

Semiconductor IP Design

Theory, Production Code, and the Practice of a Design House

A graduate curriculum for the design, modelling and verification of semiconductor IP blocks, written from first principles and grounded throughout in source code that is shipping in production silicon.

Corpus. 56,199 files, **19,925,868 lines** of real design and modelling code were collected and indexed mechanically. Every quantity quoted in this book is computed from that index; none is written from memory.

How to read. Green boxes recall undergraduate material; purple boxes carry graduate-level detail and the reasoning a practising engineer needs; red boxes mark places where real projects have failed.

Contents

Z1. How a Chip Is Described: Levels of Abstraction	9
Z2. Boolean Algebra and Combinational Logic	11
Z3. Sequential Logic and State.....	13
Z4. Circuit Analysis: the Minimum Needed	15
A1. Signals, Systems and Linearity	16
A2. Transforms and Spectral Representation	19
A3. Sampling, Reconstruction and Quantisation	21
A4. Probability, Random Processes and Noise	23
A5. Linear Algebra and Numerical Methods for Hardware	25
B1. The MOSFET — What a Digital Designer Must Know	27
B2. CMOS Logic, Delay and Power	29
B3. Timing, Clocking and Static Timing Analysis.....	31
B4. Interconnect, Transmission Lines and Signal Integrity.....	33
B5. Analogue Building Blocks.....	36
C1. Arithmetic Units — The Core of Every Datapath.....	38
C2. Control: State Machines, Pipelines and Flow Control.....	41
C3. Clock Domain Crossing, Reset and Low Power	43
D1. Instruction Set Architecture	46
D2. Pipelining, Hazards and Speculation	48
D3. Memory Hierarchy and Coherence.....	50
E1. Digital Filters as Hardware	52
E2. Transforms in Hardware: FFT and Friends	55
F1. The Wireline Link, End to End.....	57
F2. Error Control Coding.....	60
G1. Electromagnetics for the Digital Engineer	63
G2. Radar: Signal Processing as an IP Block Family.....	65
H1. Verification: Theory and Organisation	68
H2. The Implementation Flow	71
H3. Hardware Security.....	73

I1. Information Theory	75
I2. Algebraic Structures for Hardware	77
I3. Optimisation and Numerical Methods	79
I4. Discrete Mathematics and Graphs in Hardware	80
J1. On-Chip Memory	81
J2. Accelerators: Systolic Arrays and the Roofline.....	82
J3. On-Chip Interconnect and Networks	85
J4. Image and Video Coding	87
K1. Wireless Physical Layers	88
K2. Protocol Stacks as Hardware.....	90
K3. Reliability, Safety and Yield.....	92
K4. Current Research Directions	94
L1. Semiconductor Physics: What Survives into Design	95
L2. Power Delivery and Conversion.....	97
L3. Thermal and Packaging	98
L4. Feedback and Control.....	99
M1. The Hardware–Software Interface.....	100
M2. Machine Learning Hardware.....	102
M3. Image Signal Processing	104
M4. Measurement and Characterisation.....	105
N1. Asynchronous and Globally Asynchronous Design	106
N2. Physical Design in More Depth.....	107
N3. Statistics for Silicon.....	108
N4. Optical Links and Photonics.....	109
W1. What a One-Person Design House Actually Sells.....	110
W2. The Solo Process, Stage by Stage	113
W3. Automation: Running Verification Around the Clock	116
W4. Worked Exercises	118
W5. From Literature to a Block.....	120
W6. Working on a Modelling Team.....	122
W7. Debug and Bring-Up	124

W8. Positioning, Pricing and Support	127
Q1. Performance Modelling and Queueing	129
Q2. Compilers and High-Level Synthesis Internals.....	131
Q3. Embedded Software Around the Hardware	133
Q4. Electromagnetic Compatibility	134
R1. Cryptographic Mathematics.....	135
R2. Manufacturing Test	137
R3. Quantum Computing: What a Hardware Engineer Should Know.....	138
P1. MIPI CSI-2 and DSI — Camera and Display Interfaces	139
P2. PCI Express, Generations 3 to 6	142
P3. The Gigabit Ethernet MAC and Its Transceiver Interfaces	145
S1. Storage Interfaces	148
S2. Display and Audio Interfaces	149
S3. Clock, Reset and Power Architecture of an IP Block.....	150
S4. Packaging an IP for Delivery	152
X1. Fixed-Point Arithmetic, Worked End to End	154
X2. Timing Closure Arithmetic, Worked	161
X3. Metastability and Clock-Domain Crossing, Worked	167
X4. The Wireline Link Budget, Worked	172
X5. Reed–Solomon Decoding, Traced and Measured	178
X6. The FFT as a Hardware Block, Worked	183
X7. Power and Energy, Worked	187
X8. The Mathematics of Verification, Worked	191
X9. Computer Arithmetic Beyond the Adder, Worked	196
X10. Memory: The Block That Decides the Floorplan	201
X11. Interconnect: Arbitration, Flow Control and Deadlock	205
X12. Data Converters and PLLs, Worked	210
X13. Machine-Learning Hardware, Worked	214
X14. Physical Design as the RTL Designer Sees It	218
X15. Hardware Security, Worked	221
X16. Reliability, Yield and Functional Safety, Worked	224

X17. Reading the Current Literature	228
X18. Performance Modelling: Queues, Bursts and Tails	231
X19. Processors as IP, Worked.....	234
X20. Image and Video Coding as a Hardware Block	237
X21. Feedback and Control Inside Digital Blocks	240
X22. Resampling and Timing Recovery, Worked.....	243
X23. Complex Analysis and Transforms, for People Who Build Them.....	246
X24. Numerical Linear Algebra in Finite Precision	251
X25. Detection and Estimation, the Receiver's Mathematics	255
X26. Optimisation, the Tool Behind Every EDA Algorithm	259
X27. Design for Test: Scan, BIST and ATPG, Worked	263
X28. Caches and Coherence, Worked	268
X29. LDPC and Polar Codes, Worked	272
X30. OFDM and MIMO, Worked	276
X31. Analogue Building Blocks, Budgeted	280
X32. The Power Delivery Network, Worked.....	284
X33. Boolean Reasoning: BDDs, SAT and Equivalence Checking	287
X34. Logic Synthesis: What the Tool Is Actually Doing	290
X35. Hardware Data Structures: CAM, Hashing and Bloom Filters	293
X36. Cryptographic Engines, Sized	297
X37. Bus Protocols: Ordering, Outstanding Transactions and Exclusives	301
X38. Colour, the ISP Pipeline and Display Processing	305
X39. Clock Generation, Distribution and Time Synchronisation	309
X40. The Coverage Map: Which Theory an IP Block Uses	312
X41. DRAM: The Timing That Shapes Every Memory Controller	315
X42. Reset, Initialisation and the States Nobody Simulates.....	318
X43. Power Intent: Domains, Retention and Isolation.....	321
Z1. The Two-Member IP Team: You and the Agent	9
Z2. The Daily and Weekly Loop	11
Z3. The Quality Bar: What 'Sellable' Means as Numbers.....	13
Z4. The Deliverable Pack, Built and Gated	15

Z5. Choosing the First Product, Concretely	336
Z6. What the Agent Cannot Do	339
Z7. Solo Bring-Up: The Bench You Actually Need.....	342
Z8. The Money.....	345
Z9. Scaling the Agent to a Block That Is Actually Big	348
Z10. The RS-KP4 Decoder, Planned Block by Block	351
Z11. Writing the AXI Wrapper a Customer Will Accept.....	354
Z12. Writing the Documents	358
Z13. Week One	361
Z14. The C++ Model Layer — One Source, Two Destinations	364
Z15. HLS — What It Costs, and How to Make It Win.....	368
Z16. Before Any Code — System, Spec, Schematic.....	372
Z17. From One Transistor to Our Datapath	377
Z18. SystemVerilog for the C++ Programmer	385
Z19. The C++ Used for Hardware, Keyword by Keyword.....	398
Z20. The Free Toolchain, Command by Command	404
Z21. Building Protocol IP When You Cannot Get the Spec.....	411
Z22. 8b/10b — One Block, Two Markets	415
Z23. The Packet Header ECC — Why Headers Get More Than Payloads	421
Y1. MIPI CSI-2 and DSI, Worked as a Deliverable	426
Y2. PCI Express, Generations 3 to 6, Worked	431
Y3. The Ethernet MAC and Its Transceiver, Worked.....	435
Y4. A Verification Agent That Runs While You Sleep	439
Y5. The Commercial Mechanics of Selling an IP Block	444
Y6. From a Paper to a Block, and Back to a Paper.....	448
Y7. Debug and Bring-Up, With the Arithmetic.....	452
Y8. The One-Person Path from Model to Product	455
Y9. Standards, Compliance and Interoperability	459
Y10. The Register Map and the Software Interface	462
Y11. The Modelling Engineer’s Working Day	465
Y12. High-Level Synthesis in a Small Design House	468

Y13. Getting Hired into a Modelling Team	471
Y14. The Flow a One-Person House Can Actually Run.....	474
Y15. A Complete Worked Project: RS-KP4.....	477
Volume II — Production Source Code	480
MIPI CSI-2 수신기 -- 패킷 디코더 · 레인 정렬 · RAW 언팩	481
MIPI CSI-2 수신기 (1세대).....	506
이더넷 MAC · 10G BASE-R PCS · XGMII/GMII	532
PCIe 코어 (추가).....	574
AES 골든 모델 (C).....	627
AES·SHA·MD5 C 참조 구현.....	636
AES 코어 (Verilog).....	671
SHA-256 코어 (Verilog)	692
OpenTitan AES -- 양산 보안 RTL.....	702
OpenTitan HMAC-SHA256	748
OpenTitan KMAC (Keccak/SHA-3)	779
OpenTitan CSRNG (SP800-90A)	814
OpenTitan 엔트로피원 (SP800-90B)	841
OpenTitan OTBN -- 빅넘버 코프로세서.....	881
SPI 호스트	921
I2C 컨트롤러.....	956
UART	990
USB 디바이스.....	1013
RISC-V 타이머	1055
RISC-V 디버그 모듈.....	1065
TL-UL 버스 구조.....	1087
SRAM 컨트롤러(스크램블).....	1137
OTP 컨트롤러	1153
키 관리자	1156
수명주기 컨트롤러	1190
플래시 컨트롤러.....	1231
OpenTitan 기본 셀 라이브러리	1233

Caliptra SHA-512	1282
Caliptra SHA-256	1317
Caliptra HMAC	1352
Caliptra ECC (P-384)	1386
Caliptra AES	1422
PULP 기본 셀 (FIFO·CDC·아비터)	1468
AXI4 상호연결	1528
APB	1577
RISC-V 디버그	1592
CV32E40P RISC-V 코어	1627
CV32E40P RVFI 트레이스	1657
PicoRV32 -- 초소형 RISC-V	1695
SERV -- 비트직렬 RISC-V	1740
형식검증된 AXI/Wishbone 브리지	1781
PCIe 코어	1814
RVFI UVM 에이전트	1867
step-compare 테스트벤치	1883

Z1. How a Chip Is Described: Levels of Abstraction

This chapter is the gentlest in the book and the one that everything else refers back to. A modern chip contains on the order of 10^{10} transistors; no person reasons about them individually. The field copes by stacking abstractions, each hiding the one beneath. Knowing the stack — and knowing where it leaks — is the organising skill of the discipline.

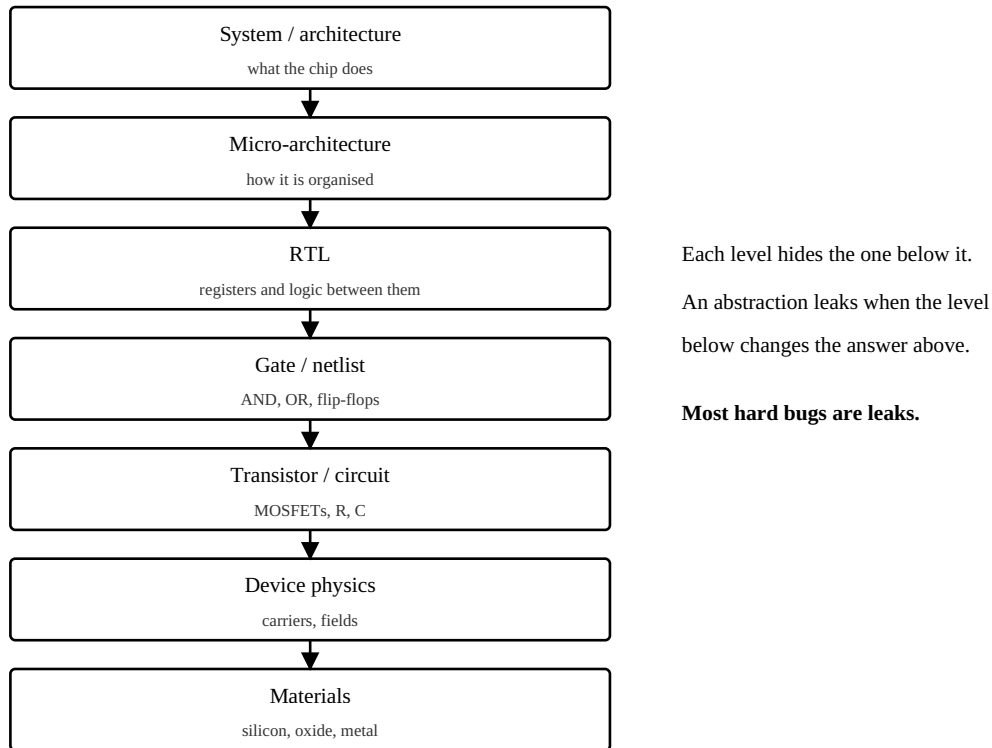


Figure 1. The abstraction stack. Work at one level, but know which level below will eventually intrude.

Table 1. What each level is, and who works there

Level	Unit of thought	Typical artefact	Role
System	Throughput, latency, power budget	Architecture document, performance model	System architect
Micro-architecture	Pipelines, buffers, state machines	Block diagrams, cycle-accurate model	Micro-architect, modelling engineer
RTL	Registers and the logic between them	SystemVerilog / VHDL	Design engineer
Gate	Boolean functions, standard cells	Netlist	Synthesis tool (not usually a person)
Circuit	Transistors, resistance, capacitance	SPICE netlist	Analogue / custom designer
Device	Carrier transport, fields	Compact models (BSIM)	Device engineer
Materials	Doping, oxides, metals	Process recipe	Process engineer

Undergraduate review · The middle of this stack is where digital IP work happens. A digital designer writes RTL and trusts the synthesis tool to produce gates, the library to characterise them, and the process to build them. The trust is well-founded almost all of the time.

Graduate · **Almost all of the time is the important qualifier, and the exceptions form a checklist worth memorising.** Abstractions leak in a small number of recurring ways, each of which appears later in this book:

- **Timing** — RTL describes behaviour without delay, but a real gate takes time, so a functionally correct design can fail (Chapter B3).
- **Metastability** — a flip-flop is modelled as producing 0 or 1, but a real one can do neither for a while (Chapter C3).
- **Power** — RTL has no notion of energy, yet switching activity and glitching are decided by RTL structure (Chapter B2).
- **Physical size** — RTL has no geometry, but wire delay depends on it (Chapter B4).
- **Side channels** — two circuits computing the same function can be distinguished by their power consumption (Chapter H3).
- **Radiation and ageing** — a stored bit is assumed permanent; physically it is not (Chapter K3).

A useful way to read the rest of this book is as a systematic tour of these leaks, because they are exactly the places where an engineer's judgement is still required and a tool cannot be trusted alone.

Z1.2 The design flow in one page

Table 2. From idea to silicon

Phase	Input	Output	Principal risk
Specification	Requirements	Written specification	Ambiguity
Modelling	Specification	Reference model	Misreading the specification
RTL design	Specification	Synthesisable RTL	Functional error
Verification	RTL + model	Coverage, bug reports	Missing a case
Synthesis	RTL + constraints + library	Gate netlist	Wrong constraints
Place and route	Netlist + floorplan	Layout	Congestion, timing
Sign-off	Layout	GDSII	An unchecked corner
Fabrication	GDSII	Wafers	Yield
Bring-up	Packaged parts	Working system	No visibility
Production test	—	Sorted devices	Test escape

The cost of fixing a defect rises by roughly an order of magnitude at each phase boundary. A specification ambiguity caught while writing the specification costs an hour. Caught in verification it costs a week. Caught at bring-up it costs a mask set — hundreds of thousands to millions of dollars and a three-month schedule slip. This asymmetry, not perfectionism, is why the industry invests so heavily in front-end rigour, and it is the economic reason a modelling team exists at all.

Z2. Boolean Algebra and Combinational Logic

Undergraduate review · Digital logic rests on a two-element Boolean algebra with operations AND ($\cdot$), OR ($+$) and NOT ($'$). Its axioms give the identities used to simplify expressions, and every combinational circuit is the realisation of some Boolean function.

Table 3. Identities used constantly

Name	Statement
Identity	$A \cdot 1 = A, \quad A + 0 = A$
Null	$A \cdot 0 = 0, \quad A + 1 = 1$
Idempotence	$A \cdot A = A$
Complement	$A \cdot A' = 0, \quad A + A' = 1$
De Morgan	$(A \cdot B)' = A' + B', \quad (A + B)' = A' \cdot B'$
Absorption	$A + A \cdot B = A$
Consensus	$AB + A'C + BC = AB + A'C$
Shannon expansion	$f = x \cdot f _{x=1} + x' \cdot f _{x=0}$

Graduate · **Two of these are more than algebra.** De Morgan's laws are the reason CMOS is built from NAND and NOR rather than AND and OR: a CMOS gate is naturally inverting, so *every* logic function must be expressible in inverting gates, which De Morgan guarantees. The **consensus term** is the redundant product that can be removed without changing the function — but removing it can create a *static hazard*, a momentary glitch when inputs change. In synchronous logic a glitch before the next clock edge is harmless and the term is removed for area; in asynchronous logic or in a clock-gating enable it is not, and the redundant term must be kept. **The same simplification is correct in one context and a bug in another, which is why hazard-free synthesis is a distinct option in the tools.**

Table 4. Representations of a Boolean function

Form	Size	Canonical?	Use
Truth table	2^n rows	Yes	Definition; small functions
Karnaugh map	2^n	No	Hand minimisation up to ~5 variables
Sum of products	Varies	No (unless canonical)	PLA, two-level synthesis
Quine–McCluskey	—	—	Exact minimisation; exponential
Espresso	—	—	Heuristic; what tools actually use
BDD	Order-dependent	Yes	Equivalence checking
AIG	Compact	No	Modern synthesis and formal engines

Z2.2 Standard combinational blocks

Table 5. The building blocks every design contains

Block	Function	Cost	Note
Multiplexer	Select one of n inputs	$n-1$ two-input muxes, or a tree	A mux is functionally complete — any function can be built from muxes

Block	Function	Cost	Note
Decoder	$n \rightarrow 2^n$ one-hot	2^n gates	Address decoding, instruction decode
Encoder / priority encoder	one-hot $\rightarrow$ binary	—	Interrupt arbitration, leading-zero detection
Comparator	Equality or magnitude	XOR tree / subtract	Magnitude comparison is an adder in disguise
Shifter / barrel shifter	Shift by a variable amount	$\log_2 n$ mux stages	Used in floating point, in gearboxes
Adder	See Chapter C1	—	The most studied block in the field
Parity / population count	XOR tree / adder tree	—	ECC, binary neural networks

Graduate · The observation that a multiplexer is functionally complete is more than a curiosity: it is why FPGAs work.

An FPGA's logic element is a small lookup table, which is physically a multiplexer tree driven by configuration bits. Any four-input function is realised by loading sixteen bits. The entire programmable-logic industry rests on the Shannon expansion identity in the table above. **Recognising that a familiar identity is the foundation of a product category is the kind of connection this book is trying to make routine.**

Z3. Sequential Logic and State

Undergraduate review · A combinational circuit's output depends only on its present inputs. Adding memory — elements that hold a value — produces sequential logic, whose output depends on the history of inputs. That history is summarised in the **state**.

Table 6. Storage elements

Element	Behaviour	Use	Hazard
SR latch	Set/reset, level sensitive	Building block	Forbidden input combination
D latch	Transparent when enabled	Time borrowing, low-power designs	Transparency complicates timing analysis
D flip-flop	Captures on a clock edge	The standard element	Setup/hold must be met
Master–slave	Two latches in series	How a flip-flop is built	—
Pulsed latch	Narrow transparency window	High performance	Timing complexity
Register file	Array with multiple ports	Processors	Port count costs area

Unintentional latches are one of the most common RTL defects. In a combinational always block, if some path through the logic leaves a signal unassigned, the synthesiser must remember its previous value and therefore infers a latch. The design usually still simulates correctly, but a latch in a nominally combinational path is a timing analysis problem and often a functional one. The defence is mechanical: assign a default value at the top of every combinational block, cover every branch, and treat the lint warning as an error. **This is the first check in any RTL review.**

Table 7. Finite state machine styles

Style	Output depends on	Property
Moore	State only	Outputs are registered and glitch-free; one cycle of latency
Mealy	State and inputs	Faster response; outputs can glitch and create combinational paths
Registered Mealy	State and registered inputs	Compromise; usually preferred

Graduate · **The Moore/Mealy choice has a direct consequence at module boundaries.** A Mealy output is a combinational function of an input, so connecting two Mealy modules can create a path that runs through both in a single cycle — or, if each one's input depends on the other's output, a combinational loop. This is the general form of the valid/ready loop described in Chapter C2. **For an IP block, registering all outputs (Moore style) costs one cycle of latency and removes an entire class of integration failures,** which is usually the right trade for something a stranger will instantiate.

Z3.2 Counters, shift registers and LFSRs

Table 8. Sequential building blocks

Block	Function	Where used
Binary counter	Increment	Addressing, timing
Gray counter	One bit changes per step	CDC-safe pointers , low-power encoding
Johnson counter	Twisted ring	Multi-phase clock generation

Block	Function	Where used
Shift register	Serial movement	Serialisers, delay lines
LFSR	Shift with XOR feedback	PRBS generation, scramblers, CRC, BIST, test patterns
CRC	Polynomial remainder	Frame check sequences

Graduate · **The LFSR is the most reused structure in communications hardware, and all of its uses are the same mathematics.** An LFSR computes a remainder modulo a polynomial over GF(2). Choose a primitive polynomial and it cycles through all $2^n - 1$ non-zero states, giving a maximal-length pseudo-random sequence — that is a PRBS generator, and also a scrambler. Feed data in as you shift and the final state is the remainder — that is a CRC. Use it to produce stimulus and to compact responses and you have built-in self test. **One circuit, four products, because they are one piece of algebra.** Chapter I2 develops the algebra; this is where it first appears.

Z4. Circuit Analysis: the Minimum Needed

Table 9. Laws and elements

Item	Statement	Digital-design relevance
KCL	Currents into a node sum to zero	Power grid analysis
KVL	Voltages around a loop sum to zero	IR drop
Ohm	$V = IR$	Wire and switch resistance
Capacitor	$i = C \, dv/dt$	The dominant load in CMOS
Inductor	$v = L \, di/dt$	Package and board; cause of supply noise
RC time constant	$\tau = RC$	Gate delay, wire delay, rise time
Thévenin / Norton	Source equivalents	Driver modelling, termination
Superposition	Linear circuits only	Noise analysis

Undergraduate review · For digital work the essential relation is the RC charging curve: a capacitor charged through a resistor reaches 63% of its final value in one time constant $\tau = RC$, and about 50% in 0.69τ . Since a logic gate switches when its input crosses the threshold near mid-supply, **propagation delay is approximately $0.69R_{\text{on}}C_{\text{load}}$** .

Graduate · **That single expression explains most of digital design practice.** Delay is proportional to load capacitance, so driving many gates is slow — hence buffer trees. Delay is proportional to on-resistance, which falls as transistor width rises, so wider devices are faster — but a wider device presents a larger capacitance to *its* driver, which is why sizing is a chain optimisation rather than a local one, and why the logical-effort method of Chapter B2 exists. Dynamic energy is CV^2 per transition, which is why lowering supply voltage is the strongest power lever and why it costs speed. **Four consequences from one equation; this is the level of leverage that makes a small amount of circuit theory worth carrying.**

Table 10. Frequency-domain concepts a digital engineer meets

Concept	Meaning	Where encountered
Impedance $Z(\omega)$	Generalised resistance	Power delivery, termination
Pole / zero	Roots of the denominator / numerator	Filter and loop design
Bode plot	Magnitude and phase versus frequency	PLL loop stability
Q factor	Energy stored / dissipated per cycle	Resonance in power networks, oscillator phase noise
Phase margin	Stability measure	Any feedback loop, including a PLL
Bandwidth $\times$ gain	Roughly constant for an amplifier	Why high gain and high speed conflict

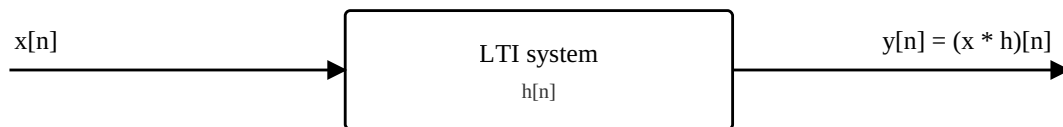
A1. Signals, Systems and Linearity

Every digital IP block is a *system*: it maps input sequences to output sequences. Before discussing any particular block we fix the vocabulary that makes such maps analysable, and — more importantly for a hardware engineer — we identify exactly which property of a system makes it **cheap** to build and which makes it **expensive**.

A1.1 Definitions

Table 11. System properties and their hardware consequences

Property	Definition	Hardware consequence
Memoryless	$y[n]$ depends only on $x[n]$	Pure combinational logic; no registers, no state to reset or scan
Causal	$y[n]$ depends only on $x[k]$, $k \leq n$	Required for real-time streaming. Non-causal filters are realised by adding latency , not by seeing the future
Linear	Superposition holds	Enables decomposition into impulse responses; permits frequency-domain design
Time-invariant	Delay in $\rightarrow$ same delay out	Coefficients are constants $\rightarrow$ can be hard-wired or stored once
BIBO stable	Bounded input $\rightarrow$ bounded output	$\sum h[n] < \infty$. In fixed point, determines whether accumulators can overflow
Passive lossless	/ Energy not increased	Guarantees no growth in word length



Linearity + time invariance $\Rightarrow$ one impulse response is the whole system

$Y(z) = H(z) \cdot X(z)$ — convolution becomes multiplication

Figure 2. An LTI system is completely characterised by its impulse response. This single fact underpins all filter design.

Graduate The practical importance of linearity is not elegance but **separability of verification**. If a block is LTI you can characterise it with a single impulse and predict its response to every input; a random-stimulus campaign is then a redundancy check rather than the primary evidence. If it is *not* LTI — a decision-feedback equaliser, a μ -law compander, a saturating accumulator, any adaptive loop — superposition fails and you must verify over the operating region, which is exponentially more work. **The first architectural question about any new block is therefore "where exactly is the nonlinearity?"** Isolating it into the smallest possible sub-block is a verification cost decision, not an aesthetic one.

A1.2 Convolution and its cost

$$y[n] = \sum_{k=0}^{N-1} h[k] x[n-k]$$

Table 12. Realisations of the same convolution

Method	Operations per output	Latency	When preferred
Direct convolution	N MACs	0 (plus pipeline)	$N \leq 64$
Overlap-add FFT	$\sim(2 \log_2 L + 1)$ per sample	One block (L samples)	Long filters; latency tolerable
Overlap-save FFT	Same order	One block	Streaming; avoids add stage
Frequency-domain adaptive	FFT + per-bin update	One block	Long adaptive filters (echo cancellation)
Distributed arithmetic	LUT reads only	W cycles	Fixed coefficients, LUT-rich fabric
Multiplierless (CSD)	Shifts and adds	0	Fixed coefficients, ASIC area critical

Graduate · The crossover between direct and FFT-based convolution is not a fixed number of taps; it depends on the **cost ratio of a multiplier to a memory word** in the target technology, and on whether block latency is acceptable. In an ASIC at an advanced node, multipliers are comparatively cheap and SRAM is comparatively expensive, which pushes the crossover *up*. On an FPGA with hard DSP slices and abundant block RAM it moves the other way. **Quoting a fixed crossover (“use FFT above 128 taps”) without naming the technology is a common and costly error in architecture documents.**

A1.3 Correlation, matched filtering and the origin of SNR gain

Cross-correlation is convolution with a time-reversed conjugated kernel. When the kernel is the transmitted pulse itself, the operation is a **matched filter**, and it maximises output SNR for a signal in additive white Gaussian noise.

$$SNR_{out,max} = 2E_s / N_0$$

Graduate · The result is worth internalising because it recurs in radar, in spread spectrum, in synchronisation preambles and in every correlator IP. **The gain comes from integrating signal coherently while noise adds incoherently:** over M samples, signal amplitude grows as M while noise amplitude grows as $\sqrt{M}$, hence power SNR improves by M ($10 \log_{10} M$ dB). Two engineering consequences follow. First, coherent integration requires **phase stability over the integration window**; if the oscillator drifts, gain saturates — this is the practical limit on radar integration time and on preamble length. Second, the correlator's dynamic range must accommodate the M -fold growth, which is a fixed-point word-growth problem, not a signal-processing one.

A1.4 Group delay, phase linearity and why FIR dominates

$$\tau_g(\omega) = -d\phi(\omega)/d\omega$$

Undergraduate review · A filter has **linear phase** when τ_g is constant: all frequency components are delayed equally, so pulse shapes are preserved and only shifted. An FIR filter with symmetric (or antisymmetric) coefficients has exactly linear phase. An IIR filter cannot, except trivially.

Graduate · This is the decisive argument for FIR in data transmission. Non-constant group delay **spreads a pulse asymmetrically**, which is indistinguishable from channel dispersion and therefore directly closes the eye. In audio the ear is comparatively insensitive to phase and IIR is preferred for its efficiency; in a wireline receiver an IIR equaliser would introduce the very distortion it is meant to remove. **The same mathematics leads to opposite engineering choices in two domains, and knowing why is the difference between applying a rule and understanding it.** Note also the hardware corollary: symmetric FIR coefficients permit the *folded* structure that halves the multiplier count, so linear phase is not only free, it is cheaper.

A1.5 Stability in the Z domain

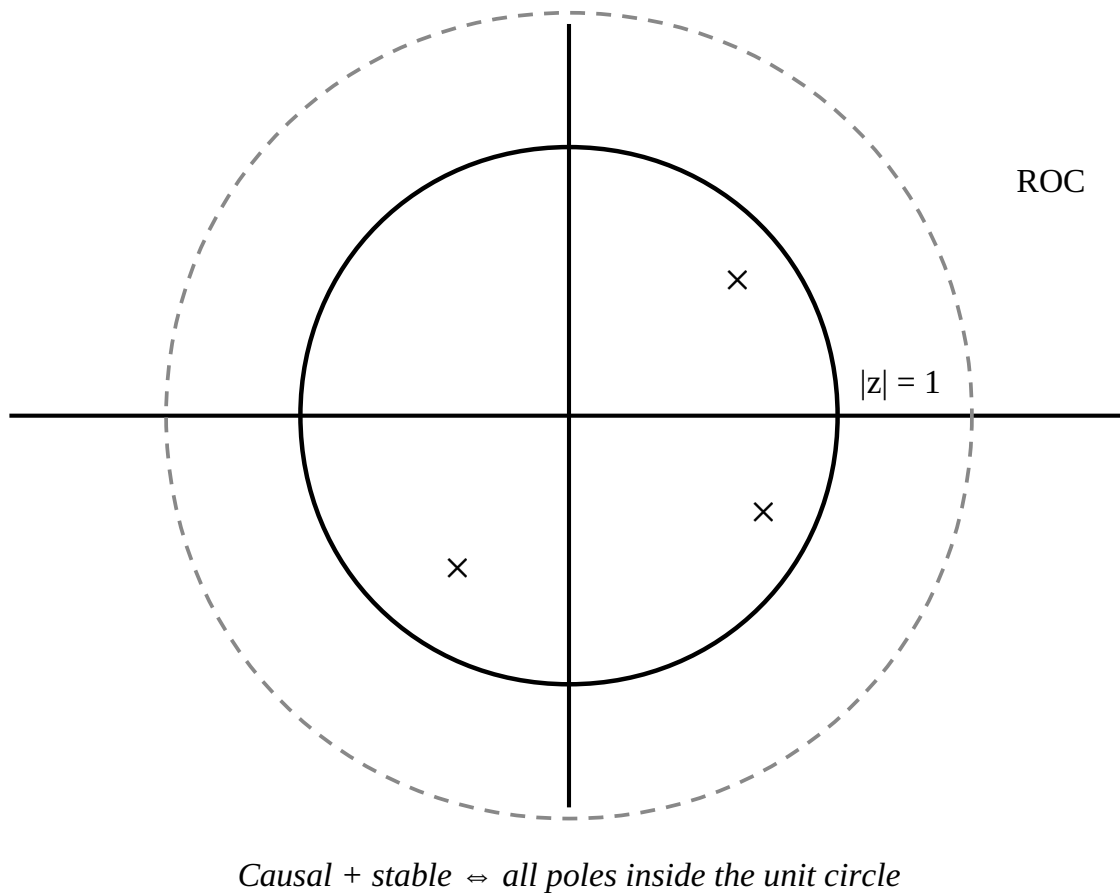


Figure 3. Pole-zero plot and region of convergence. For a causal system the ROC is outside the outermost pole; stability additionally requires that the ROC contain the unit circle.

Table 13. Pole location and behaviour

Pole radius $ p $	Behaviour	Fixed-point implication
$ p < 1$	Decaying; stable	Accumulator bounded by $1/(1- p)$
$ p = 1$	Marginally stable (oscillator, integrator)	Accumulator grows without bound — needs explicit leakage or reset
$ p > 1$	Divergent	Unusable
$ p \rightarrow 1^-$	Very narrow band; long memory	Coefficient quantisation can push the pole outside — the classic failure mode of high-order direct-form IIR

Graduate · Quantifying the last row: for a direct-form IIR of order N , the sensitivity of pole p_i to a denominator coefficient a_k is $\partial p_i / \partial a_k = -p_i^{N-k} / \prod_{j \neq i} (p_i - p_j)$. The denominator is a product of *pole separations*: **clustered poles make the sensitivity explode**. Cascaded second-order sections avoid this because each section contains only one conjugate pair, so no small separations appear in its own denominator. This is the quantitative reason behind the universal practice of implementing high-order IIR filters as cascaded biquads, and it is worth being able to state it rather than merely follow it.

A2. Transforms and Spectral Representation

Table 14. The transform family

Transform	Domain	Kernel	Principal use in IP work
CTFT	continuous time $\rightarrow \omega$	$e^{-j\omega t}$	Analogue filter and antenna analysis
Laplace	continuous time $\rightarrow s$	e^{-st}	PLL loop dynamics, control, transmission lines
DTFT	discrete time $\rightarrow \omega$	$e^{-j\omega n}$	Theoretical filter response (continuous in ω)
DFT / FFT	finite sequence $\rightarrow$ bins	$e^{-j2\pi kn/N}$	Everything computable
Z	discrete time $\rightarrow z$	z^{-n}	Poles, zeros, stability, filter structures
Hilbert	real $\rightarrow$ analytic	$-j \operatorname{sgn}(\omega)$	I/Q generation, envelope, SSB
DCT	finite $\rightarrow$ cosine basis	$\cos$	Image/video coding (energy compaction)
Wavelet	time–scale	scaled basis	Transient detection, JPEG2000

A2.1 The DFT is not the DTFT

The DFT computes samples of the DTFT of a **windowed, periodically extended** sequence. Three artefacts follow, and all three appear as bugs in practice.

Table 15. DFT artefacts

Artefact	Cause	Symptom	Mitigation
Spectral leakage	Implicit rectangular window	A pure tone smears across bins	Apply a window (Hann, Blackman–Harris)
Scalloping loss	Tone falls between bin centres	Up to 3.9 dB amplitude error (rectangular)	Window, or interpolate the peak
Picket fence	Only N samples of a continuum	Frequency resolution limited to f_s/N	Zero-pad (interpolates, does not add resolution)
Circular convolution	Periodic extension	Wrap-around in fast convolution	Zero-pad to $N \geq L+M-1$

Zero-padding is routinely misdescribed as improving resolution. It does not. **Resolution is set by the observation time** $T = N/f_s$; two tones closer than $\approx 1/T$ cannot be separated no matter how many zeros are appended. Padding interpolates the displayed spectrum, which makes peaks easier to *locate* but not easier to *resolve*. In a specification, resolution and interpolation must be stated separately.

Table 16. Common windows

Window	Main-lobe width (bins)	First sidelobe	Use
Rectangular	2	−13 dB	Transient capture; maximum resolution
Hann	4	−31 dB	General purpose
Hamming	4	−41 dB	Slightly better near sidelobe
Blackman–Harris (4-term)	8	−92 dB	Spur hunting in converter test
Kaiser (β adjustable)	tunable	tunable	Design-to-spec
Flat-top	~9	−90 dB	Amplitude accuracy (calibration)

A2.2 Parseval, power spectral density, and dB conventions

$$\sum_n |x[n]|^2 = (1/N) \sum_k |X[k]|^2$$

Table 17. Decibel conventions that are routinely confused

Unit	Reference	Note
dB	ratio	10 log for power, 20 log for amplitude
dBm	1 mW	Absolute power
dBc	carrier	Spur levels, phase noise
dBFS	converter full scale	Digital level; not a physical power
dBm/Hz	1 mW per Hz	Spectral density; thermal floor -174 dBm/Hz @ 290 K
dBc/Hz	carrier per Hz	Phase noise $L(\Delta f)$

Graduate · A recurring measurement error is comparing a **single-bin** FFT magnitude with a **spectral density**. The FFT bin contains power integrated over the equivalent noise bandwidth of the window, which for a Hann window is 1.5 bins, not 1. Reporting “noise floor at -110 dBFS” without stating N , f_s and the window is not a measurement, because the number changes when any of the three changes. **Processing gain $10 \log_{10}(N/2)$ is often mistaken for converter performance.**

A3. Sampling, Reconstruction and Quantisation

A3.1 The sampling theorem and its practical corollaries

$$X_s(f) = f_s \sum_k X(f - k f_s)$$

Table 18. Sampling regimes

Regime	Condition	Where used	Hardware cost
Nyquist	$f_s > 2B$, base-band	Most baseband DSP	Anti-alias filter must be steep
Oversampling (OSR > 1)	$f_s \gg 2B$	$\Sigma\Delta$ converters, relaxed AAF	Quantisation noise spread over wider band: +3 dB SNR per doubling
Bandpass / under-sampling	$2f_H/n \leq f_s \leq 2f_L/(n-1)$	RF direct sampling receivers	Sampler analogue bandwidth must exceed f_H , not f_s
Time-interleaved	M converters at f_s/M	>10 GS/s ADCs (112G SerDes uses 32–64 way)	Offset / gain / timing skew mismatch creates spurs at kf_s/M

Graduate · **Time-interleaving is where the design house earns its fee.** An M -way interleaved ADC has three mismatch mechanisms, each producing a distinct spur family: offset mismatch gives tones at kf_s/M independent of input; gain mismatch gives sidebands at $kf_s/M \pm f_{in}$; timing skew gives the same sideband locations but with amplitude proportional to f_{in} . **The frequency dependence is the diagnostic:** sweep the input and see which spur grows. Digital background calibration of all three is a standard deliverable in a modern ADC-DSP receiver, and it is a *digital* block — which means a fabless digital IP vendor can sell it even though the converter itself is analogue. This is a concrete example of finding a sellable digital block adjacent to an analogue one you cannot build.

A3.2 Quantisation noise

$$SQNR_{dB} = 6.02 B + 1.76 + 10 \log_{10}(OSR)$$

Table 19. Where the ideal formula breaks

Effect	Mechanism	Consequence	Design response
Aperture jitter	Sampling instant uncertainty σ_t	$SNR = -20 \log_{10}(2\pi f_{in} \sigma_t)$	Dominates above ~1 GHz input; clock design becomes the ADC spec
Thermal noise	KT/C of sampling capacitor	Floors SNR regardless of bits	Increase C (costs power and bandwidth)
DNL / INL	Element mismatch	Harmonic distortion, missing codes	Calibration, dynamic element matching
Comparator meta-stability	Finite regeneration time	Rare large errors	More regeneration time or error detection
Correlated quantisation	Low-amplitude periodic input	Quantisation noise becomes tonal, not white	Dither — deliberately added noise to decorrelate

Graduate · The dither entry deserves emphasis because it is counter-intuitive: **adding noise improves the result.** Quantisation error is only white and signal-independent when the signal moves over many LSBs; for a small or slowly varying signal

the error becomes periodic and produces discrete tones, which are far more objectionable than an equivalent amount of broad-band noise (in audio they are audible; in a radar they look like targets). Subtractive dither — add a known pseudo-random sequence before quantising and subtract it after — removes the tones without the SNR penalty. **A dither generator is a small, precisely specifiable digital block and therefore exactly the kind of thing a one-person IP house can build and sell as part of a converter-adjacent package.**

A3.3 Reconstruction

Table 20. Reconstruction effects

Effect	Expression	Compensation
Zero-order hold droop	$\text{sinc}(f/f_s)$	−3.92 dB at Nyquist; pre-compensate with an inverse-sinc FIR
Image replicas	at $kf_s \pm f$	Reconstruction (anti-imaging) filter
Interpolation	up-sample then low-pass	Moves images away so the analogue filter can be gentle
Mismatch-shaping DAC	—	Pushes element mismatch error out of band; a digital technique for an analogue defect

A4. Probability, Random Processes and Noise

A4.1 Distributions that occur in hardware

Table 21. Distributions and where they arise

Distribution	Arises from	Appears in
Gaussian	Sum of many small independent effects (CLT)	Thermal noise, random jitter, process variation
Rayleigh	Magnitude of complex Gaussian	Fading envelope, noise magnitude
Rician	Complex Gaussian + deterministic component	Fading with line of sight
Poisson	Rare independent events	Shot noise, soft-error (SER) counts
Uniform	Quantisation residue (ideal)	Quantisation noise model
Exponential	Waiting time between Poisson events	Time between soft errors
Log-normal	Product of many factors	Shadowing, some reliability lifetimes
Weibull	Weakest-link failure	Oxide breakdown, reliability
Binomial	n Bernoulli trials	Bit errors in a block, yield
Chi-square	Sum of squared Gaussians	Energy detection, CFAR statistics

A4.2 Physical noise mechanisms

Table 22. Noise sources in silicon

Mechanism	Spectrum	Expression	Design lever
Thermal (Johnson)	White	$v_n^2 = 4kTR\Delta f$	Lower R ; lower T ; narrower band
Channel thermal (MOS)	White	$i_n^2 = 4kTyg_m\Delta f$	Higher g_m improves noise <i>figure</i> though noise power rises
Flicker (1/f)	1/f	$\propto K/(WLC_{ox}f)$	Larger devices ; chopping; auto-zeroing
Shot	White	$i_n^2 = 2qI\Delta f$	Present where carriers cross a barrier (junctions, photodiodes)
kT/C	Sampled	$v^2 = kT/C$	Independent of resistance ; only C helps
Substrate / supply coupling	Structured	—	Guard rings, separate supplies, differential design
Phase noise	$1/f^3$, $1/f^2$, flat	Leeson model	Higher Q , higher power, lower flicker

Graduate · Why flicker noise sets the architecture of precision analogue. $1/f$ noise is not reducible by filtering, because it lives at the frequencies where the signal of interest often lives (DC to kHz). The standard responses — chopping and auto-zeroing — both work by *moving the signal to a higher frequency where only white noise remains*, and both require digital support: a chopping clock, demodulation, and residual-ripple filtering. In a modern mixed-signal IP the digital half of that scheme is a well-defined block with a clean interface, and it is again a piece that a digital design house can own.

A4.3 Jitter and phase noise — the same thing in two languages

$$\sigma_t^2 = (1/(2\pi f_0)^2) \int 2 L(f) df$$

Table 23. Jitter taxonomy used in link specifications

Type	Symbol	Statistics	Cause	How it is separated
Random jitter	RJ	Gaussian, unbounded	Thermal, flicker	Fit tails of the bathtub curve
Deterministic jitter	DJ	Bounded	—	Peak-to-peak; decomposed below
Periodic	PJ	Sinusoidal	Supply coupling, spurs	FFT of the jitter sequence
Data-dependent	DDJ / ISI	Pattern-correlated	Channel loss	Compare per-pattern edge times
Duty-cycle distortion	DCD	Two-valued	Rise/fall asymmetry, threshold offset	Even/odd UI comparison
Bounded uncorrelated	BUJ	Bounded, random	Crosstalk	Statistical separation
Total jitter at BER	TJ(BER)	RJ⊗DJ convolution	—	$TJ = DJ + 2 \cdot Q(BER) \cdot RJ_{rms}$

Graduate · The last row is the one that appears in every SerDes specification, and the factor $Q(BER)$ is why: at $BER\ 10^{-12}$, $Q = 7.03$, so $TJ = DJ + 14.07\ RJ_{rms}$. **Random jitter is multiplied by fourteen.** This is why a small improvement in oscillator or phase noise is worth a large amount of design effort, and why jitter separation — distinguishing RJ from DJ in a measurement — is a required capability rather than a nicety. A modelling engineer asked to produce a link budget must implement this convolution correctly; adding RMS values in quadrature, which is the intuitive move, is wrong because DJ is bounded and does not add that way.

A4.4 Random processes

Table 24. Concepts and their use

Concept	Statement	Use in IP design
Stationarity (WSS)	Mean constant, autocorrelation depends on lag only	Precondition for spectral analysis
Ergodicity	Time average = ensemble average	Justifies measuring one long record instead of many devices
Wiener–Khinchin	PSD = FT of autocorrelation	Links time and spectrum
Markov chain	Memoryless state evolution	Bus arbitration analysis, FIFO occupancy, error models
Gauss–Markov	Correlated Gaussian	Coloured noise models
Cyclostationarity	Statistics periodic in time	All digitally modulated signals ; exploited for blind synchronisation
Point process	Random event times	Soft errors, packet arrivals

Graduate · Cyclostationarity is the theoretical basis for **blind** timing and carrier recovery: a modulated signal has periodically varying statistics at the symbol rate, so a suitable nonlinearity produces a spectral line at that rate even though the signal itself has no such line. Gardner and Mueller–Müller timing detectors are practical exploitations of this. Knowing the principle rather than the recipe is what lets an engineer design a detector for a modulation format that has no textbook detector — a situation that occurs constantly in proprietary links.

A5. Linear Algebra and Numerical Methods for Hardware

A5.1 Decompositions and their hardware character

Table 25. Matrix decompositions as IP blocks

Decomposition	Form	Cost	Streaming?	Numerical character
LU	$A = LU$	$n^3/3$	No (pivoting needs lookahead)	Needs pivoting for stability
Cholesky	$A = LL^*$	$n^3/6$	Yes	No pivoting needed for SPD; half the work of LU
QR (Givens)	$A = QR$	$2mn^2$	Yes	Backward stable; rotations are well conditioned
QR (Householder)	same	fewer ops	Less so	Best stability
SVD	$A = U\Sigma V^*$	iterative	No	Most robust ; reveals rank and conditioning
Eigen (symmetric)	$A = Q\Lambda Q^*$	iterative	No	Jacobi parallelises well
Schur / Hessenberg	—	n^3	No	Control applications

Graduate · The streaming column is the commercial column. Cholesky and Givens QR touch each input element once and can therefore be built with an `hls::stream`-style interface and no external memory; SVD and LU must revisit the array and therefore need an AXI master and a buffer. That single architectural difference changes the IP's integration story, its area, and its price. When a customer asks for “a matrix inverse block”, the first engineering question is whether the matrix is symmetric positive definite — because if it is, Cholesky gives a streaming solution at half the arithmetic, and if it is not, you are quoting a much larger block.

A5.2 Conditioning

$$\kappa(A) = \sigma_{\max}/\sigma_{\min} \quad \text{relative error} \lesssim \kappa(A) \cdot \varepsilon_{\text{machine}}$$

Graduate · The rule of thumb that follows is directly usable: **you lose roughly $\log_{10}\kappa$ decimal digits**. In fixed point, substitute $\varepsilon = 2^{-F}$ for F fractional bits and the statement becomes a word-length requirement: $F \gtrsim \log_2\kappa + (\text{bits of accuracy required})$. This is how a modelling engineer converts a mathematical property of the customer's data into a hardware parameter. It also explains why **iterative refinement** works: solve in low precision, compute the residual in high precision, correct, repeat. Each iteration roughly squares the accuracy, so three refinements from a rough start can reach near-full precision — but only if the condition number is modest, because κ enters the convergence factor. **Measured behaviour: for a well-conditioned (isotropic) system the scheme reached 24.2 bits of accuracy; at correlation 0.99 it reached only 12.4 bits**. An IP that advertises mixed-precision refinement must therefore state a conditioning limit, or it will fail at a customer whose data is correlated.

A5.3 Iterative solvers and their hardware mapping

Table 26. Iterative methods

Method	Update	Parallelism	Convergence
Jacobi	uses only previous iterate	Fully parallel	Slow; needs diagonal dominance
Gauss–Seidel	uses updated values immediately	Sequential within a sweep	Roughly twice Jacobi's rate
SOR	over-relaxed GS	Sequential	Tunable ω

Method	Update	Parallelism	Convergence
Conjugate gradient	Krylov subspace	Parallel (dot products)	n steps exactly; $\sqrt{k}$ in practice
Richardson	$x \leftarrow x + \alpha(b - Ax)$	Fully parallel	Simplest; α must be tuned

Graduate · There is a physical realisation worth knowing because it is repeatedly rediscovered and mis-described. An RC network that solves $C \, dv/dt = -(Av - b)$ settles to $A^{-1}b$, and is often presented as an “analogue Gauss–Seidel solver”. **It is not:** because all nodes evolve simultaneously from the same previous state, the continuous-time limit corresponds to **Jacobi / Richardson**, not Gauss–Seidel, which is defined by sequential in-sweep updates. The settling time is governed by $1/\lambda_{\min}$, i.e. by the condition number again. Under a normalisation that fixes $\lambda_{\max} = 1$, the measured settling-time-to-condition ratio was constant at 13.2 — the analogue solver buys constant factors, not asymptotic ones. **Being able to state this precisely prevents a design house from accepting a proposal whose speedup claim rests on a misidentification.**

B1. The MOSFET — What a Digital Designer Must Know

A digital IP engineer does not size transistors, but every timing number, every power figure and several verification hazards trace back to device behaviour. This chapter keeps only what changes decisions.

B1.1 Regions of operation

Table 27. MOSFET regions

Region	Condition	Current	Where it matters
Cut-off	$V_{GS} < V_{TH}$	$I_{sub} \propto e^{(V_{GS}-V_{TH})/nV_T}$	Subthreshold leakage — the dominant static power term below 65 nm
Triode / linear	$V_{DS} < V_{GS}-V_{TH}$	$\mu C_{ox}(W/L)[(V_{GS}-V_{TH})V_{DS} - V_{DS}^2/2]$	Switch resistance R_{on} ; pass gates; the ON device of an inverter at end of transition
Saturation	$V_{DS} > V_{GS}-V_{TH}$	$\frac{1}{2}\mu C_{ox}(W/L)(V_{GS}-V_{TH})^2(1+\lambda V_{DS})$	Analogue gain stages; the beginning of a digital transition
Velocity saturation	Short channel, high field	$\propto (V_{GS}-V_{TH})^1$ — linear, not square	Modern nodes live here. Textbook square-law intuition overestimates drive

Graduate · The last row is the single most important correction for anyone who learned the square law. In a short-channel device carriers reach their saturation velocity, so drive current becomes roughly proportional to overdrive rather than its square. Two practical consequences: **(i)** increasing supply voltage buys less speed than square-law intuition suggests, while dynamic power still rises as V^2 , so voltage scaling is a poor speed lever; **(ii)** stacking devices (as in a NAND with many inputs) costs more than the naive series-resistance estimate, which is why wide fan-in gates are avoided and logic is restructured into trees. When a synthesis tool reports that a 4-input gate is slower than two 2-input gates in series, this is why.

B1.2 Threshold voltage and its enemies

Table 28. V_{TH} modulation effects

Effect	Mechanism	Design consequence
Body effect	Source–bulk bias raises V_{TH}	Stacked devices are slower than expected
DIBL	Drain field lowers the barrier	V_{TH} falls with $V_{DS} \rightarrow$ more leakage
Random dopant fluctuation	Discrete dopant atoms	$\sigma_{V_{TH}} \propto 1/\sqrt{WL}$ — small devices mismatch more
Temperature	–1 to –2 mV/K	Hot chip is <i>faster</i> at low V_{DD} (temperature inversion)
NBTI / PBTI ageing	Trap generation under stress	V_{TH} drifts over years — timing must include an ageing margin
Multi- V_{TH} offering	Process option	LVT for critical paths, HVT elsewhere: the primary leakage-vs-speed lever

Graduate · **Temperature inversion is a verification trap.** At high supply voltage a chip is slowest when hot, so signing off at the high-temperature corner is conservative. At low supply voltage — which is where modern low-power modes operate — the V_{TH} reduction outweighs mobility degradation and the chip is slowest when **cold**. Teams that carried forward a "hot is

worst case" habit have taped out parts that fail at $-40\text{ }^{\circ}\text{C}$. The corner list is not a formality; it is a statement about which physics dominates in the operating region you are selling into.

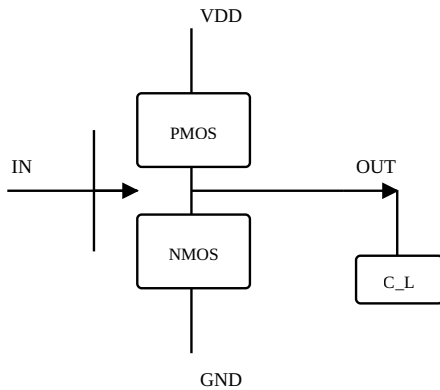
B1.3 Scaling and what it stopped giving

Table 29. Dennard scaling and its breakdown

Quantity	Ideal scaling (factor $s > 1$)	What actually happened
Dimensions	$1/s$	Continued (with FinFET, GAA)
Supply V_{DD}	$1/s$	Stalled near 0.7–0.9 V — V_{TH} cannot follow without leakage explosion
Delay	$1/s$	Slower than ideal; interconnect does not scale
Power density	constant	Rose → dark silicon, thermal limits
Leakage	negligible	Comparable to dynamic below 65 nm
Interconnect RC	—	Worsens : resistance rises as cross-section shrinks

Graduate · The end of Dennard scaling is the reason the industry turned to **specialised accelerators**, and therefore the reason the IP business grew. When a single general-purpose core could no longer be made faster within a power budget, the way to get performance became putting the right fixed-function block next to the processor. **Every FEC, crypto, codec and NPU IP sold today exists because of this graph.** Understanding it lets a design house argue its value proposition in the customer's own terms: not "our block is fast", but "your general-purpose alternative costs x times the energy per operation".

B2. CMOS Logic, Delay and Power



$$t_{pd} \approx 0.69 R_{on} C_L$$

$R_{on} \propto 1/(W/L) \Rightarrow$ bigger device, faster, more input cap

FO4 = delay driving four identical inverters

— the technology-independent speed unit

Figure 4. The inverter, and the delay model every timing number ultimately rests on.

B2.1 Delay models

Table 30. Delay models by fidelity

Model	Form	Use
RC (Elmore)	$t_{pd} = 0.69 R_{on} C_L$	Hand estimates, intuition
Logical effort	$d = gh + p$	Optimal gate sizing and stage count without simulation
NLDM (lookup)	table over input slew $\times$ output load	Standard-cell STA
CCS / ECSM	current-source model	Advanced nodes, accurate slew propagation
SPICE	device equations	Characterisation, critical custom paths

Graduate · Logical effort is the one hand method worth memorising. With g the logical effort of a gate (1 for an inverter, 4/3 for NAND2, 5/3 for NOR2), h the electrical effort (fanout), and p parasitic delay, the path delay is minimised when **every stage carries equal effort** $f = (GH)^{1/N}$, and the optimal per-stage effort is ≈ 3.6 (≈ 4 in practice, whence "fanout-of-4"). This yields the optimal number of stages $N \approx \log_4(GH)$ directly. In an IP context it answers questions such as "how many buffer stages should the clock tree have?" or "is this 12-input decoder better as one stage or three?" before a single tool is run — which matters because those questions arise during architecture, when no netlist exists.

Graduate · FO4 as a unit. Quoting delay in FO4 units makes a design comparable across processes: a 30-FO4 pipeline stage is 30 FO4 whether the node is 130 nm or 5 nm. A measured data point for calibration: in the open SkyWater 130 nm PDK, an inverter chain with $W_p = 2.0 \mu\text{m}$, $W_n = 1.0 \mu\text{m}$, $L = 0.15 \mu\text{m}$ and $C_L = 6 \text{ fF}$ gives **FO4 $\approx 48 \text{ ps}$** (ngspice; stage delays 47.5 and 44.5 ps, rise time 67.9 ps). That implies a practical ceiling near **1 GHz** for a ~ 20 -FO4 pipeline stage in that process — which is the quantitative reason a 56 Gbd analogue decision loop is impossible there, and why an open-PDK project must choose a digital, sub-rate architecture.

B2.2 Power

$$P = \alpha C V_{DD}^2 f + I_{SC} V_{DD} + I_{leak} V_{DD}$$

Table 31. Power components and levers

Component	Driver	Lever	Cost of the lever
Dynamic (switching)	αCV^2f	Clock gating; reduce α by data gating; lower V_{DD}	Gating adds skew and verification states
Short-circuit	Both devices on during transition	Sharp input slews	Bigger drivers $\rightarrow$ more C
Subthreshold leakage	e^{-V_{TH}/nV_T}	HVT cells, power gating, body bias	Slower; wake-up latency; state retention
Gate leakage	Tunnelling through oxide	High- κ dielectric	Process choice, not design
Glitch power	Unbalanced arrival times	Balance logic depth ; pipeline	Area, latency

Graduate · Glitch power is an IP-level design decision that rarely appears in textbooks. In an unbalanced arithmetic tree, a node can toggle several times before settling, and each toggle costs full switching energy. Ripple-carry adders and unbalanced multiplier trees are notorious. Balancing the tree — or inserting a pipeline register that stops glitch propagation — can reduce datapath power by tens of percent at no functional cost. Because glitching is invisible in RTL simulation (it appears only in gate-level simulation with timing), it is systematically underestimated during architecture. **A design house that reports power measured from a back-annotated gate-level run, with the customer's own data pattern, differentiates itself from one that reports a synthesis estimate.**

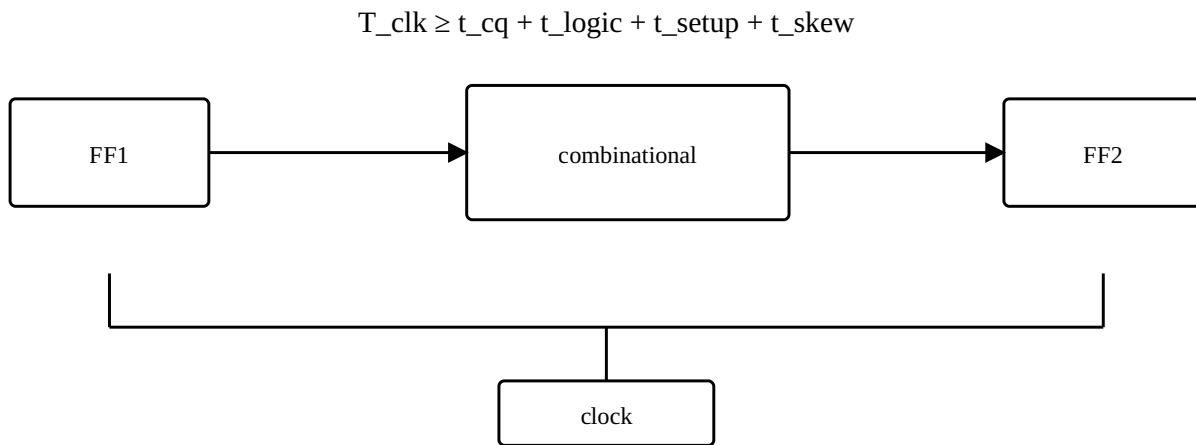
B2.3 Logic families

Table 32. Logic styles

Style	Principle	Advantage	Why it is rarely used in IP
Static CMOS	Complementary pull-up/down	Robust, no clock, easy to verify	— the default
Transmission gate	Pass logic	Fewer devices for MUX/XOR	Threshold loss, charge sharing
Domino / dynamic	Precharge then evaluate	Fast, small	Noise-sensitive, needs custom timing ; not synthesisable
Pass-transistor (CPL)	—	Compact XOR	Level restoration needed
Current-mode (CML/ECL)	Differential current steering	Very high speed, constant current	Static power; used only in SerDes front ends
Adiabatic	Slow charge recovery	Low energy	Needs special clocks; niche

Graduate · A synthesisable IP is static CMOS by definition — everything else requires custom timing verification that a customer cannot repeat in their own flow. **This is a commercial constraint, not a technical preference.** The exception is the SerDes front end, where CML is unavoidable above a few GHz; that is precisely the part that is sold as a hard macro rather than RTL, and it is why the PMA/PCS boundary in a PHY falls where it does.

B3. Timing, Clocking and Static Timing Analysis



$$\text{Hold: } t_{cq} + t_{logic,min} \geq t_{hold} + t_{skew}$$

Setup fails $\Rightarrow$ slow the clock. Hold fails $\Rightarrow$ the chip is dead.

Figure 5. The two timing constraints. Setup limits frequency; hold limits correctness at any frequency.

B3.1 The two inequalities

$$T_{clk} \geq t_{cq} + t_{logic,max} + t_{setup} + t_{skew} + t_{jitter}$$

$$t_{cq,min} + t_{logic,min} \geq t_{hold} + t_{skew}$$

The asymmetry between the two is the single most important operational fact in digital timing. A setup violation is fixed by lowering the clock frequency, so a chip that fails setup still works, just slower. A hold violation has no frequency term: the chip is **broken at every frequency** and the only remedy is a mask change. Consequently hold is fixed exhaustively before tape-out, by buffer insertion, while setup is traded against performance targets. In IP delivery this means the constraint file (SDC) you ship must make the customer's hold analysis possible — omitting clock definitions or false-path declarations can cause a customer to close hold on a path that does not exist, or worse, to miss one that does.

B3.2 Clock distribution

Table 33. Clock network structures

Structure	Skew	Power	Use
H-tree	Low by symmetry	High	Regular arrays, large blocks
Buffered tree (CTS)	Tool-balanced	Moderate	Standard ASIC flow
Mesh / grid	Very low	Very high	High-performance CPUs
Spine + local trees	Moderate	Moderate	Common compromise
Useful skew	Deliberately non-zero	—	Borrow time from a fast stage to a slow one

Graduate · **Useful skew (clock borrowing)** is worth knowing because it turns a timing failure into a scheduling problem. If stage A has slack and stage B does not, delaying B's launch clock moves margin from A to B. The tool does this automatically, but the consequence for an IP vendor is subtle: **a block whose internal paths are already borrowing has little tolerance for the customer's clock tree**. Reporting timing with an assumed skew budget, and stating that budget explicitly in the datasheet, prevents the integration failure where the block meets timing in the vendor's flow and misses it in the customer's.

B3.3 What STA cannot see

Table 34. Limits of static timing analysis

Issue	Why STA misses it	What catches it
False paths	STA is structural, not functional	Designer-declared exceptions — and a wrong declaration hides a real path
Multicycle paths	Same	Declaration plus functional verification
Asynchronous cross-ings	No single clock relation	CDC static analysis , not STA
Metastability	Probabilistic, not deterministic	MTBF calculation; synchroniser depth
Glitches	STA reports arrival, not toggle count	Gate-level power simulation
Crosstalk-induced delay	Depends on aggressor timing	SI-aware STA (signal-integrity mode)
IR drop	Voltage assumed constant	Power/IR co-analysis
On-chip variation	Single delay per cell	OCV / AOCV / POCV derating

Graduate · **Exception declarations are the most dangerous artefact in a timing constraint file**. Each `set_false_path` removes a path from analysis; if the declaration is wrong, that path is simply never checked and the failure appears only in silicon. A mature IP deliverable therefore ships its SDC with a written justification for every exception, and ideally with an assertion or formal property that proves the path is genuinely unexercisable. **This is a place where a modelling engineer contributes directly: the argument that a path cannot be sensitised is a functional argument, and the person who wrote the reference model is usually the one who can make it.**

B3.4 Metastability quantified

$$MTBF = e^{t_r/\tau} / (T_0 f_{clk} f_{data})$$

Graduate · The exponential in t_r (resolution time allowed) is why adding one synchroniser stage improves MTBF by orders of magnitude rather than by a factor of two. With typical 28 nm flip-flop parameters ($\tau \approx 20$ ps), one extra stage at 1 GHz multiplies MTBF by $e^{1000/20} = e^{50}$ — an astronomically large factor. The corollary is that **two stages is almost always enough and three is almost always superstition**, except at very high clock rates where t_r per stage is small. Being able to compute this rather than follow a rule is what lets an IP vendor justify a latency budget to a customer who wants the crossing to be faster.

B4. Interconnect, Transmission Lines and Signal Integrity

Below roughly 250 nm, wires stopped being ideal connections and became circuit elements. Above a few GHz they stop being lumped elements and become transmission lines. Both transitions change what a designer must do.

B4.1 On-chip interconnect as distributed RC

$$t_{\text{delay}} \approx 0.38 R_w C_w L^2 \quad (\text{distributed RC line of length } L)$$

Table 35. Interconnect scaling and its remedies

Problem	Cause	Remedy	Cost
Delay grows as L^2	Distributed RC	Repeater insertion → delay becomes linear in L	Power and area; repeaters can be 30% of a large SoC's dynamic power
Resistance rises with scaling	Smaller cross-section, surface scattering	Thicker upper metal layers for global routes	Fewer routing tracks
Coupling capacitance dominates	Tall narrow wires, close spacing	Increase spacing; shield with ground lines	Routing area
Electromigration	Current density in narrow wires	Width rules, current limits	Area; constrains power grid design
IR drop	Resistance of the power grid	Grid density, decoupling capacitance	Metal resources

Graduate · Optimal repeater insertion follows from minimising the total delay of a line broken into k segments each driven by a repeater of size h . The classic result is that the optimum makes the repeater's intrinsic delay comparable to the wire segment delay, giving $k_{\text{opt}} = L\sqrt{(R_w C_w)/(2R_0 C_0)}$. The practical consequence for an IP vendor is that **a block's timing cannot be guaranteed independent of its physical size**: a 2 mm-wide block has internal wire delays that a synthesis-only estimate will miss entirely. This is why serious IP deliverables include a floorplan hint or a hardened layout, and why "it met timing in synthesis" is not a claim a customer will accept for a large block.

B4.2 When a wire becomes a transmission line

Undergraduate review · A connection must be treated as a transmission line when its electrical length is a significant fraction of the signal's wavelength — in practice when the propagation delay exceeds roughly one tenth of the signal rise time. Since t_r shrinks with every process generation, the threshold length keeps falling.

$$Z_0 = \sqrt{L/C} \quad v_p = 1/\sqrt{LC} = c/\sqrt{\epsilon_r} \quad \Gamma = (Z_L - Z_0)/(Z_L + Z_0)$$

Table 36. Transmission-line effects in a link

Effect	Physical cause	Frequency behaviour	Consequence
Conductor (skin) loss	Current crowds into a skin depth $\delta \propto 1/\sqrt{f}$	$\propto \sqrt{f}$	Dominant loss below ~20 GHz on PCB
Dielectric loss	$\tan\delta$ of the substrate	$\propto f$	Dominant at high frequency; chooses the laminate

Effect	Physical cause	Frequency behaviour	Consequence
Reflections	Impedance discontinuities (vias, connectors)	Resonant dips	Return loss; creates ISI that FFE cannot fully fix
Crosstalk (NEXT/FEXT)	Mutual L and C between traces	Rises with f	Limits lane density
Fibre weave effect	Inhomogeneous ϵ_r of woven glass	—	Skew between differential pair halves
Mode conversion	Asymmetry in a differential pair	—	Differential energy becomes common mode → EMI

Graduate · Why skin effect makes channel loss go as $\sqrt{f}$ and why that shapes the equaliser. Skin depth $\delta = \sqrt{2\rho/\omega\mu}$ shrinks as $1/\sqrt{f}$, so the effective conductor cross-section shrinks and resistance rises as $\sqrt{f}$. The resulting channel magnitude response is approximately $\exp(-k\sqrt{f})$, which in the time domain produces a **long, slowly decaying pulse tail** rather than a sharp one. That specific tail shape is why decision-feedback equalisation works so well on wireline channels: the post-cursor ISI is long but the symbols causing it have already been decided, so they can be subtracted exactly without noise enhancement. **The physics of the conductor selects the equaliser architecture.** An engineer who knows only that "DFE cancels post-cursor ISI" cannot explain why a different channel — say a multipath radio channel with pre-cursor energy — needs a different structure.

B4.3 S-parameters

Table 37. Scattering parameters for a two-port

Parameter	Meaning	Specification name
S_{11}	Input reflection	Return loss (dB, negative is good)
S_{21}	Forward transmission	Insertion loss
S_{12}	Reverse transmission	Isolation
S_{22}	Output reflection	Output return loss
S_{dd}, S_{cc}	Differential / common mode	Mixed-mode S-parameters
S_{cd}, S_{dc}	Mode conversion	Asymmetry metric — an EMI predictor

Graduate · S-parameters are used because at high frequency you cannot measure voltage and current at a port without perturbing it, but you can measure incident and reflected power waves. For a modelling engineer the practical point is the **conversion to a time-domain model**: a channel is delivered as a Touchstone (.s4p) file, and a link simulation needs its pulse response. Doing that conversion correctly requires enforcing **causality and passivity** on measured data that generally violates both because of measurement noise and band limitation. Non-causal channel models produce link simulations that are optimistic in a way that is very hard to notice. **A channel-model conditioning tool is a small, genuinely sellable piece of software**, and it is squarely modelling-team work.

B4.4 Power delivery and simultaneous switching noise

$$V_{noise} = L_{package} \cdot dI/dt + R_{grid} \cdot I$$

Table 38. Power integrity mechanisms

Mechanism	Where	Mitigation
Static IR drop	On-chip grid resistance	Grid density, more straps
d//dt noise (SSN)	Package and board inductance	Decoupling capacitance at three levels: on-die, package, board
Resonance	L_{pkg} with on-die C	Anti-resonance peak typically 50–200 MHz ; add damping
Ground bounce	Shared return inductance	More ground pins, differential IO
Supply-induced jitter	V_{DD} modulates delay	Regulate locally; use supply-insensitive delay cells in clock paths

Graduate · Supply-induced jitter is the mechanism by which a *digital* design decision damages an *analogue* specification. A large synchronous block that starts and stops — a clock-gated accelerator, a burst-mode FEC decoder — produces a current step at the burst rate, which modulates the supply, which modulates the delay of the PLL's delay cells, which appears as a spur in the phase noise, which appears as periodic jitter (PJ) in the link budget. **The FEC block is the root cause of a jitter specification failure**, and nothing in either block's own verification would reveal it. This class of cross-domain coupling is why chip-level power-aware simulation exists, and why an IP datasheet should state the block's *current profile*, not merely its average power.

B5. Analogue Building Blocks

A digital IP house does not design these, but it must model them, specify them, and understand which digital block can compensate which analogue imperfection — because that compensation is often the sellable part.

B5.1 Amplifiers

Table 39. Amplifier topologies

Topology	Gain	Bandwidth	Where used
Common source	$-g_m r_o$	Moderate	Basic gain stage
Cascode	$-g_m (g_m r_o) r_o$	High (reduced Miller)	High-gain stages
Differential pair	—	—	Rejects common-mode noise ; the basis of nearly everything
Folded cascode	High	Good	Wide input range op-amps
Two-stage Miller	Very high	Compensated	General-purpose op-amp
CTLE	peaked	—	Source degeneration creates a zero that boosts high frequency
TIA	$-R_f$	—	Photodiode front end

Graduate · Why a CTLE amplifies noise and an FFE does not have to. A CTLE is a linear analogue filter placed *before* the point where noise is added by the rest of the receiver, but *after* the channel has already attenuated the signal and after thermal noise from the channel and the front end. Boosting high frequency therefore boosts the noise already present in that band by the same factor. A digital FFE placed after the ADC has exactly the same problem. **ADFE** escapes it because it subtracts a *decided* symbol — a noiseless quantity — rather than amplifying a received one. This is the whole reason receivers combine a modest CTLE with a DFE instead of using a large CTLE alone, and it is a piece of reasoning that a modelling engineer must be able to reproduce when asked to justify an architecture.

B5.2 Comparators and references

Table 40. Comparator and reference issues

Block	Key imperfection	Digital compensation
Strong-arm latch	Input-referred offset (mismatch)	Offset calibration DAC trimmed digitally
	Metastability when inputs are close	Allow regeneration time; detect and flag
	Kickback onto the input	Sample-and-hold isolation
Bandgap reference	Curvature over temperature	Piecewise digital trim
	Start-up failure mode	Start-up circuit plus watchdog
Current DAC	Element mismatch	Dynamic element matching (digital)

Graduate · Each row's right-hand column is a digital block with a clean specification, and that is the commercial observation: **the calibration and trim logic around an analogue block is frequently larger than the analogue block itself, and it is synthesisable.** A design house that cannot build a comparator can still own the offset-calibration engine, the DEM scrambler, and the background interleaving-mismatch corrector. These are sold as part of converter IP, and they are where the modelling effort concentrates because their correctness is statistical rather than logical.

B5.3 PLLs and clock generation

$$H(s) = K_{PD}K_{VCO}F(s) / (s + K_{PD}K_{VCO}F(s)/N)$$

Table 41. PLL noise transfer

Noise source	Transfer to output	Design implication
Reference noise	Low-pass ($\times N$ in band)	In-band noise is multiplied by $20 \log_{10} N$
VCO noise	High-pass	Wide loop suppresses VCO noise
Divider noise	Low-pass	—
Charge-pump noise	Low-pass	Dominates in-band floor
$\Sigma\Delta$ quantisation (fractional-N)	High-pass shaped	Loop must be narrow enough to filter it

Graduate · The opposing shapes create the **optimal loop bandwidth**: wide enough to suppress VCO phase noise, narrow enough to suppress reference and fractional- N quantisation noise. The optimum is where the two contributions cross. This is a genuinely quantitative design decision that a system modeller can and should compute, and it is directly connected to the link jitter budget of Chapter A4 — the integrated phase noise becomes the RJ term that is multiplied by 14 at 10^{-12} BER. **Chapters A4, B4 and B5 are three views of the same budget**, and one of the marks of a senior engineer is holding them together.

B5.4 Data converters

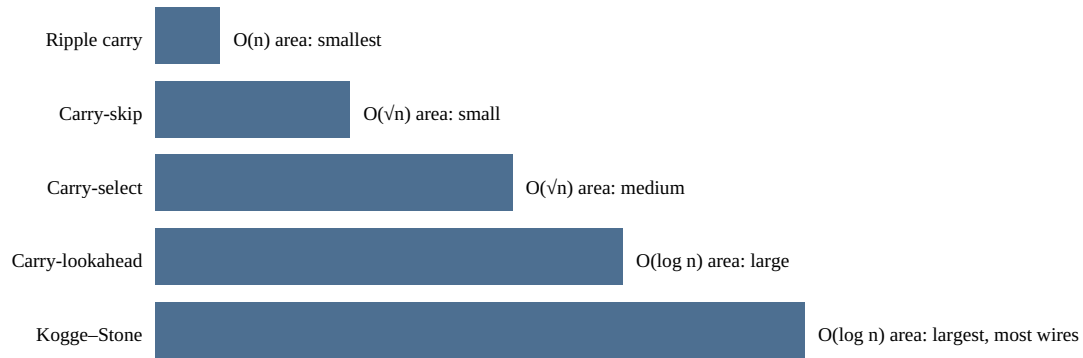
Table 42. ADC architectures

Architecture	Speed	Resolution	Principle	Digital support needed
Flash	Highest	≤ 6 bit	$2^N - 1$ comparators	Thermometer $\rightarrow$ binary, bubble correction
SAR	Moderate–high	8–14 bit	Binary search with a DAC	Redundancy and digital error correction
Pipeline	High	10–14 bit	Stages with residue amplification	Inter-stage gain calibration
$\Sigma\Delta$	Low–moderate	16–24 bit	Oversample and noise-shape	Decimation filter (CIC + FIR)
Time-interleaved SAR	>10 GS/s	6–8 bit	M SARs in parallel	Offset/gain/skew background calibration — the 112G receiver case
Folding/interpolating	High	8–10 bit	Reduce comparator count	—

Graduate · **$\Sigma\Delta$ noise shaping** is worth stating exactly because it is the clearest example of trading bandwidth for resolution. An L th-order modulator with oversampling ratio OSR gives SNR gain $\approx (6L+3)$ dB per doubling of OSR, versus 3 dB for plain oversampling. The price is that the modulator output is a low-resolution, very fast stream that must be decimated — and **that decimation filter is a substantial digital IP block** whose passband ripple and stopband attenuation determine the converter's final specification. A converter vendor's datasheet number is in part a statement about a digital filter. This is the second time in this chapter that the saleable digital block sits next to an analogue one.

C1. Arithmetic Units — The Core of Every Datapath

C1.1 Addition



Same function. Five points on the area–delay curve.

Figure 6. Adder architectures span an area–delay curve of roughly an order of magnitude in each axis.

Table 43. Adder architectures

Architecture	Delay	Area	Notes
Ripple carry	$O(n)$	$O(n)$	Baseline; fine for $n \leq 8$ at low frequency
Carry-skip	$O(\sqrt{n})$	$O(n)$	Cheap improvement
Carry-select	$O(\sqrt{n})$	$\sim 2\times$	Duplicate blocks, select on carry
Carry-lookahead	$O(\log n)$	$O(n \log n)$	Classic
Kogge–Stone	$O(\log n)$	Large, heavy wiring	Minimum depth; wire-limited in practice
Brent–Kung	$O(\log n)$	Fewer wires	Depth $2 \log n$; better area
Carry-save	$O(1)$ per add	$O(n)$	Does not propagate carry — the key to fast multipliers and accumulators

Graduate · **Carry-save is the single most useful idea in this table.** A carry-save adder takes three operands and produces two (sum and carry vectors) in constant time, because no carry propagates. A chain of them accumulates many terms at constant delay per term, and only the *final* conversion to a normal binary number requires a carry-propagate adder. This is why multiplier trees, FIR accumulators, and MAC units are built as "compress many partial products with carry-save, then one fast adder at the end". **In an accumulation loop — the feedback path that limits every recursive filter — keeping the state in carry-save form removes the carry-propagate delay from the loop entirely.** That single transformation has rescued more timing-critical datapaths than any other.

C1.2 Multiplication

Table 44. Multiplier construction

Stage	Technique	Effect
Partial product generation	Booth radix-4	Halves the number of partial products; needs 2's-complement handling
	Booth radix-8	Fewer still, but needs a 3× multiple (extra adder)
Reduction	Wallace tree	$O(\log n)$ depth, irregular layout
	Dadda tree	Slightly fewer adders than Wallace
	Array	Regular layout, $O(n)$ depth
Final add	Fast carry-propagate adder	Kogge–Stone or similar
Whole unit	Pipelining	Cut the tree with registers to raise $f_{\max}$

Graduate · **Constant multiplication is a different problem** and is worth separating in any specification. When one operand is a compile-time constant — filter coefficients, twiddle factors, scaling constants — the multiplier collapses into shifts and adds. Canonical signed digit (CSD) representation minimises the number of non-zero digits, and **multiple constant multiplication (MCM)** algorithms share sub-expressions across a whole coefficient set. For a 32-tap FIR with fixed coefficients this routinely gives a 3–5× area reduction over generic multipliers. The commercial point: **an IP that is parameterised for run-time coefficients cannot use this, so "programmable coefficients" is an expensive feature** that should be priced and specified as such, not thrown in because it seems flexible.

C1.3 Division, square root, and reciprocal

Table 45. Division approaches

Method	Convergence	Hardware	Use
Restoring / non-restoring	1 bit per cycle	Adder + shifter	Small area, high latency
SRT (radix-4/8)	2–3 bits per cycle	Lookup for quotient digit	Standard in FPU's
Newton–Raphson	Quadratic (doubles digits)	Multipliers	Reuses the multiplier; good when one exists
Goldschmidt	Quadratic	Multipliers	Better pipelining than Newton
Reciprocal LUT + refine	—	Small ROM + multiplier	The usual answer in DSP

The most important division fact for an IP designer is architectural, not numerical: never put a divider in a feedback loop. Its latency enters the loop directly and sets the initiation interval. The standard transformation is to compute a reciprocal *once* outside the loop and multiply inside it. The Cholesky decomposition is a textbook example — the diagonal reciprocal is formed once per column and the column is then scaled by multiplication, which is why the inner loop can be pipelined at $II = 1$. This is also a place where the reference model and the RTL can legitimately differ: the model may divide, the RTL multiplies by a reciprocal, and the two agree only to within the reciprocal's rounding. **The specification must state which one is normative.**

C1.4 Fixed-point and floating-point units

Table 46. Number format decisions

Format	Range	Hardware cost	Where it belongs
Fixed point $Q_{m.n}$	Limited, uniform	Adder/multiplier only	Most DSP and comms datapaths
Block floating point	Shared exponent per block	Small extra	FFT, where dynamic range grows predictably

Format	Range	Hardware cost	Where it belongs
IEEE 754 binary32	Wide	Align, normalise, round — expensive	Scientific, some solvers
binary16 / bfloat16	Wide, low precision	Cheaper	Neural inference
Integer with saturation	Limited, clamped	Compare and clamp	Where wrap would be catastrophic (LLRs, audio)
Posit / logarithmic	Tapered	Unusual	Research; rarely in shipped IP

Graduate · **Block floating point deserves more attention than it gets.** In an FFT the signal magnitude grows by up to a factor of two per stage. Full floating point is overkill; fixed point requires either wide words or per-stage scaling that loses SNR. Block floating point keeps one exponent per stage (or per block) and normalises once, recovering most of floating point's dynamic range at a small fraction of the cost. **For a modelling engineer this format is a trap**, because a naive floating-point reference model will not match the hardware's block-normalised behaviour bit for bit — the reference must model the normalisation decisions too. Specifying "bit-accurate reference model" without specifying the scaling policy produces months of false mismatches.

C2. Control: State Machines, Pipelines and Flow Control

C2.1 FSM encoding

Table 47. State encodings

Encoding	Re-gisters	Next-state logic	When
Binary	$\log_2 N$	Complex	Many states, area critical
Gray	$\log_2 N$	Complex	Adjacent states differ by one bit — low power, safer across CDC
One-hot	N	Simple (one AND per transition)	FPGA default ; fast; easy to check
Johnson	$N/2$	Simple	Counters, ring sequencers
Hamming-protected	$>\log_2 N$	—	Safety/security : illegal states detectable

Graduate · **Security-grade FSM encoding** is a concrete example of a requirement that comes from a threat model rather than from function. If an attacker can flip a state bit with a laser or a voltage glitch, a binary-encoded FSM may land in a valid but wrong state — for instance skipping rounds of a cipher. Countermeasures are: sparse encodings with large Hamming distance so that a single flip lands in an illegal state; a default branch that raises an alarm rather than doing nothing; and **duplicated FSMs with complementary encodings** whose outputs must remain complementary. The last technique requires a synthesis barrier so the tool does not optimise the inverters away and merge the two copies — if it does, the redundancy disappears while *functional verification still passes*. That silent failure mode is the reason such designs contain explicit buffer primitives whose only purpose is to stop the optimiser.

C2.2 Pipelining and its hazards

Table 48. Pipeline transformations

Transformation	What it does	Cost	Limit
Pipelining	Insert registers between logic stages	Latency, area	Cannot cut a feedback loop
Retiming	Move existing registers across logic	None (if legal)	Total registers on each loop is invariant
Loop un-rolling	Process k iterations per cycle	$k \times$ area	The loop's critical path is divided by k only in the feedforward part
Speculation	Compute all outcomes, select afterwards	Exponential in the number of speculated bits	Removes the decision from the loop
C-slow	Interleave k independent streams	Registers $\times k$	Needs k independent problems
Algebraic transform	Restructure the recurrence itself	Design effort	The only way past a fundamental loop bound

Graduate · **The iteration bound** is the hard limit that these transformations negotiate with. For a recursive system, the minimum achievable iteration period is $T_\infty = \max_{\text{loops}} (t_{\text{loop}}/w_{\text{loop}})$, where t_{loop} is the total computation delay around a loop

and w_{loop} the number of registers in it. Retiming and pipelining cannot beat it; only changing the recurrence can. Recognising this quantity turns a vague "we need more speed" into a specific question: *which loop, and can its algebra be rewritten?* Look-ahead transformation (computing two steps of a recurrence in one) doubles w and thereby halves T_{∞} , at the cost of more arithmetic. **This is the abstract form of the unrolled DFE, radix-4 Viterbi ACS, and reformulated Berlekamp-Massey — three blocks that look unrelated and are the same transformation.**

C2.3 Flow control and backpressure

Table 49. Handshake styles

Style	Signals	Property	Hazard
Valid only	valid	Producer never stalls	Consumer must always accept — needs guaranteed rate
Valid/Ready	valid, ready	Full backpressure	Combinational loop if ready depends on valid
Credit-based	credits returned	Decouples latency from throughput	Credit accounting bugs
Req/Ack (4-phase)	(4- handshake pairs)	Async-safe	Slow (two round trips)

The valid/ready combinational loop is one of the most common integration failures in modular RTL. If module A's ready depends combinationaly on its input valid, and module B's valid depends combinationaly on its input ready, connecting them creates a zero-delay cycle. Each module is individually correct and passes its own tests. The standard remedies are a *skid buffer* (also called a spill register), which breaks the path at the cost of one register stage and one entry of buffering, or a rule that ready is registered. **An IP that documents which of its interfaces are combinationaly coupled — and ships a skid buffer — saves its customers a week each.** This is exactly the kind of detail that distinguishes a product from a piece of code.

C2.4 Arbitration

Table 50. Arbiter types

Type	Fairness	Latency	Use
Fixed priority	None (starvation possible)	Lowest	Strict QoS classes
Round robin	Fair	Low	General purpose
Weighted round robin	Proportional	Low	Mixed traffic classes
Matrix / LRU	Fair, true least-recently-used	Higher	Caches
Lottery	Statistical	Low	Rare
Age-based	Strict FIFO fairness	Higher	Avoids starvation strictly

Graduate · Arbitration is where **fairness and forward progress** become formal properties rather than intentions. A round-robin arbiter is fair by construction; a fixed-priority arbiter is not, and a low-priority requester can starve forever. Whether starvation matters is a system question — it is acceptable for a debug port, fatal for a DMA that feeds a real-time codec. Because starvation is a *liveness* property ("eventually granted") rather than a safety property, it is natural to express and prove with formal tools, and arbiters are one of the few places where formal verification is both easy and genuinely valuable. **Shipping a formally proven arbiter with its properties is a differentiator that costs a design house little.**

C3. Clock Domain Crossing, Reset and Low Power

C3.1 The CDC problem stated precisely

Undergraduate review · A signal crossing between unrelated clock domains violates setup or hold at the receiving flip-flop with non-zero probability. The flip-flop may enter a metastable state whose resolution time is unbounded (though exponentially unlikely to be long). **This cannot be simulated:** a logic simulator has no metastable value and will always resolve to 0 or 1.

Table 51. CDC techniques by payload type

Payload	Technique	Why	Failure if misapplied
1-bit level	Two-flop synchroniser	Allows a full cycle to resolve	—
1-bit pulse	Toggle synchroniser (level convert, then edge detect)	A pulse shorter than the destination period would be lost	Lost events
Multi-bit, unrelated	Synchronise a request; use a data hold register	Bits arriving in different cycles corrupt the value	Corrupted data words
Multi-bit counter	Gray code	Only one bit changes per increment	Wrong pointer → FIFO corruption
Stream	Asynchronous FIFO (Gray pointers, dual-port RAM)	Decouples rate and phase	—
Bus transaction	Handshake (req/ack) or credit	Explicit acknowledgement	—
Reset	Asynchronous assert, synchronous de-assert	De-assertion must not violate recovery/removal	Part of the design leaves reset a cycle late

Graduate · Why Gray coding a FIFO pointer is safe even though the pointer is multi-bit. The synchroniser may sample a Gray pointer mid-transition, but since only one bit changes per increment, the sampled value is either the old or the new pointer — never a third value. An off-by-one pointer makes the FIFO report itself slightly *fuller* or *emptier* than it is, which is conservative: the FIFO may stall a cycle early but never overflows or underflows. **The design is safe because its error mode is biased in the harmless direction.** This principle — arrange for uncertainty to fail conservatively — generalises far beyond FIFOs and is one of the most transferable ideas in digital design.

C3.2 Reset architecture

Table 52. Reset strategies

Strategy	Advantage	Problem
Synchronous	No recovery/removal issue; clean STA	Needs a running clock — fails at power-up
Asynchronous	Works without a clock	De-assertion can violate recovery
Async assert, sync de-assert	Both advantages	Requires a reset synchroniser per domain — the standard solution
Reset-less registers	Area and routing saving	Must be provably initialised by data flow; a verification obligation

Reset de-assertion order is a specification item that is almost always omitted. If two blocks leave reset in different cycles and one samples the other's outputs, the receiving block may latch garbage. Within a block the designer controls this; across an IP boundary the *customer* controls it, and they cannot get it right unless the datasheet states the requirement. A good IP datasheet specifies: the reset polarity, whether it is synchronous or asynchronous, the minimum assertion width in cycles of each clock, and any required ordering between multiple resets. **Omitting this produces intermittent bring-up failures that are blamed on the IP.**

C3.3 Low power design

Table 53. Power reduction techniques by level

Level	Technique	Saving	Verification burden
RTL	Clock gating	Large (clock tree is 20–40% of dynamic power)	Low — tools insert it automatically
RTL	Operand isolation / data gating	Moderate	Low
RTL	Reduce switching activity (encoding, bus inversion)	Moderate	Low
Architecture	Parallelism + voltage scaling	Large — power $\propto V^2$	Moderate
Architecture	Memory hierarchy (avoid far accesses)	Large	Moderate
Physical	Multi- V_{TH} cells	Leakage	None
System	Power gating	Leakage $\rightarrow \sim 0$	High: isolation, retention, wake sequencing
System	DVFS	Large	High: timing at every operating point
System	Multiple power domains	Large	Highest: UPF, isolation cells

Graduate · Parallelism as a power technique is the least intuitive entry and the most powerful. Two copies of a block running at half the clock frequency deliver the same throughput; because they now have twice the time budget, the supply voltage can be lowered until the slower path just meets timing. Dynamic power scales as $V^2 f$, so halving f and reducing V by, say, 30% gives roughly $0.5 \times 0.49 = 0.25$ of the original power, at the cost of about twice the area. **This trade — area for energy — is the fundamental reason accelerators are more efficient than processors**, and it is the argument an IP vendor uses when selling a block against a software implementation. State it with the exponents and it is persuasive; state it as "our block is more efficient" and it is not.

C3.4 Design for test

Table 54. DFT structures

Structure	Tests what	Cost	IP implication
Scan chains	All flip-flops and combinational logic	Mux in every FF (~5–10% area, some delay)	IP must be scan-insertable: no gated clocks without test bypass, no latches, no async loops
MBIST	Embedded memories	Controller per memory group	Memory interfaces must be accessible
BSD (JTAG 1149.1)	Board-level interconnect	Boundary cells	—

Structure	Tests what	Cost	IP implication
LBIST	Logic, in-field	PRPG + MISR	Automotive/safety
ATPG coverage	—	—	Stuck-at >99%, transition >90% typical targets
Test compression	—	Decompressor/compactor	Reduces tester time and cost

Scan is a security hole. A scan chain can read and write every flip-flop in the design, including key registers. Production chips must therefore disable scan after manufacturing test, typically by blowing a fuse or advancing a life-cycle state. A security IP that does not document its scan-isolation requirement will be integrated insecurely. **Conversely, an IP that cannot be scanned at all is unsellable**, because the customer cannot achieve their test coverage target. The resolution — scannable, but with key registers excluded from the chain or cleared on scan entry — must be designed in, and it is a requirement that comes from neither function nor timing.

D1. Instruction Set Architecture

An ISA is a contract between hardware and software. For a modelling engineer it is the *specification document* in the most literal sense: the reference model **is** the ISA, written executably.

Table 55. ISA design axes

Axis	Options	Consequence for hardware
Operand model	Stack / accumulator / register-memory / load-store	Load-store simplifies pipelining; every RISC uses it
Instruction length	Fixed / variable / compressed	Fixed simplifies fetch; compressed (RVC, Thumb) saves 25–30% code size at the cost of alignment logic
Addressing modes	Few / many	Many modes need an address generation unit; few modes need more instructions
Condition handling	Flags / compare-and-branch	Flags create a serialising dependency ; RISC-V deliberately has none
Delay slots	Present (classic MIPS) / absent	Delay slots expose the pipeline in the ISA — a design mistake later reversed
Register count	8 / 16 / 32 / more	More registers reduce spills but widen every instruction field
Exceptions	Precise / imprecise	Precise exceptions constrain the whole microarchitecture

Graduate · Delay slots are the canonical example of leaking microarchitecture into architecture. MIPS exposed its single branch-delay slot so that the compiler could fill it, saving a stall in a five-stage pipeline. When later implementations had deeper pipelines and branch prediction, the delay slot became useless baggage that every implementation still had to honour. **The lesson for IP design generalises: a specification that encodes today's implementation constrains tomorrow's.** When writing a block's interface, ask which parts describe *what* and which describe *how*, and expose only the former.

Graduate · Precise exceptions mean that when an exception is taken, all instructions before it have completed and none after it has had any effect. This is what makes an ISS a valid reference: the model can execute one instruction at a time and still describe a machine that executes many concurrently. Achieving it in an out-of-order machine requires a reorder buffer and speculative state, which is a large fraction of such a core's complexity. **The entire step-and-compare verification methodology rests on precise exceptions**; without them there would be no well-defined point at which model and RTL should agree.

D1.2 RISC-V as a case study

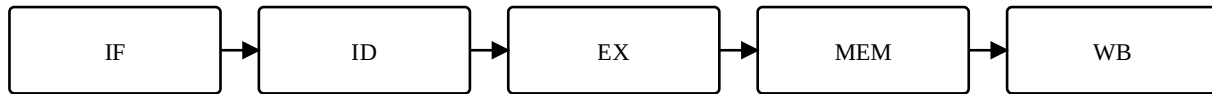
Table 56. RISC-V structure

Element	Content	Modelling note
Base integer	RV32I / RV64I — ~40 instructions	Small enough to model exhaustively
M	Multiply/divide	Division by zero is defined , not trapping
A	Atomics (LR/SC, AMO)	Requires a memory consistency model
F/D/Q	Floating point	IEEE 754; needs SoftFloat, not host arithmetic
C	Compressed 16-bit	Alignment: instructions may straddle a word
V	Vector	Vector-length agnostic — a model must track <code>v1</code> , <code>vtype</code>
B, Zk, Zc...	Bit manipulation, crypto, code size	—

Element	Content	Modelling note
Privileged	M/S/U modes, CSRs, PMP, virtual memory	Where most model bugs live: exception priority and CSR side effects

Exception priority is the most common source of reference-model defects in a CPU project. One instruction can simultaneously satisfy the conditions for several exceptions — misaligned address, page fault, access fault, breakpoint. The specification gives a strict priority order. A model that implements a different order is correct on almost every test and wrong on a narrow class of inputs that constrained-random stimulus may take weeks to find. The remedy is not more random testing but **directed tests derived from the priority table itself:** enumerate the table, construct one case per row, and check. This is a half-day of work that reliably finds bugs a month of random testing misses.

D2. Pipelining, Hazards and Speculation



Throughput 1 instr/cycle; latency 5 cycles

hazards: structural · data · control

Forwarding removes most data hazards; load-use needs one stall

Control hazards cost the branch penalty × misprediction rate

Figure 7. The classic five-stage pipeline and its three hazard classes.

Table 57. Hazards and remedies

Hazard	Cause	Remedy	Residual cost
Structural	Two instructions need the same resource	Duplicate the resource, or stall	Area
Data RAW	Read after write	Forwarding from EX/MEM	Load-use still costs one cycle
Data WAR / WAW	Anti- and output dependencies	Register renaming	Only in out-of-order designs
Control	Branch outcome unknown at fetch	Prediction + speculative execution	Misprediction penalty × rate
Memory ordering	Loads and stores to the same address	Store buffer with forwarding, memory disambiguation	Complexity

Table 58. Branch prediction

Predictor	Mechanism	Typical accuracy
Static (backward taken)	Loop heuristic	~65%
Bimodal	2-bit saturating counter per PC	~85%
gshare	Global history XOR PC indexes the table	~93%
Tournament	Choose between local and global predictors	~95%
TAGE	Multiple history lengths with tags	>97%
Perceptron	Linear classifier over history	>97%
Return address stack	Dedicated stack for call/return	>99% for returns

Graduate · Why a misprediction is expensive and getting more so. The penalty is the pipeline depth from fetch to resolve. In a 5-stage machine that is 2–3 cycles; in a 15-stage out-of-order machine it is 15–20, and the machine may have to discard dozens of in-flight instructions. With a 5% miss rate and a 15-cycle penalty, branches alone cost 0.75 cycles per branch — often more than every other stall combined. This is why predictor research continues to be worthwhile and why deep pipelines

have diminishing returns. **For an accelerator IP designer the lesson is the opposite and liberating: a fixed-function datapath has no branches, so none of this cost exists.** Quantifying that difference — "your software loop spends 0.75 cycles per iteration on branch misprediction alone" — is one of the strongest arguments an accelerator vendor has.

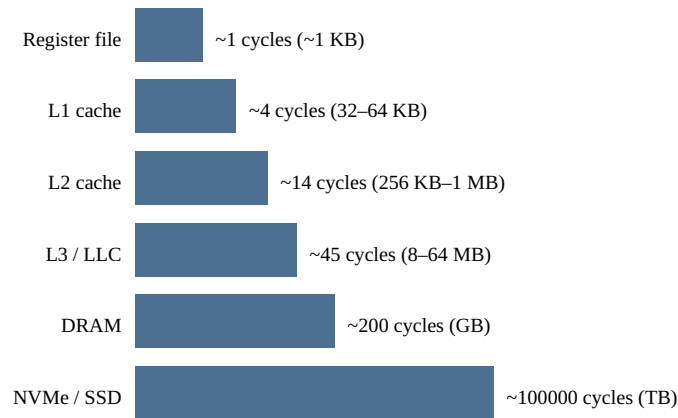
D2.2 Out-of-order execution

Table 59. Out-of-order machinery

Structure	Purpose	Scaling limit
Register renaming (RAT)	Removes WAR/WAW	Physical register file size
Reservation stations / issue queue	Hold instructions until operands ready	Wakeup-select loop is the critical path
Reorder buffer (ROB)	In-order commit → precise exceptions	Entries scale with memory latency
Load/store queue	Memory disambiguation	Associative search cost
Scoreboard	Simpler alternative (no renaming)	Stalls on WAR/WAW

Graduate · The **wakeup-select loop** — broadcasting a result tag, waking dependent instructions, selecting which to issue — must complete in one cycle for back-to-back dependent execution. It is a broadcast across the whole issue queue, so its delay grows with queue size and with issue width. This single loop, not transistor speed, is why issue widths have plateaued around 4–8 and why further performance has come from more cores rather than wider ones. **It is the same phenomenon as the DFE feedback loop and the Viterbi ACS loop: a recurrence whose latency cannot be pipelined away.** Recognising the pattern across domains is exactly the transferable skill this book aims to build.

D3. Memory Hierarchy and Coherence



Each level is roughly an order of magnitude slower and larger

Figure 8. The memory hierarchy. Each level trades capacity against latency by roughly an order of magnitude.

Table 60. Cache design parameters

Parameter	Effect	Trade
Block size	Exploits spatial locality	Too large wastes bandwidth and causes false sharing
Associativity	Reduces conflict misses	More comparators, longer hit time; 4–8 way is the usual optimum
Capacity	Reduces capacity misses	Area, leakage, hit latency
Write policy	Write-through vs write-back	Write-back reduces bandwidth; needs dirty bits and coherence
Allocation	Write-allocate vs not	Depends on write locality
Replacement	LRU / pseudo-LRU / random	True LRU is expensive beyond 4-way; pseudo-LRU is within 1%
Inclusion	Inclusive / exclusive / NINE	Affects snoop filtering

Undergraduate review · Misses classify as **compulsory** (first reference), **capacity** (working set exceeds the cache), **conflict** (limited associativity), and in multiprocessors **coherence** (invalidated by another core). The classification is useful because each class has a different remedy: prefetching for compulsory, larger cache for capacity, higher associativity or better indexing for conflict, and data layout changes for coherence.

Table 61. Coherence protocols

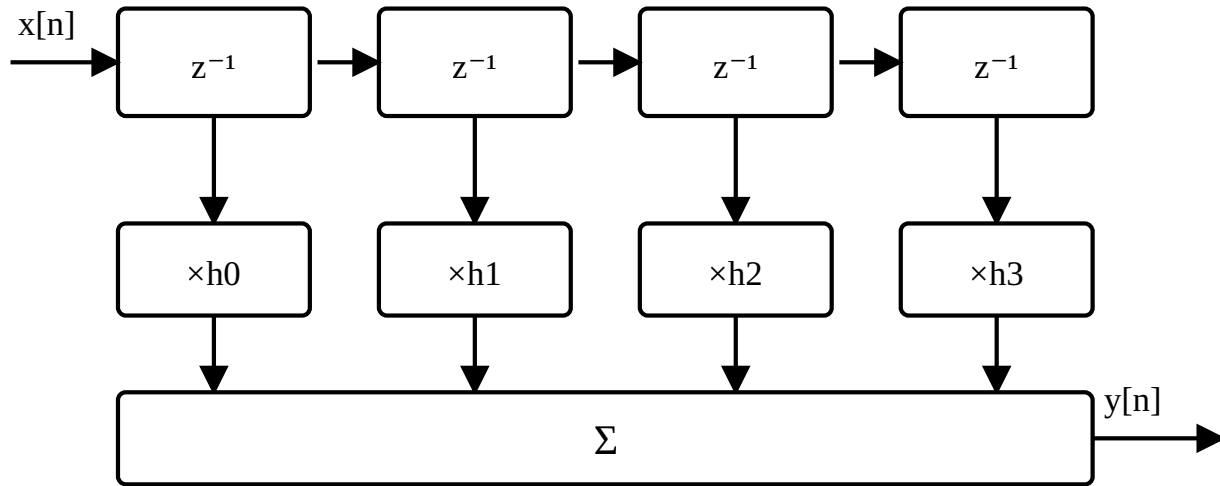
Protocol	States	Note
MSI	Modified, Shared, Invalid	Minimal
MESI	+ Exclusive	Avoids a bus transaction on read–modify
MOESI	+ Owned	Allows dirty sharing; saves writebacks
Directory-based	—	Scales beyond a bus ; required for many-core
Snoop filter	—	Reduces useless snoops in large systems

Graduate · Memory consistency is not coherence. Coherence concerns a single address — all cores must see one order of writes to it. Consistency concerns the relative order of operations to *different* addresses. A machine can be perfectly coherent and still reorder a store to A before a store to B as observed by another core. The consistency model (sequential, TSO, release, RVWMO) specifies what reorderings are allowed and which fences prevent them. **For a modelling engineer this is the hardest specification to capture**, because the reference model is inherently sequential while the hardware is not: a simple step-and-compare against an ISS *cannot* verify a weak memory model. The correct approach is **litmus tests** — short multi-threaded programs with enumerated permitted outcomes — checked against the architecture's formal memory model. Knowing that a different verification technique is required, and why, is the point.

Table 62. DRAM characteristics that leak into system design

Property	Meaning	Design response
Row buffer	A row must be activated before access	Exploit row locality; schedule for open rows
$t_{\text{RCD}}, t_{\text{RP}}, t_{\text{RAS}}$	Activate/precharge timings	Memory controller scheduling
Refresh	Every ~32–64 ms	Blocks access; matters for real-time guarantees
Bank/rank parallelism	Independent banks	Interleave addresses to exploit it
Burst length	Minimum transfer granularity	A 4-byte read still fetches 32–64 bytes
Rowhammer	Repeated activation disturbs neighbours	A security problem solved by refresh management — a DRAM controller feature

E1. Digital Filters as Hardware



Direct form: adder tree depth grows with N

Transposed form: critical path is one mult + one add, independent of N

Figure 9. Direct-form FIR. The tap count sets the multiplier count; the *form* sets the critical path.

Table 63. FIR structures and their costs

Structure	Multipliers	Critical path	Choose when
Direct form	N	1 mult + $\log N$ adds	N small
Transposed direct form	N	1 mult + 1 add (constant)	Almost always, for speed. Cost: high fanout on $x[n]$
Symmetric (folded)	$\lceil N/2 \rceil$	1 add + 1 mult + 1 add	Linear phase — free 2× saving
Polyphase	N/M effective	—	Combined with rate change
Distributed arithmetic	0 (LUTs)	W cycles	Fixed coefficients, LUT fabric
CSD / multiplierless	0	adds only	Fixed coefficients, ASIC area
Frequency domain	—	—	$N \gtrsim 100$ and block latency acceptable

Graduate · The fanout cost of the transposed form is real and is often overlooked in architecture documents. The input sample must reach all N multipliers in one cycle. For $N = 128$ at 500 MHz this is a significant buffer tree whose delay and power must be budgeted. The usual remedy is to *partition* the filter into segments with a pipelined input distribution network — effectively reintroducing a small amount of direct-form structure. **Real designs are hybrids, and a specification that names only "transposed FIR" is incomplete.**

E1.2 Filter design methods

Table 64. FIR design methods

Method	Optimality	Control	Use
Windowing	Suboptimal	Simple	Quick designs
Frequency sampling	Suboptimal	Direct in frequency	—
Parks–McClellan (Remez)	Minimax optimal	Ripple in each band	Standard for equiripple specs
Least squares	L2 optimal	Weighted error	When energy matters more than peak
Convex optimisation	Flexible	Arbitrary constraints	Handles quantised coefficients directly

Graduate · Designing in floating point and rounding afterwards is a mistake that costs stopband attenuation. Rounding a Parks–McClellan design to B bits perturbs each coefficient by up to half an LSB, and the resulting stopband degradation can be 10–20 dB. The correct procedure is to include the quantisation in the design: either use an integer-programming or convex formulation that searches over representable coefficients, or re-optimize the remaining coefficients after fixing some. **For an IP vendor this is a concrete deliverable: ship the coefficient generator, not just the coefficients**, so the customer can re-tune for their own band plan without repeating the analysis.

E1.3 Multirate structures

Table 65. Rate conversion building blocks

Block	Function	Key property
Decimator ($\downarrow M$)	Filter then discard $M-1$ of every M	Filter first , or aliasing is irreversible
Interpolator ($\uparrow L$)	Insert $L-1$ zeros then filter	Zeros create images; the filter removes them
Polyphase decomposition	Split the filter into M sub-filters	Avoids computing samples that are discarded — $M\times$ saving
CIC	Integrator–comb cascade	No multipliers ; passband droop must be compensated
Half-band	Every other coefficient is zero	$\sim 2\times$ saving for 2:1 rates
Farrow	Polynomial interpolation with variable delay	Arbitrary, time-varying resampling ratio — used in timing recovery

Graduate · The CIC word-growth question is a classic fixed-point trap. An N -stage CIC with rate change R and differential delay M has DC gain $(RM)^N$, so the integrators need $W_{\text{out}} = W_{\text{in}} + N \log_2(RM)$ bits. The textbook trick is to use **two's-complement wrap-around** in the integrators: intermediate values overflow, but because the comb stages later subtract, the overflow cancels exactly, provided the *final* result fits. This works because wrap-around arithmetic is a ring homomorphism. **Replace wrap with saturation — an apparently safer choice — and the cancellation is destroyed and the filter breaks.** A specification that says only "use saturating arithmetic throughout" would silently kill a correct CIC. This is why Part A's number-system chapter insists that overflow behaviour is a per-block decision.

E1.4 Adaptive filters

Table 66. Adaptive algorithms in hardware terms

Algorithm	Update	Multipliers per tap per update	Convergence
LMS	$\mathbf{w} \leftarrow \mathbf{w} + \mu \mathbf{e} \mathbf{x}^*$	2	Slow; sensitive to eigenvalue spread
NLMS	μ normalised by $\ \mathbf{x}\ ^2$	2 + norm	Robust to input level
Sign-error LMS	uses $\text{sgn}(\mathbf{e})$	1	Slower

Algorithm	Update	Multipliers per tap per update	Convergence
Sign-data LMS	uses $\text{sgn}(x)$	1	Slower
Sign-sign LMS	both signs	0 — adds only	Slowest; used in every high-speed SerDes
RLS	inverse correlation update	$O(N)$ per tap	Fast; expensive
Frequency-domain LMS	block FFT	—	Fast for long filters

Graduate · Why sign-sign LMS dominates at 56 GBd. Two independent facts combine. First, the channel changes on a timescale of milliseconds while symbols arrive every 18 ps — **adaptation can be nine orders of magnitude slower than the datapath**. Second, multipliers cannot run at the symbol rate at all. Taking only signs reduces the update to increment/decrement, which can be accumulated over thousands of symbols and applied occasionally. The resulting architecture has a *fast datapath* and a *slow adaptation loop* in a different clock domain. **For a modelling engineer this separation must be explicit in the model**, because a model that updates taps every symbol will not match RTL that updates them every 4096 symbols, even though both "implement sign-sign LMS". The update schedule is part of the specification.

E2. Transforms in Hardware: FFT and Friends

Table 67. FFT algorithm families

Algorithm	Complex multiplies	Structure	Note
Radix-2 DIT/DIF	$(N/2)\log_2 N$	Regular butterflies	Simplest control
Radix-4	fewer	4-point butterflies	~25% fewer multiplies
Split-radix	fewest for radix-2 ^k	Irregular	Control complexity
Winograd / PFA	Minimum multiplies	Length must factor into coprimes	Niche
Pipelined SDF	—	One butterfly per stage + delay line	Streaming; the standard for OFDM
MDC	—	Multiple data paths	Higher throughput
SSR / parallel	—	Several samples per clock	When the data rate exceeds the clock rate

Graduate · The resource that actually limits an FFT is memory and the twiddle factors, not the butterflies. A pipelined N -point SDF FFT needs $\log_2 N$ butterflies but a total of $N-1$ words of delay line, and a twiddle ROM whose size grows with N . For $N = 4096$ that is thousands of words of storage against a dozen arithmetic units. Two standard optimisations follow: exploit twiddle symmetry to store only one eighth of a period and derive the rest by sign and swap, or generate twiddles with a CORDIC instead of a ROM. **Which one wins depends on whether the target has cheap memory (FPGA) or cheap logic (ASIC)** — another instance of the rule that architecture choices are technology-dependent and must be re-derived, not memorised.

Bit-reversed output order is an interface decision, not an implementation detail. A DIF FFT naturally produces its output in bit-reversed order. Reordering costs a full N -word buffer and its bandwidth. In many systems the consumer — a channel equaliser, a magnitude detector — does not care about order, so the reordering is pure waste. Competent designs pass the bit-reversed order through and let the next block absorb it. **This means the reference model and the RTL will disagree about output order unless the specification says which convention is normative**, and a large fraction of "FFT mismatch" bug reports are exactly this.

Table 68. Numerical behaviour of fixed-point FFT

Issue	Cause	Handling
Magnitude growth	Each radix-2 stage can double magnitude	Scale by $\frac{1}{2}$ per stage (lose SNR), grow the word, or block floating point
Twiddle quantisation	Finite coefficient precision	Adds a noise floor; usually 2–4 bits more than data width
Rounding vs truncation	Bias accumulates over $\log N$ stages	Convergent rounding avoids DC bias
Overflow detection	Input-dependent	Saturate and flag, or scale conservatively

Table 69. Other transforms used in IP

Transform	Property	Application
DCT-II	Energy compaction for correlated signals	JPEG, H.264/HEVC (integer approximations)
Integer DCT	Exactly invertible, multiplier-free	Video codecs — avoids encoder/decoder drift
Hadamard	± 1 only	Secondary transform, spreading

Transform	Property	Application
Number-theoretic (NTT)	Exact modular arithmetic	Post-quantum crypto (ML-KEM/ML-DSA) , homomorphic encryption
Wavelet (DWT)	Time–frequency localisation	JPEG2000, denoising

Graduate · **The integer DCT in video codecs is a beautiful example of specification-driven design.** An exact DCT requires irrational coefficients; any two implementations would round differently, and since the decoder's output feeds back into the encoder's prediction loop, the tiny differences would accumulate into visible *drift*. H.264 solved this by **defining the transform to be a specific integer matrix**, so every conforming implementation produces bit-identical results. The lesson generalises directly to IP work: **when an algorithm sits in a feedback loop that spans two independently implemented systems, the specification must be bit-exact, not mathematical.** The same reasoning applies to a DFE's tap update, to an adaptive equaliser shared between link partners, and to any distributed control loop.

F1. The Wireline Link, End to End

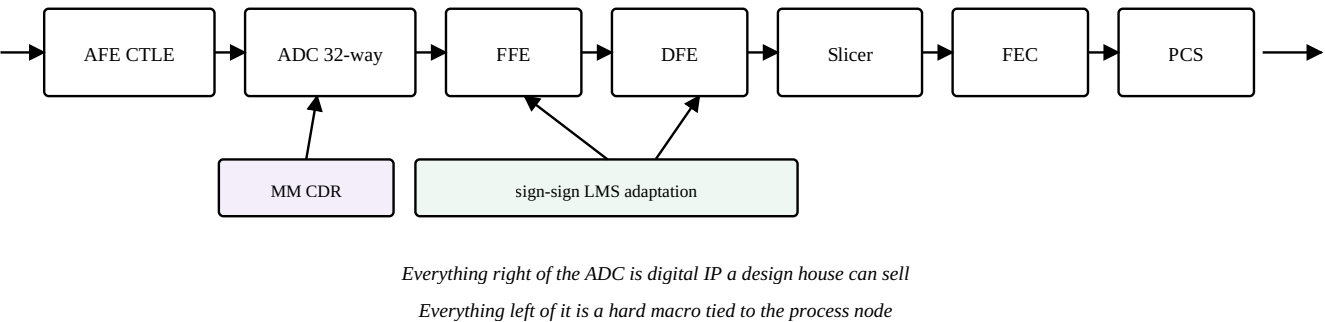


Figure 10. A modern ADC-DSP receiver. The PMA/PCS boundary is also the boundary of what can be sold as soft IP.

F1.1 Why the architecture looks like this

The structure above is not arbitrary; each block exists because a specific physical limit made the previous approach fail. Following that chain is the fastest way to understand the field.

Table 70. The forcing chain

Limit encountered	Response	New problem created
Channel loss rises as $\sqrt{f}$ (skin effect)	Equalise	Analogue boost amplifies noise as well as signal
CTLE alone cannot open the eye at 30 dB loss	Add DFE (subtracts decided symbols, no noise gain)	DFE feedback must close within 1 UI
1 UI = 17.86 ps at 56 GBd; latch + summer + wire + setup > 23 ps	Speculate (unroll) and parallelise (sub-rate)	Area grows; still not enough alone
Baud rate cannot be raised further — the channel will not pass it	PAM4 : two bits per symbol at the same baud rate	9.54 dB SNR penalty
PAM4 raw BER is $\sim 10^{-4}$, far from 10^{-15}	Forward error correction (RS-FEC KP4)	Latency, and FEC becomes mandatory in the standard
Analogue equalisation cannot be adapted precisely enough	Sample first with a fast ADC, equalise digitally	Time-interleaved ADC mismatch; huge digital block
200 Gb/s per lane: even KP4 is not enough	Concatenated FEC (outer RS-KP4 + inner Hamming)	More latency; more complex decoder

Graduate · This table is the single most useful thing to memorise about high-speed links, because it lets you derive the architecture rather than recall it, and it identifies where a newcomer can contribute. Notice that the last four rows are all *digital* responses to *analogue* limits. That migration — from analogue equalisation to digital — is why a digital IP house can participate in SerDes at all, and it is still in progress.

F1.2 Timing budget arithmetic

Table 71. DFE first-tap loop budget at 56 GBd (1 UI = 17.86 ps)

Element	Delay	Comment
Slicer (latch) clock-to-Q	12 ps	Regeneration plus output drive
Summer	8 ps	Analogue summing node settling
Wiring	6 ps	Layout-dependent; often underestimated
Setup at next slicer	5 ps	—
Total loop	31.0 ps	—
Available	17.86 ps	1 UI
Margin	-13.14 ps	Infeasible
Latch + wire + setup alone	23 ps	Still exceeds 1 UI

The last row is decisive and is often missed. Unrolling removes the *summer* from the loop by pre-computing both hypotheses, but the latch, the wiring and the setup time remain — and they already exceed one UI. **Therefore no amount of speculation alone reaches 56 GBd; sub-rate parallelism is mandatory.** Stating this as a measured budget rather than an opinion is what turns an architecture argument into an engineering conclusion, and it is exactly the kind of analysis a modelling engineer is expected to produce before any RTL exists.

F1.3 Equaliser taxonomy

Table 72. Equalisers compared

Type	Location	Removes	Noise effect	Constraint
TX FFE (de-emphas-is)	Transmitter	Pre- and post-cursor	None (pre-distortion)	Reduces transmitted amplitude
CTLE	RX ana- logue	Broad high-frequency loss	Amplifies noise	Limited number of poles/zeros
RX FFE	RX digital	Pre- and post-cursor	Amplifies noise	Needs ADC; tap count $\times$ rate
DFE	RX digital	Post-cursor only	None (uses de- cisions)	1 UI loop; error propagation
MLSE / Viterbi	RX digital	Optimal sequence detec- tion	None	Complexity grows as M^L
Nonlinear (Volterra, NN)	RX digital	Nonlinear channel ef- fects	—	Training, complexity; an active research area

Graduate · Error propagation is the price of the DFE's noise-free subtraction. A wrong decision is fed back and corrupts the next few symbols, so errors arrive in bursts rather than independently. Two consequences follow, and both are design decisions rather than analysis details. First, the raw BER is worse than an independent-error model predicts, so a link budget built on Q-function arithmetic alone is optimistic. Second — and more usefully — **bursty errors are exactly what a symbol-oriented code like Reed–Solomon handles well**, because several wrong bits inside one symbol still count as one symbol error. The choice of RS over a binary code at the FEC layer is therefore partly a consequence of the DFE at the equaliser layer. **Layers that look independent in a block diagram are coupled through their error statistics.**

F1.4 Clock and data recovery

Table 73. CDR phase detectors

Detector	Samples needed	Characteristic	Used with
Alexander (bang-bang)	Data + edge (2×)	Binary output; nonlinear	Classic NRZ links
Hogge	Data + edge	Linear output	Lower jitter, more complex
Gardner	2× oversampling	Modulation-independent	Wireless, some wireline
Mueller–Müller	Baud-rate only	Uses ISI structure; needs equalised signal	ADC-DSP receivers — no 2× sampling needed
Oversampling / majority	3–8×	Simple, robust, low rate	Low-speed links, FPGA

Graduate · Why Mueller–Müller won in ADC-based receivers. At 56 GBd, sampling at 2× the baud rate means 112 GS/s — roughly doubling the ADC's power and area, which are already the dominant cost. MM extracts a timing error from baud-rate samples alone by exploiting the asymmetry of the equalised pulse response: it drives the sampling phase to the point where the first pre-cursor and first post-cursor are equal. The subtlety is that **MM's lock point depends on the equaliser**, so CDR and equaliser adaptation interact; a modelling engineer who simulates them independently will not see the interaction, and joint convergence failures are a classic late-stage surprise. **Model the loops together or state explicitly that you have not.**

F2. Error Control Coding

F2.1 The coding landscape

Table 74. Codes by structure and application

Family	Alphabet	Decoding	Strength	Typical use
Parity / SECDED	binary	syndrome	1-bit correct, 2-bit detect	Memory ECC , register files
Hamming / extended	binary	syndrome	—	Inner code in concatenation
BCH	binary	BM + Chien	t bit errors	Flash, optical
Reed–Solomon	$GF(2^m)$	syndrome $\rightarrow$ BM $\rightarrow$ Chien $\rightarrow$ Forney	t symbol errors — burst tolerant	Ethernet KP4 , storage, DVB
Convolutional	binary	Viterbi (ML)	Random errors	Legacy wireless, satellite
Turbo	binary	iterative BCJR	Near capacity	3G/4G
LDPC	binary	belief propagation (min-sum)	Near capacity	5G NR data , Wi-Fi, 802.3ca, NAND
Polar	binary	SC / SC-List	Provably capacity-achieving	5G control
Fountain (LT, Raptor)	binary	BP	Rateless	Broadcast, storage
Concatenated	mixed	inner then outer	Burst + random	802.3dj 224G , deep space

Graduate · **Choosing a code is choosing an error model.** A binary code assumes errors are independent bit flips; a symbol code assumes they cluster. Getting this wrong is expensive in both directions — a strong binary code performs poorly on bursts, and an RS code wastes redundancy on isolated bit errors. The correct procedure is to **measure the error statistics of the actual slicer output**, including DFE error propagation and any burst noise, and choose accordingly. In concatenated schemes the division of labour is explicit: the inner code cleans up random errors and the outer code mops up the bursts that the inner decoder's own failures produce. **The inner decoder's failure mode is the outer decoder's input distribution** — a subtlety that must be in the system model.

F2.2 Reed–Solomon in detail

Table 75. RS decoder pipeline

Stage	Computation	Parallelism	Hardware character
Syndrome	$S_i = r(\alpha^{tcr+i}), i=0\dots 2t-1$	Fully parallel ($2t$ independent evaluations)	$2t$ constant-multiplier accumulators
Key equation (BM)	Find $\sigma(x)$	Sequential — $2t$ dependent iterations	This is the critical path
Chien search	Roots of $\sigma(x)$	Parallel over positions	n/P cycles
Forney	$e_j = \omega/\sigma'$ evaluated at roots	Parallel	Needs GF inversion (ROM or Itoh–Tsujii)
Correction	XOR into the buffer	Parallel	Requires the codeword buffered

Graduate · The Berlekamp–Massey iteration is the same kind of obstacle as the DFE loop and the OOO wakeup loop. Each iteration depends on the previous discrepancy, so it cannot be pipelined away. The standard remedies mirror those in the other two cases: *inversionless* BM removes the GF inversion from the loop (replacing it with an extra multiplication), and *re-formulated* BM restructures the recurrence so that fewer operations sit inside it. A design house that recognises this pattern can move an engineer from a SerDes project to an FEC project and expect them to be productive quickly, because the underlying problem is identical.

Table 76. RS decoder behaviour that must appear in the datasheet

Condition	Behaviour	Why it must be documented
$\leq t$ symbol errors	Always corrected	The guarantee being sold
$> t$ errors	Detected or mis-corrected	Not all are detected. Claiming otherwise is false
Mis-correction probability	Roughly $n^t / (t! 2^{m(n-k-t)})$	Customers at 10^{-15} FLR need this number
Erasure decoding	Corrects up to $n-k$ erasures	If the receiver can flag unreliable symbols, capacity doubles
Latency	Fixed, = pipeline depth	Real-time systems need the number

“All errors beyond t are detected” is a false statement that appears in datasheets. A received word with more than t errors may land within distance t of a *different* valid codeword, in which case the decoder confidently produces the wrong answer. A measured illustration on a small code, RS(15,9) with $t = 3$: injecting exactly four symbol errors 120 times gave **117 detected failures and 3 mis-corrections** — never a correct recovery, but not always a detection either. An honest specification states the mis-correction probability; a verification plan must distinguish the three outcomes (corrected / detected / mis-corrected) rather than counting pass and fail.

F2.3 LDPC

Table 77. LDPC decoding choices

Choice	Options	Effect
Message form	Probability vs log-likelihood ratio	LLR turns products into sums — essential in hardware
Check-node update	Sum-product (tanh) vs min-sum	Min-sum loses ~ 0.5 dB, removes transcendental functions
Correction	Normalised or offset min-sum	Recovers most of the loss for one constant
Schedule	Flooding vs layered	Layered halves the iteration count; creates pipeline hazards
Parallelism	Fully parallel / row / block	Quasi-cyclic structure enables block parallelism
Early termination	Syndrome check per iteration	Large average power saving ; worst-case latency unchanged

LDPC's fixed-point rule is the opposite of the CIC's. An LLR's sign carries the hard decision. If an LLR wraps in two's complement, a strong belief in “0” becomes a strong belief in “1”, and the decoder does not merely lose precision — it is actively misled, and iteration amplifies the damage. Measured on a 5G NR base-graph decoder, reducing the message width by three bits gave BLER 0.250 with saturation and **1.000 — complete failure — with wrap-around**. Meanwhile the CIC filter of Chapter E1 *requires* wrap-around to function. **There is no global “correct” overflow policy; it is a per-block property that must be stated in the specification and enforced by tests.**

Graduate · **Layered scheduling and pipeline hazards.** In a layered decoder, layer j may need a variable node that layer i is still updating, forcing a stall whose length depends on the pipeline depth D and on the layer ordering. It is tempting to optimise the ordering against a closed-form stall count, but that formula is only an **upper bound**: it assumes every potential conflict costs a full stall. A measured comparison on 5G NR base graph 1 at $D = 4$ gave the formula 108 stall cycles while a cycle-accurate model and the RTL both gave **80** — the difference between a 70.1% and a 62.0% efficiency loss. **Optimising**

a schedule against the wrong objective function finds the wrong schedule. Build the cycle-accurate model first; it is a day of work and it changes the answer.

G1. Electromagnetics for the Digital Engineer

A digital IP engineer meets electromagnetics at three boundaries: the package and board that carry the block's signals, the channel a SerDes must equalise, and the antenna of a radio or radar system whose baseband processing is the block itself. This chapter covers what is needed at those boundaries.

G1.1 From Maxwell to circuits

Table 78. Regimes and the model that applies

Regime	Condition	Model	Consequence
Lumped	size $\ll \lambda$	KCL/KVL, R L C	Voltage is well defined everywhere
Quasi-static	size $\leq \lambda/10$	Lumped + parasitics	Inductance begins to matter
Distributed	size $\geq \lambda/10$	Transmission line	Position matters; reflections occur
Full wave	structure comparable to λ	Maxwell solvers	Radiation, resonance, modes

Graduate · **The threshold keeps moving toward the designer.** The criterion is not clock frequency but edge rate: significant spectral content extends to roughly $0.35/t_r$. A 50 ps edge has content to 7 GHz regardless of whether the clock is 100 MHz, and at 7 GHz a 2 cm trace is a transmission line. **Slow clocks do not protect you from fast edges.** This is why deliberately slowing output slew rates is a legitimate design technique — and why an IP's output driver strength is a specification item with EMI consequences, not just a timing one.

G1.2 Transmission lines

$$Z_0 = \sqrt{(R + j\omega L)/(G + j\omega C)} \rightarrow \sqrt{L/C} \text{ (lossless)}$$

Table 79. Line types

Type	Z_0 range	Where
Microstrip	50–100 Ω	Outer PCB layer; radiates more
Stripline	50–100 Ω	Inner layer; better shielded, slower (higher ϵ_{eff})
Coplanar waveguide	tunable	On-chip RF; ground on the same layer
Differential pair	85–100 Ω differential	The standard for high-speed serial
On-chip global wire	—	Usually RC-dominated, not a true TL

Table 80. Termination strategies

Scheme	Placement	Power	Note
Series (source)	At driver	Low	Doubles the driver's output swing requirement
Parallel	At receiver	High (static)	Best match
Thevenin	At receiver	High	Sets a DC bias
AC-coupled	In line	Low	Requires DC-balanced coding (8b/10b, 64b/66b)
On-die termination (ODT)	Inside the chip	Moderate	Calibrated against an external precision resistor

Graduate · **AC coupling and line coding are linked requirements.** A series capacitor blocks DC, so a long run of identical symbols causes the signal to droop toward the decision threshold — baseline wander. The line code must therefore bound the run length and keep the running disparity small. 8b/10b does this with an explicit disparity rule at 25% overhead; 64b/66b uses a scrambler and accepts a small probability of a long run, at 3% overhead. **The coding choice is thus determined by an analogue component.** An engineer who knows only the coding layer cannot explain why 64b/66b needs a scrambler with a particular polynomial, nor why certain data patterns are used as stress tests.

G1.3 Antennas and propagation

$$P_r = P_t G_t G_r (\lambda/4\pi d)^2 \quad (\text{Friis})$$

Table 81. Antenna and propagation quantities

Quantity	Definition	Design relevance
Gain G	Directivity × efficiency	Trades coverage for range
Effective aperture	$A_e = G\lambda^2/4\pi$	Higher frequency → smaller aperture for the same gain
Beamwidth	$\approx 70^\circ/\sqrt{D}$	Array size sets angular resolution
Path loss exponent	2 (free space) to 4+ (urban)	Link budget
Multipath / delay spread	—	Sets the equaliser or OFDM CP length
Doppler shift	$f_d = 2v/\lambda$	Coherence time; tracking loop bandwidth
Fading	Rayleigh / Rician	Diversity, MIMO

Graduate · **Delay spread converts directly into a hardware parameter.** If the channel's impulse response lasts $\tau_{\max}$, a single-carrier receiver needs an equaliser spanning $\tau_{\max}/T_s$ symbols, and an OFDM receiver needs a cyclic prefix at least that long. The first costs taps (area, adaptation time); the second costs throughput (the CP is overhead). **This is the quantitative form of the choice between single-carrier and OFDM**, and it explains why wireline links — where delay spread is long but the channel is static — use equalisers, while wireless links with time-varying multipath use OFDM. The same physics, opposite conclusions, because the *rate of change* differs.

G1.4 Noise figure and receiver chains

$$F_{\text{total}} = F_1 + (F_2 - 1)/G_1 + (F_3 - 1)/(G_1 G_2) + \dots$$

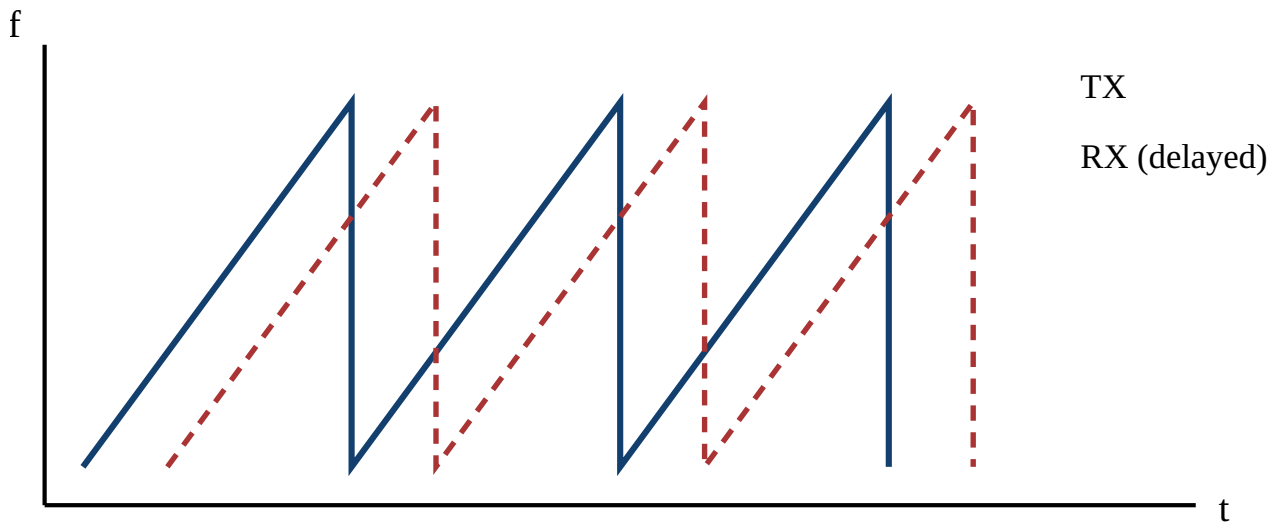
Graduate · **Friis's formula justifies the entire architecture of a receiver front end.** Because later stages are divided by the cumulative gain ahead of them, the first amplifier dominates the noise figure — hence the low-noise amplifier, placed as close to the antenna as possible, often before any filter whose insertion loss would add directly to F_1 . The counterweight is linearity: gain early in the chain means large signals later, so the cascaded third-order intercept point degrades as $1/\text{IIP3}_{\text{total}} = 1/\text{IIP3}_1 + G_1/\text{IIP3}_2 + \dots$ **Noise pushes gain forward; linearity pushes it backward.** Every receiver's gain distribution is a solution to that tension, and the digital back end inherits the result as its required dynamic range — which is to say, as its ADC bit count.

G2. Radar: Signal Processing as an IP Block Family

Automotive and industrial radar is one of the largest current markets for signal-processing IP, and its chain is a clean illustration of how physical parameters become hardware parameters.

$$P_r = P_t G^2 \lambda^2 \sigma / ((4\pi)^3 R^4) \quad (\text{radar equation})$$

Undergraduate review · The fourth-power range dependence is the defining fact of radar: doubling the detection range requires sixteen times the received power. Since transmit power and antenna gain are bounded, the remaining lever is **processing gain** — coherent integration — which is entirely a digital matter and therefore entirely IP.



$$\text{Beat frequency } f_b = 2 \cdot R \cdot S / c \quad (S = \text{chirp slope})$$

Range from frequency; velocity from phase change across chirps

Figure 11. FMCW radar. Range appears as a beat frequency; velocity appears as phase progression across successive chirps.

G2.1 FMCW parameter derivation

Table 82. From requirement to hardware parameter

Requirement	Relation	Hardware consequence
Range resolution ΔR	$\Delta R = c/2B$	Sweep bandwidth — 4 GHz gives 3.75 cm
Maximum range $R_{\max}$	$f_{b,\max} = 2R_{\max}S/c$	ADC sample rate and IF bandwidth
Velocity resolution Δv	$\Delta v = \lambda/(2N_c T_c)$	Number of chirps per frame → frame memory
Maximum velocity	$v_{\max} = \lambda/(4T_c)$	Chirp repetition period
Angular resolution	$\approx \lambda/(N_a d \cos \theta)$	Number of antennas → number of RX chains
Processing gain	$10\log(N_s N_c)$	FFT sizes → compute and memory

Graduate · **The range–Doppler map is a two-dimensional FFT** and its dimensions come straight from the table: a fast-time FFT across samples within one chirp yields range bins; a slow-time FFT across chirps at each range bin yields Doppler bins. A typical automotive frame — 256 samples $\times$ 128 chirps $\times$ 8 virtual antennas — requires a corner-turn between the two FFTs, and **that transpose, not the FFTs, dominates the memory bandwidth**. Designing the memory organisation (ping-pong banks, tiled transpose, or on-the-fly second-stage accumulation) is the real architectural work. This is a recurring theme: in data-intensive blocks the arithmetic is rarely the bottleneck.

G2.2 Detection

Table 83. CFAR detectors

Type	Noise estimate	Strength	Weakness
CA-CFAR	Mean of surrounding cells	Simple, efficient in homogeneous noise	Masks targets near clutter edges
GO-CFAR	Greater of the two halves	Handles clutter edges	Loses closely spaced targets
SO-CFAR	Smaller of the two halves	Resolves close targets	High false alarm at edges
OS-CFAR	k -th order statistic	Robust to interfering targets	Needs a sorter — more hardware
Adaptive / ML	learned	Best in complex scenes	Verification difficulty

Graduate · **CFAR sets a threshold, not a decision**. The multiplier α is chosen so that the probability of false alarm is fixed regardless of the noise level — that is what "constant false alarm rate" means. For CA-CFAR with N reference cells, $\alpha = N(P_{FA}^{-1/N} - 1)$. Two hardware consequences: the block needs a divider or a reciprocal table, and it needs the N reference cells buffered, which for a 2-D CFAR means a sliding window over a two-dimensional array. **OS-CFAR's sorter is the most hardware-hungry element in a typical radar back end**, and choosing between a full sorting network and an approximate selection is a genuine PPA decision that a design house can differentiate on.

G2.3 Angle estimation and MIMO

Table 84. Angle-of-arrival methods

Method	Resolution	Computation	Robustness
FFT beamforming	Rayleigh limit	One FFT	Very robust
Capon / MVDR	Super-resolution	Matrix inverse per snapshot	Needs good covariance estimate
MUSIC	Super-resolution	Eigendecomposition	Needs the source count
ESPRIT	Super-resolution	SVD + rotation	Needs array structure
Compressive sensing	High	Iterative solver	Sensitive to model mismatch

Graduate · **This is where Part A's matrix decompositions become products**. MUSIC needs the eigendecomposition of an $N \times N$ covariance matrix per snapshot; Capon needs its inverse. For an 8–16 element array that is a small, dense, Hermitian, **positive-definite** problem — exactly the case where Cholesky applies and streams. A design house that owns a parameterised streaming Cholesky or Jacobi eigenvalue block can sell it into radar, into MIMO communications, and into adaptive beamforming, because all three reduce to the same kernel. **Recognising that one kernel serves three markets is how a small IP portfolio becomes viable**.

Graduate · **MIMO radar's virtual array** deserves a precise statement because it is the main reason automotive radar improved so quickly. With N_t transmitters and N_r receivers, transmitting orthogonal waveforms (time-, frequency-, or code-division) and separating them at the receiver yields $N_t \times N_r$ distinct transmit–receive paths, which behave like a physical array of

that many elements. Three transmitters and four receivers give a twelve-element virtual array — a threefold improvement in angular resolution at the cost of **digital** separation logic rather than antennas and RF chains. **Once again the solution to a physical limit is a digital block.**

H1. Verification: Theory and Organisation

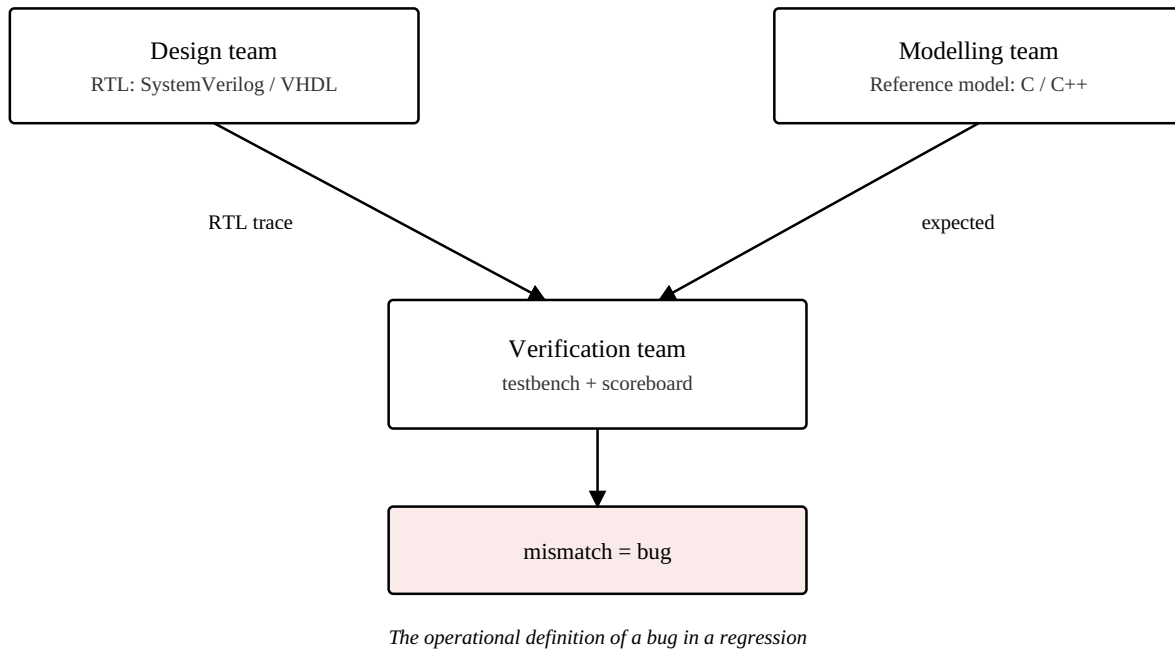


Figure 12. The three-team structure. It exists to make “correct” an operational, testable notion.

Verification consumes more effort than design in every serious IP project — commonly cited at 60–70% of the schedule. Understanding why it is organised as it is prevents a great deal of wasted work.

H1.1 Why two independent implementations

Undergraduate review · A specification written in prose is ambiguous; a single implementation is self-consistent by construction and therefore cannot reveal its own misreadings. Two implementations built independently from the same specification will agree only where both read it the same way. **Disagreement localises ambiguity.** This is the same principle as dissimilar redundancy in avionics, applied to development rather than to operation.

Graduate · **The argument fails if the two implementations are not independent.** If the modeller and the designer sit together, share an interpretation, and one consults the other's code when unsure, the common-mode error rate rises and the regression turns green on a shared misunderstanding. Organisations counter this in three ways: physical or organisational separation; deriving one implementation from a **formal, executable specification** (the RISC-V Sail model is the canonical example, generating both prose and a C emulator from one source); and an **external oracle** — standard test vectors from NIST, IEEE or a conformance suite. **The third is the strongest and the cheapest, and it is the reason standards-based blocks are the best first product for a small IP house.**

H1.2 The verification stack

Table 85. Verification levels

Level	Checks	Catches	Cannot catch
Lint	Coding rules	Latches, width mismatches, incomplete sensitivity	Anything functional
CDC / RDC static	Crossing structure	Missing synchronisers, reconvergence	Functional protocol errors
Unit simulation	Block behaviour vs model	Most functional bugs	Integration, timing
Formal property	Exhaustive over state space	Deadlock, unreachable states, protocol violations	Deep datapath equivalence (state explosion)
Formal equivalence (LEC)	RTL vs netlist	Synthesis and ECO errors	Specification errors
Integration / SoC sim	Blocks together	Interface mismatches, arbitration	Long-running system behaviour
Gate-level with timing	Netlist + SDF	X-propagation, timing-dependent bugs	Slow; limited coverage
Emulation / FPGA prototype	Real software on real RTL	Software-visible bugs, long sequences	Analogue, timing corners
Post-silicon	Actual device	Everything else	— but fixes are expensive

X-propagation deserves a specific warning because it is asymmetric between RTL and gates. In RTL simulation, an if (x) with an unknown condition may take a branch and produce a definite value, hiding the uncertainty — this is *X-optimism*. In gate-level simulation the same unknown spreads — *X-pessimism*. A design that relies on registers being initialised by data flow rather than by reset can therefore pass RTL simulation and fail in gates, or vice versa. The remedies are X-propagation-aware simulation modes and, better, **explicit initialisation of any register whose value is read before it is written**. For an IP vendor this is also a deliverable question: if your block requires *n* cycles of reset before its outputs are valid, say so.

H1.3 Coverage and closure

Table 86. Coverage metrics

Metric	Measures	Automat-ic?	Weakness
Line / block	Statements executed	Yes	Says nothing about values
Branch / condition	Decisions taken	Yes	—
Expression (MC/DC)	Each condition independently affects the outcome	Yes	Required for DO-254 / ISO 26262
Toggle	Every bit changed both ways	Yes	Weak functional signal
FSM state / transition	States and arcs visited	Yes	—
Functional (coverage groups)	Scenarios the engineer defined	No	Blind to scenarios nobody thought of
Assertion	Properties actually exercised	Partly	Vacuous passes must be excluded
Mutation / fault injection	Whether the testbench would notice a bug	Yes	The only metric that tests the testbench

Graduate · Mutation coverage is the metric that answers the question the others avoid: **would this testbench detect a bug if there were one?** The procedure is to inject a small, semantically meaningful change — invert a comparison, drop a term, stick a bit — and confirm the regression turns red. A mutant that survives identifies a check that does not exist or a stimulus that never reaches the code. This is cheap to automate and routinely reveals that a large fraction of a mature suite is

inert. **The same technique applies to the reference model itself**, which is otherwise unverified: break the model deliberately and confirm the model's own self-checks fail. A verified reference model is a differentiator that very few IP vendors can claim.

H1.4 The scoreboard problem

Table 87. Comparison strategies

Situation	Strategy	Pitfall
In-order, fixed latency	FIFO compare	—
In-order, variable latency	FIFO + timeout	Timeout too tight → false failures
Out-of-order, tagged	Per-tag queues	A single queue produces both false failures and false passes
Multiple legal orders	Compare against a set of allowed outcomes	Requires the model to enumerate them
Don't-care fields	Mask before comparing	Masking too much hides real bugs
Weak memory model	Litmus tests , not trace comparison	Trace comparison is simply invalid here

Graduate · The AXI case is worth stating concretely because it is so common. AXI guarantees ordering only *within* an ID; transactions with different IDs may complete in any order. A scoreboard with one queue will report a failure when the DUT legitimately reorders across IDs, and — more dangerously — may accept a genuine intra-ID reordering as long as the totals match. **The correct structure is one queue per ID**, and knowing this is a direct consequence of having read the protocol specification rather than inferring behaviour from waveforms. It is a frequent and productive point of contact between the modelling and verification teams.

H1.5 Regression engineering

Table 88. Regression discipline

Practice	Reason
Record the seed with every run	Without it a failure is not reproducible
Classify failures automatically	Triage time dominates otherwise
Track failure <i>signatures</i> , not counts	Distinguishes new from known
Never close an intermittent failure as 'flaky'	Intermittent failures are usually real races
Keep a golden log of the last known-good	Enables bisection
Measure regression runtime	A suite nobody can run nightly is not a suite
Separate infrastructure failures	Licence and disk errors must not be counted as bugs

The prohibition on dismissing intermittent failures is the most frequently violated rule in this book. An intermittent failure means the outcome depends on something the test does not control — a race, an uninitialised value, a timing-dependent path. All three exist in the silicon too. **Closing such a ticket does not remove the bug; it removes the evidence.** The correct disposition is to keep it open with the failing seed recorded, even if it cannot be root-caused immediately.

H2. The Implementation Flow

Table 89. From RTL to GDSII

Step	Input → output	Key decisions	Common failure
Elaboration	RTL → generic netlist	Parameter resolution	Unintended latch inference
Logic synthesis	generic → standard cells	Constraints, effort, V_{TH} mix	Over-constrained clock → area and power explosion
DFT insertion	+ scan chains	Chain count and length	Scan shift timing
Floorplan	—	Block placement, pin assignment, power grid	The decision with the largest downstream impact
Placement	cells → locations	Congestion vs timing	Routing congestion
CTS	+ clock tree	Skew target, useful skew	Clock power, OCV
Routing	—	Layer assignment, shielding	DRC violations, crosstalk
Sign-off STA	—	Corners, derating, SI	Missing a corner
Physical verification	DRC, LVS, antenna	—	—
Extraction + final timing	RC → delays	—	—

Graduate · Floorplanning is where an IP block's shape becomes a commercial issue. A block delivered as RTL will be placed by the customer, whose aspect ratio, pin locations and macro placement may be nothing like the vendor's. If the block's internal timing depends on a particular arrangement — a wide datapath that must stay compact, a memory that must sit next to its controller — then the RTL alone is not a sufficient deliverable. Serious IP therefore ships a **floorplan guideline**, sometimes a hardened macro, and always a statement of the assumed aspect ratio and utilisation. **A block that met timing in the vendor's flow and misses in the customer's is the single most damaging support event a small IP house can have**, because it is expensive to debug remotely and it destroys confidence.

Table 90. Sign-off corners

Axis	Values	Why
Process	SS, TT, FF, SF, FS	Global and local variation
Voltage	min, nominal, max	IR drop and regulation tolerance
Temperature	−40, 25, 125 °C	Both extremes — temperature inversion makes cold the worst case at low V
RC extraction	Cmin, Cmax, RCmin, RCmax	Metal thickness variation
Mode	functional, scan, BIST, low power	Different constraints apply
Ageing	fresh, end-of-life	NBTI/PBTI drift

Graduate · The corner count multiplies. Five process points × three voltages × three temperatures × four RC × four modes is 720 combinations; full sign-off on all of them is not affordable. The industry response is **statistical or parametric OCV** (POCV), which models variation as a distribution rather than enumerating corners, plus careful selection of dominant corners. For an IP vendor the practical obligation is to state **which corners were signed off** in the PPA report. "500 MHz" without a corner is not a specification, and a customer who assumes typical-typical will be disappointed at slow-slow.

Table 91. High-level synthesis in the flow

Aspect	Reality
Input	C/C++/SystemC with vendor pragmas — not portable across tools
Strength	Design-space exploration: one source, many area/throughput points
Weakness	Generated RTL is hard to read and debug; timing closure less predictable
Where it wins	Datapath-dominated blocks with regular loops (filters, transforms, crypto)
Where it loses	Control-dominated logic, precise cycle-level interfaces
Verification role	The same C source doubles as the reference model , because a general compiler ignores the pragmas — convenient, but it shares every algorithmic assumption with the design

The last row is a genuine methodological trap. Using the HLS source as the golden model gives *zero* independence: the two implementations are literally the same text. It verifies the HLS tool, not the design. **If the HLS C is the design, the reference model must be written separately** — ideally in a different language, by a different person, from the specification.

H3. Hardware Security

Table 92. Threat model, mechanism, countermeasure

Threat	Mechanism	Countermeasure	Verifiable by simulation?
Timing side channel	Execution time depends on secret	Constant-time algorithms; no secret-dependent branches or table indices	Partly — can check control flow
Power / EM (SPA, DPA, CPA)	Switching activity correlates with data	Masking (Boolean or arithmetic), threshold implementations, DOM	No — requires TVLA on silicon or power simulation
Fault injection (laser, glitch, EM)	Flip a bit to skip a round or corrupt a check	Redundant/complementary FSMs, computation duplication, output checks	Partly — by explicit fault simulation
Probing / FIB	Physically read an internal wire	Shields, sensors, routing obfuscation	No
Scan / debug access	Read state through test infrastructure	Lock scan after test; life-cycle state control	Yes
Rowhammer memory disturb	Repeated activation flips neighbours	Refresh management, tracking	Partly
Supply chain	Trojan insertion, counterfeiting	Split manufacturing, logic locking, PUF identity	No

Graduate · The fourth column is the key organisational insight for a modelling engineer. A functional regression, no matter how complete, is blind to four of the seven rows. Saying “the AES block is verified” is therefore ambiguous, and in a security review the ambiguity will be challenged. The mature formulation is: *functional equivalence to FIPS-197 is verified by NIST vectors and constrained-random comparison against an independent model; side-channel resistance is assessed separately by TVLA on the test chip; fault resistance is assessed by gate-level fault injection. Three claims, three methods, three owners.*

Table 93. Masking: the principle and its cost

Concept	Statement	Consequence
Boolean masking	x represented as $(x \oplus m, m)$	Each share is independent of $x \rightarrow$ first-order leakage removed
Linear operations	Apply to each share separately	Nearly free
Nonlinear operations	Shares must interact	The expensive part — this is why the AES S-box is rebuilt in arithmetic
Glitches	Transient values can momentarily depend on x	Registers must separate share-combining stages
DOM	Domain separation + fresh randomness + registers	Provably glitch-resistant; needs a continuous randomness supply
Order d masking	$d+1$ shares resist d -th order analysis	Cost grows roughly as $(d+1)^2$

Graduate · Randomness becomes a system-level resource. A DOM-protected S-box consumes fresh random bits on every evaluation. An AES core performing sixteen S-box lookups per round for fourteen rounds therefore needs a sustained random-bit rate that a slow true-RNG cannot supply, so the design includes a fast PRNG reseeded from the TRNG. That PRNG's quality, its reseed policy, and its behaviour when starved are now **functional** requirements of the cipher block, and they are

verifiable. **A security countermeasure has turned into an ordinary functional specification with a throughput requirement** — which is precisely the kind of boundary a modelling engineer should be able to identify and take ownership of.

Table 94. Standards a security IP is measured against

Standard	Scope
FIPS 140-3	Cryptographic module validation
Common Criteria (ISO 15408)	Evaluation assurance levels
ISO/IEC 17825	Side-channel test methods
NIST CAVP	Algorithm test vectors — the practical oracle
SP 800-90A/B/C	RNG construction, entropy, and validation
FIPS 203/204/205	Post-quantum: ML-KEM, ML-DSA, SLH-DSA
OCP Caliptra	Datacentre root of trust specification
ISO 26262 / IEC 61508	Functional safety (distinct from security)

I1. Information Theory

I1.1 Entropy and its operational meanings

$$H(X) = -\sum_x p(x) \log_2 p(x) \quad I(X;Y) = H(X) - H(X|Y)$$

Table 95. Information-theoretic quantities and where they bind hardware

Quantity	Meaning	Operational limit it sets
Entropy $H(X)$	Average information per symbol	Lossless compression cannot go below it — bounds a co-dec's ratio
Min-entropy H_∞	$-\log_2 \max_x p(x)$	The measure NIST SP 800-90B requires for an entropy source; always $\leq H$
Conditional entropy	Remaining uncertainty given side information	Leakage analysis in side-channel evaluation
Mutual information	Information one variable carries about another	Side-channel leakage metric ; also the quantity maximised by capacity
Channel capacity C	$\max I(X;Y)$	No code can exceed it; tells you how far your FEC is from optimal
Rate-distortion $R(D)$	Minimum rate for distortion D	Lossy codec bound — the theoretical floor of a video encoder
Kolmogorov complexity	Shortest program producing the string	Uncomputable; conceptual only

Graduate · **Min-entropy is the one an IP engineer must get right, and it is not Shannon entropy.** A biased source that outputs 0 with probability 0.9 has Shannon entropy 0.47 bits but min-entropy only $-\log_2(0.9) = 0.152$ bits. Certification uses min-entropy because security must hold against the *most likely* guess, not the average case. A TRNG that claims "0.5 bits of entropy per sample" on Shannon grounds and is conditioned on that basis will fail validation and, worse, will produce keys with less unpredictability than advertised. **The conditioning ratio — how many raw bits are compressed into one output bit — is derived from min-entropy, and it is a hardware parameter.**

I1.2 Capacity results that matter in practice

Table 96. Capacity expressions

Channel	Capacity	Design meaning
AWGN	$B \log_2(1+\text{SNR})$	The reference against which every modem is measured
Binary symmetric (BSC)	$1 - H_b(p)$	Hard-decision decoding loses ~2 dB versus soft
Binary erasure (BEC)	$1 - \epsilon$	Erasure decoding doubles RS correction capability
MIMO ($N_t \times N_r$)	$\approx \min(N_t, N_r) \log(1+\text{SNR})$	Capacity scales with antennas, not power
Fading with CSI	$E[\log(1+ h ^2 \text{SNR})]$	Motivates adaptive modulation and coding
Band-limited with ISI	water-filling over the channel response	The theoretical justification for OFDM bit loading

Graduate · **The soft-versus-hard decision gap is the most actionable number here.** Passing hard decisions to a decoder discards the reliability information the slicer had, and costs roughly 2 dB on an AWGN channel. In a link budget 2 dB is enormous — it is the difference between needing an extra equaliser tap and not. This is why modern FEC interfaces carry

LLRs rather than bits, why the slicer in an ADC-based receiver reports a multi-bit soft value, and why the fixed-point width of that LLR is a system-level parameter rather than a local choice. **An interface that carries one bit where it could carry four has thrown away 2 dB before the decoder starts.**

I1.3 Source coding

Table 97. Compression methods in hardware

Method	Principle	Hardware character
Huffman	Prefix code by symbol probability	Table lookup; variable-length decode is serial
Canonical Huffman	Ordered code lengths	Compact tables, faster decode
Arithmetic / range	Interval subdivision	Better ratio; strictly serial — a throughput bottleneck
CABAC	Context-adaptive binary arithmetic	The critical path of an H.264/HEVC decoder
ANS / rANS	Asymmetric numeral systems	Arithmetic-coding ratio with table-driven speed; used in AV1, Zstd
LZ family	Dictionary matching	Match search is the expensive part
RLE	Run lengths	Trivial; still used for coefficient maps

Entropy coding is where video decoders stop parallelising. Transform, prediction and filtering all parallelise over blocks, but arithmetic decoding is inherently sequential: the next symbol's interval depends on the current state. This is why real decoders introduce architectural workarounds — slices and tiles that reset the context so they can be decoded independently, wavefront parallel processing, or the bypass modes that skip context modelling for near-uniform symbols. **The bitstream syntax was designed around a hardware constraint**, which is the clearest possible illustration that standards and architectures co-evolve.

I2. Algebraic Structures for Hardware

I2.1 Groups, rings, fields

Table 98. Structures and the hardware they license

Structure	Operations	Guarantee	Hardware consequence
Group	one associative op with inverses	Cancellation	Reversible transforms
Ring	+ and $\times$, no division	Closure under both	Two's-complement W-bit arithmetic is a ring — wrap-around is exact, so operation order does not matter
Field	+ $\times$ and division	Every non-zero element invertible	$GF(2^m)$ arithmetic for codes and crypto
Vector space over a field	—	Linearity	Linear codes: encoding and syndrome are matrix products
Polynomial ring $F[x]/(f)$	—	—	Cyclic codes, NTT, lattice cryptography

Graduate · The ring property of two's-complement arithmetic is the formal reason behind a practical rule. Because W -bit two's complement is the ring $\mathbb{Z}/2^W\mathbb{Z}$, addition and multiplication are exact modulo 2^W , associative and commutative. Intermediate overflow therefore cancels exactly, provided the final result is in range, and **the order of accumulation cannot change the answer**. Saturation destroys the ring structure: once a value is clamped, information is lost and order matters. This is why a CIC filter *requires* wrap-around and why an LDPC decoder *requires* saturation — the two blocks rely on opposite algebraic properties. **Stating this as algebra rather than as folklore makes it possible to decide the question for a new block instead of guessing.**

I2.2 Finite fields in detail

Table 99. $GF(2^m)$ implementation choices

Representation	Multiplication	Inversion	Best for
Polynomial basis	Shift-and-XOR, reduce	Extended Euclid or Itoh–Tsujii	General purpose; most codes
Normal basis	More complex	Squaring is a free cyclic shift	Inversion-heavy workloads (ECC)
Log/antilog tables	Add logs	Negate the log	Small fields ($m \leq 12$); the usual RS choice
Tower field	$GF(2^8) \rightarrow GF(2^4) \rightarrow GF(2^2)$	Recursive	Maskable — the AES DOM S-box
Composite field	—	—	Area reduction

Graduate · The choice of irreducible polynomial is a specification item, not an implementation detail. Two implementations of $GF(2^{10})$ using different primitive polynomials produce entirely different — though isomorphic — arithmetic, and codewords from one will not decode in the other. Standards therefore fix the polynomial: AES uses $x^8+x^4+x^3+x+1$, and IEEE 802.3 Clause 91 fixes the field for RS(544,514). **A reference model that hard-codes a plausible polynomial without checking the standard will interoperate with nothing.** The defensive practice is to make the polynomial a parameter, assert that it is primitive (verify that α generates all 2^m-1 non-zero elements), and record in the code whether the value has been confirmed against the specification text.

Table 100. Where finite fields appear

Application	Field	Operation that dominates
Reed–Solomon	$\text{GF}(2^8), \text{GF}(2^{10})$	Multiply, invert
BCH	$\text{GF}(2^m)$	Multiply
AES	$\text{GF}(2^8)$	Inverse (S-box), $\times 2$ and $\times 3$ (MixColumns)
AES-GCM	$\text{GF}(2^{128})$	Carry-less multiply
Elliptic curves	$\text{GF}(p)$ or $\text{GF}(2^m)$	Modular multiply (Montgomery)
ML-KEM / ML-DSA	$\mathbb{Z}_q[x]/(x^n+1)$	Number-theoretic transform
CRC	$\text{GF}(2)[x]$	Polynomial remainder (LFSR)
Erasure coding (RAID)	$\text{GF}(2^8)$	Matrix multiply

Graduate · Post-quantum cryptography brings a new arithmetic requirement into mainstream silicon. ML-KEM and ML-DSA operate in a polynomial ring modulo a small prime (3329 and 8380417 respectively), and their bottleneck is the number-theoretic transform — an FFT over that ring. The hardware looks like an FFT but with modular multipliers instead of complex ones, and modular reduction by a fixed small prime admits efficient tricks (Montgomery, Barrett, or prime-specific shift–add chains). **For a design house, an NTT block parameterised over modulus and length is a single kernel serving several standards**, in the same way that a Cholesky block serves radar, MIMO and beamforming. Finding these shared kernels is how a small portfolio covers a large market.

I3. Optimisation and Numerical Methods

Table 101. Optimisation problems that appear in IP design

Problem	Class	Method	Where
Filter coefficient design	Convex (minimax)	Remez, convex solvers	FIR design with quantised coefficients
Equaliser tap solution	Least squares	Normal equations, QR	MMSE equaliser initialisation
Bit allocation	Integer, convex relaxation	Water-filling then rounding	OFDM loading, quantisation budgeting
Scheduling / binding	NP-hard	ILP, list scheduling, force-directed	HLS internals
Retiming	Polynomial	Leiserson–Saxe	Timing optimisation
Placement	NP-hard	Analytic + partitioning	Physical design
Layer/schedule ordering	Combinatorial	Local search, simulated annealing	LDPC layered decoding order
Word-length assignment	Mixed integer	Heuristic + simulation	Fixed-point conversion

The objective function must be the true cost, not a convenient proxy. A measured example: optimising an LDPC layer ordering against a closed-form stall estimate produced a schedule that a cycle-accurate model scored differently — the formula predicted 108 stall cycles where the true figure was 80. Because the formula is an upper bound that assumes every potential conflict materialises, it ranks schedules incorrectly, and the search converges on the wrong one. **Build the accurate evaluator before the optimiser;** a search is only as good as the function it is minimising, and a fast wrong objective is worse than a slow right one.

Table 102. Numerical methods and their hardware suitability

Method	Convergence	Hardware note
Newton–Raphson	Quadratic	Needs a multiplier and a good seed; used for reciprocal and rsqrt
Goldschmidt	Quadratic	Better pipelining than Newton (no dependency chain)
CORDIC	Linear (one bit per iteration)	Shifts and adds only
Polynomial (minimax)	Fixed degree	Remez-fitted polynomials beat Taylor for equal degree
Piecewise polynomial + LUT	—	The usual production answer
Iterative refinement	Doubles accuracy per step	Limited by conditioning — see A5.2

Graduate · Taylor series should almost never appear in shipped hardware. A Taylor expansion minimises error at a single point and degrades away from it; a minimax (Remez) polynomial of the same degree distributes error evenly and typically achieves the same worst-case accuracy with one or two fewer terms. Since each term is a multiply–add, that is a direct area and latency saving. The standard production recipe is: reduce the argument to a small interval using the number's exponent, evaluate a low-degree minimax polynomial on that interval, then reconstruct. **Knowing that the textbook series is the wrong tool is a small piece of knowledge with a consistently measurable payoff.**

I4. Discrete Mathematics and Graphs in Hardware

Table 103. Graph problems in the design flow

Problem	Where it appears	Complexity	Practical method
Topological sort	Netlist evaluation, scheduling	Linear	Exact
Longest path	Static timing analysis	Linear on a DAG	Exact — this is what STA computes
Cycle detection	Combinational loop check	Linear	Exact
Min-cut / max-flow	Partitioning, retiming	Polynomial	Exact
Graph colouring	Register allocation, channel assignment	NP-hard	Heuristic
Clique / independent set	Binding in HLS	NP-hard	Heuristic
Shortest path	Routing	Polynomial	A*, maze routing
Steiner tree	Multi-pin net routing	NP-hard	Heuristic
Isomorphism	LVS (layout vs schematic)	Unknown class	Practical algorithms

Graduate · Recognising that a design task is a known graph problem is worth more than knowing an algorithm for it. Static timing analysis is longest path on a DAG — which is why combinational loops must be forbidden: they make the problem ill-posed, not merely difficult. Retiming is a min-cost flow problem, which is why it is exactly solvable while placement (a quadratic assignment problem) is not. **The tractability boundary explains the tool landscape:** tasks on the polynomial side have exact tools that engineers trust, and tasks on the NP-hard side have heuristics whose results engineers must inspect. A designer who knows which side a problem lies on knows whether to argue with the tool.

Table 104. Boolean algebra and representations

Representation	Property	Use
Truth table	Exponential size	Small functions, LUT mapping
SOP / POS	Two-level	PLA, simple synthesis
BDD	Canonical for a fixed variable order	Equivalence checking ; order-sensitive size
AIG (And-Inverter Graph)	Compact, not canonical	Modern synthesis and formal engines
CNF + SAT	—	Bounded model checking, equivalence
SMT	Theories beyond Boolean	Word-level formal verification

Graduate · **The shift from BDDs to SAT reshaped formal verification.** BDDs are canonical, which makes equivalence a pointer comparison, but their size explodes for arithmetic functions such as multipliers and is acutely sensitive to variable ordering. Modern SAT solvers, with conflict-driven clause learning, handle far larger instances without canonicity. The practical consequence for an IP engineer is that **formal property checking is now feasible on real control logic** — arbiters, FIFOs, protocol adapters, FSM reachability — where a decade ago it was a specialist activity. It remains infeasible on deep datapaths, so the division of labour is: formal for control, simulation with a golden model for datapath. **Knowing which side of that line a block falls on is how a verification plan is written.**

J1. On-Chip Memory

Table 105. Memory types available to an IP block

Type	Bit cell	Density	Notes
6T SRAM	6 transistors	High	Standard; compiled by the foundry
8T / 10T SRAM	8–10 T	Lower	Better read stability, low-voltage operation
Register file	Latch or flop array	Low	Multi-port, fast, expensive
Flip-flop array	—	Lowest	Synthesisable — the only portable option for small storage
Dual-port SRAM	8T	Lower	True dual port costs ~30–40% more area
Pseudo dual port	6T, time-multiplexed	High	Half the bandwidth
ROM	—	Very high	Constants, microcode, S-boxes
eDRAM / MRAM / RRAM	—	Very high	Process-specific; limited availability

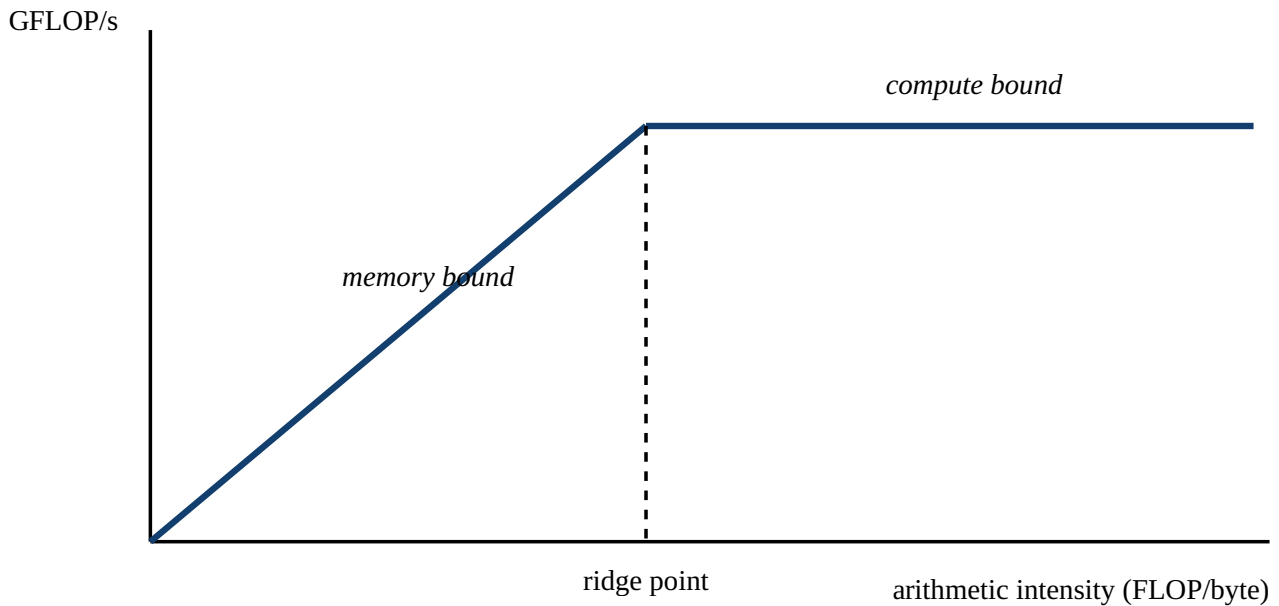
Graduate · **For an IP vendor the portability question dominates the density question.** A compiled SRAM is smaller and faster than a flip-flop array, but it is a foundry-specific macro: shipping RTL that instantiates one ties the customer to a particular process and library. The standard resolution is a **memory wrapper** — the IP instantiates a thin module with a defined interface, and the customer maps it to their compiler's output or to a behavioural model. A well-built IP ships the wrapper, a synthesisable fallback, and a document specifying the required timing (read latency, write behaviour on a read–write collision). **Omitting the collision behaviour is a frequent and expensive omission**, because different compilers do different things and the block's correctness may depend on which.

Table 106. Memory design considerations

Issue	Consequence	Handling
Read/write collision	Behaviour is compiler-dependent	Specify it ; forbid it in the design if possible
Banking	Enables parallel access	Address mapping becomes a design choice
Wide words vs many banks	Power and area trade	Follow the access pattern
ECC	Soft-error protection	Adds a cycle of latency and ~12–25% bits
Scrubbing	Corrects accumulated errors	Background read–correct–write
Redundancy and repair	Yield	Foundry feature; needs BIST/BIRA
Retention voltage	Low-power state	Below it, data is lost

Graduate · **ECC on memory is a reliability calculation, not a preference.** Soft-error rate is quoted in FIT per megabit (failures in 10^9 device-hours), typically a few hundred FIT/Mb at sea level for SRAM. A 10 Mb on-chip memory at 300 FIT/Mb gives 3000 FIT, or roughly one upset per 38 years per device — negligible for a phone, unacceptable for a fleet of a million servers, where it means an upset every twenty minutes somewhere in the fleet. **The same silicon needs ECC in one market and not in another**, which is why it must be a parameter rather than a fixed design choice, and why the datasheet must state the FIT contribution with and without it.

J2. Accelerators: Systolic Arrays and the Roofline

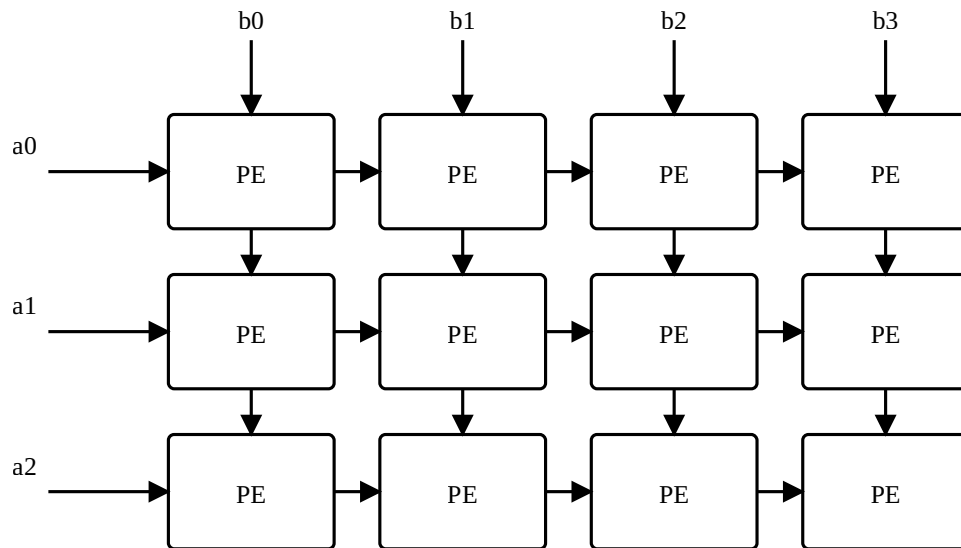


Left of the ridge, adding compute units buys nothing

Figure 13. The roofline model. A kernel's arithmetic intensity determines whether more compute or more bandwidth helps.

$$\text{Attainable performance} = \min(\text{Peak compute}, \text{Bandwidth} \times \text{Arithmetic intensity})$$

Graduate · **The roofline is the first slide of any accelerator proposal, and it decides the architecture.** Arithmetic intensity — useful operations per byte moved from memory — is a property of the algorithm and its dataflow, not of the hardware. A naive matrix multiply that re-reads operands has low intensity and is memory-bound: adding multipliers achieves nothing. Tiling the computation so that each loaded element is reused n times multiplies the intensity by n and moves the kernel rightward past the ridge, at which point additional compute finally pays. **Every accelerator architecture is, at bottom, a scheme for raising arithmetic intensity.**



Each datum is read from memory once and reused across the array

Arithmetic intensity rises with array size — that is the whole point

Figure 14. A systolic array. Operands flow through the array and are reused by every processing element they pass.

Table 107. Accelerator dataflow taxonomy

Dataflow	What stays resident	Best when
Weight stationary	Weights in the PE	Weights are reused across many activations (CNN inference)
Output stationary	Partial sums in the PE	Long reduction chains; minimises partial-sum movement
Row stationary	A row of the convolution	Balances all three reuse types
No local reuse	Nothing	Simple, memory-bound

Table 108. Precision in inference accelerators

Format	Bits	Effect	Hardware cost
FP32	32	Baseline	Reference
BF16	16	Same exponent range as FP32, fewer mantissa bits	~1/2 multiplier area; drop-in for training
FP16	16	Smaller range — needs loss scaling	~1/2
INT8	8	Requires calibration or QAT	~1/4 area, ~1/4 energy of FP32
INT4 / INT2	4/2	Significant accuracy loss without care	Smaller still
Binary / ternary	1–2	XNOR–popcount replaces multiply	No multipliers at all
Mixed / per-channel scale	—	Recovers most INT8 accuracy	Per-channel scaling logic

Graduate · Binary networks replace arithmetic with logic, which changes the whole cost structure. When weights and activations are ± 1 , a multiply becomes an XNOR and the accumulation becomes a population count. A 256-input dot product is then 256 XNOR gates and a popcount tree — a few hundred gates instead of 256 multipliers. The trade is accuracy, recovered partially by keeping the first and last layers at higher precision and by batch normalisation folded into a threshold

comparison. **For an IP house this is attractive because the resulting block is small, fully synthesisable, and has no dependence on DSP hard macros**, which makes it portable across FPGA and ASIC targets.

Table 109. Accelerator interface decisions

Decision	Options	Consequence
Weight delivery	Resident in the bitstream vs streamed at run time	Dominates the system architecture: bandwidth vs flexibility
Granularity	Layer-at-a-time vs fused layers	Fusion removes intermediate memory traffic
Control	Fixed sequence vs programmable	Flexibility vs area
Memory	Scratchpad vs cache	Scratchpad is deterministic; cache is easier to program
Sparsity	Dense vs structured vs unstructured	Unstructured sparsity rarely pays in hardware

J3. On-Chip Interconnect and Networks

Table 110. Interconnect topologies

Topology	Scaling	Latency	Use
Shared bus	Poor beyond ~8 masters	Low	Small SoC
Crossbar	$O(N^2)$ area	Low	Moderate port counts
Mesh NoC	Good	$O(\sqrt{N})$ hops	Many-core
Torus	Good	Lower diameter	—
Ring	Simple	$O(N)$	Moderate core counts
Tree / fat tree	Good for hierarchical traffic	—	Cache hierarchies
Butterfly / Clos	Good	Uniform	Switches

Table 111. NoC design issues

Issue	Cause	Solution
Deadlock	Cyclic dependency among buffers	Virtual channels, turn-model routing (e.g. dimension-order)
Livelock	Packets circulate without progress	Bounded misrouting, ageing
Head-of-line blocking	One blocked packet stalls the queue	Virtual channels
Fairness	Some sources starve	Arbitration policy
QoS	Mixed traffic classes	Priority virtual channels, rate limiting
Flow control	Buffer overflow	Credit-based

Graduate · **Deadlock freedom is a structural property and should be established structurally, not tested.** Dimension-order routing (traverse X fully, then Y) makes cyclic buffer dependencies impossible in a mesh, and the argument is a short proof rather than a simulation campaign. Adaptive routing performs better under load but reintroduces the possibility, which is then removed by reserving an escape virtual channel that uses dimension-order. **This is an unusually clean example of the general principle that architecture should make undesirable states unreachable rather than merely unlikely** — the same principle behind sparse FSM encodings and Gray-coded FIFO pointers.

Table 112. Chip-to-chip and chiplet interfaces

Standard	Scope	Note
PCIe	Board-level, packetised	Gen6 uses PAM4 and FLIT mode
CXL	Cache-coherent over PCIe PHY	Memory expansion and pooling
UCIe	Die-to-die in a package	Very short reach → very low energy per bit
BoW / AIB	Die-to-die	Open alternatives
HBM	Memory via silicon interposer	Very wide, moderate rate
Ethernet	Rack and beyond	—

Graduate · **Chiplets change the economics that an IP business operates in.** When a system is partitioned across dies, the interface between them becomes a standardised product boundary, and a company can sell an entire die rather than an RTL block. Energy per bit is the defining metric: on-chip wires are ~0.1 pJ/bit, UCIe die-to-die ~0.5 pJ/bit, and long-reach SerDes ~5–10 pJ/bit. **That two-order-of-magnitude spread is why partitioning decisions are made at the package level and why**

die-to-die PHY IP became valuable so quickly. A digital design house cannot build the PHY, but the link layer, the retry and CRC logic, and the protocol adapters above it are all synthesisable — the familiar pattern of a sellable digital block adjacent to an unreachable analogue one.

J4. Image and Video Coding

Table 113. Codec pipeline stages

Stage	Purpose	Hardware character
Prediction (intra)	Exploit spatial correlation	Many modes → mode decision is a search
Prediction (inter / motion)	Exploit temporal correlation	Motion estimation dominates encoder cost
Transform	Energy compaction (integer DCT)	Multiplier-free integer approximations
Quantisation	Lossy step — rate control lives here	Divide by a table value
Entropy coding	Remove remaining redundancy (CABAC)	Serial bottleneck
Loop filter (deblocking, SAO, ALF)	Remove artefacts	In the reconstruction loop — must be bit-exact
Reference frame store	—	DRAM bandwidth dominates

Graduate · **Encoder and decoder are asymmetric by design.** The standard specifies only the decoder; the encoder may choose modes however it likes, provided the bitstream is conformant. This means **encoder quality is a product differentiator while decoder correctness is a conformance requirement**, and the two have completely different verification strategies: a decoder is verified against conformance bitstreams with bit-exact expected output, whereas an encoder is evaluated on rate-distortion curves and must only produce decodable output. A design house entering video should know which side it is selling into, because the effort profiles differ by an order of magnitude.

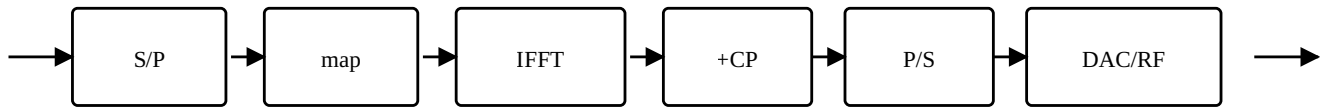
Table 114. Standards and their distinguishing features

Codec	Year	Key additions	Hardware impact
JPEG	1992	8×8 DCT, Huffman	Simple, still ubiquitous
H.264/AVC	2003	Integer transform, CABAC, variable block size	Integer transform removes encoder/decoder drift
HEVC/H.265	2013	CTU up to 64×64, 35 intra modes, SAO	Much larger mode search
VP9 / AV1	2013/2018	Royalty-free, ANS coding, many partitions	Very large encoder search space
VVC/H.266	2020	More partitions, ALF, affine motion	Further complexity
JPEG XS / light-weight	—	Low latency, visually lossless	Line-based, small buffers — a good niche for a small IP house

Bit-exactness inside the reconstruction loop is non-negotiable. The decoder's output feeds the encoder's prediction, so any arithmetic difference accumulates frame after frame — drift. This is why every in-loop operation (transform, quantisation, interpolation filters, deblocking) is specified as exact integer arithmetic with defined rounding, and why a reference model for a codec must reproduce those operations bit for bit rather than mathematically. **A floating-point reference model of a video codec is not a reference model.**

K1. Wireless Physical Layers

K1.1 OFDM



Cyclic prefix $\geq$ channel delay spread

$\Rightarrow$ multipath becomes circular convolution

$\Rightarrow$ one complex tap per subcarrier equalises the channel

The CP converts a hard equalisation problem into a division

Figure 15. OFDM transmitter. The cyclic prefix is the trick that makes frequency-domain equalisation valid.

Graduate · Why the cyclic prefix works, stated precisely. A linear convolution with the channel is not diagonalised by the DFT; a *circular* convolution is. Prepending the last N_{cp} samples of the symbol makes the channel's linear convolution look circular to the receiver, provided N_{cp} exceeds the channel's delay spread. The channel matrix then becomes diagonal in the DFT basis, so equalisation reduces to dividing each subcarrier by one complex number. **An N -tap time-domain equalisation problem has been exchanged for N_{cp}/N of lost throughput.** That trade is the entire justification for OFDM, and it explains why the CP length in every standard is tied to the expected delay spread of its deployment environment.

Table 115. OFDM impairments and their remedies

Impairment	Effect	Remedy	Where implemented
PAPR	Sum of subcarriers has a high peak	Clipping, tone reservation, SLM; or use SC-FDMA uplink	Digital pre-distortion; amplifier back-off
Carrier frequency offset	Destroys subcarrier orthogonality (ICI)	Estimate from preamble/pilots, correct before FFT	Digital
Sampling clock offset	Progressive phase rotation	Track with pilots	Digital
Phase noise	Common phase error + ICI	Estimate CPE from pilots	Digital
I/Q imbalance	Mirror-image interference	Estimate and compensate	Digital
Doppler	Time-varying channel within a symbol	Shorter symbols, or per-symbol tracking	Design-level

Graduate · Frequency offset is the impairment that must be fixed first and fixed well. A residual offset destroys orthogonality and produces inter-carrier interference that no subsequent equaliser can undo, because the subcarriers are no longer separable. The tolerance is roughly 1% of the subcarrier spacing for negligible loss; in 5G NR with 30 kHz spacing that is 300 Hz — at 3.5 GHz carrier, under 0.1 ppm. **This single requirement sets the crystal specification for the whole device,** which is a good illustration of a digital-layer requirement propagating down to a bill-of-materials cost.

K1.2 MIMO

Table 116. MIMO techniques

Technique	Objective	Requirement	Processing
Spatial multiplexing	Throughput	Rich scattering	Matrix inversion or successive cancellation per sub-carrier
Space-time coding	Diversity	None	Simple combining (Alamouti)
Beamforming	SNR, interference	Channel knowledge at the TX	Weight computation
Massive MIMO	Both	Many antennas, reciprocity	Large matrix operations; Cholesky/QR territory
MU-MIMO	Multi-user capacity	Multi-user CSI	Precoding (ZF, MMSE, dirty paper)

Table 117. MIMO detection algorithms

Detector	Complexity	Performance	Hardware
Zero forcing	One matrix inverse	Noise enhancement	Cholesky on $H^H H$
MMSE	One matrix inverse	Better at low SNR	Same, with a regularising term
V-BLAST / SIC	Iterative	Better	Ordering + repeated inversion
Sphere decoding	Variable	Near-ML	Data-dependent latency — awkward
K-best	Fixed	Near-ML	Fixed latency — preferred in hardware
Belief propagation	Iterative	Good	Fits with LDPC decoding

Graduate · **K-best beats sphere decoding in hardware for a reason that has nothing to do with performance.** Sphere decoding prunes a search tree adaptively, so its runtime depends on the channel and the noise realisation; a pipelined receiver with a fixed symbol rate cannot accommodate variable latency without buffering for the worst case, which negates the saving. K-best keeps a fixed number of candidates at every level, giving constant throughput and latency at slightly worse error performance. **Determinism is worth more than average-case efficiency in a streaming datapath,** and this principle recurs in FEC (fixed iteration counts), in SVD (fixed sweep counts), and in CFAR (fixed window sizes).

K2. Protocol Stacks as Hardware

Table 118. Ethernet layering and what each layer does

Layer	Function	Hardware	Clause (802.3)
MAC	Framing, addressing, CRC, flow control	Digital	4, 31
RS-FEC	Reed–Solomon KP4/KR4	Digital — soft IP	91, 161
PCS	64b/66b or 256b/257b coding, scrambling, lane distribution, alignment markers, gearbox	Digital — soft IP	49, 82, 119
PMA	Serialisation, clock recovery, equalisation	Analogue/mixed — hard macro	—
PMD	Electrical or optical interface	Analogue	—
AN / LT	Auto-negotiation, link training	Digital + control	73, 72

Graduate · **The PCS/PMA boundary is the commercial boundary of the whole industry.** Vendors describe their products in exactly these terms — "hardened PMA with a soft PCS layer" — because the PMA is tied to a process node and must be delivered as a hard macro, while the PCS is process-independent RTL. For a digital design house this boundary defines what is reachable: everything from the PCS upward. It is also where the open-source landscape is thinnest — 10G BASE-R PCS exists in open form, but 100G-and-above multi-lane PCS and RS-FEC do not — which makes it a reasonable place to look for an entry product.

Table 119. 64b/66b coding: what each piece is for

Element	Purpose	Failure if omitted
2-bit sync header	Distinguishes data from control blocks	Receiver cannot find block boundaries
Self-synchronising scrambler ($x^{58} + x^{39} + 1$)	DC balance and transition density	Baseline wander through AC coupling; CDR loses lock
Block type field	Encodes control block layout	—
Block lock state machine	Finds the header position	No synchronisation
BER monitor	Counts invalid sync headers	No link quality signal
Gearbox	Maps 66 bits onto a 64-bit datapath	Rate mismatch — 66 does not divide the bus width

Graduate · **The gearbox is the piece newcomers underestimate.** A 66-bit block does not fit a 64-bit datapath, so the PCS must repack a 66-bit stream into 64-bit words, which realigns every block by two bits and returns to alignment only after 32 blocks. The resulting barrel shifter and control state machine are a surprisingly large fraction of the PCS. Multi-lane 100G+ adds alignment markers, lane deskew and reordering on top. **The "simple" coding layer is mostly plumbing, and the plumbing is where the engineering effort goes** — a pattern that recurs in every protocol block.

Table 120. PCIe layering

Layer	Function	Notes
Transaction (TLP)	Read/write/completion semantics, ordering rules	Ordering rules are subtle and a frequent source of bugs

Layer	Function	Notes
Data link (DLLP)	Sequence numbers, ACK/NAK, credit flow control	Replay buffer sizing matters
Physical logical	Encoding (8b/10b, 128b/130b, FLIT), scrambling, lane management	—
Physical electrical	SerDes	Hard macro
LTSSM	Link training state machine	Large, intricate FSM — a classic formal-verification target

Table 121. DDR memory interface

Element	Function	Difficulty
Command scheduling	Bank, row, column timing	Performance-critical
Training	Write levelling, read gate, per-bit deskew	Runs at boot; a mix of hardware and firmware
PHY	DLL/PLL, DQS strobing	Hard macro
ECC	SECEDED or chipkill	Latency and bandwidth cost
Refresh	Periodic; blocks access	Real-time guarantee impact
Rowhammer mitigation	Track and refresh victims	Security requirement

K3. Reliability, Safety and Yield

Table 122. Wear-out mechanisms

Mechanism	Physics	Accelerated by	Design response
NBTI / PBTI	Interface trap generation under gate bias	Voltage, temperature, duty cycle	Timing guard band; adaptive body bias
HCI	Hot carriers damage the oxide	Switching activity, high field	Reduce voltage; limit slew
TDDB	Oxide breakdown	Field, temperature	Thickness rules; Weibull statistics
Electromigration	Momentum transfer moves metal atoms	Current density, temperature	Wire width rules, via arrays
Stress migration	Thermo-mechanical stress	Thermal cycling	Layout rules
Soft errors	Particle strikes	Altitude, neutron flux	ECC, TMR, scrubbing

Graduate · Wear-out is a timing constraint, not an afterthought. NBTI shifts $|V_{TH}|$ over the device lifetime, so a path that just met timing on day one may fail after five years. Sign-off therefore uses an aged library or an explicit derate — commonly 5–10%. For an IP vendor this matters in a specific way: **a frequency claim is implicitly a claim about a lifetime**, and the two must be quoted together. "500 MHz" at fresh silicon and "500 MHz over ten years at 125 °C" are different products.

Table 123. Functional safety (ISO 26262) concepts

Concept	Meaning	Hardware implication
ASIL A–D	Risk classification	D is the most stringent
SPFM	Single-point fault metric	≥99% for ASIL D
LFM	Latent fault metric	≥90% for ASIL D
PMHF	Probabilistic metric for random hardware failures	<10 FIT for ASIL D
Safety mechanism	Detects or controls a fault	ECC, lockstep, watchdogs, BIST
Diagnostic coverage	Fraction of faults detected	Drives the metrics
Lockstep	Duplicate cores compared cycle by cycle	Delayed lockstep avoids common-cause failures
FMEDA	Failure mode and diagnostic analysis	A required deliverable for safety IP

Safety certification changes the deliverable list, not just the design. An ASIL-D block must ship with an FMEDA showing computed SPFM, LFM and PMHF, a safety manual stating the assumptions of use, and evidence of a qualified development process. A technically excellent block without these documents cannot be used in a safety application at any price. **For a small design house this is the clearest example of a market that is closed not by engineering capability but by process obligations**, and it should be entered deliberately or not at all.

Table 124. Yield

Concept	Expression	Meaning
Defect density D_0	defects/cm ²	Process maturity
Poisson yield	$Y = e^{-AD_0}$	Pessimistic for large dies
Negative binomial	$Y = (1 + AD_0/\alpha)^{-\alpha}$	Accounts for defect clustering
Die area effect	—	Yield falls steeply with area — the economic driver behind chiplets

Concept	Expression	Meaning
Repair / redundancy	—	Spare rows and columns in memories raise yield
Parametric yield	—	Devices that work but miss specification

K4. Current Research Directions

This chapter surveys the areas where current literature is active, so that a reader who has absorbed the preceding chapters can approach a recent paper knowing what problem it is addressing and which established constraint it is trying to relax.

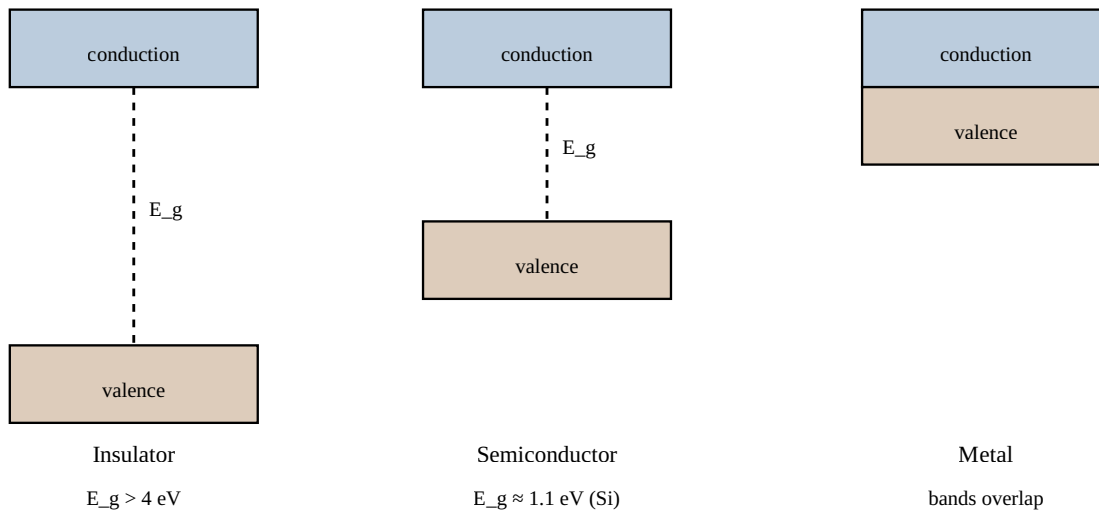
Table 125. Active areas and the constraint each attacks

Area	Constraint attacked	Approach	Open problem
Post-quantum cryptography	Shor's algorithm breaks RSA/ECC	Lattice (ML-KEM/ML-DSA), hash-based (SLH-DSA)	Large keys; NTT throughput; side-channel protection of lattice arithmetic
In-memory computing	Von Neumann data movement energy	Analogue MAC in memory arrays (RRAM, SRAM)	ADC overhead often cancels the saving; device variability
Approximate computing	Exactness is expensive	Truncated multipliers, stochastic computing, voltage overscaling	Error bounds must be guaranteed, not measured
Chiplets / UCle	Reticle limit and yield vs die area	Disaggregation with standard die-to-die links	Test, thermal, known-good-die
Nonlinear equalisation	Linear equalisers fail on nonlinear channels	Volterra series, neural equalisers	Power and latency; verification of a learned block
Concatenated FEC for 224G	KP4 alone insufficient at 200G/lane	Outer RS-KP4 + inner Hamming(128,120)	Latency budget; decoder complexity
Formal at scale	Simulation cannot cover state space	Word-level model checking, SMT	Datapath equivalence remains hard
ML for EDA	Heuristics in placement/routing	Learned cost models, RL for scheduling	Reproducibility and trust in sign-off
Open PDK / open EDA	Tool licence cost as a barrier	SkyWater/IHP PDKs, OpenROAD, Yosys	Analogue and advanced-node support
Cryogenic and superconducting	CMOS energy floor	SFQ logic, cryo-CMOS	Infrastructure; interface to room temperature

Graduate · How to read a paper in any of these areas. Each row names a constraint from earlier chapters. In-memory computing attacks the roofline's memory-bound region (J2); approximate computing attacks the fixed-point accuracy–area trade (A3, C1); neural equalisation attacks the linear-model assumption behind every equaliser in F1; chiplets attack the yield-versus-area relation in K3. **Identifying which constraint a paper is relaxing, and what it pays for the relaxation, is a faster route to understanding than reading the method first** — and it immediately tells you whether the result transfers to your own problem, because you know which assumption it depends on.

A caution that applies to all of these areas. Reported gains are measured at an operating point the authors chose. Before accepting a claim, locate three things: the baseline (is it a strong implementation or a straw man?), the operating conditions (process, voltage, data set, channel), and the point at which the advantage disappears. A result quoted without the crossover point is a data point, not a design rule — and in an IP business, mistaking one for the other is how a year is lost.

L1. Semiconductor Physics: What Survives into Design



Doping moves the Fermi level; the band gap sets the intrinsic carrier density

Figure 16. Band structure. The gap determines whether a material conducts, and silicon's 1.1 eV gap is what makes it useful.

Table 126. Physical quantities and where a designer meets them

Quantity	Symbol / value	Design consequence
Band gap of Si	1.12 eV at 300 K	Falls with temperature $\rightarrow$ leakage rises, V_{TH} falls
Intrinsic carrier density	$n_i \approx 10^{10} \text{ cm}^{-3}$	Doubles roughly every 8 K $\rightarrow$ junction leakage doubles with it
Thermal voltage	$kT/q = 25.9 \text{ mV}$ at 300 K	Sets the subthreshold slope: 60 mV/decade minimum
Mobility μ	$\approx 1400 \text{ (e)}, 450 \text{ (h)} \text{ cm}^2/\text{Vs}$	Why PMOS is sized $\sim 2\text{--}3\times$ wider than NMOS
Saturation velocity	$\approx 10^7 \text{ cm/s}$	Short-channel current becomes linear in overdrive
Oxide capacitance	$C_{ox} = \epsilon_{ox}/t_{ox}$	Thinner oxide $\rightarrow$ more drive and more gate leakage
Depletion width	$\propto \sqrt{(V/N)}$	Short-channel effects; junction capacitance

Graduate The 60 mV/decade subthreshold slope is a hard physical limit and it ended Dennard scaling. Below threshold, drain current depends exponentially on gate voltage with a slope of at best $(kT/q) \ln 10 \approx 60 \text{ mV}$ per decade at room temperature, because the mechanism is thermal emission over a barrier. To keep leakage constant while lowering the supply, the threshold must fall by the same amount — but each 100 mV of threshold reduction multiplies leakage by about fifty. **This is why supply voltage stopped scaling near 0.8 V and why power density rose**, which in turn is why the industry turned to specialised accelerators and therefore why the IP market grew. A thermodynamic constant is visible in a business model.

Table 127. Devices beyond the planar MOSFET

Device	Principle	Motivation
FinFET	Gate wraps a vertical fin on three sides	Better electrostatic control $\rightarrow$ less DIBL and leakage
GAA / nanosheet	Gate surrounds the channel completely	Further control; width quantised by sheet count

Device	Principle	Motivation
FD-SOI	Thin body on buried oxide	Low leakage; body bias as a live tuning knob
Tunnel FET	Band-to-band tunnelling	Can beat 60 mV/decade ; low on-current so far
Negative-capacitance FET	Ferroelectric gate stack	Internal voltage amplification; reliability unproven
2D materials	MoS ₂ and similar	Ultra-thin body; research stage

L2. Power Delivery and Conversion

Table 128. Voltage regulator types

Type	Efficiency	Ripple	Re- sponse	Where
LDO	V_{out}/V_{in} — poor if the drop is large	Very low	Fast	Sensitive analogue, small currents
Buck (switching)	85–95%	Higher (switching ripple)	Slower	Digital cores, large currents
Boost	—	—	—	Raising a voltage
Charge pump	Moderate	—	—	Small currents, no inductor
Integrated voltage regulator	—	—	Very fast	Per-domain DVFS on die
Switched-capacitor	Good at fixed ratios	—	—	On-die, no inductor needed

Graduate · Regulator response time is a digital design constraint. When a large synchronous block starts, it draws a current step; the regulator takes time to respond, and during that time the supply droops. The droop reduces gate drive and therefore increases delay, which can violate timing. Designers respond with **di/dt management** — staggering the release of clock gates so a block wakes in stages rather than all at once. **This is a purely digital technique introduced to solve an analogue problem, and it must be specified:** an IP that draws its full current in one cycle after an enable is harder to integrate than one that ramps.

Table 129. Decoupling hierarchy

Level	Capacitance	Effective range	Limit
On-die (MOS, MIM)	nF	>100 MHz	Area
Package	μF	10–100 MHz	Package inductance
Board ceramic	μF	1–10 MHz	ESL
Bulk electrolytic	100 μF+	<1 MHz	—
Anti-resonance	—	50–200 MHz typically	Where package L resonates with die C — must be damped

The anti-resonance peak is a common cause of otherwise inexplicable failures. Package inductance and on-die capacitance form a parallel resonance at which the power delivery network's impedance is much higher than at neighbouring frequencies. A workload whose current draw happens to contain energy at that frequency — a loop running at the resonant rate, a burst-mode accelerator — produces supply noise far larger than the average current would suggest. The symptom is a design that passes all tests but fails on one specific piece of software. **A digital block's current spectrum, not only its average power, is a system-level property.**

L3. Thermal and Packaging

$$T_{junction} = T_{ambient} + P \cdot \theta_{JA}$$

Table 130. Thermal resistance path

Segment	Symbol	Typical	Improved by
Junction to case	θ_{JC}	0.1–1 K/W	Die attach, lid material
Case to heatsink	—	0.1–0.5 K/W	Thermal interface material
Heatsink to ambient	θ_{SA}	0.5–10 K/W	Fin area, airflow
Total	θ_{JA}	—	—
Hot spot	—	—	Local power density, not average — floorplanning matters

Graduate · **Hot spots are a floorplanning problem, and they couple back into timing.** Average die temperature may be comfortable while a dense arithmetic block runs 20 K hotter than its surroundings. Since delay and leakage both depend on temperature, that block is slower and leakier than the sign-off corner assumed, and its leakage raises its temperature further — a positive feedback known as thermal runaway in the extreme. The design responses are spreading high-activity logic, inserting thermal sensors and throttling, and accounting for the gradient in timing analysis. **For an IP vendor, a block with very high local power density should say so,** because the customer's floorplanner needs to know.

Table 131. Package types

Package	Density	Thermal	Use
Wire-bond QFN/BGA	Low	Moderate	Cost-sensitive
Flip-chip BGA	High	Good (heat exits through the lid)	High performance
Silicon interposer (2.5D)	Very high	Good	HBM integration
3D stacking	Highest	Difficult — heat is trapped	Memory on logic
Fan-out wafer level	High	Moderate	Mobile
Chiplet in package	High	Manageable	Disaggregated designs

Graduate · **Thermal constraints are what limit 3D integration, not manufacturing.** Stacking a logic die under another die places its heat behind a thermal barrier, so the achievable power density falls sharply. This is why current 3D products stack memory (low power) on logic rather than logic on logic, and why thermal modelling now appears early in architecture rather than at the end. **The physical constraint determines the partitioning, which determines which interfaces are needed, which determines which IP is valuable.**

L4. Feedback and Control

Feedback loops appear throughout an IP portfolio: PLLs, CDRs, adaptive equalisers, AGC, DVFS controllers, thermal throttling, and the convergence of iterative decoders. The same small set of concepts governs all of them.

Table 132. Control concepts and their hardware instances

Concept	Meaning	Instance
Open-loop gain	Gain around the loop	PLL $K_{PD}K_{VCO}$
Loop bandwidth	Where gain crosses unity	The central design parameter of every loop
Phase margin	Phase at crossover, relative to -180°	$<45^\circ$ gives peaking and ringing
Type / order	Number of integrators / poles	Type-2 PLL tracks a frequency step with zero error
Damping ζ	—	$\zeta=0.707$ is the usual target
Steady-state error	—	Determines whether an offset persists
Settling time	—	Lock time; adaptation time
Discrete-time effects	Sampling and delay in the loop	Digital loops must include their own latency

Latency inside a digital loop is a stability problem, not merely a performance one. A digital adaptation loop that computes an error, pipelines it through several stages, and then applies a correction has inserted pure delay, which subtracts phase without reducing gain — exactly what destroys phase margin. The loop can oscillate even though every block in it is correct. The standard remedies are to reduce the loop gain (slower adaptation), to shorten the loop, or to compensate explicitly. **A model that applies the correction instantaneously will not show the instability**, so the pipeline delay must be in the model. This is one of the most common gaps between a working model and non-working hardware.

Table 133. Adaptation loops in a receiver and their interactions

Loop	Controls	Typical bandwidth	Interacts with
AGC	Input amplitude	Fast	Everything downstream
CDR	Sampling phase	Moderate	Equaliser — MM lock point depends on it
CTLE adaptation	Analogue peaking	Slow	FFE
FFE/DFE adaptation	Tap weights	Slow	CDR
Offset calibration	Comparator offsets	Very slow	Slicer thresholds
Interleaving calibration	ADC mismatch	Very slow	All of the above

Graduate · Six loops sharing one signal is the hardest part of a modern receiver, and it is a systems problem rather than a circuit one. Each loop is stable alone; together they can hunt, lock to a wrong point, or fail to converge from certain initial conditions. The standard engineering response is **bandwidth separation** — giving each loop a rate an order of magnitude apart so that faster loops appear instantaneous to slower ones and slower loops appear constant to faster ones — plus a defined start-up sequence. **Both the separation and the sequence belong in the specification**, and verifying convergence from adverse initial conditions is a system-model task that no block-level testbench will perform.

M1. The Hardware–Software Interface

An IP block is useless until software can drive it. The interface between them is a design artefact in its own right, and getting it wrong produces integration cost that dwarfs the block's development.

Table 134. Interface mechanisms

Mechanism	Direction	Cost	When appropriate
Memory-mapped registers	CPU → block	Low	Configuration, status; the default
Polling	CPU reads status	Wastes CPU cycles and power	Very short operations
Interrupt	block → CPU	Latency and context switch	Completion of long operations
DMA	block ↔ memory	Bus bandwidth	Bulk data; avoids CPU copying
Descriptor rings	—	Memory traffic	Streaming with many buffers (network, storage)
Mailbox / doorbell	bidirectional	Low	Command submission
Shared memory + fences	—	Coherence traffic	Tight coupling; requires a memory model

Graduate · **The single most consequential interface decision is who moves the data.** A block that requires the CPU to write its input and read its output word by word can be limited by the CPU even when the block itself is fast; a DMA-capable block moves data in bursts and leaves the CPU free. The difference is frequently an order of magnitude in system throughput, and it is invisible in block-level benchmarks. **For an IP vendor this means the example design matters as much as the block:** a customer who integrates it through programmed I/O will measure disappointing performance and conclude the IP is slow.

Table 135. Register map design rules

Rule	Reason
Generate RTL, header, documentation and tests from one source	Three hand-maintained copies always diverge
Reserve and document unused bits	Forward compatibility
Make read-clear and write-one-clear behaviour explicit	Ambiguity here produces lost interrupts
Provide an identification and version register	Software can adapt to the actual hardware
Group by function, align to natural boundaries	Simplifies access
Avoid side effects on read where possible	A debugger that dumps registers must not change behaviour
State atomicity for wide registers	Prevents tearing

Read side effects break debuggers, and the failure is baffling when it happens. If reading a FIFO status register pops an entry, then attaching a debugger that displays the register window silently consumes data, and the bug disappears when the debugger is detached. Where a read side effect is genuinely required — a pop-on-read data port, for example — it must be isolated in its own register and clearly marked, so that a debug tool can be told to avoid it. This is a small documentation duty with a disproportionate payoff.

Table 136. Driver and firmware concerns an IP vendor should anticipate

Concern	What the IP must provide
Initialisation sequence	Exact required order, including any wait conditions
Reset semantics	What a soft reset does and does not clear
Error reporting	Sticky status bits with a defined clearing mechanism
Concurrency	Which registers may be accessed while the block is running
Endianness and bit order	Stated explicitly
Timeouts	Maximum time any operation can take
Low-power entry/exit	Sequence and state retention

M2. Machine Learning Hardware

Table 137. Operations that dominate inference

Operation	Share of compute	Arithmetic intensity	Hardware mapping
Convolution	High in CNNs	High with tiling	Systolic array or vector units
Matrix multiply (GEMM)	Dominant in transformers	High	The universal accelerator primitive
Attention (QK^TV)	Dominant in LLMs	Moderate	Memory bound at long sequence lengths
Element-wise / activation	Low compute	Very low	Memory bound; fuse with the previous layer
Normalisation	Low	Low	Fuse; needs reductions
Softmax	Low	Low	Needs exp and a reduction; numerically delicate
Embedding lookup	—	Very low	Random memory access

Graduate · **Fusion is the main architectural lever and it follows directly from the roofline.** An activation function performs one operation per element and therefore has arithmetic intensity near zero — it is entirely memory bound. Executed as a separate pass it costs a full read and write of the tensor. Fused into the preceding matrix multiply's output stage it costs essentially nothing. Real accelerators therefore expose fused operations rather than primitive ones, and their compilers spend most of their effort deciding what to fuse. **An IP block that implements a single layer type in isolation will underperform in a system for reasons that have nothing to do with its internal quality.**

Table 138. Quantisation approaches

Approach	Accuracy loss	Effort	Hardware
Post-training, per-tensor INT8	Moderate	Low	One scale per tensor
Post-training, per-channel INT8	Small	Low	Per-channel scale multiply — usually worth it
Quantisation-aware training	Very small	High	Same as above
INT4 with grouping	Noticeable	High	Group-wise scales
Mixed precision	Small	Moderate	Sensitive layers kept wider
Binary / ternary	Large	High	XNOR-popcount; no multipliers

Graduate · **Quantisation is a fixed-point problem and the vocabulary of Chapter A3 applies unchanged.** A per-tensor scale is a single Q format for the whole tensor; per-channel scaling is the recognition that different channels have different dynamic ranges, which is exactly the block-floating-point idea from the FFT chapter. Understanding this correspondence prevents an engineer from treating ML quantisation as a separate discipline: the questions — where to round, when to saturate, how dynamic range grows through a computation — are the same, and the answers are reached the same way.

Table 139. Sparsity

Kind	Speedup potential	Hardware feasibility
Unstructured (random zeros)	High in theory	Poor — irregular access defeats the memory system
Structured (2:4, block)	Moderate	Good — regular indexing
Channel / filter pruning	Moderate	Excellent — it is just a smaller dense model
Activation sparsity (ReLU)	Moderate	Dynamic; needs run-time detection

Unstructured sparsity is the clearest case of a metric that does not translate into hardware. A paper reporting 90% of weights pruned implies a tenfold reduction in operations, but realising it requires indexing individual non-zeros, which destroys the regular access patterns that give an accelerator its efficiency. The achieved speedup is commonly a small fraction of the nominal one, and can be below unity. **Structured sparsity exists because hardware needs regularity more than it needs fewer operations** — a principle that recurs whenever an algorithmic saving is proposed.

M3. Image Signal Processing

Table 140. The ISP pipeline

Stage	Function	Why it is in hardware
Black level and defect correction	Sensor offset, dead pixels	Per-pixel rate
Lens shading correction	Vignetting	Per-pixel with a gain surface
Demosaic	Bayer → RGB	Neighbourhood operation at pixel rate
White balance	Illuminant compensation	Per-pixel gains
Colour correction matrix	Sensor → standard colour space	3×3 per pixel
Gamma / tone mapping	Perceptual encoding	LUT per pixel
Noise reduction	Spatial and temporal filtering	Line buffers dominate the area
Sharpening	Edge enhancement	Convolution
Scaling	Resize	Polyphase filter

Graduate · Line buffers, not arithmetic, determine ISP cost. A 5×5 neighbourhood operation at 4K resolution requires four lines of 3840 pixels each held on chip; at 12 bits per component that is roughly 184 kbit per stage before any compute. A pipeline with several such stages spends most of its silicon on storage. The architectural consequences are standard: process in tiles rather than full lines, fuse stages so that a neighbourhood is read once and used by several operations, and choose separable filters wherever possible. **This is the same memory-versus-compute analysis as the roofline of Chapter J2, applied to a different domain.**

Table 141. Colour and vision concepts that reach the hardware

Concept	Meaning	Implementation note
Bayer pattern	One colour per photosite	Demosaic must interpolate two of three
Colour space (sRGB, YCbCr)	—	YCbCr separates luma for chroma subsampling
Chroma subsampling (4:2:0)	Half resolution chroma	Exploits lower chroma acuity — halves the data
Gamma	Non-linear encoding	Must be undone before linear operations
HDR / tone mapping	Wide dynamic range compressed for display	Local operators need neighbourhood statistics
Rolling shutter	Rows exposed at different times	Motion artefacts; a system issue

Performing linear operations on gamma-encoded data is a persistent and quiet error. Gamma encoding is non-linear, so averaging two gamma-encoded pixels does not give the gamma encoding of their average. Scaling, blending and filtering must therefore be done in linear light, which means decoding before and re-encoding after. The visible symptom is subtly wrong brightness at edges — easy to miss in a test pattern and obvious on real images. **Specify the colour encoding of every interface**, in the same way and for the same reason that fixed-point formats must be specified.

M4. Measurement and Characterisation

Everything in this book that is stated as a number came from a measurement, and the difference between a useful number and a misleading one is usually in the method rather than the instrument.

Table 142. Instruments and what they actually measure

Instrument	Measures	Principal limitation
Oscilloscope	Voltage versus time	Bandwidth and probe loading
Real-time versus sampling scope	—	Sampling scopes need a repetitive signal
Spectrum analyser	Power versus frequency	Resolution bandwidth changes the displayed noise floor
Vector network analyser	S-parameters	Calibration plane definition
BER tester	Errors per bit	Time: 10^{-12} at 10 Gb/s takes ~100 s per point
Time interval analyser	Jitter statistics	Trigger accuracy
Logic analyser	Digital state	No analogue detail
Thermal camera	Surface temperature	Emissivity; not junction temperature
Power analyser / shunt	Current	Bandwidth versus resolution

Graduate · **Bathtub extrapolation exists because direct BER measurement does not scale.** Confirming 10^{-15} by counting errors would take days per operating point, which is impossible for a characterisation sweep. Instead the eye is scanned at several sampling phases, the error rate is measured where it is high enough to be quick, and the Gaussian random-jitter tails are extrapolated to the target BER. **The extrapolation is only valid if the random component really is Gaussian and has been correctly separated from the bounded deterministic component** — which is why jitter decomposition is a required capability rather than a convenience, and why a reported margin should always name the method.

Table 143. Characterisation practices

Practice	Reason
Sweep, do not spot-check	A single operating point can be unrepresentative or accidentally favourable
Confirm the operating point is healthy first	Measurements taken on a broken setup look plausible
Obtain the same quantity a second way	An independent estimate catches methodological errors
List the trivial explanations and rule each out by measurement	A result that could be an artefact is a hypothesis, not a finding
Record conditions with every number	Temperature, voltage, part, instrument, settings
Report the condition under which the claim fails	A claim with no stated failure condition is advertising

Four measurement failures, each of which produced a plausible number that meant nothing. Taking the slope of a pulse at its peak — which is zero by definition — and reporting it as a sensitivity. Measuring residuals at an operating point where BER was already 0.49, so the link carried no information. Asking a tool for the longest path in an entire netlist when the quantity of interest was the delay around one feedback loop. Reporting a lookup table's accuracy at an operating point where the feedback taps were so small that the loop was effectively inactive, so the table was never exercised. **In every case the instrument worked and the number was wrong, because the quantity measured was not the quantity intended.** The four rules above are written to prevent exactly these.

N1. Asynchronous and Globally Asynchronous Design

Synchronous design — one clock, one timing constraint — is overwhelmingly dominant because it makes timing analysis tractable. Asynchronous techniques remain relevant in three specific places, and knowing which is which prevents both over-application and blind avoidance.

Table 144. Asynchronous styles

Style	Timing assumption	Property	Practical use
Bundled data	Matched delay line models the logic	Simple, small	Local handshakes, self-timed SRAM
Dual-rail / QDI	None (delay insensitive)	Robust to any delay; $\sim 2\times$ area	Extreme reliability, some security uses
Micropipelines	Bundled	Elastic pipelines	—
GALS	Synchronous islands, async between	Avoids a global clock tree	The practically important case — multi-domain SoCs
Self-timed memory	Internal	Hidden from the user	Nearly all SRAM

Graduate · **GALS is what most engineers actually encounter, usually without the name.** Any SoC with several clock domains connected by asynchronous FIFOs is globally asynchronous and locally synchronous. Framing it that way is useful because it makes the design rules explicit: each island has its own timing closure, and every boundary between islands is a CDC problem requiring one of the techniques of Chapter C3. **The failure mode of an undisciplined multi-domain design is not a timing violation but an intermittent data corruption,** which is far harder to diagnose — which is why the boundaries must be enumerated and checked by a tool rather than by inspection.

Table 145. Where fully asynchronous logic is genuinely used

Application	Reason
SRAM internal timing	Self-timed sense amplifier control is faster than a clock
Security	No clock to glitch; power profile is less data-correlated
Ultra-low power / energy harvesting	No clock power when idle
Very wide dynamic voltage range	Works at whatever speed the supply allows
Reliability at extremes	Delay-insensitive circuits tolerate huge variation

The barrier to asynchronous design is tooling, not theory. Standard synthesis, static timing analysis, scan insertion and equivalence checking all assume a clock. An asynchronous block therefore needs custom flows, and — decisively for an IP vendor — the customer must be able to integrate, verify and test it in *their* flow. **An asynchronous block is usually unsellable as RTL for this reason alone,** regardless of its merits. Where the technique is used commercially it is nearly always hidden inside a hard macro with a synchronous interface.

N2. Physical Design in More Depth

Table 146. Placement and routing concerns

Concern	Cause	Symptom	Remedy
Congestion	More nets than routing tracks in a region	Routing failures, detours, timing loss	Spread cells, reduce utilisation, re-structure logic
Timing/congestion conflict	Timing wants cells close; routing wants them apart	—	Iterate; sometimes an architectural change is needed
Antenna effect	Charge accumulation on a long metal segment during etch	Gate oxide damage	Diode insertion, layer jumping
Crosstalk delay and noise	Coupling between adjacent nets	Timing variation, glitches	Spacing, shielding, net ordering
Electromigration	Current density	Long-term failure	Wire widening, via arrays
IR drop	Grid resistance	Local slowdown	Grid strengthening, decap
Hold violations after CTS	Skew changes after the clock tree is built	—	Buffer insertion; usually many thousands

Graduate · **Utilisation is the number that governs the whole back-end schedule.** Utilisation — the fraction of core area occupied by cells — trades area against routability. Below roughly 60% a design routes easily and wastes area; above roughly 80% congestion begins to dominate and each iteration becomes slower and less predictable. A block delivered with a recommended utilisation lets the customer floorplan correctly the first time. **Omitting it is one of the common causes of "the IP does not meet timing in our flow"**, because the customer packed it at 85% while the vendor characterised it at 65%.

Table 147. Standard cell library concepts

Concept	Meaning	Selection consequence
Track height (e.g. 7.5T, 9T)	Cell height in routing tracks	Taller cells are faster and larger
Drive strength (X1, X2, X4...)	Transistor width multiplier	Higher drive for larger loads
Multi- V_{TH} (LVT/SVT/HVT)	Threshold variants	Leakage versus speed, per cell
Characterisation corner	PVT point of the timing data	Must match the sign-off corner
NLDM / CCS	Delay model fidelity	CCS is needed at advanced nodes
Engineering change order cells	Spare gates scattered in the layout	Enable metal-only fixes after tape-out

Graduate · **Spare cells are cheap insurance that is often omitted and then badly missed.** Scattering a small percentage of unused gates across the floorplan allows a post-silicon logic fix to be implemented by changing only the metal masks, which costs a fraction of a full mask set and weeks rather than months. The area cost is typically one or two percent. **The decision is made at floorplan time and cannot be revisited**, which is why it belongs on the checklist of Chapter W1 rather than being left to back-end judgement.

N3. Statistics for Silicon

Table 148. Variation sources

Source	Scale	Correlation	Handled by
Lot to lot	Global	Fully correlated on a die	Process corners
Wafer to wafer	Global	Correlated	Corners
Die to die	Global	Correlated	Corners
Within die, systematic	Local	Position dependent	Layout-dependent effects, proximity rules
Within die, random	Local	Uncorrelated	OCV / AOCV / POCV derating
Random dopant fluctuation	Per device	None	$\sigma \propto 1/\sqrt{WL}$ — matters most for small devices

Graduate · The distinction between correlated and uncorrelated variation is what makes modern timing analysis tractable. If every device on a path varied independently, a long path would average out and show *less* relative variation than a short one — which is exactly what statistical on-chip variation (POCV) captures, and why it recovers pessimism that flat derating throws away. Flat OCV applies the same percentage to every path regardless of depth, which over-penalises long paths and under-penalises short ones (making hold analysis optimistic). **The statistics are not a refinement; using the wrong model can make a design that signs off fail in silicon.**

Table 149. Statistical concepts used in characterisation

Concept	Use
Mean, σ , $\pm 3\sigma$	Basic spread; $3\sigma \approx 99.73\%$
6σ design	Memory bit cells — billions of instances require extreme tails to be safe
Importance sampling	Simulating a 6 σ failure directly is infeasible; bias the sampling and correct
Monte Carlo	General variation analysis
Design of experiments	Efficient exploration of many parameters
Regression / response surface	Build a fast surrogate model
Confidence intervals	A measurement without one is an anecdote
Process capability C_{pk}	Manufacturing margin

Graduate · Memory bit cells illustrate why tail statistics matter so much. A 100 Mbit memory contains 10^8 cells; for the chip to yield, the per-cell failure probability must be well below 10^{-8} , which corresponds to roughly 5.7σ . Plain Monte Carlo would need on the order of 10^9 simulations to see such an event, so importance sampling — deliberately sampling the tail and reweighting — is mandatory. **This is why memory design is a specialised discipline**, and why memories come from compilers rather than being drawn by hand.

N4. Optical Links and Photonics

Beyond a few metres of copper, electrical links become impractical and the industry switches to optics. The digital blocks on either side — the FEC, the PCS, the DSP — are the same kind of IP discussed throughout this book, which is why the domain deserves a chapter.

Table 150. Optical link components

Component	Function	Impairment introduced
Laser (DFB, VCSEL)	Source	RIN, chirp, temperature drift
Modulator (direct, EML, MZM)	Impress data on light	Extinction ratio, chirp, nonlinearity
Fibre (SMF, MMF)	Transport	Chromatic dispersion, modal dispersion, attenuation, nonlinearity
Photodiode (PIN, APD)	Detect	Shot noise; APD adds excess noise
TIA	Current to voltage	Noise; bandwidth
Coherent receiver	Recover amplitude and phase	Requires a local oscillator and heavy DSP

Table 151. Modulation and detection schemes

Scheme	Detection	Reach	DSP required
NRZ / PAM4 IM-DD	Direct (intensity only)	≤ 10 km	Equalisation, FEC — 802.3 territory
Coherent QPSK / 16-QAM	Coherent (amplitude and phase)	Long haul	Very heavy: CD and PMD compensation, carrier recovery
Probabilistic shaping	Coherent	Long haul	Shaping encoder/decoder

Graduate • **Chromatic dispersion compensation is where photonics becomes a large digital problem.** Different wavelengths travel at slightly different speeds in fibre, so a pulse spreads over distance — and over hundreds of kilometres the spread reaches hundreds of symbol periods. Compensating it optically requires dispersion-compensating fibre; compensating it digitally requires a very long filter, which is implemented in the frequency domain because a direct FIR of that length is uneconomical. **The result is that a coherent optical receiver contains one of the largest FFT blocks in commercial electronics,** and the chapter on multirate and transform architectures applies directly. This is an unusually clear case of a physical-layer impairment creating a large, well-specified digital IP opportunity.

Table 152. Co-packaged and silicon photonics

Development	Motivation	Consequence
Pluggable optics	Serviceability	Electrical link to the module costs power
Co-packaged optics	Remove the electrical link	Large power saving; serviceability and thermal challenges
Silicon photonics	CMOS-compatible fabrication	Integration density, cost
Optical I/O for chiplets	Bandwidth density	Research to early product

W1. What a One-Person Design House Actually Sells

The central proposition of this volume is that a customer does not buy RTL. They buy **transferred risk**. A competent customer could usually write the block themselves; what they cannot cheaply produce is the evidence that it is correct, the documentation that makes it integrable, and a commitment to fix it when it is not. **Sellability is therefore a property of the evidence package, not of the code.** Everything in this volume follows from that.

W1.1 The deliverable set

Table 153. Minimum viable IP package

#	Deliverable	Content	What its absence signals to a buyer
1	Product brief (2–4 pp)	Function, performance, resources, interfaces, standards conformance	Not a product
2	User guide	Parameters and legal combinations, ports with timing diagrams, register map, integration procedure, reset and clocking requirements	Integration will consume the vendor's time, not the customer's
3	Synthesisable RTL	Parameterised, lint-clean, no tool-specific constructs	—
4	Reference model (C/C++ or Python)	Bit-accurate; usable in the customer's own environment	The customer cannot verify integration independently
5	Testbench and vectors	Self-checking, runnable on an open simulator if possible; standard vectors included	No acceptance test is possible
6	Verification report	Method, coverage achieved, tests passed, known limitations	The most common omission and the most revealing
7	Constraints (SDC) and synthesis scripts	Clock definitions, exceptions with justification, a reproduction recipe	Customer cannot reproduce the timing claim
8	PPA report	Area, frequency, power — with process, library, corner, utilisation stated	The numbers are unusable
9	Example design	Minimal system on a named FPGA board or simulation target	Evaluation takes weeks instead of a day
10	Release notes and versioning	Change history, compatibility policy	No basis for a long-term commitment
11	Licence and third-party notices	Scope of use, open-source bill of materials	Blocked in legal review
12	IP-XACT metadata (recommended)	Machine-readable ports, parameters, register map	Manual integration

Stating known limitations increases the probability of a sale. This is counter-intuitive and consistently true. The reviewer is an engineer who knows that every block has boundaries; a report without them reads as either incomplete verification or concealment. A limitation stated precisely — "throughput is guaranteed only when the sink asserts `ready` at least once every four cycles" — is information the integrator needs, and it demonstrates that the vendor has characterised their own block.

W1.2 Choosing the first product

Table 154. Suitability screen for a solo developer

Criterion	Good	Bad	Reason
Source of truth	Public standard with test vectors	Proprietary or informal	A solo developer cannot arbitrate correctness
Demand	Mandated by a standard	Nice to have	The customer must <i>need</i> it, not merely like it
Size	One function, 3–5 kLOC	Subsystem	Verification volume scales superlinearly
Domain	Digital, process-independent	Analogue/mixed-signal	PDK, tools and measurement equipment are out of reach
Interface	One standard bus	Custom protocol	Reuse and integration
Lifetime	Consumer/datacentre	Automotive/aerospace	10–20 year support obligations
Novelty	Well-understood algorithm	Novel algorithm	Proving novelty costs more than implementing it

Graduate · The last row is the one most often got wrong by technical founders. An IP business sells power, performance, area, interfaces, verification and support — not algorithmic novelty. A well-known algorithm has an external oracle, comparable prior implementations to benchmark against, and a customer who already knows they need it. A novel algorithm has none of these, and the first question every customer asks ("how do I know it works?") has no cheap answer. **Existing implementations are an asset, not competition: they give you a baseline to quantify against.**

W1.3 Licensing and contracts

Table 155. Licence structures

Model	Mechanics	Suits
Per-design licence	Fixed fee, one tape-out	Most common; predictable for both sides
Multi-use / site	Fee for unlimited designs in a period	Larger customers
Royalty per unit	Fee per shipped device	High-volume; requires audit rights
Hybrid	Reduced up-front plus royalty	Industry norm for significant IP
Source vs obfuscated	RTL source, encrypted RTL, or netlist	Source access materially increases price
Maintenance	15–20% of licence per year	The recurring revenue that makes a solo practice viable

Table 156. Contract clauses that matter most to a solo vendor

Clause	Position to take	Why
Scope of use	Enumerate designs, derivatives, resale explicitly	The most frequent source of dispute
Acceptance criteria	Name the exact tests that constitute acceptance	Otherwise payment can be withheld indefinitely
Warranty period	Bounded (e.g. 12 months) defect correction	Unbounded is uninsurable
Limitation of liability	Cap at fees received	A consequential-damage claim on a tape-out would end a solo business
IP indemnity	'To the best of knowledge', capped	Unlimited patent indemnity is not survivable
Third-party components	Disclose the open-source bill of materials	Legal review will ask
Escrow	Offer it	Removes the 'what if you disappear' objection cheaply

Clause	Position to take	Why
Support response	Define business-hours response, not resolution	Resolution cannot be promised

Graduate · The indemnity clause deserves specific preparation. An unqualified promise that the IP infringes no patent exposes a one-person business to a liability unrelated to its size. The defensible position combines three things: a knowledge qualifier, a liability cap tied to fees received, and — crucially — **evidence that a search was actually performed**. A documented patent search, even a modest one, changes the character of any later dispute and is worth the few days it takes. Record the databases searched, the classification codes, the date, and the conclusions, exactly as you would record a verification campaign.

W2. The Solo Process, Stage by Stage

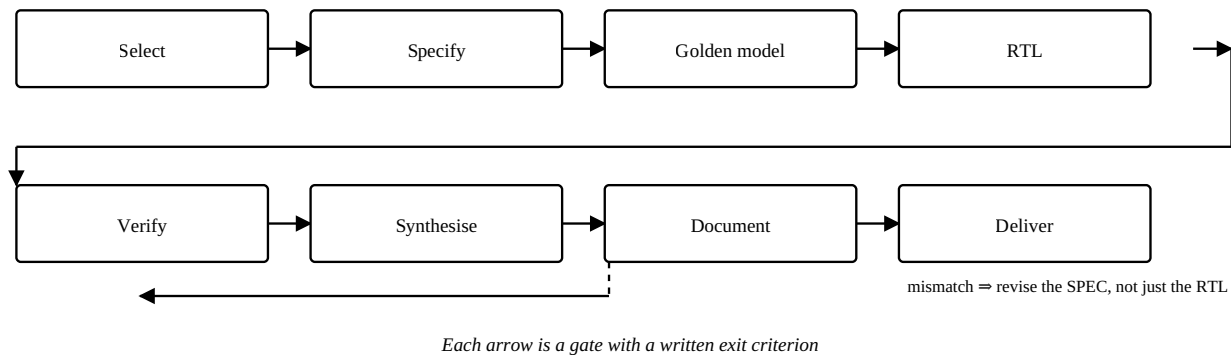


Figure 17. The development flow. What makes it a process rather than a sequence of activities is that each arrow has a written exit criterion.

Table 157. Stage gates

Stage	Output	Exit criterion (all must be true)
Select	Prior-art note	A named standard mandates it • prior implementations listed by name and size • the specification text is obtainable • a patent search has been performed and recorded
Specify	Specification document	Every parameter has a range and legal-combination rules • undefined behaviour is marked as such • word lengths, rounding, overflow policy and reset values are written down • reset and clocking requirements stated
Golden model	Reference model + self-tests	Standard vectors pass • deliberate mutation of the model turns the tests red • the model is deterministic • a second independent implementation or an external oracle agrees
RTL	Synthesisable RTL	Lint clean • no inferred latches • CDC static analysis clean • synthesises without warnings at the target frequency
Verify	Testbench, coverage, regression logs	Zero mismatches against the model • functional coverage goals met • no unexplained intermittent failures • mutation testing performed
Synthesise	PPA report	Timing closed at the stated corner • conditions fully documented
Document	The twelve deliverables	No section empty • the example design runs from a clean checkout
Deliver	Release	Acceptance tests defined and demonstrated

Graduate · The gate that solo developers skip is the third one: verifying the reference model. Because the model defines correctness, there is a natural tendency to treat it as correct by definition. It is not, and an error there propagates into the RTL (which is written to match it) and into the testbench (which checks against it), so the regression is green and the silicon is wrong. The three cheap defences are standard vectors, a second independent implementation, and mutation testing of the model's own checks. **In a one-person operation, where independence between designer and modeller cannot be organisational, these substitutes are the only thing standing in for it.**

W2.1 Specification: the parts that are always missing

Table 158. Specification checklist for numerical blocks

Item	Question to answer explicitly
Word lengths	Every signal's Q format, including internal accumulators
Rounding	Truncate, round-half-up, or round-half-even — at every reduction point
Overflow	Saturate or wrap , per signal — they are not interchangeable
Reset values	What is every state element after reset
Accumulation order	Fixed or unspecified — it matters under saturation
Latency	Exact cycle count, and whether it is data-dependent
Throughput	Initiation interval and any conditions on backpressure
Error behaviour	What happens on illegal input — and is it flagged
Undefined regions	Marked as such, with the expectation that DV masks them

A measured illustration of why reset values belong in the specification. In a feedback structure (DFE, IIR, accumulator), a mismatch in the initial register value causes the first outputs to differ, and the feedback carries the difference forward. In one measured comparison, 103 of 200 vectors mismatched; after aligning the model's and the RTL's reset value the count fell to 10, and the remaining ten had a separate cause. **Half the apparent failures were one unwritten specification line.** Debugging that without the specification costs days; writing it costs a sentence.

W2.2 Building the golden model

Table 159. Reference model construction rules

Rule	Rationale
Do not use host arithmetic for fixed or floating point	Results become machine-dependent; use explicit integer or SoftFloat
Hide state access behind accessors	Allows the representation to change without touching every operation
One file (or function) per operation where possible	Extensions are additive; the Spike ISS is the model for this
Make it fast	It will run millions of times in regression
Make it deterministic	Seeded randomness only; no wall-clock, no hash order
Emit a trace	The DV team cannot compare what it cannot see
Annotate with specification section numbers	Three years later, this is the only record of why
Write tests that bite	Deliberately break the model and confirm the tests fail

Graduate · **A concrete pattern for “tests that bite”.** For an error-correcting decoder, a test that injects zero errors passes even if the decoder does nothing at all. The useful test injects *exactly* t errors and requires full correction, injects $t+1$ and requires that the original is *not* recovered, and separately counts detected failures versus mis-corrections. Then it breaks the decoder on purpose — forcing the syndrome to zero — and asserts that the first test now fails. A suite that does not contain that last step has never been tested itself.

W2.3 Parameterisation

Table 160. Parameter design rules

Rule	Mechanism	Effect
Expose orthogonal axes only	Design, not syntax	Prevents combinatorial explosion
Document legal combinations	A table in the user guide	Customer cannot guess
Enforce them at elaboration	\$error inside generate; static_assert in C++	Illegal configurations fail to build rather than misbehave
Declare the verified set	A table of tested combinations	Honest scope statement; exhaustive testing is impossible
Safe defaults	Most conservative values	Wrong usage degrades, not breaks

Graduate · Elaboration-time enforcement is the cheapest quality feature available. An illegal parameter combination that merely produces wrong behaviour will be discovered by the customer, late, and reported as a bug in your IP. The same combination rejected with a clear message at elaboration costs the customer five minutes. The difference in support burden over a product's life is large, and it requires perhaps twenty lines of code.

W3. Automation: Running Verification Around the Clock

A solo developer cannot out-work a verification team, but can out-schedule one: machines run while people sleep. The objective is a loop that, unattended, generates stimulus, compares against the golden model, classifies failures, and leaves a prioritised queue for the morning.

W3.1 The continuous verification loop

Table 161. Loop stages

Stage	Action	Requirement
1. Generate	Random parameters and stimulus from declared legal ranges	Seeds recorded
2. Run model	Reference model produces expected output	Fast and deterministic
3. Run RTL	Open-source simulator (Verilator/Icarus) or vendor tool	Scriptable, headless
4. Compare	Scoreboard with the correct ordering semantics	See H1.4
5. Classify	Group failures by signature	Automated triage
6. Minimise	Shrink a failing case to a minimal reproducer	The single highest-value automation step
7. Report	Ranked queue with reproduction commands	—
8. Coverage	Accumulate and identify gaps	Feeds back into stage 1

Graduate · Stage 6 is where automation earns its keep. A randomly generated failing case may be thousands of transactions long; a human cannot read it. Delta-debugging — repeatedly removing parts of the stimulus and re-running to see whether the failure persists — typically reduces such a case to a handful of transactions in a few minutes of machine time. The engineer then arrives to a minimal reproducer rather than a log file. **This is the difference between an overnight run that produces work and one that produces guilt.**

W3.2 Where AI helps and where it must not be trusted

Table 162. Machine assistance in an IP flow

Task	Suitability	Condition
Writing testbench boilerplate	High	Reviewed; it is code like any other
Generating directed tests from a specification table	High	The table is the source of truth, not the model
Triage and clustering of failures	High	Statistical, low risk
Test case minimisation	High	Mechanical; verifiable by re-running
Documentation drafting	Moderate	Every number must be regenerated from data
Suggesting RTL fixes	Moderate	Must be re-verified from scratch
Writing the reference model	Low	It would share the same misreading as generated RTL — independence is lost
Judging whether coverage is sufficient	Low	Requires domain judgement
Deciding that an intermittent failure is benign	None	This is the judgement that must never be delegated

The independence argument constrains automation more than capability does. Verification works because two implementations were derived separately. If the same tool writes the model and the RTL from the same prompt, the two share every misunderstanding and the regression proves only internal consistency. Automation is therefore safest on the *mechanical* parts of the loop — generation, execution, comparison, minimisation, reporting — and least safe on the parts that define what correct means. **Automate the pipeline; keep the oracle human or standards-derived.**

W3.3 Infrastructure a solo practice needs

Table 163. Toolchain

Function	Open-source option	Note
RTL simulation	Verilator (cycle-based), Icarus Verilog (event)	Verilator is fast enough for overnight regressions
Testbench	cocotb (Python) or plain SystemVerilog	cocotb lets the golden model and testbench share a language
Formal	SymbiYosys / Yosys	Sufficient for arbiters, FIFOs, protocol properties
Synthesis (estimation)	Yosys + an open PDK	Relative area/timing trends, not sign-off numbers
Lint	Verilator --lint-only, Verible	—
Coverage	Verilator coverage, simulator built-ins	—
Waveforms	GTKWave, Surfer	—
CI	Any runner	The regression must run on a clean checkout
Documentation	Generated from the same source as the RTL	See W2.3

Graduate · On open-source synthesis results: report trends, not absolutes. Yosys with an open PDK will not reproduce a commercial tool's area or frequency, and quoting its numbers as a specification invites embarrassment. What it *does* reliably show is *relative* behaviour — that configuration B is 40% larger than A, that removing a pipeline stage raises the critical path by 30%. Those comparisons are legitimate, useful during architecture, and free. **State the tool and PDK next to every number and the report stays honest.**

W4. Worked Exercises

Each exercise below is a self-contained project that can be completed with open tools and produces an artefact of the kind described in W1. They are ordered so that each reuses the infrastructure built by the previous one.

Exercise 1 — A verified $\text{GF}(2^m)$ arithmetic library

Table 164. Exercise 1

Element	Specification
Objective	Parameterised finite-field arithmetic in software, with a test suite that verifies the <i>field axioms</i> rather than sample values
Deliverable	Library, test suite, short note on the primitive-polynomial requirement
Key decisions	Log/antilog tables versus direct reduction; whether the polynomial is a parameter or a constant
Tests that bite	Every non-zero element has an inverse and $a \cdot a^{-1} = 1$; the exponent table is a bijection onto the non-zero elements; a non-primitive polynomial is rejected rather than silently accepted; corrupting the multiply makes the axiom tests fail
Why it matters	Every code and cipher block rests on this. A silent error here is invisible everywhere above it

Graduate · **The non-primitive-polynomial test is the instructive one.** An irreducible polynomial that is not primitive still defines a field, but $\alpha = x$ no longer generates the whole multiplicative group, so a log/antilog implementation built on it is wrong — and wrong in a way that ordinary arithmetic tests may not reveal, because most products are still correct. The defence is to verify at construction that the powers of α enumerate all $2^m - 1$ non-zero elements, and to refuse otherwise. **A library that validates its own parameters is the difference between a component and a hazard.**

Exercise 2 — Reed–Solomon encoder and decoder (model)

Table 165. Exercise 2

Element	Specification
Objective	$\text{RS}(n, k)$ over the library from Exercise 1: systematic encoder, syndrome/BM/Chien/Forney decoder
Parameters	n, k , first consecutive root — all arguments, because the standard's values may not yet be confirmed
Tests that bite	Encoder output has zero syndrome; exactly t errors are always corrected with zero mis-corrections; $t+1$ errors are never recovered , and the detected-versus-mis-corrected split is counted rather than assumed; a test with zero injected errors is explicitly flagged as meaningless; forcing the syndrome to zero makes the suite fail
Honest reporting	Do not claim that all errors beyond t are detected. Measure the mis-correction rate and state it

Graduate · **Parameterising the unknown constants is the professional move here.** When the standard's field polynomial and first-root index have not been confirmed from the specification text, hard-coding a plausible value creates a model that is confidently wrong. Making them arguments, defaulting to a documented guess, and marking the guess as unverified in the code keeps the work moving without creating a false record. **The tests verify properties — t corrected, $t+1$ not recovered — which hold for any legal parameter choice, so they remain valid when the constants are later confirmed.**

Exercise 3 — RTL and co-simulation

Table 166. Exercise 3

Element	Specification
Objective	Synthesisable RTL for the Exercise 2 decoder's syndrome stage, co-simulated against the model
Method	DPI-C or cocotb; random vectors; seeds recorded
Tests that bite	Break the RTL deliberately and confirm the comparison fails; break the <i>model</i> deliberately and confirm it also fails — a harness that has only ever seen a correct design has not been tested
Extension	Sweep the parallelism parameter and plot cells versus throughput with Yosys; report the trend, not absolute area

Exercise 4 — Fixed-point conversion

Table 167. Exercise 4

Element	Specification
Objective	Convert a floating-point reference to fixed point and find the minimum word length meeting an accuracy target
Method	Sweep integer and fractional bits; measure the metric that matters (BER, SNR, residual) rather than raw error
Required experiments	Saturation versus wrap-around at each width; rounding mode comparison; reset-value alignment between model and RTL
Expected finding	The correct overflow policy is block-dependent: a CIC needs wrap, an LLR-based decoder needs saturation. Reproduce both results rather than accepting the claim

Check the operating point before believing any fixed-point result. If the floating-point baseline is already failing — BER near 0.5, a decoder not converging, a sampling phase misaligned — then every quantisation comparison made on top of it is meaningless, and the numbers will look plausible. The discipline is to sweep first for a healthy operating point, assert that the baseline metric lies in a sensible range, and only then vary word length. An assertion such as $0.02 < \text{BLER} < 0.6$ in the test harness makes this automatic.

Exercise 5 — A complete deliverable package

Table 168. Exercise 5

Element	Specification
Objective	Assemble the twelve deliverables of W1.1 for the Exercise 3 block
Hardest parts	The verification report's <i>known limitations</i> section; the PPA report's <i>conditions</i> ; the example design that runs from a clean checkout
Self-test	Hand the package to someone else and ask them to integrate it without speaking to you. Every question they ask is a documentation defect
Outcome	This is the artefact that demonstrates capability to an employer or a customer — not the RTL alone

W5. From Literature to a Block

W5.1 What published work is for

Table 169. Four uses of literature in an IP practice

Use	What to extract	Where to look	Trap
Architecture selection	Which structure, and under what conditions	JSSC, ISSCC, VLSI Symposia, DAC/ICCAD	Reported numbers come from the authors' best operating point
Algorithmic justification	Correctness conditions, convergence bounds	IEEE Transactions, arXiv	Confusing a theory paper with an implementable one
Attack awareness	Whether a countermeasure is already broken	CHES, CCS, USENIX Security, IACR ePrint	Mandatory for security IP
Standards rationale	Why a specification says what it says	Standards-body contribution archives	Contributions are proposals; not all were adopted

Graduate · **Standards contributions are the most under-used resource.** The published standard states *what*; the working-group contributions that preceded it state *why*, with the measurements and the alternatives that were rejected. For IEEE 802 these are publicly archived. Reading the contributions for the clause you are implementing typically answers the questions the standard's terse language leaves open, and it does so faster than deriving the answer. **It also reveals which parameter choices were contentious, which is exactly where implementations are most likely to differ.**

W5.2 Reading efficiently

Table 170. A triage procedure

Step	Action	Decision
1	Read the figures and tables first	What was measured?
2	Find the experimental conditions	Is this comparable to my case?
3	Identify the baseline	Is the comparison fair?
4	Find the crossover	Where does the claimed advantage disappear?
5	Only now read the method	How does it work?
6	Read limitations and future work	What do the authors already know is wrong?

Record your confidence level with every citation. A note that says "full text read", "abstract only", or "search snippet only" costs nothing to write and prevents the most damaging failure mode in technical work: building a plan on a claim nobody actually verified. If a design decision rests on a paper you have not read in full, that fact should be visible in the document where the decision is recorded. **Unverified citations propagate quietly and are discovered late.**

W5.3 Turning a paper into a specification

Table 171. Translation checklist

Question	Why it must be answered before design starts
What exactly is the input and output format?	Papers routinely omit scaling, ordering and normalisation conventions

Question	Why it must be answered before design starts
What are the numerical requirements?	Most papers assume floating point; the word lengths are yours to determine
What is the initialisation?	Iterative methods rarely state the starting point; it affects convergence
What is the termination condition?	'Until converged' is not implementable; fixed iteration counts are
Which parameters were tuned to the reported data set?	Those parameters will not transfer
What is the failure behaviour?	Papers report average performance; you must specify the worst case
Is there a patent?	Published does not mean free to use

Graduate · **The termination-condition question separates research from engineering.** An algorithm described as iterating until a residual falls below a threshold has data-dependent latency, which a streaming datapath cannot accept. The engineering translation is a fixed iteration count chosen so that the worst case in the operating region still meets the accuracy requirement — and then the accuracy becomes a design-time constant rather than a run-time property. This is exactly what the Jacobi SVD does with a fixed sweep count, what an LDPC decoder does with a maximum iteration limit, and what K-best detection does relative to sphere decoding. **Converting an adaptive algorithm into a fixed-latency one is a recurring and characteristic act of hardware design,** and the specification must record the accuracy that the fixed budget guarantees.

W6. Working on a Modelling Team

W6.1 The first week

Table 172. Questions to ask, and what the answers reveal

Question	What a good answer looks like	What a poor answer means
What is our golden model, and who owns it?	A named artefact with an owner	Correctness is decided ad hoc
How does the model connect to the RTL?	A defined trace interface (RVFI-like) or DPI harness	Building one is your first project
How often does the regression run, and how long does it take?	Nightly, under eight hours	Feedback is too slow to steer design
What is the specification?	A standard document or a maintained internal spec	The model is the specification — large responsibility
How is undefined behaviour handled?	Explicitly marked and masked in comparison	The model may have been tuned to match the RTL — independence lost
Who verifies the model?	Standard vectors, a second implementation, mutation testing	Nobody — this is your highest-value contribution
What escaped in the last year, and why?	A specific analysis	No escape analysis means no learning loop

Graduate · **The last question is the most informative one you can ask.** An organisation that can describe a specific escape, its root cause, and the check that was added in response has a functioning improvement loop. One that cannot recall any escape either has not shipped much or does not trace failures back. The answer tells you more about how the team works than any description of its methodology.

W6.2 Habits that compound

Table 173. Working practices

Practice	Effect
Annotate model code with specification section numbers	Makes the model auditable and survivable
Record the seed and the exact command with every failure	Turns an anecdote into a reproducible fact
Minimise a failing case before reporting it	Preserves the designer's trust and time
Suspect the comparison before the design	Most mismatches are scoreboard or timing issues
Write the check that would have caught it	Converts each bug into permanent coverage
Deliberately break things to test the tests	The only way to know a suite is alive
State confidence levels in reports	Distinguishes measured from assumed
Never close an unreproduced intermittent failure	Removes evidence, not the bug

Graduate · **“Suspect the comparison first” is worth making a reflex.** In a mature project the ordering of likelihood is: the scoreboard or its timing assumptions, then the model, then the RTL. Reporting a suspected RTL bug that turns out to be a test-bench issue twice in a row is enough to make the third report — the real one — go unexamined for a week. Running the mod-

el and the RTL separately on the same stimulus and reading both logs takes an hour and settles the question before anyone else is involved.

W6.3 Writing that gets read

Table 174. Reporting a discrepancy

Section	Content
One-line summary	What differs, where, under what condition
Minimal reproducer	Exact command, seed, and the shortest failing stimulus
Evidence	Model output and RTL output side by side, with the first differing cycle identified
Triage	Which of the three causes has been ruled out, and how
Specification reference	The clause or section that decides the question
Proposed disposition	Model change, RTL change, or specification clarification
Confidence	What you verified versus what you inferred

The last row is the habit that most distinguishes experienced engineers in written communication. A report that separates “I measured this” from “I believe this follows” can be acted on immediately; one that blends them forces every reader to re-derive the distinction. Over a project's life this single discipline saves more time than any tool.

W7. Debug and Bring-Up

The chapters so far describe how to avoid defects. This one describes what to do when one is present anyway, which is a distinct skill and the one most visibly separating experienced engineers from new ones.

W7.1 A debugging method

Table 175. Systematic debugging

Step	Action	Why this order
1	Reproduce reliably	An unreproducible failure cannot be verified as fixed. Record the seed, the command and the environment
2	Minimise	Shrink the stimulus until any further reduction removes the failure. Often reveals the cause without further work
3	Bisect in time	Find the first cycle at which observed and expected diverge — not where the symptom appears
4	Bisect in space	Move the observation point upstream until the signal is correct; the boundary contains the fault
5	Bisect in history	If it worked before, bisect the revisions
6	Form one hypothesis and test it	Changing several things at once destroys the evidence
7	Explain the mechanism	A fix that works without an explanation has probably moved the bug
8	Add the check that would have caught it	Otherwise the class of defect will recur

Step 7 is the one under schedule pressure most often skipped, and it is the one that causes recurrence. A change that makes a symptom disappear without an understood mechanism has frequently only altered timing or masking, leaving the defect present and less observable. The question that settles it is: *can I re-create the failure on demand by undoing exactly this change, and does my explanation predict that?* If not, the investigation is not finished.

W7.2 Observation techniques by level

Table 176. What can be observed where

Level	Technique	Visibility	Cost
Simulation	Waveforms, printouts, assertions	Complete	Slow
Formal	Counterexample trace	Complete, minimal	Limited scope
Emulation	Trace buffers, triggers	Large but finite	Expensive
FPGA prototype	Integrated logic analyser	Small window	Compile time
Silicon	Scan dump, on-chip trace, debug bus	Very limited	Must be designed in beforehand
Silicon, physical	Laser probing, FIB, e-beam	Specialist	Very expensive

Graduate · Observability in silicon is decided at design time, and it is decided by people who are not yet debugging. Once a chip exists, the only internal state visible is what someone chose to expose: scan chains, trace buffers, performance counters, a debug bus. A block that provides no status registers, no error counters and no way to inject or capture data is effectively opaque, and diagnosing a problem in it means diagnosing the whole subsystem. **For an IP vendor, debug features**

are a support-cost investment: an error counter and a loopback mode cost a few hundred gates and can save a week of remote debugging on a customer's board.

W7.3 Bring-up

Table 177. Bring-up order

Stage	Check	If it fails
Power	Rails at expected voltages and sequence	Board or PMIC; no chip involvement
Clocks	Reference present; PLL locks	Check the reference before the PLL
Reset	Released cleanly, in the right order	Reset sequencing
Basic life	JTAG IDCODE reads back	If this fails, nothing else can be diagnosed
Debug access	Core halts and registers are readable	Debug module or its clocking
Memory	Write and read back patterns	Interface training
Boot	First instructions execute	ROM, fuses, boot mode pins
Console	UART output	From here the system can explain itself
Peripherals	One at a time	—
Performance	Against the model's prediction	Configuration or a real shortfall

Graduate · **The ordering is not arbitrary: each stage is the prerequisite for diagnosing the next.** Chasing a peripheral fault before the console works means debugging blind. This is also why the UART and the debug module deserve disproportionate care in design and verification — they are the instruments with which everything else will be measured. **A bug in the debug path costs far more than its size suggests,** because it removes the means of finding other bugs.

W7.4 Classes of bug and where each is found

Table 178. Bug taxonomy

Class	Typical cause	Found by	Missed by
Functional	Misread specification	Model comparison	Directed tests alone
Protocol	Interface misuse	Protocol checkers, formal	Block-level tests
Corner case	Unusual combination	Constrained random, formal	Directed tests
Race / CDC	Missing synchroniser	CDC analysis, gate-level simulation	RTL simulation
Reset	Uninitialised state	X-propagation analysis	RTL with optimistic X
Timing	Path too long	STA	Any simulation
Power/IR	Local droop	Power analysis	All functional verification
Performance	Wrong architecture	System model	Block verification
Integration	Misunderstood interface	System simulation	Block verification
Specification	Requirement wrong	Review, prototype	All verification

The last row is the one no amount of verification can address. If the specification asks for the wrong thing, a perfectly verified implementation of it is still useless. This is the argument for early prototypes, for system models built before RTL,

and for showing a specification to whoever will ultimately use the block. **Verification answers "did we build it right"; only review and prototyping answer "did we build the right thing".**

W8. Positioning, Pricing and Support

W8.1 Who buys IP and why

Table 179. Buyer motivations

Buyer situation	What they are really buying	What convinces them
No in-house expertise	Capability	Evidence the block works; references
Expertise but no schedule	Time	Maturity; a working example design
Risk aversion	Someone to hold responsible	Verification evidence and support terms
Standard conformance needed	Conformance evidence	Test vector results, certification
Cost reduction	Lower total cost than building	A credible effort comparison
Second source	Supply security	Compatibility with the incumbent

Graduate · The third row is the most common and the least discussed. An engineering manager who licenses a block has converted an internal schedule risk into a contractual relationship. That is worth paying for, and it explains why the verification report and the support agreement carry more weight in the decision than the RTL's elegance. **It also explains why a first sale is so much harder than a second one to the same customer:** the first requires establishing that the risk transfer is real.

Table 180. Pricing inputs

Factor	Direction	Note
Development cost	Floor	Must be recovered across expected licences
Customer's build-versus-buy cost	Ceiling	Their engineer-years times their rate
Value of the schedule saved	Raises the ceiling	Often larger than the build cost
Competitive alternatives	Anchors	Including open-source options
Volume	—	Royalty structures capture upside
Source access	Raises price substantially	Source enables the customer to fork
Support scope	Recurring revenue	Usually 15–20% of licence per year
Exclusivity	Large premium	Rarely wise for a small vendor

Free alternatives set the floor even when they are not equivalent. If an open-source block exists that does approximately the same thing, the value proposition must be stated against it explicitly: verification evidence, documentation, support, conformance testing, indemnity. “Ours is better” is not an argument a procurement process can act on; “here is the verification report and the conformance results that the open version does not have” is. **This is another reason to invest in the evidence package rather than in additional features.**

W8.2 Support

Table 181. Support categories and their true cost

Category	Frequency	Cost driver	How to reduce it
Integration questions	High initially	Documentation gaps	Better user guide and example design
Configuration errors	High	Unclear parameter rules	Elaboration-time assertions

Category	Frequency	Cost driver	How to reduce it
Environment differences	Moderate	Tool and version variety	State tested tool versions
Real defects	Low if verification was good	—	Verification investment
Feature requests	Moderate	—	Roadmap and paid customisation
Performance shortfalls	Moderate	Unstated assumptions	Document measurement conditions

Graduate · Most support load is documentation debt, not defects. A useful discipline is to treat every support question as a documentation bug: when the answer is given, the document is updated so that the same question cannot be asked twice. Over a few customers this converges, and support cost per customer falls sharply. **For a one-person practice this is the difference between a viable business and one that cannot take a second customer,** because support is the cost that scales with customers rather than with product.

Table 182. What to track from the first release

Item	Why
Every question asked, and by whom	Reveals documentation gaps
Time spent per customer	Tells you whether the price is right
Defects found after release, and how	Shows what verification missed
Configurations actually used	Focuses future verification effort
Tool and version combinations	Compatibility matrix
Reasons for lost sales	Product direction

Q1. Performance Modelling and Queueing

Functional correctness is verified against a golden model; performance is predicted by a different kind of model entirely. Confusing the two is a common and expensive mistake, because a functionally perfect block can still fail its throughput requirement in a system.

Table 183. Model types and what each answers

Model	Question answered	Cost	Fidelity
Analytical (closed form)	Rough sizing, bounds	Minutes	Low but instant
Spreadsheet / static	Bandwidth and utilisation budgets	Hours	Adequate for first-order decisions
Queueing model	Latency under load, buffer sizing	Days	Good when arrivals are stochastic
Trace-driven simulation	Behaviour on real workloads	Days	High
Cycle-approximate (SystemC TLM)	System behaviour before RTL exists	Weeks	Good
Cycle-accurate	Exact cycle counts	Weeks–months	Highest
RTL simulation	Ground truth	—	Exact but far too slow for system studies

Graduate · The practical value of the middle rows is that they exist before RTL does. An architecture decision — how deep a buffer, how many outstanding transactions, whether to double the datapath width — must be made months before there is anything to simulate. A cycle-approximate SystemC model built from the specification answers it in days, and being wrong by 10% is usually irrelevant when the alternatives differ by a factor of two. **Insisting on cycle-accurate fidelity for an architectural question delays the decision until it is too late to act on.**

Table 184. Queueing results worth knowing

Result	Statement	Use
Little's law	$L = \lambda W$	Occupancy = arrival rate × latency — sizes any buffer
M/M/1 latency	$W = 1/(\mu - \lambda)$	Latency diverges as utilisation → 1
M/M/1 occupancy	$\rho/(1-\rho)$	At 90% utilisation the mean queue is 9
M/D/1	Deterministic service	Half the waiting time of M/M/1
Erlang B	Blocking probability	Sizing a finite resource pool
Burstiness	—	Real traffic is burstier than Poisson — Poisson results are optimistic

Graduate · Little's law is the single most useful formula in system design and it requires no assumptions about distributions. To sustain 10 GB/s with a 500 ns round trip, the system must keep $10 \times 10^9 \times 500 \times 10^{-9} = 5$ kB in flight; if the interface supports 32 outstanding 64-byte requests, that is 2 kB and the link will run at 40% of its rate no matter how fast it is. **This is the quantitative form of the PCIe tag-capacity argument in Chapter P2**, and the same calculation sizes DMA descriptor rings, memory controller queues and cache miss-status registers. Learning to apply it in thirty seconds is worth more than any simulator.

Designing for average load is designing to fail. The M/M/1 result shows latency rising hyperbolically with utilisation: at 50% the queue is 1, at 90% it is 9, at 99% it is 99. Systems provisioned at their average demand spend a substantial fraction of their time near saturation, where latency is dominated by queueing rather than service. **Buffer sizing must be driven by**

the tail of the arrival distribution, not its mean, and a performance specification that quotes only average throughput is incomplete.

Q2. Compilers and High-Level Synthesis Internals

HLS is often treated as a black box. Knowing what it does internally makes the difference between fighting the tool and directing it, and the internal steps are the same scheduling and binding problems that appear elsewhere in design.

Table 185. The HLS pipeline

Step	Function	What the designer controls
Front end	Parse to an intermediate representation	Language subset used
Optimisation	Constant folding, loop transforms, inlining	Code structure; pragmas
Scheduling	Assign operations to clock cycles	Target II and latency constraints; dependences
Allocation	Decide how many of each resource	Resource constraints, binding directives
Binding	Map operations to specific units, values to registers	Storage directives
Interface synthesis	Turn arguments into ports/protocols	Interface pragmas
RTL generation	Emit Verilog/VHDL	—

Graduate · **Scheduling is a constrained optimisation problem and that explains the tool's behaviour.** Given a dependence graph and resource limits, the scheduler minimises latency or the initiation interval. Because the problem is NP-hard in general, tools use heuristics (list scheduling, modulo scheduling for pipelines, occasionally ILP for small regions). Two consequences follow. First, a small change in the source can produce a disproportionate change in the schedule, because the heuristic follows a different path — which is why HLS results can be unstable. Second, **the tool cannot schedule around a dependence it cannot disprove**, so telling it the truth about aliasing is the highest-leverage thing a designer does. That is what a dependence directive is: not an optimisation hint, but a fact the tool could not derive.

Table 186. Directive families and what they actually do

Family	Examples	Nature	Risk of misuse
Scheduling	pipeline, unroll, inline, loop_flatten, loop_merge	Requests — the tool may not achieve them	Silent failure to meet the target; check the report
Structural	array_partition, bind_storage, bind_op, dataflow, stream	Change what is built	Area explosion
Assertions	dependence, loop_tripcount	Facts the tool trusts	A false assertion produces silently wrong hardware
Interface	interface, axis, m_axi, s_axilite	Define ports	Protocol mismatch

The third family is categorically different from the other two and should be reviewed differently. A scheduling directive that cannot be satisfied produces a worse result and a report message. A structural directive produces a bigger design. A false dependence assertion produces hardware that computes the wrong answer, with no message and no simulation failure — because the C semantics are unaffected. **Every dependence directive is therefore a proof obligation:** the reviewer must be able to state why the asserted independence holds for every legal parameter value. Treating it as an optimisation switch is how silent bugs enter.

Table 187. What HLS is good and bad at

Suited	Reason	Unsuited	Reason
Loop-dominated datapaths	Regular dependence structure	Cycle-exact interface protocols	The tool owns the schedule

Suited	Reason	Unsuited	Reason
Design-space exploration	One source, many points	Control-dominated logic	Irregular; RTL is clearer
Filters, transforms, crypto rounds	Well-structured loops	Blocks needing precise latency	Latency is an output, not an input
Algorithm-level changes	Edit the C, not the RTL	Deep pipelines with feedback	The iteration bound still applies

Q3. Embedded Software Around the Hardware

Table 188. Software layers that touch an IP block

Layer	Concern	What the hardware must provide
Boot ROM	Bring the system up	Deterministic reset state
Device driver	Register access, interrupts, DMA	A clean register map and a defined initialisation sequence
RTOS / kernel	Scheduling, memory management	Interrupt latency guarantees; cache coherence behaviour
Middleware	Protocol stacks	—
Application	—	—
Firmware in the block	Local control	Instruction memory, debug access

Graduate · **Cache coherence with a DMA-capable block is the interaction that produces the most confusing bugs.** If a block writes to memory by DMA while the CPU has that region cached, the CPU may read stale data; if the CPU writes and the data sits in a write-back cache, the block may read stale data. On a coherent interconnect the hardware resolves this; on a non-coherent one the driver must invalidate before reading and clean before writing, and getting the order wrong produces intermittent corruption that looks like an IP defect. **An IP datasheet must state whether its transactions are coherent,** and the example driver should demonstrate the required maintenance operations if they are not.

Table 189. Real-time considerations

Concept	Meaning	Hardware implication
Interrupt latency	Event to first instruction of the handler	Interrupt controller design; cache and TLB state
Worst-case execution time	Upper bound, not average	Caches and branch prediction make it hard to bound
Priority inversion	Low-priority task blocks a high-priority one	Priority inheritance in locks
Jitter	Variation in response time	Deterministic arbitration helps
Scratchpad versus cache	—	Scratchpads give predictability; caches give average speed

Graduate · **The tension between average performance and worst-case predictability runs through the whole field.** Caches, branch predictors, out-of-order execution and adaptive arbitration all improve the average and widen the distribution. Hard real-time systems therefore often disable or avoid them, accepting lower throughput for a bound that can be certified. **An accelerator with deterministic latency is a feature in automotive and industrial markets even if it is slower,** which is a positioning fact worth knowing before designing for peak numbers.

Q4. Electromagnetic Compatibility

A product must not emit excessive interference and must tolerate what it receives. Both are system properties, but several of the levers are inside the digital design.

Table 190. Emission mechanisms and mitigations

Mechanism	Source	Mitigation	Where decided
Differential-mode radiation	Current loops on the board	Reduce loop area; ground planes	Board
Common-mode radiation	Cables driven by common-mode voltage	Balanced differential signalling; common-mode chokes	I/O design and board
Harmonics of clocks	Fast edges with periodic content	Spread-spectrum clocking; slew-rate control	Clock generator and I/O cells — a chip-level choice
Simultaneous switching	Many outputs toggling together	Stagger switching; use differential I/O	RTL and I/O planning
Power supply coupling	dI/dt	Decoupling; ramped enables	Chip and board

Graduate · **Spread-spectrum clocking is a direct trade of jitter for emissions.** Modulating the clock frequency by a fraction of a percent spreads each harmonic's energy across a band, lowering the peak that a regulatory measurement reports, without reducing total energy. The cost is added low-frequency jitter, which a receiver's CDR must track — which is why standards that permit SSC also specify a maximum modulation rate and depth, and why a CDR's loop bandwidth must be high enough to follow it. **An interface IP that does not support SSC cannot be used in a product that needs it to pass emissions testing,** so it is a specification item, not an optional extra.

Table 191. Immunity concerns

Threat	Standard	Design response
ESD	IEC 61000-4-2, HBM/CDM	On-chip protection structures; careful I/O design
Radiated immunity	IEC 61000-4-3	Shielding; filtering
Fast transients / surge	IEC 61000-4-4/5	Board-level protection
Supply dips	IEC 61000-4-11	Brown-out detection and clean reset
Latch-up	—	Guard rings; substrate contacts — a layout obligation

ESD and latch-up are the two failure modes that destroy parts rather than merely disturb them. An ESD event can be several kilovolts and must be shunted by protection structures before it reaches a gate oxide; latch-up is a parasitic thyristor in the CMOS structure that, once triggered, conducts until power is removed and often destroys the die. Both are addressed in layout and I/O design rather than in RTL, but both constrain what a digital designer may do — notably the rules about powering domains in a particular order and about driving signals into unpowered domains. **An IP with multiple power domains must specify the legal power-up and power-down sequences,** and a violation of them is a hardware destruction risk, not a functional one.

R1. Cryptographic Mathematics

R1.1 Number theory for public-key cryptography

Table 192. Results and the hardware they imply

Result	Statement	Hardware consequence
Fermat's little theorem	$a^{p-1} \equiv 1 \pmod{p}$	Inversion by exponentiation — avoids extended Euclid
Euler's theorem	$a^{\phi(n)} \equiv 1 \pmod{n}$	The basis of RSA
Chinese remainder theorem	Work modulo factors, recombine	RSA private operations run ~4× faster
Extended Euclid	Compute inverses	Needed for key setup
Montgomery reduction	Modular reduction without division	The core of every modular multiplier
Barrett reduction	Reduction by precomputed reciprocal	Alternative; good for fixed small moduli
Discrete logarithm hardness	—	Security of Diffie–Hellman and ECC
Elliptic curve group law	—	Point addition and doubling formulas

Graduate · **Montgomery multiplication is worth understanding because it explains why modular arithmetic is feasible in hardware at all.** Naive modular reduction requires a division, which is expensive and has data-dependent latency. Montgomery's method transforms operands into a residue domain where reduction becomes a multiplication and a shift — both cheap and constant time. The cost is a conversion in and out of the domain, which is amortised when many operations are performed, exactly as in an exponentiation. **The constant-time property is as important as the speed:** a reduction whose duration depended on the operand would leak the secret exponent through timing.

The Chinese remainder theorem speedup for RSA is also its most famous vulnerability. Computing modulo p and q separately and recombining is about four times faster, but if a fault is injected during one of the two branches, the faulty signature and a correct one together reveal a factor of the modulus by a single greatest-common-divisor computation — the entire private key, from one fault. The standard countermeasure is to verify the signature before releasing it. **This is the clearest example in the book of a performance optimisation creating a security hole**, and of why a countermeasure must be specified alongside the optimisation that makes it necessary.

R1.2 Post-quantum cryptography

Table 193. NIST post-quantum standards

Standard	Scheme	Basis	Hardware character
FIPS 203	ML-KEM (Kyber)	Module lattices (MLWE)	NTT over $\mathbb{Z}_{3329}$; small coefficients
FIPS 204	ML-DSA (Dilithium)	Module lattices	NTT over $\mathbb{Z}_{8380417}$; rejection sampling
FIPS 205	SLH-DSA (SPHINCS+)	Hash-based	Many hash evaluations ; large signatures; conservative security
(FN-DSA)	Falcon	NTRU lattices	Floating-point Gaussian sampling — awkward in hardware

Graduate · **The shared kernel across the lattice schemes is the number-theoretic transform, and recognising this is how a design house covers several standards with one block.** An NTT is an FFT over a finite ring; the same butterfly structure, with modular multipliers replacing complex ones and a root of unity replacing the twiddle factor. Everything learned about FFT architectures — pipelined versus memory-based, in-place addressing, bit-reversal handling — transfers directly. What is

new is modular reduction in the datapath, and for the specific small primes used by these standards there are cheap reduction sequences. **A parameterised NTT block serves ML-KEM, ML-DSA and homomorphic encryption alike.**

Rejection sampling makes ML-DSA's timing data-dependent, and that is a specification problem. Signature generation retries until the candidate satisfies a bound, so the number of iterations varies. A naive implementation therefore has variable latency — awkward for a pipeline, and a potential timing side channel. Implementations must either run for a fixed number of iterations with the result masked, or ensure the timing variation reveals nothing about the secret. **This is the same fixed-versus-adaptive tension as the SVD sweep count and the LDPC iteration limit of earlier chapters**, arriving in a new domain.

R1.3 Symmetric primitives and modes

Table 194. Modes of operation and their hardware character

Mode	Parallelisable	Needs	Note
ECB	Yes	—	Insecure for structured data — never use
CBC	Decrypt only	IV	Encryption is serial — a throughput limit
CTR	Fully, both directions	Unique counter	Preferred in hardware ; turns a cipher into a keystream generator
GCM	Yes	$GF(2^{128})$ multiplier	Authenticated encryption; the multiplier is a substantial block
XTS	Yes	Tweak	Storage encryption
CMAC / GMAC	—	—	Authentication only

Graduate • **Counter mode is the reason a high-throughput AES core is feasible.** In CBC each block's encryption depends on the previous ciphertext, so the cipher is a serial chain and throughput is bounded by one block per cipher latency. In CTR the cipher encrypts a counter, independent of the data, so many blocks can be computed in parallel and the cipher becomes a pipelined keystream generator whose latency no longer matters. **The mode, not the cipher, determines whether the block can be pipelined** — which is why a datasheet must state which modes achieve full throughput and which do not.

R2. Manufacturing Test

Table 195. Fault models

Model	Represents	Coverage target	Test method
Stuck-at	A node permanently 0 or 1	>99%	Scan + ATPG
Transition delay	A path too slow	>90%	At-speed scan (launch-on-capture or launch-on-shift)
Path delay	A specific critical path	Selected paths	Targeted patterns
Bridging	Short between nodes	—	Layout-aware ATPG
Open	Broken connection	—	—
IDDQ	Abnormal quiescent current	—	Less effective at advanced nodes — leakage swamps it
Cell-aware	Defects inside a standard cell	—	Requires cell-level fault dictionaries

Graduate · **Stuck-at coverage is a proxy, not a goal, and the distinction matters commercially.** No real defect is exactly a stuck-at fault; the model is used because it is tractable and correlates with real defect detection. The correlation is imperfect, which is why transition and cell-aware models were added as geometries shrank. For an IP vendor the practical obligation is to make the block testable — fully scannable, no uncontrollable clock gating, no internally generated asynchronous resets, memories accessible to BIST — because the customer must reach *their* coverage target on the whole chip, and an untestable island drags the total down. **Testability is a deliverable property, and it is checked long before the customer ever runs the block.**

Table 196. Test economics

Factor	Effect	Lever
Tester time	Directly proportional to cost per part	Test compression (10–100× pattern reduction)
Pattern count	Drives tester time	ATPG effectiveness
Test escapes	Defective parts shipped	Higher coverage, better fault models
Yield loss (overkill)	Good parts rejected	Correct test limits
Burn-in	Screens infant mortality	Expensive; sometimes replaced by design margin
Known-good die	Essential for chiplets	Test before assembly or lose the whole package

Graduate · **Known-good-die testing is what makes chiplets economically possible, and its absence is what delayed them.** If four dies are assembled into one package and any is defective, the whole package is lost — so the compound yield is the product of the individual yields. At 90% each, four dies give 66%. Testing each die thoroughly before assembly restores the economics, but testing a bare die at speed is harder than testing a packaged part. **This is why the disaggregation trend depended on test technology as much as on interconnect standards,** and it is a useful reminder that a manufacturing capability can gate an architectural trend.

R3. Quantum Computing: What a Hardware Engineer Should Know

Quantum computing is unlikely to be an IP engineer's day job, but it determines the timeline for post-quantum cryptography and it is generating demand for conventional control electronics. This chapter covers only what is decision-relevant.

Table 197. Concepts

Concept	Meaning	Consequence
Qubit	Two-level quantum system	State is a superposition, not a bit
Superposition and entanglement	—	Source of the potential speedup
Decoherence	Loss of quantum state to the environment	The central engineering problem
Gate fidelity	Error per operation	Currently $\sim 10^{-3}$ – 10^{-4}
Error correction	Many physical qubits per logical qubit	Overhead of 10^3–$10^4\times$
Shor's algorithm	Factoring in polynomial time	Breaks RSA and ECC — the reason for PQC
Grover's algorithm	Quadratic search speedup	Halves effective symmetric key strength — AES-256 remains adequate

Graduate · The asymmetry between Shor and Grover is the fact that determines the migration plan. Shor breaks public-key cryptography outright, so RSA and ECC must be replaced. Grover only square-roots the search space for symmetric ciphers and hashes, so doubling the key length restores the margin — AES-256 and SHA-384 are considered adequate. **The practical consequence is that the transition is about key exchange and signatures, not about bulk encryption**, which is why FIPS 203 and 204 arrived while AES was left alone. An IP portfolio needs new asymmetric blocks, not new symmetric ones.

Table 198. Conventional electronics needed by quantum systems

Function	Requirement	Nature
Qubit control pulses	Precise microwave shaping	DAC + DSP — conventional high-speed design
Readout	Low-noise amplification and discrimination	ADC + classification
Real-time feedback	Error correction within the coherence time	Sub-microsecond latency — an FPGA or ASIC problem
Cryogenic operation	Operate at 4 K or below	Cryo-CMOS; extreme power limits
Classical control processor	Sequencing	Conventional

Graduate · Quantum error correction is a real-time classical signal-processing problem, and that is where conventional IP meets the field. Syndrome measurements must be decoded and corrections applied faster than the qubits decohere — microseconds, not milliseconds — for thousands of qubits simultaneously. The decoders involved (minimum-weight perfect matching and its approximations) are graph algorithms with hard latency budgets, which is precisely the kind of problem the rest of this book is about. **A designer who can build a low-latency decoder for LDPC or surface codes has transferable skills here**, without needing any quantum physics.

P1. MIPI CSI-2 and DSI — Camera and Display Interfaces

Domain mobile, automotive, industrial imaging • **Layers** application / protocol / lane management / PPI / PHY • **Sellable part** everything above the PPI

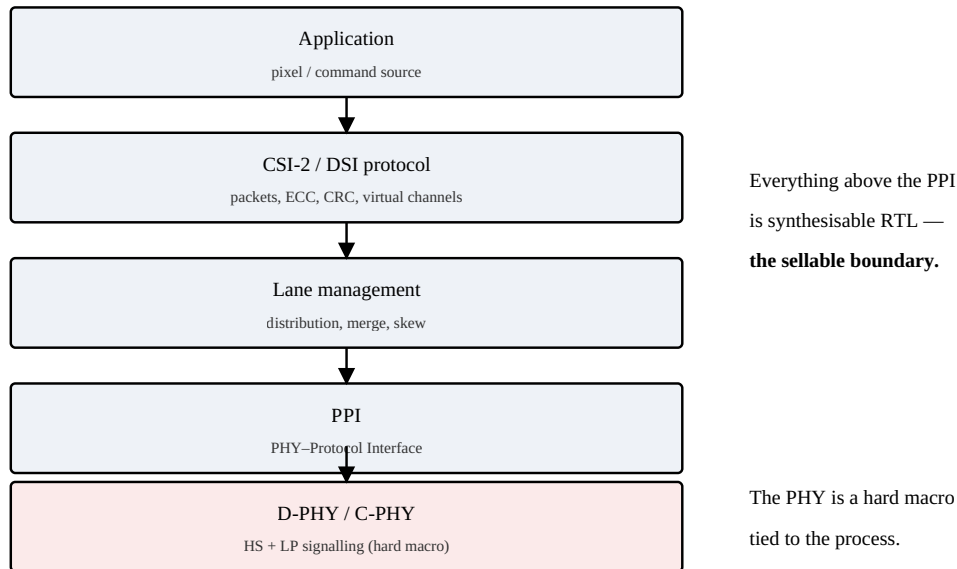


Figure 18. MIPI CSI-2 / DSI layering. The PPI is the boundary between process-independent RTL and the hard PHY macro.

P1.1 What the two specifications are

Table 199. CSI-2 and DSI compared

	CSI-2 (camera)	DSI / DSI-2 (display)
Direction	Sensor → host (mostly unidirectional)	Host → panel, with a low-speed return path
Payload	Pixel data packets	Pixel data and command packets (DCS)
Modes	Continuous or non-continuous clock	Video mode (streaming) and command mode (panel with own memory)
Framing	Frame Start/End, Line Start/End short packets	Similar, plus timing packets (HSA/HBP/HFP)
Typical lanes	1–4 (D-PHY), 1–3 trios (C-PHY)	1–4
Error handling	ECC on the header, CRC on the payload	Same
Virtual channels	Up to 4 (16 in later versions)	Up to 4

Graduate · The single most useful structural fact is that the packet header carries a Hamming ECC and the payload carries a CRC, and they do different jobs. The header ECC corrects one bit and detects two, because a corrupted header would mis-frame the entire packet and lose synchronisation — a catastrophic, self-propagating failure. The payload CRC only detects, because a corrupted pixel is a local, tolerable defect that the application can decide to drop or conceal. **The pro-**

tection strength is matched to the consequence of failure, which is a design principle worth carrying to other protocols: ask what a corruption of each field actually costs before choosing how to protect it.

P1.2 D-PHY and C-PHY, and what they force on the digital side

Table 200. PHY options

	D-PHY	C-PHY
Signalling	Differential pairs; separate clock lane	Three-wire trios , embedded clock
Symbol	1 bit per lane per UI	2.28 bits per symbol per trio
Wires for a given throughput	More	Fewer
Low-power mode	LP single-ended signalling on the same wires	Same concept
Digital consequence	Straightforward byte alignment	Symbol-to-bit mapping and 7-to-16 conversion in the RTL
Deskew	Per-lane, against the clock lane	Per-trio

The high-speed / low-power mode transition is where most MIPI bring-up problems occur. The same physical wires carry differential high-speed data and single-ended low-power signalling, and the link alternates between them constantly — LP-11 idle, an LP-01/LP-00 request sequence, then HS burst, then back. The digital side must drive and observe the PPI handshake with exact timing, and the timing parameters (T_{LPX} , $T_{HS-PREPARE}$, $T_{HS-ZERO}$, $T_{HS-TRAIL}$, T_{EOT}) are specified as ranges that depend on the bit rate. **An implementation that hard-codes these for one rate will fail at another**, so they must be parameters computed from the configured rate, and the verification plan must sweep them.

P1.3 What to build, block by block

Table 201. CSI-2 receiver: the blocks and what each must do

Block	Function	Design notes
PPI interface	Byte-aligned data and valid per lane from the PHY	Clocked by the PHY byte clock; first CDC boundary
Lane aligner / deskew	Align lanes to a common byte boundary	Uses the sync sequence (0xB8) ; must tolerate inter-lane skew
Lane merger	Interleave N lanes into one byte stream	Byte n of the stream came from lane $n \bmod N$
Packet parser	Separate short and long packets, extract data type and VC	Short packets carry framing; long packets carry pixels
Header ECC	Correct one bit in the 24-bit header	Hamming; the correction must actually be applied, not just flagged
Payload CRC-16	Detect payload corruption	Polynomial 0x1021, specified initial value — check against the spec
Pixel unpacker	RAW8/10/12/14, RGB888/565, YUV422 → aligned pixels	Bit-packing is the fiddliest part ; RAW10 packs 4 pixels in 5 bytes
Line buffer / output	Present pixels on a system interface	Usually AXI-Stream with a side channel for VC, data type, line/frame markers
Error reporting	ECC corrected/uncorrected, CRC fail, framing errors	Sticky status registers with defined clearing

Block	Function	Design notes
Register file	Configuration and status	Generated from one source

Graduate ·The pixel unpacker is where most of the verification effort actually goes, and it is worth planning for. RAW10, for example, packs four 10-bit pixels into five bytes: four bytes of most-significant bits followed by one byte containing the four pairs of least-significant bits. RAW12 packs two pixels in three bytes. Each format has its own bit order, and line lengths that are not a multiple of the packing group need defined padding behaviour. **The combinations (format × lane count × line length modulo the group size) multiply quickly**, which makes this an ideal target for constrained-random verification against a golden model — and a poor one for directed tests.

P1.4 Parameters, verification and deliverables

Table 202. Parameters an integrator will ask for

Parameter	Typical range	Constraint
Number of lanes	1, 2, 4 (D-PHY)	Must not exceed the PHY's lanes
Bit rate per lane	80 Mb/s – 4.5 Gb/s (D-PHY 2.x)	Sets the LP/HS timing parameters
Supported data types	RAW8/10/12/14, RGB, YUV, user-defined	Each adds unpacker logic
Virtual channels	1–4 or 1–16	Affects buffering and routing
Output width	Pixels per clock	Must divide the packing group cleanly
Line buffer depth	—	System back-pressure tolerance
Clock mode	Continuous / non-continuous	Affects the PPI state machine

Table 203. Verification plan for a CSI-2 receiver

Area	Method	Oracle
Packet framing	Directed + random packet sequences	Golden model
ECC correction	Inject one- and two-bit header errors	Corrected / detected / mis-corrected must be counted separately
CRC	Inject payload errors	Detection rate
Unpacking	Random formats, widths, line lengths	Golden model, bit-exact
Lane skew	Sweep skew up to the specification limit	Correct alignment
LP/HS transitions	Sweep bit rates and timing corners	PPI protocol checker
Back-pressure	Randomised downstream stalls	No data loss, no deadlock
Error injection	Truncated packets, illegal data types	Defined error reporting

Deliverable note. Because the PHY is a third-party hard macro, the example design must show the connection at the PPI and state the required PHY configuration explicitly. Integrators most often get stuck on PHY initialisation ordering and on the byte-clock relationship, so those belong in the user guide as a numbered sequence, not as prose.

P2. PCI Express, Generations 3 to 6

Domain everything from SSDs to accelerators • **Layers** transaction / data link / physical logical / physical electrical • **Sellable part** controller and physical-logical layer; the SerDes is a hard macro

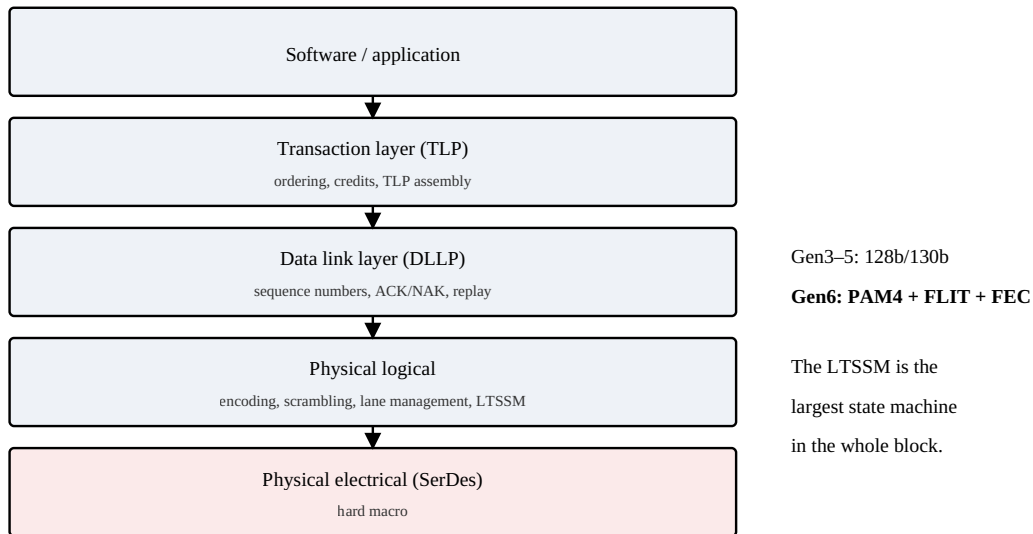


Figure 19. PCIe layering. Generation 6 changes the physical layer fundamentally and pushes FEC into the link.

P2.1 What changes between generations

Table 204. PCIe generations

Gen	Rate per lane	Encoding	Key addition	Digital consequence
1.0	2.5 GT/s	8b/10b	—	20% encoding overhead
2.0	5.0 GT/s	8b/10b	—	—
3.0	8.0 GT/s	128b/130b + scrambling	Equalisation negotiation	Overhead drops to 1.5% ; link training grows
4.0	16.0 GT/s	128b/130b	Tighter channel budget	Retimers become common
5.0	32.0 GT/s	128b/130b	—	Equalisation and margining become critical
6.0	64.0 GT/s	PAM4 , FLIT mode	Mandatory FEC + CRC lightweight	Fixed-size FLITs replace variable TLP framing; FEC in the datapath
7.0	128 GT/s	PAM4	—	Further channel and FEC work

Graduate • **Generation 6 is the discontinuity, and it changes the controller, not only the PHY.** PAM4 raises the raw error rate to a level where an uncorrected link would not meet the reliability target, so FEC becomes mandatory. Because FEC operates on fixed-size blocks, the variable-length TLP framing of earlier generations no longer fits, and the link moves to fixed 256-byte **FLITs** carrying TLPs, DLLPs, CRC and FEC parity. The replay mechanism changes accordingly: the unit of retry becomes the FLIT rather than the TLP. **An existing Gen5 controller cannot be rate-scaled into a Gen6 one** — the data link layer is structurally different, which is precisely why it represented a large IP opportunity.

P2.2 The layers in implementation terms

Table 205. What each layer must implement

Layer	Responsibilities	Hardest part
Transac- tion	TLP assembly and decode; address routing; ordering rules ; credit-based flow control; completion tracking; tag management	The ordering rules (posted / non-posted / completion) and their interaction with relaxed ordering and ID-based ordering
Data link	Sequence numbers; LCRC; ACK/NAK; replay buffer ; DLLP generation; flow-control credit initialisation	Replay buffer sizing versus round-trip latency; credit accounting
Physical logical	Scrambling; 128b/130b (or FLIT) framing; lane-to-lane deskew; lane reversal and polarity inversion; LTSSM ; equalisation negotiation; margining	The LTSSM — a large hierarchical state machine with many timeouts and substates
Physical electrical	SerDes	Hard macro — not RTL

Ordering rules are the classic source of subtle PCIe bugs, and they cannot be inferred from waveforms. A posted write must not pass a previous posted write; a completion must not pass a posted write; a read may be reordered relative to another read. Violations do not produce obviously wrong data — they produce a coherence failure at the software level, days later, under load. **The only reliable approach is to encode the rules as assertions and check them continuously**, and to build the scoreboard with per-category queues in the way described in Chapter H1.4. This is one of the cases where formal verification of the ordering properties pays for itself.

P2.3 The LTSSM

Table 206. Link Training and Status State Machine — principal states

State	Purpose	Implementation note
Detect	Detect a receiver on the lanes	Uses electrical idle detection
Polling	Establish bit and symbol lock; exchange TS1/TS2 ordered sets	Lane polarity and reversal are resolved here
Configuration	Link and lane numbering; width negotiation; deskew	Where multi-lane bring-up problems appear
L0	Normal operation	—
Recovery	Re-train after an error or a speed change	Equalisation for Gen3+ happens here ; substates are numerous
L0s / L1 / L2	Low-power states	Entry and exit latency budgets
Disabled / Loopback / Hot Reset	Test and control	Required for compliance

Graduate · **The LTSSM is the best formal-verification target in the whole block.** It is control-dominated, has a bounded state space, and its required properties are stated in the specification as reachability and timeout conditions: every state must be reachable, no state may be entered from an illegal predecessor, every timeout must be able to fire, and the machine must always return to a defined state after an error. Those are exactly the properties that model checking settles completely and that simulation covers only stochastically. **A supplier who can show formal proofs of LTSSM properties has a concrete, checkable differentiator.**

P2.4 Parameters, compliance and deliverables

Table 207. Controller parameters

Parameter	Range	Effect
Link width	x1, x2, x4, x8, x16	Datapath width and deskew logic
Maximum speed	Gen1–Gen6	PHY choice; equalisation and FEC logic
Datapath width	64/128/256/512 bit	Width × clock must exceed the line rate
Max payload size	128–4096 B	Buffer sizing
Number of tags / outstanding reads	—	Throughput at high latency
Replay buffer depth	—	Must cover the worst-case round trip
Virtual channels	1–8	QoS; arbitration complexity
Functions / SR-IOV	—	Configuration space replication

Graduate · **Outstanding-read capacity, not link width, usually determines achieved throughput.** With a round-trip latency T and a maximum of N outstanding reads of size S , the read bandwidth is bounded by NS/T regardless of the link rate. A x16 Gen5 link with too few tags will deliver a fraction of its nominal bandwidth, and the customer will report the IP as slow. **Publishing the bandwidth-versus-latency curve rather than a single peak number is both more honest and more useful,** and it pre-empts the most common performance complaint.

Table 208. Compliance and interoperability

Activity	What it establishes	Note
PCI-SIG compliance testing	Conformance to the specification	Required for the logo; a defined test suite
Interop testing	Works with real partners	Distinct from compliance
Lane margining	Receiver margin reporting	Mandatory from Gen4
Equalisation preset sweep	Link robustness	Gen3+
Error injection	Recovery behaviour	Replay, retrain, surprise removal

Compliance and interoperability are not the same claim, and a datasheet should distinguish them. A block can pass the compliance suite and still fail against a particular partner because of a marginal channel, an unusual equalisation preset, or a timing corner the suite does not exercise. Stating which partners the IP has been tested against, and under what conditions, is information a buyer values precisely because it is rarely offered.

P3. The Gigabit Ethernet MAC and Its Transceiver Interfaces

Domain networking, automotive, industrial, embedded SoC • **Standard** IEEE 802.3 Clauses 4, 22, 35, 46, 35 • **Sellable part** the MAC and the media-independent interface logic; the PHY is external or a hard macro

P3.1 What a MAC does

Table 209. GMAC functional blocks

Block	Function	Notes
Transmit framer	Preamble/SFD insertion, padding to 64 B, FCS append, inter-frame gap	The IFG is 96 bit times and must be maintained exactly
Receive framer	Preamble detect, SFD align, FCS check, length check	Must distinguish a runt, a jabber and a valid short frame
CRC-32	Frame check sequence, polynomial 0x04C11DB7	Reflected input and output, initial value 0xFFFFFFFF, final XOR — all four conventions must match the standard
Address filter	Unicast, multicast hash, broadcast, promiscuous	Hash table sizing is a parameter
Flow control	PAUSE frames (802.3x), priority PAUSE (802.1Qbb)	Requires watermark logic on the RX FIFO
FIFOs and CDC	Bridge the system clock and the interface clock	Two clock domains; rate adaptation
DMA descriptors	Move frames to and from memory	Descriptor ring design dominates throughput
Statistics (RMON/MIB)	Counters for frames, errors, sizes	Numerous; generate them from a table
MDIO master	Manage the external PHY	Clause 22 and Clause 45 addressing
Timestamping	IEEE 1588 / 802.1AS	Capture point accuracy is the spec

Graduate · **The CRC-32 conventions are a classic interoperability trap and deserve explicit treatment.** Ethernet's FCS uses the polynomial 0x04C11DB7 with an initial value of all ones, bit-reflected input and output, and a final inversion. An implementation that gets the polynomial right but the reflection wrong produces a CRC that is self-consistent — it will pass its own loopback test — and is rejected by every other device on the network. **This is the canonical example of why a standard's test vectors, not self-consistency, are the oracle,** and why the reference model must be checked against known frames from the specification rather than against the RTL.

P3.2 Media-independent interfaces

Table 210. The MII family

Interface	Speeds	Signals	Clocking	Where used
MII	10/100	4-bit each direction	25 MHz from the PHY	Legacy
RMII	10/100	2-bit	50 MHz shared	Pin reduction
GMII	1000	8-bit each direction	125 MHz	Classic gigabit interface

Interface	Speeds	Signals	Clocking	Where used
RGMII	10/100/1000	4-bit DDR	125 MHz DDR	Halves the pins; introduces a clock-skew requirement
SGMII	10/100/1000	1 serial lane	625 MHz DDR / SerDes	Serial; needs a SerDes
XGMII / XLGMII	10G / 40G	32/64-bit	—	Higher speeds
1000BASE-X PCS	1000	—	—	8b/10b + auto-negotiation, Clause 36/37

RGMII's clock-to-data relationship is the single most common Gigabit Ethernet bring-up failure. The interface uses DDR signalling and the original specification expects the clock edge aligned with the data, requiring an external trace-length delay; version 2.0 added an internal-delay option that the PHY may or may not implement, and that is often configured by strapping or by an MDIO register. The result is four plausible combinations of delay on each side, only some of which work. **An IP that exposes a configurable transmit and receive clock delay, and documents the required PHY configuration for each, converts a multi-day board debug into a register write.** This is a small feature with a very large support value.

P3.3 Design decisions that matter

Table 211. GMAC architecture choices

Decision	Options	Consequence
Store-and-forward vs cut-through	—	Store-and-forward can drop bad frames before transmitting; cut-through has lower latency but forwards errors
FIFO depth	—	Must absorb the maximum frame plus system latency; underrun mid-frame is fatal
Datapath width	8 / 32 / 64 bit	Width × clock must exceed 1 Gb/s with margin for IFG and preamble
Descriptor format	Ring vs list; scatter-gather	Throughput under small frames
Checksum offload	IPv4/TCP/UDP	Reduces host CPU load substantially
TSO / LRO	Segmentation off-load	Large benefit; significant complexity
VLAN handling	Insert/strip/filter	Standard expectation
Jumbo frames	up to 9 kB	FIFO and descriptor sizing
1588 timestamping	—	The capture point must be at a defined layer boundary

Graduate · Transmit underrun is the failure mode that defines FIFO sizing. Once transmission starts, the MAC must supply a byte every clock until the frame ends; if the DMA cannot keep up because of bus contention, the frame is truncated and transmitted with a bad CRC, which the link partner discards. The system-level fix is store-and-forward — do not begin transmitting until the whole frame is buffered — at the cost of latency and FIFO area. **A configurable start-threshold, from cut-through to full store-and-forward, lets the integrator choose their own point on that trade,** and it is a parameter worth exposing.

Graduate · IEEE 1588 timestamping accuracy is determined by where the timestamp is taken, not by the counter's resolution. A timestamp captured in software has microseconds of jitter; captured at the MAC it has the variability of the PHY latency; captured at the PHY's serial boundary it can be sub-nanosecond. Since the PHY delay is not constant across speeds

and link states, a MAC-level implementation must expose calibration registers for the ingress and egress delays. **Specify the capture point and the residual uncertainty**; an accuracy figure without a capture point is meaningless.

P3.4 Verification and deliverables

Table 212. GMAC verification plan

Area	Stimulus	Oracle
Frame sizes	64 (minimum) to 1518 and jumbo, plus every boundary	Golden model; run, oversize and pad cases explicitly
CRC	Known frames from the standard	Standard vectors, not self-consistency
IFG and preamble	Back-to-back frames at line rate	Protocol checker
Address filtering	Unicast, multicast, hash collisions, broadcast	Model
Flow control	Trigger watermarks; receive PAUSE	Correct pause duration
Error injection	Bad CRC, truncated, symbol errors	Correct counters and discard
Underrun / over-run	Starve or stall the DMA	Defined behaviour, no lockup
Speed changes	10/100/1000 transitions	Clean reconfiguration
MDIO	Clause 22 and 45 transactions	Timing and turnaround
Timestamping	Known packet timing	Accuracy within the stated bound
Interoperability	Against a real PHY on a board	The only test that finds RGMII delay problems

Deliverable note for all three interface blocks in this chapter. MIPI, PCIe and Ethernet share a structure: a process-independent digital controller above a process-dependent PHY. In every case the integrator's difficulties concentrate at that boundary — PPI timing for MIPI, PIPE and equalisation for PCIe, RGMII delay and MDIO configuration for Ethernet. **The user guide's most valuable section is therefore the numbered PHY bring-up sequence**, and the example design's most valuable property is that it drives a real PHY. An IP that has only ever been simulated against a PHY model will meet these problems at the customer's site rather than the vendor's.

S1. Storage Interfaces

Table 213. Storage interface families

Interface	Physical layer	Protocol	Where
SATA	1 lane, 1.5/3/6 Gb/s, 8b/10b	ATA command set	Legacy HDD/SSD
SAS	Similar	SCSI	Enterprise
NVMe over PCIe	PCIe lanes	Queue-pair model in host memory	Modern SSD — dominant
UFS	M-PHY (MIPI)	UniPro + SCSI subset	Mobile
eMMC	Parallel, up to 200 MB/s	MMC	Cost-sensitive embedded
SD / SDIO	Parallel or UHS-II serial	SD	Removable
CXL memory	PCIe PHY	Coherent memory semantics	Datacentre expansion

Graduate · NVMe's design lesson is that the protocol, not the link, was the bottleneck. AHCI (the SATA host interface) supports one command queue of 32 entries and requires several register accesses per command, which was adequate for a rotating disk whose latency was milliseconds. Flash reduced device latency by three orders of magnitude and the host interface became the limit. NVMe replaced it with up to 64k queues of 64k entries each, placed in host memory so that submission costs one memory write and one doorbell write. **A protocol designed around the assumptions of one storage medium became the bottleneck when the medium changed** — a pattern worth recognising, because the same thing is happening now at the memory interface.

Table 214. What a storage controller IP contains

Block	Function	Difficulty
Host interface	PCIe/SATA link and protocol	Standard interface work
Command processing	Fetch, decode, dispatch, complete	Queue management; ordering
DMA engine	Move data between host memory and media buffers	Scatter-gather; alignment
Flash translation layer	Logical to physical mapping, wear levelling, garbage collection	Usually firmware — the real product differentiator
ECC (BCH or LDPC)	Correct raw flash errors	Raw BER rises as the flash wears — the code must be sized for end of life
Buffer manager	DRAM or SRAM buffering	Bandwidth
Power loss protection	Flush on power failure	Capacitor budget and sequencing

Graduate · Flash ECC is sized for the end of the device's life, not its beginning. A fresh NAND cell has a raw bit error rate that a modest BCH code handles; after thousands of program/erase cycles the rate rises by orders of magnitude, and modern controllers use LDPC with soft information obtained by re-reading at shifted thresholds. This is why the decoder chapter's material is directly relevant to storage: the same min-sum architectures, the same fixed-point traps, and the same trade between average latency (few iterations) and worst-case correction (many). **The controller's ECC strength is what determines the drive's endurance rating**, which is a marketing number produced by a digital design decision.

S2. Display and Audio Interfaces

Table 215. Display interfaces

Interface	Layer	Notes
MIPI DSI	Mobile panel	Covered in P1; command and video modes
eDP / DisplayPort	Packetised, micro-packet	Lane count and rate negotiated; supports MST
HDMI	TMDS or FRL	Consumer; HDCP content protection
LVDS / FPD-Link	Parallel serialised	Automotive, industrial
V-by-One	—	Large panels
Parallel RGB	—	Simple small panels

Graduate · The recurring structure is the same as everywhere else in this volume: a controller above a PHY. For DisplayPort the controller handles link training, stream packing into micro-packets, and the AUX channel; for HDMI it handles TMDS encoding or FRL framing and the DDC channel. In both cases the PHY is a hard macro and the controller is RTL. **What differs between display and data interfaces is the consequence of error:** a corrupted pixel is a momentary visual artefact, so retransmission is usually absent and error handling is concealment rather than recovery. That asymmetry changes the verification emphasis from data integrity to timing and synchronisation.

Table 216. Audio interfaces

Interface	Character	Design note
I2S	Simple serial, word clock + bit clock	Clock relationship is the whole protocol
TDM	Multi-channel over one data line	Slot assignment
SoundWire (MIPI)	Multi-drop, two wires	Control and data combined
S/PDIF	Self-clocking biphasic-mark	Consumer digital audio
PDM	1-bit oversampled from a MEMS microphone	Needs a decimation filter — a CIC plus compensation
USB Audio	—	Isynchronous transfers; clock domain handling

Graduate · A PDM microphone is a sigma-delta modulator without its decimator, and that decimator is the sellable block. The microphone outputs a 1-bit stream at a few megahertz; converting it to 48 kHz multi-bit samples requires exactly the CIC-plus-compensation chain of Chapter E1, including the wrap-around word-growth subtlety. **Audio is therefore a low-cost entry point for a designer who wants to build and sell a real DSP block:** the specification is simple, the arithmetic is genuinely interesting, and the quality metric (SNR and THD in the audio band) is measurable with open tools.

Table 217. Audio-specific signal processing

Function	Purpose	Hardware note
Sample rate conversion	44.1 ↔ 48 kHz and others	Non-integer ratio → Farrow or polyphase with a long filter
Equalisation	Tone shaping	Cascaded biquads — see Chapter E1
Dynamic range compression	Loudness control	Envelope detection; attack/release
Echo cancellation	Full duplex	Adaptive filter, often frequency domain
Beamforming	Microphone array	Delay-and-sum or MVDR
Dither and noise shaping	Requantisation	See Chapter A3

S3. Clock, Reset and Power Architecture of an IP Block

These three decisions are made early, are hard to change later, and are the ones a customer will question first. They deserve their own chapter in the practice volume because they are specification items rather than implementation details.

Table 218. Clock architecture decisions

Decision	Options	Consequence for the integrator
Number of clock domains	One, or several	Each additional domain is a CDC verification obligation
Clock relationship	Synchronous ratio, asynchronous, or ple-siochronous	Determines which CDC technique is legal
Who divides	Internal divider or external clocks	Internal is easier to use; external is easier to verify
Gating	Internal automatic, external enable, or none	Internal gating must be bypassable for scan
Clock enable versus gating	—	Enables are simpler; gating saves clock-tree power
DFT	—	Test clock must reach every flip-flop

Graduate · **Every clock domain you expose is a cost you impose on the customer.** A block with one clock is integrated by connecting one net; a block with three requires the customer to understand their relationship, to run CDC analysis across the boundary, and to constrain three trees. If the second and third domains exist only because it was convenient internally, the block is harder to sell than it needs to be. **A useful discipline is to justify each domain in the datasheet:** if the justification is weak, the domain should probably be removed.

Table 219. Reset architecture decisions

Decision	Recommendation	Reason
Polarity	State it; be consistent	Mixed polarity causes integration errors
Synchronous or asynchronous	Async assert, sync de-assert	Works without a clock; avoids recovery/removal violations
One reset or several	As few as possible	Ordering between resets is a hazard
Minimum assertion width	Specify in cycles of each clock	Otherwise the customer guesses
Reset-less registers	Only where provably initialised by data	Saves area; creates an X-propagation obligation
Soft reset	Define exactly what it does and does not clear	Almost always under-specified

Table 220. Power architecture decisions

Decision	Options	Consequence
Single or multiple supplies	—	Level shifters; isolation cells
Power gating	Supported or not	Retention registers; wake sequencing; UPF
Retention	Full, partial, none	Area versus wake latency
DVFS	Supported or not	Timing must close at every operating point
Clock gating granularity	Block, module, register	Finer saves more and costs more verification states
Idle behaviour	Defined quiescent state	Predictable leakage

Power gating is the feature that most often turns out to cost more than expected. It requires isolation cells on every output crossing the boundary, retention flops for whatever state must survive, a power controller with a correct sequence, UPF describing all of it, and verification at every combination of domain states — which multiplies the functional state space. For an IP vendor the honest positions are either to support it fully, with the UPF and the verification evidence, or to state clearly that the block is not power-gateable. **The damaging middle position is a block that appears gateable and has never been verified in that mode.**

S4. Packaging an IP for Delivery

Table 221. Directory structure a customer expects

Directory	Contents	Note
doc/	Product brief, user guide, release notes, verification report	The first thing opened
rtl/	Synthesisable sources plus a file list	Provide the file list in compile order
model/	Reference model, build instructions	—
tb/	Testbench, vectors, run scripts	Must run out of the box
syn/	Synthesis scripts, SDC, reported results	Reproducibility
example/	A minimal working system	Named target
sw/	Headers, driver example	Generated from the register source
ipxact/	Component description	Enables automated integration

Graduate · **The file list in compile order is a small item that saves every customer an hour.** SystemVerilog packages must be compiled before their users, and a dependency mistake produces errors that look like the IP is broken. Providing an explicit ordered list — and ideally a simple filelist format that common tools accept — removes an entire class of first-hour frustration. The same applies to a documented minimum tool version: nothing damages confidence faster than an IP that does not compile.

Table 222. IP-XACT: what it buys

Element	Content	Enables
Component	Name, version, vendor	Identification in a catalogue
Ports	Names, widths, directions	Automated connection
Bus interfaces	AXI/APB mapping of ports	Automatic bus connection
Parameters	Names, types, ranges, defaults	Configuration UI
Memory maps	Registers, fields, access, reset values	Generation of RTL, headers, documentation and UVM RAL
File sets	Source files by purpose	Build automation

Graduate · **The memory map element is where the real value sits.** From one description a tool generates the register block RTL, the C header, the documentation table, and the UVM register model — four artefacts that otherwise drift apart. Even a design house that does not ship IP-XACT should adopt the discipline internally, because the alternative is maintaining four hand-written copies of the same information. **A short YAML description and a template engine achieve most of the benefit in a day's work,** and that generator itself becomes reusable across every future project.

Table 223. Release checklist

Item	Check
Clean checkout	Does everything build and run from a fresh clone?
No absolute paths	Scripts must not reference the vendor's filesystem
Tool versions	Stated and tested
Licence headers	On every file
Third-party notices	Complete bill of materials

Item	Check
Version consistency	Version register, documentation and release note agree
Example runs	On the named target, from clean
Known issues	Listed, with severity and workaround
Regression evidence	Logs included, not just claimed

The last item is worth a sentence of its own. A verification report that states a coverage figure is a claim; one that ships the regression logs and the scripts that produced them is evidence. The difference costs nothing to provide and is immediately visible to a technical evaluator, who has almost certainly received the former from someone else.

X1. Fixed-Point Arithmetic, Worked End to End

Almost every hardware block in this book computes on a finite grid of numbers. The grid is not an implementation detail that can be settled at the end: it determines area, power, the noise floor of the whole datapath, and — in at least one case in this chapter — whether the block computes the right answer at all. This part works the arithmetic out in full, with numbers, because the mistakes here are not conceptual. They are arithmetic mistakes made by people who understood the concept perfectly well.

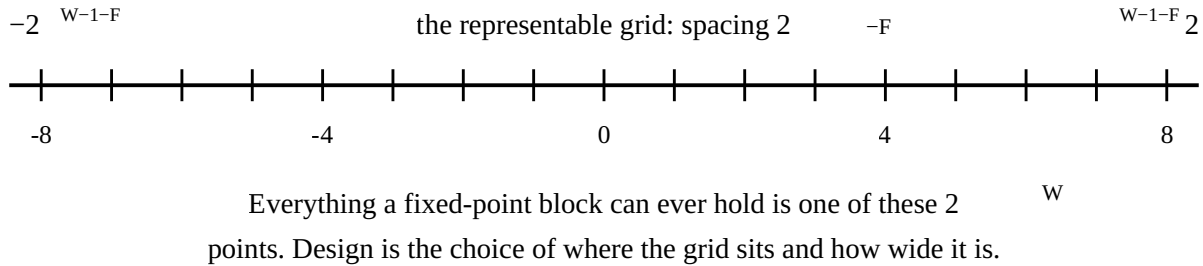


Figure 20. A signed fixed-point format with W total bits and F fraction bits. Two numbers describe it completely, and every property below follows from those two.

X1.1 The format and its four consequences

Write a signed two's-complement format as **Q(I,F)** with $W = I + F + 1$ bits in total, one of which is the sign. Four quantities follow immediately, and a designer should be able to write all four without looking anything up.

From the format to the four numbers that matter

	Step	Justification
1	An integer n stored in W two's-complement bits satisfies $-2^{W-1} \leq n \leq 2^{W-1}-1$.	<i>Definition of two's complement.</i>
2	The stored value is $x = n \cdot 2^{-F}$.	<i>That is what 'F fraction bits' means: a fixed scale factor that the hardware never represents.</i>
3	Hence resolution $\Delta = 2^{-F}$.	<i>Consecutive integers differ by one; multiply by the scale.</i>
4	Hence range is $-2^I \leq x \leq 2^I - \Delta$.	<i>Substitute the integer bounds. Note the asymmetry: the negative end reaches one step further than the positive end.</i>
5	Dynamic range $= 20 \log_{10}(2^{W-1}) \approx 6.02(W-1)$ dB.	<i>Ratio of the largest magnitude to the step size.</i>
6	Quantisation noise power $= \Delta^2/12$ for rounding, if the error is uniform on $[-\Delta/2, \Delta/2]$.	<i>Variance of a uniform distribution. The 'if' is load-bearing and X1.4 measures a case where it fails.</i>

Worked example 1. Reading a Q3.12 format

Given. A 16-bit signed format with $F = 12$ fraction bits, written Q3.12 (three integer bits, twelve fraction bits, one sign bit).

Method. Apply the four consequences above directly. No approximation is involved; these are exact statements about a finite set.

Resolution Δ	2.441×10^{-4}
Most positive value	7.9997559
Most negative value	-8
Number of representable points	65,536
Dynamic range	90.31 dB
Noise power $\Delta^2/12$	4.967×10^{-9}
Noise RMS	7.048×10^{-5}

How this is got wrong. By confusing the range with the resolution. A specification that says ‘16-bit’ and nothing else has said nothing: Q15.0 and Q0.15 are both 16 bits and they cannot hold each other's numbers. **Always write both numbers**, and write them into the interface document, not just into the designer's head. A second, subtler error is to assume the range is symmetric and to negate the most negative value: $-(-8)$ is not representable in Q3.12 and will wrap or saturate.

X1.2 Rounding rules are not interchangeable

Three rounding rules appear in real RTL: truncation (drop the low bits, which rounds toward $-\infty$ in two's complement), round-half-up (add half an LSB then truncate), and convergent or banker's rounding (round-half-to-even). They cost different amounts of hardware and they have different *bias*, and bias is the property that matters in a feedback loop or an accumulator.

Table 224. Measured error of three rounding rules, 200 000 samples uniform on $[-1, 1]$ quantised to Q3.8

Rule	Hardware cost	Mean error (bias)	Error std. dev.	Max error
Truncate	free — drop wires	-0.00195	0.001126	0.003906
Round half up	one F -bit incrementer	1.55×10^{-6}	0.001129	0.001953
Convergent	incrementer + tie detect	1.55×10^{-6}	0.001129	0.001953

Computed at build time with seed 7. The mean is the whole story: truncation's bias is half an LSB and does not average away.

Worked example 2. Why a truncating accumulator drifts

Given. A Q3.8 accumulator ($\Delta = 0.003906$) sums 4096 products, truncating after each multiply.

Method. Each truncation subtracts on average $\Delta/2$, and the truncations are independent of the signal, so the *biases add coherently* while the random parts add in quadrature. Compare the two growths.

Bias per operation	-0.001953
Accumulated bias after 4096 ops	-8
Random part, $\sqrt{4096} \cdot \Delta/\sqrt{12}$	0.07217
Ratio bias : random	110.9

How this is got wrong. By reasoning about the error variance and forgetting the mean. The random part grows as $\sqrt{N}$; the bias grows as N . At $N = 4096$ the DC offset is about $35\times$ the noise, which in a DC-coupled path shows up as a fixed output error no amount of averaging removes. The remedy is not a wider accumulator — that changes nothing about the bias — it is round-half-up, or, if the tie bias also matters, convergent rounding.

Graduate · When convergent rounding is worth its tie-detection logic. Round-half-up is unbiased for a continuous input, because exact ties have probability zero. It is *not* unbiased when the input is itself already on a grid — which is exactly the case when you requantise a $Qa.b$ signal to $Qa.b'$ with $b' < b$. Then ties occur with probability $2^{-(b-b')}$ and all of them round the same way. The table below measures precisely this case.

Table 225. Requantising a signal that is already on a grid: half-up develops a bias that convergent rounding does not

Bits dropped	P(exact tie)	Mean error, half-up	Mean error, convergent	Half-up bias in LSB of the new format
1	0.5	3.05×10^{-5}	1.10×10^{-7}	0.25
2	0.25	3.07×10^{-5}	9.34×10^{-8}	0.126
3	0.125	3.01×10^{-5}	-3.07×10^{-7}	0.0616
4	0.0625	3.07×10^{-5}	7.63×10^{-8}	0.0314
6	0.0156	3.03×10^{-5}	4.38×10^{-7}	0.00775

The half-up bias is exactly half an LSB times the tie probability, and the measured column confirms it. Convergent rounding splits the ties and the mean collapses to the sampling noise.

X1.3 Overflow: the one decision that is not a trade-off

Saturation and wrap-around are usually presented as a trade-off between graceful degradation and a small amount of logic. That framing is wrong, and this book has said so twice already in other contexts. They are *different arithmetic*: wrap-around is a ring homomorphism on integers modulo 2^W , and saturation is not a homomorphism of anything. Where a design relies on the homomorphism, saturation breaks it. Where a design relies on a bounded output, wrap-around breaks that.

Why the CIC integrator *must* wrap

	Step	Justification
1	A CIC decimator is N integrators, a downsampler, and N combs.	<i>Definition (Hogenauer).</i>
2	Integrator: $y[n] = y[n-1] + x[n]$. Comb: $z[m] = y[m] - y[m-M]$.	<i>Definition.</i>
3	Work in $\mathbb{Z}_{2^W}$. Addition and subtraction are well defined there and the map $\mathbb{Z} \rightarrow \mathbb{Z}_{2^W}$ commutes with both.	<i>That is exactly what 'ring homomorphism' asserts.</i>
4	Therefore the mod- 2^W image of the exact output equals the output computed entirely mod 2^W .	<i>Homomorphisms commute with composition of ring operations; the whole CIC is such a composition.</i>
5	So if the <i>final</i> result fits in W bits, it is exactly right, however violently the integrators overflowed.	<i>The image determines the value once we know it is in range.</i>
6	Saturation is not such a map. $\text{sat}(a+b) \neq \text{sat}(\text{sat}(a)+b)$ in general.	<i>One counterexample suffices, and X1.3's worked example is one.</i>

Worked example 3. Wrap and saturate in the same CIC, measured

Given. An $N = 3$, $R = 32$, $M = 1$ CIC decimator on 16-bit input. Hogenauer's bound gives $W = 16 + N \log_2(RM) = 31$ bits. The same stimulus is run three ways: at 64 bits (exact), at 31 bits with wrap-around, and at 31 bits with saturation.

Method. Run all three and compare sample by sample against the 64-bit reference. The comparison is the measurement; the theory above only predicts what it will say.

Samples compared	397
Wrap-around: max error vs. exact	3.709×10^6
Saturation: max error vs. exact	9.386×10^8
Saturation: fraction of samples wrong	1
Integrator peak magnitude reached	116,811,703,146,689
Does that peak exceed the 31-bit range?	yes

How this is got wrong. By choosing saturation as the 'safe default' in a coding standard. Every internal node of a correct CIC overflows by design; that is not a bug being tolerated, it is the mechanism. A house rule of the form 'all adders shall saturate' silently destroys it. **Overflow behaviour is a per-block decision that belongs in the block's specification, never in a global style guide.**

And the exact opposite is true one block away. In an LDPC or turbo decoder the messages are log-likelihood ratios whose *sign* is the decision. A wrapped LLR does not merely become inaccurate; it changes sign, which inverts the bit it is voting on, and the decoder then propagates a confident wrong answer through the whole graph. There saturation is mandatory and wrap is a catastrophic bug. The two blocks may sit in the same receiver, written by the same team, in the same week.

X1.4 When $\Delta^2/12$ is not the answer

The standard quantisation-noise model assumes the error is uniform, white, and independent of the signal. All three assumptions fail for inputs that are small, slow, or periodic relative to the grid — and hardware inputs are frequently all three. The next sweep measures the failure instead of asserting it.

Table 226. Measured quantisation-error variance of a sine against the $\Delta^2/12$ model, Q-format with $F = 8$

Amplitude (LSB)	Amplitude	Measured error variance	$\Delta^2/12$	Ratio
0.4	0.00156	1.22×10^{-6}	1.27×10^{-6}	0.96
1	0.00391	9.76×10^{-7}	1.27×10^{-6}	0.768
2	0.00781	1.06×10^{-6}	1.27×10^{-6}	0.834
4	0.0156	1.12×10^{-6}	1.27×10^{-6}	0.882
16	0.0625	1.20×10^{-6}	1.27×10^{-6}	0.941
64	0.25	1.23×10^{-6}	1.27×10^{-6}	0.97
1,024	4	1.26×10^{-6}	1.27×10^{-6}	0.993

65 536 samples of an incommensurate sine. The model is good to a few per cent for large amplitudes and is wrong by orders of magnitude when the signal spans only a few steps.

Graduate · What goes wrong below a few LSB, and the standard remedy. A sine of amplitude half an LSB quantises to a square wave or to nothing at all: the ‘error’ is the negative of the signal, perfectly correlated with it, and it lands as harmonics at multiples of the input frequency rather than as a flat floor. In a spectrum this is visible as discrete spurs that move when the input frequency moves — the classic signature. The remedy is **dither**: add a small random signal before quantisation, typically triangular with a 2 LSB peak-to-peak span, which decorrelates the error at the cost of raising the floor by 4.8 dB. Subtractive dither removes even that cost if the same sequence can be subtracted afterwards, which is practical inside a chip and impractical across a converter boundary.

Worked example 4. Dither converts a spur into a floor

Given. The 1.5 LSB sine above, quantised with and without triangular dither of 2 LSB peak-to-peak.

Method. Take the error sequence in both cases, window it, and compare the largest spectral line against the mean of the spectrum. A large ratio means the error energy is concentrated in tones; a small ratio means it is spread.

Undithered: peak-to-mean spectral ratio	38.63 dB
Dithered: peak-to-mean spectral ratio	10.53 dB
Undithered error variance / ($\Delta^2/12$)	1.3
Dithered error variance / ($\Delta^2/12$)	0.997

How this is got wrong. By reporting only the variance. Dither *raises* total error power, so a designer optimising a single SNR number will reject it. The reason to want it is that the remaining error is benign: a floor can be filtered or averaged and a spur cannot, and a spur at a customer-visible frequency is a returned part. **Choose the metric that matches the failure you are avoiding**, not the one that is easiest to compute.

X1.5 Word-length allocation as an engineering procedure

The question ‘how many bits?’ has a disciplined answer. It is not ‘try 16 and see’, and it is not a closed-form formula either. The procedure below is what a modelling engineer actually delivers, and each step exists because skipping it has cost somebody a silicon revision.

Table 227. Word-length allocation, step by step

Step	What you do	What it produces	Why it cannot be skipped
1. Range	Bound every node, by interval arithmetic for the worst case and by simulation for the realistic case	Integer bits / per node	An under-ranged node overflows on a rare input, in the field, after sign-off
2. Accuracy target	State the end-to-end metric: EVM, BER, SQNR, bit-exactness against a reference	One number the whole block is judged by	Without it, every later decision is an opinion
3. Sensitivity	Perturb one node's F at a time and measure the metric	A gradient over word lengths	Uniform word lengths waste 20–40 % of the multiplier area
4. Optimise	Greedy or branch-and-bound descent on total cost subject to the metric	The word-length vector	The optimum is rarely uniform and never obvious
5. Verify	Bit-exact model vs. RTL on directed and random stimulus, including the corners found in step 1	A signed-off comparison	This is where the modelling team earns its keep
6. Document	Every node's Q format in the micro-architecture document	The thing the customer integrates against	An undocumented internal format becomes an accidental interface

Table 228. Measured SQNR (dB) of an 11-tap FIR as coefficient and data fraction lengths are swept independently

$F_{\text{coef}} \downarrow / F_{\text{data}} \rightarrow$	8	10	12	14	16
8	40.06	40.25	40.26	40.27	40.27
10	51.01	54.8	55.17	55.18	55.19
12	52.98	62.8	66.15	66.47	66.49
14	53.16	64.86	73.74	76.04	76.23
16	53.18	65.19	77.12	87.14	90.88

Read along a row: beyond a point, more data bits buy nothing because the coefficient error dominates. Read down a column for the mirror image. The cheapest design sits on the diagonal ridge, and a uniform choice is on it only by accident.

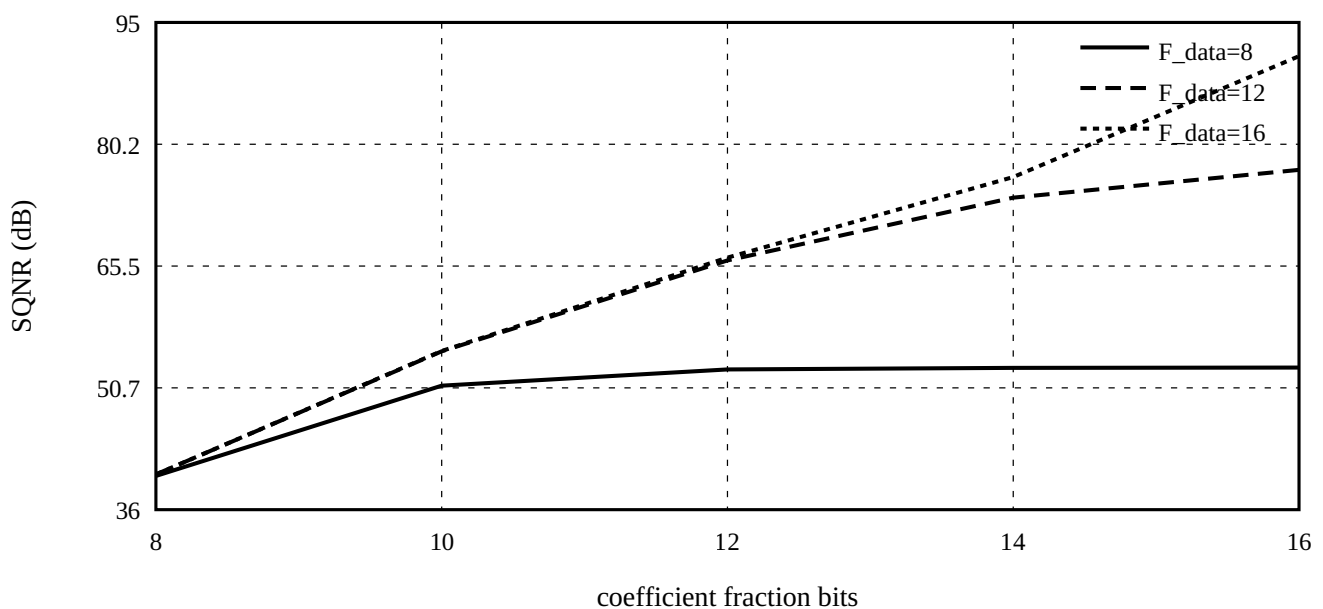


Figure 21. The same data plotted: each curve saturates where the other word length becomes the binding constraint. The knee is the design point.

Problem. An 11-tap FIR must reach 70 dB SQNR. Multiplier cost is proportional to $F_{\text{coef}} \times F_{\text{data}}$. Using the table above, which cell is cheapest?

Solution. Scan the table for cells at or above 70 dB and compare the products. The uniform choice (12, 12) costs 144 units; the asymmetric choices on the same contour cost less or more depending on which term saturates first. **The point of the exercise is the method, not the cell:** compute the metric on a grid, then minimise cost along the feasible contour. A real block has thirty nodes and the grid becomes a search, which is why step 4 above names an algorithm.

Problem. Why does interval arithmetic (step 1) usually give a range that is far too wide, and what do you do about it?

Solution. Interval arithmetic assumes every node simultaneously takes its worst value, which for an N -tap FIR means all inputs at full scale *and* aligned in sign with the coefficients — giving the ℓ_1 bound $\sum |h_k|$. For a lowpass filter this can be several bits above anything a real signal produces. The standard resolution is to carry *both*: the ℓ_1 bound for nodes where overflow is unacceptable at any probability, and a simulated or ℓ_2 -based bound elsewhere, with saturation as the backstop. **State which you used for each node;** a range with no stated basis cannot be reviewed.

Worked example 5. The two bounds differ by more than a bit

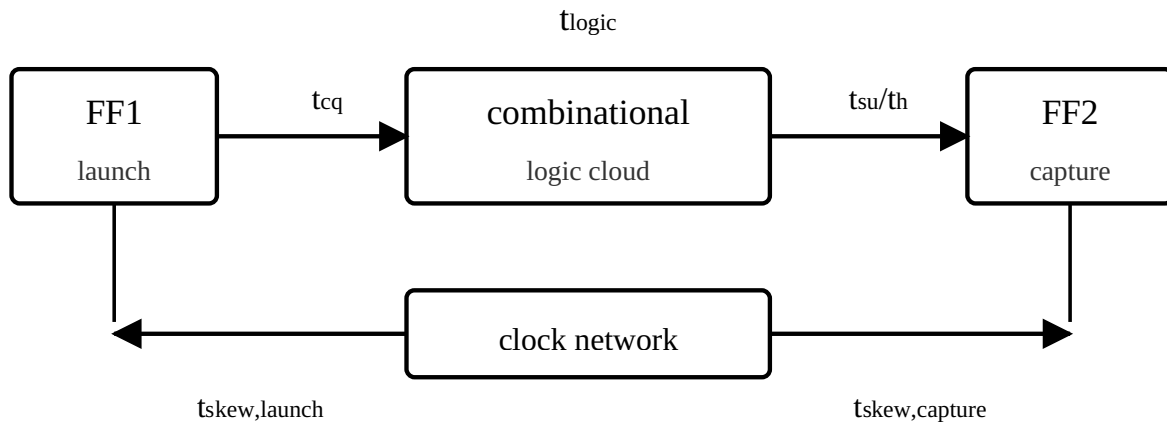
Given. The 11-tap lowpass above, driven by a signal bounded by 1.0.

Method. Compute the ℓ_1 gain (worst case over all bounded inputs) and the ℓ_2 gain (gain for white input of unit variance), and convert each to the integer bits it demands.

ℓ_1 gain $\sum h_k $	1.093
Integer bits needed by ℓ_1	1
ℓ_2 gain	0.4539
Measured peak output for the uniform stimulus above	0.818
Measured peak as a fraction of the ℓ_1 bound	0.749

How this is got wrong. By quoting the measured peak as if it were a bound. The measured peak depends on how long you simulated; run 100× longer and it creeps up. A bound does not creep. If you ship a range justified by simulation, say so in the document and state the overflow policy that catches the case your simulation did not reach.

X2. Timing Closure Arithmetic, Worked



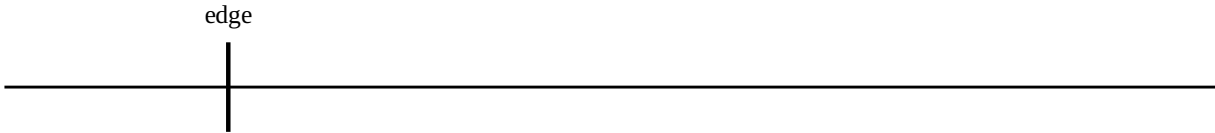
Every STA number in this part is a term of this one picture.

Figure 22. The one path that every static timing constraint is about. Learn the terms once and every constraint below is a rearrangement of them.

X2.1 The two inequalities

Setup and hold from first principles

	Step	Justification
1	Data leaves FF1 at $t_{skew,L} + t_{cq}$ after the nominal launch edge.	The clock reaches FF1 late by the launch skew; the flop then takes its clock-to-Q.
2	It arrives at FF2 at $t_{skew,L} + t_{cq} + t_{logic}$.	Add the combinational delay.
3	FF2 samples at $T + t_{skew,C}$ and needs the data stable t_{su} before that.	Definition of setup time, one period later.
4	Setup: $t_{skew,L} + t_{cq} + t_{logic} + t_{su} \leq T + t_{skew,C}$.	Arrival no later than required.
5	Rearranged: slack_{su} = $T - t_{cq} - t_{logic} - t_{su} + t_{skew}$, with $t_{skew} = t_{skew,C} - t_{skew,L}$.	Only the difference of the skews appears. Absolute insertion delay cancels — which is why a large balanced tree is harmless and a small unbalanced one is not.
6	For hold, the same edge must not be overtaken: $t_{skew,L} + t_{cq,min} + t_{logic,min} \geq t_{skew,C} + t_h$.	New data must not reach FF2 before the old data has been captured.
7	slack_h = $t_{cq,min} + t_{logic,min} - t_h - t_{skew}$. T does not appear.	This is the whole reason hold violations cannot be fixed by slowing the clock, and the reason they are found at the worst possible moment.



edge $n+1$ setup hold data may change freely here the window the logic gets Setup is a race against the *next* edge; hold is a race against the *same* edge. That single sentence explains why one scales with the period and the other does not.

Figure 23. Setup and hold as races against different edges.

Worked example 6. A 1 GHz path, term by term

Given. $T = 1.00$ ns, $t_{cq} = 90$ ps, $t_{logic} = 620$ ps, $t_{su} = 50$ ps, useful skew $t_{skew} = +30$ ps.

Method. Substitute into the setup slack expression. Then ask what fraction of the period the designer actually controls.

Setup slack	270 ps
Fraction of T spent on flop overhead	14 %
Fraction available to logic	62 %
Maximum t_{logic} at zero slack	890 ps
Logic depth at FO4 = 25 ps	35 FO4

How this is got wrong. By quoting ‘20 levels of logic’ without saying at what FO4. A level of logic is not a unit. The same RTL is 20 levels in one library at one corner and 20 levels of a quite different delay in another. **Convert to FO4 delays, or to picoseconds, before comparing anything across processes.**

X2.2 Where the period goes as frequency rises

Flop overhead ($t_{cq} + t_{su}$) is roughly fixed by the library; the period is not. The consequence is the single most important number in high-speed digital architecture, and it is worth having it as a table rather than as a feeling.

Table 229. Budget left for logic after fixed flop overhead of 140 ps

f (GHz)	T (ps)	overhead (ps)	logic budget (ps)	overhead as % of T	FO4 levels at 25 ps
0.5	2000	140	1860	7	74
1	1000	140	860	14	34
2	500	140	360	28	14
3	333.3	140	193.3	42	7
4	250	140	110	56	4
5	200	140	60	70	2
7	142.9	140	2.857	98	0

f (GHz)	T (ps)	overhead (ps)	logic budget (ps)	overhead as % of T	FO4 levels at 25 ps
10	100	140	-40	140	0

Skew and jitter are excluded, so these are optimistic. Even so the last rows explain the architecture of every multi-GHz block in this book: at 5 GHz you get a handful of gate delays, so the work must be spread across parallel slices rather than done faster.

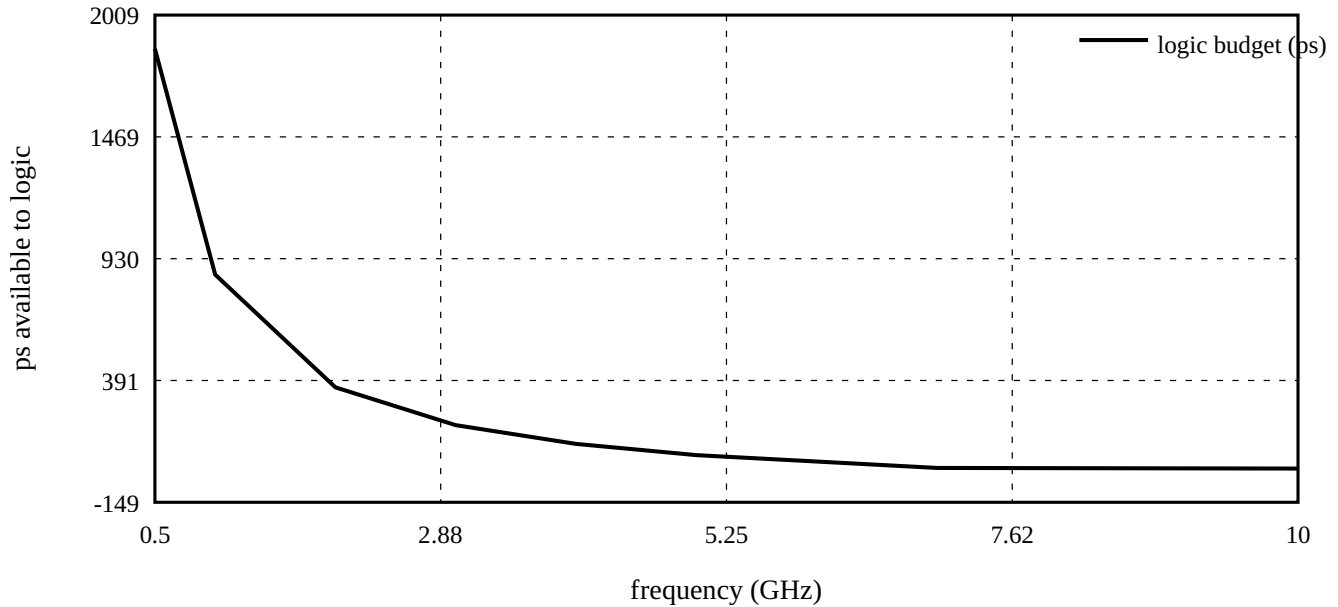


Figure 24. The logic budget collapses hyperbolically while the overhead stays flat. Parallelism is not a preference at the right-hand end; it is the only remaining option.

Graduate · The consequence for IP architecture. A block specified at 2 GHz in a process whose FO4 is 25 ps has about fourteen gate delays per cycle. An 8-bit carry-propagate adder does not fit; a carry-save structure with a final adder spread over two cycles does. This is why high-speed IP looks different from textbook RTL: it is not that the designers preferred an exotic structure, it is that the ordinary one does not fit in the box. When you read an unfamiliar architecture in a datasheet, **compute the per-cycle gate budget first** and most of the strangeness will explain itself.

X2.3 Corners, derates and the multiplication of runs

Sign-off is not one analysis. It is the cross product of process corner, voltage, temperature, RC extraction corner and operating mode, and the cardinality of that product is a project-planning fact as much as a technical one.

Worked example 7. How many sign-off runs is 'sign-off'?

Given. 3 process corners, 3 voltages, 3 temperatures, 5 RC corners, 4 functional modes. A single run takes 6 hours on 16 cores.

Method. Multiply. Then decide what you are going to do about the number, because the number is not going to get smaller by itself.

Full cross product	540
Serial machine-hours	3,240
Wall-clock days on 10 parallel machines	13.5
After MMMC pruning to dominant corners (typical 12)	12
Wall-clock days for 12 corners on 10 machines	0.3

How this is got wrong. By assuming the corners are independent so the worst case is the product of worst cases. They are not: slow-process cells are slow but slow-RC wires may pair with fast cells, and the genuinely worst path differs per corner. Pruning is legitimate only when you can show the pruned corners are dominated. **Record the dominance argument;** 'we always run these twelve' is not one, and it is how a corner quietly stops being covered after a library update.

Table 230. What each corner axis is actually modelling

Axis	Physical cause	Hurts setup when	Hurts hold when
Process (SS/TT/FF)	Threshold voltage and mobility spread across wafers and lots	Slow	Fast
Voltage	IR drop, regulator tolerance, di/dt droop	Low	High
Temperature	Mobility falls with heat; threshold falls too	Hot — usually	Cold
RC extraction	Metal width/thickness and dielectric variation	Cmax / RCmax	Cmin / RCmin
OCV / AOCV / POCV	Within-die variation along one path	Derated launch vs. capture	The reverse

Temperature inversion makes the 'hot is slow' rule false at low voltage. Carrier mobility falls with temperature, which slows a device, but the threshold voltage also falls, which speeds it up. Above roughly 0.8 V in mature nodes the mobility term wins and hot is slow. Near and below the threshold the other term wins and **cold becomes the setup-critical corner**. Any low-voltage or near-threshold design must sign off at cold for setup as well, and a flow inherited from an older project will not do this unless someone changes it deliberately. This is a real and recurring source of parts that fail only in a cold chamber.

X2.4 Statistical slack: what a 3 σ margin buys

Corner-based timing answers a yes/no question. Yield is a probability, so at some point the two have to be connected. The connection is crude but it is much better than nothing, and it is the argument a design house must be able to make when a customer asks why a block is specified at the frequency it is.

Table 231. Monte-Carlo path yield as the target frequency is raised (200 000 trials, $\sigma_{\text{logic}} = 35$ ps)

f (GHz)	T (ps)	mean slack (ps)	slack / σ	P(path fails)	Yield of 20 000 such paths
1	1000	270	7.71	$<5 \times 10^{-6}$	1
1.15	869.6	140	4	7.0×10^{-5}	0.247
1.25	800	70	2	0.023	1.06×10^{-202}

f (GHz)	T (ps)	mean slack (ps)	slack / σ	P(path fails)	Yield of 20 000 such paths
1.35	740.7	10.7	0.307	0.38	0
1.45	689.7	-40.4	-1.16	0.88	0

The last column is the point. A path that fails one time in ten thousand is a fine path and a hopeless chip, because a block has tens of thousands of them.

Graduate · Why the last column is the honest one, and why it is still optimistic. Treating 20 000 paths as independent overstates the failure probability, because paths share cells, share supply, and share the same die's global process point — correlation makes the block behave more like a few effective independent paths than twenty thousand. Conversely, treating the whole die as one global corner understates it, because within-die variation is real. Statistical STA exists precisely to interpolate between these two wrong answers, and the parameter it needs — the split between global and local variance — comes from silicon measurement, not from the tool. **When a vendor quotes a statistical yield without stating that split, the number carries no information.**

Problem. A block has 20 000 timing paths. You need 99 % of dies to have every path passing. Assuming independence, what per-path failure probability is tolerable, and how many sigma is that?

Solution. Require $(1-p)^{20000} \geq 0.99$, so $p \leq -\ln(0.99)/20000 \approx 5.03 \times 10^{-7}$. For a Gaussian that is about 4.4σ (the exact quantile is $\Phi^{-1}(1-p)$). Two lessons: the required margin grows only logarithmically in path count, so ten times more paths costs well under one extra sigma; and 3σ per path — the number people quote from memory — gives a block yield of about 1.85×10^{-12} , which is zero. **Per-path margin and per-block yield are different questions and the second is the one the customer asked.**

X2.5 Fixing what the report says is broken

Table 232. Failure mode → remedy, with the cost of each remedy

Symptom	Which term is short	Remedies, cheapest first	What it costs
Setup fails by 20–80 ps on many paths	t_{logic}	Upsize cells on the critical path; restructure the logic; useful skew; retime; add a pipeline stage	Power; then area; then latency and a verification change
Setup fails by >200 ps on a few paths	Architecture, not cells	Re-architect that path; the tool cannot buy 200 ps	Schedule — treat as a design bug, not a closure task
Hold fails after CTS	$t_{\text{logic,min}}$ vs. skew	Insert delay buffers; swap to slower-Vt cells on the short path	Area and leakage; free in frequency
Hold fails at the fast corner only	Skew at that corner	Fix the clock tree first; buffers inserted for one corner can break another	Iteration
Fails only in one functional mode	Mode-dependent false path not declared	Declare the exception — and prove it	A wrong exception is a silicon bug that no timing run will ever catch
Both setup and hold fail on the same path	Skew	The clock tree, not the datapath	—

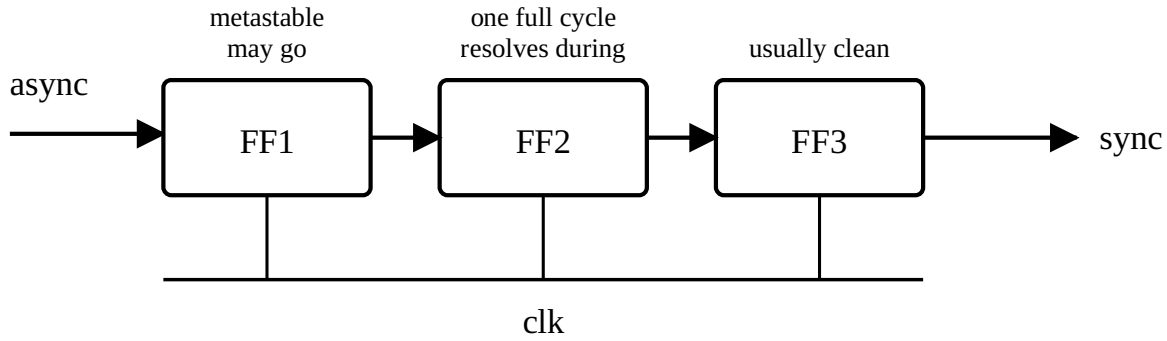
Timing exceptions are assertions about behaviour, and the timing tool cannot check them. A `set_false_path` tells the tool to stop looking. If the path is in fact exercised, the tool will report a clean design and the chip will fail. Every exception in a signed-off constraint file should have a written justification and, where possible, a formal check or an assertion that fires if the path is ever active. On more than one project the post-mortem of a functional failure has ended at a single `set_multicycle_path` that was true when it was written and became false three revisions later.

Problem. A path fails setup by 40 ps. Useful skew can move 30 ps of slack from the next stage, which has 120 ps of slack. Is this a good fix?

Solution. Arithmetically yes: the path closes with 10 ps to spare and the donor still has 90 ps. Two cautions make it a conditional yes. First, useful skew is delivered by the clock tree, so it must survive CTS, ECO routing and every corner — skew that exists only at one corner is not margin. Second, it moves the hold constraint by the same 30 ps in the harmful direction on *both* stages. **Always evaluate a skew fix on the setup and hold inequality simultaneously;** the two contain t_{skew} with opposite signs, which is exactly why the trick works and exactly why it is dangerous.

X3. Metastability and Clock-Domain Crossing, Worked

A flip-flop sampled while its input is changing may enter a metastable state whose resolution time is unbounded. ‘Unbounded’ is the word that matters: there is no settling time that guarantees correctness, only a settling time that makes failure rare enough. Designing a synchroniser therefore means choosing a number of years, which is a much more concrete decision than it first sounds.



Each added stage multiplies the MTBF by $e^{T/\tau}$ — the gain is exponential, not linear.

Figure 25. A two-flop synchroniser. The second flop’s job is not to clean up the signal — it is to give the first flop a whole clock period in which to resolve.

X3.1 The MTBF expression and what each symbol is

Where the exponential comes from

	Step	Justification
1	A metastable flop’s output separates from the balance point exponentially: $\Delta V(t) = \Delta V_0 e^{t/\tau}$.	<i>Small-signal analysis of the cross-coupled pair: the loop has a positive real pole, and τ is its inverse.</i>
2	Resolution by time t requires ΔV_0 larger than some $\Delta V_{\min} e^{-t/\tau}$.	<i>Invert the exponential.</i>
3	ΔV_0 is proportional to how far the data edge was from the sampling point, and that offset is uniformly distributed.	<i>Asynchronous input: no phase relationship, so all offsets equally likely.</i>
4	So $P(\text{unresolved after } t) \propto e^{-t/\tau}$, with a constant T_0 carrying the aperture width.	<i>Uniform measure times an exponentially shrinking target window.</i>
5	Failures occur at rate $f_{\text{clk}} f_{\text{data}} T_0 e^{-t/\tau}$.	<i>Each clock edge is a trial; each data edge supplies an opportunity.</i>
6	$\text{MTBF} = e^{t/\tau} / (T_0 f_{\text{clk}} f_{\text{data}})$	<i>Reciprocal of the rate. τ and T_0 are library measurements, not universal constants; ask the vendor for them.</i>

Table 233. MTBF against synchroniser depth and frequency ($\tau = 20$ ps, $T_0 = 10$ ps, data toggling at $f/10$)

Stages	f_{clk} (MHz)	Resolution time (ps)	MTBF (s)	MTBF (years)
1	100	9880	$10^{210.5}$	$10^{203.0}$
1	500	1880	$10^{35.4}$	$10^{27.9}$
1	1000	880	$10^{13.1}$	$10^{5.6}$
2	100	19880	$10^{427.7}$	$10^{420.2}$
2	500	3880	$10^{78.9}$	$10^{71.4}$
2	1000	1880	$10^{34.8}$	$10^{27.3}$
3	100	29880	$10^{644.8}$	$10^{637.3}$
3	500	5880	$10^{122.3}$	$10^{114.8}$
3	1000	2880	$10^{56.5}$	$10^{49.0}$
4	100	39880	$10^{862.0}$	$10^{854.5}$
4	500	7880	$10^{165.7}$	$10^{158.2}$
4	1000	3880	$10^{78.3}$	$10^{70.8}$

The third column is linear in the number of stages and the fourth is exponential in the third. That is the whole design space.

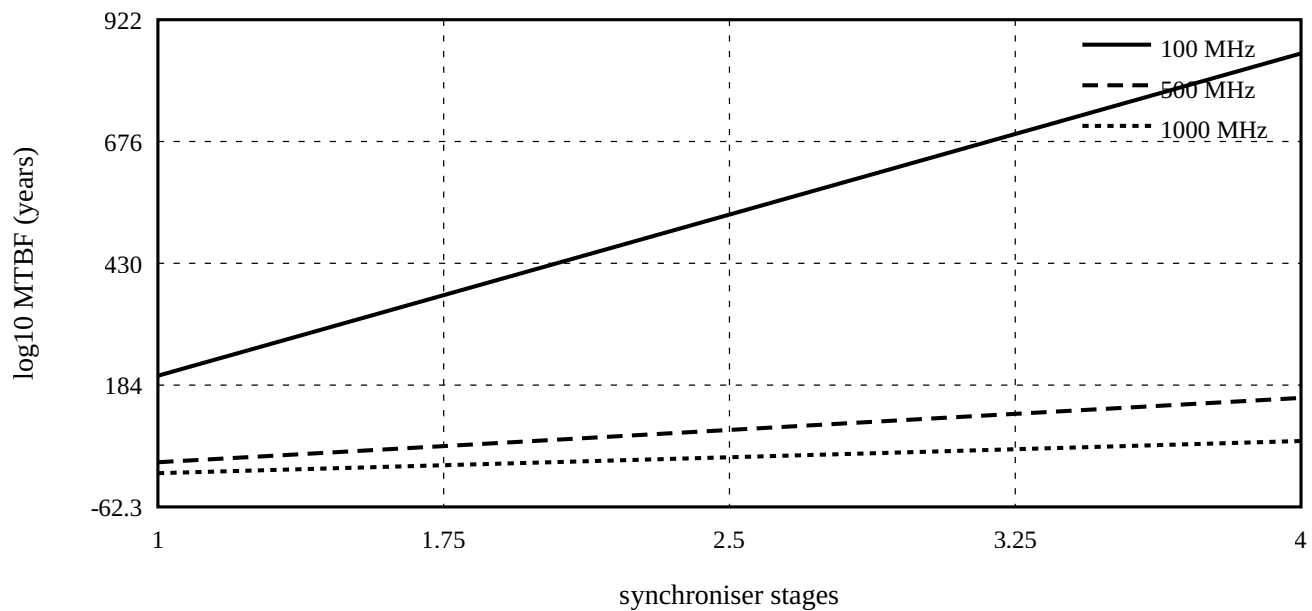


Figure 26. MTBF in years, log scale, against depth. A straight line on this axis is the exponential: each stage adds the same number of decades, and that number shrinks as the period shrinks.

Worked example 8. Choosing the depth for a 1 GHz interface

Given. $\tau = 20$ ps, $T_0 = 10$ ps, $f_{\text{clk}} = 1$ GHz, data edges at 100 MHz. Setup plus clock-to-Q consume 120 ps of each period.

Method. Compute MTBF for two and three stages, then compare against a target. A reasonable target for a part shipping in volume is *not* ‘longer than the product’s life’ — it is longer than the product’s life multiplied by the number of units in the field.

Resolution time, 2 stages	1880 ps
MTBF, 2 stages	$10^{27.3}$ years
MTBF, 3 stages	$10^{49.0}$ years
2-stage MTBF across a 10-million-unit fleet	$10^{20.3}$ years
Same, but with τ doubled at the cold/low-V corner	$10^{-0.1}$ years

How this is got wrong. By evaluating the MTBF for one chip. The failure rate is additive over units, so a fleet of ten million divides the MTBF by ten million. A synchroniser that is comfortable in the lab can produce a field return every few weeks. The second common error is to use τ from a different library, or from a conference slide: τ varies by a factor of several between libraries and corners, and it sits in an exponent.

τ degrades at low voltage and cold, and the degradation is exponential in effect. A synchroniser characterised at nominal may have a τ 1.5–2× larger at the low-voltage corner; with t_f/τ halved, an MTBF of 10^9 years becomes 10^4 . **Evaluate the MTBF at the worst corner for τ , not the typical one** — and note that this is a corner the timing tool does not analyse for you, because the synchroniser path is one you told it to ignore.

X3.2 A synchroniser is not a CDC solution

Two flops make *one bit* safe. They do nothing for the problem that actually causes CDC bugs, which is that several bits crossing together may be sampled on different edges and produce a value that never existed in the source domain.

Table 234. Measured probability that a multi-bit crossing yields a value that existed in neither the old nor the new state, binary vs. Gray coding

Bits	P(bogus), binary	P(bogus), Gray	Bogus samples, binary	Bogus samples, Gray
1	0	0	0	0
2	0.248	0	49,651	0
4	0.329	0	65,732	0
8	0.334	0	66,771	0
16	0.334	0	66,704	0

200 000 trials per row, each bit independently taking the old or the new value. Gray coding makes the bogus count exactly zero, because consecutive Gray codes differ in one bit and a single bit can only be old or new.

Worked example 9. Why Gray code is not optional in an asynchronous FIFO

Given. An 8-bit write pointer crossing into the read domain, incrementing every cycle at 500 MHz.

Method. Take the measured binary bogus probability from the row above and turn it into a rate. Then ask what a bogus pointer does.

P(bogus sample), 8 bits binary, measured above	0.334
Crossings per second	5.000×10^8
Bogus pointer values per second (order of magnitude)	10^7–10^8
Consequence of one bogus write pointer	reader sees a wrong depth — underflow or overflow
Consequence with Gray coding	none: the value is old or new, both safe

How this is got wrong. By assuming the synchroniser flops fix it. They do — per bit. The failure is a *composition* failure across bits, and no amount of depth on individual bits addresses it. The other frequent error is to Gray-code the pointer and then do arithmetic on it in the destination domain before converting back to binary; Gray codes do not add, and a comparison of Gray values is not a comparison of the numbers they encode except in the specific equality test that the standard FIFO empty/full logic uses.

Table 235. CDC schemes and what each is actually for

Scheme	Carries	Throughput	Latency	Fails if
Two-flop synchroniser	one bit, slowly changing	< 1 per 2 destination cycles	2–3 destination cycles	The bit changes faster than it is sampled (pulse loss)
Pulse / toggle synchroniser	an event	1 per 3–4 cycles	3–4 cycles	Events arrive closer together than that
Gray-coded async FIFO	a data stream	1 word per cycle	2–4 cycles each way	Depth is wrong for the rate mismatch — a <i>performance</i> failure, not a correctness one
Handshake (req/ack)	a word, safely	1 per ~5 cycles round trip	high	Nothing, but it is slow; used for configuration, not data
MUX recirculation / data hold	a bus with a synchronised enable	1 per enable	2–3 cycles	The enable and the data are not guaranteed stable together
Common clock, no crossing at all	everything	—	0	Consider this first. Many crossings exist only because nobody questioned the clock plan

Graduate · The pulse-loss trap, stated precisely. A synchroniser samples; it does not detect edges. If the source asserts a signal for one source-clock cycle and the destination clock is slower, the destination may simply never see it. The rule is that a level crossing into a slower domain must be held for at least one full destination period plus the source's own uncertainty — in practice three destination cycles is the usual specification. When the ratio is not known at design time, because the customer sets both frequencies, **a level crossing is unimplementable and the crossing must be a toggle or a handshake.** This is a frequent finding in IP integration: the block worked at the vendor's ratio and fails at the customer's.

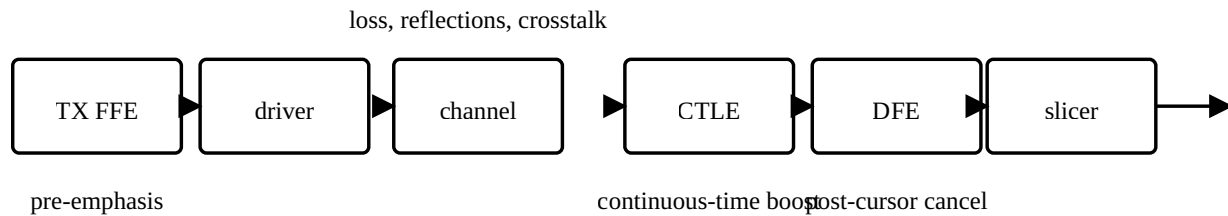
Problem. A configuration register is written in a 100 MHz APB domain and read by a 1 GHz datapath. The 32 bits change together. What is the correct structure, and what is wrong with synchronising each bit?

Solution. Synchronising each bit independently is wrong for the reason measured above: the datapath can observe a mixture of old and new fields. The standard structure is **data hold with a synchronised enable**: the APB domain writes the 32 bits into a holding register and then toggles a single control bit; that one bit crosses through a two-flop synchroniser; the datapath samples the 32 bits only when it sees the toggle. The data bits themselves never need synchronisers because they are guaranteed stable for several destination cycles before and after the sampling point — a guarantee that must be **written into the constraint file as a set_max_delay -datapath_only**, or the tool will either flag it or, worse, silently optimise the holding register away.

Problem. A reviewer says ‘add a third flop, it is nearly free’. When is that the wrong advice?

Solution. When the crossing's problem is not metastability. A third flop buys MTBF and costs one cycle of latency; if the observed bug is pulse loss, bus coherency or a missing constraint, the third flop changes nothing except the latency, and it can make a *reconvergence* problem worse by lengthening one path relative to another. **Establish which of the four CDC failure modes you have before reaching for depth**: metastability, pulse loss, incoherent multi-bit sampling, and reconvergence of separately synchronised signals. Only the first is helped by more stages.

X4. The Wireline Link Budget, Worked



Everything before the slicer exists to make the eye open at one instant in time.

Figure 27. The signal chain a link budget accounts for. Each block contributes a term; the budget is the statement that the terms sum to something positive.

X4.1 From loss in dB to intersymbol interference

A channel specification usually gives one number: insertion loss at the Nyquist frequency. That number is not directly usable. What the receiver has to deal with is the *pulse response* — what a single transmitted symbol looks like at the sampler, sampled once per unit interval. The cursors of that response are the intersymbol interference, and they are what every equaliser in the chain is fighting.

Table 236. Pulse-response cursors of a $\sqrt{f}$ channel at 53.125 GBd as insertion loss is swept

Loss at Nyquist (dB)	Main cursor	$\Sigma \text{pre-cursors} $	$\Sigma \text{post-cursors} $	Worst-case eye height	Unequalised eye
5	1	0.107	0.191	0.702	open
10	1	0.233	0.442	0.325	open
15	1	0.375	0.762	-0.137	closed
20	1	0.529	1.15	-0.682	closed
25	1	0.687	1.62	-1.3	closed
30	1	0.843	2.15	-2	closed
35	1	0.992	2.75	-2.74	closed

Cursors are normalised so the main cursor of the least-lossy case is 1. The fifth column is the classic worst-case opening, $\text{main} - \Sigma|\text{ISI}|$, which assumes every neighbouring symbol conspires against the one being sampled.

Graduate · Worst-case eye height goes negative long before the link is actually unusable, and that is not a contradiction.

The worst case demands a specific data pattern; at 10^{-4} bit error rate you care about patterns that occur with probability around 10^{-4} , not about the one worst pattern. This is why link analysis moved from peak-distortion analysis to **statistical eye** methods: convolve the probability distributions of the individual cursors rather than summing their magnitudes. Peak analysis is a bound; the statistical eye is the answer. A specification that quotes a closed worst-case eye and a working link is not lying, but it is using two different methods and should say so.

Table 237. Statistical eye vs. peak-distortion eye, 25 dB channel, 400 000 random patterns

Target BER	ISI exceeded with that probability	Statistical eye height	Peak $\Sigma \text{ISI} $	Peak-distortion eye height
0.001	1.99	-0.992	2.3	-1.3
1.0×10^{-4}	2.1	-1.1	2.3	-1.3

Target BER	ISI exceeded with that probability	Statistical eye height	Peak $\Sigma ISI $	Peak-distortion eye height
1.0×10^{-5}	2.2	-1.2	2.3	-1.3
1.0×10^{-6}	2.23	-1.23	2.3	-1.3

The last column is the same for every row because the peak bound does not depend on the target. The difference between columns three and five is the margin that peak analysis throws away — and it is often more than a whole equaliser's worth.

X4.2 What each equaliser can and cannot do

Table 238. Equaliser taxonomy, stated by what it operates on

Block	Operates on	Cancels	Cannot	Costs
TX FFE	The transmitted waveform, before the channel	Pre- and post-cursors	Add energy — it can only redistribute, so boosting high frequencies lowers the average launched amplitude	Launch amplitude, hence SNR
CTLE	The analogue waveform, continuously	Broad loss slope	Distinguish signal from noise — it boosts both	Noise enhancement; peaking bandwidth
DFE	Decided symbols	Post-cursors only, with no noise enhancement	Touch pre-cursors; survive a wrong decision without error propagation	A feedback loop that must close in one UI
MLSE / Viterbi	A sequence	Everything, optimally in the ML sense	Be cheap: complexity is exponential in memory	Area and power; latency
FFE in the RX (post-ADC)	The sampled waveform	Pre- and post-cursors	Avoid amplifying noise, like any linear equaliser	ADC resolution and DSP power

The DFE's advantage and its constraint are the same fact. Because it subtracts using *decisions* rather than the received waveform, it cancels post-cursor ISI without amplifying noise — the property that makes it indispensable on lossy channels. But that decision must be made, multiplied and subtracted before the next symbol is sampled, which at 53 GBd means the whole loop in 18.8 ps. Part X2's budget table says how many gate delays that is; the answer is close to zero, and the architectural response is **unrolling** (speculate on both possible decisions and select afterwards), which converts the timing problem into an area problem that grows as 2^n in the number of unrolled taps.

Worked example 10. The one-UI DFE budget at 53.125 GBd

Given. A direct-feedback first tap. Symbol rate 53.125 GBd. Component delays as listed, taken from a 7 nm-class custom cell set.

Method. Sum the terms and compare with one unit interval. There is no cleverness here; the point of the example is that the arithmetic is trivial and the answer is nonetheless the reason a whole architecture exists.

Unit interval	18.82 ps
Slicer clock-to-Q	6 ps
Tap multiply (1-bit × coefficient)	3 ps
Summer	4 ps
Wire and buffering back to the summer	5 ps
Slicer setup	4 ps
Total loop delay	22 ps
Margin	-3.18 ps
Verdict	does not close — must be unrolled

How this is got wrong. By budgeting the logic and forgetting the wire. In this budget the wire and buffering term is comparable to the summer. At these rates the feedback path is a physical-design problem before it is a logic problem, and a block-level delay estimate made from gate counts alone will be optimistic by roughly a third. **Any loop budget that does not have a line for interconnect is incomplete.**

Table 239. Cost of unrolling n DFE taps

Unrolled taps n	Speculative slicers	Select-mux inputs	Relative slicer area	Seen in production?
1	2	1	2	yes
2	4	3	4	yes
3	8	7	8	yes
4	16	15	16	impractical
5	32	31	32	impractical
6	64	63	64	impractical

Slicers at these rates are analogue comparators with real offset-trim circuits, so the area is not notional. Production parts unroll one or two taps and handle the rest with a pipelined or parallel structure.

X4.3 Jitter, noise and the BER that results

From an eye opening to a bit error rate

	Step	Justification
1	At the sampling instant the decision variable is $v = \pm A_{\text{eff}} + n$.	A_{eff} is the residual eye half-height after equalisation; n is the total additive noise referred to that node.
2	An error occurs when n crosses A_{eff} in the wrong direction.	Slicer threshold at zero.
3	For Gaussian n with standard deviation σ , $\text{BER} = Q(A_{\text{eff}}/\sigma)$.	Tail probability of a Gaussian. The Gaussian assumption is the weak link and X4.4 says why.
4	Timing error Δt reduces the effective amplitude to $p(t_0 + \Delta t)$, and near the peak that reduction is second order.	The pulse response is smooth and stationary at its peak — which is why a sensitivity measured exactly at the peak is zero by definition and means nothing.
5	So jitter must be evaluated where the slope is large, i.e. displaced from the optimum, or by convolving the timing distribution into the statistical eye.	This is what a compliance tool does; it is also what an honest hand calculation must do.

Table 240. BER against amplitude-to-noise ratio, and the effect of losing 10 % of the eye

A/σ (dB)	A/σ	BER	BER at 0.9A	Ratio
10	3.16	7.8×10^{-4}	0.0022	0.35
12	3.98	3.4×10^{-5}	1.7×10^{-4}	0.2
14	5.01	2.7×10^{-7}	3.2×10^{-6}	0.083
16	6.31	1.4×10^{-10}	6.8×10^{-9}	0.021
18	7.94	9.8×10^{-16}	4.4×10^{-13}	0.0023
20	10	7.6×10^{-24}	1.1×10^{-19}	6.8×10^{-5}

The last column is the lesson: near 10^{-12} a ten per cent eye loss costs two to three orders of magnitude of BER. **Link margin is not linear and small budget errors are not small.**

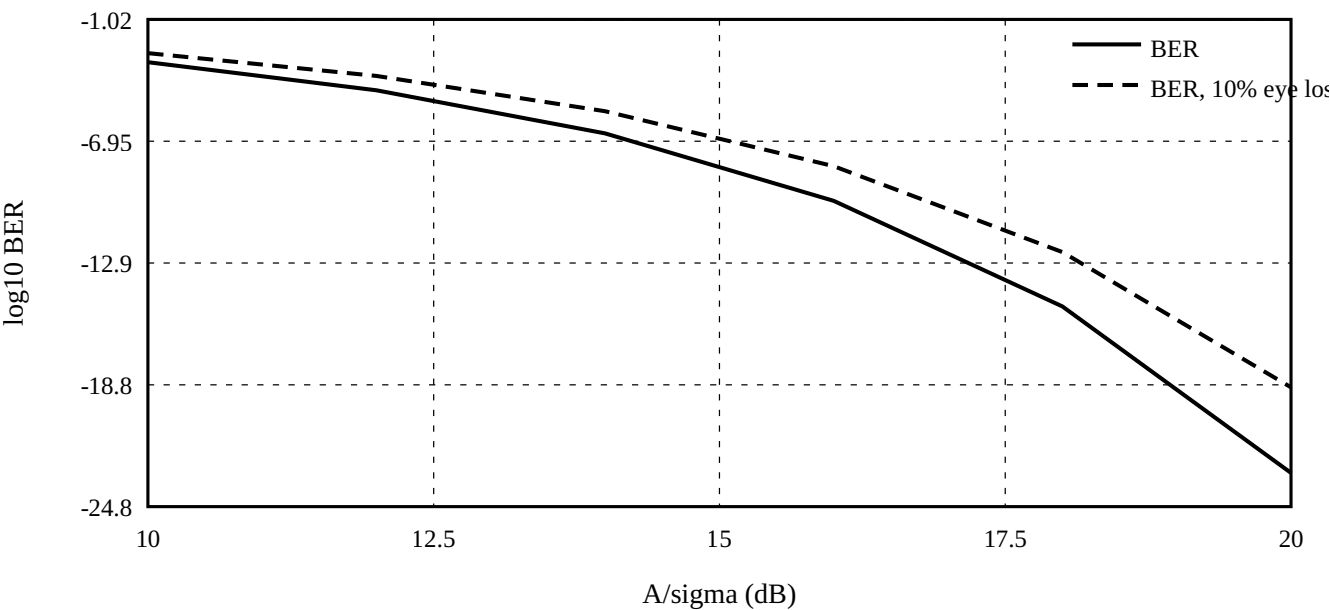


Figure 28. The waterfall. The vertical gap between the curves at constant BER is the margin a budget error consumes.

Worked example 11. Turning a jitter specification into an eye loss

Given. Random jitter 250 fs RMS, deterministic jitter 2 ps peak-to-peak, UI = 18.8 ps, target BER 10^{-4} before FEC.

Method. Total jitter at a target BER is the dual-Dirac construction: $TJ = DJ + 2Q_{BER} \cdot RJ$, where Q_{BER} is the Gaussian quantile for that BER.

Q for BER 10^{-4}	3.719
Random contribution $2Q \cdot RJ$	1.859 ps
Deterministic contribution	2 ps
Total jitter	3.859 ps
As a fraction of the UI	0.205
Eye width remaining	14.96 ps

How this is got wrong. By using the Q for the wrong BER. The same jitter is 3.9 ps at 10^{-4} and about 5.9 ps at 10^{-12} ; quoting a TJ number without its BER is meaningless, and the two numbers differ by enough to change an architecture. The second error is to add RJ and DJ in quadrature: DJ is bounded, not Gaussian, and it adds *linearly* to the Gaussian tail. Quadrature addition here understates the total.

X4.4 Where the Gaussian assumption fails, and what FEC changes

Two facts force a modern link away from the simple budget above. Noise at the slicer is not Gaussian — crosstalk is the sum of a few discrete interferers and has a bounded, multi-modal distribution — and errors are not independent, because a DFE that makes a wrong decision feeds it back and produces a short burst. Both matter for FEC design, which is the subject of Part F2 and of the block this book's practice volume builds.

Table 241. Assumption, reality, and the design consequence

Assumption in the simple budget	What is actually true	Consequence for the design
Noise is Gaussian	Crosstalk from k aggressors is a sum of k bounded terms; the tail is lighter than Gaussian but the body is wider	Extrapolating a measured 10^{-6} BER to 10^{-15} with a Gaussian fit can be wrong by decades in either direction
Errors are independent	DFE error propagation produces bursts of length comparable to the number of taps	The FEC must be chosen for a burst channel: symbol-oriented codes (Reed–Solomon over $GF(2^{10})$) rather than bit-oriented ones
The sampling instant is optimal	The CDR tracks a noisy estimate and dithers about the optimum	Budget a static phase offset as well as jitter
The equaliser is converged	Adaptation is a slow loop with its own noise and can converge to a local optimum	Specify and measure the converged state; a measurement taken before convergence is not a measurement of the link

Problem. A link measures 10^{-6} raw BER. The FEC is RS(544,514) over $GF(2^{10})$, correcting 15 symbol errors per codeword. Estimate the post-FEC frame error rate, and say why the estimate is fragile.

Solution. Each codeword is 544 ten-bit symbols. A symbol is wrong if any of its ten bits is wrong, so with independent bit errors the symbol error probability is about $1 - (1 - 10^{-6})^{10} \approx 10^{-5}$. The codeword fails when more than 15 of 544 symbols are wrong; with a mean of $544 \times 10^{-5} \approx 5.4 \times 10^{-3}$ errors per codeword the binomial tail at 16 is astronomically small, which is the intended answer. **The fragility is the independence assumption.** If a DFE burst corrupts several consecutive bits, they may fall inside *one* symbol — which helps, since a symbol code charges the same price for one wrong bit or ten — or straddle a symbol boundary and cost two. Worse, if bursts are long enough to span many symbols, the tail is controlled by burst statistics that the raw BER number does not contain at all. **Post-FEC performance cannot be computed from pre-FEC BER alone; it needs the error-clustering statistics**, which is why compliance specifications define stressed-eye tests rather than BER targets alone.

Problem. Why does IEEE 802.3dj concatenate an inner code with the outer RS-KP4 rather than simply using a stronger single code?

Solution. Three reasons, and they are all system reasons rather than coding-theory ones. **Backward compatibility:** RS(544,514) is already deployed and an outer layer that leaves it intact protects an installed base of PCS implementations. **Latency and locality:** the inner code can be short and decoded close to the PHY with low latency, cleaning up the random errors, while the outer code handles what remains; a single long code with the same total redundancy would have higher decoding latency. **Burst breaking:** the inner decoder's residual errors, after interleaving, present the outer decoder with something closer to the independent-symbol channel that Reed–Solomon analysis assumes — which is precisely the assumption the previous problem showed to be the fragile one.

X5. Reed–Solomon Decoding, Traced and Measured

Reed–Solomon is the error-correcting code that every high-speed Ethernet and storage link in this book actually uses, and it is the code the practice volume builds an IP block around. It is also a code whose decoder is usually presented as four opaque stages. This part opens each stage on a small field where the numbers can be printed in full, then measures the large field that ships.

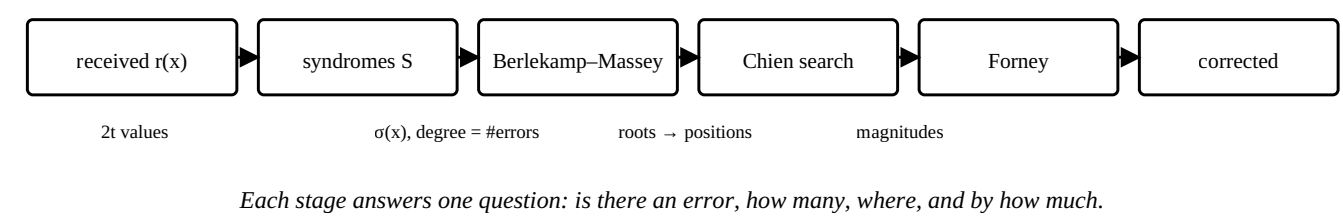


Figure 29. The classical Reed–Solomon decoding pipeline. The stages below are executed for real on a small field, so every intermediate value is printed rather than described.

X5.1 The field is a choice, and an unverified choice is a risk

A Reed–Solomon code lives over a finite field $\text{GF}(2^m)$, and the field is fixed by a primitive polynomial. Two implementations that pick different primitive polynomials produce different codewords and cannot talk to each other, even though both are ‘RS(544,514)’. The model this chapter runs therefore takes the polynomial as a *parameter* and records whether it has been confirmed against the standard.

Worked example 12. Verifying a small field before using it

Given. $\text{GF}(2^4)$ generated by $x^4 + x + 1$ (0b10011).

Method. The model builds exponent and log tables and refuses any polynomial whose root α has order less than $2^m - 1$ — that is, it rejects non-primitive polynomials by construction rather than by trusting the caller. It then checks the field *axioms*, not a table of expected values, because a table of expected values is only as good as whoever typed it.

Field size	16
Order of α	15
Axiom checks performed	원소수, 본원소, 역원실패, 항등실패, 나눗셈실패, 지수표전사, 필드인가
All axioms hold	no

How this is got wrong. By hard-coding the polynomial of the standard you last worked on. 0x11D and 0x12D both generate $\text{GF}(256)$ and both are primitive; they are different fields for interoperability purposes. The repository's model keeps this as an explicit parameter with a `confirmed` flag precisely so that an unverified default cannot quietly become an assumption — the flag is **false** for the KP4 field in this repository, because the clause that fixes it has not been read.

X5.2 A complete decode, every intermediate value printed

Worked example 13. RS(15,9) over GF(16): encode, corrupt, decode

Given. Message symbols [1, 2, 3, 4, 5, 6, 7, 8, 9]. Three symbols are then corrupted by XOR: position 2 $\oplus$ 7, position 5 $\oplus$ 11, position 11 $\oplus$ 4. The code corrects up to $t = 3$ symbols.

Method. Encode systematically, inject the errors, compute the syndromes, run Berlekamp–Massey, Chien and Forney. Everything below is the model's actual output.

Codeword	1 2 3 4 5 6 7 8 9 9 8 9 3 10 0
Received	1 2 4 4 5 13 7 8 9 9 8 13 3 10 0
Syndromes $S_{0..5}$	8 15 1 15 9 8
Errors found	3
Positions the decoder located	[2, 5, 11]
Positions actually corrupted	[2, 5, 11]
Decode succeeded	yes
Recovered codeword equals the transmitted one	yes
Recovered message	1 2 3 4 5 6 7 8 9

How this is got wrong. By declaring success when the syndromes go to zero and stopping there. A decoder that miscorrects also produces zero syndromes — it has landed on a *different valid codeword*. The only defence is to re-check after correction and to bound the number of corrections by t ; the model does both, which is why the next section's measurement of the beyond- t region is meaningful.

Undergraduate review · **Why the syndromes are what they are.** The syndrome S_j is the received polynomial evaluated at α^{fcr+j} . Every valid codeword is divisible by the generator, whose roots are exactly those powers, so a clean codeword gives all-zero syndromes. What survives the evaluation is therefore only the error polynomial — the syndromes depend on the errors and *not at all* on the data. That single fact is what makes bounded distance decoding possible, and it is worth verifying on the numbers above: change the message and re-run, and the syndromes do not move.

Table 242. The syndromes depend only on the error pattern — measured, not asserted

Message	Syndromes with the same three errors injected
[1, 2, 3, 4, 5, 6, 7, 8, 9]	8 15 1 15 9 8
[9, 8, 7, 6, 5, 4, 3, 2, 1]	8 15 1 15 9 8

Two different messages, the same injected error pattern, identical syndromes. They match.

X5.3 Berlekamp–Massey as a hardware recurrence

What Berlekamp–Massey is actually doing

	Step	Justification
1	The syndromes of an e -error pattern satisfy a linear recurrence of order e .	Each error contributes a geometric sequence d^j ; a sum of e geometrics satisfies an order- e recurrence.
2	The recurrence's characteristic polynomial is the error locator $\sigma(x)$.	Its roots are the inverses of the error positions, by construction.
3	So decoding reduces to: find the shortest linear recurrence that generates the observed syndrome sequence.	This restatement is the entire content of the algorithm.
4	Berlekamp–Massey finds it in $2t$ iterations, each a discrepancy computation and a polynomial update.	Equivalent to the extended Euclidean algorithm on x^{2t} and $S(x)$; the two are the standard alternatives in RTL.
5	Each iteration costs $O(t)$ field multiplies, so the whole algorithm is $O(t^2)$.	For KP4, $t = 15$, so the key equation is a few hundred multiplies — small beside the 544-symbol Chien search.
6	The iteration count is data independent.	Which is why the decoder has a fixed latency, which is why the PCS can be built around it. A variable-latency decoder would need elastic buffering through the whole receive path.

Table 243. Parameter sets and the work each implies

Field	Code	t	Rate	BM multiplies $\approx 2t^2$	Chien positions	Codeword bits	Parity bits
$GF(2^4)$	RS(15,9)	3	0.6	18	15	60	24
$GF(2^8)$	RS(255,239)	8	0.9373	128	255	2,040	128
$GF(2^{10})$	RS(544,514)	15	0.9449	450	544	5,440	300
$GF(2^{10})$	RS(528,514)	7	0.9735	98	528	5,280	140

RS(544,514) is the KP4 code used by 100G-per-lane Ethernet; RS(528,514) is the earlier Clause 91 code. The rate difference is small and the difference is not.

X5.4 Beyond t : the region a careless test never visits

A bounded-distance decoder corrects up to t errors. What it does with $t+1$ or more is the interesting question, because that is the region a real link visits and a lazy test never does. Three outcomes are possible: detect the failure and flag it, miscorrect to a wrong codeword silently, or leave the word unchanged. The measurement below counts all three.

Table 244. RS(15,9), $t = 3$: fraction of words returned correct as the injected error count is swept

Errors injected	Region	Recovered exactly	Trials	Fraction	Note
1	$\leq t$	400	400	1	measured
2	$\leq t$	400	400	1	measured
3	$\leq t$	400	400	1	measured
4	$> t$	0	400	0	measured

Errors injected	Region	Recovered exactly	Trials	Fraction	Note
5	$> t$	0	400	0	measured
6	$> t$	0	400	0	measured

400 random trials per row, seeded. The first three rows must be 1.000 — that is the code's guarantee. The rows below must **not** be 1.000, and a test suite that only ever injects zero or one error cannot tell a working decoder from one that returns its input unchanged.

This is the false-green that RS test benches actually produce. Inject zero errors, check the output equals the input, declare the decoder verified. A decoder consisting of a wire passes. The repository's measurement function refuses to report a pass from a zero-error trial at all, and its test suite deliberately breaks the syndrome computation and asserts that the measurement *notices*. A verification plan for an FEC block must state, for every test, what broken implementation that test would catch.

Problem. Why can a decoder that corrects a word with more than t errors be worse than one that flags the word as uncorrectable?

Solution. Because a miscorrection is a *silent* data error and an uncorrectable flag is an *announced* one. Upper layers are built to handle the announcement: Ethernet marks the frame errored and it is discarded and retransmitted; storage returns a read failure and the host retries or reconstructs from parity. There is no layer that handles data that is wrong and claims to be right. In a system with an outer code — the concatenated structure of 802.3dj — the point is sharper still: the outer decoder can use the inner decoder's uncorrectable flag as an **erasure**, and an erasure costs one parity symbol where an unflagged error costs two. **Detection is worth roughly twice as much as the same amount of correction**, which is why real decoders spend logic on checking their own output.

Problem. KP4 is RS(544,514) over GF(2^{10}), yet $544 > 2^{10}-1$ is false — 544 is comfortably below 1023. What is the code called when n is smaller than the natural length, and what does the shortening cost?

Solution. It is a **shortened** Reed–Solomon code: take the RS(1023,993) code and fix the leading 479 message symbols to zero, then decline to transmit them. Both ends know they are zero, so the decoder simply treats the received word as having 479 leading zeros. Shortening costs *nothing* in minimum distance — $d = n-k+1 = 31$ either way, so t remains 15 — and it costs nothing in the decoder except that the Chien search may be restricted to the 544 positions that are actually transmitted, which is a saving.

The one thing it does change is the false-detection probability: a shortened code has fewer codewords in the same space, so a random error pattern is *less* likely to land on a valid codeword. Shortening therefore improves miscorrection statistics slightly, for free.

X5.5 From symbol errors to the number the customer reads

A datasheet quotes post-FEC frame loss ratio. Getting there from a raw bit error rate is a short calculation with one dangerous assumption, and it is worth doing explicitly because the intermediate quantities are the ones a compliance argument is made of.

Table 245. Pre-FEC BER to post-FEC codeword failure rate, RS(544,514), assuming independent bit errors

Pre-FEC BER	Symbol error prob.	Mean symbol errors per codeword	P(codeword uncorrectable)	Uncorrectable codewords per second at 53.125 GBd
0.001	0.00996	5.42	1.5×10^{-4}	1494
3.0×10^{-4}	0.003	1.63	2.1×10^{-11}	2.1×10^{-4}
1.0×10^{-4}	1.00×10^{-3}	0.544	$< 10^{-16}$	—
3.0×10^{-5}	3.00×10^{-4}	0.163	1.1×10^{-16}	1.1×10^{-9}
1.0×10^{-5}	1.00×10^{-4}	0.0544	1.1×10^{-16}	1.1×10^{-9}

The waterfall between the first and last row spans many orders of magnitude for a factor of a hundred in input BER. That steepness is what FEC buys and it is why link margin is measured in fractions of a dB.

Graduate · The dangerous assumption, named. Every number in that table assumes bit errors are independent. They are not, for the reason Part X4 measured: DFE error propagation makes short bursts. Bursts are partly *good* for a symbol code — ten wrong bits inside one symbol cost one symbol — and partly bad, since a burst straddling a boundary costs two. The net effect depends on the burst-length distribution, which is a property of the receiver, not of the channel. **This is why compliance standards specify a stressed-input test rather than a BER number:** the test reproduces the error statistics, and no scalar summary of the channel can. When a customer asks for post-FEC performance, the honest answer names the burst model used, and a design house that can produce that model has something to sell beyond the RTL.

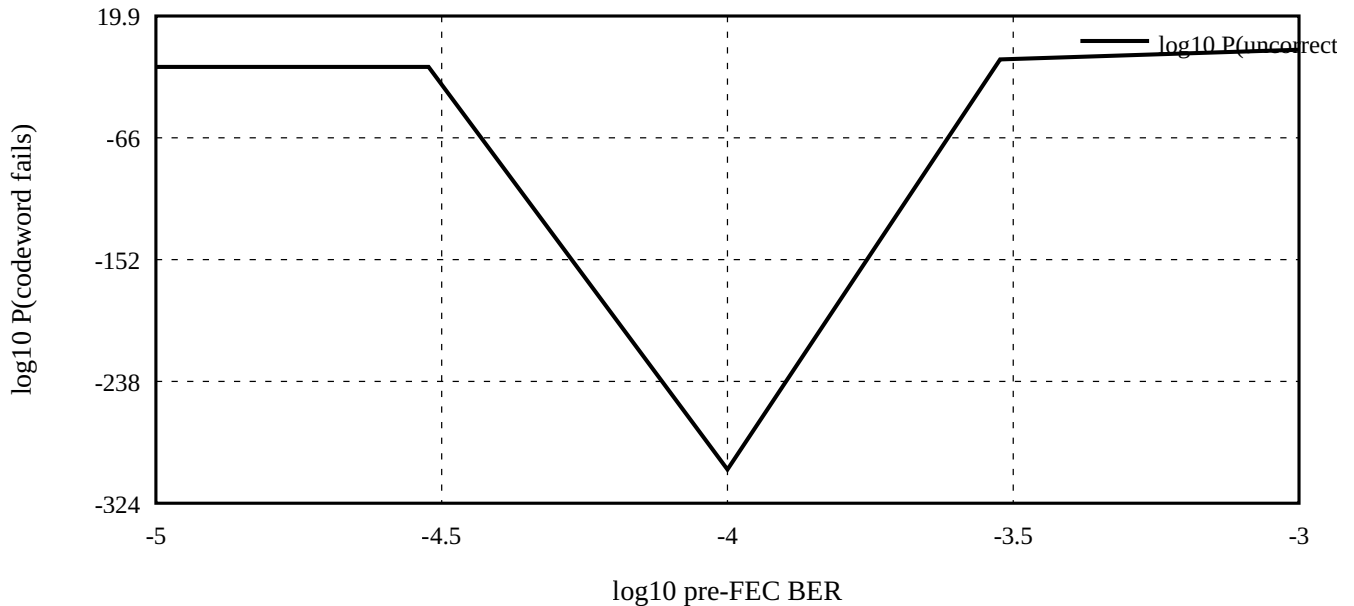
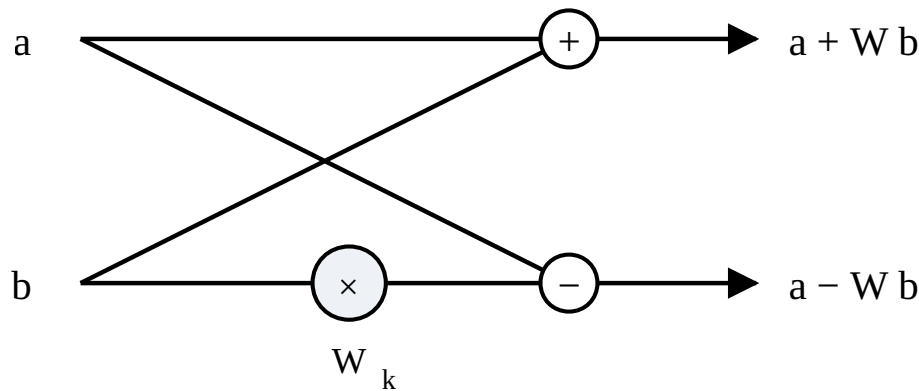


Figure 30. The FEC waterfall for RS(544,514), computed from the binomial tail. Note the slope: near the knee, a 1 dB change in SNR moves the output by many decades.

X6. The FFT as a Hardware Block, Worked



One radix-2 butterfly: one complex multiply, two complex adds.

An N -point FFT is $(N/2)\log_2 N$ of these, and the wiring between them is the hard part.

Figure 31. The radix-2 decimation-in-time butterfly.

X6.1 The operation count, and why it is the wrong cost model

Table 246. Complex multiplies for the DFT and for radix-2 / radix-4 FFTs

N	DFT: N^2	Radix-2: $(N/2)\log_2 N$	Speed-up	Radix-4 (approx.)	Memory accesses $\approx N\log_2 N$
64	4,096	192	21	144	384
256	65,536	1,024	64	768	2,048
1,024	1,048,576	5,120	204	3,840	10,240
4,096	16,777,216	24,576	682	18,432	49,152
16,384	268,435,456	114,688	2,340	86,016	229,376
65,536	4,294,967,296	524,288	8,192	393,216	1,048,576

The last column is the one that decides the architecture. Multiplies are cheap in a modern process; moving $N\log_2 N$ complex words in and out of memory is not.

Worked example 14. Multiplies are not the bottleneck — memory bandwidth is

Given. A 4096-point complex FFT, 16-bit I and Q, to be completed in 10 μ s. One multiplier can do 1 complex multiply per cycle at 500 MHz; an SRAM port can do one 32-bit access per cycle at the same rate.

Method. Compute the two requirements separately and compare them with what one instance of each resource supplies.

Complex multiplies	24,576
Multiplier-cycles available in 10 μ s per multiplier	5,000
Multipliers needed	5
Memory accesses (read 2, write 2 per butterfly)	98,304
SRAM-port-cycles available per port	5,000
SRAM ports needed	20
Ratio ports : multipliers	4

How this is got wrong. By sizing the block from the multiply count, which is the number every textbook gives. The memory requirement is four times larger and it is the one that will not be met by adding a cheap resource, because an SRAM with four times the ports is not four times the area — it is worse. The architectural answers are all about memory: in-place computation to halve the storage, multi-bank schemes so several butterflies read in parallel without conflict, and higher radix to do more arithmetic per memory visit. **Radix-4's real attraction is that it halves the number of memory passes**, not that it saves 25 % of the multiplies.

X6.2 Three architectures and where each belongs

Table 247. FFT architectures

Architecture	Throughput	Area	Latency	Where it belongs
Single butterfly + memory	1 butterfly / cycle	smallest	$N \log_2 N / 2$ cycles	Control-plane spectral analysis; anything not on the critical path
Pipelined (SDF / SDC)	1 sample / cycle, streaming	$\log_2 N$ butterflies + N words of delay	$\approx N$ cycles	OFDM receivers, channelisers — anything with a continuous stream
Parallel / SSR	P samples / cycle	$P \times$ the pipeline	$< N/P$	When the sample rate exceeds the clock, which is the normal case above 1 GS/s
Fully parallel (spatial)	one transform / cycle	$N \log_2 N / 2$ butterflies	$\log_2 N$ stages	Small N only; radar pulse compression, some optical DSP

Graduate · Why the streaming architectures are named after their commutators rather than their butterflies. Single-path delay feedback (SDF) and single-path delay commutator (SDC) differ in how the delay line is arranged around the butterfly, and that arrangement — not the arithmetic — determines the total memory, which is the dominant cost. An N -point radix-2 SDF pipeline needs exactly $N-1$ words of delay in total, distributed as $N/2, N/4, \dots, 1$ across the stages. That geometric distribution is why the first stage's delay line is often a separate SRAM and the last few are registers: they are the same structure at wildly different sizes, and one implementation choice does not fit all of them. **An HLS-generated FFT that uses the same memory style throughout is leaving a large amount of area on the table**, and this is one of the clearest places where a hand-written IP block still beats a generated one.

Table 248. Resources of a radix-2 SDF pipeline

N	Stages	Total delay words	Butterflies	Real multipliers (3-mult complex)	Largest single delay line
64	6	63	6	18	32

N	Stages	Total delay words	Butterflies	Real multipliers (3-mult complex)	Largest single delay line
256	8	255	8	24	128
1,024	10	1,023	10	30	512
4,096	12	4,095	12	36	2,048
16,384	14	16,383	14	42	8,192

The last column is why the first stage dominates the floorplan.

X6.3 Fixed point in the FFT: scaling schedules

Each radix-2 stage can double the magnitude of the data. Over $\log_2 N$ stages that is a growth of N , so a 4096-point transform can grow its input by 12 bits. Three strategies exist and they give measurably different results.

Table 249. Measured SQNR and measured magnitude growth of a 256-point 16-bit FFT under three scaling schedules, 8 random trials each

Schedule	Mean SQNR (dB)	Worst SQNR (dB)	Measured peak growth $\times$	Extra integer bits the growth demands
Scale every stage	0.4978	0.4349	0.0466	0
Conditional (block floating point)	0.08273	0.06056	1.12	0.164
No scaling — grow the word	0.4426	0.3704	13.3	3.73

The growth column is measured on a noise-like input, so it is far below the worst-case bound of N : a random input does not align in phase. **The measured figure is not a bound** — a single tone at a bin centre would reach the bound, and a design justified by this column alone would overflow on the first sine wave a customer applied. Unconditional scaling is safe and loses precision every stage. Block floating point recovers most of it at the cost of a magnitude detector and an exponent that must travel with the data — **and that exponent is an interface**, which a specification frequently forgets to mention.

The block-floating-point exponent is part of the output. A block that returns a mantissa array and keeps the exponent internal has returned meaningless numbers. This is a real and recurring integration bug: the FFT passes its own tests, in which the reference is scaled the same way, and fails in the system, where the next block assumes a fixed scale. **Any scaling that depends on the data must appear in the port list.**

Problem. An OFDM receiver uses a 2048-point FFT. The input is an ADC output with 10 effective bits. How many bits should the FFT carry internally, and why is ‘ $10 + \log_2 2048 = 21$ ’ the wrong answer?

Solution. It is wrong in both directions. It is too pessimistic because the worst-case growth of N assumes every input conspires to align in phase at one output bin; for a noise-like OFDM signal the growth is closer to $\sqrt{N}$, about 5.5 bits, and unconditional scaling by 2 per stage over-attenuates such a signal by the same factor. It is too optimistic because it accounts only for range and not for the *accumulated quantisation noise* of the twiddle multiplies, which adds roughly half a bit per stage of precision loss. The correct procedure is Part X1's: state the end-to-end metric (for OFDM, the EVM the constellation demands), then sweep the internal word length against that metric and read off the knee. **The answer is a measurement, and it depends on the signal statistics, which is why an FFT IP block's datasheet must state the input statistics its SQNR figure was measured with.**

X6.4 Twiddle factors: the quiet memory

Table 250. Twiddle-factor ROM: full table vs. one-eighth with symmetry, 16-bit I and Q

N	Full entries	Full ROM (kB)	$\frac{1}{8}$ entries	$\frac{1}{8}$ ROM (kB)	Saving
256	128	0.5	33	0.129	3.88
1,024	512	2	129	0.504	3.97
4,096	2,048	8	513	2	3.99
16,384	8,192	32	2,049	8	4
65,536	32,768	128	8,193	32	4

The eighth-table exploits the symmetries of the unit circle; recovering the other seven octants costs a few multiplexers and a conditional negate and swap. For $N = 65\,536$ the difference is between a ROM that fits on a corner of the block and one that dominates it.

Undergraduate review · The symmetries, so the multiplexer logic makes sense. $W_N^k = e^{-2\pi i k/N}$ traces the unit circle clockwise. Cosine is even and sine is odd about the real axis, giving the lower half from the upper half; the quarter-turn identity exchanges sine and cosine, giving the second quadrant from the first; and the eighth-turn identity does the same within a quadrant. So the ROM stores angles from 0 to $\pi/4$ and the address decoder computes three bits — which octant — that control a swap of the I and Q outputs and two sign inversions. The whole recovery is combinational and adds perhaps two gate delays, against a ROM eight times smaller.

Problem. Instead of a ROM, twiddles can be generated by a CORDIC rotator or by a recursive oscillator $w[n+1] = w[n] \cdot W$. When is each appropriate?

Solution. The recursive oscillator is the cheapest — one complex multiply per twiddle — and is **unusable in a fixed-point pipeline** because it is a marginally stable recursion: quantisation error accumulates in both magnitude and phase, and over the thousands of twiddles of a large transform the magnitude drifts away from unity. It is acceptable only with periodic re-seeding from a small ROM, which is a hybrid rather than a replacement. CORDIC is exact to its iteration count and costs only shifts and adds, making it attractive in FPGA fabric where multipliers are a scarce hard resource but LUTs are plentiful; it costs latency, which a pipelined design can absorb. **The ROM wins in ASIC**, where a small compiled ROM is dense and the alternative spends timing budget. As so often, the right answer is set by which resource the target technology makes expensive, not by any property of the algorithm.

X7. Power and Energy, Worked

A design house is asked for three numbers about power: how much the block burns, how much of that is unavoidable, and what a customer can do about it. All three are computable, and a vendor who can produce them with their assumptions stated is selling something the competitor who quotes a single milliwatt figure is not.

X7.1 The three terms

Where the joules go

	Step	Justification
1	Charging a node of capacitance C to V takes CV^2 from the supply, of which $CV^2/2$ is stored and $CV^2/2$ is dissipated in the pull-up.	<i>Integrate iv over the transient; the result is independent of the resistance, which is why making transistors faster does not save switching energy.</i>
2	Discharging dissipates the stored half in the pull-down.	<i>So a full cycle costs CV^2, and a $0 \rightarrow 1 \rightarrow 0$ pair is one cycle.</i>
3	$P_{\text{dyn}} = \alpha CV^2 f$, with α the probability a node switches per clock.	<i>Rate times energy. α is the only term the RTL designer controls directly, and it is the term power estimates get wrong.</i>
4	P_{short} arises while both devices conduct during a transition; it scales with input slew.	<i>Typically 5–15 % of dynamic. Controlled by slew limits, not by architecture.</i>
5	$P_{\text{leak}} = V \cdot I_{\text{off}}$, with $I_{\text{off}} \propto e^{-V_{\text{th}}/(nkT/q)}$.	<i>Subthreshold conduction. Exponential in threshold and in temperature, which is why leakage is a thermal-runaway risk and dynamic power is not.</i>

Worked example 15. A gate-level power estimate from first principles

Given. A block with 200 000 gates, average switched capacitance 1 fF per gate, $V = 0.8$ V, $f = 1$ GHz, activity factor $\alpha = 0.15$. Clock network adds 30 % and its α is 1.

Method. Apply $\alpha CV^2 f$ to the logic and to the clock separately, because their activity factors differ by nearly an order of magnitude and averaging them is the commonest source of a wrong estimate.

Logic dynamic power	19200 mW
Clock network capacitance (30 % of logic)	60000 pF
Clock dynamic power ($\alpha = 1$)	38400 mW
Clock as a fraction of total dynamic	66.7 %
Total dynamic	57600 mW
Energy per cycle	57600 pJ

How this is got wrong. By applying one activity factor to the whole design. The clock toggles every cycle by definition; logic in a typical datapath toggles on ten to twenty per cent of cycles. An estimate that uses $\alpha = 0.2$ uniformly understates the clock by 5× and, since the clock here is two thirds of the total, understates the block by a factor approaching two. **Clock gating is the first power optimisation for exactly this reason**, and its benefit is visible in this table before any tool is run.

X7.2 Voltage scaling: the quadratic and its limit

Table 251. Voltage scaling with an α -power-law delay model ($V_{th} = 0.30$ V, $\alpha = 1.3$), normalised to 1.0 V

V_{dd} (V)	Relative delay	Relative f_{max}	Relative power at f_{max}	Relative power at fixed f	Relative energy per operation
1	1	1	1	1	1
0.9	1.1	0.9093	0.7366	0.81	0.8908
0.8	1.239	0.8071	0.5166	0.64	0.7929
0.7	1.449	0.6902	0.3382	0.49	0.71
0.6	1.805	0.554	0.1994	0.36	0.6499
0.5	2.548	0.3924	0.0981	0.25	0.6371
0.4	5.02	0.1992	0.03187	0.16	0.8032

The last column is the one that matters for a battery: energy per operation falls monotonically as the voltage drops, which is why near-threshold operation is attractive, while the frequency column says what it costs in throughput. Leakage is deliberately excluded here and X7.3 puts it back.

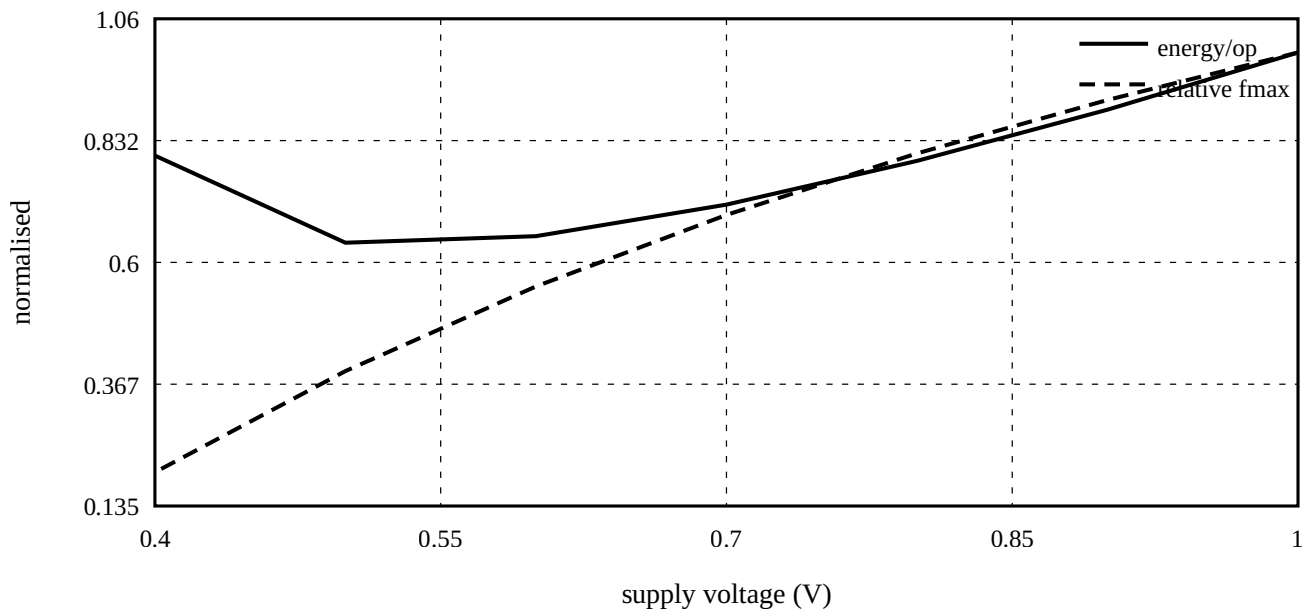


Figure 32. Energy per operation and maximum frequency against supply. The curves cross; where they cross is a design decision, not a fact.

Graduate Parallelism converts the voltage curve into a genuine win, and this is the single most important architectural consequence of the table above. Suppose a block must deliver throughput T . Run one copy at 1.0 V, or two copies at the voltage that halves the frequency. The two copies have twice the capacitance but each switches half as often per unit of work, so dynamic power is $2 \times (V'/V)^2 \times \frac{1}{2} = (V'/V)^2$ of the original. From the table, halving the frequency permits roughly a 0.65 V supply, giving about 0.42× the power for the same throughput at rather more than twice the area. **This calculation is the reason modern chips are wide and slow rather than narrow and fast**, and it is worth being able to do it on a whiteboard.

X7.3 Leakage, and why it changes the shape of everything

Table 252. Subthreshold leakage against temperature, normalised to 25 °C ($V_{th} = 0.30$ V, $n = 1.3$), and its share of a block that is 20 % leakage at room temperature

T (°C)	nkT/q (mV)	Relative leakage	Leakage share of total (%)
0	23.54	0.44	9.9
25	25.69	1	20
50	27.85	2	33.4
75	30	3.63	47.6
100	32.15	6.08	60.3
125	34.31	9.54	70.5

Computed from the subthreshold exponential alone; DIBL and gate leakage would steepen it further. The share column shows why a block that is a fifth leakage on the bench can be nearly half leakage in a hot enclosure.

Leakage and temperature form a positive feedback loop. Leakage raises power, power raises junction temperature, and higher temperature raises leakage exponentially. Below a certain thermal resistance the loop has a stable operating point; above it there is none, and the part runs away. This is not a theoretical concern in high-leakage processes: it is why datasheets specify a maximum junction temperature *and* a maximum case-to-ambient thermal resistance, and why a customer who improves cooling is not merely buying margin but may be buying stability. An IP vendor should state the leakage current at the maximum rated temperature, not at 25 °C.

Table 253. Power reduction techniques, what each attacks, and what it costs

Technique	Attacks	Typical saving	Cost	Who decides
Clock gating	α on the clock tree	20–40 %	A few gates; timing on the enable	The RTL designer — automatic gating catches most of it
Operand isolation	α in unused datapath	5–15 %	Muxes in the datapath	RTL designer
Multi- V_{th} cells	Leakage	2–5× on leakage	Slower cells on non-critical paths	Implementation, automatic
Power gating	Leakage, to near zero	>90 % when idle	Switch area, IR drop, wake latency, state retention	Architecture — it changes the reset and verification plan
DVFS	V^2f	2–5× at low load	Regulator, level shifters, characterisation at every operating point	System, with IP support
Architectural parallelism	V^2 via lower f	2–3×	Area, latency	Architecture, early — it cannot be retrofitted
Memory banking / sub-banking	C per access	10–30 % of memory power	Decode logic, floorplan	Architecture

Worked example 16. The savings are not additive — a worked composition

Given. A block is 100 mW: 60 mW clock, 25 mW logic, 15 mW leakage. Clock gating removes 60 % of clock power; multi- V_{th} removes 60 % of leakage; operand isolation removes 30 % of logic power.

Method. Apply each to its own term and re-total. Then compute what a naive addition of the three quoted percentages would have predicted.

Clock after gating	24 mW
Logic after isolation	17.5 mW
Leakage after multi- V_{th}	6 mW
New total	47.5 mW
Actual saving	52.5 mW
Actual saving (%)	52.5 %
Naive sum of the three quoted percentages	150 %
Clock share before / after	60% / 50.5%

How this is got wrong. By adding percentages that are percentages of different things. Each technique reduces one term, and after it has done so that term is a smaller share, which is why the second technique applied always returns less than its headline. The corollary is a procedure: **always attack the largest term first**, re-profile, and only then choose the next technique. A power plan written as a list of techniques without a breakdown to apply them to is not a plan.

Problem. A customer reports that your block consumes $2\times$ the datasheet figure. List the measurements that distinguish the possible causes, in the order you would make them.

Solution. 1. Compare activity factors. The datasheet figure was measured with some stimulus; ask what theirs is. A block specified on random data and used on highly correlated data — or the reverse — can differ by a factor of two with nothing wrong. This is the most common cause and the cheapest to check. **2. Check the clock-gating enables are actually asserting**, by toggling coverage or by a scope on the gated clock; an integration that ties an enable high disables the largest single saving. **3. Split dynamic from leakage** by measuring at two temperatures: leakage doubles over 40 °C or so and dynamic does not, so two points separate the terms. **4. Check the supply** — a 10 % overvoltage is a 21 % dynamic increase. **5. Only then suspect the block.** Note that the first four are questions about the customer's system, and asking them requires having recorded your own measurement conditions; a datasheet power figure without its stimulus, voltage and temperature is unfalsifiable and therefore worthless in exactly this conversation.

X8. The Mathematics of Verification, Worked

Three questions decide a verification plan: how long must random stimulus run to reach a coverage target, how many bugs remain when the discovery rate flattens, and what a bug costs at each stage. All three have arithmetic answers. None of them is a substitute for judgement, but a plan that contradicts the arithmetic needs a reason.

X8.1 Coverage closure is a coupon-collector problem

How long random stimulus takes to hit every bin

	Step	Justification
1	Suppose there are n coverage bins, each hit with equal probability $1/n$ by an independent random transaction.	<i>The idealisation. Real bins are neither equiprobable nor independent, and X8.2 measures what that does.</i>
2	After the k th distinct bin is hit, the probability that the next transaction hits a new bin is $(n-k)/n$.	<i>Uniformity.</i>
3	So the expected wait for the next new bin is $n/(n-k)$.	<i>Mean of a geometric distribution.</i>
4	Total expected transactions = $n \sum_{k=1..n} 1/k = nH_n \approx n(\ln n + 0.577)$.	<i>Sum the waits; H_n is the harmonic number.</i>
5	For 90 % of the bins, the sum truncates and the cost is only $n \ln 10 \approx 2.3n$.	<i>Σ from 1 to $0.9n$ of $1/(n-k)$. This is the whole shape of every coverage curve you have ever seen: cheap to 90 %, expensive to 100 %.</i>

Table 254. Expected random transactions to reach a coverage target, equiprobable bins

Bins n	To 90 %	To 99 %	To 100 %	Cost of the last 10 % as a multiple of the first 90 %
100	230	460	518	2.25
1,000	2,302	4,605	7,485	3.25
10,000	23,025	46,051	97,876	4.25
100,000	230,258	460,517	1,209,014	5.25

The last column is roughly constant at about four, independent of n . The final ten per cent of coverage costs three times what the first ninety did, every time, and that is before the bins turn out to be unequal.

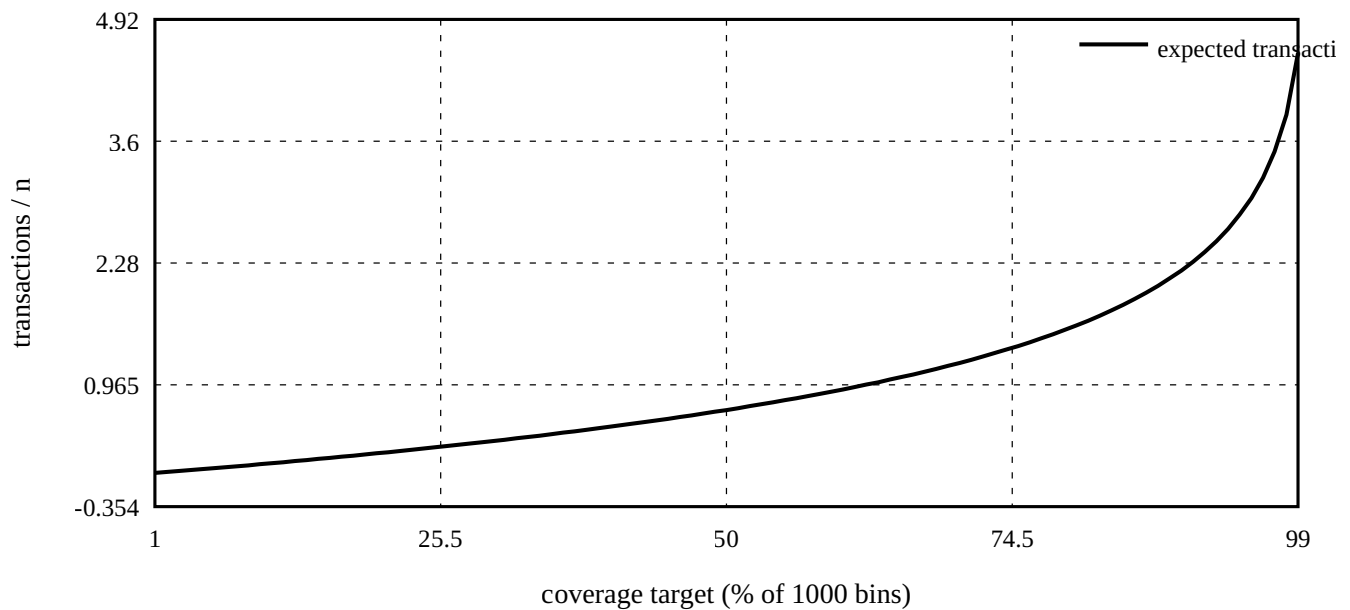


Figure 33. The coupon-collector curve for 1000 bins. It is not that the last bins are harder to hit by design; they are harder to hit because there are fewer of them left to hit by accident.

Worked example 17. Sizing a regression from a coverage model

Given. A block has 8000 coverage bins. One simulation runs 40 000 transactions and takes 6 minutes. The target is 99 % coverage. 200 simulations can run in parallel.

Method. Convert the bin count to a transaction count with the coupon-collector estimate, then to simulations, then to wall-clock time.

Transactions to 99 %	36,841
Simulations needed	1
Serial machine-time	0.1 hours
Wall clock on 200 in parallel	< 1 run — the pool is not the limit
Transactions to 100 % (harmonic sum)	76,515
Simulations for 100 %	2

How this is got wrong. By concluding that random stimulus will therefore close coverage. The model assumes every bin is reachable by random stimulus with equal probability. In practice a handful of bins require a specific sequence — a FIFO full while a retry is in flight and a parity error arrives — whose random probability is not $1/n$ but 10^{-9} . **Those bins never close randomly and no amount of compute changes that;** they close with directed tests or constrained scenarios. The arithmetic above tells you how much compute to buy for the easy 99 %, and identifying the remaining bins as requiring direction is the actual verification engineering.

Table 255. Measured transactions to 99 % coverage of 1000 bins as the bin probabilities are made unequal

Skew parameter	Most likely : least likely bin	Transactions measured	Multiple of the equiprobable estimate
0	1	6,000	1.3
0.5	1.65	6,000	1.3
1	2.72	6,000	1.3
1.5	4.47	8,000	1.74

Skew parameter	Most likely : least likely bin	Transactions measured	Multiple of the equiprobable estimate
2	7.37	8,000	1.74

An exponential weighting with the stated skew, measured by simulation with seed 9. Even a 7:1 spread between the most and least likely bin multiplies the cost noticeably; real constraint solvers produce far larger spreads than that, which is why **coverage convergence is slower than any model predicts and why the remedy is to reshape the constraints, not to buy machines.**

X8.2 Bug discovery curves and when to stop

Bugs are found at a rate that decays roughly exponentially with effort, because the easy ones go first. Fitting that decay gives an estimate of how many remain — an estimate that is crude, defensible, and far better than the alternative, which is a feeling.

Table 256. An exponential bug-discovery model: 240 latent bugs, decay constant 0.18 per week

Week	Cumulative found	Bugs found this week	Estimated remaining	Remaining (%)
2	72	30.1	167	69.8
4	123	21	116	48.7
6	158	14.7	81	34
8	183	10.2	56	23.7
10	200	7.14	39	16.5
12	212	4.98	27	11.5
16	226	2.43	13	5.61
20	233	1.18	6	2.73

The parameters are illustrative; the method is not. Fit the two parameters to your own weekly find-rate data and the extrapolation is a real estimate with a real confidence interval.

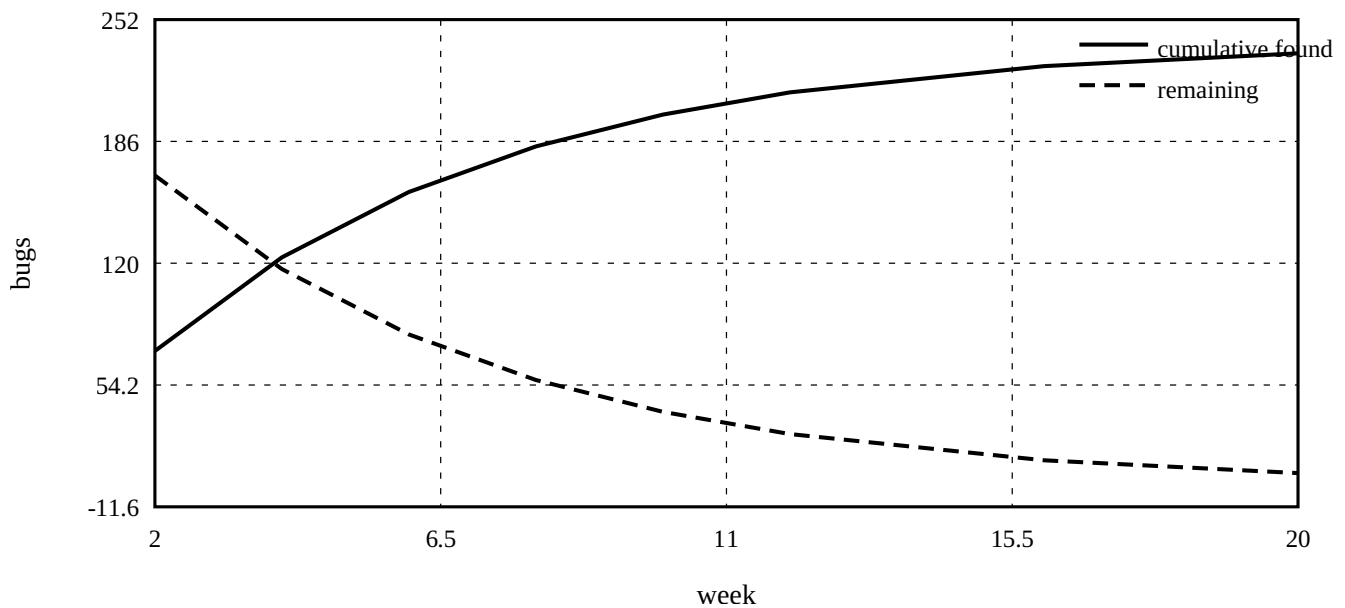


Figure 34. The two curves sum to a constant by construction. The decision to tape out is the decision that the lower curve is acceptable.

The find-rate falling is ambiguous and this is where projects deceive themselves. A falling rate means either that the bugs are running out or that the *testing* is running out — the same tests re-run on the same stimulus find nothing new whether or not bugs remain. The two are distinguished by a measurement, not by an argument: **introduce new stimulus or a new checker and see whether the rate recovers.** Mutation testing does this systematically by injecting known faults and asking what fraction the existing suite catches; a suite that catches 60 % of injected faults is telling you your find-rate curve has flattened for the wrong reason. This repository runs exactly such a hunt against its own checks.

Table 257. Relative cost of finding one bug at each stage, and the cost of letting 2 % of 240 bugs reach that stage

Stage found	Relative cost	vs. spec review	Cost of 2 % escaping to here (relative units)
Specification review	1	—	4.8
Modelling / C++ reference	3	3×	14.4
Block-level simulation	10	10×	48
Chip-level simulation	40	40×	192
Emulation / FPGA prototype	100	100×	480
First silicon bring-up	1,000	1000×	4800
Customer escape	10,000	10 000×	48000

*The multipliers are the industry's rule of thumb and vary by an order of magnitude between sources; the **shape** is what is robust. It is the reason a modelling team that catches specification ambiguities pays for itself many times over, and the reason design houses invest in the left-hand rows.*

Worked example 18. Is another month of verification worth it?

Given. At week 12 the model above estimates 28 bugs remaining. A month of verification costs 4 engineer-months. Each bug that reaches silicon costs, on average, 1 engineer-month of debug; a respin costs 40 engineer-months plus three months of schedule, and the probability of a respin is roughly the probability that at least one remaining bug is functional-critical, estimated at 5 % per remaining bug.

Method. Compute the expected cost of stopping now against the expected cost of continuing, using the discovery model to say how many bugs the extra month removes.

Bugs remaining at week 12	27
Bugs remaining at week 16	13
Bugs removed by the extra month	14
P(no respin) if we stop now	0.242
P(no respin) if we continue	0.501
Expected respin cost now (engineer-months)	30.33
Expected respin cost after one more month	19.96
Saving from the extra month	10.37
Cost of the extra month	4
Verdict	continue

How this is got wrong. By treating the model's parameters as known. The decay constant is fitted from noisy weekly counts and its uncertainty propagates into the remaining-bug estimate, which sits inside an exponent. The right use of this calculation is not to produce a verdict but to find out *which assumption the verdict is sensitive to*: here it is the 5 % criticality figure, and that figure can be improved by classifying the bugs already found, which is free. **Do the sensitivity analysis and go and measure the sensitive term.**

Problem. Your regression has 100 % code coverage and 98 % functional coverage, and has found no new bug in three weeks. A reviewer asks for evidence that the suite would catch a bug if one existed. What do you provide?

Solution. Code coverage is the weakest of the three and answering with it is the mistake: it shows lines executed, not behaviour checked, and a testbench with the checkers disabled reaches 100 %. The evidence a reviewer should accept is **fault injection**: take the design, introduce a set of realistic mutations — invert a comparison, drop a reset term, change a FIFO threshold by one, delete a wait state — and report the fraction the existing suite catches, with the escapes listed and explained. A suite that catches 95 % of mutations has made a falsifiable claim about itself. Supplement it with **checker coverage** (did each assertion ever fire in a passing run? an assertion that never even evaluated its antecedent is not checking anything) and with the **bug-find-rate recovery test** above: add genuinely new stimulus and see whether the rate moves. **Every one of these is a measurement of the test-bench rather than of the design**, which is precisely what was asked for.

Problem. Formal verification proves a property over all inputs. Why does a project still simulate?

Solution. Three reasons, in decreasing order of how often they bite. **Capacity**: proofs complete on control logic, arbiters, FIFOs and protocol interfaces, and commonly do not on a datapath with wide multipliers or on a whole subsystem; what comes back is a bounded proof, which is a very good simulation and not a proof. **Specification**: a formal tool proves the properties you wrote. A missing property is an unproven behaviour, and there is no coverage metric for ‘properties I did not think of’ — though mutation coverage of the property set comes closest. **Environment**: proofs need constraints to exclude illegal inputs, and an over-constraint silently removes the case that would have failed — the formal equivalent of a false-green, and the one that has ended more than one post-silicon investigation. The two techniques are complementary in a specific way worth remembering: **formal is strong where state space is deep and narrow, simulation is strong where it is shallow and wide.**

X9. Computer Arithmetic Beyond the Adder, Worked

Addition and multiplication are solved problems with known structures. Division, square root, and the transcendental functions are not: each admits several algorithms with genuinely different latency, area and accuracy, and choosing among them is a recurring task for anyone specifying a datapath. This part computes the trade-offs instead of listing them.

X9.1 Adders: the latency–area curve, computed

Table 258. Adder structures: delay in gate levels and area in full-adder equivalents

Width n	Ripple delay	Kogge–Stone delay	Kogge–Stone area	Carry-select delay ($\sqrt{n}$ blocks)	Carry-select area
8	8	8	12	5	16
16	16	10	32	8	32
32	32	12	80	11	64
64	64	14	192	16	128
128	128	16	448	22	256

Delays are counted in two-input gate levels from the standard prefix constructions; they are architectural counts, not library timing. Kogge–Stone is logarithmic in delay and $n \log n$ in area and wiring — and the wiring, not the gates, is what makes it expensive in a real floorplan.

Graduate · Why the prefix-adder family is a design space rather than a single answer. Every parallel-prefix adder computes the same carry prefixes; they differ in how the prefix tree is shaped, and the shape trades three quantities against each other: logic depth, fan-out, and wiring tracks. Kogge–Stone minimises depth at maximum wire count; Brent–Kung minimises wires at twice the depth; Han–Carlson interleaves the two and sits between; Ladner–Fischer trades fan-out for wires. In a modern process **the wiring term usually dominates**, so the textbook's favourite (Kogge–Stone) is often not the synthesiser's choice, and a designer who insists on it by instantiation can make the block slower. The right procedure is to let the tool choose from its library and to check the result, not to specify the topology in the RTL.

X9.2 Multipliers: partial products and how to not add them

Table 259. Partial-product counts and reduction depth

Operand width n	Simple PP rows	Radix-4 Booth rows	Radix-8 Booth rows	Wallace 3:2 reduction levels (from radix-4)	Approx. full adders in the tree
8	8	5	4	3	24
16	16	9	7	4	86
24	24	13	9	5	187
32	32	17	12	6	326
53	53	28	19	7	890

Radix-8 Booth needs a $3\times$ multiple, which costs an adder and a carry chain up front; it pays only when the row saving outweighs that, which for $n \geq 32$ it usually does. The 53 row is the IEEE-754 double-precision significand.

Worked example 19. Why a multiplier ends in one wide adder

Given. A 32×32 multiplier built as radix-4 Booth encoding plus a carry-save reduction tree.

Method. Count the stages and see where the delay goes. Carry-save addition has *constant* delay per level regardless of width, because no carry propagates; only the final conversion from carry-save to binary needs a carry-propagate adder.

Booth rows	17
Reduction levels to reach 2 rows	6
Delay of the tree (≈ 2 gate levels per level)	12 gate levels
Final 64-bit carry-propagate adder (Kogge–Stone)	14 gate levels
Final adder as a fraction of total delay	53.8 %

How this is got wrong. By optimising the reduction tree and leaving the final adder to the synthesiser's default. The final adder is a third of the delay of the whole multiplier, and it is the one part that is a plain carry-propagate problem. It is also the reason **a multiply-accumulate is cheaper than a multiply followed by an add**: keep the product in carry-save form, feed it into the accumulator's reduction tree, and pay for exactly one carry-propagate adder at the end of the whole accumulation rather than one per operation. Any datapath that multiplies and then sums should be examined for this.

X9.3 Division and square root: digit recurrence against Newton

Table 260. Two families, and what each is for

	Digit recurrence (SRT)	Multiplicative (Newton–Raphson, Goldschmidt)
Produces	One radix- r digit per iteration	Doubles the number of correct bits per iteration
Iterations for 53 bits	27 at radix 4	About 4 from an 8-bit seed
Per-iteration cost	A small adder and a quotient-digit selection table	Two full multiplies
Exact remainder	Yes — available for free	No; needs a final correction step
Correct rounding	Straightforward from the remainder	Requires care ; this is where implementations go wrong
Latency	Long, predictable	Short if multipliers are already there
Reuses	Nothing	The existing multiplier array
Chosen when	Area matters, or the remainder is needed (integer division, modulo)	A fast multiplier already exists (every FPU)

The Pentium FDIV bug was a digit-selection table with five missing entries. An SRT divider chooses each quotient digit from a lookup indexed by truncated partial remainder and divisor; the table has redundant regions so that a slightly wrong estimate can be corrected later, which is exactly what makes the algorithm fast and exactly what makes a missing entry hard to find by random testing. The entries were unreachable for almost all operand pairs. The lesson for a verification plan is specific and permanent: **a table-driven arithmetic unit must be verified over the table's index space, not over the operand space.** Random operands reach the interesting table entries with probability near zero; enumerating the index space takes minutes.

Table 261. Iteration counts: Newton–Raphson against SRT

Result bits	Seed table bits	Newton iterations	Multiplies	SRT radix-4 iterations	SRT radix-8 iterations
24	8	2	4	12	8

Result bits	Seed table bits	Newton iterations	Multiplies	SRT radix-4 iterations	SRT radix-8 iterations
24	12	1	2	12	8
24	16	1	2	12	8
53	8	3	6	27	18
53	12	3	6	27	18
53	16	2	4	27	18
113	8	4	8	57	38
113	12	4	8	57	38
113	16	3	6	57	38

Newton's iteration count is logarithmic in the precision and the seed table's size trades directly against it — four extra table bits routinely remove an entire iteration, which is two multiplies. **This is the cheapest optimisation in the whole unit** and is frequently left on the table.

Problem. A block needs $y = a/b$ where b changes rarely and a changes every cycle. What is the right structure?

Solution. Not a divider. Compute $r = 1/b$ once, when b changes, and multiply every cycle. The reciprocal can take as long as it likes — a slow iterative unit, or even a microcoded sequence — because it is off the critical path, and the per-cycle cost collapses to one multiply. **The general principle is worth naming: look for the operand that changes slowly and move work onto it.** The same observation produces the coefficient-folding in a fixed-coefficient FIR, the precomputed tables in a modular-arithmetic unit, and the slow adaptation loop of a SerDes equaliser. The one caution is accuracy: $a \cdot (1/b)$ is not correctly rounded even when both operations are, so a unit claiming IEEE compliance cannot take this shortcut without a correction step.

X9.4 CORDIC: rotations without multipliers

Why the strange constants work

	Step	Justification
1	A rotation by θ is $(x\cos\theta - y\sin\theta, x\sin\theta + y\cos\theta)$.	<i>Definition.</i>
2	Factor out $\cos\theta$: the rotation becomes $\cos\theta$ times a shear with $\tan\theta$.	<i>Algebra. The scale factor is now separate from the shape.</i>
3	Choose θ_i with $\tan\theta_i = 2^{-i}$. Then the shear is two shifts and two adds.	This is the whole trick: restrict the allowed angles so the multiplication becomes a shift.
4	Any angle in the convergence range is reachable as $\Sigma\pm\theta_i$.	<i>Because $\theta_i < \Sigma_{j>i}\theta_j$, so the remaining steps can always correct an overshoot — the same redundancy argument that makes SRT division work.</i>
5	The accumulated scale $\Pi\cos\theta_i \rightarrow 0.607253$ is independent of the angle , because every step rotates by the same magnitude and only the sign varies.	<i>So it can be compensated once, at the end, or folded into surrounding coefficients for free.</i>

Table 262. Measured CORDIC accuracy against iteration count (rotation mode, 41 angles)

Iterations	Scale factor K	Max error in cos	Effective bits	Adds
8	0.60725911	0.00453	7.79	16
12	0.60725296	2.65×10^{-4}	11.9	24
16	0.60725294	2.17×10^{-5}	15.5	32

Iterations	Scale factor K	Max error in cos	Effective bits	Adds
20	0.60725294	9.02×10^{-7}	20.1	40
24	0.60725294	7.05×10^{-8}	23.8	48

One bit of accuracy per iteration, exactly as the theory predicts, and the scale factor converges quickly to its limit. The last column is the cost: two adds and two shifts per iteration and no multiplier anywhere.

Undergraduate review · The three CORDIC modes, and what each computes. In *rotation* mode the algorithm drives the angle accumulator to zero and the coordinates come out rotated — giving sine and cosine, and the complex rotation a mixer or an FFT twiddle needs. In *vectoring* mode it drives y to zero instead, and what accumulates is the original angle — giving arctangent and magnitude, which is exactly the operation a Cartesian-to-polar conversion, an AM detector or a phase detector performs. Substituting hyperbolic angles ($\tanh$ instead of $\tan$) gives a third mode that computes exponentials, logarithms and square roots, with some iterations repeated to preserve convergence. **One structure, three modes, most of the transcendental functions a DSP block needs** — which is why CORDIC survives in fabric where multipliers are the scarce resource.

X9.5 Floating point: what the standard actually requires

Table 263. Floating-point formats in current use

Format	Total bits	Exponent bits	Stored mantissa bits	Significand bits (with hidden 1)	Machine epsilon	Decimal digits
binary16 (half)	16	5	10	11	4.88×10^{-4}	3.31
bfloat16	16	8	7	8	0.00391	2.41
binary32 (single)	32	8	23	24	5.96×10^{-8}	7.22
TF32	19	8	10	11	4.88×10^{-4}	3.31
binary64 (double)	64	11	52	53	1.11×10^{-16}	16
FP8 E4M3	8	4	3	4	0.0625	1.2
FP8 E5M2	8	5	2	3	0.125	0.903

bfloat16 and *binary16* are the same width and differ entirely in how they split it: *bfloat16* keeps *binary32*'s exponent range and throws away precision, which is the right trade for neural-network training because the failure mode there is overflow of a gradient, not rounding of a weight.

Worked example 20. Why $0.1 + 0.2 \neq 0.3$, in bits

Given. IEEE-754 binary64 arithmetic.

Method. 0.1 and 0.2 are not representable: in binary they are infinite repeating fractions, so each is rounded to 53 significant bits. The sum of the two rounded values is then itself rounded. Print the exact decimal expansions.

0.1 stored as	0.1000000000000000555
0.2 stored as	0.2000000000000000110
0.1 + 0.2 =	0.3000000000000000441
0.3 stored as	0.29999999999999998890
Difference	5.55×10^{-17}
Difference in units in the last place	1

How this is got wrong. By concluding that floating point is unreliable. It is exactly reliable: every operation above returned the correctly rounded result, and the standard guarantees that. What is unreliable is the assumption that decimal literals are representable. The consequences for hardware verification are direct: a bit-exact comparison against a reference model is only meaningful if both use the same rounding mode and the same order of operations, **and floating-point addition is not associative**, so a reference model that sums an array in a different order than the RTL will disagree with it while both are correct. Specify the summation order, or specify a tolerance and justify it.

Problem. A neural-network accelerator uses bfloat16 multipliers with binary32 accumulation. Why not accumulate in bfloat16 too?

Solution. Because accumulation is where precision is lost and multiplication is not. A bfloat16 multiply of two 8-bit significands produces a 16-bit exact product, and rounding it back to 8 bits costs one rounding. Summing N such products in bfloat16 costs N roundings, each of half an LSB of a number with only 8 significand bits, and — worse — once the running sum is 2^8 times larger than the next term, that term is **absorbed entirely** and contributes nothing. For N in the hundreds or thousands, which a convolution reduction routinely is, the sum silently stops growing. Binary32 accumulation has 24 significand bits and pushes the absorption threshold out by 2^{16} . **The general rule is that a reduction needs more precision than its operands**, which is why every practical mixed-precision unit, from a DSP MAC to a tensor core, is asymmetric in exactly this way.

X10. Memory: The Block That Decides the Floorplan

In most IP blocks the memory is the largest single object, the slowest path, and the thing that cannot be changed late. It is also the part of a design that RTL hides most completely: a two-dimensional array in SystemVerilog becomes, in silicon, a compiled macro with a fixed aspect ratio, a fixed number of ports and its own timing model. This part computes what that substitution costs.

X10.1 The bit cell and what follows from it

Table 264. Storage cells compared

Cell	Transistors / bit	Relative area	Volatile	Read	Used for
6T SRAM	6	1	Yes	Non-destructive	Caches, buffers, everything on-chip
8T SRAM	8	~1.3	Yes	Separate read port	Low-voltage operation — decouples read and write stability
Dual-port (2RW)	8–10	~1.8–2	Yes	Two ports	FIFOs across clock domains
Register file cell	~8–12	1.5–3	Yes	Many ports	Small, highly ported storage
1T-1C DRAM	1 + capacitor	~0.15	Yes, needs refresh	Destructive — must write back	Main memory, eDRAM
ROM (via / contact)	1 or 0	~0.1	No	Non-destructive	Twiddles, microcode, constants
Flip-flop	~20–24	4–5	Yes	—	What synthesis gives you if you do not instantiate a macro

Worked example 21. The cost of forgetting to instantiate a RAM

Given. A 4 kB buffer, 32 768 bits. The 6T cell is $0.03 \mu\text{m}^2$; a flip-flop with its multiplexer occupies roughly $150\times$ a bit cell once the write-enable logic is counted.

Method. Compute both areas and the array overhead. SRAM macros carry decoders, sense amplifiers and control that a small array cannot amortise; assume 40 % overhead at this size.

Bit-cell area	$983 \mu\text{m}^2$
Compiled macro with 40 % overhead	$1376 \mu\text{m}^2$
Same storage in flip-flops	$1.475 \times 10^5 \mu\text{m}^2$
Ratio	$107\times$
Flip-flop clock power penalty (all bits clocked every cycle)	very large — every bit is on the clock tree

How this is got wrong. By writing an array in RTL and letting synthesis decide. Synthesis will infer flip-flops unless the coding style matches the inference template exactly — and the template differs between tools and between memory compilers. The failure is quiet: the design works, meets timing, and is a hundred times too large. **Always check the synthesis report for inferred registers against your expected memory instances**, and prefer explicit instantiation with a wrapper for anything above a few hundred bits.

X10.2 Aspect ratio, banking and the access-energy curve

Table 265. Same 32 768 bits, six aspect ratios (relative units)

Words (depth)	Bits per word	Bit-line length	Word-line length	Relative access energy	Relative access delay
128	256	128	256	0.23	14.8
256	128	256	128	0.307	20
512	64	512	64	0.538	27.1
1,024	32	1,024	32	1.04	37
2,048	16	2,048	16	2.05	50.8
4,096	8	4,096	8	4.1	70

A tall narrow array has long bit lines, which cost energy and delay; a short wide one has long word lines and wastes energy activating bits it will not return. **The optimum is in the middle and memory compilers know it**, which is why a compiler refuses extreme aspect ratios and folds internally instead.

Graduate · Sub-banking is why a 1 MB cache does not cost 32× the energy of a 32 kB one. A large array is built as many small arrays with a hierarchical decoder, and an access activates one of them. Energy per access therefore grows roughly with the *square root* of capacity rather than linearly, and the same structure bounds the delay growth. This is the reason capacity-versus-energy intuitions borrowed from a small block do not transfer, and the reason a cache model that charges energy proportional to size will mispredict a memory-bound design badly. **When you need real numbers, use a memory compiler's datasheet or a model such as CACTI** — and state which, because the two disagree.

X10.3 Multi-port memory: three ways, all of them compromises

Table 266. Getting more ports than the compiler offers

Technique	Ports obtained	Area cost	Restriction
True multi-port cell	$nRmW$	Grows as $(n+m)^2$ — wires per cell	Only 1RW and 2RW are usually in the compiler library
Banking by address	n accesses if to different banks	Decode and crossbar	Conflicts stall — needs arbitration and back-pressure
Replication (one copy per reader)	$nR1W$	$n \times$ the array	Writes must go to every copy
Double clocking	2 accesses per functional cycle	None in area	Halves the frequency budget ; needs a clean 2× clock
XOR / LVT multi-port	$nRmW$ from 1RW banks	$m(n+m)$ banks plus a live-value table	Complexity; well suited to FPGA, rarely worth it in ASIC

Worked example 22. A register file's port count is a quadratic cost

Given. A 32-entry $\times$ 64-bit register file for a 2-wide machine: 4 read ports and 2 write ports.

Method. Model the cell area as growing with the square of the total port count, which is the standard first-order model because each port adds a bit line and a word line to every cell.

Ports	6
Area relative to a 1R1W cell	9×
Bits	2,048
If widened to 4-wide (8R4W): ports	12
Area relative to 1R1W	36×
Area penalty of doubling issue width	4×

How this is got wrong. By treating issue width as a free architectural parameter. Doubling it quadruples the register-file area and lengthens its access, which is one of the two reasons superscalar width stopped growing (the other is the scheduler's own quadratic wake-up logic). The architectural responses are all about avoiding the quadratic: clustering the register file and accepting a cross-cluster penalty, banking it by register number, or moving to a structure where most operands never enter the file at all. **A cost model that is quadratic in a parameter is a signal to change the structure, not to tune the parameter.**

X10.4 ECC: the arithmetic of protecting a memory

Why SECDED needs exactly the bits it needs

	Step	Justification
1	To correct a single error among k data bits, the syndrome must name one of $k+r$ positions or say 'no error'.	<i>Every correctable case needs its own syndrome value.</i>
2	So $2^r \geq k + r + 1$.	<i>The Hamming bound for single-error correction.</i>
3	Detecting double errors as well costs one more bit: an overall parity.	<i>With it, a double error gives a non-zero syndrome and even overall parity, which single errors never do.</i>
4	SECDED overhead is $r+1$ bits for k data bits.	<i>And because r grows logarithmically, wide words are far cheaper to protect than narrow ones — the single most useful consequence.</i>

Table 267. SECDED overhead against word width

Data bits k	Hamming bits r	SECDED bits	Overhead (%)	Total stored bits
8	4	5	62.5	13
16	5	6	37.5	22
32	6	7	21.9	39
64	7	8	12.5	72
128	8	9	7.03	137
256	9	10	3.91	266
512	10	11	2.15	523

Protecting a 64-bit word costs 12.5 %; protecting a 512-bit line costs 2 %. **This is why ECC is applied to cache lines and DRAM bursts rather than to bytes**, and why a design that must protect narrow fields should consider grouping them first.

ECC changes the timing path and the failure modes, not only the area. The syndrome must be computed and the correction applied before the data is used, which inserts an XOR tree — typically 3 to 5 gate levels for a 64-bit word — into the read path. At high frequency this forces the correction into a later pipeline stage, which means **the consumer must be able to tolerate a late correction or be stalled**, and that is an architectural decision, not an implementation detail. There is a second trap: a memory with ECC that is never scrubbed accumulates single-bit errors until two coincide in one word, at which point a correctable fault population becomes an uncorrectable event. **Scrubbing is part of the ECC design, not an optional extra**, and its period follows from the soft-error rate and the word count.

Table 268. Soft-error arithmetic at 1000 FIT/Mbit (one failure per 10⁹ device-hours per Mbit)

Memory (Mbit)	Errors per hour	MTBF (years, one device)	Errors per device-year	Errors per year across a million units
1	1.00×10^{-6}	114	0.00877	8766
4	4.00×10^{-6}	28.5	0.0351	35064
16	1.60×10^{-5}	7.13	0.14	1.40×10^5
64	6.40×10^{-5}	1.78	0.561	5.61×10^5
256	2.56×10^{-4}	0.446	2.24	2.24×10^6

The last column is the one a product manager needs. A part that sees one upset per device-century produces thousands of field events per year across a large deployment, which is why ECC appears in consumer parts and not only in servers. The FIT figure itself depends on altitude, process and cell design and must come from the foundry.

Problem. Your block has a 2 kB configuration RAM written once at boot and read every cycle thereafter. Does it need ECC, and of what kind?

Solution. It needs protection more than a data buffer does, for a reason that has nothing to do with its size: **a corrupted configuration bit is permanent and systematic**, where a corrupted data word is transient. An upset in a filter coefficient degrades every subsequent output, silently, until the next reboot, and there is no downstream check that will catch it. The economical answer is usually **parity plus a reload path** rather than full SECDED: since the contents are known and re-writable, detection is enough — flag the error, interrupt, and have software rewrite the block. That costs one bit per word instead of eight and needs no correction logic in the read path, so it does not disturb the timing at all. **Choose detection over correction whenever a recovery path exists**; the recovery path is usually cheaper than the correction hardware, and it also covers failures the ECC cannot, such as a corrupted address.

Problem. Why does a FIFO built from a dual-port SRAM need its pointers in flip-flops rather than in the same memory?

Solution. Because the pointers must be readable in the same cycle they are compared, and a synchronous SRAM read has a cycle of latency. The full and empty conditions gate the write and read enables of the current cycle, so a pointer that arrives a cycle late produces a FIFO that overflows by one entry under back-to-back traffic — a bug that appears only at full rate, which is to say only in the system and never in the unit test that writes one word at a time. **The general form of this trap is worth internalising: any state that participates in this cycle's flow-control decision cannot live behind a synchronous read port.** It is the same reason a cache tag array is read a cycle ahead of the data array, and the reason credit counters in an interconnect are registers however large the credit table becomes.

X11. Interconnect: Arbitration, Flow Control and Deadlock

An IP block that talks to anything talks through an interconnect, and the interconnect's properties leak into the block's specification whether or not the specification mentions them. The three that leak hardest are how bandwidth is shared, how back-pressure propagates, and whether the system can lock up. All three are computable and the third is provable.

X11.1 Credit-based flow control: sizing the buffer

Why the buffer must cover the round trip

	Step	Justification
1	The sender may transmit only while it holds credits; each transmission consumes one.	<i>Definition of credit-based flow control.</i>
2	A credit is returned when the receiver frees a buffer entry; the return takes t_{ret} to arrive.	<i>Physical delay: the return path is wires and pipeline stages like any other.</i>
3	For the sender never to stall while the receiver is draining, credits must outlast the round trip.	<i>Otherwise there is an interval in which the sender has no credit and the receiver has space.</i>
4	$N_{\text{buf}} \geq B \cdot \text{RTT}$, where B is the peak rate and RTT is forward latency plus return latency.	<i>The bandwidth–delay product. It is the same formula as a TCP window and for exactly the same reason.</i>
5	Any shortfall shows up as a throughput loss of $N_{\text{buf}} / (B \cdot \text{RTT})$, not as a functional failure.	<i>Which is why an undersized buffer is found in performance bring-up rather than in verification — late, and by the customer.</i>

Table 269. Credit buffer needed against pipeline depth, one flit per cycle

Forward stages	Return stages	Round trip (cycles)	Minimum credits	Buffer bytes at 32 B/flit	Throughput if only one credit is provided
1	1	2	2	8	0.5
2	2	4	4	16	0.25
4	4	8	8	32	0.125
8	8	16	16	64	0.0625
16	16	32	32	128	0.0312

The last column is what an under-provisioned link actually delivers: one flit per round trip. A four-deep pipeline with one credit runs at an eighth of the wire's rate, and nothing in the design is broken.

Worked example 23. A block on a pipelined AXI interconnect

Given. A DMA engine issues read bursts across an interconnect with 6 cycles of forward latency and 6 of response, at 500 MHz, 128-bit data, and must sustain 6 GB/s.

Method. Convert the required bandwidth into outstanding transactions via the bandwidth–delay product, then check it against the interface's ID width, which is what actually limits outstanding transactions on AXI.

Bytes per beat	16
Beats per second required	3.750×10^8
Cycles per second	5.000×10^8
Beats per cycle required	0.75
Round-trip latency	12 cycles
Outstanding beats needed	9
If bursts are 16 beats: outstanding bursts needed	1
Read data buffer required	144 B

How this is got wrong. By sizing the buffer for the burst and not for the round trip. A common specification says ‘one burst of read data’, which permits exactly one outstanding burst and throttles the engine to burst-length divided by round-trip — here about a quarter of the wire rate. The second frequent error is to size for the *average* latency: the interconnect's latency has a tail whenever another master is active, and a buffer sized for the mean stalls whenever the tail occurs. **Size for the latency at the load you will actually run at**, and state that load in the datasheet, because a customer whose system is busier than your bench will not reach your quoted bandwidth.

X11.2 Arbitration: fairness has a precise meaning and a cost

Table 270. Measured mean / maximum wait in cycles, four masters offering 0.5, 0.2, 0.15 and 0.1 requests per cycle, 200 000 cycles

Policy	M0 (busiest)	M1	M2	M3 (quietest)
Fixed priority	1.0 / 1	2.0 / 21	3.3 / 29	5.4 / 62
Round robin	1.6 / 4	1.5 / 4	1.5 / 4	1.5 / 4
Oldest first (LRG)	1.4 / 4	1.8 / 4	2.0 / 4	2.1 / 4

Simulated with seed 13. Fixed priority starves the low-priority masters when the high-priority one is busy — read M3's maximum. Round robin bounds everyone's wait; oldest-first minimises the maximum further at the cost of an age counter per requester.

Table 271. Arbitration policies and what each guarantees

Policy	Guarantees	Cost	Fails at
Fixed priority	Lowest latency for the top requester	A priority encoder — cheapest	Starvation: no bound for the bottom requester
Round robin	Bounded wait of $n-1$ grants	A rotating pointer	Does not respect differing bandwidth needs
Weighted round robin	Bandwidth in proportion to weights	Per-requester counters	Latency for a low-weight but urgent requester
Deficit round robin	Bandwidth proportion with variable-size requests	A deficit counter per requester	— this is the standard answer for packets

Policy	Guarantees	Cost	Fails at
Oldest first	Minimises maximum wait	Age counters and a comparison tree — the tree is the timing problem	Scaling: the comparison is $O(\log n)$ deep and wide
Lottery / ran- domised	Proportional in expectation	An LFSR	No bound , only an expectation — unacceptable for real-time

Graduate · **Quality of service is not a policy, it is two different requirements that must be separated before a policy can be chosen.** A display controller needs *guaranteed bandwidth* and tolerates latency, because it has a line buffer; a CPU fetching a cache miss needs *low latency* and consumes little bandwidth. One arbiter cannot serve both well with a single knob, which is why real interconnects separate them: a bandwidth regulator (a token bucket or a deficit counter) enforces the first, and a priority class enforces the second, with the regulator demoting a requester that exceeds its allocation so that its priority cannot starve anyone. **When writing an IP block's requirements, state which of the two you need;** 'high performance' is not an interconnect requirement and cannot be designed against.

X11.3 Deadlock: a property of the dependency graph

The four conditions, and which one hardware removes

	Step	Justification
1	Mutual exclusion: a resource is held by one holder at a time.	<i>True of every buffer; not removable.</i>
2	Hold and wait: a holder may request another resource while holding one.	<i>True of any pipelined request; removing it means allocating everything at once, which wastes buffers.</i>
3	No pre-emption: a buffer cannot be taken back.	<i>Removable — by dropping and retrying, which is what a network does and what an on-chip fabric usually cannot afford.</i>
4	Circular wait: the wait-for graph contains a cycle.	<i>This is the one hardware removes.</i> Break every cycle and deadlock is impossible regardless of the other three.
5	Virtual channels break cycles by splitting one physical buffer into several logically independent ones with an ordering between them.	<i>A message may only move from a lower-numbered channel to a higher one, so the dependency graph becomes acyclic by construction.</i>
6	Dimension-ordered routing breaks cycles in a mesh by forbidding a turn.	<i>X then Y: the turns that would close a cycle are simply not permitted. Cheap, and it costs path diversity, which costs throughput under adversarial traffic.</i>

The most common on-chip deadlock has nothing to do with routing. It is a read response that cannot make progress because the response buffer is full of responses waiting behind a write that cannot complete because the write channel is blocked by a request from the same master. Protocol specifications address this with ordering rules — AXI's requirement that read and write channels be independent, and that a slave must not make a read response depend on a future write — and **those rules are routinely broken by IP blocks that share one internal queue between the two channels.** If your block merges AXI's read and write paths into a single ordered structure anywhere, you have created this deadlock, and it will appear in a customer's system and not in yours.

Problem. A bridge accepts an AXI write and must issue a read to fetch the rest of a partially written cache line, then complete the write. Why is this a deadlock hazard, and what are the fixes?

Solution. It creates a dependency from the write channel to the read channel. If the read travels through the same interconnect and that interconnect is congested with writes — including this one, which is holding a slot — the read cannot be issued and the write cannot retire. The wait-for graph has a cycle. Three fixes, in increasing order of cost and decreasing order of fragility. **Reserve resources before accepting:** do not accept the write until a read slot and a line buffer are already allocated, so the bridge never holds and waits. **Use a separate virtual or physical channel** for the fetch, so the read's path cannot be blocked by writes. **Buffer the entire write** and acknowledge it immediately, converting the dependency into a local one — which costs a full line of storage per outstanding write and changes the coherence behaviour. Whichever is chosen, **the argument must be written down as an acyclicity claim about the dependency graph**, because it is not something a simulation will refute reliably: deadlocks appear only under specific congestion, and the bench that would produce it is rarely the bench that is written.

X11.4 Topology and the wire budget

Table 272. Topologies: links and worst-case hop count

Nodes	Full crossbar links	Ring: worst hops	2-D mesh: worst hops	Hypercube: worst hops	Ring links
4	6	3	2	2	2
8	28	7	—	3	4
16	120	15	6	4	8
32	496	31	—	5	16
64	2,016	63	14	6	32

The crossbar column is the reason crossbars stop at modest radix: the link count is quadratic and each link is a full-width bus. Beyond about 16 ports a hierarchy of smaller crossbars wins on area even though it adds hops.

Worked example 24. Bisection bandwidth decides, not link count

Given. A 16-node system, 128-bit links at 1 GHz. Compare a ring, a 4×4 mesh and a full crossbar under uniform random traffic, where on average half the traffic crosses the bisection.

Method. Compute the bisection bandwidth — the bandwidth across the worst cut that divides the network in half — and compare with the offered load. Bisection, not aggregate link bandwidth, is what limits uniform traffic.

Per-link bandwidth	16 GB/s
Ring: links crossing the bisection	2
Ring bisection bandwidth	32 GB/s
4×4 mesh: links crossing	4
Mesh bisection bandwidth	64 GB/s
Crossbar bisection bandwidth	128 GB/s
Offered load if each node sends 8 GB/s, half crossing	64 GB/s
Ring sufficient?	no
Mesh sufficient?	no
Crossbar sufficient?	yes

How this is got wrong. By quoting aggregate bandwidth. The ring in this example has 16 links and 256 GB/s of aggregate link bandwidth, which sounds ample and is irrelevant: uniform traffic is limited by the two links at the cut. The corollary is that **topology should be chosen from the traffic pattern**. If the real traffic is nearest-neighbour, the ring's bisection never binds and it is the right answer; if it is all-to-all, only the bisection matters. An interconnect specified without a traffic model has not been specified.

X12. Data Converters and PLLs, Worked

Very few IP blocks are purely digital at their boundary. A SerDes has an analogue front end; a sensor interface has a converter; every synchronous block has a clock that came from a PLL. A digital designer does not have to design these, but does have to *budget* for them, and budgeting requires the same arithmetic the analogue designer uses. This part supplies it.

X12.1 The converter equations, and what they leave out

SNR, ENOB and the $6.02N+1.76$ formula

	Step	Justification
1	Quantisation error is bounded by $\pm\Delta/2$ with variance $\Delta^2/12$, if it is uniform.	<i>Part X1 established this and also measured where it fails.</i>
2	A full-scale sine of peak $A = 2^{N-1}\Delta$ has power $A^2/2$.	<i>RMS of a sinusoid.</i>
3	$\text{SNR} = 10\log_{10}(A^2/2 \div \Delta^2/12) = \mathbf{6.02N + 1.76 \text{ dB}}$.	<i>Substitute and simplify. The 1.76 is $10\log_{10}(1.5)$, the sine's crest-factor advantage over a uniform signal.</i>
4	Real converters fall short; the shortfall is expressed as ENOB = (SNDR – 1.76)/6.02 .	<i>Invert the formula using the measured signal-to-noise-and-distortion ratio. ENOB is a measurement, N is a pin count, and quoting the second as if it were the first is the standard datasheet deception.</i>
5	Aperture jitter adds a noise term $\text{SNR}_{\text{jitter}} = -20\log_{10}(2\pi f_{\text{in}}\sigma_t)$.	<i>The sampled error is the signal's slew times the timing error; slew is proportional to input frequency. This term does not depend on N at all and eventually dominates every high-speed converter.</i>

Table 273. Effective bits achievable with 1 ps RMS aperture jitter, against nominal resolution and input frequency

Nominal N	Ideal SNR (dB)	ENOB at 1 MHz	at 10 MHz	at 100 MHz	at 1 GHz
8	49.92	8	8	7.97	6.86
10	61.96	10	10	9.65	7.01
12	74	12	11.9	10.3	7.02
14	86.04	14	13.3	10.3	7.02
16	98.08	15.8	13.6	10.3	7.02

Read down the last column: **at 1 GHz input, 1 ps of jitter caps the converter at about 8 bits no matter how many bits the part has**. Buying a 16-bit converter without buying a cleaner clock buys nothing.

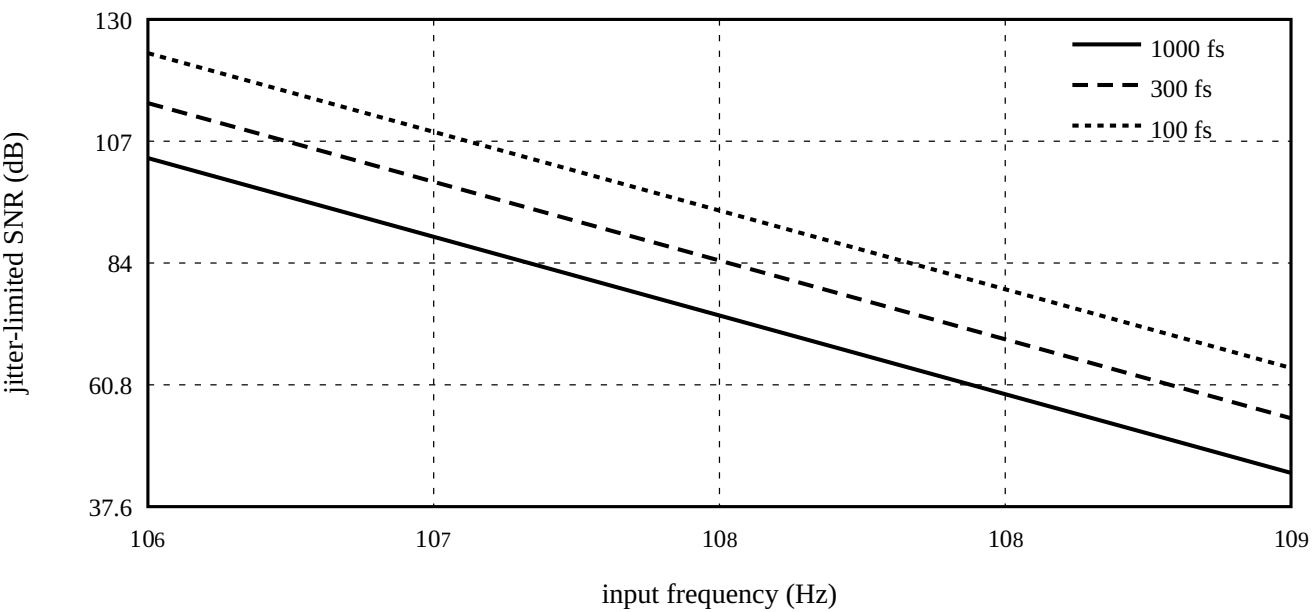


Figure 35. The jitter wall. Every curve falls 20 dB per decade of input frequency; the only way up is a better clock.

Worked example 25. Specifying the clock for an ADC, backwards from the requirement

Given. A receiver needs 11 effective bits on a 500 MHz input. The converter's own thermal and distortion contribution is 13 ENOB.

Method. Convert the requirement to an SNR, subtract the converter's own contribution in the power domain to find the jitter budget, then invert the jitter formula.

Required SNDR	67.98 dB
Converter's own SNR	80.02 dB
Permitted jitter-noise SNR	68.26 dB
Permitted RMS aperture jitter	123 fs
Is that achievable with an on-chip PLL?	demanding — it is at the edge of a good integrated LC PLL

How this is got wrong. By subtracting decibels instead of powers. Noise contributions add in power, so budgets must be combined as $10^{-\text{SNR}/10}$ terms and only then converted back. Subtracting 11 from 13 in dB gives a meaningless number. The second error, subtler, is to forget that the *clock buffer* between the PLL and the sampler adds its own jitter, often more than the PLL, so a budget that spends the whole allowance on the PLL has already failed.

Table 274. Converter architectures and where each lives

Architecture	Resolu- tion	Speed	Latency	Cost driver
Flash	4–6 bits	Highest	1 cycle	2^N comparators — area doubles per bit
Pipelined	10–14 bits	High	$N/2$ cycles	Op-amps; inter-stage gain accuracy
SAR	8–16 bits	Moderate	N cycles	Capacitor matching; scales beautifully with process
Sigma-delta	16–24 bits	Low signal band- width	High (filter)	Oversampling ratio; digital decimation filter

Architecture	Resolution	Speed	Latency	Cost driver
Time-inter-leaved	8–12 bits	Extreme	As the sub-ADC	Mismatch between channels — gain, offset and timing skew, all needing digital calibration

Graduate · Time interleaving is where the digital team earns its place in an analogue block. Running M converters in round robin multiplies the sample rate by M and introduces three mismatches: offset, which appears as spurs at multiples of f_s/M ; gain, which appears as spurs around the signal; and timing skew, which appears as signal-dependent spurs that grow with input frequency. None is removable by better layout alone at modern speeds; all three are corrected by **digital background calibration** that estimates the mismatch from the data and applies a correction. That estimator is a piece of DSP with a convergence time, a lock detector and a failure mode, and it is specified, modelled and verified by the digital side. **A SerDes receiver's ADC is time-interleaved 32 or 64 ways, and its calibration loops are a substantial part of the digital block's area.**

X12.2 Phase-locked loops: the transfer functions that matter

Why a PLL is a lowpass for reference noise and a highpass for VCO noise

	Step	Justification
1	Open loop: $G(s) = K_{pd}F(s)K_{vco}/s$, divided by N .	Phase detector gain, loop filter, VCO as a phase integrator, feedback divider.
2	Reference to output: $N G/(1+G)$.	Standard feedback algebra. At low frequency $G \gg 1$ so this is N ; at high frequency it rolls off. Lowpass.
3	VCO noise to output: $1/(1+G)$.	The VCO's noise enters after the phase detector. At low frequency the loop suppresses it; above the bandwidth it passes. Highpass.
4	So the loop bandwidth is a crossover, not a quality setting.	Wide bandwidth tracks the reference and suppresses VCO noise; narrow bandwidth rejects reference noise and lets the VCO run free. The optimum is where the two noise densities cross, and that point is measured , not chosen by taste.
5	Total output jitter is the integral of the combined phase-noise spectrum over the band of interest.	Which is why a jitter specification without an integration band is incomplete — the same part can be quoted at 200 fs or 2 ps depending on limits.

Table 275. Integrated jitter of a 2.5 GHz PLL against loop bandwidth, with a toy noise model (reference -100 dBc/Hz flat, VCO -80 dBc/Hz at 1 MHz, $N = 20$)

Loop bandwidth (MHz)	Output phase noise at 10 kHz (dBc/Hz)	at 1 MHz (dBc/Hz)	Integrated RMS jitter 10 kHz–100 MHz (fs)
0.1	-59.87	-79.87	35204
0.5	-70.97	-77.96	22387
1	-73.01	-76.02	25070
2	-73.72	-74.69	32635
5	-73.94	-74.11	49869

Integrated numerically over the stated band. There is a minimum: too narrow and the VCO dominates, too wide and the reference multiplied by N^2 dominates. **The $20\log_{10}N$ multiplication of reference noise is the term designers forget, and it is 26 dB here.**

A PLL specification needs five numbers and is usually given two. Output frequency and jitter are not enough. The five are: output frequency range; **integrated jitter with its integration band**; spurious tone level, since a single reference spur can violate a mask that broadband jitter passes; lock time, which sets how long a power-gated block takes to become usable; and the **input reference's own phase noise**, since the PLL multiplies it by $20\log_{10}N$. A block that is specified without the fifth will work on the vendor's evaluation board with a laboratory clock source and fail in the customer's system with a crystal oscillator.

Table 276. Reference phase noise is multiplied by the divide ratio

Divide ratio N	$20\log_{10}N$ (dB)	Output noise from a -100 dBc/Hz reference	Output noise from a -130 dBc/Hz reference
2	6.021	-93.98	-124
4	12.04	-87.96	-118
10	20	-80	-110
20	26.02	-73.98	-104
50	33.98	-66.02	-96.02
100	40	-60	-90

In-band only — above the loop bandwidth the reference is filtered. This is the reason a high-speed SerDes wants a low reference multiplication, and the reason a 100 MHz reference often beats a 25 MHz one for the same output frequency.

Problem. A clock-and-data-recovery loop and a frequency-synthesis PLL are both PLLs. Why is the CDR's bandwidth chosen by a standard rather than by the designer?

Solution. Because in a CDR the loop bandwidth determines **jitter tolerance**, which is an interoperability property rather than a performance one. Jitter below the loop bandwidth is tracked by the receiver and causes no error; jitter above it is not tracked and eats the eye directly. So the receiver's bandwidth and the transmitter's permitted jitter spectrum are two halves of one specification: if each vendor chose its own bandwidth, a compliant transmitter and a compliant receiver could still fail together. Standards therefore specify a **jitter tolerance mask** — how much sinusoidal jitter the receiver must survive at each frequency — which constrains the loop bandwidth from below, and a **jitter transfer limit**, which constrains it from above so that jitter is not amplified around the loop's peaking and accumulated through a chain of repeaters. **The designer's freedom is what remains between the two masks**, and in modern standards that gap is narrow.

Problem. Why does a fractional- N synthesiser need a delta-sigma modulator, and what does it cost?

Solution. A divider divides by an integer. To reach a fractional average the divide ratio must be switched between neighbouring integers, and the *pattern* of switching determines where the resulting phase error lands in frequency. A simple periodic pattern puts it at a low frequency inside the loop bandwidth, where it appears as a **fractional spur** — a discrete tone that violates masks. A delta-sigma modulator chooses the pattern so that the quantisation error is *shaped* to high frequency, where the loop filters it. The costs are real: the shaped noise is larger in total than the unshaped noise, so the loop bandwidth must be kept below the point where the shaping rises; a higher-order modulator shapes harder but can produce idle tones if its input sits near a simple fraction, which is why implementations dither the input; and the divider must now handle a range of ratios, which complicates its design. **This is a clean example of a digital block bought to relax an analogue constraint**, which is the dominant trend in mixed-signal design and the reason the boundary keeps moving toward the digital side.

X13. Machine-Learning Hardware, Worked

Neural-network accelerators are the largest category of new IP by value, and they are also the category where the gap between a quoted peak number and delivered performance is widest. The arithmetic that closes that gap is the roofline model and its refinements, and it is worth doing carefully because it decides the architecture before a line of RTL is written.

X13.1 The roofline, computed for real layers

Arithmetic intensity and where the roof bends

	Step	Justification
1	A kernel performs W operations and moves Q bytes.	<i>Definitions.</i>
2	Arithmetic intensity $I = W/Q$, operations per byte.	<i>A property of the algorithm and the dataflow, not of the machine.</i>
3	Achievable performance = $\min(P_{\text{peak}}, I \cdot B)$, with B the memory bandwidth.	<i>You cannot compute faster than the machine, nor faster than the data arrives.</i>
4	The ridge point is at $I = P_{\text{peak}}/B$.	<i>Below it the kernel is memory bound and peak FLOPS are irrelevant.</i>
5	For a modern accelerator P_{peak}/B is in the hundreds.	<i>Which means most layers are memory bound unless the dataflow is designed to raise I — and raising I is what the whole accelerator architecture is for.</i>

Table 277. Arithmetic intensity of common layers, int8, no reuse beyond the layer

Layer	MOPs	Bytes moved (kB)	Ops per byte	Bound by (ridge = 200)
Conv 3×3, 256 → 256, 56×56	3699	2144	1685	compute
Conv 1×1, 512 → 128, 28×28	102.8	554	181.1	memory
Conv 3×3 depthwise, 512, 14×14	0	98.19	0	memory
Fully connected 4096 → 4096	33.55	16392	1.999	memory
Attention QK ^T , $d=64$, $n=1024$	1.374×10^5	1.114×10^6	120.5	memory

Computed assuming each weight and activation is read once from off-chip, which is the worst dataflow. **The point of an accelerator's on-chip memory hierarchy is to make this table wrong** by reusing data; how much it can reuse is the design.

Graduate · Depthwise convolution is the instructive row. It has excellent parameter efficiency — which is why MobileNet-style networks use it — and terrible arithmetic intensity, because each weight is used for only one input channel. On an accelerator designed around dense convolution it can run at a few per cent of peak, so a network that is 4× cheaper in operations can be *slower* in wall-clock time. **This is the clearest example in the field of why operation counts are not a performance metric**, and it is a question worth asking any vendor: not ‘what is your TOPS’ but ‘what is your utilisation on a depthwise layer’.

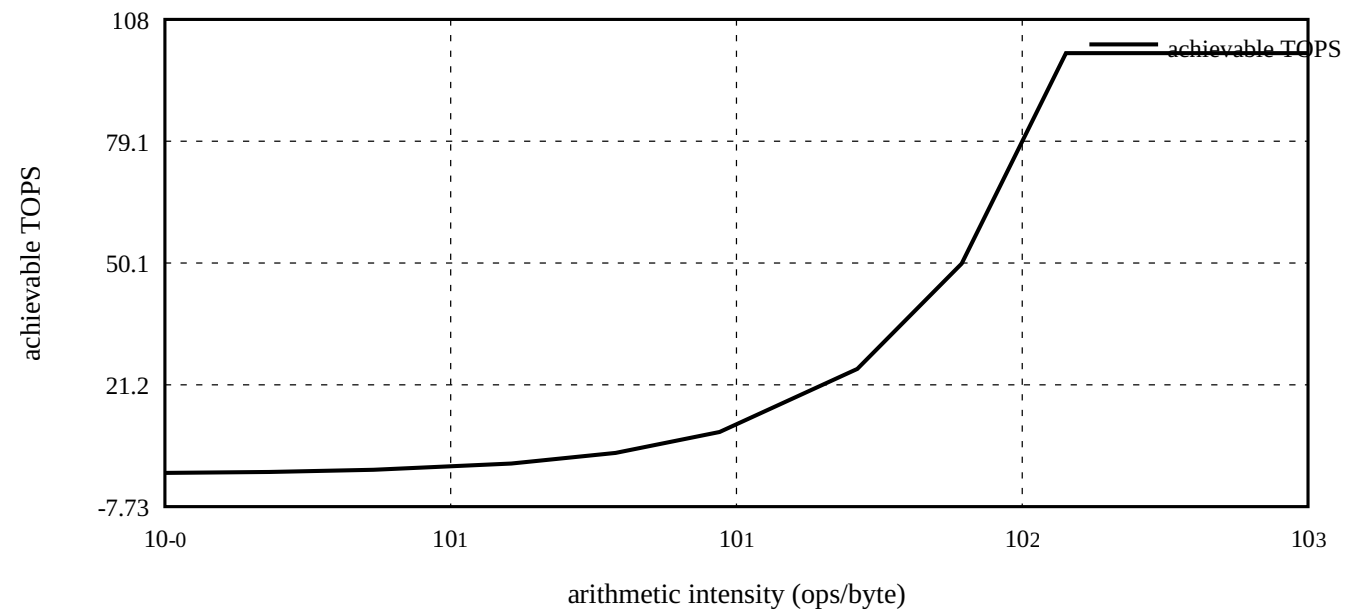


Figure 36. A roofline for a 100 TOPS accelerator with 500 GB/s. The ridge is at 200 ops/byte; everything to its left is a bandwidth problem wearing a compute problem's clothes.

X13.2 Dataflow: what a systolic array is actually reusing

Table 278. Dataflow taxonomy by what stays still

Dataflow	Held stationary	Reuse obtained	Weak when
Weight stationary	Weights in the PE	Each weight is used for a whole batch or feature map	Batch is 1 and the feature map is small
Output stationary	The partial sum in the PE	No partial sums move — saves the widest data	Few accumulations per output (1×1 convolutions)
Input stationary	The activation	Each activation feeds many filters	Few output channels
Row stationary	A row of the convolution in each PE	Balances all three reuse types	Complex control and mapping
No local reuse	Nothing	—	Always — it is the baseline that makes the others look good

Worked example 26. Sizing a systolic array from a bandwidth limit

Given. A 256×256 int8 multiply–accumulate array at 1 GHz, weight stationary. Off-chip bandwidth 100 GB/s.

Method. Compute the peak rate, then compute the byte rate the array would demand with no reuse, then find the reuse factor required to keep it fed.

MACs per cycle	65,536
Peak operations	131.1 TOPS
Operands needed per cycle with no reuse	131,072 bytes
Byte rate demanded	131.1 TB/s
Available	0.1 TB/s
Reuse factor required	1311×
On-chip SRAM to achieve it (order)	megabytes

How this is got wrong. By quoting the TOPS figure and stopping. The array is 131 TOPS and the memory system can feed it 1/1300th of what it wants. Every architectural feature of a real accelerator — the weight buffer, the activation buffer, the tiling, the fusion of adjacent layers — exists to close that factor of 1300, and **the accelerator is really a memory system with multipliers attached**. A design house pitching an accelerator should lead with the memory hierarchy, because that is what a knowledgeable buyer will ask about first.

X13.3 Quantisation: where the bits actually go

Table 279. Measured SQNR of a 512×512 int matrix product, per-tensor vs. per-channel weight scaling

Bits	Per-tensor SQNR (dB)	Per-channel SQNR (dB)	Gain (dB)	Weight storage (kB)
8	39.08	41.29	2.21	256
6	26.8	29.15	2.35	192
4	13.8	16.26	2.47	128
3	6.414	9.003	2.59	96
2	-0.08753	0.6042	0.692	64

Gaussian weights and post-ReLU activations, seed 21. **Per-channel scaling is nearly free in hardware** — one extra multiply per output channel, not per MAC — and buys several dB, which at low bit widths is the difference between a usable and an unusable network.

The outlier problem, and why it changed transformer accelerators. The table above uses Gaussian weights, for which max-based scaling is reasonable. Real transformer activations are not Gaussian: a few channels carry values tens of times larger than the rest, and a per-tensor scale chosen to contain them leaves the remaining channels occupying two or three bits. The measured consequence is that naive int8 quantisation of a large language model loses far more accuracy than the same procedure applied to a convolutional network, and the remedies — per-channel and per-group scales, keeping outlier channels in higher precision, or migrating the scale between activations and weights — all cost hardware that a convolution-era accelerator does not have. **An accelerator's quantisation support is a workload-specific design decision, not a generic feature.**

Problem. Why does int8 inference usually retain accuracy while int8 training does not?

Solution. Because the two use the numbers differently. Inference computes a forward pass whose outputs pass through nonlinearities that are insensitive to small perturbations, and errors do not accumulate across iterations — each inference is independent. Training computes gradients, which are (a) small differences of large numbers, so relative precision matters far more, (b) accumulated over many steps, so bias accumulates exactly as Part X1's truncating accumulator did, and (c) spread over an enormous dynamic range, because gradients of different layers differ by orders of magnitude. The practical resolution is the asymmetric one: **low-precision multiplies with higher-precision accumulation and master weights**, plus loss scaling to move small gradients out of the denormal region. bfloat16 exists for exactly the (c) reason — it sacrifices mantissa, which inference cares about, to keep exponent range, which training cares about.

Problem. A customer wants your accelerator to be ‘future proof’. What can you honestly offer?

Solution. Not architectural coverage of models that do not exist. What you can offer is specific and worth more than a vague claim: **a programmable dataflow** rather than a fixed one, so a new layer shape maps at some efficiency rather than not at all; **a well-specified fallback path** to a host or a DSP for operators the array cannot do, with measured numbers for that path so the customer can compute the Amdahl penalty themselves; **a compiler that is open enough to extend**, since in practice the compiler ages faster than the silicon; and **measured utilisation on a set of named, versioned models**, so the customer can extrapolate from evidence rather than from adjectives. **Offering the measurement methodology is itself a differentiator**: most data-sheets in this field quote one peak number and one optimistic model, and a buyer who has been burned once recognises the difference immediately.

X14. Physical Design as the RTL Designer Sees It

A soft-IP vendor does not run place-and-route on the customer's chip, but the vendor's RTL determines whether that run succeeds. This part covers the physical facts that propagate back into RTL decisions, with the arithmetic that makes them checkable.

X14.1 Wire delay does not scale, and that changes architecture

Why a long wire gets relatively slower every generation

	Step	Justification
1	A wire's distributed RC delay is $0.38rcL^2$ for resistance r and capacitance c per unit length.	<i>Elmore delay of a distributed line. Quadratic in length — the single most important fact in this chapter.</i>
2	Scaling reduces wire cross-section, so r rises roughly as $1/S^2$ while c stays roughly constant.	<i>Thinner and narrower conductor, same dielectric spacing ratio.</i>
3	A wire of <i>fixed physical length</i> therefore gets slower as the process shrinks.	<i>While gates get faster. The ratio moves against the wire every generation.</i>
4	Buffering breaks the quadratic: k equal segments give $k \cdot (L/k)^2 = L^2/k$ plus k buffer delays.	<i>Optimise over k and the delay becomes linear in L.</i>
5	So a long wire costs area and power, and it costs a fixed delay per millimetre that no amount of cleverness removes.	<i>Which makes distance an architectural quantity. A design whose blocks must exchange data every cycle across a chip is not implementable at high frequency, however good the RTL.</i>

Table 280. Wire delay with and without optimal buffering (250 Ω /mm, 200 fF/mm, 20 ps buffers)

Length (mm)	Unbuffered delay (ps)	Buffers	Buffered delay (ps)	ps per mm	Speed-up
0.1	0.19	1	20.19	201.9	0.00941
0.5	4.75	1	24.75	49.5	0.192
1	19	2	49.5	49.5	0.384
2	76	5	115.2	57.6	0.66
5	475	12	279.6	55.92	1.7
10	1900	25	576	57.6	3.3

The constants are indicative of an intermediate metal layer in a mature node and are not from a specific PDK. **The shape is what matters:** the unbuffered column is quadratic, the buffered column is linear, and the per-millimetre figure is roughly constant — which is why architects budget in millimetres.

Worked example 27. How far can a signal travel in one cycle?

Given. A 1 GHz design in the process above. Allow half the period for wire and half for logic and flop overhead.

Method. Divide the available wire time by the per-millimetre delay from the table.

Period	1,000 ps
Wire budget	500 ps
Delay per mm, buffered	57.6 ps/mm
Reachable distance in one cycle	8.68 mm
At 3 GHz	2.89 mm
Typical die edge	10 mm

How this is got wrong. By assuming any module can talk to any module in one cycle. At 3 GHz a signal crosses a fraction of a large die per cycle, so a chip-spanning bus is physically impossible at that frequency and the architecture must be pipelined, hierarchical, or both. **This is the physical origin of networks-on-chip:** not a preference for packets, but the observation that a global broadcast wire no longer exists. For an IP vendor the practical consequence is to keep the block's internal critical paths local and to **register every boundary signal**, so the customer's floorplanner has somewhere to put the pipeline stages the distance demands.

X14.2 What synthesis can and cannot fix

Table 281. Where a problem must be solved

Problem	Can synthesis fix it?	Who must
Slow gate on the critical path	Yes — upsizing, Vt swap	The tool
Badly structured arithmetic	Usually — it re-implements operators	The tool, if the RTL uses + and * rather than hand-built structures
Too many logic levels between flops	No	The RTL — add a pipeline stage
High fan-out net	Partly — it buffers	The RTL, if the fan-out is architectural (a global enable)
Congestion from a crossbar	No	The architecture — reduce the radix or add a stage
Clock domain crossing	No — and it may <i>break</i> one by optimising away a synchroniser	The RTL, plus constraints marking the crossing
Latch inferred from an incomplete case	It will report it	The RTL — always
Multi-cycle path	No	The constraint file, with justification

Synthesis can delete your synchroniser. The two flops of a synchroniser are functionally redundant to a tool that believes the input is a normal signal: it may merge them, retime through them, or move one across a boundary, any of which destroys the resolution time the MTBF calculation of Part X3 depends on. The defences are to mark the flops with a synthesis attribute that forbids merging and retiming, to place them in a module the tool is told not to ungroup, and **to check the gate-level netlist that they are still there** — a check that can be automated and belongs in the sign-off list. A design that relies on the tool's good behaviour here has an MTBF nobody has verified.

X14.3 Utilisation, congestion and the numbers that predict trouble

Table 282. Relative routing difficulty against placement utilisation (heuristic model)

Utilisation (%)	Relative difficulty	vs. 50 %	Practical verdict
50	2.828	1	comfortable
60	3.953	1.4	comfortable
70	6.086	2.15	comfortable
75	8	2.83	tight
80	11.18	3.95	tight
85	17.21	6.09	expect congestion
90	31.62	11.2	expect congestion

The model is a heuristic, not a PDK number: difficulty grows as $(1-u)^{-1.5}$. **Its only job is to make the shape visible** — the cost is mild to 70 % and then rises very fast, which is why floorplans target 65–75 % and why ‘we will just raise utilisation’ is rarely the answer to an area problem.

Graduate · Congestion is caused by pin density and connectivity, not by cell area, and this distinction decides what an RTL designer can do about it. A block that is 60 % utilised can be unroutable if its cells are small and richly connected — a crossbar, a large multiplexer tree, a register file read network — because the routing demand per unit area is set by nets crossing a boundary, not by transistors under it. The RTL-level remedies are therefore about *connectivity*: reduce the radix of a switch and add a stage; replace a fully-connected structure with a hierarchical one; move a wide mux into a memory where the decoder is dense and local; and **avoid the one-hot broadcast structures that look cheap in gate count and are expensive in tracks**. An area report will not show any of this; a congestion map will.

Problem. Your block meets timing standalone at 800 MHz but the customer reports it fails at 600 MHz in their chip. Name four causes, in the order you would check them.

Solution. 1. Different constraints. Your standalone run may have used ideal clocks, no input/output delay, or a different corner set. Ask for their SDC and compare it with yours line by line; this is the cause more often than the other three together. **2. Boundary paths.** Standalone runs frequently constrain inputs and outputs generously; in the chip your block's inputs arrive late and its outputs must arrive early. This is why registering every boundary is worth the latency — it makes your timing independent of their floorplan. **3. Physical context.** In their chip your block may be stretched around a macro, sit far from its clock source, or share power rails with a noisy neighbour, all of which add delay your standalone run did not model. **4. A different library or corner.** Same process, different vendor library or different Vt mix, and your critical path is no longer their critical path. **The general lesson for a vendor is to ship the constraints and the conditions with the frequency claim;** a frequency without them is not a specification, and this exact conversation is why.

Problem. Why do IP vendors quote area in gate equivalents rather than square micrometres?

Solution. Because square micrometres are meaningless without naming the process, the library and the utilisation, and a vendor who names all three has effectively told a competitor which foundry and library they used. A gate equivalent — conventionally the area of a two-input NAND in the same library — is a ratio, so it survives the translation between processes approximately, and it lets a customer scale to their own library by multiplying by their NAND area. **The approximation is worse than it looks,** because the ratio of flop area to NAND area and of SRAM bit-cell area to NAND area both vary between libraries, so a flop-heavy block converts less accurately than a logic-heavy one. The honest datasheet therefore gives **gate equivalents, flop count and memory bits separately**, which lets the customer do the conversion properly, and costs the vendor nothing.

X15. Hardware Security, Worked

Security is where a plausible-looking design is most often wrong, because the adversary is not a random input but a person who reads your datasheet. This part covers the parts of hardware security that are arithmetic — how many traces an attack needs, how much entropy a source really has, how long a key survives — because those are the parts a designer can actually bound.

X15.1 Side channels: the attack is a statistics problem

Why correlation power analysis works, and how many traces it needs

	Step	Justification
1	Dynamic power depends on the number of bits that switch, which depends on the data being processed.	Part X7's $\alpha CV^2 f$: α is data dependent.
2	Choose an intermediate value $v = f(\text{plaintext}, \text{key guess})$ that the device computes, e.g. an AES S-box output.	It must depend on a small part of the key so the guesses can be enumerated.
3	Predict the leakage as a model, commonly the Hamming weight of v .	A crude model that works because switching count correlates with it.
4	Correlate the predicted leakage with the measured power across many traces, for every key guess.	The correct guess correlates; the others do not, because their predicted values are effectively random with respect to the device's actual computation.
5	The number of traces needed is $n \approx 3 + 8(Z_\alpha / \ln((1+p)/(1-p)))^2$.	Fisher's z-transform of the correlation coefficient. Quadratic in $1/p$, which is why reducing the signal by a factor of ten costs the attacker a factor of a hundred in traces.

Table 283. Traces needed for a correlation attack against the leakage correlation ρ (99.99 % confidence)

ρ	Traces needed	Relative to $\rho = 0.5$	Acquisition hours at 1 ms per trace
0.5	109	1	3.03×10^{-5}
0.2	781	7.167	2.17×10^{-4}
0.1	3,181	29.18	8.84×10^{-4}
0.05	12,781	117.2	0.00355
0.02	79,981	733.4	0.0222
0.01	319,981	2934	0.0889
0.005	1,279,981	11737	0.356

Computed from the z-transform expression. **Read the last column as the security margin:** an unprotected implementation falls in minutes, and each factor of ten in leakage reduction costs the attacker a factor of a hundred in time. A countermeasure does not have to be perfect, it has to move this column past the value of the key.

Table 284. Countermeasures, what each does to ρ , and what it costs

Counter-measure	Mechanism	Effect	Cost
Masking	Split each secret into shares whose individual distributions are independent of the secret	Removes first-order leakage entirely; the attacker must combine $d+1$ points, and the trace count grows exponentially in d	2–4× area and a source of fresh randomness every cycle

Counter-measure	Mechanism	Effect	Cost
Hiding (dual rail, WDDL)	Make switching activity constant	Reduces p , does not remove it — routing imbalance leaks	2× area, careful layout
Shuffling	Randomise the order of independent operations	Divides p by roughly the number of permuted slots	Cheap; combines well with masking
Noise injection	Add uncorrelated activity	Reduces p by the amplitude ratio	Power; attacker averages it away — buys time, not security
Frequency randomisation	Jitter the clock	Misaligns traces; defeated by alignment preprocessing	Cheap and weak on its own
Key refresh	Limit the traces available per key	Caps the attack directly	Protocol support

A masked implementation that is not verified at the gate level is usually not masked. Masking is a property of the *computation*, and synthesis optimises computations. If the tool finds that two shares are combined in a common subexpression — which is exactly the kind of optimisation it is good at — the shares meet in one gate and the first-order leakage returns, while the RTL still looks masked. There is a second, subtler failure: even without optimisation, a **glitch** in combinational logic can momentarily present a function of both shares, which is why threshold implementations are designed to be glitch-resistant by construction rather than merely share-correct. **The verification here is formal leakage analysis on the netlist**, not inspection of the source.

X15.2 Entropy: counting what a random number generator really has

Worked example 28. A ring-oscillator TRNG, entropy per bit

Given. Two free-running ring oscillators, one sampled by the other. Jitter accumulates over the sampling interval; the accumulated timing uncertainty is $\sigma = 12$ ps and the sampled oscillator's period is $T = 400$ ps.

Method. The min-entropy of the sampled bit depends on how much of a period the uncertainty spans. A standard stochastic-model estimate is $H \approx 1 - (4/\pi^2)e^{-4\pi^2\sigma^2/T^2}$, which goes to 1 when the jitter covers a period and to 0 when it is small.

σ/T	0.03
Exponent argument	0.03553
Estimated entropy per raw bit	0.6089
Raw bits needed per full-entropy bit	1.642
If σ falls to 3 ps (a quieter supply): entropy	0.5956
Raw bits per full-entropy bit then	1.679

How this is got wrong. By testing the output of the conditioner instead of the source. A cryptographic hash applied to a biased source produces output that passes every statistical test — that is what a hash does — while containing only as much entropy as went in. **The test must be applied to the raw source**, with an online health test that keeps checking it in the field, because the quantity being relied on (σ) depends on supply noise, temperature and ageing, and an attacker with access to the supply can reduce it deliberately. The second row from the bottom is that attack expressed as arithmetic.

Graduate • Why the standards insist on min-entropy rather than Shannon entropy. Shannon entropy is an average; an adversary guesses the most likely value. A source that emits 0 with probability 0.9 has Shannon entropy 0.47 bits and min-entropy $-\log_2(0.9) = 0.15$ bits, and the second number is the one that bounds an attacker's success. For a key of k bits assembled

from such a source, the attacker's work is 2 raised to the total *min*-entropy, not the total Shannon entropy — a factor of $2^{0.32}$ per bit, which over 256 bits is a factor of 10^{24} . **Using the wrong entropy measure is not conservative rounding, it is a wrong security claim.**

X15.3 Fault attacks and the cost of detection

Table 285. Fault-attack countermeasures

Countermeasure	Mechanism	Catches	Overhead
None	—	0 %	0 %
Parity on registers	1 bit per word	single-bit faults	3–12 %
Duplication	Run twice, compare	any single fault	>100 %
Time redundancy	Run twice in sequence	transient faults only	0 % area, 100 % time
Inverse check	Decrypt the ciphertext and compare	any fault in the datapath	~100 % time
Infective computation	Spread the fault so the output is useless	prevents exploitation rather than detecting	moderate

*The classic differential fault attack on RSA-CRT recovers the private key from **one** faulty signature, which is why the inverse check — verify the signature before releasing it — is standard there despite its cost. **When one fault is enough, probabilistic countermeasures are not.***

Problem. A customer asks for a ‘secure’ AES block. What do you ask them before quoting?

Solution. Four questions, because ‘secure’ alone does not specify anything. **What is the attacker's physical access?** A block in a server that an attacker cannot touch needs no side-channel countermeasures; a smart card that the attacker owns needs every one of them, and the area difference is a factor of three. **How many operations per key?** This bounds the traces available and therefore the required leakage reduction directly, via the table in X15.1. **What certification is required?** Common Criteria or FIPS levels prescribe specific countermeasures and specific evidence, and the evidence costs more than the design. **What is the throughput and latency requirement?** Masking costs randomness per cycle, and a design that must run at line rate may not be able to afford the randomness generation. **A quote given without these answers will be wrong by a large factor in one direction or the other,** and a vendor who asks them is demonstrating exactly the competence being bought.

Problem. Why is a hardware root of trust harder than storing a key in fuses?

Solution. Because the key is the easy part. A root of trust must establish, at every boot, that the first code to run is the code the owner intended — which requires immutable boot code, a verified signature chain, and protection of the verification itself from the fault and side-channel attacks above. It must survive the states nobody designs for: reset during an update, a power glitch mid-verification, a debug interface left enabled. It needs a way to revoke a compromised key and to roll forward a version counter that cannot be rolled back, which means monotonic storage, which means more fuses and a policy for them. And it must do all this while remaining **debuggable in the field by the legitimate owner**, which is the requirement that quietly reopens every door the rest closed. **The key in fuses is perhaps five per cent of the work;** the rest is the state machine around it, and that state machine is where the published attacks land.

X16. Reliability, Yield and Functional Safety, Worked

Three quantities decide whether a chip can be sold into an automotive, industrial or medical market, and all three are arithmetic: the fraction of dies that work, the rate at which working dies stop working, and the fraction of dangerous failures that are detected. A design house that can compute them can quote for markets that pay considerably more than consumer.

X16.1 Yield: the model and its parameter

From defect density to die yield

	Step	Justification
1	Defects fall on the wafer at an average density D_0 per unit area.	<i>Empirical; the foundry supplies it and it falls over a process's life.</i>
2	If defects were Poisson and independent, yield would be e^{-AD_0} .	<i>Probability of zero defects in area A.</i>
3	They are not independent — defects cluster — so Poisson is pessimistic for large dies.	<i>Clustering means some wafers are bad and others clean, which raises the average yield of the clean ones.</i>
4	The negative-binomial (Murphy/Seeds) model adds a clustering parameter α : $Y = (1 + AD_0/\alpha)^{-\alpha}$.	<i>$\alpha \approx 2-5$ in practice; $\alpha \rightarrow \infty$ recovers Poisson.</i>
5	Yield falls superlinearly with die area , and total good dies per wafer falls faster still.	<i>Which is the whole argument for chiplets, and the reason a large monolithic die is a business decision as much as a technical one.</i>

Table 286. Yield against die area at $D_0 = 0.10 / \text{cm}^2$, $\alpha = 3$, 300 mm wafer

Die area (mm ²)	Poisson yield	Negative-binomial yield	Gross die/wafer	Good die/wafer	Relative cost per good die
10	0.99	0.9901	6,503	6,438	1.01
25	0.9753	0.9754	2,601	2,537	1.025
50	0.9512	0.9516	1,300	1,237	1.051
100	0.9048	0.9063	650	589	1.103
200	0.8187	0.824	325	267	1.214
400	0.6703	0.687	162	111	1.456
800	0.4493	0.4921	81	39	2.032

The last column is cost per working die relative to a perfect-yield part, holding wafer cost fixed. An 800 mm² die costs far more than eighty times a 10 mm² one, which is the economics that produced chiplets.

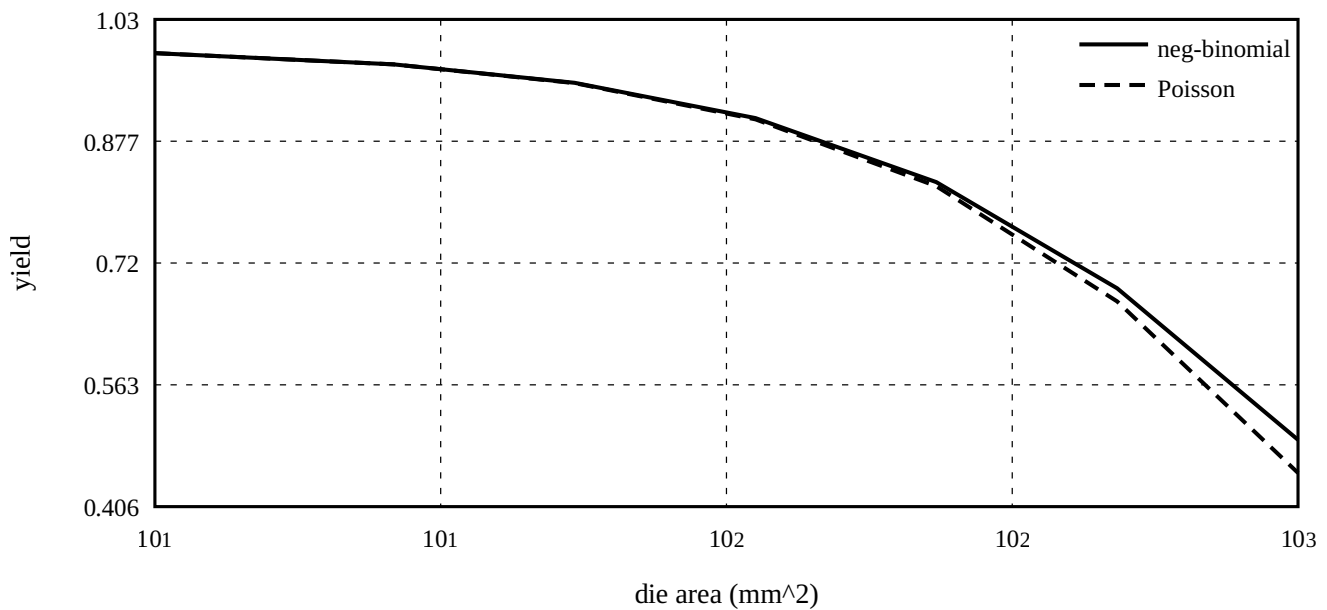


Figure 37. The two models diverge exactly where the decision is hard. Using Poisson for a large die understates yield and can kill a viable product; using it for a small die makes no difference.

Worked example 29. Redundancy in a memory, and when it pays

Given. A 16 Mbit SRAM occupying 20 mm². Adding 2 % area of spare rows and columns with laser or fuse repair can repair any single defect in a row or column.

Method. Compare the yield of the memory with and without repair, using the negative-binomial model, and weigh the 2 % area against the yield gain.

Memory area	20 mm ²
Yield without repair	0.9803
With 2 % area added	0.9799
Defects repairable (assume 80 % are single row/column)	80 %
Effective yield with repair	0.996
Gain	0.016
Area cost	2 %

How this is got wrong. By applying the same reasoning to random logic. Memory repair works because memory is regular: a spare row is interchangeable with any row, so one spare covers a whole class of defects. Random logic has no such structure, and the equivalent — triple modular redundancy — costs over 200 % rather than 2 %. **This asymmetry is why yield-enhancement effort concentrates on memory**, and why a design that is 60 % memory by area has a very different yield story from one that is 60 % logic. It is worth knowing which yours is.

X16.2 Wear-out: the four mechanisms and their accelerations

Table 287. Why parts that passed test fail later

Mechanism	Physics	Accelerated by	Design response
Elec-tromigration	Momentum transfer from electrons moves metal atoms; voids open	Current density and temperature	Width rules per current; the tool checks them if told the currents

Mechanism	Physics	Accelerated by	Design response
NBTI / PBTI	Interface traps shift the threshold voltage under bias	Voltage, temperature, and duty cycle	Guardband; some recovery when the stress is removed, which makes measurement subtle
TDDDB	Gate oxide accumulates damage until it conducts	Electric field, exponentially	Voltage limits; thicker oxide for I/O
Hot carrier injection	Energetic carriers embed in the oxide	Switching activity and short channels	Slew limits; affects high-activity nodes such as clocks
Soft errors	Ionising particle upsets a node	Altitude, cell size, node capacitance	ECC and scrubbing — Part X10

Table 288. Arrhenius acceleration relative to 55 °C, $E_a = 0.7$ eV

Junction temperature (°C)	Acceleration factor	Field years equivalent to 10 test hours	Field years equivalent to 1000 test hours
55	1	10	1000
85	7.953	1.257	125.7
105	26.4	0.3788	37.88
125	77.66	0.1288	12.88
150	259.2	0.03857	3.857

*This is how a fifteen-year automotive lifetime is qualified in weeks: stress at 150 °C and multiply. **The activation energy is per mechanism and is the parameter the whole extrapolation rests on;** quoting a lifetime without naming E_a and the mechanism it was measured for is not a lifetime claim.*

Acceleration factors multiply the error in E_a enormously. At 150 °C the factor above is roughly 170; had E_a been 0.5 eV instead of 0.7, it would be about 40. So a 10 % uncertainty in an exponent's parameter becomes a factor of four in the predicted lifetime, and the direction of the error is not conservative — assuming a high E_a makes the test look more severe than it was. **This is the same structural hazard as the metastability τ of Part X3:** a quantity inside an exponent, measured with modest precision, on which a large claim rests. Both deserve the same treatment: use the worst plausible value, and say which value you used.

X16.3 Functional safety: what ISO 26262 actually asks for

Table 289. The safety metrics, in plain arithmetic

Metric	Definition	ASIL B	ASIL D
SPFM	Fraction of single-point faults that are safe or detected	$\geq 90 \%$	$\geq 99 \%$
LFM	Fraction of latent multi-point faults detected	$\geq 60 \%$	$\geq 90 \%$
PMHF	Probabilistic metric for random hardware failures	< 100 FIT	< 10 FIT
FMEDA	The document that computes all three, gate by gate	Required	Required

Worked example 30. What a 99 % SPFM target means for a block

Given. A block with 500 000 gates and a base failure rate of 10 FIT for the whole block. ASIL D requires $\text{SPFM} \geq 99\%$.

Method. Compute the undetected fault rate the target permits, and then ask what fraction of the design must be covered by a safety mechanism to reach it.

Block failure rate	10 FIT
Permitted undetected single-point rate	0.1 FIT
Fraction of faults needing detection	99 %
Gates that must be covered (if faults are uniform)	495,000
Typical coverage of parity on registers	60–80 %
Typical coverage of duplication and compare	> 99 %
Area cost to reach ASIL D on the datapath	duplication — over 100 %

How this is got wrong. By assuming safety is a verification activity. It is an architecture activity: 99 % coverage is not reachable by testing harder, it is reachable by building detection into the design, and detection costs area that must be budgeted at the start. The usual structure is a **mixed** one — duplication on the control path, where it is small and a fault is dangerous; ECC or parity on memories, where the regularity makes it cheap; and end-to-end checks such as a CRC over a datapath, where a single check covers a great deal of logic. **A safety manual naming those mechanisms, with the coverage claimed for each, is a deliverable a customer pays for.**

Problem. Why is a ‘latent’ fault treated separately, and why does it need periodic testing?

Solution. A latent fault is one in a *safety mechanism* rather than in the function: the comparator that would detect a duplication mismatch has itself failed stuck at ‘equal’. On its own it is harmless — the function still works — which is exactly why it is dangerous: nothing reveals it, so it accumulates, and when a real fault eventually occurs the detection that was counted on is absent. The metric that bounds this is LFM, and the mechanism that serves it is a **periodic self-test**: at start-up and at intervals during operation, deliberately inject a fault and confirm the detector fires. **This is the same argument as the verification agent's self-check in Part Y4 and as this repository's whole rule about checking the checker** — a detector that is never exercised is not known to work, and in a safety context that is not a style preference but a numbered requirement with a quantitative target.

X17. Reading the Current Literature

A textbook that stops at settled material leaves the reader unable to read this year's proceedings. This part covers the directions that a designer entering the field now will meet, treated the way the rest of the book treats everything: by the arithmetic that makes each one attractive and the arithmetic that limits it. No performance claim is attributed to a specific paper here, because this repository's rule is that a citation carries a confidence level and an unread paper cannot carry a high one.

X17.1 Chiplets: the arithmetic that made them inevitable

Part X16 computed that yield falls superlinearly with die area. Chiplets exploit that directly, and the trade is entirely quantitative.

Table 290. Splitting 600 mm² of silicon into *n* chiplets, yield only

Chiplets	Area each (mm²)	Die yield	All- <i>n</i> -good yield	Relative silicon cost	Interfaces to design
1	600	0.5787	0.5787	1.728	0
2	300	0.7513	0.5645	1.772	1
4	150	0.8638	0.5568	1.796	6
8	75	0.9286	0.5529	1.809	28

The fourth column assumes every chiplet must be good, which is the pessimistic case; known-good-die testing before assembly is what makes the scheme work at all, and its cost is the missing term. *The gain is real and the last column is why it is not free.*

Worked example 31. What a die-to-die link costs, and why it decides the partition

Given. Two chiplets exchanging 1 TB/s. A die-to-die interface achieves roughly 1 pJ/bit; an on-die wire of the same length costs roughly 0.05 pJ/bit.

Method. Compute the power of the link and compare with a monolithic implementation. The number decides where the partition can be drawn.

Bits per second	8.00×10 ¹²
Die-to-die power at 1 pJ/bit	8 W
On-die equivalent at 0.05 pJ/bit	0.4 W
Penalty	7.6 W
As a fraction of a 300 W package	2.53 %
Bandwidth affordable in a 15 W budget	1.88 TB/s

How this is got wrong. By partitioning on functional boundaries instead of on bandwidth. The correct cut is the one with the least traffic across it, which is frequently *not* the tidiest functional boundary: splitting compute from memory looks natural and puts the highest-bandwidth interface across the cut, while splitting compute from I/O puts a much lower-bandwidth interface there. **Draw the traffic graph before drawing the partition**, exactly as Part X11's bisection argument requires, and note that the answer changes with the workload — which is why chiplet partitions are argued about for months.

X17.2 Near- and in-memory computing

Why moving computation to the data is attractive, and what bounds it

	Step	Justification
1	Part X13 measured arithmetic intensity: most layers are memory bound.	<i>So the machine spends its energy on movement, not arithmetic.</i>
2	Reading a 64-bit word from DRAM costs on the order of 100× the energy of a 64-bit add.	<i>The ratio varies with process and distance; the order of magnitude is the robust part.</i>
3	If the computation happens where the data is, that ratio is avoided for whatever it can do there.	<i>The gain is bounded by the fraction of work that can move, which is Amdahl's law applied to data movement rather than to time.</i>
4	Analogue in-memory computing performs the multiply–accumulate in the array itself, using conductance as the weight.	<i>One column sums currents; the addition is free and the multiply is Ohm's law.</i>
5	Its limits are precision, variability and the converters.	<i>The ADC at the bottom of each column frequently dominates area and power, which returns the design to Part X12's converter arithmetic. A technique that removes one bottleneck by creating another has to be evaluated on the sum.</i>

Table 291. A crude accounting for an analogue in-memory column: the converter dominates as precision rises (relative energy units)

Output bits	Array MAC energy	Column ADC energy	ADC share of the column (%)	Levels to resolve
2	0.02	0.5	96.2	4
4	0.02	2	99	16
6	0.02	8	99.8	64
8	0.02	32	99.9	256

*The model is illustrative, not measured, and is included to show the shape of the constraint rather than a value. **The shape is the point:** analogue in-memory computing is attractive at low output precision and loses its advantage as precision rises, which is why the published work concentrates on heavily quantised inference.*

Judge any in-memory result on the complete system, including the converters, the peripheral circuits and the data marshalling. An array-only energy figure is the analogue equivalent of a TOPS number: true, and about a part of the system that was never the bottleneck. The questions that separate a result from a product are: what precision at the output, what does the ADC cost, how is a weight updated and how often, what happens to accuracy over temperature and drift, and what fraction of a real network maps onto the array at all. **Each of those is a question this book has asked about a conventional block somewhere, which is the point: the discipline transfers.**

X17.3 Machine learning applied to the design flow itself

Table 292. Where learning is used in EDA, and what the oracle is

Application	What is learned	The oracle	Why it can work
Placement and floorplanning	A policy for placing macros	The downstream tool's own metrics	The reward is computable, if slowly — a real oracle exists

Application	What is learned	The oracle	Why it can work
Routing congestion prediction	A map from placement to congestion	The router's result	Plentiful labelled data from past runs
Timing prediction before routing	Post-route delay from pre-route features	The signoff tool	Saves iterations rather than replacing signoff
Parameter tuning	Good tool settings for a design	The tool's output	The highest-return and least glamorous application
RTL generation from natural language	Code	Nothing, unless you supply one	This is the case that differs in kind, not degree
Testbench and assertion generation	Properties and stimulus	Coverage, mutation score	An oracle exists and is measurable

Graduate · **The organising question for every row is: what verifies the output?** Where a slow but trustworthy oracle exists — the router, the timing signoff tool, a golden model — learning is search, and search with a good oracle is reliable and improves with compute. Where no oracle exists, the output must be read by a human, and the failure mode is code that looks correct, which is the most expensive kind. Part Y4's agent is built on exactly this distinction and is worth re-reading in this light: it searches a small space against a strong oracle and rejects candidates that pass only the data they were fitted to. **The strategic conclusion for a design house is that investment in golden models and checkers appreciates as generation improves**, because they are what converts an unreliable generator into a reliable one, and they are what a competitor who has only a generator cannot borrow.

Problem. A tool vendor claims their learned placer improves timing by 10 %. What do you ask?

Solution. **Against what baseline, tuned how hard?** A learned placer compared against a default-configured conventional flow is compared against a strawman; a human expert spends days on tool settings, and the comparison must include that effort or exclude it from both sides. **On how many designs, and were they in the training set?** Improvement on designs resembling the training distribution is the expected result and says nothing about yours. **What is the variance?** Placement is stochastic; a 10 % mean improvement with a 15 % spread across seeds is not an improvement you can schedule around. **What does it cost in runtime and in setup?** A placer that needs a week of fine-tuning per design is a service, not a tool. **And what happens when it fails?** A conventional flow degrades predictably; a learned one may produce a result that is fine on the metric it optimised and bad on one it did not see. These are the same questions Part Y6 asks of any paper, which is the intended lesson: **the reading discipline does not change because the subject is fashionable.**

Problem. Your customer asks whether your IP is 'AI-generated'. How do you answer?

Solution. Directly, because the question behind it is about provenance and liability rather than about tooling. Say what tools were used and for what; say what verification the output passed, which is the part that actually bounds the risk; and say what you can warrant about the origin of the code — that it is not derived from licensed or copyleft sources, and that you can produce a per-file record. **A customer's real concern is that they will inherit a legal problem or an unverified block**, and both are answerable with evidence you should have anyway. Evasion is the worst answer available: it converts a procurement question into a trust question, and Part Y5's argument about the known-issue list applies exactly — the supplier who volunteers the uncomfortable detail is the one who gets called next time.

X18. Performance Modelling: Queues, Bursts and Tails

Most hardware performance questions are queueing questions in disguise: how deep must this FIFO be, how long will this request wait, what happens when two masters are busy at once. The answers are available in closed form for the simple cases and by simulation for the rest, and having both is what lets a designer say ‘sixteen entries’ with a reason attached.

X18.1 Little's law, which is almost too simple to be useful and is not

Little's law and its three uses

	Step	Justification
1	In any stable system, $L = \lambda W$: average number in the system equals arrival rate times average time in the system.	<i>It requires only stability — no distributional assumption whatever, which is why it is the most portable result in the subject.</i>
2	Use 1: from occupancy and rate, get latency.	<i>Measure the average FIFO occupancy and the throughput, and the latency comes out without instrumenting it.</i>
3	Use 2: from rate and latency, get the buffer needed.	<i>This is the bandwidth–delay product</i> , which Parts X11, Y2 and Y3 each derived separately. They are the same theorem.
4	Use 3: as a consistency check on a simulation.	<i>If measured L, λ and W do not satisfy it, the measurement is wrong — an independent cross-check of exactly the kind Part Y6 insists on, available free.</i>

X18.2 Utilisation and the tail

Table 293. Queue length against utilisation: exponential service (M/M/1) and deterministic service (M/D/1)

Utilisation ρ	Mean queue, M/M/1	Mean queue, M/D/1	Mean time in system (service units)	Depth for 1 % overflow, M/M/1
0.3	0.4286	0.06429	1.429	4
0.5	1	0.25	2	7
0.7	2.333	0.8167	3.333	13
0.8	4	1.6	5	21
0.9	9	4.05	10	44
0.95	19	9.025	20	90
0.99	99	49	100	459

*Deterministic service halves the queue, which is the quantitative reason fixed-size flits and fixed-latency pipelines are preferred in hardware: **re-moving service-time variance is worth as much as halving the load.***

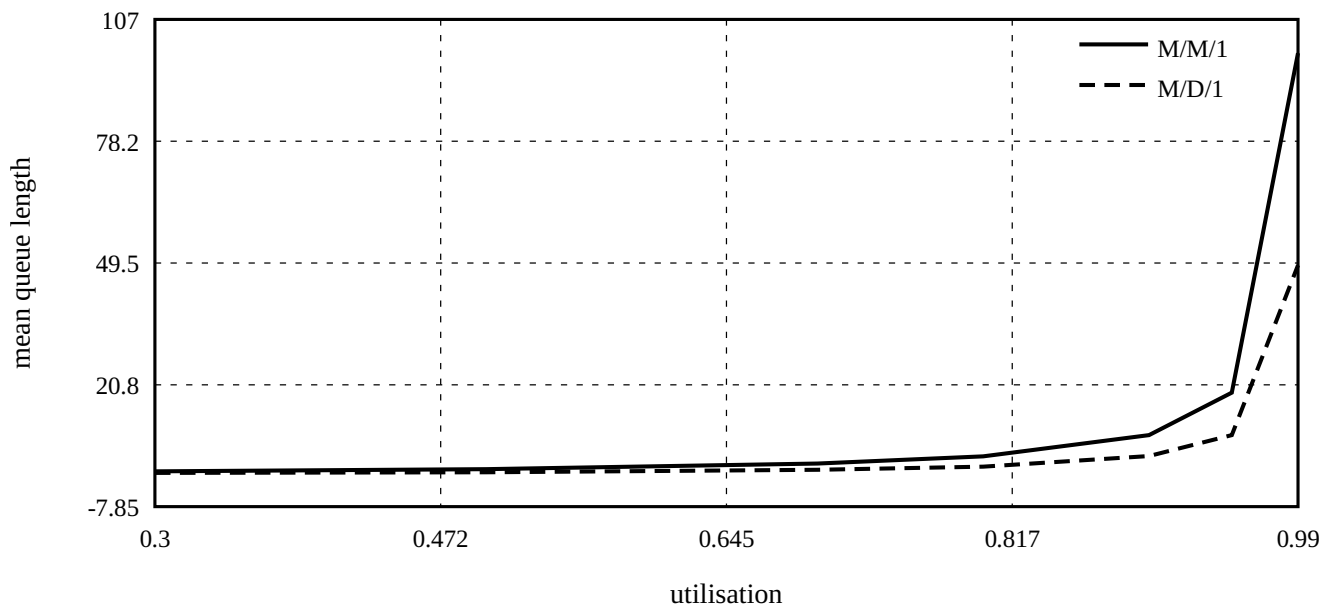


Figure 38. Both curves have a pole at $\rho = 1$. A design that targets 90 % utilisation is targeting the steep part, and small errors in the load estimate become large errors in the buffer requirement.

Graduate · Why hardware designers can be more aggressive about utilisation than the curve suggests, and where that reasoning fails. The classical curves assume Poisson arrivals, which model independent users. On-chip traffic is frequently *self-limited*: a master with four outstanding transactions cannot offer more than four, so the arrival process is closed rather than open and the queue cannot blow up — the right model is a closed network, and utilisation near one is achievable. The reasoning fails exactly when the traffic becomes open: a DMA engine with deep outstanding support, several masters that do not coordinate, or a packet interface fed from outside the chip. **Ask whether the source can be throttled by the queue itself;** if it can, be aggressive, and if it cannot, respect the pole.

Table 294. Measured drop rate of a FIFO against depth, bursty arrivals (8-cycle bursts at 1/cycle, 20 % of slots start a burst, service one every 2 cycles)

Depth	Arrivals	Drops	Drop fraction	Peak occupancy reached
1	39,192	15,642	0.399	1
2	39,192	11,688	0.298	2
4	39,192	3,780	0.0964	4
8	39,192	852	0.0217	8
16	39,192	72	0.00184	16
32	39,192	0	0	28

Simulated with seed 37. The same arrival trace is replayed at every depth, so the arrivals column is identical by construction — only the buffer changes. The average offered load is 0.196 items per cycle against a service rate of 0.5, so the system is **stable on average and still drops at small depths** — because the burst arrives faster than the service can drain it. The depth at which drops stop is set by the burst, not by the average.

Worked example 32. Sizing a FIFO from a burst specification rather than an average

Given. A source delivers a 64-byte burst at one byte per cycle, then idles for at least 192 cycles. The consumer drains at one byte every 2 cycles.

Method. Compute the accumulation during the burst and the drain afterwards; the peak occupancy is the answer, and it depends only on the burst, not on the average.

Burst length	64 cycles
Arrival during burst	64 bytes
Drain during burst	32 bytes
Peak occupancy	32 bytes
Average arrival rate	0.25 B/cycle
Average service rate	0.5 B/cycle
Utilisation	0.5
Depth an average-based calculation would suggest	small — and wrong

How this is got wrong. By computing the utilisation, seeing 0.5, and choosing a shallow FIFO. The system is only half loaded on average and still needs 32 bytes of storage, because the buffer's job is to absorb the *difference between the arrival and service patterns*, not to absorb the average. **The general procedure is to integrate the difference between cumulative arrivals and cumulative service and take the maximum** — a construction that generalises to any traffic pattern you can describe, and that is the basis of the network-calculus bounds used where a hard guarantee is required rather than an average.

Problem. Your interconnect meets its bandwidth requirement in simulation but a customer reports missed real-time deadlines. What is the likely disagreement?

Solution. You measured throughput and they need **bounded latency**, and the two are almost unrelated once utilisation is high. An arbiter that delivers full bandwidth can make one requester wait arbitrarily long — fixed priority does this by construction, as Part X11's measurement showed. The resolution has three parts. **Establish which requester has the deadline and what it is**, because a real-time requirement is a per-requester property and an aggregate number cannot express it. **Measure the latency distribution, not the mean:** report the 99.9th percentile and the maximum observed, since a deadline is violated by the tail. **Then change the mechanism, not the tuning:** a bandwidth regulator plus a priority class, or a reserved slot, gives a bound; adjusting weights gives an average. **No amount of throughput tuning produces a latency guarantee**, and recognising that these are different requirements early saves a redesign late.

X19. Processors as IP, Worked

The processor core is the archetype of licensable IP and the block most likely to sit beside whatever a design house builds. This part treats it the way the rest of the book treats blocks: by the arithmetic that sets its size, its speed and its verification cost.

X19.1 Where the cycles go

The performance equation and what each term belongs to

	Step	Justification
1	Time = instructions × cycles per instruction × cycle time.	<i>The identity. Its value is that the three terms belong to different people.</i>
2	Instructions: the compiler and the instruction set.	<i>A richer ISA lowers this and usually raises one of the other two.</i>
3	CPI: the micro-architecture — pipeline, caches, branch prediction.	<i>Where a core designer spends their effort.</i>
4	Cycle time: the implementation and the process.	<i>Where Part X2's budget applies directly.</i>
5	An improvement that lowers one term and raises another by more is a regression , and this is the standard failure of ISA extensions.	<i>A new instruction that saves two instructions but lengthens the critical path by 5 % has to save 5 % of all instructions to break even.</i>

Table 295. Illustrative core points: performance is frequency divided by CPI

Core	CPI	MHz	MIPS (approx.)	Relative performance
Simple 3-stage, in-order	1.35	400	0.2963	1
5-stage, in-order, no cache	1.2	600	0.5	1.688
5-stage with I/D caches	1.45	900	0.6207	2.095
Dual-issue in-order	0.95	900	0.9474	3.197
Out-of-order, 3-wide	0.55	1,200	2.182	7.364

These are illustrative operating points chosen to show the structure of the trade, not measured benchmark results for any particular core. Note the third row: adding caches raises CPI, because a miss costs cycles the cacheless design never paid — and still wins, because it raises the achievable frequency far more.

Graduate · **The third row is the most instructive line in this chapter.** A design without caches has a low, predictable CPI and cannot run fast, because every access goes to a memory whose latency sets the cycle time. Adding caches introduces misses and therefore raises average CPI, and simultaneously removes the slow memory from the critical path, which raises frequency by more. **Optimising CPI in isolation would have rejected the change.** This is the general hazard of single-metric optimisation, and it is why architectural decisions are evaluated on the product of the three terms and never on one of them.

Table 296. Memory stall as a fraction of CPI

Miss rate (%)	Miss penalty (cycles)	CPI	Stall share of CPI (%)	Relative performance
1	10	1.1	9.09	0.9091
2	10	1.2	16.7	0.8333

Miss rate (%)	Miss penalty (cycles)	CPI	Stall share of CPI (%)	Relative performance
5	10	1.5	33.3	0.6667
2	30	1.6	37.5	0.625
2	100	3	66.7	0.3333
0.5	100	1.5	33.3	0.6667

The last two rows are the same miss penalty at 2 % and 0.5 % miss rates. **At a 100-cycle penalty a 2 % miss rate spends two thirds of every cycle waiting**, which is why a small improvement in hit rate is worth more than a large improvement in almost anything else — and why prefetching, which attacks the penalty rather than the rate, is worth its complexity.

X19.2 What a processor IP licence actually contains

Table 297. The deliverable, and why a core is expensive

Item	Note
RTL or a hard macro	The smallest part of the value
The compiler toolchain	A core without a compiler is unusable. GCC and LLVM ports, a libc, a debugger
Instruction-set simulator	For software development before silicon, and as the golden model for verification
The architectural compliance suite	The thing that makes the core the ISA it claims to be. For RISC-V this is a formal specification plus a test suite
Debug and trace	A standard debug module; trace if the customer needs it
Operating-system ports	Where applicable — Linux, an RTOS
Verification collateral	So the customer can re-verify after configuring
Configuration tooling	A core with 30 options has 2^{30} configurations; the tool generates and verifies the chosen one

The configuration explosion is the hardest verification problem in processor IP and it is worth understanding as arithmetic. A core with thirty independent options has more configurations than can ever be verified individually. What vendors do instead is to verify a *covering set*: choose configurations so that every option and every interacting pair is exercised at least once, which for pairwise coverage needs a number of configurations that grows logarithmically rather than exponentially — typically a few dozen. **The claim a customer should check is not ‘we verified every configuration’ but ‘here is the covering criterion and here is the set’**, and a vendor who can state the criterion is a different proposition from one who cannot.

Table 298. Configuration coverage: exhaustive versus pairwise

Independent options	All configurations	Options to cover singly	Option pairs	Configurations for pairwise coverage (approx.)
5	32	5	10	10
10	1,024	10	45	14
20	1,048,576	20	190	18
30	1,073,741,824	30	435	20

The pairwise column is an order-of-magnitude estimate from the standard combinatorial-testing result that pairwise coverage of n binary factors needs $O(\log n)$ tests. **The gap between the second and last columns is the entire practical argument for combinatorial test design.**

X19.3 Verifying a core against its specification

The code appendix of this book contains the machinery a processor-verification engineer uses, in its real form: **100,524 lines** of the Sail formal specification of RISC-V, **87,547 lines** of the Spike instruction-set simulator, and **9,365,313 lines** of the ar-

chitectural test suites. Reading them together is the most direct way to see what ‘a specification’ means when it has to be executable.

Table 299. Four ways to verify a core, and what each one catches

Method	How	Catches	Misses
Directed tests	Hand-written assembly per feature	What you thought of	What you did not
Architectural suite	The ISA body's own tests	Non-conformance to the ISA	Micro-architectural bugs the ISA does not constrain
Random in-instruction generation	Generate legal programs, run on core and on a reference, compare state	Interactions nobody enumerated	Rare sequences; needs enormous volume
Step-and-compare (co-simulation)	After every instruction, compare the core's architectural state with the reference simulator's	The exact instruction where they diverge — the debug cost collapses	Only what the interface exposes
Formal	Prove properties of the pipeline, the ISA decode, the memory ordering	Deep control bugs, over all inputs	Needs the properties written; capacity limits on the datapath

Graduate · **Step-and-compare is the technique worth internalising, because it generalises far beyond processors.** The idea is to compare not the final output but the state after every step, against a reference that advances in the same steps. The payoff is that the first divergence is the bug, so debug time falls from hours of backward reasoning to inspecting one instruction. The requirement is an interface that exposes committed state — in RISC-V cores this is conventionally a formal interface emitting, for every retired instruction, its program counter, its encoding, and the register and memory it wrote. **Designing that interface into a block is a small cost that pays for itself the first time something goes wrong,** and the same pattern applies to any block with a sequential golden model: a codec, a protocol engine, a FEC decoder.

Problem. Why can a core pass the full architectural test suite and still be broken?

Solution. Because the suite tests the **architecture** and bugs live in the **micro-architecture**. The ISA says what the visible state is after an instruction; it says nothing about what happens when a branch mispredicts while a load is outstanding and an interrupt arrives. Those interactions are where pipeline bugs live, and an architectural test that executes instructions in isolation never creates them. Three further gaps are worth naming. **Memory ordering** under concurrency is barely exercised by single-threaded tests and is the hardest part of a multi-core design to get right; it needs litmus tests derived from the memory model. **Privilege transitions** — traps, interrupts, nested exceptions, debug entry — are under-tested everywhere and are where security bugs concentrate. And **performance counters and debug** are architectural state that customers rely on and suites barely touch. The remedy is the one above: random program generation with step-and-compare, run for a very long time, with a stimulus generator biased toward the interactions rather than toward the instructions.

X20. Image and Video Coding as a Hardware Block

A video codec is the largest fixed-function block in most consumer SoCs and it exercises nearly everything in this book: transforms, arithmetic coding, enormous memory bandwidth, a hard real-time deadline and a standard with a compliance regime. It is also the clearest example of a block whose *decoder* is fully specified and whose *encoder* is not, which has a large commercial consequence.

X20.1 The asymmetry that defines the business

Table 300. Decoder and encoder are different products

	Decoder	Encoder
Specified by the standard	Exactly — bit-exact output is required	Only the bitstream syntax; how to choose it is free
Conformance	Run the conformance streams; output must match bit for bit	Any legal stream is conformant, however bad
Where the value is	Correctness and area	Rate-distortion decisions — the quality at a given bitrate
Differentiation	Little — all correct decoders agree	Large and lasting
Verification	Against the reference decoder, bit-exact	Against a quality metric on a corpus, statistically
A design house should	Build it if the volume justifies it	Build it if you have an algorithmic advantage to sell

Graduate · The bit-exactness requirement on decoders is what makes them verifiable and therefore commoditisable.

Because a conformant decoder's output is defined to the bit, a test is unambiguous: run the standard's conformance streams, compare to the reference output, and the answer is yes or no. That property makes verification tractable and makes every correct decoder equivalent, which drives margins to the cost of implementation. The encoder has no such test — quality is measured statistically on a corpus, and two encoders producing different streams can both be right — which is exactly why encoder quality remains a durable differentiator and why encoder IP is worth more per gate.

X20.2 Bandwidth: the number that sizes the block

Table 301. Frame store and bandwidth for video coding

Format	Frame (MB)	Raw read once (GB/s)	Decoder: ~3× (read ref, write, display)	Encoder: ~8× with motion search
1080p60 8-bit 4:2:0	2.966	0.1866	0.5599	1.493
4K60 10-bit 4:2:0	14.83	0.9331	2.799	7.465
8K60 10-bit 4:2:0	59.33	3.732	11.2	29.86
4K120 10-bit 4:2:2	19.78	2.488	7.465	19.91

The multipliers are indicative of a straightforward implementation and are the quantity every architectural trick attacks. **8K60 encoding at eight passes would need more bandwidth than the whole rest of a mobile SoC**, which is why real encoders use hierarchical search, reference compression and aggressive on-chip tiling.

Worked example 33. Why reference-frame compression exists

Given. 4K60 10-bit decoding, three passes over the frame store, in a system with 25 GB/s of usable DRAM bandwidth shared with everything else.

Method. Compare the requirement with a realistic allocation, then compute what a lossless 2:1 reference compressor buys.

Raw bandwidth required	2.799 GB/s
Plausible allocation to video	6 GB/s
Shortfall	-3.201 GB/s
With 2:1 lossless reference compression	1.4 GB/s
Still short by	0 GB/s
Energy saved at 20 pJ/byte	0.02799 W

How this is got wrong. By treating compression as a capacity optimisation. Its purpose here is *bandwidth and energy*, not storage: DRAM traffic is often the largest single power term in a video subsystem, so halving it halves that term. Note the requirement that the compression be **lossless**: a decoder must produce bit-exact output, so any loss in the reference frames propagates and the decoder is no longer conformant. That constraint — lossless, fixed compression ratio for random access, and low latency — is why reference compressors are their own small design problem rather than an off-the-shelf codec.

X20.3 Entropy coding: the serial bottleneck

Why arithmetic coding resists parallelisation

	Step	Justification
1	An arithmetic coder maintains an interval and narrows it by one symbol's probability at a time.	<i>Definition.</i>
2	The interval after symbol k depends on every previous symbol.	<i>Strictly serial by construction.</i>
3	Context-adaptive coding makes it worse: the probability model is updated from the decoded symbols.	<i>So even the model is serial.</i>
4	Throughput is therefore bins per cycle, and a high-bitrate stream needs several.	<i>Where the decoder's frequency wall comes from.</i>
5	The architectural answers change the standard, not the hardware.	<i>Slices and tiles create independent partitions; wave-front parallel processing allows rows to start once the row above is far enough ahead; bypass bins skip the model. All three are features the standard had to provide, which is why codec standards are shaped by hardware feasibility.</i>

Table 302. Entropy-decoding throughput required at 600 MHz

Bitrate (Mbit/s)	Mbins/s	Bins per cycle needed	Parallel engines at 1 bin/cycle	Utilisation of a 4-bin engine (%)
50	60	0.1	1	2.5
100	120	0.2	1	5
400	480	0.8	1	20
1,000	1200	2	2	50

Assuming 1.2 bins per coded bit. **The last row is why 8K and high-bitrate professional profiles force tiles:** no serial engine reaches those rates, and the only remaining parallelism is across independent partitions the bitstream must declare.

Problem. Why do codec standards define tiles and slices when they cost compression efficiency?

Solution. Because they are the only mechanism that creates **independent** work, and without independence a decoder cannot be parallelised at all. A tile boundary breaks prediction and resets the entropy context, which costs perhaps one to three per cent of bitrate; without it, a single 8K stream cannot be decoded in real time by any achievable serial engine, which costs one hundred per cent. **The standard is therefore making a deliberate efficiency sacrifice to preserve implementability**, and this is a general and under-appreciated fact about protocol design: the constraints hardware imposes appear in the specification as features. Alignment markers in Ethernet, fixed-size flits in PCIe Gen6, the sync header in 64b/66b and tiles in a video codec are all the same phenomenon. **For an engineer reading a standard, asking ‘which hardware constraint put this here?’ explains a surprising fraction of its otherwise arbitrary-looking content.**

Problem. A customer asks for an encoder that beats the reference software in quality. Is that possible, and what does it cost?

Solution. Possible, and the framing matters. Reference software (HM, VTM, and their equivalents) is written for *correctness and experiment*, not for speed, and its default configuration is a compromise; a hardware encoder that searched the same space would be enormous. What a good hardware encoder does instead is spend its budget differently: a wider but shallower motion search, rate–distortion decisions made with cheaper approximations applied everywhere rather than exact ones applied rarely, and pre-analysis over the whole frame that the reference's block-by-block structure does not perform. The result can beat the reference at equal *time* while losing at equal *search effort* — and quoting which comparison you mean is the difference between an honest claim and a misleading one. **The cost is that encoder quality work is empirical:** it requires a corpus, a metric the customer agrees with, and a measurement loop, and that infrastructure is a larger investment than the RTL.

X21. Feedback and Control Inside Digital Blocks

Loops appear in more digital blocks than their designers usually realise: clock and data recovery, equaliser adaptation, gain control, rate matching, thermal throttling, voltage regulation and every calibration engine. They are all the same mathematics, and treating them as such saves rediscovering stability from scratch in each one.

X21.1 The digital loop and where its delay comes from

Why loop delay, not loop gain, usually sets the limit

	Step	Justification
1	A first-order digital loop is $x[n+1] = x[n] + \mu e[n-d]$, with d the delay in the feedback path.	<i>Accumulator plus a delayed error. Every adaptation engine in this book has this form.</i>
2	With $d = 0$ it is stable for $0 < \mu < 2$ (for unit error gain).	<i>Standard result for the integrator.</i>
3	Each unit of delay adds phase lag ωd at frequency ω .	<i>A pure delay is all phase and no magnitude — which is what makes it so destructive: it costs stability margin without showing up in a gain plot.</i>
4	Stability requires the loop gain to have fallen below one before the phase reaches 180° .	<i>Nyquist. With delay d, that caps the bandwidth at roughly $\pi/(2d)$.</i>
5	So μ must be reduced roughly in proportion to $1/d$, and the loop's settling time grows with d twice over.	<i>Once because each correction is smaller, once because corrections arrive later. Pipelining an adaptation loop is not free even though it meets timing.</i>

Table 303. Measured settling of a first-order loop as feedback delay grows (step response to within 2 %, μ set to half the stable maximum)

Delay d (cycles)	Approx. stable μ limit	μ used	Cycles to settle	Normalised bandwidth
0	2	1	0	0.5
1	0.6667	0.3333	4	0.25
2	0.4	0.2	8	0.1667
4	0.2222	0.1111	15	0.1
8	0.1176	0.05882	29	0.05556
16	0.06061	0.0303	58	0.02941

Simulated directly. The settling column is the cost of pipelining: the loop still works at every delay, and it takes an order of magnitude longer at the bottom of the table than at the top.

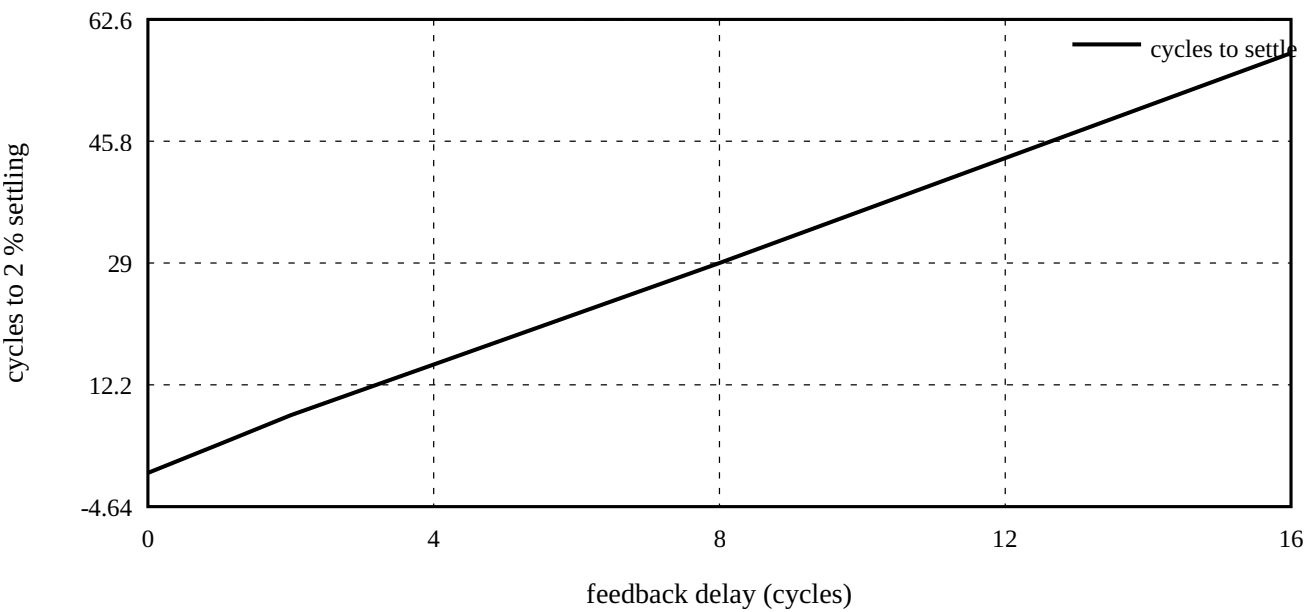


Figure 39. Settling time against loop delay, measured. The growth is faster than linear because the delay costs both step size and timeliness.

Worked example 34. Where the delay in a real adaptation loop comes from

Given. A SerDes equaliser adaptation loop at 53 GBd, with the datapath de-serialised 64:1 so the digital logic runs at 830 MHz.

Method. Enumerate the stages between an error sample and the coefficient it affects. Each is a pipeline register somebody added to meet timing.

Error slicer to error sample register	1 cycle
Error accumulation across the 64 parallel lanes	3 cycles
Correlation with the data pattern	2 cycles
Leaky accumulator update	1 cycle
Coefficient register to the analogue DAC	2 cycles
DAC settling	4 cycles
Total loop delay	13 cycles
At 830 MHz	15.66 ns
Symbols elapsed	832

How this is got wrong. By budgeting only the digital pipeline. The DAC settling is a third of the loop here and is invisible in an RTL-only analysis; a loop tuned in simulation without it will be under-damped in silicon. **Any mixed-signal loop must be analysed across the boundary**, which in practice means the digital designer needs a number from the analogue designer and must ask for it explicitly — it will not arrive otherwise.

X21.2 The loops in a typical block, and what each is for

Table 304. Digital control loops and their characteristic time constants

Loop	Controls	Time constant	Failure if too fast	Failure if too slow
CDR	Sampling phase	10 ³ –10 ⁵ UI	Tracks jitter into the eye; fails jitter transfer	Fails jitter tolerance — cannot follow real wander

Loop	Controls	Time constant	Failure if too fast	Failure if too slow
Equaliser adaptation	Tap coefficients	10^4 – 10^6 symbols	Noisy coefficients; the taps chase noise	Cannot follow temperature drift
AGC	Analogue gain	10^3 – 10^4 symbols	Modulates the signal it is measuring	Clips on a level change
Offset cancellation	Slicer threshold	Slow	Interacts with the data pattern	Residual offset eats the eye
Thermal throttle	Clock frequency	Milliseconds to seconds	Oscillates audibly in performance	Overheats
Rate matching / elastic buffer	Idle insertion and deletion	Per packet	—	Overflow or underflow

Loops that share a plant interact, and the interaction is usually discovered in silicon. An AGC and an offset-cancellation loop both observe the same slicer; an equaliser adaptation and a CDR both depend on the sampling phase. Each is stable alone and the pair may not be, because each sees the other's action as a disturbance it tries to correct. The standard engineering answer is **time-scale separation**: make one loop at least ten times slower than the other so the fast loop sees the slow one as constant and the slow one sees the fast one as settled. **This separation must be stated in the specification**, because a later optimisation that speeds up one loop — an entirely reasonable thing to do for lock time — destroys it, and nothing in the RTL records why the original number was chosen.

Problem. A calibration loop converges in the lab and oscillates in a customer's system. What changed?

Solution. Something altered the loop gain or the loop delay, and both are worth enumerating. **Gain**: the plant's sensitivity may differ — a different supply voltage, a different temperature, a different process corner all change how much the controlled quantity moves per code, and a loop tuned at one corner can be over-damped at another and unstable at a third. **Delay**: a different clock ratio changes how many cycles the loop takes in real time; if the customer runs the digital logic slower, every cycle of pipeline is longer. **Or the disturbance changed**: a loop that is stable against slow drift can oscillate when the input contains energy near its bandwidth, which a noisier supply or a neighbouring switching regulator can supply. **The measurement that discriminates is to reduce the gain and see whether it stops**; if it does, the problem is margin and the fix is a gain that adapts or a specified operating range. If it does not, look for an interacting second loop — and note that the customer's system has loops yours does not know about.

X22. Resampling and Timing Recovery, Worked

Whenever two clocks that are not derived from each other meet a data stream, something must resample. In an audio codec it is a sample-rate converter; in a receiver it is the timing-recovery interpolator; in a video pipeline it is a scaler. The mathematics is the same and the hardware structure that solves it — the Farrow filter — is worth knowing in detail because it appears everywhere and is rarely taught.

X22.1 The problem: a sample that was never taken

Interpolation as filtering, and why a fixed filter is not enough

	Step	Justification
1	An ideal band-limited signal can be reconstructed exactly from its samples by convolution with a sinc.	<i>Sampling theorem.</i>
2	So the value at any intermediate instant μ is a weighted sum of neighbouring samples, with weights $\text{sinc}(k - \mu)$.	<i>Evaluate the reconstruction at that instant. The weights depend on μ.</i>
3	For a fixed μ this is one FIR filter, and a set of L fixed filters gives L discrete phases.	<i>The polyphase interpolator: fine when the phase set is known in advance.</i>
4	Timing recovery needs a μ that varies continuously and is not known in advance.	<i>The sampling phase drifts with the frequency offset between transmitter and receiver, by an amount nobody can tabulate.</i>
5	Farrow's insight: approximate each weight as a <i>polynomial</i> in μ , $h_k(\mu) = \sum c_{k,m} \mu^m$.	<i>Then the interpolator becomes $M+1$ fixed FIR filters whose outputs are combined by Horner's rule in μ.</i>
6	The coefficients are constant; only μ changes.	<i>So the multipliers can be constant-coefficient, and a continuously variable delay costs M general multiplies rather than a new filter per phase.</i>

Table 305. Measured worst-case error of a cubic Farrow interpolator against input frequency, swept over all fractional delays

Normalised frequency	Fraction of Nyquist	Max error	Equivalent SNR (dB)
0.02	0.04	5.84×10^{-6}	104.7
0.05	0.1	2.24×10^{-4}	73.01
0.1	0.2	0.00353	49.03
0.2	0.4	0.0487	26.25
0.35	0.7	0.361	8.843
0.45	0.9	0.758	2.409

Computed directly by comparing against the exact sine. **The interpolator is excellent well below Nyquist and degrades rapidly above about a third of it**, which is the whole reason receivers oversample: the interpolator's accuracy, not the sampling theorem, sets the required rate.

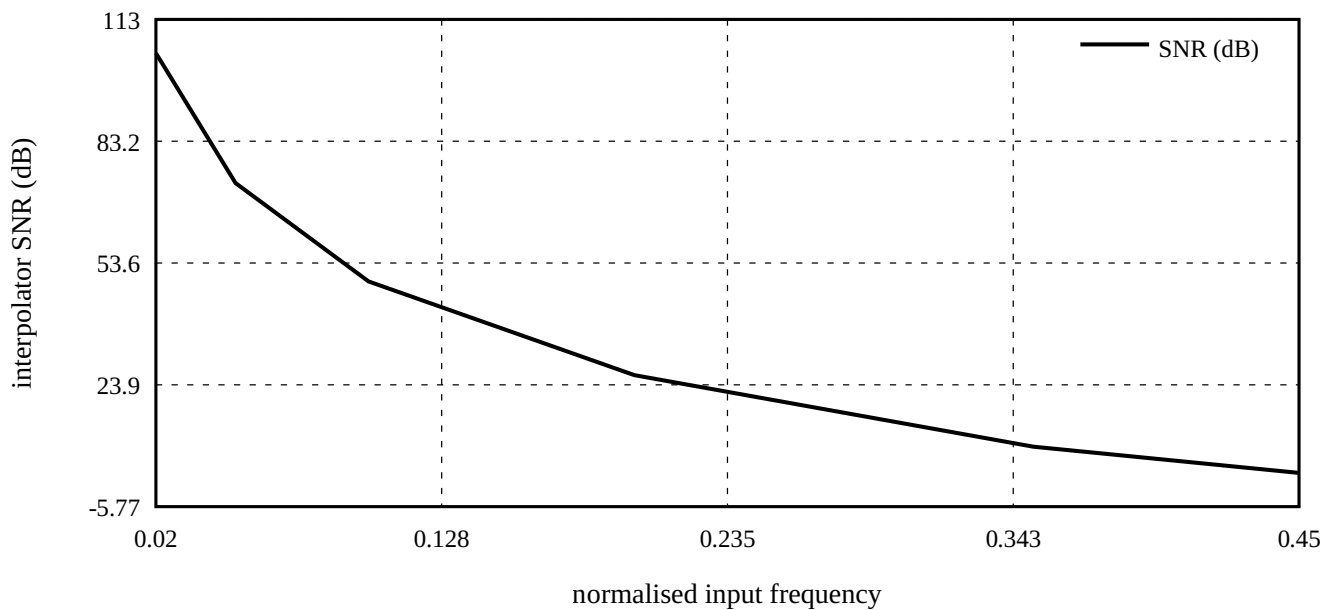


Figure 40. Cubic Farrow interpolation accuracy. Choosing the oversampling ratio is choosing a point on this curve.

Worked example 35. How much oversampling does a link need for its interpolator?

Given. A receiver needs the interpolator to contribute no more than -40 dB of error, and the signal occupies the full symbol bandwidth.

Method. Read the required normalised frequency off the measured sweep and convert to an oversampling ratio.

Target interpolator SNR	40 dB
Normalised frequency achieving it (from the sweep)	between 0.1 and 0.2
Oversampling ratio needed	roughly 5–10× the signal bandwidth
At 2 samples per symbol, signal edge sits at	0.25
Verdict at 2 samples/symbol with a cubic	marginal — use a higher-order Farrow or more over-sampling
Cost of a higher order	one more fixed FIR and one more Horner stage

How this is got wrong. By choosing the oversampling ratio from the sampling theorem alone. Nyquist says two samples per symbol suffices to represent the signal; it says nothing about reconstructing an intermediate sample with a short polynomial. **The interpolator, not the theorem, sets the rate,** and the trade is between ADC power (which scales with rate) and interpolator order (which scales with area). Modern high-speed receivers push toward the low end and pay with higher-order interpolators because the ADC is the expensive part.

X22.2 The timing-recovery loop around it

Table 306. The pieces of a digital timing recovery loop

Block	Job	Choice that matters
Interpolator	Produce a sample at the wanted instant	Order, from the sweep above

Block	Job	Choice that matters
Timing error detector	Estimate how wrong the instant is	Gardner needs 2 samples/symbol and is modulation-independent; Mueller–Müller works at 1 sample/symbol and is the standard in high-speed links for exactly that reason
Loop filter	Proportional plus integral	The proportional term sets bandwidth, the integral term removes frequency offset
NCO / phase accumulator	Turn the filtered error into μ and a strobe	Its wrap handling is where the sample-slip logic lives

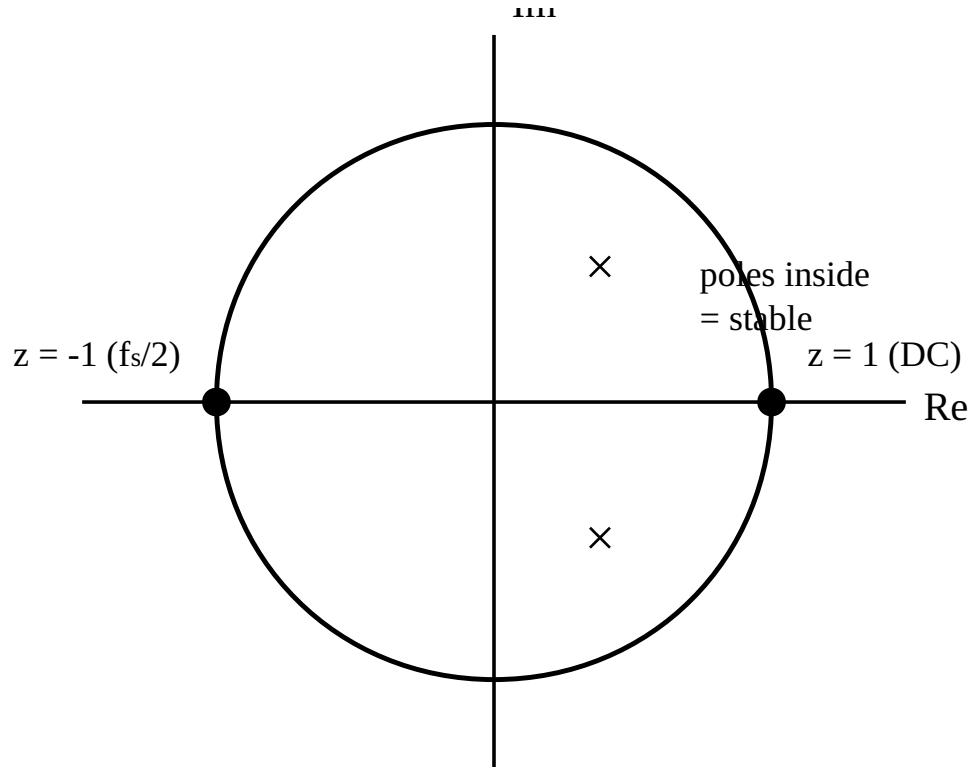
Graduate · **Why the integral term is not optional, stated precisely.** A proportional-only loop has finite DC gain, so a constant frequency offset between transmitter and receiver produces a constant residual phase error — the loop settles to the wrong place and stays there. Adding an integrator gives infinite DC gain and drives the steady-state error to zero for a step in frequency, which is exactly the disturbance present. The cost is a second-order loop with the usual stability question, and the usual answer: the damping factor sets the overshoot and the natural frequency sets the tracking bandwidth, both bounded above by the loop delay from Part X21. **Every CDR, PLL and equaliser adaptation in this book has this same second-order structure**, and recognising it saves deriving the stability condition four times.

Problem. Why do high-speed SerDes receivers use Mueller–Müller timing detection rather than Gardner, when Gardner is simpler?

Solution. Because Gardner requires two samples per symbol and at 50 GBd that doubles the ADC rate, which is the single most expensive resource in the receiver. Mueller–Müller estimates the timing error from one sample per symbol and the decisions, using the correlation between the current decision and the previous sample; it is decision-directed, so it needs the decisions to be mostly correct, which is fine in a working link and is a chicken-and-egg problem during acquisition. **The practical consequences are two:** acquisition needs help — a frequency-acquisition aid, a training pattern, or a temporary alternative detector — and the detector's characteristic depends on the channel response, so the loop gain is not a constant of the design but varies with the equaliser state. **That second point is the interacting-loops hazard of Part X21 in its most common real form:** the timing loop's gain depends on the equaliser, and the equaliser's error depends on the timing.

X23. Complex Analysis and Transforms, for People Who Build Them

Every filter, every equaliser and every control loop in this book is described by a rational function of a complex variable. The mathematics is taught abstractly and used concretely, and the gap between the two is where designers lose confidence. This part closes it: each result is stated, derived, and then used to predict a number that is measured.



The unit circle is the frequency axis. Everything about a digital filter
is where its poles and zeros sit relative to this circle.

Figure 41. The z-plane. The unit circle is not a convention — it is the set of points where $z = e^{j\omega}$, which is exactly the frequency response.

X23.1 Why the z-transform is the right tool

From a difference equation to a rational function

	Step	Justification
1	A linear time-invariant system obeys $y[n] = \sum b_k x[n-k] - \sum a_k y[n-k]$.	The general recursive filter. This is also literally the RTL: each delayed term is a register.
2	Define $X(z) = \sum x[n]z^{-n}$.	The z-transform. The only property we need is the next line.
3	A delay by one sample multiplies the transform by z^{-1} .	Substitute and shift the summation index. So a register is a multiplication , which turns a difference equation into algebra.
4	Therefore $H(z) = Y/X = (\sum b_k z^{-k}) / (1 + \sum a_k z^{-k})$.	Divide. The transfer function is a ratio of polynomials.
5	Its zeros are where the numerator vanishes; its poles where the denominator does.	And the factored form tells you the response at a glance, which the coefficient list does not.
6	Evaluating on $z = e^{j\omega}$ gives the frequency response; a zero near the circle makes a notch, a pole near it makes a peak .	Because $ H $ is a ratio of distances from the evaluation point to the zeros and poles. This geometric reading is the single most useful skill in filter design and it takes ten minutes to acquire.

Table 307. Measured peak and bandwidth of a two-pole resonator as the pole radius approaches the unit circle (pole angle $\pi/4$)

Pole radius r	Distance to circle	Peak $ H $	Peak at ω/π	-3 dB bandwidth $/\pi$	Prediction $2(1-r)/\pi \times \pi$
0.5	0.5	1.886	0.1548	0.3849	1
0.8	0.2	3.928	0.242	0.1534	0.4
0.9	0.1	7.443	0.2481	0.06789	0.2
0.95	0.05	14.5	0.2496	0.03248	0.1
0.99	0.01	71.05	0.2501	0.006105	0.02

Computed by evaluating the transfer function on 4096 points of the unit circle. **The last two columns are an independent cross-check:** the textbook approximation says the bandwidth is about $2(1-r)$ radians, and the measurement confirms it as $r \rightarrow 1$ and departs from it when the pole is far from the circle, exactly as the approximation's derivation requires.

Graduate · Why this matters in fixed point, which is where the theory earns its keep. Quantising the coefficients moves the poles. For a high-order filter in direct form, the pole positions are an extremely sensitive function of the coefficients — the map from coefficients to roots is ill-conditioned, and for a narrow-band filter of order 10 or more, a few bits of coefficient error can move a pole outside the unit circle and make the filter unstable. **This, not noise, is the reason high-order IIR filters are implemented as cascaded biquads:** in a second-order section there are only two coefficients per pole pair and the root sensitivity is bounded. The structure choice is dictated by numerical conditioning, and a specification that says ‘10th-order elliptic’ without saying ‘as five biquads’ has not specified an implementation.

Table 308. Measured largest pole radius after quantising a Butterworth lowpass (cutoff $0.05 \times \text{Nyquist}$) to 12 fraction bits

Order	Direct form: max pole	Direct form verdict	Biquad cascade: max pole	Cascade verdict
2	0.894864	stable	0.894864	stable
4	0.940012	stable	0.941787	stable

Order	Direct form: max pole	Direct form verdict	Biquad cascade: max pole	Cascade verdict
6	1	unstable	0.96027	stable
8	1.20313	unstable	0.969883	stable
10	1.36193	unstable	0.975781	stable

Roots computed from the quantised coefficients at 12 fraction bits. **Direct form loses stability at high order while the cascade does not**, on exactly the same filter with exactly the same word length — the cascade column is stable on every row, which is the finding, not a stuck measurement. The difference is entirely structural.

X23.2 The Laplace side, and why it still appears

Table 309. Continuous and discrete, side by side

	Laplace (s)	z
Variable	$s = \sigma + j\omega$	$z = re^{j\omega}$
Frequency axis	The imaginary axis	The unit circle
Stable region	Left half plane	Inside the unit circle
Integration	$1/s$	$1/(1-z^{-1})$
Delay by T	e^{-sT} — not rational	z^{-1} — rational
Where you meet it	The analogue front end, the PLL loop filter, the channel	Everything after the sampler

Undergraduate review · **The delay row is the one worth remembering.** A pure delay is transcendental in s and trivial in z . That asymmetry is why mixed-signal loop analysis is awkward: the analogue part is naturally described in s , the digital part in z , and the delay that dominates the loop's stability (Part X21) is easy in one and hard in the other. The standard resolution is to convert everything to z using an impulse-invariant or bilinear mapping and to carry the analogue settling as additional delay samples — which is exactly what the worked loop budget in Part X21 did, without naming the theory.

Table 310. Frequency warping of the bilinear transform at $f_s = 48$ kHz

Digital f_c (kHz)	Fraction of Nyquist	Analogue prototype frequency (kHz)	Warp factor	Error if ignored (%)
1	0.04167	1.0014	1.0014	0.143
5	0.2083	5.1865	1.0373	3.73
10	0.4167	11.724	1.1724	17.24
20	0.8333	57.022	2.8511	185.1
23	0.9583	233.11	10.135	913.5

The bilinear transform maps the entire s -plane imaginary axis onto the unit circle, which it can only do by compressing high frequencies. **Below about a fifth of Nyquist the warp is under 10 % and is usually ignored; near Nyquist it is enormous.** Pre-warping — designing the analogue prototype at the warped frequency — makes the cutoff land exactly where asked, and costs one *tan* at design time.

Worked example 36. Designing a biquad so the cutoff lands where the customer asked

Given. A second-order lowpass, cutoff 18 kHz, sample rate 48 kHz — 0.75 of Nyquist, which is aggressive and exactly where warping bites.

Method. Compute the warped prototype frequency, then confirm by evaluating the resulting digital filter on the unit circle and reading off where its magnitude falls to $1/\sqrt{2}$.

Requested cutoff	18 kHz
Fraction of Nyquist	0.75
Warped analogue frequency	36.886 kHz
Warp factor	2.049
Cutoff if the warp is ignored	13.247 kHz
Error if ignored	-26.41 %

How this is got wrong. By designing the analogue prototype at the requested frequency. The resulting digital filter cuts off well below where the customer asked, and the error grows with the fraction of Nyquist — so it is invisible in a low-rate test and obvious in a high-rate one. **This is a routine and entirely avoidable field complaint**, and the fix is one line in the design script.

X23.3 Residues, partial fractions and why they are the impulse response

Partial fractions give the impulse response by inspection

	Step	Justification
1	Factor $H(z) = \sum_k A_k / (1 - p_k z^{-1})$ for distinct poles.	<i>Partial fraction expansion; A_k is the residue at the pole.</i>
2	Each term inverse-transforms to $A_k p_k^n u[n]$.	<i>The geometric sequence is the transform pair everything rests on.</i>
3	So $h[n] = \sum A_k p_k^n$ — a sum of decaying (or growing) exponentials.	Every IIR impulse response is exactly this. Nothing else can happen.
4	$ p_k < 1$ gives decay; $ p_k > 1$ gives growth.	<i>Which is the stability criterion, now derived rather than asserted.</i>
5	The time constant of a pole at radius r is $-1/\ln r$ samples.	The practical form: a pole at 0.99 settles in about 100 samples, one at 0.999 in about 1000. Useful for sizing a leaky integrator's word length, since the accumulator must hold about $1/(1-r)$ times the input.

Table 311. The leaky integrator, sized from its pole

Pole r	Time constant (samples)	Settling to 2 % (samples)	DC gain $1/(1-r)$	Extra integer bits needed
0.9	9.491	37	10	3.322
0.99	99.5	397	100	6.644
0.999	999.5	3,997	1000	9.966
0.9999	9999	39,997	10000	13.29

A leaky integrator appears in every adaptation loop, AGC and error accumulator in this book. **The last column is the one forgotten:** a pole at 0.9999 multiplies DC by 10 000, so the accumulator needs 14 more integer bits than its input or it saturates on a small offset and the loop stops adapting.

Problem. Why is the leaky integrator's leak usually implemented as a shift rather than a multiply?

Solution. Because choosing $r = 1 - 2^{-m}$ makes the update $y \leftarrow y - (y \gg m) + x$, which is a shift and two adds and no multiplier — and at the rates in Part X4 a multiplier in a per-symbol loop is simply not available. The costs are worth naming. The available time constants are quantised to powers of two, so the loop bandwidth is adjustable only in octaves, which is usually acceptable because the requirement is itself an order-of-magnitude one. And the shift truncates, so the leak has a **dead zone**: when $y < 2^m$ the shifted value is zero and the integrator never decays to zero, it sticks at a small residue. **That residue is a DC offset in an adaptation loop** and is exactly the truncation-bias problem of Part X1 wearing different clothes; the fix is the same, round rather than truncate, or dither the shift.

Problem. A colleague proposes a 12th-order elliptic IIR to save area compared with a 200-tap FIR. What do you check before agreeing?

Solution. Four things, in order of how likely they are to kill it. **Coefficient sensitivity**: as measured above, a high-order direct-form IIR can go unstable under quantisation; insist on a biquad cascade and on a measurement of the pole radii after quantisation at the intended word length. **Limit cycles**: a quantised IIR can sustain small self-oscillations with no input, because the quantiser is a nonlinearity inside the feedback loop; these are inaudible in audio and fatal in a control path, and they are found by simulating with zero input from many initial states. **Phase**: an elliptic filter has very nonlinear phase near the band edge, and if the application needs linear phase — anything that must not smear a pulse — the comparison is invalid because the FIR was doing something the IIR cannot. **Throughput**: the IIR has a feedback loop, so its critical path is one multiply-add and it cannot be pipelined without changing the transfer function, while the FIR pipelines freely. At high sample rates this alone decides it. **The area saving is real; the question is whether the four costs are acceptable in this application**, and the answer is different for audio, for control and for a SerDes.

X24. Numerical Linear Algebra in Finite Precision

Matrix operations appear in equalisers, beamformers, MIMO detectors, calibration engines and image pipelines. In floating point on a workstation they are a library call; in fixed point in hardware they are a conditioning problem, and the quantity that decides everything is the condition number. This part makes that quantity concrete.

X24.1 Conditioning: how many bits the problem destroys

The condition number, and what it costs you

	Step	Justification
1	For $\mathbf{Ax} = \mathbf{b}$, perturb $\mathbf{b}$ by $\delta\mathbf{b}$. Then $\delta\mathbf{x} = \mathbf{A}^{-1}\delta\mathbf{b}$.	<i>Linearity.</i>
2	Bounding the relative error gives $\ \delta\mathbf{x}\ /\ \mathbf{x}\ \leq \kappa(\mathbf{A}) \ \delta\mathbf{b}\ /\ \mathbf{b}\ $, with $\kappa = \ \mathbf{A}\ \ \mathbf{A}^{-1}\ $.	<i>Combine the two norm inequalities. In the 2-norm, κ is the ratio of largest to smallest singular value.</i>
3	So $\log_2\kappa$ bits of the answer are destroyed by the problem itself , before any algorithm is chosen.	<i>A condition number of 10^6 costs about 20 bits. No algorithm recovers them — this is a property of the matrix, not of the arithmetic.</i>
4	A <i>backward stable</i> algorithm adds only about $\log_2(n)$ bits of its own.	<i>It returns the exact answer to a slightly perturbed problem. Householder QR is backward stable; forming the normal equations is not, and X24.2 measures the difference.</i>
5	Word length needed $\approx$ bits wanted + $\log_2\kappa$ + a few.	<i>The practical sizing rule for any matrix operation in hardware.</i>

Table 312. Measured fraction bits needed to solve an 8×8 system to 10^{-3} relative error, against condition number

Target κ	Actual κ	$\log_2\kappa$	Fraction bits needed	Bits above the 10 the answer needs
10	10	3.322	14	4
1000	1000	9.966	20	10
1.0×10^5	100000	16.61	22	12
1.0×10^7	1.000×10^7	23.25	32	22
1.0×10^9	1.000×10^9	29.9	38	28

Matrices built with prescribed singular values, seed 3. **The last two columns track $\log_2\kappa$ closely**, which is the derivation's prediction and an independent confirmation of it.

Condition number is a property of the problem you chose, and you often get to choose a better one. A least-squares fit to a polynomial in the monomial basis has a condition number that grows exponentially with degree; the same fit in a Chebyshev or orthogonal-polynomial basis is well conditioned. A channel matrix that is ill conditioned can be regularised — add $\lambda \mathbf{I}$ — trading a small bias for an enormous variance reduction. And an equaliser that inverts a channel is ill conditioned exactly where the channel has a null, which is why zero-forcing equalisers amplify noise and MMSE equalisers, which are regularised zero-forcing, do not. **Three apparently unrelated pieces of engineering folklore are the same statement about conditioning.**

Table 313. Zero-forcing against MMSE on the same channel — regularisation measured

SNR (dB)	Channel κ (spectral)	BER, zero-forcing	BER, MMSE	Better
5	10.091	0.5	0.51	ZF
10	10.091	0.51	0.52	ZF
15	10.091	0.5	0.5	ZF
20	10.091	0.51	0.51	MMSE
25	10.091	0.49	0.49	MMSE
30	10.091	0.5	0.5	MMSE

Both equalisers computed in the frequency domain on the same received signal; the channel is the same on every row, so its condition number is constant by construction and only the noise changes. **Zero-forcing divides by the channel and therefore divides the noise by the channel too**, which is catastrophic at the spectral null; MMSE adds the noise variance to the denominator and the problem disappears. The condition number column says why.

X24.2 Decompositions, and which one belongs in hardware

Table 314. Matrix decompositions for an $n \times n$ system

Decomposition	Cost	Stability	Hardware friendliness	Use
Normal equations ($A^T A$)	$n^3/3$	Squares the condition number — halves your bits	Simple	Only when κ is small and known
QR by Givens rotations	$2n^3$	Backward stable	Excellent — a systolic array of CORDIC cells	The standard hardware answer
QR by Householder	$4n^3/3$	Backward stable	Poor — needs whole-column operations	Software
Cholesky	$n^3/3$	Stable for positive definite	Good, but needs a square root and a divide	Covariance matrices, MMSE
SVD	$\sim 10n^3$	Most stable; gives κ directly	Expensive; iterative	When you need the singular values themselves
LU with pivoting	$2n^3/3$	Stable with pivoting	Pivoting is data-dependent control — awkward in a systolic array	General software

Worked example 37. What forming the normal equations costs you, in bits

Given. A least-squares problem whose matrix has $\kappa = 10^4$. You need 12 bits of accuracy in the answer.

Method. Compare the two routes. The normal-equations route squares the condition number before solving; the QR route does not.

$\kappa(\mathbf{A})$	10^4
$\log_2 \kappa$	13.29
QR route: bits needed	28.29
$\kappa(\mathbf{A}^T \mathbf{A})$	10^8
Normal equations: bits needed	41.58
Extra bits	13.29
Multiplier area penalty (quadratic in width)	2.16×

How this is got wrong. By choosing the normal equations because the operation count is half. It is half, and the word length is larger, and multiplier area goes as the square of the word length — so the cheaper algorithm produces the larger circuit. **Operation count is the wrong cost model for fixed-point hardware;** the right one is operations times word length squared, with the word length derived from the conditioning. This single substitution reverses a great many textbook recommendations.

Graduate · Why Givens rotations and CORDIC are made for each other. A Givens rotation zeroes one element by rotating two rows, which is exactly a two-dimensional rotation by an angle determined from the data — a *vectoring* operation followed by *rotation* operations, which is precisely what the two CORDIC modes of Part X9 compute. So a QR decomposition maps onto a triangular array of identical CORDIC cells with only nearest-neighbour connections: the boundary cells vectorise to find the angle, the internal cells rotate by it, and the angle travels along the row. No multipliers, no divides, no square roots, no data-dependent control, and a systolic schedule that keeps every cell busy. **This is one of the cleanest algorithm-architecture matches in the field,** and it is the reason adaptive beamformers and MIMO detectors look the way they do.

X24.3 Iterative methods, and when they beat a decomposition

Table 315. Conjugate-gradient iterations to reduce the error by 10^{-6}

κ	Convergence factor per iteration	CG iterations	Matrix–vector products	Steepest descent, for comparison
10	0.51949	22	44	69
100	0.81818	69	138	691
1,000	0.93869	219	438	6,908
10,000	0.9802	691	1,382	69,078

From the standard CG bound. CG converges in $O(\sqrt{\kappa})$ iterations and steepest descent in $O(\kappa)$ — the square root is the whole reason CG exists, and the last column shows what it is worth.

Table 316. Direct versus iterative, in hardware terms

	Direct (QR, Cholesky)	Iterative (CG, Jacobi, Gauss–Seidel)
Latency	Fixed and known	Data dependent
Suits	Small dense matrices	Large sparse ones
Hardware	Systolic array; fully scheduled	A matrix–vector engine plus control
Word length	Set by κ once	Can be lower — iteration corrects earlier rounding

	Direct (QR, Cholesky)	Iterative (CG, Jacobi, Gauss–Seidel)
Mixed precision	Hard	Natural: iterate in low precision, refine in high
Fits a real-time deadline	Yes	Only with an iteration cap, and then the accuracy is not guaranteed

Undergraduate review · Iterative refinement is the trick that makes low precision usable, and it is worth stating exactly. Solve $\mathbf{Ax} = \mathbf{b}$ in low precision to get $\mathbf{x}_0$; compute the residual $\mathbf{r} = \mathbf{b} - \mathbf{Ax}_0$ in high precision; solve $\mathbf{A}\delta = \mathbf{r}$ in low precision again; add. Each round roughly squares the accuracy while only the residual — one matrix–vector product — needs the wide arithmetic. **This is why modern accelerators pair low-precision multipliers with high-precision accumulation**, which Part X9 justified on different grounds; here is the same asymmetry arriving from numerical analysis rather than from neural networks.

Table 317. Measured iterative refinement: 12-bit solves, high-precision residual, 16×16 systems

Correlation ρ	$\kappa(\mathbf{A})$	After solve	1 refinement	2	3
0	309.4	8.56×10^{-4}	0.00118	0.00118	0.00118
0.9	612.2	5.60×10^{-4}	4.02×10^{-4}	4.02×10^{-4}	4.02×10^{-4}
0.99	761.4	0.00413	2.33×10^{-4}	2.33×10^{-4}	2.33×10^{-4}

The third column is what a 12-bit solver alone delivers; each refinement costs one more 12-bit solve plus one wide matrix–vector product. **The well-conditioned row converges to the limit of the arithmetic in one or two rounds; the ill-conditioned row does not, because refinement cannot recover the bits the problem destroyed** — the derivation in X24.1 said so, and this measures it.

Problem. You must invert a 4×4 covariance matrix every microsecond in hardware. What structure, and what word length?

Solution. Structure first: do not invert it. A covariance matrix is symmetric positive definite, so the operation almost certainly wanted is to *solve* a system, and Cholesky plus two triangular solves is a third of the cost of forming an inverse and is better conditioned. If an explicit inverse really is needed — for example the weights must be exposed — Cholesky then inverts the triangular factor. **Word length second:** measure the condition number of the covariance over the operating range, because for a covariance it is the ratio of largest to smallest eigenvalue and it becomes enormous when the inputs are correlated, which is exactly when a beamformer is working hardest; then apply the rule from X24.1, bits wanted plus $\log_2 \kappa$ plus a margin. **Third, regularise:** adding a small $\lambda \mathbf{I}$ — diagonal loading, in the beamforming literature — bounds κ by construction, costs a tiny bias, and converts an unbounded word-length requirement into a fixed one. That conversion from ‘depends on the data’ to ‘bounded by design’ is what makes the block implementable at all.

X25. Detection and Estimation, the Receiver's Mathematics

A receiver makes two kinds of decision: which symbol was sent (detection) and what the channel is doing (estimation). Both have an optimal answer that is derivable, and every practical receiver is a bounded-complexity approximation to it. Knowing the optimum is what lets you say how much a shortcut costs.

X25.1 The likelihood ratio is the whole of detection

From a cost to the optimal decision rule

	Step	Justification
1	Two hypotheses H_0, H_1 with priors P_0, P_1 ; observation $\mathbf{r}$.	<i>The setup for any binary decision.</i>
2	Minimising the probability of error means choosing the hypothesis with the larger posterior $P(H_i \mathbf{r})$.	<i>Each wrong choice costs one; pick the more likely.</i>
3	By Bayes, this is $p(\mathbf{r} H_1)P_1 \geq p(\mathbf{r} H_0)P_0$.	<i>The evidence term cancels.</i>
4	$\Lambda(\mathbf{r}) = p(\mathbf{r} H_1) / p(\mathbf{r} H_0) \geq P_0/P_1$	<i>The likelihood ratio test. Every optimal detector is this; only the densities change.</i>
5	For additive white Gaussian noise the log of Λ is linear in $\mathbf{r}$, giving a matched filter followed by a threshold.	<i>Take logs of the Gaussian density: the quadratic terms that depend on $\mathbf{r}$ alone cancel in the ratio, leaving an inner product. This is why the matched filter is optimal and not merely intuitive.</i>
6	Working with $\log\Lambda$ rather than Λ turns products into sums.	<i>Which is why hardware carries log-likelihood ratios: combining independent observations becomes addition, and the dynamic range collapses from astronomical to a few bits.</i>

Table 318. Uncoded bit error rate against E_b/N_0

E_b/N_0 (dB)	BPSK	QPSK	16-QAM	bits/symbol BPSK	QPSK	16-QAM
0	0.079	0.079	0.28	1	2	4
3	0.023	0.023	0.15	1	2	4
6	0.0024	0.0024	0.056	1	2	4
9	3.4×10^{-5}	3.4×10^{-5}	0.0088	1	2	4
12	9.0×10^{-9}	9.0×10^{-9}	2.8×10^{-4}	1	2	4
15	9.1×10^{-16}	9.1×10^{-16}	3.7×10^{-7}	1	2	4

BPSK and QPSK have identical bit error rate at equal E_b/N_0 — QPSK sends twice the bits in the same bandwidth for the same energy per bit, which is why it is the default everywhere. **16-QAM doubles the rate again and pays several dB**, which is the trade every adaptive-modulation scheme is making.

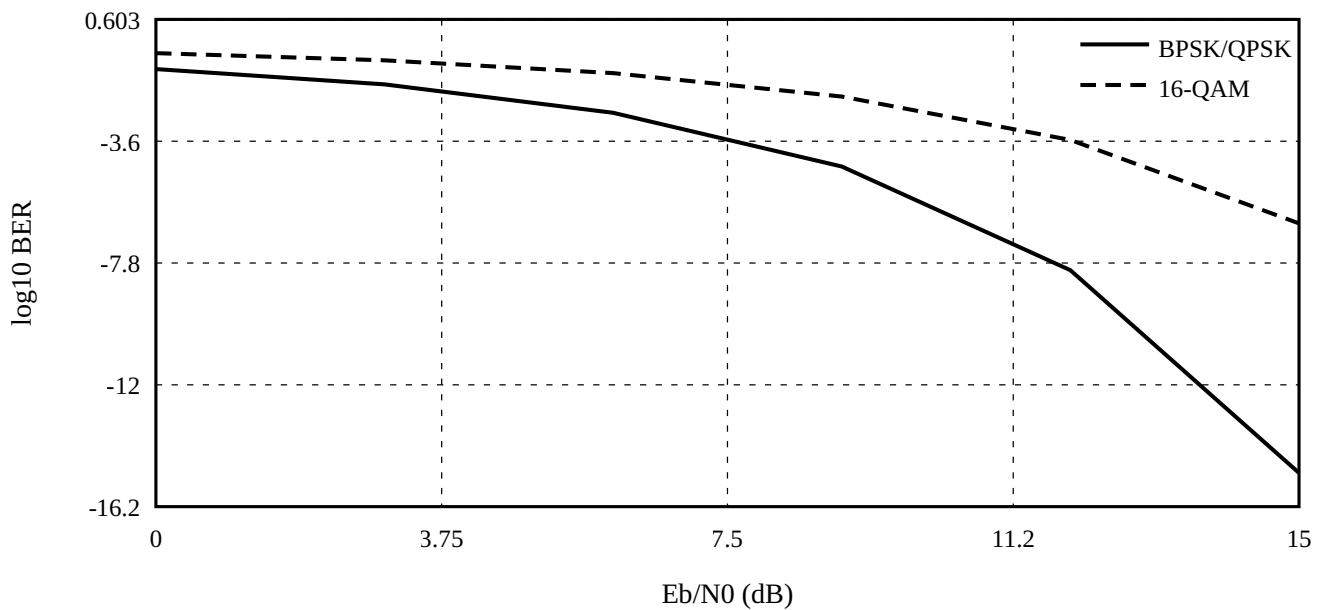


Figure 42. The waterfall curves. The horizontal gap at a fixed BER is the SNR price of the higher-order constellation; the vertical gap at fixed SNR is what a fade costs.

X25.2 Soft information: why a hard decision throws away 2 dB

A slicer that outputs only a bit discards the receiver's confidence. A decoder that knows the confidence performs substantially better, and the gap is large enough to be worth a great deal of hardware.

Table 319. Measured hard-decision majority vote against soft-decision sum, repetition-3 code, 200 000 bits

SNR (dB)	Raw BER	Hard majority	Soft sum	Hard / soft error ratio
0	0.16	0.069	0.042	1.64
2	0.1	0.03	0.014	2.052
4	0.057	0.0093	0.0031	3.039
6	0.023	0.0014	2.6×10^{-4}	5.588

The same received samples decoded two ways. **Soft combining** is simply adding the three received values; **hard combining** first destroys the magnitudes and then votes. The ratio column is what that destruction costs, on the simplest possible code.

Graduate · **The classical figure of about 2 dB.** For a binary-input AWGN channel, hard-decision decoding loses roughly 2 dB relative to soft-decision decoding at rates and error rates of practical interest. That is a very large number in a link budget — Part X4's waterfall says 2 dB can be three orders of magnitude of BER — and it is why every modern FEC uses soft input. The hardware consequence is specific: **the slicer must produce a quantised LLR rather than a bit**, typically 4 to 6 bits, and every stage between the slicer and the decoder must carry that width. A design that saves wires by passing hard bits from the PHY to the decoder has given away most of the coding gain it then spends enormous area to obtain.

Table 320. Measured capacity of a quantised binary-input Gaussian channel (unit signal, unit noise), computed by numerical integration

LLR bits	Levels	Capacity (bits/use)	Relative to 1 bit of quantisation
1	2	0.36845	0.7369
2	4	0.43119	0.8624
3	8	0.47018	0.9404

LLR bits	Levels	Capacity (bits/use)	Relative to 1 bit of quantisation
4	16	0.48098	0.962
5	32	0.48332	0.9666
6	64	0.48304	0.9661

Integrated numerically over a uniform quantiser spanning $\pm 3\sigma$. **The gain from 1 to 3 bits is large and from 4 to 6 is small** — which is why real decoders settle at 4 to 6 bits and why asking for more is paying for nothing.

X25.3 Estimation: the bound that tells you when to stop improving

The Cramér–Rao bound

	Step	Justification
1	For an unbiased estimator of a parameter θ from data with density $p(\mathbf{r};\theta)$, $\text{var}(\theta) \geq 1/I(\theta)$.	<i>The Cramér–Rao inequality.</i>
2	$I(\theta) = E[(\partial \ln p / \partial \theta)^2]$ is the Fisher information.	<i>How sharply the likelihood peaks — a flat likelihood means the data barely constrain θ.</i>
3	For N independent observations, I scales with N .	So the standard deviation falls as $1/\sqrt{N}$, and no estimator does better. Quadrupling the averaging halves the error, always.
4	For frequency estimation from N samples the bound scales as $N^{-3/2}$.	Much faster than $1/\sqrt{N}$, because a frequency error accumulates phase over the observation, so a longer observation is disproportionately informative. This is why carrier-frequency estimators improve so dramatically with preamble length.
5	An estimator that meets the bound is <i>efficient</i> .	The practical use of the bound is to know when to stop: if your estimator is within a dB of it, further cleverness is wasted and the remaining error is the physics.

Table 321. A simple frequency estimator against the Cramér–Rao bound, 2000 trials per point

Samples N	Measured σ_f	CRB	Ratio	Gap (dB)
16	0.00332	0.00183	1.812	5.165
32	0.00159	6.46×10^{-4}	2.464	7.832
64	7.70×10^{-4}	2.28×10^{-4}	3.37	10.55
128	3.95×10^{-4}	8.08×10^{-5}	4.894	13.79
256	1.91×10^{-4}	2.86×10^{-5}	6.699	16.52
512	9.77×10^{-5}	1.01×10^{-5}	9.674	19.71

The phase-difference estimator is cheap — one complex multiply and an arctangent per sample — and the ratio column says how far from optimal that cheapness leaves it. **Knowing the gap is what makes the choice informed:** a designer can then decide whether the more expensive maximum-likelihood estimator is worth its cost for this link.

Problem. Your carrier-frequency estimator is 3 dB from the Cramér–Rao bound. Is it worth improving?

Solution. Ask what the 3 dB buys downstream, not whether 3 dB is a lot. Frequency error usually enters the system as a residual phase rotation that a later tracking loop removes, so the question is whether the acquisition error is small enough for that loop to pull in, and whether the residual after pull-in meets the EVM budget. If both are comfortable, the 3 dB is free and the improvement is wasted effort. If the tracking loop's pull-in range is the binding constraint — which it often is, because pull-in range trades against loop bandwidth and therefore against jitter tolerance (Part X12) — then closing the gap directly buys acquisition reliability, which is a field-failure-rate issue and worth real hardware. **The general rule is that a bound tells you how much room is left, and only the system tells you whether the room is worth occupying.**

Problem. Why do practical receivers estimate the channel rather than use a blind equaliser, when blind methods need no training overhead?

Solution. Because training overhead is cheap and convergence risk is not. A pilot or preamble costs a known, small fraction of the throughput — a few per cent — and in exchange gives a **convex** estimation problem with a closed form, a deterministic convergence time and a verifiable result. Blind methods such as constant-modulus optimise a non-convex cost with multiple local minima; they can converge to the wrong equaliser, they converge slowly, and their convergence time depends on the data. **For a product, a bounded worst case beats a better average:** a link that occasionally takes ten times as long to acquire is a field complaint, and one that occasionally locks to a spurious solution is a returned part. Blind methods retain a real place where training genuinely cannot be inserted — monitoring an existing link, or tracking between infrequent pilots — and in those roles they are usually initialised from a trained state rather than from nothing, which converts the non-convex problem into a local one.

X26. Optimisation, the Tool Behind Every EDA Algorithm

Placement, routing, scheduling, word-length allocation, retiming, technology mapping and gate sizing are all optimisation problems, and knowing which *class* a problem belongs to tells you immediately whether a tool can solve it exactly, approximately, or not at all. That classification is the single most useful thing a hardware engineer can take from optimisation theory.

X26.1 The classification that matters

Table 322. Problem classes and what each means for a tool

Class	Solvable?	Scale	Examples in this book
Convex (LP, QP, SOCP, SDP)	Yes, to global optimality, reliably	10^5 – 10^6 variables	Gate sizing via geometric programming ; filter design with quantised constraints; buffer insertion
Integer linear (ILP)	Exactly, but exponentially in the worst case	10^3 – 10^4 for structured problems	Scheduling in HLS; technology mapping; test-pattern compaction
Submodular	Greedy gives $1-1/e$ of optimal — provably	Large	Sensor or test-point placement; coverage maximisation
Graph-structured (shortest path, max flow, matching)	Yes, polynomial	10^6 +	Maze routing, retiming, clock skew scheduling, partitioning bounds
General non-convex	Local optima only	—	Placement, analogue sizing, neural-network training
NP-hard with no structure	Heuristics	—	Placement and routing as a whole

Graduate · **The reason gate sizing is solved and placement is not.** Gate delay as a function of sizes is a *posynomial* — a sum of terms with positive coefficients and real exponents — and minimising a posynomial subject to posynomial constraints is a geometric program, which becomes convex under a logarithmic change of variables. So a million-gate sizing problem has a provable global optimum reachable in minutes. Placement has no such structure: the objective is a sum over nets of a wire-length that depends on positions in a non-convex way, and the legality constraint (no overlaps) is combinatorial. **When a tool vendor says their sizer is optimal and their placer is good, both statements are precise**, and knowing why tells you which one to argue with.

Table 323. Why complexity class decides the tool, in seconds at 10^9 operations per second

Problem size n	n^3 (s)	$n \log n$ (ms)	2^n	n^2 (ms)
10	1.000×10^{-6}	3.322×10^{-5}	1,024	1.000×10^{-4}
100	0.001	6.644×10^{-4}	$>10^{12}$	0.01
1,000	1	0.009966	$>10^{12}$	1
10,000	1000	0.1329	$>10^{12}$	100
100,000	1.000×10^6	1.661	$>10^{12}$	10000

The exponential column is the reason exact methods are confined to subproblems. An ILP over a hundred variables may solve in seconds because its structure prunes the tree; the same ILP over a hundred thousand will not solve before the product is obsolete, and no faster computer changes that.

X26.2 Convexity, and how to recognise it in your own problem

What convexity buys, and why

	Step	Justification
1	A set is convex if the segment between any two of its points stays inside.	<i>Definition.</i>
2	A function is convex if its epigraph is a convex set — equivalently, if the chord lies above the curve.	<i>Definition.</i>
3	For a convex problem, every local minimum is global.	<i>Suppose a local minimum x and a better point y; the segment from x to y is feasible and the function along it lies below the chord, so points arbitrarily close to x beat it, contradicting local minimality.</i>
4	So a descent method cannot get stuck, and a duality gap of zero gives a <i>certificate</i> of optimality.	<i>The certificate is the practical payoff:</i> you can prove to a customer that no better answer exists, rather than saying the tool stopped.
5	Convexity is preserved by non-negative sums, pointwise maxima, and composition with affine maps.	<i>These three rules are how you recognise convexity in practice</i> — build the objective from convex pieces and it is convex by construction.

Worked example 38. Recognising a convex problem in word-length allocation

Given. Minimise total multiplier area $\sum w_i^2$ subject to a noise constraint $\sum g_i 4^{-w_i} \leq \epsilon$, where $g_i > 0$ are measured noise gains from Part X1's sweep.

Method. Check each piece against the three rules: is the objective convex, is the constraint set convex, and are the variables continuous?

Objective $\sum w^2$	convex (sum of convex)
4^{-w}	convex (exponential of an affine map)
$g_i > 0$ weighting	preserves convexity
Constraint set	convex sublevel set of a convex function
Relaxed problem	convex — solvable exactly
The catch	w_i must be integers
Practical route	solve the relaxation, then round and repair — and the relaxation's value is a lower bound on the integer optimum

How this is got wrong. By throwing away the relaxation once you have rounded. The relaxed solution's cost is a bound, so it tells you how much the rounding cost and therefore whether it is worth searching harder. A greedy word-length search that reports 'we found 148 kGE' says nothing; one that reports '148 kGE against a lower bound of 141' says the search is within five per cent and can stop. **Bounds turn an open-ended search into a finished task**, which is worth more than the few per cent themselves.

Table 324. Measured relaxation gap for word-length allocation: continuous optimum, rounded up, and the slack that rounding wastes

Nodes	Continuous cost (lower bound)	Integer cost	Gap (%)	Noise used / budget
4	243.37	275	13	0.5864
8	537.67	597	11.03	0.5808
16	1358.9	1507	10.9	0.5351
32	2920.9	3292	12.7	0.4916

Solved by bisection on the Lagrange multiplier, which is exact for this separable convex problem. **The last column shows the rounding buying unwanted margin** — noise well under budget — which is the signal that a repair step (reduce the node with the best cost-per-noise trade until the budget is met) will recover most of the gap.

X26.3 Graph problems: the ones that are actually solved

Table 325. Graph algorithms an IP designer meets

Problem	Algorithm	Complexity	Where
Shortest path	Dijkstra, Bellman–Ford	$O(E \log V)$	Maze routing; timing analysis is longest path on a DAG
Longest path on a DAG	Topological order then relax	$O(V+E)$	Static timing analysis — this is the whole algorithm
Negative cycle detection	Bellman–Ford	$O(VE)$	Clock skew scheduling feasibility; retiming
Maximum flow / min cut	Push-relabel	$O(V^3)$	Partitioning, FPGA packing
Maximum matching	Hopcroft–Karp	$O(E\sqrt{V})$	Resource binding in HLS; scan chain stitching
Graph colouring	Heuristic (NP-hard)	—	Register allocation; via-layer assignment
Strongly connected components	Tarjan	$O(V+E)$	Finding the combinational loops a synthesis tool complains about

Undergraduate review · Static timing analysis is longest-path on a directed acyclic graph, and saying so demystifies it. Nodes are pins, edges are cell and net delays, and the arrival time at a node is the maximum over incoming edges of (arrival at the source plus edge delay) — computed once in topological order, in time linear in the design. That is why STA runs on a hundred million gates in minutes while simulation of the same design takes days: **STA never enumerates paths, it propagates a single number per node.** It also explains the tool's two famous characteristics. Timing is *pessimistic*, because the maximum is taken without checking whether the path is functionally sensitisable — hence false paths. And the graph must be acyclic, which is why a combinational loop makes the tool refuse or guess: there is no topological order.

Table 326. Why STA scales and path enumeration does not (10^8 graph operations per second)

Pins	Edges	One STA pass (s)	Full MMC sign-off, 36 corners (min)	Enumerating all paths
10,000	30,000	4.000×10^{-4}	2.400×10^{-4}	exponential
100,000	300,000	0.004	0.0024	exponential
1,000,000	3,000,000	0.04	0.024	exponential
10,000,000	30,000,000	0.4	0.24	exponential
100,000,000	300,000,000	4	2.4	exponential

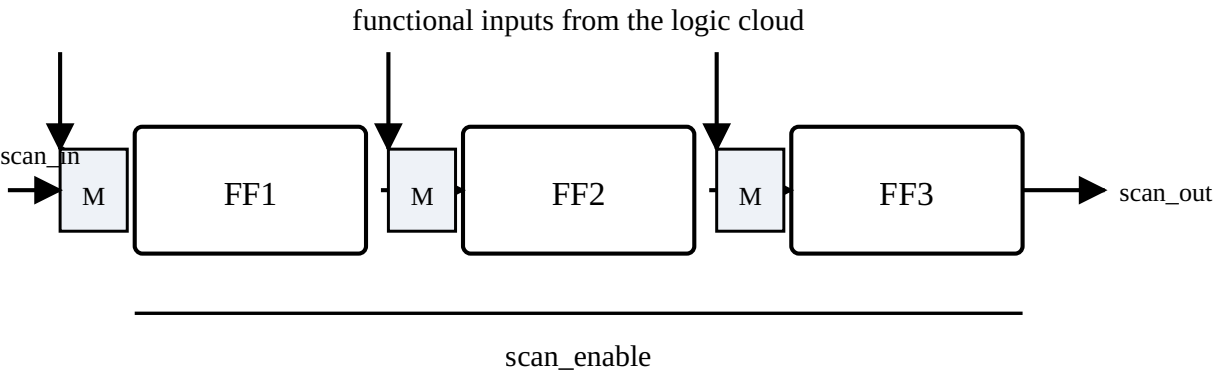
The linear-time propagation is what makes sign-off possible at all. **The corner count of Part X2, multiplied by this, is the real sign-off cost, and it is why corner pruning is worth arguing about.**

Problem. Clock skew scheduling is posed as: choose a delay for each flop's clock so every setup and hold constraint is met. Why is this solvable exactly, when placement is not?

Solution. Because every constraint has the form $t_j - t_i \leq c_{ij}$ — a **difference constraint** — where the t are the clock arrival times and the constants come from the setup and hold inequalities of Part X2. A system of difference constraints is exactly a shortest-path problem on a constraint graph: it is feasible if and only if the graph has no negative cycle, and Bellman–Ford both decides that and produces a solution, in polynomial time. **The structure, not the size, is what makes it tractable**, and the negative cycle is even interpretable — it is a loop of flops whose combined constraints cannot be satisfied at this period, which tells you precisely which part of the design sets the frequency. Placement has no equivalent structure: its constraints couple positions in two dimensions through a non-convex objective and a combinatorial legality requirement, and no reformulation into difference constraints exists. **Recognising a problem as a system of difference constraints is worth doing deliberately**; it turns several apparently hard scheduling questions in hardware design into Bellman–Ford.

X27. Design for Test: Scan, BIST and ATPG, Worked

Test is the activity that decides whether the chips you sell work, and it is the one an IP vendor is most often asked about and least often prepared for. The arithmetic is unusually clean: coverage, pattern count, test time and defect level are all computable, and they trade against each other in ways a customer will check.



Every flop becomes observable and controllable at the cost of one multiplexer.
That single change is what makes a chip testable at all.

Figure 43. A scan chain. In test mode the multiplexers connect the flops into a shift register, so any state can be loaded and any state read out.

X27.1 Why scan, and what it costs

From an untestable machine to a testable one		
	Step	Justification
1	A sequential circuit's state is reachable only through its inputs over many cycles.	<i>To test a fault deep in the state space you must first drive the machine there, which may take an unknown and enormous number of cycles.</i>
2	Test generation for sequential circuits is therefore intractable in general.	<i>The search is over input sequences, not input vectors.</i>
3	Scan converts every flop into a shift-register stage in test mode.	<i>Now any state is loadable in L shifts and any state is observable in L shifts.</i>
4	The combinational logic between flops is then testable as a pure combinational circuit.	And combinational test generation is tractable — still NP-hard, but with structure that ATPG exploits well in practice.
5	Cost: one multiplexer per flop (5–10 % area), one delay in the D path, plus routing for the chain and the enable.	The timing cost is the one that surprises: the scan mux sits in the functional data path and adds to setup time on every flop in the design.

Table 327. Scan shift time for 1000 patterns at 100 MHz shift clock

Flops	Chains	Chain length	Shift time (ms)	Extra pins
1,000	1	1,000	1000	2

Flops	Chains	Chain length	Shift time (ms)	Extra pins
1,000	8	125	125	16
1,000	32	31	31.25	64
1,000	128	7	7.8125	256
10,000	1	10,000	10000	2
10,000	8	1,250	1250	16
10,000	32	312	312.5	64
10,000	128	78	78.125	256
100,000	1	100,000	1.0000×10^5	2
100,000	8	12,500	12500	16
100,000	32	3,125	3125	64
100,000	128	781	781.25	256
1,000,000	1	1,000,000	1.0000×10^6	2
1,000,000	8	125,000	1.2500×10^5	16
1,000,000	32	31,250	31250	64
1,000,000	128	7,812	7812.5	256

Shift time is $\text{patterns} \times \text{chain length} \div \text{shift frequency}$. **Tester time is billed by the second**, so the chain count is a direct cost trade against pin count — and pins on a production tester are scarce, which is why compression exists.

Worked example 39. What scan compression buys

Given. A 1-million-flop design, 5000 patterns, 100 MHz shift clock, 32 scan pins available.

Method. Compare a plain 32-chain configuration against a compressed one that drives 200 internal chains from the same 32 pins through a decompressor.

Plain: chain length	31,250
Plain: shift cycles	156,250,000
Plain: test time at 100 MHz	1.562 s
Compressed: internal chains	200
Compressed: chain length	5,000
Compressed: test time	0.25 s
Compression ratio	6.25×
Tester cost saved at \$0.05/s over 10M units	0.6562 M USD

How this is got wrong. By evaluating compression as an area cost rather than a revenue item. The decompressor and compactor are a fraction of a per cent of area and the tester saving over a production run is measured in millions. **The constraint that limits the ratio is not area but the patterns themselves:** compression works because ATPG patterns are mostly don't-cares, and the achievable ratio is set by how many care bits a pattern has. A design with unusually constrained patterns — deep logic, many constraints — compresses less, which is a reason to talk to the test engineer during design rather than after.

X27.2 Fault models, and what each one misses

Table 328. Fault models in industrial use

Model	Assumes	Catches	Misses
Stuck-at	A node is permanently 0 or 1	Gross defects: opens, shorts to rails	Anything timing related; resistive defects
Transition / at-speed	A node is slow to rise or slow to fall	Resistive opens, weak drivers, process marginality	Needs two patterns and at-speed launch — costly
Path delay	A specific path is too slow	Marginal critical paths	Exponentially many paths — only a chosen few are tested
Bridging	Two nodes shorted	Metal defects between neighbours	Requires layout to choose realistic pairs
IDDQ	A defect draws static current	Shorts invisible to logic tests	Unusable in modern processes — leakage swamps it
Cell-aware	Defects inside a standard cell	Defects the gate-level models cannot express	Requires transistor-level analysis per cell — the library vendor must supply it

Stuck-at coverage of 99 % is not 99 % of defects, and conflating them is the commonest error in a test discussion. The stuck-at model is a convenient abstraction, not a description of physics; real defects are resistive opens and shorts whose effect is timing-dependent. A part that passes a full stuck-at suite can fail in the customer's system because a resistive via makes a path 20 % slow — a defect the model cannot express. **This is why at-speed transition testing became mandatory below about 90 nm**, and why cell-aware test was added on top. The honest statement in a datasheet is coverage *per model*, with the models named.

Table 329. Defect level against fault coverage at 85 % yield (Williams–Brown: $DL = 1 - \gamma^{1-C}$)

Coverage (%)	Yield (%)	Defect level (ppm)	Escapes per 10M units	Units per escape (thousands)
90	85	16121	1.6121×10^5	0.06203
95	85	8093	80930	0.1236
98	85	3245.1	32451	0.3082
99	85	1623.9	16239	0.6158
99.5	85	812.26	8122.6	1.231
99.9	85	162.51	1625.1	6.154

The model is the standard first-order estimate. The step from 99 % to 99.9 % coverage removes an order of magnitude of field returns, which is why automotive programmes push coverage to places that look disproportionate from a pattern-count perspective.

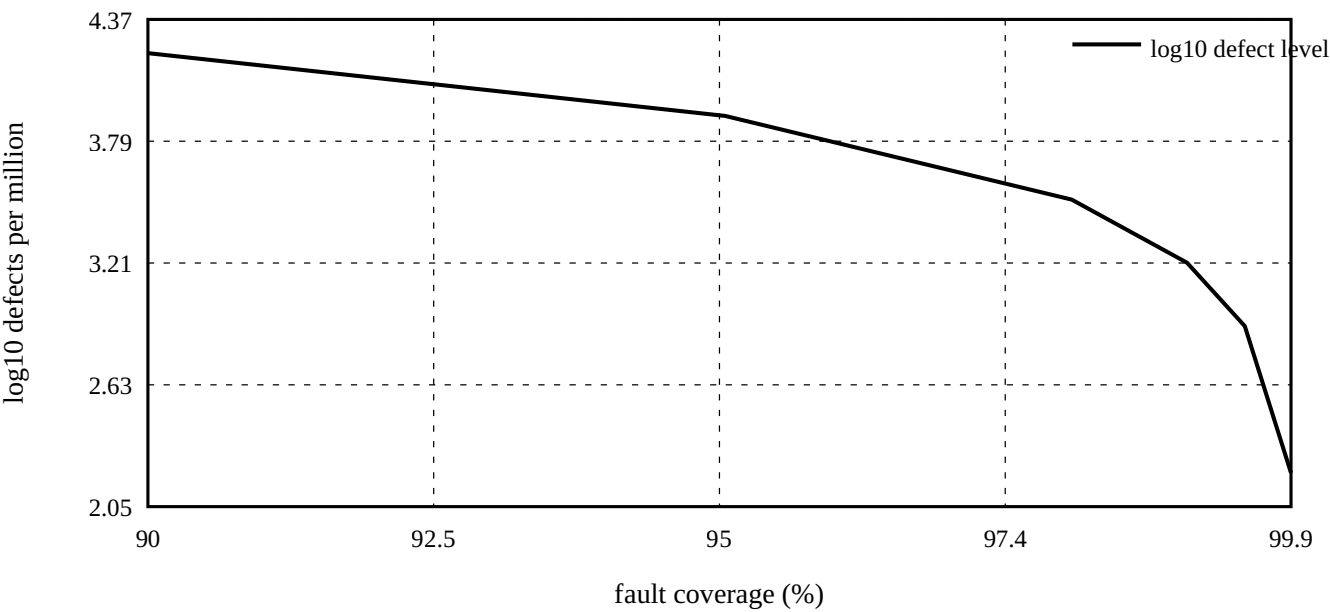


Figure 44. Defect level falls roughly a decade per decade of uncovered faults. The curve is why the last per cent of coverage is bought.

X27.3 BIST: testing without a tester

Table 330. Built-in self test

Kind	Mechanism	Why it exists
Memory BIST	An on-chip engine walks March algorithms through the array	Memories are most of the transistors and their faults are array-specific (coupling, retention, address decoder); a scan test cannot express them
Logic BIST	An LFSR generates pseudo-random patterns; a MISR compacts the responses	In-field self test for safety; reduces pattern storage
Repair	Spare rows and columns swapped in by fuses	Turns a failing die into a good one — Part X16's yield arithmetic
In-system test	BIST run at power-up or periodically	ISO 26262 latent-fault metric — Part X16

Worked example 40. Why random patterns alone are not enough, measured

Given. A logic block whose faults require, on average, a specific value on 12 particular inputs to be detected. Pseudo-random patterns set each input independently to 0 or 1.

Method. Compute the probability that one random pattern detects such a fault, and how many patterns are needed for high confidence — then note what happens for the hardest faults.

Probability one pattern detects	2.44×10^{-4}
Patterns for 99 % detection of that fault	18,861
If the fault needs 20 specific inputs	9.54×10^{-7}
Patterns then	4,828,869
At 100 MHz, time for those patterns	0.04829 s
Standard remedy	test points, or deterministic top-up patterns

How this is got wrong. By reporting the average and ignoring the tail. Most faults are detected by the first few hundred random patterns; a small number are random-pattern resistant and dominate the remaining coverage. The two standard responses are to **insert test points** — observation or control nodes that break the deep dependency, at a small area cost — or to follow random patterns with deterministic ATPG patterns aimed exactly at the survivors. **Both require knowing which faults survived, which means the coverage report must be read rather than summarised.**

Problem. A customer asks for your IP's fault coverage. What do you actually give them, and what do you need from them?

Solution. You cannot give a single number, and explaining why is part of the answer. Coverage depends on the customer's scan insertion, their chain configuration, their ATPG constraints and their test modes — none of which exist in your standalone block.

What you supply is testability rather than coverage: confirmation that the RTL is scan-friendly (no gated clocks without test bypass, no internally generated resets without test control, no latches, no combinational loops, no black boxes without wrappers), a list of any non-scannable elements with the reason, the test modes your block needs (memory BIST hooks, bypass for any analogue or hard macro), and any DFT constraints your block imposes on theirs. **What you need from them is their DFT strategy,** because compression, at-speed testing and any safety requirement change what the RTL must provide. **Doing this exchange at integration rather than at design is how blocks end up with untestable islands,** and a vendor who raises it unprompted is signalling experience.

X28. Caches and Coherence, Worked

Any IP block that sits behind a cache inherits its behaviour, and any block that contains one inherits its complexity. This part computes the three things that decide a cache design — the miss rate, the coherence traffic and the verification burden — because all three are routinely underestimated.

X28.1 The three Cs, measured rather than recited

Why a cache works at all

	Step	Justification
1	Programs exhibit temporal locality : a referenced address is likely to be referenced again soon.	<i>Loops, stacks, repeatedly used variables.</i>
2	And spatial locality : addresses near a referenced one are likely next.	<i>Arrays, instruction streams, structures.</i>
3	A cache exploits the first by keeping recent data and the second by fetching a whole line.	<i>The line size is the knob that trades one against the other.</i>
4	Misses classify as compulsory (first touch), capacity (working set exceeds the cache) and conflict (associativity too low).	<i>Hill's three Cs. Each has a different fix, which is why the classification is worth measuring rather than guessing.</i>
5	Compulsory misses fall with line size; conflict misses fall with associativity; capacity misses fall only with capacity.	<i>So a miss-rate problem must be classified before it can be addressed. Doubling associativity does nothing for a capacity problem, and the tool will not tell you which you have.</i>

Table 331. Measured three-Cs breakdown for a 16 kB cache (256 lines of 64 B) on two reference streams

Stream	Ways	Misses	Miss rate	Compulsory	Capacity	Conflict
mixed strided + loop	1	33,707	0.5618	16,384	17,264	59
	2	33,648	0.5608	16,384	17,264	0
	4	33,648	0.5608	16,384	17,264	0
	8	33,650	0.5608	16,384	17,266	0
four colliding arrays	1	30,000	1	256	0	29,744
	2	30,000	1	256	0	29,744
	4	256	0.008533	256	0	0
	8	256	0.008533	256	0	0

Each miss is classified by running a fully associative cache of the same capacity alongside: a miss the fully associative cache would have hit is a conflict miss. **The two streams are the two regimes.** The first is capacity-bound — it touches far more data than fits, so associativity buys almost nothing (59 conflict misses out of 33 707). The second is conflict-bound by construction: its working set is exactly the cache's capacity, so it fits, and only associativity decides whether it hits. **A miss-rate number without this breakdown cannot tell you which you have, and the two have different fixes.**

Table 332. Same 16 kB cache, 4-way, as line size is swept

Line (B)	Lines	Misses	Miss rate	Compulsory	Bytes fetched (kB)
16	1,024	33,841	0.564	16,576	528.77

Line (B)	Lines	Misses	Miss rate	Compulsory	Bytes fetched (kB)
32	512	33,713	0.5619	16,448	1053.5
64	256	33,648	0.5608	16,384	2103
128	128	16,823	0.2804	8,192	2102.9
256	64	8,411	0.1402	4,096	2102.8

The last column is the point that miss rate alone hides: **a longer line lowers the miss rate and raises the bandwidth**, because each miss moves more data. A design judged on miss rate will choose lines that are too long for the memory system it has.

X28.2 Average memory access time, and why it is the only metric

Worked example 41. Two caches, one better miss rate, and the wrong winner

Given. Cache A: 2 % miss rate, 2-cycle hit. Cache B: 1.4 % miss rate, 3-cycle hit (it is larger and more associative). Miss penalty 60 cycles.

Method. Average memory access time is hit time plus miss rate times penalty. Compute both and then find the penalty at which they cross.

Cache A AMAT	3.2 cycles
Cache B AMAT	3.84 cycles
Winner at 60-cycle penalty	A
Crossover penalty	166.7 cycles
Winner at 200-cycle penalty (DRAM)	B — 5.8 vs 6
What this depends on	the memory system, which is the customer's, not yours

How this is got wrong. By optimising the miss rate. Miss rate is a property of the cache and the workload; AMAT is a property of the whole system, and the two rank designs differently whenever the hit time changes. The crossover above is at 167 cycles, so the same two caches swap places between an SRAM-backed system and a DRAM-backed one. **An IP block containing a cache must therefore state the miss penalty its configuration was chosen for**, and ideally make the configuration a parameter, because the customer's penalty is not yours.

Table 333. The same two caches across miss penalties

Miss penalty (cycles)	AMAT A	AMAT B	Winner	Margin (%)
10	2.2	3.14	A	42.7
20	2.4	3.28	A	36.7
40	2.8	3.56	A	27.1
80	3.6	4.12	A	14.4
160	5.2	5.24	A	0.769
320	8.4	7.48	B	12.3

Computed from the definitions. **The winner changes in the middle of the range a real product spans**, which is the general reason cache parameters are made configurable rather than fixed.

X28.3 Coherence: the protocol and its traffic

Table 334. MESI states and what each permits

State	Meaning	Read hit	Write hit	On a remote read
Modified	Dirty, exclusive	Yes	Yes, silently	Write back, go to S
Exclusive	Clean, exclusive	Yes	Yes, silently — this is why E exists	Go to S
Shared	Clean, possibly shared	Yes	No — must invalidate others first	Stay S
Invalid	Not present	Miss	Miss	—

Graduate · **The E state exists entirely to make private data cheap.** Without it, a line loaded by a read would be S, and the first write to it would need a bus transaction to invalidate copies that do not exist. Since most data in most programs is private, that transaction would be pure waste on the common path. E says ‘clean and I am the only holder’, so the write is silent. The cost is that the interconnect must be able to tell a requester whether anyone else has a copy — a shared signal on a bus, or a directory lookup — which is a real structural requirement that exists solely to support this optimisation. **It is a good example of a protocol state that looks redundant on a state diagram and is load-bearing in the traffic.**

Table 335. Coherence cost against core count

Cores	Snoop messages per miss (all-to-all)	Snoops each cache must absorb	Directory bits per line (full bit vector)	Directory overhead (% of line)	Practical choice
2	2	1	2	0.3906	snoop
4	12	3	4	0.7812	snoop
8	56	7	8	1.562	snoop
16	240	15	16	3.125	directory
32	992	31	32	6.25	directory
64	4,032	63	64	12.5	directory

*Snooping broadcasts, so its traffic is quadratic in cores and every cache must absorb every other cache's misses — which becomes the bottleneck long before the wires do. A **directory is linear in traffic and pays a storage overhead**, and the crossover in practice is around eight to sixteen cores.*

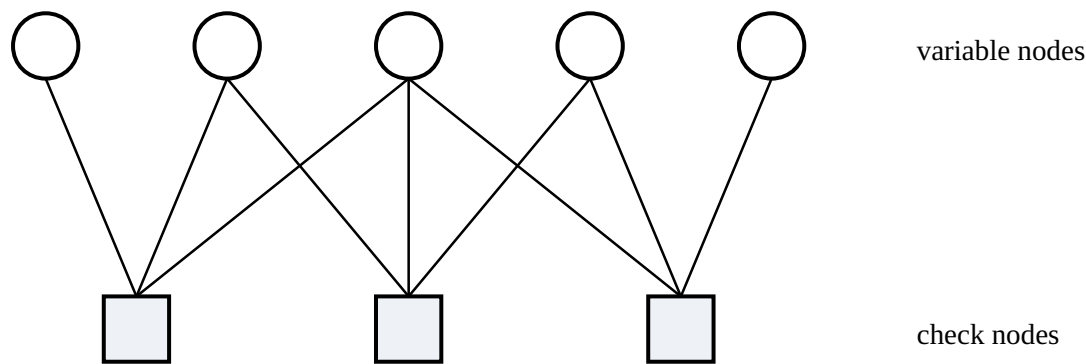
Coherence is where verification cost explodes, and the reason is combinatorial. A four-state protocol with four caches has 4^4 combinations of state for one address, and the interesting bugs involve two addresses, an eviction in flight, and a request that arrives between the two halves of another transaction. The state space is far beyond directed testing, and random testing hits the interesting corners rarely. **This is the clearest case in the book for formal verification:** a coherence protocol is exactly the kind of deep, narrow state space that model checking handles and simulation does not, and proving the protocol at the specification level — before any RTL — is standard practice at companies that ship coherent systems. What formal cannot do is prove the *implementation* matches the protocol at scale, so both are needed.

Problem. Your accelerator has a private cache and the host writes its input buffer. What are the choices, and what does each cost?

Solution. Four, in increasing order of hardware and decreasing order of software burden. **Uncached access:** the accelerator does not cache the buffer at all; correct by construction, and it gives up locality entirely, which for a streaming accelerator that touches each byte once is often the right answer. **Software coherence:** the driver flushes and invalidates around each transfer; cheap in hardware, and it is a source of extremely nasty bugs because a missing invalidate is intermittent and data-dependent, and the failure appears in the customer's driver rather than in your block. **IO coherence (one-way):** the accelerator's accesses snoop the host caches but not the reverse; this is what most SoC interconnects offer today, it removes the flush burden for host-written data, and it costs interconnect support rather than accelerator complexity. **Full coherence:** the accelerator participates in the protocol as a peer, which gives the best performance for fine-grained sharing and costs a coherent cache controller plus the verification burden described above. **For a design house the recommendation is usually the third option:** it captures most of the benefit, the complexity lives in the customer's interconnect where it already exists, and it does not put a protocol state machine into your verification scope.

X29. LDPC and Polar Codes, Worked

These are the codes that reach within a fraction of a decibel of capacity and they are, between them, the error correction of 5G, Wi-Fi, high-speed Ethernet and modern storage. Their hardware is dominated by memory and interconnect rather than arithmetic, which makes them an unusually good case study in the gap between an algorithm's operation count and its silicon.



Decoding passes messages along the edges. A short cycle in this graph makes a message come back as if it were independent evidence.

Figure 45. A Tanner graph. The code is the graph; the decoder is message passing on it.

X29.1 Belief propagation, and the approximation everyone ships

From the check equation to the min-sum rule

	Step	Justification
1	A check node enforces that the XOR of its connected bits is zero.	Parity check.
2	The log-likelihood ratio of a XOR of independent bits satisfies $\tanh(L/2) = \prod \tanh(L_i/2)$.	The standard identity: probabilities of a parity multiply in the tanh domain.
3	So the exact check update is $L = 2 \tanh^{-1}(\prod \tanh(L_i/2))$.	Accurate and unimplementable: a transcendental function per edge, at hundreds of gigabits per second.
4	Because tanh saturates, the product is dominated by its smallest term.	Every factor with large $ L $ is close to ± 1 .
5	Min-sum: $L \approx (\prod \text{sgn } L_i) \cdot \min L_i $.	A comparison and a sign product — no multipliers, no tables.
6	Min-sum overestimates the magnitude, so it is scaled or offset: $\alpha \cdot \min$ or $\max(\min - \beta, 0)$.	One constant recovers most of the loss, and X29.2 measures how much.

Table 336. Measured bit error rate of a (96, $d_v=3$, $d_c=6$) LDPC code under three check-node rules, 8 iterations, 60 trials per point

SNR (dB)	Uncoded (last trial)	Plain min-sum	Scaled min-sum ($\alpha=0.75$)	Offset min-sum ($\beta=0.5$)
1	0.146	0.106	0.0767	0.0774
2	0.0625	0.0436	0.024	0.0234
3	0.104	0.00955	0.00642	0.00712
4	0.0938	0	0	0

Decoded by a direct implementation of the message-passing rules, all-ones codeword. **The scaling and offset constants cost one multiplier or one subtractor in the whole decoder and recover a measurable part of the loss; this is the cheapest improvement available in an LDPC decoder and it is universal in shipping designs.**

Graduate · Why the all-ones codeword is a legitimate test and not a shortcut. For a linear code over a symmetric channel, the error probability is independent of the transmitted codeword: the decoder's behaviour depends only on the noise realisation relative to the sent word, and the code's symmetry makes every word equivalent. So simulating the all-zero (or all-ones, under the sign convention used here) word is exact, not approximate, and it removes the encoder from the simulation entirely. **The condition to check is that the decoder is genuinely symmetric** — a quantised implementation with an asymmetric saturation or a biased rounding rule is not, and then the shortcut becomes invalid. That is a real trap in fixed-point LDPC verification and a good reason to run a full encoder at least once against the shortcut.

X29.2 Where the hardware cost actually is

Table 337. LDPC decoder memory and interconnect, 6-bit messages

Block length n	Rate	Checks m	Edges	Message memory (kB)	Message accesses per iteration
648	0.5	324	1,944	1.4238	3,888
1,944	0.5	972	5,832	4.2715	11,664
8,448	0.8	1,689	33,792	24.75	67,584
26,112	0.86	3,655	104,448	76.5	208,896

The edge count sets everything. **A decoder is a memory with comparators attached:** the message memory dominates area, and moving those messages between the variable and check phases dominates power. Arithmetic — comparisons and adds — is almost free by comparison.

Table 338. Decoder architectures

Architecture	Parallelism	Memory	Throughput	Where used
Fully parallel	Every edge has wires	Registers	Highest	Short codes; the interconnect is the limit and routing congestion sets the block length
Layered / row-parallel	One block-row of the parity matrix at a time	SRAM banks	High; roughly halves the iterations needed	The standard for 5G and Wi-Fi quasi-cyclic codes
Serial	One check at a time	Single memory	Low	Area-critical, low-rate applications
Flooding	All checks then all variables	Two message arrays	Moderate	Simple; needs about twice the iterations of layered

Undergraduate review · Why quasi-cyclic codes are the ones that get standardised. A quasi-cyclic parity-check matrix is built from circulant blocks — identity matrices shifted by a stored amount. Three consequences follow, and all three are hardware consequences rather than coding ones. The matrix is described by a small table of shift values instead of a full incidence list, so the code fits in a few hundred bytes of ROM. A whole block-row can be processed in parallel with perfectly regular addressing, because a circulant is a barrel shifter. And the same hardware serves many block lengths by changing the circulant

size, which is what lets one decoder cover a standard's whole rate and length family. **The structure exists for the decoder's benefit, at a small cost in threshold performance**, and that trade is why every modern LDPC standard is quasi-cyclic.

X29.3 Polar codes, and why they won the control channel

Channel polarisation in one page

	Step	Justification
1	Combine two uses of a channel W as $u_1 \oplus u_2$ and u_2 .	<i>The basic 2×2 transform.</i>
2	Decoding u_1 first sees a worse channel; decoding u_2 given u_1 sees a better one.	<i>The mutual information is conserved but redistributed: $I(W^-) + I(W^+) = 2I(W)$.</i>
3	Recurse n times over 2^n uses.	<i>The fraction of synthetic channels that are nearly perfect tends to $I(W)$ and the rest tend to useless.</i>
4	Put information on the good channels and freeze the rest to known values.	<i>That is the entire code construction — no search, no optimisation, and it provably achieves capacity as the length grows.</i>
5	Decoding is successive cancellation: decide bits in order, using earlier decisions.	<i>Inherently serial, which is the hardware problem; latency grows with block length even though the operation count is $N \log N$.</i>
6	Adding a CRC and keeping L candidate paths (SCL) closes most of the gap to maximum likelihood.	<i>At $L \times$ the memory and the sorting network, which is what 5G specifies for control channels.</i>

Table 339. Successive-cancellation list decoding cost

N	List size L	SC latency (cycles, unoptimised)	Path memory (bytes, 6-bit LLR)	Sorter comparisons per stage
128	1	254	96	7
128	2	254	192	14
128	4	254	384	28
128	8	254	768	56
256	1	510	192	8
256	2	510	384	16
256	4	510	768	32
256	8	510	1536	64
512	1	1,022	384	9
512	2	1,022	768	18
512	4	1,022	1536	36
512	8	1,022	3072	72
1,024	1	2,046	768	10
1,024	2	2,046	1536	20
1,024	4	2,046	3072	40
1,024	8	2,046	6144	80

Latency is the polar decoder's problem and list size is its area problem. The published architectural work is almost entirely about shortening the serial schedule — recognising sub-blocks that can be decoded in one step rather than bit by bit.

Problem. 5G uses polar codes for control channels and LDPC for data. Why not one code for both?

Solution. Because the two channels have opposite requirements and each code is better at one of them. **Control messages are short** — tens of bits — and at those lengths LDPC's performance degrades badly, because the Tanner graph cannot avoid short cycles and the asymptotic analysis that makes LDPC excellent does not apply. Polar codes with CRC-aided list decoding are close to optimal at short lengths, and their **error detection** is excellent, which matters enormously for control: a mis-decoded control message is worse than a detected failure, exactly as Part X5 argued for Reed–Solomon. **Data blocks are long and need throughput**, and there LDPC's parallel decoding wins decisively while polar's serial successive cancellation would set a latency floor. **The standard is not being indecisive; it is applying the right tool at each length**, and the lesson generalises: code choice is dominated by block length and by whether detection or throughput is the binding requirement.

X30. OFDM and MIMO, Worked

Every wireless standard of the last twenty years is OFDM, and most are MIMO. Both are, from a hardware point of view, ways of converting a hard problem into a large number of easy ones — and the conversion has costs that are computable and are the subject of most of the implementation literature.

X30.1 Why OFDM turns equalisation into division

The cyclic prefix is the whole trick

	Step	Justification
1	A multipath channel convolves the transmitted signal with an impulse response of length L .	<i>Linear convolution.</i>
2	Circular convolution in time is multiplication in the DFT domain.	<i>The convolution theorem — but only for circular convolution.</i>
3	A channel performs <i>linear</i> convolution, which is not circular, so the theorem does not apply.	<i>The blocks bleed into each other.</i>
4	Prepend the last $L-1$ samples of each block to itself.	<i>Now the tail that bleeds in is the same as what would have wrapped around, so over the retained window the linear convolution is circular.</i>
5	Therefore $Y_k = H_k X_k + N_k$, one complex multiply per subcarrier.	<i>Equalisation becomes one complex division per subcarrier</i> , replacing a time-domain equaliser of length L . This is the entire reason OFDM exists.
6	Cost: the prefix is redundancy, so the rate falls by $N/(N+L-1)$.	<i>And the peak-to-average ratio rises</i> , because a sum of many independent subcarriers is nearly Gaussian — X30.2.

Table 340. Cyclic-prefix overhead and the delay spread it tolerates

Standard	FFT size	CP samples	Rate lost (%)	Symbol duration (μ s)	Max echo path (m)
802.11a	64	16	20	4	240
802.11ax	256	32	11.11	14.4	480
LTE 20 MHz	2,048	144	6.569	109.6	2160
5G NR 100 MHz SCS30	4,096	288	6.569	43.84	864

The last column is the design driver: the prefix must exceed the channel's delay spread, and delay spread is set by the environment — tens of metres indoors, kilometres in a large cell. **The prefix is therefore not a free parameter but a property of the deployment**, which is why standards define several and negotiate.

Worked example 42. Sizing an OFDM receiver's FFT from the standard

Given. 5G NR, 100 MHz channel, 30 kHz subcarrier spacing, 4096-point FFT, 14 symbols per slot, 0.5 ms slot.

Method. Compute the FFT rate the receiver must sustain, then apply Part X6's resource arithmetic to see what that implies.

Symbols per second	28000
FFT size	4,096
Complex multiplies per FFT	24,576
Complex multiplies per second	6.88×10^8
At 500 MHz, multipliers needed	2
Memory accesses per second (4 per butterfly)	2.75×10^9
SRAM ports needed at 500 MHz	6

How this is got wrong. By sizing for one FFT and forgetting the antennas and the layers. A four-antenna receiver needs four of these, and channel estimation needs more transforms again. The multiplier count is modest and the **memory port count is not** — which is Part X6's conclusion arriving in a specific product, and the reason an OFDM receiver's floorplan is dominated by buffer SRAM rather than by arithmetic.

X30.2 Peak-to-average ratio: the cost OFDM does not advertise

Table 341. Measured peak-to-average power ratio of QPSK-OFDM, 400 symbols per point

Subcarriers N	Mean PAPR (dB)	99.9th percentile (dB)	Observed max (dB)	Theoretical worst case $10\log N$ (dB)
64	6.791	9.886	9.926	18.06
256	7.893	9.95	10.03	24.08
1,024	8.78	10.93	11.04	30.1
4,096	9.469	11.87	12.26	36.12

Computed by transforming random QPSK symbols. **The theoretical worst case is never approached** — it needs all subcarriers to align in phase, which has vanishing probability — but the 99.9th percentile grows slowly and is the number the power amplifier must be backed off by.

PAPR is a power-amplifier problem, and the power amplifier is the most expensive and least efficient thing in a radio.

An amplifier must be operated below its compression point by the PAPR or the peaks clip, generating out-of-band emission that violates the spectral mask. Backing off 8 dB means operating at a small fraction of the amplifier's capability, and amplifier efficiency falls steeply with back-off — which in a handset is battery life and in a base station is the electricity bill and the cooling. **This is why PAPR reduction (clipping and filtering, tone reservation, selected mapping) and digital pre-distortion exist**, and why single-carrier FDMA was chosen for the LTE uplink: the handset's amplifier is the constraint, so the uplink gave up OFDM's convenience to get several dB of PAPR back. **An architectural decision in a standard, driven entirely by one analogue component's efficiency curve.**

Table 342. Indicative amplifier efficiency against back-off (class-AB model, 78 % peak)

Back-off (dB)	Efficiency (%)	DC power per watt of RF	Heat per watt of RF
3	55.22	1.811	0.8109
5	43.86	2.28	1.28
7	34.84	2.87	1.87
9	27.68	3.613	2.613

Back-off (dB)	Efficiency (%)	DC power per watt of RF	Heat per watt of RF
11	21.98	4.549	3.549

A first-order model, not a measurement of any particular device. **Its only job is to show the shape:** efficiency falls roughly as the voltage back-off, so every dB of PAPR costs real power, and a few dB of PAPR reduction pays for a great deal of digital signal processing.

X30.3 MIMO: capacity, and the detector that costs it

Why multiple antennas multiply capacity

	Step	Justification
1	With N_t transmit and N_r receive antennas, $\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{n}$.	$\mathbf{H}$ is $N_r \times N_t$ complex.
2	Decompose $\mathbf{H} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^H$; the channel becomes $\min(N_t, N_r)$ parallel scalar channels with gains σ_i .	The singular value decomposition of Part X24, used as a change of basis.
3	Capacity = $\sum \log_2(1 + P_i \sigma_i^2 / N_0)$.	Sum of parallel Gaussian channels, with power allocated by water-filling.
4	So capacity grows linearly with $\min(N_t, N_r)$, not logarithmically with power.	This is the whole point: spatial multiplexing buys rate that more power cannot.
5	It requires the σ_i to be comparable — a well-conditioned $\mathbf{H}$.	Line-of-sight channels are ill conditioned and give almost no multiplexing gain. Rich scattering, which sounds like a bad channel, is what makes MIMO work — and Part X24's condition number is again the deciding quantity.

Table 343. Measured ergodic capacity at 10 dB SNR: rich scattering against pure line of sight, 400 channel realisations

Configuration	Rich scattering (bit/s/Hz)	Line of sight	Ratio	$\min(N_t, N_r)$
1×1	2.931	3.459	0.8472	1
2×2	5.618	4.392	1.279	2
4×4	11	5.358	2.054	4
8×8	21.71	6.34	3.424	8
2×4	8.036	5.358	1.5	2
4×2	6.277	4.392	1.429	2

A line-of-sight channel is rank one however many antennas it has, so its capacity does not grow with antenna count — the measurement confirms it. The second column tracks the last column, which is the linear growth the derivation predicted.

Table 344. MIMO detectors, and what each costs

Detector	Complexity	Performance	Hardware
Zero forcing	$O(N^3)$ once per channel	Poor at low SNR — noise enhancement (Part X24)	A matrix inverse
MMSE	Same	Better; regularised	Same plus a diagonal add
Ordered SIC (V-BLAST)	N passes	Noticeably better than linear	Sequential — latency grows with N ; error propagation
Sphere / K-best	Tree search, bounded	Near maximum likelihood	A sorter and a tree walk — the standard hardware compromise

Detector		Complexity	Performance	Hardware
Maximum likelihood	likeli-hood	M^{N_t} candidates	Optimal	Infeasible beyond small cases: 16-QAM 4×4 is 65 536 candidates per symbol

Problem. Why does massive MIMO make the detector *easier* rather than harder, despite having far more antennas?

Solution. Because when N_r is much larger than N_t the columns of $\mathbf{H}$ become nearly orthogonal — a consequence of concentration of measure, since independent high-dimensional random vectors are nearly orthogonal with high probability. Nearly orthogonal columns mean $\mathbf{H}^H \mathbf{H}$ is nearly diagonal, so the simple matched filter (conjugate beamforming) is nearly optimal and the noise enhancement that makes zero forcing bad at low SNR essentially vanishes. **The hard detector problem is a symptom of having barely enough antennas**, and adding antennas at the base station — where size and power are affordable — dissolves it. The cost moves elsewhere and should be named: channel estimation for many antennas needs pilot resources and becomes the limiting factor (pilot contamination), the number of RF chains dominates cost and power, and the digital front end must move enormous sample rates from the antennas to the processing. **Massive MIMO trades a detection problem for an interconnect and calibration problem**, which is a trade the digital designer should recognise as favourable and should be ready to quantify.

X31. Analogue Building Blocks, Budgeted

A digital designer does not size transistors, but does have to say whether an analogue requirement is reasonable before committing to an architecture. The quantities that answer that are few and computable: thermal noise, matching, gain-bandwidth and settling. This part gives them.

X31.1 kT/C — the noise floor you cannot design away

Why sampled noise depends only on the capacitor

	Step	Justification
1	A switch has resistance R ; its thermal noise density is $4kTR$ V^2/Hz .	<i>Johnson–Nyquist.</i>
2	The RC network's noise bandwidth is $1/(4RC)$.	<i>Integrating a single-pole response gives $\pi/2$ times the 3 dB bandwidth.</i>
3	So the sampled noise power is $4kTR \cdot 1/(4RC) = kT/C$.	The resistance cancels. A better switch does not help; only a bigger capacitor does.
4	Doubling resolution means halving the noise voltage, which means quadrupling the capacitor .	<i>And capacitance is area, and it is also the load the driver must charge, so power rises too.</i>
5	Each extra bit of a sampled system costs 4× the capacitor and roughly 4× the power.	<i>This single relation explains the shape of every converter's energy-per-conversion curve above about 10 bits.</i>

Table 345. kT/C noise and the resolution it permits (1 V full scale, 300 K)

Capacitance (fF)	RMS noise (μV)	SNR (dB)	ENOB	Approx. area (μm^2 at 2 fF/ μm^2)
10	643.58	54.8	8.81	5
100	203.52	64.8	10.47	50
1000	64.358	74.8	12.13	500
10000	20.352	84.8	13.79	5000
1.0000×10^5	6.4358	94.8	15.45	50000

Computed from kT/C alone, ignoring every other noise source, so these are optimistic. A 14-bit sampled system needs picofarads, and the area and power that implies is why high-resolution converters are large and why oversampling architectures, which trade speed for resolution, exist.

Worked example 43. Can this ADC specification be met?

Given. A customer asks for a 16-bit, 100 MS/s SAR ADC on 1 V supply, ‘small area’.

Method. Work the kT/C requirement, then the settling requirement, then compare against what the process offers.

Required SNR for 16 bits	98.08 dB
Signal RMS at 1 V full scale	353.6 mV
Permitted noise RMS	4.41 μV
Required capacitance	212.96 pF
Array area at 2 fF/ μ m ²	1.0648$\times 10^5$ μm²
Settling time available at 100 MS/s, 16 SAR cycles	625 ps
Time constants needed for 16-bit settling	11.09
Required RC	56.36 ps
Verdict	the capacitor and the settling requirement are in direct conflict

How this is got wrong. By treating ‘16-bit’ and ‘100 MS/s’ as independent requirements. The noise requirement forces a large capacitor; the speed requirement forces that capacitor to settle in picoseconds, which forces a very low switch resistance and therefore an enormous switch, which loads the previous stage. This is why 16 bits at 100 MS/s is a time-interleaved or pipelined architecture and not a single SAR, and why the answer to the customer is a **question about their actual SNR requirement in the signal band** — if they need 16 bits over 1 MHz rather than over 50 MHz, oversampling changes the arithmetic completely.

X31.2 Matching: why analogue area is set by statistics

Pelgrom's law

	Step	Justification
1	Threshold voltage varies between nominally identical devices because of random dopant fluctuation and line-edge roughness.	<i>A counting phenomenon: fewer dopants under a smaller gate means larger relative variation.</i>
2	The standard deviation of the mismatch between two devices is $\sigma_{\Delta V_{th}} = A_{VT}/\sqrt{WL}$.	<i>Pelgrom's law, verified across many processes. A_{VT} is a process constant of a few mV$\cdot\mu$m.</i>
3	So halving the mismatch requires four times the area.	<i>The same 4$\times$ law as kT/C, arriving from a completely different mechanism.</i>
4	A comparator's input-referred offset is this mismatch; a current mirror's error is proportional to it.	<i>Which sets the minimum size of every matched pair in a converter.</i>
5	Digital calibration converts area into computation.	<i>Measure the offset once, store it, subtract it. This is the dominant trend in mixed-signal design and the reason the digital content of converters keeps growing.</i>

Table 346. Pelgrom matching against device area ($A_{VT} = 3.5$ mV $\cdot\mu$ m)

WL (μm^2)	$\sigma_{\Delta V_{th}}$ (mV)	3σ spread (mV)	Bits resolvable without calibration at 1 V FS	Pair area (μm^2)
0.01	35	210	2.252	0.02
0.1	11.07	66.41	3.913	0.2
1	3.5	21	5.573	2
10	1.107	6.641	7.234	20
100	0.35	2.1	8.895	200

Without calibration, matching alone limits an uncalibrated comparator to about eight bits at sane areas. Every converter above that resolution calibrates, and the calibration engine is digital — which is how a digital designer ends up owning a large part of an analogue block.

Graduate · The calibration trade, stated as arithmetic. Suppose a comparator needs 1 mV of offset. Pelgrom says that costs about $12 \mu\text{m}^2$ of gate area per device, and in a 64-way time-interleaved ADC that is repeated 64 times, in the fastest and most power-hungry part of the chip. Alternatively, build a minimum-size comparator with 10 mV of offset and add a trim DAC plus a digital loop that measures and cancels it. The trim DAC is small, the loop is a counter and a comparator, and the whole thing is synthesised. **The crossover is far to the digital side and moves further every process node**, because digital area shrinks and A_{VT} improves only slowly. This is the single clearest instance of the general trend this book keeps meeting: **buy analogue precision with digital computation.**

X31.3 Gain, bandwidth and settling

Table 347. What a resolution demands of an amplifier

Bits	Settling accuracy	Time constants needed	Minimum open-loop DC gain (dB)	Settling in units of 1/GBW
8	0.00195	6.238	54.19	0.9929
10	4.88×10^{-4}	7.625	66.23	1.213
12	1.22×10^{-4}	9.011	78.27	1.434
14	3.05×10^{-5}	10.4	90.31	1.655
16	7.63×10^{-6}	11.78	102.4	1.875

*Static accuracy needs loop gain (finite gain leaves a proportional error); dynamic accuracy needs time constants. **16-bit accuracy needs about 100 dB of DC gain and twelve time constants**, and those two requirements fight each other because high gain means more stages means more poles means harder compensation.*

Table 348. The analogue specifications a digital architect should be able to sanity check

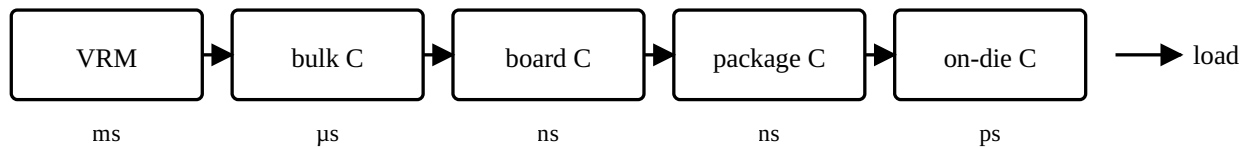
Quantity	Rule of thumb	If the requirement exceeds it
Sampled SNR	kT/C ; 14 bits needs $>1 \text{ pF}$ at 1 V	Oversample, or relax full scale, or use a different architecture
Uncalibrated comparator offset	Pelgrom; ~ 8 bits at sensible area	Add digital calibration
Amplifier DC gain	$\sim 6 \text{ dB per bit}$	Gain boosting, or a correlated-double-sampling scheme
Aperture jitter	$\text{SNR} = -20 \log(2\pi f_{in} \sigma_t)$ — Part X12	A better clock, not a better converter
Reference noise	Appears directly at the output	Filter it; and remember the filter capacitor is area too
Supply rejection	Falls with frequency; poor above the loop bandwidth	This is why the PDN of Part X32 matters to analogue blocks

Problem. Your digital block shares a supply with an ADC and the customer reports spurs at your clock frequency. Whose problem is it?

Solution. Both of yours, and the useful response is to find out which mechanism rather than which party. There are three paths and they are distinguishable by measurement. **Supply coupling:** your switching current modulates the shared rail, and the converter's finite power-supply rejection turns that into an input offset; test by changing your block's activity and watching whether the spur amplitude tracks, and by separating the supplies if the board permits. **Substrate coupling:** switching current injects into the substrate and reaches the converter's sensitive nodes; this is layout, and the remedies are guard rings, deep n-well isolation and distance. **Clock coupling:** your clock capacitively couples into the sampling network, which is the worst of the three because it lands exactly at a fixed frequency and its harmonics. **The digital side's contribution, and it is a real one, is to reduce di/dt :** spread the clock tree's switching with deliberate skew, gate clocks aggressively so the average current is lower, add on-chip decoupling, and avoid large simultaneous switching events. **None of that is exotic, and all of it is far cheaper than asking the analogue designer for another 20 dB of supply rejection,** which may not be available at any price.

X32. The Power Delivery Network, Worked

Every timing number in Part X2 assumed a supply voltage. The network that delivers it is a distributed RLC structure whose impedance must be low across six decades of frequency, and when it is not, the result is a voltage droop that shows up as a timing failure nobody can reproduce. This is a digital designer's problem even though it looks like an analogue one.



Each stage covers a different decade of time.
A gap between two of them is an impedance peak, and an impedance peak is a droop.

Figure 46. The power delivery hierarchy. Each element handles a different timescale because each is limited by its own parasitic inductance.

X32.1 Target impedance, and where the number comes from

From a droop budget to an impedance specification

	Step	Justification
1	The supply must stay within $\pm\delta$ of nominal for the timing libraries to apply.	<i>Typically 5 %. Outside it, the delays are not the ones you signed off with.</i>
2	A current step ΔI produces a voltage $Z(f)\Delta I$.	<i>Ohm's law for an impedance.</i>
3	Target impedance $Z_t = \delta V_{dd} / \Delta I_{max}$.	<i>The one number the whole PDN is designed to, and it must be met at every frequency, not on average.</i>
4	ΔI_{max} is the worst-case step, not the average current.	<i>A clock-gated block waking up, or a test pattern that toggles everything, is a far larger step than any functional average — which is why maximum-toggle test patterns can fail a part that works in its application.</i>
5	Falling voltage and rising current per node make Z_t shrink every generation, quadratically.	<i>This is why PDN design has become progressively harder rather than benefiting from scaling.</i>

Table 349. Target impedance across process generations (5 % droop budget)

Node	V_{dd} (V)	Peak current (A)	Z_t (m Ω)	Peak power (W)	Droop budget (mV)
180 nm	1.8	2	45	3.6	90
90 nm	1.2	10	6	12	60
28 nm	1	40	1.25	40	50
7 nm	0.8	120	0.3333	96	40
5 nm	0.75	250	0.15	187.5	37.5

The current figures are indicative of a large SoC at each generation rather than any specific part. **The impedance target falls by more than two orders of magnitude across the table**, which is why on-die decoupling and integrated voltage regulators appeared.

Worked example 44. Sizing on-die decoupling capacitance

Given. A block draws 5 A when active and is clock-gated. On wake-up the current rises in 2 ns. The package inductance is 100 pH and the droop budget is 40 mV.

Method. The package cannot supply current faster than its inductance allows, so on-die capacitance must cover the interval until the package catches up. Compute the charge needed and hence the capacitance.

Current step	5 A
Rise time	2 ns
$L di/dt$ if the package had to supply it	250 mV
Droop budget	40 mV
Verdict	the package alone exceeds the budget by 6×
Charge to supply during the step	5 nC
Capacitance for a 40 mV droop	125 nF
Area at 10 fF/ μm^2 (MOS cap)	12.5 mm ²

How this is got wrong. By budgeting only the resistive drop. At a nanosecond timescale the package is an inductor, not a wire, and $L di/dt$ dominates the static IR term by a large factor. **The remedy is not lower resistance but local charge storage**, and the area it costs is real — on-die decoupling is commonly several per cent of die area and in some designs more than ten. A second, cheaper remedy belongs to the digital designer: **ramp the current instead of stepping it**, by staging the clock-gate release across several cycles, which is a few gates of logic and can reduce the required capacitance by a large factor.

X32.2 Resonance: the peak that puts the droop where you are not looking

Table 350. Resonant peaks formed by each inductance with the capacitance below it

L (pH)	C (nF)	R (mΩ)	Resonant frequency (MHz)	Q	Peak impedance (mΩ)
1000	100	5	15.915	20	100
1000	1000	5	5.0329	6.325	31.62
100	100	2	50.329	15.81	31.62
100	10	2	159.15	50	100
10	100	1	159.15	10	10

Every inductance in the hierarchy resonates with the capacitance on the load side of it, and at resonance the impedance is Q times the series resistance. **A high- Q peak is the dangerous case:** if the current spectrum has energy there, the droop is far worse than the target impedance suggests.

The first-droop, second-droop, third-droop language engineers use is just this resonance hierarchy, and the dangerous one is usually the mid-frequency droop. The first droop (hundreds of megahertz) is handled by on-die capacitance; the third (kilohertz) by the regulator's control loop. The second — tens to a couple of hundred megahertz, set by package inductance against on-die capacitance — sits in a gap between the two remedies and is where the highest- Q peak usually lands. **It is also, unhelpfully, right where a processor's activity changes:** a loop that alternates between a heavy and a light phase at that rate excites it directly, which is the origin of the classic 'this benchmark fails and that one does not' report. The digital remedies are to add damping (series resistance in some decoupling, deliberately), to spread activity in time, and to use an adaptive

clock that stretches the period when a droop is detected — which converts a functional failure into a small performance loss and is now common in large processors.

X32.3 Electromigration and the static budget

Table 351. Current a wire can carry before electromigration limits its life (indicative 1.5×10^6 A/cm², 0.2 μ m thick)

Width (μ m)	Cross-section ($\times 10^{-8}$ cm ²)	Max current (mA)	mA per μ m of width	Resistance per μ m length ($\mu\Omega$)
0.1	0.02	0.3	3	2.2000×10^5
0.2	0.04	0.6	3	1.1000×10^5
0.5	0.1	1.5	3	44000
1	0.2	3	3	22000
2	0.4	6	3	11000

Indicative constants, not a PDK. The useful content is the third column: a few milliamps per micron of width is the order of magnitude, so a 5 A block needs of the order of a millimetre of total metal width to feed it — which is why power grids are wide, numerous and on the thick upper metal layers.

Problem. Your block passes timing in every corner and fails intermittently in silicon at high activity. How do you decide whether it is the PDN?

Solution. Look for the signature, which is specific and distinguishable. **A PDN-induced failure is activity dependent rather than pattern dependent:** it appears when many things switch at once, not when a particular logical case occurs, so a test that toggles a lot fails while a functionally harder but quieter test passes. **It moves with supply and with frequency in a particular way:** raising the supply helps more than it should for a simple timing problem, because it increases the margin against droop as well as speeding the logic. **It correlates with di/dt, not with average power,** so a test with the same average current but smoother activity passes. The measurements that settle it are an on-die droop monitor if one exists — and if none exists, that is the lesson for the next design, because a ring-oscillator-based droop detector is a few hundred gates — and an experiment that staggers the wake-up of the block's clock-gating domains, which changes only di/dt. **If staggering fixes it, the PDN is the cause and the cheapest fix is the staggering you just used to diagnose it.**

X33. Boolean Reasoning: BDDs, SAT and Equivalence Checking

Three tools an IP designer uses every week — logic equivalence checking, formal property verification and automatic test pattern generation — are the same engine underneath: a decision procedure for Boolean formulas. Knowing what that engine can and cannot do explains why one proof finishes in seconds and another never finishes, which is otherwise the most frustrating thing about these tools.

X33.1 Representations, and why the choice is everything

Table 352. Ways to represent a Boolean function

Representation	Size	Equivalence check	Where it is used
Truth table	2^n always	Trivial	Small cells; library characterisation
Sum of products	Can be exponential	Hard	Two-level minimisation, PLAs
BDD (reduced, ordered)	Canonical — and variable-order dependent	Pointer comparison	Symbolic model checking; some equivalence checking
AIG (and-inverter graph)	Linear in the circuit	Needs a solver	The working representation of every modern synthesis and verification tool
CNF for SAT	Linear via Tseitin encoding	NP-complete, and fast in practice	Formal verification, ATPG, equivalence

Graduate · **Canonicity is the property that makes BDDs both magical and fragile.** Two functions are equal if and only if their reduced ordered BDDs are the identical graph, so equivalence is a pointer comparison rather than a search. The price is that the size depends catastrophically on the variable order: the same function can be linear under one order and exponential under another, and finding the best order is itself NP-hard. The canonical example is a multiplier, whose BDD is exponential under *every* order — which is exactly why BDD-based equivalence checking fails on datapaths and why modern tools use AIGs plus SAT instead. **When a tool reports that it is ‘building BDDs’ and then runs out of memory, this is what happened,** and the remedy is a different engine rather than a bigger machine.

Table 353. Measured decision-diagram size for $x_0y_0 + x_1y_1 + \dots$ under two variable orders

Variables	Product terms	Interleaved order (nodes)	Separated order (nodes)	Ratio
4	2	4	6	1.5
8	4	8	30	3.75
12	6	12	126	10.5
16	8	16	510	31.88

Built by exhaustive construction with subfunction sharing, which is what reduction does. **The interleaved column is exactly n ; the separated column is $2^{n/2+1}-2$.** Linear against exponential, on the same function with the same tool. The ratio is modest at these sizes because the exponent is small — at 40 variables it is a factor of 52 000 — $(2^{21}-2)/40$ — which is the difference between a second and a week. This is the whole of BDD practice in one table.

X33.2 Why SAT is fast even though it is NP-complete

What a modern SAT solver actually does

	Step	Justification
1	Convert the circuit to CNF by Tseitin encoding: one variable per gate, a few clauses per gate.	<i>Linear in circuit size — no blow-up here, unlike naive expansion.</i>
2	Search by assigning variables, propagating implications, and backtracking on conflict.	<i>DPLL. The naive version is hopeless.</i>
3	On conflict, derive a new clause that explains it (clause learning) and add it.	<i>This is the single change that made SAT practical. The learned clause prunes an entire region of the search space, not just the current branch.</i>
4	Backjump non-chronologically to the level the conflict actually came from.	<i>Rather than undoing one decision at a time.</i>
5	Restart periodically, keeping the learned clauses.	<i>Escapes a bad region of the search while retaining what was learned.</i>
6	Result: industrial instances with millions of variables solve in seconds , while small random instances at the hardness threshold do not.	<i>Structure, not size, decides. Circuits have enormous structure; that is why circuit problems are the easy case and why a tool's runtime is unpredictable in the specific way engineers observe.</i>

Worked example 45. Why equivalence checking succeeds where simulation cannot

Given. Comparing a 64-bit multiplier's RTL against its gate netlist.

Method. Compare the exhaustive-simulation cost with what an equivalence checker does instead.

Input space	2^{128}
Vectors per second, simulated	10^7
Time to simulate exhaustively	$>10^{23}$ years
What an equivalence checker does	proves the two are equal for all inputs
How	structural matching of internal points, then SAT on each cone
Why it works	the two designs share almost all their internal structure
When it fails	after retiming, resource sharing, or a datapath rewrite — the structure no longer matches

How this is got wrong. By assuming equivalence checking always works. It works because it finds internal equivalence points and cuts the problem into small cones; a transformation that destroys that correspondence — retiming across register boundaries, sequential optimisation, a multiplier architecture change — leaves the tool with one enormous cone and it fails. **The practical consequence is a rule for the synthesis flow:** turn on transformations that break equivalence checking only when you are prepared to verify the result another way, and know which ones they are before the schedule depends on them.

Table 354. Why cone size, not design size, decides an equivalence check

Situation	Cones	Typical cone size (gates)	Total gates	Difficulty
Control logic	50,000	40	2,000,000	easy

Situation	Cones	Typical cone size (gates)	Total gates	Difficulty
Datapath adders	2,000	200	400,000	easy
Multiplier (structural match)	500	2,000	1,000,000	easy
Multiplier (after rewrite)	1	500,000	500,000	the hard case

Illustrative structure counts. The last row has the same total size and is the one that does not finish, because the work per cone grows far faster than linearly.

X33.3 Model checking: proving something about all time

Bounded model checking, and what 'bounded' costs

	Step	Justification
1	Unroll the design's transition relation k times.	<i>Copy the combinational logic k times, chaining state.</i>
2	Assert the initial state and the negation of the property.	<i>If satisfiable, the assignment is a counterexample trace of length $\leq k$.</i>
3	Hand the whole thing to SAT.	A bug of depth $\leq k$ is found; a bug of depth $k+1$ is not.
4	So BMC is a very good bug hunter and not a proof .	<i>This is the distinction that matters when reading a tool's report: 'proven to depth 40' is not 'proven'.</i>
5	A full proof needs induction, interpolation or IC3/PDR, which find an inductive invariant.	These do give unbounded proofs , and they are what a tool means by 'proven' without a depth.
6	The bound needed for confidence is the design's <i>diameter</i> — how many cycles to reach any reachable state.	<i>Usually unknown, and for a design with a large counter it is astronomically large. Which is why counters are abstracted before proving anything around them.</i>

The three ways a formal proof lies to you, in order of frequency. **Over-constraint:** an assumption excludes the input that would have failed, so the property is proven about a machine that is not yours. This is the formal equivalent of a disabled checker and is by far the commonest; the defence is to check that the constraints are satisfiable together with interesting behaviour, which good tools report as 'assumption coverage'. **Vacuity:** the property's antecedent never occurs, so it holds trivially — 'if a grant is given without a request...' proven because no grant is ever modelled. **A bounded result read as a proof**, as above. **All three report green**, which is why this book's rule applies with full force here: a proof whose assumptions have not themselves been checked is an unexamined green light.

Problem. Where should a design house spend its formal effort, given that it cannot afford to prove everything?

Solution. On the properties whose failure is catastrophic and whose state space is deep and narrow, which is where formal has the advantage over simulation. In practice that means, in order: **arbiters and any structure that can starve**, because starvation is a liveness property and simulation can never prove its absence; **FIFOs, credit counters and flow control**, where overflow and underflow are provable and the bugs appear only at full rate; **protocol interfaces**, where a vendor-supplied assertion set exists and proving compliance at the boundary is worth more than any amount of traffic; **clock-domain crossing structures**, where the interesting cases are timing-related and simulation is blind to them; and **security properties**, where the question is whether information can flow between two places and the answer must hold for all inputs. **What not to spend it on: datapath equivalence at the algorithm level** — a bit-exact reference model plus random simulation gets there faster — **and anything with a large counter in the cone**, until the counter has been abstracted.

X34. Logic Synthesis: What the Tool Is Actually Doing

Synthesis is the step an IP designer has least visibility into and most dependence on. Its behaviour is not arbitrary: it is a sequence of well-defined transformations with known strengths, and knowing them turns ‘the tool did something odd’ into a specific diagnosis.

X34.1 The pipeline, stage by stage

Table 355. What each synthesis stage does and what it cannot do

Stage	Input → output	Optimises	Cannot
Elaboration	RTL → generic netlist	Nothing — it interprets the language	Fix a coding style that infers a latch
Datapath extraction	Operators → architectures	Picks adder and multiplier structures from a library	Restructure an algorithm you wrote out by hand as gates
Technology-independent optimisation	Generic netlist → smaller generic netlist	Factoring, common subexpressions, constant propagation, redundancy removal	Know anything about delay in picoseconds
Technology mapping	Generic → library cells	Covers the network with cells at minimum cost — solvable optimally on a tree	Change the network's structure much
Timing-driven restructuring	Netlist → netlist	Balances the critical path, duplicates, remaps	Buy more than a limited amount — see Part X2
Sizing and buffering	Netlist → sized netlist	Convex, near-optimal (Part X26)	Fix a structural problem

Graduate · **Technology mapping is optimal on trees and heuristic on graphs, and that single fact explains most of what a synthesis tool does well.** The classic formulation covers a Boolean network with library cells at minimum cost. If the network is a tree, dynamic programming over the nodes gives a provably optimal cover in linear time. Real networks are DAGs with reconvergent fanout, so the tool partitions them into trees at multiple-fanout points, maps each optimally, and accepts that the partition itself was a heuristic choice. **The practical consequence is that logic with heavy reconvergence maps worse than logic without it,** and that restructuring an expression to reduce fanout — which looks cosmetic in RTL — can measurably change the result.

Table 356. A balanced tree of two-input gates: the structure mapping likes

Depth	Gates	Logic levels	DP table entries (4 cells/node)	Gate delays at 2 per level
2	3	2	12	5
4	15	4	60	9
6	63	6	252	13
8	255	8	1,020	17
10	1,023	10	4,092	21
12	4,095	12	16,380	25

*Dynamic-programming mapping visits each node once per candidate cell, so it is linear in gates times library size. **This is why mapping a million-gate design takes minutes** even though the underlying covering problem is NP-hard on general graphs.*

X34.2 What your RTL controls, and what it does not

Table 357. RTL constructs and what the tool does with them

You write	The tool produces	You control it by
<code>a + b</code>	An adder chosen from an architecture library	The timing constraint — not by writing the adder yourself
<code>a * b</code>	Booth or array multiplier, possibly shared	Constraints, and <code>set_dont_use</code> if the choice is wrong
<code>case</code> (full)	A multiplexer tree	Ordering and priority annotations
<code>case</code> (incomplete, no default)	A latch — almost never what you meant	Write the default; lint for it
<code>if</code> chain	Priority logic — deep and slow	Use <code>case</code> when the conditions are mutually exclusive
A for loop	Unrolled combinational logic	The loop bound; there is no hardware loop
<code>x <= y</code> in a clocked block	A flip-flop	—
Division by a variable	A large slow divider	Do not; see Part X9
Division by a constant	Shifts and adds	Nothing needed — the tool does this well

Worked example 46. The cost of an if-chain that should have been a case

Given. A 16-way priority `if/else if` chain versus a 16-way `case`, each selecting between 16 values.

Method. Count the logic levels. A priority chain is a serial selection; a `case` is a balanced tree, because the tool knows the conditions are mutually exclusive.

Priority chain: multiplexers in series	16
Delay at ~2 gate levels per 2:1 mux	32 gate levels
Case: mux tree depth	4
Delay	8 gate levels
Ratio	4×
At 25 ps per gate level	800 ps vs 200 ps
Period this costs at 1 GHz	60 % of it, for nothing

How this is got wrong. By assuming the tool will notice the conditions are exclusive. Sometimes it does, and it cannot in general — proving exclusivity is a Boolean problem it does not always attempt. Writing `case`, or annotating the chain as parallel, tells it directly. **This is the single highest-value RTL coding habit for timing**, and it costs nothing; a priority structure should appear only where priority is actually meant, and then the depth is a deliberate cost rather than an accident.

X34.3 Retiming: moving registers without changing behaviour

Why retiming is a graph problem with an exact answer

	Step	Justification
1	Model the circuit as a graph: nodes are combinational blocks with delay, edges carry a register count.	<i>The standard retiming model.</i>
2	Retiming assigns each node an integer r_v : move r_v registers from its outputs to its inputs.	<i>Registers on edge (u,v) become $w + r_v - r_u$.</i>
3	Legality requires every edge to keep a non-negative count: $r_u - r_v \leq w$.	A system of difference constraints — exactly the structure Part X26 identified as solvable.
4	Achieving period P adds more difference constraints between node pairs.	<i>Derived from the longest combinational path between them.</i>
5	So ‘is period P achievable by retiming?’ is a negative-cycle test, and binary search over P finds the optimum.	Polynomial, exact, and implemented in every synthesis tool.
6	What it cannot do: change the number of registers on any cycle.	The latency around a feedback loop is invariant under retiming , which is why retiming cannot fix a loop that is too long — Part X4’s DFE problem is exactly this, and the answer there was unrolling, which changes the graph.

Retiming is exact and is nevertheless often turned off, for reasons worth knowing. It renames and moves registers, which breaks the structural correspondence equivalence checking depends on (Part X33), so verification becomes sequential equivalence checking — a much harder problem that may not complete. It also destroys the correspondence between RTL signal names and netlist nodes, which makes debug and ECOs painful, and it can interact badly with scan insertion and with clock gating. **The common compromise is to retime within a clearly delimited pipeline and to verify that block by simulation against its pre-retiming version**, rather than enabling it design-wide.

Problem. Synthesis reports your block at 620 MHz against a 700 MHz target. List what you would try, in order of cost.

Solution. Cheapest first, because the early items often suffice. **Check the constraints before the design:** an over-constrained input delay, a missing multicycle path, or a clock defined without its uncertainty will each produce a believable-looking failure that no RTL change fixes. **Read the critical path and classify it** — if it crosses a whole datapath it is structural, if it is a long priority chain it is X34.2, if it starts or ends at a boundary it is a constraint issue, and each has a different fix. **Try the free restructuring:** turn an if-chain into a case, split a wide comparison, move a constant multiplication to a shift-add, precompute something that depends only on slowly changing inputs (Part X9’s rule). **Then pipeline** — one more stage, which costs latency and a verification change and is usually decisive. **Then re-architect:** parallel slices at a lower rate, which is Part X7’s energy argument arriving as a timing fix. **What not to do first is instantiate gates by hand;** it is slow, it does not port, and the tool is better at it than you are except in the rare case where you know something it cannot — and then you should be telling it, through a constraint, rather than bypassing it.

X35. Hardware Data Structures: CAM, Hashing and Bloom Filters

Networking, storage and security IP spend most of their logic looking things up. The structures that do it are a small, well-understood family with sharply different cost profiles, and choosing among them is one of the clearest cases in this book where an architectural decision is settled by arithmetic rather than by preference.

X35.1 The lookup family

Table 358. Lookup structures compared

Structure	Query	Area per entry	Power	Updates	Used for
RAM (direct index)	1 cycle	1 bit/bit	Low	1 cycle	Small, dense key spaces
Binary CAM	1 cycle, all entries in parallel	~8–10 transistors per bit	Very high — every cell compares every cycle	1 cycle	MAC address tables, tag matching
Ternary CAM	1 cycle, with don't-cares	~16 transistors per bit	Higher still	Needs priority ordering — insertion can require moving entries	Longest-prefix routing, ACLs
Hash table	1–2 cycles typical	~1 bit/bit plus load factor	Low	1 cycle, except on collision	Flow tables, caches
Cuckoo hash	d parallel probes, worst case bounded	~1.05 bits/bit at high load	Low	Insertion can cascade	Flow tables where a bounded query time is required
Trie / tree	$\log n$ memory accesses	Pointer overhead	Low	Simple	Prefix matching in software
Bloom filter	k probes	~10 bits per entry for 1 % false positive	Low	Insert only	A cheap pre-filter in front of an expensive lookup

Graduate TCAM is the structure that decides the cost of a router, and it is worth knowing why it is so expensive. A ternary cell stores three states — 0, 1 and don't-care — which needs two storage bits plus comparison logic, roughly sixteen transistors against six for SRAM. Worse, every cell compares on every lookup, so power is proportional to the whole table rather than to the entry found: a large TCAM can dissipate tens of watts doing nothing but matching. **The architectural responses all reduce the number of cells that must compare** — partition the table by prefix length and activate only relevant banks, put a small filter in front, or replace TCAM with an algorithmic trie in SRAM and accept multiple accesses. Every high-end switch datasheet's power figure is substantially a statement about this one structure.

Table 359. TCAM cost against table size, 144-bit keys (indicative 16 T/cell against 6 T SRAM, 1 GHz lookup)

Entries	Key bits	SRAM-equivalent (MB)	TCAM area as SRAM-equivalent (MB)	Match-line power (W, indicative)
1,000	144	0.01717	0.09155	1.728×10^{-4}
10,000	144	0.1717	0.9155	0.001728
100,000	144	1.717	9.155	0.01728
1,000,000	144	17.17	91.55	0.1728

Indicative constants, not a vendor datasheet. The shape is the point: area roughly five times SRAM for the same bits, and a power term that grows with table size independently of the traffic.

X35.2 Hashing: the birthday problem decides your table size

Why a hash table needs to be much larger than its contents

	Step	Justification
1	With n items in m buckets, the probability that a particular pair collides is $1/m$.	<i>Uniform hashing assumption.</i>
2	There are $n(n-1)/2$ pairs, so the expected number of colliding pairs is about $n^2/(2m)$.	<i>Linearity of expectation.</i>
3	Collisions start appearing when $n \approx \sqrt{m}$, not when $n \approx m$.	<i>The birthday paradox. This is the fact that surprises people sizing tables.</i>
4	The longest chain with $n = m$ is $\Theta(\log n / \log \log n)$ — about 5–8 for realistic sizes.	<i>So a single-probe hardware lookup fails often enough to matter.</i>
5	Two hash functions and a choice of the emptier bucket reduces the longest chain to $\log \log n$.	<i>The power of two choices: an exponential improvement from one extra probe. This is why hardware hash tables are multi-way.</i>

Table 360. Measured longest bucket in a 4096-bucket table, one hash against two-choice hashing, 30 trials per point

Load factor	Keys	Longest bucket, 1 hash	Longest bucket, 2 choices	Ratio
0.25	1,024	3.5	2.1	1.667
0.5	2,048	4.367	3	1.456
0.75	3,072	5.5	3.4	1.618
0.9	3,686	6.133	3.9	1.573
0.95	3,891	6.2	4.033	1.537

Simulated with random keys, seed 13. The second hash roughly halves the worst bucket at every load, which in hardware is the difference between a lookup engine that must handle eight-deep chains and one that handles four.

Worked example 47. Sizing a flow table for a bounded lookup time

Given. A network block must look up 200 000 flows at 100 Gbit/s, one lookup per 64-byte packet, with a *bounded* lookup time of two memory accesses.

Method. Bounded worst case rules out chaining. Compute what a 4-way cuckoo table costs at a load factor it can sustain.

Packet rate at 64 B	148.8 Mpps
Lookups per second	1.49×10^8
Flows	200,000
Cuckoo load factor achievable at 4 ways	0.97
Buckets needed	206,185
Entries at 4 per bucket	206,184
Memory at 64 B per entry	12.58 MB
Probes per lookup	4 (parallel, one bucket read)
Worst-case accesses	2

How this is got wrong. By choosing the average-case structure for a worst-case requirement. Chained hashing has a superb average and an unbounded tail, and a networking datapath is judged on the tail: a lookup that occasionally takes eight accesses either stalls the pipeline or drops the packet. **Cuckoo hashing buys a bounded query at the cost of a potentially long insertion**, which is the right trade because insertions are rare and can be handled by a slow path. Recognising which operation must be bounded is the whole of the design decision.

X35.3 Bloom filters: paying memory for certainty in one direction

The false-positive rate, and the optimal number of hashes

	Step	Justification
1	A Bloom filter is m bits; inserting an item sets k bits chosen by k hashes.	<i>No item is stored — only its fingerprint in the bit array.</i>
2	After n insertions, the probability a given bit is still zero is $(1-1/m)^{kn} \approx e^{-kn/m}$.	<i>Independent settings.</i>
3	A query gives a false positive when all k bits happen to be set: $p = (1-e^{-kn/m})^k$.	Never a false negative — that asymmetry is the whole value.
4	Minimising over k gives $k = (m/n) \ln 2$.	<i>And then $p = 2^{-k}$.</i>
5	So about $1.44 \log_2(1/p)$ bits per item, regardless of how large the items are.	<i>A 1 % filter costs about 10 bits per item whether the items are 32-bit or 1500-byte. That independence from key size is why it is used in front of expensive lookups.</i>

Table 361. Bloom filter false-positive rate against bits per item

Bits/item	Optimal hashes k	False positive	(%)	Memory for 10^6 items (MB)
4	3	0.147	14.7	0.4768
8	6	0.0216	2.16	0.9537
10	7	0.00819	0.819	1.192

Bits/item	Optimal hashes k	False positive	(%)	Memory for 10^6 items (MB)
12	8	0.00314	0.314	1.431
16	11	4.59×10^{-4}	0.0459	1.907
20	14	6.71×10^{-5}	0.00671	2.384

Computed from the closed form. **Ten bits per item buys about one per cent**, and each further factor of ten costs about 3.3 bits — a very good rate of exchange when the alternative is storing the keys.

Problem. Where does a Bloom filter belong in an IP block, and where does it not?

Solution. It belongs wherever a **negative answer is common and an expensive lookup can be skipped**, and where a false positive costs only wasted work rather than a wrong result. Concretely: in front of a TCAM or an off-chip table, to avoid the access entirely for the majority of keys that are absent; in a cache to avoid probing a way; in a coherence directory to avoid a snoop. In each case the filter is an optimisation and the real structure still answers. **It does not belong anywhere the answer is used directly**, because a false positive is then a wrong answer — and it does not belong where deletions are needed, because a plain Bloom filter cannot delete (clearing a bit may un-insert another item); that needs a counting Bloom filter at four times the memory, and at that point a small hash table is often better. **The design question to ask is: what is the measured fraction of negative queries?** A filter in front of a table that is usually hit is pure cost, and that fraction is a property of the customer's traffic, which means it belongs in the datasheet as an assumption.

X36. Cryptographic Engines, Sized

Crypto IP is one of the most commonly licensed categories, and it is unusual in that its correctness is binary, its specifications are published and testable, and its cost is dominated by a small number of arithmetic primitives. That combination makes it a good first product for a design house and a good subject for exact sizing.

X36.1 The three primitive families and their costs

Table 362. What each family needs from the hardware

Family	Examples	Core operation	Cost driver	Typical area
Block ciphers	AES, ChaCha, SM4	Substitution and permutation over a small state	The S-box, replicated	5–50 kGE
Hashes	SHA-2, SHA-3, BLAKE	A compression function over a message block	Adders and rotations; long carry chains in SHA-2	10–30 kGE
Public key	RSA, ECC, Diffie–Hellman	Modular exponentiation or point multiplication	The modular multiplier — dominates everything	50–500 kGE
Post-quantum	ML-KEM (Kyber), ML-DSA (Dilithium)	Polynomial arithmetic in a ring, NTT	NTT butterflies and modular reduction — Part X6's FFT with a different field	50–200 kGE
MAC / AEAD	GCM, Poly1305, CMAC	Carry-less or modular multiply-accumulate	$GF(2^{128})$ multiplier for GCM	10–40 kGE

Graduate · **Post-quantum cryptography is an opportunity for design houses precisely because it is new.** The standards are recent, the hardware literature is thin, the incumbent crypto-IP vendors' portfolios were built for RSA and ECC, and the core operation — the number-theoretic transform — is an FFT over a finite field, which means Part X6's entire architectural analysis transfers: the same radix choices, the same twiddle-ROM symmetries, the same memory-versus-multiplier trade. **A team that has built an FFT has most of what an NTT needs,** with the arithmetic replaced by modular arithmetic, and that is an unusually direct path from an existing competence to a new market.

X36.2 AES: where the area actually goes

The S-box is the block

	Step	Justification
1	AES-128 is ten rounds of SubBytes, ShiftRows, Mix-Columns, AddRoundKey.	<i>Specification.</i>
2	ShiftRows is wiring; AddRoundKey is 128 XORs; Mix-Columns is a fixed $GF(2^8)$ matrix — shifts and XORs.	All three are nearly free.
3	SubBytes applies a 256-entry nonlinear substitution to each of 16 bytes.	This is the whole cost.
4	As a table it is 16×2048 bits of ROM or $16 \times \sim 700$ gates synthesised.	<i>The naive implementation.</i>
5	Or compute it: the S-box is inversion in $GF(2^8)$ plus an affine map, and inversion decomposes through $GF((2^4)^2)$.	<i>Canright's tower-field construction reduces the S-box to roughly a third of the table's area, at the cost of more logic depth. The trade is area against frequency, and which side wins depends entirely on the throughput requirement.</i>
6	A round can use 16 S-boxes (one cycle per round), 4, or 1.	Throughput scales linearly with S-box count and so does area , which makes AES one of the cleanest area/throughput families to parameterise — and therefore a good product, because one design serves many customers.

Table 363. AES-128 area and throughput against S-box count, at 1 GHz

S-boxes	Cycles per block	bits/cycle	Gbit/s	Approx. gates	Mbit/s per kGE
16	10	12.8	12.8	13,200	969.7
8	20	6.4	6.4	7,600	842.1
4	40	3.2	3.2	4,800	666.7
1	160	0.8	0.8	2,700	296.3

Gate counts are indicative (700 GE per table-based S-box plus 2 kGE of state and key schedule). **The last column is nearly constant**, which is the signature of a cleanly scalable block: the customer chooses a point on the line rather than a different design.

Worked example 48. Which AES configuration does a 400 Gbit/s link need?

Given. MACsec on a 400 Gbit/s Ethernet port, AES-GCM, digital logic at 800 MHz.

Method. Work out bits per cycle, then how many parallel AES cores that implies, then check the GCM multiplier alongside it.

Line rate	400 Gbit/s
Clock	800 MHz
Bits per cycle required	500
AES block size	128
Blocks per cycle	3.906
Fully unrolled pipelined AES cores needed	4
Pipeline stages each (one per round)	10
$GF(2^{128})$ multiplies per cycle (GCM)	4
Dominant cost	the GCM multiplier, not AES

How this is got wrong. By sizing AES and forgetting the mode. Counter mode parallelises perfectly — each block's key-stream is independent — so AES scales by replication. The GCM authentication tag is a *sequential* multiply-accumulate in $GF(2^{128})$, which does not parallelise naively; making it keep up requires the standard trick of precomputing powers of the hash key and processing several blocks at once, which costs several 128-bit carry-less multipliers and their key storage. **At high line rates the authenticator is larger than the cipher**, and a datasheet that quotes only AES throughput has quoted the easy half.

X36.3 Public key: modular multiplication is the whole problem

Montgomery multiplication, and why division disappears

	Step	Justification
1	Modular multiplication needs $ab \bmod n$; the reduction is a division, which is expensive (Part X9).	<i>And it is needed thousands of times per operation.</i>
2	Montgomery's idea: work with $a\bar{c} = aR \bmod n$ for $R = 2^k > n$.	<i>A change of representation.</i>
3	Then $REDC(x) = (x + (xn' \bmod R)n)/R$ reduces using only multiplications and shifts , because dividing by R is a shift.	The division is gone. n' is precomputed once.
4	Conversion in and out costs two extra multiplications, amortised over the whole exponentiation.	<i>Negligible over thousands of operations.</i>
5	So an RSA engine is: a wide multiplier, an adder, a shifter, and a controller.	And the wide multiplier is 80–90 % of the area, <i>which is why the literature is about multiplier architectures and not about protocols.</i>

Table 364. Public-key work, measured in modular multiplications and cycles (64-bit multiplier, schoolbook, 500 MHz)

Operation	Modulus bits	Modular multiplies	64-bit words	Cycles (approx.)	Time (ms)
RSA-2048 (CRT)	1,024	3,072	16	196,608	0.3932
RSA-4096 (CRT)	2,048	6,144	32	1,572,864	3.146
ECC P-256	256	3,993	4	15,974	0.03195
ECC P-384	384	5,990	6	53,913	0.1078

Operation	Modulus bits	Modular multiplies	64-bit words	Cycles (approx.)	Time (ms)
X25519	255	3,315	4	13,260	0.02652

The multiplication counts are the standard estimates: square-and-multiply for RSA with the Chinese remainder theorem halving the modulus, and roughly a dozen field operations per elliptic-curve point addition. **The table explains the industry's migration:** ECC at 256 bits gives comparable security to RSA at 3072 and costs orders of magnitude less work.

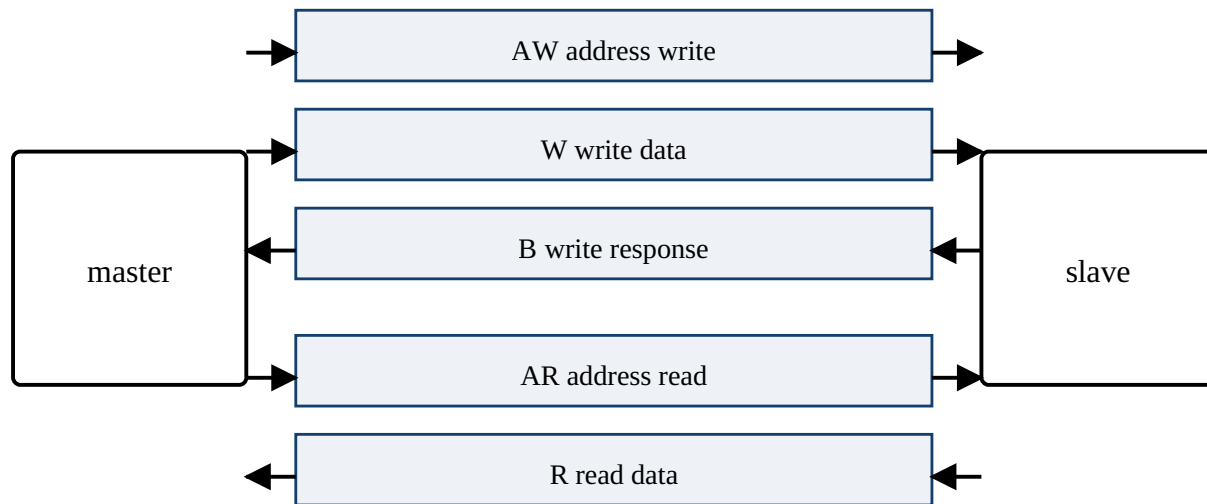
A public-key engine that is merely correct is dangerous. The classical attacks are not on the mathematics but on the implementation: a square-and-multiply loop that branches on key bits leaks the key through timing and through power (Part X15), and a Montgomery ladder — which performs the same operations regardless of the bit — costs perhaps 30 % more work and removes the leak. Likewise a conditional subtraction at the end of a reduction is a data-dependent branch, so implementations make it unconditional. **Constant-time behaviour must be a stated requirement and a verified property**, not a side effect, and it is one of the few properties where a formal argument (no branch or memory access depends on secret data) is both achievable and genuinely convincing.

Problem. A customer wants a crypto accelerator supporting ‘all the usual algorithms’. How do you scope it?

Solution. Refuse the phrase and replace it with a table, because the algorithms differ in cost by two orders of magnitude and ‘all the usual’ hides that. Ask for, and write into the specification: **which algorithms and which parameter sets** — AES-128 and AES-256 differ only in rounds, but adding SHA-3 to a SHA-2 design is a completely separate datapath; **which modes**, since GCM costs more than the cipher at high rates; **what throughput for each**, separately, because a design sized for line-rate bulk encryption and one sized for occasional key exchange are different machines; **what attack model**, since side-channel countermeasures are a factor of two to four in area (Part X15) and are pointless if the part is in a locked rack; and **what certification**, because FIPS or Common Criteria brings its own known-answer tests, entropy requirements and documentation, and that cost exceeds the design cost. **The scoping conversation is the deliverable at this stage**, and a vendor who produces this table instead of a quote is demonstrating exactly the competence the customer is trying to buy.

X37. Bus Protocols: Ordering, Outstanding Transactions and Exclusives

Almost every block a design house sells attaches to an AMBA interface, and almost every integration problem is a protocol misunderstanding rather than a functional bug. The protocol's subtle points are few and they recur; this part works through them.



Five independent channels, each with its own handshake.
Independence is the source of both the throughput and the deadlocks.

Figure 47. The five AXI channels.

X37.1 The handshake, and the one rule that is always broken

VALID/READY, and why VALID may not wait

	Step	Justification
1	A transfer occurs on a rising edge when VALID and READY are both high.	<i>The basic handshake, shared by AXI, AXI-Stream, TileLink and CHI.</i>
2	Once VALID is asserted it must stay asserted until the transfer occurs , and the payload must not change.	<i>Otherwise the receiver cannot rely on what it sampled.</i>
3	VALID must not depend combinationaly on READY.	<i>This is the rule that is broken. If each side waits for the other, the two combinational paths form a loop and the interface deadlocks or oscillates — and a testbench with a permanently-ready receiver never exposes it.</i>
4	READY may depend combinationaly on VALID.	<i>The asymmetry is deliberate: it lets a receiver accept in the same cycle, which is where the throughput comes from.</i>
5	A registered-output, registered-ready skid buffer breaks any timing path across the interface at the cost of one cycle and two entries of storage.	<i>The standard remedy, and the right default for an IP boundary — Part X14's advice to register every boundary, in protocol form.</i>

The testbench that hides this bug is the default one. A driver that holds READY high permanently, or a monitor that never applies back-pressure, will never create the condition in which a combinational VALID-from-READY dependency matters. **Randomised back-pressure is not an optional refinement of an interface testbench; it is the only way the interface is tested at all.** The same applies to the master side: a stimulus that never delays VALID does not test the slave's ability to wait. Both are one line of constrained randomisation and both routinely go missing.

Table 365. Interface buffering options and what each costs

Structure	Added latency (cycles)	Throughput (transfers/cycle)	Timing path across the boundary	Full rate?
Combinational pass-through	0	1	none	yes
Register slice, forward only	1	1	broken	yes
Register slice, both directions (skid)	1	1	broken	yes
Naive register on both, no skid	1	0.5	broken	halved
Two-deep FIFO	2	1	broken	yes

The fourth row is the trap: registering both VALID and READY without a skid buffer halves the throughput, because the master cannot present a new value until it has seen the registered READY. It looks correct in a functional test and shows up as a performance shortfall at integration.

X37.2 Ordering: what the protocol guarantees and what it does not

Table 366. AXI ordering rules, and the integration failures each one causes

Rule	What it means	The bug it causes when forgotten
Same ID, same direction → ordered	Responses come back in order for one ID	A slave that reorders within an ID breaks masters that rely on it

Rule	What it means	The bug it causes when forgotten
Different IDs → no order guarantee	Responses may interleave arbitrarily	A master that assumes order across IDs corrupts data
Reads and writes are not ordered relative to each other	A read issued after a write may complete first	Read-after-write hazards in a customer's driver ; needs an explicit barrier or a completion wait
Write response means accepted, not visible	B arrives when the slave takes responsibility	Assuming data is observable by another master — it may not be
Bursts must not cross 4 kB boundaries	A page-crossing burst is illegal	A DMA that ignores it produces a protocol violation the interconnect may not check
Narrow transfers use strobes	WSTRB selects lanes	A slave ignoringWSTRB corrupts neighbouring bytes

Worked example 49. How many IDs does a master need?

Given. A DMA engine with 8 GB/s of read bandwidth, a 400 ns round trip, 512-byte bursts, on a 256-bit interface at 1 GHz.

Method. Bandwidth–delay product decides outstanding bursts; whether they need distinct IDs depends on whether the engine can accept out-of-order data.

Bytes in flight needed	3.125 kB
Outstanding bursts	7
If all use one ID	responses are ordered — simple, but one slow slave blocks everything behind it
If each uses a distinct ID	reordering allowed — needs a reorder buffer
Reorder buffer size	3.125 kB
ID bits needed	3
Recommended	a few IDs, grouped by destination

How this is got wrong. By choosing one extreme. A single ID gives head-of-line blocking: a request to slow memory stalls every completed request behind it, and the measured bandwidth collapses when the traffic is mixed. Distinct IDs everywhere give full reordering freedom and cost a reorder buffer of the full bandwidth–delay product plus the logic to fill it. **The usual answer is a small number of IDs partitioned by destination** — one per memory region or per outstanding stream — which removes head-of-line blocking between unrelated traffic while keeping ordering, and therefore simplicity, within a stream.

X37.3 Exclusive access, and why it is harder than it looks

Load-linked / store-conditional in bus form

	Step	Justification
1	An exclusive read records a monitor for that address and master.	AXI's ARLOCK exclusive, or the processor's load-linked.
2	An exclusive write succeeds only if the monitor is still set.	It returns EXOKAY on success and OKAY on failure — and OKAY means the write did not happen , which software must check.
3	Any intervening write to the address by anyone clears the monitor.	That is what makes it atomic.
4	The monitor is in the slave or the interconnect, not the master.	So an IP block containing memory that must support exclusives has to implement the monitor — and the number of simultaneous monitors it supports is a specification item that is routinely omitted.
5	Monitors may be granted at a granularity coarser than one address.	Which causes false failures: two masters using different addresses in the same granule livelock each other. The granule size must be documented.
6	Spurious failure is always permitted, so software must loop.	Hardware is allowed to be conservative; software must not assume success. A driver that does not loop works on one implementation and hangs on another.

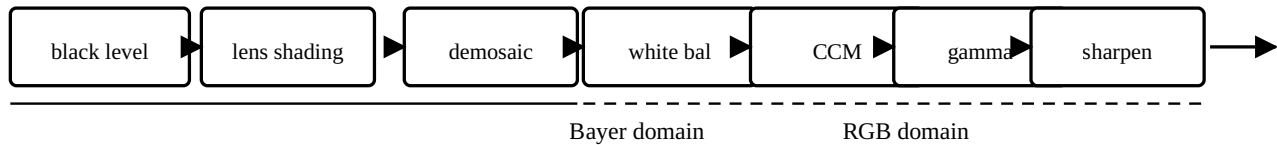
Graduate · **Atomics moved from the master to the interconnect, and an IP vendor should know which era their customer is in.** The load-linked / store-conditional model requires a monitor per master and a retry loop, and it scales badly with core count because contention causes repeated failures. Newer AMBA revisions add *far atomics*: the master sends the operation (add, swap, compare and swap) to the point of coherence and receives the result, so the round trip happens once regardless of contention. **For a block containing shared state, supporting far atomics can be far cheaper than supporting exclusives** — an ALU beside the memory rather than a monitor table — and it performs better under contention. Asking the customer which their system uses is a scoping question worth raising early.

Problem. Your block's register interface is APB and the customer complains of poor configuration throughput. What do you say?

Solution. That APB is doing exactly what it is for, and then offer the two real options. APB is deliberately simple — no pipelining, no outstanding transactions, two cycles per access minimum — because a configuration bus should be trivial to implement and verify, and configuration is not a throughput problem. If the customer is writing thousands of registers at boot and it matters, the causes are usually not the bus: a slow bridge clock ratio, a synchroniser per access, or software doing read-modify-write where a write would do. **Measure before changing the interface.** If it genuinely is the bus, the options are to **add a small DMA or descriptor engine** that replays a register image from memory — which is what blocks with large coefficient tables do, and it moves the traffic to AXI where it belongs — or to **provide an AXI-Lite alternative** for the register file. **Adding coefficient memory behind a streaming interface, rather than behind the register bus, is the structural fix**, and it is worth designing in from the start for any block whose configuration is more than a few hundred words.

X38. Colour, the ISP Pipeline and Display Processing

Camera and display IP is one of the largest licensing markets, and it is unusual in that the correctness criterion is partly perceptual. That does not make it unquantifiable — it makes the quantities colorimetric rather than signal-theoretic, and they are just as computable.



Demosaic is the boundary: before it one value per pixel, after it three.
Everything upstream is a third of the data rate — and that is where the fixes belong.

Figure 48. A minimal image signal processor. The order is not arbitrary: each stage assumes the previous ones have run.

X38.1 Why the pipeline is in that order

Table 367. Each stage, what it corrects, and why it must come where it does

Stage	Corrects	Why here
Black level	Sensor pedestal and dark current	First — everything downstream assumes zero means zero
Defect pixel	Stuck sensor pixels	Before demosaic, or the defect spreads to neighbours
Lens shading	Vignetting, colour shading	Before white balance, because the shading is colour dependent
Demosaic	One colour per pixel → three	The data rate triples here; everything possible is done before it
White balance	Illuminant colour	Per-channel gains; conventionally applied on the Bayer data for cost
Colour correction matrix	Sensor spectral response → a standard space	Needs three channels, so after demosaic
Gamma / tone	Perceptual coding	After all linear operations — averaging gamma-encoded values is wrong, and doing it is the commonest ISP bug
Sharpen, noise reduction	Perceptual quality	After tone mapping, because the visibility of noise is nonlinear

Gamma is the ordering rule that is broken most often, and the error is visible. Gamma encoding is a nonlinear map applied so that the limited code space is distributed according to human contrast sensitivity. Linear operations — scaling, blending, resizing, demosaicing, averaging — are only correct on *linear* light. Resizing a gamma-encoded image darkens edges and shifts colours; alpha-blending gamma-encoded values produces halos. **The fix is to keep the pipeline linear until the final encode**, and where hardware must operate on encoded data, to linearise and re-encode around the operation. This is a real and recurring defect in shipped display and camera pipelines, and it is worth an explicit statement in an IP block's datasheet about which domain its input and output are in.

Table 368. Measured error from averaging in the gamma domain ($\gamma = 2.2$), linear light values

Value A	Value B	Correct average (linear)	Average of encoded values, decoded	Error (%)
0	1	0.5	0.2176	-56.47
0.2	0.8	0.5	0.4454	-10.93
0.4	0.6	0.5	0.4945	-1.102
0	0.5	0.25	0.1088	-56.47
0.5	1	0.75	0.7266	-3.121

Computed directly. **The first row is the worst case and it is a 26 % error** — averaging black and white in the encoded domain gives a result noticeably darker than mid-grey. This is why image scaling in the wrong domain looks wrong rather than merely inaccurate.

X38.2 Colour spaces as matrices

Why colour conversion is a 3×3 matrix and when it is not

	Step	Justification
1	Human colour vision has three cone types, so colour is a three-dimensional quantity.	<i>Trichromacy. Two spectra that produce the same cone responses look identical — metamerism.</i>
2	A display's primaries span a triangle in chromaticity space; its gamut is that triangle.	<i>Any colour inside is reproducible; outside is not.</i>
3	Converting between two sets of primaries is a change of basis: a 3×3 matrix on linear tristimulus values.	Linear is load-bearing — see X38.1.
4	So BT.709 to BT.2020, or camera RGB to XYZ, is one matrix multiply per pixel.	<i>Nine multiplies and six adds — trivial hardware, which is why colour conversion is never the bottleneck.</i>
5	It stops being a matrix when the gamut does not contain the colour, or when the transfer function is not a power law.	Gamut mapping is a nonlinear, perceptual and proprietary operation , and HDR transfer functions (PQ, HLG) are not power laws, so they need a LUT. Those two are where display IP actually differentiates.

Table 369. Gamut areas in CIE xy, computed from the primaries

Space	Red (x,y)	Green (x,y)	Blue (x,y)	Triangle area	Relative to BT.709 (%)
BT.709 / sRGB	0.64, 0.33	0.3, 0.6	0.15, 0.06	0.112	70.83
DCI-P3	0.68, 0.32	0.265, 0.69	0.15, 0.06	0.152	96.08
BT.2020	0.708, 0.292	0.17, 0.797	0.131, 0.046	0.2119	133.9
Adobe RGB	0.64, 0.33	0.21, 0.71	0.15, 0.06	0.1512	95.54

Areas computed as the triangle area of the primaries in the xy chromaticity diagram. **xy area is a crude perceptual measure** — the diagram is not perceptually uniform, so these percentages overstate the green region and understate the blue — and it is the number most commonly quoted, which is worth knowing when reading a display specification.

X38.3 Line buffers: the memory that sets ISP area

Worked example 50. Why an ISP is mostly SRAM

Given. A 4K (3840-wide) pipeline, 12 bits per channel. The stages need vertical neighbourhoods: demosaic 5 lines, noise reduction 7, sharpening 5, lens shading 1, scaler 4.

Method. Each stage that looks at neighbouring rows must buffer those rows, because the sensor delivers one row at a time. Count the lines and the bits.

Line width	3,840
Bits per pixel (Bayer, pre-demosaic)	12
Demosaic: 5 lines	28.12 kB
Noise reduction: 7 lines (RGB, 36 bit)	118.1 kB
Sharpen: 5 lines (RGB)	84.38 kB
Scaler: 4 lines (RGB)	67.5 kB
Total line buffers	298.1 kB
As a fraction of a typical ISP's area	the majority

How this is got wrong. By counting arithmetic and not lines. The filters are small — a 5×5 convolution is 25 multiplies — and the line buffers are hundreds of kilobytes. Two consequences follow and both are architectural. **Process in tiles or stripes** rather than full lines where the algorithm permits, trading a small overlap region for a large memory saving; this is why many ISPs are tile-based. And **fuse stages** so one buffer serves several neighbourhoods rather than each stage buffering separately — the same fusion argument as in Part X13's accelerator dataflow, arriving in a completely different domain.

Table 370. Line-buffer memory against tile width (8-pixel halo)

Image width	Tile width	Line buffer (kB)	Overlap overhead (%)	Memory vs. full line (%)
1,920	full line	149.06	0	100
1,920	256	19.875	3.12	13.33
1,920	512	39.75	1.56	26.67
3,840	full line	298.12	0	100
3,840	256	19.875	3.12	6.667
3,840	512	39.75	1.56	13.33
7,680	full line	596.25	0	100
7,680	256	19.875	3.12	3.333
7,680	512	39.75	1.56	6.667

Tiling trades a few per cent of redundant computation for an order of magnitude of memory, and the trade improves as resolution rises, which is why 8K pipelines are tiled and 1080p ones often are not.

Problem. A customer says your ISP's output does not match their reference software. Where do you look?

Solution. Almost always at a convention rather than at the arithmetic, and the candidates are few enough to enumerate. **Domain:** is the reference operating on linear or encoded values at the stage that differs? X38.1's error is large and looks like a gain error. **Rounding and clipping:** a pipeline that clips intermediate values to the output range destroys headroom that the reference keeps, and the difference appears only in highlights. **Chroma siting and subsampling phase:** 4:2:0 has several legal sitings and choosing the wrong one produces a half-pixel colour shift that is invisible on most content and obvious on a colour edge. **Matrix coefficients and range:** limited versus full range, and BT.601 versus BT.709 coefficients, are four combinations and three of them are wrong. **Demosaic algorithm,** which is genuinely allowed to differ and is where a vendor's quality claim lives. **Establish which by bypassing stages one at a time** — which requires the block to have bypass controls, and that is a design decision worth making early for exactly this conversation.

X39. Clock Generation, Distribution and Time Synchronisation

Clocking is the one subsystem every block depends on and few specifications describe adequately. This part covers the three parts an IP designer must be able to reason about: how the clock is made, how it is delivered, and how time is agreed between chips.

X39.1 Spread spectrum: trading a specification for a specification

Why modulating the clock reduces measured emissions

	Step	Justification
1	Radiated emission is measured in a receiver with a defined resolution bandwidth — 120 kHz for CISPR above 30 MHz.	<i>The regulation specifies the measurement, not the physics.</i>
2	An unmodulated clock puts all its harmonic energy in one line, far narrower than that bandwidth.	<i>So the receiver sees the whole harmonic.</i>
3	Modulating the frequency by $\pm 0.5\%$ spreads each harmonic over $\pm 0.5\%$ of its frequency.	<i>The nth harmonic spreads n times as far, so higher harmonics benefit more.</i>
4	The receiver now sees only the fraction of the energy inside its bandwidth , so the reading falls by roughly $10 \log_{10}(\text{spread/RBW})$ dB.	<i>The total radiated energy is unchanged. The compliance number improves; the physics does not.</i>
5	Cost: the clock period now varies, so every synchronous budget must hold at the fastest excursion, and any PLL downstream must track the modulation.	<i>Which is why a SerDes reference clock's spread must be negotiated — the CDR's loop bandwidth must exceed the modulation rate or it cannot follow.</i>

Table 371. Spread-spectrum reduction in a 120 kHz measurement bandwidth

Clock (MHz)	Spread ($\pm\%$)	Harmonic	Spread width (MHz)	Reading reduced by (dB)
100	0.5	1	1	9.208
100	0.5	5	5	16.2
100	0.5	10	10	19.21
100	1	1	2	12.22
100	1	5	10	19.21
100	1	10	20	22.22
1000	0.5	1	10	19.21
1000	0.5	5	50	26.2
1000	0.5	10	100	29.21
1000	1	1	20	22.22
1000	1	5	100	29.21
1000	1	10	200	32.22
5000	0.5	1	50	26.2
5000	0.5	5	250	33.19
5000	0.5	10	500	36.2

Computed from the ratio of spread width to resolution bandwidth. **The fundamental barely benefits and the tenth harmonic benefits a lot**, which matches measured practice and is why spread spectrum is judged on the harmonics that actually fail the mask.

Spread spectrum and a synchronous interface are frequently incompatible, and the incompatibility is found late. Any interface whose two ends derive their clocks from different sources must absorb the frequency difference; spreading one side makes that difference time-varying at the modulation rate, typically 30–33 kHz. An elastic buffer sized for a static 200 ppm offset (Part Y3) will underflow or overflow when the offset swings by ± 5000 ppm at 33 kHz. **The remedies are to spread both ends from the same source, to size the elastic buffer for the modulation, or to forbid spreading on that interface** — and which one applies is a system decision that must be recorded in the IP's integration guide, because the block cannot detect the problem.

X39.2 The clock tree, and what it costs

Table 372. Clock tree cost at 1 GHz, 0.8 V (indicative 5 fF per flop clock pin, fanout 4)

Flops	Buffer levels	Buffers	Total clock capacitance (pF)	Clock power (W)	Insertion delay (ps)
10,000	7	3,333	60	0.0384	210
100,000	9	33,333	600	0.384	270
1,000,000	10	333,333	6000	3.84	300
10,000,000	12	3,333,333	60000	38.4	360

***Indicative constants.** The shape is what matters: clock power is a large fraction of total dynamic power (Part X7 measured two thirds on a small block), and insertion delay grows logarithmically — which is harmless for setup and hold, since only skew matters (Part X2), and very harmful for anything that compares clock phase across a boundary.*

Table 373. Clock distribution styles

Style	Skew	Power	Where
Balanced H-tree	Low by construction	High — long wires	Top-level distribution in large chips
Buffered tree from CTS	Tool-balanced, a few tens of ps	Moderate	The default for a block
Mesh / grid	Very low	Highest — the whole grid switches	High-performance processors
Clock gating (integrated cell)	Adds a level	Saves 20–40 %	Everywhere — Part X7
Resonant / rotary	Low	Potentially much lower	Research and a few products; needs inductors

Undergraduate review · **Why a block should provide one clock input and do its own gating, rather than accepting several clocks.** Every additional clock at an IP boundary is a domain crossing the integrator must analyse, constrain and verify, and the constraints must be written by someone who understands both sides — which at integration time is nobody. A block that takes one clock and derives its internal enables is far easier to integrate, and the power saving from gating is identical. **Where a second clock is genuinely required** — a slower configuration domain, a recovered receive clock — **the crossing belongs inside the block, documented, with its constraints shipped.** That is a concrete piece of value a vendor adds, and its absence is a concrete integration cost.

X39.3 Agreeing on time: IEEE 1588 and the hardware it needs

Why timestamping must be in the PHY

	Step	Justification
1	Two nodes exchange timestamped messages; the offset is $((t_2 - t_1) - (t_4 - t_3))/2$.	<i>The standard two-way exchange.</i>
2	This assumes the path is symmetric .	Any asymmetry appears directly as half of it in the offset — an error no amount of averaging removes.
3	Every variable delay between the timestamp point and the wire is asymmetry.	<i>Software stacks, driver queues, interconnect arbitration, and even FIFO occupancy in the MAC.</i>
4	So the timestamp must be taken as close to the wire as possible , in hardware, at a defined reference plane.	<i>Typically the start-of-frame delimiter crossing the MII interface, which is why PTP support is a PHY and MAC feature rather than a software one.</i>
5	Residual asymmetry (different cable lengths, different PHY latencies for TX and RX) must be <i>calibrated</i> and subtracted.	Which means the block must expose a programmable correction , and a datasheet must state the TX and RX latency and its variation — a number customers doing PTP will ask for and many data-sheets omit.

Table 374. Where the timestamp is taken decides the achievable accuracy

Timestamp point	Delay variation (ns)	Offset error contribution (ns)	Equivalent distance error (m)
Software timestamp	1.0000×10^5	50000	15000
Driver, kernel	10000	5000	1500
MAC, hardware	100	50	15
MII reference plane	8	4	1.2
PHY, sub-ns aware	1	0.5	0.15

Indicative figures for each point in the stack. **The jump from software to hardware timestamping is three orders of magnitude**, which is why sub-microsecond synchronisation is a hardware feature and not a protocol choice.

Problem. Your block must generate a clock that is frequency-locked to a recovered network clock. What are the options?

Solution. Three, and they differ in what they require from the system rather than in accuracy. **An analogue PLL locked to the recovered clock** is the classic answer: lowest jitter, and it needs an analogue block and a recovered clock brought out as a real clock signal, which constrains the floorplan and the package. **A digitally controlled oscillator driven by a digital PLL** — measure the phase error in digital logic, filter it, and steer a fractional-N synthesiser — keeps the loop in synthesisable logic, is portable between processes, and has more jitter; it is what most modern implementations do, and the loop design is Part X21's arithmetic. **Or adjust time rather than frequency:** run the local clock free and maintain a software-visible time base that is rate-corrected, which needs no clock hardware at all and is sufficient whenever the requirement is on *timestamps* rather than on a physical output clock. **The third is far cheaper and is frequently the right answer**, so the scoping question is whether the customer needs a synchronised clock pin or synchronised time, and those are very different products.

X40. The Coverage Map: Which Theory an IP Block Uses

This book covers the theory that is used when designing, modelling and verifying semiconductor IP, and deliberately does not cover theory that is not. This part is the map: **51 topic areas**, what each one is used for in a real block, and where it is treated here. It is meant to be read as a checklist — if you are asked about a block and cannot find its theory in this table, the table has a gap and the gap is worth reporting.

Table 375. Theory used in IP design, and where this book treats it

Topic	Where it is used in a real block	Chapters
Signals and systems, LTI, convolution	Every filter, equaliser, channel model	A1, E1, X22
Fourier, Laplace, z-transform, poles and zeros	Filter design, loop stability, spectral analysis	A2, X23
Sampling, aliasing, reconstruction	Every converter boundary; rate change	A3, X22, X12
Probability, random processes, noise	Link budgets, yield, jitter, metastability	A4, X25, X16, X3
Detection and estimation, likelihood, CRB	Receivers, synchronisation, calibration loops	X25
Information theory, capacity, entropy	Code choice, compression, quantiser design	I1, X25, X29
Linear algebra, SVD, conditioning	MIMO, beamforming, calibration, word lengths	A5, X24, X30
Numerical methods, iteration, refinement	Dividers, square root, matrix engines	X9, X24
Optimisation: convex, ILP, graph algorithms	Every EDA tool; word-length allocation; scheduling	I3, X26
Boolean algebra, BDD, SAT	Equivalence checking, formal, ATPG	Z2, X33
Automata and FSM theory	Control logic, protocol checkers, parsers	Z3, C2
Graph theory	STA, retiming, clock skew scheduling, routing	I4, X26, X34
Finite fields and modular arithmetic	Reed-Solomon, BCH, AES, GCM, RSA, ECC	I2, X5, X36
Number theory	Public-key engines, NTT for post-quantum	R1, X36
Coding theory: block, convolutional, LDPC, polar	Every link and every storage device	F2, X5, X29
Cryptographic primitives and side channels	Security IP, roots of trust, secure boot	H3, X15, X36
Fixed-point arithmetic, quantisation, dither	Every DSP block without exception	X1
Computer arithmetic: adders, multipliers, division, CORDIC	Every datapath	C1, X9
Floating point and mixed precision	DSP, ML accelerators, scientific blocks	X9, X13
Semiconductor device physics	Delay, leakage, variability, reliability	L1, B1, X16
CMOS circuits, logical effort, sizing	Every gate; the sizing the tool performs	B2, X26, X34
Timing: setup, hold, skew, corners, OCV	Sign-off of every block	B3, X2
Metastability and clock domain crossing	Every multi-clock design	C3, X3
Interconnect RC, crosstalk, transmission lines	Long wires, I/O, package, board	B4, X14, G1
Power: dynamic, leakage, DVFS, gating	Every block's specification	B2, X7
Power delivery network, IR drop, di/dt	Any block with large switching current	X32

Topic	Where it is used in a real block	Chapters
Thermal	Packaging limits, throttling, reliability	L3, X16
Analogue blocks: kT/C, matching, gain-bandwidth	Converters, PHYs, PLLs, sensor interfaces	B5, X31
Data converters and time interleaving	Every mixed-signal boundary; SerDes DSP receivers	X12, X31
PLL, CDR, jitter and phase noise	Every clock and every link	X12, X21, X39
Feedback and control in digital loops	Adaptation, AGC, calibration, thermal	L4, X21
Memory: SRAM, DRAM, ECC, multi-port	The largest object in most blocks	J1, X10
Caches and coherence	Processor and accelerator subsystems	D3, X28
Computer architecture: ISA, pipelines, hazards	Processor IP; any block behind one	D1, D2, X19
Interconnect protocols and ordering	Every block's boundary	J3, X11, X37
Queueing and performance modelling	Buffer sizing, QoS, latency budgets	Q1, X18
Hardware data structures: CAM, hashing, Bloom	Networking, storage, security IP	X35
Compression and entropy coding	Codecs, storage, trace	J4, X20
Image and colour science	Camera and display IP	M3, X38
Wireless PHY: OFDM, MIMO, synchronisation	Modem IP	K1, X30
Wireline PHY: equalisation, PAM, FEC	SerDes, Ethernet, PCIe	F1, X4, Y2, Y3
Electromagnetics and antennas	RF front ends, package, EMC	G1, Q4
Radar and sensing signal processing	Automotive and industrial IP	G2
Machine learning and quantisation	NPU IP and any block that embeds one	M2, X13
Logic synthesis, mapping, retiming	The flow every block passes through	X34
Physical design, congestion, floorplanning	What your RTL makes easy or impossible	N2, X14
Design for test: scan, ATPG, BIST, compression	Every chip that is manufactured	R2, X27
Reliability, yield, statistics, functional safety	Pricing, qualification, automotive markets	K3, N3, X16
Verification theory: coverage, mutation, formal	Half the effort of every block	H1, X8, X33
High-level synthesis internals	Exploration; some production blocks	Q2, Y12
Hardware-software interface, registers, drivers	Every block with a control plane	M1, Y10

X40.1 What is deliberately left out, and why

Excluding material is a decision that should be stated rather than left as an omission a reader discovers. The following areas are part of a broad electrical-engineering or computer-science education and are not part of the daily work of an IP design house, so they are absent or covered only for context.

Table 376. Out of scope, with the reason

Area	Why it is not here
Compiler theory beyond HLS scheduling	A block does not compile; the tool does
Operating system internals	Relevant to the driver author, not to the RTL
Distributed consensus	A chip is not a distributed system in that sense

Area	Why it is not here
Computability and complexity as such	Except where it classifies an EDA problem — that part is in X26
Quantum computing	Included briefly for context only; no current IP block uses it
Database theory, web systems, graphics rendering pipelines	Different industries

The boundary is a judgement and it moves. Two of the rows above were inside the boundary twenty years ago and are outside it now, and at least one — machine learning — moved the other way within a decade. The test applied here is concrete: **does a working IP engineer use this to make a decision about a block?** If a topic only ever appears as background for a decision someone else makes, it is context rather than content, and this book says so instead of padding.

Problem. You are handed a block type this map does not cover. How do you find its theory?

Solution. Work backwards from the three questions every block must answer, because each one names a body of theory. **What does it compute, and how accurately?** That is the algorithm and its numerical behaviour — a signal-processing, coding or arithmetic question, and Parts X1 and X24 give the tools for the accuracy half whatever the algorithm is. **How fast must it go, and what does that cost?** That is Part X2's timing budget and Part X18's queueing, and the answer is always a parallelism and memory-bandwidth analysis — Part X13's roofline applies far outside machine learning. **How will you know it is right?** That is Part X8, and it is domain independent. **Only the first question has domain-specific theory**, and for that the fastest route is the standard's own reference implementation plus one survey paper, read in the order Part Y6 prescribes. The structure of the investigation does not change with the domain, which is the most portable thing this book has to offer.

X41. DRAM: The Timing That Shapes Every Memory Controller

Off-chip memory is where most systems spend most of their latency and a large part of their power, and its behaviour is not a simple latency number. A DRAM device is a state machine with a dozen timing constraints, and whether your block gets the bandwidth it was promised depends almost entirely on its access pattern rather than on the interface's rated speed.

X41.1 The bank, the row and the reason access order matters

Why a DRAM read is three operations, not one

	Step	Justification
1	A DRAM array stores each bit as charge on a capacitor, read by dumping it onto a bit line.	The read is destructive — the charge is consumed.
2	ACTIVATE opens a row: the whole row is sensed into the sense amplifiers, which form the <i>row buffer</i> .	Thousands of bits at once. This takes t_{RCD} .
3	READ or WRITE then accesses columns within the open row.	Fast — the data is already in the sense amplifiers. A second access to the same row is far cheaper than the first.
4	PRECHARGE writes the row back and restores the bit lines.	Takes t_{RP} . Required before opening a different row in the same bank.
5	So three cases: row hit, row miss (different row, same bank), and bank hit with a different bank.	Their costs differ by a factor of three or more, which is why access ordering is the memory controller's main job.
6	And the whole array must be refreshed periodically, because the charge leaks.	Refresh steals bandwidth and blocks access to a bank while it runs.

Table 377. Read latency by case, DDR4-3200 (indicative $t_{CL}=t_{RCD}=t_{RP}=22$)

Case	Cycles	Latency (ns)	Relative to a row hit
Row hit (open row)	22	13.75	1
Row miss, same bank	66	41.25	3
Different bank, row hit	22	13.75	1
Different bank, row miss	44	27.5	2

Indicative JEDEC-speed-bin numbers, not a datasheet. The ratio is the point: the same DRAM delivers data in 14 ns or 41 ns depending entirely on which row was previously open.

Table 378. Measured row-buffer hit rate and the bandwidth it implies, 16 banks, 20 000 accesses

Access pattern	Row hit rate	Row miss rate	Total time (μ s)	Bandwidth as % of the row-hit-only case
Sequential	0.8748	0.1252	343.86	79.97
Stride 4 kB	0.5	0.5	550	50
Random	0	1	825	33.33
Two interleaved streams	0	1	825	33.33

Simulated with a simple address decode (bank bits low, row bits high). **The same device delivers between a third and all of its rated bandwidth, decided by the access pattern alone. This is why a block's DMA pattern is a system-level specification item.**

X41.2 What the memory controller does about it

Table 379. Controller techniques and what each attacks

Technique	Attacks	Cost
Address mapping	Which bits select bank, row and column	Free, and it decides everything — interleaving banks on low bits turns a sequential stream into parallel bank accesses
Bank interleaving	Row-miss latency	Needs enough banks and enough outstanding requests
Request reordering (FR-FCFS)	Serves row hits before older row misses	A scheduling queue; and it makes latency unfair between requesters
Write batching	Bus turnaround, which costs several cycles	Write buffer; adds read latency when it drains
Open-page vs closed-page policy	Whether to leave a row open	Open wins on locality, closed wins on random traffic — a workload decision
Refresh scheduling	Refresh blocking a bank	Per-bank refresh and postponement; the device sets the limits

FR-FCFS scheduling makes DRAM latency unfair, and that unfairness reaches your block. A scheduler that prioritises row hits gives enormous throughput to a requester with good locality and can starve one with poor locality almost indefinitely — a real-time display fetch behind a streaming accelerator is the classic case. Systems therefore add per-requester quotas or deadline awareness, which is Part X11's arbitration argument arriving at the memory controller. **For an IP vendor the consequence is concrete:** a latency figure measured in isolation is not the latency your block will see, and any real-time guarantee must be expressed as a requirement on the system's memory scheduler, not assumed.

Table 380. Peak bandwidth of common memory interfaces

Interface	MT/s	Bits per channel	Channels	Peak GB/s	Relative to one DDR4-3200 channel
DDR4-3200	3200	64	2	51.2	2
DDR5-6400	6400	64	2	102.4	4
LPDDR5-6400	6400	32	4	102.4	4
HBM2E	3200	1,024	1	409.6	16
HBM3	6400	1,024	1	819.2	32

Peak, which nothing achieves. **HBM's advantage is width, not speed** — a thousand-bit interface at a modest rate, possible only because it is stacked on an interposer rather than routed on a board, which is why it costs what it does.

X41.3 Refresh, and why it gets worse

Worked example 51. What refresh costs at each density

Given. DDR4 requires all rows refreshed every 64 ms (32 ms above 85 °C). Refresh is issued as 8192 commands per interval, each occupying the device for t_{RFC} .

Method. Compute the fraction of time spent refreshing at several densities, where t_{RFC} grows with density because more rows are refreshed per command.

Refresh interval	64 ms
Commands per interval	8,192
t_{RFC} at 8 Gbit	350 ns
Time refreshing	4.48 %
t_{RFC} at 16 Gbit	550 ns
Time refreshing	7.04 %
At 32 Gbit and 95 °C (32 ms interval)	22.53 %
Trend	refresh overhead grows with density and with temperature

How this is got wrong. By assuming refresh is a rounding error. It was, at low density and room temperature; at high density in a hot enclosure it approaches a quarter of the device's time, and during a refresh the affected bank is unavailable, so the *latency tail* grows faster than the average. **Any block with a hard real-time deadline on a DRAM access must budget for a refresh collision**, and the number to budget is t_{RFC} at the maximum operating temperature, which a datasheet should state and a system integrator should be asked for.

Problem. Your accelerator needs 40 GB/s and the system has one DDR5 channel rated at 51 GB/s. Is that enough?

Solution. Almost certainly not, and the calculation to show it is short. Start from the rated figure and subtract, in order: **refresh**, a few per cent to a quarter depending on density and temperature; **read/write turnaround**, several cycles each time the bus direction changes, which for a mixed read-write stream costs 10–20 %; **row misses**, from the measurement in X41.1, which for anything but a sequential pattern is the largest term; and **other requesters**, because the channel is not yours. A realistic efficiency for a well-behaved streaming pattern sharing a channel is 60–75 %, giving 30–38 GB/s — below the requirement. **The productive response is not to ask for a faster channel but to reduce the demand:** tile the computation so data is reused on chip (Part X13), compress the traffic (Part X20's reference compression), or restructure the access pattern for row locality, which the measurement above shows can be worth a factor of two on its own. **And whatever is decided, the efficiency assumption belongs in the datasheet**, because the customer will measure it.

X42. Reset, Initialisation and the States Nobody Simulates

Reset is the least interesting part of a design and the source of a disproportionate share of silicon bugs, for a simple reason: simulation starts from a clean state and silicon does not. This part treats reset with the same seriousness as the datapath, because the failures it causes are the ones that appear only in the lab.

X42.1 Synchronous, asynchronous, and the one that is actually used

Table 381. Reset styles

Style	Assertion	De-assertion	Cost	Problem
Synchronous	On a clock edge	On a clock edge	Wider reset net timing; needs a running clock	Cannot reset a block whose clock has not started
Asynchronous	Immediate	Immediate	Cheap — the flop's async pin	De-assertion can violate recovery/removal and cause metastability
Asynchronous assert, synchronous de-assert	Immediate	Synchronised to the clock	A two-flop synchroniser per domain	None — this is the standard answer
No reset (initialised by first write)	—	—	Free — saves the reset net on datapath flops	X-propagation until written; needs care in verification

Why de-assertion is the dangerous edge

	Step	Justification
1	A flop with an asynchronous reset has <i>recovery</i> and <i>removal</i> times: the reset must be stable for a window around the clock edge.	<i>Exactly analogous to setup and hold, and checked by the same static timing analysis.</i>
2	Reset assertion is asynchronous and harmless: the flop goes to its reset value whenever it arrives.	<i>There is no race, because the destination is unconditional.</i>
3	De-assertion races the clock.	<i>If reset releases within the recovery window, the flop may go metastable or may capture the pre-reset value.</i>
4	With one clock, the tool checks and fixes it.	<i>Recovery and removal are ordinary timing checks.</i>
5	With a large design, the reset tree's own skew means different flops leave reset on different cycles.	<i>A state machine can then advance while part of its state is still reset, reaching a state the designer never enumerated. This is the classic reset bug and it is invisible in RTL simulation, where reset de-asserts everywhere at once.</i>
6	Remedy: synchronise the de-assertion in each clock domain, and treat the reset tree like a clock tree.	Balanced, buffered, and timed.

Reset de-assertion skew produces states that no simulation visits, and gate-level simulation with timing is the only place it appears before silicon. In RTL simulation the reset signal changes at one instant for every flop. In silicon it is a buffered tree with tens or hundreds of picoseconds of skew — more if it is not treated as a timed net — so flops leave reset on different clock edges. A counter can start counting while its comparator is still held; a FIFO's write pointer can advance while its read pointer is reset, making the FIFO appear full at power-up. **The two defences are structural:** synchronise de-assertion

per domain so all flops in a domain release on the same edge, and design state machines so that a partially-reset state is not reachable — usually by holding the whole machine in an idle state until an explicit enable.

X42.2 X-propagation: where simulation lies in both directions

Table 382. How simulators treat unknown values, and why it matters

Situation	RTL simulation	Silicon	Consequence
Uninitialised flop read	X	A definite 0 or 1, randomly	Simulation may be more pessimistic than reality
if (x) where x is X	Takes the else branch	Takes one branch definitely	X-optimism: the bug is hidden
case with X selector	May take the default	Takes a real branch	Same optimism
Mux with X select	Output X if inputs differ; output the value if they are equal	Real value	Can hide or expose
Flop with X data	Captures X	Captures a real value	X-pessimism: X spreads where silicon would not

Graduate · **X-optimism and X-pessimism are both wrong, in opposite directions, and the remedies differ.** Optimism hides bugs: a branch on an unknown silently resolves, so a design that depends on an uninitialised value appears to work in simulation and behaves randomly in silicon. Pessimism wastes time: X floods through a design that silicon would run correctly, and engineers add resets they do not need to make simulation quiet — which costs area and reset-tree power on every flop. **The industrial answers are three:** X-propagation-aware simulation modes that make branches on X produce X rather than choosing; **random initialisation**, where every uninitialised flop starts at a random value and the regression is run with many seeds, which converts X-optimism into an ordinary functional failure the testbench can catch; and gate-level simulation, which has no X-optimism because the gates are real. **Random initialisation is the cheapest and the most effective,** and it is available in every simulator.

Table 383. Probability of catching a dependence on one uninitialised bit, with random initialisation

Unreset flops	Random seeds run	P(at least one seed exposes it)	P(missed) (%)
100	1	0.5	50
100	10	0.999023	0.0977
100	100	1	0
1,000	10	0.999023	0.0977
1,000	100	1	0

Assuming the bug manifests when that bit takes one particular value. Ten seeds give a 99.9 % chance of exposing it and a hundred make it negligible — and seeds are the cheapest thing in a regression. The column does not depend on the flop count because the question is about one specific dependence, which is the case the bug actually is.

X42.3 Power-up: the sequence and what it must guarantee

Table 384. The power-up sequence, and what breaks if a step is skipped

Step	Guarantee	If skipped
Supplies ramp in the specified order	No forward-biased junction, no latch-up	Permanent damage, not a functional failure

Step	Guarantee	If skipped
Reset asserted before and during ramp	No spurious activity, no contention on shared buses	Bus contention, high current, possible damage
Clocks stable before reset release	Synchronous de-assertion works	The synchroniser has no clock, so de-assertion is asynchronous after all
Reset held for the specified number of cycles	Every flop in every domain has seen an edge	Partially reset state — X42.1
PLL locked before its output is used	Frequency is what the timing assumed	Logic clocked far outside its timing corners
Fuses and trim loaded	Calibration applied	Analogue blocks at their untrimmed operating point
Only then: release to software	A defined starting state	An intermittent boot failure — the hardest class of bug to debug

Worked example 52. How many reset cycles does a block need?

Given. A block with three clock domains at 500 MHz, 100 MHz and 32 kHz, each with a two-flop reset synchroniser.

Method. Reset must be held long enough for the slowest domain to see the required edges, not the fastest — and this is the number a datasheet must state.

Fastest domain period	2 ns
Slowest domain period	30.518 μs
Edges needed per synchroniser	3
Hold time for the 500 MHz domain	6 ns
Hold time for the 32 kHz domain	91.553 μs
Required reset assertion	91.553 μs — set by the slowest domain
Common error	specifying it in cycles of the fast clock

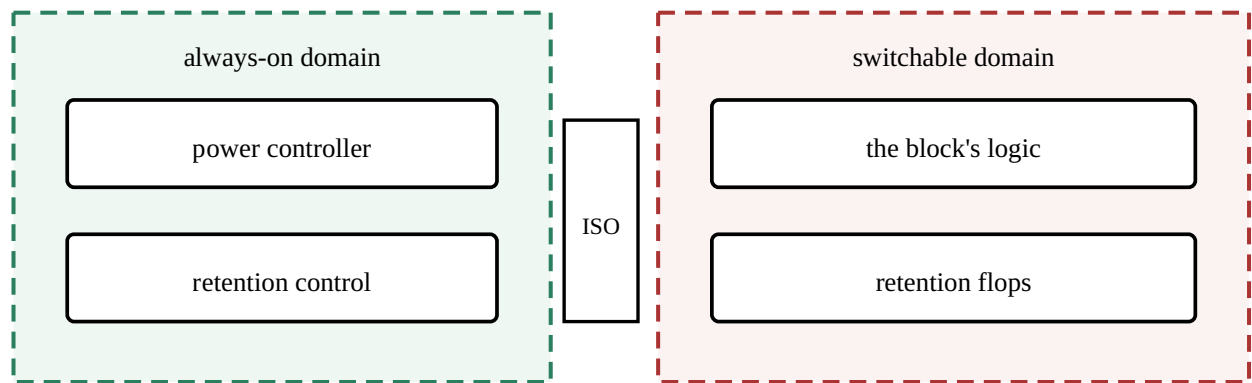
How this is got wrong. By quoting the reset requirement in clock cycles without saying which clock. ‘Assert reset for 16 cycles’ is ambiguous in a multi-clock block and is wrong by four orders of magnitude if the reader picks the fast clock. **State the requirement in absolute time, and state it for the slowest clock the block can be configured with** — including the case where a clock is stopped, which requires either that reset be held until it starts or that the domain be reset asynchronously and independently.

Problem. Your block occasionally fails to come out of reset, roughly one boot in a thousand. Where do you look?

Solution. The frequency is the clue: one in a thousand is a race, not a logic error, because a logic error would be deterministic. Three candidates, in order. **Reset de-assertion recovery:** if any domain's reset is not synchronised, de-assertion races the clock and metastability resolves the wrong way occasionally — the rate is computable from Part X3's MTBF formula with the reset edge as the asynchronous event, and one in a thousand boots is entirely consistent with an unsynchronised reset. **Clock-before-reset ordering:** if reset is released in the same window that a PLL locks or a clock gate opens, some flops see edges and some do not, giving the partial-reset state of X42.1; this shows a dependence on temperature or supply, because they change the lock time. **An uninitialised state element that is usually benign:** a flop without reset whose value normally does not matter, except in the rare case — and this is what random-initialisation regression is for. **The diagnostic that separates them is to vary the reset pulse width and the clock start time:** a recovery race moves with clock phase, an ordering problem moves with pulse width, and an uninitialised value moves with neither.

X43. Power Intent: Domains, Retention and Isolation

Power gating is the only technique that attacks leakage to near zero, and it is the one that changes an IP block's interface, its reset behaviour and its verification plan. A vendor whose block supports it properly is selling something materially different from one whose block does not.



Isolation cells clamp every signal leaving the switched domain.
Without them, a floating input burns static current in the domain that is still on.

Figure 49. A switchable power domain and the cells that make it safe.

X43.1 The four special cells, and what each prevents

Table 385. Low-power cells

Cell	Sits	Prevents	Cost
Isolation	On every signal leaving a switched domain	A floating value driving the always-on domain , which causes static crowbar current in the receiving gate	One gate per signal, plus the enable's routing
Level shifter	Between domains at different voltages	A low-voltage 1 failing to turn off a high-voltage PMOS — again static current	One cell per signal; slower than a buffer
Retention flop	Replacing flops whose state must survive	Losing state on power-down	20–30 % larger than a normal flop, plus always-on routing for the save/restore signals
Power switch	Between the always-on rail and the domain	—	Area, and IR drop through the switch when on ; the switching itself is an inrush event (Part X32)
Always-on buffer	On signals crossing a domain that is off	A signal path being cut	Special placement

Worked example 53. Is retention worth it, or should the block re-initialise?

Given. A block with 20 000 flops, of which 3 000 hold configuration and 17 000 hold datapath state. Leakage when on is 12 mW. The alternative to retention is for software to rewrite the configuration on wake.

Method. Compare three options on area, wake latency and residual leakage. Retention flops still leak, because their storage is always on.

Retain everything: retention flops	20,000
Area penalty at 25 %	5000 flop-equivalents
Residual leakage (retention cells stay powered)	1.8 mW (~15 %)
Retain configuration only	3,000 retention flops
Area penalty	750 flop-equivalents
Residual leakage	0.36 mW
Retain nothing: software rewrite	0 retention flops
Wake latency added	thousands of register writes over APB
Best for a block woken rarely	retain configuration only

How this is got wrong. By retaining everything because it is simplest. Datapath state is usually worthless across a power-down — the block was idle, so there is nothing in flight — and retaining it costs area and leakage for nothing. **The design work is to partition state into what must survive and what must not**, and then to make the latter's reset unconditional so that a wake-up is indistinguishable from a cold start. That partition is a specification item and is the single most useful thing a vendor can document about power behaviour.

X43.2 What UPF adds, and why RTL alone is not enough

Why power intent lives outside the RTL

	Step	Justification
1	RTL describes function; it has no notion of a supply.	<i>A Verilog module does not say which rail powers it.</i>
2	Power domains, switches, isolation and retention are <i>structural</i> and depend on the floorplan and the process.	<i>The same RTL is used with and without power gating in different products.</i>
3	So they are described in a separate file — UPF or CPF — that the tools read alongside.	<i>Synthesis inserts the cells, verification simulates the power states, and place-and-route builds the switch network.</i>
4	Simulation must then model a powered-down domain as X, and check that isolation is enabled before power goes away.	<i>This is the verification that cannot be done in RTL alone, and it is where the bugs are.</i>
5	The power state table enumerates the legal combinations of domain states.	<i>And the illegal ones are what must be proven unreachable — a formal problem of exactly the kind Part X33 recommends.</i>

Table 386. Power-state space against domain count

Switchable domains	Combinations	Typically legal states	Illegal states to exclude	Ordered transitions
2	4	3	1	12
3	8	4	4	56

Switchable domains	Combinations	Typically legal states	Illegal states to exclude	Ordered transitions
4	16	5	11	240
5	32	6	26	992
6	64	7	57	4,032

The legal column assumes a simple nested hierarchy — each domain requires the ones below it. **The transition column is the verification burden**, and it is quadratic, which is why power-state machines are kept deliberately simple and why a block that demands its own independent domain is expensive to integrate.

The classic power-gating bug is an isolation enable that arrives too late, and it is not a functional bug in the usual sense. If power is removed before isolation is enabled, the outputs float for a short interval; downstream gates in the always-on domain then sit at an intermediate level and conduct, drawing a current spike that is invisible to functional simulation and shows up as excess power or, in the worst case, as damage. **The sequence is: assert isolation, assert retention save, then remove power — and the reverse on wake.** Getting it right is a small state machine; proving it right requires power-aware simulation, and asserting it is what a UPF-aware formal check does. An IP block that supplies its own power-control sequence and the assertions that check it is removing a whole class of integration risk.

X43.3 DVFS: the controller and its constraints

Table 387. A DVFS operating-point table, normalised to the top point

V (V)	f (MHz)	Relative power	Relative energy per operation	Relative throughput	Power per unit throughput
0.9	1,200	1	1	1	1
0.8	1,000	0.6584	0.7901	0.8333	0.7901
0.7	800	0.4033	0.6049	0.6667	0.6049
0.6	600	0.2222	0.4444	0.5	0.4444
0.5	400	0.1029	0.3086	0.3333	0.3086

Energy per operation falls monotonically as voltage drops (the V^2 term), which is Part X7's result in the form a system actually uses. The last column is what a governor optimises when the work must be finished by a deadline rather than as fast as possible.

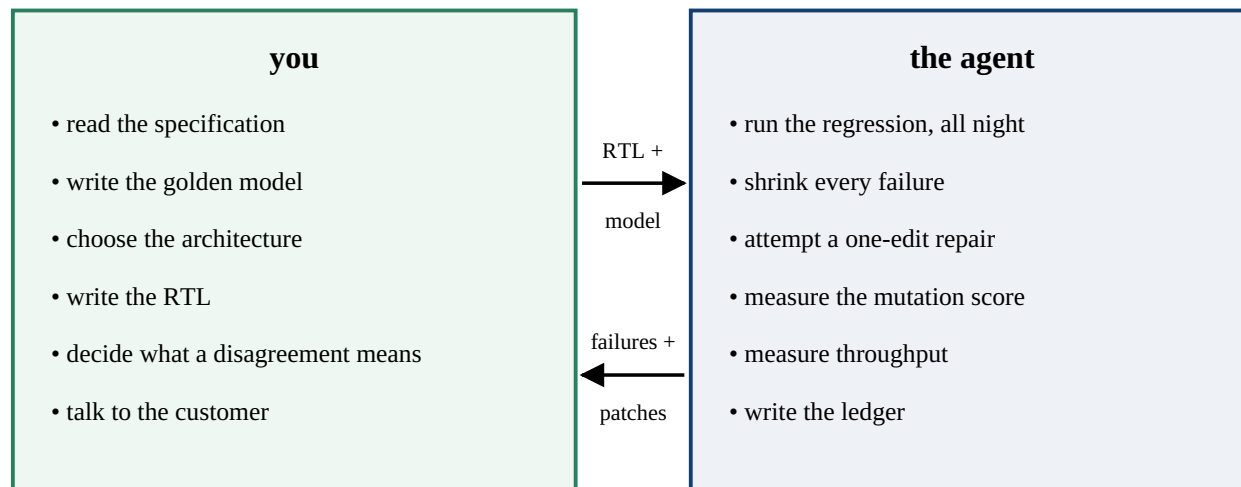
Problem. Why is ‘race to idle’ sometimes better than running slowly, when X43.2 shows energy per operation is lowest at low voltage?

Solution. Because the block is not the only thing consuming power. Running at the top operating point finishes the work sooner and lets the *whole subsystem* — memory, interconnect, PLLs, the always-on logic, and any peripheral waiting on the result — enter a low-power state earlier. That static and shared component does not scale with the block's voltage, so the comparison is between (fast, high dynamic, short time, then everything off) and (slow, low dynamic, long time, everything on throughout).

Race to idle wins when the fixed overhead per unit time is large relative to the block's own dynamic power, which is common in small IP blocks inside large systems and uncommon in a block that dominates its chip. **The crossover is computable** from the block's dynamic power at each point, the platform's idle power, and the entry/exit energy of the low-power state — and it is worth computing rather than asserting, because both answers appear in the literature and both are right for their own systems.

Z1. The Two-Member IP Team: You and the Agent

This volume is written for a specific arrangement: one engineer and one automated verification agent, producing IP that is good enough to sell. That is a smaller team than any company would assign to the job, and it can work — but only if the division of labour is decided deliberately rather than by what happens to be convenient. This part sets it out, and every claim about the agent in it is measured by running the agent while this page is rendered.



The division is not by difficulty. It is by whether a decision is needed.

The agent does what is checkable; you do what is judgeable.

Figure 50. The division of labour. Everything on the right is mechanically checkable; everything on the left needs a judgement.

Z1.1 The rule that decides who does what

Why the boundary falls where it does

	Step	Justification
1	The agent can only act where there is an oracle — something that says right or wrong without a human.	<i>A golden model, a standard's test vectors, a compiler, a timing report.</i>
2	Where an oracle exists, the agent's work is <i>search plus check</i> , and search with a good oracle is reliable and improves with compute.	<i>This is the regression, the shrink, the repair attempt and the mutation score.</i>
3	Where no oracle exists, any output must be read by a human, and the failure mode is work that looks right.	Which is the expensive kind of failure , because it is found late.
4	So: the agent owns everything downstream of a model; you own the model and everything upstream of it.	<i>Reading the specification, deciding what the block should do, deciding which side of a disagreement is wrong — none of these has an oracle.</i>
5	The corollary is where to invest: make the oracle better and the agent gets better for free.	<i>A stronger golden model, a wider stimulus, a throughput bound, a lint rule — each one extends the region the agent can work in, permanently.</i>

Table 388. Task by task, and why

Task	Who	Because
Read the standard, resolve ambiguities	You	No oracle; and the ambiguities are what you will be asked about
Write the bit-accurate golden model	You	It is the oracle. Writing it is how you find out what the specification actually says
Choose the architecture, the word lengths, the parallelism	You	Judgement against cost; Parts X1–X2 give the arithmetic
Write the RTL	You	For now. The agent repairs small defects; it does not architect
Run the regression on every seed, all night	Agent	Pure search against an oracle
Reduce a failure to one stimulus	Agent	Mechanical, and it is the difference between a debuggable and an un-debuggable report
Attempt a single-edit repair	Agent	Bounded search; three gates bound the false-accept rate
Measure the mutation score	Agent	The number that says whether the suite bites — and the escapes say where the stimulus is thin
Measure throughput against a stated bound	Agent	A value-only regression cannot see a block getting slower
Decide whether an escape is an equivalent mutant	You	A machine cannot decide behavioural equivalence in general
Decide which side of a disagreement is wrong	You	Only the specification settles it, and only you have read it
Talk to the customer, price the work, scope it	You	Part Y5

The temptation to let the agent write the model is the one thing that destroys the arrangement. If the model and the RTL come from the same process, they share its misunderstandings and their agreement proves nothing — this is the independence argument of Part Y11, and it does not change because one of the two authors is a machine. The agent may write *stimulus*, may write *checkers derived from the model*, and may repair RTL; it must not be the source of the reference. **Your reading of the specification is the asset the business is built on**, and it is the one part that cannot be delegated.

Z1.2 What the agent actually does, measured now

Table 389. The agent's blocks, run while this page was rendered

Block	Checker bites?	Stimuli	Mismatches	Distinct outputs	Throughput bound?	Wall time
crc32_8	yes	600	0	599	—	0.0284 s
crc32_stream	yes	31	0	31	yes	0.033 s
enc8b10b	yes	1,200	0	355	yes	0.0349 s
gf_mul	yes	600	0	444	—	0.0268 s
regfile	yes	600	0	78	—	0.0334 s
rs_syn_step	yes	600	0	458	—	0.0248 s

Run with Icarus Verilog at build time. **The ‘checker bites’ column is produced by deliberately corrupting a golden value and asserting that the comparison notices** — without it the mismatch column would be an unexamined green light.

Table 390. Mutation score, measured at build time

Block	Mutants run	Caught	Score (%)	Escapes	Time
crc32_8	6	6	100	0	8.1 s
crc32_stream	8	8	100	0	8.4 s
enc8b10b	16	16	100	0	0.6 s
gf_mul	6	6	100	0	8.1 s
regfile	13	13	100	0	0.5 s
rs_syn_step	6	6	100	0	8.1 s

Each mutant is a single-token edit to the DUT — not to the testbench, and not inside a comment, because mutating either produces escapes that mean nothing. **An escape is a finding, not a defect in the score:** it is either an equivalent mutant or a place the stimulus does not reach, and both need a human to say which. **Every block reads 100 % with zero unjudged escapes, which is the state the release gate of Part Z3 requires** — a column that is constant because the bar was met, not because the measurement is stuck. It was not always so: the register file scored 58.3 % until the escapes were read.

Z1.3 A worked morning: what the escapes actually told us

The mutation score is not a grade; it is a pointer. The following happened while this book's own register-file block was being built, and it is the clearest illustration in the book of why the number is worth measuring.

Table 391. From a score to two real bugs

Step	What happened	What it cost
Regression written, all green	400 APB accesses, no mismatch	—
Mutation score run: 58.3 %	Five escapes, all pointing at the same place	20 minutes
Escapes read	$r \mid hw_set \rightarrow r \wedge hw_set$ not caught — because hw_set was tied to zero, and 0 behaves identically under both	The W1C and RC paths had never executed
Testbench fixed to pulse the hardware inputs	Plus a directed sequence: set three bits, read, clear one, check the other two survive	an hour
Regression re-run: two failures	Both were real generator bugs — an ID register that could not hold a constant, and an RC register cleared during the APB setup phase because <code>penable</code> was missing from the read term	—
Fixed, re-scored	100 % (13 of 13)	Two bugs that would otherwise have been found by a customer's driver

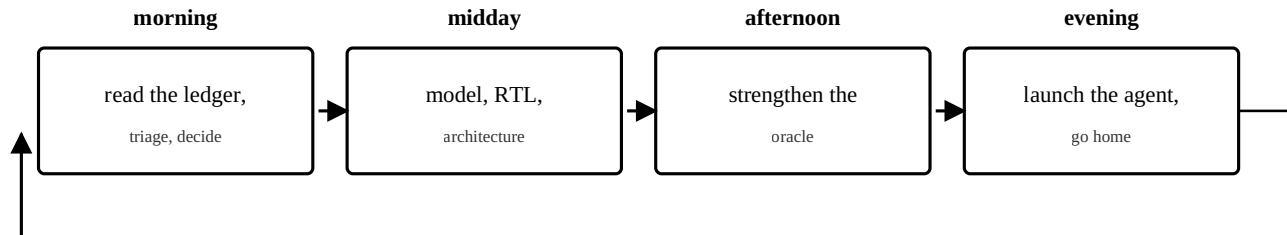
Graduate · Note what the green regression was worth before the mutation score ran. It was not wrong — those 400 accesses really did match — and it was nearly worthless, because the stimulus never reached the logic that was broken. **This is the difference between ‘the tests pass’ and ‘the tests would fail if the design were wrong’**, and only the second is evidence. For a one-person house it is also the only affordable substitute for a verification team's scepticism: you cannot review your own work with fresh eyes, and the mutation score does not need fresh eyes.

Problem. You are one person. How much of the day should go to verification?

Solution. More than feels reasonable, and the arithmetic from Part Y1's effort table says roughly how much: verification and everything downstream of RTL is about three quarters of the work in a sellable block. But the useful answer for a day rather than a project is a **shape**, not a fraction. **Verification infrastructure is built once and runs forever**, so it belongs early and its cost is amortised; **the agent runs it while you sleep**, so the marginal cost of *running* verification is zero and only the marginal cost of *improving* it is real. That changes the allocation: spend your hours on the golden model, the stimulus and the checkers — the things that decide what the agent can catch — and almost none on running or triaging, which the agent does. **A practical daily target is that every session leaves the oracle stronger than it found it:** one more property checked, one more corner in the stimulus, one more bound measured. That compounds, and it is the only thing in a one-person house that does.

Z2. The Daily and Weekly Loop

A two-member team has an unusual property: one member works when the other does not. That makes the shape of the day the main scheduling decision, and getting it right is worth more than working longer.



The agent works the interval you are not there.
Your afternoon decides what it is able to find that night.

Figure 51. One day.

Z2.1 The day

Table 392. Four slots, and what each is for

When	You do	Why then	What it must not become
Morn- ing	Read the night's ledger. Triage: real failure, equivalent mutant, infrastructure noise. Apply or reject the proposed patches.	The information is freshest and the decisions unblock the rest of the day	Debugging. Triage decides <i>what</i> to debug; the debugging happens midday
Midday	The work that needs an unbroken stretch: the golden model, architecture, RTL, the hard bug from triage.	It is the only slot long enough	Email, or fiddling with the flow
After- noon	Strengthen the oracle: one more property, one more corner in the stimulus, one measured bound, one lint rule.	This is what decides what the agent can catch tonight	Optional. It is the compounding part and it is always the first thing sacrificed
Even- ing	Launch the run and stop. Check it is alive, not that it is finished.	The agent has the night	Watching it. If you are watching, the ledger is not good enough

```

The evening command, and the morning one
# 저녁 -- 밤새 돌린다 (setsid/nohup/disown; 확인은 pgrep)
bash edu/agent/run.sh 12

# 아침 -- 무엇이 있었나
bash edu/agent/status.sh
python3 edu/agent/mutscore.py          # 주 1회면 충분하다

# 밤새 빨간불이 났다면: 최소 사례와 제안된 편집이 원장에 있다
grep '빨간불' edu/agent/ledger/regress.jsonl | tail -5
  
```

The failure mode of this loop is that the afternoon slot disappears. It is the only slot with no immediate consequence — skip it and tonight's run still happens, this week's deliverable still moves — so it is the one that gets spent on whatever is urgent. Three weeks later the agent is finding nothing, not because the design is clean but because the oracle stopped growing

while the design kept growing. **The measurable symptom is a flat mutation score against a rising line count**, and it is worth plotting: if the design grew 30 % and the mutant count did not, the suite is covering a smaller fraction of it than it was.

Z2.2 The week

Table 393. The week

When	What	Why
Monday	Plan against the deliverable list; pick this week's one thing	A week with two goals finishes neither
Tuesday–Thursday	The daily loop	—
Friday morning	Mutation score across every block; read every escape	Weekly is the right cadence — daily is noise, monthly is too late to act on
Friday afternoon	Documentation for what changed this week	Written now it takes twenty minutes; written at release it takes days and is wrong
Friday, last thing	Tag, and run the full gate from a clean checkout	A green working directory is not a green repository — the commit may be missing a file

The Friday items are the ones that make a release possible without a release scramble. Both are cheap weekly and expensive in a batch, which is the general shape of everything in this chapter.

Worked example 54. What a night is actually worth

Given. The agent runs 12 hours. Each block's regression takes about 0.02 s of simulation per 600 stimuli, and a full mutation pass over five blocks takes about a minute.

Method. Count what the night can cover, then note what limits it — which is not the machine.

Regression rounds per hour (order)	10^5
Distinct seeds consumed in 12 hours	$\approx 10^6$
Mutation passes possible in one night	hundreds
Engineer-hours consumed	0
What actually limits the value	the stimulus distribution and the golden model
Marginal value of the 500th mutation pass	zero
Marginal value of one new checker	large and permanent

How this is got wrong. By measuring the night in rounds. A million rounds of the same distribution explore the same region a thousand times — Part X8's coupon-collector arithmetic says the return flattens far below that. **The honest use of the night is variety, not volume:** rotate the constraint sets, replay every previous minimal reproducer as a growing regression, and run the mutation pass. A loop that only turns the seed stops learning within the first hour and then keeps the machine warm until morning.

Z2.3 Triage: what to do with each thing in the ledger

Table 394. The morning decision table

Ledger says	You do	Time
초록 on every block	Nothing. Read the mutation trend instead	1 min

Ledger says	You do	Time
자해검사 failed	Stop. The night's results mean nothing. Fix the harness first	Highest priority — a broken checker is worse than a broken design
자극이 DUT 를 안 건드렸다	The stimulus degenerated — a constraint is over-tight or a parameter is wrong	10 min
빨간불 -- 수리 적용됨	Read the patch. It passed three gates; it still has to be right for the right reason	5 min per patch
빨간불 -- 수리 후보 있음 (적용 안 함)	The agent found an edit and was not allowed to apply it. Decide	5 min
빨간불 -- 수리 실패	This is the real work. The minimal reproducer is in the ledger; it is outside the repair library, which usually means it is structural	The morning's budget goes here
적용했다가 되돌림	The edit passed the first seeds and failed after application — look at why, it is usually an interacting change	15 min
An escape in the mutation report	Equivalent mutant, or a hole in the stimulus. Write down which, and the reason	5 min each, weekly

Graduate · **Record the verdict on every escape, permanently.** An escape you have judged to be an equivalent mutant will reappear in every run forever, and re-deciding it each week is both wasted time and an invitation to stop reading the list. Keep a file of judged escapes with the reason — ‘extra loop iteration reads an out-of-range bit, which is x, so the branch is never taken’ — and have the report mark them as known. **What must not happen is suppressing them silently:** a suppressed escape whose reasoning was wrong is a permanent blind spot, and the file is the thing that lets a reviewer, or you in six months, check the reasoning.

Problem. The agent applied a patch overnight and the regression is green. Do you trust it?

Solution. Trust the *evidence* and check the *reason*, which are different things. The evidence is strong: the edit passed the original seed, passed a seed never used during the search (the overfitting gate), and passed the self-check confirming the comparison still bites. That rules out the classic automated-repair failure of special-casing the failing input. **What it does not establish is that the edit is the change a human would have made.** An edit that makes the symptom go away by tightening a condition can be correct on every stimulus the suite has and wrong in a case the suite does not reach — and if the suite reached that case, the bug would have been caught before the patch. So: **read the diff, and ask what the original defect was.** If you can state the defect in one sentence, keep the patch. If you cannot, the patch is masking something and the right response is to extend the stimulus until the failure is understood. **Five minutes per patch, and it is the five minutes that keeps the arrangement honest.**

Z3. The Quality Bar: What 'Sellable' Means as Numbers

'Industry level' is not a feeling. A customer's technical evaluation is a list of questions with numerical answers, and a block either has them or does not. This part is that list, with the acceptance criterion for each and the command that produces it. **A block that passes every row is sellable; one that passes most of them is a prototype.**

Z3.1 The gate list

Table 395. The gates a sellable block passes

Gate	What it asserts	How it is measured	Why
Functional agreement	Zero mismatches against an independent golden model, on many seeds	agent: <code>잔다() 틀림 == 0, ≥ 20</code> seeds	Non-negotiable
The checker bites	Corrupting a golden value makes the comparison fail	<code>자해검사() == True</code> every run	Without it the row above is meaningless
Stimulus is non-degenerate	The golden model produced many distinct outputs	<code>서로다른출력 ≥ 2</code> , in practice > 20	A constant stimulus passes any design
Mutation score	Injected defects are caught	<code>mutscore.py ≥ 90 %</code> , every escape judged in writing	The number a knowledgeable buyer asks for
Throughput	Cycles per unit of work within a stated bound	<code>성능한계()</code> declared and met	A value-only suite cannot see a block getting slower
Code and toggle coverage	Every line and every bit exercised	Verilator or a coverage-capable simulator	Weakest of the metrics; necessary, not sufficient
Lint clean	No inferred latches, no width mismatches, no incomplete sensitivity	<code>verilator --lint-only -Wall</code> , zero warnings or a written waiver	The customer will run this and judge you by it
Synthesises	To a gate netlist, with reported area and frequency	<code>yosys</code> plus a timing run, scripts shipped	An unsynthesisable block is not a product
CDC analysed	Every crossing identified, structured and constrained	A CDC tool, or a documented manual analysis with the constraints	The commonest integration failure
Reset verified	Random initialisation, many seeds; de-assertion synchronised per domain	Regression with randomised initial state	The commonest silicon failure
Register map generated	RTL, header, docs and IP-XACT from one source	<code>house/regmap.py --전부</code>	Hand-maintained copies diverge
Documentation	Datasheet, integration guide, known issues, release notes	They exist and match the code	Support cost is inversely proportional to this
Reproducibility	A clean checkout reproduces every number	<code>scripts/precheck.sh</code> from HEAD, not the working directory	A missing file passes locally and fails for the customer

Nine of the thirteen are runnable by the agent, unattended. The four that are not — judging escapes, CDC analysis, documentation, and deciding what the block should do — are the ones this volume says belong to you.

The bar is not 'no known bugs'; it is 'here is the evidence, and here is what is not covered'. Every real IP block ships with limitations, and Part Y5 argues that naming them is what distinguishes a professional supplier. The gate list above is

what converts that from a posture into a document: each row either passes, or fails with a stated reason and a workaround. **A customer can evaluate that. They cannot evaluate an assurance.**

Z3.2 Running the whole bar in one command

A gate that requires remembering to run it is a gate that will be skipped in the week it matters. The repository's own discipline applies: one command, from a clean checkout, that runs everything and refuses to report green on anything it could not check.

```
edu/house/release.sh — the release gate
bash edu/house/release.sh <블록이름>

1. 깨끗한 판을 꺼낸다          -- 작업 디렉터리를 안 본다
2. 자해검사                   -- 비교가 무는가
3. 회귀 (씨앗 20개)           -- 값과 순서
4. 성능                       -- 선언된 한계 안인가
5. 변이 점수                   -- 몇 %, 탈출은 몇 개
6. lint (verilator -Wall)
7. 합성 (yosys)               -- 면적과 셀 수
8. 레지스터 맵 생성물         -- 컴파일되는가
9. 문서가 있는가              -- 데이터시트·통합가이드·알려진문제
10. 보고서를 낸다             -- 통과/실패와 **못詹 것**을 같이

**못詹 것을 초록이라고 하지 않는다.**  도구가 없으면 '없어서 못詹다' 로 낸다.
```

Worked example 55. What the gate costs to run

Given. Five blocks, the regression at 20 seeds each, a mutation pass, lint and synthesis.

Method. Estimate from the measured per-step times in this book's own runs.

Regression, 600 stimuli	0.03 s per seed per block
20 seeds × 5 blocks	3 s
Self-check	0.15 s
Mutation pass, 5 blocks	≈60 s
Lint	seconds
Synthesis	seconds to minutes per block
Whole gate	a few minutes
How often it should run	every push, and nightly

How this is got wrong. By assuming the gate is expensive and running it at release. At a few minutes it is cheap enough to run on every push, and a gate that runs on every push finds the regression on the commit that caused it rather than three weeks later. The expensive parts — a full mutation pass over a large block, a long synthesis — belong in the nightly run, which costs nothing at all because nobody is waiting.

Z3.3 What the numbers should look like

Table 396. Acceptance thresholds: internal and at release

Metric	Internal (keep working)	Release	Note
Mismatches	0	0	Any non-zero is a stop
Self-check	passes	passes	A failure invalidates the run

Metric	Internal (keep working)	Release	Note
Distinct outputs	> 20	> 20	Low means the stimulus degenerated
Mutation score	$\geq 90\%$	$\geq 95\%$	Escapes all judged and recorded
Escapes unjudged	0	0	An unjudged escape is an unexamined hole
Lint warnings	0	0	Or a written waiver per warning
Coverage (line)	> 95 %	> 99 %	Unreached lines explained
Throughput margin	> 10 %	> 20 %	Against the declared bound
Documented registers	100 %	100 %	Generated, not typed
Known issues listed	yes	yes	Absence is not a clean sheet

Two columns because they serve different purposes. The internal column is what you refuse to go home below; the release column is what a customer's evaluation will find. Setting the internal bar lower than the release bar is how a release becomes a scramble.

Problem. A customer's technical evaluation asks for your verification plan. What do you send?

Solution. Not a narrative. Send four things, and their existence is most of the answer. **The gate list of Z3.1 with each row's current value** — this is a table of facts and it is what they are actually trying to establish. **The mutation report, escapes included**, because it is the one artefact that demonstrates the suite bites rather than merely runs, and very few suppliers have one; including the escapes with their judgements is more convincing than a higher score without them. **The known-issue list**, dated, with workarounds. **And the command they can run themselves** on the delivered package to reproduce every number. **The fourth is the one that converts the first three from claims into evidence**, and it is the reason the release gate must run from a clean checkout rather than from your working directory — if it only works on your machine, they will find that out, and it will cost more than the bug it hides.

Z4. The Deliverable Pack, Built and Gated

Part Y5 lists what a customer receives. This part builds it. Every artefact named here exists in this repository, every generator named here runs, and the gate that checks them is run while this page is rendered. A book that describes a deliverable pack without producing one is teaching the habit it warns against.

Z4.1 What is in the repository

Table 397. The pack, with line counts read from disk at build time

File	What it is	Lines
edu/house/regmap.py	한 맵에서 RTL · C 헤더 · 문서 · IP-XACT 를 낸다	387
edu/house/release.sh	판매 가능 관문 전체를 한 번에 돌린다	188
edu/house/DATASHEET.md	데이터시트 (젠 수와 그 조건)	72
edu/house/INTEGRATION.md	통합 가이드 (전화하지 않고 끝낼 수 있게)	62
edu/house/KNOWN_ISSUES.md	알려진 제한 (없다고 적지 않는다)	27
edu/agent/harness.py	골든모델 대 RTL, 자해검사와 처리량 포함	186
edu/agent/mutscore.py	변이 점수 -- 검사가 무는지를 수로	165
edu/agent/vrepair.py	한 곳 편집 수리, 과적합 관문 포함	195
edu/agent/agent.py	밤새 도는 고리	196
tests/test_verif_agent.py	에이전트를 실제로 돌려 보는 검사	279
tests/test_release_gate.py	**관문 자체를 망가뜨려 보는 검사**	123

Counted by reading the files as this page was rendered. A row showing zero would mean the book is describing something that does not exist, which is the failure this table is designed to make visible.

Z4.2 One source, four outputs — measured

Table 398. Generated from one map of 7 registers and 16 fields

Output	Lines	Characters
RTL (Verilog)	101	4,575
C header	99	5,554
Markdown (datasheet section)	78	2,316
IP-XACT	186	8,962

Generated at build time. Part Y10 computed that maintaining six copies by hand over a nine-month project has an expected nine missed edits; the generator is a day's work and removes all nine.

Graduate · The map validates itself, and that is most of the value. The definition refuses overlapping bit ranges, offsets that are not word-aligned, reset values too wide for their field, unknown access types, duplicate offsets, and — the one that matters most — a hardware-written field declared RW, which is the read-modify-write race of Part Y10. **Each of those is a defect that a hand-written map produces regularly and that nothing downstream detects:** the RTL compiles, the header compiles, the documentation looks right, and the bug appears in the customer's driver.

24.3 What the gate says today

Table 399. The release gate, run while this page was rendered

Block	Passed	Failed	Unmeasured	Mutation score	Cells (yosys, open library)
crc32	7	0	0	100.0 %	216
crc32_stream	7	0	0	100.0 %	1,983
enc8b10b	7	0	0	100.0 %	208
gf_mul	7	0	0	100.0 %	208
regfile	7	0	0	100.0 %	419
rs_syndrome	7	0	0	100.0 %	218

*Totals: 46 passed, 0 failed, 0 unmeasured. The cell counts are from an open-source synthesis flow and an open library; **they are not an area figure in any customer's process**, and Part Y14's rule says to present them with that stated rather than to quote a bigger number without it.*

24.4 Three generator bugs the gate found

The register-map generator was written from Part Y10's argument and looked correct. The agent and the gate found three defects in it, none of which any amount of reading would have found, and each is instructive about a different failure mode.

Table 400. What was found, and by which gate

Bug	Found by	Why reading would not have found it
An ID register could not hold a constant. Every RO field was wired to a hardware input, so a vendor code read as zero	The regression, against an independent model	The generator was internally consistent — it did exactly what its author intended, and the intention was incomplete
A read-clear register cleared during the APB setup phase. rd was psel & ~pwrite, missing penable	The regression, after the stimulus was extended	It reads as correct; the defect is in the protocol's two-phase timing, which only a running comparison exposes
A software-set field had an unused hardware port. W1S defaulted to hardware-driven, which is wrong for the common case	verilator -Wall (UNUSED SIGNAL)	An unused port is harmless functionally and is a question the integrator has to ask you

And the mutation score found the stimulus hole that hid the second one. Before it ran, the register-file regression was green on 400 accesses and scored 58.3 %. Five escapes all pointed at the same place: the hardware-side inputs were tied to zero, so the W1C and RC paths had never executed. Driving them turned up two real failures within minutes. **The green regression was not wrong — those 400 accesses did match — and it was nearly worthless.** That distinction is the whole argument for measuring whether a suite bites rather than whether it passes.

Problem. You have one week before a customer evaluation. What do you build first?

Solution. Not more features. Build, in this order, the things the evaluation will actually test. **First the gate**, because everything else is easier to trust once one command produces the numbers; a day. **Then the mutation score**, because it is the single artefact that distinguishes you from a supplier who says ‘we tested it thoroughly’, and because it will find the stimulus holes you do not know about; half a day plus whatever it turns up. **Then the known-issue list**, which costs an afternoon and is read as competence rather than as weakness. **Then the integration guide**, written as if for someone you will never speak to. **Only then the datasheet**, because by that point every number in it exists and is reproducible — which means writing it takes an hour instead of a day and none of it is invented. **A week spent this way produces a smaller block that can be bought; a week spent on features produces a larger one that cannot.**

Z5. Choosing the First Product, Concretely

Part Y8 gives the criteria. This part applies them to real candidates with weights chosen for a one-person house, and arrives at a recommendation — including the one column that is currently zero for every candidate, which is the honest headline.

Z5.1 The scorecard

Table 401. Criteria and weights for a one-person house

Criterion	Weight	Why this weight
규격을 구할 수 있나	3	못 구하면 못 만든다. 끝
독립 골든모델이 있나	3	혼자 검증하는 유일한 길
소프트 IP 인가	3	하드 매크로는 아날로그와 공정 접근이 필요하다
표준이 바뀌는 중인가	2	기존 공급자의 블록이 낡는다 -- 들어갈 틈
공급자가 적은가	2	많으면 가격으로 진다
가족으로 번지나	2	둘째 셋째가 싸진다
이름 델 고객이 있나	3	없으면 투기다
검증이 기계화되나	2	에이전트가 밤새 할 수 있나

Weight 3 means a zero here kills the candidate regardless of the rest. A larger company would weight ‘specification obtainable’ at 1, because a membership is a rounding error in its budget; for one person it is a gate.

Table 402. Scored 0–3 per criterion, weighted; maximum possible 60

Candidate	규격 을...	독립 립...	소프 트...	표준 이...	공급자 가...	가족으 로...	이름...	검증 이...	Total	Note
RS-FEC (KP4) 복호기	1	3	3	3	2	3	0	3	43	규격 조문(Clause 91)을 아직 못 봤다 -- 그것이 1번 걸림돌
AES-GCM 엔진	3	3	3	0	0	2	0	3	37	규격도 시험벡터도 공짜인데 **공급자가 너무 많다**
ML-KEM (Kyber) NTT 코어	3	3	3	3	3	2	0	3	49	규격 공개, 참조 구현 공개, 표준이 막 확정, 공급자 적음
MIPI CSI-2 수신기	1	1	2	1	1	2	0	2	24	규격이 유료 회원제, PHY 가 하드 매크로 -- 1인에게 무겁다
이더넷 PCS (802.3dj 내부코드)	1	2	3	3	3	3	0	3	42	소프트·바뀌는 중·공급자 적음. 규격 조문이 다시 걸림돌

Scores are judgements, and stating them as numbers is what makes them arguable. A reader who disagrees can change one cell and see whether the ranking moves — which is the point of a scorecard and the reason it beats a paragraph of reasoning.

Graduate · On these weights the winner is ML-KEM (Kyber) NTT 코어, and the reason is worth reading off the table rather than taking on trust: it is the only candidate that scores 3 on *all four* of specification availability, independent golden model, soft RTL, and a changing standard with few suppliers. The post-quantum transition put a new, fully public, algorithmically demanding block into a market whose incumbents built their portfolios around RSA and ECC. **And Part X6's entire FFT analysis transfers to the NTT** — same radix choices, same twiddle symmetries, same memory-versus-multiplier trade, with modular arithmetic replacing complex arithmetic. A team that has built an FFT has most of it.

Every candidate scores zero on ‘a customer you can name’, and that is the real finding. The scorecard ranks technical and market fit; it cannot manufacture demand. Six good rows and a zero in that column describes a project, not a business. **The action the table implies is not to start building the winner — it is to spend the next two weeks turning that column into a non-zero**, by talking to people who buy this kind of block, and to let what they say re-score the other rows. A candidate that scores 40 with a named customer beats one that scores 48 without.

25.2 Sizing the winner before committing

Worked example 56. What an NTT core costs to build, estimated from the structure

Given. ML-KEM uses a 256-point NTT over the prime $q = 3329$, so coefficients are 12 bits and the modulus is small. Compare against the FFT arithmetic of Part X6.

Method. Count butterflies, multipliers and memory the same way Part X6 does, then convert to weeks with Part Y8's plan.

Transform size	256
Stages (radix 2)	8
Butterflies per transform	1,024
Coefficient width	12 bits
Modular multiplier (12×12 then reduce)	small — a few hundred gates
Coefficient memory	384 bytes
Twiddle ROM (128 entries)	192 bytes
Compared with a 4096-point complex FFT	two orders of magnitude smaller
Plausible effort from Part Y8's plan	half of the 39 weeks — the block is small, the verification is not

How this is got wrong. By sizing the arithmetic and forgetting that the verification does not shrink with the datapath. The core is small; the golden model, the known-answer vectors from the standard, the constant-time argument (Part X36), the register map, the documentation and the release gate are the same work they would be for a block ten times larger. **That asymmetry is good news for a first product** — it means a small block can carry a full-quality wrapper, which is exactly what a first product needs to demonstrate.

25.3 The two weeks before you write any RTL

Table 403. Two weeks, and what each part produces

When	What	Why it is in this order
Day 1–2	Prior-art note (Part Y6): closest work, how we differ, queries run, **where we have not looked**	Committed before any code — the repository's own rule
Day 3–5	Read the standard; write down every ambiguity	The ambiguity list is your first deliverable and your first sales document
Day 6–8	Golden model in Python, checked against the standard's known-answer vectors	If the vectors do not match, you have misread the standard — find out now, not in month four
Day 9–10	Architecture study: word lengths, parallelism, memory (Parts X1, X6)	Produces the area and throughput numbers you will quote
Day 11–12	Talk to three potential customers with the ambiguity list and the architecture study	This is the step that turns the zero column into a number

When	What	Why it is in this order
Day 13–14	Re-score the candidates with what they said	The scorecard was a guess until this point

No RTL in fourteen days, and that is deliberate. Every item above changes what the RTL should be, and each is cheap to redo and expensive to redo after the RTL exists.

Problem. You cannot get three customer conversations. Now what?

Solution. Treat that as data about the market rather than as an obstacle to route around. If nobody will spend twenty minutes discussing a block they might buy, the likeliest explanations are that the block is not a problem they have, that they already have a supplier they are satisfied with, or that you have not found the right people — and the third is the only one that is your mistake to fix. **Widen the search before widening the scope:** the people who answer are rarely procurement; they are engineers, and they are reachable through the standards body's working group, the open-source projects around the protocol, and conference and plugfest attendee lists. **A useful substitute when conversations genuinely cannot be had is a public artefact that attracts them** — the golden model published with the standard's vectors, a note on an ambiguity you found in the specification, a comparison of the open implementations. Part Y6's argument for publishing applies exactly here: it is how a house nobody has heard of becomes findable. **What is not an answer is to start building and hope**, because the 39 weeks are spent either way and only one of the two ends with someone to sell to.

Z6. What the Agent Cannot Do

A tool's limits are more useful than its features, because the limits decide where your attention has to go. This part states the agent's honestly, each with the evidence from this repository's own use of it, and then says what to do about each one.

Z6.1 The limits, named

Table 404. What the agent does not do, and why not

Limit	Why	What you do instead
It cannot decide what the block should do	No oracle exists for a specification reading	You read the standard. This is the asset
It cannot write the golden model	The model <i>is</i> the oracle; a machine-written oracle checked by the same machine proves nothing	You write it, from the standard, before the RTL
It cannot architect	Architecture is a judgement against cost, and the cost model is yours	Parts X1, X2, X6, X13 give the arithmetic; you apply it
Its repairs are single-token edits from a narrow library	A wider library finds something for every failure and most of it is wrong	Structural bugs come to you with a minimal reproducer — which is the valuable part
It cannot judge an equivalent mutant	Behavioural equivalence is undecidable in general	You judge each escape once and record the reason
It does not know what the stimulus is missing	It measures what the stimulus reaches, not what it should reach	The mutation escapes point at the holes — read them
It cannot verify timing, area or power	Those need a library and a flow, not a comparison	The release gate runs lint and synthesis; sign-off is the customer's
It cannot talk to a customer	—	Part Y5. This is most of whether the business works

The limit that does the most damage when forgotten is the second. Under deadline it is very tempting to have the machine write the reference too — it is faster, it looks like the same kind of work, and the result will agree with the RTL. It will agree because both came from the same reading, which is the one thing the reference exists to be independent of. **Every disagreement you delete this way is a check you cannot get back**, and nobody will remember six months later which checks were deleted or why. Part Y11 states the rule for a human team; it does not change when one member is a machine.

Z6.2 What happened when the limits were tested

Table 405. This repository's own record

What the agent did	What it could not do	Who closed the gap
Found the ID-register bug on the first run	It could not say the generator's <i>design</i> was incomplete	A human decided RO fields should default to constants
Found the RC/penable bug after the stimulus was extended	It could not find it before — the stimulus never reached the logic	The mutation escapes pointed at the hole; a human read them
Scored the register file at 58.3 %	It could not say <i>why</i> the five escapes shared a cause	A human read three lines and saw hw_set tied to zero
Repaired three injected Verilog faults exactly	None of the three was a structural bug	— the library is deliberately narrow

What the agent did	What it could not do	Who closed the gap
Caught a throughput-only regression once the bound existed	It could not invent the bound	A human measured the clean block and chose 2.6 cycles/byte
Ran all night, every night, for free	—	This is the part that is genuinely free, and it is not the valuable part

Graduate · Read the last two rows together. The compute is free and the bound is not; the agent contributed the checking and a human contributed the number being checked against. Every one of the agent's real finds in this repository traces back to a human decision about what to compare, what to drive, or what bound to declare. **That is not a complaint about the agent — it is the investment thesis.** An hour spent on the oracle is repaid every night indefinitely; an hour spent running the agent is repaid once and would have happened anyway.

Z6.3 The failure modes of working with it

Table 406. How a two-member team goes wrong

Failure	How it looks	The early symptom
The oracle stops growing	The agent finds nothing and everyone is pleased	Flat mutant count against a rising line count — plot it
Patches accumulate unread	Green regressions, RTL nobody can explain	A patch whose original defect you cannot state in one sentence
Escapes suppressed rather than judged	The score rises and the blind spots stay	A suppression file with no reasons in it
Triage becomes debugging	The morning consumes the day; the afternoon slot disappears	Part Z2's first symptom, and the commonest
Stimulus tuned until it passes	A constraint narrowed to make a failure go away	Distinct-output count falling — the gate checks it for this reason
Believing the mutation score is the goal	Mutants chosen because they are easy to catch	The library stops growing while the score stays at 100 %

Worked example 57. Detecting the first failure mode before it costs a month

Given. Plot, weekly: lines of DUT code, number of mutants generated, mutation score, and distinct outputs in the regression.

Method. Each pair of series answers a question that neither answers alone.

Lines rising, mutants flat	the suite covers a shrinking fraction
Mutants rising, score flat	healthy — new code is being checked
Score rising, escapes flat at zero	healthy
Score 100 %, mutants flat for weeks	the library has stopped growing, not the design
Distinct outputs falling	the stimulus is narrowing
Everything flat	you are not working on this block — fine, if deliberate

How this is got wrong. By watching the score alone. A mutation score is a ratio, and a ratio hides both of its terms: 100 % of eight mutants on a block that has grown to four thousand lines is a much weaker statement than 90 % of two hundred. **Plot the numerator and the denominator**, which costs nothing because the agent already records both, and the failure mode announces itself weeks before it matters.

Problem. Should the agent be allowed to apply its own patches unattended?

Solution. Yes for a narrow class, and the class is worth defining precisely rather than deciding by feel. **Allow it when all four hold:** the edit is a single token from the declared library; it passed a seed never used during the search; the self-check confirms the comparison still bites; and the block is one whose golden model you trust because it is independent. **Refuse it otherwise,** and in particular refuse it on any block whose reference model you wrote by reading the RTL — there the whole gate structure rests on nothing. Two practical additions make the yes safe. Keep the default as report-only (‘--적용’ is opt-in, which is how this repository ships it), so an unattended night cannot silently change the design. And require that every applied patch be reviewed in the morning with the one question of Part Z2: *what was the original defect?* **If you cannot state it, the patch is masking something,** and the right response is to extend the stimulus rather than to keep the patch.

Z7. Solo Bring-Up: The Bench You Actually Need

At some point a block has to run on real hardware, because a customer will ask whether it has and because simulation does not model the things that break. This part is about the smallest bench that answers that honestly, what it costs, and what it cannot tell you.

Z7.1 What an FPGA prototype proves — and what it does not

Table 407. Claims a prototype supports

Claim	Prototype evidence?	Why
The RTL is functionally correct at speed	Strong	Billions of cycles against real traffic — far beyond simulation
It interoperates with a real partner	Strong, and unobtainable any other way	The partner's implementation is the thing you cannot simulate
Reset and initialisation work	Strong	Real power-up, real clock start — Part X42's untestable cases
The clock-domain crossings are sound	Weak	Metastability at the FPGA's rates and MTBF is not the ASIC's
It meets the ASIC frequency	No	Different fabric entirely. An FPGA number says nothing about an ASIC number and quoting it as if it did is the classic misrepresentation
The area estimate is right	No	LUTs are not gates
Power is acceptable	No	Unrelated
Timing closes at the customer's corners	No	Their library, their flow, their corners

The temptation is to quote the FPGA frequency. It is a real number that was really measured, and it is about a different technology. A customer's evaluator knows this and will read the substitution as either ignorance or misdirection, both of which cost more than the number was worth. **Quote it as what it is:** ‘runs at 200 MHz on *this* part, with this build, and here is the project’, alongside a separately-labelled synthesis estimate with its library named. Part Y14's rule again: the honest smaller claim is more persuasive than the unattributed larger one.

Z7.2 The bench, itemised

Table 408. A first bench, with what each item is for

Item	Order of cost	Note
FPGA board with the right I/O	\$200–\$3,000	Choose by the interface you must prove , not by logic capacity. A board without the connector is useless however large its fabric
Vendor toolchain	\$0 (free tier) – \$3,000	The free tier covers small and mid parts; check your part is in it before buying the board
Logic analyser, on-chip	\$0 — bundled	The single most valuable debug tool ; budget the block RAM it eats
USB protocol analyser / scope	\$300–\$5,000	Only if the interface is external and slow enough to see
High-speed scope or BERT	\$20,000+ or rented	Rent by the day, or use the far end's own error counters

Item	Order of cost	Note
Partner hardware to interoperate with	varies	Often the largest real cost and the highest value
Bench supplies, cables, adapters	\$200–\$800	Underestimated

Ranges, not quotations. The point is the shape: the board and the partner hardware matter and the instruments mostly do not, because modern high-speed links carry their own error counters and a rented instrument answers the rest.

Worked example 58. What the prototype has to demonstrate, in order

Given. Part Y7's bring-up sequence, applied to an IP block on an FPGA.

Method. Each step is a claim you will make to a customer. Do them in dependency order and stop at the first that fails.

1. Bitstream loads, clock runs	the board and the build
2. ID register reads the right constant	power, clock, reset, bus, decode — all in one read
3. Scratch register writes and reads back	the write path
4. Internal loopback passes traffic	the datapath, no external risk
5. One external transaction	debuggable; the channel now included
6. Line-rate traffic for an hour	buffering, back pressure, rate matching
7. Overnight with error counters read	the number you quote
8. Against a second vendor's part	interoperability — the claim worth the most

How this is got wrong. By starting at step 6. Running traffic before the ID register reads is how a week disappears: every failure has a dozen candidate causes and none of the cheap tests has been done. Part Y7's information argument applies exactly — each early step splits the hypothesis space evenly and costs minutes, and skipping them does not save the minutes, it spends days.

27.3 Instrumenting for a bench you will be alone at

Table 409. What to build into the RTL before the board arrives

Feature	Cost	What it saves
ID register with a known constant	a dozen gates	Six hypotheses in one read
Scratch register	32 flops	Separates the write path from the function
Internal loopback	a multiplexer	Splits 'inside the chip' from 'outside' — the one clean partition
Per-stage error counters, saturating	a counter each	Tells you <i>which</i> stage, not just that something failed
Snapshot register on first error	a register set and a sticky bit	The first failure is the debuggable one; the thousandth is not
State register readable	wires	A stuck state machine names itself
Pattern generator and checker	a small LFSR pair	Removes the host from the loop — tests at line rate with no software
On-chip logic analyser trigger outputs	a few wires	Lets the bundled analyser trigger on your condition, not on time

Graduate · **All eight are cheap, and all eight must exist before the board arrives.** Adding instrumentation during bring-up means a rebuild, and a rebuild on a large FPGA is an hour you spend staring at a progress bar while the failure is still unexplained. The habit worth forming is to write the debug features in the same sitting as the datapath they observe, when you still remember what could go wrong with it. **They are also a sales feature:** a customer bringing your block up in their chip has exactly the same problem you did, and a block that names its own failures is materially cheaper for them to integrate — which is the argument Part Y5 makes about support cost, arriving as concrete RTL.

Problem. You have no partner hardware and no budget for it. Can you still claim interoperability?

Solution. No, and saying so is better than the alternatives. What you *can* claim, precisely, is conformance: ‘passes the standard’s test vectors’, ‘matches the reference implementation bit for bit on n random cases’, ‘runs at line rate against an open-source implementation of the far end’. That last one is the most useful and the most overlooked — for most protocols in this book an open implementation exists, and running against it is a genuine second implementation even though it is not a product. **Then say what is untested and what it would take.** A customer who reads ‘interoperability not yet demonstrated; we have run against X and the gap is a plugfest’ can decide whether to lend you hardware, which happens more often than a first-time vendor expects, or to treat it as a risk they price. **A customer who reads an interoperability claim that turns out to mean ‘against ourselves’ stops reading the rest of the datasheet.**

Z8. The Money

Part Y5 covers licensing structures. This part is narrower and more uncomfortable: what a one-person house's first three years look like as numbers, where the money actually goes, and which of the failure modes are arithmetic rather than bad luck.

Z8.1 The cost base

Table 410. Where the money goes in year one

Item	Order of cost	Note
Your living cost	\$40k–\$120k / yr	The dominant term. Everything else is noise beside it
Standards membership and documents	\$0–\$50k / yr	Zero for open standards, five figures for some bodies — and this is a gate, not a preference (Part Z5)
EDA tools	\$0–\$50k / yr	\$0 is viable for soft IP (Part Y14); the line appears only if you must sign off
Compute	\$0–\$3k / yr	Genuinely small. The agent runs on one machine overnight
FPGA bench	\$500–\$5k one-off	Part Z7
Legal (contract templates, one review)	\$2k–\$10k one-off	Do not skip the indemnity review — Part Y5
Accounting, insurance, entity	\$1k–\$5k / yr	Jurisdiction dependent
Conferences and plugfests	\$2k–\$10k / yr	The highest-return line per dollar for a house nobody has heard of

Ranges, not quotations — they vary by country, body and year, and this repository has not obtained current price lists. The shape is what transfers: your time is the cost base and almost everything else is optional or small, with the two exceptions in bold.

Graduate · Read the tools line against the standards line. The received wisdom is that EDA licences are what keep individuals out of chip design. For *soft IP* that is no longer true — Part Y14 measures a complete model-to-synthesis flow at zero — and the barrier has moved to a less discussed place: **the specification**. A protocol block you cannot legally read the standard for cannot be built at any price, and some bodies' membership is a five-figure annual commitment before a line of RTL exists. **That is the real gate, and it is why Part Z5 weights 'specification obtainable' at 3** and why the post-quantum candidate scores so well: NIST published everything.

Z8.2 Three years, three scenarios

Table 411. Three years at a \$120k/yr cost base, revenue in thousands

Path	Year 1	Year 2	Year 3	Cumulative	Verdict
서비스만 (계약 검증·모델링)	90	130	150	10	생존
서비스 + 블록 하나	70	140	200	50	생존
블록만, 고객 없이 시작	0	80	240	-40	못 버틴다
블록 가족 (둘째가 싸다)	60	200	340	240	아슬

Illustrative numbers, computed from the stated assumptions — not a forecast and not anyone's actual results. The purpose is the ranking, which is robust to changing the inputs: the row that starts with no revenue is the row that does not survive, and it is the row most first-time houses choose because it is the most technically satisfying.

Worked example 59. How long the runway has to be

Given. You have savings S , a cost base of \$120k/yr, and the first licence revenue lands in month M .

Method. Compute the required savings for several values of M , and note what determines M .

Cost per month	10,000 USD
First revenue at month 12	120,000 USD needed
At month 18	180,000 USD
At month 24	240,000 USD
Part Y8's block takes	39 weeks $\approx$ 9 months to build
Then evaluation, negotiation, integration	3–9 months more before money moves
So realistic M without services	18–24
With services from month 1	M does not matter

How this is got wrong. By budgeting the build and not the sale. The nine months are the part you control and the least of it; a customer's evaluation, legal review and purchase cycle is measured in quarters and does not start until the block exists. **That gap is why Part Y8 recommends services first**, and the recommendation is not about risk appetite — it is that the alternative requires two years of savings, and the failure mode is running out three weeks before the first sale, at which point the asset is worth nothing to you.

28.3 Pricing the first block

Table 412. Four ways to set the first price, and what each signals

Approach	Number	What the customer reads
Cost plus margin	Part Y5's arithmetic	Nothing — they cannot see your cost
Fraction of their alternative	What it would cost them to build it	The only argument that lands. Their engineer-year is the comparison
Market comparable	What incumbents charge	Credible if you can name the comparables; weak if you cannot
Below cost for a reference	Low, explicitly one-off	Rational once , and only with a written reference commitment in exchange

Do not discount to win the first deal without getting something back for it. The discount is defensible — a first customer takes a real risk on an unproven supplier and a public reference is worth more to you than the margin. What is not defensible is giving it away quietly: the price becomes the anchor for every subsequent negotiation, the customer's procurement remembers it, and you have bought nothing. **Trade it explicitly:** a named reference, a quotable result, an introduction, or a commitment to the second block in the family at full price. Write the exchange into the contract, because a verbal promise of a reference does not survive a change of contact.

Problem. A customer asks for source code escrow, unlimited indemnity, and a five-year support commitment. Which do you agree to?

Solution. Escrow: yes, and budget the annual fee. It is a reasonable request from anyone betting a product on a one-person supplier, and refusing it confirms exactly the risk they are worried about. Keep the deposit current — a stale escrow is worse than none, because it creates false comfort. **Unlimited indemnity: no,** and this is the one to hold. Cap it at fees received, which is standard and which a competent counterparty will accept; uncapped IP indemnity can exceed the value of everything you own, and no price compensates for that because the loss is unbounded. If they will not move, the deal is not worth doing at any price. **Five-year support: yes, with the scope defined.** Define what an update is — bug fixes in the delivered version, yes; a new revision of the standard, no, that is a new product — define a response time you can actually meet alone including holidays, and price the years beyond the first separately. **The general rule is that the ones to resist are the unbounded ones,** not the expensive ones; expensive is a number you can put in the quote, unbounded is not.

Z9. Scaling the Agent to a Block That Is Actually Big

The blocks this book's agent verifies are small — tens to hundreds of lines. A sellable block is thousands. This part is about what changes, because several things that work at the small scale stop working, and knowing which ones in advance is the difference between a method and a demonstration.

Z9.1 What breaks as the block grows

Table 413. Each mechanism at both scales

Mechanism	At 30 lines	At 3000 lines	Holds?	What replaces it
Regression runtime	0.03 s	minutes	Fine — it runs at night	Use Verilator for the bulk run; keep Icarus for X-propagation
Mutation pass	15 s	hours to days	Breaks — mutants × seeds × runtime is a triple product	Sample the mutants; run the full set weekly
Repair search	8 candidates	thousands	Breaks — and most edits are irrelevant to the failure	Restrict candidates to the cone of the failing output
Minimal reproducer	one stimulus line	a sequence	Breaks — shrinking a sequence is a different algorithm	Delta-debugging over the transaction sequence
Golden model runtime	micro-seconds	comparable to the RTL	Can break	Model in C++ and link it, or compare at checkpoints
Stimulus generation	random in a loop	constrained, layered	Breaks — random reaches nothing interesting	Constraint solving, sequences, and directed corners
Distinct-output check	trivially satisfied	still trivially satisfied	Holds — but becomes weak	Replace with functional coverage bins

Three of seven break, and they break for the same reason: the work is a product of several quantities that each grow with the design. The replacements are all standard verification practice — the agent's structure does not change, only the components inside it.

Worked example 60. When the mutation pass stops fitting in a night

Given. Mutants scale roughly with lines; each mutant is run on k seeds; each run takes time proportional to the design.

Method. Compute the triple product at three sizes and find where twelve hours runs out.

30 lines: mutants × seeds × run	$8 \times 1 \times 0.03 \text{ s} = 0.24 \text{ s}$
300 lines	$80 \times 2 \times 0.3 \text{ s} = 48 \text{ s}$
3000 lines	$800 \times 2 \times 3 \text{ s} = 1.333 \text{ h}$
30 000 lines	$8000 \times 2 \times 30 \text{ s} = 133.3 \text{ h}$
Hours available per night	12
Where it stops fitting	around 3000–10 000 lines
The fix	sample mutants nightly, full set weekly
Sampling 10 % nightly at 30k lines	13.33 h

How this is got wrong. By treating the mutation score as something run once at release. Sampled nightly it is a *trend*, and the trend is what detects Part Z6's first failure mode — the oracle standing still while the design grows. A single number at release cannot detect that, and by then it is too late to act on it anyway.

Z9.2 Shrinking a sequence, not a value

Delta debugging, and why it is worth implementing

	Step	Justification
1	A sequential failure is a <i>sequence</i> of transactions, not one input.	<i>So the current shrink — replay the one failing stimulus — does not apply.</i>
2	Naive minimisation removes one transaction at a time and re-runs: $O(n^2)$ runs for length n .	<i>At $n = 10\,000$ and one second a run, that is weeks.</i>
3	Delta debugging halves instead: try the first half, then the second, then quarters, keeping any subset that still fails.	<i>$O(n \log n)$ in the worst case and far better in practice, because failures usually depend on a few transactions.</i>
4	The result is a <i>1-minimal</i> sequence: removing any single transaction makes it pass.	<i>Which is usually two or three transactions out of ten thousand, and that is a debuggable report.</i>
5	It needs only a predicate: does this sequence still fail?	<i>Which the harness already provides. That is why it is cheap to add — the expensive part, the oracle, already exists.</i>

Table 414. Shrinking a failing sequence: one-at-a-time against delta debugging

Transactions	Naive runs	Delta-debug runs (bound)	Speed-up	Delta-debug time
100	10,000	1,328	7.526	0.3691 h at 1 s/run
1,000	1,000,000	19,931	50.17	5.537 h at 1 s/run
10,000	100,000,000	265,754	376.3	73.82 h at 1 s/run
100,000	10,000,000,000	3,321,928	3010	922.8 h at 1 s/run

Worst-case bounds; in practice delta debugging does far better because it finds the small dependent subset early. The naive column is why nobody shrinks long sequences by hand, and the third column is why the agent should.

Z9.3 Constrained stimulus: where random stops working

Table 415. Stimulus layers, and what each reaches

Layer	Reaches	Cost	When you need it
Uniform random	The bulk of the state space	free	Always — the first 90 % of coverage (Part X8)
Weighted random	Rare values, boundaries	A distribution per field	As soon as any field has a rare-but-legal value
Constrained random	Only legal combinations	A solver	When illegal stimulus wastes the night
Sequences	Protocol states that need a history	A sequence library	The moment the block has state — which is immediately
Directed corners	What random never reaches	One test each; you write them	Part X8's argument: some bins have probability 10^{-9}
Replay of past failures	Everything previously found	Almost nothing — keep the minimal reproducers	Every night. This is the cheapest coverage in existence

Graduate · **The last row is the one to build first, and it is routinely skipped.** Every failure the agent has ever found, reduced to its minimal reproducer, is a test that costs nothing to re-run and that is known to have caught a real bug once. A growing file of them is a regression suite assembled for free from work already done, and it is the only part of the stimulus that is guaranteed to be relevant to *this* design rather than to a designer's idea of it. **This repository's agent writes the minimal reproducer into its ledger; turning that ledger into a replay set is a few lines and should be the first thing added when a block gets big.**

Problem. At what size should you stop using one agent and split the block?

Solution. The question is better posed as: at what size does the *oracle* stop being a single comparison? The agent's structure is indifferent to size; what fails is the golden model's granularity. While one model consumes the block's inputs and produces its outputs, one comparison suffices however large the block is. **The break comes when a failure's distance from its cause becomes too large** — an error injected in a pipeline's first stage that manifests as a wrong packet ten thousand cycles later, where the minimal reproducer is still ten thousand transactions and the repair search has no cone to restrict to. At that point split the *comparison*, not the block: add internal checkpoints where the model and the RTL can be compared — after the syndrome stage, after the key equation, after Chien — each with its own small oracle. **This is step-and-compare from Part X19 applied to a datapath**, and it converts one hard debugging problem into several easy ones. The cost is that the internal interfaces become part of the verification contract and cannot be changed freely, which is a real constraint and is usually worth it.

Z10. The RS-KP4 Decoder, Planned Block by Block

Part Y15 states where this project stands. This part is the plan that would finish it: each sub-block, what its golden model is, how the agent verifies it, what its gate looks like, and how long it takes. It is written so that it can be executed rather than admired.

Z10.1 The decomposition

Table 416. Sub-blocks, each with its own oracle and its own gate

Sub-block	What it is	Golden model	State	Effort	Note
Syndrome	30 accumulators, 64 symbols/cycle	<code>rs.신드롬()</code>	Exists as a one-step block ; widen it	3 weeks	Dominates area — Part Y15's count
Key equation (BM)	Serial, $2t$ iterations	<code>rs._BM()</code>	New	4 weeks	Small area, fixed latency
Chien search	Evaluate σ at 544 positions, 64/cycle	<code>rs._치엔()</code>	New	3 weeks	Second-largest block
Forney	Magnitudes from σ and ω	<code>rs._포니()</code>	New	2 weeks	Needs a field inverse — table or Itoh–Tsujii
Delay FIFO	One codeword in flight	trivial	New	1 week	5,440 bits — the block's main memory
Re-check after correction	Recompute syndromes on the corrected word	<code>rs.복호()</code> does this	New	1 week	Turns silent miscorrection into a flag — Part X5
Top level, AXI-Stream	Handshake, packet boundaries, back pressure	the Python decoder end to end	New	3 weeks	Part X37's rules

Every golden model already exists, in this repository's `rs.py`, verified by 13 tests including mutation guards. That is the unusual part of this plan and the reason it is credible: the hardest thing to get right is already done and already checked.

Graduate · **Total 17 weeks of RTL, against Part Y8's 39-week plan for a sellable block.** The difference is everything downstream: the verification environment, the release gate, the documentation, the packaging, the synthesis and timing runs, and the buffer. **The RTL is about a third**, which matches Part Y1's effort table for MIPI and is the ratio a first-time vendor most consistently gets wrong.

Z10.2 How each sub-block is verified, concretely

Each sub-block becomes an agent block in exactly the form Part Z1 describes: a DUT file, a testbench, and a `block.py` whose `자극()` drives it and whose golden values come from `rs.py`. The pattern is already demonstrated by `blocks/rs_syndrome`.

Table 417. The verification plan per sub-block

Sub-block	Stimulus that must be in the set	Why
Syn-drome	Zero errors; 1 error; exactly $t=15$; $t+1$; all-zero codeword; all-ones	Zero errors must not pass as success — <code>rs.측정()</code> refuses to report a pass from a zero-error trial
Key equation	Every error count $0 \dots t+2$; degenerate syndromes	The iteration count is data independent, so the corners are about degree, not timing

Sub-block	Stimulus that must be in the set	Why
Chien	Roots at position 0, at 543, adjacent roots, repeated roots	Repeated roots mean the locator is wrong — the decoder must flag, not correct
Forney	Error magnitudes at the field's extremes; a zero denominator	Part X5's decoder returns failure on a zero denominator; the RTL must too
Top level	Back-to-back codewords with no gap; a stall in the middle of a codeword; reset mid-codeword	Part X37 and Part X42 — the integration failures

Worked example 61. What the agent can check that a human review cannot

Given. RS(544,514) over GF(2¹⁰), $t=15$. The decoder must correct every pattern of up to 15 symbol errors and must not silently miscorrect beyond.

Method. Count the patterns. Then note which of the two properties is checkable by sampling and which is not.

Error positions of weight 15	$C(544,15) \approx 10^{29}$
Magnitudes per position	1,023
Total patterns	astronomically many
What sampling establishes	a bound on the failure rate, not correctness
What the algebra guarantees	all patterns of weight $\leq t$, by the code's minimum distance
So the RTL must be checked against	the model, not the space
And the model against	the algebra — which tests/test_rs.py does by property, not by sample

How this is got wrong. By trying to verify the RTL directly against the specification's guarantee. The guarantee is over a space nobody can enumerate; what is checkable is agreement with a model whose own correctness rests on properties that *are* checkable. **This two-level structure is why the golden model must be verified by properties rather than by vectors** — and it is exactly what this repository's test_rs.py does when it breaks the syndrome computation and asserts the measurement notices.

Z10.3 The three unconfirmed constants, and the plan around them

Table 418. What is not known, and what it blocks

Unknown	Fixed by	Blocks	Work that proceeds anyway
Field polynomial	IEEE 802.3 Clause 91	Interoperability — not correctness	Everything. It is a parameter; the RTL takes it as one
First consecutive root fcr	Same clause	Interoperability	Same — a parameter
Symbol bit order	Same clause	The byte/symbol packing at the boundary	The core; only the top-level packer depends on it

Parameterising is not a substitute for reading the clause, and the plan should not pretend otherwise. A parameterised decoder is correct for whatever parameters it is given and interoperable with nothing until the parameters are right. What parameterising buys is that *the unknown stays visible* — a hard-coded constant dissolves into an assumption nobody ques-

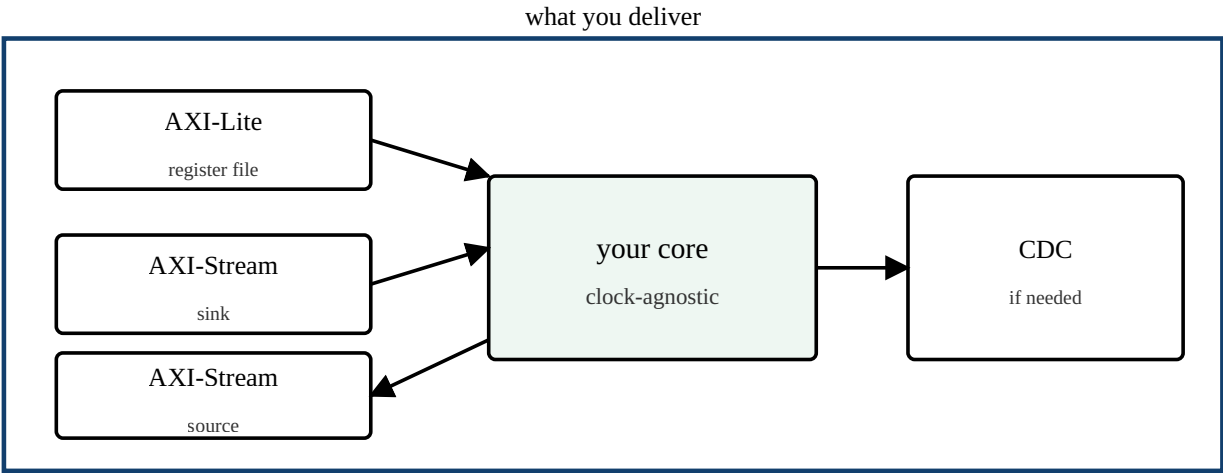
tions, and Part Y15's scorecard would quietly lose its most important row. **The first item in the plan remains: obtain the document.** Everything else is work that can proceed in parallel, and none of it is work that can substitute.

Problem. Sixteen weeks of RTL and the specification is still unobtainable. Do you build it anyway?

Solution. Build the parts that are parameter-independent and stop before the parts that are not, and be explicit with yourself about which is which. The field arithmetic, the Berlekamp–Massey iteration, the Chien search structure and the Forney computation are all *generic* — they are correct for any primitive polynomial and any first root, and building them is not speculative. The top-level packing, the interoperability claim and the datasheet are parameter-dependent and should not be written. **What you must not do is pick a plausible value and proceed as though it were confirmed**, because that converts a known unknown into an unknown one, and the failure surfaces at a plugfest in front of the people you most want to impress. **The strategic answer, though, is to reconsider the product.** Part Z5's scorecard weights specification availability at 3 precisely so that this situation is visible before sixteen weeks are spent, and its winner — a post-quantum core whose standard, reference implementation and test vectors are all public — exists because that row was weighted honestly. A block you cannot obtain the specification for is not a cheaper project; it is a different one.

Z11. Writing the AXI Wrapper a Customer Will Accept

Almost every block a design house sells is bought as an AXI peripheral, and almost every integration complaint is about the wrapper rather than the core. This part is the wrapper — what it must do, what it must not do, and the specific mistakes that cost a week at the customer's end.



The core knows nothing about AXI. The wrapper knows nothing about the algorithm.
That separation is what lets you sell the same core with three different wrappers.

Figure 52. The separation that makes a core sellable more than once.

Z11.1 Keep the protocol out of the core

Why the boundary pays for itself three times

	Step	Justification
1	A core that consumes and produces values, with a simple ready/valid or an enable, has no protocol in it.	<i>It can be tested against the golden model directly — Part Z1's arrangement.</i>
2	The wrapper translates AXI into that.	<i>And it is the same wrapper for every core you build.</i>
3	First payment: verification. The core is verified against the model; the wrapper is verified against the protocol.	<i>Two small problems instead of one large one, and the protocol half is reusable.</i>
4	Second payment: the customer. Some want AXI-Stream, some AXI4, some a bare FIFO, some APB for configuration only.	<i>A core with the protocol baked in has to be re-written for each.</i>
5	Third payment: the standard changes and the core does not.	<i>AMBA revisions, a move to CHI, a customer's proprietary fabric — all wrapper work.</i>
6	The cost is one extra level of hierarchy and a handful of registers.	<i>Which Part X37's skid-buffer argument says you wanted anyway, because registering the boundary is what keeps your timing independent of their floor-plan.</i>

Table 419. What belongs on each side of the line

Belongs in the wrapper	Belongs in the core
AXI channels and their handshakes	The algorithm
Burst handling, ID management, ordering	Word lengths and pipelining
The register file (generated — Part Z4)	Nothing about registers — it takes parameters and strobes
Clock domain crossing, if the customer's is different	One clock; no assumptions about its rate
Error responses (SLVERR, DECERR)	An error <i>flag</i> , not a bus code
Back pressure and buffering	A stall input it honours — nothing more
Low-power and isolation (Part X43)	A clock enable

Z11.2 The wrapper mistakes that cost a week

Table 420. Eight wrapper defects, what each looks like, and the fix

Mistake	Symptom	Fix
s_ready depends combinationally on s_valid	Deadlock or oscillation when the partner does the same	Register one side; a skid buffer costs two entries and one cycle
No skid buffer, both directions registered	Throughput halves and functional tests still pass	Part X37's table — measure transfers per cycle, not correctness
Read and write share an internal queue	Deadlock in the customer's system, never in yours	Part X11 — keep the AXI channels independent all the way in
Bursts allowed to cross a 4 kB boundary	Protocol violation the interconnect may not check	Split in the wrapper; it is a comparison and an adder
WSTRB ignored	Neighbouring bytes corrupted on a narrow write	Honour it, or declare the register file 32-bit-only in the guide
Write response returned before the write is visible	A driver's read-after-write returns stale data	Define and document the visibility point
Reset not synchronised per domain	One boot in a thousand (Part X42)	Two flops per domain; the constraint shipped with the block
Register map hand-written into the wrapper	Diverges from the header and the data-sheet	Generate it — Part Z4

Six of the eight pass a functional test. They are found at integration, at line rate, or in the customer's system — which is why the release gate of Part Z3 checks throughput and why the testbench must apply randomised back pressure rather than accepting everything.

Worked example 62. Proving the wrapper does not halve your throughput

Given. An AXI-Stream wrapper around a core that accepts one word per cycle. The customer will measure sustained throughput with back pressure applied at random.

Method. State the bound, measure it, and put the measurement in the datasheet — the same three steps the agent's throughput check performs.

Ideal	1 word/cycle
With random back pressure at 50 %	0.5 word/cycle — expected
With a missing skid buffer	0.25 — and every value is still correct
What a value-only regression says	pass
What the throughput check says	cycles > bound → red
Where the bound comes from	measured on the clean block over several seeds
This repository's example	crc32_stream: 2.18 clean vs 3.16 broken cycles/byte, bound 2.6

How this is got wrong. By declaring a bound from the architecture rather than from a measurement. The architectural ideal is one word per cycle and no real testbench achieves it, so a bound set from the ideal never fires and a bound set by guessing is either useless or a false alarm. **Measure the clean block on several seeds, check the spread is small, and set the bound just above it** — which in this repository's case caught a mutation that a value-only suite had missed.

Z11.3 What ships with the wrapper

Table 421. The wrapper's own deliverables

Item	Why the customer needs it
The SDC constraints for the boundary	Your timing claim is meaningless without them — Part X14
CDC constraints, if the block crosses domains	Their tool will flag the crossing and stop; the waiver must come from you
Protocol assertions (SVA) on the AXI interfaces	They can bind them in their environment and see your block behaving
A bus functional model or a trivial testbench	So they can run it in ten minutes, which is when they decide
The register map in IP-XACT	Automated integration
A reference integration: your block plus a stub master	Removes the whole class of 'it does not work' reports where the wiring was wrong

Graduate · **The reference integration is the item with the best return and the one most often left out.** It is an hour's work: your block, a minimal master that writes the registers and streams some data, and a script that runs it. What it removes is the first two days of every customer's evaluation, during which they are wiring your block up for the first time and every problem looks like your problem. **A customer who runs your reference in ten minutes and sees traffic move has formed an opinion about your competence before reading the datasheet**, and that opinion is expensive to change later in either direction.

Problem. The customer's interconnect is CHI, not AXI. How much work is that?

Solution. Ask first whether it has to be, because the answer is often no. Most CHI systems provide AXI bridges for peripherals, and a block that does not need coherence has no reason to speak CHI — so the first move is to establish whether the block is a coherent agent or a device behind a bridge, and for a datapath accelerator it is almost always the latter. **If it genuinely must be a coherent agent**, the work is substantial and is *not* a wrapper change: CHI is a coherence protocol with a snoop interface, ordering rules and a state machine, and Part X28's verification argument applies — the state space is the kind that needs formal methods, and the effort is comparable to the core itself. **For a one-person house that is a reason to decline or to scope it as a separate project**, not a line item in a wrapper quote. The middle case worth naming: some blocks need only IO-coherence, where the block's accesses snoop the host caches but nothing snoops the block. **That is an interconnect feature, not a block feature**, and it costs you nothing — which is worth knowing so you do not quote for work the customer's fabric already does.

Z12. Writing the Documents

Documentation is where a one-person house is at its biggest disadvantage and, unusually, where it can be at its best: a document written by the person who made the decisions is better than one written by a technical writer three months later. This part is the structure, the rules, and what this repository's own documents look like.

Z12.1 Three documents, three readers

Table 422. Who reads each one, and what they are deciding

Document	Reader	The decision they are making	Fails when
Datasheet	An evaluator, before buying	Does this block do what we need, at the rate we need?	Numbers without conditions; claims without evidence
Integration guide	An engineer, after buying	How do I wire this in without calling them?	It documents the block and not the <i>integration</i>
Known issues	Both	Can I trust this supplier?	It is absent, or it says 'none'

Graduate · The known-issue list is read as a character reference and should be written as one. An evaluator has seen datasheets before and knows every block has limitations; what varies is whether the supplier says so. A list with eight dated entries, each with impact, workaround and status, says that someone has thought about the failure modes and is willing to be held to them. An absent list says either that nobody looked or that somebody decided not to tell you, and the evaluator cannot tell which — so they assume the worse one. **This is the cheapest credibility available to a supplier nobody has heard of,** and it costs an afternoon.

Z12.2 The rules that make the numbers usable

Table 423. Six rules, each with the version that fails it

Rule	Good	Bad	Why
Every number carries its conditions	'2.18 cycles/byte, random back pressure, seeds 1/7/99'	'1 byte per cycle'	A number without conditions cannot be reproduced or disputed
Name the tool, the library and the version	'208 cells, yosys 0.33, open library'	'~2 kGE'	Part Y14 — the honest smaller claim beats the unattributed larger one
Say what was not measured	'Area in your process: not measured. Scripts included'	Silence	Silence is read as a claim
Separate measured from derived	'measured 2.18; bound set at 2.6'	'2.6 cycles/byte'	A bound and a measurement are different facts
Give the reproduction command	bash edu/house/release.sh crc32_stream	—	This is what turns claims into evidence
Date everything	'2026-09-19, v1.0'	undated	An undated known-issue list is a list of excuses

All six are about the reader's ability to check you, which is the only thing that distinguishes a datasheet from marketing. A vendor who makes checking easy is making a bet that the numbers hold, and evaluators read that bet correctly.

Table 424. This repository's own documents, counted at build time

File	Reader	Lines	Table cells	Measured-value mentions	Links to the issue list
DATASHEET.md	evaluator	72	100	3	yes

File	Reader	Lines	Table cells	Measured-value mentions	Links to the issue list
INTEGRATION.md	integrator	62	0	0	—
KNOWN_ISSUES.md	both	27	80	0	yes

Counted by reading the files as this page rendered. The measured-value column is the one to watch: a datasheet whose count is zero is a list of intentions.

Z12.3 The datasheet, section by section

Table 425. What each section must answer

Section	The question	The commonest omission
1. What it is	Does this solve my problem?	Naming the standard but not the profile or the options
2. Interface	Can I connect it?	The handshake rules — which side may depend on which
3. Performance	Is it fast enough?	The conditions the number was measured under
4. Area and frequency	Does it fit?	Either silence, or an FPGA number presented as an ASIC one
5. Verification	Can I trust it?	Evidence that the tests bite — the mutation score
6. Configuration	Can I tune it?	Parameters listed without their legal ranges or their interactions
7. Known issues	What will bite me?	The whole section
8. Deliverables	What do I get?	Whether the testbench and scripts are included

Worked example 63. How long the documents take if written weekly

Given. A 39-week block. Documentation written at the end, versus twenty minutes at the end of each week.

Method. Compare the totals, and then the quality, which is the part that does not appear in the total.

Written at the end	1–2 weeks solid
Written weekly	13 hours total
Ratio	about 3× cheaper
And the quality difference	weekly notes record <i>why</i>; end-of-project notes record <i>what</i>
What is lost by waiting	the reason a constant has its value, the corner that forced a structure, the thing that nearly went wrong
Who needs the why	the customer's integrator, and you in a year

How this is got wrong. By treating documentation as transcription. It is not — it is the record of decisions, and decisions are only recoverable while they are fresh. The three-times cost difference is real and is the smaller of the two effects; the larger one is that a document written at the end cannot contain the information that makes it worth reading. **Part Z2 puts this in the Friday slot for exactly this reason**, and it is the slot that disappears first.

Problem. Your block has 40 configuration parameters. How do you document them without writing 40 pages?

Solution. Generate the list and write prose only about the interactions, because the list is mechanical and the interactions are not. **Generated:** name, type, legal range, default, and what it affects — from the same source that produces the RTL parameters, exactly as Part Z4's register map is generated. That is a table nobody writes and nobody can get out of sync. **Written by hand:** the three or four *combinations* that matter — which parameters must move together, which ones are ignored when another is set, which ones change the area or the latency and by roughly how much. **And a short list of the two or three configurations you have actually verified and synthesised**, named, with their numbers. That last item is the honest core of it: 40 independent parameters is 2^{40} configurations and you have verified a handful, so say which handful and state the covering argument for the rest (Part X19's pairwise criterion is the standard one). **A customer choosing an unverified combination should know that is what they are doing**, and a supplier who tells them is worth more than one who implies all 2^{40} were tested.

Z13. Week One

This book is long and the first Monday is short. This part is what to do in the first five days if the aim is a two-member team producing sellable IP, in an order that leaves something working at the end of each one. Everything it names exists in this repository and can be copied.

Z13.1 The five days

Table 426. Week one

Day	Goal	What you do	Why this and not something else
Monday	Get the flow running	Install iverilog, verilator, yosys, Python. Clone this repository. Run <code>bash edu/house/release.sh gf_mul</code> and watch it pass.	You now have a working gate on somebody else's block , which is the fastest way to see what one looks like
Tuesday	Make it your block	Copy <code>edu/agent/blocks/gf_mul</code> . Replace the RTL with the smallest thing you actually want to build. Write <code>block.py</code> 's <code>자극()</code> against a golden model you did not write.	The golden model must be independent — a library, a reference implementation, the standard's vectors
Wednesday	Make the checks bite	Run <code>mutscore.py</code> . Read every escape. Fix the stimulus until the score is above 90 % with every remaining escape judged in writing.	This is the day that teaches the most , and it is the day this repository's register file went from 58 % to 100 % and surfaced two real bugs
Thursday	Declare a bound	Measure the clean block's cycles over several seeds. Declare <code>성능한계()</code> just above the spread. Break the block so it is only slower and confirm the gate catches it.	A value-only suite cannot see a block getting slower
Friday	Write the three documents and run the gate	Datasheet, integration guide, known issues — short, with the numbers you measured this week and their conditions. Then <code>release.sh</code> and read what it says.	It will say you cannot sell it yet, and it will say exactly why

No architecture, no algorithm, no big block. The week builds the apparatus; the apparatus is what makes every later week cheap. A week spent the other way round produces a nicer block and no way to know whether it works.

The instinct is to start with the interesting block. It is the reason to be doing this at all, and it is the wrong first week: a large block with no gate is a large block you cannot make claims about, and the gate is far easier to build against something trivial. **The 30-line GF multiplier in this repository exists for exactly that reason** — it is small enough that every part of the apparatus can be seen working, and the apparatus transfers unchanged to a block a hundred times larger.

Z13.2 What each day leaves behind

Table 427. The artefacts, and what each proves

After	You have	You can say
Monday	A green gate on a known-good block	'my flow works' — which you could not say before
Tuesday	Your RTL compared against an independent model	'it agrees with X on n cases'
Wednesday	A mutation score with judged escapes	'the suite catches k % of injected defects, and here are the ones it does not and why'
Thursday	A measured throughput bound that fires	'it sustains x , and the gate proves it has not regressed'

After	You have	You can say
Friday	Three documents and a gate report	'here is what is done, here is what is not'

Worked example 64. What the first week costs and what it saves

Given. Five days building apparatus instead of five days building the block.

Method. Count what the apparatus does over the following months, using this book's own measurements.

Cost	5 days
Nightly regression thereafter	free
Bugs the mutation score surfaced here in one afternoon	2
Where those bugs would otherwise have been found	the customer's driver
Cost of a bug found by a customer (Part X8)	$10^3-10^4 \times$ a specification-stage fix
Break-even	the first real bug
Additional return	every block after this one starts on Tuesday

How this is got wrong. By counting the week as lost time. It is the only week whose output is reusable without modification across every block the house will ever build, and the ratio in the fifth row means it pays for itself the first time the gate catches something. **The honest caution is that it does not *feel* like progress**, which is why it is worth deciding in advance to spend it.

Z13.3 Where to go next in this book

Table 428. Reading order after week one

If you are	Read	Because
Choosing what to build	Part Z5, then Part Y6	Scorecard first, then the prior-art discipline that stops you rebuilding something
Writing the golden model	Parts X1 and Y11	Fixed point is most of the work and independence is the whole point
Choosing an architecture	Parts X2, X6, X13	Timing budget, memory-versus-arithmetic, the roofline
Stuck on verification	Parts X8, X33, Z6	Coverage arithmetic, where formal helps, what the agent cannot do
Getting it to close timing	Parts X2, X14, X34	The budget, the physical consequences, what synthesis can and cannot fix
Preparing to sell	Parts Z3, Z4, Y5, Z12	The bar, the pack, the commercial terms, the documents
Talking to a customer	Parts Y5, Y9, Z8	Scoping, compliance, money
Scaling past a toy	Part Z9	What breaks and what replaces it

Problem. You have read this book and you have a day job. Is any of this possible part-time?

Solution. The apparatus is, and the block is not — and separating the two is the useful answer rather than a discouraging one. **The apparatus is compatible with part-time work** because the agent runs when you are not there: an hour in the evening spent strengthening the oracle is repaid by a night of unattended checking, which is the one part of this arrangement that does not scale with your hours. Week one is five evenings rather than five days. **What is not compatible is the customer side:** evaluations, plugfests, support response times and the conversations of Part Z5 all happen during business hours and cannot be deferred, and Part Z8's arithmetic says the customer side is what decides whether the business exists. **So the honest part-time plan is to build the apparatus and one small, complete, publishable block** — which is a portfolio (Part Y13) and an asset and a genuine thing to point at — and to treat the transition to selling as a separate decision that needs the day job to end. **Conflating the two is how people spend two years part-time and arrive with neither.**

Z14. The C++ Model Layer — One Source, Two Destinations

Ask a working design house where its models live and you rarely hear “SystemVerilog.” You hear C or C++. The RTL is what you sell; the C++ is what tells you the RTL is right, and increasingly it is also what *generates* the RTL. This part builds that layer for real. Every file named here is in this repository, compiles with g++, and is exercised while this page renders.

Z14.1 Why the model is C++ and not Python

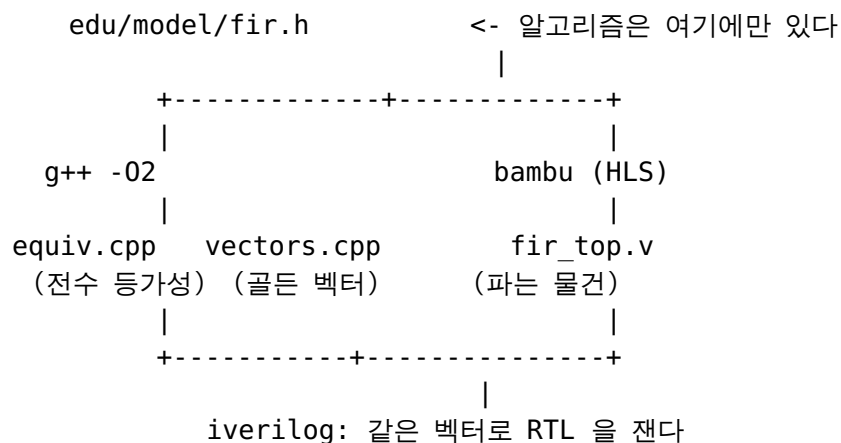
Earlier parts of this book used Python golden models (zlib for CRC-32, gf.py for the Galois field). That was right for those blocks: the model existed already, independently, which is the property that makes a golden model worth having. It stops being right the moment the model has to do two jobs:

Table 429. What the model layer has to do, and what language can do it

Job	Python	C++	Why
Be the verification reference	yes	yes	Both express the algorithm exactly.
Run billions of vectors	no	yes	Measured below: 144 M evaluations/s in C++. CPython is ~1000× slower.
Be the HLS input	no	yes	Every HLS tool in industry takes C, C++ or SystemC. None takes Python.
Be bit-accurate to the RTL	awkward	yes	Python ints are arbitrary precision. Overflow and truncation must be simulated by hand; in C++ the fixed-width types do it.

The third row is the one that changes the architecture of the whole flow. If the model is also the HLS input, then the model and the RTL *cannot disagree*, because there is only one description:

One source, two destinations



The conventional flow has three descriptions — a Python model, a C++ HLS source and hand-written RTL — and therefore three chances to disagree. When they disagree, nobody knows which is right, because each was written from the spec by a different person on a different day. The structure above removes two of the three.

Z14.2 The shared header, and why it has no vendor types

edu/model/ipmodel.h declares the fixed-point contract. It deliberately uses `int32_t` and hand-written saturation rather than `ap_fixed` or `hls::stream`. That was not a style choice; it was measured. Part Z15 gives the four one-variable controls that forced it.

```
// C++ 에서 음수 우시프트는 C++20 이전에는 구현정의지만, gcc·clang 은
// 산술 시프트로 하고 Bambu 도 같다 -- 그래도 기대는 것이 싫어서
// **나눗셈으로 적지 않는다**: 나눗셈은 0 쪽으로 자르고 시프트는 -무한 쪽으로
// 자른다. 둘이 음수에서 다르다. RTL 은 시프트를 하므로 모델도 시프트다.
static inline int32_t asr(int32_t v, int n) {
    return (v < 0) ? ~(~v >> n) : (v >> n);    // 부호와 무관하게 floor 시프트
}

// n비트 부호있는 범위로 포화. 자르지(wrap) 않고 **포화**하는 이유:
// 링크 등화기에서 wrap 은 큰 양수를 큰 음수로 뒤집어 슬라이서를 정반대로
// 틀리게 만든다. 포화는 틀려도 같은 부호 쪽으로 틀린다.
static inline int32_t sat(int32_t v, int n) {
    const int32_t hi = (int32_t)((1u << (n - 1)) - 1);
    const int32_t lo = -hi - 1;
    return v > hi ? hi : (v < lo ? lo : v);
}

// 아래 n비트만 남긴다 (부호 없음). RTL 의 `[n-1:0]` 과 같다.
static inline uint32_t mask(uint32_t v, int n) {
    return n >= 32 ? v : (v & ((1u << n) - 1));
}
```

Two details in that code are worth the ink, because both are places where a model and an RTL silently part company.

Why the shift is written as a shift and not a division

	Step	Justification
1	The RTL computes <code>acc[16:7]</code> — an arithmetic right shift by 7.	<i>That is what the hardware does; a shift is free, a divider is not.</i>
2	In C, <code>acc / 128</code> rounds toward zero. <code>acc >> 7</code> rounds toward negative infinity.	<i>Integer division truncates toward zero by definition (C99 6.5.5); a shift discards low bits, which floors.</i>
3	For <code>acc = -1</code> : division gives 0, shift gives <code>-1</code> .	<i>$-1/128 = -0.0078$ truncates to 0; $-1 >> 7 = -1$.</i>
4	So writing the model with division makes it disagree with the RTL on every negative value whose magnitude is below the shift amount.	<i>That is roughly half of all small negative accumulator values — common in an equaliser sitting near zero.</i>
5	Therefore <code>asr()</code> is written as a shift, with the sign handled explicitly so it does not rely on implementation-defined behaviour.	<i>C++ left negative right-shift implementation-defined until C++20.</i>

The second is saturation versus wrapping. A wrap turns a large positive accumulator into a large negative one, which drives the slicer to exactly the wrong decision; a saturate is still wrong but wrong in the same direction. The model saturates because

the RTL saturates, and the RTL saturates because the specification says so. All three have to agree or the comparison is theatre.

Z14.3 Proving three architectures equal — by exhaustion

`fir.h` contains the same filter written three ways: the textbook form, a width-narrowed form, and a symmetry-folded form that uses two multipliers instead of four. They have to compute the same function. Most projects establish that by running a few thousand random vectors and declaring victory. Here the input space is four 8-bit samples, so it has exactly 4,294,967,296 points — and that is small enough to check *all of them*.

Table 430. Exhaustive equivalence, measured

Quantity	Value	What it means
Input space	4,294,967,296	256^4 — every possible input
Wall time	29.7 s	g++ -O2, one core
Rate	144,419,484/s	Each point evaluates all three variants
Disagreements	0	Not “none found” — none exist
Distinct outputs	256	All 256 output codes occur, so the check is not degenerate

That last row is the gate this repository puts on every measurement. A comparison that only ever sees one output value has proved nothing, however many vectors it ran. `equiv.cpp` counts distinct outputs and exits non-zero if there are fewer than two — the same rule the RTL harness applies in Part Z1.

The word “proof” is used carefully. This is exhaustive over the *stated* input space, which is the combinational function of four samples. It says nothing about the sequential wrapper, the reset behaviour, or the handshake. Those need different arguments — and in Part Z15 one of them costs us a bug.

Z14.4 What verification time actually looks like

“C++ simulates a thousand times faster than RTL” is repeated everywhere and is almost never measured by the person repeating it. Measuring it on this design produces a more useful answer than the folklore.

Table 431. Vectors per second, same algorithm, same machine

Path	Vectors/s	Relative	What is included
C++, compute only	144,419,484	1,616×	The arithmetic, nothing else (<code>equiv.cpp</code>)
C++, formatted to a file	5,110,455	57×	Same arithmetic plus <code>printf</code> of five integers
RTL, iverilog + vvp	89,345	1×	Event-driven simulation of the synthesisable design

Worked example 65. Where the speed advantage goes

Given. C++ computes at 144,419,484/s but emits formatted vectors at only 5,110,455/s.

Method. Divide. The ratio is the cost of text formatting relative to the algorithm itself.

Numbers. 28×. Formatting five integers as decimal text costs about 28 times more than computing the filter output.

The practical rule: a C++ reference that talks to the RTL through a text file throws away almost all of its speed advantage. Use the file for the few thousand vectors the RTL will actually run, and keep the billion-vector sweeps entirely inside C++.

How this is got wrong. The obvious reading is “file I/O is slow.” It was measured and it is not: writing 200,000 vectors to a file ran at 5,110,455/s and discarding them to `/dev/null` ran at 5,398,353/s — a 5 % difference. The cost is `printf`'s decimal conversion, not the disk. Replacing text with a binary buffer, or comparing in memory and never serialising at all, recovers the other 96 %.

So the honest version of the folklore is: the algorithm is about 1,600× faster in C++, and a badly plumbed harness will give you 57×. Both are worth having; they buy different things. The 57× path is how you drive the RTL. The 1,600× path is how you settle the algorithm before any RTL exists — and that is where the schedule is actually won, because a bug found in the C++ costs minutes and the same bug found in a gate-level regression costs days.

Z14.5 Order of work, and what it saves

Table 432. Where each class of defect should die

Defect class	Cheapest place to catch it	Cost if it escapes to RTL
Algorithm wrong (wrong formula, wrong rounding)	C++, exhaustive or bulk	Days — the RTL is right and the spec is wrong, which is the slowest kind of debug
Overflow / width	C++ with fixed-width types	Weeks — appears as “fails occasionally”
Handshake, reset, backpressure	RTL only	Cannot be found in C++; budget RTL time for exactly this
Timing / area	Synthesis and P&R only	Cannot be found in simulation at all

The two lower rows are why HLS does not remove the need for RTL verification. It removes the top two rows, which are most of the defects by count, and leaves the bottom two, which are most of the defects by *difficulty*. Part Z15 hits one of the third-row defects within ten minutes of first running the generated RTL.

Z15. HLS — What It Costs, and How to Make It Win

High-level synthesis has two well-known drawbacks. The first is that it does not always compile what you give it. The second is quality of results: the generated RTL can use noticeably more of the device than RTL a person would have written. Both are real, and both are measured in this part on a filter that exists in this repository.

But “generated RTL is worse than hand RTL” is not a law. It is what happens when the C++ is written as if it were software. If you understand the specification, the architecture you actually want, and what the HLS tool does with what you wrote, the generated RTL can beat hand-written RTL — and below it does, by 1.38× in clock frequency. The rest of this part is how.

Z15.1 Drawback one: what the tool refuses

Before any quality question there is a compile question. These are one-variable controls run against PandA Bambu 2024.10 in this container: each row changes exactly one thing from the row above, so the cause is not inferred.

Table 433. What Bambu accepts and refuses — one-variable controls

Control	What changed	Result
1	Pure C, 8-tap FIR	rc=0, Verilog produced, 101.0 MHz
2	Control 1 plus one line : <code>#pragma HLS PIPELINE II=1</code>	rc=11 — Loop pipelining pragma not supported
3	C++ templates, no pragmas	rc=0, Verilog produced
4a	<code>ap_uint</code> only	rc=0, Verilog, 119–147 MHz
4b	<code>hls::stream</code> only	rc=1 — crash in <code>FixStructsPassedByValue</code>
5	<code>ap_fixed</code> complex 8×8 Cholesky	rc=124 — 40-minute timeout in the front end

Control 2 is the important one: the same file, one line added, a hard failure. The blocker is not C++ and not the algorithm — it is the *pragma dialect*. Every vendor has its own, and pragmas are exactly the part of an HLS source that does not port. Control 4a is the pleasant surprise: Xilinx’s arbitrary-precision integer type compiles fine on a tool that has nothing to do with Xilinx, because it is ordinary C++ templates. Control 4b is the unpleasant one, and it has no workaround.

The design rule that falls out of this table, and the one `edu/model/ipmodel.h` follows: **put the algorithm in portable C++ and keep every tool-specific thing out of it**. No pragmas in the header, no vendor stream types, no heavy template arithmetic. Then the same header compiles under g++ as the golden model and under the HLS tool as the design, and retargeting to a different HLS tool is a build-script change rather than a rewrite.

Z15.2 Drawback two: quality of results, measured

The filter is four taps, 8-bit samples, constant symmetric coefficients. Written naturally in C++ — `int` parameters, `int` accumulator, the way anybody would write it — and handed to Bambu, it produces this:

Table 434. The same algorithm, the same tool, only the C types differ

HLS source	Port width	Cells	SB_LUT4	SB_CARRY
<code>int fir_top(int,int,int,int)</code>	32 bit	1,218	890	199
<code>signed char fir_top_tuned(signed char×4)</code>	8 bit	338	228	97

Worked example 66. Where a 4-tap filter's 890 LUTs came from

Given. A 4-tap FIR with constant coefficients needs, by hand, about 200 LUT4 on this device. The naive HLS version used 890.

Method. Compare the two rows above. The algorithm is identical — the same header, the same arithmetic, verified bit-identical. Only the C parameter types differ.

Numbers. 3.60× reduction (1,218 cells to 338) from changing `int` to `signed char` in the function signature. Nothing else changed.

This is the single largest QoR lever in HLS and it costs one line. It is also the first thing to check when generated RTL comes back four times too big.

How this is got wrong. The tempting explanation is “HLS tools are bloated.” It is not that. C's integer promotion rules widen `char` and `short` operands to `int` before arithmetic, so a C++ source that says `int` is *requesting* a 32-bit datapath and 32-bit ports. The tool built what the source asked for. A person writing Verilog types `[7:0]` and cannot make this mistake — which is most of why naive HLS loses to hand RTL.

Z15.3 Making the comparison honest

Three things had to be fixed before any number in the next table meant anything. Each of them, left unfixed, produces a confident and wrong result.

Table 435. Three ways this comparison was wrong before it was right

Problem	Symptom	Fix
The two designs computed different functions	None — area numbers looked fine and meant nothing	Verify both against the same C++ golden vectors first. Every design in the next table passes 400 vectors with 0 mismatches.
Reset polarity differed	The generated RTL never asserted done; 400/400 “failed”	Bambu defaults to active-low reset; the hand RTL used active-high. <code>--reset-level=high --reset-type=sync</code> .
Frequency was measured on the wrong path	Adding pipeline registers made Fmax <i>fall</i> (324 MHz → 74 MHz), and the hand RTL reported no frequency at all	<code>nextpnr</code> reports register-to-register paths only. Wrap every design in an identical registered timing shell (<code>fir_shell.v</code>) so the path being measured is the datapath.

The third deserves emphasis because it is the kind of error that survives review. The numbers before the fix were not noise — they were stable across seeds and they were monotonic in the wrong direction. A stable, reproducible, completely meaningless measurement is the most dangerous kind, because every instinct says to trust it. What made it visible was not a tool warning; it was noticing that *more registers cannot make a design slower*, and refusing to write the number down until that contradiction was explained.

Z15.4 The sweep — where HLS loses and where it wins

Table 436. Area, frequency and latency — all verified against the same C++ golden model (0 mismatches in 400 vectors each)

Design	LC	Fmax (MHz)	Latency (cyc)	Fmax vs hand pipe	Area vs hand pipe
Hand combinational	252	47.2	1	0.61×	0.98×
Hand 3-stage pipeline	256	76.9	3	1.00×	1.00×
HLS @ 10ns	299	57.2	2	0.74×	1.17×
HLS @ 6ns	307	72.7	4	0.95×	1.20×
HLS @ 3ns	386	105.9	9	1.38×	1.51×

Design	LC	Fmax (MHz)	Latency (cyc)	Fmax vs hand pipe	Area vs hand pipe
HLS @ 2ns	460	92.3	12	1.20×	1.80×
HLS @ 1ns	544	82.6	18	1.08×	2.12×

Frequency is the mean of three placer seeds; the spread was under 7 % on every row, so the differences below are larger than the noise. Device is ice40 HX8K, nextpnr-ice40 --freq 300 --placer heap.

Worked example 67. Does the generated RTL beat the hand-written RTL?

Given. Hand-written 3-stage pipeline: 256 LC at 76.9 MHz, latency 3. Best HLS point (HLS clock=3ns): 386 LC at 105.9 MHz, latency 9.

Method. Compare each axis separately. There is no single QoR number; area, frequency and latency trade against each other.

Numbers. Frequency: 1.38× **better**. Area: 1.51× worse. Latency: 3.0× worse.

So the answer is: yes on frequency, no on area and latency, and only at one point in the sweep. Anyone quoting the 1.38× without the 1.51× area and 3× latency is selling, not measuring.

How this is got wrong. Two trivial explanations had to be killed. First, that the hand RTL was a straw man: the comparison is against a hand-written *pipelined* design, not the single-cycle one, precisely so the answer is not rigged. Second, that tightening the constraint always helps: it does not. At 2 ns and 1 ns the generated design is both **larger and slower** than at 3 ns. The knee is at 3 ns and you only find it by sweeping.

Z15.5 The method — how to get better QoR than hand RTL

The 1.38× did not come from the tool being clever. It came from four things, in this order. Each is a thing you do, not a thing the tool does.

Why the generated RTL could win at all

	Step	Justification
1	Write the interface at the true width, not the C default.	<i>Measured above: 3.60× area from this alone. Until this is done, nothing else matters — the design is four times too big and every later measurement is taken on the wrong design.</i>
2	Express the architecture you want, not just the arithmetic you want.	<i>The folded form computes $h_0(x_0+x_3)+h_1(x_1+x_2)$: two multipliers instead of four. The tool will not discover the symmetry; you state it, and <code>equiv.cpp</code> proves you stated it correctly.</i>
3	Let the tool re-schedule, and sweep the constraint.	<i>One number, --clock-period, produced five different microarchitectures with latencies 2, 4, 9, 12 and 18 cycles. In hand RTL each of those is a re-design; here each is a rebuild.</i>
4	Pick the knee from the sweep rather than the tightest point.	<i>The best frequency is at 3 ns, not at 1 ns. Constraining harder past the knee adds registers that the fabric cannot exploit, so area rises and frequency falls.</i>

Step three is where the structural advantage lives, and it is worth being precise about what it is. The tool is not out-scheduling a human. A human given a week could hand-pipeline this filter to 3 ns and would probably land smaller than 386 LC. The advantage is that the sweep above cost **four rebuilds and no design work**, so five architectures were evaluated instead of one.

Hand RTL usually gets pipelined once, for one target, because re-pipelining is expensive and risky; it then stays that way when the target moves. The generated design is re-derived from the specification every time.

That is the real claim, and it is narrower and more useful than “HLS beats hand RTL.” **Hand RTL beats HLS at a point you have already designed for. HLS beats hand RTL at a point you have not.** In a one-person design house, where nobody has a week to re-pipeline a block because a customer moved the clock from 75 to 100 MHz, the second case is the one that keeps happening.

Z15.6 Where this result stops being true

Stating the conditions under which a measurement flips is not a disclaimer; it is part of the measurement. This one flips under any of these:

Table 437. Invalidation conditions for Z15.4

Condition	Why the result changes
A different fabric	ice40 has no DSP blocks usable here, so constant multiplies became LUT shift-adds. On a device with hard multipliers the area picture changes completely.
Wider data or more taps	At 4 taps the hand design is small enough for one person to hold in mind. The argument for HLS gets <i>stronger</i> with size, but this table does not prove that — it was not measured here.
A hand design with more than 3 stages	The comparison is against a 3-stage hand pipeline. A 6-stage hand pipeline would likely beat the 3 ns HLS point on both axes. What it would cost in engineer-days is not in the table.
A different HLS tool	Bambu's scheduler is not Vitis HLS's. The pragma dialect result in Z15.1 guarantees the sources are not portable unchanged.

The last row is the one to plan around commercially. An IP built on one HLS tool's pragmas is tied to that tool. An IP built the way `edu/model/ipmodel.h` is built — portable C++, tool-specific settings on the command line — is not, and that portability is worth more than the few percent a vendor pragma might buy.

Z16. Before Any Code — System, Spec, Schematic

Everything so far in this volume started from a piece of RTL or a piece of C++. That is the wrong end. In a real design house the code is the *fifth* thing you produce, and by the time you type it most of the decisions that matter are already made. This part walks the four things that come first, for the same 4-tap equaliser the previous parts built: where it sits in a system, what it is responsible for, what its specification says, and what it looks like as a drawing — all the way down to transistors.

Nothing here assumes prior hardware knowledge. If you have written C++ and never seen a schematic, this part and the two after it are written for you.

Z16.1 What problem this IP exists to solve

A serial link sends bits down a copper trace. At 10 Gb/s a bit is 100 picoseconds wide, and a 30 cm PCB trace behaves like a low-pass filter: high frequencies are attenuated more than low ones. The consequence has a name — **inter-symbol interference** (ISI). Energy from bit n is still arriving when bit $n+1$ is being sampled, so the receiver's decision about one bit depends on the bits around it.

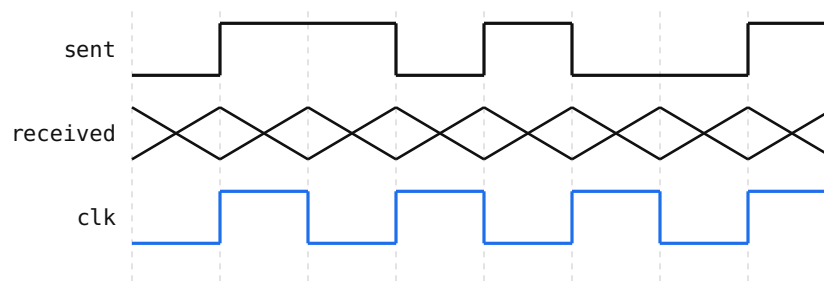


Figure 53. What the channel does. The transmitted waveform (top) is clean. The received waveform is smeared across bit boundaries — drawn as ‘unknown’ because at the sampling instant its value depends on neighbouring bits, not just the current one. Recovering the top row from the middle row is the equaliser's job.

An equaliser undoes that smearing. A **feed-forward equaliser** (FFE) does it with a filter whose response approximates the inverse of the channel: take the last few samples, scale each by a coefficient, add them up. That sum is the filter this book has been building. Four taps, 8-bit samples, coefficients $\{-18, 111, 111, -18\}$ in Q1.7 — a mild high-frequency boost, which is exactly the shape that undoes a mild low-pass channel.

Worked example 68. Why those coefficients have that shape

Given. The coefficients are $\{-18, 111, 111, -18\}$ in Q1.7, i.e. $\{-0.141, 0.867, 0.867, -0.141\}$.

Method. Sum them to get the DC (zero-frequency) gain. Alternate their signs and sum to get the Nyquist (highest-frequency) gain.

Numbers. DC gain = $(-18+111+111-18)/128 = 186/128 = \mathbf{1.453}$. Nyquist gain = $(-18-111+111+18)/128 = 0/128 = \mathbf{\text{exactly zero}}$. The filter passes DC at $1.45\times$ and has a perfect null at the highest representable frequency.

This is the difference between a teaching example and a product. The structure, widths, saturation, handshake and verification in this book are all real. The coefficient values are placeholders, and the book says so rather than letting the reader assume otherwise.

How this is got wrong. That is a **low-pass** response — and not a mild one: a null at Nyquist is what a smoothing filter does, the opposite of what an equaliser is supposed to do. The negative outer taps make it *look* like a boost, but the adjacent equal centre taps dominate and their sum $111-111$ cancels exactly at Nyquist. Treat this filter as what it is — a worked example with realistic structure, arithmetic and saturation behaviour — not as a coefficient set anyone should ship. Real FFE coefficients are adapted at run time by an LMS loop against the measured channel, and the sign pattern is typically the other way round.

Z16.2 Where it sits — the whole receive chain

The filter is one box in a chain. Knowing the boxes on either side is not background reading; it is what determines your interface, your latency budget and your clock.

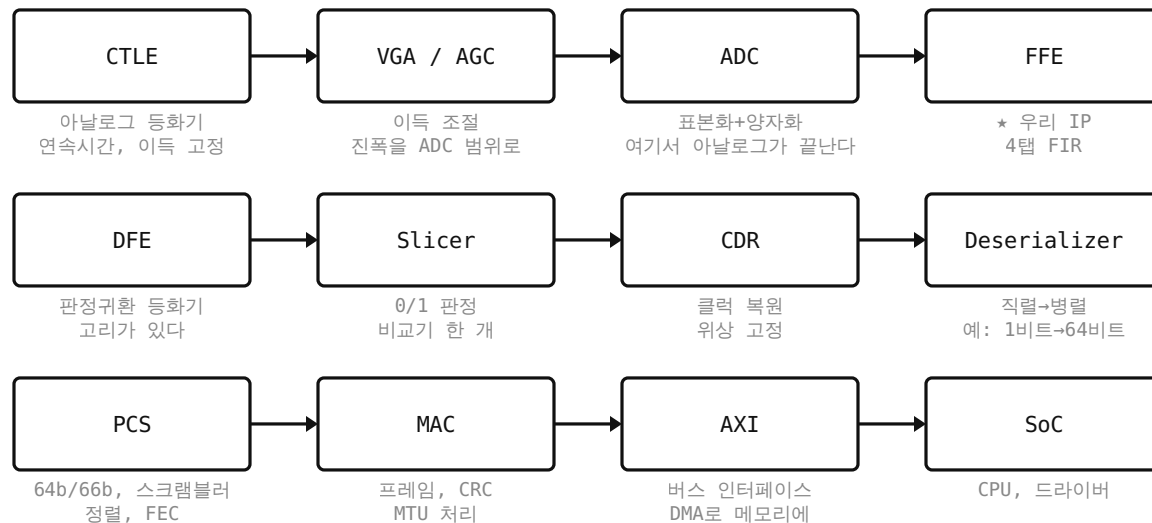


Figure 54. The 10 Gb/s receive chain. Our IP is the fourth box. Everything before it is analogue or mixed-signal; everything after it is ordinary synchronous digital logic. The boundary sits at the ADC — which is exactly why the IP's input is an 8-bit two's-complement sample and not a voltage.

Table 438. Each neighbour, and what it forces on our design

Block	What it does	What it forces on us
CTLE	Continuous-time linear equaliser: an analogue filter that boosts high frequencies before sampling.	It removes the bulk of the ISI, so our digital FFE only has to clean up the residue — which is why 4 taps is enough and 40 is not needed.
VGA / AGC	Variable-gain amplifier with automatic gain control: scales the signal so it fills the ADC's input range.	It guarantees our input occupies most of the 8-bit range. If AGC is broken our input is tiny and the filter output is mostly quantisation noise — a failure that looks like our bug.
ADC	Analogue-to-digital converter: turns a voltage into an 8-bit number, once per unit interval.	Defines our input format. 8-bit, two's complement, Q1.7, one sample per clock. Everything about our widths comes from this box.
DFE	Decision-feedback equaliser: subtracts the known ISI contribution of already-decided bits. Contains a feedback loop.	It sits immediately after us and its loop must close in one unit interval. Our latency is in series with nothing critical, but our throughput must be one sample per clock or the DFE starves.
Slicer	A comparator: decides 0 or 1 from the equalised sample.	It consumes our 8-bit output and only really needs the sign plus a few bits of margin — which is why saturating (rather than wrapping) matters so much: a wrap flips the sign and the slicer decides exactly wrong.
CDR	Clock and data recovery: extracts the sampling clock from the data.	It provides our clock. Our design must be fully synchronous to it and must tolerate its reset sequencing.
PCS / MAC	Protocol layers: alignment, 64b/66b coding, FEC, framing, CRC.	Far enough away not to constrain us directly, but they set the overall latency budget the whole chain is measured against.

Z16.3 “Front end” and “back end” mean two different things

This trips up every newcomer, because the same two words name two unrelated axes and practitioners switch between them without warning.

Table 439. Two meanings of front-end / back-end

Axis	Front end	Back end	Where our IP is
Signal chain (where in the data path)	Analogue and mixed-signal: CTLE, VGA, ADC, CDR. Deals in volts and picoseconds.	Digital: PCS, MAC, bus, software. Deals in bits and packets.	First block of the digital back end — immediately after the ADC.
Design flow (where in the schedule)	RTL design, verification, synthesis-for-estimate. Produces a netlist. This is where this book lives.	Floorplan, place and route, clock tree, timing closure, DRC/LVS, GDSII tape-out.	Parts Z14–Z15 are flow front-end. <code>synth.sh</code> 's <code>nextpnr</code> step is the smallest possible taste of flow back-end.

So “our IP is in the back end” and “we are doing front-end work” are both true at once, and mean different things. When someone says it, ask which axis.

Z16.4 The specification, written before the code

A specification is not a description of what you built. It is the contract you write first, and then check what you built against. Here is the real one for this block. Every row is a number a customer can hold you to, and every row was measured in Parts Z14–Z15 — the spec came first and the measurements came back to fill it in.

Table 440. IP specification: 4-tap feed-forward equaliser

Item	Value	How it is verified
Function	$y[n] = \text{sat}_g((\sum h[i] \cdot x[n-i]) \gg 7)$	Exhaustive C++ equivalence over all 2^{32} inputs (Z14.3)
Input	4×8 -bit signed, Q1.7, one sample per clock	Testbench drives all corner combinations plus random
Output	1×8 -bit signed, Q1.7, saturating	Saturation exercised by corner vectors; all 256 output codes seen
Coefficients	Compile-time constants, symmetric	Symmetry asserted at build; folding verified bit-exact against unfolded
Throughput	1 sample/clock, no back-pressure	Cycle count checked in the harness, not just values
Latency	1 clock (combinational variant) / 3 clocks (pipelined)	Measured start-to-done in simulation
Clock	Single clock, synchronous, rising edge	Lint (no latches, no multiple drivers) plus synthesis
Reset	Synchronous, active high	Explicitly stated because the HLS tool defaults to active low — see Z15.3
Area	252 LC (hand) / 386 LC (HLS @3ns) on iCE40 HX8K	yosys + nextpnr, three placer seeds
$F_{\max}$	76.9 MHz (hand, 3-stage) / 106.0 MHz (HLS @3ns)	nextpnr with a registered timing shell
Deliverables	RTL, C++ model, testbench, vectors, synthesis script, data-sheet, integration guide, known issues	<code>edu/house/release.sh</code> gates all of them

Two rows are worth pausing on. The **reset polarity** row exists because getting it wrong cost a debugging session in Part Z15 — a specification that does not pin down polarity, edge and synchronicity has not specified the reset at all. And **throughput**

is a separate row from latency because a value-only regression cannot see a block getting slower; Part Z1 caught a mutant that broke throughput while keeping every output value correct.

Z16.5 What else you must own to ship this

A 4-tap filter is not a product. A customer who buys “an equaliser” expects a block they can drop into a chip, and that means a surprising amount of surrounding machinery. For a one-person design house the question is not whether you need these — you do — but which you write, which you take from an open collection, and which you decline to support.

Table 441. Supporting IP, and the one-person make/buy call

What	Why it is needed	Call
Register file / CSR block	Coefficients, bypass, status, ID. No customer accepts a block with no software interface.	Make. Generated from one description — <code>edu/house/regmap.py</code> emits RTL, C header, docs and IP-XACT from a single source.
APB or AXI-Lite bridge	How the CSRs are reached from the SoC.	Make (APB is ~100 lines and fully specified) or take an open one. AXI-Lite is bigger; buy or adapt.
Clock-domain crossing / FIFO	The CSR bus and the data path are usually on different clocks.	Buy or adopt. CDC is the single most common source of bugs that pass simulation and fail silicon. Do not hand-roll a synchroniser.
Reset synchroniser	An asynchronous reset release violates recovery/removal timing on every flop it reaches.	Make , from the standard two-flop pattern, and lint for it.
PLL / CDR	Provides the clock.	Decline. This is analogue IP. A one-person digital house does not build a CDR; you specify the clock you need and document it.
SerDes PHY	Serialiser, driver, receiver front end.	Decline. Same reason, larger.
DFT / scan insertion	Manufacturing test. Without it the chip cannot be screened.	Defer to the integrator , but write RTL that is scan-friendly: no gated clocks you invented, no latches, no async logic.
Lint and CDC rule decks	Customers run them and send you the report.	Make. <code>verilator --lint-only -Wall</code> is free and catches most of it; ship the clean report as a deliverable.
Documentation set	Datasheet, integration guide, known issues.	Make. This is the cheapest thing you can do to look like a real vendor, and the thing most one-person efforts skip.

The honest summary of that table: the filter is perhaps 15 % of the work of selling the filter. Parts Y and Z4 build the other 85 %.

Z16.6 The five levels of drawing

“Schematic” is not one thing. The same design is drawn at five levels of detail and each answers a different question. Knowing which level you are being shown is half of reading a drawing.

Table 442. Levels of representation, from system to silicon

Level	What a box is	What a line is	Question it answers	Where in this book
System	A whole function (ADC, MAC)	A stream of data	Does the architecture close?	Z16.2, above
Block / RTL	A module with ports	A named bus of bits	What are the interfaces and widths?	Z16.7

Level	What a box is	What a line is	Question it answers	Where in this book
Datapath	An adder, a multiplier, a register	A bundle of wires	Where does the critical path run?	Z17.4
Gate	A NAND, a flip-flop	One wire carrying one bit	How many gates deep is it?	Z17.3
Transistor	One MOSFET	One node	Why does a gate behave as it does?	Z17.1–Z17.2

Z16.7 Our IP at block level

This is the drawing that belongs at the top of the datasheet. Boxes are modules, lines are buses, and the widths are on the lines because a width mismatch is the most common integration failure there is.

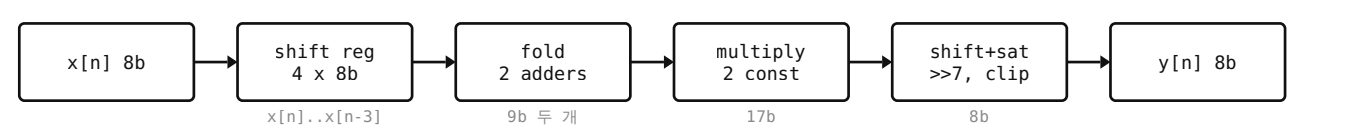


Figure 55. The equaliser at block level. Reading left to right: samples enter and are held in a 4-deep shift register; symmetric pairs are added (9 bits, because adding two 8-bit numbers needs 9); each sum is multiplied by one constant coefficient (17 bits); the result is shifted right by 7 to return to Q1.7 and saturated back to 8 bits. Every width change on this diagram is a place where a bug can hide silently.

Why every width on that diagram is what it is		
	Step	Justification
1	The input is 8 bits because the ADC produces 8 bits.	<i>The interface is set by the neighbour, not by us.</i>
2	The fold adders are 9 bits, not 8.	<i>Adding two n-bit signed numbers can need n+1 bits: 127+127 = 254, which does not fit in 8-bit signed. Truncating here would be wrong only for large inputs — a bug that passes random testing.</i>
3	The multiply result is 17 bits.	<i>9-bit operand times 8-bit coefficient gives up to 17 bits. The accumulator is declared 18 in the C++ model, and the maximum magnitude is 127×(18+111+111+18) = 32,766, which fits in 17 bits signed with room to spare — so the 18-bit saturation in the model never actually fires.</i>
4	The shift is by 7, not 8.	<i>Both sample and coefficient are Q1.7, so their product is Q2.14. Returning to Q1.7 means discarding 7 fractional bits.</i>
5	The output saturates to 8 bits rather than wrapping.	<i>The next block is a slicer that decides sign. A wrap turns a large positive into a large negative and the slicer decides exactly wrong; a saturate is wrong in magnitude but right in sign.</i>

That last derivation is the whole argument for drawing before coding. It took five steps and no Verilog, and it fixed every width in the design. Written the other way round — code first, widths discovered during debug — each of those five steps becomes a regression failure somebody has to chase.

Z17. From One Transistor to Our Datapath

This part builds the equaliser upward from a single transistor. Nothing is assumed. If you know that a computer is made of transistors and nothing more specific than that, start here; by the end you will be able to read the gate-level schematic of the adder inside the filter and say why it is shaped the way it is.

The reason this matters to someone writing RTL or HLS is not nostalgia. Every decision the previous parts made — why saturation costs more than truncation, why a 9-bit adder is not an 8-bit adder, why the critical path runs where it does, why you register a signal before it leaves the chip — is a fact about these drawings. You can write RTL without them and you will write slower RTL.

Z17.1 The switch

A MOSFET is a voltage-controlled switch with three terminals that matter: **gate**, **source** and **drain**. Current flows between source and drain when the gate says so. There are two flavours and they are opposites:

Table 443. The two switches

Type	Turns on when its gate is	Good at passing	Where it goes
NMOS	high (1)	a strong 0	between the output and ground
PMOS	low (0)	a strong 1	between the output and VDD

The third column is the one people skip and then get wrong. An NMOS can pull a node down to 0 V cleanly but cannot pull it all the way up to VDD — it stops roughly a threshold voltage short. A PMOS is the mirror image. So neither type alone can build a good logic gate, and *that* is why every static CMOS gate contains both: PMOS above to pull up, NMOS below to pull down. The circuit family is called **complementary** MOS for exactly this reason.

Z17.2 The inverter, and why CMOS won

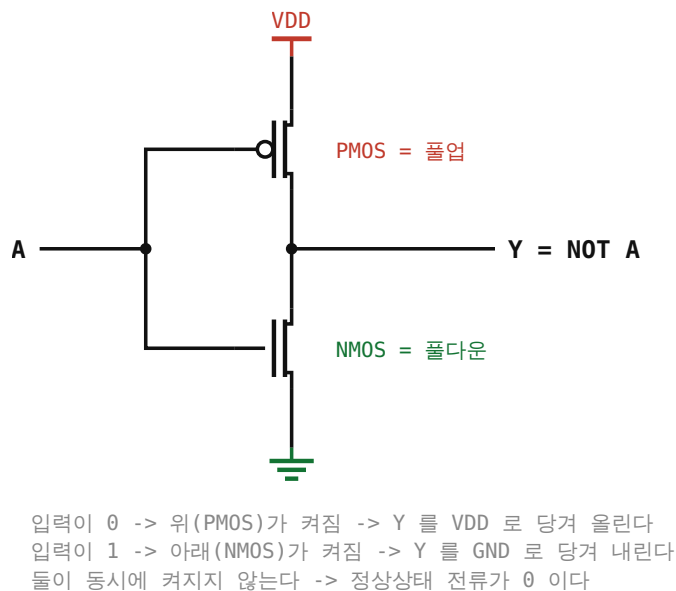


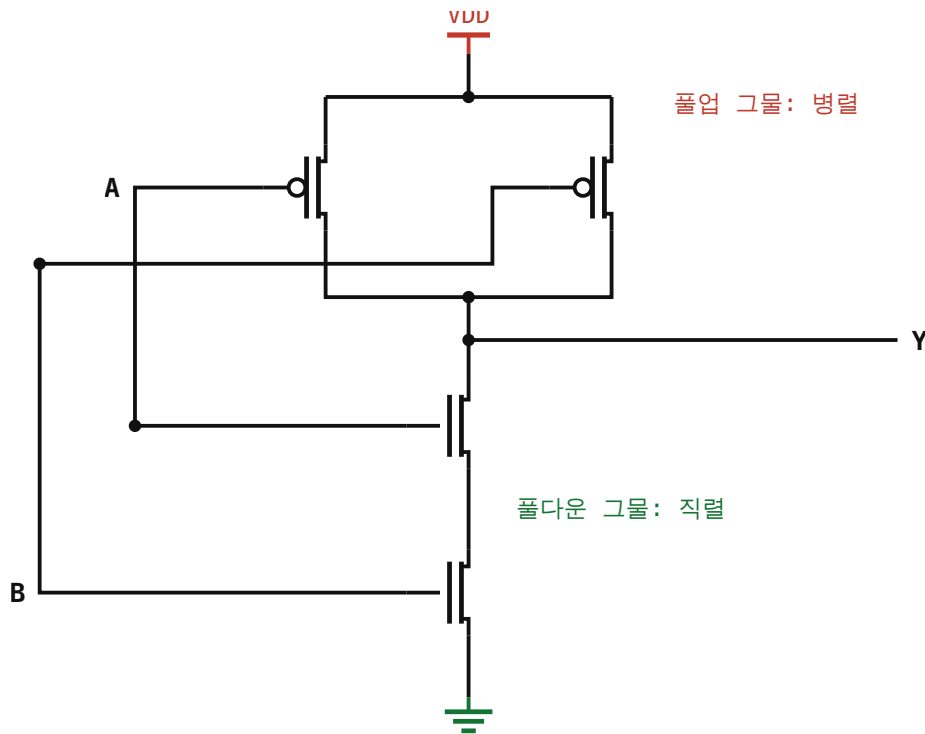
Figure 56. The CMOS inverter: one PMOS pull-up, one NMOS pull-down, gates tied together. This is the smallest complete CMOS gate and the template for every other one. The red bar at the top is the supply (VDD); the striped symbol at the bottom is ground (0 V).

Why this circuit inverts, and why it costs no static power

	Step	Justification
1	Input A = 0. The PMOS gate sees 0, so the PMOS is on . The NMOS gate sees 0, so the NMOS is off .	<i>From the table above: PMOS turns on with a low gate, NMOS with a high gate.</i>
2	With only the PMOS on, the output node is connected to VDD and disconnected from ground. Y is pulled up to 1.	<i>The only conducting path from Y goes to the supply.</i>
3	Input A = 1. Now the PMOS is off and the NMOS is on, so Y is connected to ground and disconnected from VDD. Y is pulled down to 0.	<i>The mirror image of the previous step.</i>
4	In both steady states exactly one of the two devices conducts, so there is no path from VDD to ground .	<i>A path from supply to ground would be a short circuit drawing continuous current.</i>
5	Therefore a CMOS gate that is not switching draws (almost) no current.	<i>This is the property that let CMOS displace every earlier logic family. Power is consumed charging and discharging capacitance when the output changes, which is why dynamic power is proportional to activity and frequency.</i>
6	The word 'almost' is doing real work at modern nodes.	<i>Sub-threshold leakage and gate leakage mean an idle gate does draw a small current, and below roughly 28 nm leakage becomes a first-order design problem. Part X43 covers what is done about it.</i>

Z17.3 NAND: pull-up and pull-down are duals

Now the gate that everything is actually built from. The rule that generates every static CMOS gate is a single sentence: **build the pull-down network from NMOS so that it conducts exactly when the output should be 0, then build the pull-up network as its dual.**



A·B 가 둘 다 1 일 때만 아래 길이 이어져 Y=0
 하나라도 0 이면 위 길이 이어져 Y=1 -> Y = NOT(A AND B)
 풀업과 풀다운은 언제나 서로의 쌍대(dual)다 -- 직렬 <-> 병렬

Figure 57. A 2-input CMOS NAND. The two NMOS are in **series**: current only reaches ground when A and B are both 1, which is exactly when NAND's output should be 0. The two PMOS are in **parallel**: a path to VDD exists when A is 0 or B is 0. Series means AND; parallel means OR.

Table 444. Reading the NAND schematic as a truth table

A	B	Pull-down (series NMOS)	Pull-up (parallel PMOS)	Y
0	0	both off → open	both on → conducts	1
0	1	top off → open	left on → conducts	1
1	0	bottom off → open	right on → conducts	1
1	1	both on → conducts	both off → open	0

Why the pull-up is the dual of the pull-down, always

	Step	Justification
1	The output must be driven to exactly one value in every input combination.	<i>If neither network conducts the output floats (high-impedance, value undefined); if both conduct you have a short from VDD to ground.</i>
2	So the pull-up must conduct exactly when the pull-down does not — the two conditions are logical complements.	<i>That is what ‘exactly one’ means.</i>
3	The pull-down for NAND conducts when (A AND B).	<i>Series transistors both have to be on, and NMOS is on for a 1.</i>
4	Its complement is $\text{NOT}(A \text{ AND } B) = (\text{NOT } A) \text{ OR } (\text{NOT } B)$, by De Morgan's law.	<i>De Morgan converts a negated conjunction into a disjunction of negations.</i>
5	PMOS is on for a 0, so a PMOS whose gate is A implements (NOT A). An OR of two conditions is a parallel connection.	<i>Either path alone is enough to conduct.</i>
6	Hence: series NMOS below, parallel PMOS above. Swap series and parallel and you get NOR instead.	<i>NOR's pull-down is parallel (output 0 when A OR B) and its pull-up is series.</i>

Two consequences worth carrying into RTL. First, **NAND and NOR are the cheap gates** — four transistors each. AND and OR are *more* expensive, because CMOS naturally produces an inversion and a non-inverting gate is a NAND followed by an inverter: six transistors. This is why synthesis tools cheerfully rewrite your logic into seas of NANDs, and why the classic identity $\text{AND}(a,b) = \text{NOT}(\text{NAND}(a,b))$ is worth recognising in a netlist.

Second, **series transistors are slow**. A 4-input NAND has four NMOS in series and its pull-down is roughly four times weaker than an inverter's. That is why wide gates get decomposed into trees, and why a 17-bit ripple adder is not simply ‘a bit slower’ than an 8-bit one.

Z17.4 Pull-up resistors — a different thing with a similar name

The phrase “pull-up” is used for two different circuits and conflating them causes real design errors.

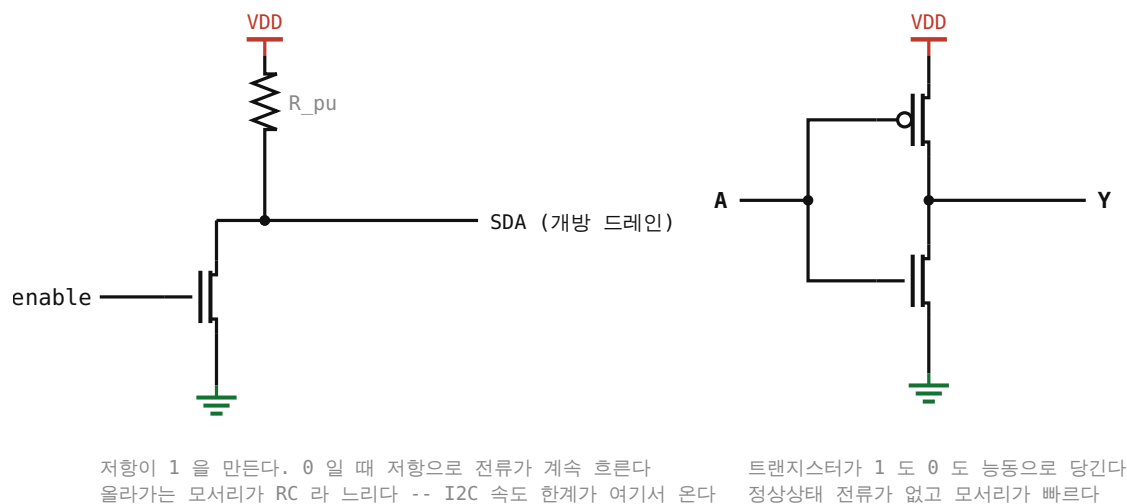


Figure 58. Two things called “pull-up”. **Left:** a real resistor on a board, holding an open-drain line high until some device pulls it low. **Right:** a PMOS pull-up network inside a CMOS gate. Only the left one is a resistor; the right one is the transistor network of Z17.2.

Table 445. Resistor pull-up versus transistor pull-up

	Pull-up resistor (board level)	PMOS pull-up (inside a gate)
What it is	A physical resistor, typically 1–10 k Ω	A transistor network, the dual of the pull-down
Where	On the PCB or inside an I/O pad	Inside every static CMOS gate
Why	The line has no active driver for the high state — open-drain or open-collect-or outputs can only pull down	Every state is actively driven
Static current	Yes — whenever the line is held low, current flows through the resistor to ground	No — exactly one network conducts
Speed	Slow rising edge: RC charging of the bus capacitance. This is why I ² C is limited to 100 kHz/400 kHz/1 MHz modes	Fast: the PMOS actively charges the node
Typical use	I ² C SDA/SCL, shared interrupt lines, reset lines, strapping pins, unused inputs	All ordinary logic
Relevance to our IP	None inside the block. It appears at the chip boundary and in the CSR bus pads	Every gate the filter synthesises into

Worked example 69. Sizing a pull-up resistor, and why the choice is a squeeze

Given. An I²C bus at 3.3 V with 200 pF of bus capacitance. The standard requires the low level to stay below 0.4 V while a device sinks at most 3 mA, and the rise time to be under 300 ns for 400 kHz fast mode.

Method. The lower bound comes from current: R must be large enough that $(V_{DD} - V_{OL})/R$ does not exceed the sink current. The upper bound comes from speed: the RC rise to the input threshold must fit in the rise-time budget, which for $0 \rightarrow 0.7 V_{DD}$ takes about 1.2 RC.

Numbers. Lower bound: $R \geq (3.3 - 0.4)/3 \text{ mA} = 967 \text{ }\Omega$. Upper bound: $R \leq 300 \text{ ns}/(1.2 \times 200 \text{ pF}) = 1250 \text{ }\Omega$. So R must sit between roughly 1.0 and 1.25 k Ω — a narrow window.

Note what dominates: capacitance. Halving C doubles the upper bound. On a board, capacitance is trace length and the number of devices, so the electrical problem is really a layout problem.

How this is got wrong. The window being this narrow is the finding, not an arithmetic slip. The usual reflex is to grab the familiar 4.7 k Ω part, which violates the upper bound by nearly 4 $\times$ and produces a bus that works on the bench at low speed and fails at 400 kHz with long traces. If the window closes entirely, the fix is not a different resistor — it is less bus capacitance, or an active terminator.

Z17.5 The clock, and why anything is clocked at all

Combinational logic has no memory: outputs follow inputs after a delay. To build anything that accumulates, you need an element that *holds*. The universal one is the edge-triggered **D flip-flop**: it samples its D input at the rising edge of the clock and holds that value on Q until the next rising edge.

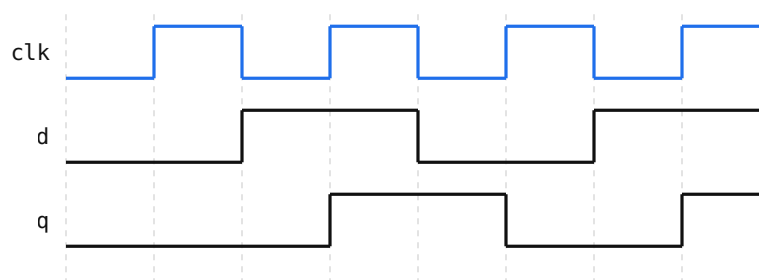


Figure 59. A D flip-flop. Q takes the value D had at the rising edge and holds it for the whole cycle. Note that Q changes only at rising edges, never in between — that is the entire point.

The clock is not there to make things go. It is there to make things *agree on when*. Combinational paths through different amounts of logic finish at different times; without a clock, a downstream block cannot know whether what it is looking at is the final answer or a value still settling. The clock defines an instant at which everything is required to have settled, and the design's job is to make that true.

Table 446. The two timing constraints, and what violating each looks like

Con-straint	What it requires	Fix if violated	Symptom
Setup	Data must arrive at the flop at least t_{su} before the edge. Limits the longest path.	Shorten the logic, add a pipeline stage, or slow the clock.	Works at low frequency, fails above some clock rate. Reproducible.
Hold	Data must remain stable at least t_h after the edge. Limits the shortest path.	Add delay (buffers). Slow-ing the clock does not help.	Fails at every frequency. Usually a silicon respin — which is why hold violations are feared more than setup.

That asymmetry is worth memorising: a setup failure is a performance bug, a hold failure is a functional bug. It is also why Part Z15's measurement needed a registered timing shell — a path that starts or ends outside the chip has no flop at one end, so neither constraint is defined and the tool has nothing to report.

Z17.6 Our adder, at gate level

Now put it together. The filter's first operation is $a0 = x0 + x3$ on 8-bit signed values producing 9 bits. An adder is built from full adders, and a full adder is built from the gates above.

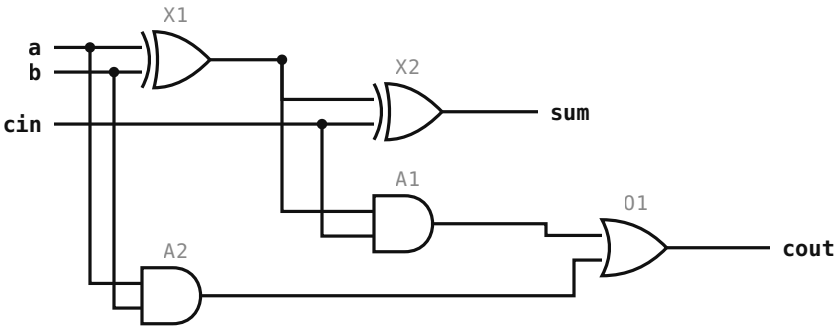


Figure 60. One full adder: $sum = a \text{ XOR } b \text{ XOR } cin$, $cout = (a \text{ AND } b) \text{ OR } (cin \text{ AND } (a \text{ XOR } b))$. Five gates. The 9-bit fold adder in the filter is nine of these chained, with each cout feeding the next cin — a ripple-carry adder.

Worked example 70. Why the carry chain sets the critical path

Given. A ripple-carry adder of N bits. Each full adder's carry output takes about 2 gate delays (one AND, one OR) after its carry input is valid.

Method. The last bit's carry cannot be computed until every earlier carry has rippled through. Count gate delays along that chain.

Numbers. Roughly $2N$ gate delays for N bits: **18** for the 9-bit fold adder, **34** for a 17-bit accumulator. The sum bits themselves add only one more XOR after their carry arrives.

This is also why fast-adder architectures exist — carry-lookahead, carry-select, prefix adders — trading area for a logarithmic rather than linear carry depth. On an FPGA you usually get the vendor's carry chain and the question does not arise; in an ASIC the synthesiser picks from a library of adder architectures based on your timing constraint.

How this is got wrong. The tempting conclusion is that our filter's critical path is the adder. It is not, and assuming so sends you optimising the wrong thing. The measured designs in Z15 use iCE40's dedicated carry chain (SB_CARRY), which is far faster than generic gates — the hand design used 48 of them. On this fabric the multiply-by-constant shift-add network, not the ripple, dominates. **Count gates to understand the structure; measure the tool output to know the number.**

Z17.7 Why saturation costs more than wrapping

Part Z16 justified saturation on system grounds: a wrap flips the sign and the slicer decides exactly wrong. Here is what it costs, in gates.

Table 447. Truncate versus saturate, at gate level

Operation	What the hardware does	Cost
Truncate (wrap) to 8 bits	Discard the upper bits. Route 8 wires, ignore the rest.	Zero gates. It is a renaming of wires.
Saturate to 8 bits	Detect whether the value exceeds the 8-bit range, and if so substitute the limit. Needs two comparisons and an 8-bit 2-to-1 multiplexer.	Two comparators plus a mux — tens of gates, and it sits after the adder, so it adds directly to the critical path.

This explains a result in Part Z15 that looked backwards at first. Narrowing the accumulator from 32 bits to 18 *should* have shrunk the design, but the narrowed variant came out larger in Bambu's estimate — because the narrowing was implemented with `sat()` at every accumulation step, and each of those saturations bought comparators that the plain 32-bit accumulation did not need. The width saving was real and the saturation cost was larger.

The lesson generalises: **narrowing a datapath is only free if you narrow it somewhere the value provably cannot overflow.** In the final tuned design the accumulator is wide enough that no intermediate saturation is needed at all — Z16.7 derived the bound, 32,766, which fits in 17 bits — and saturation happens exactly once, at the output, where the specification demands it.

Z17.8 The whole stack, one page

Table 448. Every level, and what our filter looks like there

Level	Our filter at this level	Who works here
System	One box in the receive chain, between ADC and DFE	Architect; the spec of Z16.4 is written here
RTL / HLS	40 lines of Verilog, or 12 lines of C++ plus a build command	You, in Parts Z14–Z15
Netlist	338–1218 cells depending on how the C++ was typed	Synthesis (yosys); you read its report

Level	Our filter at this level	Who works here
Gate	SB_LUT4 ×228, SB_CARRY ×97 — NANDs and carry chains in a specific arrangement	The tool; you influence it through coding style and constraints
Transistor	Each LUT4 is a small memory plus a mux tree; each is CMOS pull-up/pull-down as in Z17.2	The FPGA vendor, or in an ASIC the standard-cell library team
Silicon	Configuration bits in an iCE40, or diffused transistors in a mask set	The foundry

The practical point of having walked down this stack is that the levels are not independent. A choice at the top — “saturate rather than wrap” — became comparators and a mux at the gate level, which became area and delay in the measurements of Part Z15, which fed back into the specification in Z16.4. An engineer who can only see one level cannot follow that chain, and will make the choice at the top without knowing what it costs at the bottom.

Z18. SystemVerilog for the C++ Programmer

SystemVerilog looks enough like C to be dangerous. The braces are begin/end, the types are unfamiliar, and about a third of the constructs mean something that has no software equivalent at all. This part is a complete tour, written for somebody who knows C++ and has never seen a hardware description language. It covers the synthesisable language in full, then the verification subset, and says at every point which of the two you are in.

The single most important idea, before any syntax: **you are not writing instructions that execute. You are describing hardware that exists.** Every always block in a design is a piece of circuitry that is powered on all the time, simultaneously with every other block. Sequence exists only inside a single block, and even there it is often a fiction the synthesiser unrolls into wires.

Z18.1 The thing you asked about: ::

:: is the scope resolution operator, and it means very nearly what it means in C++ — “this name, from inside that namespace.” SystemVerilog has three scopes that use it.

Table 449. The three things that appear to the left of ::

Form	What the left side is	C++ analogue
my_pkg::WIDTH	A package — a named bundle of types, constants and functions shared across files.	namespace my_pkg { }, and this is the same syntax
state_e::IDLE	An enum type , naming one of its members explicitly.	enum class state_e { IDLE } — also the same syntax
std::mailbox	The built-in package, which holds mailbox, semaphore, process.	std::, and even the same name
\$unit::foo	The compilation unit scope — everything declared outside any package or module.	The global namespace, ::foo

```
// — 패키지: C++ 의 namespace 와 같은 자리다
package fir_pkg;
    // parameter = 컴파일 시간 상수. C++ 의 `constexpr` 에 해당한다.
    parameter int NTAP = 4;
    parameter int SAMP_W = 8;

    // typedef: C 와 똑같다. `logic signed [7:0]` 에 이름을 붙인다.
    typedef logic signed [SAMP_W-1:0] samp_t;

    // enum: C++11 의 `enum class` 처럼 타입이 붙는다.
    // 괄호 안 `logic [1:0]` 은 **이 enum 이 하드웨어에서 2비트로 인코딩된다**는
    // 뜻이다. C++ enum 의 underlying type 과 같은 개념인데, 여기서는
    // 실제 플립플롭 개수를 정한다.
    typedef enum logic [1:0] { IDLE, RUN, DONE } state_e;

    // 함수도 답을 수 있다. `automatic` 은 아래 Z18.9 에서 설명한다.
    function automatic int unsigned clog2_up(int unsigned v);
        clog2_up = 0;
        while (v > 1) begin v = v >> 1; clog2_up++; end
    endfunction
```

endpackage

```
// — 쓰는 쪽 —————
module fir_core;
    // (1) 매번 `fir_pkg::` 를 붙여 쓴다 -- C++ 의 `std::vector` 와 같다
    fir_pkg::samp_t a;
    localparam int N = fir_pkg::NTAP;

    // (2) enum 멤버를 타입 이름으로 한정한다
    fir_pkg::state_e st = fir_pkg::state_e::IDLE;

    // (3) import 로 통째로 끌어온다 -- C++ 의 `using namespace` 다
    import fir_pkg::*;
    samp_t b;                // 이제 접두사 없이 쓴다
endmodule
```

The difference from C++ that bites: **an import inside a module is visible only in that module**, and a package must be compiled *before* anything that imports it. There is no header-guard mechanism and no forward declaration; file order on the compiler command line is part of the build. Most projects keep one package per file and list packages first in the file list.

One more ::-adjacent thing that surprises C++ programmers: the dot. dut.internal_reg reaches *inside* another module instance from a testbench, past every encapsulation boundary. This is a **hierarchical reference**, it is legal, it is not synthesizable, and it is how most white-box testbenches are written. There is no private in SystemVerilog modules.

Z18.2 What a module is, and why it is not a class

```
// 모듈 = 하드웨어 한 덩어리. C++ 의 class 가 아니라 **회로 그 자체**다.
//
// class 와 다른 점:
// * 생성/소멸이 없다. 전원이 켜져 있는 동안 늘 존재한다
// * 메서드 호출이 없다. 포트에 신호가 들어오고 나갈 뿐이다
// * 인스턴스가 **컴파일 시간에 고정**된다. 런타임에 new 할 수 없다
// (verification 쪽 `class` 는 할 수 있다 -- Z18.11)
module fir4 #(
    // 파라미터 = 인스턴스마다 다르게 줄 수 있는 컴파일 시간 값.
    // C++ 의 **템플릿 인자**에 해당한다: `template<int SAMP_W>`
    parameter int SAMP_W = 8,
    parameter int NTAP    = 4
) (
    // 포트 목록. 방향이 반드시 붙는다.
    // input   읽기만 한다
    // output  이 모듈이 구동한다
    // inout   양방향 (삼상태). 칩 안에서는 거의 안 쓴다
    input logic                clk,
    input logic                rst,
    input logic signed [SAMP_W-1:0] x,
    output logic signed [SAMP_W-1:0] y
);
// ... 내용 ...
```

```
endmodule
```

```
// 인스턴스화.  `.포트이름(연결할신호)` 를 **이름으로** 잇는다.
// 순서로 잇는 옛 문법(`fir4 u(clk, rst, x, y)`도 되지만 **쓰지 마라** --
// 포트를 하나 추가하면 그 아래가 전부 조용히 밀린다.
fir4 #(.SAMP_W(8), .NTAP(4)) u_fir (
    .clk (clk_100m),
    .rst (rst_sync),
    .x   (adc_sample),
    .y   (eq_out)
);
```

Z18.3 Types: the four-state trap

C++ has `bool`. SystemVerilog has a bit that can hold four values, and the two extra ones are the most useful debugging tool in the language.

Table 450. The four values a logic bit can take

Value	Means	Where it comes from	Why you want it
0	low	driven low	—
1	high	driven high	—
x	unknown	uninitialised register, two drivers fighting, reading before reset	It propagates. An x in a sum makes the sum x, so a missing reset shows up as garbage rather than as a plausible zero.
z	high impedance (not driven)	a tri-state driver that is off, an unconnected input	Models buses where several drivers share a wire.

Table 451. The types, and which to use

Type	States	Use it for	C++ analogue
logic	4	Almost everything. One bit, or logic [7:0] for a vector.	Closest to <code>bool</code> / <code>uint8_t</code> , but 4-state
wire	4	Legacy. A net that must be continuously driven. <code>logic</code> covers it in SystemVerilog.	—
reg	4	Legacy Verilog name for a procedural variable. It does not mean ‘register’ — this misnaming has confused people for thirty years. Use <code>logic</code> .	—
bit	2	Testbench code where you want C semantics and speed.	<code>bool</code>
int	2	Loop counters, testbench arithmetic. 32-bit signed.	<code>int32_t</code>
byte, shortint, longint	2	8/16/64-bit signed testbench integers.	<code>int8_t</code> , <code>int16_t</code> , <code>int64_t</code>
integer	4	Legacy 32-bit 4-state.	—
real	—	Floating point, testbench only.	<code>double</code>
string	—	Testbench messages.	<code>std::string</code>

```
// — 벡터와 부호 —————
logic [7:0] u;    // 8비트 **부호 없음** (기본값)
```

```

logic signed [7:0] a;    // 8비트 부호 있음 -- `signed`를 빼면 조용히 틀린다

// [7:0] 과 [0:7] 은 다르다. [7:0] 이 관례다 (왼쪽이 MSB).
// C++ 배열과 달리 **비트 번호를 내가 정한다** -- [15:8] 도 legal 하다.

// 부호를 빠뜨리면 어떻게 되나 -- 실제로 물리는 자리다
logic      [7:0] p = 8'hFF;    // 255 로 읽힌다
logic signed [7:0] q = 8'hFF;    // -1 로 읽힌다
// p > 0 은 참, q > 0 은 거짓. **같은 비트, 다른 답.**

// — 리터럴 —————
// <폭>'<진법><값>
8'd10      // 8비트 십진 10
8'hFF      // 8비트 16진
8'b1010_1010 // 밑줄은 자리 구분용. 무시된다 (C++14 의 ' 와 같다)
8'sd10      // 8비트 **부호있는** 십진
'0          // 폭을 문맥에서 가져오는 전부-0. `1` 은 전부-1 (0001 아님!)
8'bx       // 전부 unknown -- 리셋 전 상태를 흉내낼 때

// — 구조체와 배열 —————
typedef struct packed {          // packed = **비트가 연속**이다. 합성된다
    logic      valid;            // [16]
    logic [7:0] data;            // [15:8]
    logic [7:0] tag;             // [7:0]
} beat_t;                        // 통째로 17비트 벡터처럼 쓸 수 있다

typedef struct {                 // unpacked = 그냥 묶음. **합성 안 된다**
    int count;
    string name;
} stat_t;                        // 테스트벤치용

logic [7:0] mem [0:255];        // unpacked 배열 = 메모리. `[개수]` 가 **뒤**에 온다
logic [3:0][7:0] lanes;         // packed 배열 = 32비트를 4x8 로 본 것
beat_t queue [$];               // 동적 큐 (테스트벤치). C++ 의 `std::deque`

```


Worked example 71. Why signed is the most expensive missing keyword in the language

Given. The filter adds two 8-bit samples: $a0 = x0 + x3$, where both may be negative. Written as `logic [7:0] x0` without `signed`.

Method. Work out what the comparison and the shift do under unsigned interpretation.

Numbers. $x0 = -1$ is stored as 8'hFF. Unsigned, that reads as 255. Adding -1 and -1 should give -2 ; unsigned it gives 510, and the right shift that follows brings in zeros from the top instead of sign bits, so the result is a large positive number instead of a small negative one.

The rule in SystemVerilog: an expression is unsigned if **any** operand is unsigned. One unsigned literal in a long signed expression silently converts the whole thing. When in doubt, wrap in `$signed()`.

How this is got wrong. Nothing warns you. The bit patterns are identical — only the *interpretation* differs, and the interpretation lives in a keyword you forgot to type. The design simulates, synthesises, and is wrong for exactly the half of the input space where the sample is negative. Random testing finds it; directed testing with positive-only stimulus does not. This is why `edu/model/fir4_hand.v` writes `$signed({x0[7], x0})` explicitly rather than relying on declarations being right.

Z18.4 The two assignments, and when to use which

This is the rule that separates working RTL from RTL that simulates and then fails in hardware. C++ has one assignment; SystemVerilog has two and they mean different things about *time*.

Table 452. Blocking versus non-blocking

	= blocking	<= non-blocking
Reads like	C's <code>=</code> : compute, store, move on.	"Sample all right-hand sides now; update all left-hand sides at the end of the time step."
Models	A wire — combinational logic.	A flip-flop — every register in the block updates together on the edge.
Use in	<code>always_comb</code>	<code>always_ff</code>
If you get it wrong	Non-blocking in combinational logic: simulates slowly and can need extra delta cycles, may mismatch synthesis.	Blocking in sequential logic: creates a race . Simulation result depends on the order the simulator happens to evaluate blocks in; hardware does something else.

// — 왜 순차 논리에 `<=` 를 쓰는가 —————
 // 하드웨어에서 이 두 플립플롭은 ****동시에**** 잡힌다. a 가 b 로 가고 b 가
 // c 로 가는 시프트 레지스터를 생각하면 된다.

```
always_ff @(posedge clk) begin
    b <= a;      // 이 edge 의 **옛** a 값을 잡는다
    c <= b;      // 이 edge 의 **옛** b 값을 잡는다
end
// 결과: 시프트 레지스터. a -> b -> c, 한 edge 에 한 칸. **맞다.**
```

```
always_ff @(posedge clk) begin
    b = a;       // b 가 **즉시** a 가 된다
    c = b;       // 그래서 c 도 **같은 edge 에** a 가 되어 버린다
end
// 결과: b 와 c 가 둘 다 a. 시프트 레지스터가 아니라 그냥 팬아웃이다.
// 더 나쁜 것: 다른 always 블록이 b 를 읽고 있으면, 그 블록이 이 블록보다
```

```
// 먼저 도느냐 나중에 도느냐에 따라 **옛 b 를 보거나 새 b 를 본다.**
// 시뮬레이터가 순서를 정하고, 그 순서는 명세에 없다 -- 그것이 경주다.

// — 조합 논리에는 = 를 쓴다 —————
always_comb begin
    sum = a + b;          // 중간값
    outv = sum + c;        // **방금 계산한** sum 을 쓴다 -- 그래서 blocking
end
// <= 로 쓰면 outv 가 이번이 아니라 **다음** delta 의 sum 을 본다.
```

The rule, stated once and never broken: **non-blocking in a `always_ff`, blocking in a `always_comb`, never both in one block, and never assign the same signal from two blocks**. Every lint tool checks it. The generated RTL in Part Z15 obeys it because the HLS tool cannot make this mistake — which is one genuine, if unglamorous, advantage of generating RTL.

Z18.5 The `always` blocks

```
// — SystemVerilog 의 세 가지 (Verilog 의 `always` 를 갈라놓은 것) —

// 1. 조합 논리. 감도 목록을 **도구가 채운다** -- 빠뜨릴 수가 없다
always_comb begin
    y = a & b;
end
// 옛 Verilog: always @(a or b) <- b 를 빠뜨리면 래치가 생긴다.
// `always_comb` 는 그 사고를 문법으로 막는다. **항상 이것을 써라.**

// 2. 순차 논리. edge 를 적는다
always_ff @(posedge clk) begin          // 동기 리셋
    if (rst) q <= '0;
    else    q <= d;
end

always_ff @(posedge clk or negedge rst_n) begin // 비동기 리셋
    if (!rst_n) q <= '0;                // 리셋이 감도 목록에 **있어야** 한다
    else        q <= d;
end

// 3. 래치. **의도적으로** 래치를 만들 때만. ASIC 에서는 거의 금지다
always_latch begin
    if (en) q = d;
end

// — 래치가 사고로 생기는 자리 —————
always_comb begin
    if (sel) y = a;          // sel 이 0 일 때 y 를 안 정했다
end                          // -> y 가 **이전 값을 기억해야** 한다 -> 래치!
// 고치는 법 두 가지:
always_comb begin
    y = '0;                  // (a) 먼저 기본값을 준다
```

```

    if (sel) y = a;
end
always_comb begin
    if (sel) y = a;          // (b) 모든 갈래를 채운다
    else    y = '0;
end
// `always_comb` 를 쓰면 도구가 "latch inferred" 라고 **경고**해 준다.
// 옛 `always @(*)` 는 조용히 만든다.

```

Z18.6 Operators: what is the same and what is not

Table 453. Operators a C++ programmer will misread

Operator	Meaning	The C++ trap
& ^ ~	Bitwise, on vectors.	Same as C++.
&& !	Logical, on truth of the whole value.	Same as C++.
&a a ^ a	Reduction — unary. &a is AND of every bit of a; ^a is parity.	No C++ equivalent. ^data computing parity in one character is the single most useful operator in the language.
== !=	Equality; result is x if either operand has an x.	Can return unknown, not just true/false.
=== !==	Case equality — compares x and z literally, always gives 0 or 1.	Use in testbenches to ask 'is this actually X?'. Not synthesizable.
>> <<	Logical shift — brings in zeros.	Same as C++ unsigned shift.
>>>	Arithmetic right shift — replicates the sign bit. Only if the operand is signed.	C++ >> on a signed int does this; SystemVerilog needs the third angle bracket and a signed operand.
{a, b}	Concatenation — glue bits together.	No C++ equivalent. {x[7], x} is 9-bit sign extension.
{4{a}}	Replication — four copies of a.	{8{x[7]}} is the idiomatic sign-extend.
a inside {1,2, [5:9]}	Set membership, ranges allowed.	Like a switch without fallthrough; also used in constraints.
?:	Conditional. Synthesises to a multiplexer.	Same as C++, but remember it is <i>hardware</i> — both sides are always computed.

```

// — 이 저장소의 실제 코드로 본 연산자 —————

// 부호 확장: 8비트를 9비트로. {x0[7], x0} 이 최상위 비트를 한 번 더 붙인다
wire signed [8:0] a0 = $signed({x0[7], x0}) + $signed({x3[7], x3});

// 왜 $signed() 로 또 감싸나?
// {} 연결의 결과는 **언제나 unsigned** 다 -- 안에 뭐가 들었든.
// 이것이 SystemVerilog 에서 가장 흔한 조용한 버그 중 하나다.

// 비트 뽑기와 자르기
acc[16:7]          // 고정 범위
acc[i +: 8]        // i 부터 위로 8비트. i 가 변수여도 된다
acc[i -: 8]        // i 부터 아래로 8비트
// `acc[i+7 : i]` 는 **불법**이다 (범위 양끝이 다 변수면 안 된다).

```

```
// +:/-: 가 그것을 푸는 문법이다.

// 리덕션으로 패리티 한 줄
wire parity = ^data;           // data 의 모든 비트를 XOR

// 전부 0 인지 검사 -- 세 가지가 다 같은 뜻이다
wire empty1 = (count == '0);
wire empty2 = ~|count;         // 리덕션 OR 의 부정
wire empty3 = !count;
```

Z18.7 Control flow, and what it becomes

```
// if / else -> 멀티플렉서.  **양쪽 다 하드웨어로 존재한다.**
// C++ 처럼 "안 타는 가지는 실행 안 됨" 이 아니다. 둘 다 늘 계산되고
// 마지막에 mux 가 고른다. 그래서 `if (rare) 비싼연산()` 은 하드웨어에서
// 아무것도 아껴 주지 않는다.

// case -> 역시 mux. 수식어가 중요하다
unique case (state)           // 겹치는 가지가 있으면 **시뮬레이션 오류**
    IDLE: ...                 // 빠진 가지가 있어도 오류
    RUN: ...
    DONE: ...
endcase

priority case (sel)           // 위에서부터 우선순위. 겹쳐도 됨
    sel[0]: ...
    sel[1]: ...
endcase

casez (opcode)                // ? 를 don't-care 로 본다
    8'b1010_????: ...        // 상위 4비트만 맞으면 된다
endcase

// `casex` 도 있는데 **쓰지 마라** -- x 를 don't-care 로 취급해서
// 합성 전후가 달라지는 고전적 버그를 낸다.

// for -> **언롤링된다**. 루프가 아니라 하드웨어 복제다
always_comb begin
    acc = '0;
    for (int i = 0; i < NTAP; i++)
        acc = acc + x[i] * H[i];    // 곱셈기 NTAP 개가 **동시에** 생긴다
end

// 경계가 컴파일 시간에 정해져야 한다. `for (i = 0; i < n; i++)` 에서
// n 이 신호면 합성 안 된다 -- 하드웨어 개수를 못 정하므로.

// generate -> 모듈/블록을 복제한다. for 와 다르게 **구조**를 만든다
genvar g;
generate
    for (g = 0; g < NTAP; g++) begin : gen_tap    // 라벨을 **반드시** 붙인다
```

```

        tap_mul u_mul (.x(x[g]), .h(H[g]), .y(prod[g]));
    end
endgenerate
// 라벨이 없으면 계층 이름이 `genblk1` 이 되고, 나중에 탭을 하나 추가하면
// 이름이 전부 밀려서 파형·제약·커버리지가 통째로 어긋난다.
// verilator -Wall 의 GENUNNAMED 가 이것을 잡는다.

```

Z18.8 Interfaces — the one big idea with no C++ analogue

A bus has a dozen signals and they always travel together. Declaring them twelve at a time in every module is where port-list bugs come from. An interface bundles them and a modport says which direction each end sees.

```

interface axis_if #(parameter int W = 8) (input logic clk);
    logic [W-1:0] tdata;
    logic          tvalid, tready, tlast;

    // modport = 한쪽 끝에서 본 방향. 같은 다발을 양쪽이 반대로 본다
    modport src  (output tdata, tvalid, tlast, input  tready, input clk);
    modport dst  (input  tdata, tvalid, tlast, output tready, input clk);

    // 인터페이스 안에 검사도 넣을 수 있다 -- **한 번 적으면 모든 인스턴스에 걸린다**
    // (아래 assert 문법은 Z18.10)
    property valid_stable;
        @(posedge clk) (tvalid && !tready) | => $stable(tdata);
    endproperty
    assert property (valid_stable)
        else $error("tvalid 중에 tready 없이 tdata 가 바뀌었다");
endinterface

// 쓰는 쪽: 포트 열두 개가 하나가 된다
module producer (axis_if.src out);
    always_ff @(posedge out.clk) begin
        out.tvalid <= 1'b1;
        out.tdata  <= next_byte;
    end
endmodule

module top;
    logic clk;
    axis_if #(W(8)) bus (.clk(clk));
    producer u_p (.out(bus));
    consumer u_c (.in (bus));
endmodule

```

The property embedded in the interface is the point. AXI-Stream requires that once `tvalid` is asserted, the data may not change until `tready` is seen. Written inside the interface, that rule is checked at every instance of the bus in the whole design, automatically. Written in each testbench, it is checked wherever somebody remembered.

Z18.9 Functions and tasks

Table 454. function versus task

	function	task
Can consume time	No — no @, no #, no wait	Yes
Returns a value	Yes	No (use output arguments)
Synthesisable	Yes, if it has no time control	No
C++ analogue	A constexpr-ish pure function	A coroutine — it can block and resume

```
// `automatic` 이 왜 필요한가 -- C++ 프로그래머가 가장 놀라는 자리
//
// Verilog 의 기본은 `static` 이다. 함수의 지역 변수가 **하나만 존재하고
// 모든 호출이 그것을 공유한다.** C 의 함수 안 `static int` 가 기본값인 셈이다.
// 동시에 두 번 불리면 서로의 변수를 덮어쓴다.

function automatic int unsigned clog2_up(int unsigned v);
    int unsigned r = 0;          // automatic 이라 호출마다 새로 생긴다
    while (v > 1) begin v >>= 1; r++; end
    return r;
endfunction
// **테스트벤치 함수에는 늘 `automatic` 을 붙여라.** 안 붙이면 여러
// 스레드에서 부를 때 조용히 틀린 값이 나온다.
// (SystemVerilog 의 `module` 안 기본은 static, `program`/`class` 안은 automatic)

// task: 시간을 쓴다
task automatic send_byte(input logic [7:0] b);
    @(negedge clk);
    tdata <= b;
    tvalid <= 1'b1;
    @(posedge clk);
    while (!tready) @(posedge clk); // **기다린다** -- function 은 못 하는 일
    tvalid <= 1'b0;
endtask
```

Z18.10 The verification half of the language

Everything above describes hardware. The rest of SystemVerilog is a testbench language, and it is much closer to C++ — classes, inheritance, randomisation, dynamic allocation. **None of it synthesises**, and keeping the two halves separate in your head is most of learning the language.

```
// — 어서션: 무엇이 참이어야 하는가를 적는다 —————
// 즉시 어서션 -- 절차 코드 안에서
always_ff @(posedge clk)
    if (!rst) assert (!(full && empty))
        else $error("FIFO 가 full 이면서 empty 다");

// 동시 어서션 -- 시간에 걸친 성질. 이것이 진짜 물건이다
```

```

property req_gets_ack;
    @(posedge clk) disable iff (rst)
        req |-> ##[1:8] ack;      // req 가 뜨면 1~8 사이클 안에 ack 가 온다
endproperty
assert property (req_gets_ack);
cover property (req_gets_ack);   // **정말 일어났는지**도 센다

// |-> 같은 사이클부터 (overlapping)
// |=> 다음 사이클부터 (non-overlapping)
// ##n n 사이클 뒤
// $rose $fell $stable $past edge 와 과거 값

// — 커버리지: 무엇을 **봤는지** 센다 —————
covergroup cg_sample @(posedge clk);
    cp_val: coverpoint x {
        bins neg_sat = {-128};      // 포화 경계를 실제로 밟았나
        bins neg     = {[ -127:-1]};
        bins zero    = {0};
        bins pos     = {[1:126]};
        bins pos_sat = {127};
    }
    cp_pair: cross cp_val, cp_state; // 조합까지 봤나
endgroup
// 어서션이 "틀린 것이 없었나" 라면 커버리지는 "볼 것을 다 봤나" 다.
// 이 책의 변이 점수(Part Z1)는 세 번째 질문에 답한다:
// ***검사가 정말 무는가***. 셋이 서로를 대신하지 못한다.

// — 무작위화 —————
class packet;
    rand bit [7:0] payload [];      // 동적 배열
    rand int      len;
    constraint c_len { len inside {[1:64]};
                    payload.size() == len; }
    // 경계를 **일부러 자주** 뺏는다 -- 무작위만으로는 len==1 이 거의 안 나온다
    constraint c_edge { len dist {1 := 20, [2:8] := 40, [9:64] := 40}; }
endclass

packet p = new();
if (!p.randomize()) $fatal(1, "제약을 못 풀었다");

```

Z18.11 Classes: where SystemVerilog really is C++

```

// 검증 쪽 class 는 C++ class 와 거의 같다. 다른 점만 적는다.
class base_driver;
    // 모든 객체가 **참조**다. C++ 의 `T*` 나 `shared_ptr` 에 해당하고,
    // 값 복사(`=`)는 포인터 복사다. 깊은 복사는 직접 적어야 한다.
    protected virtual axis_if vif;    // `virtual interface` = 인터페이스 핸들

```

```

function new(virtual axis_if vif);    // 생성자 이름이 **언제나 `new`** 다
    this.vif = vif;
endfunction

// `virtual` 은 C++ 과 같은 뜻 (동적 디스패치).
// 다만 **기본이 non-virtual** 이라 붙여야 한다.
virtual task run();
    $display("base");
endtask
endclass

class fir_driver extends base_driver;    // `:` 이 아니라 `extends`
    function new(virtual axis_if vif);
        super.new(vif);                // C++ 의 base(vif) 초기화
    endfunction
    virtual task run();
        super.run();
        // ...
    endtask
endclass

// 없는 것들 (C++ 에서 오면 찾게 된다):
// * 다중 상속 없음. 인터페이스 클래스로 흉내낸다
// * 연산자 오버로딩 없음
// * 소멸자 없음 -- 가비지 컬렉션이다
// * const 멤버 함수 없음
// * 템플릿은 있다: `class stack #(type T = int);` (parameterized class)

```

Z18.12 What is and is not synthesisable

Table 455. The dividing line

Synthesisable	Not synthesisable
always_comb, always_ff, assign	initial (except FPGA memory init), final
if, case, for with constant bounds	while/forever with data-dependent exit
logic, struct packed, packed arrays, enum	class, string, real, queues ([\$]), associative arrays, dynamic arrays
function without time control	task, fork/join, @, #delay
parameter, localparam, generate	\$display, \$fopen, randomize(), assertions (they are <i>checked</i> , not built)
interface, modport	virtual interface, hierarchical references (dut.internal)
\$signed, \$unsigned, \$clog2, \$bits	\$random, \$time, \$past

The practical test, and the one this repository uses: if yosys reads it and verilator `--lint-only -Wall` is quiet, it is synthesisable. Both are free, both run in seconds, and edu/house/synth.sh runs them in that order for exactly this reason — Part Z20 walks that script line by line.

Z18.13 The house style, in ten rules

Table 456. Rules worth enforcing with a linter, and why each exists

Rule	The failure it prevents
<code>always_ff/always_comb</code> , never bare <code>always</code>	Accidental latches and missing sensitivity items.
<code><=</code> in <code>always_ff</code> , <code>=</code> in <code>always_comb</code> , never mixed	Simulation/synthesis mismatch and evaluation-order races.
One signal is driven by exactly one block	Multiple drivers — produces <code>x</code> in simulation and a short in silicon.
Always declare <code>signed</code> when the value is signed, and wrap concatenations in <code>\$signed()</code>	The half-the-input-space bug of Z18.3.
Name every <code>generate</code> block	Hierarchical names shifting when a loop bound changes.
Connect ports by name, never by position	Silent mis-connection when a port is inserted.
One module per file, file named after the module	Verilator's <code>DECLFILENAME</code> ; also how every build tool finds things.
No <code>casex</code> ; use <code>casez</code> or explicit comparison	<code>x</code> treated as don't-care, differing pre- and post-synthesis.
Reset every flop that needs a known initial value, and say which ones do not	Simulation starting in <code>x</code> and hardware starting in whatever.
No magic numbers for widths — use <code>parameter</code> and <code>\$clog2</code>	The width change that gets made in four of five places.

Nine of those ten are checked automatically by `verilator --lint-only -Wall`. Running it costs under a second and it is the highest-yield thing in the entire flow — which is why it is step one of `synth.sh` and not an afterthought.

Z19. The C++ Used for Hardware, Keyword by Keyword

The model layer in `edu/model/` is about 250 lines of C++, and almost every line uses a keyword that means something slightly different when the output is a circuit rather than a program. This part goes through them. It assumes you can write C++ and explains what each construct *costs in hardware*, which is the part no C++ book covers.

Z19.1 `inline` — and why it matters more here

```
static inline int32_t asr(int32_t v, int n) {
    return (v < 0) ? ~((-v) >> n) : (v >> n);
}
```

Table 457. What `inline` actually does

Belief	Reality
"It tells the compiler to inline the function."	It is a hint , and modern compilers ignore it for inlining decisions. <code>-O2</code> inlines small functions whether or not you write it.
"It is about speed."	Its actual required meaning is a linkage rule: it permits the same function to be defined in multiple translation units without a duplicate-symbol error. That is why it is mandatory for a function body in a header.
"In HLS it makes things faster."	In HLS it changes what hardware is built . An inlined function is flattened into the caller's datapath; a non-inlined one may become a separate module that is <i>shared</i> between call sites, with arbitration logic.

That last row is the hardware-specific part. If four call sites share one non-inlined multiply-accumulate, the tool builds one instance and a state machine that takes turns — small and slow. Inline them and you get four instances in parallel — large and fast. In C++ this is a performance micro-decision; in HLS it is an architecture decision.

In `edu/model/` everything is `static inline` in headers because the same header is compiled by `g++` (where the linkage rule is what matters) and by `bambu` (where full flattening into one datapath is what we want). `static` here means internal linkage — the function is private to each translation unit — and combined with `inline` it is the conventional way to put a small helper in a header.

Z19.2 The five meanings of `static`

C++ reused one keyword for several unrelated jobs. Three of them appear in the model code, and one of them is a bug waiting to happen in HLS.

Table 458. `static`, by position

Where it appears	What it means	Hardware consequence
On a function at namespace scope <code>static inline int32_t asr(...)</code>	Internal linkage: not visible to other translation units.	None — a linker concept.
On a variable at namespace scope <code>static const int32_t TH[4] = {...}</code>	Internal linkage, plus (with <code>const</code>) a compile-time-known value.	Large. Constant coefficients let the tool replace multipliers with shift-adds. See Z19.4.

Where it appears	What it means	Hardware consequence
On a local variable inside a function <code>static int count = 0;</code>	One instance, shared by all calls, persisting between them.	Becomes state. In HLS this infers a register or a RAM that survives across invocations — usually not what you meant, and it silently makes a pure function impure.
On a class member	Shared by all objects.	Not used in synthesisable code.
In an anonymous namespace (the modern alternative)	Same as internal linkage for names.	None.

The third row is the trap. A C++ programmer writes `static` inside a function to keep a counter cheaply. In HLS that counter becomes a register whose value carries between calls, so the generated block is no longer a function of its inputs — and the golden-model comparison that passed in C++ can fail in RTL, or worse, pass in both while the block is unusable because two instances would interfere. **In synthesisable C++, a function-local `static` is almost always a mistake.**

Z19.3 `const`, `constexpr`, and `enum`

// 이 저장소가 실제로 쓰는 세 가지, 그리고 왜 각각인가

// (1) `enum` -- 정수 상수를 묶는다. ``ipmodel.h`` 가 이걸 쓴다

```
enum {
    SAMP_F = 7,
    COEF_F = 7,
    ACC_F = SAMP_F + COEF_F,    // 다른 멤버를 참조할 수 있다
    NTAP = 4
};
```

// 왜 `enum` 인가: ****저장 공간을 절대 차지하지 않는다****. 주소를 가질 수 없으므로
 // 컴파일러가 메모리를 잡을 여지가 없다. 옛날 C++ 에서 "enum hack" 이라
 // 불리던 관용구인데, HLS 에서는 여전히 가장 안전한 컴파일 시간 상수다.

// (2) `static const` 배열 -- 값 여러 개를 묶을 때

```
static const int32_t FIR_H[NTAP] = { -18, 111, 111, -18 };
// 배열은 enum 으로 못 만든다. `static const` 가 대신한다.
// **이 값이 컴파일 시간에 보이는 것이 면적의 핵심이다** (Z19.4).
```

// (3) `constexpr` -- 컴파일 시간 계산이 필요할 때

```
constexpr int32_t sat_hi(int n) { return (1 << (n - 1)) - 1; }
static const int32_t HI = sat_hi(8);    // 127, 컴파일 시간에 계산됨
// `const` 는 "안 바꾼다" 이고 `constexpr` 은 "컴파일 시간에 값을 안다" 이다.
// 둘은 다르다:
const int a = rand();    // 합법. 런타임 값인데 안 바뀔 뿐
constexpr int b = rand(); // **컴파일 오류**. 컴파일 시간에 알 수 없다
```

Table 459. Which to use, in synthesisable C++

Need	Use	Why not the others
A single integer constant	<code>enum</code> or <code>constexpr</code>	<code>static const int</code> also works but may take storage if its address is ever taken.

Need	Use	Why not the others
An array of constants	<code>static const T[]</code>	enum cannot hold arrays. Mark it <code>const</code> or the tool must assume it can change and builds a RAM.
A value computed from other constants	<code>constexpr</code> function	A normal function may not be evaluated at compile time, so the result is not a constant to the HLS tool.
A run-time-settable coefficient	A function parameter, deliberately	This is the opposite choice and it costs real multipliers — make it consciously.

Z19.4 Why `const` on the coefficients is worth hundreds of gates

Worked example 72. Constant multiply versus variable multiply

Given. The filter computes `a1 * 111`. In one version 111 is a `static const`; in the other it arrives as a function parameter.

Method. A constant multiply can be expanded into shifts and adds at compile time. $111 = 64+32+8+4+2+1$, so it is a sum of six shifted copies — or, better, $111 = 128-16-1$, which is three terms.

Numbers. Constant: about 2 adders and 1 subtractor of 9–17 bits — roughly 40–50 LUT4 on iCE40. Variable: a full 9×8 array multiplier, roughly 150–200 LUT4, or a hard DSP block where one exists.

The design consequence for a sellable IP: if customers need programmable coefficients, you pay for real multipliers and you must say so in the datasheet. If the coefficients are fixed at build time, you can offer a block a quarter the size. These are two different products, and the difference is one `const`.

How this is got wrong. The trivial explanation to kill is “the tool is smart.” It is not being smart; it is being given different information. Make the coefficient a parameter and the same tool produces the big version, because it now must handle every possible coefficient value. The measured evidence is in Z15: the tuned design used 97 `SB_CARRY` and **zero** hard multipliers, because every multiply had a compile-time-constant operand.

Z19.5 Fixed-width types, and integer promotion

```
#include <stdint.h>      // int8_t, int16_t, int32_t, uint32_t ...

// `int` 은 쓰지 마라 -- 크기가 구현마다 다르고, HLS 에서는 **포트 폭이 된다**.
// Z15 의 실측: 인자를 `int` 로 두면 포트가 32비트가 되어 면적이 3.6배가 됐다.

// 정수 승격 (integer promotion) -- C++ 이 조용히 32비트로 올리는 규칙
int8_t  a = 100, b = 100;
auto    c = a + b;        // c 의 타입은 int8_t 가 **아니라 int** 다!
// a+b = 200 이고, int8_t 였다면 -56 으로 넘쳤을 것이다. int 라서 200 이 된다.
//
// 이것이 모델과 RTL 이 갈라지는 대표적인 자리다:
// C++ 모델: 200 (int 로 승격돼서)
// RTL:      -56 (8비트로 짤으면)
// **모델이 하드웨어보다 관대하다.** 그래서 `ipmodel.h` 는 폭을 손으로
// 못박는다 -- `sat(v, 8)` 이 그 일을 한다.

// 규칙: `char`·`short`·`bool` 과 그 unsigned 판은 산술 연산 전에
// 전부 `int` 로 올라간다. `int` 보다 큰 타입은 안 올라간다.

// 부호 있는 것과 없는 것을 섞으면 더 나쁘다
```

```
int32_t x = -1;
uint32_t y = 1;
bool z = (x < y);    // **false** 다. x 가 unsigned 로 변환돼 4294967295 가 된다
// 컴파일러가 -Wsign-compare 로 경고한다. **경고를 켜고 오류로 다뤄라.**
```

Z19.6 References, pointers, and what they become

// C++ 의 세 가지 전달 방식과, HLS 가 각각 무엇으로 만드는가

```
int32_t f1(int32_t x);           // 값 전달 -> 입력 포트 하나
int32_t f2(const int32_t *x);    // 포인터 -> **메모리 인터페이스**
int32_t f3(const int32_t &x);    // 참조 -> 값 전달과 같게 취급되는 경우가 많다
void f4(int32_t &out);           // 비-const 참조 -> 출력 포트
```

// 포인터가 위험한 이유 -- `fir\_top.cpp` 가 그래서 스칼라 넷을 받는다:

```
//
// int fir_top(const int *x) -> bambu 가 주소·요청·응답 포트를 만든다.
//                               재는 것이 FIR 이 아니라 메모리 컨트롤러가 된다
// int fir_top(int,int,int,int) -> 포트 넷. 데이터패스만 남는다
//
// 그래서 모델 안에서는 배열(`const int32_t *x`)을 쓰고,
// **HLS 최상위에서만** 스칼라로 퍼서 넘긴다. 안쪽은 인라인되므로
// 포인터가 남지 않는다.
```

```
// 합성 가능한 C++ 에서 **없는** 것들:
// * 동적 할당 (new/delete/malloc) -- 하드웨어에 힘이 없다
// * 재귀 -- 깊이가 컴파일 시간에 안 정해지면 회로 크기를 못 정한다
// * 가상 함수 -- 런타임 디스패치는 하드웨어에 없다
// * 함수 포인터 -- 같은 이유
// * 예외 (try/catch) -- 스택 되감기가 없다
// * std::vector, std::string, std::map -- 전부 동적 할당이다
```

The list of absences is the real lesson. Synthesisable C++ is a *subset*: fixed-size data, no dynamic anything, no indirection that cannot be resolved at compile time. What remains is essentially C with templates — which is why the model layer looks the way it does, and why a C++ programmer's instincts about abstraction have to be held in check.

Z19.7 Templates — the one abstraction that survives

```
// 템플릿은 **컴파일 시간에 전부 풀린다**. 그래서 합성된다.
// 하드웨어에서 템플릿 인자는 곧 **비트폭과 개수**다.
```

```
template<int W>
static inline int32_t sat_t(int32_t v) {
    constexpr int32_t hi = (1 << (W - 1)) - 1;
    constexpr int32_t lo = -hi - 1;
    return v > hi ? hi : (v < lo ? lo : v);
}
// sat_t<8>(x) 와 sat_t<18>(x) 는 **서로 다른 하드웨어**를 낳는다.
```

```
// C++ 의 `template<int>` 가 SystemVerilog 의 `parameter` 와 같은 자리다.

// 탭 수까지 템플릿으로 빼면 4탭과 16탭이 한 소스에서 나온다
template<int N>
static inline int32_t fir_n(const int32_t *x, const int32_t *h) {
    int32_t acc = 0;
    for (int i = 0; i < N; i++) acc += x[i] * h[i];    // N 이 상수 -> 언롤됨
    return acc;
}

// 그런데 대가가 있다 -- 실측이다.
// survey/synth/합성결과.md: `ap_fixed` 로 된 복소 8x8 Cholesky 는
// Bambu 프론트엔드에서 **40분 타임아웃(rc=124)** 이 났다.  템플릿 깊이가
// 깊어지면 HLS 도구가 프론트엔드에서 죽는다.
//
// 규칙: **얕은 템플릿은 공짜, 깊은 템플릿은 도구를 죽인다.**
// `template<int W>` 정도는 안전하고, 템플릿이 템플릿을 부르는 수학
// 라이브러리는 위험하다.
```

Z19.8 Namespaces and ::, the C++ side

```
namespace fir {
    enum { NTAP = 4 };
    static const int32_t H[NTAP] = { -18, 111, 111, -18 };
    static inline int32_t tap(const int32_t *x);
}

// 쓰는 쪽 -- SystemVerilog 의 package 와 문법이 거의 같다
int32_t y = fir::tap(x);
int      n = fir::NTAP;

using namespace fir;    // SV 의 `import fir_pkg::*;`
int32_t y2 = tap(x);

// `::` 의 다른 얼굴들
::global_thing          // 전역 네임스페이스 (SV 의 $unit:: 에 해당)
std::int32_t             // 표준 라이브러리
MyClass::method          // 클래스 멤버
MyClass::Nested::value   // 중첩
enum class E { A };      E::A // 범위 있는 enum (SV 의 state_e::IDLE 과 같다)
```

Table 460. ::, side by side

Purpose	C++	SystemVerilog
Named scope	namespace fir { } → fir::H	package fir_pkg; endpackage → fir_pkg::H
Bring it all in	using namespace fir;	import fir_pkg::*;

Purpose	C++	SystemVerilog
Scoped enum member	enum class E{A}; → E::A	typedef enum {A} e_t; → e_t::A
Standard library	std::	std:: (mailbox, semaphore)
Global scope	::name	\$unit::name
Class member	C::m	C::m (verification classes)
Not a thing	—	There is no -> for scope; SystemVerilog uses . for instance members and hierarchy

So the answer to “I only know C++, what is ::?” is: it is the same operator you already know, and SystemVerilog borrowed it deliberately. The construct behind it — package — is the direct analogue of namespace, with one difference that matters: a SystemVerilog package must be *compiled before* its users, because there is no two-pass name resolution and no forward declaration.

Z19.9 The flags the build actually uses

골든모델을 컴파일할 때

```
g++ -O2 -I. -o vectors vectors.cpp
```

```
# -O2    최적화.    등가성 전수 확인이 29.7초가 된 것이 이 덕이다.
#       -O0 이면 수 분이 걸린다 -- 그러면 전수를 포기하게 되고,
#       포기하면 표본으로 내려간다.  **최적화 플래그가 검증 전략을 바꾼다.**
# -I.    헤더 탐색 경로.  `fir.h` 를 찾는다
```

```
# 커 둘 만한 경고 -- 모델과 RTL 이 갈라지는 자리를 짚는다
g++ -O2 -Wall -Wextra -Wsign-compare -Wconversion -Werror ...
# -Wconversion    폭이 줄어드는 암묵 변환을 경고한다.  HLS 에서 이것이
#                  곧 비트 잘림이다
# -Wsign-compare   Z19.5 의 부호 섞임
# -Werror          경고를 오류로.  경고는 안 읽히고 오류는 읽힌다
```

```
# HLS 로 넘길 때 -- 같은 헤더, 다른 도구
bambu --top-fname=fir_top_tuned -I/home/user/SE/edu/model \
      --clock-period=3 --reset-level=high --reset-type=sync \
      fir_top_tuned.cpp
```

```
# --top-fname      어느 함수가 최상위 모듈이 되는가
# -I              g++ 와 **같은** 경로.  같은 헤더를 읽는다는 뜻이다
# --clock-period   목표 주기(ns).  Z15 에서 이 숫자 하나로
#                  마이크로아키텍처 다섯 개를 얻었다
# --reset-level    active high/low.  **기본값이 low 라서** 한 번 물렸다
# --reset-type     sync/async/no
```

The two commands read the same header with the same include path. That is the whole design of the model layer in one line of shell: if the paths ever diverge, the golden model stops being the thing that was synthesised, and the comparison becomes decoration.

Z20. The Free Toolchain, Command by Command

Every number in Parts Z14–Z19 was produced by tools that cost nothing and install from a package manager. A one-person design house can take a block from C++ to a placed-and-routed netlist with a timing number without a single licence. This part is how, command by command, with the real output each one produces.

Table 461. The toolchain, as measured in this container

Tool	Version here	Job	Licence
Verilator	5.020	Lint, and fast C++-based simulation	LGPL-3 / Artistic-2
Icarus Verilog	12.0	Event-driven simulation of the full language	GPL-2
Yosys	0.33	Synthesis: RTL → gate netlist	ISC
nextpnr-ice40	0.6	Place and route, static timing	ISC
PandA Bambu	2024.10	High-level synthesis: C/C++ → RTL	GPL-3
GCC	g++ -O2	Compiles the golden model	GPL-3

Two things are missing from that list and it is worth being honest about them. There is **no free sign-off static timing analyzer for ASICs** (OpenSTA exists but was not available here), and there is **no free formal equivalence checker**. For FPGA work the gap does not bite; for an ASIC tape-out it does, and Part Y13 covers what that means commercially.

Z20.1 Step one: lint, because it is the cheapest

```
verilator --lint-only -Wall --top-module fir4_hand edu/model/fir4_hand.v
```

Table 462. What the flags do

Flag	Meaning	Why
--lint-only	Parse and check; do not build a simulator.	Runs in well under a second. This is the whole point.
-Wall	Enable all the style and width warnings.	The default set is too quiet. -Wall is what industry actually turns on.
--top-module	Name the top of the hierarchy.	Without it, a file with several modules gives confusing 'no top' errors.

The warnings that matter most, and what each is telling you:

Table 463. Verilator warnings you will actually see

Warning	Means	Take it seriously?
WIDTHEXPAND / WIDTHTRUNC	An assignment or operation changes bit width implicitly.	Yes, always. This is the class of bug Z16.7 spent five derivation steps preventing. Every silent truncation lives here.
UNUSED SIGNAL	A signal is declared or driven and never read.	Usually. Either it is dead logic (remove it) or you forgot to connect something. Suppress deliberately with a comment when a port is genuinely reserved.
LATCH	Combinational block does not assign in all paths.	Yes. Almost never intended. See Z18.5.
MULTIDRIVEN	Two blocks drive one signal.	Yes. This is a hard bug: x in simulation, contention in silicon.

Warning	Means	Take it seriously?
BLKSEQ / COMBDLY	Blocking assignment in sequential logic, or vice versa.	Yes. The race of Z18.4.
DECLFILENAME	Module name does not match file name.	Cosmetic, but fix it — every build system assumes the convention.
GENUNNAMED	Unnamed generate block.	Fix it. Hierarchical names shift when loop bounds change.
SYNCASYNCNET	A net is used as both sync and async reset.	Investigate. Usually a real reset-architecture problem.

Worked example 73. Reading a lint report on generated code

Given. Running the command above on Bambu's generated `fir_top.v` returns 19 warnings: 7 `WIDTHEXPAND`, 6 `UNUSED SIGNAL`, 3 `WIDTHTRUNC`, 1 `UNUSED PARAM`, 1 `SYNCASYNCNET`, 1 `DECLFILENAME`.

Method. Separate warnings that indicate a defect from warnings that indicate a generator's house style.

Numbers. Effectively **all 19** are style, not defects. Generated RTL uses parameterised width-adaptor modules, so `WIDTHEXPAND` and `UNUSED SIGNAL` appear wherever a generic module is instantiated narrower than its maximum.

The same reasoning applies to hand-written RTL with a different conclusion: there, a clean `-Wall` report is achievable and should be a release gate. `edu/model/fir4_hand.v` is clean.

How this is got wrong. The wrong conclusion is “lint is useless on generated code.” The right one is that you must **lint the generator's output once**, triage it, and then **watch for new warning types**. A `WIDTHTRUNC` that was not there yesterday is a real signal; the same six `UNUSED SIGNAL` every build is noise. Counting warnings by type — which is what `synth.sh` prints — makes the difference visible.

Z20.2 Step two: simulate

```
# 컴파일 -- 소스 여러 개를 한 실행파일로
iverilog -g2012 -DDUT=fir4_hand -o a.out edu/model/fir_tb.v edu/model/fir4_hand.v

# -g2012 SystemVerilog-2012 문법을 켜다. **안 켜면 always_ff 부터 막힌다**
# -D 매크로 정의. C 의 -D 와 같다. 여기서는 테스트벤치 하나로
# 손RTL 과 HLS RTL 을 갈아 끼우는 데 쓴다
# -o 출력 파일
# -s <톱> 톱 모듈 지정 (여러 개일 때)

# 실행 -- vvp 가 iverilog 의 가상머신이다
vvp a.out

# 결과
# 잔것 400 틀림 0

# 파형을 보고 싶으면 테스트벤치에 이것을 넣는다
# initial begin $dumpfile("w.vcd"); $dumpvars(0, tb); end
# 그리고 gtkwave 로 연다 (이 컨테이너에는 없다 -- GUI 가 없으므로)
```

Verilator can also simulate, and it is 10–100× faster than Icarus because it compiles to C++ rather than interpreting events. The trade is that Verilator's simulation is 2-state and cycle-based: it will not show you x propagation, which is exactly the thing you want to see when debugging a reset problem. **Use Icarus to find the bug and Verilator to run the regression.**

Z20.3 Step three: synthesise with Yosys

```
yosys -p "read_verilog -sv edu/model/fir4_hand.v; \
        hierarchy -check -top fir4_hand; \
        proc; opt; fsm; opt; memory; opt; techmap; opt; \
        stat"
```

Table 464. The Yosys script, pass by pass

Pass	What it does
read_verilog -sv	Parse. -sv enables SystemVerilog; without it a lways_ff is a syntax error.
hierarchy -check -top M	Resolve the module tree from M downward. -check makes a missing module an error rather than a black box — without it, a typo in a module name silently synthesises to nothing.
proc	Convert always blocks into multiplexers and registers. This is where <= becomes a flip-flop.
opt	Constant folding, dead-code removal, simplification. Run it between every other pass; it is cheap.
fsm	Detect state machines and re-encode them.
memory	Turn array accesses into RAM primitives.
techmap	Map to generic gates (\$_AND_, \$_XOR_, ...).
stat	Print the cell count. Do not pass -q to yosys — it swallows this output, which was measured here the hard way.

The real output for the hand-written filter:

```
=== fir4_hand ===
```

```
Number of wires:           268
Number of wire bits:       1681
Number of public wires:    14
Number of public wire bits: 97
Number of cells:           780
  $_AND_                   340
  $_MUX_                   15
  $_NOT_                   10
  $_OR_                    142
  $_SDFF_PP0_              9
  $_XOR_                   264
```

Reading that report without guessing

	Step	Justification
1	<code>\$_SDFF_PP0_</code> × 9 — nine synchronous flip-flops, positive edge, positive reset, reset-to-0.	<i>The design has exactly nine state bits: <code>done_port</code> (1) plus <code>return_port</code> (8). The number matching the count you expected is the first thing to check — a surprise here means inferred state you did not intend.</i>
2	<code>\$_XOR_</code> × 264 dominates the combinational logic.	<i>XOR is the sum output of a full adder (Z17.6). Two 9-bit adds plus a 17-bit add plus the constant-multiply shift-add network accounts for it.</i>
3	<code>\$_AND_</code> × 340 and <code>\$_OR_</code> × 142 are the carry logic.	<i><code>cout = (a AND b) OR (cin AND (a XOR b))</code> — two ANDs and an OR per bit.</i>
4	<code>\$_MUX_</code> × 15 is the saturation.	<i>Two comparisons selecting between three 8-bit values. This is the gate-level price of saturating rather than wrapping, as Z17.7 predicted.</i>
5	But 780 generic cells is not the area of the design.	<i>These are technology-independent gates. The device has 4-input lookup tables and a dedicated carry chain, so the mapping in the next step collapses this dramatically — 780 generic cells became 257 iCE40 cells. Never quote generic cell counts as area.</i>

Z20.4 Step four: map to a real device

```
yosys -p "read_verilog -sv fir4_hand.v; \
        synth_ice40 -top fir4_hand -json fir4_hand.json; \
        stat"
```

```
# synth_ice40 은 위의 긴 스크립트를 **대신한다** -- 읽기부터 기술 매핑까지
# 한 명령이다. 다른 판을 쓰면 명령만 바꾼다:
# synth_ice40      Lattice iCE40
# synth_ecp5       Lattice ECP5   (더 크다. nextpnr-ecp5 와 짝)
# synth_gowin      Gowin
# synth_xilinx     Xilinx 7-series (nextpnr-xilinx 는 실험적이다)
# synth_intel      Intel/Altera
# ABC9 기반 매핑을 쓰려면 `-abc9` 를 붙인다 (보통 더 작고 빠르다)
```

```
=== fir4_hand ===
Number of cells:          257
  SB_CARRY                 48
  SB_DFFSR                 9
  SB_LUT4                  200
```

Now the numbers mean something physical. SB_LUT4 is a 4-input lookup table, SB_CARRY is one bit of the dedicated carry chain, SB_DFFSR is a flip-flop with set/reset. The 48 carry cells are the adders using hardware the fabric provides for free — which is why the generic 780 collapsed to 257, and why the gate-counting exercise in Z17.6 explains the *structure* but not the *number*.

Z20.5 Step five: place, route, and get a real frequency

```
nextpnr-ice40 --hx8k --package ct256 --json fir4_hand.json \
    --pcf-allow-unconstrained --placer heap --freq 300 --seed 1
```

Table 465. The flags that change the answer

Flag	Meaning	Why it matters
--hx8k --package ct256	Device and package.	Pin count is a real constraint. Measured here: up5k/sg48 has 39 I/O and refused to place a design needing 43, with Unable to find a placement location. That is not a quality problem with the design.
--pcf-allow-unconstrained	Allow pins with no physical assignment.	Needed when you are measuring a block rather than building a board.
--placer heap	Analytic placer.	Better and more repeatable than the default SA placer.
--freq 300	Target frequency in MHz.	Without this the target defaults to 12 MHz and the tool stops optimising the moment it passes — the reported Fmax then reflects routing luck, not the design.
--seed N	Placement random seed.	Run three and report the spread. A single seed is a sample of one.

Info: ICESTORM_LC: 211/ 7680 2%

Info: Max frequency for clock 'clock\$SB_IO_IN\$glb_clk': 455.79 MHz (PASS at 12.00 MHz)

Worked example 74. Why that 455.79 MHz was not usable and 47 MHz was

Given. Placing the hand-written filter directly gives 455.79 MHz. Wrapping it in a registered timing shell and re-placing gives 47.4 MHz. Same design.

Method. Ask which paths the tool is measuring in each case.

Numbers. Without the shell the design has 9 flops, all at the output. The only register-to-register paths are trivial, and the real datapath runs from an **input pin** to those flops — unconstrained, therefore unreported. With the shell, every path starts and ends at a flop, so the 47.4 MHz is the actual combinational depth of the filter.

The general rule: **a frequency number is meaningless unless you can say which two registers the path runs between.** Ask that question of every Fmax anyone quotes you, including your own.

How this is got wrong. The 455.79 MHz figure is stable across seeds and looks entirely credible. A reproducible measurement of the wrong path is the most dangerous kind, because every instinct says to trust it. What exposed it was a contradiction, not a tool warning: adding pipeline registers made the reported frequency *fall*, and more registers cannot make a design slower.

Z20.6 Step six: HLS, if you are generating the RTL

```
bambu --top-fname=fir_top_tuned \
    -I/home/user/SE/edu/model \
    --clock-period=3 \
    --reset-level=high --reset-type=sync \
    fir_top_tuned.cpp
```

```
# --top-fname      최상위가 될 C 함수.  **이것이 모듈 이름이 된다**
# -I              헤더 경로.  g++ 와 **같은 경로**를 줘야 같은 모델이 된다
# --clock-period   목표 주기 (ns).  스케줄링이 여기에 맞춰진다.
```

```
#          Z15 에서 10·6·3·2·1 로 쓸어 무릎을 찾았다
#  --reset-level    high / low.  **기본이 low 다**  -- 집 관례와 다르면 여기서 맞춘다
#  --reset-type     sync / async / no
#  --simulate       내장 테스트벤치로 돌려 본다 (골든이 따로 있으면 안 써도 된다)
#  --device-name    목표 소자.  타이밍 추정에 쓰인다

# 안 되는 것들 -- survey/synth/합성결과.md 의 1변수 대조
#  #pragma HLS ...      rc=11.  Xilinx 방언을 안 받는다
#  hls::stream         rc=1.  FixStructsPassedByValue 에서 크래시
#  깊은 ap_fixed 템플릿 rc=124.  프론트엔드 40분 타임아웃
```

Z20.7 One command that runs all of it

edu/house/synth.sh is the whole flow in one script. It is written to be read as well as run, and two of its choices came from being wrong first.

```
bash edu/house/synth.sh <file.v> <top_module> [--pnr]
```

예

```
bash edu/house/synth.sh edu/model/fir4_hand.v fir4_hand --pnr
```

출력

```
# == 1. verilator lint ==
#   경고+오류 0
#   lint rc=0
# == 2. yosys 기술독립 합성 ==
#   Number of cells: 780 ...
# == 3. yosys iCE40 매핑 ==
#   SB_LUT4 200, SB_CARRY 48, SB_DFFSR 9
# == 4. nextpnr 배치·배선 ==
#   ICESTORM_LC: 211/7680,  Max frequency 47.4 MHz
```

Table 466. Two things the script does that look fussy and are not

Choice	What went wrong without it
Tool output goes to a file , then is filtered — never piped straight into head.	head closes the pipe, the tool dies of SIGPIPE, and the exit code comes back 141 . A lint that passed and a lint that crashed became indistinguishable. Measured here.
Cell counts are extracted from inside the stat block, not by grepping the whole log.	Yosys prints Generating RTLIL representation for module \SB_LUT4 while loading its cell library. A naive grep SB_LUT4 counts those lines as if they were cells.

Both are the same failure in different clothes: **a measurement harness that reports a plausible number when it has actually failed**. That is the defect this whole book is organised against, and it turns out to be just as easy to build into a twelve-line shell script as into a verification environment.

Z20.8 Installing the lot

Debian / Ubuntu -- 앞의 넷은 패키지가 있다

```
sudo apt install verilator yosys nextpnr-ice40 iverilog gtkwave
```

```

# Bambu 는 패키지가 없다. 소스에서 짓는다 (이 컨테이너에서 한 일이다).
#
# 이 줄만은 배포가 안 깎다 -- 그리고 그것이 맞다. 이유 셋:
# 1. Bambu 는 **선택**이다. 이 책의 08단계까지 한 번도 안 쓴다
# 2. 소스 빌드가 한 시간대다. 배포마다 이것을 하면 배포가 못 끝난다
# 3. 이 줄은 봇이 돌릴 명령이 아니라 **읽는 사람의 데스크톱** 지시다
# 그래서 배포에 넣지 않고 표식을 단다. 조용히 넘어가는 것이 아니라
# 눈에 보이게 고르는 것이다.
sudo apt install gcc-multilib libc6-dev-i386 gcc-13-plugin-dev \ # G021: 사람 몫
                  clang-16 llvm-16-dev libclang-16-dev \
                  autoconf automake libtool
git clone --recursive https://github.com/ferrandi/PandA-bambu.git
# 막혔던 자리들 (survey/synth/합성결과.md 에 전부 적혀 있다):
# * --depth 1 로 클론하면 서브모듈이 빠져 Makefile.in 이 없다
# * ext/Coin-Cbc 와 ext/lpsolve5 는 선택인데도 automake 가 디렉터리 존재를 요구한다
# -> 빈 디렉터리를 만들면 넘어간다
# * clang 은 16 이어야 한다. 18 은 너무 새것이라 configure 가 거른다

```

The install list is short and the whole thing fits on a laptop. That is the point of this part: the barrier to running a real IP flow is not money any more. What the free tools do not give you is ASIC sign-off timing, formal equivalence, and somebody to call — and those are exactly the things Part Y13 says to budget for when a block stops being an exercise and starts being a product.

Z21. Building Protocol IP When You Cannot Get the Spec

Every previous part of this volume built a block whose definition was ours to choose. A protocol block is different: its correctness is defined by a document somebody else wrote, and for the protocols a design house most wants to sell into — MIPI CSI-2, PCIe, Ethernet — that document is expensive, membership-gated, or both.

This part is about what to do then. It is not a workaround chapter. The discipline it describes — separate what you can prove from what you have assumed, and publish the boundary — is the same discipline you need *with* the spec in hand, because having the document does not mean you implemented it correctly.

Z21.1 The honest starting position

The container this book is built in has no outbound network: a `CONNECT` to `ieee802.org` returns 403. So the normative tables for 8b/10b and the MIPI packet-header ECC were not available while the blocks in Parts Z22 and Z23 were written. That is stated first, because everything else in those parts has to be read against it.

There are exactly three things you can do in that position, and only the third is defensible:

Table 467. Three responses to a missing specification

Response	What it produces	Verdict
Write the constants from memory and ship	A block that looks finished and is wrong in ways nobody will notice until integration.	No. This part demonstrates concretely why — three sets of remembered constants were wrong, and one was not even a valid code.
Refuse to build anything	Nothing. The structure, the verification harness and the microarchitecture are all independent of the exact table values.	No. It throws away the 90 % of the work that does not depend on the document.
Build it, prove everything provable, and publish the boundary	A block that is correct by construction on every property you can state, with a written list of what remains unverified and how to close it.	Yes. This is what Parts Z22–Z23 do.

Z21.2 What “provable without the document” means

A protocol constant is not arbitrary. It was chosen to make the code satisfy properties — and those properties are usually stated in the open literature even when the tables are not. That gives you a check with real teeth: **assert the properties the constants exist to provide, and test them exhaustively.**

Table 468. The two blocks, and what each property buys

Block	Property asserted	What a wrong constant would do
8b/10b	Bijjective within each running disparity	Two data bytes encode to the same codeword → the link cannot decode at all.
	Running disparity stays in {−1, +1}	DC wander on an AC-coupled link → the receiver's slicer drifts off centre.
	No more than 5 identical bits in a row, across code-word boundaries	The CDR loses lock. This is the property that caught two of the three defects below.
	The comma pattern never occurs in data	The receiver aligns to a false byte boundary and every subsequent byte is garbage.
24+6 ECC	All 29 single-error syndromes distinct and non-zero	Two different single-bit errors look identical → the decoder ‘corrects’ the wrong bit.

Block	Property asserted	What a wrong constant would do
	No double-error syndrome equals a single-error syndrome	A 2-bit error is silently 'fixed' into a 3-bit error. Worse than no ECC.

Every one of those is checkable by enumeration on a laptop in under a second, and every one of them failed at least once during this work.

Z21.3 What the checks actually caught

Six defects, all found by machine, none found by reading the code. They are listed in full because the pattern matters more than any one of them.

Table 469. Every defect found while building the two protocol blocks

Where	What was wrong	How it surfaced
enc8b10b.h API	새 RD(-1 또는 +1)를 돌려주면서 -1 을 오류 코드로도 썼다. 부르는 쪽이 성한 결과 절반(528 중 264)을 버렸다.	proto_check 가 첫 실행에서 코드워드 개수가 절반이라고 알렸다
3b/4b 표 압축	'불균형 0 이면 RD 양쪽이 같은 코드' 라고 가정해 한 벌만 적었다. 틀렸다 -- 불균형 0 인데 다른 짝이 있고, 그 선택은 DC 균형이 아니라 연속비트가 정한다.	P3 이 1082 군데, P4 가 2048 번 깨졌다
5b/6b D.07	같은 가정. D.07 은 111000/000111 로 둘 다 불균형 0 인데 다르다.	P3 이 556 군데에서 깨졌다. D7.7 이 RD+1 에서 1110000001 -- 0 이 여섯 개
기억하고 있던 CSI-2 ECC 마스크	신드롬 30 개 중 서로 다른 것이 27 개. 단일오류 정정 부호조차 아니었다.	쓰기 전에 신드롬 distinctness 를 썼다. 그래서 **안 실행**
ecc24_dec 의 전 체패리티	다시 부호화해 여섯째 비트를 비교했더니 신드롬 자신의 패리티가 섞였다. 신드롬 무게가 홀수인 단일오류 90/180 이 이중오류로 오인됐다.	전수 단일오류 확인이 절반 실패
tb.v 의 자극	valid 가 0 인 구간을 아예 안 밟아, code_valid 를 가짜로 띄우는 변이가 통과했다.	변이 점수 71.4 % 의 탈출 하나. 놀림 사이클을 관찰하게 고쳐 100 % 가 됐다

Worked example 75. The defect that would have shipped

Given. The MIPI CSI-2 packet-header ECC parity masks, written from memory, were about to go into both the C++ model and the RTL.

Method. Before writing any code with them, count the syndromes: each of the 24 data bits and 6 parity bits must produce a distinct, non-zero syndrome, or the code cannot even correct single errors.

Numbers. 27 distinct syndromes out of the 30 required, and 96 pairs where a double-bit error is indistinguishable from a single-bit error.

The check that caught it is four lines of Python and it ran before any Verilog existed. The cost of not running it would have been discovered at a customer's integration bench, months later, as 'your ECC sometimes makes things worse'.

How this is got wrong. The tempting reading is "close enough, it is mostly right." It is not mostly right — it is **not a single-error-correcting code at all**. Three of the 24 data bits share a syndrome with another bit, so roughly one single-bit header error in ten would be 'corrected' by flipping the wrong bit, turning a recoverable error into a corrupted packet length. A block like this passes a random-data regression perfectly, because random data contains no bit errors.

Z21.4 The pattern in the three 8b/10b defects

Three separate defects in the encoder all had the same root, and it was not a typo. It was **a compression rule that was really an unstated hypothesis**.

How a good idea became three bugs

	Step	Justification
1	The 8b/10b tables have two columns — one per running disparity. Writing both invites a typo in one column only.	<i>A typo in one column is invisible to any check that only exercises the other.</i>
2	Observation: most second-column entries are the bitwise complement of the first, and entries with zero disparity appear in both columns unchanged.	<i>True for the majority of entries, which is what made it convincing.</i>
3	So: store one column, derive the other. Half the table, half the typo surface.	<i>This reasoning is correct as far as it goes, and the comment in the first draft said exactly this.</i>
4	But ‘zero disparity implies one codeword’ is a hypothesis, not a fact.	<i>A 4-bit code with two ones and two zeros has disparity zero — and so does its complement. Both are legal on disparity grounds, so disparity alone cannot pick one.</i>
5	What picks between them is run length , not disparity.	<i>D.x.3 is 1100 after one six-bit prefix and 0011 after another, because 001111 followed by 1100 puts six ones in a row and breaks the CDR.</i>
6	The compression therefore collapsed nine genuinely-distinct table entries into one each, and the property check found it: 1082 run-length violations and 2048 false commas.	<i>Then, after fixing the 4-bit tables, 556 violations remained — the same hypothesis was also wrong for exactly one 5b/6b entry, D.07 (111000/000111).</i>

The lesson generalises past 8b/10b: **every scheme for not writing something down twice encodes an assumption about why the two copies agree**. When that assumption is right, the scheme removes a whole class of error — the register-map generator of Part Z4 and the single-C++-source flow of Part Z14 both work exactly this way. When it is wrong, it replaces typos with a *systematic* error, which is much harder to see, because every instance is wrong in the same plausible way.

The difference between the two cases is not cleverness. It is whether you checked.

Z21.5 The provenance ledger

The repository already requires this for paper citations (Gate G022): every claim carries how well it was verified. Protocol constants get the same treatment, because they are the same kind of claim.

Table 470. Confirmation level for every constant in edu/proto

Constant	Level	What that means here
8b/10b 5b/6b and 3b/4b tables	property-proven, not source-verified	All four defining properties hold exhaustively over all 528 codewords and every codeword pair. The normative tables were not available for comparison.
8b/10b well-known codewords (K.28.5, D21.5, D10.2, ...)	corroborated	Seven widely-quoted codewords match exactly. Weak evidence — the same memory produced both — but it is independent of the property proof.
K.28.x selection (K.23.7 etc. omitted)	scoped out, deliberately	Only K.28.x is implemented. Stated in the datasheet rather than left for a customer to discover.

Constant	Level	What that means here
24+6 ECC parity masks	constructed, explicitly not MIPI	A valid SEC-DED built from first principles and proven exhaustively. It is not byte-compatible with CSI-2 and the header of <code>ecc24.h</code> says so in the first paragraph.
Ethernet FCS CRC-32 (Part Z1)	independently verified	Checked against Python's <code>zlib.crc32</code> , an implementation written by somebody else from the standard. This is the strongest level available and it is available only because a free reference implementation exists.

Notice how much the last row differs from the rest. Ethernet's CRC-32 is fully pinned down not because the Ethernet standard is easier to read, but because **somebody shipped a free, widely-used implementation of it**. When you are choosing which protocol block to build first as a one-person house, the existence of an independent reference implementation is worth more than the clarity of the specification — it is the difference between “proven” and “argued”.

Z21.6 What to write in the datasheet

All of the above is worthless if it stays in a commit message. The boundary between proven and assumed belongs on the page the customer reads, phrased without hedging:

확인 수준 (Verification status)

이 블록은 다음을 ****전수로**** 확인했다:

- * 코드워드 528 개가 RD 마다 서로 다르다 (전단사)
- * running disparity 가 -1 / +1 을 벗어나지 않는다
- * 같은 비트가 5 개를 넘게 이어지지 않는다 -- ****코드워드 이음 전부에서****
- * 콤마 무늬가 데이터 흐름에 나타나지 않는다

이 블록은 다음을 ****확인하지 못했다****:

- * IEEE 802.3 Clause 36 Table 36-1/36-2 와 바이트까지 같은가
 - > 상호운용 시험 전에 그 표로 대조할 것. 구조는 그대로 쓸 수 있고
 - 바꿀 곳은 ``enc8b10b.v`` 의 case 문 두 개뿐이다.
- * K.23.7 · K.27.7 · K.29.7 · K.30.7 (일부러 뺐다)
- * 복호기 (이 판은 부호기만이다)

A customer who reads that knows exactly what they are buying and what work remains. A customer who reads “IEEE 802.3 compliant” on a block that was never compared to IEEE 802.3 finds out later, and tells other people.

There is a commercial argument here too, and it is not the obvious one. Publishing the boundary does not make the block look weaker — it makes it look *measured*, and it is the only version of the claim that survives contact with a reviewer. Every vendor says “compliant”. Almost none says which of their properties were checked by enumeration and which by reading.

Z22. 8b/10b — One Block, Two Markets

Of everything a one-person design house could build first, an 8b/10b encoder is unusually well chosen, for a reason that has nothing to do with the technology: **the same block sells into PCIe and into Ethernet**. PCIe Gen1 and Gen2 use it as the physical-layer line code; so does 1000BASE-X, the gigabit Ethernet PCS. So do SATA, DisplayPort, Fibre Channel and USB 3 Gen1. One verification effort, one datasheet, several customers who do not compete with each other.

Z22.1 Where it sits

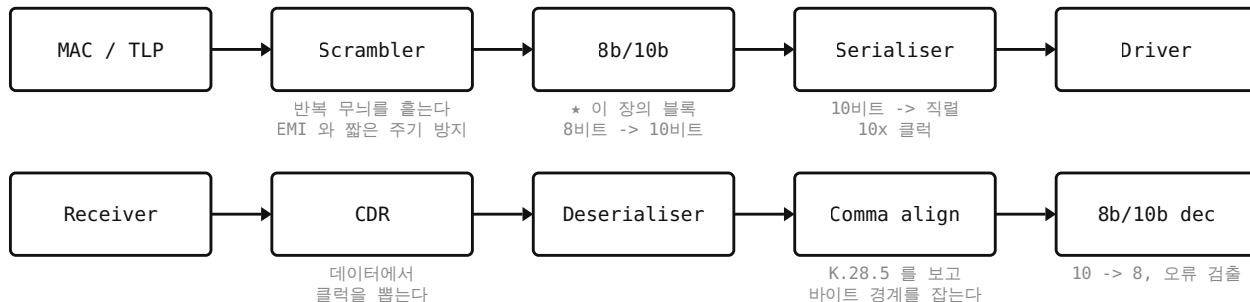


Figure 61. The transmit chain (top) and receive chain (bottom). Our encoder is the last digital stage before serialisation. The two blocks that depend on it most are drawn on the receive side: the CDR needs the transitions 8b/10b guarantees, and comma alignment works only because the comma pattern cannot occur in data.

Read that diagram as a list of obligations rather than a picture. The CDR cannot recover a clock from a long run of identical bits, so the code must bound run length. The AC coupling capacitors in the channel cannot pass DC, so the code must bound the imbalance between ones and zeros. The deserialiser has no idea where a byte starts, so the code must provide a pattern that means ‘here’ and cannot be forged by data. Each of those is one of the four properties Part Z21 proved.

Z22.2 What the code does, in one worked byte

Encoding 0x00 with running disparity -1

	Step	Justification
1	Split the byte into low 5 bits and high 3 bits: 0x00 → x = 0, y = 0. This is written D0.0.	<i>The code is built from a 5b/6b stage and a 3b/4b stage. The naming Dx.y is universal in the literature.</i>
2	Look up x = 0 in the 5b/6b table for RD = -1: 100111.	<i>Four ones and two zeros, so its disparity is +2.</i>
3	Running disparity was -1; adding +2 gives +1.	<i>This is why the table for RD = -1 holds the codeword with positive disparity: it pulls the imbalance back toward zero.</i>
4	Now look up y = 0 in the 3b/4b table, using the updated disparity RD = +1: 0100.	<i>One one and three zeros, disparity -2. The 3b/4b stage sees the disparity after the 6-bit stage, not the disparity the byte started with — a detail that is easy to get wrong and that the property check catches immediately.</i>
5	RD becomes +1 - 2 = -1, and the codeword is 100111 0100.	<i>Back where we started, which is exactly what bounded running disparity means.</i>
6	Check the result: 1001110100 has five ones and five zeros, and its longest run is two.	<i>Verified against the published value for D0.0 as one of the seven corroboration vectors.</i>

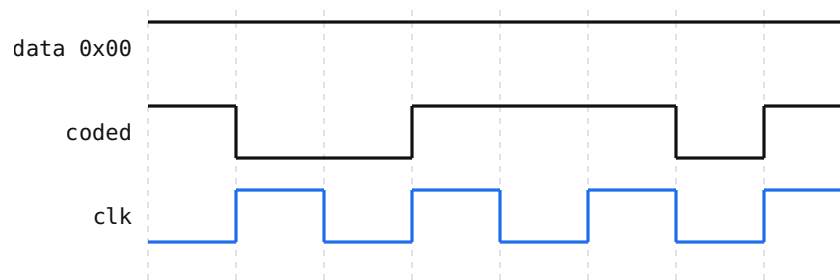


Figure 62. Why the code exists, drawn. An all-zero byte sent raw (top, shown held) offers the receiver's clock-recovery loop no transitions at all. The same byte encoded (middle) carries five transitions in ten bits. The encoder's job is to make every byte look like the middle trace.

Z22.3 The four properties, proven

Part Z21 explained why these are the right things to check. Here is what they came back as, over all 528 codewords and every ordered pair of codewords that can actually follow one another:

Table 471. Exhaustive property check, `edu/proto/proto_check.cpp`

Property	Why the code needs it	Result
P1 — bijective within each disparity	Otherwise two bytes share a codeword and the link cannot decode.	0 collisions
P2 — running disparity stays in {-1, +1}	DC balance on an AC-coupled channel.	0 excursions
P3 — at most 5 identical bits in a row	The CDR needs transitions to hold lock.	longest run 5 , violations 0

Property	Why the code needs it	Result
P4 — comma never appears in data	Byte alignment depends on the comma being unforgeable.	0 occurrences
Not degenerate	A check that only ever sees a few values proves nothing.	456 distinct codewords of 1024

The third row is worth dwelling on, because it is the one that cannot be checked by looking at a codeword. A run of six can straddle a boundary: each codeword is individually fine and the pair is not. That is why the checker enumerates *pairs*, filtered by which disparity each one can follow — and it is why the two table-compression defects of Part Z21 were invisible until it ran.

Worked example 76. Why the longest run is exactly 5 and not 4

Given. P3 reports a longest run of 5, and that is a pass rather than a failure.

Method. Ask what produces the run of 5 and whether it is avoidable.

Numbers. The comma itself. K.28.5 is 0011111010 — five ones in a row, by construction. Any code that also bounded data runs at 5 without exception would have no pattern left that data cannot produce.

This is the general shape of the run-length/alignment trade. The comma must be distinguishable from data, so it must do something data cannot. Making the bound tighter for data is what buys the comma its distinctiveness.

How this is got wrong. The tempting reading of ‘longest run = 5’ is that the design only just passes and a small change might push it to 6. The opposite is true: the 5 is **designed in**, it occurs only in K codes, and the data codewords are strictly better behaved. Reporting the maximum without saying where it comes from would leave a reader with the wrong impression of the margin.

Z22.4 The circuit

The encoder is almost entirely combinational. There is exactly one bit of state in the whole block.

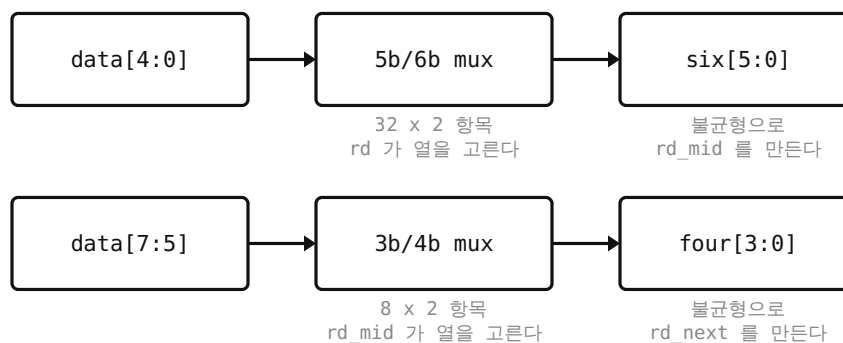


Figure 63. The datapath. Two table lookups (which synthesise to multiplexers), two popcount-and-compare blocks to update the disparity, and one flip-flop holding it. The 3b/4b stage is selected by *rd_mid* — the disparity *after* the 6-bit stage — which is the sequencing the worked example in Z22.2 walked through.

Why the running disparity register is one bit wide

	Step	Justification
1	Running disparity is conventionally described as a signed accumulator.	<i>That is how the literature explains it, and it invites an implementation with a width and a saturation question.</i>
2	But P2 proves it only ever takes the values -1 and $+1$.	<i>Every codeword has disparity -2, 0 or $+2$, and the table selection always moves it back toward zero.</i>
3	Two possible values need exactly one bit: 0 means -1 , 1 means $+1$.	<i>The accumulator, its width, and its saturation logic all disappear.</i>
4	So the entire sequential state of an 8b/10b encoder is one flip-flop.	<i>Measured: the synthesised block has 13 flip-flops — code 10 + rd 1 + code_valid 1 + err 1. Exactly one of them is the protocol state.</i>
5	Knowing an invariant is what bought that.	<i>The proof did not just check the design; it simplified it. This is the usual return on proving a property rather than assuming it.</i>

Z22.5 The one-bit-state rule, in code

```
// 6비트 코드워드의 불균형만큼 RD 가 움직인다. -2 / 0 / +2 뿐이다.
// $countones 대신 손으로 더한다 -- iverilog 가 -g2012 에서도 그것을
// 안 받는 판이 있다. **도구에 안 기대는 쪽을 고른다.**
wire [2:0] six_ones = {2'b0, six[5]} + {2'b0, six[4]} + {2'b0, six[3]}
                  + {2'b0, six[2]} + {2'b0, six[1]} + {2'b0, six[0]};

// 1의 개수가 4 면 +2, 3 이면 0, 2 면 -2. rd 는 1비트다.
wire rd_mid = (six_ones == 3'd4) ? 1'b1 :
              (six_ones == 3'd2) ? 1'b0 : rd;
```

Two details in six lines. The manual popcount avoids \$countones, which some simulator builds reject even in -g2012 mode — when a construct buys you nothing but a dependency, do not take the dependency. And the rd fall-through in the last line is the disparity-zero case: the codeword does not move the imbalance, so the register holds. Writing that as an explicit else rather than letting it fall out of an incomplete assignment is what keeps always_comb from inferring a latch (Part Z18.5).

Z22.6 Verification

Table 472. What was run against the encoder

Check	Result	What it establishes
Exhaustive property proof (C++)	P1–P4 all pass, 528 codewords	The code is internally consistent and does what 8b/10b exists to do.
Published codeword spot-check	7 of 7 match	Weak but independent corroboration that these are <i>the</i> tables.
RTL vs C++ golden, streamed	3000 vectors, 0 wrong, 520 distinct outputs	The hand-written Verilog agrees with the independently written C++, including the disparity carried between codewords.
Self-check (corrupt one golden value)	통과	The comparison can actually fail. Without this the row above means nothing.

Check	Result	What it establishes
Mutation score	16/16 = 100 %, 5 judged equivalent	The tests <i>bite</i> . This is a different question from 'do the tests pass'.

The stream matters more than the count. The golden generator threads running disparity from one vector to the next, so the expected value on line 2000 depends on every line before it. A single disagreement anywhere makes everything after it wrong — the regression cannot quietly pass with a broken state machine, which is the failure mode a per-codeword test would have.

Worked example 77. What the mutation score found that 3000 passing vectors did not

Given. The regression passed 3000 streamed vectors with 0 mismatches. Mutation scoring then reported 71.4 % with six escapes.

Method. Look at each surviving mutant and ask why the tests cannot see it.

Numbers. Five were **provably equivalent** — OR replaced by XOR in a condition whose terms are mutually exclusive, verified over all $32 \times 2 = 64$ inputs with zero differences. The sixth was real: `code_valid <= valid & ~bad_k` mutated to `|` survived. The fix was to observe `code_valid` during a deliberate idle beat after each transaction, which took four lines and moved the score to 100 %. This is the same lesson as the throughput mutant in Part Z1: a regression measures what you told it to look at, and mutation scoring is how you find out what you forgot.

How this is got wrong. The real one is not a subtle bug — the mutant asserts `code_valid` on every idle cycle, which would break any downstream consumer immediately. It survived because the testbench **never looked at an idle cycle**. The stimulus drove a transaction every beat, so the entire idle path was unexercised. No amount of additional random data would have found it.

Z22.7 What it costs

Table 473. Synthesis and place-and-route, iCE40 HX8K

Measure	Value	Note
Cells (standalone)	91	yosys synth_ice40
SB_LUT4	74	The two table multiplexers dominate
SB_CARRY	4	The popcount adders
Flip-flops	13	code 10 + rd 1 + code_valid 1 + err 1
ICESTORM_LC (with timing shell)	106	Includes the shell's input registers
$F_{\max}$	86.3 / 84.9 / 86.3 MHz	Three placer seeds; spread under 2 %
Verilator -Wall	0 warnings	Clean, not suppressed

74 LUTs and 13 flip-flops for a block that sits in the datapath of every PCIe Gen2 lane in the world. But the frequency is the number that decides what this block is actually good for, and it is worth doing that arithmetic rather than assuming.

Worked example 78. Is this fast enough to sell?

Given. The encoder reaches 84.9 MHz on iCE40 HX8K at one byte per clock. 1000BASE-X carries 1 Gb/s of payload; PCIe Gen2 runs 5 GT/s per lane and 8b/10b passes 8 payload bits per 10 transmitted.

Method. Convert each protocol's payload rate into the byte rate the encoder must accept, then compare.

Numbers. 1000BASE-X needs $1\text{ Gb/s} \div 8 = \mathbf{125\text{ Mbyte/s}}$. PCIe Gen2 needs $5\text{ GT/s} \times 8/10 \div 8 = \mathbf{500\text{ Mbyte/s}}$ per lane. At one byte per clock this block delivers 84.9 Mbyte/s — **68 % of gigabit Ethernet and 17 % of one PCIe Gen2 lane**. It meets neither. The fix is architectural, not a matter of squeezing the timing. Encoding two bytes per clock gives 170 Mbyte/s, which clears 1000BASE-X with margin; the two encoders are independent except that the second must take the first's updated disparity, so the disparity path becomes the critical path. PCIe Gen2 needs six bytes per clock at this frequency, which on this fabric is not the right answer — that part belongs on a faster process.

How this is got wrong. The tempting sentence to write was ‘comfortably past gigabit Ethernet’ — it was in the first draft of this page, and it was wrong by a factor of 1.5. 84.9 MHz sounds fast, and a block being small and clean invites the assumption that it is also fast enough. **Neither area nor cleanliness implies throughput; only the arithmetic does.** The same draft also multiplied 84.88 by 8 and got 686 instead of 679.

So the honest positioning is: on this FPGA, at one byte per clock, the block is a demonstration rather than a shippable PHY. Widened to two bytes per clock it is a plausible 1000BASE-X part. PCIe Gen2 is a different process, not a different day of optimisation. A datasheet that says that is more useful than one that quotes 84.9 MHz and leaves the division to the reader.

Z22.8 What is not in the box

Stated here rather than left for a customer to find:

Table 474. Known limitations

Gap	Consequence	Effort to close
Encoder only — no decoder	Half a link. A decoder needs the inverse tables plus invalid-codeword and disparity-error detection.	Comparable to the encoder, plus error reporting. The property harness is reusable as-is.
Only K.28.x control codes	K.23.7, K.27.7, K.29.7 and K.30.7 are unavailable. Enough for alignment and framing; not enough for every protocol's ordered sets.	Small — those four take the 5b/6b data table with the K 3b/4b table.
Not compared to the normative tables	Byte-exact interoperability is unproven. See Part Z21.	One afternoon with the document, and only the two case statements change.
No scrambler	8b/10b bounds run length but does not break up repeating patterns, which concentrate EMI at one frequency.	Separate block; an LFSR and an XOR.

That table is the honest version of “production ready”. The verification is real and unusually thorough for a block this size; the scope is genuinely narrow. Both halves belong in the datasheet, because a customer who discovers the second half after buying will not believe the first.

Z23. The Packet Header ECC — Why Headers Get More Than Payloads

A MIPI CSI-2 short packet is four bytes: one Data Identifier, two Word Count, one ECC. A long packet adds a payload and a 16-bit CRC at the end. So the header gets an **error-correcting** code and the payload gets a merely **error-detecting** one, and the header's code costs a quarter of the header's own size.

That asymmetry looks wasteful until you ask what each failure costs. This part builds the header code, proves it, and puts it on silicon.

Z23.1 Why the header is worth more protection

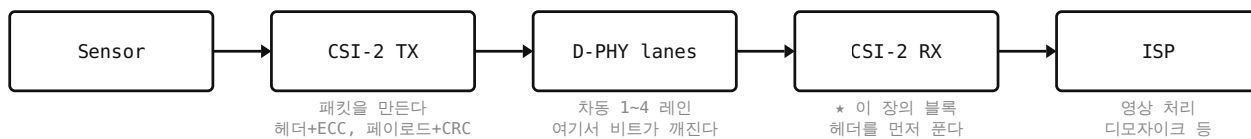


Figure 64. The camera path. The receiver must read Word Count out of the header to know where the payload ends — and therefore where the *next* packet begins.

Why a 1-bit header error is not a 1-bit problem

	Step	Justification
1	The header's Word Count says how many payload bytes follow.	<i>The receiver has no other way to find the packet boundary; CSI-2 packets are not self-delimiting.</i>
2	A single bit flipped in Word Count changes that length — by 1, or by 32768 if it lands in the top bit.	<i>Binary weighting means the damage is not proportional to the number of bits flipped.</i>
3	The receiver then looks for the next packet header at the wrong offset.	<i>It reads payload bytes as a header, whose ECC will not check, whose Word Count is garbage.</i>
4	The error does not stop. Every subsequent packet in the frame is misaligned.	<i>One bit has cost an entire video frame, and possibly re-synchronisation time beyond it.</i>
5	Whereas a single bit flipped in the payload corrupts one pixel.	<i>The CRC flags the packet as bad; the frame is imperfect but its structure survives. An ISP can interpolate one pixel.</i>
6	Therefore: correct the header, merely detect on the payload.	<i>The cost asymmetry is thousands to one, so the protection asymmetry is justified. This is the same reasoning that puts ECC on DRAM address lines before data lines.</i>

Z23.2 What SEC-DED means, precisely

Table 475. Three codes, three guarantees

Code	1-bit error	2-bit error	Cost for 24 data bits
Parity (1 bit)	detected	missed	1 bit
Hamming SEC (5 bits)	corrected	miscorrected — silently turned into a 3-bit error	5 bits

Code	1-bit error	2-bit error	Cost for 24 data bits
Extended Hamming SEC-DED (6 bits)	corrected	detected, not corrected	6 bits

The middle row is the trap, and it is why the sixth bit is not optional. A plain Hamming code always produces a syndrome that names some bit, so given a 2-bit error it confidently flips a third one. You end up worse off than with no ECC at all. The extra overall-parity bit is what lets the decoder say “I do not know” — the same distinction this repository insists on everywhere else, here implemented in six wires.

Worked example 79. How one extra bit buys double-error detection

Given. 5 Hamming parity bits give each of the 30 bit positions a distinct 5-bit syndrome. Add a 6th bit that is the parity of the entire codeword.

Method. Work out what each error class does to the syndrome and to the overall parity, separately.

Numbers. A **single** error flips an odd number of bits, so overall parity becomes 1, and the syndrome is non-zero and names the bit. A **double** error flips an even number, so overall parity returns to 0, while the syndrome (the XOR of two distinct non-zero syndromes) is still non-zero. So *syndrome non-zero with parity 0* is a signature no single error can produce.

That bug is instructive because it is *conservative*: it never corrupts data, it just declines to fix things it could have fixed. In the field it would look like an unexplained doubling of the dropped-frame rate, with no corrupted output to point at — the kind of defect that gets blamed on the cable for a year.

How this is got wrong. The decoder must compute overall parity **directly from the received codeword**, not by re-encoding the received data and comparing the 6th bit. Re-encoding folds the syndrome's own parity into the comparison: the result is 1 XOR parity(syndrome), which is 0 for every data bit whose syndrome has odd weight. Measured — the first version of `ec-c24_dec` did exactly this and reported **90 of 180 single errors as double errors**, i.e. refused to correct half of the correctable ones.

Z23.3 Constructing the code

Part Z21 explained why the remembered MIPI parity masks were discarded: counted before use, they produced 27 distinct syndromes out of the 30 required. So the code here is built from first principles instead, and is **not** byte-compatible with CSI-2. The construction is four lines:

```
# 24 데이터 비트에 신드롬을 배정한다.
# 무게 1 짜리 패턴은 **패리티 비트 자신의 오류**가 쓰므로 데이터에 주면 안 된다.
cand = [s for s in range(1, 32) if bin(s).count('1') >= 2] # 26 개
syn = cand[:24] # 24 개만 쓴다

# 패리티 i 의 마스크 = 신드롬의 i 번째 비트가 1 인 데이터 비트들
masks = [[b for b in range(24) if (syn[b] >> i) & 1] for i in range(5)]
```

Why the syndromes must have weight ≥ 2

	Step	Justification
1	Each parity bit can itself be corrupted in transit.	<i>The ECC field travels over the same lanes as the data.</i>
2	If parity bit i flips, the recomputed syndrome is 2^i — a weight-1 pattern.	<i>Only that one parity check disagrees.</i>
3	So the five weight-1 patterns are already spoken for, and no data bit may be assigned one.	<i>Otherwise a flipped parity bit and a flipped data bit look identical, and the decoder corrupts good data trying to fix a parity bit that nothing depends on.</i>
4	Of the 31 non-zero 5-bit patterns, 26 have weight ≥ 2 , and we need 24.	<i>It fits, with two spare. Had we needed 27 data bits, 5 parity bits would not have been enough.</i>
5	Verified rather than argued: all 29 syndromes distinct, none zero, no double-error syndrome colliding with a single-error syndrome.	29/29, 0 인 것 없음

Z23.4 Proven exhaustively

Table 476. edu/proto/ecc_check.cpp

Case	Enumerated	Result
No error	6 representative data values	all report ECC_OK, data unchanged
Single error	180 (30 bit positions $\times$ 6 data values)	0 wrong — every one corrected
Double error	2610 (all C(30,2) pairs $\times$ 6 data values)	2610 detected, 0 miscorrections
Not degenerate	20,000 random headers	64 of 64 possible ECC values seen

Six representative data values rather than all 2^{24} , because the code is **linear**: the syndrome of a corrupted codeword depends only on the error pattern, not on the data underneath it. That is a property worth stating rather than assuming, so the six values span all-zeros, all-ones and four irregular patterns, and they all agree — which is the evidence that linearity holds in the implementation and not merely in the theory.

Z23.5 The circuit is a pile of XOR trees

```
// `^(vector)` 는 리덕션 XOR -- 모든 비트를 XOR 한다.
// 이 한 글자가 14입력 XOR 나무 하나다. C++ 에는 이런 연산자가 없다.
wire [4:0] p;
assign p[0] = ^(data_in & M0);
assign p[1] = ^(data_in & M1);
assign p[2] = ^(data_in & M2);
assign p[3] = ^(data_in & M3);
assign p[4] = ^(data_in & M4);
wire p5 = (^data_in) ^ (^p);

// 신드롬 -> 비트 자리. 각 데이터 비트의 신드롬은 그 비트가 어느 마스크에
// 들어 있는가로 정해진다. 전부 상수라 합성기가 통째로 푼다.
genvar g;
generate
```

```

    for (g = 0; g < 24; g = g + 1) begin : gen_fix
        wire [4:0] s_g = { M4[g], M3[g], M2[g], M1[g], M0[g] };
        assign fix[g] = (syn == s_g) & opar & (syn != 5'd0);
    end
endgenerate
assign data_fixed = data_in ^ fix;

```

Three things in that fragment are worth naming. The reduction XOR (^vector) has no C++ equivalent and collapses a 14-input parity tree into one character — Part Z18.6 lists the others. The generate loop builds 24 comparators whose right-hand sides are all compile-time constants, so synthesis evaluates `s_g` away entirely and what remains is 24 five-bit constant comparisons. And the block is named (`gen_fix`) because an unnamed generate block gets called `genblk1`, and every constraint and waveform that references it breaks the day somebody inserts another loop above it.

Z23.6 What it costs

Table 477. Synthesis, iCE40 HX8K

Measure	Value	Note
Cells	68	yosys synth_ice40
SB_LUT4	68	All of them — pure combinational logic
Flip-flops	0	순수 XOR 나무. 조합 논리뿐이라 플립플롭이 없다.
Verilator -Wall	0 warnings	Clean
RTL vs C++ golden	1500 vectors, 0 wrong	Stimulus mixes no-error, single-error and double-error 1:1:1
Self-check	통과	Corrupting one golden value makes the comparison fail

68 LUTs, zero flip-flops, and it corrects any single-bit error in a camera packet header. For comparison the 8b/10b encoder of Part Z22 is 74 LUTs. Both are small enough that the interesting engineering is entirely in the verification and the specification, not in the gates — which is the usual situation for protocol IP and the reason Parts Y and Z4 spend so much more time on deliverables than on RTL.

The stimulus mix in that table is the part most easily got wrong. A regression built from random headers contains **no bit errors at all**, so it exercises only the `ECC_OK` path — the correction logic, which is the entire point of the block, would never run. Errors have to be *injected*, and injected in the proportions you care about.

Z23.7 Swapping in the real masks

Because the construction is separated from the structure, making this block CSI-2 interoperable is a table change, not a re-design:

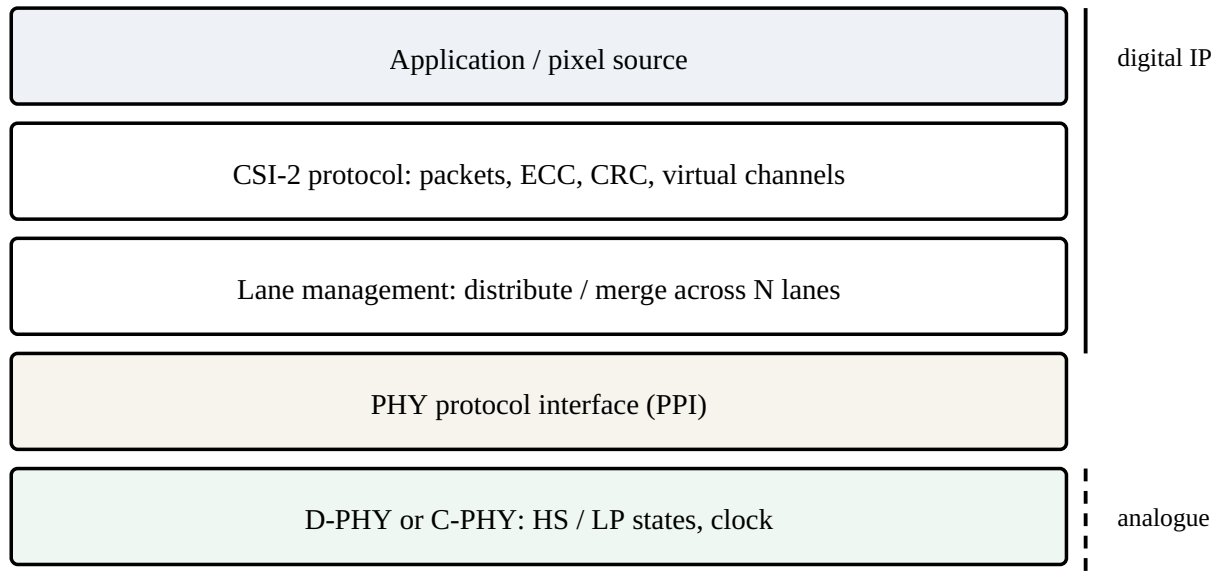
Table 478. What changes, and what does not

Item	Changes?
<code>ECC_MASK[5]</code> in <code>ecc24.h</code>	Yes — five constants
<code>M0–M4</code> in <code>ecc24.v</code>	Yes — the same five values
Encoder, decoder, syndrome decode, correction logic	No
<code>ecc_check.cpp</code> and all four proofs	No — re-run them
Testbench and golden generator	No
Area, timing, interface	No

And the proofs are exactly what you re-run after the swap, because a transcription error in five hex constants is precisely the failure this part opened with. The value of having built the harness first is that checking somebody else's constants costs one command.

Y1. MIPI CSI-2 and DSI, Worked as a Deliverable

Every phone camera, most automotive cameras and a large fraction of embedded vision sensors speak CSI-2; every phone display and most embedded panels speak DSI. The two share a PHY and a packet structure, which is why a design house that builds one has most of the other. This part works the numbers a customer will ask for, and states where the answers come from.



The commercial boundary is the PPI: above it is soft RTL a design house can sell,
below it is a hard macro tied to one foundry process.

Figure 65. The CSI-2 / DSI stack, with the boundary that determines what a soft-IP vendor can actually sell.

Y1.1 Bandwidth: from a sensor specification to a lane count

This is the first calculation a customer will do with your datasheet, so it must be the first one in it. It is also the calculation that decides whether your block is in the design at all.

Table 479. Required D-PHY bandwidth and lane count at 2.5 Gbps per lane

Mode	Mpixel/s	bits/pixel	Payload (Gbps)	With 15 % overhead (Gbps)	Lanes at 4 Gbps/lane	Lanes at 2.5 Gbps/lane
1080p30	62.21	12	0.7465	0.8585	1	1
1080p60	124.4	12	1.493	1.717	1	1
4K30	248.8	12	2.986	3.434	1	2
4K60	497.7	12	5.972	6.868	1	3
8MP automotive 30	248.8	16	3.981	4.579	1	2
12MP still burst	360	14	5.04	5.796	1	3

The 15 % allowance covers line blanking, packet headers and footers, and the low-power intervals between bursts. **It is an assumption and it belongs in the datasheet beside the number it produced;** a customer with a different blanking ratio will get a different lane count and should be able to see why.

Worked example 80. Why the overhead is not a fixed percentage

Given. A 1920×1080 frame, 12 bits per pixel, RAW12 packing. Each line is one long packet: 4 bytes of header, the payload, and 2 bytes of footer. Between lines the link may drop to low-power state and return, costing $T_{LPX} + T_{HS-PREPARE} + T_{HS-ZERO} + \text{sync}$, taken here as 300 ns.

Method. Compute the per-line overhead in bytes and in time, then express it as a fraction. The fraction depends on the line length, which is why a single percentage cannot be right for every mode.

Payload bytes per line	2,880
Packet header + footer	6 bytes
Packet overhead	0.208 %
Line payload time at 4 lanes × 2.5 Gbps	2.304 μs
LP transition cost per line	0.3 μs
Transition overhead	13 %
Total overhead for this mode	13.2 %
Same for a 320-pixel-wide line	79.4 %

How this is got wrong. By quoting one efficiency figure for the whole product. The overhead is dominated by a per-line *constant*, so efficiency collapses for short lines: a narrow region-of-interest readout — exactly what a machine-vision customer does — can spend more time in transitions than in data. The design response is to **stay in high-speed state across lines** when the blanking interval is short, which the specification permits and which many controllers do not implement. That feature is worth naming in a datasheet because a competitor's block that lacks it will lose the ROI benchmark.

Y1.2 The D-PHY state machine and its timing parameters

Table 480. D-PHY lane states and what each costs

State	Electrical	Purpose	Entry cost
LP-11 (Stop)	Both lines high, 1.2 V single-ended	Idle	—
LP-01 / LP-00	Low-power signalling	Request and bridge states	T_{LPX} per transition, min 50 ns
HS-0 / HS-1	200 mV differential about a 200 mV common mode	Data burst	$T_{HS-PREPARE} + T_{HS-ZERO} + \text{sync}$, 150–300 ns
ULPS	LP-00 held	Ultra-low power; clock may stop	$T_{WAKEUP} \geq 1 \text{ ms}$ — a millisecond, not a microsecond
Escape mode	LP pulse-coded	Trigger, LPDT side-channel	Slow by design

The 1 ms ULPS wake-up is a system-level number that routinely surprises integrators. A design that powers the link down between frames to save energy cannot wake it in a line blanking interval; it can only wake it in a frame interval, and only if the frame rate leaves a millisecond spare. 30 fps gives 33 ms per frame, so it fits; a 240 fps burst mode does not.

Power state transitions have latencies that are three to five orders of magnitude apart — 50 ns for LPX, 1 ms for ULPS — and a power management policy that treats them as interchangeable will either waste energy or drop frames.

Table 481. D-PHY timing parameters evaluated at three lane rates (ns)

Parameter	Specified as	at 1.0 Gbps	at 1.5 Gbps	at 2.5 Gbps
T_LPX	≥ 50	50	50	50
T_HS-PREPARE	$40 + 4 \times UI \dots 85 + 6 \times UI$	44	42.67	41.6
T_HS-ZERO	$\geq 145 + 10 \times UI$	155	151.7	149
T_HS-TRAIL	$\geq \max(8 \times UI, 60 + 4 \times UI)$	64	62.67	61.6
T_HS-EXIT	≥ 100	100	100	100
T_CLK-PRE	$\geq 8 \times UI$	8	5.333	3.2
T_CLK-POST	$\geq 60 + 52 \times UI$	112	94.67	80.8

These values are computed from the formulae as written here and have not been checked against a current copy of the MIPI D-PHY specification, which is a paid document this repository has not obtained. Treat the shape as correct and the constants as requiring confirmation before they enter an implementation.

On quoting a specification you have not read. The table above is deliberately marked. A design house that builds a MIPI controller buys the specification; there is no substitute and no legitimate way around it. Writing formulae from secondary sources into RTL is how a block comes to fail interoperability at a plugfest, and the failure is expensive precisely because it is found late and in public. **The correct entry in a project plan is a line item for the specification and for membership, not an assumption that the numbers can be inferred.**

Y1.3 Packet structure, ECC and CRC — and why both exist

Why the header has ECC and the payload has CRC

	Step	Justification
1	The short packet header is 4 bytes: data identifier, two word-count bytes, and one ECC byte.	<i>CSI-2 packet structure.</i>
2	The ECC is a Hamming code over the preceding 24 bits, giving single-error correction and double-error detection.	<i>From Part X10's bound: 24 data bits need $r = 5$ for correction, plus one overall parity, which fits in the byte with two bits spare.</i>
3	A corrupted header is unrecoverable at any higher layer , because the word count tells the receiver where the packet ends.	<i>A wrong length desynchronises the stream for the remainder of the frame. Correction, not detection, is therefore required here.</i>
4	The payload carries a 16-bit CRC and no correction .	<i>A corrupted pixel is one bad pixel; the frame continues. Detection lets the application decide, and correction would cost far more for far less.</i>
5	The asymmetry is a design principle, not a quirk of this standard.	<i>Spend correction where a failure is structural and detection where it is local. The same reasoning produced the parity-plus-reload answer for configuration memory in Part X10.</i>

Problem. A customer reports occasional frame corruption that disappears when they reduce the lane rate. Your ECC error counter is zero and your CRC counter is non-zero. What does that tell you, and what would the opposite pattern tell you?

Solution. CRC errors without ECC errors mean the *payload* is being corrupted while the headers survive. That is a strong hint, because headers and payload travel the same wires but not the same way: headers occur at the start of a burst, just after the HS settle time, and are short; payload is long and continuous. Corruption that spares the header and hits the payload points at something that accumulates over a burst — **clock drift or an under-specified $T_{\text{HS-SETTLE}}$ window that is fine for the first bytes and drifts later**, or an SSC/jitter interaction, or simply insufficient eye at the sampling point for long runs. The opposite pattern, ECC errors without CRC errors, would point at the *transition*: the receiver is mis-sampling the first bytes after leaving low-power state, which is the classic symptom of an incorrect settle-time programming or of a PHY that reports ready too early. **The instrumentation that makes this diagnosis possible — separate, readable counters for header and payload errors, with the ability to capture the first failing packet — is a feature to design in, not a debug hack; it is also a strong differentiator in a datasheet.**

Y1.4 What the repository's real CSI-2 source shows

The code appendix of this book reproduces **32,676 lines** across 55 files of a working MIPI CSI-2 receiver written for FPGA fabric. It is worth reading for one reason above all: it shows how much of the block is *not* the protocol. Word alignment, lane de-skew, the byte-to-pixel unpacker and the clock-domain crossing to the pixel clock take more code than the packet decoder.

Table 482. The twelve largest files of the CSI-2 receiver in the corpus

File	Lines
line_ram_dp.v	5,781
out_line_ram_dp.v	5,781
mipi_csi_phy.v	4,697
out_line_ram_ldp.v	4,635
uvc.c	2,094
cyfxtx.c	1,188
cyfxuvcdscr.c	1,024
sensor_imx219.c	587
debayer_filter.v	458
line_ram_dp.v	373
debayer_filter.v	350
tb_mipi_bridge.v	334

Counted from the indexed corpus at build time.

Graduate · What a design house adds on top of an open-source receiver like this one. The open block demonstrates the datapath. What it does not contain is what a customer pays for: a register map with a defined reset state and an IP-XACT description; error counters and an interrupt controller; support for more than one virtual channel and for the data types the customer's sensor actually emits; a verification environment with the protocol checkers and a compliance test list; synthesis and timing closure at a specified frequency in a named process with reports; lint and CDC sign-off; and documentation that lets an integrator finish without calling. **Each of those is weeks of work, and together they are the product.** A design house that believes the RTL is the product will price the work at a fraction of its cost and deliver something a customer cannot integrate.

Table 483. An honest effort estimate for a sellable CSI-2 receiver, one engineer

Activity	Weeks	Share of a person-year (%)
Specification study and register map	3	5.77
RTL: protocol layer	4	7.69
RTL: lane management and de-skew	3	5.77
RTL: PPI adaptation and CDC	3	5.77
C reference model for the packet layer	2	3.85
Verification environment and checkers	6	11.5
Directed and random test suites	5	9.62
Lint, CDC and synthesis sign-off	3	5.77
Timing closure at target frequency	2	3.85
IP-XACT packaging and integration guide	2	3.85
Documentation and datasheet	3	5.77
Customer support allowance, first year	6	11.5

*Total 42 weeks — about 0.8 person-years. **Verification and everything downstream of RTL is 64 % of it.** An estimate that counts only the RTL rows produces 10 weeks and is the reason first-time IP vendors run out of money.*

Y2. PCI Express, Generations 3 to 6, Worked

PCIe is the interface a design house is most likely to be asked for and least likely to build from scratch, because the controller is enormous and the compliance burden is severe. It is nevertheless worth understanding in detail, because almost every block a design house sells will sit behind one, and the properties of that link — its latency, its credit behaviour, its error model — propagate into the specification of whatever is attached.

Y2.1 The rate table, computed rather than remembered

Table 484. PCIe generations: raw rate, encoding and usable bandwidth

Generation	GT/s per lane	Encoding	Encoding efficiency	GB/s per lane (one direction)	GB/s at x16	Modulation
Gen1	2.5	8b/10b	0.8	0.25	4	NRZ
Gen2	5	8b/10b	0.8	0.5	8	NRZ
Gen3	8	128b/130b	0.9846	0.9846	15.75	NRZ
Gen4	16	128b/130b	0.9846	1.969	31.51	NRZ
Gen5	32	128b/130b	0.9846	3.938	63.02	NRZ
Gen6	64	242B FLIT / PAM4	0.9453	7.562	121	PAM4

Computed from the transfer rate and the encoding efficiency alone; protocol overheads (TLP headers, DLLPs, ordered sets) are additional and Y2.3 computes them. The 8b/10b to 128b/130b change at Gen3 is why Gen3 is more than 1.6× Gen2 despite the rate being only 1.6×.

Worked example 81. Why Gen6 went to PAM4 and what it cost

Given. Gen5 runs 32 GT/s NRZ; Gen6 runs 64 GT/s. The channel budget is roughly unchanged, because the connectors and boards are the same.

Method. Compare the Nyquist frequency each modulation requires and the eye each produces, then note what follows.

Gen5 NRZ Nyquist	16 GHz
Gen6 at 64 GT/s with NRZ would need	32 GHz
Gen6 PAM4 symbol rate	32 GBd
Gen6 PAM4 Nyquist	16 GHz
PAM4 levels	4
Eye height relative to NRZ at equal amplitude	0.333
SNR penalty	9.542 dB
Consequence	a raw BER near 10^{-6}, so FEC becomes mandatory

How this is got wrong. By reading PAM4 as free bandwidth. It keeps the Nyquist frequency — and therefore the channel loss — unchanged, which is the whole point, and it pays 9.5 dB of eye for it. That payment forces three things into the specification at once: **forward error correction**, which adds latency; a **flit-based protocol**, because FEC needs fixed-size blocks and the old variable-length TLP framing does not provide them; and **retry at the flit level** rather than the packet level. Gen6 is therefore not an incremental speed bump but a protocol change, and any IP behind it sees different latency statistics.

Y2.2 Link training: the LTSSM as a bring-up tool

Table 485. LTSSM states a bring-up engineer actually watches

State	What it establishes	Stuck here means
Detect	A receiver is present on the far end	No far end, or AC coupling caps missing / wrong value
Polling	Bit lock and symbol lock at 2.5 GT/s	Reference clock, PHY configuration, or gross signal integrity
Configuration	Lane numbering, link width, lane reversal, polarity inversion	Lane mapping error in the board or the controller
Recovery	Speed change and equalisation	The commonest stall: the link comes up at Gen1 and will not train higher
L0	Normal operation	—
L0s / L1 / L2	Power states	Exit latency violations, or a partner that does not support the state
Hot Reset / Disabled	Explicit teardown	Software

Graduate · ‘Trains at Gen1, fails at Gen3’ is a diagnosis, not a symptom, and it is worth knowing why. Every PCIe link starts at 2.5 GT/s with no equalisation, because both ends must be able to talk before they can negotiate. The move to Gen3 and beyond runs a four-phase equalisation procedure in Recovery in which each end requests transmitter coefficient settings from the other and evaluates the result. A link that trains at Gen1 and fails above it has working wires and a failing equalisation: the channel is worse than the preset assumes, the far end's coefficient requests are being ignored or mishandled, or the receiver's evaluation is reporting success when the eye is marginal. **The debug sequence is therefore: read the link status registers for the actual speed and width; read the lane error status; capture the equalisation phase the link fails in; and only then look at the board.** Controllers that expose per-phase equalisation status are much faster to bring up, and this is a feature worth asking a vendor for.

Table 486. Unit intervals and the budgets they imply

Generation	GT/s	UI (ps)	30 % of UI (ps)	10 % of UI (ps)	Typical channel loss budget (dB)
Gen3	8	125	37.5	12.5	20
Gen4	16	62.5	18.8	6.25	20
Gen5	32	31.25	9.38	3.13	28
Gen6	64	15.63	4.69	1.56	32

The loss budgets are the figures commonly cited for the base specification's channel; they have not been checked against the current specification text in this repository, which is a paid document. The UI column is exact arithmetic.

Y2.3 Protocol overhead: from GB/s on the box to GB/s in the driver

Table 487. TLP efficiency against maximum payload size, Gen5 x16

Max payload (B)	Overhead bytes	Total bytes	TLP efficiency	× encoding efficiency	Usable GB/s
64	24	88	0.7273	0.7161	45.83
128	24	152	0.8421	0.8291	53.07
256	24	280	0.9143	0.9002	57.61
512	24	536	0.9552	0.9405	60.19
4,096	24	4,120	0.9942	0.9789	62.65

The overhead figure counts a 12-byte header, sequence number, LCRC and framing; it ignores DLLP acknowledgements and flow-control updates, which consume a further few per cent. **The lesson is the first column: a device restricted to 128-byte payloads gets roughly three quarters of the bandwidth of one that negotiates 512, and maximum payload size is negotiated to the minimum of the whole path, so one old switch in the chain sets it for everybody.**

Worked example 82. Why a DMA engine behind PCIe needs deep outstanding-read support

Given. A Gen4 x8 link, 4 μ s round-trip read latency to host memory, requesting 512-byte reads. The engine must sustain 12 GB/s.

Method. Bandwidth–delay product again, exactly as in Part X11 — the formula does not care that the wire is now a PCIe link.

Gen4 x8 usable bandwidth	15.75 GB/s
Required bandwidth	12 GB/s
Bytes in flight needed	46.88 kB
512-byte reads outstanding	94
Tag space needed (10-bit tags allow 768)	94 of 768
Completion buffer required	46.88 kB

How this is got wrong. By sizing from the link rate and ignoring the latency. A device with 32 outstanding reads on this link achieves $512 \times 32 / 4 \mu\text{s} = 4 \text{ GB/s}$ regardless of the link being capable of 15. The second trap is the completion buffer: those bytes in flight must land somewhere, so the buffer is not optional and it is tens of kilobytes. **Outstanding-transaction capability and completion-buffer size are the two numbers that decide whether a PCIe device performs**, and they belong on the first page of its datasheet, not in an appendix.

Y2.4 What the corpus contains, and what it does not

This book's code appendix includes **198,332 lines** of a working open-source PCIe datapath: DMA engines, TLP generation and parsing, and AXI bridging. It is genuinely useful and it is worth being precise about what it is. **It is the transaction and data-link layer above a vendor hard macro** — the PHY, the LTSSM, the equalisation procedure and the serialiser are not in it and cannot be, because they are analogue and foundry-specific.

Table 488. Where PCIe functionality actually comes from, and who can sell it

Layer	Implemented as	Who supplies it	Can a small design house sell this?
Serialiser, CDR, equaliser	Hard macro in a specific process	PHY vendor or foundry	No — requires analogue design and process access
PIPE interface	Defined digital interface	Boundary between the two	—
LTSSM, lane management	RTL	Controller vendor	In principle; compliance is the barrier
Data link layer: ack/nak, flow control	RTL	Controller vendor	Yes, with effort
Transaction layer, config space	RTL	Controller vendor	Yes
Application logic behind the controller	RTL	Anyone	Yes — and this is where a design house should be

Compliance is the commercial fact about PCIe. A controller that works is not a controller that can be sold: the PCI-SIG compliance programme, its test fixtures, its interoperability workshops and the membership that permits them are a substantial and recurring cost, and a customer buying a PCIe controller is buying that compliance as much as the RTL. **For a one-**

person design house the rational position is above the controller, not inside it — build the accelerator, the DMA engine, the protocol offload that sits behind a licensed controller, and let the customer bring the controller they have already qualified.

Problem. A customer asks you to quote for ‘a PCIe Gen4 endpoint with a custom accelerator’. How do you structure the quote?

Solution. Split it explicitly, and say why in the quote itself. **Item one: the accelerator** — your design, your verification, your price. **Item two: integration with a controller the customer licenses**, quoted as integration effort with the controller named, because the effort depends heavily on which one; a controller with a clean AXI-Stream application interface is a fortnight and one with an idiosyncratic interface is two months. **Item three: what you are not supplying** — the controller licence, the PHY, the compliance testing, and the driver, each listed with who you expect to provide it. A quote that silently includes the controller either loses on price to competitors who excluded it, or wins and then discovers the licence costs more than the project. **Being explicit about scope is not a weakness in a proposal; it is the clearest signal that the quoter has done this before**, and experienced buyers read it that way.

Problem. Gen6 introduced FEC. What does that do to the latency your block sees, and why can the FEC not simply be made stronger to allow a cheaper channel?

Solution. FEC operates on fixed-size flits, so the receiver must collect a whole flit before it can correct it, and the correction itself takes time. The added latency is small in absolute terms — tens of nanoseconds — but it is **added to every transaction**, and for a latency-sensitive accelerator performing fine-grained host synchronisation it can matter more than the doubled bandwidth helps. It also changes the *distribution*: retries now happen at flit granularity with a different tail than Gen5's packet-level replay. Stronger FEC is not the answer to a worse channel because latency is the constraint that a stronger code violates first: a longer code corrects more and waits longer, and PCIe's entire value proposition rests on being a low-latency load/store fabric rather than a network. **The design point was chosen to bound added latency, and the code strength followed from that bound** — not the other way round, which is the usual direction in communications and the reason PCIe's FEC looks weak to someone arriving from Ethernet.

Y3. The Ethernet MAC and Its Transceiver, Worked

The Gigabit MAC is the most-integrated communications block in existence and a good vehicle for the whole of this book: it has a datapath with hard real-time constraints, a protocol with a long tail of options, a clock-domain problem at its centre, and an interface stack whose names (GMII, RGMII, SGMII, QSGMII) encode a series of commercial compromises worth understanding.

Y3.1 Frame timing: the arithmetic that sets every buffer

Table 489. Frame times, packet rates and the inter-packet gap

Rate	Mbit/s	Min frame + preamble + IPG (ns)	Max frame + overhead (μ s)	Max frames/s (M)	IPG (ns)
10BASE-T	10	67200	1230	0.01488	9600
100BASE-TX	100	6720	123	0.1488	960
1000BASE-T	1000	672	12.3	1.488	96
2.5GBASE-T	2500	268.8	4.922	3.72	38.4
10GBASE-R	10000	67.2	1.23	14.88	9.6
25GBASE-R	25000	26.88	0.4922	37.2	3.84

Computed from 8 bytes of preamble and start delimiter, 12 bytes of inter-packet gap, and the frame itself. **The fifth column is the number that sizes the control path:** at 25 Gbit/s a new frame can start every 33 ns, so any per-frame decision — a lookup, a filter, a statistics update — must complete in that time or be pipelined.

Worked example 83. Why a MAC at 10 Gbit/s cannot process one byte per cycle

Given. 10 Gbit/s line rate. The digital logic runs at 156.25 MHz, which is the standard XGMII clock.

Method. Divide the bit rate by the clock to find the datapath width, then check what that width does to frame alignment.

Bits per clock	64
Datapath width	64 bits = 8 bytes
Bytes per clock	8
Minimum frame (64 B) in clocks	8
Can a frame start mid-word?	yes — the IPG is 12 bytes, not a multiple of 8
Alignment cases the datapath must handle	8
At 25 Gbit/s on the same clock: width	160 bits

How this is got wrong. By designing the datapath for aligned frames. Because the inter-packet gap is 12 bytes and the datapath is 8 wide, successive frames start at different byte lanes, so every parser, every CRC engine and every filter must handle all eight alignments. This is the single largest source of complexity in a wide Ethernet MAC and it is invisible in a specification that describes the protocol in bytes. **A block that assumes alignment will pass a test bench that sends one frame at a time and fail on back-to-back traffic at line rate** — which is the only traffic that matters.

Y3.2 The interface zoo, and what each trade is

Table 490. MAC-to-PHY interfaces: the same function, very different pin counts

Interface	Data bits each way	Clocking	Carries	Approx. pins
MII	4	25 MHz	100 Mbit/s	16
GMII	8	125 MHz	1 Gbit/s	24
RGMII	4	125 MHz DDR	1 Gbit/s	12
SGMII	1	625 MHz DDR serial	1 Gbit/s	4
QSGMII	1	5 Gbit/s serial	4 × 1 Gbit/s	4
XGMII	32	156.25 MHz DDR	10 Gbit/s	72
XFI / SFI	1	10.3125 Gbit/s	10 Gbit/s	4
USXGMII	1	10.3125 Gbit/s	up to 10 Gbit/s, multi-port	4

The progression is one long trade of pins against signalling difficulty. **RGMII halves GMII's pins by clocking on both edges and pays with a notorious source-synchronous timing problem:** the specification's delay requirement is met either by the board, by the PHY or by the MAC, and mismatched assumptions between the three are the most common Gigabit Ethernet bring-up failure in existence.

The RGMII delay question, stated once so it can be checked. RGMII requires the clock to be delayed relative to the data by roughly 1.5–2 ns at the receiver. That delay may be provided by a long clock trace on the board, by the PHY's internal delay (enabled by strap or register), or by the MAC's output delay. **Exactly one of the three must provide it.** Zero gives a link that does not pass traffic or passes it only at 10 Mbit/s; two gives the same symptom for the opposite reason. Because each is configured by a different party — layout, PHY firmware, RTL — nobody owns the question, and it is worth writing the answer into the integration guide of any MAC a design house ships, along with the register bits that set it.

Y3.3 64b/66b and the PCS: what the transcoding buys

Why 64b/66b replaced 8b/10b

	Step	Justification
1	8b/10b maps each byte to a 10-bit symbol chosen for DC balance and transition density.	<i>Original Gigabit encoding.</i>
2	Overhead is 25 %: a 1.25 GBd line carries 1 Gbit/s.	<i>10/8. At 10 Gbit/s that would demand 12.5 GBd, which the channel of the day could not support.</i>
3	64b/66b prefixes two bits of sync header to 64 bits of scrambled payload.	<i>Overhead 3.1 %.</i>
4	Transition density comes from a self-synchronous scrambler ($x^{58}+x^{39}+1$), not from the code.	<i>The sync header is the only guaranteed transition, which is enough for a CDR with a long time constant and would not have been enough for the receivers of 1998.</i>
5	The scrambler is self-synchronising, so a single bit error produces three errors at the descrambler.	<i>The error is fed back at the two tap positions. This multiplication is the reason the FEC above it is symbol-oriented, and it is a fact a link budget must account for.</i>

Table 491. Line-rate consequences of the encoding choice

Encoding	Expansion	Payload Gbit/s	Line GBd (NRZ)	Overhead (%)
8b/10b	1.25	1	1.25	25
64b/66b	1.0312	10	10.312	3.12

Encoding	Expansion	Payload Gbit/s	Line GBd (NRZ)	Overhead (%)
64b/66b	1.0312	25	25.781	3.12
256b/257b + RS-FEC	1.0625	50	53.125	6.25
256b/257b + RS-FEC	1.0625	100	106.25	6.25

The last two rows include the RS(544,514) FEC expansion, which is why 100 Gbit/s per lane runs at about 53.125 GBd — the number Part X4's link budget used.

Graduate · **Where the design-house opportunity is in this stack.** The PMA — serialiser, CDR, equaliser — is a hard macro and is not available to a soft-IP vendor. The MAC is a commodity: dozens of implementations exist, several of them open, and margins are thin. **The PCS and FEC sublayer between them is soft RTL, is algorithmically demanding, changes with every new standard, and has far fewer credible suppliers.** That combination — soft, hard, and changing — is the profile of a good product for a small house, and it is the reason this book's prior-art survey chose it. The specific opening at the time of writing is IEEE 802.3dj's concatenated FEC, where the outer RS-KP4 is well understood and the inner code and its interleaving are new.

The code appendix contains **260,596 lines** of an open-source Ethernet family, of which **29,336 lines** in 89 files are the BASE-R PCS itself — the 64b/66b encoder and decoder, the scrambler, the block synchroniser and the gearbox. Reading those files against the derivation above is the fastest way to see how a standard's prose becomes RTL.

Y3.4 Flow control, buffering and the numbers a customer checks

Table 492. Buffer needed to stop a sender in time, 100 m of copper

Rate (Gbit/s)	One-way propagation (µs)	Round trip (µs)	Bandwidth–delay product (kB)	Max-size frames in flight
1	0.5	1	0.1221	1
10	0.5	1	1.221	1
25	0.5	1	3.052	3
100	0.5	1	12.21	9

This is the buffer a receiver must have after it decides to send a PAUSE frame, because frames already in flight will arrive regardless. Add the PAUSE frame's own transmission and the partner's reaction time and the requirement grows further. **The bandwidth–delay product appears for the third time in this book**, in its third guise; it is the same formula that sized the credit buffer and the PCIe completion buffer.

Problem. A customer reports frame loss under burst traffic although your MAC reports no errors and the link is not saturated on average. Where do you look?

Solution. ‘Not saturated on average’ is the clue: averages hide bursts, and Ethernet traffic is bursty at every timescale. The sequence is: **read the drop counters and find out which stage dropped** — a MAC that cannot say whether a frame was dropped at the PHY interface, in the receive FIFO, or by the filter is much harder to defend, so this instrumentation is worth building. Then **check the receive FIFO's depth against the consumer's worst-case stall**, not its average service rate; if the consumer is a bus master, the stall is the interconnect's worst-case arbitration latency from Part X11, which is usually far longer than anyone estimated. Then **check whether flow control is enabled at both ends**, since PAUSE is negotiated and frequently disabled by default. Finally, **check the clock-rate mismatch**: two ends nominally at the same rate differ by up to 200 ppm, so a receiver without rate adaptation slowly falls behind and drops a frame every few seconds — a distinctive signature worth recognising, because the loss rate is remarkably constant.

Problem. Why does a MAC need an idle-insertion and idle-deletion mechanism at all, given that both ends are specified at the same rate?

Solution. Because ‘the same rate’ means the same nominal rate within a tolerance, typically ± 100 ppm each, so the two can differ by 200 ppm — one part in five thousand. Over a continuous stream that is one extra or missing byte every 5000, which accumulates without bound. The mechanism uses the inter-packet gap as elastic: if the local clock is slower, delete an idle byte from the gap; if faster, insert one. The gap can be shortened to a floor (five bytes in the 10 Gbit/s specifications, from twelve) which is what limits how much rate difference can be absorbed. **The consequence for a design is that the IPG a MAC sees on receive is not the IPG the far end transmitted**, so any logic that assumes twelve bytes is wrong, and any logic that uses IPG timing to infer anything about the far end is wrong twice. This elasticity, and its floor, is also why the FEC and PCS layers above must tolerate the alignment markers that periodically consume the same budget.

Y4. A Verification Agent That Runs While You Sleep

A one-person design house has one engineer and twenty-four hours in a day, and the engineer needs some of them. The only way the arithmetic works is for verification to run unattended overnight and to leave behind something better than a log file. This part documents such a system, which lives in this repository under `edu/agent/` and which is exercised by `tests/test_verif_agent.py`. It is deliberately small — roughly the size one person can maintain — and it is deliberately suspicious of itself.

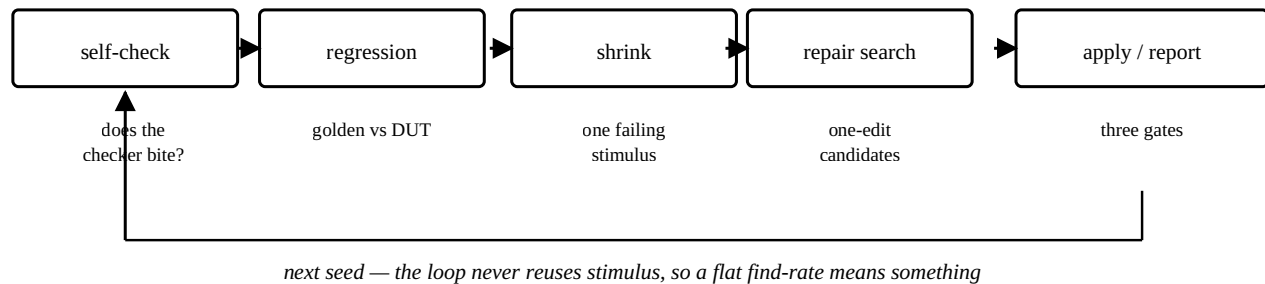


Figure 66. One iteration of the loop. The first box is the one most regression systems lack, and it is the reason the rest can be believed.

Y4.1 The design, and the two failures it is built around

An unattended regression can fail in two directions, and the second is far more dangerous than the first. It can report a failure that is not there, which wastes a morning. Or it can report success while checking nothing, which wastes a tape-out. The loop is structured so that the second is caught by construction.

Table 493. What each stage is for, and what would go wrong without it

Stage	What it does	Without it
Self-check	Corrupts one golden value and asserts that the comparison notices	A comparison that silently stopped comparing reports green forever. This is the failure mode this whole repository is organised against
Regression	Runs the DUT and the golden model on the same stimulus and compares every output	—
Distinct-output count	Counts how many <i>different</i> values the golden model produced	A stimulus that drives one constant passes trivially. The loop treats fewer than two distinct outputs as a failure, not a pass
Shrink	Re-runs the single failing stimulus on its own	A 4000-transaction failure report that nobody can debug
Repair search	Generates single-edit candidates from a rule library and tests each	The engineer starts every morning from a raw failure
Three gates	Original seed, an unused seed, and the self-check	An edit that special-cases the failing input passes. The overfitting gate is the reason this can be left running unattended

Graduate · The overfitting gate deserves its own sentence, because automated repair without it is actively harmful. A search that accepts any edit making the failing test pass will, given a large enough rule library, eventually find one that hard-codes the answer. Such an edit is worse than the original bug; the bug was visible and the patch is not. The rule here is that a candidate must pass a seed that was *never used during the search*, which is the same train/test separation that any honest em-

pirical claim requires. This book's Part X8 makes the same point about a bug-find-rate curve, and Part X1 about a range justified by simulation: **a number validated on the data that produced it is not validated.**

Y4.2 The blocks under test, measured now

Table 494. The agent's blocks, run at build time against their golden models

Block	Golden model	Stimuli	Mismatches	Distinct outputs	Checker bites?	Wall time
crc32_8	Python's <code>zlib.crc32</code>	1,500	0	1,499	yes	0.0497 s
crc32_stream	—	52	0	51	yes	0.0553 s
enc8b10b	—	3,000	0	488	yes	0.0496 s
gf_mul	the repository's <code>gf.py</code>	1,500	0	808	yes	0.0442 s
regfile	—	1,500	0	169	yes	0.063 s
rs_syn_step	the repository's <code>gf.py</code>	1,500	0	796	yes	0.0431 s

Run by this chapter as the book is rendered, with Icarus Verilog. The 'distinct outputs' column is the one that makes the 'mismatches' column mean something; a zero in the fourth column beside a one in the fifth would be a false green.

Undergraduate review · Why the CRC block's golden model is `zlib` and not a hand-written CRC. If the reference and the design are written by the same person from the same understanding, they share that person's misunderstandings, and the comparison certifies the misunderstanding rather than detecting it. This is not hypothetical: reflected versus non-reflected CRC, initial value, and final XOR are four independent conventions and a hand-written reference gets at least one of them wrong about as often as the RTL does. **An independent reference is worth more than a more accurate one.** The same argument is why a modelling team is organisationally separate from the design team — the independence is the product.

Y4.3 Repair, measured on injected faults

The claim that the agent repairs Verilog is only worth making with numbers behind it. The following table injects known faults into the GF multiplier and reports what the search did, computed while this page was rendered.

Table 495. Injected faults and what the repair search did, run at build time

Injected fault	Mismatches before	Candidates tried	Rule that repaired it	Rejected for overfitting	Search time
overflow bit read one position low	393 / 400	7 of 7	MSB 인덱스 M-2 / M-1	0	10.2 s
accumulate with OR instead of XOR	384 / 400	3 of 7	OR / XOR	0	0.102 s
reduction shift in the wrong direction	390 / 400	5 of 7	우시프트 / 좌시프트	0	0.148 s

The search is exhaustive over single edits from the rule library, so a 'not repaired' result means the fault is outside the library — which is information, not a failure. The candidate counts are small because the rule library is deliberately narrow; a library that edits anything would find something for every fault and most of it would be wrong.

What this system is not. It is not a bug finder for logic it has no golden model of, and it is not a substitute for a verification plan. The rule library covers the small syntactic faults that a tired engineer introduces at two in the morning — an inverted comparison, a shift the wrong way, an index off by one — and nothing else. A missing state in a state machine, a protocol rule that was never implemented, or a specification misread will all pass the repair search untouched, because there is no single-

token edit that fixes them. **The value is not that it repairs everything; it is that it removes the trivial failures from the morning's queue**, with a patch and a minimal reproducer attached to each of the rest.

Y4.4 Running it for twenty-four hours

The launcher follows this repository's rule for background work, which exists because the obvious way does not work: a process started with a bare & from inside an agent session is reaped when that session's process exits, and the evidence for that is a night in which nothing ran and a message the next morning saying it had.

```
edu/agent/run.sh - the launcher
#!/usr/bin/env bash
# 24시간 회귀 에이전트를 **부모가 죽어도 사는** 방식으로 띄운다.
#
# CLAUDE.md 의 백그라운드 규칙 그대로: setsid + nohup + disown, 그리고
# stdin/stdout/stderr 를 전부 리다이렉트한다. claude -p 가 응답 직후 끝나도
# 이 프로세스는 남는다.
#
# 사용: bash run.sh [--적용] [시간]
#
# 확인은 `ps -p $!` 가 아니라 `pgrep -af agent.py` 로 한다 -- setsid 가 fork 하므로
# $! 는 이미 끝난 래퍼다. 셸 변수명에 한글을 쓰지 않는다 (bash 는 받지 않는다).
set -u

HERE="$(cd "$(dirname "$0")" && pwd)"
LOGDIR="${SE_AGENT_LOG:-/home/user/SE/logs}"
mkdir -p "$LOGDIR"
LOG="$LOGDIR/verif_agent.log"

APPLY=""
HOURS="24"
for a in "$@"; do
  case "$a" in
    --적용|--apply) APPLY="--적용" ;;
    *) HOURS="$a" ;;
  esac
done

setsid nohup python3 "$HERE/agent.py" --시간 "$HOURS" ${APPLY:+$APPLY} \
  > "$LOG" 2>&1 < /dev/null &
disown

sleep 3
if pgrep -af "agent.py" > /dev/null; then
  echo "살아 있다:"
  pgrep -af "agent.py"
  echo "로그: $LOG"
  sleep 2
  echo "--- 로그 첫 줄 ---"
  head -3 "$LOG"
else
  echo "띄우지 못했다. 로그:"
  cat "$LOG"
  exit 1
fi
```

Table 496. The three things that make an unattended launch survive

Element	What it detaches from	What happens without it
setsid	The parent's session and process group	The process dies with the session that started it

Element	What it detaches from	What happens without it
nohup	SIGHUP	The process dies when the terminal closes
disown	The shell's job table	The shell may still signal it on exit
< /dev/null and redirected output	The terminal's pipes	A write to a closed pipe kills the process, often hours later
pgrep -af agent.py	—	ps -p \$! reports nothing even when the job is running, because <code>setsid</code> forks when it is already a group leader, so <code>\$!</code> is the wrapper that has already exited. Believing it leads to starting a second copy that overwrites the first one's files

Do not put non-ASCII text in a `pgrep -f` pattern. The pattern is matched against `/proc/<pid>/cmdline`, whose bytes may not decode in the current locale; a pattern containing characters that cannot be decoded matches nothing, and a match of nothing reads as ‘finished’. A job that is running is then reported as done, or started a second time. Wait on a **PID** with `kill -0`, or match an ASCII-only fragment of the command line.

Worked example 84. What a night actually buys

Given. The loop measured above runs roughly 900–2000 regression rounds per minute on this container for these small combinational blocks, each round a fresh seed.

Method. Extrapolate to eight hours and compare with what a person can supervise. Then note the limit, which is not the compute.

Rounds per hour (order of magnitude)	10^5
Rounds in an eight-hour night	$\approx 10^6$
Distinct seeds consumed	one per block per round
Engineer-hours consumed	0
What limits the value	the golden models and the stimulus, not the machine

How this is got wrong. By reading the round count as coverage. A million rounds of the same random distribution explore the same region a thousand times. Part X8's coupon-collector arithmetic says where the return flattens, and the answer is far below a million for any of these blocks. **The honest use of an overnight run is not volume but variety:** rotate the constraint sets, inject faults deliberately to measure whether the suite still bites, and re-run the previous night's minimal reproducers as a growing regression. A loop that only turns the seed is a loop that stops learning after the first hour.

Problem. You arrive in the morning to nine failures, all in one block, all with the same minimal reproducer shape. What do you do first?

Solution. Deduplicate before debugging. Nine reports with one shape are one bug, and the first task is to confirm that by checking whether a single edit makes all nine reproducers pass — which the repair search has already attempted and logged. If it found an edit and the overfitting gate rejected it, **that rejection is the most informative line in the whole night's ledger:** it says a plausible local fix exists and is wrong, which usually means the bug is one level up from where it manifests. If the search found nothing, read the minimal reproducer and ask which of the four X8 categories it belongs to before opening the RTL. The failure the agent cannot repair is the one worth your morning; the ones it repaired are worth only a review of the patch.

Problem. Your manager asks whether the agent could write the RTL as well as fix it. What is the honest answer?

Solution. The honest answer distinguishes two things this chapter has kept separate. **Search over a small edit space with a strong oracle is reliable**, and that is what this agent does: the golden model is the oracle, the edit space is a dozen rules, and the three gates bound the false-accept rate. **Generation without an oracle is not**, because nothing is checking the result except a human reading it, and the failure mode is code that looks right. The useful framing for a design house is therefore: let generation propose and let the oracle dispose, and **invest in the oracle**. A team with an excellent golden model and a mediocre generator will out-ship a team with the reverse, because the first can accept help from any source and the second cannot tell good help from bad. That is also why the modelling role — the one this book is written for — becomes more valuable as generation improves, not less.

Y5. The Commercial Mechanics of Selling an IP Block

A design house that cannot price, license and support its work does not have a business, it has a hobby with good documentation. This part gives the arithmetic of the commercial side with the same seriousness as the technical parts, because the failure modes are just as specific and rather less discussed.

Y5.1 The three revenue models and when each applies

Table 497. How semiconductor IP is actually sold

Model	Customer pays	Suits	Risk to you	Risk to them
Licence royalty +	A licence fee, then a per-unit royalty	High volume, standard function	Royalty depends on their success and their honesty — audit rights matter	Ongoing cost; they will try to convert it to a buyout
Paid-up / perpetual licence	One fee, unlimited units	Customers who cannot tolerate per-unit accounting	You capture none of their upside	Large up-front commitment
Services NRE /	Engineering time	Where a one-person house should start	No leverage — you are paid once for each hour	Low; they own the result
Subscription	Annual fee for access and updates	Rapidly changing standards	Churn	Dependence on your survival
Open core + support	Support and customisation	Building reputation from nothing	Hard to monetise	Very low

Worked example 85. When a royalty beats a buyout, from the customer's side

Given. A block licensed at \$150 000 plus \$0.20 per unit, or \$600 000 paid up. The customer's forecast is 1 million units over three years, but forecasts in this market are routinely out by 3× in both directions.

Method. Compute the break-even volume, then compute what each party's exposure is above and below it. The break-even is arithmetic; the interesting part is the asymmetry around it.

Break-even volume	2,250,000
Royalty model cost at 1M units	3.5000×10^5 USD
Buyout cost	600,000 USD
Royalty cost at 0.3M units (forecast ÷ 3)	2.1000×10^5 USD
Royalty cost at 3M units	7.5000×10^5 USD
Your revenue at 3M units under buyout	600,000 USD
Your revenue at 3M units under royalty	7.5000×10^5 USD

How this is got wrong. By negotiating the break-even and not the distribution. Both parties can compute 2.25 million units; what decides the deal is who is more confident and who can bear the downside. A one-person house with no cash reserve should prefer the larger up-front fee even at a worse expected value, because **surviving to the next contract dominates expected value when a single bad quarter ends the business**. The standard compromise is a royalty with a cap and a floor, which bounds both parties' exposure and is easier to agree than either extreme.

Y5.2 What a customer actually receives

Table 498. The delivery package, item by item

Item	Form	Why it is not optional
RTL	Encrypted or plain System-Verilog	The product — but the smallest part of the package
Testbench and tests	UVM or equivalent, runnable	The customer must be able to re-verify after editing; without it your block becomes untouchable and therefore unusable
Reference model	C or C++, bit-accurate	Their system model needs it; also it is what makes your claims checkable
Synthesis scripts and constraints	SDC plus a script	Timing closure in <i>their</i> flow is where integrations fail
Lint and CDC waivers	Files with justifications	Their sign-off will run these tools; unexplained violations stop the project
IP-XACT / integration metadata	XML	Automated integration; increasingly a purchase requirement
Register map	Generated from one source	Hand-maintained duplicates diverge, and the divergence is discovered by software at the worst time
Documentation	Datasheet, integration guide, release notes	Support cost is inversely proportional to documentation quality
Known-issue list	A document	Omitting it does not remove the issues, it removes your credibility when they are found

Graduate · The known-issue list is the item that distinguishes a professional supplier, and it is counter-intuitive to first-time vendors. Every block has limitations: a configuration that was never verified, a corner that fails a lint rule for a defensible reason, a performance figure that holds only above a certain clock ratio. A customer who finds these themselves concludes you did not know; a customer who reads them in your release notes concludes you did. **The second customer calls you when they have a problem instead of calling your competitor.** Write the list, keep it honest, and put dates and work-arounds beside each entry.

Y5.3 Support: the cost nobody budgets

Table 499. Support load against customer count, at 1.5 engineer-hours per query

Customers	Queries per customer per year	Queries per year	Engineer-hours	% of a person-year (1800 h)	Working weeks
1	40	40	60	3.33	1.5
3	30	90	135	7.5	3.38
5	25	125	187.5	10.4	4.69
10	20	200	300	16.7	7.5
20	18	360	540	30	13.5

Queries per customer fall with experience because the documentation improves and the frequent questions get answered in it. **At twenty customers support is a full-time job**, which for a one-person house means either hiring, raising prices to slow growth, or investing hard in documentation and self-service — and the third is the only one available immediately.

Worked example 86. Pricing that accounts for support

Given. You expect 8 customers over three years for a block that took 44 weeks to build. Target: recover the build and support and earn a living. Your cost of living plus overhead is \$120 000 a year.

Method. Total the engineering, add the support from the table above, and divide. Then compare with what the market will bear, because the two are independent numbers and the smaller one wins.

Build effort	44 weeks
Support over 3 years at 8 customers	19.8 weeks
Total effort	63.8 weeks
Cost at \$120k/year	1.4723×10^5 USD
Break-even price per customer	18404 USD
With a 40 % margin	30673 USD
If only 3 customers materialise	65929 USD

How this is got wrong. By pricing from cost and ignoring the market. The last row is the real risk: with three customers instead of eight, the price that covers cost is more than double, and there is no reason a customer will pay it. The standard mitigations are to **start with services revenue**, which is paid whether or not the block sells; to **sell the first block below cost to a reference customer** in exchange for a public reference, which is worth more than the margin; and to **build blocks whose adjacent variants share most of the effort**, so the second customer in a family costs a fraction of the first.

Y5.4 Legal and quality items that bite a small vendor

Table 500. Contractual terms and what each one means in practice

Term	What the customer wants	What you should watch
Indemnification	You cover patent claims against the block	Cap it at fees received. Uncapped indemnity can exceed the company's value; it is the single most dangerous clause for a small vendor
Warranty	The block conforms to the specification	Define the specification precisely and tie the warranty to it, not to 'fitness for purpose'
Escrow	Source held by a third party in case you disappear	Reasonable; budget the fee and keep the deposit current
Acceptance criteria	A defined test they run	Insist the criteria be written and runnable before delivery. Undefined acceptance is an unbounded obligation
Exclusivity	Nobody else gets the block	Price it as the lost market, and time-limit it
Support term	Years of updates	Define what an update is; a new standard revision is a new product

Provenance of the source you ship is a hard requirement, not a formality. If any part of your block derives from code with a copyleft licence, or from a previous employer's work, or from a reference design with redistribution limits, a customer's legal review will find it — they scan for this — and the deal ends there. Keep a per-file record of origin, do not paste from sources you cannot name, and be able to produce that record on request. **The code appendix of this book is an illustration of the standard:** every file reproduced in it is named with its repository and its line count, and a reader can trace any line back to its source.

Problem. A prospective customer asks for a three-month free evaluation with full source. How do you respond?

Solution. Not with a flat refusal, and not with the source. The request is reasonable in intent — they cannot buy what they cannot assess — and unreasonable in form, because full source for three months is the product. The standard structures, in increasing order of what you give away: an **evaluation package** of documentation, the datasheet, the register map and simulation results, which costs you nothing and answers most questions; an **encrypted or obfuscated netlist or FPGA bitstream** they can integrate and measure but not read, which answers the performance question; and a **paid evaluation licence** whose fee is credited against the full licence if they proceed, which aligns both parties and filters out tyre-kickers at no cost to a serious buyer. **The paid evaluation is the one to propose**; a customer who will not pay a small fee to evaluate is very unlikely to pay a large one to license.

Problem. Your block has been shipping for a year when a customer reports a bug that affects every unit they have built. What do you do, in order?

Solution. Reproduce it before saying anything about cause. Ask for the configuration, the stimulus and the failing condition, and reproduce it in your own environment; a report is a hypothesis. **Then determine the blast radius** — which configurations, which versions, which other customers — because the answer determines whether this is one email or a notification to everyone. **Then notify affected customers before they find it**, with what is known, what is not, and when the next update will come; a vendor who reports their own bug keeps the account, and one who is caught concealing it does not. **Then fix, with a regression test that would have caught it**, and say in the release notes what that test is. **Finally, ask why the verification plan missed it** and fix the plan, not just the code — Part X8's mutation argument applies directly: if the suite did not catch this, inject it and find out what else of the same shape it would miss.

Y6. From a Paper to a Block, and Back to a Paper

Research papers are the cheapest source of architecture a design house has, and the most dangerous, because a paper's incentives are not a product's incentives. A paper must be novel; a product must be robust. A paper reports the configuration that worked; a product must survive the ones that did not. This part is a procedure for extracting value from the literature without inheriting its optimism — and, in its second half, for publishing without committing the same offence.

Y6.1 Reading a hardware paper in the order that matters

Table 501. What to read, in order, and what you are looking for

Read	You are asking	Stop if
The results table	What was actually measured, in what units, against what baseline	The baseline is the authors' own strawman rather than a published comparable
The methodology section	Simulated, synthesised, or fabricated? At what node? Post-layout or post-synthesis?	Post-synthesis numbers compared against someone else's post-layout numbers — a 20–40 % difference presented as a contribution
The assumptions	What is held fixed, and is it fixed in your application?	The gain depends on a parameter your customer sets differently
The limitations paragraph	What do the authors admit?	There is not one
The related work	Who else did this, and how does this compare?	It is a list of citations rather than a comparison
The algorithm	Can you implement it from the text alone?	Only now — reading the algorithm first is how a fortnight disappears

The recurring pattern in hardware papers is the unstated operating point. A result is reported at the configuration where the proposed technique wins, and the sweep that shows where it loses is absent. This is not usually dishonesty; it is page limits and the reviewing process. But it means **the first thing to do with a promising technique is to reproduce the sweep the paper did not print**, in your own model, and find the crossover. If the crossover is on the wrong side of your customer's operating point, the technique is a publication and not a product — and you have learned that in a week rather than a quarter. This repository's own record contains the mirror image of this mistake: a headline figure of '1/59 the area' taken from the most favourable corner of a sweep, with the loss condition present elsewhere in the same document.

Worked example 87. Reproducing the missing sweep: a worked reading

Given. A paper claims an unrolled architecture removes a timing bottleneck and reports a 59× area advantage over the standard-cell baseline at one operating point.

Method. Identify the parameter the advantage depends on, sweep it, and find where the sign changes. Here the parameter is the number of taps that must be unrolled, which grows with channel loss; area grows as 2^n .

Claimed advantage at the reported point	59×
Unrolled taps at that point	2
Area factor at $n=2$	4
Unrolled taps at a realistic 30 dB channel	5
Area factor at $n=5$	32
Advantage rescaled	7.38×
Taps at which the advantage disappears	7.88

How this is got wrong. By quoting the headline and moving on. The arithmetic above takes ten minutes and changes the conclusion from ‘adopt’ to ‘adopt for short channels only’, which is a completely different product decision. The general procedure is: **find the exponential, and find where it eats the constant.** Almost every architecture paper contains exactly one such exponential, and the paper's operating point is chosen just below where it bites.

Y6.2 The prior-art search, done so that it means something

This book's repository enforces a rule that a prior-art note must be committed before the code for a topic. The rule exists because the same failure occurred five times: choose a topic, build, measure, write, and only then discover the work already existed. The procedure below is what the rule requires.

Table 502. A prior-art note that is worth the time it takes

Section	Content	The failure it prevents
Closest prior work	arXiv or DOI, year, and one sentence on what it did	‘I could not find anything’, which is not a finding
How we differ	One paragraph, specific, falsifiable	A difference that evaporates under a reviewer's first question
Queries run	The literal search strings, at least three, with the databases	An unrepeatable search
Where you have not looked	Named: patents, a paywalled venue, a language, an industry body's documents	Treating absence of evidence as evidence of absence
Confidence of each citation	Full text, abstract, listing, or snippet	Citing something you have not read as though you had
Possibilities not yet eliminated	A list	A false sense of completion

Graduate · **The confidence marker is the mechanically enforceable part and therefore the part worth automating.** This repository requires every citation in a paper to carry, on the document's face, whether the author read the full text, the abstract, a listing entry, or only a search snippet; a gate checks the document against a ledger of citations and fails the build if they disagree. What the gate *cannot* check is whether the marker is honest, and the repository's notes say so explicitly. **That is**

the correct division: the machine forces the claim to be stated, and a human is accountable for its truth. A process that pretends the machine can verify reading is worse than one that admits it cannot, because it invites everyone to stop looking.

Problem. You find a patent that appears to cover the architecture you were about to build. What are the possible responses, and which are realistic for a one-person house?

Solution. Six responses exist and only some are realistic. **Design around:** read the *claims*, not the abstract — the claims are the legal scope and are usually much narrower than the description — and find an element of the independent claim you can avoid. This is the normal answer and it is realistic. **Check the status:** the patent may have lapsed for unpaid maintenance fees, may never have been granted in your customers' jurisdictions, or may expire before your product ships; all are cheap to check and all are common. **License it:** realistic only if the holder licenses, which large semiconductor holders often do not to small parties. **Challenge validity:** not realistic — the cost exceeds a small house's annual revenue. **Ignore it:** not an option; wilful infringement carries enhanced damages, and your customer's legal review will find it anyway. **Change topic:** often the correct answer, and the reason the prior-art search comes first. **The meta-point is that the patent is information, not a verdict,** and reading the claims carefully is a skill worth acquiring because it converts a project-ending discovery into a design constraint about half the time.

Y6.3 Measuring so that the number survives a reviewer

The four rules below are this repository's response to four specific measurements that were reported and were wrong. They are stated as a checklist because each failure looked, at the time, like a result.

Table 503. Four checks before a number is said aloud

Check	What went wrong without it	How to do it
Did you measure the quantity you defined?	A tool reported the longest path in the whole netlist; the quantity wanted was the longest path <i>around a loop</i> . Different number, same units	Isolate the quantity — cut the loop, build the combinational equivalent, and measure that
Is the operating point healthy?	A residual analysis was run at a point where the bit error rate was 0.49 — the link was not working, so every derived number was noise	Sweep first, find a working point, then measure. Report the point
Is there an independent cross-check?	A reconstruction double-counted a term; the error was invisible until the same quantity was computed a second way	Closed form, a second estimator, or a hand calculation — and print the difference
Have you killed the trivial explanations?	'Seven gates, zero mismatch' — because at that operating point the equalizer taps were near zero and the loop was doing nothing	Write the list of boring reasons the number could look good, then measure each one away

If you cannot do all four, say which one you could not do. A number with a stated gap is usable by a reader, who can weigh it. A number presented as complete when it is not is a claim the reader cannot calibrate, and when it turns out to be wrong it costs more than the information was ever worth. This applies with full force inside a company: the colleague reading your slide has no way to know which checks you ran unless you say.

Problem. Your simulation shows a $3\times$ improvement. What do you do before telling anyone?

Solution. Assume it is a bug, because at that magnitude it usually is, and spend an hour trying to prove it. **Check the baseline actually runs:** the commonest cause of a large speed-up is a baseline that is mis-configured, unoptimised, or silently failing, and comparing against a broken baseline is the strawman failure from Y6.1 committed against yourself. **Check the units and the normalisation** — per operation, per cycle, per joule and per second are four different claims. **Check that both sides did the same work:** a fast path that skips a case the baseline handles is not faster. **Compute the improvement a second way,** from first principles, and see whether $3\times$ is even plausible given what changed; if the change removed 20 % of the operations, a $3\times$ speed-up needs an explanation beyond the operation count. **Only then look for the mechanism,** and when you report it, report the mechanism rather than the multiplier — a number with an explanation survives scrutiny and a number without one does not.

Problem. Should a one-person design house publish at all?

Solution. Yes, and for reasons that are commercial rather than academic. A publication is **evidence a customer can check** before trusting a supplier they have never heard of, which is the central problem of a new house. It is **defensive prior art:** published material cannot later be patented against you, which is cheap insurance compared with a patent of your own. It **forces the four checks above,** because a reviewer will ask, and work that survives review is work you can quote in a datasheet. And it **brings the people** — collaborators, customers and the occasional acquirer read the literature of their field. The cost is real: a paper is several weeks, and a rejection is several more. The pragmatic compromise many small houses take is an arXiv preprint plus a workshop or industry conference talk, which captures most of the benefit at a fraction of the cost of a top-tier submission, and **which is perfectly compatible with holding back the implementation details that are actually the product.**

Y7. Debug and Bring-Up, With the Arithmetic

Bring-up is the phase in which the difference between engineers is largest, and most of that difference is method rather than knowledge. The method is binary search over hypotheses, executed with discipline, and it can be made quantitative: each observation should halve the space of possible causes, and an observation that does not is a waste of an afternoon.

Y7.1 Debugging as information gain

Why the right measurement is the one that splits the space evenly

	Step	Justification
1	Let there be N plausible causes, each with prior probability p_j .	<i>The hypothesis space after reading the symptom.</i>
2	A test partitions the space: outcome A is consistent with a subset, outcome B with the rest.	<i>Any observation does this, well or badly.</i>
3	The information gained is the entropy reduction, maximised when the two subsets have equal probability mass.	Not equal count — equal probability. <i>Splitting off one unlikely cause teaches almost nothing.</i>
4	k ideal tests reduce N equiprobable causes to $N/2^k$.	<i>So twenty causes need about five tests, not twenty.</i>
5	A test whose outcome you can predict has zero information.	<i>If you are confident what will happen, do not run it. This single rule eliminates most wasted bring-up time.</i>

Table 504. Tests needed under binary search versus one-at-a-time elimination

Plausible causes	Ideal tests ($\log_2$)	One-at-a-time, worst case	One-at-a-time, average	Ideal as % of worst case
4	2	4	2	50
8	3	8	4	37.5
16	4	16	8	25
32	5	32	16	15.6
64	6	64	32	9.38

A bring-up with 32 plausible causes is five good measurements or thirty-two bad ones. **The whole skill is in choosing tests that split the space rather than tests that confirm a favourite hypothesis.**

Graduate · **The favourite-hypothesis trap has a specific signature.** An engineer who believes the problem is in module X runs tests that would be consistent with X, gets results consistent with X, and becomes more confident — while every one of those tests was also consistent with Y and Z. The discipline that prevents it is to **write down what each outcome would rule out before running the test**. If the answer is ‘nothing, either way’, the test is confirmation and not evidence. This is the same rule as the verification one in Part X8: a test that no broken implementation would fail is not a test.

Y7.2 The bring-up sequence, in dependency order

Table 505. Bring up in this order, because each step depends on the previous

Step	Check	If it fails
1. Power	Every rail at the right voltage, in the right sequence, with the ripple in spec	Nothing above this means anything. Sequence violations can damage parts

Step	Check	If it fails
2. Clocks	Every clock present, at the right frequency, with acceptable jitter	Measure at the die if possible; a clock present at the oscillator is not a clock present at the block
3. Reset	Asserted long enough, released cleanly and synchronously	The most common silent failure — a block that half-resets produces a state that no simulation ever visited
4. Register access	Read an ID register; write and read back a scratch register	If this fails, everything above is suspect; if it works, the bus, clock, reset and power are all proven at once
5. Static configuration	Program the block; read back every register	Read-back mismatch localises to the register file immediately
6. Loopback	Internal loopback first, then external	Separates the block from the channel — the single most valuable feature to design in
7. One transaction	One packet, one frame, one transfer	Debuggable; a stream is not
8. Line rate	Continuous traffic	Reveals everything the single transaction hid: buffering, back pressure, rate adaptation, alignment
9. Corner conditions	Temperature, voltage, rate extremes	Where the margin problems live

Step 4 is worth more than its position suggests. A readable identification register with a known constant value proves, in one transaction, that the supply is up, the clock is running, the reset released, the bus is connected, the address decode is right and the endianness matches. There is no other single test with that much power, and providing one costs a design a dozen gates. **An IP block without an ID register and a scratch register is harder to bring up by a factor that is hard to overstate**, and their absence is a reasonable reason for a customer to choose a competitor.

Worked example 88. Why internal loopback pays for itself in one bring-up

Given. A SerDes-attached block fails to pass traffic. Without loopback there are, say, 24 plausible causes spanning the transmitter, the channel, the connector, the far end and the receiver. Internal loopback splits them into 'inside the chip' and 'outside'.

Method. Apply the information argument: count the tests needed with and without the feature, at roughly two hours per test including setup.

Plausible causes	24
Ideal tests without loopback	5
Causes remaining after a loopback result	12
Ideal tests after loopback	5
Hours saved if each test costs 2 h	0
But: tests without loopback are <i>not</i> even splits	this is the real saving
Realistic sequence without loopback	10–15 tests, several requiring board rework
Realistic sequence with loopback	4–6 tests, none requiring rework

How this is got wrong. By counting only the ideal test count, which barely changes. The value of loopback is not that it reduces $\log_2 N$ by one; it is that **without it, no available test splits the space evenly** — every measurement is at one end of the chain and rules out little. A feature that creates an even split where none existed is worth far more than its effect on the count suggests. The same argument justifies per-stage error counters, snapshot registers and a pattern generator: each one manufactures a clean partition of the hypothesis space.

Y7.3 Reading a symptom

Table 506. Symptom to first hypothesis

Symptom	Most likely class	The distinguishing test
Works at low rate, fails at high rate	Timing or bandwidth	Vary the rate and find the exact threshold; a sharp threshold means timing, a gradual one means buffering
Fails after a fixed time, always the same	A counter wrapping, or a buffer filling	Compute what wraps in that time at that rate — the arithmetic usually names the register
Fails randomly, low rate of failure	Metastability, marginal timing, or a rare data pattern	Vary temperature and voltage: timing moves, a data pattern does not
Works in simulation, fails in silicon	Something not modelled	Reset, clock, power, X-propagation, or an unmodelled analogue behaviour. List what the simulation did not contain
Fails only with a specific partner	Interoperability	Capture the link and compare against the specification, not against your other partner
First transaction after idle fails	A power or clock state exit	Disable the low-power state and see if it disappears
Corrupt data with a regular period	Rate mismatch or an alignment marker	Compute the period in symbols and compare with the marker interval

Problem. A link passes traffic for exactly 71 minutes and then drops a frame, reliably. What wraps in 71 minutes?

Solution. Do the arithmetic rather than guessing. At 1 Gbit/s and 8 bits per byte a byte counter increments 125 million times a second; 71 minutes is about 4260 s, giving roughly 5.3×10^{11} bytes — close to 2^{39} . At a 156.25 MHz clock, 4260 s is about 6.7×10^{11} cycles, near $2^{39.3}$. So look for a counter of about 39 or 40 bits — a 40-bit statistics counter, or a timestamp. **The technique generalises: convert the failure period into counts of every periodic event in the system and see which lands on a power of two.** It also works for suspiciously round numbers in other bases: 71 minutes is not obviously special, which is exactly why the conversion is necessary. A failure at 49.7 days would be a 32-bit millisecond counter and is recognisable on sight; the same reasoning found it.

Problem. Silicon fails a case that simulation passes. Where do you look first, and what does that tell you about your verification plan?

Solution. Look at the list of things the simulation did not model, because that is where the answer is by definition. The usual entries: **reset** — simulation often starts from an initialised state that silicon never has, so an uninitialised register that simulation reads as a known value reads as garbage in silicon; **X-propagation** — a simulator's optimistic X handling can let an unknown resolve to a definite value through a multiplexer, hiding exactly this class of bug, which is why X-pessimistic or four-state gate-level simulation exists; **clock and power sequencing**, rarely modelled at all; **analogue behaviour**, from supply droop to the settling of an off-chip device; and **real timing**, since RTL simulation is untimed. What this says about the plan is concrete: each of these has a known mitigation — randomised reset values, X-pessimistic simulation, gate-level simulation with back-annotated timing, and a modelled power sequence — and **the plan should name which ones it uses and which it consciously declines**. A plan that does not mention them has not declined them; it has forgotten them, and the difference shows up here.

Y8. The One-Person Path from Model to Product

The three-team structure of a large company — design, modelling, verification — exists because independence catches bugs. A single engineer cannot have organisational independence, so it has to be manufactured by process: the golden model is written before the RTL, from the specification rather than from the design, and it is never modified to agree with the RTL. This part is the procedure, with the realistic schedule and the places it goes wrong.

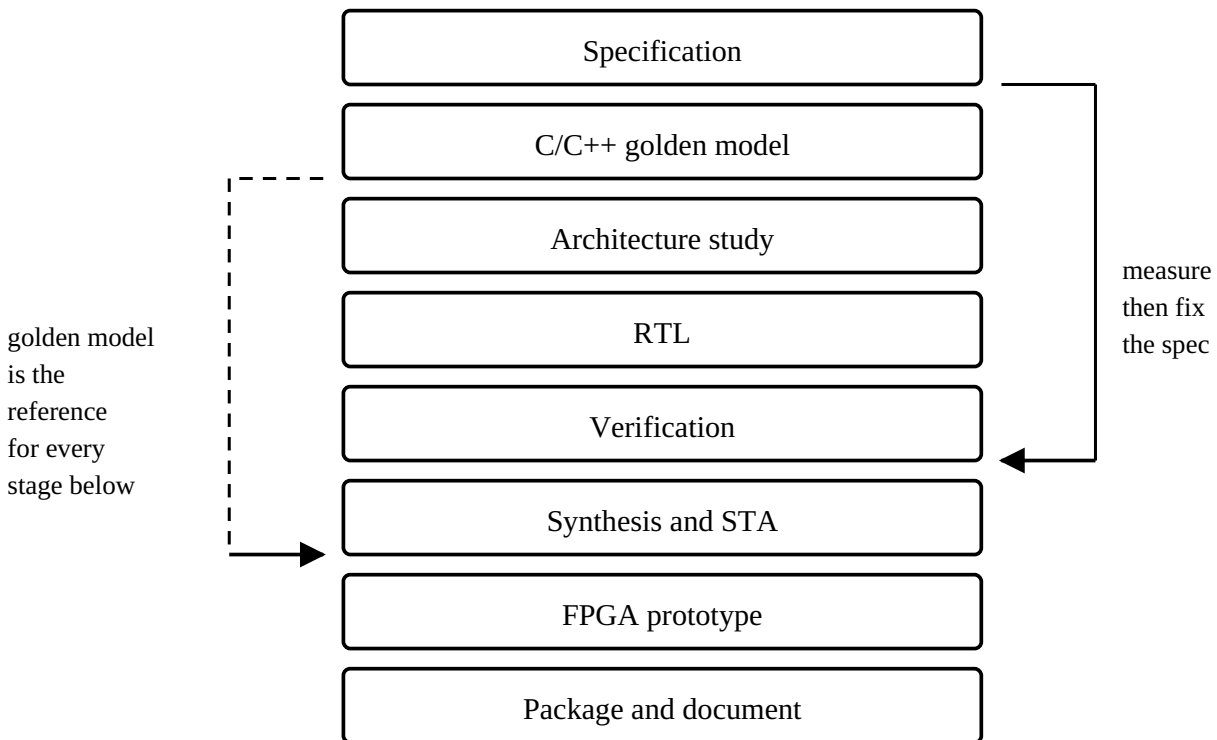


Figure 67. The solo flow. The dashed line is the one that matters: the golden model is the reference for every stage below it, which is what replaces organisational independence.

Y8.1 The rule that replaces a second engineer

Table 507. How to be your own independent checker

Rule	Why	What breaks without it
Write the model before the RTL	The model then encodes your reading of the specification, not your design	The model inherits the design's misunderstanding and certifies it
Never edit the model to match the RTL	A disagreement is information; resolving it by editing destroys the information	The commonest single failure of solo verification — and it feels productive while you do it
Resolve every disagreement by returning to the specification	One of the two is wrong and the text says which	You pick the one that is less work to change
Use a third source where one exists	zlib, a reference implementation, a vendor model, published vectors	Two of your own artefacts agreeing proves only that you are consistent
Write the test before the fix	Otherwise the fix is unverified and may be wrong	Regressions reappear

Rule	Why	What breaks without it
Record why each parameter has its value	You will not remember in six months, and a customer will ask	Undocumented constants become unchangeable

‘I will fix the model later’ is the moment a solo project loses its verification. When the RTL and the model disagree and the deadline is close, editing the model makes the test pass in minutes and investigating takes hours. Every such edit removes one independent check permanently, and they are not recoverable later because nobody remembers which ones were principled. The practical defence is mechanical: **keep the model in a separate commit history and require a written reason in the commit message for any change to it after the RTL exists.** The friction is the point.

Y8.2 A realistic schedule, with the failure points marked

Table 508. A 39-week single-engineer plan for a medium protocol block

Stage	Weeks	Cumulative	Where it goes wrong
Specification study and requirements	3	3	Under-reading the standard; the parameters you assume are the ones that fail interoperability
Golden model in C++ or Python	3	6	Modelling what you intend to build rather than what the spec says
Architecture study and word lengths	2	8	Choosing uniform word lengths — Part X1's sweep exists for this
RTL, first pass	5	13	Writing before the architecture is settled
Testbench and checkers	4	17	Checkers that cannot fail; run mutation testing here
Random and directed regression	5	22	Believing a flat find rate; Part X8's recovery test
Lint, CDC, synthesis	2	24	Discovering a CDC problem that needs an architecture change
Timing closure at target	2	26	Finding that the target was never achievable — check in week 2, not week 24
FPGA prototype and bring-up	4	30	No loopback or ID register — Part Y7
Documentation and packaging	3	33	Leaving it to the end, when the details are forgotten
Buffer for the unforeseen	6	39	Omitting it

The buffer is 15 % of the total and it is the row most often deleted by someone who wants the number smaller. A plan without a buffer is not an optimistic plan, it is a plan that has already failed, because the unforeseen is the only certainty in the list.

Worked example 89. What the schedule implies about what you can build

Given. 39 weeks for one block, a working year of 46 weeks, and a need to spend some of it selling and supporting.

Method. Compute how many blocks a single engineer can produce and what that means for the product strategy.

Weeks per block	39
Working weeks per year	46
Weeks available for sales and support	7
Blocks per year	1.18
Blocks in three years, ignoring support growth	3.54
With support at 20 % from year two	3.07
Implication	a family of related blocks, not unrelated ones

How this is got wrong. By planning a catalogue. At roughly one block a year, a catalogue of unrelated blocks takes a decade and each one is supported alone. The strategy that fits the arithmetic is a **family**: one architecture parameterised across a range — the same FEC core at three code rates, the same MAC at three line rates — where the second member costs perhaps a quarter of the first and shares its verification environment, its documentation structure and its support material. **Choose the first block for the family it opens, not for its own merits.**

Y8.3 What to build first

Table 509. Criteria for a first block, weighted for a house with no reputation

Criterion	Why it matters more than it looks	Good sign
A specification you can obtain	A block you cannot legally read the standard for cannot be built	Open standard, or a membership you can afford
A checkable golden model	Without an independent reference, solo verification is much weaker	Published test vectors, or an open reference implementation
Soft, not hard	A hard macro needs analogue skills and foundry access	The block is RTL from end to end
Changing standard	Incumbents' blocks go stale; a new revision resets the field	A draft is in ballot now
Few credible suppliers	You can win	Searching finds two vendors, not twenty
Adjacent to a family	Second and third members are cheap	The same core serves several rates or profiles
A customer you can name	Speculative IP is how houses die	Someone has said they would evaluate it

Graduate · **Applying these criteria is what selected this book's own subject.** The PCS and FEC sublayer of a high-speed Ethernet link scores on every row: the sublayer is soft RTL between two hard boundaries; Reed–Solomon has an independent reference in any number of open implementations and a golden model that can be verified by its algebraic properties rather than by sample values; the 802.3dj revision is in progress, which resets the incumbents; the supplier list is short; and the family extends naturally across code rates and generations. **The row that is not yet satisfied is the last one**, and this book says so rather than quietly omitting it — a named first customer is the thing that converts the other six rows from an argument into a business.

Problem. You have 39 weeks and no income. What is the order of operations that keeps you solvent?

Solution. Invert the order the technical plan suggests. **Sell services first:** contract verification, modelling or integration work pays weekly and, crucially, puts you in front of the customers who will later buy the block — the conversations are the market research. **Build the golden model and the architecture study in the gaps,** because they are the parts that need thinking rather than long uninterrupted stretches, and because they are what you show a prospective customer to prove you understand the problem. **Do the RTL and verification in a funded block** — ideally funded by a customer who wants the first instance and will accept that you retain the rights, which is a negotiation worth having explicitly and early. **Do not build speculatively for 39 weeks on savings:** the failure mode is not that the block is bad, it is that you run out of runway three weeks before the first sale, and at that point the asset is worth nothing to you and a great deal to whoever buys it from the liquidator.

Y9. Standards, Compliance and Interoperability

A protocol block is worth what it can be shown to interoperate with. That showing is a separate activity from verification, with its own vocabulary, its own costs and its own failure modes, and it is the part of the work that first-time vendors most consistently under-budget. This part lays it out.

Y9.1 Three different things called ‘testing’

Table 510. What each activity proves and what it does not

Activity	Question answered	Who runs it	What it misses
Verifica- tion	Does the block do what I specified?	You	Whether your reading of the standard is right
Conform- ance	Does the block do what the standard says?	You, or a test house, against a published test suite	Whether real partners interoperate — conformant implementations can still fail together
Interoper- ability	Does it work with this specific partner?	A plugfest, or a lab with partner equipment	Partners not present; the next revision
Certifica- tion	Will the body let me use the logo?	An accredited laboratory	Everything above — it is a process outcome, not a technical one

Two conformant implementations can fail to interoperate, and the reason is structural rather than anyone's fault. Standards contain optional features, permitted ranges and ambiguous sentences. If your block implements an option and the partner does not, both conform and neither works with the other unless the negotiation that selects it is itself implemented correctly on both sides — and negotiation code is under-tested everywhere, because testing it requires a partner that behaves differently from you. **The practical consequence is that the negotiation and fallback paths deserve more verification effort than the steady-state datapath**, which is the opposite of where effort naturally goes. In every protocol in this book, the bugs found at plugfests are concentrated in link training, capability exchange and error recovery.

Y9.2 The cost of a standard, itemised

Table 511. What conformance costs before any revenue arrives

Item	Order of magnitude	Note
The specification document	\$0–\$5,000 per document	Some are free (IETF); most industry standards are not
Body membership	\$5,000–\$50,000+ per year	Often required to obtain drafts, to attend, or to use the name
Test suite or test equipment	\$10,000–\$200,000+	Protocol analysers and compliance fixtures; rentable
Plugfest attendance	\$2,000–\$10,000 per event plus travel	The highest-value line in this table per dollar
Accredited laboratory testing	\$10,000–\$100,000 per campaign	Where a logo is required
Engineering time to fix what it finds	4–12 weeks	The line that is always omitted from the budget

Ranges, not quotations — the actual figures depend on the body and the year, and this repository has not obtained current price lists. The purpose of the table is to prevent the commonest planning error, which is budgeting zero for all six rows.

Worked example 90. Whether to certify at all

Given. A block that could be sold with or without a logo. Certification costs \$60 000 and eight weeks. Without it you expect 4 customers over three years at \$80 000; with it you expect 7, and you can charge \$95 000 because the logo removes a risk from the buyer's evaluation.

Method. Compute both revenues net of cost and time. Then note the factor the arithmetic does not contain.

Revenue without certification	320,000 USD
Revenue with certification	665,000 USD
Certification cost	60,000 USD
Eight weeks of engineering at \$120k/yr	18462 USD
Net with certification	5.8654×10 ⁵ USD
Net without	320,000 USD
Difference	2.6654×10 ⁵ USD

How this is got wrong. By treating the customer counts as data. They are forecasts, and the forecast with certification is the one you have least evidence for, since it is the counterfactual. The robust way to use this calculation is not to trust its output but to find the break-even customer count and ask whether it is plausible: here certification pays if it brings even one additional customer at the higher price. **Framed that way the decision is usually easy**, and the arithmetic has done its job by converting an argument about strategy into a question about one number.

Y9.3 Getting value from a plugfest

Table 512. Before, during and after

Phase	Do	Do not
Weeks before	Test against every partner implementation you can obtain, including open-source ones; exercise every negotiation path	Arrive to discover a problem you could have found at home
Bring	Two of everything, a logic analyser or protocol analyser, a way to re-build and reload on site, and a colleague if possible	Rely on borrowing; the other vendors are busy
Instrument	Counters, snapshot registers, and a way to capture the first failing exchange	Debug by re-running and watching
Record	Every partner, every configuration, every result — including the ones that worked	Remember it afterwards; you will not
When something fails	Determine which side deviates from the text before discussing it	Assert that the partner is wrong — roughly half the time it is you
After	Turn every failure into a regression test; publish an interoperability list	Fix the symptom and move on

Graduate ·The interoperability list is a sales document as much as an engineering one. ‘Tested against these seven implementations, at these configurations, on these dates’ answers the question a buyer cannot otherwise answer about a supplier they do not know, and it answers it with something checkable. It also has a defensive function: when a customer reports an interoperability failure with an eighth partner, the list makes it immediately clear that this is new territory rather than a regression, which changes the conversation entirely. **Keep it current and date every row**; a list without dates is read as a list of claims rather than of tests.

Problem. A partner at a plugfest insists your implementation is wrong. You believe theirs is. How do you proceed?

Solution. Stop arguing and go to the text, together. Find the clause, read it aloud, and establish what each implementation does at that point in the exchange with a capture rather than with an assertion. Three outcomes follow and all three are useful. **The text is clear and you are wrong:** fix it, and you have discovered a real bug cheaply. **The text is clear and they are wrong:** show them, courteously, and consider whether to interoperate with their behaviour anyway — if they ship in volume, your customers will meet them, and a conformant block that fails against a popular partner is a commercial problem whoever is right. **The text is ambiguous:** this is the most valuable outcome, because it means everyone has the problem. Raise it with the working group; a clarification with your name on it is reputation, and it is one of the few ways a small house becomes visible to the people who buy. **In all three cases the conduct matters more than the verdict:** plugfests are small worlds and the engineer across the table is a future customer, partner or employer.

Problem. Why should a soft-IP vendor care about compliance at all, when the customer's chip is what gets certified?

Solution. Because the customer's certification failure becomes your problem regardless of whose name is on the certificate. Practically, three things follow. **Your block must be testable in their chip:** the compliance tests require modes — loopback, pattern generation, error injection, counter read-out — that must exist in the RTL and be reachable from their software, and adding them late is expensive. **You must supply the evidence they will be asked for:** which clauses your block implements, which options, which are the customer's responsibility. And **you should have run what you can run yourself**, because a block that has never seen a compliance suite will fail one, and it is far cheaper to fail it in your lab than in theirs. **A vendor who arrives with a clause-by-clause implementation statement and their own test results is selling a lower-risk product**, and that is precisely what the premium in protocol IP is paid for.

Y10. The Register Map and the Software Interface

The register map is the part of a block that software sees, the part a customer integrates against, and the part that is hardest to change after release. It is also, in most projects, the artefact with the least design attention per hour of downstream consequence. This part works through one map in full and states the rules that each entry follows.

Y10.1 A worked map

Table 513. A minimal but complete register map, and why each register exists

Off-set	Register	Access	Reset	Why it is here
0x00	ID	RO	0x1A2B0100	Proves supply, clock, reset, bus and decode in one read (Part Y7).
0x04	SCRATCH	RW	0x00000000	Proves the write path independently of any functional side effect.
0x08	CTRL	RW	0x00000000	Everything that changes behaviour, in one register the driver can read-modify-write atomically.
0x0C	STATUS	RO	0x00000000	Live state. Never latched -- a latched status needs a clear, and a clear is a race.
0x10	IRQ_STATUS	W1C	0x00000000	Write-1-to-clear so two drivers cannot clear each other's bits.
0x14	IRQ_ENABLE	RW	0x00000000	Separate from status so masking never loses an event.
0x18	ERR_COUNT	RC	0x00000000	Read-to-clear and saturating: a counter that wraps silently is worse than one that sticks at maximum.

Seven registers. *Three of them (ID, SCRATCH, LOOPBACK) exist only for bring-up and debug, and they are the ones most often omitted by a designer who is thinking about function rather than about the week the customer spends getting first traffic.*

Table 514. ID at offset 0x00, reset 0x1A2B0100

Bits	Field	Access	Description
31:16	VENDOR	RO	Vendor identifier, constant
15:8	BLOCK	RO	Block identifier
7:0	VERSION	RO	Major.minor, BCD

Table 515. SCRATCH at offset 0x04, reset 0x00000000

Bits	Field	Access	Description
31:0	DATA	RW	Any value; no hardware effect

Table 516. CTRL at offset 0x08, reset 0x00000000

Bits	Field	Access	Description
0	ENABLE	RW	1 starts the datapath
1	SOFT_RESET	W1S	Self-clearing; resets the datapath only
2	LOOPBACK	RW	Internal loopback for bring-up
7:4	LANES	RW	Active lane count minus one

Table 517. STATUS at offset 0x0C, reset 0x00000000

Bits	Field	Access	Description
0	READY	RO	Datapath has completed initialisation
1	ALIGNED	RO	Lane alignment achieved
7:4	STATE	RO	Current link state, for debug

Y10.2 Access types, and the race each one prevents

Table 518. Register access types

Type	Behaviour	The problem it solves	Where it goes wrong
RO	Read only	—	Writing is silently ignored — specify that, or software will assume it faulted
RW	Read and write	—	Read-modify-write on a register hardware also writes loses the hardware's update
W1C	Write 1 to clear a bit	Two drivers clearing different bits do not clobber each other	A driver that writes 0xFFFFFFFF clears events it never handled
W1S / W1T	Write 1 to set / toggle	Same, for setting	—
RC	Read clears	A counter read without a separate write	A debugger that reads registers disturbs state — mark such registers so tools skip them
WO	Write only	Command registers	Unreadable state cannot be restored after suspend; prefer RW with a self-clearing bit
RW1	Writable once until reset	Lockable configuration	Software must know the lock exists

The read-modify-write race is the defect this table exists to prevent, and it is worth stating concretely. Suppose STATUS bits lived in CTRL. A driver reads CTRL, changes ENABLE, and writes it back. Between the read and the write, hardware set an error bit. The write puts back the old value of that bit and the error is lost — silently, rarely, and only under load. **The rule is structural: never mix software-written and hardware-written bits in one register**, and where an event must be cleared by software, use W1C so that the write names exactly the bits it intends. This costs one extra register and removes a class of bug that is close to undebuggable in the field.

Y10.3 One source, many outputs

A register map appears in at least six places. Maintaining them by hand guarantees they diverge, and the divergence is discovered by a driver author at the worst moment.

Table 519. What is generated from the register description, and who consumes it

Artefact	Consumer	What divergence costs
RTL register file	The design	—
C header with offsets and field masks	The driver author	A driver that writes the wrong bit
IP-XACT or SystemRDL	The customer's integration tooling	Manual integration — increasingly a purchase blocker
Documentation tables	Everyone	A datasheet that lies
UVM register model	Verification	Tests that pass against a stale model

Artefact	Consumer	What divergence costs
Debugger description	Bring-up	Wrong values read during the week they matter most

Worked example 91. The cost of maintaining six copies by hand

Given. A 60-register map, six artefacts, and a change rate of roughly 8 register changes per month during development.

Method. Count the edits and the probability that at least one is missed, assuming a 2 % chance of missing any individual edit.

Artefacts	6
Edits per month	48
Edits over a 9-month project	432
P(no edit missed) at 2 % per edit	1.62×10^{-4}
Expected number missed	8.64
Effort to build a generator	1–2 weeks
Effort saved	0.54 weeks, plus the bugs

How this is got wrong. By evaluating the generator against the edit effort alone. The edits are cheap; the missed ones are not, and the expected count here is about nine. Each is a divergence between what software believes and what hardware does, found at integration. **The generator's value is in the defects it removes, not the keystrokes,** and stating it that way is also how to justify the two weeks to a manager who sees only the keystrokes.

Problem. What should a register do when software writes a reserved bit?

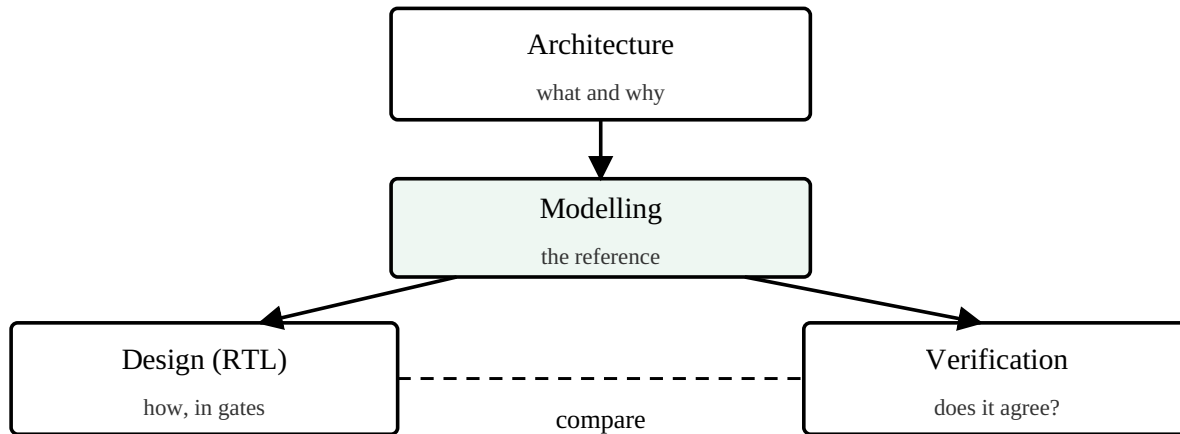
Solution. Define it, and define it in the direction that leaves you room. The two defensible choices are **‘reserved, write zero, read undefined’** and **‘reserved, preserve on write, read as written’**. The second is much better for forward compatibility: software does a read-modify-write and unknowingly preserves bits a future version defines, so an old driver keeps working with new hardware. The first is simpler and breaks that property. What is *not* acceptable is leaving it unspecified, because software will discover the behaviour empirically and depend on it, and you will then be unable to change it. **The same principle applies to every ‘don’t care’ in a specification:** an unspecified behaviour is not free, it is a behaviour your customers will come to rely on before you have decided what it should be.

Problem. Your block is in a power domain that can be switched off. What does that do to the register map?

Solution. It adds a requirement that is easy to miss and expensive to retrofit: **every register software may need after resume must be restorable**. That means no write-only registers whose value cannot be read back, no state that exists only in hardware counters software cannot reconstruct, and a documented save/restore list telling the driver exactly which offsets to store. It also raises a design question about which registers should be in an always-on domain — typically the ones controlling wake-up and the power sequence itself, since a register that controls power-up cannot live in the domain being powered up. **Getting this wrong produces a block that works until the first suspend/resume cycle,** which in a mobile or automotive product is within minutes of the customer first running real software on it.

Y11. The Modelling Engineer's Working Day

This book was written for someone joining a modelling team. This part describes the job concretely: what is produced, how it is judged, what the first ninety days look like, and the specific habits that distinguish a modelling engineer whose work is trusted from one whose work is checked.



A bug is a disagreement between the model and the design.

Which one is wrong is a separate question, answered by the specification.

Figure 68. The three roles and the place of the model. The dashed line is the comparison that defines a bug.

Y11.1 What a modelling engineer produces

Table 520. The deliverables, in the order they are usually needed

Deliverable	Form	Judged by
Algorithm study	Python or MATLAB, plots, a written recommendation	Whether the architecture chosen from it survives implementation
Bit-accurate reference model	C or C++, callable from the testbench	Whether it agrees with the RTL, and whether it is right when it does not
Word-length study	A sweep and a recommendation per node	Area saved against the metric held
Performance model	A cycle-approximate or cycle-accurate model	Whether the predicted throughput matches silicon
Test vectors	Stimulus and expected output	Coverage of the corners the RTL must handle
The specification's numbers	Tables in the micro-architecture document	Whether anyone has to re-derive them later

Graduate · The bit-accurate model is the deliverable the role is named for, and its value rests entirely on one property: **independence**. It must encode the *specification's* behaviour, not the design's. A model written by reading the RTL is a paraphrase, and a paraphrase agrees with the original for exactly the wrong reason. The practical discipline is to write it from the standard or the algorithm description before the RTL exists, and to resist the pressure — which is constant and comes from

everyone — to ‘just make the model match’ when the two disagree near a deadline. **Every time you make the model match, you delete a check that cannot be recovered**, and nobody will remember later which checks were deleted.

Y11.2 Fixed point is most of the work

The floating-point algorithm is usually short. Turning it into something implementable is not, and that conversion is where a modelling engineer spends most of their time. Part X1 gives the arithmetic; this is the workflow.

Table 521. The six models, and what each one answers

Stage	Model	Question	Output
1	Floating-point reference	Does the algorithm work at all?	A plot against the metric
2	Range analysis	What does each node hold?	Integer bits per node, with the basis stated
3	Quantised model with parameterised word lengths	How does the metric degrade?	A sweep, as in Part X1
4	Word-length optimisation	What is the cheapest point that passes?	A vector of word lengths and the cost model used
5	Bit-exact model	Exactly what the RTL must produce, every bit	The reference the testbench compares against
6	Corner vectors	What must the RTL handle that random stimulus will not reach?	A directed vector set

These are six models, not one model in six states. Keeping them separate means the floating-point reference remains available to answer ‘is the algorithm wrong or is the quantisation wrong?’ — a question that arises constantly and is unanswerable once the models have been merged.

Worked example 92. A disagreement, diagnosed properly

Given. The RTL and the bit-exact model differ on 3 samples in 100 000, always by one LSB, always when the input is near full scale.

Method. Do not guess. Use the model hierarchy: compare the bit-exact model against the quantised model, and the quantised against floating point, on the same three inputs. The stage where the discrepancy appears names the cause.

Differing samples	3 / 100,000
Magnitude	1 LSB
Correlated with	near-full-scale input
First hypothesis	rounding at a saturation boundary
Test	compare rounding mode at the saturating node in both
Second hypothesis	the RTL saturates before rounding, the model after
Which is right?	whichever the specification says — check

How this is got wrong. By deciding which is right from which is easier to change. The order of rounding and saturation is a real specification question with a real answer, and getting it wrong produces a block that differs from every other implementation by one LSB at full scale — which will be found by a customer comparing against a competitor. **One-LSB disagreements are the ones most often waved away and most often genuine**; the discipline is that a disagreement is closed by a citation to the specification, not by a judgement about magnitude.

Y11.3 The first ninety days

Table 522. A first quarter that builds credibility

When	What	Why this and not something more impressive
Week 1–2	Build and run the existing models and the regression	You cannot contribute to a flow you cannot run. Fix the first thing that does not build — it is a real contribution and it teaches the flow
Week 2–4	Read the specification of the block you will model, and the existing model beside it	Note every place the model differs from your reading; some are your misunderstanding and some are bugs
Week 4–6	Take a small, complete piece — one function, one corner case, one test	Complete and small beats large and partial ; the goal is a closed loop through review and merge
Week 6–10	Own a module's model end to end	Including its verification against the RTL and its documentation
Week 10–13	Find something the process misses and fix it	A missing check, an unreproducible result, a manual step. This is what distinguishes you

*The pattern is deliberate: **run before reading, read before writing, small before large, and process improvement last** — because a process criticism from someone who has not yet shipped anything through the process is ignored, and the same criticism after ten weeks is acted on.*

The failure mode of a new modelling engineer is silent disagreement. You read the specification, you believe the existing model is wrong, and you say nothing because you assume the experienced people have a reason. Sometimes they do, and hearing it is how you learn. Sometimes they do not, and the bug ships. **The correct move is neither silence nor an accusation: it is a question with the clause number attached.** ‘Clause 7.3.2 says the rounding happens before saturation and I read the model as doing the reverse — what am I missing?’ costs nothing if you are wrong and is extremely valuable if you are right. Engineers who ask this way get answers; engineers who assert get arguments.

Problem. You are asked to model a block whose specification is an internal document written by an architect who has left. What do you do?

Solution. Treat the gap as the first deliverable. **Write down what the document does not determine** — every place where two readings are possible, every constant with no stated basis, every behaviour on an illegal input — and circulate that list. It is immediately useful to everyone and it is the cheapest thing you can produce in week one. **Then resolve the list by evidence in this order:** existing RTL or a previous generation, since what shipped is a fact; test vectors or captured data, which settle behaviour without settling intent; the people who integrated against it, who know what they relied on; and finally a decision recorded as a decision, with your name and the date, for the items nothing resolves. **What you must not do is make the choices silently inside the model.** A model containing twenty undocumented decisions is not a reference; it is a second design, and when it disagrees with the RTL nobody will be able to say which is right.

Problem. Your model and the RTL agree perfectly on the first day of integration. Is that good news?

Solution. It is news that needs checking before it is good. Perfect agreement on day one has three explanations and only one of them is happy. **The comparison may not be running** — a checker that is disabled, a stimulus that never reaches the block, a comparison of two constants; this is the first thing to rule out, and the way to rule it out is to **break something deliberately and confirm the comparison notices**, which is the same self-check the agent in Part Y4 performs every round. **The model may have been written from the RTL**, in which case agreement is tautological. **Or the block may genuinely be simple and correct**, which does happen. Establishing which costs half an hour and is the single highest-value half hour in an integration, because everything downstream rests on the answer.

Y12. High-Level Synthesis in a Small Design House

High-level synthesis promises to turn C++ into RTL, which for a one-person house sounds like a way to multiply headcount. It is a real tool with a real place, and the place is narrower than the marketing and wider than the scepticism. This part states where it wins, where it does not, and what this repository measured when it tried to synthesise a production C++ library with an open-source tool.

Y12.1 What HLS actually decides for you

The three transformations, and which one is the hard one

	Step	Justification
1	Scheduling: assign each operation to a clock cycle, respecting data dependences and the target period.	<i>The tool solves a constrained scheduling problem, typically with a list-scheduling heuristic or an ILP formulation for small regions.</i>
2	Allocation: decide how many of each resource exist.	<i>Two multiplies in different cycles may share one multiplier.</i>
3	Binding: assign operations to specific resource instances, and variables to registers or memories.	<i>Introduces multiplexers, which is where a naive binding loses the timing the scheduler thought it had.</i>
4	All three are driven by pragmas , not by the C++ semantics.	<i>This is the fact that determines everything else.</i> <i>The same C++ with different pragmas produces designs differing by orders of magnitude in area and throughput, which means the C++ is not the design — the C++ plus the pragmas plus the tool version is the design.</i>
5	So HLS source is not portable between tools in any practical sense.	<i>The pragma dialects differ, the library types (<code>ap_fixed</code>, <code>hls::stream</code>) differ, and the scheduling behaviour differs. Measured in this repository: a production Xilinx Vitis library did not build under an open-source HLS tool without modification.</i>

Table 523. Where HLS wins and where it does not

Situation	HLS	Why
Datapath-heavy, regular loops (filters, transforms, linear algebra)	Strong	Scheduling a loop nest is exactly what the tool is good at
Design-space exploration — ten variants of one algorithm	Very strong	Changing a pragma is minutes; rewriting RTL is days. This is the best argument for HLS
Algorithm still changing	Strong	The C++ stays readable as the algorithm moves
Control-dominated protocol logic	Weak	A state machine expressed as C++ control flow is harder to read and to verify than the FSM it becomes
Cycle-exact interface timing	Weak	The tool owns the schedule; you are negotiating with it
Area- or timing-critical blocks at the frequency limit	Weak	The last 20 % needs control the abstraction removes
Code you must hand to a customer as portable RTL	Fine	Ship the generated RTL; the customer never needs the C++

Graduate · **The strongest case for HLS in a small house is exploration, not production.** The question ‘what does this algorithm cost at four different parallelism factors?’ takes an afternoon in HLS and a fortnight in RTL, and the answer changes the architecture. Having taken the decision, writing the production block in RTL is often still the right call — and the HLS model remains useful as a golden reference, since it is C++ that is known to produce the same numbers. **Used this way, HLS is a modelling tool that happens to emit RTL**, which fits the role this book is written for.

Y12.2 What this repository measured

This repository built an open-source HLS tool from source and attempted to synthesise a production vendor C++ library with it. The findings are recorded in `survey/synth/합성결과.md` (144 lines) and are summarised here because they are the kind of result that is rarely written down.

Table 524. Three independent blockers, isolated by single-variable experiments

Blocker	Evidence	What it means for a user
Pragma dialect	Vendor pragmas are not recognised by the other tool; the code compiles and schedules differently	HLS source does not port. Budget a rewrite, not a recompile
Library types	A crash in the tool's own pass while handling a streaming class passed by value	The vendor's stream and fixed-point classes are part of the vendor's tool, not part of C++
Template instantiation cost	Heavily templated fixed-point code expands enormously before synthesis begins	Compile time can exceed the time budget before any hardware is generated

Two lessons from that exercise are worth more than the result itself. First, **the blockers were only separable by controlled experiments:** all three appeared as ‘it does not work’ until each was isolated with one variable changed at a time, and before that isolation it would have been easy — and wrong — to report a single cause. Second, an early attempt to name the cause was withdrawn because the run had timed out before reaching the pass in question, so the claim was a guess and not a measurement. **The rule that produced both corrections is the one this whole book applies: a number or a cause you have not measured is not a finding, however confident you are.**

Y12.3 Verifying HLS output

Table 525. The three levels of HLS verification

Level	What it compares	Catches	Cost
C simulation	The C++ against your test vectors	Algorithm bugs	Seconds — run constantly
C/RTL co-simulation	The generated RTL against the same C++ on the same vectors	Tool bugs and pragma misunderstandings	Minutes to hours
RTL verification	The generated RTL in its real environment	Interface and integration problems the co-simulation harness hides	As for any RTL

Undergraduate review · **Why C/RTL co-simulation is not optional.** It is tempting to reason that if the C++ is correct and the tool is correct then the RTL is correct. The second premise is the problem: HLS tools have bugs, and more commonly the *pragmas do not mean what the writer thought* — a dependence the tool was told to ignore was real, a loop it was told to pipeline could not be, an interface protocol was inferred differently from what the integrator expects. Co-simulation catches all of these for the cost of running the vectors twice. **It is also the only thing that makes the C++ usable as a golden model afterwards**, since it establishes that the two really do agree.

Worked example 93. When HLS pays for itself, as arithmetic

Given. An exploration of one algorithm at four parallelism factors and three word lengths — twelve design points. RTL: 3 days per point. HLS: 2 days to set up plus 2 hours per point. Both need the results to be trustworthy.

Method. Count the days for each route, including the HLS learning and verification overhead that is easy to omit.

Design points	12
RTL route, days	36
HLS setup, days	2
HLS per point, days	0.25
HLS route, days	5
Plus co-simulation of the chosen point, days	2
HLS total	7
Saving	29 days

How this is got wrong. By counting the saving and then shipping the HLS output without the verification days. The arithmetic above holds only because the last two rows are included; an HLS flow without co-simulation is faster and produces RTL nobody should trust. The second common error is to count the setup as a one-off when it is per-project: tool versions, library versions and pragma behaviour change, and a flow that worked last year needs re-establishing. **Budget the setup every time and the saving is still large for exploration and small or negative for a single production block** — which is exactly the recommendation in Y12.1, now with numbers behind it.

Problem. A customer asks whether your block was written in RTL or generated from HLS. Does it matter, and how do you answer?

Solution. It matters to them for three legitimate reasons and you should answer plainly. **Maintainability:** if they must modify the block, generated RTL is unpleasant to edit, so they need either the C++ and the flow, or a commitment that you will make changes. **Quality of results:** generated RTL is usually larger and slower than good hand RTL at the same function, and if they are area- or frequency-constrained that is their business. **Predictability:** regenerating with a different tool version can change the result, so they will want to know the RTL they receive is frozen and verified as delivered, not regenerated on demand. **The answer that satisfies all three is to ship frozen, verified RTL with its synthesis and timing reports, and to state the provenance.** Concealing it is the bad option: it is discoverable from the generated code's shape in about five minutes, and being caught concealing something harmless costs more than the thing concealed.

Y13. Getting Hired into a Modelling Team

This part is about the interview and the portfolio, treated as an engineering problem: what does the hiring side actually need to learn about you, and what evidence answers it fastest? The answer is unusually concrete for this kind of advice, because modelling work produces artefacts that can be shown.

Y13.1 What the interviewer is trying to find out

Table 526. The four questions behind a modelling interview

The question	How it is usually asked	What actually answers it
Can you write a correct model?	A coding exercise, often a fixed-point or protocol task	Code that handles the corners and says which it handles
Do you know why the model exists?	'How would you verify this block?'	An answer that mentions independence and what a disagreement means
Do you understand the hardware?	'Why is this architecture like this?'	Reasoning from timing, area or bandwidth — numbers, not adjectives
Will you say when something is wrong?	'Tell me about a time...', or a deliberately flawed premise in a question	Noticing the flaw and saying so, politely, is the whole test

Graduate · The fourth question is weighted more heavily than candidates expect, and it is the one this book's discipline speaks to directly. A modelling engineer's value is that they are an independent check. An engineer who defers to the design team's reading of the specification is not performing the function they were hired for, however good their code is. Interviewers probe this deliberately — a question with a wrong assumption embedded, a requirement that contradicts an earlier one — and the answer they want is neither silence nor combativeness but the thing this book keeps recommending: **name the discrepancy, cite the source, and ask.**

Y13.2 A portfolio that is evidence rather than assertion

Table 527. Portfolio items, ordered by what they prove per hour of effort

Item	Concretely	What it proves
A bit-accurate model of a standard block	RS or BCH decoder, a CIC decimator, a 64b/66b PCS	Shows you can read a specification and produce something checkable
Its test suite, with mutation results	'These 14 injected faults; the suite catches 13; here is the escape and why'	Almost nobody does this, and it is the single most convincing artefact
A word-length study	A sweep, a cost model, a chosen point with a reason	Shows you understand the actual daily work
An RTL block compared against the model	Even a small one, with the comparison automated	Shows you have closed the loop end to end
A written page on a bug you found	Symptom, hypotheses, the measurement that discriminated, the fix	Demonstrates method, which is what is being hired
A contribution to an open project	A fix, a test, a documentation improvement in a real codebase	Independent evidence that someone reviewed and merged your work

Depth beats breadth here. One block modelled properly, with its verification and its written reasoning, is worth more than six half-finished repositories — because the thing being assessed is whether you finish and whether you check, and only a finished, checked thing can show that.

Worked example 94. Where the hours go, and what each hour buys

Given. 120 available hours over three months, to be spent on a portfolio.

Method. Allocate against the four interview questions, weighting by how much evidence each hour produces.

Model of one standard block	40 h
Test suite with mutation measurement	30 h
Small RTL implementation and comparison	25 h
Written documentation and a bug write-up	15 h
Polish, README, reproducibility	10 h
Total	120 h
Questions from Y13.1 this evidences	all four

How this is got wrong. By spending all 120 hours on the model. A beautiful model with no test suite answers one of the four questions; the same model with 30 hours moved into verification answers three. The most common portfolio failure is an imbalance in exactly this direction, because modelling is the enjoyable part. **The hours spent proving the model is right are the hours that differentiate**, precisely because they are the ones most candidates skip.

Y13.3 Interview questions you should expect, and how to think about them

Table 528. Common questions and the shape of a good answer

Question	What they want to hear
‘How do you know your model is right?’	Independence of source; a third-party reference where one exists; property-based checks rather than sample values; fault injection to prove the checks bite
‘The RTL and your model disagree. What now?’	Reproduce minimally, then go to the specification. Do not edit the model to match , and say so
‘How many bits does this filter need?’	‘Against what metric?’ — then range, then sweep, then the knee. Asking for the metric first is most of the answer
‘Why not just use floating point?’	Area, power and determinism; then the accumulation and associativity arguments from Part X9
‘Our coverage is 100 %. Are we done?’	Code coverage measures execution, not checking. Offer mutation score and checker coverage as the evidence that would settle it
‘Tell me about a hard bug.’	The method, not the heroics: hypotheses, the measurement that discriminated, what you changed in the process afterwards

One answer to avoid: ‘I would add more tests.’ It is the reflex response to every verification question and it is almost never the best one, because it does not say *which* tests or how you would know they were enough. The answers that distinguish a candidate name a measurement: mutation score, checker coverage, a comparison against an independent reference, a property that holds for all inputs rather than a vector that holds for one. **Every one of those is something this book has computed somewhere**, and being able to say ‘I would measure X and here is what X would tell us’ is the difference between sounding careful and being useful.

Problem. You are asked to estimate something you do not know — how many gates a block needs, how long a task takes. What do you do?

Solution. Estimate out loud, from decomposition, and say what would sharpen it. The interviewer is not testing recall of a number; they are watching whether you can bound an unknown, which is most of engineering judgement. A good answer names the decomposition ('the multiplier array is about n^2 full adders, a full adder is about six gates, so...'), states the assumption that matters most, gives a range rather than a point, and ends with the measurement that would collapse the range. **Refusing to estimate is the worst answer** and a confident unqualified number is the second worst; the habit being looked for is the one Part X1 applies to word lengths and Part Y8 to schedules — a bound, its basis, and what would improve it.

Problem. You have no industry experience and no degree from a famous university. What actually compensates?

Solution. Artefacts, and one specific kind of them. A repository containing a model, its test suite, a mutation measurement of that suite, and a written page explaining a bug it caught is evidence that nobody can produce without having done the work — and it is checkable in ten minutes by the person deciding whether to interview you. Two things amplify it. **Contributions to a real open project carry an independent review signal:** someone with no reason to flatter you merged your change. And **a written explanation of something difficult** — a blog post deriving a result, a clear note on a standard's ambiguity — demonstrates the communication half of the job, which is half of it. What does not compensate is a list of technologies; every candidate has one, and it distinguishes nobody. **Build one thing completely and write down how you know it works.**

Y14. The Flow a One-Person House Can Actually Run

Commercial EDA licences cost more per year than a one-person house earns. That fact shapes what such a house can sell, and the honest answer is that a great deal is possible with free tools and one thing is not. This part sets out which is which, and measures what is actually installed in the environment this book was built in.

Y14.1 What is installed here, measured

Table 529. Tools present in the container this book was rendered in, checked at build time

Tool	Role	Present	Version reported
iverilog	Event-driven Verilog/SystemVerilog simulation	yes	Icarus Verilog version 12.0 (stable) ()
verilator	Cycle-accurate compiled simulation; fast regressions	yes	Verilator 5.020 2024-01-01 rev (Debian 5.020-1)
yosys	RTL synthesis to a gate netlist	yes	Yosys 0.33 (git sha1 2584903a060)
gtkwave	Waveform viewing	not installed	—
python3	Modelling, scripting, this book	yes	Python 3.11.15
gcc	Reference models in C/C++	yes	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0

Probed by running each tool. *The verification agent of Part Y4 uses the first of these and nothing else, which is the point: a complete model-versus-RTL regression loop needs one free simulator.*

Y14.2 What the free flow can and cannot do

Table 530. Capability, honestly assessed

Activity	Free flow	Note
Modelling and algorithm work	Complete	Python, NumPy, C++ — nothing is missing
RTL simulation	Complete for most purposes	Verilator is fast; Icarus handles constructs Verilator does not
UVM	Partial	UVM needs constrained randomisation and full class support; free simulators have improved but are not equivalent. cocotb is a viable different answer
Formal property checking	Usable	SymbiYosys and its solvers; capacity below commercial tools
Lint and CDC	Partial	Verilator lint is good; dedicated CDC sign-off tools have no free equivalent
Synthesis to a gate netlist	Yes	Yosys, with open cell libraries
Place and route	Yes for open PDKs	OpenROAD and the open PDK flows
Sign-off STA in a commercial process	No	Needs the foundry's timing libraries, which come with an NDA and a commercial tool
Tape-out to a commercial foundry	No	This is the line a free flow does not cross

Graduate · **The line falls exactly where a soft-IP business wants it.** Everything on the seller's side of a soft-IP transaction — modelling, RTL, verification, and enough synthesis to quote area and frequency credibly — is reachable with free tools. Everything on the buyer's side — sign-off in their process, with their libraries, in their flow — is theirs to run and they already have the licences. **A design house selling RTL therefore does not need a tape-out flow**, and recognising that is the difference between a viable plan and one that requires six figures of tooling before the first sale. The caveat to state to customers is equally clear: your frequency and area numbers come from an open flow and an open library, and they will re-run them; give them the scripts so they can.

Worked example 95. What credible area and frequency numbers cost to produce

Given. You want to quote '150 kGE at 500 MHz' for a block, with evidence, using only free tools.

Method. Enumerate what is needed and what each step actually establishes.

Synthesis with Yosys to an open library	area in gate equivalents
Static timing with OpenSTA on that netlist	a frequency, in that library
What this does <i>not</i> establish	performance in the customer's process
What makes it credible anyway	the <i>ratio</i> transfers; publish the library and the scripts
Additional cost	0 USD
What a customer will still do	re-run it in their flow
What they are really buying	the assurance that the number is not invented

How this is got wrong. By quoting a number from an open flow as though it were a sign-off number. It is not, and a customer who discovers the elision stops trusting the rest of the datasheet. The correct presentation names the library, the tool and the version, states that it is a pre-layout estimate in an open process, and offers the scripts. **That presentation is more persuasive than an unattributed larger claim**, for the same reason the known-issue list of Part Y5 is: it is checkable.

Y14.3 A regression setup that survives one person

Table 531. What to automate first, in order of return

Automate	Why it is first	Effort
One command that runs everything	If the regression is three commands, it will be run less often, and the one that is skipped is the one that would have caught it	Hours
Deterministic seeds recorded with results	A failure you cannot reproduce is a failure you cannot fix	Hours
A self-check that the checkers bite	Otherwise a green regression means nothing — the theme of this entire book	A day
Results to a ledger, not to a terminal	You will be asked what changed between last week and now	A day
Overnight running with the correct detachment	Part Y4's setsid/nohup/disown pattern — the obvious way silently does nothing	Hours
Mutation testing of the suite	Turns 'we have tests' into a number you can quote	Days — and it is what a customer remembers

The failure that ends solo projects is not a hard bug; it is losing the ability to reproduce. Six months in, with several branches, a half-changed model and results in a terminal scrollbar, the question 'did this ever pass?' becomes unanswerable, and from that point every decision is a guess. The defence is cheap and must be built before it is needed: **every run records**

its commit, its seeds, its tool versions and its results to a file that is committed. This book's own repository does exactly this, and the rule that produced it is worth repeating: a check that leaves no record is a check nobody can audit, including you.

Problem. Which free simulator should a one-person house standardise on?

Solution. Both, for different jobs, and the reasoning generalises. **Verilator** for the bulk regression: it compiles to C++ and runs one to two orders of magnitude faster than event-driven simulation, which is what makes an overnight run worth having; its restrictions (two-state by default, synthesisable subset, no timing constructs) are mostly acceptable for a testbench driven from C++ or cocotb. **Icarus** for the cases Verilator refuses: four-state X-propagation, gate-level simulation with delays, and legacy constructs. **The important discipline is to keep the RTL compiling under both**, because each catches things the other permits — Verilator's lint is stricter than most commercial tools, and Icarus's four-state semantics catch reset and initialisation bugs that a two-state run hides. Two simulators disagreeing about your design is information, and it is free.

Y15. A Complete Worked Project: RS-KP4

This part takes one block from a specification through to a delivery package, with the numbers computed rather than estimated. It uses the code that is already in this repository — the field arithmetic and the decoder of Part X5 — and it states, at each step, what exists and what does not, so the reader can see the difference between a worked example and a finished product.

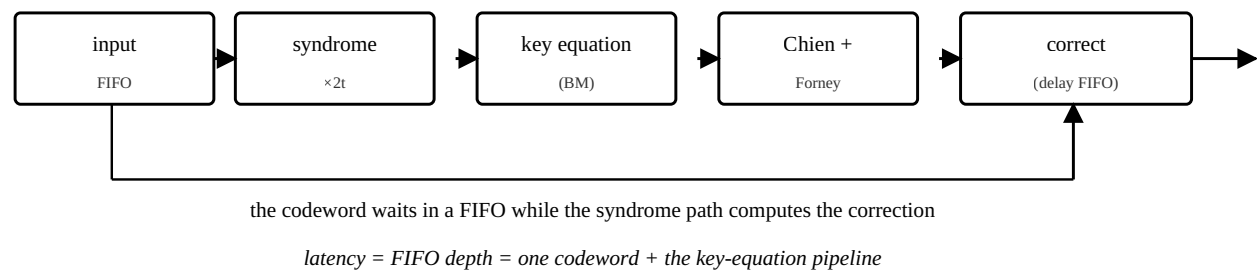


Figure 69. The decoder architecture. The delay FIFO is the block’s dominant memory and the reason latency is quoted in code-words.

Y15.1 The parameters, and which are still unconfirmed

Table 532. The parameter set, with the confidence of each entry

Parameter	Value used	Confidence	Where it is fixed
Field	$GF(2^{10})$	confirmed by the code length	—
Primitive polynomial	0x409 assumed	NOT confirmed	Fixed by IEEE 802.3 Clause 91, which this repository has not read
First consecutive root fcr	0 assumed	NOT confirmed	Same clause
Symbol bit order	MSB first assumed	NOT confirmed	Same clause
n, k	544, 514	confirmed — the code’s name	—
t	15	derived: $(n-k)/2$	—

Three of six are assumptions. The model in this repository takes all three as arguments with the assumption marked, and its tests check properties that hold whatever the values are — that up to t errors are always corrected and that beyond t the decoder does not silently return the input. When the clause is obtained, only the defaults change.

This is what it looks like to build honestly on an unread specification. The alternative — pick a polynomial, hard-code it, and proceed — produces a block that works perfectly against itself and fails at the first interoperability test, with the cause buried in a constant nobody questions. Parameterising costs nothing and the marked confidence costs one column in a table. **What it buys is that the unread clause remains visible** instead of dissolving into an assumption, which is precisely the failure Part Y6 is written to prevent.

Y15.2 Sizing the block from the standard's rate

Table 533. Parallelism required at 53.125 GBd, one symbol per 10 bits

De-serialisation	Digital clock (MHz)	Symbols per clock	Symbols processed in parallel	Datapath width (bits)	Clocks per code-word
4	830	64.01	65	650	8.369
8	415	128	129	1,290	4.217
16	208	255.4	256	2,560	2.125
32	104	510.8	511	5,110	1.065

The third column is the block's fundamental parallelism: at 830 MHz the decoder must absorb 64 symbols every cycle. **Every stage of the architecture must be replicated or restructured to that width, and that replication, not the algorithm, is where the area goes.**

Worked example 96. Where the area actually goes

Given. RS(544,514), $t = 15$, 64 symbols per cycle at 830 MHz.

Method. Count the three structures separately: the syndrome computation (parallel, wide), the key equation (serial, narrow) and the Chien search (parallel, wide). The asymmetry is the architectural fact.

Syndromes to compute	30
GF multipliers for syndromes, unparallelised	30
At 64 symbols per cycle	1,920
Berlekamp–Massey multipliers (serial, $\sim 2t$)	30
BM cycles per codeword	30
Chien: evaluate σ at 64 positions per cycle	960
Ratio of wide structures to the key equation	96×
Delay FIFO	5,440 bits per codeword in flight

How this is got wrong. By optimising Berlekamp–Massey. It is the algorithmically interesting stage, it is what the literature discusses, and here it is about one per cent of the multipliers. **The syndrome and Chien stages dominate because they are replicated 64 ways** and BM is not. The architectural leverage is therefore in sharing and folding those wide stages — the standard techniques being common-subexpression sharing across the syndrome constants and a folded Chien search — and in nothing else. **Compute the replication factor before deciding what to optimise**, in any block.

Y15.3 What exists in this repository and what does not

Table 534. State of the project, stated plainly

Item	State	Where
Field arithmetic with verified axioms	Exists	gf.py, 10 tests
Encoder and bounded-distance decoder	Exists	rs.py, 13 tests including mutation guards
Measurement that refuses a zero-error pass	Exists	rs. 측정()
Prior-art survey with confidence markers	Exists	paper/선행조사/PCS_FEC계층_IP.md
A syndrome-step RTL block verified against the model	Exists	edu/agent/blocks/rs_syndrome/
Full parallel decoder RTL	Does not exist	—
Berlekamp–Massey RTL	Does not exist	—

Item	State	Where
The three unconfirmed constants	Still unconfirmed	Needs the IEEE document
Concatenated inner code (802.3dj)	Not started	—
A named first customer	None	—

Graduate · Listing what does not exist is the point of this section. A worked example that presents a partial project as a finished one teaches the wrong lesson twice: it misleads about the effort, and it models the behaviour — quiet omission of the incomplete parts — that this book spends two chapters warning against. The honest summary is that this repository contains a *verified reference model and one verified RTL primitive* for a block whose full implementation is most of Part Y8's 39-week plan, and that three of its parameters await a document that must be purchased.

Problem. Given that state, what are the next three actions, in order?

Solution. First, obtain the clause. Three unconfirmed constants block every downstream claim of interoperability, and no amount of RTL removes that block; until they are confirmed the project can produce a correct decoder for a code that may not be the standard's. This is a purchase and a fortnight of reading, and it is the highest-value action available. **Second, build the syndrome stage at full parallelism and verify it against the model,** because the sizing example above says that stage dominates the area, so the area and frequency claims — the numbers a customer asks for first — are determined by it. One wide stage, synthesised and timed, converts an estimate into a measurement. **Third, find a customer conversation before writing the key equation,** because everything after the syndrome stage is a large effort whose requirements (latency budget, code rate set, concatenation) the customer determines, and guessing them is how the 39 weeks turn into 60. **Note that the technically interesting work is third on this list,** which is usually where it belongs.

Volume II — Production Source Code

This volume reproduces **156,976 lines** of source from 44 blocks that are shipping in silicon or sold as commercial IP. Nothing is paraphrased: every line is read from disk and printed with its real line number, so a citation of the form `file:line` anywhere in this book can be followed directly.

Each block is preceded by four notes — **What** it is, **Why** it exists (what fails without it), the **EECS concepts** it rests on, and a **practice note** for engineers who must work with it. An **API dictionary** then lists every module, class, function, parameter, port, macro and typedef that the block declares, with the arguments each one takes, because names alone are ambiguous.

The API dictionary is generated by pattern extraction, not by a full language front end. It is therefore complete enough to navigate by and may miss unusual declarations; the verbatim source that follows each table is authoritative.

MIPI CSI-2 수신기 -- 패킷 디코더 · 레인 정렬 · RAW 언팩

What this is. A working MIPI CSI-2 receiver in Verilog: byte and lane aligners, the packet decoder, RAW8/10/12 depackers, an output reformatter, and a debayer filter.

Why it exists. Because it is the concrete form of every block named in Chapter P1. Reading `mipi_rx_lane_aligner.v` alongside the description of lane deskew, and `mipi_rx_raw12_depacker.v` alongside the explanation of RAW packing, turns an abstract block diagram into something that can be modified.

Undergraduate review · EECS concepts. Packet framing and synchronisation; ECC and CRC; bit-packing and unpacking; multi-lane deskew; line buffering for neighbourhood operations.

Graduate · Practice note. Start with `mipi_rx_raw_depacker.v`. The bit-slicing there is exactly the part that Chapter P1 identifies as where verification effort concentrates, and seeing how few lines it takes — and how many ways it can be wrong — makes the point better than any description.

15 files • 2,841 lines

API — debayer_filter.v

Table 535. `circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/debayer_filter.v` — module / interface / package (1개)

이름	종류
<code>debayer_filter</code>	module

Table 536. `circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/debayer_filter.v` — localparam (3개)

상수	형/폭	값
<code>PIXEL_WIDTH</code>		16
<code>INPUT_WIDTH</code>		64
<code>OUTPUT_WIDTH</code>		192

Table 537. `circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/debayer_filter.v` — 포트 (7개)

포트	방향	형/폭
<code>clk_i</code>	input	
<code>reset_i</code>	input	
<code>line_valid_i</code>	input	
<code>data_valid_i</code>	input	
<code>1</code>	input	<code>[(INPUT_WIDTH -</code>
<code>output_valid_o</code>	output	reg
<code>1</code>	output	reg <code>[(OUTPUT_WIDTH-</code>

API — mipi_csi_bridge.v**Table 538. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/mipi_csi_bridge.v — module / interface / package (17개)**

이름	종류
mipi_bridge	module

Table 539. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/mipi_csi_bridge.v — parameter (1개)

파라미터	형/폭	기본값
MIPI_LANES		4

Table 540. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/mipi_csi_bridge.v — 포트 (11개)

포트	방향	형/폭
frame_sync_in	input	
line_sync_in	input	
reset_in	input	
mipi_clk_p_in	input	
mipi_clk_n_in	input	
mipi_data_p_in	input	[MIPI_LANES-1:0]
mipi_data_n_in	input	[MIPI_LANES-1:0]
pclk_o	output	
data_o	output	[31:0]
fsync_o	output	
lsync_o	output	

API — mipi_csi_packet_decoder.v**Table 541. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/mipi_csi_packet_decoder.v — module / interface / package (17개)**

이름	종류
mipi_csi_packet_decoder	module

Table 542. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/mipi_csi_packet_decoder.v — localparam (5개)

상수	형/폭	값
SYNC_BYTE	[7:0]	8'hB8
LANES	[3:0]	3'h4
MIPI_CSI_PACKET_10bRAW	[7:0]	8'h2B
MIPI_CSI_PACKET_12bRAW	[7:0]	8'h2C
MIPI_CSI_PACKET_14bRAW	[7:0]	8'h2D

Table 543. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/mipi_csi_packet_decoder.v — 포트 (7개)

포트	방향	형/폭
clk_i	input	
data_valid_i	input	
data_i	input	[31:0]
output_valid_o	output	reg
data_o	output	reg [31:0]
packet_length_o	output	reg [15:0]
packet_type_o	output	reg [2:0]

API — mipi_rx_byte_aligner.v

Table 544. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/mipi_rx_byte_aligner.v — module / interface / package (1개)

이름	종류
mipi_rx_byte_aligner	module

Table 545. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/mipi_rx_byte_aligner.v — localparam (1개)

상수	형/폭	값
SYNC_BYTE	[7:0]	8'hB8

Table 546. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/mipi_rx_byte_aligner.v — 포트 (5개)

포트	방향	형/폭
clk_i	input	
reset_i	input	
byte_i	input	[7:0]
byte_o	output	reg [7:0]
byte_valid_o	output	reg

API — mipi_rx_lane_aligner.v

Table 547. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/mipi_rx_lane_aligner.v — module / interface / package (1개)

이름	종류
mipi_rx_lane_aligner	module

Table 548. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/mipi_rx_lane_aligner.v — localparam (2개)

상수	형/폭	값
ALIGN_DEPTH	[3:0]	4'h7
LANES	[2:0]	3'h4

Table 549. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/mipi_rx_lane_aligner.v — 포트 (6개)

포트	방향	형/폭
reset_i	input	
clk_i	input	
1	input	[(LANES-
LANES	input	[(8*
LANES	output	reg [(8*
lane_valid_o	output	reg

API — mipi_rx_raw12_depacker.v

Table 550. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/mipi_rx_raw12_depacker.v — module / interface / package (1개)

이름	종류
mipi_rx_raw10_depacker	module

Table 551. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/mipi_rx_raw12_depacker.v — localparam (1개)

상수	형/폭	값
TRANSFERS_PERPACK	[2:0]	3'h2

Table 552. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/mipi_rx_raw12_depacker.v — 포트 (7개)

포트	방향	형/폭
delayed	output	is one clock cycle
cycle	output	come in group of 2x48bit chunk on each clock
clk_i	input	
data_valid_i	input	
data_i	input	[31:0]
output_valid_o	output	reg
output_o	output	reg [47:0]

API — mipi_rx_raw_depacker.v

Table 553. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/mipi_rx_raw_depacker.v — module / interface / package (1개)

이름	종류
mipi_rx_raw_depacker	module

Table 554. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/mipi_rx_raw_depacker.v — localparam (4개)

상수	형/폭	값
TRANSFERS_PERCHUNK	[2:0]	3'h5
MIPI_CSI_PACKET_10bRAW	[7:0]	8'h2B
MIPI_CSI_PACKET_12bRAW	[7:0]	8'h2C
MIPI_CSI_PACKET_14bRAW	[7:0]	8'h2D

Table 555. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/mipi_rx_raw_depacker.v — 포트 (8개)

포트	방향	형/폭
delayed	output	is one clock cycle
cycle	output	come in group of 5x40bit chunk on each clock
clk_i	input	
data_valid_i	input	
data_i	input	[31:0]
packet_type_i	input	[2:0]
output_valid_o	output	reg
output_o	output	reg [63:0]

API — output_reformatter.v

Table 556. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/output_reformatter.v — module / interface / package (1개)

이름	종류
output_reformatter	module

Table 557. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/output_reformatter.v — 포트 (8개)

포트	방향	형/폭
line_sync_i	input	
frame_sync_i	input	
clk_i	input	
data_in_valid_i	input	
data_i	input	[63:0]
output_valid_o	output	reg
output_o	output	[31:0]

포트	방향	형/폭
output_clk_i	input	

API — rgb_to_yuv.v**Table 558. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/rgb_to_yuv.v — module / interface / package (1개)**

이름	종류
rgb_to_yuv	module

Table 559. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/rgb_to_yuv.v — localparam (2개)

상수	형/폭	값
PIXEL_DEPTH		5'd16
PIXEL_PER_CLK		4'd4

Table 560. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/rgb_to_yuv.v — 포트 (6개)

포트	방향	형/폭
clk_i	input	
reset_i	input	
3	input	(((PIXEL_DEPTH * PIXEL_PER_CLK *
rgb_valid_i	input	
8	output	reg (((PIXEL_PER_CLK * 2 *
yuv_valid_o	output	reg

API — tb_debayer_filter.v**Table 561. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/tb_debayer_filter.v — module / interface / package (1개)**

이름	종류
tb_debayer_filter	module

Table 562. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/tb_debayer_filter.v — 포트 (1개)

포트	방향	형/폭
bytes	input	[31:0]

API — tb_mipi_bridge.v**Table 563. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/tb_mipi_bridge.v — module / interface / package (1개)**

이름	종류
tb_mipi_bridge	module

Table 564. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/tb_mipi_bridge.v — 포트 (1개)

포트	방향	형/폭
bytes	input	[31:0]

API — tb_mipi_csi_packet_decoder.v

Table 565. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/tb_mipi_csi_packet_decoder.v — module / interface / package (1개)

이름	종류
tb_mipi_csi_packet_decoder	module

Table 566. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/tb_mipi_csi_packet_decoder.v — 포트 (1개)

포트	방향	형/폭
bytes	input	[31:0]

API — tb_mipi_rx_raw_depacker.v

Table 567. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/tb_mipi_rx_raw_depacker.v — module / interface / package (1개)

이름	종류
tb_mipi_rx_raw_depacker	module

Table 568. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/tb_mipi_rx_raw_depacker.v — 포트 (1개)

포트	방향	형/폭
bytes	input	[31:0]

API — tb_rx_byte_aligner.v

Table 569. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/tb_rx_byte_aligner.v — module / interface / package (1개)

이름	종류
tb_rx_byte_aligner	module

Table 570. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/tb_rx_byte_aligner.v — 포트 (1개)

포트	방향	형/폭
byte	input	[7:0]

API — tb_rx_lane_aligner.v

Table 571. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/tb_rx_lane_aligner.v — module / interface / package (1개)

이름	종류
tb_rx_lane_aligner	module

Table 572. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/FPGA_Source/source/mipi_csi_bridge/tb_rx_lane_aligner.v — 포트 (1개)

포트	방향	형/폭
bytes	input	[31:0]

```

1. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/
FPGA_Source/source/mipi_csi_bridge/debayer_filter.v (458
lines)
1 `timescale 1ns/1ns
2
3 /*
4 MIPI CSI RX to Parallel Bridge (c) by Gaurav Singh www.Cir-
cuitValley.com
5
6 MIPI CSI RX to Parallel Bridge is licensed under a
7 Creative Commons Attribution 3.0 Unported License.
8
9 You should have received a copy of the license along with this
10 work. If not, see <http://creativecommons.org/licenses/by/
3.0/>.
11 */
12
13 /*
14 Takes 4x10bit pixel from RAW10 depacker module @mipi byte
15 clock output 4x24bit RGB for each pixel , output is delayed by 2
16 lines
17 Implement Basic Debayer filter, As debayer need pixel infrom
18 neighboring pixel which may be on next or previous display line,
19 so input data is written onto RAM, only 4 lines are stored in
20 RAM at one time and only three of the readable at any give time ,
21 RAM to which data is written to can not be read.
22 as we have enough info in RAM, 4 10bit pixel will be converted
23 to 4x24bit RGB output
24 First line is expected to RGRG , second line GBGB
25 */
26
27 module debayer_filter(clk_i,
28                      reset_i,
29                      line_valid_i,
30                      data_i,
31                      data_valid_i,
32                      output_valid_o,
33                      output_o
34                      );
35
36 localparam PIXEL_WIDTH = 16; //bits per color
37 localparam INPUT_WIDTH = 64; //4 x 16bit pixels from raw de-
38 packer module
39 localparam OUTPUT_WIDTH = 192; //4 x 48bit RGB output
40
41 input clk_i;
42 input reset_i;
43 input line_valid_i;
44 input data_valid_i;
45 input [(INPUT_WIDTH-1):0] data_i;
46 output reg output_valid_o;
47 output reg [(OUTPUT_WIDTH-1):0] output_o;
48
49
50 reg [1:0]line_counter; //counts lines of the frame , needed
51 determine if have enough data ins line rams to start outputting RGB
52 data
53 reg data_valid_reg;
54
55 reg [(PIXEL_WIDTH - 1):0]R1[3:0];
56 reg [(PIXEL_WIDTH - 1):0]R2[3:0];
57 reg [(PIXEL_WIDTH - 1):0]R3[3:0];
58 reg [(PIXEL_WIDTH - 1):0]R4[3:0];
59
60 reg [(PIXEL_WIDTH - 1):0]B1[3:0];
61 reg [(PIXEL_WIDTH - 1):0]B2[3:0];
62 reg [(PIXEL_WIDTH - 1):0]B3[3:0];
63 reg [(PIXEL_WIDTH - 1):0]B4[3:0];
64
65 reg [(PIXEL_WIDTH - 1):0]G1[3:0];
66 reg [(PIXEL_WIDTH - 1):0]G2[3:0];
67 reg [(PIXEL_WIDTH - 1):0]G3[3:0];
68 reg [(PIXEL_WIDTH - 1):0]G4[3:0];
69
70 reg [(PIXEL_WIDTH - 1):0]R1_even[3:0];
71 reg [(PIXEL_WIDTH - 1):0]R2_even[3:0];
72 reg [(PIXEL_WIDTH - 1):0]R3_even[3:0];
73 reg [(PIXEL_WIDTH - 1):0]R4_even[3:0];
74
75 reg [(PIXEL_WIDTH - 1):0]B1_odd[3:0];
76 reg [(PIXEL_WIDTH - 1):0]B2_odd[3:0];
77 reg [(PIXEL_WIDTH - 1):0]B3_odd[3:0];
78 reg [(PIXEL_WIDTH - 1):0]B4_odd[3:0];
79
80 reg [(PIXEL_WIDTH - 1):0]G1_odd[3:0];
81 reg [(PIXEL_WIDTH - 1):0]G2_odd[3:0];
82 reg [(PIXEL_WIDTH - 1):0]G3_odd[3:0];
83 reg [(PIXEL_WIDTH - 1):0]G4_odd[3:0];
84
85 reg [1:0]read_ram_index_even; //which line RAM is being fo-
86 cused to read for even lines, (not which address is being read from
87 line RAM)
88 reg [1:0]read_ram_index_odd; //which line RAM is being fo-
89 cused to read for odd lines, (not which address is being read from
90 line RAM)
91 reg [1:0]read_ram_index_even_plus_1;
92 reg [1:0]read_ram_index_odd_plus_1;
93 reg [1:0]read_ram_index_even_minus_1;
94 reg [1:0]read_ram_index_odd_minus_1;
95
96 reg [3:0]write_ram_select; //which line RAM is begin writ-
97 ten
98 reg [10:0]line_address; //which address is be-
99 ing read and written
100 reg [(INPUT_WIDTH-1):0]last_ram_outputs[3:0]; //one clock
101 cycle delayed output of line RAMs
102 reg [(INPUT_WIDTH-1):0]last_ram_outputs_stage2[3:0]; //two
103 clock cycle delayed output of RAMs
104
105 reg [1:0]not_used2b;
106
107 wire [(INPUT_WIDTH-1):0]RAM_out[3:0];
108 reg [(INPUT_WIDTH-1):0]RAM_out_reg[3:0];
109 wire ram_write_enable;
110 wire ram_clk;
111
112 assign ram_clk = !clk_i;
113 assign ram_write_enable = data_valid_i;
114
115 reg [2:0]i;
116 //line rams, total 4,
117 //The way debayer is implemented in this code. Depending on
118 pixel, we need minimum 2 lines and maximum 3 lines in the ram, To
119 be able to have access to neighboring pixels from previous and next
120 line
121 //There are many ways to implemented debayer, this code imple-
122 ment simplest possible bare minimum.
123 // IMX219 Camera only output BGGR as defined by the IMX219
124 Driver in linux repo MEDIA_BUS_FMT_SBGGR10_1X10, Camera datasheet
125 incorrrectly defines output as RGRG and GBGB. Data sheet is incor-
126 rect in this case.
127 // Bayer filter type does not affect test pattern.
128
129 line_ram_dp line0(.wr_clk_i(ram_clk), //data and ad-
130 address latch in on rising edge

```

```

66 reg [(PIXEL_WIDTH - 1):0]R3_even[3:0];
67 reg [(PIXEL_WIDTH - 1):0]R4_even[3:0];
68
69 reg [(PIXEL_WIDTH - 1):0]B1_even[3:0];
70 reg [(PIXEL_WIDTH - 1):0]B2_even[3:0];
71 reg [(PIXEL_WIDTH - 1):0]B3_even[3:0];
72 reg [(PIXEL_WIDTH - 1):0]B4_even[3:0];
73
74 reg [(PIXEL_WIDTH - 1):0]G1_even[3:0];
75 reg [(PIXEL_WIDTH - 1):0]G2_even[3:0];
76 reg [(PIXEL_WIDTH - 1):0]G3_even[3:0];
77 reg [(PIXEL_WIDTH - 1):0]G4_even[3:0];
78
79
80
81
82 reg [(PIXEL_WIDTH - 1):0]R1_odd[3:0];
83 reg [(PIXEL_WIDTH - 1):0]R2_odd[3:0];
84 reg [(PIXEL_WIDTH - 1):0]R3_odd[3:0];
85 reg [(PIXEL_WIDTH - 1):0]R4_odd[3:0];
86
87 reg [(PIXEL_WIDTH - 1):0]B1_odd[3:0];
88 reg [(PIXEL_WIDTH - 1):0]B2_odd[3:0];
89 reg [(PIXEL_WIDTH - 1):0]B3_odd[3:0];
90 reg [(PIXEL_WIDTH - 1):0]B4_odd[3:0];
91
92 reg [(PIXEL_WIDTH - 1):0]G1_odd[3:0];
93 reg [(PIXEL_WIDTH - 1):0]G2_odd[3:0];
94 reg [(PIXEL_WIDTH - 1):0]G3_odd[3:0];
95 reg [(PIXEL_WIDTH - 1):0]G4_odd[3:0];
96
97
98
99
100 reg [1:0]read_ram_index_even; //which line RAM is being fo-
101 cused to read for even lines, (not which address is being read from
102 line RAM)
103 reg [1:0]read_ram_index_odd; //which line RAM is being fo-
104 cused to read for odd lines, (not which address is being read from
105 line RAM)
106 reg [1:0]read_ram_index_even_plus_1;
107 reg [1:0]read_ram_index_odd_plus_1;
108 reg [1:0]read_ram_index_even_minus_1;
109 reg [1:0]read_ram_index_odd_minus_1;
110
111 reg [3:0]write_ram_select; //which line RAM is begin writ-
112 ten
113 reg [10:0]line_address; //which address is be-
114 ing read and written
115 reg [(INPUT_WIDTH-1):0]last_ram_outputs[3:0]; //one clock
116 cycle delayed output of line RAMs
117 reg [(INPUT_WIDTH-1):0]last_ram_outputs_stage2[3:0]; //two
118 clock cycle delayed output of RAMs
119
120 reg [1:0]not_used2b;
121
122 wire [(INPUT_WIDTH-1):0]RAM_out[3:0];
123 reg [(INPUT_WIDTH-1):0]RAM_out_reg[3:0];
124 wire ram_write_enable;
125 wire ram_clk;
126
127 assign ram_clk = !clk_i;
128 assign ram_write_enable = data_valid_i;
129
130 reg [2:0]i;
131 //line rams, total 4,
132 //The way debayer is implemented in this code. Depending on
133 pixel, we need minimum 2 lines and maximum 3 lines in the ram, To
134 be able to have access to neighboring pixels from previous and next
135 line
136 //There are many ways to implemented debayer, this code imple-
137 ment simplest possible bare minimum.
138 // IMX219 Camera only output BGGR as defined by the IMX219
139 Driver in linux repo MEDIA_BUS_FMT_SBGGR10_1X10, Camera datasheet
140 incorrrectly defines output as RGRG and GBGB. Data sheet is incor-
141 rect in this case.
142 // Bayer filter type does not affect test pattern.
143
144 line_ram_dp line0(.wr_clk_i(ram_clk), //data and ad-
145 address latch in on rising edge

```

```

132         .rd_clk_i(ram_clk),
133         .rst_i(reset_i),
134         .wr_clk_en_i(ram_write_enable
),
//TODO : Fix This
135         .rd_en_i(1'b1),
136         .rd_clk_en_i(1'b1),
137         .wr_en_i(write_ram_select[0] )
,
138         .wr_data_i(data_i),
139         .wr_addr_i(line_address),
140         .rd_addr_i(line_address),
141         .rd_data_o(RAM_out[0])) ;
142
143 line_ram_dp line1(.wr_clk_i(ram_clk),
144         .rd_clk_i(ram_clk),
145         .rst_i(reset_i),
146         .wr_clk_en_i(ram_write_enable
),
147         .rd_en_i(1'b1),
148         .rd_clk_en_i(1'b1),
149         .wr_en_i(write_ram_select[1])
,
150         .wr_data_i(data_i),
151         .wr_addr_i(line_address),
152         .rd_addr_i(line_address),
153         .rd_data_o(RAM_out[1])) ;
154
155 line_ram_dp line2(.wr_clk_i(ram_clk),
156         .rd_clk_i(ram_clk),
157         .rst_i(reset_i),
158         .wr_clk_en_i(ram_write_enable
),
159         .rd_en_i(1'b1),
160         .rd_clk_en_i(1'b1),
161         .wr_en_i(write_ram_select[2])
,
162         .wr_data_i(data_i),
163         .wr_addr_i(line_address),
164         .rd_addr_i(line_address),
165         .rd_data_o(RAM_out[2])) ;
166
167 line_ram_dp line3(.wr_clk_i(ram_clk),
168         .rd_clk_i(ram_clk),
169         .rst_i(reset_i),
170         .wr_clk_en_i(ram_write_enable
),
171         .rd_en_i(1'b1),
172         .rd_clk_en_i(1'b1),
173         .wr_en_i(write_ram_select[3])
,
174         .wr_data_i(data_i),
175         .wr_addr_i(line_address),
176         .rd_addr_i(line_address),
177         .rd_data_o(RAM_out[3])) ;
178
179
180
181 always @(posedge clk_i) //address should increment at falling
edge of ram_clk. It is inverted from clk_i
182 begin
183     if (!line_valid_i )
184     begin
185         line_address <= 10'h0;
186     end
187     else
188     begin
189         if (data_valid_i)
190         begin
191             line_address <= line_address + 1'b1;
192         end
193     end
194 end
195
196
197 always @(posedge reset_i or posedge line_valid_i)
198 begin
199     if (reset_i)
200     begin
201         write_ram_select <= 4'b1000;
202         line_counter <= 2'b0;
203         read_ram_index_odd <= 2'b01;
204         read_ram_index_even <= 2'b01;
205         read_ram_index_odd_plus_1 <= 2'd2;
206         read_ram_index_odd_minus_1 <= 2'd0;
207         read_ram_index_even_plus_1 <= 2'd2;
208         read_ram_index_even_minus_1 <= 2'd0;
209     end
210     else
211     begin
212         write_ram_select <= {write_ram_select[2:0],
write_ram_select[3]};
213
214
215         read_ram_index_odd <= read_ram_index_odd +
1'b1;
216         read_ram_index_even <= read_ram_index_even +
1'b1;
217         read_ram_index_odd_plus_1 <= read_ram_in-
dex_odd_plus_1 + 1'b1;
218         read_ram_index_odd_minus_1 <= read_ram_in-

```

```

dex_odd_minus_1 + 1'b1;
219         read_ram_index_even_plus_1 <= read_ram_in-
dex_even_plus_1 + 1'b1;
220         read_ram_index_even_minus_1 <= read_ram_in-
dex_even_minus_1 + 1'b1;
221
222         if (line_counter < 2'd3)
223         begin
224             line_counter <= line_counter + 1'b1;
225         end
226     end
227 end
228
229 always @(posedge ram_clk)
230 begin
231
232         RAM_out_reg[0] <= RAM_out[0];
233         RAM_out_reg[1] <= RAM_out[1];
234         RAM_out_reg[2] <= RAM_out[2];
235         RAM_out_reg[3] <= RAM_out[3];
236
237         last_ram_outputs[0] <= RAM_out_reg[0];
238         last_ram_outputs[1] <= RAM_out_reg[1];
239         last_ram_outputs[2] <= RAM_out_reg[2];
240         last_ram_outputs[3] <= RAM_out_reg[3];
241
242         last_ram_outputs_stage2[0] <= last_ram_out-
puts[0];
243         last_ram_outputs_stage2[1] <= last_ram_out-
puts[1];
244         last_ram_outputs_stage2[2] <= last_ram_out-
puts[2];
245         last_ram_outputs_stage2[3] <= last_ram_out-
puts[3];
246     end
247
248     always @(negedge clk_i)
249     begin
250         if (reset_i)
251         begin
252             output_valid_o <= 1'b0;
253             data_valid_reg <= 1'b0;
254         end
255         else
256         begin
257             if(line_counter > 9'd2)
258             begin
259                 data_valid_reg <= data_valid_i;
260                 output_valid_o <= data_valid_reg;
261             end
262         end
263     end
264 end
265
266
267
268
269 always @(negedge clk_i)
270 begin
271
272         B1_even[0] = last_ram_out-
puts[ read_ram_index_even_plus_1 ][63:48];
273         B2_even[0] = last_ram_out-
puts[ read_ram_index_even_plus_1 ][63:48];
274         B3_even[0] = last_ram_out-
puts[ read_ram_index_even_minus_1 ][63:48];
275         B4_even[0] = last_ram_out-
puts[ read_ram_index_even_minus_1 ][63:48];
276
277         G1_even[0] =
last_ram_outputs[ read_ram_index_even ][63:48];
278         G2_even[0] =
last_ram_outputs[ read_ram_index_even ][63:48];
279         G3_even[0] =
last_ram_outputs[ read_ram_index_even ][63:48];
280         G4_even[0] =
last_ram_outputs[ read_ram_index_even ][63:48];
281
282         R1_even[0] =
last_ram_outputs[ read_ram_index_even ][47:32];
283         R2_even[0] = last_ram_out-
puts_stage2[ read_ram_index_even ][15:0 ];
284         R3_even[0] =
last_ram_outputs[ read_ram_index_even ][47:32];
285         R4_even[0] = last_ram_out-
puts_stage2[ read_ram_index_even ][15:0 ];
286
287         B1_even[1] = last_ram_out-
puts[ read_ram_index_even_minus_1 ][63:48];
288         B2_even[1] = last_ram_out-
puts[ read_ram_index_even_plus_1 ][63:48];
289         B3_even[1] = last_ram_out-
puts[ read_ram_index_even_minus_1 ][31:16];
290         B4_even[1] = last_ram_out-
puts[ read_ram_index_even_plus_1 ][31:16];
291
292         G1_even[1] = last_ram_out-
puts[ read_ram_index_even ][63:48];
293         G2_even[1] = last_ram_out-
puts[ read_ram_index_even_minus_1 ][47:32];

```

```

294      G3_even[1] = last_ram_out-
puts[ read_ram_index_even_plus_1 ][47:32];
295      G4_even[1] = last_ram_out-
puts[ read_ram_index_even          ][31:16];
296
297      R1_even[1] = last_ram_out-
puts[ read_ram_index_even          ][47:32];
298      R2_even[1] = last_ram_out-
puts[ read_ram_index_even          ][47:32];
299      R3_even[1] = last_ram_out-
puts[ read_ram_index_even          ][47:32];
300      R4_even[1] = last_ram_out-
puts[ read_ram_index_even          ][47:32];
301
302      B1_even[2] = last_ram_out-
puts[ read_ram_index_even_minus_1 ][31:16];
303      B2_even[2] = last_ram_out-
puts[ read_ram_index_even_plus_1  ][31:16];
304      B3_even[2] = last_ram_out-
puts[ read_ram_index_even_minus_1 ][31:16];
305      B4_even[2] = last_ram_out-
puts[ read_ram_index_even_plus_1  ][31:16];
306
307      G1_even[2] = last_ram_out-
puts[ read_ram_index_even          ][31:16];
308      G2_even[2] = last_ram_out-
puts[ read_ram_index_even          ][31:16];
309      G3_even[2] = last_ram_out-
puts[ read_ram_index_even          ][31:16];
310      G4_even[2] = last_ram_out-
puts[ read_ram_index_even          ][31:16];
311
312      R1_even[2] = last_ram_out-
puts[ read_ram_index_even          ][ 15:0 ];
313      R2_even[2] = last_ram_out-
puts[ read_ram_index_even          ][47:32];
314      R3_even[2] = last_ram_out-
puts[ read_ram_index_even          ][ 15:0 ];
315      R4_even[2] = last_ram_out-
puts[ read_ram_index_even          ][47:32];
316
317      B1_even[3] =
RAM_out_reg[ read_ram_index_even_minus_1 ][63:48];
318      B2_even[3] = last_ram_out-
puts[ read_ram_index_even_minus_1 ][31:16];
319      B3_even[3] =
RAM_out_reg[ read_ram_index_even_plus_1 ][63:48];
320      B4_even[3] = last_ram_out-
puts[ read_ram_index_even_plus_1 ][31:16];
321
322
323      G1_even[3] = last_ram_out-
puts[ read_ram_index_even          ][31:16];
324      G2_even[3] = last_ram_out-
puts[ read_ram_index_even_minus_1 ][ 15:0 ];
325      G3_even[3] = last_ram_out-
puts[ read_ram_index_even_plus_1 ][ 15:0 ];
326      G4_even[3] =
RAM_out_reg[ read_ram_index_even          ][63:48];
327
328      R1_even[3] = last_ram_out-
puts[ read_ram_index_even          ][ 15:0 ];
329      R2_even[3] = last_ram_out-
puts[ read_ram_index_even          ][ 15:0 ];
330      R3_even[3] = last_ram_out-
puts[ read_ram_index_even          ][ 15:0 ];
331      R4_even[3] = last_ram_out-
puts[ read_ram_index_even          ][ 15:0 ];
332
333
334
335
336      B1_odd[0] =
last_ram_outputs[ read_ram_index_odd          ][63:48];
337      B2_odd[0] =
last_ram_outputs[ read_ram_index_odd          ][63:48];
338      B3_odd[0] =
last_ram_outputs[ read_ram_index_odd          ][63:48];
339      B4_odd[0] =
last_ram_outputs[ read_ram_index_odd          ][63:48];
340
341      G1_odd[0] =
last_ram_outputs[ read_ram_index_odd_minus_1 ][63:48];
342      G2_odd[0] =
last_ram_outputs[ read_ram_index_odd_plus_1 ][63:48];
343      G3_odd[0] =
last_ram_outputs[ read_ram_index_odd          ][47:32];
344      G4_odd[0] =
puts_stage2[ read_ram_index_odd          ][15:0 ];
345
346      R1_odd[0] = last_ram_out-
puts_stage2[ read_ram_index_odd_minus_1 ][ 15:0 ];
347      R2_odd[0] =
last_ram_outputs[ read_ram_index_odd_minus_1 ][47:32];
348      R3_odd[0] = last_ram_out-
puts_stage2[ read_ram_index_odd_plus_1 ][ 15:0 ];
349      R4_odd[0] =
last_ram_outputs[ read_ram_index_odd_plus_1 ][47:32];
350
351

```

```

352      B1_odd[1] = last_ram_out-
puts[ read_ram_index_odd          ][63:48];
353      B2_odd[1] = last_ram_out-
puts[ read_ram_index_odd          ][31:16];
354      B3_odd[1] = last_ram_out-
puts[ read_ram_index_odd          ][63:48];
355      B4_odd[1] = last_ram_out-
puts[ read_ram_index_odd          ][31:16];
356
357      G1_odd[1] = last_ram_out-
puts[ read_ram_index_odd          ][47:32];
358      G2_odd[1] = last_ram_out-
puts[ read_ram_index_odd          ][47:32];
359      G3_odd[1] = last_ram_out-
puts[ read_ram_index_odd          ][47:32];
360      G4_odd[1] = last_ram_out-
puts[ read_ram_index_odd          ][47:32];
361
362      R1_odd[1] = last_ram_out-
puts[ read_ram_index_odd_minus_1 ][47:32];
363      R2_odd[1] = last_ram_out-
puts[ read_ram_index_odd_plus_1 ][47:32];
364      R3_odd[1] = last_ram_out-
puts[ read_ram_index_odd_minus_1 ][47:32];
365      R4_odd[1] = last_ram_out-
puts[ read_ram_index_odd_plus_1 ][47:32];
366
367      B1_odd[2] = last_ram_out-
puts[ read_ram_index_odd          ][31:16];
368      B2_odd[2] = last_ram_out-
puts[ read_ram_index_odd          ][31:16];
369      B3_odd[2] = last_ram_out-
puts[ read_ram_index_odd          ][31:16];
370      B4_odd[2] = last_ram_out-
puts[ read_ram_index_odd          ][31:16];
371
372      G1_odd[2] = last_ram_out-
puts[ read_ram_index_odd_minus_1 ][31:16];
373      G2_odd[2] = last_ram_out-
puts[ read_ram_index_odd_plus_1 ][31:16];
374      G3_odd[2] = last_ram_out-
puts[ read_ram_index_odd          ][47:32];
375      G4_odd[2] = last_ram_out-
puts[ read_ram_index_odd          ][ 15:0 ];
376
377      R1_odd[2] = last_ram_out-
puts[ read_ram_index_odd_minus_1 ][ 15:0 ];
378      R2_odd[2] = last_ram_out-
puts[ read_ram_index_odd_minus_1 ][47:32];
379      R3_odd[2] = last_ram_out-
puts[ read_ram_index_odd_plus_1 ][ 15:0 ];
380      R4_odd[2] = last_ram_out-
puts[ read_ram_index_odd_plus_1 ][47:32];
381
382      B1_odd[3] =
RAM_out_reg[ read_ram_index_odd          ][63:48];
383      B2_odd[3] = last_ram_out-
puts[ read_ram_index_odd          ][31:16];
384      B3_odd[3] =
RAM_out_reg[ read_ram_index_odd          ][63:48];
385      B4_odd[3] = last_ram_out-
puts[ read_ram_index_odd          ][31:16];
386
387      G1_odd[3] = last_ram_out-
puts[ read_ram_index_odd          ][ 15:0 ];
388      G2_odd[3] = last_ram_out-
puts[ read_ram_index_odd          ][ 15:0 ];
389      G3_odd[3] = last_ram_out-
puts[ read_ram_index_odd          ][ 15:0 ];
390      G4_odd[3] =
RAM_out_reg[ read_ram_index_odd          ][ 15:0 ];
391
392      R1_odd[3] = last_ram_out-
puts[ read_ram_index_odd_minus_1 ][ 15:0 ];
393      R2_odd[3] = last_ram_out-
puts[ read_ram_index_odd_plus_1 ][ 15:0 ];
394      R3_odd[3] = last_ram_out-
puts[ read_ram_index_odd_minus_1 ][ 15:0 ];
395      R4_odd[3] = last_ram_out-
puts[ read_ram_index_odd_plus_1 ][ 15:0 ];
396
397
398      if (!line_counter[0]) //even
399          begin
400              for (i=3'b0; i < 4 ; i = i + 1)
401                  begin
402                      R1[i] <= R1_even[i];
403                      G1[i] <= G1_even[i];
404                      B1[i] <= B1_even[i];
405                      R2[i] <= R2_even[i];
406                      G2[i] <= G2_even[i];
407                      B2[i] <= B2_even[i];
408                      R3[i] <= R3_even[i];
409                      G3[i] <= G3_even[i];
410
411                      B3[i] <= B3_even[i];
412                      R4[i] <= R4_even[i];
413                      G4[i] <= G4_even[i];
414                      B4[i] <= B4_even[i];
415
416                  end

```

```

416         end //end even rows
417     else
418         begin //odd rows //First line
419
420         for (i=3'b0; i < 4 ; i = i + 1)
421             begin
422                 R1[i] <= R1_odd[i];
423                 G1[i] <= G1_odd[i];
424                 B1[i] <= B1_odd[i];
425                 R2[i] <= R2_odd[i];
426                 G2[i] <= G2_odd[i];
427                 B2[i] <= B2_odd[i];
428                 R3[i] <= R3_odd[i];
429                 G3[i] <= G3_odd[i];
430
431                 B3[i] <= B3_odd[i];
432                 R4[i] <= R4_odd[i];
433                 G4[i] <= G4_odd[i];
434                 B4[i] <= B4_odd[i];
435
436             end
437         end //end odd rows
438
439         {not_used2b,output_o[191:176]} <=
440         {{2'd0, R1[0]} + R2[0] + R3[0] + R4[0]} >> 2; //R
441         {not_used2b,output_o[175:160]} <=
442         {{2'd0, G1[0]} + G2[0] + G3[0] + G4[0]} >> 2; //G
443         {not_used2b,output_o[159:144]} <=
444         {{2'd0, B1[0]} + B2[0] + B3[0] + B4[0]} >> 2; //B
445         {not_used2b,output_o[143:128]} <=
446         {{2'd0, R1[1]} + R2[1] + R3[1] + R4[1]} >> 2; //R
447         {not_used2b,output_o[127:112]} <=
448         {{2'd0, G1[1]} + G2[1] + G3[1] + G4[1]} >> 2; //G
449         {not_used2b,output_o[111:96]} <=
450         {{2'd0, B1[1]} + B2[1] + B3[1] + B4[1]} >> 2; //B
451
452         {not_used2b,output_o[95:80]} <= {{2'd0,
453         R1[2]} + R2[2] + R3[2] + R4[2]} >> 2; //R
454         {not_used2b,output_o[79:64]} <= {{2'd0,
455         G1[2]} + G2[2] + G3[2] + G4[2]} >> 2; //G
456         {not_used2b,output_o[63:48]} <= {{2'd0,
457         B1[2]} + B2[2] + B3[2] + B4[2]} >> 2; //B
458
459         {not_used2b,output_o[47:32]} <= {{2'd0,
460         R1[3]} + R2[3] + R3[3] + R4[3]} >> 2; //R
461         {not_used2b,output_o[31:16]} <= {{2'd0,
462         G1[3]} + G2[3] + G3[3] + G4[3]} >> 2; //G
463         {not_used2b,output_o[15:0]} <=
464         {{2'd0, B1[3]} + B2[3] + B3[3] + B4[3]} >> 2; //B
465
466     end
467 endmodule

```

2. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/ FPGA_Source/source/mipi_csi_bridge/mipi_csi_bridge.v (203 lines)

```

1 `timescale 1ns/1ns
2
3 /*
4 MIPI CSI RX to Parallel Bridge (c) by Gaurav Singh www.Cir-
5 cuitValley.com
6 MIPI CSI RX to Parallel Bridge is licensed under a
7 Creative Commons Attribution 3.0 Unported License.
8
9 You should have received a copy of the license along with this
10 work. If not, see <http://creativecommons.org/licenses/by/
11 3.0/>.
12 */
13 /*
14 MIPI CSI 4 Lane Receiver To Parallel Bridge
15 Tested with Lattice MachX03LF-6900 with IMX219 Camera
16 Takes MIPI Clock and 4 Data lane as input convert into Paral-
17 lel YUV output
18 Outputs 32bit YUV data with Frame sync, lsync and pixel clock
19 */
20 module mipi_bridge(    reset_in,
21
22                        mipi_clk_p_in,
23                        mipi_clk_n_in,
24                        mipi_data_p_in,
25                        mipi_data_n_in,
26
27                        pclk_o,
28                        data_o,
29                        fsync_o,
30                        lsync_o,

```

```

31                        frame_sync_in,
32                        line_sync_in,
33                    );
34
35 parameter MIPI_LANES = 4;
36 input frame_sync_in;
37 input line_sync_in;
38
39 input reset_in;
40 input mipi_clk_p_in;
41 input mipi_clk_n_in;
42 input [MIPI_LANES-1:0]mipi_data_p_in;
43 input [MIPI_LANES-1:0]mipi_data_n_in;
44
45 output pclk_o;
46 output [31:0]data_o;
47 output fsync_o;
48 output lsync_o;
49
50
51 wire osc_clk;
52
53 wire reset ;
54
55
56
57 wire output_clock;
58 wire mipi_byte_clock; //byte clock from mipi phy
59
60
61
62 wire [3:0]is_byte_valid;
63 wire is_lane_aligned_valid;
64 wire is_decoded_valid;
65 wire is_unpacked_valid;
66 wire is_rgb_valid;
67 wire is_yuv_valid;
68
69 wire mipi_out_clk;
70
71 wire [31:0]mipi_data_raw;
72 wire [31:0]byte_aligned;
73 wire [31:0]lane_aligned;
74 wire [31:0]decoded_data;
75 wire [2:0]packet_type;
76 wire [15:0]packet_length;
77 wire [63:0]unpacked_data;
78 wire [191:0]rgb_data;
79 wire [63:0]yuv_data;
80
81 wire byte_aligner_reset;
82 //wire frame_sync_in;
83 assign reset = !reset_in;
84
85
86 assign byte_aligner_reset = line_sync_in;
87
88 oscillator oscillator_inst0(.hf_out_en_i(1'b1),
89
90                                .hf_
91                                .lf_
92                                );
93 mipi_csi_phy mipi_csi_phy_inst0(
94
95                                .sync_rst_i(reset),
96
97                                // .lmmi_clk_i(osc_clk),
98
99                                //.lmmi_resetrn_i(),
100                                //.lmmi_wdata_i(),
101                                //.lmmi_wr_rdn_i(),
102                                //.lmmi_offset_i(),
103                                //.lmmi_request_i(),
104                                //.lmmi_ready_o(),
105                                //.lmmi_rdata_o(),
106                                //.lmmi_rdata_valid_o(),
107                                //.hs_rx_en_i(1'b1),
108                                .hs_rx_data_o(mipi_data_raw),

```

```

106 //hs_rx_data_sync_o(),
107 .lp_rx_en_i(1'b0),
108 .lp_rx_data_p_o(), //byte_aligner_reset),
109 .lp_rx_data_n_o(),
110 .lp_rx_clk_p_o(), //frame_sync_in),
111 .lp_rx_clk_n_o(ready),
112 .pll_lock_i(1'b1),
113 .clk_p_io(mipi_clk_p_in),
114 .clk_n_io(mipi_clk_n_in),
115 .data_p_io(mipi_data_p_in),
116 .data_n_io(mipi_data_n_in),
117 .pd_dphy_i(1'b0),
118 .clk_byte_o(mipi_byte_clock),
119 .ready_o()) ;
120 //oclk(mipi_out_clk), //double to mipi_clock
121
122     mipi_rx_byte_aligner
123 mipi_rx_byte_aligner_0( .clk_i(mipi_byte_clock),
124 .reset_i(byte_aligner_reset),
125 .byte_i(mipi_data_raw[7:0]),
126 .byte_o( byte_aligned[7:0]),
127 .byte_valid_o(is_byte_valid[0]));
128
129     mipi_rx_byte_aligner
130 mipi_rx_byte_aligner_1( .clk_i(mipi_byte_clock),
131 .reset_i(byte_aligner_reset),
132 .byte_i(mipi_data_raw[15:8]),
133 .byte_o(byte_aligned[15:8]),
134 .byte_valid_o(is_byte_valid[1]));
135
136     mipi_rx_byte_aligner
137 mipi_rx_byte_aligner_2( .clk_i(mipi_byte_clock),
138 .reset_i(byte_aligner_reset),
139 .byte_i(mipi_data_raw[23:16]),
140 .byte_o( byte_aligned[23:16]),
141 .byte_valid_o(is_byte_valid[2]));
142
143     mipi_rx_byte_aligner
144 mipi_rx_byte_aligner_3( .clk_i(mipi_byte_clock),
145 .reset_i(byte_aligner_reset),

```

```

145 .byte_i(mipi_data_raw[31:24]),
146 .byte_o( byte_aligned[31:24]),
147 .byte_valid_o(is_byte_valid[3]));
148
149     mipi_rx_lane_aligner
150 mipi_rx_lane_aligner( .clk_i(mipi_byte_clock),
151 .reset_i(byte_aligner_reset),
152 .bytes_valid_i(is_byte_valid),
153 .byte_i(byte_aligned),
154 .lane_valid_o(is_lane_aligned_valid),
155 .lane_byte_o(lane_aligned));
156
157     mipi_csi_packet_decoder mipi_csi_packet_decoder_0( .clk_i(mipi_byte_clock),
158 .data_valid_i(is_lane_aligned_valid),
159 .data_i(lane_aligned),
160 .output_valid_o(is_decoded_valid),
161 .data_o(decoded_data),
162 .packet_length_o(packet_length),
163 .packet_type_o(packet_type));
164
165     mipi_rx_raw_depacker mipi_rx_raw_depacker_0( .clk_i(mipi_byte_clock),
166 .data_valid_i(is_decoded_valid),
167 .data_i(decoded_data),
168 .packet_type_i(packet_type),
169 .output_o(unpacked_data),
170 .output_valid_o(is_unpacked_valid));
171
172     debayer_filter debayer_filter_0( .clk_i(mipi_byte_clock),
173 .reset_i(!frame_sync_in),
174 .line_valid_i(is_decoded_valid),
175 .data_i(unpacked_data),
176 .data_valid_i(is_unpacked_valid),
177 .output_o(rgb_data),
178 .output_valid_o(is_rgb_valid));
179
180     rgb_to_yuv rgb_to_yuv_0( .clk_i(mipi_byte_clock),
181 .reset_i(!frame_sync_in),
182 .rgb_i(rgb_data),
183 .rgb_val-
184 .yuv_o(yuv_data),
185 .yuv_val-
186 id_i(is_rgb_valid),
187 id_o(is_yuv_valid));
188
189

```



```

190 output_reformatter out_reformat-
ter_0( .clk_i(mipi_byte_clock),
191 .line_sync_i(is_decoded_valid),
192 .frame_sync_i(frame_sync_in),
193 .output_clk_i(mipi_out_clk),
194 .data_i(yuv_data),
195 .data_in_valid_i(is_yuv_valid),
196 .output_o(data_o),
197 .output_valid_o(lsync_o));
198 //assign pclk_o = mipi_byte_clock;
199 //assign pclk_o = frame_sync_in? osc_clk: mipi_out_clk ; //out-
put clock always available, slow when there is no mipi frame , fast
from mipi_clk when mipi_clock is active
200 assign fsync_o = !
frame_sync_in; //activate fsync
as soon as mipi frame is active
201 assign fsync_o = !
202
203 endmodule

```

3. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/ FPGA_Source/source/mipi_csi_bridge/ mipi_csi_packet_decoder.v (90 lines)

```

1 `timescale 1ns/1ns
2
3 /*
4  MIPI CSI RX to Parallel Bridge (c) by Gaurav Singh www.Cir-
cuitValley.com
5
6  MIPI CSI RX to Parallel Bridge is licensed under a
7  Creative Commons Attribution 3.0 Unported License.
8
9  You should have received a copy of the license along with this
10 work. If not, see <http://creativecommons.org/licenses/by/
3.0/>.
11 */
12
13 /*
14  Basically a packet Stripper, removes header and footer from
packet
15 Takes lane aligned data from lane aligner @ mipi byte clock
16 looks for specific packet types, in this case RAW10bit 0x2B
RAW12bit 0x2C and RAW14bit 0x2D , Packet type is also output to be
used in next modules
17 outputs Stripped bytes in exactly the way they were received.
18 this module also fetch packet length and output_valid is active
as long as input data is valid and packet length is still valid.
19 */
20
21 module mipi_csi_packet_decoder(
22
23 clk_i,
24
25 data_valid_i,
26
27 data_i,
28
29 output_valid_o,
30
31 data_o,
32
33 packet_length_o,
34
35 packet_type_o
36 )
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

```

39 output reg output_valid_o;
40 output reg [31:0]data_o;
41 output reg [15:0]packet_length_o;
42 output reg [2:0]packet_type_o;
43
44 reg [15:0]packet_length_reg;
45 reg [31:0]data_reg;
46 reg [31:0]last_data_i;
47
48 always @(negedge clk_i)
49 begin
50     if (data_valid_i)
51     begin
52         last_data_i <= data_reg;
53         output_valid_o <= |packet_length_reg;
54
55         if (|packet_length_reg)
56         begin
57             packet_length_reg <= packet_length_reg
58         end
59     end
60     else if (last_data_i[7:0] == SYNC_BYTE &&
61 (data_reg[7:0] == MIPI_CSI_PACKET_10bRAW || data_reg[7:0] ==
MIPI_CSI_PACKET_12bRAW || data_reg[7:0] == MIPI_CSI_PACKET_14bRAW))
62     begin
63         //TODO: better timings could be achived by
just copy whole data_reg into a reg and later take individual values
from that reg
64         packet_type_o <= data_reg[2:0];
65         packet_length_o <= {data_reg[23:16],
data_reg[15:8]};
66         packet_length_reg <= {data_reg[23:16],
data_reg[15:8]};
67     end
68     else
69     begin
70         packet_length_reg <= 32'h0;
71         packet_type_o <= 3'h0;
72         packet_length_o <= 32'h0;
73     end
74     end
75     end
76     end
77     end
78     end
79     end
80     end
81     end
82     end
83     end
84     end
85     end
86     end
87     end
88     end
89     end
90     end
91     end
92     end
93     end
94     end
95     end
96     end
97     end
98     end
99     end
100     end
101     end
102     end
103     end
104     end
105     end
106     end
107     end
108     end
109     end
110     end
111     end
112     end
113     end
114     end
115     end
116     end
117     end
118     end
119     end
120     end
121     end
122     end
123     end
124     end
125     end
126     end
127     end
128     end
129     end
130     end
131     end
132     end
133     end
134     end
135     end
136     end
137     end
138     end
139     end
140     end
141     end
142     end
143     end
144     end
145     end
146     end
147     end
148     end
149     end
150     end
151     end
152     end
153     end
154     end
155     end
156     end
157     end
158     end
159     end
160     end
161     end
162     end
163     end
164     end
165     end
166     end
167     end
168     end
169     end
170     end
171     end
172     end
173     end
174     end
175     end
176     end
177     end
178     end
179     end
180     end
181     end
182     end
183     end
184     end
185     end
186     end
187     end
188     end
189     end
190     end
191     end
192     end
193     end
194     end
195     end
196     end
197     end
198     end
199     end
200     end
201     end
202     end
203     end
204     end
205     end
206     end
207     end
208     end
209     end
210     end
211     end
212     end
213     end
214     end
215     end
216     end
217     end
218     end
219     end
220     end
221     end
222     end
223     end
224     end
225     end
226     end
227     end
228     end
229     end
230     end
231     end
232     end
233     end
234     end
235     end
236     end
237     end
238     end
239     end
240     end
241     end
242     end
243     end
244     end
245     end
246     end
247     end
248     end
249     end
250     end
251     end
252     end
253     end
254     end
255     end
256     end
257     end
258     end
259     end
260     end
261     end
262     end
263     end
264     end
265     end
266     end
267     end
268     end
269     end
270     end
271     end
272     end
273     end
274     end
275     end
276     end
277     end
278     end
279     end
280     end
281     end
282     end
283     end
284     end
285     end
286     end
287     end
288     end
289     end
290     end
291     end
292     end
293     end
294     end
295     end
296     end
297     end
298     end
299     end
300     end
301     end
302     end
303     end
304     end
305     end
306     end
307     end
308     end
309     end
310     end
311     end
312     end
313     end
314     end
315     end
316     end
317     end
318     end
319     end
320     end
321     end
322     end
323     end
324     end
325     end
326     end
327     end
328     end
329     end
330     end
331     end
332     end
333     end
334     end
335     end
336     end
337     end
338     end
339     end
340     end
341     end
342     end
343     end
344     end
345     end
346     end
347     end
348     end
349     end
350     end
351     end
352     end
353     end
354     end
355     end
356     end
357     end
358     end
359     end
360     end
361     end
362     end
363     end
364     end
365     end
366     end
367     end
368     end
369     end
370     end
371     end
372     end
373     end
374     end
375     end
376     end
377     end
378     end
379     end
380     end
381     end
382     end
383     end
384     end
385     end
386     end
387     end
388     end
389     end
390     end
391     end
392     end
393     end
394     end
395     end
396     end
397     end
398     end
399     end
400     end
401     end
402     end
403     end
404     end
405     end
406     end
407     end
408     end
409     end
410     end
411     end
412     end
413     end
414     end
415     end
416     end
417     end
418     end
419     end
420     end
421     end
422     end
423     end
424     end
425     end
426     end
427     end
428     end
429     end
430     end
431     end
432     end
433     end
434     end
435     end
436     end
437     end
438     end
439     end
440     end
441     end
442     end
443     end
444     end
445     end
446     end
447     end
448     end
449     end
450     end
451     end
452     end
453     end
454     end
455     end
456     end
457     end
458     end
459     end
460     end
461     end
462     end
463     end
464     end
465     end
466     end
467     end
468     end
469     end
470     end
471     end
472     end
473     end
474     end
475     end
476     end
477     end
478     end
479     end
480     end
481     end
482     end
483     end
484     end
485     end
486     end
487     end
488     end
489     end
490     end
491     end
492     end
493     end
494     end
495     end
496     end
497     end
498     end
499     end
500     end
501     end
502     end
503     end
504     end
505     end
506     end
507     end
508     end
509     end
510     end
511     end
512     end
513     end
514     end
515     end
516     end
517     end
518     end
519     end
520     end
521     end
522     end
523     end
524     end
525     end
526     end
527     end
528     end
529     end
530     end
531     end
532     end
533     end
534     end
535     end
536     end
537     end
538     end
539     end
540     end
541     end
542     end
543     end
544     end
545     end
546     end
547     end
548     end
549     end
550     end
551     end
552     end
553     end
554     end
555     end
556     end
557     end
558     end
559     end
560     end
561     end
562     end
563     end
564     end
565     end
566     end
567     end
568     end
569     end
570     end
571     end
572     end
573     end
574     end
575     end
576     end
577     end
578     end
579     end
580     end
581     end
582     end
583     end
584     end
585     end
586     end
587     end
588     end
589     end
590     end
591     end
592     end
593     end
594     end
595     end
596     end
597     end
598     end
599     end
600     end
601     end
602     end
603     end
604     end
605     end
606     end
607     end
608     end
609     end
610     end
611     end
612     end
613     end
614     end
615     end
616     end
617     end
618     end
619     end
620     end
621     end
622     end
623     end
624     end
625     end
626     end
627     end
628     end
629     end
630     end
631     end
632     end
633     end
634     end
635     end
636     end
637     end
638     end
639     end
640     end
641     end
642     end
643     end
644     end
645     end
646     end
647     end
648     end
649     end
650     end
651     end
652     end
653     end
654     end
655     end
656     end
657     end
658     end
659     end
660     end
661     end
662     end
663     end
664     end
665     end
666     end
667     end
668     end
669     end
670     end
671     end
672     end
673     end
674     end
675     end
676     end
677     end
678     end
679     end
680     end
681     end
682     end
683     end
684     end
685     end
686     end
687     end
688     end
689     end
690     end
691     end
692     end
693     end
694     end
695     end
696     end
697     end
698     end
699     end
700     end
701     end
702     end
703     end
704     end
705     end
706     end
707     end
708     end
709     end
710     end
711     end
712     end
713     end
714     end
715     end
716     end
717     end
718     end
719     end
720     end
721     end
722     end
723     end
724     end
725     end
726     end
727     end
728     end
729     end
730     end
731     end
732     end
733     end
734     end
735     end
736     end
737     end
738     end
739     end
740     end
741     end
742     end
743     end
744     end
745     end
746     end
747     end
748     end
749     end
750     end
751     end
752     end
753     end
754     end
755     end
756     end
757     end
758     end
759     end
760     end
761     end
762     end
763     end
764     end
765     end
766     end
767     end
768     end
769     end
770     end
771     end
772     end
773     end
774     end
775     end
776     end
777     end
778     end
779     end
780     end
781     end
782     end
783     end
784     end
785     end
786     end
787     end
788     end
789     end
790     end
791     end
792     end
793     end
794     end
795     end
796     end
797     end
798     end
799     end
800     end
801     end
802     end
803     end
804     end
805     end
806     end
807     end
808     end
809     end
810     end
811     end
812     end
813     end
814     end
815     end
816     end
817     end
818     end
819     end
820     end
821     end
822     end
823     end
824     end
825     end
826     end
827     end
828     end
829     end
830     end
831     end
832     end
833     end
834     end
835     end
836     end
837     end
838     end
839     end
840     end
841     end
842     end
843     end
844     end
845     end
846     end
847     end
848     end
849     end
850     end
851     end
852     end
853     end
854     end
855     end
856     end
857     end
858     end
859     end
860     end
861     end
862     end
863     end
864     end
865     end
866     end
867     end
868     end
869     end
870     end
871     end
872     end
873     end
874     end
875     end
876     end
877     end
878     end
879     end
880     end
881     end
882     end
883     end
884     end
885     end
886     end
887     end
888     end
889     end
890     end
891     end
892     end
893     end
894     end
895     end
896     end
897     end
898     end
899     end
900     end
901     end
902     end
903     end
904     end
905     end
906     end
907     end
908     end
909     end
910     end
911     end
912     end
913     end
914     end
915     end
916     end
917     end
918     end
919     end
920     end
921     end
922     end
923     end
924     end
925     end
926     end
927     end
928     end
929     end
930     end
931     end
932     end
933     end
934     end
935     end
936     end
937     end
938     end
939     end
940     end
941     end
942     end
943     end
944     end
945     end
946     end
947     end
948     end
949     end
950     end
951     end
952     end
953     end
954     end
955     end
956     end
957     end
958     end
959     end
960     end
961     end
962     end
963     end
964     end
965     end
966     end
967     end
968     end
969     end
970     end
971     end
972     end
973     end
974     end
975     end
976     end
977     end
978     end
979     end
980     end
981     end
982     end
983     end
984     end
985     end
986     end
987     end
988     end
989     end
990     end
991     end
992     end
993     end
994     end
995     end
996     end
997     end
998     end
999     end
1000     end

```

4. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/ FPGA_Source/source/mipi_csi_bridge/mipi_rx_byte_aligner.v (97 lines)

```

1 `timescale 1ns/1ns
2
3 /*
4  MIPI CSI RX to Parallel Bridge (c) by Gaurav Singh www.Cir-
cuitValley.com
5
6  MIPI CSI RX to Parallel Bridge is licensed under a
7  Creative Commons Attribution 3.0 Unported License.
8
9  You should have received a copy of the license along with this
10 work. If not, see <http://creativecommons.org/licenses/by/
3.0/>.
11 */
12
13 /*
14  Received Raw unaligned bits from DDR RX module outputs Aligned
bytes
15 Bytes on MIPI lane does not have any defined byte boundary so
this modules Looks for always constant first byte 0xB8 on wire,
16 once 0xB8 is found, byte boundary offset is determined, set
output valid to active and start outputting correct bytes
17 stays reset when data lane are in MIPI LP state
18 */
19
20
21 module mipi_rx_byte_aligner(
22
23 clk_i,
24 reset_i,
25 byte_i,
26 byte_o,
27 byte_valid_o
28 )
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
37
```

```

29 localparam [7:0]SYNC_BYTE = 8'h8;
30
31 input clk_i;
32 input reset_i;
33 input [7:0]byte_i;
34 output reg [7:0]last_byte;
35 output reg byte_valid_o;
36
37 reg [2:0]offset;
38 reg [3:0]i;
39 reg [7:0] last_byte;
40 wire [16:0]word;
41
42 reg [7:0] output_reg;
43 reg valid_reg;
44 assign word = {byte_i, last_byte};
45
46
47 always @(negedge clk_i or posedge reset_i)
48 begin
49     if (reset_i)
50     begin
51         valid_reg <= 1'b0;
52         last_byte <= 8'h0;
53         offset <= 3'h0;
54
55         output_reg <= SYNC_BYTE; //first byte output is
56         always sync byte once byte_valid_o is high
57         end
58         else
59         begin
60             last_byte <= byte_i;
61
62             if (!offset) //offset minimum 1
63             begin
64                 for (i= 8'h0; i < 8; i = i + 1'h1) //need to
65                 have loop 8 time not 9 because if input bytes are already aligned
66                 they will fall on last_byte or byte_i
67                 begin
68                     if ((word[(i + 1'h1) +: 8] ==
69                     SYNC_BYTE))
70                     begin
71                         valid_reg <=
72                         1'b1;
73                         offset <=
74                         i[2:0] + 1'b1;
75                     end
76                     end
77                 end
78                 end
79                 end
80                 end
81                 end
82                 end
83                 end
84                 end
85                 end
86                 end
87                 end
88                 end
89                 end
90                 end
91                 end
92                 end
93                 end
94                 end
95                 end
96                 end
97                 end

```

5. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/ FPGA_Source/source/mipi_csi_bridge/mipi_rx_lane_aligner.v (137 lines)

```

1 `timescale 1ns/1ns
2
3 /*
4  * MIPI CSI RX to Parallel Bridge (c) by Gaurav Singh www.CircuitValley.com
5  *
6  * MIPI CSI RX to Parallel Bridge is licensed under a
7  * Creative Commons Attribution 3.0 Unported License.
8  *
9  * You should have received a copy of the license along with this
10  * work. If not, see <http://creativecommons.org/licenses/by/3.0/>.
11  */
12
13 /*
14  * Receives multiple lane, byte aligned data from mipi rx byte

```

aligner @mipi byte clock outputs lane aligned data

15 in a multi-lane mipi bus, data on different lane may appear at different offset so this module will wait till of the all lanes have valid output

16 start outputting lane aligned data so byte x from all the lanes outputted at same timescale

```

17 */
18
19 module mipi_rx_lane_aligner (
20     clk_i,
21     reset_i,
22     bytes_valid_i,
23     byte_i,
24     lane_valid_o,
25     lane_byte_o);
26
27 input clk_i;
28 input reset_i;
29 localparam [3:0]ALIGN_DEPTH = 4'h7; //how many byte misalignment is allowed, whole package length must be also longer than this
30 localparam [2:0]LANES = 3'h4; //mipi csi have 2 or 4 lanes
31
32 input [LANES-1:0]bytes_valid_i;
33 input [((8*LANES)-1):0]byte_i;
34 output reg [((8*LANES)-1):0]lane_byte_o;
35 output reg lane_valid_o;
36
37 reg [((8*LANES)-1):0]last_bytes[ALIGN_DEPTH-1:0];
38 reg [2:0]sync_byte_index[LANES-1:0];
39 reg [2:0]sync_byte_index_reg[LANES-1:0];
40 reg [2:0]offset;
41
42 reg [3:0]valid;
43 reg lane_valid_reg;
44 reg [3:0]i;
45
46 // TODO: Find why async reset with !(|bytes_valid_i) cause whole system reset;
47
48 always @(posedge clk_i)
49 begin
50     lane_byte_o = {last_bytes[sync_byte_index_reg[3]][31:24],
51     last_bytes[sync_byte_index_reg[2]][23:16],
52     last_bytes[sync_byte_index_reg[1]][15:8],
53     last_bytes[sync_byte_index_reg[0]][7:0]};
54
55 end
56
57 always @(posedge clk_i)
58 begin
59     if (reset_i)
60     begin
61         for (i= 4'h0; i < ALIGN_DEPTH; i = i + 1'h1)
62         begin
63             last_bytes[i] <= 8'h0;
64         end
65     end
66     else
67     begin
68         last_bytes[0] <= byte_i;
69         for (i= 1'b1; i < (ALIGN_DEPTH); i = i + 1'h1)
70         begin
71             last_bytes[i] <= last_bytes[i-1'h1];
72         end
73     end
74 end
75
76 end
77
78
79
80 always @(posedge clk_i)
81 begin
82     if (reset_i)
83     begin
84         valid <= 4'b0;
85     end
86     else
87     begin
88         valid <= bytes_valid_i;
89     end
90 end
91
92 always @(posedge clk_i)
93 begin
94     if (reset_i || (!lane_valid_o && (!(|valid)))) //always reset when lane_valid_o is active, valid
95     begin
96         for (i= 4'h0; i < LANES; i = i + 1'h1)
97         begin

```

```

98             sync_byte_index[i] <= ALIGN_DEPTH -
1'b1;
99         end
100         offset <= ALIGN_DEPTH - 2'h2;
101     end
102     else
103     begin
104         offset <= offset - 1'b1;
105     for (i= 4'h0; i < LANES; i = i + 1'h1)
106     begin
107         sync_byte_index[i] <=
sync_byte_index[i] - !valid[i]; //count delay of each sync, first
one will be 0 delay, last one will max
108     end
109     end
110 end
111
112 always @(posedge clk_i)
113 begin
114     if (reset_i)
115     begin
116         for (i= 4'h0; i < LANES; i = i + 1'h1)
117         begin
118             sync_byte_index_reg[i] <= 3'h0;
119         end
120         lane_valid_o <= 1'h0;
121     end
122     else
123     begin
124         lane_valid_reg <= (&valid)? 1'h1:(lane_valid_o
&&[valid]? 1'h1 : 1'h0; //one clock delay to the last packet
125         lane_valid_o <= lane_valid_reg;
126         if (!lane_valid_reg)
127         begin
128             for (i= 4'h0; i < LANES; i = i + 1'h1)
129             begin
130                 sync_byte_index_reg[i] <=
sync_byte_index[i] - offset;
131             end
132         end
133     end
134 end
135 end
136
137 endmodule

```

6. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/ FPGA_Source/source/mipi_csi_bridge/ mipi_rx_raw12_depacker.v (78 lines)

```

1 `timescale 1ns/1ns
2
3 /*
4 MIPI CSI RX to Parallel Bridge (c) by Gaurav Singh www.Cir-
cuitValley.com
5
6 MIPI CSI RX to Parallel Bridge is licensed under a
7 Creative Commons Attribution 3.0 Unported License.
8
9 You should have received a copy of the license along with this
10 work. If not, see <http://creativecommons.org/licenses/by/
3.0/>.
11 */
12
13 /*
14 Receives 4 lane raw mipi bytes from packet decoder, rearrange
bytes to output 4 pixel 12bit each
15 output is one clock cycle delayed, because the way , MIPI
RAW12 is packed
16 output come in group of 2x48bit chunk on each clock cycle,
output_valid_o remains active only while 8 pixel chunk is outputted
17 */
18
19 module mipi_rx_raw12_depacker( clk_i,
20
21 data_valid_i,
22
23 data_i,
24
25 output_valid_o,
26
27 output_o);
28
29 localparam [2:0]TRANSFERS_PERPACK = 3'h2; // RAW 12 is packed
<Sample0[10:4]> <Sample1[10:4]> <Sample1[3:0],
Sample0[3:0]><Sample2[10:4]> <Sample3[10:4]>
<Sample3[3:0],Sample2[3:0]>
30 input clk_i;
31 input data_valid_i;
32 input [31:0]data_i;
33 output reg output_valid_o;
34 output reg [47:0]output_o;

```

```

31
32 reg [7:0]offset;
33 reg [2:0]byte_count;
34 reg [31:0]last_data_i;
35
36 wire [63:0]word;
37 assign word = {data_i,last_data_i}; //would need last bytes as
well as current data to get full 4 pixel
38
39 always @(posedge clk_i)
40 begin
41     if (data_valid_i)
42     begin
43         last_data_i <= data_i;
44         //RAW 12 , Byte1 -> Byte2 -> Byte3 -> Byte4 ->
[LSbB4[1:0]LSbB3[1:0]LSbB2[1:0]LSbB1[1:0]]
45         output_o[47:36] <= {word [(offset + 7) -:
8], word [(offset + 23) -:4]}; //lane 1
46         output_o[35:24] <= {word [(offset + 15) -:
8], word [(offset + 19) -:4]}; //lane 2
47         output_o[23:12] <= {word [(offset + 31) -:
8], word [(offset + 51) -:4]}; //lane 3
48         output_o[11:0] <= {word [(offset + 39) -:
8], word [(offset + 47) -:4]}; //lane 4
49
50         if (byte_count < (TRANSFERS_PERPACK))
51         begin
52             byte_count <= byte_count + 1'd1;
53             if (byte_count )
54             begin
55                 offset <= ((offset + 8'd16) &
8'h1F);
56                 output_valid_o <= 1'h1;
57             end
58         end
59     end
60     else
61     begin
62         offset <= 8'h0;
63
64         byte_count <= 4'b1; //this
byte is the first byte
65         output_valid_o <= 1'h0;
66     end
67 end
68 else
69 begin
70     output_o <= 40'h0;
71     last_data_i <= 1'h0;
72     offset <= 8'h0;
73     byte_count <= 3'b0;
74     output_valid_o <= 1'h0;
75 end
76 end
77
78 endmodule

```

7. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/ FPGA_Source/source/mipi_csi_bridge/ mipi_rx_raw_depacker.v (249 lines)

```

1 `timescale 1ns/1ns
2
3 /*
4 MIPI CSI RX to Parallel Bridge (c) by Gaurav Singh www.Cir-
cuitValley.com
5
6 MIPI CSI RX to Parallel Bridge is licensed under a
7 Creative Commons Attribution 3.0 Unported License.
8
9 You should have received a copy of the license along with this
10 work. If not, see <http://creativecommons.org/licenses/by/
3.0/>.
11 */
12
13 /*
14 Receives 4 lane raw mipi bytes from packet decoder, rearrange
bytes to output 4 pixel 10bit each
15 output is one clock cycle delayed, because the way , MIPI
RAW10 is packed
16 output come in group of 5x40bit chunk on each clock cycle,
output_valid_o remains active only while 20 pixel chunk is output-
ted
17 */
18
19 module mipi_rx_raw_depacker( clk_i,
20
21 data_valid_i,
22
23 data_i,
24
25 packet_type_i,
26
27

```

```

output_valid_o,
24
output_o);
25
26 localparam [2:0]TRANSFERS_PERCHUNK= 3'h5; // RAW 10 is packed
<Sample0[9:2]> <Sample1[9:2]> <Sample2[9:2]> <Sample3[9:2]>
<Sample0[1:0], Sample1[1:0], Sample2[1:0], Sample3[1:0]>
27 localparam [7:0]MIPI_CSI_PACKET_10bRAW = 8'h2B;
28 localparam [7:0]MIPI_CSI_PACKET_12bRAW = 8'h2C;
29 localparam [7:0]MIPI_CSI_PACKET_14bRAW = 8'h2D;
30
31 input clk_i;
32 input data_valid_i;
33 input [31:0]data_i;
34 input [2:0]packet_type_i;
35
36 output reg output_valid_o;
37 output reg [63:0]output_o;
38
39 reg [7:0]offset;
40
41 reg [7:0]offset_7;
42 reg [7:0]offset_15;
43 reg [7:0]offset_23;
44 reg [7:0]offset_31;
45 reg [7:0]offset_37;
46 reg [7:0]offset_43;
47 reg [7:0]offset_49;
48 reg [7:0]offset_55;
49 reg [7:0]offset_71;
50 reg [7:0]offset_79;
51 reg [7:0]offset_83;
52 reg [7:0]offset_87;
53 reg [7:0]offset_95;
54 reg [7:0]offset_97;
55 reg [7:0]offset_99;
56 reg [7:0]offset_101;
57 reg [7:0]offset_103;
58 reg [7:0]offset_107;
59 reg [7:0]offset_111;
60
61
62 reg [2:0]byte_count;
63 reg [31:0]last_data_i[3:0];
64 reg [1:0]idle_count;
65
66 reg data_valid_reg;
67 reg [31:0]data_reg;
68 reg [7:0]offset_factor_reg;
69 reg [2:0]burst_length_reg;
70 reg [1:0]idle_length_reg;
71 reg [2:0]packet_type_reg;
72
73 wire [7:0]offset_factor;
74 wire [2:0]burst_length;
75 wire [1:0]idle_length;
76 wire [127:0]word;
77
78 reg [63:0]output_10b;
79 reg [63:0]output_12b;
80 reg [63:0]output_14b;
81 reg output_valid_reg;
82 reg output_valid_reg_2;
83
84 assign word = {last_data_i[0], last_data_i[1],
last_data_i[2], last_data_i[3]}; //would need last bytes as well as
current data to get full 4 pixel
85
86 assign offset_factor = (packet_type_i == (MIPI_CSI_PACK-
ET_10bRAW & 8'h07)) ? 8'd8: (packet_type_i == (MIPI_CSI_PACKET_12bRAW
& 8'h07)) ? 8'd16:8'd24;
87
88 assign burst_length = ((packet_type_i == (MIPI_CSI_PACK-
ET_10bRAW & 8'h07)) || (packet_type_i == (MIPI_CSI_PACKET_14bRAW &
8'h07))) ? 8'd5:8'd3;
89
90 assign idle_length = ((packet_type_i == (MIPI_CSI_PACK-
ET_10bRAW & 8'h07)) || (packet_type_i == (MIPI_CSI_PACKET_12bRAW &
8'h07))) ? 2'd1: 2'd3;
91
92 reg [15:0]pixel_counter_depacker;
93
94 always @(posedge clk_i)
95 begin
96     output_10b[63:48] <= {word [(offset_71) -:8],
word [(offset_97) -:2]} << 6; //lane 1
TODO:Reverify
97     output_10b[47:32] <= {word [(offset_79) -:8],
word [(offset_99) -:2]} << 6;
98     output_10b[31:16] <= {word [(offset_87) -:8],
word [(offset_101) -:2]} << 6;
99     output_10b[15:0] <= {word [(offset_95) -:8],
word [(offset_103) -:2]} << 6; //lane 4
100     output_12b[63:48] <= {word [(offset_71) -:8],
word [(offset_83) -:4]} << 4; //lane 1
101     output_12b[47:32] <= {word [(offset_79) -:8],
word [(offset_87) -:4]} << 4;
102     output_12b[31:16] <= {word [(offset_95) -:8],
word [(offset_107) -:4]} << 4;
103     output_12b[15:0] <= {word [(offset_103) -:8],
word [(offset_111) -:4]} << 4; //lane 4
104     output_14b[63:48] <= {word [offset_7 -:8], word
[(offset_37) -:6]} << 2; //lane 1
105     output_14b[47:32] <= {word [offset_15 -:8], word
[(offset_43) -:6]} << 2;
106     output_14b[31:16] <= {word [offset_23 -:8], word
[(offset_49) -:6]} << 2;
107     output_14b[15:0] <= {word [offset_31 -:8], word
[(offset_55) -:6]} << 2; //lane 4
108
109     if (packet_type_reg == (MIPI_CSI_PACKET_10bRAW &
8'h07))
110     begin
111         output_o <= output_10b;
112     end
113     else if (packet_type_reg == (MIPI_CSI_PACKET_12bRAW &
8'h07))
114     begin
115         output_o <= output_12b;
116     end
117     else // if (packet_type_i == (MIPI_CSI_PACKET_14bRAW &
8'h07))
118     begin
119         output_o <= output_14b;
120     end
121 end
122 end
123
124
125 always @(posedge clk_i)
126 begin
127
128     output_valid_reg_2 <= output_valid_reg;
129     output_valid_o <= output_valid_reg_2;
130
131     if (output_valid_reg)
132     begin
133
134         offset_7 <= offset_7 + off-
set_factor_reg;
135         offset_15 <= offset_15 + off-
set_factor_reg;
136         offset_23 <= offset_23 + off-
set_factor_reg;
137         offset_31 <= offset_31 + off-
set_factor_reg;
138         offset_37 <= offset_37 + off-
set_factor_reg;
139         offset_43 <= offset_43 + off-
set_factor_reg;
140         offset_49 <= offset_49 + off-
set_factor_reg;
141         offset_55 <= offset_55 + off-
set_factor_reg;
142         offset_71 <= offset_71 + off-
set_factor_reg;
143         offset_79 <= offset_79 + off-
set_factor_reg;
144         offset_83 <= offset_83 + off-
set_factor_reg;
145         offset_87 <= offset_87 + off-
set_factor_reg;
146         offset_95 <= offset_95 + off-
set_factor_reg;
147         offset_97 <= offset_97 + off-
set_factor_reg;
148         offset_99 <= offset_99 + off-
set_factor_reg;
149         offset_101 <= offset_101 + off-
set_factor_reg;
150         offset_103 <= offset_103 + off-
set_factor_reg;
151         offset_107 <= offset_107 + off-
set_factor_reg;
152         offset_111 <= offset_111 + off-
set_factor_reg;
153
154     end
155     else
156     begin
157         offset_7 <= 8'd7;
158         offset_15 <= 8'd15;
159         offset_23 <= 8'd23;
160         offset_31 <= 8'd31;
161         offset_37 <= 8'd37;
162         offset_43 <= 8'd43;
163         offset_49 <= 8'd49;
164         offset_55 <= 8'd55;
165         offset_71 <= 8'd71;
166         offset_79 <= 8'd79;
167         offset_83 <= 8'd83;
168         offset_87 <= 8'd87;
169         offset_95 <= 8'd97;

```

```

170         offset_97 <= 8'd97;
171         offset_99 <= 8'd99;
172         offset_101 <= 8'd101;
173         offset_103 <= 8'd103;
174         offset_107 <= 8'd107;
175         offset_111 <= 8'd111;
176     end
177 end
178
179 always @(posedge clk_i )//or negedge data_valid_reg)
180 begin
181     if (data_valid_reg)
182     begin
183         last_data_i[0] <= data_reg;
184         last_data_i[1] <= last_data_i[0];
185         last_data_i[2] <= last_data_i[1];
186         last_data_i[3] <= last_data_i[2];
187         pixel_counter_depacker <=
188         pixel_counter_depacker + 1'b1;
189         //RAW 10 , Byte1 -> Byte2 -> Byte3 -> Byte4 ->
190         [ LSBB1[1:0] LSBB2[1:0] LSBB3[1:0] LSBB4[1:0] ]
191         if (byte_count < (burst_length_reg))
192         begin
193             byte_count <= byte_count + 1'd1;
194             idle_count <= idle_length_reg - 1'b1;
195             output_valid_reg <= 1'b1;
196         end
197     else
198     begin
199         idle_count <= idle_count - 1'b1;
200         if (!idle_count)
201         begin
202             byte_count <=
203             //set to 1 to enable output_valid_o with next edge
204             end
205             output_valid_reg <= 1'h0;
206         end
207     end
208     else
209     begin
210         pixel_counter_depacker <= 0;
211         last_data_i[0] <= 32'h0;
212         last_data_i[1] <= 32'h0;
213         last_data_i[2] <= 32'h0;
214         last_data_i[3] <= 32'h0;
215         byte_count <= burst_length;
216         if (packet_type_i == (MIPI_CSI_PACKET_14bRAW &
217         8'h07)) // for 14bit need to wait for 3 sample while 12bit
218         and 10bit only need 1 sample delay
219         begin
220             idle_count <= 3'd2;
221         end
222     else
223     begin
224         idle_count <= 3'b0; //need to be
225         zero to wait for 1 sample after data become valid
226         end
227         output_valid_reg <= 1'h0;
228         offset_factor <= offset_factor;
229         burst_length_reg <= burst_length;
230         idle_length_reg <= idle_length;
231         packet_type_reg <= packet_type_i;
232     end
233 end
234
235 always @(posedge clk_i)
236 begin
237     data_valid_reg <= data_valid_i;
238     data_reg <= data_i;
239 end
240 endmodule

```

8. circuitvalley USB3_MIPI_CSI2_RX_V2_Crosslink_NX/ FPGA_Source/source/mipi_csi_bridge/output_reformatter.v (163 lines)

```

1 `timescale 1ns/1ns
2
3 /*
4  * MIPI CSI RX to Parallel Bridge (c) by Gaurav Singh www.Cir-
5  * cuitValley.com
6  * MIPI CSI RX to Parallel Bridge is licensed under a

```

```

7  Creative Commons Attribution 3.0 Unported License.
8
9  You should have received a copy of the license along with this
10 work. If not, see <http://creativecommons.org/licenses/by/
11 3.0/>.
12  */
13 /*
14  * Takes 64bit 4pixel yuv input from rgb2yuv module @ mipi byte
15  * clock outputs 32bit 2pixel yuv output @output_clk_i ,
16  * output_clk_i must be generated by same way as mipi byte clock,
17  * output_clk_i must be exactly double to mipi byteclock
18  * This implementation of Output reformatter outputs data which
19  * which meant to send out of the system to a 32bit receiver
20  * depending on requirement this will be need to be adapted as
21  * per the receiver
22  */
23 module output_reformatter(
24     clk_i, //data
25     changes on negedge
26     out-
27     put_clk_i, //output clock double to clk_i to get in 64bit and output
28     32bit
29     data_i,
30     data_in_val-
31     id_i,
32     line_sync_i,
33     output_o,
34     out-
35     put_valid_o,
36     frame_sync_i
37 );
38
39 input line_sync_i;
40 input frame_sync_i;
41 input clk_i;
42 input data_in_valid_i;
43 input [63:0]data_i;
44 output reg output_valid_o;
45 output [31:0]output_o;
46 input output_clk_i;
47
48 reg [10:0] write_address;
49 reg [11:0] read_address;
50
51 wire [31:0]ram_even_o;
52 wire [31:0]ram_odd_o;
53
54 reg [10:0] input_pixel_count;
55 reg line_even_nodd; //select
56 //between two different RAM
57 reg last_line_sync; //helps to de-
58 termine edge of line sync for write address reset
59 reg last_line_even_nodd; //helps to determine
60 edge of line sync for read address reset
61
62 out_line_ram_dp line_odd(
63     .wr_clk_i(!clk_i),
64     .rd_c
65     lk_i(output_clk_i),
66     .rst_i
67     (!frame_sync_i),
68     .wr_c
69     lk_en_i(data_in_valid_i),
70     .rd_e
71     n_i(line_even_nodd),
72     .rd_c
73     lk_en_i(1'b1),
74     .wr_e
75     n_i(!line_even_nodd),
76     .wr_d
77     ata_i({data_i[31:0], data_i[63:32]}),
78     .wr_a
79     ddr_i(write_address),
80     .rd_a
81     ddr_i(read_address),
82     .rd_d
83     ata_o(ram_odd_o));
84
85 out_line_ram_dp line_even(
86     .wr_clk_i(!clk_i),
87     .rd_c
88     lk_i(output_clk_i),
89     .rst_i
90     (!frame_sync_i),
91     .wr_c
92     lk_en_i(data_in_valid_i),
93     .rd_e
94     n_i(!line_even_nodd),
95     .rd_c
96     lk_en_i(1'b1),
97     .wr_e
98     n_i(line_even_nodd),
99     .wr_d
100    ata_i({data_i[31:0], data_i[63:32]}),
101    .wr_a

```

```

ddr_i(write_address),
76 .rd_a
ddr_i(read_address),
77 .rd_d
ata_o(ram_even_o));
78
79 /*
80 out_line_ram_ldp line_odd( .clk_i((line_even_nodd)?!
clk_i:output_clk_i),
81 .dps_i
(1'b1),
82 .rst_i
(!frame_sync_i),
83 .wr_c
lk_en_i(data_in_valid_i),
84 .rd_c
lk_en_i(!line_even_nodd),
85 .wr_e
n_i(line_even_nodd),
86 .wr_d
ata_i(data_i),
87 .wr_a
ddr_i(write_address),
88 .rd_a
ddr_i(read_address),
89 .rd_d
ata_o(ram_even_o),
90 .lram-
ready_o(),
91 .rd_d
atavalid_o());
92
93 out_line_ram_ldp line_even( .clk_i((!line_even_nodd)?!
clk_i:output_clk_i),
94 .dps_i
(1'b1),
95 .rst_i
(!frame_sync_i),
96 .wr_c
lk_en_i(data_in_valid_i),
97 .rd_c
lk_en_i(line_even_nodd),
98 .wr_e
n_i(!line_even_nodd),
99 .wr_d
ata_i(data_i),
100 .wr_a
ddr_i(write_address),
101 .rd_a
ddr_i(read_address),
102 .rd_d
ata_o(ram_odd_o),
103 .lram-
ready_o(),
104 .rd_d
atavalid_o()) ;
105 */
106 //assign output_o = line_even_nodd? ram_odd_o[:(read_ad-
dress[0]?6'd32:6'd0) +:32]: ram_even_o[:(read_address[0]?
6'd32:6'd0) +:32]; //depeding on line select even or odd ram , also
select correct 32bit word from 64 bit ramoutput
107
108 assign output_o = line_even_nodd? ram_odd_o:ram_even_o; //de-
peding on line select even or odd ram
109
110 always @(posedge line_sync_i or negedge frame_sync_i)
111 begin
112     if (!frame_sync_i)
113     begin
114         line_even_nodd <= 0;
115     end
116     else
117     begin
118         line_even_nodd <= !line_even_nodd;
119     end
120 end
121
122
123 always @(negedge clk_i )
124 begin
125     last_line_sync <= line_sync_i;
126     if (!last_line_sync && line_sync_i)
127     begin
128         write_address <= 9'b0;
129         input_pixel_count <= write_address << 1 ; //
Double write_address as each write_address has 64 bit while output
width is 32bit
131     end
132     else
133     begin
134         write_address <= write_address + data_in_val-
id_i;
135     end
136 end
137
138
139 always @(posedge output_clk_i)
140 begin
141     last_line_even_nodd <= line_even_nodd;
142

```

```

143     if ( last_line_even_nodd !=
line_even_nodd) //reset read address for each new line
144     begin
145         read_address <= 12'b0;
146         output_valid_o <= 1'b0;
147     end
148     else
149     begin
150         if (read_address < input_pixel_count)
151         begin
152             read_address <= read_address +
1'b1;
153             output_valid_o <= 1'b1;
154         end
155         else
156         begin
157             output_valid_o <= 1'b0;
158         end
159     end
160 end
161
162
163 endmodule

```

9. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/ FPGA_Source/source/mipi_csi_bridge/rgb_to_yuv.v (220 lines)

```

1 `timescale 1ns/1ns
2
3 /*
4 MIPI CSI RX to Parallel Bridge (c) by Gaurav Singh www.Cir-
cuitValley.com
5
6 MIPI CSI RX to Parallel Bridge is licensed under a
7 Creative Commons Attribution 3.0 Unported License.
8
9 You should have received a copy of the license along with this
10 work. If not, see <http://creativecommons.org/licenses/by/
3.0/>.
11 */
12
13 /*
14 received 4 pixel 120bit RGB from the Debayer filter output
64bit 4pixel yuv422
15 Calculation is done based on integer YUV formula from the YUV
wiki page
16 */
17
18 module rgb_to_yuv(clk_i, //data changes on rising edge ,
latched in on falling edge
19     reset_i,
20     rgb_i,
21     rgb_valid_i,
22     yuv_o,
23     yuv_valid_o);
24
25 localparam PIXEL_DEPTH = 5'd16; //10bit per col-
or
26 localparam PIXEL_PER_CLK = 4'd4; //4pixels per clock cycle
comes , should be even
27 input clk_i;
28 input reset_i;
29 input [(PIXEL_DEPTH * PIXEL_PER_CLK * 3) - 1'd1]:0]rgb_i;
30 input rgb_valid_i;
31
32 output reg [(PIXEL_PER_CLK * 2 * 8) - 1'd1]:0]yuv_o;
33 output reg yuv_valid_o;
34
35 reg [7:0]Y[3:0]; // result
36 reg [7:0]U[3:0];
37 reg [7:0]V[3:0];
38
39
40 //from YUV wiki page full swing
41 // Y = ((77 R + 150G + 29B + 128) >>10)
42 // U = ((-43R - 84G + 127B + 128) >>10) + 128
43 // V = ((127R -106G -21B +128) >>10) + 128
44
45 reg [23:0]Y0_R;
46 reg [23:0]Y0_G;
47 reg [23:0]Y0_B;
48 reg [23:0]Y1_R;
49 reg [23:0]Y1_G;
50 reg [23:0]Y1_B;
51 reg [23:0]Y2_R;
52 reg [23:0]Y2_G;
53 reg [23:0]Y2_B;
54 reg [23:0]Y3_R;
55 reg [23:0]Y3_G;
56 reg [23:0]Y3_B;
57
58 reg [23:0]U0_R;
59 reg [23:0]U0_G;
60 reg [23:0]U0_B;
61 reg [23:0]U2_R;
62 reg [23:0]U2_G;
63 reg [23:0]U2_B;
64
65 reg [23:0]V0_R;
66 reg [23:0]V0_G;

```



```

67 reg [23:0]V0_B;
68 reg [23:0]V2_R;
69 reg [23:0]V2_G;
70 reg [23:0]V2_B;
71
72 reg [23:0]Y0_ADD;
73 reg [23:0]Y1_ADD;
74 reg [23:0]Y2_ADD;
75 reg [23:0]Y3_ADD;
76
77 reg [23:0]U0_ADD;
78 reg [23:0]U2_ADD;
79
80 reg [23:0]V0_ADD;
81 reg [23:0]V2_ADD;
82
83 reg [23:0]Y0_ADD_STAGE2;
84 reg [23:0]Y0_ADD_STAGE3;
85 reg [23:0]Y0_ADD_STAGE4;
86
87 reg [23:0]Y1_ADD_STAGE2;
88 reg [23:0]Y1_ADD_STAGE3;
89 reg [23:0]Y1_ADD_STAGE4;
90
91 reg [23:0]Y2_ADD_STAGE2;
92 reg [23:0]Y2_ADD_STAGE3;
93 reg [23:0]Y2_ADD_STAGE4;
94
95 reg [23:0]Y3_ADD_STAGE2;
96 reg [23:0]Y3_ADD_STAGE3;
97 reg [23:0]Y3_ADD_STAGE4;
98
99
100 reg [23:0]U0_ADD_STAGE2;
101 reg [23:0]U0_ADD_STAGE3;
102 reg [23:0]U0_ADD_STAGE4;
103
104 reg [23:0]V0_ADD_STAGE2;
105 reg [23:0]V0_ADD_STAGE3;
106 reg [23:0]V0_ADD_STAGE4;
107
108
109 reg [23:0]U2_ADD_STAGE2;
110 reg [23:0]U2_ADD_STAGE3;
111 reg [23:0]U2_ADD_STAGE4;
112
113 reg [23:0]V2_ADD_STAGE2;
114 reg [23:0]V2_ADD_STAGE3;
115 reg [23:0]V2_ADD_STAGE4;
116
117
118 reg [23:0]not_used24; //to suppress warning from the tool
119 always @(negedge clk_i)
120 begin
121     yuv_valid_o <= rgb_valid_i;
122
123     Y0_R <= ( 77 * rgb_i[176 +: 16]);
124     Y0_G <= (150 * rgb_i[160 +: 16]);
125     Y0_B <= ( 29 * rgb_i[144 +: 16]);
126
127     U0_R <= (127 * rgb_i[144 +: 16]);
128     U0_G <= ( 43 * rgb_i[176 +: 16]);
129     U0_B <= ( 84 * rgb_i[160 +: 16]);
130
131     V0_R <= (127 * rgb_i[176 +: 16]);
132     V0_G <= (106 * rgb_i[160 +: 16]);
133     V0_B <= ( 21 * rgb_i[144 +: 16]);
134
135     Y0_ADD <= Y0_R + Y0_G;
136
137     U0_ADD <= U0_R - U0_G;
138
139     V0_ADD <= V0_R - V0_G;
140
141     Y0_ADD_STAGE2 <= Y0_ADD + Y0_B;
142     Y0_ADD_STAGE3 <= (Y0_ADD_STAGE2 + 18'd128) >> 16;
143     Y0_ADD_STAGE4 <= Y0_ADD_STAGE3;
144
145     U0_ADD_STAGE2 <= U0_ADD - U0_B;
146     U0_ADD_STAGE3 <= (U0_ADD_STAGE2 + 18'd128) >> 16;
147     U0_ADD_STAGE4 <= U0_ADD_STAGE3 + 32'd128;
148
149     V0_ADD_STAGE2 <= V0_ADD - V0_B;
150     V0_ADD_STAGE3 <= (V0_ADD_STAGE2 + 18'd128) >> 16;
151     V0_ADD_STAGE4 <= V0_ADD_STAGE3 + 32'd128;
152
153     {not_used24,Y[0]} = Y0_ADD_STAGE4;
154     {not_used24,U[0]} = U0_ADD_STAGE4;
155     {not_used24,V[0]} = V0_ADD_STAGE4;
156
157     Y1_R <= ( 77 * rgb_i[128 +: 16]);
158     Y1_G <= (150 * rgb_i[112 +: 16]);
159     Y1_B <= ( 29 * rgb_i[96 +: 16]);
160
161     Y1_ADD <= Y1_R + Y1_G;
162     Y1_ADD_STAGE2 <= Y1_ADD + Y1_B;
163     Y1_ADD_STAGE3 <= (Y1_ADD_STAGE2 + 18'd128) >> 16;
164     Y1_ADD_STAGE4 <= Y1_ADD_STAGE3;
165
166     {not_used24,Y[1]} = Y1_ADD_STAGE4;
167     //U[1] and V[1] not need to yuv422 sub sampling

```

```

168
169     Y2_R <= ( 77 * rgb_i[128 +:
160     16]); //T000 May be incorrect index , be-
161     cause same as Y1_R
162     Y2_G <= (150 * rgb_i[64 +: 16]);
163     Y2_B <= ( 29 * rgb_i[48 +: 16]);
164
165     U2_R <= (127 * rgb_i[48 +: 16]);
166     U2_G <= ( 43 * rgb_i[128 +: 16]);
167     U2_B <= (84 * rgb_i[64 +: 16]);
168
169     V2_R <= (127 * rgb_i[128 +: 16]);
170     V2_G <= (106 * rgb_i[64 +: 16]);
171     V2_B <= ( 21 * rgb_i[48 +: 16]);
172
173     Y2_ADD <= Y2_R + Y2_G;
174
175     U2_ADD <= U2_R - U2_G;
176
177     V2_ADD <= V2_R - V2_G;
178
179     Y2_ADD_STAGE2 <= Y2_ADD + Y2_B;
180     Y2_ADD_STAGE3 <= (Y2_ADD_STAGE2 + 18'd128) >> 16;
181     Y2_ADD_STAGE4 <= Y2_ADD_STAGE3;
182
183     U2_ADD_STAGE2 <= U2_ADD - U2_B;
184     U2_ADD_STAGE3 <= (U2_ADD_STAGE2 + 18'd128) >> 16;
185     U2_ADD_STAGE4 <= U2_ADD_STAGE3 + 32'd128;
186
187     V2_ADD_STAGE2 <= V2_ADD - V2_B;
188     V2_ADD_STAGE3 <= (V2_ADD_STAGE2 + 18'd128) >> 16;
189     V2_ADD_STAGE4 <= V2_ADD_STAGE3 + 32'd128;
190
191     {not_used24,Y[2]} = Y2_ADD_STAGE4;
192     {not_used24,U[2]} = U2_ADD_STAGE4;
193     {not_used24,V[2]} = V2_ADD_STAGE4;
194
195     Y3_R <= ( 77 * rgb_i[32 +: 16]);
196     Y3_G <= (150 * rgb_i[16 +: 16]);
197     Y3_B <= ( 29 * rgb_i[0 +: 16]);
198
199     Y3_ADD <= Y3_R + Y3_G;
200     Y3_ADD_STAGE2 <= Y3_ADD + Y3_B;
201     Y3_ADD_STAGE3 <= (Y3_ADD_STAGE2 + 18'd128) >> 16;
202     Y3_ADD_STAGE4 <= Y3_ADD_STAGE3;
203
204     {not_used24,Y[3]} = Y3_ADD_STAGE4;
205     //U[3] and V[3] not need to yuv422 sub sampling
206
207     yuv_o = { Y[0], U[0], Y[1], V[0], Y[2],
208     U[2], Y[3], V[2]};
209
210 end
211
212 endmodule

```

10. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/ FPGA_Source/source/mipi_csi_bridge/tb_debayer_filter.v (326 lines)

```

1 `timescale 1ns/1ns
2
3 module tb_debayer_filter();
4     reg clk;
5     reg reset;
6     reg line_valid;
7     wire [63:0]bytes_o;
8     wire unpacked_valid;
9     wire [119:0]rgb_output;
10    wire output_valid;
11    wire [63:0] yuv_data;
12    wire is_yuv_valid;
13
14    wire [31:0]data_out;
15    wire lsync_out;
16    reg out_clock;
17    reg frame_sync;
18    wire reset_g;
19    //GSR GSR_INST (.GSR (reset_g));
20    PUR PUR_INST (.PUR (reset_g));
21
22    GSR
23    GSR_INST (
24        .GSR_N(1'b1),
25        .CLK(1'b0)
26    );
27
28    debayer_filter ins1( .clk_i(clk),
29
30    .re-
31    set_i(reset),
32    id_i(line_valid),
33    .data_i(bytes_o
34    ),
35    .data_val-
36    id_i(unpacked_valid),
37    .out-
38    put_valid_o(output_valid),

```

```

34      .out-
put_o(rgb_output));
35
36      rgb_to_yuv rgb_to_yuv1(.clk_i(clk),
37
38      .reset_i(reset),
39      .rgb_i(rgb_output),
40      .rgb_val-
id_i(output_valid),
41
42      .yuv_o(yuv_data),
43      .yuv_val-
id_o(is_yuv_valid));
44
45      output_reformatter out_reformat1(
46
47      .clk_i(clk),
48
49      .line_sync_i(line_valid),
50
51      .output_clk_i(out_clock),
52
53      .data_i(yuv_data),
54
55      .data_in_valid_i(is_yuv_valid),
56
57      .output_o(data_out),
58
59      .output_valid_o(lsync_out),
60
61      .frame_sync_i(frame_sync));
62
63      53 reg bytes_valid;
64      54 reg [31:0]bytes_i;
65      55 wire synced;
66
67      56
68      57
69      58
70      59 mipi_rx_raw_depacker ins2(
71      .clk_i(clk),
72
73      .pack-
et_type_i(3'd3),
74
75      .data_val-
id_i(bytes_valid),
76
77      .data_i(bytes_i
78      ),
79
80      .out-
put_valid_o(unpacked_valid),
81
82      .out-
put_o(bytes_o));
83
84      66 task sendbytes;
85      67     input [31:0]bytes;
86      68     begin
87      69         bytes_i = bytes;
88      70         clk = 1'b0;
89      71         out_clock = 1'b0;
90      72         #10
91      73         out_clock = 1'b1;
92      74         #10
93      75         out_clock = 1'b0;
94      76         #10
95      77         out_clock = 1'b1;
96      78         #10
97      79         out_clock = 1'b0;
98      80         clk = 1'b1;
99      81         #10
100     82         out_clock = 1'b1;
101     83         #10;
102     84         end
103     endtask
104
105     initial begin
106
107         clk = 1'b0;
108         out_clock = 1'b0;
109         line_valid = 1'h0;
110         bytes_valid = 4'h0;
111         frame_sync = 0;
112         reset = 1'h1;
113         sendbytes(32'h0);
114         reset = 1'h0;
115         #50
116         frame_sync = 1;
117         #10;
118         sendbytes(32'h0);
119         sendbytes(32'h00000000);
120         sendbytes(32'h00000000);
121         sendbytes(32'h00000000);
122         sendbytes(32'h00000000);
123         sendbytes(32'h00000000);
124         sendbytes(32'h00000000);
125         sendbytes(32'h00000000);
126         bytes_valid = 1'h1;
127         line_valid = 1'h1;
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209

```



```

210      sendbytes(32'h9ABC00F0);
211      sendbytes(32'hBBBBB800);
212
213      sendbytes(32'h12345678);
214      sendbytes(32'h00BCDEF0);
215      sendbytes(32'h12005678);
216      sendbytes(32'h9ABC00F0);
217      sendbytes(32'hAAAAA00);
218
219      sendbytes(32'h12345678);
220      sendbytes(32'h00BCDEF0);
221      sendbytes(32'h12005678);
222      sendbytes(32'h9ABC00F0);
223      sendbytes(32'hDDDDDD00);
224      bytes_valid = 1'h0;
225      line_valid = 1'h0;
226
227      send-
bytes(32'h00000000);
228      sendbytes(32'h00000000);
229      sendbytes(32'h00000000);
230      sendbytes(32'h00000000);
231      sendbytes(32'h00000000);
232      sendbytes(32'h00000000);
233      sendbytes(32'h00000000);
234      sendbytes(32'h00000000);
235      sendbytes(32'h0);
236      sendbytes(32'h0);
237      bytes_valid = 1'h1;
238      line_valid = 1'h1;
239      sendbytes(32'h12345678);
240      sendbytes(32'h00BCDEF0);
241      sendbytes(32'h12005678);
242      sendbytes(32'h9ABC00F0);
243      sendbytes(32'hBBBBB800);
244
245      sendbytes(32'h12345678);
246      sendbytes(32'h00BCDEF0);
247      sendbytes(32'h12005678);
248      sendbytes(32'h9ABC00F0);
249      sendbytes(32'hAAAAA00);
250
251      sendbytes(32'h12345678);
252      sendbytes(32'h00BCDEF0);
253      sendbytes(32'h12005678);
254      sendbytes(32'h9ABC00F0);
255      sendbytes(32'hDDDDDD00);
256      bytes_valid = 1'h0;
257      line_valid = 1'h0;
258
259
sendbytes(32'h00000000);
260      sendbytes(32'h00000000);
261      sendbytes(32'h00000000);
262      sendbytes(32'h00000000);
263      sendbytes(32'h00000000);
264      sendbytes(32'h00000000);
265      sendbytes(32'h00000000);
266      sendbytes(32'h00000000);
267      sendbytes(32'h0);
268      sendbytes(32'h0);
269      bytes_valid = 1'h1;
270      line_valid = 1'h1;
271      sendbytes(32'h12345678);
272      sendbytes(32'h00BCDEF0);
273      sendbytes(32'h12005678);
274      sendbytes(32'h9ABC00F0);
275      sendbytes(32'hBBBBB800);
276
277      sendbytes(32'h12345678);
278      sendbytes(32'h00BCDEF0);
279      sendbytes(32'h12005678);
280      sendbytes(32'h9ABC00F0);
281      sendbytes(32'hAAAAA00);
282
283      sendbytes(32'h12345678);
284      sendbytes(32'h00BCDEF0);
285      sendbytes(32'h12005678);
286      sendbytes(32'h9ABC00F0);
287      sendbytes(32'hDDDDDD00);
288      bytes_valid = 1'h0;
289      line_valid = 1'h0;
290
291
sendbytes(32'h00000000);
292      sendbytes(32'h00000000);
293      sendbytes(32'h00000000);
294      sendbytes(32'h00000000);
295      sendbytes(32'h00000000);
296      sendbytes(32'h00000000);
297      sendbytes(32'h00000000);
298      sendbytes(32'h00000000);
299      sendbytes(32'h0);
300      sendbytes(32'h0);
301      bytes_valid = 1'h1;
302      line_valid = 1'h1;
303      sendbytes(32'h12345678);
304      sendbytes(32'h00BCDEF0);
305      sendbytes(32'h12005678);

```

```

306      sendbytes(32'h9ABC00F0);
307      sendbytes(32'hBBBBB800);
308
309      sendbytes(32'h12345678);
310      sendbytes(32'h00BCDEF0);
311      sendbytes(32'h12005678);
312      sendbytes(32'h9ABC00F0);
313      sendbytes(32'hAAAAA00);
314
315      sendbytes(32'h12345678);
316      sendbytes(32'h00BCDEF0);
317      sendbytes(32'h12005678);
318      sendbytes(32'h9ABC00F0);
319      sendbytes(32'hDDDDDD00);
320      bytes_valid = 1'h0;
321      line_valid = 1'h0;
322      #10;
323      frame_sync = 0;
324      end
325
326      endmodule

```

11. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/ FPGA_Source/source/mipi_csi_bridge/tb_mipi_bridge.v (334 lines)

```

1 `timescale 1ns/1ns
2
3 module tb_mipi_bridge();
4     reg clk;
5     reg reset;
6     reg mipi_clk;
7     wire clk_lpp;
8     wire clk_lpn;
9     wire [3:0]data_lpp;
10    wire [3:0]data_lpn;
11
12    wire mipi_clk_n;
13    wire [3:0]mipi_data_n;
14    reg clk_lpp_r;
15    reg clk_lpn_r;
16    reg [3:0]data_lpp_r;
17    reg [3:0]data_lpn_r;
18
19    reg [3:0]mipi_data;
20
21    wire pclk;
22    wire [31:0]data;
23    wire fsync;
24    wire lsync;
25    wire reset_g;
26    GSR
27    GSR_INST (
28        .GSR_N(1'b1),
29        .CLK(1'b0)
30    );
31
32
33    assign mipi_clk_n = !mipi_clk;
34    assign mipi_data_n = ~mipi_data;
35
36    mipi_bridge ins1(
37        .reset_in(reset),
38        .mipi_clk_n_in(mipi_clk_n),
39        .mipi_clk_p_in(mipi_clk),
40        .mipi_data_p_in(mipi_data),
41        .mipi_data_n_in(mipi_data_n),
42
43        .pclk_o(pclk),
44        .data_o(data),
45        .fsync_o(fsync),
46        .lsync_o(lsync),
47        .frame_sync_in(!
48            .line_sync_in(data_lpn_r[0])
49        );
50
51
52    initial be-
gin
53    //generate 90 phase clock and slow sync clock
54    clk = 1'b0;
55    end
56    always begin
57        #6 clk = ~clk;
58    end
59    task send_mipi_frame;
60        reg [15:0]i= 16'd0;
61        begin
62            clk_lpp_r = 0;
63            clk_lpn_r = 0;
64            #5
65            send_mipi_clock();

```

```

66     send_mipi_clock();
67     send_mipi_clock();
68     send_mipi_clock();
69     send_mipi_clock();
70     send_mipi_clock();
71     send_mipi_clock();
72     send_mipi_clock();
73     send_mipi_clock();
74     send_mipi_clock();
75     for (i= 16'd0; i <16'd10 ; i = i +1'd1)
76     begin
77         send_line();
78     send_mipi_clock();
79         send_mipi_clock();
80         send_mipi_clock();
81         send_mipi_clock();
82         send_mipi_clock();
83         send_mipi_clock();
84         #20;
85     end
86     send_mipi_clock();
87     send_mipi_clock();
88     send_mipi_clock();
89     send_mipi_clock();
90     send_mipi_clock();
91     send_mipi_clock();
92     send_mipi_clock();
93     send_mipi_clock();
94     send_mipi_clock();
95     #5
96     clk_lpp_r = 1;
97     clk_lpn_r = 1;
98
99 end
100 endtask
101
102
103 task send_mipi_frame12;
104     reg [15:0]i= 16'd0;
105     begin
106         clk_lpp_r = 0;
107         clk_lpn_r = 0;
108         #5
109         send_mipi_clock();
110         send_mipi_clock();
111         send_mipi_clock();
112         send_mipi_clock();
113         send_mipi_clock();
114         send_mipi_clock();
115         send_mipi_clock();
116         send_mipi_clock();
117         send_mipi_clock();
118         send_mipi_clock();
119         for (i= 16'd0; i <16'd10 ; i = i +1'd1)
120         begin
121             send_line12();
122             #20;
123         end
124         send_mipi_clock();
125         send_mipi_clock();
126         send_mipi_clock();
127         send_mipi_clock();
128         send_mipi_clock();
129         send_mipi_clock();
130         send_mipi_clock();
131         send_mipi_clock();
132         send_mipi_clock();
133         #5
134         clk_lpp_r = 1;
135         clk_lpn_r = 1;
136
137     end
138 endtask
139
140 task send_mipi_frame14;
141     reg [15:0]i= 16'd0;
142     begin
143         clk_lpp_r = 0;
144         clk_lpn_r = 0;
145         #5
146         send_mipi_clock();
147         send_mipi_clock();
148         send_mipi_clock();
149         send_mipi_clock();
150         send_mipi_clock();
151         send_mipi_clock();
152         send_mipi_clock();
153         send_mipi_clock();
154         send_mipi_clock();
155         send_mipi_clock();
156         for (i= 16'd0; i <16'd10 ; i = i +1'd1)
157         begin
158             send_line14();
159             #20;
160         end
161         send_mipi_clock();
162         send_mipi_clock();
163         send_mipi_clock();
164         send_mipi_clock();
165         send_mipi_clock();
166         send_mipi_clock();

```

```

167     send_mipi_clock();
168     send_mipi_clock();
169     send_mipi_clock();
170     #5
171     clk_lpp_r = 1;
172     clk_lpn_r = 1;
173
174     end
175 endtask
176
177 task send_line14;
178     reg[15:0]i = 16'b0;
179     begin
180
181
182         send_mipi_clock();
183         send_mipi_clock();
184         send_mipi_clock();
185         data_lpp_r = 4'b0;
186         send_mipi_clock();
187         send_mipi_clock();
188         send_mipi_clock();
189         data_lpn_r = 4'b0;
190         send_mipi_byte(32'hB8B8B8B8);
191         send_mipi_byte(32'h2D200D00);
192         for (i= 16'b0; i < 16'd120 ; i = i+1)
193         begin
194             send_mipi_byte(32'h00FF55AA);
195             send_mipi_byte(32'h00000000);
196             send_mipi_byte(32'hFF55AA00);
197             send_mipi_byte(32'h000000FF);
198             send_mipi_byte(32'h55AA0000);
199             send_mipi_byte(32'h0000FF55);
200             send_mipi_byte(32'hAA000000);
201
202         end
203         send_mipi_clock();
204         send_mipi_clock();
205         send_mipi_clock();
206         send_mipi_clock();
207         send_mipi_clock();
208         send_mipi_clock();
209         data_lpp_r = 4'b1111;
210         data_lpn_r = 4'b1111;
211
212     end
213 endtask
214
215 task send_line12;
216     reg[15:0]i = 16'b0;
217     begin
218
219
220         send_mipi_clock();
221         send_mipi_clock();
222         send_mipi_clock();
223         data_lpp_r = 4'b0;
224         send_mipi_clock();
225         send_mipi_clock();
226         send_mipi_clock();
227         data_lpn_r = 4'b0;
228         send_mipi_byte(32'hB8B8B8B8);
229         send_mipi_byte(32'h2C400B00);
230         for (i= 16'b0; i < 16'd120 ; i = i+1)
231         begin
232             send_mipi_byte(32'h00FF0055);
233             send_mipi_byte(32'hAA0000FF);
234             send_mipi_byte(32'h0055AA00);
235             send_mipi_byte(32'h00FF0055);
236             send_mipi_byte(32'hAA0000FF);
237             send_mipi_byte(32'h0055AA00);
238
239         end
240         send_mipi_clock();
241         send_mipi_clock();
242         send_mipi_clock();
243         send_mipi_clock();
244         send_mipi_clock();
245         send_mipi_clock();
246         data_lpp_r = 4'b1111;
247         data_lpn_r = 4'b1111;
248
249     end
250 endtask
251
252 task send_line;
253     reg[15:0]i = 16'b0;
254     begin
255
256
257         send_mipi_clock();
258         send_mipi_clock();
259         send_mipi_clock();
260         data_lpp_r = 4'b0;
261         send_mipi_clock();
262         send_mipi_clock();
263         send_mipi_clock();
264         data_lpn_r = 4'b0;
265         send_mipi_byte(32'hB8B8B8B8);
266         send_mipi_byte(32'h2B009000);
267         for (i= 16'b0; i < 16'd120 ; i = i+1)

```

```

268         begin
269             send_mipi_byte(32'h00FF55AA); //MSB to
lane 0 and LSB to lane 3
270             send_mipi_byte(32'h0000FF55);
271             send_mipi_byte(32'hAA0000FF);
272             send_mipi_byte(32'h55AA0000);
273             send_mipi_byte(32'hFF55AA00);
274         end
275         send_mipi_byte(32'h33333333);
276         //send_mipi_clock();
277         send_mipi_byte(32'h12345678);
278         send_mipi_clock();
279         send_mipi_clock();
280         send_mipi_clock();
281         send_mipi_clock();
282         send_mipi_clock();
283         data_lpp_r = 4'b1111;
284         data_lpn_r = 4'b1111;
285     end
286 endtask
287
288 task send_mipi_byte;
289     input [31:0]bytes;
290     reg [7:0] i = 8'b0;
291     begin
292         for(i = 8'b0; i < 8'h8; i = i+2) begin
293             #1;
294             mipi_data = {bytes[i], bytes[8'd8 +
i] ,bytes[8'd16 + i] , bytes[8'd24 + i]};
295             #1;
296             mipi_clk = 1'b1;
297             #1;
298             mipi_data = {bytes[i+1], bytes[8'd8 + i
+ 1] ,bytes[8'd16 + i + 1] , bytes[8'd24 + i + 1]};
299             #1;
300             mipi_clk = 1'b0;
301         end
302     end
303 endtask
304
305 task send_mipi_clock;
306     begin
307         send_mipi_byte(32'h00000000);
308     end
309 endtask
310
311 initial begin
312     mipi_data = 0;
313     mipi_clk = 0;
314     data_lpp_r = 4'b1111;
315     data_lpn_r = 4'b1111;
316     reset = 0;
317     clk_lpp_r = 1;
318     clk_lpn_r = 1;
319     #20
320     reset = 1;
321     #200
322     send_mipi_frame();
323     #200
324     send_mipi_frame();
325     #200
326     send_mipi_frame14();
327     #200
328     send_mipi_frame12();
329     $finish;
330 end
331
332 endmodule

```

12. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/ FPGA_Source/source/mipi_csi_bridge/ tb_mipi_csi_packet_decoder.v (76 lines)

```

1 `timescale 1ns/1ns
2
3 module tb_mipi_csi_packet_decoder;
4     reg clk;
5     reg reset;
6     reg input_valid;
7     reg [31:0]bytes_i;
8     wire [31:0]bytes_o;
9     wire synced;
10    wire [31:0]packet_length;
11    wire [2:0]packet_type;
12
13    wire reset_g;
14
15    GSR
16    GSR_INST (
17        .GSR_N(1'b1),
18        .CLK(1'b0)
19    );
20
21    mipi_csi_packet_decoder dec1(.clk_i(clk),
22                                .dat
a_valid_i(input_valid),
23                                .dat

```

```

a_i(bytes_i),
24    put_valid_o(synced),
25    .dat
a_o(bytes_o),
26    .pac
ket_length_o(packet_length),
27    .pac
ket_type_o(packet_type));
28
29    task sendbytes;
30        input [31:0]bytes;
31        begin
32            bytes_i = bytes;
33            clk = 1'b1;
34            #4
35            clk = 1'b0;
36            #4;
37        end
38    endtask
39
40    task sendpacket;
41        reg [31:0]i;
42        for ( i= 32'b0; i < 32'h980; i = i + 4)
43            begin
44                sendbytes(i*10000);
45            end
46    endtask
47
48    initial begin
49        clk = 1'b0;
50        input_valid = 1'h0;
51        #50
52        reset = 1'b0;
53        sendbytes(32'h00000000);
54        sendbytes(32'h00000000);
55        sendbytes(32'h00000000);
56        sendbytes(32'h00000000);
57        input_valid = 1'h1;
58        sendbytes(32'h00000000);
59        sendbytes(32'h00000000);
60        sendbytes(32'h88888888);
61        sendbytes(32'hAB09602B);
62        sendpacket();
63        input_valid = 1'h0;
64        sendbytes(32'h00000000);
65        sendbytes(32'h00000000);
66        sendbytes(32'h00000000);
67        sendbytes(32'h00000000);
68        sendbytes(32'h00000000);
69        sendbytes(32'h00000000);
70        sendbytes(32'h00000000);
71        sendbytes(32'h00000000);
72
73        reset = 1'b1;
74    end
75
76 endmodule

```

13. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/ FPGA_Source/source/mipi_csi_bridge/ tb_mipi_rx_raw_depacker.v (139 lines)

```

1 `timescale 1ns/1ns
2
3 module tb_mipi_rx_raw_depacker();
4
5     reg clk;
6     reg bytes_valid;
7     reg [31:0]bytes_i;
8     wire [39:0]bytes_o;
9     wire synced;
10    reg [2:0]packet_type;
11    wire reset_g;
12
13    GSR
14    GSR_INST (
15        .GSR_N(1'b1),
16        .CLK(1'b0)
17    );
18
19
20    mipi_rx_raw_depacker ins1(
21        .clk_i(clk),
22        .data_val-
id_i(bytes_valid),
23        .data_i(bytes_i
),
24        .pack-
et_type_i(packet_type),
25        .out-
put_valid_o(synced),
26        .out-
put_o(bytes_o));
27
28    task sendbytes;
29        input [31:0]bytes;
30        begin
31            bytes_i = bytes;
32            clk = 1'b0;

```

```

32      #4
33      clk = 1'b1;
34      #4;
35      end
36  endtask
37
38  initial begin
39      clk = 1'b0;
40      bytes_valid = 4'h0;
41      packet_type = 3;
42      #50
43      sendbytes(32'h0);
44      sendbytes(32'h0);
45      bytes_valid = 1'h1;
46      sendbytes(32'h12345678);
47      sendbytes(32'h00BCDEF0);
48      sendbytes(32'h12005678);
49      sendbytes(32'h9ABC00F0);
50      sendbytes(32'hBBBBB000);
51
52      sendbytes(32'h12345678);
53      sendbytes(32'h00BCDEF0);
54      sendbytes(32'h12005678);
55      sendbytes(32'h9ABC00F0);
56      sendbytes(32'hAAAAA000);
57
58      sendbytes(32'h12345678);
59      sendbytes(32'h00BCDEF0);
60      sendbytes(32'h12005678);
61      sendbytes(32'h9ABC00F0);
62      sendbytes(32'hDDDDDD00);
63      bytes_valid = 1'h0;
64      sendbytes(32'h00000000);
65      sendbytes(32'h00000000);
66      sendbytes(32'h00000000);
67      sendbytes(32'h00000000);
68      sendbytes(32'h00000000);
69      sendbytes(32'h00000000);
70      sendbytes(32'h00000000);
71      sendbytes(32'h00000000);
72
73      packet_type = 4;
74
75      sendbytes(32'h0);
76      sendbytes(32'h0);
77      bytes_valid = 1'h1;
78      sendbytes(32'h12345678);
79      sendbytes(32'h00BCDEF0);
80      sendbytes(32'h12005678);
81      sendbytes(32'h9ABC00F0);
82      sendbytes(32'hBBBBB000);
83
84      sendbytes(32'h12345678);
85      sendbytes(32'h00BCDEF0);
86      sendbytes(32'h12005678);
87      sendbytes(32'h9ABC00F0);
88      sendbytes(32'hAAAAA000);
89
90      sendbytes(32'h12345678);
91      sendbytes(32'h00BCDEF0);
92      sendbytes(32'h12005678);
93      sendbytes(32'h9ABC00F0);
94      sendbytes(32'hDDDDDD00);
95      bytes_valid = 1'h0;
96      sendbytes(32'h00000000);
97      sendbytes(32'h00000000);
98      sendbytes(32'h00000000);
99      sendbytes(32'h00000000);
100     sendbytes(32'h00000000);
101     sendbytes(32'h00000000);
102     sendbytes(32'h00000000);
103     sendbytes(32'h00000000);
104
105     packet_type = 5;
106
107     sendbytes(32'h0);
108     sendbytes(32'h0);
109     bytes_valid = 1'h1;
110     sendbytes(32'h12345678);
111     sendbytes(32'h00BCDEF0);
112     sendbytes(32'h12005678);
113     sendbytes(32'h9ABC00F0);
114     sendbytes(32'hBBBBB000);
115
116     sendbytes(32'h12345678);
117     sendbytes(32'h00BCDEF0);
118     sendbytes(32'h12005678);
119     sendbytes(32'h9ABC00F0);
120     sendbytes(32'hAAAAA000);
121
122     sendbytes(32'h12345678);
123     sendbytes(32'h00BCDEF0);
124     sendbytes(32'h12005678);
125     sendbytes(32'h9ABC00F0);
126     sendbytes(32'hDDDDDD00);
127     bytes_valid = 1'h0;
128     sendbytes(32'h00000000);
129     sendbytes(32'h00000000);
130     sendbytes(32'h00000000);
131     sendbytes(32'h00000000);
132     sendbytes(32'h00000000);

```

```

133     sendbytes(32'h00000000);
134     sendbytes(32'h00000000);
135     sendbytes(32'h00000000);
136
137     $finish;
138 end
139 endmodule

```

14. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/ FPGA_Source/source/mipi_csi_bridge/tb_rx_byte_aligner.v (164 lines)

```

1  `timescale 1ns/1ns
2
3  module tb_rx_byte_aligner;
4
5      reg clk;
6      reg reset;
7      reg [7:0]byte_i;
8      wire [7:0]byte_o;
9      wire synced;
10
11  wire reset_g;
12  GSR
13  GSR_INST (
14      .GSR_N(1'b1),
15      .CLK(1'b0)
16  );
17
18  mipi_rx_byte_aligner inst1(
19      .clk_i(clk),
20      .re-
21      .byte_i(byte_i)
22      .byte_o(byte_o)
23      .byte_val-
24      id_o(synced));
25
26  task sendbyte;
27      input [7:0]byte;
28      begin
29          byte_i = byte;
30          clk = 1'b1;
31          #4
32          clk = 1'b0;
33          #4;
34      end
35  endtask
36
37  initial begin
38      clk = 1'b1;
39      reset = 1'b1;
40      clk = 1'b1;
41      #4
42      clk = 1'b0;
43      #4;
44      #50
45      reset = 1'b0;
46      sendbyte(8'h00);
47      sendbyte(8'h00);
48      sendbyte(8'h00);
49      sendbyte(8'h00);
50      sendbyte(8'h77); //B8 2B 11
51      sendbyte(8'h25);
52      sendbyte(8'h42);
53      sendbyte(8'hCE);
54      sendbyte(8'h22);
55      sendbyte(8'h22);
56      sendbyte(8'h22);
57      sendbyte(8'h62);
58      sendbyte(8'h30);
59      sendbyte(8'h22);
60      sendbyte(8'h02);
61      reset = 1'h1;
62      #5
63      clk = 1'b1;
64      #4
65      clk = 1'b0;
66      #4;
67      #50
68
69      reset = 1'b0;
70      sendbyte(8'h00);
71      sendbyte(8'h00);
72      sendbyte(8'h00);
73      sendbyte(8'h00);
74      sendbyte(8'h70); //B8 20 50 11
75      sendbyte(8'h41);
76      sendbyte(8'hA0);
77      sendbyte(8'h22);
78      sendbyte(8'h72);
79      sendbyte(8'h22);
80      sendbyte(8'h22);
81      sendbyte(8'h22);
82      sendbyte(8'h3A);
83      sendbyte(8'h22);

```

```

84      sendbyte(8'h22);
85      reset = 1'h1;
86      #5
87      clk = 1'b1;
88      #4
89      clk = 1'b0;
90      #4;
91
92      #50
93      reset = 1'b0;
94      sendbyte(8'h00);
95      sendbyte(8'h00);
96      sendbyte(8'h00);
97      sendbyte(8'h00);
98      sendbyte(8'h00);
99      sendbyte(8'h5C); //B8 2A 11
100     sendbyte(8'h95);
101     sendbyte(8'h08);
102     sendbyte(8'h1F);
103     sendbyte(8'h08);
104     sendbyte(8'h08);
105     sendbyte(8'h88);
106     sendbyte(8'h08);
107     sendbyte(8'h17);
108     sendbyte(8'h08);
109     sendbyte(8'h08);
110     reset = 1'h1;
111     #5
112     clk = 1'b1;
113     #4
114     clk = 1'b0;
115     #4;
116
117     #50
118     reset = 1'b0;
119     sendbyte(8'h00);
120     sendbyte(8'h00);
121     sendbyte(8'h00);
122     sendbyte(8'h00);
123     sendbyte(8'h00);
124     sendbyte(8'h5C); //B8 60 10
125     sendbyte(8'h30);
126     sendbyte(8'h88);
127     sendbyte(8'h08);
128     sendbyte(8'hA9);
129     sendbyte(8'h88);
130     sendbyte(8'h88);
131     sendbyte(8'h08);
132     sendbyte(8'h17);
133     sendbyte(8'h08);
134     sendbyte(8'h08);
135     reset = 1'h1;
136     #5
137     clk = 1'b1;
138     #4
139     clk = 1'b0;
140     #4;
141
142     #50
143     reset = 1'b0;
144     sendbyte(8'h00);
145     sendbyte(8'h00);
146     sendbyte(8'h00);
147     sendbyte(8'h00);
148     sendbyte(8'h00);
149     sendbyte(8'hE0); //B8 60 11
150     sendbyte(8'h82);
151     sendbyte(8'h45);
152     sendbyte(8'h40);
153     sendbyte(8'hC4);
154     sendbyte(8'h45);
155     sendbyte(8'h40);
156     sendbyte(8'h08);
157     sendbyte(8'h17);
158     sendbyte(8'h08);
159     sendbyte(8'h08);
160     reset = 1'h1;
161
162 end
163
164 endmodule

```

15. circuitvalley_USB3_MIPI_CSI2_RX_V2_Crosslink_NX/ FPGA_Source/source/mipi_csi_bridge/tb_rx_lane_aligner.v (107 lines)

```

1  `timescale 1ns/1ns
2
3  module tb_rx_lane_aligner;
4      reg clk;
5      reg [3:0]bytes_valid;
6      reg [31:0]bytes_i;
7      wire [31:0]bytes_o;
8      wire synced;
9      reg reset;
10
11  wire reset_g;
12  GSR
13  GSR_INST (

```

```

14      .GSR_N(1'b1),
15      .CLK(1'b0)
16  );
17  mipi_rx_lane_aligner ins1(      .clk_i(clk),
18
19  set_i(reset),
20
21  bytes_valid_i(
22
23  ),
24  id_o(synced),
25  bytes_o));
26
27  task sendbytes;
28      input [31:0]bytes;
29      begin
30          bytes_i = bytes;
31          clk = 1'b0;
32          #4
33          clk = 1'b1;
34          #4;
35      end
36  endtask
37
38  initial begin
39      reset = 1'b0;
40      clk = 1'b0;
41      bytes_valid = 4'h0;
42      #50
43
44      sendbytes(32'h00000000);
45      reset = 1'h1;
46      sendbytes(32'h00000000);
47      reset = 1'h0;
48      sendbytes(32'h00000000);
49      sendbytes(32'h00000000);
50      bytes_valid[1] = 1'h1;
51      sendbytes(32'h0000B800);
52      sendbytes(32'h00001100);
53      sendbytes(32'h00002200);
54      sendbytes(32'h00003300);
55      bytes_valid[2] = 1'h1;
56      bytes_valid[3] = 1'h1;
57      sendbytes(32'hB8B84400);
58      bytes_valid[0] = 1'h1;
59      sendbytes(32'h111155B8);
60      sendbytes(32'h22226611);
61      sendbytes(32'h33337722);
62      sendbytes(32'h44448833);
63      bytes_valid[1] = 1'h0;
64      sendbytes(32'h55556644);
65      sendbytes(32'h66667755);
66      sendbytes(32'h77778866);
67
68      sendbytes(32'h88889977);
69      bytes_valid[2] = 1'h0;
70      bytes_valid[3] = 1'h0;
71
72      sendbytes(32'h9999AA88);
73      bytes_valid[0] = 1'h0;
74      sendbytes(32'h00000000);
75      sendbytes(32'h00000000);
76      sendbytes(32'h00000000);
77      sendbytes(32'h00000000);
78      sendbytes(32'h00000000);
79      sendbytes(32'h00000000);
80      bytes_valid[1] = 1'h1;
81      sendbytes(32'h0000B800);
82      sendbytes(32'h00001100);
83      bytes_valid[2] = 1'h1;
84      bytes_valid[3] = 1'h1;
85      sendbytes(32'hB8B82200);
86      bytes_valid[0] = 1'h1;
87      sendbytes(32'h111133B8);
88      sendbytes(32'h22224411);
89      sendbytes(32'h33335522);
90      sendbytes(32'h44446633);
91      sendbytes(32'h55557744);
92      sendbytes(32'h66668855);
93      bytes_valid[1] = 1'h0;
94      sendbytes(32'h77778866);
95
96
97      sendbytes(32'h88889977);
98      bytes_valid[2] = 1'h0;
99      bytes_valid[3] = 1'h0;
100
101      sendbytes(32'h9999AA88);
102      bytes_valid[0] = 1'h0;
103
104  end
105
106 endmodule

```

MIPI CSI-2 수신기 (1세대)

What this is. An earlier generation of the same receiver, targeting a different FPGA family.

Why it exists. Because comparing two implementations of one specification is instructive. The differences show which parts are forced by the standard and which are choices — the distinction a specification writer must be able to make.

Undergraduate review · EECS concepts. Design portability; the boundary between specification and implementation.

Graduate · Practice note. A useful exercise: list the differences between the two versions and classify each as required by the standard, forced by the target device, or a free choice. That classification is the substance of a good specification document.

17 files • 2,586 lines

API — debayer_filter.v

Table 573. circuitvalley_mipi_csi_receiver_FPGA/FPGA/debayer_filter.v — module / interface / package (1개)

이름	종류
debayer_filter	module

Table 574. circuitvalley_mipi_csi_receiver_FPGA/FPGA/debayer_filter.v — localparam (2개)

상수	형/폭	값
INPUT_WIDTH		40
OUTPUT_WIDTH		120

Table 575. circuitvalley_mipi_csi_receiver_FPGA/FPGA/debayer_filter.v — 포트 (8개)

포트	방향	형/폭
clk_i	input	
reset_i	input	
line_valid_i	input	
data_valid_i	input	
1	input	[(INPUT_WIDTH -
output_valid_o	output	reg
1	output	reg [(OUTPUT_WIDTH-
debug_out	output	reg [7:0]

API — line_ram_dp.v

Table 576. circuitvalley_mipi_csi_receiver_FPGA/FPGA/line_ram_dp.v — module / interface / package (1개)

이름	종류
line_ram_dp	module

Table 577. circuitvalley_mipi_csi_receiver_FPGA/FPGA/line_ram_dp.v — 포트 (10개)

포트	방향	형/폭
WrAddress	input	wire [9:0]
RdAddress	input	wire [9:0]
Data	input	wire [39:0]
WE	input	wire
RdClock	input	wire
RdClockEn	input	wire
Reset	input	wire
WrClock	input	wire
WrClockEn	input	wire
Q	output	wire [39:0]

API — mipi_bridge.v**Table 578. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_bridge.v — module / interface / package (1개)**

이름	종류
mipi_bridge	module

Table 579. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_bridge.v — parameter (1개)

파라미터	형/폭	기본값
MIPI_LANES		4

Table 580. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_bridge.v — 포트 (13개)

포트	방향	형/폭
clk_i	input	
reset_in	input	
mipi_clk_in	input	
mipi_clk_lpp_io	inout	
mipi_clk_lpn_io	inout	
mipi_data_in	input	[MIPI_LANES-1:0]
mipi_data_lpp_io	inout	[MIPI_LANES-1:0]
mipi_data_lpn_io	inout	[MIPI_LANES-1:0]
pclk_out	output	
data_out	output	[31:0]
fsyn_out	output	
lsync_out	output	
debug_b8	output	wire [7:0]

API — mipi_csi_packet_decoder.v**Table 581. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_csi_packet_decoder.v — module / interface / package (1개)**

이름	종류
mipi_csi_packet_decoder	module

Table 582. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_csi_packet_decoder.v — localparam (3개)

상수	형/폭	값
SYNC_BYTE	[7:0]	8'hB8
LANES	[3:0]	3'h4
MIPI_CSI_PACKET_10bRAW	[7:0]	8'h2B

Table 583. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_csi_packet_decoder.v — 포트 (6개)

포트	방향	형/폭
clk_i	input	
data_valid_i	input	
data_i	input	[31:0]
output_valid_o	output	reg
data_o	output	reg [31:0]
packet_length	output	reg [31:0]

API — mipi_rx_byte_aligner.v**Table 584. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_rx_byte_aligner.v — module / interface / package (1개)**

이름	종류
mipi_rx_byte_aligner	module

Table 585. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_rx_byte_aligner.v — localparam (1개)

상수	형/폭	값
SYNC_BYTE	[7:0]	8'hB8

Table 586. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_rx_byte_aligner.v — 포트 (5개)

포트	방향	형/폭
clk_i	input	
reset_i	input	
byte_i	input	[7:0]
byte_o	output	reg [7:0]
byte_valid_o	output	reg

API — mipi_rx_dds.v**Table 587. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_rx_dds.v — module / interface / package (1개)**

이름	종류
mipi_rx_dds	module

Table 588. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_rx_dds.v — 포트 (26개)

포트	방향	형/폭
alignwd	input	wire
buf_clk_lp0o	input	wire
buf_clk_lp0t	input	wire
buf_clk_lp1o	input	wire
buf_clk_lp1t	input	wire
clk	input	wire
clk_s	input	wire
init	input	wire
reset	input	wire
buf_data_lp0o	input	wire [3:0]
buf_data_lp0t	input	wire [3:0]
buf_data_lp1o	input	wire [3:0]
buf_data_lp1t	input	wire [3:0]
datain	input	wire [3:0]
buf_clk_lp0i	output	wire
buf_clk_lp1i	output	wire
rx_ready	output	wire
sclk	output	wire
oclk	output	wire
buf_data_lp0i	output	wire [3:0]
buf_data_lp1i	output	wire [3:0]
q	output	wire [31:0]
clk_lp0	inout	wire
clk_lp1	inout	wire
data_lp0	inout	wire [3:0]
data_lp1	inout	wire [3:0]

API — mipi_rx_lane_aligner.v**Table 589. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_rx_lane_aligner.v — module / interface / package (1개)**

이름	종류
mipi_rx_lane_aligner	module

Table 590. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_rx_lane_aligner.v — localparam (2개)

상수	형/폭	값
ALIGN_DEPTH	[3:0]	4'h3
LANES	[2:0]	3'h4

Table 591. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_rx_lane_aligner.v — 포트 (6개)

포트	방향	형/폭
reset_i	input	
clk_i	input	
1	input	[(LANES-
LANES	input	[(8*
LANES	output	reg [(8*
lane_valid_o	output	reg

API — mipi_rx_raw10_depacker.v**Table 592. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_rx_raw10_depacker.v — module / interface / package (1개)**

이름	종류
mipi_rx_raw10_depacker	module

Table 593. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_rx_raw10_depacker.v — localparam (1개)

상수	형/폭	값
BYTES_PERPACK	[2:0]	3'h5

Table 594. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_rx_raw10_depacker.v — 포트 (7개)

포트	방향	형/폭
delayed	output	is one clock cycle
cycle	output	come in group of 5x40bit chunk on each clock
clk_i	input	
data_valid_i	input	
data_i	input	[31:0]
output_valid_o	output	reg
output_o	output	reg [39:0]

API — output_reformatter.v**Table 595. circuitvalley_mipi_csi_receiver_FPGA/FPGA/output_reformatter.v — module / interface / package (1개)**

이름	종류
output_reformatter	module

Table 596. circuitvalley_mipi_csi_receiver_FPGA/FPGA/output_reformatter.v — 포트 (6개)

포트	방향	형/폭
clk_i	input	
data_in_valid_i	input	
data_i	input	[63:0]
output_valid_o	output	reg
output_o	output	reg [31:0]
output_clk_i	input	

API — rgb_to_yuv.v**Table 597. circuitvalley_mipi_csi_receiver_FPGA/FPGA/rgb_to_yuv.v — module / interface / package (1개)**

이름	종류
rgb_to_yuv	module

Table 598. circuitvalley_mipi_csi_receiver_FPGA/FPGA/rgb_to_yuv.v — localparam (2개)

상수	형/폭	값
PIXEL_DEPTH		4'd10
PIXEL_PER_CLK		4'd4

Table 599. circuitvalley_mipi_csi_receiver_FPGA/FPGA/rgb_to_yuv.v — 포트 (6개)

포트	방향	형/폭
clk_i	input	
reset_i	input	
3	input	[((PIXEL_DEPTH * PIXEL_PER_CLK *
rgb_valid_i	input	
8	output	reg [((PIXEL_PER_CLK * 2 *
yuv_valid_o	output	reg

API — tb_debayer_filter.v**Table 600. circuitvalley_mipi_csi_receiver_FPGA/FPGA/tb_debayer_filter.v — module / interface / package (1개)**

이름	종류
tb_debayer_filter	module

Table 601. circuitvalley_mipi_csi_receiver_FPGA/FPGA/tb_debayer_filter.v — 포트 (1개)

포트	방향	형/폭
bytes	input	[31:0]

API — tb_line_ram_dp_tmpl.v**Table 602. circuitvalley_mipi_csi_receiver_FPGA/FPGA/tb_line_ram_dp_tmpl.v — module / interface / package (1개)**

이름	종류
tb_ram	module

API — tb_mipi_bridge.v**Table 603. circuitvalley_mipi_csi_receiver_FPGA/FPGA/tb_mipi_bridge.v — module / interface / package (1개)**

이름	종류
tb_mipi_bridge	module

Table 604. circuitvalley_mipi_csi_receiver_FPGA/FPGA/tb_mipi_bridge.v — 포트 (1개)

포트	방향	형/폭
bytes	input	[31:0]

API — tb_mipi_csi_packet_decoder.v**Table 605. circuitvalley_mipi_csi_receiver_FPGA/FPGA/tb_mipi_csi_packet_decoder.v — module / interface / package (1개)**

이름	종류
tb_mipi_csi_packet_decoder	module

Table 606. circuitvalley_mipi_csi_receiver_FPGA/FPGA/tb_mipi_csi_packet_decoder.v — 포트 (1개)

포트	방향	형/폭
bytes	input	[31:0]

API — tb_mipi_rx_raw10_repacker.v**Table 607. circuitvalley_mipi_csi_receiver_FPGA/FPGA/tb_mipi_rx_raw10_repacker.v — module / interface / package (1개)**

이름	종류
tb_mipi_rx_raw10_depacker	module

Table 608. circuitvalley_mipi_csi_receiver_FPGA/FPGA/tb_mipi_rx_raw10_repacker.v — 포트 (1개)

포트	방향	형/폭
bytes	input	[31:0]

API — tb_rx_byte_aligner.v**Table 609. circuitvalley_mipi_csi_receiver_FPGA/FPGA/tb_rx_byte_aligner.v — module / interface / package (1개)**

이름	종류
tb_rx_byte_aligner	module

Table 610. circuitvalley_mipi_csi_receiver_FPGA/FPGA/tb_rx_byte_aligner.v — 포트 (1개)

포트	방향	형/폭
byte	input	[7:0]

API — tb_rx_lane_aligner.v**Table 611. circuitvalley_mipi_csi_receiver_FPGA/FPGA/tb_rx_lane_aligner.v — module / interface / package (1개)**

이름	종류
tb_rx_lane_aligner	module

Table 612. circuitvalley_mipi_csi_receiver_FPGA/FPGA/tb_rx_lane_aligner.v — 포트 (1개)

포트	방향	형/폭
bytes	input	[31:0]

1. circuitvalley_mipi_csi_receiver_FPGA/FPGA/debayer_filter.v (349 lines)

```

1 `timescale 1ns/1ns
2
3 /*
4 MIPI CSI RX to Parallel Bridge (c) by Gaurav Singh www.CircuitValley.com
5
6 MIPI CSI RX to Parallel Bridge is licensed under a
7 Creative Commons Attribution 3.0 Unported License.
8
9 You should have received a copy of the license along with this
10 work. If not, see <http://creativecommons.org/licenses/by/3.0/>.
11 */
12
13 /*
14 Takes 4x10bit pixel from RAW10 depacker module @mipi byte
15 clock output 4x30bit RGB for each pixel , output is delayed by 2
16 lines
17 Implement Basic Debayer filter, As debayer need pixel infrom
18 neighboring pixel which may be on next or previous display line,
19 so input data is written onto RAM, only 4 lines are stored in
20 RAM at one time and only three of the readable at any give time ,
21 RAM to which data is written to can not be read.
22 as we have enough info in RAM, 4 10bit pixel will be converted
23 to 4x24bit RGB output
24 First line is expected to RGRG , second line GBGB
25 */
26
27 module debayer_filter(clk_i,
28                     reset_i,
29                     line_valid_i,
30                     data_i,
31                     data_valid_i,
32                     output_valid_o,
33                     output_o,
34                     debug_out);
35
36 localparam INPUT_WIDTH = 40; //4 x 10bit pixels from raw10
37 depacker module
38 localparam OUTPUT_WIDTH = 120; //4 x 30bit RGB output
39
40 input clk_i;
41 input reset_i;
42 input line_valid_i;
43 input data_valid_i;
44 input [(INPUT_WIDTH-1):0] data_i;
45 output reg output_valid_o;
46 output reg [(OUTPUT_WIDTH-1):0] output_o;

```

```

40
41 output reg [7:0]debug_out;
42
43
44
45 reg [11:0]line_counter; //counts lines of the frame , needed
46 determine if have enough data ins line rams to start outputting RGB
47 data
48 reg data_valid_reg;
49
50 reg [9:0]R1[3:0];
51 reg [9:0]R2[3:0];
52 reg [9:0]R3[3:0];
53 reg [9:0]R4[3:0];
54
55 reg [9:0]B1[3:0];
56 reg [9:0]B2[3:0];
57 reg [9:0]B3[3:0];
58 reg [9:0]B4[3:0];
59
60 reg [9:0]G1[3:0];
61 reg [9:0]G2[3:0];
62 reg [9:0]G3[3:0];
63 reg [9:0]G4[3:0];
64
65 reg [1:0]read_ram_index; //which line RAM is being fo-
66 cused to read, not which address is being read from line RAM
67 reg [3:0]write_ram_select; //which line RAM is being writ-
68 ten
69 reg [9:0]line_address; //which address is being read
70 and written
71 reg [(INPUT_WIDTH-1):0]last_ram_outputs[3:0]; //one clock
72 cycle delayed output of line RAMs
73 reg [(INPUT_WIDTH-1):0]last_ram_outputs_stage2[3:0]; //two
74 clock cycle delayed output of RAMs
75
76 reg [1:0]not_used2b;
77
78 wire [(INPUT_WIDTH-1):0]RAM_out[3:0];
79 wire ram_write_enable;
80 wire ram_clk;
81
82 assign ram_clk = !clk_i;
83 assign ram_write_enable = data_valid_i;
84
85 //line rams, total 4,
86 //The way debayer is implemented in this code. Depending on
87 pixel, we need minimum 2 lines and maximum 3 lines in the ram, To
88 be able to have access to neighboring pixels from previous and next
89 line
90 //There are many ways to implemented debayer, this code imple-

```

```

ment simplest possible bare minimum.
83 // IMX219 Camera only output BGGR as defined by the IMX219
Driver in linux repo MEDIA_BUS_FMT_SBGGR10_1X10, Camera datasheet
incorrectly defines output as RGGB and GBRG. Data sheet is incor-
rect in this case.
84 // Bayer filter type does not affect test pattern.
85
86 line_ram_dp line0(      .WrAddress(line_address),
87                        .RdAd-
dress(line_address),
88                        .Data(data_i),
89                        .WE(((write_ram_select &
4'b001))) && ram_write_enable),
90                        .RdClock(ram_clk),
91                        .RdClockEn(1'b1),
92                        .Reset(reset_i),
93                        .WrClock(ram_clk),
94                        .WrClockEn(1'b1),
95                        .Q(RAM_out[0]));
96
97 line_ram_dp line1(      .WrAddress(line_address),
98                        .RdAd-
dress(line_address),
99                        .Data(data_i),
100                       .WE(((write_ram_select &
4'b0010))) && ram_write_enable),
101                       .RdClock(ram_clk),
102                       .RdClockEn(1'b1),
103                       .Reset(reset_i),
104                       .WrClock(ram_clk),
105                       .WrClockEn(1'b1),
106                       .Q(RAM_out[1]));
107
108 line_ram_dp line2(      .WrAddress(line_address),
109                        .RdAd-
dress(line_address),
110                        .Data(data_i),
111                        .WE(((write_ram_select &
4'b0100))) && ram_write_enable),
112                        .RdClock(ram_clk),
113                        .RdClockEn(1'b1),
114                        .Reset(reset_i),
115                        .WrClock(ram_clk),
116                        .WrClockEn(1'b1),
117                        .Q(RAM_out[2]));
118
119 line_ram_dp line3(      .WrAddress(line_address),
120                        .RdAd-
dress(line_address),
121                        .Data(data_i),
122                        .WE(((write_ram_select &
4'b1000))) && ram_write_enable),
123                        .RdClock(ram_clk),
124                        .RdClockEn(1'b1),
125                        .Reset(reset_i),
126                        .WrClock(ram_clk),
127                        .WrClockEn(1'b1),
128                        .Q(RAM_out[3]));
129
130
131 always @(posedge clk_i) //address should increment at falling
edge of ram_clk. It is inverted from clk_i
132 begin
133     if (reset_i)
134     begin
135         line_address <= 9'h0;
136     end
137     else
138     begin
139         if (!line_valid_i)
140         begin
141             line_address <= 9'h0;
142         end
143         else if (data_valid_i)
144         begin
145             line_address <= line_address + 1'b1;
146         end
147     end
148 end
149
150
151 always @(posedge reset_i or posedge line_valid_i)
152 begin
153     if (reset_i)
154     begin
155         write_ram_select <= 4'b1000;
156         line_counter <= 12'b0;
157         read_ram_index <= 2'b01;
158     end
159     else
160     begin
161         write_ram_select <= {write_ram_select[2:0],
write_ram_select[3]};
162         read_ram_index <= read_ram_index + 1'b1;
163         line_counter <= line_counter + 1'b1;
164     end
165 end

```

```

166
167
168 always @(posedge clk_i)
169 begin
170     if (reset_i)
171     begin
172         output_valid_o <= 1'b0;
173         data_valid_reg <= 1'b0;
174     end
175     else
176     begin
177         last_ram_outputs[0] <= RAM_out[0];
178         last_ram_outputs[1] <= RAM_out[1];
179         last_ram_outputs[2] <= RAM_out[2];
180         last_ram_outputs[3] <= RAM_out[3];
181
182         last_ram_outputs_stage2[0] <= last_ram_out-
puts[0];
183         last_ram_outputs_stage2[1] <= last_ram_out-
puts[1];
184         last_ram_outputs_stage2[2] <= last_ram_out-
puts[2];
185         last_ram_outputs_stage2[3] <= last_ram_out-
puts[3];
186
187         if (line_counter > 9'd2)
188         begin
189             data_valid_reg <= data_valid_i;
190             output_valid_o <= data_valid_reg;
191             if (!line_counter[0]) //even
192             begin
193                 B1[0] = last_ram_out-
puts[ read_ram_index + 1'd1 ][39:30];
194                 B2[0] = last_ram_out-
puts[ read_ram_index + 1'd1 ][39:30];
195                 B3[0] = last_ram_out-
puts[ read_ram_index - 1'd1 ][39:30];
196                 B4[0] = last_ram_out-
puts[ read_ram_index - 1'd1 ][39:30];
197
198                 G1[0] = last_ram_out-
puts[ read_ram_index ][39:30];
199                 G2[0] = last_ram_out-
puts[ read_ram_index ][39:30];
200                 G3[0] = last_ram_out-
puts[ read_ram_index ][39:30];
201                 G4[0] = last_ram_out-
puts[ read_ram_index ][39:30];
202
203                 R1[0] = last_ram_out-
puts[ read_ram_index ][29:20];
204                 R2[0] = last_ram_out-
puts_stage2[ read_ram_index ][ 9:0 ];
205                 R3[0] = last_ram_out-
puts[ read_ram_index ][29:20];
206                 R4[0] = last_ram_out-
puts_stage2[ read_ram_index ][ 9:0 ];
207
208                 B1[1] = last_ram_out-
puts[ read_ram_index - 1'd1 ][39:30];
209                 B2[1] = last_ram_out-
puts[ read_ram_index + 1'd1 ][39:30];
210                 B3[1] = last_ram_out-
puts[ read_ram_index - 1'd1 ][19:10];
211                 B4[1] = last_ram_out-
puts[ read_ram_index + 1'd1 ][19:10];
212
213                 G1[1] = last_ram_out-
puts[ read_ram_index ][39:30];
214                 G2[1] = last_ram_out-
puts[ read_ram_index - 1'h1 ][29:20];
215                 G3[1] = last_ram_out-
puts[ read_ram_index + 1'h1 ][29:20];
216                 G4[1] = last_ram_out-
puts[ read_ram_index ][19:10];
217
218                 R1[1] = last_ram_out-
puts[ read_ram_index ][29:20];
219                 R2[1] = last_ram_out-
puts[ read_ram_index ][29:20];
220                 R3[1] = last_ram_out-
puts[ read_ram_index ][29:20];
221                 R4[1] = last_ram_out-
puts[ read_ram_index ][29:20];
222
223                 B1[2] = last_ram_out-
puts[ read_ram_index - 1'd1 ][19:10];
224                 B2[2] = last_ram_out-
puts[ read_ram_index + 1'd1 ][19:10];
225                 B3[2] = last_ram_out-
puts[ read_ram_index - 1'd1 ][19:10];
226                 B4[2] = last_ram_out-
puts[ read_ram_index + 1'd1 ][19:10];
227
228                 G1[2] = last_ram_out-
puts[ read_ram_index ][19:10];
229                 G2[2] = last_ram_out-
puts[ read_ram_index ][19:10];
230                 G3[2] = last_ram_out-
puts[ read_ram_index ][19:10];
231

```

```

puts[ read_ram_index      ][19:10];
232      G4[2] = last_ram_out-
puts[ read_ram_index      ][19:10];
233
234      R1[2] = last_ram_out-
puts[ read_ram_index      ][ 9:0 ];
235      R2[2] = last_ram_out-
puts[ read_ram_index      ][29:20];
236      R3[2] = last_ram_out-
puts[ read_ram_index      ][ 9:0 ];
237      R4[2] = last_ram_out-
puts[ read_ram_index      ][29:20];
238
239      B1[3] =
RAM_out[ read_ram_index - 1'd1 ][39:30];
240      B2[3] = last_ram_out-
puts[ read_ram_index - 1'd1 ][19:10];
241      B3[3] =
RAM_out[ read_ram_index + 1'd1 ][39:30];
242      B4[3] = last_ram_out-
puts[ read_ram_index + 1'd1 ][19:10];
243
244
245      G1[3] = last_ram_out-
puts[ read_ram_index      ][19:10];
246      G2[3] = last_ram_out-
puts[ read_ram_index - 1'h1 ][ 9:0 ];
247      G3[3] = last_ram_out-
puts[ read_ram_index + 1'h1 ][ 9:0 ];
248      G4[3] =
RAM_out[ read_ram_index      ][39:30];
249
250      R1[3] = last_ram_out-
puts[ read_ram_index      ][ 9:0 ];
251      R2[3] = last_ram_out-
puts[ read_ram_index      ][ 9:0 ];
252      R3[3] = last_ram_out-
puts[ read_ram_index      ][ 9:0 ];
253      R4[3] = last_ram_out-
puts[ read_ram_index      ][ 9:0 ];
254
255      end          //end even rows
256      else
257      begin        //odd rows //First line
258
259      B1[0] =      last_ram_out-
puts[ read_ram_index      ][39:30];
260      B2[0] =      last_ram_out-
puts[ read_ram_index      ][39:30];
261      B3[0] =      last_ram_out-
puts[ read_ram_index      ][39:30];
262      B4[0] =      last_ram_out-
puts[ read_ram_index      ][39:30];
263
264      G1[0] =      last_ram_out-
puts[ read_ram_index - 1'd1 ][39:30];
265      G2[0] =      last_ram_out-
puts[ read_ram_index + 1'd1 ][39:30];
266      G3[0] =      last_ram_out-
puts[ read_ram_index      ][29:20];
267      G4[0] = last_ram_out-
puts_stage2[ read_ram_index ][ 9:0 ];
268
269      R1[0] = last_ram_out-
puts_stage2[ read_ram_index - 1'd1 ][ 9:0 ];
270      R2[0] =      last_ram_out-
puts[ read_ram_index - 1'd1 ][29:20];
271      R3[0] = last_ram_out-
puts_stage2[ read_ram_index + 1'd1 ][ 9:0 ];
272      R4[0] =      last_ram_out-
puts[ read_ram_index + 1'd1 ][29:20];
273
274
275      B1[1] = last_ram_out-
puts[ read_ram_index      ][39:30];
276      B2[1] = last_ram_out-
puts[ read_ram_index      ][19:10];
277      B3[1] = last_ram_out-
puts[ read_ram_index      ][39:30];
278      B4[1] = last_ram_out-
puts[ read_ram_index      ][19:10];
279
280      G1[1] = last_ram_out-
puts[ read_ram_index      ][29:20];
281      G2[1] = last_ram_out-
puts[ read_ram_index      ][29:20];
282      G3[1] = last_ram_out-
puts[ read_ram_index      ][29:20];
283      G4[1] = last_ram_out-
puts[ read_ram_index      ][29:20];
284
285      R1[1] = last_ram_out-
puts[ read_ram_index - 1'd1 ][29:20];
286      R2[1] = last_ram_out-
puts[ read_ram_index + 1'd1 ][29:20];
287      R3[1] = last_ram_out-
puts[ read_ram_index - 1'd1 ][29:20];
288      R4[1] = last_ram_out-
puts[ read_ram_index + 1'd1 ][29:20];
289

```

```

290      B1[2] = last_ram_out-
puts[ read_ram_index      ][19:10];
291      B2[2] = last_ram_out-
puts[ read_ram_index      ][19:10];
292      B3[2] = last_ram_out-
puts[ read_ram_index      ][19:10];
293      B4[2] = last_ram_out-
puts[ read_ram_index      ][19:10];
294
295      G1[2] = last_ram_out-
puts[ read_ram_index - 1'd1 ][19:10];
296      G2[2] = last_ram_out-
puts[ read_ram_index + 1'd1 ][19:10];
297      G3[2] = last_ram_out-
puts[ read_ram_index      ][29:20];
298      G4[2] = last_ram_out-
puts[ read_ram_index      ][ 9:0 ];
299
300      R1[2] = last_ram_out-
puts[ read_ram_index - 1'd1 ][ 9:0 ];
301      R2[2] = last_ram_out-
puts[ read_ram_index - 1'd1 ][29:20];
302      R3[2] = last_ram_out-
puts[ read_ram_index + 1'd1 ][ 9:0 ];
303      R4[2] = last_ram_out-
puts[ read_ram_index + 1'd1 ][29:20];
304
305      B1[3] =
RAM_out[ read_ram_index      ][39:30];
306      B2[3] = last_ram_out-
puts[ read_ram_index      ][19:10];
307      B3[3] =
RAM_out[ read_ram_index      ][39:30];
308      B4[3] = last_ram_out-
puts[ read_ram_index      ][19:10];
309
310      G1[3] = last_ram_out-
puts[ read_ram_index      ][ 9:0 ];
311      G2[3] = last_ram_out-
puts[ read_ram_index      ][ 9:0 ];
312      G3[3] = last_ram_out-
puts[ read_ram_index      ][ 9:0 ];
313      G4[3] =
RAM_out[ read_ram_index      ][ 9:0 ];
314
315      R1[3] = last_ram_out-
puts[ read_ram_index - 1'd1 ][ 9:0 ];
316      R2[3] = last_ram_out-
puts[ read_ram_index + 1'd1 ][ 9:0 ];
317      R3[3] = last_ram_out-
puts[ read_ram_index - 1'd1 ][ 9:0 ];
318      R4[3] = last_ram_out-
puts[ read_ram_index + 1'd1 ][ 9:0 ];
319
320      end
321
322      //debug_out <= R1[0][9:2] & R2[0][9:2]
& R3[0][9:2] & R4[0][9:2];
323      debug_out <= {{2'd0, R1[1]} + R2[1] +
R3[1] + R4[1]} >> 4;
324
325      {not_used2b,output_o[119:110]} <=
{{2'd0, R1[0]} + R2[0] + R3[0] + R4[0]} >> 2; //R
326      {not_used2b,output_o[109:100]} <=
{{2'd0, G1[0]} + G2[0] + G3[0] + G4[0]} >> 2; //G
327      {not_used2b,output_o[99:90]} <=
{{2'd0, B1[0]} + B2[0] + B3[0] + B4[0]} >> 2; //B
328
329      {not_used2b,output_o[89:80]} <= {{2'd0,
R1[1]} + R2[1] + R3[1] + R4[1]} >> 2; //R
330
331      {not_used2b,output_o[79:70]} <= {{2'd0,
G1[1]} + G2[1] + G3[1] + G4[1]} >> 2; //G
332
333      {not_used2b,output_o[69:60]} <= {{2'd0,
B1[1]} + B2[1] + B3[1] + B4[1]} >> 2; //B
334
335      {not_used2b,output_o[59:50]} <= {{2'd0,
R1[2]} + R2[2] + R3[2] + R4[2]} >> 2; //R
336
337      {not_used2b,output_o[49:40]} <= {{2'd0,
G1[2]} + G2[2] + G3[2] + G4[2]} >> 2; //G
338
339      {not_used2b,output_o[39:30]} <= {{2'd0,
B1[2]} + B2[2] + B3[2] + B4[2]} >> 2; //B
340
341      {not_used2b,output_o[29:20]} <= {{2'd0,
R1[3]} + R2[3] + R3[3] + R4[3]} >> 2; //R
342
343      end
end

```

```

344 end
345
346
347
348
349 endmodule

```

2. circuitvalley_mipi_csi_receiver_FPGA/FPGA/line_ram_dp.v

[illegible][illegible]

[illegible]

[illegible]

[illegible]

```

363 // exemplar attribute line_ram_dp_0_1_3 MEM_INIT_FILE
INIT_ALL_0s
364 // exemplar attribute line_ram_dp_0_2_2 MEM_LPC_FILE
line_ram_dp.lpc
365 // exemplar attribute line_ram_dp_0_2_2 MEM_INIT_FILE
INIT_ALL_0s
366 // exemplar attribute line_ram_dp_0_3_1 MEM_LPC_FILE
line_ram_dp.lpc
367 // exemplar attribute line_ram_dp_0_3_1 MEM_INIT_FILE
INIT_ALL_0s
368 // exemplar attribute line_ram_dp_0_4_0 MEM_LPC_FILE
line_ram_dp.lpc
369 // exemplar attribute line_ram_dp_0_4_0 MEM_INIT_FILE
INIT_ALL_0s
370 // exemplar end
371
372 endmodule

```

3. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_bridge.v (194 lines)

```

1 `timescale 1ns/1ns
2
3 /*
4 MIPI CSI RX to Parallel Bridge (c) by Gaurav Singh www.Cir-
cuiValley.com
5
6 MIPI CSI RX to Parallel Bridge is licensed under a
7 Creative Commons Attribution 3.0 Unported License.
8
9 You should have received a copy of the license along with this
10 work. If not, see <http://creativecommons.org/licenses/by/
3.0/>.
11 */
12
13 /*
14 MIPI CSI 4 Lane Receiver To Parallel Bridge
15 Tested with Lattice MachX03LF-6900 with IMX219 Camera
16 Takes MIPI Clock and 4 Data lane as input convert into Paral-
lel YUV output
17 Outputs 32bit YUV data with Frame sync, lsync and pixel clock
18 */
19
20 module mipi_bridge(    clk_i,
21
22                      reset_in,
23                      mipi_clk_in,
24                      mipi_clk_lpp_io,
25                      mipi_clk_lpn_io,
26                      mipi_data_in,
27                      mipi_data_lpp_io,
28                      mipi_data_lpn_io,
29
30                      pclk_out,
31                      data_out,
32                      fsyn_out,
33                      lsync_out,
34
35                      debug_b8
36                      );
37
38 parameter MIPI_LANES = 4;
39 input clk_i;
40
41 input reset_in;
42 input mipi_clk_in;
43 inout mipi_clk_lpp_io;
44 inout mipi_clk_lpn_io;
45 input [MIPI_LANES-1:0]mipi_data_in;
46 inout [MIPI_LANES-1:0]mipi_data_lpp_io;
47 inout [MIPI_LANES-1:0]mipi_data_lpn_io;
48
49 output pclk_out;
50 output [31:0]data_out;
51 output fsyn_out;
52 output lsync_out;
53
54 output wire [7:0]debug_b8;
55
56 wire reset ;
57 wire [MIPI_LANES-1:0]mipi_data_lpp_in;
58 wire [MIPI_LANES-1:0]mipi_data_lpn_in;
59
60
61 wire output_clock;
62 wire mipi_byte_clock; //byte clock from mipi phy
63
64 wire mipi_clk_n_lp;
65 wire mipi_clk_p_lp;
66
67 wire [3:0]is_byte_valid;
68 wire is_lane_aligned_valid;
69 wire is_decoded_valid;
70 wire is_unpacked_valid;
71 wire is_rgb_valid;
72 wire is_yuv_valid;
73
74 wire mipi_out_clk;
75
76 wire [31:0]mipi_data_raw;

```

```

77 wire [31:0]byte_aligned;
78 wire [31:0]lane_aligned;
79 wire [31:0]decoded_data;
80 wire [31:0]packet_length;
81 wire [39:0]unpacked_data;
82 wire [119:0]rgb_data;
83 wire [63:0]yuv_data;
84
85 wire byte_aligner_reset = mipi_data_lpn_in[0];
86 assign reset = mipi_clk_n_lp | !reset_in;
87
88 mipi_rx_ddr mipi_rx_ddr_inst_0(.alignwd(1'b0),
89
90 buf_clk_lp0i(mipi_clk_n_lp),
91
92 buf_clk_lp0o(),
93
94 buf_clk_lp0t(1'b1),
95
96 buf_clk_lp1i(mipi_clk_p_lp),
97
98 buf_clk_lp1o(),
99
100 buf_clk_lp1t(1'b1),
101
102 clk(mipi_clk_in),
103
104 clk_lp0(mipi_clk_lpn_io),
105
106 clk_lp1(mipi_clk_lpp_io),
107
108 clk_s(clk_i),
109
110 init(1'b1),
111
112 .reset(reset),
113
114 .rx_ready(),
115
116 .sclk(mipi_byte_clock),
117
118 .oclk(mipi_out_clk), //double to mipi_clock
119
120 .buf_data_lp0i(mipi_data_lpn_in),
121
122 .buf_data_lp0o(4'b0),
123
124 .buf_data_lp0t(4'b1111),
125
126 .buf_data_lp1i(mipi_data_lpp_in),
127
128 .buf_data_lp1o(4'b0),
129
130 .buf_data_lp1t(4'b1111),
131
132 .data_lp0(mipi_data_lpn_io),
133
134 .data_lp1(mipi_data_lpp_io),
135
136 .datain(mipi_data_in),
137
138 .q(mipi_data_raw));
139
140
141
142
143
144
145
146 mipi_rx_byte_aligner
mipi_rx_byte_aligner_0( .clk_i(mipi_byte_clock),
147
148 .reset_i(byte_aligner_reset),
149
150 .byte_i(mipi_data_raw[7:0]),
151
152 .byte_o( byte_aligned[7:0]),

```

```

120 .byte_valid_o(is_byte_valid[0]));
121
122
123 mipi_rx_byte_aligner
mipi_rx_byte_aligner_1( .clk_i(mipi_byte_clock),

124 .reset_i(byte_aligner_reset),

125 .byte_i(mipi_data_raw[15:8]),

126 .byte_o(byte_aligned[15:8]),

127 .byte_valid_o(is_byte_valid[1]));
128
129
130 mipi_rx_byte_aligner
mipi_rx_byte_aligner_2( .clk_i(mipi_byte_clock),

131 .reset_i(byte_aligner_reset),

132 .byte_i(mipi_data_raw[23:16]),

133 .byte_o( byte_aligned[23:16]),

134 .byte_valid_o(is_byte_valid[2]));
135
136 mipi_rx_byte_aligner
mipi_rx_byte_aligner_3( .clk_i(mipi_byte_clock),

137 .reset_i(byte_aligner_reset),

138 .byte_i(mipi_data_raw[31:24]),

139 .byte_o( byte_aligned[31:24]),

140 .byte_valid_o(is_byte_valid[3]));
141
142 mipi_rx_lane_aligner
mipi_rx_lane_aligner( .clk_i(mipi_byte_clock),

143 .reset_i(reset),

144 .bytes_valid_i(is_byte_valid),

145 .byte_i(byte_aligned),

146 .lane_valid_o(is_lane_aligned_valid),

147 .lane_byte_o(lane_aligned));
148
149
150 mipi_csi_packet_decoder mipi_csi_pack-
et_decoder_0( .clk_i(mipi_byte_clock),

151 .data_valid_i(is_lane_aligned_valid),

152 .data_i(lane_aligned),

153 .output_valid_o(is_decoded_valid),

154 .data_o(decoded_data),

155 .packet_length(packet_length));
156
157
158 mipi_rx_raw10_depacker
mipi_rx_raw10_depacker_0(.clk_i(mipi_byte_clock),

159 .data_valid_i(is_decoded_valid),

160 .data_i(decoded_data),

161 .output_o(unpacked_data),

162

```

```

.output_valid_o(is_unpacked_valid));
163
164
165 debayer_filter debayer_filter_0(.clk_i(mipi_byte_clock),

166 .reset_i(reset),

167 .line_valid_i(is_decoded_valid),

168 .data_i(unpacked_data),

169 .data_valid_i(is_unpacked_valid),

170 .output_o(rgb_data),

171 .output_valid_o(is_rgb_valid),

172 .debug_out(debug_b8));
173
174 rgb_to_yuv rgb_to_yuv_0(.clk_i(mipi_byte_clock),
175 .reset_i(reset),
176 .rgb_i(rgb_data),
177 .rgb_val-
id_i(is_rgb_valid),
178 .yuv_o(yuv_data),
179 .yuv_val-
id_o(is_yuv_valid));
180
181
182 output_reformatter out_reformatter_0(.clk_i(mipi_byte_clock),

183 .output_clk_i(mipi_out_clk),

184 .data_i(yuv_data),

185 .data_in_valid_i(is_yuv_valid),

186 .output_o(data_out),

187 .output_valid_o(lsync_out));
188
189
190
191 assign pclk_out = reset? clk_i: mipi_out_clk ; //output clock
always available, slow when there is no mipi frame , fast from
mipi_clk when mipi_clock is active
192 assign fsyn_out = !re-
set; //activate fsync as soon
as mipi frame is active
193
194 endmodule

```

4. circuitvalley_mipi_csi_receiver_FPGA/FPGA/ mipi_csi_packet_decoder.v (71 lines)

```

1 `timescale 1ns/1ns
2
3 /*
4 MIPI CSI RX to Parallel Bridge (c) by Gaurav Singh www.Cir-
cuitValley.com
5
6 MIPI CSI RX to Parallel Bridge is licensed under a
7 Creative Commons Attribution 3.0 Unported License.
8
9 You should have received a copy of the license along with this
10 work. If not, see <http://creativecommons.org/licenses/by/
3.0/>.
11 */
12
13 /*
14 Basically a packet Stripper, removes header and footer from
packet
15 Takes lane aligned data from lane aligner @ mipi byte clock
16 looks for specific packet type, in this case RAW10bit 0x2B
17 outputs Stripped bytes in exactly the way they were received.
18 this module also fetch packet length and output_valid is act-
ive as long as input data is valid and packet length is still valid.
19 */
20
21 module mipi_csi_packet_decoder(

22 clk_i,

23 data_valid_i,

24

```

```

data_i,
25
output_valid_o,

26
data_o,

27
packet_length

28
;

29
30 localparam [7:0]SYNC_BYTE = 8'hB8;
31 localparam [3:0]LANES = 3'h4;
32 localparam [7:0]MIPI_CSI_PACKET_10bRAW = 8'h2B;
33 input clk_i;
34 input data_valid_i;
35 input [31:0]data_i;
36 output reg output_valid_o;
37 output reg [31:0]data_o;
38 output reg [31:0]packet_length;
39 reg [31:0]packet_length_reg;
40
41 reg [31:0]last_data_i;
42
43 always @(negedge clk_i)
44 begin
45     if (data_valid_i)
46     begin
47         last_data_i <= data_i;
48         output_valid_o <= |packet_length_reg;
49         data_o <= data_i;
50
51         if (!|packet_length_reg)
52         begin
53             packet_length_reg <= packet_length_reg
- LANES;
54         end
55     else if (last_data_i[7:0] == SYNC_BYTE &&
data_i[7:0] == MIPI_CSI_PACKET_10bRAW)
56     begin
57         packet_length_reg <= {data_i[23:16],
data_i[15:8]];
58         packet_length <= {data_i[23:16],
data_i[15:8]];
59     end
60     end
61     else
62     begin
63         last_data_i <=32'h0;
64         data_o <=32'h0;
65         packet_length <= 32'h0;
66         packet_length_reg <= 32'h0;
67         output_valid_o <= 1'h0;
68     end
69 end
70
71 endmodule

```

5. circuitvalley_mipi_csi_receiver_FPGA/FPGA/ mipi_rx_byte_aligner.v (80 lines)

```

1 `timescale 1ns/1ns
2
3 /*
4  * MIPI CSI RX to Parallel Bridge (c) by Gaurav Singh www.Cir-
cuiValley.com
5
6  * MIPI CSI RX to Parallel Bridge is licensed under a
7  * Creative Commons Attribution 3.0 Unported License.
8
9  * You should have received a copy of the license along with this
10  * work. If not, see <http://creativecommons.org/licenses/by/
3.0/>.
11  */
12
13 /*
14  * Received Raw unaligned bits from DDR RX module outputs Aligned
bytes
15  * Bytes on MIPI lane does not have any defined byte boundary so
this module Looks for always constant first byte 0xB8 on wire,
16  * once 0xB8 is found, byte boundary offset is determined, set
output valid to active and start outputting correct bytes
17  * stays reset when data lane are in MIPI LP state
18  */
19
20
21 module mipi_rx_byte_aligner(
22
23     clk_i,
24     reset_i,
25     byte_i,
26     byte_o,
27     byte_valid_o
28 );
29 localparam [7:0]SYNC_BYTE = 8'hB8;

```

```

30
31 input clk_i;
32 input reset_i;
33 input [7:0]byte_i;
34 output reg [7:0]byte_o;
35 output reg byte_valid_o;
36
37 reg [2:0]offset;
38 reg [3:0]i;
39 reg [7:0] last_byte;
40 wire [16:0]word;
41
42 // TODO: Optimize first byte output;
43
44 assign word = {byte_i, last_byte};
45 always @(negedge clk_i)
46 begin
47     if (reset_i)
48     begin
49         last_byte <= 8'h0;
50         byte_valid_o <= 1'b0;
51         offset <= 3'h0;
52     end
53     else
54     begin
55
56         last_byte <= byte_i;
57
58         if (!byte_valid_o)
59         begin
60             for (i= 8'h0; i < 8; i = i + 1'h1)
61                 begin
62                     if ( ( word[(i + 1'h1) +: 8] ==
SYNC_BYTE))
63                         begin
64                             byte_valid_o <=
1'h1;
65                             offset <=
i[2:0] + 1'b1;
66                             byte_o <=
SYNC_BYTE; //first byte output if sync found is always going to be
the syncbyte itself
67                             end
68                         end
69                     end
70                 else
71                 begin
72                     byte_o <= word[offset +:8]; //
from offset 8bits upwards
73                 end
74             end
75         end
76     end
77 endmodule
78
79
80

```

6. circuitvalley_mipi_csi_receiver_FPGA/FPGA/mipi_rx_ddr.v (271 lines)

```

1 /* Verilog netlist generated by SCUBA Diamond (64-bit)
3.11.0.396.4 */
2 /* Module Version: 5.8 */
3 /* C:\lscd\diamond\3.11_x64\isppfpga\bin\nt64\scuba.exe -w -n
mipi_rx_ddr -lang verilog -synth lse -bus_exp 7 -bb -arch xo3c00f -
type iol -mode in -io_type MIPI_LP width 4 -freq_in 100 -gear 4 -
clk_eclk -del -1 */
4 /* Thu Jan 23 21:47:24 2020 */
5
6
7 // Generated by IPX but Modified output bus bit order
8 `timescale 1 ns / 1 ps
9 module mipi_rx_ddr (alignwd, buf_clk_lp0i, buf_clk_lp0o,
buf_clk_lp0t,
10     buf_clk_lp1i, buf_clk_lp1o, buf_clk_lp1t, clk, clk_lp0,
clk_lp1,
11     clk_s, init, reset, rx_ready, sclck, oclck, buf_data_lp0i,
buf_data_lp0o,
12     buf_data_lp0t, buf_data_lp1i, buf_data_lp1o,
buf_data_lp1t, data_lp0,
13     data_lp1, datain, q)/* synthesis NGD_DRC_MASK=1 */;
14 input wire alignwd;
15 input wire buf_clk_lp0o;
16 input wire buf_clk_lp0t;
17 input wire buf_clk_lp1o;
18 input wire buf_clk_lp1t;
19 input wire clk;
20 input wire clk_s;
21 input wire init;
22 input wire reset;
23 input wire [3:0] buf_data_lp0o;
24 input wire [3:0] buf_data_lp0t;
25 input wire [3:0] buf_data_lp1o;
26 input wire [3:0] buf_data_lp1t;
27 input wire [3:0] datain;
28 output wire buf_clk_lp0i;
29 output wire buf_clk_lp1i;

```



```

30 output wire rx_ready;
31 output wire sclk;
32 output wire oclk;
33 output wire [3:0] buf_data_lp0i;
34 output wire [3:0] buf_data_lp1i;
35 output wire [31:0] q;
36 inout wire clk_lp0;
37 inout wire clk_lp1;
38 inout wire [3:0] data_lp0;
39 inout wire [3:0] data_lp1;
40
41 wire scuba_vlo;
42 wire opensync_0;
43 wire opensync_1;
44 wire opensync_cken;
45 wire opensync_2;
46 wire q7_3;
47 wire q6_3;
48 wire q5_3;
49 wire q4_3;
50 wire q3_3;
51 wire q2_3;
52 wire q1_3;
53 wire q0_3;
54 wire q7_2;
55 wire q6_2;
56 wire q5_2;
57 wire q4_2;
58 wire q3_2;
59 wire q2_2;
60 wire q1_2;
61 wire q0_2;
62 wire q7_1;
63 wire q6_1;
64 wire q5_1;
65 wire q4_1;
66 wire q3_1;
67 wire q2_1;
68 wire q1_1;
69 wire q0_1;
70 wire q7_0;
71 wire q6_0;
72 wire q5_0;
73 wire q4_0;
74 wire q3_0;
75 wire q2_0;
76 wire q1_0;
77 wire q0_0;
78 wire sclk_t;
79 wire oclk_t;
80 wire cdiv1;
81 wire eclko;
82 wire xstop;
83 wire xstart;
84 wire opensync_3;
85 wire eclki;
86 wire lock_chk;
87 wire scuba_vhi;
88 wire dataini_t3;
89 wire dataini_t2;
90 wire dataini_t1;
91 wire dataini_t0;
92 wire buf_clk;
93 wire buf_dataini3;
94 wire buf_dataini2;
95 wire buf_dataini1;
96 wire buf_dataini0;
97
98 defparam LUT4_1.initval = 16'h0a78 ;
99 ROM16X1A LUT4_1 (.AD3(open-
sync_0), .AD2(opensync_3), .AD1(lock_chk),
100 .AD0(init), .D00(opensync_cken));
101
102 VLO scuba_vlo_inst (.Z(scuba_vlo));
103
104 defparam LUT4_0.initval = 16'hfffe ;
105 ROM16X1A LUT4_0 (.AD3(open-
sync_0), .AD2(opensync_1), .AD1(scuba_vlo),
106 .AD0(scuba_vlo), .D00(xstop));
107
108 FD1P3BX FF_3 (.D(open-
sync_3), .SP(opensync_cken), .CK(clk_s), .PD(reset),
109 .Q(opensync_0))
110 /* synthesis GSR="ENABLED" */;
111
112 FD1P3DX FF_2 (.D(open-
sync_0), .SP(opensync_cken), .CK(clk_s), .CD(reset),
113 .Q(opensync_1))
114 /* synthesis GSR="ENABLED" */;
115
116 FD1P3DX FF_1 (.D(open-
sync_1), .SP(opensync_cken), .CK(clk_s), .CD(reset),
117 .Q(opensync_2))
118 /* synthesis GSR="ENABLED" */;
119
120 FD1P3DX FF_0 (.D(open-
sync_2), .SP(opensync_cken), .CK(clk_s), .CD(reset),
121 .Q(opensync_3))
122 /* synthesis GSR="ENABLED" */;
123
124 BB Inst9_BB
(.I(buf_clk_lp1o), .T(buf_clk_lp1t), .O(buf_clk_lp1i), .B(clk_lp1))
125 /* synthesis IO_TYPE="MIPI_LP" */;
126
127 BB Inst8_BB
(.I(buf_clk_lp0o), .T(buf_clk_lp0t), .O(buf_clk_lp0i), .B(clk_lp0))
128 /* synthesis IO_TYPE="MIPI_LP" */;
129
130 BB Inst7_BB3
(.I(buf_data_lp1o[3]), .T(buf_data_lp1t[3]), .O(buf_data_lp1i[3]),
131 .B(data_lp1[3]))
132 /* synthesis IO_TYPE="MIPI_LP" */;
133
134 BB Inst7_BB2
(.I(buf_data_lp1o[2]), .T(buf_data_lp1t[2]), .O(buf_data_lp1i[2]),
135 .B(data_lp1[2]))
136 /* synthesis IO_TYPE="MIPI_LP" */;
137
138 BB Inst7_BB1
(.I(buf_data_lp1o[1]), .T(buf_data_lp1t[1]), .O(buf_data_lp1i[1]),
139 .B(data_lp1[1]))
140 /* synthesis IO_TYPE="MIPI_LP" */;
141
142 BB Inst7_BB0
(.I(buf_data_lp1o[0]), .T(buf_data_lp1t[0]), .O(buf_data_lp1i[0]),
143 .B(data_lp1[0]))
144 /* synthesis IO_TYPE="MIPI_LP" */;
145
146 BB Inst6_BB3
(.I(buf_data_lp0o[3]), .T(buf_data_lp0t[3]), .O(buf_data_lp0i[3]),
147 .B(data_lp0[3]))
148 /* synthesis IO_TYPE="MIPI_LP" */;
149
150 BB Inst6_BB2
(.I(buf_data_lp0o[2]), .T(buf_data_lp0t[2]), .O(buf_data_lp0i[2]),
151 .B(data_lp0[2]))
152 /* synthesis IO_TYPE="MIPI_LP" */;
153
154 BB Inst6_BB1
(.I(buf_data_lp0o[1]), .T(buf_data_lp0t[1]), .O(buf_data_lp0i[1]),
155 .B(data_lp0[1]))
156 /* synthesis IO_TYPE="MIPI_LP" */;
157
158 BB Inst6_BB0
(.I(buf_data_lp0o[0]), .T(buf_data_lp0t[0]), .O(buf_data_lp0i[0]),
159 .B(data_lp0[0]))
160 /* synthesis IO_TYPE="MIPI_LP" */;
161
162 IDDRX4B Inst5_IDDRX4B3
(.D(dataini_t3), .ECLK(eclko), .SCLK(sclk_t),
163 .RST(re-
set), .ALIGNWD(algnwd), .Q0(q0_3), .Q1(q1_3), .Q2(q2_3),
164 .Q3(q3_3), .Q4(q4_3), .Q5(q5_3), .Q6(q6_3), .Q7(q7_3))
;
165
166 IDDRX4B Inst5_IDDRX4B2
(.D(dataini_t2), .ECLK(eclko), .SCLK(sclk_t),
167 .RST(re-
set), .ALIGNWD(algnwd), .Q0(q0_2), .Q1(q1_2), .Q2(q2_2),
168 .Q3(q3_2), .Q4(q4_2), .Q5(q5_2), .Q6(q6_2), .Q7(q7_2))
;
169
170 IDDRX4B Inst5_IDDRX4B1
(.D(dataini_t1), .ECLK(eclko), .SCLK(sclk_t),
171 .RST(re-
set), .ALIGNWD(algnwd), .Q0(q0_1), .Q1(q1_1), .Q2(q2_1),
172 .Q3(q3_1), .Q4(q4_1), .Q5(q5_1), .Q6(q6_1), .Q7(q7_1))
;
173
174 IDDRX4B Inst5_IDDRX4B0
(.D(dataini_t0), .ECLK(eclko), .SCLK(sclk_t),
175 .RST(re-
set), .ALIGNWD(algnwd), .Q0(q0_0), .Q1(q1_0), .Q2(q2_0),
176 .Q3(q3_0), .Q4(q4_0), .Q5(q5_0), .Q6(q6_0), .Q7(q7_0))
;
177
178 defparam Inst4_CLKDIVC.DIV = "4.0" ;
179 CLKDIVC Inst4_CLKDIVC (.RST(re-
set), .CLKI(eclko), .ALIGNWD(algnwd),
180 .CDIV1(cdiv1), .CDIVX(sclk_t));
181
182 defparam Inst5_CLKDIVC.DIV = "2.0" ;
183 CLKDIVC Inst5_CLKDIVC (.RST(re-
set), .CLKI(eclko), .ALIGNWD(algnwd),
184 .CDIV1(cdiv1), .CDIVX(oclk_t));
185
186 ECLKSYNCA Inst3_ECLKSYNCA
(.ECLKI(eclki), .STOP(xstop), .ECLKO(eclko));
187
188 VHI scuba_vhi_inst (.Z(scuba_vhi));
189
190 IB Inst2_IB (.I(clk), .O(buf_clk))
191 /* synthesis IO_TYPE="MIPI" */;
192
193 IB Inst1_IB3 (.I(datain[3]), .O(buf_dataini3))
194 /* synthesis IO_TYPE="MIPI" */;
195
196 IB Inst1_IB2 (.I(datain[2]), .O(buf_dataini2))
197 /* synthesis IO_TYPE="MIPI" */;
198
199 IB Inst1_IB1 (.I(datain[1]), .O(buf_dataini1))
200 /* synthesis IO_TYPE="MIPI" */;

```

```

201
202 IB Inst1_IB0 (.I(datain[0]), .O(buf_datain10))
203 /* synthesis IO_TYPE="MIPI" */;
204
205 assign oclk = oclk_t;
206 assign sclck = sclck_t;
207 assign q[31] = q7_3;
208 assign q[30] = q6_3;
209 assign q[29] = q5_3;
210 assign q[28] = q4_3;
211 assign q[27] = q3_3;
212 assign q[26] = q2_3;
213 assign q[25] = q1_3;
214 assign q[24] = q0_3;
215 assign q[23] = q7_2;
216 assign q[22] = q6_2;
217 assign q[21] = q5_2;
218 assign q[20] = q4_2;
219 assign q[19] = q3_2;
220 assign q[18] = q2_2;
221 assign q[17] = q1_2;
222 assign q[16] = q0_2;
223 assign q[15] = q7_1;
224 assign q[14] = q6_1;
225 assign q[13] = q5_1;
226 assign q[12] = q4_1;
227 assign q[11] = q3_1;
228 assign q[10] = q2_1;
229 assign q[9] = q1_1;
230 assign q[8] = q0_1;
231 assign q[7] = q7_0;
232 assign q[6] = q6_0;
233 assign q[5] = q5_0;
234 assign q[4] = q4_0;
235 assign q[3] = q3_0;
236 assign q[2] = q2_0;
237 assign q[1] = q1_0;
238 assign q[0] = q0_0;
239 assign rx_ready = xstart;
240 assign xstart = opensync_3;
241 assign eclk_i = buf_clk;
242 assign lock_chk = scuba_vhi;
243 assign datain_t3 = buf_datain3;
244 assign datain_t2 = buf_datain2;
245 assign datain_t1 = buf_datain1;
246 assign datain_t0 = buf_datain0;
247
248 // exemplar begin
249 // exemplar attribute FF_3 GSR ENABLED
250 // exemplar attribute FF_2 GSR ENABLED
251 // exemplar attribute FF_1 GSR ENABLED
252 // exemplar attribute FF_0 GSR ENABLED
253 // exemplar attribute Inst9_BB IO_TYPE MIPI_LP
254 // exemplar attribute Inst8_BB IO_TYPE MIPI_LP
255 // exemplar attribute Inst7_BB3 IO_TYPE MIPI_LP
256 // exemplar attribute Inst7_BB2 IO_TYPE MIPI_LP
257 // exemplar attribute Inst7_BB1 IO_TYPE MIPI_LP
258 // exemplar attribute Inst7_BB0 IO_TYPE MIPI_LP
259 // exemplar attribute Inst6_BB3 IO_TYPE MIPI_LP
260 // exemplar attribute Inst6_BB2 IO_TYPE MIPI_LP
261 // exemplar attribute Inst6_BB1 IO_TYPE MIPI_LP
262 // exemplar attribute Inst6_BB0 IO_TYPE MIPI_LP
263 // exemplar attribute Inst2_IB IO_TYPE MIPI
264 // exemplar attribute Inst1_IB3 IO_TYPE MIPI
265 // exemplar attribute Inst1_IB2 IO_TYPE MIPI
266 // exemplar attribute Inst1_IB1 IO_TYPE MIPI
267 // exemplar attribute Inst1_IB0 IO_TYPE MIPI
268 // exemplar end
269
270
271 endmodule

```

7. circuitvalley_mipi_csi_receiver_FPGA/FPGA/ mipi_rx_lane_aligner.v (104 lines)

```

1 `timescale 1ns/1ns
2
3 /*
4 MIPI CSI RX to Parallel Bridge (c) by Gaurav Singh www.Cir-
5 cuitValley.com
6 MIPI CSI RX to Parallel Bridge is licensed under a
7 Creative Commons Attribution 3.0 Unported License.
8
9 You should have received a copy of the license along with this
10 work. If not, see <http://creativecommons.org/licenses/by/
11 3.0/>.
12 */
13 /*
14 Receives multiple lane, byte aligned data from mipi rx byte
15 aligner @mipi byte clock outputs lane aligned data
16
17 in a multi-lane mipi bus, data on different lane may appear at
18 different offset so this module will wait till of the all lanes have
19 valid output
20 start outputting lane aligned data so byte x from all the
21 lanes outputted at same timescale
22 */
23
24

```

```

19 module mipi_rx_lane_aligner ( clk_i, re-
20 set_i,
21 bytes_valid_i,
22 byte_i,
23 lane_valid_o,
24 lane_byte_o);
25
26 input reset_i;
27
28 localparam [3:0]ALIGN_DEPTH = 4'h3; //how many byte misalign-
29 ment is allowed, whole package length must be also longer than this
30 localparam [2:0]LANES = 3'h4; //mipi csi have 2 or 4
31 lanes
32
33 input clk_i;
34 input [(LANES-1):0]bytes_valid_i;
35 input [(8*LANES-1):0]byte_i;
36 output reg [(8*LANES-1):0]lane_byte_o;
37 output reg lane_valid_o;
38
39 reg [(8*LANES)-1:0]last_bytes[(ALIGN_DEPTH-1):0];
40 reg [(LANES-1):0]last_bytes_valid;
41 reg [3:0]sync_byte_index[(LANES-1):0];
42 reg lane_valid_reg;
43
44 reg [3:0]i;
45 reg [3:0]last_sync; //which lane was delayed most , which re-
46 ceived sync byte at last
47 // TODO: Find why async reset with !(|bytes_valid_i) cause
48 whole system reset;
49
50 always @(posedge clk_i)
51 begin
52 if (reset_i || (!lane_valid_o && (!|bytes_valid_i)))
53 begin
54 lane_valid_o <= 1'h0;
55 last_bytes_valid <= 0;
56
57 for (i= 4'h0; i < ALIGN_DEPTH; i = i + 1'h1)
58 begin
59 last_bytes[i] <= 8'h0;
60 end
61
62 for (i= 4'h0; i < LANES; i = i + 1'h1)
63 begin
64 sync_byte_index[i] = 0;
65 end
66
67 lane_byte_o <= 0;
68 lane_valid_reg <= 1'h0;
69 last_sync <= 0;
70
71 end
72 else
73 begin
74 last_bytes[0] <= byte_i;
75
76 for (i= 4'h1; i < ALIGN_DEPTH; i = i + 1'h1)
77 begin
78 last_bytes[i] <= last_bytes[i-1'h1];
79 end
80
81 last_bytes_valid <= bytes_valid_i;
82 lane_valid_o <= last_bytes_valid[last_sync]; //
83 one clock delay to the last packet
84
85 if ((!lane_valid_o) && (!|bytes_valid_i))
86 begin
87
88 for (i= 4'h0; i < LANES; i = i + 1'h1)
89 begin
90 if (!bytes_valid_i[i])
91 begin
92 last_sync = i; //
93 which lane was last
94 sync_byte_index[i] <=
95 sync_byte_index[i] + 1'b1; //count delay of each sync, first one
96 will be 0 delay last will max
97 end
98 end
99 end
100
101 lane_byte_o <=
102 {last_bytes[sync_byte_index[last_sync] - sync_byte_index[3]][31:24],
103 last_bytes[sync_byte_index[last_sync] - sync_byte_index[2]][23:16],
104 last_bytes[sync_byte_index[last_sync] - sync_byte_index[1]][15:8],
105 last_bytes[sync_byte_index[last_sync] - sync_byte_index[0]][7:0]};
106 end

```



```

102 end
103
104 endmodule

```

8. circuitvalley_mipi_csi_receiver_FPGA/FPGA/ mipi_rx_raw10_depacker.v (78 lines)

```

1 `timescale 1ns/1ns
2
3 /*
4 MIPI CSI RX to Parallel Bridge (c) by Gaurav Singh www.Cir-
5 cuitValley.com
6 MIPI CSI RX to Parallel Bridge is licensed under a
7 Creative Commons Attribution 3.0 Unported License.
8
9 You should have received a copy of the license along with this
10 work. If not, see <http://creativecommons.org/licenses/by/
11 3.0/>.
12 */
13 /*
14 Receives 4 lane raw mipi bytes from packet decoder, rearrange
15 bytes to output 4 pixel 10bit each
16 output is one clock cycle delayed, because the way , MIPI
17 RAW10 is packed
18 output come in group of 5x40bit chunk on each clock cycle,
19 output_valid_o remains active only while 20 pixel chunk is output-
20 ted
21 */
22 module mipi_rx_raw10_depacker( clk_i,
23 data_valid_i,
24 data_i,
25 output_valid_o,
26 output_o);
27
28 localparam [2:0]BYTES_PERPACK = 3'h5; // RAW 10 is packed
29 <Sample0[9:2]> <Sample1[9:2]> <Sample2[9:2]> <Sample3[9:2]>
30 <Sample0[1:0],Sample1[1:0],Sample2[1:0],Sample3[1:0]>
31
32 input clk_i;
33 input data_valid_i;
34 input [31:0]data_i;
35 output reg output_valid_o;
36 output reg [39:0]output_o;
37
38 reg [7:0]offset;
39 reg [2:0]byte_count;
40 reg [31:0]last_data_i;
41
42 wire [63:0]word;
43
44 assign word = {data_i,last_data_i}; //would need last bytes as
45 well as current data to get full 4 pixel
46
47 always @(posedge clk_i)
48 begin
49     if (data_valid_i)
50     begin
51         last_data_i <= data_i;
52         //RAW 10 , Byte1 -> Byte2 -> Byte3 -> Byte4 ->
53         [LSbB1[1:0]LSbB2[1:0]LSbB3[1:0]LSbB4[1:0]]
54         output_o[39:30] <= {word [(offset + 7) -:
55 8], word [(offset + 39) -:2]}; //lane 1
56         output_o[29:20] <= {word [(offset + 15) -:
57 8], word [(offset + 37) -:2]};
58         output_o[19:10] <= {word [(offset + 23) -:
59 8], word [(offset + 35) -:2]};
60         output_o[9:0] <= {word [(offset + 31) -:
61 8], word [(offset + 33) -:2]}; //lane 4
62
63         if (byte_count < (BYTES_PERPACK))
64         begin
65             byte_count <= byte_count + 1'd1;
66             if (byte_count )
67             begin
68                 offset <= ((offset + 8'd8) &
69 8'h1F);
70                 output_valid_o <= 1'h1;
71             end
72         end
73     end
74     else
75     begin
76         offset <= 8'h0;
77         byte_count <= 4'b1; //this
78         byte is the first byte
79         output_valid_o <= 1'h0;
80     end
81 end
82 endmodule

```

```

68 else
69 begin
70     output_o <= 40'h0;
71     last_data_i <= 1'h0;
72     offset <= 8'h0;
73     byte_count <= 3'b0;
74     output_valid_o <= 1'h0;
75 end
76 end
77
78 endmodule

```

9. circuitvalley_mipi_csi_receiver_FPGA/FPGA/ output_reformatter.v (53 lines)

```

1 `timescale 1ns/1ns
2
3 /*
4 MIPI CSI RX to Parallel Bridge (c) by Gaurav Singh www.Cir-
5 cuitValley.com
6 MIPI CSI RX to Parallel Bridge is licensed under a
7 Creative Commons Attribution 3.0 Unported License.
8
9 You should have received a copy of the license along with this
10 work. If not, see <http://creativecommons.org/licenses/by/
11 3.0/>.
12 */
13 /*
14 Takes 64bit 4pixel yuv input from rgb2yuv module @ mipi byte
15 clock outputs 32bit 2pixel yuv output @output_clk_i ,
16
17 output_clk_i must be generated by same way as mipi byte clock,
18 output_clk_i must be exactly double to mipi byteclock
19 This implementation of Output reformatter outputs data which
20 which meant to send out of the system to a 32bit receiver
21 depending on requirement this will be need to be adapted as
22 per the receiver
23 */
24 module output_reformatter(clk_i, //data changes on negedge
25 out-
26 clk_i, //output clock double to clk_i to get in 64bit and output
27 32bit
28 data_i,
29 data_in_val-
30 id_i,
31 output_o,
32 out-
33 put_valid_o
34 );
35
36 input clk_i;
37 input data_in_valid_i;
38 input [63:0]data_i;
39 output reg output_valid_o;
40 output reg [31:0]output_o;
41 input output_clk_i;
42
43 reg lsw; //select if lower 32bit or higher 32bit
44
45 always @(posedge output_clk_i)
46 begin
47     if (!data_in_valid_i)
48     begin
49         lsw <= 1'd0;
50         output_valid_o <= 1'd0;
51     end
52     else
53     begin
54         lsw <= !lsw;
55         output_valid_o <= 1'd1;
56         output_o <= data_i[(lsw)?6'd0:6'd32] +:32];
57     end
58 end
59 endmodule

```

10. circuitvalley_mipi_csi_receiver_FPGA/FPGA/rgb_to_yuv.v (69 lines)

```

1 `timescale 1ns/1ns
2
3 /*
4 MIPI CSI RX to Parallel Bridge (c) by Gaurav Singh www.Cir-
5 cuitValley.com
6 MIPI CSI RX to Parallel Bridge is licensed under a
7 Creative Commons Attribution 3.0 Unported License.
8
9 You should have received a copy of the license along with this
10 work. If not, see <http://creativecommons.org/licenses/by/
11 3.0/>.
12 */
13 /*
14 received 4 pixel 120bit RGB from the Debayer filter output

```

```

64bit 4pixel yuv422
15 Calculation is done based on integer YUV formula from the YUV
wiki page
16 */
17
18 module rgb_to_yuv(clk_i, //data changes on rising edge ,
latched in on falling edge
19
20 reset_i,
21 rgb_i,
22 rgb_valid_i,
23 yuv_o,
24 yuv_valid_o);
25
26 localparam PIXEL_DEPTH = 4'd10; //10bit per col-
or
27
28 localparam PIXEL_PER_CLK = 4'd4; //4pixels per clock cycle
comes , should be even
29
30 input clk_i;
31 input reset_i;
32 input [(PIXEL_DEPTH * PIXEL_PER_CLK * 3) - 1'd1]:0]rgb_i;
33 input rgb_valid_i;
34
35 output reg [(PIXEL_PER_CLK * 2 * 8) - 1'd1]:0]yuv_o;
36 output reg yuv_valid_o;
37
38 reg [7:0]Y[3:0]; // result
39 reg [7:0]U[3:0];
40 reg [7:0]V[3:0];
41
42 //from YUV wiki page full swing
43 // Y = ((77 R + 150G + 29B + 128) >>10)
44 // U = ((-43R - 84G + 127B + 128) >>10) + 128
45 // V = ((127R -106G -21B +128) >>10) + 128
46
47 reg [23:0]not_used24; //to suppress warning from the tool
48
49 always @(negedge clk_i)
50 begin
51     yuv_valid_o <= rgb_valid_i;
52
53 {not_used24,Y[0]} = (( 77 * rgb_i[110+: 10]) + (150 *
rgb_i[100+: 10]) + (29 * rgb_i[90+: 10]) + 18'd128) >> 10;
54
55 {not_used24,U[0]} = (((127 * rgb_i[90+: 10]) - (43 *
rgb_i[110+: 10]) - (84 * rgb_i[100+: 10]) + 18'd128) >> 10) +
32'd128;
56
57 {not_used24,V[0]} = (((127 * rgb_i[110+: 10]) - (106 *
rgb_i[100+: 10]) - (21 * rgb_i[90+: 10]) + 18'd128) >> 10) +
32'd128;
58
59 {not_used24,Y[1]} = (( 77 * rgb_i[80+: 10]) + (150 *
rgb_i[70+: 10]) + (29 * rgb_i[60+: 10]) + 18'd128) >> 10;
60 //U[1] and V[1] not need to yuv422 sub sampling
61
62 {not_used24,Y[2]} = (( 77 * rgb_i[50+: 10]) + (150 *
rgb_i[40+: 10]) + (29 * rgb_i[30+: 10]) + 18'd128) >> 10;
63 {not_used24,U[2]} = (((127 * rgb_i[30+: 10]) - ( 43 *
rgb_i[50+: 10]) - (84 * rgb_i[40+: 10]) + 18'd128) >> 10) +
32'd128;
64
65 {not_used24,V[2]} = (((127 * rgb_i[50+: 10]) - (106 *
rgb_i[40+: 10]) - (21 * rgb_i[30+: 10]) + 18'd128) >> 10) +
32'd128;
66
67 {not_used24,Y[3]} = (( 77 * rgb_i[20+: 10]) + (150 *
rgb_i[10+: 10]) + (29 * rgb_i[0+: 10]) + 18'd128) >> 10;
68 //U[3] and V[3] not need to yuv422 sub sampling
69
70 yuv_o <= { Y[0], U[0], Y[1], V[0], Y[2],
U[2], Y[3], V[2]};
71
72 end
73
74 endmodule

```

11. circuitvalley_mipi_csi_receiver_FPGA/FPGA/ tb_debayer_filter.v (307 lines)

```

1 `timescale 1ns/1ns
2
3 module tb_debayer_filter();
4     reg clk;
5     reg reset;
6     reg line_valid;
7     wire [39:0]bytes_o;
8     wire unpacked_valid;
9     wire [119:0]rgb_output;
10    wire output_valid;
11    wire [63:0] yuv_data;
12    wire is_yuv_valid;
13
14    wire [31:0]data_out;
15    wire lsync_out;
16    reg out_clock;
17
18    wire reset_g;
19    GSR GSR_INST (.GSR (reset_g));
20    PUR PUR_INST (.PUR (reset_g));

```

```

21 debayer_filter ins1( .clk_i(clk),
22
23 set_i(reset),
24
25 id_i(line_valid),
26
27 ),
28
29 .data_i(bytes_o
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97

```

98	sendbytes(32'h12345678);	198	sendbytes(32'h9ABC00F0);
99	sendbytes(32'h00BCDEF0);	199	sendbytes(32'hAAAAA00);
100	sendbytes(32'h12005678);	200	
101	sendbytes(32'h9ABC00F0);	201	sendbytes(32'h12345678);
102	sendbytes(32'hAAAAA00);	202	sendbytes(32'h00BCDEF0);
103		203	sendbytes(32'h12005678);
104	sendbytes(32'h12345678);	204	sendbytes(32'h9ABC00F0);
105	sendbytes(32'h00BCDEF0);	205	sendbytes(32'hDDDDDD00);
106	sendbytes(32'h12005678);	206	bytes_valid = 1'h0;
107	sendbytes(32'h9ABC00F0);	207	line_valid = 1'h0;
108	sendbytes(32'hDDDDDD00);	208	
109	bytes_valid = 1'h0;	209	
110	line_valid = 1'h0;		send-
111	sendbytes(32'h00000000);	bytes(32'h00000000);	
112	sendbytes(32'h00000000);	210	sendbytes(32'h00000000);
113	sendbytes(32'h00000000);	211	sendbytes(32'h00000000);
114	sendbytes(32'h00000000);	212	sendbytes(32'h00000000);
115	sendbytes(32'h00000000);	213	sendbytes(32'h00000000);
116	sendbytes(32'h00000000);	214	sendbytes(32'h00000000);
117	sendbytes(32'h00000000);	215	sendbytes(32'h00000000);
118	sendbytes(32'h00000000);	216	sendbytes(32'h00000000);
119		217	sendbytes(32'h0);
120	sendbytes(32'h0);	218	sendbytes(32'h0);
121	sendbytes(32'h0);	219	bytes_valid = 1'h1;
122	bytes_valid = 1'h1;	220	line_valid = 1'h1;
123	line_valid = 1'h1;	221	sendbytes(32'h12345678);
124	sendbytes(32'h12345678);	222	sendbytes(32'h00BCDEF0);
125	sendbytes(32'h00BCDEF0);	223	sendbytes(32'h12005678);
126	sendbytes(32'h12005678);	224	sendbytes(32'h9ABC00F0);
127	sendbytes(32'h9ABC00F0);	225	sendbytes(32'hBBBBBB00);
128	sendbytes(32'hBBBBBB00);	226	
129		227	sendbytes(32'h12345678);
130	sendbytes(32'h12345678);	228	sendbytes(32'h00BCDEF0);
131	sendbytes(32'h00BCDEF0);	229	sendbytes(32'h12005678);
132	sendbytes(32'h12005678);	230	sendbytes(32'h9ABC00F0);
133	sendbytes(32'h9ABC00F0);	231	sendbytes(32'hAAAAA00);
134	sendbytes(32'hAAAAA00);	232	
135		233	sendbytes(32'h12345678);
136	sendbytes(32'h12345678);	234	sendbytes(32'h00BCDEF0);
137	sendbytes(32'h00BCDEF0);	235	sendbytes(32'h12005678);
138	sendbytes(32'h12005678);	236	sendbytes(32'h9ABC00F0);
139	sendbytes(32'h9ABC00F0);	237	sendbytes(32'hDDDDDD00);
140	sendbytes(32'hDDDDDD00);	238	bytes_valid = 1'h0;
141	bytes_valid = 1'h0;	239	line_valid = 1'h0;
142	line_valid = 1'h0;	240	
143		241	
144	sendbytes(32'h00000000);	sendbytes(32'h00000000);	
145	sendbytes(32'h00000000);	242	sendbytes(32'h00000000);
146	sendbytes(32'h00000000);	243	sendbytes(32'h00000000);
147	sendbytes(32'h00000000);	244	sendbytes(32'h00000000);
148	sendbytes(32'h00000000);	245	sendbytes(32'h00000000);
149	sendbytes(32'h00000000);	246	sendbytes(32'h00000000);
150	sendbytes(32'h00000000);	247	sendbytes(32'h00000000);
151	sendbytes(32'h00000000);	248	sendbytes(32'h00000000);
152	sendbytes(32'h0);	249	sendbytes(32'h0);
153	sendbytes(32'h0);	250	sendbytes(32'h0);
154	bytes_valid = 1'h1;	251	bytes_valid = 1'h1;
155	line_valid = 1'h1;	252	line_valid = 1'h1;
156	sendbytes(32'h12345678);	253	sendbytes(32'h12345678);
157	sendbytes(32'h00BCDEF0);	254	sendbytes(32'h00BCDEF0);
158	sendbytes(32'h12005678);	255	sendbytes(32'h12005678);
159	sendbytes(32'h9ABC00F0);	256	sendbytes(32'h9ABC00F0);
160	sendbytes(32'hBBBBBB00);	257	sendbytes(32'hBBBBBB00);
161		258	
162	sendbytes(32'h12345678);	259	sendbytes(32'h12345678);
163	sendbytes(32'h00BCDEF0);	260	sendbytes(32'h00BCDEF0);
164	sendbytes(32'h12005678);	261	sendbytes(32'h12005678);
165	sendbytes(32'h9ABC00F0);	262	sendbytes(32'h9ABC00F0);
166	sendbytes(32'hAAAAA00);	263	sendbytes(32'hAAAAA00);
167		264	
168	sendbytes(32'h12345678);	265	sendbytes(32'h12345678);
169	sendbytes(32'h00BCDEF0);	266	sendbytes(32'h00BCDEF0);
170	sendbytes(32'h12005678);	267	sendbytes(32'h12005678);
171	sendbytes(32'h9ABC00F0);	268	sendbytes(32'h9ABC00F0);
172	sendbytes(32'hDDDDDD00);	269	sendbytes(32'hDDDDDD00);
173	bytes_valid = 1'h0;	270	bytes_valid = 1'h0;
174	line_valid = 1'h0;	271	line_valid = 1'h0;
175		272	
176		273	
177	bytes(32'h00000000);	sendbytes(32'h00000000);	
178	sendbytes(32'h00000000);	274	sendbytes(32'h00000000);
179	sendbytes(32'h00000000);	275	sendbytes(32'h00000000);
180	sendbytes(32'h00000000);	276	sendbytes(32'h00000000);
181	sendbytes(32'h00000000);	277	sendbytes(32'h00000000);
182	sendbytes(32'h00000000);	278	sendbytes(32'h00000000);
183	sendbytes(32'h00000000);	279	sendbytes(32'h00000000);
184	sendbytes(32'h00000000);	280	sendbytes(32'h00000000);
185	sendbytes(32'h0);	281	sendbytes(32'h0);
186	sendbytes(32'h0);	282	sendbytes(32'h0);
187	bytes_valid = 1'h1;	283	bytes_valid = 1'h1;
188	line_valid = 1'h1;	284	line_valid = 1'h1;
189	sendbytes(32'h12345678);	285	sendbytes(32'h12345678);
190	sendbytes(32'h00BCDEF0);	286	sendbytes(32'h00BCDEF0);
191	sendbytes(32'h12005678);	287	sendbytes(32'h12005678);
192	sendbytes(32'h9ABC00F0);	288	sendbytes(32'h9ABC00F0);
193	sendbytes(32'hBBBBBB00);	289	sendbytes(32'hBBBBBB00);
194		290	
195	sendbytes(32'h12345678);	291	sendbytes(32'h12345678);
196	sendbytes(32'h00BCDEF0);	292	sendbytes(32'h00BCDEF0);
197	sendbytes(32'h12005678);	293	sendbytes(32'h12005678);

```

294         sendbytes(32'h9ABC00F0);
295         sendbytes(32'hAAAAA000);
296
297         sendbytes(32'h12345678);
298         sendbytes(32'h00BCDEF0);
299         sendbytes(32'h12005678);
300         sendbytes(32'h9ABC00F0);
301         sendbytes(32'hDDDDDD00);
302         bytes_valid = 1'h0;
303         line_valid = 1'h0;
304
305     end
306
307 endmodule

```

12. circuitvalley_mipi_csi_receiver_FPGA/FPGA/ tb_line_ram_dp_tmpl.v (92 lines)

```

1 //Verilog testbench template generated by SCUBA Diamond (64-
bit) 3.11.0.396.4
2 `timescale 1 ns / 1 ps
3 module tb_ram;
4     reg [9:0] WrAddress = 10'b0;
5     reg [9:0] RdAddress = 10'b0;
6     reg [39:0] Data = 40'b0;
7     reg WE = 0;
8     reg RdClock = 0;
9     reg RdClockEn = 0;
10    reg Reset = 0;
11    reg WrClock = 0;
12    reg WrClockEn = 0;
13    wire [39:0] Q;
14
15    integer i0 = 0, i1 = 0, i2 = 0, i3 = 0, i4 = 0, i5 = 0, i6 =
0, i7 = 0, i8 = 0, i9 = 0;
16
17    GSR GSR_INST (.GSR(1'b1));
18    PUR PUR_INST (.PUR(1'b1));
19
20    line_ram_dp u1 (.WrAd-
dress(WrAddress), .RdAddress(RdAddress), .Data(Data),
21    .WE(WE), .Rd-
Clock(RdClock), .RdClockEn(RdClockEn), .Reset(Reset),
22    .WrClock(WrClock), .WrClockEn(WrClockEn), .Q(Q)
23    );
24
25    initial
26    begin
27        WrAddress <= 0;
28        #100;
29        @(Reset == 1'b0);
30        for (i1 = 0; i1 < 2054; i1 = i1 + 1) begin
31            @(posedge WrClock);
32            #1 WrAddress <= WrAddress + 1'b1;
33        end
34    end
35    initial
36    begin
37        RdAddress <= 0;
38        #100;
39        @(Reset == 1'b0);
40        for (i2 = 0; i2 < 2054; i2 = i2 + 1) begin
41            @(posedge RdClock);
42            #1 RdAddress <= RdAddress + 1'b1;
43        end
44    end
45    initial
46    begin
47        Data <= 5;
48        #100;
49        @(Reset == 1'b0);
50        for (i3 = 0; i3 < 1027; i3 = i3 + 1) begin
51            @(posedge WrClock);
52            #1 Data <= Data + 1'b1;
53        end
54    end
55
56    initial
57    begin
58        WE <= 1'b0;
59        @(Reset == 1'b0);
60        for (i4 = 0; i4 < 1034; i4 = i4 + 1) begin
61            @(posedge WrClock);
62            #1 WE <= 1'b1;
63        end
64        WE <= 1'b0;
65    end
66    always
67    #5.00 RdClock <= ~ RdClock;
68
69    initial
70    begin
71        RdClockEn <= 1'b0;
72        #100;
73        @(Reset == 1'b0);
74        RdClockEn <= 1'b1;
75    end
76    initial
77    begin

```

```

78        Reset <= 1'b1;
79        #100;
80        Reset <= 1'b0;
81    end
82    always
83    #5.00 WrClock <= ~ WrClock;
84
85    initial
86    begin
87        WrClockEn <= 1'b0;
88        #100;
89        @(Reset == 1'b0);
90        WrClockEn <= 1'b1;
91    end
92 endmodule

```

13. circuitvalley_mipi_csi_receiver_FPGA/FPGA/ tb_mipi_bridge.v (170 lines)

```

1 `timescale 1ns/1ns
2
3 module tb_mipi_bridge();
4     reg clk;
5     reg reset;
6     reg mipi_clk;
7     wire clk_lpp;
8     wire clk_lpn;
9     wire [3:0]data_lpp;
10    wire [3:0]data_lpn;
11
12    reg clk_lpp_r;
13    reg clk_lpn_r;
14    reg [3:0]data_lpp_r;
15    reg [3:0]data_lpn_r;
16
17    reg [3:0]mipi_data;
18
19    wire pclk;
20    wire [31:0]data;
21    wire fsync;
22    wire lsync;
23    wire reset_g;
24    GSR GSR_INST (.GSR (reset_g));
25    PUR PUR_INST (.PUR (reset_g));
26
27    assign clk_lpp = clk_lpp_r;
28    assign clk_lpn = clk_lpn_r;
29    assign data_lpp = data_lpp_r;
30    assign data_lpn = data_lpn_r;
31
32    mipi_bridge ins1(
33        .clk_i(clk),
34        .reset_in(reset),
35        .mipi_clk_in(mipi_clk),
36        .mipi_clk_lpp_io(clk_l
37        .mipi_clk_lpn_io(clk_l
38        .mipi_data_in(mipi_data
39        .mipi_data_lpp_io(data_
40        .mipi_data_lpn_io(data_
41        .clk_out(pclk),
42        .data_out(data),
43        .fsync_out(fsync),
44        .lsync_out(lsync),
45        .debug_E3(),
46        .debug_F3(),
47        .debug_B1(),
48        .debug_D3(),
49        .byte_c-
lock_debug_H6());
50
51
52
53    initial be-
gin
54    //generate 90 phase clock and slow sync clock
55    clk = 1'b0;
56    always begin
57        #6 clk = ~clk;
58    end
59
60    task send_mipi_frame;
61        reg [15:0]i= 16'd0;
62        begin
63            clk_lpp_r = 0;
64            clk_lpn_r = 0;
65            #5
66            send_mipi_clock();
67            send_mipi_clock();
68            send_mipi_clock();
69            send_mipi_clock();
70            send_mipi_clock();
71            send_mipi_clock();
72            send_mipi_clock();
73            send_mipi_clock();

```

```

74         send_mipi_clock();
75         send_mipi_clock();
76         for (i= 16'd0; i <16'd10 ; i = i +1'd1)
77             begin
78                 send_line();
79                 #20;
80             end
81         send_mipi_clock();
82         send_mipi_clock();
83         send_mipi_clock();
84         send_mipi_clock();
85         send_mipi_clock();
86         send_mipi_clock();
87         send_mipi_clock();
88         send_mipi_clock();
89         send_mipi_clock();
90         #5
91         clk_lpp_r = 1;
92         clk_lpn_r = 1;
93
94     end
95 endtask
96
97 task send_line;
98     reg[15:0]i = 16'b0;
99     begin
100
101         send_mipi_clock();
102         send_mipi_clock();
103         send_mipi_clock();
104         data_lpp_r = 4'b0;
105         send_mipi_clock();
106         send_mipi_clock();
107         send_mipi_clock();
108         data_lpn_r = 4'b0;
109         send_mipi_byte(32'hB8B8B8B8);
110         send_mipi_byte(32'h2B600900);
111         for (i= 16'b0; i < 16'd120 ; i = i+1)
112             begin
113                 send_mipi_byte(32'h00FF55AA);
114                 send_mipi_byte(32'h0000FF55);
115                 send_mipi_byte(32'hAA0000FF);
116                 send_mipi_byte(32'h55AA0000);
117                 send_mipi_byte(32'hFF55AA00);
118             end
119         end
120         send_mipi_clock();
121         send_mipi_clock();
122         data_lpp_r = 4'b1111;
123         data_lpn_r = 4'b1111;
124
125     end
126 endtask
127
128 task send_mipi_byte;
129     input [31:0]bytes;
130     reg [7:0] i = 8'b0;
131     begin
132         for(i = 8'b0; i < 8'h8; i = i+2) begin
133             #1;
134             mipi_data = {bytes[i], bytes[8'd8 +
135 i] ,bytes[8'd16 + i] , bytes[8'd24 + i]};
136             #1;
137             mipi_clk = 1'b1;
138             #1;
139             mipi_data = {bytes[i+1], bytes[8'd8 + i
140 + 1] ,bytes[8'd16 + i + 1] , bytes[8'd24 + i + 1]};
141             #1;
142             mipi_clk = 1'b0;
143         end
144     end
145 endtask
146
147 task send_mipi_clock;
148     begin
149         send_mipi_byte(32'h00000000);
150     end
151 endtask
152
153 initial begin
154     mipi_data = 0;
155     mipi_clk = 0;
156     data_lpp_r = 4'b1111;
157     data_lpn_r = 4'b1111;
158     reset = 0;
159     clk_lpp_r = 1;
160     clk_lpn_r = 1;
161     #20
162     reset = 1;
163     #200
164     send_mipi_frame();
165     #200
166     send_mipi_frame();
167     #200
168     send_mipi_frame();
169 end
170 endmodule

```

14. circuitvalley_mipi_csi_receiver_FPGA/FPGA/ tb_mipi_csi_packet_decoder.v (70 lines)

```

1 `timescale 1ns/1ns
2
3 module tb_mipi_csi_packet_decoder;
4     reg clk;
5     reg reset;
6     reg input_valid;
7     reg [31:0]bytes_i;
8     wire [31:0]bytes_o;
9     wire synced;
10    wire [31:0]packet_length;
11
12    wire reset_g;
13    GSR GSR_INST (.GSR (reset_g));
14    PUR PUR_INST (.PUR (reset_g));
15
16    mipi_csi_packet_decoder dec1(.clk_i(clk),
17                                .dat
18                                .dat
19                                .out-
20                                .dat
21                                .pac
22                                ket_length(packet_length));
23
24    task sendbytes;
25        input [31:0]bytes;
26        begin
27            bytes_i = bytes;
28            clk = 1'b1;
29            #4
30            clk = 1'b0;
31            #4;
32        end
33    endtask
34
35    task sendpacket;
36        reg [31:0]i;
37        for ( i= 32'b0; i < 32'h980; i = i + 4)
38            begin
39                sendbytes(i*10000);
40            end
41    endtask
42
43    initial begin
44        clk = 1'b0;
45        input_valid = 1'h0;
46        #50
47        reset = 1'b0;
48        sendbytes(32'h00000000);
49        sendbytes(32'h00000000);
50        sendbytes(32'h00000000);
51        sendbytes(32'h00000000);
52        input_valid = 1'h1;
53        sendbytes(32'h00000000);
54        sendbytes(32'h00000000);
55        sendbytes(32'hB8B8B8B8);
56        sendbytes(32'hAB09602B);
57        sendpacket();
58        input_valid = 1'h0;
59        sendbytes(32'h00000000);
60        sendbytes(32'h00000000);
61        sendbytes(32'h00000000);
62        sendbytes(32'h00000000);
63        sendbytes(32'h00000000);
64        sendbytes(32'h00000000);
65        sendbytes(32'h00000000);
66
67        reset = 1'b1;
68    end
69
70 endmodule

```

15. circuitvalley_mipi_csi_receiver_FPGA/FPGA/ tb_mipi_rx_raw10_repacker.v (66 lines)

```

1 `timescale 1ns/1ns
2
3 module tb_mipi_rx_raw10_repacker();
4
5     reg clk;
6     reg bytes_valid;
7     reg [31:0]bytes_i;
8     wire [39:0]bytes_o;
9     wire synced;
10
11    wire reset_g;
12    GSR GSR_INST (.GSR (reset_g));
13    PUR PUR_INST (.PUR (reset_g));
14
15    mipi_rx_raw10_repacker ins1( .clk_i(clk),
16                                .data_val-

```

```

id_i(bytes_valid),
17
),
18
put_valid_o(synced),
19
put_o(bytes_o));
20
21 task sendbytes;
22     input [31:0]bytes;
23     begin
24         bytes_i = bytes;
25         clk = 1'b0;
26         #4
27         clk = 1'b1;
28         #4;
29     end
30 endtask
31
32 initial begin
33     clk = 1'b0;
34     bytes_valid = 4'h0;
35     #50
36     sendbytes(32'h0);
37     sendbytes(32'h0);
38     bytes_valid = 1'h1;
39     sendbytes(32'h12345678);
40     sendbytes(32'h00BCDEF0);
41     sendbytes(32'h12005678);
42     sendbytes(32'h9ABC00F0);
43     sendbytes(32'hBBBBBB00);
44
45     sendbytes(32'h12345678);
46     sendbytes(32'h00BCDEF0);
47     sendbytes(32'h12005678);
48     sendbytes(32'h9ABC00F0);
49     sendbytes(32'hAAAAAA00);
50
51     sendbytes(32'h12345678);
52     sendbytes(32'h00BCDEF0);
53     sendbytes(32'h12005678);
54     sendbytes(32'h9ABC00F0);
55     sendbytes(32'hDDDDDD00);
56     bytes_valid = 1'h0;
57     sendbytes(32'h00000000);
58     sendbytes(32'h00000000);
59     sendbytes(32'h00000000);
60     sendbytes(32'h00000000);
61     sendbytes(32'h00000000);
62     sendbytes(32'h00000000);
63     sendbytes(32'h00000000);
64     sendbytes(32'h00000000);
65 end
66 endmodule

```

16. circuitvalley_mipi_csi_receiver_FPGA/FPGA/ tb_rx_byte_aligner.v (136 lines)

```

1 `timescale 1ns/1ns
2
3 module tb_rx_byte_aligner;
4
5     reg clk;
6     reg reset;
7     reg [7:0]byte_i;
8     wire [7:0]byte_o;
9     wire synced;
10
11 wire reset_g;
12 GSR GSR_INST (.GSR (reset_g));
13 PUR PUR_INST (.PUR (reset_g));
14
15 rx_byte_aligner inst1( .clk_i(clk),
16
17 set_i(reset),
18
19 ,
20
21 ,
22 id_o(synced));
23
24 task sendbyte;
25     input [7:0]byte;
26     begin
27         byte_i = byte;
28         clk = 1'b1;
29         #4
30         clk = 1'b0;
31         #4;
32     end
33 endtask
34
35 initial begin
36     clk = 1'b1;
37     reset = 1'b1;
38     #50
39     reset = 1'b0;
40     sendbyte(8'h00);
41
42     .data_i(bytes_i
43
44     .out-
45
46     .out-
47
48     .re-
49
50     .byte_i(byte_i)
51
52     .byte_o(byte_o)
53
54     .byte_val-
55
56     .re-
57
58     .byte_i(byte_i)
59
60     .byte_o(byte_o)
61
62     .byte_val-
63
64     .re-
65
66     .byte_i(byte_i)
67
68     .byte_o(byte_o)
69
70     .byte_val-
71
72     .re-
73
74     .byte_i(byte_i)
75
76     .byte_o(byte_o)
77
78     .byte_val-
79
80     .re-
81
82     .byte_i(byte_i)
83
84     .byte_o(byte_o)
85
86     .byte_val-
87
88     .re-
89
90     .byte_i(byte_i)
91
92     .byte_o(byte_o)
93
94     .byte_val-
95
96     .re-
97
98     .byte_i(byte_i)
99
100    .byte_o(byte_o)
101
102    .byte_val-
103
104    .re-
105
106    .byte_i(byte_i)
107
108    .byte_o(byte_o)
109
110    .byte_val-
111
112    .re-
113
114    .byte_i(byte_i)
115
116    .byte_o(byte_o)
117
118    .byte_val-
119
120    .re-
121
122    .byte_i(byte_i)
123
124    .byte_o(byte_o)
125
126    .byte_val-
127
128    .re-
129
130    .byte_i(byte_i)
131
132    .byte_o(byte_o)
133
134    .byte_val-
135
136    .re-
137
138    .byte_i(byte_i)
139
140    .byte_o(byte_o)
141
142    .byte_val-
143
144    .re-
145
146    .byte_i(byte_i)
147
148    .byte_o(byte_o)
149
150    .byte_val-
151
152    .re-
153
154    .byte_i(byte_i)
155
156    .byte_o(byte_o)
157
158    .byte_val-
159
160    .re-
161
162    .byte_i(byte_i)
163
164    .byte_o(byte_o)
165
166    .byte_val-
167
168    .re-
169
170    .byte_i(byte_i)
171
172    .byte_o(byte_o)
173
174    .byte_val-
175
176    .re-
177
178    .byte_i(byte_i)
179
180    .byte_o(byte_o)
181
182    .byte_val-
183
184    .re-
185
186    .byte_i(byte_i)
187
188    .byte_o(byte_o)
189
190    .byte_val-
191
192    .re-
193
194    .byte_i(byte_i)
195
196    .byte_o(byte_o)
197
198    .byte_val-
199
200    .re-
201
202    .byte_i(byte_i)
203
204    .byte_o(byte_o)
205
206    .byte_val-
207
208    .re-
209
210    .byte_i(byte_i)
211
212    .byte_o(byte_o)
213
214    .byte_val-
215
216    .re-
217
218    .byte_i(byte_i)
219
220    .byte_o(byte_o)
221
222    .byte_val-
223
224    .re-
225
226    .byte_i(byte_i)
227
228    .byte_o(byte_o)
229
230    .byte_val-
231
232    .re-
233
234    .byte_i(byte_i)
235
236    .byte_o(byte_o)
237
238    .byte_val-
239
240    .re-
241
242    .byte_i(byte_i)
243
244    .byte_o(byte_o)
245
246    .byte_val-
247
248    .re-
249
250    .byte_i(byte_i)
251
252    .byte_o(byte_o)
253
254    .byte_val-
255
256    .re-
257
258    .byte_i(byte_i)
259
260    .byte_o(byte_o)
261
262    .byte_val-
263
264    .re-
265
266    .byte_i(byte_i)
267
268    .byte_o(byte_o)
269
270    .byte_val-
271
272    .re-
273
274    .byte_i(byte_i)
275
276    .byte_o(byte_o)
277
278    .byte_val-
279
280    .re-
281
282    .byte_i(byte_i)
283
284    .byte_o(byte_o)
285
286    .byte_val-
287
288    .re-
289
290    .byte_i(byte_i)
291
292    .byte_o(byte_o)
293
294    .byte_val-
295
296    .re-
297
298    .byte_i(byte_i)
299
300    .byte_o(byte_o)
301
302    .byte_val-
303
304    .re-
305
306    .byte_i(byte_i)
307
308    .byte_o(byte_o)
309
310    .byte_val-
311
312    .re-
313
314    .byte_i(byte_i)
315
316    .byte_o(byte_o)
317
318    .byte_val-
319
320    .re-
321
322    .byte_i(byte_i)
323
324    .byte_o(byte_o)
325
326    .byte_val-
327
328    .re-
329
330    .byte_i(byte_i)
331
332    .byte_o(byte_o)
333
334    .byte_val-
335
336    .re-
337
338    .byte_i(byte_i)
339
340    .byte_o(byte_o)
341
342    .byte_val-
343
344    .re-
345
346    .byte_i(byte_i)
347
348    .byte_o(byte_o)
349
350    .byte_val-
351
352    .re-
353
354    .byte_i(byte_i)
355
356    .byte_o(byte_o)
357
358    .byte_val-
359
360    .re-
361
362    .byte_i(byte_i)
363
364    .byte_o(byte_o)
365
366    .byte_val-
367
368    .re-
369
370    .byte_i(byte_i)
371
372    .byte_o(byte_o)
373
374    .byte_val-
375
376    .re-
377
378    .byte_i(byte_i)
379
380    .byte_o(byte_o)
381
382    .byte_val-
383
384    .re-
385
386    .byte_i(byte_i)
387
388    .byte_o(byte_o)
389
390    .byte_val-
391
392    .re-
393
394    .byte_i(byte_i)
395
396    .byte_o(byte_o)
397
398    .byte_val-
399
400    .re-
401
402    .byte_i(byte_i)
403
404    .byte_o(byte_o)
405
406    .byte_val-
407
408    .re-
409
410    .byte_i(byte_i)
411
412    .byte_o(byte_o)
413
414    .byte_val-
415
416    .re-
417
418    .byte_i(byte_i)
419
420    .byte_o(byte_o)
421
422    .byte_val-
423
424    .re-
425
426    .byte_i(byte_i)
427
428    .byte_o(byte_o)
429
430    .byte_val-
431
432    .re-
433
434    .byte_i(byte_i)
435
436    .byte_o(byte_o)
437
438    .byte_val-
439
440    .re-
441
442    .byte_i(byte_i)
443
444    .byte_o(byte_o)
445
446    .byte_val-
447
448    .re-
449
450    .byte_i(byte_i)
451
452    .byte_o(byte_o)
453
454    .byte_val-
455
456    .re-
457
458    .byte_i(byte_i)
459
460    .byte_o(byte_o)
461
462    .byte_val-
463
464    .re-
465
466    .byte_i(byte_i)
467
468    .byte_o(byte_o)
469
470    .byte_val-
471
472    .re-
473
474    .byte_i(byte_i)
475
476    .byte_o(byte_o)
477
478    .byte_val-
479
480    .re-
481
482    .byte_i(byte_i)
483
484    .byte_o(byte_o)
485
486    .byte_val-
487
488    .re-
489
490    .byte_i(byte_i)
491
492    .byte_o(byte_o)
493
494    .byte_val-
495
496    .re-
497
498    .byte_i(byte_i)
499
500    .byte_o(byte_o)
501
502    .byte_val-
503
504    .re-
505
506    .byte_i(byte_i)
507
508    .byte_o(byte_o)
509
510    .byte_val-
511
512    .re-
513
514    .byte_i(byte_i)
515
516    .byte_o(byte_o)
517
518    .byte_val-
519
520    .re-
521
522    .byte_i(byte_i)
523
524    .byte_o(byte_o)
525
526    .byte_val-
527
528    .re-
529
530    .byte_i(byte_i)
531
532    .byte_o(byte_o)
533
534    .byte_val-
535
536    .re-
537
538    .byte_i(byte_i)
539
540    .byte_o(byte_o)
541
542    .byte_val-
543
544    .re-
545
546    .byte_i(byte_i)
547
548    .byte_o(byte_o)
549
550    .byte_val-
551
552    .re-
553
554    .byte_i(byte_i)
555
556    .byte_o(byte_o)
557
558    .byte_val-
559
560    .re-
561
562    .byte_i(byte_i)
563
564    .byte_o(byte_o)
565
566    .byte_val-
567
568    .re-
569
570    .byte_i(byte_i)
571
572    .byte_o(byte_o)
573
574    .byte_val-
575
576    .re-
577
578    .byte_i(byte_i)
579
580    .byte_o(byte_o)
581
582    .byte_val-
583
584    .re-
585
586    .byte_i(byte_i)
587
588    .byte_o(byte_o)
589
590    .byte_val-
591
592    .re-
593
594    .byte_i(byte_i)
595
596    .byte_o(byte_o)
597
598    .byte_val-
599
600    .re-
601
602    .byte_i(byte_i)
603
604    .byte_o(byte_o)
605
606    .byte_val-
607
608    .re-
609
610    .byte_i(byte_i)
611
612    .byte_o(byte_o)
613
614    .byte_val-
615
616    .re-
617
618    .byte_i(byte_i)
619
620    .byte_o(byte_o)
621
622    .byte_val-
623
624    .re-
625
626    .byte_i(byte_i)
627
628    .byte_o(byte_o)
629
630    .byte_val-
631
632    .re-
633
634    .byte_i(byte_i)
635
636    .byte_o(byte_o)
637
638    .byte_val-
639
640    .re-
641
642    .byte_i(byte_i)
643
644    .byte_o(byte_o)
645
646    .byte_val-
647
648    .re-
649
650    .byte_i(byte_i)
651
652    .byte_o(byte_o)
653
654    .byte_val-
655
656    .re-
657
658    .byte_i(byte_i)
659
660    .byte_o(byte_o)
661
662    .byte_val-
663
664    .re-
665
666    .byte_i(byte_i)
667
668    .byte_o(byte_o)
669
670    .byte_val-
671
672    .re-
673
674    .byte_i(byte_i)
675
676    .byte_o(byte_o)
677
678    .byte_val-
679
680    .re-
681
682    .byte_i(byte_i)
683
684    .byte_o(byte_o)
685
686    .byte_val-
687
688    .re-
689
690    .byte_i(byte_i)
691
692    .byte_o(byte_o)
693
694    .byte_val-
695
696    .re-
697
698    .byte_i(byte_i)
699
700    .byte_o(byte_o)
701
702    .byte_val-
703
704    .re-
705
706    .byte_i(byte_i)
707
708    .byte_o(byte_o)
709
710    .byte_val-
711
712    .re-
713
714    .byte_i(byte_i)
715
716    .byte_o(byte_o)
717
718    .byte_val-
719
720    .re-
721
722    .byte_i(byte_i)
723
724    .byte_o(byte_o)
725
726    .byte_val-
727
728    .re-
729
730    .byte_i(byte_i)
731
732    .byte_o(byte_o)
733
734    .byte_val-
735
736    .re-
737
738    .byte_i(byte_i)
739
740    .byte_o(byte_o)
741
742    .byte_val-
743
744    .re-
745
746    .byte_i(byte_i)
747
748    .byte_o(byte_o)
749
750    .byte_val-
751
752    .re-
753
754    .byte_i(byte_i)
755
756    .byte_o(byte_o)
757
758    .byte_val-
759
760    .re-
761
762    .byte_i(byte_i)
763
764    .byte_o(byte_o)
765
766    .byte_val-
767
768    .re-
769
770    .byte_i(byte_i)
771
772    .byte_o(byte_o)
773
774    .byte_val-
775
776    .re-
777
778    .byte_i(byte_i)
779
780    .byte_o(byte_o)
781
782    .byte_val-
783
784    .re-
785
786    .byte_i(byte_i)
787
788    .byte_o(byte_o)
789
790    .byte_val-
791
792    .re-
793
794    .byte_i(byte_i)
795
796    .byte_o(byte_o)
797
798    .byte_val-
799
800    .re-
801
802    .byte_i(byte_i)
803
804    .byte_o(byte_o)
805
806    .byte_val-
807
808    .re-
809
810    .byte_i(byte_i)
811
812    .byte_o(byte_o)
813
814    .byte_val-
815
816    .re-
817
818    .byte_i(byte_i)
819
820    .byte_o(byte_o)
821
822    .byte_val-
823
824    .re-
825
826    .byte_i(byte_i)
827
828    .byte_o(byte_o)
829
830    .byte_val-
831
832    .re-
833
834    .byte_i(byte_i)
835
836    .byte_o(byte_o)
837
838    .byte_val-
839
840    .re-
841
842    .byte_i(byte_i)
843
844    .byte_o(byte_o)
845
846    .byte_val-
847
848    .re-
849
850    .byte_i(byte_i)
851
852    .byte_o(byte_o)
853
854    .byte_val-
855
856    .re-
857
858    .byte_i(byte_i)
859
860    .byte_o(byte_o)
861
862    .byte_val-
863
864    .re-
865
866    .byte_i(byte_i)
867
868    .byte_o(byte_o)
869
870    .byte_val-
871
872    .re-
873
874    .byte_i(byte_i)
875
876    .byte_o(byte_o)
877
878    .byte_val-
879
880    .re-
881
882    .byte_i(byte_i)
883
884    .byte_o(byte_o)
885
886    .byte_val-
887
888    .re-
889
890    .byte_i(byte_i)
891
892    .byte_o(byte_o)
893
894    .byte_val-
895
896    .re-
897
898    .byte_i(byte_i)
899
900    .byte_o(byte_o)
901
902    .byte_val-
903
904    .re-
905
906    .byte_i(byte_i)
907
908    .byte_o(byte_o)
909
910    .byte_val-
911
912    .re-
913
914    .byte_i(byte_i)
915
916    .byte_o(byte_o)
917
918    .byte_val-
919
920    .re-
921
922    .byte_i(byte_i)
923
924    .byte_o(byte_o)
925
926    .byte_val-
927
928    .re-
929    .byte_i(byte_i)
930
931    .byte_o(byte_o)
932
933    .byte_val-
934
935    .re-
936
937    .byte_i(byte_i)
938
939    .byte_o(byte_o)
940
941    .byte_val-
942
943    .re-
944
945    .byte_i(byte_i)
946
947    .byte_o(byte_o)
948
949    .byte_val-
950
951    .re-
952
953    .byte_i(byte_i)
954
955    .byte_o(byte_o)
956
957    .byte_val-
958
959    .re-
960
961    .byte_i(byte_i)
962
963    .byte_o(byte_o)
964
965    .byte_val-
966
967    .re-
968
969    .byte_i(byte_i)
970
971    .byte_o(byte_o)
972
973    .byte_val-
974
975    .re-
976
977    .byte_i(byte_i)
978
979    .byte_o(byte_o)
980
981    .byte_val-
982
983    .re-
984
985    .byte_i(byte_i)
986
987    .byte_o(byte_o)
988
989    .byte_val-
990
991    .re-
992
993    .byte_i(byte_i)
994
995    .byte_o(byte_o)
996
997    .byte_val-
998
999    .re-
1000
1001    .byte_i(byte_i)
1002
1003    .byte_o(byte_o)
1004
1005    .byte_val-
1006
1007    .re-
1008
1009    .byte_i(byte_i)
1010
1011    .byte_o(byte_o)
1012
1013    .byte_val-
1014
1015    .re-
1016
1017    .byte_i(byte_i)
1018
1019    .byte_o(byte_o)
1020
1021    .byte_val-
1022
1023    .re-
1024
1025    .byte_i(byte_i)
1026
1027    .byte_o(byte_o)
1028
1029    .byte_val-
1030
1031    .re-
1032
1033    .byte_i(byte_i)
1034
1035    .byte_o(byte_o)
1036
1037    .byte_val-
1038
1039    .re-
1040
1041    .byte_i(byte_i)
1042
1043    .byte_o(byte_o)
1044
1045    .byte_val-
1046
1047    .re-
1048
1049    .byte_i(byte_i)
1050
1051    .byte_o(byte_o)
1052
1053    .byte_val-
1054
1055    .re-
1056
1057    .byte_i(byte_i)
1058
1059    .byte_o(byte_o)
1060
1061    .byte_val-
1062
1063    .re-
1064
1065    .byte_i(byte_i)
1066
1067    .byte_o(byte_o)
1068
1069    .byte_val-
1070
1071    .re-
1072
1073    .byte_i(byte_i)
1074
1075    .byte_o(byte_o)
1076
1077    .byte_val-
1078
1079    .re-
1080
1081    .byte_i(byte_i)
1082
1083    .byte_o(byte_o)
1084
1085    .byte_val-
1086
1087    .re-
1088
1089    .byte_i(byte_i)
1090
1091    .byte_o(byte_o)
1092
1093    .byte_val-
1094
1095    .re-
1096
1097    .byte_i(byte_i)
1098
1099    .byte_o(byte_o)
1100
1101    .byte_val-
1102
1103    .re-
1104
1105    .byte_i(byte_i)
1106
1107    .byte_o(byte_o)
1108
1109    .byte_val-
1110
1111    .re-
1112
1113    .byte_i(byte_i)
1114
1115    .byte_o(byte_o)
1116
1117    .byte_val-
1118
1119    .re-
1120
1121    .byte_i(byte_i)
1122
1123    .byte_o(byte_o)
1124
1125    .byte_val-
1126
1127    .re-
1128
1129    .byte_i(byte_i)
1130
1131    .byte_o(byte_o)
1132
1133    .byte_val-
1134
1135    .re-
1136
1137    .byte_i(byte_i)
1138
1139    .byte_o(byte_o)
1140
1141    .byte_val-
1142
1143    .re-
1144
1145    .byte_i(byte_i)
1146
1147    .byte_o(byte_o)
1148
1149    .byte_val-
1150
1151    .re-
1152
1153    .byte_i(byte_i)
1154
1155    .byte_o(byte_o)
1156
1157    .byte_val-
1158
1159    .re-
1160
1161    .byte_i(byte_i)
1162
1163    .byte_o(byte_o)
1164
1165    .byte_val-
1166
1167    .re-
1168
1169    .byte_i(byte_i)
1170
1171    .byte_o(byte_o)
1172
1173    .byte_val-
1174
1175    .re-
1176
1177    .byte_i(byte_i)
1178
1179    .byte_o(byte_o)
1180
1181    .byte_val-
1182
1183    .re-
1184
1185    .byte_i(byte_i)
1186
1187    .byte_o(byte_o)
1188
1189    .byte_val-
1190
1191    .re-
1192
1193    .byte_i(byte_i)
1194
1195    .byte_o(byte_o)
1196
1197    .byte_val-
1198
1199    .re-
1200
1201    .byte_i(byte_i)
1202
1203    .byte_o(byte_o)
1204
1205    .byte_val-
1206
1207    .re-
1208
1209    .byte_i(byte_i)
1210
1211    .byte_o(byte_o)
1212
1213    .byte_val-
1214
1215    .re-
1216
1217    .byte_i(byte_i)
1218
1219    .byte_o(byte_o)
1220
1221    .byte_val-
1222
1223    .re-
1224
1225    .byte_i(byte_i)
1226
1227    .byte_o(byte_o)
1228
1229    .byte_val-
1230
1231    .re-
1232
1233    .byte_i(byte_i)
1234
1235    .byte_o(byte_o)
1236
1237    .byte_val-
1238
1239    .re-
1240
1241    .byte_i(byte_i)
1242
1243    .byte_o(byte_o)
1244
1245    .byte_val-
1246
1247    .re-
1248
1249    .byte_i(byte_i)
1250
1251    .byte_o(byte_o)
1252
1253    .byte_val-
1254
1255    .re-
1256
1257    .byte_i(byte_i)
1258
1259    .byte_o(byte_o)
1260
1261    .byte_val-
1262
1263    .re-
1264
1265    .byte_i(byte_i)
1266
1267    .byte_o(byte_o)
1268
1269    .byte_val-
1270
1271    .re-
1272
1273    .byte_i(byte_i)
1274
1275    .byte_o(byte_o)
1276
1277    .byte_val-
1278
1279    .re-
1280
1281    .byte_i(byte_i)
1282
1283    .byte_o(byte_o)
1284
1285    .byte_val-
1286
1287    .re-
1288
1289    .byte_i(byte_i)
1290
1291    .byte_o(byte_o)
1292
1293    .byte_val-
1294
1295    .re-
1296
1297    .byte_i(byte_i)
1298
1299    .byte_o(byte_o)
1300
1301    .byte_val-
1302
1303    .re-
1304
1305    .byte_i(byte_i)
1306
1307    .byte_o(byte_o)
1308
1309    .byte_val-
1310
1311    .re-
1312
1313    .byte_i(byte_i)
1314
1315    .byte_o(byte_o)
1316
1317    .byte_val-
1318
1319    .re-
1320
1321    .byte_i(byte_i)
1322
1323    .byte_o(byte_o)
1324
1325    .byte_val-
1326
1327    .re-
1328
1329    .byte_i(byte_i)
1330
1331    .byte_o(byte_o)
1332
1333    .byte_val-
1334
1335    .re-
1336
1337    .byte_i(byte_i)
1338
1339    .byte_o(byte_o)
1340
1341    .byte_val-
1342
1343    .re-
1344
1345    .byte_i(byte_i)
1346
1347    .byte_o(byte_o)
1348
1349    .byte_val-
1350
1351    .re-
1352
1353    .byte_i(byte_i)
1354
1355    .byte_o(byte_o)
1356
1357    .byte_val-
1358
1359    .re-
1360
1361    .byte_i(byte_i)
1362
1363    .byte_o(byte_o)
1364
1365    .byte_val-
1366
1367    .re-
1368
1369    .byte_i(byte_i)
1370
1371    .byte_o(byte_o)
1372
1373    .byte_val-
1374
1375    .re-
1376
1377    .byte_i(byte_i)
1378
1379    .byte_o(byte_o)
1380
1381    .byte_val-
1382
1383    .re-
1384
1385    .byte_i(byte_i)
1386
1387    .byte_o(byte_o)
1388
1389    .byte_val-
1390
1391    .re-
1392
1393    .byte_i(byte_i)
1394
1395    .byte_o(byte_o)
1396
1397    .byte_val-
1398
1399    .re-
1400
1401    .byte_i(byte_i)
1402
1403    .byte_o(byte_o)
1404
1405    .byte_val-
1406
1407    .re-
1408
1409    .byte_i(byte_i)
1410
1411    .byte_o(byte_o)
1412
1413    .byte_val-
1414
1415    .re-
1416
1417    .byte_i(byte_i)
1418
1419    .byte_o(byte_o)
1420
1421    .byte_val-
1422
1423    .re-
1424
1425    .byte_i(byte_i)
1426
1427    .byte_o(byte_o)
1428
1429    .byte_val-
1430
1431    .re-
1432
1433    .byte_i(byte_i)
1434
1435    .byte_o(byte_o)
1436
1437    .byte_val-
1438
1439    .re-
1440
1441    .byte_i(byte_i)
1442
1443    .byte_o(byte_o)
1444
1445    .byte_val-
1446
1447    .re-
1448
1449    .byte_i(byte_i)
1450
1451    .byte_o(byte_o)
1452
1453    .byte_val-
1454
1455    .re-
1456
1457    .byte_i(byte_i)
1458
1459    .byte_o(byte_o)
1460
1461    .byte_val-
1462
1463    .re-
1464
1465    .byte_i(byte_i)
1466
1467    .byte_o(byte_o)
1468
1469    .byte_val-
1470
1471    .re-
1472
1473    .byte_i(byte_i)
1474
1475    .byte_o(byte_o)
1476
1477    .byte_val-
1478
1479    .re-
1480
1481    .byte_i(byte_i)
1482
1483    .byte_o(byte_o)
1484
1485    .byte_val-
1486
1487    .re-
1488
1489    .byte_i(byte_i)
1490
1491    .byte_o(byte_o)
1492
1493    .byte_val-
1494
1495    .re-
1496
1497    .byte_i(byte_i)
1498
1499    .byte_o(byte_o)
1500
1501    .byte_val-
1502
1503    .re-
1504
1505    .byte_i(byte_i)
1506
1507    .byte_o(byte_o)
1508
1509    .byte_val-
1510
1511    .re-
1512
1513    .byte_i(byte_i)
1514
1515    .byte_o(byte_o)
1516
1517    .byte_val-
1518
1519    .re-
1520
1521    .byte_i(byte_i)
1522
1523    .byte_o(byte_o)
1524
1525    .byte_val-
1526
1527    .re-
1528
1529    .byte_i(byte_i)
1530
1531    .byte_o(byte_o)
1532
1533    .byte_val-
1534
1535    .re-
1536
1537    .byte_i(byte_i)
1538
1539    .byte_o(byte_o)
1540
1541    .byte_val-
1542
1543    .re-
1544
1545    .byte_i(byte_i)
1546
1547    .byte_o(byte_o)
1548
1549    .byte_val-
1550
1551    .re-
1552
1553    .byte_i(byte_i)
1554
1555    .byte_o(byte_o)
1556
1557    .byte_val-
1558
1559    .re-
1560
1561    .byte_i(byte_i)
1562
1563    .byte_o(byte_o)
1564
1565    .byte_val-
1566
1567    .re-
1568
1569    .byte_i(byte_i)
1570
1571    .byte_o(byte_o)
1572
1573    .byte_val-
1574
1575    .re-
1576
1577    .byte_i(byte_i)
1578
1579    .byte_o(byte_o)
1580
1581    .byte_val-
1582
1583    .re-
1584
1585    .byte_i(byte_i)
1586
1587    .byte_o(byte_o)
1588
1589    .byte_val-
1590
1591    .re-
1592
1593    .byte_i(byte_i)
1594
1595    .byte_o(byte_o)
1596
1597    .byte_val-
1598
15
```

17. circuitvalley_mipi_csi_receiver_FPGA/FPGA/ tb_rx_lane_aligner.v (104 lines)

```

1 `timescale 1ns/1ns
2
3 module tb_rx_lane_aligner;
4     reg clk;
5     reg [3:0]bytes_valid;
6     reg [31:0]bytes_i;
7     wire [31:0]bytes_o;
8     wire synced;
9     reg reset;
10
11 wire reset_g;
12 GSR GSR_INST (.GSR (reset_g));
13 PUR PUR_INST (.PUR (reset_g));
14
15 rx_lane_aligner ins1( .clk_i(clk),
16
17 set_i(reset),
18
19 bytes_valid),
20
21 .byte_i(bytes_i
22 ),
23
24 .lane_val-
25 id_o(synced),
26
27 .lane_byte_o(b
28 ytes_o));
29
30 task sendbytes;
31     input [31:0]bytes;
32     begin
33         bytes_i = bytes;
34         clk = 1'b0;
35         #4
36         clk = 1'b1;
37         #4;
38     end
39 endtask
40
41 initial begin
42     clk = 1'b0;
43     bytes_valid = 4'h0;
44     #50
45
46     sendbytes(32'h00000000);
47     reset = 1'h1;
48     sendbytes(32'h00000000);
49     reset = 1'h0;
50     sendbytes(32'h00000000);
51     sendbytes(32'h00000000);
52     bytes_valid[1] = 1'h1;
53     sendbytes(32'h0000B800);
54     sendbytes(32'h00001100);
55     sendbytes(32'h00002200);
56
57     sendbytes(32'h00003300);
58     bytes_valid[2] = 1'h1;
59     bytes_valid[3] = 1'h1;
60     sendbytes(32'hB8B84400);
61     bytes_valid[0] = 1'h1;
62     sendbytes(32'h111155B8);
63     sendbytes(32'h22226611);
64     sendbytes(32'h33337722);
65     sendbytes(32'h44448833);
66     bytes_valid[1] = 1'h0;
67     sendbytes(32'h55556644);
68     sendbytes(32'h66667755);
69     sendbytes(32'h77778866);
70
71     sendbytes(32'h88889977);
72     bytes_valid[2] = 1'h0;
73     bytes_valid[3] = 1'h0;
74
75     sendbytes(32'h9999AA88);
76     bytes_valid[0] = 1'h0;
77     sendbytes(32'h00000000);
78     sendbytes(32'h00000000);
79     sendbytes(32'h00000000);
80     sendbytes(32'h00000000);
81     bytes_valid[1] = 1'h1;
82     sendbytes(32'h0000B800);
83     sendbytes(32'h00001100);
84     bytes_valid[2] = 1'h1;
85     bytes_valid[3] = 1'h1;
86     sendbytes(32'hB8B82200);
87     bytes_valid[0] = 1'h1;
88     sendbytes(32'h111133B8);
89     sendbytes(32'h22224411);
90     sendbytes(32'h33335522);
91     sendbytes(32'h44446633);
92     sendbytes(32'h55557744);
93     sendbytes(32'h66668855);
94     bytes_valid[1] = 1'h0;
95     sendbytes(32'h77778866);
96
97     sendbytes(32'h88889977);
98     bytes_valid[2] = 1'h0;
99     bytes_valid[3] = 1'h0;
100
101     sendbytes(32'h9999AA88);
102     bytes_valid[0] = 1'h0;
103
104 endmodule

```


이더넷 MAC · 10G BASE-R PCS · XGMII/GMII

What this is. A widely used open Ethernet implementation: 1G and 10G MACs, the 10G BASE-R PCS (64b/66b encode and decode, block synchronisation, BER monitor), and the GMII/XGMII interface adapters.

Why it exists. Because it is the reference point for Chapter P3. It shows what a complete MAC contains, and — equally usefully — what it does not: there is no RS-FEC and no 100G multi-lane PCS here, which is precisely the gap a new IP could fill.

Undergraduate review · EECS concepts. Framing; CRC-32; inter-frame gap; 64b/66b line coding; scrambling for DC balance; block synchronisation state machines.

Graduate · Practice note. Read `xgmii_baser_enc_64.v` and `xgmii_baser_dec_64.v` together with Chapter K2's account of 64b/66b. The two-bit sync header and the block type field are visible directly in the code, and the scrambler is in `lfsr.v` — the same LFSR structure that Chapter Z3 introduces as CRC, PRBS and scrambler at once.

12 files • 5,169 lines

API — `arp.v`

Table 613. `alexforencich_verilog-ethernet/rtl/arp.v` — module / interface / package (1개)

이름	종류
<code>arp</code>	module

Table 614. `alexforencich_verilog-ethernet/rtl/arp.v` — parameter (7개)

파라미터	형/폭	기본값
<code>DATA_WIDTH</code>		8
<code>KEEP_ENABLE</code>		(<code>DATA_WIDTH</code> >8)
<code>KEEP_WIDTH</code>		(<code>DATA_WIDTH</code> /8)
<code>CACHE_ADDR_WIDTH</code>		9
<code>REQUEST_RETRY_COUNT</code>		4
<code>REQUEST_RETRY_INTERVAL</code>		125000000*2
<code>REQUEST_TIMEOUT</code>		125000000*30

Table 615. `alexforencich_verilog-ethernet/rtl/arp.v` — localparam (1개)

상수	형/폭	값
<code>ARP_OPER_ARP_REQUEST</code>	[15:0]	16'h0001

Table 616. `alexforencich_verilog-ethernet/rtl/arp.v` — 포트 (36개)

포트	방향	형/폭
<code>clk</code>	input	wire
<code>rst</code>	input	wire

포트	방향	형/폭
s_eth_hdr_valid	input	wire
s_eth_hdr_ready	output	wire
s_eth_dest_mac	input	wire [47:0]
s_eth_src_mac	input	wire [47:0]
s_eth_type	input	wire [15:0]
s_eth_payload_axis_tdata	input	wire [DATA_WIDTH-1:0]
s_eth_payload_axis_tkeep	input	wire [KEEP_WIDTH-1:0]
s_eth_payload_axis_tvalid	input	wire
s_eth_payload_axis_tready	output	wire
s_eth_payload_axis_tlast	input	wire
s_eth_payload_axis_tuser	input	wire
m_eth_hdr_valid	output	wire
m_eth_hdr_ready	input	wire
m_eth_dest_mac	output	wire [47:0]
m_eth_src_mac	output	wire [47:0]
m_eth_type	output	wire [15:0]
m_eth_payload_axis_tdata	output	wire [DATA_WIDTH-1:0]
m_eth_payload_axis_tkeep	output	wire [KEEP_WIDTH-1:0]
m_eth_payload_axis_tvalid	output	wire
m_eth_payload_axis_tready	input	wire
m_eth_payload_axis_tlast	output	wire
m_eth_payload_axis_tuser	output	wire
arp_request_valid	input	wire
arp_request_ready	output	wire
arp_request_ip	input	wire [31:0]
arp_response_valid	output	wire
arp_response_ready	input	wire
arp_response_error	output	wire
arp_response_mac	output	wire [47:0]
local_mac	input	wire [47:0]
local_ip	input	wire [31:0]
gateway_ip	input	wire [31:0]
subnet_mask	input	wire [31:0]
clear_cache	input	wire

API — arp_cache.v**Table 617. alexforencich_verilog-ethernet/rtl/arp_cache.v — module / interface / package (1개)**

이름	종류
arp_cache	module

Table 618. alexforencich_verilog-ethernet/rtl/arp_cache.v — parameter (1개)

파라미터	형/폭	기본값
CACHE_ADDR_WIDTH		9

Table 619. alexforencich_verilog-ethernet/rtl/arp_cache.v — 포트 (14개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
query_request_valid	input	wire
query_request_ready	output	wire
query_request_ip	input	wire [31:0]
query_response_valid	output	wire
query_response_ready	input	wire
query_response_error	output	wire
query_response_mac	output	wire [47:0]
write_request_valid	input	wire
write_request_ready	output	wire
write_request_ip	input	wire [31:0]
write_request_mac	input	wire [47:0]
clear_cache	input	wire

API — arp_eth_rx.v**Table 620. alexforencich_verilog-ethernet/rtl/arp_eth_rx.v — module / interface / package (1개)**

이름	종류
arp_eth_rx	module

Table 621. alexforencich_verilog-ethernet/rtl/arp_eth_rx.v — parameter (8개)

파라미터	형/폭	기본값
DATA_WIDTH		8
KEEP_ENABLE		(DATA_WIDTH>8)
KEEP_WIDTH		(DATA_WIDTH/8)
BYTE_LANES		KEEP_ENABLE ? KEEP_WIDTH : 1
HDR_SIZE		28

파라미터	형/폭	기본값
CYCLE_COUNT		(HDR_SIZE+BYTE_LANES-1)/BYTE_LANES
PTR_WIDTH		\$clog2(CYCLE_COUNT)
OFFSET		HDR_SIZE % BYTE_LANES

Table 622. alexforencich_verilog-ethernet/rtl/arp_eth_rx.v — 포트 (30개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_eth_hdr_valid	input	wire
s_eth_hdr_ready	output	wire
s_eth_dest_mac	input	wire [47:0]
s_eth_src_mac	input	wire [47:0]
s_eth_type	input	wire [15:0]
s_eth_payload_axis_tdata	input	wire [DATA_WIDTH-1:0]
s_eth_payload_axis_tkeep	input	wire [KEEP_WIDTH-1:0]
s_eth_payload_axis_tvalid	input	wire
s_eth_payload_axis_tready	output	wire
s_eth_payload_axis_tlast	input	wire
s_eth_payload_axis_tuser	input	wire
m_frame_valid	output	wire
m_frame_ready	input	wire
m_eth_dest_mac	output	wire [47:0]
m_eth_src_mac	output	wire [47:0]
m_eth_type	output	wire [15:0]
m_arp_htype	output	wire [15:0]
m_arp_ptype	output	wire [15:0]
m_arp_hlen	output	wire [7:0]
m_arp_plen	output	wire [7:0]
m_arp_oper	output	wire [15:0]
m_arp_sha	output	wire [47:0]
m_arp_spa	output	wire [31:0]
m_arp_tha	output	wire [47:0]
m_arp_tpa	output	wire [31:0]
busy	output	wire
error_header_early_termination	output	wire

포트	방향	형/폭
error_invalid_header	output	wire

Table 623. alexforencich_verilog-ethernet/rtl/arp_eth_rx.v — #define / `define 매크로 (1개)

매크로	인자	정의
_HEADER_FIELD_	(offset, field)	\

API — arp_eth_tx.v**Table 624. alexforencich_verilog-ethernet/rtl/arp_eth_tx.v — module / interface / package (1개)**

이름	종류
arp_eth_tx	module

Table 625. alexforencich_verilog-ethernet/rtl/arp_eth_tx.v — parameter (8개)

파라미터	형/폭	기본값
DATA_WIDTH		8
KEEP_ENABLE		(DATA_WIDTH>8)
KEEP_WIDTH		(DATA_WIDTH/8)
BYTE_LANES		KEEP_ENABLE ? KEEP_WIDTH : 1
HDR_SIZE		28
CYCLE_COUNT		(HDR_SIZE+BYTE_LANES-1)/BYTE_LANES
PTR_WIDTH		\$clog2(CYCLE_COUNT)
OFFSET		HDR_SIZE % BYTE_LANES

Table 626. alexforencich_verilog-ethernet/rtl/arp_eth_tx.v — 포트 (26개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_frame_valid	input	wire
s_frame_ready	output	wire
s_eth_dest_mac	input	wire [47:0]
s_eth_src_mac	input	wire [47:0]
s_eth_type	input	wire [15:0]
s_arp_htype	input	wire [15:0]
s_arp_ptype	input	wire [15:0]
s_arp_oper	input	wire [15:0]
s_arp_sha	input	wire [47:0]
s_arp_spa	input	wire [31:0]
s_arp_tha	input	wire [47:0]

포트	방향	형/폭
s_arp_tpa	input	wire [31:0]
m_eth_hdr_valid	output	wire
m_eth_hdr_ready	input	wire
m_eth_dest_mac	output	wire [47:0]
m_eth_src_mac	output	wire [47:0]
m_eth_type	output	wire [15:0]
m_eth_payload_axis_tdata	output	wire [DATA_WIDTH-1:0]
m_eth_payload_axis_tkeep	output	wire [KEEP_WIDTH-1:0]
m_eth_payload_axis_tvalid	output	wire
m_eth_payload_axis_tready	input	wire
m_eth_payload_axis_tlast	output	wire
m_eth_payload_axis_tuser	output	wire
busy	output	wire

Table 627. alexforencich_verilog-ethernet/rtl/arp_eth_tx.v — #define / `define 매크로 (1개)

매크로	인자	정의
_HEADER_FIELD_	(offset, field)	\

API — axis_baser_rx_64.v

Table 628. alexforencich_verilog-ethernet/rtl/axis_baser_rx_64.v — module / interface / package (1개)

이름	종류
axis_baser_rx_64	module

Table 629. alexforencich_verilog-ethernet/rtl/axis_baser_rx_64.v — parameter (7개)

파라미터	형/폭	기본값
DATA_WIDTH		64
KEEP_WIDTH		(DATA_WIDTH/8)
HDR_WIDTH		2
PTP_TS_ENABLE		0
PTP_TS_FMT_TOD		1
PTP_TS_WIDTH		PTP_TS_FMT_TOD ? 96 : 64
USER_WIDTH		(PTP_TS_ENABLE ? PTP_TS_WIDTH : 0) + 1

Table 630. alexforencich_verilog-ethernet/rtl/axis_baser_rx_64.v — localparam (8개)

상수	형/폭	값
ETH_PRE	[7:0]	8'h55
XGMII_IDLE	[7:0]	8'h07

상수	형/폭	값
CTRL_IDLE	[6:0]	7'h00
O_SEQ_OS	[3:0]	4'h0
SYNC_DATA	[1:0]	2'b10
BLOCK_TYPE_CTRL	[7:0]	8'h1e
INPUT_TYPE_IDLE	[3:0]	4'd0
STATE_IDLE	[1:0]	2'd0

Table 631. alexforencich_verilog-ethernet/rtl/axis_baser_rx_64.v — 포트 (15개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
encoded_rx_data	input	wire [DATA_WIDTH-1:0]
encoded_rx_hdr	input	wire [HDR_WIDTH-1:0]
m_axis_tdata	output	wire [DATA_WIDTH-1:0]
m_axis_tkeep	output	wire [KEEP_WIDTH-1:0]
m_axis_tvalid	output	wire
m_axis_tlast	output	wire
m_axis_tuser	output	wire [USER_WIDTH-1:0]
ptp_ts	input	wire [PTP_TS_WIDTH-1:0]
cfg_rx_enable	input	wire
start_packet	output	wire [1:0]
error_bad_frame	output	wire
error_bad_fcs	output	wire
rx_bad_block	output	wire

API — axis_baser_tx_64.v**Table 632. alexforencich_verilog-ethernet/rtl/axis_baser_tx_64.v — module / interface / package (1개)**

이름	종류
axis_baser_tx_64	module

Table 633. alexforencich_verilog-ethernet/rtl/axis_baser_tx_64.v — parameter (15개)

파라미터	형/폭	기본값
DATA_WIDTH		64
KEEP_WIDTH		(DATA_WIDTH/8)
HDR_WIDTH		2
ENABLE_PADDING		1

파라미터	형/폭	기본값
ENABLE_DIC		1
MIN_FRAME_LENGTH		64
PTP_TS_ENABLE		0
PTP_TS_FMT_TOD		1
PTP_TS_WIDTH		PTP_TS_FMT_TOD ? 96 : 64
PTP_TS_CTRL_IN_TUSER		0
PTP_TAG_ENABLE		PTP_TS_ENABLE
PTP_TAG_WIDTH		16
USER_WIDTH		(PTP_TS_ENABLE ? (PTP_TAG_ENABLE ? PTP_TAG_WIDTH : 0) + (PTP...
EMPTY_WIDTH		$\$clog2(KEEP_WIDTH)$
MIN_LEN_WIDTH		$\$clog2(MIN_FRAME_LENGTH-4-KEEP_WIDTH+1)$

Table 634. alexforencich_verilog-ethernet/rtl/axis_baser_tx_64.v — localparam (7개)

상수	형/폭	값
ETH_PRE	[7:0]	8'h55
CTRL_IDLE	[6:0]	7'h00
O_SEQ_OS	[3:0]	4'h0
SYNC_DATA	[1:0]	2'b10
BLOCK_TYPE_CTRL	[7:0]	8'h1e
OUTPUT_TYPE_IDLE	[3:0]	4'd0
STATE_IDLE	[2:0]	3'd0

Table 635. alexforencich_verilog-ethernet/rtl/axis_baser_tx_64.v — 포트 (19개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_axis_tdata	input	wire [DATA_WIDTH-1:0]
s_axis_tkeep	input	wire [KEEP_WIDTH-1:0]
s_axis_tvalid	input	wire
s_axis_tready	output	wire
s_axis_tlast	input	wire
s_axis_tuser	input	wire [USER_WIDTH-1:0]
encoded_tx_data	output	wire [DATA_WIDTH-1:0]
encoded_tx_hdr	output	wire [HDR_WIDTH-1:0]
ptp_ts	input	wire [PTP_TS_WIDTH-1:0]
m_axis_ptp_ts	output	wire [PTP_TS_WIDTH-1:0]

포트	방향	형/폭
m_axis_ptp_ts_tag	output	wire [PTP_TAG_WIDTH-1:0]
m_axis_ptp_ts_valid	output	wire
cfg_ifg	input	wire [7:0]
cfg_tx_enable	input	wire
start_packet	output	wire [1:0]
error_underflow	output	wire
k	input	[7:0]

API — axis_eth_fcs.v

Table 636. alexforencich_verilog-ethernet/rtl/axis_eth_fcs.v — module / interface / package (1개)

이름	종류
axis_eth_fcs	module

Table 637. alexforencich_verilog-ethernet/rtl/axis_eth_fcs.v — parameter (3개)

파라미터	형/폭	기본값
DATA_WIDTH		8
KEEP_ENABLE		(DATA_WIDTH>8)
KEEP_WIDTH		(DATA_WIDTH/8)

Table 638. alexforencich_verilog-ethernet/rtl/axis_eth_fcs.v — 포트 (10개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_axis_tdata	input	wire [DATA_WIDTH-1:0]
s_axis_tkeep	input	wire [KEEP_WIDTH-1:0]
s_axis_tvalid	input	wire
s_axis_tready	output	wire
s_axis_tlast	input	wire
s_axis_tuser	input	wire
output_fcs	output	wire [31:0]
output_fcs_valid	output	wire

API — axis_eth_fcs_check.v

Table 639. alexforencich_verilog-ethernet/rtl/axis_eth_fcs_check.v — module / interface / package (1개)

이름	종류
axis_eth_fcs_check	module

Table 640. alexforencich_verilog-ethernet/rtl/axis_eth_fcs_check.v — localparam (1개)

상수	형/폭	값
STATE_IDLE	[1:0]	2'd0

Table 641. alexforencich_verilog-ethernet/rtl/axis_eth_fcs_check.v — 포트 (14개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_axis_tdata	input	wire [7:0]
s_axis_tvalid	input	wire
s_axis_tready	output	wire
s_axis_tlast	input	wire
s_axis_tuser	input	wire
m_axis_tdata	output	wire [7:0]
m_axis_tvalid	output	wire
m_axis_tready	input	wire
m_axis_tlast	output	wire
m_axis_tuser	output	wire
busy	output	wire
error_bad_fcs	output	wire

API — axis_eth_fcs_check_64.v**Table 642. alexforencich_verilog-ethernet/rtl/axis_eth_fcs_check_64.v — module / interface / package (1개)**

이름	종류
axis_eth_fcs_check_64	module

Table 643. alexforencich_verilog-ethernet/rtl/axis_eth_fcs_check_64.v — localparam (1개)

상수	형/폭	값
STATE_IDLE	[1:0]	2'd0

Table 644. alexforencich_verilog-ethernet/rtl/axis_eth_fcs_check_64.v — 포트 (16개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_axis_tdata	input	wire [63:0]
s_axis_tkeep	input	wire [7:0]
s_axis_tvalid	input	wire

포트	방향	형/폭
s_axis_tready	output	wire
s_axis_tlast	input	wire
s_axis_tuser	input	wire
m_axis_tdata	output	wire [63:0]
m_axis_tkeep	output	wire [7:0]
m_axis_tvalid	output	wire
m_axis_tready	input	wire
m_axis_tlast	output	wire
m_axis_tuser	output	wire
busy	output	wire
error_bad_fcs	output	wire

API — axis_eth_fcs_insert.v

Table 645. alexforencich_verilog-ethernet/rtl/axis_eth_fcs_insert.v — module / interface / package (1개)

이름	종류
axis_eth_fcs_insert	module

Table 646. alexforencich_verilog-ethernet/rtl/axis_eth_fcs_insert.v — parameter (2개)

파라미터	형/폭	기본값
ENABLE_PADDING		0
MIN_FRAME_LENGTH		64

Table 647. alexforencich_verilog-ethernet/rtl/axis_eth_fcs_insert.v — localparam (1개)

상수	형/폭	값
STATE_IDLE	[1:0]	2'd0

Table 648. alexforencich_verilog-ethernet/rtl/axis_eth_fcs_insert.v — 포트 (13개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_axis_tdata	input	wire [7:0]
s_axis_tvalid	input	wire
s_axis_tready	output	wire
s_axis_tlast	input	wire
s_axis_tuser	input	wire
m_axis_tdata	output	wire [7:0]
m_axis_tvalid	output	wire

포트	방향	형/폭
m_axis_tready	input	wire
m_axis_tlast	output	wire
m_axis_tuser	output	wire
busy	output	wire

API — axis_eth_fcs_insert_64.v

Table 649. alexforencich_verilog-ethernet/rtl/axis_eth_fcs_insert_64.v — module / interface / package (1개)

이름	종류
axis_eth_fcs_insert_64	module

Table 650. alexforencich_verilog-ethernet/rtl/axis_eth_fcs_insert_64.v — parameter (2개)

파라미터	형/폭	기본값
ENABLE_PADDING		0
MIN_FRAME_LENGTH		64

Table 651. alexforencich_verilog-ethernet/rtl/axis_eth_fcs_insert_64.v — localparam (1개)

상수	형/폭	값
STATE_IDLE	[1:0]	2'd0

Table 652. alexforencich_verilog-ethernet/rtl/axis_eth_fcs_insert_64.v — 포트 (17개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_axis_tdata	input	wire [63:0]
s_axis_tkeep	input	wire [7:0]
s_axis_tvalid	input	wire
s_axis_tready	output	wire
s_axis_tlast	input	wire
s_axis_tuser	input	wire
m_axis_tdata	output	wire [63:0]
m_axis_tkeep	output	wire [7:0]
m_axis_tvalid	output	wire
m_axis_tready	input	wire
m_axis_tlast	output	wire
m_axis_tuser	output	wire
busy	output	wire

포트	방향	형/폭
k	input	[7:0]
k	input	[3:0]

API — axis_gmii_rx.v

Table 653. alexforencich_verilog-ethernet/rtl/axis_gmii_rx.v — module / interface / package (1개)

이름	종류
axis_gmii_rx	module

Table 654. alexforencich_verilog-ethernet/rtl/axis_gmii_rx.v — parameter (4개)

파라미터	형/폭	기본값
DATA_WIDTH		8
PTP_TS_ENABLE		0
PTP_TS_WIDTH		96
USER_WIDTH		(PTP_TS_ENABLE ? PTP_TS_WIDTH : 0) + 1

Table 655. alexforencich_verilog-ethernet/rtl/axis_gmii_rx.v — localparam (2개)

상수	형/폭	값
ETH_PRE	[7:0]	8'h55
STATE_IDLE	[2:0]	3'd0

Table 656. alexforencich_verilog-ethernet/rtl/axis_gmii_rx.v — 포트 (16개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
gmii_rxd	input	wire [DATA_WIDTH-1:0]
gmii_rx_dv	input	wire
gmii_rx_er	input	wire
m_axis_tdata	output	wire [DATA_WIDTH-1:0]
m_axis_tvalid	output	wire
m_axis_tlast	output	wire
m_axis_tuser	output	wire [USER_WIDTH-1:0]
ptp_ts	input	wire [PTP_TS_WIDTH-1:0]
clk_enable	input	wire
mii_select	input	wire
cfg_rx_enable	input	wire
start_packet	output	wire
error_bad_frame	output	wire

포트	방향	형/폭
error_bad_fcs	output	wire

1. alexforenrich_verilog-ethernet/rtl/arp.v (452 lines)

```

1  /*
2
3  Copyright (c) 2014-2020 Alex Forenrich
4
5  Permission is hereby granted, free of charge, to any person
obtaining a copy
6  of this software and associated documentation files (the
"Software"), to deal
7  in the Software without restriction, including without limita-
tion the rights
8  to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9  copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24 // Language: Verilog 2001
25
26 `resetall
27 `timescale 1ns / 1ps
28 `default_nettype none
29
30 /*
31 * ARP block for IPv4, ethernet frame interface
32 */
33 module arp #
34 (
35     // Width of AXI stream interfaces in bits
36     parameter DATA_WIDTH = 8,
37     // Propagate tkeep signal
38     // If disabled, tkeep assumed to be 1'b1
39     parameter KEEP_ENABLE = (DATA_WIDTH>8),
40     // tkeep signal width (words per cycle)
41     parameter KEEP_WIDTH = (DATA_WIDTH/8),
42     // Log2 of ARP cache size
43     parameter CACHE_ADDR_WIDTH = 9,
44     // ARP request retry count
45     parameter REQUEST_RETRY_COUNT = 4,
46     // ARP request retry interval (in cycles)
47     parameter REQUEST_RETRY_INTERVAL = 125000000*2,
48     // ARP request timeout (in cycles)
49     parameter REQUEST_TIMEOUT = 125000000*30
50 )
51 (
52     input wire clk,
53     input wire rst,
54
55     /*
56     * Ethernet frame input
57     */
58     input wire s_eth_hdr_valid,
59     output wire s_eth_hdr_ready,
60     input wire [47:0] s_eth_dest_mac,
61     input wire [47:0] s_eth_src_mac,
62     input wire [15:0] s_eth_type,
63     input wire [DATA_WIDTH-1:0] s_eth_payload_axis_tdata,
64     input wire [KEEP_WIDTH-1:0] s_eth_payload_axis_tkeep,
65     input wire s_eth_payload_axis_tvalid,
66     output wire s_eth_payload_axis_tready,
67     input wire s_eth_payload_axis_tlast,
68     input wire s_eth_payload_axis_tuser,
69
70     /*
71     * Ethernet frame output
72     */
73     output wire m_eth_hdr_valid,
74     input wire m_eth_hdr_ready,
75     output wire [47:0] m_eth_dest_mac,
76     output wire [47:0] m_eth_src_mac,

```

```

78     output wire [15:0] m_eth_type,
79     output wire [DATA_WIDTH-1:0] m_eth_payload_axis_tdata,
80     output wire [KEEP_WIDTH-1:0] m_eth_payload_axis_tkeep,
81     output wire m_eth_payload_axis_tvalid,
82     input wire m_eth_payload_axis_tready,
83     output wire m_eth_payload_axis_tlast,
84     output wire m_eth_payload_axis_tuser,
85
86     /*
87     * ARP requests
88     */
89     input wire arp_request_valid,
90     output wire arp_request_ready,
91     input wire [31:0] arp_request_ip,
92     output wire arp_response_valid,
93     input wire arp_response_ready,
94     output wire arp_response_error,
95     output wire [47:0] arp_response_mac,
96
97     /*
98     * Configuration
99     */
100    input wire [47:0] local_mac,
101    input wire [31:0] local_ip,
102    input wire [31:0] gateway_ip,
103    input wire [31:0] subnet_mask,
104    input wire clear_cache
105 );
106
107 localparam [15:0]
108     ARP_OPER_ARP_REQUEST = 16'h0001,
109     ARP_OPER_ARP_REPLY = 16'h0002,
110     ARP_OPER_INARP_REQUEST = 16'h0008,
111     ARP_OPER_INARP_REPLY = 16'h0009;
112
113 wire incoming_frame_valid;
114 reg incoming_frame_ready;
115 wire [47:0] incoming_eth_dest_mac;
116 wire [47:0] incoming_eth_src_mac;
117 wire [15:0] incoming_eth_type;
118 wire [15:0] incoming_arp_htype;
119 wire [15:0] incoming_arp_ptype;
120 wire [7:0] incoming_arp_hlen;
121 wire [7:0] incoming_arp_plen;
122 wire [15:0] incoming_arp_oper;
123 wire [47:0] incoming_arp_sha;
124 wire [31:0] incoming_arp_spa;
125 wire [47:0] incoming_arp_tha;
126 wire [31:0] incoming_arp_tpa;
127
128 /*
129 * ARP frame processing
130 */
131 arp_eth_rx #(
132     .DATA_WIDTH(DATA_WIDTH),
133     .KEEP_ENABLE(KEEP_ENABLE),
134     .KEEP_WIDTH(KEEP_WIDTH)
135 )
136 arp_eth_rx_inst (
137     .clk(clk),
138     .rst(rst),
139     // Ethernet frame input
140     .s_eth_hdr_valid(s_eth_hdr_valid),
141     .s_eth_hdr_ready(s_eth_hdr_ready),
142     .s_eth_dest_mac(s_eth_dest_mac),
143     .s_eth_src_mac(s_eth_src_mac),
144     .s_eth_type(s_eth_type),
145     .s_eth_payload_axis_tdata(s_eth_payload_axis_tdata),
146     .s_eth_payload_axis_tkeep(s_eth_payload_axis_tkeep),
147     .s_eth_payload_axis_tvalid(s_eth_payload_axis_tvalid),
148     .s_eth_payload_axis_tready(s_eth_payload_axis_tready),
149     .s_eth_payload_axis_tlast(s_eth_payload_axis_tlast),
150     .s_eth_payload_axis_tuser(s_eth_payload_axis_tuser),
151     // ARP frame output
152     .m_frame_valid(incoming_frame_valid),
153     .m_frame_ready(incoming_frame_ready),
154     .m_eth_dest_mac(incoming_eth_dest_mac),
155     .m_eth_src_mac(incoming_eth_src_mac),
156     .m_eth_type(incoming_eth_type),
157     .m_arp_htype(incoming_arp_htype),
158     .m_arp_ptype(incoming_arp_ptype),
159     .m_arp_hlen(incoming_arp_hlen),
160     .m_arp_plen(incoming_arp_plen),
161     .m_arp_oper(incoming_arp_oper),
162     .m_arp_sha(incoming_arp_sha),
163     .m_arp_spa(incoming_arp_spa),
164     .m_arp_tha(incoming_arp_tha),
165     .m_arp_tpa(incoming_arp_tpa),
166     // Status signals
167     .busy(),
168     .error_header_early_termination(),
169     .error_invalid_header()
170 );
171

```

```

172 reg outgoing_frame_valid_reg = 1'b0, outgoing_frame_valid_next;
173 wire outgoing_frame_ready;
174 reg [47:0] outgoing_eth_dest_mac_reg = 48'd0, outgoing_eth_dest_mac_next;
175 reg [15:0] outgoing_arp_oper_reg = 16'd0, outgoing_arp_oper_next;
176 reg [47:0] outgoing_arp_tha_reg = 48'd0, outgoing_arp_tha_next;
177 reg [31:0] outgoing_arp_tpa_reg = 32'd0, outgoing_arp_tpa_next;
178
179 arp_eth_tx #(
180     .DATA_WIDTH(DATA_WIDTH),
181     .KEEP_ENABLE(KEEP_ENABLE),
182     .KEEP_WIDTH(KEEP_WIDTH)
183 )
184 arp_eth_tx_inst (
185     .clk(clk),
186     .rst(rst),
187     // ARP frame input
188     .s_frame_valid(outgoing_frame_valid_reg),
189     .s_frame_ready(outgoing_frame_ready),
190     .s_eth_dest_mac(outgoing_eth_dest_mac_reg),
191     .s_eth_src_mac(local_mac),
192     .s_eth_type(16'h0806),
193     .s_arp_htype(16'h0001),
194     .s_arp_ptype(16'h0800),
195     .s_arp_oper(outgoing_arp_oper_reg),
196     .s_arp_sha(local_mac),
197     .s_arp_spa(local_ip),
198     .s_arp_tha(outgoing_arp_tha_reg),
199     .s_arp_tpa(outgoing_arp_tpa_reg),
200     // Ethernet frame output
201     .m_eth_hdr_valid(m_eth_hdr_valid),
202     .m_eth_hdr_ready(m_eth_hdr_ready),
203     .m_eth_dest_mac(m_eth_dest_mac),
204     .m_eth_src_mac(m_eth_src_mac),
205     .m_eth_type(m_eth_type),
206     .m_eth_payload_axis_tdata(m_eth_payload_axis_tdata),
207     .m_eth_payload_axis_tkeep(m_eth_payload_axis_tkeep),
208     .m_eth_payload_axis_tvalid(m_eth_payload_axis_tvalid),
209     .m_eth_payload_axis_tready(m_eth_payload_axis_tready),
210     .m_eth_payload_axis_tlast(m_eth_payload_axis_tlast),
211     .m_eth_payload_axis_tuser(m_eth_payload_axis_tuser),
212     // Status signals
213     .busy()
214 );
215
216 reg cache_query_request_valid_reg = 1'b0,
cache_query_request_valid_next;
217 reg [31:0] cache_query_request_ip_reg = 32'd0,
cache_query_request_ip_next;
218 wire cache_query_response_valid;
219 wire cache_query_response_error;
220 wire [47:0] cache_query_response_mac;
221
222 reg cache_write_request_valid_reg = 1'b0,
cache_write_request_valid_next;
223 reg [31:0] cache_write_request_ip_reg = 32'd0,
cache_write_request_ip_next;
224 reg [47:0] cache_write_request_mac_reg = 48'd0,
cache_write_request_mac_next;
225
226 /*
227  * ARP cache
228  */
229 arp_cache #(
230     .CACHE_ADDR_WIDTH(CACHE_ADDR_WIDTH)
231 )
232 arp_cache_inst (
233     .clk(clk),
234     .rst(rst),
235     // Query cache
236     .query_request_valid(cache_query_request_valid_reg),
237     .query_request_ready(),
238     .query_request_ip(cache_query_request_ip_reg),
239     .query_response_valid(cache_query_response_valid),
240     .query_response_ready(1'b1),
241     .query_response_error(cache_query_response_error),
242     .query_response_mac(cache_query_response_mac),
243     // Write cache
244     .write_request_valid(cache_write_request_valid_reg),
245     .write_request_ready(),
246     .write_request_ip(cache_write_request_ip_reg),
247     .write_request_mac(cache_write_request_mac_reg),
248     // Configuration
249     .clear_cache(clear_cache)
250 );
251
252 reg arp_request_operation_reg = 1'b0, arp_request_operation_next;
253
254 reg arp_request_ready_reg = 1'b0, arp_request_ready_next;
255 reg [31:0] arp_request_ip_reg = 32'd0, arp_request_ip_next;
256
257 reg arp_response_valid_reg = 1'b0, arp_response_valid_next;
258 reg arp_response_error_reg = 1'b0, arp_response_error_next;
259 reg [47:0] arp_response_mac_reg = 48'd0, arp_response_mac_next;
260

```

```

261 reg [5:0] arp_request_retry_cnt_reg = 6'd0, arp_request_retry_cnt_next;
262 reg [35:0] arp_request_timer_reg = 36'd0, arp_request_timer_next;
263
264 assign arp_request_ready = arp_request_ready_reg;
265
266 assign arp_response_valid = arp_response_valid_reg;
267 assign arp_response_error = arp_response_error_reg;
268 assign arp_response_mac = arp_response_mac_reg;
269
270 always @* begin
271     incoming_frame_ready = 1'b0;
272
273     outgoing_frame_valid_next = outgoing_frame_valid_reg && !
outgoing_frame_ready;
274     outgoing_eth_dest_mac_next = outgoing_eth_dest_mac_reg;
275     outgoing_arp_oper_next = outgoing_arp_oper_reg;
276     outgoing_arp_tha_next = outgoing_arp_tha_reg;
277     outgoing_arp_tpa_next = outgoing_arp_tpa_reg;
278
279     cache_query_request_valid_next = 1'b0;
280     cache_query_request_ip_next = cache_query_request_ip_reg;
281
282     cache_write_request_valid_next = 1'b0;
283     cache_write_request_mac_next =
cache_write_request_mac_reg;
284     cache_write_request_ip_next = cache_write_request_ip_reg;
285
286     arp_request_ready_next = 1'b0;
287     arp_request_ip_next = arp_request_ip_reg;
288     arp_request_operation_next = arp_request_operation_reg;
289     arp_request_retry_cnt_next = arp_request_retry_cnt_reg;
290     arp_request_timer_next = arp_request_timer_reg;
291     arp_response_valid_next = arp_response_valid_reg && !
arp_response_ready;
292     arp_response_error_next = 1'b0;
293     arp_response_mac_next = 48'd0;
294
295     // manage incoming frames
296     incoming_frame_ready = outgoing_frame_ready;
297     if (incoming_frame_valid && incoming_frame_ready) begin
298         if (incoming_eth_type == 16'h0806 && incoming_arp_htype == 16'h0001 && incoming_arp_ptype == 16'h0800) begin
299             // store sender addresses in cache
300             cache_write_request_valid_next = 1'b1;
301             cache_write_request_ip_next = incoming_arp_spa;
302             cache_write_request_mac_next = incoming_arp_sha;
303             if (incoming_arp_oper == ARP_OPER_ARP_REQUEST) begin
304                 // ARP request
305                 if (incoming_arp_tpa == local_ip) begin
306                     // send reply frame to valid incoming request
307                     outgoing_frame_valid_next = 1'b1;
308                     outgoing_eth_dest_mac_next = incoming_eth_src_mac;
309                     outgoing_arp_oper_next = ARP_OP-
ER_ARP_REPLY;
310                     outgoing_arp_tha_next = incoming_arp_sha;
311                     outgoing_arp_tpa_next = incoming_arp_spa;
312                     end
313                     end else if (incoming_arp_oper == ARP_OP-
ER_INARP_REQUEST) begin
314                         // INARP request
315                         if (incoming_arp_tha == local_mac) begin
316                             // send reply frame to valid incoming request
317                             outgoing_frame_valid_next = 1'b1;
318                             outgoing_eth_dest_mac_next = incoming_eth_src_mac;
319                             outgoing_arp_oper_next = ARP_OP-
ER_INARP_REPLY;
320                             outgoing_arp_tha_next = incoming_arp_sha;
321                             outgoing_arp_tpa_next = incoming_arp_spa;
322                             end
323                             end
324                             end
325                             end
326                             end
327                             // manage ARP lookup requests
328                             if (arp_request_operation_reg) begin
329                                 arp_request_ready_next = 1'b0;
330                                 cache_query_request_valid_next = 1'b1;
331                                 arp_request_timer_next = arp_request_timer_reg - 1;
332                                 // if we got a response, it will go in the cache, so
when the query succeeds, we're done
333                                 if (cache_query_response_valid && !
cache_query_response_error) begin
334                                     arp_request_operation_next = 1'b0;
335                                     cache_query_request_valid_next = 1'b0;
336                                     arp_response_valid_next = 1'b1;
337                                     arp_response_error_next = 1'b0;
338                                     arp_response_mac_next = cache_query_response_mac;
339                                 end
340                                 // timer timeout
341                                 if (arp_request_timer_reg == 0) begin
342                                     if (arp_request_retry_cnt_reg > 0) begin
343                                         // have more retries
344                                         // send ARP request frame
345                                         outgoing_frame_valid_next = 1'b1;

```

```

346     outgoing_eth_dest_mac_next = 48'hfffffffffff;
347     outgoing_arp_oper_next = ARP_OPER_ARP_REQUEST;
348     outgoing_arp_tha_next = 48'h000000000000;
349     outgoing_arp_tpa_next = arp_request_ip_reg;
350     arp_request_retry_cnt_next = arp-re-
quest_retry_cnt_reg - 1;
351     if (arp_request_retry_cnt_reg > 1) begin
352         arp_request_timer_next = RE-
QUEST_RETRY_INTERVAL;
353     end else begin
354         arp_request_timer_next = REQUEST_TIMEOUT;
355     end
356 end else begin
357     // out of retries
358     arp_request_operation_next = 1'b0;
359     arp_response_valid_next = 1'b1;
360     arp_response_error_next = 1'b1;
361     cache_query_request_valid_next = 1'b0;
362 end
363 end
364 end else begin
365     arp_request_ready_next = !arp_response_valid_next;
366     if (cache_query_request_valid_reg) begin
367         cache_query_request_valid_next = 1'b1;
368         if (cache_query_response_valid) begin
369             if (cache_query_response_error) begin
370                 arp_request_operation_next = 1'b1;
371                 // send ARP request frame
372                 outgoing_frame_valid_next = 1'b1;
373                 outgoing_eth_dest_mac_next =
48'hfffffffffff;
374                 outgoing_arp_oper_next = ARP_OP-
ER_ARP_REQUEST;
375                 outgoing_arp_tha_next = 48'h000000000000;
376                 outgoing_arp_tpa_next = arp-re-
quest_ip_reg;
377                 arp_request_retry_cnt_next = RE-
QUEST_RETRY_COUNT-1;
378                 arp_request_timer_next = RE-
QUEST_RETRY_INTERVAL;
379             end else begin
380                 cache_query_request_valid_next = 1'b0;
381                 arp_response_valid_next = 1'b1;
382                 arp_response_error_next = 1'b0;
383                 arp_response_mac_next =
cache_query_response_mac;
384             end
385         end
386         end else if (arp_request_valid && arp_request_ready)
begin
387             if (arp_request_ip == 32'hffffffff) begin
388                 // broadcast address; use broadcast MAC ad-
dress
389                 arp_response_valid_next = 1'b1;
390                 arp_response_error_next = 1'b0;
391                 arp_response_mac_next = 48'hfffffffffff;
392             end else if (((arp_request_ip ^ gateway_ip) & sub-
net_mask) == 0) begin
393                 // within subnet
394                 // (no bits differ between request IP and
gateway IP where subnet mask is set)
395                 if (~(arp_request_ip | subnet_mask) == 0) be-
gin
396                     // broadcast address; use broadcast MAC
address
397                     // (all bits in request IP set where sub-
net mask is clear)
398                     arp_response_valid_next = 1'b1;
399                     arp_response_error_next = 1'b0;
400                     arp_response_mac_next = 48'hfffffffffff;
401                 end else begin
402                     // unicast address; look up IP directly
403                     cache_query_request_valid_next = 1'b1;
404                     cache_query_request_ip_next = arp-re-
quest_ip;
405                     arp_request_ip_next = arp_request_ip;
406                 end
407             end else begin
408                 // outside of subnet, so look up gateway ad-
dress
409                 cache_query_request_valid_next = 1'b1;
410                 cache_query_request_ip_next = gateway_ip;
411                 arp_request_ip_next = gateway_ip;
412             end
413         end
414     end
415 end
416
417 always @(posedge clk) begin
418     if (rst) begin
419         outgoing_frame_valid_reg <= 1'b0;
420         cache_query_request_valid_reg <= 1'b0;
421         cache_write_request_valid_reg <= 1'b0;
422         arp_request_ready_reg <= 1'b0;
423         arp_request_operation_reg <= 1'b0;
424         arp_request_retry_cnt_reg <= 6'd0;
425         arp_request_timer_reg <= 36'd0;
426         arp_response_valid_reg <= 1'b0;
427     end else begin
428         outgoing_frame_valid_reg <= outgoing_frame_valid_next;
429         cache_query_request_valid_reg <=

```

```

cache_query_request_valid_next;
430         cache_write_request_valid_reg <=
cache_write_request_valid_next;
431         arp_request_ready_reg <= arp_request_ready_next;
432         arp_request_operation_reg <= arp-re-
quest_operation_next;
433         arp_request_retry_cnt_reg <= arp-re-
quest_retry_cnt_next;
434         arp_request_timer_reg <= arp_request_timer_next;
435         arp_response_valid_reg <= arp_response_valid_next;
436     end
437
438     cache_query_request_ip_reg <= cache_query_request_ip_next;
439     outgoing_eth_dest_mac_reg <= outgoing_eth_dest_mac_next;
440     outgoing_arp_oper_reg <= outgoing_arp_oper_next;
441     outgoing_arp_tha_reg <= outgoing_arp_tha_next;
442     outgoing_arp_tpa_reg <= outgoing_arp_tpa_next;
443     cache_write_request_mac_reg <=
cache_write_request_mac_next;
444     cache_write_request_ip_reg <= cache_write_request_ip_next;
445     arp_request_ip_reg <= arp_request_ip_next;
446     arp_response_error_reg <= arp_response_error_next;
447     arp_response_mac_reg <= arp_response_mac_next;
448 end
449
450 endmodule
451
452 `resetall

```

2. alexforenrich_verilog-ethernet/rtl/arp_cache.v (247 lines)

```

1 /*
2
3 Copyright (c) 2014-2018 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * ARP cache
33 */
34 module arp_cache #(
35     parameter CACHE_ADDR_WIDTH = 9
36 )
37 (
38     input wire clk,
39     input wire rst,
40
41     /*
42     * Cache query
43     */
44     input wire query_request_valid,
45     output wire query_request_ready,
46     input wire [31:0] query_request_ip,
47
48     output wire query_response_valid,
49     input wire query_response_ready,
50     output wire query_response_error,
51     output wire [47:0] query_response_mac,
52
53     /*
54     * Cache write
55     */
56     input wire write_request_valid,

```



```

57  output wire      write_request_ready,
58  input  wire [31:0] write_request_ip,
59  input  wire [47:0] write_request_mac,
60
61  /*
62   * Configuration
63   */
64  input wire      clear_cache
65  );
66
67  reg mem_write = 0;
68  reg store_query = 0;
69  reg store_write = 0;
70
71  reg query_ip_valid_reg = 0, query_ip_valid_next;
72  reg [31:0] query_ip_reg = 0;
73  reg write_ip_valid_reg = 0, write_ip_valid_next;
74  reg [31:0] write_ip_reg = 0;
75  reg [47:0] write_mac_reg = 0;
76  reg clear_cache_reg = 0, clear_cache_next;
77
78  reg [CACHE_ADDR_WIDTH-1:0] wr_ptr_reg =
{CACHE_ADDR_WIDTH{1'b0}}, wr_ptr_next;
79  reg [CACHE_ADDR_WIDTH-1:0] rd_ptr_reg =
{CACHE_ADDR_WIDTH{1'b0}}, rd_ptr_next;
80
81  reg valid_mem[(2**CACHE_ADDR_WIDTH)-1:0];
82  reg [31:0] ip_addr_mem[(2**CACHE_ADDR_WIDTH)-1:0];
83  reg [47:0] mac_addr_mem[(2**CACHE_ADDR_WIDTH)-1:0];
84
85  reg query_request_ready_reg = 0, query_request_ready_next;
86
87  reg query_response_valid_reg = 0, query_response_valid_next;
88  reg query_response_error_reg = 0, query_response_error_next;
89  reg [47:0] query_response_mac_reg = 0;
90
91  reg write_request_ready_reg = 0, write_request_ready_next;
92
93  wire [31:0] query_request_hash;
94  wire [31:0] write_request_hash;
95
96  assign query_request_ready = query_request_ready_reg;
97
98  assign query_response_valid = query_response_valid_reg;
99  assign query_response_error = query_response_error_reg;
100 assign query_response_mac = query_response_mac_reg;
101
102 assign write_request_ready = write_request_ready_reg;
103
104 lfsr #(
105     .LFSR_WIDTH(32),
106     .LFSR_POLY(32'h4c11db7),
107     .LFSR_CONFIG("GALOIS"),
108     .LFSR_FEED_FORWARD(0),
109     .REVERSE(1),
110     .DATA_WIDTH(32),
111     .STYLE("AUTO")
112 )
113 rd_hash (
114     .data_in(query_request_ip),
115     .state_in(32'hffffffff),
116     .data_out(),
117     .state_out(query_request_hash)
118 );
119
120 lfsr #(
121     .LFSR_WIDTH(32),
122     .LFSR_POLY(32'h4c11db7),
123     .LFSR_CONFIG("GALOIS"),
124     .LFSR_FEED_FORWARD(0),
125     .REVERSE(1),
126     .DATA_WIDTH(32),
127     .STYLE("AUTO")
128 )
129 wr_hash (
130     .data_in(write_request_ip),
131     .state_in(32'hffffffff),
132     .data_out(),
133     .state_out(write_request_hash)
134 );
135
136 integer i;
137
138 initial begin
139     for (i = 0; i < 2**CACHE_ADDR_WIDTH; i = i + 1) begin
140         valid_mem[i] = 1'b0;
141         ip_addr_mem[i] = 32'd0;
142         mac_addr_mem[i] = 48'd0;
143     end
144 end
145
146 always @* begin
147     mem_write = 1'b0;
148     store_query = 1'b0;
149     store_write = 1'b0;
150
151     wr_ptr_next = wr_ptr_reg;
152     rd_ptr_next = rd_ptr_reg;
153
154     clear_cache_next = clear_cache_reg | clear_cache;
155
156     query_ip_valid_next = query_ip_valid_reg;
157
158     query_request_ready_next = (~query_ip_valid_reg ||
~query_request_valid || query_response_ready) && !clear_cache_next;
159
160     query_response_valid_next = query_response_valid_reg &
~query_response_ready;
161     query_response_error_next = query_response_error_reg;
162
163     if (query_ip_valid_reg && (~query_request_valid ||
query_response_ready)) begin
164         query_response_valid_next = 1;
165         query_ip_valid_next = 0;
166         if (valid_mem[rd_ptr_reg] && ip_addr_mem[rd_ptr_reg]
== query_ip_reg) begin
167             query_response_error_next = 0;
168         end else begin
169             query_response_error_next = 1;
170         end
171     end
172
173     if (query_request_valid && query_request_ready &&
(~query_ip_valid_reg || ~query_request_valid ||
query_response_ready)) begin
174         store_query = 1;
175         query_ip_valid_next = 1;
176         rd_ptr_next =
query_request_hash[CACHE_ADDR_WIDTH-1:0];
177     end
178
179     write_ip_valid_next = write_ip_valid_reg;
180
181     write_request_ready_next = !clear_cache_next;
182
183     if (write_ip_valid_reg) begin
184         write_ip_valid_next = 0;
185         mem_write = 1;
186     end
187
188     if (write_request_valid && write_request_ready) begin
189         store_write = 1;
190         write_ip_valid_next = 1;
191         wr_ptr_next =
write_request_hash[CACHE_ADDR_WIDTH-1:0];
192     end
193
194     if (clear_cache) begin
195         clear_cache_next = 1'b1;
196         wr_ptr_next = 0;
197     end else if (clear_cache_reg) begin
198         wr_ptr_next = wr_ptr_reg + 1;
199         clear_cache_next = wr_ptr_next != 0;
200         mem_write = 1;
201     end
202 end
203
204 always @(posedge clk) begin
205     if (rst) begin
206         query_ip_valid_reg <= 1'b0;
207         query_request_ready_reg <= 1'b0;
208         query_response_valid_reg <= 1'b0;
209         write_ip_valid_reg <= 1'b0;
210         write_request_ready_reg <= 1'b0;
211         clear_cache_reg <= 1'b1;
212         wr_ptr_reg <= 0;
213     end else begin
214         query_ip_valid_reg <= query_ip_valid_next;
215         query_request_ready_reg <= query_request_ready_next;
216         query_response_valid_reg <= query_response_valid_next;
217         write_ip_valid_reg <= write_ip_valid_next;
218         write_request_ready_reg <= write_request_ready_next;
219         clear_cache_reg <= clear_cache_next;
220         wr_ptr_reg <= wr_ptr_next;
221     end
222
223     query_response_error_reg <= query_response_error_next;
224
225     if (store_query) begin
226         query_ip_reg <= query_request_ip;
227     end
228
229     if (store_write) begin
230         write_ip_reg <= write_request_ip;
231         write_mac_reg <= write_request_mac;
232     end
233
234     rd_ptr_reg <= rd_ptr_next;
235
236     query_response_mac_reg <= mac_addr_mem[rd_ptr_reg];
237
238     if (mem_write) begin
239         valid_mem[wr_ptr_reg] <= !clear_cache_reg;
240         ip_addr_mem[wr_ptr_reg] <= write_ip_reg;
241         mac_addr_mem[wr_ptr_reg] <= write_mac_reg;
242     end
243 end
244
245 endmodule
246
247 `resetall

```


3. alexforenrich_verilog-ethernet/rtl/arp_eth_rx.v (331 lines)

```

1 /*
2
3 Copyright (c) 2014-2020 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * ARP ethernet frame receiver (Ethernet frame in, ARP frame
out)
33 */
34 module arp_eth_rx #
35 (
36     // Width of AXI stream interfaces in bits
37     parameter DATA_WIDTH = 8,
38     // Propagate tkeep signal
39     // If disabled, tkeep assumed to be 1'b1
40     parameter KEEP_ENABLE = (DATA_WIDTH>8),
41     // tkeep signal width (words per cycle)
42     parameter KEEP_WIDTH = (DATA_WIDTH/8)
43 )
44 (
45     input wire          clk,
46     input wire          rst,
47
48     /*
49      * Ethernet frame input
50      */
51     input wire          s_eth_hdr_valid,
52     output wire         s_eth_hdr_ready,
53     input wire [47:0]   s_eth_dest_mac,
54     input wire [47:0]   s_eth_src_mac,
55     input wire [15:0]   s_eth_type,
56     input wire [DATA_WIDTH-1:0] s_eth_payload_axis_tdata,
57     input wire [KEEP_WIDTH-1:0] s_eth_payload_axis_tkeep,
58     input wire          s_eth_payload_axis_tvalid,
59     output wire         s_eth_payload_axis_tready,
60     input wire          s_eth_payload_axis_tlast,
61     input wire          s_eth_payload_axis_tuser,
62
63     /*
64      * ARP frame output
65      */
66     output wire         m_frame_valid,
67     input wire          m_frame_ready,
68     output wire [47:0]  m_eth_dest_mac,
69     output wire [47:0]  m_eth_src_mac,
70     output wire [15:0]  m_eth_type,
71     output wire [15:0]  m_arp_htype,
72     output wire [15:0]  m_arp_ptype,
73     output wire [7:0]   m_arp_hlen,
74     output wire [7:0]   m_arp_plen,
75     output wire [15:0]  m_arp_oper,
76     output wire [47:0]  m_arp_sha,
77     output wire [31:0]  m_arp_spa,
78     output wire [47:0]  m_arp_tha,
79     output wire [31:0]  m_arp_tpa,
80
81     /*
82      * Status signals
83      */

```

```

84     output wire          busy,
85     output wire          error_header_early_termination,
86     output wire          error_invalid_header
87 );
88
89 parameter BYTE_LANES = KEEP_ENABLE ? KEEP_WIDTH : 1;
90
91 parameter HDR_SIZE = 28;
92
93 parameter CYCLE_COUNT = (HDR_SIZE+BYTE_LANES-1)/BYTE_LANES;
94
95 parameter PTR_WIDTH = $clog2(CYCLE_COUNT);
96
97 parameter OFFSET = HDR_SIZE % BYTE_LANES;
98
99 // bus width assertions
100 initial begin
101     if (BYTE_LANES * 8 != DATA_WIDTH) begin
102         $error("Error: AXI stream interface requires byte (8-
bit) granularity (instance %m)");
103         $finish;
104     end
105 end
106
107 /*
108 ARP Frame
109
110 Field                Length
111 Destination MAC address 6 octets
112 Source MAC address      6 octets
113 Ethertype (0x0806)      2 octets
114 HTYPE (1)               2 octets
115 PTYPE (0x0800)          2 octets
116 HLEN (6)                1 octets
117 PLEN (4)                1 octets
118 OPER                   2 octets
119 SHA Sender MAC          6 octets
120 SPA Sender IP           4 octets
121 THA Target MAC          6 octets
122 TPA Target IP           4 octets
123
124 This module receives an Ethernet frame with header fields in
parallel and
125 payload on an AXI stream interface, decodes the ARP packet
fields, and
126 produces the frame fields in parallel.
127
128 */
129
130 // datapath control signals
131 reg store_eth_hdr;
132
133 reg read_eth_header_reg = 1'b1, read_eth_header_next;
134 reg read_arp_header_reg = 1'b0, read_arp_header_next;
135 reg [PTR_WIDTH-1:0] ptr_reg = 0, ptr_next;
136
137 reg s_eth_hdr_ready_reg = 1'b0, s_eth_hdr_ready_next;
138 reg s_eth_payload_axis_tready_reg = 1'b0, s_eth_pay-
load_axis_tready_next;
139
140 reg m_frame_valid_reg = 1'b0, m_frame_valid_next;
141 reg [47:0] m_eth_dest_mac_reg = 48'd0;
142 reg [47:0] m_eth_src_mac_reg = 48'd0;
143 reg [15:0] m_eth_type_reg = 16'd0;
144 reg [15:0] m_arp_htype_reg = 16'd0, m_arp_htype_next;
145 reg [15:0] m_arp_ptype_reg = 16'd0, m_arp_ptype_next;
146 reg [7:0] m_arp_hlen_reg = 8'd0, m_arp_hlen_next;
147 reg [7:0] m_arp_plen_reg = 8'd0, m_arp_plen_next;
148 reg [15:0] m_arp_oper_reg = 16'd0, m_arp_oper_next;
149 reg [47:0] m_arp_sha_reg = 48'd0, m_arp_sha_next;
150 reg [31:0] m_arp_spa_reg = 32'd0, m_arp_spa_next;
151 reg [47:0] m_arp_tha_reg = 48'd0, m_arp_tha_next;
152 reg [31:0] m_arp_tpa_reg = 32'd0, m_arp_tpa_next;
153
154 reg busy_reg = 1'b0;
155 reg error_header_early_termination_reg = 1'b0, er-
ror_header_early_termination_next;
156 reg error_invalid_header_reg = 1'b0, er-
ror_invalid_header_next;
157
158 assign s_eth_hdr_ready = s_eth_hdr_ready_reg;
159 assign s_eth_payload_axis_tready = s_eth_pay-
load_axis_tready_reg;
160
161 assign m_frame_valid = m_frame_valid_reg;
162 assign m_eth_dest_mac = m_eth_dest_mac_reg;
163 assign m_eth_src_mac = m_eth_src_mac_reg;
164 assign m_eth_type = m_eth_type_reg;
165 assign m_arp_htype = m_arp_htype_reg;
166 assign m_arp_ptype = m_arp_ptype_reg;
167 assign m_arp_hlen = m_arp_hlen_reg;
168 assign m_arp_plen = m_arp_plen_reg;
169 assign m_arp_oper = m_arp_oper_reg;
170 assign m_arp_sha = m_arp_sha_reg;
171 assign m_arp_spa = m_arp_spa_reg;
172 assign m_arp_tha = m_arp_tha_reg;
173 assign m_arp_tpa = m_arp_tpa_reg;
174
175 assign busy = busy_reg;
176

```

```

177 assign error_header_early_termination = er-
ror_header_early_termination_reg;
178 assign error_invalid_header = error_invalid_header_reg;
179
180 always @* begin
181   read_eth_header_next = read_eth_header_reg;
182   read_arp_header_next = read_arp_header_reg;
183   ptr_next = ptr_reg;
184
185   s_eth_hdr_ready_next = 1'b0;
186   s_eth_payload_axis_tready_next = 1'b0;
187
188   store_eth_hdr = 1'b0;
189
190   m_frame_valid_next = m_frame_valid_reg && !m_frame_ready;
191
192   m_arp_htype_next = m_arp_htype_reg;
193   m_arp_ptype_next = m_arp_ptype_reg;
194   m_arp_hlen_next = m_arp_hlen_reg;
195   m_arp_plen_next = m_arp_plen_reg;
196   m_arp_oper_next = m_arp_oper_reg;
197   m_arp_sha_next = m_arp_sha_reg;
198   m_arp_spa_next = m_arp_spa_reg;
199   m_arp_tha_next = m_arp_tha_reg;
200   m_arp_tpa_next = m_arp_tpa_reg;
201
202   error_header_early_termination_next = 1'b0;
203   error_invalid_header_next = 1'b0;
204
205   if (s_eth_hdr_ready && s_eth_hdr_valid) begin
206     if (read_eth_header_reg) begin
207       store_eth_hdr = 1'b1;
208       ptr_next = 0;
209       read_eth_header_next = 1'b0;
210       read_arp_header_next = 1'b1;
211     end
212   end
213
214   if (s_eth_payload_axis_tready && s_eth_pay-
load_axis_tvalid) begin
215     if (read_arp_header_reg) begin
216       // word transfer in - store it
217       ptr_next = ptr_reg + 1;
218
219       `define _HEADER_FIELD(offset, field) \
220         if (ptr_reg == offset/BYTE_LANES && (!KEEP_EN-
ABLE || s_eth_payload_axis_tkeep[offset%BYTE_LANES])) begin \
221           field = s_eth_pay-
load_axis_tdata[(offset%BYTE_LANES)*8 +: 8]; \
222         end
223
224         `_HEADER_FIELD(0, m_arp_htype_next[1*8 +: 8])
225         `_HEADER_FIELD(1, m_arp_htype_next[0*8 +: 8])
226         `_HEADER_FIELD(2, m_arp_ptype_next[1*8 +: 8])
227         `_HEADER_FIELD(3, m_arp_ptype_next[0*8 +: 8])
228         `_HEADER_FIELD(4, m_arp_hlen_next[0*8 +: 8])
229         `_HEADER_FIELD(5, m_arp_plen_next[0*8 +: 8])
230         `_HEADER_FIELD(6, m_arp_oper_next[1*8 +: 8])
231         `_HEADER_FIELD(7, m_arp_oper_next[0*8 +: 8])
232         `_HEADER_FIELD(8, m_arp_sha_next[5*8 +: 8])
233         `_HEADER_FIELD(9, m_arp_sha_next[4*8 +: 8])
234         `_HEADER_FIELD(10, m_arp_sha_next[3*8 +: 8])
235         `_HEADER_FIELD(11, m_arp_sha_next[2*8 +: 8])
236         `_HEADER_FIELD(12, m_arp_sha_next[1*8 +: 8])
237         `_HEADER_FIELD(13, m_arp_sha_next[0*8 +: 8])
238         `_HEADER_FIELD(14, m_arp_spa_next[3*8 +: 8])
239         `_HEADER_FIELD(15, m_arp_spa_next[2*8 +: 8])
240         `_HEADER_FIELD(16, m_arp_spa_next[1*8 +: 8])
241         `_HEADER_FIELD(17, m_arp_spa_next[0*8 +: 8])
242         `_HEADER_FIELD(18, m_arp_tha_next[5*8 +: 8])
243         `_HEADER_FIELD(19, m_arp_tha_next[4*8 +: 8])
244         `_HEADER_FIELD(20, m_arp_tha_next[3*8 +: 8])
245         `_HEADER_FIELD(21, m_arp_tha_next[2*8 +: 8])
246         `_HEADER_FIELD(22, m_arp_tha_next[1*8 +: 8])
247         `_HEADER_FIELD(23, m_arp_tha_next[0*8 +: 8])
248         `_HEADER_FIELD(24, m_arp_tpa_next[3*8 +: 8])
249         `_HEADER_FIELD(25, m_arp_tpa_next[2*8 +: 8])
250         `_HEADER_FIELD(26, m_arp_tpa_next[1*8 +: 8])
251         `_HEADER_FIELD(27, m_arp_tpa_next[0*8 +: 8])
252
253         if (ptr_reg == 27/BYTE_LANES && (!KEEP_ENABLE ||
s_eth_payload_axis_tkeep[27%BYTE_LANES])) begin
254           read_arp_header_next = 1'b0;
255         end
256
257         `undef _HEADER_FIELD_
258       end
259
260       if (s_eth_payload_axis_tlast) begin
261         if (read_arp_header_next) begin
262           // don't have the whole header
263           error_header_early_termination_next = 1'b1;
264         end else if (m_arp_hlen_next != 4'd6 ||
m_arp_plen_next != 4'd4) begin
265           // lengths not valid
266           error_invalid_header_next = 1'b1;
267         end else begin
268           // otherwise, transfer tuser
269           m_frame_valid_next = !s_eth_pay-
load_axis_tuser;
270         end

```

```

271
272   ptr_next = 1'b0;
273   read_eth_header_next = 1'b1;
274   read_arp_header_next = 1'b0;
275 end
276
277   if (read_eth_header_next) begin
278     s_eth_hdr_ready_next = !m_frame_valid_next;
279   end else begin
280     s_eth_payload_axis_tready_next = 1'b1;
281   end
282 end
283
284 always @(posedge clk) begin
285   read_eth_header_reg <= read_eth_header_next;
286   read_arp_header_reg <= read_arp_header_next;
287   ptr_reg <= ptr_next;
288
289   s_eth_hdr_ready_reg <= s_eth_hdr_ready_next;
290   s_eth_payload_axis_tready_reg <= s_eth_pay-
load_axis_tready_next;
291
292   m_frame_valid_reg <= m_frame_valid_next;
293
294   m_arp_htype_reg <= m_arp_htype_next;
295   m_arp_ptype_reg <= m_arp_ptype_next;
296   m_arp_hlen_reg <= m_arp_hlen_next;
297   m_arp_plen_reg <= m_arp_plen_next;
298   m_arp_oper_reg <= m_arp_oper_next;
299   m_arp_sha_reg <= m_arp_sha_next;
300   m_arp_spa_reg <= m_arp_spa_next;
301   m_arp_tha_reg <= m_arp_tha_next;
302   m_arp_tpa_reg <= m_arp_tpa_next;
303
304   error_header_early_termination_reg <= er-
ror_header_early_termination_next;
305   error_invalid_header_reg <= error_invalid_header_next;
306
307   busy_reg <= read_arp_header_next;
308
309   // datapath
310   if (store_eth_hdr) begin
311     m_eth_dest_mac_reg <= s_eth_dest_mac;
312     m_eth_src_mac_reg <= s_eth_src_mac;
313     m_eth_type_reg <= s_eth_type;
314   end
315
316   if (rst) begin
317     read_eth_header_reg <= 1'b1;
318     read_arp_header_reg <= 1'b0;
319     ptr_reg <= 0;
320     s_eth_payload_axis_tready_reg <= 1'b0;
321     m_frame_valid_reg <= 1'b0;
322     busy_reg <= 1'b0;
323     error_header_early_termination_reg <= 1'b0;
324     error_invalid_header_reg <= 1'b0;
325   end
326 end
327 endmodule
328
329 endmodule
330
331 `resetall

```

4. alexforenrich_verilog-ethernet/rtl/arp_eth_tx.v (368 lines)

```

1 /*
2
3 Copyright (c) 2014-2020 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.

```

```

22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * ARP ethernet frame transmitter (ARP frame in, Ethernet
33 frame out)
34 */
35 module arp_eth_tx #
36 (
37     // Width of AXI stream interfaces in bits
38     parameter DATA_WIDTH = 8,
39     // Propagate tkeep signal
40     // If disabled, tkeep assumed to be 1'b1
41     parameter KEEP_ENABLE = (DATA_WIDTH>8),
42     // tkeep signal width (words per cycle)
43     parameter KEEP_WIDTH = (DATA_WIDTH/8)
44 )
45 (
46     input wire          clk,
47     input wire          rst,
48
49     /*
50      * ARP frame input
51      */
52     input wire          s_frame_valid,
53     output wire         s_frame_ready,
54     input wire [47:0]   s_eth_dest_mac,
55     input wire [47:0]   s_eth_src_mac,
56     input wire [15:0]   s_eth_type,
57     input wire [15:0]   s_arp_htype,
58     input wire [15:0]   s_arp_ptype,
59     input wire [15:0]   s_arp_oper,
60     input wire [47:0]   s_arp_sha,
61     input wire [31:0]   s_arp_spa,
62     input wire [47:0]   s_arp_tha,
63     input wire [31:0]   s_arp_tpa,
64
65     /*
66      * Ethernet frame output
67      */
68     output wire         m_eth_hdr_valid,
69     input wire          m_eth_hdr_ready,
70     output wire [47:0]  m_eth_dest_mac,
71     output wire [47:0]  m_eth_src_mac,
72     output wire [15:0]  m_eth_type,
73     output wire [DATA_WIDTH-1:0] m_eth_payload_axis_tdata,
74     output wire [KEEP_WIDTH-1:0] m_eth_payload_axis_tkeep,
75     output wire         m_eth_payload_axis_tvalid,
76     input wire          m_eth_payload_axis_tready,
77     output wire         m_eth_payload_axis_tlast,
78     output wire         m_eth_payload_axis_tuser,
79
80     /*
81      * Status signals
82      */
83     output wire         busy
84 );
85
86 parameter BYTE_LANES = KEEP_ENABLE ? KEEP_WIDTH : 1;
87
88 parameter HDR_SIZE = 28;
89
90 parameter CYCLE_COUNT = (HDR_SIZE+BYTE_LANES-1)/BYTE_LANES;
91
92 parameter PTR_WIDTH = $clog2(CYCLE_COUNT);
93
94 parameter OFFSET = HDR_SIZE % BYTE_LANES;
95
96 // bus width assertions
97 initial begin
98     if (BYTE_LANES * 8 != DATA_WIDTH) begin
99         $error("Error: AXI stream interface requires byte (8-
100 bit) granularity (instance %m)");
101     end
102 end
103
104 /*
105 ARP Frame
106
107 Field                Length
108 Destination MAC address 6 octets
109 Source MAC address      6 octets
110 Ethertype (0x0806)      2 octets
111 HTYPE (1)               2 octets
112 PTYPE (0x0800)          2 octets
113 HLEN (6)                1 octets
114 PLEN (4)                1 octets
115 OPER                    2 octets
116 SHA Sender MAC          6 octets
117 SPA Sender IP           4 octets
118 THA Target MAC          6 octets
119 TPA Target IP           4 octets
120

```

```

121 This module receives an ARP frame with header fields in paral-
122 lel and
123 transmits the complete Ethernet payload on an AXI interface.
124
125 */
126 // datapath control signals
127 reg store_frame;
128
129 reg send_arp_header_reg = 1'b0, send_arp_header_next;
130 reg [PTR_WIDTH-1:0] ptr_reg = 0, ptr_next;
131
132 reg [15:0] arp_htype_reg = 16'd0;
133 reg [15:0] arp_ptype_reg = 16'd0;
134 reg [15:0] arp_oper_reg = 16'd0;
135 reg [47:0] arp_sha_reg = 48'd0;
136 reg [31:0] arp_spa_reg = 32'd0;
137 reg [47:0] arp_tha_reg = 48'd0;
138 reg [31:0] arp_tpa_reg = 32'd0;
139
140 reg s_frame_ready_reg = 1'b0, s_frame_ready_next;
141
142 reg m_eth_hdr_valid_reg = 1'b0, m_eth_hdr_valid_next;
143 reg [47:0] m_eth_dest_mac_reg = 48'd0;
144 reg [47:0] m_eth_src_mac_reg = 48'd0;
145 reg [15:0] m_eth_type_reg = 16'd0;
146
147 reg busy_reg = 1'b0;
148
149 // internal datapath
150 reg [DATA_WIDTH-1:0] m_eth_payload_axis_tdata_int;
151 reg [KEEP_WIDTH-1:0] m_eth_payload_axis_tkeep_int;
152 reg m_eth_payload_axis_tvalid_int;
153 reg m_eth_payload_axis_tready_int_reg = 1'b0;
154 reg m_eth_payload_axis_tlast_int;
155 reg m_eth_payload_axis_tuser_int;
156 wire m_eth_payload_axis_tready_int_early;
157
158 assign s_frame_ready = s_frame_ready_reg;
159
160 assign m_eth_hdr_valid = m_eth_hdr_valid_reg;
161 assign m_eth_dest_mac = m_eth_dest_mac_reg;
162 assign m_eth_src_mac = m_eth_src_mac_reg;
163 assign m_eth_type = m_eth_type_reg;
164
165 assign busy = busy_reg;
166
167 always @* begin
168     send_arp_header_next = send_arp_header_reg;
169     ptr_next = ptr_reg;
170
171     s_frame_ready_next = 1'b0;
172
173     store_frame = 1'b0;
174
175     m_eth_hdr_valid_next = m_eth_hdr_valid_reg && !
176 m_eth_hdr_ready;
177
178     m_eth_payload_axis_tdata_int = {DATA_WIDTH{1'b0}};
179     m_eth_payload_axis_tkeep_int = {KEEP_WIDTH{1'b0}};
180     m_eth_payload_axis_tvalid_int = 1'b0;
181     m_eth_payload_axis_tlast_int = 1'b0;
182     m_eth_payload_axis_tuser_int = 1'b0;
183
184     if (s_frame_ready && s_frame_valid) begin
185         store_frame = 1'b1;
186         m_eth_hdr_valid_next = 1'b1;
187         ptr_next = 0;
188         send_arp_header_next = 1'b1;
189     end
190
191     if (m_eth_payload_axis_tready_int_reg) begin
192         if (send_arp_header_reg) begin
193             ptr_next = ptr_reg + 1;
194
195             m_eth_payload_axis_tdata_int = {DATA_WIDTH{1'b0}};
196             m_eth_payload_axis_tkeep_int = {KEEP_WIDTH{1'b0}};
197             m_eth_payload_axis_tvalid_int = 1'b1;
198             m_eth_payload_axis_tlast_int = 1'b0;
199             m_eth_payload_axis_tuser_int = 1'b0;
200
201             `define HEADER_FIELD(offset, field) \
202                 if (ptr_reg == offset/BYTE_LANES) begin \
203                     m_eth_pay-
204 load_axis_tdata_int[(offset%BYTE_LANES)*8 + : 8] = field; \
205                     m_eth_pay-
206 load_axis_tkeep_int[(offset%BYTE_LANES) + : 8] = 1'b1; \
207                 end
208
209             `HEADER_FIELD(0, arp_htype_reg[1*8 + : 8])
210             `HEADER_FIELD(1, arp_htype_reg[0*8 + : 8])
211             `HEADER_FIELD(2, arp_ptype_reg[1*8 + : 8])
212             `HEADER_FIELD(3, arp_ptype_reg[0*8 + : 8])
213             `HEADER_FIELD(4, 8'd6)
214             `HEADER_FIELD(5, 8'd4)
215             `HEADER_FIELD(6, arp_oper_reg[1*8 + : 8])
216             `HEADER_FIELD(7, arp_oper_reg[0*8 + : 8])
217             `HEADER_FIELD(8, arp_sha_reg[5*8 + : 8])
218             `HEADER_FIELD(9, arp_sha_reg[4*8 + : 8])
219             `HEADER_FIELD(10, arp_sha_reg[3*8 + : 8])
220             `HEADER_FIELD(11, arp_sha_reg[2*8 + : 8])

```

```

218 `HEADER_FIELD_ (12, arp_sha_reg[1*8 +: 8])
219 `HEADER_FIELD_ (13, arp_sha_reg[0*8 +: 8])
220 `HEADER_FIELD_ (14, arp_spa_reg[3*8 +: 8])
221 `HEADER_FIELD_ (15, arp_spa_reg[2*8 +: 8])
222 `HEADER_FIELD_ (16, arp_spa_reg[1*8 +: 8])
223 `HEADER_FIELD_ (17, arp_spa_reg[0*8 +: 8])
224 `HEADER_FIELD_ (18, arp_tha_reg[5*8 +: 8])
225 `HEADER_FIELD_ (19, arp_tha_reg[4*8 +: 8])
226 `HEADER_FIELD_ (20, arp_tha_reg[3*8 +: 8])
227 `HEADER_FIELD_ (21, arp_tha_reg[2*8 +: 8])
228 `HEADER_FIELD_ (22, arp_tha_reg[1*8 +: 8])
229 `HEADER_FIELD_ (23, arp_tha_reg[0*8 +: 8])
230 `HEADER_FIELD_ (24, arp_tpa_reg[3*8 +: 8])
231 `HEADER_FIELD_ (25, arp_tpa_reg[2*8 +: 8])
232 `HEADER_FIELD_ (26, arp_tpa_reg[1*8 +: 8])
233 `HEADER_FIELD_ (27, arp_tpa_reg[0*8 +: 8])
234
235 if (ptr_reg == 27/BYTE_LANES) begin
236     m_eth_payload_axis_tlast_int = 1'b1;
237     send_arp_header_next = 1'b0;
238 end
239
240 `undef `HEADER_FIELD_
241 end
242 end
243
244 s_frame_ready_next = !m_eth_hdr_valid_next && !
send_arp_header_next;
245 end
246
247 always @(posedge clk) begin
248     send_arp_header_reg <= send_arp_header_next;
249     ptr_reg <= ptr_next;
250
251     s_frame_ready_reg <= s_frame_ready_next;
252
253     m_eth_hdr_valid_reg <= m_eth_hdr_valid_next;
254
255     busy_reg <= send_arp_header_next;
256
257     if (store_frame) begin
258         m_eth_dest_mac_reg <= s_eth_dest_mac;
259         m_eth_src_mac_reg <= s_eth_src_mac;
260         m_eth_type_reg <= s_eth_type;
261         arp_htype_reg <= s_arp_htype;
262         arp_ptype_reg <= s_arp_ptype;
263         arp_oper_reg <= s_arp_oper;
264         arp_sha_reg <= s_arp_sha;
265         arp_spa_reg <= s_arp_spa;
266         arp_tha_reg <= s_arp_tha;
267         arp_tpa_reg <= s_arp_tpa;
268     end
269
270     if (rst) begin
271         send_arp_header_reg <= 1'b0;
272         ptr_reg <= 0;
273         s_frame_ready_reg <= 1'b0;
274         m_eth_hdr_valid_reg <= 1'b0;
275         busy_reg <= 1'b0;
276     end
277 end
278
279 // output datapath logic
280 reg [DATA_WIDTH-1:0] m_eth_payload_axis_tdata_reg =
{DATA_WIDTH{1'b0}};
281 reg [KEEP_WIDTH-1:0] m_eth_payload_axis_tkeep_reg =
{KEEP_WIDTH{1'b0}};
282 reg m_eth_payload_axis_tvalid_reg = 1'b0,
m_eth_payload_axis_tvalid_next;
283 reg m_eth_payload_axis_tlast_reg = 1'b0;
284 reg m_eth_payload_axis_tuser_reg = 1'b0;
285
286 reg [DATA_WIDTH-1:0] temp_m_eth_payload_axis_tdata_reg =
{DATA_WIDTH{1'b0}};
287 reg [KEEP_WIDTH-1:0] temp_m_eth_payload_axis_tkeep_reg =
{KEEP_WIDTH{1'b0}};
288 reg temp_m_eth_payload_axis_tvalid_reg =
1'b0, temp_m_eth_payload_axis_tvalid_next;
289 reg temp_m_eth_payload_axis_tlast_reg = 1'b0;
290 reg temp_m_eth_payload_axis_tuser_reg = 1'b0;
291
292 // datapath control
293 reg store_eth_payload_int_to_output;
294 reg store_eth_payload_int_to_temp;
295 reg store_eth_payload_axis_temp_to_output;
296
297 assign m_eth_payload_axis_tdata = m_eth_pay-
load_axis_tdata_reg;
298 assign m_eth_payload_axis_tkeep = KEEP_ENABLE ? m_eth_pay-
load_axis_tkeep_reg : {KEEP_WIDTH{1'b1}};
299 assign m_eth_payload_axis_tvalid = m_eth_pay-
load_axis_tvalid_reg;
300 assign m_eth_payload_axis_tlast = m_eth_pay-
load_axis_tlast_reg;
301 assign m_eth_payload_axis_tuser = m_eth_pay-
load_axis_tuser_reg;
302
303 // enable ready input next cycle if output is ready or if both
output registers are empty
304 assign m_eth_payload_axis_tready_int_early = m_eth_pay-

```

```

load_axis_tready || (!temp_m_eth_payload_axis_tvalid_reg && !
m_eth_payload_axis_tvalid_reg);
305
306 always @* begin
307     // transfer sink ready state to source
308     m_eth_payload_axis_tvalid_next = m_eth_pay-
load_axis_tvalid_reg;
309     temp_m_eth_payload_axis_tvalid_next = temp_m_eth_pay-
load_axis_tvalid_reg;
310
311     store_eth_payload_int_to_output = 1'b0;
312     store_eth_payload_int_to_temp = 1'b0;
313     store_eth_payload_axis_temp_to_output = 1'b0;
314
315     if (m_eth_payload_axis_tready_int_reg) begin
316         // input is ready
317         if (m_eth_payload_axis_tready || !m_eth_pay-
load_axis_tvalid_reg) begin
318             // output is ready or currently not valid, trans-
fer data to output
319             m_eth_payload_axis_tvalid_next = m_eth_pay-
load_axis_tvalid_int;
320             store_eth_payload_int_to_output = 1'b1;
321         end else begin
322             // output is not ready, store input in temp
323             temp_m_eth_payload_axis_tvalid_next = m_eth_pay-
load_axis_tvalid_int;
324             store_eth_payload_int_to_temp = 1'b1;
325         end
326     end else if (m_eth_payload_axis_tready) begin
327         // input is not ready, but output is ready
328         m_eth_payload_axis_tvalid_next = temp_m_eth_pay-
load_axis_tvalid_reg;
329         temp_m_eth_payload_axis_tvalid_next = 1'b0;
330         store_eth_payload_axis_temp_to_output = 1'b1;
331     end
332 end
333
334 always @(posedge clk) begin
335     m_eth_payload_axis_tvalid_reg <= m_eth_pay-
load_axis_tvalid_next;
336     m_eth_payload_axis_tready_int_reg <= m_eth_pay-
load_axis_tready_int_early;
337     temp_m_eth_payload_axis_tvalid_reg <= temp_m_eth_pay-
load_axis_tvalid_next;
338
339     // datapath
340     if (store_eth_payload_int_to_output) begin
341         m_eth_payload_axis_tdata_reg <= m_eth_pay-
load_axis_tdata_int;
342         m_eth_payload_axis_tkeep_reg <= m_eth_pay-
load_axis_tkeep_int;
343         m_eth_payload_axis_tlast_reg <= m_eth_pay-
load_axis_tlast_int;
344         m_eth_payload_axis_tuser_reg <= m_eth_pay-
load_axis_tuser_int;
345     end else if (store_eth_payload_axis_temp_to_output) begin
346         m_eth_payload_axis_tdata_reg <= temp_m_eth_pay-
load_axis_tdata_reg;
347         m_eth_payload_axis_tkeep_reg <= temp_m_eth_pay-
load_axis_tkeep_reg;
348         m_eth_payload_axis_tlast_reg <= temp_m_eth_pay-
load_axis_tlast_reg;
349         m_eth_payload_axis_tuser_reg <= temp_m_eth_pay-
load_axis_tuser_reg;
350     end
351
352     if (store_eth_payload_int_to_temp) begin
353         temp_m_eth_payload_axis_tdata_reg <= m_eth_pay-
load_axis_tdata_int;
354         temp_m_eth_payload_axis_tkeep_reg <= m_eth_pay-
load_axis_tkeep_int;
355         temp_m_eth_payload_axis_tlast_reg <= m_eth_pay-
load_axis_tlast_int;
356         temp_m_eth_payload_axis_tuser_reg <= m_eth_pay-
load_axis_tuser_int;
357     end
358
359     if (rst) begin
360         m_eth_payload_axis_tvalid_reg <= 1'b0;
361         m_eth_payload_axis_tready_int_reg <= 1'b0;
362         temp_m_eth_payload_axis_tvalid_reg <= 1'b0;
363     end
364 end
365
366 endmodule
367
368 `resetall

```

5. alexforenrich_verilog-ethernet/rtl/axis_baser_rx_64.v (560 lines)

```

1 /*
2
3 Copyright (c) 2019 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal

```

```

7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32  * AXI4-Stream 10GBASE-R frame receiver (10GBASE-R in, AXI
out)
33  */
34 module axis_baser_rx_64 #
35 (
36     parameter DATA_WIDTH = 64,
37     parameter KEEP_WIDTH = (DATA_WIDTH/8),
38     parameter HDR_WIDTH = 2,
39     parameter PTP_TS_ENABLE = 0,
40     parameter PTP_TS_FMT_TOD = 1,
41     parameter PTP_TS_WIDTH = PTP_TS_FMT_TOD ? 96 : 64,
42     parameter USER_WIDTH = (PTP_TS_ENABLE ? PTP_TS_WIDTH : 0)
+ 1
43 )
44 (
45     input wire          clk,
46     input wire          rst,
47
48     /*
49      * 10GBASE-R encoded input
50      */
51     input wire [DATA_WIDTH-1:0] encoded_rx_data,
52     input wire [HDR_WIDTH-1:0] encoded_rx_hdr,
53
54     /*
55      * AXI output
56      */
57     output wire [DATA_WIDTH-1:0] m_axis_tdata,
58     output wire [KEEP_WIDTH-1:0] m_axis_tkeep,
59     output wire                m_axis_tvalid,
60     output wire                m_axis_tlast,
61     output wire [USER_WIDTH-1:0] m_axis_tuser,
62
63     /*
64      * PTP
65      */
66     input wire [PTP_TS_WIDTH-1:0] ptp_ts,
67
68     /*
69      * Configuration
70      */
71     input wire                cfg_rx_enable,
72
73     /*
74      * Status
75      */
76     output wire [1:0]          start_packet,
77     output wire                error_bad_frame,
78     output wire                error_bad_fcs,
79     output wire                rx_bad_block
80 );
81
82 // bus width assertions
83 initial begin
84     if (DATA_WIDTH != 64) begin
85         $error("Error: Interface width must be 64");
86         $finish;
87     end
88
89     if (KEEP_WIDTH * 8 != DATA_WIDTH) begin
90         $error("Error: Interface requires byte (8-bit) granu-
larity");
91         $finish;
92     end
93 end

```

```

94     if (HDR_WIDTH != 2) begin
95         $error("Error: HDR_WIDTH must be 2");
96         $finish;
97     end
98 end
99
100 localparam [7:0]
101     ETH_PRE = 8'h55,
102     ETH_SFD = 8'hD5;
103
104 localparam [7:0]
105     XGMII_IDLE = 8'h07,
106     XGMII_START = 8'hfb,
107     XGMII_TERM = 8'hfd,
108     XGMII_ERROR = 8'hfe;
109
110 localparam [6:0]
111     CTRL_IDLE = 7'h00,
112     CTRL_LPI = 7'h06,
113     CTRL_ERROR = 7'h1e,
114     CTRL_RES_0 = 7'h2d,
115     CTRL_RES_1 = 7'h33,
116     CTRL_RES_2 = 7'h4b,
117     CTRL_RES_3 = 7'h55,
118     CTRL_RES_4 = 7'h66,
119     CTRL_RES_5 = 7'h78;
120
121 localparam [3:0]
122     O_SEQ_OS = 4'h0,
123     O_SIG_OS = 4'hf;
124
125 localparam [1:0]
126     SYNC_DATA = 2'b10,
127     SYNC_CTRL = 2'b01;
128
129 localparam [7:0]
130     BLOCK_TYPE_CTRL = 8'h1e, // C7 C6 C5 C4 C3 C2 C1 C0 BT
131     BLOCK_TYPE_OS_4 = 8'h2d, // D7 D6 D5 D4 C3 C2 C1 C0 BT
132     BLOCK_TYPE_START_4 = 8'h33, // D7 D6 D5 C3 C2 C1 C0 BT
133     BLOCK_TYPE_OS_START = 8'h66, // D7 D6 D5 00 D3 D2 D1 BT
134     BLOCK_TYPE_OS_04 = 8'h55, // D7 D6 D5 D4 00 D3 D2 D1 BT
135     BLOCK_TYPE_START_0 = 8'h78, // D7 D6 D5 D4 D3 D2 D1 BT
136     BLOCK_TYPE_OS_0 = 8'h4b, // C7 C6 C5 C4 00 D3 D2 D1 BT
137     BLOCK_TYPE_TERM_0 = 8'h87, // C7 C6 C5 C4 C3 C2 C1 BT
138     BLOCK_TYPE_TERM_1 = 8'h99, // C7 C6 C5 C4 C3 C2 00 BT
139     BLOCK_TYPE_TERM_2 = 8'haa, // C7 C6 C5 C4 C3 D1 00 BT
140     BLOCK_TYPE_TERM_3 = 8'hb4, // C7 C6 C5 C4 D2 D1 00 BT
141     BLOCK_TYPE_TERM_4 = 8'hcc, // C7 C6 C5 D3 D2 D1 00 BT
142     BLOCK_TYPE_TERM_5 = 8'hd2, // C7 C6 D4 D3 D2 D1 00 BT
143     BLOCK_TYPE_TERM_6 = 8'he1, // C7 D5 D4 D3 D2 D1 00 BT
144     BLOCK_TYPE_TERM_7 = 8'hff, // D6 D5 D4 D3 D2 D1 00 BT
145
146 localparam [3:0]
147     INPUT_TYPE_IDLE = 4'd0,
148     INPUT_TYPE_ERROR = 4'd1,
149     INPUT_TYPE_START_0 = 4'd2,
150     INPUT_TYPE_START_4 = 4'd3,
151     INPUT_TYPE_DATA = 4'd4,
152     INPUT_TYPE_TERM_0 = 4'd8,
153     INPUT_TYPE_TERM_1 = 4'd9,
154     INPUT_TYPE_TERM_2 = 4'd10,
155     INPUT_TYPE_TERM_3 = 4'd11,
156     INPUT_TYPE_TERM_4 = 4'd12,
157     INPUT_TYPE_TERM_5 = 4'd13,
158     INPUT_TYPE_TERM_6 = 4'd14,
159     INPUT_TYPE_TERM_7 = 4'd15;
160
161 localparam [1:0]
162     STATE_IDLE = 2'd0,
163     STATE_PAYLOAD = 2'd1,
164     STATE_LAST = 2'd2;
165
166 reg [1:0] state_reg = STATE_IDLE, state_next;
167
168 // datapath control signals
169 reg reset_crc;
170
171 reg lanes_swapped = 1'b0;
172 reg [31:0] swap_data = 32'd0;
173
174 reg delay_type_valid = 1'b0;
175 reg [3:0] delay_type = INPUT_TYPE_IDLE;
176
177 reg [DATA_WIDTH-1:0] encoded_rx_data_masked =
{DATA_WIDTH{1'b0}};
178
179 reg [DATA_WIDTH-1:0] input_data_d0 = {DATA_WIDTH{1'b0}};
180 reg [DATA_WIDTH-1:0] input_data_d1 = {DATA_WIDTH{1'b0}};
181
182 reg [3:0] input_type_d0 = INPUT_TYPE_IDLE;
183 reg [3:0] input_type_d1 = INPUT_TYPE_IDLE;
184
185 reg [DATA_WIDTH-1:0] m_axis_tdata_reg = {DATA_WIDTH{1'b0}},
m_axis_tdata_next;
186 reg [KEEP_WIDTH-1:0] m_axis_tkeep_reg = {KEEP_WIDTH{1'b0}},
m_axis_tkeep_next;
187 reg m_axis_tvalid_reg = 1'b0, m_axis_tvalid_next;
188 reg m_axis_tlast_reg = 1'b0, m_axis_tlast_next;
189 reg [USER_WIDTH-1:0] m_axis_tuser_reg = {USER_WIDTH{1'b0}},
m_axis_tuser_next;
190

```



```

191 reg [1:0] start_packet_reg = 2'b00;
192 reg error_bad_frame_reg = 1'b0, error_bad_frame_next;
193 reg error_bad_fcs_reg = 1'b0, error_bad_fcs_next;
194 reg rx_bad_block_reg = 1'b0;
195
196 reg [PTP_TS_WIDTH-1:0] ptp_ts_reg = 0;
197 reg [PTP_TS_WIDTH-1:0] ptp_ts_adj_reg = 0;
198 reg ptp_ts_borrow_reg = 0;
199
200 reg [31:0] crc_state = 32'hFFFFFFF;
201
202 wire [31:0] crc_next;
203
204 wire [7:0] crc_valid;
205 reg [7:0] crc_valid_save;
206
207 assign crc_valid[7] = crc_next == ~32'h2144df1c;
208 assign crc_valid[6] = crc_next == ~32'hc622f71d;
209 assign crc_valid[5] = crc_next == ~32'hb1c2a1a3;
210 assign crc_valid[4] = crc_next == ~32'h9d6cdf7e;
211 assign crc_valid[3] = crc_next == ~32'h6522df69;
212 assign crc_valid[2] = crc_next == ~32'he60914ae;
213 assign crc_valid[1] = crc_next == ~32'he38a6876;
214 assign crc_valid[0] = crc_next == ~32'h6b87b1ec;
215
216 reg [4+16-1:0] last_ts_reg = 0;
217 reg [4+16-1:0] ts_inc_reg = 0;
218
219 assign m_axis_tdata = m_axis_tdata_reg;
220 assign m_axis_tkeep = m_axis_tkeep_reg;
221 assign m_axis_tvalid = m_axis_tvalid_reg;
222 assign m_axis_tlast = m_axis_tlast_reg;
223 assign m_axis_tuser = m_axis_tuser_reg;
224
225 assign start_packet = start_packet_reg;
226 assign error_bad_frame = error_bad_frame_reg;
227 assign error_bad_fcs = error_bad_fcs_reg;
228 assign rx_bad_block = rx_bad_block_reg;
229
230 lfsr #(
231     .LFSR_WIDTH(32),
232     .LFSR_POLY(32'h4c11db7),
233     .LFSR_CONFIG("GALOIS"),
234     .LFSR_FEED_FORWARD(0),
235     .REVERSE(1),
236     .DATA_WIDTH(64),
237     .STYLE("AUTO")
238 )
239 eth_crc (
240     .data_in(input_data_d0),
241     .state_in(crc_state),
242     .data_out(),
243     .state_out(crc_next)
244 );
245
246 // Mask input data
247 integer j;
248
249 always @* begin
250     // minimal checks of control info to simplify datapath logic, full checks performed later
251     if (encoded_rx_hdr[0] == 0) begin
252         encoded_rx_data_masked = encoded_rx_data;
253     end else if (encoded_rx_data[7]) begin
254         // terminate
255         case (encoded_rx_data[6:4])
256             3'd0: encoded_rx_data_masked = 64'd0;
257             3'd1: encoded_rx_data_masked = {56'd0, encoded_rx_data[15:8]};
258             3'd2: encoded_rx_data_masked = {48'd0, encoded_rx_data[23:8]};
259             3'd3: encoded_rx_data_masked = {40'd0, encoded_rx_data[31:8]};
260             3'd4: encoded_rx_data_masked = {32'd0, encoded_rx_data[39:8]};
261             3'd5: encoded_rx_data_masked = {24'd0, encoded_rx_data[47:8]};
262             3'd6: encoded_rx_data_masked = {16'd0, encoded_rx_data[55:8]};
263             3'd7: encoded_rx_data_masked = {8'd0, encoded_rx_data[63:8]};
264         endcase
265     end else begin
266         // start, OS, etc.
267         encoded_rx_data_masked = {encoded_rx_data[63:8], 8'd0};
268     end
269 end
270
271 always @* begin
272     state_next = STATE_IDLE;
273     reset_crc = 1'b0;
274
275     m_axis_tdata_next = input_data_d1;
276     m_axis_tkeep_next = 8'd0;
277     m_axis_tvalid_next = 1'b0;
278     m_axis_tlast_next = 1'b0;
279     m_axis_tuser_next = m_axis_tuser_reg;
280     m_axis_tuser_next[0] = 1'b0;
281
282

```

```

283     error_bad_frame_next = 1'b0;
284     error_bad_fcs_next = 1'b0;
285
286     case (state_reg)
287     STATE_IDLE: begin
288         // idle state - wait for packet
289         reset_crc = 1'b1;
290
291         if (input_type_d1 == INPUT_TYPE_START_0 &&
292             cfg_rx_enable) begin
293             // start condition
294             reset_crc = 1'b0;
295             state_next = STATE_PAYLOAD;
296         end else begin
297             state_next = STATE_IDLE;
298         end
299     end
300     STATE_PAYLOAD: begin
301         // read payload
302         m_axis_tdata_next = input_data_d1;
303         m_axis_tkeep_next = 8'hff;
304         m_axis_tvalid_next = 1'b1;
305         m_axis_tlast_next = 1'b0;
306         m_axis_tuser_next[0] = 1'b0;
307
308         if (input_type_d0[3]) begin
309             // INPUT_TYPE_TERM_*
310             reset_crc = 1'b1;
311         end
312
313         if (PTP_TS_ENABLE) begin
314             m_axis_tuser_next[1+: PTP_TS_WIDTH] = (!
315                 PTP_TS_FMT_TOD || ptp_ts_borrow_reg) ? ptp_ts_reg : ptp_ts_adj_reg;
316         end
317
318         if (input_type_d0 == INPUT_TYPE_DATA) begin
319             state_next = STATE_PAYLOAD;
320         end else if (input_type_d0[3]) begin
321             // INPUT_TYPE_TERM_*
322             if (input_type_d0 <= INPUT_TYPE_TERM_4) begin
323                 // end this cycle
324                 case (input_type_d0)
325                     INPUT_TYPE_TERM_0: m_axis_tkeep_next =
326                         8'b00001111;
327                     INPUT_TYPE_TERM_1: m_axis_tkeep_next =
328                         8'b00011111;
329                     INPUT_TYPE_TERM_2: m_axis_tkeep_next =
330                         8'b00111111;
331                     INPUT_TYPE_TERM_3: m_axis_tkeep_next =
332                         8'b01111111;
333                     INPUT_TYPE_TERM_4: m_axis_tkeep_next =
334                         8'b11111111;
335                 endcase
336                 m_axis_tlast_next = 1'b1;
337             end
338             if ((input_type_d0 == INPUT_TYPE_TERM_0 &&
339                 crc_valid_save[7]) ||
340                 (input_type_d0 == INPUT_TYPE_TERM_1 &&
341                 crc_valid[0]) ||
342                 (input_type_d0 == INPUT_TYPE_TERM_2 &&
343                 crc_valid[1]) ||
344                 (input_type_d0 == INPUT_TYPE_TERM_3 &&
345                 crc_valid[2]) ||
346                 (input_type_d0 == INPUT_TYPE_TERM_4 &&
347                 crc_valid[3])) begin
348                 // CRC valid
349                 end else begin
350                     m_axis_tuser_next[0] = 1'b1;
351                     error_bad_frame_next = 1'b1;
352                     error_bad_fcs_next = 1'b1;
353                 end
354                 state_next = STATE_IDLE;
355             end else begin
356                 // need extra cycle
357                 state_next = STATE_LAST;
358             end
359         end
360     end
361     STATE_LAST: begin
362         // last cycle of packet
363         m_axis_tdata_next = input_data_d1;
364         m_axis_tkeep_next = 8'hff;
365         m_axis_tvalid_next = 1'b1;
366         m_axis_tlast_next = 1'b1;
367         m_axis_tuser_next[0] = 1'b0;
368

```

```

362
363     reset_crc = 1'b1;
364
365     case (input_type_d1)
366     INPUT_TYPE_TERM_5: m_axis_tkeep_next =
8'b00000001;
367     INPUT_TYPE_TERM_6: m_axis_tkeep_next =
8'b00000011;
368     INPUT_TYPE_TERM_7: m_axis_tkeep_next =
8'b00000111;
369     endcase
370
371     if ((input_type_d1 == INPUT_TYPE_TERM_5 &&
crc_valid_save[4]) ||
372     (input_type_d1 == INPUT_TYPE_TERM_6 &&
crc_valid_save[5]) ||
373     (input_type_d1 == INPUT_TYPE_TERM_7 &&
crc_valid_save[6])) begin
374         // CRC valid
375     end else begin
376         m_axis_tuser_next[0] = 1'b1;
377         error_bad_frame_next = 1'b1;
378         error_bad_fcs_next = 1'b1;
379     end
380
381     state_next = STATE_IDLE;
382 end
383 endcase
384 end
385
386 always @(posedge clk) begin
387     state_reg <= state_next;
388
389     m_axis_tdata_reg <= m_axis_tdata_next;
390     m_axis_tkeep_reg <= m_axis_tkeep_next;
391     m_axis_tvalid_reg <= m_axis_tvalid_next;
392     m_axis_tlast_reg <= m_axis_tlast_next;
393     m_axis_tuser_reg <= m_axis_tuser_next;
394
395     start_packet_reg <= 2'b00;
396     error_bad_frame_reg <= error_bad_frame_next;
397     error_bad_fcs_reg <= error_bad_fcs_next;
398     rx_bad_block_reg <= 1'b0;
399
400     delay_type_valid <= 1'b0;
401
402     swap_data <= encoded_rx_data_masked[63:32];
403
404     if (PTP_TS_ENABLE && PTP_TS_FMT_TOD) begin
405         // ns field rollover
406         ptp_ts_adj_reg[15:0] <= ptp_ts_reg[15:0];
407         {ptp_ts_borrow_reg, ptp_ts_adj_reg[45:16]} <=
$signed({1'b0, ptp_ts_reg[45:16]}) - $signed(31'd1000000000);
408         ptp_ts_adj_reg[47:46] <= 0;
409         ptp_ts_adj_reg[95:48] <= ptp_ts_reg[95:48] + 1;
410     end
411
412     if (encoded_rx_hdr == SYNC_CTRL && encoded_rx_data[7:0] ==
BLOCK_TYPE_START_0) begin
413         lanes_swapped <= 1'b0;
414         input_type_d0 <= INPUT_TYPE_START_0;
415         input_data_d0 <= encoded_rx_data_masked;
416     end else if (encoded_rx_hdr == SYNC_CTRL && (en-
coded_rx_data[7:0] == BLOCK_TYPE_START_4 || encoded_rx_data[7:0] ==
BLOCK_TYPE_OS_START)) begin
417         lanes_swapped <= 1'b1;
418         delay_type_valid <= 1'b1;
419
420         if (delay_type_valid) begin
421             input_type_d0 <= delay_type;
422         end else begin
423             input_type_d0 <= INPUT_TYPE_IDLE;
424         end
425         input_data_d0 <= {encoded_rx_data_masked[31:0],
swap_data};
426     end else if (lanes_swapped) begin
427         if (delay_type_valid) begin
428             input_type_d0 <= delay_type;
429         end else if (encoded_rx_hdr == SYNC_DATA) begin
430             input_type_d0 <= INPUT_TYPE_DATA;
431         end else if (encoded_rx_hdr == SYNC_CTRL) begin
432             case (encoded_rx_data[7:0])
433             BLOCK_TYPE_TERM_0: input_type_d0 <= IN-
PUT_TYPE_TERM_4;
434             BLOCK_TYPE_TERM_1: input_type_d0 <= IN-
PUT_TYPE_TERM_5;
435             BLOCK_TYPE_TERM_2: input_type_d0 <= IN-
PUT_TYPE_TERM_6;
436             BLOCK_TYPE_TERM_3: input_type_d0 <= IN-
PUT_TYPE_TERM_7;
437             BLOCK_TYPE_TERM_4: begin
438                 delay_type_valid <= 1'b1;
439                 input_type_d0 <= INPUT_TYPE_DATA;
440             end
441             BLOCK_TYPE_TERM_5: begin
442                 delay_type_valid <= 1'b1;
443                 input_type_d0 <= INPUT_TYPE_DATA;
444             end
445             BLOCK_TYPE_TERM_6: begin
446                 delay_type_valid <= 1'b1;

```

```

447         input_type_d0 <= INPUT_TYPE_DATA;
448     end
449     BLOCK_TYPE_TERM_7: begin
450         delay_type_valid <= 1'b1;
451         input_type_d0 <= INPUT_TYPE_DATA;
452     end
453     default: begin
454         rx_bad_block_reg <= 1'b1;
455         input_type_d0 <= INPUT_TYPE_ERROR;
456     end
457 endcase
458 end else begin
459     rx_bad_block_reg <= 1'b1;
460     input_type_d0 <= INPUT_TYPE_ERROR;
461 end
462 input_data_d0 <= {encoded_rx_data_masked[31:0],
swap_data};
463 end else begin
464     if (encoded_rx_hdr == SYNC_DATA) begin
465         input_type_d0 <= INPUT_TYPE_DATA;
466     end else if (encoded_rx_hdr == SYNC_CTRL) begin
467         case (encoded_rx_data[7:0])
468         BLOCK_TYPE_TERM_0: input_type_d0 <= IN-
PUT_TYPE_TERM_0;
469         BLOCK_TYPE_TERM_1: input_type_d0 <= IN-
PUT_TYPE_TERM_1;
470         BLOCK_TYPE_TERM_2: input_type_d0 <= IN-
PUT_TYPE_TERM_2;
471         BLOCK_TYPE_TERM_3: input_type_d0 <= IN-
PUT_TYPE_TERM_3;
472         BLOCK_TYPE_TERM_4: input_type_d0 <= IN-
PUT_TYPE_TERM_4;
473         BLOCK_TYPE_TERM_5: input_type_d0 <= IN-
PUT_TYPE_TERM_5;
474         BLOCK_TYPE_TERM_6: input_type_d0 <= IN-
PUT_TYPE_TERM_6;
475         BLOCK_TYPE_TERM_7: input_type_d0 <= IN-
PUT_TYPE_TERM_7;
476         default: begin
477             rx_bad_block_reg <= 1'b1;
478             input_type_d0 <= INPUT_TYPE_ERROR;
479         end
480     endcase
481     end else begin
482         rx_bad_block_reg <= 1'b1;
483         input_type_d0 <= INPUT_TYPE_ERROR;
484     end
485     input_data_d0 <= encoded_rx_data_masked;
486 end
487
488     if (encoded_rx_hdr == SYNC_DATA) begin
489         delay_type <= INPUT_TYPE_DATA;
490     end else if (encoded_rx_hdr == SYNC_CTRL) begin
491         case (encoded_rx_data[7:0])
492         BLOCK_TYPE_START_4: delay_type <= IN-
PUT_TYPE_START_0;
493         BLOCK_TYPE_TERM_0: delay_type <= IN-
PUT_TYPE_TERM_4;
494         BLOCK_TYPE_TERM_1: delay_type <= IN-
PUT_TYPE_TERM_5;
495         BLOCK_TYPE_TERM_2: delay_type <= IN-
PUT_TYPE_TERM_6;
496         BLOCK_TYPE_TERM_3: delay_type <= IN-
PUT_TYPE_TERM_7;
497         BLOCK_TYPE_TERM_4: delay_type <= IN-
PUT_TYPE_TERM_0;
498         BLOCK_TYPE_TERM_5: delay_type <= IN-
PUT_TYPE_TERM_1;
499         BLOCK_TYPE_TERM_6: delay_type <= IN-
PUT_TYPE_TERM_2;
500         BLOCK_TYPE_TERM_7: delay_type <= IN-
PUT_TYPE_TERM_3;
501         default: delay_type <= INPUT_TYPE_ERROR;
502     endcase
503     end else begin
504         delay_type <= INPUT_TYPE_ERROR;
505     end
506
507     if (delay_type == INPUT_TYPE_START_0 && delay_type_valid)
begin
508         start_packet_reg <= 2'b10;
509         if (PTP_TS_FMT_TOD) begin
510             ptp_ts_reg[45:0] <= ptp_ts[45:0] + (ts_inc_reg >>
1);
511             ptp_ts_reg[95:48] <= ptp_ts[95:48];
512         end else begin
513             ptp_ts_reg <= ptp_ts + (ts_inc_reg >> 1);
514         end
515     end
516
517     if (input_type_d0 == INPUT_TYPE_START_0) begin
518         if (!lanes_swapped) begin
519             start_packet_reg <= 2'b01;
520             ptp_ts_reg <= ptp_ts;
521         end
522     end
523
524     input_type_d1 <= input_type_d0;
525     input_data_d1 <= input_data_d0;
526
527     if (reset_crc) begin

```

```

528     crc_state <= 32'hFFFFFFF;
529 end else begin
530     crc_state <= crc_next;
531 end
532
533     crc_valid_save <= crc_valid;
534
535     last_ts_reg <= ptp_ts;
536     ts_inc_reg <= ptp_ts - last_ts_reg;
537
538     if (rst) begin
539         state_reg <= STATE_IDLE;
540
541         m_axis_tvalid_reg <= 1'b0;
542
543         start_packet_reg <= 2'b00;
544         error_bad_frame_reg <= 1'b0;
545         error_bad_fcs_reg <= 1'b0;
546         rx_bad_block_reg <= 1'b0;
547
548         input_type_d0 <= INPUT_TYPE_IDLE;
549         input_type_d1 <= INPUT_TYPE_IDLE;
550
551         lanes_swapped <= 1'b0;
552
553         delay_type_valid <= 1'b0;
554         delay_type <= INPUT_TYPE_IDLE;
555     end
556 end
557
558 endmodule
559
560 `resetall

```

6. alexforenrich_verilog-ethernet/rtl/axis_baser_tx_64.v (792 lines)

```

1 /*
2
3 Copyright (c) 2019 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * AXI4-Stream 10GBASE-R frame transmitter (AXI in, 10GBASE-R
out)
33 */
34 module axis_baser_tx_64 #
35 (
36     parameter DATA_WIDTH = 64,
37     parameter KEEP_WIDTH = (DATA_WIDTH/8),
38     parameter HDR_WIDTH = 2,
39     parameter ENABLE_PADDING = 1,
40     parameter ENABLE_DIC = 1,
41     parameter MIN_FRAME_LENGTH = 64,
42     parameter PTP_TS_ENABLE = 0,
43     parameter PTP_TS_FMT_TOD = 1,
44     parameter PTP_TS_WIDTH = PTP_TS_FMT_TOD ? 96 : 64,
45     parameter PTP_TS_CTRL_IN_TUSER = 0,
46     parameter PTP_TAG_ENABLE = PTP_TS_ENABLE,
47     parameter PTP_TAG_WIDTH = 16,
48     parameter USER_WIDTH = (PTP_TS_ENABLE ? (PTP_TAG_ENABLE ?
PTP_TAG_WIDTH : 0) + (PTP_TS_CTRL_IN_TUSER ? 1 : 0) : 0) + 1

```

```

49 )
50 (
51     input wire          clk,
52     input wire          rst,
53
54     /*
55      * AXI input
56      */
57     input wire [DATA_WIDTH-1:0] s_axis_tdata,
58     input wire [KEEP_WIDTH-1:0] s_axis_tkeep,
59     input wire          s_axis_tvalid,
60     output wire          s_axis_tready,
61     input wire          s_axis_tlast,
62     input wire [USER_WIDTH-1:0] s_axis_tuser,
63
64     /*
65      * 10GBASE-R encoded interface
66      */
67     output wire [DATA_WIDTH-1:0] encoded_tx_data,
68     output wire [HDR_WIDTH-1:0] encoded_tx_hdr,
69
70     /*
71      * PTP
72      */
73     input wire [PTP_TS_WIDTH-1:0] ptp_ts,
74     output wire [PTP_TS_WIDTH-1:0] m_axis_ptp_ts,
75     output wire [PTP_TAG_WIDTH-1:0] m_axis_ptp_ts_tag,
76     output wire          m_axis_ptp_ts_valid,
77
78     /*
79      * Configuration
80      */
81     input wire [7:0]          cfg_ifg,
82     input wire          cfg_tx_enable,
83
84     /*
85      * Status
86      */
87     output wire [1:0]          start_packet,
88     output wire          error_underflow
89 );
90
91 parameter EMPTY_WIDTH = $clog2(KEEP_WIDTH);
92 parameter MIN_LEN_WIDTH = $clog2(MIN_FRAME_LENGTH-4-
KEEP_WIDTH+1);
93
94 // bus width assertions
95 initial begin
96     if (DATA_WIDTH != 64) begin
97         $error("Error: Interface width must be 64");
98         $finish;
99     end
100
101     if (KEEP_WIDTH * 8 != DATA_WIDTH) begin
102         $error("Error: Interface requires byte (8-bit) granu-
larity");
103         $finish;
104     end
105
106     if (HDR_WIDTH != 2) begin
107         $error("Error: HDR_WIDTH must be 2");
108         $finish;
109     end
110 end
111
112 localparam [7:0]
113     ETH_PRE = 8'h55,
114     ETH_SFD = 8'hD5;
115
116 localparam [6:0]
117     CTRL_IDLE = 7'h00,
118     CTRL_LPI = 7'h06,
119     CTRL_ERROR = 7'h1e,
120     CTRL_RES_0 = 7'h2d,
121     CTRL_RES_1 = 7'h33,
122     CTRL_RES_2 = 7'h4b,
123     CTRL_RES_3 = 7'h55,
124     CTRL_RES_4 = 7'h66,
125     CTRL_RES_5 = 7'h78;
126
127 localparam [3:0]
128     O_SEQ_OS = 4'h0,
129     O_SIG_OS = 4'hf;
130
131 localparam [1:0]
132     SYNC_DATA = 2'b10,
133     SYNC_CTRL = 2'b01;
134
135 localparam [7:0]
136     BLOCK_TYPE_CTRL = 8'h1e, // C7 C6 C5 C4 C3 C2 C1 C0 BT
137     BLOCK_TYPE_OS_4 = 8'h2d, // D7 D6 D5 D4 C3 C2 C1 C0 BT
138     BLOCK_TYPE_START_4 = 8'h33, // D7 D6 D5 C3 C2 C1 C0 BT
139     BLOCK_TYPE_OS_START = 8'h66, // D7 D6 D5 00 D3 D2 D1 BT
140     BLOCK_TYPE_OS_04 = 8'h55, // D7 D6 D5 D4 00 D3 D2 D1 BT
141     BLOCK_TYPE_START_0 = 8'h78, // D7 D6 D5 D4 D3 D2 D1 BT
142     BLOCK_TYPE_OS_0 = 8'h4b, // C7 C6 C5 C4 00 D3 D2 D1 BT
143     BLOCK_TYPE_TERM_0 = 8'h87, // C7 C6 C5 C4 C3 C2 C1 BT
144     BLOCK_TYPE_TERM_1 = 8'h99, // C7 C6 C5 C4 C3 C2 00 BT
145     BLOCK_TYPE_TERM_2 = 8'haa, // C7 C6 C5 C4 C3 D1 D0 BT
146     BLOCK_TYPE_TERM_3 = 8'hb4, // C7 C6 C5 C4 D2 D1 D0 BT
147     BLOCK_TYPE_TERM_4 = 8'hcc, // C7 C6 C5 D3 D2 D1 D0 BT

```



```

148 BLOCK_TYPE_TERM_5 = 8'h2, // C7 C6 D4 D3 D2 D1 D0 BT
149 BLOCK_TYPE_TERM_6 = 8'h1, // C7 D5 D4 D3 D2 D1 D0 BT
150 BLOCK_TYPE_TERM_7 = 8'hff, // D6 D5 D4 D3 D2 D1 D0 BT
151
152 localparam [3:0]
153 OUTPUT_TYPE_IDLE = 4'd0,
154 OUTPUT_TYPE_ERROR = 4'd1,
155 OUTPUT_TYPE_START_0 = 4'd2,
156 OUTPUT_TYPE_START_4 = 4'd3,
157 OUTPUT_TYPE_DATA = 4'd4,
158 OUTPUT_TYPE_TERM_0 = 4'd8,
159 OUTPUT_TYPE_TERM_1 = 4'd9,
160 OUTPUT_TYPE_TERM_2 = 4'd10,
161 OUTPUT_TYPE_TERM_3 = 4'd11,
162 OUTPUT_TYPE_TERM_4 = 4'd12,
163 OUTPUT_TYPE_TERM_5 = 4'd13,
164 OUTPUT_TYPE_TERM_6 = 4'd14,
165 OUTPUT_TYPE_TERM_7 = 4'd15;
166
167 localparam [2:0]
168 STATE_IDLE = 3'd0,
169 STATE_PAYLOAD = 3'd1,
170 STATE_PAD = 3'd2,
171 STATE_FCS_1 = 3'd3,
172 STATE_FCS_2 = 3'd4,
173 STATE_ERR = 3'd5,
174 STATE_IFG = 3'd6;
175
176 reg [2:0] state_reg = STATE_IDLE, state_next;
177
178 // datapath control signals
179 reg reset_crc;
180 reg update_crc;
181
182 reg swap_lanes_reg = 1'b0, swap_lanes_next;
183 reg [31:0] swap_data = 32'd0;
184
185 reg delay_type_valid = 1'b0;
186 reg [3:0] delay_type = OUTPUT_TYPE_IDLE;
187
188 reg [DATA_WIDTH-1:0] s_axis_tdata_masked;
189
190 reg [DATA_WIDTH-1:0] s_tdata_reg = 0, s_tdata_next;
191 reg [EMPTY_WIDTH-1:0] s_empty_reg = 0, s_empty_next;
192
193 reg [DATA_WIDTH-1:0] fcs_output_data_0;
194 reg [DATA_WIDTH-1:0] fcs_output_data_1;
195 reg [3:0] fcs_output_type_0;
196 reg [3:0] fcs_output_type_1;
197
198 reg [7:0] ifg_offset;
199
200 reg frame_start_reg = 1'b0, frame_start_next;
201 reg frame_reg = 1'b0, frame_next;
202 reg frame_error_reg = 1'b0, frame_error_next;
203 reg [MIN_LEN_WIDTH-1:0] frame_min_count_reg = 0,
frame_min_count_next;
204
205 reg [7:0] ifg_count_reg = 8'd0, ifg_count_next;
206 reg [1:0] deficit_idle_count_reg = 2'd0, defi-
cit_idle_count_next;
207
208 reg s_axis_tready_reg = 1'b0, s_axis_tready_next;
209
210 reg [PTP_TS_WIDTH-1:0] m_axis_ptp_ts_reg = 0;
211 reg [PTP_TS_WIDTH-1:0] m_axis_ptp_ts_adj_reg = 0;
212 reg [PTP_TAG_WIDTH-1:0] m_axis_ptp_ts_tag_reg = 0;
213 reg m_axis_ptp_ts_valid_reg = 1'b0;
214 reg m_axis_ptp_ts_valid_int_reg = 1'b0;
215 reg m_axis_ptp_ts_borrow_reg = 1'b0;
216
217 reg [31:0] crc_state_reg[7:0];
218 wire [31:0] crc_state_next[7:0];
219
220 reg [DATA_WIDTH-1:0] encoded_tx_data_reg = {{8{CTRL_IDLE}},
BLOCK_TYPE_CTRL};
221 reg [HDR_WIDTH-1:0] encoded_tx_hdr_reg = SYNC_CTRL;
222
223 reg [DATA_WIDTH-1:0] output_data_reg = {DATA_WIDTH{1'b0}},
output_data_next;
224 reg [3:0] output_type_reg = OUTPUT_TYPE_IDLE, out-
put_type_next;
225
226 reg [1:0] start_packet_reg = 2'b00;
227 reg error_underflow_reg = 1'b0, error_underflow_next;
228
229 reg [4+16-1:0] last_ts_reg = 0;
230 reg [4+16-1:0] ts_inc_reg = 0;
231
232 assign s_axis_tready = s_axis_tready_reg;
233
234 assign encoded_tx_data = encoded_tx_data_reg;
235 assign encoded_tx_hdr = encoded_tx_hdr_reg;
236
237 assign m_axis_ptp_ts = PTP_TS_ENABLE ? (!PTP_TS_FMT_TOD ||
m_axis_ptp_ts_borrow_reg) ? m_axis_ptp_ts_reg : m_ax-
is_ptp_ts_adj_reg : 0;
238 assign m_axis_ptp_ts_tag = PTP_TAG_ENABLE ? m_ax-
is_ptp_ts_tag_reg : 0;
239 assign m_axis_ptp_ts_valid = PTP_TS_ENABLE || PTP_TAG_ENABLE ?

```

```

m_axis_ptp_ts_valid_reg : 1'b0;
240
241 assign start_packet = start_packet_reg;
242 assign error_underflow = error_underflow_reg;
243
244 generate
245     genvar n;
246
247     for (n = 0; n < 8; n = n + 1) begin : crc
248         lfsr #(
249             .LFSR_WIDTH(32),
250             .LFSR_POLY(32'h4c11db7),
251             .LFSR_CONFIG("GALOIS"),
252             .LFSR_FEED_FORWARD(0),
253             .REVERSE(1),
254             .DATA_WIDTH(8*(n+1)),
255             .STYLE("AUTO")
256         )
257         eth_crc (
258             .data_in(s_tdata_reg[0 +: 8*(n+1)]),
259             .state_in(crc_state_reg[7]),
260             .data_out(),
261             .state_out(crc_state_next[n])
262         );
263     end
264
265 endgenerate
266
267 function [2:0] keep2empty;
268     input [7:0] k;
269     casez (k)
270         8'bzzzzzz0: keep2empty = 3'd7;
271         8'bzzzzzz01: keep2empty = 3'd7;
272         8'bzzzzzz011: keep2empty = 3'd6;
273         8'bzzzzzz0111: keep2empty = 3'd5;
274         8'bzzzz01111: keep2empty = 3'd4;
275         8'bzz011111: keep2empty = 3'd3;
276         8'bz0111111: keep2empty = 3'd2;
277         8'b01111111: keep2empty = 3'd1;
278         8'b11111111: keep2empty = 3'd0;
279     endcase
280 endfunction
281
282 // Mask input data
283 integer j;
284
285 always @* begin
286     for (j = 0; j < 8; j = j + 1) begin
287         s_axis_tdata_masked[j*8 +: 8] = s_axis_tkeep[j] ?
s_axis_tdata[j*8 +: 8] : 8'd0;
288     end
289 end
290
291 // FCS cycle calculation
292 always @* begin
293     casez (s_empty_reg)
294         3'd7: begin
295             fcs_output_data_0 = {24'd0, ~crc_state_next[0]
[31:0], s_tdata_reg[7:0]};
296             fcs_output_data_1 = 64'd0;
297             fcs_output_type_0 = OUTPUT_TYPE_TERM_5;
298             fcs_output_type_1 = OUTPUT_TYPE_IDLE;
299             ifg_offset = 8'd3;
300         end
301         3'd6: begin
302             fcs_output_data_0 = {16'd0, ~crc_state_next[1]
[31:0], s_tdata_reg[15:0]};
303             fcs_output_data_1 = 64'd0;
304             fcs_output_type_0 = OUTPUT_TYPE_TERM_6;
305             fcs_output_type_1 = OUTPUT_TYPE_IDLE;
306             ifg_offset = 8'd2;
307         end
308         3'd5: begin
309             fcs_output_data_0 = {8'd0, ~crc_state_next[2]
[31:0], s_tdata_reg[23:0]};
310             fcs_output_data_1 = 64'd0;
311             fcs_output_type_0 = OUTPUT_TYPE_TERM_7;
312             fcs_output_type_1 = OUTPUT_TYPE_IDLE;
313             ifg_offset = 8'd1;
314         end
315         3'd4: begin
316             fcs_output_data_0 = {~crc_state_next[3][31:0],
s_tdata_reg[31:0]};
317             fcs_output_data_1 = 64'd0;
318             fcs_output_type_0 = OUTPUT_TYPE_DATA;
319             fcs_output_type_1 = OUTPUT_TYPE_TERM_0;
320             ifg_offset = 8'd8;
321         end
322         3'd3: begin
323             fcs_output_data_0 = {~crc_state_next[4][23:0],
s_tdata_reg[39:0]};
324             fcs_output_data_1 = {56'd0, ~crc_state_reg[4]
[31:24]};
325             fcs_output_type_0 = OUTPUT_TYPE_DATA;
326             fcs_output_type_1 = OUTPUT_TYPE_TERM_1;
327             ifg_offset = 8'd7;
328         end
329         3'd2: begin
330             fcs_output_data_0 = {~crc_state_next[5][15:0],
s_tdata_reg[47:0]};
331             fcs_output_data_1 = {48'd0, ~crc_state_reg[5]

```

```

[31:16]];
332         fcs_output_type_0 = OUTPUT_TYPE_DATA;
333         fcs_output_type_1 = OUTPUT_TYPE_TERM_2;
334         ifg_offset = 8'd6;
335     end
336     3'd1: begin
337         fcs_output_data_0 = {~crc_state_next[6][7:0], s_t-
data_reg[55:0]];
338         fcs_output_data_1 = {40'd0, ~crc_state_reg[6]
[31:8]];
339         fcs_output_type_0 = OUTPUT_TYPE_DATA;
340         fcs_output_type_1 = OUTPUT_TYPE_TERM_3;
341         ifg_offset = 8'd5;
342     end
343     3'd0: begin
344         fcs_output_data_0 = s_tdata_reg;
345         fcs_output_data_1 = {32'd0, ~crc_state_reg[7]
[31:0]];
346         fcs_output_type_0 = OUTPUT_TYPE_DATA;
347         fcs_output_type_1 = OUTPUT_TYPE_TERM_4;
348         ifg_offset = 8'd4;
349     end
350 endcase
351 end
352
353 always @* begin
354     state_next = STATE_IDLE;
355
356     reset_crc = 1'b0;
357     update_crc = 1'b0;
358
359     swap_lanes_next = swap_lanes_reg;
360
361     frame_start_next = 1'b0;
362     frame_next = frame_reg;
363     frame_error_next = frame_error_reg;
364     frame_min_count_next = frame_min_count_reg;
365
366     ifg_count_next = ifg_count_reg;
367     deficit_idle_count_next = deficit_idle_count_reg;
368
369     s_axis_tready_next = 1'b0;
370
371     s_tdata_next = s_tdata_reg;
372     s_empty_next = s_empty_reg;
373
374     output_data_next = s_tdata_reg;
375     output_type_next = OUTPUT_TYPE_IDLE;
376
377     error_underflow_next = 1'b0;
378
379     if (s_axis_tvalid && s_axis_tready) begin
380         frame_next = !s_axis_tlast;
381     end
382
383     case (state_reg)
384     STATE_IDLE: begin
385         // idle state - wait for data
386         frame_error_next = 1'b0;
387         frame_min_count_next = MIN_FRAME_LENGTH-4-
KEEP_WIDTH;
388         reset_crc = 1'b1;
389         s_axis_tready_next = 1'b1;
390
391         output_data_next = s_tdata_reg;
392         output_type_next = OUTPUT_TYPE_IDLE;
393
394         s_tdata_next = s_axis_tdata_masked;
395         s_empty_next = keep2empty(s_axis_tkeep);
396
397         if (s_axis_tvalid && cfg_tx_enable) begin
398             // Preamble and SFD
399             output_data_next = {ETH_SFD, {7{ETH_PRE}}};
400             output_type_next = OUTPUT_TYPE_START_0;
401             frame_start_next = 1'b1;
402             s_axis_tready_next = 1'b1;
403             state_next = STATE_PAYLOAD;
404         end else begin
405             swap_lanes_next = 1'b0;
406             ifg_count_next = 8'd0;
407             deficit_idle_count_next = 2'd0;
408             state_next = STATE_IDLE;
409         end
410     end
411     STATE_PAYLOAD: begin
412         // transfer payload
413         update_crc = 1'b1;
414         s_axis_tready_next = 1'b1;
415
416         if (frame_min_count_reg > KEEP_WIDTH) begin
417             frame_min_count_next = frame_min_count_reg -
KEEP_WIDTH;
418         end else begin
419             frame_min_count_next = 0;
420         end
421
422         output_data_next = s_tdata_reg;
423         output_type_next = OUTPUT_TYPE_DATA;
424
425         s_tdata_next = s_axis_tdata_masked;
426         s_empty_next = keep2empty(s_axis_tkeep);
427
428         if (!s_axis_tvalid || s_axis_tlast) begin
429             s_axis_tready_next = frame_next; // drop frame
430             frame_error_next = !s_axis_tvalid || s_ax-
is_tuser[0];
431             error_underflow_next = !s_axis_tvalid;
432
433             if (ENABLE_PADDING && frame_min_count_reg) be-
gin
434                 if (frame_min_count_reg > KEEP_WIDTH) be-
gin
435                     s_empty_next = 0;
436                     state_next = STATE_PAD;
437                 end else begin
438                     if (keep2empty(s_axis_tkeep) >
KEEP_WIDTH-frame_min_count_reg) begin
439                         s_empty_next = KEEP_WIDTH-
frame_min_count_reg;
440                     end
441                     if (frame_error_next) begin
442                         state_next = STATE_ERR;
443                     end else begin
444                         state_next = STATE_FCS_1;
445                     end
446                 end
447             end else begin
448                 if (frame_error_next) begin
449                     state_next = STATE_ERR;
450                 end else begin
451                     state_next = STATE_FCS_1;
452                 end
453             end
454             end else begin
455                 state_next = STATE_PAYLOAD;
456             end
457         end
458     STATE_PAD: begin
459         // pad frame to MIN_FRAME_LENGTH
460         s_axis_tready_next = frame_next; // drop frame
461
462         output_data_next = s_tdata_reg;
463         output_type_next = OUTPUT_TYPE_DATA;
464
465         s_tdata_next = 64'd0;
466         s_empty_next = 0;
467
468         update_crc = 1'b1;
469
470         if (frame_min_count_reg > KEEP_WIDTH) begin
471             frame_min_count_next = frame_min_count_reg -
KEEP_WIDTH;
472         end else begin
473             state_next = STATE_PAD;
474         end
475         if (frame_min_count_next = 0;
476         s_empty_next = KEEP_WIDTH-frame_min_count_reg;
477         if (frame_error_reg) begin
478             state_next = STATE_ERR;
479         end else begin
480             state_next = STATE_FCS_1;
481         end
482     end
483     STATE_FCS_1: begin
484         // last cycle
485         s_axis_tready_next = frame_next; // drop frame
486
487         output_data_next = fcs_output_data_0;
488         output_type_next = fcs_output_type_0;
489
490         update_crc = 1'b1;
491
492         ifg_count_next = (cfg_ifg > 8'd12 ? cfg_ifg :
8'd12) - ifg_offset + (swap_lanes_reg ? 8'd4 : 8'd0) + defi-
cit_idle_count_reg;
493         if (s_empty_reg <= 4) begin
494             state_next = STATE_FCS_2;
495         end else begin
496             state_next = STATE_IFG;
497         end
498     end
499     STATE_FCS_2: begin
500         // last cycle
501         s_axis_tready_next = frame_next; // drop frame
502
503         output_data_next = fcs_output_data_1;
504         output_type_next = fcs_output_type_1;
505
506         reset_crc = 1'b1;
507
508         if (ENABLE_DIC) begin
509             if (ifg_count_next > 8'd7) begin
510                 state_next = STATE_IFG;
511             end else begin
512                 if (ifg_count_next >= 8'd4) begin
513                     deficit_idle_count_next =
ifg_count_next - 8'd4;
514                     swap_lanes_next = 1'b1;
515                 end else begin
516                     deficit_idle_count_next =
ifg_count_next;
517                     ifg_count_next = 8'd0;

```

```

518         swap_lanes_next = 1'b0;
519     end
520     s_axis_tready_next = 1'b1;
521     state_next = STATE_IDLE;
522 end
523 end else begin
524     if (ifg_count_next > 8'd4) begin
525         state_next = STATE_IFG;
526     end else begin
527         s_axis_tready_next = 1'b1;
528         swap_lanes_next = ifg_count_next != 0;
529         state_next = STATE_IDLE;
530     end
531 end
532 end
533 STATE_ERR: begin
534     // terminate packet with error
535     s_axis_tready_next = frame_next; // drop frame
536
537     output_data_next = s_tdata_reg;
538     output_type_next = OUTPUT_TYPE_ERROR;
539
540     ifg_count_next = 8'd12;
541
542     state_next = STATE_IFG;
543 end
544 STATE_IFG: begin
545     // send IFG
546     s_axis_tready_next = frame_next; // drop frame
547
548     output_data_next = s_tdata_reg;
549     output_type_next = OUTPUT_TYPE_IDLE;
550
551     if (ifg_count_reg > 8'd8) begin
552         ifg_count_next = ifg_count_reg - 8'd8;
553     end else begin
554         ifg_count_next = 8'd0;
555     end
556
557     reset_crc = 1'b1;
558
559     if (ENABLE_DIC) begin
560         if (ifg_count_next > 8'd7 || frame_reg) begin
561             state_next = STATE_IFG;
562         end else begin
563             if (ifg_count_next >= 8'd4) begin
564                 deficit_idle_count_next =
565                     ifg_count_next - 8'd4;
566                 swap_lanes_next = 1'b1;
567             end else begin
568                 deficit_idle_count_next =
569                     ifg_count_next;
570                 swap_lanes_next = 1'b0;
571             end
572             s_axis_tready_next = 1'b1;
573             state_next = STATE_IDLE;
574         end
575     end else begin
576         if (ifg_count_next > 8'd4 || frame_reg) begin
577             state_next = STATE_IFG;
578         end else begin
579             s_axis_tready_next = 1'b1;
580             swap_lanes_next = ifg_count_next != 0;
581             state_next = STATE_IDLE;
582         end
583     end
584 endcase
585 end
586
587 always @(posedge clk) begin
588     state_reg <= state_next;
589
590     swap_lanes_reg <= swap_lanes_next;
591
592     frame_start_reg <= frame_start_next;
593     frame_reg <= frame_next;
594     frame_error_reg <= frame_error_next;
595     frame_min_count_reg <= frame_min_count_next;
596
597     ifg_count_reg <= ifg_count_next;
598     deficit_idle_count_reg <= deficit_idle_count_next;
599
600     s_tdata_reg <= s_tdata_next;
601     s_empty_reg <= s_empty_next;
602
603     s_axis_tready_reg <= s_axis_tready_next;
604
605     m_axis_ptp_ts_valid_reg <= 1'b0;
606     m_axis_ptp_ts_valid_int_reg <= 1'b0;
607
608     start_packet_reg <= 2'b00;
609     error_underflow_reg <= error_underflow_next;
610
611     delay_type_valid <= 1'b0;
612     delay_type <= output_type_next ^ 4'd4;
613
614     swap_data <= output_data_next[63:32];
615
616     if (swap_lanes_reg) begin
617         output_data_reg <= {output_data_next[31:0],
618                               swap_data};
619         if (delay_type_valid) begin
620             output_type_reg <= delay_type;
621         end else if (output_type_next == OUTPUT_TYPE_START_0)
622             begin
623             output_type_reg <= OUTPUT_TYPE_START_4;
624             end else if (output_type_next[3]) begin
625             // OUTPUT_TYPE_TERM_*
626             if (output_type_next[2]) begin
627                 delay_type_valid <= 1'b1;
628                 output_type_reg <= OUTPUT_TYPE_DATA;
629             end else begin
630                 output_type_reg <= output_type_next ^ 4'd4;
631             end
632         end else begin
633             output_type_reg <= output_type_next;
634         end
635     end else begin
636         output_data_reg <= output_data_next;
637         output_type_reg <= output_type_next;
638     end
639
640     if (PTP_TS_ENABLE && PTP_TS_FMT_TOD) begin
641         m_axis_ptp_ts_valid_reg <= m_ax-
642             is_ptp_ts_valid_int_reg;
643         m_axis_ptp_ts_adj_reg[15:0] <= m_ax-
644             is_ptp_ts_reg[15:0];
645         {m_axis_ptp_ts_borrow_reg, m_ax-
646             is_ptp_ts_adj_reg[45:16]} <= $signed({1'b0, m_ax-
647             is_ptp_ts_reg[45:16]}) - $signed(31'd1000000000);
648         m_axis_ptp_ts_adj_reg[47:46] <= 0;
649         m_axis_ptp_ts_adj_reg[95:48] <= m_ax-
650             is_ptp_ts_reg[95:48] + 1;
651         end
652         if (frame_start_reg) begin
653             if (swap_lanes_reg) begin
654                 if (PTP_TS_ENABLE) begin
655                     if (PTP_TS_FMT_TOD) begin
656                         m_axis_ptp_ts_reg[45:0] <= ptp_ts[45:0] +
657                             (ts_inc_reg >> 1);
658                         m_axis_ptp_ts_reg[95:48] <= ptp_ts[95:48];
659                     end else begin
660                         m_axis_ptp_ts_reg <= ptp_ts + (ts_inc_reg
661                             >> 1);
662                     end
663                 end
664                 start_packet_reg <= 2'b10;
665             end else begin
666                 if (PTP_TS_ENABLE) begin
667                     m_axis_ptp_ts_reg <= ptp_ts;
668                 end
669                 start_packet_reg <= 2'b01;
670             end
671         end
672         if (PTP_TS_ENABLE) begin
673             if (PTP_TS_CTRL_IN_TUSER) begin
674                 m_axis_ptp_ts_tag_reg <= s_axis_tuser >> 2;
675                 if (PTP_TS_FMT_TOD) begin
676                     m_axis_ptp_ts_valid_int_reg <= s_ax-
677                         is_tuser[1];
678                 end else begin
679                     m_axis_ptp_ts_valid_reg <= s_ax-
680                         is_tuser[1];
681                 end
682             end
683             end else begin
684                 end
685             end else begin
686                 m_axis_ptp_ts_tag_reg <= s_axis_tuser >> 1;
687                 if (PTP_TS_FMT_TOD) begin
688                     m_axis_ptp_ts_valid_int_reg <= 1'b1;
689                 end else begin
690                     m_axis_ptp_ts_valid_reg <= 1'b1;
691                 end
692             end
693         end
694     end
695 end
696
697 case (output_type_reg)
698     OUTPUT_TYPE_IDLE: begin
699         encoded_tx_data_reg <= {{8{CTRL_IDLE}}},
700         BLOCK_TYPE_CTRL;
701         encoded_tx_hdr_reg <= SYNC_CTRL;
702     end
703     OUTPUT_TYPE_ERROR: begin
704         encoded_tx_data_reg <= {{8{CTRL_ERROR}}},
705         BLOCK_TYPE_CTRL;
706         encoded_tx_hdr_reg <= SYNC_CTRL;
707     end
708     OUTPUT_TYPE_START_0: begin
709         encoded_tx_data_reg <= {output_data_reg[63:8],
710             BLOCK_TYPE_START_0};
711         encoded_tx_hdr_reg <= SYNC_CTRL;
712     end
713     OUTPUT_TYPE_START_4: begin
714         encoded_tx_data_reg <= {output_data_reg[63:40],
715             4'd0, {4{CTRL_IDLE}}}, BLOCK_TYPE_START_4;
716         encoded_tx_hdr_reg <= SYNC_CTRL;
717     end
718     OUTPUT_TYPE_DATA: begin
719         encoded_tx_data_reg <= output_data_reg;
720         encoded_tx_hdr_reg <= SYNC_DATA;
721     end
722 end

```

```

703     OUTPUT_TYPE_TERM_0: begin
704         encoded_tx_data_reg <= {{7{CTRL_IDLE}}, 7'd0,
BLOCK_TYPE_TERM_0};
705     encoded_tx_hdr_reg <= SYNC_CTRL;
706     end
707     OUTPUT_TYPE_TERM_1: begin
708         encoded_tx_data_reg <= {{6{CTRL_IDLE}}, 6'd0, out-
put_data_reg[7:0], BLOCK_TYPE_TERM_1};
709     encoded_tx_hdr_reg <= SYNC_CTRL;
710     end
711     OUTPUT_TYPE_TERM_2: begin
712         encoded_tx_data_reg <= {{5{CTRL_IDLE}}, 5'd0, out-
put_data_reg[15:0], BLOCK_TYPE_TERM_2};
713     encoded_tx_hdr_reg <= SYNC_CTRL;
714     end
715     OUTPUT_TYPE_TERM_3: begin
716         encoded_tx_data_reg <= {{4{CTRL_IDLE}}, 4'd0, out-
put_data_reg[23:0], BLOCK_TYPE_TERM_3};
717     encoded_tx_hdr_reg <= SYNC_CTRL;
718     end
719     OUTPUT_TYPE_TERM_4: begin
720         encoded_tx_data_reg <= {{3{CTRL_IDLE}}, 3'd0, out-
put_data_reg[31:0], BLOCK_TYPE_TERM_4};
721     encoded_tx_hdr_reg <= SYNC_CTRL;
722     end
723     OUTPUT_TYPE_TERM_5: begin
724         encoded_tx_data_reg <= {{2{CTRL_IDLE}}, 2'd0, out-
put_data_reg[39:0], BLOCK_TYPE_TERM_5};
725     encoded_tx_hdr_reg <= SYNC_CTRL;
726     end
727     OUTPUT_TYPE_TERM_6: begin
728         encoded_tx_data_reg <= {{1{CTRL_IDLE}}, 1'd0, out-
put_data_reg[47:0], BLOCK_TYPE_TERM_6};
729     encoded_tx_hdr_reg <= SYNC_CTRL;
730     end
731     OUTPUT_TYPE_TERM_7: begin
732         encoded_tx_data_reg <= {output_data_reg[55:0],
BLOCK_TYPE_TERM_7};
733     encoded_tx_hdr_reg <= SYNC_CTRL;
734     end
735     default: begin
736         encoded_tx_data_reg <= {{8{CTRL_ERROR}},
BLOCK_TYPE_CTRL};
737     encoded_tx_hdr_reg <= SYNC_CTRL;
738     end
739 endcase
740
741     crc_state_reg[0] <= crc_state_next[0];
742     crc_state_reg[1] <= crc_state_next[1];
743     crc_state_reg[2] <= crc_state_next[2];
744     crc_state_reg[3] <= crc_state_next[3];
745     crc_state_reg[4] <= crc_state_next[4];
746     crc_state_reg[5] <= crc_state_next[5];
747     crc_state_reg[6] <= crc_state_next[6];
748
749     if (update_crc) begin
750         crc_state_reg[7] <= crc_state_next[7];
751     end
752
753     if (reset_crc) begin
754         crc_state_reg[7] <= 32'hFFFFFFFF;
755     end
756
757     last_ts_reg <= ptp_ts;
758     ts_inc_reg <= ptp_ts - last_ts_reg;
759
760     if (rst) begin
761         state_reg <= STATE_IDLE;
762
763         frame_start_reg <= 1'b0;
764         frame_reg <= 1'b0;
765
766         swap_lanes_reg <= 1'b0;
767
768         ifg_count_reg <= 8'd0;
769         deficit_idle_count_reg <= 2'd0;
770
771         s_axis_tready_reg <= 1'b0;
772
773         m_axis_ptp_ts_valid_reg <= 1'b0;
774         m_axis_ptp_ts_valid_int_reg <= 1'b0;
775
776         encoded_tx_data_reg <= {{8{CTRL_IDLE}},
BLOCK_TYPE_CTRL};
777         encoded_tx_hdr_reg <= SYNC_CTRL;
778
779         output_data_reg <= {DATA_WIDTH{1'b0}};
780         output_type_reg <= OUTPUT_TYPE_IDLE;
781
782         start_packet_reg <= 2'b00;
783         error_underflow_reg <= 1'b0;
784
785         delay_type_valid <= 1'b0;
786         delay_type <= OUTPUT_TYPE_IDLE;
787     end
788 end
789
790 endmodule
791
792 `resetall

```

7. alexforenrich_verilog-ethernet/rtl/axis_eth_fcs.v (141 lines)

```

1 /*
2
3 Copyright (c) 2015-2018 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * AXI4-Stream Ethernet FCS Generator
33 */
34 module axis_eth_fcs #
35 (
36     // Width of AXI stream interfaces in bits
37     parameter DATA_WIDTH = 8,
38     // Propagate tkeep signal
39     // If disabled, tkeep assumed to be 1'b1
40     parameter KEEP_ENABLE = (DATA_WIDTH>8),
41     // tkeep signal width (words per cycle)
42     parameter KEEP_WIDTH = (DATA_WIDTH/8)
43 )
44 (
45     input wire          clk,
46     input wire          rst,
47
48     /*
49     * AXI input
50     */
51     input wire [DATA_WIDTH-1:0] s_axis_tdata,
52     input wire [KEEP_WIDTH-1:0] s_axis_tkeep,
53     input wire                  s_axis_tvalid,
54     output wire                 s_axis_tready,
55     input wire                  s_axis_tlast,
56     input wire                  s_axis_tuser,
57
58     /*
59     * FCS output
60     */
61     output wire [31:0]          output_fcs,
62     output wire                output_fcs_valid
63 );
64
65 // bus width assertions
66 initial begin
67     if (KEEP_WIDTH * 8 != DATA_WIDTH) begin
68         $error("Error: AXI stream interface requires byte (8-
bit) granularity (instance %m)");
69     $finish;
70     end
71 end
72
73 reg [31:0] crc_state = 32'hFFFFFFFF;
74 reg [31:0] fcs_reg = 32'h00000000;
75 reg fcs_valid_reg = 1'b0;
76
77 wire [31:0] crc_next[KEEP_WIDTH-1:0];
78
79 assign s_axis_tready = 1;
80 assign output_fcs = fcs_reg;
81 assign output_fcs_valid = fcs_valid_reg;
82
83 generate

```

```

84
85   genvar n;
86
87   for (n = 0; n < KEEP_WIDTH; n = n + 1) begin : crc
88
89       lfsr #(
90           .LFSR_WIDTH(32),
91           .LFSR_POLY(32'h4c11db7),
92           .LFSR_CONFIG("GALOIS"),
93           .LFSR_FEED_FORWARD(0),
94           .REVERSE(1),
95           .DATA_WIDTH(DATA_WIDTH/KEEP_WIDTH*(n+1)),
96           .STYLE("AUTO")
97       )
98       eth_crc_inst (
99           .data_in(s_axis_tdata[DATA_WIDTH/
100             KEEP_WIDTH*(n+1)-1:0]),
101           .state_in(crc_state),
102           .data_out(),
103           .state_out(crc_next[n])
104       );
105   end
106
107 endgenerate
108
109 integer i;
110
111 always @(posedge clk) begin
112     fcs_valid_reg <= 1'b0;
113
114     if (s_axis_tvalid) begin
115         crc_state <= crc_next[KEEP_WIDTH-1];
116
117         if (s_axis_tlast) begin
118             crc_state <= 32'hFFFFFFF;
119             if (KEEP_ENABLE) begin
120                 fcs_reg <= ~crc_next[0];
121                 for (i = 0; i < KEEP_WIDTH; i = i + 1) begin
122                     if (s_axis_tkeep[i]) begin
123                         fcs_reg <= ~crc_next[i];
124                     end
125                 end
126             end else begin
127                 fcs_reg <= ~crc_next[KEEP_WIDTH-1];
128             end
129             fcs_valid_reg <= 1'b1;
130         end
131     end
132
133     if (rst) begin
134         crc_state <= 32'hFFFFFFF;
135         fcs_valid_reg <= 1'b0;
136     end
137 end
138
139 endmodule
140
141 `resetall

```

8. alexforenrich_verilog-ethernet/rtl/axis_eth_fcs_check.v (345 lines)

```

1  /*
2
3  Copyright (c) 2015-2018 Alex Forenrich
4
5  Permission is hereby granted, free of charge, to any person
6  obtaining a copy
7  of this software and associated documentation files (the
8  "Software"), to deal
9  in the Software without restriction, including without limita-
10 tion the rights
11 to use, copy, modify, merge, publish, distribute, sublicense,
12 and/or sell
13 copies of the Software, and to permit persons to whom the
14 Software is
15 furnished to do so, subject to the following conditions:
16
17 The above copyright notice and this permission notice shall be
18 included in
19 all copies or substantial portions of the Software.
20
21 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
22 KIND, EXPRESS OR
23 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
24 CHANTABILITY
25 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
26 EVENT SHALL THE
27 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
28 OR OTHER
29 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
30 WISE, ARISING FROM,
31 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
32 DEALINGS IN
33 THE SOFTWARE.
34
35 */

```

```

25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32  * AXI4-Stream Ethernet FCS checker
33  */
34 module axis_eth_fcs_check
35 (
36     input wire      clk,
37     input wire      rst,
38
39     /*
40      * AXI input
41      */
42     input wire [7:0] s_axis_tdata,
43     input wire      s_axis_tvalid,
44     output wire     s_axis_tready,
45     input wire      s_axis_tlast,
46     input wire      s_axis_tuser,
47
48     /*
49      * AXI output
50      */
51     output wire [7:0] m_axis_tdata,
52     output wire      m_axis_tvalid,
53     input wire      m_axis_tready,
54     output wire      m_axis_tlast,
55     output wire      m_axis_tuser,
56
57     /*
58      * Status
59      */
60     output wire      busy,
61     output wire      error_bad_fcs
62 );
63
64 localparam [1:0]
65     STATE_IDLE = 2'd0,
66     STATE_PAYLOAD = 2'd1;
67
68 reg [1:0] state_reg = STATE_IDLE, state_next;
69
70 // datapath control signals
71 reg reset_crc;
72 reg update_crc;
73 reg shift_in;
74 reg shift_reset;
75
76 reg [7:0] s_axis_tdata_d0 = 8'd0;
77 reg [7:0] s_axis_tdata_d1 = 8'd0;
78 reg [7:0] s_axis_tdata_d2 = 8'd0;
79 reg [7:0] s_axis_tdata_d3 = 8'd0;
80
81 reg s_axis_tvalid_d0 = 1'b0;
82 reg s_axis_tvalid_d1 = 1'b0;
83 reg s_axis_tvalid_d2 = 1'b0;
84 reg s_axis_tvalid_d3 = 1'b0;
85
86 reg busy_reg = 1'b0;
87 reg error_bad_fcs_reg = 1'b0, error_bad_fcs_next;
88
89 reg s_axis_tready_reg = 1'b0, s_axis_tready_next;
90
91 reg [31:0] crc_state = 32'hFFFFFFF;
92 wire [31:0] crc_next;
93
94 // internal datapath
95 reg [7:0] m_axis_tdata_int;
96 reg      m_axis_tvalid_int;
97 reg      m_axis_tready_int_reg = 1'b0;
98 reg      m_axis_tlast_int;
99 reg      m_axis_tuser_int;
100 wire     m_axis_tready_int_early;
101
102 assign s_axis_tready = s_axis_tready_reg;
103
104 assign busy = busy_reg;
105 assign error_bad_fcs = error_bad_fcs_reg;
106
107 lfsr #(
108     .LFSR_WIDTH(32),
109     .LFSR_POLY(32'h4c11db7),
110     .LFSR_CONFIG("GALOIS"),
111     .LFSR_FEED_FORWARD(0),
112     .REVERSE(1),
113     .DATA_WIDTH(8),
114     .STYLE("AUTO")
115 )
116 eth_crc_8 (
117     .data_in(s_axis_tdata_d3),
118     .state_in(crc_state),
119     .data_out(),
120     .state_out(crc_next)
121 );
122
123 always @* begin
124     state_next = STATE_IDLE;
125

```



```

126 reset_crc = 1'b0;
127 update_crc = 1'b0;
128 shift_in = 1'b0;
129 shift_reset = 1'b0;
130
131 s_axis_tready_next = 1'b0;
132
133 m_axis_tdata_int = 8'd0;
134 m_axis_tvalid_int = 1'b0;
135 m_axis_tlast_int = 1'b0;
136 m_axis_tuser_int = 1'b0;
137
138 error_bad_fcs_next = 1'b0;
139
140 case (state_reg)
141   STATE_IDLE: begin
142     // idle state - wait for data
143     s_axis_tready_next = m_axis_tready_int_early;
144     reset_crc = 1'b1;
145
146     m_axis_tdata_int = s_axis_tdata_d3;
147     m_axis_tvalid_int = s_axis_tvalid_d3 && s_ax-
is_tvalid;
148     m_axis_tlast_int = 1'b0;
149     m_axis_tuser_int = 1'b0;
150
151     if (s_axis_tready && s_axis_tvalid) begin
152       shift_in = 1'b1;
153
154       if (s_axis_tvalid_d3) begin
155         reset_crc = 1'b0;
156         update_crc = 1'b1;
157         if (s_axis_tlast) begin
158           shift_reset = 1'b1;
159           reset_crc = 1'b1;
160           m_axis_tlast_int = 1'b1;
161           m_axis_tuser_int = s_axis_tuser;
162           if ({s_axis_tdata, s_axis_tdata_d0,
s_axis_tdata_d1, s_axis_tdata_d2} != ~crc_next) begin
163             m_axis_tuser_int = 1'b1;
164             error_bad_fcs_next = 1'b1;
165           end
166           s_axis_tready_next = m_ax-
is_tready_int_early;
167           state_next = STATE_IDLE;
168         end else begin
169           state_next = STATE_PAYLOAD;
170         end
171       end else begin
172         state_next = STATE_IDLE;
173       end
174     end else begin
175       state_next = STATE_IDLE;
176     end
177   end
178   STATE_PAYLOAD: begin
179     // transfer payload
180     s_axis_tready_next = m_axis_tready_int_early;
181
182     m_axis_tdata_int = s_axis_tdata_d3;
183     m_axis_tvalid_int = s_axis_tvalid_d3 && s_ax-
is_tvalid;
184     m_axis_tlast_int = 1'b0;
185     m_axis_tuser_int = 1'b0;
186
187     if (s_axis_tready && s_axis_tvalid) begin
188       shift_in = 1'b1;
189       update_crc = 1'b1;
190       if (s_axis_tlast) begin
191         shift_reset = 1'b1;
192         reset_crc = 1'b1;
193         m_axis_tlast_int = 1'b1;
194         m_axis_tuser_int = s_axis_tuser;
195         if ({s_axis_tdata, s_axis_tdata_d0, s_ax-
is_tdata_d1, s_axis_tdata_d2} != ~crc_next) begin
196           m_axis_tuser_int = 1'b1;
197           error_bad_fcs_next = 1'b1;
198         end
199         s_axis_tready_next = m_ax-
is_tready_int_early;
200         state_next = STATE_IDLE;
201       end else begin
202         state_next = STATE_PAYLOAD;
203       end
204     end else begin
205       state_next = STATE_PAYLOAD;
206     end
207   end
208 endcase
209 end
210
211 always @(posedge clk) begin
212   if (rst) begin
213     state_reg <= STATE_IDLE;
214
215     s_axis_tready_reg <= 1'b0;
216
217     busy_reg <= 1'b0;
218     error_bad_fcs_reg <= 1'b0;
219
220     s_axis_tvalid_d0 <= 1'b0;
221
222     s_axis_tvalid_d1 <= 1'b0;
223     s_axis_tvalid_d2 <= 1'b0;
224     s_axis_tvalid_d3 <= 1'b0;
225
226     crc_state <= 32'hFFFFFFF;
227   end else begin
228     state_reg <= state_next;
229
230     s_axis_tready_reg <= s_axis_tready_next;
231
232     busy_reg <= state_next != STATE_IDLE;
233     error_bad_fcs_reg <= error_bad_fcs_next;
234
235     // datapath
236     if (reset_crc) begin
237       crc_state <= 32'hFFFFFFF;
238     end else if (update_crc) begin
239       crc_state <= crc_next;
240     end
241
242     if (shift_reset) begin
243       s_axis_tvalid_d0 <= 1'b0;
244       s_axis_tvalid_d1 <= 1'b0;
245       s_axis_tvalid_d2 <= 1'b0;
246       s_axis_tvalid_d3 <= 1'b0;
247     end else if (shift_in) begin
248       s_axis_tvalid_d0 <= s_axis_tvalid;
249       s_axis_tvalid_d1 <= s_axis_tvalid_d0;
250       s_axis_tvalid_d2 <= s_axis_tvalid_d1;
251       s_axis_tvalid_d3 <= s_axis_tvalid_d2;
252     end
253   end
254
255   if (shift_in) begin
256     s_axis_tdata_d0 <= s_axis_tdata;
257     s_axis_tdata_d1 <= s_axis_tdata_d0;
258     s_axis_tdata_d2 <= s_axis_tdata_d1;
259     s_axis_tdata_d3 <= s_axis_tdata_d2;
260   end
261
262   // output datapath logic
263   reg [7:0] m_axis_tdata_reg = 8'd0;
264   reg m_axis_tvalid_reg = 1'b0, m_axis_tvalid_next;
265   reg m_axis_tlast_reg = 1'b0;
266   reg m_axis_tuser_reg = 1'b0;
267
268   reg [7:0] temp_m_axis_tdata_reg = 8'd0;
269   reg temp_m_axis_tvalid_reg = 1'b0, temp_m_ax-
is_tvalid_next;
270   reg temp_m_axis_tlast_reg = 1'b0;
271   reg temp_m_axis_tuser_reg = 1'b0;
272
273   // datapath control
274   reg store_axis_int_to_output;
275   reg store_axis_int_to_temp;
276   reg store_axis_temp_to_output;
277
278   assign m_axis_tdata = m_axis_tdata_reg;
279   assign m_axis_tvalid = m_axis_tvalid_reg;
280   assign m_axis_tlast = m_axis_tlast_reg;
281   assign m_axis_tuser = m_axis_tuser_reg;
282
283   // enable ready input next cycle if output is ready or if both
284   // output registers are empty
285   assign m_axis_tready_int_early = m_axis_tready || (!temp_m_ax-
is_tvalid_reg && !m_axis_tvalid_reg);
286
287   always @* begin
288     // transfer sink ready state to source
289     m_axis_tvalid_next = m_axis_tvalid_reg;
290     temp_m_axis_tvalid_next = temp_m_axis_tvalid_reg;
291
292     store_axis_int_to_output = 1'b0;
293     store_axis_int_to_temp = 1'b0;
294     store_axis_temp_to_output = 1'b0;
295
296     if (m_axis_tready_int_reg) begin
297       // input is ready
298       if (m_axis_tready || !m_axis_tvalid_reg) begin
299         // output is ready or currently not valid, trans-
300         fer data to output
301         m_axis_tvalid_next = m_axis_tvalid_int;
302         store_axis_int_to_output = 1'b1;
303       end else begin
304         // output is not ready, store input in temp
305         temp_m_axis_tvalid_next = m_axis_tvalid_int;
306         store_axis_int_to_temp = 1'b1;
307       end
308     end else if (m_axis_tready) begin
309       // input is not ready, but output is ready
310       m_axis_tvalid_next = temp_m_axis_tvalid_reg;
311       temp_m_axis_tvalid_next = 1'b0;
312       store_axis_temp_to_output = 1'b1;
313     end
314   end
315
316   always @(posedge clk) begin
317     m_axis_tvalid_reg <= m_axis_tvalid_next;
318     m_axis_tready_int_reg <= m_axis_tready_int_early;

```

```

317 temp_m_axis_tvalid_reg <= temp_m_axis_tvalid_next;
318
319 // datapath
320 if (store_axis_int_to_output) begin
321   m_axis_tdata_reg <= m_axis_tdata_int;
322   m_axis_tlast_reg <= m_axis_tlast_int;
323   m_axis_tuser_reg <= m_axis_tuser_int;
324 end else if (store_axis_temp_to_output) begin
325   m_axis_tdata_reg <= temp_m_axis_tdata_reg;
326   m_axis_tlast_reg <= temp_m_axis_tlast_reg;
327   m_axis_tuser_reg <= temp_m_axis_tuser_reg;
328 end
329
330 if (store_axis_int_to_temp) begin
331   temp_m_axis_tdata_reg <= m_axis_tdata_int;
332   temp_m_axis_tlast_reg <= m_axis_tlast_int;
333   temp_m_axis_tuser_reg <= m_axis_tuser_int;
334 end
335
336 if (rst) begin
337   m_axis_tvalid_reg <= 1'b0;
338   m_axis_tready_int_reg <= 1'b0;
339   temp_m_axis_tvalid_reg <= 1'b0;
340 end
341 end
342 endmodule
343
344 `resetall
345

```

9. alexforenrich_verilog-ethernet/rtl/axis_eth_fcs_check_64.v

(482 lines)

```

1 /*
2
3 Copyright (c) 2015-2018 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
6 obtaining a copy
7 of this software and associated documentation files (the
8 "Software"), to deal
9 in the Software without restriction, including without limita-
10 tion the rights
11 to use, copy, modify, merge, publish, distribute, sublicense,
12 and/or sell
13 copies of the Software, and to permit persons to whom the
14 Software is
15 furnished to do so, subject to the following conditions:
16
17 The above copyright notice and this permission notice shall be
18 included in
19 all copies or substantial portions of the Software.
20
21 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
22 KIND, EXPRESS OR
23 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
24 CHANTABILITY
25 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
26 EVENT SHALL THE
27 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
28 OR OTHER
29 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
30 WISE, ARISING FROM,
31 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
32 DEALINGS IN
33 THE SOFTWARE.
34
35 */
36 // Language: Verilog 2001
37
38 `resetall
39 `timescale 1ns / 1ps
40 `default_nettype none
41
42 /*
43 * AXI4-Stream Ethernet FCS checker (64 bit datapath)
44 */
45 module axis_eth_fcs_check_64
46 (
47   input wire      clk,
48   input wire      rst,
49
50   /*
51    * AXI input
52    */
53   input wire [63:0] s_axis_tdata,
54   input wire [7:0]  s_axis_tkeep,
55   input wire        s_axis_tvalid,
56   output wire       s_axis_tready,
57   input wire        s_axis_tlast,
58   input wire        s_axis_tuser,
59
60   /*
61    * AXI output
62    */
63   output wire [63:0] m_axis_tdata,
64   output wire [7:0]  m_axis_tkeep,
65   output wire        m_axis_tvalid,

```

```

55   input wire      m_axis_tready,
56   output wire     m_axis_tlast,
57   output wire     m_axis_tuser,
58
59   /*
60    * Status
61    */
62   output wire     busy,
63   output wire     error_bad_fcs
64 );
65
66 localparam [1:0]
67   STATE_IDLE = 2'd0,
68   STATE_PAYLOAD = 2'd1,
69   STATE_LAST = 2'd2;
70
71 reg [1:0] state_reg = STATE_IDLE, state_next;
72
73 // datapath control signals
74 reg reset_crc;
75 reg update_crc;
76 reg shift_in;
77 reg shift_reset;
78
79 reg [7:0] last_cycle_tkeep_reg = 8'd0, last_cycle_tkeep_next;
80 reg last_cycle_tuser_reg = 1'b0, last_cycle_tuser_next;
81
82 reg [63:0] s_axis_tdata_d0 = 64'd0;
83 reg [7:0] s_axis_tkeep_d0 = 8'd0;
84 reg s_axis_tvalid_d0 = 1'b0;
85 reg s_axis_tuser_d0 = 1'b0;
86
87 reg busy_reg = 1'b0;
88 reg error_bad_fcs_reg = 1'b0, error_bad_fcs_next;
89
90 reg s_axis_tready_reg = 1'b0, s_axis_tready_next;
91
92 reg [31:0] crc_state = 32'hFFFFFFF;
93 reg [31:0] crc_state3 = 32'hFFFFFFF;
94
95 wire [31:0] crc_next0;
96 wire [31:0] crc_next1;
97 wire [31:0] crc_next2;
98 wire [31:0] crc_next3;
99 wire [31:0] crc_next7;
100
101 wire crc_valid0 = crc_next0 == ~32'h2144df1c;
102 wire crc_valid1 = crc_next1 == ~32'h2144df1c;
103 wire crc_valid2 = crc_next2 == ~32'h2144df1c;
104 wire crc_valid3 = crc_next3 == ~32'h2144df1c;
105
106 // internal datapath
107 reg [63:0] m_axis_tdata_int;
108 reg [7:0] m_axis_tkeep_int;
109 reg        m_axis_tvalid_int;
110 reg        m_axis_tready_int_reg = 1'b0;
111 reg        m_axis_tlast_int;
112 reg        m_axis_tuser_int;
113 wire       m_axis_tready_int_early;
114
115 assign s_axis_tready = s_axis_tready_reg;
116
117 assign busy = busy_reg;
118 assign error_bad_fcs = error_bad_fcs_reg;
119
120 wire last_cycle = state_reg == STATE_LAST;
121
122 lfsr #(
123   .LFSR_WIDTH(32),
124   .LFSR_POLY(32'h4c11db7),
125   .LFSR_CONFIG("GAL0IS"),
126   .LFSR_FEED_FORWARD(0),
127   .REVERSE(1),
128   .DATA_WIDTH(8),
129   .STYLE("AUTO")
130 )
131 eth_crc_8 (
132   .data_in(last_cycle ? s_axis_tdata_d0[39:32] : s_ax-
133 is_tdata[7:0]),
134   .state_in(last_cycle ? crc_state3 : crc_state),
135   .data_out(),
136   .state_out(crc_next0)
137 );
138
139 lfsr #(
140   .LFSR_WIDTH(32),
141   .LFSR_POLY(32'h4c11db7),
142   .LFSR_CONFIG("GAL0IS"),
143   .LFSR_FEED_FORWARD(0),
144   .REVERSE(1),
145   .DATA_WIDTH(16),
146   .STYLE("AUTO")
147 )
148 eth_crc_16 (
149   .data_in(last_cycle ? s_axis_tdata_d0[47:32] : s_ax-
150 is_tdata[15:0]),
151   .state_in(last_cycle ? crc_state3 : crc_state),
152   .data_out(),
153   .state_out(crc_next1)
154 );
155

```

```

154 lfsr #(
155     .LFSR_WIDTH(32),
156     .LFSR_POLY(32'h4c11db7),
157     .LFSR_CONFIG("GALOIS"),
158     .LFSR_FEED_FORWARD(0),
159     .REVERSE(1),
160     .DATA_WIDTH(24),
161     .STYLE("AUTO")
162 )
163 eth_crc_24 (
164     .data_in(last_cycle ? s_axis_tdata_d0[55:32] : s_ax-
is_tdata[23:0]),
165     .state_in(last_cycle ? crc_state3 : crc_state),
166     .data_out(),
167     .state_out(crc_next2)
168 );
169
170 lfsr #(
171     .LFSR_WIDTH(32),
172     .LFSR_POLY(32'h4c11db7),
173     .LFSR_CONFIG("GALOIS"),
174     .LFSR_FEED_FORWARD(0),
175     .REVERSE(1),
176     .DATA_WIDTH(32),
177     .STYLE("AUTO")
178 )
179 eth_crc_32 (
180     .data_in(last_cycle ? s_axis_tdata_d0[63:32] : s_ax-
is_tdata[31:0]),
181     .state_in(last_cycle ? crc_state3 : crc_state),
182     .data_out(),
183     .state_out(crc_next3)
184 );
185
186 lfsr #(
187     .LFSR_WIDTH(32),
188     .LFSR_POLY(32'h4c11db7),
189     .LFSR_CONFIG("GALOIS"),
190     .LFSR_FEED_FORWARD(0),
191     .REVERSE(1),
192     .DATA_WIDTH(64),
193     .STYLE("AUTO")
194 )
195 eth_crc_64 (
196     .data_in(s_axis_tdata[63:0]),
197     .state_in(crc_state),
198     .data_out(),
199     .state_out(crc_next7)
200 );
201
202 always @* begin
203     state_next = STATE_IDLE;
204
205     reset_crc = 1'b0;
206     update_crc = 1'b0;
207     shift_in = 1'b0;
208     shift_reset = 1'b0;
209
210     last_cycle_tkeep_next = last_cycle_tkeep_reg;
211     last_cycle_tuser_next = last_cycle_tuser_reg;
212
213     s_axis_tready_next = 1'b0;
214
215     m_axis_tdata_int = 64'd0;
216     m_axis_tkeep_int = 8'd0;
217     m_axis_tvalid_int = 1'b0;
218     m_axis_tlast_int = 1'b0;
219     m_axis_tuser_int = 1'b0;
220
221     error_bad_fcs_next = 1'b0;
222
223     case (state_reg)
224         STATE_IDLE: begin
225             // idle state - wait for data
226             s_axis_tready_next = m_axis_tready_int_early;
227             reset_crc = 1'b1;
228
229             m_axis_tdata_int = s_axis_tdata_d0;
230             m_axis_tkeep_int = s_axis_tkeep_d0;
231             m_axis_tvalid_int = s_axis_tvalid_d0 && s_ax-
is_tvalid;
232
233             m_axis_tlast_int = 1'b0;
234             m_axis_tuser_int = 1'b0;
235
236             if (s_axis_tready && s_axis_tvalid) begin
237                 shift_in = 1'b1;
238                 reset_crc = 1'b0;
239                 update_crc = 1'b1;
240                 if (s_axis_tlast) begin
241                     if (s_axis_tkeep[7:4] == 0) begin
242                         shift_reset = 1'b1;
243                         reset_crc = 1'b1;
244                         m_axis_tlast_int = 1'b1;
245                         m_axis_tuser_int = s_axis_tuser;
246
247                         m_axis_tkeep_int = {s_axis_tkeep[3:0],
4'b1111};
248                         if ((s_axis_tkeep[3:0] == 4'b0001 &&
crc_valid0) ||
249                             (s_axis_tkeep[3:0] == 4'b0011 &&
crc_valid1) ||

```

```

248         (s_axis_tkeep[3:0] == 4'b0111 &&
crc_valid2) ||
249             (s_axis_tkeep[3:0] == 4'b1111 &&
crc_valid3)) begin
250             // CRC valid
251         end else begin
252             m_axis_tuser_int = 1'b1;
253             error_bad_fcs_next = 1'b1;
254         end
255         s_axis_tready_next = m_ax-
is_tready_int_early;
256             state_next = STATE_IDLE;
257         end else begin
258             last_cycle_tkeep_next = {4'b0000,
s_axis_tkeep[7:4]};
259             last_cycle_tuser_next = s_axis_tuser;
260             s_axis_tready_next = 1'b0;
261             state_next = STATE_LAST;
262         end
263         end else begin
264             state_next = STATE_PAYLOAD;
265         end
266         end else begin
267             state_next = STATE_IDLE;
268         end
269     end
270     STATE_PAYLOAD: begin
271         // transfer payload
272         s_axis_tready_next = m_axis_tready_int_early;
273
274         m_axis_tdata_int = s_axis_tdata_d0;
275         m_axis_tkeep_int = s_axis_tkeep_d0;
276         m_axis_tvalid_int = s_axis_tvalid_d0 && s_ax-
is_tvalid;
277         m_axis_tlast_int = 1'b0;
278         m_axis_tuser_int = 1'b0;
279
280         if (s_axis_tready && s_axis_tvalid) begin
281             shift_in = 1'b1;
282             update_crc = 1'b1;
283             if (s_axis_tlast) begin
284                 if (s_axis_tkeep[7:4] == 0) begin
285                     shift_reset = 1'b1;
286                     reset_crc = 1'b1;
287                     m_axis_tlast_int = 1'b1;
288                     m_axis_tuser_int = s_axis_tuser;
289
290                     m_axis_tkeep_int = {s_axis_tkeep[3:0],
4'b1111};
291                     if ((s_axis_tkeep[3:0] == 4'b0001 &&
crc_valid0) ||
292                         (s_axis_tkeep[3:0] == 4'b0011 &&
crc_valid1) ||
293                         (s_axis_tkeep[3:0] == 4'b0111 &&
crc_valid2) ||
294                         (s_axis_tkeep[3:0] == 4'b1111 &&
crc_valid3)) begin
295                             // CRC valid
296                         end else begin
297                             m_axis_tuser_int = 1'b1;
298                             error_bad_fcs_next = 1'b1;
299                         end
300                         s_axis_tready_next = m_ax-
is_tready_int_early;
301                             state_next = STATE_IDLE;
302                         end else begin
303                             last_cycle_tkeep_next = {4'b0000,
s_axis_tkeep[7:4]};
304                             last_cycle_tuser_next = s_axis_tuser;
305                             s_axis_tready_next = 1'b0;
306                             state_next = STATE_LAST;
307                         end
308                         end else begin
309                             state_next = STATE_PAYLOAD;
310                         end
311                     end else begin
312                         state_next = STATE_PAYLOAD;
313                     end
314                 end
315             STATE_LAST: begin
316                 // last cycle
317                 s_axis_tready_next = 1'b0;
318
319                 m_axis_tdata_int = s_axis_tdata_d0;
320                 m_axis_tkeep_int = last_cycle_tkeep_reg;
321                 m_axis_tvalid_int = s_axis_tvalid_d0;
322                 m_axis_tlast_int = 1'b1;
323                 m_axis_tuser_int = last_cycle_tuser_reg;
324
325                 if ((s_axis_tkeep_d0[7:4] == 4'b0001 && crc_val-
id0) ||
326                     (s_axis_tkeep_d0[7:4] == 4'b0011 && crc_val-
id1) ||
327                     (s_axis_tkeep_d0[7:4] == 4'b0111 && crc_val-
id2) ||
328                     (s_axis_tkeep_d0[7:4] == 4'b1111 && crc_val-
id3)) begin
329                     // CRC valid
330                 end else begin
331                     m_axis_tuser_int = 1'b1;
332                     error_bad_fcs_next = 1'b1;

```



```

332     end
333
334     if (m_axis_tready_int_reg) begin
335         shift_reset = 1'b1;
336         reset_crc = 1'b1;
337         s_axis_tready_next = m_axis_tready_int_early;
338         state_next = STATE_IDLE;
339     end else begin
340         state_next = STATE_LAST;
341     end
342 end
343 endcase
344 end
345
346 always @(posedge clk) begin
347     if (rst) begin
348         state_reg <= STATE_IDLE;
349
350         s_axis_tready_reg <= 1'b0;
351
352         busy_reg <= 1'b0;
353         error_bad_fcs_reg <= 1'b0;
354
355         s_axis_tvalid_d0 <= 1'b0;
356
357         crc_state <= 32'hFFFFFFF;
358         crc_state3 <= 32'hFFFFFFF;
359     end else begin
360         state_reg <= state_next;
361
362         s_axis_tready_reg <= s_axis_tready_next;
363
364         busy_reg <= state_next != STATE_IDLE;
365         error_bad_fcs_reg <= error_bad_fcs_next;
366
367         // datapath
368         if (reset_crc) begin
369             crc_state <= 32'hFFFFFFF;
370             crc_state3 <= 32'hFFFFFFF;
371         end else if (update_crc) begin
372             crc_state <= crc_next7;
373             crc_state3 <= crc_next3;
374         end
375
376         if (shift_reset) begin
377             s_axis_tvalid_d0 <= 1'b0;
378         end else if (shift_in) begin
379             s_axis_tvalid_d0 <= s_axis_tvalid;
380         end
381     end
382
383     last_cycle_tkeep_reg <= last_cycle_tkeep_next;
384     last_cycle_tuser_reg <= last_cycle_tuser_next;
385
386     if (shift_in) begin
387         s_axis_tdata_d0 <= s_axis_tdata;
388         s_axis_tkeep_d0 <= s_axis_tkeep;
389         s_axis_tuser_d0 <= s_axis_tuser;
390     end
391 end
392
393 // output datapath logic
394 reg [63:0] m_axis_tdata_reg = 64'd0;
395 reg [7:0] m_axis_tkeep_reg = 8'd0;
396 reg m_axis_tvalid_reg = 1'b0, m_axis_tvalid_next;
397 reg m_axis_tlast_reg = 1'b0;
398 reg m_axis_tuser_reg = 1'b0;
399
400 reg [63:0] temp_m_axis_tdata_reg = 64'd0;
401 reg [7:0] temp_m_axis_tkeep_reg = 8'd0;
402 reg temp_m_axis_tvalid_reg = 1'b0, temp_m_ax-
is_tvalid_next;
403 reg temp_m_axis_tlast_reg = 1'b0;
404 reg temp_m_axis_tuser_reg = 1'b0;
405
406 // datapath control
407 reg store_axis_int_to_output;
408 reg store_axis_int_to_temp;
409 reg store_axis_temp_to_output;
410
411 assign m_axis_tdata = m_axis_tdata_reg;
412 assign m_axis_tkeep = m_axis_tkeep_reg;
413 assign m_axis_tvalid = m_axis_tvalid_reg;
414 assign m_axis_tlast = m_axis_tlast_reg;
415 assign m_axis_tuser = m_axis_tuser_reg;
416
417 // enable ready input next cycle if output is ready or if both
output registers are empty
418 assign m_axis_tready_int_early = m_axis_tready || (!temp_m_ax-
is_tvalid_reg && !m_axis_tvalid_reg);
419
420 always @* begin
421     // transfer sink ready state to source
422     m_axis_tvalid_next = m_axis_tvalid_reg;
423     temp_m_axis_tvalid_next = temp_m_axis_tvalid_reg;
424
425     store_axis_int_to_output = 1'b0;
426     store_axis_int_to_temp = 1'b0;
427     store_axis_temp_to_output = 1'b0;
428

```

```

429     if (m_axis_tready_int_reg) begin
430         // input is ready
431         if (m_axis_tready || !m_axis_tvalid_reg) begin
432             // output is ready or currently not valid, trans-
fer data to output
433             m_axis_tvalid_next = m_axis_tvalid_int;
434             store_axis_int_to_output = 1'b1;
435         end else begin
436             // output is not ready, store input in temp
437             temp_m_axis_tvalid_next = m_axis_tvalid_int;
438             store_axis_int_to_temp = 1'b1;
439         end
440     end else if (m_axis_tready) begin
441         // input is not ready, but output is ready
442         m_axis_tvalid_next = temp_m_axis_tvalid_reg;
443         temp_m_axis_tvalid_next = 1'b0;
444         store_axis_temp_to_output = 1'b1;
445     end
446 end
447
448 always @(posedge clk) begin
449     m_axis_tvalid_reg <= m_axis_tvalid_next;
450     m_axis_tready_int_reg <= m_axis_tready_int_early;
451     temp_m_axis_tvalid_reg <= temp_m_axis_tvalid_next;
452
453     // datapath
454     if (store_axis_int_to_output) begin
455         m_axis_tdata_reg <= m_axis_tdata_int;
456         m_axis_tkeep_reg <= m_axis_tkeep_int;
457         m_axis_tlast_reg <= m_axis_tlast_int;
458         m_axis_tuser_reg <= m_axis_tuser_int;
459     end else if (store_axis_temp_to_output) begin
460         m_axis_tdata_reg <= temp_m_axis_tdata_reg;
461         m_axis_tkeep_reg <= temp_m_axis_tkeep_reg;
462         m_axis_tlast_reg <= temp_m_axis_tlast_reg;
463         m_axis_tuser_reg <= temp_m_axis_tuser_reg;
464     end
465
466     if (store_axis_int_to_temp) begin
467         temp_m_axis_tdata_reg <= m_axis_tdata_int;
468         temp_m_axis_tkeep_reg <= m_axis_tkeep_int;
469         temp_m_axis_tlast_reg <= m_axis_tlast_int;
470         temp_m_axis_tuser_reg <= m_axis_tuser_int;
471     end
472
473     if (rst) begin
474         m_axis_tvalid_reg <= 1'b0;
475         m_axis_tready_int_reg <= 1'b0;
476         temp_m_axis_tvalid_reg <= 1'b0;
477     end
478 end
479
480 endmodule
481
482 `resetall

```

10. alexforenrich_verilog-ethernet/rtl/axis_eth_fcs_insert.v (372 lines)

```

1 /*
2
3 Copyright (c) 2015-2018 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps

```

```

29 `default_nettype none
30
31 /*
32  * AXI4-Stream Ethernet FCS inserter
33  */
34 module axis_eth_fcs_insert #
35 (
36     parameter ENABLE_PADDING = 0,
37     parameter MIN_FRAME_LENGTH = 64
38 )
39 (
40     input wire      clk,
41     input wire      rst,
42
43     /*
44      * AXI input
45      */
46     input wire [7:0] s_axis_tdata,
47     input wire      s_axis_tvalid,
48     output wire     s_axis_tready,
49     input wire      s_axis_tlast,
50     input wire      s_axis_tuser,
51
52     /*
53      * AXI output
54      */
55     output wire [7:0] m_axis_tdata,
56     output wire     m_axis_tvalid,
57     input wire      m_axis_tready,
58     output wire     m_axis_tlast,
59     output wire     m_axis_tuser,
60
61     /*
62      * Status
63      */
64     output wire      busy
65 );
66
67 localparam [1:0]
68     STATE_IDLE = 2'd0,
69     STATE_PAYLOAD = 2'd1,
70     STATE_PAD = 2'd2,
71     STATE_FCS = 2'd3;
72
73 reg [1:0] state_reg = STATE_IDLE, state_next;
74
75 // datapath control signals
76 reg reset_crc;
77 reg update_crc;
78
79 reg [15:0] frame_ptr_reg = 16'd0, frame_ptr_next;
80
81 reg busy_reg = 1'b0;
82
83 reg s_axis_tready_reg = 1'b0, s_axis_tready_next;
84
85 reg [31:0] crc_state = 32'hFFFFFFF;
86 wire [31:0] crc_next;
87
88 // internal datapath
89 reg [7:0] m_axis_tdata_int;
90 reg      m_axis_tvalid_int;
91 reg      m_axis_tready_int_reg = 1'b0;
92 reg      m_axis_tlast_int;
93 reg      m_axis_tuser_int;
94 wire     m_axis_tready_int_early;
95
96 assign s_axis_tready = s_axis_tready_reg;
97
98 assign busy = busy_reg;
99
100 lfsr #(
101     .LFSR_WIDTH(32),
102     .LFSR_POLY(32'h4c11db7),
103     .LFSR_CONFIG("GALOIS"),
104     .LFSR_FEED_FORWARD(0),
105     .REVERSE(1),
106     .DATA_WIDTH(8),
107     .STYLE("AUTO")
108 )
109 eth_crc_8 (
110     .data_in(m_axis_tdata_int),
111     .state_in(crc_state),
112     .data_out(),
113     .state_out(crc_next)
114 );
115
116 always @* begin
117     state_next = STATE_IDLE;
118
119     reset_crc = 1'b0;
120     update_crc = 1'b0;
121
122     frame_ptr_next = frame_ptr_reg;
123
124     s_axis_tready_next = 1'b0;
125
126     m_axis_tdata_int = 8'd0;
127     m_axis_tvalid_int = 1'b0;
128     m_axis_tlast_int = 1'b0;
129     m_axis_tuser_int = 1'b0;

```

```

130
131 case (state_reg)
132     STATE_IDLE: begin
133         // idle state - wait for data
134         s_axis_tready_next = m_axis_tready_int_early;
135         frame_ptr_next = 16'd0;
136         reset_crc = 1'b1;
137
138         m_axis_tdata_int = s_axis_tdata;
139         m_axis_tvalid_int = s_axis_tvalid;
140         m_axis_tlast_int = 1'b0;
141         m_axis_tuser_int = 1'b0;
142
143         if (s_axis_tready && s_axis_tvalid) begin
144             frame_ptr_next = 16'd1;
145             reset_crc = 1'b0;
146             update_crc = 1'b1;
147             if (s_axis_tlast) begin
148                 if (s_axis_tuser) begin
149                     m_axis_tlast_int = 1'b1;
150                     m_axis_tuser_int = 1'b1;
151                     reset_crc = 1'b1;
152                     frame_ptr_next = 16'd0;
153                     state_next = STATE_IDLE;
154                 end else begin
155                     s_axis_tready_next = 1'b0;
156                     if (ENABLE_PADDING && frame_ptr_reg <
157 MIN_FRAME_LENGTH-5) begin
158                         state_next = STATE_PAD;
159                     end else begin
160                         frame_ptr_next = 16'd0;
161                         state_next = STATE_FCS;
162                     end
163                 end
164             end else begin
165                 state_next = STATE_PAYLOAD;
166             end
167         end else begin
168             state_next = STATE_IDLE;
169         end
170     end
171     STATE_PAYLOAD: begin
172         // transfer payload
173         s_axis_tready_next = m_axis_tready_int_early;
174
175         m_axis_tdata_int = s_axis_tdata;
176         m_axis_tvalid_int = s_axis_tvalid;
177         m_axis_tlast_int = 1'b0;
178         m_axis_tuser_int = 1'b0;
179
180         if (s_axis_tready && s_axis_tvalid) begin
181             frame_ptr_next = frame_ptr_reg + 16'd1;
182             update_crc = 1'b1;
183             if (s_axis_tlast) begin
184                 if (s_axis_tuser) begin
185                     m_axis_tlast_int = 1'b1;
186                     m_axis_tuser_int = 1'b1;
187                     reset_crc = 1'b1;
188                     frame_ptr_next = 16'd0;
189                     state_next = STATE_IDLE;
190                 end else begin
191                     s_axis_tready_next = 1'b0;
192                     if (ENABLE_PADDING && frame_ptr_reg <
193 MIN_FRAME_LENGTH-5) begin
194                         state_next = STATE_PAD;
195                     end else begin
196                         frame_ptr_next = 16'd0;
197                         state_next = STATE_FCS;
198                     end
199                 end
200             end else begin
201                 state_next = STATE_PAYLOAD;
202             end
203         end else begin
204             state_next = STATE_PAYLOAD;
205         end
206     end
207     STATE_PAD: begin
208         // insert padding
209         s_axis_tready_next = 1'b0;
210
211         m_axis_tdata_int = 8'd0;
212         m_axis_tvalid_int = 1'b1;
213         m_axis_tlast_int = 1'b0;
214         m_axis_tuser_int = 1'b0;
215
216         if (m_axis_tready_int_reg) begin
217             frame_ptr_next = frame_ptr_reg + 16'd1;
218             update_crc = 1'b1;
219             if (frame_ptr_reg < MIN_FRAME_LENGTH-5) begin
220                 state_next = STATE_PAD;
221             end else begin
222                 frame_ptr_next = 16'd0;
223                 state_next = STATE_FCS;
224             end
225         end else begin
226             state_next = STATE_PAD;
227         end
228     end
229     STATE_FCS: begin
230         // send FCS

```

```

229     s_axis_tready_next = 1'b0;
230
231     case (frame_ptr_reg)
232     2'd0: m_axis_tdata_int = ~crc_state[7:0];
233     2'd1: m_axis_tdata_int = ~crc_state[15:8];
234     2'd2: m_axis_tdata_int = ~crc_state[23:16];
235     2'd3: m_axis_tdata_int = ~crc_state[31:24];
236     endcase
237     m_axis_tvalid_int = 1'b1;
238     m_axis_tlast_int = 1'b0;
239     m_axis_tuser_int = 1'b0;
240
241     if (m_axis_tready_int_reg) begin
242         frame_ptr_next = frame_ptr_reg + 16'd1;
243
244         if (frame_ptr_reg < 16'd3) begin
245             state_next = STATE_FCS;
246         end else begin
247             reset_crc = 1'b1;
248             frame_ptr_next = 16'd0;
249             m_axis_tlast_int = 1'b1;
250             s_axis_tready_next = m_ax-
is_tready_int_early;
251             state_next = STATE_IDLE;
252         end
253     end else begin
254         state_next = STATE_FCS;
255     end
256 end
257 endcase
258 end
259
260 always @(posedge clk) begin
261     if (rst) begin
262         state_reg <= STATE_IDLE;
263
264         frame_ptr_reg <= 1'b0;
265
266         s_axis_tready_reg <= 1'b0;
267
268         busy_reg <= 1'b0;
269
270         crc_state <= 32'hFFFFFFFF;
271     end else begin
272         state_reg <= state_next;
273
274         frame_ptr_reg <= frame_ptr_next;
275
276         s_axis_tready_reg <= s_axis_tready_next;
277
278         busy_reg <= state_next != STATE_IDLE;
279
280         // datapath
281         if (reset_crc) begin
282             crc_state <= 32'hFFFFFFFF;
283         end else if (update_crc) begin
284             crc_state <= crc_next;
285         end
286     end
287 end
288
289 // output datapath logic
290 reg [7:0] m_axis_tdata_reg = 8'd0;
291 reg      m_axis_tvalid_reg = 1'b0, m_axis_tvalid_next;
292 reg      m_axis_tlast_reg = 1'b0;
293 reg      m_axis_tuser_reg = 1'b0;
294
295 reg [7:0] temp_m_axis_tdata_reg = 8'd0;
296 reg      temp_m_axis_tvalid_reg = 1'b0, temp_m_ax-
is_tvalid_next;
297 reg      temp_m_axis_tlast_reg = 1'b0;
298 reg      temp_m_axis_tuser_reg = 1'b0;
299
300 // datapath control
301 reg store_axis_int_to_output;
302 reg store_axis_int_to_temp;
303 reg store_axis_temp_to_output;
304
305 assign m_axis_tdata = m_axis_tdata_reg;
306 assign m_axis_tvalid = m_axis_tvalid_reg;
307 assign m_axis_tlast = m_axis_tlast_reg;
308 assign m_axis_tuser = m_axis_tuser_reg;
309
310 // enable ready input next cycle if output is ready or if both
output registers are empty
311 assign m_axis_tready_int_early = m_axis_tready || (!temp_m_ax-
is_tvalid_reg && !m_axis_tvalid_reg);
312
313 always @* begin
314     // transfer sink ready state to source
315     m_axis_tvalid_next = m_axis_tvalid_reg;
316     temp_m_axis_tvalid_next = temp_m_axis_tvalid_reg;
317
318     store_axis_int_to_output = 1'b0;
319     store_axis_int_to_temp = 1'b0;
320     store_axis_temp_to_output = 1'b0;
321
322     if (m_axis_tready_int_reg) begin
323         // input is ready
324         if (m_axis_tready || !m_axis_tvalid_reg) begin

```

```

325         // output is ready or currently not valid, trans-
fer data to output
326         m_axis_tvalid_next = m_axis_tvalid_int;
327         store_axis_int_to_output = 1'b1;
328     end else begin
329         // output is not ready, store input in temp
330         temp_m_axis_tvalid_next = m_axis_tvalid_int;
331         store_axis_int_to_temp = 1'b1;
332     end
333 end else if (m_axis_tready) begin
334     // input is not ready, but output is ready
335     m_axis_tvalid_next = temp_m_axis_tvalid_reg;
336     temp_m_axis_tvalid_next = 1'b0;
337     store_axis_temp_to_output = 1'b1;
338 end
339 end
340
341 always @(posedge clk) begin
342     m_axis_tvalid_reg <= m_axis_tvalid_next;
343     m_axis_tready_int_reg <= m_axis_tready_int_early;
344     temp_m_axis_tvalid_reg <= temp_m_axis_tvalid_next;
345
346     // datapath
347     if (store_axis_int_to_output) begin
348         m_axis_tdata_reg <= m_axis_tdata_int;
349         m_axis_tlast_reg <= m_axis_tlast_int;
350         m_axis_tuser_reg <= m_axis_tuser_int;
351     end else if (store_axis_temp_to_output) begin
352         m_axis_tdata_reg <= temp_m_axis_tdata_reg;
353         m_axis_tlast_reg <= temp_m_axis_tlast_reg;
354         m_axis_tuser_reg <= temp_m_axis_tuser_reg;
355     end
356
357     if (store_axis_int_to_temp) begin
358         temp_m_axis_tdata_reg <= m_axis_tdata_int;
359         temp_m_axis_tlast_reg <= m_axis_tlast_int;
360         temp_m_axis_tuser_reg <= m_axis_tuser_int;
361     end
362
363     if (rst) begin
364         m_axis_tvalid_reg <= 1'b0;
365         m_axis_tready_int_reg <= 1'b0;
366         temp_m_axis_tvalid_reg <= 1'b0;
367     end
368 end
369
370 endmodule
371
372 `resetall

```

11. alexforench_verilog-ethernet/rtl/axis_eth_fcs_insert_64.v (721 lines)

```

1 /*
2
3 Copyright (c) 2015-2018 Alex Forench
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * AXI4-Stream Ethernet FCS inserter (64 bit datapath)
33 */
34 module axis_eth_fcs_insert_64 #

```

```

35 (
36     parameter ENABLE_PADDING = 0,
37     parameter MIN_FRAME_LENGTH = 64
38 )
39 (
40     input wire      clk,
41     input wire      rst,
42
43     /*
44      * AXI input
45      */
46     input wire [63:0] s_axis_tdata,
47     input wire [7:0]  s_axis_tkeep,
48     input wire        s_axis_tvalid,
49     output wire        s_axis_tready,
50     input wire        s_axis_tlast,
51     input wire        s_axis_tuser,
52
53     /*
54      * AXI output
55      */
56     output wire [63:0] m_axis_tdata,
57     output wire [7:0]  m_axis_tkeep,
58     output wire        m_axis_tvalid,
59     input wire        m_axis_tready,
60     output wire        m_axis_tlast,
61     output wire        m_axis_tuser,
62
63     /*
64      * Status
65      */
66     output wire        busy
67 );
68
69 localparam [1:0]
70     STATE_IDLE = 2'd0,
71     STATE_PAYLOAD = 2'd1,
72     STATE_PAD = 2'd2,
73     STATE_FCS = 2'd3;
74
75 reg [1:0] state_reg = STATE_IDLE, state_next;
76
77 // datapath control signals
78 reg reset_crc;
79 reg update_crc;
80
81 reg [63:0] s_axis_tdata_masked;
82
83 reg [63:0] fcs_s_tdata;
84 reg [7:0]  fcs_s_tkeep;
85
86 reg [63:0] fcs_m_tdata_0;
87 reg [63:0] fcs_m_tdata_1;
88 reg [7:0]  fcs_m_tkeep_0;
89 reg [7:0]  fcs_m_tkeep_1;
90
91 reg [15:0] frame_ptr_reg = 16'd0, frame_ptr_next;
92
93 reg [63:0] last_cycle_tdata_reg = 64'd0,
last_cycle_tdata_next;
94 reg [7:0] last_cycle_tkeep_reg = 8'd0, last_cycle_tkeep_next;
95
96 reg busy_reg = 1'b0;
97
98 reg s_axis_tready_reg = 1'b0, s_axis_tready_next;
99
100 reg [31:0] crc_state = 32'hFFFFFFF;
101
102 wire [31:0] crc_next0;
103 wire [31:0] crc_next1;
104 wire [31:0] crc_next2;
105 wire [31:0] crc_next3;
106 wire [31:0] crc_next4;
107 wire [31:0] crc_next5;
108 wire [31:0] crc_next6;
109 wire [31:0] crc_next7;
110
111 // internal datapath
112 reg [63:0] m_axis_tdata_int;
113 reg [7:0]  m_axis_tkeep_int;
114 reg        m_axis_tvalid_int;
115 reg        m_axis_tready_int_reg = 1'b0;
116 reg        m_axis_tlast_int;
117 reg        m_axis_tuser_int;
118 wire        m_axis_tready_int_early;
119
120 assign s_axis_tready = s_axis_tready_reg;
121
122 assign busy = busy_reg;
123
124 lfsr #(
125     .LFSR_WIDTH(32),
126     .LFSR_POLY(32'h4c11db7),
127     .LFSR_CONFIG("GALOIS"),
128     .LFSR_FEED_FORWARD(0),
129     .REVERSE(1),
130     .DATA_WIDTH(8),
131     .STYLE("AUTO")
132 )
133 eth_crc_8 (
134     .data_in(fcs_s_tdata[7:0]),
135     .state_in(crc_state),
136     .data_out(),
137     .state_out(crc_next0)
138 );
139
140 lfsr #(
141     .LFSR_WIDTH(32),
142     .LFSR_POLY(32'h4c11db7),
143     .LFSR_CONFIG("GALOIS"),
144     .LFSR_FEED_FORWARD(0),
145     .REVERSE(1),
146     .DATA_WIDTH(16),
147     .STYLE("AUTO")
148 )
149 eth_crc_16 (
150     .data_in(fcs_s_tdata[15:0]),
151     .state_in(crc_state),
152     .data_out(),
153     .state_out(crc_next1)
154 );
155
156 lfsr #(
157     .LFSR_WIDTH(32),
158     .LFSR_POLY(32'h4c11db7),
159     .LFSR_CONFIG("GALOIS"),
160     .LFSR_FEED_FORWARD(0),
161     .REVERSE(1),
162     .DATA_WIDTH(24),
163     .STYLE("AUTO")
164 )
165 eth_crc_24 (
166     .data_in(fcs_s_tdata[23:0]),
167     .state_in(crc_state),
168     .data_out(),
169     .state_out(crc_next2)
170 );
171
172 lfsr #(
173     .LFSR_WIDTH(32),
174     .LFSR_POLY(32'h4c11db7),
175     .LFSR_CONFIG("GALOIS"),
176     .LFSR_FEED_FORWARD(0),
177     .REVERSE(1),
178     .DATA_WIDTH(32),
179     .STYLE("AUTO")
180 )
181 eth_crc_32 (
182     .data_in(fcs_s_tdata[31:0]),
183     .state_in(crc_state),
184     .data_out(),
185     .state_out(crc_next3)
186 );
187
188 lfsr #(
189     .LFSR_WIDTH(32),
190     .LFSR_POLY(32'h4c11db7),
191     .LFSR_CONFIG("GALOIS"),
192     .LFSR_FEED_FORWARD(0),
193     .REVERSE(1),
194     .DATA_WIDTH(40),
195     .STYLE("AUTO")
196 )
197 eth_crc_40 (
198     .data_in(fcs_s_tdata[39:0]),
199     .state_in(crc_state),
200     .data_out(),
201     .state_out(crc_next4)
202 );
203
204 lfsr #(
205     .LFSR_WIDTH(32),
206     .LFSR_POLY(32'h4c11db7),
207     .LFSR_CONFIG("GALOIS"),
208     .LFSR_FEED_FORWARD(0),
209     .REVERSE(1),
210     .DATA_WIDTH(48),
211     .STYLE("AUTO")
212 )
213 eth_crc_48 (
214     .data_in(fcs_s_tdata[47:0]),
215     .state_in(crc_state),
216     .data_out(),
217     .state_out(crc_next5)
218 );
219
220 lfsr #(
221     .LFSR_WIDTH(32),
222     .LFSR_POLY(32'h4c11db7),
223     .LFSR_CONFIG("GALOIS"),
224     .LFSR_FEED_FORWARD(0),
225     .REVERSE(1),
226     .DATA_WIDTH(56),
227     .STYLE("AUTO")
228 )
229 eth_crc_56 (
230     .data_in(fcs_s_tdata[55:0]),
231     .state_in(crc_state),
232     .data_out(),
233     .state_out(crc_next6)
234 );
235

```

```

236 lfsr #(
237     .LFSR_WIDTH(32),
238     .LFSR_POLY(32'h4c11db7),
239     .LFSR_CONFIG("GALOIS"),
240     .LFSR_FEED_FORWARD(0),
241     .REVERSE(1),
242     .DATA_WIDTH(64),
243     .STYLE("AUTO")
244 )
245 eth_crc_64 (
246     .data_in(fcs_s_tdata[63:0]),
247     .state_in(crc_state),
248     .data_out(),
249     .state_out(crc_next7)
250 );
251
252 function [3:0] keep2count;
253     input [7:0] k;
254     casez (k)
255         8'bzzzzzz0: keep2count = 4'd0;
256         8'bzzzzzz01: keep2count = 4'd1;
257         8'bzzzzzz011: keep2count = 4'd2;
258         8'bzzzzzz0111: keep2count = 4'd3;
259         8'bzzzz01111: keep2count = 4'd4;
260         8'bzzzz011111: keep2count = 4'd5;
261         8'bzz0111111: keep2count = 4'd6;
262         8'b01111111: keep2count = 4'd7;
263         8'b11111111: keep2count = 4'd8;
264     endcase
265 endfunction
266
267 function [7:0] count2keep;
268     input [3:0] k;
269     case (k)
270         4'd0: count2keep = 8'b00000000;
271         4'd1: count2keep = 8'b00000001;
272         4'd2: count2keep = 8'b00000011;
273         4'd3: count2keep = 8'b00000111;
274         4'd4: count2keep = 8'b00001111;
275         4'd5: count2keep = 8'b00011111;
276         4'd6: count2keep = 8'b00111111;
277         4'd7: count2keep = 8'b01111111;
278         4'd8: count2keep = 8'b11111111;
279     endcase
280 endfunction
281
282 // Mask input data
283 integer j;
284
285 always @* begin
286     for (j = 0; j < 8; j = j + 1) begin
287         s_axis_tdata_masked[j*8 +: 8] = s_axis_tkeep[j] ?
s_axis_tdata[j*8 +: 8] : 8'd0;
288     end
289 end
290
291 // FCS cycle calculation
292 always @* begin
293     casez (fcs_s_tkeep)
294         8'bzzzzzz01: begin
295             fcs_m_tdata_0 = {24'd0, ~crc_next0[31:0], fcs_s_t-
data[7:0]};
296             fcs_m_tdata_1 = 64'd0;
297             fcs_m_tkeep_0 = 8'b00011111;
298             fcs_m_tkeep_1 = 8'b00000000;
299         end
300         8'bzzzzzz011: begin
301             fcs_m_tdata_0 = {16'd0, ~crc_next1[31:0], fcs_s_t-
data[15:0]};
302             fcs_m_tdata_1 = 64'd0;
303             fcs_m_tkeep_0 = 8'b00111111;
304             fcs_m_tkeep_1 = 8'b00000000;
305         end
306         8'bzzzzzz0111: begin
307             fcs_m_tdata_0 = {8'd0, ~crc_next2[31:0], fcs_s_t-
data[23:0]};
308             fcs_m_tdata_1 = 64'd0;
309             fcs_m_tkeep_0 = 8'b01111111;
310             fcs_m_tkeep_1 = 8'b00000000;
311         end
312         8'bzzzz01111: begin
313             fcs_m_tdata_0 = {~crc_next3[31:0], fcs_s_t-
data[31:0]};
314             fcs_m_tdata_1 = 64'd0;
315             fcs_m_tkeep_0 = 8'b11111111;
316             fcs_m_tkeep_1 = 8'b00000000;
317         end
318         8'bzz011111: begin
319             fcs_m_tdata_0 = {~crc_next4[23:0], fcs_s_t-
data[39:0]};
320             fcs_m_tdata_1 = {56'd0, ~crc_next4[31:24]};
321             fcs_m_tkeep_0 = 8'b11111111;
322             fcs_m_tkeep_1 = 8'b00000001;
323         end
324         8'bzz0111111: begin
325             fcs_m_tdata_0 = {~crc_next5[15:0], fcs_s_t-
data[47:0]};
326             fcs_m_tdata_1 = {48'd0, ~crc_next5[31:16]};
327             fcs_m_tkeep_0 = 8'b11111111;
328             fcs_m_tkeep_1 = 8'b00000011;
329         end

```

```

330         8'b01111111: begin
331             fcs_m_tdata_0 = {~crc_next6[7:0], fcs_s_t-
data[55:0]};
332             fcs_m_tdata_1 = {40'd0, ~crc_next6[31:8]};
333             fcs_m_tkeep_0 = 8'b11111111;
334             fcs_m_tkeep_1 = 8'b00000111;
335         end
336         8'b11111111: begin
337             fcs_m_tdata_0 = fcs_s_tdata;
338             fcs_m_tdata_1 = {32'd0, ~crc_next7[31:0]};
339             fcs_m_tkeep_0 = 8'b11111111;
340             fcs_m_tkeep_1 = 8'b00000111;
341         end
342     default: begin
343         fcs_m_tdata_0 = 64'd0;
344         fcs_m_tdata_1 = 64'd0;
345         fcs_m_tkeep_0 = 8'd0;
346         fcs_m_tkeep_1 = 8'd0;
347     end
348 endcase
349 end
350
351 always @* begin
352     state_next = STATE_IDLE;
353
354     reset_crc = 1'b0;
355     update_crc = 1'b0;
356
357     frame_ptr_next = frame_ptr_reg;
358
359     last_cycle_tdata_next = last_cycle_tdata_reg;
360     last_cycle_tkeep_next = last_cycle_tkeep_reg;
361
362     s_axis_tready_next = 1'b0;
363
364     fcs_s_tdata = 64'd0;
365     fcs_s_tkeep = 8'd0;
366
367     m_axis_tdata_int = 64'd0;
368     m_axis_tkeep_int = 8'd0;
369     m_axis_tvalid_int = 1'b0;
370     m_axis_tlast_int = 1'b0;
371     m_axis_tuser_int = 1'b0;
372
373     case (state_reg)
374         STATE_IDLE: begin
375             // idle state - wait for data
376             s_axis_tready_next = m_axis_tready_int_early;
377             frame_ptr_next = 16'd0;
378             reset_crc = 1'b1;
379
380             m_axis_tdata_int = s_axis_tdata_masked;
381             m_axis_tkeep_int = s_axis_tkeep;
382             m_axis_tvalid_int = s_axis_tvalid;
383             m_axis_tlast_int = 1'b0;
384             m_axis_tuser_int = 1'b0;
385
386             fcs_s_tdata = s_axis_tdata_masked;
387             fcs_s_tkeep = s_axis_tkeep;
388
389             if (s_axis_tready && s_axis_tvalid) begin
390                 reset_crc = 1'b0;
391                 update_crc = 1'b1;
392                 frame_ptr_next = keep2count(s_axis_tkeep);
393                 if (s_axis_tlast) begin
394                     if (s_axis_tuser) begin
395                         m_axis_tlast_int = 1'b1;
396                         m_axis_tuser_int = 1'b1;
397                         reset_crc = 1'b1;
398                         frame_ptr_next = 16'd0;
399                         state_next = STATE_IDLE;
400                     end else begin
401                         if (ENABLE_PADDING && frame_ptr_next <
MIN_FRAME_LENGTH-4) begin
402                             m_axis_tkeep_int = 8'hff;
403                             fcs_s_tkeep = 8'hff;
404                             frame_ptr_next = frame_ptr_reg +
16'd8;
405                         end
406                         if (frame_ptr_next <
MIN_FRAME_LENGTH-4) begin
407                             s_axis_tready_next = 1'b0;
408                             state_next = STATE_PAD;
409                         end else begin
410                             m_axis_tkeep_int = 8'hff >>
(8-((MIN_FRAME_LENGTH-4) & 7));
411                             fcs_s_tkeep = 8'hff >> (8-
((MIN_FRAME_LENGTH-4) & 7));
412                             m_axis_tdata_int = fcs_m_t-
data_0;
413                             last_cycle_tdata_next =
fcs_m_tdata_1;
414                             m_axis_tkeep_int = fcs_m_t-
keep_0;
415                             last_cycle_tkeep_next =
fcs_m_tkeep_1;
416                             reset_crc = 1'b1;
417
418
419

```

```

420         if (fcs_m_tkeep_1 == 8'd0) be-
gin
421             m_axis_tlast_int = 1'b1;
422             s_axis_tready_next = m_ax-
is_tready_int_early;
423             frame_ptr_next = 1'b0;
424             state_next = STATE_IDLE;
425         end else begin
426             s_axis_tready_next = 1'b0;
427             state_next = STATE_FCS;
428         end
429     end
430 end else begin
431     m_axis_tdata_int = fcs_m_tdata_0;
432     last_cycle_tdata_next = fcs_m_t-
data_1;
433     m_axis_tkeep_int = fcs_m_tkeep_0;
434     last_cycle_tkeep_next = fcs_m_t-
keep_1;
435
436     reset_crc = 1'b1;
437
438     if (fcs_m_tkeep_1 == 8'd0) begin
439         m_axis_tlast_int = 1'b1;
440         s_axis_tready_next = m_ax-
is_tready_int_early;
441         frame_ptr_next = 16'd0;
442         state_next = STATE_IDLE;
443     end else begin
444         s_axis_tready_next = 1'b0;
445         state_next = STATE_FCS;
446     end
447 end
448 end
449 end else begin
450     state_next = STATE_PAYLOAD;
451 end
452 end else begin
453     state_next = STATE_IDLE;
454 end
455 end
456 STATE_PAYLOAD: begin
457     // transfer payload
458     s_axis_tready_next = m_axis_tready_int_early;
459
460     m_axis_tdata_int = s_axis_tdata_masked;
461     m_axis_tkeep_int = s_axis_tkeep;
462     m_axis_tvalid_int = s_axis_tvalid;
463     m_axis_tlast_int = 1'b0;
464     m_axis_tuser_int = 1'b0;
465
466     fcs_s_tdata = s_axis_tdata_masked;
467     fcs_s_tkeep = s_axis_tkeep;
468
469     if (s_axis_tready && s_axis_tvalid) begin
470         update_crc = 1'b1;
471         frame_ptr_next = frame_ptr_reg + keep2-
count(s_axis_tkeep);
472         if (s_axis_tlast) begin
473             if (s_axis_tuser) begin
474                 m_axis_tlast_int = 1'b1;
475                 m_axis_tuser_int = 1'b1;
476                 reset_crc = 1'b1;
477                 frame_ptr_next = 16'd0;
478                 state_next = STATE_IDLE;
479             end else begin
480                 if (ENABLE_PADDING && frame_ptr_next <
MIN_FRAME_LENGTH-4) begin
481                     m_axis_tkeep_int = 8'hff;
482                     fcs_s_tkeep = 8'hff;
483                     frame_ptr_next = frame_ptr_reg +
16'd8;
484
485                     if (frame_ptr_next <
MIN_FRAME_LENGTH-4) begin
486                         s_axis_tready_next = 1'b0;
487                         state_next = STATE_PAD;
488                     end else begin
489                         m_axis_tkeep_int = 8'hff >>
(8-((MIN_FRAME_LENGTH-4) & 7));
490                         fcs_s_tkeep = 8'hff >> (8-
((MIN_FRAME_LENGTH-4) & 7));
491
492                         m_axis_tdata_int = fcs_m_t-
data_0;
493                         last_cycle_tdata_next =
fcs_m_tdata_1;
494                         m_axis_tkeep_int = fcs_m_t-
keep_0;
495                         last_cycle_tkeep_next =
fcs_m_tkeep_1;
496
497                         reset_crc = 1'b1;
498
499                         if (fcs_m_tkeep_1 == 8'd0) be-
gin
500                             m_axis_tlast_int = 1'b1;
501                             s_axis_tready_next = m_ax-
is_tready_int_early;
502                             frame_ptr_next = 16'd0;

```

```

503                             state_next = STATE_IDLE;
504                         end else begin
505                             s_axis_tready_next = 1'b0;
506                             state_next = STATE_FCS;
507                         end
508                     end else begin
509                         m_axis_tdata_int = fcs_m_tdata_0;
510                         last_cycle_tdata_next = fcs_m_t-
data_1;
511                         m_axis_tkeep_int = fcs_m_tkeep_0;
512                         last_cycle_tkeep_next = fcs_m_t-
keep_1;
513
514                         reset_crc = 1'b1;
515
516                         if (fcs_m_tkeep_1 == 8'd0) begin
517                             m_axis_tlast_int = 1'b1;
518                             s_axis_tready_next = m_ax-
is_tready_int_early;
519                             frame_ptr_next = 16'd0;
520                             state_next = STATE_IDLE;
521                         end else begin
522                             s_axis_tready_next = 1'b0;
523                             state_next = STATE_FCS;
524                         end
525                     end
526                 end
527             end else begin
528                 state_next = STATE_PAYLOAD;
529             end
530         end else begin
531             state_next = STATE_PAYLOAD;
532         end
533     end
534 end
535 STATE_PAD: begin
536     s_axis_tready_next = 1'b0;
537
538     m_axis_tdata_int = 64'd0;
539     m_axis_tkeep_int = 8'hff;
540     m_axis_tvalid_int = 1'b1;
541     m_axis_tlast_int = 1'b0;
542     m_axis_tuser_int = 1'b0;
543
544     fcs_s_tdata = 64'd0;
545     fcs_s_tkeep = 8'hff;
546
547     if (m_axis_tready_int_reg) begin
548         update_crc = 1'b1;
549         frame_ptr_next = frame_ptr_reg + 16'd8;
550
551         if (frame_ptr_next < MIN_FRAME_LENGTH-4) begin
552             state_next = STATE_PAD;
553         end else begin
554             m_axis_tkeep_int = 8'hff >> (8-
((MIN_FRAME_LENGTH-4) & 7));
555             fcs_s_tkeep = 8'hff >> (8-
((MIN_FRAME_LENGTH-4) & 7));
556
557             m_axis_tdata_int = fcs_m_tdata_0;
558             last_cycle_tdata_next = fcs_m_tdata_1;
559             m_axis_tkeep_int = fcs_m_tkeep_0;
560             last_cycle_tkeep_next = fcs_m_tkeep_1;
561
562             reset_crc = 1'b1;
563
564             if (fcs_m_tkeep_1 == 8'd0) begin
565                 m_axis_tlast_int = 1'b1;
566                 s_axis_tready_next = m_ax-
is_tready_int_early;
567                 frame_ptr_next = 16'd0;
568                 state_next = STATE_IDLE;
569             end else begin
570                 s_axis_tready_next = 1'b0;
571                 state_next = STATE_FCS;
572             end
573         end
574     end else begin
575         state_next = STATE_PAD;
576     end
577 end
578 STATE_FCS: begin
579     // last cycle
580     s_axis_tready_next = 1'b0;
581
582     m_axis_tdata_int = last_cycle_tdata_reg;
583     m_axis_tkeep_int = last_cycle_tkeep_reg;
584     m_axis_tvalid_int = 1'b1;
585     m_axis_tlast_int = 1'b1;
586     m_axis_tuser_int = 1'b0;
587
588     if (m_axis_tready_int_reg) begin
589         reset_crc = 1'b1;
590         s_axis_tready_next = m_axis_tready_int_early;
591         frame_ptr_next = 1'b0;
592         state_next = STATE_IDLE;
593     end else begin
594         state_next = STATE_FCS;
595     end
596 end
597 endcase

```



```

598 end
599
600 always @(posedge clk) begin
601   if (rst) begin
602     state_reg <= STATE_IDLE;
603
604     frame_ptr_reg <= 1'b0;
605
606     s_axis_tready_reg <= 1'b0;
607
608     busy_reg <= 1'b0;
609
610     crc_state <= 32'hFFFFFFF;
611   end else begin
612     state_reg <= state_next;
613
614     frame_ptr_reg <= frame_ptr_next;
615
616     s_axis_tready_reg <= s_axis_tready_next;
617
618     busy_reg <= state_next != STATE_IDLE;
619
620     // datapath
621     if (reset_crc) begin
622       crc_state <= 32'hFFFFFFF;
623     end else if (update_crc) begin
624       crc_state <= crc_next;
625     end
626   end
627
628   last_cycle_tdata_reg <= last_cycle_tdata_next;
629   last_cycle_tkeep_reg <= last_cycle_tkeep_next;
630 end
631
632 // output datapath logic
633 reg [63:0] m_axis_tdata_reg = 64'd0;
634 reg [7:0] m_axis_tkeep_reg = 8'd0;
635 reg m_axis_tvalid_reg = 1'b0, m_axis_tvalid_next;
636 reg m_axis_tlast_reg = 1'b0;
637 reg m_axis_tuser_reg = 1'b0;
638
639 reg [63:0] temp_m_axis_tdata_reg = 64'd0;
640 reg [7:0] temp_m_axis_tkeep_reg = 8'd0;
641 reg temp_m_axis_tvalid_reg = 1'b0, temp_m_ax-
is_tvalid_next;
642 reg temp_m_axis_tlast_reg = 1'b0;
643 reg temp_m_axis_tuser_reg = 1'b0;
644
645 // datapath control
646 reg store_axis_int_to_output;
647 reg store_axis_int_to_temp;
648 reg store_axis_temp_to_output;
649
650 assign m_axis_tdata = m_axis_tdata_reg;
651 assign m_axis_tkeep = m_axis_tkeep_reg;
652 assign m_axis_tvalid = m_axis_tvalid_reg;
653 assign m_axis_tlast = m_axis_tlast_reg;
654 assign m_axis_tuser = m_axis_tuser_reg;
655
656 // enable ready input next cycle if output is ready or if both
output registers are empty
657 assign m_axis_tready_int_early = m_axis_tready || (!temp_m_ax-
is_tvalid_reg && !m_axis_tvalid_reg);
658
659 always @* begin
660   // transfer sink ready state to source
661   m_axis_tvalid_next = m_axis_tvalid_reg;
662   temp_m_axis_tvalid_next = temp_m_axis_tvalid_reg;
663
664   store_axis_int_to_output = 1'b0;
665   store_axis_int_to_temp = 1'b0;
666   store_axis_temp_to_output = 1'b0;
667
668   if (m_axis_tready_int_reg) begin
669     // input is ready
670     if (m_axis_tready || !m_axis_tvalid_reg) begin
671       // output is ready or currently not valid, trans-
fer data to output
672       m_axis_tvalid_next = m_axis_tvalid_int;
673       store_axis_int_to_output = 1'b1;
674     end else begin
675       // output is not ready, store input in temp
676       temp_m_axis_tvalid_next = m_axis_tvalid_int;
677       store_axis_int_to_temp = 1'b1;
678     end
679   end else if (m_axis_tready) begin
680     // input is not ready, but output is ready
681     m_axis_tvalid_next = temp_m_axis_tvalid_reg;
682     temp_m_axis_tvalid_next = 1'b0;
683     store_axis_temp_to_output = 1'b1;
684   end
685 end
686
687 always @(posedge clk) begin
688   m_axis_tvalid_reg <= m_axis_tvalid_next;
689   m_axis_tready_int_reg <= m_axis_tready_int_early;
690   temp_m_axis_tvalid_reg <= temp_m_axis_tvalid_next;
691
692   // datapath
693   if (store_axis_int_to_output) begin

```

```

694     m_axis_tdata_reg <= m_axis_tdata_int;
695     m_axis_tkeep_reg <= m_axis_tkeep_int;
696     m_axis_tlast_reg <= m_axis_tlast_int;
697     m_axis_tuser_reg <= m_axis_tuser_int;
698   end else if (store_axis_temp_to_output) begin
699     m_axis_tdata_reg <= temp_m_axis_tdata_reg;
700     m_axis_tkeep_reg <= temp_m_axis_tkeep_reg;
701     m_axis_tlast_reg <= temp_m_axis_tlast_reg;
702     m_axis_tuser_reg <= temp_m_axis_tuser_reg;
703   end
704
705   if (store_axis_int_to_temp) begin
706     temp_m_axis_tdata_reg <= m_axis_tdata_int;
707     temp_m_axis_tkeep_reg <= m_axis_tkeep_int;
708     temp_m_axis_tlast_reg <= m_axis_tlast_int;
709     temp_m_axis_tuser_reg <= m_axis_tuser_int;
710   end
711
712   if (rst) begin
713     m_axis_tvalid_reg <= 1'b0;
714     m_axis_tready_int_reg <= 1'b0;
715     temp_m_axis_tvalid_reg <= 1'b0;
716   end
717 end
718
719 endmodule
720
721 `resetall

```

12. alexforenrich_verilog-ethernet/rtl/axis_gmii_rx.v (358 lines)

```

1 /*
2
3 Copyright (c) 2015-2018 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * AXI4-Stream GMII frame receiver (GMII in, AXI out)
33 */
34 module axis_gmii_rx #
35 (
36   parameter DATA_WIDTH = 8,
37   parameter PTP_TS_ENABLE = 0,
38   parameter PTP_TS_WIDTH = 96,
39   parameter USER_WIDTH = (PTP_TS_ENABLE ? PTP_TS_WIDTH : 0)
+ 1
40 )
41 (
42   input wire clk,
43   input wire rst,
44
45   /*
46    * GMII input
47    */
48   input wire [DATA_WIDTH-1:0] gmii_rxd,
49   input wire gmii_rx_dv,
50   input wire gmii_rx_er,
51
52   /*
53    * AXI output
54    */
55   output wire [DATA_WIDTH-1:0] m_axis_tdata,

```

```

56 output wire m_axis_tvalid,
57 output wire m_axis_tlast,
58 output wire [USER_WIDTH-1:0] m_axis_tuser,
59
60 /*
61  * PTP
62  */
63 input wire [PTP_TS_WIDTH-1:0] ptp_ts,
64
65 /*
66  * Control
67  */
68 input wire clk_enable,
69 input wire mii_select,
70
71 /*
72  * Configuration
73  */
74 input wire cfg_rx_enable,
75
76 /*
77  * Status
78  */
79 output wire start_packet,
80 output wire error_bad_frame,
81 output wire error_bad_fcs
82 );
83
84 // bus width assertions
85 initial begin
86   if (DATA_WIDTH != 8) begin
87     $error("Error: Interface width must be 8");
88     $finish;
89   end
90 end
91
92 localparam [7:0]
93   ETH_PRE = 8'h55,
94   ETH_SFD = 8'hD5;
95
96 localparam [2:0]
97   STATE_IDLE = 3'd0,
98   STATE_PAYLOAD = 3'd1,
99   STATE_WAIT_LAST = 3'd2;
100
101 reg [2:0] state_reg = STATE_IDLE, state_next;
102
103 // datapath control signals
104 reg reset_crc;
105 reg update_crc;
106
107 reg mii_odd = 1'b0;
108 reg in_frame = 1'b0;
109
110 reg [DATA_WIDTH-1:0] gmii_rxd_d0 = {DATA_WIDTH{1'b0}};
111 reg [DATA_WIDTH-1:0] gmii_rxd_d1 = {DATA_WIDTH{1'b0}};
112 reg [DATA_WIDTH-1:0] gmii_rxd_d2 = {DATA_WIDTH{1'b0}};
113 reg [DATA_WIDTH-1:0] gmii_rxd_d3 = {DATA_WIDTH{1'b0}};
114 reg [DATA_WIDTH-1:0] gmii_rxd_d4 = {DATA_WIDTH{1'b0}};
115
116 reg gmii_rx_dv_d0 = 1'b0;
117 reg gmii_rx_dv_d1 = 1'b0;
118 reg gmii_rx_dv_d2 = 1'b0;
119 reg gmii_rx_dv_d3 = 1'b0;
120 reg gmii_rx_dv_d4 = 1'b0;
121
122 reg gmii_rx_er_d0 = 1'b0;
123 reg gmii_rx_er_d1 = 1'b0;
124 reg gmii_rx_er_d2 = 1'b0;
125 reg gmii_rx_er_d3 = 1'b0;
126 reg gmii_rx_er_d4 = 1'b0;
127
128 reg [DATA_WIDTH-1:0] m_axis_tdata_reg = {DATA_WIDTH{1'b0}},
m_axis_tdata_next;
129 reg m_axis_tvalid_reg = 1'b0, m_axis_tvalid_next;
130 reg m_axis_tlast_reg = 1'b0, m_axis_tlast_next;
131 reg m_axis_tuser_reg = 1'b0, m_axis_tuser_next;
132
133 reg start_packet_int_reg = 1'b0;
134 reg start_packet_reg = 1'b0;
135 reg error_bad_frame_reg = 1'b0, error_bad_frame_next;
136 reg error_bad_fcs_reg = 1'b0, error_bad_fcs_next;
137
138 reg [PTP_TS_WIDTH-1:0] ptp_ts_reg = 0;
139
140 reg [31:0] crc_state = 32'hFFFFFFF;
141 wire [31:0] crc_next;
142
143 assign m_axis_tdata = m_axis_tdata_reg;
144 assign m_axis_tvalid = m_axis_tvalid_reg;
145 assign m_axis_tlast = m_axis_tlast_reg;
146 assign m_axis_tuser = PTP_TS_ENABLE ? {ptp_ts_reg, m_ax-
is_tuser_reg} : m_axis_tuser_reg;
147
148 assign start_packet = start_packet_reg;
149 assign error_bad_frame = error_bad_frame_reg;
150 assign error_bad_fcs = error_bad_fcs_reg;
151
152 lfsr #(
153   .LFSR_WIDTH(32),
154   .LFSR_POLY(32'h4c11db7),

```

```

155   .LFSR_CONFIG("GALOIS"),
156   .LFSR_FEED_FORWARD(0),
157   .REVERSE(1),
158   .DATA_WIDTH(8),
159   .STYLE("AUTO")
160 )
161 eth_crc_8 (
162   .data_in(gmii_rxd_d4),
163   .state_in(crc_state),
164   .data_out(),
165   .state_out(crc_next)
166 );
167
168 always @* begin
169   state_next = STATE_IDLE;
170
171   reset_crc = 1'b0;
172   update_crc = 1'b0;
173
174   m_axis_tdata_next = {DATA_WIDTH{1'b0}};
175   m_axis_tvalid_next = 1'b0;
176   m_axis_tlast_next = 1'b0;
177   m_axis_tuser_next = 1'b0;
178
179   error_bad_frame_next = 1'b0;
180   error_bad_fcs_next = 1'b0;
181
182   if (!clk_enable) begin
183     // clock disabled - hold state
184     state_next = state_reg;
185   end else if (mii_select && !mii_odd) begin
186     // MII even cycle - hold state
187     state_next = state_reg;
188   end else begin
189     case (state_reg)
190       STATE_IDLE: begin
191         // idle state - wait for packet
192         reset_crc = 1'b1;
193
194         if (gmii_rx_dv_d4 && !gmii_rx_er_d4 &&
gmii_rxd_d4 == ETH_SFD && cfg_rx_enable) begin
195           state_next = STATE_PAYLOAD;
196         end else begin
197           state_next = STATE_IDLE;
198         end
199       end
200       STATE_PAYLOAD: begin
201         // read payload
202         update_crc = 1'b1;
203
204         m_axis_tdata_next = gmii_rxd_d4;
205         m_axis_tvalid_next = 1'b1;
206
207         if (gmii_rx_dv_d4 && gmii_rx_er_d4) begin
208           // error
209           m_axis_tlast_next = 1'b1;
210           m_axis_tuser_next = 1'b1;
211           error_bad_frame_next = 1'b1;
212           state_next = STATE_WAIT_LAST;
213         end else if (!gmii_rx_dv) begin
214           // end of packet
215           m_axis_tlast_next = 1'b1;
216           if (gmii_rx_er_d0 || gmii_rx_er_d1 ||
gmii_rx_er_d2 || gmii_rx_er_d3) begin
217             // error received in FCS bytes
218             m_axis_tuser_next = 1'b1;
219             error_bad_frame_next = 1'b1;
220           end else if ({gmii_rxd_d0, gmii_rxd_d1,
gmii_rxd_d2, gmii_rxd_d3} == ~crc_next) begin
221             // FCS good
222             m_axis_tuser_next = 1'b0;
223           end else begin
224             // FCS bad
225             m_axis_tuser_next = 1'b1;
226             error_bad_frame_next = 1'b1;
227             error_bad_fcs_next = 1'b1;
228           end
229           state_next = STATE_IDLE;
230         end else begin
231           state_next = STATE_PAYLOAD;
232         end
233       end
234       STATE_WAIT_LAST: begin
235         // wait for end of packet
236
237         if (~gmii_rx_dv) begin
238           state_next = STATE_IDLE;
239         end else begin
240           state_next = STATE_WAIT_LAST;
241         end
242       end
243     endcase
244   end
245 end
246
247 always @(posedge clk) begin
248   state_reg <= state_next;
249
250   m_axis_tdata_reg <= m_axis_tdata_next;
251   m_axis_tvalid_reg <= m_axis_tvalid_next;
252   m_axis_tlast_reg <= m_axis_tlast_next;

```



```

253 m_axis_tuser_reg <= m_axis_tuser_next;
254
255 start_packet_int_reg <= 1'b0;
256 start_packet_reg <= 1'b0;
257
258 if (start_packet_int_reg) begin
259     ptp_ts_reg <= ptp_ts;
260     start_packet_reg <= 1'b1;
261 end
262
263 if (clk_enable) begin
264     if (mii_select) begin
265         mii_odd <= !mii_odd;
266
267         if (in_frame) begin
268             in_frame <= gmii_rx_dv;
269         end else if (gmii_rx_dv && {gmii_rxd[3:0],
gmii_rxd_d0[7:4]} == ETH_SFD) begin
270             in_frame <= 1'b1;
271             start_packet_int_reg <= 1'b1;
272             mii_odd <= 1'b1;
273         end
274
275         gmii_rxd_d0 <= {gmii_rxd[3:0], gmii_rxd_d0[7:4]};
276
277         if (mii_odd) begin
278             gmii_rxd_d1 <= gmii_rxd_d0;
279             gmii_rxd_d2 <= gmii_rxd_d1;
280             gmii_rxd_d3 <= gmii_rxd_d2;
281             gmii_rxd_d4 <= gmii_rxd_d3;
282
283             gmii_rx_dv_d0 <= gmii_rx_dv & gmii_rx_dv_d0;
284             gmii_rx_dv_d1 <= gmii_rx_dv_d0 & gmii_rx_dv;
285             gmii_rx_dv_d2 <= gmii_rx_dv_d1 & gmii_rx_dv;
286             gmii_rx_dv_d3 <= gmii_rx_dv_d2 & gmii_rx_dv;
287             gmii_rx_dv_d4 <= gmii_rx_dv_d3 & gmii_rx_dv;
288
289             gmii_rx_er_d0 <= gmii_rx_er | gmii_rx_er_d0;
290             gmii_rx_er_d1 <= gmii_rx_er_d0;
291             gmii_rx_er_d2 <= gmii_rx_er_d1;
292             gmii_rx_er_d3 <= gmii_rx_er_d2;
293             gmii_rx_er_d4 <= gmii_rx_er_d3;
294         end else begin
295             gmii_rx_dv_d0 <= gmii_rx_dv;
296             gmii_rx_er_d0 <= gmii_rx_er;
297         end
298     end else begin
299         if (in_frame) begin
300             in_frame <= gmii_rx_dv;
301         end else if (gmii_rx_dv && gmii_rxd == ETH_SFD)
begin
302             in_frame <= 1'b1;
303             start_packet_int_reg <= 1'b1;
304         end

```

```

305
306         gmii_rxd_d0 <= gmii_rxd;
307         gmii_rxd_d1 <= gmii_rxd_d0;
308         gmii_rxd_d2 <= gmii_rxd_d1;
309         gmii_rxd_d3 <= gmii_rxd_d2;
310         gmii_rxd_d4 <= gmii_rxd_d3;
311
312         gmii_rx_dv_d0 <= gmii_rx_dv;
313         gmii_rx_dv_d1 <= gmii_rx_dv_d0 & gmii_rx_dv;
314         gmii_rx_dv_d2 <= gmii_rx_dv_d1 & gmii_rx_dv;
315         gmii_rx_dv_d3 <= gmii_rx_dv_d2 & gmii_rx_dv;
316         gmii_rx_dv_d4 <= gmii_rx_dv_d3 & gmii_rx_dv;
317
318         gmii_rx_er_d0 <= gmii_rx_er;
319         gmii_rx_er_d1 <= gmii_rx_er_d0;
320         gmii_rx_er_d2 <= gmii_rx_er_d1;
321         gmii_rx_er_d3 <= gmii_rx_er_d2;
322         gmii_rx_er_d4 <= gmii_rx_er_d3;
323     end
324 end
325
326 if (reset_crc) begin
327     crc_state <= 32'hFFFFFFFF;
328 end else if (update_crc) begin
329     crc_state <= crc_next;
330 end
331
332 error_bad_frame_reg <= error_bad_frame_next;
333 error_bad_fcs_reg <= error_bad_fcs_next;
334
335 if (rst) begin
336     state_reg <= STATE_IDLE;
337
338     m_axis_tvalid_reg <= 1'b0;
339
340     start_packet_int_reg <= 1'b0;
341     start_packet_reg <= 1'b0;
342     error_bad_frame_reg <= 1'b0;
343     error_bad_fcs_reg <= 1'b0;
344
345     in_frame <= 1'b0;
346     mii_odd <= 1'b0;
347
348     gmii_rx_dv_d0 <= 1'b0;
349     gmii_rx_dv_d1 <= 1'b0;
350     gmii_rx_dv_d2 <= 1'b0;
351     gmii_rx_dv_d3 <= 1'b0;
352     gmii_rx_dv_d4 <= 1'b0;
353 end
354 end
355
356 endmodule
357
358 `resetall

```

PCIe 코어 (추가)

What this is. Additional PCIe interface logic: transaction handling and DMA.

Why it exists. Because Chapter P2 describes the layering abstractly and this shows the transaction layer as code. DMA and descriptor handling, in particular, are where the throughput analysis of that chapter becomes concrete.

Undergraduate review · EECS concepts. Layered protocols; credit-based flow control; descriptor rings; DMA engines.

Graduate · Practice note. Trace how outstanding requests are tracked. Chapter P2 argues that tag capacity rather than link width limits read throughput; the tag management logic here is where that limit physically lives.

9 files • 5,198 lines

API — arbiter.v

Table 657. alexforencich_verilog-pcie/rtl/arbiter.v — module / interface / package (1개)

이름	종류
arbiter	module

Table 658. alexforencich_verilog-pcie/rtl/arbiter.v — parameter (5개)

파라미터	형/폭	기본값
PORTS		4
ARB_TYPE_ROUND_ROBIN		0
ARB_BLOCK		0
ARB_BLOCK_ACK		1
ARB_LSB_HIGH_PRIORITY		0

Table 659. alexforencich_verilog-pcie/rtl/arbiter.v — 포트 (7개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
request	input	wire [PORTS-1:0]
acknowledge	input	wire [PORTS-1:0]
grant	output	wire [PORTS-1:0]
grant_valid	output	wire
PORTS	output	wire [\$clog2(

API — axis_arb_mux.v

Table 660. alexforencich_verilog-pcie/rtl/axis_arb_mux.v — module / interface / package (1개)

이름	종류
axis_arb_mux	module

Table 661. alexforencich_verilog-pcie/rtl/axis_arb_mux.v — parameter (17개)

파라미터	형/폭	기본값
S_COUNT		4
DATA_WIDTH		8
KEEP_ENABLE		(DATA_WIDTH>8)
KEEP_WIDTH		((DATA_WIDTH+7)/8)
ID_ENABLE		0
S_ID_WIDTH		8
M_ID_WIDTH		S_ID_WIDTH+\$clog2(S_COUNT)
DEST_ENABLE		0
DEST_WIDTH		8
USER_ENABLE		1
USER_WIDTH		1
LAST_ENABLE		1
UPDATE_TID		0
ARB_TYPE_ROUND_ROBIN		0
ARB_LSB_HIGH_PRIORITY		1
CL_S_COUNT		\$clog2(S_COUNT)
S_ID_WIDTH_INT		S_ID_WIDTH > 0 ? S_ID_WIDTH : 1

Table 662. alexforencich_verilog-pcie/rtl/axis_arb_mux.v — 포트 (18개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_axis_tdata	input	wire [S_COUNT*DATA_WIDTH-1:0]
s_axis_tkeep	input	wire [S_COUNT*KEEP_WIDTH-1:0]
s_axis_tvalid	input	wire [S_COUNT-1:0]
s_axis_tready	output	wire [S_COUNT-1:0]
s_axis_tlast	input	wire [S_COUNT-1:0]
s_axis_tid	input	wire [S_COUNT*S_ID_WIDTH-1:0]
s_axis_tdest	input	wire [S_COUNT*DEST_WIDTH-1:0]
s_axis_tuser	input	wire [S_COUNT*USER_WIDTH-1:0]
m_axis_tdata	output	wire [DATA_WIDTH-1:0]
m_axis_tkeep	output	wire [KEEP_WIDTH-1:0]

포트	방향	형/폭
m_axis_tvalid	output	wire
m_axis_tready	input	wire
m_axis_tlast	output	wire
m_axis_tid	output	wire [M_ID_WIDTH-1:0]
m_axis_tdest	output	wire [DEST_WIDTH-1:0]
m_axis_tuser	output	wire [USER_WIDTH-1:0]

API — dma_client_axis_sink.v

Table 663. alexforencich_verilog-pcie/rtl/dma_client_axis_sink.v — module / interface / package (1개)

이름	종류
dma_client_axis_sink	module

Table 664. alexforencich_verilog-pcie/rtl/dma_client_axis_sink.v — parameter (33개)

파라미터	형/폭	기본값
RAM_ADDR_WIDTH		16
SEG_COUNT		2
SEG_DATA_WIDTH		64
SEG_BE_WIDTH		SEG_DATA_WIDTH/8
SEG_ADDR_WIDTH		RAM_ADDR_WIDTH-\$clog2(SEG_COUNT*SEG_BE_WIDTH)
AXIS_DATA_WIDTH		SEG_DATA_WIDTH*SEG_COUNT/2
AXIS_KEEP_ENABLE		(AXIS_DATA_WIDTH>8)
AXIS_KEEP_WIDTH		(AXIS_DATA_WIDTH/8)
AXIS_LAST_ENABLE		1
AXIS_ID_ENABLE		0
AXIS_ID_WIDTH		8
AXIS_DEST_ENABLE		0
AXIS_DEST_WIDTH		8
AXIS_USER_ENABLE		1
AXIS_USER_WIDTH		1
LEN_WIDTH		16
TAG_WIDTH		8
RAM_WORD_WIDTH		SEG_BE_WIDTH
RAM_WORD_SIZE		SEG_DATA_WIDTH/RAM_WORD_WIDTH
AXIS_KEEP_WIDTH_INT		AXIS_KEEP_ENABLE ? AXIS_KEEP_WIDTH : 1
AXIS_WORD_WIDTH		AXIS_KEEP_WIDTH_INT
AXIS_WORD_SIZE		AXIS_DATA_WIDTH/AXIS_WORD_WIDTH

파라미터	형/폭	기본값
PART_COUNT		SEG_COUNT*SEG_BE_WIDTH / AXIS_KEEP_WIDTH_INT
PART_COUNT_WIDTH		PART_COUNT > 1 ? \$clog2(PART_COUNT) : 1
PART_OFFSET_WIDTH		AXIS_KEEP_WIDTH_INT > 1 ? \$clog2(AXIS_KEEP_WIDTH_INT) : 1
PARTS_PER_SEG		(SEG_BE_WIDTH + AXIS_KEEP_WIDTH_INT - 1) / AXIS_KEEP_WIDTH_I...
SEGS_PER_PART		(AXIS_KEEP_WIDTH_INT + SEG_BE_WIDTH - 1) / SEG_BE_WIDTH
OFFSET_WIDTH		AXIS_KEEP_WIDTH_INT > 1 ? \$clog2(AXIS_KEEP_WIDTH_INT) : 1
OFFSET_MASK		AXIS_KEEP_WIDTH_INT > 1 ? {OFFSET_WIDTH{1'b1}} : 0
ADDR_MASK		{RAM_ADDR_WIDTH{1'b1}} << \$clog2(AXIS_KEEP_WIDTH_INT)
CYCLE_COUNT_WIDTH		LEN_WIDTH - \$clog2(AXIS_KEEP_WIDTH_INT) + 1
STATUS_FIFO_ADDR_WIDTH		5
OUTPUT_FIFO_ADDR_WIDTH		5

Table 665. alexforenchich_verilog-pcie/rtl/dma_client_axis_sink.v — localparam (1개)

상수	형/폭	값
STATE_IDLE	[1:0]	2'd0

Table 666. alexforenchich_verilog-pcie/rtl/dma_client_axis_sink.v — 포트 (30개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_axis_write_desc_ram_addr	input	wire [RAM_ADDR_WIDTH-1:0]
s_axis_write_desc_len	input	wire [LEN_WIDTH-1:0]
s_axis_write_desc_tag	input	wire [TAG_WIDTH-1:0]
s_axis_write_desc_valid	input	wire
s_axis_write_desc_ready	output	wire
m_axis_write_desc_status_len	output	wire [LEN_WIDTH-1:0]
m_axis_write_desc_status_tag	output	wire [TAG_WIDTH-1:0]
m_axis_write_desc_status_id	output	wire [AXIS_ID_WIDTH-1:0]
m_axis_write_desc_status_dest	output	wire [AXIS_DEST_WIDTH-1:0]
m_axis_write_desc_status_user	output	wire [AXIS_USER_WIDTH-1:0]
m_axis_write_desc_status_error	output	wire [3:0]
m_axis_write_desc_status_valid	output	wire
s_axis_write_data_tdata	input	wire [AXIS_DATA_WIDTH-1:0]
s_axis_write_data_tkeep	input	wire [AXIS_KEEP_WIDTH-1:0]
s_axis_write_data_tvalid	input	wire
s_axis_write_data_tready	output	wire

포트	방향	형/폭
s_axis_write_data_tlast	input	wire
s_axis_write_data_tid	input	wire [AXIS_ID_WIDTH-1:0]
s_axis_write_data_tdest	input	wire [AXIS_DEST_WIDTH-1:0]
s_axis_write_data_tuser	input	wire [AXIS_USER_WIDTH-1:0]
ram_wr_cmd_be	output	wire [SEG_COUNT*SEG_BE_WIDTH-1:0]
ram_wr_cmd_addr	output	wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0]
ram_wr_cmd_data	output	wire [SEG_COUNT*SEG_DATA_WIDTH-1:0]
ram_wr_cmd_valid	output	wire [SEG_COUNT-1:0]
ram_wr_cmd_ready	input	wire [SEG_COUNT-1:0]
ram_wr_done	input	wire [SEG_COUNT-1:0]
enable	input	wire
abort	input	wire

API — dma_client_axis_source.v

Table 667. alexforencich_verilog-pcie/rtl/dma_client_axis_source.v — module / interface / package (1개)

이름	종류
dma_client_axis_source	module

Table 668. alexforencich_verilog-pcie/rtl/dma_client_axis_source.v — parameter (32개)

파라미터	형/폭	기본값
RAM_ADDR_WIDTH		16
SEG_COUNT		2
SEG_DATA_WIDTH		64
SEG_BE_WIDTH		SEG_DATA_WIDTH/8
SEG_ADDR_WIDTH		RAM_ADDR_WIDTH-\$clog2(SEG_COUNT*SEG_BE_WIDTH)
AXIS_DATA_WIDTH		SEG_DATA_WIDTH*SEG_COUNT/2
AXIS_KEEP_ENABLE		(AXIS_DATA_WIDTH>8)
AXIS_KEEP_WIDTH		(AXIS_DATA_WIDTH/8)
AXIS_LAST_ENABLE		1
AXIS_ID_ENABLE		0
AXIS_ID_WIDTH		8
AXIS_DEST_ENABLE		0
AXIS_DEST_WIDTH		8
AXIS_USER_ENABLE		1
AXIS_USER_WIDTH		1
LEN_WIDTH		16

파라미터	형/폭	기본값
TAG_WIDTH		8
RAM_WORD_WIDTH		SEG_BE_WIDTH
RAM_WORD_SIZE		SEG_DATA_WIDTH/RAM_WORD_WIDTH
AXIS_KEEP_WIDTH_INT		AXIS_KEEP_ENABLE ? AXIS_KEEP_WIDTH : 1
AXIS_WORD_WIDTH		AXIS_KEEP_WIDTH_INT
AXIS_WORD_SIZE		AXIS_DATA_WIDTH/AXIS_WORD_WIDTH
PART_COUNT		SEG_COUNT*SEG_BE_WIDTH / AXIS_KEEP_WIDTH_INT
PART_COUNT_WIDTH		PART_COUNT > 1 ? \$clog2(PART_COUNT) : 1
PART_OFFSET_WIDTH		AXIS_KEEP_WIDTH_INT > 1 ? \$clog2(AXIS_KEEP_WIDTH_INT) : 1
PARTS_PER_SEG		(SEG_BE_WIDTH + AXIS_KEEP_WIDTH_INT - 1) / AXIS_KEEP_WIDTH_I...
SEGS_PER_PART		(AXIS_KEEP_WIDTH_INT + SEG_BE_WIDTH - 1) / SEG_BE_WIDTH
OFFSET_WIDTH		AXIS_KEEP_WIDTH_INT > 1 ? \$clog2(AXIS_KEEP_WIDTH_INT) : 1
OFFSET_MASK		AXIS_KEEP_WIDTH_INT > 1 ? {OFFSET_WIDTH{1'b1}} : 0
ADDR_MASK		{RAM_ADDR_WIDTH{1'b1}} << \$clog2(AXIS_KEEP_WIDTH_INT)
CYCLE_COUNT_WIDTH		LEN_WIDTH - \$clog2(AXIS_KEEP_WIDTH_INT) + 1
OUTPUT_FIFO_ADDR_WIDTH		5

Table 669. alexforencich_verilog-pcie/rtl/dma_client_axis_source.v — localparam (2개)

상수	형/폭	값
READ_STATE_IDLE	[0:0]	1'd0
AXIS_STATE_IDLE	[0:0]	1'd0

Table 670. alexforencich_verilog-pcie/rtl/dma_client_axis_source.v — 포트 (28개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_axis_read_desc_ram_addr	input	wire [RAM_ADDR_WIDTH-1:0]
s_axis_read_desc_len	input	wire [LEN_WIDTH-1:0]
s_axis_read_desc_tag	input	wire [TAG_WIDTH-1:0]
s_axis_read_desc_id	input	wire [AXIS_ID_WIDTH-1:0]
s_axis_read_desc_dest	input	wire [AXIS_DEST_WIDTH-1:0]
s_axis_read_desc_user	input	wire [AXIS_USER_WIDTH-1:0]
s_axis_read_desc_valid	input	wire
s_axis_read_desc_ready	output	wire
m_axis_read_desc_status_tag	output	wire [TAG_WIDTH-1:0]
m_axis_read_desc_status_error	output	wire [3:0]

포트	방향	형/폭
m_axis_read_desc_status_valid	output	wire
m_axis_read_data_tdata	output	wire [AXIS_DATA_WIDTH-1:0]
m_axis_read_data_tkeep	output	wire [AXIS_KEEP_WIDTH-1:0]
m_axis_read_data_tvalid	output	wire
m_axis_read_data_tready	input	wire
m_axis_read_data_tlast	output	wire
m_axis_read_data_tid	output	wire [AXIS_ID_WIDTH-1:0]
m_axis_read_data_tdest	output	wire [AXIS_DEST_WIDTH-1:0]
m_axis_read_data_tuser	output	wire [AXIS_USER_WIDTH-1:0]
ram_rd_cmd_addr	output	wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0]
ram_rd_cmd_valid	output	wire [SEG_COUNT-1:0]
ram_rd_cmd_ready	input	wire [SEG_COUNT-1:0]
ram_rd_resp_data	input	wire [SEG_COUNT*SEG_DATA_WIDTH-1:0]
ram_rd_resp_valid	input	wire [SEG_COUNT-1:0]
ram_rd_resp_ready	output	wire [SEG_COUNT-1:0]
enable	input	wire

API — dma_if_axi.v

Table 671. alexforenrich_verilog-pcie/rtl/dma_if_axi.v — module / interface / package (17개)

이름	종류
dma_if_axi	module

Table 672. alexforenrich_verilog-pcie/rtl/dma_if_axi.v — parameter (19개)

파라미터	형/폭	기본값
AXI_DATA_WIDTH		32
AXI_ADDR_WIDTH		16
AXI_STRB_WIDTH		(AXI_DATA_WIDTH/8)
AXI_ID_WIDTH		8
AXI_MAX_BURST_LEN		256
RAM_SEL_WIDTH		2
RAM_ADDR_WIDTH		16
RAM_SEG_COUNT		2
RAM_SEG_DATA_WIDTH		AXI_DATA_WIDTH*2/RAM_SEG_COUNT
RAM_SEG_BE_WIDTH		RAM_SEG_DATA_WIDTH/8
RAM_SEG_ADDR_WIDTH		RAM_ADDR_WIDTH-\$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH)
IMM_ENABLE		0

파라미터	형/폭	기본값
IMM_WIDTH		32
LEN_WIDTH		16
TAG_WIDTH		8
READ_OP_TABLE_SIZE		2**(AXI_ID_WIDTH)
WRITE_OP_TABLE_SIZE		2**(AXI_ID_WIDTH)
READ_USE_AXI_ID		0
WRITE_USE_AXI_ID		1

Table 673. alexforencich_verilog-pcie/rtl/dma_if_axi.v — 포트 (105개) — 앞 60개만 표시(전체 105개, 본문 소스 참조)

포트	방향	형/폭
clk	input	wire
rst	input	wire
m_axi_awid	output	wire [AXI_ID_WIDTH-1:0]
m_axi_awaddr	output	wire [AXI_ADDR_WIDTH-1:0]
m_axi_awlen	output	wire [7:0]
m_axi_awsz	output	wire [2:0]
m_axi_awburst	output	wire [1:0]
m_axi_awlock	output	wire
m_axi_awcache	output	wire [3:0]
m_axi_awprot	output	wire [2:0]
m_axi_awvalid	output	wire
m_axi_awready	input	wire
m_axi_wdata	output	wire [AXI_DATA_WIDTH-1:0]
m_axi_wstrb	output	wire [AXI_STRB_WIDTH-1:0]
m_axi_wlast	output	wire
m_axi_wvalid	output	wire
m_axi_wready	input	wire
m_axi_bid	input	wire [AXI_ID_WIDTH-1:0]
m_axi_bresp	input	wire [1:0]
m_axi_bvalid	input	wire
m_axi_bready	output	wire
m_axi_arid	output	wire [AXI_ID_WIDTH-1:0]
m_axi_araddr	output	wire [AXI_ADDR_WIDTH-1:0]
m_axi_arlen	output	wire [7:0]

포트	방향	형/폭
m_axi_arsize	output	wire [2:0]
m_axi_arburst	output	wire [1:0]
m_axi_arlock	output	wire
m_axi_arcache	output	wire [3:0]
m_axi_arprot	output	wire [2:0]
m_axi_arvalid	output	wire
m_axi_arready	input	wire
m_axi_rid	input	wire [AXI_ID_WIDTH-1:0]
m_axi_rdata	input	wire [AXI_DATA_WIDTH-1:0]
m_axi_rresp	input	wire [1:0]
m_axi_rlast	input	wire
m_axi_rvalid	input	wire
m_axi_rready	output	wire
s_axis_read_desc_axi_addr	input	wire [AXI_ADDR_WIDTH-1:0]
s_axis_read_desc_ram_sel	input	wire [RAM_SEL_WIDTH-1:0]
s_axis_read_desc_ram_addr	input	wire [RAM_ADDR_WIDTH-1:0]
s_axis_read_desc_len	input	wire [LEN_WIDTH-1:0]
s_axis_read_desc_tag	input	wire [TAG_WIDTH-1:0]
s_axis_read_desc_valid	input	wire
s_axis_read_desc_ready	output	wire
m_axis_read_desc_status_tag	output	wire [TAG_WIDTH-1:0]
m_axis_read_desc_status_error	output	wire [3:0]
m_axis_read_desc_status_valid	output	wire
s_axis_write_desc_axi_addr	input	wire [AXI_ADDR_WIDTH-1:0]
s_axis_write_desc_ram_sel	input	wire [RAM_SEL_WIDTH-1:0]
s_axis_write_desc_ram_addr	input	wire [RAM_ADDR_WIDTH-1:0]
s_axis_write_desc_imm	input	wire [IMM_WIDTH-1:0]
s_axis_write_desc_imm_en	input	wire
s_axis_write_desc_len	input	wire [LEN_WIDTH-1:0]
s_axis_write_desc_tag	input	wire [TAG_WIDTH-1:0]
s_axis_write_desc_valid	input	wire
s_axis_write_desc_ready	output	wire
m_axis_write_desc_status_tag	output	wire [TAG_WIDTH-1:0]
m_axis_write_desc_status_error	output	wire [3:0]
m_axis_write_desc_status_valid	output	wire

포트	방향	형/폭
ram_wr_cmd_sel	output	wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]

API — dma_if_axi_rd.v

Table 674. alexforencich_verilog-pcie/rtl/dma_if_axi_rd.v — module / interface / package (1개)

이름	종류
dma_if_axi_rd	module

Table 675. alexforencich_verilog-pcie/rtl/dma_if_axi_rd.v — parameter (32개)

파라미터	형/폭	기본값
AXI_DATA_WIDTH		32
AXI_ADDR_WIDTH		16
AXI_STRB_WIDTH		(AXI_DATA_WIDTH/8)
AXI_ID_WIDTH		8
AXI_MAX_BURST_LEN		256
RAM_SEL_WIDTH		2
RAM_ADDR_WIDTH		16
RAM_SEG_COUNT		2
RAM_SEG_DATA_WIDTH		AXI_DATA_WIDTH*2/RAM_SEG_COUNT
RAM_SEG_BE_WIDTH		RAM_SEG_DATA_WIDTH/8
RAM_SEG_ADDR_WIDTH		RAM_ADDR_WIDTH-\$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH)
LEN_WIDTH		16
TAG_WIDTH		8
OP_TABLE_SIZE		2**(AXI_ID_WIDTH)
USE_AXI_ID		0
RAM_DATA_WIDTH		RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH
RAM_WORD_WIDTH		RAM_SEG_BE_WIDTH
RAM_WORD_SIZE		RAM_SEG_DATA_WIDTH/RAM_WORD_WIDTH
AXI_WORD_WIDTH		AXI_STRB_WIDTH
AXI_WORD_SIZE		AXI_DATA_WIDTH/AXI_WORD_WIDTH
AXI_BURST_SIZE		\$clog2(AXI_STRB_WIDTH)
AXI_MAX_BURST_SIZE		AXI_MAX_BURST_LEN << AXI_BURST_SIZE
OFFSET_WIDTH		AXI_STRB_WIDTH > 1 ? \$clog2(AXI_STRB_WIDTH) : 1
OFFSET_MASK		AXI_STRB_WIDTH > 1 ? {OFFSET_WIDTH{1'b1}} : 0
RAM_OFFSET_WIDTH		\$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH)
ADDR_MASK		{AXI_ADDR_WIDTH{1'b1}} << \$clog2(AXI_STRB_WIDTH)
CYCLE_COUNT_WIDTH		LEN_WIDTH - AXI_BURST_SIZE + 1

파라미터	형/폭	기본값
OP_TAG_WIDTH		$\$clog2(OP_TABLE_SIZE)$
OP_TABLE_READ_COUNT_WIDTH		AXI_ID_WIDTH+1
OP_TABLE_WRITE_COUNT_WIDTH		LEN_WIDTH
STATUS_FIFO_ADDR_WIDTH		5
OUTPUT_FIFO_ADDR_WIDTH		5

Table 676. alexforencich_verilog-pcie/rtl/dma_if_axi_rd.v — localparam (4개)

상수	형/폭	값
AXI_RESP_OKAY	[1:0]	2'b00
DMA_ERROR_NONE	[3:0]	4'd0
REQ_STATE_IDLE	[0:0]	1'd0
AXI_STATE_IDLE	[0:0]	1'd0

Table 677. alexforencich_verilog-pcie/rtl/dma_if_axi_rd.v — 포트 (51개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
m_axi_arid	output	wire [AXI_ID_WIDTH-1:0]
m_axi_araddr	output	wire [AXI_ADDR_WIDTH-1:0]
m_axi_arlen	output	wire [7:0]
m_axi_arsize	output	wire [2:0]
m_axi_arburst	output	wire [1:0]
m_axi_arlock	output	wire
m_axi_arcache	output	wire [3:0]
m_axi_arprot	output	wire [2:0]
m_axi_arvalid	output	wire
m_axi_arready	input	wire
m_axi_rid	input	wire [AXI_ID_WIDTH-1:0]
m_axi_rdata	input	wire [AXI_DATA_WIDTH-1:0]
m_axi_rresp	input	wire [1:0]
m_axi_rlast	input	wire
m_axi_rvalid	input	wire
m_axi_rready	output	wire
s_axis_read_desc_axi_addr	input	wire [AXI_ADDR_WIDTH-1:0]
s_axis_read_desc_ram_sel	input	wire [RAM_SEL_WIDTH-1:0]
s_axis_read_desc_ram_addr	input	wire [RAM_ADDR_WIDTH-1:0]

포트	방향	형/폭
s_axis_read_desc_len	input	wire [LEN_WIDTH-1:0]
s_axis_read_desc_tag	input	wire [TAG_WIDTH-1:0]
s_axis_read_desc_valid	input	wire
s_axis_read_desc_ready	output	wire
m_axis_read_desc_status_tag	output	wire [TAG_WIDTH-1:0]
m_axis_read_desc_status_error	output	wire [3:0]
m_axis_read_desc_status_valid	output	wire
ram_wr_cmd_sel	output	wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]
ram_wr_cmd_be	output	wire [RAM_SEG_COUNT*RAM_SEG_BE_WIDTH-1:0]
ram_wr_cmd_addr	output	wire [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0]
ram_wr_cmd_data	output	wire [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0]
ram_wr_cmd_valid	output	wire [RAM_SEG_COUNT-1:0]
ram_wr_cmd_ready	input	wire [RAM_SEG_COUNT-1:0]
ram_wr_done	input	wire [RAM_SEG_COUNT-1:0]
enable	input	wire
status_busy	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_rd_op_start_len	output	wire [LEN_WIDTH-1:0]
stat_rd_op_start_valid	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_rd_op_finish_status	output	wire [3:0]
stat_rd_op_finish_valid	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_rd_req_start_len	output	wire [12:0]
stat_rd_req_start_valid	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_rd_req_finish_status	output	wire [3:0]
stat_rd_req_finish_valid	output	wire
stat_rd_op_table_full	output	wire
stat_rd_tx_stall	output	wire

API — dma_if_axi_wr.v

Table 678. alexforencich_verilog-pcie/rtl/dma_if_axi_wr.v — module / interface / package (1개)

이름	종류
dma_if_axi_wr	module

Table 679. alexforencich_verilog-pcie/rtl/dma_if_axi_wr.v — parameter (32개)

파라미터	형/폭	기본값
AXI_DATA_WIDTH		32
AXI_ADDR_WIDTH		16
AXI_STRB_WIDTH		(AXI_DATA_WIDTH/8)
AXI_ID_WIDTH		8
AXI_MAX_BURST_LEN		256
RAM_SEL_WIDTH		2
RAM_ADDR_WIDTH		16
RAM_SEG_COUNT		2
RAM_SEG_DATA_WIDTH		AXI_DATA_WIDTH*2/RAM_SEG_COUNT
RAM_SEG_BE_WIDTH		RAM_SEG_DATA_WIDTH/8
RAM_SEG_ADDR_WIDTH		RAM_ADDR_WIDTH-\$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH)
IMM_ENABLE		0
IMM_WIDTH		32
LEN_WIDTH		16
TAG_WIDTH		8
OP_TABLE_SIZE		2**(AXI_ID_WIDTH)
USE_AXI_ID		1
RAM_DATA_WIDTH		RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH
RAM_WORD_WIDTH		RAM_SEG_BE_WIDTH
RAM_WORD_SIZE		RAM_SEG_DATA_WIDTH/RAM_WORD_WIDTH
AXI_WORD_WIDTH		AXI_STRB_WIDTH
AXI_WORD_SIZE		AXI_DATA_WIDTH/AXI_WORD_WIDTH
AXI_BURST_SIZE		\$clog2(AXI_STRB_WIDTH)
AXI_MAX_BURST_SIZE		AXI_MAX_BURST_LEN << AXI_BURST_SIZE
OFFSET_WIDTH		AXI_STRB_WIDTH > 1 ? \$clog2(AXI_STRB_WIDTH) : 1
OFFSET_MASK		AXI_STRB_WIDTH > 1 ? {OFFSET_WIDTH{1'b1}} : 0
RAM_OFFSET_WIDTH		\$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH)
ADDR_MASK		{AXI_ADDR_WIDTH{1'b1}} << \$clog2(AXI_STRB_WIDTH)
CYCLE_COUNT_WIDTH		LEN_WIDTH - AXI_BURST_SIZE + 1
MASK_FIFO_ADDR_WIDTH		\$clog2(OP_TABLE_SIZE)+1
OP_TAG_WIDTH		\$clog2(OP_TABLE_SIZE)
OUTPUT_FIFO_ADDR_WIDTH		5

Table 680. alexforencich_verilog-pcie/rtl/dma_if_axi_wr.v — localparam (5개)

상수	형/폭	값
AXI_RESP_OKAY	[1:0]	2'b00
DMA_ERROR_NONE	[3:0]	4'd0
REQ_STATE_IDLE	[0:0]	1'd0
READ_STATE_IDLE	[0:0]	1'd0
AXI_STATE_IDLE	[0:0]	1'd0

Table 681. alexforencich_verilog-pcie/rtl/dma_if_axi_wr.v — 포트 (56개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
m_axi_awid	output	wire [AXI_ID_WIDTH-1:0]
m_axi_awaddr	output	wire [AXI_ADDR_WIDTH-1:0]
m_axi_awlen	output	wire [7:0]
m_axi_awsz	output	wire [2:0]
m_axi_awburst	output	wire [1:0]
m_axi_awlock	output	wire
m_axi_awcache	output	wire [3:0]
m_axi_awprot	output	wire [2:0]
m_axi_awvalid	output	wire
m_axi_awready	input	wire
m_axi_wdata	output	wire [AXI_DATA_WIDTH-1:0]
m_axi_wstrb	output	wire [AXI_STRB_WIDTH-1:0]
m_axi_wlast	output	wire
m_axi_wvalid	output	wire
m_axi_wready	input	wire
m_axi_bid	input	wire [AXI_ID_WIDTH-1:0]
m_axi_bresp	input	wire [1:0]
m_axi_bvalid	input	wire
m_axi_bready	output	wire
s_axis_write_desc_axi_addr	input	wire [AXI_ADDR_WIDTH-1:0]
s_axis_write_desc_ram_sel	input	wire [RAM_SEL_WIDTH-1:0]
s_axis_write_desc_ram_addr	input	wire [RAM_ADDR_WIDTH-1:0]
s_axis_write_desc_imm	input	wire [IMM_WIDTH-1:0]
s_axis_write_desc_imm_en	input	wire
s_axis_write_desc_len	input	wire [LEN_WIDTH-1:0]

포트	방향	형/폭
s_axis_write_desc_tag	input	wire [TAG_WIDTH-1:0]
s_axis_write_desc_valid	input	wire
s_axis_write_desc_ready	output	wire
m_axis_write_desc_status_tag	output	wire [TAG_WIDTH-1:0]
m_axis_write_desc_status_error	output	wire [3:0]
m_axis_write_desc_status_valid	output	wire
ram_rd_cmd_sel	output	wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]
ram_rd_cmd_addr	output	wire [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0]
ram_rd_cmd_valid	output	wire [RAM_SEG_COUNT-1:0]
ram_rd_cmd_ready	input	wire [RAM_SEG_COUNT-1:0]
ram_rd_resp_data	input	wire [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0]
ram_rd_resp_valid	input	wire [RAM_SEG_COUNT-1:0]
ram_rd_resp_ready	output	wire [RAM_SEG_COUNT-1:0]
enable	input	wire
status_busy	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_wr_op_start_len	output	wire [LEN_WIDTH-1:0]
stat_wr_op_start_valid	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_wr_op_finish_status	output	wire [3:0]
stat_wr_op_finish_valid	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_wr_req_start_len	output	wire [12:0]
stat_wr_req_start_valid	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_wr_req_finish_status	output	wire [3:0]
stat_wr_req_finish_valid	output	wire
stat_wr_op_table_full	output	wire
stat_wr_tx_stall	output	wire

API — dma_if_desc_mux.v

Table 682. alexforencich_verilog-pcie/rtl/dma_if_desc_mux.v — module / interface / package (1개)

이름	종류
dma_if_desc_mux	module

Table 683. alexforencich_verilog-pcie/rtl/dma_if_desc_mux.v — parameter (16개)

파라미터	형/폭	기본값
PORTS		2
EXTEND_RAM_SEL		0
S_RAM_SEL_WIDTH		2
M_RAM_SEL_WIDTH		S_RAM_SEL_WIDTH+(EXTEND_RAM_SEL ? \$clog2(PORTS) : 0)
RAM_ADDR_WIDTH		16
DMA_ADDR_WIDTH		64
IMM_ENABLE		0
IMM_WIDTH		32
LEN_WIDTH		16
S_TAG_WIDTH		8
M_TAG_WIDTH		S_TAG_WIDTH+\$clog2(PORTS)
ARB_TYPE_ROUND_ROBIN		0
ARB_LSB_HIGH_PRIORITY		1
CL_PORTS		\$clog2(PORTS)
S_RAM_SEL_WIDTH_INT		S_RAM_SEL_WIDTH > 0 ? S_RAM_SEL_WIDTH : 1
FIFO_ADDR_WIDTH		5

Table 684. alexforencich_verilog-pcie/rtl/dma_if_desc_mux.v — 포트 (26개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
m_axis_desc_dma_addr	output	wire [DMA_ADDR_WIDTH-1:0]
m_axis_desc_ram_sel	output	wire [M_RAM_SEL_WIDTH-1:0]
m_axis_desc_ram_addr	output	wire [RAM_ADDR_WIDTH-1:0]
m_axis_desc_imm	output	wire [IMM_WIDTH-1:0]
m_axis_desc_imm_en	output	wire
m_axis_desc_len	output	wire [LEN_WIDTH-1:0]
m_axis_desc_tag	output	wire [M_TAG_WIDTH-1:0]
m_axis_desc_valid	output	wire
m_axis_desc_ready	input	wire
s_axis_desc_status_tag	input	wire [M_TAG_WIDTH-1:0]
s_axis_desc_status_error	input	wire [3:0]
s_axis_desc_status_valid	input	wire
s_axis_desc_dma_addr	input	wire [PORTS*DMA_ADDR_WIDTH-1:0]
s_axis_desc_ram_sel	input	wire [PORTS*S_RAM_SEL_WIDTH-1:0]

포트	방향	형/폭
s_axis_desc_ram_addr	input	wire [PORTS*RAM_ADDR_WIDTH-1:0]
s_axis_desc_imm	input	wire [PORTS*IMM_WIDTH-1:0]
s_axis_desc_imm_en	input	wire [PORTS-1:0]
s_axis_desc_len	input	wire [PORTS*LEN_WIDTH-1:0]
s_axis_desc_tag	input	wire [PORTS*S_TAG_WIDTH-1:0]
s_axis_desc_valid	input	wire [PORTS-1:0]
s_axis_desc_ready	output	wire [PORTS-1:0]
m_axis_desc_status_tag	output	wire [PORTS*S_TAG_WIDTH-1:0]
m_axis_desc_status_error	output	wire [PORTS*4-1:0]
m_axis_desc_status_valid	output	wire [PORTS-1:0]

API — dma_if_mux.v

Table 685. alexforencich_verilog-pcie/rtl/dma_if_mux.v — module / interface / package (1개)

이름	종류
dma_if_mux	module

Table 686. alexforencich_verilog-pcie/rtl/dma_if_mux.v — parameter (16개)

파라미터	형/폭	기본값
PORTS		2
SEG_COUNT		2
SEG_DATA_WIDTH		64
SEG_ADDR_WIDTH		8
SEG_BE_WIDTH		SEG_DATA_WIDTH/8
S_RAM_SEL_WIDTH		2
M_RAM_SEL_WIDTH		S_RAM_SEL_WIDTH+\$clog2(PORTS)
RAM_ADDR_WIDTH		SEG_ADDR_WIDTH+\$clog2(SEG_COUNT)+\$clog2(SEG_BE_WIDTH)
DMA_ADDR_WIDTH		64
IMM_ENABLE		0
IMM_WIDTH		32
LEN_WIDTH		16
S_TAG_WIDTH		8
M_TAG_WIDTH		S_TAG_WIDTH+\$clog2(PORTS)
ARB_TYPE_ROUND_ROBIN		0
ARB_LSB_HIGH_PRIORITY		1

Table 687. alexforencich_verilog-pcie/rtl/dma_if_mux.v — 포트 (74개) — 앞 60개만 표시(전체 74개, 본문 소스 참조)

포트	방향	형/폭
clk	input	wire
rst	input	wire
m_axis_read_desc_dma_addr	output	wire [DMA_ADDR_WIDTH-1:0]
m_axis_read_desc_ram_sel	output	wire [M_RAM_SEL_WIDTH-1:0]
m_axis_read_desc_ram_addr	output	wire [RAM_ADDR_WIDTH-1:0]
m_axis_read_desc_len	output	wire [LEN_WIDTH-1:0]
m_axis_read_desc_tag	output	wire [M_TAG_WIDTH-1:0]
m_axis_read_desc_valid	output	wire
m_axis_read_desc_ready	input	wire
s_axis_read_desc_status_tag	input	wire [M_TAG_WIDTH-1:0]
s_axis_read_desc_status_error	input	wire [3:0]
s_axis_read_desc_status_valid	input	wire
m_axis_write_desc_dma_addr	output	wire [DMA_ADDR_WIDTH-1:0]
m_axis_write_desc_ram_sel	output	wire [M_RAM_SEL_WIDTH-1:0]
m_axis_write_desc_ram_addr	output	wire [RAM_ADDR_WIDTH-1:0]
m_axis_write_desc_imm	output	wire [IMM_WIDTH-1:0]
m_axis_write_desc_imm_en	output	wire
m_axis_write_desc_len	output	wire [LEN_WIDTH-1:0]
m_axis_write_desc_tag	output	wire [M_TAG_WIDTH-1:0]
m_axis_write_desc_valid	output	wire
m_axis_write_desc_ready	input	wire
s_axis_write_desc_status_tag	input	wire [M_TAG_WIDTH-1:0]
s_axis_write_desc_status_error	input	wire [3:0]
s_axis_write_desc_status_valid	input	wire
s_axis_read_desc_dma_addr	input	wire [PORTS*DMA_ADDR_WIDTH-1:0]
s_axis_read_desc_ram_sel	input	wire [PORTS*S_RAM_SEL_WIDTH-1:0]
s_axis_read_desc_ram_addr	input	wire [PORTS*RAM_ADDR_WIDTH-1:0]
s_axis_read_desc_len	input	wire [PORTS*LEN_WIDTH-1:0]
s_axis_read_desc_tag	input	wire [PORTS*S_TAG_WIDTH-1:0]
s_axis_read_desc_valid	input	wire [PORTS-1:0]
s_axis_read_desc_ready	output	wire [PORTS-1:0]
m_axis_read_desc_status_tag	output	wire [PORTS*S_TAG_WIDTH-1:0]
m_axis_read_desc_status_error	output	wire [PORTS*4-1:0]
m_axis_read_desc_status_valid	output	wire [PORTS-1:0]
s_axis_write_desc_dma_addr	input	wire [PORTS*DMA_ADDR_WIDTH-1:0]

포트	방향	형/폭
s_axis_write_desc_ram_sel	input	wire [PORTS*S_RAM_SEL_WIDTH-1:0]
s_axis_write_desc_ram_addr	input	wire [PORTS*RAM_ADDR_WIDTH-1:0]
s_axis_write_desc_imm	input	wire [PORTS*IMM_WIDTH-1:0]
s_axis_write_desc_imm_en	input	wire [PORTS-1:0]
s_axis_write_desc_len	input	wire [PORTS*LEN_WIDTH-1:0]
s_axis_write_desc_tag	input	wire [PORTS*S_TAG_WIDTH-1:0]
s_axis_write_desc_valid	input	wire [PORTS-1:0]
s_axis_write_desc_ready	output	wire [PORTS-1:0]
m_axis_write_desc_status_tag	output	wire [PORTS*S_TAG_WIDTH-1:0]
m_axis_write_desc_status_error	output	wire [PORTS*4-1:0]
m_axis_write_desc_status_valid	output	wire [PORTS-1:0]
if_ram_wr_cmd_sel	input	wire [SEG_COUNT*M_RAM_SEL_WIDTH-1:0]
if_ram_wr_cmd_be	input	wire [SEG_COUNT*SEG_BE_WIDTH-1:0]
if_ram_wr_cmd_addr	input	wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0]
if_ram_wr_cmd_data	input	wire [SEG_COUNT*SEG_DATA_WIDTH-1:0]
if_ram_wr_cmd_valid	input	wire [SEG_COUNT-1:0]
if_ram_wr_cmd_ready	output	wire [SEG_COUNT-1:0]
if_ram_wr_done	output	wire [SEG_COUNT-1:0]
if_ram_rd_cmd_sel	input	wire [SEG_COUNT*M_RAM_SEL_WIDTH-1:0]
if_ram_rd_cmd_addr	input	wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0]
if_ram_rd_cmd_valid	input	wire [SEG_COUNT-1:0]
if_ram_rd_cmd_ready	output	wire [SEG_COUNT-1:0]
if_ram_rd_resp_data	output	wire [SEG_COUNT*SEG_DATA_WIDTH-1:0]
if_ram_rd_resp_valid	output	wire [SEG_COUNT-1:0]
if_ram_rd_resp_ready	input	wire [SEG_COUNT-1:0]

1. alexforenchich_verilog-pcie/rtl/arbiter.v (159 lines)

```

1 /*
2
3 Copyright (c) 2014-2021 Alex Forenchich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR

```

```

16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * Arbiter module
33 */
34 module arbiter #
35 (
36     parameter PORTS = 4,

```

```

37 // select round robin arbitration
38 parameter ARB_TYPE_ROUND_ROBIN = 0,
39 // blocking arbiter enable
40 parameter ARB_BLOCK = 0,
41 // block on acknowledge assert when nonzero, request deas-
sert when 0
42 parameter ARB_BLOCK_ACK = 1,
43 // LSB priority selection
44 parameter ARB_LSB_HIGH_PRIORITY = 0
45 )
46 (
47     input wire clk,
48     input wire rst,
49     input wire [PORTS-1:0] request,
50     input wire [PORTS-1:0] acknowledge,
51     output wire [PORTS-1:0] grant,
52     output wire grant_valid,
53     output wire [$clog2(PORTS)-1:0] grant_encoded
54 );
55 reg [PORTS-1:0] grant_reg = 0, grant_next;
56 reg grant_valid_reg = 0, grant_valid_next;
57 reg [$clog2(PORTS)-1:0] grant_encoded_reg = 0,
grant_encoded_next;
58 assign grant_valid = grant_valid_reg;
59 assign grant = grant_reg;
60 assign grant_encoded = grant_encoded_reg;
61
62 wire request_valid;
63 wire [$clog2(PORTS)-1:0] request_index;
64 wire [PORTS-1:0] request_mask;
65
66 priority_encoder #(
67     .WIDTH(PORTS),
68     .LSB_HIGH_PRIORITY(ARB_LSB_HIGH_PRIORITY)
69 )
70 priority_encoder_inst (
71     .input_unencoded(request),
72     .output_valid(request_valid),
73     .output_encoded(request_index),
74     .output_unencoded(request_mask)
75 );
76
77 reg [PORTS-1:0] mask_reg = 0, mask_next;
78
79 wire masked_request_valid;
80 wire [$clog2(PORTS)-1:0] masked_request_index;
81 wire [PORTS-1:0] masked_request_mask;
82
83 priority_encoder #(
84     .WIDTH(PORTS),
85     .LSB_HIGH_PRIORITY(ARB_LSB_HIGH_PRIORITY)
86 )
87 priority_encoder_masked (
88     .input_unencoded(request & mask_reg),
89     .output_valid(masked_request_valid),
90     .output_encoded(masked_request_index),
91     .output_unencoded(masked_request_mask)
92 );
93
94 always @* begin
95     grant_next = 0;
96     grant_valid_next = 0;
97     grant_encoded_next = 0;
98     mask_next = mask_reg;
99
100     if (ARB_BLOCK && !ARB_BLOCK_ACK && grant_reg & request)
101     begin
102         // granted request still asserted; hold it
103         grant_valid_next = grant_valid_reg;
104         grant_next = grant_reg;
105         grant_encoded_next = grant_encoded_reg;
106     end else if (ARB_BLOCK && ARB_BLOCK_ACK && grant_valid
&& !(grant_reg & acknowledge)) begin
107         // granted request not yet acknowledged; hold it
108         grant_valid_next = grant_valid_reg;
109         grant_next = grant_reg;
110         grant_encoded_next = grant_encoded_reg;
111     end else if (request_valid) begin
112         if (ARB_TYPE_ROUND_ROBIN) begin
113             if (masked_request_valid) begin
114                 grant_valid_next = 1;
115                 grant_next = masked_request_mask;
116                 grant_encoded_next = masked_request_index;
117                 if (ARB_LSB_HIGH_PRIORITY) begin
118                     mask_next = {PORTS{1'b1}} <<
(masked_request_index + 1);
119                 end else begin
120                     mask_next = {PORTS{1'b1}} >> (PORTS -
masked_request_index);
121                 end
122             end else begin
123                 grant_valid_next = 1;
124                 grant_next = request_mask;
125                 grant_encoded_next = request_index;
126                 if (ARB_LSB_HIGH_PRIORITY) begin
127                     mask_next = {PORTS{1'b1}} << (re-
quest_index + 1);
128                 end else begin
129                     mask_next = {PORTS{1'b1}} >> (PORTS -
request_index);
130                 end
131             end
132         end
133     end
134 end else begin
135     grant_valid_next = 1;
136     grant_next = request_mask;
137     grant_encoded_next = request_index;
138     mask_next = {PORTS{1'b1}} >> (PORTS - re-
quest_index);
139 end
140 end
141 end
142
143 always @(posedge clk) begin
144     if (rst) begin
145         grant_reg <= 0;
146         grant_valid_reg <= 0;
147         grant_encoded_reg <= 0;
148         mask_reg <= 0;
149     end else begin
150         grant_reg <= grant_next;
151         grant_valid_reg <= grant_valid_next;
152         grant_encoded_reg <= grant_encoded_next;
153         mask_reg <= mask_next;
154     end
155 end
156
157 endmodule
158
159 `resetall

```

```

131     end else begin
132         mask_next = {PORTS{1'b1}} >> (PORTS - re-
quest_index);
133     end
134 end
135 end else begin
136     grant_valid_next = 1;
137     grant_next = request_mask;
138     grant_encoded_next = request_index;
139 end
140 end
141 end
142
143 always @(posedge clk) begin
144     if (rst) begin
145         grant_reg <= 0;
146         grant_valid_reg <= 0;
147         grant_encoded_reg <= 0;
148         mask_reg <= 0;
149     end else begin
150         grant_reg <= grant_next;
151         grant_valid_reg <= grant_valid_next;
152         grant_encoded_reg <= grant_encoded_next;
153         mask_reg <= mask_next;
154     end
155 end
156
157 endmodule
158
159 `resetall

```

2. alexforenrich_verilog-pcie/rtl/axis_arb_mux.v (310 lines)

```

1 /*
2
3 Copyright (c) 2014-2018 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * AXI4-Stream arbitrated multiplexer
33 */
34 module axis_arb_mux #
35 (
36     // Number of AXI stream inputs
37     parameter S_COUNT = 4,
38     // Width of AXI stream interfaces in bits
39     parameter DATA_WIDTH = 8,
40     // Propagate tkeep signal
41     parameter KEEP_ENABLE = (DATA_WIDTH>8),
42     // tkeep signal width (words per cycle)
43     parameter KEEP_WIDTH = ((DATA_WIDTH+7)/8),
44     // Propagate tid signal
45     parameter ID_ENABLE = 0,
46     // input tid signal width
47     parameter S_ID_WIDTH = 8,
48     // output tid signal width
49     parameter M_ID_WIDTH = S_ID_WIDTH+$clog2(S_COUNT),
50     // Propagate tdest signal
51     parameter DEST_ENABLE = 0,
52     // tdest signal width
53     parameter DEST_WIDTH = 8,
54     // Propagate tuser signal

```

```

55 parameter USER_ENABLE = 1,
56 // tuser signal width
57 parameter USER_WIDTH = 1,
58 // Propagate tlast signal
59 parameter LAST_ENABLE = 1,
60 // Update tid with routing information
61 parameter UPDATE_TID = 0,
62 // select round robin arbitration
63 parameter ARB_TYPE_ROUND_ROBIN = 0,
64 // LSB priority selection
65 parameter ARB_LSB_HIGH_PRIORITY = 1
66 )
67 (
68   input wire clk,
69   input wire rst,
70
71   /*
72    * AXI Stream inputs
73    */
74   input wire [S_COUNT*DATA_WIDTH-1:0] s_axis_tdata,
75   input wire [S_COUNT*KEEP_WIDTH-1:0] s_axis_tkeep,
76   input wire [S_COUNT-1:0] s_axis_tvalid,
77   output wire [S_COUNT-1:0] s_axis_tready,
78   input wire [S_COUNT-1:0] s_axis_tlast,
79   input wire [S_COUNT*S_ID_WIDTH-1:0] s_axis_tid,
80   input wire [S_COUNT*DEST_WIDTH-1:0] s_axis_tdest,
81   input wire [S_COUNT*USER_WIDTH-1:0] s_axis_tuser,
82
83   /*
84    * AXI Stream output
85    */
86   output wire [DATA_WIDTH-1:0] m_axis_tdata,
87   output wire [KEEP_WIDTH-1:0] m_axis_tkeep,
88   output wire m_axis_tvalid,
89   input wire m_axis_tready,
90   output wire m_axis_tlast,
91   output wire [M_ID_WIDTH-1:0] m_axis_tid,
92   output wire [DEST_WIDTH-1:0] m_axis_tdest,
93   output wire [USER_WIDTH-1:0] m_axis_tuser
94 );
95
96 parameter CL_S_COUNT = $clog2(S_COUNT);
97
98 parameter S_ID_WIDTH_INT = S_ID_WIDTH > 0 ? S_ID_WIDTH : 1;
99
100 // check configuration
101 initial begin
102   if (UPDATE_TID) begin
103     if (!ID_ENABLE) begin
104       $error("Error: UPDATE_TID set requires ID_ENABLE");
105     end
106   end
107
108   if (M_ID_WIDTH < CL_S_COUNT) begin
109     $error("Error: M_ID_WIDTH too small for port count (instance %m)");
110   end
111   $finish;
112 end
113
114 wire [S_COUNT-1:0] request;
115 wire [S_COUNT-1:0] acknowledge;
116 wire [S_COUNT-1:0] grant;
117 wire grant_valid;
118 wire [CL_S_COUNT-1:0] grant_encoded;
119
120 // input registers to pipeline arbitration delay
121 reg [S_COUNT*DATA_WIDTH-1:0] s_axis_tdata_reg = 0;
122 reg [S_COUNT*KEEP_WIDTH-1:0] s_axis_tkeep_reg = 0;
123 reg [S_COUNT-1:0] s_axis_tvalid_reg = 0;
124 reg [S_COUNT-1:0] s_axis_tlast_reg = 0;
125 reg [S_COUNT*S_ID_WIDTH-1:0] s_axis_tid_reg = 0;
126 reg [S_COUNT*DEST_WIDTH-1:0] s_axis_tdest_reg = 0;
127 reg [S_COUNT*USER_WIDTH-1:0] s_axis_tuser_reg = 0;
128
129 // internal datapath
130 reg [DATA_WIDTH-1:0] m_axis_tdata_int;
131 reg [KEEP_WIDTH-1:0] m_axis_tkeep_int;
132 reg m_axis_tvalid_int;
133 reg m_axis_tready_int_reg = 1'b0;
134 reg m_axis_tlast_int;
135 reg [M_ID_WIDTH-1:0] m_axis_tid_int;
136 reg [DEST_WIDTH-1:0] m_axis_tdest_int;
137 reg [USER_WIDTH-1:0] m_axis_tuser_int;
138 wire m_axis_tready_int_early;
139
140 assign s_axis_tready = ~s_axis_tvalid_reg |
141 ({S_COUNT{m_axis_tready_int_reg}} & grant);
142
143 // mux for incoming packet
144 wire [DATA_WIDTH-1:0] current_s_tdata = s_ax-
145 is_tdata_reg[grant_encoded*DATA_WIDTH+: DATA_WIDTH];
146 wire [KEEP_WIDTH-1:0] current_s_tkeep = s_ax-
147 is_tkeep_reg[grant_encoded*KEEP_WIDTH+: KEEP_WIDTH];
148 wire current_s_tvalid = s_ax-
149 is_tvalid_reg[grant_encoded];
150 wire current_s_tready = s_ax-
151 is_tready_reg[grant_encoded];

```

```

148 wire current_s_tlast = s_ax-
149 is_tlast_reg[grant_encoded];
150 wire [S_ID_WIDTH-1:0] current_s_tid = s_ax-
151 is_tid_reg[grant_encoded*S_ID_WIDTH+: S_ID_WIDTH_INT];
152 wire [DEST_WIDTH-1:0] current_s_tdest = s_ax-
153 is_tdest_reg[grant_encoded*DEST_WIDTH+: DEST_WIDTH];
154 wire [USER_WIDTH-1:0] current_s_tuser = s_ax-
155 is_tuser_reg[grant_encoded*USER_WIDTH+: USER_WIDTH];
156
157 // arbiter instance
158 arbiter #(
159   .PORTS(S_COUNT),
160   .ARB_TYPE_ROUND_ROBIN(ARB_TYPE_ROUND_ROBIN),
161   .ARB_BLOCK(1),
162   .ARB_BLOCK_ACK(1),
163   .ARB_LSB_HIGH_PRIORITY(ARB_LSB_HIGH_PRIORITY)
164 )
165 arb_inst (
166   .clk(clk),
167   .rst(rst),
168   .request(request),
169   .acknowledge(acknowledge),
170   .grant(grant),
171   .grant_valid(grant_valid),
172   .grant_encoded(grant_encoded)
173 );
174
175 assign request = (s_axis_tvalid_reg & ~grant) | (s_axis_tvalid &
176 grant);
177 assign acknowledge = grant & s_axis_tvalid_reg &
178 {S_COUNT{m_axis_tready_int_reg}} & (LAST_ENABLE ? s_axis_tlast_reg :
179 {S_COUNT{1'b1}});
180
181 always @* begin
182   // pass through selected packet data
183   m_axis_tdata_int = current_s_tdata;
184   m_axis_tkeep_int = current_s_tkeep;
185   m_axis_tvalid_int = current_s_tvalid && m_ax-
186 is_tready_int_reg && grant_valid;
187   m_axis_tlast_int = current_s_tlast;
188   m_axis_tid_int = current_s_tid;
189   if (UPDATE_TID && S_COUNT > 1) begin
190     m_axis_tid_int[M_ID_WIDTH-1:M_ID_WIDTH-CL_S_COUNT] =
191 grant_encoded;
192   end
193   m_axis_tdest_int = current_s_tdest;
194   m_axis_tuser_int = current_s_tuser;
195 end
196
197 integer i;
198
199 always @(posedge clk) begin
200   // register inputs
201   for (i = 0; i < S_COUNT; i = i + 1) begin
202     if (s_axis_tready[i]) begin
203       s_axis_tdata_reg[i*DATA_WIDTH+: DATA_WIDTH] <=
204 s_axis_tdata[i*DATA_WIDTH+: DATA_WIDTH];
205       s_axis_tkeep_reg[i*KEEP_WIDTH+: KEEP_WIDTH] <=
206 s_axis_tkeep[i*KEEP_WIDTH+: KEEP_WIDTH];
207       s_axis_tvalid_reg[i] <= s_axis_tvalid[i];
208       s_axis_tlast_reg[i] <= s_axis_tlast[i];
209       s_axis_tid_reg[i*S_ID_WIDTH+: S_ID_WIDTH_INT] <=
210 s_axis_tid[i*S_ID_WIDTH+: S_ID_WIDTH_INT];
211       s_axis_tdest_reg[i*DEST_WIDTH+: DEST_WIDTH] <=
212 s_axis_tdest[i*DEST_WIDTH+: DEST_WIDTH];
213       s_axis_tuser_reg[i*USER_WIDTH+: USER_WIDTH] <=
214 s_axis_tuser[i*USER_WIDTH+: USER_WIDTH];
215     end
216   end
217
218 if (rst) begin
219   s_axis_tvalid_reg <= 0;
220 end
221
222 // output datapath logic
223 reg [DATA_WIDTH-1:0] temp_m_axis_tdata_reg =
224 {DATA_WIDTH{1'b0}};
225 reg [KEEP_WIDTH-1:0] temp_m_axis_tkeep_reg =
226 {KEEP_WIDTH{1'b0}};
227 reg temp_m_axis_tvalid_reg = 1'b0, temp_m_ax-
228 is_tvalid_next;
229 reg temp_m_axis_tlast_reg = 1'b0;
230 reg [M_ID_WIDTH-1:0] temp_m_axis_tid_reg =
231 {M_ID_WIDTH{1'b0}};
232 reg [DEST_WIDTH-1:0] temp_m_axis_tdest_reg =
233 {DEST_WIDTH{1'b0}};
234 reg [USER_WIDTH-1:0] temp_m_axis_tuser_reg =
235 {USER_WIDTH{1'b0}};
236
237 // datapath control

```

```

227 reg store_axis_int_to_output;
228 reg store_axis_int_to_temp;
229 reg store_axis_temp_to_output;
230
231 assign m_axis_tdata = m_axis_tdata_reg;
232 assign m_axis_tkeep = KEEP_ENABLE ? m_axis_tkeep_reg :
{KEEP_WIDTH{1'b1}};
233 assign m_axis_tvalid = m_axis_tvalid_reg;
234 assign m_axis_tlast = LAST_ENABLE ? m_axis_tlast_reg : 1'b1;
235 assign m_axis_tid = ID_ENABLE ? m_axis_tid_reg :
{M_ID_WIDTH{1'b0}};
236 assign m_axis_tdest = DEST_ENABLE ? m_axis_tdest_reg :
{DEST_WIDTH{1'b0}};
237 assign m_axis_tuser = USER_ENABLE ? m_axis_tuser_reg :
{USER_WIDTH{1'b0}};
238
239 // enable ready input next cycle if output is ready or if both
output registers are empty
240 assign m_axis_tready_int_early = m_axis_tready || (!temp_m_ax-
is_tvalid_reg && !m_axis_tvalid_reg);
241
242 always @* begin
243 // transfer sink ready state to source
244 m_axis_tvalid_next = m_axis_tvalid_reg;
245 temp_m_axis_tvalid_next = temp_m_axis_tvalid_reg;
246
247 store_axis_int_to_output = 1'b0;
248 store_axis_int_to_temp = 1'b0;
249 store_axis_temp_to_output = 1'b0;
250
251 if (m_axis_tready_int_reg) begin
252 // input is ready
253 if (m_axis_tready || !m_axis_tvalid_reg) begin
254 // output is ready or currently not valid, trans-
fer data to output
255 m_axis_tvalid_next = m_axis_tvalid_int;
256 store_axis_int_to_output = 1'b1;
257 end else begin
258 // output is not ready, store input in temp
259 temp_m_axis_tvalid_next = m_axis_tvalid_int;
260 store_axis_int_to_temp = 1'b1;
261 end
262 end else if (m_axis_tready) begin
263 // input is not ready, but output is ready
264 m_axis_tvalid_next = temp_m_axis_tvalid_reg;
265 temp_m_axis_tvalid_next = 1'b0;
266 store_axis_temp_to_output = 1'b1;
267 end
268 end
269
270 always @(posedge clk) begin
271 m_axis_tvalid_reg <= m_axis_tvalid_next;
272 m_axis_tready_int_reg <= m_axis_tready_int_early;
273 temp_m_axis_tvalid_reg <= temp_m_axis_tvalid_next;
274
275 // datapath
276 if (store_axis_int_to_output) begin
277 m_axis_tdata_reg <= m_axis_tdata_int;
278 m_axis_tkeep_reg <= m_axis_tkeep_int;
279 m_axis_tlast_reg <= m_axis_tlast_int;
280 m_axis_tid_reg <= m_axis_tid_int;
281 m_axis_tdest_reg <= m_axis_tdest_int;
282 m_axis_tuser_reg <= m_axis_tuser_int;
283 end else if (store_axis_temp_to_output) begin
284 m_axis_tdata_reg <= temp_m_axis_tdata_reg;
285 m_axis_tkeep_reg <= temp_m_axis_tkeep_reg;
286 m_axis_tlast_reg <= temp_m_axis_tlast_reg;
287 m_axis_tid_reg <= temp_m_axis_tid_reg;
288 m_axis_tdest_reg <= temp_m_axis_tdest_reg;
289 m_axis_tuser_reg <= temp_m_axis_tuser_reg;
290 end
291
292 if (store_axis_int_to_temp) begin
293 temp_m_axis_tdata_reg <= m_axis_tdata_int;
294 temp_m_axis_tkeep_reg <= m_axis_tkeep_int;
295 temp_m_axis_tlast_reg <= m_axis_tlast_int;
296 temp_m_axis_tid_reg <= m_axis_tid_int;
297 temp_m_axis_tdest_reg <= m_axis_tdest_int;
298 temp_m_axis_tuser_reg <= m_axis_tuser_int;
299 end
300
301 if (rst) begin
302 m_axis_tvalid_reg <= 1'b0;
303 m_axis_tready_int_reg <= 1'b0;
304 temp_m_axis_tvalid_reg <= 1'b0;
305 end
306 end
307 endmodule
308
309 `resetall
310

```

3. alexforenrich_verilog-pcie/rtl/dma_client_axis_sink.v (635 lines)

```

1 /*
2
3 Copyright (c) 2019-2021 Alex Forenrich
4

```

```

5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * AXI stream sink DMA client
33 */
34 module dma_client_axis_sink #
35 (
36 // RAM address width
37 parameter RAM_ADDR_WIDTH = 16,
38 // RAM segment count
39 parameter SEG_COUNT = 2,
40 // RAM segment data width
41 parameter SEG_DATA_WIDTH = 64,
42 // RAM segment byte enable width
43 parameter SEG_BE_WIDTH = SEG_DATA_WIDTH/8,
44 // RAM segment address width
45 parameter SEG_ADDR_WIDTH = RAM_ADDR_WIDTH-$clo-
g2(SEG_COUNT*SEG_BE_WIDTH),
46 // Width of AXI stream interfaces in bits
47 parameter AXIS_DATA_WIDTH = SEG_DATA_WIDTH*SEG_COUNT/2,
48 // Use AXI stream tkeep signal
49 parameter AXIS_KEEP_ENABLE = (AXIS_DATA_WIDTH>8),
50 // AXI stream tkeep signal width (words per cycle)
51 parameter AXIS_KEEP_WIDTH = (AXIS_DATA_WIDTH/8),
52 // Use AXI stream tlast signal
53 parameter AXIS_LAST_ENABLE = 1,
54 // Propagate AXI stream tid signal
55 parameter AXIS_ID_ENABLE = 0,
56 // AXI stream tid signal width
57 parameter AXIS_ID_WIDTH = 8,
58 // Propagate AXI stream tdest signal
59 parameter AXIS_DEST_ENABLE = 0,
60 // AXI stream tdest signal width
61 parameter AXIS_DEST_WIDTH = 8,
62 // Propagate AXI stream tuser signal
63 parameter AXIS_USER_ENABLE = 1,
64 // AXI stream tuser signal width
65 parameter AXIS_USER_WIDTH = 1,
66 // Width of length field
67 parameter LEN_WIDTH = 16,
68 // Width of tag field
69 parameter TAG_WIDTH = 8
70 )
71 (
72 input wire clk,
73 input wire rst,
74
75 /*
76 * AXI write descriptor input
77 */
78 input wire [RAM_ADDR_WIDTH-1:0] s_ax-
is_write_desc_ram_addr,
79 input wire [LEN_WIDTH-1:0] s_ax-
is_write_desc_len,
80 input wire [TAG_WIDTH-1:0] s_ax-
is_write_desc_tag,
81 input wire s_ax-
is_write_desc_valid,
82 output wire s_ax-
is_write_desc_ready,
83
84 /*
85 * AXI write descriptor status output
86 */

```



```

87   output wire [LEN_WIDTH-1:0]          m_ax-
is_write_desc_status_len,                m_ax-
88   output wire [TAG_WIDTH-1:0]          m_ax-
is_write_desc_status_tag,                m_ax-
89   output wire [AXIS_ID_WIDTH-1:0]      m_ax-
is_write_desc_status_id,                  m_ax-
90   output wire [AXIS_DEST_WIDTH-1:0]    m_ax-
is_write_desc_status_dest,                m_ax-
91   output wire [AXIS_USER_WIDTH-1:0]    m_ax-
is_write_desc_status_user,                m_ax-
92   output wire [3:0]                    m_ax-
is_write_desc_status_error,                m_ax-
93   output wire                          m_ax-
is_write_desc_status_valid,
94
95   /*
96   * AXI stream write data input
97   */
98   input wire [AXIS_DATA_WIDTH-1:0]      s_ax-
is_write_data_tdata,                      s_ax-
99   input wire [AXIS_KEEP_WIDTH-1:0]      s_ax-
is_write_data_tkeep,                      s_ax-
100  input wire                             s_ax-
is_write_data_tvalid,                     s_ax-
101  output wire                             s_ax-
is_write_data_tready,                     s_ax-
102  input wire                             s_ax-
is_write_data_tlast,                      s_ax-
103  input wire [AXIS_ID_WIDTH-1:0]        s_ax-
is_write_data_tid,                        s_ax-
104  input wire [AXIS_DEST_WIDTH-1:0]      s_ax-
is_write_data_tdest,                      s_ax-
105  input wire [AXIS_USER_WIDTH-1:0]      s_ax-
is_write_data_tuser,
106
107  /*
108  * RAM interface
109  */
110  output wire [SEG_COUNT*SEG_BE_WIDTH-1:0] ram_wr_cmd_be,
111  output wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0] ram_wr_cmd_ad-
dr,
112  output wire [SEG_COUNT*SEG_DATA_WIDTH-1:0] ram_wr_cmd_data,
113  output wire [SEG_COUNT-1:0] ram_wr_cmd_valid,
114  input wire [SEG_COUNT-1:0] ram_wr_cmd_ready,
115  input wire [SEG_COUNT-1:0] ram_wr_done,
116
117  /*
118  * Configuration
119  */
120  input wire enable,
121  input wire abort
122 );
123
124 parameter RAM_WORD_WIDTH = SEG_BE_WIDTH;
125 parameter RAM_WORD_SIZE = SEG_DATA_WIDTH/RAM_WORD_WIDTH;
126
127 parameter AXIS_KEEP_WIDTH_INT = AXIS_KEEP_ENABLE ?
AXIS_KEEP_WIDTH : 1;
128 parameter AXIS_WORD_WIDTH = AXIS_KEEP_WIDTH_INT;
129 parameter AXIS_WORD_SIZE = AXIS_DATA_WIDTH/AXIS_WORD_WIDTH;
130
131 parameter PART_COUNT = SEG_COUNT*SEG_BE_WIDTH /
AXIS_KEEP_WIDTH_INT;
132 parameter PART_COUNT_WIDTH = PART_COUNT > 1 ? $clog-
2(PART_COUNT) : 1;
133 parameter PART_OFFSET_WIDTH = AXIS_KEEP_WIDTH_INT > 1 ? $clog-
2(AXIS_KEEP_WIDTH_INT) : 1;
134 parameter PARTS_PER_SEG = (SEG_BE_WIDTH + AXIS_KEEP_WIDTH_INT
- 1) / AXIS_KEEP_WIDTH_INT;
135 parameter SEG_PER_PART = (AXIS_KEEP_WIDTH_INT + SEG_BE_WIDTH
- 1) / SEG_BE_WIDTH;
136
137 parameter OFFSET_WIDTH = AXIS_KEEP_WIDTH_INT > 1 ? $clog-
2(AXIS_KEEP_WIDTH_INT) : 1;
138 parameter OFFSET_MASK = AXIS_KEEP_WIDTH_INT > 1 ? {OFF-
SET_WIDTH{1'b1}} : 0;
139 parameter ADDR_MASK = {RAM_ADDR_WIDTH{1'b1}} << $clog-
2(AXIS_KEEP_WIDTH_INT);
140 parameter CYCLE_COUNT_WIDTH = LEN_WIDTH - $clog-
2(AXIS_KEEP_WIDTH_INT) + 1;
141
142 parameter STATUS_FIFO_ADDR_WIDTH = 5;
143 parameter OUTPUT_FIFO_ADDR_WIDTH = 5;
144
145 // bus width assertions
146 initial begin
147   if (RAM_WORD_SIZE * SEG_BE_WIDTH != SEG_DATA_WIDTH) begin
148     $error("Error: RAM data width not evenly divisble (in-
stance %m)");
149   $finish;
150   end
151
152   if (AXIS_WORD_SIZE * AXIS_KEEP_WIDTH_INT !=
AXIS_DATA_WIDTH) begin
153     $error("Error: AXI stream data width not evenly divis-
ble (instance %m)");
154   $finish;
155   end

```

```

156
157   if (RAM_WORD_SIZE != AXIS_WORD_SIZE) begin
158     $error("Error: word size mismatch (instance %m)");
159   $finish;
160   end
161
162   if (2**$clog2(RAM_WORD_WIDTH) != RAM_WORD_WIDTH) begin
163     $error("Error: RAM word width must be even power of
two (instance %m)");
164   $finish;
165   end
166
167   if (RAM_ADDR_WIDTH != SEG_ADDR_WIDTH+$clog2(SEG_COUNT)+
$clog2(SEG_BE_WIDTH)) begin
168     $error("Error: RAM_ADDR_WIDTH does not match RAM con-
figuration (instance %m)");
169   $finish;
170   end
171
172   if (AXIS_DATA_WIDTH > SEG_COUNT*SEG_DATA_WIDTH) begin
173     $error("Error: AXI stream interface width must not be
wider than RAM interface width (instance %m)");
174   $finish;
175   end
176
177   if (AXIS_DATA_WIDTH*2**$clog2(PART_COUNT) !=
SEG_COUNT*SEG_DATA_WIDTH) begin
178     $error("Error: AXI stream interface width must be a
power of two fraction of RAM interface width (instance %m)");
179   $finish;
180   end
181 end
182
183 localparam [1:0]
184   STATE_IDLE = 2'd0,
185   STATE_WRITE = 2'd1,
186   STATE_DROP_DATA = 2'd2;
187
188 reg [1:0] state_reg = STATE_IDLE, state_next;
189
190 integer i;
191 reg [OFFSET_WIDTH:0] cycle_size;
192
193 reg [RAM_ADDR_WIDTH-1:0] addr_reg = {RAM_ADDR_WIDTH{1'b0}},
addr_next;
194 reg [AXIS_KEEP_WIDTH_INT-1:0] keep_mask_reg =
{AXIS_KEEP_WIDTH_INT{1'b0}}, keep_mask_next;
195 reg [OFFSET_WIDTH-1:0] last_cycle_offset_reg = {OFF-
SET_WIDTH{1'b0}}, last_cycle_offset_next;
196 reg [LEN_WIDTH-1:0] length_reg = {LEN_WIDTH{1'b0}},
length_next;
197 reg [CYCLE_COUNT_WIDTH-1:0] cycle_count_reg =
{CYCLE_COUNT_WIDTH{1'b0}}, cycle_count_next;
198 reg last_cycle_reg = 1'b0, last_cycle_next;
199
200 reg [TAG_WIDTH-1:0] tag_reg = {TAG_WIDTH{1'b0}}, tag_next;
201
202 reg [STATUS_FIFO_ADDR_WIDTH+1-1:0] status_fifo_wr_ptr_reg = 0;
203 reg [STATUS_FIFO_ADDR_WIDTH+1-1:0] status_fifo_rd_ptr_reg = 0,
status_fifo_rd_ptr_next;
204 reg [LEN_WIDTH-1:0] status_fifo_len[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
205 reg [TAG_WIDTH-1:0] status_fifo_tag[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
206 reg [AXIS_ID_WIDTH-1:0] status_fifo_id[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
207 reg [AXIS_DEST_WIDTH-1:0] status_fifo_dest[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
208 reg [AXIS_USER_WIDTH-1:0] status_fifo_user[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
209 reg [SEG_COUNT-1:0] status_fifo_mask[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
210 reg status_fifo_last[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
211 reg [LEN_WIDTH-1:0] status_fifo_wr_len;
212 reg [TAG_WIDTH-1:0] status_fifo_wr_tag;
213 reg [AXIS_ID_WIDTH-1:0] status_fifo_wr_id;
214 reg [AXIS_DEST_WIDTH-1:0] status_fifo_wr_dest;
215 reg [AXIS_USER_WIDTH-1:0] status_fifo_wr_user;
216 reg [SEG_COUNT-1:0] status_fifo_wr_mask;
217 reg status_fifo_wr_last;
218 reg status_fifo_we = 1'b0;
219 reg status_fifo_half_full_reg = 1'b0;
220
221 reg [STATUS_FIFO_ADDR_WIDTH+1-1:0] active_count_reg = 0;
222 reg active_count_av_reg = 1'b1;
223 reg inc_active;
224 reg dec_active;
225
226 reg s_axis_write_desc_ready_reg = 1'b0, s_ax-
is_write_desc_ready_next;
227
228 reg [LEN_WIDTH-1:0] m_axis_write_desc_status_len_reg =
{LEN_WIDTH{1'b0}}, m_axis_write_desc_status_len_next;
229 reg [TAG_WIDTH-1:0] m_axis_write_desc_status_tag_reg =
{TAG_WIDTH{1'b0}}, m_axis_write_desc_status_tag_next;
230 reg [AXIS_ID_WIDTH-1:0] m_axis_write_desc_status_id_reg =
{AXIS_ID_WIDTH{1'b0}}, m_axis_write_desc_status_id_next;
231 reg [AXIS_DEST_WIDTH-1:0] m_axis_write_desc_status_dest_reg =
{AXIS_DEST_WIDTH{1'b0}}, m_axis_write_desc_status_dest_next;
232 reg [AXIS_USER_WIDTH-1:0] m_axis_write_desc_status_user_reg =

```



```

[AXIS_USER_WIDTH{1'b0}], m_axis_write_desc_status_user_next;
233 reg m_axis_write_desc_status_valid_reg = 1'b0, m_ax-
is_write_desc_status_valid_next;
234
235 reg s_axis_write_data_tready_reg = 1'b0, s_ax-
is_write_data_tready_next;
236
237 // internal datapath
238 reg [SEG_COUNT*SEG_BE_WIDTH-1:0] ram_wr_cmd_be_int;
239 reg [SEG_COUNT*SEG_ADDR_WIDTH-1:0] ram_wr_cmd_addr_int;
240 reg [SEG_COUNT*SEG_DATA_WIDTH-1:0] ram_wr_cmd_data_int;
241 reg [SEG_COUNT-1:0] ram_wr_cmd_valid_int;
242 wire [SEG_COUNT-1:0] ram_wr_cmd_ready_int;
243
244 reg [SEG_COUNT-1:0] ram_wr_cmd_mask;
245
246 wire [SEG_COUNT-1:0] out_done;
247 reg [SEG_COUNT-1:0] out_done_ack;
248
249 assign s_axis_write_desc_ready = s_axis_write_desc_ready_reg;
250
251 assign m_axis_write_desc_status_len = m_ax-
is_write_desc_status_len_reg;
252 assign m_axis_write_desc_status_tag = m_ax-
is_write_desc_status_tag_reg;
253 assign m_axis_write_desc_status_id = m_ax-
is_write_desc_status_id_reg;
254 assign m_axis_write_desc_status_dest = m_ax-
is_write_desc_status_dest_reg;
255 assign m_axis_write_desc_status_user = m_ax-
is_write_desc_status_user_reg;
256 assign m_axis_write_desc_status_error = 4'd0;
257 assign m_axis_write_desc_status_valid = m_ax-
is_write_desc_status_valid_reg;
258
259 assign s_axis_write_data_tready = s_ax-
is_write_data_tready_reg;
260
261 always @* begin
262     state_next = STATE_IDLE;
263
264     s_axis_write_desc_ready_next = 1'b0;
265
266     m_axis_write_desc_status_len_next = m_ax-
is_write_desc_status_len_reg;
267     m_axis_write_desc_status_tag_next = m_ax-
is_write_desc_status_tag_reg;
268     m_axis_write_desc_status_id_next = m_ax-
is_write_desc_status_id_reg;
269     m_axis_write_desc_status_dest_next = m_ax-
is_write_desc_status_dest_reg;
270     m_axis_write_desc_status_user_next = m_ax-
is_write_desc_status_user_reg;
271     m_axis_write_desc_status_valid_next = 1'b0;
272
273     s_axis_write_data_tready_next = 1'b0;
274
275     if (PART_COUNT > 1) begin
276         ram_wr_cmd_be_int = (s_axis_write_data_tkeep &
keep_mask_reg) << (addr_reg & ({PART_COUNT_WIDTH{1'b1}} << PART_OFF-
SET_WIDTH));
277     end else begin
278         ram_wr_cmd_be_int = s_axis_write_data_tkeep &
keep_mask_reg;
279     end
280     ram_wr_cmd_addr_int =
{PART_COUNT{addr_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-SEG_AD-
DR_WIDTH]}};
281     ram_wr_cmd_data_int =
{PART_COUNT{s_axis_write_data_tdata}};
282     ram_wr_cmd_valid_int = {SEG_COUNT{1'b0}};
283     for (i = 0; i < SEG_COUNT; i = i + 1) begin
284         ram_wr_cmd_mask[i] = ram_wr_cmd_be_int[i*SEG_BE_WIDTH
+: SEG_BE_WIDTH] != 0;
285     end
286
287     cycle_size = AXIS_KEEP_WIDTH_INT;
288
289     addr_next = addr_reg;
290     keep_mask_next = keep_mask_reg;
291     last_cycle_offset_next = last_cycle_offset_reg;
292     length_next = length_reg;
293     cycle_count_next = cycle_count_reg;
294     last_cycle_next = last_cycle_reg;
295
296     tag_next = tag_reg;
297
298     status_fifo_rd_ptr_next = status_fifo_rd_ptr_reg;
299
300     status_fifo_wr_len = 0;
301     status_fifo_wr_tag = tag_reg;
302     status_fifo_wr_id = s_axis_write_data_tid;
303     status_fifo_wr_dest = s_axis_write_data_tdest;
304     status_fifo_wr_user = s_axis_write_data_tuser;
305     status_fifo_wr_mask = ram_wr_cmd_mask;
306     status_fifo_wr_last = 1'b0;
307     status_fifo_we = 1'b0;
308
309     inc_active = 1'b0;
310     dec_active = 1'b0;
311

```

```

312     out_done_ack = {SEG_COUNT{1'b0}};
313
314     case (state_reg)
315     STATE_IDLE: begin
316         // idle state - load new descriptor to start oper-
ation
317         s_axis_write_desc_ready_next = enable && act-
ive_count_av_reg;
318
319         addr_next = s_axis_write_desc_ram_addr &
ADDR_MASK;
320         last_cycle_offset_next = s_axis_write_desc_len &
OFFSET_MASK;
321
322         tag_next = s_axis_write_desc_tag;
323
324         length_next = 0;
325
326         cycle_count_next = (s_axis_write_desc_len - 1) >>
$log2(AXIS_KEEP_WIDTH_INT);
327         last_cycle_next = cycle_count_next == 0;
328         if (cycle_count_next == 0 &&
last_cycle_offset_next != 0) begin
329             keep_mask_next = {AXIS_KEEP_WIDTH_INT{1'b1}}
>> (AXIS_KEEP_WIDTH_INT - last_cycle_offset_next);
330         end else begin
331             keep_mask_next = {AXIS_KEEP_WIDTH_INT{1'b1}};
332         end
333
334         if (s_axis_write_desc_ready && s_ax-
is_write_desc_valid) begin
335             s_axis_write_desc_ready_next = 1'b0;
336             s_axis_write_data_tready_next =
&ram_wr_cmd_ready_int && !status_fifo_half_full_reg;
337
338             inc_active = 1'b1;
339
340             state_next = STATE_WRITE;
341         end else begin
342             state_next = STATE_IDLE;
343         end
344     end
345     STATE_WRITE: begin
346         // write state - generate write operations
347         s_axis_write_data_tready_next =
&ram_wr_cmd_ready_int && !status_fifo_half_full_reg;
348
349         if (s_axis_write_data_tready && s_ax-
is_write_data_tvalid) begin
350
351             // update counters
352             addr_next = addr_reg + AXIS_KEEP_WIDTH_INT;
353             length_next = length_reg +
AXIS_KEEP_WIDTH_INT;
354             cycle_count_next = cycle_count_reg - 1;
355             last_cycle_next = cycle_count_next == 0;
356             if (cycle_count_next == 0 &&
last_cycle_offset_reg != 0) begin
357                 keep_mask_next =
{AXIS_KEEP_WIDTH_INT{1'b1}} >> (AXIS_KEEP_WIDTH_INT -
last_cycle_offset_reg);
358             end else begin
359                 keep_mask_next =
{AXIS_KEEP_WIDTH_INT{1'b1}};
360             end
361
362             if (PART_COUNT > 1) begin
363                 ram_wr_cmd_be_int = (s_ax-
is_write_data_tkeep & keep_mask_reg) << (addr_reg &
({PART_COUNT_WIDTH{1'b1}} << PART_OFFSET_WIDTH));
364             end else begin
365                 ram_wr_cmd_be_int = s_ax-
is_write_data_tkeep & keep_mask_reg;
366             end
367             ram_wr_cmd_addr_int =
{SEG_COUNT{addr_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-SEG_AD-
DR_WIDTH]}};
368             ram_wr_cmd_data_int =
{PART_COUNT{s_axis_write_data_tdata}};
369             ram_wr_cmd_valid_int = ram_wr_cmd_mask;
370
371             // enqueue status FIFO entry for write comple-
tion
372             status_fifo_wr_len = length_next;
373             status_fifo_wr_tag = tag_reg;
374             status_fifo_wr_id = s_axis_write_data_tid;
375             status_fifo_wr_dest = s_axis_write_data_tdest;
376             status_fifo_wr_user = s_axis_write_data_tuser;
377             status_fifo_wr_mask = ram_wr_cmd_mask;
378             status_fifo_wr_last = 1'b0;
379             status_fifo_we = 1'b1;
380
381             if (AXIS_LAST_ENABLE && s_ax-
is_write_data_tlast) begin
382                 if (AXIS_KEEP_ENABLE) begin
383                     cycle_size = AXIS_KEEP_WIDTH_INT;
384                     for (i = AXIS_KEEP_WIDTH_INT-1; i >=
0; i = i - 1) begin
385                         if (~(s_axis_write_data_tkeep &
keep_mask_reg) & (1 << i)) begin
386                             cycle_size = i;

```

```

387         end
388     end
389 end else begin
390     cycle_size = AXIS_KEEP_WIDTH_INT;
391 end
392
393 // no more data to transfer, finish operation
394 if (last_cycle_reg &&
last_cycle_offset_reg > 0) begin
395     if (AXIS_KEEP_ENABLE && !(s_ax-
is_write_data_tkeep & keep_mask_reg & ~({AXIS_KEEP_WIDTH_INT{1'b1}}
>> (AXIS_KEEP_WIDTH_INT - last_cycle_offset_reg)))) begin
396         length_next = length_reg +
cycle_size;
397     end else begin
398         length_next = length_reg +
last_cycle_offset_reg;
399     end
400 end else begin
401     if (AXIS_KEEP_ENABLE) begin
402         length_next = length_reg +
cycle_size;
403     end
404 end
405
406 // enqueue status FIFO entry for write
completion
407     status_fifo_wr_len = length_next;
408     status_fifo_wr_tag = tag_reg;
409     status_fifo_wr_id = s_axis_write_data_tid;
410     status_fifo_wr_dest = s_ax-
is_write_data_tdest;
411     status_fifo_wr_user = s_ax-
is_write_data_tuser;
412     status_fifo_wr_mask = ram_wr_cmd_mask;
413     status_fifo_wr_last = 1'b1;
414     status_fifo_we = 1'b1;
415
416     s_axis_write_data_tready_next = 1'b0;
417     s_axis_write_desc_ready_next = enable &&
active_count_av_reg;
418     state_next = STATE_IDLE;
419 end else if (last_cycle_reg) begin
420     if (last_cycle_offset_reg > 0) begin
421         length_next = length_reg +
last_cycle_offset_reg;
422     end
423
424     // enqueue status FIFO entry for write
completion
425     status_fifo_wr_len = length_next;
426     status_fifo_wr_tag = tag_reg;
427     status_fifo_wr_id = s_axis_write_data_tid;
428     status_fifo_wr_dest = s_ax-
is_write_data_tdest;
429     status_fifo_wr_user = s_ax-
is_write_data_tuser;
430     status_fifo_wr_mask = ram_wr_cmd_mask;
431     status_fifo_wr_last = 1'b1;
432     status_fifo_we = 1'b1;
433
434     if (AXIS_LAST_ENABLE) begin
435         s_axis_write_data_tready_next = 1'b1;
436         state_next = STATE_DROP_DATA;
437     end else begin
438         s_axis_write_data_tready_next = 1'b0;
439         s_axis_write_desc_ready_next = enable
&& active_count_av_reg;
440         state_next = STATE_IDLE;
441     end
442 end else begin
443     state_next = STATE_WRITE;
444 end
445 end else begin
446     state_next = STATE_WRITE;
447 end
448 end
449 STATE_DROP_DATA: begin
450     // drop excess AXI stream data
451     s_axis_write_data_tready_next = 1'b1;
452
453     if (s_axis_write_data_tready && s_ax-
is_write_data_tvalid) begin
454         if (s_axis_write_data_tlast) begin
455             s_axis_write_data_tready_next = 1'b0;
456             s_axis_write_desc_ready_next = enable &&
active_count_av_reg;
457             state_next = STATE_IDLE;
458         end else begin
459             state_next = STATE_DROP_DATA;
460         end
461     end else begin
462         state_next = STATE_DROP_DATA;
463     end
464 end
465 endcase
466
467 m_axis_write_desc_status_len_next =
status_fifo_len[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]];
468 m_axis_write_desc_status_tag_next =

```

```

status_fifo_tag[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]];
469 m_axis_write_desc_status_id_next =
status_fifo_id[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]];
470 m_axis_write_desc_status_dest_next =
status_fifo_dest[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
;
471 m_axis_write_desc_status_user_next =
status_fifo_user[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
;
472 m_axis_write_desc_status_valid_next = 1'b0;
473
474 if (status_fifo_rd_ptr_reg != status_fifo_wr_ptr_reg) be-
gin
475     // status FIFO not empty
476     if
((status_fifo_mask[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]] & ~out_done) == 0) begin
477         // got write completion, pop and return status
478         status_fifo_rd_ptr_next = status_fifo_rd_ptr_reg +
1;
479
480         out_done_ack =
status_fifo_mask[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
;
481
482         if
(status_fifo_last[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]]) begin
483             m_axis_write_desc_status_valid_next = 1'b1;
484
485             dec_active = 1'b1;
486         end
487     end
488 end
489 end
490
491 always @(posedge clk) begin
492     state_reg <= state_next;
493
494     s_axis_write_desc_ready_reg <= s_ax-
is_write_desc_ready_next;
495
496     m_axis_write_desc_status_len_reg <= m_ax-
is_write_desc_status_len_next;
497     m_axis_write_desc_status_tag_reg <= m_ax-
is_write_desc_status_tag_next;
498     m_axis_write_desc_status_id_reg <= m_ax-
is_write_desc_status_id_next;
499     m_axis_write_desc_status_dest_reg <= m_ax-
is_write_desc_status_dest_next;
500     m_axis_write_desc_status_user_reg <= m_ax-
is_write_desc_status_user_next;
501     m_axis_write_desc_status_valid_reg <= m_ax-
is_write_desc_status_valid_next;
502
503     s_axis_write_data_tready_reg <= s_ax-
is_write_data_tready_next;
504
505     addr_reg <= addr_next;
506     keep_mask_reg <= keep_mask_next;
507     last_cycle_offset_reg <= last_cycle_offset_next;
508     length_reg <= length_next;
509     cycle_count_reg <= cycle_count_next;
510     last_cycle_reg <= last_cycle_next;
511
512     tag_reg <= tag_next;
513
514     if (status_fifo_we) begin
515
516         status_fifo_len[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_len;
517
518         status_fifo_tag[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_tag;
519
520         status_fifo_id[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_id;
521
522         status_fifo_dest[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_dest;
523
524         status_fifo_user[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_user;
525
526         status_fifo_mask[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_mask;
527
528         status_fifo_last[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_last;
529         status_fifo_wr_ptr_reg <= status_fifo_wr_ptr_reg + 1;
530
531         status_fifo_rd_ptr_reg <= status_fifo_rd_ptr_next;
532
533         status_fifo_half_full_reg <= $un-
signed(status_fifo_wr_ptr_reg - status_fifo_rd_ptr_reg) >=
2** (STATUS_FIFO_ADDR_WIDTH-1);
534
535         if (active_count_reg < 2**STATUS_FIFO_ADDR_WIDTH &&
inc_active && !dec_active) begin
536             active_count_reg <= active_count_reg + 1;

```

```

530     active_count_av_reg <= active_count_reg <
(2**STATUS_FIFO_ADDR_WIDTH-1);
531     end else if (active_count_reg > 0 && !inc_active &&
dec_active) begin
532         active_count_reg <= active_count_reg - 1;
533         active_count_av_reg <= 1'b1;
534     end else begin
535         active_count_av_reg <= active_count_reg <
2**STATUS_FIFO_ADDR_WIDTH;
536     end
537
538     if (rst) begin
539         state_reg <= STATE_IDLE;
540
541         s_axis_write_desc_ready_reg <= 1'b0;
542         m_axis_write_desc_status_valid_reg <= 1'b0;
543
544         s_axis_write_data_tready_reg <= 1'b0;
545
546         status_fifo_wr_ptr_reg <= 0;
547         status_fifo_rd_ptr_reg <= 0;
548
549         active_count_reg <= 0;
550         active_count_av_reg <= 1'b1;
551     end
552 end
553
554 // output datapath logic (write data)
555 generate
556     genvar n;
557     for (n = 0; n < SEG_COUNT; n = n + 1) begin
558         reg [SEG_BE_WIDTH-1:0] ram_wr_cmd_be_reg =
{SEG_BE_WIDTH{1'b0}};
559         reg [SEG_ADDR_WIDTH-1:0] ram_wr_cmd_addr_reg = {SEG_AD-
DR_WIDTH{1'b0}};
560         reg [SEG_DATA_WIDTH-1:0] ram_wr_cmd_data_reg =
{SEG_DATA_WIDTH{1'b0}};
561         reg ram_wr_cmd_valid_reg = 1'b0;
562
563         reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] out_fifo_wr_ptr_reg =
0;
564         reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] out_fifo_rd_ptr_reg =
0;
565         reg out_fifo_half_full_reg = 1'b0;
566
567         wire out_fifo_full = out_fifo_wr_ptr_reg ==
(out_fifo_rd_ptr_reg ^ {1'b1, {OUTPUT_FIFO_ADDR_WIDTH{1'b0}}});
568         wire out_fifo_empty = out_fifo_wr_ptr_reg ==
out_fifo_rd_ptr_reg;
569
570         (* ram_style = "distributed", ramstyle = "no_rw_check,
mlab" *)
571         reg [SEG_BE_WIDTH-1:0]
out_fifo_wr_cmd_be[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
572         (* ram_style = "distributed", ramstyle = "no_rw_check,
mlab" *)
573         reg [SEG_ADDR_WIDTH-1:0]
out_fifo_wr_cmd_addr[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
574         (* ram_style = "distributed", ramstyle = "no_rw_check,
mlab" *)
575         reg [SEG_DATA_WIDTH-1:0]
out_fifo_wr_cmd_data[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
576         reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] done_count_reg = 0;
577         reg done_reg = 1'b0;
578         assign ram_wr_cmd_ready_int[n +: 1] = !
out_fifo_half_full_reg;
579         assign ram_wr_cmd_be[n*SEG_BE_WIDTH +: SEG_BE_WIDTH] =
ram_wr_cmd_be_reg;
580         assign ram_wr_cmd_addr[n*SEG_ADDR_WIDTH +: SEG_ADDR_WIDTH] =
ram_wr_cmd_addr_reg;
581         assign ram_wr_cmd_data[n*SEG_DATA_WIDTH +: SEG_DATA_WIDTH] =
ram_wr_cmd_data_reg;
582         assign ram_wr_cmd_valid[n +: 1] = ram_wr_cmd_valid_reg;
583         assign out_done[n] = done_reg;
584         always @(posedge clk) begin
585             ram_wr_cmd_valid_reg <= ram_wr_cmd_valid_reg && !
ram_wr_cmd_ready[n +: 1];
586
587             out_fifo_half_full_reg <= $un-
signed(out_fifo_wr_ptr_reg - out_fifo_rd_ptr_reg) >= 2**(OUT-
PUT_FIFO_ADDR_WIDTH-1);
588
589             if (!out_fifo_full && ram_wr_cmd_valid_int[n +: 1])
begin
590                 out_fifo_wr_cmd_be[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
<= ram_wr_cmd_be_int[n*SEG_BE_WIDTH +: SEG_BE_WIDTH];
591                 out_fifo_wr_cmd_addr[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
<= ram_wr_cmd_addr_int[n*SEG_ADDR_WIDTH +: SEG_ADDR_WIDTH];
592                 out_fifo_wr_cmd_data[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
<= ram_wr_cmd_data_int[n*SEG_DATA_WIDTH +: SEG_DATA_WIDTH];
593             end
594         end
595     end
596 endgenerate
597 endmodule
598
599 `resetall
600

```

```

out_fifo_wr_cmd_data[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
] <= ram_wr_cmd_data_int[n*SEG_DATA_WIDTH +: SEG_DATA_WIDTH];
601     out_fifo_wr_ptr_reg <= out_fifo_wr_ptr_reg + 1;
602 end
603
604     if (!out_fifo_empty && (!ram_wr_cmd_valid_reg ||
ram_wr_cmd_ready[n +: 1])) begin
605         ram_wr_cmd_be_reg <=
out_fifo_wr_cmd_be[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
606         ram_wr_cmd_addr_reg <=
out_fifo_wr_cmd_addr[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
];
607         ram_wr_cmd_data_reg <=
out_fifo_wr_cmd_data[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
];
608         ram_wr_cmd_valid_reg <= 1'b1;
609         out_fifo_rd_ptr_reg <= out_fifo_rd_ptr_reg + 1;
610     end
611
612     if (done_count_reg < 2**OUTPUT_FIFO_ADDR_WIDTH &&
ram_wr_done[n] && !out_done_ack[n]) begin
613         done_count_reg <= done_count_reg + 1;
614         done_reg <= 1;
615     end else if (done_count_reg > 0 && !ram_wr_done[n] &&
out_done_ack[n]) begin
616         done_count_reg <= done_count_reg - 1;
617         done_reg <= done_count_reg > 1;
618     end
619
620     if (rst) begin
621         out_fifo_wr_ptr_reg <= 0;
622         out_fifo_rd_ptr_reg <= 0;
623         ram_wr_cmd_valid_reg <= 1'b0;
624         done_count_reg <= 0;
625         done_reg <= 1'b0;
626     end
627 end
628
629 end
630
631 endgenerate
632
633 endmodule
634
635 `resetall

```

4. alexforench_verilog-pcie/rtl/dma_client_axis_source.v

(568 lines)

```

1 /*
2
3 Copyright (c) 2019-2021 Alex Forench
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * AXI stream source DMA client
33 */
34 module dma_client_axis_source #
35 (
36     // RAM address width
37     parameter RAM_ADDR_WIDTH = 16,
38     // RAM segment count

```

```

39 parameter SEG_COUNT = 2,
40 // RAM segment data width
41 parameter SEG_DATA_WIDTH = 64,
42 // RAM segment byte enable width
43 parameter SEG_BE_WIDTH = SEG_DATA_WIDTH/8,
44 // RAM segment address width
45 parameter SEG_ADDR_WIDTH = RAM_ADDR_WIDTH-$clog2(
g2(SEG_COUNT*SEG_BE_WIDTH)),
46 // Width of AXI stream interfaces in bits
47 parameter AXIS_DATA_WIDTH = SEG_DATA_WIDTH*SEG_COUNT/2,
48 // Use AXI stream tkeep signal
49 parameter AXIS_KEEP_ENABLE = (AXIS_DATA_WIDTH>8),
50 // AXI stream tkeep signal width (words per cycle)
51 parameter AXIS_KEEP_WIDTH = (AXIS_DATA_WIDTH/8),
52 // Use AXI stream tlast signal
53 parameter AXIS_LAST_ENABLE = 1,
54 // Propagate AXI stream tid signal
55 parameter AXIS_ID_ENABLE = 0,
56 // AXI stream tid signal width
57 parameter AXIS_ID_WIDTH = 8,
58 // Propagate AXI stream tdest signal
59 parameter AXIS_DEST_ENABLE = 0,
60 // AXI stream tdest signal width
61 parameter AXIS_DEST_WIDTH = 8,
62 // Propagate AXI stream tuser signal
63 parameter AXIS_USER_ENABLE = 1,
64 // AXI stream tuser signal width
65 parameter AXIS_USER_WIDTH = 1,
66 // Width of length field
67 parameter LEN_WIDTH = 16,
68 // Width of tag field
69 parameter TAG_WIDTH = 8
70 )
71 (
72 input wire clk,
73 input wire rst,
74
75 /*
76 * AXI read descriptor input
77 */
78 input wire [RAM_ADDR_WIDTH-1:0] s_ax-
is_read_desc_ram_addr,
79 input wire [LEN_WIDTH-1:0] s_ax-
is_read_desc_len,
80 input wire [TAG_WIDTH-1:0] s_ax-
is_read_desc_tag,
81 input wire [AXIS_ID_WIDTH-1:0] s_ax-
is_read_desc_id,
82 input wire [AXIS_DEST_WIDTH-1:0] s_ax-
is_read_desc_dest,
83 input wire [AXIS_USER_WIDTH-1:0] s_ax-
is_read_desc_user,
84 input wire s_ax-
is_read_desc_valid,
85 output wire s_ax-
is_read_desc_ready,
86
87 /*
88 * AXI read descriptor status output
89 */
90 output wire [TAG_WIDTH-1:0] m_ax-
is_read_desc_status_tag,
91 output wire [3:0] m_ax-
is_read_desc_status_error,
92 output wire m_ax-
is_read_desc_status_valid,
93
94 /*
95 * AXI stream read data output
96 */
97 output wire [AXIS_DATA_WIDTH-1:0] m_ax-
is_read_data_tdata,
98 output wire [AXIS_KEEP_WIDTH-1:0] m_ax-
is_read_data_tkeep,
99 output wire m_ax-
is_read_data_tvalid,
100 input wire m_ax-
is_read_data_tready,
101 output wire m_ax-
is_read_data_tlast,
102 output wire [AXIS_ID_WIDTH-1:0] m_ax-
is_read_data_tid,
103 output wire [AXIS_DEST_WIDTH-1:0] m_ax-
is_read_data_tdest,
104 output wire [AXIS_USER_WIDTH-1:0] m_ax-
is_read_data_tuser,
105
106 /*
107 * RAM interface
108 */
109 output wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0] ram_rd_cmd_ad-
dr,
110 output wire [SEG_COUNT-1:0] ram_rd_cmd_valid,
111 input wire [SEG_COUNT-1:0] ram_rd_cmd_ready,
112 input wire [SEG_COUNT*SEG_DATA_WIDTH-1:0] ram_rd_resp_data,
113 input wire [SEG_COUNT-1:0] ram_rd_resp_valid,
114 output wire [SEG_COUNT-1:0]

```

```

ram_rd_resp_ready,
115
116 /*
117 * Configuration
118 */
119 input wire enable
120 );
121
122 parameter RAM_WORD_WIDTH = SEG_BE_WIDTH;
123 parameter RAM_WORD_SIZE = SEG_DATA_WIDTH/RAM_WORD_WIDTH;
124
125 parameter AXIS_KEEP_WIDTH_INT = AXIS_KEEP_ENABLE ?
AXIS_KEEP_WIDTH : 1;
126 parameter AXIS_WORD_WIDTH = AXIS_KEEP_WIDTH_INT;
127 parameter AXIS_WORD_SIZE = AXIS_DATA_WIDTH/AXIS_WORD_WIDTH;
128
129 parameter PART_COUNT = SEG_COUNT*SEG_BE_WIDTH /
AXIS_KEEP_WIDTH_INT;
130 parameter PART_COUNT_WIDTH = PART_COUNT > 1 ? $clog2-
g2(PART_COUNT) : 1;
131 parameter PART_OFFSET_WIDTH = AXIS_KEEP_WIDTH_INT > 1 ? $clog2-
g2(AXIS_KEEP_WIDTH_INT) : 1;
132 parameter PARTS_PER_SEG = (SEG_BE_WIDTH + AXIS_KEEP_WIDTH_INT
- 1) / AXIS_KEEP_WIDTH_INT;
133 parameter SEG_SIZE_PER_PART = (AXIS_KEEP_WIDTH_INT + SEG_BE_WIDTH
- 1) / SEG_BE_WIDTH;
134
135 parameter OFFSET_WIDTH = AXIS_KEEP_WIDTH_INT > 1 ? $clog2-
g2(AXIS_KEEP_WIDTH_INT) : 1;
136 parameter OFFSET_MASK = AXIS_KEEP_WIDTH_INT > 1 ? {OFF-
SET_WIDTH{1'b1}} : 0;
137 parameter ADDR_MASK = {RAM_ADDR_WIDTH{1'b1}} << $clog2-
g2(AXIS_KEEP_WIDTH_INT);
138 parameter CYCLE_COUNT_WIDTH = LEN_WIDTH - $clog2-
g2(AXIS_KEEP_WIDTH_INT) + 1;
139
140 parameter OUTPUT_FIFO_ADDR_WIDTH = 5;
141
142 // bus width assertions
143 initial begin
144 if (RAM_WORD_SIZE * SEG_BE_WIDTH != SEG_DATA_WIDTH) begin
145 Error("Error: RAM data width not evenly divisble (in-
stance %m)");
146 $finish;
147 end
148
149 if (AXIS_WORD_SIZE * AXIS_KEEP_WIDTH_INT !=
AXIS_DATA_WIDTH) begin
150 Error("Error: AXI stream data width not evenly divis-
ble (instance %m)");
151 $finish;
152 end
153
154 if (RAM_WORD_SIZE != AXIS_WORD_SIZE) begin
155 Error("Error: word size mismatch (instance %m)");
156 $finish;
157 end
158
159 if (2**$clog2(RAM_WORD_WIDTH) != RAM_WORD_WIDTH) begin
160 Error("Error: RAM word width must be even power of
two (instance %m)");
161 $finish;
162 end
163
164 if (RAM_ADDR_WIDTH != SEG_ADDR_WIDTH+$clog2(SEG_COUNT)+
$clog2(SEG_BE_WIDTH)) begin
165 Error("Error: RAM_ADDR_WIDTH does not match RAM con-
figuration (instance %m)");
166 $finish;
167 end
168
169 if (AXIS_DATA_WIDTH > SEG_COUNT*SEG_DATA_WIDTH) begin
170 Error("Error: AXI stream interface width must not be
wider than RAM interface width (instance %m)");
171 $finish;
172 end
173
174 if (AXIS_DATA_WIDTH*2**$clog2(PART_COUNT) !=
SEG_COUNT*SEG_DATA_WIDTH) begin
175 Error("Error: AXI stream interface width must be a
power of two fraction of RAM interface width (instance %m)");
176 $finish;
177 end
178 end
179
180 localparam [0:0]
181 READ_STATE_IDLE = 1'd0,
182 READ_STATE_READ = 1'd1;
183
184 reg [0:0] read_state_reg = READ_STATE_IDLE, read_state_next;
185
186 localparam [0:0]
187 AXIS_STATE_IDLE = 1'd0,
188 AXIS_STATE_READ = 1'd1;
189
190 reg [0:0] axis_state_reg = AXIS_STATE_IDLE, axis_state_next;
191
192 // datapath control signals
193 reg axis_cmd_ready;
194
195 integer i;

```

```

196
197 reg [RAM_ADDR_WIDTH-1:0] read_addr_reg = {RAM_AD-
DR_WIDTH{1'b0}}, read_addr_next;
198 reg [SEG_COUNT-1:0] read_ram_mask_reg = 0, read_ram_mask_next;
199 reg [CYCLE_COUNT_WIDTH-1:0] read_cycle_count_reg =
{CYCLE_COUNT_WIDTH{1'b0}}, read_cycle_count_next;
200
201 reg [RAM_ADDR_WIDTH-1:0] axis_cmd_addr_reg = {RAM_AD-
DR_WIDTH{1'b0}}, axis_cmd_addr_next;
202 reg [OFFSET_WIDTH-1:0] axis_cmd_last_cycle_offset_reg = {OFF-
SET_WIDTH{1'b0}}, axis_cmd_last_cycle_offset_next;
203 reg [CYCLE_COUNT_WIDTH-1:0] axis_cmd_cycle_count_reg =
{CYCLE_COUNT_WIDTH{1'b0}}, axis_cmd_cycle_count_next;
204 reg [TAG_WIDTH-1:0] axis_cmd_tag_reg = {TAG_WIDTH{1'b0}},
axis_cmd_tag_next;
205 reg [AXIS_ID_WIDTH-1:0] axis_cmd_axis_id_reg =
{AXIS_ID_WIDTH{1'b0}}, axis_cmd_axis_id_next;
206 reg [AXIS_DEST_WIDTH-1:0] axis_cmd_axis_dest_reg = {AXIS_D-
EST_WIDTH{1'b0}}, axis_cmd_axis_dest_next;
207 reg [AXIS_USER_WIDTH-1:0] axis_cmd_axis_user_reg =
{AXIS_USER_WIDTH{1'b0}}, axis_cmd_axis_user_next;
208 reg axis_cmd_valid_reg = 1'b0, axis_cmd_valid_next;
209
210 reg [RAM_ADDR_WIDTH-1:0] addr_reg = {RAM_ADDR_WIDTH{1'b0}},
addr_next;
211 reg [SEG_COUNT-1:0] ram_mask_reg = 0, ram_mask_next;
212 reg [OFFSET_WIDTH-1:0] last_cycle_offset_reg = {OFF-
SET_WIDTH{1'b0}}, last_cycle_offset_next;
213 reg [CYCLE_COUNT_WIDTH-1:0] cycle_count_reg =
{CYCLE_COUNT_WIDTH{1'b0}}, cycle_count_next;
214 reg last_cycle_reg = 1'b0, last_cycle_next;
215
216 reg [AXIS_ID_WIDTH-1:0] axis_id_reg = {AXIS_ID_WIDTH{1'b0}},
axis_id_next;
217 reg [AXIS_DEST_WIDTH-1:0] axis_dest_reg = {AXIS_D-
EST_WIDTH{1'b0}}, axis_dest_next;
218 reg [AXIS_USER_WIDTH-1:0] axis_user_reg =
{AXIS_USER_WIDTH{1'b0}}, axis_user_next;
219
220 reg s_axis_read_desc_ready_reg = 1'b0, s_ax-
is_read_desc_ready_next;
221
222 reg [TAG_WIDTH-1:0] m_axis_read_desc_status_tag_reg =
{TAG_WIDTH{1'b0}}, m_axis_read_desc_status_tag_next;
223 reg m_axis_read_desc_status_valid_reg = 1'b0, m_ax-
is_read_desc_status_valid_next;
224
225 reg [SEG_COUNT*SEG_ADDR_WIDTH-1:0] ram_rd_cmd_addr_reg = 0,
ram_rd_cmd_addr_next;
226 reg [SEG_COUNT-1:0] ram_rd_cmd_valid_reg = 0, ram_rd_cmd_val-
id_next;
227 reg [SEG_COUNT-1:0] ram_rd_resp_ready_cmb;
228
229 // internal datapath
230 reg [AXIS_DATA_WIDTH-1:0] m_axis_read_data_tdata_int;
231 reg [AXIS_KEEP_WIDTH-1:0] m_axis_read_data_tkeep_int;
232 reg [AXIS_DATA_WIDTH-1:0] m_axis_read_data_tvalid_int;
233 wire [AXIS_DATA_WIDTH-1:0] m_axis_read_data_tready_int;
234 reg [AXIS_DATA_WIDTH-1:0] m_axis_read_data_tlast_int;
235 reg [AXIS_ID_WIDTH-1:0] m_axis_read_data_tid_int;
236 reg [AXIS_DEST_WIDTH-1:0] m_axis_read_data_tdest_int;
237 reg [AXIS_USER_WIDTH-1:0] m_axis_read_data_tuser_int;
238
239 assign s_axis_read_desc_ready = s_axis_read_desc_ready_reg;
240
241 assign m_axis_read_desc_status_tag = m_ax-
is_read_desc_status_tag_reg;
242 assign m_axis_read_desc_status_error = 4'd0;
243 assign m_axis_read_desc_status_valid = m_ax-
is_read_desc_status_valid_reg;
244
245 assign ram_rd_cmd_addr = ram_rd_cmd_addr_reg;
246 assign ram_rd_cmd_valid = ram_rd_cmd_valid_reg;
247 assign ram_rd_resp_ready = ram_rd_resp_ready_cmb;
248
249 always @* begin
250   read_state_next = READ_STATE_IDLE;
251
252   s_axis_read_desc_ready_next = 1'b0;
253
254   ram_rd_cmd_addr_next = ram_rd_cmd_addr_reg;
255   ram_rd_cmd_valid_next = ram_rd_cmd_valid_reg &
~ram_rd_cmd_ready;
256
257   read_addr_next = read_addr_reg;
258   read_ram_mask_next = read_ram_mask_reg;
259   read_cycle_count_next = read_cycle_count_reg;
260
261   axis_cmd_addr_next = axis_cmd_addr_reg;
262   axis_cmd_last_cycle_offset_next =
axis_cmd_last_cycle_offset_reg;
263   axis_cmd_cycle_count_next = axis_cmd_cycle_count_reg;
264   axis_cmd_tag_next = axis_cmd_tag_reg;
265   axis_cmd_axis_id_next = axis_cmd_axis_id_reg;
266   axis_cmd_axis_dest_next = axis_cmd_axis_dest_reg;
267   axis_cmd_axis_user_next = axis_cmd_axis_user_reg;
268   axis_cmd_valid_next = axis_cmd_valid_reg && !
axis_cmd_ready;
269
270   case (read_state_reg)
271     READ_STATE_IDLE: begin

```

```

272   // idle state - load new descriptor to start oper-
ation
273   s_axis_read_desc_ready_next = !axis_cmd_valid_reg
&& enable;
274
275   if (s_axis_read_desc_ready && s_ax-
is_read_desc_valid) begin
276     read_addr_next = s_axis_read_desc_ram_addr &
ADDR_MASK;
277
278     if (PART_COUNT > 1) begin
279
280       read_ram_mask_next = {SEGS_PER_PART{1'b1}}
<< (((read_addr_next >> PART_OFFSET_WIDTH) &
({PART_COUNT_WIDTH{1'b1}})) / PARTS_PER_SEG) * SEGS_PER_PART);
281       end else begin
282         read_ram_mask_next = {SEG_COUNT{1'b1}};
283       end
284
285       axis_cmd_addr_next = s_axis_read_desc_ram_addr &
ADDR_MASK;
286       axis_cmd_last_cycle_offset_next = s_ax-
is_read_desc_len & OFFSET_MASK;
287
288       axis_cmd_tag_next = s_axis_read_desc_tag;
289
290       axis_cmd_axis_id_next = s_axis_read_desc_id;
291       axis_cmd_axis_dest_next = s_ax-
is_read_desc_dest;
292       axis_cmd_axis_user_next = s_ax-
is_read_desc_user;
293
294       axis_cmd_cycle_count_next = (s_ax-
is_read_desc_len - 1) >> $clog2(AXIS_KEEP_WIDTH_INT);
295       read_cycle_count_next = (s_axis_read_desc_len
- 1) >> $clog2(AXIS_KEEP_WIDTH_INT);
296
297       axis_cmd_valid_next = 1'b1;
298
299       s_axis_read_desc_ready_next = 1'b0;
300       read_state_next = READ_STATE_READ;
301     end else begin
302       read_state_next = READ_STATE_IDLE;
303     end
304   end
305   READ_STATE_READ: begin
306     // read state - start new read operations
307
308     if (!(ram_rd_cmd_valid & ~ram_rd_cmd_ready &
read_ram_mask_reg)) begin
309
310       // update counters
311       read_addr_next = read_addr_reg +
AXIS_KEEP_WIDTH_INT;
312
313       read_cycle_count_next = read_cycle_count_reg -
1;
314
315       if (PART_COUNT > 1) begin
316
317         read_ram_mask_next = {SEGS_PER_PART{1'b1}}
<< (((read_addr_next >> PART_OFFSET_WIDTH) &
({PART_COUNT_WIDTH{1'b1}})) / PARTS_PER_SEG) * SEGS_PER_PART);
318         end else begin
319           read_ram_mask_next = {SEG_COUNT{1'b1}};
320         end
321
322         for (i = 0; i < SEG_COUNT; i = i + 1) begin
323           if (read_ram_mask_reg[i]) begin
324             ram_rd_cmd_addr_next[i*SEG_ADDR_WIDTH
+: SEG_ADDR_WIDTH] = read_addr_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-
SEG_ADDR_WIDTH];
325             ram_rd_cmd_valid_next[i] = 1'b1;
326           end
327         end
328
329         if (read_cycle_count_reg == 0) begin
330           s_axis_read_desc_ready_next = !
axis_cmd_valid_reg && enable;
331           read_state_next = READ_STATE_IDLE;
332         end else begin
333           read_state_next = READ_STATE_READ;
334         end
335       end
336     end
337   endcase
338 end
339
340 always @* begin
341   axis_state_next = AXIS_STATE_IDLE;
342
343   m_axis_read_desc_status_tag_next = m_ax-
is_read_desc_status_tag_reg;
344   m_axis_read_desc_status_valid_next = 1'b0;
345
346   if (PART_COUNT > 1) begin
347     m_axis_read_data_rdata_int = ram_rd_resp_data >>

```



```

(((addr_reg >> PART_OFFSET_WIDTH) & {PART_COUNT_WIDTH{1'b1}}) *
AXIS_DATA_WIDTH);
348 end else begin
349     m_axis_read_data_tdata_int = ram_rd_resp_data;
350 end
351 m_axis_read_data_tkeep_int = {AXIS_KEEP_WIDTH{1'b1}};
352 m_axis_read_data_tlast_int = 1'b0;
353 m_axis_read_data_tvalid_int = 1'b0;
354 m_axis_read_data_tid_int = axis_id_reg;
355 m_axis_read_data_tdest_int = axis_dest_reg;
356 m_axis_read_data_tuser_int = axis_user_reg;
357
358 ram_rd_resp_ready_cmb = {SEG_COUNT{1'b0}};
359
360 axis_cmd_ready = 1'b0;
361
362 addr_next = addr_reg;
363 ram_mask_next = ram_mask_reg;
364 last_cycle_offset_next = last_cycle_offset_reg;
365 cycle_count_next = cycle_count_reg;
366 last_cycle_next = last_cycle_reg;
367
368 axis_id_next = axis_id_reg;
369 axis_dest_next = axis_dest_reg;
370 axis_user_next = axis_user_reg;
371
372 case (axis_state_reg)
373     AXIS_STATE_IDLE: begin
374         // idle state - load new descriptor to start oper-
375         ation
376         // store transfer parameters
377         addr_next = axis_cmd_addr_reg;
378         last_cycle_offset_next =
axis_cmd_last_cycle_offset_reg;
379         cycle_count_next = axis_cmd_cycle_count_reg;
380         last_cycle_next = axis_cmd_cycle_count_reg == 0;
381
382         if (PART_COUNT > 1) begin
383             ram_mask_next = {SEGS_PER_PART{1'b1}} <<
(((addr_next >> PART_OFFSET_WIDTH) & ({PART_COUNT_WIDTH{1'b1}})) /
PARTS_PER_SEG) * SEGS_PER_PART;
384         end else begin
385             ram_mask_next = {SEG_COUNT{1'b1}};
386         end
387
388         m_axis_read_desc_status_tag_next =
axis_cmd_tag_reg;
389         axis_id_next = axis_cmd_axis_id_reg;
390         axis_dest_next = axis_cmd_axis_dest_reg;
391         axis_user_next = axis_cmd_axis_user_reg;
392
393         if (axis_cmd_valid_reg) begin
394             axis_cmd_ready = 1'b1;
395             axis_state_next = AXIS_STATE_READ;
396         end
397     end
398     AXIS_STATE_READ: begin
399         // handle read data
400         ram_rd_resp_ready_cmb = {SEG_COUNT{1'b0}};
401
402         if (!(ram_mask_reg & ~ram_rd_resp_valid) && m_ax-
is_read_data_tready_int) begin
403             // transfer in read data
404             ram_rd_resp_ready_cmb = ram_mask_reg;
405
406             // update counters
407             addr_next = addr_reg + AXIS_KEEP_WIDTH_INT;
408             cycle_count_next = cycle_count_reg - 1;
409             last_cycle_next = cycle_count_next == 0;
410
411             if (PART_COUNT > 1) begin
412                 ram_mask_next = {SEGS_PER_PART{1'b1}} <<
(((addr_next >> PART_OFFSET_WIDTH) & ({PART_COUNT_WIDTH{1'b1}})) /
PARTS_PER_SEG) * SEGS_PER_PART;
413             end else begin
414                 ram_mask_next = {SEG_COUNT{1'b1}};
415             end
416
417             if (PART_COUNT > 1) begin
418                 m_axis_read_data_tdata_int =
ram_rd_resp_data >> (((addr_reg >> PART_OFFSET_WIDTH) &
{PART_COUNT_WIDTH{1'b1}}) * AXIS_DATA_WIDTH);
419             end else begin
420                 m_axis_read_data_tdata_int =
ram_rd_resp_data;
421             end
422             m_axis_read_data_tkeep_int =
{AXIS_KEEP_WIDTH_INT{1'b1}};
423             m_axis_read_data_tvalid_int = 1'b1;
424
425             if (last_cycle_reg) begin
426                 // no more data to transfer, finish opera-
427                 tion
428                 if (last_cycle_offset_reg > 0) begin
429                     m_axis_read_data_tkeep_int =
{AXIS_KEEP_WIDTH_INT{1'b1}} >> (AXIS_KEEP_WIDTH_INT -
last_cycle_offset_reg);
430                 end
431                 m_axis_read_data_tlast_int = 1'b1;
432
433                 m_axis_read_desc_status_valid_next = 1'b1;
434
435                 axis_state_next = AXIS_STATE_IDLE;
436             end else begin
437                 // more cycles in AXI transfer
438                 axis_state_next = AXIS_STATE_READ;
439             end
440         end else begin
441             axis_state_next = AXIS_STATE_READ;
442         end
443     endcase
444 end
445
446 always @(posedge clk) begin
447     read_state_reg <= read_state_next;
448     axis_state_reg <= axis_state_next;
449
450     s_axis_read_desc_ready_reg <= s_axis_read_desc_ready_next;
451
452     m_axis_read_desc_status_tag_reg <= m_ax-
is_read_desc_status_tag_next;
453     m_axis_read_desc_status_valid_reg <= m_ax-
is_read_desc_status_valid_next;
454
455     ram_rd_cmd_addr_reg <= ram_rd_cmd_addr_next;
456     ram_rd_cmd_valid_reg <= ram_rd_cmd_valid_next;
457
458     read_addr_reg <= read_addr_next;
459     read_ram_mask_reg <= read_ram_mask_next;
460     read_cycle_count_reg <= read_cycle_count_next;
461
462     axis_cmd_addr_reg <= axis_cmd_addr_next;
463     axis_cmd_last_cycle_offset_reg <=
axis_cmd_last_cycle_offset_next;
464     axis_cmd_cycle_count_reg <= axis_cmd_cycle_count_next;
465     axis_cmd_tag_reg <= axis_cmd_tag_next;
466     axis_cmd_axis_id_reg <= axis_cmd_axis_id_next;
467     axis_cmd_axis_dest_reg <= axis_cmd_axis_dest_next;
468     axis_cmd_axis_user_reg <= axis_cmd_axis_user_next;
469     axis_cmd_valid_reg <= axis_cmd_valid_next;
470
471     addr_reg <= addr_next;
472     ram_mask_reg <= ram_mask_next;
473     last_cycle_offset_reg <= last_cycle_offset_next;
474     cycle_count_reg <= cycle_count_next;
475     last_cycle_reg <= last_cycle_next;
476
477     axis_id_reg <= axis_id_next;
478     axis_dest_reg <= axis_dest_next;
479     axis_user_reg <= axis_user_next;
480
481     if (rst) begin
482         read_state_reg <= READ_STATE_IDLE;
483         axis_state_reg <= AXIS_STATE_IDLE;
484
485         axis_cmd_valid_reg <= 1'b0;
486
487         s_axis_read_desc_ready_reg <= 1'b0;
488         m_axis_read_desc_status_valid_reg <= 1'b0;
489
490         ram_rd_cmd_valid_reg <= {SEG_COUNT{1'b0}};
491     end
492 end
493
494 // output datapath logic
495 reg [AXIS_DATA_WIDTH-1:0] m_axis_read_data_tdata_reg =
{AXIS_DATA_WIDTH{1'b0}};
496 reg [AXIS_KEEP_WIDTH-1:0] m_axis_read_data_tkeep_reg =
{AXIS_KEEP_WIDTH{1'b0}};
497 reg [AXIS_ID_WIDTH-1:0] m_axis_read_data_tvalid_reg = 1'b0;
498 reg [AXIS_DEST_WIDTH-1:0] m_axis_read_data_tlast_reg = 1'b0;
499 reg [AXIS_ID_WIDTH-1:0] m_axis_read_data_tid_reg =
{AXIS_ID_WIDTH{1'b0}};
500 reg [AXIS_DEST_WIDTH-1:0] m_axis_read_data_tdest_reg =
{AXIS_DEST_WIDTH{1'b0}};
501 reg [AXIS_USER_WIDTH-1:0] m_axis_read_data_tuser_reg =
{AXIS_USER_WIDTH{1'b0}};
502
503 reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] out_fifo_wr_ptr_reg = 0;
504 reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] out_fifo_rd_ptr_reg = 0;
505 reg out_fifo_half_full_reg = 1'b0;
506
507 wire out_fifo_full = out_fifo_wr_ptr_reg ==
(out_fifo_rd_ptr_reg ^ {1'b1, {OUTPUT_FIFO_ADDR_WIDTH{1'b0}}});
508 wire out_fifo_empty = out_fifo_wr_ptr_reg ==
out_fifo_rd_ptr_reg;
509
510 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
511 reg [AXIS_DATA_WIDTH-1:0]
out_fifo_tdata[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
512 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
513 reg [AXIS_KEEP_WIDTH-1:0]
out_fifo_tkeep[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
514 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
515 reg
out_fifo_tlast[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
516 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"

```

```

*)
517 reg [AXIS_ID_WIDTH-1:0]
out_fifo_tid[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
518 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
519 reg [AXIS_DEST_WIDTH-1:0]
out_fifo_tdest[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
520 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
521 reg [AXIS_USER_WIDTH-1:0]
out_fifo_tuser[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
522
523 assign m_axis_read_data_tready_int = !out_fifo_half_full_reg;
524
525 assign m_axis_read_data_tdata = m_axis_read_data_tdata_reg;
526 assign m_axis_read_data_tkeep = AXIS_KEEP_ENABLE ? m_ax-
is_read_data_tkeep_reg : {AXIS_KEEP_WIDTH{1'b1}};
527 assign m_axis_read_data_tvalid = m_axis_read_data_tvalid_reg;
528 assign m_axis_read_data_tlast = AXIS_LAST_ENABLE ? m_ax-
is_read_data_tlast_reg : 1'b1;
529 assign m_axis_read_data_tid = AXIS_ID_ENABLE ? m_ax-
is_read_data_tid_reg : {AXIS_ID_WIDTH{1'b0}};
530 assign m_axis_read_data_tdest = AXIS_DEST_ENABLE ? m_ax-
is_read_data_tdest_reg : {AXIS_DEST_WIDTH{1'b0}};
531 assign m_axis_read_data_tuser = AXIS_USER_ENABLE ? m_ax-
is_read_data_tuser_reg : {AXIS_USER_WIDTH{1'b0}};
532
533 always @(posedge clk) begin
534     m_axis_read_data_tvalid_reg <= m_axis_read_data_tvalid_reg
&& !m_axis_read_data_tready;
535
536     out_fifo_half_full_reg <= $unsigned(out_fifo_wr_ptr_reg -
out_fifo_rd_ptr_reg) >= 2**(OUTPUT_FIFO_ADDR_WIDTH-1);
537
538     if (!out_fifo_full && m_axis_read_data_tvalid_int) begin
539
out_fifo_tdata[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axis_read_data_tdata_int;
540
out_fifo_tkeep[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axis_read_data_tkeep_int;
541
out_fifo_tlast[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axis_read_data_tlast_int;
542
out_fifo_tid[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axis_read_data_tid_int;
543
out_fifo_tdest[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axis_read_data_tdest_int;
544
out_fifo_tuser[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axis_read_data_tuser_int;
545         out_fifo_wr_ptr_reg <= out_fifo_wr_ptr_reg + 1;
546     end
547
548     if (!out_fifo_empty && (!m_axis_read_data_tvalid_reg ||
m_axis_read_data_tready)) begin
549         m_axis_read_data_tdata_reg <=
out_fifo_tdata[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
550         m_axis_read_data_tkeep_reg <=
out_fifo_tkeep[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
551         m_axis_read_data_tvalid_reg <= 1'b1;
552         m_axis_read_data_tlast_reg <=
out_fifo_tlast[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
553         m_axis_read_data_tid_reg <=
out_fifo_tid[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
554         m_axis_read_data_tdest_reg <=
out_fifo_tdest[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
555         m_axis_read_data_tuser_reg <=
out_fifo_tuser[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
556         out_fifo_rd_ptr_reg <= out_fifo_rd_ptr_reg + 1;
557     end
558
559     if (rst) begin
560         out_fifo_wr_ptr_reg <= 0;
561         out_fifo_rd_ptr_reg <= 0;
562         m_axis_read_data_tvalid_reg <= 1'b0;
563     end
564 end
565 endmodule
566
567 `resetall
568

```

5. alexforenrich_verilog-pcie/rtl/dma_if_axi.v (426 lines)

```

1 /*
2
3 Copyright (c) 2021 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell

```

```

9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * AXI DMA interface
33 */
34 module dma_if_axi #
35 (
36     // Width of AXI data bus in bits
37     parameter AXI_DATA_WIDTH = 32,
38     // Width of AXI address bus in bits
39     parameter AXI_ADDR_WIDTH = 16,
40     // Width of AXI wstrb (width of data bus in words)
41     parameter AXI_STRB_WIDTH = (AXI_DATA_WIDTH/8),
42     // Width of AXI ID signal
43     parameter AXI_ID_WIDTH = 8,
44     // Maximum AXI burst length to generate
45     parameter AXI_MAX_BURST_LEN = 256,
46     // RAM select width
47     parameter RAM_SEL_WIDTH = 2,
48     // RAM address width
49     parameter RAM_ADDR_WIDTH = 16,
50     // RAM segment count
51     parameter RAM_SEG_COUNT = 2,
52     // RAM segment data width
53     parameter RAM_SEG_DATA_WIDTH = AXI_DATA_WIDTH*2/
RAM_SEG_COUNT,
54     // RAM segment byte enable width
55     parameter RAM_SEG_BE_WIDTH = RAM_SEG_DATA_WIDTH/8,
56     // RAM segment address width
57     parameter RAM_SEG_ADDR_WIDTH = RAM_ADDR_WIDTH-$clo-
g2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH),
58     // Immediate enable
59     parameter IMM_ENABLE = 0,
60     // Immediate width
61     parameter IMM_WIDTH = 32,
62     // Length field width
63     parameter LEN_WIDTH = 16,
64     // Tag field width
65     parameter TAG_WIDTH = 8,
66     // Operation table size (read)
67     parameter READ_OP_TABLE_SIZE = 2**(AXI_ID_WIDTH),
68     // Operation table size (write)
69     parameter WRITE_OP_TABLE_SIZE = 2**(AXI_ID_WIDTH),
70     // Use AXI ID signals (read)
71     parameter READ_USE_AXI_ID = 0,
72     // Use AXI ID signals (write)
73     parameter WRITE_USE_AXI_ID = 1
74 )
75 (
76     input wire clk,
77     input wire rst,
78
79     /*
80     * AXI master interface
81     */
82     output wire [AXI_ID_WIDTH-1:0]
m_axi_awid,
83     output wire [AXI_ADDR_WIDTH-1:0]
m_axi_awaddr,
84     output wire [7:0]
m_axi_awlen,
85     output wire [2:0]
m_axi_awsiz,
86     output wire [1:0]
m_axi_awburst,
87     output wire
m_axi_awlock,
88     output wire [3:0]
m_axi_awcache,
89     output wire [2:0]
m_axi_awprot,
90     output wire

```

```

m_axi_awvalid,
91   input wire
m_axi_awready,
92   output wire [AXI_DATA_WIDTH-1:0]
m_axi_wdata,
93   output wire [AXI_STRB_WIDTH-1:0]
m_axi_wstrb,
94   output wire
m_axi_wlast,
95   output wire
m_axi_wvalid,
96   input wire
m_axi_wready,
97   input wire [AXI_ID_WIDTH-1:0]
m_axi_bid,
98   input wire [1:0]
m_axi_bresp,
99   input wire
m_axi_bvalid,
100  output wire
m_axi_bready,
101  output wire [AXI_ID_WIDTH-1:0]
m_axi_arid,
102  output wire [AXI_ADDR_WIDTH-1:0]
m_axi_araddr,
103  output wire [7:0]
m_axi_arlen,
104  output wire [2:0]
m_axi_arsize,
105  output wire [1:0]
m_axi_arburst,
106  output wire
m_axi_arlock,
107  output wire [3:0]
m_axi_arsize,
108  output wire [2:0]
m_axi_arprot,
109  output wire
m_axi_arvalid,
110  input wire
m_axi_arready,
111  input wire [AXI_ID_WIDTH-1:0]
m_axi_rid,
112  input wire [AXI_DATA_WIDTH-1:0]
m_axi_rdata,
113  input wire [1:0]
m_axi_rresp,
114  input wire
m_axi_rlast,
115  input wire
m_axi_rvalid,
116  output wire
m_axi_rready,
117
118  /*
119   * AXI read descriptor input
120   */
121  input wire [AXI_ADDR_WIDTH-1:0]
is_read_desc_axi_addr,
122  input wire [RAM_SEL_WIDTH-1:0]
is_read_desc_ram_sel,
123  input wire [RAM_ADDR_WIDTH-1:0]
is_read_desc_ram_addr,
124  input wire [LEN_WIDTH-1:0]
is_read_desc_len,
125  input wire [TAG_WIDTH-1:0]
is_read_desc_tag,
126  input wire
is_read_desc_valid,
127  output wire
is_read_desc_ready,
128
129  /*
130   * AXI read descriptor status output
131   */
132  output wire [TAG_WIDTH-1:0]
is_read_desc_status_tag,
133  output wire [3:0]
is_read_desc_status_error,
134  output wire
is_read_desc_status_valid,
135
136  /*
137   * AXI write descriptor input
138   */
139  input wire [AXI_ADDR_WIDTH-1:0]
is_write_desc_axi_addr,
140  input wire [RAM_SEL_WIDTH-1:0]
is_write_desc_ram_sel,
141  input wire [RAM_ADDR_WIDTH-1:0]
is_write_desc_ram_addr,
142  input wire [IMM_WIDTH-1:0]
is_write_desc_imm,
143  input wire
is_write_desc_imm_en,
144  input wire [LEN_WIDTH-1:0]
is_write_desc_len,
145  input wire [TAG_WIDTH-1:0]
is_write_desc_tag,
146  input wire
is_write_desc_valid,

```

```

147  output wire
is_write_desc_ready,
148
149  /*
150   * AXI write descriptor status output
151   */
152  output wire [TAG_WIDTH-1:0]
is_write_desc_status_tag,
153  output wire [3:0]
is_write_desc_status_error,
154  output wire
is_write_desc_status_valid,
155
156  /*
157   * RAM interface
158   */
159  output wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]
ram_wr_cmd_sel,
160  output wire [RAM_SEG_COUNT*RAM_SEG_BE_WIDTH-1:0]
ram_wr_cmd_be,
161  output wire [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0]
ram_wr_cmd_addr,
162  output wire [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0]
ram_wr_cmd_data,
163  output wire [RAM_SEG_COUNT-1:0]
ram_wr_cmd_valid,
164  input wire [RAM_SEG_COUNT-1:0]
ram_wr_cmd_ready,
165  input wire [RAM_SEG_COUNT-1:0]
ram_wr_done,
166  output wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]
ram_rd_cmd_sel,
167  output wire [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0]
ram_rd_cmd_addr,
168  output wire [RAM_SEG_COUNT-1:0]
ram_rd_cmd_valid,
169  input wire [RAM_SEG_COUNT-1:0]
ram_rd_cmd_ready,
170  input wire [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0]
ram_rd_resp_data,
171  input wire [RAM_SEG_COUNT-1:0]
ram_rd_resp_valid,
172  output wire [RAM_SEG_COUNT-1:0]
ram_rd_resp_ready,
173
174  /*
175   * Configuration
176   */
177  input wire
read_enable,
178  input wire
write_enable,
179
180  /*
181   * Status
182   */
183  output wire
status_rd_busy,
184  output wire
status_wr_busy,
185
186  /*
187   * Statistics
188   */
189  output wire [$clog2(READ_OP_TABLE_SIZE)-1:0]
stat_rd_op_start_tag,
190  output wire [LEN_WIDTH-1:0]
stat_rd_op_start_len,
191  output wire
stat_rd_op_start_valid,
192  output wire [$clog2(READ_OP_TABLE_SIZE)-1:0]
stat_rd_op_finish_tag,
193  output wire [3:0]
stat_rd_op_finish_status,
194  output wire
stat_rd_op_finish_valid,
195  output wire [$clog2(READ_OP_TABLE_SIZE)-1:0]
stat_rd_req_start_tag,
196  output wire [12:0]
stat_rd_req_start_len,
197  output wire
stat_rd_req_start_valid,
198  output wire [$clog2(READ_OP_TABLE_SIZE)-1:0]
stat_rd_req_finish_tag,
199  output wire [3:0]
stat_rd_req_finish_status,
200  output wire
stat_rd_req_finish_valid,
201  output wire
stat_rd_op_table_full,
202  output wire
stat_rd_tx_stall,
203  output wire [$clog2(WRITE_OP_TABLE_SIZE)-1:0]
stat_wr_op_start_tag,
204  output wire [LEN_WIDTH-1:0]
stat_wr_op_start_len,
205  output wire
stat_wr_op_start_valid,
206  output wire [$clog2(WRITE_OP_TABLE_SIZE)-1:0]
stat_wr_op_finish_tag,
207  output wire [3:0]

```



```

stat_wr_op_finish_status,
208     output wire
stat_wr_op_finish_valid,
209     output wire [$clog2(WRITE_OP_TABLE_SIZE)-1:0]
stat_wr_req_start_tag,
210     output wire [12:0]
stat_wr_req_start_len,
211     output wire
stat_wr_req_start_valid,
212     output wire [$clog2(WRITE_OP_TABLE_SIZE)-1:0]
stat_wr_req_finish_tag,
213     output wire [3:0]
stat_wr_req_finish_status,
214     output wire
stat_wr_req_finish_valid,
215     output wire
stat_wr_op_table_full,
216     output wire
stat_wr_tx_stall
217 );
218
219 dma_if_axi_rd #(
220     .AXI_DATA_WIDTH(AXI_DATA_WIDTH),
221     .AXI_ADDR_WIDTH(AXI_ADDR_WIDTH),
222     .AXI_STRB_WIDTH(AXI_STRB_WIDTH),
223     .AXI_ID_WIDTH(AXI_ID_WIDTH),
224     .AXI_MAX_BURST_LEN(AXI_MAX_BURST_LEN),
225     .RAM_SEL_WIDTH(RAM_SEL_WIDTH),
226     .RAM_ADDR_WIDTH(RAM_ADDR_WIDTH),
227     .RAM_SEG_COUNT(RAM_SEG_COUNT),
228     .RAM_SEG_DATA_WIDTH(RAM_SEG_DATA_WIDTH),
229     .RAM_SEG_BE_WIDTH(RAM_SEG_BE_WIDTH),
230     .RAM_SEG_ADDR_WIDTH(RAM_SEG_ADDR_WIDTH),
231     .LEN_WIDTH(LEN_WIDTH),
232     .TAG_WIDTH(TAG_WIDTH),
233     .OP_TABLE_SIZE(READ_OP_TABLE_SIZE),
234     .USE_AXI_ID(READ_USE_AXI_ID)
235 )
236 dma_if_axi_rd_inst (
237     .clk(clk),
238     .rst(rst),
239
240     /*
241      * AXI master interface
242      */
243     .m_axi_arid(m_axi_arid),
244     .m_axi_araddr(m_axi_araddr),
245     .m_axi_arlen(m_axi_arlen),
246     .m_axi_arsize(m_axi_arsize),
247     .m_axi_arburst(m_axi_arburst),
248     .m_axi_arlock(m_axi_arlock),
249     .m_axi_arcache(m_axi_arcache),
250     .m_axi_arprot(m_axi_arprot),
251     .m_axi_arvalid(m_axi_arvalid),
252     .m_axi_arready(m_axi_arready),
253     .m_axi_rid(m_axi_rid),
254     .m_axi_rdata(m_axi_rdata),
255     .m_axi_rresp(m_axi_rresp),
256     .m_axi_rlast(m_axi_rlast),
257     .m_axi_rvalid(m_axi_rvalid),
258     .m_axi_rready(m_axi_rready),
259
260     /*
261      * AXI read descriptor input
262      */
263     .s_axis_read_desc_axi_addr(s_axis_read_desc_axi_addr),
264     .s_axis_read_desc_ram_sel(s_axis_read_desc_ram_sel),
265     .s_axis_read_desc_ram_addr(s_axis_read_desc_ram_addr),
266     .s_axis_read_desc_len(s_axis_read_desc_len),
267     .s_axis_read_desc_tag(s_axis_read_desc_tag),
268     .s_axis_read_desc_valid(s_axis_read_desc_valid),
269     .s_axis_read_desc_ready(s_axis_read_desc_ready),
270
271     /*
272      * AXI read descriptor status output
273      */
274     .m_axis_read_desc_status_tag(m_axis_read_desc_status_tag),
275     .m_ax-
is_read_desc_status_error(m_axis_read_desc_status_error),
276     .m_ax-
is_read_desc_status_valid(m_axis_read_desc_status_valid),
277
278     /*
279      * RAM interface
280      */
281     .ram_wr_cmd_sel(ram_wr_cmd_sel),
282     .ram_wr_cmd_be(ram_wr_cmd_be),
283     .ram_wr_cmd_addr(ram_wr_cmd_addr),
284     .ram_wr_cmd_data(ram_wr_cmd_data),
285     .ram_wr_cmd_valid(ram_wr_cmd_valid),
286     .ram_wr_cmd_ready(ram_wr_cmd_ready),
287     .ram_wr_done(ram_wr_done),
288
289     /*
290      * Configuration
291      */
292     .enable(read_enable),
293
294     /*
295      * Status
296      */

```

```

297     .status_busy(status_rd_busy),
298
299     /*
300      * Statistics
301      */
302     .stat_rd_op_start_tag(stat_rd_op_start_tag),
303     .stat_rd_op_start_len(stat_rd_op_start_len),
304     .stat_rd_op_start_valid(stat_rd_op_start_valid),
305     .stat_rd_op_finish_tag(stat_rd_op_finish_tag),
306     .stat_rd_op_finish_status(stat_rd_op_finish_status),
307     .stat_rd_op_finish_valid(stat_rd_op_finish_valid),
308     .stat_rd_req_start_tag(stat_rd_req_start_tag),
309     .stat_rd_req_start_len(stat_rd_req_start_len),
310     .stat_rd_req_start_valid(stat_rd_req_start_valid),
311     .stat_rd_req_finish_tag(stat_rd_req_finish_tag),
312     .stat_rd_req_finish_status(stat_rd_req_finish_status),
313     .stat_rd_req_finish_valid(stat_rd_req_finish_valid),
314     .stat_rd_op_table_full(stat_rd_op_table_full),
315     .stat_rd_tx_stall(stat_rd_tx_stall)
316 );
317
318 dma_if_axi_wr #(
319     .AXI_DATA_WIDTH(AXI_DATA_WIDTH),
320     .AXI_ADDR_WIDTH(AXI_ADDR_WIDTH),
321     .AXI_STRB_WIDTH(AXI_STRB_WIDTH),
322     .AXI_ID_WIDTH(AXI_ID_WIDTH),
323     .AXI_MAX_BURST_LEN(AXI_MAX_BURST_LEN),
324     .RAM_SEL_WIDTH(RAM_SEL_WIDTH),
325     .RAM_ADDR_WIDTH(RAM_ADDR_WIDTH),
326     .RAM_SEG_COUNT(RAM_SEG_COUNT),
327     .RAM_SEG_DATA_WIDTH(RAM_SEG_DATA_WIDTH),
328     .RAM_SEG_BE_WIDTH(RAM_SEG_BE_WIDTH),
329     .RAM_SEG_ADDR_WIDTH(RAM_SEG_ADDR_WIDTH),
330     .IMM_ENABLE(IMM_ENABLE),
331     .IMM_WIDTH(IMM_WIDTH),
332     .LEN_WIDTH(LEN_WIDTH),
333     .TAG_WIDTH(TAG_WIDTH),
334     .OP_TABLE_SIZE(WRITE_OP_TABLE_SIZE),
335     .USE_AXI_ID(WRITE_USE_AXI_ID)
336 )
337 dma_if_axi_wr_inst (
338     .clk(clk),
339     .rst(rst),
340
341     /*
342      * AXI master interface
343      */
344     .m_axi_awid(m_axi_awid),
345     .m_axi_awaddr(m_axi_awaddr),
346     .m_axi_awlen(m_axi_awlen),
347     .m_axi_awsz(m_axi_awsz),
348     .m_axi_awburst(m_axi_awburst),
349     .m_axi_awlock(m_axi_awlock),
350     .m_axi_awcache(m_axi_awcache),
351     .m_axi_awprot(m_axi_awprot),
352     .m_axi_awvalid(m_axi_awvalid),
353     .m_axi_awready(m_axi_awready),
354     .m_axi_wdata(m_axi_wdata),
355     .m_axi_wstrb(m_axi_wstrb),
356     .m_axi_wlast(m_axi_wlast),
357     .m_axi_wvalid(m_axi_wvalid),
358     .m_axi_wready(m_axi_wready),
359     .m_axi_bid(m_axi_bid),
360     .m_axi_bresp(m_axi_bresp),
361     .m_axi_bvalid(m_axi_bvalid),
362     .m_axi_bready(m_axi_bready),
363
364     /*
365      * AXI write descriptor input
366      */
367     .s_axis_write_desc_axi_addr(s_axis_write_desc_axi_addr),
368     .s_axis_write_desc_ram_sel(s_axis_write_desc_ram_sel),
369     .s_axis_write_desc_ram_addr(s_axis_write_desc_ram_addr),
370     .s_axis_write_desc_imm(s_axis_write_desc_imm),
371     .s_axis_write_desc_imm_en(s_axis_write_desc_imm_en),
372     .s_axis_write_desc_len(s_axis_write_desc_len),
373     .s_axis_write_desc_tag(s_axis_write_desc_tag),
374     .s_axis_write_desc_valid(s_axis_write_desc_valid),
375     .s_axis_write_desc_ready(s_axis_write_desc_ready),
376
377     /*
378      * AXI write descriptor status output
379      */
380     .m_ax-
is_write_desc_status_tag(m_axis_write_desc_status_tag),
381     .m_ax-
is_write_desc_status_error(m_axis_write_desc_status_error),
382     .m_ax-
is_write_desc_status_valid(m_axis_write_desc_status_valid),
383
384     /*
385      * RAM interface
386      */
387     .ram_rd_cmd_sel(ram_rd_cmd_sel),
388     .ram_rd_cmd_addr(ram_rd_cmd_addr),
389     .ram_rd_cmd_valid(ram_rd_cmd_valid),
390     .ram_rd_cmd_ready(ram_rd_cmd_ready),
391     .ram_rd_resp_data(ram_rd_resp_data),
392     .ram_rd_resp_valid(ram_rd_resp_valid),
393     .ram_rd_resp_ready(ram_rd_resp_ready),
394

```

```

395  /*
396  * Configuration
397  */
398  .enable(write_enable),
399
400  /*
401  * Status
402  */
403  .status_busy(status_wr_busy),
404
405  /*
406  * Statistics
407  */
408  .stat_wr_op_start_tag(stat_wr_op_start_tag),
409  .stat_wr_op_start_len(stat_wr_op_start_len),
410  .stat_wr_op_start_valid(stat_wr_op_start_valid),
411  .stat_wr_op_finish_tag(stat_wr_op_finish_tag),
412  .stat_wr_op_finish_status(stat_wr_op_finish_status),
413  .stat_wr_op_finish_valid(stat_wr_op_finish_valid),
414  .stat_wr_req_start_tag(stat_wr_req_start_tag),
415  .stat_wr_req_start_len(stat_wr_req_start_len),
416  .stat_wr_req_start_valid(stat_wr_req_start_valid),
417  .stat_wr_req_finish_tag(stat_wr_req_finish_tag),
418  .stat_wr_req_finish_status(stat_wr_req_finish_status),
419  .stat_wr_req_finish_valid(stat_wr_req_finish_valid),
420  .stat_wr_op_table_full(stat_wr_op_table_full),
421  .stat_wr_tx_stall(stat_wr_tx_stall)
422  );
423
424 endmodule
425
426 `resetall

```

6. alexforenrich_verilog-pcie/rtl/dma_if_axi_rd.v (1,132 lines)

```

1  /*
2
3  Copyright (c) 2021 Alex Forenrich
4
5  Permission is hereby granted, free of charge, to any person
6  obtaining a copy
7  of this software and associated documentation files (the
8  "Software"), to deal
9  in the Software without restriction, including without limita-
10 tion the rights
11 to use, copy, modify, merge, publish, distribute, sublicense,
12 and/or sell
13 copies of the Software, and to permit persons to whom the
14 Software is
15 furnished to do so, subject to the following conditions:
16
17 The above copyright notice and this permission notice shall be
18 included in
19 all copies or substantial portions of the Software.
20
21 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
22 KIND, EXPRESS OR
23 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
24 CHANTABILITY
25 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
26 EVENT SHALL THE
27 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
28 OR OTHER
29 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
30 WISE, ARISING FROM,
31 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
32 DEALINGS IN
33 THE SOFTWARE.
34
35 */
36 // Language: Verilog 2001
37
38 `resetall
39 `timescale 1ns / 1ps
40 `default_nettype none
41
42 /*
43 * AXI DMA read interface
44 */
45 module dma_if_axi_rd #
46 (
47     // Width of AXI data bus in bits
48     parameter AXI_DATA_WIDTH = 32,
49     // Width of AXI address bus in bits
50     parameter AXI_ADDR_WIDTH = 16,
51     // Width of AXI wstrb (width of data bus in words)
52     parameter AXI_STRB_WIDTH = (AXI_DATA_WIDTH/8),
53     // Width of AXI ID signal
54     parameter AXI_ID_WIDTH = 8,
55     // Maximum AXI burst length to generate
56     parameter AXI_MAX_BURST_LEN = 256,
57     // RAM select width
58     parameter RAM_SEL_WIDTH = 2,
59     // RAM address width
60     parameter RAM_ADDR_WIDTH = 16,
61     // RAM segment count
62     parameter RAM_SEG_COUNT = 2,
63     // RAM segment data width

```

```

53     parameter RAM_SEG_DATA_WIDTH = AXI_DATA_WIDTH*2/
RAM_SEG_COUNT,
54     // RAM segment byte enable width
55     parameter RAM_SEG_BE_WIDTH = RAM_SEG_DATA_WIDTH/8,
56     // RAM segment address width
57     parameter RAM_SEG_ADDR_WIDTH = RAM_ADDR_WIDTH-$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH),
58     // Length field width
59     parameter LEN_WIDTH = 16,
60     // Tag field width
61     parameter TAG_WIDTH = 8,
62     // Operation table size
63     parameter OP_TABLE_SIZE = 2** (AXI_ID_WIDTH),
64     // Use AXI ID signals
65     parameter USE_AXI_ID = 0
66 )
67 (
68     input wire clk,
69     input wire rst,
70
71     /*
72     * AXI master interface
73     */
74     output wire [AXI_ID_WIDTH-1:0] m_axi_arid,
75     output wire [AXI_ADDR_WIDTH-1:0] m_axi_araddr,
76     output wire [7:0] m_axi_arlen,
77     output wire [2:0] m_axi_arsize,
78     output wire [1:0] m_axi_arburst,
79     output wire m_axi_arlock,
80     output wire [3:0] m_axi_arcache,
81     output wire [2:0] m_axi_arprot,
82     output wire m_axi_arvalid,
83     input wire m_axi_arready,
84     input wire [AXI_ID_WIDTH-1:0] m_axi_rid,
85     input wire [AXI_DATA_WIDTH-1:0] m_axi_rdata,
86     input wire [1:0] m_axi_rresp,
87     input wire m_axi_rlast,
88     input wire m_axi_rvalid,
89     output wire m_axi_rready,
90
91     /*
92     * AXI read descriptor input
93     */
94     input wire [AXI_ADDR_WIDTH-1:0] is_read_desc_axi_addr,
95     input wire [RAM_SEL_WIDTH-1:0] is_read_desc_ram_sel,
96     input wire [RAM_ADDR_WIDTH-1:0] is_read_desc_ram_addr,
97     input wire [LEN_WIDTH-1:0] is_read_desc_len,
98     input wire [TAG_WIDTH-1:0] is_read_desc_tag,
99     input wire is_read_desc_valid,
100     output wire [3:0] is_read_desc_status_error,
101     output wire is_read_desc_status_valid,
102
103     /*
104     * AXI read descriptor status output
105     */
106     output wire [TAG_WIDTH-1:0] is_read_desc_status_tag,
107     output wire [3:0] is_read_desc_status_error,
108     output wire is_read_desc_status_valid,
109
110     /*
111     * RAM interface
112     */
113     output wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0] ram_wr_cmd_sel,
114     output wire [RAM_SEG_COUNT*RAM_SEG_BE_WIDTH-1:0] ram_wr_cmd_be,
115     output wire [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0] ram_wr_cmd_addr,
116     output wire [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0] ram_wr_cmd_data,
117     output wire [RAM_SEG_COUNT-1:0] ram_wr_cmd_valid,
118     input wire [RAM_SEG_COUNT-1:0] ram_wr_cmd_ready,
119     input wire [RAM_SEG_COUNT-1:0] ram_wr_done,

```

```

119
120 /*
121  * Configuration
122  */
123 input wire en-
able,
124
125 /*
126  * Status
127  */
128 output wire
status_busy,
129
130 /*
131  * Statistics
132  */
133 output wire [$clog2(OP_TABLE_SIZE)-1:0]
stat_rd_op_start_tag,
134 output wire [LEN_WIDTH-1:0]
stat_rd_op_start_len,
135 output wire
stat_rd_op_start_valid,
136 output wire [$clog2(OP_TABLE_SIZE)-1:0]
stat_rd_op_finish_tag,
137 output wire [3:0]
stat_rd_op_finish_status,
138 output wire
stat_rd_op_finish_valid,
139 output wire [$clog2(OP_TABLE_SIZE)-1:0]
stat_rd_req_start_tag,
140 output wire [12:0]
stat_rd_req_start_len,
141 output wire
stat_rd_req_start_valid,
142 output wire [$clog2(OP_TABLE_SIZE)-1:0]
stat_rd_req_finish_tag,
143 output wire [3:0]
stat_rd_req_finish_status,
144 output wire
stat_rd_req_finish_valid,
145 output wire
stat_rd_op_table_full,
146 output wire
stat_rd_tx_stall
147 );
148
149 parameter RAM_DATA_WIDTH = RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH;
150 parameter RAM_WORD_WIDTH = RAM_SEG_BE_WIDTH;
151 parameter RAM_WORD_SIZE = RAM_SEG_DATA_WIDTH/RAM_WORD_WIDTH;
152
153 parameter AXI_WORD_WIDTH = AXI_STRB_WIDTH;
154 parameter AXI_WORD_SIZE = AXI_DATA_WIDTH/AXI_WORD_WIDTH;
155 parameter AXI_BURST_SIZE = $clog2(AXI_STRB_WIDTH);
156 parameter AXI_MAX_BURST_SIZE = AXI_MAX_BURST_LEN <<
AXI_BURST_SIZE;
157
158 parameter OFFSET_WIDTH = AXI_STRB_WIDTH > 1 ? $clog-
2(AXI_STRB_WIDTH) : 1;
159 parameter OFFSET_MASK = AXI_STRB_WIDTH > 1 ? {OFF-
SET_WIDTH{1'b1}} : 0;
160 parameter RAM_OFFSET_WIDTH = $clog-
2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH);
161 parameter ADDR_MASK = {AXI_ADDR_WIDTH{1'b1}} << $clog-
2(AXI_STRB_WIDTH);
162 parameter CYCLE_COUNT_WIDTH = LEN_WIDTH - AXI_BURST_SIZE + 1;
163
164 parameter OP_TAG_WIDTH = $clog2(OP_TABLE_SIZE);
165 parameter OP_TABLE_READ_COUNT_WIDTH = AXI_ID_WIDTH+1;
166 parameter OP_TABLE_WRITE_COUNT_WIDTH = LEN_WIDTH;
167
168 parameter STATUS_FIFO_ADDR_WIDTH = 5;
169 parameter OUTPUT_FIFO_ADDR_WIDTH = 5;
170
171 // bus width assertions
172 initial begin
173     if (AXI_WORD_SIZE * AXI_STRB_WIDTH != AXI_DATA_WIDTH) be-
gin
174         $error("Error: AXI data width not evenly divisble (in-
stance %m)");
175     $finish;
176 end
177
178 if (AXI_WORD_SIZE != RAM_WORD_SIZE) begin
179     $error("Error: word size mismatch (instance %m)");
180     $finish;
181 end
182
183 if (2**$clog2(AXI_WORD_WIDTH) != AXI_WORD_WIDTH) begin
184     $error("Error: AXI word width must be even power of
two (instance %m)");
185     $finish;
186 end
187
188 if (AXI_MAX_BURST_LEN < 1 || AXI_MAX_BURST_LEN > 256) be-
gin
189     $error("Error: AXI_MAX_BURST_LEN must be between 1 and
256 (instance %m)");
190     $finish;
191 end
192

```

```

193     if (RAM_SEG_COUNT < 2) begin
194         $error("Error: RAM interface requires at least 2 seg-
ments (instance %m)");
195         $finish;
196     end
197
198     if (RAM_DATA_WIDTH != AXI_DATA_WIDTH*2) begin
199         $error("Error: RAM interface width must be double the
AXI interface width (instance %m)");
200         $finish;
201     end
202
203     if (2**$clog2(RAM_WORD_WIDTH) != RAM_WORD_WIDTH) begin
204         $error("Error: RAM word width must be even power of
two (instance %m)");
205         $finish;
206     end
207
208     if (RAM_ADDR_WIDTH != RAM_SEG_ADDR_WIDTH+$clog-
2(RAM_SEG_COUNT)+$clog2(RAM_SEG_BE_WIDTH)) begin
209         $error("Error: RAM_ADDR_WIDTH does not match RAM con-
figuration (instance %m)");
210         $finish;
211     end
212
213     if (OP_TABLE_SIZE > 2**AXI_ID_WIDTH) begin
214         $error("Error: AXI_ID_WIDTH insufficient for requested
OP_TABLE_SIZE (instance %m)");
215         $finish;
216     end
217 end
218
219 localparam [1:0]
220     AXI_RESP_OKAY = 2'b00,
221     AXI_RESP_EXOKAY = 2'b01,
222     AXI_RESP_SLVERR = 2'b10,
223     AXI_RESP_DECERR = 2'b11;
224
225 localparam [3:0]
226     DMA_ERROR_NONE = 4'd0,
227     DMA_ERROR_TIMEOUT = 4'd1,
228     DMA_ERROR_PARITY = 4'd2,
229     DMA_ERROR_AXI_RD_SLVERR = 4'd4,
230     DMA_ERROR_AXI_RD_DECERR = 4'd5,
231     DMA_ERROR_AXI_WR_SLVERR = 4'd6,
232     DMA_ERROR_AXI_WR_DECERR = 4'd7,
233     DMA_ERROR_PCIE_FLR = 4'd8,
234     DMA_ERROR_PCIE_CPL_POISONED = 4'd9,
235     DMA_ERROR_PCIE_CPL_STATUS_UR = 4'd10,
236     DMA_ERROR_PCIE_CPL_STATUS_CA = 4'd11;
237
238 localparam [0:0]
239     REQ_STATE_IDLE = 1'd0,
240     REQ_STATE_START = 1'd1;
241
242 reg [0:0] req_state_reg = REQ_STATE_IDLE, req_state_next;
243
244 localparam [0:0]
245     AXI_STATE_IDLE = 1'd0,
246     AXI_STATE_WRITE = 1'd1;
247
248 reg [0:0] axi_state_reg = AXI_STATE_IDLE, axi_state_next;
249
250 reg [AXI_ADDR_WIDTH-1:0] req_axi_addr_reg = {AXI_AD-
DR_WIDTH{1'b0}}, req_axi_addr_next;
251 reg [RAM_SEL_WIDTH-1:0] req_ram_sel_reg =
{RAM_SEL_WIDTH{1'b0}}, req_ram_sel_next;
252 reg [RAM_ADDR_WIDTH-1:0] req_ram_addr_reg = {RAM_AD-
DR_WIDTH{1'b0}}, req_ram_addr_next;
253 reg [LEN_WIDTH-1:0] req_op_count_reg = {LEN_WIDTH{1'b0}},
req_op_count_next;
254 reg [LEN_WIDTH-1:0] req_tr_count_reg = {LEN_WIDTH{1'b0}},
req_tr_count_next;
255 reg req_zero_len_reg = 1'b0, req_zero_len_next;
256 reg [TAG_WIDTH-1:0] req_tag_reg = {TAG_WIDTH{1'b0}},
req_tag_next;
257
258 reg [RAM_SEL_WIDTH-1:0] ram_sel_reg = {RAM_SEL_WIDTH{1'b0}},
ram_sel_next;
259 reg [RAM_ADDR_WIDTH-1:0] addr_reg = {RAM_ADDR_WIDTH{1'b0}},
addr_next;
260 reg [RAM_ADDR_WIDTH-1:0] addr_delay_reg = {RAM_AD-
DR_WIDTH{1'b0}}, addr_delay_next;
261 reg [12:0] op_count_reg = 13'd0, op_count_next;
262 reg zero_len_reg = 1'b0, zero_len_next;
263 reg [RAM_SEG_COUNT-1:0] ram_mask_reg = {RAM_SEG_COUNT{1'b0}},
ram_mask_next;
264 reg [RAM_SEG_COUNT-1:0] ram_mask_0_reg =
{RAM_SEG_COUNT{1'b0}}, ram_mask_0_next;
265 reg [RAM_SEG_COUNT-1:0] ram_mask_1_reg =
{RAM_SEG_COUNT{1'b0}}, ram_mask_1_next;
266 reg ram_wrap_reg = 1'b0, ram_wrap_next;
267 reg [OFFSET_WIDTH+1-1:0] cycle_byte_count_reg = {OFF-
SET_WIDTH+1{1'b0}}, cycle_byte_count_next;
268 reg [RAM_OFFSET_WIDTH-1:0] start_offset_reg = {RAM_OFF-
SET_WIDTH{1'b0}}, start_offset_next;
269 reg [RAM_OFFSET_WIDTH-1:0] end_offset_reg = {RAM_OFF-
SET_WIDTH{1'b0}}, end_offset_next;
270 reg [OFFSET_WIDTH-1:0] offset_reg = {OFFSET_WIDTH{1'b0}}, off-
set_next;

```

```

271 reg [OP_TAG_WIDTH-1:0] op_tag_reg = {OP_TAG_WIDTH{1'b0}},
op_tag_next;
272
273 reg [STATUS_FIFO_ADDR_WIDTH+1-1:0] status_fifo_wr_ptr_reg = 0;

274 reg [STATUS_FIFO_ADDR_WIDTH+1-1:0] status_fifo_rd_ptr_reg = 0,
status_fifo_rd_ptr_next;
275 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
276 reg [OP_TAG_WIDTH-1:0]
status_fifo_op_tag[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
277 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
278 reg [RAM_SEG_COUNT-1:0]
status_fifo_mask[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
279 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
280 reg status_fifo_finish[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
281 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
282 reg [3:0] status_fifo_error[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
283 reg [OP_TAG_WIDTH-1:0] status_fifo_wr_op_tag;
284 reg [RAM_SEG_COUNT-1:0] status_fifo_wr_mask;
285 reg status_fifo_wr_finish;
286 reg [3:0] status_fifo_wr_error;
287 reg status_fifo_we;
288 reg status_fifo_mask_reg = 1'b0, status_fifo_mask_next;
289 reg status_fifo_finish_reg = 1'b0, status_fifo_finish_next;
290 reg [3:0] status_fifo_error_reg = 4'd0,
status_fifo_error_next;
291 reg status_fifo_we_reg = 1'b0, status_fifo_we_next;
292 reg status_fifo_half_full_reg = 1'b0;
293 reg [OP_TAG_WIDTH-1:0] status_fifo_rd_op_tag_reg = 0,
status_fifo_rd_op_tag_next;
294 reg [RAM_SEG_COUNT-1:0] status_fifo_rd_mask_reg = 0,
status_fifo_rd_mask_next;
295 reg status_fifo_rd_finish_reg = 1'b0,
status_fifo_rd_finish_next;
296 reg [3:0] status_fifo_rd_error_reg = 4'd0,
status_fifo_rd_error_next;
297 reg status_fifo_rd_valid_reg = 1'b0,
status_fifo_rd_valid_next;
298
299 reg [OP_TAG_WIDTH+1-1:0] active_op_count_reg = 0;
300 reg inc_active_op;
301 reg dec_active_op;
302
303 reg [AXI_DATA_WIDTH-1:0] m_axi_rdata_int_reg =
{AXI_DATA_WIDTH{1'b0}}, m_axi_rdata_int_next;
304 reg m_axi_rvalid_int_reg = 1'b0, m_axi_rvalid_int_next;
305
306 reg [AXI_ID_WIDTH-1:0] m_axi_arid_reg = {AXI_ID_WIDTH{1'b0}},
m_axi_arid_next;
307 reg [AXI_ADDR_WIDTH-1:0] m_axi_araddr_reg = {AXI_AD-
DR_WIDTH{1'b0}}, m_axi_araddr_next;
308 reg [7:0] m_axi_arlen_reg = 8'd0, m_axi_arlen_next;
309 reg m_axi_arvalid_reg = 1'b0, m_axi_arvalid_next;
310 reg m_axi_rready_reg = 1'b0, m_axi_rready_next;
311
312 reg s_axis_read_desc_ready_reg = 1'b0, s_ax-
is_read_desc_ready_next;
313
314 reg [TAG_WIDTH-1:0] m_axis_read_desc_status_tag_reg =
{TAG_WIDTH{1'b0}}, m_axis_read_desc_status_tag_next;
315 reg [3:0] m_axis_read_desc_status_error_reg = 4'd0, m_ax-
is_read_desc_status_error_next;
316 reg m_axis_read_desc_status_valid_reg = 1'b0, m_ax-
is_read_desc_status_valid_next;
317
318 reg status_busy_reg = 1'b0;
319
320 reg [OP_TAG_WIDTH-1:0] stat_rd_op_start_tag_reg = 0,
stat_rd_op_start_tag_next;
321 reg [LEN_WIDTH-1:0] stat_rd_op_start_len_reg = 0,
stat_rd_op_start_len_next;
322 reg stat_rd_op_start_valid_reg = 1'b0,
stat_rd_op_start_valid_next;
323 reg [OP_TAG_WIDTH-1:0] stat_rd_op_finish_tag_reg = 0,
stat_rd_op_finish_tag_next;
324 reg [3:0] stat_rd_op_finish_status_reg = 4'd0, stat_rd_op_fin-
ish_status_next;
325 reg stat_rd_op_finish_valid_reg = 1'b0, stat_rd_op_fin-
ish_valid_next;
326 reg [OP_TAG_WIDTH-1:0] stat_rd_req_start_tag_reg = 0,
stat_rd_req_start_tag_next;
327 reg [12:0] stat_rd_req_start_len_reg = 13'd0,
stat_rd_req_start_len_next;
328 reg stat_rd_req_start_valid_reg = 1'b0,
stat_rd_req_start_valid_next;
329 reg [OP_TAG_WIDTH-1:0] stat_rd_req_finish_tag_reg = 0,
stat_rd_req_finish_tag_next;
330 reg [3:0] stat_rd_req_finish_status_reg = 4'd0,
stat_rd_req_finish_status_next;
331 reg stat_rd_req_finish_valid_reg = 1'b0, stat_rd_req_fin-
ish_valid_next;
332 reg stat_rd_op_table_full_reg = 1'b0, stat_rd_op_t-
able_full_next;
333 reg stat_rd_tx_stall_reg = 1'b0, stat_rd_tx_stall_next;
334
335 // internal datapath
336 reg [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]

```

```

ram_wr_cmd_sel_int;
337 reg [RAM_SEG_COUNT*RAM_SEG_BE_WIDTH-1:0] ram_wr_cmd_be_int;
338 reg [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0] ram_wr_cmd_ad-
dr_int;
339 reg [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0]
ram_wr_cmd_data_int;
340 reg [RAM_SEG_COUNT-1:0] ram_wr_cmd_val-
id_int;
341 wire [RAM_SEG_COUNT-1:0]
ram_wr_cmd_ready_int;
342
343 wire [RAM_SEG_COUNT-1:0] out_done;
344 reg [RAM_SEG_COUNT-1:0] out_done_ack;
345
346 assign m_axi_arid = USE_AXI_ID ? m_axi_arid_reg :
{AXI_ID_WIDTH{1'b0}};
347 assign m_axi_araddr = m_axi_araddr_reg;
348 assign m_axi_arlen = m_axi_arlen_reg;
349 assign m_axi_arsize = AXI_BURST_SIZE;
350 assign m_axi_arburst = 2'b01;
351 assign m_axi_arlock = 1'b0;
352 assign m_axi_arcache = 4'b0011;
353 assign m_axi_arprot = 3'b010;
354 assign m_axi_arvalid = m_axi_arvalid_reg;
355 assign m_axi_rready = m_axi_rready_reg;
356
357 assign s_axis_read_desc_ready = s_axis_read_desc_ready_reg;
358
359 assign m_axis_read_desc_status_tag = m_ax-
is_read_desc_status_tag_reg;
360 assign m_axis_read_desc_status_error = m_ax-
is_read_desc_status_error_reg;
361 assign m_axis_read_desc_status_valid = m_ax-
is_read_desc_status_valid_reg;
362
363 assign status_busy = status_busy_reg;
364
365 assign stat_rd_op_start_tag = stat_rd_op_start_tag_reg;
366 assign stat_rd_op_start_len = stat_rd_op_start_len_reg;
367 assign stat_rd_op_start_valid = stat_rd_op_start_valid_reg;
368 assign stat_rd_op_finish_tag = stat_rd_op_finish_tag_reg;
369 assign stat_rd_op_finish_status = stat_rd_op_fin-
ish_status_reg;
370 assign stat_rd_op_finish_valid = stat_rd_op_finish_valid_reg;
371 assign stat_rd_req_start_tag = stat_rd_req_start_tag_reg;
372 assign stat_rd_req_start_len = stat_rd_req_start_len_reg;
373 assign stat_rd_req_start_valid = stat_rd_req_start_valid_reg;
374 assign stat_rd_req_finish_tag = stat_rd_req_finish_tag_reg;
375 assign stat_rd_req_finish_status = stat_rd_req_fin-
ish_status_reg;
376 assign stat_rd_req_finish_valid = stat_rd_req_fin-
ish_valid_reg;
377 assign stat_rd_op_table_full = stat_rd_op_table_full_reg;
378 assign stat_rd_tx_stall = stat_rd_tx_stall_reg;
379
380 // operation tag management
381 reg [OP_TAG_WIDTH+1-1:0] op_table_start_ptr_reg = 0;
382 reg [AXI_ADDR_WIDTH-1:0] op_table_start_axi_addr;
383 reg [RAM_SEL_WIDTH-1:0] op_table_start_ram_sel;
384 reg [RAM_ADDR_WIDTH-1:0] op_table_start_ram_addr;
385 reg [11:0] op_table_start_len;
386 reg op_table_start_zero_len;
387 reg [CYCLE_COUNT_WIDTH-1:0] op_table_start_cycle_count;
388 reg [TAG_WIDTH-1:0] op_table_start_tag;
389 reg op_table_start_last;
390 reg op_table_start_en;
391 reg [OP_TAG_WIDTH+1-1:0] op_table_read_complete_ptr_reg = 0;
392 reg op_table_read_complete_en;
393 reg [OP_TAG_WIDTH-1:0] op_table_update_status_ptr;
394 reg [3:0] op_table_update_status_error;
395 reg op_table_update_status_en;
396 reg [OP_TAG_WIDTH-1:0] op_table_write_complete_ptr;
397 reg op_table_write_complete_en;
398 reg [OP_TAG_WIDTH+1-1:0] op_table_finish_ptr_reg = 0;
399 reg op_table_finish_en;
400
401 reg [2**OP_TAG_WIDTH-1:0] op_table_active = 0;
402 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
403 reg [AXI_ADDR_WIDTH-1:0] op_table_axi_addr
[2**OP_TAG_WIDTH-1:0];
404 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
405 reg [RAM_SEL_WIDTH-1:0] op_table_ram_sel
[2**OP_TAG_WIDTH-1:0];
406 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
407 reg [RAM_ADDR_WIDTH-1:0] op_table_ram_addr
[2**OP_TAG_WIDTH-1:0];
408 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
409 reg [11:0] op_table_len[2**OP_TAG_WIDTH-1:0];
410 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
411 reg op_table_zero_len[2**OP_TAG_WIDTH-1:0];
412 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
413 reg [CYCLE_COUNT_WIDTH-1:0] op_t-
able_cycle_count[2**OP_TAG_WIDTH-1:0];
414 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)

```

```

415 reg [TAG_WIDTH-1:0] op_table_tag[2**OP_TAG_WIDTH-1:0];
416 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
417 reg op_table_last[2**OP_TAG_WIDTH-1:0];
418 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
419 reg op_table_write_complete[2**OP_TAG_WIDTH-1:0];
420 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
421 reg op_table_error_a [2**OP_TAG_WIDTH-1:0];
422 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
423 reg op_table_error_b [2**OP_TAG_WIDTH-1:0];
424 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
425 reg [3:0] op_table_error_code [2**OP_TAG_WIDTH-1:0];
426
427 integer i;
428
429 initial begin
430   for (i = 0; i < 2**OP_TAG_WIDTH; i = i + 1) begin
431     op_table_axi_addr[i] = 0;
432     op_table_ram_sel[i] = 0;
433     op_table_ram_addr[i] = 0;
434     op_table_len[i] = 0;
435     op_table_zero_len[i] = 1'b0;
436     op_table_cycle_count[i] = 0;
437     op_table_tag[i] = 0;
438     op_table_last[i] = 0;
439     op_table_write_complete[i] = 0;
440     op_table_error_a[i] = 0;
441     op_table_error_b[i] = 0;
442     op_table_error_code[i] = 0;
443   end
444 end
445
446 always @* begin
447   req_state_next = REQ_STATE_IDLE;
448
449   s_axis_read_desc_ready_next = 1'b0;
450
451   stat_rd_op_start_tag_next = stat_rd_op_start_tag_reg;
452   stat_rd_op_start_len_next = stat_rd_op_start_len_reg;
453   stat_rd_op_start_valid_next = 1'b0;
454   stat_rd_req_start_tag_next = stat_rd_req_start_tag_reg;
455   stat_rd_req_start_len_next = stat_rd_req_start_len_reg;
456   stat_rd_req_start_valid_next = 1'b0;
457   stat_rd_op_table_full_next = !(op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2**OP_TAG_WIDTH));
458   stat_rd_tx_stall_next = m_axi_arvalid_reg && !m_axi_ar-
ready;
459
460   req_axi_addr_next = req_axi_addr_reg;
461   req_ram_sel_next = req_ram_sel_reg;
462   req_ram_addr_next = req_ram_addr_reg;
463   req_op_count_next = req_op_count_reg;
464   req_tr_count_next = req_tr_count_reg;
465   req_zero_len_next = req_zero_len_reg;
466   req_tag_next = req_tag_reg;
467
468   m_axi_arid_next = m_axi_arid_reg;
469   m_axi_araddr_next = m_axi_araddr_reg;
470   m_axi_arlen_next = m_axi_arlen_reg;
471   m_axi_arvalid_next = m_axi_arvalid_reg && !m_axi_arready;
472
473   op_table_start_axi_addr = req_axi_addr_reg;
474   op_table_start_ram_sel = req_ram_sel_reg;
475   op_table_start_ram_addr = req_ram_addr_reg;
476   op_table_start_len = 0;
477   op_table_start_zero_len = req_zero_len_reg;
478   op_table_start_tag = req_tag_reg;
479   op_table_start_cycle_count = 0;
480   op_table_start_last = 0;
481   op_table_start_en = 1'b0;
482
483   inc_active_op = 1'b0;
484
485   // segmentation and request generation
486   case (req_state_reg)
487     REQ_STATE_IDLE: begin
488       s_axis_read_desc_ready_next = !op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2**OP_TAG_WIDTH) && enable;
489
490       req_axi_addr_next = s_axis_read_desc_axi_addr;
491       req_ram_sel_next = s_axis_read_desc_ram_sel;
492       req_ram_addr_next = s_axis_read_desc_ram_addr;
493       if (s_axis_read_desc_len == 0) begin
494         // zero-length operation
495         req_op_count_next = 1;
496         req_zero_len_next = 1'b1;
497       end else begin
498         req_op_count_next = s_axis_read_desc_len;
499         req_zero_len_next = 1'b0;
500       end
501       req_tag_next = s_axis_read_desc_tag;
502
503       if (req_op_count_next <= AXI_MAX_BURST_SIZE -

```

```

(req_axi_addr_next & OFFSET_MASK) || AXI_MAX_BURST_SIZE >= 4096) be-
gin
504         // packet smaller than max burst size
505         if (((req_axi_addr_next & 12'hfff) +
(req_op_count_next & 12'hfff)) >> 12 != 0 || req_op_count_next >>
12 != 0) begin
506           // crosses 4k boundary
507           req_tr_count_next = 13'h1000 - req_axi_ad-
dr_next[11:0];
508         end else begin
509           // does not cross 4k boundary
510           req_tr_count_next = req_op_count_next;
511         end
512       end else begin
513         // packet larger than max burst size
514         if (((req_axi_addr_next & 12'hfff) +
AXI_MAX_BURST_SIZE) >> 12 != 0) begin
515           // crosses 4k boundary
516           req_tr_count_next = 13'h1000 - req_axi_ad-
dr_next[11:0];
517         end else begin
518           // does not cross 4k boundary
519           req_tr_count_next = AXI_MAX_BURST_SIZE -
(req_axi_addr_next & OFFSET_MASK);
520         end
521       end
522
523       if (s_axis_read_desc_ready && s_ax-
is_read_desc_valid) begin
524         s_axis_read_desc_ready_next = 1'b0;
525
526         stat_rd_op_start_tag_next =
stat_rd_op_start_tag_reg+1;
527         stat_rd_op_start_len_next = s_ax-
is_read_desc_len;
528         stat_rd_op_start_valid_next = 1'b1;
529
530         req_state_next = REQ_STATE_START;
531       end else begin
532         req_state_next = REQ_STATE_IDLE;
533       end
534     end
535     REQ_STATE_START: begin
536       if (!op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2**OP_TAG_WIDTH) && (!m_axi_arvalid || m_axi_arready)) begin
537         req_axi_addr_next = req_axi_addr_reg +
req_tr_count_reg;
538         req_ram_addr_next = req_ram_addr_reg +
req_tr_count_reg;
539         req_op_count_next = req_op_count_reg -
req_tr_count_reg;
540
541         op_table_start_axi_addr = req_axi_addr_reg;
542         op_table_start_ram_sel = req_ram_sel_reg;
543         op_table_start_ram_addr = req_ram_addr_reg;
544         op_table_start_len = req_tr_count_next;
545         op_table_start_zero_len = req_zero_len_reg;
546         op_table_start_tag = req_tag_reg;
547         op_table_start_cycle_count =
(req_tr_count_next + (req_axi_addr_reg & OFFSET_MASK) - 1) >>
AXI_BURST_SIZE;
548         op_table_start_last = req_op_count_reg ==
req_tr_count_next;
549         op_table_start_en = 1'b1;
550         inc_active_op = 1'b1;
551
552         stat_rd_req_start_tag_next = op_t-
able_start_ptr_reg[OP_TAG_WIDTH-1:0];
553         stat_rd_req_start_len_next =
req_zero_len_reg ? 0 : req_tr_count_reg;
554         stat_rd_req_start_valid_next = 1'b1;
555
556         m_axi_arid_next = op_t-
able_start_ptr_reg[OP_TAG_WIDTH-1:0];
557         m_axi_araddr_next = req_axi_addr_reg;
558         m_axi_arlen_next = op_table_start_cycle_count;
559         m_axi_arvalid_next = 1'b1;
560
561         if (req_op_count_next <= AXI_MAX_BURST_SIZE -
(req_axi_addr_next & OFFSET_MASK) || AXI_MAX_BURST_SIZE >= 4096) be-
gin
562           // packet smaller than max burst size
563           if (((req_axi_addr_next & 12'hfff) +
(req_op_count_next & 12'hfff)) >> 12 != 0 || req_op_count_next >>
12 != 0) begin
564             // crosses 4k boundary
565             req_tr_count_next = 13'h1000 -
req_axi_addr_next[11:0];
566           end else begin
567             // does not cross 4k boundary
568             req_tr_count_next = req_op_count_next;
569           end
570         end else begin
571           // packet larger than max burst size
572           if (((req_axi_addr_next & 12'hfff) +
AXI_MAX_BURST_SIZE) >> 12 != 0) begin
573             // crosses 4k boundary
574             req_tr_count_next = 13'h1000 -
req_axi_addr_next[11:0];

```



```

575         end else begin
576             // does not cross 4k boundary
577             req_tr_count_next = AXI_MAX_BURST_SIZE -
(req_axi_addr_next & OFFSET_MASK);
578         end
579     end
580
581     if (!op_table_start_last) begin
582         req_state_next = REQ_STATE_START;
583     end else begin
584         s_axis_read_desc_ready_next = !op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2*OP_TAG_WIDTH) && enable;
585         req_state_next = REQ_STATE_IDLE;
586     end
587 end else begin
588     req_state_next = REQ_STATE_START;
589 end
590 end
591 endcase
592 end
593
594 always @* begin
595     axi_state_next = AXI_STATE_IDLE;
596
597     m_axi_rready_next = 1'b0;
598
599     stat_rd_op_finish_tag_next = stat_rd_op_finish_tag_reg;
600     stat_rd_op_finish_status_next = stat_rd_op_fin-
ish_status_reg;
601     stat_rd_op_finish_valid_next = 1'b0;
602     stat_rd_req_finish_tag_next = stat_rd_req_finish_tag_reg;
603     stat_rd_req_finish_status_next = stat_rd_req_fin-
ish_status_reg;
604     stat_rd_req_finish_valid_next = 1'b0;
605
606     ram_sel_next = ram_sel_reg;
607     addr_next = addr_reg;
608     addr_delay_next = addr_delay_reg;
609     op_count_next = op_count_reg;
610     zero_len_next = zero_len_reg;
611     ram_mask_next = ram_mask_reg;
612     ram_mask_0_next = ram_mask_0_reg;
613     ram_mask_1_next = ram_mask_1_reg;
614     ram_wrap_next = ram_wrap_reg;
615     cycle_byte_count_next = cycle_byte_count_reg;
616     start_offset_next = start_offset_reg;
617     end_offset_next = end_offset_reg;
618     offset_next = offset_reg;
619     op_tag_next = op_tag_reg;
620
621     op_table_read_complete_en = 1'b0;
622
623     m_axi_rdata_int_next = m_axi_rdata_int_reg;
624     m_axi_rvalid_int_next = 1'b0;
625
626     status_fifo_mask_next = 1'b1;
627     status_fifo_finish_next = 1'b0;
628     status_fifo_error_next = DMA_ERROR_NONE;
629     status_fifo_we_next = 1'b0;
630
631     out_done_ack = {RAM_SEG_COUNT{1'b0}};
632
633     // Write generation
634     ram_wr_cmd_sel_int = {RAM_SEG_COUNT{ram_sel_reg}};
635     if (!ram_wrap_reg) begin
636         ram_wr_cmd_be_int =
({RAM_SEG_COUNT*RAM_SEG_BE_WIDTH{1'b1}} << start_offset_reg) &
({RAM_SEG_COUNT*RAM_SEG_BE_WIDTH{1'b1}} >>
(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH-1-end_offset_reg));
637     end else begin
638         ram_wr_cmd_be_int =
({RAM_SEG_COUNT*RAM_SEG_BE_WIDTH{1'b1}} << start_offset_reg) |
({RAM_SEG_COUNT*RAM_SEG_BE_WIDTH{1'b1}} >>
(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH-1-end_offset_reg));
639     end
640     for (i = 0; i < RAM_SEG_COUNT; i = i + 1) begin
641         ram_wr_cmd_addr_int[i*RAM_SEG_ADDR_WIDTH +:
RAM_SEG_ADDR_WIDTH] =
addr_delay_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-RAM_SEG_ADDR_WIDTH];
642         if (ram_mask_1_reg[i]) begin
643             ram_wr_cmd_addr_int[i*RAM_SEG_ADDR_WIDTH +:
RAM_SEG_ADDR_WIDTH] =
addr_delay_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-RAM_SEG_ADDR_WIDTH]
+1;
644         end
645     end
646     ram_wr_cmd_data_int = {3{m_axi_rdata_int_reg}} >>
(AXI_DATA_WIDTH - offset_reg*AXI_WORD_SIZE);
647     ram_wr_cmd_valid_int = {RAM_SEG_COUNT{1'b0}};
648
649     if (m_axi_rvalid_int_reg) begin
650         ram_wr_cmd_valid_int = ram_mask_reg;
651     end
652
653     // AXI read response handling
654     case (axi_state_reg)
655         AXI_STATE_IDLE: begin
656             // idle state, wait for read data

```

```

657         m_axi_rready_next = &ram_wr_cmd_ready_int && !
status_fifo_half_full_reg;
658
659         if (USE_AXI_ID) begin
660             op_tag_next = m_axi_rid[OP_TAG_WIDTH-1:0];
661         end else begin
662             op_tag_next = op_table_read_complete_ptr_reg;
663         end
664         ram_sel_next = op_table_ram_sel[op_tag_next];
665         addr_next = op_table_ram_addr[op_tag_next];
666         op_count_next = op_table_len[op_tag_next];
667         zero_len_next = op_table_zero_len[op_tag_next];
668         offset_next = op_table_ram_addr[op_tag_next]
[RAM_OFFSET_WIDTH-1:0]-(op_table_axi_addr[op_tag_next] & OFF-
SET_MASK);
669
670         if (m_axi_rready && m_axi_rvalid) begin
671             if (op_count_next > AXI_WORD_WIDTH-(op_t-
able_axi_addr[op_tag_next] & OFFSET_MASK)) begin
672                 cycle_byte_count_next = AXI_WORD_WIDTH-
(op_table_axi_addr[op_tag_next] & OFFSET_MASK);
673             end else begin
674                 cycle_byte_count_next = op_count_next;
675             end
676             start_offset_next = addr_next;
677             {ram_wrap_next, end_offset_next} =
start_offset_next+cycle_byte_count_next-1;
678
679             ram_mask_0_next = {RAM_SEG_COUNT{1'b1}} <<
(start_offset_next >> $clog2(RAM_SEG_BE_WIDTH));
680             ram_mask_1_next = {RAM_SEG_COUNT{1'b1}} >>
(RAM_SEG_COUNT-1-(end_offset_next >> $clog2(RAM_SEG_BE_WIDTH)));
681
682             if (!ram_wrap_next) begin
683                 ram_mask_next = ram_mask_0_next &
ram_mask_1_next;
684                 ram_mask_0_next = ram_mask_0_next &
ram_mask_1_next;
685                 ram_mask_1_next = 0;
686             end else begin
687                 ram_mask_next = ram_mask_0_next |
ram_mask_1_next;
688             end
689
690             addr_delay_next = addr_next;
691             addr_next = addr_next + cycle_byte_count_next;
692             op_count_next = op_count_next -
cycle_byte_count_next;
693
694             m_axi_rdata_int_next = m_axi_rdata;
695             m_axi_rvalid_int_next = 1'b1;
696
697             status_fifo_mask_next = 1'b1;
698             status_fifo_finish_next = 1'b0;
699             status_fifo_error_next = DMA_ERROR_NONE;
700             status_fifo_we_next = 1'b1;
701
702             if (zero_len_next) begin
703                 m_axi_rvalid_int_next = 1'b0;
704                 status_fifo_mask_next = 1'b0;
705             end
706
707             if (m_axi_rresp == AXI_RESP_SLVERR) begin
708                 m_axi_rvalid_int_next = 1'b0;
709                 status_fifo_mask_next = 1'b0;
710                 status_fifo_error_next = DMA_ER-
ROR_AXI_RD_SLVERR;
711             end else if (m_axi_rresp == AXI_RESP_DECERR)
begin
712                 m_axi_rvalid_int_next = 1'b0;
713                 status_fifo_mask_next = 1'b0;
714                 status_fifo_error_next = DMA_ER-
ROR_AXI_RD_DECERR;
715             end
716
717             stat_rd_req_finish_tag_next = op_tag_next;
718             stat_rd_req_finish_status_next =
status_fifo_error_next;
719             stat_rd_req_finish_valid_next = 1'b0;
720
721             if (!USE_AXI_ID) begin
722                 op_table_read_complete_en = 1'b1;
723             end
724
725             if (m_axi_rlast) begin
726                 status_fifo_finish_next = 1'b1;
727                 stat_rd_req_finish_valid_next = 1'b1;
728                 axi_state_next = AXI_STATE_IDLE;
729             end else begin
730                 axi_state_next = AXI_STATE_WRITE;
731             end
732             end else begin
733                 axi_state_next = AXI_STATE_IDLE;
734             end
735         end
736         AXI_STATE_WRITE: begin
737             // write state - generate write operations
738             m_axi_rready_next = &ram_wr_cmd_ready_int && !
status_fifo_half_full_reg;
739
740             if (m_axi_rready && m_axi_rvalid) begin

```

```

741
742     if (op_count_next > AXI_WORD_WIDTH) begin
743         cycle_byte_count_next = AXI_WORD_WIDTH;
744     end else begin
745         cycle_byte_count_next = op_count_next;
746     end
747     start_offset_next = addr_next;
748     {ram_wrap_next, end_offset_next} =
start_offset_next+cycle_byte_count_next-1;
749
750     ram_mask_0_next = {RAM_SEG_COUNT{1'b1}} <<
(start_offset_next >> $clog2(RAM_SEG_BE_WIDTH));
751     ram_mask_1_next = {RAM_SEG_COUNT{1'b1}} >>
(RAM_SEG_COUNT-1-(end_offset_next >> $clog2(RAM_SEG_BE_WIDTH)));
752
753     if (!ram_wrap_next) begin
754         ram_mask_next = ram_mask_0_next &
ram_mask_1_next;
755         ram_mask_0_next = ram_mask_0_next &
ram_mask_1_next;
756         ram_mask_1_next = 0;
757     end else begin
758         ram_mask_next = ram_mask_0_next |
ram_mask_1_next;
759     end
760
761     addr_delay_next = addr_next;
762     addr_next = addr_next + cycle_byte_count_next;
763     op_count_next = op_count_next -
cycle_byte_count_next;
764
765     m_axi_rdata_int_next = m_axi_rdata;
766     m_axi_rvalid_int_next = 1'b1;
767
768     status_fifo_mask_next = 1'b1;
769     status_fifo_finish_next = 1'b0;
770     status_fifo_error_next = DMA_ERROR_NONE;
771     status_fifo_we_next = 1'b1;
772
773     if (m_axi_rresp == AXI_RESP_SLVERR) begin
774         m_axi_rvalid_int_next = 1'b0;
775         status_fifo_mask_next = 1'b0;
776         status_fifo_error_next = DMA_ER-
ROR_AXI_RD_SLVERR;
777     end else if (m_axi_rresp == AXI_RESP_DECERR)
begin
778         m_axi_rvalid_int_next = 1'b0;
779         status_fifo_mask_next = 1'b0;
780         status_fifo_error_next = DMA_ER-
ROR_AXI_RD_DECERR;
781     end
782
783     stat_rd_req_finish_tag_next = op_tag_next;
784     stat_rd_req_finish_status_next =
status_fifo_error_next;
785     stat_rd_req_finish_valid_next = 1'b0;
786
787     if (m_axi_rlast) begin
788         status_fifo_finish_next = 1'b1;
789         stat_rd_req_finish_valid_next = 1'b1;
790         axi_state_next = AXI_STATE_IDLE;
791     end else begin
792         axi_state_next = AXI_STATE_WRITE;
793     end
794     end else begin
795         axi_state_next = AXI_STATE_WRITE;
796     end
797 end
798 endcase
799
800 status_fifo_rd_ptr_next = status_fifo_rd_ptr_reg;
801
802 status_fifo_wr_op_tag = op_tag_reg;
803 status_fifo_wr_mask = status_fifo_mask_reg ?
ram_mask_reg : 0;
804 status_fifo_wr_finish = status_fifo_finish_reg;
805 status_fifo_wr_error = status_fifo_error_reg;
806 status_fifo_we = 1'b0;
807
808 if (status_fifo_we_reg) begin
809     status_fifo_wr_op_tag = op_tag_reg;
810     status_fifo_wr_mask = status_fifo_mask_reg ?
ram_mask_reg : 0;
811     status_fifo_wr_finish = status_fifo_finish_reg;
812     status_fifo_wr_error = status_fifo_error_reg;
813     status_fifo_we = 1'b1;
814 end
815
816 status_fifo_rd_op_tag_next = status_fifo_rd_op_tag_reg;
817 status_fifo_rd_mask_next = status_fifo_rd_mask_reg;
818 status_fifo_rd_finish_next = status_fifo_rd_finish_reg;
819 status_fifo_rd_valid_next = status_fifo_rd_valid_reg;
820 status_fifo_rd_error_next = status_fifo_rd_error_reg;
821
822 op_table_update_status_ptr = status_fifo_rd_op_tag_reg;
823 op_table_update_status_error = status_fifo_rd_error_reg;
824 op_table_update_status_en = 1'b0;
825
826 op_table_write_complete_ptr = status_fifo_rd_op_tag_reg;
827 op_table_write_complete_en = 1'b0;
828

```

```

829     if (status_fifo_rd_valid_reg && (status_fifo_rd_mask_reg &
~out_done) == 0) begin
830         // got write completion, pop and return status
831         status_fifo_rd_valid_next = 1'b0;
832         op_table_update_status_en = 1'b1;
833
834         out_done_ack = status_fifo_rd_mask_reg;
835
836         if (status_fifo_rd_finish_reg) begin
837             // mark done
838             op_table_write_complete_ptr =
status_fifo_rd_op_tag_reg;
839             op_table_write_complete_en = 1'b1;
840         end
841     end
842
843     if (!status_fifo_rd_valid_next && status_fifo_rd_ptr_reg !
= status_fifo_wr_ptr_reg) begin
844         // status FIFO not empty
845         status_fifo_rd_op_tag_next =
status_fifo_op_tag[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]];
846         status_fifo_rd_mask_next =
status_fifo_mask[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
;
847         status_fifo_rd_finish_next =
status_fifo_finish[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]];
848         status_fifo_rd_error_next =
status_fifo_error[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]];
849         status_fifo_rd_valid_next = 1'b1;
850         status_fifo_rd_ptr_next = status_fifo_rd_ptr_reg + 1;
851     end
852
853     // commit operations in-order
854     op_table_finish_en = 1'b0;
855     dec_active_op = 1'b0;
856
857     if (m_axis_read_desc_status_valid_reg) begin
858         m_axis_read_desc_status_error_next = DMA_ERROR_NONE;
859     end else begin
860         m_axis_read_desc_status_error_next = m_ax-
is_read_desc_status_error_reg;
861     end
862
863     m_axis_read_desc_status_tag_next = op_t-
able_tag[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
864     m_axis_read_desc_status_valid_next = 1'b0;
865
866     stat_rd_op_finish_tag_next = stat_rd_op_finish_tag_reg;
867     stat_rd_op_finish_status_next = m_ax-
is_read_desc_status_error_next;
868     stat_rd_op_finish_valid_next = 1'b0;
869
870     if (op_t-
able_active[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] && op_t-
able_write_complete[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] &&
op_table_finish_ptr_reg != op_table_start_ptr_reg) begin
871         op_table_finish_en = 1'b1;
872         dec_active_op = 1'b1;
873
874         if (op_t-
able_error_a[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] != op_t-
able_error_b[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]]) begin
875             m_axis_read_desc_status_error_next = op_t-
able_error_code[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
876         end
877
878         stat_rd_op_finish_status_next = m_ax-
is_read_desc_status_error_next;
879
880         if (op_t-
able_last[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]]) begin
881             m_axis_read_desc_status_tag_next = op_t-
able_tag[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
882             m_axis_read_desc_status_valid_next = 1'b1;
883             stat_rd_op_finish_tag_next = stat_rd_op_fin-
ish_tag_reg + 1;
884             stat_rd_op_finish_valid_next = 1'b1;
885         end
886     end
887 end
888
889 always @(posedge clk) begin
890     req_state_reg <= req_state_next;
891     axi_state_reg <= axi_state_next;
892
893     req_axi_addr_reg <= req_axi_addr_next;
894     req_ram_sel_reg <= req_ram_sel_next;
895     req_ram_addr_reg <= req_ram_addr_next;
896     req_op_count_reg <= req_op_count_next;
897     req_tr_count_reg <= req_tr_count_next;
898     req_zero_len_reg <= req_zero_len_next;
899     req_tag_reg <= req_tag_next;
900
901     ram_sel_reg <= ram_sel_next;
902     addr_reg <= addr_next;
903     addr_delay_reg <= addr_delay_next;
904     op_count_reg <= op_count_next;

```

```

905 zero_len_reg <= zero_len_next;
906 ram_mask_reg <= ram_mask_next;
907 ram_mask_0_reg <= ram_mask_0_next;
908 ram_mask_1_reg <= ram_mask_1_next;
909 ram_wrap_reg <= ram_wrap_next;
910 cycle_byte_count_reg <= cycle_byte_count_next;
911 start_offset_reg <= start_offset_next;
912 end_offset_reg <= end_offset_next;
913 offset_reg <= offset_next;
914 op_tag_reg <= op_tag_next;
915
916 m_axi_rdata_int_reg <= m_axi_rdata_int_next;
917 m_axi_rvalid_int_reg <= m_axi_rvalid_int_next;
918
919 m_axi_arid_reg <= m_axi_arid_next;
920 m_axi_araddr_reg <= m_axi_araddr_next;
921 m_axi_arlen_reg <= m_axi_arlen_next;
922 m_axi_arvalid_reg <= m_axi_arvalid_next;
923 m_axi_rready_reg <= m_axi_rready_next;
924
925 s_axis_read_desc_ready_reg <= s_axis_read_desc_ready_next;
926
927 m_axis_read_desc_status_tag_reg <= m_ax-
is_read_desc_status_tag_next;
928 m_axis_read_desc_status_error_reg <= m_ax-
is_read_desc_status_error_next;
929 m_axis_read_desc_status_valid_reg <= m_ax-
is_read_desc_status_valid_next;
930
931 status_busy_reg <= active_op_count_reg != 0;
932
933 stat_rd_op_start_tag_reg <= stat_rd_op_start_tag_next;
934 stat_rd_op_start_len_reg <= stat_rd_op_start_len_next;
935 stat_rd_op_start_valid_reg <= stat_rd_op_start_valid_next;
936 stat_rd_op_finish_tag_reg <= stat_rd_op_finish_tag_next;
937 stat_rd_op_finish_status_reg <= stat_rd_op_fin-
ish_status_next;
938 stat_rd_op_finish_valid_reg <= stat_rd_op_fin-
ish_valid_next;
939 stat_rd_req_start_tag_reg <= stat_rd_req_start_tag_next;
940 stat_rd_req_start_len_reg <= stat_rd_req_start_len_next;
941 stat_rd_req_start_valid_reg <=
stat_rd_req_start_valid_next;
942 stat_rd_req_finish_tag_reg <= stat_rd_req_finish_tag_next;
943 stat_rd_req_finish_status_reg <= stat_rd_req_fin-
ish_status_next;
944 stat_rd_req_finish_valid_reg <= stat_rd_req_fin-
ish_valid_next;
945 stat_rd_op_table_full_reg <= stat_rd_op_table_full_next;
946 stat_rd_tx_stall_reg <= stat_rd_tx_stall_next;
947
948 if (status_fifo_we) begin
949
status_fifo_op_tag[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]] <= status_fifo_wr_op_tag;
950
status_fifo_mask[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_mask;
951
status_fifo_finish[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]] <= status_fifo_wr_finish;
952
status_fifo_error[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]
] <= status_fifo_wr_error;
953
status_fifo_wr_ptr_reg <= status_fifo_wr_ptr_reg + 1;
954 end
955 status_fifo_rd_ptr_reg <= status_fifo_rd_ptr_next;
956
957 status_fifo_mask_reg <= status_fifo_mask_next;
958 status_fifo_finish_reg <= status_fifo_finish_next;
959 status_fifo_error_reg <= status_fifo_error_next;
960 status_fifo_we_reg <= status_fifo_we_next;
961
962 status_fifo_rd_op_tag_reg <= status_fifo_rd_op_tag_next;
963 status_fifo_rd_mask_reg <= status_fifo_rd_mask_next;
964 status_fifo_rd_finish_reg <= status_fifo_rd_finish_next;
965 status_fifo_rd_error_reg <= status_fifo_rd_error_next;
966 status_fifo_rd_valid_reg <= status_fifo_rd_valid_next;
967
968 status_fifo_half_full_reg <= $un-
signed(status_fifo_wr_ptr_reg - status_fifo_rd_ptr_reg) >=
2*(STATUS_FIFO_ADDR_WIDTH-1);
969
970 active_op_count_reg <= active_op_count_reg + inc_active_op -
dec_active_op;
971
972 if (op_table_start_en) begin
973   op_table_start_ptr_reg <= op_table_start_ptr_reg + 1;
974   op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= 1'b1;
975   op_t-
able_axi_addr[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_axi_addr;
976   op_t-
able_ram_sel[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_ram_sel;
977   op_t-
able_ram_addr[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_ram_addr;

```

```

978   op_table_len[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]]
<= op_table_start_len;
979   op_t-
able_zero_len[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_zero_len;
980   op_t-
able_cycle_count[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_cycle_count;
981
982   op_table_tag[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]]
<= op_table_start_tag;
983   op_t-
able_last[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_last;
984   op_t-
able_write_complete[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <=
1'b0;
985   op_t-
able_error_a[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_error_b[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]];
986   end
987   if (!USE_AXI_ID && op_table_read_complete_en) begin
988     op_table_read_complete_ptr_reg <= op_t-
able_read_complete_ptr_reg + 1;
989   end
990
991   if (op_table_update_status_en) begin
992     if (op_table_update_status_error != 0) begin
993       op_table_error_code[op_table_update_status_ptr] <=
op_table_update_status_error;
994       op_table_error_b[op_table_update_status_ptr] <= !
op_table_error_a[op_table_update_status_ptr];
995     end
996   end
997
998   if (op_table_write_complete_en) begin
999     op_table_write_complete[op_table_write_complete_ptr]
<= 1'b1;
1000   end
1001
1002   if (op_table_finish_en) begin
1003     op_table_finish_ptr_reg <= op_table_finish_ptr_reg +
1;
1004     op_t-
able_active[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] <= 1'b0;
1005   end
1006
1007   if (rst) begin
1008     req_state_reg <= REQ_STATE_IDLE;
1009     axi_state_reg <= AXI_STATE_IDLE;
1010
m_axi_rdata_int_reg <= 1'b0;
1011
m_axi_arvalid_reg <= 1'b0;
1012
m_axi_rready_reg <= 1'b0;
1013
s_axis_read_desc_ready_reg <= 1'b0;
1014
m_axis_read_desc_status_error_reg = 4'd0;
1015
m_axis_read_desc_status_valid_reg <= 1'b0;
1016
status_busy_reg <= 1'b0;
1017
stat_rd_op_start_tag_reg <= 0;
1018
stat_rd_op_start_valid_reg <= 1'b0;
1019
stat_rd_op_finish_tag_reg <= 0;
1020
stat_rd_op_finish_valid_reg <= 1'b0;
1021
stat_rd_req_start_valid_reg <= 1'b0;
1022
stat_rd_req_finish_valid_reg <= 1'b0;
1023
stat_rd_op_table_full_reg <= 1'b0;
1024
stat_rd_tx_stall_reg <= 1'b0;
1025
status_fifo_wr_ptr_reg <= 0;
1026
status_fifo_rd_ptr_reg <= 0;
1027
status_fifo_we_reg <= 1'b0;
1028
status_fifo_rd_valid_reg <= 1'b0;
1029
active_op_count_reg <= 0;
1030
op_table_start_ptr_reg <= 0;
1031
op_table_read_complete_ptr_reg <= 0;
1032
op_table_finish_ptr_reg <= 0;
1033
op_table_active <= 0;
1034
end
1035 end
1036
1037 // output datapath logic (write data)
1038 generate
1039   genvar n;
1040   for (n = 0; n < RAM_SEG_COUNT; n = n + 1) begin
1041     reg [RAM_SEL_WIDTH-1:0] ram_wr_cmd_sel_reg =
{RAM_SEL_WIDTH{1'b0}};
1042     reg [RAM_SEG_BE_WIDTH-1:0] ram_wr_cmd_be_reg =
{RAM_SEG_BE_WIDTH{1'b0}};
1043     reg [RAM_SEG_ADDR_WIDTH-1:0] ram_wr_cmd_addr_reg =
{RAM_SEG_ADDR_WIDTH{1'b0}};
1044     reg [RAM_SEG_DATA_WIDTH-1:0] ram_wr_cmd_data_reg =

```



```

(RAM_SEG_DATA_WIDTH{1'b0});
1056 reg ram_wr_cmd_valid_reg = 1'b0;
1057
1058 reg [OUTPUT_FIFO_ADDR_WIDTH-1:0] out_fifo_wr_ptr_reg = 0;
1059 reg [OUTPUT_FIFO_ADDR_WIDTH-1:0] out_fifo_rd_ptr_reg = 0;
1060 reg out_fifo_half_full_reg = 1'b0;
1061
1062 wire out_fifo_full = out_fifo_wr_ptr_reg ==
(out_fifo_rd_ptr_reg ^ {1'b1, {OUTPUT_FIFO_ADDR_WIDTH{1'b0}}});
1063 wire out_fifo_empty = out_fifo_wr_ptr_reg ==
out_fifo_rd_ptr_reg;
1064
1065 (* ram_style = "distributed", ramstyle = "no_rw_check,
mlab" *)
1066 reg [RAM_SEL_WIDTH-1:0]
out_fifo_wr_cmd_sel[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
1067 (* ram_style = "distributed", ramstyle = "no_rw_check,
mlab" *)
1068 reg [RAM_SEG_BE_WIDTH-1:0]
out_fifo_wr_cmd_be[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
1069 (* ram_style = "distributed", ramstyle = "no_rw_check,
mlab" *)
1070 reg [RAM_SEG_ADDR_WIDTH-1:0]
out_fifo_wr_cmd_addr[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
1071 (* ram_style = "distributed", ramstyle = "no_rw_check,
mlab" *)
1072 reg [RAM_SEG_DATA_WIDTH-1:0]
out_fifo_wr_cmd_data[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
1073
1074 reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] done_count_reg = 0;
1075 reg done_reg = 1'b0;
1076
1077 assign ram_wr_cmd_ready_int[n +: 1] = !
out_fifo_half_full_reg;
1078
1079 assign ram_wr_cmd_sel[n*RAM_SEL_WIDTH +: RAM_SEL_WIDTH] =
ram_wr_cmd_sel_reg;
1080 assign ram_wr_cmd_be[n*RAM_SEG_BE_WIDTH +:
RAM_SEG_BE_WIDTH] = ram_wr_cmd_be_reg;
1081 assign ram_wr_cmd_addr[n*RAM_SEG_ADDR_WIDTH +: RAM_SEG_AD-
DR_WIDTH] = ram_wr_cmd_addr_reg;
1082 assign ram_wr_cmd_data[n*RAM_SEG_DATA_WIDTH +:
RAM_SEG_DATA_WIDTH] = ram_wr_cmd_data_reg;
1083 assign ram_wr_cmd_valid[n +: 1] = ram_wr_cmd_valid_reg;
1084
1085 assign out_done[n] = done_reg;
1086
1087 always @(posedge clk) begin
1088     ram_wr_cmd_valid_reg <= ram_wr_cmd_valid_reg && !
ram_wr_cmd_ready[n +: 1];
1089
1090     out_fifo_half_full_reg <= $un-
signed(out_fifo_wr_ptr_reg - out_fifo_rd_ptr_reg) >= 2** (OUT-
PUT_FIFO_ADDR_WIDTH-1);
1091
1092     if (!out_fifo_full && ram_wr_cmd_valid_int[n +: 1])
begin
1093
out_fifo_wr_cmd_sel[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
<= ram_wr_cmd_sel_int[n*RAM_SEL_WIDTH +: RAM_SEL_WIDTH];
1094
out_fifo_wr_cmd_be[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
<= ram_wr_cmd_be_int[n*RAM_SEG_BE_WIDTH +: RAM_SEG_BE_WIDTH];
1095
out_fifo_wr_cmd_addr[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]
] <= ram_wr_cmd_addr_int[n*RAM_SEG_ADDR_WIDTH +: RAM_SEG_AD-
DR_WIDTH];
1096
out_fifo_wr_cmd_data[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]
] <= ram_wr_cmd_data_int[n*RAM_SEG_DATA_WIDTH +:
RAM_SEG_DATA_WIDTH];
1097     out_fifo_wr_ptr_reg <= out_fifo_wr_ptr_reg + 1;
1098     end
1099
1100     if (!out_fifo_empty && (!ram_wr_cmd_valid_reg ||
ram_wr_cmd_ready[n +: 1])) begin
1101         ram_wr_cmd_sel_reg <=
out_fifo_wr_cmd_sel[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
;
1102         ram_wr_cmd_be_reg <=
out_fifo_wr_cmd_be[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
;
1103         ram_wr_cmd_addr_reg <=
out_fifo_wr_cmd_addr[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]
];
1104         ram_wr_cmd_data_reg <=
out_fifo_wr_cmd_data[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]
];
1105         ram_wr_cmd_valid_reg <= 1'b1;
1106         out_fifo_rd_ptr_reg <= out_fifo_rd_ptr_reg + 1;
1107     end
1108
1109     if (done_count_reg < 2**OUTPUT_FIFO_ADDR_WIDTH &&
ram_wr_done[n] && !out_done_ack[n]) begin
1110         done_count_reg <= done_count_reg + 1;
1111         done_reg <= 1;
1112     end else if (done_count_reg > 0 && !ram_wr_done[n] &&
out_done_ack[n]) begin
1113         done_count_reg <= done_count_reg - 1;
1114         done_reg <= done_count_reg > 1;
1115     end
1116

```

```

1117     if (rst) begin
1118         out_fifo_wr_ptr_reg <= 0;
1119         out_fifo_rd_ptr_reg <= 0;
1120         ram_wr_cmd_valid_reg <= 1'b0;
1121         done_count_reg <= 0;
1122         done_reg <= 1'b0;
1123     end
1124 end
1125
1126 end
1127
1128 endgenerate
1129
1130 endmodule
1131
1132 `resetall

```

7. alexforenrich_verilog-pcie/rtl/dma_if_axi_wr.v (1,249 lines)

```

1 /*
2
3 Copyright (c) 2021 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * AXI DMA write interface
33 */
34 module dma_if_axi_wr #
35 (
36     // Width of AXI data bus in bits
37     parameter AXI_DATA_WIDTH = 32,
38     // Width of AXI address bus in bits
39     parameter AXI_ADDR_WIDTH = 16,
40     // Width of AXI wstrb (width of data bus in words)
41     parameter AXI_STRB_WIDTH = (AXI_DATA_WIDTH/8),
42     // Width of AXI ID signal
43     parameter AXI_ID_WIDTH = 8,
44     // Maximum AXI burst length to generate
45     parameter AXI_MAX_BURST_LEN = 256,
46     // RAM select width
47     parameter RAM_SEL_WIDTH = 2,
48     // RAM address width
49     parameter RAM_ADDR_WIDTH = 16,
50     // RAM segment count
51     parameter RAM_SEG_COUNT = 2,
52     // RAM segment data width
53     parameter RAM_SEG_DATA_WIDTH = AXI_DATA_WIDTH*2/
RAM_SEG_COUNT,
54     // RAM segment byte enable width
55     parameter RAM_SEG_BE_WIDTH = RAM_SEG_DATA_WIDTH/8,
56     // RAM segment address width
57     parameter RAM_SEG_ADDR_WIDTH = RAM_ADDR_WIDTH-$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH),
58     // Immediate enable
59     parameter IMM_ENABLE = 0,
60     // Immediate width
61     parameter IMM_WIDTH = 32,
62     // Length field width
63     parameter LEN_WIDTH = 16,
64     // Tag field width
65     parameter TAG_WIDTH = 8,
66     // Operation table size

```

```

67     parameter OP_TABLE_SIZE = 2**(AXI_ID_WIDTH),
68     // Use AXI ID signals
69     parameter USE_AXI_ID = 1
70 )
71 (
72     input wire          clk,
73     input wire          rst,
74
75     /*
76      * AXI master interface
77      */
78     output wire [AXI_ID_WIDTH-1:0] m_axi_awid,
79     output wire [AXI_ADDR_WIDTH-1:0] m_axi_awaddr,
80     output wire [7:0] m_axi_awlen,
81     output wire [2:0] m_axi_awsz,
82     output wire [1:0] m_axi_awburst,
83     output wire m_axi_awlock,
84     output wire [3:0] m_axi_awcache,
85     output wire [2:0] m_axi_awprot,
86     output wire m_axi_awvalid,
87     input wire m_axi_awready,
88     output wire [AXI_DATA_WIDTH-1:0] m_axi_wdata,
89     output wire [AXI_STRB_WIDTH-1:0] m_axi_wstrb,
90     output wire m_axi_wlast,
91     output wire m_axi_wvalid,
92     input wire m_axi_wready,
93     input wire [AXI_ID_WIDTH-1:0] m_axi_bid,
94     input wire [1:0] m_axi_bresp,
95     input wire m_axi_bvalid,
96     output wire m_axi_bready,
97
98     /*
99      * AXI write descriptor input
100     */
101     input wire [AXI_ADDR_WIDTH-1:0] is_write_desc_axi_addr,
102     input wire [RAM_SEL_WIDTH-1:0] is_write_desc_ram_sel,
103     input wire [RAM_ADDR_WIDTH-1:0] is_write_desc_ram_addr,
104     input wire [IMM_WIDTH-1:0] is_write_desc_imm,
105     input wire is_write_desc_imm_en,
106     input wire [LEN_WIDTH-1:0] is_write_desc_len,
107     input wire [TAG_WIDTH-1:0] is_write_desc_tag,
108     input wire is_write_desc_valid,
109     output wire is_write_desc_ready,
110
111     /*
112      * AXI write descriptor status output
113      */
114     output wire [TAG_WIDTH-1:0] is_write_desc_status_tag,
115     output wire [3:0] is_write_desc_status_error,
116     output wire is_write_desc_status_valid,
117
118     /*
119      * RAM interface
120      */
121     output wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0] ram_rd_cmd_sel,
122     output wire [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0] ram_rd_cmd_addr,
123     output wire [RAM_SEG_COUNT-1:0] ram_rd_cmd_valid,
124     input wire [RAM_SEG_COUNT-1:0] ram_rd_cmd_ready,
125     input wire [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0] ram_rd_resp_data,
126     input wire [RAM_SEG_COUNT-1:0] ram_rd_resp_valid,
127     output wire [RAM_SEG_COUNT-1:0] ram_rd_resp_ready,
128
129     /*

```

```

130     * Configuration
131     */
132     input wire          en-
133
134     /*
135      * Status
136      */
137     output wire status_busy,
138
139     /*
140      * Statistics
141      */
142     output wire [$clog2(OP_TABLE_SIZE)-1:0] stat_wr_op_start_tag,
143     output wire [LEN_WIDTH-1:0] stat_wr_op_start_len,
144     output wire stat_wr_op_start_valid,
145     output wire [$clog2(OP_TABLE_SIZE)-1:0] stat_wr_op_finish_tag,
146     output wire [3:0] stat_wr_op_finish_status,
147     output wire stat_wr_op_finish_valid,
148     output wire [$clog2(OP_TABLE_SIZE)-1:0] stat_wr_req_start_tag,
149     output wire [12:0] stat_wr_req_start_len,
150     output wire stat_wr_req_start_valid,
151     output wire [$clog2(OP_TABLE_SIZE)-1:0] stat_wr_req_finish_tag,
152     output wire [3:0] stat_wr_req_finish_status,
153     output wire stat_wr_req_finish_valid,
154     output wire stat_wr_op_table_full,
155     output wire stat_wr_tx_stall
156 );
157
158 parameter RAM_DATA_WIDTH = RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH;
159 parameter RAM_WORD_WIDTH = RAM_SEG_BE_WIDTH;
160 parameter RAM_WORD_SIZE = RAM_SEG_DATA_WIDTH/RAM_WORD_WIDTH;
161
162 parameter AXI_WORD_WIDTH = AXI_STRB_WIDTH;
163 parameter AXI_WORD_SIZE = AXI_DATA_WIDTH/AXI_WORD_WIDTH;
164 parameter AXI_BURST_SIZE = $clog2(AXI_STRB_WIDTH);
165 parameter AXI_MAX_BURST_SIZE = AXI_MAX_BURST_LEN <<
AXI_BURST_SIZE;
166
167 parameter OFFSET_WIDTH = AXI_STRB_WIDTH > 1 ? $clog2(AXI_STRB_WIDTH) : 1;
168 parameter OFFSET_MASK = AXI_STRB_WIDTH > 1 ? {OFF-
SET_WIDTH{1'b1}} : 0;
169 parameter RAM_OFFSET_WIDTH = $clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH);
170 parameter ADDR_MASK = {AXI_ADDR_WIDTH{1'b1}} << $clog2(AXI_STRB_WIDTH);
171 parameter CYCLE_COUNT_WIDTH = LEN_WIDTH - AXI_BURST_SIZE + 1;
172
173 parameter MASK_FIFO_ADDR_WIDTH = $clog2(OP_TABLE_SIZE)+1;
174
175 parameter OP_TAG_WIDTH = $clog2(OP_TABLE_SIZE);
176
177 parameter OUTPUT_FIFO_ADDR_WIDTH = 5;
178
179 // bus width assertions
180 initial begin
181     if (AXI_WORD_SIZE * AXI_STRB_WIDTH != AXI_DATA_WIDTH) be-
gin
182         $error("Error: AXI data width not evenly divisble (in-
stance %m)");
183         $finish;
184     end
185
186     if (AXI_WORD_SIZE != RAM_WORD_SIZE) begin
187         $error("Error: word size mismatch (instance %m)");
188         $finish;
189     end
190
191     if (2**$clog2(AXI_WORD_WIDTH) != AXI_WORD_WIDTH) begin
192         $error("Error: AXI word width must be even power of
two (instance %m)");
193         $finish;
194     end
195
196     if (AXI_MAX_BURST_LEN < 1 || AXI_MAX_BURST_LEN > 256) be-
gin
197         $error("Error: AXI_MAX_BURST_LEN must be between 1 and
256 (instance %m)");
198         $finish;
199     end
200
201     if (RAM_SEG_COUNT < 2) begin
202         $error("Error: RAM interface requires at least 2 seg-
ments (instance %m)");

```

```

203     $finish;
204 end
205
206 if (RAM_DATA_WIDTH != AXI_DATA_WIDTH*2) begin
207     $error("Error: RAM interface width must be double the
AXI interface width (instance %m)");
208     $finish;
209 end
210
211 if (2**$clog2(RAM_WORD_WIDTH) != RAM_WORD_WIDTH) begin
212     $error("Error: RAM word width must be even power of
two (instance %m)");
213     $finish;
214 end
215
216 if (RAM_ADDR_WIDTH != RAM_SEG_ADDR_WIDTH+$clog2(RAM_SEG_COUNT)+$clog2(RAM_SEG_BE_WIDTH)) begin
217     $error("Error: RAM_ADDR_WIDTH does not match RAM configuration (instance %m)");
218     $finish;
219 end
220
221 if (OP_TABLE_SIZE > 2**AXI_ID_WIDTH) begin
222     $error("Error: AXI_ID_WIDTH insufficient for requested
OP_TABLE_SIZE (instance %m)");
223     $finish;
224 end
225
226 if (IMM_ENABLE && IMM_WIDTH > AXI_DATA_WIDTH) begin
227     $error("Error: IMM_WIDTH must not be larger than the
AXI interface width (instance %m)");
228     $finish;
229 end
230 end
231
232 localparam [1:0]
233     AXI_RESP_OKAY = 2'b00,
234     AXI_RESP_EXOKAY = 2'b01,
235     AXI_RESP_SLVERR = 2'b10,
236     AXI_RESP_DECERR = 2'b11;
237
238 localparam [3:0]
239     DMA_ERROR_NONE = 4'd0,
240     DMA_ERROR_TIMEOUT = 4'd1,
241     DMA_ERROR_PARITY = 4'd2,
242     DMA_ERROR_AXI_RD_SLVERR = 4'd4,
243     DMA_ERROR_AXI_RD_DECERR = 4'd5,
244     DMA_ERROR_AXI_WR_SLVERR = 4'd6,
245     DMA_ERROR_AXI_WR_DECERR = 4'd7,
246     DMA_ERROR_PCIE_FLR = 4'd8,
247     DMA_ERROR_PCIE_CPL_POISONED = 4'd9,
248     DMA_ERROR_PCIE_CPL_STATUS_UR = 4'd10,
249     DMA_ERROR_PCIE_CPL_STATUS_CA = 4'd11;
250
251 localparam [0:0]
252     REQ_STATE_IDLE = 1'd0,
253     REQ_STATE_START = 1'd1;
254
255 reg [0:0] req_state_reg = REQ_STATE_IDLE, req_state_next;
256
257 localparam [0:0]
258     READ_STATE_IDLE = 1'd0,
259     READ_STATE_READ = 1'd1;
260
261 reg [0:0] read_state_reg = READ_STATE_IDLE, read_state_next;
262
263 localparam [0:0]
264     AXI_STATE_IDLE = 1'd0,
265     AXI_STATE_TRANSFER = 1'd1;
266
267 reg [0:0] axi_state_reg = AXI_STATE_IDLE, axi_state_next;
268
269 // datapath control signals
270 reg mask_fifo_we;
271
272 reg read_cmd_ready;
273
274 reg [AXI_ADDR_WIDTH-1:0] req_axi_addr_reg = {AXI_ADDR_WIDTH{1'b0}}, req_axi_addr_next;
275 reg [RAM_SEL_WIDTH-1:0] ram_sel_reg = {RAM_SEL_WIDTH{1'b0}}, ram_sel_next;
276 reg [RAM_ADDR_WIDTH-1:0] ram_addr_reg = {RAM_ADDR_WIDTH{1'b0}}, ram_addr_next;
277 reg [IMM_WIDTH-1:0] imm_reg = {IMM_WIDTH{1'b0}}, imm_next;
278 reg [imm_en_reg = 1'b0, imm_en_next;
279 reg [LEN_WIDTH-1:0] op_count_reg = {LEN_WIDTH{1'b0}}, op_count_next;
280 reg [zero_len_reg = 1'b0, zero_len_next;
281 reg [LEN_WIDTH-1:0] tr_count_reg = {LEN_WIDTH{1'b0}}, tr_count_next;
282 reg [12:0] tr_word_count_reg = 13'd0, tr_word_count_next;
283 reg [TAG_WIDTH-1:0] tag_reg = {TAG_WIDTH{1'b0}}, tag_next;
284
285 reg [AXI_ADDR_WIDTH-1:0] read_axi_addr_reg = {AXI_ADDR_WIDTH{1'b0}}, read_axi_addr_next;
286 reg [RAM_SEL_WIDTH-1:0] read_ram_sel_reg = {RAM_SEL_WIDTH{1'b0}}, read_ram_sel_next;
287 reg [RAM_ADDR_WIDTH-1:0] read_ram_addr_reg = {RAM_ADDR_WIDTH{1'b0}}, read_ram_addr_next;
288 reg [read_imm_en_reg = 1'b0, read_imm_en_next;

```

```

289 reg [LEN_WIDTH-1:0] read_len_reg = {LEN_WIDTH{1'b0}}, read_len_next;
290 reg [RAM_SEG_COUNT-1:0] read_ram_mask_reg = {RAM_SEG_COUNT{1'b0}}, read_ram_mask_next;
291 reg [RAM_SEG_COUNT-1:0] read_ram_mask_0_reg = {RAM_SEG_COUNT{1'b0}}, read_ram_mask_0_next;
292 reg [RAM_SEG_COUNT-1:0] read_ram_mask_1_reg = {RAM_SEG_COUNT{1'b0}}, read_ram_mask_1_next;
293 reg [ram_wrap_reg = 1'b0, ram_wrap_next;
294 reg [CYCLE_COUNT_WIDTH-1:0] read_cycle_count_reg = {CYCLE_COUNT_WIDTH{1'b0}}, read_cycle_count_next;
295 reg [read_last_cycle_reg = 1'b0, read_last_cycle_next;
296 reg [OFFSET_WIDTH+1-1:0] cycle_byte_count_reg = {OFFSET_WIDTH+1{1'b0}}, cycle_byte_count_next;
297 reg [RAM_OFFSET_WIDTH-1:0] start_offset_reg = {RAM_OFFSET_WIDTH{1'b0}}, start_offset_next;
298 reg [RAM_OFFSET_WIDTH-1:0] end_offset_reg = {RAM_OFFSET_WIDTH{1'b0}}, end_offset_next;
299
300 reg [AXI_ADDR_WIDTH-1:0] axi_addr_reg = {AXI_ADDR_WIDTH{1'b0}}, axi_addr_next;
301 reg [IMM_WIDTH-1:0] axi_imm_reg = {IMM_WIDTH{1'b0}}, axi_imm_next;
302 reg [axi_imm_en_reg = 1'b0, axi_imm_en_next;
303 reg [12:0] axi_len_reg = 13'd0, axi_len_next;
304 reg [axi_zero_len_reg = 1'b0, axi_zero_len_next;
305 reg [RAM_OFFSET_WIDTH-1:0] offset_reg = {RAM_OFFSET_WIDTH{1'b0}}, offset_next;
306 reg [AXI_STRB_WIDTH-1:0] strb_offset_mask_reg = {AXI_STRB_WIDTH{1'b1}}, strb_offset_mask_next;
307 reg [OFFSET_WIDTH-1:0] last_cycle_offset_reg = {OFFSET_WIDTH{1'b0}}, last_cycle_offset_next;
308 reg [RAM_SEG_COUNT-1:0] ram_mask_reg = {RAM_SEG_COUNT{1'b0}}, ram_mask_next;
309 reg [ram_mask_valid_reg = 1'b0, ram_mask_valid_next;
310 reg [CYCLE_COUNT_WIDTH-1:0] cycle_count_reg = {CYCLE_COUNT_WIDTH{1'b0}}, cycle_count_next;
311 reg [last_cycle_reg = 1'b0, last_cycle_next;
312
313 reg [AXI_ADDR_WIDTH-1:0] read_cmd_axi_addr_reg = {AXI_ADDR_WIDTH{1'b0}}, read_cmd_axi_addr_next;
314 reg [RAM_SEL_WIDTH-1:0] read_cmd_ram_sel_reg = {RAM_SEL_WIDTH{1'b0}}, read_cmd_ram_sel_next;
315 reg [RAM_ADDR_WIDTH-1:0] read_cmd_ram_addr_reg = {RAM_ADDR_WIDTH{1'b0}}, read_cmd_ram_addr_next;
316 reg [read_cmd_imm_en_reg = 1'b0, read_cmd_imm_en_next;
317 reg [12:0] read_cmd_len_reg = 13'd0, read_cmd_len_next;
318 reg [CYCLE_COUNT_WIDTH-1:0] read_cmd_cycle_count_reg = {CYCLE_COUNT_WIDTH{1'b0}}, read_cmd_cycle_count_next;
319 reg [read_cmd_last_cycle_reg = 1'b0, read_cmd_last_cycle_next;
320 reg [read_cmd_valid_reg = 1'b0, read_cmd_valid_next;
321
322 reg [MASK_FIFO_ADDR_WIDTH+1-1:0] mask_fifo_wr_ptr_reg = 0;
323 reg [MASK_FIFO_ADDR_WIDTH+1-1:0] mask_fifo_rd_ptr_reg = 0, mask_fifo_rd_ptr_next;
324 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab" *)
325 reg [RAM_SEG_COUNT-1:0]
mask_fifo_mask[(2**MASK_FIFO_ADDR_WIDTH)-1:0];
326 reg [RAM_SEG_COUNT-1:0] mask_fifo_wr_mask;
327
328 wire mask_fifo_empty = mask_fifo_wr_ptr_reg == mask_fifo_rd_ptr_reg;
329 wire mask_fifo_full = mask_fifo_wr_ptr_reg == (mask_fifo_rd_ptr_reg ^ (1 << MASK_FIFO_ADDR_WIDTH));
330
331 reg [OP_TAG_WIDTH+1-1:0] active_op_count_reg = 0;
332 reg [inc_active_op;
333 reg [dec_active_op;
334
335 reg [AXI_ID_WIDTH-1:0] m_axi_awid_reg = {AXI_ID_WIDTH{1'b0}}, m_axi_awid_next;
336 reg [AXI_ADDR_WIDTH-1:0] m_axi_awaddr_reg = {AXI_ADDR_WIDTH{1'b0}}, m_axi_awaddr_next;
337 reg [7:0] m_axi_awlen_reg = 8'd0, m_axi_awlen_next;
338 reg [m_axi_awvalid_reg = 1'b0, m_axi_awvalid_next;
339 reg [m_axi_bready_reg = 1'b0, m_axi_bready_next;
340
341 reg [s_axis_write_desc_ready_reg = 1'b0, s_axis_write_desc_ready_next;
342
343 reg [TAG_WIDTH-1:0] m_axis_write_desc_status_tag_reg = {TAG_WIDTH{1'b0}}, m_axis_write_desc_status_tag_next;
344 reg [3:0] m_axis_write_desc_status_error_reg = 4'd0, m_axis_write_desc_status_error_next;
345 reg [m_axis_write_desc_status_valid_reg = 1'b0, m_axis_write_desc_status_valid_next;
346
347 reg [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0] ram_rd_cmd_sel_reg = 0, ram_rd_cmd_sel_next;
348 reg [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0] ram_rd_cmd_addr_reg = 0, ram_rd_cmd_addr_next;
349 reg [RAM_SEG_COUNT-1:0] ram_rd_cmd_valid_reg = 0, ram_rd_cmd_valid_next;
350 reg [RAM_SEG_COUNT-1:0] ram_rd_resp_ready_cmb;
351
352 reg [status_busy_reg = 1'b0;
353
354 reg [OP_TAG_WIDTH-1:0] stat_wr_op_start_tag_reg = 0, stat_wr_op_start_tag_next;

```

```

355 reg [LEN_WIDTH-1:0] stat_wr_op_start_len_reg = 0,
stat_wr_op_start_len_next;
356 reg stat_wr_op_start_valid_reg = 1'b0,
stat_wr_op_start_valid_next;
357 reg [OP_TAG_WIDTH-1:0] stat_wr_op_finish_tag_reg = 0,
stat_wr_op_finish_tag_next;
358 reg [3:0] stat_wr_op_finish_status_reg = 0, stat_wr_op_fin-
ish_status_next;
359 reg stat_wr_op_finish_valid_reg = 1'b0, stat_wr_op_fin-
ish_valid_next;
360 reg [OP_TAG_WIDTH-1:0] stat_wr_req_start_tag_reg = 0,
stat_wr_req_start_tag_next;
361 reg [12:0] stat_wr_req_start_len_reg = 13'd0,
stat_wr_req_start_len_next;
362 reg stat_wr_req_start_valid_reg = 1'b0,
stat_wr_req_start_valid_next;
363 reg [OP_TAG_WIDTH-1:0] stat_wr_req_finish_tag_reg = 0,
stat_wr_req_finish_tag_next;
364 reg [3:0] stat_wr_req_finish_status_reg = 0, stat_wr_req_fin-
ish_status_next;
365 reg stat_wr_req_finish_valid_reg = 1'b0, stat_wr_req_fin-
ish_valid_next;
366 reg stat_wr_op_table_full_reg = 1'b0, stat_wr_op_t-
able_full_next;
367 reg stat_wr_tx_stall_reg = 1'b0, stat_wr_tx_stall_next;
368
369 // internal datapath
370 reg [AXI_DATA_WIDTH-1:0] m_axi_wdata_int;
371 reg [AXI_STRB_WIDTH-1:0] m_axi_wstrb_int;
372 reg m_axi_wlast_int;
373 reg m_axi_wvalid_int;
374 wire m_axi_wready_int;
375
376 assign m_axi_awid = USE_AXI_ID ? m_axi_awid_reg :
{AXI_ID_WIDTH{1'b0}};
377 assign m_axi_awaddr = m_axi_awaddr_reg;
378 assign m_axi_awlen = m_axi_awlen_reg;
379 assign m_axi_awsz = AXI_BURST_SIZE;
380 assign m_axi_awburst = 2'b01;
381 assign m_axi_awlock = 1'b0;
382 assign m_axi_awcache = 4'b0011;
383 assign m_axi_awprot = 3'b010;
384 assign m_axi_awvalid = m_axi_awvalid_reg;
385 assign m_axi_bready = m_axi_bready_reg;
386
387 assign s_axis_write_desc_ready = s_axis_write_desc_ready_reg;
388
389 assign m_axis_write_desc_status_tag = m_ax-
is_write_desc_status_tag_reg;
390 assign m_axis_write_desc_status_error = m_ax-
is_write_desc_status_error_reg;
391 assign m_axis_write_desc_status_valid = m_ax-
is_write_desc_status_valid_reg;
392
393 assign ram_rd_cmd_sel = ram_rd_cmd_sel_reg;
394 assign ram_rd_cmd_addr = ram_rd_cmd_addr_reg;
395 assign ram_rd_cmd_valid = ram_rd_cmd_valid_reg;
396 assign ram_rd_resp_ready = ram_rd_resp_ready_cmb;
397
398 assign status_busy = status_busy_reg;
399
400 assign stat_wr_op_start_tag = stat_wr_op_start_tag_reg;
401 assign stat_wr_op_start_len = stat_wr_op_start_len_reg;
402 assign stat_wr_op_start_valid = stat_wr_op_start_valid_reg;
403 assign stat_wr_op_finish_tag = stat_wr_op_finish_tag_reg;
404 assign stat_wr_op_finish_status = stat_wr_op_fin-
ish_status_reg;
405 assign stat_wr_op_finish_valid = stat_wr_op_finish_valid_reg;
406 assign stat_wr_req_start_tag = stat_wr_req_start_tag_reg;
407 assign stat_wr_req_start_len = stat_wr_req_start_len_reg;
408 assign stat_wr_req_start_valid = stat_wr_req_start_valid_reg;
409 assign stat_wr_req_finish_tag = stat_wr_req_finish_tag_reg;
410 assign stat_wr_req_finish_status = stat_wr_req_fin-
ish_status_reg;
411 assign stat_wr_req_finish_valid = stat_wr_req_fin-
ish_valid_reg;
412 assign stat_wr_op_table_full = stat_wr_op_table_full_reg;
413 assign stat_wr_tx_stall = stat_wr_tx_stall_reg;
414
415 // operation tag management
416 reg [OP_TAG_WIDTH+1-1:0] op_table_start_ptr_reg = 0;
417 reg [AXI_ADDR_WIDTH-1:0] op_table_start_axi_addr;
418 reg [IMM_WIDTH-1:0] op_table_start_imm;
419 reg op_table_start_imm_en;
420 reg [11:0] op_table_start_len;
421 reg op_table_start_zero_len;
422 reg [CYCLE_COUNT_WIDTH-1:0] op_table_start_cycle_count;
423 reg [RAM_OFFSET_WIDTH-1:0] op_table_start_offset;
424 reg [TAG_WIDTH-1:0] op_table_start_tag;
425 reg op_table_start_last;
426 reg op_table_start_en;
427 reg [OP_TAG_WIDTH+1-1:0] op_table_tx_start_ptr_reg = 0;
428 reg op_table_tx_start_en;
429 reg [OP_TAG_WIDTH+1-1:0] op_table_tx_finish_ptr_reg = 0;
430 reg op_table_tx_finish_en;
431 reg [OP_TAG_WIDTH-1:0] op_table_write_complete_ptr;
432 reg [3:0] op_table_write_complete_error;
433 reg op_table_write_complete_en;
434 reg [OP_TAG_WIDTH+1-1:0] op_table_finish_ptr_reg = 0;
435 reg op_table_finish_en;
436

```

```

437 reg [2**OP_TAG_WIDTH-1:0] op_table_active = 0;
438 reg [2**OP_TAG_WIDTH-1:0] op_table_write_complete = 0;
439 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
440 reg [AXI_ADDR_WIDTH-1:0] op_t-
able_axi_addr[2**OP_TAG_WIDTH-1:0];
441 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
442 reg [IMM_WIDTH-1:0] op_table_imm[2**OP_TAG_WIDTH-1:0];
443 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
444 reg op_table_imm_en[2**OP_TAG_WIDTH-1:0];
445 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
446 reg [11:0] op_table_len[2**OP_TAG_WIDTH-1:0];
447 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
448 reg op_table_zero_len[2**OP_TAG_WIDTH-1:0];
449 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
450 reg [CYCLE_COUNT_WIDTH-1:0] op_t-
able_cycle_count[2**OP_TAG_WIDTH-1:0];
451 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
452 reg [RAM_OFFSET_WIDTH-1:0] op_t-
able_offset[2**OP_TAG_WIDTH-1:0];
453 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
454 reg [TAG_WIDTH-1:0] op_table_tag[2**OP_TAG_WIDTH-1:0];
455 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
456 reg op_table_last[2**OP_TAG_WIDTH-1:0];
457 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
458 reg [3:0] op_table_error_code [2**OP_TAG_WIDTH-1:0];
459
460 integer i;
461
462 initial begin
463   for (i = 0; i < 2**OP_TAG_WIDTH; i = i + 1) begin
464     op_table_axi_addr[i] = 0;
465     op_table_imm[i] = 0;
466     op_table_imm_en[i] = 0;
467     op_table_len[i] = 0;
468     op_table_zero_len[i] = 1'b0;
469     op_table_cycle_count[i] = 0;
470     op_table_offset[i] = 0;
471     op_table_tag[i] = 0;
472     op_table_last[i] = 0;
473     op_table_error_code[i] = 0;
474   end
475 end
476
477 always @* begin
478   req_state_next = REQ_STATE_IDLE;
479
480   s_axis_write_desc_ready_next = 1'b0;
481
482   stat_wr_op_start_tag_next = stat_wr_op_start_tag_reg;
483   stat_wr_op_start_len_next = stat_wr_op_start_len_reg;
484   stat_wr_op_start_valid_next = 1'b0;
485   stat_wr_req_start_tag_next = stat_wr_req_start_tag_reg;
486   stat_wr_req_start_len_next = stat_wr_req_start_len_reg;
487   stat_wr_req_start_valid_next = 1'b0;
488   stat_wr_op_table_full_next = !(op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2**OP_TAG_WIDTH));
489   stat_wr_tx_stall_next = (m_axi_awvalid && !m_axi_wready)
|| (m_axi_wvalid && !m_axi_wready);
490
491   tag_next = tag_reg;
492   req_axi_addr_next = req_axi_addr_reg;
493   ram_sel_next = ram_sel_reg;
494   ram_addr_next = ram_addr_reg;
495   imm_next = imm_reg;
496   imm_en_next = imm_en_reg;
497   op_count_next = op_count_reg;
498   zero_len_next = zero_len_reg;
499   tr_count_next = tr_count_reg;
500   tr_word_count_next = tr_word_count_reg;
501
502   read_cmd_axi_addr_next = read_cmd_axi_addr_reg;
503   read_cmd_ram_sel_next = read_cmd_ram_sel_reg;
504   read_cmd_ram_addr_next = read_cmd_ram_addr_reg;
505   read_cmd_imm_en_next = read_cmd_imm_en_reg;
506   read_cmd_len_next = read_cmd_len_reg;
507   read_cmd_cycle_count_next = read_cmd_cycle_count_reg;
508   read_cmd_last_cycle_next = read_cmd_last_cycle_reg;
509   read_cmd_valid_next = read_cmd_valid_reg && !
read_cmd_ready;
510
511   op_table_start_axi_addr = req_axi_addr_reg;
512   op_table_start_imm = imm_reg;
513   op_table_start_imm_en = imm_en_reg;
514   op_table_start_len = 0;
515   op_table_start_zero_len = zero_len_reg;
516   op_table_start_cycle_count = 0;
517   op_table_start_offset = (req_axi_addr_reg & OFFSET_MASK)-
ram_addr_reg[RAM_OFFSET_WIDTH-1:0];
518   op_table_start_tag = tag_reg;

```



```

519   op_table_start_last = 0;
520   op_table_start_en = 1'b0;
521
522   inc_active_op = 1'b0;
523
524   // TLP segmentation
525   case (req_state_reg)
526     REQ_STATE_IDLE: begin
527       // idle state, wait for incoming descriptor
528       s_axis_write_desc_ready_next = !op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2**OP_TAG_WIDTH) && enable;
529
530       req_axi_addr_next = s_axis_write_desc_axi_addr;
531       if (IMM_ENABLE && s_axis_write_desc_imm_en) begin
532         ram_sel_next = 0;
533         ram_addr_next = 0;
534       end else begin
535         ram_sel_next = s_axis_write_desc_ram_sel;
536         ram_addr_next = s_axis_write_desc_ram_addr;
537       end
538       imm_next = s_axis_write_desc_imm;
539       imm_en_next = IMM_ENABLE && s_ax-
is_write_desc_imm_en;
540       if (s_axis_write_desc_len == 0) begin
541         // zero-length operation
542         op_count_next = 1;
543         zero_len_next = 1'b1;
544       end else begin
545         op_count_next = s_axis_write_desc_len;
546         zero_len_next = 1'b0;
547       end
548       tag_next = s_axis_write_desc_tag;
549
550       if (op_count_next <= AXI_MAX_BURST_SIZE -
(req_axi_addr_next & OFFSET_MASK) || AXI_MAX_BURST_SIZE >= 4096) be-
gin
551         // packet smaller than max burst size
552         if (((req_axi_addr_next & 12'hfff) +
(op_count_next & 12'hfff)) >> 12 != 0 || op_count_next >> 12 != 0)
begin
553           // crosses 4k boundary
554           tr_word_count_next = 13'h1000 -
req_axi_addr_next[11:0];
555         end else begin
556           // does not cross 4k boundary
557           tr_word_count_next = op_count_next;
558         end
559       end else begin
560         // packet larger than max burst size
561         if (((req_axi_addr_next & 12'hfff) +
AXI_MAX_BURST_SIZE) >> 12 != 0) begin
562           // crosses 4k boundary
563           tr_word_count_next = 13'h1000 -
req_axi_addr_next[11:0];
564         end else begin
565           // does not cross 4k boundary
566           tr_word_count_next = AXI_MAX_BURST_SIZE -
(req_axi_addr_next & OFFSET_MASK);
567         end
568       end
569
570       if (s_axis_write_desc_ready & s_ax-
is_write_desc_valid) begin
571         s_axis_write_desc_ready_next = 1'b0;
572
573         stat_wr_op_start_tag_next =
stat_wr_op_start_tag_reg+1;
574         stat_wr_op_start_len_next = s_ax-
is_write_desc_len;
575         stat_wr_op_start_valid_next = 1'b1;
576
577         req_state_next = REQ_STATE_START;
578       end else begin
579         req_state_next = REQ_STATE_IDLE;
580       end
581     end
582     REQ_STATE_START: begin
583       // start state, compute length
584       if (!op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2**OP_TAG_WIDTH) && (!ram_rd_cmd_valid_reg || ram_rd_cmd_ready) &&
(!read_cmd_valid_reg || read_cmd_ready)) begin
585         read_cmd_axi_addr_next = req_axi_addr_reg;
586         read_cmd_ram_sel_next = ram_sel_reg;
587         read_cmd_ram_addr_next = ram_addr_reg;
588         read_cmd_imm_en_next = imm_en_reg;
589         read_cmd_len_next = tr_word_count_next;
590         read_cmd_cycle_count_next =
(tr_word_count_next + (req_axi_addr_reg & OFFSET_MASK) - 1) >>
AXI_BURST_SIZE;
591         op_table_start_cycle_count =
read_cmd_cycle_count_next;
592         read_cmd_last_cycle_next =
read_cmd_cycle_count_next == 0;
593         read_cmd_valid_next = 1'b1;
594
595         req_axi_addr_next = req_axi_addr_reg +
tr_word_count_next;

```

```

596         ram_addr_next = ram_addr_reg +
tr_word_count_next;
597         op_count_next = op_count_reg -
tr_word_count_next;
598
599         op_table_start_axi_addr = req_axi_addr_reg;
600         op_table_start_imm = imm_reg;
601         op_table_start_imm_en = imm_en_reg;
602         op_table_start_len = tr_word_count_next;
603         op_table_start_zero_len = zero_len_reg;
604         op_table_start_offset = (req_axi_addr_reg &
OFFSET_MASK)-ram_addr_reg[RAM_OFFSET_WIDTH-1:0];
605         op_table_start_tag = tag_reg;
606         op_table_start_last = op_count_reg ==
tr_word_count_next;
607         op_table_start_en = 1'b1;
608         inc_active_op = 1'b1;
609
610         stat_wr_req_start_tag_next = op_t-
able_start_ptr_reg[OP_TAG_WIDTH-1:0];
611         stat_wr_req_start_len_next = zero_len_reg ?
0 : tr_word_count_next;
612         stat_wr_req_start_valid_next = 1'b1;
613
614         if (op_count_next <= AXI_MAX_BURST_SIZE -
(req_axi_addr_next & OFFSET_MASK) || AXI_MAX_BURST_SIZE >= 4096) be-
gin
615           // packet smaller than max burst size
616           if (((req_axi_addr_next & 12'hfff) +
(op_count_next & 12'hfff)) >> 12 != 0 || op_count_next >> 12 != 0)
begin
617             // crosses 4k boundary
618             tr_word_count_next = 13'h1000 -
req_axi_addr_next[11:0];
619           end else begin
620             // does not cross 4k boundary
621             tr_word_count_next = op_count_next;
622           end
623         end else begin
624           // packet larger than max burst size
625           if (((req_axi_addr_next & 12'hfff) +
AXI_MAX_BURST_SIZE) >> 12 != 0) begin
626             // crosses 4k boundary
627             tr_word_count_next = 13'h1000 -
req_axi_addr_next[11:0];
628           end else begin
629             // does not cross 4k boundary
630             tr_word_count_next =
AXI_MAX_BURST_SIZE - (req_axi_addr_next & OFFSET_MASK);
631           end
632         end
633
634         if (!op_table_start_last) begin
635           req_state_next = REQ_STATE_START;
636         end else begin
637           s_axis_write_desc_ready_next = !op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2**OP_TAG_WIDTH) && enable;
638           req_state_next = REQ_STATE_IDLE;
639         end
640       end else begin
641         req_state_next = REQ_STATE_START;
642       end
643     end
644   endcase
645 end
646
647 always @* begin
648   read_state_next = READ_STATE_IDLE;
649
650   read_cmd_ready = 1'b0;
651
652   ram_rd_cmd_sel_next = ram_rd_cmd_sel_reg;
653   ram_rd_cmd_addr_next = ram_rd_cmd_addr_reg;
654   ram_rd_cmd_valid_next = ram_rd_cmd_valid_reg &
~ram_rd_cmd_ready;
655
656   read_axi_addr_next = read_axi_addr_reg;
657   read_ram_sel_next = read_ram_sel_reg;
658   read_ram_addr_next = read_ram_addr_reg;
659   read_imm_en_next = read_imm_en_reg;
660   read_len_next = read_len_reg;
661   read_ram_mask_next = read_ram_mask_reg;
662   read_ram_mask_0_next = read_ram_mask_0_reg;
663   read_ram_mask_1_next = read_ram_mask_1_reg;
664   ram_wrap_next = ram_wrap_reg;
665   read_cycle_count_next = read_cycle_count_reg;
666   read_last_cycle_next = read_last_cycle_reg;
667   cycle_byte_count_next = cycle_byte_count_reg;
668   start_offset_next = start_offset_reg;
669   end_offset_next = end_offset_reg;
670
671   mask_fifo_wr_mask = read_ram_mask_reg;
672   mask_fifo_we = 1'b0;
673
674   // Read request generation
675   case (read_state_reg)
676     READ_STATE_IDLE: begin
677       // idle state, wait for read command
678

```

```

679         read_axi_addr_next = read_cmd_axi_addr_reg;
680         read_ram_sel_next = read_cmd_ram_sel_reg;
681         read_ram_addr_next = read_cmd_ram_addr_reg;
682         read_imm_en_next = read_cmd_imm_en_reg;
683         read_len_next = read_cmd_len_reg;
684         read_cycle_count_next = read_cmd_cycle_count_reg;
685         read_last_cycle_next = read_cmd_last_cycle_reg;
686
687         if (read_len_next > AXI_STRB_WIDTH-(read_axi_ad-
dr_next & OFFSET_MASK)) begin
688             cycle_byte_count_next = AXI_STRB_WIDTH-
(read_axi_addr_next & OFFSET_MASK);
689         end else begin
690             cycle_byte_count_next = read_len_next;
691         end
692         start_offset_next = read_ram_addr_next;
693         {ram_wrap_next, end_offset_next} =
start_offset_next+cycle_byte_count_next-1;
694
695         read_ram_mask_0_next = {RAM_SEG_COUNT{1'b1}} <<
(start_offset_next >> $clog2(RAM_SEG_BE_WIDTH));
696         read_ram_mask_1_next = {RAM_SEG_COUNT{1'b1}} >>
(RAM_SEG_COUNT-1-(end_offset_next >> $clog2(RAM_SEG_BE_WIDTH)));
697
698         if (!ram_wrap_next) begin
699             read_ram_mask_next = read_ram_mask_0_next &
read_ram_mask_1_next;
700             read_ram_mask_0_next = read_ram_mask_0_next &
read_ram_mask_1_next;
701             read_ram_mask_1_next = 0;
702         end else begin
703             read_ram_mask_next = read_ram_mask_0_next |
read_ram_mask_1_next;
704         end
705
706         if (read_cmd_valid_reg) begin
707             read_cmd_ready = 1'b1;
708             read_state_next = READ_STATE_READ;
709         end else begin
710             read_state_next = READ_STATE_IDLE;
711         end
712     end
713     READ_STATE_READ: begin
714         // read state - start new read operations
715
716         if (!ram_rd_cmd_valid & ~ram_rd_cmd_ready &
read_ram_mask_reg) && !mask_fifo_full) begin
717
718             // update counters
719             read_ram_addr_next = read_ram_addr_reg +
cycle_byte_count_reg;
720             read_len_next = read_len_reg -
cycle_byte_count_reg;
721
722             read_cycle_count_next = read_cycle_count_reg -
1;
723             read_last_cycle_next = read_cycle_count_next
== 0;
724
725             for (i = 0; i < RAM_SEG_COUNT; i = i + 1) be-
gin
726                 if (read_ram_mask_reg[i]) begin
727                     ram_rd_cmd_sel_next[i*RAM_SEL_WIDTH +:
RAM_SEL_WIDTH] = read_ram_sel_reg;
728                     ram_rd_cmd_ad-
dr_next[i*RAM_SEG_ADDR_WIDTH +: RAM_SEG_ADDR_WIDTH] = read_ram_ad-
dr_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-RAM_SEG_ADDR_WIDTH];
729                     ram_rd_cmd_valid_next[i] = !(IMM_EN-
ABLE && read_imm_en_reg);
730                 end
731                 if (read_ram_mask_1_reg[i]) begin
732                     ram_rd_cmd_ad-
dr_next[i*RAM_SEG_ADDR_WIDTH +: RAM_SEG_ADDR_WIDTH] = read_ram_ad-
dr_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-RAM_SEG_ADDR_WIDTH]+1;
733                 end
734             end
735             mask_fifo_wr_mask = (IMM_ENABLE &&
read_imm_en_reg) ? 0 : read_ram_mask_reg;
736             mask_fifo_we = 1'b1;
737
738             if (read_len_next > AXI_STRB_WIDTH) begin
739                 cycle_byte_count_next = AXI_STRB_WIDTH;
740             end else begin
741                 cycle_byte_count_next = read_len_next;
742             end
743             start_offset_next = read_ram_addr_next;
744             {ram_wrap_next, end_offset_next} =
start_offset_next+cycle_byte_count_next-1;
745
746             read_ram_mask_0_next = {RAM_SEG_COUNT{1'b1}}
<< (start_offset_next >> $clog2(RAM_SEG_BE_WIDTH));
747             read_ram_mask_1_next = {RAM_SEG_COUNT{1'b1}}
>> (RAM_SEG_COUNT-1-(end_offset_next >> $clog2(RAM_SEG_BE_WIDTH)));
748
749             if (!ram_wrap_next) begin
750                 read_ram_mask_next = read_ram_mask_0_next
& read_ram_mask_1_next;
751                 read_ram_mask_0_next =
read_ram_mask_0_next & read_ram_mask_1_next;

```

```

752                 read_ram_mask_1_next = 0;
753             end else begin
754                 read_ram_mask_next = read_ram_mask_0_next
| read_ram_mask_1_next;
755             end
756
757             if (!read_last_cycle_reg) begin
758                 read_state_next = READ_STATE_READ;
759             end else if (read_cmd_valid_reg) begin
760
761                 read_axi_addr_next = read_cmd_axi_ad-
dr_reg;
762                 read_ram_sel_next = read_cmd_ram_sel_reg;
763                 read_ram_addr_next = read_cmd_ram_ad-
dr_reg;
764                 read_imm_en_next = read_cmd_imm_en_reg;
765                 read_len_next = read_cmd_len_reg;
766                 read_cycle_count_next =
read_cmd_cycle_count_reg;
767                 read_last_cycle_next =
read_cmd_last_cycle_reg;
768
769                 if (read_len_next > AXI_STRB_WIDTH-
(read_axi_addr_next & OFFSET_MASK)) begin
770                     cycle_byte_count_next =
AXI_STRB_WIDTH-(read_axi_addr_next & OFFSET_MASK);
771                 end else begin
772                     cycle_byte_count_next = read_len_next;
773                 end
774                 start_offset_next = read_ram_addr_next;
775                 {ram_wrap_next, end_offset_next} =
start_offset_next+cycle_byte_count_next-1;
776
777                 read_ram_mask_0_next =
{RAM_SEG_COUNT{1'b1}} << (start_offset_next >> $clo-
g2(RAM_SEG_BE_WIDTH));
778                 read_ram_mask_1_next =
{RAM_SEG_COUNT{1'b1}} >> (RAM_SEG_COUNT-1-(end_offset_next >> $clo-
g2(RAM_SEG_BE_WIDTH)));
779
780                 if (!ram_wrap_next) begin
781                     read_ram_mask_next =
read_ram_mask_0_next & read_ram_mask_1_next;
782                     read_ram_mask_0_next =
read_ram_mask_0_next & read_ram_mask_1_next;
783                     read_ram_mask_1_next = 0;
784                 end else begin
785                     read_ram_mask_next =
read_ram_mask_0_next | read_ram_mask_1_next;
786                 end
787
788                 read_cmd_ready = 1'b1;
789
790                 read_state_next = READ_STATE_READ;
791             end else begin
792                 read_state_next = READ_STATE_IDLE;
793             end
794         end else begin
795             read_state_next = READ_STATE_READ;
796         end
797     end
798 endcase
799 end
800
801 always @* begin
802     axi_state_next = AXI_STATE_IDLE;
803
804     ram_rd_resp_ready_cmb = {RAM_SEG_COUNT{1'b0}};
805
806     stat_wr_op_finish_tag_next = stat_wr_op_finish_tag_reg;
807     stat_wr_op_finish_status_next = stat_wr_op_fin-
ish_status_reg;
808     stat_wr_op_finish_valid_next = 1'b0;
809     stat_wr_req_finish_tag_next = stat_wr_req_finish_tag_reg;
810     stat_wr_req_finish_status_next = stat_wr_req_fin-
ish_status_reg;
811     stat_wr_req_finish_valid_next = 1'b0;
812
813     axi_addr_next = axi_addr_reg;
814     axi_imm_next = axi_imm_reg;
815     axi_imm_en_next = axi_imm_en_reg;
816     axi_len_next = axi_len_reg;
817     axi_zero_len_next = axi_zero_len_reg;
818     offset_next = offset_reg;
819     strb_offset_mask_next = strb_offset_mask_reg;
820     last_cycle_offset_next = last_cycle_offset_reg;
821     ram_mask_next = ram_mask_reg;
822     ram_mask_valid_next = ram_mask_valid_reg;
823     cycle_count_next = cycle_count_reg;
824     last_cycle_next = last_cycle_reg;
825
826     mask_fifo_rd_ptr_next = mask_fifo_rd_ptr_reg;
827
828     op_table_tx_start_en = 1'b0;
829     op_table_tx_finish_en = 1'b0;
830
831     m_axi_awid_next = m_axi_awid_reg;
832     m_axi_awaddr_next = m_axi_awaddr_reg;
833     m_axi_awlen_next = m_axi_awlen_reg;
834     m_axi_awvalid_next = m_axi_awvalid_reg && !m_axi_awready;
835     m_axi_bready_next = 1'b0;

```

```

836
837   m_axi_wdata_int = ((IMM_ENABLE && axi_imm_en_reg) ?
{2{RAM_DATA_WIDTH{1'b0}} | axi_imm_reg}} : {2{ram_rd_resp_data}})
>> (RAM_DATA_WIDTH-offset_reg*AXI_WORD_SIZE);
838   m_axi_wstrb_int = strb_offset_mask_reg;
839   m_axi_wlast_int = 1'b0;
840   m_axi_wvalid_int = 1'b0;
841
842   // read response processing and AXI write generation
843   case (axi_state_reg)
844     AXI_STATE_IDLE: begin
845       // idle state, wait for command
846       ram_rd_resp_ready_cmb = {RAM_SEG_COUNT{1'b0}};
847
848       axi_addr_next = op_t-
able_axi_addr[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
849       axi_imm_next = op_t-
able_imm[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
850       axi_imm_en_next = op_t-
able_imm_en[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
851       axi_len_next = op_t-
able_len[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
852       axi_zero_len_next = op_t-
able_zero_len[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
853       offset_next = op_t-
able_offset[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
854       strb_offset_mask_next = axi_zero_len_next ?
{AXI_STRB_WIDTH{1'b0}} : ({AXI_STRB_WIDTH{1'b1}} << (axi_addr_next &
OFFSET_MASK));
855       last_cycle_offset_next = axi_addr_next +
(axi_len_next & OFFSET_MASK);
856       cycle_count_next = op_t-
able_cycle_count[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
857       last_cycle_next = op_t-
able_cycle_count[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]] == 0;
858
859       if (op_t-
able_active[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]] && op_t-
able_tx_start_ptr_reg != op_table_start_ptr_reg && (!m_axi_awval-
id_reg || m_axi_awready)) begin
860         m_axi_awid_next = op_t-
able_tx_start_ptr_reg[OP_TAG_WIDTH-1:0];
861         m_axi_awaddr_next = axi_addr_next;
862         m_axi_awlen_next = cycle_count_next;
863         m_axi_awvalid_next = 1'b1;
864         op_table_tx_start_en = 1'b1;
865         axi_state_next = AXI_STATE_TRANSFER;
866       end else begin
867         axi_state_next = AXI_STATE_IDLE;
868       end
869     end
870     AXI_STATE_TRANSFER: begin
871       // transfer state, transfer data
872       ram_rd_resp_ready_cmb = {RAM_SEG_COUNT{1'b0}};
873
874       if (!(ram_mask_reg & ~ram_rd_resp_valid) &&
ram_mask_valid_reg && m_axi_wready_int) begin
875         // transfer in read data
876         ram_rd_resp_ready_cmb = ram_mask_reg;
877         ram_mask_valid_next = 1'b0;
878
879         // update counters
880         cycle_count_next = cycle_count_reg - 1;
881         last_cycle_next = cycle_count_next == 0;
882         offset_next = offset_reg + AXI_STRB_WIDTH;
883         strb_offset_mask_next =
{AXI_STRB_WIDTH{1'b1}};
884
885         m_axi_wdata_int = ((IMM_ENABLE &&
axi_imm_en_reg) ? {2{RAM_DATA_WIDTH{1'b0}} | axi_imm_reg}} :
{2{ram_rd_resp_data}}) >> (RAM_DATA_WIDTH-offset_reg*AXI_WORD_SIZE);
886         m_axi_wstrb_int = strb_offset_mask_reg;
887         m_axi_wlast_int = 1'b0;
888         m_axi_wvalid_int = 1'b1;
889
890         if (last_cycle_reg) begin
891           // no more data to transfer, finish opera-
tion
892             m_axi_wlast_int = 1'b1;
893             op_table_tx_finish_en = 1'b1;
894
895             if (last_cycle_offset_reg) begin
896               m_axi_wstrb_int = strb_offset_mask_reg &
{AXI_STRB_WIDTH{1'b1}} >> (AXI_STRB_WIDTH - last_cycle_offset_reg);
897             end
898
899             // skip idle state if possible
900             axi_addr_next = op_t-
able_axi_addr[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
901             axi_imm_next = op_t-
able_imm[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
902             axi_imm_en_next = op_t-
able_imm_en[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
903             axi_len_next = op_t-
able_len[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
904             axi_zero_len_next = op_t-
able_zero_len[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
905             offset_next = op_t-
able_offset[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
906             strb_offset_mask_next =

```

```

axi_zero_len_next ? {AXI_STRB_WIDTH{1'b0}} : ({AXI_STRB_WIDTH{1'b1}}
<< (axi_addr_next & OFFSET_MASK));
907       last_cycle_offset_next = axi_addr_next +
(axi_len_next & OFFSET_MASK);
908       cycle_count_next = op_t-
able_cycle_count[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
909       last_cycle_next = op_t-
able_cycle_count[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]] == 0;
910
911       if (op_t-
able_active[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]] && op_t-
able_tx_start_ptr_reg != op_table_start_ptr_reg && (!m_axi_awval-
id_reg || m_axi_awready)) begin
912         m_axi_awid_next = op_t-
able_tx_start_ptr_reg[OP_TAG_WIDTH-1:0];
913         m_axi_awaddr_next = axi_addr_next;
914         m_axi_awlen_next = cycle_count_next;
915         m_axi_awvalid_next = 1'b1;
916         op_table_tx_start_en = 1'b1;
917         axi_state_next = AXI_STATE_TRANSFER;
918       end else begin
919         axi_state_next = AXI_STATE_IDLE;
920       end
921     end else begin
922       axi_state_next = AXI_STATE_TRANSFER;
923     end
924   end else begin
925     axi_state_next = AXI_STATE_TRANSFER;
926   end
927 end
928 endcase
929
930 if (!ram_mask_valid_next && !mask_fifo_empty) begin
931   ram_mask_next =
mask_fifo_mask[mask_fifo_rd_ptr_reg[MASK_FIFO_ADDR_WIDTH-1:0]];
932   ram_mask_valid_next = 1'b1;
933   mask_fifo_rd_ptr_next = mask_fifo_rd_ptr_reg+1;
934 end
935
936 op_table_write_complete_ptr = m_axi_bid;
937 if (m_axi_bresp == AXI_RESP_SLVERR) begin
938   op_table_write_complete_error = DMA_ER-
ROR_AXI_WR_SLVERR;
939 end else if (m_axi_bresp == AXI_RESP_DECERR) begin
940   op_table_write_complete_error = DMA_ER-
ROR_AXI_WR_DECERR;
941 end else begin
942   op_table_write_complete_error = DMA_ERROR_NONE;
943 end
944 op_table_write_complete_en = 1'b0;
945
946 m_axis_write_desc_status_tag_next = op_t-
able_tag[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
947 if (m_axis_write_desc_status_valid_reg) begin
948   m_axis_write_desc_status_error_next = DMA_ERROR_NONE;
949 end else begin
950   m_axis_write_desc_status_error_next = m_ax-
is_write_desc_status_error_reg;
951 end
952 m_axis_write_desc_status_valid_next = 1'b0;
953
954 stat_wr_req_finish_status_next = op_t-
able_write_complete_error;
955 stat_wr_req_finish_valid_next = 1'b0;
956
957 stat_wr_op_finish_tag_next = stat_wr_op_finish_tag_reg;
958 stat_wr_op_finish_status_next = m_ax-
is_write_desc_status_error_next;
959 stat_wr_op_finish_valid_next = 1'b0;
960
961 if (USE_AXI_ID) begin
962   // accept write completions
963   stat_wr_req_finish_tag_next = m_axi_bid;
964
965   m_axi_bready_next = 1'b1;
966   if (m_axi_bready && m_axi_bvalid) begin
967     op_table_write_complete_ptr = m_axi_bid;
968     op_table_write_complete_en = 1'b1;
969     stat_wr_req_finish_valid_next = 1'b1;
970   end
971
972   // commit operations in-order
973   op_table_finish_en = 1'b0;
974   dec_active_op = 1'b0;
975
976   if (op_t-
able_active[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] && op_t-
able_write_complete[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] &&
op_table_finish_ptr_reg != op_table_tx_finish_ptr_reg) begin
977     op_table_finish_en = 1'b1;
978     dec_active_op = 1'b1;
979
980     if (op_t-
able_error_code[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] !=
DMA_ERROR_NONE) begin
981       m_axis_write_desc_status_error_next = op_t-
able_error_code[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
982     end
983
984     stat_wr_op_finish_status_next = m_ax-
is_write_desc_status_error_next;

```

```

985
986     if (op_t-
able_last[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]]) begin
987         m_axis_write_desc_status_tag_next = op_t-
able_tag[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
988         m_axis_write_desc_status_valid_next = 1'b1;
989         stat_wr_op_finish_tag_next = stat_wr_op_fin-
ish_tag_reg + 1;
990         stat_wr_op_finish_valid_next = 1'b1;
991     end
992 end
993 end else begin
994     // accept write completions
995     op_table_finish_en = 1'b0;
996     dec_active_op = 1'b0;
997
998     stat_wr_req_finish_tag_next = op_t-
able_finish_ptr_reg[OP_TAG_WIDTH-1:0];
999
1000     m_axi_bready_next = 1'b1;
1001     if (m_axi_bready && m_axi_bvalid) begin
1002         op_table_finish_en = 1'b1;
1003         dec_active_op = 1'b1;
1004         stat_wr_req_finish_valid_next = 1'b1;
1005
1006         if (m_axi_bresp == AXI_RESP_SLVERR) begin
1007             m_axis_write_desc_status_error_next = DMA_ER-
ROR_AXI_WL_SLVERR;
1008         end else if (m_axi_bresp == AXI_RESP_DECERR) begin
1009             m_axis_write_desc_status_error_next = DMA_ER-
ROR_AXI_WL_DECERR;
1010         end
1011
1012         stat_wr_op_finish_status_next = m_ax-
is_write_desc_status_error_next;
1013
1014         if (op_t-
able_last[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]]) begin
1015             m_axis_write_desc_status_tag_next = op_t-
able_tag[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
1016             m_axis_write_desc_status_valid_next = 1'b1;
1017             stat_wr_op_finish_tag_next = stat_wr_op_fin-
ish_tag_reg + 1;
1018             stat_wr_op_finish_valid_next = 1'b1;
1019         end
1020     end
1021 end
1022 end
1023
1024 always @(posedge clk) begin
1025     req_state_reg <= req_state_next;
1026     read_state_reg <= read_state_next;
1027     axi_state_reg <= axi_state_next;
1028
1029     req_axi_addr_reg <= req_axi_addr_next;
1030     ram_sel_reg <= ram_sel_next;
1031     ram_addr_reg <= ram_addr_next;
1032     imm_reg <= imm_next;
1033     imm_en_reg <= imm_en_next;
1034     op_count_reg <= op_count_next;
1035     zero_len_reg <= zero_len_next;
1036     tr_count_reg <= tr_count_next;
1037     tr_word_count_reg <= tr_word_count_next;
1038     tag_reg <= tag_next;
1039
1040     read_axi_addr_reg <= read_axi_addr_next;
1041     read_ram_sel_reg <= read_ram_sel_next;
1042     read_ram_addr_reg <= read_ram_addr_next;
1043     read_imm_en_reg <= read_imm_en_next;
1044     read_len_reg <= read_len_next;
1045     read_ram_mask_reg <= read_ram_mask_next;
1046     read_ram_mask_0_reg <= read_ram_mask_0_next;
1047     read_ram_mask_1_reg <= read_ram_mask_1_next;
1048     ram_wrap_reg <= ram_wrap_next;
1049     read_cycle_count_reg <= read_cycle_count_next;
1050     read_last_cycle_reg <= read_last_cycle_next;
1051     cycle_byte_count_reg <= cycle_byte_count_next;
1052     start_offset_reg <= start_offset_next;
1053     end_offset_reg <= end_offset_next;
1054
1055     axi_addr_reg <= axi_addr_next;
1056     axi_imm_reg <= axi_imm_next;
1057     axi_imm_en_reg <= axi_imm_en_next;
1058     axi_len_reg <= axi_len_next;
1059     axi_zero_len_reg <= axi_zero_len_next;
1060     offset_reg <= offset_next;
1061     strb_offset_mask_reg <= strb_offset_mask_next;
1062     last_cycle_offset_reg <= last_cycle_offset_next;
1063     ram_mask_reg <= ram_mask_next;
1064     ram_mask_valid_reg <= ram_mask_valid_next;
1065     cycle_count_reg <= cycle_count_next;
1066     last_cycle_reg <= last_cycle_next;
1067
1068     read_cmd_axi_addr_reg <= read_cmd_axi_addr_next;
1069     read_cmd_ram_sel_reg <= read_cmd_ram_sel_next;
1070     read_cmd_ram_addr_reg <= read_cmd_ram_addr_next;
1071     read_cmd_imm_en_reg <= read_cmd_imm_en_next;
1072     read_cmd_len_reg <= read_cmd_len_next;
1073     read_cmd_cycle_count_reg <= read_cmd_cycle_count_next;
1074     read_cmd_last_cycle_reg <= read_cmd_last_cycle_next;
1075     read_cmd_valid_reg <= read_cmd_valid_next;

```

```

1076
1077     m_axi_awid_reg <= m_axi_awid_next;
1078     m_axi_awaddr_reg <= m_axi_awaddr_next;
1079     m_axi_awlen_reg <= m_axi_awlen_next;
1080     m_axi_awvalid_reg <= m_axi_awvalid_next;
1081     m_axi_bready_reg <= m_axi_bready_next;
1082
1083     s_axis_write_desc_ready_reg <= s_ax-
is_write_desc_ready_next;
1084
1085     m_axis_write_desc_status_tag_reg <= m_ax-
is_write_desc_status_tag_next;
1086     m_axis_write_desc_status_error_reg <= m_ax-
is_write_desc_status_error_next;
1087     m_axis_write_desc_status_valid_reg <= m_ax-
is_write_desc_status_valid_next;
1088
1089     status_busy_reg <= active_op_count_reg != 0;
1090
1091     stat_wr_op_start_tag_reg <= stat_wr_op_start_tag_next;
1092     stat_wr_op_start_len_reg <= stat_wr_op_start_len_next;
1093     stat_wr_op_start_valid_reg <= stat_wr_op_start_valid_next;
1094     stat_wr_op_finish_tag_reg <= stat_wr_op_finish_tag_next;
1095     stat_wr_op_finish_status_reg <= stat_wr_op_fin-
ish_status_next;
1096     stat_wr_op_finish_valid_reg <= stat_wr_op_fin-
ish_valid_next;
1097     stat_wr_req_start_tag_reg <= stat_wr_req_start_tag_next;
1098     stat_wr_req_start_len_reg <= stat_wr_req_start_len_next;
1099     stat_wr_req_start_valid_reg <=
stat_wr_req_start_valid_next;
1100     stat_wr_req_finish_tag_reg <= stat_wr_req_finish_tag_next;
1101     stat_wr_req_finish_status_reg <= stat_wr_req_fin-
ish_status_next;
1102     stat_wr_req_finish_valid_reg <= stat_wr_req_fin-
ish_valid_next;
1103     stat_wr_op_table_full_reg <= stat_wr_op_table_full_next;
1104     stat_wr_tx_stall_reg <= stat_wr_tx_stall_next;
1105
1106     ram_rd_cmd_sel_reg <= ram_rd_cmd_sel_next;
1107     ram_rd_cmd_addr_reg <= ram_rd_cmd_addr_next;
1108     ram_rd_cmd_valid_reg <= ram_rd_cmd_valid_next;
1109
1110     active_op_count_reg <= active_op_count_reg + inc_active_op
- dec_active_op;
1111
1112     if (mask_fifo_we) begin
1113         mask_fifo_mask[mask_fifo_wr_ptr_reg[MASK_FIFO_ADDR_WIDTH-1:0]] <=
mask_fifo_wr_mask;
1114         mask_fifo_wr_ptr_reg <= mask_fifo_wr_ptr_reg + 1;
1115     end
1116     mask_fifo_rd_ptr_reg <= mask_fifo_rd_ptr_next;
1117
1118     if (op_table_start_en) begin
1119         op_table_start_ptr_reg <= op_table_start_ptr_reg + 1;
1120         op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= 1'b1;
1121         op_t-
able_write_complete[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <=
1'b0;
1122         op_t-
able_axi_addr[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_axi_addr;
1123         op_table_imm[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]]
<= op_table_start_imm;
1124         op_t-
able_imm_en[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_imm_en;
1125         op_table_len[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]]
<= op_table_start_len;
1126         op_t-
able_zero_len[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_zero_len;
1127         op_t-
able_cycle_count[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_cycle_count;
1128         op_t-
able_offset[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_offset;
1129         op_table_tag[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]]
<= op_table_start_tag;
1130         op_t-
able_last[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_last;
1131     end
1132
1133     if (op_table_tx_start_en) begin
1134         op_table_tx_start_ptr_reg <= op_table_tx_start_ptr_reg
+ 1;
1135     end
1136
1137     if (op_table_tx_finish_en) begin
1138         op_table_tx_finish_ptr_reg <= op_t-
able_tx_finish_ptr_reg + 1;
1139     end

```



```

1140
1141 if (USE_AXI_ID && op_table_write_complete_en) begin
1142   op_table_write_complete[op_table_write_complete_ptr]
<= 1'b1;
1143   op_table_error_code[op_table_write_complete_ptr] <=
op_table_write_complete_error;
1144   end
1145
1146   if (op_table_finish_en) begin
1147     op_table_finish_ptr_reg <= op_table_finish_ptr_reg +
1;
1148     op_t-
able_active[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] <= 1'b0;
1149   end
1150
1151   if (rst) begin
1152     req_state_reg <= REQ_STATE_IDLE;
1153     read_state_reg <= READ_STATE_IDLE;
1154     axi_state_reg <= AXI_STATE_IDLE;
1155
1156     read_cmd_valid_reg <= 1'b0;
1157
1158     ram_mask_valid_reg <= 1'b0;
1159
1160     m_axi_awvalid_reg <= 1'b0;
1161     m_axi_bready_reg <= 1'b0;
1162
1163     s_axis_write_desc_ready_reg <= 1'b0;
1164     m_axis_write_desc_status_error_reg <= 4'd0;
1165     m_axis_write_desc_status_valid_reg <= 1'b0;
1166
1167     status_busy_reg <= 1'b0;
1168
1169     stat_wr_op_start_tag_reg <= 0;
1170     stat_wr_op_start_valid_reg <= 1'b0;
1171     stat_wr_op_finish_tag_reg <= 0;
1172     stat_wr_op_finish_valid_reg <= 1'b0;
1173     stat_wr_req_start_valid_reg <= 1'b0;
1174     stat_wr_req_finish_valid_reg <= 1'b0;
1175     stat_wr_op_table_full_reg <= 1'b0;
1176     stat_wr_tx_stall_reg <= 1'b0;
1177
1178     ram_rd_cmd_valid_reg <= {RAM_SEG_COUNT{1'b0}};
1179
1180     active_op_count_reg <= 0;
1181
1182     mask_fifo_wr_ptr_reg <= 0;
1183     mask_fifo_rd_ptr_reg <= 0;
1184
1185     op_table_start_ptr_reg <= 0;
1186     op_table_tx_start_ptr_reg <= 0;
1187     op_table_tx_finish_ptr_reg <= 0;
1188     op_table_finish_ptr_reg <= 0;
1189     op_table_active <= 0;
1190   end
1191 end
1192
1193 // output datapath logic
1194 reg [AXI_DATA_WIDTH-1:0] m_axi_wdata_reg =
{AXI_DATA_WIDTH{1'b0}};
1195 reg [AXI_STRB_WIDTH-1:0] m_axi_wstrb_reg =
{AXI_STRB_WIDTH{1'b0}};
1196 reg m_axi_wlast_reg = 1'b0;
1197 reg m_axi_wvalid_reg = 1'b0;
1198
1199 reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] out_fifo_wr_ptr_reg = 0;
1200 reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] out_fifo_rd_ptr_reg = 0;
1201 reg out_fifo_half_full_reg = 1'b0;
1202
1203 wire out_fifo_full = out_fifo_wr_ptr_reg ==
(out_fifo_rd_ptr_reg ^ {1'b1, {OUTPUT_FIFO_ADDR_WIDTH{1'b0}}});
1204 wire out_fifo_empty = out_fifo_wr_ptr_reg ==
out_fifo_rd_ptr_reg;
1205
1206 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
1207 reg [AXI_DATA_WIDTH-1:0]
out_fifo_wdata[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
1208 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
1209 reg [AXI_STRB_WIDTH-1:0]
out_fifo_wstrb[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
1210 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
1211 reg
out_fifo_wlast[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
1212
1213 assign m_axi_wready_int = !out_fifo_half_full_reg;
1214
1215 assign m_axi_wdata = m_axi_wdata_reg;
1216 assign m_axi_wstrb = m_axi_wstrb_reg;
1217 assign m_axi_wvalid = m_axi_wvalid_reg;
1218 assign m_axi_wlast = m_axi_wlast_reg;
1219
1220 always @(posedge clk) begin
1221   m_axi_wvalid_reg <= m_axi_wvalid_reg && !m_axi_wready;
1222
1223   out_fifo_half_full_reg <= $unsigned(out_fifo_wr_ptr_reg -
out_fifo_rd_ptr_reg) >= 2**(OUTPUT_FIFO_ADDR_WIDTH-1);
1224
1225   if (!out_fifo_full && m_axi_wvalid_int) begin

```

```

1226
out_fifo_wdata[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axi_wdata_int;
1227
out_fifo_wstrb[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axi_wstrb_int;
1228
out_fifo_wlast[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axi_wlast_int;
1229   out_fifo_wr_ptr_reg <= out_fifo_wr_ptr_reg + 1;
1230   end
1231
1232   if (!out_fifo_empty && (!m_axi_wvalid_reg ||
m_axi_wready)) begin
1233     m_axi_wdata_reg <=
out_fifo_wdata[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
1234     m_axi_wstrb_reg <=
out_fifo_wstrb[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
1235     m_axi_wlast_reg <=
out_fifo_wlast[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
1236     m_axi_wvalid_reg <= 1'b1;
1237     out_fifo_rd_ptr_reg <= out_fifo_rd_ptr_reg + 1;
1238   end
1239
1240   if (rst) begin
1241     out_fifo_wr_ptr_reg <= 0;
1242     out_fifo_rd_ptr_reg <= 0;
1243     m_axi_wvalid_reg <= 1'b0;
1244   end
1245 end
1246
1247 endmodule
1248
1249 `resetall

```

8. alexforenrich_verilog-pcie/rtl/dma_if_desc_mux.v (365 lines)

```

1 /*
2
3 Copyright (c) 2019-2021 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * DMA interface descriptor mux
33 */
34 module dma_if_desc_mux #
35 (
36   // Number of ports
37   parameter PORTS = 2,
38   // Extend RAM select signal
39   parameter EXTEND_RAM_SEL = 0,
40   // Input RAM segment select width
41   parameter S_RAM_SEL_WIDTH = 2,
42   // Output RAM segment select width
43   // Additional bits required for response routing
44   parameter M_RAM_SEL_WIDTH = S_RAM_SEL_WIDTH+(EX-
TEND_RAM_SEL ? $clog2(PORTS) : 0),
45   // RAM address width
46   parameter RAM_ADDR_WIDTH = 16,
47   // DMA address width
48   parameter DMA_ADDR_WIDTH = 64,
49   // Immediate enable

```

```

50 parameter IMM_ENABLE = 0,
51 // Immediate width
52 parameter IMM_WIDTH = 32,
53 // Length field width
54 parameter LEN_WIDTH = 16,
55 // Input tag field width
56 parameter S_TAG_WIDTH = 8,
57 // Output tag field width (towards DMA interface module)
58 // Additional bits required for response routing
59 parameter M_TAG_WIDTH = S_TAG_WIDTH+$clog2(PORTS),
60 // select round robin arbitration
61 parameter ARB_TYPE_ROUND_ROBIN = 0,
62 // LSB priority selection
63 parameter ARB_LSB_HIGH_PRIORITY = 1
64 )
65 (
66     input wire          clk,
67     input wire          rst,
68
69     /*
70      * Read descriptor output (to DMA interface)
71      */
72     output wire [DMA_ADDR_WIDTH-1:0] m_ax-
is_desc_dma_addr,
73     output wire [M_RAM_SEL_WIDTH-1:0] m_ax-
is_desc_ram_sel,
74     output wire [RAM_ADDR_WIDTH-1:0] m_ax-
is_desc_ram_addr,
75     output wire [IMM_WIDTH-1:0] m_ax-
is_desc_imm,
76     output wire          m_ax-
is_desc_imm_en,
77     output wire [LEN_WIDTH-1:0] m_ax-
is_desc_len,
78     output wire [M_TAG_WIDTH-1:0] m_ax-
is_desc_tag,
79     output wire          m_ax-
is_desc_valid,
80     input wire          m_ax-
is_desc_ready,
81
82     /*
83      * Read descriptor status input (from DMA interface)
84      */
85     input wire [M_TAG_WIDTH-1:0] s_ax-
is_desc_status_tag,
86     input wire [3:0] s_ax-
is_desc_status_error,
87     input wire s_ax-
is_desc_status_valid,
88
89     /*
90      * Read descriptor input
91      */
92     input wire [PORTS*DMA_ADDR_WIDTH-1:0] s_ax-
is_desc_dma_addr,
93     input wire [PORTS*S_RAM_SEL_WIDTH-1:0] s_ax-
is_desc_ram_sel,
94     input wire [PORTS*RAM_ADDR_WIDTH-1:0] s_ax-
is_desc_ram_addr,
95     input wire [PORTS*IMM_WIDTH-1:0] s_ax-
is_desc_imm,
96     input wire [PORTS:1:0] s_ax-
is_desc_imm_en,
97     input wire [PORTS*LEN_WIDTH-1:0] s_ax-
is_desc_len,
98     input wire [PORTS*S_TAG_WIDTH-1:0] s_ax-
is_desc_tag,
99     input wire [PORTS:1:0] s_ax-
is_desc_valid,
100    output wire [PORTS:1:0] s_ax-
is_desc_ready,
101
102    /*
103     * Read descriptor status output
104     */
105    output wire [PORTS*S_TAG_WIDTH-1:0] m_ax-
is_desc_status_tag,
106    output wire [PORTS*4-1:0] m_ax-
is_desc_status_error,
107    output wire [PORTS:1:0] m_ax-
is_desc_status_valid
108 );
109 parameter CL_PORTS = $clog2(PORTS);
110
111 parameter S_RAM_SEL_WIDTH_INT = S_RAM_SEL_WIDTH > 0 ?
S_RAM_SEL_WIDTH : 1;
112 parameter FIFO_ADDR_WIDTH = 5;
113
114 // check configuration
115 initial begin
116     if (M_TAG_WIDTH < S_TAG_WIDTH+$clog2(PORTS)) begin
117         $error("Error: M_TAG_WIDTH must be at least $clog2(PORTS) larger than S_TAG_WIDTH (instance %m)");
118     $finish;
119     end
120 end
121
122 if (EXTEND_RAM_SEL) begin
123     if (M_RAM_SEL_WIDTH < S_RAM_SEL_WIDTH+$clog2(PORTS))

```

```

begin
125     $error("Error: M_RAM_SEL_WIDTH must be at least
$clog2(PORTS) larger than S_RAM_SEL_WIDTH (instance %m)");
126     $finish;
127 end
128 end else begin
129     if (M_RAM_SEL_WIDTH < S_RAM_SEL_WIDTH) begin
130         $error("Error: M_RAM_SEL_WIDTH must be no smaller
than S_RAM_SEL_WIDTH (instance %m)");
131     $finish;
132     end
133 end
134 end
135
136 // descriptor mux
137 wire [PORTS-1:0] request;
138 wire [PORTS-1:0] acknowledge;
139 wire [PORTS-1:0] grant;
140 wire grant_valid;
141 wire [CL_PORTS-1:0] grant_encoded;
142
143 // input registers to pipeline arbitration delay
144 reg [PORTS*DMA_ADDR_WIDTH-1:0] s_axis_desc_dma_addr_reg = 0;
145 reg [PORTS*S_RAM_SEL_WIDTH-1:0] s_axis_desc_ram_sel_reg = 0;
146 reg [PORTS*RAM_ADDR_WIDTH-1:0] s_axis_desc_ram_addr_reg = 0;
147 reg [PORTS*IMM_WIDTH-1:0] s_axis_desc_imm_reg = 0;
148 reg [PORTS:1:0] s_axis_desc_imm_en_reg = 0;
149 reg [PORTS*LEN_WIDTH-1:0] s_axis_desc_len_reg = 0;
150 reg [PORTS*S_TAG_WIDTH-1:0] s_axis_desc_tag_reg = 0;
151 reg [PORTS:1:0] s_axis_desc_valid_reg = 0;
152
153 // internal datapath
154 reg [DMA_ADDR_WIDTH-1:0] m_axis_desc_dma_addr_int;
155 reg [M_RAM_SEL_WIDTH-1:0] m_axis_desc_ram_sel_int;
156 reg [RAM_ADDR_WIDTH-1:0] m_axis_desc_ram_addr_int;
157 reg [IMM_WIDTH-1:0] m_axis_desc_imm_int;
158 reg [LEN_WIDTH-1:0] m_axis_desc_imm_en_int;
159 reg [LEN_WIDTH-1:0] m_axis_desc_len_int;
160 reg [M_TAG_WIDTH-1:0] m_axis_desc_tag_int;
161 reg m_axis_desc_valid_int;
162 reg m_axis_desc_ready_int_reg = 1'b0;
163 wire m_axis_desc_ready_int_early;
164
165 assign s_axis_desc_ready = ~s_axis_desc_valid_reg |
({PORTS{m_axis_desc_ready_int_reg}} & grant);
166
167 // mux for incoming packet
168 wire [DMA_ADDR_WIDTH-1:0] current_s_desc_dma_addr = s_ax-
is_desc_dma_addr_reg[grant_encoded*DMA_ADDR_WIDTH +: DMA_AD-
DR_WIDTH];
169 wire [S_RAM_SEL_WIDTH-1:0] current_s_desc_ram_sel = s_ax-
is_desc_ram_sel_reg[grant_encoded*S_RAM_SEL_WIDTH +:
S_RAM_SEL_WIDTH_INT];
170 wire [RAM_ADDR_WIDTH-1:0] current_s_desc_ram_addr = s_ax-
is_desc_ram_addr_reg[grant_encoded*RAM_ADDR_WIDTH +: RAM_AD-
DR_WIDTH];
171 wire [IMM_WIDTH-1:0] current_s_desc_imm = s_ax-
is_desc_imm_reg[grant_encoded*IMM_WIDTH +: IMM_WIDTH];
172 wire [LEN_WIDTH-1:0] current_s_desc_imm_en = s_ax-
is_desc_imm_en_reg[grant_encoded];
173 wire [LEN_WIDTH-1:0] current_s_desc_len = s_ax-
is_desc_len_reg[grant_encoded*LEN_WIDTH +: LEN_WIDTH];
174 wire [S_TAG_WIDTH-1:0] current_s_desc_tag = s_ax-
is_desc_tag_reg[grant_encoded*S_TAG_WIDTH +: S_TAG_WIDTH];
175 wire current_s_desc_valid = s_ax-
is_desc_valid_reg[grant_encoded];
176 wire current_s_desc_ready = s_ax-
is_desc_ready[grant_encoded];
177
178 // arbiter instance
179 arbiter #(
180     .PORTS(PORTS),
181     .ARB_TYPE_ROUND_ROBIN(ARB_TYPE_ROUND_ROBIN),
182     .ARB_BLOCK(1),
183     .ARB_BLOCK_ACK(1),
184     .ARB_LSB_HIGH_PRIORITY(ARB_LSB_HIGH_PRIORITY)
185 )
186 arb_inst (
187     .clk(clk),
188     .rst(rst),
189     .request(request),
190     .acknowledge(acknowledge),
191     .grant(grant),
192     .grant_valid(grant_valid),
193     .grant_encoded(grant_encoded)
194 );
195
196 assign request = (s_axis_desc_valid_reg & ~grant) | (s_ax-
is_desc_valid & grant);
197 assign acknowledge = grant & s_axis_desc_valid_reg &
{PORTS{m_axis_desc_ready_int_reg}};
198
199 always @* begin
200     // pass through selected packet data
201     m_axis_desc_dma_addr_int = current_s_desc_dma_addr;
202     m_axis_desc_ram_sel_int = current_s_desc_ram_sel;
203     if (EXTEND_RAM_SEL && PORTS > 1) begin
204         m_ax-
is_desc_ram_sel_int[M_RAM_SEL_WIDTH-1:M_RAM_SEL_WIDTH-CL_PORTS] =
grant_encoded;
205     end

```

```

206 m_axis_desc_ram_addr_int = current_s_desc_ram_addr;
207 m_axis_desc_imm_int      = current_s_desc_imm;
208 m_axis_desc_imm_en_int   = current_s_desc_imm_en;
209 m_axis_desc_len_int      = current_s_desc_len;
210 m_axis_desc_tag_int      = current_s_desc_tag;
211 if (PORTS > 1) begin
212     m_axis_desc_tag_int[M_TAG_WIDTH-1:M_TAG_WIDTH-
CL_PORTS] = grant_encoded;
213 end
214 m_axis_desc_valid_int    = current_s_desc_valid && m_ax-
is_desc_ready_int_reg && grant_valid;
215 end
216
217 integer i;
218
219 always @(posedge clk) begin
220     // register inputs
221     for (i = 0; i < PORTS; i = i + 1) begin
222         if (s_axis_desc_ready[i]) begin
223             s_axis_desc_dma_addr_reg[i*DMA_ADDR_WIDTH +:
DMA_ADDR_WIDTH] <= s_axis_desc_dma_addr[i*DMA_ADDR_WIDTH +: DMA_AD-
DR_WIDTH];
224             s_axis_desc_ram_sel_reg[i*S_RAM_SEL_WIDTH +:
S_RAM_SEL_WIDTH_INT] <= s_axis_desc_ram_sel[i*S_RAM_SEL_WIDTH +:
S_RAM_SEL_WIDTH_INT];
225             s_axis_desc_ram_addr_reg[i*RAM_ADDR_WIDTH +:
RAM_ADDR_WIDTH] <= s_axis_desc_ram_addr[i*RAM_ADDR_WIDTH +: RAM_AD-
DR_WIDTH];
226             s_axis_desc_imm_reg[i*IMM_WIDTH +: IMM_WIDTH] <=
s_axis_desc_imm[i*IMM_WIDTH +: IMM_WIDTH];
227             s_axis_desc_imm_en_reg[i] <= s_ax-
is_desc_imm_en[i];
228             s_axis_desc_len_reg[i*LEN_WIDTH +: LEN_WIDTH] <=
s_axis_desc_len[i*LEN_WIDTH +: LEN_WIDTH];
229             s_axis_desc_tag_reg[i*S_TAG_WIDTH +: S_TAG_WIDTH]
<= s_axis_desc_tag[i*S_TAG_WIDTH +: S_TAG_WIDTH];
230             s_axis_desc_valid_reg[i] <= s_axis_desc_valid[i];
231         end
232     end
233
234     if (rst) begin
235         s_axis_desc_valid_reg <= 0;
236     end
237 end
238
239 // output datapath logic
240 reg [DMA_ADDR_WIDTH-1:0] m_axis_desc_dma_addr_reg = {DMA_AD-
DR_WIDTH{1'b0}};
241 reg [M_RAM_SEL_WIDTH-1:0] m_axis_desc_ram_sel_reg =
{M_RAM_SEL_WIDTH{1'b0}};
242 reg [RAM_ADDR_WIDTH-1:0] m_axis_desc_ram_addr_reg = {RAM_AD-
DR_WIDTH{1'b0}};
243 reg [IMM_WIDTH-1:0] m_axis_desc_imm_reg =
{IMM_WIDTH{1'b0}};
244 reg m_axis_desc_imm_en_reg = 1'b0;
245 reg [LEN_WIDTH-1:0] m_axis_desc_len_reg =
{LEN_WIDTH{1'b0}};
246 reg [M_TAG_WIDTH-1:0] m_axis_desc_tag_reg =
{M_TAG_WIDTH{1'b0}};
247 reg m_axis_desc_valid_reg = 1'b0,
m_axis_desc_valid_next;
248
249 reg [DMA_ADDR_WIDTH-1:0] temp_m_axis_desc_dma_addr_reg =
{DMA_ADDR_WIDTH{1'b0}};
250 reg [M_RAM_SEL_WIDTH-1:0] temp_m_axis_desc_ram_sel_reg =
{M_RAM_SEL_WIDTH{1'b0}};
251 reg [RAM_ADDR_WIDTH-1:0] temp_m_axis_desc_ram_addr_reg =
{RAM_ADDR_WIDTH{1'b0}};
252 reg [IMM_WIDTH-1:0] temp_m_axis_desc_imm_reg =
{IMM_WIDTH{1'b0}};
253 reg temp_m_axis_desc_imm_en_reg =
1'b0;
254 reg [LEN_WIDTH-1:0] temp_m_axis_desc_len_reg =
{LEN_WIDTH{1'b0}};
255 reg [M_TAG_WIDTH-1:0] temp_m_axis_desc_tag_reg =
{M_TAG_WIDTH{1'b0}};
256 reg temp_m_axis_desc_valid_reg =
1'b0, temp_m_axis_desc_valid_next;
257
258 // datapath control
259 reg store_axis_int_to_output;
260 reg store_axis_int_to_temp;
261 reg store_axis_temp_to_output;
262
263 assign m_axis_desc_dma_addr = m_axis_desc_dma_addr_reg;
264 assign m_axis_desc_ram_sel = m_axis_desc_ram_sel_reg;
265 assign m_axis_desc_ram_addr = m_axis_desc_ram_addr_reg;
266 assign m_axis_desc_imm = IMM_ENABLE ? m_ax-
is_desc_imm_reg : {IMM_WIDTH{1'b0}};
267 assign m_axis_desc_imm_en = IMM_ENABLE ? m_ax-
is_desc_imm_en_reg : 1'b0;
268 assign m_axis_desc_len = m_axis_desc_len_reg;
269 assign m_axis_desc_tag = m_axis_desc_tag_reg;
270 assign m_axis_desc_valid = m_axis_desc_valid_reg;
271
272 // enable ready input next cycle if output is ready or if both
output registers are empty
273 assign m_axis_desc_ready_int_early = m_axis_desc_ready || (!
temp_m_axis_desc_valid_reg && !m_axis_desc_valid_reg);
274

```

```

275 always @* begin
276     // transfer sink ready state to source
277     m_axis_desc_valid_next = m_axis_desc_valid_reg;
278     temp_m_axis_desc_valid_next = temp_m_axis_desc_valid_reg;
279
280     store_axis_int_to_output = 1'b0;
281     store_axis_int_to_temp = 1'b0;
282     store_axis_temp_to_output = 1'b0;
283
284     if (m_axis_desc_ready_int_reg) begin
285         // input is ready
286         if (m_axis_desc_ready || !m_axis_desc_valid_reg) begin
287             // output is ready or currently not valid, trans-
fer data to output
288             m_axis_desc_valid_next = m_axis_desc_valid_int;
289             store_axis_int_to_output = 1'b1;
290         end else begin
291             // output is not ready, store input in temp
292             temp_m_axis_desc_valid_next = m_ax-
is_desc_valid_int;
293             store_axis_int_to_temp = 1'b1;
294         end
295     end else if (m_axis_desc_ready) begin
296         // input is not ready, but output is ready
297         m_axis_desc_valid_next = temp_m_axis_desc_valid_reg;
298         temp_m_axis_desc_valid_next = 1'b0;
299         store_axis_temp_to_output = 1'b1;
300     end
301 end
302
303 always @(posedge clk) begin
304     m_axis_desc_valid_reg <= m_axis_desc_valid_next;
305     m_axis_desc_ready_int_reg <= m_axis_desc_ready_int_early;
306     temp_m_axis_desc_valid_reg <= temp_m_axis_desc_valid_next;
307
308     // datapath
309     if (store_axis_int_to_output) begin
310         m_axis_desc_dma_addr_reg <= m_axis_desc_dma_addr_int;
311         m_axis_desc_ram_sel_reg <= m_axis_desc_ram_sel_int;
312         m_axis_desc_ram_addr_reg <= m_axis_desc_ram_addr_int;
313         m_axis_desc_imm_reg <= m_axis_desc_imm_int;
314         m_axis_desc_imm_en_reg <= m_axis_desc_imm_en_int;
315         m_axis_desc_len_reg <= m_axis_desc_len_int;
316         m_axis_desc_tag_reg <= m_axis_desc_tag_int;
317     end else if (store_axis_temp_to_output) begin
318         m_axis_desc_dma_addr_reg <= temp_m_ax-
is_desc_dma_addr_reg;
319         m_axis_desc_ram_sel_reg <= temp_m_ax-
is_desc_ram_sel_reg;
320         m_axis_desc_ram_addr_reg <= temp_m_ax-
is_desc_ram_addr_reg;
321         m_axis_desc_imm_reg <= temp_m_axis_desc_imm_reg;
322         m_axis_desc_imm_en_reg <= temp_m_axis_desc_imm_en_reg;
323         m_axis_desc_len_reg <= temp_m_axis_desc_len_reg;
324         m_axis_desc_tag_reg <= temp_m_axis_desc_tag_reg;
325     end
326
327     if (store_axis_int_to_temp) begin
328         temp_m_axis_desc_dma_addr_reg <= m_ax-
is_desc_dma_addr_int;
329         temp_m_axis_desc_ram_sel_reg <= m_ax-
is_desc_ram_sel_int;
330         temp_m_axis_desc_ram_addr_reg <= m_ax-
is_desc_ram_addr_int;
331         temp_m_axis_desc_imm_reg <= m_axis_desc_imm_int;
332         temp_m_axis_desc_imm_en_reg <= m_axis_desc_imm_en_int;
333         temp_m_axis_desc_len_reg <= m_axis_desc_len_int;
334         temp_m_axis_desc_tag_reg <= m_axis_desc_tag_int;
335     end
336
337     if (rst) begin
338         m_axis_desc_valid_reg <= 1'b0;
339         m_axis_desc_ready_int_reg <= 1'b0;
340         temp_m_axis_desc_valid_reg <= 1'b0;
341     end
342 end
343
344 // descriptor status demux
345 reg [S_TAG_WIDTH-1:0] m_axis_desc_status_tag_reg =
{S_TAG_WIDTH{1'b0}};
346 reg [3:0] m_axis_desc_status_error_reg = 4'd0;
347 reg [PORTS-1:0] m_axis_desc_status_valid_reg = {PORTS{1'b0}};
348
349 assign m_axis_desc_status_tag =
{PORTS{m_axis_desc_status_tag_reg}};
350 assign m_axis_desc_status_error =
{PORTS{m_axis_desc_status_error_reg}};
351 assign m_axis_desc_status_valid = m_ax-
is_desc_status_valid_reg;
352
353 always @(posedge clk) begin
354     m_axis_desc_status_tag_reg <= s_axis_desc_status_tag;
355     m_axis_desc_status_error_reg <= s_axis_desc_status_error;
356     m_axis_desc_status_valid_reg <= s_axis_desc_status_valid
<< (PORTS > 1 ? s_axis_desc_status_tag[M_TAG_WIDTH-1:M_TAG_WIDTH-
CL_PORTS] : 0);
357
358     if (rst) begin
359         m_axis_desc_status_valid_reg <= {PORTS{1'b0}};
360     end
361 end

```

```

362
363 endmodule
364
365 `resetall

```

9. alexforenrich_verilog-pcie/rtl/dma_if_mux.v (354 lines)

```

1 /*
2
3 Copyright (c) 2019-2021 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
6 obtaining a copy
7 of this software and associated documentation files (the
8 "Software"), to deal
9 in the Software without restriction, including without limita-
10 tion the rights
11 to use, copy, modify, merge, publish, distribute, sublicense,
12 and/or sell
13 copies of the Software, and to permit persons to whom the
14 Software is
15 furnished to do so, subject to the following conditions:
16
17 The above copyright notice and this permission notice shall be
18 included in
19 all copies or substantial portions of the Software.
20
21 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
22 KIND, EXPRESS OR
23 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
24 CHANTABILITY
25 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
26 EVENT SHALL THE
27 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
28 OR OTHER
29 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
30 WISE, ARISING FROM,
31 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
32 DEALINGS IN
33 THE SOFTWARE.
34 */
35 // Language: Verilog 2001
36
37 `resetall
38 `timescale 1ns / 1ps
39 `default_nettype none
40
41 /*
42 * DMA interface mux
43 */
44 module dma_if_mux #
45 (
46     // Number of ports
47     parameter PORTS = 2,
48     // RAM segment count
49     parameter SEG_COUNT = 2,
50     // RAM segment data width
51     parameter SEG_DATA_WIDTH = 64,
52     // RAM segment address width
53     parameter SEG_ADDR_WIDTH = 8,
54     // RAM segment byte enable width
55     parameter SEG_BE_WIDTH = SEG_DATA_WIDTH/8,
56     // Input RAM segment select width
57     parameter S_RAM_SEL_WIDTH = 2,
58     // Output RAM segment select width
59     // Additional bits required for response routing
60     parameter M_RAM_SEL_WIDTH = S_RAM_SEL_WIDTH+$clog2(PORTS),
61     // RAM address width
62     parameter RAM_ADDR_WIDTH = SEG_ADDR_WIDTH+$clog2-
63     g2(SEG_COUNT)+$clog2(SEG_BE_WIDTH),
64     // DMA address width
65     parameter DMA_ADDR_WIDTH = 64,
66     // Immediate enable
67     parameter IMM_ENABLE = 0,
68     // Immediate width
69     parameter IMM_WIDTH = 32,
70     // Length field width
71     parameter LEN_WIDTH = 16,
72     // Input tag field width
73     parameter S_TAG_WIDTH = 8,
74     // Output tag field width (towards DMA module)
75     // Additional bits required for response routing
76     parameter M_TAG_WIDTH = S_TAG_WIDTH+$clog2(PORTS),
77     // select round robin arbitration
78     parameter ARB_TYPE_ROUND_ROBIN = 0,
79     // LSB priority selection
80     parameter ARB_LSB_HIGH_PRIORITY = 1
81 )
82 (
83     input wire clk,
84     input wire rst,
85
86     /*
87      * Read descriptor output (to DMA interface)
88      */
89     output wire [DMA_ADDR_WIDTH-1:0] m_ax-
90     is_read_desc_dma_addr,

```

```

79     output wire [M_RAM_SEL_WIDTH-1:0] m_ax-
is_read_desc_ram_sel,
80     output wire [RAM_ADDR_WIDTH-1:0] m_ax-
is_read_desc_ram_addr,
81     output wire [LEN_WIDTH-1:0] m_ax-
is_read_desc_len,
82     output wire [M_TAG_WIDTH-1:0] m_ax-
is_read_desc_tag,
83     output wire m_ax-
is_read_desc_valid,
84     input wire m_ax-
is_read_desc_ready,
85
86     /*
87      * Read descriptor status input (from DMA interface)
88      */
89     input wire [M_TAG_WIDTH-1:0] s_ax-
is_read_desc_status_tag,
90     input wire [3:0] s_ax-
is_read_desc_status_error,
91     input wire s_ax-
is_read_desc_status_valid,
92
93     /*
94      * Write descriptor output (to DMA interface)
95      */
96     output wire [DMA_ADDR_WIDTH-1:0] m_ax-
is_write_desc_dma_addr,
97     output wire [M_RAM_SEL_WIDTH-1:0] m_ax-
is_write_desc_ram_sel,
98     output wire [RAM_ADDR_WIDTH-1:0] m_ax-
is_write_desc_ram_addr,
99     output wire [IMM_WIDTH-1:0] m_ax-
is_write_desc_imm,
100     output wire m_ax-
is_write_desc_imm_en,
101     output wire [LEN_WIDTH-1:0] m_ax-
is_write_desc_len,
102     output wire [M_TAG_WIDTH-1:0] m_ax-
is_write_desc_tag,
103     output wire m_ax-
is_write_desc_valid,
104     input wire m_ax-
is_write_desc_ready,
105
106     /*
107      * Write descriptor status input (from DMA interface)
108      */
109     input wire [M_TAG_WIDTH-1:0] s_ax-
is_write_desc_status_tag,
110     input wire [3:0] s_ax-
is_write_desc_status_error,
111     input wire s_ax-
is_write_desc_status_valid,
112
113     /*
114      * Read descriptor input
115      */
116     input wire [PORTS*DMA_ADDR_WIDTH-1:0] s_ax-
is_read_desc_dma_addr,
117     input wire [PORTS*S_RAM_SEL_WIDTH-1:0] s_ax-
is_read_desc_ram_sel,
118     input wire [PORTS*RAM_ADDR_WIDTH-1:0] s_ax-
is_read_desc_ram_addr,
119     input wire [PORTS*LEN_WIDTH-1:0] s_ax-
is_read_desc_len,
120     input wire [PORTS*S_TAG_WIDTH-1:0] s_ax-
is_read_desc_tag,
121     input wire [PORTS-1:0] s_ax-
is_read_desc_valid,
122     output wire [PORTS-1:0] s_ax-
is_read_desc_ready,
123
124     /*
125      * Read descriptor status output
126      */
127     output wire [PORTS*S_TAG_WIDTH-1:0] m_ax-
is_read_desc_status_tag,
128     output wire [PORTS*4-1:0] m_ax-
is_read_desc_status_error,
129     output wire [PORTS-1:0] m_ax-
is_read_desc_status_valid,
130
131     /*
132      * Write descriptor input
133      */
134     input wire [PORTS*DMA_ADDR_WIDTH-1:0] s_ax-
is_write_desc_dma_addr,
135     input wire [PORTS*S_RAM_SEL_WIDTH-1:0] s_ax-
is_write_desc_ram_sel,
136     input wire [PORTS*RAM_ADDR_WIDTH-1:0] s_ax-
is_write_desc_ram_addr,
137     input wire [PORTS*IMM_WIDTH-1:0] s_ax-
is_write_desc_imm,
138     input wire [PORTS-1:0] s_ax-
is_write_desc_imm_en,
139     input wire [PORTS*LEN_WIDTH-1:0] s_ax-
is_write_desc_len,
140     input wire [PORTS*S_TAG_WIDTH-1:0] s_ax-
is_write_desc_tag,
141     input wire [PORTS-1:0] s_ax-

```

```

is_write_desc_valid,
142     output wire [PORTS:1:0]
is_write_desc_ready,
143
144     /*
145     * Write descriptor status output
146     */
147     output wire [PORTS*S_TAG_WIDTH-1:0]
is_write_desc_status_tag,
148     output wire [PORTS*4-1:0]
is_write_desc_status_error,
149     output wire [PORTS:1:0]
is_write_desc_status_valid,
150
151     /*
152     * RAM interface (from DMA interface)
153     */
154     input wire [SEG_COUNT*M_RAM_SEL_WIDTH-1:0]
if_ram_wr_cmd_sel,
155     input wire [SEG_COUNT*SEG_BE_WIDTH-1:0]
if_ram_wr_cmd_be,
156     input wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0]
if_ram_wr_cmd_addr,
157     input wire [SEG_COUNT*SEG_DATA_WIDTH-1:0]
if_ram_wr_cmd_data,
158     input wire [SEG_COUNT-1:0]
if_ram_wr_cmd_valid,
159     output wire [SEG_COUNT-1:0]
if_ram_wr_cmd_ready,
160     output wire [SEG_COUNT-1:0]
if_ram_wr_done,
161     input wire [SEG_COUNT*M_RAM_SEL_WIDTH-1:0]
if_ram_rd_cmd_sel,
162     input wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0]
if_ram_rd_cmd_addr,
163     input wire [SEG_COUNT-1:0]
if_ram_rd_cmd_valid,
164     output wire [SEG_COUNT-1:0]
if_ram_rd_cmd_ready,
165     output wire [SEG_COUNT*SEG_DATA_WIDTH-1:0]
if_ram_rd_resp_data,
166     output wire [SEG_COUNT-1:0]
if_ram_rd_resp_valid,
167     input wire [SEG_COUNT-1:0]
if_ram_rd_resp_ready,
168
169     /*
170     * RAM interface (towards RAM)
171     */
172     output wire [PORTS*SEG_COUNT*S_RAM_SEL_WIDTH-1:0]
ram_wr_cmd_sel,
173     output wire [PORTS*SEG_COUNT*SEG_BE_WIDTH-1:0]
ram_wr_cmd_be,
174     output wire [PORTS*SEG_COUNT*SEG_ADDR_WIDTH-1:0]
ram_wr_cmd_addr,
175     output wire [PORTS*SEG_COUNT*SEG_DATA_WIDTH-1:0]
ram_wr_cmd_data,
176     output wire [PORTS*SEG_COUNT-1:0]
ram_wr_cmd_valid,
177     input wire [PORTS*SEG_COUNT-1:0]
ram_wr_cmd_ready,
178     input wire [PORTS*SEG_COUNT-1:0]
ram_wr_done,
179     output wire [PORTS*SEG_COUNT*S_RAM_SEL_WIDTH-1:0]
ram_rd_cmd_sel,
180     output wire [PORTS*SEG_COUNT*SEG_ADDR_WIDTH-1:0]
ram_rd_cmd_addr,
181     output wire [PORTS*SEG_COUNT-1:0]
ram_rd_cmd_valid,
182     input wire [PORTS*SEG_COUNT-1:0]
ram_rd_cmd_ready,
183     input wire [PORTS*SEG_COUNT*SEG_DATA_WIDTH-1:0]
ram_rd_resp_data,
184     input wire [PORTS*SEG_COUNT-1:0]
ram_rd_resp_valid,
185     output wire [PORTS*SEG_COUNT-1:0]
ram_rd_resp_ready
186 );
187
188 dma_if_mux_rd #(
189     .PORTS(PORTS),
190     .SEG_COUNT(SEG_COUNT),
191     .SEG_DATA_WIDTH(SEG_DATA_WIDTH),
192     .SEG_ADDR_WIDTH(SEG_ADDR_WIDTH),
193     .SEG_BE_WIDTH(SEG_BE_WIDTH),
194     .S_RAM_SEL_WIDTH(S_RAM_SEL_WIDTH),
195     .M_RAM_SEL_WIDTH(M_RAM_SEL_WIDTH),
196     .RAM_ADDR_WIDTH(RAM_ADDR_WIDTH),
197     .DMA_ADDR_WIDTH(DMA_ADDR_WIDTH),
198     .LEN_WIDTH(LEN_WIDTH),
199     .S_TAG_WIDTH(S_TAG_WIDTH),
200     .M_TAG_WIDTH(M_TAG_WIDTH),
201     .ARB_TYPE_ROUND_ROBIN(ARB_TYPE_ROUND_ROBIN),
202     .ARB_LSB_HIGH_PRIORITY(ARB_LSB_HIGH_PRIORITY)
203 )
204 dma_if_mux_rd_inst (
205     .clk(clk),
206     .rst(rst),
207
208     /*
209     * Descriptor output (to DMA interface)
210
211     */
212     .m_axis_read_desc_dma_addr(m_axis_read_desc_dma_addr),
213     .m_axis_read_desc_ram_sel(m_axis_read_desc_ram_sel),
214     .m_axis_read_desc_ram_addr(m_axis_read_desc_ram_addr),
215     .m_axis_read_desc_len(m_axis_read_desc_len),
216     .m_axis_read_desc_tag(m_axis_read_desc_tag),
217     .m_axis_read_desc_valid(m_axis_read_desc_valid),
218     .m_axis_read_desc_ready(m_axis_read_desc_ready),
219
220     /*
221     * Descriptor status input (from DMA interface)
222     */
223     .s_axis_read_desc_status_tag(s_axis_read_desc_status_tag),
224     .s_axis_read_desc_status_error(s_axis_read_desc_status_error),
225     .s_axis_read_desc_status_valid(s_axis_read_desc_status_valid),
226
227     /*
228     * Descriptor input
229     */
230     .s_axis_read_desc_dma_addr(s_axis_read_desc_dma_addr),
231     .s_axis_read_desc_ram_sel(s_axis_read_desc_ram_sel),
232     .s_axis_read_desc_ram_addr(s_axis_read_desc_ram_addr),
233     .s_axis_read_desc_len(s_axis_read_desc_len),
234     .s_axis_read_desc_tag(s_axis_read_desc_tag),
235     .s_axis_read_desc_valid(s_axis_read_desc_valid),
236     .s_axis_read_desc_ready(s_axis_read_desc_ready),
237
238     /*
239     * Descriptor status output
240     */
241     .m_axis_read_desc_status_tag(m_axis_read_desc_status_tag),
242     .m_axis_read_desc_status_error(m_axis_read_desc_status_error),
243     .m_axis_read_desc_status_valid(m_axis_read_desc_status_valid),
244
245     /*
246     * RAM interface (from DMA interface)
247     */
248     .if_ram_wr_cmd_sel(if_ram_wr_cmd_sel),
249     .if_ram_wr_cmd_be(if_ram_wr_cmd_be),
250     .if_ram_wr_cmd_addr(if_ram_wr_cmd_addr),
251     .if_ram_wr_cmd_data(if_ram_wr_cmd_data),
252     .if_ram_wr_cmd_valid(if_ram_wr_cmd_valid),
253     .if_ram_wr_cmd_ready(if_ram_wr_cmd_ready),
254     .if_ram_wr_done(if_ram_wr_done),
255
256     /*
257     * RAM interface (towards RAM)
258     */
259     .ram_wr_cmd_sel(ram_wr_cmd_sel),
260     .ram_wr_cmd_be(ram_wr_cmd_be),
261     .ram_wr_cmd_addr(ram_wr_cmd_addr),
262     .ram_wr_cmd_data(ram_wr_cmd_data),
263     .ram_wr_cmd_valid(ram_wr_cmd_valid),
264     .ram_wr_cmd_ready(ram_wr_cmd_ready),
265     .ram_wr_done(ram_wr_done)
266 );
267
268 dma_if_mux_wr #(
269     .PORTS(PORTS),
270     .SEG_COUNT(SEG_COUNT),
271     .SEG_DATA_WIDTH(SEG_DATA_WIDTH),
272     .SEG_ADDR_WIDTH(SEG_ADDR_WIDTH),
273     .SEG_BE_WIDTH(SEG_BE_WIDTH),
274     .S_RAM_SEL_WIDTH(S_RAM_SEL_WIDTH),
275     .M_RAM_SEL_WIDTH(M_RAM_SEL_WIDTH),
276     .RAM_ADDR_WIDTH(RAM_ADDR_WIDTH),
277     .DMA_ADDR_WIDTH(DMA_ADDR_WIDTH),
278     .IMM_ENABLE(IMM_ENABLE),
279     .IMM_WIDTH(IMM_WIDTH),
280     .LEN_WIDTH(LEN_WIDTH),
281     .S_TAG_WIDTH(S_TAG_WIDTH),
282     .M_TAG_WIDTH(M_TAG_WIDTH),
283     .ARB_TYPE_ROUND_ROBIN(ARB_TYPE_ROUND_ROBIN),
284     .ARB_LSB_HIGH_PRIORITY(ARB_LSB_HIGH_PRIORITY)
285 )
286 dma_if_mux_wr_inst (
287     .clk(clk),
288     .rst(rst),
289
290     /*
291     * Descriptor output (to DMA interface)
292     */
293     .m_axis_write_desc_dma_addr(m_axis_write_desc_dma_addr),
294     .m_axis_write_desc_ram_sel(m_axis_write_desc_ram_sel),
295     .m_axis_write_desc_ram_addr(m_axis_write_desc_ram_addr),
296     .m_axis_write_desc_imm(m_axis_write_desc_imm),
297     .m_axis_write_desc_imm_en(m_axis_write_desc_imm_en),
298     .m_axis_write_desc_len(m_axis_write_desc_len),
299     .m_axis_write_desc_tag(m_axis_write_desc_tag),
300     .m_axis_write_desc_valid(m_axis_write_desc_valid),
301     .m_axis_write_desc_ready(m_axis_write_desc_ready),
302
303     /*
304     * Descriptor status input (from DMA interface)
305     */
306     .s_axis_write_desc_status_tag(s_axis_write_desc_status_tag),

```



```

306      .s_ax-
is_write_desc_status_error(s_axis_write_desc_status_error),
307      .s_ax-
is_write_desc_status_valid(s_axis_write_desc_status_valid),
308
309      /*
310      * Descriptor input
311      */
312      .s_axis_write_desc_dma_addr(s_axis_write_desc_dma_addr),
313      .s_axis_write_desc_ram_sel(s_axis_write_desc_ram_sel),
314      .s_axis_write_desc_ram_addr(s_axis_write_desc_ram_addr),
315      .s_axis_write_desc_imm(s_axis_write_desc_imm),
316      .s_axis_write_desc_imm_en(s_axis_write_desc_imm_en),
317      .s_axis_write_desc_len(s_axis_write_desc_len),
318      .s_axis_write_desc_tag(s_axis_write_desc_tag),
319      .s_axis_write_desc_valid(s_axis_write_desc_valid),
320      .s_axis_write_desc_ready(s_axis_write_desc_ready),
321
322      /*
323      * Descriptor status output
324      */
325      .m_ax-
is_write_desc_status_tag(m_axis_write_desc_status_tag),
326      .m_ax-
is_write_desc_status_error(m_axis_write_desc_status_error),
327      .m_ax-
is_write_desc_status_valid(m_axis_write_desc_status_valid),

```

```

328
329      /*
330      * RAM interface (from DMA interface)
331      */
332      .if_ram_rd_cmd_sel(if_ram_rd_cmd_sel),
333      .if_ram_rd_cmd_addr(if_ram_rd_cmd_addr),
334      .if_ram_rd_cmd_valid(if_ram_rd_cmd_valid),
335      .if_ram_rd_cmd_ready(if_ram_rd_cmd_ready),
336      .if_ram_rd_resp_data(if_ram_rd_resp_data),
337      .if_ram_rd_resp_valid(if_ram_rd_resp_valid),
338      .if_ram_rd_resp_ready(if_ram_rd_resp_ready),
339
340      /*
341      * RAM interface (towards RAM)
342      */
343      .ram_rd_cmd_sel(ram_rd_cmd_sel),
344      .ram_rd_cmd_addr(ram_rd_cmd_addr),
345      .ram_rd_cmd_valid(ram_rd_cmd_valid),
346      .ram_rd_cmd_ready(ram_rd_cmd_ready),
347      .ram_rd_resp_data(ram_rd_resp_data),
348      .ram_rd_resp_valid(ram_rd_resp_valid),
349      .ram_rd_resp_ready(ram_rd_resp_ready)
350 );
351
352 endmodule
353
354 `resetall

```

AES 골든 모델 (C)

What this is. A 573-line C implementation of FIPS-197 AES: the S-box table, `xtime`, the four round functions, key expansion, and ECB/CBC/CTR modes.

Why it exists. To define what is correct. Judging an RTL implementation requires an expected value computed independently. This file produces it. The complete absence of hardware concepts — no clock, no ports, no state machine — is deliberate: only the *meaning* remains.

Undergraduate review · EECS concepts. Finite-field $GF(2^8)$ arithmetic; substitution-permutation network structure; Shannon's confusion and diffusion.

Graduate · Practice note. Note that `xtime` multiplies by a mask rather than branching. This keeps execution time independent of the data — the most basic timing-side-channel discipline — and it is worth preserving even in a model.

3 files • 979 lines

API — `aes.c`

Table 688. `kokke_tiny-AES-c/aes.c` — 함수 (23개)

함수	반환형	받는 파라미터
<code>getSBoxValue</code>	<code>static uint8_t</code>	<code>uint8_t num</code>
<code>KeyExpansion</code>	<code>static void</code>	<code>uint8_t* RoundKey</code> , <code>const uint8_t* Key</code>
<code>AES_init_ctx</code>	<code>void</code>	<code>struct AES_ctx* ctx</code> , <code>const uint8_t* key</code>
<code>AES_init_ctx_iv</code>	<code>void</code>	<code>struct AES_ctx* ctx</code> , <code>const uint8_t* key</code> , <code>const uint8_t* iv</code>
<code>AES_ctx_set_iv</code>	<code>void</code>	<code>struct AES_ctx* ctx</code> , <code>const uint8_t* iv</code>
<code>AddRoundKey</code>	<code>static void</code>	<code>uint8_t round</code> , <code>state_t* state</code> , <code>const uint8_t* RoundKey</code>
<code>SubBytes</code>	<code>static void</code>	<code>state_t* state</code>
<code>ShiftRows</code>	<code>static void</code>	<code>state_t* state</code>
<code>xtime</code>	<code>static uint8_t</code>	<code>uint8_t x</code>
<code>MixColumns</code>	<code>static void</code>	<code>state_t* state</code>
<code>Multiply</code>	<code>static uint8_t</code>	<code>uint8_t x</code> , <code>uint8_t y</code>
<code>getSBoxInvert</code>	<code>static uint8_t</code>	<code>uint8_t num</code>
<code>InvMixColumns</code>	<code>static void</code>	<code>state_t* state</code>
<code>InvSubBytes</code>	<code>static void</code>	<code>state_t* state</code>
<code>InvShiftRows</code>	<code>static void</code>	<code>state_t* state</code>
<code>Cipher</code>	<code>static void</code>	<code>state_t* state</code> , <code>const uint8_t* RoundKey</code>
<code>InvCipher</code>	<code>static void</code>	<code>state_t* state</code> , <code>const uint8_t* RoundKey</code>
<code>AES_ECB_encrypt</code>	<code>void</code>	<code>const struct AES_ctx* ctx</code> , <code>uint8_t* buf</code>
<code>AES_ECB_decrypt</code>	<code>void</code>	<code>const struct AES_ctx* ctx</code> , <code>uint8_t* buf</code>

함수	반환형	받는 파라미터
XorWithIv	static void	uint8_t* buf, const uint8_t* lv
AES_CBC_encrypt_buffer	void	struct AES_ctx* ctx, uint8_t* buf, size_t length
AES_CBC_decrypt_buffer	void	struct AES_ctx* ctx, uint8_t* buf, size_t length
AES_CTR_xcrypt_buffer	void	struct AES_ctx* ctx, uint8_t* buf, size_t length

Table 689. kokke_tiny-AES-c/aes.c — #define / `define 매크로 (11개)

매크로	인자	정의
Nb		4
Nk		8
Nr		14
Nk		6
Nr		12
Nk		4 // The number of 32 bit words in a key.
Nr		10 // The number of rounds in AES Cipher.
MULTIPLY_AS_A_FUNCTION		0
getSBoxValue	(num)	(sbox[(num)])
Multiply	(x, y)	\
getSBoxInvert	(num)	(rsbox[(num)])

API — aes.h**Table 690. kokke_tiny-AES-c/aes.h — 클래스/구조체 (1개)**

이름	템플릿 인자	종류	상속
AES_ctx		struct	

Table 691. kokke_tiny-AES-c/aes.h — #define / `define 매크로 (12개)

매크로	인자	정의
_AES_H_		#include <stdint.h>
CBC		1
ECB		1
CTR		1
AES128		1
AES_BLOCKLEN		16 // Block length in bytes - AES is 128b block only
AES_KEYLEN		32
AES_keyExpSize		240
AES_KEYLEN		24
AES_keyExpSize		208

매크로	인자	정의
AES_KEYLEN		16 // Key length in bytes
AES_keyExpSize		176

API — test.c

Table 692. kokke_tiny-AES-c/test.c — 함수 (10개)

함수	반환형	받는 파라미터
main	int	void
phex	static void	uint8_t* str
test_encrypt_ecb_verbose	static void	void
test_encrypt_ecb	static int	void
test_decrypt_cbc	static int	void
test_encrypt_cbc	static int	void
test_encrypt_ctr	static int	void
test_decrypt_ctr	static int	void
test_xcrypt_ctr	static int	const char* xcrypt
test_decrypt_ecb	static int	void

Table 693. kokke_tiny-AES-c/test.c — #define / `define 매크로 (3개)

매크로	인자	정의
CBC		1
CTR		1
ECB		1

1. kokke_tiny-AES-c/aes.c (572 lines)

```

1 /*
2
3 This is an implementation of the AES algorithm, specifically
4 ECB, CTR and CBC mode.
5 Block size can be chosen in aes.h - available choices are
6 AES128, AES192, AES256.
7
8 The implementation is verified against the test vectors in:
9 National Institute of Standards and Technology Special Pub-
10 lication 800-38A 2001 ED
11
12 ECB-AES128
13 -----
14
15 plain-text:
16 6bc1bee22e409f96e93d7e117393172a
17 ae2d8a571e03ac9c9eb76fac45af8e51
18 30c81c46a35ce411e5fbc1191a0a52ef
19 f69f2445df4f9b17ad2b417be66c3710
20
21 key:
22 2b7e151628aed2a6abf7158809cf4f3c
23
24 resulting cipher
25 3ad77bb40d7a3660a89ecaf32466ef97
26 f5d3d58503b9699de785895a96fdbaaf
27 43b1cd7f598ece23881b00e3ed030688
28 7b0c785e27e8ad3f8223207104725dd4
29
30 NOTE: String length must be evenly divisible by 16byte
31 (str_len % 16 == 0)
32 You should pad the end of the string with zeros if
33 this is not the case.
```

```

30 For AES192/256 the key size is proportionally larger.
31
32 */
33
34
35 /
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2
```

```

key.
55 #define Nr 10 // The number of rounds in AES Cipher.
56 #endif
57
58 // jcallan@github points out that declaring Multiply as a
function
59 // reduces code size considerably with the Keil ARM compiler.
60 // See this link for more information: https://github.com/
kokke/tiny-AES-C/pull/3
61 #ifndef MULTIPLY_AS_A_FUNCTION
62 #define MULTIPLY_AS_A_FUNCTION 0
63 #endif
64
65
66
67
68 /
*****
69 /* Private vari-
ables: */
70 /
*****
71 // state - array holding the intermediate results during de-
cryption.
72 typedef uint8_t state_t[4][4];
73
74
75
76 // The lookup-tables are marked const so they can be placed in
read-only storage instead of RAM
77 // The numbers below can be computed dynamically trading ROM
for RAM -
78 // This can be useful in (embedded) bootloader applications,
where ROM is often limited.
79 static const uint8_t sbox[256] = {

80 //0 1 2 3 4 5 6 7 8 9
A B C D E F
81 0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01,
0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
82 0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4,
0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
83 0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5,
0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
84 0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12,
0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
85 0x09, 0x83, 0x2c, 0x1a, 0x1b, 0xe6, 0x5a, 0xa0, 0x52, 0x3b,
0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
86 0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb,
0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
87 0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9,
0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
88 0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6,
0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,
89 0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7,
0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
90 0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee,
0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,
91 0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3,
0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,
92 0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56,
0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,
93 0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd,
0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
94 0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35,
0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,
95 0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e,
0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,
96 0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99,
0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16 };

97
98 #if (defined(CBC) && CBC == 1) || (defined(ECB) && ECB == 1)
99 static const uint8_t rsbox[256] = {
100 0x52, 0x09, 0x6a, 0xd5, 0x30, 0x36, 0xa5, 0x38, 0xbf, 0x40,
0xa3, 0x9e, 0x81, 0xf3, 0xd7, 0xfb,
101 0x7c, 0xe3, 0x39, 0x82, 0x9b, 0x2f, 0xff, 0x87, 0x34, 0x8e,
0x43, 0x44, 0xc4, 0xde, 0xe9, 0xcb,
102 0x54, 0x7b, 0x94, 0x32, 0xa6, 0xc2, 0x23, 0x3d, 0xee, 0x4c,
0x95, 0x0b, 0x42, 0xfa, 0xc3, 0x4e,
103 0x08, 0x2e, 0xa1, 0x66, 0x28, 0xd9, 0x24, 0xb2, 0x76, 0x5b,
0xa2, 0x49, 0x6d, 0x8b, 0xd1, 0x25,
104 0x72, 0xf8, 0xf6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xd4, 0xa4,
0x5c, 0xcc, 0x5d, 0x65, 0xb6, 0x92,
105 0x6c, 0x70, 0x48, 0x50, 0xfd, 0xed, 0xb9, 0xda, 0x5e, 0x15,
0x46, 0x57, 0xa7, 0x8d, 0x9d, 0x84,
106 0x90, 0xd8, 0xab, 0x00, 0x8c, 0xbc, 0xd3, 0x0a, 0xf7, 0xe4,
0x58, 0x05, 0xb8, 0xb3, 0x45, 0x06,
107 0xd0, 0x2c, 0x1e, 0x8f, 0xca, 0x3f, 0x0f, 0x02, 0xc1, 0xaf,
0xbd, 0x03, 0x01, 0x13, 0x8a, 0x6b,
108 0x3a, 0x91, 0x11, 0x41, 0x4f, 0x67, 0xdc, 0xea, 0x97, 0xf2,
0xcf, 0xce, 0xf0, 0xb4, 0xe6, 0x73,
109 0x96, 0xac, 0x74, 0x22, 0xe7, 0xad, 0x35, 0x85, 0xe2, 0xf9,
0x37, 0xe8, 0x1c, 0x75, 0xdf, 0x6e,
110 0x47, 0xf1, 0x1a, 0x71, 0x1d, 0x29, 0xc5, 0x89, 0x6f, 0xb7,
0x62, 0x0e, 0xaa, 0x18, 0xbe, 0x1b,
111 0xfc, 0x56, 0x3e, 0x4b, 0xc6, 0xd2, 0x79, 0x20, 0x9a, 0xdb,
0xc0, 0xfe, 0x78, 0xcd, 0x5a, 0xf4,
112 0x1f, 0xdd, 0xa8, 0x33, 0x88, 0x07, 0xc7, 0x31, 0xb1, 0x12,

```

```

0x10, 0x59, 0x27, 0x80, 0xec, 0x5f,
113 0x60, 0x51, 0x7f, 0xa9, 0x19, 0xb5, 0x4a, 0x0d, 0x2d, 0xe5,
0x7a, 0x9f, 0x93, 0xc9, 0x9c, 0xef,
114 0xa0, 0xe0, 0x3b, 0x4d, 0xae, 0x2a, 0xf5, 0xb0, 0xc8, 0xeb,
0xbb, 0x3c, 0x83, 0x53, 0x99, 0x61,
115 0x17, 0x2b, 0x04, 0x7e, 0xba, 0x77, 0xd6, 0x26, 0xe1, 0x69,
0x14, 0x63, 0x55, 0x21, 0x0c, 0x7d };
116 #endif
117

118 // The round constant word array, Rcon[i], contains the values
given by
119 // x to the power (i-1) being powers of x (x is denoted as
{02}) in the field GF(2^8)
120 static const uint8_t Rcon[11] = {
121 0x8d, 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1b,
0x36 };
122
123 /*
124 * Jordan Goulder points out in PR #12 (https://github.com/
kokke/tiny-AES-C/pull/12),
125 * that you can remove most of the elements in the Rcon array,
because they are unused.
126 *
127 * From Wikipedia's article on the Rijndael key schedule @ ht-
tps://en.wikipedia.org/wiki/Rijndael_key_schedule#Rcon
128 *
129 * "Only the first some of these constants are actually used -
up to rcon[10] for AES-128 (as 11 round keys are needed),
130 * up to rcon[8] for AES-192, up to rcon[7] for AES-256.
rcon[0] is not used in AES algorithm."
131 */
132
133
134 /
*****
135 /* Private func-
tions: */
136 /
*****
137 /*
138 static uint8_t getSBoxValue(uint8_t num)
139 {
140 return sbox[num];
141 }
142 */
143 #define getSBoxValue(num) (sbox[(num)])
144
145 // This function produces Nb(Nr+1) round keys. The round keys
are used in each round to decrypt the states.
146 static void KeyExpansion(uint8_t* RoundKey, const uint8_t*
Key)
147 {
148 unsigned i, j, k;
149 uint8_t tempa[4]; // Used for the column/row operations
150
151 // The first round key is the key itself.
152 for (i = 0; i < Nk; ++i)
153 {
154 RoundKey[(i * 4) + 0] = Key[(i * 4) + 0];
155 RoundKey[(i * 4) + 1] = Key[(i * 4) + 1];
156 RoundKey[(i * 4) + 2] = Key[(i * 4) + 2];
157 RoundKey[(i * 4) + 3] = Key[(i * 4) + 3];
158 }
159
160 // All other round keys are found from the previous round
keys.
161 for (i = Nk; i < Nb * (Nr + 1); ++i)
162 {
163 {
164 k = (i - 1) * 4;
165 tempa[0] = RoundKey[k + 0];
166 tempa[1] = RoundKey[k + 1];
167 tempa[2] = RoundKey[k + 2];
168 tempa[3] = RoundKey[k + 3];
169
170 }
171
172 if (i % Nk == 0)
173 {
174 // This function shifts the 4 bytes in a word to the
left once.
175 // [a0,a1,a2,a3] becomes [a1,a2,a3,a0]
176
177 // Function RotWord()
178 {
179 const uint8_t u8tmp = tempa[0];
180 tempa[0] = tempa[1];
181 tempa[1] = tempa[2];
182 tempa[2] = tempa[3];
183 tempa[3] = u8tmp;
184 }
185
186 // SubWord() is a function that takes a four-byte input
word and
187 // applies the S-box to each of the four bytes to pro-
duce an output word.

```

```

188
189 // Function Subword()
190 {
191     tempa[0] = getSBoxValue(tempa[0]);
192     tempa[1] = getSBoxValue(tempa[1]);
193     tempa[2] = getSBoxValue(tempa[2]);
194     tempa[3] = getSBoxValue(tempa[3]);
195 }
196
197     tempa[0] = tempa[0] ^ Rcon[i/Nk];
198 }
199 #if defined(AES256) && (AES256 == 1)
200 if (i % Nk == 4)
201 {
202     // Function Subword()
203     {
204         tempa[0] = getSBoxValue(tempa[0]);
205         tempa[1] = getSBoxValue(tempa[1]);
206         tempa[2] = getSBoxValue(tempa[2]);
207         tempa[3] = getSBoxValue(tempa[3]);
208     }
209 }
210 #endif
211 j = i * 4; k=(i - Nk) * 4;
212 RoundKey[j + 0] = RoundKey[k + 0] ^ tempa[0];
213 RoundKey[j + 1] = RoundKey[k + 1] ^ tempa[1];
214 RoundKey[j + 2] = RoundKey[k + 2] ^ tempa[2];
215 RoundKey[j + 3] = RoundKey[k + 3] ^ tempa[3];
216 }
217 }
218
219 void AES_init_ctx(struct AES_ctx* ctx, const uint8_t* key)
220 {
221     KeyExpansion(ctx->RoundKey, key);
222 }
223 #if (defined(CBC) && (CBC == 1)) || (defined(CTR) && (CTR ==
1))
224 void AES_init_ctx_iv(struct AES_ctx* ctx, const uint8_t* key,
const uint8_t* iv)
225 {
226     KeyExpansion(ctx->RoundKey, key);
227     memcpy(ctx->IV, iv, AES_BLOCKLEN);
228 }
229 void AES_ctx_set_iv(struct AES_ctx* ctx, const uint8_t* iv)
230 {
231     memcpy(ctx->IV, iv, AES_BLOCKLEN);
232 }
233 #endif
234
235 // This function adds the round key to state.
236 // The round key is added to the state by an XOR function.
237 static void AddRoundKey(uint8_t round, state_t* state, const
uint8_t* RoundKey)
238 {
239     uint8_t i, j;
240     for (i = 0; i < 4; ++i)
241     {
242         for (j = 0; j < 4; ++j)
243         {
244             (*state)[i][j] ^= RoundKey[(round * Nb * 4) + (i * Nb) +
j];
245         }
246     }
247 }
248
249 // The SubBytes Function Substitutes the values in the
250 // state matrix with values in an S-box.
251 static void SubBytes(state_t* state)
252 {
253     uint8_t i, j;
254     for (i = 0; i < 4; ++i)
255     {
256         for (j = 0; j < 4; ++j)
257         {
258             (*state)[j][i] = getSBoxValue((*state)[j][i]);
259         }
260     }
261 }
262
263 // The ShiftRows() function shifts the rows in the state to
the left.
264 // Each row is shifted with different offset.
265 // Offset = Row number. So the first row is not shifted.
266 static void ShiftRows(state_t* state)
267 {
268     uint8_t temp;
269
270     // Rotate first row 1 columns to left
271     temp = (*state)[0][1];
272     (*state)[0][1] = (*state)[1][1];
273     (*state)[1][1] = (*state)[2][1];
274     (*state)[2][1] = (*state)[3][1];
275     (*state)[3][1] = temp;
276
277     // Rotate second row 2 columns to left
278     temp = (*state)[0][2];
279     (*state)[0][2] = (*state)[2][2];
280     (*state)[2][2] = temp;
281
282     temp = (*state)[1][2];

```

```

283     (*state)[1][2] = (*state)[3][2];
284     (*state)[3][2] = temp;
285
286     // Rotate third row 3 columns to left
287     temp = (*state)[0][3];
288     (*state)[0][3] = (*state)[3][3];
289     (*state)[3][3] = (*state)[2][3];
290     (*state)[2][3] = (*state)[1][3];
291     (*state)[1][3] = temp;
292 }
293
294 static uint8_t xtime(uint8_t x)
295 {
296     return ((x<1) ^ (((x>7) & 1) * 0x1b));
297 }
298
299 // MixColumns function mixes the columns of the state matrix
300 static void MixColumns(state_t* state)
301 {
302     uint8_t i;
303     uint8_t Tmp, Tm, t;
304     for (i = 0; i < 4; ++i)
305     {
306         t = (*state)[i][0];
307         Tmp = (*state)[i][0] ^ (*state)[i][1] ^ (*state)[i][2] ^
(*state)[i][3];
308         Tm = (*state)[i][0] ^ (*state)[i][1]; Tm = xtime(Tm);
309         (*state)[i][0] ^= Tm ^ Tmp;
310         Tm = (*state)[i][1] ^ (*state)[i][2]; Tm = xtime(Tm);
311         (*state)[i][1] ^= Tm ^ Tmp;
312         Tm = (*state)[i][2] ^ (*state)[i][3]; Tm = xtime(Tm);
313         (*state)[i][2] ^= Tm ^ Tmp;
314         Tm = (*state)[i][3] ^ t; Tm = xtime(Tm);
315         (*state)[i][3] ^= Tm ^ Tmp;
316     }
317 }
318
319 // Multiply is used to multiply numbers in the field GF(2^8)
320 // Note: The last call to xtime() is unneeded, but often ends
up generating a smaller binary
321 // The compiler seems to be able to vectorize the opera-
tion better this way.
322 // See https://github.com/kokke/tiny-AES-c/pull/34
323 #if MULTIPLY_AS_A_FUNCTION
324 static uint8_t Multiply(uint8_t x, uint8_t y)
325 {
326     return (((y & 1) * x) ^
327             ((y>1 & 1) * xtime(x)) ^
328             ((y>2 & 1) * xtime(xtime(x))) ^
329             ((y>3 & 1) * xtime(xtime(xtime(x)))) ^
330             ((y>4 & 1) * xtime(xtime(xtime(xtime(x)))))); /* this
last call to xtime() can be omitted */
331 }
332 #else
333 #define Multiply(x, y) \
334     ( ((y & 1) * x) ^ \
335       ((y>1 & 1) * xtime(x)) ^ \
336       ((y>2 & 1) * xtime(xtime(x))) ^ \
337       ((y>3 & 1) * xtime(xtime(xtime(x)))) ^ \
338       ((y>4 & 1) * xtime(xtime(xtime(xtime(x)))))) )
339 #endif
340
341 #if (defined(CBC) && (CBC == 1)) || (defined(ECB) && (ECB == 1)
/*
342 static uint8_t getSBoxInvert(uint8_t num)
343 {
344     return rsbox[num];
345 }
346 */
347 #define getSBoxInvert(num) (rsbox[(num)])
348
349 // MixColumns function mixes the columns of the state matrix.
350 // The method used to multiply may be difficult to understand
for the inexperienced.
351 // Please use the references to gain more information.
352 static void InvMixColumns(state_t* state)
353 {
354     int i;
355     uint8_t a, b, c, d;
356     for (i = 0; i < 4; ++i)
357     {
358         a = (*state)[i][0];
359         b = (*state)[i][1];
360         c = (*state)[i][2];
361         d = (*state)[i][3];
362
363         (*state)[i][0] = Multiply(a, 0x0e) ^ Multiply(b, 0x0b) ^
Multiply(c, 0x0d) ^ Multiply(d, 0x09);
364         (*state)[i][1] = Multiply(a, 0x09) ^ Multiply(b, 0x0e) ^
Multiply(c, 0x0b) ^ Multiply(d, 0x0d);
365         (*state)[i][2] = Multiply(a, 0x0d) ^ Multiply(b, 0x09) ^
Multiply(c, 0x0e) ^ Multiply(d, 0x0b);
366         (*state)[i][3] = Multiply(a, 0x0b) ^ Multiply(b, 0x0d) ^
Multiply(c, 0x09) ^ Multiply(d, 0x0e);
367     }
368 }
369
370 // The SubBytes Function Substitutes the values in the
state matrix with values in an S-box.

```

```

371 static void InvSubBytes(state_t* state)
372 {
373     uint8_t i, j;
374     for (i = 0; i < 4; ++i)
375     {
376         for (j = 0; j < 4; ++j)
377         {
378             (*state)[j][i] = getSBoxInvert((*state)[j][i]);
379         }
380     }
381 }
382
383 static void InvShiftRows(state_t* state)
384 {
385     uint8_t temp;
386
387     // Rotate first row 1 columns to right
388     temp = (*state)[3][1];
389     (*state)[3][1] = (*state)[2][1];
390     (*state)[2][1] = (*state)[1][1];
391     (*state)[1][1] = (*state)[0][1];
392     (*state)[0][1] = temp;
393
394     // Rotate second row 2 columns to right
395     temp = (*state)[0][2];
396     (*state)[0][2] = (*state)[2][2];
397     (*state)[2][2] = temp;
398
399     temp = (*state)[1][2];
400     (*state)[1][2] = (*state)[3][2];
401     (*state)[3][2] = temp;
402
403     // Rotate third row 3 columns to right
404     temp = (*state)[0][3];
405     (*state)[0][3] = (*state)[1][3];
406     (*state)[1][3] = (*state)[2][3];
407     (*state)[2][3] = (*state)[3][3];
408     (*state)[3][3] = temp;
409 }
410 #endif // #if (defined(CBC) && CBC == 1) || (defined(ECB) &&
ECB == 1)
411
412 // Cipher is the main function that encrypts the PlainText.
413 static void Cipher(state_t* state, const uint8_t* RoundKey)
414 {
415     uint8_t round = 0;
416
417     // Add the First round key to the state before starting the
rounds.
418     AddRoundKey(0, state, RoundKey);
419
420     // There will be Nr rounds.
421     // The first Nr-1 rounds are identical.
422     // These Nr rounds are executed in the loop below.
423     // Last one without MixColumns()
424     for (round = 1; ; ++round)
425     {
426         SubBytes(state);
427         ShiftRows(state);
428         if (round == Nr) {
429             break;
430         }
431         MixColumns(state);
432         AddRoundKey(round, state, RoundKey);
433     }
434     // Add round key to last round
435     AddRoundKey(Nr, state, RoundKey);
436 }
437
438 #if (defined(CBC) && CBC == 1) || (defined(ECB) && ECB == 1)
439 static void InvCipher(state_t* state, const uint8_t* RoundKey)
440 {
441     uint8_t round = 0;
442
443     // Add the First round key to the state before starting the
rounds.
444     AddRoundKey(Nr, state, RoundKey);
445
446     // There will be Nr rounds.
447     // The first Nr-1 rounds are identical.
448     // These Nr rounds are executed in the loop below.
449     // Last one without InvMixColumn()
450     for (round = (Nr - 1); ; --round)
451     {
452         InvShiftRows(state);
453         InvSubBytes(state);
454         AddRoundKey(round, state, RoundKey);
455         if (round == 0) {
456             break;
457         }
458         InvMixColumns(state);
459     }
460 }
461 #endif // #if (defined(CBC) && CBC == 1) || (defined(ECB) &&
ECB == 1)
462
463
464 /
*****
*****
465 /* Public func-

```

```

tions:
466 /
*****
*****
467 #if defined(ECB) && (ECB == 1)
468
469 void AES_ECB_encrypt(const struct AES_ctx* ctx, uint8_t* buf)
470 {
471     // The next function call encrypts the PlainText with the
Key using AES algorithm.
472     Cipher((state_t*)buf, ctx->RoundKey);
473 }
474
475 void AES_ECB_decrypt(const struct AES_ctx* ctx, uint8_t* buf)
476 {
477     // The next function call decrypts the PlainText with the
Key using AES algorithm.
478     InvCipher((state_t*)buf, ctx->RoundKey);
479 }
480
481 #endif // #if defined(ECB) && (ECB == 1)
482
483 #if defined(CBC) && (CBC == 1)
484
485 static void XorWithIv(uint8_t* buf, const uint8_t* Iv)
486 {
487     uint8_t i;
488     for (i = 0; i < AES_BLOCKLEN; ++i) // The block in AES is
always 128bit no matter the key size
489     {
490         buf[i] ^= Iv[i];
491     }
492 }
493
494 void AES_CBC_encrypt_buffer(struct AES_ctx* ctx, uint8_t* buf,
size_t length)
495 {
496     size_t i;
497     uint8_t* Iv = ctx->Iv;
498     for (i = 0; i < length; i += AES_BLOCKLEN)
499     {
500         XorWithIv(buf, Iv);
501         Cipher((state_t*)buf, ctx->RoundKey);
502         Iv = buf;
503         buf += AES_BLOCKLEN;
504     }
505     /* store Iv in ctx for next call */
506     memcpy(ctx->Iv, Iv, AES_BLOCKLEN);
507 }
508
509 void AES_CBC_decrypt_buffer(struct AES_ctx* ctx, uint8_t* buf,
size_t length)
510 {
511     size_t i;
512     uint8_t storeNextIv[AES_BLOCKLEN];
513     for (i = 0; i < length; i += AES_BLOCKLEN)
514     {
515         memcpy(storeNextIv, buf, AES_BLOCKLEN);
516         InvCipher((state_t*)buf, ctx->RoundKey);
517         XorWithIv(buf, ctx->Iv);
518         memcpy(ctx->Iv, storeNextIv, AES_BLOCKLEN);
519         buf += AES_BLOCKLEN;
520     }
521 }
522 #endif // #if defined(CBC) && (CBC == 1)
523
524 #if defined(CTR) && (CTR == 1)
525
526 /* Symmetrical operation: same function for encrypting as for
decrypting. Note any IV/nonce should never be reused with the same
key */
527 void AES_CTR_xcrypt_buffer(struct AES_ctx* ctx, uint8_t* buf,
size_t length)
528 {
529     uint8_t buffer[AES_BLOCKLEN];
530
531     size_t i;
532     int bi;
533     for (i = 0, bi = AES_BLOCKLEN; i < length; ++i, ++bi)
534     {
535         if (bi == AES_BLOCKLEN) /* we need to regen xor compliment
in buffer */
536         {
537             memcpy(buffer, ctx->Iv, AES_BLOCKLEN);
538             Cipher((state_t*)buffer, ctx->RoundKey);
539         }
540     }

```

```

552 /* Increment Iv and handle overflow */
553 for (bi = (AES_BLOCKLEN - 1); bi >= 0; --bi)
554 {
555     /* inc will overflow */
556     if (ctx->Iv[bi] == 255)
557     {
558         ctx->Iv[bi] = 0;
559         continue;
560     }
561     ctx->Iv[bi] += 1;
562     break;
563 }
564 bi = 0;
565 }
566
567 buf[i] = (buf[i] ^ buffer[bi]);
568 }
569 }
570
571 #endif // #if defined(CTR) && (CTR == 1)
572

```

2. kokke_tiny-AES-c/aes.h (91 lines)

```

1 #ifndef _AES_H_
2 #define _AES_H_
3
4 #include <stdint.h>
5 #include <stddef.h>
6
7 // #define the macros below to 1/0 to enable/disable the mode
of operation.
8 //
9 // CBC enables AES encryption in CBC-mode of operation.
10 // CTR enables encryption in counter-mode.
11 // ECB enables the basic ECB 16-byte block algorithm. All can
be enabled simultaneously.
12
13 // The #ifndef-guard allows it to be configured before #in-
clude'ing or at compile time.
14 #ifndef CBC
15 #define CBC 1
16 #endif
17
18 #ifndef ECB
19 #define ECB 1
20 #endif
21
22 #ifndef CTR
23 #define CTR 1
24 #endif
25
26
27 #define AES128 1
28 // #define AES192 1
29 // #define AES256 1
30
31 #define AES_BLOCKLEN 16 // Block length in bytes - AES is 128b
block only
32
33 #if defined(AES256) && (AES256 == 1)
34 #define AES_KEYLEN 32
35 #define AES_keyExpSize 240
36 #elif defined(AES192) && (AES192 == 1)
37 #define AES_KEYLEN 24
38 #define AES_keyExpSize 208
39 #else
40 #define AES_KEYLEN 16 // Key length in bytes
41 #define AES_keyExpSize 176
42 #endif
43
44 struct AES_ctx
45 {
46     uint8_t RoundKey[AES_keyExpSize];
47 #if (defined(CBC) && (CBC == 1)) || (defined(CTR) && (CTR ==
1))
48     uint8_t Iv[AES_BLOCKLEN];
49 #endif
50 };
51
52 void AES_init_ctx(struct AES_ctx* ctx, const uint8_t* key);
53 #if (defined(CBC) && (CBC == 1)) || (defined(CTR) && (CTR ==
1))
54 void AES_init_ctx_iv(struct AES_ctx* ctx, const uint8_t* key,
const uint8_t* iv);
55 void AES_ctx_set_iv(struct AES_ctx* ctx, const uint8_t* iv);
56 #endif
57
58 #if defined(ECB) && (ECB == 1)
59 // buffer size is exactly AES_BLOCKLEN bytes;
60 // you need only AES_init_ctx as IV is not used in ECB
61 // NB: ECB is considered insecure for most uses
62 void AES_ECB_encrypt(const struct AES_ctx* ctx, uint8_t* buf);
63 void AES_ECB_decrypt(const struct AES_ctx* ctx, uint8_t* buf);
64
65 #endif // #if defined(ECB) && (ECB == 1)
66
67
68 #if defined(CBC) && (CBC == 1)

```

```

69 // buffer size MUST be mutile of AES_BLOCKLEN;
70 // Suggest https://en.wikipedia.org/wiki/Pad-
ding_(cryptography)#PKCS7 for padding scheme
71 // NOTES: you need to set IV in ctx via AES_init_ctx_iv() or
AES_ctx_set_iv()
72 // no IV should ever be reused with the same key
73
74 void AES_CBC_encrypt_buffer(struct AES_ctx* ctx, uint8_t* buf,
size_t length);
75
76 #endif // #if defined(CBC) && (CBC == 1)
77
78
79 #if defined(CTR) && (CTR == 1)
80
81 // Same function for encrypting as for decrypting.
82 // IV is incremented for every block, and used after encryp-
tion as XOR-compliment for output
83 // Suggesting https://en.wikipedia.org/wiki/Pad-
ding_(cryptography)#PKCS7 for padding scheme
84 // NOTES: you need to set IV in ctx with AES_init_ctx_iv() or
AES_ctx_set_iv()
85 // no IV should ever be reused with the same key
86 void AES_CTR_xcrypt_buffer(struct AES_ctx* ctx, uint8_t* buf,
size_t length);
87
88 #endif // #if defined(CTR) && (CTR == 1)
89
90
91 #endif // _AES_H_

```

3. kokke_tiny-AES-c/test.c (316 lines)

```

1 #include <stdio.h>
2 #include <string.h>
3 #include <stdint.h>
4
5 // Enable ECB, CTR and CBC mode. Note this can be done before
including aes.h or at compile-time.
6 // E.g. with GCC by using the -D flag: gcc -c aes.c -DCBC=0 -
DCTR=1 -DECBC=1
7 #define CBC 1
8 #define CTR 1
9 #define ECB 1
10
11 #include "aes.h"
12
13
14 static void phex(uint8_t* str);
15 static int test_encrypt_cbc(void);
16 static int test_decrypt_cbc(void);
17 static int test_encrypt_ctr(void);
18 static int test_decrypt_ctr(void);
19 static int test_encrypt_ecb(void);
20 static int test_decrypt_ecb(void);
21 static void test_encrypt_ecb_verbose(void);
22
23
24 int main(void)
25 {
26     int exit;
27
28     #if defined(AES256)
29     printf("\nTesting AES256\n\n");
30     #elif defined(AES192)
31     printf("\nTesting AES192\n\n");
32     #elif defined(AES128)
33     printf("\nTesting AES128\n\n");
34     #else
35     printf("You need to specify a symbol between AES128,
AES192 or AES256. Exiting");
36     return 0;
37     #endif
38
39     exit = test_encrypt_cbc() + test_decrypt_cbc() +
test_encrypt_ctr() + test_decrypt_ctr() +
test_decrypt_ecb() + test_encrypt_ecb();
42     test_encrypt_ecb_verbose();
43
44     return exit;
45 }
46
47
48 // prints string as hex
49 static void phex(uint8_t* str)
50 {
51
52     #if defined(AES256)
53     uint8_t len = 32;
54     #elif defined(AES192)
55     uint8_t len = 24;
56     #elif defined(AES128)
57     uint8_t len = 16;
58     #endif
59
60     unsigned char i;
61     for (i = 0; i < len; ++i)

```

```

62     printf("%.2x", str[i]);
63     printf("\n");
64 }
65
66 static void test_encrypt_ecb_verbose(void)
67 {
68     // Example of more verbose verification
69
70     uint8_t i;
71
72     // 128bit key
73
74     uint8_t key[16] = { (uint8_t) 0x2b, (uint8_t) 0x7e,
75 (uint8_t) 0x15, (uint8_t) 0x16, (uint8_t) 0x28, (uint8_t) 0xae,
76 (uint8_t) 0xd2, (uint8_t) 0xa6, (uint8_t) 0xab, (uint8_t) 0xf7,
77 (uint8_t) 0x15, (uint8_t) 0x88, (uint8_t) 0x09, (uint8_t) 0xcf,
78 (uint8_t) 0x4f, (uint8_t) 0x3c };
79     // 512bit text
80
81     uint8_t plain_text[64] = { (uint8_t) 0x6b, (uint8_t) 0xc1,
82 (uint8_t) 0xbe, (uint8_t) 0xe2, (uint8_t) 0xe2, (uint8_t) 0x40,
83 (uint8_t) 0x9f, (uint8_t) 0x96, (uint8_t) 0xe9, (uint8_t) 0x3d,
84 (uint8_t) 0x7e, (uint8_t) 0x11, (uint8_t) 0x73, (uint8_t) 0x93,
85 (uint8_t) 0x17, (uint8_t) 0x2a,
86
87 (uint8_t) 0x8a, (uint8_t) 0x57, (uint8_t) 0x1e, (uint8_t) 0x03,
88 (uint8_t) 0xac, (uint8_t) 0x9c, (uint8_t) 0x9e, (uint8_t) 0xb7,
89 (uint8_t) 0x6f, (uint8_t) 0xac, (uint8_t) 0x45, (uint8_t) 0xaf,
90 (uint8_t) 0x8e, (uint8_t) 0x51,
91
92 (uint8_t) 0x1c, (uint8_t) 0x46, (uint8_t) 0xa3, (uint8_t) 0x5c,
93 (uint8_t) 0xe4, (uint8_t) 0x11, (uint8_t) 0xe5, (uint8_t) 0xfb,
94 (uint8_t) 0xc1, (uint8_t) 0x19, (uint8_t) 0x1a, (uint8_t) 0x0a,
95 (uint8_t) 0x52, (uint8_t) 0xef,
96
97 (uint8_t) 0xf6, (uint8_t) 0x9f,
98 (uint8_t) 0x24, (uint8_t) 0x45, (uint8_t) 0xdf, (uint8_t) 0x4f,
99 (uint8_t) 0x9b, (uint8_t) 0x17, (uint8_t) 0xad, (uint8_t) 0x2b,
100 (uint8_t) 0x41, (uint8_t) 0x7b, (uint8_t) 0xe6, (uint8_t) 0x6c,
101 (uint8_t) 0x37, (uint8_t) 0x10 };
102
103     // print text to encrypt, key and IV
104     printf("ECB encrypt verbose:\n");
105     printf("plain text:\n");
106     for (i = (uint8_t) 0; i < (uint8_t) 4; ++i)
107     {
108         phex(plain_text + i * (uint8_t) 16);
109     }
110     printf("\n");
111
112     printf("key:\n");
113     phex(key);
114     printf("\n");
115
116     // print the resulting cipher as 4 x 16 byte strings
117     printf("ciphertext:\n");
118
119     struct AES_ctx ctx;
120     AES_init_ctx(&ctx, key);
121
122     for (i = 0; i < 4; ++i)
123     {
124         AES_ECB_encrypt(&ctx, plain_text + (i * 16));
125         phex(plain_text + (i * 16));
126     }
127     printf("\n");
128 }
129
130 static int test_encrypt_ecb(void)
131 {
132     #if defined(AES256)
133         uint8_t key[] = { 0x60, 0x3d, 0xeb, 0x10, 0x15, 0xca,
134 0x71, 0xbe, 0x2b, 0x73, 0xae, 0xf0, 0x85, 0x7d, 0x77, 0x81,
135 0x1f, 0x35, 0x2c, 0x07, 0x3b, 0x61,
136 0x08, 0xd7, 0x2d, 0x98, 0x10, 0xa3, 0x09, 0x14, 0xdf, 0xf4 };
137         uint8_t out[] = { 0xf3, 0xee, 0xd1, 0xbd, 0xb5, 0xd2,
138 0xa0, 0x3c, 0x06, 0x4b, 0x5a, 0x7e, 0x3d, 0xb1, 0x81, 0xf8 };
139     #elif defined(AES192)
140         uint8_t key[] = { 0x8e, 0x73, 0xb0, 0xf7, 0xda, 0x0e,
141 0x64, 0x52, 0xc8, 0x10, 0xf3, 0x2b, 0x80, 0x90, 0x79, 0xe5,
142 0x62, 0xf8, 0xea, 0xd2, 0x52, 0x2c,
143 0x6b, 0x7b };
144         uint8_t out[] = { 0xbd, 0x33, 0x4f, 0x1d, 0x6e, 0x45,
145 0xf2, 0x5f, 0xf7, 0x12, 0xa2, 0x14, 0x57, 0x1f, 0xa5, 0xcc };
146     #elif defined(AES128)
147         uint8_t key[] = { 0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae,
148 0xd2, 0xa6, 0xab, 0xf7, 0x15, 0x88, 0x09, 0xcf, 0x4f, 0x3c };
149         uint8_t out[] = { 0x3a, 0xd7, 0x7b, 0xb4, 0x0d, 0x7a,
150 0x36, 0x60, 0xa8, 0x9e, 0xca, 0xf3, 0x24, 0x66, 0xef, 0x97 };
151     #endif
152
153     uint8_t in[] = { 0x6b, 0xc1, 0xbe, 0xe2, 0xe2, 0x40,
154 0x9f, 0x96, 0xe9, 0x3d, 0x7e, 0x11, 0x73, 0x93, 0x17, 0x2a };
155     struct AES_ctx ctx;
156
157     AES_init_ctx(&ctx, key);
158     AES_ECB_encrypt(&ctx, in);
159 }

```

```

129     printf("ECB encrypt: ");
130
131     if (0 == memcmp((char*) out, (char*) in, 16)) {
132         printf("SUCCESS!\n");
133         return(0);
134     } else {
135         printf("FAILURE!\n");
136         return(1);
137     }
138 }
139
140 static int test_decrypt_cbc(void)
141 {
142     #if defined(AES256)
143         uint8_t key[] = { 0x60, 0x3d, 0xeb, 0x10, 0x15, 0xca,
144 0x71, 0xbe, 0x2b, 0x73, 0xae, 0xf0, 0x85, 0x7d, 0x77, 0x81,
145 0x1f, 0x35, 0x2c, 0x07, 0x3b, 0x61,
146 0x08, 0xd7, 0x2d, 0x98, 0x10, 0xa3, 0x09, 0x14, 0xdf, 0xf4 };
147         uint8_t in[] = { 0xf5, 0x8c, 0x4c, 0x04, 0xd6, 0xe5,
148 0xf1, 0xba, 0x77, 0x9e, 0xab, 0xfb, 0x5f, 0x7b, 0xfb, 0xd6,
149 0x9c, 0xfc, 0x4e, 0x96, 0x7e, 0xdb,
150 0x80, 0x8d, 0x67, 0x9f, 0x77, 0x7b, 0xc6, 0x70, 0x2c, 0x7d,
151 0x39, 0xf2, 0x33, 0x69, 0xa9, 0xd9,
152 0xba, 0xcf, 0xa5, 0x30, 0xe2, 0x63, 0x04, 0x23, 0x14, 0x61,
153 0xb2, 0xeb, 0x05, 0xe2, 0xc3, 0x9b,
154 0xe9, 0xfc, 0xda, 0x6c, 0x19, 0x07, 0x8c, 0x6a, 0x9d, 0x1b };
155     #elif defined(AES192)
156         uint8_t key[] = { 0x8e, 0x73, 0xb0, 0xf7, 0xda, 0x0e,
157 0x64, 0x52, 0xc8, 0x10, 0xf3, 0x2b, 0x80, 0x90, 0x79, 0xe5, 0x62,
158 0xf8, 0xea, 0xd2, 0x52, 0x2c, 0x6b, 0x7b };
159         uint8_t in[] = { 0x4f, 0x02, 0x1d, 0xb2, 0x43, 0xbc,
160 0x63, 0x3d, 0x71, 0x78, 0x18, 0x3a, 0x9f, 0xa0, 0x71, 0xe8,
161 0xb4, 0xd9, 0xad, 0xa9, 0xad, 0x7d,
162 0xed, 0xf4, 0xe5, 0xe7, 0x38, 0x76, 0x3f, 0x69, 0x14, 0x5a,
163 0x57, 0x1b, 0x24, 0x20, 0x12, 0xfb,
164 0x7a, 0xe0, 0x7f, 0xa9, 0xba, 0xac, 0x3d, 0xf1, 0x02, 0xe0,
165 0x08, 0xb0, 0xe2, 0x79, 0x88, 0x59,
166 0x88, 0x81, 0xd9, 0x20, 0xa9, 0xe6, 0x4f, 0x56, 0x15, 0xcd };
167     #elif defined(AES128)
168         uint8_t key[] = { 0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae,
169 0xd2, 0xa6, 0xab, 0xf7, 0x15, 0x88, 0x09, 0xcf, 0x4f, 0x3c };
170         uint8_t in[] = { 0x76, 0x49, 0xab, 0xac, 0x81, 0x19,
171 0xb2, 0x46, 0xce, 0xe9, 0x8e, 0x9b, 0x12, 0xe9, 0x19, 0x7d,
172 0x50, 0x86, 0xcb, 0x9b, 0x50, 0x72,
173 0x19, 0xee, 0x95, 0xdb, 0x11, 0x3a, 0x91, 0x76, 0x78, 0xb2,
174 0x73, 0xbe, 0xd6, 0xb8, 0xe3, 0xc1,
175 0x74, 0x3b, 0x71, 0x16, 0xe6, 0x9e, 0x22, 0x22, 0x95, 0x16,
176 0x3f, 0xf1, 0xca, 0xa1, 0x68, 0x1f,
177 0xac, 0x09, 0x12, 0x0e, 0xca, 0x30, 0x75, 0x86, 0xe1, 0xa7 };
178     #endif
179         uint8_t iv[] = { 0x00, 0x01, 0x02, 0x03, 0x04, 0x05,
180 0x06, 0x07, 0x08, 0x09, 0x0a, 0x0b, 0x0c, 0x0d, 0x0e, 0x0f };
181         uint8_t out[] = { 0x6b, 0xc1, 0xbe, 0xe2, 0xe2, 0x40,
182 0x9f, 0x96, 0xe9, 0x3d, 0x7e, 0x11, 0x73, 0x93, 0x17, 0x2a,
183 0xae, 0x2d, 0x8a, 0x57, 0x1e, 0x03,
184 0xac, 0x9c, 0x9e, 0xb7, 0x6f, 0xac, 0x45, 0xaf, 0x8e, 0x51,
185 0x30, 0xc8, 0x1c, 0x46, 0x0a, 0x33, 0x5c,
186 0xe4, 0x11, 0xe5, 0xfb, 0xc1, 0x19, 0x1a, 0x0a, 0x52, 0xef,
187 0xf6, 0x9f, 0x24, 0x45, 0xdf, 0x4f,
188 0x9b, 0x17, 0xad, 0x2b, 0x41, 0x7b, 0xe6, 0x6c, 0x37, 0x10 };
189         struct AES_ctx ctx;
190
191         AES_init_ctx_iv(&ctx, key, iv);
192         AES_CBC_decrypt_buffer(&ctx, in, 64);
193
194         printf("CBC decrypt: ");
195
196         if (0 == memcmp((char*) out, (char*) in, 64)) {
197             printf("SUCCESS!\n");
198             return(0);
199         } else {
200             printf("FAILURE!\n");
201             return(1);
202         }
203     }
204 }
205
206 static int test_encrypt_cbc(void)
207 {
208     #if defined(AES256)
209         uint8_t key[] = { 0x60, 0x3d, 0xeb, 0x10, 0x15, 0xca,
210 0x71, 0xbe, 0x2b, 0x73, 0xae, 0xf0, 0x85, 0x7d, 0x77, 0x81,
211 0x1f, 0x35, 0x2c, 0x07, 0x3b, 0x61,
212 0x08, 0xd7, 0x2d, 0x98, 0x10, 0xa3, 0x09, 0x14, 0xdf, 0xf4 };
213         uint8_t out[] = { 0xf5, 0x8c, 0x4c, 0x04, 0xd6, 0xe5,
214 0xf1, 0xba, 0x77, 0x9e, 0xab, 0xfb, 0x5f, 0x7b, 0xfb, 0xd6,
215 0x9c, 0xfc, 0x4e, 0x96, 0x7e, 0xdb,
216 0x80, 0x8d, 0x67, 0x9f, 0x77, 0x7b, 0xc6, 0x70, 0x2c, 0x7d,
217 0x39, 0xf2, 0x33, 0x69, 0xa9, 0xd9,
218 0xba, 0xcf, 0xa5, 0x30, 0xe2, 0x63, 0x04, 0x23, 0x14, 0x61,
219 0xb2, 0xeb, 0x05, 0xe2, 0xc3, 0x9b,
220 0xe9, 0xfc, 0xda, 0x6c, 0x19, 0x07, 0x8c, 0x6a, 0x9d, 0x1b };
221     #elif defined(AES192)
222         uint8_t key[] = { 0x8e, 0x73, 0xb0, 0xf7, 0xda, 0x0e,
223 0x64, 0x52, 0xc8, 0x10, 0xf3, 0x2b, 0x80, 0x90, 0x79, 0xe5, 0x62,
224 0xf8, 0xea, 0xd2, 0x52, 0x2c, 0x6b, 0x7b };
225         uint8_t out[] = { 0x4f, 0x02, 0x1d, 0xb2, 0x43, 0xbc,
226 0x63, 0x3d, 0x71, 0x78, 0x18, 0x3a, 0x9f, 0xa0, 0x71, 0xe8,
227 0xb4, 0xd9, 0xad, 0xa9, 0xad, 0x7d,
228 0xed, 0xf4, 0xe5, 0xe7, 0x38, 0x76, 0x3f, 0x69, 0x14, 0x5a,

```



```

198         0x57, 0x1b, 0x24, 0x20, 0x12, 0xfb,
0x7a, 0xe0, 0x7f, 0xa9, 0xba, 0xac, 0x3d, 0xf1, 0x02, 0xe0,
199         0x08, 0xb0, 0xe2, 0x79, 0x88, 0x59,
0x88, 0x81, 0xd9, 0x20, 0xa9, 0xe6, 0x4f, 0x56, 0x15, 0xcd };
200 #elif defined(AES128)
201     uint8_t key[] = { 0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae,
0xd2, 0xa6, 0xab, 0xf7, 0x15, 0x88, 0x09, 0xcf, 0x4f, 0x3c };
202     uint8_t out[] = { 0x76, 0x49, 0xab, 0xac, 0x81, 0x19,
0xb2, 0x46, 0xce, 0xe9, 0x8e, 0x9b, 0x12, 0xe9, 0x19, 0x7d,
203         0x50, 0x86, 0xcb, 0x9b, 0x50, 0x72,
0x19, 0xee, 0x95, 0xdb, 0x11, 0x3a, 0x91, 0x76, 0x78, 0xb2,
204         0x73, 0xbe, 0xd6, 0xb8, 0xe3, 0xc1,
0x74, 0x3b, 0x71, 0x16, 0xe6, 0x9e, 0x22, 0x22, 0x95, 0x16,
205         0x3f, 0xf1, 0xca, 0xa1, 0x68, 0x1f,
0xac, 0x09, 0x12, 0xe0, 0xca, 0x30, 0x75, 0x86, 0xe1, 0xa7 };
206 #endif
207     uint8_t iv[] = { 0x00, 0x01, 0x02, 0x03, 0x04, 0x05,
0x06, 0x07, 0x08, 0x09, 0x0a, 0x0b, 0x0c, 0x0d, 0x0e, 0x0f };
208     uint8_t in[] = { 0x6b, 0xc1, 0xbe, 0xe2, 0x2e, 0x40,
0x9f, 0x96, 0xe9, 0x3d, 0x7e, 0x11, 0x73, 0x93, 0x17, 0x2a,
209         0xae, 0x2d, 0x8a, 0x57, 0x1e, 0x03,
0xac, 0x9c, 0x9e, 0xb7, 0x6f, 0xac, 0x45, 0xaf, 0x8e, 0x51,
210         0x30, 0xc8, 0x1c, 0x46, 0xa3, 0x5c,
0xe4, 0x11, 0xe5, 0xfb, 0xc1, 0x19, 0x1a, 0x0a, 0x52, 0xef,
211         0xf6, 0x9f, 0x24, 0x45, 0xdf, 0x4f,
0x9b, 0x17, 0xad, 0x2b, 0x41, 0x7b, 0xe6, 0x6c, 0x37, 0x10 };
212     struct AES_ctx ctx;
213
214     AES_init_ctx_iv(&ctx, key, iv);
215     AES_CBC_encrypt_buffer(&ctx, in, 64);
216
217     printf("CBC encrypt: ");
218
219     if (0 == memcmp((char*) out, (char*) in, 64)) {
220         printf("SUCCESS!\n");
221         return(0);
222     } else {
223         printf("FAILURE!\n");
224         return(1);
225     }
226 }
227
228 static int test_xcrypt_ctr(const char* xcrypt);
229 static int test_encrypt_ctr(void)
230 {
231     return test_xcrypt_ctr("encrypt");
232 }
233
234 static int test_decrypt_ctr(void)
235 {
236     return test_xcrypt_ctr("decrypt");
237 }
238
239 static int test_xcrypt_ctr(const char* xcrypt)
240 {
241     #if defined(AES256)
242         uint8_t key[32] = { 0x60, 0x3d, 0xeb, 0x10, 0x15, 0xca,
0x71, 0xbe, 0x2b, 0x73, 0xae, 0xf0, 0x85, 0x7d, 0x77, 0x81,
243         0x1f, 0x35, 0x2c, 0x07, 0x3b, 0x61,
0x08, 0xd7, 0x2d, 0x98, 0x10, 0xa3, 0x09, 0x14, 0xdf, 0xf4 };
244         uint8_t in[64] = { 0x60, 0x1e, 0xc3, 0x13, 0x77, 0x57,
0x89, 0xa5, 0xb7, 0xa7, 0xf5, 0x04, 0xbb, 0xf3, 0xd2, 0x28,
245         0xf4, 0x43, 0xe3, 0xca, 0x4d, 0x62,
0xb5, 0x9a, 0xca, 0x84, 0xe9, 0x90, 0xca, 0xca, 0xf5, 0xc5,
246         0x2b, 0x09, 0x30, 0xda, 0xa2, 0x3d,
0xe9, 0x4c, 0xe8, 0x70, 0x17, 0xba, 0x2d, 0x84, 0x98, 0x8d,
247         0xdf, 0xc9, 0xc5, 0x8d, 0xb6, 0x7a,
0xad, 0xa6, 0x13, 0xc2, 0xdd, 0x08, 0x45, 0x79, 0x41, 0xa6 };
248     #elif defined(AES192)
249         uint8_t key[24] = { 0x8e, 0x73, 0xb0, 0xf7, 0xda, 0x0e,
0x64, 0x52, 0xc8, 0x10, 0xf3, 0x2b, 0x80, 0x90, 0x79, 0xe5,
250         0x62, 0xf8, 0xea, 0xd2, 0x52, 0x2c,
0x6b, 0x7b };
251         uint8_t in[64] = { 0x1a, 0xbc, 0x93, 0x24, 0x17, 0x52,
0x1c, 0xa2, 0x4f, 0x2b, 0x04, 0x59, 0xfe, 0x7e, 0x6e, 0x0b,
252         0x09, 0x03, 0x39, 0xec, 0x0a, 0xa6,
0xfa, 0xef, 0xd5, 0xcc, 0xc2, 0xc6, 0xf4, 0xce, 0x8e, 0x94,
253         0x1e, 0x36, 0xb2, 0x6b, 0xd1, 0xeb,
0xc6, 0x70, 0xd1, 0xbd, 0x1d, 0x66, 0x56, 0x20, 0xab, 0xf7,
254         0x4f, 0x78, 0xa7, 0xf6, 0xd2, 0x98,
0x09, 0x58, 0x5a, 0x97, 0xda, 0xec, 0x58, 0xc6, 0xb0, 0x50 };

```

```

255 #elif defined(AES128)
256     uint8_t key[16] = { 0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae,
0xd2, 0xa6, 0xab, 0xf7, 0x15, 0x88, 0x09, 0xcf, 0x4f, 0x3c };
257     uint8_t in[64] = { 0x87, 0x4d, 0x61, 0x91, 0xb6, 0x20,
0xe3, 0x26, 0x1b, 0xef, 0x68, 0x64, 0x99, 0x0d, 0xb6, 0xce,
258         0x98, 0x06, 0xf6, 0x6b, 0x79, 0x70,
0xfd, 0xff, 0x86, 0x17, 0x18, 0x7b, 0xb9, 0xff, 0xfd, 0xff,
259         0x5a, 0xe4, 0xdf, 0x3e, 0xdb, 0xd5,
0xd3, 0x5e, 0x5b, 0x4f, 0x09, 0x02, 0x0d, 0xb0, 0x3e, 0xab,
260         0x1e, 0x03, 0x1d, 0xda, 0x2f, 0x8e,
0x03, 0xd1, 0x79, 0x21, 0x70, 0x70, 0xa0, 0xf3, 0x00, 0x9c, 0xee };
261 #endif
262     uint8_t iv[16] = { 0xf0, 0xf1, 0xf2, 0xf3, 0xf4, 0xf5,
0xf6, 0xf7, 0xf8, 0xf9, 0xfa, 0xfb, 0xfc, 0xfd, 0xfe, 0xff };
263     uint8_t out[64] = { 0x6b, 0xc1, 0xbe, 0xe2, 0x2e, 0x40,
0x9f, 0x96, 0xe9, 0x3d, 0x7e, 0x11, 0x73, 0x93, 0x17, 0x2a,
264         0xae, 0x2d, 0x8a, 0x57, 0x1e, 0x03,
0xac, 0x9c, 0x9e, 0xb7, 0x6f, 0xac, 0x45, 0xaf, 0x8e, 0x51,
265         0x30, 0xc8, 0x1c, 0x46, 0xa3, 0x5c,
0xe4, 0x11, 0xe5, 0xfb, 0xc1, 0x19, 0x1a, 0x0a, 0x52, 0xef,
266         0xf6, 0x9f, 0x24, 0x45, 0xdf, 0x4f,
0x9b, 0x17, 0xad, 0x2b, 0x41, 0x7b, 0xe6, 0x6c, 0x37, 0x10 };
267     struct AES_ctx ctx;
268
269     AES_init_ctx_iv(&ctx, key, iv);
270     AES_CTR_xcrypt_buffer(&ctx, in, 64);
271
272     printf("CTR %s: ", xcrypt);
273
274     if (0 == memcmp((char *) out, (char *) in, 64)) {
275         printf("SUCCESS!\n");
276         return(0);
277     } else {
278         printf("FAILURE!\n");
279         return(1);
280     }
281 }
282
283 static int test_decrypt_ecb(void)
284 {
285     #if defined(AES256)
286         uint8_t key[] = { 0x60, 0x3d, 0xeb, 0x10, 0x15, 0xca,
0x71, 0xbe, 0x2b, 0x73, 0xae, 0xf0, 0x85, 0x7d, 0x77, 0x81,
287         0x1f, 0x35, 0x2c, 0x07, 0x3b, 0x61,
0x08, 0xd7, 0x2d, 0x98, 0x10, 0xa3, 0x09, 0x14, 0xdf, 0xf4 };
288         uint8_t in[] = { 0xf3, 0xee, 0xd1, 0xbd, 0xb5, 0xd2,
0xa0, 0x3c, 0x06, 0x4b, 0x5a, 0x7e, 0x3d, 0xb1, 0x81, 0xf8 };
289         #elif defined(AES192)
290         uint8_t key[] = { 0x8e, 0x73, 0xb0, 0xf7, 0xda, 0x0e,
0x64, 0x52, 0xc8, 0x10, 0xf3, 0x2b, 0x80, 0x90, 0x79, 0xe5,
291         0x62, 0xf8, 0xea, 0xd2, 0x52, 0x2c,
0x6b, 0x7b };
292         uint8_t in[] = { 0xbd, 0x33, 0x4f, 0x1d, 0x6e, 0x45,
0xf2, 0x5f, 0xf7, 0x12, 0xa2, 0x14, 0x57, 0x1f, 0xa5, 0xcc };
293         #elif defined(AES128)
294         uint8_t key[] = { 0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae,
0xd2, 0xa6, 0xab, 0xf7, 0x15, 0x88, 0x09, 0xcf, 0x4f, 0x3c };
295         uint8_t in[] = { 0x3a, 0xd7, 0x7b, 0xb4, 0x0d, 0x7a,
0x36, 0x60, 0xa8, 0x9e, 0xca, 0xf3, 0x24, 0x66, 0xef, 0x97 };
296     #endif
297
298     uint8_t out[] = { 0x6b, 0xc1, 0xbe, 0xe2, 0x2e, 0x40,
0x9f, 0x96, 0xe9, 0x3d, 0x7e, 0x11, 0x73, 0x93, 0x17, 0x2a };
299     struct AES_ctx ctx;
300
301     AES_init_ctx(&ctx, key);
302     AES_ECB_decrypt(&ctx, in);
303
304     printf("ECB decrypt: ");
305
306     if (0 == memcmp((char*) out, (char*) in, 16)) {
307         printf("SUCCESS!\n");
308         return(0);
309     } else {
310         printf("FAILURE!\n");
311         return(1);
312     }
313 }
314 }
315
316

```

AES·SHA·MD5 C 참조 구현

What this is. Brad Conte's reference implementations of common cryptographic primitives, each self-contained in one file.

Why it exists. To serve as a second, independent implementation. A single golden model cannot reveal its own misreading of a standard. Two implementations written by different authors from the same document agree only where both read it the same way.

Undergraduate review · EECS concepts. Merkle–Damgård construction; compression functions; padding rules; collision resistance.

Graduate · Practice note. Cross-validating two independent models early makes the later triage of an RTL mismatch much cheaper, because the possibility “the model is wrong” can be dismissed in minutes rather than days.

30 files • 3,673 lines

API — aes.c

Table 694. B-Con_crypto-algorithms/aes.c — 함수 (26개)

함수	반환형	받는 파라미터
xor_buf	void	const BYTE in[], BYTE out[], size_t len
aes_encrypt_cbc	int	const BYTE in[], size_t in_len, BYTE out[], const WORD key[], int keysize, const BYTE iv[]
aes_encrypt_cbc_mac	int	const BYTE in[], size_t in_len, BYTE out[], const WORD key[], int keysize, const BYTE iv[]
aes_decrypt_cbc	int	const BYTE in[], size_t in_len, BYTE out[], const WORD key[], int keysize, const BYTE iv[]
increment_iv	void	BYTE iv[], int counter_size
aes_encrypt_ctr	void	const BYTE in[], size_t in_len, BYTE out[], const WORD key[], int keysize, const BYTE iv[]
aes_decrypt_ctr	void	const BYTE in[], size_t in_len, BYTE out[], const WORD key[], int keysize, const BYTE iv[]
aes_encrypt_ccm	int	const BYTE payload[], WORD payload_len, const BYTE assoc[], unsigned short assoc_len, const BYTE nonce[], unsigned short nonce_len, BYTE out[], WORD *...
aes_decrypt_ccm	int	const BYTE ciphertext[], WORD ciphertext_len, const BYTE assoc[], unsigned short assoc_len, const BYTE nonce[], unsigned short nonce_len, BYTE plainte...
ccm_prepare_first_ctr_blk	void	BYTE counter[], const BYTE nonce[], int nonce_len, int payload_len_store_size
ccm_prepare_first_format_blk	void	BYTE buf[], int assoc_len, int payload_len, int payload_len_store_size, int mac_len, const BYTE nonce[], int nonce_len
ccm_format_assoc_data	void	BYTE buf[], int *end_of_buf, const BYTE assoc[], int assoc_len
ccm_format_payload_data	void	BYTE buf[], int *end_of_buf, const BYTE payload[], int payload_len
SubWord	WORD	WORD word

함수	반환형	받는 파라미터
aes_key_setup	void	const BYTE key[], WORD w[], int keysize
AddRoundKey	void	BYTE state[][4], const WORD w[]
SubBytes	void	BYTE state[][4]
InvSubBytes	void	BYTE state[][4]
ShiftRows	void	BYTE state[][4]
InvShiftRows	void	BYTE state[][4]
MixColumns	void	BYTE state[][4]
InvMixColumns	void	BYTE state[][4]
aes_encrypt	void	const BYTE in[], BYTE out[], const WORD key[], int keysize
aes_decrypt	void	const BYTE in[], BYTE out[], const WORD key[], int keysize
print_state	void	BYTE state[][4]
print_rnd_key	void	WORD key[]

Table 695. B-Con_crypto-algorithms/aes.c — #define / `define 매크로 (6개)

매크로	인자	정의
KE_ROTWORD	(x)	((x) << 8) ((x) >> 24)
TRUE		1
FALSE		0
AES_128_ROUNDS		10
AES_192_ROUNDS		12
AES_256_ROUNDS		14

API — aes.h

Table 696. B-Con_crypto-algorithms/aes.h — typedef / using (2개)

새 이름	원래 형
BYTE	unsigned char
WORD	unsigned int

Table 697. B-Con_crypto-algorithms/aes.h — #define / `define 매크로 (2개)

매크로	인자	정의
AES_H		/***** HEADER FILES *****/
AES_BLOCK_SIZE		16 // AES operates on 16 bytes at a time

API — aes_test.c

Table 698. B-Con_crypto-algorithms/aes_test.c — 함수 (7개)

함수	반환형	받는 파라미터
print_hex	void	BYTE str[], int len

함수	반환형	받는 파라미터
aes_ecb_test	int	
aes_cbc_test	int	
aes_ctr_test	int	
aes_ccm_test	int	
aes_test	int	
main	int	int argc, char *argv[]

API — arcfour.c

Table 699. B-Con_crypto-algorithms/arcfour.c — 함수 (2개)

함수	반환형	받는 파라미터
arcfour_key_setup	void	BYTE state[], const BYTE key[], int len
arcfour_generate_stream	void	BYTE state[], BYTE out[], size_t len

API — arcfour.h

Table 700. B-Con_crypto-algorithms/arcfour.h — typedef / using (1개)

새 이름	원래 형
BYTE	unsigned char

Table 701. B-Con_crypto-algorithms/arcfour.h — #define / `define 매크로 (1개)

매크로	인자	정의
ARCFOUR_H		/***** HEADER FILES *****/

API — arcfour_test.c

Table 702. B-Con_crypto-algorithms/arcfour_test.c — 함수 (2개)

함수	반환형	받는 파라미터
rc4_test	int	
main	int	

API — base64.c

Table 703. B-Con_crypto-algorithms/base64.c — 함수 (3개)

함수	반환형	받는 파라미터
revchar	BYTE	char ch
base64_encode	size_t	const BYTE in[], BYTE out[], size_t len, int newline_flag
base64_decode	size_t	const BYTE in[], BYTE out[], size_t len

Table 704. B-Con_crypto-algorithms/base64.c — #define / `define 매크로 (1개)

매크로	인자	정의
NEWLINE_INVL		76

API — base64.h**Table 705. B-Con_crypto-algorithms/base64.h — typedef / using (1개)**

새 이름	원래 형
BYTE	unsigned char

Table 706. B-Con_crypto-algorithms/base64.h — #define / `define 매크로 (1개)

매크로	인자	정의
BASE64_H		/***** HEADER FILES *****/

API — base64_test.c**Table 707. B-Con_crypto-algorithms/base64_test.c — 함수 (2개)**

함수	반환형	받는 파라미터
base64_test	int	
main	int	

API — blowfish.c**Table 708. B-Con_crypto-algorithms/blowfish.c — 함수 (3개)**

함수	반환형	받는 파라미터
blowfish_encrypt	void	const BYTE in[], BYTE out[], const BLOWFISH_KEY *keystruct
blowfish_decrypt	void	const BYTE in[], BYTE out[], const BLOWFISH_KEY *keystruct
blowfish_key_setup	void	const BYTE user_key[], BLOWFISH_KEY *keystruct, size_t len

Table 709. B-Con_crypto-algorithms/blowfish.c — #define / `define 매크로 (3개)

매크로	인자	정의
F	(x,t)	t = keystruct->s[0][(x) >> 24]; \
swap	(r,l,t)	t = l; l = r; r = t;
ITERATION	(l,r,t,pval)	l ^= keystruct->p[pval]; F(l,t); r ^= t; swap(r,l,t);

API — blowfish.h**Table 710. B-Con_crypto-algorithms/blowfish.h — typedef / using (2개)**

새 이름	원래 형
BYTE	unsigned char
WORD	unsigned int

Table 711. B-Con_crypto-algorithms/blowfish.h — #define / `define 매크로 (2개)

매크로	인자	정의
BLOWFISH_H		/***** HEADER FILES *****/
BLOWFISH_BLOCK_SIZE		8 // Blowfish operates on 8 bytes at a time

API — blowfish_test.c**Table 712. B-Con_crypto-algorithms/blowfish_test.c — 함수 (2개)**

함수	반환형	받는 파라미터
blowfish_test	int	
main	int	

API — des.c**Table 713. B-Con_crypto-algorithms/des.c — 함수 (7개)**

함수	반환형	받는 파라미터
IP	void	WORD state[], const BYTE in[]
InvIP	void	WORD state[], BYTE in[]
f	WORD	WORD state, const BYTE key[]
des_key_setup	void	const BYTE key[], BYTE schedule[][6], DES_MODE mode
des_crypt	void	const BYTE in[], BYTE out[], const BYTE key[][6]
three_des_key_setup	void	const BYTE key[], BYTE schedule[][16][6], DES_MODE mode
three_des_crypt	void	const BYTE in[], BYTE out[], const BYTE key[][16][6]

Table 714. B-Con_crypto-algorithms/des.c — #define / `define 매크로 (4개)

매크로	인자	정의
BITNUM	(a,b,c)	$((a[(b)/8] \gg (7 - (b\%8))) \& 0x01) \ll (c))$
BITNUMINTR	(a,b,c)	$((a \gg (31 - (b))) \& 0x00000001) \ll (c))$
BITNUMINTL	(a,b,c)	$((a \ll (b)) \& 0x80000000) \gg (c))$
SBOXBIT	(a)	$((a \& 0x20) ((a \& 0x1f) \gg 1) ((a \& 0x01) \ll 4))$

API — des.h**Table 715. B-Con_crypto-algorithms/des.h — typedef / using (2개)**

새 이름	원래 형
BYTE	unsigned char
WORD	unsigned int

Table 716. B-Con_crypto-algorithms/des.h — #define / `define 매크로 (2개)

매크로	인자	정의
DESH		/***** HEADER FILES *****/
DES_BLOCK_SIZE		8 // DES operates on 8 bytes at a time

API — des_test.c**Table 717. B-Con_crypto-algorithms/des_test.c — 함수 (2개)**

함수	반환형	받는 파라미터
des_test	int	
main	int	

API — md2.c

Table 718. B-Con_crypto-algorithms/md2.c — 함수 (4개)

함수	반환형	받는 파라미터
md2_transform	void	MD2_CTX *ctx, BYTE data[]
md2_init	void	MD2_CTX *ctx
md2_update	void	MD2_CTX *ctx, const BYTE data[], size_t len
md2_final	void	MD2_CTX *ctx, BYTE hash[]

API — md2.h

Table 719. B-Con_crypto-algorithms/md2.h — typedef / using (1개)

새 이름	원래 형
BYTE	unsigned char

Table 720. B-Con_crypto-algorithms/md2.h — #define / `define 매크로 (2개)

매크로	인자	정의
MD2_H		/***** HEADER FILES *****/
MD2_BLOCK_SIZE		16

API — md2_test.c

Table 721. B-Con_crypto-algorithms/md2_test.c — 함수 (2개)

함수	반환형	받는 파라미터
md2_test	int	
main	int	

API — md5.c

Table 722. B-Con_crypto-algorithms/md5.c — 함수 (4개)

함수	반환형	받는 파라미터
md5_transform	void	MD5_CTX *ctx, const BYTE data[]
md5_init	void	MD5_CTX *ctx
md5_update	void	MD5_CTX *ctx, const BYTE data[], size_t len
md5_final	void	MD5_CTX *ctx, BYTE hash[]

Table 723. B-Con_crypto-algorithms/md5.c — #define / `define 매크로 (9개)

매크로	인자	정의
ROTLLEFT	(a,b)	((a << b) (a >> (32-b)))

매크로	인자	정의
F	(x,y,z)	$((x \& y) \mid (\sim x \& z))$
G	(x,y,z)	$((x \& z) \mid (y \& \sim z))$
H	(x,y,z)	$(x \wedge y \wedge z)$
I	(x,y,z)	$(y \wedge (x \mid \sim z))$
FF	(a,b,c,d,m,s,t)	{ a += F(b,c,d) + m + t; \
GG	(a,b,c,d,m,s,t)	{ a += G(b,c,d) + m + t; \
HH	(a,b,c,d,m,s,t)	{ a += H(b,c,d) + m + t; \
II	(a,b,c,d,m,s,t)	{ a += I(b,c,d) + m + t; \

API — md5.h

Table 724. B-Con_crypto-algorithms/md5.h — typedef / using (2개)

새 이름	원래 형
BYTE	unsigned char
WORD	unsigned int

Table 725. B-Con_crypto-algorithms/md5.h — #define / `define 매크로 (2개)

매크로	인자	정의
MD5_H		/***** HEADER FILES *****/
MD5_BLOCK_SIZE		16 // MD5 outputs a 16 byte digest

API — md5_test.c

Table 726. B-Con_crypto-algorithms/md5_test.c — 함수 (2개)

함수	반환형	받는 파라미터
md5_test	int	
main	int	

API — rot-13.c

Table 727. B-Con_crypto-algorithms/rot-13.c — 함수 (1개)

함수	반환형	받는 파라미터
rot13	void	char str[]

API — rot-13.h

Table 728. B-Con_crypto-algorithms/rot-13.h — #define / `define 매크로 (1개)

매크로	인자	정의
ROT13_H		/***** HEADER FILES *****/

API — rot-13_test.c**Table 729. B-Con_crypto-algorithms/rot-13_test.c — 함수 (2개)**

함수	반환형	받는 파라미터
rot13_test	int	
main	int	

API — sha1.c**Table 730. B-Con_crypto-algorithms/sha1.c — 함수 (4개)**

함수	반환형	받는 파라미터
sha1_transform	void	SHA1_CTX *ctx, const BYTE data[]
sha1_init	void	SHA1_CTX *ctx
sha1_update	void	SHA1_CTX *ctx, const BYTE data[], size_t len
sha1_final	void	SHA1_CTX *ctx, BYTE hash[]

Table 731. B-Con_crypto-algorithms/sha1.c — #define / `define 매크로 (1개)

매크로	인자	정의
ROTLLEFT	(a, b)	((a << b) (a >> (32 - b)))

API — sha1.h**Table 732. B-Con_crypto-algorithms/sha1.h — typedef / using (2개)**

새 이름	원래 형
BYTE	unsigned char
WORD	unsigned int

Table 733. B-Con_crypto-algorithms/sha1.h — #define / `define 매크로 (2개)

매크로	인자	정의
SHA1_H		/***** HEADER FILES *****/
SHA1_BLOCK_SIZE		20 // SHA1 outputs a 20 byte digest

API — sha1_test.c**Table 734. B-Con_crypto-algorithms/sha1_test.c — 함수 (2개)**

함수	반환형	받는 파라미터
sha1_test	int	
main	int	

API — sha256.c**Table 735. B-Con_crypto-algorithms/sha256.c — 함수 (4개)**

함수	반환형	받는 파라미터
sha256_transform	void	SHA256_CTX *ctx, const BYTE data[]

함수	반환형	받는 파라미터
sha256_init	void	SHA256_CTX *ctx
sha256_update	void	SHA256_CTX *ctx, const BYTE data[], size_t len
sha256_final	void	SHA256_CTX *ctx, BYTE hash[]

Table 736. B-Con_crypto-algorithms/sha256.c — #define / `define 매크로 (8개)

매크로	인자	정의
ROTLLEFT	(a,b)	$((a) \ll (b)) \mid ((a) \gg (32-(b)))$
ROTRIGHT	(a,b)	$((a) \gg (b)) \mid ((a) \ll (32-(b)))$
CH	(x,y,z)	$((x) \& (y)) \wedge \neg(x) \& (z))$
MAJ	(x,y,z)	$((x) \& (y)) \wedge ((x) \& (z)) \wedge ((y) \& (z))$
EP0	(x)	$(\text{ROTRIGHT}(x,2) \wedge \text{ROTRIGHT}(x,13) \wedge \text{ROTRIGHT}(x,22))$
EP1	(x)	$(\text{ROTRIGHT}(x,6) \wedge \text{ROTRIGHT}(x,11) \wedge \text{ROTRIGHT}(x,25))$
SIG0	(x)	$(\text{ROTRIGHT}(x,7) \wedge \text{ROTRIGHT}(x,18) \wedge ((x) \gg 3))$
SIG1	(x)	$(\text{ROTRIGHT}(x,17) \wedge \text{ROTRIGHT}(x,19) \wedge ((x) \gg 10))$

API — sha256.h

Table 737. B-Con_crypto-algorithms/sha256.h — typedef / using (2개)

새 이름	원래 형
BYTE	unsigned char
WORD	unsigned int

Table 738. B-Con_crypto-algorithms/sha256.h — #define / `define 매크로 (2개)

매크로	인자	정의
SHA256_H		/***** HEADER FILES *****/
SHA256_BLOCK_SIZE		32 // SHA256 outputs a 32 byte digest

API — sha256_test.c

Table 739. B-Con_crypto-algorithms/sha256_test.c — 함수 (2개)

함수	반환형	받는 파라미터
sha256_test	int	
main	int	

1. B-Con_crypto-algorithms/aes.c (1,095 lines)

```

1 /
*****
2 * Filename:   aes.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   This code is the implementation of the AES al-
gorithm and
7             the CTR, CBC, and CCM modes of operation it can
be used in.

```

```

8             AES is, specified by the NIST in in publication
FIPS PUB 197,
9             available at:
10            * http://csrc.nist.gov/publications/fips/
fips197/fips-197.pdf .
11            The CBC and CTR modes of operation are specified by
12            NIST SP 800-38 A, available at:
13            * http://csrc.nist.gov/publications/nistpubs/
800-38a/sp800-38a.pdf .
14            The CCM mode of operation is specified by NIST
SP800-38 C, available at:
15            * http://csrc.nist.gov/publications/nistpubs/
800-38C/SP800-38C_updated-July20_2007.pdf
16

```



```

*****
*/
17
18 /***** HEADER FILES
*****
19 #include <stdlib.h>
20 #include <memory.h>
21 #include "aes.h"
22
23 #include <stdio.h>
24
25 /***** MACROS
*****
26 // The least significant byte of the word is rotated to the
end.
27 #define KE_ROTWORD(x) (((x) << 8) | ((x) >> 24))
28
29 #define TRUE 1
30 #define FALSE 0
31
32 /***** DATA TYPES
*****
33 #define AES_128_ROUNDS 10
34 #define AES_192_ROUNDS 12
35 #define AES_256_ROUNDS 14
36
37 /***** FUNCTION DECLARATIONS
*****
38 void ccm_prepare_first_ctr_blk(BYTE counter[], const BYTE
nonce[], int nonce_len, int payload_len_store_size);
39 void ccm_prepare_first_format_blk(BYTE buf[], int assoc_len,
int payload_len, int payload_len_store_size, int mac_len, const BYTE
nonce[], int nonce_len);
40 void ccm_format_assoc_data(BYTE buf[], int *end_of_buf, const
BYTE assoc[], int assoc_len);
41 void ccm_format_payload_data(BYTE buf[], int *end_of_buf,
const BYTE payload[], int payload_len);
42
43 /***** VARIABLES
*****
44 // This is the specified AES SBox. To look up a substitution
value, put the first
45 // nibble in the first index (row) and the second nibble in
the second index (column).
46 static const BYTE aes_sbox[16][16] = {
47     {0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F,
0xC5, 0x30, 0x01, 0x67, 0x2B, 0xFE, 0xD7, 0xAB, 0x76},
48     {0xCA, 0x82, 0xC9, 0x7D, 0xFA, 0x59, 0x47, 0xF0, 0xAD,
0xD4, 0xA2, 0xAF, 0x9C, 0xA4, 0x72, 0xC0},
49     {0xB7, 0xFD, 0x93, 0x26, 0x36, 0x3F, 0xF7, 0xCC,
0x34, 0xA5, 0xE5, 0xF1, 0x71, 0xD8, 0x31, 0x15},
50     {0x04, 0xC7, 0x23, 0xC3, 0x18, 0x96, 0x05, 0x9A,
0x07, 0x12, 0x80, 0xE2, 0xEB, 0x27, 0xB2, 0x75},
51     {0x09, 0x83, 0x2C, 0x1A, 0x1B, 0x6E, 0x5A, 0xA0, 0x52, 0x3B,
0xD6, 0xB3, 0x29, 0xE3, 0x2F, 0x84},
52     {0x53, 0xD1, 0x00, 0xED, 0x20, 0xFC, 0xB1, 0x5B, 0x6A, 0xCB,
0xBE, 0x39, 0x4A, 0x4C, 0x58, 0xCF},
53     {0xD0, 0xEF, 0xAA, 0xFB, 0x43, 0x4D,
0x33, 0x85, 0x45, 0xF9, 0x02, 0x7F, 0x50, 0x3C, 0x9F, 0xA8},
54     {0x51, 0xA3, 0x40, 0x8F, 0x92, 0x9D, 0x38, 0xF5, 0xBC,
0xB6, 0xDA, 0x21, 0x10, 0xFF, 0xF3, 0xD2},
55     {0xCD, 0x0C, 0x13, 0xEC, 0x5F,
0x97, 0x44, 0x17, 0xC4, 0xA7, 0x7E, 0x3D, 0x64, 0x5D, 0x19, 0x73},
56     {0x60, 0x81, 0x4F, 0xDC, 0x22, 0x2A, 0x90, 0x88, 0x46, 0xEE,
0xB8, 0x14, 0xDE, 0x5E, 0xB0, 0xDB},
57     {0xE0, 0x32, 0x3A, 0x0A, 0x49, 0x06, 0x24, 0x5C,
0xC2, 0xD3, 0xAC, 0x62, 0x91, 0x95, 0xE4, 0x79},
58     {0xE7, 0xC8, 0x37, 0x6D, 0x80, 0xD5, 0x4E, 0xA9, 0x6C,
0x56, 0xF4, 0xEA, 0x65, 0x7A, 0xAE, 0x08},
59     {0xBA, 0x78, 0x25, 0x2E, 0x1C, 0xA6, 0xB4, 0xC6, 0xE8, 0xDD,
0x74, 0x1F, 0x4B, 0xBD, 0x8B, 0x8A},
60     {0x70, 0x3E, 0xB5, 0x66, 0x48, 0x03, 0xF6, 0x0E,
0x61, 0x35, 0x57, 0xB9, 0x86, 0xC1, 0x1D, 0x9E},
61     {0xE1, 0xF8, 0x98, 0x11, 0x69, 0xD9, 0x8E, 0x94, 0x9B, 0x1E,
0x87, 0xE9, 0xCE, 0x55, 0x28, 0xDF},
62     {0x8C, 0xA1, 0x89, 0x0D, 0xBF,
0xE6, 0x42, 0x68, 0x41, 0x99, 0x2D, 0x0F, 0xB0, 0x54, 0xBB, 0x16}
63 };
64
65 static const BYTE aes_invbox[16][16] = {
66     {0x52, 0x09, 0x6A, 0xD5, 0x30, 0x36, 0xA5, 0x38, 0xBF,
0x40, 0xA3, 0x9E, 0x81, 0xF3, 0xD7, 0xFB},
67     {0x7C, 0xE3, 0x39, 0x82, 0x9B, 0x2F, 0xFF, 0x87, 0x34, 0x8E,
0x43, 0x44, 0xC4, 0xDE, 0xE9, 0xCB},
68     {0x54, 0x7B, 0x94, 0x32, 0xA6, 0xC2, 0x23, 0x3D, 0xEE, 0x4C,
0x95, 0x0B, 0x42, 0xFA, 0xC3, 0x4E},
69     {0x08, 0x2E, 0xA1, 0x66, 0x28, 0xD9, 0x24, 0xB2, 0x76, 0x5B,
0xA2, 0x49, 0x6D, 0x88, 0xD1, 0x25},
70
71     {0x72, 0xF8, 0xF6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xD4, 0xA4, 0x5C, 0xCC, 0x5D,
0x65, 0xB6, 0x92},
72     {0x6C, 0x70, 0x48, 0x50, 0xFD, 0xED, 0xB9, 0xDA, 0x5E,
0x15, 0x46, 0x57, 0xA7, 0x8D, 0x9D, 0x84},
73     {0x90, 0xD8, 0xAB, 0x00, 0x8C, 0xBC, 0xD3, 0x0A,
0xF7, 0xE4, 0x58, 0x05, 0xB8, 0xB3, 0x45, 0x06},
74     {0xD0, 0x2C, 0x1E, 0x8F, 0xCA, 0x3F, 0x0F, 0x02, 0xC1, 0xAF,
0xBD, 0x03, 0x01, 0x13, 0x8A, 0x6B},
75     {0x3A, 0x91, 0x11, 0x41, 0x4F, 0x67, 0xDC, 0xEA,
0x97, 0xF2, 0xCF, 0xCE, 0xF0, 0xB4, 0xE6, 0x73},
76     {0x96, 0xAC, 0x74, 0x22, 0xE7, 0xAD,
0x35, 0x85, 0xE2, 0xF9, 0x37, 0xE8, 0x1C, 0x75, 0xDF, 0x6E},

```

```

76     {0x47, 0xF1, 0x1A, 0x71, 0x1D, 0x29, 0xC5, 0x89, 0x6F,
0xB7, 0x62, 0x0E, 0xAA, 0x18, 0xBE, 0x1B},
77     {0xFC, 0x56, 0x3E, 0x4B, 0xC6, 0xD2, 0x79, 0x20, 0x9A, 0xDB,
0xC0, 0xFE, 0x78, 0xCD, 0x5A, 0xF4},
78     {0x1F, 0xDD,
0xA8, 0x33, 0x88, 0x07, 0xC7, 0x31, 0xB1, 0x12, 0x10, 0x59, 0x27, 0x80, 0xEC,
0x5F},
79     {0x60, 0x51, 0x7F, 0xA9, 0x19, 0xB5, 0x4A, 0x0D, 0x2D,
0xE5, 0x7A, 0x9F, 0x93, 0xC9, 0x9C, 0xEF},
80     {0xA0, 0xE0, 0x3B, 0x4D, 0xAE, 0x2A, 0xF5, 0xB0, 0xC8, 0xEB,
0xBB, 0x3C, 0x83, 0x53, 0x99, 0x61},
81     {0x17, 0x2B, 0x04, 0x7E, 0xBA,
0x77, 0xD6, 0x26, 0xE1, 0x69, 0x14, 0x63, 0x55, 0x21, 0x0C, 0x7D}
82 };
83
84 // This table stores pre-calculated values for all possible
GF(2^8) calculations. This
85 // table is only used by the (Inv)MixColumns steps.
86 // USAGE: The second index (column) is the coefficient of mul-
tiplication. Only 7 different
87 // coefficients are used: 0x01, 0x02, 0x03, 0x09, 0x0b, 0x0d,
0x0e, but multiplication by
88 // 1 is negligible leaving only 6 coefficients. Each column of
the table is devoted to one
89 // of these coefficients, in the ascending order of value,
from values 0x00 to 0xFF.
90 static const BYTE gf_mul[256][6] = {
91     {0x00, 0x00, 0x00, 0x00, 0x00, 0x00}, {0x02, 0x03, 0x09, 0x0b,
0x0d, 0x0e},
92     {0x04, 0x06, 0x12, 0x16, 0x1a, 0x1c}, {0x06, 0x05, 0x1b, 0x1d,
0x17, 0x12},
93     {0x08, 0x0c, 0x24, 0x2c, 0x34, 0x38}, {0x0a, 0x0f, 0x2d,
0x27, 0x39, 0x36},
94     {0x0c, 0x0a, 0x36, 0x3a, 0x2e, 0x24}, {0x0e, 0x09, 0x3f,
0x31, 0x23, 0x2a},
95     {0x10, 0x18, 0x48, 0x58, 0x68, 0x70}, {0x12, 0x1b,
0x41, 0x53, 0x65, 0x7e},
96     {0x14, 0x1e, 0x5a, 0x4e, 0x72, 0x6c}, {0x16, 0x1d,
0x53, 0x45, 0x7f, 0x62},
97     {0x18, 0x14, 0x6c, 0x74, 0x5c, 0x48}, {0x1a, 0x17, 0x65, 0x7f,
0x51, 0x46},
98     {0x1c, 0x12, 0x7e, 0x62, 0x46, 0x54}, {0x1e,
0x11, 0x77, 0x69, 0x4b, 0x5a},
99     {0x20, 0x30, 0x90, 0xb0, 0xd0, 0xe0}, {0x22, 0x33, 0x99, 0xbb,
0xdd, 0xee},
100    {0x24, 0x36, 0x82, 0xa6, 0xca, 0xfc}, {0x26, 0x35, 0x8b, 0xad,
0xc7, 0xf2},
101    {0x28, 0x3c, 0xb4, 0x9c, 0xe4, 0xd8}, {0x2a, 0x3f, 0xbd,
0x97, 0xe9, 0xd6},
102    {0x2c, 0x3a, 0xa6, 0x8a, 0xfe, 0xc4}, {0x2e, 0x39, 0xaf,
0x81, 0xf3, 0xca},
103    {0x30, 0x28, 0xd8, 0xe8, 0xb8, 0x90}, {0x32, 0x2b,
0xd1, 0xe3, 0xb5, 0x9e},
104    {0x34, 0x2e, 0xca, 0xfe, 0xa2, 0x8c}, {0x36, 0x2d,
0xc3, 0xf5, 0xaf, 0x82},
105    {0x38, 0x24, 0xfc, 0xc4, 0x8c, 0xa8}, {0x3a, 0x27, 0xf5, 0xcf,
0x81, 0xa6},
106    {0x3c, 0x22, 0xee, 0xd2, 0x96, 0xb4}, {0x3e,
0x21, 0xe7, 0xd9, 0x9b, 0xba},
107    {0x40, 0x60, 0x3b, 0x7b, 0xbb, 0xdb},
108    {0x42, 0x63, 0x32, 0x70, 0xb6, 0xd5},
109    {0x44, 0x66, 0x29, 0x6d, 0xa1, 0xc7},
110    {0x46, 0x65, 0x20, 0x66, 0xac, 0xc9},
111    {0x48, 0x6c, 0x1f, 0x57, 0x8f, 0xe3}, {0x4a, 0x6f, 0x16, 0x5c,
0x82, 0xed},
112    {0x4c, 0x6a, 0x0d, 0x41, 0x95, 0xff}, {0x4e, 0x69, 0x04, 0x4a,
0x98, 0xf1},
113    {0x50, 0x78, 0x73, 0x23, 0xd3, 0xab}, {0x52, 0x7b, 0x7a,
0x28, 0xde, 0xa5},
114    {0x54, 0x7e, 0x61, 0x35, 0xc9, 0xb7}, {0x56, 0x7d, 0x68, 0x3e,
0xc4, 0xb9},
115    {0x58, 0x74, 0x57, 0x0f, 0xe7, 0x93}, {0x5a, 0x77, 0x5e,
0x04, 0xea, 0x9d},
116    {0x5c, 0x72, 0x45, 0x19, 0xfd, 0x8f}, {0x5e, 0x71, 0x4c,
0x12, 0xf0, 0x81},
117    {0x60, 0x50, 0xab, 0xcb, 0x6b, 0x3b},
118    {0x62, 0x53, 0xa2, 0xc0, 0x66, 0x35},
119    {0x64, 0x56, 0xb9, 0xdd, 0x71, 0x27},
120    {0x66, 0x55, 0xb0, 0xd6, 0x7c, 0x29},
121    {0x68, 0x5c, 0x8f, 0xe7, 0x5f, 0x03}, {0x6a, 0x5f, 0x86, 0xec,
0x52, 0x0d},
122    {0x6c, 0x5a, 0x9d, 0xf1, 0x45, 0x1f}, {0x6e, 0x59, 0x94, 0xfa,
0x48, 0x11},
123    {0x70, 0x48, 0xe3, 0x93, 0x03, 0x4b}, {0x72, 0x4b, 0xea,
0x98, 0x0e, 0x45},
124    {0x74, 0x4e, 0xf1, 0x85, 0x19, 0x57}, {0x76, 0x4d, 0xf8, 0x8e,
0x14, 0x59},
125    {0x78, 0x44, 0xc7, 0xbf, 0x37, 0x73}, {0x7a, 0x47, 0xce,
0xb4, 0x3a, 0x7d},
126    {0x7c, 0x42, 0xd5, 0xa9, 0x2d, 0x6f}, {0x7e, 0x41, 0xdc,
0xa2, 0x20, 0x61},
127    {0x80, 0xc0, 0x76, 0xf6, 0x6d, 0xad}, {0x82, 0xc3, 0x7f, 0xfd,
0x00, 0xa3},
128    {0x84, 0xc6, 0x64, 0xe0, 0x77, 0xb1}, {0x86, 0xc5, 0x6d, 0xeb,
0x7a, 0xbf},
129    {0x88, 0xcc, 0x52, 0xda, 0x59, 0x95}, {0x8a, 0xcf, 0x5b,
0xd1, 0x54, 0x9b},
130    {0x8c, 0xca, 0x40, 0xcc, 0x43, 0x89}, {0x8e,
0xc9, 0x49, 0xc7, 0x4e, 0x87},
131    {0x90, 0xd8, 0x3e, 0xae, 0x05, 0xdd}, {0x92, 0xdb,

```

```

0x37, 0xa5, 0x08, 0xd3},
128 {0x94, 0xde, 0x2c, 0xb8, 0x1f, 0xc1}, {0x96, 0xdd,
0x25, 0xb3, 0x12, 0xc0},
129 {0x98, 0xd4, 0x1a, 0x82, 0x31, 0xe5}, {0x9a,
0xd7, 0x13, 0x89, 0x3c, 0xeb},
130 {0x9c, 0xd2, 0x08, 0x94, 0x2b, 0xf9}, {0x9e, 0xd1, 0x01, 0x9f,
0x26, 0xf7},
131 {0xa0, 0xf0, 0xe6, 0x46, 0xbd, 0x4d}, {0xa2, 0xf3, 0xef, 0x4d,
0xb0, 0x43},
132 {0xa4, 0xf6, 0xf4, 0x50, 0xa7, 0x51}, {0xa6, 0xf5, 0xfd, 0x5b,
0xaa, 0x5f},
133 {0xa8, 0xfc, 0xc2, 0x6a, 0x89, 0x75}, {0xaa, 0xff, 0xcb,
0x61, 0x84, 0x7b},
134 {0xac, 0xfa, 0xd0, 0x7c, 0x93, 0x69}, {0xae,
0xf9, 0xd9, 0x77, 0x9e, 0x67},
135 {0xb0, 0xe8, 0xae, 0x1e, 0xd5, 0x3d}, {0xb2, 0xeb,
0xa7, 0x15, 0xd8, 0x33},
136 {0xb4, 0xee, 0xbc, 0x08, 0xcf, 0x21}, {0xb6, 0xed,
0xb5, 0x03, 0xc2, 0x2f},
137 {0xb8, 0xe4, 0x8a, 0x32, 0xe1, 0x05}, {0xba,
0xe7, 0x83, 0x39, 0xec, 0x0b},
138 {0xbc, 0xe2, 0x98, 0x24, 0xfb, 0x19}, {0xbe, 0xe1, 0x91, 0x2f,
0xf6, 0x17},
139 {0xc0, 0xa0, 0x4d, 0x8d, 0xd6, 0x76},
{0xc2, 0xa3, 0x44, 0x86, 0xdb, 0x78},
140 {0xc4, 0xa6, 0x5f, 0x9b, 0xcc, 0x6a},
{0xc6, 0xa5, 0x56, 0x90, 0xc1, 0x64},
141 {0xc8, 0xac, 0x69, 0xa1, 0xe2, 0x4e}, {0xca, 0xaf, 0x60, 0xaa,
0xef, 0x40},
142 {0xcc, 0xaa, 0x7b, 0xb7, 0xf8, 0x52}, {0xce, 0xa9, 0x72, 0xbc,
0xf5, 0x5c},
143 {0xd0, 0xb8, 0x05, 0xd5, 0xbe, 0x06}, {0xd2, 0xbb, 0x0c, 0xde,
0xb3, 0x08},
144 {0xd4, 0xbe, 0x17, 0xc3, 0xa4, 0x1a}, {0xd6, 0xbd, 0x1e,
0xc8, 0xa9, 0x14},
145 {0xd8, 0xb4, 0x21, 0xf9, 0x8a, 0x3e}, {0xda,
0xb7, 0x28, 0xf2, 0x87, 0x30},
146 {0xdc, 0xb2, 0x33, 0xef, 0x90, 0x22}, {0xde, 0xb1, 0x3a,
0xe4, 0x9d, 0x2c},
147 {0xe0, 0x90, 0xdd, 0x3d, 0x06, 0x96},
{0xe2, 0x93, 0xd4, 0x36, 0x0b, 0x98},
148 {0xe4, 0x96, 0xcf, 0x2b, 0x1c, 0x8a},
{0xe6, 0x95, 0xc6, 0x20, 0x11, 0x84},
149 {0xe8, 0x9c, 0xf9, 0x11, 0x32, 0xae}, {0xea, 0x9f, 0xf0, 0x1a,
0x3f, 0xa0},
150 {0xec, 0x9a, 0xeb, 0x07, 0x28, 0xb2}, {0xee, 0x99, 0xe2, 0x0c,
0x25, 0xbc},
151 {0xf0, 0x88, 0x95, 0x65, 0x6e, 0xe6}, {0xf2, 0x8b, 0x9c, 0x6e,
0x63, 0xe8},
152 {0xf4, 0x8e, 0x87, 0x73, 0xf4, 0xfa}, {0xf6, 0x8d, 0x8e,
0x78, 0x79, 0xf4},
153 {0xf8, 0x84, 0xb1, 0x49, 0x5a, 0xde}, {0xfa,
0x87, 0xb8, 0x42, 0x57, 0xd0},
154 {0xfc, 0x82, 0xa3, 0x5f, 0x40, 0xc2}, {0xfe, 0x81, 0xaa,
0x54, 0x4d, 0xcc},
155 {0x1b, 0x9b, 0xec, 0xf7, 0xda, 0x41}, {0x19, 0x98, 0xe5, 0xfc,
0xd7, 0x4f},
156 {0x1f, 0x9d, 0xfe, 0xe1, 0xc0, 0x5d}, {0x1d, 0x9e, 0xf7, 0xea,
0xcd, 0x53},
157 {0x13, 0x97, 0xc8, 0xdb, 0xee, 0x79},
{0x11, 0x94, 0xc1, 0xd0, 0xe3, 0x77},
158 {0x17, 0x91, 0xda, 0xcd, 0xf4, 0x65},
{0x15, 0x92, 0xd3, 0xc6, 0xf9, 0x6b},
159 {0x0b, 0x83, 0xa4, 0xaf, 0xb2, 0x31}, {0x09, 0x80, 0xad,
0xa4, 0xbf, 0x3f},
160 {0x0f, 0x85, 0xb6, 0xb9, 0xa8, 0x2d}, {0x0d, 0x86, 0xbf,
0xb2, 0xa5, 0x23},
161 {0x03, 0x8f, 0x80, 0x83, 0x86, 0x09}, {0x01, 0x8c,
0x89, 0x88, 0x8b, 0x07},
162 {0x07, 0x89, 0x92, 0x95, 0x9c, 0x15}, {0x05, 0x8a, 0x9b, 0x9e,
0x91, 0x1b},
163 {0x3b, 0xab, 0x7c, 0x47, 0x0a, 0xa1}, {0x39, 0xa8, 0x75, 0x4c,
0x07, 0xaf},
164 {0x3f, 0xad, 0x6e, 0x51, 0x10, 0xbd}, {0x3d, 0xae, 0x67, 0x5a,
0x1d, 0xb3},
165 {0x33, 0xa7, 0x58, 0x6b, 0x3e, 0x99},
{0x31, 0xa4, 0x51, 0x60, 0x33, 0x97},
166 {0x37, 0xa1, 0x4a, 0x7d, 0x24, 0x85},
{0x35, 0xa2, 0x43, 0x76, 0x29, 0x8b},
167 {0x2b, 0xb3, 0x34, 0x1f, 0x62, 0xd1}, {0x29, 0xb0, 0x3d,
0x14, 0x6f, 0xdf},
168 {0x2f, 0xb5, 0x26, 0x09, 0x78, 0xcd}, {0x2d, 0xb6, 0x2f,
0x02, 0x75, 0xc3},
169 {0x23, 0xbf, 0x10, 0x33, 0x56, 0xe9}, {0x21, 0xbc,
0x19, 0x38, 0x5b, 0xe7},
170 {0x27, 0xb9, 0x02, 0x25, 0x4c, 0xf5}, {0x25, 0xba, 0x0b, 0x2e,
0x41, 0xfb},
171 {0x5b, 0xfb, 0xd7, 0x8c, 0x61, 0x9a}, {0x59, 0xf8, 0xde,
0x87, 0x6c, 0x94},
172 {0x5f, 0xfd, 0xc5, 0x9a, 0x7b, 0x86}, {0x5d, 0xfe, 0xcc,
0x91, 0x76, 0x88},
173 {0x53, 0xf7, 0xf3, 0xa0, 0x55, 0xa2}, {0x51, 0xf4, 0xfa, 0xab,
0x58, 0xac},
174 {0x57, 0xf1, 0xe1, 0xb6, 0x4f, 0xbe}, {0x55, 0xf2, 0xe8, 0xbd,
0x42, 0xb0},
175 {0x4b, 0xe3, 0x9f, 0xd4, 0x09, 0xea}, {0x49, 0xe0, 0x96, 0xdf,
0x04, 0xe4},
176 {0x4f, 0xe5, 0x8d, 0xc2, 0x13, 0xf6}, {0x4d,
0xe6, 0x84, 0xc9, 0x1e, 0xf8},
177 {0x43, 0xef, 0xbb, 0xf8, 0x3d, 0xd2}, {0x41, 0xec,
0xb2, 0xf3, 0x30, 0xdc},

```

```

178 {0x47, 0xe9, 0xa9, 0xee, 0x27, 0xce}, {0x45, 0xea,
0xa0, 0xe5, 0x2a, 0xc0},
179 {0x7b, 0xcb, 0x47, 0x3c, 0xb1, 0x7a}, {0x79, 0xc8, 0x4e,
0x37, 0xbc, 0x74},
180 {0x7f, 0xcd, 0x55, 0x2a, 0xab, 0x66}, {0x7d, 0xce, 0x5c,
0x21, 0xa6, 0x68},
181 {0x73, 0xc7, 0x63, 0x10, 0x85, 0x42}, {0x71, 0xc4, 0x6a, 0x1b,
0x88, 0x4c},
182 {0x77, 0xc1, 0x71, 0x06, 0x9f, 0x5e}, {0x75, 0xc2, 0x78, 0x0d,
0x92, 0x50},
183 {0x6b, 0xd3, 0x0f, 0x64, 0xd9, 0x0a}, {0x69, 0xd0, 0x06, 0x6f,
0xd4, 0x04},
184 {0x6f, 0xd5, 0x1d, 0x72, 0xc3, 0x16}, {0x6d,
0xd6, 0x14, 0x79, 0xce, 0x18},
185 {0x63, 0xdf, 0x2b, 0x48, 0xed, 0x32}, {0x61, 0xdc,
0x22, 0x43, 0xe0, 0x3c},
186 {0x67, 0xd9, 0x39, 0x5e, 0xf7, 0x2e}, {0x65, 0xda,
0x30, 0x55, 0xfa, 0x20},
187 {0x9b, 0x5b, 0x9a, 0x01, 0xb7, 0xec}, {0x99, 0x58, 0x93, 0x0a,
0xba, 0xe2},
188 {0x9f, 0x5d, 0x88, 0x17, 0xad, 0xf0}, {0x9d, 0x5e, 0x81, 0x1c,
0xa0, 0xfe},
189 {0x93, 0x57, 0xbe, 0x2d, 0x83, 0xd4},
{0x91, 0x54, 0xb7, 0x26, 0x8e, 0xda},
190 {0x97, 0x51, 0xac, 0x3b, 0x99, 0xc8},
{0x95, 0x52, 0xa5, 0x30, 0x94, 0xc6},
191 {0x8b, 0x43, 0xd2, 0x59, 0xdf, 0x9c}, {0x89, 0x40, 0xdb,
0x52, 0xd2, 0x92},
192 {0x8f, 0x45, 0xc0, 0x4f, 0xc5, 0x80}, {0x8d,
0x46, 0xc9, 0x44, 0xc8, 0x8e},
193 {0x83, 0x4f, 0xf6, 0x75, 0xeb, 0xa4}, {0x81, 0x4c, 0xff, 0x7e,
0xe6, 0xaa},
194 {0x87, 0x49, 0xe4, 0x63, 0xf1, 0xb8}, {0x85, 0x4a, 0xed,
0xe8, 0xfc, 0xb6},
195 {0xbb, 0x6b, 0x0a, 0xb1, 0x67, 0x0c}, {0xb9, 0x68, 0x03, 0xba,
0x6a, 0x02},
196 {0xbf, 0x6d, 0x18, 0xa7, 0x7d, 0x10}, {0xbd, 0x6e, 0x11, 0xac,
0x70, 0x1e},
197 {0xb3, 0x67, 0x2e, 0x9d, 0x53, 0x34},
{0xb1, 0x64, 0x27, 0x96, 0x5e, 0x3a},
198 {0xb7, 0x61, 0x3c, 0x8b, 0x49, 0x28},
{0xb5, 0x62, 0x35, 0x80, 0x44, 0x26},
199 {0xab, 0x73, 0x42, 0xe9, 0xf7, 0xc7}, {0xa9, 0x70, 0x4b,
0xe2, 0x02, 0x72},
200 {0xaf, 0x75, 0x50, 0xff, 0x15, 0x60}, {0xad,
0x76, 0x59, 0xf4, 0x18, 0x6e},
201 {0xa3, 0x7f, 0x66, 0xc5, 0x3b, 0x44}, {0xa1, 0x7c, 0x6f, 0xce,
0x36, 0x4a},
202 {0xa7, 0x79, 0x74, 0xd3, 0x21, 0x58}, {0xa5, 0x7a, 0x7d,
0xd8, 0x2c, 0x56},
203 {0xdb, 0x3b, 0xa1, 0x7a, 0x0c, 0x37},
{0xd9, 0x38, 0xa8, 0x71, 0x01, 0x39},
204 {0xdf, 0x3d, 0xb3, 0x6c, 0x16, 0x2b}, {0xdd, 0x3e, 0xba,
0x67, 0x1b, 0x25},
205 {0xd3, 0x37, 0x85, 0x56, 0x38, 0xf0}, {0xd1, 0x34, 0x8c, 0x5d,
0x35, 0x01},
206 {0xd7, 0x31, 0x97, 0x40, 0x22, 0x13}, {0xd5, 0x32, 0x9e, 0x4b,
0x2f, 0x1d},
207 {0xcb, 0x23, 0xe9, 0x22, 0x64, 0x47},
{0xc9, 0x20, 0xe0, 0x29, 0x69, 0x49},
208 {0xcf, 0x25, 0xfb, 0x34, 0x7e, 0x5b}, {0xcd, 0x26, 0xf2, 0x3f,
0x73, 0x55},
209 {0xc3, 0x2f, 0xcd, 0x0e, 0x50, 0x7f}, {0xc1, 0x2c,
0xc4, 0x05, 0x5d, 0x71},
210 {0xc7, 0x29, 0xdf, 0x18, 0x4a, 0x63}, {0xc5, 0x2a,
0xd6, 0x13, 0x47, 0x6d},
211 {0xfb, 0x0b, 0x31, 0xca, 0xdc, 0xd7},
{0xf9, 0x08, 0x38, 0xc1, 0xd1, 0xd9},
212 {0xff, 0x0d, 0x23, 0xdc, 0xc6, 0xcb}, {0xfd, 0x0e, 0x2a,
0xd7, 0xcb, 0xc5},
213 {0xf3, 0x07, 0x15, 0xe6, 0xe8, 0xef}, {0xf1, 0x04, 0x1c, 0xed,
0xe5, 0xe1},
214 {0xf7, 0x01, 0x07, 0xf0, 0xf2, 0xf3}, {0xf5, 0x02, 0x0e, 0xfb,
0xff, 0xfd},
215 {0xeb, 0x13, 0x79, 0x92, 0xb4, 0xa7},
{0xe9, 0x10, 0x70, 0x99, 0xb9, 0xa9},
216 {0xef, 0x15, 0x6b, 0x84, 0xae, 0xbb}, {0xed, 0x16, 0x62, 0x8f,
0xa3, 0xb5},
217 {0xe3, 0x1f, 0x5d, 0xbe, 0x80, 0x9f}, {0xe1, 0x1c,
0x54, 0xb5, 0x8d, 0x91},
218 {0xe7, 0x19, 0x4f, 0xa8, 0x9a, 0x83}, {0xe5, 0x1a,
0x46, 0xa3, 0x97, 0x8d}
219 };
220
221 /***** FUNCTION DEFINITIONS *****/
222 // XORs the in and out buffers, storing the result in out.
Length is in bytes.
223 void xor_buf(const BYTE in[], BYTE out[], size_t len)
224 {
225     size_t idx;
226     for (idx = 0; idx < len; idx++)
227         out[idx] ^= in[idx];
228 }
229
230 /*****
231 * AES - CBC
232 *****/
233
234 int aes_encrypt_cbc(const BYTE in[], size_t in_len, BYTE
out[], const WORD key[], int keysize, const BYTE iv[])

```

```

235 {
236     BYTE buf_in[AES_BLOCK_SIZE], buf_out[AES_BLOCK_SIZE],
iv_buf[AES_BLOCK_SIZE];
237     int blocks, idx;
238
239     if (in_len % AES_BLOCK_SIZE != 0)
240         return(FALSE);
241
242     blocks = in_len / AES_BLOCK_SIZE;
243
244     memcpy(iv_buf, iv, AES_BLOCK_SIZE);
245
246     for (idx = 0; idx < blocks; idx++) {
247         memcpy(buf_in, &in[idx * AES_BLOCK_SIZE],
AES_BLOCK_SIZE);
248         xor_buf(iv_buf, buf_in, AES_BLOCK_SIZE);
249         aes_encrypt(buf_in, buf_out, key, keysize);
250         memcpy(&out[idx * AES_BLOCK_SIZE], buf_out,
AES_BLOCK_SIZE);
251         memcpy(iv_buf, buf_out, AES_BLOCK_SIZE);
252     }
253
254     return(TRUE);
255 }
256
257 int aes_encrypt_cbc_mac(const BYTE in[], size_t in_len, BYTE
out[], const WORD key[], int keysize, const BYTE iv[])
258 {
259     BYTE buf_in[AES_BLOCK_SIZE], buf_out[AES_BLOCK_SIZE],
iv_buf[AES_BLOCK_SIZE];
260     int blocks, idx;
261
262     if (in_len % AES_BLOCK_SIZE != 0)
263         return(FALSE);
264
265     blocks = in_len / AES_BLOCK_SIZE;
266
267     memcpy(iv_buf, iv, AES_BLOCK_SIZE);
268
269     for (idx = 0; idx < blocks; idx++) {
270         memcpy(buf_in, &in[idx * AES_BLOCK_SIZE],
AES_BLOCK_SIZE);
271         xor_buf(iv_buf, buf_in, AES_BLOCK_SIZE);
272         aes_encrypt(buf_in, buf_out, key, keysize);
273         memcpy(iv_buf, buf_out, AES_BLOCK_SIZE);
274         // Do not output all encrypted blocks.
275     }
276
277     memcpy(out, buf_out, AES_BLOCK_SIZE); // Only output
the last block.
278
279     return(TRUE);
280 }
281
282 int aes_decrypt_cbc(const BYTE in[], size_t in_len, BYTE
out[], const WORD key[], int keysize, const BYTE iv[])
283 {
284     BYTE buf_in[AES_BLOCK_SIZE], buf_out[AES_BLOCK_SIZE],
iv_buf[AES_BLOCK_SIZE];
285     int blocks, idx;
286
287     if (in_len % AES_BLOCK_SIZE != 0)
288         return(FALSE);
289
290     blocks = in_len / AES_BLOCK_SIZE;
291
292     memcpy(iv_buf, iv, AES_BLOCK_SIZE);
293
294     for (idx = 0; idx < blocks; idx++) {
295         memcpy(buf_in, &in[idx * AES_BLOCK_SIZE],
AES_BLOCK_SIZE);
296         aes_decrypt(buf_in, buf_out, key, keysize);
297         xor_buf(iv_buf, buf_out, AES_BLOCK_SIZE);
298         memcpy(&out[idx * AES_BLOCK_SIZE], buf_out,
AES_BLOCK_SIZE);
299         memcpy(iv_buf, buf_in, AES_BLOCK_SIZE);
300     }
301
302     return(TRUE);
303 }
304
305 /*****
306 * AES - CTR
307 *****/
308 void increment_iv(BYTE iv[], int counter_size)
309 {
310     int idx;
311
312     // Use counter_size bytes at the end of the IV as the
big-endian
integer to increment.
313     for (idx = AES_BLOCK_SIZE - 1; idx >= AES_BLOCK_SIZE -
counter_size; idx--) {
314         iv[idx]++;
315         if (iv[idx] != 0 || idx == AES_BLOCK_SIZE -
counter_size)
316             break;
317     }
318 }
319
320 // Performs the encryption in-place, the input and output buf-
fers may be the same.

```

```

321 // Input may be an arbitrary length (in bytes).
322 void aes_encrypt_ctr(const BYTE in[], size_t in_len, BYTE
out[], const WORD key[], int keysize, const BYTE iv[])
323 {
324     size_t idx = 0, last_block_length;
325     BYTE iv_buf[AES_BLOCK_SIZE], out_buf[AES_BLOCK_SIZE];
326
327     if (in != out)
328         memcpy(out, in, in_len);
329
330     memcpy(iv_buf, iv, AES_BLOCK_SIZE);
331     last_block_length = in_len - AES_BLOCK_SIZE;
332
333     if (in_len > AES_BLOCK_SIZE) {
334         for (idx = 0; idx < last_block_length; idx +=
AES_BLOCK_SIZE) {
335             aes_encrypt(iv_buf, out_buf, key, key-
size);
336             xor_buf(out_buf, &out[idx], AES_B-
LOCK_SIZE);
337             increment_iv(iv_buf, AES_BLOCK_SIZE);
338         }
339     }
340
341     aes_encrypt(iv_buf, out_buf, key, keysize);
342
343     xor_buf(out_buf, &out[idx], in_len - idx); // Use the
Most Significant bytes.
344 }
345
346 void aes_decrypt_ctr(const BYTE in[], size_t in_len, BYTE
out[], const WORD key[], int keysize, const BYTE iv[])
347 {
348     // CTR encryption is its own inverse function.
349     aes_encrypt_ctr(in, in_len, out, key, keysize, iv);
350 }
351 /*****
352 * AES - CCM
353 *****/
354 // out_len = payload_len + assoc_len
355 int aes_encrypt_ccm(const BYTE payload[], WORD payload_len,
const BYTE assoc[], unsigned short assoc_len,
const BYTE nonce[], unsigned short
nonce_len, BYTE out[], WORD *out_len,
WORD mac_len, const BYTE key_str[], int
keysize)
356 {
357     BYTE temp_iv[AES_BLOCK_SIZE], counter[AES_BLOCK_SIZE],
mac[16], *buf;
358     int end_of_buf, payload_len_store_size;
359     WORD key[60];
360
361     if (mac_len != 4 && mac_len != 6 && mac_len != 8 &&
mac_len != 10 &&
362         mac_len != 12 && mac_len != 14 && mac_len != 16)
363         return(FALSE);
364
365     if (nonce_len < 7 || nonce_len > 13)
366         return(FALSE);
367
368     if (assoc_len > 32768 /* = 2^15 */)
369         return(FALSE);
370
371     buf = (BYTE*)malloc(payload_len + assoc_len + 48 /
*Round both payload and associated data up a block size and add an
extra block.*);
372     if (! buf)
373         return(FALSE);
374
375     // Prepare the key for usage.
376     aes_key_setup(key_str, key, keysize);
377
378     // Format the first block of the formatted data.
379     payload_len_store_size = AES_BLOCK_SIZE - 1 -
nonce_len;
380     ccm_prepare_first_format_blk(buf, assoc_len, pay-
load_len, payload_len_store_size, mac_len, nonce, nonce_len);
381     end_of_buf = AES_BLOCK_SIZE;
382
383     // Format the Associated Data, aka, assoc[.
384     ccm_format_assoc_data(buf, &end_of_buf, assoc, as-
soc_len);
385
386     // Format the Payload, aka payload[.
387     ccm_format_payload_data(buf, &end_of_buf, payload, pay-
load_len);
388
389     // Create the first counter block.
390     ccm_prepare_first_ctr_blk(counter, nonce, nonce_len,
payload_len_store_size);
391
392     // Perform the CBC operation with an IV of zeros on the
formatted buffer to calculate the MAC.
393     memset(temp_iv, 0, AES_BLOCK_SIZE);
394
395     aes_encrypt_cbc_mac(buf, end_of_buf, mac, key, keysize,
temp_iv);
396
397     // Copy the Payload and MAC to the output buffer.

```

```

399     memcpy(out, payload, payload_len);
400     memcpy(&out[payload_len], mac, mac_len);
401
402     // Encrypt the Payload with CTR mode with a counter
starting at 1.
403     memcpy(temp_iv, counter, AES_BLOCK_SIZE);
404     increment_iv(temp_iv, AES_BLOCK_SIZE - 1 -
mac_len); // Last argument is the byte size of the counting por-
tion of the counter block. /*BUG?*/
405     aes_encrypt_ctr(out, payload_len, out, key, keysize,
temp_iv);
406
407     // Encrypt the MAC with CTR mode with a counter start-
ing at 0.
408     aes_encrypt_ctr(&out[payload_len], mac_len, &out[pay-
load_len], key, keysize, counter);
409
410     free(buf);
411     *out_len = payload_len + mac_len;
412
413     return(TRUE);
414 }
415
416 // plaintext_len = ciphertext_len - mac_len
417 // Needs a flag for whether the MAC matches.
418 int aes_decrypt_ccm(const BYTE ciphertext[], WORD cipher-
text_len, const BYTE assoc[], unsigned short assoc_len,
419 const BYTE nonce[], unsigned short
nonce_len, BYTE plaintext[], WORD *plaintext_len,
420 WORD mac_len, int *mac_auth, const BYTE
key_str[], int keysize)
421 {
422     BYTE temp_iv[AES_BLOCK_SIZE], counter[AES_BLOCK_SIZE],
mac[16], mac_buf[16], *buf;
423     int end_of_buf, plaintext_len_store_size;
424     WORD key[60];
425
426     if (ciphertext_len <= mac_len)
427         return(FALSE);
428
429     buf = (BYTE*)malloc(assoc_len + ciphertext_len /
*ciphertext_len = plaintext_len + mac_len*/ + 48);
430     if (! buf)
431         return(FALSE);
432
433     // Prepare the key for usage.
434     aes_key_setup(key_str, key, keysize);
435
436     // Copy the plaintext and MAC to the output buffers.
437     *plaintext_len = ciphertext_len - mac_len;
438     plaintext_len_store_size = AES_BLOCK_SIZE - 1 -
nonce_len;
439     memcpy(plaintext, ciphertext, *plaintext_len);
440     memcpy(mac, &ciphertext[*plaintext_len], mac_len);
441
442     // Prepare the first counter block for use in decryp-
tion.
443     ccm_prepare_first_ctr_blk(counter, nonce, nonce_len,
plaintext_len_store_size);
444
445     // Decrypt the Payload with CTR mode with a counter
starting at 1.
446     memcpy(temp_iv, counter, AES_BLOCK_SIZE);
447     increment_iv(temp_iv, AES_BLOCK_SIZE - 1 -
mac_len); // (AES_BLOCK_SIZE - 1 - mac_len) is the byte size of
the counting portion of the counter block.
448     aes_decrypt_ctr(plaintext, *plaintext_len, plaintext,
key, keysize, temp_iv);
449
450     // Setting mac_auth to NULL disables the authentication
check.
451     if (mac_auth != NULL) {
452
453         // Decrypt the MAC with CTR mode with a counter
starting at 0.
454         aes_decrypt_ctr(mac, mac_len, mac, key, key-
size, counter);
455
456         // Format the first block of the formatted
data.
457         plaintext_len_store_size = AES_BLOCK_SIZE - 1 -
nonce_len;
458         ccm_prepare_first_format_blk(buf, assoc_len,
*plaintext_len, plaintext_len_store_size, mac_len, nonce,
nonce_len);
459         end_of_buf = AES_BLOCK_SIZE;
460
461         // Format the Associated Data into the authen-
tication buffer.
462         ccm_format_assoc_data(buf, &end_of_buf, assoc,
assoc_len);
463
464         // Format the Payload into the authentication
buffer.
465         ccm_format_payload_data(buf, &end_of_buf,
plaintext, *plaintext_len);
466
467         // Perform the CBC operation with an IV of zer-
os on the formatted buffer to calculate the MAC.

```

```

467     memset(temp_iv, 0, AES_BLOCK_SIZE);
468     aes_encrypt_cbc_mac(buf, end_of_buf, mac_buf,
key, keysize, temp_iv);
469
470     // Compare the calculated MAC against the MAC
embedded in the ciphertext to see if they are the same.
471     if (! memcmp(mac, mac_buf, mac_len)) {
472         *mac_auth = TRUE;
473     }
474     else {
475         *mac_auth = FALSE;
476         memset(plaintext, 0, *plaintext_len);
477     }
478 }
479
480 free(buf);
481
482 return(TRUE);
483 }
484
485 // Creates the first counter block. First byte is flags, then
the nonce, then the incremented part.
486 void ccm_prepare_first_ctr_blk(BYTE counter[], const BYTE
nonce[], int nonce_len, int payload_len_store_size)
487 {
488     memset(counter, 0, AES_BLOCK_SIZE);
489     counter[0] = (payload_len_store_size - 1) & 0x07;
490     memcpy(&counter[1], nonce, nonce_len);
491 }
492
493 void ccm_prepare_first_format_blk(BYTE buf[], int assoc_len,
int payload_len, int payload_len_store_size, int mac_len, const BYTE
nonce[], int nonce_len)
494 {
495     // Set the flags for the first byte of the first block.
496     buf[0] = (((mac_len - 2) / 2) & 0x07) << 3 | ((pay-
load_len_store_size - 1) & 0x07);
497     if (assoc_len > 0)
498         buf[0] += 0x40;
499     // Format the rest of the first block, storing the
nonce and the size of the payload.
500     memcpy(&buf[1], nonce, nonce_len);
501     memset(&buf[1 + nonce_len], 0, AES_BLOCK_SIZE - 1 -
nonce_len);
502     buf[15] = payload_len & 0x000000FF;
503     buf[14] = (payload_len >> 8) & 0x000000FF;
504 }
505
506 void ccm_format_assoc_data(BYTE buf[], int *end_of_buf, const
BYTE assoc[], int assoc_len)
507 {
508     int pad;
509
510     buf[*end_of_buf + 1] = assoc_len & 0x00FF;
511     buf[*end_of_buf] = (assoc_len >> 8) & 0x00FF;
512     *end_of_buf += 2;
513     memcpy(&buf[*end_of_buf], assoc, assoc_len);
514     *end_of_buf += assoc_len;
515     pad = AES_BLOCK_SIZE - (*end_of_buf % AES_B-
LOCK_SIZE); /*BUG?*/
516     memset(&buf[*end_of_buf], 0, pad);
517     *end_of_buf += pad;
518 }
519
520 void ccm_format_payload_data(BYTE buf[], int *end_of_buf,
const BYTE payload[], int payload_len)
521 {
522     int pad;
523
524     memcpy(&buf[*end_of_buf], payload, payload_len);
525     *end_of_buf += payload_len;
526     pad = *end_of_buf % AES_BLOCK_SIZE;
527     if (pad != 0)
528         pad = AES_BLOCK_SIZE - pad;
529     memset(&buf[*end_of_buf], 0, pad);
530     *end_of_buf += pad;
531 }
532
533 /*****
534 * AES
535 *****/
536 // KEY EXPANSION
537 // Substitutes a word using the AES S-Box.
538 WORD SubWord(WORD word)
539 {
540     unsigned int result;
541
542     result = (int)aes_sbox[(word >> 4) & 0x0000000F][word &
0x0000000F];
543     result += (int)aes_sbox[(word >> 12) & 0x0000000F]
[(word >> 8) & 0x0000000F] << 8;
544     result += (int)aes_sbox[(word >> 20) & 0x0000000F]
[(word >> 16) & 0x0000000F] << 16;
545     result += (int)aes_sbox[(word >> 28) & 0x0000000F]
[(word >> 24) & 0x0000000F] << 24;
546     return(result);
547 }

```

```

551
552 // Performs the action of generating the keys that will be
used in every round of
553 // encryption. "key" is the user-supplied input key, "w" is
the output key schedule,
554 // "keysize" is the length in bits of "key", must be 128, 192,
or 256.
555 void aes_key_setup(const BYTE key[], WORD w[], int keysize)
556 {
557     int Nb=4,Nr,Nk,idx;
558     WORD
temp,Rcon[]={0x01000000,0x02000000,0x04000000,0x08000000,0x10000000,
0x20000000,
559 0x40000000,0x80000000,0x1b000000,0x36000000,0x6c000000,0xd8000000,
0xab000000,0x4d000000,0x9a000000};
560
561     switch (keysize) {
562         case 128: Nr = 10; Nk = 4; break;
563         case 192: Nr = 12; Nk = 6; break;
564         case 256: Nr = 14; Nk = 8; break;
565         default: return;
566     }
567
568     for (idx=0; idx < Nk; ++idx) {
569         w[idx] = ((key[4 * idx] << 24) | ((key[4 * idx +
1] << 16) |
570 ((key[4 * idx + 2] << 8) |
571 ((key[4 * idx + 3])));
572     }
573
574     for (idx = Nk; idx < Nb * (Nr+1); ++idx) {
575         temp = w[idx - 1];
576         if ((idx % Nk) == 0)
577             temp = SubWord(KE_ROTWORD(temp)) ^
Rcon[(idx-1)/Nk];
578         else if (Nk > 6 && (idx % Nk) == 4)
579             temp = SubWord(temp);
580         w[idx] = w[idx-Nk] ^ temp;
581     }
582 }
583
584 //////////////////////////////////////////////////
585 // ADD ROUND KEY
586 //////////////////////////////////////////////////
587
588 // Performs the AddRoundKey step. Each round has its own pre-
generated 16-byte key in the
589 // form of 4 integers (the "w" array). Each integer is XOR'd
by one column of the state.
590 // Also performs the job of InvAddRoundKey(); since the func-
tion is a simple XOR process,
591 // it is its own inverse.
592 void AddRoundKey(BYTE state[][4], const WORD w[])
593 {
594     BYTE subkey[4];
595
596     // memcpy(subkey,&w[idx],4); // Not accurate for big
endian machines
597     // Subkey 1
598     subkey[0] = w[0] >> 24;
599     subkey[1] = w[0] >> 16;
600     subkey[2] = w[0] >> 8;
601     subkey[3] = w[0];
602     state[0][0] ^= subkey[0];
603     state[1][0] ^= subkey[1];
604     state[2][0] ^= subkey[2];
605     state[3][0] ^= subkey[3];
606     // Subkey 2
607     subkey[0] = w[1] >> 24;
608     subkey[1] = w[1] >> 16;
609     subkey[2] = w[1] >> 8;
610     subkey[3] = w[1];
611     state[0][1] ^= subkey[0];
612     state[1][1] ^= subkey[1];
613     state[2][1] ^= subkey[2];
614     state[3][1] ^= subkey[3];
615     // Subkey 3
616     subkey[0] = w[2] >> 24;
617     subkey[1] = w[2] >> 16;
618     subkey[2] = w[2] >> 8;
619     subkey[3] = w[2];
620     state[0][2] ^= subkey[0];
621     state[1][2] ^= subkey[1];
622     state[2][2] ^= subkey[2];
623     state[3][2] ^= subkey[3];
624     // Subkey 4
625     subkey[0] = w[3] >> 24;
626     subkey[1] = w[3] >> 16;
627     subkey[2] = w[3] >> 8;
628     subkey[3] = w[3];
629     state[0][3] ^= subkey[0];
630     state[1][3] ^= subkey[1];
631     state[2][3] ^= subkey[2];
632     state[3][3] ^= subkey[3];
633 }
634
635 //////////////////////////////////////////////////
636 // (Inv)SubBytes

```

```

637 //////////////////////////////////////////////////
638
639 // Performs the SubBytes step. All bytes in the state are sub-
stituted with a
640 // pre-calculated value from a lookup table.
641 void SubBytes(BYTE state[][4])
642 {
643     state[0][0] = aes_sbox[state[0][0] >> 4][state[0][0] &
0x0F];
644     state[0][1] = aes_sbox[state[0][1] >> 4][state[0][1] &
0x0F];
645     state[0][2] = aes_sbox[state[0][2] >> 4][state[0][2] &
0x0F];
646     state[0][3] = aes_sbox[state[0][3] >> 4][state[0][3] &
0x0F];
647     state[1][0] = aes_sbox[state[1][0] >> 4][state[1][0] &
0x0F];
648     state[1][1] = aes_sbox[state[1][1] >> 4][state[1][1] &
0x0F];
649     state[1][2] = aes_sbox[state[1][2] >> 4][state[1][2] &
0x0F];
650     state[1][3] = aes_sbox[state[1][3] >> 4][state[1][3] &
0x0F];
651     state[2][0] = aes_sbox[state[2][0] >> 4][state[2][0] &
0x0F];
652     state[2][1] = aes_sbox[state[2][1] >> 4][state[2][1] &
0x0F];
653     state[2][2] = aes_sbox[state[2][2] >> 4][state[2][2] &
0x0F];
654     state[2][3] = aes_sbox[state[2][3] >> 4][state[2][3] &
0x0F];
655     state[3][0] = aes_sbox[state[3][0] >> 4][state[3][0] &
0x0F];
656     state[3][1] = aes_sbox[state[3][1] >> 4][state[3][1] &
0x0F];
657     state[3][2] = aes_sbox[state[3][2] >> 4][state[3][2] &
0x0F];
658     state[3][3] = aes_sbox[state[3][3] >> 4][state[3][3] &
0x0F];
659 }
660
661 void InvSubBytes(BYTE state[][4])
662 {
663     state[0][0] = aes_invbox[state[0][0] >> 4][state[0][0] &
0x0F];
664     state[0][1] = aes_invbox[state[0][1] >> 4][state[0][1] &
0x0F];
665     state[0][2] = aes_invbox[state[0][2] >> 4][state[0][2] &
0x0F];
666     state[0][3] = aes_invbox[state[0][3] >> 4][state[0][3] &
0x0F];
667     state[1][0] = aes_invbox[state[1][0] >> 4][state[1][0] &
0x0F];
668     state[1][1] = aes_invbox[state[1][1] >> 4][state[1][1] &
0x0F];
669     state[1][2] = aes_invbox[state[1][2] >> 4][state[1][2] &
0x0F];
670     state[1][3] = aes_invbox[state[1][3] >> 4][state[1][3] &
0x0F];
671     state[2][0] = aes_invbox[state[2][0] >> 4][state[2][0] &
0x0F];
672     state[2][1] = aes_invbox[state[2][1] >> 4][state[2][1] &
0x0F];
673     state[2][2] = aes_invbox[state[2][2] >> 4][state[2][2] &
0x0F];
674     state[2][3] = aes_invbox[state[2][3] >> 4][state[2][3] &
0x0F];
675     state[3][0] = aes_invbox[state[3][0] >> 4][state[3][0] &
0x0F];
676     state[3][1] = aes_invbox[state[3][1] >> 4][state[3][1] &
0x0F];
677     state[3][2] = aes_invbox[state[3][2] >> 4][state[3][2] &
0x0F];
678     state[3][3] = aes_invbox[state[3][3] >> 4][state[3][3] &
0x0F];
679 }
680
681 //////////////////////////////////////////////////
682 // (Inv)ShiftRows
683 //////////////////////////////////////////////////
684
685 // Performs the ShiftRows step. All rows are shifted cyl-
indrically to the left.
686 void ShiftRows(BYTE state[][4])
687 {

```



```

688     int t;
689
690     // Shift left by 1
691     t = state[1][0];
692     state[1][0] = state[1][1];
693     state[1][1] = state[1][2];
694     state[1][2] = state[1][3];
695     state[1][3] = t;
696     // Shift left by 2
697     t = state[2][0];
698     state[2][0] = state[2][2];
699     state[2][2] = t;
700     t = state[2][1];
701     state[2][1] = state[2][3];
702     state[2][3] = t;
703     // Shift left by 3
704     t = state[3][0];
705     state[3][0] = state[3][3];
706     state[3][3] = state[3][2];
707     state[3][2] = state[3][1];
708     state[3][1] = t;
709 }
710
711 // All rows are shifted cylindrically to the right.
712 void InvShiftRows(BYTE state[][4])
713 {
714     int t;
715
716     // Shift right by 1
717     t = state[1][3];
718     state[1][3] = state[1][2];
719     state[1][2] = state[1][1];
720     state[1][1] = state[1][0];
721     state[1][0] = t;
722     // Shift right by 2
723     t = state[2][3];
724     state[2][3] = state[2][1];
725     state[2][1] = t;
726     t = state[2][2];
727     state[2][2] = state[2][0];
728     state[2][0] = t;
729     // Shift right by 3
730     t = state[3][3];
731     state[3][3] = state[3][0];
732     state[3][0] = state[3][1];
733     state[3][1] = state[3][2];
734     state[3][2] = t;
735 }
736
737 ///////////////////////////////////////////////////
738 // (Inv)MixColumns
739 ///////////////////////////////////////////////////
740
741 // Performs the MixColumns step. The state is multiplied by it-
742 self using matrix
743
744 // multiplication in a Galios Field 2^8. All multiplication is
745 pre-computed in a table.
746 // Addition is equivalent to XOR. (Must always make a copy of
747 the column as the original
748 // values will be destroyed.)
749 void MixColumns(BYTE state[][4])
750 {
751     BYTE col[4];
752
753     // Column 1
754     col[0] = state[0][0];
755     col[1] = state[1][0];
756     col[2] = state[2][0];
757     col[3] = state[3][0];
758     state[0][0] = gf_mul[col[0]][0];
759     state[0][0] ^= gf_mul[col[1]][1];
760     state[0][0] ^= col[2];
761     state[0][0] ^= col[3];
762     state[1][0] = col[0];
763     state[1][0] ^= gf_mul[col[1]][0];
764     state[1][0] ^= gf_mul[col[2]][1];
765     state[1][0] ^= col[3];
766     state[2][0] = col[0];
767     state[2][0] ^= gf_mul[col[2]][0];
768     state[2][0] ^= gf_mul[col[3]][1];
769     state[2][0] ^= col[1];
770     state[3][0] = col[0];
771     state[3][0] ^= gf_mul[col[3]][0];
772     state[3][0] ^= gf_mul[col[1]][1];
773     state[3][0] ^= col[2];
774     state[3][0] ^= col[3];
775     state[1][1] = col[0];
776     state[1][1] ^= gf_mul[col[1]][0];
777     state[1][1] ^= gf_mul[col[2]][1];
778     state[1][1] ^= col[3];
779     state[2][1] = col[0];
780     state[2][1] ^= gf_mul[col[3]][0];
781     state[2][1] ^= col[1];
782     state[3][1] = col[0];
783     state[3][1] ^= gf_mul[col[1]][0];
784     state[3][1] ^= gf_mul[col[2]][1];
785     state[3][1] ^= col[3];
786     state[2][2] = col[0];
787     state[2][2] ^= gf_mul[col[3]][0];
788     state[2][2] ^= col[1];
789     state[3][2] = col[0];
790     state[3][2] ^= gf_mul[col[1]][0];
791     state[3][2] ^= gf_mul[col[2]][1];
792     state[3][2] ^= col[3];
793     state[1][2] = col[0];
794     state[1][2] ^= gf_mul[col[3]][0];
795     state[1][2] ^= col[1];
796     state[2][2] = col[0];
797     state[2][2] ^= gf_mul[col[1]][0];
798     state[2][2] ^= gf_mul[col[2]][1];
799     state[2][2] ^= col[3];
800     state[3][2] = col[0];
801     state[3][2] ^= gf_mul[col[3]][0];
802     state[3][2] ^= col[1];
803     state[1][2] = col[0];
804     state[1][2] ^= gf_mul[col[1]][0];
805     state[1][2] ^= gf_mul[col[2]][1];
806     state[1][2] ^= col[3];
807     state[2][2] = col[0];
808     state[2][2] ^= gf_mul[col[3]][0];
809     state[2][2] ^= col[1];
810     state[3][2] = col[0];
811     state[3][2] ^= gf_mul[col[1]][0];
812     state[3][2] ^= gf_mul[col[2]][1];
813     state[3][2] ^= col[3];
814     state[1][2] = col[0];
815     state[1][2] ^= gf_mul[col[3]][0];
816     state[1][2] ^= col[1];
817     state[2][2] = col[0];
818     state[2][2] ^= gf_mul[col[1]][0];
819     state[2][2] ^= gf_mul[col[2]][1];
820     state[2][2] ^= col[3];
821     state[3][2] = col[0];
822     state[3][2] ^= gf_mul[col[3]][0];
823     state[3][2] ^= col[1];
824     state[1][2] = col[0];
825     state[1][2] ^= gf_mul[col[1]][0];
826     state[1][2] ^= gf_mul[col[2]][1];
827     state[1][2] ^= col[3];
828     state[2][2] = col[0];
829     state[2][2] ^= gf_mul[col[3]][0];
830     state[2][2] ^= col[1];
831     state[3][2] = col[0];
832     state[3][2] ^= gf_mul[col[1]][0];
833     state[3][2] ^= gf_mul[col[2]][1];
834     state[3][2] ^= col[3];
835 void InvMixColumns(BYTE state[][4])
836 {
837     BYTE col[4];
838
839     // Column 1
840     col[0] = state[0][0];
841     col[1] = state[1][0];
842     col[2] = state[2][0];
843     col[3] = state[3][0];
844     state[0][0] = gf_mul[col[0]][5];
845     state[0][0] ^= gf_mul[col[1]][3];
846     state[0][0] ^= gf_mul[col[2]][4];
847     state[0][0] ^= gf_mul[col[3]][2];
848     state[1][0] = gf_mul[col[0]][2];
849     state[1][0] ^= gf_mul[col[1]][5];
850     state[1][0] ^= gf_mul[col[2]][3];
851     state[1][0] ^= gf_mul[col[3]][4];
852     state[2][0] = gf_mul[col[0]][4];
853     state[2][0] ^= gf_mul[col[1]][2];
854     state[2][0] ^= gf_mul[col[2]][5];
855     state[2][0] ^= gf_mul[col[3]][3];
856     state[3][0] = gf_mul[col[0]][3];
857     state[3][0] ^= gf_mul[col[1]][4];
858     state[3][0] ^= gf_mul[col[2]][2];
859     state[3][0] ^= gf_mul[col[3]][5];
860     // Column 2
861     col[0] = state[0][1];
862     col[1] = state[1][1];
863     col[2] = state[2][1];
864     col[3] = state[3][1];
865     state[0][1] = gf_mul[col[0]][5];
866     state[0][1] ^= gf_mul[col[1]][3];
867     state[0][1] ^= gf_mul[col[2]][4];
868     state[0][1] ^= gf_mul[col[3]][2];
869     state[1][1] = gf_mul[col[0]][2];
870     state[1][1] ^= gf_mul[col[1]][5];
871     state[1][1] ^= gf_mul[col[2]][3];
872     state[1][1] ^= gf_mul[col[3]][4];
873     state[2][1] = gf_mul[col[0]][4];
874     state[2][1] ^= gf_mul[col[1]][2];
875     state[2][1] ^= gf_mul[col[2]][5];
876     state[2][1] ^= gf_mul[col[3]][3];
877     state[3][1] = gf_mul[col[0]][3];
878     state[3][1] ^= gf_mul[col[1]][4];
879     state[3][1] ^= gf_mul[col[2]][2];
880     state[3][1] ^= gf_mul[col[3]][5];
881     // Column 3
882     col[0] = state[0][2];
883     col[1] = state[1][2];
884     col[2] = state[2][2];
885     col[3] = state[3][2];

```

```

886 state[0][2] = gf_mul[col[0]][5];
887 state[0][2] ^= gf_mul[col[1]][3];
888 state[0][2] ^= gf_mul[col[2]][4];
889 state[0][2] ^= gf_mul[col[3]][2];
890 state[1][2] = gf_mul[col[0]][2];
891 state[1][2] ^= gf_mul[col[1]][5];
892 state[1][2] ^= gf_mul[col[2]][3];
893 state[1][2] ^= gf_mul[col[3]][4];
894 state[2][2] = gf_mul[col[0]][4];
895 state[2][2] ^= gf_mul[col[1]][2];
896 state[2][2] ^= gf_mul[col[2]][5];
897 state[2][2] ^= gf_mul[col[3]][3];
898 state[3][2] = gf_mul[col[0]][3];
899 state[3][2] ^= gf_mul[col[1]][4];
900 state[3][2] ^= gf_mul[col[2]][2];
901 state[3][2] ^= gf_mul[col[3]][5];
902 // Column 4
903 col[0] = state[0][3];
904 col[1] = state[1][3];
905 col[2] = state[2][3];
906 col[3] = state[3][3];
907 state[0][3] = gf_mul[col[0]][5];
908 state[0][3] ^= gf_mul[col[1]][3];
909 state[0][3] ^= gf_mul[col[2]][4];
910 state[0][3] ^= gf_mul[col[3]][2];
911 state[1][3] = gf_mul[col[0]][2];
912 state[1][3] ^= gf_mul[col[1]][5];
913 state[1][3] ^= gf_mul[col[2]][3];
914 state[1][3] ^= gf_mul[col[3]][4];
915 state[2][3] = gf_mul[col[0]][4];
916 state[2][3] ^= gf_mul[col[1]][2];
917 state[2][3] ^= gf_mul[col[2]][5];
918 state[2][3] ^= gf_mul[col[3]][3];
919 state[3][3] = gf_mul[col[0]][3];
920 state[3][3] ^= gf_mul[col[1]][4];
921 state[3][3] ^= gf_mul[col[2]][2];
922 state[3][3] ^= gf_mul[col[3]][5];
923 }
924
925 ///////////////////////////////////////////////////
926 // (En/De)Crypt
927 ///////////////////////////////////////////////////
928
929 void aes_encrypt(const BYTE in[], BYTE out[], const WORD
key[], int keysize)
930 {
931     BYTE state[4][4];
932
933     // Copy input array (should be 16 bytes long) to a mat-
934     // rix (sequential bytes are ordered
935     // by row, not col) called "state" for processing.
936     // *** Implementation note: The official AES documenta-
937     // tion references the state by
938     // column, then row. Accessing an element in C requires
939     // row then column. Thus, all state
940     // references in AES must have the column and row in-
941     // dices reversed for C implementation.
942     state[0][0] = in[0];
943     state[1][0] = in[1];
944     state[2][0] = in[2];
945     state[3][0] = in[3];
946     state[0][1] = in[4];
947     state[1][1] = in[5];
948     state[2][1] = in[6];
949     state[3][1] = in[7];
950     state[0][2] = in[8];
951     state[1][2] = in[9];
952     state[2][2] = in[10];
953     state[3][2] = in[11];
954     state[0][3] = in[12];
955     state[1][3] = in[13];
956     state[2][3] = in[14];
957     state[3][3] = in[15];
958
959     // Perform the necessary number of rounds. The round
960     // key is added first.
961     // The last round does not perform the MixColumns step.
962     AddRoundKey(state,&key[0]);
963     SubBytes(state); ShiftRows(state); MixColumns(state);
964     AddRoundKey(state,&key[4]);
965     SubBytes(state); ShiftRows(state); MixColumns(state);
966     AddRoundKey(state,&key[8]);
967     SubBytes(state); ShiftRows(state); MixColumns(state);
968     AddRoundKey(state,&key[12]);
969     SubBytes(state); ShiftRows(state); MixColumns(state);
970     AddRoundKey(state,&key[16]);
971     SubBytes(state); ShiftRows(state); MixColumns(state);
972     AddRoundKey(state,&key[20]);
973     SubBytes(state); ShiftRows(state); MixColumns(state);
974     AddRoundKey(state,&key[24]);
975     SubBytes(state); ShiftRows(state); MixColumns(state);
976     AddRoundKey(state,&key[28]);
977     SubBytes(state); ShiftRows(state); MixColumns(state);
978     AddRoundKey(state,&key[32]);
979     SubBytes(state); ShiftRows(state); MixColumns(state);
980     AddRoundKey(state,&key[36]);
981     if (keysize != 128) {
982         SubBytes(state); ShiftRows(state); Mix-
983         Columns(state); AddRoundKey(state,&key[40]);
984         SubBytes(state); ShiftRows(state); Mix-

```

```

970     Columns(state); AddRoundKey(state,&key[44]);
971         if (keysize != 192) {
972             SubBytes(state); ShiftRows(state); Mix-
Columns(state); AddRoundKey(state,&key[48]);
973             SubBytes(state); ShiftRows(state); Mix-
Columns(state); AddRoundKey(state,&key[52]);
974             SubBytes(state); ShiftRows(state); Ad-
dRoundKey(state,&key[56]);
975         }
976         else {
977             SubBytes(state); ShiftRows(state); Ad-
dRoundKey(state,&key[48]);
978         }
979     }
980     else {
981         SubBytes(state); ShiftRows(state); Ad-
dRoundKey(state,&key[40]);
982     }
983     // Copy the state to the output array.
984     out[0] = state[0][0];
985     out[1] = state[1][0];
986     out[2] = state[2][0];
987     out[3] = state[3][0];
988     out[4] = state[0][1];
989     out[5] = state[1][1];
990     out[6] = state[2][1];
991     out[7] = state[3][1];
992     out[8] = state[0][2];
993     out[9] = state[1][2];
994     out[10] = state[2][2];
995     out[11] = state[3][2];
996     out[12] = state[0][3];
997     out[13] = state[1][3];
998     out[14] = state[2][3];
999     out[15] = state[3][3];
1000 }
1001
1002 void aes_decrypt(const BYTE in[], BYTE out[], const WORD
key[], int keysize)
1003 {
1004     BYTE state[4][4];
1005
1006     // Copy the input to the state.
1007     state[0][0] = in[0];
1008     state[1][0] = in[1];
1009     state[2][0] = in[2];
1010     state[3][0] = in[3];
1011     state[0][1] = in[4];
1012     state[1][1] = in[5];
1013     state[2][1] = in[6];
1014     state[3][1] = in[7];
1015     state[0][2] = in[8];
1016     state[1][2] = in[9];
1017     state[2][2] = in[10];
1018     state[3][2] = in[11];
1019     state[0][3] = in[12];
1020     state[1][3] = in[13];
1021     state[2][3] = in[14];
1022     state[3][3] = in[15];
1023
1024     // Perform the necessary number of rounds. The round
key is added first.
1025     // The last round does not perform the MixColumns step.
1026     if (keysize > 128) {
1027         if (keysize > 192) {
1028             AddRoundKey(state,&key[56]);
1029
InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[52]);
InvMixColumns(state);
1030
InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[48]);
InvMixColumns(state);
1031         }
1032         else {
1033             AddRoundKey(state,&key[48]);
1034         }
1035
InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[44]);
InvMixColumns(state);
1036
InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[40]);
InvMixColumns(state);
1037     }
1038     else {
1039         AddRoundKey(state,&key[40]);
1040     }
1041
InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[36]);
InvMixColumns(state);
1042
InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[32]);
InvMixColumns(state);
1043
InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[28]);
InvMixColumns(state);
1044
InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[24]);
InvMixColumns(state);
1045
InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[20]);

```

```

InvMixColumns(state);
1046 InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[16]);
InvMixColumns(state);
1047 InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[12]);
InvMixColumns(state);
1048 InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[8]);I
nvMixColumns(state);
1049 InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[4]);I
nvMixColumns(state);
1050 InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[0]);
1051
1052 // Copy the state to the output array.
1053 out[0] = state[0][0];
1054 out[1] = state[1][0];
1055 out[2] = state[2][0];
1056 out[3] = state[3][0];
1057 out[4] = state[0][1];
1058 out[5] = state[1][1];
1059 out[6] = state[2][1];
1060 out[7] = state[3][1];
1061 out[8] = state[0][2];
1062 out[9] = state[1][2];
1063 out[10] = state[2][2];
1064 out[11] = state[3][2];
1065 out[12] = state[0][3];
1066 out[13] = state[1][3];
1067 out[14] = state[2][3];
1068 out[15] = state[3][3];
1069 }
1070
1071 /*****
1072 ** AES DEBUGGING FUNCTIONS
1073 *****/
1074 /*
1075 // This prints the "state" grid as a linear hex string.
1076 void print_state(BYTE state[][4])
1077 {
1078     int idx,idx2;
1079
1080     for (idx=0; idx < 4; idx++)
1081         for (idx2=0; idx2 < 4; idx2++)
1082             printf("%02x",state[idx2][idx]);
1083     printf("\n");
1084 }
1085
1086 // This prints the key (4 consecutive ints) used for a given
1087 // round as a linear hex string.
1088 void print_rnd_key(WORD key[])
1089 {
1090     int idx;
1091
1092     for (idx=0; idx < 4; idx++)
1093         printf("%08x",key[idx]);
1094     printf("\n");
1095 }
1096 */

```

2. B-Con_crypto-algorithms/aes.h (123 lines)

```

1 /
2 *
3 * Filename: aes.h
4 * Author: Brad Conte (brad AT bradconte.com)
5 * Copyright:
6 * Disclaimer: This code is presented "as is" without any guar-
7 * antees.
8 * Details: Defines the API for the corresponding AES imple-
9 * mentation.
10 *
11 *****/
12
13 #ifndef AES_H
14 #define AES_H
15
16 /***** HEADER FILES *****/
17 #include <stdint.h>
18
19 /***** MACROS *****/
20 #define AES_BLOCK_SIZE 16 // AES operates on 16
21 // bytes at a time
22
23 /***** DATA TYPES *****/
24 typedef unsigned char BYTE; // 8-bit byte
25 typedef unsigned int WORD; // 32-bit word, change
26 // to "long" for 16-bit machines
27
28 /***** FUNCTION DECLARATIONS *****/
29
30 void aes_key_setup(const BYTE key[], // The key, must be
31 // 128, 192, or 256 bits
32 // Output key
33 // schedule to be used later
34 int keysz, // Bit length of
35 // the key, 128, 192, or 256
36 void aes_encrypt(const BYTE in[], // 16 bytes of
37 // plaintext
38 // 16 bytes of
39 // ciphertext
40 const WORD key[], // From the key
41 // setup
42 int keysz, // Bit length of
43 // the key, 128, 192, or 256
44 void aes_decrypt(const BYTE in[], // 16 bytes of
45 // ciphertext
46 // 16 bytes of
47 // plaintext
48 const WORD key[], // From the key
49 // setup
50 int keysz, // Bit length of
51 // the key, 128, 192, or 256
52 const BYTE iv[], // IV, must be
53 // AES_BLOCK_SIZE bytes long
54 // Only output the CBC-MAC of the input.
55 int aes_encrypt_cbc_mac(const BYTE in[], // plaintext
56 // size_t in_len, // Must be a
57 // multiple of AES_BLOCK_SIZE
58 // 16 bytes of
59 // ciphertext
60 const WORD key[], // From the key
61 // setup
62 int keysz, // Bit length of
63 // the key, 128, 192, or 256
64 const BYTE iv[], // IV, must be
65 // AES_BLOCK_SIZE bytes long
66 void increment_iv(BYTE iv[], // Must be a
67 // multiple of AES_BLOCK_SIZE
68 int counter_size, // Bytes of the
69 // IV used for counting (low end)
70 void aes_encrypt_ctr(const BYTE in[], // Plaintext
71 // size_t in_len, // Any byte
72 // length
73 // 16 bytes of
74 // ciphertext
75 const WORD key[], // From the key
76 // setup
77 int keysz, // Bit length of
78 // the key, 128, 192, or 256
79 const BYTE iv[], // IV, must be
80 // AES_BLOCK_SIZE bytes long
81 void aes_decrypt_ctr(const BYTE in[], // Ciphertext
82 // size_t in_len, // Any byte
83 // length
84 // 16 bytes of
85 // plaintext
86 const WORD key[], // From the key
87 // setup
88 int keysz, // Bit length of
89 // the key, 128, 192, or 256
90 const BYTE iv[], // IV, must be
91 // AES_BLOCK_SIZE bytes long
92 // Returns True if the input parameters do not violate any
93 // constraint.
94 int aes_encrypt_ccm(const BYTE plaintext[], //

```

```

25 // Key setup must be done before any AES en/de-cryption func-
26 // tions can be used.
27 void aes_key_setup(const BYTE key[], // The key, must be
28 // 128, 192, or 256 bits
29 // Output key
30 // schedule to be used later
31 int keysz, // Bit length of
32 // the key, 128, 192, or 256
33 void aes_encrypt(const BYTE in[], // 16 bytes of
34 // plaintext
35 // 16 bytes of
36 // ciphertext
37 const WORD key[], // From the key
38 // setup
39 int keysz, // Bit length of
40 // the key, 128, 192, or 256
41 void aes_decrypt(const BYTE in[], // 16 bytes of
42 // ciphertext
43 // 16 bytes of
44 // plaintext
45 const WORD key[], // From the key
46 // setup
47 int keysz, // Bit length of
48 // the key, 128, 192, or 256
49 void aes_encrypt_cbc(const BYTE in[], // Plaintext
50 // size_t in_len, // Must be a
51 // multiple of AES_BLOCK_SIZE
52 // 16 bytes of
53 // ciphertext
54 const WORD key[], // From the key
55 // setup
56 int keysz, // Bit length of
57 // the key, 128, 192, or 256
58 const BYTE iv[], // IV, must be
59 // AES_BLOCK_SIZE bytes long
60 // Only output the CBC-MAC of the input.
61 int aes_encrypt_cbc_mac(const BYTE in[], // plaintext
62 // size_t in_len, // Must be a
63 // multiple of AES_BLOCK_SIZE
64 // 16 bytes of
65 // ciphertext
66 const WORD key[], // From the key
67 // setup
68 int keysz, // Bit length of
69 // the key, 128, 192, or 256
70 const BYTE iv[], // IV, must be
71 // AES_BLOCK_SIZE bytes long
72 void increment_iv(BYTE iv[], // Must be a
73 // multiple of AES_BLOCK_SIZE
74 int counter_size, // Bytes of the
75 // IV used for counting (low end)
76 void aes_encrypt_ctr(const BYTE in[], // Plaintext
77 // size_t in_len, // Any byte
78 // length
79 // 16 bytes of
80 // ciphertext
81 const WORD key[], // From the key
82 // setup
83 int keysz, // Bit length of
84 // the key, 128, 192, or 256
85 const BYTE iv[], // IV, must be
86 // AES_BLOCK_SIZE bytes long
87 void aes_decrypt_ctr(const BYTE in[], // Ciphertext
88 // size_t in_len, // Any byte
89 // length
90 // 16 bytes of
91 // plaintext
92 const WORD key[], // From the key
93 // setup
94 int keysz, // Bit length of
95 // the key, 128, 192, or 256
96 const BYTE iv[], // IV, must be
97 // AES_BLOCK_SIZE bytes long
98 // Returns True if the input parameters do not violate any
99 // constraint.
100 int aes_encrypt_ccm(const BYTE plaintext[], //

```



```

IN - Plaintext.
84 WORD plaintext_len, //
IN - Plaintext length.
85 const BYTE associated_data[], //
IN - Associated Data included in authentication, but not encryption.
86 unsigned short associated_data_len, //
IN - Associated Data length in bytes.
87 const BYTE nonce[], //
IN - The Nonce to be used for encryption.
88 unsigned short nonce_len, //
IN - Nonce length in bytes.
89 BYTE ciphertext[], //
OUT - Ciphertext, a concatenation of the plaintext and the MAC.
90 WORD *ciphertext_len, //
OUT - The length of the ciphertext, always plaintext_len + mac_len.
91 WORD mac_len, //
IN - The desired length of the MAC, must be 4, 6, 8, 10, 12, 14, or 16.
92 const BYTE key[], //
IN - The AES key for encryption.
93 int keysize; //
IN - The length of the key in bits. Valid values are 128, 192, 256.
94 // Returns True if the input parameters do not violate any constraint.
95 // Use mac_auth to ensure decryption/validation was preformed correctly.
96 // If authentication does not succeed, the plaintext is zeroed out. To overwrite
97 // this, call with mac_auth = NULL. The proper procedure is to decrypt with
98 // authentication enabled (mac_auth != NULL) and make a second call to that
99 // ignores authentication explicitly if the first call failes.
100 int aes_decrypt_ccm(const BYTE ciphertext[], //
IN - Ciphertext, the concatenation of encrypted plaintext and MAC.
101 WORD ciphertext_len, //
IN - Ciphertext length in bytes.
102 const BYTE assoc[], //
IN - The Associated Data, required for authentication.
103 unsigned short assoc_len, //
IN - Associated Data length in bytes.
104 const BYTE nonce[], //
IN - The Nonce to use for decryption, same one as for encryption.
105 unsigned short nonce_len, //
IN - Nonce length in bytes.
106 BYTE plaintext[], //
OUT - The plaintext that was decrypted. Will need to be large enough to hold ciphertext_len - mac_len.
107 WORD *plaintext_len, //
OUT - Length in bytes of the output plaintext, always ciphertext_len - mac_len.
108 WORD mac_len, //
IN - The length of the MAC that was calculated.
109 int *mac_auth, //
OUT - TRUE if authentication succeeded, FALSE if it did not. NULL pointer will ignore the authentication.
110 const BYTE key[], //
IN - The AES key for decryption.
111 int keysize; //
IN - The length of the key in BITS. Valid values are 128, 192, 256.
112 // Test functions
113 // Test functions
114 // Test functions
115 int aes_test();
116 int aes_ecb_test();
117 int aes_cbc_test();
118 int aes_ctr_test();
119 int aes_ccm_test();
120 // Test functions
121 // Test functions
122 // Test functions
123 #endif // AES_H

```

3. B-Con_crypto-algorithms/aes_test.c (285 lines)

```

1 /
*****
*
2 * Filename: aes_test.c
3 * Author: Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guarantees.
6 * Details: Performs known-answer tests on the corresponding AES
7 implementation. These tests do not encompass the
8 full range of available test vectors and are not sufficient
9 for FIPS-140 certification. However, if the
10 tests pass it is very, very likely that the code is correct
11 and was compiled properly. This code also serves as
12 example usage of the functions.

```

```

13
*****
*/
14
15 /***** HEADER FILES *****/
16 #include <stdio.h>
17 #include <memory.h>
18 #include "aes.h"
19
20 /***** FUNCTION DEFINITIONS *****/
21 void print_hex(BYTE str[], int len)
22 {
23     int idx;
24     for(idx = 0; idx < len; idx++)
25         printf("%02x", str[idx]);
26 }
27
28 int aes_ecb_test()
29 {
30     WORD key_schedule[60], idx;
31     BYTE enc_buf[128];
32     BYTE plaintext[2][16] = {
33         {0x6b, 0xc1, 0xbe, 0xe2, 0x2e, 0x40, 0x9f,
34          0x96, 0xe9, 0x3d, 0x7e, 0x11, 0x73, 0x93, 0x17, 0x2a},
35         {0xae, 0x2d, 0x8a, 0x57, 0x1e, 0x03, 0xac, 0x9c, 0x9e,
36          0xb7, 0x6f, 0xac, 0x45, 0xaf, 0x8e, 0x51}
37     };
38     BYTE ciphertext[2][16] = {
39         {0xf3, 0xee, 0xd1, 0xbd, 0xb5, 0xd2, 0xa0, 0x3c,
40          0x06, 0x4b, 0x5a, 0x7e, 0x3d, 0xb1, 0x81, 0xf8},
41         {0x59, 0x1c, 0xcb, 0x10, 0xd4, 0x10, 0xed, 0x26, 0xdc,
42          0x5b, 0xa7, 0x4a, 0x31, 0x36, 0x28, 0x70}
43     };
44     int pass = 1;
45     // Raw ECB mode.
46     //printf(" ECB mode:\n");
47     aes_key_setup(key[0], key_schedule, 256);
48     //printf(" Key : ");
49     //print_hex(key[0], 32);
50
51     for(idx = 0; idx < 2; idx++) {
52         aes_encrypt(plaintext[idx], enc_buf, key_schedule, 256);
53         //printf("\nPlaintext : ");
54         //print_hex(plaintext[idx], 16);
55         //printf("\n-n-encrypted to: ");
56         //print_hex(enc_buf, 16);
57         pass = pass && !memcmp(enc_buf, ciphertext[idx], 16);
58     }
59     aes_decrypt(ciphertext[idx], enc_buf, key_schedule, 256);
60     //printf("\nCiphertext : ");
61     //print_hex(ciphertext[idx], 16);
62     //printf("\n-n-decrypted to: ");
63     //print_hex(enc_buf, 16);
64
65     pass = pass && !memcmp(enc_buf, plaintext[idx], 16);
66
67     //printf("\n\n");
68 }
69
70 return(pass);
71 }
72
73 int aes_cbc_test()
74 {
75     WORD key_schedule[60];
76     BYTE enc_buf[128];
77     BYTE plaintext[1][32] = {
78         {0x6b, 0xc1, 0xbe, 0xe2, 0x2e, 0x40, 0x9f,
79          0x96, 0xe9, 0x3d, 0x7e, 0x11, 0x73, 0x93, 0x17, 0x2a, 0xae, 0x2d, 0x8a,
80          0x57, 0x1e, 0x03, 0xac, 0x9c, 0x9e, 0xb7, 0x6f, 0xac, 0x45, 0xaf, 0x8e, 0x51}
81     };
82     BYTE ciphertext[1][32] = {
83         {0xf5, 0x8c, 0x4c, 0x04, 0xd6, 0xe5, 0xf1, 0xba,
84          0x77, 0x9e, 0xab, 0xfb, 0x5f, 0x7b, 0xfb, 0xd6, 0x9c, 0xfc, 0x4e, 0x96, 0x7e,
85          0xdb, 0x80, 0x8d, 0x67, 0x9f, 0x77, 0x7b, 0xc6, 0x70, 0x2c, 0x7d}
86     };
87     BYTE iv[1][16] = {
88         {0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0a, 0x0b, 0x0c,
89          0xd0, 0xe0, 0xf0}
90     };
91     BYTE key[1][32] = {
92         {0x60, 0x3d, 0xeb, 0x10, 0x15, 0xca, 0x71, 0xbe, 0x2b,
93          0x73, 0xae, 0xf0, 0x85, 0x7d, 0x77, 0x81, 0x1f, 0x35, 0x2c, 0x07, 0x3b,
94          0x61, 0x08, 0xd7, 0x2d, 0x98, 0x10, 0xa3, 0x09, 0x14, 0xdf, 0xf4}
95     };
96     int pass = 1;

```

```

91 //printf(" CBC mode:\n");
92 aes_key_setup(key[0], key_schedule, 256);
93
94 //printf("Key : ");
95 //print_hex(key[0], 32);
96 //printf("\nIV : ");
97 //print_hex(iv[0], 16);
98
99 aes_encrypt_cbc(plaintext[0], 32, enc_buf, key_schedule, 256, iv[0]);
100 //printf("\nPlaintext : ");
101 //print_hex(plaintext[0], 32);
102 //printf("\n-n-encrypted to: ");
103 //print_hex(enc_buf, 32);
104 //printf("\nCiphertext : ");
105 //print_hex(ciphertext[0], 32);
106 pass = pass && !memcmp(enc_buf, ciphertext[0], 32);
107
108 aes_decrypt_cbc(ciphertext[0], 32, enc_buf, key_schedule, 256, iv[0]);
109 //printf("\nCiphertext : ");
110 //print_hex(ciphertext[0], 32);
111 //printf("\n-decrypted to: ");
112 //print_hex(enc_buf, 32);
113 //printf("\nPlaintext : ");
114 //print_hex(plaintext[0], 32);
115 pass = pass && !memcmp(enc_buf, plaintext[0], 32);
116
117 //printf("\n\n");
118 return(pass);
119 }
120
121 int aes_ctr_test()
122 {
123     WORD key_schedule[60];
124     BYTE enc_buf[128];
125     BYTE plaintext[1][32] = {
126         {0x6b, 0xc1, 0xbe, 0xe2, 0x2e, 0x40, 0x9f,
0x96, 0xe9, 0x3d, 0x7e, 0x11, 0x73, 0x93, 0x17, 0x2a, 0xae, 0x2d, 0x8a,
0x57, 0x1e, 0x03, 0xac, 0x9c, 0x9e, 0xb7, 0x6f, 0xac, 0x45, 0xaf, 0x8e, 0x51}
127     };
128     BYTE ciphertext[1][32] = {
129         {0x60, 0x1e,
0xc3, 0x13, 0x77, 0x57, 0x89, 0xa5, 0xb7, 0xa7, 0xf5, 0x04, 0xbb,
0xf3, 0xd2, 0x28, 0xf4, 0x43, 0xe3, 0xca, 0x4d, 0x62, 0xb5, 0x9a, 0xca,
0x84, 0xe9, 0x90, 0xca, 0xca, 0xf5, 0xc5}
130     };
131     BYTE iv[1][16] = {
132         {0xf0, 0xf1, 0xf2, 0xf3, 0xf4, 0xf5, 0xf6, 0xf7, 0xf8, 0xf9, 0xfa, 0xfb, 0xfc,
0xfd, 0xfe, 0xff},
133     };
134     BYTE key[1][32] = {
135         {0x60, 0x3d, 0xeb, 0x10, 0x15, 0xca, 0x71, 0xbe, 0x2b,
0x73, 0xae, 0xf0, 0x85, 0x7d, 0x77, 0x81, 0x1f, 0x35, 0x2c, 0x07, 0x3b,
0x61, 0x08, 0xd7, 0x2d, 0x98, 0x10, 0xa3, 0x09, 0x14, 0xdf, 0xf4}
136     };
137     int pass = 1;
138
139     //printf(" CTR mode:\n");
140     aes_key_setup(key[0], key_schedule, 256);
141
142     //printf("Key : ");
143     //print_hex(key[0], 32);
144     //printf("\nIV : ");
145     //print_hex(iv[0], 16);
146
147     aes_encrypt_ctr(plaintext[0], 32, enc_buf, key_schedule, 256, iv[0]);
148     //printf("\nPlaintext : ");
149     //print_hex(plaintext[0], 32);
150     //printf("\n-n-encrypted to: ");
151     //print_hex(enc_buf, 32);
152     pass = pass && !memcmp(enc_buf, ciphertext[0], 32);
153
154     aes_decrypt_ctr(ciphertext[0], 32, enc_buf, key_schedule, 256, iv[0]);
155     //printf("\nCiphertext : ");
156     //print_hex(ciphertext[0], 32);
157     //printf("\n-decrypted to: ");
158     //print_hex(enc_buf, 32);
159     pass = pass && !memcmp(enc_buf, plaintext[0], 32);
160
161     //printf("\n\n");
162     return(pass);
163 }
164
165 int aes_ccm_test()
166 {
167     int mac_auth;
168     WORD enc_buf_len;
169     BYTE enc_buf[128];
170     BYTE plaintext[3][32] = {
171         {0x20, 0x21, 0x22, 0x23},
172         {0x20, 0x21, 0x22, 0x23, 0x24, 0x25, 0x26, 0x27, 0x28, 0x29, 0x2a, 0x2b, 0x2c,
0x2d, 0x2e, 0x2f},
173         {0x20, 0x21, 0x22, 0x23, 0x24, 0x25, 0x26, 0x27, 0x28, 0x29, 0x2a, 0x2b, 0x2c,
0x2d, 0x2e, 0x2f, 0x30, 0x31, 0x32, 0x33, 0x34, 0x35, 0x36, 0x37}
174     };

```

```

175     BYTE assoc[3][32] = {
176         {0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
0x08, 0x09, 0x0a, 0x0b, 0x0c,
0x0d, 0x0e, 0x0f},
177         {0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0a, 0x0b, 0x0c,
0x0d, 0x0e, 0x0f},
178         {0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0a, 0x0b, 0x0c,
0x0d, 0x0e, 0x0f, 0x10, 0x11, 0x12, 0x13}
179     };
180     BYTE ciphertext[3][32 + 16] = {
181         {0x71, 0x62, 0x01, 0x5b, 0x4d, 0xac, 0x25, 0x5d},
182         {0xd2, 0xa1, 0xf0, 0xe0, 0x51, 0xea, 0x5f,
0x62, 0x08, 0x1a, 0x77, 0x92, 0x07, 0x3d, 0x59, 0x3d, 0x1f, 0xc6, 0x4f, 0xbf,
0xac, 0xcd},
183         {0xe3, 0xb2, 0x01, 0xa9, 0xf5, 0xb7, 0x1a, 0x7a, 0x9b,
0x1c, 0xea, 0xec, 0xcd, 0x97, 0xe7, 0x0b, 0x61, 0x76, 0xaa,
0xd9, 0xa4, 0x42, 0x8a, 0xa5, 0x48, 0x43, 0x92, 0xfb, 0xc1, 0xb0, 0x99, 0x51}
184     };
185     BYTE iv[3][16] = {
186         {0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16},
187         {0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17},
188         {0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17, 0x18, 0x19, 0x1a, 0x1b}
189     };
190     BYTE key[1][32] = {
191         {0x40, 0x41, 0x42, 0x43, 0x44, 0x45, 0x46, 0x47, 0x48, 0x49, 0x4a, 0x4b, 0x4c,
0x4d, 0x4e, 0x4f}
192     };
193     int pass = 1;
194
195     //printf(" CCM mode:\n");
196     //printf("Key : ");
197     //print_hex(key[0], 16);
198
199     //print_hex(plaintext[0], 4);
200     //print_hex(assoc[0], 8);
201     //print_hex(ciphertext[0], 8);
202     //print_hex(iv[0], 7);
203     //print_hex(key[0], 16);
204
205     aes_encrypt_ccm(plaintext[0], 4, assoc[0], 8, iv[0], 7,
enc_buf, &enc_buf_len, 4, key[0], 128);
206     //printf("\nNONCE : ");
207     //print_hex(iv[0], 7);
208     //printf("\nAssoc. Data : ");
209     //print_hex(assoc[0], 8);
210     //printf("\nPayload : ");
211     //print_hex(plaintext[0], 4);
212     //printf("\n-n-encrypted to: ");
213     //print_hex(enc_buf, enc_buf_len);
214     pass = pass && !memcmp(enc_buf, ciphertext[0],
enc_buf_len);
215
216     aes_decrypt_ccm(ciphertext[0], 8, assoc[0], 8, iv[0],
7, enc_buf, &enc_buf_len, 4, &mac_auth, key[0], 128);
217     //printf("\n-Ciphertext : ");
218     //print_hex(ciphertext[0], 8);
219     //printf("\n-decrypted to: ");
220     //print_hex(enc_buf, enc_buf_len);
221     //printf("\nAuthenticated: %d ", mac_auth);
222     pass = pass && !memcmp(enc_buf, plaintext[0],
enc_buf_len) && mac_auth;
223
224
225     aes_encrypt_ccm(plaintext[1], 16, assoc[1], 16, iv[1],
8, enc_buf, &enc_buf_len, 6, key[0], 128);
226     //printf("\n\nNONCE : ");
227     //print_hex(iv[1], 8);
228     //printf("\nAssoc. Data : ");
229     //print_hex(assoc[1], 16);
230     //printf("\nPayload : ");
231     //print_hex(plaintext[1], 16);
232     //printf("\n-n-encrypted to: ");
233     //print_hex(enc_buf, enc_buf_len);
234     pass = pass && !memcmp(enc_buf, ciphertext[1],
enc_buf_len);
235
236     aes_decrypt_ccm(ciphertext[1], 22, assoc[1], 16, iv[1],
8, enc_buf, &enc_buf_len, 6, &mac_auth, key[0], 128);
237     //printf("\n-Ciphertext : ");
238     //print_hex(ciphertext[1], 22);
239     //printf("\n-decrypted to: ");
240     //print_hex(enc_buf, enc_buf_len);
241     //printf("\nAuthenticated: %d ", mac_auth);
242     pass = pass && !memcmp(enc_buf, plaintext[1],
enc_buf_len) && mac_auth;
243
244
245     aes_encrypt_ccm(plaintext[2], 24, assoc[2], 20, iv[2],
12, enc_buf, &enc_buf_len, 8, key[0], 128);
246     //printf("\n\nNONCE : ");
247     //print_hex(iv[2], 12);
248     //printf("\nAssoc. Data : ");
249     //print_hex(assoc[2], 20);
250     //printf("\nPayload : ");
251     //print_hex(plaintext[2], 24);
252     //printf("\n-n-encrypted to: ");
253     //print_hex(enc_buf, enc_buf_len);

```

```

254     pass = pass && !memcmp(enc_buf, ciphertext[2],
enc_buf_len);
255
256     aes_decrypt_ccm(ciphertext[2], 32, assoc[2], 20, iv[2],
12, enc_buf, &enc_buf_len, 8, &mac_auth, key[0], 128);
257     //printf("\n-Ciphertext : ");
258     //print_hex(ciphertext[2], 32);
259     //printf("\n-decrypted to: ");
260     //print_hex(enc_buf, enc_buf_len);
261     //printf("\nAuthenticated: %d ", mac_auth);
262     pass = pass && !memcmp(enc_buf, plaintext[2],
enc_buf_len) && mac_auth;
263
264     //printf("\n\n");
265     return(pass);
266 }
267
268 int aes_test()
269 {
270     int pass = 1;
271
272     pass = pass && aes_ecb_test();
273     pass = pass && aes_cbc_test();
274     pass = pass && aes_ctr_test();
275     pass = pass && aes_ccm_test();
276
277     return(pass);
278 }
279
280 int main(int argc, char *argv[])
281 {
282     printf("AES Tests: %s\n", aes_test() ? "SUCCEEDED" :
"FAILED");
283
284     return(0);
285 }

```

4. B-Con_crypto-algorithms/arcfour.c (47 lines)

```

1 /
*****
2 * Filename:   arcfour.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   Implementation of the ARCFOUR encryption al-
gorithm.
7             Algorithm specification can be found here:
8             * http://en.wikipedia.org/wiki/RC4
9
*****
*/
10
11 /***** HEADER FILES *****/
12 #include <stdlib.h>
13 #include "arcfour.h"
14
15 /***** FUNCTION DEFINITIONS *****/
16 void arcfour_key_setup(BYTE state[], const BYTE key[], int
len)
17 {
18     int i, j;
19     BYTE t;
20
21     for (i = 0; i < 256; ++i)
22         state[i] = i;
23     for (i = 0, j = 0; i < 256; ++i) {
24         j = (j + state[i] + key[i % len]) % 256;
25         t = state[i];
26         state[i] = state[j];
27         state[j] = t;
28     }
29 }
30
31 // This does not hold state between calls. It always generates
the
32 // stream starting from the first output byte.
33 void arcfour_generate_stream(BYTE state[], BYTE out[], size_t
len)
34 {
35     int i, j;
36     size_t idx;
37     BYTE t;
38
39     for (idx = 0, i = 0, j = 0; idx < len; ++idx) {
40         i = (i + 1) % 256;
41         j = (j + state[i]) % 256;
42         t = state[i];
43         state[i] = state[j];
44         state[j] = t;
45         out[idx] = state[(state[i] + state[j]) % 256];
46     }
47 }

```

5. B-Con_crypto-algorithms/arcfour.h (30 lines)

```

1 /
*****
2 * Filename:   arcfour.h
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   Defines the API for the corresponding ARCFOUR
implementation.
7
*****
*/
8
9 #ifndef ARCFOUR_H
10 #define ARCFOUR_H
11
12 /***** HEADER FILES *****/
13 #include <stddef.h>
14
15 /***** DATA TYPES *****/
16 typedef unsigned char BYTE;          // 8-bit byte
17
18 /***** FUNCTION DECLARATIONS *****/
19 // Input: state - the state used to generate the keystream
20 //          key - Key to use to initialize the state
21 //          len - length of key in bytes (valid len is 1 to
256)
22 void arcfour_key_setup(BYTE state[], const BYTE key[], int
len);
23
24 // Pseudo-Random Generator Algorithm
25 // Input: state - the state used to generate the keystream
26 //          out - Must be allocated to be of at least "len"
length
27 //          len - number of bytes to generate
28 void arcfour_generate_stream(BYTE state[], BYTE out[], size_t
len);
29
30 #endif // ARCFOUR_H

```

6. B-Con_crypto-algorithms/arcfour_test.c (47 lines)

```

1 /
*****
2 * Filename:   arcfour_test.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   Performs known-answer tests on the corresponding AR-
CFOUR
7             implementation. These tests do not encompass
the full
8             range of available test vectors, however, if
the tests
9
10             pass it is very, very likely that the code is
correct
11             and was compiled properly. This code also
serves as
12             example usage of the functions.
13
*****
14 /***** HEADER FILES *****/
15 #include <stdio.h>
16 #include <memory.h>
17 #include "arcfour.h"
18
19 /***** FUNCTION DEFINITIONS *****/
20 int rc4_test()
21 {
22     BYTE state[256];
23     BYTE key[3][10] = {{ "Key", "Wiki", "Secret" }};
24     BYTE stream[3][10] = {{ 0xEB, 0x9F,
0x77, 0x81, 0xB7, 0x34, 0xCA, 0x72, 0xA7, 0x19 },
25                             { 0x60, 0x44, 0xdb, 0x6d, 0x41, 0xb7 },
26                             { 0x04, 0xd4, 0x6b, 0x05, 0x3c,
0xa8, 0x7b, 0x59 }};
27     int stream_len[3] = {10, 6, 8};
28     BYTE buf[1024];
29     int idx;
30     int pass = 1;
31
32     // Only test the output stream. Note that the state can be
reused.
33     for (idx = 0; idx < 3; idx++) {

```

```

34         arcfour_key_setup(state, key[idx],
strlen(key[idx]));
35         arcfour_generate_stream(state, buf,
stream_len[idx]);
36         pass = pass && !memcmp(stream[idx], buf,
stream_len[idx]);
37     }
38 }
39     return(pass);
40 }
41
42 int main()
43 {
44     printf("ARCFOUR tests: %s\n", rc4_test() ? "SUC-
CEDED" : "FAILED");
45
46     return(0);
47 }

```

7. B-Con_crypto-algorithms/base64.c (135 lines)

```

1 /
*****
2 * Filename:   base64.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   Implementation of the Base64 encoding algorithm.
7
*****
8
9 /***** HEADER FILES *****/
10 #include <stdlib.h>
11 #include "base64.h"
12
13 /***** MACROS *****/
14 #define NEWLINE_INVL 76
15
16 /***** VARIABLES *****/
17 // Note: To change the charset to a URL encoding, replace the
'+' and '/' with '*' and '-'
18 static const BYTE char-
set[]={"ABCDEFHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"};
19
20 /***** FUNCTION DEFINITIONS *****/
21 BYTE revchar(char ch)
22 {
23     if (ch >= 'A' && ch <= 'Z')
24         ch -= 'A';
25     else if (ch >= 'a' && ch <= 'z')
26         ch = ch - 'a' + 26;
27     else if (ch >= '0' && ch <= '9')
28         ch = ch - '0' + 52;
29     else if (ch == '+')
30         ch = 62;
31     else if (ch == '/')
32         ch = 63;
33
34     return(ch);
35 }
36
37 size_t base64_encode(const BYTE in[], BYTE out[], size_t len,
int newline_flag)
38 {
39     size_t idx, idx2, blks, blk_ceiling, left_over,
newline_count = 0;
40
41     blks = (len / 3);
42     left_over = len % 3;
43
44     if (out == NULL) {
45         idx2 = blks * 4;
46         if (left_over)
47             idx2 += 4;
48         if (newline_flag)
49             idx2 += len / 57; // (NEWLINE_INVL /
4) * 3 = 57. One newline per 57 input bytes.
50     }
51     else {
52         // Since 3 input bytes = 4 output bytes, de-
termine out how many even sets of
53         // 3 bytes the input has.
54         blk_ceiling = blks * 3;
55         for (idx = 0, idx2 = 0; idx < blk_ceiling; idx
+= 3, idx2 += 4) {
56             out[idx2] = charset[in[idx] >> 2];
57             out[idx2 + 1] = charset[(in[idx] &
0x03) << 4] | (in[idx + 1] >> 4)];
58             out[idx2 + 2] = charset[(in[idx + 1] &
0x0f) << 2] | (in[idx + 2] >> 6)];
59             out[idx2 + 3] = charset[in[idx + 2] &

```

```

0x3f];
60         // The official standard requires a
newline every 76 characters.
61         // (Eg, first newline is character 77
of the output.)
62         if (((idx2 - newline_count + 4) %
NEWLINE_INVL == 0) && newline_flag) {
63             out[idx2 + 4] = '\n';
64             idx2++;
65             newline_count++;
66         }
67     }
68
69     if (left_over == 1) {
70         out[idx2] = charset[in[idx] >> 2];
71         out[idx2 + 1] = charset[(in[idx] &
0x03) << 4];
72         out[idx2 + 2] = '=';
73         out[idx2 + 3] = '=';
74         idx2 += 4;
75     }
76     else if (left_over == 2) {
77         out[idx2] = charset[in[idx] >> 2];
78         out[idx2 + 1] = charset[(in[idx] &
0x03) << 4] | (in[idx + 1] >> 4)];
79         out[idx2 + 2] = charset[(in[idx + 1] &
0x0f) << 2];
80         out[idx2 + 3] = '=';
81         idx2 += 4;
82     }
83 }
84
85 return(idx2);
86 }
87
88 size_t base64_decode(const BYTE in[], BYTE out[], size_t len)
89 {
90     BYTE ch;
91     size_t idx, idx2, blks, blk_ceiling, left_over;
92
93     if (in[len - 1] == '=')
94         len--;
95     if (in[len - 1] == '=')
96         len--;
97
98     blks = len / 4;
99     left_over = len % 4;
100
101     if (out == NULL) {
102         if (len >= 77 && in[NEWLINE_INVL] == '\n') //
Verify that newlines where used.
103             len -= len / (NEWLINE_INVL + 1);
104         blks = len / 4;
105         left_over = len % 4;
106
107         idx = blks * 3;
108         if (left_over == 2)
109             idx++;
110         else if (left_over == 3)
111             idx += 2;
112     }
113     else {
114         blk_ceiling = blks * 4;
115
116         for (idx = 0, idx2 = 0; idx2 < blk_ceiling; idx
+= 3, idx2 += 4) {
117             if (in[idx2] == '\n')
118                 idx2++;
119             out[idx] = (revchar(in[idx2]) << 2) |
((revchar(in[idx2 + 1]) & 0x30) >> 4);
120             out[idx + 1] = (revchar(in[idx2 + 1])
<< 4) | (revchar(in[idx2 + 2]) >> 2);
121             out[idx + 2] = (revchar(in[idx2 + 2])
<< 6) | revchar(in[idx2 + 3]);
122         }
123         if (left_over == 2) {
124             out[idx] = (revchar(in[idx2]) << 2) |
((revchar(in[idx2 + 1]) & 0x30) >> 4);
125             idx++;
126         }
127         else if (left_over == 3) {
128             out[idx] = (revchar(in[idx2]) << 2) |
((revchar(in[idx2 + 1]) & 0x30) >> 4);
129             out[idx + 1] = (revchar(in[idx2 + 1])
<< 4) | (revchar(in[idx2 + 2]) >> 2);
130             idx += 2;
131         }
132     }
133
134     return(idx);
135 }

```

8. B-Con_crypto-algorithms/base64.h (27 lines)

```

1 /
*****
2 * Filename:   base64.h
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   Defines the API for the corresponding Base64 im-
plementation.
7
*****
*/
8
9 #ifndef BASE64_H
10 #define BASE64_H
11
12 /***** HEADER FILES *****/
13 #include <stddef.h>
14
15 /***** DATA TYPES *****/
16 typedef unsigned char BYTE;          // 8-bit byte
17
18 /***** FUNCTION DECLARATIONS *****/
19 // Returns the size of the output. If called with out = NULL,
will just return
20 // the size of what the output would have been (without a ter-
minating NULL).
21 size_t base64_encode(const BYTE in[], BYTE out[], size_t len,
int newline_flag);
22
23 // Returns the size of the output. If called with out = NULL,
will just return
24 // the size of what the output would have been (without a ter-
minating NULL).
25 size_t base64_decode(const BYTE in[], BYTE out[], size_t len);
26
27 #endif // BASE64_H

```

9. B-Con_crypto-algorithms/base64_test.c (54 lines)

```

1 /
*****
2 * Filename:   blowfish_test.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6
7 * Details:   Performs known-answer tests on the corresponding
Base64
8               implementation. These tests do not encompass
the full
9               range of available test vectors, however, if
the tests
10               pass it is very, very likely that the code is
correct
11               and was compiled properly. This code also
serves as
12               example usage of the functions.
13
*****
14 /***** HEADER FILES *****/
15 #include <stdio.h>
16 #include <memory.h>
17 #include "base64.h"
18
19 /***** FUNCTION DEFINITIONS *****/
20 int base64_test()
21 {
22     BYTE text[3][1024] = {{"fo"},
23                             {"foobar"},
24                             {"Man is distinguished, not only
by his reason, but by this singular passion from other animals,
which is a lust of the mind, that by a perseverance of delight in
the continued and indefatigable generation of knowledge, exceeds the
short vehemence of any carnal pleasure."}};
25     BYTE code[3][1024] = {"Zm8="},
26                             {"Zm9vYmFy"},
27                             {"TWFuIGlZIGRpc3Rpbm-
d1aXNoZWQsIG5vdCBvbmx5IGJ5IGhpcyByZWZzb24sIGJldCBieS-
B0aGZlbnINpbmd1bGFyIHh3c3Rpb24gZnJvbSBvdGhlc1Bhbm1tYWxzLCB3aG1jaCB
pcyBhIGxlc3Qg-
b2YgYndGh1IGljbmlQsIHROXQyYXNkYYSBmXZjZjZmFuY2UgY2YgZGVzawdodCBp-
biB-
0aGUGY29udGluYndwIGFvZCBpbmRlZmF0aWdhYmxlIGd1bmVvYXNkY2YgY2Yga25v
d2x1ZG-
d1LCBleGlnZWZlIHROZnZSBzaG9yZCB2ZWhlbWVuY2UgY2YgY2Y5IGh3cm5hbnBwGv

```

```

hc3VyZS4="}};
28     BYTE buf[1024];
29     size_t buf_len;
30     int pass = 1;
31     int idx;
32
33     for (idx = 0; idx < 3; idx++) {
34         buf_len = base64_encode(text[idx], buf,
strlen(text[idx]), 1);
35         pass = pass && ((buf_len == strlen(code[idx]))
&&
36                     (buf_len ==
base64_encode(text[idx], NULL, strlen(text[idx]), 1)));
37         pass = pass && !strcmp(code[idx], buf);
38
39         memset(buf, 0, sizeof(buf));
40         buf_len = base64_decode(code[idx], buf,
strlen(code[idx]));
41         pass = pass && ((buf_len == strlen(text[idx]))
&&
42                     (buf_len ==
base64_decode(code[idx], NULL, strlen(code[idx]), 1)));
43         pass = pass && !strcmp(text[idx], buf);
44     }
45     return(pass);
46 }
47
48 int main()
49 {
51     printf("Base64 tests: %s\n", base64_test() ? "PASSED" :
"FAILED");
52
53     return 0;
54 }

```

10. B-Con_crypto-algorithms/blowfish.c (269 lines)

```

1 /
*****
2 * Filename:   blowfish.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   Implementation of the Blowfish encryption al-
gorithm.
7               Modes of operation (such as CBC) are not in-
cluded.
8               Algorithm specification can be found here:
9               * http://www.schneier.com/blowfish.html
10
*****
11
12 /***** HEADER FILES *****/
13 #include <stdlib.h>
14 #include <memory.h>
15 #include "blowfish.h"
16
17 /***** MACROS *****/
18 #define F(x,t) t = keystream->s[0][((x) >> 24); \
19             t += keystream->s[1][((x) >> 16) & 0xff]; \
20             t ^= keystream->s[2][((x) >> 8) & 0xff]; \
21             t += keystream->s[3][((x) & 0xff)];
22 #define swap(r,l,t) t = l; l = r; r = t;
23
24 #define ITERATION(l,r,t,pval) l ^= keystream->p[pval]; F(l,t);
r ^= t; swap(r,l,t);
25
26 /***** VARIABLES *****/
27 static const WORD p_perm[18] = {
28     0x243F6A88, 0x85A308D3, 0x13198A2E,
0x03707344, 0xA4093822, 0x299F31D0, 0x082EFA98,
29     0xEC4E6C89, 0x452821E6, 0x38D01377, 0xBE5466CF, 0x34E90C6C, 0x-
C0AC29B7, 0xC97C50DD,
30     0x3F84D5B5, 0xB5470917, 0x9216D5D9, 0x8979FB1B
31 };
32
33 static const WORD s_perm[4][256] = { {
34     0xD1310BA6, 0x98DFB5AC, 0x2FFD72DB, 0XD01ADF7, 0xB8E1AFED,
0x6A267E96, 0xBA7C9045, 0xF12C7F99,
35     0x24A19947, 0xB3916CF7, 0x0801F2E2, 0x858EFC16, 0x636920D8, 0x71574E69, 0x-
XA458FEA3, 0xF4933D7E,
36     0x0D95748F, 0x728EB658, 0x718BCD58, 0x82154AEE, 0x7B54A41D, 0x-
C25A59B5, 0x9C30D539, 0x2AF26013,
37     0xC5D1B023, 0x28698F50, 0xCA417918, 0xB8DB38EF, 0x8E79D-
CB0, 0x603A180E, 0x6C9E0E8B, 0xB01E8A3E,
38     0xD71577C1, 0xBD314B27, 0x78FAF2FDA,
0x556605C6, 0xE6552F3, 0x0AA5A894, 0x57489862, 0x63E81440,
39     0x7C7E993, 0xB3EE1411, 0x636FBC2A,
0x2BA9C55D, 0x741831F6, 0xCE5C3E16, 0x9B87931E, 0xAF-
D6BA33, 0x6C24CF5C, 0x7A325381, 0x28958677,

```



```

40 0x3B8F4898, 0x6B4BB9AF, 0xC4BFE81B, 0x66282193, 0x61D809CC, 0xF-
B21A991, 0x487CAC60, 0x5DEC8032,
41 0xEF845D5D,
0xE98575B1, 0xDC262302, 0xEB651B88, 0x23893E81, 0xD396ACCS, 0x0F6D6FF3, 0
x83F44239,
42 0x2E0B4482, 0xA4842004, 0x69C8F04A, 0x9E1F9B5E,
0x21C66842, 0xF6E96C9A, 0x670C9C61, 0xAB0388F0,
43 0x6A51A0D2, 0xD8542F68, 0x960FA728, 0xAB5133A3, 0x6EEF0B6C,
0x137A3BE4, 0xBA3BF050, 0x7EFF2A98,
44 0xA1F1651D, 0x39AF0176, 0x66CA593E,
0x82430E88, 0x8CEE8619, 0x456F9FB4, 0x7D84A5C3, 0x3B8B5EBE,
45 0xE06F75D8, 0x85C12073, 0x401A449F,
0x56C16AA6, 0x4ED3AA62, 0x363F7706, 0x1BFEDF72, 0x429B023D,
46 0x37D0D724, 0xD00A1248, 0xDB0FEAD3, 0x49F1C09B,
0x075372C9, 0x80991B78, 0x25D479D8, 0xF6E8DEF7,
47 0xE3FE501A, 0xB86794C3B, 0x976CE0BD, 0x04C006BA, 0xC1A94F-
B6, 0x409F60C4, 0x5E5C9EC2, 0x196A2463,
48 0x68FB6FAF, 0x3E6C53B5, 0x1339B2EB, 0x3B52EC6F, 0x6DFC511F,
0x9B30952C, 0xCC814544, 0xAF5EBD09,
49 0xBEE3D004, 0xDE334AFD,
0x660F2807, 0x192E4BB3, 0x0C0BA857, 0x45C8740F, 0xD20B5F39, 0xB9D3F8DB,
50 0x5579C0BD, 0x1A60320A, 0xD6A100C6, 0x402C7279, 0x679F25FE, 0xF-
B1FA3CC, 0x8EA5E9F8, 0xD83222F8,
51 0x3C7516DF, 0xFD616B15, 0x2F501EC8, 0xAD0552AB, 0x323DB5FA, 0xF-
D238760, 0x53317B48, 0x3E00DF82,
52 0x9E5C57B8, 0xCA6F8CA0, 0x1A87562E, 0xDF1769DB,
0xD542A8F6, 0x287EFFC3, 0xACA6732C6, 0x8C4F5573,
53 0x695B27B0, 0xBBCA58C8, 0xE1FFA35D,
0xB8F011A0, 0x10FA3D98, 0xF02183B8, 0x4AFCB56C, 0x2DD1D35B,
54 0x9A53C479, 0xB6F84565, 0xD28E49BC, 0x4BF8790, 0xE1DDF2DA, 0x-
A4CB7E33, 0x62FB1341, 0xCEEA4C68,
55 0xEF20CADA, 0x36774C01, 0xD07E9EFE, 0x2BF11FB4, 0x95DBDA4D, 0x-
AE909198, 0xEAA8DE71, 0x68993D5A0,
56 0xD08ED1D0, 0xAF7275E0, 0x8E3C5B2F, 0x8E7594B7, 0x8FF6E2FB,
0xF2122B64, 0x8888B812, 0x900DF01C,
57 0x4FAD5EA0, 0x688FC31C, 0xD1CFF191, 0xB3A8C1AD,
0x2F2F2218, 0xBE0E1777, 0xEA752DFE, 0xB8021FA1,
58 0xE5A0CC0F,
0xB56F74E8, 0x18ACF3D6, 0xCE89E299, 0xB484FE0, 0xFD13E0B7, 0x7C443B81, 0
xD2ADAD89,
59 0x165-
FA266, 0x80957705, 0x93CC7314, 0x211A1477, 0xE6AD2065, 0x77B5FA86, 0xC7544
2F5, 0xFB9D35CF,
60 0xEBCDAF0C, 0x7B3E89A0, 0xD6411BD3, 0xAE1E7E49, 0x00250E2D,
0x2071B35E, 0x226800BB, 0x57B8E0AF,
61 0x24643698, 0xF009B91E, 0x5563911D, 0x59DFA6AA,
0x78C14389, 0xD95A537F, 0x207D5BA2, 0x02E5B9C5,
62 0x83260376, 0x6295CFA9, 0x11C81968, 0x4E734A41, 0xB3472DCA,
0x7B14A94A, 0x1B510052, 0x9A532915,
63 0xD60F573F, 0xBC9B-
C6E4, 0x2B60A476, 0x81E67400, 0x08BA6FB5, 0x571BE91F, 0xF296EC6B, 0x2A0D-
D915,
64 0xB6636521, 0xE7B79FB6, 0xFF34052E, 0xC5855664, 0x53B02D5D, 0x-
A998FA1, 0x08BA4799, 0x6E85076A
65 }, {
66 0x4B7A70E9, 0xB5B32944, 0xDB75092E, 0x-
C4192623, 0xAD6EA6B0, 0x49A7DF7D, 0x9CEEE60B8, 0x8FEDB266,
67 0xECAA8C71, 0x699A17FF, 0x5664526C, 0x-
C2B19EE1, 0x193602A5, 0x75094C29, 0xA0591340, 0xE4183A3E,
68 0x3F54989A,
0x5B429D65, 0x6B8FE4D6, 0x99F73FD6, 0xA1D29C07, 0xEFE830F5, 0x4D2D38E6, 0
xF0255DC1,
69 0x4CDD2086, 0x8470EB26, 0x6382E9C6, 0x021ECC5E, 0x09686B3F,
0x3EBAEFC9, 0x3C971814, 0x6B6A70A1,
70 0x687F3584, 0x52A0E286, 0xB79C5305, 0xAA500737, 0x3E07841C,
0x7FDEAESC, 0x8E7D44EC, 0x5716F2B8,
71 0xB03ADA37, 0xF0500C0D, 0xF01C1F04, 0x0200B3FF, 0xAE0CF51A,
0x3CB574B2, 0x25837A58, 0xDC0921BD,
72
0xD19113F9, 0x7CA92FF6, 0x94324773, 0x22F54701, 0x3AE5E581, 0x37C2DADC,
0x8B57634, 0x9AF3DDA7,
73 0xA9446146, 0x0FD0030E, 0xECC8C73E, 0x-
A4751E41, 0xE238CD99, 0x3BEA0E2F, 0x3280BBA1, 0x183EB331,
74 0x4E548B38, 0x4F60B908, 0x6F42D0D3, 0xF60A04BF,
0x2CB81290, 0x24977C79, 0x5679B072, 0xBCAF89AF,
75 0xDE9A771F, 0xD9930810, 0xB38BAE12, 0xDCCF3F2E, 0x5512721F,
0x2E6B7124, 0x501ADD06, 0x9F84CD87,
76 0x7A584718, 0x7408DA17, 0xB0C9F9ABC, 0xE94B7D8C, 0xEC7AEC3A, 0xD-
B851DFA, 0x63094366, 0xC464C3D2,
77
0xEF1C1847, 0x3215D908, 0xDD433B37, 0x24C2BA16, 0x12A14D43, 0x2A65C451, 0
x50940002, 0x133AE4DD,
78 0x71DDF89E, 0x10314E55, 0x81AC77D6, 0x5F11199B,
0x043556F1, 0xD7A3C76B, 0x3C11183B, 0x5924A509,
79 0xF28FE6ED, 0x9771FBFA, 0x9EAB8F2C, 0x1E153C6E,
0x86E34570, 0xEAE96FB1, 0x806E5E0A, 0x5A3E2AB3,
80 0x771FE71C, 0x4E3D06FA, 0x2965DCB9, 0x99E71D0F,
0x803E89D6, 0x5266C825, 0x2E4CC978, 0x94C10B36A,
81 0xC6150EBA,
0x94E2EA78, 0xA5FC3C53, 0x1E0A2DF4, 0xF2F74EA7, 0x361D2B3D, 0x1939260F,
0x19C27960,
82 0x5223A708, 0xF71312B6, 0xEBADF6E6,
0xEAC31F66, 0xE3BC4595, 0xA67BC883, 0xB17F37D1, 0x018CFF28,
83 0xC332DDEF, 0xBE6C5A85, 0x65582185, 0x68AB9802, 0xEECEAE50F, 0xD-
B2F953B, 0x2AEF7DAD, 0x586E2F84,
84
0x1521B628, 0x29076170, 0xECDD4775, 0x619F1510, 0x13CCA830, 0xEB61BD96, 0
x0334FE1E, 0xAA0363CF,
85 0xB573C90, 0x4C70A239, 0xD59E9E0B, 0xCBAADE14, 0xEECC86BC,
0x60622CA7, 0x9CAB5CAB, 0xB2F3846E,
86 0x648B1EAF, 0x19BDF0CA, 0x-

```

```

A02369B9, 0x655ABB50, 0x40685A32, 0x3C2AB4B3, 0x319EE9D5, 0xC021B8F7,
87 0x9B540B19, 0x875FA099, 0x95F7997E, 0x623D7DA8, 0xF837889A,
0x97FE32D7, 0x11ED935F, 0x16681281,
88
0x0E358829, 0xC7E61FD6, 0x96DEDAF1, 0x7858BA99, 0x57F584A5, 0x1B227263, 0
x98B3C3FF, 0x1AC24696,
89 0xCDB30AEB, 0x532E3054, 0x8FD948E4, 0x6DBC3128, 0x58EBF2EF,
0x34C6FFEA, 0xFE28ED61, 0xEE7C3C73,
90
0x5D4A14D9, 0xE864B7E3, 0x42105D14, 0x203E13E0, 0x45EEE2B6, 0xA3AAABEA,
0xDB6C4F15, 0xFACB4FD0,
91 0xC742F442, 0xEF6ABBB5, 0x654F3B1D, 0x41CD2105, 0xD81E799E,
0x86854DC7, 0xE44B476A, 0x3D816250,
92 0x-
CF62A1F2, 0x58BD2646, 0xFC8883A0, 0xC1C7B6A3, 0x7F1524C3, 0x69CB7492, 0x47
84BA0B, 0x5692B285,
93 0x095BBF00, 0xAD19489D, 0x1462B174, 0x23820E00, 0x58428D2A,
0x0C55F5EA, 0x1DADF43E, 0x233F7061,
94 0x3372F092, 0x8D937E41, 0xD65FECF1, 0x6C223BDB,
0x7CDE3759, 0xCBE7460, 0x4085F2A7, 0xCE77326E,
95 0xA6078084, 0x19F8509E, 0xCE8EFD85, 0x61D99735, 0xA969A7AA, 0x-
C50C6C2, 0x5A04ABFC, 0x800BCADC,
96 0x9E447A2E, 0x-
C3453484, 0xFDD56705, 0x0E1E9EC9, 0xDB73DBD3, 0x105588CD, 0x675F-
DA79, 0xE3674340,
97 0xC5C43465, 0x713E38D8, 0x3D28F89E, 0xF16D-
FF20, 0x153E21E7, 0x8FB03D4A, 0xE6E39F28, 0xDB83ADF-
98 }, {
99 0xE93D5A68, 0x948140F7, 0xF64C261C,
0x94692934, 0x411520F7, 0x7602D4F7, 0xBCF46B2E, 0xD4A20068,
100 0xD4082471, 0x3320F46A, 0x43B7D4B7, 0x500061AF, 0x1E39F62E,
0x97244546, 0x14214F74, 0xBF8B8840,
101 0x4D95FC1D, 0x96B591AF, 0x70F4DD3, 0x66A02F45, 0xBFBC09EC,
0x03BD9785, 0x7FAC6DD0, 0x31CB8504,
102 0x96EB27B3, 0x55FD3941, 0xDA2547E6, 0xABCA0A9A,
0x28507825, 0x530429F4, 0x0A2C86DA, 0xEB966DFB,
103 0x68DC1462, 0xD7486900, 0x680EC0A4, 0x27A18DEE, 0x4F3F-
FEA2, 0xE887AD8C, 0xB58CE006, 0x7AF4D6B6,
104 0xAACE1E7C, 0xD3375FEC, 0x-
CE78A399, 0x406B2A42, 0x20FE9E35, 0xD9F385B9, 0xEE39D7AB, 0x3B124EBB,
105 0x1DC9-
FAF7, 0x4B6D1856, 0x26A36631, 0xEAE397B2, 0x3A6EFA74, 0xD5B4332, 0x6841E7
F7, 0xCA7820FB,
106 0xFB0AF54E, 0xD8FEB397, 0x454056AC, 0xBA489527, 0x55533A3A,
0x20838D87, 0xFE6BA9B7, 0xD096954B,
107 0x55A867BC, 0x-
A1159A58, 0xCCA92963, 0x99E1DB33, 0xA62A4A56, 0x3F3125F9, 0x5EF47E1C,
0x9029317C,
108 0xFDF8E802, 0x04272F70, 0x80BB155C,
0x05282CE3, 0x95C11548, 0xE4C66D22, 0x48C1133F, 0xC70F86DC,
109 0x07F9C9EE, 0x41041F0F, 0x404779A4, 0x5D886E17, 0x325F51EB,
0xD59BC0D1, 0xF2BCC18F, 0x41113564,
110 0x257B7834, 0x0620A9C60, 0xDFF8E8A3, 0x1F636C1B,
0x0E12B4C2, 0x02E1329E, 0xAF664FD1, 0xCAD18115,
111 0x6B2395E0, 0x333E92E1, 0x3B240B62, 0xEEBE9922, 0x85B2A20E,
0xE6BA0D99, 0xDE720C8C, 0x2DA2F728,
112 0xD0127845, 0x95B794FD, 0x647D0862, 0xE7CCF5F0, 0x5449A36F,
0xB77D48FA, 0xC39D0FD7, 0xF33E8D1E,
113 0x0A476341, 0x992EFF74, 0x3A6F6EAB, 0xF4F8F-
D37, 0xA812DC60, 0xA1EBDDF8, 0x991BE14C, 0xDBE66B0D,
114 0xC67B5510, 0x6D672C37, 0x2765D43B, 0xD-
CD0E804, 0xF1290DC7, 0xCC00FFA3, 0xB5390F92, 0x690FED0B,
115 0x667B9FFB, 0xCEDB7D9C, 0xA091CF0B,
0xD9155EA3, 0xBB132F88, 0x515BADA4, 0x7B9479BF, 0x763BD6EB,
116 0x37392EB3, 0xCC115979, 0x8026E297, 0xF42E312D,
0x6842ADA7, 0xC66A2B3B, 0x12754CCC, 0x782EF11C,
117
0x6A124237, 0xB79251E7, 0x06A1BBE6, 0x4BF6B350, 0x1A6B1018, 0x11CAEDFA,
0x3D25BD08, 0xE2E1C3C9,
118 0x44421659, 0x0A121386, 0xD90CEC6E, 0xD5ABEA2A, 0x64AF674E,
0xDA86A85F, 0xBEBFE988, 0x64E4C3FE,
119 0x9DB8C8057, 0xF0F7C086, 0x60787BF8, 0x6003604D, 0xD1F-
D8346, 0xF6381FB0, 0x7745EA04, 0xD736FCCC,
120 0x83426B33, 0xF01EAB71, 0xB0804187, 0x3C005E5F, 0x77A057BE, 0xB-
DEBAE24, 0x55464299, 0xBF582E61,
121 0x4E58F48F, 0xF2DDF-
DA2, 0xF474E-
F38, 0x8789BDC2, 0x5366F9C3, 0xC8B38E74, 0xB475F255, 0x46FCD9B9,
122 0x7AEB2661, 0x8B1DDF84, 0x846A0E79, 0x915F95E2, 0x466E598E,
0x20B45770, 0x8CD55591, 0xC902DE4C,
123 0xB90BACE1, 0xBB8205D0, 0x11A86248, 0x7574A99E,
0xB77F19B6, 0xE0A9DC09, 0x662D09A1, 0xC4324633,
124 0xE85A1F02, 0x09F0BE8C, 0x4A99A025, 0x1D6EFE10, 0x1AB93D1D,
0x0BA5A4DF, 0xA186F20F, 0x2868F169,
125 0xDCB7DA83, 0x573906FE, 0xA1E2CE9B,
0x4FCD7F52, 0x50115E01, 0xA70683FA, 0x0DE02B5C4, 0x0DE6D027,
126
0x9AF88C27, 0x773F8641, 0xC3604C06, 0x61A806B5, 0xF0177A28, 0xC0F586E0, 0
x06058AA, 0x390CD7D62,
127
0x11E69ED7, 0x2338EA63, 0x53C2DD94, 0xC2C21634, 0xBBCEEE56, 0x90BCB6DE,
0xEBFC7DA1, 0xC5E91D76,
128 0x6F05E409, 0x4B7C0188, 0x39720A3D, 0x7C927C24, 0x86E3725F,
0x724D9DB9, 0x1AC15BB4, 0xD39EB8FC,
129 0xED545578, 0x08FCA5B5, 0xD83D7CD3, 0x4DAD0FC4, 0x1E50EFE5E,
0xB161E6F8, 0xA28514D9, 0x6C51133C,
130 0x6FD5C7E7, 0x56E14EC4, 0x362ABFCE, 0xD-
DC6C837, 0xD79A3234, 0x92638212, 0x670EFA8E, 0x46600E0
131 }, {
132 0x3A39CE37, 0xD3FAF5CF, 0xABC27737, 0x5AC52D1B, 0x5CB0679E, 0x4-
FA33742, 0xD3822740, 0x99BC9BBE,

```

```

133 0xD5118E9D, 0xBF0F7315, 0xD62D1C7E, 0xC700C47B, 0xB78C1B6B,
0x21A19045, 0xB26EB1BE, 0x6A366EB4,
134 0x5748AB2F, 0xBC946E79, 0xC6A376D2, 0x6549C2C8, 0x530FF8EE,
0x468DDE7D, 0xD5730A1D, 0x4CD04DC6,
135 0x2939BBD8, 0x-
A9BA4650, 0xAC9526E8, 0xBE5EE304, 0xA1FAD5F0, 0xA6A2D519A, 0x63E-
F8CE2, 0x9A86EE22,
136 0xC089C2B8, 0x43242EF6, 0xA51E03AA, 0x9CF2D0A4, 0x83C061BA,
0x9BE96A4D, 0x8FE51550, 0xBA645BD6,
137
0x2826A2F9, 0xA73A3AE1, 0x4BA99586, 0xEF5562E9, 0xC72FEFD3, 0xF752F7DA,
0x3F046F69, 0x77FA0A59,
138 0x80E4A915, 0x87B08601, 0x9B09E6AD, 0x3B3EE593, 0xE990FD5A,
0x9E34D797, 0x2CF0B7D9, 0x022B8B51,
139 0x96D5AC3A, 0x017DA67D, 0xD1CF3ED6, 0x7C7D2D28, 0x1F9F25CF, 0x-
ADF2B89B, 0x5AD6B472, 0x5A88F54C,
140 0xE029AC71, 0xE019A5E6, 0x47B0ACFD, 0xED93FA9B, 0xE8D3C48D,
0x283B57CC, 0xF8D56629, 0x79132E28,
141 0x785F0191, 0xED756055, 0xF7960E44, 0xE3D35E8C, 0x15056D-
D4, 0x88F46DBA, 0x03A16125, 0x0564F0BD,
142 0xC3EB9E15, 0x3C9057A2, 0x97271AEC, 0xA93A072A, 0x1B3F6D9B,
0x1E6321F5, 0xF59C66FB, 0x26DCF319,
143 0x7533D928, 0xB155FDF5, 0x03563482, 0x8ABA3CBB,
0x28517711, 0xC2AD9F8, 0xABCC5167, 0xCCAD925F,
144 0x4DE81751, 0x3830DC8E,
0x379D5862, 0x9320F991, 0xEA7A90C2, 0xFB3E7BCE, 0x5121CE64, 0x774FBE32,
145 0xA8B6E37E, 0x-
C3293D46, 0x48DE5369, 0x6413E680, 0xA2AE0810, 0xDD6DB224, 0x69852DFD,
0x09072166,
146 0xB39A460A, 0x6445C0DD, 0x586CDECF, 0x1C20C8AE, 0x5BBEF7DD,
0x1B588D40, 0xCCD2017F, 0x6BB4E3BB,
147 0xDDA26A7E,
0x3A59FF45, 0x3E350A44, 0xBCB4CDD5, 0x72EACEA8, 0xFA6484BB, 0x8D6612AE,
0xBF3C6F47,
148 0xD29BE463, 0x542F5D9E, 0xAEC2771B, 0xF64E6370, 0x740E08D0,
0xE75B1357, 0xF8721671, 0xAF537D5D,
149 0x4040CB08, 0x4EB4E2CC, 0x34D2466A,
0x0115AF84, 0xE1B00428, 0x95983A1D, 0x06B89FB4, 0xCE6EA048,
150 0x6F3F3B82, 0x3520AB82, 0x011A1D4B,
0x277227F8, 0x611560B1, 0xE7933FDC, 0xBB3A792B, 0x344525BD,
151 0xA08839E1, 0x51CE794B, 0x2F32C9B7, 0xA01FBAC9, 0xE01CC87E, 0xB-
CC7D1F6, 0xCF0111C3, 0xA1E8AAC7,
152 0x1A908749, 0xD44FB09A, 0xD0DADECB, 0xD50ADA38, 0x0339C32A, 0x-
C6913667, 0x8DF9317C, 0xE0B12B4F,
153 0xF79E59B7, 0x43F5BB3A, 0xF2D519FF, 0x27D9459C, 0xBF97222C,
0x15E6FC2A, 0x0F91FC71, 0x9B941525,
154 0xFAE59361, 0xCEB69CEB, 0xC2A86459, 0x12BAA8D1, 0xB6C1075E,
0xE3056A0C, 0x10D25065, 0xC803A442,
155 0xE0EC6E0E, 0x1698DB3B, 0x4C98A0BE,
0x3278E964, 0x9F1F9532, 0xE0D392DF, 0xD3A0342B, 0x8971F21E,
156 0x1B0A7441, 0x4BA3348C, 0xC5BE7120, 0xC37632D8, 0xDF359F8D,
0x9B992F2E, 0xE60B6F47, 0x0FE3F11D,
157 0xE54CDA54, 0x1EDAD891, 0xCE6279CF, 0xCD3E7E6F,
0x1618B166, 0xFD2C1D05, 0x848FD2C5, 0xF6FB2299,
158
0xF523F357, 0xA6327623, 0x93A83531, 0x56CCDC0D, 0xACF08162, 0x5A75EBB5, 0
x6E163697, 0x88D273CC,
159 0xDE966292, 0x81B949D0, 0x4C50901B, 0x71C65614, 0xE6C6C7BD,
0x327A140A, 0x45E1D006, 0xC3F27B9A,
160 0xC9AA53FD,
0x62A80F00, 0xBB25BF62, 0x35BDD2F6, 0x71126905, 0xB2040222, 0xB6CBCF7C,
0xCD769C2B,
161 0x53113EC0, 0x1640E3D3, 0x38ABBD60, 0x2547ADF0, 0xBA38209C,
0xF746CE76, 0x77AFA1C5, 0x20756060,
162 0x85CBFE4E, 0x8AE88DD8, 0x7AAAF9B0, 0x4CF9AA7E, 0x1948C25C,
0x02FB8A8C, 0x01C36AE4, 0xD6EBE1F9,
163 0x90D4F869, 0xA65CDEA0, 0x3F09252D, 0xC208E69F,
0xB74E6132, 0xCE77E25B, 0x578FDFE3, 0x3AC372E6
164 } };
165
166 /***** FUNCTION DEFINITIONS *****/
167 void blowfish_encrypt(const BYTE in[], BYTE out[], const BLOW-
FISH_KEY *keystuct)
168 {
169     WORD l, r, t; //, i;
170
171     l = (in[0] << 24) | (in[1] << 16) | (in[2] << 8) | (in[3]);
172     r = (in[4] << 24) | (in[5] << 16) | (in[6] << 8) | (in[7]);
173
174     ITERATION(l, r, t, 0);
175     ITERATION(l, r, t, 1);
176     ITERATION(l, r, t, 2);
177     ITERATION(l, r, t, 3);
178     ITERATION(l, r, t, 4);
179     ITERATION(l, r, t, 5);
180     ITERATION(l, r, t, 6);
181     ITERATION(l, r, t, 7);
182     ITERATION(l, r, t, 8);
183     ITERATION(l, r, t, 9);
184     ITERATION(l, r, t, 10);
185     ITERATION(l, r, t, 11);
186     ITERATION(l, r, t, 12);
187     ITERATION(l, r, t, 13);
188     ITERATION(l, r, t, 14);
189     l ^= keystuct->p[15]; F(l, t); r ^= t; //Last iteration has
no swap()
190     r ^= keystuct->p[16];
191     l ^= keystuct->p[17];
192
193     out[0] = l >> 24;

```

```

194     out[1] = l >> 16;
195     out[2] = l >> 8;
196     out[3] = l;
197     out[4] = r >> 24;
198     out[5] = r >> 16;
199     out[6] = r >> 8;
200     out[7] = r;
201 }
202
203 void blowfish_decrypt(const BYTE in[], BYTE out[], const BLOW-
FISH_KEY *keystuct)
204 {
205     WORD l, r, t; //, i;
206
207     l = (in[0] << 24) | (in[1] << 16) | (in[2] << 8) | (in[3]);
208     r = (in[4] << 24) | (in[5] << 16) | (in[6] << 8) | (in[7]);
209
210     ITERATION(l, r, t, 17);
211     ITERATION(l, r, t, 16);
212     ITERATION(l, r, t, 15);
213     ITERATION(l, r, t, 14);
214     ITERATION(l, r, t, 13);
215     ITERATION(l, r, t, 12);
216     ITERATION(l, r, t, 11);
217     ITERATION(l, r, t, 10);
218     ITERATION(l, r, t, 9);
219     ITERATION(l, r, t, 8);
220     ITERATION(l, r, t, 7);
221     ITERATION(l, r, t, 6);
222     ITERATION(l, r, t, 5);
223     ITERATION(l, r, t, 4);
224     ITERATION(l, r, t, 3);
225     l ^= keystuct->p[2]; F(l, t); r ^= t; //Last iteration has
no swap()
226     r ^= keystuct->p[1];
227     l ^= keystuct->p[0];
228
229     out[0] = l >> 24;
230     out[1] = l >> 16;
231     out[2] = l >> 8;
232     out[3] = l;
233     out[4] = r >> 24;
234     out[5] = r >> 16;
235     out[6] = r >> 8;
236     out[7] = r;
237 }
238
239 void blowfish_key_setup(const BYTE user_key[], BLOWFISH_KEY
*keystuct, size_t len)
240 {
241     BYTE block[8];
242     int idx, idx2;
243
244     // Copy over the constant init array vals (so the originals
aren't destroyed).
245     memcpy(keystuct->p, p_perm, sizeof(WORD) * 18);
246     memcpy(keystuct->s, s_perm, sizeof(WORD) * 1024);
247
248     // Combine the key with the P box. Assume key is standard
448 bits (56 bytes) or less.
249     for (idx = 0, idx2 = 0; idx < 18; ++idx, idx2 += 4)
250         keystuct->p[idx] ^= (user_key[idx2 % len] << 24) |
(user_key[(idx2+1) % len] << 16)
| (user_key[(idx2+2) % len] << 8) |
(user_key[(idx2+3) % len]);
251
252     // Re-calculate the P box.
253     memset(block, 0, 8);
254     for (idx = 0; idx < 18; idx += 2) {
255         blowfish_encrypt(block, block, keystuct);
256         keystuct->p[idx] = (block[0] << 24) | (block[1] << 16)
| (block[2] << 8) | block[3];
257         keystuct->p[idx+1] = (block[4] << 24) | (block[5] << 16)
| (block[6] << 8) | block[7];
258     }
259     // Recalculate the S-boxes.
260     for (idx = 0; idx < 4; ++idx) {
261         for (idx2 = 0; idx2 < 256; idx2 += 2) {
262             blowfish_encrypt(block, block, keystuct);
263             keystuct->s[idx][idx2] = (block[0] << 24) |
(block[1] << 16) |
264 (block[2] << 8) | block[3];
265             keystuct->s[idx][idx2+1] = (block[4] << 24) |
(block[5] << 16) |
266 (block[6] << 8) |
block[7];
267         }
268     }
269 }

```

11. B-Con_crypto-algorithms/blowfish.h (32 lines)

```

1 /
*****
*
2 * Filename:    blowfish.h
3 * Author:      Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer:   This code is presented "as is" without any guar-

```

```

antees.
6 * Details:    Defines the API for the corresponding Blowfish
implementation.
7
8 /*****
9 #ifndef BLOWFISH_H
10 #define BLOWFISH_H
11
12 /***** HEADER FILES
13 #include <stddef.h>
14
15 /***** MACROS
16 #define BLOWFISH_BLOCK_SIZE 8          // Blowfish operates
on 8 bytes at a time
17
18 /***** DATA TYPES
19 typedef unsigned char BYTE;            // 8-bit byte
20
21 typedef unsigned int WORD;             // 32-bit word, change to
"long" for 16-bit machines
22
23 typedef struct {
24     WORD p[18];
25     WORD s[4][256];
26 } BLOWFISH_KEY;
27
28 /***** FUNCTION DECLARATIONS
29 void blowfish_key_setup(const BYTE user_key[], BLOWFISH_KEY
*keystruct, size_t len);
30 void blowfish_encrypt(const BYTE in[], BYTE out[], const BLOW-
FISH_KEY *keystruct);
31 void blowfish_decrypt(const BYTE in[], BYTE out[], const BLOW-
FISH_KEY *keystruct);
32 #endif // BLOWFISH_H

```

12. B-Con_crypto-algorithms/blowfish_test.c (68 lines)

```

1 /
2 * Filename:    blowfish_test.c
3 * Author:      Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer:   This code is presented "as is" without any guar-
antees.
6
7 * Details:     Performs known-answer tests on the corresponding
Blowfish
8               implementation. These tests do not encompass
the full
9               range of available test vectors, however, if
pass it is very, very likely that the code is
correct
10              and was compiled properly. This code also
serves as
11              example usage of the functions.
12
13 /*****
14 #include <stdio.h>
15 #include <memory.h>
16 #include "blowfish.h"
17
18 /***** FUNCTION DEFINITIONS
19
20 int blowfish_test()
21 {
22     BYTE key1[8] =
{0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00};
23     BYTE key2[8] = {0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff};
24     BYTE key3[24] =
{0xf0, 0xe1, 0xd2, 0xc3, 0xb4, 0xa5, 0x96, 0x87,
25     0x78, 0x69, 0x5a, 0x4b, 0x3c, 0x2d, 0x1e,
0x0f,
26     0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77};
27     BYTE p1[BLOWFISH_BLOCK_SIZE] =
{0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00};
28     BYTE p2[BLOWFISH_BLOCK_SIZE] = {0xff, 0xff, 0xff, 0xff,
0xff, 0xff, 0xff, 0xff};
29     BYTE p3[BLOWFISH_BLOCK_SIZE] = {0xfe, 0xdc, 0xba,
0x98, 0x76, 0x54, 0x32, 0x10};
30
31     BYTE c1[BLOWFISH_BLOCK_SIZE] = {0x4e,
0xf9, 0x97, 0x45, 0x61, 0x98, 0xdd, 0x78};
32     BYTE c2[BLOWFISH_BLOCK_SIZE] = {0x51, 0x86, 0x6f,
0xd5, 0xb8, 0x5e, 0xcb, 0x8a};

```

```

33     BYTE c3[BLOWFISH_BLOCK_SIZE] = {0x05, 0x04, 0x4b,
0x62, 0xfa, 0x52, 0xd0, 0x80};
34
35     BYTE enc_buf[BLOWFISH_BLOCK_SIZE];
36     BLOWFISH_KEY key;
37     int pass = 1;
38
39     // Test vector 1.
40     blowfish_key_setup(key1, &key, BLOWFISH_BLOCK_SIZE);
41     blowfish_encrypt(p1, enc_buf, &key);
42     pass = pass && !memcmp(c1, enc_buf, BLOW-
FISH_BLOCK_SIZE);
43     blowfish_decrypt(c1, enc_buf, &key);
44     pass = pass && !memcmp(p1, enc_buf, BLOW-
FISH_BLOCK_SIZE);
45
46     // Test vector 2.
47     blowfish_key_setup(key2, &key, BLOWFISH_BLOCK_SIZE);
48     blowfish_encrypt(p2, enc_buf, &key);
49     pass = pass && !memcmp(c2, enc_buf, BLOW-
FISH_BLOCK_SIZE);
50     blowfish_decrypt(c2, enc_buf, &key);
51     pass = pass && !memcmp(p2, enc_buf, BLOW-
FISH_BLOCK_SIZE);
52
53     // Test vector 3.
54     blowfish_key_setup(key3, &key, 24);
55     blowfish_encrypt(p3, enc_buf, &key);
56     pass = pass && !memcmp(c3, enc_buf, BLOW-
FISH_BLOCK_SIZE);
57     blowfish_decrypt(c3, enc_buf, &key);
58     pass = pass && !memcmp(p3, enc_buf, BLOW-
FISH_BLOCK_SIZE);
59
60     return(pass);
61 }
62
63 int main()
64 {
65     printf("Blowfish tests: %s\n", blowfish_test() ? "SUC-
CEDED" : "FAILED");
66
67     return(0);
68 }

```

13. B-Con_crypto-algorithms/des.c (269 lines)

```

1 /
2 * Filename:    des.c
3 * Author:      Brad Conte (brad AT radconte.com)
4 * Copyright:
5 * Disclaimer:   This code is presented "as is" without any guar-
antees.
6
7 * Details:     Implementation of the DES encryption algorithm.
Modes of operation (such as CBC) are not in-
cluded.
8               The formal NIST algorithm specification can be
found here:
9               * http://csrc.nist.gov/publications/fips/fips46-3/fips46-3.pdf
10
11 /*****
12 #include <stdlib.h>
13 #include <memory.h>
14 #include "des.h"
15
16 /***** MACROS
17
18 // Obtain bit "b" from the left and shift it "c" places from
the right
19 #define BITNUM(a,b,c) (((a[(b)/8] >> (7 - (b%8))) & 0x01) <<
(c))
20 #define BITNUMINTR(a,b,c) (((a >> (31 - (b))) & 0x00000001)
<< (c))
21 #define BITNUMINTL(a,b,c) (((a << (b)) & 0x80000000) >> (c))
22
23 // This macro converts a 6 bit block with the S-Box row
defined as the first and last
24 // bits to a 6 bit block with the row defined by the first two
bits.
25 #define SBOXBIT(a) (((a) & 0x20) | (((a) & 0x1f) >> 1) | (((a) &
0x01) << 4))
26
27 /***** VARIABLES
28 static const BYTE sbox1[64] = {
29     14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12,
5, 9, 0, 7,
30     8, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11,
9, 5, 3, 0,
31     4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7,
3, 10, 5, 0,

```



```

32      15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14,
10, 0, 6, 13
33 };
34 static const BYTE sbox2[64] = {
35     15, 1, 8, 14, 6, 11, 3, 4, 9, 7, 2, 13,
12, 0, 5, 10,
36     3, 13, 4, 7, 15, 2, 8, 14, 12, 0, 1, 10,
6, 9, 11, 5,
37     0, 14, 7, 11, 10, 4, 13, 1, 5, 8, 12, 6,
9, 3, 2, 15,
38     13, 8, 10, 1, 3, 15, 4, 2, 11, 6, 7, 12,
0, 5, 14, 9
39 };
40 static const BYTE sbox3[64] = {
41     10, 0, 9, 14, 6, 3, 15, 5, 1, 13, 12, 7,
11, 4, 2, 8,
42     13, 7, 0, 9, 3, 4, 6, 10, 2, 8, 5, 14,
12, 11, 15, 1,
43     13, 6, 4, 9, 8, 15, 3, 0, 11, 1, 2, 12,
5, 10, 14, 7,
44     1, 10, 13, 0, 6, 9, 8, 7, 4, 15, 14, 3,
11, 5, 2, 12
45 };
46 static const BYTE sbox4[64] = {
47     7, 13, 14, 3, 0, 6, 9, 10, 1, 2, 8, 5,
11, 12, 4, 15,
48     13, 8, 11, 5, 6, 15, 0, 3, 4, 7, 2, 12,
1, 10, 14, 9,
49     10, 6, 9, 0, 12, 11, 7, 13, 15, 1, 3, 14,
5, 2, 8, 4,
50     3, 15, 0, 6, 10, 1, 13, 8, 9, 4, 5, 11,
12, 7, 2, 14
51 };
52 static const BYTE sbox5[64] = {
53     2, 12, 4, 1, 7, 10, 11, 6, 8, 5, 3, 15,
13, 0, 14, 9,
54     14, 11, 2, 12, 4, 7, 13, 1, 5, 0, 15, 10,
3, 9, 8, 6,
55     4, 2, 1, 11, 10, 13, 7, 8, 15, 9, 12, 5,
6, 3, 0, 14,
56     11, 8, 12, 7, 1, 14, 2, 13, 6, 15, 0, 9,
10, 4, 5, 3
57 };
58 static const BYTE sbox6[64] = {
59     12, 1, 10, 15, 9, 2, 6, 8, 0, 13, 3, 4,
14, 7, 5, 11,
60     10, 15, 4, 2, 7, 12, 9, 5, 6, 1, 13, 14,
0, 11, 3, 8,
61     9, 14, 15, 5, 2, 8, 12, 3, 7, 0, 4, 10,
1, 13, 11, 6,
62     4, 3, 2, 12, 9, 5, 15, 10, 11, 14, 1, 7,
6, 0, 8, 13
63 };
64 static const BYTE sbox7[64] = {
65     4, 11, 2, 14, 15, 0, 8, 13, 3, 12, 9, 7,
5, 10, 6, 1,
66     13, 0, 11, 7, 4, 9, 1, 10, 14, 3, 5, 12,
2, 15, 8, 6,
67     1, 4, 11, 13, 12, 3, 7, 14, 10, 15, 6, 8,
0, 5, 9, 2,
68     6, 11, 13, 8, 1, 4, 10, 7, 9, 5, 0, 15,
14, 2, 3, 12
69 };
70 static const BYTE sbox8[64] = {
71     13, 2, 8, 4, 6, 15, 11, 1, 10, 9, 3, 14,
5, 0, 12, 7,
72     1, 15, 13, 8, 10, 3, 7, 4, 12, 5, 6, 11,
0, 14, 9, 2,
73     7, 11, 4, 1, 9, 12, 14, 2, 0, 6, 10, 13,
15, 3, 5, 8,
74     2, 1, 14, 7, 4, 10, 8, 13, 15, 12, 9, 0,
3, 5, 6, 11
75 };
76
77 /***** FUNCTION DEFINITIONS *****/
78 // Initial (Inv)Permutation step
79 void IP(WORD state[], const BYTE in[])
80 {
81     state[0] = BITNUM(in, 57, 31) | BITNUM(in, 49, 30) | BIT-
NUM(in, 41, 29) | BITNUM(in, 33, 28) |
82     BITNUM(in, 25, 27) | BITNUM(in,
17, 26) | BITNUM(in, 9, 25) | BITNUM(in, 1, 24) |
83     BITNUM(in, 59, 23) | BITNUM(in,
51, 22) | BITNUM(in, 43, 21) | BITNUM(in, 35, 20) |
84     BITNUM(in, 27, 19) | BITNUM(in,
19, 18) | BITNUM(in, 11, 17) | BITNUM(in, 3, 16) |
85     BITNUM(in, 61, 15) | BITNUM(in,
53, 14) | BITNUM(in, 45, 13) | BITNUM(in, 37, 12) |
86     BITNUM(in, 29, 11) | BITNUM(in,
21, 10) | BITNUM(in, 13, 9) | BITNUM(in, 5, 8) |
87     BITNUM(in, 63, 7) | BITNUM(in,
55, 6) | BITNUM(in, 47, 5) | BITNUM(in, 39, 4) |
88     BITNUM(in, 31, 3) | BITNUM(in,
23, 2) | BITNUM(in, 15, 1) | BITNUM(in, 7, 0);
89
90     state[1] = BITNUM(in, 56, 31) | BITNUM(in, 48, 30) | BIT-
NUM(in, 40, 29) | BITNUM(in, 32, 28) |
91     BITNUM(in, 24, 27) | BITNUM(in,
16, 26) | BITNUM(in, 8, 25) | BITNUM(in, 0, 24) |
92     BITNUM(in, 58, 23) | BITNUM(in,

```

```

50, 22) | BITNUM(in, 42, 21) | BITNUM(in, 34, 20) |
93     BITNUM(in, 26, 19) | BITNUM(in,
18, 18) | BITNUM(in, 10, 17) | BITNUM(in, 2, 16) |
94     BITNUM(in, 60, 15) | BITNUM(in,
52, 14) | BITNUM(in, 44, 13) | BITNUM(in, 36, 12) |
95     BITNUM(in, 28, 11) | BITNUM(in,
20, 10) | BITNUM(in, 12, 9) | BITNUM(in, 4, 8) |
96     BITNUM(in, 62, 7) | BITNUM(in,
54, 6) | BITNUM(in, 46, 5) | BITNUM(in, 38, 4) |
97     BITNUM(in, 30, 3) | BITNUM(in,
22, 2) | BITNUM(in, 14, 1) | BITNUM(in, 6, 0);
98 }
99
100 void InvIP(WORD state[], BYTE in[])
101 {
102     in[0] = BITNUMINTR(state[1], 7, 7) | BITNUMINTR(state[0],
7, 6) | BITNUMINTR(state[1], 15, 5) |
103     BITNUMINTR(state[0], 15, 4) | BITNUMIN-
TR(state[1], 23, 3) | BITNUMINTR(state[0], 23, 2) |
104     BITNUMINTR(state[1], 31, 1) | BITNUMIN-
TR(state[0], 31, 0);
105
106     in[1] = BITNUMINTR(state[1], 6, 7) | BITNUMINTR(state[0],
6, 6) | BITNUMINTR(state[1], 14, 5) |
107     BITNUMINTR(state[0], 14, 4) | BITNUMIN-
TR(state[1], 22, 3) | BITNUMINTR(state[0], 22, 2) |
108     BITNUMINTR(state[1], 30, 1) | BITNUMIN-
TR(state[0], 30, 0);
109
110     in[2] = BITNUMINTR(state[1], 5, 7) | BITNUMINTR(state[0],
5, 6) | BITNUMINTR(state[1], 13, 5) |
111     BITNUMINTR(state[0], 13, 4) | BITNUMIN-
TR(state[1], 21, 3) | BITNUMINTR(state[0], 21, 2) |
112     BITNUMINTR(state[1], 29, 1) | BITNUMIN-
TR(state[0], 29, 0);
113
114     in[3] = BITNUMINTR(state[1], 4, 7) | BITNUMINTR(state[0],
4, 6) | BITNUMINTR(state[1], 12, 5) |
115     BITNUMINTR(state[0], 12, 4) | BITNUMIN-
TR(state[1], 20, 3) | BITNUMINTR(state[0], 20, 2) |
116     BITNUMINTR(state[1], 28, 1) | BITNUMIN-
TR(state[0], 28, 0);
117
118     in[4] = BITNUMINTR(state[1], 3, 7) | BITNUMINTR(state[0],
3, 6) | BITNUMINTR(state[1], 11, 5) |
119     BITNUMINTR(state[0], 11, 4) | BITNUMIN-
TR(state[1], 19, 3) | BITNUMINTR(state[0], 19, 2) |
120     BITNUMINTR(state[1], 27, 1) | BITNUMIN-
TR(state[0], 27, 0);
121
122     in[5] = BITNUMINTR(state[1], 2, 7) | BITNUMINTR(state[0],
2, 6) | BITNUMINTR(state[1], 10, 5) |
123     BITNUMINTR(state[0], 10, 4) | BITNUMIN-
TR(state[1], 18, 3) | BITNUMINTR(state[0], 18, 2) |
124     BITNUMINTR(state[1], 26, 1) | BITNUMIN-
TR(state[0], 26, 0);
125
126     in[6] = BITNUMINTR(state[1], 1, 7) | BITNUMINTR(state[0],
1, 6) | BITNUMINTR(state[1], 9, 5) |
127     BITNUMINTR(state[0], 9, 4) | BITNUMIN-
TR(state[1], 17, 3) | BITNUMINTR(state[0], 17, 2) |
128     BITNUMINTR(state[1], 25, 1) | BITNUMIN-
TR(state[0], 25, 0);
129
130     in[7] = BITNUMINTR(state[1], 0, 7) | BITNUMINTR(state[0],
0, 6) | BITNUMINTR(state[1], 8, 5) |
131     BITNUMINTR(state[0], 8, 4) | BITNUMIN-
TR(state[1], 16, 3) | BITNUMINTR(state[0], 16, 2) |
132     BITNUMINTR(state[1], 24, 1) | BITNUMIN-
TR(state[0], 24, 0);
133 }
134
135 WORD f(WORD state, const BYTE key[])
136 {
137     BYTE lrgstate[6]; //, i;
138     WORD t1, t2;
139
140     // Expansion Permutation
141     t1 = BITNUMINTL(state, 31, 0) | ((state & 0xf0000000) >>
1) | BITNUMINTL(state, 4, 5) |
142     BITNUMINTL(state, 3, 6) | ((state & 0xf0000000)
>> 3) | BITNUMINTL(state, 8, 11) |
143     BITNUMINTL(state, 7, 12) | ((state &
0x00f00000) >> 5) | BITNUMINTL(state, 12, 17) |
144     BITNUMINTL(state, 11, 18) | ((state &
0x000f0000) >> 7) | BITNUMINTL(state, 16, 23);
145
146     t2 = BITNUMINTL(state, 15, 0) | ((state & 0x0000f000) <<
15) | BITNUMINTL(state, 20, 5) |
147     BITNUMINTL(state, 19, 6) | ((state &
0x00000f00) << 13) | BITNUMINTL(state, 24, 11) |
148     BITNUMINTL(state, 23, 12) | ((state &
0x000000f0) << 11) | BITNUMINTL(state, 28, 17) |
149     BITNUMINTL(state, 27, 18) | ((state &
0x0000000f) << 9) | BITNUMINTL(state, 0, 23);
150
151     lrgstate[0] = (t1 >> 24) & 0x000000ff;
152     lrgstate[1] = (t1 >> 16) & 0x000000ff;
153     lrgstate[2] = (t1 >> 8) & 0x000000ff;
154     lrgstate[3] = (t2 >> 24) & 0x000000ff;

```

```

155     lrgstate[4] = (t2 >> 16) & 0x000000ff;
156     lrgstate[5] = (t2 >> 8) & 0x000000ff;
157
158     // Key XOR
159     lrgstate[0] ^= key[0];
160     lrgstate[1] ^= key[1];
161     lrgstate[2] ^= key[2];
162     lrgstate[3] ^= key[3];
163     lrgstate[4] ^= key[4];
164     lrgstate[5] ^= key[5];
165
166     // S-Box Permutation
167     state = (sbox1[SBOXBIT(lrgstate[0] >> 2)] << 28) |
168             (sbox2[SBOXBIT(((lrgstate[0] & 0x03)
169 << 4) | (lrgstate[1] >> 4))] << 24) |
170             (sbox3[SBOXBIT(((lrgstate[1] & 0x0f)
171 << 2) | (lrgstate[2] >> 6))] << 20) |
172             (sbox4[SBOXBIT(lrgstate[2] & 0x3f)]
173 << 16) |
174             (sbox5[SBOXBIT(lrgstate[3] >> 2)] <<
175 12) |
176             (sbox6[SBOXBIT(((lrgstate[3] & 0x03)
177 << 4) | (lrgstate[4] >> 4))] << 8) |
178             (sbox7[SBOXBIT(((lrgstate[4] & 0x0f)
179 << 2) | (lrgstate[5] >> 6))] << 4) |
180             sbox8[SBOXBIT(lrgstate[5] &
181 0x3f)]);
182
183     // P-Box Permutation
184     state = BITNUMINTL(state,15,0) | BITNUMINTL(state,6,1)
185 | BITNUMINTL(state,19,2) |
186     BITNUMINTL(state,20,3) | BITNU-
187 MINTL(state,28,4) | BITNUMINTL(state,11,5) |
188     BITNUMINTL(state,27,6) | BITNU-
189 MINTL(state,16,7) | BITNUMINTL(state,0,8) |
190     BITNUMINTL(state,14,9) | BITNU-
191 MINTL(state,22,10) | BITNUMINTL(state,25,11) |
192     BITNUMINTL(state,4,12) | BITNU-
193 MINTL(state,17,13) | BITNUMINTL(state,30,14) |
194     BITNUMINTL(state,9,15) | BITNU-
195 MINTL(state,1,16) | BITNUMINTL(state,7,17) |
196     BITNUMINTL(state,23,18) | BITNU-
197 MINTL(state,13,19) | BITNUMINTL(state,31,20) |
198     BITNUMINTL(state,26,21) | BITNU-
199 MINTL(state,2,22) | BITNUMINTL(state,8,23) |
200     BITNUMINTL(state,18,24) | BITNU-
201 MINTL(state,12,25) | BITNUMINTL(state,29,26) |
202     BITNUMINTL(state,5,27) | BITNU-
203 MINTL(state,21,28) | BITNUMINTL(state,10,29) |
204     BITNUMINTL(state,3,30) | BITNU-
205 MINTL(state,24,31);
206
207     // Return the final state value
208     return(state);
209 }
210
211 void des_key_setup(const BYTE key[], BYTE schedule[][6],
212 DES_MODE mode)
213 {
214     WORD i, j, to_gen, C, D;
215     const WORD key_rnd_shift[16] =
216 {1,1,2,2,2,2,2,2,1,2,2,2,2,2,1};
217     const WORD key_perm_c[28] =
218 {56,48,40,32,24,16,8,0,57,49,41,33,25,17,
219 9,1,58,50,42,34,26,18,10,2,59,51,43,35};
220     const WORD key_perm_d[28] =
221 {62,54,46,38,30,22,14,6,61,53,45,37,29,21,
222 13,5,60,52,44,36,28,20,12,4,27,19,11,3};
223     const WORD key_compression[48] =
224 {13,16,10,23,0,4,2,27,14,5,20,9,
225 22,18,11,3,25,7,15,6,26,19,12,1,
226 40,51,30,36,46,54,29,39,50,44,32,47,
227 43,48,38,55,33,52,45,41,49,35,28,31};
228
229     // Permuted Choice #1 (copy the key in, ignoring par-
230 ity bits).
231     for (i = 0, j = 31, C = 0; i < 28; ++i, --j)
232         C |= BITNUM(key, key_perm_c[i], j);
233     for (i = 0, j = 31, D = 0; i < 28; ++i, --j)
234         D |= BITNUM(key, key_perm_d[i], j);
235
236     // Generate the 16 subkeys.
237     for (i = 0; i < 16; ++i) {
238         C = ((C << key_rnd_shift[i]) | (C >> (28-
239 key_rnd_shift[i]))) & 0xffffffff;
240         D = ((D << key_rnd_shift[i]) | (D >> (28-
241 key_rnd_shift[i]))) & 0xffffffff;
242
243         // Decryption subkeys are reverse order of en-
244 cryptation subkeys so
245
246         // generate them in reverse if the key schedule
247 is for decryption useage.
248         if (mode == DES_DECRYPT)
249             to_gen = 15 - i;
250         else /*(if mode == DES_ENCRYPT)*/

```

```

222         to_gen = i;
223         // Initialize the array
224         for (j = 0; j < 6; ++j)
225             schedule[to_gen][j] = 0;
226         for (j = 0; j < 24; ++j)
227             schedule[to_gen][j/8] |= BITNUMIN-
228 TR(C, key_compression[j], 7 - (j%8));
229         for (j < 48; ++j)
230             schedule[to_gen][j/8] |= BITNUMIN-
231 TR(D, key_compression[j] - 28, 7 - (j%8));
232     }
233 }
234 void des_crypt(const BYTE in[], BYTE out[], const BYTE key[]
235 [6])
236 {
237     WORD state[2], idx, t;
238     IP(state, in);
239     for (idx=0; idx < 15; ++idx) {
240         t = state[1];
241         state[1] = f(state[1], key[idx]) ^ state[0];
242         state[0] = t;
243     }
244
245     // Perform the final loop manually as it doesn't switch
246 sides
247     state[0] = f(state[1], key[15]) ^ state[0];
248     InvIP(state, out);
249 }
250 void three_des_key_setup(const BYTE key[], BYTE schedule[][16]
251 [6], DES_MODE mode)
252 {
253     if (mode == DES_ENCRYPT) {
254         des_key_setup(&key[0], schedule[0], mode);
255         des_key_setup(&key[8], schedule[1], !mode);
256         des_key_setup(&key[16], schedule[2], mode);
257     }
258     else /*if (mode == DES_DECRYPT)*/ {
259         des_key_setup(&key[16], schedule[0], mode);
260         des_key_setup(&key[8], schedule[1], !mode);
261         des_key_setup(&key[0], schedule[2], mode);
262     }
263 }
264 void three_des_crypt(const BYTE in[], BYTE out[], const BYTE
265 key[][16][6])
266 {
267     des_crypt(in, out, key[0]);
268     des_crypt(out, out, key[1]);
269     des_crypt(out, out, key[2]);
270 }

```

14. B-Con_crypto-algorithms/des.h (37 lines)

```

1 /
2 *****
3
4 * Filename:   des.h
5 * Author:    Brad Conte (brad AT bradconte.com)
6 * Copyright:
7 * Disclaimer: This code is presented "as is" without any guar-
8 antees.
9 * Details:   Defines the API for the corresponding DES imple-
10 mentation.
11
12 Note that encryption and decryption are defined
13 by how
14 the key setup is performed, the actual en/de-
15 cryptation is
16 performed by the same function.
17
18 *****
19 */
20
21 #ifndef DES_H
22 #define DESH
23
24 /***** HEADER FILES *****/
25 #include <stddef.h>
26
27 /***** MACROS *****/
28 #define DES_BLOCK_SIZE 8 // DES operates on 8
29 bytes at a time
30
31 /***** DATA TYPES *****/
32 typedef unsigned char BYTE; // 8-bit byte
33
34 typedef unsigned int WORD; // 32-bit word, change to
35 "long" for 16-bit machines
36
37 typedef enum {
38     DES_ENCRYPT,
39     DES_DECRYPT
40 } DES_MODE;

```

```

29
30 /***** FUNCTION DECLARATIONS *****/
31 void des_key_setup(const BYTE key[], BYTE schedule[][6],
DES_MODE mode);
32 void des_crypt(const BYTE in[], BYTE out[], const BYTE key[]
[6]);
33
34 void three_des_key_setup(const BYTE key[], BYTE schedule[][16]
[6], DES_MODE mode);
35 void three_des_crypt(const BYTE in[], BYTE out[], const BYTE
key[][16][6]);
36
37 #endif // DES_H

```

15. B-Con_crypto-algorithms/des_test.c (83 lines)

```

1 /
*****
2 * Filename:   des_test.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   Performs known-answer tests on the corresponding DES
7               implementation. These tests do not encompass
the full
8               range of available test vectors, however, if
the tests
9               pass it is very, very likely that the code is
correct
10              and was compiled properly. This code also
serves as
11              example usage of the functions.
12
*****
*/
13
14 /***** HEADER FILES *****/
15 #include <stdio.h>
16 #include <memory.h>
17 #include "des.h"
18
19 /***** FUNCTION DEFINITIONS *****/
20 int des_test()
21 {
22     BYTE pt1[DES_BLOCK_SIZE] =
{0x01,0x23,0x45,0x67,0x89,0xAB,0xCD,0xE7};
23     BYTE pt2[DES_BLOCK_SIZE] =
{0x01,0x23,0x45,0x67,0x89,0xAB,0xCD,0xEF};
24     BYTE pt3[DES_BLOCK_SIZE] =
{0x54,0x68,0x65,0x20,0x71,0x75,0x66,0x63};
25     BYTE ct1[DES_BLOCK_SIZE] = {0xc9,0x57,0x44,0x25,0x6a,
0x5e,0xd3,0x1d};
26     BYTE ct2[DES_BLOCK_SIZE] = {0x85,0xe8,0x13,0x54,0x0f,
0x0a,0xb4,0x05};
27     BYTE ct3[DES_BLOCK_SIZE] = {0xc9,0x57,0x44,0x25,0x6a,
0x5e,0xd3,0x1d};
28     BYTE ct4[DES_BLOCK_SIZE] = {0xA8,0x26,0xFD,0x8C,
0xE5,0x3B,0x85,0x5F};
29     BYTE key1[DES_BLOCK_SIZE] =
{0x01,0x23,0x45,0x67,0x89,0xAB,0xCD,0xEF};
30     BYTE key2[DES_BLOCK_SIZE] = {0x13,0x34,0x57,0x79,0x9B,
0xBC,0xDF,0xF1};
31     BYTE three_key1[DES_BLOCK_SIZE * 3] =
{0x01,0x23,0x45,0x67,0x89,0xAB,0xCD,0xEF,
0x01,0x23,0x45,0x67,0x89,0xAB,0xCD,0xEF,
0x01,0x23,0x45,0x67,0x89,0xAB,0xCD,0xEF};
34     BYTE three_key2[DES_BLOCK_SIZE * 3] =
{0x01,0x23,0x45,0x67,0x89,0xAB,0xCD,0xEF,
0x23,0x45,0x67,0x89,0xAB,0xCD,0xEF,0x01,
0x45,0x67,0x89,0xAB,0xCD,0xEF,0x01,0x23};
37     BYTE schedule[16][6];
38     BYTE three_schedule[3][16][6];
39     BYTE buf[DES_BLOCK_SIZE];
40     int pass = 1;
41
42     des_key_setup(key1, schedule, DES_ENCRYPT);
43     des_crypt(pt1, buf, schedule);
44     pass = pass && !memcmp(ct1, buf, DES_BLOCK_SIZE);
45
46     des_key_setup(key1, schedule, DES_DECRYPT);
47     des_crypt(ct1, buf, schedule);
48     pass = pass && !memcmp(pt1, buf, DES_BLOCK_SIZE);
49
50     des_key_setup(key2, schedule, DES_ENCRYPT);
51     des_crypt(pt2, buf, schedule);
52     pass = pass && !memcmp(ct2, buf, DES_BLOCK_SIZE);
53
54     des_key_setup(key2, schedule, DES_DECRYPT);
55

```

```

56     des_crypt(ct2, buf, schedule);
57     pass = pass && !memcmp(pt2, buf, DES_BLOCK_SIZE);
58
59     three_des_key_setup(three_key1, three_schedule, DES_EN-
CRYPT);
60     three_des_crypt(pt1, buf, three_schedule);
61     pass = pass && !memcmp(ct3, buf, DES_BLOCK_SIZE);
62
63     three_des_key_setup(three_key1, three_schedule, DES_DE-
CRYPT);
64     three_des_crypt(ct3, buf, three_schedule);
65     pass = pass && !memcmp(pt1, buf, DES_BLOCK_SIZE);
66
67     three_des_key_setup(three_key2, three_schedule, DES_EN-
CRYPT);
68     three_des_crypt(pt3, buf, three_schedule);
69     pass = pass && !memcmp(ct4, buf, DES_BLOCK_SIZE);
70
71     three_des_key_setup(three_key2, three_schedule, DES_DE-
CRYPT);
72     three_des_crypt(ct4, buf, three_schedule);
73     pass = pass && !memcmp(pt3, buf, DES_BLOCK_SIZE);
74
75     return(pass);
76 }
77
78 int main()
79 {
80     printf("DES test: %s\n", des_test() ? "SUCCEEDED" :
"FAILED");
81
82     return(0);
83 }

```

16. B-Con_crypto-algorithms/md2.c (104 lines)

```

1 /
*****
2 * Filename:   md2.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   Implementation of the MD2 hashing algorithm.
7               Algorithm specification can
be found here:
8               * http://tools.ietf.org/
html/rfc1319 .
9               Input is little endian byte order.
10
*****
*/
11
12 /***** HEADER FILES *****/
13 #include <stdlib.h>
14 #include <memory.h>
15 #include "md2.h"
16
17 /***** VARIABLES *****/
18 static const BYTE s[256] = {
19     41, 46, 67, 201, 162, 216, 124, 1, 61, 54, 84, 161,
236, 240, 6,
20     19, 98, 167, 5, 243, 192, 199, 115, 140, 152, 147, 43,
217, 188,
21     76, 130, 202, 30, 155, 87, 60, 253, 212, 224, 22, 103,
66, 111, 24,
22     138, 23, 229, 18, 190, 78, 196, 214, 218, 158, 222, 73,
160, 251,
23     245, 142, 187, 47, 238, 122, 169, 104, 121, 145, 21,
178, 7, 63,
24     148, 194, 16, 137, 11, 34, 95, 33, 128, 127, 93, 154,
90, 144, 50,
25     39, 53, 62, 204, 231, 191, 247, 151, 3, 255, 25, 48,
179, 72, 165,
26     181, 209, 215, 94, 146, 42, 172, 86, 170, 198, 79, 184,
56, 210,
27     150, 164, 125, 182, 118, 252, 107, 226, 156, 116, 4,
241, 69, 157,
28     112, 89, 100, 113, 135, 32, 134, 91, 207, 101, 230, 45,
168, 2, 27,
29     96, 37, 173, 174, 176, 185, 246, 28, 70, 97, 105, 52,
64, 126, 15,
30     85, 71, 163, 35, 221, 81, 175, 58, 195, 92, 249, 206,
186, 197,
31     234, 38, 44, 83, 13, 110, 133, 40, 132, 9, 211, 223,
205, 244, 65,
32     129, 77, 82, 106, 220, 55, 200, 108, 193, 171, 250, 36,
225, 123,
33     8, 12, 189, 177, 74, 120, 136, 149, 139, 227, 99, 232,
109, 233,
34     203, 213, 254, 59, 0, 29, 57, 242, 239, 183, 14, 102,
88, 208, 228,
35     166, 119, 114, 248, 235, 117, 75, 10, 49, 68, 80, 180,

```

```

143, 237,
36      31, 26, 219, 153, 141, 51, 159, 17, 131, 20
37  };
38
39  /***** FUNCTION DEFINITIONS *****/
40  void md2_transform(MD2_CTX *ctx, BYTE data[])
41  {
42      int j,k,t;
43
44      //memcpy(&ctx->state[16], data);
45      for (j=0; j < 16; ++j) {
46          ctx->state[j + 16] = data[j];
47          ctx->state[j + 32] = (ctx->state[j+16] ^ ctx-
>state[j]);
48      }
49
50      t = 0;
51      for (j = 0; j < 18; ++j) {
52          for (k = 0; k < 48; ++k) {
53              ctx->state[k] ^= s[t];
54              t = ctx->state[k];
55          }
56          t = (t+j) & 0xFF;
57      }
58
59      t = ctx->checksum[15];
60      for (j=0; j < 16; ++j) {
61          ctx->checksum[j] ^= s[data[j] ^ t];
62          t = ctx->checksum[j];
63      }
64  }
65
66  void md2_init(MD2_CTX *ctx)
67  {
68      int i;
69
70      for (i=0; i < 48; ++i)
71          ctx->state[i] = 0;
72      for (i=0; i < 16; ++i)
73          ctx->checksum[i] = 0;
74      ctx->len = 0;
75  }
76
77  void md2_update(MD2_CTX *ctx, const BYTE data[], size_t len)
78  {
79      size_t i;
80
81      for (i = 0; i < len; ++i) {
82          ctx->data[ctx->len] = data[i];
83          ctx->len++;
84          if (ctx->len == MD2_BLOCK_SIZE) {
85              md2_transform(ctx, ctx->data);
86              ctx->len = 0;
87          }
88      }
89  }
90
91  void md2_final(MD2_CTX *ctx, BYTE hash[])
92  {
93      int to_pad;
94
95      to_pad = MD2_BLOCK_SIZE - ctx->len;
96
97      while (ctx->len < MD2_BLOCK_SIZE)
98          ctx->data[ctx->len++] = to_pad;
99
100     md2_transform(ctx, ctx->data);
101     md2_transform(ctx, ctx->checksum);
102
103     memcpy(hash, ctx->state, MD2_BLOCK_SIZE);
104 }

```

17. B-Con_crypto-algorithms/md2.h (33 lines)

```

1  /
2  * Filename: md2.h
3  * Author: Brad Conte (brad AT bradconte.com)
4  * Copyright:
5  * Disclaimer: This code is presented "as is" without any guar-
antees.
6  * Details: Defines the API for the corresponding MD2 imple-
mentation.
7
*****/
8
9  #ifndef MD2_H
10 #define MD2_H
11
12 /***** HEADER FILES *****/
13 #include <stddef.h>
14
15 /***** MACROS *****/
16 #define MD2_BLOCK_SIZE 16
17

```

```

18 /***** DATA TYPES *****/
19 typedef unsigned char BYTE; // 8-bit byte
20
21 typedef struct {
22     BYTE data[16];
23     BYTE state[48];
24     BYTE checksum[16];
25     int len;
26 } MD2_CTX;
27
28 /***** FUNCTION DECLARATIONS *****/
29 void md2_init(MD2_CTX *ctx);
30 void md2_update(MD2_CTX *ctx, const BYTE data[], size_t len);
31 void md2_final(MD2_CTX *ctx, BYTE hash[]); // size of hash
must be MD2_BLOCK_SIZE
32
33 #endif // MD2_H

```

18. B-Con_crypto-algorithms/md2_test.c (58 lines)

```

1  /
2  * Filename: md2_test.c
3  * Author: Brad Conte (brad AT bradconte.com)
4  * Copyright:
5  * Disclaimer: This code is presented "as is" without any guar-
antees.
6
7  * Details: Performs known-answer tests on the corresponding MD2
the full implementation. These tests do not encompass
8 range of available test vectors, however, if
the tests
9
10 pass it is very, very likely that the code is
correct and was compiled properly. This code also
11 serves as an example usage of the functions.
12
*****/
13
14 /***** HEADER FILES *****/
15 #include <stdio.h>
16 #include <string.h>
17 #include <memory.h>
18 #include "md2.h"
19
20 /***** FUNCTION DEFINITIONS *****/
21 int md2_test()
22 {
23     BYTE text1[] = {"abc"};
24     BYTE text2[] = {"abcdefghijklmnopqrstuvwxyz"};
25     BYTE text3_1[] = {"ABCDEFGHIJKLMNOPQRSTUVWXYZabcde"};
26     BYTE text3_2[] = {"fghijklmnopqrstuvwxyz0123456789"};
27     BYTE hash1[MD2_BLOCK_SIZE] = {0xda, 0x85, 0x3b, 0x0d, 0x3f,
0xb8, 0xd9, 0x9b, 0x28, 0x3a, 0x69, 0xe6, 0xde, 0xd6, 0xbb};
28     BYTE hash2[MD2_BLOCK_SIZE] = {0x4e, 0x8d, 0xdf,
0xf3, 0x65, 0x02, 0x92, 0xab, 0x5a, 0x41, 0x08, 0xc3, 0xaa, 0x47, 0x94, 0x0b};
29     BYTE hash3[MD2_BLOCK_SIZE] = {0xda, 0x33, 0xde,
0xf2, 0xa4, 0x2d, 0xf1, 0x39, 0x75, 0x35, 0x28, 0x46, 0xc3, 0x03, 0x38, 0xcd};
30     BYTE buf[16];
31     MD2_CTX ctx;
32     int pass = 1;
33
34     md2_init(&ctx);
35     md2_update(&ctx, text1, strlen(text1));
36     md2_final(&ctx, buf);
37     pass = pass && !memcmp(hash1, buf, MD2_BLOCK_SIZE);
38
39     // Note that the MD2 object can be re-used.
40     md2_init(&ctx);
41     md2_update(&ctx, text2, strlen(text2));
42     md2_final(&ctx, buf);
43     pass = pass && !memcmp(hash2, buf, MD2_BLOCK_SIZE);
44
45     // Note that the data is added in two chunks.
46     md2_init(&ctx);
47     md2_update(&ctx, text3_1, strlen(text3_1));
48     md2_update(&ctx, text3_2, strlen(text3_2));
49     md2_final(&ctx, buf);
50     pass = pass && !memcmp(hash3, buf, MD2_BLOCK_SIZE);
51
52     return(pass);
53 }
54
55 int main()
56 {
57     printf("MD2 tests: %s\n", md2_test() ? "SUCCEEDED" :
"FAILED");
58 }

```

19. B-Con_crypto-algorithms/md5.c (189 lines)

```

1 /
*****
2 * Filename:   md5.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:    Implementation of the MD5 hashing algorithm.
7               Algorithm specification can
be found here:
8               * http://tools.ietf.org/
9               This implementation uses
little endian byte order.
10
*****
*/
11
12 /***** HEADER FILES *****/
13 #include <stdlib.h>
14 #include <memory.h>
15 #include "md5.h"
16
17 /***** MACROS *****/
18 #define ROTLEFT(a,b) ((a << b) | (a >> (32-b)))
19
20 #define F(x,y,z) ((x & y) | (~x & z))
21 #define G(x,y,z) ((x & z) | (y & ~z))
22 #define H(x,y,z) (x ^ y ^ z)
23 #define I(x,y,z) (y ^ (x | ~z))
24
25 #define FF(a,b,c,d,m,s,t) { a += F(b,c,d) + m + t; \
26                             a = b + ROTLEFT(a,s); }
27 #define GG(a,b,c,d,m,s,t) { a += G(b,c,d) + m + t; \
28                             a = b + ROTLEFT(a,s); }
29 #define HH(a,b,c,d,m,s,t) { a += H(b,c,d) + m + t; \
30                             a = b + ROTLEFT(a,s); }
31 #define II(a,b,c,d,m,s,t) { a += I(b,c,d) + m + t; \
32                             a = b + ROTLEFT(a,s); }
33
34 /***** FUNCTION DEFINITIONS *****/
35 void md5_transform(MD5_CTX *ctx, const BYTE data[])
36 {
37     WORD a, b, c, d, m[16], i, j;
38
39     // MD5 specifies big endian byte order, but this imple-
40     // mentation assumes a little
41     // endian byte order CPU. Reverse all the bytes upon
42     // input, and re-reverse them
43     // on output (in md5_final()).
44     for (i = 0, j = 0; i < 16; ++i, j += 4)
45         m[i] = (data[j] + (data[j + 1] << 8) + (data[j +
46 2] << 16) + (data[j + 3] << 24));
47
48     a = ctx->state[0];
49     b = ctx->state[1];
50     c = ctx->state[2];
51     d = ctx->state[3];
52
53     FF(a,b,c,d,m[0], 7,0xd76aa478);
54     FF(d,a,b,c,m[1], 12,0xe8c7b756);
55     FF(c,d,a,b,m[2], 17,0x242070db);
56     FF(b,c,d,a,m[3], 22,0xc1bdcee5);
57     FF(a,b,c,d,m[4], 7,0xf57c0faf);
58     FF(d,a,b,c,m[5], 12,0x4787c62a);
59     FF(c,d,a,b,m[6], 17,0xa8304613);
60     FF(b,c,d,a,m[7], 22,0xfd469501);
61     FF(a,b,c,d,m[8], 7,0x698098d8);
62     FF(d,a,b,c,m[9], 12,0x8b44f7af);
63     FF(c,d,a,b,m[10],17,0xffff5bb1);
64     FF(b,c,d,a,m[11],22,0x895cd7be);
65     FF(a,b,c,d,m[12], 7,0x6b901122);
66     FF(d,a,b,c,m[13],12,0xfd987193);
67     FF(c,d,a,b,m[14],17,0xa679438e);
68     FF(b,c,d,a,m[15],22,0x49b40821);
69
70     GG(a,b,c,d,m[1], 5,0xf61e2562);
71     GG(d,a,b,c,m[6], 9,0xc940b340);
72     GG(c,d,a,b,m[11],14,0x265e5a51);
73     GG(b,c,d,a,m[0], 20,0xe9b6c7aa);
74     GG(a,b,c,d,m[5], 5,0xd62f105d);
75     GG(d,a,b,c,m[10], 9,0x02441453);
76     GG(c,d,a,b,m[15],14,0xd8a1e681);
77     GG(b,c,d,a,m[4], 20,0xe7d3fbc8);
78     GG(a,b,c,d,m[9], 5,0x21e1cde6);
79     GG(d,a,b,c,m[14], 9,0xc33707d6);
80     GG(c,d,a,b,m[3], 14,0xf4d50d87);
81     GG(b,c,d,a,m[8], 20,0xa55a14ed);
82     GG(a,b,c,d,m[13], 5,0xa9e3e905);
83     GG(d,a,b,c,m[2], 9,0xfcfa3f8);
84     GG(c,d,a,b,m[7], 14,0x676f02d9);
85     GG(b,c,d,a,m[12],20,0x8d2a4c8a);

```

```

83
84     HH(a,b,c,d,m[5], 4,0xffff3942);
85     HH(d,a,b,c,m[10],11,0x8771f681);
86     HH(c,d,a,b,m[15],16,0x6d9d6122);
87     HH(b,c,d,a,m[4], 23,0xfde5380c);
88     HH(a,b,c,d,m[9], 4,0xa4bee44);
89     HH(d,a,b,c,m[14],11,0x4bdecfa9);
90     HH(c,d,a,b,m[3], 16,0xf6bb4b60);
91     HH(b,c,d,a,m[12],23,0x9bebfb70);
92     HH(a,b,c,d,m[1], 4,0x289b7ec6);
93     HH(d,a,b,c,m[6], 11,0x9aa127fa);
94     HH(c,d,a,b,m[11],16,0xd4ef3085);
95     HH(b,c,d,a,m[16],23,0x04881d05);
96     HH(a,b,c,d,m[5], 4,0xd9d4d039);
97     HH(d,a,b,c,m[10],11,0xe6db99e5);
98     HH(c,d,a,b,m[15],16,0x1fa27cf8);
99     HH(b,c,d,a,m[4], 23,0xc4ac5665);
100
101     II(a,b,c,d,m[0], 6,0xf4292244);
102     II(d,a,b,c,m[7], 10,0x432aff97);
103     II(c,d,a,b,m[14],15,0xab9423a7);
104     II(b,c,d,a,m[5], 21,0xfc93a039);
105     II(a,b,c,d,m[12], 6,0x655b59c3);
106     II(d,a,b,c,m[3], 10,0x8f6ccc92);
107     II(c,d,a,b,m[10],15,0xffeff47d);
108     II(b,c,d,a,m[1], 21,0x85845dd1);
109     II(a,b,c,d,m[8], 6,0x6fa87e4f);
110     II(d,a,b,c,m[15],10,0xfe2ce6e0);
111     II(c,d,a,b,m[6], 15,0xa3014314);
112     II(b,c,d,a,m[13],21,0x4e0811a1);
113     II(a,b,c,d,m[4], 6,0xf7537e82);
114     II(d,a,b,c,m[11],10,0xbd3af235);
115     II(c,d,a,b,m[2], 15,0x2ad7d2bb);
116     II(b,c,d,a,m[9], 21,0xeb86d391);
117
118     ctx->state[0] += a;
119     ctx->state[1] += b;
120     ctx->state[2] += c;
121     ctx->state[3] += d;
122 }
123
124 void md5_init(MD5_CTX *ctx)
125 {
126     ctx->datalen = 0;
127     ctx->bitlen = 0;
128     ctx->state[0] = 0x67452301;
129     ctx->state[1] = 0xefcdab89;
130     ctx->state[2] = 0x98badcf5;
131     ctx->state[3] = 0x10325476;
132 }
133
134 void md5_update(MD5_CTX *ctx, const BYTE data[], size_t len)
135 {
136     size_t i;
137
138     for (i = 0; i < len; ++i) {
139         ctx->data[ctx->datalen] = data[i];
140         ctx->datalen++;
141         if (ctx->datalen == 64) {
142             md5_transform(ctx, ctx->data);
143             ctx->bitlen += 512;
144             ctx->datalen = 0;
145         }
146     }
147 }
148
149 void md5_final(MD5_CTX *ctx, BYTE hash[])
150 {
151     size_t i;
152
153     i = ctx->datalen;
154
155     // Pad whatever data is left in the buffer.
156     if (ctx->datalen < 56) {
157         ctx->data[i++] = 0x80;
158         while (i < 56)
159             ctx->data[i++] = 0x00;
160     }
161     else if (ctx->datalen == 56) {
162         ctx->data[i++] = 0x80;
163         while (i < 64)
164             ctx->data[i++] = 0x00;
165         md5_transform(ctx, ctx->data);
166         memset(ctx->data, 0, 56);
167     }
168
169     // Append to the padding the total message's length in
170     // bits and transform.
171     ctx->bitlen += ctx->datalen * 8;
172     ctx->data[56] = ctx->bitlen;
173     ctx->data[57] = ctx->bitlen >> 8;
174     ctx->data[58] = ctx->bitlen >> 16;
175     ctx->data[59] = ctx->bitlen >> 24;
176     ctx->data[60] = ctx->bitlen >> 32;
177     ctx->data[61] = ctx->bitlen >> 40;
178     ctx->data[62] = ctx->bitlen >> 48;
179     ctx->data[63] = ctx->bitlen >> 56;
180     md5_transform(ctx, ctx->data);
181
182     // Since this implementation uses little endian byte
183     // ordering and MD uses big endian,

```



```

182 // reverse all the bytes when copying the final state
to the output hash.
183 for (i = 0; i < 4; ++i) {
184     hash[i] = (ctx->state[0] >> (i * 8)) &
0x000000ff;
185     hash[i + 4] = (ctx->state[1] >> (i * 8)) &
0x000000ff;
186     hash[i + 8] = (ctx->state[2] >> (i * 8)) &
0x000000ff;
187     hash[i + 12] = (ctx->state[3] >> (i * 8)) &
0x000000ff;
188 }
189 }

```

20. B-Con_crypto-algorithms/md5.h (34 lines)

```

1 /
*****
2 * Filename:    md5.h
3 * Author:     Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:    Defines the API for the corresponding MD5 imple-
mentation.
7
*****
8
9 #ifndef MD5_H
10 #define MD5_H
11
12 /***** HEADER FILES
*****
13 #include <stddef.h>
14
15 /***** MACROS
*****
16 #define MD5_BLOCK_SIZE 16 // MD5 outputs a 16
byte digest
17
18 /***** DATA TYPES
*****
19 typedef unsigned char BYTE; // 8-bit byte
20
21 typedef unsigned int WORD; // 32-bit word, change to
"long" for 16-bit machines
22
23 typedef struct {
24     BYTE data[64];
25     WORD datalen;
26     unsigned long long bitlen;
27     WORD state[4];
28 } MD5_CTX;
29
30 /***** FUNCTION DECLARATIONS
*****
31 void md5_init(MD5_CTX *ctx);
32 void md5_update(MD5_CTX *ctx, const BYTE data[], size_t len);
33 void md5_final(MD5_CTX *ctx, BYTE hash[]);
34 #endif // MD5_H

```

21. B-Con_crypto-algorithms/md5_test.c (60 lines)

```

1 /
*****
2 * Filename:    md5_test.c
3 * Author:     Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:    Performs known-answer tests on the corresponding MD5
implementation. These tests do not encompass
the full
7
8
9
10
11
12
13
14 /***** HEADER FILES
*****
15 #include <stdio.h>
16 #include <memory.h>
17 #include <string.h>
18 #include "md5.h"
19
20 /***** FUNCTION DEFINITIONS
*****
21 int md5_test()

```

```

22 {
23     BYTE text1[] = {"";
24     BYTE text2[] = {"abc";
25     BYTE text3_1[] = {"ABCDEFGHIJKLMNOPQRSTUVWXYZabcde";
26     BYTE text3_2[] = {"fghijklmnopqrstuvwxyz0123456789";
27     BYTE hash1[MD5_BLOCK_SIZE] = {0xd4, 0x1d, 0x8c, 0xd9, 0x8f,
0x00, 0xb2, 0x04, 0xe9, 0x80, 0x09, 0x98, 0xec, 0xf8, 0x42, 0xe7};
28     BYTE hash2[MD5_BLOCK_SIZE] = {0x90, 0x01, 0x50, 0x98, 0x3c,
0xd2, 0x4f, 0xb0, 0xd6, 0x96, 0x3f, 0x7d, 0x28, 0xe1, 0x7f, 0x72};
29     BYTE hash3[MD5_BLOCK_SIZE] = {0xd1, 0x74, 0xab,
0x98, 0xd2, 0x77, 0xd9, 0xf5, 0xa5, 0x61, 0x1c, 0x2c, 0x9f, 0x41, 0x9d, 0x9f};
30     BYTE buf[16];
31     MD5_CTX ctx;
32     int pass = 1;
33
34     md5_init(&ctx);
35     md5_update(&ctx, text1, strlen(text1));
36     md5_final(&ctx, buf);
37     pass = pass && !memcmp(hash1, buf, MD5_BLOCK_SIZE);
38
39     // Note the MD5 object can be reused.
40     md5_init(&ctx);
41     md5_update(&ctx, text2, strlen(text2));
42     md5_final(&ctx, buf);
43     pass = pass && !memcmp(hash2, buf, MD5_BLOCK_SIZE);
44
45     // Note the data is being added in two chunks.
46     md5_init(&ctx);
47     md5_update(&ctx, text3_1, strlen(text3_1));
48     md5_update(&ctx, text3_2, strlen(text3_2));
49     md5_final(&ctx, buf);
50     pass = pass && !memcmp(hash3, buf, MD5_BLOCK_SIZE);
51
52     return(pass);
53 }
54
55 int main()
56 {
57     printf("MD5 tests: %s\n", md5_test() ? "SUCCEEDED" :
"FAILED");
58
59     return(0);
60 }

```

22. B-Con_crypto-algorithms/rot-13.c (35 lines)

```

1 /
*****
2 * Filename:    rot-13.c
3 * Author:     Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:    Implementation of the ROT-13 encryption al-
gorithm.
7
8
9
10
11
12 /***** HEADER FILES
*****
13 #include <string.h>
14 #include "rot-13.h"
15
16 /***** FUNCTION DEFINITIONS
*****
17 void rot13(char str[])
18 {
19     int case_type, idx, len;
20
21     for (idx = 0, len = strlen(str); idx < len; idx++) {
22         // Only process alphabetic characters.
23
24         if (str[idx] < 'A' || (str[idx] > 'Z' && str[idx] < 'a') ||
str[idx] > 'z')
25             continue;
26         // Determine if the char is upper or lower case.
27         if (str[idx] >= 'a')
28             case_type = 'a';
29         else
30             case_type = 'A';
31         // Rotate the char's value, ensuring it doesn't acci-
dentally "fall off" the end.
32         str[idx] = (str[idx] + 13) % (case_type + 26);
33         if (str[idx] < 26)
34             str[idx] += case_type;
35     }

```

23. B-Con_crypto-algorithms/rot-13.h (20 lines)

```

1 /
*****

```

```

2 * Filename:   rot-13.h
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:    Defines the API for the corresponding ROT-13 im-
plementation.
7
*****
*/
8
9 #ifndef ROT13_H
10 #define ROT13_H
11
12 /***** HEADER FILES *****/
13 #include <stddef.h>
14
15 /***** FUNCTION DECLARATIONS *****/
16 // Performs IN PLACE rotation of the input. Assumes input is
NULL terminated.
17 // Preserves each character's case. Ignores non alphabetic
characters.
18 void rot13(char str[]);
19
20 #endif // ROT13_H

```

24. B-Con_crypto-algorithms/rot-13_test.c (44 lines)

```

1 /
*****
*
2 * Filename:   rot-13_test.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:    Performs known-answer tests on the corresponding
ROT-13
7                  implementation. These tests do not encompass
the full
8                  range of available test vectors, however, if
the tests
9
10                 pass it is very, very likely that the code is
correct
11                 and was compiled properly. This code also
serves as
12                 example usage of the functions.
*****
*/
13
14 /***** HEADER FILES *****/
15 #include <stdio.h>
16 #include <string.h>
17 #include "rot-13.h"
18
19 /***** FUNCTION DEFINITIONS *****/
20 int rot13_test()
21 {
22     char text[] = {"ABCDEFGHIJKLMNopQRSTUVWXYZabcde-
fghijklmnopqrstuvwxyz"};
23     char code[] = {"NOPQRSTUVWXYZABCDEFGHIJKLMnopqr-
stuvwxyzabcdefghijklmnop"};
24     char buf[1024];
25     int pass = 1;
26
27     // To encode, just apply ROT-13.
28     strcpy(buf, text);
29     rot13(buf);
30     pass = pass && !strcmp(code, buf);
31
32     // To decode, just re-apply ROT-13.
33     rot13(buf);
34     pass = pass && !strcmp(text, buf);
35
36     return(pass);
37 }
38
39 int main()
40 {
41     printf("ROT-13 tests: %s\n", rot13_test() ? "SUC-
CEDED" : "FAILED");
42
43     return(0);
44 }

```

25. B-Con_crypto-algorithms/sha1.c (149 lines)

```

1 /
*****
*
2 * Filename:   sha1.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:

```

```

5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:    Implementation of the SHA1 hashing algorithm.
Algorithm specification can be found here:
7                * http://csrc.nist.gov/publications/fips/
fips180-2/fips180-2withchangenotice.pdf
8                This implementation uses little endian byte or-
der.
9
10 *****
11
12 /***** HEADER FILES *****/
13 #include <stdlib.h>
14 #include <memory.h>
15 #include "sha1.h"
16
17 /***** MACROS *****/
18 #define ROTLEFT(a, b) ((a << b) | (a >> (32 - b)))
19
20 /***** FUNCTION DEFINITIONS *****/
21 void sha1_transform(SHA1_CTX *ctx, const BYTE data[])
22 {
23     WORD a, b, c, d, e, i, j, t, m[80];
24
25     for (i = 0; j = 0; i < 16; ++i, j += 4)
26         m[i] = (data[j] << 24) + (data[j + 1] << 16) +
(data[j + 2] << 8) + (data[j + 3]);
27     for ( ; i < 80; ++i) {
28         m[i] = (m[i - 3] ^ m[i - 8] ^ m[i - 14] ^ m[i -
16]);
29         m[i] = (m[i] << 1) | (m[i] >> 31);
30     }
31
32     a = ctx->state[0];
33     b = ctx->state[1];
34     c = ctx->state[2];
35     d = ctx->state[3];
36     e = ctx->state[4];
37
38     for (i = 0; i < 20; ++i) {
39         t = ROTLEFT(a, 5) + ((b & c) ^ (~b & d)) + e +
ctx->k[0] + m[i];
40         e = d;
41         d = c;
42         c = ROTLEFT(b, 30);
43         b = a;
44         a = t;
45     }
46     for ( ; i < 40; ++i) {
47         t = ROTLEFT(a, 5) + (b ^ c ^ d) + e + ctx->k[1] +
m[i];
48         e = d;
49         d = c;
50         c = ROTLEFT(b, 30);
51         b = a;
52         a = t;
53     }
54     for ( ; i < 60; ++i) {
55         t = ROTLEFT(a, 5) + ((b & c) ^ (b & d) ^ (c &
d)) + e + ctx->k[2] + m[i];
56         e = d;
57         d = c;
58         c = ROTLEFT(b, 30);
59         b = a;
60         a = t;
61     }
62     for ( ; i < 80; ++i) {
63         t = ROTLEFT(a, 5) + (b ^ c ^ d) + e + ctx->k[3] +
m[i];
64         e = d;
65         d = c;
66         c = ROTLEFT(b, 30);
67         b = a;
68         a = t;
69     }
70
71     ctx->state[0] += a;
72     ctx->state[1] += b;
73     ctx->state[2] += c;
74     ctx->state[3] += d;
75     ctx->state[4] += e;
76 }
77
78 void sha1_init(SHA1_CTX *ctx)
79 {
80     ctx->datalen = 0;
81     ctx->bitlen = 0;
82     ctx->state[0] = 0x67452301;
83     ctx->state[1] = 0xEFCDAB89;
84     ctx->state[2] = 0x98BADCFE;
85     ctx->state[3] = 0x10325476;
86     ctx->state[4] = 0xC3D2E1F0;
87     ctx->k[0] = 0x5A827999;
88     ctx->k[1] = 0x6ED9EBA1;

```

```

89     ctx->k[2] = 0x8f1bbcdc;
90     ctx->k[3] = 0xca62c1d6;
91 }
92
93 void sha1_update(SHA1_CTX *ctx, const BYTE data[], size_t len)
94 {
95     size_t i;
96
97     for (i = 0; i < len; ++i) {
98         ctx->data[ctx->datalen] = data[i];
99         ctx->datalen++;
100         if (ctx->datalen == 64) {
101             sha1_transform(ctx, ctx->data);
102             ctx->bitlen += 512;
103             ctx->datalen = 0;
104         }
105     }
106 }
107
108 void sha1_final(SHA1_CTX *ctx, BYTE hash[])
109 {
110     WORD i;
111
112     i = ctx->datalen;
113
114     // Pad whatever data is left in the buffer.
115     if (ctx->datalen < 56) {
116         ctx->data[i++] = 0x80;
117         while (i < 56)
118             ctx->data[i++] = 0x00;
119     }
120     else {
121         ctx->data[i++] = 0x80;
122         while (i < 64)
123             ctx->data[i++] = 0x00;
124         sha1_transform(ctx, ctx->data);
125         memset(ctx->data, 0, 56);
126     }
127
128     // Append to the padding the total message's length in
129     bits and transform.
130     ctx->bitlen += ctx->datalen * 8;
131     ctx->data[63] = ctx->bitlen;
132     ctx->data[62] = ctx->bitlen >> 8;
133     ctx->data[61] = ctx->bitlen >> 16;
134     ctx->data[60] = ctx->bitlen >> 24;
135     ctx->data[59] = ctx->bitlen >> 32;
136     ctx->data[58] = ctx->bitlen >> 40;
137     ctx->data[57] = ctx->bitlen >> 48;
138     ctx->data[56] = ctx->bitlen >> 56;
139     sha1_transform(ctx, ctx->data);
140
141     // Since this implementation uses little endian byte
142     ordering and MD uses big endian,
143     // reverse all the bytes when copying the final state
144     to the output hash.
145     for (i = 0; i < 4; ++i) {
146         hash[i] = (ctx->state[0] >> (24 - i * 8))
147         & 0x000000ff;
148         hash[i + 4] = (ctx->state[1] >> (24 - i * 8))
149         & 0x000000ff;
150         hash[i + 8] = (ctx->state[2] >> (24 - i * 8))
151         & 0x000000ff;
152         hash[i + 12] = (ctx->state[3] >> (24 - i * 8))
153         & 0x000000ff;
154         hash[i + 16] = (ctx->state[4] >> (24 - i * 8))
155         & 0x000000ff;
156     }
157 }

```

26. B-Con_crypto-algorithms/sha1.h (35 lines)

```

1 /
2 *
3 * Filename: sha1.h
4 * Author: Brad Conte (brad AT bradconte.com)
5 * Copyright:
6 * Disclaimer: This code is presented "as is" without any guar-
7 * antees.
8 * Details: Defines the API for the corresponding SHA1 im-
9 * plementation.
10
11 /*****
12  *
13  */
14
15 #ifndef SHA1_H
16 #define SHA1_H
17
18 /***** HEADER FILES *****/
19 #include <stddef.h>
20
21 /***** MACROS *****/
22 #define SHA1_BLOCK_SIZE 20 // SHA1 outputs a 20
23 // byte digest
24
25 /***** DATA TYPES *****/

```

```

19 typedef unsigned char BYTE; // 8-bit byte
20
21 typedef unsigned int WORD; // 32-bit word, change to
22 "long" for 16-bit machines
23
24 typedef struct {
25     BYTE data[64];
26     WORD datalen;
27     unsigned long long bitlen;
28     WORD state[5];
29     WORD k[4];
30 } SHA1_CTX;
31
32 /***** FUNCTION DECLARATIONS *****/
33 void sha1_init(SHA1_CTX *ctx);
34 void sha1_update(SHA1_CTX *ctx, const BYTE data[], size_t
35 len);
36 void sha1_final(SHA1_CTX *ctx, BYTE hash[]);
37
38 #endif // SHA1_H

```

27. B-Con_crypto-algorithms/sha1_test.c (58 lines)

```

1 /
2 *
3 * Filename: sha1_test.c
4 * Author: Brad Conte (brad AT bradconte.com)
5 * Copyright:
6 * Disclaimer: This code is presented "as is" without any guar-
7 * antees.
8
9 * Details: Performs known-answer tests on the corresponding
10 SHA1 implementation. These tests do not encompass
11 the full range of available test vectors, however, if
12 the tests pass it is very, very likely that the code is
13 correct and was compiled properly. This code also
14 serves as example usage of the functions.
15
16 /*****
17  *
18  */
19
20 /***** HEADER FILES *****/
21 #include <stdio.h>
22 #include <memory.h>
23 #include <string.h>
24 #include "sha1.h"
25
26 /***** FUNCTION DEFINITIONS *****/
27
28 int sha1_test()
29 {
30     BYTE text1[] = {"abc"};
31     BYTE text2[] = {"abcdcbcdedfdefgefgh-
32 fghighijhijkijklklmklmnlmnomnopnopq"};
33     BYTE text3[] = {"aaaaaaaaaaaaa"};
34     BYTE hash1[SHA1_BLOCK_SIZE] = {0xa9, 0x99, 0x3e,
35 0x36, 0x47, 0x06, 0x81, 0x6a, 0xba, 0x3e, 0x25, 0x71, 0x78, 0x50, 0xc2, 0x6c,
36 0x9c, 0xd0, 0xd8, 0x9d};
37     BYTE hash2[SHA1_BLOCK_SIZE] = {0x84, 0x98, 0x3e,
38 0x44, 0x1c, 0x3b, 0xd2, 0x6e, 0xba, 0xae, 0x4a,
39 0xa1, 0xf9, 0x51, 0x29, 0xe5, 0xe5, 0x46, 0x70, 0xf1};
40     BYTE hash3[SHA1_BLOCK_SIZE] = {0x34, 0xaa, 0x97, 0x3c,
41 0xd4, 0xc4, 0xda, 0xa4, 0xf6, 0x1e, 0xeb, 0x2b, 0xdb, 0xad,
42 0x27, 0x31, 0x65, 0x34, 0x01, 0x6f};
43     BYTE buf[SHA1_BLOCK_SIZE];
44     int idx;
45     SHA1_CTX ctx;
46     int pass = 1;
47
48     sha1_init(&ctx);
49     sha1_update(&ctx, text1, strlen(text1));
50     sha1_final(&ctx, buf);
51     pass = pass && !memcmp(hash1, buf, SHA1_BLOCK_SIZE);
52
53     sha1_init(&ctx);
54     sha1_update(&ctx, text2, strlen(text2));
55     sha1_final(&ctx, buf);
56     pass = pass && !memcmp(hash2, buf, SHA1_BLOCK_SIZE);
57
58     sha1_init(&ctx);
59     for (idx = 0; idx < 100000; ++idx)
60         sha1_update(&ctx, text3, strlen(text3));
61     sha1_final(&ctx, buf);
62     pass = pass && !memcmp(hash3, buf, SHA1_BLOCK_SIZE);
63
64     return(pass);
65 }
66
67 int main()
68 {
69     printf("SHA1 tests: %s\n", sha1_test() ? "SUCCEEDED" :

```



```

"FAILED");
56
57     return(0);
58 }

```

28. B-Con_crypto-algorithms/sha256.c (158 lines)

```

1 /
*****
2 * Filename:   sha256.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   Implementation of the SHA-256 hashing algorithm.
SHA2
7             SHA-256 is one of the three algorithms in the
8             specification. The others, SHA-384 and SHA-512,
are not
9             offered in this implementation.
10            Algorithm specification can be found here:
11            * http://csrc.nist.gov/publications/fips/
fips180-2/fips180-2withchangenotice.pdf
12            This implementation uses little endian byte or-
der.
13
*****
14
15 /***** HEADER FILES
*****
16 #include <stdlib.h>
17 #include <memory.h>
18 #include "sha256.h"
19
20 /***** MACROS
*****
21 #define ROTLEFT(a,b) (((a) << (b)) | ((a) >> (32-(b))))
22 #define ROTRIGHT(a,b) (((a) >> (b)) | ((a) << (32-(b))))
23
24 #define CH(x,y,z) (((x) & (y)) ^ (~(x) & (z)))
25 #define MAJ(x,y,z) (((x) & (y)) ^ ((x) & (z)) ^ ((y) & (z)))
26 #define EP0(x) (ROTRIGHT(x,2) ^ ROTRIGHT(x,13) ^ ROTRIGHT(x,
22))
27 #define EP1(x) (ROTRIGHT(x,6) ^ ROTRIGHT(x,11) ^ ROTRIGHT(x,
25))
28 #define SIG0(x) (ROTRIGHT(x,7) ^ ROTRIGHT(x,18) ^ ((x) >> 3))
29 #define SIG1(x) (ROTRIGHT(x,17) ^ ROTRIGHT(x,19) ^ ((x) >>
10))
30
31 /***** VARIABLES
*****
32 static const WORD k[64] = {
33     0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b,
0x59f111f1, 0x923f82a4, 0xab1c5ed5,
34     0xd807aa98, 0x12835b01, 0x243185be, 0x550c7d-
c3, 0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
35     0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f,
0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
36     0x983e5152, 0xa831c66d,
0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
37     0x27b70a85, 0x2e1b2138, 0x4d2c6dfc,
0x53380d13, 0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
38     0xa2bfe8a1, 0xa81a664b, 0x-
c24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
39     0x19a4c116, 0x1e377c08, 0x2748774c, 0x34b0b0-
cb5, 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
40     0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x900beffa,
0xa4506ceb, 0xbef9a3f7, 0xc67f1782
41 };
42
43 /***** FUNCTION DEFINITIONS
*****
44 void sha256_transform(SHA256_CTX *ctx, const BYTE data[])
45 {
46     WORD a, b, c, d, e, f, g, h, i, j, t1, t2, m[64];
47
48     for (i = 0, j = 0; i < 16; ++i, j += 4)
49         m[i] = (data[j] << 24) | (data[j + 1] << 16) |
(data[j + 2] << 8) | (data[j + 3]);
50     for (; i < 64; ++i)
51         m[i] = SIG1(m[i - 2]) + m[i - 7] + SIG0(m[i -
15]) + m[i - 16];
52
53     a = ctx->state[0];
54     b = ctx->state[1];
55     c = ctx->state[2];
56     d = ctx->state[3];
57     e = ctx->state[4];
58     f = ctx->state[5];
59     g = ctx->state[6];
60     h = ctx->state[7];
61
62     for (i = 0; i < 64; ++i) {
63         t1 = h + EP1(e) + CH(e,f,g) + k[i] + m[i];
64         t2 = EP0(a) + MAJ(a,b,c);
65         h = g;
66         g = f;
67         f = e;

```

```

68         e = d + t1;
69         d = c;
70         c = b;
71         b = a;
72         a = t1 + t2;
73     }
74
75     ctx->state[0] += a;
76     ctx->state[1] += b;
77     ctx->state[2] += c;
78     ctx->state[3] += d;
79     ctx->state[4] += e;
80     ctx->state[5] += f;
81     ctx->state[6] += g;
82     ctx->state[7] += h;
83 }
84
85 void sha256_init(SHA256_CTX *ctx)
86 {
87     ctx->datalen = 0;
88     ctx->bitlen = 0;
89     ctx->state[0] = 0x6a09e667;
90     ctx->state[1] = 0xbb67ae85;
91     ctx->state[2] = 0x3c6ef372;
92     ctx->state[3] = 0xa54ff53a;
93     ctx->state[4] = 0x510e527f;
94     ctx->state[5] = 0x9b05688c;
95     ctx->state[6] = 0x1f83d9ab;
96     ctx->state[7] = 0x5be0cd19;
97 }
98
99 void sha256_update(SHA256_CTX *ctx, const BYTE data[], size_t
len)
100 {
101     WORD i;
102
103     for (i = 0; i < len; ++i) {
104         ctx->data[ctx->datalen] = data[i];
105         ctx->datalen++;
106         if (ctx->datalen == 64) {
107             sha256_transform(ctx, ctx->data);
108             ctx->bitlen += 512;
109             ctx->datalen = 0;
110         }
111     }
112 }
113
114 void sha256_final(SHA256_CTX *ctx, BYTE hash[])
115 {
116     WORD i;
117
118     i = ctx->datalen;
119
120     // Pad whatever data is left in the buffer.
121     if (ctx->datalen < 56) {
122         ctx->data[i++] = 0x80;
123         while (i < 56)
124             ctx->data[i++] = 0x00;
125     }
126     else {
127         ctx->data[i++] = 0x80;
128         while (i < 64)
129             ctx->data[i++] = 0x00;
130         sha256_transform(ctx, ctx->data);
131         memset(ctx->data, 0, 56);
132     }
133
134     // Append to the padding the total message's length in
bits and transform.
135     ctx->bitlen += ctx->datalen * 8;
136     ctx->data[63] = ctx->bitlen;
137     ctx->data[62] = ctx->bitlen >> 8;
138     ctx->data[61] = ctx->bitlen >> 16;
139     ctx->data[60] = ctx->bitlen >> 24;
140     ctx->data[59] = ctx->bitlen >> 32;
141     ctx->data[58] = ctx->bitlen >> 40;
142     ctx->data[57] = ctx->bitlen >> 48;
143     ctx->data[56] = ctx->bitlen >> 56;
144     sha256_transform(ctx, ctx->data);
145
146     // Since this implementation uses little endian byte
ordering and SHA uses big endian,
147     // reverse all the bytes when copying the final state
to the output hash.
148     for (i = 0; i < 4; ++i) {
149         hash[i] = (ctx->state[0] >> (24 - i * 8))
& 0x000000ff;
150         hash[i + 4] = (ctx->state[1] >> (24 - i * 8))
& 0x000000ff;
151         hash[i + 8] = (ctx->state[2] >> (24 - i * 8))
& 0x000000ff;
152         hash[i + 12] = (ctx->state[3] >> (24 - i * 8))
& 0x000000ff;
153         hash[i + 16] = (ctx->state[4] >> (24 - i * 8))
& 0x000000ff;
154         hash[i + 20] = (ctx->state[5] >> (24 - i * 8))
& 0x000000ff;
155         hash[i + 24] = (ctx->state[6] >> (24 - i * 8))
& 0x000000ff;
156         hash[i + 28] = (ctx->state[7] >> (24 - i * 8))
& 0x000000ff;

```


AES 코어 (Verilog)

What this is. Joachim Strömbergson's AES in Verilog: encipher and decipher cores, S-box, key memory.

Why it exists. To provide the plain-RTL baseline. Neither HLS C++ nor hardened security RTL, it shows how small AES is when only the function is implemented. Comparing it with OpenTitan's 17,000 lines isolates exactly what side-channel and fault countermeasures cost.

Undergraduate review · EECS concepts. Register-transfer abstraction; finite state machines; separation of datapath and control.

Graduate · Practice note. When evaluating any IP, the useful question is not “how many lines?” but “what threat model does it assume?” Two blocks implementing the same standard under different threat models are different products.

7 files • 2,714 lines

API — aes.v

Table 740. secworks_aes/src/rtl/aes.v — module / interface / package (1개)

이름	종류
aes	module

Table 741. secworks_aes/src/rtl/aes.v — localparam (21개)

상수	형/폭	값
ADDR_NAME0		8'h00
ADDR_NAME1		8'h01
ADDR_VERSION		8'h02
ADDR_CTRL		8'h08
CTRL_INIT_BIT		0
CTRL_NEXT_BIT		1
ADDR_STATUS		8'h09
STATUS_READY_BIT		0
STATUS_VALID_BIT		1
ADDR_CONFIG		8'h0a
CTRL_ENCDEC_BIT		0
CTRL_KEYLEN_BIT		1
ADDR_KEY0		8'h10
ADDR_KEY7		8'h17
ADDR_BLOCK0		8'h20
ADDR_BLOCK3		8'h23

상수	형/폭	값
ADDR_RESULT0		8'h30
ADDR_RESULT3		8'h33
CORE_NAME0		32'h61657320
CORE_NAME1		32'h20202020
CORE_VERSION		32'h302e3630

Table 742. secworks_aes/src/rtl/aes.v — 포트 (7개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
cs	input	wire
we	input	wire
address	input	wire [7 : 0]
write_data	input	wire [31 : 0]
read_data	output	wire [31 : 0]

API — aes_core.v**Table 743. secworks_aes/src/rtl/aes_core.v — module / interface / package (1개)**

이름	종류
aes_core	module

Table 744. secworks_aes/src/rtl/aes_core.v — localparam (3개)

상수	형/폭	값
CTRL_IDLE		2'h0
CTRL_INIT		2'h1
CTRL_NEXT		2'h2

Table 745. secworks_aes/src/rtl/aes_core.v — 포트 (11개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
encdec	input	wire
init	input	wire
next	input	wire
ready	output	wire
key	input	wire [255 : 0]
keylen	input	wire

포트	방향	형/폭
block	input	wire [127 : 0]
result	output	wire [127 : 0]
result_valid	output	wire

API — aes_decipher_block.v

Table 746. secworks_aes/src/rtl/aes_decipher_block.v — module / interface / package (1개)

이름	종류
aes_decipher_block	module

Table 747. secworks_aes/src/rtl/aes_decipher_block.v — localparam (13개)

상수	형/폭	값
AES_128_BIT_KEY		1'h0
AES_256_BIT_KEY		1'h1
AES128_ROUNDS		4'ha
AES256_ROUNDS		4'he
NO_UPDATE		3'h0
INIT_UPDATE		3'h1
SBOX_UPDATE		3'h2
MAIN_UPDATE		3'h3
FINAL_UPDATE		3'h4
CTRL_IDLE		2'h0
CTRL_INIT		2'h1
CTRL_SBOX		2'h2
CTRL_MAIN		2'h3

Table 748. secworks_aes/src/rtl/aes_decipher_block.v — 포트 (9개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
next	input	wire
keylen	input	wire
round	output	wire [3 : 0]
round_key	input	wire [127 : 0]
block	input	wire [127 : 0]
new_block	output	wire [127 : 0]
ready	output	wire

API — aes_encipher_block.v**Table 749. secworks_aes/src/rtl/aes_encipher_block.v — module / interface / package (1개)**

이름	종류
aes_encipher_block	module

Table 750. secworks_aes/src/rtl/aes_encipher_block.v — localparam (13개)

상수	형/폭	값
AES_128_BIT_KEY		1'h0
AES_256_BIT_KEY		1'h1
AES128_ROUNDS		4'ha
AES256_ROUNDS		4'he
NO_UPDATE		3'h0
INIT_UPDATE		3'h1
SBOX_UPDATE		3'h2
MAIN_UPDATE		3'h3
FINAL_UPDATE		3'h4
CTRL_IDLE		2'h0
CTRL_INIT		2'h1
CTRL_SBOX		2'h2
CTRL_MAIN		2'h3

Table 751. secworks_aes/src/rtl/aes_encipher_block.v — 포트 (11개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
next	input	wire
keylen	input	wire
round	output	wire [3 : 0]
round_key	input	wire [127 : 0]
sboxw	output	wire [31 : 0]
new_sboxw	input	wire [31 : 0]
block	input	wire [127 : 0]
new_block	output	wire [127 : 0]
ready	output	wire

API — aes_inv_sbox.v**Table 752. secworks_aes/src/rtl/aes_inv_sbox.v — module / interface / package (1개)**

이름	종류
aes_inv_sbox	module

Table 753. secworks_aes/src/rtl/aes_inv_sbox.v — 포트 (2개)

포트	방향	형/폭
sboxw	input	wire [31 : 0]
new_sboxw	output	wire [31 : 0]

API — aes_key_mem.v**Table 754. secworks_aes/src/rtl/aes_key_mem.v — module / interface / package (1개)**

이름	종류
aes_key_mem	module

Table 755. secworks_aes/src/rtl/aes_key_mem.v — localparam (8개)

상수	형/폭	값
AES_128_BIT_KEY		1'h0
AES_256_BIT_KEY		1'h1
AES_128_NUM_ROUNDS		10
AES_256_NUM_ROUNDS		14
CTRL_IDLE		3'h0
CTRL_INIT		3'h1
CTRL_GENERATE		3'h2
CTRL_DONE		3'h3

Table 756. secworks_aes/src/rtl/aes_key_mem.v — 포트 (10개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
key	input	wire [255 : 0]
keylen	input	wire
init	input	wire
round	input	wire [3 : 0]
round_key	output	wire [127 : 0]
ready	output	wire
sboxw	output	wire [31 : 0]
new_sboxw	input	wire [31 : 0]

API — aes_sbox.v

Table 757. secworks_aes/src/rtl/aes_sbox.v — module / interface / package (1개)

이름	종류
aes_sbox	module

Table 758. secworks_aes/src/rtl/aes_sbox.v — 포트 (2개)

포트	방향	형/폭
sboxw	input	wire [31 : 0]
new_sboxw	output	wire [31 : 0]

1. secworks_aes/src/rtl/aes.v (273 lines)

```

1 //
=====
2 //
3 // aes.v
4 // -----
5 // Top level wrapper for the AES block cipher core.
6 //
7 //
8 // Author: Joachim Strombergson
9 // Copyright (c) 2013, 2014 Secworks Sweden AB
10 // All rights reserved.
11 //
12 // Redistribution and use in source and binary forms, with or
13 // without modification, are permitted provided that the fol-
14 // lowing conditions are met:
15 //
16 // 1. Redistributions of source code must retain the above
17 // copyright notice, this list of conditions and the following dis-
18 // claimer.
19 // 2. Redistributions in binary form must reproduce the above
20 // copyright notice, this list of conditions and the following dis-
21 // claimer in the documentation and/or other materials provided with
22 // the distribution.
23 //
24 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
25 // TRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
26 // BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
27 // FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
28 // THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
29 // INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
30 // (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
31 // VICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
32 // HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
33 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
34 // WISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
35 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
36 //
37 //
=====
38
39 `default_nettype none
40
41 module aes(
42     // Clock and reset.
43     input wire      clk,
44     input wire      reset_n,
45
46     // Control.
47     input wire      cs,
48     input wire      we,
49
50     // Data ports.
51     input wire [7 : 0] address,
52     input wire [31 : 0] write_data,
53     output wire [31 : 0] read_data
54 );

```

```

55
56 //-----
57 // Internal constant and parameter definitions.
58 //-----
59
60 localparam ADDR_NAME0    = 8'h00;
61 localparam ADDR_NAME1    = 8'h01;
62 localparam ADDR_VERSION  = 8'h02;
63
64 localparam ADDR_CTRL     = 8'h08;
65 localparam CTRL_INIT_BIT = 0;
66 localparam CTRL_NEXT_BIT = 1;
67
68 localparam ADDR_STATUS   = 8'h09;
69 localparam STATUS_READY_BIT = 0;
70 localparam STATUS_VALID_BIT = 1;
71
72 localparam ADDR_CONFIG   = 8'h0a;
73 localparam CTRL_ENCDEC_BIT = 0;
74 localparam CTRL_KEYLEN_BIT = 1;
75
76 localparam ADDR_KEY0     = 8'h10;
77 localparam ADDR_KEY7     = 8'h17;
78
79 localparam ADDR_BLOCK0   = 8'h20;
80 localparam ADDR_BLOCK3   = 8'h23;
81
82 localparam ADDR_RESULT0  = 8'h30;
83 localparam ADDR_RESULT3  = 8'h33;
84
85 localparam CORE_NAME0    = 32'h61657320; // "aes "
86 localparam CORE_NAME1    = 32'h20202020; // "   "
87 localparam CORE_VERSION  = 32'h302e3630; // "0.60"
88
89 //-----
90 // Registers including update variables and write enable.
91 //-----
92
93 reg init_reg;
94 reg init_new;
95
96 reg next_reg;
97 reg next_new;
98
99 reg encdec_reg;
100 reg keylen_reg;
101 reg config_we;
102
103 reg [31 : 0] block_reg [0 : 3];
104 reg          block_we;
105
106 reg [31 : 0] key_reg [0 : 7];
107 reg          key_we;
108
109 reg [127 : 0] result_reg;
110 reg          valid_reg;
111 reg          ready_reg;
112
113 //-----
114 // Wires.
115 //-----
116
117 reg [31 : 0] tmp_read_data;
118
119 wire        core_encdec;
120 wire        core_init;
121 wire        core_next;
122 wire        core_ready;
123 wire [255 : 0] core_key;
124 wire        core_keylen;
125 wire [127 : 0] core_block;

```



```

125 wire [127 : 0] core_result;
126 wire          core_valid;
127
128 //-----
129 // Concurrent connectivity for ports etc.
130 //-----
131
132 assign read_data = tmp_read_data;
133
134 assign core_key = {key_reg[0], key_reg[1], key_reg[2],
key_reg[3],
135                  key_reg[4], key_reg[5], key_reg[6],
key_reg[7]};
136
137 assign core_block = {block_reg[0], block_reg[1],
138                     block_reg[2], block_reg[3]};
139 assign core_init  = init_reg;
140 assign core_next  = next_reg;
141 assign core_encdec = encdec_reg;
142 assign core_keylen = keylen_reg;
143
144 //-----
145 // core instantiation.
146 //-----
147
148 aes_core core(
149     .clk(clk),
150     .reset_n(reset_n),
151
152     .encdec(core_encdec),
153     .init(core_init),
154     .next(core_next),
155     .ready(core_ready),
156
157     .key(core_key),
158     .keylen(core_keylen),
159
160     .block(core_block),
161     .result(core_result),
162     .result_valid(core_valid)
163 );
164
165 //-----
166 // reg_update
167 // Update functionality for all registers in the core.
168 // All registers are positive edge triggered with asynchron-
169 // ous
170 // active low reset.
171 //-----
172
173 always @ (posedge clk or negedge reset_n)
174     begin : reg_update
175         integer i;
176
177         if (!reset_n)
178             begin
179                 for (i = 0 ; i < 4 ; i = i + 1)
180                     block_reg[i] <= 32'h0;
181
182                 for (i = 0 ; i < 8 ; i = i + 1)
183                     key_reg[i] <= 32'h0;
184
185                 init_reg  <= 1'b0;
186                 next_reg  <= 1'b0;
187                 encdec_reg <= 1'b0;
188                 keylen_reg <= 1'b0;
189
190                 result_reg <= 128'h0;
191                 valid_reg  <= 1'b0;
192                 ready_reg  <= 1'b0;
193             end
194         else
195             begin
196                 ready_reg <= core_ready;
197                 valid_reg <= core_valid;
198                 result_reg <= core_result;
199                 init_reg  <= init_new;
200                 next_reg  <= next_new;
201
202                 if (config_we)
203                     begin
204                         encdec_reg <= write_data[CTRL_ENCDEC_BIT];
205                         keylen_reg <= write_data[CTRL_KEYLEN_BIT];
206                     end
207
208                 if (key_we)
209                     key_reg[address[2 : 0]] <= write_data;
210
211                 if (block_we)
212                     block_reg[address[1 : 0]] <= write_data;
213             end
214         end // reg_update
215
216 //-----

```

```

217 // api
218 //
219 // The interface command decoding logic.
220 //-----
221
222 always @*
223     begin : api
224         init_new  = 1'b0;
225         next_new  = 1'b0;
226         config_we = 1'b0;
227         key_we    = 1'b0;
228         block_we  = 1'b0;
229         tmp_read_data = 32'h0;
230
231         if (cs)
232             begin
233                 if (we)
234                     begin
235                         if (address == ADDR_CTRL)
236                             begin
237                                 init_new = write_data[CTRL_INIT_BIT];
238                                 next_new = write_data[CTRL_NEXT_BIT];
239                             end
240
241                         if (address == ADDR_CONFIG)
242                             config_we = 1'b1;
243
244                         if ((address >= ADDR_KEY0) && (address <=
ADDR_KEY7))
245                             key_we = 1'b1;
246
247                         if ((address >= ADDR_BLOCK0) && (address <=
ADDR_BLOCK3))
248                             block_we = 1'b1;
249                     end // if (we)
250                 else
251                     begin
252                         case (address)
253                             ADDR_NAME0: tmp_read_data = CORE_NAME0;
254                             ADDR_NAME1: tmp_read_data = CORE_NAME1;
255                             ADDR_VERSION: tmp_read_data = CORE_VERSION;
256                             ADDR_CTRL:   tmp_read_data = {28'h0,
keylen_reg, encdec_reg, next_reg, init_reg};
257                             ADDR_STATUS: tmp_read_data = {30'h0, val-
id_reg, ready_reg};
258
259                         default:
260                             begin
261                                 end
262                             endcase // case (address)
263
264                         if ((address >= ADDR_RESULT0) && (address <=
ADDR_RESULT3))
265                             tmp_read_data = result_reg[{3 - (address -
ADDR_RESULT0)} * 32 +: 32];
266                     end
267                 end // addr_decoder
268             endmodule // aes
269
270 //
271 //

```

```

272 // EOF aes.v
273 //

```

2. secworks_aes/src/rtl/aes_core.v (342 lines)

```

1 //
=====
2 //
3 // aes_core.v
4 // -----
5 // The AES core. This core supports key size of 128, and 256
bits.
6 // Most of the functionality is within the submodules.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2013, 2014, Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the fol-
lowing
15 // conditions are met:
16 //
17 // 1. Redistributions of source code must retain the above
copyright
18 // notice, this list of conditions and the following dis-
claimer.
19 //
20 // 2. Redistributions in binary form must reproduce the above
copyright
21 // notice, this list of conditions and the following dis-

```

```

claimer in
22 // the documentation and/or other materials provided with
the
23 // distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
TRIBUTORS
26 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
27 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
28 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
29 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
30 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
31 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
32 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
33 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
35 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====
==
39
40 `default_nettype none
41
42 module aes_core(
43     input wire      clk,
44     input wire      reset_n,
45
46     input wire      encdec,
47     input wire      init,
48     input wire      next,
49     output wire     ready,
50
51     input wire [255 : 0] key,
52     input wire      keylen,
53
54     input wire [127 : 0] block,
55     output wire [127 : 0] result,
56     output wire      result_valid
57 );
58
59
60
61
62 //-----
63 // Internal constant and parameter definitions.
64 //-----
65 localparam CTRL_IDLE = 2'h0;
66 localparam CTRL_INIT = 2'h1;
67 localparam CTRL_NEXT = 2'h2;
68
69
70 //-----
71 // Registers including update variables and write enable.
72 //-----
73 reg [1 : 0] aes_core_ctrl_reg;
74 reg [1 : 0] aes_core_ctrl_new;
75 reg        aes_core_ctrl_we;
76
77 reg        result_valid_reg;
78 reg        result_valid_new;
79 reg        result_valid_we;
80
81 reg        ready_reg;
82 reg        ready_new;
83 reg        ready_we;
84
85
86 //-----
87 // Wires.
88 //-----
89 reg        init_state;
90
91 wire [127 : 0] round_key;
92 wire          key_ready;
93
94 reg        enc_next;
95 wire [3 : 0] enc_round_nr;
96 wire [127 : 0] enc_new_block;
97 wire          enc_ready;
98 wire [31 : 0] enc_sboxw;
99
100 reg        dec_next;
101 wire [3 : 0] dec_round_nr;
102 wire [127 : 0] dec_new_block;
103 wire          dec_ready;

```

```

104
105 reg [127 : 0] muxed_new_block;
106 reg [3 : 0]  muxed_round_nr;
107 reg          muxed_ready;
108
109 wire [31 : 0] keymem_sboxw;
110
111 /* verilator lint_off UNOPTFLAT */
112 reg [31 : 0] muxed_sboxw;
113 wire [31 : 0] new_sboxw;
114 /* verilator lint_on UNOPTFLAT */
115
116
117 //-----
118 // Instantiations.
119 //-----
120
121 aes_encipher_block enc_block(
122     .clk(clk),
123     .reset_n(reset_n),
124     .next(enc_next),
125
126     .keylen(keylen),
127     .round(enc_round_nr),
128     .round_key(round_key),
129
130     .sboxw(enc_sboxw),
131     .new_sboxw(new_sboxw),
132
133     .block(block),
134     .new_block(enc_new_block),
135     .ready(enc_ready)
136 );
137
138
139 aes_decipher_block dec_block(
140     .clk(clk),
141     .reset_n(reset_n),
142     .next(dec_next),
143
144     .keylen(keylen),
145     .round(dec_round_nr),
146     .round_key(round_key),
147
148     .block(block),
149     .new_block(dec_new_block),
150     .ready(dec_ready)
151 );
152
153
154
155 aes_key_mem keymem(
156     .clk(clk),
157     .reset_n(reset_n),
158
159     .key(key),
160     .keylen(keylen),
161     .init(init),
162
163     .round(muxed_round_nr),
164     .round_key(round_key),
165     .ready(key_ready),
166
167     .sboxw(keymem_sboxw),
168     .new_sboxw(new_sboxw)
169 );
170
171
172 aes_sbox
sbox_inst(.sboxw(muxed_sboxw), .new_sboxw(new_sboxw));
173
174
175 //-----
176 // Concurrent connectivity for ports etc.
177 //-----
178
179 assign ready      = ready_reg;
180 assign result     = muxed_new_block;
181 assign result_valid = result_valid_reg;
182
183 //-----
184 // reg_update
185 //
186 // Update functionality for all registers in the core.
187 // All registers are positive edge triggered with asynchron-
ous
188 // active low reset. All registers have write enable.
189 //-----
190
191 always @ (posedge clk or negedge reset_n)
192 begin: reg_update
193     if (!reset_n)
194     begin
195         result_valid_reg <= 1'b0;
196         ready_reg        <= 1'b1;
197         aes_core_ctrl_reg <= CTRL_IDLE;

```

```

197     end
198   else
199     begin
200       if (result_valid_we)
201         result_valid_reg <= result_valid_new;
202
203       if (ready_we)
204         ready_reg <= ready_new;
205
206       if (aes_core_ctrl_we)
207         aes_core_ctrl_reg <= aes_core_ctrl_new;
208     end
209   end // reg_update
210
211 //-----
212 // sbox_mux
213 //
214 // Controls which of the encipher datapath or the key memory
215 // that gets access to the sbox.
216 //-----
217
218 always @*
219   begin : sbox_mux
220     if (init_state)
221       begin
222         muxed_sboxw = keymem_sboxw;
223       end
224     else
225       begin
226         muxed_sboxw = enc_sboxw;
227       end
228   end // sbox_mux
229
230 //-----
231 // encdex_mux
232 //
233
234 // Controls which of the datapaths that get the next signal,
235 // have
236 // access to the memory as well as the block processing res-
237 // ult.
238 //-----
239
240 always @*
241   begin : encdec_mux
242     enc_next = 1'b0;
243     dec_next = 1'b0;
244
245     if (encdec)
246       begin
247         // Encipher operations
248         enc_next = next;
249         muxed_round_nr = enc_round_nr;
250         muxed_new_block = enc_new_block;
251         muxed_ready = enc_ready;
252       end
253     else
254       begin
255         // Decipher operations
256         dec_next = next;
257         muxed_round_nr = dec_round_nr;
258         muxed_new_block = dec_new_block;
259         muxed_ready = dec_ready;
260       end
261   end // encdec_mux
262
263 //-----
264 // aes_core_ctrl
265 //
266 // Control FSM for aes core. Basically tracks if we are in
267 // key init, encipher or decipher modes and connects the
268 // different submodules to shared resources and interface
269 // ports.
270 //-----
271
272 always @*
273   begin : aes_core_ctrl
274     init_state = 1'b0;
275     ready_new = 1'b0;
276     ready_we = 1'b0;
277     result_valid_new = 1'b0;
278     result_valid_we = 1'b0;
279     aes_core_ctrl_new = CTRL_IDLE;
280     aes_core_ctrl_we = 1'b0;
281
282     case (aes_core_ctrl_reg)
283       CTRL_IDLE:
284         begin
285           if (init)
286             begin
287               init_state = 1'b1;
288               ready_new = 1'b0;
289               ready_we = 1'b1;
290               result_valid_new = 1'b0;
291               result_valid_we = 1'b1;

```

```

288     aes_core_ctrl_new = CTRL_INIT;
289     aes_core_ctrl_we = 1'b1;
290   end
291   else if (next)
292     begin
293       init_state = 1'b0;
294       ready_new = 1'b0;
295       ready_we = 1'b1;
296       result_valid_new = 1'b0;
297       result_valid_we = 1'b1;
298       aes_core_ctrl_new = CTRL_NEXT;
299       aes_core_ctrl_we = 1'b1;
300     end
301   end
302
303 CTRL_INIT:
304   begin
305     init_state = 1'b1;
306
307     if (key_ready)
308       begin
309         ready_new = 1'b1;
310         ready_we = 1'b1;
311         aes_core_ctrl_new = CTRL_IDLE;
312         aes_core_ctrl_we = 1'b1;
313       end
314     end
315
316 CTRL_NEXT:
317   begin
318     init_state = 1'b0;
319
320     if (muxed_ready)
321       begin
322         ready_new = 1'b1;
323         ready_we = 1'b1;
324         result_valid_new = 1'b1;
325         result_valid_we = 1'b1;
326         aes_core_ctrl_new = CTRL_IDLE;
327         aes_core_ctrl_we = 1'b1;
328       end
329     end
330
331   default:
332     begin
333
334     end
335   endcase // case (aes_core_ctrl_reg)
336
337   end // aes_core_ctrl
338 endmodule // aes_core
339
340 //

```

```

=====
341 // EOF aes_core.v
342 //
=====

```

3. secworks_aes/src/rtl/aes_decipher_block.v (526 lines)

```

1 //
=====
2 //
3 // aes_decipher_block.v
4 // -----
5 // The AES decipher round. A pure combinational module that
6 // implements
7 // the initial round, main round and final round logic for
8 // decipher operations.
9 //
10 // Author: Joachim Strombergson
11 // Copyright (c) 2013, 2014, Secworks Sweden AB
12 // All rights reserved.
13 //
14 // Redistribution and use in source and binary forms, with or
15 // without modification, are permitted provided that the fol-
16 // lowing
17 // conditions are met:
18 // 1. Redistributions of source code must retain the above
19 // copyright
20 // notice, this list of conditions and the following dis-
21 // claimer.
22 // 2. Redistributions in binary form must reproduce the above
23 // copyright
24 // notice, this list of conditions and the following dis-
25 // claimer in
26 // the documentation and/or other materials provided with
27 // the
28 // distribution.
29 //
30 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
31 // TRIBUTORS
32 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
33 // BUT NOT

```

```

28 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
29 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
30 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
31 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
32 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
33 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
34 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
35 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
36 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
37 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
38 //
39 //
=====
==
40
41 `default_nettype none
42
43 module aes_decipher_block(
44     input wire          clk,
45     input wire          reset_n,
46
47     input wire          next,
48
49     input wire          keylen,
50     output wire [3 : 0] round,
51     input wire [127 : 0] round_key,
52
53     input wire [127 : 0] block,
54     output wire [127 : 0] new_block,
55     output wire          ready
56 );
57
58
59 //-----
60 // Internal constant and parameter definitions.
61 //-----
62 localparam AES_128_BIT_KEY = 1'h0;
63 localparam AES_256_BIT_KEY = 1'h1;
64
65 localparam AES128_ROUNDS = 4'h0;
66 localparam AES256_ROUNDS = 4'h1;
67
68 localparam NO_UPDATE = 3'h0;
69 localparam INIT_UPDATE = 3'h1;
70 localparam SBOX_UPDATE = 3'h2;
71 localparam MAIN_UPDATE = 3'h3;
72 localparam FINAL_UPDATE = 3'h4;
73
74 localparam CTRL_IDLE = 2'h0;
75 localparam CTRL_INIT = 2'h1;
76 localparam CTRL_SBOX = 2'h2;
77 localparam CTRL_MAIN = 2'h3;
78
79
80 //-----
81 // Gaolis multiplication functions for Inverse MixColumn.
82 //-----
83 function automatic [7 : 0] gm2(input [7 : 0] op);
84 begin
85     gm2 = {op[6 : 0], 1'b0} ^ {8'h1b & {8{op[7]}}};
86 end
87 endfunction // gm2
88
89 function automatic [7 : 0] gm3(input [7 : 0] op);
90 begin
91     gm3 = gm2(op) ^ op;
92 end
93 endfunction // gm3
94
95 function automatic [7 : 0] gm4(input [7 : 0] op);
96 begin
97     gm4 = gm2(gm2(op));
98 end
99 endfunction // gm4
100
101 function automatic [7 : 0] gm8(input [7 : 0] op);
102 begin
103     gm8 = gm2(gm4(op));
104 end
105 endfunction // gm8
106
107 function automatic [7 : 0] gm09(input [7 : 0] op);
108 begin
109     gm09 = gm8(op) ^ op;
110 end
111 endfunction // gm09
112
113 function automatic [7 : 0] gm11(input [7 : 0] op);
114 begin
115     gm11 = gm8(op) ^ gm2(op) ^ op;

```

```

116 end
117 endfunction // gm11
118
119 function automatic [7 : 0] gm13(input [7 : 0] op);
120 begin
121     gm13 = gm8(op) ^ gm4(op) ^ op;
122 end
123 endfunction // gm13
124
125 function automatic [7 : 0] gm14(input [7 : 0] op);
126 begin
127     gm14 = gm8(op) ^ gm4(op) ^ gm2(op);
128 end
129 endfunction // gm14
130
131 function automatic [31 : 0] inv_mixw(input [31 : 0] w);
132 reg [7 : 0] b0, b1, b2, b3;
133 reg [7 : 0] mb0, mb1, mb2, mb3;
134 begin
135     b0 = w[31 : 24];
136     b1 = w[23 : 16];
137     b2 = w[15 : 08];
138     b3 = w[07 : 00];
139
140     mb0 = gm14(b0) ^ gm11(b1) ^ gm13(b2) ^ gm09(b3);
141     mb1 = gm09(b0) ^ gm14(b1) ^ gm11(b2) ^ gm13(b3);
142     mb2 = gm13(b0) ^ gm09(b1) ^ gm14(b2) ^ gm11(b3);
143     mb3 = gm11(b0) ^ gm13(b1) ^ gm09(b2) ^ gm14(b3);
144
145     inv_mixw = {mb0, mb1, mb2, mb3};
146 end
147 endfunction // mixw
148
149 function automatic [127 : 0] inv_mixcolumns(input [127 : 0]
data);
150 reg [31 : 0] w0, w1, w2, w3;
151 reg [31 : 0] ws0, ws1, ws2, ws3;
152 begin
153     w0 = data[127 : 096];
154     w1 = data[095 : 064];
155     w2 = data[063 : 032];
156     w3 = data[031 : 000];
157
158     ws0 = inv_mixw(w0);
159     ws1 = inv_mixw(w1);
160     ws2 = inv_mixw(w2);
161     ws3 = inv_mixw(w3);
162
163     inv_mixcolumns = {ws0, ws1, ws2, ws3};
164 end
165 endfunction // inv_mixcolumns
166
167 function automatic [127 : 0] inv_shiftrows(input [127 : 0]
data);
168 reg [31 : 0] w0, w1, w2, w3;
169 reg [31 : 0] ws0, ws1, ws2, ws3;
170 begin
171     w0 = data[127 : 096];
172     w1 = data[095 : 064];
173     w2 = data[063 : 032];
174     w3 = data[031 : 000];
175
176     ws0 = {w0[31 : 24], w3[23 : 16], w2[15 : 08], w1[07 :
00]};
177     ws1 = {w1[31 : 24], w0[23 : 16], w3[15 : 08], w2[07 :
00]};
178     ws2 = {w2[31 : 24], w1[23 : 16], w0[15 : 08], w3[07 :
00]};
179     ws3 = {w3[31 : 24], w2[23 : 16], w1[15 : 08], w0[07 :
00]};
180
181     inv_shiftrows = {ws0, ws1, ws2, ws3};
182 end
183 endfunction // inv_shiftrows
184
185 function automatic [127 : 0] addroundkey(input [127 : 0]
data, input [127 : 0] rkey);
186 begin
187     addroundkey = data ^ rkey;
188 end
189 endfunction // addroundkey
190
191
192 //-----
193 // Registers including update variables and write enable.
194 //-----
195 reg [1 : 0] sword_ctr_reg;
196 reg [1 : 0] sword_ctr_new;
197 reg sword_ctr_we;
198 reg sword_ctr_inc;
199 reg sword_ctr_rst;
200
201 reg [3 : 0] round_ctr_reg;
202 reg [3 : 0] round_ctr_new;
203 reg round_ctr_we;
204 reg round_ctr_set;
205 reg round_ctr_dec;
206
207 reg [127 : 0] block_new;

```

```

208 reg [31 : 0] block_w0_reg;
209 reg [31 : 0] block_w1_reg;
210 reg [31 : 0] block_w2_reg;
211 reg [31 : 0] block_w3_reg;
212 reg block_w0_we;
213 reg block_w1_we;
214 reg block_w2_we;
215 reg block_w3_we;
216
217 reg ready_reg;
218 reg ready_new;
219 reg ready_we;
220
221 reg [1 : 0] dec_ctrl_reg;
222 reg [1 : 0] dec_ctrl_new;
223 reg dec_ctrl_we;
224
225 //-----
226 // Wires.
227 //-----
228
229 reg [31 : 0] tmp_sboxw;
230 wire [31 : 0] new_sboxw;
231 reg [2 : 0] update_type;
232
233 //-----
234 // Instantiations.
235 //-----
236
237 aes_inv_sbox inv_s-
box_inst(.sboxw(tmp_sboxw), .new_sboxw(new_sboxw));
238
239 //-----
240 // Concurrent connectivity for ports etc.
241 //-----
242
243 assign round = round_ctr_reg;
244 assign new_block = {block_w0_reg, block_w1_reg,
block_w2_reg, block_w3_reg};
245 assign ready = ready_reg;
246
247 //-----
248 // reg_update
249 //
250 // Update functionality for all registers in the core.
251 // All registers are positive edge triggered with synchron-
ous
252 // active low reset. All registers have write enable.
253 //-----
254
255 always @ (posedge clk or negedge reset_n)
256 begin: reg_update
257 if (!reset_n)
258 begin
259 block_w0_reg <= 32'h0;
260 block_w1_reg <= 32'h0;
261 block_w2_reg <= 32'h0;
262 block_w3_reg <= 32'h0;
263 sword_ctr_reg <= 2'h0;
264 round_ctr_reg <= 4'h0;
265 ready_reg <= 1'b1;
266 dec_ctrl_reg <= CTRL_IDLE;
267 end
268 else
269 begin
270 if (block_w0_we)
271 block_w0_reg <= block_new[127 : 096];
272
273 if (block_w1_we)
274 block_w1_reg <= block_new[095 : 064];
275
276 if (block_w2_we)
277 block_w2_reg <= block_new[063 : 032];
278
279 if (block_w3_we)
280 block_w3_reg <= block_new[031 : 000];
281
282 if (sword_ctr_we)
283 sword_ctr_reg <= sword_ctr_new;
284
285 if (round_ctr_we)
286 round_ctr_reg <= round_ctr_new;
287
288 if (ready_we)
289 ready_reg <= ready_new;
290
291 if (dec_ctrl_we)
292 dec_ctrl_reg <= dec_ctrl_new;
293 end
294 end // reg_update
295
296 //-----
297

```

```

-----
298 // round_logic
299 //
300 // The logic needed to implement init, main and final
rounds.
301 //-----
302
303 always @*
304 begin : round_logic
305 reg [127 : 0] old_block, inv_shiftrows_block, inv_mix-
columns_block;
306 reg [127 : 0] addkey_block;
307
308 inv_shiftrows_block = 128'h0;
309 inv_mixcolumns_block = 128'h0;
310 addkey_block = 128'h0;
311 block_new = 128'h0;
312 tmp_sboxw = 32'h0;
313 block_w0_we = 1'b0;
314 block_w1_we = 1'b0;
315 block_w2_we = 1'b0;
316 block_w3_we = 1'b0;
317
318 old_block = {block_w0_reg, block_w1_reg,
block_w2_reg, block_w3_reg};
319
320 // Update based on update type.
321 case (update_type)
322 // InitRound
323 INIT_UPDATE:
324 begin
325 old_block = block;
326 addkey_block = addroundkey(old_block,
round_key);
327
328 inv_shiftrows_block = inv_shiftrows(addkey_block);
329 block_new = inv_shiftrows_block;
330 block_w0_we = 1'b1;
331 block_w1_we = 1'b1;
332 block_w2_we = 1'b1;
333 block_w3_we = 1'b1;
334 end
335
336 SBUX_UPDATE:
337 begin
338 block_new = {new_sboxw, new_sboxw, new_sboxw,
new_sboxw};
339
340 case (sword_ctr_reg)
341 2'h0:
342 begin
343 tmp_sboxw = block_w0_reg;
344 block_w0_we = 1'b1;
345 end
346
347 2'h1:
348 begin
349 tmp_sboxw = block_w1_reg;
350 block_w1_we = 1'b1;
351 end
352
353 2'h2:
354 begin
355 tmp_sboxw = block_w2_reg;
356 block_w2_we = 1'b1;
357 end
358
359 2'h3:
360 begin
361 tmp_sboxw = block_w3_reg;
362 block_w3_we = 1'b1;
363 end
364 endcase // case (sbox_mux_ctrl_reg)
365 end
366
367 MAIN_UPDATE:
368 begin
369 addkey_block = addroundkey(old_block,
round_key);
370 inv_mixcolumns_block = inv_mix-
columns(addkey_block);
371 inv_shiftrows_block =
inv_shiftrows(inv_mixcolumns_block);
372 block_new = inv_shiftrows_block;
373 block_w0_we = 1'b1;
374 block_w1_we = 1'b1;
375 block_w2_we = 1'b1;
376 block_w3_we = 1'b1;
377 end
378
379 FINAL_UPDATE:
380 begin
381 block_new = addroundkey(old_block, round_key);
382 block_w0_we = 1'b1;
383 block_w1_we = 1'b1;
384 block_w2_we = 1'b1;
385 block_w3_we = 1'b1;
386 end
387
388 default:
389 begin
390 end
391 end

```

```

389     endcase // case (update_type)
390 end // round_logic
391
392 //-----
393 // sword_ctr
394 //
395 // The subbytes word counter with reset and increase logic.
396 //-----
397
398 always @*
399 begin : sword_ctr
400     sword_ctr_new = 2'h0;
401     sword_ctr_we = 1'b0;
402
403     if (sword_ctr_rst)
404     begin
405         sword_ctr_new = 2'h0;
406         sword_ctr_we = 1'b1;
407     end
408     else if (sword_ctr_inc)
409     begin
410         sword_ctr_new = sword_ctr_reg + 1'b1;
411         sword_ctr_we = 1'b1;
412     end
413 end // sword_ctr
414
415 //-----
416 // round_ctr
417 //
418 // The round counter with reset and increase logic.
419 //-----
420
421 always @*
422 begin : round_ctr
423     round_ctr_new = 4'h0;
424     round_ctr_we = 1'b0;
425
426     if (round_ctr_set)
427     begin
428         if (keylen == AES_256_BIT_KEY)
429         begin
430             round_ctr_new = AES256_ROUNDS;
431         end
432         else
433         begin
434             round_ctr_new = AES128_ROUNDS;
435         end
436         round_ctr_we = 1'b1;
437     end
438     else if (round_ctr_dec)
439     begin
440         round_ctr_new = round_ctr_reg - 1'b1;
441         round_ctr_we = 1'b1;
442     end
443 end // round_ctr
444
445 //-----
446 // decipher_ctrl
447 //
448 // The FSM that controls the decipher operations.
449 //-----
450
451 always @*
452 begin: decipher_ctrl
453     sword_ctr_inc = 1'b0;
454     sword_ctr_rst = 1'b0;
455     round_ctr_dec = 1'b0;
456     round_ctr_set = 1'b0;
457     ready_new = 1'b0;
458     ready_we = 1'b0;
459     update_type = NO_UPDATE;
460     dec_ctrl_new = CTRL_IDLE;
461     dec_ctrl_we = 1'b0;
462
463     case(dec_ctrl_reg)
464     CTRL_IDLE:
465     begin
466         if (next)
467         begin
468             round_ctr_set = 1'b1;
469             ready_new = 1'b0;
470             ready_we = 1'b1;
471             dec_ctrl_new = CTRL_INIT;
472             dec_ctrl_we = 1'b1;
473         end
474     end
475
476     CTRL_INIT:
477     begin
478         sword_ctr_rst = 1'b1;
479         update_type = INIT_UPDATE;
480         dec_ctrl_new = CTRL_SBOX;
481         dec_ctrl_we = 1'b1;
482     end
483

```

```

484 CTRL_SBOX:
485 begin
486     sword_ctr_inc = 1'b1;
487     update_type = SBOX_UPDATE;
488     if (sword_ctr_reg == 2'h3)
489     begin
490         round_ctr_dec = 1'b1;
491         dec_ctrl_new = CTRL_MAIN;
492         dec_ctrl_we = 1'b1;
493     end
494 end
495
496 CTRL_MAIN:
497 begin
498     sword_ctr_rst = 1'b1;
499     if (round_ctr_reg > 0)
500     begin
501         update_type = MAIN_UPDATE;
502         dec_ctrl_new = CTRL_SBOX;
503         dec_ctrl_we = 1'b1;
504     end
505     else
506     begin
507         update_type = FINAL_UPDATE;
508         ready_new = 1'b1;
509         ready_we = 1'b1;
510         dec_ctrl_new = CTRL_IDLE;
511         dec_ctrl_we = 1'b1;
512     end
513 end
514
515 default:
516 begin
517     // Empty. Just here to make the synthesis tool
518     happy.
519     end
520 endcase // case (dec_ctrl_reg)
521 end // decipher_ctrl
522 endmodule // aes_decipher_block
523
524 //
=====
525 // EOF aes_decipher_block.v
526 //
=====

```

4. secworks_aes/src/rtl/aes_encipher_block.v (487 lines)

```

1 //
=====
2 //
3 // aes_encipher_block.v
4 //-----
5 // The AES encipher round. A pure combinational module that
6 // implements
7 // the initial round, main round and final round logic for
8 // encipher operations.
9 //
10 // Author: Joachim Strombergson
11 // Copyright (c) 2013, 2014, Secworks Sweden AB
12 // All rights reserved.
13 //
14 // Redistribution and use in source and binary forms, with or
15 // without modification, are permitted provided that the fol-
16 // lowing
17 // conditions are met:
18 // 1. Redistributions of source code must retain the above
19 // copyright
20 // notice, this list of conditions and the following dis-
21 // claimer.
22 // 2. Redistributions in binary form must reproduce the above
23 // copyright
24 // notice, this list of conditions and the following dis-
25 // claimer in
26 // the documentation and/or other materials provided with
27 // the
28 // distribution.
29 //
30 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
31 // TRIBUTORS
32 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
33 // BUT NOT
34 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
35 // FITNESS
36 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
37 // THE
38 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
39 // INDIRECT,
40 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
41 // (INCLUDING,
42 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
43 // VICES;
44 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)

```

```

HOWEVER
34 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
35 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
36 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
37 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
38 //
39 //
=====
==
40
41 `default_nettype none
42
43 module aes_encipher_block(
44     input wire          clk,
45     input wire          reset_n,
46
47     input wire          next,
48
49     input wire          keylen,
50     output wire [3 : 0] round,
51     input wire [127 : 0] round_key,
52
53     output wire [31 : 0] sboxw,
54     input wire  [31 : 0] new_sboxw,
55
56     input wire [127 : 0] block,
57     output wire [127 : 0] new_block,
58     output wire          ready
59 );
60
61
62 //-----
63 // Internal constant and parameter definitions.
64 //-----
65 localparam AES_128_BIT_KEY = 1'h0;
66 localparam AES_256_BIT_KEY = 1'h1;
67
68 localparam AES128_ROUNDS = 4'ha;
69 localparam AES256_ROUNDS = 4'he;
70
71 localparam NO_UPDATE = 3'h0;
72 localparam INIT_UPDATE = 3'h1;
73 localparam SBOX_UPDATE = 3'h2;
74 localparam MAIN_UPDATE = 3'h3;
75 localparam FINAL_UPDATE = 3'h4;
76
77 localparam CTRL_IDLE = 2'h0;
78 localparam CTRL_INIT = 2'h1;
79 localparam CTRL_SBOX = 2'h2;
80 localparam CTRL_MAIN = 2'h3;
81
82
83 //-----
84 // Round functions with sub functions.
85 //-----
86 function automatic [7 : 0] gm2(input [7 : 0] op);
87     begin
88         gm2 = {op[6 : 0], 1'b0} ^ (8'h1b & {8{op[7]}});
89     end
90 endfunction // gm2
91
92 function automatic [7 : 0] gm3(input [7 : 0] op);
93     begin
94         gm3 = gm2(op) ^ op;
95     end
96 endfunction // gm3
97
98 function automatic [31 : 0] mixw(input [31 : 0] w);
99     reg [7 : 0] b0, b1, b2, b3;
100     reg [7 : 0] mb0, mb1, mb2, mb3;
101     begin
102         b0 = w[31 : 24];
103         b1 = w[23 : 16];
104         b2 = w[15 : 08];
105         b3 = w[07 : 00];
106
107         mb0 = gm2(b0) ^ gm3(b1) ^ b2 ^ b3;
108         mb1 = b0 ^ gm2(b1) ^ gm3(b2) ^ b3;
109         mb2 = b0 ^ b1 ^ gm2(b2) ^ gm3(b3);
110         mb3 = gm3(b0) ^ b1 ^ b2 ^ gm2(b3);
111
112         mixw = {mb0, mb1, mb2, mb3};
113     end
114 endfunction // mixw
115
116 function automatic [127 : 0] mixcolumns(input [127 : 0]
data);
117     reg [31 : 0] w0, w1, w2, w3;
118     reg [31 : 0] ws0, ws1, ws2, ws3;
119     begin
120         w0 = data[127 : 096];
121         w1 = data[095 : 064];
122         w2 = data[063 : 032];
123         w3 = data[031 : 000];
124
125         ws0 = mixw(w0);

```

```

126         ws1 = mixw(w1);
127         ws2 = mixw(w2);
128         ws3 = mixw(w3);
129
130         mixcolumns = {ws0, ws1, ws2, ws3};
131     end
132 endfunction // mixcolumns
133
134 function automatic [127 : 0] shiftrows(input [127 : 0]
data);
135     reg [31 : 0] w0, w1, w2, w3;
136     reg [31 : 0] ws0, ws1, ws2, ws3;
137     begin
138         w0 = data[127 : 096];
139         w1 = data[095 : 064];
140         w2 = data[063 : 032];
141         w3 = data[031 : 000];
142
143         ws0 = {w0[31 : 24], w1[23 : 16], w2[15 : 08], w3[07 :
00]};
144         ws1 = {w1[31 : 24], w2[23 : 16], w3[15 : 08], w0[07 :
00]};
145         ws2 = {w2[31 : 24], w3[23 : 16], w0[15 : 08], w1[07 :
00]};
146         ws3 = {w3[31 : 24], w0[23 : 16], w1[15 : 08], w2[07 :
00]};
147
148         shiftrows = {ws0, ws1, ws2, ws3};
149     end
150 endfunction // shiftrows
151
152 function automatic [127 : 0] addroundkey(input [127 : 0]
data, input [127 : 0] rkey);
153     begin
154         addroundkey = data ^ rkey;
155     end
156 endfunction // addroundkey
157
158 //-----
159 // Registers including update variables and write enable.
160 //-----
161
162 reg [1 : 0] sword_ctr_reg;
163 reg [1 : 0] sword_ctr_new;
164 reg sword_ctr_we;
165 reg sword_ctr_inc;
166 reg sword_ctr_rst;
167
168 reg [3 : 0] round_ctr_reg;
169 reg [3 : 0] round_ctr_new;
170 reg round_ctr_we;
171 reg round_ctr_rst;
172 reg round_ctr_inc;
173
174 reg [127 : 0] block_new;
175 reg [31 : 0] block_w0_reg;
176 reg [31 : 0] block_w1_reg;
177 reg [31 : 0] block_w2_reg;
178 reg [31 : 0] block_w3_reg;
179 reg block_w0_we;
180 reg block_w1_we;
181 reg block_w2_we;
182 reg block_w3_we;
183
184 reg ready_reg;
185 reg ready_new;
186 reg ready_we;
187
188 reg [1 : 0] enc_ctrl_reg;
189 reg [1 : 0] enc_ctrl_new;
190 reg enc_ctrl_we;
191
192
193 //-----
194 // Wires.
195 //-----
196 reg [2 : 0] update_type;
197 reg [31 : 0] muxed_sboxw;
198
199 //-----
200 // Concurrent connectivity for ports etc.
201 //-----
202
203 assign round = round_ctr_reg;
204 assign sboxw = muxed_sboxw;
205 assign new_block = {block_w0_reg, block_w1_reg,
block_w2_reg, block_w3_reg};
206 assign ready = ready_reg;
207
208 //-----
209 // reg_update
210 //
211 // Update functionality for all registers in the core.
212

```



```

213 // All registers are positive edge triggered with asynchron-
ous
214 // active low reset. All registers have write enable.
215 //-----
216 always @ (posedge clk or negedge reset_n)
217 begin: reg_update
218   if (!reset_n)
219     begin
220       block_w0_reg <= 32'h0;
221       block_w1_reg <= 32'h0;
222       block_w2_reg <= 32'h0;
223       block_w3_reg <= 32'h0;
224       sword_ctr_reg <= 2'h0;
225       round_ctr_reg <= 4'h0;
226       ready_reg <= 1'b1;
227       enc_ctrl_reg <= CTRL_IDLE;
228     end
229   else
230     begin
231       if (block_w0_we)
232         block_w0_reg <= block_new[127 : 096];
233       if (block_w1_we)
234         block_w1_reg <= block_new[095 : 064];
235       if (block_w2_we)
236         block_w2_reg <= block_new[063 : 032];
237       if (block_w3_we)
238         block_w3_reg <= block_new[031 : 000];
239       if (sword_ctr_we)
240         sword_ctr_reg <= sword_ctr_new;
241       if (round_ctr_we)
242         round_ctr_reg <= round_ctr_new;
243       if (ready_we)
244         ready_reg <= ready_new;
245       if (enc_ctrl_we)
246         enc_ctrl_reg <= enc_ctrl_new;
247     end
248   end // reg_update
249
250 //-----
251 // round_logic
252 //
253 // The logic needed to implement init, main and final
254 rounds.
255 //-----
256 always @*
257 begin : round_logic
258   reg [127 : 0] old_block, shiftrows_block, mix-
columns_block;
259   reg [127 : 0] addkey_init_block, addkey_main_block, ad-
dkey_final_block;
260   block_new = 128'h0;
261   muxed_sboxw = 32'h0;
262   block_w0_we = 1'b0;
263   block_w1_we = 1'b0;
264   block_w2_we = 1'b0;
265   block_w3_we = 1'b0;
266   old_block = {block_w0_reg, block_w1_reg,
block_w2_reg, block_w3_reg};
267   shiftrows_block = shiftrows(old_block);
268   mixcolumns_block = mixcolumns(shiftrows_block);
269   addkey_init_block = addroundkey(block, round_key);
270   addkey_main_block = addroundkey(mixcolumns_block,
round_key);
271   addkey_final_block = addroundkey(shiftrows_block,
round_key);
272   case (update_type)
273     INIT_UPDATE:
274       begin
275         block_new = addkey_init_block;
276         block_w0_we = 1'b1;
277         block_w1_we = 1'b1;
278         block_w2_we = 1'b1;
279         block_w3_we = 1'b1;
280       end
281     SBOX_UPDATE:
282       begin
283         block_new = {new_sboxw, new_sboxw, new_sboxw,
new_sboxw};
284         case (sword_ctr_reg)
285           2'h0:
286             begin
287               muxed_sboxw = block_w0_reg;
288               block_w0_we = 1'b1;
289             end
290           default:
291             begin
292               muxed_sboxw = block_w1_reg;
293               block_w1_we = 1'b1;
294             end
295           2'h2:
296             begin
297               muxed_sboxw = block_w2_reg;
298               block_w2_we = 1'b1;
299             end
300           2'h3:
301             begin
302               muxed_sboxw = block_w3_reg;
303               block_w3_we = 1'b1;
304             end
305         endcase // case (sword_mux_ctrl_reg)
306       end
307     endcase // case (update_type)
308   end // round_logic
309
310 //-----
311 // sword_ctr
312 //
313 // The subbytes word counter with reset and increase logic.
314 //-----
315 always @*
316 begin : sword_ctr
317   sword_ctr_new = 2'h0;
318   sword_ctr_we = 1'b0;
319   if (sword_ctr_rst)
320     begin
321       sword_ctr_new = 2'h0;
322       sword_ctr_we = 1'b1;
323     end
324   else if (sword_ctr_inc)
325     begin
326       sword_ctr_new = sword_ctr_reg + 1'b1;
327       sword_ctr_we = 1'b1;
328     end
329   end // sword_ctr
330
331 //-----
332 // round_ctr
333 //
334 // The round counter with reset and increase logic.
335 //-----
336 always @*
337 begin : round_ctr
338   round_ctr_new = 4'h0;
339   round_ctr_we = 1'b0;
340   if (round_ctr_rst)
341     begin
342       round_ctr_new = 4'h0;
343       round_ctr_we = 1'b1;
344     end
345   else if (round_ctr_inc)
346     begin
347       round_ctr_new = round_ctr_reg + 1'b1;
348       round_ctr_we = 1'b1;
349     end
350   end // round_ctr
351
352 //-----
353 // encipher_ctrl
354 //
355 // The FSM that controls the encipher operations.
356 //-----

```

```

303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398

```



```

-----
399 always @*
400 begin: encipher_ctrl
401   reg [3 : 0] num_rounds;
402
403   // Default assignments.
404   sword_ctr_inc = 1'b0;
405   sword_ctr_rst = 1'b0;
406   round_ctr_inc = 1'b0;
407   round_ctr_rst = 1'b0;
408   ready_new = 1'b0;
409   ready_we = 1'b0;
410   update_type = NO_UPDATE;
411   enc_ctrl_new = CTRL_IDLE;
412   enc_ctrl_we = 1'b0;
413
414   if (keylen == AES_256_BIT_KEY)
415   begin
416     num_rounds = AES256_ROUNDS;
417   end
418   else
419   begin
420     num_rounds = AES128_ROUNDS;
421   end
422
423   case(enc_ctrl_reg)
424   CTRL_IDLE:
425   begin
426     if (next)
427     begin
428       round_ctr_rst = 1'b1;
429       ready_new = 1'b0;
430       ready_we = 1'b1;
431       enc_ctrl_new = CTRL_INIT;
432       enc_ctrl_we = 1'b1;
433     end
434   end
435
436   CTRL_INIT:
437   begin
438     round_ctr_inc = 1'b1;
439     sword_ctr_rst = 1'b1;
440     update_type = INIT_UPDATE;
441     enc_ctrl_new = CTRL_SBOX;
442     enc_ctrl_we = 1'b1;
443   end
444
445   CTRL_SBOX:
446   begin
447     sword_ctr_inc = 1'b1;
448     update_type = SBOX_UPDATE;
449     if (sword_ctr_reg == 2'h3)
450     begin
451       enc_ctrl_new = CTRL_MAIN;
452       enc_ctrl_we = 1'b1;
453     end
454   end
455
456   CTRL_MAIN:
457   begin
458     sword_ctr_rst = 1'b1;
459     round_ctr_inc = 1'b1;
460     if (round_ctr_reg < num_rounds)
461     begin
462       update_type = MAIN_UPDATE;
463       enc_ctrl_new = CTRL_SBOX;
464       enc_ctrl_we = 1'b1;
465     end
466     else
467     begin
468       update_type = FINAL_UPDATE;
469       ready_new = 1'b1;
470       ready_we = 1'b1;
471       enc_ctrl_new = CTRL_IDLE;
472       enc_ctrl_we = 1'b1;
473     end
474   end
475
476   default:
477   begin
478     // Empty. Just here to make the synthesis tool
479     happy.
480     end
481   endcase // case (enc_ctrl_reg)
482   end // encipher_ctrl
483 endmodule // aes_encipher_block
484
485 //
=====

```

```

=====
486 // EOF aes_encipher_block.v
487 //
=====

```

5. secworks_aes/src/rtl/aes_inv_sbox.v (325 lines)

```

1 //
=====

```

```

==
2 //
3 // aes_inv_sbox.v
4 // -----
5 // The inverse AES S-box. Basically a 256 Byte ROM.
6 //
7 //
8 // Copyright (c) 2013 Secworks Sweden AB
9 // All rights reserved.
10 //
11 // Redistribution and use in source and binary forms, with or
12 // without modification, are permitted provided that the fol-
13 // lowing
14 // conditions are met:
15 //
16 // 1. Redistributions of source code must retain the above
17 // copyright
18 // notice, this list of conditions and the following dis-
19 // claimer.
20 //
21 // 2. Redistributions in binary form must reproduce the above
22 // copyright
23 // notice, this list of conditions and the following dis-
24 // claimer in
25 // the documentation and/or other materials provided with
26 // the
27 // distribution.
28 //
29 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
30 // TRIBUTORS
31 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
32 // BUT NOT
33 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
34 // FITNESS
35 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
36 // THE
37 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
38 // INDIRECT,
39 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
40 // (INCLUDING,
41 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
42 // VICES;
43 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
44 // HOWEVER
45 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
46 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
47 // WISE)
48 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
49 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
50 //
51 // -----
52 //
53 // `default_nettype none
54 //
55 module aes_inv_sbox(
56   input wire [31 : 0] sboxw,
57   output wire [31 : 0] new_sboxw
58 );
59
60 //-----
61 // The inverse sbox array.
62 //-----
63 wire [7 : 0] inv_sbox [0 : 255];
64
65 //-----
66 // Four parallel muxes.
67 //-----
68 assign new_sboxw[31 : 24] = inv_sbox[sboxw[31 : 24]];
69 assign new_sboxw[23 : 16] = inv_sbox[sboxw[23 : 16]];
70 assign new_sboxw[15 : 08] = inv_sbox[sboxw[15 : 08]];
71 assign new_sboxw[07 : 00] = inv_sbox[sboxw[07 : 00]];
72
73 //-----
74 // Creating the contents of the array.
75 //-----
76 assign inv_sbox[8'h00] = 8'h52;
77 assign inv_sbox[8'h01] = 8'h09;
78 assign inv_sbox[8'h02] = 8'h6a;
79 assign inv_sbox[8'h03] = 8'h5d;
80 assign inv_sbox[8'h04] = 8'h30;
81 assign inv_sbox[8'h05] = 8'h36;
82 assign inv_sbox[8'h06] = 8'ha5;
83 assign inv_sbox[8'h07] = 8'h38;
84 assign inv_sbox[8'h08] = 8'hbf;
85 assign inv_sbox[8'h09] = 8'h40;
86 assign inv_sbox[8'h0a] = 8'ha3;
87 assign inv_sbox[8'h0b] = 8'h9e;
88 assign inv_sbox[8'h0c] = 8'h81;
89 assign inv_sbox[8'h0d] = 8'hf3;
90 assign inv_sbox[8'h0e] = 8'hd7;

```

```

79 assign inv_sbox[8'h0f] = 8'hfb;
80 assign inv_sbox[8'h10] = 8'h7c;
81 assign inv_sbox[8'h11] = 8'he3;
82 assign inv_sbox[8'h12] = 8'h39;
83 assign inv_sbox[8'h13] = 8'h82;
84 assign inv_sbox[8'h14] = 8'h9b;
85 assign inv_sbox[8'h15] = 8'h2f;
86 assign inv_sbox[8'h16] = 8'hff;
87 assign inv_sbox[8'h17] = 8'h87;
88 assign inv_sbox[8'h18] = 8'h34;
89 assign inv_sbox[8'h19] = 8'h8e;
90 assign inv_sbox[8'h1a] = 8'h43;
91 assign inv_sbox[8'h1b] = 8'h44;
92 assign inv_sbox[8'h1c] = 8'hc4;
93 assign inv_sbox[8'h1d] = 8'hde;
94 assign inv_sbox[8'h1e] = 8'he9;
95 assign inv_sbox[8'h1f] = 8'hcb;
96 assign inv_sbox[8'h20] = 8'h54;
97 assign inv_sbox[8'h21] = 8'h7b;
98 assign inv_sbox[8'h22] = 8'h94;
99 assign inv_sbox[8'h23] = 8'h32;
100 assign inv_sbox[8'h24] = 8'ha6;
101 assign inv_sbox[8'h25] = 8'hc2;
102 assign inv_sbox[8'h26] = 8'h23;
103 assign inv_sbox[8'h27] = 8'h3d;
104 assign inv_sbox[8'h28] = 8'hee;
105 assign inv_sbox[8'h29] = 8'h4c;
106 assign inv_sbox[8'h2a] = 8'h95;
107 assign inv_sbox[8'h2b] = 8'h0b;
108 assign inv_sbox[8'h2c] = 8'h42;
109 assign inv_sbox[8'h2d] = 8'hfa;
110 assign inv_sbox[8'h2e] = 8'hc3;
111 assign inv_sbox[8'h2f] = 8'h4e;
112 assign inv_sbox[8'h30] = 8'h08;
113 assign inv_sbox[8'h31] = 8'h2e;
114 assign inv_sbox[8'h32] = 8'ha1;
115 assign inv_sbox[8'h33] = 8'h66;
116 assign inv_sbox[8'h34] = 8'h28;
117 assign inv_sbox[8'h35] = 8'hd9;
118 assign inv_sbox[8'h36] = 8'h24;
119 assign inv_sbox[8'h37] = 8'hb2;
120 assign inv_sbox[8'h38] = 8'h76;
121 assign inv_sbox[8'h39] = 8'h5b;
122 assign inv_sbox[8'h3a] = 8'ha2;
123 assign inv_sbox[8'h3b] = 8'h49;
124 assign inv_sbox[8'h3c] = 8'h6d;
125 assign inv_sbox[8'h3d] = 8'h8b;
126 assign inv_sbox[8'h3e] = 8'hdl;
127 assign inv_sbox[8'h3f] = 8'h25;
128 assign inv_sbox[8'h40] = 8'h72;
129 assign inv_sbox[8'h41] = 8'hf8;
130 assign inv_sbox[8'h42] = 8'hf6;
131 assign inv_sbox[8'h43] = 8'h64;
132 assign inv_sbox[8'h44] = 8'h86;
133 assign inv_sbox[8'h45] = 8'h68;
134 assign inv_sbox[8'h46] = 8'h98;
135 assign inv_sbox[8'h47] = 8'h16;
136 assign inv_sbox[8'h48] = 8'hd4;
137 assign inv_sbox[8'h49] = 8'ha4;
138 assign inv_sbox[8'h4a] = 8'h5c;
139 assign inv_sbox[8'h4b] = 8'hcc;
140 assign inv_sbox[8'h4c] = 8'h5d;
141 assign inv_sbox[8'h4d] = 8'h65;
142 assign inv_sbox[8'h4e] = 8'hb6;
143 assign inv_sbox[8'h4f] = 8'h92;
144 assign inv_sbox[8'h50] = 8'h6c;
145 assign inv_sbox[8'h51] = 8'h70;
146 assign inv_sbox[8'h52] = 8'h48;
147 assign inv_sbox[8'h53] = 8'h50;
148 assign inv_sbox[8'h54] = 8'hfd;
149 assign inv_sbox[8'h55] = 8'hed;
150 assign inv_sbox[8'h56] = 8'hb9;
151 assign inv_sbox[8'h57] = 8'hda;
152 assign inv_sbox[8'h58] = 8'h5e;
153 assign inv_sbox[8'h59] = 8'h15;
154 assign inv_sbox[8'h5a] = 8'h46;
155 assign inv_sbox[8'h5b] = 8'h57;
156 assign inv_sbox[8'h5c] = 8'ha7;
157 assign inv_sbox[8'h5d] = 8'h8d;
158 assign inv_sbox[8'h5e] = 8'h9d;
159 assign inv_sbox[8'h5f] = 8'h84;
160 assign inv_sbox[8'h60] = 8'h90;
161 assign inv_sbox[8'h61] = 8'hd8;
162 assign inv_sbox[8'h62] = 8'hab;
163 assign inv_sbox[8'h63] = 8'h00;
164 assign inv_sbox[8'h64] = 8'h8c;
165 assign inv_sbox[8'h65] = 8'hbc;
166 assign inv_sbox[8'h66] = 8'hd3;
167 assign inv_sbox[8'h67] = 8'h0a;
168 assign inv_sbox[8'h68] = 8'hf7;
169 assign inv_sbox[8'h69] = 8'he4;
170 assign inv_sbox[8'h6a] = 8'h58;
171 assign inv_sbox[8'h6b] = 8'h05;
172 assign inv_sbox[8'h6c] = 8'hb8;
173 assign inv_sbox[8'h6d] = 8'hb3;
174 assign inv_sbox[8'h6e] = 8'h45;
175 assign inv_sbox[8'h6f] = 8'h06;
176 assign inv_sbox[8'h70] = 8'hd0;
177 assign inv_sbox[8'h71] = 8'h2c;
178 assign inv_sbox[8'h72] = 8'h1e;
179 assign inv_sbox[8'h73] = 8'h8f;

```

```

180 assign inv_sbox[8'h74] = 8'hca;
181 assign inv_sbox[8'h75] = 8'h3f;
182 assign inv_sbox[8'h76] = 8'h0f;
183 assign inv_sbox[8'h77] = 8'h02;
184 assign inv_sbox[8'h78] = 8'hc1;
185 assign inv_sbox[8'h79] = 8'haf;
186 assign inv_sbox[8'h7a] = 8'hbd;
187 assign inv_sbox[8'h7b] = 8'h03;
188 assign inv_sbox[8'h7c] = 8'h01;
189 assign inv_sbox[8'h7d] = 8'h13;
190 assign inv_sbox[8'h7e] = 8'h8a;
191 assign inv_sbox[8'h7f] = 8'h6b;
192 assign inv_sbox[8'h80] = 8'h3a;
193 assign inv_sbox[8'h81] = 8'h91;
194 assign inv_sbox[8'h82] = 8'h11;
195 assign inv_sbox[8'h83] = 8'h41;
196 assign inv_sbox[8'h84] = 8'h4f;
197 assign inv_sbox[8'h85] = 8'h67;
198 assign inv_sbox[8'h86] = 8'hdc;
199 assign inv_sbox[8'h87] = 8'hea;
200 assign inv_sbox[8'h88] = 8'h97;
201 assign inv_sbox[8'h89] = 8'hf2;
202 assign inv_sbox[8'h8a] = 8'hcf;
203 assign inv_sbox[8'h8b] = 8'hce;
204 assign inv_sbox[8'h8c] = 8'hf0;
205 assign inv_sbox[8'h8d] = 8'hb4;
206 assign inv_sbox[8'h8e] = 8'he6;
207 assign inv_sbox[8'h8f] = 8'h73;
208 assign inv_sbox[8'h90] = 8'h96;
209 assign inv_sbox[8'h91] = 8'hac;
210 assign inv_sbox[8'h92] = 8'h74;
211 assign inv_sbox[8'h93] = 8'h22;
212 assign inv_sbox[8'h94] = 8'he7;
213 assign inv_sbox[8'h95] = 8'had;
214 assign inv_sbox[8'h96] = 8'h35;
215 assign inv_sbox[8'h97] = 8'h85;
216 assign inv_sbox[8'h98] = 8'he2;
217 assign inv_sbox[8'h99] = 8'hf9;
218 assign inv_sbox[8'h9a] = 8'h37;
219 assign inv_sbox[8'h9b] = 8'he8;
220 assign inv_sbox[8'h9c] = 8'h1c;
221 assign inv_sbox[8'h9d] = 8'h75;
222 assign inv_sbox[8'h9e] = 8'hdf;
223 assign inv_sbox[8'h9f] = 8'h6e;
224 assign inv_sbox[8'ha0] = 8'h47;
225 assign inv_sbox[8'ha1] = 8'hf1;
226 assign inv_sbox[8'ha2] = 8'h1a;
227 assign inv_sbox[8'ha3] = 8'h71;
228 assign inv_sbox[8'ha4] = 8'h1d;
229 assign inv_sbox[8'ha5] = 8'h29;
230 assign inv_sbox[8'ha6] = 8'hc5;
231 assign inv_sbox[8'ha7] = 8'h89;
232 assign inv_sbox[8'ha8] = 8'h6f;
233 assign inv_sbox[8'ha9] = 8'hb7;
234 assign inv_sbox[8'haa] = 8'h62;
235 assign inv_sbox[8'hab] = 8'h0e;
236 assign inv_sbox[8'hac] = 8'haa;
237 assign inv_sbox[8'had] = 8'h18;
238 assign inv_sbox[8'hae] = 8'hbe;
239 assign inv_sbox[8'haf] = 8'h1b;
240 assign inv_sbox[8'hb0] = 8'hfc;
241 assign inv_sbox[8'hb1] = 8'h56;
242 assign inv_sbox[8'hb2] = 8'h3e;
243 assign inv_sbox[8'hb3] = 8'h4b;
244 assign inv_sbox[8'hb4] = 8'hc6;
245 assign inv_sbox[8'hb5] = 8'hd2;
246 assign inv_sbox[8'hb6] = 8'h79;
247 assign inv_sbox[8'hb7] = 8'h20;
248 assign inv_sbox[8'hb8] = 8'h9a;
249 assign inv_sbox[8'hb9] = 8'hdb;
250 assign inv_sbox[8'hba] = 8'hc0;
251 assign inv_sbox[8'hbb] = 8'hfe;
252 assign inv_sbox[8'hbc] = 8'h78;
253 assign inv_sbox[8'hbd] = 8'hcd;
254 assign inv_sbox[8'hbe] = 8'h5a;
255 assign inv_sbox[8'hbf] = 8'hf4;
256 assign inv_sbox[8'hc0] = 8'h1f;
257 assign inv_sbox[8'hc1] = 8'hdd;
258 assign inv_sbox[8'hc2] = 8'ha8;
259 assign inv_sbox[8'hc3] = 8'h33;
260 assign inv_sbox[8'hc4] = 8'h88;
261 assign inv_sbox[8'hc5] = 8'h07;
262 assign inv_sbox[8'hc6] = 8'hc7;
263 assign inv_sbox[8'hc7] = 8'h31;
264 assign inv_sbox[8'hc8] = 8'hb1;
265 assign inv_sbox[8'hc9] = 8'h12;
266 assign inv_sbox[8'hca] = 8'h10;
267 assign inv_sbox[8'hcb] = 8'h59;
268 assign inv_sbox[8'hcc] = 8'h27;
269 assign inv_sbox[8'hcd] = 8'h80;
270 assign inv_sbox[8'hce] = 8'hec;
271 assign inv_sbox[8'hcf] = 8'h5f;
272 assign inv_sbox[8'hd0] = 8'h09;
273 assign inv_sbox[8'hd1] = 8'h51;
274 assign inv_sbox[8'hd2] = 8'h7f;
275 assign inv_sbox[8'hd3] = 8'ha9;
276 assign inv_sbox[8'hd4] = 8'h19;
277 assign inv_sbox[8'hd5] = 8'hb5;
278 assign inv_sbox[8'hd6] = 8'h4a;
279 assign inv_sbox[8'hd7] = 8'h0d;
280 assign inv_sbox[8'hd8] = 8'h2d;

```

```

281 assign inv_sbox[8'h9d] = 8'h5;
282 assign inv_sbox[8'hda] = 8'h7a;
283 assign inv_sbox[8'hdb] = 8'h9f;
284 assign inv_sbox[8'hdc] = 8'h93;
285 assign inv_sbox[8'hdd] = 8'h9c;
286 assign inv_sbox[8'hde] = 8'h9c;
287 assign inv_sbox[8'hdf] = 8'h9c;
288 assign inv_sbox[8'he0] = 8'ha0;
289 assign inv_sbox[8'he1] = 8'he0;
290 assign inv_sbox[8'he2] = 8'h3b;
291 assign inv_sbox[8'he3] = 8'h4d;
292 assign inv_sbox[8'he4] = 8'hae;
293 assign inv_sbox[8'he5] = 8'h2a;
294 assign inv_sbox[8'he6] = 8'hf5;
295 assign inv_sbox[8'he7] = 8'hb0;
296 assign inv_sbox[8'he8] = 8'h8b;
297 assign inv_sbox[8'he9] = 8'h83;
298 assign inv_sbox[8'hea] = 8'hbb;
299 assign inv_sbox[8'heb] = 8'h3c;
300 assign inv_sbox[8'hec] = 8'h83;
301 assign inv_sbox[8'hed] = 8'h53;
302 assign inv_sbox[8'hee] = 8'h99;
303 assign inv_sbox[8'hdf] = 8'h61;
304 assign inv_sbox[8'hf0] = 8'h17;
305 assign inv_sbox[8'hf1] = 8'h2b;
306 assign inv_sbox[8'hf2] = 8'h04;
307 assign inv_sbox[8'hf3] = 8'h7e;
308 assign inv_sbox[8'hf4] = 8'hba;
309 assign inv_sbox[8'hf5] = 8'h77;
310 assign inv_sbox[8'hf6] = 8'h06;
311 assign inv_sbox[8'hf7] = 8'h26;
312 assign inv_sbox[8'hf8] = 8'h61;
313 assign inv_sbox[8'hf9] = 8'h69;
314 assign inv_sbox[8'hfa] = 8'h14;
315 assign inv_sbox[8'hfb] = 8'h63;
316 assign inv_sbox[8'hfc] = 8'h55;
317 assign inv_sbox[8'hfd] = 8'h21;
318 assign inv_sbox[8'hfe] = 8'h0c;
319 assign inv_sbox[8'hff] = 8'h7d;
320
321 endmodule // aes_inv_sbox
322
323 //
=====
324 // EOF aes_inv_sbox.v
325 //
=====

```

6. secworks_aes/src/rtl/aes_key_mem.v (434 lines)

```

1 //
=====
2 //
3 // aes_key_mem.v
4 // -----
5 // The AES key memory including round key generator.
6 //
7 //
8 // Author: Joachim Strombergson
9 // Copyright (c) 2013 Secworks Sweden AB
10 // All rights reserved.
11 //
12 // Redistribution and use in source and binary forms, with or
13 // without modification, are permitted provided that the fol-
14 // lowing conditions are met:
15 //
16 // 1. Redistributions of source code must retain the above
17 // copyright notice, this list of conditions and the following dis-
18 // claimer.
19 //
20 // 2. Redistributions in binary form must reproduce the above
21 // copyright notice, this list of conditions and the following dis-
22 // claimer in the documentation and/or other materials provided with
23 // the distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
26 // TRIBUTORS
27 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
28 // BUT NOT
29 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
30 // FITNESS
31 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
32 // THE
33 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
34 // INDIRECT,
35 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
36 // (INCLUDING,
37 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
38 // VICES;
39 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
40 // HOWEVER
41 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,

```

```

33 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
34 // WISE)
35 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
=====
38
39 `default_nettype none
40
41 module aes_key_mem(
42     input wire          clk,
43     input wire          reset_n,
44
45     input wire [255 : 0] key,
46     input wire          keylen,
47     input wire          init,
48
49     input wire [3 : 0] round,
50     output wire [127 : 0] round_key,
51     output wire          ready,
52
53     output wire [31 : 0] sboxw,
54     input wire [31 : 0] new_sboxw
55 );
56
57 //-----
58 // Parameters.
59 //-----
60
61 localparam AES_128_BIT_KEY = 1'h0;
62 localparam AES_256_BIT_KEY = 1'h1;
63
64 localparam AES_128_NUM_ROUNDS = 10;
65 localparam AES_256_NUM_ROUNDS = 14;
66
67 localparam CTRL_IDLE = 3'h0;
68 localparam CTRL_INIT = 3'h1;
69 localparam CTRL_GENERATE = 3'h2;
70 localparam CTRL_DONE = 3'h3;
71
72 //-----
73 // Registers.
74 //-----
75
76 reg [127 : 0] key_mem [0 : 14];
77 reg [127 : 0] key_mem_new;
78 reg          key_mem_we;
79
80 reg [127 : 0] prev_key0_reg;
81 reg [127 : 0] prev_key0_new;
82 reg          prev_key0_we;
83
84 reg [127 : 0] prev_key1_reg;
85 reg [127 : 0] prev_key1_new;
86 reg          prev_key1_we;
87
88 reg [3 : 0] round_ctr_reg;
89 reg [3 : 0] round_ctr_new;
90 reg          round_ctr_rst;
91 reg          round_ctr_inc;
92 reg          round_ctr_we;
93
94 reg [2 : 0] key_mem_ctrl_reg;
95 reg [2 : 0] key_mem_ctrl_new;
96 reg          key_mem_ctrl_we;
97
98 reg          ready_reg;
99 reg          ready_new;
100 reg          ready_we;
101
102 reg [7 : 0] rcon_reg;
103 reg [7 : 0] rcon_new;
104 reg          rcon_we;
105 reg          rcon_set;
106 reg          rcon_next;
107
108 //-----
109 // Wires.
110 //-----
111
112 reg [31 : 0] tmp_sboxw;
113 reg          round_key_update;
114 reg [127 : 0] tmp_round_key;
115
116 //-----
117 // Concurrent assignments for ports.
118 //-----
119
120 assign round_key = tmp_round_key;
121 assign ready = ready_reg;
122

```

```

123 assign sboxw      = tmp_sboxw;
124
125
126 //-----
127 // reg_update
128 //
129 // Update functionality for all registers in the core.
130 // All registers are positive edge triggered with asynchron-
ous
131 // active low reset. All registers have write enable.
132 //-----
133 always @ (posedge clk or negedge reset_n)
134 begin: reg_update
135     integer i;
136
137     if (!reset_n)
138     begin
139         for (i = 0 ; i <= AES_256_NUM_ROUNDS ; i = i + 1)
140             key_mem [i] <= 128'h0;
141
142             ready_reg      <= 1'b1;
143             rcon_reg       <= 8'h0;
144             round_ctr_reg  <= 4'h0;
145             prev_key0_reg  <= 128'h0;
146             prev_key1_reg  <= 128'h0;
147             key_mem_ctrl_reg <= CTRL_IDLE;
148         end
149     else
150     begin
151         if (ready_we)
152             ready_reg <= ready_new;
153
154         if (rcon_we)
155             rcon_reg <= rcon_new;
156
157         if (round_ctr_we)
158             round_ctr_reg <= round_ctr_new;
159
160         if (key_mem_we)
161             key_mem[round_ctr_reg] <= key_mem_new;
162
163         if (prev_key0_we)
164             prev_key0_reg <= prev_key0_new;
165
166         if (prev_key1_we)
167             prev_key1_reg <= prev_key1_new;
168
169         if (key_mem_ctrl_we)
170             key_mem_ctrl_reg <= key_mem_ctrl_new;
171         end
172     end // reg_update
173
174
175 //-----
176 // key_mem_read
177 //
178 // Combinational read port for the key memory.
179 //-----
180 always @*
181 begin : key_mem_read
182     tmp_round_key = key_mem[round];
183 end // key_mem_read
184
185
186 //-----
187 // round_key_gen
188 //
189 // The round key generator logic for AES-128 and AES-256.
190 //-----
191 always @*
192 begin: round_key_gen
193     reg [31 : 0] w0, w1, w2, w3, w4, w5, w6, w7;
194     reg [31 : 0] k0, k1, k2, k3;
195     reg [31 : 0] rconw, rotstw, tw, trw;
196
197     // Default assignments.
198     key_mem_new = 128'h0;
199     key_mem_we = 1'b0;
200     prev_key0_new = 128'h0;
201     prev_key0_we = 1'b0;
202     prev_key1_new = 128'h0;
203     prev_key1_we = 1'b0;
204
205     k0 = 32'h0;
206     k1 = 32'h0;
207     k2 = 32'h0;
208     k3 = 32'h0;
209
210     rcon_set = 1'b1;
211     rcon_next = 1'b0;
212
213     // Extract words and calculate intermediate values.
214     // Perform rotation of sbox word etc.
215     w0 = prev_key0_reg[127 : 096];
216     w1 = prev_key0_reg[095 : 064];

```

```

217     w2 = prev_key0_reg[063 : 032];
218     w3 = prev_key0_reg[031 : 000];
219
220     w4 = prev_key1_reg[127 : 096];
221     w5 = prev_key1_reg[095 : 064];
222     w6 = prev_key1_reg[063 : 032];
223     w7 = prev_key1_reg[031 : 000];
224
225     rconw = {rcon_reg, 24'h0};
226     tmp_sboxw = w7;
227     rotstw = {new_sboxw[23 : 00], new_sboxw[31 : 24]};
228     trw = rotstw ^ rconw;
229     tw = new_sboxw;
230
231     // Generate the specific round keys.
232     if (round_key_update)
233     begin
234         rcon_set = 1'b0;
235         key_mem_we = 1'b1;
236         case (keylen)
237             AES_128_BIT_KEY:
238             begin
239                 if (round_ctr_reg == 0)
240                 begin
241                     key_mem_new = key[255 : 128];
242                     prev_key1_new = key[255 : 128];
243                     prev_key1_we = 1'b1;
244                     rcon_next = 1'b1;
245                 end
246             else
247             begin
248                 k0 = w4 ^ trw;
249                 k1 = w5 ^ w4 ^ trw;
250                 k2 = w6 ^ w5 ^ w4 ^ trw;
251                 k3 = w7 ^ w6 ^ w5 ^ w4 ^ trw;
252
253                 key_mem_new = {k0, k1, k2, k3};
254                 prev_key1_new = {k0, k1, k2, k3};
255                 prev_key1_we = 1'b1;
256                 rcon_next = 1'b1;
257             end
258         end
259
260         AES_256_BIT_KEY:
261         begin
262             if (round_ctr_reg == 0)
263             begin
264                 key_mem_new = key[255 : 128];
265                 prev_key0_new = key[255 : 128];
266                 prev_key0_we = 1'b1;
267             end
268             else if (round_ctr_reg == 1)
269             begin
270                 key_mem_new = key[127 : 0];
271                 prev_key1_new = key[127 : 0];
272                 prev_key1_we = 1'b1;
273                 rcon_next = 1'b1;
274             end
275         else
276         begin
277             if (round_ctr_reg[0] == 0)
278             begin
279                 k0 = w0 ^ trw;
280                 k1 = w1 ^ w0 ^ trw;
281                 k2 = w2 ^ w1 ^ w0 ^ trw;
282                 k3 = w3 ^ w2 ^ w1 ^ w0 ^ trw;
283             end
284             else
285             begin
286                 k0 = w0 ^ tw;
287                 k1 = w1 ^ w0 ^ tw;
288                 k2 = w2 ^ w1 ^ w0 ^ tw;
289                 k3 = w3 ^ w2 ^ w1 ^ w0 ^ tw;
290                 rcon_next = 1'b1;
291             end
292         end
293
294         // Store the generated round keys.
295         key_mem_new = {k0, k1, k2, k3};
296         prev_key1_new = {k0, k1, k2, k3};
297         prev_key1_we = 1'b1;
298         prev_key0_new = prev_key1_reg;
299         prev_key0_we = 1'b1;
300     end
301
302     default:
303     begin
304     end
305     endcase // case (keylen)
306 end
307
308 // round_key_gen
309
310 //-----
311 // rcon_logic
312 //
313 // Calculates the rcon value for the different key expansion
314 // iterations.
315 //-----

```

```

316 always @*
317 begin : rcon_logic
318   reg [7 : 0] tmp_rcon;
319   rcon_new = 8'h00;
320   rcon_we = 1'b0;
321
322   tmp_rcon = {rcon_reg[6 : 0], 1'b0} ^ (8'h1b &
323   {8{rcon_reg[7]}]);
324
325   if (rcon_set)
326   begin
327     rcon_new = 8'h8d;
328     rcon_we = 1'b1;
329   end
330
331   if (rcon_next)
332   begin
333     rcon_new = tmp_rcon[7 : 0];
334     rcon_we = 1'b1;
335   end
336 end
337
338 //-----
339 // round_ctr
340 //
341 // The round counter logic with increase and reset.
342 //-----
343
344 always @*
345 begin : round_ctr
346   round_ctr_new = 4'h0;
347   round_ctr_we = 1'b0;
348
349   if (round_ctr_rst)
350   begin
351     round_ctr_new = 4'h0;
352     round_ctr_we = 1'b1;
353   end
354
355   else if (round_ctr_inc)
356   begin
357     round_ctr_new = round_ctr_reg + 1'b1;
358     round_ctr_we = 1'b1;
359   end
360 end
361
362 //-----
363 // key_mem_ctrl
364 //
365 //
366 // The FSM that controls the round key generation.
367 //-----
368
369 always @*
370 begin: key_mem_ctrl
371   reg [3 : 0] num_rounds;
372
373   // Default assignments.
374   ready_new = 1'b0;
375   ready_we = 1'b0;
376   round_key_update = 1'b0;
377   round_ctr_rst = 1'b0;
378   round_ctr_inc = 1'b0;
379   key_mem_ctrl_new = CTRL_IDLE;
380   key_mem_ctrl_we = 1'b0;
381
382   if (keylen == AES_128_BIT_KEY)
383     num_rounds = AES_128_NUM_ROUNDS;
384   else
385     num_rounds = AES_256_NUM_ROUNDS;
386
387   case(key_mem_ctrl_reg)
388     CTRL_IDLE:
389     begin
390       if (init)
391       begin
392         ready_new = 1'b0;
393         ready_we = 1'b1;
394         key_mem_ctrl_new = CTRL_INIT;
395         key_mem_ctrl_we = 1'b1;
396       end
397     end
398
399     CTRL_INIT:
400     begin
401       round_ctr_rst = 1'b1;
402       key_mem_ctrl_new = CTRL_GENERATE;
403       key_mem_ctrl_we = 1'b1;
404     end
405
406     CTRL_GENERATE:
407     begin
408       round_ctr_inc = 1'b1;
409       round_key_update = 1'b1;
410       if (round_ctr_reg == num_rounds)
411       begin
412         key_mem_ctrl_new = CTRL_DONE;

```

```

412         key_mem_ctrl_we = 1'b1;
413       end
414     end
415
416     CTRL_DONE:
417     begin
418       ready_new = 1'b1;
419       ready_we = 1'b1;
420       key_mem_ctrl_new = CTRL_IDLE;
421       key_mem_ctrl_we = 1'b1;
422     end
423
424     default:
425     begin
426     end
427   endcase // case (key_mem_ctrl_reg)
428
429   end // key_mem_ctrl
430 endmodule // aes_key_mem
431
432 //
433 // EOF aes_key_mem.v
434 //
=====
7. secworks_aes/src/rtl/aes_sbox.v (327 lines)
=====
1 //
=====
2 //
3 // aes_sbox.v
4 // -----
5 // The AES S-box. Basically a 256 Byte ROM. This implementa-
6 // tion contains four parallel S-boxes to handle a 32 bit word.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2014, Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the fol-
15 // lowing conditions are met:
16 //
17 // 1. Redistributions of source code must retain the above
18 // copyright notice, this list of conditions and the following dis-
19 // claimer.
20 // 2. Redistributions in binary form must reproduce the above
21 // copyright notice, this list of conditions and the following dis-
22 // claimer in the documentation and/or other materials provided with
23 // the distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
26 // TRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
27 // BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
28 // FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
29 // THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
30 // INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
31 // (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
32 // VICES; HOWEVER LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
33 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
35 // WISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====
39 `default_nettype none
40
41 module aes_sbox(
42   input wire [31 : 0] sboxw,
43   output wire [31 : 0] new_sboxw
44 );
45
46
47
48 //-----
49 // The sbox array.

```

```

50  //-----
51  wire [7 : 0] sbox [0 : 255];
52
53
54  //-----
55  // Four parallel muxes.
56  //-----
57  assign new_sboxw[31 : 24] = sbox[sboxw[31 : 24]];
58  assign new_sboxw[23 : 16] = sbox[sboxw[23 : 16]];
59  assign new_sboxw[15 : 08] = sbox[sboxw[15 : 08]];
60  assign new_sboxw[07 : 00] = sbox[sboxw[07 : 00]];
61
62
63  //-----
64  // Creating the sbox array contents.
65  //-----
66  assign sbox[8'h00] = 8'h63;
67  assign sbox[8'h01] = 8'h7c;
68  assign sbox[8'h02] = 8'h77;
69  assign sbox[8'h03] = 8'h7b;
70  assign sbox[8'h04] = 8'hf2;
71  assign sbox[8'h05] = 8'h6b;
72  assign sbox[8'h06] = 8'h6f;
73  assign sbox[8'h07] = 8'hc5;
74  assign sbox[8'h08] = 8'h30;
75  assign sbox[8'h09] = 8'h01;
76  assign sbox[8'h0a] = 8'h67;
77  assign sbox[8'h0b] = 8'h2b;
78  assign sbox[8'h0c] = 8'hfe;
79  assign sbox[8'h0d] = 8'hd7;
80  assign sbox[8'h0e] = 8'hab;
81  assign sbox[8'h0f] = 8'h76;
82  assign sbox[8'h10] = 8'hca;
83  assign sbox[8'h11] = 8'h82;
84  assign sbox[8'h12] = 8'hc9;
85  assign sbox[8'h13] = 8'h7d;
86  assign sbox[8'h14] = 8'hfa;
87  assign sbox[8'h15] = 8'h59;
88  assign sbox[8'h16] = 8'h47;
89  assign sbox[8'h17] = 8'hf0;
90  assign sbox[8'h18] = 8'had;
91  assign sbox[8'h19] = 8'hd4;
92  assign sbox[8'h1a] = 8'ha2;
93  assign sbox[8'h1b] = 8'haf;
94  assign sbox[8'h1c] = 8'h9c;
95  assign sbox[8'h1d] = 8'ha4;
96  assign sbox[8'h1e] = 8'h72;
97  assign sbox[8'h1f] = 8'hc0;
98  assign sbox[8'h20] = 8'hb7;
99  assign sbox[8'h21] = 8'hfd;
100 assign sbox[8'h22] = 8'h93;
101 assign sbox[8'h23] = 8'h26;
102 assign sbox[8'h24] = 8'h36;
103 assign sbox[8'h25] = 8'h3f;
104 assign sbox[8'h26] = 8'hf7;
105 assign sbox[8'h27] = 8'hcc;
106 assign sbox[8'h28] = 8'h34;
107 assign sbox[8'h29] = 8'ha5;
108 assign sbox[8'h2a] = 8'he5;
109 assign sbox[8'h2b] = 8'hf1;
110 assign sbox[8'h2c] = 8'h71;
111 assign sbox[8'h2d] = 8'hd8;
112 assign sbox[8'h2e] = 8'h31;
113 assign sbox[8'h2f] = 8'h15;
114 assign sbox[8'h30] = 8'h04;
115 assign sbox[8'h31] = 8'hc7;
116 assign sbox[8'h32] = 8'h23;
117 assign sbox[8'h33] = 8'hc3;
118 assign sbox[8'h34] = 8'h18;
119 assign sbox[8'h35] = 8'h96;
120 assign sbox[8'h36] = 8'h05;
121 assign sbox[8'h37] = 8'h9a;
122 assign sbox[8'h38] = 8'h07;
123 assign sbox[8'h39] = 8'h12;
124 assign sbox[8'h3a] = 8'h80;
125 assign sbox[8'h3b] = 8'he2;
126 assign sbox[8'h3c] = 8'heb;
127 assign sbox[8'h3d] = 8'h27;
128 assign sbox[8'h3e] = 8'hb2;
129 assign sbox[8'h3f] = 8'h75;
130 assign sbox[8'h40] = 8'h09;
131 assign sbox[8'h41] = 8'h83;
132 assign sbox[8'h42] = 8'h2c;
133 assign sbox[8'h43] = 8'h1a;
134 assign sbox[8'h44] = 8'h1b;
135 assign sbox[8'h45] = 8'h6e;
136 assign sbox[8'h46] = 8'h5a;
137 assign sbox[8'h47] = 8'ha0;
138 assign sbox[8'h48] = 8'h52;
139 assign sbox[8'h49] = 8'h3b;
140 assign sbox[8'h4a] = 8'hd6;
141 assign sbox[8'h4b] = 8'hb3;
142 assign sbox[8'h4c] = 8'h29;
143 assign sbox[8'h4d] = 8'he3;
144 assign sbox[8'h4e] = 8'h2f;
145 assign sbox[8'h4f] = 8'h84;

146 assign sbox[8'h50] = 8'h53;
147 assign sbox[8'h51] = 8'hd1;
148 assign sbox[8'h52] = 8'h00;
149 assign sbox[8'h53] = 8'hed;
150 assign sbox[8'h54] = 8'h20;
151 assign sbox[8'h55] = 8'hfc;
152 assign sbox[8'h56] = 8'hb1;
153 assign sbox[8'h57] = 8'h5b;
154 assign sbox[8'h58] = 8'h6a;
155 assign sbox[8'h59] = 8'hcb;
156 assign sbox[8'h5a] = 8'hbe;
157 assign sbox[8'h5b] = 8'h39;
158 assign sbox[8'h5c] = 8'h4a;
159 assign sbox[8'h5d] = 8'h4c;
160 assign sbox[8'h5e] = 8'h58;
161 assign sbox[8'h5f] = 8'hcf;
162 assign sbox[8'h60] = 8'hd0;
163 assign sbox[8'h61] = 8'hcf;
164 assign sbox[8'h62] = 8'haa;
165 assign sbox[8'h63] = 8'hfb;
166 assign sbox[8'h64] = 8'h43;
167 assign sbox[8'h65] = 8'h4d;
168 assign sbox[8'h66] = 8'h33;
169 assign sbox[8'h67] = 8'h85;
170 assign sbox[8'h68] = 8'h45;
171 assign sbox[8'h69] = 8'hf9;
172 assign sbox[8'h6a] = 8'h02;
173 assign sbox[8'h6b] = 8'h7f;
174 assign sbox[8'h6c] = 8'h50;
175 assign sbox[8'h6d] = 8'h3c;
176 assign sbox[8'h6e] = 8'h9f;
177 assign sbox[8'h6f] = 8'ha8;
178 assign sbox[8'h70] = 8'h51;
179 assign sbox[8'h71] = 8'ha3;
180 assign sbox[8'h72] = 8'h40;
181 assign sbox[8'h73] = 8'h8f;
182 assign sbox[8'h74] = 8'h92;
183 assign sbox[8'h75] = 8'h9d;
184 assign sbox[8'h76] = 8'h38;
185 assign sbox[8'h77] = 8'hf5;
186 assign sbox[8'h78] = 8'hbc;
187 assign sbox[8'h79] = 8'hb6;
188 assign sbox[8'h7a] = 8'hda;
189 assign sbox[8'h7b] = 8'h21;
190 assign sbox[8'h7c] = 8'h10;
191 assign sbox[8'h7d] = 8'hff;
192 assign sbox[8'h7e] = 8'hf3;
193 assign sbox[8'h7f] = 8'hd2;
194 assign sbox[8'h80] = 8'hcd;
195 assign sbox[8'h81] = 8'h0c;
196 assign sbox[8'h82] = 8'h13;
197 assign sbox[8'h83] = 8'hec;
198 assign sbox[8'h84] = 8'h5f;
199 assign sbox[8'h85] = 8'h97;
200 assign sbox[8'h86] = 8'h44;
201 assign sbox[8'h87] = 8'h17;
202 assign sbox[8'h88] = 8'hc4;
203 assign sbox[8'h89] = 8'ha7;
204 assign sbox[8'h8a] = 8'h7e;
205 assign sbox[8'h8b] = 8'h3d;
206 assign sbox[8'h8c] = 8'h64;
207 assign sbox[8'h8d] = 8'h5d;
208 assign sbox[8'h8e] = 8'h19;
209 assign sbox[8'h8f] = 8'h73;
210 assign sbox[8'h90] = 8'h60;
211 assign sbox[8'h91] = 8'h81;
212 assign sbox[8'h92] = 8'h4f;
213 assign sbox[8'h93] = 8'hdc;
214 assign sbox[8'h94] = 8'h22;
215 assign sbox[8'h95] = 8'h2a;
216 assign sbox[8'h96] = 8'h90;
217 assign sbox[8'h97] = 8'h88;
218 assign sbox[8'h98] = 8'h46;
219 assign sbox[8'h99] = 8'hee;
220 assign sbox[8'h9a] = 8'hb8;
221 assign sbox[8'h9b] = 8'h14;
222 assign sbox[8'h9c] = 8'hde;
223 assign sbox[8'h9d] = 8'h5e;
224 assign sbox[8'h9e] = 8'h0b;
225 assign sbox[8'h9f] = 8'hdb;
226 assign sbox[8'ha0] = 8'he0;
227 assign sbox[8'ha1] = 8'h32;
228 assign sbox[8'ha2] = 8'h3a;
229 assign sbox[8'ha3] = 8'h0a;
230 assign sbox[8'ha4] = 8'h49;
231 assign sbox[8'ha5] = 8'h06;
232 assign sbox[8'ha6] = 8'h24;
233 assign sbox[8'ha7] = 8'h5c;
234 assign sbox[8'ha8] = 8'hc2;
235 assign sbox[8'ha9] = 8'hd3;
236 assign sbox[8'haa] = 8'hac;
237 assign sbox[8'hab] = 8'h62;
238 assign sbox[8'hac] = 8'h91;
239 assign sbox[8'had] = 8'h95;
240 assign sbox[8'hae] = 8'he4;
241 assign sbox[8'haf] = 8'h79;
242 assign sbox[8'hb0] = 8'he7;
243 assign sbox[8'hb1] = 8'hc8;
244 assign sbox[8'hb2] = 8'h37;
245 assign sbox[8'hb3] = 8'h6d;
246 assign sbox[8'hb4] = 8'h8d;

```



```

247 assign sbox[8'hb5] = 8'h5;
248 assign sbox[8'hb6] = 8'h4e;
249 assign sbox[8'hb7] = 8'ha9;
250 assign sbox[8'hb8] = 8'h6c;
251 assign sbox[8'hb9] = 8'h56;
252 assign sbox[8'hba] = 8'hf4;
253 assign sbox[8'hbb] = 8'hea;
254 assign sbox[8'hbc] = 8'h65;
255 assign sbox[8'hbd] = 8'h7a;
256 assign sbox[8'hbe] = 8'hae;
257 assign sbox[8'hbf] = 8'h08;
258 assign sbox[8'hc0] = 8'hba;
259 assign sbox[8'hc1] = 8'h78;
260 assign sbox[8'hc2] = 8'h25;
261 assign sbox[8'hc3] = 8'h2e;
262 assign sbox[8'hc4] = 8'h1c;
263 assign sbox[8'hc5] = 8'ha6;
264 assign sbox[8'hc6] = 8'hb4;
265 assign sbox[8'hc7] = 8'hc6;
266 assign sbox[8'hc8] = 8'he8;
267 assign sbox[8'hc9] = 8'hdd;
268 assign sbox[8'hca] = 8'h74;
269 assign sbox[8'hcb] = 8'h1f;
270 assign sbox[8'hcc] = 8'h4b;
271 assign sbox[8'hcd] = 8'hbd;
272 assign sbox[8'hce] = 8'h8b;
273 assign sbox[8'hcf] = 8'h8a;
274 assign sbox[8'hd0] = 8'h70;
275 assign sbox[8'hd1] = 8'h3e;
276 assign sbox[8'hd2] = 8'hb5;
277 assign sbox[8'hd3] = 8'h66;
278 assign sbox[8'hd4] = 8'h48;
279 assign sbox[8'hd5] = 8'h03;
280 assign sbox[8'hd6] = 8'hf6;
281 assign sbox[8'hd7] = 8'h0e;
282 assign sbox[8'hd8] = 8'h61;
283 assign sbox[8'hd9] = 8'h35;
284 assign sbox[8'hda] = 8'h57;
285 assign sbox[8'hdb] = 8'hb9;
286 assign sbox[8'hdc] = 8'h86;
287 assign sbox[8'hdd] = 8'hc1;
288 assign sbox[8'hde] = 8'h1d;
289 assign sbox[8'hdf] = 8'h9e;

```

```

290 assign sbox[8'he0] = 8'he1;
291 assign sbox[8'he1] = 8'hf8;
292 assign sbox[8'he2] = 8'h98;
293 assign sbox[8'he3] = 8'h11;
294 assign sbox[8'he4] = 8'h69;
295 assign sbox[8'he5] = 8'hd9;
296 assign sbox[8'he6] = 8'h8e;
297 assign sbox[8'he7] = 8'h94;
298 assign sbox[8'he8] = 8'h9b;
299 assign sbox[8'he9] = 8'h1e;
300 assign sbox[8'hea] = 8'h87;
301 assign sbox[8'heb] = 8'he9;
302 assign sbox[8'hec] = 8'hce;
303 assign sbox[8'hed] = 8'h55;
304 assign sbox[8'hee] = 8'h28;
305 assign sbox[8'hef] = 8'hdf;
306 assign sbox[8'hf0] = 8'h8c;
307 assign sbox[8'hf1] = 8'ha1;
308 assign sbox[8'hf2] = 8'h89;
309 assign sbox[8'hf3] = 8'h0d;
310 assign sbox[8'hf4] = 8'hbf;
311 assign sbox[8'hf5] = 8'he6;
312 assign sbox[8'hf6] = 8'h42;
313 assign sbox[8'hf7] = 8'h68;
314 assign sbox[8'hf8] = 8'h41;
315 assign sbox[8'hf9] = 8'h99;
316 assign sbox[8'hfa] = 8'h2d;
317 assign sbox[8'hfb] = 8'h0f;
318 assign sbox[8'hfc] = 8'hb0;
319 assign sbox[8'hfd] = 8'h54;
320 assign sbox[8'hfe] = 8'hbb;
321 assign sbox[8'hff] = 8'h16;
322

```

```

323 endmodule // aes_sbox

```

```

324
325 //

```

```

==
326 // EOF aes_sbox.v
327 //

```

```

==

```

SHA-256 코어 (Verilog)

What this is. SHA-256 with the compression function and the message schedule (W memory) as separate modules.

Why it exists. To show that a hash's message schedule is a block in its own right. Each of the 64 rounds consumes one W word, and the W words are generated recursively from earlier ones. Whether to expand all 64 in advance or to maintain a 16-word rotating buffer is a direct area–latency trade.

Undergraduate review · EECS concepts. Mapping a recurrence to hardware; circular buffers; area–time trade-offs.

Graduate · Practice note. Read sha256_w_mem.v first. The 16-word rotating buffer idiom recurs in LDPC message storage, FFT delay lines and FIR delay chains; recognising it once makes all of them familiar.

4 files • 1,194 lines

API — sha256.v

Table 759. secworks_sha256/src/rtl/sha256.v — module / interface / package (1개)

이름	종류
sha256	module

Table 760. secworks_sha256/src/rtl/sha256.v — localparam (19개)

상수	형/폭	값
ADDR_NAME0		8'h00
ADDR_NAME1		8'h01
ADDR_VERSION		8'h02
ADDR_CTRL		8'h08
CTRL_INIT_BIT		0
CTRL_NEXT_BIT		1
CTRL_MODE_BIT		2
ADDR_STATUS		8'h09
STATUS_READY_BIT		0
STATUS_VALID_BIT		1
ADDR_BLOCK0		8'h10
ADDR_BLOCK15		8'h1f
ADDR_DIGEST0		8'h20
ADDR_DIGEST7		8'h27
CORE_NAME0		32'h73686132
CORE_NAME1		32'h2d323536
CORE_VERSION		32'h312e3830

상수	형/폭	값
MODE_SHA_224		1'h0
MODE_SHA_256		1'h1

Table 761. secworks_sha256/src/rtl/sha256.v — 포트 (8개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
cs	input	wire
we	input	wire
address	input	wire [7 : 0]
write_data	input	wire [31 : 0]
read_data	output	wire [31 : 0]
error	output	wire

API — sha256_core.v**Table 762. secworks_sha256/src/rtl/sha256_core.v — module / interface / package (1개)**

이름	종류
sha256_core	module

Table 763. secworks_sha256/src/rtl/sha256_core.v — localparam (20개)

상수	형/폭	값
SHA224_H0_0		32'hc1059ed8
SHA224_H0_1		32'h367cd507
SHA224_H0_2		32'h3070dd17
SHA224_H0_3		32'hf70e5939
SHA224_H0_4		32'hffc00b31
SHA224_H0_5		32'h68581511
SHA224_H0_6		32'h64f98fa7
SHA224_H0_7		32'hbfa4fa4
SHA256_H0_0		32'h6a09e667
SHA256_H0_1		32'hbb67ae85
SHA256_H0_2		32'h3c6ef372
SHA256_H0_3		32'ha54ff53a
SHA256_H0_4		32'h510e527f
SHA256_H0_5		32'h9b05688c
SHA256_H0_6		32'h1f83d9ab

상수	형/폭	값
SHA256_H0_7		32'h5be0cd19
SHA256_ROUNDS		63
CTRL_IDLE		0
CTRL_ROUNDS		1
CTRL_DONE		2

Table 764. secworks_sha256/src/rtl/sha256_core.v — 포트 (9개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
init	input	wire
next	input	wire
mode	input	wire
block	input	wire [511 : 0]
ready	output	wire
digest	output	wire [255 : 0]
digest_valid	output	wire

API — sha256_k_constants.v**Table 765. secworks_sha256/src/rtl/sha256_k_constants.v — module / interface / package (1개)**

이름	종류
sha256_k_constants	module

Table 766. secworks_sha256/src/rtl/sha256_k_constants.v — 포트 (2개)

포트	방향	형/폭
round	input	wire [5 : 0]
K	output	wire [31 : 0]

API — sha256_w_mem.v**Table 767. secworks_sha256/src/rtl/sha256_w_mem.v — module / interface / package (1개)**

이름	종류
sha256_w_mem	module

Table 768. secworks_sha256/src/rtl/sha256_w_mem.v — 포트 (7개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire

포트	방향	형/폭
block	input	wire [511 : 0]
round	input	wire [5 : 0]
init	input	wire
next	input	wire
w	output	wire [31 : 0]

1. secworks_sha256/src/rtl/sha256.v (265 lines)

```

1 //
=====
2 //
3 // sha256.v
4 // -----
5 // Top level wrapper for the SHA-256 hash function providing
6 // a simple memory like interface with 32 bit data access.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2013, 201, Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the fol-
15 // lowing conditions are met:
16 //
17 // 1. Redistributions of source code must retain the above
18 // copyright notice, this list of conditions and the following dis-
19 // claimer.
20 // 2. Redistributions in binary form must reproduce the above
21 // copyright notice, this list of conditions and the following dis-
22 // claimer in the documentation and/or other materials provided with
23 // the distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
26 // TRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
27 // BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
28 // FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
29 // THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
30 // INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
31 // (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
32 // VICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
33 // HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
35 // WISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====
39
40 `default_nettype none
41
42 module sha256(
43     // Clock and reset.
44     input wire      clk,
45     input wire      reset_n,
46
47     // Control.
48     input wire      cs,
49     input wire      we,
50
51     // Data ports.
52     input wire [7 : 0] address,
53     input wire [31 : 0] write_data,
54     output wire [31 : 0] read_data,
55     output wire      error
56 );
57
58 //-----
59 // Internal constant and parameter definitions.

```

```

60 //-----
61 localparam ADDR_NAME0    = 8'h00;
62 localparam ADDR_NAME1    = 8'h01;
63 localparam ADDR_VERSION  = 8'h02;
64
65 localparam ADDR_CTRL     = 8'h08;
66 localparam CTRL_INIT_BIT = 0;
67 localparam CTRL_NEXT_BIT = 1;
68 localparam CTRL_MODE_BIT = 2;
69
70 localparam ADDR_STATUS    = 8'h09;
71 localparam STATUS_READY_BIT = 0;
72 localparam STATUS_VALID_BIT = 1;
73
74 localparam ADDR_BLOCK0    = 8'h10;
75 localparam ADDR_BLOCK15  = 8'h1f;
76
77 localparam ADDR_DIGEST0   = 8'h20;
78 localparam ADDR_DIGEST7  = 8'h27;
79
80 localparam CORE_NAME0     = 32'h73686132; // "sha2"
81 localparam CORE_NAME1     = 32'h2d323536; // "-256"
82 localparam CORE_VERSION   = 32'h312e3830; // "1.80"
83
84 localparam MODE_SHA_224   = 1'h0;
85 localparam MODE_SHA_256  = 1'h1;
86
87
88 //-----
89 // Registers including update variables and write enable.
90 //-----
91 reg init_reg;
92 reg init_new;
93
94 reg next_reg;
95 reg next_new;
96
97 reg mode_reg;
98 reg mode_new;
99 reg mode_we;
100
101 reg ready_reg;
102
103 reg [31 : 0] block_reg [0 : 15];
104 reg          block_we;
105
106 reg [255 : 0] digest_reg;
107
108 reg digest_valid_reg;
109
110
111 //-----
112 // Wires.
113 //-----
114 wire          core_ready;
115 wire [511 : 0] core_block;
116 wire [255 : 0] core_digest;
117 wire          core_digest_valid;
118
119 reg [31 : 0] tmp_read_data;
120 reg          tmp_error;
121
122
123 //-----
124 // Concurrent connectivity for ports etc.
125 //-----
126 assign core_block = {block_reg[00], block_reg[01],
127                     block_reg[02], block_reg[03],
128                     block_reg[04], block_reg[05],
129                     block_reg[06], block_reg[07],
130                     block_reg[08], block_reg[09],
131                     block_reg[10], block_reg[11],
132                     block_reg[12], block_reg[13],
133                     block_reg[14], block_reg[15]};
134
135 assign read_data = tmp_read_data;

```

```

132 assign error      = tmp_error;
133
134 //-----
135
136 // core instantiation.
137 //-----
138 sha256_core core(
139     .clk(clk),
140     .reset_n(reset_n),
141
142     .init(init_reg),
143     .next(next_reg),
144     .mode(mode_reg),
145
146     .block(core_block),
147
148     .ready(core_ready),
149
150     .digest(core_digest),
151     .digest_valid(core_digest_valid)
152 );
153
154 //-----
155
156 // reg_update
157 //
158 // Update functionality for all registers in the core.
159 // All registers are positive edge triggered with asynchron-
160 // ous
161 // active low reset. All registers have write enable.
162 //-----
163 always @ (posedge clk or negedge reset_n)
164 begin : reg_update
165     integer i;
166
167     if (!reset_n)
168     begin
169         for (i = 0 ; i < 16 ; i = i + 1)
170             block_reg[i] <= 32'h0;
171
172         init_reg      <= 0;
173         next_reg      <= 0;
174         ready_reg     <= 0;
175         mode_reg      <= MODE_SHA_256;
176         digest_reg    <= 256'h0;
177         digest_valid_reg <= 0;
178     end
179 else
180     begin
181         ready_reg      <= core_ready;
182         digest_valid_reg <= core_digest_valid;
183         init_reg       <= init_new;
184         next_reg       <= next_new;
185
186         if (mode_we)
187             mode_reg <= mode_new;
188
189         if (core_digest_valid)
190             digest_reg <= core_digest;
191
192         if (block_we)
193             block_reg[address[3 : 0]] <= write_data;
194     end
195 end // reg_update
196
197 //-----
198 // api_logic
199 //
200 // Implementation of the api logic. If cs is enabled will
201 // try to write to or read from the internal registers.
202 //-----
203 either
204 begin : api_logic
205     init_new      = 0;
206     next_new      = 0;
207     mode_new      = 0;
208     mode_we       = 0;
209     block_we      = 0;
210     tmp_read_data = 32'h0;
211     tmp_error     = 0;
212
213     if (cs)
214     begin
215         if (we)
216         begin
217             if (address == ADDR_CTRL)
218             begin
219                 init_new = write_data[CTRL_INIT_BIT];
220                 next_new = write_data[CTRL_NEXT_BIT];
221                 mode_new = write_data[CTRL_MODE_BIT];
222                 mode_we  = 1;
223             end
224

```

```

225         if ((address >= ADDR_BLOCK0) && (address <=
ADDR_BLOCK15))
226             block_we = 1;
227         end // if (we)
228
229     else
230     begin
231         if ((address >= ADDR_BLOCK0) && (address <=
ADDR_BLOCK15))
232             tmp_read_data = block_reg[address[3 : 0]];
233
234         if ((address >= ADDR_DIGEST0) && (address <=
ADDR_DIGEST7))
235             tmp_read_data = digest_reg[(7 - (address -
ADDR_DIGEST0)) * 32 +: 32];
236
237         case (address)
238             // Read operations.
239             ADDR_NAME0:
240                 tmp_read_data = CORE_NAME0;
241
242             ADDR_NAME1:
243                 tmp_read_data = CORE_NAME1;
244
245             ADDR_VERSION:
246                 tmp_read_data = CORE_VERSION;
247
248             ADDR_CTRL:
249                 tmp_read_data = {29'h0, mode_reg, next_reg,
init_reg};
250
251             ADDR_STATUS:
252                 tmp_read_data = {30'h0, digest_valid_reg,
ready_reg};
253
254             default:
255                 begin
256                     end
257             endcase // case (address)
258         end
259     end
260 end // addr_decoder
261 endmodule // sha256
262
263 //
=====
264 // EOF sha256.v
265 //
=====

```

2. secworks_sha256/src/rtl/sha256_core.v (554 lines)

```

1 //
=====
2 //
3 // sha256_core.v
4 // -----
5 // Verilog 2001 implementation of the SHA-256 hash function.
6 // This is the internal core with wide interfaces.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2013, Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the fol-
15 // lowing
16 // conditions are met:
17 // 1. Redistributions of source code must retain the above
18 // copyright
19 // notice, this list of conditions and the following dis-
20 // claimer.
21 // 2. Redistributions in binary form must reproduce the above
22 // copyright
23 // notice, this list of conditions and the following dis-
24 // claimer in
25 // the documentation and/or other materials provided with
26 // the
27 // distribution.
28 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
29 // TRIBUTORS
30 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
31 // BUT NOT
32 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
33 // FITNESS
34 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
35 // THE
36 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
37 // INDIRECT,
38 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
39 // (INCLUDING,
40 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
41 // VICES;

```

```

32 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
33 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
35 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====
39
40 `default_nettype none
41
42 module sha256_core(
43     input wire      clk,
44     input wire      reset_n,
45
46     input wire      init,
47     input wire      next,
48     input wire      mode,
49
50     input wire [511 : 0] block,
51
52     output wire      ready,
53     output wire [255 : 0] digest,
54     output wire      digest_valid
55 );
56
57
58 //-----
59 // Internal constant and parameter definitions.
60 //-----
61
62 localparam SHA224_H0_0 = 32'hc1059ed8;
63 localparam SHA224_H0_1 = 32'h367cd507;
64 localparam SHA224_H0_2 = 32'h3070dd17;
65 localparam SHA224_H0_3 = 32'hf70e5939;
66 localparam SHA224_H0_4 = 32'hffc00b31;
67 localparam SHA224_H0_5 = 32'h68581511;
68 localparam SHA224_H0_6 = 32'h64f98fa7;
69 localparam SHA224_H0_7 = 32'hbfa4fa4;
70
71 localparam SHA256_H0_0 = 32'h6a09e667;
72 localparam SHA256_H0_1 = 32'hbb67ae85;
73 localparam SHA256_H0_2 = 32'h3c6ef372;
74 localparam SHA256_H0_3 = 32'ha54ff53a;
75 localparam SHA256_H0_4 = 32'h510e527f;
76 localparam SHA256_H0_5 = 32'h9b05688c;
77 localparam SHA256_H0_6 = 32'h1f83d9ab;
78 localparam SHA256_H0_7 = 32'h5be0cd19;
79
80 localparam SHA256_ROUNDS = 63;
81
82 localparam CTRL_IDLE = 0;
83 localparam CTRL_ROUNDS = 1;
84 localparam CTRL_DONE = 2;
85
86 //-----
87 // Registers including update variables and write enable.
88 //-----
89
90 reg [31 : 0] a_reg;
91 reg [31 : 0] a_new;
92 reg [31 : 0] b_reg;
93 reg [31 : 0] b_new;
94 reg [31 : 0] c_reg;
95 reg [31 : 0] c_new;
96 reg [31 : 0] d_reg;
97 reg [31 : 0] d_new;
98 reg [31 : 0] e_reg;
99 reg [31 : 0] e_new;
100 reg [31 : 0] f_reg;
101 reg [31 : 0] f_new;
102 reg [31 : 0] g_reg;
103 reg [31 : 0] g_new;
104 reg [31 : 0] h_reg;
105 reg [31 : 0] h_new;
106 reg a_h_we;
107
108 reg [31 : 0] H0_reg;
109 reg [31 : 0] H0_new;
110 reg [31 : 0] H1_reg;
111 reg [31 : 0] H1_new;
112 reg [31 : 0] H2_reg;
113 reg [31 : 0] H2_new;
114 reg [31 : 0] H3_reg;
115 reg [31 : 0] H3_new;
116 reg [31 : 0] H4_reg;
117 reg [31 : 0] H4_new;
118 reg [31 : 0] H5_reg;
119 reg [31 : 0] H5_new;
120 reg [31 : 0] H6_reg;
121 reg [31 : 0] H6_new;
122 reg [31 : 0] H7_reg;
123 reg [31 : 0] H7_new;
124 reg H_we;

```

```

125 reg [5 : 0] t_ctr_reg;
126 reg [5 : 0] t_ctr_new;
127 reg t_ctr_we;
128 reg t_ctr_inc;
129 reg t_ctr_rst;
130
131 reg digest_valid_reg;
132 reg digest_valid_new;
133 reg digest_valid_we;
134
135 reg [1 : 0] sha256_ctrl_reg;
136 reg [1 : 0] sha256_ctrl_new;
137 reg sha256_ctrl_we;
138
139
140 //-----
141 // Wires.
142 //-----
143
144 reg digest_init;
145 reg digest_update;
146
147 reg state_init;
148 reg state_update;
149
150 reg first_block;
151
152 reg ready_flag;
153
154 reg [31 : 0] t1;
155 reg [31 : 0] t2;
156
157 wire [31 : 0] k_data;
158
159 reg w_init;
160 reg w_next;
161 reg [5 : 0] w_round;
162 wire [31 : 0] w_data;
163
164 //-----
165 // Module instantiations.
166 //-----
167
168 sha256_k_constants k_constants_inst(
169     .round(t_ctr_reg),
170     .K(k_data)
171 );
172
173 sha256_w_mem w_mem_inst(
174     .clk(clk),
175     .reset_n(reset_n),
176
177     .block(block),
178     .round(t_ctr_reg),
179
180     .init(w_init),
181     .next(w_next),
182     .w(w_data)
183 );
184
185 //-----
186 // Concurrent connectivity for ports etc.
187 //-----
188
189 assign ready = ready_flag;
190
191 assign digest = {H0_reg, H1_reg, H2_reg, H3_reg,
192                 H4_reg, H5_reg, H6_reg, H7_reg};
193
194 assign digest_valid = digest_valid_reg;
195
196 //-----
197 // reg_update
198 // Update functionality for all registers in the core.
199 // All registers are positive edge triggered with asynchron-
200 // active low reset. All registers have write enable.
201 //-----
202
203 always @ (posedge clk or negedge reset_n)
204     begin : reg_update
205         if (!reset_n)
206             begin
207                 a_reg      <= 32'h0;
208                 b_reg      <= 32'h0;
209                 c_reg      <= 32'h0;
210                 d_reg      <= 32'h0;
211                 e_reg      <= 32'h0;
212                 f_reg      <= 32'h0;
213                 g_reg      <= 32'h0;
214                 h_reg      <= 32'h0;
215                 H0_reg     <= 32'h0;
216                 H1_reg     <= 32'h0;

```

```

217     H2_reg      <= 32'h0;
218     H3_reg      <= 32'h0;
219     H4_reg      <= 32'h0;
220     H5_reg      <= 32'h0;
221     H6_reg      <= 32'h0;
222     H7_reg      <= 32'h0;
223     digest_valid_reg <= 0;
224     t_ctr_reg    <= 6'h0;
225     sha256_ctrl_reg <= CTRL_IDLE;
226 end
227 else
228 begin
229     if (a_h_we)
230     begin
231         a_reg <= a_new;
232         b_reg <= b_new;
233         c_reg <= c_new;
234         d_reg <= d_new;
235         e_reg <= e_new;
236         f_reg <= f_new;
237         g_reg <= g_new;
238         h_reg <= h_new;
239     end
240
241     if (H_we)
242     begin
243         H0_reg <= H0_new;
244         H1_reg <= H1_new;
245         H2_reg <= H2_new;
246         H3_reg <= H3_new;
247         H4_reg <= H4_new;
248         H5_reg <= H5_new;
249         H6_reg <= H6_new;
250         H7_reg <= H7_new;
251     end
252
253     if (t_ctr_we)
254     begin
255         t_ctr_reg <= t_ctr_new;
256     end
257
258     if (digest_valid_we)
259     begin
260         digest_valid_reg <= digest_valid_new;
261     end
262
263     if (sha256_ctrl_reg <= sha256_ctrl_new;
264 end // reg_update
265
266 //-----
267 // digest_logic
268 //
269 // The logic needed to init as well as update the digest.
270 //-----
271
272 always @*
273 begin : digest_logic
274     H0_new = 32'h0;
275     H1_new = 32'h0;
276     H2_new = 32'h0;
277     H3_new = 32'h0;
278     H4_new = 32'h0;
279     H5_new = 32'h0;
280     H6_new = 32'h0;
281     H7_new = 32'h0;
282     H_we = 0;
283
284     if (digest_init)
285     begin
286         H_we = 1;
287         if (mode)
288         begin
289             H0_new = SHA256_H0_0;
290             H1_new = SHA256_H0_1;
291             H2_new = SHA256_H0_2;
292             H3_new = SHA256_H0_3;
293             H4_new = SHA256_H0_4;
294             H5_new = SHA256_H0_5;
295             H6_new = SHA256_H0_6;
296             H7_new = SHA256_H0_7;
297         end
298     else
299     begin
300         H0_new = SHA224_H0_0;
301         H1_new = SHA224_H0_1;
302         H2_new = SHA224_H0_2;
303         H3_new = SHA224_H0_3;
304         H4_new = SHA224_H0_4;
305         H5_new = SHA224_H0_5;
306         H6_new = SHA224_H0_6;
307         H7_new = SHA224_H0_7;
308     end
309 end
310
311 if (digest_update)
312 begin
313     H0_new = H0_reg + a_reg;
314     H1_new = H1_reg + b_reg;
315     H2_new = H2_reg + c_reg;
316     H3_new = H3_reg + d_reg;

```

```

316     H4_new = H4_reg + e_reg;
317     H5_new = H5_reg + f_reg;
318     H6_new = H6_reg + g_reg;
319     H7_new = H7_reg + h_reg;
320     H_we = 1;
321 end
322 end // digest_logic
323
324 //-----
325 // t1_logic
326 //
327 // The logic for the T1 function.
328 //-----
329
330 always @*
331 begin : t1_logic
332     reg [31 : 0] sum1;
333     reg [31 : 0] ch;
334
335     sum1 = {e_reg[5 : 0], e_reg[31 : 6]} ^
336           {e_reg[10 : 0], e_reg[31 : 11]} ^
337           {e_reg[24 : 0], e_reg[31 : 25]};
338
339     ch = (e_reg & f_reg) ^ ((~e_reg) & g_reg);
340
341     t1 = h_reg + sum1 + ch + w_data + k_data;
342 end // t1_logic
343
344 //-----
345 // t2_logic
346 //
347 // The logic for the T2 function
348 //-----
349
350 always @*
351 begin : t2_logic
352     reg [31 : 0] sum0;
353     reg [31 : 0] maj;
354
355     sum0 = {a_reg[1 : 0], a_reg[31 : 2]} ^
356           {a_reg[12 : 0], a_reg[31 : 13]} ^
357           {a_reg[21 : 0], a_reg[31 : 22]};
358
359     maj = (a_reg & b_reg) ^ (a_reg & c_reg) ^ (b_reg &
360 c_reg);
361
362     t2 = sum0 + maj;
363 end // t2_logic
364
365 //-----
366 // state_logic
367 //
368 // The logic needed to init as well as update the state dur-
369 ing
370 // round processing.
371 //-----
372
373 always @*
374 begin : state_logic
375     a_new = 32'h0;
376     b_new = 32'h0;
377     c_new = 32'h0;
378     d_new = 32'h0;
379     e_new = 32'h0;
380     f_new = 32'h0;
381     g_new = 32'h0;
382     h_new = 32'h0;
383     a_h_we = 0;
384
385     if (state_init)
386     begin
387         a_h_we = 1;
388         if (first_block)
389         begin
390             if (mode)
391             begin
392                 a_new = SHA256_H0_0;
393                 b_new = SHA256_H0_1;
394                 c_new = SHA256_H0_2;
395                 d_new = SHA256_H0_3;
396                 e_new = SHA256_H0_4;
397                 f_new = SHA256_H0_5;
398                 g_new = SHA256_H0_6;
399                 h_new = SHA256_H0_7;
400             end
401         else
402         begin
403             a_new = SHA224_H0_0;
404             b_new = SHA224_H0_1;
405             c_new = SHA224_H0_2;
406             d_new = SHA224_H0_3;
407             e_new = SHA224_H0_4;
408             f_new = SHA224_H0_5;
409             g_new = SHA224_H0_6;
410             h_new = SHA224_H0_7;

```

```

409         end
410     end
411     else
412     begin
413         a_new = H0_reg;
414         b_new = H1_reg;
415         c_new = H2_reg;
416         d_new = H3_reg;
417         e_new = H4_reg;
418         f_new = H5_reg;
419         g_new = H6_reg;
420         h_new = H7_reg;
421     end
422 end
423
424 if (state_update)
425 begin
426     a_new = t1 + t2;
427     b_new = a_reg;
428     c_new = b_reg;
429     d_new = c_reg;
430     e_new = d_reg + t1;
431     f_new = e_reg;
432     g_new = f_reg;
433     h_new = g_reg;
434     a_h_we = 1;
435 end
436 end // state_logic
437
438
439 //-----
440 // t_ctr
441 //
442 // Update logic for the round counter, a monotonically
443 // increasing counter with reset.
444 //-----
445 always @*
446 begin : t_ctr
447     t_ctr_new = 0;
448     t_ctr_we = 0;
449
450     if (t_ctr_rst)
451     begin
452         t_ctr_new = 0;
453         t_ctr_we = 1;
454     end
455
456     if (t_ctr_inc)
457     begin
458         t_ctr_new = t_ctr_reg + 1'b1;
459         t_ctr_we = 1;
460     end
461 end // t_ctr
462
463
464 //-----
465 // sha256_ctrl_fsm
466 //
467 // Logic for the state machine controlling the core beha-
468 // viour.
469 //-----
470 always @*
471 begin : sha256_ctrl_fsm
472     digest_init = 0;
473     digest_update = 0;
474
475     state_init = 0;
476     state_update = 0;
477
478     first_block = 0;
479     ready_flag = 0;
480
481     w_init = 0;
482     w_next = 0;
483
484     t_ctr_inc = 0;
485     t_ctr_rst = 0;
486
487     digest_valid_new = 0;
488     digest_valid_we = 0;
489
490     sha256_ctrl_new = CTRL_IDLE;
491     sha256_ctrl_we = 0;
492
493     case (sha256_ctrl_reg)
494     CTRL_IDLE:
495     begin
496         ready_flag = 1;
497
498         if (init)
499         begin
500             digest_init = 1;
501             w_init = 1;
502             state_init = 1;
503             first_block = 1;
504             t_ctr_rst = 1;

```

```

505         digest_valid_new = 0;
506         digest_valid_we = 1;
507         sha256_ctrl_new = CTRL_ROUNDS;
508         sha256_ctrl_we = 1;
509     end
510
511     if (next)
512     begin
513         t_ctr_rst = 1;
514         w_init = 1;
515         state_init = 1;
516         digest_valid_new = 0;
517         digest_valid_we = 1;
518         sha256_ctrl_new = CTRL_ROUNDS;
519         sha256_ctrl_we = 1;
520     end
521 end
522
523
524 CTRL_ROUNDS:
525 begin
526     w_next = 1;
527     state_update = 1;
528     t_ctr_inc = 1;
529
530     if (t_ctr_reg == SHA256_ROUNDS)
531     begin
532         sha256_ctrl_new = CTRL_DONE;
533         sha256_ctrl_we = 1;
534     end
535 end
536
537
538 CTRL_DONE:
539 begin
540     digest_update = 1;
541     digest_valid_new = 1;
542     digest_valid_we = 1;
543
544     sha256_ctrl_new = CTRL_IDLE;
545     sha256_ctrl_we = 1;
546 end
547 endcase // case (sha256_ctrl_reg)
548 end // sha256_ctrl_fsm
549
550 endmodule // sha256_core
551
552 //

```

```

=====
553 // EOF sha256_core.v
554 //
=====

```

3. secworks_sha256/src/rtl/sha256_k_constants.v (134 lines)

```

1 //
=====
2 //
3 // sha256_k_constants.v
4 // -----
5 // The table K with constants in the SHA-256 hash function.
6 //
7 //
8 // Author: Joachim Strombergson
9 // Copyright (c) 2013, Secworks Sweden AB
10 // All rights reserved.
11 //
12 // Redistribution and use in source and binary forms, with or
13 // without modification, are permitted provided that the fol-
14 // lowing
15 // conditions are met:
16 // 1. Redistributions of source code must retain the above
17 // copyright
18 // notice, this list of conditions and the following dis-
19 // claimer.
20 // 2. Redistributions in binary form must reproduce the above
21 // copyright
22 // notice, this list of conditions and the following dis-
23 // claimer in
24 // the documentation and/or other materials provided with
25 // the
26 // distribution.
27 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
28 // TRIBUTORS
29 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
30 // BUT NOT
31 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
32 // FITNESS
33 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
34 // THE
35 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
36 // INDIRECT,
37 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
38 // (INCLUDING,

```

```

30 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
31 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
32 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
33 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
34 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
35 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
36 //
37 //
=====
38
39 `default_nettype none
40
41 module sha256_k_constants(
42     input wire [5 : 0] round,
43     output wire [31 : 0] K
44 );
45
46 //-----
47 // Wires.
48 //-----
49
50 reg [31 : 0] tmp_K;
51
52 //-----
53 // Concurrent connectivity for ports etc.
54 //-----
55
56 assign K = tmp_K;
57
58 //-----
59 // round_mux
60 //-----
61
62 always @*
63 begin : round_mux
64     case(round)
65         00: tmp_K = 32'h428a2f98;
66         01: tmp_K = 32'h71374491;
67         02: tmp_K = 32'hb5c0fbcf;
68         03: tmp_K = 32'he9b5dba5;
69         04: tmp_K = 32'h3956c25b;
70         05: tmp_K = 32'h59f111f1;
71         06: tmp_K = 32'h923f82a4;
72         07: tmp_K = 32'hab1c5ed5;
73         08: tmp_K = 32'hd807aa98;
74         09: tmp_K = 32'h12835b01;
75         10: tmp_K = 32'h243185be;
76         11: tmp_K = 32'h550c7dc3;
77         12: tmp_K = 32'h72be5d74;
78         13: tmp_K = 32'h80deb1fe;
79         14: tmp_K = 32'h9bdc06a7;
80         15: tmp_K = 32'hc19bf174;
81         16: tmp_K = 32'he49b69c1;
82         17: tmp_K = 32'hef6be4786;
83         18: tmp_K = 32'h0fc19dc6;
84         19: tmp_K = 32'h240ca1cc;
85         20: tmp_K = 32'h2de92c6f;
86         21: tmp_K = 32'h4a7484aa;
87         22: tmp_K = 32'h5cb0a9dc;
88         23: tmp_K = 32'h76f98da;
89         24: tmp_K = 32'h983e5152;
90         25: tmp_K = 32'ha831c66d;
91         26: tmp_K = 32'hb00327c8;
92         27: tmp_K = 32'hbf597fc7;
93         28: tmp_K = 32'hc6e00bf3;
94         29: tmp_K = 32'hd5a79147;
95         30: tmp_K = 32'h06ca6351;
96         31: tmp_K = 32'h14292967;
97         32: tmp_K = 32'h27b70a85;
98         33: tmp_K = 32'h2e1b2138;
99         34: tmp_K = 32'h4d2c6dfc;
100        35: tmp_K = 32'h53380d13;
101        36: tmp_K = 32'h650a7354;
102        37: tmp_K = 32'h766a0abb;
103        38: tmp_K = 32'h81c2c92e;
104        39: tmp_K = 32'h92722c85;
105        40: tmp_K = 32'ha2bfe8a1;
106        41: tmp_K = 32'ha81a664b;
107        42: tmp_K = 32'hc24b8b70;
108        43: tmp_K = 32'hc76c51a3;
109        44: tmp_K = 32'hd192e819;
110        45: tmp_K = 32'hd6990624;
111        46: tmp_K = 32'hf40e3585;
112        47: tmp_K = 32'h106aa070;
113        48: tmp_K = 32'h19a4c116;
114        49: tmp_K = 32'h1e376c08;
115        50: tmp_K = 32'h2748774c;
116        51: tmp_K = 32'h34b0bcb5;
117        52: tmp_K = 32'h391c0cb3;
118        53: tmp_K = 32'h4ed8aa4a;
119        54: tmp_K = 32'h5b9cca4f;
120        55: tmp_K = 32'h682e6fff;

```

```

120        56: tmp_K = 32'h748f82ee;
121        57: tmp_K = 32'h78a5636f;
122        58: tmp_K = 32'h84c87814;
123        59: tmp_K = 32'h8cc70208;
124        60: tmp_K = 32'h90beffff;
125        61: tmp_K = 32'ha4506ceb;
126        62: tmp_K = 32'hbef9a3f7;
127        63: tmp_K = 32'hc67178f2;
128        endcase // case (round)
129        end // block: round_mux
130    endmodule // sha256_k_constants
131
132 //
=====

```

```

133 // sha256_k_constants.v
134 //
=====

```

4. secworks_sha256/src/rtl/sha256_w_mem.v (241 lines)

```

1 //
=====
2 //
3 // sha256_w_mem_regs.v
4 //
5 // The W memory. This version uses 16 32-bit registers as a
sliding
6 // window to generate the 64 words.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2013, Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the fol-
lowing
15 // conditions are met:
16 //
17 // 1. Redistributions of source code must retain the above
copyright
18 // notice, this list of conditions and the following dis-
claimer.
19 //
20 // 2. Redistributions in binary form must reproduce the above
copyright
21 // notice, this list of conditions and the following dis-
claimer in
22 // the documentation and/or other materials provided with
the
23 // distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
TRIBUTORS
26 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
27 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
28 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
29 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
30 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
31 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
32 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
33 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
35 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====
39
40 `default_nettype none
41
42 module sha256_w_mem(
43     input wire          clk,
44     input wire          reset_n,
45
46     input wire [511 : 0] block,
47     input wire [5 : 0] round,
48
49     input wire          init,
50     input wire          next,
51     output wire [31 : 0] w
52 );
53
54 //-----
55 //
56 // Registers including update variables and write enable.
57 //-----

```



```

58 reg [31 : 0] w_mem [0 : 15];
59 reg [31 : 0] w_mem00_new;
60 reg [31 : 0] w_mem01_new;
61 reg [31 : 0] w_mem02_new;
62 reg [31 : 0] w_mem03_new;
63 reg [31 : 0] w_mem04_new;
64 reg [31 : 0] w_mem05_new;
65 reg [31 : 0] w_mem06_new;
66 reg [31 : 0] w_mem07_new;
67 reg [31 : 0] w_mem08_new;
68 reg [31 : 0] w_mem09_new;
69 reg [31 : 0] w_mem10_new;
70 reg [31 : 0] w_mem11_new;
71 reg [31 : 0] w_mem12_new;
72 reg [31 : 0] w_mem13_new;
73 reg [31 : 0] w_mem14_new;
74 reg [31 : 0] w_mem15_new;
75 reg w_mem_we;
76
77 //-----
78
79 // Wires.
80 //-----
81
81 reg [31 : 0] w_tmp;
82 reg [31 : 0] w_new;
83
84 //-----
85
86 // Concurrent connectivity for ports etc.
87 //-----
88
88 assign w = w_tmp;
89
90 //-----
91
92 // reg_update
93 // Update functionality for all registers in the core.
94 // All registers are positive edge triggered with synchron-
ous
95 // active low reset. All registers have write enable.
96 //-----
97
97 always @(posedge clk or negedge reset_n)
98   begin : reg_update
99     integer i;
100
101     if (!reset_n)
102       begin
103         for (i = 0 ; i < 16 ; i = i + 1) begin
104           w_mem[i] <= 32'h0;
105         end
106       end
107     else
108       begin
109         if (w_mem_we)
110           begin
111             w_mem[00] <= w_mem00_new;
112             w_mem[01] <= w_mem01_new;
113             w_mem[02] <= w_mem02_new;
114             w_mem[03] <= w_mem03_new;
115             w_mem[04] <= w_mem04_new;
116             w_mem[05] <= w_mem05_new;
117             w_mem[06] <= w_mem06_new;
118             w_mem[07] <= w_mem07_new;
119             w_mem[08] <= w_mem08_new;
120             w_mem[09] <= w_mem09_new;
121             w_mem[10] <= w_mem10_new;
122             w_mem[11] <= w_mem11_new;
123             w_mem[12] <= w_mem12_new;
124             w_mem[13] <= w_mem13_new;
125             w_mem[14] <= w_mem14_new;
126             w_mem[15] <= w_mem15_new;
127           end
128         end
129       end // reg_update
130
131 //-----
132
133 // select_w
134 //
135 // Mux for the external read operation. This is where we ex-
ract
136 // the W variable.
137 //-----
138
138 always @*
139   begin : select_w
140     if (round < 16)
141       w_tmp = w_mem[round[3 : 0]];
142     else
143       w_tmp = w_new;
144     end // select_w
145
146 //-----
147

```

```

148 // w_new_logic
149 //
150 // Logic that calculates the next value to be inserted into
151 // the sliding window of the memory.
152 //-----
153
153 always @*
154   begin : w_mem_update_logic
155     reg [31 : 0] w_0;
156     reg [31 : 0] w_1;
157     reg [31 : 0] w_9;
158     reg [31 : 0] w_14;
159     reg [31 : 0] d0;
160     reg [31 : 0] d1;
161
162     w_mem00_new = 32'h0;
163     w_mem01_new = 32'h0;
164     w_mem02_new = 32'h0;
165     w_mem03_new = 32'h0;
166     w_mem04_new = 32'h0;
167     w_mem05_new = 32'h0;
168     w_mem06_new = 32'h0;
169     w_mem07_new = 32'h0;
170     w_mem08_new = 32'h0;
171     w_mem09_new = 32'h0;
172     w_mem10_new = 32'h0;
173     w_mem11_new = 32'h0;
174     w_mem12_new = 32'h0;
175     w_mem13_new = 32'h0;
176     w_mem14_new = 32'h0;
177     w_mem15_new = 32'h0;
178     w_mem_we = 0;
179
180     w_0 = w_mem[0];
181     w_1 = w_mem[1];
182     w_9 = w_mem[9];
183     w_14 = w_mem[14];
184
185     d0 = {w_1[6 : 0], w_1[31 : 7]} ^
186           {w_1[17 : 0], w_1[31 : 18]} ^
187           {3'b000, w_1[31 : 3]};
188
189     d1 = {w_14[16 : 0], w_14[31 : 17]} ^
190           {w_14[18 : 0], w_14[31 : 19]} ^
191           {10'b0000000000, w_14[31 : 10]};
192
193     w_new = d1 + w_9 + d0 + w_0;
194
195     if (init)
196       begin
197         w_mem00_new = block[511 : 480];
198         w_mem01_new = block[479 : 448];
199         w_mem02_new = block[447 : 416];
200         w_mem03_new = block[415 : 384];
201         w_mem04_new = block[383 : 352];
202         w_mem05_new = block[351 : 320];
203         w_mem06_new = block[319 : 288];
204         w_mem07_new = block[287 : 256];
205         w_mem08_new = block[255 : 224];
206         w_mem09_new = block[223 : 192];
207         w_mem10_new = block[191 : 160];
208         w_mem11_new = block[159 : 128];
209         w_mem12_new = block[127 : 96];
210         w_mem13_new = block[95 : 64];
211         w_mem14_new = block[63 : 32];
212         w_mem15_new = block[31 : 0];
213         w_mem_we = 1;
214       end
215
216     if (next && (round > 15))
217       begin
218         w_mem00_new = w_mem[01];
219         w_mem01_new = w_mem[02];
220         w_mem02_new = w_mem[03];
221         w_mem03_new = w_mem[04];
222         w_mem04_new = w_mem[05];
223         w_mem05_new = w_mem[06];
224         w_mem06_new = w_mem[07];
225         w_mem07_new = w_mem[08];
226         w_mem08_new = w_mem[09];
227         w_mem09_new = w_mem[10];
228         w_mem10_new = w_mem[11];
229         w_mem11_new = w_mem[12];
230         w_mem12_new = w_mem[13];
231         w_mem13_new = w_mem[14];
232         w_mem14_new = w_mem[15];
233         w_mem15_new = w_new;
234         w_mem_we = 1;
235       end
236     end // w_mem_update_logic
237 endmodule // sha256_w_mem
238
239 //
=====
240 // sha256_w_mem.v
241 //
=====

```

OpenTitan AES -- 양산 보안 RTL

What this is. The AES block from a production security chip: cipher core, triplicated control FSMs, six S-box implementations, masking and clearing PRNGs, CTR and GCM support, shadowed registers, TL-UL interface.

Why it exists. It contains the entire difference between an AES that is functionally correct and one that can be shipped. The algorithm is identical to the plain Verilog core. Everything else defends against power analysis, fault injection, misconfiguration and data remanence.

Undergraduate review · EECS concepts. Differential power analysis; threshold implementations and probing security; fault-tolerant design through complementary redundancy; CMOS switching energy $P \propto \alpha CV^2 f$.

Graduate · Practice note. Open `aes_sbox_dom.sv` (1,077 lines) beside `aes_sbox_lut.sv` (120 lines). They compute the same mathematical function at nine times the size. **A functional regression cannot distinguish them** — which is precisely the boundary a modelling engineer must understand and state.

8 files • 4,898 lines

API — aes.sv

Table 769. lowRISC_opentitan/hw/ip/aes/rtl/aes.sv — module / interface / package (1개)

이름	종류
aes	module

Table 770. lowRISC_opentitan/hw/ip/aes/rtl/aes.sv — parameter (14개)

파라미터	형/폭	기본값
AES192Enable	bit	1
AESGCMEnable	bit	1
SecMasking	bit	1
SecSBoxImpl	sbox_impl_e	SBoxImplDom
SecStartTriggerDelay	int unsigned	0
SecAllowForcingMasks	bit	0
SecSkipPRNGReseeding	bit	0
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
RndCnstClearingLfsrSeed	clearing_lfsr_seed_t	RndCnstClearingLfsrSeedDefault
RndCnstClearingLfsrPerm	clearing_lfsr_perm_t	RndCnstClearingLfsrPermDefault
RndCnstClearingSharePerm	clearing_lfsr_perm_t	RndCnstClearingSharePermDefault
RndCnstMaskingLfsrSeed	masking_lfsr_seed_t	RndCnstMaskingLfsrSeedDefault
RndCnstMaskingLfsrPerm	masking_lfsr_perm_t	RndCnstMaskingLfsrPermDefault

Table 771. lowRISC_opentitan/hw/ip/aes/rtl/aes.sv — localparam (1개)

상수	형/폭	값
EntropyWidth	int unsigned	edn_pkg::ENDPOINT_BUS_WIDTH

Table 772. lowRISC_opentitan/hw/ip/aes/rtl/aes.sv — 포트 (16개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
rst_shadowed_ni	input	logic
idle_o	output	prim_mubi_pkg::mubi4_t
output_valid_o	output	logic
input_ready_o	output	logic
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
clk_edn_i	input	logic
rst_edn_ni	input	logic
edn_o	output	edn_pkg::edn_req_t
edn_i	input	edn_pkg::edn_rsp_t
keymgr_key_i	input	keymgr_pkg::hw_key_req_t
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]

API — aes_cipher_control.sv**Table 773. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control.sv — module / interface / package (1개)**

이름	종류
aes_cipher_control	module

Table 774. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control.sv — parameter (3개)

파라미터	형/폭	기본값
CiphOpFwdOnly	bit	0
SecMasking	bit	0
SecSBoxImpl	sbox_impl_e	SBoxImplDom

Table 775. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control.sv — localparam (2개)

상수	형/폭	값
NumSp2VSSig	int unsigned	8
Sp2VEnSecBuf	bit [NumSp2VSSig-1:0]	8'b1100_0000

Table 776. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control.sv — 포트 (45개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_i	input	sp2v_e
in_ready_o	output	sp2v_e
out_valid_o	output	sp2v_e
out_ready_i	input	sp2v_e
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_i	input	sp2v_e
crypt_o	output	sp2v_e
dec_key_gen_i	input	sp2v_e
dec_key_gen_o	output	sp2v_e
prng_reseed_i	input	logic
prng_reseed_o	output	logic
key_clear_i	input	logic
key_clear_o	output	logic
data_out_clear_i	input	logic
data_out_clear_o	output	logic
mux_sel_err_i	input	logic
sp_enc_err_i	input	logic
op_err_i	input	logic
alert_fatal_i	input	logic
alert_o	output	logic
prng_update_o	output	logic
prng_reseed_req_o	output	logic
prng_reseed_ack_i	input	logic
state_sel_o	output	state_sel_e
state_we_o	output	sp2v_e
sub_bytes_en_o	output	sp2v_e
sub_bytes_out_req_i	input	sp2v_e
sub_bytes_out_ack_o	output	sp2v_e
add_rk_sel_o	output	add_rk_sel_e
key_expand_op_o	output	ciph_op_e

포트	방향	형/폭
key_full_sel_o	output	key_full_sel_e
key_full_we_o	output	sp2v_e
key_dec_sel_o	output	key_dec_sel_e
key_dec_we_o	output	sp2v_e
key_expand_en_o	output	sp2v_e
key_expand_out_req_i	input	sp2v_e
key_expand_out_ack_o	output	sp2v_e
key_expand_clear_o	output	logic
key_expand_round_o	output	logic [3:0]
key_words_sel_o	output	key_words_sel_e
round_key_sel_o	output	round_key_sel_e

API — aes_cipher_control_fsm.sv

Table 777. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm.sv — module / interface / package (1개)

이름	종류
aes_cipher_control_fsm	module

Table 778. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm.sv — parameter (2개)

파라미터	형/폭	기본값
SecMasking	bit	0
SecSBoxImpl	sbox_impl_e	SBoxImplDom

Table 779. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm.sv — 포트 (50개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_i	input	logic
in_ready_o	output	logic
out_valid_o	output	logic
out_ready_i	input	logic
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_i	input	logic
dec_key_gen_i	input	logic
prng_reseed_i	input	logic

포트	방향	형/폭
key_clear_i	input	logic
data_out_clear_i	input	logic
mux_sel_err_i	input	logic
sp_enc_err_i	input	logic
rnd_ctr_err_i	input	logic
op_err_i	input	logic
alert_fatal_i	input	logic
alert_o	output	logic
prng_update_o	output	logic
prng_reseed_req_o	output	logic
prng_reseed_ack_i	input	logic
state_sel_o	output	state_sel_e
state_we_o	output	logic
sub_bytes_en_o	output	logic
sub_bytes_out_req_i	input	logic
sub_bytes_out_ack_o	output	logic
add_rk_sel_o	output	add_rk_sel_e
key_full_sel_o	output	key_full_sel_e
key_full_we_o	output	logic
key_dec_sel_o	output	key_dec_sel_e
key_dec_we_o	output	logic
key_expand_en_o	output	logic
key_expand_out_req_i	input	logic
key_expand_out_ack_o	output	logic
key_expand_clear_o	output	logic
rnd_ctr_o	output	logic [3:0]
key_words_sel_o	output	key_words_sel_e
round_key_sel_o	output	round_key_sel_e
crypt_q_i	input	logic
crypt_d_o	output	logic
dec_key_gen_q_i	input	logic
dec_key_gen_d_o	output	logic
prng_reseed_q_i	input	logic
prng_reseed_d_o	output	logic
key_clear_q_i	input	logic

포트	방향	형/폭
key_clear_d_o	output	logic
data_out_clear_q_i	input	logic
data_out_clear_d_o	output	logic

API — aes_cipher_control_fsm_n.sv

Table 780. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm_n.sv — module / interface / package (1개)

이름	종류
aes_cipher_control_fsm_n	module

Table 781. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm_n.sv — parameter (2개)

파라미터	형/폭	기본값
SecMasking	bit	0
SecSBoxImpl	sbox_impl_e	SBoxImplDom

Table 782. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm_n.sv — localparam (2개)

상수	형/폭	값
NumInBufBits	int	\$bits({
NumOutBufBits	int	\$bits({

Table 783. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm_n.sv — 포트 (50개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_ni	input	logic
in_ready_no	output	logic
out_valid_no	output	logic
out_ready_ni	input	logic
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_ni	input	logic
dec_key_gen_ni	input	logic
prng_reseed_i	input	logic
key_clear_i	input	logic
data_out_clear_i	input	logic
mux_sel_err_i	input	logic

포트	방향	형/폭
sp_enc_err_i	input	logic
rnd_ctr_err_i	input	logic
op_err_i	input	logic
alert_fatal_i	input	logic
alert_o	output	logic
prng_update_o	output	logic
prng_reseed_req_o	output	logic
prng_reseed_ack_i	input	logic
state_sel_o	output	state_sel_e
state_we_no	output	logic
sub_bytes_en_no	output	logic
sub_bytes_out_req_ni	input	logic
sub_bytes_out_ack_no	output	logic
add_rk_sel_o	output	add_rk_sel_e
key_full_sel_o	output	key_full_sel_e
key_full_we_no	output	logic
key_dec_sel_o	output	key_dec_sel_e
key_dec_we_no	output	logic
key_expand_en_no	output	logic
key_expand_out_req_ni	input	logic
key_expand_out_ack_no	output	logic
key_expand_clear_o	output	logic
rnd_ctr_o	output	logic [3:0]
key_words_sel_o	output	key_words_sel_e
round_key_sel_o	output	round_key_sel_e
crypt_q_ni	input	logic
crypt_d_no	output	logic
dec_key_gen_q_ni	input	logic
dec_key_gen_d_no	output	logic
prng_reseed_q_i	input	logic
prng_reseed_d_o	output	logic
key_clear_q_i	input	logic
key_clear_d_o	output	logic
data_out_clear_q_i	input	logic
data_out_clear_d_o	output	logic

API — aes_cipher_control_fsm_p.sv**Table 784. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm_p.sv — module / interface / package (1개)**

이름	종류
aes_cipher_control_fsm_p	module

Table 785. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm_p.sv — parameter (2개)

파라미터	형/폭	기본값
SecMasking	bit	0
SecSBoxImpl	sbox_impl_e	SBoxImplDom

Table 786. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm_p.sv — localparam (2개)

상수	형/폭	값
NumInBufBits	int	\$bits({
NumOutBufBits	int	\$bits({

Table 787. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm_p.sv — 포트 (50개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_i	input	logic
in_ready_o	output	logic
out_valid_o	output	logic
out_ready_i	input	logic
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_i	input	logic
dec_key_gen_i	input	logic
prng_reseed_i	input	logic
key_clear_i	input	logic
data_out_clear_i	input	logic
mux_sel_err_i	input	logic
sp_enc_err_i	input	logic
rnd_ctr_err_i	input	logic
op_err_i	input	logic
alert_fatal_i	input	logic
alert_o	output	logic

포트	방향	형/폭
prng_update_o	output	logic
prng_reseed_req_o	output	logic
prng_reseed_ack_i	input	logic
state_sel_o	output	state_sel_e
state_we_o	output	logic
sub_bytes_en_o	output	logic
sub_bytes_out_req_i	input	logic
sub_bytes_out_ack_o	output	logic
add_rk_sel_o	output	add_rk_sel_e
key_full_sel_o	output	key_full_sel_e
key_full_we_o	output	logic
key_dec_sel_o	output	key_dec_sel_e
key_dec_we_o	output	logic
key_expand_en_o	output	logic
key_expand_out_req_i	input	logic
key_expand_out_ack_o	output	logic
key_expand_clear_o	output	logic
rnd_ctr_o	output	logic [3:0]
key_words_sel_o	output	key_words_sel_e
round_key_sel_o	output	round_key_sel_e
crypt_q_i	input	logic
crypt_d_o	output	logic
dec_key_gen_q_i	input	logic
dec_key_gen_d_o	output	logic
prng_reseed_q_i	input	logic
prng_reseed_d_o	output	logic
key_clear_q_i	input	logic
key_clear_d_o	output	logic
data_out_clear_q_i	input	logic
data_out_clear_d_o	output	logic

API — aes_cipher_core.sv

Table 788. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_core.sv — module / interface / package (1개)

이름	종류
aes_cipher_core	module

Table 789. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_core.sv — parameter (9개)

파라미터	형/폭	기본값
AES192Enable	bit	1
CiphOpFwdOnly	bit	0
SecMasking	bit	1
SecSBoxImpl	sbox_impl_e	SBoxImplDom
SecAllowForcingMasks	bit	0
SecSkipPRNGReseeding	bit	0
EntropyWidth	int unsigned	edn_pkg::ENDPOINT_BUS_WIDTH
RndCnstMaskingLfsrSeed	masking_lfsr_seed_t	RndCnstMaskingLfsrSeedDefault
RndCnstMaskingLfsrPerm	masking_lfsr_perm_t	RndCnstMaskingLfsrPermDefault

Table 790. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_core.sv — localparam (5개)

상수	형/폭	값
NumShares	int	SecMasking ? 2 : 1
WidthPRDRow	int unsigned	4*WidthPRDSBox
NumSp2VSig	int unsigned	3
Sp2VEnSecBuf	bit [NumSp2VSig-1:0]	{NumSp2VSig{1'b1}}
NumCyclesPerRound	int unsigned	(SecSBoxImpl == SBoxImplDom) ? 5 : 1

Table 791. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_core.sv — 포트 (26개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_i	input	sp2v_e
in_ready_o	output	sp2v_e
out_valid_o	output	sp2v_e
out_ready_i	input	sp2v_e
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_i	input	sp2v_e
crypt_o	output	sp2v_e
dec_key_gen_i	input	sp2v_e
dec_key_gen_o	output	sp2v_e
prng_reseed_i	input	logic
prng_reseed_o	output	logic

포트	방향	형/폭
key_clear_i	input	logic
key_clear_o	output	logic
data_out_clear_i	input	logic
data_out_clear_o	output	logic
alert_fatal_i	input	logic
alert_o	output	logic
force_masks_i	input	logic
data_in_mask_o	output	logic [3:0][3:0][7:0]
entropy_req_o	output	logic
entropy_ack_i	input	logic
entropy_i	input	logic [EntropyWidth-1:0]

API — aes_control.sv

Table 792. lowRISC_opentitan/hw/ip/aes/rtl/aes_control.sv — module / interface / package (1개)

이름	종류
aes_control	module

Table 793. lowRISC_opentitan/hw/ip/aes/rtl/aes_control.sv — parameter (3개)

파라미터	형/폭	기본값
AESGCMEnable	bit	0
SecMasking	bit	0
SecStartTriggerDelay	int unsigned	0

Table 794. lowRISC_opentitan/hw/ip/aes/rtl/aes_control.sv — localparam (3개)

상수	형/폭	값
WidthCounter	int unsigned	\$clog2(SecStartTriggerDelay+1)
NumSp2VSign	int unsigned	7 + NumSlicesCtr
Sp2VEnSecBuf	bit [NumSp2VSign-1:0]	'0

Table 795. lowRISC_opentitan/hw/ip/aes/rtl/aes_control.sv — 포트 (87개) — 앞 60개만 표시(전체 87개, 본문 소스 참조)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
ctrl_qe_i	input	logic
ctrl_we_o	output	logic
ctrl_phase_i	input	logic

포트	방향	형/폭
ctrl_err_storage_i	input	logic
op_i	input	aes_op_e
mode_i	input	aes_mode_e
cipher_op_i	input	ciph_op_e
sideload_i	input	logic
prng_reseed_rate_i	input	prs_rate_e
manual_operation_i	input	logic
key_touch_forces_reseed_i	input	logic
ctrl_gcm_qe_i	input	logic
ctrl_gcm_we_o	output	logic
ctrl_gcm_phase_i	input	logic
gcm_init_done_o	output	logic
gcm_phase_i	input	gcm_phase_e
start_i	input	logic
key_iv_data_in_clear_i	input	logic
data_out_clear_i	input	logic
prng_reseed_i	input	logic
mux_sel_err_i	input	logic
sp_enc_err_i	input	logic
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
alert_fatal_i	input	logic
alert_o	output	logic
key_sideload_valid_i	input	logic
iv_qe_i	input	logic [NumRegsIv-1:0]
data_in_qe_i	input	logic [NumRegsData-1:0]
data_out_re_i	input	logic [NumRegsData-1:0]
data_in_we_o	output	logic
data_out_sel_o	output	data_out_sel_e
data_out_we_o	output	sp2v_e
data_in_prev_sel_o	output	dip_sel_e
data_in_prev_we_o	output	sp2v_e
state_in_sel_o	output	si_sel_e
add_state_in_sel_o	output	add_si_sel_e
add_state_out_sel_o	output	add_so_sel_e
ctr_inc32_o	output	sp2v_e

포트	방향	형/폭
ctr_incr_o	output	sp2v_e
ctr_ready_i	input	sp2v_e
ctr_we_i	input	sp2v_e [NumSlicesCtr-1:0]
cipher_in_valid_o	output	sp2v_e
cipher_in_ready_i	input	sp2v_e
cipher_out_valid_i	input	sp2v_e
cipher_out_ready_o	output	sp2v_e
cipher_crypt_o	output	sp2v_e
cipher_crypt_i	input	sp2v_e
cipher_dec_key_gen_o	output	sp2v_e
cipher_dec_key_gen_i	input	sp2v_e
cipher_prng_reseed_o	output	logic
cipher_prng_reseed_i	input	logic
cipher_key_clear_o	output	logic
cipher_key_clear_i	input	logic
cipher_data_out_clear_o	output	logic
cipher_data_out_clear_i	input	logic
ghash_in_valid_o	output	sp2v_e
ghash_in_ready_i	input	sp2v_e
ghash_out_valid_i	input	sp2v_e

API — aes_control_fsm.sv

Table 796. lowRISC_opentitan/hw/ip/aes/rtl/aes_control_fsm.sv — module / interface / package (1개)

이름	종류
aes_control_fsm	module

Table 797. lowRISC_opentitan/hw/ip/aes/rtl/aes_control_fsm.sv — parameter (2개)

파라미터	형/폭	기본값
AESGCMEnable	bit	0
SecMasking	bit	0

Table 798. lowRISC_opentitan/hw/ip/aes/rtl/aes_control_fsm.sv — 포트 (86개) — 앞 60개만 표시(전체 86개, 본문 소스 참조)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
ctrl_qe_i	input	logic

포트	방향	형/폭
ctrl_we_o	output	logic
ctrl_phase_i	input	logic
ctrl_err_storage_i	input	logic
op_i	input	aes_op_e
mode_i	input	aes_mode_e
cipher_op_i	input	ciph_op_e
sideload_i	input	logic
prng_reseed_rate_i	input	prs_rate_e
manual_operation_i	input	logic
key_touch_forces_reseed_i	input	logic
ctrl_gcm_qe_i	input	logic
ctrl_gcm_we_o	output	logic
ctrl_gcm_phase_i	input	logic
gcm_init_done_o	output	logic
gcm_phase_i	input	gcm_phase_e
start_i	input	logic
key_iv_data_in_clear_i	input	logic
data_out_clear_i	input	logic
prng_reseed_i	input	logic
mux_sel_err_i	input	logic
sp_enc_err_i	input	logic
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
alert_fatal_i	input	logic
alert_o	output	logic
key_sideload_valid_i	input	logic
key_init_qe_i	input	logic [NumSharesKey-1:0][NumRegsKey-1:0]
iv_qe_i	input	logic [NumRegslv-1:0]
data_in_qe_i	input	logic [NumRegsData-1:0]
data_out_re_i	input	logic [NumRegsData-1:0]
data_in_we_o	output	logic
data_out_sel_o	output	data_out_sel_e
data_out_we_o	output	logic
data_in_prev_sel_o	output	dip_sel_e
data_in_prev_we_o	output	logic
state_in_sel_o	output	si_sel_e

포트	방향	형/폭
add_state_in_sel_o	output	add_si_sel_e
add_state_out_sel_o	output	add_so_sel_e
ctr_inc32_o	output	logic
ctr_incr_o	output	logic
ctr_ready_i	input	logic
ctr_we_i	input	logic [NumSlicesCtr-1:0]
cipher_in_valid_o	output	logic
cipher_in_ready_i	input	logic
cipher_out_valid_i	input	logic
cipher_out_ready_o	output	logic
cipher_crypt_o	output	logic
cipher_crypt_i	input	logic
cipher_dec_key_gen_o	output	logic
cipher_dec_key_gen_i	input	logic
cipher_prng_reseed_o	output	logic
cipher_prng_reseed_i	input	logic
cipher_key_clear_o	output	logic
cipher_key_clear_i	input	logic
cipher_data_out_clear_o	output	logic
cipher_data_out_clear_i	input	logic
ghash_in_valid_o	output	logic
ghash_in_ready_i	input	logic

1. lowRISC_opentitan/hw/ip/aes/rtl/aes.sv (328 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES top-level wrapper
6
7 `include "prim_assert.sv"
8
9 module aes
10   import aes_pkg::*;
11   import aes_reg_pkg::*;
12   #(
13     parameter bit      AES192Enable      = 1, // Can be
0 (disable), or 1 (enable).
14     parameter bit      AESGCMEnable      = 1, // Can be
0 (disable), or 1 (enable).
15     parameter bit      SecMasking        = 1, // Can be
0 (no masking), or
16     // 1
(first-order masking) of the cipher
17     // core.
Masking requires the use of a
18     // masked
S-Box, see SecSBoxImpl parameter.
19     parameter sbox_impl_e SecSBoxImpl    = SBoxImpl-
Dom, // See aes_pkg.sv
20     parameter int unsigned SecStartTriggerDelay = 0, // Manual
start trigger delay, useful for
21     // SCA

```

```

measurements. A value of e.g. 40
22                                     // allows
the processor to go into sleep
23                                     // before
AES starts operation.
24     parameter bit      SecAllowForcingMasks = 0, // Allow
forcing masks to constant values using
25                                     //
FORCE_MASKS bit in Auxiliary Control
26                                     // Re-
gister. Useful for SCA only.
27     parameter bit      SecSkipPRNGReseeding = 0, // The
current SCA setup doesn't provide enough
28                                     // re-
sources to implement the infrastructure
29                                     // re-
quired for PRNG reseeding (CSRNG, EDN).
30                                     // To en-
able SCA resistance evaluations, we
31                                     // need to
skip reseeding requests.
32                                     // Useful
for SCA only.
33     parameter logic [NumAlerts-1:0] AlertAsyncOn = {Nu-
mAlerts{1'b1}},
34     // Number of cycles a differential skew is tolerated on the
alert signal
35     parameter int unsigned AlertSkewCycles      = 1,
36     parameter clearing_lfsr_seed_t RndCnstClearingLfsrSeed =
RndCnstClearingLfsrSeedDefault,
37     parameter clearing_lfsr_perm_t RndCnstClearingLfsrPerm =
RndCnstClearingLfsrPermDefault,
38     parameter clearing_lfsr_perm_t RndCnstClearingSharePerm =
RndCnstClearingSharePermDefault,

```



```

39  parameter masking_lfsr_seed_t  RndCnstMaskingLfsrSeed  =
    RndCnstMaskingLfsrSeedDefault,
40  parameter masking_lfsr_perm_t  RndCnstMaskingLfsrPerm  =
    RndCnstMaskingLfsrPermDefault
41 ) (
42  input  logic                    clk_i,
43  input  logic                    rst_ni,
44  input  logic                    rst_shad-
owed_ni,
45
46  // Idle indicator for clock manager
47  output prim_mubi_pkg::mubi4_t   idle_o,
48
49  // Status indicators for DMA integrations
50  output logic                    out-
put_valid_o,
51  output logic                    in-
put_ready_o,
52
53  // Life cycle
54  input  lc_ctrl_pkg::lc_tx_t     lc_escal-
ate_en_i,
55
56  // Entropy distribution network (EDN) interface
57  input  logic                    clk_edn_i,
58  input  logic                    rst_edn_ni,
59  output edn_pkg::edn_req_t       edn_o,
60  input  edn_pkg::edn_rsp_t       edn_i,
61
62  // Key manager (keymgr) key sideload interface
63  input  keymgr_pkg::hw_key_req_t keym-
gr_key_i,
64
65  // Bus interface
66  input  tlul_pkg::tl_h2d_t       tl_i,
67  output tlul_pkg::tl_d2h_t       tl_o,
68
69  // Alerts
70  input  prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_rx_i,
71  output prim_alert_pkg::alert_tx_t [NumAlerts-1:0] alert_tx_o
72 );
73
74  localparam int unsigned EntropyWidth = edn_pkg::END-
POINT_BUS_WIDTH;
75
76  // Signals
77  aes_reg2hw_t          reg2hw;
78  aes_hw2reg_t          hw2reg;
79
80  logic [NumAlerts-1:0] alert;
81  lc_ctrl_pkg::lc_tx_t  lc_escalate_en;
82
83  logic                    edn_req_int;
84  logic                    edn_req_hold_d, edn_req_hold_q;
85  logic                    edn_req;
86  logic                    edn_ack;
87  logic [EntropyWidth-1:0] edn_data;
88  logic                    unused_edn_fips;
89  logic                    entropy_clearing_req, en-
tropy_masking_req;
90  logic                    entropy_clearing_ack, en-
tropy_masking_ack;
91
92  ////////////
93  // Inputs //
94  ////////////
95
96  // SEC_CM: BUS_INTEGRITY
97  // SEC_CM: AUX.CONFIG.SHADOW
98  // SEC_CM: AUX.CONFIG.REGWEN
99  // SEC_CM: KEY.SW_UNREADABLE
100 // SEC_CM: DATA_REG.SW_UNREADABLE
101 // Register interface
102 logic intg_err_alert;
103 logic shadowed_storage_err, shadowed_update_err;
104 aes_reg_top u_reg (
105   .clk_i,
106   .rst_ni,
107   .rst_shadowed_ni,
108   .tl_i,
109   .tl_o,
110   .reg2hw,
111   .hw2reg,
112   .shadowed_storage_err_o(shadowed_storage_err),
113   .shadowed_update_err_o(shadowed_update_err),
114   .intg_err_o(intg_err_alert)
115 );
116
117 // SEC_CM: LC_ESCALATE_EN.INTERSIG.MUBI
118 // Synchronize life cycle input
119 prim_lc_sync #(
120   .NumCopies (1)
121 ) u_prim_lc_sync (
122   .clk_i,
123   .rst_ni,
124   .lc_en_i ( lc_escalate_en_i ),
125   .lc_en_o ( {lc_escalate_en} )
126 );
127

```

```

128  ////////////
129  // EDN Interface //
130  ////////////
131
132  // Internally, we have up to two PRNGs that share the EDN
interface for reseeding. Here, we just
133  // arbitrate the requests. Upsizing of the entropy to the
correct width is performed inside the
134  // PRNGs.
135  // Reseed operations for the clearing PRNG are initiated by
software. Reseed operations for the
136  // masking PRNG can also be automatically initiated.
137  assign edn_req_int = entropy_clearing_req | en-
tropy_masking_req;
138  // Only forward ACK to PRNG currently requesting entropy.
Give higher priority to clearing PRNG.
139  assign entropy_clearing_ack = entropy_clearing_req &
edn_ack;
140  assign entropy_masking_ack = ~entropy_clearing_req & en-
tropy_masking_req & edn_ack;
141
142  // Upon escalation or detection of a fatal alert, an EDN re-
quest signal can be dropped before
143  // getting acknowledged. This is okay with respect to AES as
the module will need to be reset
144  // anyway. However, to not leave EDN in a strange state, we
hold the request until it's actually
145  // acknowledged.
146  assign edn_req = edn_req_int | edn_req_hold_q;
147  assign edn_req_hold_d = (edn_req_hold_q | edn_req) &
~edn_ack;
148  always_ff @(posedge clk_i or negedge rst_ni) begin :
edn_req_reg
149    if (!rst_ni) begin
150      edn_req_hold_q <= '0;
151    end else begin
152      edn_req_hold_q <= edn_req_hold_d;
153    end
154  end
155
156  // Synchronize EDN interface
157  prim_sync_reqack_data #(
158    .Width(EntropyWidth),
159    .DataSrc2Dst(1'b0),
160    .DataReg(1'b0)
161  ) u_prim_sync_reqack_data (
162    .clk_src_i ( clk_i ),
163    .rst_src_ni ( rst_ni ),
164    .clk_dst_i ( clk_edn_i ),
165    .rst_dst_ni ( rst_edn_ni ),
166    .req_chk_i ( 1'b1 ),
167    .src_req_i ( edn_req ),
168    .src_ack_o ( edn_ack ),
169    .dst_req_o ( edn_o.edn_req ),
170    .dst_ack_i ( edn_i.edn_ack ),
171    .data_i ( edn_i.edn_bus ),
172    .data_o ( edn_data )
173  );
174  // We don't track whether the entropy is pre-FIPS or not in-
side AES.
175  assign unused_edn_fips = edn_i.edn_fips;
176
177  ////////////
178  // Core //
179  ////////////
180
181  // AES core
182  aes_core #(
183    .AES192Enable ( AES192Enable ),
184    .AESGCMEnable ( AESGCMEnable ),
185    .SecMasking ( SecMasking ),
186    .SecSBoxImpl ( SecSBoxImpl ),
187    .SecStartTriggerDelay ( SecStartTriggerDelay ),
188    .SecAllowForcingMasks ( SecAllowForcingMasks ),
189    .SecSkipPRNGReseeding ( SecSkipPRNGReseeding ),
190    .EntropyWidth ( EntropyWidth ),
191    .RndCnstClearingLfsrSeed ( RndCnstClearingLfsrSeed ),
192    .RndCnstClearingLfsrPerm ( RndCnstClearingLfsrPerm ),
193    .RndCnstClearingSharePerm ( RndCnstClearingSharePerm ),
194    .RndCnstMaskingLfsrSeed ( RndCnstMaskingLfsrSeed ),
195    .RndCnstMaskingLfsrPerm ( RndCnstMaskingLfsrPerm )
196  ) u_aes_core (
197    .clk_i
198    ( clk_i ),
199    .rst_ni
200    ( rst_ni ),
201    .rst_shadowed_ni
202    ( rst_shad-
owed_ni ),
203    .entropy_clearing_req_o ( en-
tropy_clearing_req ),
204    .entropy_clearing_ack_i ( en-
tropy_clearing_ack ),
205    .entropy_clearing_i
206    ( edn_data ),
207    .entropy_masking_req_o ( en-
tropy_masking_req ),
208    .entropy_masking_ack_i ( en-
tropy_masking_ack ),
209    .entropy_masking_i
210    ( edn_data ),

```

```

206
207 .keymgr_key_i      ( keymgr_key_i ),
gr_key_i
208
209 .lc_escalate_en_i  ( lc_escalate_en ),
ate_en
210
211 .shadowed_storage_err_i ( shadowed_storage_err ),
owed_storage_err
212
213 .shadowed_update_err_i ( shadowed_update_err ),
owed_update_err
214
215 .intg_err_alert_i   ( intg_err_alert ),
(intg_err_alert)
216
217 .alert_recov_o      ( alert_recov_o ),
(alert[AlertRecovCtrlUpdateErrIdx])
218
219 .alert_fatal_o      ( alert_fatal_o ),
(alert[AlertFatalFaultIdx])
220
221 .reg2hw             ( reg2hw ),
(reg2hw)
222
223 .hw2reg             ( hw2reg ),
(hw2reg)
224
225 );
226
227 assign idle_o =
prim_mubi_pkg::mubi4_bool_to_mubi(reg2hw.status.idle.q);
228
229 assign output_valid_o = reg2hw.status.output_valid.q;
230
231 assign input_ready_o = reg2hw.status.input_ready.q;
232
233 // Alerts
234 // Alerts
235 // Alerts
236
237 logic [NumAlerts-1:0] alert_test;
238 assign alert_test[AlertRecovCtrlUpdateErrIdx] =
reg2hw.alert_test.recov_ctrl_update_err.q &
reg2hw.alert_test.recov_ctrl_update_err.qe;
239
240 assign alert_test[AlertFatalFaultIdx] =
reg2hw.alert_test.fatal_fault.q &
reg2hw.alert_test.fatal_fault.qe;
241
242 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
243     prim_alert_sender #(
244         .AsyncOn(AlertAsyncOn[i]),
245         .SkewCycles(AlertSkewCycles),
246         .IsFatal(i == AlertFatalFaultIdx)
247     ) u_prim_alert_sender (
248         .clk_i,
249         .rst_ni,
250         .alert_test_i ( alert_test[i] ),
251         .alert_req_i ( alert[i] ),
252         .alert_ack_o ( ),
253         .alert_state_o ( ),
254         .alert_rx_i ( alert_rx_i[i] ),
255         .alert_tx_o ( alert_tx_o[i] )
256     );
257 end
258
259 // Assertions
260 // Assertions
261 // Assertions
262
263 // All outputs should have a known value after reset
264 `ASSERT_KNOWN(T10DValidKnown, tl_o.d_valid)
265 `ASSERT_KNOWN(T10AReadyKnown, tl_o.a_ready)
266 `ASSERT_KNOWN(IdleKnown, idle_o)
267 `ASSERT_KNOWN(OutputValidKnown, output_valid_o)
268 `ASSERT_KNOWN(InputReadyKnown, input_ready_o)
269 `ASSERT_KNOWN(EdnReqKnown, edn_o)
270 `ASSERT_KNOWN(AlertTxKnown, alert_tx_o)
271
272 // Alert assertions for sparse FSMs.
273 for (genvar i = 0; i < Sp2VWidth; i++) begin : gen_ctr_fsm_svas
274     if (SP2V_LOGIC_HIGH[i] == 1'b1) begin : gen_ctr_fsm_svas_p
275         `AS-
276         SERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCtrlFsmCheck_A,
277             u_aes_core.u_aes_ctrl.gen_fsm[i].gen_fsm.p,
278             u_aes_ctrl_fsm_i.u_aes_ctrl_fsm.u_state_regs,
279             alert_tx_o[1])
280         end else begin : gen_ctr_fsm_svas_n
281             `AS-
282             SERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCtrlFsmCheck_A,
283                 u_aes_core.u_aes_ctrl.gen_fsm[i].gen_fsm.n,
284                 u_aes_ctrl_fsm_i.u_aes_ctrl_fsm.u_state_regs,
285                 alert_tx_o[1])
286         end
287     end
288 end
289
290 for (genvar i = 0; i < Sp2VWidth; i++) begin :
gen_cipher_control_fsm_svas
291     if (SP2V_LOGIC_HIGH[i] == 1'b1) begin :
gen_cipher_control_fsm_svas_p
292         `AS-
293         SERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCipherControlFsmCheck_A,
294             u_aes_core.u_aes_cipher_core.u_aes_cipher_control.gen_fsm[i].gen_fsm.p,
295             u_aes_cipher_control_fsm_i.u_aes_cipher_control_fsm.u_state_regs,
296             alert_tx_o[1])
297         end else begin : gen_cipher_control_fsm_svas_n
298             `AS-
299             SERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCipherControlFsmCheck_A,
300                 u_aes_core.u_aes_cipher_core.u_aes_cipher_control.gen_fsm[i].gen_fsm.n,
301                 u_aes_cipher_control_fsm_i.u_aes_cipher_control_fsm.u_state_regs,
302                 alert_tx_o[1])
303         end
304     end
305 end
306
307 if (AESGCMEEnable) begin : gen_ghash_fsm_sva
308     `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesGhashFsmCheck_A,
309         u_aes_core.gen_ghash.u_aes_ghash.u_state_regs,
310         alert_tx_o[1])
311 end
312
313 if (AESGCMEEnable && SecMasking) begin : gen_ghash_onehot_sva
314     for (genvar s = 0; s < 2; s++) begin :
gen_ghash_onehot_add_in_sva
315         `AS-
316         SERT_PRIM_ONEHOT_ERROR_TRIGGER_ALERT(GhashAadOnehotCheck_A,
317             u_aes_core.gen_ghash.u_aes_ghash.gen_masked_add.gen_add_in_muxes[s].
318             u_prim_onehot_check_add_in_sel,
319             alert_tx_o[1])
320         end
321     end
322     `AS-
323     SERT_PRIM_ONEHOT_ERROR_TRIGGER_ALERT(GhashMultOnehotCheck_A,
324         u_aes_core.gen_ghash.u_aes_ghash.gen_gf_mult1_mux.u_prim_onehot_che
325         ck_gf_mult1_in_sel,
326         alert_tx_o[1])
327 end
328 // Alert assertions for reg_we onehot check
329 `AS-
330 SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
331     u_reg, alert_tx_o[1])
332 endmodule

```

```

286 end else begin : gen_ctr_fsm_svas_n
287     `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCtrlFsmCheck_A,
288         u_aes_core.u_aes_ctrl.gen_fsm[i].gen_fsm.n,
289         u_aes_ctrl_fsm_i.u_aes_ctrl_fsm.u_state_regs,
290         alert_tx_o[1])
291 end
292 end
293
294 for (genvar i = 0; i < Sp2VWidth; i++) begin :
gen_cipher_control_fsm_svas
295     if (SP2V_LOGIC_HIGH[i] == 1'b1) begin :
gen_cipher_control_fsm_svas_p
296         `AS-
297         SERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCipherControlFsmCheck_A,
298             u_aes_core.u_aes_cipher_core.u_aes_cipher_control.gen_fsm[i].gen_fsm.p,
299             u_aes_cipher_control_fsm_i.u_aes_cipher_control_fsm.u_state_regs,
300             alert_tx_o[1])
301         end else begin : gen_cipher_control_fsm_svas_n
302             `AS-
303             SERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCipherControlFsmCheck_A,
304                 u_aes_core.u_aes_cipher_core.u_aes_cipher_control.gen_fsm[i].gen_fsm.n,
305                 u_aes_cipher_control_fsm_i.u_aes_cipher_control_fsm.u_state_regs,
306                 alert_tx_o[1])
307         end
308     end
309 end
310
311 if (AESGCMEEnable) begin : gen_ghash_fsm_sva
312     `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesGhashFsmCheck_A,
313         u_aes_core.gen_ghash.u_aes_ghash.u_state_regs,
314         alert_tx_o[1])
315 end
316
317 if (AESGCMEEnable && SecMasking) begin : gen_ghash_onehot_sva
318     for (genvar s = 0; s < 2; s++) begin :
gen_ghash_onehot_add_in_sva
319         `AS-
320         SERT_PRIM_ONEHOT_ERROR_TRIGGER_ALERT(GhashAadOnehotCheck_A,
321             u_aes_core.gen_ghash.u_aes_ghash.gen_masked_add.gen_add_in_muxes[s].
322             u_prim_onehot_check_add_in_sel,
323             alert_tx_o[1])
324         end
325     end
326     `AS-
327     SERT_PRIM_ONEHOT_ERROR_TRIGGER_ALERT(GhashMultOnehotCheck_A,
328         u_aes_core.gen_ghash.u_aes_ghash.gen_gf_mult1_mux.u_prim_onehot_che
329         ck_gf_mult1_in_sel,
330         alert_tx_o[1])
331 end
332 // Alert assertions for reg_we onehot check
333 `AS-
334 SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
335     u_reg, alert_tx_o[1])
336 endmodule

```

2. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control.sv (485 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 //
6 // AES cipher core control
7 // This module controls the AES cipher core including the key
8 // expand module.
9 `include "prim_assert.sv"
10
11 module aes_cipher_control import aes_pkg::*;
12 #(
13     parameter bit CiphOpFwdOnly = 0,
14     parameter bit SecMasking = 0,
15     parameter sbbox_impl_e SecSBoxImpl = SBoxImplDom
16 ) (
17     input logic clk_i,
18     input logic rst_ni,
19
20     // Input handshake signals
21     input sp2v_e in_valid_i,
22     output sp2v_e in_ready_o,
23
24     // Output handshake signals
25     output sp2v_e out_valid_o,
26     input sp2v_e out_ready_i,
27
28     // Control and sync signals
29     input logic cfg_valid_i,
30     input ciph_op_e op_i,
31     input key_len_e key_len_i,
32     input sp2v_e crypt_i,

```

```

33 output sp2v_e crypt_o,
34 input sp2v_e dec_key_gen_i,
35 output sp2v_e dec_key_gen_o,
36 input logic prng_reseed_i,
37 output logic prng_reseed_o,
38 input logic key_clear_i,
39 output logic key_clear_o,
40 input logic data_out_clear_i,
41 output logic data_out_clear_o,
42 input logic mux_sel_err_i,
43 input logic sp_enc_err_i,
44 input logic op_err_i,
45 input logic alert_fatal_i,
46 output logic alert_o,
47
48 // Control signals for masking PRNG
49 output logic prng_update_o,
50 output logic prng_reseed_req_o,
51 input logic prng_reseed_ack_i,
52
53 // Control and sync signals for cipher data path
54 output state_sel_e state_sel_o,
55 output sp2v_e state_we_o,
56 output sp2v_e sub_bytes_en_o,
57 input sp2v_e sub_bytes_out_req_i,
58 output sp2v_e sub_bytes_out_ack_o,
59 output add_rk_sel_e add_rk_sel_o,
60
61 // Control and sync signals for key expand data path
62 output ciph_op_e key_expand_op_o,
63 output key_full_sel_e key_full_sel_o,
64 output sp2v_e key_full_we_o,
65 output key_dec_sel_e key_dec_sel_o,
66 output sp2v_e key_dec_we_o,
67 output sp2v_e key_expand_en_o,
68 input sp2v_e key_expand_out_req_i,
69 output sp2v_e key_expand_out_ack_o,
70 output logic key_expand_clear_o,
71 output logic [3:0] key_expand_round_o,
72 output key_words_sel_e key_words_sel_o,
73 output round_key_sel_e round_key_sel_o,
74 );
75
76 // Signals
77 logic [3:0] rnd_ctr;
78 sp2v_e crypt_d, crypt_q;
79 sp2v_e dec_key_gen_d,
80 logic prng_reseed_d, prng_reseed_o,
81 logic key_clear_d,
82 logic data_out_clear_d,
83 sp2v_e sub_bytes_out_req;
84 sp2v_e key_expand_out_req;
85 sp2v_e in_valid;
86 sp2v_e out_ready;
87 sp2v_e crypt;
88 sp2v_e dec_key_gen;
89 logic mux_sel_err;
90 logic mr_err;
91 logic sp_enc_err;
92 logic rnd_ctr_err;
93
94 // Sparsified FSM signals. These are needed for connecting
the individual bits of the Sp2V
95 // signals to the single-rail FSMs.
96 logic [Sp2VWidth-1:0] sp_in_valid;
97 logic [Sp2VWidth-1:0] sp_in_ready;
98 logic [Sp2VWidth-1:0] sp_out_valid;
99 logic [Sp2VWidth-1:0] sp_out_ready;
100 logic [Sp2VWidth-1:0] sp_crypt;
101 logic [Sp2VWidth-1:0] sp_dec_key_gen;
102 logic [Sp2VWidth-1:0] sp_state_we;
103 logic [Sp2VWidth-1:0] sp_sub_bytes_en;
104 logic [Sp2VWidth-1:0] sp_sub_bytes_out_req;
105 logic [Sp2VWidth-1:0] sp_sub_bytes_out_ack;
106 logic [Sp2VWidth-1:0] sp_key_full_we;
107 logic [Sp2VWidth-1:0] sp_key_dec_we;
108 logic [Sp2VWidth-1:0] sp_key_expand_en;
109 logic [Sp2VWidth-1:0] sp_key_expand_out_req;
110 logic [Sp2VWidth-1:0] sp_key_expand_out_ack;
111 logic [Sp2VWidth-1:0] sp_crypt_d;
112 logic [Sp2VWidth-1:0] sp_crypt_q;
113 logic [Sp2VWidth-1:0] sp_dec_key_gen_d;
114 logic [Sp2VWidth-1:0] sp_dec_key_gen_q;
115
116 // Multi-rail signals. These are outputs of the single-rail
FSMs and need combining.
117 logic [Sp2VWidth-1:0] mr_alert;
118 logic [Sp2VWidth-1:0] mr_prng_update;
119 logic [Sp2VWidth-1:0] mr_prng_reseed_req;
120 logic [Sp2VWidth-1:0] mr_key_expand_clear;
121 logic [Sp2VWidth-1:0] mr_prng_reseed_d;
122 logic [Sp2VWidth-1:0] mr_key_clear_d;
123 logic [Sp2VWidth-1:0] mr_data_out_clear_d;
124
125 state_sel_e [Sp2VWidth-1:0] mr_state_sel;
126 add_rk_sel_e [Sp2VWidth-1:0] mr_add_rk_sel;
127 key_full_sel_e [Sp2VWidth-1:0] mr_key_full_sel;
128 key_dec_sel_e [Sp2VWidth-1:0] mr_key_dec_sel;
129 key_words_sel_e [Sp2VWidth-1:0] mr_key_words_sel;
130 round_key_sel_e [Sp2VWidth-1:0] mr_round_key_sel;
131
132 logic [Sp2VWidth-1:0][3:0] mr_rnd_ctr;
133
134 // FSM
135 // FSM
136
137 // Convert sp2v_e signals to sparsified inputs.
138 assign sp_in_valid = {in_valid};
139 assign sp_out_ready = {out_ready};
140 assign sp_crypt = {crypt};
141 assign sp_dec_key_gen = {dec_key_gen};
142 assign sp_sub_bytes_out_req = {sub_bytes_out_req};
143 assign sp_key_expand_out_req = {key_expand_out_req};
144 assign sp_crypt_q = {crypt_q};
145 assign sp_dec_key_gen_q = {dec_key_gen_q};
146
147 // SEC_CM: CIPHER.FSM.RETURN
148 // SEC_CM: CIPHER.CTR.RETURN
149 // For every bit in the Sp2V signals, one separate rail is
instantiated. The inputs and outputs
150 // of every rail are buffered to prevent aggressive syntheses-
optimizations.
151 for (genvar i = 0; i < Sp2VWidth; i++) begin : gen_fsm
152 if (SP2V_LOGIC_HIGH[i] == 1'b1) begin : gen_fsm_p
153 aes_cipher_control_fsm_p # (
154 .SecMasking ( SecMasking ),
155 .SecSBoxImpl ( SecSBoxImpl )
156 ) u_aes_cipher_control_fsm_i (
157 .clk_i ( clk_i ),
158 .rst_i ( rst_i ),
159 .in_valid_i ( sp_in_val-
id[i] ), // Sparsified
160 .in_ready_o ( sp_in_ready[i] ), // Sparsified
161 .out_valid_o ( sp_out_val-
id[i] ), // Sparsified
162 .out_ready_i ( sp_out_ready[i] ), // Sparsified
163 .cfg_valid_i ( cfg_valid_i ),
164 .op_i ( op_i ),
165 .key_len_i ( key_len_i ),
166 .crypt_i ( sp_crypt[i] ), // Sparsified
167 .dec_key_gen_i ( sp_dec_key_gen[i] ), // Sparsified
168 .prng_reseed_i ( prng_reseed_i ),
169 .key_clear_i ( key_clear_i ),
170 .data_out_clear_i ( data_out_clear_i ),
171 .mux_sel_err_i ( mux_sel_err ),
172 .sp_enc_err_i ( sp_enc_err ),
173 .rnd_ctr_err_i ( rnd_ctr_err ),
174 .op_err_i ( op_err_i ),
175 .alert_fatal_i ( alert_fatal_i ),
176 .alert_o ( mr_alert[i] ), // OR-combine
177 .prng_update_o ( mr_prng_update[i] ), // OR-combine
178 .prng_reseed_req_o ( mr_prng_reseed_req[i] ), // OR-combine
179 .prng_reseed_ack_i ( prng_reseed_ack_i ),
180 .state_sel_o ( mr_state_sel[i] ), // OR-combine
181 .state_we_o ( sp_state_we[i] ), // Sparsified
182 .sub_bytes_en_o ( sp_sub_bytes_en[i] ), // Sparsified
183 .sub_bytes_out_req_i ( sp_sub_bytes_out_req[i] ), // Sparsified
184 .sub_bytes_out_ack_o ( sp_sub_bytes_out_ack[i] ), // Sparsified
185 .add_rk_sel_o ( mr_add_rk_sel[i] ), // OR-combine
186 .key_full_sel_o ( mr_key_full_sel[i] ), // OR-combine
187 .key_full_we_o ( sp_key_full_we[i] ), // Sparsified
188 .key_dec_sel_o ( mr_key_dec_sel[i] ), // OR-combine
189 .key_dec_we_o ( sp_key_dec_we[i] ), // Sparsified
190 .key_expand_en_o ( sp_key_expand_en[i] ), // Sparsified
191 .key_expand_out_req_i ( sp_key_expand_out_req[i] ), // Sparsified
192 .key_expand_out_ack_o ( sp_key_expand_out_ack[i] ), // Sparsified
193 .key_expand_clear_o ( mr_key_expand_clear[i] ), // OR-combine
194 .rnd_ctr_o ( mr_rnd_ctr[i] ), // OR-combine
195 .key_words_sel_o ( mr_key_words_sel[i] ), // OR-combine
196

```

```

( mr_key_words_sel[i] ), // OR-combine
203 .round_key_sel_o
( mr_round_key_sel[i] ), // OR-combine
204
205 .crypt_q_i
( sp_crypt_q[i] ), // Sparsified
206 .crypt_d_o
( sp_crypt_d[i] ), // Sparsified
207 .dec_key_gen_q_i
( sp_dec_key_gen_q[i] ), // Sparsified
208 .dec_key_gen_d_o
( sp_dec_key_gen_d[i] ), // Sparsified
209 .prng_reseed_q_i ( prng_reseed_q ),
210 .prng_reseed_d_o
( mr_prng_reseed_d[i] ), // AND-combine
211 .key_clear_q_i ( key_clear_q ),
212 .key_clear_d_o
( mr_key_clear_d[i] ), // AND-combine
213 .data_out_clear_q_i ( data_out_clear_q ),
214 .data_out_clear_d_o
( mr_data_out_clear_d[i] ) // AND-combine
215 );
216 end else begin : gen_fsm_n
217   aes_cipher_control_fsm_n #(
218     .SecMasking ( SecMasking ),
219     .SecSBoxImpl ( SecSBoxImpl ),
220     ) u_aes_cipher_control_fsm_i (
221     .clk_i ( clk_i ),
222     .rst_ni ( rst_ni ),
223
224     .in_valid_ni ( sp_in_val-
id[i] ), // Sparsified
225     .in_ready_no
( sp_in_ready[i] ), // Sparsified
226
227     .out_valid_no ( sp_out_val-
id[i] ), // Sparsified
228     .out_ready_ni
( sp_out_ready[i] ), // Sparsified
229
230     .cfg_valid_i ( cfg_valid_i ),
231     .op_i ( op_i ),
232     .key_len_i ( key_len_i ),
233     .crypt_ni
( sp_crypt[i] ), // Sparsified
234     .dec_key_gen_ni
( sp_dec_key_gen[i] ), // Sparsified
235     .prng_reseed_i ( prng_reseed_i ),
236     .key_clear_i ( key_clear_i ),
237     .data_out_clear_i ( data_out_clear_i ),
238     .mux_sel_err_i ( mux_sel_err ),
239     .sp_enc_err_i ( sp_enc_err ),
240     .rnd_ctr_err_i ( rnd_ctr_err ),
241     .op_err_i ( op_err_i ),
242     .alert_fatal_i ( alert_fatal_i ),
243     .alert_o
( mr_alert[i] ), // OR-combine
244
245     .prng_update_o
( mr_prng_update[i] ), // OR-combine
246     .prng_reseed_req_o
( mr_prng_reseed_req[i] ), // OR-combine
247     .prng_reseed_ack_i ( prng_reseed_ack_i ),
248
249     .state_sel_o
( mr_state_sel[i] ), // OR-combine
250     .state_we_no
( sp_state_we[i] ), // Sparsified
251     .sub_bytes_en_no
( sp_sub_bytes_en[i] ), // Sparsified
252     .sub_bytes_out_req_ni
( sp_sub_bytes_out_req[i] ), // Sparsified
253     .sub_bytes_out_ack_no
( sp_sub_bytes_out_ack[i] ), // Sparsified
254     .add_rk_sel_o
( mr_add_rk_sel[i] ), // OR-combine
255
256     .key_full_sel_o
( mr_key_full_sel[i] ), // OR-combine
257     .key_full_we_no
( sp_key_full_we[i] ), // Sparsified
258     .key_dec_sel_o
( mr_key_dec_sel[i] ), // OR-combine
259     .key_dec_we_no
( sp_key_dec_we[i] ), // Sparsified
260     .key_expand_en_no ( sp_key-ex-
pand_en[i] ), // Sparsified
261     .key_expand_out_req_ni ( sp_key-ex-
pand_out_req[i] ), // Sparsified
262     .key_expand_out_ack_no ( sp_key-ex-
pand_out_ack[i] ), // Sparsified
263     .key_expand_clear_o ( mr_key-ex-
pand_clear[i] ), // OR-combine
264     .rnd_ctr_o
( mr_rnd_ctr[i] ), // OR-combine
265     .key_words_sel_o
( mr_key_words_sel[i] ), // OR-combine
266     .round_key_sel_o
( mr_round_key_sel[i] ), // OR-combine
267
268     .crypt_q_ni

```

```

( sp_crypt_q[i] ), // Sparsified
269 .crypt_d_no
( sp_crypt_d[i] ), // Sparsified
270 .dec_key_gen_q_ni
( sp_dec_key_gen_q[i] ), // Sparsified
271 .dec_key_gen_d_no
( sp_dec_key_gen_d[i] ), // Sparsified
272 .prng_reseed_q_i ( prng_reseed_q ),
273 .prng_reseed_d_o
( mr_prng_reseed_d[i] ), // AND-combine
274 .key_clear_q_i ( key_clear_q ),
275 .key_clear_d_o
( mr_key_clear_d[i] ), // AND-combine
276 .data_out_clear_q_i ( data_out_clear_q ),
277 .data_out_clear_d_o
( mr_data_out_clear_d[i] ) // AND-combine
278 );
279 end
280 end
281
282 // Convert sparsified outputs to sp2v_e type.
283 assign in_ready_o = sp2v_e'(sp_in_ready);
284 assign out_valid_o = sp2v_e'(sp_out_valid);
285 assign state_we_o = sp2v_e'(sp_state_we);
286 assign sub_bytes_en_o = sp2v_e'(sp_sub_bytes_en);
287 assign sub_bytes_out_ack_o = sp2v_e'(sp_sub_bytes_out_ack);
288 assign key_full_we_o = sp2v_e'(sp_key_full_we);
289 assign key_dec_we_o = sp2v_e'(sp_key_dec_we);
290 assign key_expand_en_o = sp2v_e'(sp_key_expand_en);
291 assign key_expand_out_ack_o = sp2v_e'(sp_key-ex-
pand_out_ack);
292 assign crypt_d = sp2v_e'(sp_crypt_d);
293 assign dec_key_gen_d = sp2v_e'(sp_dec_key_gen_d);
294
295 // Combine single-bit FSM outputs.
296 // OR: One bit is sufficient to drive the corresponding out-
put bit high.
297 assign alert_o = |mr_alert;
298 assign prng_update_o = |mr_prng_update;
299 assign prng_reseed_req_o = |mr_prng_reseed_req;
300 assign key_expand_clear_o = |mr_key_expand_clear;
301 // AND: Only if all bits are high, the corresponding status
is signaled which will lead to
302 // the clearing of these trigger bits.
303 assign prng_reseed_d = &mr_prng_reseed_d;
304 assign key_clear_d = &mr_key_clear_d;
305 assign data_out_clear_d = &mr_data_out_clear_d;
306
307 // Combine multi-bit, sparse FSM outputs. We simply OR them
together. If the FSMs don't provide
308 // the same outputs, two cases are possible:
309 // - An invalid encoding results: A downstream checker will
fire, see mux_sel_err_i.
310 // - A valid encoding results: The outputs are compared be-
low to cover this case, see mr_err;
311 always_comb begin : combine_sparse_signals
312   state_sel_o = state_sel_e'({StateSelWidth{1'b0}});
313   add_rk_sel_o = add_rk_sel_e'({AddRKSelWidth{1'b0}});
314   key_full_sel_o =
key_full_sel_e'({KeyFullSelWidth{1'b0}});
315   key_dec_sel_o = key_dec_sel_e'({KeyDecSelWidth{1'b0}});
316   key_words_sel_o =
key_words_sel_e'({KeyWordsSelWidth{1'b0}});
317   round_key_sel_o =
round_key_sel_e'({RoundKeySelWidth{1'b0}});
318   mr_err = 1'b0;
319
320   for (int i = 0; i < Sp2VWidth; i++) begin
321     state_sel_o = state_sel_e'({state_sel_o} |
{mr_state_sel[i]});
322     add_rk_sel_o = add_rk_sel_e'({add_rk_sel_o} |
{mr_add_rk_sel[i]});
323     key_full_sel_o = key_full_sel_e'({key_full_sel_o} |
{mr_key_full_sel[i]});
324     key_dec_sel_o = key_dec_sel_e'({key_dec_sel_o} |
{mr_key_dec_sel[i]});
325     key_words_sel_o = key_words_sel_e'({key_words_sel_o} |
{mr_key_words_sel[i]});
326     round_key_sel_o = round_key_sel_e'({round_key_sel_o} |
{mr_round_key_sel[i]});
327   end
328
329   for (int i = 0; i < Sp2VWidth; i++) begin
330     if (state_sel_o != mr_state_sel[i] ||
331         add_rk_sel_o != mr_add_rk_sel[i] ||
332         key_full_sel_o != mr_key_full_sel[i] ||
333         key_dec_sel_o != mr_key_dec_sel[i] ||
334         key_words_sel_o != mr_key_words_sel[i] ||
335         round_key_sel_o != mr_round_key_sel[i]) begin
336       mr_err = 1'b1;
337     end
338   end
339 end
340
341 // Collect errors in mux selector signals.
342 assign mux_sel_err = mux_sel_err_i | mr_err;
343
344 // Combine counter signals. We simply OR them together. If
the FSMs don't provide the same
345 // outputs, rnd_ctr_err will be set.
346 always_comb begin : combine_counter_signals

```

```

347   rnd_ctr      = '0;
348   rnd_ctr_err  = 1'b0;
349   for (int i = 0; i < Sp2VWidth; i++) begin
350     rnd_ctr[i] = mr_rnd_ctr[i];
351   end
352
353   for (int i = 0; i < Sp2VWidth; i++) begin
354     if (rnd_ctr != mr_rnd_ctr[i]) begin
355       rnd_ctr_err = 1'b1;
356     end
357   end
358 end
359
360 always_ff @(posedge clk_i or negedge rst_ni) begin : reg_fsm
361   if (!rst_ni) begin
362     prng_reseed_q <= 1'b0;
363     key_clear_q   <= 1'b0;
364     data_out_clear_q <= 1'b0;
365   end else begin
366     prng_reseed_q <= prng_reseed_d;
367     key_clear_q   <= key_clear_d;
368     data_out_clear_q <= data_out_clear_d;
369   end
370 end
371
372 // Use separate signal for key expand operation, forward
round.
373   assign key_expand_op_o = (dec_key_gen_d == SP2V_HIGH ||
374     dec_key_gen_q == SP2V_HIGH) ||
CiphOpFwdOnly ? CIPH_FWD : op_i;
375   assign key_expand_round_o = rnd_ctr;
376
377   // Let the main controller know what we are doing.
378   assign crypt_o = crypt_q;
379   assign dec_key_gen_o = dec_key_gen_q;
380   assign prng_reseed_o = prng_reseed_q;
381   assign key_clear_o = key_clear_q;
382   assign data_out_clear_o = data_out_clear_q;
383
384   ///////////////////////////////////////////////////
385   // Sparsely Encoded Signals //
386   ///////////////////////////////////////////////////
387   // SEC_CM: CTRL.SPARSE
388   // We use sparse encodings for various critical signals and
must ensure that:
391   // 1. The synthesis tool doesn't optimize away the sparse
encoding.
392   // 2. The sparsely encoded signal is always valid. More pre-
cisely, an alert or SVA is triggered
393   // if a sparse signal takes on an invalid value.
394   // 3. The alert signal remains asserted until reset even if
the sparse signal becomes valid again
395   // This is achieved by driving the control FSM into the
terminal error state whenever any
396   // sparsely encoded signal becomes invalid.
397   //
398   // If any sparsely encoded signal becomes invalid, the
cipher core further immediately de-asserts
399   // the out_valid_o signal to prevent any data from being re-
leased.
400
401   // The following primitives are used to place a size-only
constraint on the
402   // flops in order to prevent optimizations on these status
signals.
403   logic [Sp2VWidth-1:0] crypt_q_raw;
404   prim_flop #(
405     .Width      ( Sp2VWidth ),
406     .ResetValue ( Sp2VWidth'(SP2V_LOW) )
407   ) u_crypt_regs (
408     .clk_i ( clk_i ),
409     .rst_ni ( rst_ni ),
410     .d_i   ( crypt_d ),
411     .q_o   ( crypt_q_raw )
412   );
413
414   logic [Sp2VWidth-1:0] dec_key_gen_q_raw;
415   prim_flop #(
416     .Width      ( Sp2VWidth ),
417     .ResetValue ( Sp2VWidth'(SP2V_LOW) )
418   ) u_dec_key_gen_regs (
419     .clk_i ( clk_i ),
420     .rst_ni ( rst_ni ),
421     .d_i   ( dec_key_gen_d ),
422     .q_o   ( dec_key_gen_q_raw )
423   );
424
425   // We use vectors of sparsely encoded signals to reduce code
duplication.
426   localparam int unsigned NumSp2VSig = 8;
427   sp2v_e [NumSp2VSig-1:0] sp2v_sig;
428   sp2v_e [NumSp2VSig-1:0] sp2v_sig_chk;
429   logic [NumSp2VSig-1:0][Sp2VWidth-1:0] sp2v_sig_chk_raw;
430   logic [NumSp2VSig-1:0] sp2v_sig_err;
431
432   assign sp2v_sig[0] = in_valid_i;
433   assign sp2v_sig[1] = out_ready_i;
434   assign sp2v_sig[2] = crypt_i;

```

```

435   assign sp2v_sig[3] = dec_key_gen_i;
436   assign sp2v_sig[4] = sp2v_e'(crypt_q_raw);
437   assign sp2v_sig[5] = sp2v_e'(dec_key_gen_q_raw);
438   assign sp2v_sig[6] = sub_bytes_out_req_i;
439   assign sp2v_sig[7] = key_expand_out_req_i;
440
441   // All signals inside sp2v_sig except for sub_bytes/key_ex-
pand_out_req_i are driven and consumed
442   // by multi-rail FSMs.
443   localparam bit [NumSp2VSig-1:0] Sp2VEnSecBuf = 8'b1100_0000;
444
445   // Individually check sparsely encoded signals.
446   for (genvar i = 0; i < NumSp2VSig; i++) begin :
gen_sel_buf_chk
447     aes_sel_buf_chk #(
448       .Num      ( Sp2VNum ),
449       .Width    ( Sp2VWidth ),
450       .EnSecBuf ( Sp2VEnSecBuf[i] )
451     ) u_aes_sp2v_sig_buf_chk_i (
452       .clk_i ( clk_i ),
453       .rst_ni ( rst_ni ),
454       .sel_i ( sp2v_sig[i] ),
455       .sel_o ( sp2v_sig_chk_raw[i] ),
456       .err_o ( sp2v_sig_err[i] )
457     );
458     assign sp2v_sig_chk[i] = sp2v_e'(sp2v_sig_chk_raw[i]);
459   end
460
461   assign in_valid      = sp2v_sig_chk[0];
462   assign out_ready     = sp2v_sig_chk[1];
463   assign crypt         = sp2v_sig_chk[2];
464   assign dec_key_gen   = sp2v_sig_chk[3];
465   assign crypt_q       = sp2v_sig_chk[4];
466   assign dec_key_gen_q = sp2v_sig_chk[5];
467   assign sub_bytes_out_req = sp2v_sig_chk[6];
468   assign key_expand_out_req = sp2v_sig_chk[7];
469
470   // Collect encoding errors.
471   // We instantiate the checker modules as close as possible
to where the sparsely encoded signals
472   // are used. Here, we collect also encoding errors detected
in other places of the cipher core.
473   assign sp_enc_err = |sp2v_sig_err | sp_enc_err_i;
474
475   ///////////////////////////////////////////////////
476   // Assertions //
477   ///////////////////////////////////////////////////
478   // Selectors must be known/valid
479   `ASSERT(AesCiphOpValid, cfg_valid_i |> op_i inside {
480     CIPH_FWD,
481     CIPH_INV
482   })
483
484
485 endmodule

```

3. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm.sv (515 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core control FSM
6 //
7 // This module contains the AES cipher core control FSM.
8
9 `include "prim_assert.sv"
10
11 module aes_cipher_control_fsm import aes_pkg::*;
12 #(
13   parameter bit          SecMasking = 0,
14   parameter sbbox_impl_e SecSBoxImpl = SBoxImplDom
15 ) (
16   input logic             clk_i,
17   input logic             rst_ni,
18
19   // Input handshake signals
20   input logic             in_valid_i,           // Sparsify
using multi-rail.
21   output logic            in_ready_o,          // Sparsify
using multi-rail.
22
23   // Output handshake signals
24   output logic            out_valid_o,         // Sparsify
using multi-rail.
25   input logic             out_ready_i,        // Sparsify
using multi-rail.
26
27   // Control and sync signals
28   input logic             cfg_valid_i,         // Used for
SVAs only.
29   input ciph_op_e         op_i,
30   input key_len_e         key_len_i,
31   input logic             crypt_i,           // Sparsify
using multi-rail.
32   input logic             dec_key_gen_i,      // Sparsify
using multi-rail.

```



```

33 input logic prng_reseed_i,
34 input logic key_clear_i,
35 input logic data_out_clear_i,
36 input logic mux_sel_err_i,
37 input logic sp_enc_err_i,
38 input logic rnd_ctr_err_i,
39 input logic op_err_i,
40 input logic alert_fatal_i,
41 output logic alert_o,
42
43 // Control signals for masking PRNG
44 output logic prng_update_o,
45 output logic prng_reseed_req_o,
46 input logic prng_reseed_ack_i,
47
48 // Control and sync signals for cipher data path
49 output state_sel_e state_sel_o,
50 output logic state_we_o, // Sparsify
using multi-rail.
51 output logic sub_bytes_en_o, // Sparsify
using multi-rail.
52 input logic sub_bytes_out_req_i, // Sparsify
using multi-rail.
53 output logic sub_bytes_out_ack_o, // Sparsify
using multi-rail.
54 output add_rk_sel_e add_rk_sel_o,
55
56 // Control and sync signals for key expand data path
57 output key_full_sel_e key_full_sel_o,
58 output logic key_full_we_o, // Sparsify
using multi-rail.
59 output key_dec_sel_e key_dec_sel_o,
60 output logic key_dec_we_o, // Sparsify
using multi-rail.
61 output logic key_expand_en_o, // Sparsify
using multi-rail.
62 input logic key_expand_out_req_i, // Sparsify
using multi-rail.
63 output logic key_expand_out_ack_o, // Sparsify
using multi-rail.
64 output logic key_expand_clear_o,
65 output key_words_sel_e rnd_ctr_o,
66 output key_words_sel_e key_words_sel_o,
67 output round_key_sel_e round_key_sel_o,
68
69 // Register signals
70 input logic crypt_q_i, // Sparsify
using multi-rail.
71 output logic crypt_d_o, // Sparsify
using multi-rail.
72 input logic dec_key_gen_q_i, // Sparsify
using multi-rail.
73 output logic dec_key_gen_d_o, // Sparsify
using multi-rail.
74 input logic prng_reseed_q_i,
75 output logic prng_reseed_d_o,
76 input logic key_clear_q_i,
77 output logic key_clear_d_o,
78 input logic data_out_clear_q_i,
79 output logic data_out_clear_d_o
80 );
81
82 // cfg_valid_i is used for SVAs only.
83 logic unused_cfg_valid;
84 assign unused_cfg_valid = cfg_valid_i;
85
86 // Tie off unused inputs.
87 if (!SecMasking) begin : gen_unused_prng_reseed
88 logic unused_prng_reseed;
89 assign unused_prng_reseed = prng_reseed_i;
90 end
91
92 // Signals
93 aes_cipher_ctrl_e aes_cipher_ctrl_ns, aes_cipher_ctrl_cs;
94 logic advance;
95 logic [2:0] cyc_ctr_d, cyc_ctr_q;
96 logic cyc_ctr_expr;
97 logic prng_reseed_done_d, prng_reseed_done_q;
98 logic [3:0] rnd_ctr_d, rnd_ctr_q;
99 logic [3:0] num_rounds_d, num_rounds_q;
100 logic [3:0] num_rounds_regular;
101
102 // Use separate signal for number of regular rounds.
103 assign num_rounds_regular = num_rounds_q - 4'd1;
104
105 // FSM
106 always_comb begin : aes_cipher_ctrl_fsm
107
108 // Handshake signals
109 in_ready_o = 1'b0;
110 out_valid_o = 1'b0;
111
112 // Masking PRNG signals
113 prng_update_o = 1'b0;
114 prng_reseed_req_o = 1'b0;
115
116 // Cipher data path
117 state_sel_o = STATE_ROUND;
118 state_we_o = 1'b0;
119 add_rk_sel_o = ADD_RK_ROUND;
120 sub_bytes_en_o = 1'b0;
121
122 sub_bytes_out_ack_o = 1'b0;
123
124 // Key expand data path
125 key_full_sel_o = KEY_FULL_ROUND;
126 key_full_we_o = 1'b0;
127 key_dec_sel_o = KEY_DEC_EXPAND;
128 key_dec_we_o = 1'b0;
129 key_expand_en_o = 1'b0;
130 key_expand_out_ack_o = 1'b0;
131 key_expand_clear_o = 1'b0;
132 key_words_sel_o = KEY_WORDS_ZERO;
133 round_key_sel_o = ROUND_KEY_DIRECT;
134
135 // FSM
136 aes_cipher_ctrl_ns = aes_cipher_ctrl_cs;
137 num_rounds_d = num_rounds_q;
138 rnd_ctr_d = rnd_ctr_q;
139 crypt_d_o = crypt_q_i;
140 dec_key_gen_d_o = dec_key_gen_q_i;
141 prng_reseed_d_o = prng_reseed_q_i;
142 key_clear_d_o = key_clear_q_i;
143 data_out_clear_d_o = data_out_clear_q_i;
144 prng_reseed_done_d = prng_reseed_done_q | prng_reseed_ack_i;
145 advance = 1'b0;
146 cyc_ctr_q + 3'd1 : 3'd0;
147
148 // Alert
149 alert_o = 1'b0;
150
151 unique case (aes_cipher_ctrl_cs)
152 CIPHER_CTRL_IDLE: begin
153 cyc_ctr_d = 3'd0;
154
155 // Signal that we are ready, wait for handshake.
156 in_ready_o = 1'b1;
157 if (in_valid_i) begin
158 if (SecMasking && prng_reseed_i && !dec_key_gen_i
&& !crypt_i) begin
159 // Reseed the masking PRNG without starting en-
160 // cryptation/decryption or generation of the
161 // start key for decryption.
162 prng_reseed_d_o = 1'b1;
163 prng_reseed_done_d = 1'b0;
164 aes_cipher_ctrl_ns = CIPHER_CTRL_PRNG_RESEED;
165
166 end else if (key_clear_i || data_out_clear_i) begin
167 // Clear internal key registers. The cipher core
168 // muxes are used to clear the data
169 // output registers.
170 key_clear_d_o = key_clear_i;
171 data_out_clear_d_o = data_out_clear_i;
172
173 // To clear the data output registers, we must
174 // first clear the state.
175 aes_cipher_ctrl_ns = data_out_clear_i ?
176 CIPHER_CTRL_CLEAR_S : CIPHER_CTRL_CLEAR_KD;
177
178 end else if (dec_key_gen_i || crypt_i) begin
179 // Start encryption/decryption or generation of
180 // start key for decryption.
181 crypt_d_o = ~dec_key_gen_i & crypt_i;
182 dec_key_gen_d_o = dec_key_gen_i;
183
184 // Latch whether we shall reseed the masking PRNG.
185 prng_reseed_d_o = SecMasking & prng_reseed_i;
186
187 // Load input data to state
188 state_sel_o = dec_key_gen_i ? STATE_CLEAR :
189 STATE_INIT;
190
191 state_we_o = 1'b1;
192
193 // Make the masking PRNG advance. The current
194 // pseudo-random data is used to mask the
195 // input data.
196 prng_update_o = SecMasking;
197
198 // Init key expand
199 key_expand_clear_o = 1'b1;
200
201 // Load full key
202 key_full_sel_o = dec_key_gen_i ?
203 KEY_FULL_ENC_INIT :
204 (op_i == CIPH_FWD) ?
205 KEY_FULL_ENC_INIT :
206 (op_i == CIPH_INV) ?
207 KEY_FULL_DEC_INIT :
208 KEY_FULL_ENC_INIT;
209
210 key_full_we_o = 1'b1;
211
212 // Load num_rounds, initialize round counter.
213 num_rounds_d = (key_len_i == AES_128) ? 4'd10 :
214 (key_len_i == AES_192) ? 4'd12 :
215 4'd14;
216
217 rnd_ctr_d = '0;
218 aes_cipher_ctrl_ns = CIPHER_CTRL_INIT;
219
220 end else begin

```

```

208      // Handshake without a valid command. We should
never get here. If we do (e.g. via a
209      // malicious glitch), error out immediately.
210      aes_cipher_ctrl_ns = CIPHER_CTRL_ERROR;
211      end
212    end
213  end
214
215  CIPHER_CTRL_INIT: begin
216    // Initial round: just add key to state
217    add_rk_sel_o = ADD_RK_INIT;
218
219    // Select key words for initial add_round_key
220    key_words_sel_o = (dec_key_gen_q_i) ?
KEY_WORDS_ZERO :
221    (key_len_i == AES_128) ?
KEY_WORDS_0123 :
222    (key_len_i == AES_192 && op_i == CIPH_FWD) ?
KEY_WORDS_0123 :
223    (key_len_i == AES_192 && op_i == CIPH_INV) ?
KEY_WORDS_2345 :
224    (key_len_i == AES_256 && op_i == CIPH_FWD) ?
KEY_WORDS_0123 :
225    (key_len_i == AES_256 && op_i == CIPH_INV) ?
KEY_WORDS_4567 : KEY_WORDS_ZERO;
226
227    // Clear masking PRNG reseed status.
228    prng_reseed_done_d = 1'b0;
229
230    // AES-256 has two round keys available right from be-
ginning. Pseudo-random data is
231    // required by KeyExpand only.
232    if (key_len_i != AES_256) begin
233      // Advance in sync with KeyExpand. Based on the S-
Box implementation, it can take
234      // multiple cycles to finish. Wait for handshake.
The DOM S-Boxes consume fresh PRD
235      // only in the first clock cycle and that PRD is
taken from the buffer stage updated
236      // based on key_full_we_o. The PRNG itself is up-
dated in every clock cycle to increase
237      // the noise.
238      advance = key_expand_out_req_i & cyc_ctr_ex-
pr;
239      prng_update_o = SecMasking;
240      key_expand_en_o = 1'b1;
241      if (advance) begin
242        key_expand_out_ack_o = 1'b1;
243        state_we_o = ~dec_key_gen_q_i;
244        key_full_we_o = 1'b1;
245        rnd_ctr_d = rnd_ctr_q + 4'b0001;
246        cyc_ctr_d = 3'd0;
247        aes_cipher_ctrl_ns = CIPHER_CTRL_ROUND;
248      end
249      end else begin
250        prng_update_o = SecMasking;
251        state_we_o = ~dec_key_gen_q_i;
252        rnd_ctr_d = rnd_ctr_q + 4'b0001;
253        cyc_ctr_d = 3'd0;
254        aes_cipher_ctrl_ns = CIPHER_CTRL_ROUND;
255      end
256    end
257
258    CIPHER_CTRL_ROUND: begin
259      // Normal rounds
260
261      // Select key words for add_round_key
262      key_words_sel_o = (dec_key_gen_q_i) ?
KEY_WORDS_ZERO :
263      (key_len_i == AES_128) ?
KEY_WORDS_0123 :
264      (key_len_i == AES_192 && op_i == CIPH_FWD) ?
KEY_WORDS_2345 :
265      (key_len_i == AES_192 && op_i == CIPH_INV) ?
KEY_WORDS_0123 :
266      (key_len_i == AES_256 && op_i == CIPH_FWD) ?
KEY_WORDS_4567 :
267      (key_len_i == AES_256 && op_i == CIPH_INV) ?
KEY_WORDS_0123 : KEY_WORDS_ZERO;
268
269      // Keep requesting PRNG reseeding until it is acknow-
ledged.
270      prng_reseed_req_o = SecMasking & prng_reseed_q_i &
~prng_reseed_done_q;
271
272      // Select round key: direct or mixed (equivalent in-
verse cipher)
273      round_key_sel_o = (op_i == CIPH_FWD) ?
ROUND_KEY_DIRECT :
274      (op_i == CIPH_INV) ?
ROUND_KEY_MIXED : ROUND_KEY_DIRECT;
275
276      // Advance in sync with SubBytes and KeyExpand. Based
on the S-Box implementation, both can
277      // take multiple cycles to finish. Wait for handshake.
The DOM S-Boxes consume fresh PRD
278      // only in the first clock cycle and that PRD is taken
from the buffer stages updated
279      // with state_we_o / based on key_full_we_o. The PRNG

```

```

itself is updated in every clock
280      // cycle to increase the noise.
281      advance = key_expand_out_req_i & cyc_ctr_expr &
(dec_key_gen_q_i | sub_bytes_out_req_i);
282      prng_update_o = SecMasking;
283      sub_bytes_en_o = ~dec_key_gen_q_i;
284      key_expand_en_o = 1'b1;
285
286      if (advance) begin
287        sub_bytes_out_ack_o = ~dec_key_gen_q_i;
288        key_expand_out_ack_o = 1'b1;
289
290        state_we_o = ~dec_key_gen_q_i;
291        key_full_we_o = 1'b1;
292
293        // Update round
294        rnd_ctr_d = rnd_ctr_q + 4'b0001;
295        cyc_ctr_d = 3'd0;
296
297        // Are we doing the last regular round?
298        if (rnd_ctr_q >= num_rounds_regular) begin
299          aes_cipher_ctrl_ns = CIPHER_CTRL_FINISH;
300
301          if (dec_key_gen_q_i) begin
302            // Write decryption key.
303            key_dec_we_o = 1'b1;
304
305            // Indicate that we are done, try to perform the
handshake. But we don't wait here.
306            // If we don't get the handshake now, we will
wait in the finish state. When using
307            // masking, we only finish if the masking PRNG
has been reseeded.
308            out_valid_o = SecMasking ? (prng_reseed_q_i ?
prng_reseed_done_q : 1'b1) : 1'b1;
309            if (out_valid_o && out_ready_i) begin
310              // Go to idle state directly.
311              dec_key_gen_d_o = 1'b0;
312              prng_reseed_d_o = 1'b0;
313              aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
314            end
315            end // rnd_ctr_q
316            end // SubBytes/KeyExpand REQ/ACK
317            end
318
319            CIPHER_CTRL_FINISH: begin
320              // Final round
321
322              // Select key words for add_round_key
323              key_words_sel_o = (dec_key_gen_q_i) ?
KEY_WORDS_ZERO :
324              (key_len_i == AES_128) ?
KEY_WORDS_0123 :
325              (key_len_i == AES_192 && op_i == CIPH_FWD) ?
KEY_WORDS_2345 :
326              (key_len_i == AES_192 && op_i == CIPH_INV) ?
KEY_WORDS_0123 :
327              (key_len_i == AES_256 && op_i == CIPH_FWD) ?
KEY_WORDS_4567 :
328              (key_len_i == AES_256 && op_i == CIPH_INV) ?
KEY_WORDS_0123 : KEY_WORDS_ZERO;
329
330              // Skip mix_columns
331              add_rk_sel_o = ADD_RK_FINAL;
332
333              // Keep requesting PRNG reseeding until it is acknow-
ledged.
334              prng_reseed_req_o = SecMasking & prng_reseed_q_i &
~prng_reseed_done_q;
335
336              // SEC_CM: DATA_REG.KEY.SCA
337              // Once we're done, we won't need the state anymore.
We actually clear it when progressing
338              // to the next state.
339              state_sel_o = STATE_CLEAR;
340
341              // SEC_CM: DATA_REG.LOCAL_ESC
342              // Advance in sync with SubBytes. Based on the S-Box
implementation, it can take multiple
343              // cycles to finish. Only indicate that we are done
if:
344              // - we have valid output (SubBytes finished),
345              // - the masking PRNG has been reseeded (if masking is
used),
346              // - all mux selector signals are valid (don't release
data in case of errors), and
347              // - all sparsely encoded signals are valid (don't re-
lease data in case of errors).
348              // Perform both handshakes simultaneously.
349              advance = (sub_bytes_out_req_i & cyc_ctr_expr)
| dec_key_gen_q_i;
350              sub_bytes_en_o = ~dec_key_gen_q_i;
351              out_valid_o = (mux_sel_err_i || sp_enc_err_i ||
op_err_i) ? 1'b0 :
352              SecMasking ? (prng_reseed_q_i ? prng_reseed_done_q &
advance : advance) : advance;

```

```

354
355 // Stop updating the cycle counter once we have valid
output.
356   cyc_ctr_d =
357     (SecSBoxImpl == SBoxImplDom) ? (!advance ?
cyc_ctr_q + 3'd1 : cyc_ctr_q) : 3'd0;
358
359 // The DOM S-Boxes consume fresh PRD only in the first
clock cycle and that PRD is taken
360 // from the buffer stages updated with state_we_o /
based on key_full_we_o. The PRNG itself
361 // is updated in every but the last processing clock
cycle to increase the noise. Once the
362 // processing is all done (e.g. if we're just waiting
for the PRNG reseeding to finish or
363 // if we're waiting for out_ready_i), the PRNG is no
longer updated to save power. In the
364 // very last clock cycle, we update the PRNG again to
get ready for the next block.
365   prng_update_o =
366     ((SecSBoxImpl == SBoxImplDom) ? !advance : 1'b0) |
(out_valid_o & out_ready_i);
367
368   if (out_valid_o && out_ready_i) begin
369     sub_bytes_out_ack_o = ~dec_key_gen_q_i;
370
371     // Clear the state.
372     state_we_o = 1'b1;
373     crypt_d_o = 1'b0;
374     cyc_ctr_d = 3'd0;
375     // If we were generating the decryption key and
didn't get the handshake in the last
376     // regular round, we should clear dec_key_gen now.
377     dec_key_gen_d_o = 1'b0;
378     prng_reseed_d_o = 1'b0;
379     aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
380   end
381 end
382
383 CIPHER_CTRL_PRNG_RESEED: begin
384 // Keep requesting PRNG reseeding until it is acknow-
ledged.
385   prng_reseed_req_o = prng_reseed_q_i & ~prng_re-
seed_done_q;
386
387 // Don't update the cycle counter as we don't need it.
388   cyc_ctr_d = 3'd0;
389
390 // Once we're done, wait for handshake.
391   out_valid_o = prng_reseed_done_q;
392   if (out_valid_o && out_ready_i) begin
393     prng_reseed_d_o = 1'b0;
394     aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
395   end
396 end
397
398 CIPHER_CTRL_CLEAR_S: begin
399 // Clear the state with pseudo-random data.
400   state_we_o = 1'b1;
401   state_sel_o = STATE_CLEAR;
402   aes_cipher_ctrl_ns = CIPHER_CTRL_CLEAR_KD;
403 end
404
405 CIPHER_CTRL_CLEAR_KD: begin
406 // Clear internal key registers and/or external data
output registers.
407   if (key_clear_q_i) begin
408     key_full_sel_o = KEY_FULL_CLEAR;
409     key_full_we_o = 1'b1;
410     key_dec_sel_o = KEY_DEC_CLEAR;
411     key_dec_we_o = 1'b1;
412   end
413   if (data_out_clear_q_i) begin
414 // Forward the state (previously cleared with
pseudo-random data).
415     // SEC_CM: DATA_REG.SEC_WIPE
416     add_rk_sel_o = ADD_RK_INIT;
417     key_words_sel_o = KEY_WORDS_ZERO;
418     round_key_sel_o = ROUND_KEY_DIRECT;
419   end
420 // Indicate that we are done, wait for handshake.
421   out_valid_o = 1'b1;
422   if (out_ready_i) begin
423     key_clear_d_o = 1'b0;
424     data_out_clear_d_o = 1'b0;
425     aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
426   end
427 end
428
429 CIPHER_CTRL_ERROR: begin
430 // SEC_CM: CIPHER.FSM.LOCAL_ESC
431 // Terminal error state
432   alert_o = 1'b1;
433 end
434
435 // We should never get here. If we do (e.g. via a mali-
cious glitch), error out immediately.
436   default: begin
437     aes_cipher_ctrl_ns = CIPHER_CTRL_ERROR;

```

```

438   alert_o = 1'b1;
439 end
440 endcase
441
442 // Unconditionally jump into the terminal error state in
case a mux selector or a sparsely
443 // encoded signal becomes invalid, in case we have detec-
ted a fault in the round counter,
444 // or if a fatal alert has been triggered.
445   if (mux_sel_err_i || sp_enc_err_i || rnd_ctr_err_i ||
op_err_i || alert_fatal_i) begin
446     aes_cipher_ctrl_ns = CIPHER_CTRL_ERROR;
447   end
448 end
449
450 // SEC_CM: CIPHER.FSM.SPARSE
451 `PRIM_FLOP_SPARSE_FSM(u_state_regs, aes_cipher_ctrl_ns,
aes_cipher_ctrl_cs, aes_cipher_ctrl_e, CIPHER_CTRL_IDLE)
452
453 always_ff @(posedge clk_i or negedge rst_ni) begin : reg_fsm
454   if (!rst_ni) begin
455     prng_reseed_done_q <= 1'b0;
456     rnd_ctr_q <= '0;
457     num_rounds_q <= '0;
458   end else begin
459     prng_reseed_done_q <= prng_reseed_done_d;
460     rnd_ctr_q <= rnd_ctr_d;
461     num_rounds_q <= num_rounds_d;
462   end
463 end
464
465 assign rnd_ctr_o = rnd_ctr_q;
466
467   if (SecSBoxImpl == SBoxImplDom) begin : gen_cyc_ctr
468     always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_cyc_ctr
469       if (!rst_ni) begin
470         cyc_ctr_q <= 3'd0;
471       end else begin
472         cyc_ctr_q <= cyc_ctr_d;
473       end
474     end
475     assign cyc_ctr_expr = cyc_ctr_q >= 3'd4;
476   end else begin : gen_no_cyc_ctr
477     logic [2:0] unused_cyc_ctr;
478     assign cyc_ctr_q = cyc_ctr_d;
479     assign unused_cyc_ctr = cyc_ctr_q;
480     assign cyc_ctr_expr = 1'b1;
481   end
482 end
483
484 // Assertions //
485 // Create a lint error to reduce the risk of accidentally
disabling the masking.
486 `AS-
487   SERT_STATIC_LINT_ERROR(AesCipherControlFsmSecMaskingNonDefault,
SecMasking == 1)
488 // Create a lint error to reduce the risk of accidentally
using a less secure SBox
implementation.
489 `AS-
490   SERT_STATIC_LINT_ERROR(AesCipherControlFsmSecSBoxImplNonDefault,
SecSBoxImpl == SBoxImplDom)
491 // Selectors must be known/valid
492 `ASSERT(AesCiphOpValid, cfg_valid_i |> op_i inside {
493   CIPH_FWD,
494   CIPH_INV
495 })
496 `ASSERT(AesKeyLenValid, cfg_valid_i |> key_len_i inside {
497   AES_128,
498   AES_192,
499   AES_256
500 })
501 `ASSERT(AesCipherControlStateValid, !alert_o |>
aes_cipher_ctrl_cs inside {
502   CIPHER_CTRL_IDLE,
503   CIPHER_CTRL_INIT,
504   CIPHER_CTRL_ROUND,
505   CIPHER_CTRL_FINISH,
506   CIPHER_CTRL_PRNG_RESEED,
507   CIPHER_CTRL_CLEAR_S,
508   CIPHER_CTRL_CLEAR_KD
509 })
510 endmodule

```

4. lowRISC_opentitan/hw/ip/aes/rtl/ aes_cipher_control_fsm_n.sv (400 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core control FSM

```



```

6 //
7 // This module contains the AES cipher core control FSM oper-
ating on and producing the negated
8 // values of important control signals. This is achieved by:
9 // - instantiating the regular AES cipher core control FSM op-
erating on and producing the positive
10 // values of these signals, and
11 // - inverting these signals between the regular FSM and the
prim_buf synthesis barriers.
12 // Synthesis tools will then push the inverters into the actu-
al FSM.
13
14 module aes_cipher_control_fsm_n import aes_pkg::*;
15 #(
16     parameter bit SecMasking = 0,
17     parameter sbbox_impl_e SecSBBoxImpl = SBBoxImplDom
18 ) (
19     input logic clk_i,
20     input logic rst_ni,
21
22     // Input handshake signals
23     input logic in_valid_ni, // Sparsify
using multi-rail.
24     output logic in_ready_no, // Sparsify
using multi-rail.
25
26     // Output handshake signals
27     output logic out_valid_no, // Sparsify
using multi-rail.
28     input logic out_ready_ni, // Sparsify
using multi-rail.
29
30     // Control and sync signals
31     input logic cfg_valid_i, // Used for
SVAs only.
32     input ciph_op_e op_i,
33     input key_len_e key_len_i,
34     input logic crypt_ni, // Sparsify
using multi-rail.
35     input logic dec_key_gen_ni, // Sparsify
using multi-rail.
36     input logic prng_reseed_i,
37     input logic key_clear_i,
38     input logic data_out_clear_i,
39     input logic mux_sel_err_i,
40     input logic sp_enc_err_i,
41     input logic rnd_ctr_err_i,
42     input logic op_err_i,
43     input logic alert_fatal_i,
44     output logic alert_o,
45
46     // Control signals for masking PRNG
47     output logic prng_update_o,
48     output logic prng_reseed_req_o,
49     input logic prng_reseed_ack_i,
50
51     // Control and sync signals for cipher data path
52     output state_sel_e state_sel_o,
53     output logic state_we_no, // Sparsify
using multi-rail.
54     output logic sub_bytes_en_no, // Sparsify
using multi-rail.
55     input logic sub_bytes_out_req_ni, // Sparsify
using multi-rail.
56     output logic sub_bytes_out_ack_no, // Sparsify
using multi-rail.
57     output add_rk_sel_e add_rk_sel_o,
58
59     // Control and sync signals for key expand data path
60     output key_full_sel_e key_full_sel_o,
61     output logic key_full_we_no, // Sparsify
using multi-rail.
62     output key_dec_sel_e key_dec_sel_o,
63     output logic key_dec_we_no, // Sparsify
using multi-rail.
64     output logic key_expand_en_no, // Sparsify
using multi-rail.
65     input logic key_expand_out_req_ni, // Sparsify
using multi-rail.
66     output logic key_expand_out_ack_no, // Sparsify
using multi-rail.
67     output logic key_expand_clear_o,
68     output logic [3:0] rnd_ctr_o,
69     output key_words_sel_e key_words_sel_o,
70     output round_key_sel_e round_key_sel_o,
71
72     // Register signals
73     input logic crypt_q_ni, // Sparsify
using multi-rail.
74     output logic crypt_d_no, // Sparsify
using multi-rail.
75     input logic dec_key_gen_q_ni, // Sparsify
using multi-rail.
76     output logic dec_key_gen_d_no, // Sparsify
using multi-rail.
77     input logic prng_reseed_q_i,
78     output logic prng_reseed_d_o,
79     input logic key_clear_q_i,
80     output logic key_clear_d_o,
81     input logic data_out_clear_q_i,
82     output logic data_out_clear_d_o

```

```

83 );
84
85 // Input Buffering //
86 // Input Buffering //
87 // Input Buffering //
88
89 localparam int NumInBufBits = $bits({
90     in_valid_ni,
91     out_ready_ni,
92     cfg_valid_i,
93     op_i,
94     key_len_i,
95     crypt_ni,
96     dec_key_gen_ni,
97     prng_reseed_i,
98     key_clear_i,
99     data_out_clear_i,
100     mux_sel_err_i,
101     sp_enc_err_i,
102     rnd_ctr_err_i,
103     op_err_i,
104     alert_fatal_i,
105     prng_reseed_ack_i,
106     sub_bytes_out_req_ni,
107     key_expand_out_req_ni,
108     crypt_q_ni,
109     dec_key_gen_q_ni,
110     prng_reseed_q_i,
111     key_clear_q_i,
112     data_out_clear_q_i
113 });
114
115 logic [NumInBufBits-1:0] in, in_buf;
116
117 assign in = {
118     in_valid_ni,
119     out_ready_ni,
120     cfg_valid_i,
121     op_i,
122     key_len_i,
123     crypt_ni,
124     dec_key_gen_ni,
125     prng_reseed_i,
126     key_clear_i,
127     data_out_clear_i,
128     mux_sel_err_i,
129     sp_enc_err_i,
130     rnd_ctr_err_i,
131     op_err_i,
132     alert_fatal_i,
133     prng_reseed_ack_i,
134     sub_bytes_out_req_ni,
135     key_expand_out_req_ni,
136     crypt_q_ni,
137     dec_key_gen_q_ni,
138     prng_reseed_q_i,
139     key_clear_q_i,
140     data_out_clear_q_i
141 };
142
143 // This primitive is used to place a size-only constraint on
the
144 // buffers to act as a synthesis optimization barrier.
145 prim_buf #(
146     .Width(NumInBufBits)
147 ) u_prim_buf_in (
148     .in_i(in),
149     .out_o(in_buf)
150 );
151
152 logic in_valid_n;
153 logic out_ready_n;
154 logic cfg_valid;
155 ciph_op_e op;
156 logic [$bits(op)-1:0] op_raw;
157 key_len_e key_len;
158 logic crypt_n;
159 logic dec_key_gen_n;
160 logic prng_reseed;
161 logic key_clear;
162 logic data_out_clear;
163 logic mux_sel_err;
164 logic sp_enc_err;
165 logic rnd_ctr_err;
166 logic op_err;
167 logic alert_fatal;
168 logic prng_reseed_ack;
169 logic sub_bytes_out_req_n;
170 logic key_expand_out_req_n;
171 logic crypt_q_n;
172 logic dec_key_gen_q_n;
173 logic prng_reseed_q;
174 logic key_clear_q;
175 logic data_out_clear_q;
176
177 assign {in_valid_n,
178     out_ready_n,
179     cfg_valid,
180     op_raw,
181     key_len,

```

```

182     crypt_n,
183     dec_key_gen_n,
184     prng_reseed,
185     key_clear,
186     data_out_clear,
187     mux_sel_err,
188     sp_enc_err,
189     rnd_ctr_err,
190     op_err,
191     alert_fatal,
192     prng_reseed_ack,
193     sub_bytes_out_req_n,
194     key_expand_out_req_n,
195     crypt_q_n,
196     dec_key_gen_q_n,
197     prng_reseed_q,
198     key_clear_q,
199     data_out_clear_q} = in_buf;
200
201     assign op = ciph_op_e'(op_raw);
202
203     // Intermediate output signals
204     logic      in_ready;
205     logic      out_valid;
206     logic      alert;
207     logic      prng_update;
208     logic      prng_reseed_req;
209     state_sel_e state_sel;
210     logic      state_we;
211     logic      sub_bytes_en;
212     logic      sub_bytes_out_ack;
213     add_rk_sel_e add_rk_sel;
214     key_full_sel_e key_full_sel;
215     logic      key_full_we;
216     key_dec_sel_e key_dec_sel;
217     logic      key_dec_we;
218     logic      key_expand_en;
219     logic      key_expand_out_ack;
220     logic      key_expand_clear;
221     key_words_sel_e key_words_sel;
222     round_key_sel_e round_key_sel;
223     logic [3:0] rnd_ctr;
224     logic      crypt_d;
225     logic      dec_key_gen_d;
226     logic      prng_reseed_d;
227     logic      key_clear_d;
228     logic      data_out_clear_d;
229
230     ///////////////////////////////////////////////////
231     // Regular FSM //
232     ///////////////////////////////////////////////////
233
234     // The regular FSM operates on and produces the positive
235     // values of important control signals.
236     // Invert *_n input signals here to get the positive values
237     // for the regular FSM. To obtain the
238     // negated outputs, important output signals are inverted
239     // further below. Thanks to the prim_buf
240     // synthesis optimization barriers, tools will push the in-
241     // verters into the regular FSM.
242     aes_cipher_control_fsm #(
243         .SecMasking ( SecMasking ),
244         .SecSBoxImpl ( SecSBoxImpl )
245     ) u_aes_cipher_control_fsm (
246         .clk_i      ( clk_i      ),
247         .rst_ni     ( rst_ni     ),
248         .in_valid_i ( ~in_valid_n ), // In-
249         .in_ready_o ( in_ready   ), // In-
250         .out_valid_o ( out_valid  ), // In-
251         .out_ready_i ( ~out_ready_n ), // In-
252         .cfg_valid_i ( cfg_valid  ),
253         .op_i        ( op        ),
254         .key_len_i   ( key_len   ),
255         .crypt_i     ( ~crypt_n  ), // In-
256         .dec_key_gen_i ( ~dec_key_gen_n ), // In-
257         .prng_reseed_i ( prng_reseed ),
258         .key_clear_i ( key_clear ),
259         .data_out_clear_i ( data_out_clear ),
260         .mux_sel_err_i ( mux_sel_err ),
261         .sp_enc_err_i ( sp_enc_err ),
262         .rnd_ctr_err_i ( rnd_ctr_err ),
263         .op_err_i    ( op_err    ),
264         .alert_fatal_i ( alert_fatal ),
265         .alert_o     ( alert     ),
266         .prng_update_o ( prng_update ),
267         .prng_reseed_req_o ( prng_reseed_req ),
268         .prng_reseed_ack_i ( prng_reseed_ack ),
269         .state_sel_o ( state_sel ),
270         .state_we_o ( state_we ), // In-
271         .sub_bytes_en_o ( sub_bytes_en ), // In-
272         .sub_bytes_out_ack_i ( ~sub_bytes_out_ack ), // In-
273         .sub_bytes_out_ack_o ( sub_bytes_out_ack ), // In-
274         .add_rk_sel_o ( add_rk_sel ),
275         .key_full_sel_o ( key_full_sel ),
276         .key_full_we_o ( key_full_we ), // In-
277         .key_dec_sel_o ( key_dec_sel ),
278         .key_dec_we_o ( key_dec_we ), // In-
279         .key_expand_en_o ( key_expand_en ), // In-
280         .key_expand_out_ack_i ( ~key_expand_out_ack ), // In-
281         .key_expand_out_ack_o ( key_expand_out_ack ), // In-
282         .key_expand_clear_o ( key_expand_clear ),
283         .rnd_ctr_o ( rnd_ctr ),
284         .key_words_sel_o ( key_words_sel ),
285         .round_key_sel_o ( round_key_sel ),
286         .crypt_q_i ( ~crypt_q_n ), // In-
287         .crypt_d_o ( crypt_d ), // In-
288         .dec_key_gen_q_i ( ~dec_key_gen_q_n ), // In-
289         .dec_key_gen_d_o ( dec_key_gen_d ), // In-
290         .key_clear_q_i ( key_clear_q ),
291         .key_clear_d_o ( key_clear_d ),
292         .prng_reseed_q_i ( prng_reseed_q ),
293         .prng_reseed_d_o ( prng_reseed_d ),
294         .data_out_clear_q_i ( data_out_clear_q ),
295         .data_out_clear_d_o ( data_out_clear_d )
296     );
297
298     ///////////////////////////////////////////////////
299     // Output Buffering //
300     ///////////////////////////////////////////////////
301
302     localparam int NumOutBufBits = $bits({
303         in_ready_no,
304         out_valid_no,
305         alert_o,
306         prng_update_o,
307         prng_reseed_req_o,
308         state_sel_o,
309         state_we_no,
310         sub_bytes_en_no,
311         sub_bytes_out_ack_no,
312         add_rk_sel_o,
313         key_full_sel_o,
314         key_full_we_no,
315         key_dec_sel_o,
316         key_dec_we_no,
317         key_expand_en_no,
318         key_expand_out_ack_no,
319         key_expand_clear_o,
320         rnd_ctr_o,
321         key_words_sel_o,
322         round_key_sel_o,
323         crypt_d_no,
324         dec_key_gen_d_no,
325         key_clear_d_o,
326         prng_reseed_d_o,
327         data_out_clear_d_o
328     });
329
330     logic [NumOutBufBits-1:0] out, out_buf;
331
332     // Important output control signals need to be inverted
333     // here. Synthesis tools will push the
334     // inverters back into the regular FSM.
335     assign out = {
336         ~in_ready,
337         ~out_valid,
338         alert,
339         prng_update,
340         prng_reseed_req,
341         state_sel,
342         ~state_we,
343         ~sub_bytes_en,
344         ~sub_bytes_out_ack,
345         add_rk_sel,
346         key_full_sel,
347         ~key_full_we,
348         key_dec_sel,
349         ~key_dec_we,
350         ~key_expand_en,
351         ~key_expand_out_ack,
352         key_expand_clear,
353         rnd_ctr,
354         key_words_sel,
355         round_key_sel,
356         ~crypt_d,
357         ~dec_key_gen_d,

```

```

272     .sub_bytes_en_o ( sub_bytes_en ), // In-
273     .sub_bytes_out_ack_i ( ~sub_bytes_out_ack ), // In-
274     .sub_bytes_out_ack_o ( sub_bytes_out_ack ), // In-
275     .add_rk_sel_o ( add_rk_sel ),
276     .key_full_sel_o ( key_full_sel ),
277     .key_full_we_o ( key_full_we ), // In-
278     .key_dec_sel_o ( key_dec_sel ),
279     .key_dec_we_o ( key_dec_we ), // In-
280     .key_expand_en_o ( key_expand_en ), // In-
281     .key_expand_out_ack_i ( ~key_expand_out_ack ), // In-
282     .key_expand_out_ack_o ( key_expand_out_ack ), // In-
283     .key_expand_clear_o ( key_expand_clear ),
284     .rnd_ctr_o ( rnd_ctr ),
285     .key_words_sel_o ( key_words_sel ),
286     .round_key_sel_o ( round_key_sel ),
287     .crypt_q_i ( ~crypt_q_n ), // In-
288     .crypt_d_o ( crypt_d ), // In-
289     .dec_key_gen_q_i ( ~dec_key_gen_q_n ), // In-
290     .dec_key_gen_d_o ( dec_key_gen_d ), // In-
291     .key_clear_q_i ( key_clear_q ),
292     .key_clear_d_o ( key_clear_d ),
293     .prng_reseed_q_i ( prng_reseed_q ),
294     .prng_reseed_d_o ( prng_reseed_d ),
295     .data_out_clear_q_i ( data_out_clear_q ),
296     .data_out_clear_d_o ( data_out_clear_d )
297 );
298
299 ///////////////////////////////////////////////////
300 // Output Buffering //
301 ///////////////////////////////////////////////////
302
303 localparam int NumOutBufBits = $bits({
304     in_ready_no,
305     out_valid_no,
306     alert_o,
307     prng_update_o,
308     prng_reseed_req_o,
309     state_sel_o,
310     state_we_no,
311     sub_bytes_en_no,
312     sub_bytes_out_ack_no,
313     add_rk_sel_o,
314     key_full_sel_o,
315     key_full_we_no,
316     key_dec_sel_o,
317     key_dec_we_no,
318     key_expand_en_no,
319     key_expand_out_ack_no,
320     key_expand_clear_o,
321     rnd_ctr_o,
322     key_words_sel_o,
323     round_key_sel_o,
324     crypt_d_no,
325     dec_key_gen_d_no,
326     key_clear_d_o,
327     prng_reseed_d_o,
328     data_out_clear_d_o
329 });
330
331 logic [NumOutBufBits-1:0] out, out_buf;
332
333 // Important output control signals need to be inverted
334 // here. Synthesis tools will push the
335 // inverters back into the regular FSM.
336 assign out = {
337     ~in_ready,
338     ~out_valid,
339     alert,
340     prng_update,
341     prng_reseed_req,
342     state_sel,
343     ~state_we,
344     ~sub_bytes_en,
345     ~sub_bytes_out_ack,
346     add_rk_sel,
347     key_full_sel,
348     ~key_full_we,
349     key_dec_sel,
350     ~key_dec_we,
351     ~key_expand_en,
352     ~key_expand_out_ack,
353     key_expand_clear,
354     rnd_ctr,
355     key_words_sel,
356     round_key_sel,
357     ~crypt_d,
358     ~dec_key_gen_d,

```

```

360   key_clear_d,
361   prng_reseed_d,
362   data_out_clear_d
363 };
364
365 // This primitive is used to place a size-only constraint on
the
366 // buffers to act as a synthesis optimization barrier.
367 prim_buf #(
368   .Width(NumOutBufBits)
369 ) u_prim_buf_out (
370   .in_i(out),
371   .out_o(out_buf)
372 );
373
374 assign {in_ready_no,
375         out_valid_no,
376         alert_o,
377         prng_update_o,
378         prng_reseed_req_o,
379         state_sel_o,
380         state_we_o,
381         sub_bytes_en_no,
382         sub_bytes_out_ack_no,
383         add_rk_sel_o,
384         key_full_sel_o,
385         key_full_we_o,
386         key_dec_sel_o,
387         key_dec_we_o,
388         key_expand_en_no,
389         key_expand_out_ack_no,
390         key_expand_clear_o,
391         rnd_ctr_o,
392         key_words_sel_o,
393         round_key_sel_o,
394         crypt_d_no,
395         dec_key_gen_d_no,
396         key_clear_d_o,
397         prng_reseed_d_o,
398         data_out_clear_d_o} = out_buf;
399
400 endmodule

```

5. lowRISC_opentitan/hw/ip/aes/rtl/ aes_cipher_control_fsm_p.sv (390 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core control FSM
6 //
7 // This module contains the AES cipher core control FSM oper-
ating on
8 // and producing the positive values of important control sig-
nals.
9
10 module aes_cipher_control_fsm_p import aes_pkg::*;
11 #(
12   parameter bit SecMasking = 0,
13   parameter sbox_impl_e SecSBoxImpl = SBoxImplDom
14 ) (
15   input logic clk_i,
16   input logic rst_ni,
17
18   // Input handshake signals
19   input logic in_valid_i, // Sparsify
using multi-rail.
20   output logic in_ready_o, // Sparsify
using multi-rail.
21
22   // Output handshake signals
23   output logic out_valid_o, // Sparsify
using multi-rail.
24   input logic out_ready_i, // Sparsify
using multi-rail.
25
26   // Control and sync signals
27   input logic cfg_valid_i, // Used for
SVAs only.
28   input ciph_op_e op_i,
29   input key_len_e key_len_i,
30   input logic crypt_i, // Sparsify
using multi-rail.
31   input logic dec_key_gen_i, // Sparsify
using multi-rail.
32   input logic prng_reseed_i,
33   input logic key_clear_i,
34   input logic data_out_clear_i,
35   input logic mux_sel_err_i,
36   input logic sp_enc_err_i,
37   input logic rnd_ctr_err_i,
38   input logic op_err_i,
39   input logic alert_fatal_i,
40   output logic alert_o,
41
42   // Control signals for masking PRNG

```

```

43   output logic prng_update_o,
44   output logic prng_reseed_req_o,
45   input logic prng_reseed_ack_i,
46
47   // Control and sync signals for cipher data path
48   output state_sel_e state_sel_o,
49   output logic state_we_o, // Sparsify
using multi-rail.
50   output logic sub_bytes_en_o, // Sparsify
using multi-rail.
51   input logic sub_bytes_out_req_i, // Sparsify
using multi-rail.
52   output logic sub_bytes_out_ack_o, // Sparsify
using multi-rail.
53   output add_rk_sel_e add_rk_sel_o,
54
55   // Control and sync signals for key expand data path
56   output key_full_sel_e key_full_sel_o,
57   output logic key_full_we_o, // Sparsify
using multi-rail.
58   output key_dec_sel_e key_dec_sel_o,
59   output logic key_dec_we_o, // Sparsify
using multi-rail.
60   output logic key_expand_en_o, // Sparsify
using multi-rail.
61   input logic key_expand_out_req_i, // Sparsify
using multi-rail.
62   output logic key_expand_out_ack_o, // Sparsify
using multi-rail.
63   output logic key_expand_clear_o,
64   output logic [3:0] rnd_ctr_o,
65   output key_words_sel_e key_words_sel_o,
66   output round_key_sel_e round_key_sel_o,
67
68   // Register signals
69   input logic crypt_q_i, // Sparsify
using multi-rail.
70   output logic crypt_d_o, // Sparsify
using multi-rail.
71   input logic dec_key_gen_q_i, // Sparsify
using multi-rail.
72   output logic dec_key_gen_d_o, // Sparsify
using multi-rail.
73   input logic prng_reseed_q_i,
74   output logic prng_reseed_d_o,
75   input logic key_clear_q_i,
76   output logic key_clear_d_o,
77   input logic data_out_clear_q_i,
78   output logic data_out_clear_d_o
79 );
80
81 //////////////////////////////////////////////////
82 // Input Buffering //
83 //////////////////////////////////////////////////
84
85 localparam int NumInBufBits = $bits({
86   in_valid_i,
87   out_ready_i,
88   cfg_valid_i,
89   op_i,
90   key_len_i,
91   crypt_i,
92   dec_key_gen_i,
93   prng_reseed_i,
94   key_clear_i,
95   data_out_clear_i,
96   mux_sel_err_i,
97   sp_enc_err_i,
98   rnd_ctr_err_i,
99   op_err_i,
100  alert_fatal_i,
101  prng_reseed_ack_i,
102  sub_bytes_out_req_i,
103  key_expand_out_req_i,
104  crypt_q_i,
105  dec_key_gen_q_i,
106  prng_reseed_q_i,
107  key_clear_q_i,
108  data_out_clear_q_i
109 });
110
111 logic [NumInBufBits-1:0] in, in_buf;
112
113 assign in = {
114   in_valid_i,
115   out_ready_i,
116   cfg_valid_i,
117   op_i,
118   key_len_i,
119   crypt_i,
120   dec_key_gen_i,
121   prng_reseed_i,
122   key_clear_i,
123   data_out_clear_i,
124   mux_sel_err_i,
125   sp_enc_err_i,
126   rnd_ctr_err_i,
127   op_err_i,
128   alert_fatal_i,
129   prng_reseed_ack_i,
130   sub_bytes_out_req_i,

```

```

131   key_expand_out_req_i,
132   crypt_q_i,
133   dec_key_gen_q_i,
134   prng_reseed_q_i,
135   key_clear_q_i,
136   data_out_clear_q_i
137   };
138
139 // This primitive is used to place a size-only constraint on
the
140 // buffers to act as a synthesis optimization barrier.
141 prim_buf #(
142   .Width(NumInBufBits)
143 ) u_prim_buf_in (
144   .in_i(in),
145   .out_o(in_buf)
146 );
147
148 logic          in_valid;
149 logic          out_ready;
150 logic          cfg_valid;
151 ciph_op_e      op;
152 logic [$bits(op)-1:0] op_raw;
153 key_len_e      key_len;
154 logic          crypt;
155 logic          dec_key_gen;
156 logic          prng_reseed;
157 logic          key_clear;
158 logic          data_out_clear;
159 logic          mux_sel_err;
160 logic          sp_enc_err;
161 logic          rnd_ctr_err;
162 logic          op_err;
163 logic          alert_fatal;
164 logic          prng_reseed_ack;
165 logic          sub_bytes_out_req;
166 logic          key_expand_out_req;
167 logic          crypt_q;
168 logic          dec_key_gen_q;
169 logic          prng_reseed_q;
170 logic          key_clear_q;
171 logic          data_out_clear_q;
172
173 assign {in_valid,
174        out_ready,
175        cfg_valid,
176        op_raw,
177        key_len,
178        crypt,
179        dec_key_gen,
180        prng_reseed,
181        key_clear,
182        data_out_clear,
183        mux_sel_err,
184        sp_enc_err,
185        rnd_ctr_err,
186        op_err,
187        alert_fatal,
188        prng_reseed_ack,
189        sub_bytes_out_req,
190        key_expand_out_req,
191        crypt_q,
192        dec_key_gen_q,
193        prng_reseed_q,
194        key_clear_q,
195        data_out_clear_q} = in_buf;
196
197 assign op = ciph_op_e'(op_raw);
198
199 // Intermediate output signals
200 logic          in_ready;
201 logic          out_valid;
202 logic          alert;
203 logic          prng_update;
204 logic          prng_reseed_req;
205 state_sel_e    state_sel;
206 logic          state_we;
207 logic          sub_bytes_en;
208 logic          sub_bytes_out_ack;
209 add_rk_sel_e   add_rk_sel;
210 key_full_sel_e key_full_sel;
211 logic          key_full_we;
212 key_dec_sel_e  key_dec_sel;
213 logic          key_dec_we;
214 logic          key_expand_en;
215 logic          key_expand_out_ack;
216 logic          key_expand_clear;
217 logic [3:0]    rnd_ctr;
218 key_words_sel_e key_words_sel;
219 round_key_sel_e round_key_sel;
220 logic          crypt_d;
221 logic          dec_key_gen_d;
222 logic          prng_reseed_d;
223 logic          key_clear_d;
224 logic          data_out_clear_d;
225
226 //////////////////////////////////////////////////
227 // Regular FSM //
228 //////////////////////////////////////////////////
229

```

```

230 aes_cipher_control_fsm #(
231   .SecMasking ( SecMasking ),
232   .SecSBoxImpl ( SecSBoxImpl )
233 ) u_aes_cipher_control_fsm (
234   .clk_i          ( clk_i          ),
235   .rst_ni         ( rst_ni         ),
236
237   .in_valid_i     ( in_valid       ),
238   .in_ready_o     ( in_ready       ),
239
240   .out_valid_o    ( out_valid      ),
241   .out_ready_i    ( out_ready      ),
242
243   .cfg_valid_i    ( cfg_valid      ),
244   .op_i           ( op             ),
245   .key_len_i      ( key_len        ),
246   .crypt_i        ( crypt          ),
247   .dec_key_gen_i  ( dec_key_gen     ),
248   .prng_reseed_i  ( prng_reseed    ),
249   .key_clear_i    ( key_clear      ),
250   .data_out_clear_i ( data_out_clear ),
251   .mux_sel_err_i  ( mux_sel_err    ),
252   .sp_enc_err_i   ( sp_enc_err     ),
253   .rnd_ctr_err_i  ( rnd_ctr_err    ),
254   .op_err_i       ( op_err         ),
255   .alert_fatal_i  ( alert_fatal     ),
256   .alert_o        ( alert          ),
257
258   .prng_update_o  ( prng_update    ),
259   .prng_reseed_req_o ( prng_reseed_req ),
260   .prng_reseed_ack_i ( prng_reseed_ack ),
261
262   .state_sel_o    ( state_sel      ),
263   .state_we_o     ( state_we       ),
264   .sub_bytes_en_o ( sub_bytes_en   ),
265   .sub_bytes_out_req_i ( sub_bytes_out_req ),
266   .sub_bytes_out_ack_o ( sub_bytes_out_ack ),
267   .add_rk_sel_o   ( add_rk_sel     ),
268
269   .key_full_sel_o ( key_full_sel    ),
270   .key_full_we_o  ( key_full_we     ),
271   .key_dec_sel_o  ( key_dec_sel     ),
272   .key_dec_we_o   ( key_dec_we     ),
273   .key_expand_en_o ( key_expand_en  ),
274   .key_expand_out_req_i ( key_expand_out_req ),
275   .key_expand_out_ack_o ( key_expand_out_ack ),
276   .key_expand_clear_o ( key_expand_clear ),
277   .rnd_ctr_o      ( rnd_ctr        ),
278   .key_words_sel_o ( key_words_sel  ),
279   .round_key_sel_o ( round_key_sel  ),
280
281   .crypt_q_i      ( crypt_q        ),
282   .crypt_d_o      ( crypt_d        ),
283   .dec_key_gen_q_i ( dec_key_gen_q  ),
284   .dec_key_gen_d_o ( dec_key_gen_d  ),
285   .key_clear_q_i  ( key_clear_q    ),
286   .key_clear_d_o  ( key_clear_d    ),
287   .prng_reseed_q_i ( prng_reseed_q  ),
288   .prng_reseed_d_o ( prng_reseed_d  ),
289   .data_out_clear_q_i ( data_out_clear_q ),
290   .data_out_clear_d_o ( data_out_clear_d ),
291 );
292
293 //////////////////////////////////////////////////
294 // Output Buffering //
295 //////////////////////////////////////////////////
296
297 localparam int NumOutBufBits = $bits({
298   in_ready_o,
299   out_valid_o,
300   alert_o,
301   prng_update_o,
302   prng_reseed_req_o,
303   state_sel_o,
304   state_we_o,
305   sub_bytes_en_o,
306   sub_bytes_out_ack_o,
307   add_rk_sel_o,
308   key_full_sel_o,
309   key_full_we_o,
310   key_dec_sel_o,
311   key_dec_we_o,
312   key_expand_en_o,
313   key_expand_out_ack_o,
314   key_expand_clear_o,
315   rnd_ctr_o,
316   key_words_sel_o,
317   round_key_sel_o,
318   crypt_d_o,
319   dec_key_gen_d_o,
320   key_clear_d_o,
321   prng_reseed_d_o,
322   data_out_clear_d_o
323 });
324
325 logic [NumOutBufBits-1:0] out, out_buf;
326
327 assign out = {
328   in_ready,
329   out_valid,
330   alert,

```

```

331 prng_update,
332 prng_reseed_req,
333 state_sel,
334 state_we,
335 sub_bytes_en,
336 sub_bytes_out_ack,
337 add_rk_sel,
338 key_full_sel,
339 key_full_we,
340 key_dec_sel,
341 key_dec_we,
342 key_expand_en,
343 key_expand_out_ack,
344 key_expand_clear,
345 rnd_ctr,
346 key_words_sel,
347 round_key_sel,
348 crypt_d,
349 dec_key_gen_d,
350 key_clear_d,
351 prng_reseed_d,
352 data_out_clear_d
353 };
354
355 // This primitive is used to place a size-only constraint on
the
356 // buffers to act as a synthesis optimization barrier.
357 prim_buf #(
358     .Width(NumOutBufBits)
359 ) u_prim_buf_out (
360     .in_i(out),
361     .out_o(out_buf)
362 );
363
364 assign {in_ready_o,
365         out_valid_o,
366         alert_o,
367         prng_update_o,
368         prng_reseed_req_o,
369         state_sel_o,
370         state_we_o,
371         sub_bytes_en_o,
372         sub_bytes_out_ack_o,
373         add_rk_sel_o,
374         key_full_sel_o,
375         key_full_we_o,
376         key_dec_sel_o,
377         key_dec_we_o,
378         key_expand_en_o,
379         key_expand_out_ack_o,
380         key_expand_clear_o,
381         rnd_ctr_o,
382         key_words_sel_o,
383         round_key_sel_o,
384         crypt_d_o,
385         dec_key_gen_d_o,
386         key_clear_d_o,
387         prng_reseed_d_o,
388         data_out_clear_d_o} = out_buf;
389
390 endmodule

```

6. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_core.sv (932 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core implementation
6 //
7 // This module contains the AES cipher core including, state
register, full key and decryption key
8 // registers as well as key expand module and control unit.
9 //
10 //
11 // Masking
12 // -----
13 //
14 // If the parameter "Masking" is set to one, first-order mask-
ing is applied to the entire
15 // cipher core including key expand module. For details, see
Rivain et al., "Provably secure
16 // higher-order masking of AES" available at https://
eprint.iacr.org/2010/441.pdf .
17 //
18 //
19 // Details on the data formats
20 // -----
21 //
22 // This implementation uses 4-dimensional SystemVerilog arrays to
represent the AES state:
23 //
24 //     logic [3:0][3:0][7:0] state_q [NumShares];
25 //
26 // The fourth dimension (unpacked) corresponds to the differ-

```

ent shares. The first element holds the
27 // (masked) data share whereas the other elements hold the
masks (masked implementation only).

28 // The three packed dimensions correspond to the 128-bit state
matrix per share. This
29 // implementation uses the same encoding as the Advanced En-
cryption Standard (AES) FIPS Publication
30 // 197 available at https://www.nist.gov/publications/ad-
vanced-encryption-standard-aes (see Section
31 // 3.4). An input sequence of 16 bytes (128-bit, left most
byte is the first one)

```

32 //
33 //     b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 b10 b11 b12 b13 b14 b15
34 //
35 // is mapped to the state matrix as
36 //
37 //     [ b0 b4 b8 b12 ]
38 //     [ b1 b5 b9 b13 ]
39 //     [ b2 b6 b10 b14 ]
40 //     [ b3 b7 b11 b15 ] .
41 //
42 // This is mapped to three packed dimensions of SystemVerilog
array as follows:
43 // - The first dimension corresponds to the rows. Thus,
state_q[0] gives
44 // - The first row of the state matrix [ b0 b4 b8
b12 ], or
45 // - A 32-bit packed SystemVerilog array 32h'{ b12, b8, b4,
b0 }.
46 //
47 // - The second dimension corresponds to the columns. To ac-
cess complete columns, the state matrix
48 // must be transposed first. Thus state_transposed =
aes_pkg::aes_transpose(state_q) and then
49 // state_transposed[1] gives
50 // - The second column of the state matrix [ b4 b5 b6
b7 ], or
51 // - A 32-bit packed SystemVerilog array 32h'{ b7, b6, b5,
b4 }.
52 //
53 // - The third dimension corresponds to the bytes.
54 //
55 // Note that the CSRs are little-endian. The input sequence
above is provided to 32-bit DATA_IN_0 -
56 // DATA_IN_3 registers as
57 //
58 //     MSB                                LSB
59 // - DATA_IN_0 32h'{ b3 , b2 , b1 , b0 }
60 // - DATA_IN_1 32h'{ b7 , b6 , b4 , b4 }
61 // - DATA_IN_2 32h'{ b11, b10, b9 , b8 }
62 // - DATA_IN_3 32h'{ b15, b14, b13, b12 } .
63 //
64 // The input state can thus be obtained by transposing the
content of the DATA_IN_0 - DATA_IN_3
65 // registers.

```

66 // Similarly, the implementation uses a 3-dimensional array to
represent the AES keys:

```

67 //
68 //     logic [7:0][31:0] key_full_q [NumShares]
69 //

```

70 // The third dimension (unpacked) corresponds to the different
shares. The first element holds the
71 // (masked) key share whereas the other elements hold the
masks (masked implementation only).

72 // The two packed dimensions correspond to the 256-bit key per
share. This implementation uses

73 // the same encoding as the Advanced Encryption Standard (AES)
FIPS Publication
74 // 197 available at https://www.nist.gov/publications/ad-
vanced-encryption-standard-aes .

75 //
76 // The first packed dimension corresponds to the 8 key words.
The second packed dimension
77 // corresponds to the 32 bits per key word. A key sequence of
32 bytes (256-bit, left most byte is
78 // the first one)

```

79 //
80 //     b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 b10 b11 b12 b13 b14
b15 ... .. b28 b29 b30 b31
81 //
82 // is mapped to the key words and registers (little-endian) as
83 //
84 //     MSB                                LSB
85 // - KEY_SHARE0_0 32h'{ b3 , b2 , b1 , b0 }
86 // - KEY_SHARE0_1 32h'{ b7 , b6 , b4 , b4 }
87 // - KEY_SHARE0_2 32h'{ b11, b10, b9 , b8 }
88 // - KEY_SHARE0_3 32h'{ b15, b14, b13, b12 }
89 // - KEY_SHARE0_4 32h'{ . . . . }
90 // - KEY_SHARE0_5 32h'{ . . . . }
91 // - KEY_SHARE0_6 32h'{ . . . . }
92 // - KEY_SHARE0_7 32h'{ b31, b30, b29, b28 } .
93 //
94 //
95 // include "prim_assert.sv"
96 //
97 // module aes_cipher_core import aes_pkg::*;
98 //
99 // parameter bit AES192Enable = 1,

```



```

98 parameter bit CiphOpFwdOnly = 0,
99 parameter bit SecMasking = 1,
100 parameter sbox_impl_e SecSBoxImpl = SBoxImplDom,
101 parameter bit SecAllowForcingMasks = 0,
102 parameter bit SecSkipPRNGReseeding = 0,
103 parameter int unsigned EntropyWidth = edn_pkg::END-
POINT_BUS_WIDTH,
104
105 localparam int NumShares = SecMasking ?
2 : 1, // derived parameter
106
107 parameter masking_lfsr_seed_t RndCnstMaskingLfsrSeed = Rnd-
CnstMaskingLfsrSeedDefault,
108 parameter masking_lfsr_perm_t RndCnstMaskingLfsrPerm = Rnd-
CnstMaskingLfsrPermDefault
109 ) (
110 input logic clk_i,
111 input logic rst_ni,
112
113 // Input handshake signals
114 input sp2v_e in_valid_i,
115 output sp2v_e in_ready_o,
116
117 // Output handshake signals
118 output sp2v_e out_valid_o,
119 input sp2v_e out_ready_i,
120
121 // Control and sync signals
122 input logic cfg_valid_i, // Used for
gating assertions only.
123 input ciph_op_e op_i,
124 input key_len_e key_len_i,
125 input sp2v_e crypt_i,
126 output sp2v_e crypt_o,
127 input sp2v_e dec_key_gen_i,
128 output sp2v_e dec_key_gen_o,
129 input logic prng_reseed_i,
130 output logic prng_reseed_o,
131 input logic key_clear_i,
132 output logic key_clear_o,
133 input logic data_out_clear_i, // Re-
use the cipher core muxes.
134 output logic data_out_clear_o,
135 input logic alert_fatal_i,
136 output logic alert_o,
137
138 // Pseudo-random data for register clearing
139 input logic [3:0][3:0][7:0] prd_clearing_state_i
[NumShares],
140 input logic [7:0][31:0] prd_clearing_key_i [Num-
Shares],
141
142 // Masking PRNG
143 input logic force_masks_i, // Useful
for SCA only.
144 output logic [3:0][3:0][7:0] data_in_mask_o,
145 output logic entropy_req_o,
146 input logic entropy_ack_i,
147 input logic [EntropyWidth-1:0] entropy_i,
148
149 // I/O data & initial key
150 input logic [3:0][3:0][7:0] state_init_i [Num-
Shares],
151 input logic [7:0][31:0] key_init_i [NumShares],
152 output logic [3:0][3:0][7:0] state_o [NumShares]
153 );
154
155 // Signals
156 logic [3:0][3:0][7:0] state_d [NumShares];
157 logic [3:0][3:0][7:0] state_q [NumShares];
158 sp2v_e state_we_ctrl;
159 sp2v_e state_we;
160 logic [StateSelWidth-1:0] state_sel_raw;
161 state_sel_e state_sel_ctrl;
162 state_sel_e state_sel;
163 logic state_sel_err;
164
165 sp2v_e sub_bytes_en;
166 sp2v_e sub_bytes_out_req;
167 sp2v_e sub_bytes_out_ack;
168 logic sub_bytes_err;
169 logic [3:0][3:0][7:0] sub_bytes_out;
170 logic [3:0][3:0][7:0] sb_in_mask;
171 logic [3:0][3:0][7:0] sb_out_mask;
172 logic [3:0][3:0][7:0] shift_rows_in [Num-
Shares];
173 logic [3:0][3:0][7:0] shift_rows_out [Num-
Shares];
174 logic [3:0][3:0][7:0] mix_columns_out [Num-
Shares];
175 logic [3:0][3:0][7:0] add_round_key_in [Num-
Shares];
176 logic [3:0][3:0][7:0] add_round_key_out [Num-
Shares];
177 logic [AddRKSelWidth-1:0] add_rk_sel_raw;
178 add_rk_sel_e add_rk_sel_ctrl;
179 add_rk_sel_e add_rk_sel;
180 logic add_rk_sel_err;
181

```

```

182 logic [7:0][31:0] key_full_d [NumShares];
183 logic [7:0][31:0] key_full_q [NumShares];
184 sp2v_e key_full_we_ctrl;
185 sp2v_e key_full_we;
186 logic [KeyFullSelWidth-1:0] key_full_sel_raw;
187 key_full_sel_e key_full_sel_ctrl;
188 key_full_sel_e key_full_sel;
189 logic key_full_sel_err;
190 logic [7:0][31:0] key_dec_d [NumShares];
191 logic [7:0][31:0] key_dec_q [NumShares];
192 sp2v_e key_dec_we_ctrl;
193 sp2v_e key_dec_we;
194 logic [KeyDecSelWidth-1:0] key_dec_sel_raw;
195 key_dec_sel_e key_dec_sel_ctrl;
196 key_dec_sel_e key_dec_sel;
197 logic key_dec_sel_err;
198 logic [7:0][31:0] key_expand_out [Num-
Shares];
199 ciph_op_e key_expand_op;
200 sp2v_e key_expand_en;
201 logic key_expand_prd_we;
202 sp2v_e key_expand_out_req;
203 sp2v_e key_expand_out_ack;
204 logic key_expand_err;
205 logic key_expand_clear;
206 logic [3:0] key_expand_round;
207 logic [KeywordsSelWidth-1:0] key_words_sel_raw;
208 key_words_sel_e key_words_sel_ctrl;
209 key_words_sel_e key_words_sel;
210 logic key_words_sel_err;
211 logic [3:0][31:0] key_words [NumShares];
212 logic [3:0][3:0][7:0] key_bytes [NumShares];
213 logic [3:0][3:0][7:0] key_mix_columns_out
[NumShares];
214 logic [3:0][3:0][7:0] round_key [NumShares];
215 logic [RoundKeySelWidth-1:0] round_key_sel_raw;
216 round_key_sel_e round_key_sel_ctrl;
217 round_key_sel_e round_key_sel;
218 logic round_key_sel_err;
219
220 logic cfg_valid;
221 logic mux_sel_err;
222 logic sp_enc_err_q;
223 logic op_err;
224
225 // Pseudo-random data for masking purposes
226 logic [WidthPRDMasking-1:0] prd_masking;
227 logic [3:0][3:0][WidthPRDSBox-1:0] prd_sub_bytes_d;
228 logic [3:0][3:0][WidthPRDSBox-1:0] prd_sub_bytes_q;
229 logic [WidthPRDKey-1:0] prd_key_expand;
230 logic prd_masking_upd;
231 logic prd_masking_rsd_req;
232 logic prd_masking_rsd_ack;
233
234 logic [3:0][3:0][7:0] data_in_mask;
235
236 // op_i is one-hot encoded. Check the provided value and
trigger an alert upon detecting invalid
237 // encodings.
238 assign op_err = ~(op_i == CIPH_FWD || op_i == CIPH_INV);
239 assign cfg_valid = cfg_valid_i & ~op_err;
240
241 // Data //
242 // Data //
243 // Data //
244
245 // SEC_CM: DATA_REG.SEC_WIPE
246 // State registers
247 always_comb begin : state_mux
248     unique case (state_sel)
249         STATE_INIT: state_d = state_init_i;
250         STATE_ROUND: state_d = add_round_key_out;
251         STATE_CLEAR: state_d = prd_clearing_state_i;
252         default: state_d = prd_clearing_state_i;
253     endcase
254 end
255
256 always_ff @(posedge clk_i or negedge rst_ni) begin :
state_reg
257     if (!rst_ni) begin
258         state_q <= '{default: '0};
259     end else if (state_we == SP2V_HIGH) begin
260         state_q <= state_d;
261     end
262 end
263
264 // Masking
265 if (!SecMasking) begin : gen_no_masks
266     // The masks are ignored anyway, they can be 0.
267     assign sb_in_mask = '0;
268     assign prd_masking = '0;
269
270 // Tie-off unused signals.
271 logic unused_entropy_ack;
272 logic [EntropyWidth-1:0] unused_entropy;
273 assign unused_entropy_ack = entropy_ack_i;
274 assign unused_entropy = entropy_i;
275 assign entropy_req_o = '1'b0;
276
277 logic unused_force_masks;

```

```

278 logic unused_prd_masking_upd;
279 logic unused_prd_masking_rsd_req;
280 assign unused_force_masks = force_masks_i;
281 assign unused_prd_masking_upd = prd_masking_upd;
282 assign unused_prd_masking_rsd_req = prd_masking_rsd_req;
283 assign prd_masking_rsd_ack = 1'b0;
284
285 logic [3:0][3:0][7:0] unused_sb_out_mask;
286 assign unused_sb_out_mask = sb_out_mask;
287
288 end else begin : gen_masks
289 // The input mask is the mask share of the state.
290 assign sb_in_mask = state_q[1];
291
292 // The masking PRNG generates:
293 // - the pseudo-random data (PRD) required by SubBytes,
294 // - the PRD required by the key expand module (has 4 S-Boxes internally).
295 aes_prng_masking #(
296     .Width ( WidthPRDMasking ),
297     .EntropyWidth ( EntropyWidth ),
298     .SecAllowForcingMasks ( SecAllowForcingMasks ),
299     .SecSkipPRNGReseeding ( SecSkipPRNGReseeding ),
300     .RndCnstLfsrcSeed ( RndCnstLfsrcSeed ),
301     .RndCnstLfsrcPerm ( RndCnstLfsrcPerm )
302 ) u_aes_prng_masking (
303     .clk_i ( clk_i ),
304     .rst_ni ( rst_ni ),
305     .force_masks_i ( force_masks_i ),
306     .data_update_i ( prd_masking_upd ),
307     .data_o ( prd_masking ),
308     .reseed_req_i ( prd_masking_rsd_req ),
309     .reseed_ack_o ( prd_masking_rsd_ack ),
310     .entropy_req_o ( entropy_req_o ),
311     .entropy_ack_i ( entropy_ack_i ),
312     .entropy_i ( entropy_i )
313 );
314 end
315
316 // Extract randomness for key expand module and SubBytes.
317 //
318 // The masking PRNG output has the following shape:
319 // prd_masking = { prd_key_expand, prd_sub_bytes_d }
320 assign prd_key_expand = prd_masking[WidthPRDMasking-1 -: WidthPRDKey];
321 assign prd_sub_bytes_d = prd_masking[WidthPRDData-1 -: WidthPRDData];
322
323 // PRD buffering
324 if (!SecMasking) begin : gen_no_prd_buffer
325 // The masks are ignored anyway.
326 assign prd_sub_bytes_q = prd_sub_bytes_d;
327
328 end else begin : gen_prd_buffer
329 // PRD buffer stage to:
330 // 1. Make sure the S-Boxes get always presented new data/
331 // mask inputs together with fresh PRD
332 // for remasking.
333
334 // 2. Prevent glitches originating from inside the masking
335 // PRNG from propagating into the
336 // masked S-Boxes.
337 always_ff @(posedge clk_i or negedge rst_ni) begin :
338 prd_sub_bytes_reg
339 if (!rst_ni) begin
340 prd_sub_bytes_q <= '0;
341 end else if (state_we == SP2V_HIGH) begin
342 prd_sub_bytes_q <= prd_sub_bytes_d;
343 end
344 end
345
346 // Convert the 3-dimensional prd_sub_bytes_q array to a 1-
347 // dimensional packed array for the
348 // aes_prd_get_lsbs() function used below.
349 logic [WidthPRDData-1:0] prd_sub_bytes;
350 assign prd_sub_bytes = prd_sub_bytes_q;
351
352 // Extract randomness for masking the input data.
353 //
354 // The masking PRNG is used for generating both the PRD for
355 // the S-Boxes/SubBytes operation as
356 // well as for the input data masks. When using any of the
357 // masked Canright S-Box implementations,
358 // it is important that the SubBytes input masks (generated
359 // by the PRNG in Round X-1) and the
360 // SubBytes output masks (generated by the PRNG in Round X)
361 // are independent. This can be achieved
362 // by using e.g. an unrolled Bivium stream cipher primitive
363 // inside the PRNG. Since the input data
364
365 // masks become the SubBytes input masks in the first round,
366 // we select the same 8 bit lanes for
367 // the input data masks which are also used to form the Sub-
368 // Bytes output mask for the masked
369 // Canright S-Box implementations, i.e., the 8 LSBs of the
370 // per S-Box PRD. In particular, we have:
371 //
372 // prd_masking = { prd_key_expand, ... , sb_prd[4],
373 // sb_out_mask[4], sb_prd[0], sb_out_mask[0] }
374 //

```

```

361 // Where sb_out_mask[x] contains the SubBytes output mask
362 // for byte x (when using a masked
363 // Canright S-Box implementation) and sb_prd[x] contains ad-
364 // ditional PRD consumed by SubBytes for
365 // byte x.
366 //
367 // When using a masked S-Box implementation other than Can-
368 // right, we still select the 8 LSBs of
369 // the per-S-Box PRD to form the input data mask of the cor-
370 // responding byte. We do this to
371 // distribute the input data masks over all output bits the
372 // masking PRNG. We do the extraction on
373 // a row basis.
374 localparam int unsigned WidthPRDRow = 4*WidthPRDSubBox;
375 for (genvar i = 0; i < 4; i++) begin : gen_in_mask
376 assign data_in_mask[i] = aes_prd_get_lsbs(prd_sub_bytes[i
377 * WidthPRDRow -: WidthPRDRow]);
378 end
379
380 // Rotate the data input masks by 64 bits to ensure the data
381 // input masks are independent
382 // from the PRD fed to the S-Boxes/SubBytes operation.
383 assign data_in_mask_o = {data_in_mask[1], data_in_mask[0],
384 data_in_mask[3], data_in_mask[2]};
385
386 // Cipher data path
387 aes_sub_bytes #(
388     .SecSBoxImpl ( SecSBoxImpl )
389 ) u_aes_sub_bytes (
390     .clk_i ( clk_i ),
391     .rst_ni ( rst_ni ),
392     .en_i ( sub_bytes_en ),
393     .out_req_o ( sub_bytes_out_req ),
394     .out_ack_i ( sub_bytes_out_ack ),
395     .op_i ( op_i ),
396     .data_i ( state_q[0] ),
397     .mask_i ( sb_in_mask ),
398     .prd_i ( prd_sub_bytes_q ),
399     .data_o ( sub_bytes_out ),
400     .mask_o ( sb_out_mask ),
401     .err_o ( sub_bytes_err )
402 );
403
404 for (genvar s = 0; s < NumShares; s++) begin :
405 gen_shares_shift_mix
406 if (s == 0) begin : gen_shift_in_data
407 // The (masked) data share
408 assign shift_rows_in[s] = sub_bytes_out;
409 end else begin : gen_shift_in_mask
410 // The mask share
411 assign shift_rows_in[s] = sb_out_mask;
412 end
413
414 aes_shift_rows u_aes_shift_rows (
415     .op_i ( op_i ),
416     .data_i ( shift_rows_in[s] ),
417     .data_o ( shift_rows_out[s] )
418 );
419
420 aes_mix_columns u_aes_mix_columns (
421     .op_i ( op_i ),
422     .data_i ( shift_rows_out[s] ),
423     .data_o ( mix_columns_out[s] )
424 );
425 end
426
427 always_comb begin : add_round_key_in_mux
428 unique case (add_rk_sel)
429 ADD_RK_INIT: add_round_key_in = state_q;
430 ADD_RK_ROUND: add_round_key_in = mix_columns_out;
431 ADD_RK_FINAL: add_round_key_in = shift_rows_out;
432 default: add_round_key_in = state_q;
433 endcase
434 end
435
436 for (genvar s = 0; s < NumShares; s++) begin :
437 gen_shares_add_round_key
438 assign add_round_key_out[s] = add_round_key_in[s] ^
439 round_key[s];
440 end
441
442 // Key
443 //
444 // SEC_CM: KEY.SEC_WIPE
445 // Full Key registers
446 always_comb begin : key_full_mux
447 unique case (key_full_sel)
448 KEY_FULL_ENC_INIT: key_full_d = key_init_i;
449 KEY_FULL_DEC_INIT: key_full_d = !CiphOpFwdOnly ?
450 key_dec_q : prd_clearing_key_i;
451 KEY_FULL_ROUND: key_full_d = key_expand_out;
452 KEY_FULL_CLEAR: key_full_d = prd_clearing_key_i;
453 default: key_full_d = prd_clearing_key_i;
454 endcase
455 end
456
457 always_ff @(posedge clk_i or negedge rst_ni) begin :
458 key_full_reg

```

```

448   if (!rst_ni) begin
449     key_full_q <= '{default: '0};
450   end else if (key_full_we == SP2V_HIGH) begin
451     key_full_q <= key_full_d;
452   end
453 end
454
455 if (!CiphOpFwdOnly) begin : gen_key_dec
456   // SEC_CM: KEY.SEC.WIPE
457   // Decryption Key registers
458   always_comb begin : key_dec_mux
459     unique case (key_dec_sel)
460       KEY_DEC_EXPAND: key_dec_d = key_expand_out;
461       KEY_DEC_CLEAR:  key_dec_d = prd_clearing_key_i;
462       default:        key_dec_d = prd_clearing_key_i;
463     endcase
464   end
465
466   always_ff @(posedge clk_i or negedge rst_ni) begin :
key_dec_reg
467     if (!rst_ni) begin
468       key_dec_q <= '{default: '0};
469     end else if (key_dec_we == SP2V_HIGH) begin
470       key_dec_q <= key_dec_d;
471     end
472   end
473 end else begin : gen_no_key_dec
474   // No Decryption Key registers
475   assign key_dec_q = '{default: '0};
476   assign key_dec_d = key_dec_q;
477
478   // Tie-off unused signals.
479   logic unused_key_dec;
480   always_comb begin
481     unused_key_dec = ^{key_dec_sel, key_dec_we};
482     for (int s = 0; s < NumShares; s++) begin
483       unused_key_dec ^= ^{key_dec_d[s]};
484     end
485   end
486 end
487
488 // Make sure that whenever the data/mask inputs of the S-
Boxes update, the internally buffered
489 // PRD is updated in sync.
490 assign key_expand_prd_we = (key_full_we == SP2V_HIGH) ?
1'b1 : 1'b0;
491
492 // Key expand data path
493 aes_key_expand #(
494   .AES192Enable ( AES192Enable ),
495   .SecMasking   ( SecMasking ),
496   .SecSBoxImpl  ( SecSBoxImpl )
497 ) u_aes_key_expand (
498   .clk_i      ( clk_i ),
499   .rst_ni     ( rst_ni ),
500   .cfg_valid_i ( cfg_valid ),
501   .op_i       ( key_expand_op ),
502   .en_i       ( key_expand_en ),
503   .prd_we_i   ( key_expand_prd_we ),
504   .out_req_o  ( key_expand_out_req ),
505   .out_ack_i  ( key_expand_out_ack ),
506   .clear_i    ( key_expand_clear ),
507   .round_i    ( key_expand_round ),
508   .key_len_i  ( key_len_i ),
509   .key_i      ( key_full_q ),
510   .key_o      ( key_expand_out ),
511   .prd_i      ( prd_key_expand ),
512   .err_o      ( key_expand_err ),
513 );
514
515 for (genvar s = 0; s < NumShares; s++) begin :
gen_shares_round_key
516   always_comb begin : key_words_mux
517     unique case (key_words_sel)
518       KEY_WORDS_0123: key_words[s] = key_full_q[s][3:0];
519       KEY_WORDS_2345: key_words[s] = AES192Enable ?
key_full_q[s][5:2] : '0;
520       KEY_WORDS_4567: key_words[s] = key_full_q[s][7:4];
521       KEY_WORDS_ZERO: key_words[s] = '0;
522       default:        key_words[s] = '0;
523     endcase
524   end
525
526   // Convert words to bytes (every key word contains one
column).
527   assign key_bytes[s] = aes_transpose(key_words[s]);
528
529   aes_mix_columns u_aes_key_mix_columns (
530     .op_i ( CIPH_INV ),
531     .data_i ( key_bytes[s] ),
532     .data_o ( key_mix_columns_out[s] )
533   );
534 end
535
536 always_comb begin : round_key_mux
537   unique case (round_key_sel)
538     ROUND_KEY_DIRECT: round_key = key_bytes;
539     ROUND_KEY_MIXED:  round_key = !CiphOpFwdOnly ?
key_mix_columns_out : key_bytes;
540     default:          round_key = key_bytes;
541   endcase

```

```

542 end
543
544 if (CiphOpFwdOnly) begin : gen_unused_key_mix_columns_out
545   // Tie-off unused signals.
546   logic unused_key_mix_columns_out;
547   always_comb begin
548     unused_key_mix_columns_out = 1'b0;
549     for (int s = 0; s < NumShares; s++) begin
550       unused_key_mix_columns_out ^=
^key_mix_columns_out[s];
551     end
552   end
553 end
554
555 // Control
556 // Control
557 // Control
558 // Control
559 aes_cipher_control #(
560   .CiphOpFwdOnly ( CiphOpFwdOnly ),
561   .SecMasking    ( SecMasking ),
562   .SecSBoxImpl   ( SecSBoxImpl )
563 ) u_aes_cipher_control (
564   .clk_i      ( clk_i ),
565   .rst_ni     ( rst_ni ),
566   .in_valid_i ( in_valid_i ),
567   .in_ready_o ( in_ready_o ),
568   .out_valid_o ( out_valid_o ),
569   .out_ready_i ( out_ready_i ),
570   .cfg_valid_i ( cfg_valid ),
571   .op_i       ( op_i ),
572   .key_len_i  ( key_len_i ),
573   .crypt_i    ( crypt_i ),
574   .crypt_o    ( crypt_o ),
575   .dec_key_gen_i ( dec_key_gen_i ),
576   .dec_key_gen_o ( dec_key_gen_o ),
577   .prng_reseed_i ( prng_reseed_i ),
578   .prng_reseed_o ( prng_reseed_o ),
579   .key_clear_i ( key_clear_i ),
580   .key_clear_o ( key_clear_o ),
581   .data_out_clear_i ( data_out_clear_i ),
582   .data_out_clear_o ( data_out_clear_o ),
583   .mux_sel_err_i ( mux_sel_err ),
584   .sp_enc_err_i ( sp_enc_err_q ),
585   .op_err_i     ( op_err ),
586   .alert_fatal_i ( alert_fatal_i ),
587   .alert_o      ( alert_o ),
588   .prng_update_o ( prd_masking_upd ),
589   .prng_reseed_req_o ( prd_masking_rsd_req ),
590   .prng_reseed_ack_i ( prd_masking_rsd_ack ),
591   .state_sel_o ( state_sel_ctrl ),
592   .state_we_o ( state_we_ctrl ),
593   .sub_bytes_en_o ( sub_bytes_en ),
594   .sub_bytes_out_req_i ( sub_bytes_out_req ),
595   .sub_bytes_out_ack_o ( sub_bytes_out_ack ),
596   .add_rk_sel_o ( add_rk_sel_ctrl ),
597   .key_expand_op_o ( key_expand_op ),
598   .key_full_sel_o ( key_full_sel_ctrl ),
599   .key_full_we_o ( key_full_we_ctrl ),
600   .key_dec_sel_o ( key_dec_sel_ctrl ),
601   .key_dec_we_o ( key_dec_we_ctrl ),
602   .key_expand_en_o ( key_expand_en ),
603   .key_expand_out_req_i ( key_expand_out_req ),
604   .key_expand_out_ack_o ( key_expand_out_ack ),
605   .key_expand_clear_o ( key_expand_clear ),
606   .key_expand_round_o ( key_expand_round ),
607   .key_words_sel_o ( key_words_sel_ctrl ),
608   .round_key_sel_o ( round_key_sel_ctrl ),
609 );
610
611 // We use sparse encodings for these mux selector signals
and must ensure that:
612 // 1. The synthesis tool doesn't optimize away the sparse
encoding.
613 // 2. The selector signal is always valid. More precisely,
an alert or SVA is triggered if a
614 // selector signal takes on an invalid value.
615 // 3. The alert signal remains asserted until reset even if
the selector signal becomes valid
616 // again. This is achieved by driving the control FSM
into the terminal error state whenever
617 // any mux selector signal becomes invalid.
618 //
619 // If any mux selector signal becomes invalid, the cipher
core further immediately de-asserts
620 // the out_valid_o signal to prevent any data from being re-
leased.
621
622 aes_sel_buf_chk #(
623   .Num ( StateSelNum ),

```



```

635 .Width ( StateSelWidth ),
636 .EnSecBuf ( 1'b1 )
637 ) u_aes_state_sel_buf_chk (
638 .clk_i ( clk_i ),
639 .rst_ni ( rst_ni ),
640 .sel_i ( state_sel_ctrl ),
641 .sel_o ( state_sel_raw ),
642 .err_o ( state_sel_err )
643 );
644 assign state_sel = state_sel_e'(state_sel_raw);
645
646 aes_sel_buf_chk #(
647 .Num ( AddrKSelNum ),
648 .Width ( AddrKSelWidth ),
649 .EnSecBuf ( 1'b1 )
650 ) u_aes_add_rk_sel_buf_chk (
651 .clk_i ( clk_i ),
652 .rst_ni ( rst_ni ),
653 .sel_i ( add_rk_sel_ctrl ),
654 .sel_o ( add_rk_sel_raw ),
655 .err_o ( add_rk_sel_err )
656 );
657 assign add_rk_sel = add_rk_sel_e'(add_rk_sel_raw);
658
659 aes_sel_buf_chk #(
660 .Num ( KeyFullSelNum ),
661 .Width ( KeyFullSelWidth ),
662 .EnSecBuf ( 1'b1 )
663 ) u_aes_key_full_sel_buf_chk (
664 .clk_i ( clk_i ),
665 .rst_ni ( rst_ni ),
666 .sel_i ( key_full_sel_ctrl ),
667 .sel_o ( key_full_sel_raw ),
668 .err_o ( key_full_sel_err )
669 );
670 assign key_full_sel = key_full_sel_e'(key_full_sel_raw);
671
672 aes_sel_buf_chk #(
673 .Num ( KeyDecSelNum ),
674 .Width ( KeyDecSelWidth ),
675 .EnSecBuf ( 1'b1 )
676 ) u_aes_key_dec_sel_buf_chk (
677 .clk_i ( clk_i ),
678 .rst_ni ( rst_ni ),
679 .sel_i ( key_dec_sel_ctrl ),
680 .sel_o ( key_dec_sel_raw ),
681 .err_o ( key_dec_sel_err )
682 );
683 assign key_dec_sel = key_dec_sel_e'(key_dec_sel_raw);
684
685 aes_sel_buf_chk #(
686 .Num ( KeyWordsSelNum ),
687 .Width ( KeyWordsSelWidth ),
688 .EnSecBuf ( 1'b1 )
689 ) u_aes_key_words_sel_buf_chk (
690 .clk_i ( clk_i ),
691 .rst_ni ( rst_ni ),
692 .sel_i ( key_words_sel_ctrl ),
693 .sel_o ( key_words_sel_raw ),
694 .err_o ( key_words_sel_err )
695 );
696 assign key_words_sel = key_words_sel_e'(key_words_sel_raw);
697
698 aes_sel_buf_chk #(
699 .Num ( RoundKeySelNum ),
700 .Width ( RoundKeySelWidth ),
701 .EnSecBuf ( 1'b1 )
702 ) u_aes_round_key_sel_buf_chk (
703 .clk_i ( clk_i ),
704 .rst_ni ( rst_ni ),
705 .sel_i ( round_key_sel_ctrl ),
706 .sel_o ( round_key_sel_raw ),
707 .err_o ( round_key_sel_err )
708 );
709 assign round_key_sel = round_key_sel_e'(round_key_sel_raw);
710
711 // Signal invalid mux selector signals to control FSM which
will lock up and trigger an alert.
712 assign mux_sel_err = state_sel_err | add_rk_sel_err |
key_full_sel_err |
713 key_dec_sel_err | key_words_sel_err | round_key_sel_err;
714
715 //////////////////////////////////////////////////
716 // Sparsely Encoded Signals //
717 //////////////////////////////////////////////////
718
719 // We use sparse encodings for various critical signals and
must ensure that:
720 // 1. The synthesis tool doesn't optimize away the sparse
encoding.
721 // 2. The sparsely encoded signal is always valid. More pre-
cisely, an alert or SVA is triggered
722 // if a sparse signal takes on an invalid value.
723 // 3. The alert signal remains asserted until reset even if
the sparse signal becomes valid again
724 // This is achieved by driving the control FSM into the
terminal error state whenever any
725 // sparsely encoded signal becomes invalid.
726 //
727 // If any sparsely encoded signal becomes invalid, the
cipher core further immediately de-asserts

```

```

728 // the out_valid_o signal to prevent any data from being re-
leased.
729
730 // We use vectors of sparsely encoded signals to reduce code
duplication.
731 localparam int unsigned NumSp2Vsig = 3;
732 sp2v_e [NumSp2Vsig-1:0] sp2v_sig;
733 sp2v_e [NumSp2Vsig-1:0] sp2v_sig_chk;
734 logic [NumSp2Vsig-1:0][Sp2VWidth-1:0] sp2v_sig_chk_raw;
735 logic [NumSp2Vsig-1:0] sp2v_sig_err;
736
737 assign sp2v_sig[0] = state_we_ctrl;
738 assign sp2v_sig[1] = key_full_we_ctrl;
739 assign sp2v_sig[2] = key_dec_we_ctrl;
740
741 // All signals inside sp2v_sig are eventually converted to
single-rail signals.
742 localparam bit [NumSp2Vsig-1:0] Sp2VEnSecBuf = {Num-
Sp2Vsig{1'b1}};
743
744 // Individually check sparsely encoded signals.
745 for (genvar i = 0; i < NumSp2Vsig; i++) begin :
gen_sel_buf_chk
746     aes_sel_buf_chk #(
747         .Num ( Sp2VNum ),
748         .Width ( Sp2VWidth ),
749         .EnSecBuf ( Sp2VEnSecBuf[i] )
750     ) u_aes_sp2v_sig_buf_chk_i (
751         .clk_i ( clk_i ),
752         .rst_ni ( rst_ni ),
753         .sel_i ( sp2v_sig[i] ),
754         .sel_o ( sp2v_sig_chk_raw[i] ),
755         .err_o ( sp2v_sig_err[i] )
756     );
757     assign sp2v_sig_chk[i] = sp2v_e'(sp2v_sig_chk_raw[i]);
758 end
759
760 assign state_we = sp2v_sig_chk[0];
761 assign key_full_we = sp2v_sig_chk[1];
762 assign key_dec_we = sp2v_sig_chk[2];
763
764 // Collect encoding errors.
765 // We instantiate the checker modules as close as possible
to where the sparsely encoded signals
766 // are used. Here, we collect also encoding errors detected
in other places of the cipher core.
767 assign sp_enc_err_d = |sp2v_sig_err | sub_bytes_err |
key_expand_err;
768
769 // We need to register the collected error signal to avoid
circular loops in the cipher core
770
771 // controller related to out_valid_o and detecting errors in
state_we_o and sub_bytes_out_ack.
772 always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_sp_enc_err
773     if (!rst_ni) begin
774         sp_enc_err_q <= 1'b0;
775     end else if (sp_enc_err_d) begin
776         sp_enc_err_q <= 1'b1;
777     end
778 end
779
780 //////////////////////////////////////////////////
781 // Outputs //
782 //////////////////////////////////////////////////
783
784 // The output of the last round is not stored into the state
register but forwarded directly.
785 assign state_o = add_round_key_out;
786
787 //////////////////////////////////////////////////
788 // Assertions //
789 //////////////////////////////////////////////////
790 // Typically assertions already contain this macro, which en-
sures that assertions are only compiled
791 // in simulation and FPV. However, we wrap the entire asser-
tion section with INC_ASSERT so that the
792 // helper logic below is not synthesized either, since that
could cause issues in DC.
793 `ifdef INC_ASSERT
794 //VCS coverage off
795 // pragma coverage off
796
797 // Create a lint error to reduce the risk of accidentally
disabling the masking.
798 `ASSERT_STATIC_LINT_ERROR(AesSecMaskingNonDefault, SecMask-
ing == 1)
799
800 // Cipher core masking requires a masked SBox and vice
versa.
801 `ASSERT_INIT(AesMaskedCoreAndSBox,
802 (SecMasking &&
803 (SecSBoxImpl == SBoxImplCanrightMasked ||
804 SecSBoxImpl == SBoxImplCanrightMaskedNoreuse ||
805 SecSBoxImpl == SBoxImplDom)) ||
806 (!SecMasking &&
807 (SecSBoxImpl == SBoxImplLut ||

```

```

808     SecSBoxImpl == SBoxImplCanright)))
809
810     // Signals used for assertions only.
811     logic prd_clearing_equals_output, un-
used_prd_clearing_equals_output;

812     assign prd_clearing_equals_output = (prd_clearing_state_i ==
add_round_key_out);
813     assign unused_prd_clearing_equals_output = prd_clear-
ing_equals_output;
814
815     // Ensure that the state register gets cleared with pseudo-
random data at the end of the last
816     // round. The following two scenarios are unlikely but not
illegal:
817     // 1. The newly loaded initial state matches the previous
output (the round counter is only
818     // cleared upon loading the new initial state).
819     // 2. The previous pseudo-random data is equal to the previ-
ous output.
820     // Otherwise, we must see an alert e.g. because the state
multiplexer got glitched.
821     `ASSERT(AesSecCmDataRegKeySca, (state_we == SP2V_HIGH) &&
822     ((key_len_i == AES_128 && u_aes_cipher_control.rnd_ctr
== 4'd10) ||
823     (key_len_i == AES_192 && u_aes_cipher_control.rnd_ctr
== 4'd12) ||
824     (key_len_i == AES_256 && u_aes_cipher_control.rnd_ctr
== 4'd14)) |>
825     (state_q != $past(add_round_key_out)) ||
826     (state_q == $past(state_init_i)) ||
827     $past(prd_clearing_equals_output) || alert_o)
828
829     if (SecMasking) begin : gen_sec_cm_key_masking_svas
830     // The number of clock cycles a regular AES round takes
- only used for assertions.
831     localparam int unsigned NumCyclesPerRound = (SecSBoxImpl
== SBoxImplDom) ? 5 : 1;
832     logic unused_param;
833     assign unused_param = (NumCyclesPerRound == 1);
834     // Ensure that SubBytes gets fresh PRD input for every
evaluation unless mask forcing is
835     // enabled. We effectively check that the PRNG has been
updated at least once within the
836     // last NumCyclesPerRound cycles. This also holds for
the very first round, as the PRNG
837     // is always updated in the last cycle of the IDLE state
and/or the first cycle of the
838     // INIT state.
839     `ASSERT(AesSecCmKeyMaskingPrdSubBytes,

840     sub_bytes_en == SP2V_HIGH && ($past(sub_bytes_en) ==
SP2V_LOW ||
841     ($past(sub_bytes_out_req) == SP2V_HIGH &&
842     $past(sub_bytes_out_ack) == SP2V_HIGH)) |>
843     $past(prd_sub_bytes_q) != $past(prd_sub_bytes_q,
NumCyclesPerRound + 1) ||
844     SecAllowForcingMasks && force_masks_i)
845
846     // Ensure that the PRNG has been updated between masking
the input and starting the first
847     // SubBytes evaluation/KeyExpand operation unless mask
forcing is enabled. For AES-256,
848     // we just spend 1 cycle in the INIT state and KeyExpand
isn't evaluating its S-Boxes,
849     // i.e., no fresh randomness is required. For the other
key lengths, KeyExpand evaluates
850     // its S-Boxes which takes NumCyclesPerRound cycles.
When computing the start key for
851     // decryption, the input isn't loaded and the PRNG is
thus not advanced.
852     `ASSERT(AesSecCmKeyMaskingInitialPrngUpdateSubBytes,
853     sub_bytes_en == SP2V_HIGH && $past(sub_bytes_en) ==
SP2V_LOW |>
854     (key_len_i == AES_256 &&
855     $past(prd_masking) != $past(prd_masking, 3)) ||
856     ((key_len_i == AES_128 || key_len_i == AES_192) &&
857     $past(prd_masking) != $past(prd_masking, Num-
CyclesPerRound + 2)) ||
858     (SecAllowForcingMasks && force_masks_i))
859     `ASSERT(AesSecCmKeyMaskingInitialPrngUpdateKeyExpand,
860     key_expand_en == SP2V_HIGH && $past(key_expand_en)
== SP2V_LOW |>
861     (key_len_i == AES_256 &&
862     $past(prd_masking) != $past(prd_masking, 3)) ||
863     ((key_len_i == AES_128 || key_len_i == AES_192) &&
864     $past(prd_masking) != $past(prd_masking, 2)) ||
865     (SecAllowForcingMasks && force_masks_i) ||
dec_key_gen_o == SP2V_HIGH)
866
867     // Ensure none of the state shares keeps being constant
during encryption/decryption
868     // unless mask forcing is enabled. Even though unlikely
it's not impossible that one
869     // share remains constant throughout one round. The SVAs
thus only fire if a share

```

```

870     // remains constant across two rounds.
871     for (genvar s = 0; s < NumShares; s++) begin :
gen_sec_cm_key_masking_share_svas
872     `ASSERT(AesSecCmKeyMaskingStateShare, state_we ==
SP2V_HIGH &&
873     (crypt_i == SP2V_HIGH || crypt_o == SP2V_HIGH) |>
874     state_q[s] != $past(state_q[s], NumCyclesPerRound)
||
875     $past(state_q[s], NumCyclesPerRound) !=
$past(state_q[s], 2*NumCyclesPerRound) ||
876     (SecAllowForcingMasks && force_masks_i) ||
dec_key_gen_o == SP2V_HIGH)
877     `ASSERT(AesSecCmKeyMaskingOutputShare,
878     (out_valid_o == SP2V_HIGH && $past(out_valid_o) ==
SP2V_LOW) &&
879     (crypt_o == SP2V_HIGH) |>
880     $past(state_o[s]) != $past(state_q[s], NumCycles-
PerRound) ||
881     $past(state_q[s], NumCyclesPerRound) !=
$past(state_q[s], 2*NumCyclesPerRound) ||
882     (SecAllowForcingMasks && force_masks_i) ||
dec_key_gen_o == SP2V_HIGH)
883     end
884     end
885
886     // Make sure the output of the masking PRNG is properly ex-
tracted without creating overlaps
887     // in the data input masks, or between the PRD fed to the
key expand module and SubBytes.
888     if (WidthPRDSBox > 8) begin : gen_prd_extract_assert
889     // For one row of the state matrix, extract the WidthPRDS-
Box-8 MSBs of the per-S-Box PRD from
890     // the PRNG output.
891     function automatic logic [3:0][(WidthPRDSBox-8)-1:0]
aes_prd_get_msbs(
892     logic [(4*WidthPRDSBox)-1:0] in
893     );
894     logic [3:0][(WidthPRDSBox-8)-1:0] prd_msbs;
895     for (int i = 0; i < 4; i++) begin
896     prd_msbs[i] = in[(i*WidthPRDSBox) + 8 +: (WidthPRDS-
Box-8)];
897     end
898     return prd_msbs;
899     endfunction
900
901     // For one row of the state matrix, undo the extraction of
LSBs and MSBs of the per-S-Box PRD
902     // from the PRNG output. This can be used to verify proper
extraction (no overlap of output
903     // masks and PRD for masked Canright S-Box implementa-
tions, no unused PRNG output).
904     function automatic logic [4*WidthPRDSBox-1:0] aes_prd_con-
cat_bits(
905     logic [3:0] [7:0] prd_lsb,
906     logic [3:0][(WidthPRDSBox-8)-1:0] prd_msbs
907     );
908     logic [(4*WidthPRDSBox)-1:0] prd;
909     for (int i = 0; i < 4; i++) begin
910     prd[(i*WidthPRDSBox) +: WidthPRDSBox] = {prd_msbs[i],
prd_lsb[i]};
911     end
912     return prd;
913     endfunction
914
915     // Check for correct extraction of masking PRNG output
without overlaps.
916     logic [WidthPRDMasking-1:0] unused_prd_masking;
917     logic [3:0][3:0][(WidthPRDSBox-8)-1:0] unused_prd_msbs;
918     for (genvar i = 0; i < 4; i++) begin : gen_unused_prd_msbs
919     assign unused_prd_msbs[i] =
aes_prd_get_msbs(prd_masking[i * WidthPRDRow +: WidthPRDRow]);
920     end
921     for (genvar i = 0; i < 4; i++) begin : gen_un-
used_prd_masking
922     assign unused_prd_masking[i * WidthPRDRow +: Width-
PRDRow] =
923     aes_prd_concat_bits(data_in_mask[i], un-
used_prd_msbs[i]);
924     end
925     assign unused_prd_masking[WidthPRDMasking-1 -: Width-
PRDRow] = prd_key_expand;
926     `ASSERT(AesMskgPrdExtraction, prd_masking == un-
used_prd_masking)
927     end
928     //VCS coverage on
929     // pragma coverage on
930     `endif
931
932 endmodule

```

7. lowRISC_opentitan/hw/ipl/aes/rtl/aes_control.sv (646 lines)

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for details.

```

3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES main control
6 //
7 // This module controls the interplay of input/output registers and the AES cipher core.
8
9 `include "prim_assert.sv"
10
11 module aes_control
12   import aes_pkg::*;
13   import aes_reg_pkg::*;
14   #(
15     parameter bit AESGCMEnable = 0,
16     parameter bit SecMasking = 0,
17     parameter int unsigned SecStartTriggerDelay = 0
18   ) (
19     input logic clk_i,
20     input logic rst_ni,
21
22     // Main control signals
23     input logic ctrl_qe_i,
24     output logic ctrl_we_o,
25     input logic ctrl_phase_i,
26     input logic ctrl_err_storage_i,
27     input aes_op_e op_i,
28     input aes_mode_e mode_i,
29     input ciph_op_e cipher_op_i,
30     input logic sideload_i,
31     input prs_rate_e prng_reseed_rate_i,
32     input logic manual_operation_i,
33     input logic key_touch_forces_reseed_i,
34     input logic ctrl_gcm_qe_i,
35     output logic ctrl_gcm_we_o,
36     input logic ctrl_gcm_phase_i,
37     output logic gcm_init_done_o,
38     input gcm_phase_e gcm_phase_i,
39     input logic start_i,
40     input logic key_iv_data_in_clear_i,
41     input logic data_out_clear_i,
42     input logic prng_reseed_i,
43     input logic mux_sel_err_i,
44     input logic sp_enc_err_i,
45     input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
46     input logic alert_fatal_i,
47     output logic alert_o,
48
49     // I/O register read/write enables
50     input logic key_sideload_valid_i,
51     input logic [NumRegsKey-1:0] key_init_qe_i [NumShares-
Key],
52     input logic [NumRegsIv-1:0] iv_qe_i,
53     input logic [NumRegsData-1:0] data_in_qe_i,
54     input logic [NumRegsData-1:0] data_out_re_i,
55     output logic data_in_we_o,
56     output data_out_sel_e data_out_sel_o,
57     output sp2v_e data_out_we_o,
58
59     // Previous input data register
60     output dip_sel_e data_in_prev_sel_o,
61     output sp2v_e data_in_prev_we_o,
62
63     // Cipher I/O muxes
64     output si_sel_e state_in_sel_o,
65     output add_si_sel_e add_state_in_sel_o,
66     output add_so_sel_e add_state_out_sel_o,
67
68     // Counter
69     output sp2v_e ctr_inc32_o,
70     output sp2v_e ctr_incr_o,
71     input sp2v_e ctr_ready_i,
72     input sp2v_e [NumSlicesCtr-1:0] ctr_we_i,
73
74     // Cipher core control and sync
75     output sp2v_e cipher_in_valid_o,
76     input sp2v_e cipher_in_ready_i,
77     input sp2v_e cipher_out_valid_i,
78     output sp2v_e cipher_out_ready_o,
79     output sp2v_e cipher_crypt_o,
80     input sp2v_e cipher_crypt_i,
81     output sp2v_e cipher_dec_key_gen_o,
82     input sp2v_e cipher_dec_key_gen_i,
83     output logic cipher_prng_reseed_o,
84     input logic cipher_prng_reseed_i,
85     output logic cipher_key_clear_o,
86     input logic cipher_key_clear_i,
87     output logic cipher_data_out_clear_o,
88     input logic cipher_data_out_clear_i,
89
90     // GHASH control and sync
91     output sp2v_e ghash_in_valid_o,
92     input sp2v_e ghash_in_ready_i,
93     input sp2v_e ghash_out_valid_i,
94     output sp2v_e ghash_out_ready_o,
95     output sp2v_e ghash_load_hash_subkey_o,
96
97     // Initial key registers
98     output key_init_sel_e key_init_sel_o,
99     output sp2v_e [NumRegsKey-1:0] key_init_we_o [NumShares-
Key],
100

```

```

101 // IV registers
102 output iv_sel_e iv_sel_o,
103 output sp2v_e [NumSlicesCtr-1:0] iv_we_o,
104
105 // Pseudo-random number generator interface
106 output logic prng_update_o,
107 output logic prng_reseed_req_o,
108 input logic prng_reseed_ack_i,
109
110 // Trigger register
111 output logic start_o,
112 output logic start_we_o,
113 output logic key_iv_data_in_clear_o,
114 output logic key_iv_data_in_clear_we_o,
115 output logic data_out_clear_o,
116 output logic data_out_clear_we_o,
117 output logic prng_reseed_o,
118 output logic prng_reseed_we_o,
119
120 // Status register
121 output logic idle_o,
122 output logic idle_we_o,
123 output logic stall_o,
124 output logic stall_we_o,
125 input logic output_lost_i,
126 output logic output_lost_o,
127 output logic output_lost_we_o,
128 output logic output_valid_o,
129 output logic output_valid_we_o,
130 output logic input_ready_o,
131 output logic input_ready_we_o,
132 );
133
134 // Optional delay of manual start trigger
135 logic start_trigger;
136
137 // Create a lint error to reduce the risk of accidentally
enabling this feature.
138 `ASSERT_STATIC_LINT_ERROR(AesSecStartTriggerDelayNonDefault,
SecStartTriggerDelay == 0)
139
140 if (SecStartTriggerDelay > 0) begin : gen_start_delay
141   // Delay the manual start trigger input for SCA measure-
ments.
142   localparam int unsigned WidthCounter = $clog2(SecStartTriggerDelay+1);
143   logic [WidthCounter-1:0] count_d, count_q;
144
145   // Clear counter when input goes low. Keep value if the
specified
146   delay is reached.
147   assign count_d = !start_i ? '0 :
start_trigger ? count_q : count_q +
1'b1;
148   assign start_trigger = (count_q == SecStartTrigger-
Delay[WidthCounter-1:0]) ? 1'b1 : 1'b0;
149
150   always_ff @(posedge clk_i or negedge rst_ni) begin
151     if (!rst_ni) begin
152       count_q <= '0;
153     end else begin
154       count_q <= count_d;
155     end
156   end
157
158 end else begin : gen_no_start_delay
159   // Directly forward the manual start trigger input.
160   assign start_trigger = start_i;
161 end
162
163 // Signals
164 sp2v_e ctr_ready;
165 sp2v_e [NumSlicesCtr-1:0] ctr_we;
166 sp2v_e cipher_in_ready;
167 sp2v_e cipher_out_valid;
168 sp2v_e cipher_crypt;
169 sp2v_e cipher_dec_key_gen;
170 sp2v_e ghash_in_ready;
171 sp2v_e ghash_out_valid;
172 logic mux_sel_err;
173 logic mr_err;
174 logic sp_enc_err;
175
176 // Sparsified FSM signals. These are needed for connecting
the individual bits of the Sp2V
177 // signals to the single-rail FSMs.
178 logic [Sp2VWidth-1:0] sp_data_out_we;
179 logic [Sp2VWidth-1:0] sp_data_in_prev_we;
180 logic [Sp2VWidth-1:0] sp_ctr_inc32;
181 logic [Sp2VWidth-1:0] sp_ctr_incr;
182 logic [Sp2VWidth-1:0] sp_ctr_ready;
183 logic [Sp2VWidth-1:0] sp_cipher_in_valid;
184 logic [Sp2VWidth-1:0] sp_cipher_in_ready;
185 logic [Sp2VWidth-1:0] sp_cipher_out_valid;
186 logic [Sp2VWidth-1:0] sp_cipher_out_ready;
187 logic [Sp2VWidth-1:0] sp_in_cipher_crypt;
188 logic [Sp2VWidth-1:0] sp_out_cipher_crypt;
189 logic [Sp2VWidth-1:0] sp_in_cipher_dec_key_gen;
190 logic [Sp2VWidth-1:0] sp_out_cipher_dec_key_gen;
191 logic [Sp2VWidth-1:0] sp_ghash_in_valid;
192 logic [Sp2VWidth-1:0] sp_ghash_in_ready;

```

```

193 logic [Sp2VWidth-1:0] sp_ghash_out_valid;
194 logic [Sp2VWidth-1:0] sp_ghash_out_ready;
195 logic [Sp2VWidth-1:0] sp_ghash_load_hash_subkey;
196
197 // Multi-rail signals. These are outputs of the single-rail
FSMs and need combining.
198 logic [Sp2VWidth-1:0] mr_ctrl_we;
199 logic [Sp2VWidth-1:0] mr_ctrl_gcm_we;
200 logic [Sp2VWidth-1:0] mr_gcm_init_done;
201 logic [Sp2VWidth-1:0] mr_alert;
202 logic [Sp2VWidth-1:0] mr_data_in_we;
203 data_out_sel_e [Sp2VWidth-1:0] mr_data_out_sel;
204 dip_sel_e [Sp2VWidth-1:0] mr_data_in_prev_sel;
205 si_sel_e [Sp2VWidth-1:0] mr_state_in_sel;
206 add_si_sel_e [Sp2VWidth-1:0] mr_add_state_in_sel;
207 add_so_sel_e [Sp2VWidth-1:0] mr_add_state_out_sel;
208 logic [Sp2VWidth-1:0] mr_cipher_prng_reseed;
209 logic [Sp2VWidth-1:0] mr_cipher_key_clear;
210 logic [Sp2VWidth-1:0] mr_cipher_data_out_clear;
211 key_init_sel_e [Sp2VWidth-1:0] mr_key_init_sel;
212 iv_sel_e [Sp2VWidth-1:0] mr_iv_sel;
213 logic [Sp2VWidth-1:0] mr_prng_update;
214 logic [Sp2VWidth-1:0] mr_prng_reseed_req;
215 logic [Sp2VWidth-1:0] mr_start_we;
216 logic [Sp2VWidth-1:0] mr_key_iv_data_in_clear_we;
217 logic [Sp2VWidth-1:0] mr_data_out_clear_we;
218 logic [Sp2VWidth-1:0] mr_prng_reseed;
219 logic [Sp2VWidth-1:0] mr_prng_reseed_we;
220 logic [Sp2VWidth-1:0] mr_idle;
221 logic [Sp2VWidth-1:0] mr_idle_we;
222 logic [Sp2VWidth-1:0] mr_stall;
223 logic [Sp2VWidth-1:0] mr_stall_we;
224 logic [Sp2VWidth-1:0] mr_output_lost;
225 logic [Sp2VWidth-1:0] mr_output_lost_we;
226 logic [Sp2VWidth-1:0] mr_output_valid;
227 logic [Sp2VWidth-1:0] mr_output_valid_we;
228 logic [Sp2VWidth-1:0] mr_input_ready;
229 logic [Sp2VWidth-1:0] mr_input_ready_we;
230
231 // To ease interfacing with the individual FSM rails, some
signals need to be converted to packed
232 // arrays.
233 logic [Sp2VWidth-1:0][NumSharesKey-1:0][NumRegs-
Key-1:0] int_key_init_we;
234 logic [NumSharesKey-1:0][NumRegsKey-1:0]
[Sp2VWidth-1:0] log_key_init_we;
235 logic [NumSharesKey-1:0][NumRegs-
Key-1:0] int_key_init_qe;
236 for (genvar s = 0; s < NumSharesKey; s++) begin :
gen_conv_key_init_wqe_shares
237 for (genvar i = 0; i < NumRegsKey; i++) begin :
gen_conv_key_init_wqe_regs
238 assign int_key_init_qe[s][i] = key_init_qe[i][s];
239 for (genvar j = 0; j < Sp2VWidth; j++) begin :
gen_conv_key_init_wqe_log
240 assign log_key_init_we[s][i][j] = int_key_init_we[j]
[s][i];
241 end
242 assign key_init_we_o[s][i] = sp2v_e'(log_key_init_we[s]
[i]);
243 end
244 end
245 logic [Sp2VWidth-1:0][NumSlicesCtr-1:0]
int_ctr_we;
246 logic [NumSlicesCtr-1:0][Sp2VWidth-1:0]
log_ctr_we;
247 logic [Sp2VWidth-1:0][NumSlicesCtr-1:0]
int_iv_we;
248 logic [NumSlicesCtr-1:0][Sp2VWidth-1:0]
log_iv_we;
249 for (genvar i = 0; i < NumSlicesCtr; i++) begin :
gen_conv_ctr_iv_we_slices
250 assign log_ctr_we[i] = {ctr_we[i]};
251 for (genvar j = 0; j < Sp2VWidth; j++) begin :
gen_conv_ctr_iv_we_log
252 assign int_ctr_we[j][i] = log_ctr_we[i][j];
253 assign log_iv_we[i][j] = int_iv_we[j][i];
254 end
255 assign iv_we_o[i] = sp2v_e'(log_iv_we[i]);
256 end
257
258 // FSM
259 //
260 //
261
262 // Convert sp2v_e signals to sparsified inputs.
263 assign sp_ctr_ready = {ctr_ready};
264 assign sp_cipher_in_ready = {cipher_in_ready};
265 assign sp_cipher_out_valid = {cipher_out_valid};
266 assign sp_in_cipher_crypt = {cipher_crypt};
267 assign sp_in_cipher_dec_key_gen = {cipher_dec_key_gen};
268 assign sp_ghash_in_ready = {ghash_in_ready};
269 assign sp_ghash_out_valid = {ghash_out_valid};
270
271 // SEC_CM: MAIN.FSM.REDUN
272 // For every bit in the Sp2V signals, one separate rail is
instantiated. The inputs and outputs
273 // of every rail are buffered to prevent aggressive syntheses
optimizations.
274 for (genvar i = 0; i < Sp2VWidth; i++) begin : gen_fsm
275 if (SP2V_LOGIC_HIGH[i] == 1'b1) begin : gen_fsm_p

```

```

276 aes_control_fsm_p #(
277 .AESGCMEnable ( AESGCMEnable ),
278 .SecMasking ( SecMasking )
279 ) u_aes_control_fsm_i (
280 .clk_i
281 .rst_ni
282 ( rst_ni ),
283 .ctrl_qe_i
284 ( ctrl_qe_i ),
285 .ctrl_we_o ( mr_c-
trl_we[i] ), // AND-combine
286 .ctrl_phase_i
287 ( ctrl_phase_i ),
288 .ctrl_err_storage_i ( ctrl_err_stor-
age_i ),
289 .op_i
290 ( op_i ),
291 .mode_i
292 ( mode_i ),
293 .cipher_op_i
294 ( cipher_op_i ),
295 .sideload_i ( side-
load_i ),
296 .prng_reseed_rate_i ( prng_re-
seed_rate_i ),
297 .manual_operation_i ( manu-
al_operation_i ),
298 .key_touch_forces_reseed_i
299 ( key_touch_forces_reseed_i ),
300 .ctrl_gcm_qe_i
301 ( ctrl_gcm_qe_i ),
302 .ctrl_gcm_we_o ( mr_c-
trl_gcm_we[i] ), // AND-combine
303 .ctrl_gcm_phase_i
304 ( ctrl_gcm_phase_i ),
305 .gcm_init_done_o ( mr_gcm_in-
it_done[i] ), // AND-combine
306 .gcm_phase_i
307 ( gcm_phase_i ),
308 .start_i
309 ( start_trigger ),
310 .key_iv_data_in_clear_i
311 ( key_iv_data_in_clear_i ),
312 .data_out_clear_i
313 ( data_out_clear_i ),
314 .prng_reseed_i ( prng_re-
seed_i ),
315 .mux_sel_err_i
316 ( mux_sel_err ),
317 .sp_enc_err_i
318 ( sp_enc_err ),
319 .lc_escalate_en_i ( lc_escal-
ate_en_i ),
320 .alert_fatal_i
321 ( alert_fatal_i ),
322 .alert_o
323 ( mr_alert[i] ), // OR-combine
324 .key_sideload_valid_i ( key_side-
load_valid_i ),
325 .key_init_qe_i ( int_key_in-
it_qe ),
326 .iv_qe_i
327 ( iv_qe_i ),
328 .data_in_qe_i
329 ( data_in_qe_i ),
330 .data_out_re_i
331 ( data_out_re_i ),
332 .data_in_we_o
333 ( mr_data_in_we[i] ), // AND-combine
334 .data_out_sel_o
335 ( mr_data_out_sel[i] ), // OR-combine
336 .data_out_we_o
337 ( sp_data_out_we[i] ), // Sparsified
338 .data_in_prev_sel_o
339 ( mr_data_in_prev_sel[i] ), // OR-combine
340 .data_in_prev_we_o
341 ( sp_data_in_prev_we[i] ), // Sparsified
342 .state_in_sel_o
343 ( mr_state_in_sel[i] ), // OR-combine
344 .add_state_in_sel_o
345 ( mr_add_state_in_sel[i] ), // OR-combine
346 .add_state_out_sel_o
347 ( mr_add_state_out_sel[i] ), // OR-combine
348 .ctr_inc32_o
349 ( sp_ctr_inc32[i] ), // Sparsified
350 .ctr_incr_o ( sp_ctr_in-
cr[i] ), // Sparsified
351 .ctr_ready_i
352 ( sp_ctr_ready[i] ), // Sparsified
353 .ctr_we_i
354 ( int_ctr_we[i] ), // Sparsified
355 .cipher_in_valid_o
356 ( sp_cipher_in_valid[i] ), // Sparsified
357 .cipher_in_ready_i

```

```

( sp_cipher_in_ready[i] ), // Sparsified
332 .cipher_out_valid_i
( sp_cipher_out_valid[i] ), // Sparsified
333 .cipher_out_ready_o
( sp_cipher_out_ready[i] ), // Sparsified
334 .cipher_crypt_o
( sp_out_cipher_crypt[i] ), // Sparsified
335 .cipher_crypt_i
( sp_in_cipher_crypt[i] ), // Sparsified
336 .cipher_dec_key_gen_o
( sp_out_cipher_dec_key_gen[i] ), // Sparsified
337 .cipher_dec_key_gen_i
( sp_in_cipher_dec_key_gen[i] ), // Sparsified
338 .cipher_prng_reseed_o
( mr_cipher_prng_reseed[i] ), // OR-combine
339 .cipher_prng_reseed_i
( cipher_prng_reseed_i ),
340 .cipher_key_clear_o
( mr_cipher_key_clear[i] ), // OR-combine
341 .cipher_key_clear_i
( cipher_key_clear_i ),
342 .cipher_data_out_clear_o
( mr_cipher_data_out_clear[i] ), // OR-combine
343 .cipher_data_out_clear_i
( cipher_data_out_clear_i ),
344
345 .ghash_in_valid_o
( sp_ghash_in_valid[i] ), // Sparsified
346 .ghash_in_ready_i
( sp_ghash_in_ready[i] ), // Sparsified
347 .ghash_out_valid_i
( sp_ghash_out_valid[i] ), // Sparsified
348 .ghash_out_ready_o
( sp_ghash_out_ready[i] ), // Sparsified
349 .ghash_load_hash_subkey_o
( sp_ghash_load_hash_subkey[i] ), // Sparsified
350
351 .key_init_sel_o ( mr_key_in-
it_sel[i] ), // OR-combine
352 .key_init_we_o ( int_key_in-
it_we[i] ), // Sparsified
353
354 .iv_sel_o
( mr_iv_sel[i] ), // OR-combine
355 .iv_we_o
( int_iv_we[i] ), // Sparsified
356
357 .prng_update_o
( mr_prng_update[i] ), // OR-combine
358 .prng_reseed_req_o
( mr_prng_reseed_req[i] ), // OR-combine
359 .prng_reseed_ack_i ( prng_re-
seed_ack_i ),
360
361 .start_we_o
( mr_start_we[i] ), // OR-combine
362 .key_iv_data_in_clear_we_o
( mr_key_iv_data_in_clear_we[i] ), // AND-combine
363 .data_out_clear_we_o
( mr_data_out_clear_we[i] ), // AND-combine
364 .prng_reseed_o
( mr_prng_reseed[i] ), // OR-combine
365 .prng_reseed_we_o
( mr_prng_reseed_we[i] ), // OR-combine
366
367 .idle_o
( mr_idle[i] ), // AND-combine
368 .idle_we_o
( mr_idle_we[i] ), // AND-combine
369 .stall_o
( mr_stall[i] ), // AND-combine
370 .stall_we_o
( mr_stall_we[i] ), // AND-combine
371 .output_lost_i ( out-
put_lost_i ), // AND-combine
372 .output_lost_o ( mr_out-
put_lost[i] ), // AND-combine
373 .output_lost_we_o ( mr_out-
put_lost_we[i] ), // AND-combine
374 .output_valid_o ( mr_out-
put_valid[i] ), // AND-combine
375 .output_valid_we_o ( mr_out-
put_valid_we[i] ), // AND-combine
376 .input_ready_o ( mr_in-
put_ready[i] ), // AND-combine
377 .input_ready_we_o ( mr_in-
put_ready_we[i] ), // AND-combine
378 );
379 end else begin : gen_fsm_n
380 aes_control_fsm_n #(
381 .AESGCMEEnable ( AESGCMEEnable ),
382 .SecMasking ( SecMasking )
383 ) u_aes_control_fsm_i (
384 .clk_i
( clk_i ),
385 .rst_ni
( rst_ni ),
386
387 .ctrl_qe_i
( ctrl_qe_i ),
388 .ctrl_we_o ( mr_c-

```

```

trl_we[i] ), // AND-combine
389 .ctrl_phase_i
( ctrl_phase_i ),
390 .ctrl_err_storage_i ( ctrl_err_stor-
age_i ),
391 .op_i
( op_i ),
392 .mode_i
( mode_i ),
393 .cipher_op_i
( cipher_op_i ),
394 .sideload_i ( side-
load_i ),
395 .prng_reseed_rate_i ( prng_re-
seed_rate_i ),
396 .manual_operation_i ( manu-
al_operation_i ),
397 .key_touch_forces_reseed_i
( key_touch_forces_reseed_i ),
398 .ctrl_gcm_qe_i
( ctrl_gcm_qe_i ),
399 .ctrl_gcm_we_o ( mr_c-
trl_gcm_we[i] ), // AND-combine
400 .ctrl_gcm_phase_i
( ctrl_gcm_phase_i ),
401 .gcm_init_done_o ( mr_gcm_in-
it_done[i] ), // AND-combine
402 .gcm_phase_i
( gcm_phase_i ),
403 .start_i
( start_trigger ),
404 .key_iv_data_in_clear_i
( key_iv_data_in_clear_i ),
405 .data_out_clear_i
( data_out_clear_i ),
406 .prng_reseed_i ( prng_re-
seed_i ),
407 .mux_sel_err_i
( mux_sel_err ),
408 .sp_enc_err_i
( sp_enc_err ),
409 .lc_escalate_en_i ( lc_escal-
ate_en_i ),
410 .alert_fatal_i
( alert_fatal_i ),
411 .alert_o
( mr_alert[i] ), // OR-combine
412
413 .key_sideload_valid_i ( key_side-
load_valid_i ),
414 .key_init_qe_i ( int_key_in-
it_qe ),
415 .iv_qe_i
( iv_qe_i ),
416 .data_in_qe_i
( data_in_qe_i ),
417 .data_out_re_i
( data_out_re_i ),
418 .data_in_we_o
( mr_data_in_we[i] ), // AND-combine
419 .data_out_sel_o
( mr_data_out_sel[i] ), // OR-combine
420 .data_out_we_no
( sp_data_out_we[i] ), // Sparsified
421
422 .data_in_prev_sel_o
( mr_data_in_prev_sel[i] ), // OR-combine
423 .data_in_prev_we_no
( sp_data_in_prev_we[i] ), // Sparsified
424
425 .state_in_sel_o
( mr_state_in_sel[i] ), // OR-combine
426 .add_state_in_sel_o
( mr_add_state_in_sel[i] ), // OR-combine
427 .add_state_out_sel_o
( mr_add_state_out_sel[i] ), // OR-combine
428
429 .ctr_inc32_no
( sp_ctr_inc32[i] ), // Sparsified
430 .ctr_incr_no ( sp_ctr_in-
cr[i] ), // Sparsified
431 .ctr_ready_ni
( sp_ctr_ready[i] ), // Sparsified
432 .ctr_we_ni
( int_ctr_we[i] ), // Sparsified
433
434 .cipher_in_valid_no
( sp_cipher_in_valid[i] ), // Sparsified
435 .cipher_in_ready_ni
( sp_cipher_in_ready[i] ), // Sparsified
436 .cipher_out_valid_ni
( sp_cipher_out_valid[i] ), // Sparsified
437 .cipher_out_ready_no
( sp_cipher_out_ready[i] ), // Sparsified
438 .cipher_crypt_no
( sp_out_cipher_crypt[i] ), // Sparsified
439 .cipher_crypt_ni
( sp_in_cipher_crypt[i] ), // Sparsified
440 .cipher_dec_key_gen_no
( sp_out_cipher_dec_key_gen[i] ), // Sparsified
441 .cipher_dec_key_gen_ni

```



```

( sp_in_cipher_dec_key_gen[i] ), // Sparsified
442 .cipher_prng_reseed_o
( mr_cipher_prng_reseed[i] ), // OR-combine
443 .cipher_prng_reseed_i
( cipher_prng_reseed_i ),
444 .cipher_key_clear_o
( mr_cipher_key_clear[i] ), // OR-combine
445 .cipher_key_clear_i
( cipher_key_clear_i ),
446 .cipher_data_out_clear_o
( mr_cipher_data_out_clear[i] ), // OR-combine
447 .cipher_data_out_clear_i
( cipher_data_out_clear_i ),
448
449 .ghash_in_valid_no
( sp_ghash_in_valid[i] ), // Sparsified
450 .ghash_in_ready_no
( sp_ghash_in_ready[i] ), // Sparsified
451 .ghash_out_valid_no
( sp_ghash_out_valid[i] ), // Sparsified
452 .ghash_out_ready_no
( sp_ghash_out_ready[i] ), // Sparsified
453 .ghash_load_hash_subkey_no
( sp_ghash_load_hash_subkey[i] ), // Sparsified
454
455 .key_init_sel_o ( mr_key_in-
it_sel[i] ), // OR-combine
456 .key_init_we_no ( int_key_in-
it_we[i] ), // Sparsified
457
458 .iv_sel_o
( mr_iv_sel[i] ), // OR-combine
459 .iv_we_no
( int_iv_we[i] ), // Sparsified
460
461 .prng_update_o
( mr_prng_update[i] ), // OR-combine
462 .prng_reseed_req_o
( mr_prng_reseed_req[i] ), // OR-combine
463 .prng_reseed_ack_i ( prng_re-
seed_ack_i ),
464
465 .start_we_o
( mr_start_we[i] ), // OR-combine
466 .key_iv_data_in_clear_we_o
( mr_key_iv_data_in_clear_we[i] ), // AND-combine
467 .data_out_clear_we_o
( mr_data_out_clear_we[i] ), // AND-combine
468 .prng_reseed_o
( mr_prng_reseed[i] ), // OR-combine
469 .prng_reseed_we_o
( mr_prng_reseed_we[i] ), // OR-combine
470
471 .idle_o
( mr_idle[i] ), // AND-combine
472 .idle_we_o
( mr_idle_we[i] ), // AND-combine
473 .stall_o
( mr_stall[i] ), // AND-combine
474 .stall_we_o
( mr_stall_we[i] ), // AND-combine
475 .output_lost_i ( out-
put_lost_i ), // AND-combine
476 .output_lost_o ( mr_out-
put_lost[i] ), // AND-combine
477 .output_lost_we_o ( mr_out-
put_lost_we[i] ), // AND-combine
478 .output_valid_o ( mr_out-
put_valid[i] ), // AND-combine
479 .output_valid_we_o ( mr_out-
put_valid_we[i] ), // AND-combine
480 .input_ready_o ( mr_in-
put_ready[i] ), // AND-combine
481 .input_ready_we_o ( mr_in-
put_ready_we[i] ), // AND-combine
482 );
483 end
484 end
485
486 // Convert sparsified outputs to sp2v_e type.
487 assign data_out_we_o = sp2v_e'(sp_data_out_we);
488 assign data_in_prev_we_o =
sp2v_e'(sp_data_in_prev_we);
489 assign ctr_inc32_o = sp2v_e'(sp_ctr_inc32);
490 assign ctr_incr_o = sp2v_e'(sp_ctr_incr);
491 assign cipher_in_valid_o =
sp2v_e'(sp_cipher_in_valid);
492 assign cipher_out_ready_o =
sp2v_e'(sp_cipher_out_ready);
493 assign cipher_crypt_o =
sp2v_e'(sp_out_cipher_crypt);
494 assign cipher_dec_key_gen_o =
sp2v_e'(sp_out_cipher_dec_key_gen);
495 assign ghash_in_valid_o =
sp2v_e'(sp_ghash_in_valid);
496 assign ghash_out_ready_o =
sp2v_e'(sp_ghash_out_ready);
497 assign ghash_load_hash_subkey_o =
sp2v_e'(sp_ghash_load_hash_subkey);
498
499 // Combine single-bit FSM outputs.

```

```

500 // OR: One bit is sufficient to drive the corresponding out-
put bit high.
501 assign alert_o = |mr_alert;
502 assign cipher_prng_reseed_o = |mr_cipher_prng_reseed;
503 assign cipher_key_clear_o = |mr_cipher_key_clear;
504 assign cipher_data_out_clear_o = |
mr_cipher_data_out_clear;
505 assign prng_update_o = |mr_prng_update;
506 assign prng_reseed_req_o = |mr_prng_reseed_req;
507 assign start_we_o = |mr_start_we;
508 assign prng_reseed_o = |mr_prng_reseed;
509 assign prng_reseed_we_o = |mr_prng_reseed_we;
510
511 // AND: Only if all bits are high, the corresponding action
should be triggered.
512 assign ctrl_we_o = &mr_ctrl_we;
513 assign ctrl_gcm_we_o = &mr_ctrl_gcm_we;
514 assign gcm_init_done_o = &mr_gcm_init_done;
515 assign data_in_we_o = &mr_data_in_we;
516 assign key_iv_data_in_clear_we_o =
&mr_key_iv_data_in_clear_we;
517 assign data_out_clear_we_o = &mr_data_out_clear_we;
518 assign idle_o = &mr_idle;
519 assign idle_we_o = &mr_idle_we;
520 assign stall_o = &mr_stall;
521 assign stall_we_o = &mr_stall_we;
522 assign output_lost_o = &mr_output_lost;
523 assign output_lost_we_o = &mr_output_lost_we;
524 assign output_valid_o = &mr_output_valid;
525 assign output_valid_we_o = &mr_output_valid_we;
526 assign input_ready_o = &mr_input_ready;
527 assign input_ready_we_o = &mr_input_ready_we;
528
529 // Combine multi-bit, sparse FSM outputs. We simply OR them
together. If the FSMs don't provide
530 // the same outputs, two cases are possible:
531 // - An invalid encoding results: A downstream checker will
fire, see mux_sel_err_i.
532 // - A valid encoding results: The outputs are compared be-
low to cover this case, see mr_err;
533 always_comb begin : combine_sparse_signals
534   data_out_sel_o = data_out_sel_e'({DataOutSel-
Width{1'b0}});
535   data_in_prev_sel_o = dip_sel_e'({DIPSelWidth{1'b0}});
536   state_in_sel_o = si_sel_e'({SISelWidth{1'b0}});
537   add_state_in_sel_o = add_si_sel_e'({AddSISel-
Width{1'b0}});
538   add_state_out_sel_o = add_so_sel_e'({AddSOSel-
Width{1'b0}});
539   key_init_sel_o = key_in-
it_sel_e'({KeyInitSelWidth{1'b0}});
540   iv_sel_o = iv_sel_e'({IVSelWidth{1'b0}});
541   mr_err = 1'b0;
542
543   for (int i = 0; i < Sp2VWidth; i++) begin
544     data_out_sel_o =
data_out_sel_e'({data_out_sel_o | {mr_data_out_sel[i]}});
545     data_in_prev_sel_o =
dip_sel_e'({data_in_prev_sel_o | {mr_data_in_prev_sel[i]}});
546     state_in_sel_o =
si_sel_e'({state_in_sel_o | {mr_state_in_sel[i]}});
547     add_state_in_sel_o =
add_si_sel_e'({add_state_in_sel_o | {mr_add_state_in_sel[i]}});
548     add_state_out_sel_o =
add_so_sel_e'({add_state_out_sel_o | {mr_add_state_out_sel[i]}});
549     key_init_sel_o = key_in-
it_sel_e'({key_init_sel_o | {mr_key_init_sel[i]}});
550     iv_sel_o =
iv_sel_e'({iv_sel_o | {mr_iv_sel[i]}});
551     end
552
553   for (int i = 0; i < Sp2VWidth; i++) begin
554     if (data_out_sel_o != mr_data_out_sel[i] ||
555         data_in_prev_sel_o != mr_data_in_prev_sel[i] ||
556         state_in_sel_o != mr_state_in_sel[i] ||
557         add_state_in_sel_o != mr_add_state_in_sel[i] ||
558         add_state_out_sel_o != mr_add_state_out_sel[i] ||
559         key_init_sel_o != mr_key_init_sel[i] ||
560         iv_sel_o != mr_iv_sel[i]) begin
561       mr_err = 1'b1;
562     end
563   end
564 end
565
566 // Collect errors in mux selector signals.
567 assign mux_sel_err = mux_sel_err_i | mr_err;
568
569 ///////////////////////////////////////////////////
570 // Sparsely Encoded Signals //
571 ///////////////////////////////////////////////////
572
573 // SEC_CM: CTRL.SPARSE
574 // We use sparse encodings for various critical signals and
must ensure that:
575 // 1. The synthesis tool doesn't optimize away the sparse
encoding.
576 // 2. The sparsely encoded signal is always valid. More pre-
cisely, an alert or SVA is triggered
577 // 3. if a sparse signal takes on an invalid value.
578 // 3. The alert signal remains asserted until reset even if
the sparse signal becomes valid again

```

```

579 // This is achieved by driving the control FSM into the
terminal error state whenever any
580 // sparsely encoded signal becomes invalid.
581 //
582 // If any sparsely encoded signal becomes invalid, the con-
troller further immediately de-asserts
583 // data_out_we_o and other write-enable signals to prevent
any data from being released.
584
585 // We use vectors of sparsely encoded signals to reduce code
duplication.
586 localparam int unsigned NumSp2VVSig = 7 + NumSlicesCtr;
587 sp2v_e [NumSp2VVSig-1:0] sp2v_sig;
588 sp2v_e [NumSp2VVSig-1:0] sp2v_sig_chk;
589 logic [NumSp2VVSig-1:0][Sp2VWidth-1:0] sp2v_sig_chk_raw;
590 logic [NumSp2VVSig-1:0] sp2v_sig_err;
591
592 assign sp2v_sig[0] = cipher_in_ready_i;
593 assign sp2v_sig[1] = cipher_out_valid_i;
594 assign sp2v_sig[2] = cipher_crypt_i;
595 assign sp2v_sig[3] = cipher_dec_key_gen_i;
596 assign sp2v_sig[4] = ghash_in_ready_i;
597 assign sp2v_sig[5] = ghash_out_valid_i;
598 assign sp2v_sig[6] = ctr_ready_i;
599 for (genvar i = 0; i < NumSlicesCtr; i++) begin :
gen_use_ctr_we_i
600     assign sp2v_sig[7+i] = ctr_we_i[i];
601 end
602
603 // All signals inside sp2v_sig are driven and consumed by
multi-rail FSMs.
604 localparam bit [NumSp2VVSig-1:0] Sp2VEnSecBuf = '0;
605
606 // Individually check sparsely encoded signals.
607 for (genvar i = 0; i < NumSp2VVSig; i++) begin :
gen_sel_buf_chk
608     aes_sel_buf_chk #(
609         .Num      ( Sp2VNum      ),
610         .Width    ( Sp2VWidth    ),
611         .EnSecBuf ( Sp2VEnSecBuf[i] )
612     ) u_aes_sp2v_sig_buf_chk_i (
613         .clk_i ( clk_i ),
614         .rst_ni ( rst_ni ),
615         .sel_i ( sp2v_sig[i] ),
616         .sel_o ( sp2v_sig_chk_raw[i] ),
617         .err_o ( sp2v_sig_err[i] )
618     );
619     assign sp2v_sig_chk[i] = sp2v_e'(sp2v_sig_chk_raw[i]);
620 end
621
622 assign cipher_in_ready    = sp2v_sig_chk[0];
623 assign cipher_out_valid  = sp2v_sig_chk[1];
624 assign cipher_crypt      = sp2v_sig_chk[2];
625 assign cipher_dec_key_gen = sp2v_sig_chk[3];
626 assign ghash_in_ready    = sp2v_sig_chk[4];
627 assign ghash_out_valid  = sp2v_sig_chk[5];
628 assign ctr_ready        = sp2v_sig_chk[6];
629 for (genvar i = 0; i < NumSlicesCtr; i++) begin : gen_ctr_we
630     assign ctr_we[i] = sp2v_sig_chk[7+i];
631 end
632
633 // Collect encoding errors.
634 // We instantiate the checker modules as close as possible
to where the sparsely encoded signals
635 // are used. Here, we collect also encoding errors detected
in other places of the core.
636 assign sp_enc_err = |sp2v_sig_err | sp_enc_err_i;
637
638 ///////////////////////////////////////////////////
639 // Trigger Register //
640 ///////////////////////////////////////////////////
641 // Most triggers are only ever cleared by control.
642 assign start_o = 1'b0;
643 assign key_iv_data_in_clear_o = 1'b0;
644 assign data_out_clear_o = 1'b0;
645
646 endmodule

```

8. lowRISC_opentitan/hw/ip/aes/rtl/aes_control_fsm.sv (1,202 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES main control FSM
6 //
7 // This module contains the main control FSM handling the in-
terplay of input/output registers and
8 // the AES cipher core.
9
10 `include "prim_assert.sv"
11
12 module aes_control_fsm
13     import aes_pkg::*;
14     import aes_reg_pkg::*;
15     #(

```

```

16     parameter bit AESGCMEEnable = 0,
17     parameter bit SecMasking    = 0
18 ) (
19     input  logic clk_i,
20     input  logic rst_ni,
21
22     // Main control signals
23     input  logic ctrl_qe_i,
24     output logic ctrl_we_o,
25     input  logic ctrl_phase_i,
26     input  logic ctrl_err_storage_i,
27     input  aes_op_e op_i,
28     input  aes_mode_e mode_i,
29     input  ciph_op_e cipher_op_i,
30     input  logic sideload_i,
31     input  prs_rate_e prng_re-
seed_rate_i,
32     input  logic manu-
al_operation_i,
33     input  logic key_touch_forces_reseed_i,
34     input  logic ctrl_gcm_qe_i,
35     output logic ctrl_gcm_we_o,
36     input  logic ctrl_gcm_phase_i,
37     output logic gcm_in-
it_done_o,
38     input  gcm_phase_e gcm_phase_i,
39     input  logic start_i,
40     input  logic key_iv_data_in_clear_i,
41     input  logic data_out_clear_i,
42     input  logic prng_re-
seed_i,
43     input  logic mux_sel_err_i,
44     input  logic sp_enc_err_i,
45     input  lc_ctrl_pkg::lc_tx_t lc_escal-
ate_en_i,
46     input  logic alert_fatal_i,
47     output logic alert_o,
48
49     // I/O register read/write enables
50     input  logic key_side-
load_valid_i,
51     input  logic [NumSharesKey-1:0][NumRegsKey-1:0] key_in-
it_qe_i,
52     input  logic [NumRegsIv-1:0] iv_qe_i,
53     input  logic [NumRegsData-1:0]
data_in_qe_i,
54     input  logic [NumRegsData-1:0]
data_out_re_i,
55     output logic data_in_we_o,
56     output logic data_out_sel_e
data_out_sel_o,
57     output logic data_out_we_o, // Sparsify
58
59     // Previous input data register
60     output dip_sel_e data_in_prev_sel_o,
61     output logic data_in_prev_we_o, // Sparsify
62
63     // Cipher I/O muxes
64     output si_sel_e state_in_sel_o,
65     output add_si_sel_e add_state_in_sel_o,
66     output add_so_sel_e add_state_out_sel_o,
67
68     // Counter
69     output logic ctr_inc32_o, // Sparsify
70     output logic ctr_o, // Sparsify
71     input  logic ctr_ready_i, // Sparsify
72     input  logic ctr_we_i, // Sparsify [NumSlicesCtr-1:0]
73
74     // Cipher core control and sync
75     output logic cipher_in_valid_o, // Sparsify
76     input  logic cipher_in_ready_i, // Sparsify
77     input  logic cipher_out_valid_i, // Sparsify
78     output logic cipher_out_ready_o, // Sparsify
79     output logic cipher_crypt_o, // Sparsify

```

```

80  input logic
cipher_crypt_i,          // Sparsify
81  output logic
cipher_dec_key_gen_o,    // Sparsify
82  input logic
cipher_dec_key_gen_i,    // Sparsify
83  output logic
cipher_prng_reseed_o,
84  input logic
cipher_prng_reseed_i,
85  output logic
cipher_key_clear_o,
86  input logic
cipher_key_clear_i,
87  output logic
cipher_data_out_clear_o,
88  input logic
cipher_data_out_clear_i,
89
90  // GHASH control and sync
91  output logic
ghash_in_valid_o,        // Sparsify
92  input logic
ghash_in_ready_i,        // Sparsify
93  input logic
ghash_out_valid_i,       // Sparsify
94  output logic
ghash_out_ready_o,       // Sparsify
95  output logic
ghash_load_hash_subkey_o, // Sparsify
96
97  // Initial key registers
98  output key_init_sel_e          key_in-
it_sel_o,
99  output logic [NumSharesKey-1:0][NumRegsKey-1:0] key_in-
it_we_o,          // Sparsify
100
101  // IV registers
102  output iv_sel_e
103  output logic [NumSlicesCtr-1:0] iv_sel_o,
iv_we_o,          // Sparsify
104
105  // Pseudo-random number generator interface
106  output logic
prng_up-
date_o,
107  output logic
prng_re-
seed_req_o,
108  input logic
prng_re-
seed_ack_i,
109
110  // Trigger register
111  output logic
start_we_o,
112  output logic
key_iv_data_in_clear_we_o,
113  output logic
data_out_clear_we_o,
114  output logic
prng_re-
seed_o,
115  output logic
prng_re-
seed_we_o,
116
117  // Status register
118  output logic
idle_o,
119  output logic
idle_we_o,
120  output logic
stall_o,
121  output logic
stall_we_o,
122  input logic
out-
put_lost_i,
123  output logic
out-
put_lost_o,
124  output logic
out-
put_lost_we_o,
125  output logic
out-
put_valid_o,
126  output logic
out-
put_valid_we_o,
127  output logic
in-
put_ready_o,
128  output logic
in-
put_ready_we_o
129 );
130
131  // Signals
132  aes_ctrl_e          aes_ctrl_ns, aes_ctrl_cs;
133  logic
prng_reseed_done_d, prng_re-
seed_done_q;
134
135  logic
key_init_clear;
136  logic
key_init_new;
137  logic
key_init_new_pulse;
138  logic
key_init_load;
139  logic
key_init_arm;
140  logic
key_init_ready;
141  logic
key_sideload;
142
143  logic [NumSlicesCtr-1:0] iv_qe;
144  logic
iv_clear;
145  logic
iv_load;
146  logic
iv_arm;
147  logic
iv_ready;
148

```

```

149  logic [NumRegsData-1:0] data_in_new_d, data_in_new_q;
150  logic
data_in_new;
151  logic
data_in_load;
152
153  logic [NumRegsData-1:0] data_out_read_d, data_out_read_q;
154  logic
data_out_read;
155  logic
output_valid_q;
156
157  logic
cfg_valid;
158  logic
no_alert;
159  logic
cipher_op_err;
160  logic
start_common, start_ecb,
start_cbc, start_cfb, start_ofb, start_ctr;
161  logic
start;
162  logic
start_core;
163  logic
finish;
164  logic
crypt;
165  logic
cipher_out_done;
166  logic
doing_cbc_enc, doing_cbc_dec;
167  logic
doing_cfb_enc, doing_cfb_dec;
168  logic
doing_ofb;
169  logic
doing_ctr;
170  logic
ctrl_we_q;
171  logic
clear_in_out_status;
172  logic
clear_on_fatal;
173
174  logic
start_we;
175  logic
key_iv_data_in_clear_we;
176  logic
data_out_clear_we;
177  logic
prng_reseed_we;
178
179  logic
idle;
180  logic
idle_we;
181  logic
stall;
182  logic
stall_we;
183  logic
output_lost;
184  logic
output_lost_we;
185  logic
output_valid;
186  logic
output_valid_we;
187  logic
input_ready;
188  logic
input_ready_we;
189
190  logic
ctrl_gcm_we_q;
191  logic
gcm_clear;
192  logic
gcm_init, gcm_restore, gcm_aad,
gcm_txt, gcm_save, gcm_tag;
193  logic
start_common_gcm, start_ghash;
194  logic
start_gcm_init, start_gcm_hsk,
start_gcm_s;
195  logic
start_gcm_restore, start_gcm_aad,
start_gcm_txt, start_gcm_save;
196  logic
start_gcm_tag;
197  logic
doing_gcm_hsk, doing_gcm_s, do-
ing_gcm_txt;
198  logic
hash_subkey_ready_d, hash_sub-
key_ready_q;
199  logic
s_ready_d, s_ready_q;
200  logic
doing_gcm_restore_d, do-
ing_gcm_restore_q;
201  logic
doing_gcm_aad_d, doing_gcm_aad_q;
202  logic
doing_gcm_tag_d, doing_gcm_tag_q;
203  logic
doing_gcm_save_d, do-
ing_gcm_save_q;
204  logic
ghash_out_done;
205  logic
ghash_idle;
206
207  logic
block_ctr_expr;
208  logic
block_ctr_decr;
209
210  // Software updates IV in chunks of 32 bits, the counter up-
dates SliceSizeCtr bits at a time.
211  // Convert word write enable to internal half-word write en-
able.
212  assign iv_qe = {iv_qe_i[3], iv_qe_i[3], iv_qe_i[2],
iv_qe_i[2],
213  iv_qe_i[1], iv_qe_i[1], iv_qe_i[0],
iv_qe_i[0]};
214
215  // The cipher core is only ever allowed to start or finish
if the control register holds a valid
216  // configuration and if no fatal alert condition occurred.
217  assign cfg_valid = ~(mode_i == AES_NONE) | ctrl_err_stor-
age_i;
218  assign no_alert = ~alert_fatal_i;
219
220  // cipher_op_i is obtained from the configuration of the
control register with additional logic.
221  assign cipher_op_err = ~(cipher_op_i == CIPH_FWD ||
cipher_op_i == CIPH_INV);
222
223  // Check common start conditions. These are needed for any
mode, unless we are running in
224  // manual mode.
225  assign start_common = key_init_ready & data_in_new &
226  // If key sideload is enabled, we only start if the key
is valid.
(sideload_i ? key_sideload_valid_i : 1'b1);
227
228
229  // Check mode-specific start conditions. If the IV (and
counter) is needed, we only start if
230  // also the IV (and counter) is ready.

```



```

231 assign start_ecb = (mode_i == AES_ECB);
232 assign start_cbc = (mode_i == AES_CBC) & iv_ready;
233 assign start_cfb = (mode_i == AES_CFB) & iv_ready;
234 assign start_ofb = (mode_i == AES_OFB) & iv_ready;
235 assign start_ctr = (mode_i == AES_CTR) & iv_ready &
ctr_ready_i;
236
237 // If set to start manually, we just wait for the trigger.
Otherwise, check common as well as
238 // mode-specific start conditions.
239 assign start = cfg_valid & no_alert &
240 // Manual operation has priority.
241 (manual_operation_i ? start_i :
242 // Check start conditions for automatic operation.
243 ((start_ecb |
244 start_cbc |
245 start_cfb |
246 start_ofb |
247 start_ctr) & start_common) |
248 // Only the initial as well as the ciphertext/plain-
text phases of GCM require operating
249 // the AES cipher core. Common start conditions are
already factored into these signals.
250 (start_gcm_init |
251 start_gcm_txt));
252
253 // If not set to overwrite data, we wait for any previous
output data to be read. data_out_read
254 // synchronously clears output_valid_q, unless new output
data is written in the exact same
255 // clock cycle.
256 assign finish = cfg_valid & no_alert &
257 // Manual operation has priority.
258 (manual_operation_i ? 1'b1 :
259 // Make sure previous output data has been read.
260 (~output_valid_q | data_out_read));
261
262 ///////////////////////////////////////////////////
263 // GCM Start Conditions //
264 ///////////////////////////////////////////////////
265 // Note that only the initial as well as the ciphertext/
plaintext phases of GCM require operating
266 // the AES cipher core. The start conditions for these
phases are factored into the regular start
267 // signal above. In contrast, the restore, AAD, save and tag
phases use the GHASH block only. The
268 // corresponding start conditions are handled separately by
the FSM and we need to explicitly
269 // factor in manual operation below.
270
271 assign gcm_init = (mode_i == AES_GCM) & (gcm_phase_i ==
GCM_INIT);
272 assign gcm_restore = (mode_i == AES_GCM) & (gcm_phase_i ==
GCM_RESTORE);
273 assign gcm_aad = (mode_i == AES_GCM) & (gcm_phase_i ==
GCM_AAD);
274 assign gcm_txt = (mode_i == AES_GCM) & (gcm_phase_i ==
GCM_TXT);
275 assign gcm_save = (mode_i == AES_GCM) & (gcm_phase_i ==
GCM_SAVE);
276 assign gcm_tag = (mode_i == AES_GCM) & (gcm_phase_i ==
GCM_TAG);
277
278 // The common start conditions for GCM don't include the in-
put data.
279 assign start_common_gcm = key_init_ready &
280 // If key sideload is enabled, we only start if the key
is valid.
281 (sideload_i ? key_sideload_valid_i : 1'b1);
282
283 // For the initial GCM phase, we need key and IV but then
encrypt two blocks. The input data
284 // is not required. Requires operating the AES cipher core
and thus factors into the regular
285 // start signal. Delay GCM initialization while clearing
GCM-related status tracking.
286 assign start_gcm_hsk = gcm_init & ~gcm_clear & ~hash_sub-
key_ready_q & iv_ready & ctr_ready_i;
287 assign start_gcm_s = gcm_init & ~gcm_clear &
~s_ready_q & iv_ready & ctr_ready_i;
288 assign start_gcm_init = (start_gcm_hsk | start_gcm_s) &
start_common_gcm;
289
290 // For the restore phase of GCM, we need key, IV and input
data. If set to start manually, we
291 // just wait for the trigger.
292 assign start_gcm_restore = gcm_restore & cfg_valid &
no_alert &
293 // Manual operation has priority.
294 (manual_operation_i ? start_i : iv_ready & ctr_ready_i &
data_in_new & start_common_gcm);
295
296 // For the AAD and ciphertext/plaintext phases of GCM, we
need key, IV and input data. But for
297 // the AAD phase, only the input data is really used and
marked as used. The start condition for
298 // the AAD phase is handled separately by the FSM and we
need to factor in the manual operation
299 // mode here.

```

```

300 assign start_gcm_aad = gcm_aad & cfg_valid & no_alert &
301 // Manual operation has priority.
302 (manual_operation_i ? start_i : iv_ready & ctr_ready_i &
start_common);
303 // The ciphertext/plaintext phase requires operating AES
cipher core and thus factors into the
304 // regular start signal.
305 assign start_gcm_txt = gcm_txt & iv_ready & ctr_ready_i &
start_common;
306
307 // For the save phase of GCM, we need no inputs. But the
hash subkey and S must not yet have been
308 // cleared (happens upon saving the GHASH state).
309 assign start_gcm_save = gcm_save & cfg_valid & no_alert &
310 // Manual operation has priority.
311 (manual_operation_i ? start_i : hash_subkey_ready_q &
s_ready_q);
312
313 // For the tag phase of GCM, we also need the input data.
314 assign start_gcm_tag = gcm_tag & cfg_valid & no_alert &
315 // Manual operation has priority.
316 (manual_operation_i ? start_i : hash_subkey_ready_q &
s_ready_q & data_in_new);
317
318 // The restore, AAD, save and tag phases use the GHASH block
only.
319 assign start_ghash = start_gcm_restore | start_gcm_aad |
start_gcm_save | start_gcm_tag;
320
321 // The GHASH block is idle whenever it's ready to receive
inputs and we're not about to start
322 // a GCM related operation.
323 assign ghash_idle = ghash_in_ready_i & ~start_ghash;
324
325 // In GCM, the counter performs inc32() instead of inc128(),
i.e., the counter wraps at 32 bits.
326 assign ctr_inc32_o = (mode_i == AES_GCM);
327
328 // Helper signals for FSM
329 assign crypt = cipher_crypt_o | cipher_crypt_i;
330
331 assign doing_cbc_enc = (mode_i == AES_CBC && op_i ==
AES_ENC) & crypt;
332 assign doing_cbc_dec = (mode_i == AES_CBC && op_i ==
AES_DEC) & crypt;
333 assign doing_cfb_enc = (mode_i == AES_CFB && op_i ==
AES_ENC) & crypt;
334 assign doing_cfb_dec = (mode_i == AES_CFB && op_i ==
AES_DEC) & crypt;
335 assign doing_ofb = (mode_i ==
AES_OFB) & crypt;
336 assign doing_ctr = (mode_i ==
AES_CTR) & crypt;
337 assign doing_gcm_hsk = gcm_init & ~hash_sub-
key_ready_q & crypt;
338 assign doing_gcm_s = gcm_init & hash_sub-
key_ready_q & crypt;
339 assign doing_gcm_txt =
gcm_txt & crypt;
340
341 // FSM
342 always_comb begin : aes_ctrl_fsm
343
344 // Previous input data register control
345 data_in_prev_sel_o = DIP_CLEAR;
346 data_in_prev_we_o = 1'b0;
347
348 // Cipher I/O mux control
349 state_in_sel_o = SI_DATA;
350 add_state_in_sel_o = ADD_SI_ZERO;
351 add_state_out_sel_o = ADD_SO_ZERO;
352
353 // Counter control
354 ctr_incr_o = 1'b0;
355
356 // Cipher core control
357 cipher_in_valid_o = 1'b0;
358 cipher_out_ready_o = 1'b0;
359 cipher_out_done = 1'b0;
360 cipher_crypt_o = 1'b0;
361 cipher_dec_key_gen_o = 1'b0;
362 cipher_prng_reseed_o = 1'b0;
363 cipher_key_clear_o = 1'b0;
364 cipher_data_out_clear_o = 1'b0;
365
366 // GHASH control
367 ghash_in_valid_o = 1'b0;
368 ghash_out_ready_o = 1'b0;
369 ghash_out_done = 1'b0;
370 ghash_load_hash_subkey_o = ~hash_subkey_ready_q;
371
372 // Initial key registers
373 key_init_sel_o = sideload_i ? KEY_INIT_KEYMGR : KEY_IN-
IT_INPUT;
374 key_init_we_o = {NumSharesKey * NumRegsKey{1'b0}};
375
376 // IV registers
377 iv_sel_o = IV_INPUT;

```

```

378   iv_we_o = {NumSlicesCtrl{1'b0}};
379
380   // Control registers
381   ctrl_we_o = 1'b0;
382   ctrl_gcm_we_o = 1'b0;
383
384   // Alert
385   alert_o = 1'b0;
386
387   // Pseudo-random number generator control
388   prng_update_o = 1'b0;
389   prng_reseed_req_o = 1'b0;
390
391   // Trigger register control
392   start_we = 1'b0;
393   key_iv_data_in_clear_we = 1'b0;
394   data_out_clear_we = 1'b0;
395   prng_reseed_we = 1'b0;
396
397   // Status register
398   idle = 1'b0;
399   idle_we = 1'b0;
400   stall = 1'b0;
401   stall_we = 1'b0;
402
403   // Key, data I/O register control
404   data_in_load = 1'b0;
405   data_in_we_o = 1'b0;
406   data_out_sel_o = DATA_OUT_CIPHER;
407   data_out_we_o = 1'b0;
408
409   // Register status tracker control
410   key_init_clear = 1'b0;
411   key_init_load = 1'b0;
412   key_init_arm = 1'b0;
413   iv_clear = 1'b0;
414   iv_load = 1'b0;
415   iv_arm = 1'b0;
416
417   // Block counter
418   block_ctr_decr = 1'b0;
419
420   // FSM
421   aes_ctrl_ns = aes_ctrl_cs;
422   start_core = 1'b0;
423   prng_reseed_done_d = prng_reseed_done_q | prng_reseed_ack_i;
424   hash_subkey_ready_d = hash_subkey_ready_q;
425   s_ready_d = s_ready_q;
426   doing_gcm_restore_d = doing_gcm_restore_q;
427   doing_gcm_aad_d = doing_gcm_aad_q;
428   doing_gcm_tag_d = doing_gcm_tag_q;
429   doing_gcm_save_d = doing_gcm_save_q;
430
431   unique case (aes_ctrl_cs)
432   CTRL_IDLE: begin
433
434       // The core is about to start encryption/decryption or
435       // another action.
436       start_core = start | key_iv_data_in_clear_i |
437       data_out_clear_i | prng_reseed_i;
438
439       // Update status register. A write to the main control
440       // register (if sideload is enabled)
441       // or writing the last key register can initiate a
442       // PRNG reseed operation via trigger
443       // register. To avoid that subsequent writes to the
444       // main control, key or IV registers
445       // collide with the start of the reseed operation, de-
446       // assert the idle bit. The GHASH block
447
448       // may still be busy while the cipher core is actually
449       // idle.
450       idle = ~(start_core | (prng_reseed_o & prng_reseed_we_o)) & ghash_idle;
451       idle_we = 1'b1;
452
453       // Clear the start trigger when seeing invalid configurations or performing automatic
454       // operation.
455       start_we = start_i & ((mode_i == AES_NONE) | ~manual_operation_i);
456
457       if (!start_core) begin
458           // Initial key and IV updates are ignored if the
459           // core is about to start. If key sideload
460           // is enabled, software writes to the initial key
461           // registers are ignored.
462           key_init_we_o = sideload_i ? {NumSharesKey * NumRegisters[key_sideload]} : key_init_we_i;
463           iv_we_o = iv_we_i;
464       end
465
466       if (!start_core && !start_ghash) begin
467
468           // Updates to the main and GCM control registers are
469           // only allowed if the cipher core and
470           // the GHASH block are not about to start and there
471           // isn't a storage error. A storage
472
473           // error is unrecoverable and requires a reset.
474           ctrl_we_o = !ctrl_err_storage_i ?
475           ctrl_we_i : 1'b0;
476           ctrl_gcm_we_o = !ctrl_err_storage_i ?
477           ctrl_gcm_we_i : 1'b0;
478
479           // Control register updates clear all register
480           // status trackers. As GCM initialization
481           // advances the IV, we have to clear the IV status
482           // tracking when re-initializing GCM.
483           key_init_clear = ctrl_we_o;
484           iv_clear = ctrl_we_o | (gcm_init & gcm_clear);
485
486           // Also clear GCM-specific status tracking.
487           if (ctrl_we_o | gcm_clear) begin
488               hash_subkey_ready_d = 1'b0;
489               s_ready_d = 1'b0;
490           end
491       end
492
493       if (prng_reseed_i) begin
494           // PRNG reseeding has highest priority.
495           if (!SecMasking) begin
496               prng_reseed_done_d = 1'b0;
497               aes_ctrl_ns = CTRL_PRNG_RESEED;
498           end else begin
499               // In case masking is enabled, also the masking
500               // PRNG inside the cipher core needs to
501               // be reseeded.
502               cipher_prng_reseed_o = 1'b1;
503
504               // Perform handshake.
505               cipher_in_valid_o = 1'b1;
506               if (cipher_in_ready_i) begin
507                   prng_reseed_done_d = 1'b0;
508                   aes_ctrl_ns = CTRL_PRNG_RESEED;
509               end
510           end
511       end else if (key_iv_data_in_clear_i ||
512       data_out_clear_i) begin
513           // To clear registers, we update the PRNG and then
514           // wait for the GHASH block.
515           prng_update_o = 1'b1;
516
517           // To clear the output data registers, we re-use the
518           // muxing resources of the cipher
519           // core. To clear all key material, some key registers
520           // inside the cipher core need to
521           // be cleared.
522           cipher_key_clear_o = key_iv_data_in_clear_i;
523           cipher_data_out_clear_o = data_out_clear_i;
524
525           // We have work for the cipher core, perform handshake.
526           cipher_in_valid_o = 1'b1;
527           if (cipher_in_ready_i) begin
528               aes_ctrl_ns = CTRL_GHASH_READY;
529           end
530       end else if (start_gcm_restore || start_gcm_aad ||
531       start_gcm_tag) begin
532           // We don't have work for the AES cipher core but
533           // for the GHASH block only. Load the
534           // input data into the previous input data register
535           // such that it can be loaded into
536           // the GHASH block in unmasked form.
537           data_in_prev_sel_o = DIP_DATA_IN;
538           data_in_prev_we_o = 1'b1;
539
540           // Advance to the loading state where the input data
541           // is marked as used.
542           doing_gcm_restore_d = start_gcm_restore;
543           doing_gcm_aad_d = start_gcm_aad;
544           doing_gcm_tag_d = start_gcm_tag;
545           start_we = 1'b1;
546           aes_ctrl_ns = CTRL_LOAD;
547       end else if (start_gcm_save) begin
548           // We don't have work for the AES cipher core but
549           // for the GHASH block only. Update the
550           // PRNG (to clear the internal state after saving
551           // it) and advance.
552           prng_update_o = 1'b1;
553           doing_gcm_save_d = 1'b1;
554           start_we = 1'b1;
555           aes_ctrl_ns = CTRL_GHASH_READY;
556       end else if (start) begin
557           // Signal that we want to start encryption/decryption.
558           cipher_crypt_o = 1'b1;
559
560           // Signal if the cipher core shall reseed the masking PRNG.
561           cipher_prng_reseed_o = block_ctr_expr;
562
563           // We got a new initial key, but want to do decryption.
564           // The cipher core must first

```

```

539 // generate the start key for decryption.
540 cipher_dec_key_gen_o = (cipher_op_i == CIPH_INV) ?
key_init_new : 1'b0;
541
542 // Previous input data register control
543 data_in_prev_sel_o = doing_cbc_dec ? DIP_DATA_IN :
544 doing_cfb_enc ? DIP_DATA_IN :
545 doing_cfb_dec ? DIP_DATA_IN :
546 doing_ofb ? DIP_DATA_IN :
547 doing_ctr ? DIP_DATA_IN :
548 doing_gcm_txt ? DIP_DATA_IN :
DIP_CLEAR;
549 data_in_prev_we_o = doing_cbc_dec |
550 doing_cfb_enc |
551 doing_cfb_dec |
552 doing_ofb |
553 doing_ctr |
554 doing_gcm_txt;
555
556 // State input mux control
557 state_in_sel_o = doing_cfb_enc ? SI_ZERO :
558 doing_cfb_dec ? SI_ZERO :
559 doing_ofb ? SI_ZERO :
560 doing_ctr ? SI_ZERO :
561 doing_gcm_hsk ? SI_ZERO :
562 doing_gcm_s ? SI_ZERO :
563 doing_gcm_txt ? SI_ZERO :
SI_DATA;
564
565 // State input addition mux control
566 add_state_in_sel_o = doing_cbc_enc ? ADD_SI_IV :
567 doing_cfb_enc ? ADD_SI_IV :
568 doing_cfb_dec ? ADD_SI_IV :
569 doing_ofb ? ADD_SI_IV :
570 doing_ctr ? ADD_SI_IV :
571 doing_gcm_s ? ADD_SI_IV :
572 doing_gcm_txt ? ADD_SI_IV :
ADD_SI_ZERO;
573
574 // We have work for the cipher core, perform hand-
shake.
575 cipher_in_valid_o = 1'b1;
576 if (cipher_in_ready_i) begin
577 // Do not yet clear a possible start trigger if we
are just starting the generation of
578 // the start key for decryption.
579 start_we = ~cipher_dec_key_gen_o;
580 aes_ctrl_ns = CTRL_LOAD;
581 end
582 end
583 end
584 CTRL_LOAD: begin
585 // Signal that we have used the current key, IV, data
input to register status tracking.
587 key_init_load = cipher_dec_key_gen_i; // This key is
no longer "new", but still clean.
588 key_init_arm = ~cipher_dec_key_gen_i; // The key is
still "new", prevent partial updates.
589 iv_load = ~cipher_dec_key_gen_i & (doing_cbc_enc |
590 doing_cbc_dec |
591 doing_cfb_enc |
592 doing_cfb_dec |
593 doing_ofb |
594 doing_ctr |
595 doing_gcm_hsk |
596 doing_gcm_s |
597 do-
ing_gcm_txt);
598 data_in_load = ~cipher_dec_key_gen_i;
599
600 // Trigger counter increment.
601 ctr_incr_o = doing_ctr | doing_gcm_hsk | doing_gcm_s |
doing_gcm_txt;
602
603 // Unless we are just generating the start key for de-
cryption, we must update the PRNG.
604 // Fresh pseudo-random data is used to:
605 // - clear the state in the final cipher round,
606 // - clear any other registers in the CLEAR_IV/CO
states.
607 prng_update_o = !cipher_dec_key_gen_i;
608
609 // Unless we are just generating the start key for de-
cryption, we may need to interface
610 // the GHASH block as well
611 aes_ctrl_ns = !cipher_dec_key_gen_i ?
CTRL_GHASH_READY : CTRL_FINISH;
612 end
613
614 CTRL_GHASH_READY: begin

```

```

615 // IV control in case of ongoing encryption/decryption
616 // - CTR: IV registers are updated by counter during
cipher operation
617 iv_sel_o = doing_ctr ||
618 doing_gcm_hsk ||
619 doing_gcm_s ||
620 doing_gcm_txt ? IV_CTR : IV_INPUT;
621 iv_we_o = doing_ctr ||
622 doing_gcm_hsk ||
623 doing_gcm_s ||
624 doing_gcm_txt ? ctr_we_i :
{NumSlicesCtr{1'b0}};
625
626 // Ongoing encryption/decryption operations have the
highest priority. The clear triggers
627 // might for example have become asserted after the
handshake with the cipher core.
628 if (cipher_crypt_i) begin
629 if (doing_gcm_hsk || doing_gcm_s || doing_gcm_txt)
begin
630 // We actually have some work for the GHASH block.
Make sure it's ready before we move.
631 // We send the valid in a following clock cycle.
Doing this check here, allows to
632 // decouple the valid from the ready.
633 if (ghash_in_ready_i) begin
634 aes_ctrl_ns = CTRL_FINISH;
635 end
636 end else begin
637 // We're not actually using the GHASH block and
don't need to check it's status.
638 aes_ctrl_ns = CTRL_FINISH;
639 end
640 end else if (doing_gcm_restore_q) begin
641 // Pass the previously saved GHASH state to the
GHASH block. We're done after the input
642 // handshake.
643 ghash_in_valid_o = 1'b1;
644 if (ghash_in_ready_i) begin
645 doing_gcm_restore_d = 1'b0;
646 aes_ctrl_ns = CTRL_IDLE;
647 end
648 end else if (doing_gcm_aad_q) begin
649 // Pass the AAD to the GHASH block. We're done after
the input handshake.
650 ghash_in_valid_o = 1'b1;
651 if (ghash_in_ready_i) begin
652 doing_gcm_aad_d = 1'b0;
653 aes_ctrl_ns = CTRL_IDLE;
654 end
655 end else if (doing_gcm_save_q) begin
656 // Perform the handshake with the GHASH block and
then wait for the GHASH block to output
657 // the state.
658 ghash_in_valid_o = 1'b1;
659 if (ghash_in_ready_i) begin
660 aes_ctrl_ns = CTRL_FINISH;
661 end
662 end else if (doing_gcm_tag_q) begin
663 // Pass the AAD and text length to the GHASH block
and wait for the final authentication
664 // tag afterwards.
665 ghash_in_valid_o = 1'b1;
666 if (ghash_in_ready_i) begin
667 aes_ctrl_ns = CTRL_FINISH;
668 end
669 end else if (key_iv_data_in_clear_i ||
data_out_clear_i ||
670 cipher_key_clear_i ||
cipher_data_out_clear_i) begin
671 // We actually have some work for the GHASH block.
Make sure it's ready before we move.
672 // We send the valid in a following clock cycle. Do-
ing this check here, allows to
673 // decouple the valid from the ready.
674 if (ghash_in_ready_i) begin
675 aes_ctrl_ns = CTRL_CLEAR_I;
676 end
677 end else begin
678 // Another write to the trigger register must have
overwritten the trigger bits that
679 // actually caused us to enter this state. Just re-
turn.
680 aes_ctrl_ns = CTRL_IDLE;
681 end
682 end
683 CTRL_PRNG_RESEED: begin
684 // Request a reseed of the clearing PRNG.
685 prng_reseed_req_o = ~prng_reseed_done_q;
686
687 if (!SecMasking) begin
688 if (prng_reseed_done_q) begin
689 // Clear the trigger and return.

```

```

696     prng_reseed_we    = 1'b1;
697     prng_reseed_done_d = 1'b0;
698     aes_ctrl_ns       = CTRL_IDLE;
699     end
700
701     end else begin
702         // In case masking is used, we must also wait for
the cipher core
703         // to reseed the internal
704         // masking PRNG. Perform handshake.
705         cipher_out_ready_o = prng_reseed_done_q;
706         if (cipher_out_ready_o && cipher_out_valid_i) begin
707             // Clear the trigger and return.
708             prng_reseed_we    = 1'b1;
709             prng_reseed_done_d = 1'b0;
710             aes_ctrl_ns       = CTRL_IDLE;
711         end
712     end
713
714     CTRL_FINISH: begin
715         // Wait for cipher core to finish.
716
717         if (cipher_dec_key_gen_i) begin
718             // We are ready.
719             cipher_out_ready_o = 1'b1;
720             if (cipher_out_valid_i) begin
721                 block_ctr_decr = 1'b1;
722                 aes_ctrl_ns     = CTRL_IDLE;
723             end
724         end else if (doing_gcm_save_q || doing_gcm_tag_q) be-
gin
725             // Handshake signals: We are ready once the output
data registers
726             // can be written. Don't
727             // let data propagate in case of mux selector or
sparsely encoded
728             // signals taking on
729             // invalid values.
730             ghash_out_ready_o = finish;
731             ghash_out_done    = finish & ghash_out_valid_i &
~mux_sel_err_i & ~sp_enc_err_i & ~cipher_op_err;
732             // Signal if the GHASH block is stalled (because
previous output
733             // has not yet been read).
734             stall    = ~finish & ghash_out_valid_i;
735             stall_we = 1'b1;
736             // Forward the GHASH output instead of the cipher
core output.
737             data_out_sel_o = DATA_OUT_GHASH;
738
739             // Proceed upon successful handshake. Mark hash sub-
key and s as not ready, the GHASH
740             // block clears those registers after the handshake.
741             if (ghash_out_done) begin
742                 doing_gcm_save_d    = 1'b0;
743                 doing_gcm_tag_d     = 1'b0;
744                 hash_subkey_ready_d = 1'b0;
745                 s_ready_d           = 1'b0;
746                 data_out_we_o       = 1'b1;
747                 aes_ctrl_ns         = CTRL_IDLE;
748             end
749         end else begin
750             // Handshake signals: We are ready once the output
data registers
751             // can be written. Don't
752             // let data propagate in case of mux selector or
sparsely encoded
753             // signals taking on
754             // invalid values.
755             cipher_out_ready_o = finish;
756             cipher_out_done    = finish & cipher_out_valid_i &
~mux_sel_err_i & ~sp_enc_err_i & ~cipher_op_err;
757             // Signal if the cipher core is stalled (because
previous output
758             // has not yet been read).
759             stall    = ~finish & cipher_out_valid_i;
760             stall_we = 1'b1;
761
762             // State out addition mux control
763             add_state_out_sel_o = doing_cbc_dec ? ADD_SO_IV :
764             doing_cfb_enc ? ADD_SO_DIP :
765             doing_cfb_dec ? ADD_SO_DIP :
766             doing_ofb ? ADD_SO_DIP :
767             doing_ctr ? ADD_SO_DIP :
768             doing_gcm_txt ? ADD_SO_DIP :
769             ADD_SO_ZERO;
770             // IV control
771
772             // - CBC/CFB/OFB: IV registers are only updated when
cipher finishes.
773             // - CTR/GCM: IV registers are updated by counter
during cipher operation.
774             // The same holds when we're computing the hash
subkey for GCM.
775             iv_sel_o = doing_cbc_enc ? IV_DATA_OUT :
776             doing_cbc_dec ? IV_DATA_IN_PREV :
777             doing_cfb_enc ? IV_DATA_OUT :
778             doing_cfb_dec ? IV_DATA_IN_PREV :
779             doing_ofb ? IV_DATA_OUT_RAW :
780             doing_ctr ? IV_CTR :
781             doing_gcm_hsk ? IV_CTR :
782             doing_gcm_s ? IV_CTR :

```

```

782             doing_gcm_txt ? IV_CTR : IV_IN-
PUT;
783             iv_we_o = doing_cbc_enc ||
784             doing_cbc_dec ||
785             doing_cfb_enc ||
786             doing_cfb_dec ||
787             doing_ofb ?
{NumSlicesCtr{cipher_out_done}} :
788             doing_ctr ?
789             doing_gcm_hsk ?
790             doing_gcm_s ?
791             doing_gcm_txt ?
792             : {NumSlicesCtr{1'b0}};
793             // Arm the IV status tracker: After finishing, the
IV registers can be written again
794             // by software. We need to make sure software does
not partially update the IV.
795             iv_arm = (doing_cbc_enc |
796             doing_cbc_dec |
797             doing_cfb_enc |
798             doing_cfb_dec |
799             doing_ofb |
800             doing_ctr |
801             doing_gcm_hsk |
802             doing_gcm_s |
803             doing_gcm_txt) & cipher_out_done;
804
805             // Proceed upon successful handshake. When initial-
izing GHASH for GCM, don't write to the
806             // cipher output to the output data registers but
forward it to the GHASH block and
807             // mark the hash subkey or S as ready.
808             if (cipher_out_done) begin
809                 block_ctr_decr = 1'b1;
810                 data_out_we_o = doing_gcm_hsk |
811                 doing_gcm_s ? 1'b0 : 1'b1;
812                 ghash_in_valid_o = doing_gcm_hsk |
813                 doing_gcm_s |
814                 doing_gcm_txt ? 1'b1 : 1'b0;
815                 hash_subkey_ready_d = doing_gcm_hsk ? 1'b1 :
816                 s_ready_d = doing_gcm_s ? 1'b1 :
817                 s_ready_q;
818                 aes_ctrl_ns = CTRL_IDLE;
819             end
820         end
821
822         CTRL_CLEAR_I: begin
823             // Clear input registers such as Initial Key, IV and
input data registers.
824             if (key_iv_data_in_clear_i) begin
825                 // Initial Key
826                 key_init_sel_o = KEY_INIT_CLEAR;
827                 key_init_we_o = {NumSharesKey * NumRegsKey{1'b1}};
828                 key_init_clear = 1'b1;
829
830                 // IV
831                 iv_sel_o = IV_CLEAR;
832                 iv_we_o = {NumSlicesCtr{1'b1}};
833                 iv_clear = 1'b1;
834
835                 // Input data
836                 data_in_we_o = 1'b1;
837                 data_in_prev_sel_o = DIP_CLEAR;
838                 data_in_prev_we_o = 1'b1;
839             end
840             aes_ctrl_ns = CTRL_CLEAR_CO;
841         end
842
843         CTRL_CLEAR_CO: begin
844             // Wait for cipher core to clear internal Full Key and
Decryption Key registers and/or
845             // the state register and clear output data registers
afterwards.
846
847             // Perform handshake with cipher core.
848             cipher_out_ready_o = 1'b1;
849             if (cipher_out_valid_i) begin
850
851                 // Full Key and Decryption Key registers are cleared
by the cipher core.
852                 // key_iv_data_in_clear_i is acknowledged by the
cipher core with cipher_key_clear_i.
853                 if (cipher_key_clear_i) begin
854                     // Clear the trigger bit.
855                     key_iv_data_in_clear_we = 1'b1;
856
857                     // The GHASH block can now clear it's internal re-
gisters.
858                     ghash_in_valid_o = 1'b1;
859                 end
860
861                 // To clear the output data registers, we re-use the

```

```

muxing resources of the cipher core.
862 // data_out_clear_i is acknowledged by the cipher
core with cipher_data_out_clear_i.
863 if (cipher_data_out_clear_i) begin
864 // Clear output data and the trigger bit. Don't
release data from cipher core in case
865 // of mux selector or sparsely encoded signals
taking on invalid values.

866 data_out_we_o = ~mux_sel_err_i & ~sp_enc_err_i &
~cipher_op_err;
867 data_out_clear_we = 1'b1;
868 end
869
870 aes_ctrl_ns = CTRL_IDLE;
871 end
872 end
873
874 CTRL_ERROR: begin
875 // SEC_CM: MAIN.FSM.GLOBAL_ESC
876 // SEC_CM: MAIN.FSM.LOCAL_ESC
877 // Terminal error state
878 alert_o = 1'b1;
879 end
880
881 // We should never get here. If we do (e.g. via a mali-
cious glitch), error out immediately.
882 default: begin
883 aes_ctrl_ns = CTRL_ERROR;
884 alert_o = 1'b1;
885 end
886 endcase
887
888 // Unconditionally jump into the terminal error state in
case a mux selector or a sparsely
889 // encoded signal becomes invalid, or if the life cycle
controller triggers an escalation.
890 if (mux_sel_err_i || sp_enc_err_i || cipher_op_err ||
lc_c-
891 trl_pkg::lc_tx_test_true_loose(lc_escalate_en_i)) begin
892 aes_ctrl_ns = CTRL_ERROR;
893 end
894 end
895
896 // SEC_CM: MAIN.FSM.SPARSE
897 'PRIM_FLOP_SPARSE_FSM(u_state_regs, aes_ctrl_ns, aes_c-
trl_cs, aes_ctrl_e, CTRL_IDLE)
898
899 always_ff @(posedge clk_i or negedge rst_ni) begin : reg_fsm
900 if (!rst_ni) begin
901 prng_reseed_done_q <= 1'b0;
902 end else begin
903 prng_reseed_done_q <= prng_reseed_done_d;
904 end
905 end
906
907 if (AESGCMEnable) begin : gen_reg_fsm_gcm
908 always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_fsm_gcm
909 if (!rst_ni) begin
910 hash_subkey_ready_q <= 1'b0;
911 s_ready_q <= 1'b0;
912 doing_gcm_restore_q <= 1'b0;
913 doing_gcm_aad_q <= 1'b0;
914 doing_gcm_save_q <= 1'b0;
915 doing_gcm_tag_q <= 1'b0;
916 end else begin
917 hash_subkey_ready_q <= hash_subkey_ready_d;
918 s_ready_q <= s_ready_d;
919 doing_gcm_restore_q <= doing_gcm_restore_d;
920 doing_gcm_aad_q <= doing_gcm_aad_d;
921 doing_gcm_save_q <= doing_gcm_save_d;
922 doing_gcm_tag_q <= doing_gcm_tag_d;
923 end
924 end
925 end else begin : gen_no_reg_fsm_gcm
926 // GCM is simply not supported.
927 assign hash_subkey_ready_q = 1'b0;
928 assign s_ready_q = 1'b0;
929 assign doing_gcm_restore_q = 1'b0;
930 assign doing_gcm_aad_q = 1'b0;
931 assign doing_gcm_save_q = 1'b0;
932 assign doing_gcm_tag_q = 1'b0;
933
934 // Tie-off unused signals.
935 logic unused_gcm_d;
936 assign unused_gcm_d = ^{hash_subkey_ready_d,
937 s_ready_d,
938 doing_gcm_restore_d,
939 doing_gcm_aad_d,
940 doing_gcm_save_d,
941 doing_gcm_tag_d};
942 end
943
944 //////////////////////////////////////////////////
945 // Status Tracking //
946 //////////////////////////////////////////////////
947
948 // We only take a new sideload key if sideload is enabled,
if the provided sideload key is marked

```

```

949 // as valid, and after the control register has been written
for the second time. After that

950 // point we don't update the key anymore, as we don't have a
notion of when it actually changes.
951 // This would be required to trigger decryption key genera-
tion for ECB/CBC decryption.
952 // To update the sideload key, software has to:
953 // 1) wait until AES is idle,
954 // 2) wait for the key manager to provide the new key,
955 // 3) start a new message by writing the control register
and providing the IV (if needed).
956 assign key_sideload = sideload_i & key_sideload_valid_i &
ctrl_we_q & ~ctrl_phase_i;
957
958 // We only use clean initial keys. Either software/counter
has updated
959 // - all initial key registers, or
960 // - none of the initial key registers but the registers
were updated in the past.
961 aes_reg_status #(
962 .Width ( $bits(key_init_we_o) )
963 ) u_reg_status_key_init (
964 .clk_i ( clk_i ),
965 .rst_ni ( rst_ni ),
966 .we_i ( key_init_we_o ),
967 .use_i ( key_init_load ),
968 .clear_i ( key_init_clear ),
969 .arm_i ( key_init_arm ),
970 .new_o ( key_init_new ),
971 .new_pulse_o ( key_init_new_pulse ),
972 .clean_o ( key_init_ready )
973 );
974
975 // We only use clean and unused IVs. Either software/counter
has updated
976 // - all IV registers, or

977 // - none of the IV registers but the registers were updated
in the past
978 // and this particular IV has not yet been used.
979 aes_reg_status #(
980 .Width ( $bits(iv_we_o) )
981 ) u_reg_status_iv (
982 .clk_i ( clk_i ),
983 .rst_ni ( rst_ni ),
984 .we_i ( iv_we_o ),
985 .use_i ( iv_load ),
986 .clear_i ( iv_clear ),
987 .arm_i ( iv_arm ),
988 .new_o ( iv_ready ),
989 .new_pulse_o ( ),
990 .clean_o ( )
991 );
992
993 // Input and output data register status tracking detects
if:
994 // - A complete new data input block is available, and
995 // - An output data block has been read completely.
996 // The status tracking needs to be cleared upon writes to
the control register. The clearing is

997 // applied one cycle later here to avoid zero-latency loops.
This additional delay is not
998 // relevant as if we are about to start encryption/decryp-
tion, we anyway don't allow writes
999 // to the control register.
1000 always_ff @(posedge clk_i or negedge rst_ni) begin : reg_c-
trl_we
1001 if (!rst_ni) begin
1002 ctrl_we_q <= 1'b0;
1003 end else begin
1004 ctrl_we_q <= ctrl_we_o;
1005 end
1006 end
1007 assign clear_in_out_status = ctrl_we_q;
1008
1009 // Collect writes to data input registers. Cleared if:
1010 // - data is loaded into cipher core,
1011 // - clearing data input registers with random data (all
data_in_qe_i bits high in next cycle),
1012 // - clearing the status tracking.
1013 assign data_in_new_d = data_in_load || &data_in_qe_i ||
clear_in_out_status ? '0 :
1014 data_in_new_q | data_in_qe_i;
1015 assign data_in_new = &data_in_new_d;
1016
1017 // Collect reads of data output registers. data_out_read is
high for one clock cycle only and

1018 // clears output_valid_q unless new output is written in the
exact same cycle. Cleared if:
1019 // - clearing data output registers with random data,
1020 // - clearing the status tracking.
1021 assign data_out_read_d = &data_out_read_q ||
clear_in_out_status ? '0 :
1022 data_out_read_q | data_out_re_i;
1023 assign data_out_read = &data_out_read_d;
1024
1025 always_ff @(posedge clk_i or negedge rst_ni) begin :

```



```

reg_edge_detection
1026   if (!rst_ni) begin
1027     data_in_new_q <= '0;
1028     data_out_read_q <= '0;
1029   end else begin
1030     data_in_new_q <= data_in_new_d;
1031     data_out_read_q <= data_out_read_d;
1032   end
1033 end
1034
1035 // Status register bits for data input and output
1036 // Cleared to 1 if:
1037 // - data is loaded into cipher core,
1038 // - clearing data input registers with random data,
1039 // - clearing the status tracking.
1040 assign input_ready = ~data_in_new;
1041 assign input_ready_we = data_in_new | data_in_load |
data_in_we_o | clear_in_out_status;
1042
1043 // Cleared if:
1044 // - all data output registers have been read (unless new
output is written in the same cycle),
1045 // - clearing data output registers with random data,
1046 // - clearing the status tracking.
1047 assign output_valid = data_out_we_o & ~data_out_clear_we;
1048 assign output_valid_we = data_out_we_o | data_out_read |
data_out_clear_we |
1049   clear_in_out_status;
1050
1051 always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_output_valid
1052   if (!rst_ni) begin
1053     output_valid_q <= '0;
1054   end else if (output_valid_we) begin
1055     output_valid_q <= output_valid;
1056   end
1057 end
1058
1059 // Output lost status register bit
1060 // Cleared when updating the Control Register. Set when
overwriting previous output data that has
1061 // not yet been read.
1062 assign output_lost = ctrl_we_o ? 1'b0 :
1063   output_lost_i ? 1'b1 : out-
put_valid_q & ~data_out_read;
1064 assign output_lost_we = ctrl_we_o | data_out_we_o;
1065
1066 ///////////////////////////////////////////////////
1067 // GCM Status Tracking //
1068 ///////////////////////////////////////////////////
1069
1070 // Clear GCM-specific status tracking if the GCM phase is
set to GCM_INIT after a second write to
1071 // the shadowed GCM control register
1072
1072 assign gcm_clear = (gcm_phase_i == GCM_INIT) & ctrl_gcm_we_q
& ~ctrl_gcm_phase_i;
1073
1074 if (AESGCMEnable) begin : gen_reg_ctrl_gcm_we
1075   always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_ctrl_gcm_we
1076     if (!rst_ni) begin
1077       ctrl_gcm_we_q <= 1'b0;
1078     end else begin
1079       ctrl_gcm_we_q <= ctrl_gcm_we_o;
1080     end
1081   end
1082 end else begin : gen_no_reg_ctrl_gcm_we
1083   assign ctrl_gcm_we_q = 1'b1;
1084 end
1085
1086 // Signal when the GCM initialization is done.
1087 assign gcm_init_done_o = hash_subkey_ready_q & s_ready_q;
1088
1089 // Should fatal alerts clear the status and trigger re-
gister?
1090 assign clear_on_fatal = ClearStatusOnFatalAlert ?
alert_fatal_i : 1'b0;
1091
1092 ///////////////////////////////////////////////////
1093 // Status Register //
1094 ///////////////////////////////////////////////////
1095 assign idle_o = clear_on_fatal ? 1'b0 : idle;
1096 assign idle_we_o = clear_on_fatal ? 1'b1 : idle_we;
1097 assign stall_o = clear_on_fatal ? 1'b0 : stall;
1098 assign stall_we_o = clear_on_fatal ? 1'b1 : stall_we;
1099 assign output_lost_o = clear_on_fatal ? 1'b0 : out-
put_lost;
1100 assign output_lost_we_o = clear_on_fatal ? 1'b1 : out-
put_lost_we;
1101 assign output_valid_o = clear_on_fatal ? 1'b0 : out-
put_valid;
1102 assign output_valid_we_o = clear_on_fatal ? 1'b1 : out-
put_valid_we;
1103 assign input_ready_o = clear_on_fatal ? 1'b0 : in-
put_ready;
1104 assign input_ready_we_o = clear_on_fatal ? 1'b1 : in-
put_ready_we;
1105
1106 ///////////////////////////////////////////////////
1107 // Trigger Register //

```

```

1108 ///////////////////////////////////////////////////
1109 // Most triggers are only ever cleared by control. Fatal
alerts clear all bits in the trigger
1110 // register.
1111 assign start_we_o = clear_on_fatal ? 1'b1 :
start_we;
1112 assign key_iv_data_in_clear_we_o = clear_on_fatal ? 1'b1 :
key_iv_data_in_clear_we;
1113 assign data_out_clear_we_o = clear_on_fatal ? 1'b1 :
data_out_clear_we;
1114
1115 // If configured, trigger the reseeding of the PRNGs used
for clearing and masking purposes after
1116 // the key has been updated.
1117 assign prng_reseed_o = clear_on_fatal ? 1'b0 :
1118   key_init_new_pulse ? 1'b1 : 1'b0;
1119 assign prng_reseed_we_o = clear_on_fatal ?
1'b1 :
1120   key_init_new_pulse ?
key_touch_forces_reseed_i : prng_reseed_we;
1121
1122 ///////////////////////////////////////////////////
1123 // PRNG Reseeding Counter //
1124 ///////////////////////////////////////////////////
1125
1125 // Count the number of blocks since the start of the message
to determine when the masking PRNG
1126 // inside the cipher core needs to be reseeded.
1127 if (SecMasking) begin : gen_block_ctr
1128   logic block_ctr_set;
1129   logic [BlockCtrWidth-1:0] block_ctr_d, block_ctr_q;
1130   logic [BlockCtrWidth-1:0] block_ctr_set_val,
block_ctr_decr_val;
1131
1132   assign block_ctr_expr = block_ctr_q == '0;
1133   assign block_ctr_set = ctrl_we_q | (block_ctr_decr &
(block_ctr_expr | cipher_prng_reseed_i));
1134
1135   assign block_ctr_set_val = prng_reseed_rate_i == PER_1 ?
'0 :
1136     prng_reseed_rate_i == PER_64 ?
BlockCtrWidth'(63) :
1137     prng_reseed_rate_i == PER_8K ?
BlockCtrWidth'(8191) : '0;
1138
1139   assign block_ctr_decr_val = block_ctr_q - BlockC-
trWidth'(1);
1140
1141   assign block_ctr_d = block_ctr_set ? block_ctr_set_val :
block_ctr_decr ? block_ctr_decr_val :
block_ctr_q;
1142
1143   always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_block_ctr
1144     if (!rst_ni) begin
1145       block_ctr_q <= '0;
1146     end else begin
1147       block_ctr_q <= block_ctr_d;
1148     end
1149   end
1150 end
1151
1152 end else begin : gen_no_block_ctr
1153   assign block_ctr_expr = 1'b0;
1154
1155   // Tie off unused signals.
1156   logic unused_block_ctr_decr;
1157   prs_rate_e unused_prng_reseed_rate;
1158   logic unused_cipher_prng_reseed;
1159   assign unused_block_ctr_decr = block_ctr_decr;
1160   assign unused_prng_reseed_rate = prng_reseed_rate_i;
1161   assign unused_cipher_prng_reseed = cipher_prng_reseed_i;
1162 end
1163
1164 ///////////////////////////////////////////////////
1165 // Assertions //
1166 ///////////////////////////////////////////////////
1167
1168 // Create a lint error to reduce the risk of accidentally
disabling the masking.
1169 `ASSERT_STATIC_LINT_ERROR(AesControlFsmSecMaskingNonDefault,
SecMasking == 1)
1170
1171 // Selectors must be known/valid
1172 `ASSERT(AesModeValid, !ctrl_err_storage_i |-> mode_i inside
{
1173   AES_ECB,
1174   AES_CBC,
1175   AES_CFB,
1176   AES_OFB,
1177   AES_CTR,
1178   AES_GCM,
1179   AES_NONE
1180 })
1181 `ASSERT(AesOpValid, !ctrl_err_storage_i |-> op_i inside {
1182   AES_ENC,
1183   AES_DEC

```

```

1184     })
1185     `ASSERT(AesCiphOpValid, !cipher_op_err |-> cipher_op_i in-
side {
1186         CIPH_FWD,
1187         CIPH_INV
1188     })
1189     `ASSERT(AesControlStateValid, !alert_o |-> aes_ctrl_cs in-
side {
1190         CTRL_IDLE,
1191         CTRL_LOAD,
1192         CTRL_GHASH_READY,

```

```

1193         CTRL_PRNG_RESEED,
1194         CTRL_FINISH,
1195         CTRL_CLEAR_I,
1196         CTRL_CLEAR_CO
1197     })
1198
1199     // Check parameters
1200     `ASSERT_INIT(AesNumSlicesCtr, NumSlicesCtr == 8)
1201
1202 endmodule

```

OpenTitan HMAC-SHA256

What this is. HMAC and SHA-256 combined in one block with a FIFO and a register interface.

Why it exists. Because the streaming interface, not the hash, is most of the work. Message lengths are arbitrary and software delivers fragments, so the hardware must handle buffering, byte alignment, final-block padding, and suspend/resume.

Undergraduate review · EECS concepts. HMAC's ipad/opad construction; stream processing; backpressure; byte-enable handling.

Graduate · Practice note. In most blocks the interface exceeds 70% of the code. Schedules that assume the algorithm is the work are systematically optimistic.

4 files • 4,632 lines

API — hmac.sv

Table 799. lowRISC_opentitan/hw/ip/hmac/rtl/hmac.sv — module / interface / package (1개)

이름	종류
hmac	module

Table 800. lowRISC_opentitan/hw/ip/hmac/rtl/hmac.sv — parameter (2개)

파라미터	형/폭	기본값
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1

Table 801. lowRISC_opentitan/hw/ip/hmac/rtl/hmac.sv — localparam (3개)

상수	형/폭	값
NumInBufBits	int	\$bits(keymgr_pkg::hw_key_req_t)
MsgFifoDepth	int	32
AlertIsFatal	logic [NumAlerts-1:0]	{1'b1}

Table 802. lowRISC_opentitan/hw/ip/hmac/rtl/hmac.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
keymgr_key_i	input	keymgr_pkg::hw_key_req_t

포트	방향	형/폭
intr_hmac_done_o	output	logic
intr_fifo_empty_o	output	logic
intr_hmac_err_o	output	logic
idle_o	output	prim_mubi_pkg::mubi4_t

API — hmac_core.sv

Table 803. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_core.sv — module / interface / package (17개)

이름	종류
hmac_core	module

Table 804. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_core.sv — localparam (7개)

상수	형/폭	값
BlockSizeSHA256	int unsigned	512
BlockSizeSHA512	int unsigned	1024
BlockSizeBitsSHA256	int unsigned	$\$clog2(\text{BlockSizeSHA256})$
BlockSizeBitsSHA512	int unsigned	$\$clog2(\text{BlockSizeSHA512})$
HashWordBitsSHA256	int unsigned	$\$clog2(\$bits(\text{sha_word32_t}))$
BlockSizeSHA256in64	bit [63:0]	64'(BlockSizeSHA256)
BlockSizeSHA512in64	bit [63:0]	64'(BlockSizeSHA512)

Table 805. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_core.sv — 포트 (28개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
secret_key_i	input	[1023:0]
hmac_en_i	input	
digest_size_i	input	digest_mode_e
key_length_i	input	key_length_e
reg_hash_start_i	input	
reg_hash_stop_i	input	
reg_hash_continue_i	input	
reg_hash_process_i	input	
hash_done_o	output	logic
sha_hash_start_o	output	logic
sha_hash_continue_o	output	logic
sha_hash_process_o	output	logic

포트	방향	형/폭
sha_hash_done_i	input	
sha_rvalid_o	output	logic
sha_rdata_o	output	sha_fifo32_t
sha_rready_i	input	
fifo_rvalid_i	input	
fifo_rdata_i	input	sha_fifo32_t
fifo_rready_o	output	logic
fifo_wsel_o	output	logic
fifo_wvalid_o	output	logic
fifo_wdata_sel_o	output	logic [3:0]
fifo_wready_i	input	
message_length_i	input	[63:0]
sha_message_length_o	output	[63:0]
idle_o	output	logic

API — hmac_reg_pkg.sv

Table 806. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_reg_pkg.sv — module / interface / package (1개)

이름	종류
hmac_reg_pkg	package

Table 807. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_reg_pkg.sv — parameter (134개) — 앞 60개만 표시(전체 134개, 본문 소스 참조)

파라미터	형/폭	기본값
NumDigestWords	int	16
NumKeyWords	int	32
NumAlerts	int	1
BlockAw	int	13
NumRegs	int	59
HMAC_INTR_STATE_OFFSET	logic [BlockAw-1:0]	13'h 0
HMAC_INTR_ENABLE_OFFSET	logic [BlockAw-1:0]	13'h 4
HMAC_INTR_TEST_OFFSET	logic [BlockAw-1:0]	13'h 8
HMAC_ALERT_TEST_OFFSET	logic [BlockAw-1:0]	13'h c
HMAC_CFG_OFFSET	logic [BlockAw-1:0]	13'h 10
HMAC_CMD_OFFSET	logic [BlockAw-1:0]	13'h 14
HMAC_STATUS_OFFSET	logic [BlockAw-1:0]	13'h 18
HMAC_ERR_CODE_OFFSET	logic [BlockAw-1:0]	13'h 1c

파라미터	형/폭	기본값
HMAC_WIPE_SECRET_OFFSET	logic [BlockAw-1:0]	13'h 20
HMAC_KEY_0_OFFSET	logic [BlockAw-1:0]	13'h 24
HMAC_KEY_1_OFFSET	logic [BlockAw-1:0]	13'h 28
HMAC_KEY_2_OFFSET	logic [BlockAw-1:0]	13'h 2c
HMAC_KEY_3_OFFSET	logic [BlockAw-1:0]	13'h 30
HMAC_KEY_4_OFFSET	logic [BlockAw-1:0]	13'h 34
HMAC_KEY_5_OFFSET	logic [BlockAw-1:0]	13'h 38
HMAC_KEY_6_OFFSET	logic [BlockAw-1:0]	13'h 3c
HMAC_KEY_7_OFFSET	logic [BlockAw-1:0]	13'h 40
HMAC_KEY_8_OFFSET	logic [BlockAw-1:0]	13'h 44
HMAC_KEY_9_OFFSET	logic [BlockAw-1:0]	13'h 48
HMAC_KEY_10_OFFSET	logic [BlockAw-1:0]	13'h 4c
HMAC_KEY_11_OFFSET	logic [BlockAw-1:0]	13'h 50
HMAC_KEY_12_OFFSET	logic [BlockAw-1:0]	13'h 54
HMAC_KEY_13_OFFSET	logic [BlockAw-1:0]	13'h 58
HMAC_KEY_14_OFFSET	logic [BlockAw-1:0]	13'h 5c
HMAC_KEY_15_OFFSET	logic [BlockAw-1:0]	13'h 60
HMAC_KEY_16_OFFSET	logic [BlockAw-1:0]	13'h 64
HMAC_KEY_17_OFFSET	logic [BlockAw-1:0]	13'h 68
HMAC_KEY_18_OFFSET	logic [BlockAw-1:0]	13'h 6c
HMAC_KEY_19_OFFSET	logic [BlockAw-1:0]	13'h 70
HMAC_KEY_20_OFFSET	logic [BlockAw-1:0]	13'h 74
HMAC_KEY_21_OFFSET	logic [BlockAw-1:0]	13'h 78
HMAC_KEY_22_OFFSET	logic [BlockAw-1:0]	13'h 7c
HMAC_KEY_23_OFFSET	logic [BlockAw-1:0]	13'h 80
HMAC_KEY_24_OFFSET	logic [BlockAw-1:0]	13'h 84
HMAC_KEY_25_OFFSET	logic [BlockAw-1:0]	13'h 88
HMAC_KEY_26_OFFSET	logic [BlockAw-1:0]	13'h 8c
HMAC_KEY_27_OFFSET	logic [BlockAw-1:0]	13'h 90
HMAC_KEY_28_OFFSET	logic [BlockAw-1:0]	13'h 94
HMAC_KEY_29_OFFSET	logic [BlockAw-1:0]	13'h 98
HMAC_KEY_30_OFFSET	logic [BlockAw-1:0]	13'h 9c
HMAC_KEY_31_OFFSET	logic [BlockAw-1:0]	13'h a0
HMAC_DIGEST_0_OFFSET	logic [BlockAw-1:0]	13'h a4
HMAC_DIGEST_1_OFFSET	logic [BlockAw-1:0]	13'h a8

파라미터	형/폭	기본값
HMAC_DIGEST_2_OFFSET	logic [BlockAw-1:0]	13'h ac
HMAC_DIGEST_3_OFFSET	logic [BlockAw-1:0]	13'h b0
HMAC_DIGEST_4_OFFSET	logic [BlockAw-1:0]	13'h b4
HMAC_DIGEST_5_OFFSET	logic [BlockAw-1:0]	13'h b8
HMAC_DIGEST_6_OFFSET	logic [BlockAw-1:0]	13'h bc
HMAC_DIGEST_7_OFFSET	logic [BlockAw-1:0]	13'h c0
HMAC_DIGEST_8_OFFSET	logic [BlockAw-1:0]	13'h c4
HMAC_DIGEST_9_OFFSET	logic [BlockAw-1:0]	13'h c8
HMAC_DIGEST_10_OFFSET	logic [BlockAw-1:0]	13'h cc
HMAC_DIGEST_11_OFFSET	logic [BlockAw-1:0]	13'h d0
HMAC_DIGEST_12_OFFSET	logic [BlockAw-1:0]	13'h d4
HMAC_DIGEST_13_OFFSET	logic [BlockAw-1:0]	13'h d8

API — hmac_reg_top.sv

Table 808. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_reg_top.sv — module / interface / package (1개)

이름	종류
hmac_reg_top	module

Table 809. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_reg_top.sv — localparam (3개)

상수	형/폭	값
AW	int	13
DW	int	32
DBW	int	DW/8

Table 810. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_reg_top.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
tl_win_o	output	tlul_pkg::tl_h2d_t
tl_win_i	input	tlul_pkg::tl_d2h_t
reg2hw	output	hmac_reg_pkg::hmac_reg2hw_t
hw2reg	input	hmac_reg_pkg::hmac_hw2reg_t
intg_err_o	output	logic

1. lowRISC_opentitan/hw/ip/hmac/rtl/hmac.sv (968 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // HMAC/SHA-2 256/384/512
6
7 `include "prim_assert.sv"
8
9 module hmac
10     import prim_sha2_pkg::*;
11     import hmac_reg_pkg::*;
12 #()
13     parameter logic [NumAlerts-1:0] AlertAsyncOn = {Num-
mAlerts{1'b1}},
14     // Number of cycles a differential skew is tolerated on the
alert signal
15     parameter int unsigned AlertSkewCycles = 1
16 ) (
17     input clk_i,
18     input rst_ni,
19
20     input tlul_pkg::tl_h2d_t tl_i,
21     output tlul_pkg::tl_d2h_t tl_o,
22
23     input prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_rx_i,
24     output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
alert_tx_o,
25
26     // Key manager (keymgr) key sideload interface
27     input keymgr_pkg::hw_key_req_t keymgr_key_i,
28
29     output logic intr_hmac_done_o,
30     output logic intr_fifo_empty_o,
31     output logic intr_hmac_err_o,
32
33     output prim_mubi_pkg::mubi4_t idle_o
34 );
35
36 // TODO(#31026): Consume the key from the keymgr_dpe
37 localparam int NumInBufBits = $bits(keymgr-
gr_pkg::hw_key_req_t);
38 keymgr_pkg::hw_key_req_t unused_key;
39 prim_buf #(
40     .Width (NumInBufBits)
41 ) u_anchor_buf (
42     .in_i (keymgr_key_i),
43     .out_o (unused_key)
44 );
45
46 ///////////////////////////////////////////////////
47 // Signal declarations //
48 ///////////////////////////////////////////////////
49 hmac_reg2hw_t reg2hw;
50 hmac_hw2reg_t hw2reg;
51
52 tlul_pkg::tl_h2d_t tl_win_h2d;
53 tlul_pkg::tl_d2h_t tl_win_d2h;
54
55 logic [1023:0] secret_key, secret_key_d;
56
57 // Logic will support key length <= block size
58 // Will default to key length = block size, if key length >
block size or unsupported value
59 key_length_e key_length_supplied, key_length;
60
61 logic wipe_secret;
62 logic [31:0] wipe_v;
63
64 logic fifo_rvalid;
65 logic fifo_rready;
66 sha_fifo32_t fifo_rdata;
67
68 logic fifo_wvalid, fifo_wready;
69 sha_fifo32_t fifo_wdata;
70 logic fifo_full;
71 logic fifo_empty;
72 logic [5:0] fifo_depth;
73
74 logic msg_fifo_req;
75 logic msg_fifo_gnt;
76 logic msg_fifo_we;
77 logic [31:0] msg_fifo_wdata;
78 logic [31:0] msg_fifo_wmask;
79 logic [31:0] msg_fifo_rdata;
80 logic [1:0] msg_fifo_rerror;
81 logic [31:0] msg_fifo_wdata_endian;
82 logic [31:0] msg_fifo_wmask_endian;
83
84 logic packer_ready;
85 logic packer_flush_done;
86
87 logic reg_fifo_wvalid;
88 sha_word32_t reg_fifo_wdata;
89 sha_word32_t reg_fifo_wmask;

```

```

90 logic hmac_fifo_wsel;
91 logic hmac_fifo_wvalid;
92 logic [3:0] hmac_fifo_wdata_sel;
93
94 logic shaf_rvalid;
95 sha_fifo32_t shaf_rdata;
96 logic shaf_rready;
97
98 logic sha_en;
99 logic hmac_en;
100 logic endian_swap;
101 logic digest_swap;
102 logic key_swap;
103
104 logic reg_hash_start;
105 logic sha_hash_start;
106 logic reg_hash_stop;
107 logic reg_hash_continue;
108 logic sha_hash_continue;
109 logic hash_start; // hash_start is reg_hash_start
gated with extra checks
110 logic hash_continue; // hash_continue is
reg_hash_continue gated with extra checks
111 logic hash_process; // hash_process is
reg_hash_process gated with extra checks
112 logic hash_start_or_continue;
113 logic hash_done_event;
114 logic reg_hash_process;
115 logic sha_hash_process;
116 logic digest_on_blk;
117
118 logic reg_hash_done;
119 logic sha_hash_done;
120
121 logic [63:0] message_length, message_length_d;
122 logic [63:0] sha_message_length;
123
124 err_code_e err_code;
125 logic err_valid;
126
127 logic invalid_config; // HMAC/SHA-2 is configured with
invalid digest size/key length
128 logic invalid_config_atstart;
129
130 sha_word64_t [7:0] digest, digest_sw;
131 logic [7:0] digest_sw_we;
132
133 digest_mode_e digest_size, digest_size_supplied;
134 // this is the digest size captured into HMAC when it gets
started
135 digest_mode_e digest_size_started_d, digest_size_started_q;
136
137 hmac_reg2hw_cfg_reg_t cfg_reg;
138 logic cfg_block; // Prevents changing con-
fig
139
140 logic msg_allowed; // MSG_FIFO from software
is allowed
141
142 logic hmac_core_idle;
143 logic sha_core_idle;
144 logic hash_running;
145 logic idle;
146
147 ///////////////////////////////////////////////////
148 // Connect registers //
149 ///////////////////////////////////////////////////
150 assign hw2reg.status_fifo_full_d = fifo_full;
151 assign hw2reg.status_fifo_empty_d = fifo_empty;
152 assign hw2reg.status_fifo_depth_d = fifo_depth;
153 assign hw2reg.status_hmac_idle_d = idle;
154
155 typedef enum logic [1:0] {
156     DoneAwaitCmd,
157     DoneAwaitHashDone,
158     DoneAwaitMessageComplete,
159     DoneAwaitHashComplete
160 } done_state_e;
161
162 done_state_e done_state_d, done_state_q;
163
164 always_comb begin
165     done_state_d = done_state_q;
166     hash_done_event = 1'b0;
167
168     unique case (done_state_q)
169         DoneAwaitCmd: begin
170             if (sha_hash_process) begin
171                 // SHA has been told to process the message, so sig-
nal *done* when the hash is done.
172                 done_state_d = DoneAwaitHashDone;
173             end else if (reg_hash_stop) begin
174                 // SHA has been told to stop, so first wait for the
current message block to be complete.
175                 done_state_d = DoneAwaitMessageComplete;
176             end
177         DoneAwaitHashDone: begin
178             if (reg_hash_done) begin

```

```

179     hash_done_event = 1'b1;
180     done_state_d = DoneAwaitCmd;
181 end
182 end
183
184 DoneAwaitMessageComplete: begin
185     if (digest_on_blk) begin
186         // Once the digest is being computed for the com-
187         plete message block, wait for the hash to
188         // complete.
189         done_state_d = DoneAwaitHashComplete;
190     end
191 end
192
193 DoneAwaitHashComplete: begin
194     if (!hash_running) begin
195         hash_done_event = 1'b1;
196         done_state_d = DoneAwaitCmd;
197     end
198 end
199
200 default: ;
201 endcase
202 end
203
204 always_ff @(posedge clk_i or negedge rst_ni) begin
205     if (!rst_ni) begin
206         done_state_q <= DoneAwaitCmd;
207     end else begin
208         done_state_q <= done_state_d;
209     end
210 end
211
212 assign wipe_secret = reg2hw.wipe_secret.qe;
213 assign wipe_v      = reg2hw.wipe_secret.q;
214
215 // update secret key
216 always_comb begin : update_secret_key
217     secret_key_d = secret_key;
218     if (wipe_secret) begin
219         secret_key_d = {32{wipe_v}};
220     end else if (!cfg_block) begin
221         // Allow updating secret key only when the engine is in
222         Idle.
223         for (int i = 0; i < 32; i++) begin
224             if (reg2hw.key[31-i].qe) begin
225                 // swap byte endianness per secret key word if
226                 key_swap = 1
227                 secret_key_d[32*i+:32] = conv_endi-
228                 an32(reg2hw.key[31-i].q, key_swap);
229             end
230         end
231     end
232 end
233
234 always_ff @(posedge clk_i or negedge rst_ni) begin
235     if (!rst_ni) secret_key <= '0;
236     else secret_key <= secret_key_d;
237 end
238
239 for (genvar i = 0; i < 32; i++) begin : gen_key
240     assign hw2reg.key[31-i].d = '0;
241 end
242
243 // Retain the previous digest in CSRs until HMAC is actually
244 // started with a valid configuration
245 always_comb begin : assign_digest_reg
246     // default
247     // digest SW -> HW
248     digest_sw = '0;
249     digest_sw_we = '0;
250     // digest HW -> SW
251     hw2reg.digest = '0;
252 end
253
254 for (int i = 0; i < 8; i++) begin
255     // digest SW -> HW (depends on digest size configured
256     // even before starting/enabling)
257     // capturing the intermediate digests written by SW when
258     // restoring context into the SHA-2
259     // engine before it is started
260     if (digest_size == SHA2_256) begin
261         // digest SW -> HW
262         digest_sw[i][31:0] = conv_endian32(reg2hw.digest[i].q,
263         digest_swap);
264         digest_sw_we[i] = reg2hw.digest[i].qe;
265     end else if ((digest_size == SHA2_384) || (digest_size
266     == SHA2_512)) begin
267         // digest SW -> HW
268         digest_sw[i][63:32] = reg2hw.digest[2*i].qe ?
269         conv_endi-
270         an32(reg2hw.digest[2*i].q, digest_swap) :
271         digest[i][63:32];
272         digest_sw[i][31:0] = reg2hw.digest[2*i+1].qe ?
273         conv_endi-
274         an32(reg2hw.digest[2*i+1].q, digest_swap) :
275         digest[i][31:0];
276         digest_sw_we[i] = reg2hw.digest[2*i].qe |
277         reg2hw.digest[2*i+1].qe;

```

```

265     end
266
267 // digest HW -> SW (depends on configuration that has
268 // been started)
269 if (digest_size_started_q == SHA2_256) begin
270     hw2reg.digest[i].d = conv_endian32(digest[i][31:0],
271     digest_swap);
272 // replicate digest[0..7] into digest[8..15]. Di-
273 // gest[8...15] are irrelevant for SHA2_256,
274 // but this ensures all digest CSRs are wiped out with
275 // random value (at wipe_secret)
276 // across different configurations.
277 hw2reg.digest[i+8].d = conv_endian32(digest[i][31:0],
278     digest_swap);
279 end else if ((digest_size_started_q == SHA2_384) || (di-
280     gest_size_started_q == SHA2_512)) begin
281     // digest HW -> SW
282     // digest swap only within each 32-bit word of the 64-
283     // bit digest word, not digest swap
284     // on the entire 64-bit digest word
285     hw2reg.digest[2*i].d = conv_endian32(digest[i]
286     [63:32], digest_swap);
287     hw2reg.digest[2*i+1].d = conv_endian32(digest[i]
288     [31:0], digest_swap);
289 end else begin // for SHA2_None
290     // to ensure secret wiping is always passed to digest
291     CSRs
292     hw2reg.digest[i].d = conv_endian32(digest[i][31:0],
293     digest_swap);
294     hw2reg.digest[i+8].d = conv_endian32(digest[i][31:0],
295     digest_swap);
296 end
297 end
298
299 logic unused_cfg_qe;
300 assign unused_cfg_qe = ^{cfg_reg.sha_en.qe,
301     cfg_reg.hmac_en.qe,
302     cfg_reg.endian_swap.qe, cfg_reg.di-
303     gest_swap.qe,
304     cfg_reg.key_swap.qe, cfg_reg.di-
305     gest_size.qe,
306     cfg_reg.key_length.qe };
307
308 assign sha_en = cfg_reg.sha_en.q;
309 assign hmac_en = cfg_reg.hmac_en.q;
310
311 assign digest_size_supplied = di-
312     gest_mode_e'(cfg_reg.digest_size.q);
313 always_comb begin : cast_digest_size
314     digest_size = SHA2_None;
315
316     unique case (digest_size_supplied)
317         SHA2_256: digest_size = SHA2_256;
318         SHA2_384: digest_size = SHA2_384;
319         SHA2_512: digest_size = SHA2_512;
320         // unsupported digest size values are mapped to
321         SHA2_None
322     // if HMAC/SHA-2 is triggered to start with this digest
323     // size, it is blocked
324     // and an error is signalled to SW
325     default: digest_size = SHA2_None;
326     endcase
327 end
328
329 // Hold the previous digest size till HMAC is started with
330 // the new digest size configured
331 assign digest_size_started_d = (hash_start_or_continue) ?
332     digest_size : digest_size_started_q;
333
334 always_ff @(posedge clk_i or negedge rst_ni) begin
335     if (!rst_ni) digest_size_started_q <= SHA2_None;
336     else digest_size_started_q <= di-
337     gest_size_started_d;
338 end
339
340 assign key_length_supplied =
341     key_length_e'(cfg_reg.key_length.q);
342 always_comb begin : cast_key_length
343     key_length = Key_None;
344
345     unique case (key_length_supplied)
346         Key_128: key_length = Key_128;
347         Key_256: key_length = Key_256;
348         Key_384: key_length = Key_384;
349         Key_512: key_length = Key_512;
350         Key_1024: key_length = Key_1024;
351         // unsupported key length values are mapped to Key_None
352         // if HMAC (not SHA-2) is triggered to start with this
353         // key length, it is blocked
354         // and an error is signalled to SW
355         default: key_length = Key_None;
356     endcase
357 end
358
359 assign endian_swap = cfg_reg.endian_swap.q;
360 assign digest_swap = cfg_reg.digest_swap.q;
361 assign key_swap = cfg_reg.key_swap.q;
362
363 assign hw2reg.cfg.hmac_en.d = cfg_reg.hmac_en.q;

```

```

342 assign hw2reg.cfg.sha_en.d = cfg_reg.sha_en.q;
343 assign hw2reg.cfg.digest_size.d = di-
gest_mode_e'(digest_size);
344 assign hw2reg.cfg.key_length.d = key_length_e'(key_length);
345 assign hw2reg.cfg.endian_swap.d = cfg_reg.endian_swap.q;
346 assign hw2reg.cfg.digest_swap.d = cfg_reg.digest_swap.q;
347 assign hw2reg.cfg.key_swap.d = cfg_reg.key_swap.q;
348
349 assign reg_hash_start = reg2hw.cmd.hash_start.qe &
reg2hw.cmd.hash_start.q;
350 assign reg_hash_stop = reg2hw.cmd.hash_stop.qe &
reg2hw.cmd.hash_stop.q;
351 assign reg_hash_continue = reg2hw.cmd.hash_continue.qe &
reg2hw.cmd.hash_continue.q;
352 assign reg_hash_process = reg2hw.cmd.hash_process.qe &
reg2hw.cmd.hash_process.q;
353
354 // Error code register
355 assign hw2reg.err_code.de = err_valid;
356 assign hw2reg.err_code.d = err_code;
357
358 // Control signals //
359 // Control signals //
360 // Control signals //
361 assign hash_start = reg_hash_start & sha_en &
~cfg_block & ~invalid_config;
362 assign hash_continue = reg_hash_continue & sha_en &
~cfg_block & ~invalid_config;
363 assign hash_process = reg_hash_process & sha_en &
cfg_block & ~invalid_config;
364 assign hash_start_or_continue = hash_start | hash_continue;
365
366 always_ff @(posedge clk_i or negedge rst_ni) begin
367 if (!rst_ni) begin
368   cfg_block <= '0;
369 end else if (hash_start_or_continue) begin
370   cfg_block <= 1'b 1;
371 end else if (reg_hash_done || reg_hash_stop) begin
372   cfg_block <= 1'b 0;
373 end
374 end
375 // Hold the configuration during the process
376 always_ff @(posedge clk_i or negedge rst_ni) begin
377 if (!rst_ni) begin
378   cfg_reg <= '{
379     hmac_en: '{
380       q: 1'b0,
381       qe: 1'b0
382     },
383     sha_en: '{
384       q: 1'b0,
385       qe: 1'b0
386     },
387     endian_swap: '{
388       q: HMAC_CFG_ENDIAN_SWAP_RESVAL,
389       qe: 1'b0
390     },
391     digest_swap: '{
392       q: HMAC_CFG_DIGEST_SWAP_RESVAL,
393       qe: 1'b0
394     },
395     key_swap: '{
396       q: HMAC_CFG_KEY_SWAP_RESVAL,
397       qe: 1'b0
398     },
399     digest_size: '{
400       q: HMAC_CFG_DIGEST_SIZE_RESVAL,
401       qe: 1'b0
402     },
403     key_length: '{
404       q: HMAC_CFG_KEY_LENGTH_RESVAL,
405       qe: 1'b0
406     },
407     default: '0
408   };
409 end else if (!cfg_block && reg2hw.cfg.hmac_en.qe) begin
410   cfg_reg <= reg2hw.cfg ;
411 end
412 end
413
414 // Open up the MSG_FIFO from the TL-UL port when it is ready
415 always_ff @(posedge clk_i or negedge rst_ni) begin
416 if (!rst_ni) begin
417   msg_allowed <= '0;
418 end else if (hash_start_or_continue) begin
419   msg_allowed <= 1'b 1;
420 end else if (packer_flush_done) begin
421   msg_allowed <= 1'b 0;
422 end
423 end
424
425 // Interrupts //
426 // Interrupts //
427 // Interrupts //
428
429 // instantiate interrupt hardware primitive
430 prim_intr_hw #(Width(1)) intr_hw_hmac_done (
431   .clk_i,

```

```

432   .rst_ni,
433   .event_intr_i (hash_done_event),
434   .reg2hw_intr_enable_q_i (reg2hw.intr_enable.hmac_done.q),
435   .reg2hw_intr_test_q_i (reg2hw.intr_test.hmac_done.q),
436   .reg2hw_intr_test_qe_i (reg2hw.intr_test.hmac_done.qe),
437   .reg2hw_intr_state_q_i (reg2hw.intr_state.hmac_done.q),
438   .hw2reg_intr_state_de_o (hw2reg.intr_state.hmac_done.de),
439   .hw2reg_intr_state_d_o (hw2reg.intr_state.hmac_done.d),
440   .intr_o (intr_hmac_done_o)
441 );
442
443 // FIFO empty interrupt
444 //
445 // The FIFO empty interrupt is **not useful** for software
if:
446 // - The HMAC block is running in HMAC mode and performing
the second round of computing the
447 // final hash of the outer key as well as the result of
the first round using the inner key.
448 // - The FIFO is then managed entirely by the hardware.
449 // - The FIFO is currently not writeable by software.
450 // - Software has already written the Process command. The
HMAC block will now empty the
451 // FIFO and load its content into the SHA2 core, add the
padding and then perform
452 // the final hashing operation. Software cannot append the
message further.
453 // - Software has written the Stop command. The HMAC block
will not wait for further input from
454 // software after finishing the current block.
455 //
456 // The FIFO empty interrupt can be **useful** for software
in particular if:
457 // - The FIFO was completely full previously. However, un-
less the HMAC block is currently
458 // processing a block, it always empties the message FIFO
faster than software can fill it up,
459 // meaning the message FIFO is empty most of the time.
Note, the empty status is signaled only
460 // once after the FIFO was completely full. The FIFO needs
to be full again for the empty
461 // status to be signaled again next time it's empty.
462 logic status_fifo_empty, fifo_empty_gate;
463 logic fifo_empty_negedge, fifo_empty_q;
464 logic fifo_full_posedge, fifo_full_q;
465 logic fifo_full_seen_d, fifo_full_seen_q;
466 assign fifo_empty_negedge = fifo_empty_q & ~fifo_empty;
467 assign fifo_full_posedge = ~fifo_full_q & fifo_full;
468
469 // Track whether the FIFO was full after being empty. We
clear the tracking:
470 // - When receiving the Start, Continue, Process or Stop
command. This is to start over for the
471 // next message.
472 // - When seeing a negative edge on the empty signal. This
signals that software has reacted to
473 // the interrupt and is filling up the FIFO again.
474 assign fifo_full_seen_d =
475   reg_hash_start || reg_hash_continue ||
476   reg_hash_process || reg_hash_stop ? 1'b 0 :
477   fifo_empty_negedge ? 1'b 0 :
478   fifo_full_posedge ? 1'b 1 :
fifo_full_seen_q;
479
480 // The interrupt is gated unless software is actually al-
lowed to write the FIFO and the FIFO was
481 // full before.
482 assign fifo_empty_gate = ~msg_allowed || ~fifo_full_seen_q;
483
484 assign status_fifo_empty = fifo_empty_gate ? 1'b 0 :
fifo_empty;
485
486 always_ff @(posedge clk_i or negedge rst_ni) begin
487 if (!rst_ni) begin
488   fifo_empty_q <= 1'b 0;
489   fifo_full_q <= 1'b 0;
490   fifo_full_seen_q <= 1'b 0;
491 end else begin
492   fifo_empty_q <= fifo_empty;
493   fifo_full_q <= fifo_full;
494   fifo_full_seen_q <= fifo_full_seen_d;
495 end
496 end
497
498 prim_intr_hw #(
499   .Width(1),
500   .IntrT("Status")
501 ) intr_hw_fifo_empty (
502   .clk_i,
503   .rst_ni,
504   .event_intr_i (status_fifo_empty),
505   .reg2hw_intr_enable_q_i (reg2hw.intr_enable.fifo_empty.q),
506   .reg2hw_intr_test_q_i (reg2hw.intr_test.fifo_empty.q),
507   .reg2hw_intr_test_qe_i (reg2hw.intr_test.fifo_empty.qe),
508   .reg2hw_intr_state_q_i (reg2hw.intr_state.fifo_empty.q),
509   .hw2reg_intr_state_de_o (hw2reg.intr_state.fifo_empty.de),
510   .hw2reg_intr_state_d_o (hw2reg.intr_state.fifo_empty.d),
511   .intr_o (intr_fifo_empty_o)
512 );

```



```

513 prim_intr_hw #(.Width(1)) intr_hw_hmac_err (
514   .clk_i,
515   .rst_ni,
516   .event_intr_i      (err_valid),
517   .reg2hw_intr_enable_q_i (reg2hw.intr_enable.hmac_err.q),
518   .reg2hw_intr_test_q_i   (reg2hw.intr_test.hmac_err.q),
519   .reg2hw_intr_test_qe_i  (reg2hw.intr_test.hmac_err.qe),
520   .reg2hw_intr_state_q_i  (reg2hw.intr_state.hmac_err.q),
521   .hw2reg_intr_state_de_o (hw2reg.intr_state.hmac_err.de),
522   .hw2reg_intr_state_d_o  (hw2reg.intr_state.hmac_err.d),
523   .intr_o                 (intr_hmac_err_o)
524 );
525
526 //////////////////////////////////////////////////
527 // Instances //
528 //////////////////////////////////////////////////
529
530 assign msg_fifo_rdata = '1; // Return all F
531 assign msg_fifo_rerror = '1; // Return error for read access
532 assign msg_fifo_gnt   = msg_fifo_req & ~hmac_fifo_wsel &
packer_ready;
533
534 //////////////////////////////////////////////////
535 // Unused Signals //
536 //////////////////////////////////////////////////
537 logic unused_signals;
538 assign unused_signals = ^{reg_fifo_wmask[7:1],
reg_fifo_wmask[15:9],
reg_fifo_wmask[23:17],
reg_fifo_wmask[31:25]};
540
541 // FIFO control: from packer into message FIFO
542 sha_fifo32_t reg_fifo_wentry;
543 assign reg_fifo_wentry.data = conv_endian32(reg_fifo_wdata,
'1'b1); // always convert
544 assign reg_fifo_wentry.mask = {reg_fifo_wmask[0],
reg_fifo_wmask[8],
reg_fifo_wmask[16],
reg_fifo_wmask[24]};
545
546 assign fifo_empty = ~fifo_rvalid;
547 assign fifo_wvalid = (hmac_fifo_wsel && fifo_wready) ?
hmac_fifo_wvalid : reg_fifo_wvalid;
548
549 logic index;
550 always_comb begin : select_fifo_wdata
551   // default when !hmac_fifo_wsel
552   index = '1'b0;
553   fifo_wdata = reg_fifo_wentry;
554
555   if (hmac_fifo_wsel) begin
556     fifo_wdata = '0;
557     if (digest_size == SHA2_256) begin
558       // only reads out lower 32 bits of each digest word
559       // and discards upper 32-bit zero padding
560       fifo_wdata = '{data: digest[hmac_fifo_wdata_sel[2:0]]
[31:0], mask: '1};
561     end else if ((digest_size == SHA2_384) || (digest_size
== SHA2_512)) begin
562       // reads out first upper 32 bits then lower 32 bits of
each digest word
563       index = !hmac_fifo_wdata_sel[0];
564       fifo_wdata = '{data: digest[hmac_fifo_wdata_sel[3:1]]
[32*index+:32], mask: '1};
565     end
566   end
567
568   // Extended for 1024-bit block
569   localparam int MsgFifoDepth = 32;
570   prim_fifo_sync #(
571     .Width ($bits(sha_fifo32_t)),
572     .Pass ('1'b1),
573     .Depth (MsgFifoDepth),
574     .NeverClears ('1'b1)
575   ) u_msg_fifo (
576     .clk_i,
577     .rst_ni,
578     .clr_i ('1'b0),
579
580     .wvalid_i(fifo_wvalid & sha_en),
581     .wready_o(fifo_wready),
582     .wdata_i (fifo_wdata),
583
584     .depth_o (fifo_depth),
585     .full_o (fifo_full),
586
587     .rvalid_o(fifo_rvalid),
588     .rready_i(fifo_rready),
589     .rdata_o (fifo_rdata),
590     .err_o   ()
591   );
592
593 // TL ADAPTER SRAM
594 tlul_adapter_sram #(
595   .SramAw (9),
596   .SramDw (32),
597   .Outstanding (1),
598   .ByteAccess (1),
599   .ErrOnRead (1)

```

```

600 ) u_tlul_adapter (
601   .clk_i,
602   .rst_ni,
603   .tl_i      (tl_win_h2d),
604   .tl_o      (tl_win_d2h),
605   .en_ifetch_i (prim_mubi_pkg::MuBi4False),
606   .req_o      (msg_fifo_req ),
607   .req_type_o  ( ),
608   .gnt_i      (msg_fifo_gnt ),
609   .we_o      (msg_fifo_we ),
610   .addr_o     ( ), // Doesn't
care the address
611 // other
than sub-word
612   .wdata_o      (msg_fifo_wdata ),
613   .wmask_o      (msg_fifo_wmask ),
614   .intg_error_o  ( ),
615   .user_rsvd_o  ( ),
616   .rdata_i      (msg_fifo_rdata ),
617   .rvalid_i     (1'b0 ),
618   .rerror_i     (msg_fifo_rerror),
619   .compound_txn_in_progress_o ( ),
620   .readback_en_i (prim_mubi_pkg::MuBi4False),
621   .readback_error_o ( ),
622   .wr_collision_i (1'b0),
623   .write_pending_i (1'b0)
624 );
625
626 // TL-UL to MSG_FIFO byte write handling
627 logic msg_write;
628
629 assign msg_write = msg_fifo_req & msg_fifo_we &
~hmac_fifo_wsel & msg_allowed;
630
631 logic [$clog2(32+1)-1:0] wmask_ones;
632
633 always_comb begin
634   wmask_ones = '0;
635   for (int i = 0 ; i < 32 ; i++) begin
636     wmask_ones = wmask_ones + msg_fifo_wmask[i];
637   end
638 end
639
640 // Calculate written message
641 always_ff @(posedge clk_i or negedge rst_ni) begin
642   if (!rst_ni) message_length <= '0;
643   else message_length <= message_length_d;
644 end
645
646 always_comb begin
647   message_length_d = message_length;
648   if (!cfg_block) begin
649     if (reg2hw.msg_length_lower.qe) begin
650       message_length_d[31:0] = reg2hw.msg_length_lower.q;
651     end
652     if (reg2hw.msg_length_upper.qe) begin
653       message_length_d[63:32] = reg2hw.msg_length_upper.q;
654     end
655   end
656
657   if (hash_start) begin
658     message_length_d = '0;
659   end else if (msg_write && sha_en && packer_ready) begin
660     message_length_d = message_length + 64*(wmask_ones);
661   end
662 end
663
664 assign hw2reg.msg_length_upper.d = message_length[63:32];
665 assign hw2reg.msg_length_lower.d = message_length[31:0];
666
667 // Convert endian here
668 // prim_packer always packs to the right, but SHA engine
assumes incoming
669 // to be big-endian, [31:24] comes first. So, the data is
reverted after
670 // prim_packer before the message fifo. here to reverse
if not big-endian
671 // before pushing to the packer.
672
672 assign msg_fifo_wdata_endian = conv_endian32(msg_fifo_wdata,
endian_swap);
673
673 assign msg_fifo_wmask_endian = conv_endian32(msg_fifo_wmask,
endian_swap);
674
675 prim_packer #(
676   .InW (32),
677   .OutW (32),
678   .EnProtection (1'b 0)
679 ) u_packer (
680   .clk_i,
681   .rst_ni,
682
683   .valid_i (msg_write & sha_en),
684   .data_i (msg_fifo_wdata_endian),
685   .mask_i (msg_fifo_wmask_endian),
686   .ready_o (packer_ready),
687
688   .valid_o (reg_fifo_wvalid),
689   .data_o (reg_fifo_wdata),

```



```

690 .mask_o      (reg_fifo_wmask),
691 .ready_i     (fifo_wready & ~hmac_fifo_wsel),
692
693 .flush_i     (hash_process),
694 .flush_done_o (packer_flush_done), // ignore at this mo-
ment
695
696 .err_o      () // Not used
697 );
698
699 hmac_core u_hmac (
700 .clk_i,
701 .rst_ni,
702 .secret_key_i (secret_key),
703 .hmac_en_i    (hmac_en),
704 .digest_size_i (digest_size),
705 .key_length_i (key_length),
706
707 .reg_hash_start_i (hash_start),
708 .reg_hash_stop_i  (reg_hash_stop),
709 .reg_hash_continue_i (hash_continue),
710 .reg_hash_process_i (packer_flush_done), // Trigger after
all msg written
711 .hash_done_o      (reg_hash_done),
712 .sha_hash_start_o (sha_hash_start),
713 .sha_hash_continue_o (sha_hash_continue),
714 .sha_hash_process_o (sha_hash_process),
715 .sha_hash_done_i   (sha_hash_done),
716
717 .sha_rvalid_o      (shaf_rvalid),
718 .sha_rdata_o       (shaf_rdata),
719 .sha_rready_i      (shaf_rready),
720
721 .fifo_rvalid_i     (fifo_rvalid),
722 .fifo_rdata_i      (fifo_rdata),
723 .fifo_rready_o     (fifo_rready),
724
725 .fifo_wsel_o       (hmac_fifo_wsel),
726 .fifo_wvalid_o     (hmac_fifo_wvalid),
727 .fifo_wdata_sel_o  (hmac_fifo_wdata_sel),
728 .fifo_wready_i     (fifo_wready),
729
730 .message_length_i  (message_length),
731 .sha_message_length_o (sha_message_length),
732
733 .idle_o            (hmac_core_idle)
734 );
735
736 // Instantiate SHA-2 256/384/512 engine
737 prim_sha2_32 #(
738 .MultimodeEn(1)
739 ) u_prim_sha2_512 (
740 .clk_i,
741 .rst_ni,
742 .wipe_secret_i      (wipe_secret),
743 .wipe_v_i           (wipe_v),
744 .fifo_rvalid_i      (shaf_rvalid),
745 .fifo_rdata_i       (shaf_rdata),
746 .fifo_rready_o      (shaf_rready),
747 .sha_en_i            (sha_en),
748 .hash_start_i        (sha_hash_start),
749 .hash_stop_i         (reg_hash_stop),
750 .hash_continue_i     (sha_hash_continue),
751 .digest_mode_i       (digest_size),
752 .hash_process_i      (sha_hash_process),
753 .message_length_i    (sha_message_length),
754 .digest_i            (digest_sw),
755 .digest_we_i         (digest_sw_we),
756 .digest_o            (digest),
757 .hash_running_o      (hash_running),
758 .digest_on_blk_o     (digest_on_blk),
759 .hash_done_o         (sha_hash_done),
760 .idle_o              (sha_core_idle)
761 );
762
763 // Register top
764 logic [NumAlerts-1:0] alert_test, alerts;
765 hmac_reg_top u_reg (
766 .clk_i,
767 .rst_ni,
768
769 .tl_i,
770 .tl_o,
771
772 .tl_win_o (tl_win_h2d),
773 .tl_win_i (tl_win_d2h),
774
775 .reg2hw,
776 .hw2reg,
777
778 // SEC_CM: BUS_INTEGRITY
779 .intg_err_o (alerts[0])
780 );
781
782 // Alerts
783 assign alert_test = {
784 reg2hw.alert_test.q &
785 reg2hw.alert_test.qe
786 };
787

```

```

788 localparam logic [NumAlerts-1:0] AlertIsFatal = {1'b1};
789 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
790     prim_alert_sender #(
791         .AsyncOn(AlertAsyncOn[i]),
792         .SkewCycles(AlertSkewCycles),
793         .IsFatal(AlertIsFatal[i])
794     ) u_prim_alert_sender (
795         .clk_i,
796         .rst_ni,
797         .alert_test_i ( alert_test[i] ),
798         .alert_req_i  ( alerts[i] ),
799         .alert_ack_o  (          ),
800         .alert_state_o (          ),
801         .alert_rx_i   ( alert_rx_i[i] ),
802         .alert_tx_o   ( alert_tx_o[i] )
803     );
804 end
805
806 //////////////////////////////////////////////////
807 // HMAC Error Handling //
808 //////////////////////////////////////////////////
809 logic hash_start_sha_disabled, update_seckey_inprocess;
810 logic hash_start_active; // `reg_hash_start` or
`reg_hash_continue` set when hash already active
811 logic msg_push_not_allowed; // Message is received when
`hash_start_or_continue` isn't set
812
813 assign hash_start_sha_disabled = (reg_hash_start |
reg_hash_continue) & ~sha_en;
814 assign hash_start_active = (reg_hash_start |
reg_hash_continue) & cfg_block;
815 assign msg_push_not_allowed = msg_fifo_req & ~msg_allowed;
816
817 // Invalid/unconfigured HMAC/SHA-2: not configured/invalid
digest size or
818 // not configured/invalid key length for HMAC mode or
819 // key_length = 1024-bit for digest_size = SHA2_256 (max
512-bit is supported for SHA-2 256)
820 assign invalid_config = ((digest_size ==
SHA2_None) |
((key_length == Key_None) &&
hmac_en) |
((key_length == Key_1024) && (di-
gest_size == SHA2_256) && hmac_en));
823
824 // invalid_config at reg_hash_start or reg_hash_continue
will signal an error to the SW
825 assign invalid_config_atstart = (reg_hash_start ||
reg_hash_continue) & invalid_config;
826
827 always_comb begin
828     update_seckey_inprocess = 1'b0;
829     if (cfg_block) begin
830         for (int i = 0; i < 32; i++) begin
831             if (reg2hw.key[i].qe) begin
832                 update_seckey_inprocess = update_seckey_inprocess |
1'b1;
833             end
834         end
835     end else begin
836         update_seckey_inprocess = 1'b0;
837     end
838 end
839
840 // Update ERR_CODE register and interrupt only when no
pending interrupt.
841 // This ensures only the first event of the series of events
can be seen to sw.
842 // It is recommended that the software reads ERR_CODE re-
gister when interrupt
843 // is pending to avoid any race conditions.
844 assign err_valid = ~reg2hw.intr_state.hmac_err.q &
845 ( hash_start_sha_disabled | up-
date_seckey_inprocess
| hash_start_active | msg_push_not_allowed
| invalid_config_atstart);
847
848 always_comb begin
849     // default
850     err_code = NoError;
851
852     priority case (1'b1)
853         // SwInvalidConfig has the highest priority: SW config-
ures HMAC incorrectly
854         invalid_config_atstart: begin
855             err_code = SwInvalidConfig;
856         end
857
858         hash_start_sha_disabled: begin
859             err_code = SwHashStartWhenShaDisabled;
860         end
861
862         hash_start_active: begin
863             err_code = SwHashStartWhenActive;
864         end
865
866         msg_push_not_allowed: begin
867             err_code = SwPushMsgWhenDisallowed;
868         end
869

```

```

870     update_seckey_inprocess: begin
871         err_code = SwUpdateSecretKeyInProcess;
872     end
873
874     default: begin
875         err_code = NoError;
876     end
877 endcase
878 end
879
880 //////////////////////////////////////////////////
881 // Idle output
882 //////////////////////////////////////////////////
883 // TBD this should be connected later
884 // Idle: AND condition of:

```

885 // - packer empty: Currently no way to guarantee the packer is empty.

```

886 // temporary, the logic uses packer output
887 (reg_fifo_wvalid)
888 // - MSG_FIFO --> fifo_rvalid
889 // - HMAC_CORE --> hmac_core_idle
890 // - SHA2_CORE --> sha_core_idle
891 // - Clean interrupt status
892 // ICEBOX(#12958): Revise prim_packer and replace
893 'reg_fifo_wvalid' to the
894 // empty status.
895 assign idle = !reg_fifo_wvalid && !fifo_rvalid
896             && hmac_core_idle && sha_core_idle;
897
898 prim_mubi_pkg::mubi4_t idle_q, idle_d;
899 assign idle_d = prim_mubi_pkg::mubi4_bool_to_mubi(idle);
900 always_ff @(posedge clk_i or negedge rst_ni) begin
901     if (!rst_ni) begin
902         idle_q <= prim_mubi_pkg::MuBi4False;
903     end else begin
904         idle_q <= idle_d;
905     end
906 end
907 assign idle_o = idle_q;
908
909 //////////////////////////////////////////////////
910 // Assertions, Assumptions, and Coverpoints
911 //////////////////////////////////////////////////
912 `ifndef VERILATOR
913 `ifndef SYNTHESIS
914 // HMAC assumes TL-UL mask is byte-aligned.
915 property wmask_bytealign_p(wmask_byte, clk, rst_n);
916 @ (posedge clk) disable iff (rst_n == 0)
917     msg_fifo_req & msg_fifo_we -> wmask_byte inside {'0,
918 '1';
919 endproperty
920
921 for (genvar i = 0; i < 4; i++) begin:
922     gen_assert_wmask_bytealign
923
924 // To pass FPV, this shouldn't add pragma translate_off even
925 // these two signals
926 // are used in Assertion only
927 logic in_process;
928 always_ff @(posedge clk_i or negedge rst_ni) begin
929     if (!rst_ni)
930         in_process <=
931 1'b0;
932     else if (hash_process || reg_hash_stop)
933         in_process <=
934 1'b1;
935     else if (reg_hash_done)
936         in_process <=
937 1'b0;
938     end
939
940 logic initiated;
941 always_ff @(posedge clk_i or negedge rst_ni) begin
942     if (!rst_ni)
943         initiated <=
944 1'b0;
945     else if (hash_start_or_continue)
946         initiated <=
947 1'b1;
948     else if (hash_process || reg_hash_stop)
949         initiated <=
950 1'b0;
951     end
952
953 // the host doesn't write data after hash_process until
954 hash_start_or_continue.
955 `ASSERT(ValidWriteAssert, msg_fifo_req -> !in_process)
956
957 // Below condition is covered by the design (2020-02-19)
958 `ASSERT(ValidHashStartAssert, hash_start_or_continue -> !
959 initiated)
960 // 'hash_process' or 'reg_hash_stop' should be toggled and
961 paired with 'hash_start_or_continue'
962 `ASSERT(ValidHashProcessAssert, (hash_process ||
963 reg_hash_stop) -> initiated)
964
965 // hmac_en should be modified only when the logic is Idle
966 `ASSERT(ValidHmacEnConditionAssert,
967         hmac_en != $past(hmac_en) -> !in_process && !initiated)

```

```

950
951 // When wipe_secret is high, sensitive internal variables
952 // are cleared by extending the wipe
953 // value specified in the register
954 `ASSERT(WipeSecretKeyAssert,
955         wipe_secret |=> (secret_key == {($bits(secret_key)/
956 $bits(wipe_v)){$past(wipe_v)}}))
957
958 // All outputs should be known value after reset
959 `ASSERT_KNOWN(IntrHmacDoneKnown, intr_hmac_done_o)
960 `ASSERT_KNOWN(IntrFifoEmptyKnown, intr_fifo_empty_o)
961 `ASSERT_KNOWN(Tl0dValidKnown, tl_o.d_valid)
962 `ASSERT_KNOWN(Tl0AReadyKnown, tl_o.a_ready)
963 `ASSERT_KNOWN(AlertKnown0_A, alert_tx_o)
964
965 `endif // SYNTHESIS
966 `endif // VERILATOR
967
968 // Alert assertions for reg_we onehot check
969 `AS-
970 SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
971 u_reg, alert_tx_o[0])
972 endmodule

```

2. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_core.sv (505 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 //
6 // HMAC Core implementation
7
8 `include "prim_assert.sv"
9
10 module hmac_core import prim_sha2_pkg::*; (
11     input clk_i,
12     input rst_ni,
13
14     input [1023:0] secret_key_i, // {word0, word1, ...,
15     word7}
16     input hmac_en_i,
17     input digest_mode_e digest_size_i,
18     input key_length_e key_length_i,
19
20     input reg_hash_start_i,
21     input reg_hash_stop_i,
22     input reg_hash_continue_i,
23     input reg_hash_process_i,
24     output logic hash_done_o,
25     output logic sha_hash_start_o,
26     output logic sha_hash_continue_o,
27     output logic sha_hash_process_o,
28     input sha_hash_done_i,
29
30     // fifo
31     output logic sha_rvalid_o,
32     output sha_fifo32_t sha_rdata_o,
33     input sha_rready_i,
34
35     input fifo_rvalid_i,
36     input sha_fifo32_t fifo_rdata_i,
37     output logic fifo_rready_o,
38
39     // fifo control (select and fifo write data)
40
41     output logic fifo_wsel_o, // 0: from reg, 1: from
42     digest
43     output logic fifo_wvalid_o,
44     // 0: digest[0][upper], 1: digest[0][lower] .. 14: digest[7]
45     [upper], 15: digest[7][lower]
46     output logic [3:0] fifo_wdata_sel_o,
47     input fifo_wready_i,
48
49     input [63:0] message_length_i,
50     output [63:0] sha_message_length_o,
51
52     output logic idle_o
53 );
54
55 localparam int unsigned BlockSizeSHA256 = 512;
56 localparam int unsigned BlockSizeSHA512 = 1024;
57
58 localparam int unsigned BlockSizeBitsSHA256 = $clog2(BlockSizeSHA256);
59 localparam int unsigned BlockSizeBitsSHA512 = $clog2(BlockSizeSHA512);
60
61 localparam int unsigned HashWordBitsSHA256 = $clog2($bits(sha_word32_t));
62
63 localparam bit [63:0] BlockSizeSHA256in64 = 64'(BlockSizeSHA256);
64 localparam bit [63:0] BlockSizeSHA512in64 = 64'(BlockSizeSHA512);
65
66 logic hash_start; // generated from internal state machine
67 logic hash_continue; // generated from internal state machine

```

```

chine
63 logic hash_process; // generated from internal state machine
chine to trigger hash
64 logic hmac_hash_done;
65
66 logic [BlockSizeSHA256-1:0] i_pad_256;
67 logic [BlockSizeSHA512-1:0] i_pad_512;
68 logic [BlockSizeSHA256-1:0] o_pad_256;
69 logic [BlockSizeSHA512-1:0] o_pad_512;
70
71 logic [63:0] txcount, txcount_d; // works for both digest
lengths
72
73 logic [BlockSizeBitsSHA512-HashWordBitsSHA256-1:0] pad_in-
dex_512;
74 logic [BlockSizeBitsSHA256-HashWordBitsSHA256-1:0] pad_in-
dex_256;
75 logic clr_txcount, load_txcount, inc_txcount;
76
77 logic hmac_sha_rvalid;
78
79 logic idle_d, idle_q;
80 logic reg_hash_stop_d, reg_hash_stop_q;
81
82 typedef enum logic [1:0] {
83     SelIPad,
84     SelOPad,
85     SelFifo
86 } sel_rdata_t;
87
88 sel_rdata_t sel_rdata;
89
90 typedef enum logic {
91     SelIPadMsg,
92     SelOPadMsg
93 } sel_msglen_t;
94
95 sel_msglen_t sel_msglen;
96
97 typedef enum logic {
98     Inner, // Update when state goes to StIPad
99     Outer // Update when state enters StOPad
100 } round_t;
101
102 logic update_round;
103 round_t round_q, round_d;
104
105 typedef enum logic [2:0] {
106     StIdle,
107     StIPad,
108     StMsg, // Actual Msg, and Digest both
109     StPushToMsgFifo, // Digest --> Msg Fifo
110     StWaitResp, // Hash done( by checking pro-
cessed_length? or hash_done)
111     StOPad,
112     StDone // hmac_done
113 } st_e;
114
115 st_e st_q, st_d;
116
117 logic clr_fifo_wdata_sel;
118 logic txcnt_eq_blksz;
119
120 logic reg_hash_process_flag;
121
122 assign sha_hash_start_o = (hmac_en_i) ? hash_start :
reg_hash_start_i;
123 assign sha_hash_continue_o = (hmac_en_i) ? hash_continue :
reg_hash_continue_i;
124
125 assign sha_hash_process_o = (hmac_en_i) ?
reg_hash_process_i | hash_process : reg_hash_process_i;
126 assign hash_done_o = (hmac_en_i) ?
hmac_hash_done : sha_hash_done_i;
127
128 assign pad_index_512 = tx-
count[BlockSizeBitsSHA512-1:HashWordBitsSHA256];
129 assign pad_index_256 = tx-
count[BlockSizeBitsSHA256-1:HashWordBitsSHA256];
130
131 // adjust inner and outer padding depending on key length
and block size
132 always_comb begin : adjust_key_pad_length
133     // set defaults
134     i_pad_256 = '{default: '0';
135     i_pad_512 = '{default: '0';
136     o_pad_256 = '{default: '0';
137     o_pad_512 = '{default: '0';
138
139     unique case (key_length_i)
140         Key_128: begin
141             i_pad_256 = {secret_key_i[1023:896],
142                 {(BlockSizeSHA256-128){1'b0}}} ^ {(BlockS-
izeSHA256/8){8'h36}};
143             i_pad_512 = {secret_key_i[1023:896],
144                 {(BlockSizeSHA512-128){1'b0}}} ^ {(BlockS-
izeSHA512/8){8'h36}};
145             o_pad_256 = {secret_key_i[1023:896],
146                 {(BlockSizeSHA256-128){1'b0}}} ^ {(BlockS-
izeSHA256/8){8'h5c}};
147             o_pad_512 = {secret_key_i[1023:896],

```

```

148                 {(BlockSizeSHA512-128){1'b0}}} ^ {(BlockS-
izeSHA512/8){8'h5c}};
149         end
150         Key_256: begin
151             i_pad_256 = {secret_key_i[1023:768],
152                 {(BlockSizeSHA256-256){1'b0}}} ^ {(BlockS-
izeSHA256/8){8'h36}};
153             i_pad_512 = {secret_key_i[1023:768],
154                 {(BlockSizeSHA512-256){1'b0}}} ^ {(BlockS-
izeSHA512/8){8'h36}};
155             o_pad_256 = {secret_key_i[1023:768],
156                 {(BlockSizeSHA256-256){1'b0}}} ^ {(BlockS-
izeSHA256/8){8'h5c}};
157             o_pad_512 = {secret_key_i[1023:768],
158                 {(BlockSizeSHA512-256){1'b0}}} ^ {(BlockS-
izeSHA512/8){8'h5c}};
159         end
160         Key_384: begin
161             i_pad_256 = {secret_key_i[1023:640],
162                 {(BlockSizeSHA256-384){1'b0}}} ^ {(BlockS-
izeSHA256/8){8'h36}};
163             i_pad_512 = {secret_key_i[1023:640],
164                 {(BlockSizeSHA512-384){1'b0}}} ^ {(BlockS-
izeSHA512/8){8'h36}};
165             o_pad_256 = {secret_key_i[1023:640],
166                 {(BlockSizeSHA256-384){1'b0}}} ^ {(BlockS-
izeSHA256/8){8'h5c}};
167             o_pad_512 = {secret_key_i[1023:640],
168                 {(BlockSizeSHA512-384){1'b0}}} ^ {(BlockS-
izeSHA512/8){8'h5c}};
169         end
170         Key_512: begin
171             i_pad_256 = secret_key_i[1023:512] ^ {(BlockSize-
SHA256/8){8'h36}};
172             i_pad_512 = {secret_key_i[1023:512],
173                 {(BlockSizeSHA512-512){1'b0}}} ^ {(BlockS-
izeSHA512/8){8'h36}};
174             o_pad_256 = secret_key_i[1023:512] ^ {(BlockSize-
SHA256/8){8'h5c}};
175             o_pad_512 = {secret_key_i[1023:512],
176                 {(BlockSizeSHA512-512){1'b0}}} ^ {(BlockS-
izeSHA512/8){8'h5c}};
177         end
178         Key_1024: begin // not allowed to be configured for
SHA-2 256
179             // zero out for SHA-2 256
180             i_pad_256 = '{default: '0';
181             i_pad_512 = secret_key_i[1023:0] ^ {(BlockSize-
SHA512/8){8'h36}};
182             // zero out for SHA-2 256
183             o_pad_256 = '{default: '0';
184             o_pad_512 = secret_key_i[1023:0] ^ {(BlockSize-
SHA512/8){8'h5c}};
185         end
186         default: begin
187             end
188         endcase
189     end
190
191     assign fifo_rready_o = (hmac_en_i) ? (st_q == StMsg) &
sha_rready_i : sha_rready_i;
192     // sha_rvalid is controlled by State Machine below.
193     assign sha_rvalid_o = (!hmac_en_i) ? fifo_rvalid_i :
hmac_sha_rvalid;
194     assign sha_rdata_o =
195         (!hmac_en_i) ?
fifo_rdata_i
196         : (sel_rdata == SelIPad && digest_size_i == SHA2_256)
? '{data: i_pad_256[(BlockSize-
SHA256-1)-32*pad_index_256-:32], mask: '1' :
197         (sel_rdata == SelIPad && ((digest_size_i == SHA2_384) ||
(digest_size_i == SHA2_512)))
? '{data: i_pad_512[(BlockSize-
SHA512-1)-32*pad_index_512-:32], mask: '1' :
198         (sel_rdata == SelOPad && digest_size_i == SHA2_256)
? '{data: o_pad_256[(BlockSize-
SHA256-1)-32*pad_index_256-:32], mask: '1' :
199         (sel_rdata == SelOPad && ((digest_size_i == SHA2_384) ||
(digest_size_i == SHA2_512)))
? '{data: o_pad_512[(BlockSize-
SHA512-1)-32*pad_index_512-:32], mask: '1' :
200         // Well-defined default case `sel_rdata == SelFifo`.
`sel_rdata == 2'b11` cannot occur.
201         fifo_rdata_i;
202
203     logic [63:0] sha_msg_len;
204
205     always_comb begin: assign_sha_message_length
206         sha_msg_len = '0;
207         if (!hmac_en_i) begin
208             sha_msg_len = message_length_i;
209             // HASH = (o_pad || HASH_INTERMEDIATE (i_pad || msg))
210             // message length for HASH_INTERMEDIATE = block size
(i_pad) + message length
211         end else if (sel_msglen == SelIPadMsg) begin
212             if (digest_size_i == SHA2_256) begin
213                 sha_msg_len = message_length_i + BlockSizeSHA256in64;
214             end else if ((digest_size_i == SHA2_384) || (di-
gest_size_i == SHA2_512)) begin
215                 sha_msg_len = message_length_i + BlockSizeSHA512in64;

```

```

220 // `SHA2_None` is a permissible value in the
`SelIPadMsg` case. It can not occur for the
221 // `SelOPadMsg` message length.
222 end else begin
223   sha_msg_len = '0';
224 end
225 end else begin // `SelOPadMsg`
226   // message length for HASH = block size (o_pad) +
HASH_INTERMEDIATE digest length
227   if (digest_size_i == SHA2_256) begin
228     sha_msg_len = BlockSizeSHA256in64 + 64'd256;
229   end else if (digest_size_i == SHA2_384) begin
230     sha_msg_len = BlockSizeSHA512in64 + 64'd384;
231   end else begin // SHA512
232     sha_msg_len = BlockSizeSHA512in64 + 64'd512;
233   end
234 end
235 end
236
237 assign sha_message_length_o = sha_msg_len;
238
239 always_comb begin
240   txcnt_eq_blkisz = '0';
241
242   unique case (digest_size_i)
243     SHA2_256: txcnt_eq_blkisz = (tx-
count[BlockSizeSHA256-1:0] == '0') && (txcount != '0');
244     SHA2_384: txcnt_eq_blkisz = (tx-
count[BlockSizeSHA512-1:0] == '0') && (txcount != '0');
245     SHA2_512: txcnt_eq_blkisz = (tx-
count[BlockSizeSHA512-1:0] == '0') && (txcount != '0');
246     default;
247   endcase
248 end
249
250 assign inc_txcount = sha_rready_i && sha_rvalid_o;
251
252 // txcount
253 // Looks like txcount can be removed entirely here in
hmac_core
254 // In the first round (InnerPaddedKey), it can just watch
process and hash_done
255 // In the second round, it only needs count 256 bits for
hash digest to trigger
256 // hash_process to SHA2
257 always_comb begin
258   txcount_d = txcount;
259   if (clr_txcount) begin
260     txcount_d = '0';
261   end else if (load_txcount) begin
262     // when loading, add block size to the message length
because the SW-visible message length
263     // does not include the block containing the key xor'ed
with the inner pad.
264     unique case (digest_size_i)
265       SHA2_256: txcount_d = message_length_i + BlockSize-
SHA256in64;
266       SHA2_384: txcount_d = message_length_i + BlockSize-
SHA512in64;
267       SHA2_512: txcount_d = message_length_i + BlockSize-
SHA512in64;
268       default : txcount_d = message_length_i + '0';
269     endcase
270   end else if (inc_txcount) begin
271     txcount_d[63:5] = txcount[63:5] + 1'b1; // increment by
32 (data word size)
272   end
273 end
274
275 always_ff @(posedge clk_i or negedge rst_ni) begin
276   if (!rst_ni) txcount <= '0';
277   else txcount <= txcount_d;
278 end
279
280 // reg_hash_process_i trigger logic
281 always_ff @(posedge clk_i or negedge rst_ni) begin
282   if (!rst_ni) begin
283     reg_hash_process_flag <= 1'b0;
284   end else if (reg_hash_process_i) begin
285     reg_hash_process_flag <= 1'b1;
286   end else if (hmac_hash_done || reg_hash_start_i ||
reg_hash_continue_i) begin
287     reg_hash_process_flag <= 1'b0;
288   end
289 end
290
291 always_ff @(posedge clk_i or negedge rst_ni) begin
292   if (!rst_ni) begin
293     round_q <= Inner;
294   end else if (update_round) begin
295     round_q <= round_d;
296   end
297 end
298
299 always_ff @(posedge clk_i or negedge rst_ni) begin
300   if (!rst_ni) begin
301     fifo_wdata_sel_o <= '0';
302   end else if (clr_fifo_wdata_sel) begin
303     fifo_wdata_sel_o <= '0';
304   end else if (fifo_wsel_o && fifo_wvalid_o) begin

```

```

305     fifo_wdata_sel_o <= fifo_wdata_sel_o + 1'b1; // incre-
ment by 1
306   end
307 end
308
309 assign sel_msglen = (round_q == Inner) ? SelIPadMsg : SelO-
PadMsg ;
310
311 always_ff @(posedge clk_i or negedge rst_ni) begin :
state_ff
312   if (!rst_ni) st_q <= StIdle;
313   else st_q <= st_d;
314 end
315
316 always_comb begin : next_state
317   hmac_hash_done = 1'b0;
318   hmac_sha_rvalid = 1'b0;
319   clr_txcount = 1'b0;
320   load_txcount = 1'b0;
321   update_round = 1'b0;
322   round_d = Inner;
323   fifo_wsel_o = 1'b0; // from register
324   fifo_wvalid_o = 1'b0;
325   clr_fifo_wdata_sel = 1'b1;
326   sel_rdata = SelFifo;
327   hash_start = 1'b0;
328   hash_continue = 1'b0;
329   hash_process = 1'b0;
330   st_d = st_q;
331
332   unique case (st_q)
333     StIdle: begin
334       // reset round to Inner
335       // we always switch context into inner round since
outer round computes once over
336       // single block at the end (outer key pad + inner
hash)
337       update_round = 1'b1;
338       round_d = Inner;
339       if (hmac_en_i && reg_hash_start_i) begin
340         st_d = StIPad; // start at StIPad if told to start
341       end
342       clr_txcount = 1'b1;
343       hash_start = 1'b1;
344       end else if (hmac_en_i && reg_hash_continue_i) begin
345         st_d = StMsg; // skip StIPad if told to continue -
assumed it finished StIPad
346       load_txcount = 1'b1;
347       hash_continue = 1'b1;
348       end else begin
349         st_d = StIdle;
350       end
351     end
352   end
353
354   StIPad: begin
355     sel_rdata = SelIPad;
356
357     if (txcnt_eq_blkisz) begin
358       st_d = StMsg;
359     end
360     hmac_sha_rvalid = 1'b0; // block new read request
361   end else begin
362     st_d = StIPad;
363
364     hmac_sha_rvalid = 1'b1;
365   end
366 end
367
368   StMsg: begin
369     sel_rdata = SelFifo;
370     fifo_wsel_o = (round_q == Outer);
371
372     if ( (((round_q == Inner) && reg_hash_process_flag) ||
(round_q == Outer))
373         && (txcount >= sha_message_length_o)) begin
374       st_d = StWaitResp;
375     end
376     hmac_sha_rvalid = 1'b0; // block reading words from
MSG FIFO
377     hash_process = (round_q == Outer);
378     end else if (txcnt_eq_blkisz && (txcount >= sha_mes-
sage_length_o)
379         && reg_hash_stop_q && (round_q == Inner))
begin
380       // wait till all MSG words are pushed out from FIFO
(txcount reaches msg length)
381       // before transitioning to StWaitResp to wait on
sha_hash_done_i and disabling
382       // reading from MSG FIFO
383       st_d = StWaitResp;
384
385       hmac_sha_rvalid = 1'b0;
386     end else begin
387       st_d = StMsg;
388       hmac_sha_rvalid = fifo_rvalid_i;
389     end
390   end
391 end

```

```

392 StWaitResp: begin
393   hmac_sha_rvalid = 1'b0;
394
395   if (sha_hash_done_i) begin
396     if (round_q == Outer) begin
397       st_d = StDone;
398     end else begin // round_q == Inner
399       if (reg_hash_stop_q) begin
400         st_d = StDone;
401       end else begin
402         st_d = StPushToMsgFifo;
403       end
404     end
405   end else begin
406     st_d = StWaitResp;
407   end
408 end
409
410 StPushToMsgFifo: begin
411   hmac_sha_rvalid = 1'b0;
412   fifo_wsel_o = 1'b1;
413   fifo_wvalid_o = 1'b1;
414   clr_fifo_wdata_sel = 1'b0;
415
416   if (fifo_wready_i && (((fifo_wdata_sel_o == 4'd7) &&
417 (digest_size_i == SHA2_256)) ||
418 ((fifo_wdata_sel_o == 4'd15) &&
419 (digest_size_i == SHA2_512)) ||
420 ((fifo_wdata_sel_o == 4'd11) &&
421 (digest_size_i == SHA2_384)))) begin
422     st_d = StOPad;
423
424     clr_txcount = 1'b1;
425     update_round = 1'b1;
426     round_d = Outer;
427     hash_start = 1'b1;
428   end else begin
429     st_d = StPushToMsgFifo;
430   end
431 end
432
433 StOPad: begin
434   sel_rdata = SelOPad;
435   fifo_wsel_o = 1'b1; // Remained HMAC select to indic-
436 ate HMAC is in second stage
437
438   if (txcnt_eq_blkisz) begin
439     st_d = StMsg;
440
441     hmac_sha_rvalid = 1'b0; // block new read request
442   end else begin
443     st_d = StOPad;
444
445     hmac_sha_rvalid = 1'b1;
446   end
447 end
448
449 StDone: begin
450   // raise interrupt (hash_done)
451   st_d = StIdle;
452
453   hmac_hash_done = 1'b1;
454 end
455
456 default: begin
457   st_d = StIdle;
458 end
459 endcase
460 end
461
462 // raise reg_hash_stop_d flag at reg_hash_stop_i and keep it
463 until sha_hash_done_i is asserted
464 // to indicate the hashing operation on current block has
465 completed
466 assign reg_hash_stop_d = (reg_hash_stop_i ==
467 1'b1) ? 1'b1 :
468 (sha_hash_done_i == 1'b1 &&
469 reg_hash_stop_q == 1'b1) ? 1'b0 :
470 reg_hash_stop_q;
471
472 reg_hash_stop_q;
473
474 always_ff @(posedge clk_i or negedge rst_ni) begin
475   if (!rst_ni) begin
476     reg_hash_stop_q <= 1'b0;
477   end else begin
478     reg_hash_stop_q <= reg_hash_stop_d;
479   end
480 end
481
482 // Idle status signaling: This module ..
483 assign idle_d =
484 // .. is not idle when told to start or continue
485 (reg_hash_start_i || reg_hash_continue_i) ? 1'b0 :
486 // .. is idle when the FSM is in the Idle state
487 (st_q == StIdle) ? 1'b1 :
488 // .. is idle when it has processed a complete block of

```

```

a message and is told to stop in any
482 // FSM state
483 (txcnt_eq_blkisz && reg_hash_stop_d) ? 1'b1 :
484 // .. and keeps the current idle state in all other
485 cases.
486 idle_q;
487 assign idle_o = idle_d;
488
489 always_ff @(posedge clk_i or negedge rst_ni) begin
490   if (!rst_ni) begin
491     idle_q <= 1'b1;
492   end else begin
493     idle_q <= idle_d;
494   end
495 end
496
497 // Assertions //
498 // Assertions //
499 // Assertions //
500
501 `ASSERT(ValidSelRdata_A, hmac_en_i |-> sel_rdata inside
502 {SelIPad, SelOPad, SelFifo})
503 `ASSERT(ValidDigestSize_A, (hmac_en_i && (sel_msglen ==
504 SelOPadMsg)) |->
505 digest_size_i inside {SHA2_256, SHA2_384, SHA2_512})
506 endmodule

```

3. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_reg_pkg.sv (511 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 details.
4 // SPDX-License-Identifier: Apache-2.0
5 // Register Package auto-generated by `reggen` containing data
6 structure
7 package hmac_reg_pkg;
8
9 // Param list
10 parameter int NumDigestWords = 16;
11 parameter int NumKeywords = 32;
12 parameter int NumAlerts = 1;
13
14 // Address widths within the block
15 parameter int BlockAw = 13;
16
17 // Number of registers for every interface
18 parameter int NumRegs = 59;
19
20 // Alert indices
21 typedef enum int {
22   AlertFatalFaultIdx = 0
23 } hmac_alert_idx_t;
24
25 // Typedefs for registers //
26 // Typedefs for registers //
27 // Typedefs for registers //
28
29 typedef struct packed {
30   struct packed {
31     logic q;
32   } hmac_err;
33   struct packed {
34     logic q;
35   } fifo_empty;
36   struct packed {
37     logic q;
38   } hmac_done;
39 } hmac_reg2hw_intr_state_reg_t;
40
41 typedef struct packed {
42   struct packed {
43     logic q;
44   } hmac_err;
45   struct packed {
46     logic q;
47   } fifo_empty;
48   struct packed {
49     logic q;
50   } hmac_done;
51 } hmac_reg2hw_intr_enable_reg_t;
52
53 typedef struct packed {
54   struct packed {
55     logic q;
56     logic qe;
57   } hmac_err;
58   struct packed {
59     logic q;
60     logic qe;
61   } fifo_empty;
62   struct packed {
63     logic q;
64     logic qe;

```



```

65     } hmac_done;
66 } hmac_reg2hw_intr_test_reg_t;
67
68 typedef struct packed {
69     logic q;
70     logic qe;
71 } hmac_reg2hw_alert_test_reg_t;
72
73 typedef struct packed {
74     struct packed {
75         logic [5:0] q;
76         logic qe;
77     } key_length;
78     struct packed {
79         logic [3:0] q;
80         logic qe;
81     } digest_size;
82     struct packed {
83         logic q;
84         logic qe;
85     } key_swap;
86     struct packed {
87         logic q;
88         logic qe;
89     } digest_swap;
90     struct packed {
91         logic q;
92         logic qe;
93     } endian_swap;
94     struct packed {
95         logic q;
96         logic qe;
97     } sha_en;
98     struct packed {
99         logic q;
100        logic qe;
101    } hmac_en;
102 } hmac_reg2hw_cfg_reg_t;
103
104 typedef struct packed {
105     struct packed {
106         logic q;
107         logic qe;
108     } hash_continue;
109     struct packed {
110         logic q;
111         logic qe;
112     } hash_stop;
113     struct packed {
114         logic q;
115         logic qe;
116     } hash_process;
117     struct packed {
118         logic q;
119         logic qe;
120     } hash_start;
121 } hmac_reg2hw_cmd_reg_t;
122
123 typedef struct packed {
124     logic [31:0] q;
125     logic qe;
126 } hmac_reg2hw_wipe_secret_reg_t;
127
128 typedef struct packed {
129     logic [31:0] q;
130     logic qe;
131 } hmac_reg2hw_key_mreg_t;
132
133 typedef struct packed {
134     logic [31:0] q;
135     logic qe;
136 } hmac_reg2hw_digest_mreg_t;
137
138 typedef struct packed {
139     logic [31:0] q;
140     logic qe;
141 } hmac_reg2hw_msg_length_lower_reg_t;
142
143 typedef struct packed {
144     logic [31:0] q;
145     logic qe;
146 } hmac_reg2hw_msg_length_upper_reg_t;
147
148 typedef struct packed {
149     struct packed {
150         logic d;
151         logic de;
152     } hmac_err;
153     struct packed {
154         logic d;
155         logic de;
156     } fifo_empty;
157     struct packed {
158         logic d;
159         logic de;
160     } hmac_done;
161 } hmac_hw2reg_intr_state_reg_t;
162
163 typedef struct packed {
164     struct packed {
165         logic [5:0] d;
166     } key_length;
167     struct packed {
168         logic [3:0] d;
169     } digest_size;
170     struct packed {
171         logic d;
172     } key_swap;
173     struct packed {
174         logic d;
175     } digest_swap;
176     struct packed {
177         logic d;
178     } endian_swap;
179     struct packed {
180         logic d;
181     } sha_en;
182     struct packed {
183         logic d;
184     } hmac_en;
185 } hmac_hw2reg_cfg_reg_t;
186
187 typedef struct packed {
188     struct packed {
189         logic [5:0] d;
190     } fifo_depth;
191     struct packed {
192         logic d;
193     } fifo_full;
194     struct packed {
195         logic d;
196     } fifo_empty;
197     struct packed {
198         logic d;
199     } hmac_idle;
200 } hmac_hw2reg_status_reg_t;
201
202 typedef struct packed {
203     logic [31:0] d;
204     logic de;
205 } hmac_hw2reg_err_code_reg_t;
206
207 typedef struct packed {
208     logic [31:0] d;
209 } hmac_hw2reg_key_mreg_t;
210
211 typedef struct packed {
212     logic [31:0] d;
213 } hmac_hw2reg_digest_mreg_t;
214
215 typedef struct packed {
216     logic [31:0] d;
217 } hmac_hw2reg_msg_length_lower_reg_t;
218
219 typedef struct packed {
220     logic [31:0] d;
221 } hmac_hw2reg_msg_length_upper_reg_t;
222
223 // Register -> HW type
224 typedef struct packed {
225     hmac_reg2hw_intr_state_reg_t intr_state; // [1726:1724]
226     hmac_reg2hw_intr_enable_reg_t intr_enable; // [1723:1721]
227     hmac_reg2hw_intr_test_reg_t intr_test; // [1720:1715]
228     hmac_reg2hw_alert_test_reg_t alert_test; // [1714:1713]
229     hmac_reg2hw_cfg_reg_t cfg; // [1712:1691]
230     hmac_reg2hw_cmd_reg_t cmd; // [1690:1683]
231     hmac_reg2hw_wipe_secret_reg_t wipe_secret; // [1682:1650]
232     hmac_reg2hw_key_mreg_t [31:0] key; // [1649:594]
233     hmac_reg2hw_digest_mreg_t [15:0] digest; // [593:66]
234     hmac_reg2hw_msg_length_lower_reg_t msg_length_lower; //
235     hmac_reg2hw_msg_length_upper_reg_t msg_length_upper; //
236 } hmac_reg2hw_t;
237
238 // HW -> register type
239 typedef struct packed {
240     hmac_hw2reg_intr_state_reg_t intr_state; // [1662:1657]
241     hmac_hw2reg_cfg_reg_t cfg; // [1656:1642]
242     hmac_hw2reg_status_reg_t status; // [1641:1633]
243     hmac_hw2reg_err_code_reg_t err_code; // [1632:1600]
244     hmac_hw2reg_key_mreg_t [31:0] key; // [1599:576]
245     hmac_hw2reg_digest_mreg_t [15:0] digest; // [575:64]
246     hmac_hw2reg_msg_length_lower_reg_t msg_length_lower; //
247     hmac_hw2reg_msg_length_upper_reg_t msg_length_upper; //
248 } hmac_hw2reg_t;
249
250 // Register offsets
251 parameter logic [BlockAw-1:0] HMAC_INTR_STATE_OFFSET = 13'h
0;
252 parameter logic [BlockAw-1:0] HMAC_INTR_ENABLE_OFFSET = 13'h
4;
253 parameter logic [BlockAw-1:0] HMAC_INTR_TEST_OFFSET = 13'h
8;
254 parameter logic [BlockAw-1:0] HMAC_ALERT_TEST_OFFSET = 13'h
c;
255 parameter logic [BlockAw-1:0] HMAC_CFG_OFFSET = 13'h 10;
256 parameter logic [BlockAw-1:0] HMAC_CMD_OFFSET = 13'h 14;
257 parameter logic [BlockAw-1:0] HMAC_STATUS_OFFSET = 13'h 18;

```

```

258 parameter logic [BlockAw-1:0] HMAC_ERR_CODE_OFFSET = 13'h
1c;

259 parameter logic [BlockAw-1:0] HMAC_WIPE_SECRET_OFFSET = 13'h
20;

260 parameter logic [BlockAw-1:0] HMAC_KEY_0_OFFSET = 13'h 24;
261 parameter logic [BlockAw-1:0] HMAC_KEY_1_OFFSET = 13'h 28;
262 parameter logic [BlockAw-1:0] HMAC_KEY_2_OFFSET = 13'h 2c;
263 parameter logic [BlockAw-1:0] HMAC_KEY_3_OFFSET = 13'h 30;
264 parameter logic [BlockAw-1:0] HMAC_KEY_4_OFFSET = 13'h 34;
265 parameter logic [BlockAw-1:0] HMAC_KEY_5_OFFSET = 13'h 38;
266 parameter logic [BlockAw-1:0] HMAC_KEY_6_OFFSET = 13'h 3c;
267 parameter logic [BlockAw-1:0] HMAC_KEY_7_OFFSET = 13'h 40;
268 parameter logic [BlockAw-1:0] HMAC_KEY_8_OFFSET = 13'h 44;
269 parameter logic [BlockAw-1:0] HMAC_KEY_9_OFFSET = 13'h 48;
270 parameter logic [BlockAw-1:0] HMAC_KEY_10_OFFSET = 13'h 4c;
271 parameter logic [BlockAw-1:0] HMAC_KEY_11_OFFSET = 13'h 50;
272 parameter logic [BlockAw-1:0] HMAC_KEY_12_OFFSET = 13'h 54;
273 parameter logic [BlockAw-1:0] HMAC_KEY_13_OFFSET = 13'h 58;
274 parameter logic [BlockAw-1:0] HMAC_KEY_14_OFFSET = 13'h 5c;
275 parameter logic [BlockAw-1:0] HMAC_KEY_15_OFFSET = 13'h 60;
276 parameter logic [BlockAw-1:0] HMAC_KEY_16_OFFSET = 13'h 64;
277 parameter logic [BlockAw-1:0] HMAC_KEY_17_OFFSET = 13'h 68;
278 parameter logic [BlockAw-1:0] HMAC_KEY_18_OFFSET = 13'h 6c;
279 parameter logic [BlockAw-1:0] HMAC_KEY_19_OFFSET = 13'h 70;
280 parameter logic [BlockAw-1:0] HMAC_KEY_20_OFFSET = 13'h 74;
281 parameter logic [BlockAw-1:0] HMAC_KEY_21_OFFSET = 13'h 78;
282 parameter logic [BlockAw-1:0] HMAC_KEY_22_OFFSET = 13'h 7c;
283 parameter logic [BlockAw-1:0] HMAC_KEY_23_OFFSET = 13'h 80;
284 parameter logic [BlockAw-1:0] HMAC_KEY_24_OFFSET = 13'h 84;
285 parameter logic [BlockAw-1:0] HMAC_KEY_25_OFFSET = 13'h 88;
286 parameter logic [BlockAw-1:0] HMAC_KEY_26_OFFSET = 13'h 8c;
287 parameter logic [BlockAw-1:0] HMAC_KEY_27_OFFSET = 13'h 90;
288 parameter logic [BlockAw-1:0] HMAC_KEY_28_OFFSET = 13'h 94;
289 parameter logic [BlockAw-1:0] HMAC_KEY_29_OFFSET = 13'h 98;
290 parameter logic [BlockAw-1:0] HMAC_KEY_30_OFFSET = 13'h 9c;
291 parameter logic [BlockAw-1:0] HMAC_KEY_31_OFFSET = 13'h a0;
292 parameter logic [BlockAw-1:0] HMAC_DIGEST_0_OFFSET = 13'h
a4;
293 parameter logic [BlockAw-1:0] HMAC_DIGEST_1_OFFSET = 13'h
a8;
294 parameter logic [BlockAw-1:0] HMAC_DIGEST_2_OFFSET = 13'h
ac;
295 parameter logic [BlockAw-1:0] HMAC_DIGEST_3_OFFSET = 13'h
b0;
296 parameter logic [BlockAw-1:0] HMAC_DIGEST_4_OFFSET = 13'h
b4;
297 parameter logic [BlockAw-1:0] HMAC_DIGEST_5_OFFSET = 13'h
b8;
298 parameter logic [BlockAw-1:0] HMAC_DIGEST_6_OFFSET = 13'h
bc;
299 parameter logic [BlockAw-1:0] HMAC_DIGEST_7_OFFSET = 13'h
c0;
300 parameter logic [BlockAw-1:0] HMAC_DIGEST_8_OFFSET = 13'h
c4;
301 parameter logic [BlockAw-1:0] HMAC_DIGEST_9_OFFSET = 13'h
c8;
302 parameter logic [BlockAw-1:0] HMAC_DIGEST_10_OFFSET = 13'h
cc;
303 parameter logic [BlockAw-1:0] HMAC_DIGEST_11_OFFSET = 13'h
d0;
304 parameter logic [BlockAw-1:0] HMAC_DIGEST_12_OFFSET = 13'h
d4;
305 parameter logic [BlockAw-1:0] HMAC_DIGEST_13_OFFSET = 13'h
d8;
306 parameter logic [BlockAw-1:0] HMAC_DIGEST_14_OFFSET = 13'h
dc;
307 parameter logic [BlockAw-1:0] HMAC_DIGEST_15_OFFSET = 13'h
e0;

308 parameter logic [BlockAw-1:0] HMAC_MSG_LENGTH_LOWER_OFFSET =
13'h e4;

309 parameter logic [BlockAw-1:0] HMAC_MSG_LENGTH_UPPER_OFFSET =
13'h e8;

310 // Reset values for hwext registers and their fields
311 parameter logic [2:0] HMAC_INTR_TEST_RESVAL = 3'h 0;
312 parameter logic [0:0] HMAC_INTR_TEST_HMAC_DONE_RESVAL = 1'h
0;

314 parameter logic [0:0] HMAC_INTR_TEST_FIFO_EMPTY_RESVAL = 1'h
0;
315 parameter logic [0:0] HMAC_INTR_TEST_HMAC_ERR_RESVAL = 1'h
0;
316 parameter logic [0:0] HMAC_ALERT_TEST_RESVAL = 1'h 0;
317 parameter logic [0:0] HMAC_ALERT_TEST_FATAL_FAULT_RESVAL =
1'h 0;

318 parameter logic [14:0] HMAC_CFG_RESVAL = 15'h 4100;
319 parameter logic [0:0] HMAC_CFG_ENDIAN_SWAP_RESVAL = 1'h 0;
320 parameter logic [0:0] HMAC_CFG_DIGEST_SWAP_RESVAL = 1'h 0;
321 parameter logic [0:0] HMAC_CFG_KEY_SWAP_RESVAL = 1'h 0;
322 parameter logic [3:0] HMAC_CFG_DIGEST_SIZE_RESVAL = 4'h 8;
323 parameter logic [5:0] HMAC_CFG_KEY_LENGTH_RESVAL = 6'h 20;
324 parameter logic [3:0] HMAC_CMD_RESVAL = 4'h 0;
325 parameter logic [9:0] HMAC_STATUS_RESVAL = 10'h 3;
326 parameter logic [0:0] HMAC_STATUS_HMAC_IDLE_RESVAL = 1'h 1;
327 parameter logic [0:0] HMAC_STATUS_FIFO_EMPTY_RESVAL = 1'h 1;
328 parameter logic [31:0] HMAC_WIPE_SECRET_RESVAL = 32'h 0;
329 parameter logic [31:0] HMAC_KEY_0_RESVAL = 32'h 0;
330 parameter logic [31:0] HMAC_KEY_1_RESVAL = 32'h 0;

331 parameter logic [31:0] HMAC_KEY_2_RESVAL = 32'h 0;
332 parameter logic [31:0] HMAC_KEY_3_RESVAL = 32'h 0;
333 parameter logic [31:0] HMAC_KEY_4_RESVAL = 32'h 0;
334 parameter logic [31:0] HMAC_KEY_5_RESVAL = 32'h 0;
335 parameter logic [31:0] HMAC_KEY_6_RESVAL = 32'h 0;
336 parameter logic [31:0] HMAC_KEY_7_RESVAL = 32'h 0;
337 parameter logic [31:0] HMAC_KEY_8_RESVAL = 32'h 0;
338 parameter logic [31:0] HMAC_KEY_9_RESVAL = 32'h 0;
339 parameter logic [31:0] HMAC_KEY_10_RESVAL = 32'h 0;
340 parameter logic [31:0] HMAC_KEY_11_RESVAL = 32'h 0;
341 parameter logic [31:0] HMAC_KEY_12_RESVAL = 32'h 0;
342 parameter logic [31:0] HMAC_KEY_13_RESVAL = 32'h 0;
343 parameter logic [31:0] HMAC_KEY_14_RESVAL = 32'h 0;
344 parameter logic [31:0] HMAC_KEY_15_RESVAL = 32'h 0;
345 parameter logic [31:0] HMAC_KEY_16_RESVAL = 32'h 0;
346 parameter logic [31:0] HMAC_KEY_17_RESVAL = 32'h 0;
347 parameter logic [31:0] HMAC_KEY_18_RESVAL = 32'h 0;
348 parameter logic [31:0] HMAC_KEY_19_RESVAL = 32'h 0;
349 parameter logic [31:0] HMAC_KEY_20_RESVAL = 32'h 0;
350 parameter logic [31:0] HMAC_KEY_21_RESVAL = 32'h 0;
351 parameter logic [31:0] HMAC_KEY_22_RESVAL = 32'h 0;
352 parameter logic [31:0] HMAC_KEY_23_RESVAL = 32'h 0;
353 parameter logic [31:0] HMAC_KEY_24_RESVAL = 32'h 0;
354 parameter logic [31:0] HMAC_KEY_25_RESVAL = 32'h 0;
355 parameter logic [31:0] HMAC_KEY_26_RESVAL = 32'h 0;
356 parameter logic [31:0] HMAC_KEY_27_RESVAL = 32'h 0;
357 parameter logic [31:0] HMAC_KEY_28_RESVAL = 32'h 0;
358 parameter logic [31:0] HMAC_KEY_29_RESVAL = 32'h 0;
359 parameter logic [31:0] HMAC_KEY_30_RESVAL = 32'h 0;
360 parameter logic [31:0] HMAC_KEY_31_RESVAL = 32'h 0;
361 parameter logic [31:0] HMAC_DIGEST_0_RESVAL = 32'h 0;
362 parameter logic [31:0] HMAC_DIGEST_1_RESVAL = 32'h 0;
363 parameter logic [31:0] HMAC_DIGEST_2_RESVAL = 32'h 0;
364 parameter logic [31:0] HMAC_DIGEST_3_RESVAL = 32'h 0;
365 parameter logic [31:0] HMAC_DIGEST_4_RESVAL = 32'h 0;
366 parameter logic [31:0] HMAC_DIGEST_5_RESVAL = 32'h 0;
367 parameter logic [31:0] HMAC_DIGEST_6_RESVAL = 32'h 0;
368 parameter logic [31:0] HMAC_DIGEST_7_RESVAL = 32'h 0;
369 parameter logic [31:0] HMAC_DIGEST_8_RESVAL = 32'h 0;
370 parameter logic [31:0] HMAC_DIGEST_9_RESVAL = 32'h 0;
371 parameter logic [31:0] HMAC_DIGEST_10_RESVAL = 32'h 0;
372 parameter logic [31:0] HMAC_DIGEST_11_RESVAL = 32'h 0;
373 parameter logic [31:0] HMAC_DIGEST_12_RESVAL = 32'h 0;
374 parameter logic [31:0] HMAC_DIGEST_13_RESVAL = 32'h 0;
375 parameter logic [31:0] HMAC_DIGEST_14_RESVAL = 32'h 0;
376 parameter logic [31:0] HMAC_DIGEST_15_RESVAL = 32'h 0;
377 parameter logic [31:0] HMAC_MSG_LENGTH_LOWER_RESVAL = 32'h
0;
378 parameter logic [31:0] HMAC_MSG_LENGTH_UPPER_RESVAL = 32'h
0;

379 // Window parameters
380 parameter logic [BlockAw-1:0] HMAC_MSG_FIFO_OFFSET = 13'h
1000;
381 parameter int unsigned HMAC_MSG_FIFO_SIZE = 'h
1000;
382 parameter int unsigned HMAC_MSG_FIFO_IDX = 0;

383 // Register index
384 typedef enum int {
385     HMAC_INTR_STATE,
386     HMAC_INTR_ENABLE,
387     HMAC_INTR_TEST,
388     HMAC_ALERT_TEST,
389     HMAC_CFG,
390     HMAC_CMD,
391     HMAC_STATUS,
392     HMAC_ERR_CODE,
393     HMAC_WIPE_SECRET,
394     HMAC_KEY_0,
395     HMAC_KEY_1,
396     HMAC_KEY_2,
397     HMAC_KEY_3,
398     HMAC_KEY_4,
399     HMAC_KEY_5,
400     HMAC_KEY_6,
401     HMAC_KEY_7,
402     HMAC_KEY_8,
403     HMAC_KEY_9,
404     HMAC_KEY_10,
405     HMAC_KEY_11,
406     HMAC_KEY_12,
407     HMAC_KEY_13,
408     HMAC_KEY_14,
409     HMAC_KEY_15,
410     HMAC_KEY_16,
411     HMAC_KEY_17,
412     HMAC_KEY_18,
413     HMAC_KEY_19,
414     HMAC_KEY_20,
415     HMAC_KEY_21,
416     HMAC_KEY_22,
417     HMAC_KEY_23,
418     HMAC_KEY_24,
419     HMAC_KEY_25,
420     HMAC_KEY_26,
421     HMAC_KEY_27,
422     HMAC_KEY_28,
423     HMAC_KEY_29,
424     HMAC_KEY_30,
425     HMAC_KEY_31,

```

```

428 HMAC_DIGEST_0,
429 HMAC_DIGEST_1,
430 HMAC_DIGEST_2,
431 HMAC_DIGEST_3,
432 HMAC_DIGEST_4,
433 HMAC_DIGEST_5,
434 HMAC_DIGEST_6,
435 HMAC_DIGEST_7,
436 HMAC_DIGEST_8,
437 HMAC_DIGEST_9,
438 HMAC_DIGEST_10,
439 HMAC_DIGEST_11,
440 HMAC_DIGEST_12,
441 HMAC_DIGEST_13,
442 HMAC_DIGEST_14,
443 HMAC_DIGEST_15,
444 HMAC_MSG_LENGTH_LOWER,
445 HMAC_MSG_LENGTH_UPPER
446 } hmac_id_e;
447
448 // Register width information to check illegal writes
449 parameter logic [3:0] HMAC_PERMIT [59] = '{
450 4'b 0001, // index[ 0] HMAC_INTR_STATE
451 4'b 0001, // index[ 1] HMAC_INTR_ENABLE
452 4'b 0001, // index[ 2] HMAC_INTR_TEST
453 4'b 0001, // index[ 3] HMAC_ALERT_TEST
454 4'b 0011, // index[ 4] HMAC_CFG
455 4'b 0001, // index[ 5] HMAC_CMD
456 4'b 0011, // index[ 6] HMAC_STATUS
457 4'b 1111, // index[ 7] HMAC_ERR_CODE
458 4'b 1111, // index[ 8] HMAC_WIPE_SECRET
459 4'b 1111, // index[ 9] HMAC_KEY_0
460 4'b 1111, // index[10] HMAC_KEY_1
461 4'b 1111, // index[11] HMAC_KEY_2
462 4'b 1111, // index[12] HMAC_KEY_3
463 4'b 1111, // index[13] HMAC_KEY_4
464 4'b 1111, // index[14] HMAC_KEY_5
465 4'b 1111, // index[15] HMAC_KEY_6
466 4'b 1111, // index[16] HMAC_KEY_7
467 4'b 1111, // index[17] HMAC_KEY_8
468 4'b 1111, // index[18] HMAC_KEY_9
469 4'b 1111, // index[19] HMAC_KEY_10
470 4'b 1111, // index[20] HMAC_KEY_11
471 4'b 1111, // index[21] HMAC_KEY_12
472 4'b 1111, // index[22] HMAC_KEY_13
473 4'b 1111, // index[23] HMAC_KEY_14
474 4'b 1111, // index[24] HMAC_KEY_15
475 4'b 1111, // index[25] HMAC_KEY_16
476 4'b 1111, // index[26] HMAC_KEY_17
477 4'b 1111, // index[27] HMAC_KEY_18
478 4'b 1111, // index[28] HMAC_KEY_19
479 4'b 1111, // index[29] HMAC_KEY_20
480 4'b 1111, // index[30] HMAC_KEY_21
481 4'b 1111, // index[31] HMAC_KEY_22
482 4'b 1111, // index[32] HMAC_KEY_23
483 4'b 1111, // index[33] HMAC_KEY_24
484 4'b 1111, // index[34] HMAC_KEY_25
485 4'b 1111, // index[35] HMAC_KEY_26
486 4'b 1111, // index[36] HMAC_KEY_27
487 4'b 1111, // index[37] HMAC_KEY_28
488 4'b 1111, // index[38] HMAC_KEY_29
489 4'b 1111, // index[39] HMAC_KEY_30
490 4'b 1111, // index[40] HMAC_KEY_31
491 4'b 1111, // index[41] HMAC_DIGEST_0
492 4'b 1111, // index[42] HMAC_DIGEST_1
493 4'b 1111, // index[43] HMAC_DIGEST_2
494 4'b 1111, // index[44] HMAC_DIGEST_3
495 4'b 1111, // index[45] HMAC_DIGEST_4
496 4'b 1111, // index[46] HMAC_DIGEST_5
497 4'b 1111, // index[47] HMAC_DIGEST_6
498 4'b 1111, // index[48] HMAC_DIGEST_7
499 4'b 1111, // index[49] HMAC_DIGEST_8
500 4'b 1111, // index[50] HMAC_DIGEST_9
501 4'b 1111, // index[51] HMAC_DIGEST_10
502 4'b 1111, // index[52] HMAC_DIGEST_11
503 4'b 1111, // index[53] HMAC_DIGEST_12
504 4'b 1111, // index[54] HMAC_DIGEST_13
505 4'b 1111, // index[55] HMAC_DIGEST_14
506 4'b 1111, // index[56] HMAC_DIGEST_15
507 4'b 1111, // index[57] HMAC_MSG_LENGTH_LOWER
508 4'b 1111 // index[58] HMAC_MSG_LENGTH_UPPER
509 };
510
511 endpackage

```

4. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_reg_top.sv (2,648 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"
8
9 module hmac_reg_top (
10   input clk_i,

```

```

11   input rst_ni,
12   input tlul_pkg::tl_h2d_t tl_i,
13   output tlul_pkg::tl_d2h_t tl_o,
14
15   // Output port for window
16   output tlul_pkg::tl_h2d_t tl_win_o,
17   input  tlul_pkg::tl_d2h_t tl_win_i,
18
19   // To HW
20   output hmac_reg_pkg::hmac_reg2hw_t reg2hw, // Write
21   input  hmac_reg_pkg::hmac_hw2reg_t hw2reg, // Read
22
23   // Integrity check errors
24   output logic intg_err_o
25 );
26
27 import hmac_reg_pkg::* ;
28
29 localparam int AW = 13;
30 localparam int DW = 32;
31 localparam int DBW = DW/8; // Byte Width
32
33 // register signals
34 logic          reg_we;
35 logic          reg_re;
36 logic [AW-1:0] reg_addr;
37 logic [DW-1:0] reg_wdata;
38 logic [DBW-1:0] reg_be;
39 logic [DW-1:0] reg_rdata;
40 logic          reg_error;
41
42 logic          addrmiss, wr_err;
43
44 logic [DW-1:0] reg_rdata_next;
45 logic reg_busy;
46
47 tlul_pkg::tl_h2d_t tl_reg_h2d;
48 tlul_pkg::tl_d2h_t tl_reg_d2h;
49
50 // incoming payload check
51 logic intg_err;
52 tlul_cmd_intg_chk u_chk (
53   .tl_i(tl_i),
54   .err_o(intg_err)
55 );
56
57 // also check for spurious write enables
58 logic reg_we_err;
59 logic [58:0] reg_we_check;
60 prim_reg_we_check #(
61   .OneHotWidth(59)
62 ) u_prim_reg_we_check (
63   .clk_i(clk_i),
64   .rst_ni(rst_ni),
65   .oh_i (reg_we_check),
66   .en_i (reg_we && !addrmiss),
67   .err_o (reg_we_err)
68 );
69
70 logic err_q;
71 always_ff @(posedge clk_i or negedge rst_ni) begin
72   if (!rst_ni) begin
73     err_q <= '0;
74   end else if (intg_err || reg_we_err) begin
75     err_q <= '1'b1;
76   end
77 end
78
79 // integrity error output is permanent and should be used
  for alert generation
81 // register errors are transactional
82 assign intg_err_o = err_q | intg_err | reg_we_err;
83
84 // outgoing integrity generation
85 tlul_pkg::tl_d2h_t tl_o_pre;
86 tlul_rsp_intg_gen #(
87   .EnableRspIntgGen(1),
88   .EnableDataIntgGen(1)
89 ) u_rsp_intg_gen (
90   .tl_i(tl_o_pre),
91   .tl_o(tl_o)
92 );
93
94 tlul_pkg::tl_h2d_t tl_socket_h2d [2];
95 tlul_pkg::tl_d2h_t tl_socket_d2h [2];
96
97 logic [0:0] reg_steer;
98
99 // socket_1n connection
100 assign tl_reg_h2d = tl_socket_h2d[1];
101 assign tl_socket_d2h[1] = tl_reg_d2h;
102
103 assign tl_win_o = tl_socket_h2d[0];
104 assign tl_socket_d2h[0] = tl_win_i;
105
106 // Create Socket_1n
107 tlul_socket_1n #(
108   .N (2),
109   .HReqPass (1'b1),
110   .HReqPass (1'b1),

```



```

111 .DReqPass    ({2{1'b1}}),
112 .DRspPass    ({2{1'b1}}),
113 .HReqDepth    (4'h0),
114 .HRspDepth    (4'h0),
115 .DReqDepth    ({2{4'h0}}),
116 .DRspDepth    ({2{4'h0}}),
117 .ExplicitErrs (1'b0)
118 ) u_socket (
119   .clk_i (clk_i),
120   .rst_ni (rst_ni),
121   .tl_h_i (tl_i),
122   .tl_h_o (tl_o_pre),
123   .tl_d_o (tl_socket_h2d),
124   .tl_d_i (tl_socket_d2h),
125   .dev_select_i (reg_steer)
126 );
127
128 // Create steering logic
129 always_comb begin
130   reg_steer =
131     tl_i.a_address[AW-1:0] inside {{4096:8191}} ? 1'd0 :
132     // Default set to register
133     1'd1;
134
135   // Override this in case of an integrity error
136   if (intg_err) begin
137     reg_steer = 1'd1;
138   end
139 end
140
141 tlul_adapter_reg #(
142   .RegAw(AW),
143   .RegDw(DW),
144   .EnableDataIntgGen(0)
145 ) u_reg_if (
146   .clk_i (clk_i),
147   .rst_ni (rst_ni),
148
149   .tl_i (tl_reg_h2d),
150   .tl_o (tl_reg_d2h),
151
152   .en_ifetch_i(prim_mubi_pkg::MuBi4False),
153   .intg_error_o(),
154
155   .we_o (reg_we),
156   .re_o (reg_re),
157   .addr_o (reg_addr),
158   .wdata_o (reg_wdata),
159   .be_o (reg_be),
160   .busy_i (reg_busy),
161   .rdata_i (reg_rdata),
162   .error_i (reg_error)
163 );
164
165 // cdc oversampling signals
166
167 assign reg_rdata = reg_rdata_next ;
168 assign reg_error = addrmiss | wr_err | intg_err;
169
170 // Define SW related signals
171 // Format: <reg>_<field>_{wd|we|qs}
172 // or <reg>_{wd|we|qs} if field == 1 or 0
173 logic intr_state_we;
174 logic intr_state_hmac_done_qs;
175 logic intr_state_hmac_done_wd;
176 logic intr_state_fifo_empty_qs;
177 logic intr_state_hmac_err_qs;
178 logic intr_state_hmac_err_wd;
179 logic intr_enable_we;
180 logic intr_enable_hmac_done_qs;
181 logic intr_enable_hmac_done_wd;
182 logic intr_enable_fifo_empty_qs;
183 logic intr_enable_fifo_empty_wd;
184 logic intr_enable_hmac_err_qs;
185 logic intr_enable_hmac_err_wd;
186 logic intr_test_we;
187 logic intr_test_hmac_done_wd;
188 logic intr_test_fifo_empty_wd;
189 logic intr_test_hmac_err_wd;
190 logic alert_test_we;
191 logic alert_test_wd;
192 logic cfg_re;
193 logic cfg_we;
194 logic cfg_hmac_en_qs;
195 logic cfg_hmac_en_wd;
196 logic cfg_sha_en_qs;
197 logic cfg_sha_en_wd;
198 logic cfg_endian_swap_qs;
199 logic cfg_endian_swap_wd;
200 logic cfg_digest_swap_qs;
201 logic cfg_digest_swap_wd;
202 logic cfg_key_swap_qs;
203 logic cfg_key_swap_wd;
204 logic [3:0] cfg_digest_size_qs;
205 logic [3:0] cfg_digest_size_wd;
206 logic [5:0] cfg_key_length_qs;
207 logic [5:0] cfg_key_length_wd;
208 logic cmd_we;
209 logic cmd_hash_start_wd;
210 logic cmd_hash_process_wd;
211 logic cmd_hash_stop_wd;
212 logic cmd_hash_continue_wd;
213 logic status_re;
214 logic status_hmac_idle_qs;
215 logic status_fifo_empty_qs;
216 logic status_fifo_full_qs;
217 logic [5:0] status_fifo_depth_qs;
218 logic [31:0] err_code_qs;
219 logic wipe_secret_we;
220 logic [31:0] wipe_secret_wd;
221 logic key_0_we;
222 logic [31:0] key_0_wd;
223 logic key_1_we;
224 logic [31:0] key_1_wd;
225 logic key_2_we;
226 logic [31:0] key_2_wd;
227 logic key_3_we;
228 logic [31:0] key_3_wd;
229 logic key_4_we;
230 logic [31:0] key_4_wd;
231 logic key_5_we;
232 logic [31:0] key_5_wd;
233 logic key_6_we;
234 logic [31:0] key_6_wd;
235 logic key_7_we;
236 logic [31:0] key_7_wd;
237 logic key_8_we;
238 logic [31:0] key_8_wd;
239 logic key_9_we;
240 logic [31:0] key_9_wd;
241 logic key_10_we;
242 logic [31:0] key_10_wd;
243 logic key_11_we;
244 logic [31:0] key_11_wd;
245 logic key_12_we;
246 logic [31:0] key_12_wd;
247 logic key_13_we;
248 logic [31:0] key_13_wd;
249 logic key_14_we;
250 logic [31:0] key_14_wd;
251 logic key_15_we;
252 logic [31:0] key_15_wd;
253 logic key_16_we;
254 logic [31:0] key_16_wd;
255 logic key_17_we;
256 logic [31:0] key_17_wd;
257 logic key_18_we;
258 logic [31:0] key_18_wd;
259 logic key_19_we;
260 logic [31:0] key_19_wd;
261 logic key_20_we;
262 logic [31:0] key_20_wd;
263 logic key_21_we;
264 logic [31:0] key_21_wd;
265 logic key_22_we;
266 logic [31:0] key_22_wd;
267 logic key_23_we;
268 logic [31:0] key_23_wd;
269 logic key_24_we;
270 logic [31:0] key_24_wd;
271 logic key_25_we;
272 logic [31:0] key_25_wd;
273 logic key_26_we;
274 logic [31:0] key_26_wd;
275 logic key_27_we;
276 logic [31:0] key_27_wd;
277 logic key_28_we;
278 logic [31:0] key_28_wd;
279 logic key_29_we;
280 logic [31:0] key_29_wd;
281 logic key_30_we;
282 logic [31:0] key_30_wd;
283 logic key_31_we;
284 logic [31:0] key_31_wd;
285 logic digest_0_re;
286 logic digest_0_we;
287 logic [31:0] digest_0_qs;
288 logic [31:0] digest_0_wd;
289 logic digest_1_re;
290 logic digest_1_we;
291 logic [31:0] digest_1_qs;
292 logic [31:0] digest_1_wd;
293 logic digest_2_re;
294 logic digest_2_we;
295 logic [31:0] digest_2_qs;
296 logic [31:0] digest_2_wd;
297 logic digest_3_re;
298 logic digest_3_we;
299 logic [31:0] digest_3_qs;
300 logic [31:0] digest_3_wd;
301 logic digest_4_re;
302 logic digest_4_we;
303 logic [31:0] digest_4_qs;
304 logic [31:0] digest_4_wd;
305 logic digest_5_re;
306 logic digest_5_we;
307 logic [31:0] digest_5_qs;
308 logic [31:0] digest_5_wd;
309 logic digest_6_re;
310 logic digest_6_we;
311 logic [31:0] digest_6_qs;
312 logic [31:0] digest_6_wd;

```

```

313 logic digest_7_re;
314 logic digest_7_we;
315 logic [31:0] digest_7_qs;
316 logic [31:0] digest_7_wd;
317 logic digest_8_re;
318 logic digest_8_we;
319 logic [31:0] digest_8_qs;
320 logic [31:0] digest_8_wd;
321 logic digest_9_re;
322 logic digest_9_we;
323 logic [31:0] digest_9_qs;
324 logic [31:0] digest_9_wd;
325 logic digest_10_re;
326 logic digest_10_we;
327 logic [31:0] digest_10_qs;
328 logic [31:0] digest_10_wd;
329 logic digest_11_re;
330 logic digest_11_we;
331 logic [31:0] digest_11_qs;
332 logic [31:0] digest_11_wd;
333 logic digest_12_re;
334 logic digest_12_we;
335 logic [31:0] digest_12_qs;
336 logic [31:0] digest_12_wd;
337 logic digest_13_re;
338 logic digest_13_we;
339 logic [31:0] digest_13_qs;
340 logic [31:0] digest_13_wd;
341 logic digest_14_re;
342 logic digest_14_we;
343 logic [31:0] digest_14_qs;
344 logic [31:0] digest_14_wd;
345 logic digest_15_re;
346 logic digest_15_we;
347 logic [31:0] digest_15_qs;
348 logic [31:0] digest_15_wd;
349 logic msg_length_lower_re;
350 logic msg_length_lower_we;
351 logic [31:0] msg_length_lower_qs;
352 logic [31:0] msg_length_lower_wd;
353 logic msg_length_upper_re;
354 logic msg_length_upper_we;
355 logic [31:0] msg_length_upper_qs;
356 logic [31:0] msg_length_upper_wd;
357
358 // Register instances
359 // R[intr_state]: V(False)
360 // F[hmac_done]: 0:0
361 prim_subreg #(
362     .DW (1),
363     .SwAccess(prim_subreg_pkg::SwAccessW1C),
364     .RESVAL (1'h0),
365     .Mubi (1'b0)
366 ) u_intr_state_hmac_done (
367     .clk_i (clk_i),
368     .rst_ni (rst_ni),
369
370     // from register interface
371     .we (intr_state_we),
372     .wd (intr_state_hmac_done_wd),
373
374     // from internal hardware
375     .de (hw2reg.intr_state.hmac_done.de),
376     .d (hw2reg.intr_state.hmac_done.d),
377
378     // to internal hardware
379     .qe (),
380     .q (reg2hw.intr_state.hmac_done.q),
381     .ds (),
382
383     // to register interface (read)
384     .qs (intr_state_hmac_done_qs)
385 );
386
387 // F[fifo_empty]: 1:1
388 prim_subreg #(
389     .DW (1),
390     .SwAccess(prim_subreg_pkg::SwAccessR0),
391     .RESVAL (1'h0),
392     .Mubi (1'b0)
393 ) u_intr_state_fifo_empty (
394     .clk_i (clk_i),
395     .rst_ni (rst_ni),
396
397     // from register interface
398     .we (1'b0),
399     .wd ('0'),
400
401     // from internal hardware
402     .de (hw2reg.intr_state_fifo_empty.de),
403     .d (hw2reg.intr_state_fifo_empty.d),
404
405     // to internal hardware
406     .qe (),
407     .q (reg2hw.intr_state_fifo_empty.q),
408     .ds (),
409
410     // to register interface (read)
411     .qs (intr_state_fifo_empty_qs)
412 );
413
414 // F[hmac_err]: 2:2
415 prim_subreg #(
416     .DW (1),
417     .SwAccess(prim_subreg_pkg::SwAccessW1C),
418     .RESVAL (1'h0),
419     .Mubi (1'b0)
420 ) u_intr_state_hmac_err (
421     .clk_i (clk_i),
422     .rst_ni (rst_ni),
423
424     // from register interface
425     .we (intr_state_we),
426     .wd (intr_state_hmac_err_wd),
427
428     // from internal hardware
429     .de (hw2reg.intr_state.hmac_err.de),
430     .d (hw2reg.intr_state.hmac_err.d),
431
432     // to internal hardware
433     .qe (),
434     .q (reg2hw.intr_state.hmac_err.q),
435     .ds (),
436
437     // to register interface (read)
438     .qs (intr_state_hmac_err_qs)
439 );
440
441 // R[intr_enable]: V(False)
442 // F[hmac_done]: 0:0
443 prim_subreg #(
444     .DW (1),
445     .SwAccess(prim_subreg_pkg::SwAccessRW),
446     .RESVAL (1'h0),
447     .Mubi (1'b0)
448 ) u_intr_enable_hmac_done (
449     .clk_i (clk_i),
450     .rst_ni (rst_ni),
451
452     // from register interface
453     .we (intr_enable_we),
454     .wd (intr_enable_hmac_done_wd),
455
456     // from internal hardware
457     .de (1'b0),
458     .d ('0'),
459
460     // to internal hardware
461     .qe (),
462     .q (reg2hw.intr_enable.hmac_done.q),
463     .ds (),
464
465     // to register interface (read)
466     .qs (intr_enable_hmac_done_qs)
467 );
468
469 // F[fifo_empty]: 1:1
470 prim_subreg #(
471     .DW (1),
472     .SwAccess(prim_subreg_pkg::SwAccessRW),
473     .RESVAL (1'h0),
474     .Mubi (1'b0)
475 ) u_intr_enable_fifo_empty (
476     .clk_i (clk_i),
477     .rst_ni (rst_ni),
478
479     // from register interface
480     .we (intr_enable_we),
481     .wd (intr_enable_fifo_empty_wd),
482
483     // from internal hardware
484     .de (1'b0),
485     .d ('0'),
486
487     // to internal hardware
488     .qe (),
489     .q (reg2hw.intr_enable_fifo_empty.q),
490     .ds (),
491
492     // to register interface (read)
493     .qs (intr_enable_fifo_empty_qs)
494 );
495
496 // F[hmac_err]: 2:2
497 prim_subreg #(
498     .DW (1),
499     .SwAccess(prim_subreg_pkg::SwAccessRW),
500     .RESVAL (1'h0),
501     .Mubi (1'b0)
502 ) u_intr_enable_hmac_err (
503     .clk_i (clk_i),
504     .rst_ni (rst_ni),
505
506     // from register interface
507     .we (intr_enable_we),
508     .wd (intr_enable_hmac_err_wd),
509
510     // from internal hardware
511     .de (1'b0),
512     .d ('0'),
513
514     // to internal hardware

```

```

515 // to internal hardware
516 .qe      (),
517 .q       (reg2hw.intr_enable.hmac_err.q),
518 .ds      (),
519
520 // to register interface (read)
521 .qs      (intr_enable_hmac_err_qs)
522 );
523
524 // R[intr_test]: V(True)
525 logic intr_test_qe;
526 logic [2:0] intr_test_flds_we;
527 assign intr_test_qe = &intr_test_flds_we;
528 // F[hmac_done]: 0:0
529 prim_subreg_ext #(
530   .DW (1)
531 ) u_intr_test_hmac_done (
532   .re (1'b0),
533   .we (intr_test_we),
534   .wd (intr_test_hmac_done_wd),
535   .d  ('0'),
536   .qre (),
537   .qe (intr_test_flds_we[0]),
538   .q  (reg2hw.intr_test.hmac_done.q),
539   .ds (),
540   .qs ()
541 );
542 assign reg2hw.intr_test.hmac_done.qe = intr_test_qe;
543
544 // F[fifo_empty]: 1:1
545 prim_subreg_ext #(
546   .DW (1)
547 ) u_intr_test_fifo_empty (
548   .re (1'b0),
549   .we (intr_test_we),
550   .wd (intr_test_fifo_empty_wd),
551   .d  ('0'),
552   .qre (),
553   .qe (intr_test_flds_we[1]),
554   .q  (reg2hw.intr_test.fifo_empty.q),
555   .ds (),
556   .qs ()
557 );
558 assign reg2hw.intr_test.fifo_empty.qe = intr_test_qe;
559
560 // F[hmac_err]: 2:2
561 prim_subreg_ext #(
562   .DW (1)
563 ) u_intr_test_hmac_err (
564   .re (1'b0),
565   .we (intr_test_we),
566   .wd (intr_test_hmac_err_wd),
567   .d  ('0'),
568   .qre (),
569   .qe (intr_test_flds_we[2]),
570   .q  (reg2hw.intr_test.hmac_err.q),
571   .ds (),
572   .qs ()
573 );
574 assign reg2hw.intr_test.hmac_err.qe = intr_test_qe;
575
576 // R[alert_test]: V(True)
577 logic alert_test_qe;
578 logic [0:0] alert_test_flds_we;
579 assign alert_test_qe = &alert_test_flds_we;
580 // F[hmac_en]: 0:0
581 prim_subreg_ext #(
582   .DW (1)
583 ) u_alert_test (
584   .re (1'b0),
585   .we (alert_test_we),
586   .wd (alert_test_wd),
587   .d  ('0'),
588   .qre (),
589   .qe (alert_test_flds_we[0]),
590   .q  (reg2hw.alert_test.q),
591   .ds (),
592   .qs ()
593 );
594 assign reg2hw.alert_test.qe = alert_test_qe;
595
596 // R[cfg]: V(True)
597 logic cfg_qe;
598 logic [6:0] cfg_flds_we;
599 assign cfg_qe = &cfg_flds_we;
600 // F[hmac_en]: 0:0
601 prim_subreg_ext #(
602   .DW (1)
603 ) u_cfg_hmac_en (
604   .re (cfg_re),
605   .we (cfg_we),
606   .wd (cfg_hmac_en_wd),
607   .d  (hw2reg.cfg.hmac_en.d),
608   .qre (),
609   .qe (cfg_flds_we[0]),
610   .q  (reg2hw.cfg.hmac_en.q),
611   .ds (),
612   .qs (cfg_hmac_en_qs)
613 );
614
615 assign reg2hw.cfg.hmac_en.qe = cfg_qe;
616
617 // F[sha_en]: 1:1
618 prim_subreg_ext #(
619   .DW (1)
620 ) u_cfg_sha_en (
621   .re (cfg_re),
622   .we (cfg_we),
623   .wd (cfg_sha_en_wd),
624   .d  (hw2reg.cfg.sha_en.d),
625   .qre (),
626   .qe (cfg_flds_we[1]),
627   .q  (reg2hw.cfg.sha_en.q),
628   .ds (),
629   .qs (cfg_sha_en_qs)
630 );
631 assign reg2hw.cfg.sha_en.qe = cfg_qe;
632
633 // F[endian_swap]: 2:2
634 prim_subreg_ext #(
635   .DW (1)
636 ) u_cfg_endian_swap (
637   .re (cfg_re),
638   .we (cfg_we),
639   .wd (cfg_endian_swap_wd),
640   .d  (hw2reg.cfg.endian_swap.d),
641   .qre (),
642   .qe (cfg_flds_we[2]),
643   .q  (reg2hw.cfg.endian_swap.q),
644   .ds (),
645   .qs (cfg_endian_swap_qs)
646 );
647 assign reg2hw.cfg.endian_swap.qe = cfg_qe;
648
649 // F[digest_swap]: 3:3
650 prim_subreg_ext #(
651   .DW (1)
652 ) u_cfg_digest_swap (
653   .re (cfg_re),
654   .we (cfg_we),
655   .wd (cfg_digest_swap_wd),
656   .d  (hw2reg.cfg.digest_swap.d),
657   .qre (),
658   .qe (cfg_flds_we[3]),
659   .q  (reg2hw.cfg.digest_swap.q),
660   .ds (),
661   .qs (cfg_digest_swap_qs)
662 );
663 assign reg2hw.cfg.digest_swap.qe = cfg_qe;
664
665 // F[key_swap]: 4:4
666 prim_subreg_ext #(
667   .DW (1)
668 ) u_cfg_key_swap (
669   .re (cfg_re),
670   .we (cfg_we),
671   .wd (cfg_key_swap_wd),
672   .d  (hw2reg.cfg.key_swap.d),
673   .qre (),
674   .qe (cfg_flds_we[4]),
675   .q  (reg2hw.cfg.key_swap.q),
676   .ds (),
677   .qs (cfg_key_swap_qs)
678 );
679 assign reg2hw.cfg.key_swap.qe = cfg_qe;
680
681 // F[digest_size]: 8:5
682 prim_subreg_ext #(
683   .DW (4)
684 ) u_cfg_digest_size (
685   .re (cfg_re),
686   .we (cfg_we),
687   .wd (cfg_digest_size_wd),
688   .d  (hw2reg.cfg.digest_size.d),
689   .qre (),
690   .qe (cfg_flds_we[5]),
691   .q  (reg2hw.cfg.digest_size.q),
692   .ds (),
693   .qs (cfg_digest_size_qs)
694 );
695 assign reg2hw.cfg.digest_size.qe = cfg_qe;
696
697 // F[key_length]: 14:9
698 prim_subreg_ext #(
699   .DW (6)
700 ) u_cfg_key_length (
701   .re (cfg_re),
702   .we (cfg_we),
703   .wd (cfg_key_length_wd),
704   .d  (hw2reg.cfg.key_length.d),
705   .qre (),
706   .qe (cfg_flds_we[6]),
707   .q  (reg2hw.cfg.key_length.q),
708   .ds (),
709   .qs (cfg_key_length_qs)
710 );
711 assign reg2hw.cfg.key_length.qe = cfg_qe;
712
713 // R[cmd]: V(True)
714 logic cmd_qe;

```

```

717 logic [3:0] cmd_flds_we;
718 assign cmd_qe = &cmd_flds_we;
719 // F[hash_start]: 0:0
720 prim_subreg_ext #(
721   .DW (1)
722 ) u_cmd_hash_start (
723   .re (1'b0),
724   .we (cmd_we),
725   .wd (cmd_hash_start_wd),
726   .d ('0'),
727   .qre (),
728   .qe (cmd_flds_we[0]),
729   .q (reg2hw.cmd.hash_start.q),
730   .ds (),
731   .qs ()
732 );
733 assign reg2hw.cmd.hash_start.qe = cmd_qe;
734
735 // F[hash_process]: 1:1
736 prim_subreg_ext #(
737   .DW (1)
738 ) u_cmd_hash_process (
739   .re (1'b0),
740   .we (cmd_we),
741   .wd (cmd_hash_process_wd),
742   .d ('0'),
743   .qre (),
744   .qe (cmd_flds_we[1]),
745   .q (reg2hw.cmd.hash_process.q),
746   .ds (),
747   .qs ()
748 );
749 assign reg2hw.cmd.hash_process.qe = cmd_qe;
750
751 // F[hash_stop]: 2:2
752 prim_subreg_ext #(
753   .DW (1)
754 ) u_cmd_hash_stop (
755   .re (1'b0),
756   .we (cmd_we),
757   .wd (cmd_hash_stop_wd),
758   .d ('0'),
759   .qre (),
760   .qe (cmd_flds_we[2]),
761   .q (reg2hw.cmd.hash_stop.q),
762   .ds (),
763   .qs ()
764 );
765 assign reg2hw.cmd.hash_stop.qe = cmd_qe;
766
767 // F[hash_continue]: 3:3
768 prim_subreg_ext #(
769   .DW (1)
770 ) u_cmd_hash_continue (
771   .re (1'b0),
772   .we (cmd_we),
773   .wd (cmd_hash_continue_wd),
774   .d ('0'),
775   .qre (),
776   .qe (cmd_flds_we[3]),
777   .q (reg2hw.cmd.hash_continue.q),
778   .ds (),
779   .qs ()
780 );
781 assign reg2hw.cmd.hash_continue.qe = cmd_qe;
782
783
784 // R[status]: V(True)
785 // F[hmac_idle]: 0:0
786 prim_subreg_ext #(
787   .DW (1)
788 ) u_status_hmac_idle (
789   .re (status_re),
790   .we (1'b0),
791   .wd ('0'),
792   .d (hw2reg.status.hmac_idle.d),
793   .qre (),
794   .qe (),
795   .q (),
796   .ds (),
797   .qs (status_hmac_idle_qs)
798 );
799
800 // F[fifo_empty]: 1:1
801 prim_subreg_ext #(
802   .DW (1)
803 ) u_status_fifo_empty (
804   .re (status_re),
805   .we (1'b0),
806   .wd ('0'),
807   .d (hw2reg.status.fifo_empty.d),
808   .qre (),
809   .qe (),
810   .q (),
811   .ds (),
812   .qs (status_fifo_empty_qs)
813 );
814
815 // F[fifo_full]: 2:2
816 prim_subreg_ext #(
817   .DW (1)
818 ) u_status_fifo_full (
819   .re (status_re),
820   .we (1'b0),
821   .wd ('0'),
822   .d (hw2reg.status.fifo_full.d),
823   .qre (),
824   .qe (),
825   .q (),
826   .ds (),
827   .qs (status_fifo_full_qs)
828 );
829
830 // F[fifo_depth]: 9:4
831 prim_subreg_ext #(
832   .DW (6)
833 ) u_status_fifo_depth (
834   .re (status_re),
835   .we (1'b0),
836   .wd ('0'),
837   .d (hw2reg.status.fifo_depth.d),
838   .qre (),
839   .qe (),
840   .q (),
841   .ds (),
842   .qs (status_fifo_depth_qs)
843 );
844
845 // R[err_code]: V(False)
846 prim_subreg_ext #(
847   .DW (32),
848   .SwAccess(prim_subreg_pkg::SwAccessR0),
849   .RESVAL (32'h0),
850   .Mubi (1'b0)
851 ) u_err_code (
852   .clk_i (clk_i),
853   .rst_ni (rst_ni),
854
855   // from register interface
856   .we (1'b0),
857   .wd ('0'),
858
859   // from internal hardware
860   .de (hw2reg.err_code.de),
861   .d (hw2reg.err_code.d),
862
863   // to internal hardware
864   .qe (),
865   .q (),
866   .ds (),
867
868   // to register interface (read)
869   .qs (err_code_qs)
870 );
871
872
873 // R[wipe_secret]: V(True)
874 logic wipe_secret_qe;
875 logic [0:0] wipe_secret_flds_we;
876 assign wipe_secret_qe = &wipe_secret_flds_we;
877 prim_subreg_ext #(
878   .DW (32)
879 ) u_wipe_secret (
880   .re (1'b0),
881   .we (wipe_secret_we),
882   .wd (wipe_secret_wd),
883   .d ('0'),
884   .qre (),
885   .qe (wipe_secret_flds_we[0]),
886   .q (reg2hw.wipe_secret.q),
887   .ds (),
888   .qs ()
889 );
890 assign reg2hw.wipe_secret.qe = wipe_secret_qe;
891
892
893 // Subregister 0 of Multireg key
894 // R[key_0]: V(True)
895 logic key_0_qe;
896 logic [0:0] key_0_flds_we;
897 assign key_0_qe = &key_0_flds_we;
898 prim_subreg_ext #(
899   .DW (32)
900 ) u_key_0 (
901   .re (1'b0),
902   .we (key_0_we),
903   .wd (key_0_wd),
904   .d (hw2reg.key[0].d),
905   .qre (),
906   .qe (key_0_flds_we[0]),
907   .q (reg2hw.key[0].q),
908   .ds (),
909   .qs ()
910 );
911 assign reg2hw.key[0].qe = key_0_qe;
912
913
914 // Subregister 1 of Multireg key
915 // R[key_1]: V(True)
916 logic key_1_qe;
917 logic [0:0] key_1_flds_we;

```

```

919 assign key_1_qe = &key_1_flds_we;
920 prim_subreg_ext #(
921     .DW (32)
922 ) u_key_1 (
923     .re (1'b0),
924     .we (key_1_we),
925     .wd (key_1_wd),
926     .d (hw2reg.key[1].d),
927     .qre (),
928     .qe (key_1_flds_we[0]),
929     .q (reg2hw.key[1].q),
930     .ds (),
931     .qs ()
932 );
933 assign reg2hw.key[1].qe = key_1_qe;
934
935 // Subregister 2 of Multireg key
936 // R[key_2]: V(True)
937 logic key_2_qe;
938 logic [0:0] key_2_flds_we;
939 assign key_2_qe = &key_2_flds_we;
940 prim_subreg_ext #(
941     .DW (32)
942 ) u_key_2 (
943     .re (1'b0),
944     .we (key_2_we),
945     .wd (key_2_wd),
946     .d (hw2reg.key[2].d),
947     .qre (),
948     .qe (key_2_flds_we[0]),
949     .q (reg2hw.key[2].q),
950     .ds (),
951     .qs ()
952 );
953 assign reg2hw.key[2].qe = key_2_qe;
954
955 // Subregister 3 of Multireg key
956 // R[key_3]: V(True)
957 logic key_3_qe;
958 logic [0:0] key_3_flds_we;
959 assign key_3_qe = &key_3_flds_we;
960 prim_subreg_ext #(
961     .DW (32)
962 ) u_key_3 (
963     .re (1'b0),
964     .we (key_3_we),
965     .wd (key_3_wd),
966     .d (hw2reg.key[3].d),
967     .qre (),
968     .qe (key_3_flds_we[0]),
969     .q (reg2hw.key[3].q),
970     .ds (),
971     .qs ()
972 );
973 assign reg2hw.key[3].qe = key_3_qe;
974
975 // Subregister 4 of Multireg key
976 // R[key_4]: V(True)
977 logic key_4_qe;
978 logic [0:0] key_4_flds_we;
979 assign key_4_qe = &key_4_flds_we;
980 prim_subreg_ext #(
981     .DW (32)
982 ) u_key_4 (
983     .re (1'b0),
984     .we (key_4_we),
985     .wd (key_4_wd),
986     .d (hw2reg.key[4].d),
987     .qre (),
988     .qe (key_4_flds_we[0]),
989     .q (reg2hw.key[4].q),
990     .ds (),
991     .qs ()
992 );
993 assign reg2hw.key[4].qe = key_4_qe;
994
995 // Subregister 5 of Multireg key
996 // R[key_5]: V(True)
997 logic key_5_qe;
998 logic [0:0] key_5_flds_we;
999 assign key_5_qe = &key_5_flds_we;
1000 prim_subreg_ext #(
1001     .DW (32)
1002 ) u_key_5 (
1003     .re (1'b0),
1004     .we (key_5_we),
1005     .wd (key_5_wd),
1006     .d (hw2reg.key[5].d),
1007     .qre (),
1008     .qe (key_5_flds_we[0]),
1009     .q (reg2hw.key[5].q),
1010     .ds (),
1011     .qs ()
1012 );
1013 assign reg2hw.key[5].qe = key_5_qe;
1014
1015 // Subregister 6 of Multireg key
1016 // R[key_6]: V(True)
1017 logic key_6_qe;
1018 logic [0:0] key_6_flds_we;
1019 assign key_6_qe = &key_6_flds_we;
1020 prim_subreg_ext #(
1021     .DW (32)
1022 ) u_key_6 (
1023     .re (1'b0),
1024     .we (key_6_we),
1025     .wd (key_6_wd),
1026     .d (hw2reg.key[6].d),
1027     .qre (),
1028     .qe (key_6_flds_we[0]),
1029     .q (reg2hw.key[6].q),
1030     .ds (),
1031     .qs ()
1032 );
1033 assign reg2hw.key[6].qe = key_6_qe;
1034
1035 // Subregister 7 of Multireg key
1036 // R[key_7]: V(True)
1037 logic key_7_qe;
1038 logic [0:0] key_7_flds_we;
1039 assign key_7_qe = &key_7_flds_we;
1040 prim_subreg_ext #(
1041     .DW (32)
1042 ) u_key_7 (
1043     .re (1'b0),
1044     .we (key_7_we),
1045     .wd (key_7_wd),
1046     .d (hw2reg.key[7].d),
1047     .qre (),
1048     .qe (key_7_flds_we[0]),
1049     .q (reg2hw.key[7].q),
1050     .ds (),
1051     .qs ()
1052 );
1053 assign reg2hw.key[7].qe = key_7_qe;
1054
1055 // Subregister 8 of Multireg key
1056 // R[key_8]: V(True)
1057 logic key_8_qe;
1058 logic [0:0] key_8_flds_we;
1059 assign key_8_qe = &key_8_flds_we;
1060 prim_subreg_ext #(
1061     .DW (32)
1062 ) u_key_8 (
1063     .re (1'b0),
1064     .we (key_8_we),
1065     .wd (key_8_wd),
1066     .d (hw2reg.key[8].d),
1067     .qre (),
1068     .qe (key_8_flds_we[0]),
1069     .q (reg2hw.key[8].q),
1070     .ds (),
1071     .qs ()
1072 );
1073 assign reg2hw.key[8].qe = key_8_qe;
1074
1075 // Subregister 9 of Multireg key
1076 // R[key_9]: V(True)
1077 logic key_9_qe;
1078 logic [0:0] key_9_flds_we;
1079 assign key_9_qe = &key_9_flds_we;
1080 prim_subreg_ext #(
1081     .DW (32)
1082 ) u_key_9 (
1083     .re (1'b0),
1084     .we (key_9_we),
1085     .wd (key_9_wd),
1086     .d (hw2reg.key[9].d),
1087     .qre (),
1088     .qe (key_9_flds_we[0]),
1089     .q (reg2hw.key[9].q),
1090     .ds (),
1091     .qs ()
1092 );
1093 assign reg2hw.key[9].qe = key_9_qe;
1094
1095 // Subregister 10 of Multireg key
1096 // R[key_10]: V(True)
1097 logic key_10_qe;
1098 logic [0:0] key_10_flds_we;
1099 assign key_10_qe = &key_10_flds_we;
1100 prim_subreg_ext #(
1101     .DW (32)
1102 ) u_key_10 (
1103     .re (1'b0),
1104     .we (key_10_we),
1105     .wd (key_10_wd),
1106     .d (hw2reg.key[10].d),
1107     .qre (),
1108     .qe (key_10_flds_we[0]),
1109     .q (reg2hw.key[10].q),
1110     .ds (),
1111     .qs ()
1112 );

```

```

1121 );
1122 assign reg2hw.key[10].qe = key_10_qe;
1123
1124
1125 // Subregister 11 of Multireg key
1126 // R[key_11]: V(True)
1127 logic key_11_qe;
1128 logic [0:0] key_11_flds_we;
1129 assign key_11_qe = &key_11_flds_we;
1130 prim_subreg_ext #(
1131     .DW (32)
1132 ) u_key_11 (
1133     .re (1'b0),
1134     .we (key_11_we),
1135     .wd (key_11_wd),
1136     .d (hw2reg.key[11].d),
1137     .qre (),
1138     .qe (key_11_flds_we[0]),
1139     .q (reg2hw.key[11].q),
1140     .ds (),
1141     .qs ()
1142 );
1143 assign reg2hw.key[11].qe = key_11_qe;
1144
1145
1146 // Subregister 12 of Multireg key
1147 // R[key_12]: V(True)
1148 logic key_12_qe;
1149 logic [0:0] key_12_flds_we;
1150 assign key_12_qe = &key_12_flds_we;
1151 prim_subreg_ext #(
1152     .DW (32)
1153 ) u_key_12 (
1154     .re (1'b0),
1155     .we (key_12_we),
1156     .wd (key_12_wd),
1157     .d (hw2reg.key[12].d),
1158     .qre (),
1159     .qe (key_12_flds_we[0]),
1160     .q (reg2hw.key[12].q),
1161     .ds (),
1162     .qs ()
1163 );
1164 assign reg2hw.key[12].qe = key_12_qe;
1165
1166
1167 // Subregister 13 of Multireg key
1168 // R[key_13]: V(True)
1169 logic key_13_qe;
1170 logic [0:0] key_13_flds_we;
1171 assign key_13_qe = &key_13_flds_we;
1172 prim_subreg_ext #(
1173     .DW (32)
1174 ) u_key_13 (
1175     .re (1'b0),
1176     .we (key_13_we),
1177     .wd (key_13_wd),
1178     .d (hw2reg.key[13].d),
1179     .qre (),
1180     .qe (key_13_flds_we[0]),
1181     .q (reg2hw.key[13].q),
1182     .ds (),
1183     .qs ()
1184 );
1185 assign reg2hw.key[13].qe = key_13_qe;
1186
1187
1188 // Subregister 14 of Multireg key
1189 // R[key_14]: V(True)
1190 logic key_14_qe;
1191 logic [0:0] key_14_flds_we;
1192 assign key_14_qe = &key_14_flds_we;
1193 prim_subreg_ext #(
1194     .DW (32)
1195 ) u_key_14 (
1196     .re (1'b0),
1197     .we (key_14_we),
1198     .wd (key_14_wd),
1199     .d (hw2reg.key[14].d),
1200     .qre (),
1201     .qe (key_14_flds_we[0]),
1202     .q (reg2hw.key[14].q),
1203     .ds (),
1204     .qs ()
1205 );
1206 assign reg2hw.key[14].qe = key_14_qe;
1207
1208
1209 // Subregister 15 of Multireg key
1210 // R[key_15]: V(True)
1211 logic key_15_qe;
1212 logic [0:0] key_15_flds_we;
1213 assign key_15_qe = &key_15_flds_we;
1214 prim_subreg_ext #(
1215     .DW (32)
1216 ) u_key_15 (
1217     .re (1'b0),
1218     .we (key_15_we),
1219     .wd (key_15_wd),
1220     .d (hw2reg.key[15].d),
1221     .qre (),
1222     .qe (key_15_flds_we[0]),
1223     .q (reg2hw.key[15].q),
1224     .ds (),
1225     .qs ()
1226 );
1227 assign reg2hw.key[15].qe = key_15_qe;
1228
1229
1230 // Subregister 16 of Multireg key
1231 // R[key_16]: V(True)
1232 logic key_16_qe;
1233 logic [0:0] key_16_flds_we;
1234 assign key_16_qe = &key_16_flds_we;
1235 prim_subreg_ext #(
1236     .DW (32)
1237 ) u_key_16 (
1238     .re (1'b0),
1239     .we (key_16_we),
1240     .wd (key_16_wd),
1241     .d (hw2reg.key[16].d),
1242     .qre (),
1243     .qe (key_16_flds_we[0]),
1244     .q (reg2hw.key[16].q),
1245     .ds (),
1246     .qs ()
1247 );
1248 assign reg2hw.key[16].qe = key_16_qe;
1249
1250
1251 // Subregister 17 of Multireg key
1252 // R[key_17]: V(True)
1253 logic key_17_qe;
1254 logic [0:0] key_17_flds_we;
1255 assign key_17_qe = &key_17_flds_we;
1256 prim_subreg_ext #(
1257     .DW (32)
1258 ) u_key_17 (
1259     .re (1'b0),
1260     .we (key_17_we),
1261     .wd (key_17_wd),
1262     .d (hw2reg.key[17].d),
1263     .qre (),
1264     .qe (key_17_flds_we[0]),
1265     .q (reg2hw.key[17].q),
1266     .ds (),
1267     .qs ()
1268 );
1269 assign reg2hw.key[17].qe = key_17_qe;
1270
1271
1272 // Subregister 18 of Multireg key
1273 // R[key_18]: V(True)
1274 logic key_18_qe;
1275 logic [0:0] key_18_flds_we;
1276 assign key_18_qe = &key_18_flds_we;
1277 prim_subreg_ext #(
1278     .DW (32)
1279 ) u_key_18 (
1280     .re (1'b0),
1281     .we (key_18_we),
1282     .wd (key_18_wd),
1283     .d (hw2reg.key[18].d),
1284     .qre (),
1285     .qe (key_18_flds_we[0]),
1286     .q (reg2hw.key[18].q),
1287     .ds (),
1288     .qs ()
1289 );
1290 assign reg2hw.key[18].qe = key_18_qe;
1291
1292
1293 // Subregister 19 of Multireg key
1294 // R[key_19]: V(True)
1295 logic key_19_qe;
1296 logic [0:0] key_19_flds_we;
1297 assign key_19_qe = &key_19_flds_we;
1298 prim_subreg_ext #(
1299     .DW (32)
1300 ) u_key_19 (
1301     .re (1'b0),
1302     .we (key_19_we),
1303     .wd (key_19_wd),
1304     .d (hw2reg.key[19].d),
1305     .qre (),
1306     .qe (key_19_flds_we[0]),
1307     .q (reg2hw.key[19].q),
1308     .ds (),
1309     .qs ()
1310 );
1311 assign reg2hw.key[19].qe = key_19_qe;
1312
1313
1314 // Subregister 20 of Multireg key
1315 // R[key_20]: V(True)
1316 logic key_20_qe;
1317 logic [0:0] key_20_flds_we;
1318 assign key_20_qe = &key_20_flds_we;
1319 prim_subreg_ext #(
1320     .DW (32)
1321 ) u_key_20 (
1322     .re (1'b0),

```



```

1323 .we (key_20_we),
1324 .wd (key_20_wd),
1325 .d (hw2reg.key[20].d),
1326 .qre (),
1327 .qe (key_20_flds_we[0]),
1328 .q (reg2hw.key[20].q),
1329 .ds (),
1330 .qs ();
1331 );
1332 assign reg2hw.key[20].qe = key_20_qe;
1333
1334 // Subregister 21 of Multireg key
1335 // R[key_21]: V(True)
1336 logic key_21_qe;
1337 logic [0:0] key_21_flds_we;
1338 assign key_21_qe = &key_21_flds_we;
1339 prim_subreg_ext #(
1340 .DW (32)
1341 ) u_key_21 (
1342 .re (1'b0),
1343 .we (key_21_we),
1344 .wd (key_21_wd),
1345 .d (hw2reg.key[21].d),
1346 .qre (),
1347 .qe (key_21_flds_we[0]),
1348 .q (reg2hw.key[21].q),
1349 .ds (),
1350 .qs ();
1351 );
1352 assign reg2hw.key[21].qe = key_21_qe;
1353
1354 // Subregister 22 of Multireg key
1355 // R[key_22]: V(True)
1356 logic key_22_qe;
1357 logic [0:0] key_22_flds_we;
1358 assign key_22_qe = &key_22_flds_we;
1359 prim_subreg_ext #(
1360 .DW (32)
1361 ) u_key_22 (
1362 .re (1'b0),
1363 .we (key_22_we),
1364 .wd (key_22_wd),
1365 .d (hw2reg.key[22].d),
1366 .qre (),
1367 .qe (key_22_flds_we[0]),
1368 .q (reg2hw.key[22].q),
1369 .ds (),
1370 .qs ();
1371 );
1372 assign reg2hw.key[22].qe = key_22_qe;
1373
1374 // Subregister 23 of Multireg key
1375 // R[key_23]: V(True)
1376 logic key_23_qe;
1377 logic [0:0] key_23_flds_we;
1378 assign key_23_qe = &key_23_flds_we;
1379 prim_subreg_ext #(
1380 .DW (32)
1381 ) u_key_23 (
1382 .re (1'b0),
1383 .we (key_23_we),
1384 .wd (key_23_wd),
1385 .d (hw2reg.key[23].d),
1386 .qre (),
1387 .qe (key_23_flds_we[0]),
1388 .q (reg2hw.key[23].q),
1389 .ds (),
1390 .qs ();
1391 );
1392 assign reg2hw.key[23].qe = key_23_qe;
1393
1394 // Subregister 24 of Multireg key
1395 // R[key_24]: V(True)
1396 logic key_24_qe;
1397 logic [0:0] key_24_flds_we;
1398 assign key_24_qe = &key_24_flds_we;
1399 prim_subreg_ext #(
1400 .DW (32)
1401 ) u_key_24 (
1402 .re (1'b0),
1403 .we (key_24_we),
1404 .wd (key_24_wd),
1405 .d (hw2reg.key[24].d),
1406 .qre (),
1407 .qe (key_24_flds_we[0]),
1408 .q (reg2hw.key[24].q),
1409 .ds (),
1410 .qs ();
1411 );
1412 assign reg2hw.key[24].qe = key_24_qe;
1413
1414 // Subregister 25 of Multireg key
1415 // R[key_25]: V(True)
1416 logic key_25_qe;
1417 logic [0:0] key_25_flds_we;
1418 assign key_25_qe = &key_25_flds_we;
1419
1420 prim_subreg_ext #(
1421 .DW (32)
1422 ) u_key_25 (
1423 .re (1'b0),
1424 .we (key_25_we),
1425 .wd (key_25_wd),
1426 .d (hw2reg.key[25].d),
1427 .qre (),
1428 .qe (key_25_flds_we[0]),
1429 .q (reg2hw.key[25].q),
1430 .ds (),
1431 .qs ();
1432 );
1433 assign reg2hw.key[25].qe = key_25_qe;
1434
1435 // Subregister 26 of Multireg key
1436 // R[key_26]: V(True)
1437 logic key_26_qe;
1438 logic [0:0] key_26_flds_we;
1439 assign key_26_qe = &key_26_flds_we;
1440 prim_subreg_ext #(
1441 .DW (32)
1442 ) u_key_26 (
1443 .re (1'b0),
1444 .we (key_26_we),
1445 .wd (key_26_wd),
1446 .d (hw2reg.key[26].d),
1447 .qre (),
1448 .qe (key_26_flds_we[0]),
1449 .q (reg2hw.key[26].q),
1450 .ds (),
1451 .qs ();
1452 );
1453 assign reg2hw.key[26].qe = key_26_qe;
1454
1455 // Subregister 27 of Multireg key
1456 // R[key_27]: V(True)
1457 logic key_27_qe;
1458 logic [0:0] key_27_flds_we;
1459 assign key_27_qe = &key_27_flds_we;
1460 prim_subreg_ext #(
1461 .DW (32)
1462 ) u_key_27 (
1463 .re (1'b0),
1464 .we (key_27_we),
1465 .wd (key_27_wd),
1466 .d (hw2reg.key[27].d),
1467 .qre (),
1468 .qe (key_27_flds_we[0]),
1469 .q (reg2hw.key[27].q),
1470 .ds (),
1471 .qs ();
1472 );
1473 assign reg2hw.key[27].qe = key_27_qe;
1474
1475 // Subregister 28 of Multireg key
1476 // R[key_28]: V(True)
1477 logic key_28_qe;
1478 logic [0:0] key_28_flds_we;
1479 assign key_28_qe = &key_28_flds_we;
1480 prim_subreg_ext #(
1481 .DW (32)
1482 ) u_key_28 (
1483 .re (1'b0),
1484 .we (key_28_we),
1485 .wd (key_28_wd),
1486 .d (hw2reg.key[28].d),
1487 .qre (),
1488 .qe (key_28_flds_we[0]),
1489 .q (reg2hw.key[28].q),
1490 .ds (),
1491 .qs ();
1492 );
1493 assign reg2hw.key[28].qe = key_28_qe;
1494
1495 // Subregister 29 of Multireg key
1496 // R[key_29]: V(True)
1497 logic key_29_qe;
1498 logic [0:0] key_29_flds_we;
1499 assign key_29_qe = &key_29_flds_we;
1500 prim_subreg_ext #(
1501 .DW (32)
1502 ) u_key_29 (
1503 .re (1'b0),
1504 .we (key_29_we),
1505 .wd (key_29_wd),
1506 .d (hw2reg.key[29].d),
1507 .qre (),
1508 .qe (key_29_flds_we[0]),
1509 .q (reg2hw.key[29].q),
1510 .ds (),
1511 .qs ();
1512 );
1513 assign reg2hw.key[29].qe = key_29_qe;
1514
1515 // Subregister 30 of Multireg key

```

```

1525 // R[key_30]: V(True)
1526 logic key_30_qe;
1527 logic [0:0] key_30_flds_we;
1528 assign key_30_qe = &key_30_flds_we;
1529 prim_subreg_ext #(
1530     .DW (32)
1531 ) u_key_30 (
1532     .re (1'b0),
1533     .we (key_30_we),
1534     .wd (key_30_wd),
1535     .d (hw2reg.key[30].d),
1536     .qre (),
1537     .qe (key_30_flds_we[0]),
1538     .q (reg2hw.key[30].q),
1539     .ds (),
1540     .qs ()
1541 );
1542 assign reg2hw.key[30].qe = key_30_qe;
1543
1544 // Subregister 31 of Multireg key
1545 // R[key_31]: V(True)
1546 logic key_31_qe;
1547 logic [0:0] key_31_flds_we;
1548 assign key_31_qe = &key_31_flds_we;
1549 prim_subreg_ext #(
1550     .DW (32)
1551 ) u_key_31 (
1552     .re (1'b0),
1553     .we (key_31_we),
1554     .wd (key_31_wd),
1555     .d (hw2reg.key[31].d),
1556     .qre (),
1557     .qe (key_31_flds_we[0]),
1558     .q (reg2hw.key[31].q),
1559     .ds (),
1560     .qs ()
1561 );
1562 assign reg2hw.key[31].qe = key_31_qe;
1563
1564 // Subregister 0 of Multireg digest
1565 // R[digest_0]: V(True)
1566 logic digest_0_qe;
1567 logic [0:0] digest_0_flds_we;
1568 assign digest_0_qe = &digest_0_flds_we;
1569 prim_subreg_ext #(
1570     .DW (32)
1571 ) u_digest_0 (
1572     .re (digest_0_re),
1573     .we (digest_0_we),
1574     .wd (digest_0_wd),
1575     .d (hw2reg.digest[0].d),
1576     .qre (),
1577     .qe (digest_0_flds_we[0]),
1578     .q (reg2hw.digest[0].q),
1579     .ds (),
1580     .qs (digest_0_qs)
1581 );
1582 assign reg2hw.digest[0].qe = digest_0_qe;
1583
1584 // Subregister 1 of Multireg digest
1585 // R[digest_1]: V(True)
1586 logic digest_1_qe;
1587 logic [0:0] digest_1_flds_we;
1588 assign digest_1_qe = &digest_1_flds_we;
1589 prim_subreg_ext #(
1590     .DW (32)
1591 ) u_digest_1 (
1592     .re (digest_1_re),
1593     .we (digest_1_we),
1594     .wd (digest_1_wd),
1595     .d (hw2reg.digest[1].d),
1596     .qre (),
1597     .qe (digest_1_flds_we[0]),
1598     .q (reg2hw.digest[1].q),
1599     .ds (),
1600     .qs (digest_1_qs)
1601 );
1602 assign reg2hw.digest[1].qe = digest_1_qe;
1603
1604 // Subregister 2 of Multireg digest
1605 // R[digest_2]: V(True)
1606 logic digest_2_qe;
1607 logic [0:0] digest_2_flds_we;
1608 assign digest_2_qe = &digest_2_flds_we;
1609 prim_subreg_ext #(
1610     .DW (32)
1611 ) u_digest_2 (
1612     .re (digest_2_re),
1613     .we (digest_2_we),
1614     .wd (digest_2_wd),
1615     .d (hw2reg.digest[2].d),
1616     .qre (),
1617     .qe (digest_2_flds_we[0]),
1618     .q (reg2hw.digest[2].q),
1619     .ds (),
1620     .qs (digest_2_qs)
1621 );
1622
1623 // Subregister 3 of Multireg digest
1624 // R[digest_3]: V(True)
1625 logic digest_3_qe;
1626 logic [0:0] digest_3_flds_we;
1627 assign digest_3_qe = &digest_3_flds_we;
1628 prim_subreg_ext #(
1629     .DW (32)
1630 ) u_digest_3 (
1631     .re (digest_3_re),
1632     .we (digest_3_we),
1633     .wd (digest_3_wd),
1634     .d (hw2reg.digest[3].d),
1635     .qre (),
1636     .qe (digest_3_flds_we[0]),
1637     .q (reg2hw.digest[3].q),
1638     .ds (),
1639     .qs (digest_3_qs)
1640 );
1641 assign reg2hw.digest[3].qe = digest_3_qe;
1642
1643 // Subregister 4 of Multireg digest
1644 // R[digest_4]: V(True)
1645 logic digest_4_qe;
1646 logic [0:0] digest_4_flds_we;
1647 assign digest_4_qe = &digest_4_flds_we;
1648 prim_subreg_ext #(
1649     .DW (32)
1650 ) u_digest_4 (
1651     .re (digest_4_re),
1652     .we (digest_4_we),
1653     .wd (digest_4_wd),
1654     .d (hw2reg.digest[4].d),
1655     .qre (),
1656     .qe (digest_4_flds_we[0]),
1657     .q (reg2hw.digest[4].q),
1658     .ds (),
1659     .qs (digest_4_qs)
1660 );
1661 assign reg2hw.digest[4].qe = digest_4_qe;
1662
1663 // Subregister 5 of Multireg digest
1664 // R[digest_5]: V(True)
1665 logic digest_5_qe;
1666 logic [0:0] digest_5_flds_we;
1667 assign digest_5_qe = &digest_5_flds_we;
1668 prim_subreg_ext #(
1669     .DW (32)
1670 ) u_digest_5 (
1671     .re (digest_5_re),
1672     .we (digest_5_we),
1673     .wd (digest_5_wd),
1674     .d (hw2reg.digest[5].d),
1675     .qre (),
1676     .qe (digest_5_flds_we[0]),
1677     .q (reg2hw.digest[5].q),
1678     .ds (),
1679     .qs (digest_5_qs)
1680 );
1681 assign reg2hw.digest[5].qe = digest_5_qe;
1682
1683 // Subregister 6 of Multireg digest
1684 // R[digest_6]: V(True)
1685 logic digest_6_qe;
1686 logic [0:0] digest_6_flds_we;
1687 assign digest_6_qe = &digest_6_flds_we;
1688 prim_subreg_ext #(
1689     .DW (32)
1690 ) u_digest_6 (
1691     .re (digest_6_re),
1692     .we (digest_6_we),
1693     .wd (digest_6_wd),
1694     .d (hw2reg.digest[6].d),
1695     .qre (),
1696     .qe (digest_6_flds_we[0]),
1697     .q (reg2hw.digest[6].q),
1698     .ds (),
1699     .qs (digest_6_qs)
1700 );
1701 assign reg2hw.digest[6].qe = digest_6_qe;
1702
1703 // Subregister 7 of Multireg digest
1704 // R[digest_7]: V(True)
1705 logic digest_7_qe;
1706 logic [0:0] digest_7_flds_we;
1707 assign digest_7_qe = &digest_7_flds_we;
1708 prim_subreg_ext #(
1709     .DW (32)
1710 ) u_digest_7 (
1711     .re (digest_7_re),
1712     .we (digest_7_we),
1713     .wd (digest_7_wd),
1714     .d (hw2reg.digest[7].d),
1715     .qre (),
1716     .qe (digest_7_flds_we[0]),
1717     .ds (),
1718     .qs ()
1719 );

```



```

1727 .q      (reg2hw.digest[7].q),
1728 .ds      (),
1729 .qs      (digest_7_qs)
1730 );
1731 assign reg2hw.digest[7].qe = digest_7_qe;
1732
1733 // Subregister 8 of Multireg digest
1734 // R[digest_8]: V(True)
1735 logic digest_8_qe;
1736 logic [0:0] digest_8_flds_we;
1737 assign digest_8_qe = &digest_8_flds_we;
1738 prim_subreg_ext #(
1739   .DW      (32)
1740 ) u_digest_8 (
1741   .re      (digest_8_re),
1742   .we      (digest_8_we),
1743   .wd      (digest_8_wd),
1744   .d      (hw2reg.digest[8].d),
1745   .qre     (),
1746   .qe      (digest_8_flds_we[0]),
1747   .q      (reg2hw.digest[8].q),
1748   .ds      (),
1749   .qs      (digest_8_qs)
1750 );
1751 assign reg2hw.digest[8].qe = digest_8_qe;
1752
1753 // Subregister 9 of Multireg digest
1754 // R[digest_9]: V(True)
1755 logic digest_9_qe;
1756 logic [0:0] digest_9_flds_we;
1757 assign digest_9_qe = &digest_9_flds_we;
1758 prim_subreg_ext #(
1759   .DW      (32)
1760 ) u_digest_9 (
1761   .re      (digest_9_re),
1762   .we      (digest_9_we),
1763   .wd      (digest_9_wd),
1764   .d      (hw2reg.digest[9].d),
1765   .qre     (),
1766   .qe      (digest_9_flds_we[0]),
1767   .q      (reg2hw.digest[9].q),
1768   .ds      (),
1769   .qs      (digest_9_qs)
1770 );
1771 assign reg2hw.digest[9].qe = digest_9_qe;
1772
1773 // Subregister 10 of Multireg digest
1774 // R[digest_10]: V(True)
1775 logic digest_10_qe;
1776 logic [0:0] digest_10_flds_we;
1777 assign digest_10_qe = &digest_10_flds_we;
1778 prim_subreg_ext #(
1779   .DW      (32)
1780 ) u_digest_10 (
1781   .re      (digest_10_re),
1782   .we      (digest_10_we),
1783   .wd      (digest_10_wd),
1784   .d      (hw2reg.digest[10].d),
1785   .qre     (),
1786   .qe      (digest_10_flds_we[0]),
1787   .q      (reg2hw.digest[10].q),
1788   .ds      (),
1789   .qs      (digest_10_qs)
1790 );
1791 assign reg2hw.digest[10].qe = digest_10_qe;
1792
1793 // Subregister 11 of Multireg digest
1794 // R[digest_11]: V(True)
1795 logic digest_11_qe;
1796 logic [0:0] digest_11_flds_we;
1797 assign digest_11_qe = &digest_11_flds_we;
1798 prim_subreg_ext #(
1799   .DW      (32)
1800 ) u_digest_11 (
1801   .re      (digest_11_re),
1802   .we      (digest_11_we),
1803   .wd      (digest_11_wd),
1804   .d      (hw2reg.digest[11].d),
1805   .qre     (),
1806   .qe      (digest_11_flds_we[0]),
1807   .q      (reg2hw.digest[11].q),
1808   .ds      (),
1809   .qs      (digest_11_qs)
1810 );
1811 assign reg2hw.digest[11].qe = digest_11_qe;
1812
1813 // Subregister 12 of Multireg digest
1814 // R[digest_12]: V(True)
1815 logic digest_12_qe;
1816 logic [0:0] digest_12_flds_we;
1817 assign digest_12_qe = &digest_12_flds_we;
1818 prim_subreg_ext #(
1819   .DW      (32)
1820 ) u_digest_12 (
1821   .re      (digest_12_re),
1822   .we      (digest_12_we),
1823   .wd      (digest_12_wd),
1824   .d      (hw2reg.digest[12].d),
1825   .qre     (),
1826   .qe      (digest_12_flds_we[0]),
1827   .q      (reg2hw.digest[12].q),
1828   .ds      (),
1829   .qs      (digest_12_qs)
1830 );
1831 assign reg2hw.digest[12].qe = digest_12_qe;
1832
1833 // Subregister 13 of Multireg digest
1834 // R[digest_13]: V(True)
1835 logic digest_13_qe;
1836 logic [0:0] digest_13_flds_we;
1837 assign digest_13_qe = &digest_13_flds_we;
1838 prim_subreg_ext #(
1839   .DW      (32)
1840 ) u_digest_13 (
1841   .re      (digest_13_re),
1842   .we      (digest_13_we),
1843   .wd      (digest_13_wd),
1844   .d      (hw2reg.digest[13].d),
1845   .qre     (),
1846   .qe      (digest_13_flds_we[0]),
1847   .q      (reg2hw.digest[13].q),
1848   .ds      (),
1849   .qs      (digest_13_qs)
1850 );
1851 assign reg2hw.digest[13].qe = digest_13_qe;
1852
1853 // Subregister 14 of Multireg digest
1854 // R[digest_14]: V(True)
1855 logic digest_14_qe;
1856 logic [0:0] digest_14_flds_we;
1857 assign digest_14_qe = &digest_14_flds_we;
1858 prim_subreg_ext #(
1859   .DW      (32)
1860 ) u_digest_14 (
1861   .re      (digest_14_re),
1862   .we      (digest_14_we),
1863   .wd      (digest_14_wd),
1864   .d      (hw2reg.digest[14].d),
1865   .qre     (),
1866   .qe      (digest_14_flds_we[0]),
1867   .q      (reg2hw.digest[14].q),
1868   .ds      (),
1869   .qs      (digest_14_qs)
1870 );
1871 assign reg2hw.digest[14].qe = digest_14_qe;
1872
1873 // Subregister 15 of Multireg digest
1874 // R[digest_15]: V(True)
1875 logic digest_15_qe;
1876 logic [0:0] digest_15_flds_we;
1877 assign digest_15_qe = &digest_15_flds_we;
1878 prim_subreg_ext #(
1879   .DW      (32)
1880 ) u_digest_15 (
1881   .re      (digest_15_re),
1882   .we      (digest_15_we),
1883   .wd      (digest_15_wd),
1884   .d      (hw2reg.digest[15].d),
1885   .qre     (),
1886   .qe      (digest_15_flds_we[0]),
1887   .q      (reg2hw.digest[15].q),
1888   .ds      (),
1889   .qs      (digest_15_qs)
1890 );
1891 assign reg2hw.digest[15].qe = digest_15_qe;
1892
1893 // R[msg_length_lower]: V(True)
1894 logic msg_length_lower_qe;
1895 logic [0:0] msg_length_lower_flds_we;
1896 assign msg_length_lower_qe = &msg_length_lower_flds_we;
1897 prim_subreg_ext #(
1898   .DW      (32)
1899 ) u_msg_length_lower (
1900   .re      (msg_length_lower_re),
1901   .we      (msg_length_lower_we),
1902   .wd      (msg_length_lower_wd),
1903   .d      (hw2reg.msg_length_lower.d),
1904   .qre     (),
1905   .qe      (msg_length_lower_flds_we[0]),
1906   .q      (reg2hw.msg_length_lower.q),
1907   .ds      (),
1908   .qs      (msg_length_lower_qs)
1909 );
1910 assign reg2hw.msg_length_lower.qe = msg_length_lower_qe;
1911
1912 // R[msg_length_upper]: V(True)
1913 logic msg_length_upper_qe;
1914 logic [0:0] msg_length_upper_flds_we;
1915 assign msg_length_upper_qe = &msg_length_upper_flds_we;
1916 prim_subreg_ext #(
1917   .DW      (32)
1918 ) u_msg_length_upper (

```

```

1929 .re (msg_length_upper_re),
1930 .we (msg_length_upper_we),
1931 .wd (msg_length_upper_wd),
1932 .d (hw2reg.msg_length_upper.d),
1933 .qre (),
1934 .qe (msg_length_upper_flds_we[0]),
1935 .q (reg2hw.msg_length_upper.q),
1936 .ds (),
1937 .qs (msg_length_upper_qs)
1938 );
1939 assign reg2hw.msg_length_upper.qe = msg_length_upper_qe;
1940
1941
1942
1943 logic [58:0] addr_hit;
1944 always_comb begin
1945   addr_hit[0] = (reg_addr == HMAC_INTR_STATE_OFFSET);
1946   addr_hit[1] = (reg_addr == HMAC_INTR_ENABLE_OFFSET);
1947   addr_hit[2] = (reg_addr == HMAC_INTR_TEST_OFFSET);
1948   addr_hit[3] = (reg_addr == HMAC_ALERT_TEST_OFFSET);
1949   addr_hit[4] = (reg_addr == HMAC_CFG_OFFSET);
1950   addr_hit[5] = (reg_addr == HMAC_CMD_OFFSET);
1951   addr_hit[6] = (reg_addr == HMAC_STATUS_OFFSET);
1952   addr_hit[7] = (reg_addr == HMAC_ERR_CODE_OFFSET);
1953   addr_hit[8] = (reg_addr == HMAC_WIPE_SECRET_OFFSET);
1954   addr_hit[9] = (reg_addr == HMAC_KEY_0_OFFSET);
1955   addr_hit[10] = (reg_addr == HMAC_KEY_1_OFFSET);
1956   addr_hit[11] = (reg_addr == HMAC_KEY_2_OFFSET);
1957   addr_hit[12] = (reg_addr == HMAC_KEY_3_OFFSET);
1958   addr_hit[13] = (reg_addr == HMAC_KEY_4_OFFSET);
1959   addr_hit[14] = (reg_addr == HMAC_KEY_5_OFFSET);
1960   addr_hit[15] = (reg_addr == HMAC_KEY_6_OFFSET);
1961   addr_hit[16] = (reg_addr == HMAC_KEY_7_OFFSET);
1962   addr_hit[17] = (reg_addr == HMAC_KEY_8_OFFSET);
1963   addr_hit[18] = (reg_addr == HMAC_KEY_9_OFFSET);
1964   addr_hit[19] = (reg_addr == HMAC_KEY_10_OFFSET);
1965   addr_hit[20] = (reg_addr == HMAC_KEY_11_OFFSET);
1966   addr_hit[21] = (reg_addr == HMAC_KEY_12_OFFSET);
1967   addr_hit[22] = (reg_addr == HMAC_KEY_13_OFFSET);
1968   addr_hit[23] = (reg_addr == HMAC_KEY_14_OFFSET);
1969   addr_hit[24] = (reg_addr == HMAC_KEY_15_OFFSET);
1970   addr_hit[25] = (reg_addr == HMAC_KEY_16_OFFSET);
1971   addr_hit[26] = (reg_addr == HMAC_KEY_17_OFFSET);
1972   addr_hit[27] = (reg_addr == HMAC_KEY_18_OFFSET);
1973   addr_hit[28] = (reg_addr == HMAC_KEY_19_OFFSET);
1974   addr_hit[29] = (reg_addr == HMAC_KEY_20_OFFSET);
1975   addr_hit[30] = (reg_addr == HMAC_KEY_21_OFFSET);
1976   addr_hit[31] = (reg_addr == HMAC_KEY_22_OFFSET);
1977   addr_hit[32] = (reg_addr == HMAC_KEY_23_OFFSET);
1978   addr_hit[33] = (reg_addr == HMAC_KEY_24_OFFSET);
1979   addr_hit[34] = (reg_addr == HMAC_KEY_25_OFFSET);
1980   addr_hit[35] = (reg_addr == HMAC_KEY_26_OFFSET);
1981   addr_hit[36] = (reg_addr == HMAC_KEY_27_OFFSET);
1982   addr_hit[37] = (reg_addr == HMAC_KEY_28_OFFSET);
1983   addr_hit[38] = (reg_addr == HMAC_KEY_29_OFFSET);
1984   addr_hit[39] = (reg_addr == HMAC_KEY_30_OFFSET);
1985   addr_hit[40] = (reg_addr == HMAC_KEY_31_OFFSET);
1986   addr_hit[41] = (reg_addr == HMAC_DIGEST_0_OFFSET);
1987   addr_hit[42] = (reg_addr == HMAC_DIGEST_1_OFFSET);
1988   addr_hit[43] = (reg_addr == HMAC_DIGEST_2_OFFSET);
1989   addr_hit[44] = (reg_addr == HMAC_DIGEST_3_OFFSET);
1990   addr_hit[45] = (reg_addr == HMAC_DIGEST_4_OFFSET);
1991   addr_hit[46] = (reg_addr == HMAC_DIGEST_5_OFFSET);
1992   addr_hit[47] = (reg_addr == HMAC_DIGEST_6_OFFSET);
1993   addr_hit[48] = (reg_addr == HMAC_DIGEST_7_OFFSET);
1994   addr_hit[49] = (reg_addr == HMAC_DIGEST_8_OFFSET);
1995   addr_hit[50] = (reg_addr == HMAC_DIGEST_9_OFFSET);
1996   addr_hit[51] = (reg_addr == HMAC_DIGEST_10_OFFSET);
1997   addr_hit[52] = (reg_addr == HMAC_DIGEST_11_OFFSET);
1998   addr_hit[53] = (reg_addr == HMAC_DIGEST_12_OFFSET);
1999   addr_hit[54] = (reg_addr == HMAC_DIGEST_13_OFFSET);
2000   addr_hit[55] = (reg_addr == HMAC_DIGEST_14_OFFSET);
2001   addr_hit[56] = (reg_addr == HMAC_DIGEST_15_OFFSET);
2002   addr_hit[57] = (reg_addr == HMAC_MSG_LENGTH_LOWER_OFFSET);
2003   addr_hit[58] = (reg_addr == HMAC_MSG_LENGTH_UPPER_OFFSET);
2004 end
2005
2006 assign addrmiss = (reg_re || reg_we) ? ~|addr_hit : 1'b0 ;
2007
2008 // Check sub-word write is permitted
2009 always_comb begin
2010   wr_err = (reg_we &
2011
2012       ((addr_hit[0] & !(HMAC_PERMIT[0] & ~reg_be)))
2013
2014       (addr_hit[1] & !(HMAC_PERMIT[1] & ~reg_be)))
2015
2016       (addr_hit[2] & !(HMAC_PERMIT[2] & ~reg_be)))
2017
2018       (addr_hit[3] & !(HMAC_PERMIT[3] & ~reg_be)))
2019
2020       (addr_hit[4] & !(HMAC_PERMIT[4] & ~reg_be)))
2021
2022       (addr_hit[5] & !(HMAC_PERMIT[5] & ~reg_be)))
2023
2024       (addr_hit[6] & !(HMAC_PERMIT[6] & ~reg_be)))
2025
2026       (addr_hit[7] & !(HMAC_PERMIT[7] & ~reg_be)))
2027
2028       (addr_hit[8] & !(HMAC_PERMIT[8] & ~reg_be)))
2029
2030       (addr_hit[9] & !(HMAC_PERMIT[9] & ~reg_be)))
2031
2032       (addr_hit[10] & !(HMAC_PERMIT[10] & ~reg_be)))
2033
2034       (addr_hit[11] & !(HMAC_PERMIT[11] & ~reg_be)))
2035
2036       (addr_hit[12] & !(HMAC_PERMIT[12] & ~reg_be)))
2037
2038       (addr_hit[13] & !(HMAC_PERMIT[13] & ~reg_be)))
2039
2040       (addr_hit[14] & !(HMAC_PERMIT[14] & ~reg_be)))
2041
2042       (addr_hit[15] & !(HMAC_PERMIT[15] & ~reg_be)))
2043
2044       (addr_hit[16] & !(HMAC_PERMIT[16] & ~reg_be)))
2045
2046       (addr_hit[17] & !(HMAC_PERMIT[17] & ~reg_be)))
2047
2048       (addr_hit[18] & !(HMAC_PERMIT[18] & ~reg_be)))
2049
2050       (addr_hit[19] & !(HMAC_PERMIT[19] & ~reg_be)))
2051
2052       (addr_hit[20] & !(HMAC_PERMIT[20] & ~reg_be)))
2053
2054       (addr_hit[21] & !(HMAC_PERMIT[21] & ~reg_be)))
2055
2056       (addr_hit[22] & !(HMAC_PERMIT[22] & ~reg_be)))
2057
2058       (addr_hit[23] & !(HMAC_PERMIT[23] & ~reg_be)))
2059
2060       (addr_hit[24] & !(HMAC_PERMIT[24] & ~reg_be)))
2061
2062       (addr_hit[25] & !(HMAC_PERMIT[25] & ~reg_be)))
2063
2064       (addr_hit[26] & !(HMAC_PERMIT[26] & ~reg_be)))
2065
2066       (addr_hit[27] & !(HMAC_PERMIT[27] & ~reg_be)))
2067
2068       (addr_hit[28] & !(HMAC_PERMIT[28] & ~reg_be)))
2069
2070       (addr_hit[29] & !(HMAC_PERMIT[29] & ~reg_be)))
2071
2072       (addr_hit[30] & !(HMAC_PERMIT[30] & ~reg_be)))
2073
2074       (addr_hit[31] & !(HMAC_PERMIT[31] & ~reg_be)))
2075
2076       (addr_hit[32] & !(HMAC_PERMIT[32] & ~reg_be)))
2077
2078       (addr_hit[33] & !(HMAC_PERMIT[33] & ~reg_be)))
2079
2080       (addr_hit[34] & !(HMAC_PERMIT[34] & ~reg_be)))
2081
2082       (addr_hit[35] & !(HMAC_PERMIT[35] & ~reg_be)))
2083
2084       (addr_hit[36] & !(HMAC_PERMIT[36] & ~reg_be)))
2085
2086       (addr_hit[37] & !(HMAC_PERMIT[37] & ~reg_be)))
2087
2088       (addr_hit[38] & !(HMAC_PERMIT[38] & ~reg_be)))
2089
2090       (addr_hit[39] & !(HMAC_PERMIT[39] & ~reg_be)))
2091
2092       (addr_hit[40] & !(HMAC_PERMIT[40] & ~reg_be)))
2093
2094       (addr_hit[41] & !(HMAC_PERMIT[41] & ~reg_be)))
2095
2096       (addr_hit[42] & !(HMAC_PERMIT[42] & ~reg_be)))
2097
2098       (addr_hit[43] & !(HMAC_PERMIT[43] & ~reg_be)))
2099
2100       (addr_hit[44] & !(HMAC_PERMIT[44] & ~reg_be)))
2101
2102       (addr_hit[45] & !(HMAC_PERMIT[45] & ~reg_be)))
2103
2104       (addr_hit[46] & !(HMAC_PERMIT[46] & ~reg_be)))
2105
2106       (addr_hit[47] & !(HMAC_PERMIT[47] & ~reg_be)))
2107
2108       (addr_hit[48] & !(HMAC_PERMIT[48] & ~reg_be)))
2109
2110       (addr_hit[49] & !(HMAC_PERMIT[49] & ~reg_be)))
2111
2112       (addr_hit[50] & !(HMAC_PERMIT[50] & ~reg_be)))
2113
2114       (addr_hit[51] & !(HMAC_PERMIT[51] & ~reg_be)))
2115
2116       (addr_hit[52] & !(HMAC_PERMIT[52] & ~reg_be)))
2117
2118       (addr_hit[53] & !(HMAC_PERMIT[53] & ~reg_be)))
2119
2120       (addr_hit[54] & !(HMAC_PERMIT[54] & ~reg_be)))
2121
2122       (addr_hit[55] & !(HMAC_PERMIT[55] & ~reg_be)))
2123
2124       (addr_hit[56] & !(HMAC_PERMIT[56] & ~reg_be)))
2125
2126       (addr_hit[57] & !(HMAC_PERMIT[57] & ~reg_be)))
2127
2128       (addr_hit[58] & !(HMAC_PERMIT[58] & ~reg_be)))
2129
2130       1'b1;
2131 end

```

```

2051      (addr_hit[40] & (!(HMAC_PERMIT[40] & ~reg_be)))
2052      |
2053      (addr_hit[41] & (!(HMAC_PERMIT[41] & ~reg_be)))
2054      |
2055      (addr_hit[42] & (!(HMAC_PERMIT[42] & ~reg_be)))
2056      |
2057      (addr_hit[43] & (!(HMAC_PERMIT[43] & ~reg_be)))
2058      |
2059      (addr_hit[44] & (!(HMAC_PERMIT[44] & ~reg_be)))
2060      |
2061      (addr_hit[45] & (!(HMAC_PERMIT[45] & ~reg_be)))
2062      |
2063      (addr_hit[46] & (!(HMAC_PERMIT[46] & ~reg_be)))
2064      |
2065      (addr_hit[47] & (!(HMAC_PERMIT[47] & ~reg_be)))
2066      |
2067      (addr_hit[48] & (!(HMAC_PERMIT[48] & ~reg_be)))
2068      |
2069      (addr_hit[49] & (!(HMAC_PERMIT[49] & ~reg_be)))
2070      |
2071      (addr_hit[50] & (!(HMAC_PERMIT[50] & ~reg_be)))
2072      |
2073      (addr_hit[51] & (!(HMAC_PERMIT[51] & ~reg_be)))
2074      |
2075      (addr_hit[52] & (!(HMAC_PERMIT[52] & ~reg_be)))
2076      |
2077      (addr_hit[53] & (!(HMAC_PERMIT[53] & ~reg_be)))
2078      |
2079      (addr_hit[54] & (!(HMAC_PERMIT[54] & ~reg_be)))
2080      |
2081      (addr_hit[55] & (!(HMAC_PERMIT[55] & ~reg_be)))
2082      |
2083      (addr_hit[56] & (!(HMAC_PERMIT[56] & ~reg_be)))
2084      |
2085      (addr_hit[57] & (!(HMAC_PERMIT[57] & ~reg_be)))
2086      |
2087      (addr_hit[58] & (!(HMAC_PERMIT[58] &
2088      ~reg_be)))));
2089      end
2090
2091      // Generate write-enables
2092      assign intr_state_we = addr_hit[0] & reg_we & !reg_error;
2093
2094      assign intr_state_hmac_done_wd = reg_wdata[0];
2095
2096      assign intr_state_hmac_err_wd = reg_wdata[2];
2097      assign intr_enable_we = addr_hit[1] & reg_we & !reg_error;
2098
2099      assign intr_enable_hmac_done_wd = reg_wdata[0];
2100
2101      assign intr_enable_fifo_empty_wd = reg_wdata[1];
2102
2103      assign intr_enable_hmac_err_wd = reg_wdata[2];
2104      assign intr_test_we = addr_hit[2] & reg_we & !reg_error;
2105
2106      assign intr_test_hmac_done_wd = reg_wdata[0];
2107
2108      assign intr_test_fifo_empty_wd = reg_wdata[1];
2109
2110      assign intr_test_hmac_err_wd = reg_wdata[2];
2111      assign alert_test_we = addr_hit[3] & reg_we & !reg_error;
2112
2113      assign alert_test_wd = reg_wdata[0];
2114      assign cfg_re = addr_hit[4] & reg_re & !reg_error;
2115      assign cfg_we = addr_hit[4] & reg_we & !reg_error;
2116
2117      assign cfg_hmac_en_wd = reg_wdata[0];
2118
2119      assign cfg_sha_en_wd = reg_wdata[1];
2120
2121      assign cfg_endian_swap_wd = reg_wdata[2];
2122
2123      assign cfg_digest_swap_wd = reg_wdata[3];
2124
2125      assign cfg_key_swap_wd = reg_wdata[4];
2126
2127      assign cfg_digest_size_wd = reg_wdata[8:5];
2128
2129      assign cfg_key_length_wd = reg_wdata[14:9];
2130      assign cmd_we = addr_hit[5] & reg_we & !reg_error;
2131
2132      assign cmd_hash_start_wd = reg_wdata[0];

```

```

2115      assign cmd_hash_process_wd = reg_wdata[1];
2116
2117      assign cmd_hash_stop_wd = reg_wdata[2];
2118
2119      assign cmd_hash_continue_wd = reg_wdata[3];
2120      assign status_re = addr_hit[6] & reg_re & !reg_error;
2121      assign wipe_secret_we = addr_hit[8] & reg_we & !reg_error;
2122
2123      assign wipe_secret_wd = reg_wdata[31:0];
2124      assign key_0_we = addr_hit[9] & reg_we & !reg_error;
2125
2126      assign key_0_wd = reg_wdata[31:0];
2127      assign key_1_we = addr_hit[10] & reg_we & !reg_error;
2128
2129      assign key_1_wd = reg_wdata[31:0];
2130      assign key_2_we = addr_hit[11] & reg_we & !reg_error;
2131
2132      assign key_2_wd = reg_wdata[31:0];
2133      assign key_3_we = addr_hit[12] & reg_we & !reg_error;
2134
2135      assign key_3_wd = reg_wdata[31:0];
2136      assign key_4_we = addr_hit[13] & reg_we & !reg_error;
2137
2138      assign key_4_wd = reg_wdata[31:0];
2139      assign key_5_we = addr_hit[14] & reg_we & !reg_error;
2140
2141      assign key_5_wd = reg_wdata[31:0];
2142      assign key_6_we = addr_hit[15] & reg_we & !reg_error;
2143
2144      assign key_6_wd = reg_wdata[31:0];
2145      assign key_7_we = addr_hit[16] & reg_we & !reg_error;
2146
2147      assign key_7_wd = reg_wdata[31:0];
2148      assign key_8_we = addr_hit[17] & reg_we & !reg_error;
2149
2150      assign key_8_wd = reg_wdata[31:0];
2151      assign key_9_we = addr_hit[18] & reg_we & !reg_error;
2152
2153      assign key_9_wd = reg_wdata[31:0];
2154      assign key_10_we = addr_hit[19] & reg_we & !reg_error;
2155
2156      assign key_10_wd = reg_wdata[31:0];
2157      assign key_11_we = addr_hit[20] & reg_we & !reg_error;
2158
2159      assign key_11_wd = reg_wdata[31:0];
2160      assign key_12_we = addr_hit[21] & reg_we & !reg_error;
2161
2162      assign key_12_wd = reg_wdata[31:0];
2163      assign key_13_we = addr_hit[22] & reg_we & !reg_error;
2164
2165      assign key_13_wd = reg_wdata[31:0];
2166      assign key_14_we = addr_hit[23] & reg_we & !reg_error;
2167
2168      assign key_14_wd = reg_wdata[31:0];
2169      assign key_15_we = addr_hit[24] & reg_we & !reg_error;
2170
2171      assign key_15_wd = reg_wdata[31:0];
2172      assign key_16_we = addr_hit[25] & reg_we & !reg_error;
2173
2174      assign key_16_wd = reg_wdata[31:0];
2175      assign key_17_we = addr_hit[26] & reg_we & !reg_error;
2176
2177      assign key_17_wd = reg_wdata[31:0];
2178      assign key_18_we = addr_hit[27] & reg_we & !reg_error;
2179
2180      assign key_18_wd = reg_wdata[31:0];
2181      assign key_19_we = addr_hit[28] & reg_we & !reg_error;
2182
2183      assign key_19_wd = reg_wdata[31:0];
2184      assign key_20_we = addr_hit[29] & reg_we & !reg_error;
2185
2186      assign key_20_wd = reg_wdata[31:0];
2187      assign key_21_we = addr_hit[30] & reg_we & !reg_error;
2188
2189      assign key_21_wd = reg_wdata[31:0];
2190      assign key_22_we = addr_hit[31] & reg_we & !reg_error;
2191
2192      assign key_22_wd = reg_wdata[31:0];
2193      assign key_23_we = addr_hit[32] & reg_we & !reg_error;
2194
2195      assign key_23_wd = reg_wdata[31:0];
2196      assign key_24_we = addr_hit[33] & reg_we & !reg_error;
2197
2198      assign key_24_wd = reg_wdata[31:0];
2199      assign key_25_we = addr_hit[34] & reg_we & !reg_error;
2200
2201      assign key_25_wd = reg_wdata[31:0];
2202      assign key_26_we = addr_hit[35] & reg_we & !reg_error;
2203
2204      assign key_26_wd = reg_wdata[31:0];
2205      assign key_27_we = addr_hit[36] & reg_we & !reg_error;
2206
2207      assign key_27_wd = reg_wdata[31:0];
2208      assign key_28_we = addr_hit[37] & reg_we & !reg_error;
2209
2210      assign key_28_wd = reg_wdata[31:0];
2211      assign key_29_we = addr_hit[38] & reg_we & !reg_error;
2212
2213      assign key_29_wd = reg_wdata[31:0];
2214      assign key_30_we = addr_hit[39] & reg_we & !reg_error;
2215

```

```

2216 assign key_30_wd = reg_wdata[31:0];
2217 assign key_31_we = addr_hit[40] & reg_we & !reg_error;
2218
2219 assign key_31_wd = reg_wdata[31:0];
2220 assign digest_0_re = addr_hit[41] & reg_re & !reg_error;
2221 assign digest_0_we = addr_hit[41] & reg_we & !reg_error;
2222
2223 assign digest_0_wd = reg_wdata[31:0];
2224 assign digest_1_re = addr_hit[42] & reg_re & !reg_error;
2225 assign digest_1_we = addr_hit[42] & reg_we & !reg_error;
2226
2227 assign digest_1_wd = reg_wdata[31:0];
2228 assign digest_2_re = addr_hit[43] & reg_re & !reg_error;
2229 assign digest_2_we = addr_hit[43] & reg_we & !reg_error;
2230
2231 assign digest_2_wd = reg_wdata[31:0];
2232 assign digest_3_re = addr_hit[44] & reg_re & !reg_error;
2233 assign digest_3_we = addr_hit[44] & reg_we & !reg_error;
2234
2235 assign digest_3_wd = reg_wdata[31:0];
2236 assign digest_4_re = addr_hit[45] & reg_re & !reg_error;
2237 assign digest_4_we = addr_hit[45] & reg_we & !reg_error;
2238
2239 assign digest_4_wd = reg_wdata[31:0];
2240 assign digest_5_re = addr_hit[46] & reg_re & !reg_error;
2241 assign digest_5_we = addr_hit[46] & reg_we & !reg_error;
2242
2243 assign digest_5_wd = reg_wdata[31:0];
2244 assign digest_6_re = addr_hit[47] & reg_re & !reg_error;
2245 assign digest_6_we = addr_hit[47] & reg_we & !reg_error;
2246
2247 assign digest_6_wd = reg_wdata[31:0];
2248 assign digest_7_re = addr_hit[48] & reg_re & !reg_error;
2249 assign digest_7_we = addr_hit[48] & reg_we & !reg_error;
2250
2251 assign digest_7_wd = reg_wdata[31:0];
2252 assign digest_8_re = addr_hit[49] & reg_re & !reg_error;
2253 assign digest_8_we = addr_hit[49] & reg_we & !reg_error;
2254
2255 assign digest_8_wd = reg_wdata[31:0];
2256 assign digest_9_re = addr_hit[50] & reg_re & !reg_error;
2257 assign digest_9_we = addr_hit[50] & reg_we & !reg_error;
2258
2259 assign digest_9_wd = reg_wdata[31:0];
2260 assign digest_10_re = addr_hit[51] & reg_re & !reg_error;
2261 assign digest_10_we = addr_hit[51] & reg_we & !reg_error;
2262
2263 assign digest_10_wd = reg_wdata[31:0];
2264 assign digest_11_re = addr_hit[52] & reg_re & !reg_error;
2265 assign digest_11_we = addr_hit[52] & reg_we & !reg_error;
2266
2267 assign digest_11_wd = reg_wdata[31:0];
2268 assign digest_12_re = addr_hit[53] & reg_re & !reg_error;
2269 assign digest_12_we = addr_hit[53] & reg_we & !reg_error;
2270
2271 assign digest_12_wd = reg_wdata[31:0];
2272 assign digest_13_re = addr_hit[54] & reg_re & !reg_error;
2273 assign digest_13_we = addr_hit[54] & reg_we & !reg_error;
2274
2275 assign digest_13_wd = reg_wdata[31:0];
2276 assign digest_14_re = addr_hit[55] & reg_re & !reg_error;
2277 assign digest_14_we = addr_hit[55] & reg_we & !reg_error;
2278
2279 assign digest_14_wd = reg_wdata[31:0];
2280 assign digest_15_re = addr_hit[56] & reg_re & !reg_error;
2281 assign digest_15_we = addr_hit[56] & reg_we & !reg_error;
2282
2283 assign digest_15_wd = reg_wdata[31:0];
2284 assign msg_length_lower_re = addr_hit[57] & reg_re & !
reg_error;
2285 assign msg_length_lower_we = addr_hit[57] & reg_we & !
reg_error;
2286
2287 assign msg_length_lower_wd = reg_wdata[31:0];
2288 assign msg_length_upper_re = addr_hit[58] & reg_re & !
reg_error;
2289 assign msg_length_upper_we = addr_hit[58] & reg_we & !
reg_error;
2290
2291 assign msg_length_upper_wd = reg_wdata[31:0];
2292
2293 // Assign write-enables to checker logic vector.
2294 always_comb begin
2295   reg_we_check[0] = intr_state_we;
2296   reg_we_check[1] = intr_enable_we;
2297   reg_we_check[2] = intr_test_we;
2298   reg_we_check[3] = alert_test_we;
2299   reg_we_check[4] = cfg_we;
2300   reg_we_check[5] = cmd_we;
2301   reg_we_check[6] = 1'b0;
2302   reg_we_check[7] = 1'b0;
2303   reg_we_check[8] = wipe_secret_we;
2304   reg_we_check[9] = key_0_we;
2305   reg_we_check[10] = key_1_we;
2306   reg_we_check[11] = key_2_we;
2307   reg_we_check[12] = key_3_we;
2308   reg_we_check[13] = key_4_we;
2309   reg_we_check[14] = key_5_we;
2310   reg_we_check[15] = key_6_we;
2311   reg_we_check[16] = key_7_we;
2312   reg_we_check[17] = key_8_we;
2313   reg_we_check[18] = key_9_we;
2314   reg_we_check[19] = key_10_we;
2315   reg_we_check[20] = key_11_we;
2316   reg_we_check[21] = key_12_we;
2317   reg_we_check[22] = key_13_we;
2318   reg_we_check[23] = key_14_we;
2319   reg_we_check[24] = key_15_we;
2320   reg_we_check[25] = key_16_we;
2321   reg_we_check[26] = key_17_we;
2322   reg_we_check[27] = key_18_we;
2323   reg_we_check[28] = key_19_we;
2324   reg_we_check[29] = key_20_we;
2325   reg_we_check[30] = key_21_we;
2326   reg_we_check[31] = key_22_we;
2327   reg_we_check[32] = key_23_we;
2328   reg_we_check[33] = key_24_we;
2329   reg_we_check[34] = key_25_we;
2330   reg_we_check[35] = key_26_we;
2331   reg_we_check[36] = key_27_we;
2332   reg_we_check[37] = key_28_we;
2333   reg_we_check[38] = key_29_we;
2334   reg_we_check[39] = key_30_we;
2335   reg_we_check[40] = key_31_we;
2336   reg_we_check[41] = digest_0_we;
2337   reg_we_check[42] = digest_1_we;
2338   reg_we_check[43] = digest_2_we;
2339   reg_we_check[44] = digest_3_we;
2340   reg_we_check[45] = digest_4_we;
2341   reg_we_check[46] = digest_5_we;
2342   reg_we_check[47] = digest_6_we;
2343   reg_we_check[48] = digest_7_we;
2344   reg_we_check[49] = digest_8_we;
2345   reg_we_check[50] = digest_9_we;
2346   reg_we_check[51] = digest_10_we;
2347   reg_we_check[52] = digest_11_we;
2348   reg_we_check[53] = digest_12_we;
2349   reg_we_check[54] = digest_13_we;
2350   reg_we_check[55] = digest_14_we;
2351   reg_we_check[56] = digest_15_we;
2352   reg_we_check[57] = msg_length_lower_we;
2353   reg_we_check[58] = msg_length_upper_we;
2354 end
2355
2356 // Read data return
2357 always_comb begin
2358   reg_rdata_next = '0;
2359   unique case (1'b1)
2360     addr_hit[0]: begin
2361       reg_rdata_next[0] = intr_state_hmac_done_qs;
2362       reg_rdata_next[1] = intr_state_fifo_empty_qs;
2363       reg_rdata_next[2] = intr_state_hmac_err_qs;
2364     end
2365     addr_hit[1]: begin
2366       reg_rdata_next[0] = intr_enable_hmac_done_qs;
2367       reg_rdata_next[1] = intr_enable_fifo_empty_qs;
2368       reg_rdata_next[2] = intr_enable_hmac_err_qs;
2369     end
2370     addr_hit[2]: begin
2371       reg_rdata_next[0] = '0;
2372       reg_rdata_next[1] = '0;
2373       reg_rdata_next[2] = '0;
2374     end
2375     addr_hit[3]: begin
2376       reg_rdata_next[0] = '0;
2377     end
2378     addr_hit[4]: begin
2379       reg_rdata_next[0] = cfg_hmac_en_qs;
2380       reg_rdata_next[1] = cfg_sha_en_qs;
2381       reg_rdata_next[2] = cfg_endian_swap_qs;
2382       reg_rdata_next[3] = cfg_digest_swap_qs;
2383       reg_rdata_next[4] = cfg_key_swap_qs;
2384       reg_rdata_next[8:5] = cfg_digest_size_qs;
2385       reg_rdata_next[14:9] = cfg_key_length_qs;
2386     end
2387     addr_hit[5]: begin
2388       reg_rdata_next[0] = '0;
2389       reg_rdata_next[1] = '0;
2390       reg_rdata_next[2] = '0;
2391       reg_rdata_next[3] = '0;
2392     end
2393     addr_hit[6]: begin
2394       reg_rdata_next[0] = status_hmac_idle_qs;
2395       reg_rdata_next[1] = status_fifo_empty_qs;
2396       reg_rdata_next[2] = status_fifo_full_qs;
2397       reg_rdata_next[9:4] = status_fifo_depth_qs;
2398     end
2399     addr_hit[7]: begin
2400       reg_rdata_next[31:0] = err_code_qs;
2401     end
2402     addr_hit[8]: begin
2403       reg_rdata_next[31:0] = '0;
2404     end
2405   end

```

```

2414     addr_hit[9]: begin
2415         reg_rdata_next[31:0] = '0;
2416     end
2417
2418     addr_hit[10]: begin
2419         reg_rdata_next[31:0] = '0;
2420     end
2421
2422     addr_hit[11]: begin
2423         reg_rdata_next[31:0] = '0;
2424     end
2425
2426     addr_hit[12]: begin
2427         reg_rdata_next[31:0] = '0;
2428     end
2429
2430     addr_hit[13]: begin
2431         reg_rdata_next[31:0] = '0;
2432     end
2433
2434     addr_hit[14]: begin
2435         reg_rdata_next[31:0] = '0;
2436     end
2437
2438     addr_hit[15]: begin
2439         reg_rdata_next[31:0] = '0;
2440     end
2441
2442     addr_hit[16]: begin
2443         reg_rdata_next[31:0] = '0;
2444     end
2445
2446     addr_hit[17]: begin
2447         reg_rdata_next[31:0] = '0;
2448     end
2449
2450     addr_hit[18]: begin
2451         reg_rdata_next[31:0] = '0;
2452     end
2453
2454     addr_hit[19]: begin
2455         reg_rdata_next[31:0] = '0;
2456     end
2457
2458     addr_hit[20]: begin
2459         reg_rdata_next[31:0] = '0;
2460     end
2461
2462     addr_hit[21]: begin
2463         reg_rdata_next[31:0] = '0;
2464     end
2465
2466     addr_hit[22]: begin
2467         reg_rdata_next[31:0] = '0;
2468     end
2469
2470     addr_hit[23]: begin
2471         reg_rdata_next[31:0] = '0;
2472     end
2473
2474     addr_hit[24]: begin
2475         reg_rdata_next[31:0] = '0;
2476     end
2477
2478     addr_hit[25]: begin
2479         reg_rdata_next[31:0] = '0;
2480     end
2481
2482     addr_hit[26]: begin
2483         reg_rdata_next[31:0] = '0;
2484     end
2485
2486     addr_hit[27]: begin
2487         reg_rdata_next[31:0] = '0;
2488     end
2489
2490     addr_hit[28]: begin
2491         reg_rdata_next[31:0] = '0;
2492     end
2493
2494     addr_hit[29]: begin
2495         reg_rdata_next[31:0] = '0;
2496     end
2497
2498     addr_hit[30]: begin
2499         reg_rdata_next[31:0] = '0;
2500     end
2501
2502     addr_hit[31]: begin
2503         reg_rdata_next[31:0] = '0;
2504     end
2505
2506     addr_hit[32]: begin
2507         reg_rdata_next[31:0] = '0;
2508     end
2509
2510     addr_hit[33]: begin
2511         reg_rdata_next[31:0] = '0;
2512     end
2513
2514     addr_hit[34]: begin
2515         reg_rdata_next[31:0] = '0;
2516     end
2517
2518     addr_hit[35]: begin
2519         reg_rdata_next[31:0] = '0;
2520     end
2521
2522     addr_hit[36]: begin
2523         reg_rdata_next[31:0] = '0;
2524     end
2525
2526     addr_hit[37]: begin
2527         reg_rdata_next[31:0] = '0;
2528     end
2529
2530     addr_hit[38]: begin
2531         reg_rdata_next[31:0] = '0;
2532     end
2533
2534     addr_hit[39]: begin
2535         reg_rdata_next[31:0] = '0;
2536     end
2537
2538     addr_hit[40]: begin
2539         reg_rdata_next[31:0] = '0;
2540     end
2541
2542     addr_hit[41]: begin
2543         reg_rdata_next[31:0] = digest_0_qs;
2544     end
2545
2546     addr_hit[42]: begin
2547         reg_rdata_next[31:0] = digest_1_qs;
2548     end
2549
2550     addr_hit[43]: begin
2551         reg_rdata_next[31:0] = digest_2_qs;
2552     end
2553
2554     addr_hit[44]: begin
2555         reg_rdata_next[31:0] = digest_3_qs;
2556     end
2557
2558     addr_hit[45]: begin
2559         reg_rdata_next[31:0] = digest_4_qs;
2560     end
2561
2562     addr_hit[46]: begin
2563         reg_rdata_next[31:0] = digest_5_qs;
2564     end
2565
2566     addr_hit[47]: begin
2567         reg_rdata_next[31:0] = digest_6_qs;
2568     end
2569
2570     addr_hit[48]: begin
2571         reg_rdata_next[31:0] = digest_7_qs;
2572     end
2573
2574     addr_hit[49]: begin
2575         reg_rdata_next[31:0] = digest_8_qs;
2576     end
2577
2578     addr_hit[50]: begin
2579         reg_rdata_next[31:0] = digest_9_qs;
2580     end
2581
2582     addr_hit[51]: begin
2583         reg_rdata_next[31:0] = digest_10_qs;
2584     end
2585
2586     addr_hit[52]: begin
2587         reg_rdata_next[31:0] = digest_11_qs;
2588     end
2589
2590     addr_hit[53]: begin
2591         reg_rdata_next[31:0] = digest_12_qs;
2592     end
2593
2594     addr_hit[54]: begin
2595         reg_rdata_next[31:0] = digest_13_qs;
2596     end
2597
2598     addr_hit[55]: begin
2599         reg_rdata_next[31:0] = digest_14_qs;
2600     end
2601
2602     addr_hit[56]: begin
2603         reg_rdata_next[31:0] = digest_15_qs;
2604     end
2605
2606     addr_hit[57]: begin
2607         reg_rdata_next[31:0] = msg_length_lower_qs;
2608     end
2609
2610     addr_hit[58]: begin
2611         reg_rdata_next[31:0] = msg_length_upper_qs;
2612     end
2613
2614     default: begin
2615         reg_rdata_next = '1;

```

```

2616     end
2617   endcase
2618 end
2619
2620 // shadow busy
2621 logic shadow_busy;
2622 assign shadow_busy = 1'b0;
2623
2624 // register busy
2625 assign reg_busy = shadow_busy;
2626
2627 // Unused signal tieoff
2628
2629 // wdata / byte enable are not always fully used
2630 // add a blanket unused statement to handle lint waivers
2631 logic unused_wdata;
2632 logic unused_be;
2633 assign unused_wdata = ^reg_wdata;
2634 assign unused_be = ^reg_be;

```

```

2635
2636 // Assertions for Register Interface
2637 `ASSERT_PULSE(wePulse, reg_we, clk_i, !rst_ni)
2638 `ASSERT_PULSE(rePulse, reg_re, clk_i, !rst_ni)
2639
2640 `ASSERT(reAfterRv, $rose(reg_re || reg_we) |>
2641 tl_o_pre.d_valid, clk_i, !rst_ni)
2642
2643 `ASSERT(en2addrHit, (reg_we || reg_re) |> $one-
2644 hot0(addr_hit), clk_i, !rst_ni)
2645
2646 // this is formulated as an assumption such that the FPV
2647 testbenches do disprove this
2648 // property by mistake
2649 `ASSUME(reqParity, tl_reg_h2d.a_valid |>
2650 tl_reg_h2d.a_user.chk_en == tlul_pkg::CheckDis)
2651
2652 endmodule

```


OpenTitan KMAC (Keccak/SHA-3)

What this is. SHA-3 family and KMAC built on the Keccak-f[1600] permutation.

Why it exists. Because its construction differs fundamentally from SHA-2. It is a sponge, not a Merkle–Damgård chain: a large state is stirred by a permutation while data is absorbed and squeezed. In hardware this becomes a 1600-bit register and very wide combinational logic.

Undergraduate review · EECS concepts. Sponge constructions; permutation-based cryptography; bit-slice implementation; routing congestion in wide datapaths.

Graduate · Practice note. Keccak is a block where **wiring, not gate count, determines area**. If post-synthesis area greatly exceeds a gate-count estimate, congestion is usually the reason and RTL changes will not fix it. Include a routing factor in early estimates.

5 files • 4,651 lines

API — keccak_2share.sv

Table 811. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_2share.sv — module / interface / package (1개)

이름	종류
keccak_2share	module

Table 812. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_2share.sv — parameter (3개)

파라미터	형/폭	기본값
Width	int	1600
EnMasking	bit	1'b0
ForceRandExt	bit	1'b0

Table 813. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_2share.sv — localparam (10개)

상수	형/폭	값
W	int	Width/25
L	int	$\$clog2(W)$
MaxRound	int	$12 + 2 \cdot L$
RndW	int	$\$clog2(\text{MaxRound} + 1)$
Share	int	$\text{EnMasking} ? 2 : 1$
WSheetHalf	int unsigned	$\$bits(\text{sheet_t})/2$
X1	int	$(x + 1) \% 5$
X2	int	$(x + 2) \% 5$
Offset	int	$\text{RhoOffset}[5 \cdot x + y] \% W$
ShiftAmt	int	W- Offset

Table 814. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_2share.sv — 포트 (10개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
rnd_i	input	[RndW-1:0]
phase_sel_i	input	mubi4_t
dom_out_low_i	input	
dom_in_low_i	input	
dom_in_rand_ext_i	input	
dom_update_i	input	
rand_i	input	[Width/2-1:0]

Table 815. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_2share.sv — typedef / using (7개)

새 이름	원래 형
box_t	logic [4:0][4:0][W-1:0]
lane_t	logic [W-1:0]
plane_t	logic [4:0] [W-1:0]
slice_t	logic [4:0][4:0]
sheet_t	logic [4:0][W-1:0]
row_t	logic [4:0]
col_t	logic [4:0]

API — keccak_round.sv**Table 816. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_round.sv — module / interface / package (1개)**

이름	종류
keccak_round	module

Table 817. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_round.sv — parameter (4개)

파라미터	형/폭	기본값
Width	int	1600
DInWidth	int	64
EnMasking	bit	1'b0
ForceRandExt	bit	1'b0

Table 818. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_round.sv — localparam (7개)

상수	형/폭	값
W	int	Width/25

상수	형/폭	값
L	int	$\$clog2(W)$
MaxRound	int	$12 + 2 * L$
RndW	int	$\$clog2(MaxRound+1)$
DInEntry	int	Width / DInWidth
DInAddr	int	$\$clog2(DInEntry)$
Share	int	EnMasking ? 2 : 1

Table 819. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_round.sv — 포트 (18개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
valid_i	input	
addr_i	input	[DInAddr-1:0]
ready_o	output	
run_i	input	
rand_valid_i	input	
rand_early_i	input	
rand_data_i	input	[Width/2-1:0]
rand_aux_i	input	
rand_update_o	output	logic
rand_consumed_o	output	logic
complete_o	output	logic
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
sparse_fsm_error_o	output	logic
round_count_error_o	output	logic
rst_storage_error_o	output	logic
0	input	prim_mubi_pkg::mubi4_t clear_i // Clear internal s...

API — kmac.sv

Table 820. lowRISC_opentitan/hw/ip/kmac/rtl/kmac.sv — module / interface / package (1개)

이름	종류
kmac	module

Table 821. lowRISC_opentitan/hw/ip/kmac/rtl/kmac.sv — parameter (11개)

파라미터	형/폭	기본값
EnMasking	bit	1

파라미터	형/폭	기본값
SwKeyMasked	bit	0
SecCmdDelay	int	0
SecIdleAcceptSwMsg	bit	1'b0
NumAppIntf	int unsigned	4
RndCnstLfsrPerm	lfsr_perm_t	RndCnstLfsrPermDefault
RndCnstLfsrSeed	lfsr_seed_t	RndCnstLfsrSeedDefault
RndCnstBufferLfsrSeed	buffer_lfsr_seed_t	RndCnstBufferLfsrSeedDefault
RndCnstMsgPerm	msg_perm_t	RndCnstMsgPermDefault
AlertAsync0n	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1

Table 822. lowRISC_opentitan/hw/ip/kmac/rtl/kmac.sv — localparam (5개)

상수	형/폭	값
Share	int	(EnMasking) ? 2 : 1
SwKeyShare	int	(EnMasking SwKeyMasked) ? 2 : 1
StateWidth	int	6
NumLcSyncCopies	int unsigned	6
WidthCounter	int unsigned	\$clog2(SecCmdDelay+1)

Table 823. lowRISC_opentitan/hw/ip/kmac/rtl/kmac.sv — 포트 (20개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
rst_shadowed_ni	input	
clk_edn_i	input	
rst_edn_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
keymgr_key_i	input	keymgr_pkg::hw_key_req_t
app_i	input	app_req_t [NumAppIntf-1:0]
app_o	output	app_rsp_t [NumAppIntf-1:0]
entropy_o	output	edn_pkg::edn_req_t
entropy_i	input	edn_pkg::edn_rsp_t
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t

포트	방향	형/폭
intr_kmac_done_o	output	logic
intr_fifo_empty_o	output	logic
intr_kmac_err_o	output	logic
en_masking_o	output	logic
idle_o	output	prim_mubi_pkg::mubi4_t

API — kmac_app.sv

Table 824. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_app.sv — module / interface / package (1개)

이름	종류
kmac_app	module

Table 825. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_app.sv — parameter (3개)

파라미터	형/폭	기본값
EnMasking	bit	1'b0
SecIdleAcceptSwMsg	bit	1'b0
NumAppIntf	int unsigned	4

Table 826. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_app.sv — localparam (9개)

상수	형/폭	값
Share	int	(EnMasking) ? 2 : 1
KeyMgrKeyW	int	\$bits(keymgr_key_i.key[0])
SelKeySize	int	(AppKeyW == 128) ? 0 :
SelDigSize	int	(AppDigestW == 128) ? 0 :
SideloadedKey	key_len_e	KeyLengths[SelKeySize]
OutLenW	int	24
AppIdxW	int unsigned	\$clog2(NumAppIntf)
MaxNumDigestParts	int	(sha3_pkg::StateW - 2 * 128) / DynAppDigestW
DigestCntW	int	\$clog2(MaxNumDigestParts)

Table 827. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_app.sv — 포트 (43개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
reg_key_len_i	input	key_len_e
reg_prefix_i	input	[sha3_pkg::NSRegisterSize*8-1:0]
reg_kmac_en_i	input	logic
reg_sha3_mode_i	input	sha3_pkg::sha3_mode_e

포트	방향	형/폭
reg_keccak_strength_i	input	sha3_pkg::keccak_strength_e
sw_valid_i	input	
sw_data_i	input	[MsgWidth-1:0]
sw_strb_i	input	[MsgWidth-1:0]
sw_ready_o	output	logic
keymgr_key_i	input	keymgr_pkg::hw_key_req_t
app_i	input	app_req_t [NumAppIntf-1:0]
app_o	output	app_rsp_t [NumAppIntf-1:0]
key_len_o	output	key_len_e
key_valid_o	output	logic
kmac_valid_o	output	logic
kmac_strb_o	output	logic [MsgWidth-1:0]
kmac_ready_i	input	logic
kmac_bypass_fifo_o	output	logic
kmac_en_o	output	logic
sha3_prefix_o	output	logic [sha3_pkg::NSRegisterSize*8-1:0]
sha3_mode_o	output	sha3_pkg::sha3_mode_e
keccak_strength_o	output	sha3_pkg::keccak_strength_e
keccak_state_valid_i	input	
reg_state_valid_o	output	logic
keymgr_key_en_i	input	logic
sw_cmd_i	input	kmac_cmd_e
absorbed_i	input	prim_mubi_pkg::mubi4_t
squeezing_i	input	logic
cmd_o	output	kmac_cmd_e
absorbed_o	output	prim_mubi_pkg::mubi4_t
app_active_o	output	logic
entropy_fast_process_i	input	logic
entropy_fast_process_o	output	logic
entropy_ready_i	input	prim_mubi_pkg::mubi4_t
error_i	input	
err_processed_i	input	
clear_after_error_o	output	prim_mubi_pkg::mubi4_t
error_o	output	kmac_pkg::err_t
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t

포트	방향	형/폭
sparse_fsm_error_o	output	logic
counter_error_o	output	logic

API — kmac_core.sv

Table 828. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_core.sv — module / interface / package (1개)

이름	종류
kmac_core	module

Table 829. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_core.sv — parameter (1개)

파라미터	형/폭	기본값
EnMasking	bit	0

Table 830. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_core.sv — localparam (2개)

상수	형/폭	값
Share	int	(EnMasking) ? 2 : 1 // derived parameter
StateWidth	int	6

Table 831. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_core.sv — 포트 (20개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
fifo_valid_i	input	
fifo_strb_i	input	[MsgStrbW-1:0]
fifo_ready_o	output	logic
msg_valid_o	output	logic
msg_strb_o	output	logic [MsgStrbW-1:0]
msg_ready_i	input	
kmac_en_i	input	
mode_i	input	sha3_pkg::sha3_mode_e
strength_i	input	sha3_pkg::keccak_strength_e
key_len_i	input	key_len_e
key_valid_i	input	logic
start_i	input	
process_i	input	
done_i	input	prim_mubi_pkg::mubi4_t
process_o	output	logic
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t

포트	방향	형/폭
sparse_fsm_error_o	output	logic
key_index_error_o	output	logic

1. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_2share.sv (563 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // This module is the single round keccak permutation module
6 // It supports Keccak with up to 1600b of state
7
8 `include "prim_assert.sv"
9
10 module keccak_2share
11     import prim_mubi_pkg::*;
12 # (
13     parameter int Width = 1600, // b= {25, 50, 100, 200, 400,
800, 1600}
14
15     // Derived
16     localparam int W      = Width/25,
17     localparam int L      = $clog2(W),
18     localparam int MaxRound = 12 + 2*L,          // Keccak-f
  only
19     localparam int RndW    = $clog2(MaxRound+1), // Represent-
  ing up to MaxRound
20
21     // Control parameters
22     parameter bit EnMasking = 1'b0, // Enable secure harden-
  ing
23     parameter bit ForceRandExt = 1'b0, // 1: Always forward ex-
  ternally provided randomness.
24                                     // 0: Switch between ex-
  ternal randomness and internal
25                                     // intermediate state
  according to dom_in_rand_ext_i.
26     localparam int Share      = EnMasking ? 2 : 1
27 ) (
28     input clk_i,
29     input rst_ni,
30
31     input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i, // Used to
  disable SVAs when escalating.
32
33     input [RndW-1:0] rnd_i, // Current round index
34
35     // Control inputs used when EnMasking = 1.
36     input mubi4_t phase_sel_i, // Output mux control
37     input dom_out_low_i, // DOM multiplier output
  mux
38     input dom_in_low_i, // DOM multiplier input mux
39     input dom_in_rand_ext_i, // DOM multiplier input
  randomness mux
40     input dom_update_i, // DOM multiplier pipeline
  reg write enable
41
42     input [Width/2-1:0] rand_i, // Randomness for remask-
  ing.
43
44     // State input and output
45     input [Width-1:0] s_i [Share],
46     output logic [Width-1:0] s_o [Share]
47 );
48 //////////////////////////////////////////////////
49 // Types //
50 //////////////////////////////////////////////////
51 //
52 // x y z
53 typedef logic [4:0][4:0][W-1:0] box_t; // (x,y,z) state
54 typedef logic [4:0][W-1:0] lane_t; // (z)
55 typedef logic [4:0][4:0] slice_t; // (x,y)
56 typedef logic [4:0][W-1:0] sheet_t; // (y,z) identical
  to plane_t
57 typedef logic [4:0] row_t; // (x)
58 typedef logic [4:0] col_t; // (y) identical to
  row_t
59
60 //////////////////////////////////////////////////
61 // Keccak_f //
62 //////////////////////////////////////////////////
63 box_t state_in [Share];
64 box_t state_out [Share];
65 box_t theta_data [Share];
66 box_t rho_data [Share];
67 box_t pi_data [Share];
68 box_t chi_data [Share];

```

```

69 box_t iota_data [Share];
70
71 box_t phase1_in [Share];
72 box_t phase1_out [Share];
73 box_t phase2_in [Share];
74 box_t phase2_out [Share];
75
76 //////////////////////////////////////////////////
77 // Unused nets //
78 //////////////////////////////////////////////////
79 // Tie off input signals that aren't used in the unmasked
  implementation.
80 if (!EnMasking) begin : gen_tie_unused
81     logic unused_clk;
82     logic unused_rst_n;
83     mubi4_t unused_phase_sel;
84     logic unused_dom_ctrl;
85     logic [Width/2-1:0] unused_rand;
86     assign unused_clk = clk_i;
87     assign unused_rst_n = rst_ni;
88     assign unused_phase_sel = phase_sel_i;
89     assign unused_dom_ctrl =
90         ^{dom_out_low_i, dom_in_low_i, dom_in_rand_ext_i,
  dom_update_i};
91     assign unused_rand = rand_i;
92 end
93
94 //////////////////////////////////////////////////
95 // Input/output type conversion and interfacing //
96 //////////////////////////////////////////////////
97 for (genvar i = 0 ; i < Share ; i++) begin : g_state_inout
98     assign state_in[i] = bitarray_to_box(s_i[i]);
99     assign s_o[i] = box_to_bitarray(state_out[i]);
100 end : g_state_inout
101
102 if (EnMasking) begin : g_2share_data
103     assign phase1_in = state_in;
104     assign phase2_in = state_in;
105
106     always_comb begin
107         unique case (phase_sel_i)
108             MUBi4False: state_out = phase1_out;
109             MUBi4True: state_out = phase2_out;
110             default: state_out = phase1_out;
111         endcase
112     end
113 end else begin : g_single_data
114     assign phase1_in = state_in;
115     assign phase2_in = phase1_out;
116     assign state_out = phase2_out;
117 end
118
119 //////////////////////////////////////////////////
120 // Datapath //
121 //////////////////////////////////////////////////
122 for (genvar i = 0 ; i < Share ; i++) begin : g_datapath
123
124     // Phase 1:
125     assign theta_data[i] = theta(phase1_in[i]);
126     // Commented out rho function as vcs complains z-Offset%W
  isn't constant
127     // assign rho_data[i] = rho(theta_data[i]);
128
129     assign pi_data[i] = pi(rho_data[i]);
130
131     // Phase 2 (Cycles 1, 2 and 3):
132     // Chi : See below
133     // Iota: See below
134 end : g_datapath
135
136 assign phase1_out = pi_data;
137
138 // Iota adds Round Constants(RC), so only one share should
  be XORed
139 if (EnMasking) begin : g_2share_iota
140     assign iota_data[0] = iota(chi_data[0], rnd_i);
141     assign iota_data[1] = chi_data[1];
142 end else begin : g_single_iota
143     assign iota_data[0] = iota(chi_data[0], rnd_i);
144 end
145
146 if (EnMasking) begin : g_2share_chi
147     // Domain-Oriented Masking
148     // reference: https://eprint.iacr.org/2017/395.pdf
149
150     localparam int unsigned WSheetHalf = $bits(sheet_t)/2;
151     logic [4:0][WSheetHalf-1:0] in_prd, out_prd;
152
153     //////////////////////////////////////////////////
154     // DOM multipliers //
155     //////////////////////////////////////////////////

```

```

156
157   for (genvar x = 0 ; x < 5 ; x++) begin : g_chi_w
158     localparam int X1 = (x + 1) % 5;
159     localparam int X2 = (x + 2) % 5;
160
161     sheet_t sheet0[Share]; // Inverted input X1
162     sheet_t sheet1[Share]; // X2
163     sheet_t sheet2[Share]; // DOM output
164
165     assign sheet0[0] = ~phase2_in[0][X1];
166     assign sheet0[1] = phase2_in[1][X1];
167
168     assign sheet1[0] = phase2_in[0][X2];
169     assign sheet1[1] = phase2_in[1][X2];
170
171     // Convert sheet_t to 1D arrays, one for the upper and
lower half lane.
172     logic [WSheetHalf-1:0] a0_l, a1_l, b0_l, b1_l;
173     logic [WSheetHalf-1:0] a0_h, a1_h, b0_h, b1_h;
174     logic [WSheetHalf-1:0] a0, a1, b0, b1, q0, q1;
175
176     assign a0_l = {sheet0[0][0][W/2-1:0],
177                   sheet0[0][1][W/2-1:0],
178                   sheet0[0][2][W/2-1:0],
179                   sheet0[0][3][W/2-1:0],
180                   sheet0[0][4][W/2-1:0]};
181     assign a1_l = {sheet0[1][0][W/2-1:0],
182                   sheet0[1][1][W/2-1:0],
183                   sheet0[1][2][W/2-1:0],
184                   sheet0[1][3][W/2-1:0],
185                   sheet0[1][4][W/2-1:0]};
186
187     assign a0_h = {sheet0[0][0][W-1:W/2],
188                   sheet0[0][1][W-1:W/2],
189                   sheet0[0][2][W-1:W/2],
190                   sheet0[0][3][W-1:W/2],
191                   sheet0[0][4][W-1:W/2]};
192     assign a1_h = {sheet0[1][0][W-1:W/2],
193                   sheet0[1][1][W-1:W/2],
194                   sheet0[1][2][W-1:W/2],
195                   sheet0[1][3][W-1:W/2],
196                   sheet0[1][4][W-1:W/2]};
197
198     assign b0_l = {sheet1[0][0][W/2-1:0],
199                   sheet1[0][1][W/2-1:0],
200                   sheet1[0][2][W/2-1:0],
201                   sheet1[0][3][W/2-1:0],
202                   sheet1[0][4][W/2-1:0]};
203     assign b1_l = {sheet1[1][0][W/2-1:0],
204                   sheet1[1][1][W/2-1:0],
205                   sheet1[1][2][W/2-1:0],
206                   sheet1[1][3][W/2-1:0],
207                   sheet1[1][4][W/2-1:0]};
208
209     assign b0_h = {sheet1[0][0][W-1:W/2],
210                   sheet1[0][1][W-1:W/2],
211                   sheet1[0][2][W-1:W/2],
212                   sheet1[0][3][W-1:W/2],
213                   sheet1[0][4][W-1:W/2]};
214     assign b1_h = {sheet1[1][0][W-1:W/2],
215                   sheet1[1][1][W-1:W/2],
216                   sheet1[1][2][W-1:W/2],
217                   sheet1[1][3][W-1:W/2],
218                   sheet1[1][4][W-1:W/2]};
219
220     // Input muxing
221     assign a0 = dom_in_low_i ? a0_l : a0_h;
222     assign a1 = dom_in_low_i ? a1_l : a1_h;
223     assign b0 = dom_in_low_i ? b0_l : b0_h;
224     assign b1 = dom_in_low_i ? b1_l : b1_h;
225
226     // Randomness muxing
227     if (!ForceRandExt) begin : gen_in_prd_mux
228       // Intermediate results are rotated across rows. The
new Row x depends on
229       // data from Rows x + 1 and x + 2. Hence we don't want
to use intermediate
230       // results from Rows x, x + 1, and x + 2 for remask-
ing.
231       assign in_prd[x] = dom_in_rand_ext_i ? rand_i[x *
WSheetHalf +: WSheetHalf] :
232
233       out_prd[rot_int(x, 5)];
234       end else begin : gen_no_in_prd_mux
235         // Always use the externally provided randomness.
236         assign in_prd[x] = rand_i[x * WSheetHalf +: WSheetH-
alf];
237         // Tie off unused signals.
238         logic unused_out_prd;
239         assign unused_out_prd = ^{dom_in_rand_ext_i,
out_prd[rot_int(x, 5)]};
240       end
241       prim_dom_and_2share #(
242         .DW (WSheetHalf), // a half sheet
243         .Pipeline(1) // Process the full sheet in 3 clock
cycles. This reduces
244         // SCA leakage.
245       ) u_dom (
246         .clk_i,

```

```

247     .rst_ni,
248
249     .a0_i    (a0),
250     .a1_i    (a1),
251     .b0_i    (b0),
252     .b1_i    (b1),
253     .z_valid_i (dom_update_i),
254     .z_i      (in_prd[x]),
255     .q0_o     (q0),
256     .q1_o     (q1),
257     .prd_o    (out_prd[x])
258   );
259
260   // Output conversion from q0, q1 to sheet_t
261
262   // For simplicity, we forward the generated lane half to
both the upper
263   // and lower lane halves at this point. The actual out-
put muxing/selection
264   // happens after the Iota step when generating
phase2_out from iota_data
265   // and state_in below.
266   assign sheet2[0][4] = {2{q0[W/2*0+:W/2]}};
267   assign sheet2[0][3] = {2{q0[W/2*1+:W/2]}};
268   assign sheet2[0][2] = {2{q0[W/2*2+:W/2]}};
269   assign sheet2[0][1] = {2{q0[W/2*3+:W/2]}};
270   assign sheet2[0][0] = {2{q0[W/2*4+:W/2]}};
271
272   assign sheet2[1][4] = {2{q1[W/2*0+:W/2]}};
273   assign sheet2[1][3] = {2{q1[W/2*1+:W/2]}};
274   assign sheet2[1][2] = {2{q1[W/2*2+:W/2]}};
275   assign sheet2[1][1] = {2{q1[W/2*3+:W/2]}};
276   assign sheet2[1][0] = {2{q1[W/2*4+:W/2]}};
277
278   // Final XOR to generate the output
279   assign chi_data[0][x] = sheet2[0] ^ phase2_in[0][x];
280   assign chi_data[1][x] = sheet2[1] ^ phase2_in[1][x];
281   end : g_chi_w
282
283   // Since Chi and thus Iota are separately applied to the
lower and upper half
284   // lanes, we need to forward the input to the other half.
285   for (genvar x = 0 ; x < 5 ; x++) begin :
g_2share_phase2_out_row
286     for (genvar y = 0 ; y < 5 ; y++) begin :
g_2share_phase2_out_col
287       assign phase2_out[0][x][y] = dom_out_low_i ?
{ state_in[0][x][y][W-1:W/2], iota_data[0][x][y]
[W/2-1:0] } :
288       {iota_data[0][x][y][W-1:W/2], state_in[0][x][y]
[W/2-1:0]};
289       assign phase2_out[1][x][y] = dom_out_low_i ?
{ state_in[1][x][y][W-1:W/2], iota_data[1][x][y]
[W/2-1:0] } :
290       {iota_data[1][x][y][W-1:W/2], state_in[1][x][y]
[W/2-1:0]};
291     end
292   end
293   end
294
295   end else begin : g_single_chi
296     assign chi_data[0] = chi(phase2_in[0]);
297     assign phase2_out = iota_data;
298   end
299
300   // Rho
=====
301   // As RhoOffset[x][y] is considered as variable int in VCS,
302   // it is replaced with generate statement.
303   // Revised to meet verifier lint. Now RhoOffset is 1-D ar-
ray
304   localparam int RhoOffset [25] = '{
305     //y  0   1   2   3   4   x
306     0, 36, 3, 105, 210, // 0: 0 1 2 3 4
307     1, 300, 10, 45, 66, // 1: 5 6 7 8 9
308     190, 6, 171, 15, 253, // 2: 10 11 12 13 14
309     28, 55, 153, 21, 120, // 3: 15 16 17 18 19
310     91, 276, 231, 136, 78 // 4: 20 21 22 23 24
311   };
312   for (genvar i = 0 ; i < Share ; i++) begin : g_rho
313     box_t rho_in, rho_out;
314     assign rho_in = theta_data[i];
315     assign rho_data[i] = rho_out;
316
317     for (genvar x = 0 ; x < 5 ; x++) begin : gen_rho_x
318       for (genvar y = 0 ; y < 5 ; y++) begin : gen_rho_y
319         localparam int Offset = RhoOffset[5*x+y]%W;
320         localparam int ShiftAmt = W- Offset;
321         if (Offset == 0) begin : gen_offset0
322           assign rho_out[x][y][W-1:0] = rho_in[x][y][W-1:0];
323         end else begin : gen_others
324           assign rho_out[x][y][W-1:0] = {rho_in[x][y]
[0+:ShiftAmt],
325                                           rho_in[x][y]
[ShiftAmt+:Offset]};
326         end
327       end
328     end
329   end : g_rho
330
331   ///////////////

```

```

332 // Assertions //
333 ////////////////
334
335 `ASSERT_INIT(ValidWidth_A,
336   EnMasking == 0 && Width inside {25, 50, 100, 200, 400,
800, 1600} ||
337   EnMasking == 1 && Width inside {50, 100, 200, 400, 800,
1600})
338 `ASSERT_INIT(ValidW_A, W inside {1, 2, 4, 8, 16, 32, 64})
339 `ASSERT_INIT(ValidL_A, L inside {0, 1, 2, 3, 4, 5, 6})
340 `ASSERT_INIT(ValidRound_A, MaxRound <= 24) // Keccak-f only
341
342 // phase_sel_i shall stay for two cycle after change to 1.
343 lc_ctrl_pkg::lc_tx_t unused_lc_sig;
344 assign unused_lc_sig = lc_escalate_en_i;
345 if (EnMasking) begin : gen_selperiod_chk
346   `ASSUME(StayTwoCycleIfTrue_A,
347     ($past(phase_sel_i) == MuBi4False) && (phase_sel_i ==
MuBi4True)
348     | => phase_sel_i == MuBi4True, clk_i, !rst_ni ||
349     lc_c-
trl_pkg::lc_tx_test_true_loose(lc_escalate_en_i))
350 end
351
352 // Functions //
353 ////////////////
354
355 // Convert bitarray to 3D box
356 // Please take a look at FIPS PUB 202
357 // https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf
358 // > For all triples (x,y,z) such that 0<=x<5, 0<=y<5, and
0<=z<w,
359 // > A[x,y,z]=S[w(5y+x)+z]
360 function automatic box_t bitarray_to_box(logic [Width-1:0]
s_in);
361   automatic box_t box;
362   for (int y = 0 ; y < 5 ; y++) begin
363     for (int x = 0 ; x < 5 ; x++) begin
364       for (int z = 0 ; z < W ; z++) begin
365         box[x][y][z] = s_in[W*(5*y+x) + z];
366       end
367     end
368   end
369   return box;
370 endfunction : bitarray_to_box
371
372 // Convert 3D cube to bitarray
373 function automatic logic [Width-1:0] box_to_bitarray(box_t
state);
374   automatic logic [Width-1:0] bitarray;
375   for (int y = 0 ; y < 5 ; y++) begin
376     for (int x = 0 ; x < 5 ; x++) begin
377       for (int z = 0 ; z < W ; z++) begin
378         bitarray[W*(5*y+x)+z] = state[x][y][z];
379       end
380     end
381   end
382   return bitarray;
383 endfunction : box_to_bitarray
384
385 // Rotate integer indices
386 function automatic integer rot_int(integer in, integer num);
387   integer out;
388   if (in == 0) begin
389     out = num - 1;
390   end else begin
391     out = in - 1;
392   end
393   return out;
394 endfunction
395
396 // Step Mapping
397 // =====
398 // theta
399 // XOR each bit in the state with the parity of two columns
400 // C[x,z] = A[x,0,z] ^ A[x,1,z] ^ A[x,2,z] ^ A[x,3,z] ^ A[x,
4,z]
401 // D[x,z] = C[x-1,z] ^ C[x+1,z-1]
402 // theta = A[x,y,z] ^ D[x,z]
403 localparam int ThetaIndexX1 [5] = '{4, 0, 1, 2, 3}; //
(x-1)%5
404 localparam int ThetaIndexX2 [5] = '{1, 2, 3, 4, 0}; //
(x+1)%5
405 function automatic box_t theta(box_t state);
406   plane_t c;
407   plane_t d;
408   box_t result;
409   for (int x = 0 ; x < 5 ; x++) begin
410     c[x] = state[x][0] ^ state[x][1] ^ state[x][2] ^
state[x][3] ^ state[x][4];
411   end
412   for (int x = 0 ; x < 5 ; x++) begin
413     for (int z = 0 ; z < W ; z++) begin
414       int index_z;
415       index_z = (z == 0) ? W-1 : z-1; // (z+1)%W
416       d[x][z] = c[ThetaIndexX1[x]][z] ^ c[ThetaIndexX2[x]]
[index_z];
417     end
418   end
419   for (int x = 0 ; x < 5 ; x++) begin

```

```

420     for (int y = 0 ; y < 5 ; y++) begin
421       result[x][y] = state[x][y] ^ d[x];
422     end
423   end
424   return result;
425 endfunction : theta
426
427 // rho
428
429 // Commented out entire rho function due to VCS elaboration
error.
430 // (z-RhoOffset[x][y]%W) isn't considered as a constant in
VCS.
431 // Even changing it to W-RhoOffset[x][y]%W and assign to
ShiftAmt
432 // creates same error.
433
434 // Offset : Look at Table 2 in FIPS PUB 202
435 //localparam int RhoOffset [5][5] = '{
436 // //y 0 1 2 3 4 x
437 // '{ 0, 36, 3, 105, 210}, // 0
438 // '{ 1, 300, 10, 45, 66}, // 1
439 // '{ 190, 6, 171, 15, 253}, // 2
440 // '{ 28, 55, 153, 21, 120}, // 3
441 // '{ 91, 276, 231, 136, 78} // 4
442 //};
443
444 // rotate bits of each lane by offset
445 // 1. rho[0,0,z] = A[0,0,z]
446 // 2. Offset swap
447 // a. (x,y) := (1,0)
448 // b. for t [0..23]
449 //   i. rho[x,y,z] = A[x,y,z-(t+1)(t+2)/2]
450 //   ii. (x,y) = (y, (2x+3y))
451 //function automatic box_t rho(box_t state);
452 // box_t result;
453 // for (int x = 0 ; x < 5 ; x++) begin
454 //   for (int y = 0 ; y < 5 ; y++) begin
455 //     for (int z = 0 ; z < W ; z++) begin
456 //       automatic int index_z;
457 //       index_z = (z-RhoOffset[x][y]%W;
458 //       result[x][y][z] = state[x][y][(z-RhoOffset[x][y])
%W];
459 //     end
460 //   end
461 // end
462 // return result;
463 //endfunction : rho
464
465 // pi
466 // rearrange the position of lanes
467 // pi[x,y,z] = state[(x+3y),x,z]
468 localparam int PiRotate [5][5] = '{
469 //y 0 1 2 3 4 x
470 // '{ 0, 3, 1, 4, 2}, // 0
471 // '{ 1, 4, 2, 0, 3}, // 1
472 // '{ 2, 0, 3, 1, 4}, // 2
473 // '{ 3, 1, 4, 2, 0}, // 3
474 // '{ 4, 2, 0, 3, 1} // 4
475 };
476 function automatic box_t pi(box_t state);
477   box_t result;
478   for (int x = 0 ; x < 5 ; x++) begin
479     for (int y = 0 ; y < 5 ; y++) begin
480       result[x][y][W-1:0] = state[PiRotate[x][y]][x][W-1:0];
481     end
482   end
483   return result;
484 endfunction : pi
485
486 // chi
487 // chi[x,y,z] = state[x,y,z] ^ ((state[x+1,y,z] ^ 1) &
state[x+2,y,z])
488 localparam int ChiIndexX1 [5] = '{1, 2, 3, 4, 0}; // (x+1)%5
489 localparam int ChiIndexX2 [5] = '{2, 3, 4, 0, 1}; // (x+2)%5
490 function automatic box_t chi(box_t state);
491   box_t result;
492   for (int x = 0 ; x < 5 ; x++) begin
493     result[x] = state[x] ^ ((~state[ChiIndexX1[x]]) &
state[ChiIndexX2[x]]);
494   end
495   return result;
496 endfunction : chi
497
498 // iota
499 // XOR (x,y) = (0,0) with Round Constant (RC)
500
501 // RC parameter: Precomputed by util/keccak_rc.py. Only up-
to 0..L-1 is used
502 // RC = '0
503 // RC[2*j-1] = rc(j+7*rnd)
504 // rc(t) =
505 // 1. t%255 == 0 -> 1
506 // 2. R[0:7] = 'b10000000
507 // 3. for i = [1..t%255]
508 //   a. R = 0 || R
509 //   b. R[0] = R[0] ^ R[8]
510 //   c. R[4] = R[4] ^ R[8]
511 //   d. R[5] = R[5] ^ R[8]
512 //   e. R[6] = R[6] ^ R[8]
513 //   f. R = R[0:7]

```



```

514 // 4. return R[0]
515 // RC has L = [0..6]

516 // for lower L case, only chopping lower part of 64bit RC is
sufficient.
517 localparam logic [63:0] RC [24] = '{
518     64'h 0000_0000_0000_0001, // Round 0
519     64'h 0000_0000_0000_8082, // Round 1
520     64'h 8000_0000_0000_808A, // Round 2
521     64'h 8000_0000_8000_8000, // Round 3
522     64'h 0000_0000_0000_808B, // Round 4
523     64'h 0000_0000_8000_0001, // Round 5
524     64'h 8000_0000_8000_8081, // Round 6
525     64'h 8000_0000_0000_8009, // Round 7
526     64'h 0000_0000_0000_008A, // Round 8
527     64'h 0000_0000_0000_0088, // Round 9
528     64'h 0000_0000_8000_8009, // Round 10
529     64'h 0000_0000_8000_000A, // Round 11
530     64'h 0000_0000_8000_808B, // Round 12
531     64'h 8000_0000_0000_008B, // Round 13
532     64'h 8000_0000_0000_8089, // Round 14
533     64'h 8000_0000_0000_8003, // Round 15
534     64'h 8000_0000_0000_8002, // Round 16
535     64'h 8000_0000_0000_0080, // Round 17
536     64'h 0000_0000_0000_800A, // Round 18
537     64'h 8000_0000_8000_000A, // Round 19
538     64'h 8000_0000_8000_8081, // Round 20
539     64'h 8000_0000_0000_8080, // Round 21
540     64'h 0000_0000_8000_0001, // Round 22
541     64'h 8000_0000_8000_8008 // Round 23
542 };
543
544 // iota: XOR with RC for (x,y) = (0,0)
545 function automatic box_t iota(box_t state, logic [RndW-1:0]
rnd);
546     box_t result;
547     result = state;
548     result[0][0][W-1:0] = state[0][0][W-1:0] ^ RC[rnd][W-1:0];
549
550     return result;
551 endfunction : iota
552
553 // Round function : Rnd(A,i_r)
554 // Not used due to rho function issue described above.
555
556 //function automatic box_t keccak_rnd(box_t state, logic
[RndW-1:0] rnd);
557 // box_t keccak_state;
558 // keccak_state = iota(chi(pi(rho(theta(state))))), rnd);
559 //
560 // return keccak_state;
561 //endfunction : keccak_rnd
562
563 endmodule

```

2. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_round.sv (611 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Keccak full round logic based on given input 'Width'
6 // e.g. Width 800 requires 22 rounds
7 //
8 // This module has two phases:
9 // 1. Compute Theta, Rho, Pi steps.
10 // 2. Computes Chi and Iota steps.
11 //
12 // If masking is not enabled, the two phases are completed
within a single
13 // clock cycle.
14 //
15 // If masking is enabled, the first phase (Phase1) completes
in one cycle.
16 // The second phase then needs three clock cycles to complete:
17 // 1. In the first clock cycle, the first stage of Chi is com-
puted for the
18 // first lane halves.

19 // 2. In the second clock cycle, the module writes the updated
first lane
20 // halves to the state.

21 // 3. In the third clock cycle, the new second lane halves are
written to
22 // to the state.
23 //
24 // To deter SCA, we randomly decide which lane halves to pro-
cess first on a
25 // per-round basis. We use additional randomness generated by
the PRNG to take
26 // this decision (rand_aux_i). For more details, refer to the
comments inside
27 // the FSM below.
28 //
29 // If masking is enabled, this implementation uses both ran-
domness provided

```

```

30 // from an external PRNG as well as intermediate results for
remasking the DOM
31 // multipliers below. Per clock cycle, 800b of pseudo-random
data (PRD) are
32 // required. A carefully designed schedule ensures to only
ever update the input
33 // data of the DOM multipliers when also providing fresh ran-
domness and vice
34 // versa. Updating one without the other could lead to un-
desired SCA leakage.
35
36 `include "prim_assert.sv"
37
38 module keccak_round
39     import prim_mubi_pkg::*;
40 # (
41     parameter int Width = 1600, // b= {25, 50, 100, 200, 400,
800, 1600}
42
43     // Derived
44     localparam int W      = Width/25,
45     localparam int L      = $clog2(W),
46     localparam int MaxRound = 12 + 2*L, // Keccak-f
only
47     localparam int RndW    = $clog2(MaxRound+1), // Represent-
ing up to MaxRound-1
48
49     // Feed parameters
50     parameter int DinWidth = 64, // currently only 64bit sup-
ported
51     localparam int DinEntry = Width / DinWidth,
52     localparam int DinAddr  = $clog2(DinEntry),
53
54     // Control parameters
55     parameter bit EnMasking = 1'b0, // Enable SCA harden-
ing, requires Width >= 50
56     parameter bit ForceRandExt = 1'b0, // 1: Always forward
externally provided randomness.
57     // 0: Switch between
external randomness and internal
58     // intermediate
state according to schedule.
59     localparam int Share     = EnMasking ? 2 : 1
60 ) (
61     input clk_i,
62     input rst_ni,
63
64     // Message Feed
65     input valid_i,
66     input [DinAddr-1:0] addr_i,
67     input [DinWidth-1:0] data_i [Share],
68     output ready_o,
69
70     // In-process control
71     input run_i, // Pulse signal to initi-
ates Keccak full round
72     input rand_valid_i,
73     input rand_early_i,
74     input [Width/2-1:0] rand_data_i,
75     input rand_aux_i,
76     output logic rand_update_o,
77     output logic rand_consumed_o,
78
79     output logic complete_o, // Indicates full round is
done
80
81     // State out. This can be used as Digest
82     output logic [Width-1:0] state_o [Share],
83
84     // Life cycle
85     input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
86
87     // Errors:
88     // sparse_fsm_error: Checking if FSM state falls into un-
known value
89     output logic sparse_fsm_error_o,
90     // round_count_error: prim_count checks round value con-
sistency
91     output logic round_count_error_o,
92
93     // rst_storage_error: check if reset signal asserted out of
the
94     // permitted window
95     output logic rst_storage_error_o,
96     input prim_mubi_pkg::mubi4_t clear_i // Clear internal
state to '0'
97 );
98
99 import sha3_pkg::*;
100
101 // Control signals //
102 ///////////////////////////////////////////////////
103 ///////////////////////////////////////////////////
104
105 // Update storage register
106 logic update_storage;
107
108 // Reset the storage to 0 to initiate new Hash operation
109 logic rst_storage;

```

```

110
111 // XOR message into storage register
112 // It only does based on the given DinWidth.
113 // If DinWidth < Width, it takes multiple cycles to XOR all
message
114   logic xor_message;
115
116 // Select Keccak_p datapath
117 // 0: Select Phase1 (Theta -> Rho -> Pi)
118 // 1: Select Phase2 (Chi -> Iota)
119 // 'phase_sel' needs to be asserted until the Chi stage is
consumed,
120   mubi4_t phase_sel;
121
122 // DOM multiplier input/output mux control
123 // 0: first compute upper, then lower lane halves
124 // 1: first compute lower, then upper lane halves
125   logic low_then_high_d, low_then_high_q;
126 // 0: drive/select upper lane halves
127 // 1: drive/select lower lane halves
128   logic dom_out_low_d, dom_out_low_q;
129   logic dom_in_low_d, dom_in_low_q;
130 // 0: forward external randomness input
131 // 1: forward partial intermediate results
132   logic dom_in_rand_ext_d, dom_in_rand_ext_q;
133 // 0: keep current intermediate results in pipeline re-
gisters
134 // 1: latch new intermediate results into pipeline registers
135   logic dom_update;
136
137 // Increase/ Reset Round number
138   logic inc_rnd_num;
139   logic rst_rnd_num;
140
141 // Round reaches end
142 // This signal indicates the round reaches desired number,
which is MaxRound -1.
143 // MaxRound is dependant on the Width. In case of SHA3/
SHAKE, MaxRound is 24.
144   logic rnd_eq_end;
145
146 // Complete of Keccak_f
147 // State machine asserts 'complete_d' when it reaches at the
end of round and
148 // operation (Phase3 if Masked). The stage, the storage
still doesn't have
149 // the valid states. So precisely it is not completed yet.
150 // State generated 'complete_d' is latched with the clock
and creates a pulse
151 // signal one cycle later. The signal is the indication of
completion.
152 //
153 // Intentionally removed any intermediate step (so called
StComplete) in order
154 // to save a clock to proceeds next round.
155   logic complete_d;
156
157 ///////////////////////////////////////////////////
158 // Datapath Signals //
159 ///////////////////////////////////////////////////
160
161 // Single round keccak output data
162   logic [width-1:0] keccak_out [Share];
163
164 // Keccak Round indicator: range from 0 .. MaxRound
165   logic [RndW-1:0] round;
166
167 // Random value and valid signal used in Keccak_p
168   logic keccak_rand_update;
169   logic keccak_rand_consumed;
170   logic [width/2-1:0] keccak_rand_data;
171
172 ///////////////////////////////////////////////////
173 // Keccak Round FSM //
174 ///////////////////////////////////////////////////
175
176 // state inputs
177   assign rnd_eq_end = (int'(round) == MaxRound - 1);
178
179   keccak_st_e keccak_st, keccak_st_d;
180   `PRIM_FLOP_SPARSE_FSM(u_state_regs, keccak_st_d, keccak_st,
keccak_st_e, KeccakStIdle)
181
182 // Next state logic and output logic
183 // SEC_CM: FSM.SPARSE
184   always_comb begin
185     // Default values
186     keccak_st_d = keccak_st;
187
188     xor_message = 1'b 0;
189     update_storage = 1'b 0;
190     rst_storage = 1'b 0;
191
192     inc_rnd_num = 1'b 0;
193     rst_rnd_num = 1'b 0;
194
195     keccak_rand_update = 1'b 0;
196     keccak_rand_consumed = 1'b 0;
197
198     phase_sel = MuBi4False;

```

```

199   low_then_high_d = low_then_high_q;
200   dom_in_low_d = dom_in_low_q;
201   dom_in_rand_ext_d = dom_in_rand_ext_q;
202   dom_update = 1'b 0;
203
204   complete_d = 1'b 0;
205
206   sparse_fsm_error_o = 1'b 0;
207
208   unique case (keccak_st)
209     KeccakStIdle: begin
210       if (valid_i) begin
211         // State machine allows Sponge Absorbing only in
Idle state.
212         keccak_st_d = KeccakStIdle;
213
214         xor_message = 1'b 1;
215         update_storage = 1'b 1;
216       end else if
(prim_mubi_pkg::mubi4_test_true_strict(clear_i)) begin
217         // Opt1. State machine allows resetting the storage
only in Idle
218         // Opt2. storage resets regardless of states but
clear_i
219         // Both are added in the design at this time. Will
choose the
220         // direction later.
221         keccak_st_d = KeccakStIdle;
222
223         rst_storage = 1'b 1;
224       end else if (EnMasking && run_i) begin
225         // Masked version of Keccak handling
226         keccak_st_d = KeccakStPhase1;
227
228         // Drive DOM multiplier I/O mux signals for Phase 1.
229         dom_in_low_d = low_then_high_q;
230         dom_in_rand_ext_d = 1'b 0;
231       end else if (!EnMasking && run_i) begin
232         // Unmasked version of Keccak handling
233         keccak_st_d = KeccakStActive;
234       end else begin
235         keccak_st_d = KeccakStIdle;
236       end
237     end
238
239     KeccakStActive: begin
240       // Run Keccak single round logic until it reaches
MaxRound - 1
241       update_storage = 1'b 1;
242
243       if (rnd_eq_end) begin
244         keccak_st_d = KeccakStIdle;
245
246         rst_rnd_num = 1'b 1;
247         complete_d = 1'b 1;
248       end else begin
249         keccak_st_d = KeccakStActive;
250
251         inc_rnd_num = 1'b 1;
252       end
253     end
254
255     KeccakStPhase1: begin
256       // Compute Theta, Rho, Pi - The DOM multipliers are
not evaluated at
257       // all: their inputs are driven by the first lane
halves (same values
258       // as in Phase2Cycle3 of the last round). Also, the
intermediate
259       // results we already had in Phase2Cycle3 didn't
change.
260       phase_sel = MuBi4False;
261       dom_update = 1'b 0;
262
263       // Only update the state and move on once we know the
auxiliary
264       // randomness required for Phase2 will be available in
the next clock
265       // cycle.
266       //
267       // It's important that the DOM multipliers inside kec-
cak_2share are
268       // presented the new state (updated with up-
date_storage) at the same
269       // time as the new randomness (updated with rand_up-
date_o). Otherwise,
270       // stale entropy is paired with fresh data or vice
versa. This could
271       // lead to undesired SCA leakage.
272       if (rand_early_i || rand_valid_i) begin
273         keccak_st_d = KeccakStPhase2Cycle1;
274         update_storage = 1'b 1;
275         keccak_rand_update = 1'b 1;
276
277         // Update lane halves processing order for this
round.
278         low_then_high_d = rand_aux_i;
279
280         // Drive DOM multiplier I/O mux signals for next
phase.

```

```

281     dom_in_low_d = low_then_high_d;
282     dom_in_rand_ext_d = 1'b 1;
283   end else begin
284     keccak_st_d = KeccakStPhase1;
285   end
286 end
287
288 KeccakStPhase2Cycle1: begin
289   // Compute first stage of Chi for first lane halves
using the DOM
290   // multipliers. Use the fresh randomness provided by
the PRNG for
291   // remasking.
292   phase_sel = MuBi4True;
293   dom_update = 1'b 1;
294
295   // Trigger randomness update for next cycle.
296   // It's important that the DOM multipliers inside kec-
cak_2share are
297   // presented the second lane halves at the same time
as the new
298   // randomness (updated with rand_update_o). Otherwise,
state entropy
299   // is paired with fresh data or vice versa. This could
lead to
300   // undesired SCA leakage.
301   keccak_rand_update = 1'b 1;
302
303   // Unconditionally move to next phase/cycle.
304   keccak_st_d = KeccakStPhase2Cycle2;
305
306   // Drive DOM multiplier I/O mux signals for next
phase.
307   dom_in_low_d = ~low_then_high_q;
308   dom_in_rand_ext_d = 1'b 1;
309 end
310
311 KeccakStPhase2Cycle2: begin
312   // Chi Stage 1 for second lane halves.
313   // Chi Stage 2 and Iota for first lane halves.
314
315   // Compute second stage of Chi and Iota for first lane
halves.
316   // Compute first stage of Chi for second lane halves.
Use the fresh
317   // randomness provided by the PRNG for remasking the
DOM multipliers.
318   phase_sel = MuBi4True;
319   dom_update = 1'b 1;
320
321   // Trigger randomness update for next cycle.
322   // It's important that the DOM multipliers inside kec-
cak_2share are
323   // presented the updated state at the same as the new
randomness
324   // (updated with rand_update_o) - even if the DOM mul-
tipliers don't
325   // update the pipeline registers in the next cycle.
Otherwise, stale
326   // entropy is paired with fresh data or vice versa.
This could lead to
327   // undesired SCA leakage.
328   keccak_rand_update = 1'b 1;
329
330   // Trigger auxiliary randomness update for next round.
The rand_aux_i
331   // signal is actually going to change in 2 clock
cycles from now
332
333   // (Phase1) based on the PRNG output in the next cycle
(Phase2Cycle3)
334   // in which the DOM multipliers don't update the
pipeline registers.
335   keccak_rand_consumed = 1'b 1;
336
337   // Update first lane halves.
338   update_storage = 1'b 1;
339
340   // Unconditionally move to next phase/cycle.
341   keccak_st_d = KeccakStPhase2Cycle3;
342
343   // Drive DOM multiplier I/O mux signals for next
phase.
344   dom_in_low_d = low_then_high_q;
345   dom_in_rand_ext_d = 1'b 0;
346 end
347
348 KeccakStPhase2Cycle3: begin
349   // Compute second stage of Chi and Iota for second
lane halves.
350   // Feed again first lane halves to DOM multiplier in-
puts (now
351   // the updated values become visible) together with
intermediate
352   // results of Phase2Cycle2. Don't update the register
stage inside
353   // the DOM multipliers.
354   phase_sel = MuBi4True;

```

```

353   dom_update = 1'b 0;
354
355   // Update second lane halves.
356   // We don't need fresh randomness for the next cycle
as the DOM
357   // multipliers inside keccak_2share will keep seeing
the first
358   // lane halves in the next cycle. If we updated the
randomness,
359   // old data got combined with fresh randomness which
is not
360   // desirable as it could lead to SCA leakage.
361   update_storage = 1'b 1;
362
363   if (rnd_eq_end) begin
364     // We're done.
365     keccak_st_d = KeccakStIdle;
366
367     rst_rnd_num = 1'b 1;
368     complete_d = 1'b 1;
369   end else begin
370     // Continue to the next round.
371     keccak_st_d = KeccakStPhase1;
372
373     inc_rnd_num = 1'b 1;
374
375     // Drive DOM multiplier I/O mux signals for next
phase.
376     dom_in_low_d = low_then_high_q;
377     dom_in_rand_ext_d = 1'b 0;
378   end
379 end
380
381 KeccakStError: begin
382   keccak_st_d = KeccakStError;
383 end
384
385 KeccakStTerminalError: begin
386   //this state is terminal
387   keccak_st_d = keccak_st;
388   sparse_fsm_error_o = 1'b 1;
389 end
390
391 default: begin
392   keccak_st_d = KeccakStTerminalError;
393   sparse_fsm_error_o = 1'b 1;
394 end
395 endcase
396
397 // SEC_CM: FSM.GLOBAL_ESC, FSM.LOCAL_ESC
398 // Unconditionally jump into the terminal error state
399 // if the life cycle controller triggers an escalation.
400 if (lc_ctrl_pkg::lc_tx_test_true_loose(lc_escalate_en_i))
begin
401   keccak_st_d = KeccakStTerminalError;
402 end
403 end
404
405 // When taking the lower lane halves in, the upper lane
halves are output and
406 // vice versa.
407 assign dom_out_low_d = ~dom_in_low_d;
408
409 if (EnMasking) begin : gen_regs_dom_ctrl
410   always_ff @(posedge clk_i or negedge rst_ni) begin
411     if (!rst_ni) begin
412       low_then_high_q <= 1'b 0;
413       dom_out_low_q <= 1'b 0;
414       dom_in_low_q <= 1'b 0;
415     end else begin
416       low_then_high_q <= low_then_high_d;
417       dom_out_low_q <= dom_out_low_d;
418       dom_in_low_q <= dom_in_low_d;
419     end
420   end
421 end
422
423 if (!ForceRandExt) begin : gen_reg_dom_in_rand_ext
424   always_ff @(posedge clk_i or negedge rst_ni) begin
425     if (!rst_ni) begin
426       dom_in_rand_ext_q <= 1'b 0;
427     end else begin
428       dom_in_rand_ext_q <= dom_in_rand_ext_d;
429     end
430   end
431 end else begin : gen_force_dom_in_rand_ext
432   // Always forward the externally provided randomness.
433   assign dom_in_rand_ext_q = 1'b 1;
434   // Tie off unused signals.
435   logic unused_dom_in_rand_ext;
436   assign unused_dom_in_rand_ext = dom_in_rand_ext_d;
437 end
438 end else begin : gen_no_regs_dom_ctrl
439   logic unused_dom_ctrl;
440   assign unused_dom_ctrl =
    ^{low_then_high_d, dom_out_low_d, dom_in_low_d,
dom_in_rand_ext_d};
441   assign low_then_high_q = 1'b 0;
442   assign dom_out_low_q = 1'b 0;
443   assign dom_in_low_q = 1'b 0;
444   assign dom_in_rand_ext_q = 1'b 0;
445 end

```

```

446
447 // Ready indicates the keccak_round is able to receive new
message.
448 // While keccak_round is processing the data, it blocks the
new message to be
449 // XORed into the current state.
450 assign ready_o = (keccak_st == KeccakStIdle) ? 1'b 1 : 1'b
0;
451
452 ///////////////////////////////////////////////////
453 // Keccak state registers //
454 ///////////////////////////////////////////////////
455
456 // SEC_CM: LOGIC_INTEGRITY
457 logic rst_n;
458 prim_sec_anchor_buf #(
459   .width(1)
460 ) u_prim_sec_anchor_buf (
461   .in_i(rst_n),
462   .out_o(rst_n)
463 );
464
465 logic [width-1:0] storage [Share];
466 logic [width-1:0] storage_d [Share];
467 always_ff @(posedge clk_i or negedge rst_n) begin
468   if (!rst_n) begin
469     storage <= '{default:'0};
470   end else if (rst_storage) begin
471     storage <= '{default:'0};
472   end else if (update_storage) begin
473     storage <= storage_d;
474   end
475 end
476
477 assign state_o = storage;
478
479 // Storage register input
480 // The incoming message is XORed with the existing storage
registers.
481 // The logic can accept not a block size incoming message
chunk but
482 // the size defined in `DInWidth` parameter with its posi-
tion.
483
484 always_comb begin
485   storage_d = keccak_out;
486   if (xor_message) begin
487     for (int j = 0 ; j < Share ; j++) begin
488       for (int unsigned i = 0 ; i < DInEntry ; i++) begin
489         // ICEBOX(#18029): handle If width is not integer
divisible by DInWidth
490         // Currently it is not allowed to have partial write
491         // Please see the Assertion `WidthDivisibleByDIn-
Width_A`
492         if (addr_i == i[DInAddr-1:0]) begin
493           storage_d[j][i*DInWidth+:DInWidth] =
494             storage[j][i*DInWidth+:DInWidth] ^ data_i[j];
495         end else begin
496           storage_d[j][i*DInWidth+:DInWidth] = storage[j]
[i*DInWidth+:DInWidth];
497         end
498       end // for i
499     end // for j
500   end // if xor_message
501 end
502
503 // Check the rst_storage integrity
504 logic rst_storage_error;
505
506 always_comb begin : chk_rst_storage
507   rst_storage_error = 1'b 0;
508
509   if (rst_storage) begin
510     // FSM should be in KeccakStIdle and clear_i should be
high
511     if ((keccak_st != KeccakStIdle) ||
512         prim_mubi_pkg::mubi4_test_false_loose(clear_i)) begin
513       rst_storage_error = 1'b 1;
514     end
515   end
516 end : chk_rst_storage
517
518 assign rst_storage_error_o = rst_storage_error ;
519
520 ///////////////////////////////////////////////////
521 // Datapath //
522 ///////////////////////////////////////////////////
523 keccak_2share #(
524   .width(width),
525   .enMasking(EnMasking),
526   .ForceRandExt(ForceRandExt)
527 ) u_keccak_p (
528   .clk_i,
529   .rst_ni,
530
531   .lc_escalate_en_i,
532
533   .rnd_i(round),
534
535   .phase_sel_i (phase_sel),
536   .dom_out_low_i (dom_out_low_q),

```

```

537   .dom_in_low_i (dom_in_low_q),
538   .dom_in_rand_ext_i(dom_in_rand_ext_q),
539   .dom_update_i (dom_update),
540
541   .rand_i(keccak_rand_data),
542
543   .s_i(storage),
544   .s_o(keccak_out)
545 );
546
547 // keccak entropy handling
548 assign rand_update_o = keccak_rand_update;
549 assign rand_consumed_o = keccak_rand_consumed;
550
551 assign keccak_rand_data = rand_data_i;
552
553 // Round number
554 // This primitive is used to place a hardened counter
555 // SEC_CM: CTR_REDUN
556 prim_count #(
557   .width(RndW)
558 ) u_round_count (
559   .clk_i,
560   .rst_ni,
561   .clr_i(rst_rnd_num),
562   .set_i(1'b0),
563   .set_cnt_i('0),
564   .incr_en_i(inc_rnd_num),
565   .decr_en_i(1'b0),
566   .step_i(RndW'(1)),
567   .commit_i(1'b1),
568   .cnt_o(round),
569   .cnt_after_commit_o(),
570   .err_o(round_count_error_o)
571 );
572
573 // completion signal
574 always_ff @(posedge clk_i or negedge rst_ni) begin
575   if (!rst_ni) begin
576     complete_o <= 1'b 0;
577   end else begin
578     complete_o <= complete_d;
579   end
580 end
581
582 ///////////////////////////////////////////////////
583 // Assertions //
584 ///////////////////////////////////////////////////
585
586 // Only allow `DInWidth` that `Width` is integer divisible
by `DInWidth`
587 `ASSERT_INIT(WidthDivisibleByDInWidth_A, (width % DInWidth)
== 0)
588
589 // If `run_i` triggered, it shall complete
590 // `ASSERT(RunResultComplete_A, run_i ##[MaxRound:] com-
plete_o, clk_i, !rst_ni)
591
592 // valid_i and run_i cannot be asserted at the same time
593 `ASSUME(OneHot0ValidAndRun_A, $onehot0({valid_i, run_i}),
clk_i, !rst_ni)
594
595 // valid_i, run_i only asserted in Idle state
596 `ASSUME(ValidRunAssertStIdle_A, valid_i || run_i -> kec-
cak_st == KeccakStIdle, clk_i, !rst_ni)
597
598 // clear_i is assumed to be asserted in Idle state
599 `ASSUME(ClearAssertStIdle_A,
600   prim_mubi_pkg::mubi4_test_true_strict(clear_i)
601   -> keccak_st == KeccakStIdle, clk_i, !rst_ni)
602
603 // EnMasking controls the valid states
604 if (EnMasking) begin : gen_mask_st_chk
605   `ASSERT(EnMaskingValidStates_A, keccak_st != KeccakStAct-
ive, clk_i, !rst_ni)
606 end else begin : gen_unmask_st_chk
607   `ASSERT(UnmaskValidStates_A, !(keccak_st
608     inside {KeccakStPhase1, KeccakStPhase2Cycle1, Keccak-
StPhase2Cycle2, KeccakStPhase2Cycle3}),
609     clk_i, !rst_ni)
610 end
611 endmodule

```

3. lowRISC_opentitan/hw/ip/kmac/rtl/kmac.sv (1,606 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // KMAC/SHA3
6
7 `include "prim_assert.sv"
8
9 module kmac
10   import kmac_pkg::*;
11   import kmac_reg_pkg::*;
12   #(
13     // EnMasking: Enable masking security hardening inside kec-

```

```

cak_round
14 // If it is enabled, the result digest will be two set of
1600bit.
15 parameter bit EnMasking = 1,
16
17 // In case EnMasking == 0, this defines whether SW can
provide a masked key or whether Share 1 of
18 // the SW key is simply ignored. In case EnMasking == 1,
this parameter has no meaning, always
19 // both shares of the key provided by SW are used.
20 // This is useful to allow both for area-optimized unmasked
designs as well as unmasked designs
21 // having a SW interface fully compatible with the masked
design.
22 parameter bit SwKeyMasked = 0,
23
24 // Command delay, useful for SCA measurements only. A value
of e.g. 40 allows the processor to go
25 // into sleep before KMAC starts operation. If a value > 0
is chosen, the processor can provide
26 // two commands subsequently and then go to sleep. The
second command is buffered internally and
27 // will be presented to the hardware SecCmdDelay number of
cycles after the first one.
28 parameter int SecCmdDelay = 0,
29
30 // Accept SW message when idle and before receiving a START
command. Useful for SCA only.
31 parameter bit SecIdleAcceptSwMsg = 1'b0,
32 parameter int unsigned NumAppIntf = 4,
33 parameter app_config_t AppCfg[NumAppIntf] = '{AppCfgKeyMgr,
AppCfgLlcCtrl,
34 AppCfgRomCtrl,
AppCfg0tbn},
35
36 parameter lfsr_perm_t RndCnstLfsrPerm = RndCnstLfsrPermDe-
fault,
37 parameter lfsr_seed_t RndCnstLfsrSeed = RndCnstLfsrSeedDe-
fault,
38 parameter buffer_lfsr_seed_t RndCnstBufferLfsrSeed = RndCn-
stBufferLfsrSeedDefault,
39 parameter msg_perm_t RndCnstMsgPerm = RndCnstMsgPermDe-
fault,
40
41 parameter logic [NumAlerts-1:0] AlertAsyncOn = {Nu-
mAlerts{1'b1}},
42 // Number of cycles a differential skew is tolerated on the
alert signal
43 parameter int unsigned AlertSkewCycles = 1
44 ) (
45 input clk_i,
46 input rst_ni,
47
48 input rst_shadowed_ni,
49
50 input clk_edn_i,
51 input rst_edn_ni,
52
53 input tlul_pkg::tl_h2d_t tl_i,
54 output tlul_pkg::tl_d2h_t tl_o,
55
56 // Alerts
57 input prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_rx_i,
58 output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
alert_tx_o,
59
60 // KeyMgr sideload (secret key) interface
61 input keymgr_pkg::hw_key_req_t keymgr_key_i,
62
63 // KeyMgr KDF data path
64 input app_req_t [NumAppIntf-1:0] app_i,
65 output app_rsp_t [NumAppIntf-1:0] app_o,
66
67 // EDN interface
68 output edn_pkg::edn_req_t entropy_o,
69 input edn_pkg::edn_rsp_t entropy_i,
70
71 // Life cycle
72 input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
73
74 // interrupts
75 output logic intr_kmac_done_o,
76 output logic intr_fifo_empty_o,
77 output logic intr_kmac_err_o,
78
79 // parameter consistency check with keymgr
80 output logic en_masking_o,
81
82 // Idle signal
83 output prim_mubi_pkg::mubi4_t idle_o
84 );
85
86 // Parameters
87 //
88 //
89 localparam int Share = (EnMasking) ? 2 : 1 ;
90 localparam int SwKeyShare = (EnMasking || SwKeyMasked) ? 2 : 1;
91

```

```

92 //
93 // Definitions //
94 //
95 // This state machine is to track the current process based
on SW input and
96 // KMAC operation.
97 // Encoding generated with:
98 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 6 -n 6 \
99 // -s 1966361510 --language=sv
100 //
101 // Hamming distance histogram:
102 //
103 // 0: --
104 // 1: --
105 // 2: --
106 // 3: ||||| (53.33%)
107 // 4: ||||| (40.00%)
108 // 5: || (6.67%)
109 // 6: --
110 //
111 // Minimum Hamming distance: 3
112 // Maximum Hamming distance: 5
113 // Minimum Hamming weight: 2
114 // Maximum Hamming weight: 5
115 //
116 localparam int StateWidth = 6;
117 typedef enum logic [StateWidth-1:0] {
118 // Idle state
119 KmacIdle = 6'b001011,
120
121 // When software writes CmdStart @ KmacIdle and kmac_en,
FSM moves to this
122 KmacPrefix = 6'b000110,
123
124 // When SHA3 engine processes Key block, FSM moves to
here.
125 KmacKeyBlock = 6'b111110,
126
127 // Message Feed
128 KmacMsgFeed = 6'b010101,
129
130 // Complete and squeeze
131 KmacDigest = 6'b101101,
132
133 // Error
134 KmacTerminalError = 6'b110000
135 } kmac_st_e;
136
137 kmac_st_e kmac_st, kmac_st_d;
138
139 //
140 // Signals //
141 //
142 //
143 kmac_reg2hw_t reg2hw;
144 kmac_hw2reg_t hw2reg;
145
146 // Window
147 typedef enum int {
148 WinState = 0,
149 WinMsgFifo = 1
150 } tl_window_e;
151
152 tlul_pkg::tl_h2d_t tl_win_h2d[2];
153 tlul_pkg::tl_d2h_t tl_win_d2h[2];
154
155 // SHA3 core control signals and its response.
156 // Sequence: start --> process(multiple) --> get absorbed
event --> {run -->} done
157 logic sha3_start, sha3_run, sha3_squeezing;
158 prim_mubi_pkg::mubi4_t sha3_done;
159 prim_mubi_pkg::mubi4_t sha3_done_d;
160 prim_mubi_pkg::mubi4_t sha3_absorbed;
161
162 // Indicate one block processed
163 logic sha3_block_processed;
164
165 // EStatus for entropy
166 logic entropy_in_keyblock;
167
168 // Application interface logic generates absorbed from
sha3_absorbed.
169 // It is active only if SW initiates the hashing engine.
170 prim_mubi_pkg::mubi4_t app_absorbed;
171 logic event_absorbed;
172
173 sha3_pkg::sha3_st_e sha3_fsm;
174
175 // Prefix: kmac_pkg defines Prefix based on N size and S
size.
176 // Then computes left_encode(len(N)) size and left_en-
code(len(S))
177 // For given default value 32, 256 bits, the max
178 // encode_string(N) || encode_string(S) is 328. So 11 Prefix
registers are
179 // created.
180 logic [sha3_pkg::NSRegisterSize*8-1:0] reg_ns_prefix;
181 logic [sha3_pkg::NSRegisterSize*8-1:0] ns_prefix;
182
183 // NumWordsPrefix from kmac_reg_pkg

```



```

184 `ASSERT_INIT(PrefixRegSameToPrefixPkg_A,
185               kmac_reg_pkg::NumWordsPrefix*4 == sha3_pkg::NS-
RegisterSize)
186
187 // NumEntriesMsgFifo from kmac_reg_pkg must match calculated
MsgFifoDepth
188 // from kmac_pkg.
189 `ASSERT_INIT(NumEntriesRegSameToNumEntriesPkg_A,
190             kmac_reg_pkg::NumEntriesMsgFifo ==
kmac_pkg::MsgFifoDepth)
191
192 // NumBytesMsgFifoEntry from kmac_reg_pkg must match the
MsgWidth calculated
193 // in kmac_pkg (although MsgWidth is in bits, so we multiply
by 8).
194 `ASSERT_INIT(EntrySizeRegSameToEntrySizePkg_A,
195             kmac_reg_pkg::NumBytesMsgFifoEntry * 8 ==
kmac_pkg::MsgWidth)
196
197 // Output state: this is used to redirect the digest to
KeyMgr or Software
198 // depends on the configuration.
199 logic state_valid;
200 logic [sha3_pkg::StateW-1:0] state [Share];
201
202 // state is de-muxed in keymgr interface logic.
203 // the output from keymgr logic goes into staterd module to
be visible to SW
204 logic reg_state_valid;
205 logic [sha3_pkg::StateW-1:0] reg_state [Share];
206
207 // SHA3 Entropy interface
208 logic sha3_rand_valid, sha3_rand_early, sha3_rand_update,
sha3_rand_consumed;
209 logic [sha3_pkg::StateW/2-1:0] sha3_rand_data;
210 logic sha3_rand_aux;
211
212 // FIFO related signals
213 logic msgfifo_empty, msgfifo_full;
214 logic [kmac_pkg::MsgFifoDepthW-1:0] msgfifo_depth;
215
216 logic msgfifo_valid ;
217 logic [kmac_pkg::MsgWidth-1:0] msgfifo_data [Share];
218 logic [kmac_pkg::MsgStrbW-1:0] msgfifo_strb ;
219 logic msgfifo_ready ;
220
221 // TL-UL Adapter(MSG_FIFO) signals
222 logic tlram_req;
223 logic tlram_gnt;
224 logic tlram_we;
225 logic [8:0] tlram_addr; // NOT_READ
226 logic [31:0] tlram_wdata;
227 logic [31:0] tlram_wmask;
228 logic [31:0] tlram_rdata;
229 logic tlram_rvalid;
230 logic [1:0] tlram_rerror;
231 logic [31:0] tlram_wdata_endian;
232 logic [31:0] tlram_wmask_endian;
233
234 logic sw_msg_valid;
235 logic [kmac_pkg::MsgWidth-1:0] sw_msg_data;
236 logic [kmac_pkg::MsgWidth-1:0] sw_msg_strb;
237 logic sw_msg_ready;
238
239 // KeyMgr interface to MSG_FIFO
240 logic mux2fifo_valid;
241 logic [kmac_pkg::MsgWidth-1:0] mux2fifo_data[Share];
242 logic [kmac_pkg::MsgWidth-1:0] mux2fifo_strb;
243 logic mux2fifo_ready;
244 logic mux2fifo_bypass;
245
246 // KMAC to SHA3 core
247 logic msg_valid ;
248 logic [kmac_pkg::MsgWidth-1:0] msg_data [Share];
249 logic [kmac_pkg::MsgWidth-1:0] msg_data_masked [Share];
250 logic [kmac_pkg::MsgStrbW-1:0] msg_strb ;
251 logic msg_ready ;
252
253 // Process control signals
254 // Process pulse propagates from register to SHA3 engine one
by one.
255 // Each module (MSG_FIFO, KMAC core, SHA3 core) generates
the process pulse
256 // after flushing internal data to the next module.
257 logic reg2msgfifo_process, msgfifo2kmac_process,
kmac2sha3_process;
258
259 // Secret Key signals
260 logic [MaxKeyLen-1:0] sw_key_data_reg [SwKeyShare];
261 logic [MaxKeyLen-1:0] sw_key_data [Share];
262 key_len_e sw_key_len;
263 logic [MaxKeyLen-1:0] key_data [Share];
264 key_valid key_valid;
265 key_len_e key_len;
266
267 // SHA3 Mode, Strength, KMAC enable for app interface
268 logic reg_kmac_en, app_k-
mac_en;
269 sha3_pkg::sha3_mode_e reg_sha3_mode,
app_sha3_mode;
270 sha3_pkg::keccak_strength_e reg_keccak_strength, app_kec-
cak_strength;
271
272 // RegIF of enabling unsupported mode & strength
273 logic cfg_en_unsupported_modestrength;
274
275 // Indicating AppIntf is active. This signal is used to
check SW error
276 logic app_active;
277
278 // SEC_CM: SW_CMD.CTRL.SPARSE
279 // Command
280 // sw_cmd is the command written by SW
281 // checked_sw_cmd is checked in the kmac_errchk module.
282 // Invalid command is filtered out in the module.
283 // kmac_cmd is generated in KeyMgr interface.
284 // If SW initiates the KMAC/SHA3, kmac_cmd represents SW
command,
285 // If KeyMgr drives the data, kmac_cmd is controlled in the
state machine
286 // in KeyMgr interface logic.
287 kmac_cmd_e sw_cmd, checked_sw_cmd, kmac_cmd, cmd_q;
288 logic cmd_update;
289
290 // Entropy configurations
291 logic [9:0] wait_timer_prescaler;
292 logic [15:0] wait_timer_limit;
293 logic entropy_refresh_req;
294 logic entropy_seed_update;
295 logic [31:0] entropy_seed_data;
296
297 logic [HashCntW-1:0] entropy_hash_threshold;
298 logic [HashCntW-1:0] entropy_hash_cnt;
299 logic entropy_hash_clr;
300
301 logic entropy_ready;
302 entropy_mode_e entropy_mode;
303 logic reg_entropy_fast_process;
304 logic entropy_fast_process;
305
306 prim_mubi_pkg::mubi4_t entropy_configured;
307
308 // Message Masking
309 logic msg_mask_en, cfg_msg_mask;
310 logic [MsgWidth-1:0] msg_mask;
311
312 // SHA3 Error response
313 sha3_pkg::err_t sha3_err;
314
315 // KeyMgr Error response
316 kmac_pkg::err_t app_err;
317
318 // Entropy Generator Error
319 kmac_pkg::err_t entropy_err;
320
321 // Error checker
322 kmac_pkg::err_t errchecker_err;
323
324 // MsgFIFO Error
325 kmac_pkg::err_t msgfifo_err;
326
327 logic err_processed;
328
329 prim_mubi_pkg::mubi4_t clear_after_error;
330
331 logic alert_fatal, alert_recov_operation;
332 logic alert_intg_err;
333
334 // Life cycle
335 localparam int unsigned NumLcSyncCopies = 6;
336 lc_ctrl_pkg::lc_tx_t [NumLcSyncCopies-1:0] lc_escal-
ate_en_sync;
337 lc_ctrl_pkg::lc_tx_t [NumLcSyncCopies-1:0] lc_escalate_en;
338
339 // Connecting Register IF to logics //
340 ////////////////////////////////////////////////////
341 ////////////////////////////////////////////////////
342
343 // Function-name N and Customization input string S
always_comb begin
344 for (int i = 0 ; i < NumWordsPrefix; i++) begin
345 reg_ns_prefix[32*i+32] = reg2hw.prefix[i].q;
346 end
347 end
348
349 // Create a lint error to reduce the risk of accidentally
enabling this feature.
350 `ASSERT_STATIC_LINT_ERROR(KmacSecCmdDelayNonDefault, SecCmd-
Delay == 0)
351
352 if (SecCmdDelay > 0) begin : gen_cmd_delay_buf
353 // Delay and buffer commands for SCA measurements.
354 localparam int unsigned WidthCounter = $cloc-
g2(SecCmdDelay+1);
355 logic [WidthCounter-1:0] count_d, count_q;
356 logic counting_d, counting_q;
357 logic cmd_buf_empty;
358 kmac_cmd_e cmd_buf_q;

```

```

361
362 assign cmd_buf_empty = (cmd_buf_q == CmdNone);
363
364 // When seeing a write to the cmd register, we start
counting. We stop counting once the
365 // counter has expired and the command buffer is empty.
366 assign counting_d = reg2hw.cmd.cmd.qe ? 1'b1 :
367 cmd_update & cmd_buf_empty ? 1'b0 :
counting_q;
368
369 // Clear counter upon writes to the cmd register or if the
specified delay is reached.
370 assign count_d = reg2hw.cmd.cmd.qe ? '0 :
371 cmd_update ? '0 :
372 counting_q ? count_q + 1'b1 :
count_q;
373
374 // The manual run command cannot be delayed. Software ex-
pects this to be triggered immediately
375 // and will poll the status register to wait for the SHA3
engine to return back to the squeeze
376 // state.
377 assign cmd_update = (cmd_q == CmdManual-
Run) ? 1'b1 :
378 (count_q == SecCmd-
Delay[WidthCounter-1:0]) ? 1'b1 : 1'b0;
379
380 always_ff @(posedge clk_i or negedge rst_ni) begin
381 if (!rst_ni) begin
382 count_q <= '0;
383 counting_q <= 1'b0;
384 end else begin
385 count_q <= count_d;
386 counting_q <= counting_d;
387 end
388 end
389
390 // cmd.q is valid while cmd.qe is high, meaning it needs
to be registered. We buffer one
391 // additional command such that software can write START
followed by PROCESS and then go to
392 // sleep.
393 always_ff @(posedge clk_i or negedge rst_ni) begin
394 if (!rst_ni) begin
395 cmd_q <= CmdNone;
396 cmd_buf_q <= CmdNone;
397 end else begin
398 if (reg2hw.cmd.cmd.qe && cmd_update) begin
399 // New write & counter expired.
400 cmd_q <= cmd_buf_q;
401 cmd_buf_q <= kmac_cmd_e'(reg2hw.cmd.cmd.q);
402
403 end else if (reg2hw.cmd.cmd.qe) begin
404 // New write.
405 if (counting_q == 1'b0) begin
406 cmd_q <= kmac_cmd_e'(reg2hw.cmd.cmd.q);
407 end else begin
408 cmd_buf_q <= kmac_cmd_e'(reg2hw.cmd.cmd.q);
409 end
410
411 end else if (cmd_update) begin
412 // Counter expired.
413 cmd_q <= cmd_buf_q;
414 cmd_buf_q <= CmdNone;
415 end
416 end
417 end
418
419 end else begin : gen_no_cmd_delay_buf
420 // Directly forward signals from register IF.
421 assign cmd_update = reg2hw.cmd.cmd.qe;
422 assign cmd_q = kmac_cmd_e'(reg2hw.cmd.cmd.q);
423 end
424
425 // Command signals
426 assign sw_cmd = (cmd_update) ? cmd_q : CmdNone;
427 `ASSERT_KNOWN(KmacCmd_A, sw_cmd)
428 always_comb begin
429 sha3_start = 1'b 0;
430 sha3_run = 1'b 0;
431 sha3_done_d = prim_mubi_pkg::MuBi4False;
432 reg2msgfifo_process = 1'b 0;
433
434 unique case (kmac_cmd)
435 CmdStart: begin
436 sha3_start = 1'b 1;
437 end
438
439 CmdProcess: begin
440 reg2msgfifo_process = 1'b 1;
441 end
442
443 CmdManualRun: begin
444 sha3_run = 1'b 1;
445 end
446
447 CmdDone: begin
448 sha3_done_d = prim_mubi_pkg::MuBi4True;
449 end
450

```

```

451 CmdNone: begin
452 // inactive state
453 end
454
455 default: begin
456 end
457 endcase
458 end
459
460 // Status register
=====
461 // status.squeeze is valid only when SHA3 engine completes
the Absorb and not
462 // running the manual keccak rounds. This status is for SW
to determine when
463 // to read the STATE values.
464 assign hw2reg.status.sha3_idle.d = sha3_fsm ==
sha3_pkg::StIdle;
465 assign hw2reg.status.sha3_absorb.d = sha3_fsm ==
sha3_pkg::StAbsorb;
466 assign hw2reg.status.sha3_squeeze.d = sha3_fsm ==
sha3_pkg::StSqueeze;
467
468 // FIFO related status
469 assign hw2reg.status.fifo_depth.d[MsgFifoDepthW-1:0] =
msgfifo_depth;
470 if ($bits(hw2reg.status.fifo_depth.d) != MsgFifoDepthW) be-
gin : gen_fifo_depth_tie
471 assign
hw2reg.status.fifo_depth.d[$bits(hw2reg.status.fifo_depth.d)-1:MsgF-
ifoDepthW] = '0;
472 end
473 assign hw2reg.status.fifo_empty.d = msgfifo_empty;
474 assign hw2reg.status.fifo_full.d = msgfifo_full;
475
476 // Configuration Register
477 logic engine_stable;
478 assign engine_stable = sha3_fsm == sha3_pkg::StIdle;
479
480 // SEC_CM: CFG_SHADOWED.CONFIG.REGWEN
481 assign hw2reg.cfg_regwen.d = engine_stable;
482
483 // Secret Key
484 // Secret key is defined as external register. So the logic
latches when SW
485 // writes to KEY_SHARE0 , KEY_SHARE1 registers.
486 // SEC_CM: SW_KEY.KEY_MASKING
487 always_ff @(posedge clk_i or negedge rst_ni) begin
488 if (!rst_ni) begin
489 sw_key_data_reg[0] <= '0;
490 end else if (engine_stable) begin
491 for (int j = 0 ; j < MaxKeyLen/32 ; j++) begin
492 if (reg2hw.key_share0[j].qe) begin
493 sw_key_data_reg[0][32*j+:32] <=
reg2hw.key_share0[j].q;
494 end
495 end // for j
496 end // else if engine_stable
497 end // always_ff
498
499 if (EnMasking || SwKeyMasked) begin : gen_key_share1_reg
500 always_ff @(posedge clk_i or negedge rst_ni) begin
501 if (!rst_ni) begin
502 sw_key_data_reg[1] <= '0;
503 end else if (engine_stable) begin
504 for (int j = 0 ; j < MaxKeyLen/32 ; j++) begin
505 if (reg2hw.key_share1[j].qe) begin
506 sw_key_data_reg[1][32*j+:32] <=
reg2hw.key_share1[j].q;
507 end
508 end // for j
509 end // else if engine_stable
510 end // always_ff
511 end else begin : gen_no_key_share1_reg
512 logic unused_key_share1;
513 assign unused_key_share1 = ^reg2hw.key_share1;
514 end
515
516 if (EnMasking || !SwKeyMasked) begin : gen_key_forward
517 // Forward all available key shares as is.
518 assign sw_key_data = sw_key_data_reg;
519 end else begin : gen_key_unmask
520 // Masking is disabled but the SW still provides the key
in two shares.
521 // Unmask the key for processing.
522 assign sw_key_data[0] = sw_key_data_reg[0] ^
sw_key_data_reg[1];
523 end
524
525 assign sw_key_len = key_len_e'(reg2hw.key_len.q);
526
527 // Entropy configurations
528 assign wait_timer_prescaler =
reg2hw.entropy_period.prescaler.q;
529 assign wait_timer_limit =
reg2hw.entropy_period.wait_timer.q;
530 assign entropy_refresh_req = reg2hw.cmd.entropy_req.q
&& reg2hw.cmd.entropy_req.qe;
531 assign entropy_seed_update = reg2hw.entropy_seed.qe;
532 assign entropy_seed_data = reg2hw.entropy_seed.q;
533

```

```

535 assign entropy_hash_threshold =
reg2hw.entropy_refresh_threshold_shadowed.q;
536 assign hw2reg.entropy_refresh_hash_cnt.de = 1'b 1;
537 assign hw2reg.entropy_refresh_hash_cnt.d = en-
tropy_hash_cnt;
538
539 assign entropy_hash_clr = reg2hw.cmd.hash_cnt_clr.qe
&& reg2hw.cmd.hash_cnt_clr.q;
540
541 // Entropy config
542 assign entropy_ready = reg2hw.cfg_shadowed.entropy_ready.q
&& reg2hw.cfg_shadowed.entropy_ready.qe;
543
544 assign entropy_mode = en-
tropy_mode_e'(reg2hw.cfg_shadowed.entropy_mode.q);
545 assign reg_entropy_fast_process =
reg2hw.cfg_shadowed.entropy_fast_process.q;
546
547 // msg_mask_en turns on the message LFSR when KMAC is en-
abled.
548 assign cfg_msg_mask = reg2hw.cfg_shadowed.msg_mask.q;
549 assign msg_mask_en = cfg_msg_mask & msg_valid & msg_ready;
550
551 // Enable unsupported mode & strength combination
552 assign cfg_en_unsupported_modestrength =
reg2hw.cfg_shadowed.en_unsupported_modestrength.q;
553
554 `ASSERT(EntropyReadyLatched_A, $rose(entropy_ready) |> !en-
tropy_ready)
555
556 // Idle control (registered output)
557 // The logic checks idle of SHA3 engine, MSG_FIFO,
KMAC_CORE, KEYMGR interface
558 always_ff @(posedge clk_i or negedge rst_ni) begin
559   if (!rst_ni) begin
560     idle_o <= prim_mubi_pkg::MuBi4True;
561   end else if ((sha3_fsm == sha3_pkg::StIdle) &&
(msgfifo_empty || SecIdleAcceptSwMsg)) begin
562     idle_o <= prim_mubi_pkg::MuBi4True;
563   end else begin
564     idle_o <= prim_mubi_pkg::MuBi4False;
565   end
566 end
567
568 // Clear the error processed
569 assign err_processed = reg2hw.cmd.err_processed.q &
reg2hw.cmd.err_processed.qe;
570
571 // Make sure the field has latch in reg_top
572 `ASSERT(ErrProcessedLatched_A, $rose(err_processed) |> !
err_processed)
573
574 // App mode, strength, kmac_en
575 assign reg_kmac_en = reg2hw.cfg_shadowed.kmac_en.q;
576 assign reg_sha3_mode =
sha3_pkg::sha3_mode_e'(reg2hw.cfg_shadowed.mode.q);
577 assign reg_keccak_strength = sha3_pkg::kec-
cak_strength_e'(reg2hw.cfg_shadowed.kstrength.q);
578
579 // Interrupt //
580
581 // Hash process absorbed interrupt
582 // Convert mubi4_t to logic to generate interrupts
583 assign event_absorbed =
prim_mubi_pkg::mubi4_test_true_strict(app_absorbed);
584
585 prim_intr_hw #(Width(1)) intr_kmac_done (
586   .clk_i,
587   .rst_ni,
588   .event_intr_i (event_absorbed),
589   .reg2hw_intr_enable_q_i (reg2hw.intr_enable.kmac_done.q),
590   .reg2hw_intr_test_q_i (reg2hw.intr_test.kmac_done.q),
591   .reg2hw_intr_test_qe_i (reg2hw.intr_test.kmac_done.qe),
592   .reg2hw_intr_state_q_i (reg2hw.intr_state.kmac_done.q),
593   .hw2reg_intr_state_de_o (hw2reg.intr_state.kmac_done.de),
594   .hw2reg_intr_state_d_o (hw2reg.intr_state.kmac_done.d),
595   .intr_o (intr_kmac_done_o)
596 );
597
598 `ASSERT(Sha3AbsorbedPulse_A,
$rose(prim_mubi_pkg::mubi4_test_true_strict(sha3_absorbed)) |>
prim_mubi_pkg::mubi4_test_false_strict(sha3_absorbed))
599
600 // Message FIFO empty interrupt
601 //
602 // The message FIFO empty interrupt is **not useful** for
software if:
603 // - One of the hardware application interfaces is actively
using the KMAC block. In this case
604 // the message FIFO is managed entirely by the application
interface.
605 // - The SHA3 core is not in the Absorb state. Only in this
state, the FIFO is writeable by
606 // software anyway.
607 // - Software has already written the Process command. The
KMAC block will now empty the
608 // message FIFO and load its content into the SHA3 core,
add the padding and then perform

```

```

615 // the final absorption. Software cannot append the mes-
sage further.
616 //
617 // The message FIFO empty interrupt can be **useful** for
software in particular if:
618 // - The message FIFO was completely full previously. How-
ever, unless the KMAC block is currently
619 // processing a block or waiting for fresh entropy from
EDN, it always empties the message FIFO
620 // faster than software can fill it up, meaning the mes-
sage FIFO is empty most of the time.
621 // Note, the empty status is signaled only once after the
FIFO was completely full. The FIFO
622 // needs to be full again for the empty status to be
signaled again next time it's empty.
623 //
624 // For further details see also:
625 // https://opentitan.org/book/hw/ip/kmac/doc/the-
ory_of_operation.html#fifo-depth-and-empty-status
626 logic status_msgfifo_empty, msgfifo_empty_gate;
627 logic msgfifo_empty_negedge, msgfifo_empty_q;
628 logic msgfifo_full_seen_d, msgfifo_full_seen_q;
629 assign msgfifo_empty_negedge = msgfifo_empty_q &
~msgfifo_empty;
630
631 // Track whether the message FIFO was full after being
empty. We clear the tracking:
632 // - When receiving the Process command. This is to start
over for the next message.
633 // - When seeing a negative edge on the empty signal. This
signals that software has reacted to
634 // the interrupt and is filling up the FIFO again.
635 assign msgfifo_full_seen_d =
636   msgfifo_full ? 1'b 1 :
637   msgfifo_empty_negedge ? 1'b 0 :
638   msgfifo2kmac_process ? 1'b 0 : msgfifo_full_seen_q;
639
640 // The interrupt is gated unless software is performing an
absorption operation (but not the
641 // final block) and the FIFO was full before. The msgfifo2k-
mac_process pulse is arriving from the
642 // FIFO together with the empty signal.
643 assign msgfifo_empty_gate =
644   app_active ? 1'b 1 :
645   sha3_fsm != sha3_pkg::StAbsorb ? 1'b 1 :
646   msgfifo2kmac_process ? 1'b 1 :
~msgfifo_full_seen_q;
647
648 assign status_msgfifo_empty = msgfifo_empty_gate ? 1'b 0 :
msgfifo_empty;
649
650 always_ff @(posedge clk_i or negedge rst_ni) begin
651   if (!rst_ni) begin
652     msgfifo_empty_q <= 1'b 0;
653     msgfifo_full_seen_q <= 1'b 0;
654   end else begin
655     msgfifo_empty_q <= msgfifo_empty;
656     msgfifo_full_seen_q <= msgfifo_full_seen_d;
657   end
658 end
659
660 prim_intr_hw #(
661   .Width(1),
662   .IntrT("Status")
663 ) intr_fifo_empty (
664   .clk_i,
665   .rst_ni,
666   .event_intr_i (status_msgfifo_empty),
667   .reg2hw_intr_enable_q_i (reg2hw.intr_enable.fifo_empty.q),
668   .reg2hw_intr_test_q_i (reg2hw.intr_test.fifo_empty.q),
669   .reg2hw_intr_test_qe_i (reg2hw.intr_test.fifo_empty.qe),
670   .reg2hw_intr_state_q_i (reg2hw.intr_state.fifo_empty.q),
671   .hw2reg_intr_state_de_o (hw2reg.intr_state.fifo_empty.de),
672   .hw2reg_intr_state_d_o (hw2reg.intr_state.fifo_empty.d),
673   .intr_o (intr_fifo_empty_o)
674 );
675
676 // Error
677
678 logic event_error;
679 assign event_error = sha3_err.valid | app_err.valid
| entropy_err.valid | errcheck-
er_err.valid
680
681 //
682 // Assigning error code to the register
683 assign hw2reg.err_code.de = event_error;
684
685 always_comb begin
686   hw2reg.err_code.d = '0;
687
688   priority case (1'b 1)
689     // app_err has the highest priority. If SW issues an in-
correct command
690     // while app is in active state, the error from AppIntf
is passed
691     // through.
692     app_err.valid: begin
693       hw2reg.err_code.d = {app_err.code, app_err.info};
694     end
695   end
696

```



```

697     errchecker_err.valid: begin
698         hw2reg.err_code.d = {errchecker_err.code , errcheck-
err_err.info};
699     end
700
701     sha3_err.valid: begin
702         hw2reg.err_code.d = {sha3_err.code , sha3_err.info};
703     end
704
705     entropy_err.valid: begin
706         hw2reg.err_code.d = {entropy_err.code, en-
tropy_err.info};
707     end
708
709     msgfifo_err.valid: begin
710         hw2reg.err_code.d = {msgfifo_err.code,
msgfifo_err.info};
711     end
712
713     default: begin
714         hw2reg.err_code.d = '0';
715     end
716 endcase
717 end
718
719 // Counter errors
720 logic counter_error, sha3_count_error, key_index_error;
721 logic msgfifo_counter_error;
722 logic kmac_entropy_hash_counter_error;
723 logic kmac_app_counter_error;
724 assign counter_error = sha3_count_error
725 | kmac_entropy_hash_counter_error
726 | key_index_error
727 | msgfifo_counter_error
728 | kmac_app_counter_error;
729
730 assign msgfifo_counter_error = msgfifo_err.valid;
731
732 // State Errors
733 logic sparse_fsm_error;
734 logic sha3_state_error, kmac_errchk_state_error;
735 logic kmac_core_state_error, kmac_app_state_error;
736 logic kmac_entropy_state_error, kmac_state_error;
737 assign sparse_fsm_error = sha3_state_error
738 | kmac_errchk_state_error
739 | kmac_core_state_error
740 | kmac_app_state_error
741 | kmac_entropy_state_error
742 | kmac_state_error;
743
744 // Control Signal Integrity Errors
745 logic control_integrity_error;
746 logic sha3_storage_rst_error;
747 assign control_integrity_error = sha3_storage_rst_error;
748
749 prim_intr_hw #(Width(1)) intr_kmac_err (
750     .clk_i,
751     .rst_ni,
752     .event_intr_i (event_error),
753     .reg2hw_intr_enable_q_i (reg2hw.intr_enable.kmac_err.q),
754     .reg2hw_intr_test_q_i (reg2hw.intr_test.kmac_err.q),
755     .reg2hw_intr_test_qe_i (reg2hw.intr_test.kmac_err.qe),
756     .reg2hw_intr_state_q_i (reg2hw.intr_state.kmac_err.q),
757     .hw2reg_intr_state_de_o (hw2reg.intr_state.kmac_err.de),
758     .hw2reg_intr_state_d_o (hw2reg.intr_state.kmac_err.d),
759     .intr_o (intr_kmac_err_o)
760 );
761
762 //////////////////////////////////////////////////
763 // State Machine //
764 //////////////////////////////////////////////////
765
766 // State FF
767 `PRIM_FLOP_SPARSE_FSM(u_state_regs, kmac_st_d, kmac_st,
kmac_st_e, KmacIdle)
768
769 always_comb begin
770     // Default value
771     kmac_st_d = kmac_st;
772
773     entropy_in_keyblock = 1'b 0;
774     kmac_state_error = 1'b 0;
775
776     unique case (kmac_st)
777         KmacIdle: begin
778             if (kmac_cmd == CmdStart) begin
779                 // If cSHAKE turned on
780                 if (sha3_pkg::cShake == app_sha3_mode) begin
781                     kmac_st_d = KmacPrefix;
782                 end else begin
783                     // Jump to Msg feed directly
784                     kmac_st_d = KmacMsgFeed;
785                 end
786             end else begin
787                 kmac_st_d = KmacIdle;
788             end
789         end
790
791         KmacPrefix: begin
792             // Wait until SHA3 processes one block
793             if (sha3_block_processed) begin

```

```

794         kmac_st_d = (app_kmac_en) ? KmacKeyBlock :
KmacMsgFeed ;
795     end else begin
796         kmac_st_d = KmacPrefix;
797     end
798 end
799
800 KmacKeyBlock: begin
801     entropy_in_keyblock = 1'b 1;
802     if (sha3_block_processed) begin
803         kmac_st_d = KmacMsgFeed;
804     end else begin
805         kmac_st_d = KmacKeyBlock;
806     end
807 end
808
809 KmacMsgFeed: begin
810     // If absorbed, move to Digest
811     if
812         (prim_mubi_pkg::mubi4_test_true_strict(sha3_absorbed) &&
prim_mubi_pkg::mubi4_test_true_strict(sha3_done))
813     begin
814         // absorbed and done can be asserted at a cycle if
Applications have
815         // requested the hash operation. kmac_app FSM issues
CmdDone command
816         // if it receives absorbed signal.
817         kmac_st_d = KmacIdle;
818     end else if
819         (prim_mubi_pkg::mubi4_test_true_strict(sha3_absorbed) &&
prim_mubi_pkg::mubi4_test_false_loose(sha3_done))
820     begin
821         kmac_st_d = KmacDigest;
822     end else begin
823         kmac_st_d = KmacMsgFeed;
824     end
825 end
826
827 KmacDigest: begin
828     // SW can manually run it, wait till done
829     if (prim_mubi_pkg::mubi4_test_true_strict(sha3_done))
830     begin
831         kmac_st_d = KmacIdle;
832     end else begin
833         kmac_st_d = KmacDigest;
834     end
835 end
836
837 KmacTerminalError: begin
838     //this state is terminal
839     kmac_st_d = KmacTerminalError;
840     kmac_state_error = 1'b 1;
841 end
842
843 default: begin
844     kmac_st_d = KmacTerminalError;
845     kmac_state_error = 1'b 1;
846 end
847 endcase
848
849 // SEC_CM: FSM.GLOBAL_ESC, FSM.LOCAL_ESC
850 // Unconditionally jump into the terminal error state
851 // if the life cycle controller triggers an escalation.
852 if (lc_ctrl_pkg::lc_tx_test_true_loose(lc_escalate_en[0]))
853 begin
854     kmac_st_d = KmacTerminalError;
855     end
856 end
857 `ASSERT_KNOWN(KmacStKnown_A, kmac_st)
858
859 //////////////////////////////////////////////////
860 // Instances //
861 //////////////////////////////////////////////////
862
863 // KMAC core
864 kmac_core #(
865     .EnMasking (EnMasking)
866 ) u_kmac_core (
867     .clk_i,
868     .rst_ni,
869     // from Msg FIFO
870     .fifo_valid_i (msgfifo_valid),
871     .fifo_data_i (msgfifo_data ),
872     .fifo_strb_i (msgfifo_strb ),
873     .fifo_ready_o (msgfifo_ready),
874     // to SHA3 core
875     .msg_valid_o (msg_valid),
876     .msg_data_o (msg_data ),
877     .msg_strb_o (msg_strb ),
878     .msg_ready_i (msg_ready),
879     // Configurations
880     .kmac_en_i (app_kmac_en),
881     .mode_i (app_sha3_mode),
882     .strength_i (app_keccak_strength),
883     // Secret key interface

```

```

884 .key_data_i (key_data),
885 .key_len_i (key_len),
886 .key_valid_i (key_valid),
887
888 // Controls
889 .start_i (sha3_start ),
890 .process_i (msgfifo2kmac_process),
891 .done_i (sha3_done ),
892 .process_o (kmac2sha3_process ),
893
894 // LC escalation
895 .lc_escalate_en_i (lc_escalate_en[1]),
896
897 // Error detection
898 .sparse_fsm_error_o (kmac_core_state_error),
899 .key_index_error_o (key_index_error)
900 );
901
902 // SHA3 hashing engine
903
904 // msg_data masking
905 if (EnMasking == 1) begin: g_msg_mask
906   logic [MsgWidth-1:0] msg_mask_permuted;
907
908 // Permute the LFSR output to avoid same lfsr applied to
multiple times
909   always_comb begin
910     msg_mask_permuted = '0;
911     for (int unsigned i = 0 ; i < MsgWidth ; i++) begin
912       // Loop through the MsgPerm constant and swap between
the bits
913         msg_mask_permuted[i] = msg_mask[RndCnstMsgPerm[i]];
914       end
915     end
916
917     for (genvar i = 0 ; i < Share ; i++) begin:
g_msg_data_mask
918       assign msg_data_masked[i] = msg_data[i]
919         ^ ({MsgWidth{cfg_msg_mask}} &
msg_mask_permuted);
920     end : g_msg_data_mask
921   end else begin : g_no_msg_mask
922     assign msg_data_masked[0] = msg_data[0];
923     assign msg_mask = '0;
924
925     logic unused_msgmask;
926     assign unused_msgmask = ^{msg_mask, cfg_msg_mask,
msg_mask_en};
927   end
928   sha3 #(
929     .EnMasking (EnMasking)
930   ) u_sha3 (
931     .clk_i,
932     .rst_ni,
933
934     // MSG_FIFO interface (or from KMAC)
935     .msg_valid_i (msg_valid),
936     .msg_data_i (msg_data_masked ),
937     .msg_strb_i (msg_strb ),
938     .msg_ready_o (msg_ready),
939
940     // Entropy interface
941     .rand_valid_i (sha3_rand_valid),
942     .rand_early_i (sha3_rand_early),
943     .rand_data_i (sha3_rand_data),
944     .rand_aux_i (sha3_rand_aux),
945     .rand_update_o (sha3_rand_update),
946     .rand_consumed_o (sha3_rand_consumed),
947
948     // N, S: Used in cSHAKE mode
949     .ns_data_i (ns_prefix),
950
951     // Configurations
952     .mode_i (app_sha3_mode),
953     .strength_i (app_keccak_strength),
954
955     // Controls (CMD register)
956     .start_i (sha3_start ),
957     .process_i (kmac2sha3_process),
958     .run_i (sha3_run ),
959     .done_i (sha3_done ),
960
961     // LC escalation
962     .lc_escalate_en_i (lc_escalate_en[2]),
963
964     .absorbed_o (sha3_absorbed),
965     .squeezing_o (sha3_squeezing),
966
967     .block_processed_o (sha3_block_processed),
968
969     .sha3_fsm_o (sha3_fsm),
970
971     .state_valid_o (state_valid),
972     .state_o (state), // [Share]
973
974     // REQ/ACK interface to avoid power spikes
975     .run_req_o ( ), // Not used
976     .run_ack_i (1'b 1), // The SHA3 core is always allowed to
process.
977
978     .error_o (sha3_err),

```

```

979 .sparse_fsm_error_o (sha3_state_error),
980 .count_error_o (sha3_count_error),
981 .keccak_storage_rst_error_o (sha3_storage_rst_error)
982 );
983
984 // MSG_FIFO window interface to FIFO interface
=====
985 // Tie the read path
986 assign tlram_rvalid = 1'b 0;
987 assign tlram_rdata = '0;
988 assign tlram_rerror = '0;
989
990 // Convert endian here
991 // prim_packer always packs to the right(bit0). If the
input DWORD is
992 // big-endian, it needs to be swapped to little-endian to
maintain the
993 // order. Internal SHA3(Keccak) runs in little-endian in
contrast to HMAC
994 // So, no endian-swap after prim_packer.
995 assign tlram_wdata_endian = conv_endian32(tlram_wdata,
996
reg2hw.cfg_shadowed.msg_endianness.q);
997 assign tlram_wmask_endian = conv_endian32(tlram_wmask,
998
reg2hw.cfg_shadowed.msg_endianness.q);
999
1000 // TL Adapter
1001 tlul_adapter_sram #(
1002   .SramAw ($clog2(MsgWindowDepth)),
1003   .SramDw (MsgWindowWidth),
1004   .Outstanding (1),
1005   .ByteAccess (1),
1006   .ErrOnRead (1)
1007 ) u_tlul_adapter_msgfifo (
1008   .clk_i,
1009   .rst_ni,
1010   .en_ifetch_i (prim_mubi_pkg::MuBi4False),
1011   .tl_i (tl_win_h2d[WinMsgFifo]),
1012   .tl_o (tl_win_d2h[WinMsgFifo]),
1013
1014   .req_o (tlram_req),
1015   .req_type_o ( ),
1016   .gnt_i (tlram_gnt),
1017   .we_o (tlram_we ),
1018   .addr_o (tlram_addr),
1019   .wdata_o (tlram_wdata),
1020   .wmask_o (tlram_wmask),
1021   .intg_error_o ( ),
1022   .user_rsvd_o ( ),
1023   .rdata_i (tlram_rdata),
1024   .rvalid_i (tlram_rvalid),
1025   .rerror_i (tlram_rerror),
1026   .compound_txn_in_progress_o ( ),
1027   .readback_en_i (prim_mubi_pkg::MuBi4False),
1028   .readback_error_o ( ),
1029   .wr_collision_i (1'b0),
1030   .write_pending_i (1'b0)
1031 );
1032
1033 assign sw_msg_valid = tlram_req & tlram_we ;
1034 if (MsgWidth == MsgWindowWidth) begin : gen_sw_msg_samewidth
1035   assign sw_msg_data = tlram_wdata_endian ;
1036   assign sw_msg_strb = tlram_wmask_endian ;
1037 end else begin : gen_sw_msg_diff
1038   assign sw_msg_data = {{MsgWidth-MsgWindowWidth{1'b0}},
tlram_wdata_endian};
1039   assign sw_msg_strb = {{MsgWidth-MsgWindowWidth{1'b0}},
tlram_wmask_endian};
1040 end
1041 assign tlram_gnt = sw_msg_ready ;
1042
1043 logic unused_tlram_addr;
1044 assign unused_tlram_addr = &{1'b0, tlram_addr};
1045
1046 // Application interface Mux/Demux
kmac_app #(
1047   .EnMasking(EnMasking),
1048   .SecIdleAcceptSwMsg(SecIdleAcceptSwMsg),
1049   .NumAppIntf(NumAppIntf),
1050   .AppCfg(AppCfg)
1051 ) u_app_intf (
1052   .clk_i,
1053   .rst_ni,
1054
1055   .reg_key_data_i (sw_key_data),
1056   .reg_key_len_i (sw_key_len),
1057
1058   .reg_prefix_i (reg_ns_prefix),
1059
1060   .reg_kmac_en_i (reg_kmac_en),
1061   .reg_sha3_mode_i (reg_sha3_mode),
1062   .reg_keccak_strength_i (reg_keccak_strength),
1063
1064   // data from tl_adapter
1065   .sw_valid_i (sw_msg_valid),
1066   .sw_data_i (sw_msg_data),
1067   .sw_strb_i (sw_msg_strb),
1068   .sw_ready_o (sw_msg_ready),
1069
1070

```

```

1071 // KeyMgr sideloaded key interface
1072 .keymgr_key_i,
1073
1074 // Application data in / digest out interface
1075 .app_i,
1076 .app_o,
1077
1078 // Secret Key output to KMAC Core
1079 .key_data_o (key_data),
1080 .key_len_o (key_len),
1081 .key_valid_o (key_valid),
1082
1083 // to MSG_FIFO
1084 .kmac_valid_o (mux2fifo_valid),
1085 .kmac_data_o (mux2fifo_data),
1086 .kmac_strb_o (mux2fifo_strb),
1087 .kmac_ready_i (mux2fifo_ready),
1088 .kmac_bypass_fifo_o(mux2fifo_bypass),
1089
1090 // to KMAC Core
1091 .kmac_en_o (app_kmac_en),
1092
1093 // to SHA3 Core
1094 .sha3_prefix_o (ns_prefix),
1095 .sha3_mode_o (app_sha3_mode),
1096 .keccak_strength_o (app_keccak_strength),
1097
1098 // Keccak state from SHA3 core
1099 .keccak_state_valid_i (state_valid),
1100 .keccak_state_i (state),
1101
1102 // to STATE TL Window
1103 .reg_state_valid_o (reg_state_valid),
1104 .reg_state_o (reg_state),
1105
1106 // Configuration: Sideloaded Key
1107 .keymgr_key_en_i (reg2hw.cfg_shadowed.sideload.q),
1108
1109 .absorbed_i (sha3_absorbed), // from SHA3
1110 .squeezing_i (sha3_squeezing), // from SHA3
1111 .absorbed_o (app_absorbed), // to SW
1112
1113 .app_active_o (app_active),
1114
1115 .entropy_fast_process_i (reg_entropy_fast_process),
1116 .entropy_fast_process_o (entropy_fast_process),
1117
1118 .error_i (sha3_err.valid),
1119 .err_processed_i (err_processed),
1120
1121 .clear_after_error_o (clear_after_error),
1122
1123 // Command interface
1124 .sw_cmd_i (checked_sw_cmd),
1125 .cmd_o (kmac_cmd),
1126
1127 // Status
1128 .entropy_ready_i (entropy_configured),
1129
1130 // LC escalation
1131 .lc_escalate_en_i (lc_escalate_en[3]),
1132
1133 // Error report
1134 .error_o (app_err),
1135 .sparse_fsm_error_o (kmac_app_state_error),
1136 .counter_error_o (kmac_app_counter_error)
1137 );
1138
1139 // Message FIFO
1140 kmac_msgfifo #(
1141   .OutWidth (kmac_pkg::MsgWidth),
1142   .MsgDepth (kmac_pkg::MsgFifoDepth),
1143   .EnMasking (EnMasking)
1144 ) u_msgfifo (
1145   .clk_i,
1146   .rst_ni,
1147
1148   .fifo_valid_i (mux2fifo_valid),
1149   .fifo_data_i (mux2fifo_data),
1150   .fifo_strb_i (mux2fifo_strb),
1151   .fifo_ready_o (mux2fifo_ready),
1152   .fifo_bypass_i (mux2fifo_bypass),
1153
1154   .msg_valid_o (msgfifo_valid),
1155   .msg_data_o (msgfifo_data),
1156   .msg_strb_o (msgfifo_strb),
1157   .msg_ready_i (msgfifo_ready),
1158
1159   .fifo_empty_o (msgfifo_empty), // intr and status
1160   .fifo_full_o (msgfifo_full), // connected to status only
1161   .fifo_depth_o (msgfifo_depth),
1162
1163   .clear_i (sha3_done),
1164
1165   .process_i (reg2msgfifo_process),
1166   .process_o (msgfifo2kmac_process),
1167
1168   .err_o (msgfifo_err)
1169 );
1170
1171 logic [sha3_pkg::StateW-1:0] reg_state_tl [Share];
1172
1173 always_comb begin
1174   for (int i = 0 ; i < Share; i++) begin
1175     reg_state_tl[i] = reg_state_valid ? reg_state[i] : '0;
1176   end
1177 end
1178
1179 // State (Digest) reader
1180 kmac_staterd #(
1181   .AddrW (9), // 512B
1182   .EnMasking (EnMasking)
1183 ) u_staterd (
1184   .clk_i,
1185   .rst_ni,
1186
1187   .tl_i (tl_win_h2d[winState]),
1188   .tl_o (tl_win_d2h[winState]),
1189
1190   .state_i (reg_state_tl),
1191
1192   .endian_swap_i (reg2hw.cfg_shadowed.state_endianness.q)
1193 );
1194
1195 // Error checker
1196 kmac_errchk #(
1197   .EnMasking (EnMasking)
1198 ) u_errchk (
1199   .clk_i,
1200   .rst_ni,
1201
1202   // Configurations
1203   .cfg_mode_i (reg_sha3_mode),
1204   .cfg_strength_i (reg_keccak_strength),
1205
1206   .kmac_en_i (reg_kmac_en),
1207   .cfg_prefix_6B_i (reg_ns_prefix[47:0]), // first 6B of PRE-
1208   FIX
1209   .cfg_en_unsupported_modestrength_i (cfg_en_unsupported_modestrength),
1210
1211   .entropy_ready_pulse_i (entropy_ready),
1212
1213   // SW commands
1214   .sw_cmd_i (sw_cmd),
1215   .sw_cmd_o (checked_sw_cmd),
1216
1217   // Status from KMAC_APP
1218   .app_active_i (app_active),
1219
1220   // Status from SHA3 core
1221   .sha3_absorbed_i (sha3_absorbed),
1222   .keccak_done_i (sha3_block_processed),
1223
1224   // LC escalation
1225   .lc_escalate_en_i (lc_escalate_en[4]),
1226
1227   .err_processed_i (err_processed),
1228   .clear_after_error_i (clear_after_error),
1229
1230   .error_o (errchecker_err),
1231   .sparse_fsm_error_o (kmac_errchk_state_error)
1232 );
1233
1234 // Entropy Generator
1235 if (EnMasking == 1) begin : gen_entropy
1236   logic entropy_req, entropy_ack;
1237   logic [edn_pkg::ENDPOINT_BUS_WIDTH-1:0] entropy_data;
1238   logic unused_entropy_fips;
1239
1240   // Synchronize EDN interface
1241   prim_sync_reqack_data #(
1242     .Width (edn_pkg::ENDPOINT_BUS_WIDTH),
1243     .DataSrc2Dst (1'b0),
1244     .DataReg (1'b0)
1245   ) u_prim_sync_reqack_data (
1246     .clk_src_i (clk_i),
1247     .rst_src_ni (rst_ni),
1248     .clk_dst_i (clk_edn_i),
1249     .rst_dst_ni (rst_edn_ni),
1250     .req_chk_i (1'b1),
1251     .src_req_i (entropy_req),
1252     .src_ack_o (entropy_ack),
1253     .dst_req_o (entropy_o.edn_req),
1254     .dst_ack_i (entropy_i.edn_ack),
1255     .data_i (entropy_i.edn_bus),
1256     .data_o (entropy_data)
1257   );
1258
1259   // We don't track whether the entropy is pre-FIPS or not
1260   inside KMAC
1261   assign unused_entropy_fips = entropy_i.edn_fips;
1262
1263   kmac_entropy #(
1264     .RndCnstLfsrcPerm (RndCnstLfsrcPerm),
1265     .RndCnstLfsrcSeed (RndCnstLfsrcSeed),
1266     .RndCnstBufferLfsrcSeed (RndCnstBufferLfsrcSeed)
1267   ) u_entropy (
1268     .clk_i,
1269     .rst_ni,

```

```

1270 // EDN interface
1271 .entropy_req_o (entropy_req),
1272 .entropy_ack_i (entropy_ack),
1273 .entropy_data_i(entropy_data),
1274
1275 // Entropy to internal logic (DOM AND)
1276 .rand_valid_o   (sha3_rand_valid),
1277 .rand_early_o   (sha3_rand_early),
1278 .rand_data_o    (sha3_rand_data),
1279 .rand_aux_o     (sha3_rand_aux),
1280 .rand_update_i  (sha3_rand_update),
1281 .rand_consumed_i (sha3_rand_consumed),
1282
1283 // Status from internal logic
1284 /// KMAC secret block handling indicator
1285 .in_keyblock_i (entropy_in_keyblock),
1286
1287 // Configuration
1288 .mode_i        (entropy_mode),
1289 .entropy_ready_i (entropy_ready),
1290 .fast_process_i (entropy_fast_process),
1291
1292 /// Entropy refresh period in clk cycles
1293 .wait_timer_prescaler_i (wait_timer_prescaler),
1294 .wait_timer_limit_i    (wait_timer_limit),
1295
1296 /// Message Masking
1297 .msg_mask_en_i (msg_mask_en),
1298 .msg_mask_o    (msg_mask),
1299
1300 /// SW update of seed
1301 .seed_update_i (entropy_seed_update),
1302 .seed_data_i   (entropy_seed_data),
1303 .entropy_refresh_req_i (entropy_refresh_req),
1304
1305 // Status
1306 .hash_cnt_o   (entropy_hash_cnt),
1307 .hash_cnt_clr_i (entropy_hash_clr),
1308 .hash_threshold_i (entropy_hash_threshold),
1309
1310 .entropy_configured_o (entropy_configured),
1311
1312 // LC escalation
1313 .lc_escalate_en_i (lc_escalate_en[5]),
1314
1315 // Error
1316 .err_o (entropy_err),
1317 .sparse_fsm_error_o (kmac_entropy_state_error),
1318 .count_error_o (kmac_entropy_hash_counter_error),
1319 .err_processed_i (err_processed)
1320 );
1321 end else begin : gen_empty_entropy
1322 // If Masking is not used, no need of entropy. Ignore in-
1323 puts and config; tie output to 0.
1324 edn_pkg::edn_rsp_t unused_entropy_input;
1325 entropy_mode_e      unused_entropy_mode;
1326 logic               unused_entropy_fast_process;
1327
1328 assign unused_entropy_input = entropy_i;
1329 assign unused_entropy_mode = entropy_mode;
1330 assign unused_entropy_fast_process = entropy_fast_process;
1331
1332 assign entropy_o = '{default: '0};
1333
1334 logic unused_sha3_rand_update;
1335 logic unused_sha3_rand_consumed;
1336 assign sha3_rand_valid = 1'b 1;
1337 assign sha3_rand_early = 1'b 1;
1338 assign sha3_rand_data = '0;
1339 assign sha3_rand_aux = '0;
1340 assign unused_sha3_rand_update = sha3_rand_update;
1341 assign unused_sha3_rand_consumed = sha3_rand_consumed;
1342
1343 logic unused_seed_update;
1344 logic unused_seed_data;
1345 logic unused_refresh_period;
1346 assign unused_seed_data = ^entropy_seed_data;
1347 assign unused_seed_update = entropy_seed_update;
1348 assign unused_refresh_period = ^{wait_timer_limit,
1349 wait_timer_prescaler};
1350 assign unused_entropy_refresh_req = entropy_refresh_req;
1351
1352 logic unused_entropy_hash;
1353 assign unused_entropy_hash = ^{entropy_hash_clr, en-
1354 tropy_hash_threshold};
1355 assign entropy_hash_cnt = '0;
1356
1357 assign entropy_err = '{valid: 1'b 0, code: ErrNone, info:
1358 '0};
1359
1360 assign kmac_entropy_state_error = 1'b 0;
1361 assign kmac_entropy_hash_counter_error = 1'b 0;
1362
1363 logic unused_entropy_status;
1364 assign unused_entropy_status = entropy_in_keyblock;
1365
1366 // If Masking is off, always entropy configured
1367 assign entropy_configured = prim_mubi_pkg::MuBi4True;
1368
1369 logic unused_edn_clk_rst;

```

```

1367 assign unused_edn_clk_rst = ^{clk_edn_i, rst_edn_ni};
1368
1369 logic unused_lc_escalate_en5;
1370 assign unused_lc_escalate_en5 = ^lc_escalate_en[5];
1371 end
1372
1373 // MUBI4 buf
1374 prim_mubi4_sender #(
1375 .AsyncOn (0)
1376 ) u_sha3_done_sender (
1377 .clk_i,
1378 .rst_ni,
1379 .mubi_i (sha3_done_d),
1380 .mubi_o (sha3_done)
1381 );
1382
1383 // Register top
1384 logic [NumAlerts-1:0] alert_test, alerts, alerts_q;
1385
1386 logic shadowed_storage_err, shadowed_update_err;
1387 kmac_reg_top u_reg (
1388 .clk_i,
1389 .rst_ni,
1390 .rst_shadowed_ni,
1391
1392 .tl_i,
1393 .tl_o,
1394
1395 .tl_win_o (tl_win_h2d),
1396 .tl_win_i (tl_win_d2h),
1397
1398 .reg2hw,
1399 .hw2reg,
1400
1401 // SEC_CM: CFG_SHADOWED.CONFIG.SHADOW
1402 .shadowed_storage_err_o (shadowed_storage_err),
1403 .shadowed_update_err_o (shadowed_update_err),
1404 // SEC_CM: BUS.INTEGRITY
1405 .intg_err_o (alert_intg_err)
1406 );
1407
1408 logic unused_cfg_shadowed_qe;
1409 assign unused_cfg_shadowed_qe = ^{
1410 reg2hw.cfg_shadowed.kmac_en.qe
1411 , reg2hw.cfg_shadowed.kstrength.qe
1412 , reg2hw.cfg_shadowed.mode.qe
1413 , reg2hw.cfg_shadowed.msg_endianness.qe
1414 , reg2hw.cfg_shadowed.state_endianness.qe
1415 , reg2hw.cfg_shadowed.sideload.qe
1416 , reg2hw.cfg_shadowed.entropy_mode.qe
1417 , reg2hw.cfg_shadowed.entropy_fast_process.qe
1418 , reg2hw.cfg_shadowed.msg_mask.qe
1419 , reg2hw.cfg_shadowed.en_unsupported_modestrength.qe
1420 };
1421
1422 // Alerts
1423 assign alert_test = {
1424 reg2hw.alert_test.fatal_fault_err.q
1425 & reg2hw.alert_test.fatal_fault_err.qe, // [1]
1426 reg2hw.alert_test.recov_operation_err.q
1427 & reg2hw.alert_test.recov_operation_err.qe // [0]
1428 };
1429
1430 assign alerts = {
1431 alert_fatal, // Alerts[1]
1432 alert_recov_operation // Alerts[0]
1433 };
1434
1435 assign alert_recov_operation = shadowed_update_err;
1436
1437 // The recoverable alert is observable via status register
1438 until the KMAC operation is restarted
1439 // by re-writing the Control Register.
1440 logic status_alert_recov_ctrl_update_err;
1441 always_ff @(posedge clk_i or negedge rst_ni) begin
1442 if (!rst_ni) begin
1443 status_alert_recov_ctrl_update_err <= 1'b 0;
1444 end else if (alert_recov_operation) begin
1445 status_alert_recov_ctrl_update_err <= 1'b 1;
1446 end else if (err_processed) begin
1447 status_alert_recov_ctrl_update_err <= 1'b 0;
1448 end
1449 end
1450
1451 assign hw2reg.status.alert_recov_ctrl_update_err.d =
1452 status_alert_recov_ctrl_update_err;
1453
1454 assign alert_fatal = shadowed_storage_err
1455 | alert_intg_err
1456 | sparse_fsm_error
1457 | counter_error
1458 | control_integrity_error
1459 ;
1460
1461 // Make the fatal alert observable via status register.
1462 // Cannot be reset except the hardware reset
1463 logic status_alert_fatal_fault;
1464
1465 always_ff @(posedge clk_i or negedge rst_ni) begin
1466 if (!rst_ni) begin
1467 status_alert_fatal_fault <= 1'b 0;

```

```

1466     end else if (alert_fatal) begin
1467         status_alert_fatal_fault <= 1'b 1;
1468     end
1469 end
1470 assign hw2reg.status.alert_fatal_fault.d =
status_alert_fatal_fault;
1471
1472 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
1473     prim_alert_sender #(
1474         .AsyncOn(AlertAsyncOn[i]),
1475         .SkewCycles(AlertSkewCycles),
1476         .IsFatal(i)
1477     ) u_prim_alert_sender (
1478         .clk_i,
1479         .rst_ni,
1480         .alert_test_i ( alert_test[i] ),
1481         .alert_req_i ( alerts[i] ),
1482         .alert_ack_o ( ),
1483         .alert_state_o ( ),
1484         .alert_rx_i ( alert_rx_i[i] ),
1485         .alert_tx_o ( alert_tx_o[i] )
1486     );
1487 end
1488
1489 // Below assumes NumAlerts == 2
1490 `ASSERT_INIT(NumAlerts2_A, NumAlerts == 2)
1491
1492 always_ff @(posedge clk_i or negedge rst_ni) begin
1493     // break up the combinatorial path for local escalation
1494     if (!rst_ni) begin
1495         alerts_q[1] <= 1'b0;
1496     end else if (alerts[i]) begin
1497         // fatal alerts cannot be cleared
1498         alerts_q[1] <= 1'b1;
1499     end
1500 end
1501
1502 always_ff @(posedge clk_i or negedge rst_ni) begin
1503     // break up the combinatorial path for local escalation
1504     if (!rst_ni) begin
1505         alerts_q[0] <= 1'b0;
1506     end else begin
1507         // recoverable alerts can be cleared so just latch the
1508         alerts_q[0] <= alerts[0];
1509     end
1510 end
1511
1512 // Latched recoverable alert[0] is not used. Rather removing
1513 // above,
1514 // keep alert_q[1:0] and make alert_q[0] unused (lint
1515 // waive).
1516 logic unused_alerts_q0;
1517 assign unused_alerts_q0 = alerts_q[0];
1518
1519 // SEC_CM: LC_ESCALATE_EN.INTERSIG.MUBI, FSM.GLOBAL_ESC,
1520 FSM.LOCAL_ESC
1521 lc_ctrl_pkg::lc_tx_t alert_to_lc_tx;
1522 assign alert_to_lc_tx = lc_c-
trl_pkg::lc_tx_bool_to_lc_tx(alerts_q[1]);
1523 for (genvar i = 0; i < NumLcSyncCopies; i++) begin :
gen_or_alert_lc_sync
1524     assign lc_escalate_en[i] = lc_c-
trl_pkg::lc_tx_or_hi(alert_to_lc_tx, lc_escalate_en_sync[i]);
1525 end
1526
1527 // Synchronize life cycle input
1528 prim_lc_sync #(
1529     .NumCopies (NumLcSyncCopies)
1530 ) u_prim_lc_sync (
1531     .clk_i,
1532     .rst_ni,
1533     .lc_en_i ( lc_escalate_en_i ),
1534     .lc_en_o ( lc_escalate_en_sync )
1535 );
1536
1537 assign en_masking_o = EnMasking;
1538
1539 // Assertions //
1540 // Assert known for output values
1541 `ASSERT_KNOWN(KmacDone_A, intr_kmac_done_o)
1542 `ASSERT_KNOWN(FifoEmpty_A, intr_fifo_empty_o)
1543 `ASSERT_KNOWN(KmacErr_A, intr_kmac_err_o)
1544 `ASSERT_KNOWN(TLODValidKnown_A, tl_o.d_valid)
1545 `ASSERT_KNOWN(TLOAReadyKnown_A, tl_o.a_ready)
1546 `ASSERT_KNOWN(AlertKnown0_A, alert_tx_o)
1547 `ASSERT_KNOWN(EnMaskingKnown_A, en_masking_o)
1548
1549 // Parameter as desired
1550 `ASSERT_INIT(SecretKeyDivideBy32_A, (kmac_pkg::MaxKeyLen %
32) == 0)
1551
1552 // Command input should be sparse
1553 `ASSUME(CmdSparse_M, reg2hw.cmd.cmd.qe |-> reg2hw.cmd.cmd.q
inside {CmdStart, CmdProcess,
1554

```

```

CmdManualRun, CmdDone, CmdNone})
1555
1556 // redundant counter error
1557 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(SentMsgCountCheck_A,
u_sha3.u_pad.u_sentmsg_count,
1558     alert_tx_o[1])
1559 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(RoundCountCheck_A,
u_sha3.u_keccak.u_round_count,
1560     alert_tx_o[1])
1561
1562 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(KeyIndexCountCheck_A,
u_kmac_core.u_key_index_count,
1563     alert_tx_o[1])
1564
1565 `AS-
SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(AppDigestCountCheck_A,
u_app_intf.u_digest_part_counter,
1566     alert_tx_o[1])
1567
1568 // Sparse FSM state error
1569 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(KmacCoreFsmCheck_A,
u_kmac_core.u_state_regs, alert_tx_o[1])
1570 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(KmacAppFsmCheck_A,
u_app_intf.u_state_regs, alert_tx_o[1])
1571 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(SHA3FsmCheck_A,
u_sha3.u_state_regs, alert_tx_o[1])
1572 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(SHA3padFsmCheck_A,
u_sha3.u_pad.u_state_regs, alert_tx_o[1])
1573 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(KeccakFsmCheck_A,
u_sha3.u_keccak.u_state_regs,
1574     alert_tx_o[1])
1575
1576 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(ErrorCheckFsmCheck_A,
u_errchk.u_state_regs, alert_tx_o[1])
1577 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(KmacFsmCheck_A,
u_state_regs, alert_tx_o[1])
1578
1579 // prim is only instantiated if masking is enabled
1580 if (EnMasking == 1) begin : g_testassertion
1581     `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(EntropyFsmCheck_A,
gen_entropy.u_entropy.u_state_regs,
1582     alert_tx_o[1])
1583
1584 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(HashCountCheck_A,
gen_entropy.u_entropy.u_hash_count,
1585     alert_tx_o[1])
1586
1587 // MsgFifo.Packer
1588 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(
PackerCountCheck_A,
1589     u_msgfifo.u_packer.g_pos_dupcnt.u_pos,
1590     alert_tx_o[1])
1591
1592 // MsgFifo.Fifo
1593 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(
MsgFifoWptrCheck_A,
1594     u_msgfifo.u_msgfifo.gen_normal_fifo.u_fifo_cnt.gen_secure_ptrs.u_wp
tr,
1595     alert_tx_o[1])
1596
1597 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(
MsgFifoRptrCheck_A,
1598     u_msgfifo.u_msgfifo.gen_normal_fifo.u_fifo_cnt.gen_secure_ptrs.u_rp
tr,
1599     alert_tx_o[1])
1600
1601 end
1602
1603 // Alert assertions for reg_we onehot check
1604 `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
u_reg, alert_tx_o[1])
1605
1606 endmodule

```

4. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_app.sv (1,401 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 // KMAC Application interface
6 //
7 // This module implements the app interface which arbitrates
8 // between the SW interface and up to
9 // NumAppIntf hardware app interfaces. While a session is active
10 // (either an app or SW), other
11 // requests are stalled.
12 //
13 // There are two kind of app interfaces: Static interfaces
14 // which have a configuration defined at
15 //
16 // compile-time and only a one-shot digest can be retrieved. A
17 // dynamic interface can send the
18 // desired hashing configuration at run time and supports XOF
19 // operation.
20
21 `include "prim_assert.sv"

```



```

16
17 module kmac_app
18     import kmac_pkg::*;
19     #(
20         // App specific configs are defined in kmac_pkg
21         parameter bit      EnMasking      = 1'b0,
22         localparam int      Share          = (EnMasking) ?
23 : 1, // derived parameter
24         parameter bit      SecIdleAcceptSwMsg = 1'b0,
25         parameter int unsigned NumAppIntf    = 4,
26
27         parameter app_config_t AppCfg[NumAppIntf] = '{AppCfgKeyMgr,
AppCfgLcCtrl,
28                                     AppCfgRomC-
trl, AppCfg0tbn}
29     ) (
30         input clk_i,
31         input rst_ni,
32
33         // Secret Key from register
34         input [MaxKeyLen-1:0] reg_key_data_i[Share],
35         input key_len_e        reg_key_len_i,
36
37         // Prefix from register
38         input [sha3_pkg::NSRegisterSize*8-1:0] reg_prefix_i,
39
40         // mode, strength, kmac_en from register
41         input logic reg_kmac_en_i,
42         input sha3_pkg::sha3_mode_e reg_sha3_mode_i,
43         input sha3_pkg::keccak_strength_e reg_keccak_strength_i,
44
45         // Data from Software
46         input sw_valid_i,
47         input [MsgWidth-1:0] sw_data_i,
48         input [MsgWidth-1:0] sw_strb_i,
49         output logic sw_ready_o,
50
51         // KeyMgr Sideload Key interface
52         input keymgr_pkg::hw_key_req_t keymgr_key_i,
53
54         // Application Message in/ Digest out interface + control
55         signals
56         input app_req_t [NumAppIntf-1:0] app_i,
57         output app_rsp_t [NumAppIntf-1:0] app_o,
58
59         // to KMAC Core: Secret key
60         output logic [MaxKeyLen-1:0] key_data_o[Share],
61         output key_len_e key_len_o,
62         output logic key_valid_o,
63
64         // to MSG_FIFO
65         output logic kmac_valid_o,
66         output logic [MsgWidth-1:0] kmac_data_o[Share],
67
68         64 // This strobe is on bit level for the packer. The FIFO will
69         then convert it again to byte level.
70         output logic [MsgWidth-1:0] kmac_strb_o,
71         input logic kmac_ready_i,
72         output logic kmac_bypass_fifo_o,
73
74         // Signal to KMAC core that a KMAC operation is ongoing.
75         output logic kmac_en_o,
76
77         // To Sha3 Core
78         output logic [sha3_pkg::NSRegisterSize*8-1:0] sha3_prefix_o,
79         output sha3_pkg::sha3_mode_e sha3_mode_o,
80         output sha3_pkg::keccak_strength_e keccak_strength_o,
81
82         // The current keccak state from SHA3 Core
83         input keccak_state_valid_i,
84         input [sha3_pkg::StateW-1:0] keccak_state_i[Share],
85
86         // The keccak state is exposed to the STATE TL-window if no
87         application is active. If a key from
88         // the KeyMgr is used, the capacity region is zeroed. If an
89         app is active the state reads as
90         // zero.
91         output logic reg_state_valid_o,
92         output logic [sha3_pkg::StateW-1:0] reg_state_o[Share],
93
94         // Controls for SW whether to take the key from the KeyMgr
95         sideload interface or registers. For
96         // KMAC operations initiated by an app interface, we always
97         take the sideloaded key.
98         // If 1, the key for KMAC is taken from the KeyMgr sideload
99         interface.
100        // If 0, the key is taken from the registers.
101        input logic keymgr_key_en_i,
102
103        // Command from software
104        input kmac_cmd_e sw_cmd_i,
105
106        // Status signals from SHA3 core
107        input prim_mubi_pkg::mubi4_t absorbed_i,
108        input logic squeezing_i,
109
110        // to KMAC
111        output kmac_cmd_e cmd_o,
112
113        // to SW
114
115        output prim_mubi_pkg::mubi4_t absorbed_o,
116
117        // To status
118        output logic app_active_o,
119
120        // The entropy_fast_process bit is forced to 0 when recover-
121        ing an error state.
122        input logic entropy_fast_process_i,
123        output logic entropy_fast_process_o,
124
125        // Entropy must be ready before a KMAC operation can be per-
126        formed.
127        input prim_mubi_pkg::mubi4_t entropy_ready_i,
128
129        // Error input
130
131        117 // This error comes from KMAC/SHA3 engine and is pulsed if a
132        wrong command sequence is detected
133
134        118 // or kept high as long as any control signal has an invalid
135        value (e.g. if the done mubi signal
136        119 // is attacked to enforce a known digest value).
137
138        120 // This error is reported to the ERR_CODE regardless whether
139        app or SW is active.
140        input error_i,
141
142        // Pulsed when SW acknowledges error by writing to the
143        err_processed bit. Lets the interface
144        124 // return to idle after errors.
145        input err_processed_i,
146
147        output prim_mubi_pkg::mubi4_t clear_after_error_o,
148
149        // Reports error and its type if an FIFO or FSM error oc-
150        curred.
151        output kmac_pkg::err_t error_o,
152
153        // Life cycle
154        input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
155
156        // Fatal errors
157        output logic sparse_fsm_error_o,
158        output logic counter_error_o
159    );
160
161    // Create a lint error to reduce the risk of accidentally
162    enabling this feature.
163    141 `ASSERT_STATIC_LINT_ERROR(KmacSecIdleAcceptSwMsgNonDefault,
164    SecIdleAcceptSwMsg == 0)
165
166    import sha3_pkg::KeccakBitCapacity;
167    import sha3_pkg::L128;
168    import sha3_pkg::L224;
169    import sha3_pkg::L256;
170    import sha3_pkg::L384;
171    import sha3_pkg::L512;
172
173    // Definitions
174
175    localparam int KeyMgrKeyW = $bits(keymgr_key_i.key[0]);
176
177    localparam key_len_e KeyLengths [5] = '{Key128, Key192,
178    Key256, Key384, Key512};
179
180    localparam int SelKeySize = (AppKeyW == 128) ? 0 :
181    (AppKeyW == 192) ? 1 :
182    (AppKeyW == 256) ? 2 :
183    (AppKeyW == 384) ? 3 :
184    (AppKeyW == 512) ? 4 : 0 ;
185
186    localparam int SelDigSize = (AppDigestW == 128) ? 0 :
187    (AppDigestW == 192) ? 1 :
188    (AppDigestW == 256) ? 2 :
189    (AppDigestW == 384) ? 3 :
190    (AppDigestW == 512) ? 4 : 0 ;
191
192    // Key length is always the same as the full app digest
193    width.
194    localparam key_len_e SideloadedKey = KeyLengths[SelKeySize];
195
196    // Define right_encode(outlen) value here
197    // Look at kmac_pkg::key_len_e for the kinds of key size
198    //
199
200    174 // These values should be exactly the same as the key length
201    encodings
202    175 // in kmac_core.sv, with the only difference being that the
203    byte representing
204    176 // the byte-length of the encoded value is in the MSB posi-
205    tion due to right encoding
206    177 // instead of in the LSB position (left encoding).
207    localparam int OutLenW = 24;
208    localparam logic [OutLenW-1:0] EncodedOutLen [5] = '{
209    180 24'h 0001_80, // Key128
210    181 24'h 0001_C0, // Key192
211    182 24'h 02_0001, // Key256
212    183 24'h 02_8001, // Key384
213    184 24'h 02_0002 // Key512
214    };

```

```

187 localparam logic [OutLenW-1:0] EncodedOutLenStrb [5] = '{
188   24'h 00FFFF, // Key128,
189   24'h 00FFFF, // Key192
190   24'h FFFFFFF, // Key256
191   24'h FFFFFFF, // Key384
192   24'h FFFFFFF, // Key512
193 };
194
195 //////////////////////////////////////////////////
196 // Signals //
197 //////////////////////////////////////////////////
198 st_e st, st_d;
199
200 logic keymgr_key_used;
201
202 // Response channel signals
203
204 // The state machine controls the datapath mux selection and
handles the requests and responses.
205 app_rsp_t app_rsp;
206 logic app_data_ready, app_req_ready, app_error_req_ready;
207 logic app_digest_valid;
208 logic app_finish_rsp_valid, app_error_rsp_valid;
209 logic app_finish_rsp_is_error;
210 logic [AppDigestW-1:0] app_digest[2];
211
212 localparam int unsigned AppIdxW = $clog2(NumAppIntf);
213
214 // app_id indicates, which app interface was chosen. various
logic use this
215 // value to get the config or return the data.
216 logic [AppIdxW-1:0] app_id, app_id_d;
217 logic
clr_appid, set_appid;
218
219 // Output length
220 logic [OutLenW-1:0] encoded_outlen, encoded_outlen_strb;
221
222 // state output
223 // Mux selection signal
224 app_mux_sel_e mux_sel;
225 app_mux_sel_e mux_sel_buf_output;
226 app_mux_sel_e mux_sel_buf_err_check;
227 app_mux_sel_e mux_sel_buf_kmac;
228
229 // Tracking and error signals
230 kmac_pkg::err_t fsm_err, mux_err;
231 logic err_sha3_during_app_set, err_sha3_during_app_d,
err_sha3_during_app_q;
232
233 //////////////////////////////////////////////////
234 // Application arbitration //
235 //////////////////////////////////////////////////
236 // Latch the selected app ID when FSM grants a request. Per
default select ID 0. The FSM and
237 // datapath ensure that no data is leaked if interface is
not active.
238 always_ff @(posedge clk_i or negedge rst_ni) begin
239   if (!rst_ni) app_id <= AppIdxW'(0);
240   else if (clr_appid) app_id <= AppIdxW'(0);
241   else if (set_appid) app_id <= app_id_d;
242 end
243
244 // The arbitration uses a fixed priority. The assumption is
that the requests normally do no
245 // collide. If this assumption doesn't hold anymore, con-
sider RR arbiter.
246 logic [NumAppIntf-1:0] app_req_valids;
247 logic [$clog2(NumAppIntf)-1:0] arb_idx;
248 logic arb_valid;
249 logic arb_ready;
250
251 // Pick all request valid signals from the interfaces
252 always_comb begin
253   app_req_valids = '0;
254   for (int unsigned i = 0; i < NumAppIntf; i++) begin
255     app_req_valids[i] = app_i[i].req_valid;
256   end
257 end
258
259 prim_arbiter_fixed #(
260   .N (NumAppIntf),
261   .DW(1),
262   .EnDataPort(1'b 0)
263 ) u_appid_arb (
264   .clk_i,
265   .rst_ni,
266
267   .req_i (app_req_valids),
268   .data_i ('{default:'0}),
269   .gnt_o (), // not used
270   .idx_o (arb_idx),
271
272   .valid_o (arb_valid),
273   .data_o (), // not used
274   .ready_i (arb_ready)
275 );
276
277 assign app_id_d = AppIdxW'(arb_idx);
278 assign arb_ready = set_appid;

```

```

279
280 //////////////////////////////////////////////////
281 // App config selection //
282 //////////////////////////////////////////////////
283
284 // Select the new configuration from either the compile-time
defined configuration or from the
285 // supplied session configuration. This operates on the non-
latched app ID so the configuration
286 // can be latched at the same time the ID is.
287 app_config_t app_cfg;
288 app_cfg.session_cfg.mode = Ap-
pCfg[arb_idx].session_cfg.mode;
289 app_cfg.session_cfg.kstrength = Ap-
pCfg[arb_idx].session_cfg.kstrength;
290 app_cfg.session_cfg.en_xof = Ap-
pCfg[arb_idx].session_cfg.en_xof;
291 app_cfg.session_cfg.prefix_mode = Ap-
pCfg[arb_idx].session_cfg.prefix_mode;
292
293 // Note, the parsed value of prefix_mode is not actually
used but just here to have a complete
294 // type. It is overwritten below.
295 assign app_cfg.session_cfg.en_xof = app_cfg.en_xof;
296
297 always_comb begin
298   app_cfg.session_cfg.mode = Ap-
pCfg[arb_idx].session_cfg.mode;
299   app_cfg.session_cfg.kstrength = Ap-
pCfg[arb_idx].session_cfg.kstrength;
300   app_cfg.session_cfg.en_xof = Ap-
pCfg[arb_idx].session_cfg.en_xof;
301   app_cfg.session_cfg.prefix_mode = Ap-
pCfg[arb_idx].session_cfg.prefix_mode;
302
303 // Override configuration with configuration supplied by
dynamic interface.
304 if (AppCfg[arb_idx].if_type == AppDynamic) begin
305   app_cfg.session_cfg.mode = app_cfg.session_cfg.mode;
306   app_cfg.session_cfg.kstrength = app_cfg.session_cfg.kstrength;
307   app_cfg.session_cfg.en_xof = app_cfg.session_cfg.en_xof;
308   app_cfg.session_cfg.prefix_mode = app_cfg.session_cfg.prefix_mode;
309 end
310
311 // For KMAC always use prefix defined at compile time.
This saves the prefix check.
312 // Otherwise use prefix from CSR.
313 app_cfg.session_cfg.prefix_mode = app_cfg.session_cfg.prefix_mode;
314
315 // KMAC en / SHA3 mode / Strength / configuration latching.
316 logic kmac_en_d, kmac_en_q;
317 sha3_pkg::sha3_mode_e sha3_mode_d, sha3_mode_q;
318 sha3_pkg::keccak_strength_e keccak_strength_d, keccak_strength_q;
319
320 always_comb begin
321   app_cfg.session_cfg.mode = app_cfg.session_cfg.mode;
322   app_cfg.session_cfg.kstrength = app_cfg.session_cfg.kstrength;
323   app_cfg.session_cfg.en_xof = app_cfg.session_cfg.en_xof;
324   app_cfg.session_cfg.prefix_mode = app_cfg.session_cfg.prefix_mode;
325   kmac_en_d = reg_kmac_en_i;
326   sha3_mode_d = reg_sha3_mode_i;
327   keccak_strength_d = reg_keccak_strength_i;
328 end else if (set_appid) begin
329   // When an app starts, latch its configuration.
330   app_cfg.session_cfg.mode = app_cfg.session_cfg.mode;
331   app_cfg.session_cfg.kstrength = app_cfg.session_cfg.kstrength;
332   app_cfg.session_cfg.en_xof = app_cfg.session_cfg.en_xof;
333   app_cfg.session_cfg.prefix_mode = app_cfg.session_cfg.prefix_mode;
334   kmac_en_d = app_cfg.session_cfg.kstrength == App-
KMAC ? 1'b 1 : 1'b 0;
335   sha3_mode_d = app_cfg.session_cfg.en_xof == App-
SHA3 ? sha3_pkg::Sha3 :
336   app_cfg.session_cfg.prefix_mode == App-
Shake ? sha3_pkg::Shake : sha3_pkg::CShake;
337   keccak_strength_d = app_cfg.session_cfg.kstrength;
338 end else if (st == StIdle) begin
339   // In idle always propagate the latest CSR values as
there is no latch trigger when SW starts
340   // a hashing operation.
341   app_cfg.session_cfg.mode = AppSesCfgDefault;
342   kmac_en_d = reg_kmac_en_i;
343   sha3_mode_d = reg_sha3_mode_i;
344   keccak_strength_d = reg_keccak_strength_i;
345 end
346
347 always_ff @(posedge clk_i or negedge rst_ni) begin
348   if (!rst_ni) begin
349     app_cfg.session_cfg.mode <= AppSesCfgDefault;
350     kmac_en_q <= 1'b 0;
351     sha3_mode_q <= sha3_pkg::Sha3;
352     keccak_strength_q <= sha3_pkg::L256;
353   end else begin
354     app_cfg.session_cfg.mode <= app_cfg.session_cfg.mode;
355     kmac_en_q <= kmac_en_d;
356     sha3_mode_q <= sha3_mode_d;
357     keccak_strength_q <= keccak_strength_d;
358   end
359 end
360
361 assign kmac_en_o = kmac_en_q;
362 assign sha3_mode_o = sha3_mode_q;

```

```

363 assign keccak_strength_o = keccak_strength_q;
364
365 // Force the entropy_fast_process bit to 0 when recovering
from error.
366 logic disable_entropy_fast_process;
367 assign entropy_fast_process_o = dis-
able_entropy_fast_process ? 1'b0 : entropy_fast_process_i;
368
369 // Construct the complete configuration for the active app
from the static and latched
370 // session configuration.
371 always_comb begin
372     app_cfg = AppCfg[app_id];
373     app_cfg.session_cfg = app_ses_cfg_q;
374 end
375
376 ///////////////////////////////////////////////////
377 // App config checker //
378 ///////////////////////////////////////////////////
379 logic valid_app_sha3_strength;
380 logic valid_app_shake_strength;
381 logic valid_app_kmac_cfg;
382 logic valid_app_mode_strength_raw;
383 logic valid_app_mode_strength;
384 logic valid_en_xof_cfg;
385 logic valid_app_cfg;
386
387 assign valid_app_sha3_strength = app_cfg.ses-
sion_cfg.kstrength inside {sha3_pkg::L224,
388
389     sha3_pkg::L256,
390
391     sha3_pkg::L384,
392
393     sha3_pkg::L512};
394
395 assign valid_app_shake_strength = app_cfg.ses-
sion_cfg.kstrength inside {sha3_pkg::L128,
396
397     sha3_pkg::L256};
398
399 assign valid_app_mode_strength_raw =
app_cfg.session_cfg.mode == App-
SHA3
? valid_app_sha3_strength :
397 app_cfg.session_cfg.mode inside {AppShake, AppCShake,
AppKMAC} ? valid_app_shake_strength :
398
1'b0;
399
400 // Ignore the mode and strength check if app allows unsup-
ported combinations.
401
402 assign valid_app_mode_strength = valid_app_mode_strength_raw
|| app_cfg.en_unsup_comb;
403
404 assign valid_app_kmac_cfg = app_cfg.session_cfg.mode == App-
KMAC ?
405
prim_mubi_pkg::mubi4_test_true_strict(entropy_ready_i) : 1'b1;
406
407 // XOF operation is not supported for SHA3 or KMAC.
408 assign valid_en_xof_cfg = app_cfg.session_cfg.mode inside
{AppSHA3, AppKMAC} ?
409
app_cfg.session_cfg.en_xof ==
1'b0 : 1'b1;
410
411 assign valid_app_cfg = valid_app_mode_strength && val-
id_en_xof_cfg && valid_app_kmac_cfg;
412
413 // A compile-time defined configuration must always result
in a valid mode-strength and en_xof
414 // configuration.
415 `ASSERT(ConfigAlwaysValidIfStatic_A,
app_active_o && app_cfg.if_type == AppStatic
416
|-> valid_app_mode_strength && valid_en_xof_cfg)
417
418 ///////////////////////////////////////////////////
419 // Application Mux / Demux //
420 ///////////////////////////////////////////////////
421 // The active app's request and response channel is muxed
here from / to local signals.
422 app_req_t app_req;
423 assign app_req = app_i[app_id];
424
425 // There are three ready sources which can accept a request.
The app_data_ready active in the
426 // message phase and is driven by the FIFO. The
app_req_ready is controlled by the FSM to accept
427 // the control requests (start/config, process, and termina-
tion). The app_error_req_ready is used
428 // to accept messages and control requests during the error
handling.
429 // Similarly, there are three response generators. One
drives digest responses, one the finish
430 // response and one the error responses.

```

```

431 assign app_rsp = '{
432     req_ready: app_data_ready | app_error_req_ready |
app_req_ready,
433     rsp_valid: app_digest_valid | app_finish_rsp_valid |
app_error_rsp_valid,
434     digest_s0: app_digest[0],
435     digest_s1: app_digest[1],
436     error: app_error_rsp_valid | app_finish_rsp_is_error
|
437     // Must use _d as error is relevant in cycle
the digest is returned.
438     (err_sha3_during_app_d & app_cfg.if_type ==
AppStatic),
439     rsp_finish: app_finish_rsp_valid
440 };
441
442 // The three response generators may never be active at the
same time because otherwise responses
443 // would collide.
444 `ASSERT(AppOnlyOneRspSourceActive_A,
445     $onehot0({app_digest_valid, app_finish_rsp_valid,
app_error_rsp_valid}))
446
447 always_comb begin
448     for (int unsigned i = 0; i < NumAppIntf; i++) begin
449         if (i == app_id) begin
450             app_o[i] = app_rsp;
451         end else begin
452             app_o[i] = APP_RSP_DEFAULT;
453         end
454     end
455 end
456
457 ///////////////////////////////////////////////////
458 // FSM //
459 ///////////////////////////////////////////////////
460 logic clear_app_trackers;
461
462 logic any_request_outstanding;
463 logic last_req_pending;
464 logic process_cmd_pending;
465 logic last_req_hs;
466
467 logic last_msg_part_received_d, last_msg_part_received_q;
468 logic last_msg_part_received_set;
469
470 logic process_cmd_sent_d, process_cmd_sent_q;
471 logic process_cmd_sent_set;
472
473 logic pending_digest_rsp_d, pending_digest_rsp_q;
474 logic pending_error_rsp_d, pending_error_rsp_q;
475
476 logic digest_parts_available;
477 logic squeeze_again;
478 logic app_push_digest;
479 logic clear_digest_pusher;
480
481 logic service_rejected_error_d, service_rejected_error_q;
482 logic service_rejected_error_set;
483
484 logic err_key_used_but_invalid;
485 logic err_key_used_but_invalid_set;
486 logic err_key_used_but_invalid_clear;
487 logic err_key_used_but_invalid_d,
err_key_used_but_invalid_q;
488 logic err_during_sw_d, err_during_sw_q;
489 logic err_during_sw_set;
490 logic err_processed_d, err_processed_q;
491
492 // State register
493 `PRIM_FLOP_SPARSE_FSM(u_state_regs, st_d, st, st_e, stIdle)
494
495 // Next State & output logic
496 // SEC_CM: FSM.SPARSE
497 always_comb begin
498     st_d = st;
499
500     mux_sel = SecIdleAcceptSwMsg ? SelSw : SelNone;
501
502     // app_id control
503     set_appid = 1'b 0;
504     clr_appid = 1'b 0;
505
506     // Command towards the KMAC core
507     cmd_o = CmdNone;
508
509     // Software output
510     absorbed_o = prim_mubi_pkg::MuBi4False;
511
512     // Bypass FIFO if masked app is active.
513     kmac_bypass_fifo_o = 1'b0;
514
515     // Ready signal to handshake requests for dynamic inter-
faces.
516     app_req_ready = 1'b0;
517
518     // Control signals for the digest pusher.
519     app_push_digest = 1'b0;
520     clear_digest_pusher = 1'b0;
521
522     // Finish response

```



```

523 app_finish_rsp_valid = 1'b0;
524 app_finish_rsp_is_error = 1'b0;
525
526 // Error tracking and handling
527 fsm_err = '{valid: 1'b0, code: Er-
rNone, info: '0};
528 sparse_fsm_error_o = 1'b0;
529 clear_after_error_o =
prim_mubi_pkg::Mubi4False;
530 service_rejected_error_set = 1'b0;
531 err_sha3_during_app_set = 1'b0;
532 err_key_used_but_invalid_clear = 1'b0;
533 process_cmd_sent_set = 1'b0;
534 last_msg_part_received_set = 1'b0;
535 err_during_sw_set = 1'b0;
536 clear_app_trackers = 1'b0;
537 app_error_req_ready = 1'b0;
538 app_error_rsp_valid = 1'b0;
539
540 disable_entropy_fast_process = 1'b0;
541
542 unique case (st)
543   StIdle: begin
544     clear_app_trackers = 1'b1;
545
546     if (arb_valid) begin
547       // An app was chosen, latch the ID.
548       st_d = StAppCfg;
549       set_appid = 1'b1;
550     end else if (sw_cmd_i == CmdStart) begin
551       // Software initiates the sequence
552       st_d = StSw;
553       cmd_o = CmdStart;
554     end
555   end
556
557   StAppCfg: begin
558     if (!valid_app_cfg) begin
559       // Either the entropy source was not marked as ready
560       // for a KMAC operation, or the App
561       // configuration is invalid, or the configuration in
562       // the registers supplied by SW is
563       // invalid. In this error case we still go to the
564       // "message absorb" phase but no data is
565       // forwarded to the actual SHA3 core. This simpli-
566       // fies the error handling on the
567       // application side.
568       st_d = StErrorAwaitMsg;
569       service_rejected_error_set = 1'b1;
570     end else begin
571       // The configuration is valid and also latched. We
572       // can now send the start command and
573       // begin to absorb the message.
574       st_d = StAppMsg;
575       cmd_o = CmdStart;
576     end
577   end
578   // Handshake the configuration request also if there
579   // is an error. This is required as an
580   // application interface still must send the message.
581
582   // For static interfaces we stall the first request as
583   // it already contains data.
584   app_req_ready = app_cfg.if_type == AppDynamic;
585 end
586
587 StAppMsg: begin
588   mux_sel = SelApp;
589   kmac_bypass_fifo_o = app_cfg.masked;
590
591   // Accept any process command immediately and do not
592   // forward the request to the message
593   // FIFO.
594   app_req_ready = process_cmd_pending;
595   last_msg_part_received_set = last_req_hs;
596
597   if (last_req_hs) begin
598     if (app_cfg.session_cfg.mode == AppKMAC) begin
599       st_d = StAppOutLen;
600     end else begin
601       st_d = StAppProcess;
602     end
603   end else if (err_sha3_during_app_q) begin
604     // Handshaking last message part has priority as
605     // this error is also checked afterwards.
606     st_d = StErrorAwaitMsg;
607   end
608 end
609
610 StAppOutLen: begin
611   mux_sel = SelOutLen;
612   kmac_bypass_fifo_o = app_cfg.masked;
613
614   if (kmac_valid_o && kmac_ready_i) begin
615     st_d = StAppProcess;
616   end else begin
617     st_d = StAppOutLen;
618   end
619 end

```

```

610 StAppProcess: begin
611   cmd_o = CmdProcess;
612   st_d = StAppWait;
613   // Bypass must be stable until process command has
614   // been sent.
615   kmac_bypass_fifo_o = app_cfg.masked;
616   process_cmd_sent_set = 1'b1;
617 end
618
619 StAppWait: begin
620   // absorbed_i is pulsed once when the first process
621   // command ends. squeezing_i is high once
622   // the processing or subsequent squeezes have fin-
623   // ished. This is only relevant for dynamic
624   // interfaces.
625   if (prim_mubi_pkg::mubi4_test_true_strict(absorbed_i)
626   ||
627   (squeezing_i && app_cfg.if_type == AppDynamic))
628   begin
629     st_d = StAppPushDigest;
630     clear_digest_pusher = 1'b1;
631   end
632 end
633
634 StAppPushDigest: begin
635   // Static interface:
636   // - Return full digest in one response and return to
637   // idle (via finish).
638   // Dynamic interface:
639   // - Push the available digest / rate in parts.
640   // - For SHAKE and CSHAKE, if digest / rate is fully
641   // pushed, trigger a squeeze.
642   // - End the session if a termination request arrives.
643   app_push_digest = 1'b1;
644
645   if (app_cfg.if_type == AppStatic) begin
646     if (app_rsp.rsp_valid && app_req.rsp_ready) begin
647       // Must be on _d signal to capture the error in
648       // the same cycle.
649       st_d = err_sha3_during_app_d ? StErrorAwaitSw :
650       StAppFinish;
651     end
652   end else begin
653     // Ending a session by handshaking the request takes
654     // priority over 1) handling a SHA3
655     // error as it is checked for when sending the re-
656     // sponse and 2) squeezing to avoid
657     // starting an obsolete squeeze operation.
658     if (last_req_pending) begin
659       st_d = StAppFinish;
660       app_req_ready = 1'b1;
661     end else if (err_sha3_during_app_q) begin
662       st_d = StErrorPush;
663     end
664   end else if (squeeze_again && !pending_digest_rsp_d)
665   begin
666     // Trigger a squeeze if there should be sent more
667     // digest parts. Ensure that there
668     // is no pending response which would be 'killed'
669     // when changing the state.
670     cmd_o = CmdManualRun;
671     st_d = StAppWait;
672   end
673 end
674
675 StAppFinish: begin
676   if (app_cfg.if_type == AppStatic) begin
677     // Immediately end the session for static inter-
678     // faces.
679     st_d = StIdle;
680     cmd_o = CmdDone;
681     clr_appid = 1'b1;
682   end else begin
683     // Await handshake of pending response from last
684     // cycle(s) (digest or error response).
685     // Otherwise the finish response would violate the
686     // valid locked-in principle.
687     app_push_digest = pending_digest_rsp_q;
688     app_error_rsp_valid = pending_error_rsp_q;
689     if (!pending_digest_rsp_q && !pending_error_rsp_q)
690     begin
691       // We now can send the finish response. Send again
692       // the error flag to cover the case the
693       // error occurred whilst the last digest response
694       // was pending.
695       app_finish_rsp_valid = 1'b1;
696       app_finish_rsp_is_error = err_sha3_during_app_q;
697     end
698     // Once the finish response is handshaked the ses-
699     // sion can be terminated.
700     if (app_finish_rsp_valid && app_req.rsp_ready) be-
701     gin
702       st_d = StIdle;
703       cmd_o = CmdDone;
704       clr_appid = 1'b1;
705       if (err_sha3_during_app_q) begin
706         clear_after_error_o =
707         prim_mubi_pkg::Mubi4True;

```

```

685         end
686     end
687 end
688 end
689 end
690
691 StSw: begin
692     mux_sel = SelSw;
693
694     cmd_o = sw_cmd_i;
695     absorbed_o = absorbed_i;
696
697     if (sw_cmd_i == CmdDone) begin
698         st_d = StIdle;
699     end else begin
700         st_d = StSw;
701     end
702 end
703
704 StErrorKeyNotValid: begin
705     // Signal the error to SW and start the error recovery. This state won't accept any
706     // requests, so we cannot miss the last message request.
707     st_d = StErrorAwaitMsg;
708
709     // Clear the error as we now start handling it.
710     err_key_used_but_invalid_clear = 1'b1;
711
712     // Report the error to SW.
713     fsm_err.valid = 1'b 1;
714     fsm_err.code = ErrKeyNotValid;
715     fsm_err.info = 24'(app_id);
716 end
717
718 StErrorAwaitMsg: begin
719     if (err_during_sw_q) begin
720         // Only wait for SW to ack the error.
721         st_d = StErrorAwaitSw;
722     end else begin
723         // Track if the last message arrives.
724         last_msg_part_received_set = last_req_hs;
725
726         // Once the last message arrived, send error response in next cycle. We must use the _q
727         // signal to avoid 1) a timing loop with the ready and 2) handshaking a pending
728         // termination request in case we enter this error state after the
729         // message has already
730         // been received.
731         if (last_msg_part_received_q) begin
732             st_d = StErrorNotify;
733         end else begin
734             // Continue to absorb data or a process command on the app interface.
735             app_error_req_ready = 1'b1;
736         end
737     end
738
739 StErrorNotify: begin
740     // Send error response back to app. Once response is
741     // handshake, wait for termination
742     // request. Static interfaces don't wait for a termination request and skip this state.
743     app_error_rsp_valid = 1'b1;
744
745     if (app_rsp.rsp_valid && app_req.rsp_ready) begin
746         st_d = app_cfg.if_type == AppDynamic ? StErrorAwaitTermination : StErrorFinish;
747     end
748
749 StErrorAwaitTermination: begin
750     // A state entered only for dynamic interfaces. Wait for the app to send a termination
751     // request.
752     if (app_req.req_valid && app_req.req_last) begin
753         st_d = StErrorFinish;
754         app_req_ready = 1'b1;
755     end
756 end
757
758 StErrorFinish: begin
759     // Send finish response (dynamic) or exit immediately (static).
760
761     // For service-rejected errors, return to idle without SW interaction.
762     // For other errors, wait for SW to acknowledge the error.
763
764     // A SHA3 error can occur after the service rejected error is latched. Thus only send an
765     // error-free finish response if the only error case is a service rejected error.
766     app_finish_rsp_valid = app_cfg.if_type == AppDynamic;
767
768     app_finish_rsp_is_error = !service_rejected_error_q ||

```

```

err_sha3_during_app_q;
767
768     if ((app_finish_rsp_valid && app_req.rsp_ready) ||
769     (app_cfg.if_type == AppStatic)) begin
770         if (!app_finish_rsp_is_error) begin
771             clr_appid = 1'b1;
772             clear_after_error_o = prim_mubi_pkg::MuBi4True;
773             st_d = StIdle;
774         end else begin
775             st_d = StErrorAwaitSw;
776         end
777     end
778
779 StErrorAwaitSw: begin
780     // Wait for SW to have processed the error.
781     if (err_processed_q) begin
782         // Flush the message FIFO and let the SHA3 engine compute a digest (which won't be used
783         // but serves to bring the SHA3 engine back to the idle state). Skip the command if
784         // error occurred after regular PROCESS command has been sent and the processing is
785         // already ongoing or has already finished. Note, the process command is not tracked
786         // during SW operation to have the same behaviour as the KMAC before the extended app
787         // interface was added.
788         st_d = StErrorAwaitAbsorbed;
789
790         if (!process_cmd_sent_q) begin
791             cmd_o = CmdProcess;
792         end
793     end
794 end
795
796 StErrorAwaitAbsorbed: begin
797     // Wait until the hashing engine has finished computing the garbage digest. Use the
798     // squeezing_i signal to cover the case when the engine finishes before we wait for it.
799
800     if (squeezing_i) begin
801         // Clear internal variables, send done command, and return to idle.
802         clr_appid = 1'b1;
803         clear_after_error_o = prim_mubi_pkg::MuBi4True;
804         cmd_o = CmdDone;
805         st_d = StIdle;
806
807         // If error originated from SW, report 'absorbed' to SW.
808         if (err_during_sw_q) begin
809             absorbed_o = prim_mubi_pkg::MuBi4True;
810         end
811     end
812
813 StErrorPush: begin
814     // State for dynamic interfaces only.
815     // An error occurred while pushing digest parts. Keep sending error responses until the
816     // app sends a termination request. Then close the session normally.
817
818     // Continue sending pending response to adhere to valid locked-in principle.
819     if (pending_digest_rsp_q) begin
820         app_push_digest = 1'b1;
821     end else begin
822         // Now continuously send error responses until termination request arrives.
823         app_error_rsp_valid = 1'b1;
824         if (app_req.req_valid && app_req.req_last) begin
825             app_req_ready = 1'b1;
826             st_d = StAppFinish;
827         end
828     end
829 end
830
831 StTerminalError: begin
832     // this state is terminal
833     st_d = st;
834     sparse_fsm_error_o = 1'b 1;
835     fsm_err.valid = 1'b 1;
836     fsm_err.code = ErrFatalError;
837     fsm_err.info = 24'(app_id);
838 end
839
840 default: begin
841     st_d = StTerminalError;
842     sparse_fsm_error_o = 1'b 1;
843 end
844 endcase
845
846 // Error out if the key is used but it is invalid. Do not transition if:
847 // - A request is outstanding (valid request but no handshake in this cycle). Otherwise a valid

```

```

848 // could be dropped which could bring the hashing engine
in a non recoverable state.

849 // - When the key error handling starts. Otherwise the FSM
would get trapped.
850 // Note, the key error is cleared when the error handling
starts.
851 if (err_key_used_but_invalid && !any_request_outstanding
&& st != StErrorKeyNotValid) begin
852     st_d = StErrorKeyNotValid;
853     // Track if error occurred during SW.
854     err_during_sw_set = st == StSw;
855 end
856
857 // Track whether a hashing engine error occurred.
858 err_sha3_during_app_set =
859     error_i &&
860     (st inside {StAppCfg, StAppMsg, StAppOutLen, StAppPro-
cess, StAppWait, StAppPushDigest});
861
862 // Enforce the entropy_fast_process bit is set to 0 for
the duration of the error handling.

863 // This ensures the hashing updates the PRNG and hence the
masking is fully enabled. This
864 // in turn ensures that no improperly masked hashing oper-
ation can be triggered by an
865 // adversary.
866 disable_entropy_fast_process =
867     st inside {StErrorKeyNotValid, StErrorAwaitMsg, StEr-
rorNotify, StErrorAwaitTermination,
868     StErrorFinish, StErrorAwaitSw, StError-
AwaitAbsorbed};
869
870 // SEC_CM: FSM.GLOBAL_ESC, FSM.LOCAL_ESC
871 // Unconditionally jump into the terminal error state if
the life cycle controller triggers an
872 // escalation.
873 if (lc_ctrl_pkg::lc_tx_test_true_loose(lc_escalate_en_i))
begin
874     st_d = StTerminalError;
875 end
876 end
877
878 assign any_request_outstanding = app_req.req_valid && !
app_rsp.req_ready;
879
880 assign last_req_pending = app_req.req_valid &&
app_req.req_last;
881 // An empty message request issues a process command and
does not forward any message data.
882 assign process_cmd_pending = last_req_pending &&
app_req.strb == '0';
883 assign last_req_hs = last_req_pending &&
app_rsp.req_ready;
884
885 // Track when last message part is handshaked. Note, this
also tracks process commands.
886 assign last_msg_part_received_d =
887     clear_app_trackers ?
1'b0 : // clear
888     last_msg_part_received_set ?
1'b1 : // set
889
last_msg_part_received_q; // hold
890
891 assign process_cmd_sent_d =
892     clear_app_trackers ? 1'b0 : // clear
893     process_cmd_sent_set ? 1'b1 : // set
894     process_cmd_sent_q; // hold
895
896 // Track when SW acknowledges the error. Only set flag if
this bit is relevant for the app
897 // interface, i.e., a hashing operation is ongoing (this in-
cludes SW- and app-triggered
898 // operations).
899 assign err_processed_d =
900     clear_app_trackers ?
1'b0 : // clear
901     (err_processed_i && (st != StIdle)) ?
1'b1 : // set
902     err_pro-
cessed_q; // hold
903
904 // Track errors occurring in SW mode.
905 assign err_during_sw_d = clear_app_trackers ? 1'b0 : //
clear
906     err_during_sw_set ? 1'b1 : // set
907     err_during_sw_q; // hold
908
909 // Track hashing engine errors
910 assign err_sha3_during_app_d = clear_app_trackers ?
1'b0 : // clear
911     err_sha3_during_app_set ?
1'b1 : // set
912
err_sha3_during_app_q; // hold
913
914 // Track key invalid errors
915 assign err_key_used_but_invalid_d =
clear_app_trackers ? 1'b0 : // clear

```

```

916
err_key_used_but_invalid_clear ? 1'b0 : // clear when handled
917
err_key_used_but_invalid_set ? 1'b1 : // set
918
err_key_used_but_invalid_q; // hold
919
920 // Factor out the clearing as it would affect the state
transition logic but we want all
921 // state dependent transition logic inside the FSM.
922 assign err_key_used_but_invalid =
err_key_used_but_invalid_set || err_key_used_but_invalid_q;
923
924 // Track service rejected errors
925 assign service_rejected_error_d =
clear_app_trackers ? 1'b0 : // clear
926     ser-
vice_rejected_error_set ? 1'b1 : // set
927
service_rejected_error_q; // hold
928
929 always_ff @(posedge clk_i or negedge rst_ni) begin
930     if (!rst_ni) begin
931         last_msg_part_received_q <= 1'b0;
932         process_cmd_sent_q <= 1'b0;
933         pending_digest_rsp_q <= 1'b0;
934         pending_error_rsp_q <= 1'b0;
935         service_rejected_error_q <= 1'b0;
936         err_processed_q <= 1'b0;
937         err_during_sw_q <= 1'b0;
938         err_sha3_during_app_q <= 1'b0;
939         err_key_used_but_invalid_q <= 1'b0;
940     end else begin
941         last_msg_part_received_q <= last_msg_part_received_d;
942         process_cmd_sent_q <= process_cmd_sent_d;
943         pending_digest_rsp_q <= pending_digest_rsp_d;
944         pending_error_rsp_q <= pending_error_rsp_d;
945         service_rejected_error_q <= service_rejected_error_d;
946         err_processed_q <= err_processed_d;
947         err_during_sw_q <= err_during_sw_d;
948         err_sha3_during_app_q <= err_sha3_during_app_d;
949         err_key_used_but_invalid_q <=
err_key_used_but_invalid_d;
950     end
951 end
952
953 // Check that dynamic app only states are never entered for
a static app.
954 `ASSERT(InvalidStateForStaticAppA,
955     st inside {StErrorPush, StErrorAwaitTermination} |->
(app_cfg.if_type == AppDynamic),
956     clk_i, rst_ni)
957
958 // Datapath //
959 // Encoded output length for the KMAC operation. The length
is based upon the full app interface
960 // response width.
961 // assign encoded_outlen = EncodedOutLen[SelDigSize];
962 // assign encoded_outlen_strb = EncodedOutLenStrb[SelDigSize];
963
964 // Data mux
965 // This is the main datapath part of the app interface lo-
gic.
966
967 // Once the FSM selected an interface and checked its con-
figuration the message FIFO interface is
968 // exposed to the interface. For a KMAC operation, as soon
as the app has sent the last message
969
970 // the FSM pushes the encoded output length into the message
FIFO. This is a predefined value,
971 // see 'EncodeOutLen' parameter above.
972
973 // The data is forwarded depending on the masking paramet-
ers:
974
975 // EnMasking = 1:
976 // - For static and dynamic interface:
977 //   - Masked = 0: Forward share 0. Set share 1 to '0'.
978 //   - Masked = 1: Forward both shares.
979 // EnMasking = 0
980 // - For static and dynamic interface:
981 //   - Masked = 0: Forward share 0. Ignore share 1.
982 //   - Masked = 1: Forward XOR of both shares.
983 always_comb begin
984     app_data_ready = 1'b 0;
985
986     // Always accept a SW request even if SW is not active to
avoid deadlocking the software. If
987     // SW is not active, this request is ignored and will
trigger an error (handled outside).
988     sw_ready_o = 1'b 1;
989
990     kmac_valid_o = 1'b 0;
991     kmac_data_o = '{default: '0};
992     kmac_strb_o = '0;
993
994     unique case (mux_sel_buf_kmac)

```

```

995     SelApp: begin
996         // Forward message only if it contains data. An empty
message / process command is not
997         // forwarded.
998         kmac_valid_o = app_req.req_valid && !pro-
cess_cmd_pending;
999         if (EnMasking) begin
1000             kmac_data_o[0] = app_req.data_s0;
1001             kmac_data_o[1] = app_cfg.masked ? app_req.data_s1 :
'0';
1002         end else begin
1003             // If the interface is masked but the KMAC core not,
then we unmask the input data.
1004             kmac_data_o[0] = app_cfg.masked ? app_req.data_s0 ^
app_req.data_s1 : app_req.data_s0;
1005         end
1006         // Expand byte strobe to bits for the prim_packer in-
side MSG_FIFO.
1007         for (int i = 0; i < MsgStrbW; i++) begin
1008             kmac_strb_o[8 * i +: 8] = {8{app_req.strb[i]}};
1009         end
1010         app_data_ready = kmac_ready_i;
1011     end
1012
1013     SelOutLen: begin
1014         // Write encoded output length value as unmasked data
(share 1 = '0').
1015         kmac_valid_o = 1'b1; // always write
1016         kmac_data_o[0] = MsgWidth'(encoded_outlen);
1017         kmac_strb_o = MsgWidth'(encoded_outlen_strb);
1018     end
1019
1020     SelSw: begin
1021         // SW supports only one share
1022         kmac_valid_o = sw_valid_i;
1023         kmac_data_o[0] = sw_data_i;
1024         for (int i = 1; i < Share; i++) begin
1025             kmac_data_o[i] = '0';
1026         end
1027         kmac_strb_o = sw_strb_i;
1028         sw_ready_o = kmac_ready_i;
1029     end
1030
1031     default: begin // Incl. SelNone
1032         kmac_valid_o = 1'b0;
1033         kmac_data_o = '{default: '0}';
1034         kmac_strb_o = '0';
1035     end
1036
1037 endcase
1038 end
1039
1040 // Error checking for Mux
1041 always_comb begin
1042     mux_err = '{valid: 1'b0, code: ErrNone, info: '0}';
1043
1044     if (mux_sel_buf_err_check != SelSw && sw_valid_i) begin
1045         // If SW writes message into FIFO
1046         mux_err = '{
1047             valid: 1'b1,
1048             code: ErrSwPushedMsgFifo,
1049             info: 24'({8'h00, 8'(st), 8'(mux_sel_buf_err_check)})
1050         };
1051     end else if (app_active_o && sw_cmd_i != CmdNone) begin
1052         // If SW issues command except start
1053         mux_err = '{
1054             valid: 1'b1,
1055             code: ErrSwIssuedCmdInAppActive,
1056             info: 24'(sw_cmd_i)
1057         };
1058     end
1059 end
1060
1061 logic [AppMuxWidth-1:0] mux_sel_buf_output_logic;
1062 assign mux_sel_buf_output = app_mux_sel_e'(mux_sel_buf_out-
put_logic);
1063
1064 // SEC_CM: LOGIC.INTEGRITY
1065 prim_sec_anchor_buf #{
1066     .Width(AppMuxWidth)
1067 } u_prim_buf_state_output_sel (
1068     .in_i(mux_sel),
1069     .out_o(mux_sel_buf_output_logic)
1070 );
1071
1072 logic [AppMuxWidth-1:0] mux_sel_buf_err_check_logic;
1073 assign mux_sel_buf_err_check =
app_mux_sel_e'(mux_sel_buf_err_check_logic);
1074
1075 // SEC_CM: LOGIC.INTEGRITY
1076 prim_sec_anchor_buf #{
1077     .Width(AppMuxWidth)
1078 } u_prim_buf_state_err_check (
1079     .in_i(mux_sel),
1080     .out_o(mux_sel_buf_err_check_logic)
1081 );
1082
1083 logic [AppMuxWidth-1:0] mux_sel_buf_kmac_logic;
1084 assign mux_sel_buf_kmac = app_mux_sel_e'(mux_sel_buf_k-
mac_logic);

```

```

1085
1086 // SEC_CM: LOGIC.INTEGRITY
1087 prim_sec_anchor_buf #{
1088     .Width(AppMuxWidth)
1089 } u_prim_buf_state_kmac_sel (
1090     .in_i(mux_sel),
1091     .out_o(mux_sel_buf_kmac_logic)
1092 );
1093
1094 // SEC_CM: LOGIC.INTEGRITY
1095 logic reg_state_valid;
1096 prim_sec_anchor_buf #{
1097     .Width(1)
1098 } u_prim_buf_state_output_valid (
1099     .in_i(reg_state_valid),
1100     .out_o(reg_state_valid_o)
1101 );
1102
1103 // Keccak state Demux
1104 // The state is only exposed to the registers if SW is act-
ive.
1105 always_comb begin
1106     reg_state_valid = 1'b0;
1107     reg_state_o = '{default: '0}';
1108
1109     if ((mux_sel_buf_output == SelSw) &&
lc_c-
1110         tr_lpkg::lc_tx_test_false_strict(lc_escalate_en_i)) begin
1111         reg_state_valid = keccak_state_valid_i;
1112         reg_state_o = keccak_state_i;
1113         // If key is sideloaded and KMAC is SW initiated hide
the capacity from SW by zeroing.
1114         // See https://github.com/lowRISC/opentitan/issues/
17508.
1115         if (keymgr_key_en_i) begin
1116             for (int i = 0; i < Share; i++) begin
1117                 unique case (keccak_strength_q)
1118                     L128: reg_state_o[i]
[sha3_pkg::StateW-1::KeccakBitCapacity[L128]] = '0';
1119                     L224: reg_state_o[i]
[sha3_pkg::StateW-1::KeccakBitCapacity[L224]] = '0';
1120                     L256: reg_state_o[i]
[sha3_pkg::StateW-1::KeccakBitCapacity[L256]] = '0';
1121                     L384: reg_state_o[i]
[sha3_pkg::StateW-1::KeccakBitCapacity[L384]] = '0';
1122                     L512: reg_state_o[i]
[sha3_pkg::StateW-1::KeccakBitCapacity[L512]] = '0';
1123                     default: reg_state_o[i] = '0';
1124                 endcase
1125             end
1126         end
1127     end
1128 end
1129
1130 // Digest pusher //
1131 // The maximal number of digest chunks is defined by the op-
eration with the largest rate. The
1132 // bit-width of the rate for any XOF algorithm can be com-
puted with: StateW - 2 * Strength.
1133 // And as SHA3, SHAKE, cSHAKE and KMAC have always a state
width of 1600, the largest rate has
1134 // SHAKE-128 / cSHAKE-128 / KMAC-128 (the lowest strength).
1135 localparam int MaxNumDigestParts = (sha3_pkg::StateW - 2 *
128) / DynAppDigestW;
1136 localparam int DigestCntW = $clog-
2(MaxNumDigestParts);
1137 logic [DigestCntW-1:0] digest_top;
1138 logic [DigestCntW-1:0] current_digest_idx;
1139
1140 // Expose only the relevant part of the digest depending on
the mode and strength. In case an
1141 // invalid mode or strength is detected, we set the di-
gest_top to the smallest value.
1142 always_comb begin
1143     unique case (sha3_mode_q)
1144         sha3_pkg::Sha3: begin
1145             unique case (keccak_strength_q)
1146                 sha3_pkg::L128: digest_top = DigestCntW'(128 / Dyn-
AppDigestW);
1147                 // 224 is not a multiple of 64. Round up the number
of digest parts. Note, due to this
1148                 sha3_pkg::L224: digest_top = DigestCntW'(4);
1149                 sha3_pkg::L256: digest_top = DigestCntW'(256 / Dyn-
AppDigestW);
1150                 sha3_pkg::L384: digest_top = DigestCntW'(384 / Dyn-
AppDigestW);
1151                 sha3_pkg::L512: digest_top = DigestCntW'(512 / Dyn-
AppDigestW);
1152                 default: digest_top = DigestCntW'(128 / Dyn-
AppDigestW);
1153             endcase
1154         end
1155         sha3_pkg::Shake,
1156         sha3_pkg::cShake: begin
1157             // Expose the requested output length for KMAC. Expose
1158         end
1159     end

```

```

the full rate for SHAKE and cSHAKE.
1161   if (kmac_en_q) begin
1162     digest_top = AppDigestW / DynAppDigestW;
1163   end else begin
1164     unique case (keccak_strength_q)
1165       sha3_pkg::L128: digest_top = Digest-
CntW'((sha3_pkg::StateW - 2 * 128) / DynAppDigestW);
1166       sha3_pkg::L224: digest_top = Digest-
CntW'((sha3_pkg::StateW - 2 * 224) / DynAppDigestW);
1167       sha3_pkg::L256: digest_top = Digest-
CntW'((sha3_pkg::StateW - 2 * 256) / DynAppDigestW);
1168       sha3_pkg::L384: digest_top = Digest-
CntW'((sha3_pkg::StateW - 2 * 384) / DynAppDigestW);
1169       sha3_pkg::L512: digest_top = Digest-
CntW'((sha3_pkg::StateW - 2 * 512) / DynAppDigestW);
1170       default: digest_top = Digest-
CntW'((sha3_pkg::StateW - 2 * 512) / DynAppDigestW);
1171     endcase
1172   end
1173   default: digest_top = DigestCntW'(128 / DynAppDigestW);
1174 endcase
1175 end
1176
1177 // Limit the response width to 64 bits because its the GCD
for all modes except SHA-224.
1178 `ASSERT_INIT(DynAppDigestWIs64Bit_A, DynAppDigestW == 64)
1179 // Ensure all counter top computations divide properly as
the response channel does not carry any
1180 // valid-strobe information.
1181
1182 `ASSERT_INIT(DigestTopDividesSha3L128_A, 128 % DynAppDigestW
== 0)
1183 // An exception is SHA3-224, where we fix the number of di-
gest parts to 4, so the full hash and
1184 // some additional rate bits are sent.
1185
1186 `ASSERT_INIT(DigestTopDividesSha3L256_A, 256 % DynAppDigestW
== 0)
1187
1188 `ASSERT_INIT(DigestTopDividesSha3L384_A, 384 % DynAppDigestW
== 0)
1189
1190 `ASSERT_INIT(DigestTopDividesSha3L512_A, 512 % DynAppDigestW
== 0)
1191
1192 `ASSERT_INIT(DigestTopDividesKmacAppDigestW_A, AppDigestW %
DynAppDigestW == 0)
1193
1194 `ASSERT_INIT(DigestTopDividesShakeL128_A, (sha3_pkg::StateW
- 2 * 128) % DynAppDigestW == 0)
1195
1196 `ASSERT_INIT(DigestTopDividesShakeL224_A, (sha3_pkg::StateW
- 2 * 224) % DynAppDigestW == 0)
1197
1198 `ASSERT_INIT(DigestTopDividesShakeL256_A, (sha3_pkg::StateW
- 2 * 256) % DynAppDigestW == 0)
1199
1200 `ASSERT_INIT(DigestTopDividesShakeL384_A, (sha3_pkg::StateW
- 2 * 384) % DynAppDigestW == 0)
1201
1202 `ASSERT_INIT(DigestTopDividesShakeL512_A, (sha3_pkg::StateW
- 2 * 512) % DynAppDigestW == 0)
1203
1204 assign digest_parts_available = current_digest_idx < di-
gest_top;
1205
1206 // Send parts of the digest if we are pushing.
1207 // Gate with digest_parts_available to prevent sending capa-
city-region bits when
1208 // current_digest_idx has already reached digest_top (the
rate boundary).
1209 assign app_digest_valid = app_push_digest && di-
gest_parts_available &&
1210 lc_c-
trl_pkg::lc_tx_test_false_strict(lc_escalate_en_i);
1211
1212 // Only allow a squeeze if the app allows it.
1213 logic squeeze_again_allowed;
1214
1215 assign squeeze_again_allowed = app_cfg.if_type == AppDynamic
&& app_cfg.session_cfg.en_xof &&
1216 app_cfg.session_cfg.mode !=
AppSHA3 &&
1217 app_cfg.session_cfg.mode !=
AppKMAC;
1218
1219 // Request a squeeze once we are out of digest parts.
1220 assign squeeze_again = squeeze_again_allowed && !di-
gest_parts_available;
1221
1222 logic digest_part_pushed;
1223 assign digest_part_pushed = app_digest_valid &&
app_req_rsp_ready;
1224
1225 // Register if there is a digest response pending. If one is
pending, any following response
1226 // (finish or error) must wait as otherwise data is changed
which would violate the valid
1227 // locked-in principle.
1228 assign pending_digest_rsp_d = app_digest_valid && !di-
gest_part_pushed;
1229
1230 // Similarly, register if there is a pending error response.

```

```

1223 logic error_rsp_pushed;
1224 assign error_rsp_pushed = app_error_rsp_valid &&
app_req_rsp_ready;
1225 assign pending_error_rsp_d = app_error_rsp_valid && !er-
ror_rsp_pushed;
1226
1227 logic [DynAppDigestW-1:0] digest_part[Share];
1228 for (genvar i = 0; i < Share; i++) begin : g_digest_mux
1229   assign digest_part[i] = keccak_state_i[i][DynAppDigestW *
current_digest_idx +: DynAppDigestW];
1230 end
1231
1232 always_comb begin
1233   app_digest = '{default:'0';
1234   if (app_digest_valid) begin
1235     // Digest has always 2 entries. If !EnMasking, second is
tied to 0.
1236     for (int i = 0; i < Share; i++) begin
1237       // Return the portion of state.
1238       if (app_cfg.if_type == AppStatic) begin
1239         app_digest[i] = keccak_state_i[i][AppDigestW-1:0];
1240       end else if (app_cfg.if_type == AppDynamic) begin
1241         // Only expose a sub part. For SHA3-224, we set the
capacity bits of the last response to
1242         // '0. See digest_top calculation above.
1243         app_digest[i][DynAppDigestW-1:0] = digest_part[i];
1244         if (sha3_mode_q == sha3_pkg::Sha3 && kec-
cak_strength_q == sha3_pkg::L224 &&
1245             current_digest_idx == digest_top - 1) begin
1246           app_digest[i][DynAppDigestW-1:DynAppDigestW/2] =
'0;
1247         end
1248       end
1249     end
1250   end
1251 end
1252
1253 // SEC_CM CTR.REDUN
1254 prim_count #(
1255   .Width(DigestCntW),
1256   .PossibleActions(prim_count_pkg::Clr |
prim_count_pkg::Incr)
1257 ) u_digest_part_counter (
1258   .clk_i,
1259   .rst_ni,
1260   .clr_i (clear_digest_pusher),
1261   .set_i (1'b0),
1262   .set_cnt_i ('0),
1263   .incr_en_i (1'b1),
1264   .decr_en_i (1'b0),
1265   .step_i (DigestCntW'(1)),
1266   .commit_i (digest_part_pushed ||
clear_digest_pusher),
1267   .cnt_o (current_digest_idx),
1268   .cnt_after_commit_o(),
1269   .err_o (counter_error_o)
1270 );
1271
1272 // Key handling //
1273
1274 // Secret Key Mux
1275 // Prepare merged key if EnMasking is not set.
1276
1277 // Combine share keys into unpacked array for logic below to
assign easily.
1278 // SEC_CM: KEY.SIDELOAD
1279 logic [MaxKeyLen-1:0] keymgr_key[Share];
1280 if (EnMasking == 1) begin : g_masked_key
1281   for (genvar i = 0; i < Share; i++) begin : gen_key_pad
1282     assign keymgr_key[i] = {(MaxKeyLen-KeyMgrKeyW)'(0),
keymgr_key_i.key[i]};
1283   end
1284 end else begin : g_unmasked_key
1285   always_comb begin
1286     keymgr_key[0] = '0;
1287     for (int i = 0; i < keymgr_pkg::Shares; i++) begin
1288       keymgr_key[0][KeyMgrKeyW-1:0] ^= keymgr_key_i.key[i];
1289     end
1290   end
1291 end
1292
1293 // Sideloaded key expose control
1294 assign err_key_used_but_invalid_set = keymgr_key_used && !
keymgr_key_i.valid;
1295
1296 always_comb begin
1297   keymgr_key_used = 1'b0;
1298   key_len_o = reg_key_len_i;
1299   for (int i = 0; i < Share; i++) begin
1300     key_data_o[i] = reg_key_data_i[i];
1301   end
1302
1303 // The key is considered invalid in all cases that are not
listed below (which includes idle and
1304 // error states).
1305 key_valid_o = 1'b0;
1306
1307 unique case (st)

```



```

1308     StAppMsg, StAppOutLen, StAppProcess, StAppWait: begin
1309         // The key from KeyMgr is used if the current HW app
interface does *keyed* MAC. This
1310         // module does not know the precise timing related to
when the kmac_core module uses the
1311         // key during processing. To be on the safe side, we
check that the key remains valid
1312         // directly after latching the configuration and
throughout the entire processing until the
1313         // digest is received. The key is used by the
kmac_core also after the kmac_app finished

1314         // sending the message. So we must check the key until
we receive the digest because we
1315         // don't know here when the hashing engine starts the
processing.
1316         keymgr_key_used = app_cfg.session_cfg.mode == AppKMAC;
1317         key_len_o       = SideloadedKey;
1318         for (int i = 0; i < Share; i++) begin
1319             key_data_o[i] = keymgr_key[i];
1320         end
1321         // Key is valid if the current HW app interface does
*keyed* MAC and the key provided by
1322         // KeyMgr is valid.
1323         key_valid_o = keymgr_key_used && keymgr_key_i.valid;
1324     end
1325
1326     StSw: begin
1327         if (keymgr_key_en_i) begin
1328             // Key from KeyMgr is actually used if *keyed* MAC
is enabled.
1329             keymgr_key_used = kmac_en_q;
1330             key_len_o       = SideloadedKey;
1331             for (int i = 0; i < Share; i++) begin
1332                 key_data_o[i] = keymgr_key[i];
1333             end
1334         end
1335         // Key is valid if SW does *keyed* MAC and ...
1336         if (kmac_en_q) begin
1337             if (!keymgr_key_en_i) begin
1338                 // ... it uses the key from kmac's CSR, or ...
1339                 key_valid_o = 1'b1;
1340             end else begin
1341                 // ... it uses the key provided by KeyMgr and that
one is valid.
1342                 key_valid_o = keymgr_key_i.valid;
1343             end
1344         end
1345     end
1346
1347     default: ;
1348 endcase
1349 end
1350
1351 // Prefix Demux
1352 // For SW, always take the prefix from the register. For
apps chose depending on configuration.
1353 always_comb begin
1354     sha3_prefix_o = '0;
1355
1356     unique case (st)
1357         StAppCfg, StAppMsg, StAppOutLen, StAppProcess,
StAppWait, StAppPushDigest: begin
1358             if (app_cfg.session_cfg.prefix_mode == 1'b0) begin
1359                 sha3_prefix_o = reg_prefix_i;
1360             end else begin
1361                 sha3_prefix_o = app_cfg.prefix;
1362             end
1363         end
1364     end
1365
1366     StSw: begin
1367         sha3_prefix_o = reg_prefix_i;
1368     end
1369
1370     default: begin
1371         sha3_prefix_o = reg_prefix_i;
1372     end
1373 endcase
1374 end
1375
1376 // Status - exclude error states so SW still can write mes-
sages during error recovery.
1377 assign app_active_o = (st inside {StAppCfg, StAppMsg, StAp-
pOutLen, StAppProcess, StAppWait,
                                StAppPushDigest, StAppFin-
ish});
1378
1379 // Error Reporting
=====
1380 always_comb begin
1381     priority casez ({fsm_err.valid, mux_err.valid})
1382         2'b ?1: error_o = mux_err;
1383         2'b 10: error_o = fsm_err;
1384         default: error_o = '{valid: 1'b0, code: ErrNone, info:
'0};
1385     endcase
1386 end
1387
1388 // Assertions //
1389

```

```

1390 //////////////////////////////////////////////////
1391
1392 // KeyMgr sideload key and the digest should be in the Key
Length value
1393 `ASSERT_INIT(SideloadKeySameToDigest_A, KeyMgrKeyW <= AppDi-
gestW)
1394 `ASSERT_INIT(AppIntfInRange_A, AppDigestW inside {128, 192,
256, 384, 512})
1395
1396 // Issue(#13655): Having a coverage that sideload keylen and
CSR keylen are
1397 // different.
1398 `COVER(AppIntfUseDifferentSizeKey_C,
1399         (st == StAppCfg && kmac_en_q) |-> reg_key_len_i != Side-
loadedKey)
1400
1401 endmodule

```

5. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_core.sv (470 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // KMAC control and padding logic
6
7 `include "prim_assert.sv"
8
9 module kmac_core
10     import kmac_pkg::*;
11 #()
12 // EnMasking: Enable masking security hardening inside kec-
cak_round
13 // If it is enabled, the result digest will be two set of
1600bit.
14     parameter bit EnMasking = 0,
15     localparam int Share = (EnMasking) ? 2 : 1 // derived para-
meter
16 ) (
17     input clk_i,
18     input rst_ni,
19
20     // From Message FIFO
21     input fifo_valid_i,
22     input [MsgWidth-1:0] msg_data_i [Share],
23     input [MsgStrbW-1:0] fifo_strb_i,
24     output logic fifo_ready_o,
25
26     // to SHA3 Core
27     output logic msg_valid_o,
28     output logic [MsgWidth-1:0] msg_data_o [Share],
29     output logic [MsgStrbW-1:0] msg_strb_o,
30     input msg_ready_i,
31
32     // Configurations
33
34     // If kmac_en is cleared, Core logic doesn't function but
forward incoming
35     // message to SHA3 core
36     input kmac_en_i,
37     input sha3_pkg::sha3_mode_e mode_i,
38     input sha3_pkg::keccak_strength_e strength_i,
39
40     // Key input from CSR
41     input [MaxKeyLen-1:0] key_data_i [Share],
42     input key_len_e key_len_i,
43     input logic key_valid_i,
44
45     // Controls : same to SHA3 core
46     input start_i,
47     input process_i,
48     input prim_mubi_pkg::mubi4_t done_i,
49
50     // Control to SHA3 core
51     output logic process_o,
52
53     // Life cycle
54     input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
55
56     output logic sparse_fsm_error_o,
57     output logic key_index_error_o
58 );
59
60 import sha3_pkg::KeccakMsgAddrW;
61 import sha3_pkg::KeccakCountW;
62 import sha3_pkg::KeccakRate;
63 import sha3_pkg::L128;
64 import sha3_pkg::L224;
65 import sha3_pkg::L256;
66 import sha3_pkg::L384;
67 import sha3_pkg::L512;
68
69 //////////////////////////////////////////////////
70 // Definitions //
71 //////////////////////////////////////////////////
72
73 // Encoding generated with:

```

```

74 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 5 -n 6 \
75 // -s 401658243 --language=sv
76 //
77 // Hamming distance histogram:
78 //
79 // 0: --
80 // 1: --
81 // 2: --
82 // 3: ||||| (50.00%)
83 // 4: ||||| (40.00%)
84 // 5: |||| (10.00%)
85 // 6: --
86 //
87 // Minimum Hamming distance: 3
88 // Maximum Hamming distance: 5
89 // Minimum Hamming weight: 1
90 // Maximum Hamming weight: 4
91 //
92 localparam int StateWidth = 6;
93 typedef enum logic [StateWidth-1:0] {
94     StKmacIdle = 6'b011000,
95
96     // Secret Key pushing stage
97
98 // The key is sliced by prim_slicer. This state pushes the
99 // SHA3 hashing engine. When it hits the block size limit,
100 // (same as in sha3pad) the state machine moves to Mes-
101 sage.
102 StKey = 6'b010111,
103
104 // Incoming Message
105 // The core does nothing but forwarding the incoming mes-
106 sage to SHA3 hashing
107 // engine by turning off `en_kmac_datapath`.
108 StKmacMsg = 6'b001110,
109
110 // Wait till done signal
111 StKmacFlush = 6'b101011,
112
113 // Terminal Error
114 StTerminalError = 6'b100000
115 } kmac_st_e ;
116
117 // Signals //
118 // represents encode_string(K)
119 logic [MaxEncodedKeyW-1:0] encoded_key [Share];
120
121 // Key slice address
122 // This signal controls the 64 bit output of the sliced
123 secret_key.
124 logic [sha3_pkg::KeccakMsgAddrW-1:0] key_index;
125 logic inc_keyidx, clr_keyidx;
126
127 // `sent_blocksize` indicates that the encoded key is sent
128 to sha3 hashing
129 // engine. If this hits at StKey stage, the state moves to
130 message
131 state.
132 logic [sha3_pkg::KeccakCountW-1:0] block_addr_limit;
133 logic sent_blocksize;
134
135 // Internal message signals
136 logic kmac_valid ;
137 logic [MsgWidth-1:0] kmac_data [Share];
138 logic [MsgStrbW-1:0] kmac_strb ;
139
140 // Control SHA3 core
141 // `kmac_process` is to forward the process signal to SHA3
142 core only after
143
144 // the KMAC core writes the key block in case of the message
145 is empty.
146
147 // If the incoming message is empty, there's chance that the
148 `process_i`
149
150 // signal can be asserted while KMAC core processing the key
151 block.
152 logic kmac_process, process_latched;
153
154 // Indication of Secret key write stage. Only in this stage,
155 the internal
156 // message interface is active.
157 logic en_key_write;
158 logic en_kmac_datapath;
159
160 // Encoded key has wider bits. `key_sliced` is the data to
161 send to sha3
162 logic [MsgWidth-1:0] key_sliced [Share];
163
164 // The following signals are only used in assertions.
165 logic unused_signals;
166 assign unused_signals = ^{mode_i, key_valid_i};
167
168 // FSM //
169

```

```

158 kmac_st_e st, st_d;
159
160 // State register
161 PRIM_FLOP_SPARSE_FSM(u_state_regs, st_d, st, kmac_st_e,
162 StKmacIdle)
163
164 // Next state and output logic
165 // SEC_CM: FSM.SPARSE
166 always_comb begin
167     st_d = st;
168
169     en_kmac_datapath = 1'b 0;
170     en_key_write = 1'b 0;
171
172     clr_keyidx = 1'b 0;
173
174     kmac_valid = 1'b 0;
175     kmac_process = 1'b 0;
176
177     sparse_fsm_error_o = 1'b 0;
178
179     unique case (st)
180         StKmacIdle: begin
181             if (kmac_en_i && start_i) begin
182                 st_d = StKey;
183             end else begin
184                 st_d = StKmacIdle;
185             end
186         end
187         // If State enters here, regardless of the `process_i`,
188         the state writes
189         // full block size of the key into SHA3 hashing engine.
190         StKey: begin
191             en_kmac_datapath = 1'b 1;
192             en_key_write = 1'b 1;
193
194             if (sent_blocksize) begin
195                 st_d = StKmacMsg;
196
197                 kmac_valid = 1'b 0;
198                 clr_keyidx = 1'b 1;
199             end else begin
200                 st_d = StKey;
201
202                 kmac_valid = 1'b 1;
203             end
204         end
205         StKmacMsg: begin
206             // If process is previously latched, it is sent to
207             SHA3 here.
208             if (process_i || process_latched) begin
209                 st_d = StKmacFlush;
210
211                 kmac_process = 1'b 1;
212             end else begin
213                 st_d = StKmacMsg;
214             end
215         end
216         StKmacFlush: begin
217             if (prim_mubi_pkg::mubi4_test_true_strict(done_i)) be-
218 gin
219                 st_d = StKmacIdle;
220             end else begin
221                 st_d = StKmacFlush;
222             end
223         end
224         StTerminalError: begin
225             // this state is terminal
226             st_d = st;
227             sparse_fsm_error_o = 1'b 1;
228         end
229         default: begin
230             // this state is terminal
231             st_d = StTerminalError;
232             sparse_fsm_error_o = 1'b 1;
233         end
234     endcase
235
236     // SEC_CM: FSM.GLOBAL_ESC, FSM.LOCAL_ESC
237     // Unconditionally jump into the terminal error state
238     // if the life cycle controller triggers an escalation.
239     if (lc_ctrl_pkg::lc_tx_test_true_loose(lc_escalate_en_i))
240         begin
241             st_d = StTerminalError;
242         end
243     end
244
245     // Datapath //
246
247     // DATA Mux depending on kmac_en
248     // When Key write happens, hold the FIFO request. so
249     fifo_ready_o is tied to 0
250     assign msg_valid_o = (en_kmac_datapath) ? kmac_valid :
251     fifo_valid_i;

```

```

252 assign msg_data_o = (en_kmac_datapath) ? kmac_data :
fifo_data_i ;
253 assign msg_strb_o = (en_kmac_datapath) ? kmac_strb :
fifo_strb_i ;
254 assign fifo_ready_o = (en_kmac_datapath) ? 1'b 0 :
msg_ready_i ;
255

256 // secret key write request to SHA3 hashing engine is always
full width write.

257 // KeyMgr is fixed 256 bit output. So `right_encode(256)` is
0x020100 --> strb 3
258 assign kmac_strb = (en_key_write) ? 1 : '0;
259
260 assign kmac_data = (en_key_write) ? key_sliced : '{de-
fault:'0;
261
262 // Process is controlled by the KMAC core always.
263 // This is mainly to prevent process_i asserted while KMAC
core is writing
264 // the secret key to SHA3 hashing engine (the empty message
case)
265 assign process_o = (kmac_en_i) ? kmac_process : process_i ;
266
267 always_ff @(posedge clk_i or negedge rst_ni) begin
268   if (!rst_ni) begin
269     process_latched <= 1'b 0;
270   end else if (process_i && !process_o) begin
271     process_latched <= 1'b 1;
272   end else if (process_o ||
273     prim_mubi_pkg::mubi4_test_true_strict(done_i)) begin
274     process_latched <= 1'b 0;
275   end
276 end
277
278 // bytepad(encode_string(K), 168 or 136)
=====
279 // 1. Prepare left_encode(w)
280 // 2. Prepare left_encode(len(secret_key))
281 // 3. Concatenate left_encode(len(secret_key)) || secret_key
282 // 4. Concatenate left_encode(w) || en-
code_string(secret_key)

283 // 5. Based on the address, slice out the data into MsgWidth
bits
284
285 // left_encode(w): Same as used in sha3pad logic.
286 logic [15:0] encode_bytepad;
287 assign encode_bytepad = sha3_pkg::en-
code_bytepad_len(strength_i);
288
289 // left_encode(len(secret_key))
290 // encoded length is always byte size. Use MaxEncodedKeyLen-
Byte parameter
291 // from kmac_pkg and add one more byte to indicate how many
bytes used to
292 // represent len(secret_key)
293 // Note that if the secret_key is 128 bit, only lower 16
bits of
294 // `encode_keylen` are valid. Refer `encoded_key` concatena-
tion logic below.
295 // As the encoded string in the spec big-endian, The endian
swap is a must.
296 logic [MaxEncodedKeyLenSize + 8 - 1:0] encode_keylen
[Share];
297
298 always_comb begin
299   // the spec mentioned the key length is encoded in
left_encode()
300   // The number is represented in big-endian. For example:
301   // 384 ==> 0x02 0x01 0x80
302   // The first byte is the number of bytes to represent 384
303   // The second byte represents 2**8 number, which is 256
here.
304   // The third byte represents 2**16 number, which is 128.
305   // The data put into MsgFIFO is little-endian and
SHA3(Keccak) processes in
306   // little-endian. So, below keylen swaps the byte order
307   unique case (key_len_i)
308     //
endian-swapped
key_length      num_bytes
309   // Key128: encode_keylen[0] = {<<8{MaxEncodedKeyLenS-
ize'(128)}}, 8'h 01;
310   // Key192: encode_keylen[0] = {<<8{MaxEncodedKeyLenS-
ize'(192)}}, 8'h 01;
311   // Key256: encode_keylen[0] = {<<8{MaxEncodedKeyLenS-
ize'(256)}}, 8'h 02;
312   // Key384: encode_keylen[0] = {<<8{MaxEncodedKeyLenS-
ize'(384)}}, 8'h 02;
313   // Key512: encode_keylen[0] = {<<8{MaxEncodedKeyLenS-
ize'(512)}}, 8'h 02;
314
315   // Vivado does not support stream swap for non context
value. So assign
316   // the value directly.
317   Key128: encode_keylen[0] = (MaxEncodedKeyLenSize+8)'('h
0080_01);
318   Key192: encode_keylen[0] = (MaxEncodedKeyLenSize+8)'('h
00C0_01);
319   Key256: encode_keylen[0] = (MaxEncodedKeyLenSize+8)'('h

```

```

0001_02);
320   Key384: encode_keylen[0] = (MaxEncodedKeyLenSize+8)'('h
8001_02);
321   Key512: encode_keylen[0] = (MaxEncodedKeyLenSize+8)'('h
0002_02);
322   default: encode_keylen[0] = '0;
323 endcase
324 end
325
326 if (EnMasking) begin: gen_encode_keylen_masked
327   assign encode_keylen[1] = '0;
328 end
329
330 // encode_string(secret_key): Concatenate key
331 // Based on the left_encode(len(secret_key)) size, the con-
catenation logic
332 // should be changed. If key length is 128 bit, only lower
16 bits of the
333 // encoded length are used so that the upper 8 bits are pad-
ded with 0 as
334 // defined in bytepad() function.
335
336 for (genvar i = 0 ; i < Share; i++) begin : gen_encoded_key
337   always_comb begin
338     unique case (key_len_i)
339       // In Key 128, 192 case, only lower parts of en-
code_keylen signal is
340       // used. So upper padding requires 8 more bits than
MaxKeyLen - keylen
341       Key128: encoded_key[i] = {(8 + MaxKeyLen - 128)'(0),
key_data_i[i][0+:128],
342       encode_keylen[i][0+:MaxEn-
codedKeyLenSize]};
343
344       Key192: encoded_key[i] = {(8 + MaxKeyLen - 192)'(0),
key_data_i[i][0+:192],
345       encode_keylen[i][0+:MaxEn-
codedKeyLenSize]};
346
347       Key256: encoded_key[i] = {(MaxKeyLen - 256)'(0),
key_data_i[i][0+:256],
348       encode_keylen[i]};
349
350       Key384: encoded_key[i] = {(MaxKeyLen - 384)'(0),
key_data_i[i][0+:384],
351       encode_keylen[i]};
352
353       // Assume 512bit is the MaxKeyLen
354       Key512: encoded_key[i] = {key_data_i[i][0+:512],
encode_keylen[i]};
355
356       default: encoded_key[i] = '0;
357     endcase
358   end
359 end : gen_encoded_key
360
361 // Above logic assumes MaxKeyLen as 512 bits. Revise if it
is not.
362 `ASSERT_INIT(MaxKeyLenMatchToKey512_A, kmac_pkg::MaxKeyLen
== 512)
363
364 // Combine the bytepad `left_encode(w)` and the `en-
code_string(secret_key)`
365 logic [MaxEncodedKeyW + 16 - 1:0] encoded_key_block [Share];
366
367 assign encoded_key_block[0] = {encoded_key[0], en-
code_bytepad};
368
369 if (EnMasking) begin : gen_encoded_key_block_masked
370   assign encoded_key_block[1] = {encoded_key[1], 16'h 0};
371 end
372
373 // Slicer to slice out 64 bits
374 for (genvar i = 0 ; i < Share ; i++) begin : gen_key_slicer
375   prim_slicer #(
376     .InW (MaxEncodedKeyW+16),
377     .IndexW (KeccakMsgAddrW),
378     .OutW (MsgWidth)
379   ) u_key_slicer (
380     .sel_i (key_index),
381     .data_i (encoded_key_block[i]),
382     .data_o (key_sliced[i])
383   );
384 end
385
386 // `key_index` logic
387 // key_index is used to select MsgWidth data from long `en-
coded_key_block`
388 // It behaves same as `keccak_addr` or `prefix_index` in
sha3pad module.
389 assign inc_keyidx = kmac_valid & msg_ready_i ;
390
391 // This primitive is used to place a hardened counter
392 // SEC_CM: CTR.REDUN
393 prim_count #(
394   .Width (sha3_pkg::KeccakMsgAddrW)
395 ) u_key_index_count (
396   .clk_i,
397   .rst_ni,
398   .clr_i (clr_keyidx),
399   .set_i (1'b0),

```



```

405 .set_cnt_i('0),
406 .incr_en_i(inc_keyidx),
407 .decr_en_i(1'b0),
408 .step_i(sha3_pkg::KeccakMsgAddrW'(1)),
409 .commit_i(1'b1),
410 .cnt_o(key_index),
411 .cnt_after_commit_o(),
412 .err_o(key_index_error_o)
413 );
414
415 // Block size based on the address.
416 // This is used for bytepad() and also pad10*1()
417 // assign block_addr_limit = KeccakRate[strength_i];
418 // but below is easier to understand
419 always_comb begin
420   unique case (strength_i)
421     L128: block_addr_limit = KeccakCoun-
tw'(KeccakRate[L128]);
422     L224: block_addr_limit = KeccakCoun-
tw'(KeccakRate[L224]);
423     L256: block_addr_limit = KeccakCoun-
tw'(KeccakRate[L256]);
424     L384: block_addr_limit = KeccakCoun-
tw'(KeccakRate[L384]);
425     L512: block_addr_limit = KeccakCoun-
tw'(KeccakRate[L512]);
426     default: block_addr_limit = '0;
427   endcase
428 end
429
430 assign sent_blocksize = (key_index == block_addr_limit);
431
432
433 // Encoded Output Length
434 =====
435 //
436 // KMAC(K,X,L,S) := cSHAKE(newX,L,"KMAC",S)
437 // K : Secret Key
438 // X : Input Message
439 // L : Output Length

```

```

440 // S : Customization input string
441 // newX = bytepad(encode_string(key), 168or136) || X ||
right_encode(L)
442 //
443 // Software writes desired output length as encoded value
into the message
444 // FIFO at the end of the message prior to set !!CMD.pro-
cess.
445
446 ///////////////////////////////////////////////////////////////////
447 // Assertions //
448 ///////////////////////////////////////////////////////////////////
449
450
451 // If process_latched is set, then at Message state, it
should be cleared
452
453 `ASSERT(ProcessLatchedCleared_A,
454   st == StKmacMsg && process_latched ==> !pro-
cess_latched)
455
456 // Assume configuration is stable during the operation
457 `ASSUME(KmacEnStable_M, $changed(kmac_en_i) ==> st inside
{StKmacIdle, StTerminalError})
458 `ASSUME(ModeStable_M, $changed(mode_i) ==> st inside {StKma-
cIdle, StTerminalError})
459 `ASSUME(StrengthStable_M,
460   $changed(strength_i) ==>
461   (st inside {StKmacIdle, StTerminalError}) ||
462   ($past(st) == StKmacIdle))
463 `ASSUME(KeyLengthStableWhenValid_M, key_valid_i && !
$rose(key_valid_i) ==> $stable(key_len_i))
464 `ASSUME(KeyDataStableWhenValid_M, key_valid_i && !
$rose(key_valid_i) ==> $stable(key_data_i))
465
466 // no acked to MsgFIFO in StKmacMsg
467 `ASSERT(AckOnlyInMessageState_A,
468   fifo_valid_i && fifo_ready_o && kmac_en_i ==> st ==
StKmacMsg)
469
470 endmodule : kmac_core

```

OpenTitan CSRNG (SP800-90A)

What this is. A CTR_DRBG deterministic random bit generator with arbitration among multiple requesters.

Why it exists. Because the standard prescribes a procedure, not merely an output. Instantiate, reseed, generate and un-instantiate form a specified state machine with specified counter limits, and certification requires conformance to it.

Undergraduate review · EECS concepts. Pseudorandom versus cryptographically secure generation; forward and backward secrecy; entropy pool management; state machine design.

Graduate · Practice note. Verification here means “does it follow the specified procedure?” The NIST CAVP vectors are effectively the specification. This is a block where inventiveness in the model is a liability rather than an asset.

8 files • 3,151 lines

API — csrng.sv

Table 832. lowRISC_opentitan/hw/ip/csrng/rtl/csrng.sv — module / interface / package (1개)

이름	종류
csrng	module

Table 833. lowRISC_opentitan/hw/ip/csrng/rtl/csrng.sv — parameter (5개)

파라미터	형/폭	기본값
SBoxImpl	aes_pkg::sbox_impl_e	aes_pkg::SBoxImplCanright
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
RndCnstCsKeymgrDivNonProduction	cs_keymgr_div_t	CsKeymgrDivWidth'(0)
RndCnstCsKeymgrDivProduction	cs_keymgr_div_t	CsKeymgrDivWidth'(0)

Table 834. lowRISC_opentitan/hw/ip/csrng/rtl/csrng.sv — localparam (1개)

상수	형/폭	값
NumHwApps	int unsigned	NumApps - 1 // derived parameter

Table 835. lowRISC_opentitan/hw/ip/csrng/rtl/csrng.sv — 포트 (16개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
otp_en_csrng_sw_app_read_i	input	prim_mubi_pkg::mubi8_t
lc_hw_debug_en_i	input	lc_ctrl_pkg::lc_tx_t

포트	방향	형/폭
entropy_src_hw_if_o	output	entropy_src_pkg::entropy_src_hw_if_req_t
entropy_src_hw_if_i	input	entropy_src_pkg::entropy_src_hw_if_rsp_t
csrng_cmd_i	input	csrng_req_t [NumHwApps-1:0]
csrng_cmd_o	output	csrng_rsp_t [NumHwApps-1:0]
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
intr_cs_cmd_req_done_o	output	logic
intr_cs_entropy_req_o	output	logic
intr_cs_hw_inst_exc_o	output	logic
intr_cs_fatal_err_o	output	logic

API — csrng_block_encrypt.sv

Table 836. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_block_encrypt.sv — module / interface / package (1개)

이름	종류
csrng_block_encrypt	module

Table 837. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_block_encrypt.sv — parameter (1개)

파라미터	형/폭	기본값
SBoxImpl	aes_pkg::sbox_impl_e	aes_pkg::SBoxImplLut

Table 838. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_block_encrypt.sv — localparam (1개)

상수	형/폭	값
NumShares	int unsigned	1

Table 839. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_block_encrypt.sv — 포트 (10개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
enable_i	input	logic
req_vld_i	input	logic
req_rdy_o	output	logic
req_data_i	input	csrng_key_v_t
rsp_vld_o	output	logic
rsp_rdy_i	input	logic
rsp_data_o	output	logic [BlkLen-1:0]
cipher_sm_err_o	output	logic

API — csrng_cmd_stage.sv**Table 840. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_cmd_stage.sv — module / interface / package (1개)**

이름	종류
csrng_cmd_stage	module

Table 841. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_cmd_stage.sv — localparam (3개)

상수	형/폭	값
GenBitsFifoWidth	int	1 + BlkLen
GenBitsFifoDepth	int	1
StateWidth	int	8

Table 842. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_cmd_stage.sv — 포트 (32개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
cs_enable_i	input	logic
cmd_stage_vld_i	input	logic
cmd_stage_shid_i	input	logic [InstIdWidth-1:0]
cmd_stage_bus_i	input	logic [CmdBusWidth-1:0]
cmd_stage_rdy_o	output	logic
reseed_cnt_reached_i	input	logic
reseed_cnt_alert_o	output	logic
invalid_cmd_seq_alert_o	output	logic
invalid_acmd_alert_o	output	logic
cmd_arb_req_o	output	logic
cmd_arb_sop_o	output	logic
cmd_arb_mop_o	output	logic
cmd_arb_eop_o	output	logic
cmd_arb_gnt_i	input	logic
cmd_arb_bus_o	output	logic [CmdBusWidth-1:0]
cmd_ack_i	input	logic
cmd_ack_sts_i	input	csrng_cmd_sts_e
cmd_stage_ack_o	output	logic
cmd_stage_ack_sts_o	output	csrng_cmd_sts_e
genbits_vld_i	input	logic
genbits_bus_i	input	logic [127:0]
genbits_fips_i	input	logic

포트	방향	형/폭
genbits_vld_o	output	logic
genbits_rdy_i	input	logic
genbits_bus_o	output	logic [127:0]
genbits_fips_o	output	logic
cmd_stage_sfifo_cmd_err_o	output	logic [2:0]
cmd_stage_sfifo_genbits_err_o	output	logic [2:0]
cmd_gen_cnt_err_o	output	logic
cmd_stage_sm_err_o	output	logic

API — csrng_core.sv

Table 843. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_core.sv — module / interface / package (1개)

이름	종류
csrng_core	module

Table 844. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_core.sv — parameter (4개)

파라미터	형/폭	기본값
SBoxImpl	aes_pkg::sbox_impl_e	aes_pkg::SBoxImplLut
NumHwApps	int	2
RndCnstCsKeymgrDivNonProduction	cs_keymgr_div_t	CsKeymgrDivWidth'(0)
RndCnstCsKeymgrDivProduction	cs_keymgr_div_t	CsKeymgrDivWidth'(0)

Table 845. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_core.sv — localparam (4개)

상수	형/폭	값
ADataDepthClog	int unsigned	\$clog2(CmdMaxClen) + 1
CsEnableCopies	int unsigned	51
LcHwDebugCopies	int unsigned	1
Flag0Copies	int unsigned	3

Table 846. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_core.sv — 포트 (18개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
reg2hw	input	csrng_reg_pkg::csrng_reg2hw_t
hw2reg	output	csrng_reg_pkg::csrng_hw2reg_t
otp_en_csrng_sw_app_read_i	input	prim_mubi_pkg::mubi8_t
lc_hw_debug_en_i	input	lc_ctrl_pkg::lc_tx_t
entropy_src_hw_if_o	output	entropy_src_pkg::entropy_src_hw_if_req_t

포트	방향	형/폭
entropy_src_hw_if_i	input	entropy_src_pkg::entropy_src_hw_if_rsp_t
csrng_cmd_i	input	csrng_req_t [NumHwApps-1:0]
csrng_cmd_o	output	csrng_rsp_t [NumHwApps-1:0]
recov_alert_test_o	output	logic
fatal_alert_test_o	output	logic
recov_alert_o	output	logic
fatal_alert_o	output	logic
intr_cs_cmd_req_done_o	output	logic
intr_cs_entropy_req_o	output	logic
intr_cs_hw_inst_exc_o	output	logic
intr_cs_fatal_err_o	output	logic

API — csrng_ctr_drbg.sv

Table 847. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_ctr_drbg.sv — module / interface / package (1개)

이름	종류
csrng_ctr_drbg	module

Table 848. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_ctr_drbg.sv — localparam (3개)

상수	형/폭	값
NumBlkPerUpd	int unsigned	SeedLen/BlkLen
NumBlkPerUpdLg	int unsigned	$\$clog2(\text{NumBlkPerUpd})$
StateWidth	int	6

Table 849. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_ctr_drbg.sv — 포트 (24개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
enable_i	input	logic
req_vld_i	input	logic
req_rdy_o	output	logic
req_data_i	input	csrng_core_data_t
req_entropy_i	input	logic [SeedLen-1:0]
req_entropy_fips_i	input	logic
req_glast_i	input	logic
state_db_wr_o	output	logic
rsp_vld_o	output	logic

포트	방향	형/폭
rsp_data_o	output	csrng_core_data_t
rsp_sts_o	output	csrng_cmd_sts_e
bits_vld_o	output	logic
bits_data_o	output	logic [BlkLen-1:0]
bits_fips_o	output	logic
block_encrypt_req_vld_o	output	logic
block_encrypt_req_rdy_i	input	logic
block_encrypt_req_data_o	output	csrng_key_v_t
block_encrypt_rsp_vld_i	input	logic
block_encrypt_rsp_rdy_o	output	logic
block_encrypt_rsp_data_i	input	logic [BlkLen-1:0]
ctr_err_o	output	logic
sm_err_o	output	logic

API — csrng_main_sm.sv

Table 850. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_main_sm.sv — module / interface / package (1개)

이름	종류
csrng_main_sm	module

Table 851. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_main_sm.sv — 포트 (17개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
enable_i	input	logic
acmd_avail_i	input	logic
acmd_accept_o	output	logic
acmd_i	input	acmd_e
acmd_eop_i	input	logic
flag0_i	input	logic
cmd_entropy_req_o	output	logic
cmd_entropy_avail_i	input	logic
cmd_vld_o	output	logic
cmd_rdy_i	input	logic
clr_adata_packer_o	output	logic
cmd_complete_i	input	logic
local_escalate_i	input	logic

포트	방향	형/폭
main_sm_state_o	output	logic [MainSmStateWidth-1:0]
main_sm_err_o	output	logic

API — csrng_pkg.sv

Table 852. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_pkg.sv — module / interface / package (1개)

이름	종류
csrng_pkg	package

Table 853. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_pkg.sv — parameter (21개)

파라미터	형/폭	기본값
NumAppsLg	int unsigned	\$clog2(csrng_reg_pkg::NumApps)
BlkLen	int unsigned	128
KeyLen	int unsigned	256
SeedLen	int unsigned	BlkLen + KeyLen
CtrLen	int unsigned	32
RsCtrWidth	int unsigned	32
GenBitsCtrWidth	int unsigned	12
CmdWidth	int unsigned	3
InstIdWidth	int unsigned	4
CmdBusWidth	int unsigned	32
CmdMaxClen	int unsigned	12
CmdFifoDepth	int unsigned	2
CmdFifoDepthLg	int unsigned	\$clog2(CmdFifoDepth)
GENBITS_BUS_WIDTH	int unsigned	BlkLen
FIPS_GENBITS_BUS_WIDTH	int unsigned	entropy_src_pkg::FIPS_BUS_WIDTH +
CmdStatusWidth	int unsigned	3
CSRNG_REQ_DEFAULT	csrng_req_t	{default: '0}
CSRNG_RSP_DEFAULT	csrng_rsp_t	'0
StateWidth	int unsigned	\$bits(csrng_state_t)
MainSmStateWidth	int unsigned	6
CsKeymgrDivWidth	int	384

Table 854. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_pkg.sv — typedef / using (1개)

새 이름	원래 형
cs_keymgr_div_t	logic [CsKeymgrDivWidth-1:0]

API — csrng_reg_pkg.sv**Table 855. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_reg_pkg.sv — module / interface / package (1개)**

이름	종류
csrng_reg_pkg	package

Table 856. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_reg_pkg.sv — parameter (45개)

파라미터	형/폭	기본값
NumApps	int	3
NumAlerts	int	2
BlockAw	int	7
NumRegs	int	24
CSRNG_INTR_STATE_OFFSET	logic [BlockAw-1:0]	7'h 0
CSRNG_INTR_ENABLE_OFFSET	logic [BlockAw-1:0]	7'h 4
CSRNG_INTR_TEST_OFFSET	logic [BlockAw-1:0]	7'h 8
CSRNG_ALERT_TEST_OFFSET	logic [BlockAw-1:0]	7'h c
CSRNG_REGWEN_OFFSET	logic [BlockAw-1:0]	7'h 10
CSRNG_CTRL_OFFSET	logic [BlockAw-1:0]	7'h 14
CSRNG_CMD_REQ_OFFSET	logic [BlockAw-1:0]	7'h 18
CSRNG_RESEED_INTERVAL_OFFSET	logic [BlockAw-1:0]	7'h 1c
CSRNG_RESEED_COUNTER_0_OFFSET	logic [BlockAw-1:0]	7'h 20
CSRNG_RESEED_COUNTER_1_OFFSET	logic [BlockAw-1:0]	7'h 24
CSRNG_RESEED_COUNTER_2_OFFSET	logic [BlockAw-1:0]	7'h 28
CSRNG_SW_CMD_STS_OFFSET	logic [BlockAw-1:0]	7'h 2c
CSRNG_GENBITS_VLD_OFFSET	logic [BlockAw-1:0]	7'h 30
CSRNG_GENBITS_OFFSET	logic [BlockAw-1:0]	7'h 34
CSRNG_INT_STATE_READ_ENABLE_OFFSET	logic [BlockAw-1:0]	7'h 38
CSRNG_INT_STATE_READ_ENABLE_REGWEN_OFFSET	logic [BlockAw-1:0]	7'h 3c
CSRNG_INT_STATE_NUM_OFFSET	logic [BlockAw-1:0]	7'h 40
CSRNG_INT_STATE_VAL_OFFSET	logic [BlockAw-1:0]	7'h 44
CSRNG_FIPS_FORCE_OFFSET	logic [BlockAw-1:0]	7'h 48
CSRNG_HW_EXC_STS_OFFSET	logic [BlockAw-1:0]	7'h 4c
CSRNG_RECOV_ALERT_STS_OFFSET	logic [BlockAw-1:0]	7'h 50
CSRNG_ERR_CODE_OFFSET	logic [BlockAw-1:0]	7'h 54
CSRNG_ERR_CODE_TEST_OFFSET	logic [BlockAw-1:0]	7'h 58
CSRNG_MAIN_SM_STATE_OFFSET	logic [BlockAw-1:0]	7'h 5c
CSRNG_INTR_TEST_RESVAL	logic [3:0]	4'h 0

파라미터	형/폭	기본값
CSRNG_INTR_TEST_CS_CMD_REQ_DONE_RESVAL	logic [0:0]	1'h 0
CSRNG_INTR_TEST_CS_ENTROPY_REQ_RESVAL	logic [0:0]	1'h 0
CSRNG_INTR_TEST_CS_HW_INST_EXC_RESVAL	logic [0:0]	1'h 0
CSRNG_INTR_TEST_CS_FATAL_ERR_RESVAL	logic [0:0]	1'h 0
CSRNG_ALERT_TEST_RESVAL	logic [1:0]	2'h 0
CSRNG_ALERT_TEST_RECOV_ALERT_RESVAL	logic [0:0]	1'h 0
CSRNG_ALERT_TEST_FATAL_ALERT_RESVAL	logic [0:0]	1'h 0
CSRNG_RESEED_COUNTER_0_RESVAL	logic [31:0]	32'h 0
CSRNG_RESEED_COUNTER_0_RESEED_COUNTER_0_RESVAL	logic [31:0]	32'h 0
CSRNG_RESEED_COUNTER_1_RESVAL	logic [31:0]	32'h 0
CSRNG_RESEED_COUNTER_1_RESEED_COUNTER_1_RESVAL	logic [31:0]	32'h 0
CSRNG_RESEED_COUNTER_2_RESVAL	logic [31:0]	32'h 0
CSRNG_RESEED_COUNTER_2_RESEED_COUNTER_2_RESVAL	logic [31:0]	32'h 0
CSRNG_GENBITS_VLD_RESVAL	logic [1:0]	2'h 0
CSRNG_GENBITS_RESVAL	logic [31:0]	32'h 0
CSRNG_INT_STATE_VAL_RESVAL	logic [31:0]	32'h 0

1. lowRISC_opentitan/hw/ip/CSRNG/rtl/CSRNG.sv (203 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: CSRNG top level wrapper file
6
7 `include "prim_assert.sv"
8
9 module CSRNG
10   import CSRNG_pkg::*;
11   import CSRNG_reg_pkg::*;
12   #(
13     parameter aes_pkg::sbox_impl_e SBoxImpl = aes_pkg::SBoxIm-
  pLCanright,
14     parameter logic [NumAlerts-1:0] AlertAsyncOn = {Nu-
  mAlerts{1'b1}},
15     // Number of cycles a differential skew is tolerated on the
  alert signal
16     parameter int unsigned AlertSkewCycles = 1,
17     parameter cs_keymgr_div_t RndCnstCsKeymgrDivNonProduction =
  CsKeymgrDivWidth'(0),
18     parameter cs_keymgr_div_t RndCnstCsKeymgrDivProduction = Cs-
  KeymgrDivWidth'(0),
19     localparam int unsigned NumHwApps = NumApps - 1 // derived
  parameter
20   ) (
21     input  logic clk_i,
22     input  logic rst_ni,
23
24     // TileLink Bus Interface
25     input  tlul_pkg::tl_h2d_t tl_i,
26     output tlul_pkg::tl_d2h_t tl_o,
27
28     // OTP Interface
29     // SEC_CM: INTERSIG.MUBI
30     input  prim_mubi_pkg::mubi8_t otp_en_CSRNG_sw_app_read_i,
31
32     // Life Cycle Broadcast Inputs
33     input  lc_ctrl_pkg::lc_tx_t lc_hw_debug_en_i,
34
35     // Entropy Source Interface
36     output entropy_src_pkg::entropy_src_hw_if_req_t en-
  tropy_src_hw_if_o,
37     input  entropy_src_pkg::entropy_src_hw_if_rsp_t en-
  tropy_src_hw_if_i,

```

```

39
40 // Application Interfaces
41 input  CSRNG_req_t [NumHwApps-1:0] CSRNG_cmd_i,
42 output CSRNG_rsp_t [NumHwApps-1:0] CSRNG_cmd_o,
43
44 // Alerts
45 input  prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
  alert_rx_i,
46 output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
  alert_tx_o,
47
48 // Interrupts
49 output logic intr_CS_CMD_REQ_DONE_o,
50 output logic intr_CS_ENTROPY_REQ_o,
51 output logic intr_CS_HW_INST_EXC_o,
52 output logic intr_CS_FATAL_ERR_o
53 );
54
55 CSRNG_reg2hw_t reg2hw;
56 CSRNG_hw2reg_t hw2reg;
57
58 logic [NumAlerts-1:0] alert_test;
59 logic [NumAlerts-1:0] alert;
60
61 logic [NumAlerts-1:0] intg_err_alert;
62 assign intg_err_alert[0] = 1'b0;
63
64 // SEC_CM: CONFIG.REGWEN
65 // SEC_CM: TILE_LINK.BUS.INTEGRITY
66
67 CSRNG_reg_top u_reg (
68   .clk_i,
69   .rst_ni,
70   .tl_i,
71   .tl_o,
72   .reg2hw,
73   .hw2reg,
74   .intg_err_o(intg_err_alert[1])
75 );
76
77 CSRNG_core #(
78   .SBoxImpl(SBoxImpl),
79   .NumHwApps(NumHwApps),
80   .RndCnstCsKeymgrDivNonProduc-
  tion(RndCnstCsKeymgrDivNonProduction),
81   .RndCnstCsKeymgrDivProduc-
  tion(RndCnstCsKeymgrDivProduction)
82 ) u_CSRNG_core (
83   .clk_i,
84   .rst_ni,
85   .reg2hw,
86   .hw2reg,

```

```

87
88 // misc inputs
89 .otp_en_csrng_sw_app_read_i(otp_en_csrng_sw_app_read_i),
90 .lc_hw_debug_en_i,
91
92 // Entropy Interface
93 .entropy_src_hw_if_o,
94 .entropy_src_hw_if_i,
95
96 // Application Interfaces
97 .csrng_cmd_i,
98 .csrng_cmd_o,
99
100 // Alerts
101 .recov_alert_test_o(alert_test[0]),
102 .fatal_alert_test_o(alert_test[1]),
103 .recov_alert_o(alert[0]),
104 .fatal_alert_o(alert[1]),
105
106 .intr_cs_cmd_req_done_o,
107 .intr_cs_entropy_req_o,
108 .intr_cs_hw_inst_exc_o,
109 .intr_cs_fatal_err_o
110 );
111
112 // Alert generation
113 // Alert generation
114 // Alert generation
115 // Alert generation
116 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
117     prim_alert_sender #(
118         .AsyncOn(AlertAsyncOn[i]),
119         .SkewCycles(AlertSkewCycles),
120         .IsFatal(i)
121     ) u_prim_alert_sender (
122         .clk_i,
123         .rst_ni,
124         .alert_test_i ( alert_test[i] ),
125         .alert_req_i ( alert[i] || intg_err_alert[i] ),
126         .alert_ack_o ( ),
127         .alert_state_o ( ),
128         .alert_rx_i ( alert_rx[i] ),
129         .alert_tx_o ( alert_tx_o[i] )
130     );
131 end
132
133 // Assertions
134
135 `ASSERT_KNOWN(TlDValidKnown0_A, tl_o.d_valid)
136 `ASSERT_KNOWN(TlAReadyKnown0_A, tl_o.a_ready)
137 `ASSERT_KNOWN(EsReqKnown0_A, entropy_src_hw_if_o.es_req)
138
139 // Application Interface Asserts
140 for (genvar i = 0; i < NumHwApps; i = i+1) begin :
141 gen_app_if_asserts
142     `ASSERT_KNOWN(CsrngReqReadyKnown0_A,
143 csrng_cmd_o[i].csrng_req_ready)
144     `ASSERT_KNOWN(CsrngRspAckKnown0_A,
145 csrng_cmd_o[i].csrng_rsp_ack)
146     `ASSERT_KNOWN(CsrngRspStsKnown0_A,
147 csrng_cmd_o[i].csrng_rsp_sts)
148     `ASSERT_KNOWN(CsrngGenbitsValidKnown0_A,
149 csrng_cmd_o[i].genbits_valid)
150     `ASSERT_KNOWN_IF(CsrngGenbitsFipsKnown0_A,
151 csrng_cmd_o[i].genbits_fips,
152 csrng_cmd_o[i].genbits_valid)
153     `ASSERT_KNOWN_IF(CsrngGenbitsBusKnown0_A,
154 csrng_cmd_o[i].genbits_bus,
155 csrng_cmd_o[i].genbits_valid)
156 end : gen_app_if_asserts
157
158 // Alerts
159 `ASSERT_KNOWN(AlertTxKnown0_A, alert_tx_o)
160
161 `ASSERT_KNOWN(IntrCsCmdReqDoneKnown0_A,
162 intr_cs_cmd_req_done_o)
163 `ASSERT_KNOWN(IntrCsEntropyReqKnown0_A, intr_cs_en-
164 tropy_req_o)
165 `ASSERT_KNOWN(IntrCsHwInstExcKnown0_A,
166 intr_cs_hw_inst_exc_o)
167 `ASSERT_KNOWN(IntrCsFatalErrKnown0_A, intr_cs_fatal_err_o)
168
169 `AS-
170 SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CtrDrbgGenAlertCheck_A,
171 u_csrng_core.u_csrng_ctr_drbg.u_prim_count_ctr_drbg,
172 alert_tx_o[1])
173
174 for (genvar i = 0; i < NumHwApps + 1; i++) begin :
175 gen_cnt_asserts
176     `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck_A,
177 u_c-
178 srng_core.gen_cmd_stage[i].u_csrng_cmd_stage.u_prim_count_cmd_gen_c-
179 ntr,
180 alert_tx_o[1])
181
182     `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(DrbgCmdFsmCheck_A,
183 u_c-
184 srng_core.gen_cmd_stage[i].u_csrng_cmd_stage.u_state_regs,
185 alert_tx_o[1])
186 end

```

```

173
174 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlMainFsmCheck_A,
175 u_csrng_core.u_csrng_main_sm.u_state_regs,
176 alert_tx_o[1])
177
178 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(DrbgGenFsmCheck_A,
179 u_csrng_core.u_csrng_ctr_drbg.u_state_regs,
180 alert_tx_o[1])
181
182
183 for (genvar i = 0; i < aes_pkg::Sp2VWidth; i++) begin :
184 gen_aes_cipher_control_fsm_svas
185     if (aes_pkg::SP2V_LOGIC_HIGH[i] == 1'b1) begin :
186 gen_aes_cipher_control_fsm_svas_p
187         `AS-
188 SERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCipherControlFsmCheck_A,
189 u_c-
190 srng_core.u_csrng_block_encrypt.u_aes_cipher_core.u_aes_cipher_cont-
191 rol.gen_fsm[i].
192 gen_fsm_p.u_aes_cipher_control_fsm_i.u_aes_cipher_control_fsm.u_sta-
193 te_regs,
194 alert_tx_o[1])
195     end else begin : gen_aes_cipher_control_fsm_svas_n
196         `AS-
197 SERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCipherControlFsmCheck_A,
198 u_c-
199 srng_core.u_csrng_block_encrypt.u_aes_cipher_core.u_aes_cipher_cont-
200 rol.gen_fsm[i].
201 gen_fsm_n.u_aes_cipher_control_fsm_i.u_aes_cipher_control_fsm.u_sta-
202 te_regs,
203 alert_tx_o[1])
204     end
205 end
206
207 // Alert assertions for reg_we onehot check
208 `AS-
209 SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
210 u_reg, alert_tx_o[1])
211
212
213 // The total number of application interfaces defined in the
214 hjson must be at least two:
215 // One SW interface and at least one HW interface. Zero HW
216 interfaces is currently not supported.
217 `ASSERT_INIT(CsrngNumAppsMatch_A, NumApps >= 2)
218 endmodule

```

2. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_block_encrypt.sv (113 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 // Description: AES-based block encrypt module for CSRNG
6
7 module csrng_block_encrypt import csrng_pkg::*; #(
8     parameter aes_pkg::sbox_impl_e SBoxImpl = aes_pkg::SBoxIm-
9     plLut
10 ) (
11     input logic clk_i,
12     input logic rst_ni,
13
14     // Global enable
15     input logic enable_i,
16
17     // Request from ctr_drbg
18     input logic req_vld_i,
19     output logic req_rdy_o,
20     input csrng_key_v_t req_data_i,
21
22     // Response to ctr_drbg
23     output logic rsp_vld_o,
24     input logic rsp_rdy_i,
25     output logic [BlkLen-1:0] rsp_data_o,
26
27     // Status and error signals
28     output logic cipher_sm_err_o
29 );
30
31 localparam int unsigned NumShares = 1;
32
33 // Signals
34 aes_pkg::sp2v_e cipher_in_valid;
35 aes_pkg::sp2v_e cipher_in_ready;
36 aes_pkg::sp2v_e cipher_out_valid;
37 aes_pkg::sp2v_e cipher_out_ready;
38
39 logic [3:0][3:0][7:0] state_init[NumShares];
40
41 logic [7:0][31:0] key_init[NumShares];
42 logic [3:0][3:0][7:0] state_done[NumShares];
43 logic [3:0][3:0][7:0] state_out;
44
45 //-----

```

```

46 // AES cipher core
47 //-----
48
49 assign state_init[0] = aes_pkg::aes_trans-
pose({<<8{req_data_i.v}});
50 assign key_init[0] = {<<8{req_data_i.key}};
51
52 assign state_out = aes_pkg::aes_transpose(state_done[0]);
53 assign rsp_data_o = {<<8{state_out}};
54
55 assign cipher_in_valid = (enable_i && req_vld_i) ?
aes_pkg::SP2V_HIGH : aes_pkg::SP2V_LOW;
56
57 // SEC_CM: AES_CIPHER.FSM.SPARSE
58 // SEC_CM: AES_CIPHER.FSM.REDUN
59 // SEC_CM: AES_CIPHER.CTRL.SPARSE
60 // SEC_CM: AES_CIPHER.FSM.LOCAL_ESC
61 // SEC_CM: AES_CIPHER.CTR.REDUN
62 // SEC_CM: AES_CIPHER.DATA_REG.LOCAL_ESC
63
64 aes_cipher_core #(
65   .AES192Enable (1'b0), // AES192Enable disabled
66   .CiphOpFwdOnly(1'b1), // Forward operation only
67   .SecMasking   (1'b0), // Masking disable
68   .SecSBoxImpl  (SBoxImpl)
69 ) u_aes_cipher_core (
70   .clk_i (clk_i),
71   .rst_ni(rst_ni),
72
73   .cfg_valid_i      (1'b1),
74   .in_valid_i       (cipher_in_valid),
75   .in_ready_o        (cipher_in_ready),
76   .out_valid_o       (cipher_out_valid),
77   .out_ready_i       (cipher_out_ready),
78   .op_i              (aes_pkg::CIPH_FWD),
79   .key_len_i         (aes_pkg::AES_256),
80   .crypt_i           (aes_pkg::SP2V_HIGH), // Enable
81   .crypt_o           (),
82   .alert_fatal_i     (1'b0),
83   .alert_o           (cipher_sm_err_o),
84   .dec_key_gen_i     (aes_pkg::SP2V_LOW), // Disable
85   .dec_key_gen_o     (),
86   .prng_reseed_i     (1'b0), // Disable
87   .prng_reseed_o     (),
88   .key_clear_i       (1'b0), // Disable
89   .key_clear_o       (),
90   .data_out_clear_i  (1'b0), // Disable
91   .data_out_clear_o  (),
92   // These two init values are provided to allow synthesis
to perform optimizations.
93   // We don't care about SCA leakage in this context.
94   .prd_clearing_state_i(state_init),
95   .prd_clearing_key_i  (key_init),
96   .force_masks_i      (1'b0),
97   .data_in_mask_o     (),
98   .entropy_req_o      (),
99   .entropy_ack_i      (1'b0),
100  .entropy_i           ('0),
101  .state_init_i        (state_init),
102  .key_init_i          (key_init),
103  .state_o             (state_done)
104 );
105
106 // The cipher determines whether the response is ready for
consumption
107 assign rsp_vld_o = (cipher_out_valid == aes_pkg::SP2V_HIGH);
108
109 // Type conversion for AES compatibility
110 assign req_rdy_o = (cipher_in_ready == aes_pkg::SP2V_HIGH);
111 assign cipher_out_ready = rsp_rdy_i ? aes_pkg::SP2V_HIGH :
aes_pkg::SP2V_LOW;
112
113 endmodule

```

3. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_cmd_stage.sv (548 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: CSRNG command staging module.
6 //
7 `include "prim_assert.sv"
8
9 module csrng_cmd_stage import csrng_pkg::*; (
10   input logic      clk_i,
11   input logic      rst_ni,
12   // Command input.
13   input logic      cs_enable_i,
14   input logic      cmd_stage_vld_i,
15   input logic [InstIdWidth-1:0] cmd_stage_shid_i,
16   input logic [CmdBusWidth-1:0] cmd_stage_bus_i,
17   output logic     cmd_stage_rdy_o,
18   // Command checking interface.
19   input logic      reseed_cnt_reached_i,
20   output logic     reseed_cnt_alert_o,
21   output logic     invalid_cmd_seq_alert_o,

```

```

22   output logic     invalid_acmd_alert_o,
23   // Command to arbiter.
24   output logic     cmd_arb_req_o,
25   output logic     cmd_arb_sop_o,
26   output logic     cmd_arb_mop_o,
27   output logic     cmd_arb_eop_o,
28   input logic      cmd_arb_gnt_i,
29   output logic [CmdBusWidth-1:0] cmd_arb_bus_o,
30   // Ack from core.
31   input logic      cmd_ack_i,
32   input csrng_cmd_sts_e cmd_ack_sts_i,
33   // Ack to app i/f.
34   output logic     cmd_stage_ack_o,
35   output csrng_cmd_sts_e cmd_stage_ack_sts_o,
36   // Genbits from core.
37   input logic      genbits_vld_i,
38   input logic [127:0] genbits_bus_i,
39   input logic      genbits_fips_i,
40   // Genbits to app i/f.
41   output logic     genbits_vld_o,
42   input logic      genbits_rdy_i,
43   output logic [127:0] genbits_eop_o,
44   output logic     genbits_fips_o,
45   // Error indication.
46   output logic [2:0] cmd_stage_sfifo_cmd_err_o,
47   output logic [2:0] cmd_stage_sfifo_genbits_err_o,
48   output logic     cmd_gen_cnt_err_o,
49   output logic     cmd_stage_sm_err_o
50 );
51
52 // Genbits parameters.
53 localparam int GenBitsFifoWidth = 1 + BlkLen;
54 localparam int GenBitsFifoDepth = 1;
55
56 // Command FIFO.
57 logic [CmdBusWidth-1:0] sfifo_cmd_rdata;
58 logic [CmdFifoDepthLg:0] sfifo_cmd_depth;
59 logic sfifo_cmd_wvld;
60 logic sfifo_cmd_wrdy;
61 logic [CmdBusWidth-1:0] sfifo_cmd_wdata;
62 logic sfifo_cmd_rrdy;
63 logic [2:0] sfifo_cmd_err;
64 logic sfifo_cmd_rvld;
65
66 // Genbits FIFO.
67 logic [GenBitsFifoWidth-1:0] sfifo_genbits_rdata;
68 logic sfifo_genbits_wvld;
69 logic sfifo_genbits_wrdy;
70 logic [GenBitsFifoWidth-1:0] sfifo_genbits_wdata;
71 logic sfifo_genbits_rrdy;
72 logic [2:0] sfifo_genbits_err;
73 logic sfifo_genbits_rvld;
74
75 // Command signals.
76 logic [3:0] cmd_len;
77 logic cmd_fifo_zero;
78 logic cmd_fifo_pop;
79 logic cmd_len_dec;
80 logic cmd_gen_cnt_dec;
81 logic cmd_gen_1st_req;
82 logic cmd_gen_inc_req;
83 logic cmd_gen_cnt_last;
84 logic cmd_final_ack;
85 logic cmd_err_ack;
86 logic [GenBitsCtrWidth-1:0] cmd_gen_cnt;
87 csrng_cmd_sts_e err_sts;
88 logic reseed_cnt_exceeded;
89 logic invalid_cmd_seq;
90 logic invalid_acmd;
91 logic [2:0] acmd;
92 logic local_escalate;
93
94 // Flops.
95 logic cmd_ack_q, cmd_ack_d;
96 csrng_cmd_sts_e cmd_ack_sts_q, cmd_ack_sts_d;
97 logic [3:0] cmd_len_q, cmd_len_d;
98 logic cmd_gen_flag_q, cmd_gen_flag_d;
99 logic [11:0] cmd_gen_cmd_q, cmd_gen_cmd_d;
100 logic instantiated_d, instantiated_q;
101
102 always_ff @(posedge clk_i or negedge rst_ni) begin
103   if (!rst_ni) begin
104     cmd_ack_q <= '0;
105     cmd_ack_sts_q <= CMD_STS_SUCCESS;
106     cmd_len_q <= '0;
107     cmd_gen_flag_q <= '0;
108     cmd_gen_cmd_q <= '0;
109     instantiated_q <= '0;
110   end else begin
111     cmd_ack_q <= cmd_ack_d;
112     cmd_ack_sts_q <= cmd_ack_sts_d;
113     cmd_len_q <= cmd_len_d;
114     cmd_gen_flag_q <= cmd_gen_flag_d;
115     cmd_gen_cmd_q <= cmd_gen_cmd_d;
116     instantiated_q <= instantiated_d;
117   end
118 end
119
120 assign cmd_stage_sfifo_cmd_err_o = sfifo_cmd_err;

```

```

121 assign cmd_stage_sfifo_genbits_err_o = sfifo_genbits_err;
122
123 //-----
124 // Capture the transfer length of data behind the command.
125 //-----
126
127 prim_fifo_sync #(
128     .Width(CmdBusWidth),
129     .Pass(0),
130     .Depth(CmdFifoDepth),
131     .OutputZeroIfEmpty(1'b0)
132 ) u_prim_fifo_cmd (
133     .clk_i    (clk_i),
134     .rst_ni   (rst_ni),
135     .clr_i    (!cs_enable_i),
136     .wvalid_i (sfifo_cmd_wvld),
137     .wready_o (sfifo_cmd_wrdy),
138     .wdata_i  (sfifo_cmd_wdata),
139     .rvalid_o (sfifo_cmd_rvld),
140     .rready_i (sfifo_cmd_rrdy),
141     .rdata_o  (sfifo_cmd_rdata),
142     .full_o   (),
143     .depth_o  (sfifo_cmd_depth),
144     .err_o    ()
145 );
146
147 assign sfifo_cmd_wdata = cmd_stage_bus_i;
148
149 assign sfifo_cmd_wvld = cs_enable_i && cmd_stage_rdy_o &&
cmd_stage_vld_i;
150
151 assign sfifo_cmd_rrdy = cs_enable_i && cmd_fifo_pop;
152
153 assign cmd_arb_bus_o =
154     cmd_gen_inc_req ?
155     {15'b0, cmd_gen_cnt_last, cmd_stage_shid_i, cmd_gen_cmd_q} :
156     // pad, glast, id, f, clen, cmd
157     cmd_gen_1st_req ?
158     {15'b0, cmd_gen_cnt_last, cmd_stage_shid_i, sfifo_cmd_rdata[11:0]} :
159     cmd_arb_mop_o ? sfifo_cmd_rdata :
160     '0;
161
162 assign cmd_stage_rdy_o = sfifo_cmd_wrdy;
163
164 assign sfifo_cmd_err =
165     {( sfifo_cmd_wvld && !sfifo_cmd_wrdy),
166      ( sfifo_cmd_rrdy && !sfifo_cmd_rvld),
167      (!sfifo_cmd_wrdy && !sfifo_cmd_rvld)};
168
169 // State machine controls.
170 assign cmd_fifo_zero = (sfifo_cmd_depth == '0);
171 assign cmd_len = sfifo_cmd_rdata[7:4];
172
173 // Capture the length of csrng command.
174 assign cmd_len_d =
175     (!cs_enable_i) ? '0 :
176     cmd_arb_sop_o ? cmd_len :
177     cmd_len_dec ? (cmd_len_q-1) :
178     cmd_len_q;
179
180 // Capture the application command type.
181 assign acmd = sfifo_cmd_rdata[2:0];
182
183 // For gen commands, capture information from the original
command for use later.
184 assign cmd_gen_flag_d =
185     (!cs_enable_i) ? '0 :
186     cmd_gen_1st_req ? (acmd == GEN) :
187     cmd_gen_flag_q;
188
189 assign cmd_gen_cmd_d =
190     (!cs_enable_i) ? '0 :
191     cmd_gen_1st_req ? {sfifo_cmd_rdata[11:0]} :
192     cmd_gen_cmd_q;
193
194 // SEC_CM: GEN_CMD.CTR.REDUN
195 prim_count #(
196     .Width(GenBitsCtrWidth),
197     .ResetValue({GenBitsCtrWidth{1'b1}})
198 ) u_prim_count_cmd_gen_cntr (
199     .clk_i,
200     .rst_ni,
201     .clr_i(!cs_enable_i),
202     .set_i(cmd_gen_1st_req),
203     .set_cnt_i(sfifo_cmd_rdata[12 +: GenBitsCtrWidth]),
204     .incr_en_i(1'b0),
205     .decr_en_i(cmd_gen_cnt_dec), // Count down.
206     .step_i(GenBitsCtrWidth'(1)),
207     .commit_i(1'b1),
208     .cnt_o(cmd_gen_cnt),
209     .cnt_after_commit_o(),
210     .err_o(cmd_gen_cnt_err_o)
211 );
212
213 // For naming consistency.
214 assign local_escalate = cmd_gen_cnt_err_o;
215
216 //-----
217 // state machine to process command
218 //-----

```

```

218 // Encoding generated with:
219 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 11 -n 8 \
220 //   -s 170131814 --language=sv
221 //
222 // Hamming distance histogram:
223 //
224 // 0: --
225 // 1: --
226 // 2: --
227 // 3: ||||| (21.82%)
228 // 4: ||||| (36.36%)
229 // 5: ||||| (30.91%)
230 // 6: |||| (9.09%)
231 // 7: | (1.82%)
232 // 8: --
233 //
234 // Minimum Hamming distance: 3
235 // Maximum Hamming distance: 7
236 // Minimum Hamming weight: 1
237 // Maximum Hamming weight: 6
238 //
239 localparam int StateWidth = 8;
240 typedef enum logic [StateWidth-1:0] {
241     Idle    = 8'b1110101, // idle
242     Flush   = 8'b01011011, // flush command FIFO and start
over
243     ArbGnt  = 8'b00011100, // general arbiter request
244     SendSOP = 8'b00000001, // send sop (start of packet)
245     SendMOP = 8'b01010110, // send mop (middle of packet)
246     GenCmdChk = 8'b10001101, // gen cmd check
247     CmdAck   = 8'b11000000, // wait for command ack
248     GenReq   = 8'b10010011, // process gen requests
249     GenArbGnt = 8'b11101110, // generate subsequent arb re-
quest
250     GenSOP   = 8'b10111010, // generate subsequent request
251     Error    = 8'b01100111 // illegal state reached and hang
} state_e;
252
253 // SEC_CM: CMD_STAGE.FSM.SPARSE
254 state_e state_d, state_q;
255 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
state_e, Idle)
256
257 always_comb begin
258     state_d = state_q;
259     cmd_fifo_pop = 1'b0;
260     cmd_len_dec = 1'b0;
261     cmd_gen_cnt_dec = 1'b0;
262     cmd_gen_1st_req = 1'b0;
263     cmd_gen_inc_req = 1'b0;
264     cmd_gen_cnt_last = 1'b0;
265     cmd_final_ack = 1'b0;
266     cmd_arb_req_o = 1'b0;
267     cmd_arb_sop_o = 1'b0;
268     cmd_arb_mop_o = 1'b0;
269     cmd_arb_eop_o = 1'b0;
270     cmd_stage_sm_err_o = 1'b0;
271     cmd_err_ack = 1'b0;
272     reseed_cnt_exceeded = 1'b0;
273     invalid_cmd_seq = 1'b0;
274     invalid_acmd = 1'b0;
275     instantiated_d = instantiated_q;
276
277     if (state_q == Error) begin
278         // In case we are in the Error state we must ignore the
local escalate and enable signals.
279         cmd_stage_sm_err_o = 1'b1;
280         end else if (local_escalate) begin
281             // In case local escalate is high we must transition to
the error state.
282             state_d = Error;
283         end else if (!cs_enable_i && state_q inside {Idle, Flush,
ArbGnt, SendSOP, SendMOP, GenCmdChk,
284             CmdAck, Gen-
Req, GenArbGnt, GenSOP}) begin
285             // In case the module is disabled and we are in a legal
state we must go into idle state.
286             state_d = Idle;
287             instantiated_d = 1'b0;
288         end else begin
289             // Otherwise do the state machine as normal.
290             unique case (state_q)
291                 Idle: begin
292                     // Because of the if statement above we won't leave
idle if enable is low.
293                     if (!cmd_fifo_zero) begin
294                         if (acmd == INS) begin
295                             if (!instantiated_q) begin
296                                 state_d = ArbGnt;
297                                 instantiated_d = 1'b1;
298                             end
299                             if (instantiated_q) begin
300                                 cmd_err_ack = 1'b1;
301                                 invalid_cmd_seq = 1'b1;
302                                 state_d = Idle;
303                             end
304                         end else if (acmd == RES) begin
305                             if (instantiated_q) begin
306                                 state_d = ArbGnt;
307                             end
308                             if (!instantiated_q) begin
309

```

```

310     cmd_err_ack = 1'b1;
311     invalid_cmd_seq = 1'b1;
312     state_d = Idle;
313 end
314 end else if (acmd == GEN) begin
315     if (!instantiated_q) begin
316         // If the issued command is GEN and the reseed
count has already been reached,
317         // send an ack with an error status response.
318         if ((acmd == GEN) && reseed_cnt_reached_i) be-
gin
319             cmd_err_ack = 1'b1;
320             reseed_cnt_exceeded = 1'b1;
321             state_d = Idle;
322         end else begin
323             state_d = ArbGnt;
324         end
325     end
326     if (!instantiated_q) begin
327         cmd_err_ack = 1'b1;
328         invalid_cmd_seq = 1'b1;
329         state_d = Idle;
330     end
331 end else if (acmd == UPD) begin
332     if (instantiated_q) begin
333         state_d = ArbGnt;
334     end
335     if (!instantiated_q) begin
336         cmd_err_ack = 1'b1;
337         invalid_cmd_seq = 1'b1;
338         state_d = Idle;
339     end
340 end else if (acmd == UNI) begin
341     // Set the instantiation to zero.
342     instantiated_d = 1'b0;
343     state_d = ArbGnt;
344 end else begin
345     // Command was not supported.
346     cmd_err_ack = 1'b1;
347     invalid_acmd = 1'b1;
348     state_d = Idle;
349 end
350 // If we received an invalid command, pop it from
the FIFO. Afterwards, absorb any
351 // additional data belonging to the same invalid
command and empty the FIFO.
352 if (cmd_err_ack) begin
353     cmd_fifo_pop = 1'b1;
354     state_d = Flush;
355 end
356 end
357 Flush: begin
358     // Keep popping the FIFO until it's empty and we're
not getting new command input.
359     // The decision whether a command is invalid is
taken based on the command header but
360     // EDN might continue and send additional data be-
longing to the same command. We have
361     // to absorb the full invalid command before we can
continue.
362     cmd_fifo_pop = sfifo_cmd_rvld;
363     if (!sfifo_cmd_rvld && !cmd_stage_vld_i) begin
364         state_d = Idle;
365     end
366 end
367 ArbGnt: begin
368     cmd_arb_req_o = 1'b1;
369     if (cmd_arb_gnt_i) begin
370         state_d = SendSOP;
371     end
372 end
373 SendSOP: begin
374     cmd_gen_1st_req = 1'b1;
375     cmd_arb_sop_o = 1'b1;
376     cmd_fifo_pop = 1'b1;
377     if (sfifo_cmd_rdata[12+: GenBitsCtrWidth] == Gen-
BitsCtrWidth'(1)) begin
378         cmd_gen_cnt_last = 1'b1;
379     end
380     if (cmd_len == '0) begin
381         cmd_arb_eop_o = 1'b1;
382         state_d = GenCmdChk;
383     end else begin
384         state_d = SendMOP;
385     end
386 end
387 SendMOP: begin
388     if (!cmd_fifo_zero) begin
389         cmd_fifo_pop = 1'b1;
390         cmd_len_dec = 1'b1;
391         if (cmd_len_q == 4'h1) begin
392             cmd_arb_mop_o = 1'b1;
393             cmd_arb_eop_o = 1'b1;
394             state_d = GenCmdChk;
395         end else begin
396             cmd_arb_mop_o = 1'b1;
397         end
398     end
399 end
400 end

```

```

401 GenCmdChk: begin
402     if (cmd_gen_flag_q) begin
403         cmd_gen_cnt_dec = 1'b1;
404     end
405     state_d = CmdAck;
406 end
407 CmdAck: begin
408     if (cmd_ack_i) begin
409         // The state database has successfully been up-
dated.
410         // In case of Generate commands, we get the gener-
ated bits one clock cycle before
411         // receiving the ACK from the state database (from
csrng_ctr_drbg_gen).
412         state_d = GenReq;
413     end
414 end
415 GenReq: begin
416     // Flag set if a gen request.
417     if (cmd_gen_flag_q) begin
418         // Must stall if genbits fifo is not clear.
419         if (sfifo_genbits_wrdy) begin
420             if (cmd_gen_cnt == '0) begin
421                 cmd_final_ack = 1'b1;
422                 state_d = Idle;
423             end else begin
424                 // Issue a subsequent gen request.
425                 state_d = GenArbGnt;
426             end
427         end
428     end else begin
429         // Ack for the non-gen request case.
430         cmd_final_ack = 1'b1;
431         state_d = Idle;
432     end
433 end
434 GenArbGnt: begin
435     cmd_arb_req_o = 1'b1;
436     if (cmd_arb_gnt_i) begin
437         state_d = GenSOP;
438     end
439 end
440 GenSOP: begin
441     cmd_arb_sop_o = 1'b1;
442     cmd_arb_eop_o = 1'b1;
443     cmd_gen_inc_req = 1'b1;
444     state_d = GenCmdChk;
445     // Check for final genbits beat.
446     if (cmd_gen_cnt == GenBitsCtrWidth'(1)) begin
447         cmd_gen_cnt_last = 1'b1;
448     end
449 end
450 // Error: The error state is now covered by the if
statement above.
451 default: begin
452     state_d = Error;
453     cmd_stage_sm_err_o = 1'b1;
454 end
455 endcase // unique case (state_q)
456 end
457 end
458 //-----
459 // Genbits FIFO.
460 //-----
461 prim_fifo_sync #(
462     .Width(GenBitsFifoWidth),
463     .Pass(0),
464     .Depth(GenBitsFifoDepth),
465     .OutputZeroIfEmpty(0) // Set to 0, and let last data drive
out.
466 ) u_prim_fifo_genbits (
467     .clk_i (clk_i),
468     .rst_ni (rst_ni),
469     .clr_i (!cs_enable_i),
470     .wvalid_i(sfifo_genbits_wvld),
471     .wready_o(sfifo_genbits_wrdy),
472     .wdata_i(sfifo_genbits_wdata),
473     .rvalid_o(sfifo_genbits_rvld),
474     .rready_i(sfifo_genbits_rrdy),
475     .rdata_o (sfifo_genbits_rdata),
476     .full_o (),
477     .depth_o (), // sfifo_genbits_depth)
478     .err_o ()
479 );
480 assign sfifo_genbits_wdata = {genbits_fips_i, genbits_bus_i};
481 // The prim_fifo_sync primitive is constructed to only ac-
cept pushes if there is indeed space
482 // available. Backpressure would actually need to be handled
at the sender (csrng_ctr_drbg_gen).
483 // In this particular case, it is safe to unconditionally
push the genbits FIFO because
484 // the GenSOP FSM state which triggers csrng_ctr_drbg_gen to
generate bits can only be reached

```



```

489 // after checking that the genbits FIFO isn't full already.
This condition is checked using an
490 // SVA below.
491 //

492 // If the genbits FIFO got pushed without having space, this
either means the output of a genbits
493 // request is routed to the wrong application interface
(which would be a critical design bug)

494 // or that some fault injection attack is going on. Thus, we
track such cases both with an SVA
495 // and with a fatal alert (identifiable via the ERR_CODE re-
gister).
496 assign sfifo_genbits_wvld = cs_enable_i && genbits_vld_i;
497
498 assign sfifo_genbits_rrdy = genbits_vld_o && genbits_rdy_i;
499
500 assign genbits_vld_o = cs_enable_i && sfifo_genbits_rvld;
501 assign {genbits_fips_o, genbits_bus_o} =
sfifo_genbits_rdata;
502
503 assign sfifo_genbits_err =
504   {( sfifo_genbits_wvld && !sfifo_genbits_wrdy),
505     ( sfifo_genbits_rrdy && !sfifo_genbits_rvld),
506     (!sfifo_genbits_wrdy && !sfifo_genbits_rvld)};
507
508 // We're only allowed to request more bits if the genbits
FIFO has indeed space.
509 `ASSERT(CsrngCmdStageGenbitsFifoFull_A, state_q == GenSOP |->
sfifo_genbits_wrdy)
510
511 // Pushes to the genbits FIFO outside of the GenCmdChk and
CmdAck states or while handling a
512 // command other than Generate are not allowed.
513 `ASSERT(CsrngCmdStageGenbitsFifoPushExpected_A,
sfifo_genbits_wvld |-> state_q inside {GenCmdChk, Cm-
dAck} && cmd_gen_flag_q)
514
515 //-----
516 // Ack logic.
517 //-----
518
519 assign cmd_ack_d =
520   (!cs_enable_i) ? '0 :
521   cmd_final_ack || cmd_err_ack;
522
523 assign cmd_stage_ack_o = cmd_ack_q;
524
525
526 assign err_sts = reseed_cnt_exceeded ? CMD_STS_RE-
SEED_CNT_EXCEEDED :
527   invalid_cmd_seq ? CMD_STS_INVALID-
ID_CMD_SEQ :
528   invalid_acmd ? CMD_STS_INVALID-
ID_ACMD : CMD_STS_INVALID_ACMD;
529
530 assign cmd_ack_sts_d =
531   (!cs_enable_i) ? CMD_STS_SUCCESS :
532   cmd_err_ack ? err_sts :
533   cmd_final_ack ? cmd_ack_sts_i :
534   cmd_ack_sts_q;
535
536 assign cmd_stage_ack_sts_o = cmd_ack_sts_q;
537
538 assign reseed_cnt_alert_o = reseed_cnt_exceeded;
539 assign invalid_cmd_seq_alert_o = invalid_cmd_seq;
540 assign invalid_acmd_alert_o = invalid_acmd;
541
542 // Make sure that the state machine has a stable error
state. This means that after the error
543 // state is entered it will not exit it unless a reset sig-
nal is received.
544 `ASSERT(CsrngCmdStageErrorStStable_A, state_q == Error |>=
$stable(state_q))
545 // If in error state, the error output must be high.
546 `ASSERT(CsrngCmdStageErrorOutput_A, state_q == Error |->
cmd_stage_sm_err_o)
547
548 endmodule

```

4. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_core.sv (1,036 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: csrng core module
6 //
7
8 module csrng_core import csrng_pkg::*; #(
9   parameter aes_pkg::sbox_impl_e SBoxImpl = aes_pkg::SBoxIm-
pLut,
10   parameter int NumHwApps = 2,
11   parameter cs_keymgr_div_t RndCnstCsKeymgrDivNonProduction =
CsKeymgrDivWidth'(0),
12   parameter cs_keymgr_div_t RndCnstCsKeymgrDivProduction = Cs-

```

```

KeymgrDivWidth'(0)
14 ) (
15   input logic clk_i,
16   input logic rst_ni,
17
18   input csrng_reg_pkg::csrng_reg2hw_t reg2hw,
19   output csrng_reg_pkg::csrng_hw2reg_t hw2reg,
20
21   // OTP Interface
22   input prim_mubi_pkg::mubi8_t otp_en_c-
srng_sw_app_read_i,
23
24   // Lifecycle broadcast inputs
25   input lc_ctrl_pkg::lc_tx_t lc_hw_de-
bug_en_i,
26
27   // Entropy Interface
28   output entropy_src_pkg::entropy_src_hw_if_req_t en-
tropy_src_hw_if_o,
29   input entropy_src_pkg::entropy_src_hw_if_rsp_t en-
tropy_src_hw_if_i,
30
31   // Application Interfaces
32   input csrng_req_t [NumHwApps-1:0] csrng_cmd_i,
33   output csrng_rsp_t [NumHwApps-1:0] csrng_cmd_o,
34
35   // Alerts
36   output logic re-
cov_alert_test_o,
37   output logic fatal_alert_test_o,
38   output logic re-
cov_alert_o,
39   output logic fatal_alert_o,
40
41   // Interrupts
42   output logic intr_cs_cmd_req_done_o,
43   output logic intr_cs_en-
tropy_req_o,
44   output logic intr_cs_hw_inst_exc_o,
45   output logic intr_cs_fatal_err_o
46 );
47
48 import csrng_reg_pkg::*;
49
50 import prim_mubi_pkg::mubi4_t;
51 import prim_mubi_pkg::mubi4_test_true_strict;
52 import prim_mubi_pkg::mubi4_test_invalid;
53
54 localparam int unsigned ADataDepthClog = $clog2(CmdMaxClen)
+ 1;
55 localparam int unsigned CsEnableCopies = 51;
56 localparam int unsigned LcHwDebugCopies = 1;
57 localparam int unsigned Flag0Copies = 3;
58
59 // signals
60 // interrupt signals
61 logic event_cs_fatal_err;
62 logic [CsEnableCopies-1:1] cs_enable_fo;
63 logic [Flag0Copies-1:0] flag0_fo;
64 logic acmd_flag0_pfa;
65 logic cs_enable_pfa;
66 logic sw_app_enable;
67 logic sw_app_enable_pfe;
68 logic sw_app_enable_pfa;
69 logic read_int_state;
70 logic read_int_state_pfe;
71 logic read_int_state_pfa;
72 logic fips_force_enable;
73 logic fips_force_enable_pfe;
74 logic fips_force_enable_pfa;
75 logic recov_alert_event;
76 logic acmd_avail;
77 logic acmd_sop;
78 logic acmd_mop;
79 logic acmd_eop;
80
81 logic state_db_wr_vld;
82 csrng_state_t state_db_rd_data;
83
84 logic [CmdBusWidth-1:0] acmd_bus;
85 acmd_e acmd_hold;
86
87 logic [SeedLen-1:0] packer_adata;
88 logic [ADataDepthClog-1:0] packer_adata_depth;
89 logic packer_adata_pop;
90 logic packer_adata_clr;
91 logic [SeedLen-1:0] seed_diversification;
92
93 logic cmd_entropy_req;
94 logic cmd_entropy_avail;
95
96 logic ctr_drbg_req_vld;
97 logic ctr_drbg_req_rdy;
98 csrng_core_data_t ctr_drbg_req_data;
99
100 logic ctr_drbg_rsp_vld;

```

```

101 csrng_core_data_t      ctr_drbg_rsp_data;
102 csrng_cmd_sts_e        ctr_drbg_rsp_sts;
103
104 logic                  ctr_drbg_bits_vld;
105 logic [BlkLen-1:0]    ctr_drbg_bits_data;
106 logic                  ctr_drbg_bits_fips;
107
108 logic                  acmd_accept;
109 logic                  main_sm_cmd_vld;
110 logic                  clr_adata_packer;
111
112 logic                  fifo_write_err_sum;
113 logic                  fifo_read_err_sum;
114 logic                  fifo_status_err_sum;
115 logic                  ctr_err_sum;
116
117 logic                  cmd_stage_sm_err_sum;
118 logic                  main_sm_err_sum;
119 logic                  cs_main_sm_err;
120 logic [MainSmStateWidth-1:0] cs_main_sm_state;
121 logic                  ctr_drbg_sm_err_sum;
122 logic                  ctr_drbg_sm_err;
123 logic                  ctr_drbg_v_ctr_err;
124 logic                  block_encrypt_sm_err_sum;
125 logic                  block_encrypt_sm_err;
126
127 // Signals to and from block cipher
128 logic                  block_encrypt_req_vld;
129 logic                  block_encrypt_req_rdy;
130 csrng_key_v_t          block_encrypt_req_data;
131
132 logic                  block_encrypt_rsp_vld;
133 logic                  block_encrypt_rsp_rdy;
134 logic [BlkLen-1:0]    block_encrypt_rsp_data;
135
136
137 logic [2:0]
cmd_stage_sfifo_cmd_err[NumApps];
138 logic [NumApps-1:0]    cmd_stage_sfifo_cmd_err_sum;
139 logic [NumApps-1:0]    cmd_stage_sfifo_cmd_err_wr;
140 logic [NumApps-1:0]    cmd_stage_sfifo_cmd_err_rd;
141 logic [NumApps-1:0]    cmd_stage_sfifo_cmd_err_st;
142 logic [2:0]
cmd_stage_sfifo_genbits_err[NumApps];
143 logic [NumApps-1:0]    cmd_stage_sfifo_genbits_err_wr;
144 logic [NumApps-1:0]    cmd_stage_sfifo_genbits_err_rd;
145 logic [NumApps-1:0]    cmd_stage_sfifo_genbits_err_st;
146 logic [NumApps-1:0]    cmd_stage_ctr_err;
147 logic [NumApps-1:0]    cmd_stage_sm_err;
148
149
150 logic [NumApps-1:0]    cmd_stage_vld;
151 logic [InstIdWidth-1:0] cmd_stage_shid[NumApps];
152 logic [CmdBusWidth-1:0] cmd_stage_bus[NumApps];
153 logic [NumApps-1:0]    cmd_stage_rdy;
154 logic [NumApps-1:0]    cmd_arb_req;
155 logic [NumApps-1:0]    cmd_arb_gnt;
156 logic [$clog2(NumApps)-1:0] cmd_arb_idx;
157 logic [NumApps-1:0]    cmd_arb_sop;
158 logic [NumApps-1:0]    cmd_arb_mop;
159 logic [NumApps-1:0]    cmd_arb_eop;
160 logic [CmdBusWidth-1:0] cmd_arb_bus[NumApps];
161 logic [NumApps-1:0]    cmd_core_ack;
162 csrng_cmd_sts_e [NumApps-1:0] cmd_core_ack_sts;
163 logic [NumApps-1:0]    cmd_stage_ack;
164 csrng_cmd_sts_e [NumApps-1:0] cmd_stage_ack_sts;
165 logic [NumApps-1:0]    genbits_core_vld;
166 logic [BlkLen-1:0]    genbits_core_bus[NumApps];
167 logic [NumApps-1:0]    genbits_core_fips;
168 logic [NumApps-1:0]    genbits_stage_vld;
169 logic [NumApps-1:0]    genbits_stage_fips;
170 logic [BlkLen-1:0]    genbits_stage_bus[NumApps];
171 logic [NumApps-1:0]    genbits_stage_rdy;
172 logic                  genbits_stage_vld_sw;
173 logic                  genbits_stage_bus_rd_sw;
174 logic [31:0]          genbits_stage_bus_sw;
175
176 logic [15:0]          hw_exception_sts;
177 logic [LcHwDebugCopies-1:0] lc_hw_debug_on_fo;
178 logic
179
180 logic [30:0]          err_code_test_bit;
181
182 logic                  cs_rdata_capt_vld;
183 logic                  cs_bus_cmp_alert;
184 logic [NumApps-1:0]    invalid_cmd_seq_alert;
185 logic [NumApps-1:0]    invalid_acmd_alert;
186 logic [NumApps-1:0]    reseed_cnt_alert;
187 logic [1:0]          otp_sw_app_read_en;
188
189 logic [NumApps-1:0][31:0] reseed_counter;
190
191 prim_mubi_pkg::mubi8_t [1:0]
otp_sw_app_read_en_mubi;
192 prim_mubi_pkg::mubi4_t [CsEnableCopies-1:0] mubi_cs_en-
able_fanout;
193 prim_mubi_pkg::mubi4_t [Flag0Copies-1:0]
mubi_flag0_fanout;
194
195 // flops
196 acmd_e                  acmd_q, acmd_d;
197 logic [NumAppsLg-1:0]  inst_id_q, inst_id_d;
198 logic                  gen_last_q, gen_last_d;
199 mubi4_t                flag0_q, flag0_d;
200 logic [NumAppsLg-1:0]  cmd_arb_idx_q, cmd_arb_idx_d;
201 logic                  genbits_stage_fips_sw_q, gen-
bits_stage_fips_sw_d;
202 acmd_e                  ctr_drbg_cmd_q, ctr_drbg_cmd_d;
203 logic [SeedLen-1:0]    entropy_src_seed_q, en-
tropy_src_seed_d;
204 logic                  entropy_src_fips_q, en-
tropy_src_fips_d;
205 logic [63:0]          cs_rdata_capt_q, cs_rdata_capt_d;
206 logic                  cs_rdata_capt_vld_q,
cs_rdata_capt_vld_d;
207 logic                  sw_rdy_sts_q, sw_rdy_sts_d;
208 logic                  sw_sts_ack_q, sw_sts_ack_d;
209 logic [NumApps-1:0]    reseed_cnt_reached_q, re-
seed_cnt_reached_d;
210
211 always_ff @(posedge clk_i or negedge rst_ni) begin
212   if (!rst_ni) begin
213     acmd_q <= INV;
214     inst_id_q <= '0;
215     gen_last_q <= '0;
216     flag0_q <= prim_mubi_pkg::MuBi4False;
217     cmd_arb_idx_q <= '0;
218     genbits_stage_fips_sw_q <= '0;
219     ctr_drbg_cmd_q <= INV;
220     entropy_src_seed_q <= '0;
221     entropy_src_fips_q <= '0;
222     cs_rdata_capt_q <= '0;
223     cs_rdata_capt_vld_q <= '0;
224     sw_rdy_sts_q <= '0;
225     sw_sts_ack_q <= '0;
226     reseed_cnt_reached_q <= '0;
227   end else begin
228     acmd_q <= acmd_d;
229     inst_id_q <= inst_id_d;
230     gen_last_q <= gen_last_d;
231     flag0_q <= flag0_d;
232     cmd_arb_idx_q <= cmd_arb_idx_d;
233     genbits_stage_fips_sw_q <= genbits_stage_fips_sw_d;
234     ctr_drbg_cmd_q <= ctr_drbg_cmd_d;
235     entropy_src_seed_q <= entropy_src_seed_d;
236     entropy_src_fips_q <= entropy_src_fips_d;
237     cs_rdata_capt_q <= cs_rdata_capt_d;
238     cs_rdata_capt_vld_q <= cs_rdata_capt_vld_d;
239     sw_rdy_sts_q <= sw_rdy_sts_d;
240     sw_sts_ack_q <= sw_sts_ack_d;
241     reseed_cnt_reached_q <= reseed_cnt_reached_d;
242   end
243 end
244
245 //-----
246 // instantiate interrupt hardware primitives
247 //-----
248 // All TLUL interrupts are collect in the section.
249
250 prim_intr_hw #(
251   .Width(1)
252 ) u_intr_hw_cs_cmd_req_done (
253   .clk_i (clk_i),
254   .rst_ni (rst_ni),
255   .event_intr_i (cmd_stage_ack[NumApps-1]),
256   .reg2hw_intr_enable_q_i (reg2hw.intr_enable.cs_cmd_req_done
.q),
257   .reg2hw_intr_test_q_i
(reg2hw.intr_test.cs_cmd_req_done.q),
258   .reg2hw_intr_test_qe_i
(reg2hw.intr_test.cs_cmd_req_done.qe),
259   .reg2hw_intr_state_q_i
(reg2hw.intr_state.cs_cmd_req_done.q),
260   .hw2reg_intr_state_de_o (hw2reg.intr_state.cs_cmd_req_done
.de),
261   .hw2reg_intr_state_d_o
(hw2reg.intr_state.cs_cmd_req_done.d),
262   .intr_o (intr_cs_cmd_req_done_o)
);
263
264 prim_intr_hw #(
265   .Width(1)
266 ) u_intr_hw_cs_entropy_req (
267   .clk_i (clk_i),
268   .rst_ni (rst_ni),
269   .event_intr_i (entropy_src_hw_if_o.es_req),
270   .reg2hw_intr_enable_q_i (reg2hw.intr_enable.cs_entropy_req
.q),
271   .reg2hw_intr_test_q_i
(reg2hw.intr_test.cs_entropy_req.q),
272   .reg2hw_intr_test_qe_i
(reg2hw.intr_test.cs_entropy_req.qe),
273   .reg2hw_intr_state_q_i
(reg2hw.intr_state.cs_entropy_req.q),
274   .hw2reg_intr_state_de_o (hw2reg.intr_state.cs_entropy_req
.de),
275   .hw2reg_intr_state_d_o
(hw2reg.intr_state.cs_entropy_req.d),
276   .intr_o (intr_cs_entropy_req_o)
);
277
278
279

```



```

280 prim_intr_hw #(
281   .Width(1)
282 ) u_intr_hw_cs_hw_inst_exc (
283   .clk_i      (clk_i),
284   .rst_ni     (rst_ni),
285   .event_intr_i (|hw_exception_sts),
286   .reg2hw_intr_enable_q_i (reg2hw.intr_enable.cs_hw_inst_exc.
q),
287   .reg2hw_intr_test_q_i
(reg2hw.intr_test.cs_hw_inst_exc.q),
288   .reg2hw_intr_test_qe_i
(reg2hw.intr_test.cs_hw_inst_exc.qe),
289   .reg2hw_intr_state_q_i
(reg2hw.intr_state.cs_hw_inst_exc.q),
290   .hw2reg_intr_state_de_o (hw2reg.intr_state.cs_hw_inst_exc.
de),
291   .hw2reg_intr_state_d_o
(hw2reg.intr_state.cs_hw_inst_exc.d),
292   .intr_o      (intr_cs_hw_inst_exc_o)
293 );
294
295 prim_intr_hw #(
296   .Width(1)
297 ) u_intr_hw_cs_fatal_err (
298   .clk_i      (clk_i),
299   .rst_ni     (rst_ni),
300   .event_intr_i (event_cs_fatal_err),
301   .reg2hw_intr_enable_q_i (reg2hw.intr_enable.cs_fatal_err.q)
302 ,
303   .reg2hw_intr_test_q_i (reg2hw.intr_test.cs_fatal_err.q),
304   .reg2hw_intr_test_qe_i (reg2hw.intr_test.cs_fatal_err.qe),
305   .reg2hw_intr_state_q_i (reg2hw.intr_state.cs_fatal_err.q),
306   .hw2reg_intr_state_de_o (hw2reg.intr_state.cs_fatal_err.de)
307 ,
308   .hw2reg_intr_state_d_o (hw2reg.intr_state.cs_fatal_err.d),
309   .intr_o      (intr_cs_fatal_err_o)
310 );
311 // Counter and FSM errors are structural errors and are al-
ways active regardless of the
312
313 // functional state. main_sm_err_sum is not included here to
prevent some tools from
314 // inferring combo loops. However, to include the state ma-
chine error for testing,
315 // we use the error code test bit (index 21) directly.
316 logic fatal_loc_events;
317 assign fatal_loc_events = block_encrypt_sm_err_sum ||
cmd_stage_sm_err_sum || ctr_drbg_sm_err_sum
318 || ctr_err_sum ||
err_code_test_bit[21];
319
320 // set the interrupt sources
321 assign event_cs_fatal_err = fatal_loc_events ||
main_sm_err_sum || (cs_enable_fo[1] &&
(|cmd_stage_sfifo_cmd_err_sum)
|| (|cmd_stage_sfifo_genbits_err_sum)
|| fifo_write_err_sum ||
fifo_read_err_sum || fifo_status_err_sum));
322
323 // Individual error signals/sources
324 assign cmd_stage_sm_err_sum = (|cmd_stage_sm_err) ||
err_code_test_bit[20];
325 assign main_sm_err_sum = cs_main_sm_err ||
err_code_test_bit[21];
326 assign ctr_drbg_sm_err_sum = ctr_drbg_sm_err ||
err_code_test_bit[22];
327 assign block_encrypt_sm_err_sum = block_encrypt_sm_err ||
err_code_test_bit[25];
328
329 // Collective error signals for each type of error
330 assign ctr_err_sum = (|cmd_stage_ctr_err) ||
ctr_drbg_v_ctr_err || err_code_test_bit[26];
331 assign fifo_write_err_sum = (|
cmd_stage_sfifo_genbits_err_wr) || (|cmd_stage_sfifo_cmd_err_wr) ||
err_code_test_bit[28];
332 assign fifo_read_err_sum = (|
cmd_stage_sfifo_genbits_err_rd) || (|cmd_stage_sfifo_cmd_err_rd) ||
err_code_test_bit[29];
333 assign fifo_status_err_sum = (|
cmd_stage_sfifo_genbits_err_st) || (|cmd_stage_sfifo_cmd_err_st) ||
err_code_test_bit[30];
334
335 // set the err code source bits
336 assign hw2reg.err_code.sfifo_cmd_err.d = 1'b1;
337
338 assign hw2reg.err_code.sfifo_cmd_err.de = cs_enable_fo[2] &&
(|cmd_stage_sfifo_cmd_err_sum);
339
340 assign hw2reg.err_code.sfifo_genbits_err.d = 1'b1;
341 assign hw2reg.err_code.sfifo_genbits_err.de = cs_en-
able_fo[3] &&
(|cmd_stage_sfifo_genbits_err_sum);
342
343 assign hw2reg.err_code.cmd_stage_sm_err.d = 1'b1;
344 assign hw2reg.err_code.cmd_stage_sm_err.de = cs_en-
able_fo[19] && cmd_stage_sm_err_sum;
345
346 assign hw2reg.err_code.main_sm_err.d = 1'b1;
347 assign hw2reg.err_code.main_sm_err.de = cs_enable_fo[20] &&

```

```

main_sm_err_sum;
352
353 assign hw2reg.err_code.ctr_drbg_sm_err.d = 1'b1;
354
355 assign hw2reg.err_code.ctr_drbg_sm_err.de = cs_enable_fo[21]
&& ctr_drbg_sm_err_sum;
356
357 assign hw2reg.err_code.aes_cipher_sm_err.d = 1'b1;
358 assign hw2reg.err_code.aes_cipher_sm_err.de = cs_en-
able_fo[22] && block_encrypt_sm_err_sum;
359
360 // set the err code type bits
361 assign hw2reg.err_code.ctr_err.d = 1'b1;
362 assign hw2reg.err_code.ctr_err.de = cs_enable_fo[23] &&
ctr_err_sum;
363
364 assign hw2reg.err_code.fifo_write_err.d = 1'b1;
365 assign hw2reg.err_code.fifo_write_err.de = cs_enable_fo[24]
&& fifo_write_err_sum;
366
367 assign hw2reg.err_code.fifo_read_err.d = 1'b1;
368 assign hw2reg.err_code.fifo_read_err.de = cs_enable_fo[25]
&& fifo_read_err_sum;
369
370 assign hw2reg.err_code.fifo_state_err.d = 1'b1;
371 assign hw2reg.err_code.fifo_state_err.de = cs_enable_fo[26]
&& fifo_status_err_sum;
372
373 // Error testing decoding
374 for (genvar i = 0; i < 31; i++) begin :
gen_err_code_test_bit
375   assign err_code_test_bit[i] = (reg2hw.err_code_test.q ==
i) && reg2hw.err_code_test.qe;
376   end : gen_err_code_test_bit
377
378 // alert - send all interrupt sources to the alert for the
fatal case
379 assign fatal_alert_o = event_cs_fatal_err;
380
381 // alert test
382 assign recov_alert_test_o = {
383   reg2hw.alert_test.recov_alert.q &&
384   reg2hw.alert_test.recov_alert.qe
385 };
386 assign fatal_alert_test_o = {
387   reg2hw.alert_test.fatal_alert.q &&
388   reg2hw.alert_test.fatal_alert.qe
389 };
390
391 assign recov_alert_event = cs_enable_pfa ||
sw_app_enable_pfa ||
read_int_state_pfa ||
acmd_flag0_pfa ||
|reseed_cnt_alert ||
|invalid_cmd_seq_alert ||
|invalid_acmd_alert ||
cs_bus_cmp_alert;
392
393 prim_edge_detector #(
394   .Width(1),
395   .ResetValue(0),
396   .EnSync(0)
397 ) u_prim_edge_detector_recov_alert (
398   .clk_i,
399   .rst_ni,
400   .d_i(recov_alert_event),
401   .q_sync_o(),
402   .q_posedge_pulse_o(recov_alert_o),
403   .q_negedge_pulse_o()
404 );
405
406 // check for illegal enable field states, and set alert if
detected
407
408 // SEC_CM: CONFIG.MUBI
409 mubi4_t mubi_cs_enable;
410 assign mubi_cs_enable = mubi4_t'(reg2hw.ctrl.enable.q);
411 assign cs_enable_pfa = mu-
bi4_test_invalid(mubi_cs_enable.fanout[0]);
412 assign hw2reg.recov_alert_sts.enable_field_alert.de = cs_en-
able_pfa;
413 assign hw2reg.recov_alert_sts.enable_field_alert.d = cs_en-
able_pfa;
414
415 for (genvar i = 1; i < CsEnableCopies; i = i+1) begin :
gen_mubi_en_copies
416   assign cs_enable_fo[i] = mu-
bi4_test_true_strict(mubi_cs_enable.fanout[i]);
417   end : gen_mubi_en_copies
418
419 prim_mubi4_sync #(
420   .NumCopies(CsEnableCopies),
421   .AsyncOn(0)
422 ) u_prim_mubi4_sync_cs_enable (
423   .clk_i,
424   .rst_ni,
425   .mubi_i(mubi_cs_enable),
426   .mubi_o(mubi_cs_enable.fanout)
427

```

```

436 );
437
438 // SEC_CM: CONFIG.MUBI
439 mubi4_t mubi_sw_app_enable;
440 mubi4_t [1:0] mubi_sw_app_enable_fanout;
441 assign mubi_sw_app_enable = mu-
bi4_t'(reg2hw.ctrl.sw_app_enable.q);
442 assign sw_app_enable_pfe = mu-
bi4_test_true_strict(mubi_sw_app_enable_fanout[0]);
443 assign sw_app_enable_pfa = mu-
bi4_test_invalid(mubi_sw_app_enable_fanout[1]);
444 assign hw2reg.recov_alert_sts.sw_app_enable_field_alert.de =
sw_app_enable_pfa;
445 assign hw2reg.recov_alert_sts.sw_app_enable_field_alert.d =
sw_app_enable_pfa;
446
447 prim_mubi4_sync #(
448   .NumCopies(2),
449   .AsyncOn(0)
450 ) u_prim_mubi4_sync_sw_app_enable (
451   .clk_i,
452   .rst_ni,
453   .mubi_i(mubi_sw_app_enable),
454   .mubi_o(mubi_sw_app_enable_fanout)
455 );
456
457 // SEC_CM: CONFIG.MUBI
458 mubi4_t mubi_read_int_state;
459 mubi4_t [1:0] mubi_read_int_state_fanout;
460 assign mubi_read_int_state = mu-
bi4_t'(reg2hw.ctrl.read_int_state.q);
461 assign read_int_state_pfe = mu-
bi4_test_true_strict(mubi_read_int_state_fanout[0]);
462 assign read_int_state_pfa = mu-
bi4_test_invalid(mubi_read_int_state_fanout[1]);
463 assign hw2reg.recov_alert_sts.read_int_state_field_alert.de
= read_int_state_pfa;
464 assign hw2reg.recov_alert_sts.read_int_state_field_alert.d
= read_int_state_pfa;
465
466 prim_mubi4_sync #(
467   .NumCopies(2),
468   .AsyncOn(0)
469 ) u_prim_mubi4_sync_read_int_state (
470   .clk_i,
471   .rst_ni,
472   .mubi_i(mubi_read_int_state),
473   .mubi_o(mubi_read_int_state_fanout)
474 );
475
476 // SEC_CM: CONFIG.MUBI
477 mubi4_t mubi_fips_force_enable;
478 mubi4_t [1:0] mubi_fips_force_enable_fanout;
479 assign mubi_fips_force_enable = mu-
bi4_t'(reg2hw.ctrl.fips_force_enable.q);
480 assign fips_force_enable_pfe = mu-
bi4_test_true_strict(mubi_fips_force_enable_fanout[0]);
481 assign fips_force_enable_pfa = mu-
bi4_test_invalid(mubi_fips_force_enable_fanout[1]);
482 assign
hw2reg.recov_alert_sts.fips_force_enable_field_alert.de =
fips_force_enable_pfa;
483 assign
hw2reg.recov_alert_sts.fips_force_enable_field_alert.d =
fips_force_enable_pfa;
484
485 prim_mubi4_sync #(
486   .NumCopies(2),
487   .AsyncOn(0)
488 ) u_prim_mubi4_sync_fips_force_enable (
489   .clk_i,
490   .rst_ni,
491   .mubi_i(mubi_fips_force_enable),
492   .mubi_o(mubi_fips_force_enable_fanout)
493 );
494
495 // master module enable
496 assign sw_app_enable = sw_app_enable_pfe;
497 assign read_int_state = read_int_state_pfe;
498 assign fips_force_enable = fips_force_enable_pfe;
499
500 //-----
501 // application interface
502 //-----
503 // Each application port has its own
504 // csrng_cmd_stage block to receive the
505 // command, track the state of its completion,
506 // and return any genbits if the command
507 // is a generate command.
508
509 for (genvar ai = 0; ai < NumApps; ai = ai+1) begin :
gen_cmd_stage
510
511   csrng_cmd_stage u_csrng_cmd_stage (
512     .clk_i (clk_i),
513     .rst_ni (rst_ni),
514     .cs_enable_i (cs_enable_fo[27]),
515     .cmd_stage_vld_i (cmd_stage_vld[ai]),
516     .cmd_stage_shd_i (cmd_stage_shd[ai]),

```

```

517     .cmd_stage_bus_i (cmd_stage_bus[ai]),
518     .cmd_stage_rdy_o (cmd_stage_rdy[ai]),
519     .reseed_cnt_reached_i (re-
seed_cnt_reached_q[ai]),
520     .reseed_cnt_alert_o (reseed_cnt_alert[ai]),
521     .invalid_cmd_seq_alert_o (inval-
id_cmd_seq_alert[ai]),
522     .invalid_acmd_alert_o (invalid_acmd_alert[ai]),
523     .cmd_arb_req_o (cmd_arb_req[ai]),
524     .cmd_arb_sop_o (cmd_arb_sop[ai]),
525     .cmd_arb_mop_o (cmd_arb_mop[ai]),
526     .cmd_arb_eop_o (cmd_arb_eop[ai]),
527     .cmd_arb_gnt_i (cmd_arb_gnt[ai]),
528     .cmd_arb_bus_o (cmd_arb_bus[ai]),
529     .cmd_ack_i (cmd_core_ack[ai]),
530     .cmd_ack_sts_i (cmd_core_ack_sts[ai]),
531     .cmd_stage_ack_o (cmd_stage_ack[ai]),
532     .cmd_stage_ack_sts_o (cmd_stage_ack_sts[ai]),
533     .genbits_vld_i (genbits_core_vld[ai]),
534     .genbits_bus_i (genbits_core_bus[ai]),
535     .genbits_fips_i (genbits_core_fips[ai]),
536     .genbits_vld_o (genbits_stage_vld[ai]),
537     .genbits_rdy_i (genbits_stage_rdy[ai]),
538     .genbits_bus_o (genbits_stage_bus[ai]),
539     .genbits_fips_o (genbits_stage_fips[ai]),
540     .cmd_stage_sfifo_cmd_err_o
(cmd_stage_sfifo_cmd_err[ai]),
541     .cmd_stage_sfifo_genbits_err_o(cmd_stage_sfifo_genbits_
err[ai]),
542     .cmd_gen_cnt_err_o (cmd_stage_ctr_err[ai]),
543     .cmd_stage_sm_err_o (cmd_stage_sm_err[ai])
544   );
545
546   // Set reseed_cnt_reached_d to true if the max number of
generate requests between reseeds
547   // has been reached for the respective counter.
548   assign reseed_cnt_reached_d[ai] = (ctr_drbg_rsp_vld &&
(ctr_drbg_rsp_data.inst_id == ai)) ?
549
550   (ctr_drbg_rsp_data.rs_ctr >= reg2hw.reseed_interval.q) :
re-
seed_cnt_reached_q[ai];
551
552   end : gen_cmd_stage
553
554   // SW interface connection (only 1, and must be present)
555   // cmd req
556   assign cmd_stage_vld[NumApps-1] = reg2hw.cmd_req.q;
557   assign cmd_stage_bus[NumApps-1] = reg2hw.cmd_req.q;
558   assign cmd_stage_shd[NumApps-1] = InstIdWidth'(NumApps-1);
559
560   assign hw2reg.sw_cmd_sts.cmd_rdy.de = 1'b1;
561   assign hw2reg.sw_cmd_sts.cmd_rdy.d = !
cmd_stage_vld[NumApps-1] && sw_rdy_sts_q;
562
563   assign sw_rdy_sts_d =
!cs_enable_fo[28] ? 1'b0 :
cmd_stage_vld[NumApps-1] ? 1'b0 :
cmd_stage_rdy[NumApps-1] ? 1'b1 :
sw_rdy_sts_q;
564
565   // cmd sts ack
566   assign hw2reg.sw_cmd_sts.cmd_ack.de = 1'b1;
567   assign hw2reg.sw_cmd_sts.cmd_ack.d = sw_sts_ack_d;
568   assign sw_sts_ack_d =
!cs_enable_fo[28] ? 1'b0 :
cmd_stage_vld[NumApps-1] ? 1'b0 :
cmd_stage_ack[NumApps-1] ? 1'b1 :
sw_sts_ack_q;
569
570   // cmd ack sts
571   assign hw2reg.sw_cmd_sts.cmd_sts.de =
cmd_stage_ack[NumApps-1];
572   assign hw2reg.sw_cmd_sts.cmd_sts.d =
cmd_stage_ack_sts[NumApps-1];
573
574   // genbits
575   assign hw2reg.genbits_vld.genbits_vld.d = gen-
bits_stage_vld_sw;
576   assign hw2reg.genbits_vld.genbits_fips.d = gen-
bits_stage_fips_sw_q;
577   assign hw2reg.genbits.d = (sw_app_enable &&
otp_sw_app_read_en[0]) ? genbits_stage_bus_sw : '0;
578   assign genbits_stage_bus_rd_sw = reg2hw.genbits.re;
579
580   assign otp_sw_app_read_en[0] =
prim_mubi_pkg:mubi8_test_true_strict(otp_sw_app_read_en_mubi[0]);
581   assign otp_sw_app_read_en[1] =
prim_mubi_pkg:mubi8_test_true_strict(otp_sw_app_read_en_mubi[1]);
582
583   prim_mubi8_sync #(
584     .NumCopies(2),
585     .AsyncOn(1)
586   ) u_prim_mubi8_sync_sw_app_read (
587     .clk_i,
588     .rst_ni,
589     .mubi_i(otp_en_csrng_sw_app_read_i),
590     .mubi_o(otp_sw_app_read_en_mubi)
591   );
592
593   // pack the gen bits into a 32 bit register sized word
594   prim_packer_fifo #(
595     .InW(BlkLen),

```

```

602 .outW(32),
603 .ClearOnRead(1'b0)
604 ) u_prim_packer_fifo_sw_genbits (
605   .clk_i (clk_i),
606   .rst_ni (rst_ni),
607   .clr_i (!cs_enable_fo[29]),
608   .wvalid_i(genbits_stage_vld[NumApps-1]),
609   .wdata_i (genbits_stage_bus[NumApps-1]),
610   .wready_o(genbits_stage_rdy[NumApps-1]),
611   .rvalid_o(genbits_stage_vld_sw),
612   .rdata_o (genbits_stage_bus_sw),
613   .rready_i(genbits_stage_bus_rd_sw),
614   .depth_o ()
615 );
616
617 // flops for SW fips status
618 assign genbits_stage_fips_sw_d =
619   (!cs_enable_fo[30]) ? 1'b0 :
620   (genbits_stage_rdy[NumApps-1] && gen-
bits_stage_vld[NumApps-1]) ?
621   gen-
bits_stage_fips[NumApps-1] :
622   gen-
bits_stage_fips_sw_q;
623
624 //-----
625 // data path integrity check
626 // - a countermeasure to detect entropy bus tampering at-
tempts
627 // - checks to make sure repeated data sets off an alert for
sw to handle
628 //-----
629
630 // SEC_CM: SW_GENBITS.BUS.CONSTISTENCY
631
632 // capture a copy of the genbits data
633 assign cs_rdata_capt_vld = (genbits_stage_vld[NumApps-1] &&
genbits_stage_rdy[NumApps-1]);
634
635 assign cs_rdata_capt_d = cs_rdata_capt_vld ? gen-
bits_stage_bus[NumApps-1][63:0] : cs_rdata_capt_q;
636
637 assign cs_rdata_capt_vld_d =
638   !cs_enable_fo[31] ? 1'b0 :
639   cs_rdata_capt_vld ? 1'b1 :
640   cs_rdata_capt_vld_q;
641
642 // continuous compare of the entropy data for sw port
643 assign cs_bus_cmp_alert = cs_rdata_capt_vld &&
cs_rdata_capt_vld_q &&
644   (cs_rdata_capt_q == genbits_stage_bus[NumApps-1]
[63:0]); // only look at 64 bits
645
646 assign hw2reg.recov_alert_sts.cs_bus_cmp_alert.de =
cs_bus_cmp_alert;
647 assign hw2reg.recov_alert_sts.cs_bus_cmp_alert.d =
cs_bus_cmp_alert;
648
649 assign
hw2reg.recov_alert_sts.cmd_stage_invalid_acmd_alert.de = |inval-
id_acmd_alert;
650 assign
hw2reg.recov_alert_sts.cmd_stage_invalid_acmd_alert.d = |inval-
id_acmd_alert;
651
652 assign
hw2reg.recov_alert_sts.cmd_stage_invalid_cmd_seq_alert.de = |inval-
id_cmd_seq_alert;
653 assign
hw2reg.recov_alert_sts.cmd_stage_invalid_cmd_seq_alert.d = |inval-
id_cmd_seq_alert;
654
655 assign hw2reg.recov_alert_sts.cmd_stage_reseed_cnt_alert.de =
|reseed_cnt_alert;
656 assign hw2reg.recov_alert_sts.cmd_stage_reseed_cnt_alert.d =
|reseed_cnt_alert;
657
658 // HW interface connections (up to 16, numbered 0-14)
659 for (genvar hai = 0; hai < (NumApps-1); hai = hai+1) begin :
gen_app_if
660   // cmd req
661   assign cmd_stage_vld[hai] =
csrng_cmd_i[hai].csrng_req_valid;
662   assign cmd_stage_shid[hai] = hai;
663   assign cmd_stage_bus[hai] =
csrng_cmd_i[hai].csrng_req_bus;
664   assign csrng_cmd_o[hai].csrng_req_ready =
cmd_stage_rdy[hai];
665   // cmd ack
666   assign csrng_cmd_o[hai].csrng_rsp_ack =
cmd_stage_ack[hai];
667   assign csrng_cmd_o[hai].csrng_rsp_sts =
cmd_stage_ack_sts[hai];
668   // genbits
669   assign csrng_cmd_o[hai].genbits_valid = gen-
bits_stage_vld[hai];
670   assign csrng_cmd_o[hai].genbits_fips = gen-
bits_stage_fips[hai];
671   assign csrng_cmd_o[hai].genbits_bus = gen-

```

```

bits_stage_bus[hai];
672   assign genbits_stage_rdy[hai] =
csrng_cmd_i[hai].genbits_ready;
673   end : gen_app_if
674
675   // Assign ack status
676   for (genvar i = 0; i < 16; i = i+1) begin : gen_app_if_sts
677     if (i < NumApps) begin : gen_assign
678       assign hw_exception_sts[i] = cmd_stage_ack[i] &&
(cmd_stage_ack_sts[i] != CMD_STS_SUCCESS);
679     end else begin : gen_dummy_assign
680       assign hw_exception_sts[i] = 1'b0;
681     end
682   end : gen_app_if_sts
683
684   // set fifo err status bits
685   for (genvar i = 0; i < NumApps; i = i+1) begin :
gen_fifo_sts
686     assign cmd_stage_sfifo_cmd_err_sum[i] = (|
cmd_stage_sfifo_cmd_err[i] ||
687     err_code_test_bit[0]);
688     assign cmd_stage_sfifo_cmd_err_wr[i] =
cmd_stage_sfifo_cmd_err[i][2];
689     assign cmd_stage_sfifo_cmd_err_rd[i] =
cmd_stage_sfifo_cmd_err[i][1];
690     assign cmd_stage_sfifo_cmd_err_st[i] =
cmd_stage_sfifo_cmd_err[i][0];
691     assign cmd_stage_sfifo_genbits_err_sum[i] = (|
cmd_stage_sfifo_genbits_err[i] ||
692     err_code_test_bit[1]);
693     assign cmd_stage_sfifo_genbits_err_wr[i] =
cmd_stage_sfifo_genbits_err[i][2];
694     assign cmd_stage_sfifo_genbits_err_rd[i] =
cmd_stage_sfifo_genbits_err[i][1];
695     assign cmd_stage_sfifo_genbits_err_st[i] =
cmd_stage_sfifo_genbits_err[i][0];
696   end : gen_fifo_sts
697
698   //-----
699   // app command arbiter and state machine
700   //-----
701   // All commands that arrive from the
702   // application ports are arbitrated for
703   // and processed by the main state machine
704   // logic block.
705
706   assign cmd_arb_idx_d = (acmd_avail && acmd_accept) ?
cmd_arb_idx : cmd_arb_idx_q;
707
708   assign acmd_sop = cmd_arb_sop[cmd_arb_idx_q]; // Start-of-
Packet
709   assign acmd_mop = cmd_arb_mop[cmd_arb_idx_q]; // Mid-of-
Packet
710   assign acmd_eop = cmd_arb_eop[cmd_arb_idx_q]; // End-of-
Packet
711   assign acmd_bus = cmd_arb_bus[cmd_arb_idx_q]; // Actual bus
data; content changes with s/m/eop
712
713   prim_arbiter_ppc #(
714     .EnDataPort(0), // Ignore data port
715     .N(NumApps), // Number of request ports
716     .DW(1) // Data width
717   ) u_prim_arbiter_ppc_acmd (
718     .clk_i (clk_i),
719     .rst_ni (rst_ni),
720     .req_chk_i(cs_enable_fo[1]),
721     .req_i (cmd_arb_req),
722     .data_i ('{default: 1'b0}),
723     .gnt_o (cmd_arb_gnt),
724     .idx_o (cmd_arb_idx),
725     .valid_o (acmd_avail), // 1 req
726     .data_o (), //NC
727     .ready_i (acmd_accept) // 1 fsm rdy
728   );
729
730   assign acmd_flag0_pfa = mubi4_test_invalid(flag0_q);
731   assign hw2reg.recov_alert_sts.acmd_flag0_field_alert.de =
acmd_flag0_pfa;
732   assign hw2reg.recov_alert_sts.acmd_flag0_field_alert.d =
acmd_flag0_pfa;
733
734   // parse the command bus
735   assign acmd_hold = acmd_sop ? acmd_e'(acmd_bus[Cmd-
Width-1:0]) : acmd_q;
736
737   // TODO(#28153) rewrite as an always_comb block
738   assign acmd_d =
739     (!cs_enable_fo[32]) ? INV :
740     acmd_sop ? acmd_e'(acmd_bus[CmdWidth-1:0]) :
741     acmd_q;
742
743   assign inst_id_d =
744     (!cs_enable_fo[33]) ? '0 :
745     acmd_sop ? acmd_bus[12+: NumAppsLg] :
746     inst_id_q;
747
748   assign gen_last_d =
749     (!cs_enable_fo[34]) ? '0 :
750     acmd_sop ? acmd_bus[16] :

```

```

751     gen_last_q;
752
753     assign flag0_d =
754         (!cs_enable_fo[35]) ? prim_mubi_pkg::MuBi4False :
755         (acmd_sop && ((acmd_bus[2:0] == INS) ||
756         (acmd_bus[2:0] == RES))) ?
757         mubi4_t'(acmd_bus[11:8]) : flag0_q;
758
759     // SEC_CM: CTRL.MUBI
760     mubi4_t mubi_flag0;
761     assign mubi_flag0 = flag0_q;
762
763     for (genvar i = 0; i < Flag0Copies; i = i+1) begin :
764     gen_mubi_flag0_copies
765         assign flag0_fo[i] = mu-
766         bi4_test_true_strict(mubi_flag0_fanout[i]);
767     end : gen_mubi_flag0_copies
768
769     prim_mubi4_sync #(
770         .NumCopies(Flag0Copies),
771         .AsyncOn(0)
772     ) u_prim_mubi4_sync_flag0 (
773         .clk_i,
774         .rst_ni,
775         .mubi_i(mubi_flag0),
776         .mubi_o(mubi_flag0_fanout)
777     );
778
779     // sm to process all instantiation requests
780     // SEC_CM: MAIN_SM.CTR.LOCAL_ESC
781     csrng_main_sm u_csrng_main_sm (
782         .clk_i (clk_i),
783         .rst_ni (rst_ni),
784         .enable_i (cs_enable_fo[36]),
785         .acmd_avail_i (acmd_avail),
786         .acmd_accept_o (acmd_accept),
787         .acmd_i (acmd_hold),
788         .acmd_eop_i (acmd_eop),
789         .flag0_i (flag0_fo[0]),
790         .cmd_entropy_req_o (cmd_entropy_req),
791         .cmd_entropy_avail_i (cmd_entropy_avail),
792         .cmd_vld_o (main_sm_cmd_vld),
793         .cmd_rdy_i (ctr_drbg_req_rdy),
794         .clr_adata_packer_o (clr_adata_packer),
795         .cmd_complete_i (ctr_drbg_rsp_vld),
796         .local_escalate_i (fatal_loc_events),
797         .main_sm_state_o (cs_main_sm_state),
798         .main_sm_err_o (cs_main_sm_err)
799     );
800
801     // entropy available
802     assign cmd_entropy_avail = entropy_src_hw_if_i.es_ack;
803
804     for (genvar csi = 0; csi < NumApps; csi = csi+1) begin :
805     gen_cmd_ack
806         assign cmd_core_ack[csi] = ctr_drbg_rsp_vld &&
807         (ctr_drbg_rsp_data.inst_id == csi);
808         assign cmd_core_ack_sts[csi] = ctr_drbg_rsp_sts;
809         assign genbits_core_vld[csi] = ctr_drbg_bits_vld &&
810         (ctr_drbg_rsp_data.inst_id == csi);
811         assign genbits_core_bus[csi] = ctr_drbg_bits_data;
812         assign genbits_core_fips[csi] = ctr_drbg_bits_fips;
813     end : gen_cmd_ack
814
815     prim_packer_fifo #(
816         .InW(32),
817         .OutW(SeedLen),
818         .ClearOnRead(1'b1)
819     ) u_prim_packer_fifo_adata (
820         .clk_i (clk_i),
821         .rst_ni (rst_ni),
822         .clr_i (!cs_enable_fo[37] || packer_adata_clr),
823         .wvalid_i(acmd_mop),
824         .wdata_i (acmd_bus),
825         .wready_o(),
826         .rvalid_o(),
827         .rddata_o (packer_adata),
828         .rready_i(packer_adata_pop),
829         .depth_o (packer_adata_depth)
830     );
831
832     assign packer_adata_pop = cs_enable_fo[38] &&
833     clr_adata_packer && (packer_adata_depth == AData-
834     aDepthClog'(CmdMaxClen));
835
836     assign packer_adata_clr = cs_enable_fo[39] &&
837     clr_adata_packer && (packer_adata_depth < ADataDepth-
838     Clog'(CmdMaxClen));
839
840     //-----
841     // csrng_state_db instantiation
842     //-----
843     // Holds the internal state of each csrng instance. Gets up-
844     dated after every command.
845
846     assign state_db_reg_read_en = cs_enable_fo[40] &&
847     read_int_state && otp_sw_app_read_en[1];
848
849     csrng_state_db u_csrng_state_db (
850         .clk_i (clk_i),
851
852         .rst_ni (rst_ni),
853         .enable_i(cs_enable_fo[41]),
854
855         .rd_inst_id_i(inst_id_q),
856         .rd_state_o (state_db_rd_data),
857
858         .wr_vld_i (state_db_wr_vld),
859         .wr_data_i (ctr_drbg_rsp_data),
860
861         .reg_rd_otp_en_i (state_db_reg_read_en),
862         .reg_rd_regfile_en_i(reg2hw.int_state_read_enable.q),
863
864         .reg_rd_id_vld_i(reg2hw.int_state_num.qe),
865         .reg_rd_id_i (reg2hw.int_state_num.q),
866         .reg_rd_strb_i (reg2hw.int_state_val.re),
867         .reg_rd_val_o (hw2reg.int_state_val.d),
868
869         .reseed_counter_o(reseed_counter)
870     );
871
872     // Forward the reseed counter values to the register inter-
873     face.
874     always_comb begin : reseed_counter_assign
875         for (int i = 0; i < NumApps; i++) begin
876             hw2reg.reseed_counter[i].d = reseed_counter[i];
877         end
878     end
879
880     //-----
881     // entropy interface
882     //-----
883     // Basic interface logic with the entropy_src block
884
885     assign entropy_src_hw_if_o.es_req = cs_enable_fo[43] &&
886     cmd_entropy_req;
887
888     // SEC_CM: CONSTANTS.LC_GATED
889     assign seed_diversification = lc_hw_debug_on_fo[0] ? RndCn-
890     stsCsKeymgrDivNonProduction :
891     RndCn-
892     stsCsKeymgrDivProduction;
893
894     // Capture entropy from entropy_src
895     assign entropy_src_seed_d =
896         flag0_fo[1] ? '0 : // special case where zero is used
897         cmd_entropy_req && cmd_entropy_avail ?
898         (entropy_src_hw_if_i.es_bits ^ seed_diversifica-
899         tion) :
900         entropy_src_seed_q;
901     assign entropy_src_fips_d =
902         // Use inst_id_d here such that u_csrng_ctr_drbg_cmd
903         gets the inst_id_q and the proper
904         // entropy_src_fips_q in the next clock cycle.
905         fips_force_enable && reg2hw.fips_force.q[inst_id_d] ?
906         1'b1 :
907         flag0_fo[2] ? '0 : // special case where zero is used
908         cmd_entropy_req && cmd_entropy_avail ? en-
909         tropy_src_hw_if_i.es_fips :
910         entropy_src_fips_q;
911
912     //-----
913     // CTR DRBG instantiation
914     //-----
915
916     assign ctr_drbg_req_vld = !cs_enable_fo[45] ? 1'b0 :
917     main_sm_cmd_vld;
918     assign ctr_drbg_cmd_d = !cs_enable_fo[44] ? INV :
919     acmd_hold;
920
921     assign ctr_drbg_req_data = '{
922         inst_id: inst_id_q,
923         cmd: ctr_drbg_cmd_q,
924         key: state_db_rd_data.key,
925         v: state_db_rd_data.v,
926         pdata: packer_adata,
927         rs_ctr: state_db_rd_data.rs_ctr,
928         fips: state_db_rd_data.fips
929     };
930
931     csrng_ctr_drbg u_csrng_ctr_drbg (
932         .clk_i (clk_i),
933         .rst_ni (rst_ni),
934         .enable_i(cs_enable_fo[49]),
935
936         .req_vld_i (ctr_drbg_req_vld),
937         .req_rdy_o (ctr_drbg_req_rdy),
938         .req_data_i (ctr_drbg_req_data),
939         .req_entropy_i (entropy_src_seed_q),
940         .req_entropy_fips_i(entropy_src_fips_q),
941         .req_glast_i (gen_last_q),
942
943         .rsp_vld_o (ctr_drbg_rsp_vld),
944         .rsp_data_o(ctr_drbg_rsp_data),
945         .rsp_sts_o (ctr_drbg_rsp_sts),
946
947         .bits_vld_o (ctr_drbg_bits_vld),
948         .bits_data_o(ctr_drbg_bits_data),
949         .bits_fips_o(ctr_drbg_bits_fips),
950
951

```

```

932 .state_db_wr_o(state_db_wr_vld),
933
934 .block_encrypt_req_vld_o (block_encrypt_req_vld),
935 .block_encrypt_req_rdy_i (block_encrypt_req_rdy),
936 .block_encrypt_req_data_o(block_encrypt_req_data),
937
938 .block_encrypt_rsp_vld_i (block_encrypt_rsp_vld),
939 .block_encrypt_rsp_rdy_o (block_encrypt_rsp_rdy),
940 .block_encrypt_rsp_data_i(block_encrypt_rsp_data),
941
942 .ctr_err_o(ctr_drbg_v_ctr_err),
943 .sm_err_o (ctr_drbg_sm_err)
944 );
945
946 //-----
947 // csrng_block_encrypt instantiation
948 //-----
949
950 csrng_block_encrypt #(
951   .SBoxImpl(SBoxImpl)
952 ) u_csrng_block_encrypt (
953   .clk_i   (clk_i),
954   .rst_ni  (rst_ni),
955   .enable_i(cs_enable_fo[48]),
956
957   .req_vld_i (block_encrypt_req_vld),
958   .req_rdy_o (block_encrypt_req_rdy),
959   .req_data_i(block_encrypt_req_data),
960
961   .rsp_vld_o (block_encrypt_rsp_vld),
962   .rsp_rdy_i (block_encrypt_rsp_rdy),
963   .rsp_data_o(block_encrypt_rsp_data),
964
965   .cipher_sm_err_o(block_encrypt_sm_err)
966 );
967
968 //-----
969 // life cycle logic
970 //-----
971 // The chip level life cycle control
972 // provide control logic to determine
973 // how certain debug features are controlled.
974
975 lc_ctrl_pkg::lc_tx_t [LcHwDebugCopies-1:0] lc_hw_de-
bug_en_out;
976
977 prim_lc_sync #(
978   .NumCopies(LcHwDebugCopies)
979 ) u_prim_lc_sync (
980   .clk_i,
981   .rst_ni,
982   .lc_en_i(lc_hw_debug_en_i),
983   .lc_en_o({lc_hw_debug_en_out})
984 );
985
986 for (genvar i = 0; i < LcHwDebugCopies; i = i+1) begin :
gen_lc_dbg_copies
987   assign lc_hw_debug_on_fo[i] = lc_c-
trl_pkg::lc_tx_test_true_strict(lc_hw_debug_en_out[i]);
988 end : gen_lc_dbg_copies
989
990 //-----
991 // observe state machine
992 //-----
993
994 assign hw2reg.main_sm_state.de = 1'b1;
995 assign hw2reg.main_sm_state.d = cs_main_sm_state;
996
997 //-----
998 // report csrng request summary
999 //-----
1000 // Misc status
1001
1002 assign hw2reg.hw_exc_sts.de = cs_enable_fo[50];
1003 assign hw2reg.hw_exc_sts.d = hw_exception_sts;
1004
1005 // unused signals
1006 logic unused_err_code_test_bit;
1007 logic unused_enable_fo;
1008 logic unused_reg2hw_genbits;
1009 logic unused_int_state_val;
1010 logic unused_reseed_interval;
1011 logic [SeedLen-1:0] unused_gen_rsp_pdata;
1012 logic unused_state_db_inst_state;
1013
1014 assign unused_err_code_test_bit = err_code_test_bit[27] ||
(!err_code_test_bit[24:23]) ||
1015
(|
err_code_test_bit[19:2]);
1016 assign unused_enable_fo = (!cs_enable_fo[47:46]) || cs_en-
able_fo[42] || (!cs_enable_fo[18:4]);
1017 assign unused_reg2hw_genbits = (!reg2hw.genbits.q);
1018 assign unused_int_state_val = (!reg2hw.int_state_val.q);
1019 assign unused_reseed_interval = reg2hw.reseed_interval.q;
1020 assign unused_gen_rsp_pdata = ctr_drbg_rsp_data.pdata;
1021 assign unused_state_db_inst_state =
state_db_rd_data.inst_state;
1022
1023 //-----
1024 // Assertions
1025 //-----

```

```

1026 `ifdef INC_ASSERT
1027   logic state_db_zeroize;
1028   assign state_db_zeroize = state_db_wr_vld &&
(ctr_drbg_rsp_data.cmd == UNI);
1029   `ASSERT(CsrngUniZeroizeFips_A, state_db_zeroize ->
(ctr_drbg_rsp_data.fips == '0))
1030   `ASSERT(CsrngUniZeroizeKey_A, state_db_zeroize ->
(ctr_drbg_rsp_data.key == '0))
1031   `ASSERT(CsrngUniZeroizeV_A, state_db_zeroize ->
(ctr_drbg_rsp_data.v == '0))
1032   `ASSERT(CsrngUniZeroizeRc_A, state_db_zeroize ->
(ctr_drbg_rsp_data.rs_ctr == '0))
1033   `ASSERT(CsrngUniZeroizeSts_A, state_db_zeroize ->
(ctr_drbg_rsp_sts == CMD_STS_SUCCESS))
1034 `endif
1035
1036 endmodule // csrng_core

```

5. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_ctr_drbg.sv (533 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Implementation of the NIST SP800-90A CTR DRBG algorithm, no de-
rivation function.
6
7 `include "prim_assert.sv"
8
9 module csrng_ctr_drbg import csrng_pkg::*; (
10   input logic clk_i,
11   input logic rst_ni,
12
13   // Global enable
14   input logic enable_i,
15
16   // Command interface request, arbitrated from command stages
17   input logic req_vld_i,
18   output logic req_rdy_o,
19   input csrng_core_data_t req_data_i,
20   input logic [SeedLen-1:0] req_entropy_i,
21   input logic req_entropy_fips_i,
22   input logic req_glast_i,
23
24   // Command interface response to state db and command stages
25   output logic state_db_wr_o,
26   output logic rsp_vld_o,
27   output csrng_core_data_t rsp_data_o,
28   output csrng_cmd_sts_e rsp_sts_o,
29
30   // Generated bits output to command stages
31   output logic bits_vld_o,
32   output logic [BlkLen-1:0] bits_data_o,
33   output logic bits_fips_o,
34
35   // Block encrypt interface request
36   output logic block_encrypt_req_vld_o,
37   input logic block_encrypt_req_rdy_i,
38   output csrng_key_v_t block_encrypt_req_data_o,
39
40   // Block encrypt interface response
41   input logic block_encrypt_rsp_vld_i,
42   output logic block_encrypt_rsp_rdy_o,
43   input logic [BlkLen-1:0] block_encrypt_rsp_data_i,
44
45   // Error status outputs
46   output logic ctr_err_o,
47   output logic sm_err_o
48 );
49
50 import csrng_reg_pkg::NumApps;
51
52 // Number of calls to the block cipher required for each
call of UPDate()
53 localparam int unsigned NumBlkPerUpd = SeedLen/BlkLen;
54 localparam int unsigned NumBlkPerUpdLg = $clog2(NumBlkPerUpd);
55
56 // The GENERate command consists of up to three sub commands
typedef enum logic [1:0] {
57   UPD_BEGIN,
58   GEN_LOOP,
59   UPD_FINAL
60 } gen_subcmd_e;
61
62 //-----
63 // Sparse FSM encoding and instantiation
64 //-----
65
66 // Encoding generated with:
67 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 8 -n 6 \
68 // -s 923483574589 --language=sv
69 //
70 //
71 // Hamming distance histogram:
72 //
73 // 0: --

```



```

74 // 1: --
75 // 2: --
76 // 3: ||| (57.14%)
77 // 4: ||| (42.86%)
78 // 5: --
79 // 6: --
80 //
81 // Minimum Hamming distance: 3
82 // Maximum Hamming distance: 4
83 // Minimum Hamming weight: 1
84 // Maximum Hamming weight: 5
85 //
86 localparam int StateWidth = 6;
87 typedef enum logic [StateWidth-1:0] {
88     Idle = 6'b101110,
89     CtrInc = 6'b100001,
90     ReqSend = 6'b110100,
91     RspWait = 6'b010111,
92     Hndshk = 6'b001101,
93     HndshkGen = 6'b011000,
94     HndshkLoad = 6'b111011,
95     Error = 6'b000010
96 } state_e;
97
98 state_e state_q, state_d;
99
100 // SEC_CM: CTR_DRBG.FSM.SPARSE
101 PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
state_e, Idle)
102
103 //-----
104 // Signals
105 //-----
106
107 // Main data structure that encapsulates most of the
(meta)data required in this module
108 csrng_core_data_t core_data;
109
110 // Control and data signals for the v and concat counters
111 logic [BlkLen-1:0] v_ctr_sized;
112 logic [CtrLen-1:0] v_ctr;
113 logic v_ctr_load;
114 logic v_ctr_inc;
115 csrng_key_v_t concat_key_v;
116 logic concat_ctr_inc;
117 logic concat_ctr_done;
118
119 //-----
120 // Registers, including sequential process
121 //-----
122
123 // adata flops for each app interface, plus one valid bit
for each
124 // This rather large bank of flops is unfortunately required
due to the way that we handle
125 // GENERate commands with more than one block of generated
output bits: Instead of generating all
126 // blocks in one go, we always generate one block and then
allow the arbiter to give another app
127 // interface access to the data path. However, on the last
generated block, we require the adata
128 // that are supplied at the very beginning of the GENERate
command for the final update() step
129 // (compare NIST specification). Since all app interfaces
share a single packer for adata, its
130 // content can get overwritten after the first block of gen-
erated bits, and hence we must buffer
131 // adata for each app interface locally for the full "life-
time" of a multi-block GENERate command.
132 // TODO(#28153) Find a smaller solution (giving each app in-
terface an unpacker does not help)
133 csrng_key_v_t [NumApps-1:0] generate_adata_q, gener-
ate_adata_d;
134 logic [NumApps-1:0] generate_adata_vld_q, gener-
ate_adata_vld_d;
135
136 // Scratch register for commands that call block_encrypt
more than once and pointer into it
137 logic [SeedLen-1:0] concat_key_v_q, concat_key_v_d;
138 logic [NumBlkPerUpdLg-1:0] concat_ctr_q, concat_ctr_d;
139
140 // Keep track of current sub command for GENERates
141 gen_subcmd_e gen_subcmd_q, gen_subcmd_d;
142
143 always_ff @(posedge clk_i or negedge rst_ni) begin
144     if (!rst_ni) begin
145         generate_adata_q <= '{default: '0};
146         generate_adata_vld_q <= '0;
147         concat_ctr_q <= '0;
148         concat_key_v_q <= '0;
149         gen_subcmd_q <= UPD_BEGIN;
150     end else begin
151         generate_adata_q <= generate_adata_d;
152         generate_adata_vld_q <= generate_adata_vld_d;
153         concat_ctr_q <= concat_ctr_d;
154         concat_key_v_q <= concat_key_v_d;
155         gen_subcmd_q <= gen_subcmd_d;
156     end
157 end
158

```

```

159 //-----
160 // Prepare/mux input values depending on cmd
161 //-----
162
163 always_comb begin
164     core_data = req_data_i;
165
166     unique case (req_data_i.cmd)
167     INS: begin
168         // Reset key, v, and reseed counter; inject entropy
and FIPS status
169         core_data.key = '0;
170         core_data.v = '0;
171         core_data.rs_ctr = '0;
172         core_data.pdata = req_data_i.pdata ^ req_entropy_i;
173         core_data.fips = req_entropy_fips_i;
174     end
175     RES: begin
176         // Reset reseed counter; inject entropy and FIPS
status
177         core_data.rs_ctr = '0;
178         core_data.pdata = req_data_i.pdata ^ req_entropy_i;
179         core_data.fips = req_entropy_fips_i;
180     end
181     UPD, GEN: begin
182         /* leave everything as-is */
183     end
184     UNI: begin
185         // Set everything to zero
186         core_data.key = '0;
187         core_data.v = '0;
188         core_data.rs_ctr = '0;
189         core_data.pdata = '0;
190         core_data.fips = 'b0;
191     end
192     default: begin
193         // Invalid command, should never happen
194         core_data.cmd = INV;
195     end
196 endcase
197 end
198
199 //-----
200 // Handling of generate_adata(_vld)
201 //-----
202
203 for (genvar i = 0; i < NumApps; i++) begin :
g_assign_gen_adata
204     logic capt_adata;
205     logic clear_adata_vld;
206     // 1) Write adata to local buffer upon a GENERate command
and _vld not being set
207     // 2) Clear _vld when the last GENERate beat appears on
the cmd_rsp port
208     assign capt_adata = req_vld_i && (core_data.cmd == GEN) &&
(core_data.inst_id == i) &&
!generate_adata_vld_q[i];
209     assign clear_adata_vld = rsp_vld_o && req_glast_i &&
(core_data.inst_id == i);
210
211     always_comb begin
212         generate_adata_vld_d[i] = generate_adata_vld_q[i];
213         generate_adata_d[i] = generate_adata_q[i];
214
215         if (!enable_i) begin
216             generate_adata_vld_d[i] = 'b0;
217             generate_adata_d[i] = '0;
218         end else if (capt_adata) begin
219             generate_adata_vld_d[i] = 'b1;
220             generate_adata_d[i] = core_data.pdata;
221         end else if (clear_adata_vld) begin
222             generate_adata_vld_d[i] = 'b0;
223         end
224     end
225 end
226
227 //-----
228 // Counter logic for v
229 //-----
230
231 // SEC_CM: CTR_DRBG.CTR.REDUN
232 prim_count #(
233     .Width(CtrLen),
234     .PossibleActions(prim_count_pkg::Clr |
prim_count_pkg::Set |
prim_count_pkg::Incr)
235 ) u_prim_count_ctr_drbg (
236     .clk_i,
237     .rst_ni,
238     .clr_i (!enable_i),
239     .set_i (v_ctr_load),
240     .set_cnt_i(core_data.v[CtrLen-1:0]),
241     .incr_en_i(v_ctr_inc),
242     .decr_en_i('b0),
243     .step_i (CtrLen'(1)),
244     .commit_i ('b1),
245     .cnt_o (v_ctr),
246

```

```

252 .cnt_after_commit_o(),
253 .err_o (ctr_err_o)
254 );
255
256 // Combine the MSBs of the initial v from the state db with
the current counter value as LSBs
257 assign v_ctr_sized = {core_data.v[Blklen-1:CtrLen], v_ctr};
258
259 //-----
260 // Block encrypt request data assign
261 //-----
262
263 assign block_encrypt_req_data_o = '{
264   key: core_data.key,
265   v: v_ctr_sized
266 }';
267
268 //-----
269 // Scratch register for block-wise derivation of new {key,v}
270 //-----
271
272 // Count the number of blocks that have been received back
from block_encrypt until we have enough
273 // blocks for at least SeedLen bits in total.
274 assign concat_ctr_done = (concat_ctr_q >= NumBlkPerUpd);
275
276 // Type conversion for easier usage
277 assign concat_key_v = csrng_key_v_t'(concat_key_v_q);
278
279 always_comb begin
280   concat_ctr_d = concat_ctr_q;
281   concat_key_v_d = concat_key_v_q;
282
283   if (!enable_i || concat_ctr_done) begin
284     concat_ctr_d = '0;
285     concat_key_v_d = '0;
286   end else if (concat_ctr_inc) begin
287     concat_ctr_d = concat_ctr_q + 1;
288     // Steer the v-response from block encrypt to the cor-
rect lane, MSB down.
289     // concat_ctr_q must be lower than NumBlkPerUpd, other-
wise concat_ctr_done is set and we end
290     // up in the if branch above, i.e., the index into con-
cat_key_v_d is always valid.
291     concat_key_v_d[(NumBlkPerUpd - 1 - concat_ctr_q) *
Blklen +: Blklen]
292     = block_encrypt_rsp_data_i;
293   end
294 end
295
296 //-----
297 // FSM to orchestrate everything
298 //-----
299
300 always_comb begin
301   state_d = state_q;
302   gen_subcmd_d = gen_subcmd_q;
303
304   // Handshake and data validity
305   req_rdy_o = 'b0;
306   rsp_vld_o = 'b0;
307   bits_vld_o = 'b0;
308   state_db_wr_o = 'b0;
309
310   block_encrypt_req_vld_o = 'b0;
311   block_encrypt_rsp_rdy_o = 'b0;
312
313   // Counter control
314   v_ctr_load = 'b0;
315   v_ctr_inc = 'b0;
316   concat_ctr_inc = 'b0;
317
318   // Error output
319   sm_err_o = 'b0;
320
321   unique case (state_q)
322     Idle: begin
323       if (enable_i && req_vld_i) begin
324         unique case (core_data.cmd)
325           UNI: begin
326             // No further action needed, write to state db
and flag response valid
327             req_rdy_o = 'b1;
328             rsp_vld_o = 'b1;
329             state_db_wr_o = 'b1;
330           end
331           INS, RES, UPD: begin
332             // Load the counter with v from state db
333             v_ctr_load = 'b1;
334             state_d = CtrInc;
335           end
336           GEN: begin
337             v_ctr_load = 'b1;
338             // Decide which GENerate sub command to execute
339             // TODO(#28153) Clarify what the exact condition
for skipping the initial UPDATE()
340             // call on GENerate commands is (pdata/adata be-
ing an all-zero vector or pdata/adata
341             // LENGTH being zero).
342             if (core_data.pdata != '0) begin

```

```

343         gen_subcmd_d = UPD_BEGIN;
344       end else begin
345         gen_subcmd_d = GEN_LOOP;
346       end
347       state_d = CtrInc;
348     end
349     default: begin
350       // Everything else is an error
351       state_d = Error;
352     end
353   endcase
354   end else begin
355     // There is no "clear" on the AES cipher, so just
flush out any outstanding responses
356     // Otherwise, there will be erroneous handshakes
when re-enabling the CSRNG
357     block_encrypt_rsp_rdy_o = 'b1;
358   end
359   end
360   ReqSend: begin
361     // Once a request has been accepted, also process its
response.
362     block_encrypt_req_vld_o = 'b1;
363     if (block_encrypt_req_rdy_i) begin
364       state_d = RspWait;
365     end
366   end
367   RspWait: begin
368     block_encrypt_rsp_rdy_o = 'b1;
369     if (block_encrypt_rsp_vld_i) begin
370       // In the actual generate part of GENerate, the re-
sponse of the cipher is the generated
371       // random bits. In all other cases, the response
ends up in the scratch register.
372       if ((core_data.cmd == GEN) && (gen_subcmd_q ==
GEN_LOOP)) begin
373         bits_vld_o = 'b1;
374       end else begin
375         v_ctr_inc = 'b1;
376         concat_ctr_inc = 'b1;
377       end
378       // GENerate requires a more involved, separate logic
on how to continue
379       if (core_data.cmd == GEN) begin
380         state_d = HndshkGen;
381       end else begin
382         state_d = Hndshk;
383       end
384     end
385   end
386   Hndshk: begin
387     if (concat_ctr_done) begin
388       // Enough data has been received to make up the new
{key,v}; command is completed
389       req_rdy_o = 'b1;
390       rsp_vld_o = 'b1;
391       state_db_wr_o = 'b1;
392       state_d = Idle;
393     end else begin
394       // More data required
395       state_d = ReqSend;
396     end
397   end
398   HndshkGen: begin
399     unique case (gen_subcmd_q)
400       UPD_BEGIN: begin
401         if (concat_ctr_done) begin
402           // We have to commit the derived {key,v} to the
state db as they are the basis for the
403           // UPD_FINAL step which we potentially must ex-
ecute, also requiring the scratch reg
404           state_db_wr_o = 'b1;
405           gen_subcmd_d = GEN_LOOP;
406           state_d = HndshkLoad;
407         end else begin
408           state_d = ReqSend;
409         end
410       end
411       GEN_LOOP: begin
412         if (req_glast_i) begin
413           // Do the initial v increment for the UPD_FINAL
step that follows
414           v_ctr_inc = 'b1;
415           gen_subcmd_d = UPD_FINAL;
416           state_d = ReqSend;
417         end else begin
418           // This was not the last beat of the generate
loop; signal the command as completed
419           req_rdy_o = 'b1;
420           rsp_vld_o = 'b1;
421           state_db_wr_o = 'b1;
422           state_d = Idle;
423         end
424       end
425       UPD_FINAL: begin
426         if (concat_ctr_done) begin
427           // Enough data has been received to make up the
new {key,v}; command is completed
428           req_rdy_o = 'b1;
429           rsp_vld_o = 'b1;

```

```

430         state_db_wr_o = 1'b1;
431         state_d = Idle;
432     end else begin
433         // More data required
434         state_d = ReqSend;
435     end
436 end
437 default: begin
438     // We should never get here
439     state_d = Error;
440 end
441 endcase
442 end
443 HndshkLoad: begin
444     // Transitional state to load the prim_count with the
new v
445     v_ctr_load = 1'b1;
446     state_d = CtrInc;
447 end
448 CtrInc: begin
449     // Transitional state to perform initial v-increment
after (re)loading it from state db
450     v_ctr_inc = 1'b1;
451     state_d = ReqSend;
452 end
453 Error: begin
454     sm_err_o = 1'b1;
455 end
456 default: begin
457     state_d = Error;
458     sm_err_o = 1'b1;
459 end
460 endcase
461
462 // Avoid wrapping every state body in an `if (enable_i)`
by collectively handling lenable_i
463 // here, at least for most states.
464 if (!enable_i && (state_q == RspWait)) begin
465     // We have to handle this edge case, where there is an
outstanding response from block_encrypt
466     // while CSRNG gets disabled. Wait until the response
arrives; otherwise, it can arrive at a
467     // somewhat random time during other states and might
corrupt or deadlock the control flow
468     if (block_encrypt_rsp_vld_i) begin
469         state_d = Idle;
470     end
471 end else if (!(state_q inside {Idle, Error}) && (state_d !=
Error)) begin
472     // Be careful to not enable error-state escape by acci-
dent (checked by assertions).
473     if (!enable_i) begin
474         state_d = Idle;
475     end
476 end
477 end
478
479 //-----
480 // Response/data out assignments
481 //-----
482
483 // Make sure the command stages get an error response in
case we received an invalid command
484 assign rsp_sts_o = (core_data.cmd == INV) ? CMD_STS_INVALID_
ID_ACMD : CMD_STS_SUCCESS;
485
486 always_comb begin
487     // Default case only relevant for UNInstantiate and inval-
id commands
488     rsp_data_o = core_data;
489
490     // For all commands that update the internal state, send
new {key,v} to state db
491
492     // For GENerate commands, we need a more involved handling
of the three sub commands
493     if (core_data.cmd inside {INS, RES, UPD}) begin
494         rsp_data_o.key = concat_key_v.key ^
core_data.pdata[BlkLen+:KeyLen];
495         rsp_data_o.v = concat_key_v.v ^
core_data.pdata[BlkLen-1:0];
496     end else if (core_data.cmd == GEN) begin
497         unique case (gen_subcmd_q)
498             UPD_BEGIN: begin
499                 rsp_data_o.key = concat_key_v.key ^
core_data.pdata[BlkLen+:KeyLen];
500                 rsp_data_o.v = concat_key_v.v ^
core_data.pdata[BlkLen-1:0];
501             end
502             GEN_LOOP: begin
503                 rsp_data_o.key = core_data.key; // Unchanged
504                 rsp_data_o.v = v_ctr_sized; // Old value + 1
505             end
506             UPD_FINAL: begin
507                 rsp_data_o.key = concat_key_v.key ^ gener-
ate_adata_q[core_data.inst_id].key;
508                 rsp_data_o.v = concat_key_v.v ^ gener-
ate_adata_q[core_data.inst_id].v;
509             end
510         end
511         // Increase the reseed counter on the last beat of
every GENerate
512         rsp_data_o.rs_ctr = core_data.rs_ctr + 1;

```

```

510     end
511     default: /* We should never get here */ ;
512 endcase
513 end
514 end
515
516 // Genbits out; actual bits taken directly from block cipher
517 assign bits_data_o = block_encrypt_rsp_data_i;
518 assign bits_fips_o = core_data.fips;
519
520 //-----
521 // Assertions and unused signal tie-off
522 //-----
523
524 // Currently not supported, but most probably never intended
(would yield a very wide counter)
525 `ASSERT_INIT(CsrngCtrLenLessBlkLen, CtrLen < BlkLen)
526
527 // Make sure the FSM has a stable error state that cannot be
escaped
528 `ASSERT(CsrngCtrDrbgSmErrorStStable_A, state_q == Error |>=
$stable(state_q))
529
530 // Outside of any non-error state, the FSM error output must
be asserted
531 `ASSERT(CsrngCtrDrbgSmErrorOutput_A, !(state_q inside
{Idle, CtrInc, ReqSend, RspWait, Hndshk, HndshkGen,
HndshkLoad}))
532 |>= sm_err_o)
533 endmodule

```

6. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_main_sm.sv (136 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: csrng app cmd request state machine module
6 //
7 // - handles all app cmd requests from all requesting inter-
faces
8
9 `include "prim_assert.sv"
10
11 module csrng_main_sm import csrng_pkg::*; (
12     input logic clk_i,
13     input logic rst_ni,
14     input logic enable_i,
15
16     input logic acmd_avail_i,
17     output logic acmd_accept_o,
18     input acmd_e acmd_i,
19     input logic acmd_eop_i,
20
21     input logic flag0_i,
22
23     output logic cmd_entropy_req_o,
24     input logic cmd_entropy_avail_i,
25
26     output logic cmd_vld_o,
27     input logic cmd_rdy_i,
28     output logic clr_adata_packer_o,
29     input logic cmd_complete_i,
30
31     input logic local_escalate_i,
32
33     output logic [MainSmStateWidth-1:0] main_sm_state_o,
34     output logic main_sm_err_o
35 );
36
37 // SEC_CM: MAIN_SM.FSM.SPARSE
38 main_sm_state_e state_d, state_q;
39 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
main_sm_state_e, MainSmIdle)
40
41 assign main_sm_state_o = {state_q};
42
43 always_comb begin
44     state_d = state_q;
45     acmd_accept_o = 1'b0;
46     cmd_entropy_req_o = 1'b0;
47     cmd_vld_o = 1'b0;
48     clr_adata_packer_o = 1'b0;
49     main_sm_err_o = 1'b0;
50
51     if (state_q == MainSmError) begin
52         // In case we are in the Error state we must ignore the
local escalate and enable signals.
53         main_sm_err_o = 1'b1;
54     end else if (local_escalate_i) begin
55         // In case local escalate is high we must transition to
the error state.
56         state_d = MainSmError;
57     end else if (!enable_i && state_q inside {MainSmIdle,
MainSmParseCmd, MainSmEntropyReq,

```



```

58                                     MainSmCmdPrep,
MainSmCmdVld,
59                                     MainSmClrAData,
MainSmCmdCompWait}} begin
60 // In case the module is disabled and we are in a legal
state we must go into idle state.
61 state_d = MainSmIdle;
62 end else begin
63 // Otherwise do the state machine as normal.
64 unique case (state_q)
65 MainSmIdle: begin
66 // Signal the arbiter to grant this request.
67 if (acmd_avail_i) begin
68 acmd_accept_o = 1'b1;
69 state_d = MainSmParseCmd;
70 end
71 end
72 MainSmParseCmd: begin
73 if (acmd_eop_i) begin
74 unique case (acmd_i)
75 INS, RES: state_d = MainSmEntropyReq; //
Command may require entropy
76 GEN, UPD, UNI: state_d = MainSmCmdPrep; //
Command does not require entropy
77 default: state_d = MainSmIdle; //
Command was not supported
78 endcase
79 end
80 end
81 MainSmEntropyReq: begin
82 if (flag0_i) begin
83 // With flag0 set, no entropy is required.
84 state_d = MainSmCmdVld;
85 end else begin
86 // Delay one clock to fix timing issue.
87 cmd_entropy_req_o = 1'b1;
88 if (cmd_entropy_avail_i) begin
89 state_d = MainSmCmdVld;
90 end
91 end
92 end
93 MainSmCmdPrep: begin
94 // Assumes all adata is present now.
95 state_d = MainSmCmdVld;
96 end
97 MainSmCmdVld: begin
98 cmd_vld_o = 1'b1;
99 if (cmd_rdy_i) begin
100 if (cmd_complete_i) begin
101 clr_adata_packer_o = 1'b1;
102 state_d = MainSmIdle;
103 end else begin
104 state_d = MainSmClrAData;
105 end
106 end
107 end
108 MainSmClrAData: begin
109 clr_adata_packer_o = 1'b1;
110 if (cmd_complete_i) begin
111 state_d = MainSmIdle;
112 end else begin
113 state_d = MainSmCmdCompWait;
114 end
115 end
116 MainSmCmdCompWait: begin
117 if (cmd_complete_i) begin
118 state_d = MainSmIdle;
119 end
120 end
121 // Error: The error state is now covered by the if
statement above.
122 default: begin
123 state_d = MainSmError;
124 main_sm_err_o = 1'b1;
125 end
126 endcase
127 end
128 end
129
130 // Make sure that the state machine has a stable error
state. This means that after the error
131 // state is entered it will not exit it unless a reset sig-
nal is received.
132 `ASSERT(CsrngMainErrorStStable_A, state_q == MainSmError |>
$Stable(state_q))
133 // If in error state, the error output must be high.
134 `ASSERT(CsrngMainErrorOutput_A, state_q == MainSmError |>
main_sm_err_o)
135
136 endmodule

```

7. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_pkg.sv (161 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0

```

```

4 //
5
6 package csrng_pkg;
7
8 parameter int unsigned NumAppsLg = $clog2(csrng_reg_pkg::NumApps);
9
10 // Main design parameters
11 // We use the suffix 'Len' (as opposed to 'Width') to stick
to the symbols used in the
12 // NIST SP 800-90A algorithm specification.
13
14 // Size of the generated random number blocks in bits and
the internal state variable 'v'
15 parameter int unsigned BlkLen = 128;
16 // Cipher key size in bits
17 parameter int unsigned KeyLen = 256;
18
19 // Seed size in bits; must be equal to the sum of BlkLen and
KeyLen
20 parameter int unsigned SeedLen = BlkLen + KeyLen;
21 // Width of the counter field used as input to the cipher
22 // Caution: Not to be confused with the reseed counter
width! These two are independent
23 parameter int unsigned CtrLen = 32;
24 parameter int unsigned RsCtrWidth = 32;
25
26 // Width of the counter in the command stages to count the
amount of generated random bits
27 parameter int unsigned GenBitsCtrWidth = 12;
28
29 // Commonly used internal signal widths
30 parameter int unsigned CmdWidth = 3;
31 parameter int unsigned InstIdWidth = 4;
32
33 // Width of the application command bus (matches the 32b
TLUL bus width)
34 parameter int unsigned CmdBusWidth = 32;
35
36 // Maximum number of 32b words additionally supplied on the
application interfaces
37 parameter int unsigned CmdMaxClen = 12;
38 parameter int unsigned CmdFifoDepth = 2;
39 parameter int unsigned CmdFifoDepthLg = $clog2(CmdFifoDepth);
40
41 parameter int unsigned GENBITS_BUS_WIDTH = BlkLen;
42 parameter int unsigned FIPS_GENBITS_BUS_WIDTH = en-
tropy_src_pkg::FIPS_BUS_WIDTH +
43 GENBITS_BUS_WIDTH;
44
45 //-----
46 // Application Interface
47 //-----
48
49 typedef struct packed {
50 logic csrng_req_valid;
51 logic [CmdBusWidth-1:0] csrng_req_bus;
52 logic genbits_ready;
53 } csrng_req_t;
54
55 parameter int unsigned CmdStatusWidth = 3;
56
57 typedef enum logic [CmdStatusWidth-1:0] {
58 CMD_STS_SUCCESS = 'h0,
59 CMD_STS_INVALID_ACMD = 'h1,
60 CMD_STS_INVALID_GEN_CMD = 'h2,
61 CMD_STS_INVALID_CMD_SEQ = 'h3,
62 CMD_STS_RESEED_CNT_EXCEEDED = 'h4,
63 CMD_STS_UNDRIVEN = 'z
64 } csrng_cmd_sts_e;
65
66 typedef struct packed {
67 logic csrng_req_ready;
68 logic csrng_rsp_ack;
69 logic [CmdStatusWidth-1:0] csrng_rsp_sts;
70 logic genbits_valid;
71 logic genbits_fips;
72 logic [GENBITS_BUS_WIDTH-1:0] genbits_bus;
73 } csrng_rsp_t;
74
75 parameter csrng_req_t CSRNG_REQ_DEFAULT = '{default: '0};
76 parameter csrng_rsp_t CSRNG_RSP_DEFAULT = '0;
77
78 typedef enum logic [CmdWidth-1:0] {
79 INV = 3'h0,
80 INS = 3'h1,
81 RES = 3'h2,
82 GEN = 3'h3,
83 UPD = 3'h4,
84 UNI = 3'h5
85 } acmd_e;
86
87 typedef struct packed {
88 logic [7:0] resv;
89 logic [11:0] glen;
90 logic [3:0] flag0;
91 logic [3:0] clen;
92 logic gap; // acmd is defined as 4 bits wide but
only 3 are used
93 acmd_e acmd;

```

```

93 } csrng_cmd_t;
94
95 //-----
96 // Struct data types for the core data path
97 //-----
98
99 typedef struct packed {
100     logic fips;
101     logic [RsCtrWidth-1:0] rs_ctr;
102     logic [SeedLen-1:0] pdata;
103     logic [NumAppsLg-1:0] inst_id;
104     acmd_e cmd;
105     logic [KeyLen-1:0] key;
106     logic [BlkLen-1:0] v;
107 } csrng_core_data_t;
108
109 typedef struct packed {
110     logic [KeyLen-1:0] key;
111     logic [BlkLen-1:0] v;
112 } csrng_key_v_t;
113
114 // Do not reorder these fields - software expects them in
this order when doing the raw
115 // state readout through the register interface.
116 typedef struct packed {
117     logic fips;
118     logic inst_state;
119     logic [KeyLen-1:0] key;
120     logic [BlkLen-1:0] v;
121     logic [RsCtrWidth-1:0] rs_ctr;
122 } csrng_state_t;
123
124 parameter int unsigned StateWidth = $bits(csrng_state_t);
125
126 parameter int unsigned MainSmStateWidth = 6;
127
128 // Encoding generated with:
129 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 8 -n 6 \
130 // -s 137328258 --language=sv
131 //
132 // Hamming distance histogram:
133 //
134 // 0: --
135 // 1: --
136 // 2: --
137 // 3: ||||| (57.14%)
138 // 4: ||||| (42.86%)
139 // 5: --
140 // 6: --
141 //
142 // Minimum Hamming distance: 3
143 // Maximum Hamming distance: 4
144 // Minimum Hamming weight: 1
145 // Maximum Hamming weight: 5
146 //
147 typedef enum logic [MainSmStateWidth-1:0] {
148     MainSmIdle = 6'b110111, // idle
149     MainSmParseCmd = 6'b011101, // parse the cmd
150     MainSmEntropyReq = 6'b001110, // request entropy if ne-
cessary
151     MainSmCmdPrep = 6'b000011, // delay cycle for command
request (?)
152     MainSmCmdVld = 6'b010000, // command request to core
data path
153     MainSmClrAdata = 6'b111010, // clear out the additional
data packer fifo
154     MainSmCmdCompWait = 6'b100100, // wait for command to com-
plete
155     MainSmError = 6'b101001 // error state, results in
fatal alert
156 } main_sm_state_e;
157
158 parameter int CsKeymgrDivWidth = 384;
159 typedef logic [CsKeymgrDivWidth-1:0] cs_keymgr_div_t;
160
161 endpackage : csrng_pkg

```

8. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_reg_pkg.sv (421 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Package auto-generated by `reggen` containing data
structure
6
7 package csrng_reg_pkg;
8
9 // Param list
10 parameter int NumApps = 3;
11 parameter int NumAlerts = 2;
12
13 // Address widths within the block
14 parameter int BlockAw = 7;
15

```

```

16 // Number of registers for every interface
17 parameter int NumRegs = 24;
18
19 // Alert indices
20 typedef enum int {
21     AlertRecovAlertIdx = 0,
22     AlertFatalAlertIdx = 1
23 } csrng_alert_idx_t;
24
25 //-----
26 // Typedefs for registers //
27 //-----
28
29 typedef struct packed {
30     struct packed {
31         logic q;
32     } cs_fatal_err;
33     struct packed {
34         logic q;
35     } cs_hw_inst_exc;
36     struct packed {
37         logic q;
38     } cs_entropy_req;
39     struct packed {
40         logic q;
41     } cs_cmd_req_done;
42 } csrng_reg2hw_intr_state_reg_t;
43
44 typedef struct packed {
45     struct packed {
46         logic q;
47     } cs_fatal_err;
48     struct packed {
49         logic q;
50     } cs_hw_inst_exc;
51     struct packed {
52         logic q;
53     } cs_entropy_req;
54     struct packed {
55         logic q;
56     } cs_cmd_req_done;
57 } csrng_reg2hw_intr_enable_reg_t;
58
59 typedef struct packed {
60     struct packed {
61         logic q;
62         logic qe;
63     } cs_fatal_err;
64     struct packed {
65         logic q;
66         logic qe;
67     } cs_hw_inst_exc;
68     struct packed {
69         logic q;
70         logic qe;
71     } cs_entropy_req;
72     struct packed {
73         logic q;
74         logic qe;
75     } cs_cmd_req_done;
76 } csrng_reg2hw_intr_test_reg_t;
77
78 typedef struct packed {
79     struct packed {
80         logic q;
81         logic qe;
82     } fatal_alert;
83     struct packed {
84         logic q;
85         logic qe;
86     } recov_alert;
87 } csrng_reg2hw_alert_test_reg_t;
88
89 typedef struct packed {
90     struct packed {
91         logic [3:0] q;
92     } fips_force_enable;
93     struct packed {
94         logic [3:0] q;
95     } read_int_state;
96     struct packed {
97         logic [3:0] q;
98     } sw_app_enable;
99     struct packed {
100         logic [3:0] q;
101     } enable;
102 } csrng_reg2hw_ctrl_reg_t;
103
104 typedef struct packed {
105     logic [31:0] q;
106     logic qe;
107 } csrng_reg2hw_cmd_req_reg_t;
108
109 typedef struct packed {
110     logic [31:0] q;
111     logic qe;
112 } csrng_reg2hw_reseed_interval_reg_t;
113
114 typedef struct packed {
115     logic [31:0] q;
116     logic re;

```

```

117 } csrng_reg2hw_genbits_reg_t;
118
119 typedef struct packed {
120     logic [2:0] q;
121 } csrng_reg2hw_int_state_read_enable_reg_t;
122
123 typedef struct packed {
124     logic [3:0] q;
125     logic qe;
126 } csrng_reg2hw_int_state_num_reg_t;
127
128 typedef struct packed {
129     logic [31:0] q;
130     logic re;
131 } csrng_reg2hw_int_state_val_reg_t;
132
133 typedef struct packed {
134     logic [2:0] q;
135 } csrng_reg2hw_fips_force_reg_t;
136
137 typedef struct packed {
138     logic [4:0] q;
139     logic qe;
140 } csrng_reg2hw_err_code_test_reg_t;
141
142 typedef struct packed {
143     struct packed {
144         logic d;
145         logic de;
146     } cs_fatal_err;
147     struct packed {
148         logic d;
149         logic de;
150     } cs_hw_inst_exc;
151     struct packed {
152         logic d;
153         logic de;
154     } cs_entropy_req;
155     struct packed {
156         logic d;
157         logic de;
158     } cs_cmd_req_done;
159 } csrng_hw2reg_intr_state_reg_t;
160
161 typedef struct packed {
162     logic [31:0] d;
163 } csrng_hw2reg_reseed_counter_mreg_t;
164
165 typedef struct packed {
166     struct packed {
167         logic [2:0] d;
168         logic de;
169     } cmd_sts;
170     struct packed {
171         logic d;
172         logic de;
173     } cmd_ack;
174     struct packed {
175         logic d;
176         logic de;
177     } cmd_rdy;
178 } csrng_hw2reg_sw_cmd_sts_reg_t;
179
180 typedef struct packed {
181     struct packed {
182         logic d;
183     } genbits_fips;
184     struct packed {
185         logic d;
186     } genbits_vld;
187 } csrng_hw2reg_genbits_vld_reg_t;
188
189 typedef struct packed {
190     logic [31:0] d;
191 } csrng_hw2reg_genbits_reg_t;
192
193 typedef struct packed {
194     logic [31:0] d;
195 } csrng_hw2reg_int_state_val_reg_t;
196
197 typedef struct packed {
198     logic [15:0] d;
199     logic de;
200 } csrng_hw2reg_hw_exc_sts_reg_t;
201
202 typedef struct packed {
203     struct packed {
204         logic d;
205         logic de;
206     } cmd_stage_reseed_cnt_alert;
207     struct packed {
208         logic d;
209         logic de;
210     } cmd_stage_invalid_cmd_seq_alert;
211     struct packed {
212         logic d;
213         logic de;
214     } cmd_stage_invalid_acmd_alert;
215     struct packed {
216         logic d;
217         logic de;

```

```

218 } cs_bus_cmp_alert;
219 struct packed {
220     logic d;
221     logic de;
222 } acmd_flag0_field_alert;
223 struct packed {
224     logic d;
225     logic de;
226 } fips_force_enable_field_alert;
227 struct packed {
228     logic d;
229     logic de;
230 } read_int_state_field_alert;
231 struct packed {
232     logic d;
233     logic de;
234 } sw_app_enable_field_alert;
235 struct packed {
236     logic d;
237     logic de;
238 } enable_field_alert;
239 } csrng_hw2reg_recov_alert_sts_reg_t;
240
241 typedef struct packed {
242     struct packed {
243         logic d;
244         logic de;
245     } fifo_state_err;
246     struct packed {
247         logic d;
248         logic de;
249     } fifo_read_err;
250     struct packed {
251         logic d;
252         logic de;
253     } fifo_write_err;
254     struct packed {
255         logic d;
256         logic de;
257     } ctr_err;
258     struct packed {
259         logic d;
260         logic de;
261     } aes_cipher_sm_err;
262     struct packed {
263         logic d;
264         logic de;
265     } ctr_drbg_sm_err;
266     struct packed {
267         logic d;
268         logic de;
269     } main_sm_err;
270     struct packed {
271         logic d;
272         logic de;
273     } cmd_stage_sm_err;
274     struct packed {
275         logic d;
276         logic de;
277     } sfifo_genbits_err;
278     struct packed {
279         logic d;
280         logic de;
281     } sfifo_cmd_err;
282 } csrng_hw2reg_err_code_reg_t;
283
284 typedef struct packed {
285     logic [5:0] d;
286     logic de;
287 } csrng_hw2reg_main_sm_state_reg_t;
288
289 // Register -> HW type
290 typedef struct packed {
291     csrng_reg2hw_intr_state_reg_t intr_state; // [184:181]
292     csrng_reg2hw_intr_enable_reg_t intr_enable; // [180:177]
293     csrng_reg2hw_intr_test_reg_t intr_test; // [176:169]
294     csrng_reg2hw_alert_test_reg_t alert_test; // [168:165]
295     csrng_reg2hw_ctrl_reg_t ctrl; // [164:149]
296     csrng_reg2hw_cmd_req_reg_t cmd_req; // [148:116]
297     csrng_reg2hw_reseed_interval_reg_t reseed_interval; //
[115:83]
298     csrng_reg2hw_genbits_reg_t genbits; // [82:50]
299     csrng_reg2hw_int_state_read_enable_reg_t
int_state_read_enable; // [49:47]
300     csrng_reg2hw_int_state_num_reg_t int_state_num; // [46:42]
301     csrng_reg2hw_int_state_val_reg_t int_state_val; // [41:9]
302     csrng_reg2hw_fips_force_reg_t fips_force; // [8:6]
303     csrng_reg2hw_err_code_test_reg_t err_code_test; // [5:0]
304 } csrng_reg2hw_t;
305
306 // HW -> register type
307 typedef struct packed {
308     csrng_hw2reg_intr_state_reg_t intr_state; // [239:232]
309     csrng_hw2reg_reseed_counter_mreg_t [2:0] re-
seed_counter; // [231:136]
310     csrng_hw2reg_sw_cmd_sts_reg_t sw_cmd_sts; // [135:128]
311     csrng_hw2reg_genbits_vld_reg_t genbits_vld; // [127:126]
312     csrng_hw2reg_genbits_reg_t genbits; // [125:94]
313     csrng_hw2reg_int_state_val_reg_t int_state_val; // [93:62]
314     csrng_hw2reg_hw_exc_sts_reg_t hw_exc_sts; // [61:45]
315     csrng_hw2reg_recov_alert_sts_reg_t recov_alert_sts; //

```

```

[44:27]
316     csrng_hw2reg_err_code_reg_t err_code; // [26:7]
317     csrng_hw2reg_main_sm_state_reg_t main_sm_state; // [6:0]
318 } csrng_hw2reg_t;
319
320 // Register offsets
321 parameter logic [BlockAw-1:0] CSRNG_INTR_STATE_OFFSET = 7'h
0;

322 parameter logic [BlockAw-1:0] CSRNG_INTR_ENABLE_OFFSET = 7'h
4;
323 parameter logic [BlockAw-1:0] CSRNG_INTR_TEST_OFFSET = 7'h
8;
324 parameter logic [BlockAw-1:0] CSRNG_ALERT_TEST_OFFSET = 7'h
c;
325 parameter logic [BlockAw-1:0] CSRNG_REGWEN_OFFSET = 7'h 10;
326 parameter logic [BlockAw-1:0] CSRNG_CTRL_OFFSET = 7'h 14;
327 parameter logic [BlockAw-1:0] CSRNG_CMD_REQ_OFFSET = 7'h 18;

328 parameter logic [BlockAw-1:0] CSRNG_RESEED_INTERVAL_OFFSET =
7'h 1c;
329 parameter logic [BlockAw-1:0] CSRNG_RESEED_COUNTER_0_OFFSET
= 7'h 20;
330 parameter logic [BlockAw-1:0] CSRNG_RESEED_COUNTER_1_OFFSET
= 7'h 24;
331 parameter logic [BlockAw-1:0] CSRNG_RESEED_COUNTER_2_OFFSET
= 7'h 28;
332 parameter logic [BlockAw-1:0] CSRNG_SW_CMD_STS_OFFSET = 7'h
2c;

333 parameter logic [BlockAw-1:0] CSRNG_GENBITS_VLD_OFFSET = 7'h
30;
334 parameter logic [BlockAw-1:0] CSRNG_GENBITS_OFFSET = 7'h 34;
335 parameter logic [BlockAw-1:0]
CSRNG_INT_STATE_READ_ENABLE_OFFSET = 7'h 38;
336 parameter logic [BlockAw-1:0]
CSRNG_INT_STATE_READ_ENABLE_REGWEN_OFFSET = 7'h 3c;
337 parameter logic [BlockAw-1:0] CSRNG_INT_STATE_NUM_OFFSET =
7'h 40;
338 parameter logic [BlockAw-1:0] CSRNG_INT_STATE_VAL_OFFSET =
7'h 44;
339 parameter logic [BlockAw-1:0] CSRNG_FIPS_FORCE_OFFSET = 7'h
48;
340 parameter logic [BlockAw-1:0] CSRNG_HW_EXC_STS_OFFSET = 7'h
4c;

341 parameter logic [BlockAw-1:0] CSRNG_RECOV_ALERT_STS_OFFSET =
7'h 50;
342 parameter logic [BlockAw-1:0] CSRNG_ERR_CODE_OFFSET = 7'h
54;
343 parameter logic [BlockAw-1:0] CSRNG_ERR_CODE_TEST_OFFSET =
7'h 58;
344 parameter logic [BlockAw-1:0] CSRNG_MAIN_SM_STATE_OFFSET =
7'h 5c;
345
346 // Reset values for hwext registers and their fields
347 parameter logic [3:0] CSRNG_INTR_TEST_RESVAL = 4'h 0;

348 parameter logic [0:0] CSRNG_INTR_TEST_CS_CMD_REQ_DONE_RESVAL =
1'h 0;
349 parameter logic [0:0] CSRNG_INTR_TEST_CS_ENTROPY_REQ_RESVAL
= 1'h 0;
350 parameter logic [0:0] CSRNG_INTR_TEST_CS_HW_INST_EXC_RESVAL
= 1'h 0;
351 parameter logic [0:0] CSRNG_INTR_TEST_CS_FATAL_ERR_RESVAL =
1'h 0;
352 parameter logic [1:0] CSRNG_ALERT_TEST_RESVAL = 2'h 0;
353 parameter logic [0:0] CSRNG_ALERT_TEST_RECOV_ALERT_RESVAL =
1'h 0;
354 parameter logic [0:0] CSRNG_ALERT_TEST_FATAL_ALERT_RESVAL =
1'h 0;
355 parameter logic [31:0] CSRNG_RESEED_COUNTER_0_RESVAL = 32'h

0;
356 parameter logic [31:0]
CSRNG_RESEED_COUNTER_0_RESEED_COUNTER_0_RESVAL = 32'h 0;
357 parameter logic [31:0] CSRNG_RESEED_COUNTER_1_RESVAL = 32'h
0;
358 parameter logic [31:0]
CSRNG_RESEED_COUNTER_1_RESEED_COUNTER_1_RESVAL = 32'h 0;
359 parameter logic [31:0] CSRNG_RESEED_COUNTER_2_RESVAL = 32'h
0;
360 parameter logic [31:0]
CSRNG_RESEED_COUNTER_2_RESEED_COUNTER_2_RESVAL = 32'h 0;
361 parameter logic [1:0] CSRNG_GENBITS_VLD_RESVAL = 2'h 0;
362 parameter logic [31:0] CSRNG_GENBITS_RESVAL = 32'h 0;
363 parameter logic [31:0] CSRNG_INT_STATE_VAL_RESVAL = 32'h 0;
364
365 // Register index
366 typedef enum int {
367     CSRNG_INTR_STATE,
368     CSRNG_INTR_ENABLE,
369     CSRNG_INTR_TEST,
370     CSRNG_ALERT_TEST,
371     CSRNG_REGWEN,
372     CSRNG_CTRL,
373     CSRNG_CMD_REQ,
374     CSRNG_RESEED_INTERVAL,
375     CSRNG_RESEED_COUNTER_0,
376     CSRNG_RESEED_COUNTER_1,
377     CSRNG_RESEED_COUNTER_2,
378     CSRNG_SW_CMD_STS,
379     CSRNG_GENBITS_VLD,
380     CSRNG_GENBITS,
381     CSRNG_INT_STATE_READ_ENABLE,
382     CSRNG_INT_STATE_READ_ENABLE_REGWEN,
383     CSRNG_INT_STATE_NUM,
384     CSRNG_INT_STATE_VAL,
385     CSRNG_FIPS_FORCE,
386     CSRNG_HW_EXC_STS,
387     CSRNG_RECOV_ALERT_STS,
388     CSRNG_ERR_CODE,
389     CSRNG_ERR_CODE_TEST,
390     CSRNG_MAIN_SM_STATE
391 } csrng_id_e;
392
393 // Register width information to check illegal writes
394 parameter logic [3:0] CSRNG_PERMIT [24] = '{
395     4'b 0001, // index[ 0] CSRNG_INTR_STATE
396     4'b 0001, // index[ 1] CSRNG_INTR_ENABLE
397     4'b 0001, // index[ 2] CSRNG_INTR_TEST
398     4'b 0001, // index[ 3] CSRNG_ALERT_TEST
399     4'b 0001, // index[ 4] CSRNG_REGWEN
400     4'b 0011, // index[ 5] CSRNG_CTRL
401     4'b 1111, // index[ 6] CSRNG_CMD_REQ
402     4'b 1111, // index[ 7] CSRNG_RESEED_INTERVAL
403     4'b 1111, // index[ 8] CSRNG_RESEED_COUNTER_0
404     4'b 1111, // index[ 9] CSRNG_RESEED_COUNTER_1
405     4'b 1111, // index[10] CSRNG_RESEED_COUNTER_2
406     4'b 0001, // index[11] CSRNG_SW_CMD_STS
407     4'b 0001, // index[12] CSRNG_GENBITS_VLD
408     4'b 1111, // index[13] CSRNG_GENBITS
409     4'b 0001, // index[14] CSRNG_INT_STATE_READ_ENABLE
410     4'b 0001, // index[15] CSRNG_INT_STATE_READ_ENABLE_REGWEN
411     4'b 0001, // index[16] CSRNG_INT_STATE_NUM
412     4'b 1111, // index[17] CSRNG_INT_STATE_VAL
413     4'b 0001, // index[18] CSRNG_FIPS_FORCE
414     4'b 0011, // index[19] CSRNG_HW_EXC_STS
415     4'b 0011, // index[20] CSRNG_RECOV_ALERT_STS
416     4'b 1111, // index[21] CSRNG_ERR_CODE
417     4'b 0001, // index[22] CSRNG_ERR_CODE_TEST
418     4'b 0001, // index[23] CSRNG_MAIN_SM_STATE
419 };
420
421 endpackage

```

OpenTitan 엔트로피원 (SP800-90B)

What this is. Extracts entropy from a physical noise source and runs continuous health tests on it.

Why it exists. Because a broken noise source that keeps producing output is worse than one that stops. Physical sources can become stuck or biased, and keys derived from them would be predictable. The standard therefore mandates repetition-count and adaptive-proportion tests running at all times.

Undergraduate review · EECS concepts. Min-entropy as the security-relevant measure; statistical hypothesis testing; ring-oscillator jitter arising from thermal noise.

Graduate · Practice note. This block cannot be validated by simulation. RTL verification covers the test logic; actual entropy quality is measured on silicon. A modelling engineer should state that boundary explicitly in the verification plan rather than let it be assumed away.

13 files • 4,920 lines

API — entropy_src.sv

Table 857. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src.sv — module / interface / package (1개)

이름	종류
entropy_src	module

Table 858. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src.sv — parameter (8개)

파라미터	형/폭	기본값
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
RngBusWidth	int	4
RngBusBitSelWidth	int	2
HealthTestWindowWidth	int	18
EsFifoDepth	int	3
DistrFifoDepth	int	3
Stub	bit	1'b0

Table 859. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src.sv — localparam (5개)

상수	형/폭	값
BucketHtDataWidth	int	bucket_ht_data_width(RngBusWidth)
NumBuckets	int	2**BucketHtDataWidth
NumBucketHtInst	int unsigned	num_bucket_ht_inst(RngBusWidth)
StubLfsrWidth	int	64
Copies	int	CSRNG_BUS_WIDTH / StubLfsrWidth

Table 860. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src.sv — 포트 (24개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
otp_en_entropy_src_fw_read_i	input	mubi8_t
otp_en_entropy_src_fw_over_i	input	mubi8_t
rng_fips_o	output	logic
entropy_src_hw_if_i	input	entropy_src_hw_if_req_t
entropy_src_hw_if_o	output	entropy_src_hw_if_rsp_t
entropy_src_rng_enable_o	output	logic
entropy_src_rng_valid_i	input	logic
entropy_src_rng_bits_i	input	logic [RngBusWidth-1:0]
entropy_src_xht_valid_o	output	logic
entropy_src_xht_bits_o	output	logic [RngBusWidth-1:0]
entropy_src_xht_bit_sel_o	output	logic [RngBusBitSelWidth-1:0]
entropy_src_xht_health_test_window_o	output	logic [HealthTestWindowWidth-1:0]
entropy_src_xht_meta_o	output	entropy_src_xht_meta_req_t
entropy_src_xht_meta_i	input	entropy_src_xht_meta_rsp_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
intr_es_entropy_valid_o	output	logic
intr_es_health_test_failed_o	output	logic
intr_es_observe_fifo_ready_o	output	logic
intr_es_fatal_err_o	output	logic

API — entropy_src_ack_sm.sv**Table 861. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_ack_sm.sv — module / interface / package (1개)**

이름	종류
entropy_src_ack_sm	module

Table 862. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_ack_sm.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic

포트	방향	형/폭
enable_i	input	logic
req_i	input	logic
ack_o	output	logic
fifo_not_empty_i	input	logic
local_escalate_i	input	logic
fifo_pop_o	output	logic
ack_sm_err_o	output	logic

API — entropy_src_ack_sm_pkg.sv

Table 863. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_ack_sm_pkg.sv — module / interface / package (1개)

이름	종류
entropy_src_ack_sm_pkg	package

Table 864. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_ack_sm_pkg.sv — localparam (1개)

상수	형/폭	값
StateWidth	int	6

API — entropy_src_adaptp_ht.sv

Table 865. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_adaptp_ht.sv — module / interface / package (1개)

이름	종류
entropy_src_adaptp_ht	module

Table 866. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_adaptp_ht.sv — parameter (3개)

파라미터	형/폭	기본값
RegWidth	int	16
RngBusWidth	int	4
RngBusBitSelWidth	int	2

Table 867. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_adaptp_ht.sv — 포트 (17개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
entropy_bit_i	input	logic [RngBusWidth-1:0]
entropy_bit_vld_i	input	logic
rng_bit_en_i	input	logic
rng_bit_sel_i	input	logic [RngBusBitSelWidth-1:0]

포트	방향	형/폭
clear_i	input	logic
active_i	input	logic
thresh_hi_i	input	logic [RegWidth-1:0]
thresh_lo_i	input	logic [RegWidth-1:0]
window_wrap_pulse_i	input	logic
threshold_scope_i	input	logic
test_cnt_hi_o	output	logic [RegWidth-1:0]
test_cnt_lo_o	output	logic [RegWidth-1:0]
test_fail_hi_pulse_o	output	logic
test_fail_lo_pulse_o	output	logic
count_err_o	output	logic

API — entropy_src_bucket_ht.sv

Table 868. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_bucket_ht.sv — module / interface / package (1개)

이름	종류
entropy_src_bucket_ht	module

Table 869. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_bucket_ht.sv — parameter (2개)

파라미터	형/폭	기본값
RegWidth	int	16
RngBusWidth	int	4

Table 870. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_bucket_ht.sv — localparam (1개)

상수	형/폭	값
NUM_BINS	int	2**RngBusWidth

Table 871. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_bucket_ht.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
entropy_bit_i	input	logic [RngBusWidth-1:0]
entropy_bit_vld_i	input	logic
clear_i	input	logic
active_i	input	logic
thresh_i	input	logic [RegWidth-1:0]
window_wrap_pulse_i	input	logic

포트	방향	형/폭
test_cnt_o	output	logic [RegWidth-1:0]
test_fail_pulse_o	output	logic
count_err_o	output	logic

API — entropy_src_cntr_reg.sv

Table 872. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_cntr_reg.sv — module / interface / package (1개)

이름	종류
entropy_src_cntr_reg	module

Table 873. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_cntr_reg.sv — parameter (1개)

파라미터	형/폭	기본값
RegWidth	int	16

Table 874. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_cntr_reg.sv — 포트 (7개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
clear_i	input	logic
event_i	input	logic
step_i	input	logic [RegWidth-1:0]
value_o	output	logic [RegWidth-1:0]
err_o	output	logic

API — entropy_src_core.sv

Table 875. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_core.sv — module / interface / package (1개)

이름	종류
entropy_src_core	module

Table 876. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_core.sv — parameter (7개)

파라미터	형/폭	기본값
RngBusWidth	int	4
RngBusBitSelWidth	int	2
HealthTestWindowWidth	int	18
EsFifoDepth	int	3
DistrFifoDepth	int	2
BucketHtDataWidth	int	4

파라미터	형/폭	기본값
NumBucketHtInst	int	prim_util_pkg::ceil_div(RngBusWidth

Table 877. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_core.sv — localparam (21개)

상수	형/폭	값
EsFifoDepthW	int	prim_util_pkg::vbits(EsFifoDepth + 1)
PostHTWidth	int	32
PostHTFifoDepthW	int	prim_util_pkg::vbits(PostHTWidth / RngBusWidth + 1)
HalfRegWidth	int	16
FullRegWidth	int	32
EighthRegWidth	int	4
SeedLen	int	384
DistrFifoWidth	int	32
DistrFifoDepthW	int	prim_util_pkg::vbits(DistrFifoDepth + 1)
ObserveFifoWidth	int	32
PreCondWidth	int	64
PreCondFifoDepthW	int	prim_util_pkg::vbits(PreCondWidth / PostHTWidth + 1)
PipelineDepthW	int	prim_util_pkg::vbits(1 + DistrFifoDepth + PreCondWidth / Pos...
Clog2ObserveFifoDepth	int	\$clog2(ObserveFifoDepth)
EsEnableCopies	int	20
EsEnPulseCopies	int	1
NumSwReadsSeed	int unsigned	SeedLen / FullRegWidth
LastSwRead	int unsigned	NumSwReadsSeed - 1
SwReadIdxWidth	int unsigned	prim_util_pkg::vbits(NumSwReadsSeed)
Sha3EnMasking	bit	0
Sha3Share	int	(Sha3EnMasking) ? 2 : 1

Table 878. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_core.sv — 포트 (26개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
reg2hw	input	entropy_src_reg_pkg::entropy_src_reg2hw_t
hw2reg	output	entropy_src_reg_pkg::entropy_src_hw2reg_t
otp_en_entropy_src_fw_read_i	input	prim_mubi_pkg::mubi8_t
otp_en_entropy_src_fw_over_i	input	prim_mubi_pkg::mubi8_t
rng_fips_o	output	logic
entropy_src_hw_if_i	input	entropy_src_hw_if_req_t

포트	방향	형/폭
entropy_src_hw_if_o	output	entropy_src_hw_if_rsp_t
entropy_src_rng_enable_o	output	logic
entropy_src_rng_valid_i	input	logic
entropy_src_rng_bits_i	input	logic [RngBusWidth-1:0]
entropy_src_xht_valid_o	output	logic
entropy_src_xht_bits_o	output	logic [RngBusWidth-1:0]
entropy_src_xht_bit_sel_o	output	logic[RngBusBitSelWidth-1:0]
entropy_src_xht_health_test_window_o	output	logic [HealthTestWindowWidth-1:0]
entropy_src_xht_meta_o	output	entropy_src_xht_meta_req_t
entropy_src_xht_meta_i	input	entropy_src_xht_meta_rsp_t
recov_alert_test_o	output	logic
fatal_alert_test_o	output	logic
recov_alert_o	output	logic
fatal_alert_o	output	logic
intr_es_entropy_valid_o	output	logic
intr_es_health_test_failed_o	output	logic
intr_es_observe_fifo_ready_o	output	logic
intr_es_fatal_err_o	output	logic

API — entropy_src_enable_delay.sv

Table 879. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_enable_delay.sv — module / interface / package (1개)

이름	종류
entropy_src_enable_delay	module

Table 880. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_enable_delay.sv — localparam (1개)

상수	형/폭	값
MaxFifoWait	int	4

Table 881. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_enable_delay.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
enable_i	input	logic
esrng_fifo_not_empty_i	input	logic
esbit_fifo_not_empty_i	input	logic
postht_fifo_not_empty_i	input	logic

포트	방향	형/폭
distr_fifo_not_empty_i	input	logic
sha3_block_busy_i	input	logic
sha3_block_processed_i	input	logic
bypass_mode_i	input	logic
enable_o	output	logic

API — entropy_src_field_en.sv

Table 882. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_field_en.sv — module / interface / package (1개)

이름	종류
entropy_src_field_en	module

Table 883. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_field_en.sv — parameter (2개)

파라미터	형/폭	기본값
FieldW	int	4
FieldEnVal	int	'ha

Table 884. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_field_en.sv — localparam (2개)

상수	형/폭	값
FieldEnValBit	logic [FieldW-1:0]	FieldW'(FieldEnVal)
FieldEnValBitRevert	logic [FieldW-1:0]	~FieldEnValBit

Table 885. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_field_en.sv — 포트 (5개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
wvalid_i	input	logic
wdata_i	input	logic [FieldW-1:0]
enable_o	output	logic

API — entropy_src_main_sm.sv

Table 886. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_main_sm.sv — module / interface / package (1개)

이름	종류
entropy_src_main_sm	module

Table 887. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_main_sm.sv — 포트 (24개)

포트	방향	형/폭
clk_i	input	logic

포트	방향	형/폭
rst_ni	input	logic
enable_i	input	logic
fw_ov_ent_insert_i	input	logic
fw_ov_sha3_start_i	input	logic
ht_done_pulse_i	input	logic
pd_cntr_zero_i	input	logic
ht_fail_pulse_i	input	logic
alert_thresh_fail_i	input	logic
alert_cntr_clr_ok_o	output	logic
bypass_mode_i	input	logic
bypass_stage_rdy_i	input	logic
sha3_state_vld_i	input	logic
main_stage_push_o	output	logic
bypass_stage_pop_o	output	logic
boot_phase_done_o	output	logic
sha3_start_o	output	logic
sha3_process_o	output	logic
sha3_done_o	output	prim_mubi_pkg::mubi4_t
local_escalate_i	input	logic
main_sm_alert_o	output	logic
main_sm_idle_o	output	logic
main_sm_state_o	output	logic [StateWidth-1:0]
main_sm_err_o	output	logic

API — entropy_src_main_sm_pkg.sv

Table 888. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_main_sm_pkg.sv — module / interface / package (1개)

이름	종류
entropy_src_main_sm_pkg	package

Table 889. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_main_sm_pkg.sv — localparam (1개)

상수	형/폭	값
StateWidth	int	9

API — entropy_src_markov_ht.sv

Table 890. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_markov_ht.sv — module / interface / package (1개)

이름	종류
entropy_src_markov_ht	module

Table 891. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_markov_ht.sv — parameter (3개)

파라미터	형/폭	기본값
RegWidth	int	16
RngBusWidth	int	4
RngBusBitSelWidth	int	2

Table 892. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_markov_ht.sv — 포트 (17개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
entropy_bit_i	input	logic [RngBusWidth-1:0]
entropy_bit_vld_i	input	logic
rng_bit_en_i	input	logic
rng_bit_sel_i	input	logic [RngBusBitSelWidth-1:0]
clear_i	input	logic
active_i	input	logic
thresh_hi_i	input	logic [RegWidth-1:0]
thresh_lo_i	input	logic [RegWidth-1:0]
window_wrap_pulse_i	input	logic
threshold_scope_i	input	logic
test_cnt_hi_o	output	logic [RegWidth-1:0]
test_cnt_lo_o	output	logic [RegWidth-1:0]
test_fail_hi_pulse_o	output	logic
test_fail_lo_pulse_o	output	logic
count_err_o	output	logic

API — entropy_src_pkg.sv**Table 893. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_pkg.sv — module / interface / package (1개)**

이름	종류
entropy_src_pkg	package

Table 894. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_pkg.sv — parameter (9개)

파라미터	형/폭	기본값
CSRNG_BUS_WIDTH	int	384

파라미터	형/폭	기본값
FIPS_BUS_WIDTH	int	1
FIPS_CSRNG_BUS_WIDTH	int	FIPS_BUS_WIDTH + CSRNG_BUS_WIDTH
BucketHtDataMaxWidth	int	4
ENTROPY_SRC_HW_IF_REQ_DEFAULT	entropy_src_hw_if_req_t	{default: '0}
ENTROPY_SRC_HW_IF_RSP_DEFAULT	entropy_src_hw_if_rsp_t	{default: '0}
ENTROPY_SRC_XHT_META_REQ_DEFAULT	entropy_src_xht_meta_req_t	{default: '0}
ENTROPY_SRC_XHT_META_RSP_DEFAULT	entropy_src_xht_meta_rsp_t	{test_cnt_lo: 16'hffff}
HT_WATERMARK_NUM_WIDTH	int	4

1. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src.sv (481 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: entropy_src top level wrapper file
6
7 `include "prim_assert.sv"
8
9
10 module entropy_src
11 import entropy_src_pkg::*;
12 import entropy_src_reg_pkg::*;
13 import prim_mubi_pkg::mubi8_t;
14 #(
15     parameter logic [NumAlerts-1:0] AlertAsyncOn    = {Num-
mAlerts{1'b1}},
16     // Number of cycles a differential skew is tolerated on the
alert signal
17     parameter int unsigned AlertSkewCycles          = 1,
18     parameter int RngBusWidth                      = 4,
19     parameter int RngBusBitSelWidth                = 2,
20     parameter int HealthTestWindowWidth            = 18,
21     parameter int EsFifoDepth                      = 3,
22     parameter int DistrFifoDepth                   = 3,
23     parameter bit Stub                             = 1'b0
24 ) (
25     input logic clk_i,
26     input logic rst_ni,
27
28     // Bus Interface
29     input  tlul_pkg::tl_h2d_t tl_i,
30     output tlul_pkg::tl_d2h_t tl_o,
31
32     // OTP Interface
33     // SEC_CM: INTERSIG.MUBI
34     input  mubi8_t otp_en_entropy_src_fw_read_i,
35     // SEC_CM: INTERSIG.MUBI
36     input  mubi8_t otp_en_entropy_src_fw_over_i,
37
38     // RNG Interface
39     output logic rng_fips_o,
40
41     // Entropy Interface
42     input  entropy_src_hw_if_req_t entropy_src_hw_if_i,
43     output entropy_src_hw_if_rsp_t entropy_src_hw_if_o,
44
45     // RNG Interface
46     output logic          entropy_src_rng_enable_o,
47     input  logic          entropy_src_rng_valid_i,
48     input  logic [RngBusWidth-1:0] entropy_src_rng_bits_i,
49
50     // External Health Test Interface
51     output logic          entropy_src_xht_valid_o,
52     output logic [RngBusWidth-1:0] entropy_src_xht_bits_o,
53     output logic [RngBusBitSelWidth-1:0] entropy_src_xht_bit_sel_o,
54     output logic [HealthTestWindowWidth-1:0] entropy_src_xht_health_test_window_o,
55     output entropy_src_xht_meta_req_t entropy_src_xht_meta_o,
56     input  entropy_src_xht_meta_rsp_t entropy_src_xht_meta_i,
57
58     // Alerts
59     input  prim_alert_pkg::alert_rx_t [NumAlerts-1:0]

```

```

alert_rx_i,
60     output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
alert_tx_o,
61
62     // Interrupts
63     output logic intr_es_entropy_valid_o,
64     output logic intr_es_health_test_failed_o,
65     output logic intr_es_observe_fifo_ready_o,
66     output logic intr_es_fatal_err_o
67 );
68
69 // For a data width of N, there are 2^N buckets in the buck-
et health test, each with its own
70 // counter. To make this health test scale reasonably well
with `RngBusWidth`, we limit the bucket
71 // health test to 4 bit. If `RngBusWidth` is larger than 4,
the bucket health test gets
72 // instantiated multiple times, once per 4 bit.
73 localparam int BucketHtDataWidth = buck-
et_ht_data_width(RngBusWidth);
74 localparam int NumBuckets = 2**BucketHtDataWidth; // bucket
health test bin count
75 localparam int unsigned NumBucketHtInst = num_buck-
et_ht_inst(RngBusWidth);
76
77 // Register interface supports up to 256-bit wide noise
source bus widths. Ensure the that the
78 // parameter does not exceed that limit.
79 `ASSERT_INIT(RngBusBitWidthMaxValue_A, RngBusWidth <= 256)
80 // The IP needs to define both RngBusWidth, RngBusBitSel-
width, and HealthTestWindowWidth, although
81 // the last two are derived from the first. This is needed
to allow topgen to pick up the widths
82 // for interfaces IP-level interfaces based on these para-
meters.
83 `ASSERT_INIT(RngBusBitSelWidthSameAsComputed_A,
RngBusBitSelWidth ==
84 prim_util_pkg::vbits(RngBusWidth))
85 `ASSERT_INIT(HealthTestWindowWidthSameAsComputed_A,
HealthTestWindowWidth == 16 +
86 prim_util_pkg::vbits(RngBusWidth))
87
88 // common signals
89 entropy_src_hw2reg_t hw2reg;
90 entropy_src_reg2hw_t reg2hw;
91 logic [NumAlerts-1:0] alert_test;
92 logic [NumAlerts-1:0] alert;
93
94 // core signals
95 logic core_rst_n;
96 entropy_src_hw2reg_t core_hw2reg;
97 entropy_src_hw_if_rsp_t core_entropy_hw_if;
98 logic core_rng_enable;
99 entropy_src_xht_meta_req_t core_xht_meta;
100 logic core_xht_valid;
101 logic [RngBusWidth-1:0] core_xht_bits;
102 logic [RngBusBitSelWidth-1:0] core_xht_bit_sel;
103 logic [HealthTestWindowWidth-1:0]
core_xht_health_test_window;
104 logic core_intr_es_entropy_valid;
105 logic core_intr_es_health_test_failed;
106 logic core_intr_es_observe_fifo_ready;
107 logic core_intr_es_fatal_err;
108 logic [NumAlerts-1:0] core_alert_test;
109 logic [NumAlerts-1:0] core_alert;
110
111 //stub signals
112 localparam int StubLfsrWidth = 64;
113 localparam int Copies = CSRNG_BUS_WIDTH / StubLfsrWidth;
114 entropy_src_hw2reg_t stub_hw2reg;
115 entropy_src_hw_if_rsp_t stub_entropy_hw_if;
116 logic stub_es_valid;
117 logic [NumAlerts-1:0] stub_alert_test;
118 logic [NumAlerts-1:0] stub_alert;

```

```

119 logic [StubLfswidth-1:0] stub_lfsr_value;
120
121 //////////////////////////////////////////////////
122 // Selecting between core and stub
123 //////////////////////////////////////////////////
124
125 assign hw2reg = Stub ? stub_h-
w2reg : core_hw2reg;
126 assign core_rst_n = Stub ?
'0 : rst_ni;
127 assign entropy_src_hw_if_o = Stub ?
stub_entropy_hw_if : core_entropy_hw_if;
128 assign entropy_src_rng_enable_o = Stub ?
'1 : core_rng_enable;
129 assign entropy_src_xht_valid_o = Stub ?
'0 : core_xht_valid;
130 assign entropy_src_xht_bits_o = Stub ?
'0 : core_xht_bits;
131 assign entropy_src_xht_bit_sel_o = Stub ?
'0 : core_xht_bit_sel;
132 assign entropy_src_xht_health_test_window_o = Stub ?
'0 :
133 core_xht_health_test_window;
134 assign entropy_src_xht_meta_o = Stub ?
'0 : core_xht_meta;
135 assign intr_es_entropy_valid_o = Stub ?
stub_es_valid :
136 core_intr_es_entropy_valid;
137 assign intr_es_health_test_failed_o = Stub ?
'0 :
138 core_intr_es_health_test_failed;
139 assign intr_es_observe_fifo_ready_o = Stub ?
'0 :
140 core_intr_es_observe_fifo_ready;
141 assign intr_es_fatal_err_o = Stub ?
'0 : core_intr_es_fatal_err;
142 assign alert_test = Stub ?
stub_alert_test : core_alert_test;
143 assign alert = Stub ?
stub_alert : core_alert;
144
145 //////////////////////////////////////////////////
146 // core entropy operation
147 //////////////////////////////////////////////////
148
149 logic [NumAlerts-1:0] intg_err_alert;
150 assign intg_err_alert[0] = 1'b0;
151
152 // SEC_CM: CONFIG.REGWEN
153 // SEC_CM: TILE_LINK.BUS.INTEGRITY
154
155 entropy_src_reg_top u_reg (
156 .clk_i,
157 .rst_ni,
158 .tl_i,
159 .tl_o,
160 .reg2hw,
161 .hw2reg(hw2reg),
162 .intg_err_o(intg_err_alert[1]) // Assign this alert to the
fatal alert index.
163 );
164
165 entropy_src_core #(
166 .RngBusWidth(RngBusWidth),
167 .RngBusBitSelWidth(RngBusBitSelWidth),
168 .HealthTestWindowWidth(HealthTestWindowWidth),
169 .EsFifoDepth(EsFifoDepth),
170 .DistrFifoDepth(DistrFifoDepth),
171 .BucketHtDataWidth(BucketHtDataWidth),
172 .NumBucketHtInst(NumBucketHtInst)
173 ) u_entropy_src_core (
174 .clk_i,
175 .rst_ni(core_rst_n),
176 .reg2hw,
177 .hw2reg(core_hw2reg),
178
179 .otp_en-en-
tropy_src_fw_read_i(otp_en_entropy_src_fw_read_i),
180 .otp_en-en-
tropy_src_fw_over_i(otp_en_entropy_src_fw_over_i),
181 .rng_fips_o,
182
183 .entropy_src_hw_if_o(core_entropy_hw_if),
184 .entropy_src_hw_if_i,
185
186 .entropy_src_xht_valid_o(core_xht_valid),
187 .entropy_src_xht_bits_o(core_xht_bits),
188 .entropy_src_xht_bit_sel_o(core_xht_bit_sel),
189 .entropy_src_xht_meta_o(core_xht_meta),
190 .en-
tropy_src_xht_health_test_window_o(core_xht_health_test_window),
191 .entropy_src_xht_meta_i,
192
193 .entropy_src_rng_enable_o(core_rng_enable),
194 .entropy_src_rng_valid_i,
195 .entropy_src_rng_bits_i,

```

```

196
197 .recov_alert_o(core_alert[0]),
198 .fatal_alert_o(core_alert[1]),
199
200 .recov_alert_test_o(core_alert_test[0]),
201 .fatal_alert_test_o(core_alert_test[1]),
202
203 .intr_es_entropy_valid_o(core_intr_es_entropy_valid),
204 .intr_es_health_test_failed_o(core_intr_es_health_test_fa
iled),
205 .intr_es_ob-
serve_fifo_ready_o(core_intr_es_observe_fifo_ready),
206 .intr_es_fatal_err_o(core_intr_es_fatal_err)
207 );
208
209 //////////////////////////////////////////////////
210 // stub entropy operation
211 //////////////////////////////////////////////////
212
213 assign stub_alert = '0;
214 assign stub_alert_test = '0;
215 assign stub_entropy_hw_if = '{
216   es_ack: '1,
217   es_bits: {Copies{stub_lfsr_value}},
218   es_fips: '1
219 };
220 // once enabled, stub entropy is always available
221
222 import prim_mubi_pkg::mubi4_t;
223 import prim_mubi_pkg::mubi4_test_true_strict;
224
225 mubi4_t mubi_module_en;
226 assign mubi_module_en = mubi4_t'(reg2hw.module_enable.q);
227 assign stub_es_valid = mu-
bi4_test_true_strict(mubi_module_en);
228
229 if (Stub) begin : gen_stub_entropy_src
230   prim_lfsr #(
231     .Lfswidth(StubLfswidth),
232     .StateOutDw(StubLfswidth)
233   ) u_prim_lfsr (
234     .clk_i (clk_i),
235     .rst_ni (rst_ni),
236     .seed_en_i ('0),
237     .seed_i ('0),
238     .lfsr_en_i (stub_es_valid),
239     .entropy_i ('0),
240     .state_o (stub_lfsr_value)
241   );
242
243 // hardware hw2reg inputs
244 always_comb begin
245   stub_hw2reg = '0;
246
247   // as long as enable is 1, do not allow registers to be
written
248   stub_hw2reg.fw_ov_rd_data.d = stub_lfsr_value[31:0];
249   stub_hw2reg.entropy_data.d = stub_lfsr_value[31:0];
250   stub_hw2reg.debug_status.main_sm_idle.d = 1'b1;
251   stub_hw2reg.main_sm_state.d = en-
tropy_src_main_sm_pkg::Idle;
252
253   stub_hw2reg.intr_state.es_entropy_valid.de =
stub_es_valid;
254   stub_hw2reg.intr_state.es_entropy_valid.d = 1'b1;
255
256 end
257 end else begin : gen_stub_tieoff
258   assign stub_hw2reg = '0;
259   assign stub_lfsr_value = '0;
260 end
261
262 //////////////////////////////////////////////////
263 // Alert generation
264 //////////////////////////////////////////////////
265 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
266   prim_alert_sender #(
267     .AsyncOn(AlertAsyncOn[i]),
268     .SkewCycles(AlertSkewCycles),
269     .IsFatal(i)
270   ) u_prim_alert_sender (
271     .clk_i,
272     .rst_ni,
273     .alert_test_i ( alert_test[i] ),
274     .alert_req_i ( alert[i] || intg_err_alert[i] ),
275     .alert_ack_o ( ),
276     .alert_state_o ( ),
277     .alert_rx_i ( alert_rx_i[i] ),
278     .alert_tx_o ( alert_tx_o[i] )
279   );
280 end
281
282 // Outputs should have a known value after reset
283 `ASSERT_KNOWN(TlDvalKnown0_A, tl_o.d_valid)
284 `ASSERT_KNOWN(TlAReadyKnown0_A, tl_o.a_ready)
285
286 // Entropy Interface
287 `ASSERT_KNOWN(EsHwIfEsAckKnown0_A, en-
tropy_src_hw_if_o.es_ack)
288 `ASSERT_KNOWN_IF(EsHwIfEsBitsKnown0_A, en-
tropy_src_hw_if_o.es_bits,

```



```

289     entropy_src_hw_if_o.es_ack)
290 `ASSERT_KNOWN_IF(EsHwIfEsFipsKnown0_A, en-
entropy_src_hw_if_o.es_fips,
291     entropy_src_hw_if_o.es_ack)
292
293 // RNG Interface
294 `ASSERT_KNOWN(EsRngEnableKnown0_A, entropy_src_rng_enable_o)
295
296 // External Health Test Interface
297 `ASSERT_KNOWN_IF(EsXhtEntropyBitKnown0_A, en-
entropy_src_xht_bits_o, entropy_src_xht_valid_o)
298 `ASSERT_KNOWN(EsXhtEntropyBitValidKnown0_A, en-
entropy_src_xht_valid_o)
299 `ASSERT_KNOWN(EsXhtClearKnown0_A, en-
entropy_src_xht_meta_o.clear)
300 `ASSERT_KNOWN(EsXhtActiveKnown0_A, en-
entropy_src_xht_meta_o.active)
301 `ASSERT_KNOWN(EsXhtThreshHiKnown0_A, en-
entropy_src_xht_meta_o.thresh_hi)
302 `ASSERT_KNOWN(EsXhtThreshLoKnown0_A, en-
entropy_src_xht_meta_o.thresh_lo)
303 `ASSERT_KNOWN(EsXhtWindowKnown0_A, en-
entropy_src_xht_meta_o.window_wrap_pulse)
304
305 // Alerts
306 `ASSERT_KNOWN(AlertTxKnown0_A, alert_tx_o)
307
308 // Interrupts
309 `ASSERT_KNOWN(IntrEsEntropyValidKnown0_A, intr_es_en-
entropy_valid_o)
310 `ASSERT_KNOWN(IntrEsHealthTestFailedKnown0_A,
intr_es_health_test_failed_o)
311 `ASSERT_KNOWN(IntrEsFifoErrKnown0_A, intr_es_fatal_err_o)
312
313 // prim_count alerts
314 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck0_A,
u_entropy_src_core.u_prim_count_window_cntr,
315     alert_tx_o[1])
316
317 for (genvar sh = 0; sh < RngBusWidth; sh = sh+1) begin :
gen_bit_cntrs
318     `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck1_A,
u_en-
319     entropy_src_core.u_entropy_src_adaptpt_ht.gen_cntrs[sh].u_prim_count_t
est_cnt,
320     alert_tx_o[1])
321 end : gen_bit_cntrs
322
323 for (genvar j = 0; j < NumBucketHtInst; j = j + 1) begin :
gen_symbol_match_inst_loop
324     for (genvar i = 0; i < NumBuckets; i = i + 1) begin :
gen_symbol_match
325         `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck_A,
u_en-
326         entropy_src_core.gen_health_test[j].u_entropy_src_bucket_ht.gen_symbo
l_match[i].
327         u_prim_count_bin_cntr, alert_tx_o[1])
328     end : gen_symbol_match
329 end : gen_symbol_match_inst_loop
330
331 for (genvar sh = 0; sh < RngBusWidth; sh = sh+1) begin :
gen_pair_cntrs
332     `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck_A,
u_en-
333     entropy_src_core.u_entropy_src_markov_ht.gen_cntrs[sh].u_prim_count_p
air_cntr,
334     alert_tx_o[1])
335 end : gen_pair_cntrs
336
337 for (genvar sh = 0; sh < RngBusWidth; sh = sh+1) begin :
gen_rep_cntrs
338     `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck_A,
u_en-
339     entropy_src_core.u_entropy_src_repnt_ht.gen_cntrs[sh].u_prim_count_r
ep_cntr,
340     alert_tx_o[1])
341 end : gen_rep_cntrs
342
343 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck7_A,
u_en-
344     entropy_src_core.u_entropy_src_repcnts_ht.u_prim_count_rep_cntr,
345     alert_tx_o[1])
346
347 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlMainFsmCheck_A,
u_entropy_src_core.u_entropy_src_main_sm.u_state_regs,
348     alert_tx_o[1])
349
350 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlAckFsmCheck_A,
u_entropy_src_core.u_entropy_src_ack_sm.u_state_regs,
351     alert_tx_o[1])
352
353 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(PipelineDepthCheck_A,
u_entropy_src_core.u_prim_count_pipeline_depth,
354     alert_tx_o[1])
355
356 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(SHA3FsmCheck_A,
u_entropy_src_core.u_sha3.u_state_regs, alert_tx_o[1])
357
358 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(KeccakRoundFsmCheck_A,
u_entropy_src_core.u_sha3.u_keccak.u_state_regs,
359     alert_tx_o[1])

```

```

360 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(SHA3padFsmCheck_A,
u_entropy_src_core.u_sha3.u_pad.u_state_regs,
361     alert_tx_o[1])
362
363 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(SentMsgCountCheck_A,
u_entropy_src_core.u_sha3.u_pad.u_sentmsg_count,
364     alert_tx_o[1])
365
366 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(RoundCountCheck_A,
u_entropy_src_core.u_sha3.u_keccak.u_round_count,
367     alert_tx_o[1])
368
369 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck8_A,
u_en-
370     entropy_src_core.u_entropy_src_cntr_reg_repnt.u_prim_count_cntr_reg,
371     alert_tx_o[1])
372
373 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck9_A,
u_en-
374     entropy_src_core.u_entropy_src_cntr_reg_repcnts.u_prim_count_cntr_reg,
375     alert_tx_o[1])
376
377 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck10_A,
u_en-
378     entropy_src_core.u_entropy_src_cntr_reg_adaptpt_hi.u_prim_count_cntr_r
eg,
379     alert_tx_o[1])
380
381 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck11_A,
u_en-
382     entropy_src_core.u_entropy_src_cntr_reg_adaptpt_lo.u_prim_count_cntr_r
eg,
383     alert_tx_o[1])
384
385 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck12_A,
u_en-
386     entropy_src_core.u_entropy_src_cntr_reg_bucket.u_prim_count_cntr_reg,
387     alert_tx_o[1])
388
389 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck13_A,
u_en-
390     entropy_src_core.u_entropy_src_cntr_reg_markov_hi.u_prim_count_cntr_r
eg,
391     alert_tx_o[1])
392
393 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck14_A,
u_en-
394     entropy_src_core.u_entropy_src_cntr_reg_markov_lo.u_prim_count_cntr_r
eg,
395     alert_tx_o[1])
396
397 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck15_A,
u_en-
398     entropy_src_core.u_entropy_src_cntr_reg_extht_hi.u_prim_count_cntr_reg
,
399     alert_tx_o[1])
400
401 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck16_A,
u_en-
402     entropy_src_core.u_entropy_src_cntr_reg_extht_lo.u_prim_count_cntr_reg
,
403     alert_tx_o[1])
404
405 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck17_A,
u_en-
406     entropy_src_core.u_entropy_src_cntr_reg_any_alert_fails.u_prim_count_
cntr_reg,
407     alert_tx_o[1])
408
409 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck18_A,
u_en-
410     entropy_src_core.u_entropy_src_cntr_reg_repnt_alert_fails.u_prim_cou
nt_cntr_reg,
411     alert_tx_o[1])
412
413 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck19_A,
u_en-
414     entropy_src_core.u_entropy_src_cntr_reg_repcnts_alert_fails.u_prim_co
unt_cntr_reg,
415     alert_tx_o[1])
416
417 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck20_A,
u_en-
418     entropy_src_core.u_entropy_src_cntr_reg_adaptpt_hi_alert_fails.u_prim_
count_cntr_reg,
419     alert_tx_o[1])
420
421 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck21_A,
u_en-
422     entropy_src_core.u_entropy_src_cntr_reg_adaptpt_lo_alert_fails.u_prim_
count_cntr_reg,
423     alert_tx_o[1])
424
425 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck22_A,
u_en-
426     entropy_src_core.u_entropy_src_cntr_reg_bucket_alert_fails.u_prim_cou
nt_cntr_reg,
427     alert_tx_o[1])
428
429 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck23_A,

```

```

435     u_en-
trophy_src_core.u_entropy_src_cntr_reg_markov_hi_alert_fails.u_prim
count_cntr_reg,
436     alert_tx_o[1])
437
438     `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck24_A,
439     u_en-
trophy_src_core.u_entropy_src_cntr_reg_markov_lo_alert_fails.u_prim
count_cntr_reg,
440     alert_tx_o[1])
441
442     `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck25_A,
443     u_en-
trophy_src_core.u_entropy_src_cntr_reg_extht_hi_alert_fails.u_prim_c
ount_cntr_reg,
444     alert_tx_o[1])
445
446     `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck26_A,
447     u_en-
trophy_src_core.u_entropy_src_cntr_reg_extht_lo_alert_fails.u_prim_c
ount_cntr_reg,
448     alert_tx_o[1])
449
450     `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(EsrngFifoWptrCheck_A,
451     u_en-
trophy_src_core.u_prim_fifo_sync_esrng.gen_normal_fifo.u_fifo_cnt.ge
n_secure_ptrs.u_wpctr,
452     alert_tx_o[1])
453     `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(EsrngFifoRptrCheck_A,
454     u_en-
trophy_src_core.u_prim_fifo_sync_esrng.gen_normal_fifo.u_fifo_cnt.ge
n_secure_ptrs.u_rpctr,
455     alert_tx_o[1])
456
457     `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(DistrFifoWptrCheck_A,
458     u_en-
trophy_src_core.u_prim_fifo_sync_distr.gen_normal_fifo.u_fifo_cnt.ge
n_secure_ptrs.u_wpctr,
459     alert_tx_o[1])
460     `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(DistrFifoRptrCheck_A,
461     u_en-
trophy_src_core.u_prim_fifo_sync_distr.gen_normal_fifo.u_fifo_cnt.ge
n_secure_ptrs.u_rpctr,
462     alert_tx_o[1])
463
464     `AS-
SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(ObserveFifoWptrCheck_A,
465     u_en-
trophy_src_core.u_prim_fifo_sync_observe.gen_normal_fifo.u_fifo_cnt.
gen_secure_ptrs.u_wpctr,
466     alert_tx_o[1])
467     `AS-
SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(ObserveFifoRptrCheck_A,
468     u_en-
trophy_src_core.u_prim_fifo_sync_observe.gen_normal_fifo.u_fifo_cnt.
gen_secure_ptrs.u_rpctr,
469     alert_tx_o[1])
470
471     `AS-
SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(EsfinalFifoWptrCheck_A,
472     u_en-
trophy_src_core.u_prim_fifo_sync_esfinal.gen_normal_fifo.u_fifo_cnt.
gen_secure_ptrs.u_wpctr,
473     alert_tx_o[1])
474     `AS-
SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(EsfinalFifoRptrCheck_A,
475     u_en-
trophy_src_core.u_prim_fifo_sync_esfinal.gen_normal_fifo.u_fifo_cnt.
gen_secure_ptrs.u_rpctr,
476     alert_tx_o[1])
477
478     // Alert assertions for reg_we onehot check
479     `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
u_reg, alert_tx_o[1])
480
481 endmodule

```

2. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_ack_sm.sv (66 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: interface between a req/ack interface and a
fifo
6 //
7
8 `include "prim_assert.sv"
9
10 module entropy_src_ack_sm (
11     input logic      clk_i,
12     input logic      rst_ni,
13
14     input logic      enable_i,
15     input logic      req_i,
16     output logic     ack_o,

```

```

17     input logic      fifo_not_empty_i,
18     input logic      local_escalate_i,
19     output logic     fifo_pop_o,
20     output logic     ack_sm_err_o
21 );
22
23 import entropy_src_ack_sm_pkg::*;
24
25 state_e state_d, state_q;
26
27 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
state_e, Idle)
28
29 always_comb begin
30     state_d = state_q;
31     ack_o = 1'b0;
32     fifo_pop_o = 1'b0;
33     ack_sm_err_o = 1'b0;
34     unique case (state_q)
35         Idle: begin
36             if (enable_i) begin
37                 if (req_i) begin
38                     state_d = Wait;
39                 end
40             end
41         end
42         Wait: begin
43             if (!enable_i) begin
44                 state_d = Idle;
45             end else begin
46                 if (fifo_not_empty_i) begin
47                     ack_o = 1'b1;
48                     fifo_pop_o = 1'b1;
49                     state_d = Idle;
50                 end
51             end
52         end
53         Error: begin
54             ack_sm_err_o = 1'b1;
55         end
56         default: begin
57             state_d = Error;
58             ack_sm_err_o = 1'b1;
59         end
60     endcase
61     if (local_escalate_i) begin
62         state_d = Error;
63     end
64 end
65
66 endmodule

```

3. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_ack_sm_pkg.sv (35 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // State definitions for entropy_src_ack_sm, provided as a
separate package for use in DV
6
7 package entropy_src_ack_sm_pkg;
8
9 // Encoding generated with:
10 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 3 -n 6 \
11 // -s 1236774883 --language=sv
12 //
13 // Hamming distance histogram:
14 //
15 // 0: --
16 // 1: --
17 // 2: --
18 // 3: ||| (33.33%)
19 // 4: ||| (33.33%)
20 // 5: ||| (33.33%)
21 // 6: --
22 //
23 // Minimum Hamming distance: 3
24 // Maximum Hamming distance: 5
25 // Minimum Hamming weight: 1
26 // Maximum Hamming weight: 4
27 //
28 localparam int StateWidth = 6;
29 typedef enum logic [StateWidth-1:0] {
30     Idle = 6'b011101, // idle
31     Wait = 6'b101100, // wait until the fifo has an entry
32     Error = 6'b000010 // illegal state reached and hang
33 } state_e;
34
35 endpackage

```

4. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_adaptp_ht.sv (138 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

```

```

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: entropy_src Adaptive Proportion Test module
6 //
7
8 module entropy_src_adaptp_ht #(
9     parameter int RegWidth      = 16,
10    parameter int RngBusWidth    = 4,
11    parameter int RngBusBitSelWidth = 2
12 ) (
13     input logic clk_i,
14     input logic rst_ni,
15
16     // ins req interface
17     input logic [RngBusWidth-1:0] entropy_bit_i,
18     input logic entropy_bit_vld_i,
19     input logic rng_bit_en_i,
20     input logic [RngBusBitSelWidth-1:0] rng_bit_sel_i,
21     input logic clear_i,
22     input logic active_i,
23     input logic [RegWidth-1:0] thresh_hi_i,
24     input logic [RegWidth-1:0] thresh_lo_i,
25     input logic window_wrap_pulse_i,
26     input logic threshold_scope_i,
27     output logic [RegWidth-1:0] test_cnt_hi_o,
28     output logic [RegWidth-1:0] test_cnt_lo_o,
29     output logic test_fail_hi_pulse_o,
30     output logic test_fail_lo_pulse_o,
31     output logic count_err_o
32 );
33
34 // signals
35 logic [RegWidth-1:0] test_cnt_max;
36 logic [RegWidth-1:0] test_cnt_min;
37 test_cnt_min_tmp;
38 logic [RegWidth-1:0] test_cnt_sum;
39 logic [RngBusWidth-1:0][RegWidth-1:0] test_cnt;
40 logic [RngBusWidth-1:0] rng_bit_cnt;
41 logic [RngBusWidth-1:0] test_cnt_err;
42
43 // Adaptive Proportion Test
44 //
45 // Test operation
46 // This is an approved modification of the NIST Adaptive
  Proportion test in that
47 // instead of counting the first sampled value (1'b1 or
  1'b0), it will count
48 // only the 1's on all four bit streams and accumulate for
  the during of the
49 // window size (W) of the test.
50 for (genvar sh = 0; sh < RngBusWidth; sh = sh+1) begin :
  gen_cntrs
51
52     // cumulative ones counter
53     // SEC_CM: CTR.REDUN
54     prim_count #(
55         .Width(RegWidth)
56     ) u_prim_count_test_cnt (
57         .clk_i,
58         .rst_ni,
59         .clr_i(window_wrap_pulse_i),
60         .set_i(!active_i || clear_i),
61         .set_cnt_i(RegWidth'(0)),
62         .incr_en_i(entropy_bit_vld_i),
63         .decr_en_i(1'b0),
64         .step_i(RegWidth'(entropy_bit_i[sh])),
65         .commit_i(1'b1),
66         .cnt_o(test_cnt[sh]),
67         .cnt_after_commit_o(),
68         .err_o(test_cnt_err[sh])
69     );
70 end : gen_cntrs
71
72 // determine the highest counter counter value
73 prim_max_tree #(
74     .NumSrc(RngBusWidth),
75     .Width(RegWidth)
76 ) u_max (
77     .clk_i      (clk_i),
78     .rst_ni     (rst_ni),
79     .values_i   (test_cnt),
80     .valid_i    ({RngBusWidth{1'b1}}),
81     .max_value_o (test_cnt_max),
82     .max_idx_o  (),
83     .max_valid_o ()
84 );
85
86 // determine the lowest counter value
87 // Negate the inputs and outputs of prim_max_tree to find
  the minimum
88 // For this unsigned application, one's complement negation
  (i.e. logical inversion) is fine.
89 prim_max_tree #(
90     .NumSrc(RngBusWidth),
91     .Width(RegWidth)
92 ) u_min (
93     .clk_i      (clk_i),
94     .rst_ni     (rst_ni),

```

```

95     .values_i   (~test_cnt),
96     .valid_i    ({RngBusWidth{1'b1}}),
97     .max_value_o (test_cnt_min_tmp),
98     .max_idx_o  (),
99     .max_valid_o ()
100 );
101
102 assign test_cnt_min = ~test_cnt_min_tmp;
103
104 prim_sum_tree #(
105     .NumSrc(RngBusWidth),
106     .InWidth(RegWidth)
107 ) u_sum (
108     .clk_i      (clk_i),
109     .rst_ni     (rst_ni),
110     .values_i   (test_cnt),
111     .valid_i    ({RngBusWidth{1'b1}}),
112     .sum_value_o (test_cnt_sum),
113     .sum_valid_o ()
114 );
115
116 always_comb begin
117     rng_bit_cnt = '0;
118
119     for (int i = 0; i < RngBusWidth; i++) begin
120         if (rng_bit_sel_i == i) begin
121             rng_bit_cnt = test_cnt[i];
122         end
123     end
124 end
125
126 assign test_cnt_hi_o = rng_bit_en_i ? rng_bit_cnt :
  threshold_scope_i ? test_cnt_sum :
  test_cnt_max;
127
128 assign test_cnt_lo_o = rng_bit_en_i ? rng_bit_cnt :
  threshold_scope_i ? test_cnt_sum :
  test_cnt_min;
129
130 // the pulses will be only one clock in length
131 assign test_fail_hi_pulse_o = active_i && win-
  dow_wrap_pulse_i && (test_cnt_hi_o > thresh_hi_i);
132 assign test_fail_lo_pulse_o = active_i && win-
  dow_wrap_pulse_i && (test_cnt_lo_o < thresh_lo_i);
133 assign count_err_o = |test_cnt_err;
134
135 endmodule

```

5. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_bucket_ht.sv (90 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: entropy_src bucket health test module
6 //
7
8 module entropy_src_bucket_ht #(
9     parameter int RegWidth = 16,
10    parameter int RngBusWidth = 4
11 ) (
12     input logic clk_i,
13     input logic rst_ni,
14
15     // ins req interface
16     input logic [RngBusWidth-1:0] entropy_bit_i,
17     input logic entropy_bit_vld_i,
18     input logic clear_i,
19     input logic active_i,
20     input logic [RegWidth-1:0] thresh_i,
21     input logic window_wrap_pulse_i,
22     output logic [RegWidth-1:0] test_cnt_o,
23     output logic test_fail_pulse_o,
24     output logic count_err_o
25 );
26
27 localparam int NUM_BINS = 2**RngBusWidth;
28
29 // signals
30 logic [NUM_BINS-1:0] bin_incr;
31 logic [NUM_BINS-1:0] bin_cnt_exceeds_thresh;
32 logic [NUM_BINS - 1:0][RegWidth - 1:0] bin_cntr;
33 logic [NUM_BINS-1:0] bin_cntr_err;
34 logic [RegWidth-1:0] bin_max;
35
36 // Bucket Test
37 //
38 // Test operation
39 // This test will look at 4 bit symbols and increment one
  of sixteen
40 // counters, or buckets, to show a histogram of the data
  stream.
41 // An error will occur if one of the counters reaches the
  thresh
42 // value.
43
44 // Analyze the incoming symbols

```

```

46
47 for (genvar i = 0; i < NUM_BINS; i = i + 1) begin : gen_sym-
bol_match
48 // set the bin incremter if the symbol matches that bin
49 assign bin_incr[i] = entropy_bit_vld_i && (entropy_bit_i
== i);
50 // use the bin incremter to increase the bin total count
51 // SEC_CM: CTR.REDUN
52 prim_count #(
53   .width(RegWidth)
54 ) u_prim_count_bin_cntr (
55   .clk_i,
56   .rst_ni,
57   .clr_i(window_wrap_pulse_i),
58   .set_i(!active_i || clear_i),
59   .set_cnt_i(RegWidth'(0)),
60   .incr_en_i(bin_incr[i]),
61   .decr_en_i(1'b0),
62   .step_i(RegWidth'(1)),
63   .commit_i(1'b1),
64   .cnt_o(bin_cntr[i]),
65   .cnt_after_commit_o(),
66   .err_o(bin_cntr_err[i])
67 );
68 assign bin_cnt_exceeds_thresh[i] = (bin_cntr[i] >
thresh_i);
69 end : gen_symbol_match
70
71 prim_max_tree #(
72   .NumSrc(NUM_BINS),
73   .width(RegWidth)
74 ) u_prim_max_tree_bin_cntr_max (
75   .clk_i,
76   .rst_ni,
77   .values_i (bin_cntr),
78   .valid_i ({RegWidth{1'b1}}),
79   .max_value_o(bin_max),
80   .max_idx_o (),
81   .max_valid_o()
82 );
83
84 // the pulses will be only one clock in length
85 assign test_fail_pulse_o = active_i && window_wrap_pulse_i
&& (|bin_cnt_exceeds_thresh);
86 assign test_cnt_o = bin_max;
87 assign count_err_o = |bin_cntr_err;
88
89
90 endmodule

```

6. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_cntr_reg.sv (45 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: entropy_src counter register module
6 //
7
8 module entropy_src_cntr_reg #(
9   parameter int RegWidth = 16
10 ) (
11   input logic          clk_i,
12   input logic          rst_ni,
13
14   // functional interface
15   input logic          clear_i,
16   input logic          event_i,
17   input logic [RegWidth-1:0] step_i,
18   output logic [RegWidth-1:0] value_o,
19   output logic          err_o
20 );
21
22 // signals
23 logic [RegWidth-1:0] counter_value;
24
25 // counter will not wrap when full value is reached
26 prim_count #(
27   .width(RegWidth)
28 ) u_prim_count_cntr_reg (
29   .clk_i,
30   .rst_ni,
31   .clr_i(clear_i),
32   .set_i(1'b0),
33   .set_cnt_i(RegWidth'(0)),
34   .incr_en_i(event_i && (~counter_value != '0)),
35   .decr_en_i(1'b0),
36   .step_i,
37   .commit_i(1'b1),
38   .cnt_o(counter_value),
39   .cnt_after_commit_o(),
40   .err_o(err_o)
41 );
42
43 assign value_o = counter_value;

```

```

44
45 endmodule

```

7. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_core.sv (3,313 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: entropy_src core module
6 //
7 `include "prim_assert.sv"
8
9 module entropy_src_core import entropy_src_pkg::*; #(
10   parameter int RngBusWidth      = 4,
11   parameter int RngBusBitSelWidth = 2,
12   parameter int HealthTestWindowWidth = 18,
13   parameter int EsFifoDepth      = 3,
14   parameter int DistrFifoDepth   = 2,
15   parameter int BucketHtDataWidth = 4,
16   parameter int NumBucketHtInst  =
prim_util_pkg::ceil_div(RngBusWidth, BucketHtDataWidth)
17 ) (
18   input logic clk_i,
19   input logic rst_ni,
20
21   input  entropy_src_reg_pkg::entropy_src_reg2hw_t reg2hw,
22   output entropy_src_reg_pkg::entropy_src_hw2reg_t hw2reg,
23
24   // Efuse Interface
25   input prim_mubi_pkg::mubi8_t otp_en_entropy_src_fw_read_i,
26   input prim_mubi_pkg::mubi8_t otp_en_entropy_src_fw_over_i,
27
28   // RNG Interface
29   output logic rng_fips_o,
30
31   // Entropy Interface
32   input  entropy_src_hw_if_req_t entropy_src_hw_if_i,
33   output entropy_src_hw_if_rsp_t entropy_src_hw_if_o,
34
35   // RNG Interface
36   output logic          entropy_src_rng_enable_o,
37   input logic          entropy_src_rng_valid_i,
38   input logic [RngBusWidth-1:0] entropy_src_rng_bits_i,
39
40   // External Health Test Interface
41   output logic          entropy_src_xht_valid_o,
42   output logic [RngBusWidth-1:0] entropy_src_xht_bits_o,
43   output logic [RngBusBitSelWidth-1:0] entropy_src_xht_bit_sel_o,
44   output logic [HealthTestWindowWidth-1:0] entropy_src_xht_health_test_window_o,
45   output entropy_src_xht_meta_req_t entropy_src_xht_meta_o,
46   input  entropy_src_xht_meta_rsp_t entropy_src_xht_meta_i,
47
48   output logic          recov_alert_test_o,
49   output logic          fatal_alert_test_o,
50   output logic          recov_alert_o,
51   output logic          fatal_alert_o,
52
53   output logic          intr_es_entropy_valid_o,
54   output logic          intr_es_health_test_failed_o,
55   output logic          intr_es_observe_fifo_ready_o,
56   output logic          intr_es_fatal_err_o
57 );
58
59 import entropy_src_reg_pkg::*;
60 import prim_mubi_pkg::mubi4_t;
61 import prim_mubi_pkg::mubi4_test_true_strict;
62 import prim_mubi_pkg::mubi4_test_true_loose;
63 import prim_mubi_pkg::mubi4_and_hi;
64 import prim_mubi_pkg::mubi4_test_false_loose;
65 import prim_mubi_pkg::mubi4_test_invalid;
66
67 localparam int EsFifoDepthW =
prim_util_pkg::vbits(EsFifoDepth + 1);
68 localparam int PostHTWidth = 32;
69 localparam int PostHTFifoDepthW =
prim_util_pkg::vbits(PostHTWidth / RngBusWidth + 1);
70 localparam int HalfRegWidth = 16;
71 localparam int FullRegWidth = 32;
72 localparam int EighthRegWidth = 4;
73 localparam int SeedLen = 384;
74 localparam int DistrFifoWidth = 32;
75 localparam int DistrFifoDepthW =
prim_util_pkg::vbits(DistrFifoDepth + 1);
76 localparam int ObserveFifoWidth = 32;
77 localparam int PreCondWidth = 64;
78 localparam int PreCondFifoDepthW =
prim_util_pkg::vbits(PreCondWidth / PostHTWidth + 1);
79 // We can have 1 32-bit word in the post HT FIFO, DistrFi-
foDepth words in the distr FIFO, and
80 // 2 words in the precon FIFO. The final +1 is required for

```

```

when the pipeline is entirely empty.
81   localparam int PipelineDepthW =

82   prim_util_pkg::vbits(1 + DistrFifoDepth + PreCondWidth /
PostHTWidth + 1);
83   localparam int Clog2ObserveFifoDepth = $clog2(
g2(ObserveFifoDepth));
84   localparam int EsEnableCopies = 20;
85   localparam int EsEnPulseCopies = 1;
86   localparam int unsigned NumSwReadsSeed = SeedLen / FullReg-
Width;
87   localparam int unsigned LastSwRead = NumSwReadsSeed - 1;
88   localparam int unsigned SwReadIdxWidth =
prim_util_pkg::vbits(NumSwReadsSeed);
89
90   //-----
91   // SHA3parameters
92   //-----
93   // Do not enable masking
94   localparam bit Sha3EnMasking = 0;
95   // derived parameter
96   localparam int Sha3Share = (Sha3EnMasking) ? 2 : 1;
97
98   // signals
99   logic      fw_ov_mode;
100  logic      fw_ov_mode_pfe;
101  logic      fw_ov_mode_pfa;
102  logic      fw_ov_wr_fifo_full;
103  logic      fw_ov_mode_entropy_insert;
104  logic      fw_ov_entropy_insert_pfe;
105  logic      fw_ov_entropy_insert_pfa;
106  logic      fw_ov_sha3_start_pfe;
107  logic      fw_ov_sha3_start_pfe_q;
108  logic      fw_ov_sha3_start_pfa;
109  logic      fw_ov_sha3_disable_pulse;
110  logic [ObserveFifoWidth-1:0] fw_ov_wr_data;
111  logic      fw_ov_fifo_rd_pulse;
112  logic      fw_ov_fifo_wr_pulse;
113  logic      es_enable_pfa;
114
115  logic      fips_enable_pfe;
116  logic      fips_enable_pfa;
117  logic      fips_flag_pfe;
118  logic      fips_flag_pfa;
119  logic      rng_fips_pfe;
120  logic      rng_fips_pfa;
121
122  logic      rng_bit_en;
123  logic      rng_bit_enable_pfe;
124  logic      rng_bit_enable_pfa;
125  logic [RngBusBitSelWidth-1:0] rng_bit_sel;
126  logic      rng_enable_q, rng_enable_d;
127  logic      entropy_data_reg_en_pfe;
128  logic      entropy_data_reg_en_pfa;
129  logic      es_data_reg_rd_en;
130  logic      event_es_entropy_valid;
131  logic      event_es_health_test_failed;
132  logic      event_es_observe_fifo_ready;
133  logic      event_es_fatal_err;
134
135  logic [RngBusWidth-1:0] sfifo_esrng_wdata;
136  logic [RngBusWidth-1:0] sfifo_esrng_rdata;
137  logic      sfifo_esrng_push;
138  logic      sfifo_esrng_pop;
139  logic      sfifo_esrng_clr;
140  logic      sfifo_esrng_full;
141  logic      sfifo_esrng_not_empty;
142  logic      sfifo_esrng_not_full;
143  logic      sfifo_esrng_int_err;
144  logic [2:0]      sfifo_esrng_err;
145
146  logic [DistrFifoWidth-1:0] sfifo_distr_wdata;
147  logic [DistrFifoWidth-1:0] sfifo_distr_rdata;
148  logic      sfifo_distr_push;
149  logic      sfifo_distr_pop;
150  logic      sfifo_distr_clr;
151  logic      sfifo_distr_not_full;
152  logic      sfifo_distr_full;
153  logic      sfifo_distr_not_empty;
154  logic [DistrFifoDepthW-1:0] sfifo_distr_depth;
155  logic      sfifo_distr_int_err;
156  logic [2:0]      sfifo_distr_err;
157
158  logic [ObserveFifoWidth-1:0] sfifo_observe_wdata;
159  logic [ObserveFifoWidth-1:0] sfifo_observe_rdata;
160  logic      sfifo_observe_push;
161  logic      sfifo_observe_pop;
162  logic      sfifo_observe_full;
163  logic      sfifo_observe_clr;
164  logic      sfifo_observe_not_empty;
165  logic [Clog2ObserveFifoDepth:0] sfifo_observe_depth;
166  logic      sfifo_observe_int_err;
167  logic [2:0]      sfifo_observe_err;
168
169  logic [EsFifoDepthW-1:0] sfifo_esfinal_depth;
170  logic [(1+SeedLen)-1:0] sfifo_esfinal_wdata;
171  logic [(1+SeedLen)-1:0] sfifo_esfinal_rdata;
172  logic      sfifo_esfinal_push;
173  logic      sfifo_esfinal_pop;
174  logic      sfifo_esfinal_clr;
175  logic      sfifo_esfinal_not_full;

176  logic      sfifo_esfinal_full;
177  logic      sfifo_esfinal_not_empty;
178  logic      sfifo_esfinal_int_err;
179  logic [2:0]      sfifo_esfinal_err;
180
181  logic      es_sfifo_int_err;
182
183  logic [SeedLen-1:0] esfinal_data;
184  logic      esfinal_fips_flag;
185
186  logic      any_fail_pulse;
187  logic      main_stage_push;
188  logic      main_stage_push_raw;
189  logic      bypass_stage_pop;
190  logic      boot_phase_done;
191  logic [HalfRegWidth-1:0] any_fail_count;
192  logic      any_fails_cntr_err;
193  logic      alert_threshold_fail;
194  logic [HalfRegWidth-1:0] alert_threshold;
195  logic [HalfRegWidth-1:0] alert_threshold_inv;
196  logic [Clog2ObserveFifoDepth:0] observe_fifo_thresh;
197  logic      observe_fifo_thresh_met;
198  logic      repcnt_active;
199  logic      repcnts_active;
200  logic      adaptp_active;
201  logic      bucket_active;
202  logic      markov_active;
203  logic      extht_active;
204  logic      alert_cntrs_clr;
205  logic      health_test_clr;
206  logic      health_test_done_pulse;
207  logic [RngBusWidth-1:0] health_test_esbus;
208  logic      health_test_esbus_vld;
209  logic      es_route_pfe;
210  logic      es_route_pfa;
211  logic      es_type_pfe;
212  logic      es_type_pfa;
213  logic      es_route_to_sw;
214  logic      es_bypass_to_sw;
215  logic      es_bypass_mode;
216  logic      alert_cntr_clr_ok;
217  logic      threshold_scope;
218  logic      threshold_scope_pfe;
219  logic      threshold_scope_pfa;
220  logic      fips_compliance;
221
222  logic [HalfRegWidth-1:0] health_test_fips_window;
223  logic [HalfRegWidth-1:0] health_test_bypass_window;
224  logic [HealthTestWindowWidth-1:0] health_test_window;
225  logic [HealthTestWindowWidth-1:0] window_cntr_step;
226
227  logic threshold_oneway_pfe;
228  logic threshold_oneway_pfa;
229
230  logic [HalfRegWidth-1:0] repcnt_threshold;
231  logic [HalfRegWidth-1:0] repcnt_threshold_oneway;
232  logic      repcnt_threshold_wr;
233  logic [HalfRegWidth-1:0] repcnt_event_cnt;
234  logic [FullRegWidth-1:0] repcnt_total_fails;
235  logic [EighthRegWidth-1:0] repcnt_fail_count;
236  logic      repcnt_fail_pulse;
237  logic      repcnt_fails_cntr_err;
238  logic      repcnt_alert_cntr_err;
239
240  logic [HalfRegWidth-1:0] repcnts_threshold;
241  logic [HalfRegWidth-1:0] repcnts_threshold_oneway;
242  logic      repcnts_threshold_wr;
243  logic [HalfRegWidth-1:0] repcnts_event_cnt;
244  logic [FullRegWidth-1:0] repcnts_total_fails;
245  logic [EighthRegWidth-1:0] repcnts_fail_count;
246  logic      repcnts_fail_pulse;
247  logic      repcnts_fails_cntr_err;
248  logic      repcnts_alert_cntr_err;
249
250  logic [HalfRegWidth-1:0] adaptp_hi_threshold;
251  logic [HalfRegWidth-1:0] adaptp_hi_threshold_oneway;
252  logic      adaptp_hi_threshold_wr;
253  logic [HalfRegWidth-1:0] adaptp_lo_threshold;
254  logic [HalfRegWidth-1:0] adaptp_lo_threshold_oneway;
255  logic      adaptp_lo_threshold_wr;
256  logic [HalfRegWidth-1:0] adaptp_hi_event_cnt;
257  logic [HalfRegWidth-1:0] adaptp_lo_event_cnt;
258  logic [FullRegWidth-1:0] adaptp_hi_total_fails;
259  logic [FullRegWidth-1:0] adaptp_lo_total_fails;
260  logic [EighthRegWidth-1:0] adaptp_hi_fail_count;
261  logic [EighthRegWidth-1:0] adaptp_lo_fail_count;
262  logic      adaptp_hi_fail_pulse;
263  logic      adaptp_lo_fail_pulse;
264  logic      adaptp_hi_fails_cntr_err;
265  logic      adaptp_lo_fails_cntr_err;
266  logic      adaptp_hi_alert_cntr_err;
267  logic      adaptp_lo_alert_cntr_err;
268
269  logic [HalfRegWidth-1:0] bucket_threshold;
270  logic [HalfRegWidth-1:0] bucket_threshold_oneway;
271  logic      bucket_threshold_wr;
272  logic [NumBucketHtInst-1:0] [HalfRegWidth-1:0] buck-
et_event_cnt;
273  logic [HalfRegWidth-1:0] bucket_event_cnt_max;
274  logic [FullRegWidth-1:0] bucket_total_fails;
275  logic [EighthRegWidth-1:0] bucket_fail_count;

```



```

276 logic [NumBucketHtInst-1:0] bucket_fail_pulse;
277 logic [prim_util_pkg::vbits(NumBucketHtInst+1)-1:0] bucket_fail_pulse_step;
278 logic bucket_fails_cntr_err;
279 logic bucket_alert_cntr_err;
280
281 logic [HalfRegWidth-1:0] markov_hi_threshold;
282 logic [HalfRegWidth-1:0] markov_hi_threshold_oneway;
283 logic markov_hi_threshold_wr;
284 logic [HalfRegWidth-1:0] markov_lo_threshold;
285 logic [HalfRegWidth-1:0] markov_lo_threshold_oneway;
286 logic markov_lo_threshold_wr;
287 logic [HalfRegWidth-1:0] markov_hi_event_cnt;
288 logic [HalfRegWidth-1:0] markov_lo_event_cnt;
289 logic [FullRegWidth-1:0] markov_hi_total_fails;
290 logic [FullRegWidth-1:0] markov_lo_total_fails;
291 logic [EighthRegWidth-1:0] markov_hi_fail_count;
292 logic [EighthRegWidth-1:0] markov_lo_fail_count;
293 logic markov_hi_fail_pulse;
294 logic markov_lo_fail_pulse;
295 logic markov_hi_fails_cntr_err;
296 logic markov_lo_fails_cntr_err;
297 logic markov_hi_alert_cntr_err;
298 logic markov_lo_alert_cntr_err;
299
300 logic [HalfRegWidth-1:0] extht_hi_threshold;
301 logic [HalfRegWidth-1:0] extht_hi_threshold_oneway;
302 logic extht_hi_threshold_wr;
303 logic [HalfRegWidth-1:0] extht_lo_threshold;
304 logic [HalfRegWidth-1:0] extht_lo_threshold_oneway;
305 logic extht_lo_threshold_wr;
306 logic [HalfRegWidth-1:0] extht_event_cnt_hi;
307 logic [HalfRegWidth-1:0] extht_event_cnt_lo;
308 logic [FullRegWidth-1:0] extht_hi_total_fails;
309 logic [FullRegWidth-1:0] extht_lo_total_fails;
310 logic [EighthRegWidth-1:0] extht_hi_fail_count;
311 logic [EighthRegWidth-1:0] extht_lo_fail_count;
312 logic extht_hi_fail_pulse;
313 logic extht_lo_fail_pulse;
314 logic extht_cnt_test;
315 logic extht_hi_fails_cntr_err;
316 logic extht_lo_fails_cntr_err;
317 logic extht_hi_alert_cntr_err;
318 logic extht_lo_alert_cntr_err;
319
320 ht_watermark_num_e ht_watermark_num_reg_if;
321 logic ht_watermark_high;
322 logic ht_watermark_event;
323 logic ht_watermark_event_pre;
324 logic [HalfRegWidth-1:0] ht_watermark_cnt;
325 logic [HalfRegWidth-1:0] ht_watermark;
326
327 logic pfifo_esbit_wdata;
328 logic [RngBusWidth-1:0] pfifo_esbit_rdata;
329 logic pfifo_esbit_not_empty;
330 logic pfifo_esbit_not_full;
331 logic pfifo_esbit_push;
332 logic pfifo_esbit_clr;
333 logic pfifo_esbit_pop;
334
335 logic [RngBusWidth-1:0] pfifo_postht_wdata;
336 logic [PostHTWidth-1:0] pfifo_postht_rdata;
337 logic pfifo_postht_not_empty;
338 logic pfifo_postht_not_full;
339 logic pfifo_postht_push;
340 logic pfifo_postht_clr;
341 logic pfifo_postht_pop;
342 logic [PostHTFifoDepthW-1:0] pfifo_postht_depth;
343
344 logic [PreCondWidth-1:0] pfifo_cond_wdata;
345 logic [SeedLen-1:0] pfifo_cond_rdata;
346 logic pfifo_cond_push;
347
348 logic [ObserveFifoWidth-1:0] pfifo_precon_wdata;
349 logic [PreCondWidth-1:0] pfifo_precon_rdata;
350 logic pfifo_precon_not_empty;
351 logic pfifo_precon_not_full;
352 logic pfifo_precon_push;
353 logic pfifo_precon_clr;
354 logic pfifo_precon_pop;
355 logic [PreCondFifoDepthW-1:0] pfifo_precon_depth;
356
357 logic [PostHTWidth-1:0] pfifo_bypass_wdata;
358 logic [SeedLen-1:0] pfifo_bypass_rdata;
359 logic pfifo_bypass_not_empty;
360 logic pfifo_bypass_not_full;
361 logic pfifo_bypass_push;
362 logic pfifo_bypass_clr;
363 logic pfifo_bypass_pop;
364
365 logic [PipelineDepthW-1:0] pipeline_depth;
366 logic pipeline_depth_cntr_err;
367
368 logic [SwReadIdxWidth-1:0] swread_idx_d, swread_idx_q;
369 logic swread_idx_incr, swread_idx_clr;
370 logic [FullRegWidth-1:0] swread_data, swread_data_buf;
371 logic swread_done;
372
373 logic [SeedLen-1:0] final_es_data;
374 logic es_hw_if_req;
375 logic es_hw_if_ack;

```

```

376 logic es_hw_if_fifo_pop;
377 logic sfifo_esrng_err_sum;
378 logic sfifo_distr_err_sum;
379 logic sfifo_observe_err_sum;
380 logic sfifo_esfinal_err_sum;
381 // For fifo errors that are generated through the
382 // ERR_CODE_TEST register, but are not associated
383 // with any errors:
384 logic sfifo_test_err_sum;
385 logic es_ack_sm_err_sum;
386 logic es_ack_sm_err;
387 logic es_main_sm_err_sum;
388 logic es_main_sm_err;
389 logic es_main_sm_alert;
390 logic es_bus_cmp_alert;
391 logic es_thresh_cfg_alert;
392 logic es_main_sm_idle;
393 logic [8:0] es_main_sm_state;
394 logic fifo_write_err_sum;
395 logic fifo_read_err_sum;
396 logic fifo_status_err_sum;
397 logic [30:0] err_code_test_bit;
398 logic sha3_msgfifo_ready;
399 logic sha3_state_vld;
400 logic sha3_start_raw;
401 logic sha3_start;
402 logic sha3_process;
403 logic sha3_block_processed;
404 prim_mubi_pkg::mubi4_t sha3_done;
405 prim_mubi_pkg::mubi4_t sha3_absorbed;
406 logic sha3_squeezing;
407 logic [2:0] sha3_fsm;
408 logic [32:0] sha3_err;
409 logic sha3_block_busy;
410 logic [HealthTestWindowWidth-1:0] window_cntr;
411 logic window_cntr_incr_en;
412
413 logic [sha3_pkg::StateW-1:0] sha3_state[Sha3Share];
414 logic [PreCondWidth-1:0] msg_data[Sha3Share];
415 logic es_rdata_capt_vld;
416 logic window_cntr_err;
417 logic repcnts_cntr_err;
418 logic repcnts_cntr_err;
419 logic adaptp_cntr_err;
420 logic [NumBucketHtInst-1:0] bucket_cntr_err;
421 logic markov_cntr_err;
422 logic es_cntr_err;
423 logic es_cntr_err_sum;
424 logic sha3_state_error_sum;
425 logic sha3_rst_storage_err_sum;
426 logic efuse_es_sw_reg_en;
427 logic efuse_es_sw_ov_en;
428
429 logic sha3_state_error;
430 logic sha3_count_error;
431 logic sha3_rst_storage_err;
432 logic es_hw_regwen;
433 logic recov_alert_state;
434 logic es_fw_ov_wr_alert;
435 logic es_fw_ov_disable_alert;
436 logic fw_ov_corrupted;
437 logic postht_entropy_drop_alert;
438
439 logic stale_seed_processing;
440 logic main_sm_enable;
441 logic main_sm_done_pulse;
442 logic main_sm_ht_failed;
443
444 logic unused_err_code_test_bit;
445 logic unused_sha3_state;
446 logic unused_entropy_data;
447 logic unused_fw_ov_rd_data;
448 logic unused_sfifo_esrng_not_full;
449 logic unused_sfifo_esfinal_not_full;
450
451 prim_mubi_pkg::mubi8_t en_entropy_src_fw_read;
452 prim_mubi_pkg::mubi8_t en_entropy_src_fw_over;
453
454 mubi4_t mubi_es_enable;
455 mubi4_t mubi_module_en_pulse;
456
457 mubi4_t mubi_module_en_raw;
458 mubi4_t [2:0] mubi_module_en_raw_fanout;
459
460 mubi4_t [EsEnableCopies-1:0] mubi_es_enable_fanout;
461 logic [EsEnableCopies-1:0] es_enable_fo;
462
463 mubi4_t [EsEnPulseCopies-1:0] mubi_module_en_pulse_fanout;
464 logic [EsEnPulseCopies-1:0] module_en_pulse_fo;
465
466 // A delayed copy of the enable signal, and enable pulse
467 // which are needed to cleanly handle any
468 // long-duration operations from the previous run.
469 logic es_delayed_enable;
470
471 mubi4_t [1:0] mubi_rng_bit_en_fanout;
472 mubi4_t mubi_rng_bit_en;
473
474 // flops
475 logic ht_failed_qq, ht_failed_q, ht_failed_d;
476 logic ht_done_pulse_qq, ht_done_pulse_q;

```

```

ht_done_pulse_d;
476 logic      sha3_err_q, sha3_err_d;
477 logic [63:0] es_rdata_capt_q, es_rdata_capt_d;
478 logic      es_rdata_capt_vld_q, es_rdata_capt_vld_d;
479 mubi4_t     mubi_mod_en_dly_d, mubi_mod_en_dly_q;
480
481 ht_watermark_num_e ht_watermark_num_q, ht_watermark_num_d;
482
483 logic      sha3_start_mask_q, sha3_start_mask_d;
484 logic      sha3_flush_q, sha3_flush_d;
485 logic [1:0] fw_ov_corrupted_q, fw_ov_corrupted_d;
486
487 always_ff @(posedge clk_i or negedge rst_ni) begin
488   if (!rst_ni) begin
489     ht_failed_q      <= '0;
490     ht_failed_qq     <= '0;
491     ht_done_pulse_q  <= '0;
492     ht_done_pulse_qq <= '0;
493     sha3_err_q       <= '0;
494     es_rdata_capt_q  <= '0;
495     es_rdata_capt_vld_q <= '0;
496     fw_ov_sha3_start_pfe_q <= '0;
497     mubi_mod_en_dly_q <= prim_mubi_pkg::MuBi4False;
498     sha3_flush_q     <= '0;
499     sha3_start_mask_q <= '0;
500     fw_ov_corrupted_q <= 2'b00;
501     rng_enable_q     <= 1'b 0;
502   end else begin
503     ht_failed_q      <= ht_failed_d;
504     ht_failed_qq     <= ht_failed_q;
505     ht_done_pulse_q  <= ht_done_pulse_d;
506     ht_done_pulse_qq <= ht_done_pulse_q;
507     sha3_err_q       <= sha3_err_d;
508     es_rdata_capt_q  <= es_rdata_capt_d;
509     es_rdata_capt_vld_q <= es_rdata_capt_vld_d;
510     fw_ov_sha3_start_pfe_q <= fw_ov_sha3_start_pfe_d;
511     sha3_flush_q     <= sha3_flush_d;
512     sha3_start_mask_q <= sha3_start_mask_d;
513     mubi_mod_en_dly_q <= mubi_mod_en_dly_d;
514     fw_ov_corrupted_q <= fw_ov_corrupted_d;
515     rng_enable_q     <= rng_enable_d;
516   end
517 end
518
519 assign fw_ov_sha3_disable_pulse = fw_ov_sha3_start_pfe_q &
~fw_ov_sha3_start_pfe_q;
520
521 //-----
522 // register lock gating
523 //-----
524
525 // Allow writes only if
526 // 1. SW_REGUPD is true,
527 // 2. The DUT is disabled
528 // Block writes if enabled or if internal activities are
529 // still in progress (as indicated by
530 // es_delayed_enable).
531 assign es_hw_regwen = reg2hw.sw_regupd.q &&
mu-
532 bi4_test_false_loose(mubi_module_en_raw_fanout[0]) &&
!es_delayed_enable;
533 assign hw2reg.regwen.de = 1'b1;
534 assign hw2reg.regwen.d = es_hw_regwen;
535
536 //-----
537 // set up secure enable bits
538 //-----
539
540 // check for illegal enable field states, and set alert if
541 // detected
542 // SEC_CM: CONFIG.MUBI
543 assign mubi_module_en_raw = mu-
544 bi4_t'(reg2hw.module_enable.q);
545 assign es_enable_pfa = mu-
546 bi4_test_invalid(mubi_module_en_raw_fanout[1]);
547
548 assign hw2reg.recov_alert_sts.module_enable_field_alert.de =
es_enable_pfa;
549
550 assign hw2reg.recov_alert_sts.module_enable_field_alert.d =
es_enable_pfa;
551
552 prim_mubi4_sync #(
553   .NumCopies(3),
554   .AsyncOn(0)
555 ) u_prim_mubi4_sync_entropy_module_en (
556   .clk_i,
557   .rst_ni,
558   .mubi_i(mubi_module_en_raw),
559   .mubi_o(mubi_module_en_raw_fanout)
560 );
561
562 // Generation of enable pulse, and main enable signals.
563 // This module creates a single mubi_encoded mod-
ule_enable_pulse, as well as
564 // a general mubi_es_enable signal.
565 //
566 // The module enable pulse is MuBi4True for a single clock
567 // after the

```

```

564 // module_enable register is asserted MuBi4True. However in
565 // this first clock cycle
566 // most of the module is not enabled. (The enable pulse is
567 // for clearing the
568 // residual internal state.)
569 //
570 // The rest of the module is enabled in the second clock
571 // cycle after setting the
572 // module_enable register to MuBi4True.
573 //
574 // When module_enable is set to MuBi4False the module is
575 // disabled. No further
576 // RNG input is accepted and the state machines transition
577 // to Idle as soon as
578 // possible. Some delay may occur if the SHA3 conditioning
579 // block is busy, though
580 // otherwise the main_sm transitions to Idle without delay.
581
582 assign mubi_mod_en_dly_d = mubi_module_en_raw_fanout[2];
583 assign mubi_module_en_pulse = mu-
584 bi4_and_hi(mubi_mod_en_dly_d, mubi4_t'(~mubi_mod_en_dly_q));
585 assign mubi_es_enable = mubi4_and_hi(mubi_mod_en_dly_d,
586 mubi_mod_en_dly_q);
587
588 for (genvar i = 0; i < EsEnableCopies; i = i+1) begin :
589   gen_mubi_en_copies
590     assign es_enable_fo[i] = mu-
591     bi4_test_true_strict(mubi_es_enable_fanout[i]);
592   end : gen_mubi_en_copies
593
594 prim_mubi4_sync #(
595   .NumCopies(EsEnableCopies),
596   .AsyncOn(0)
597 ) u_prim_mubi4_sync_es_enable (
598   .clk_i,
599   .rst_ni,
600   .mubi_i(mubi_es_enable),
601   .mubi_o(mubi_es_enable_fanout)
602 );
603
604 for (genvar i = 0; i < EsEnPulseCopies; i = i+1) begin :
605   gen_mubi_en_pulse_copies
606     assign module_en_pulse_fo[i] = mu-
607     bi4_test_true_strict(mubi_module_en_pulse_fanout[i]);
608   end : gen_mubi_en_pulse_copies
609
610 prim_mubi4_sync #(
611   .NumCopies(EsEnPulseCopies),
612   .AsyncOn(0)
613 ) u_prim_mubi4_sync_es_enable_pulse (
614   .clk_i,
615   .rst_ni,
616   .mubi_i(mubi_module_en_pulse),
617   .mubi_o(mubi_module_en_pulse_fanout)
618 );
619
620 entropy_src_enable_delay u_enable_delay (
621   .clk_i,
622   .rst_ni,
623   .enable_i(es_enable_fo[0]),
624   .esrng_fifo_not_empty_i(sfifo_esrng_not_empty),
625   .esbit_fifo_not_empty_i(pfifo_esbit_not_empty),
626   .postht_fifo_not_empty_i(pfifo_postht_not_empty),
627   .distr_fifo_not_empty_i(sfifo_distr_not_empty),
628   .sha3_block_busy_i(sha3_block_busy),
629   .sha3_block_processed_i(sha3_block_processed),
630   .bypass_mode_i(es_bypass_mode),
631   .enable_o(es_delayed_enable)
632 );
633
634 mubi4_t mubi_fips_en;
635 mubi4_t [1:0] mubi_fips_en_fanout;
636 assign mubi_fips_en = mubi4_t'(reg2hw.conf.fips_enable.q);
637 assign fips_enable_pfe = mu-
638 bi4_test_true_strict(mubi_fips_en_fanout[0]);
639 assign fips_enable_pfa = mu-
640 bi4_test_invalid(mubi_fips_en_fanout[1]);
641 assign hw2reg.recov_alert_sts.fips_enable_field_alert.de =
fips_enable_pfa;
642 assign hw2reg.recov_alert_sts.fips_enable_field_alert.d =
fips_enable_pfa;
643
644 prim_mubi4_sync #(
645   .NumCopies(2),
646   .AsyncOn(0)
647 ) u_prim_mubi4_sync_entropy_fips_en (
648   .clk_i,
649   .rst_ni,
650   .mubi_i(mubi_fips_en),
651   .mubi_o(mubi_fips_en_fanout)
652 );
653
654 mubi4_t mubi_fips_flag;
655 mubi4_t [1:0] mubi_fips_flag_fanout;
656 assign mubi_fips_flag = mubi4_t'(reg2hw.conf.fips_flag.q);
657 assign fips_flag_pfa = mu-
658 bi4_test_invalid(mubi_fips_flag_fanout[1]);
659 assign fips_flag_pfe = mu-
660 bi4_test_true_strict(mubi_fips_flag_fanout[0]);
661 assign hw2reg.recov_alert_sts.fips_flag_field_alert.de =
fips_flag_pfa;

```

```

646 assign hw2reg.recov_alert_sts.fips_flag_field_alert.d =
fips_flag_pfa;
647
648 prim_mubi4_sync #(
649     .NumCopies(2),
650     .AsyncOn(0)
651 ) u_prim_mubi4_sync_entropy_fips_flag (
652     .clk_i,
653     .rst_ni,
654     .mubi_i(mubi_fips_flag),
655     .mubi_o(mubi_fips_flag_fanout)
656 );
657
658 mubi4_t mubi_rng_fips;
659 mubi4_t [1:0] mubi_rng_fips_fanout;
660 assign mubi_rng_fips = mubi4_t'(reg2hw.conf.rng_fips.q);
661 assign rng_fips_pfa = mu-
bi4_test_invalid(mubi_rng_fips_fanout[1]);
662 assign rng_fips_pfe =
prim_mubi_pkg::mubi4_test_true_loose(mubi_rng_fips_fanout[0]);
663 assign hw2reg.recov_alert_sts.rng_fips_field_alert.de =
rng_fips_pfa;
664 assign hw2reg.recov_alert_sts.rng_fips_field_alert.d =
rng_fips_pfa;
665
666 prim_mubi4_sync #(
667     .NumCopies(2),
668     .AsyncOn(0)
669 ) u_prim_mubi4_sync_entropy_rng_fips (
670     .clk_i,
671     .rst_ni,
672     .mubi_i(mubi_rng_fips),
673     .mubi_o(mubi_rng_fips_fanout)
674 );
675
676 // SEC_CM: CONFIG.MUBI
677 mubi4_t mubi_entropy_reg_en;
678 mubi4_t [1:0] mubi_entropy_reg_en_fanout;
679 assign mubi_entropy_reg_en = mu-
bi4_t'(reg2hw.conf.entropy_data_reg_enable.q);
680 assign entropy_data_reg_en_pfe = mu-
bi4_test_true_strict(mubi_entropy_reg_en_fanout[0]);
681 assign entropy_data_reg_en_pfa = mu-
bi4_test_invalid(mubi_entropy_reg_en_fanout[1]);
682 assign
hw2reg.recov_alert_sts.entropy_data_reg_en_field_alert.de = en-
tropy_data_reg_en_pfa;
683 assign
hw2reg.recov_alert_sts.entropy_data_reg_en_field_alert.d = en-
tropy_data_reg_en_pfa;
684
685 prim_mubi4_sync #(
686     .NumCopies(2),
687     .AsyncOn(0)
688 ) u_prim_mubi4_sync_entropy_data_reg_en (
689     .clk_i,
690     .rst_ni,
691     .mubi_i(mubi_entropy_reg_en),
692     .mubi_o(mubi_entropy_reg_en_fanout)
693 );
694
695 // SEC_CM: CONFIG.MUBI
696 mubi4_t mubi_threshold_oneway;
697 mubi4_t [1:0] mubi_threshold_oneway_fanout;
698 assign mubi_threshold_oneway = mu-
bi4_t'(reg2hw.threshold_oneway.q);
699 assign threshold_oneway_pfe = mu-
bi4_test_true_loose(mubi_threshold_oneway_fanout[0]);
700 assign threshold_oneway_pfa = mu-
bi4_test_invalid(mubi_threshold_oneway_fanout[1]);
701 assign
hw2reg.recov_alert_sts.threshold_oneway_field_alert.de =
threshold_oneway_pfa;
702 assign hw2reg.recov_alert_sts.threshold_oneway_field_alert.d =
threshold_oneway_pfa;
703
704 prim_mubi4_sync #(
705     .NumCopies(2),
706     .AsyncOn(0)
707 ) u_prim_mubi4_sync_treshold_oneway (
708     .clk_i,
709     .rst_ni,
710     .mubi_i(mubi_threshold_oneway),
711     .mubi_o(mubi_threshold_oneway_fanout)
712 );
713
714 assign observe_fifo_thresh = reg2hw.observe_fifo_thresh.q;
715
716 // SEC_CM: CONFIG.MUBI
717 mubi4_t mubi_fw_ov_mode;
718 mubi4_t [1:0] mubi_fw_ov_mode_fanout;
719 assign mubi_fw_ov_mode = mu-
bi4_t'(reg2hw.fw_ov_control.fw_ov_mode.q);
720 assign fw_ov_mode_pfe = mu-
bi4_test_true_strict(mubi_fw_ov_mode_fanout[0]);
721 assign fw_ov_mode_pfa = mu-
bi4_test_invalid(mubi_fw_ov_mode_fanout[1]);
722 assign hw2reg.recov_alert_sts.fw_ov_mode_field_alert.de =
fw_ov_mode_pfa;
723 assign hw2reg.recov_alert_sts.fw_ov_mode_field_alert.d =

```

```

fw_ov_mode_pfa;
724
725 prim_mubi4_sync #(
726     .NumCopies(2),
727     .AsyncOn(0)
728 ) u_prim_mubi4_sync_fw_ov_mode (
729     .clk_i,
730     .rst_ni,
731     .mubi_i(mubi_fw_ov_mode),
732     .mubi_o(mubi_fw_ov_mode_fanout)
733 );
734
735 // SEC_CM: CONFIG.MUBI
736 mubi4_t mubi_fw_ov_entropy_insert;
737 mubi4_t [1:0] mubi_fw_ov_entropy_insert_fanout;
738 assign mubi_fw_ov_entropy_insert = mu-
bi4_t'(reg2hw.fw_ov_control.fw_ov_entropy_insert.q);
739 assign fw_ov_entropy_insert_pfe = mu-
bi4_test_true_strict(mubi_fw_ov_entropy_insert_fanout[0]);
740 assign fw_ov_entropy_insert_pfa = mu-
bi4_test_invalid(mubi_fw_ov_entropy_insert_fanout[1]);
741 assign
hw2reg.recov_alert_sts.fw_ov_entropy_insert_field_alert.de =
fw_ov_entropy_insert_pfa;
742 assign
hw2reg.recov_alert_sts.fw_ov_entropy_insert_field_alert.d =
fw_ov_entropy_insert_pfa;
743
744 prim_mubi4_sync #(
745     .NumCopies(2),
746     .AsyncOn(0)
747 ) u_prim_mubi4_sync_fw_ov_entropy_insert (
748     .clk_i,
749     .rst_ni,
750     .mubi_i(mubi_fw_ov_entropy_insert),
751     .mubi_o(mubi_fw_ov_entropy_insert_fanout)
752 );
753
754 // SEC_CM: CONFIG.MUBI
755 mubi4_t mubi_fw_ov_sha3_start;
756 mubi4_t [1:0] mubi_fw_ov_sha3_start_fanout;
757 assign mubi_fw_ov_sha3_start = mu-
bi4_t'(reg2hw.fw_ov_sha3_start.q);
758 assign fw_ov_sha3_start_pfe = mu-
bi4_test_true_strict(mubi_fw_ov_sha3_start_fanout[0]);
759 assign fw_ov_sha3_start_pfa = mu-
bi4_test_invalid(mubi_fw_ov_sha3_start_fanout[1]);
760 assign
hw2reg.recov_alert_sts.fw_ov_sha3_start_field_alert.de =
fw_ov_sha3_start_pfa;
761 assign
hw2reg.recov_alert_sts.fw_ov_sha3_start_field_alert.d =
fw_ov_sha3_start_pfa;
762
763 prim_mubi4_sync #(
764     .NumCopies(2),
765     .AsyncOn(0)
766 ) u_prim_mubi4_sync_fw_ov_sha3_start (
767     .clk_i,
768     .rst_ni,
769     .mubi_i(mubi_fw_ov_sha3_start),
770     .mubi_o(mubi_fw_ov_sha3_start_fanout)
771 );
772
773 // firmware override controls
774 assign fw_ov_mode = efuse_es_sw_ov_en && fw_ov_mode_pfe;
775 assign fw_ov_mode_entropy_insert = fw_ov_mode && fw_ov_en-
tropy_insert_pfe;
776 assign fw_ov_fifo_rd_pulse = reg2hw.fw_ov_rd_data.re;
777 assign hw2reg.fw_ov_rd_data.d = sfifo_observe_rdata;
778 assign fw_ov_fifo_wr_pulse = reg2hw.fw_ov_wr_data.qe;
779 assign fw_ov_wr_data = reg2hw.fw_ov_wr_data.q;
780
781 assign efuse_es_sw_ov_en =
prim_mubi_pkg::mubi8_test_true_strict(en_entropy_src_fw_over);
782
783 prim_mubi8_sync #(
784     .NumCopies(1),
785     .AsyncOn(1) // must be set to one, see note below
786 ) u_prim_mubi8_sync_es_fw_over (
787     .clk_i,
788     .rst_ni,
789     .mubi_i(otp_en_entropy_src_fw_over_i),
790     .mubi_o({en_entropy_src_fw_over})
791 );
792
793 // note: the input to the above sync module is from the OTP
block.
794 // It is assumed that the source is in a different
time domain,
795 // and requires the AsyncOn parameter to be set.
796
797 // rng_enable is being used in other clock domains. Need to
latch the
798 // signal.
799 assign rng_enable_d = es_enable_fo[1] &&
800     es_delayed_enable;
801
802 assign entropy_src_rng_enable_o = rng_enable_q;
803
804 //-----

```



```

805 // instantiate interrupt hardware primitives
806 //-----
807
808 prim_intr_hw #(
809     .Width(1)
810 ) u_intr_hw_es_entropy_valid (
811     .clk_i      (clk_i),
812     .rst_ni     (rst_ni),
813     .event_intr_i (event_es_entropy_valid),
814     .reg2hw_intr_enable_q_i
815 (reg2hw.intr_enable.es_entropy_valid.q),
816     .reg2hw_intr_test_q_i
817 (reg2hw.intr_test.es_entropy_valid.q),
818     .reg2hw_intr_test_qe_i
819 (reg2hw.intr_test.es_entropy_valid.qe),
820     .reg2hw_intr_state_q_i
821 (reg2hw.intr_state.es_entropy_valid.q),
822     .hw2reg_intr_state_de_o
823 (hw2reg.intr_state.es_entropy_valid.de),
824     .hw2reg_intr_state_d_o
825 (hw2reg.intr_state.es_entropy_valid.d),
826     .intr_o      (intr_es_entropy_valid_o)
827 );
828
829 prim_intr_hw #(
830     .Width(1)
831 ) u_intr_hw_es_health_test_failed (
832     .clk_i      (clk_i),
833     .rst_ni     (rst_ni),
834     .event_intr_i (event_es_health_test_failed),
835     .reg2hw_intr_enable_q_i
836 (reg2hw.intr_enable.es_health_test_failed.q),
837     .reg2hw_intr_test_q_i
838 (reg2hw.intr_test.es_health_test_failed.q),
839     .reg2hw_intr_test_qe_i
840 (reg2hw.intr_test.es_health_test_failed.qe),
841     .reg2hw_intr_state_q_i
842 (reg2hw.intr_state.es_health_test_failed.q),
843     .hw2reg_intr_state_de_o
844 (hw2reg.intr_state.es_health_test_failed.de),
845     .hw2reg_intr_state_d_o
846 (hw2reg.intr_state.es_health_test_failed.d),
847     .intr_o      (intr_es_health_test_failed_o)
848 );
849
850 prim_intr_hw #(
851     .Width(1)
852 ) u_intr_hw_es_observe_fifo_ready (
853     .clk_i      (clk_i),
854     .rst_ni     (rst_ni),
855     .event_intr_i (event_es_observe_fifo_ready),
856     .reg2hw_intr_enable_q_i
857 (reg2hw.intr_enable.es_observe_fifo_ready.q),
858     .reg2hw_intr_test_q_i
859 (reg2hw.intr_test.es_observe_fifo_ready.q),
860     .reg2hw_intr_test_qe_i
861 (reg2hw.intr_test.es_observe_fifo_ready.qe),
862     .reg2hw_intr_state_q_i
863 (reg2hw.intr_state.es_observe_fifo_ready.q),
864     .hw2reg_intr_state_de_o
865 (hw2reg.intr_state.es_observe_fifo_ready.de),
866     .hw2reg_intr_state_d_o
867 (hw2reg.intr_state.es_observe_fifo_ready.d),
868     .intr_o      (intr_es_observe_fifo_ready_o)
869 );
870
871 prim_intr_hw #(
872     .Width(1)
873 ) u_intr_hw_es_fatal_err (
874     .clk_i      (clk_i),
875     .rst_ni     (rst_ni),
876     .event_intr_i (event_es_fatal_err),
877     .reg2hw_intr_enable_q_i
878 (reg2hw.intr_enable.es_fatal_err.q),
879     .reg2hw_intr_test_q_i
880 (reg2hw.intr_test.es_fatal_err.q),
881     .reg2hw_intr_test_qe_i
882 (reg2hw.intr_test.es_fatal_err.qe),
883     .reg2hw_intr_state_q_i
884 (reg2hw.intr_state.es_fatal_err.q),
885     .hw2reg_intr_state_de_o
886 (hw2reg.intr_state.es_fatal_err.de),
887     .hw2reg_intr_state_d_o
888 (hw2reg.intr_state.es_fatal_err.d),
889     .intr_o      (intr_es_fatal_err_o)
890 );
891
892 //-----
893 // tlul register settings
894 //-----
895
896 // set the interrupt event when enabled
897 assign event_es_entropy_valid =
898     sfifo_esfinal_not_empty && es_route_to_sw &&
899     es_data_reg_rd_en && es_enable_fo[2];
900
901
902 // Collect all internal FIFO errors.
903 assign es_sfifo_int_err = sfifo_esrng_int_err ||
904     sfifo_distr_int_err ||

```

```

905     sfifo_observe_int_err ||
906     sfifo_esfinal_int_err;
907
908 // Counter, internal FIFO errors and FSM errors are struc-
909 tural errors and are always active
910 // regardless of the functional state.
911 logic fatal_loc_events;
912 assign fatal_loc_events = es_cntr_err_sum ||
913     es_sfifo_int_err ||
914     es_main_sm_err_sum ||
915     es_ack_sm_err_sum ||
916     sha3_state_error_sum;
917
918 // set the interrupt sources
919 assign event_es_fatal_err = (es_enable_fo[3] &&
920     (sfifo_esrng_err_sum ||
921     sfifo_distr_err_sum ||
922     sfifo_observe_err_sum ||
923     sfifo_esfinal_err_sum ||
924     sfifo_test_err_sum) ) ||
925     sha3_rst_storage_err_sum ||
926     fatal_loc_events;
927
928 // set fifo errors that are single instances of source
929 assign sfifo_esrng_err_sum = (|sfifo_esrng_err) ||
930     err_code_test_bit[0];
931 assign sfifo_distr_err_sum = (|sfifo_distr_err) ||
932     err_code_test_bit[1];
933 assign sfifo_observe_err_sum = (|sfifo_observe_err) ||
934     err_code_test_bit[2];
935 assign sfifo_esfinal_err_sum = (|sfifo_esfinal_err) ||
936     err_code_test_bit[3];
937
938 // The following test bits help normally diagnose the _type_
939 of
940 // error when they are triggered by the fifo. However when
941 // they are triggered by software they are not linked to a
942 // particular sfifo and do not trigger an alert, unless
943 // we capture them here.
944 assign sfifo_test_err_sum = err_code_test_bit[28] ||
945     err_code_test_bit[29] ||
946     err_code_test_bit[30];
947
948 assign es_ack_sm_err_sum = es_ack_sm_err ||
949     err_code_test_bit[20];
950 assign es_main_sm_err_sum = es_main_sm_err ||
951     err_code_test_bit[21];
952 assign es_cntr_err_sum = es_cntr_err ||
953     err_code_test_bit[22];
954 assign sha3_state_error_sum = sha3_state_error ||
955     err_code_test_bit[23];
956 assign sha3_rst_storage_err_sum = sha3_rst_storage_err ||
957     err_code_test_bit[24];
958 assign fifo_write_err_sum =
959     sfifo_esrng_err[2] ||
960     sfifo_observe_err[2] ||
961     sfifo_distr_err[2] ||
962     sfifo_esfinal_err[2] ||
963     err_code_test_bit[28];
964 assign fifo_read_err_sum =
965     sfifo_esrng_err[1] ||
966     sfifo_observe_err[1] ||
967     sfifo_distr_err[1] ||
968     sfifo_esfinal_err[1] ||
969     err_code_test_bit[29];
970 assign fifo_status_err_sum =
971     sfifo_esrng_err[0] ||
972     sfifo_observe_err[0] ||
973     sfifo_distr_err[0] ||
974     sfifo_esfinal_err[0] ||
975     err_code_test_bit[30];
976
977 // set the err code source bits
978 assign hw2reg.err_code.sfifo_esrng_err.d = 1'b1;
979 assign hw2reg.err_code.sfifo_esrng_err.de =
980     sfifo_esrng_err_sum;
981
982 assign hw2reg.err_code.sfifo_distr_err.d = 1'b1;
983 assign hw2reg.err_code.sfifo_distr_err.de =
984     sfifo_distr_err_sum;
985
986 assign hw2reg.err_code.sfifo_observe_err.d = 1'b1;
987 assign hw2reg.err_code.sfifo_observe_err.de =
988     sfifo_observe_err_sum;
989
990 assign hw2reg.err_code.sfifo_esfinal_err.d = 1'b1;
991 assign hw2reg.err_code.sfifo_esfinal_err.de =
992     sfifo_esfinal_err_sum;
993
994 assign hw2reg.err_code.es_ack_sm_err.d = 1'b1;
995 assign hw2reg.err_code.es_ack_sm_err.de = es_ack_sm_err_sum;
996
997 assign hw2reg.err_code.es_main_sm_err.d = 1'b1;
998 assign hw2reg.err_code.es_main_sm_err.de =
999     es_main_sm_err_sum;
1000
1001 assign hw2reg.err_code.es_cntr_err.d = 1'b1;
1002 assign hw2reg.err_code.es_cntr_err.de = es_cntr_err_sum;
1003
1004 assign hw2reg.err_code.sha3_state_err.d = 1'b1;

```

```

975 assign hw2reg.err_code.sha3_state_err.de =
sha3_state_error_sum;
976
977 assign hw2reg.err_code.sha3_rst_storage_err.d = 1'b1;
978 assign hw2reg.err_code.sha3_rst_storage_err.de =
sha3_rst_storage_err_sum;
979
980
981 // set the err code type bits
982 assign hw2reg.err_code.fifo_write_err.d = 1'b1;
983 assign hw2reg.err_code.fifo_write_err.de =
fifo_write_err_sum;
984
985 assign hw2reg.err_code.fifo_read_err.d = 1'b1;
986 assign hw2reg.err_code.fifo_read_err.de = fifo_read_err_sum;
987
988 assign hw2reg.err_code.fifo_state_err.d = 1'b1;
989 assign hw2reg.err_code.fifo_state_err.de =
fifo_status_err_sum;
990
991 // Error forcing
992 for (genvar i = 0; i < 31; i = i+1) begin :
gen_err_code_test_bit
993 assign err_code_test_bit[i] = (reg2hw.err_code_test.q ==
i) && reg2hw.err_code_test.qe;
994 end : gen_err_code_test_bit
995
996 // alert - send all interrupt sources to the alert for the
fatal case
997 assign fatal_alert_o = (event_es_fatal_err |
998 sfifo_distr_int_err |
sfifo_esrng_int_err |
999 sfifo_observe_int_err |
sfifo_esfinal_int_err);
1000
1001 // alert test
1002 assign recov_alert_test_o = {
1003 reg2hw.alert_test.recov_alert.q &&
1004 reg2hw.alert_test.recov_alert.qe
1005 };
1006 assign fatal_alert_test_o = {
1007 reg2hw.alert_test.fatal_alert.q &&
1008 reg2hw.alert_test.fatal_alert.qe
1009 };
1010
1011 // set the debug status reg
1012 assign hw2reg.debug_status.entropy_fifo_depth.d =
sfifo_esfinal_depth;
1013 assign hw2reg.debug_status.sha3_fsm.d = sha3_fsm;
1014 assign hw2reg.debug_status.sha3_block_pr.d = sha3_b-
lock_processed;
1015 assign hw2reg.debug_status.sha3_squeezing.d = sha3_squeeze-
ing;
1016 assign hw2reg.debug_status.sha3_absorbed.d =
1017
1018 prim_mubi_pkg::mubi4_test_true_strict(sha3_absorbed) ? 1'b
1 : 1'b 0;
1019 assign hw2reg.debug_status.sha3_err.d = sha3_err_q;
1020
1021 assign sha3_err_d =
1022 es_enable_fo[4] ? 1'b0 :
1023 {sha3_err} ? 1'b1 :
1024 sha3_err_q;
1025
1026 // state machine status
1027 assign hw2reg.debug_status.main_sm_idle.d = es_main_sm_idle;
1028 assign hw2reg.debug_status.main_sm_boot_done.d =
boot_phase_done;
1029 assign hw2reg.main_sm_state.de = 1'b1;
1030 assign hw2reg.main_sm_state.d = es_main_sm_state;
1031
1032 // fw override wr data status indication
1033 assign fw_ov_wr_fifo_full = fw_ov_mode_entropy_insert &&
1034 (es_bypass_mode ? !
pfifo_bypass_not_full : !pfifo_precon_not_full);
1035
1036 assign hw2reg.fw_ov_wr_fifo_full.d = fw_ov_wr_fifo_full;
1037
1038
1039 //-----
1040 // receive in RNG bus input
1041 //-----
1042
1043 // SEC_CM: FIFO.CTR.REDUN
1044 prim_fifo_sync #(
1045 .Width(RngBusWidth),
1046 .Pass(0),
1047 .Depth(2),
1048 .OutputZeroIfEmpty(0),
1049 .Secure(1)
1050 ) u_prim_fifo_sync_esrng (
1051 .clk_i (clk_i),
1052 .rst_ni (rst_ni),
1053 .clr_i (sfifo_esrng_clr),
1054 .wvalid_i (sfifo_esrng_push),
1055 .wdata_i (sfifo_esrng_wdata),
1056 .wready_o (sfifo_esrng_not_full),
1057 .rvalid_o (sfifo_esrng_not_empty),
1058 .rdata_o (sfifo_esrng_rdata),
1059 .rready_i (sfifo_esrng_pop),

```

```

1060 .full_o (sfifo_esrng_full),
1061 .depth_o (sfifo_esrng_depth),
1062 .err_o (sfifo_esrng_int_err)
1063 );
1064
1065 // fifo controls
1066 // We can't handle any backpressure at this point. Unless
the ENTROPY_SRC block is turned off,
1067 // the input coming from the noise source / RNG needs to be
accepted without dropping samples.
1068 assign sfifo_esrng_push = es_enable_fo[5] &&
es_delayed_enable && entropy_src_rng_valid_i &&
1069 rng_enable_q;
1070
1071 assign sfifo_esrng_clr = ~es_delayed_enable;
1072 assign sfifo_esrng_wdata = entropy_src_rng_bits_i;
1073 // We can't apply any backpressure at this point. Every
sample is presented to the health tests
1074 // for exactly one clock cycle. If the receiving FIFO is
full, the sample is dropped but the
1075 // health tests are still performed and the results accumu-
lated.
1076 assign sfifo_esrng_pop = sfifo_esrng_not_empty;
1077
1078 // fifo err
1079 // The esrng FIFO must never be pushed when it's full as no
backpressure can be applied to the
1080 // noise source / RNG. However, we can't raise an error and
abort operation of the ENTROPY_SRC
1081 // block as this would mean a catastrophic failure of the
whole chip. Instead we need to catch
1082 // this in simulation only.
1083 assign sfifo_esrng_err =
1084 {1'b0,
1085 (sfifo_esrng_pop && !sfifo_esrng_not_empty),
1086 (sfifo_esrng_full && !sfifo_esrng_not_empty)} ||
sfifo_esrng_int_err;
1087 `ASSERT(RngBackpressureNotAllowed_A, sfifo_esrng_push |->
sfifo_esrng_not_full)
1088
1089 // Read the health test data from the esrng FIFO.
1090 assign health_test_esbus = sfifo_esrng_rdata;
1091 // Perform the health tests whenever data is valid and the
receiving FIFO is not being cleared.
1092 // This doesn't mean the receiving FIFO is pushed. The re-
ceiving FIFO is only pushed if it has
1093 // indeed space. This means in case of heavy backpressure,
we keep testing the noise source
1094 // samples, but we drop them before the postht FIFO. If this
happens, the window counter isn't
1095 // incremented. This way we can keep the number of bits go-
ing into the conditioner constant,
1096 // independent of potential backpressure within the
pipeline.
1097 assign health_test_esbus_vld =
1098 rng_bit_en ? sfifo_esrng_not_empty && !pfifo_esbit_clr :
sfifo_esrng_not_empty && !pfifo_postht_clr;
1099
1100 // Health test any data that comes in on the RNG interface.
1101 assign repcnt_active = 1'b1;
1102 assign repcnts_active = 1'b1;
1103 assign adaptp_active = 1'b1;
1104 assign bucket_active = 1'b1;
1105 assign markov_active = 1'b1;
1106 assign extht_active = 1'b1;
1107
1108 // Only reset health tests on re-enable
1109 assign health_test_clr = module_en_pulse_fo[0];
1110
1111 assign health_test_fips_window =
reg2hw.health_test_windows.fips_window.q;
1112 assign health_test_bypass_window =
reg2hw.health_test_windows.bypass_window.q;
1113
1114 // The bypass window size is given in bits. The FIPS window
is given in samples. If we are using
1115
1116 // the single lane mode, the number of samples is equivalent
to the number of bits. If we are
1117 // using the multi-channel mode, we need to multiply with
the noise source width.
1118 assign health_test_window =
1119 es_bypass_mode ? HealthTestWin-
dowWidth'(health_test_bypass_window) :
1120 rng_bit_en ? HealthTestWin-
dowWidth'(health_test_fips_window) :
1121 HealthTestWin-
dowWidth'(health_test_fips_window * RngBusWidth);
1122
1123 // Window sizes other than 384 bits (the seed length) are
currently not tested nor supported in
1124 // bypass or boot-time mode.
1125 `ASSERT(EsBootTestHtWindowSizeSupported_A,
1126 main_sm_enable && es_bypass_mode && !
fw_ov_mode_entropy_insert
1127 |-> health_test_bypass_window == HalfRegWidth'(SeedLen))
1128
1129 //-----
1130 // repcnt one-way threshold
1131 //-----

```

```

1132 assign repcnt_threshold = reg2hw.repcnt_threshold.q;
1133 assign repcnt_threshold_wr = reg2hw.repcnt_threshold.qe;
1134 assign hw2reg.repcnt_threshold.d = repcnt_threshold_oneway;
1135
1136 entropy_src_watermark_reg #(
1137     .RegWidth(HalfRegWidth),
1138     .ResVal({HalfRegWidth{1'b1}})
1139 ) u_entropy_src_watermark_reg_repcnt_thresh (
1140     .clk_i      (clk_i),
1141     .rst_ni     (rst_ni),
1142     .high_i     (1'b0),
1143     .clear_i    (1'b0),
1144     .oneway_i   (threshold_oneway_pfe),
1145     .event_i    (repcnt_threshold_wr),
1146     .value_i    (repcnt_threshold),
1147     .value_o    (repcnt_threshold_oneway)
1148 );
1149
1150 //-----
1151 // repcnts one-way threshold
1152 //-----
1153 assign repcnts_threshold = reg2hw.repcnts_threshold.q;
1154 assign repcnts_threshold_wr = reg2hw.repcnts_threshold.qe;
1155 assign hw2reg.repcnts_threshold.d = rep-
cnts_threshold_oneway;
1156
1157 entropy_src_watermark_reg #(
1158     .RegWidth(HalfRegWidth),
1159     .ResVal({HalfRegWidth{1'b1}})
1160 ) u_entropy_src_watermark_reg_repcnts_thresh (
1161     .clk_i      (clk_i),
1162     .rst_ni     (rst_ni),
1163     .high_i     (1'b0),
1164     .clear_i    (1'b0),
1165     .oneway_i   (threshold_oneway_pfe),
1166     .event_i    (repcnts_threshold_wr),
1167     .value_i    (repcnts_threshold),
1168     .value_o    (repcnts_threshold_oneway)
1169 );
1170
1171 //-----
1172 // adaptp one-way thresholds
1173 //-----
1174 assign adaptp_hi_threshold = reg2hw.adaptp_hi_threshold.q;
1175 assign adaptp_hi_threshold_wr =
reg2hw.adaptp_hi_threshold.qe;
1176 assign hw2reg.adaptp_hi_threshold.d = ad-
aptp_hi_threshold_oneway;
1177
1178 entropy_src_watermark_reg #(
1179     .RegWidth(HalfRegWidth),
1180     .ResVal({HalfRegWidth{1'b1}})
1181 ) u_entropy_src_watermark_reg_adaptp_hi_thresh (
1182     .clk_i      (clk_i),
1183     .rst_ni     (rst_ni),
1184     .high_i     (1'b0),
1185     .clear_i    (1'b0),
1186     .oneway_i   (threshold_oneway_pfe),
1187     .event_i    (adaptp_hi_threshold_wr),
1188     .value_i    (adaptp_hi_threshold),
1189     .value_o    (adaptp_hi_threshold_oneway)
1190 );
1191
1192 assign adaptp_lo_threshold = reg2hw.adaptp_lo_threshold.q;
1193 assign adaptp_lo_threshold_wr =
reg2hw.adaptp_lo_threshold.qe;
1194 assign hw2reg.adaptp_lo_threshold.d = ad-
aptp_lo_threshold_oneway;
1195
1196 entropy_src_watermark_reg #(
1197     .RegWidth(HalfRegWidth),
1198     .ResVal('0)
1199 ) u_entropy_src_watermark_reg_adaptp_lo_thresh (
1200     .clk_i      (clk_i),
1201     .rst_ni     (rst_ni),
1202     .high_i     (1'b1),
1203     .clear_i    (1'b0),
1204     .oneway_i   (threshold_oneway_pfe),
1205     .event_i    (adaptp_lo_threshold_wr),
1206     .value_i    (adaptp_lo_threshold),
1207     .value_o    (adaptp_lo_threshold_oneway)
1208 );
1209
1210 //-----
1211 // bucket one-way threshold
1212 //-----
1213 assign bucket_threshold = reg2hw.bucket_threshold.q;
1214 assign bucket_threshold_wr = reg2hw.bucket_threshold.qe;
1215 assign hw2reg.bucket_threshold.d = bucket_threshold_oneway;
1216
1217 entropy_src_watermark_reg #(
1218     .RegWidth(HalfRegWidth),
1219     .ResVal({HalfRegWidth{1'b1}})
1220 ) u_entropy_src_watermark_reg_bucket_thresh (
1221     .clk_i      (clk_i),
1222     .rst_ni     (rst_ni),
1223     .high_i     (1'b0),
1224     .clear_i    (1'b0),
1225     .oneway_i   (threshold_oneway_pfe),
1226     .event_i    (bucket_threshold_wr),
1227     .value_i    (bucket_threshold),
1228     .value_o    (bucket_threshold_oneway)
1229 );
1230
1231 //-----
1232 // markov one-way thresholds
1233 //-----
1234 assign markov_hi_threshold = reg2hw.markov_hi_threshold.q;
1235 assign markov_hi_threshold_wr =
reg2hw.markov_hi_threshold.qe;
1236 assign hw2reg.markov_hi_threshold.d =
markov_hi_threshold_oneway;
1237
1238 entropy_src_watermark_reg #(
1239     .RegWidth(HalfRegWidth),
1240     .ResVal({HalfRegWidth{1'b1}})
1241 ) u_entropy_src_watermark_reg_markov_hi_thresh (
1242     .clk_i      (clk_i),
1243     .rst_ni     (rst_ni),
1244     .high_i     (1'b0),
1245     .clear_i    (1'b0),
1246     .oneway_i   (threshold_oneway_pfe),
1247     .event_i    (markov_hi_threshold_wr),
1248     .value_i    (markov_hi_threshold),
1249     .value_o    (markov_hi_threshold_oneway)
1250 );
1251
1252 assign markov_lo_threshold = reg2hw.markov_lo_threshold.q;
1253 assign markov_lo_threshold_wr =
reg2hw.markov_lo_threshold.qe;
1254 assign hw2reg.markov_lo_threshold.d =
markov_lo_threshold_oneway;
1255
1256 entropy_src_watermark_reg #(
1257     .RegWidth(HalfRegWidth),
1258     .ResVal('0)
1259 ) u_entropy_src_watermark_reg_markov_lo_thresh (
1260     .clk_i      (clk_i),
1261     .rst_ni     (rst_ni),
1262     .high_i     (1'b1),
1263     .clear_i    (1'b0),
1264     .oneway_i   (threshold_oneway_pfe),
1265     .event_i    (markov_lo_threshold_wr),
1266     .value_i    (markov_lo_threshold),
1267     .value_o    (markov_lo_threshold_oneway)
1268 );
1269
1270 //-----
1271 // extht one-way thresholds
1272 //-----
1273 assign extht_hi_threshold = reg2hw.extht_hi_threshold.q;
1274 assign extht_hi_threshold_wr = reg2hw.extht_hi_threshold.qe;
1275 assign hw2reg.extht_hi_threshold.d =
extht_hi_threshold_oneway;
1276
1277 entropy_src_watermark_reg #(
1278     .RegWidth(HalfRegWidth),
1279     .ResVal({HalfRegWidth{1'b1}})
1280 ) u_entropy_src_watermark_reg_extht_hi_thresh (
1281     .clk_i      (clk_i),
1282     .rst_ni     (rst_ni),
1283     .high_i     (1'b0),
1284     .clear_i    (1'b0),
1285     .oneway_i   (threshold_oneway_pfe),
1286     .event_i    (extht_hi_threshold_wr),
1287     .value_i    (extht_hi_threshold),
1288     .value_o    (extht_hi_threshold_oneway)
1289 );
1290
1291 assign extht_lo_threshold = reg2hw.extht_lo_threshold.q;
1292 assign extht_lo_threshold_wr = reg2hw.extht_lo_threshold.qe;
1293 assign hw2reg.extht_lo_threshold.d =
extht_lo_threshold_oneway;
1294
1295 entropy_src_watermark_reg #(
1296     .RegWidth(HalfRegWidth),
1297     .ResVal('0)
1298 ) u_entropy_src_watermark_reg_extht_lo_thresh (
1299     .clk_i      (clk_i),
1300     .rst_ni     (rst_ni),
1301     .high_i     (1'b1),
1302     .clear_i    (1'b0),
1303     .oneway_i   (threshold_oneway_pfe),
1304     .event_i    (extht_lo_threshold_wr),
1305     .value_i    (extht_lo_threshold),
1306     .value_o    (extht_lo_threshold_oneway)
1307 );
1308
1309 //-----
1310 // misc control settings
1311 //-----
1312
1313 assign event_es_health_test_failed = es_main_sm_alert;
1314
1315 assign event_es_observe_fifo_ready = ob-
serve_fifo_thresh_met;
1316
1317 // SEC_CM: CONFIG.MUBI
1318 mubi4_t mubi_es_route;
1319 mubi4_t [1:0] mubi_es_route_fanout;
1320 assign mubi_es_route = mu-
bi4_t'(reg2hw.entropy_control.es_route.q);

```

```

1321 assign es_route_pfe = mu-
bi4_test_true_strict(mubi_es_route_fanout[0]);
1322 assign es_route_pfa = mu-
bi4_test_invalid(mubi_es_route_fanout[1]);
1323 assign hw2reg.recov_alert_sts.es_route_field_alert.de =
es_route_pfa;
1324 assign hw2reg.recov_alert_sts.es_route_field_alert.d =
es_route_pfa;
1325
1326 prim_mubi4_sync #(
1327   .NumCopies(2),
1328   .AsyncOn(0)
1329 ) u_prim_mubi4_sync_es_route (
1330   .clk_i,
1331   .rst_ni,
1332   .mubi_i(mubi_es_route),
1333   .mubi_o(mubi_es_route_fanout)
1334 );
1335
1336 // SEC_CM: CONFIG.MUBI
1337 mubi4_t mubi_es_type;
1338 mubi4_t [1:0] mubi_es_type_fanout;
1339 assign mubi_es_type = mu-
bi4_t'(reg2hw.entropy_control.es_type.q);
1340 assign es_type_pfe = mu-
bi4_test_true_strict(mubi_es_type_fanout[0]);
1341 assign es_type_pfa = mu-
bi4_test_invalid(mubi_es_type_fanout[1]);
1342 assign hw2reg.recov_alert_sts.es_type_field_alert.de =
es_type_pfa;
1343 assign hw2reg.recov_alert_sts.es_type_field_alert.d =
es_type_pfa;
1344
1345 prim_mubi4_sync #(
1346   .NumCopies(2),
1347   .AsyncOn(0)
1348 ) u_prim_mubi4_sync_es_type (
1349   .clk_i,
1350   .rst_ni,
1351   .mubi_i(mubi_es_type),
1352   .mubi_o(mubi_es_type_fanout)
1353 );
1354
1355 // SEC_CM: CONFIG.MUBI
1356 mubi4_t mubi_thresh_scope;
1357 assign mubi_thresh_scope = mu-
bi4_t'(reg2hw.conf.threshold_scope.q);
1358 assign threshold_scope_pfe = mu-
bi4_test_true_strict(mubi_thresh_scope);
1359 assign threshold_scope_pfa = mu-
bi4_test_invalid(mubi_thresh_scope);
1360 assign hw2reg.recov_alert_sts.threshold_scope_field_alert.de =
threshold_scope_pfa;
1361 assign hw2reg.recov_alert_sts.threshold_scope_field_alert.d =
threshold_scope_pfa;
1362
1363 assign es_route_to_sw = es_route_pfe;
1364 assign es_bypass_to_sw = es_type_pfe;
1365 assign threshold_scope = threshold_scope_pfe;
1366
1367 // The es_bypass_mode signal determines whether the condi-
tioner is bypassed or not.
1368 // The conditioner can be bypassed by either disabling
fips_enable_pfe or by enabling
1369 // both es_bypass_to_sw and es_route_to_sw.
1370 // The combination of es_bypass_to_sw and es_route_to_sw al-
lows for four distinct cases:
1371 //
1372 // | es_bypass_to_sw | es_route_to_sw |
1373 // |-----|-----|
1374 // | 0 | 0 | In this case the en-
1375 // | 0 | 0 | trophy passes through the conditioner
1376 // | 0 | 0 | and the entropy is
1377 // | 0 | 0 | forwarded to the HW endpoints.
1378 // | 0 | 1 | In this case the en-
1379 // | 0 | 1 | trophy passes through the conditioner
1380 // | 0 | 1 | and the entropy is
1381 // | 0 | 1 | forwarded to software.
1382 // | 1 | 0 | In this case nothing
1383 // | 1 | 0 | happens and whether the conditioner
1384 // | 1 | 0 | depends on fips_enable_pfe.
1385 // | 1 | 1 | In this case the
1386 // | 1 | 1 | conditioner is bypassed and the entropy
1387 // | 1 | 1 | is forwarded to
1388 // | 1 | 1 | software.
1389
1390 assign es_bypass_mode = (!fips_enable_pfe) || (es_by-
pass_to_sw && es_route_to_sw);
1391
1392 // send off to AST RNG for possibly faster entropy genera-
tion
1393 assign rng_fips_o = rng_fips_pfe;

```

```

1392
1393 //-----
1394 // common health test window counter
1395 //-----
1396
1397 // Window counter
1398 // SEC_CM: CTR.REDUN
1399
1400 // We only increment the counter if the currently tested
sample can be pushed to the correct FIFO
1401 // following the health tests. This is required to keep the
number of samples passed to the
1402 // conditioner constant also when experiencing backpressure
from the conditioner. At the same
1403 // time, we're not allowed to drop samples before the health
testing. This means the number of
1404 // tested samples might be slightly bigger than the number
of samples fed into the conditioner.
1405 assign window_cntr_incr_en =
1406   rng_bit_en ? pfifo_esbit_push && pfifo_esbit_not_full
&& !pfifo_esbit_clr :
1407   pfifo_postht_push && pfifo_postht_not_full
&& !pfifo_postht_clr;
1408
1409 // When in single lane mode, only increment with 1-bit, oth-
erwise use the full noise source width
1410 assign window_cntr_step = unsigned'(rng_bit_en ? HealthT-
estWindowWidth'(1) :
1411   HealthT-
estWindowWidth'(RngBusWidth));
1412
1413 prim_count #(
1414   .Width(HealthTestWindowWidth)
1415 ) u_prim_count_window_cntr (
1416   .clk_i,
1417   .rst_ni,
1418   .clr_i(!es_delayed_enable),
1419   .set_i(health_test_done_pulse),
1420   .set_cnt_i(HealthTestWindowWidth'(0)),
1421   .incr_en_i(window_cntr_incr_en),
1422   .decr_en_i(1'b0),
1423   .step_i(window_cntr_step),
1424   .commit_i(1'b1),
1425   .cnt_o(window_cntr),
1426   .cnt_after_commit_o(),
1427   .err_o(window_cntr_err)
1428 );
1429
1430 // Window wrap condition
1431 // The counter is incremented with every tested symbol and
when we've tested sufficiently many
1432 // symbols, we decide whether the current window passed or
failed the tests. Using the
1433 // health_test_done_pulse, we then restart the counter, we
update the watermark registers, and
1434 // we clear the window-based health tests to get ready for
the next window. This means, we can't
1435 // actually test another symbol in this cycle, i.e., the
maximum rate of the noise source is
1436 // limited to one symbol every two clock cycles.
1437 assign health_test_done_pulse = (window_cntr >=
health_test_window);
1438
1439 // Summary of counter errors
1440 assign es_cntr_err =
1441   (window_cntr_err ||
1442     repcnt_cntr_err ||
1443     repcnts_cntr_err ||
1444     adaptp_cntr_err ||
1445     (|bucket_cntr_err) ||
1446     markov_cntr_err ||
1447     repcnt_fails_cntr_err ||
1448     repcnt_alert_cntr_err ||
1449     repcnts_fails_cntr_err ||
1450     repcnts_alert_cntr_err ||
1451     adaptp_hi_fails_cntr_err ||
1452     adaptp_lo_fails_cntr_err ||
1453     adaptp_hi_alert_cntr_err ||
1454     adaptp_lo_alert_cntr_err ||
1455     bucket_fails_cntr_err ||
1456     bucket_alert_cntr_err ||
1457     markov_hi_fails_cntr_err ||
1458     markov_lo_fails_cntr_err ||
1459     markov_hi_alert_cntr_err ||
1460     markov_lo_alert_cntr_err ||
1461     extht_hi_fails_cntr_err ||
1462     extht_lo_fails_cntr_err ||
1463     extht_hi_alert_cntr_err ||
1464     extht_lo_alert_cntr_err ||
1465     any_fails_cntr_err ||
1466     pipeline_depth_cntr_err ||
1467     sha3_count_error);
1468
1469 //-----
1470 // repetitive count test
1471 //-----
1472
1473 // SEC_CM: RNG.BKGN_CHK
1474 entropy_src_repcnt_ht #(
1475   .RegWidth(HalfRegWidth),

```

```

1476 .RngBusBitSelWidth(RngBusBitSelWidth),
1477 .RngBusWidth(RngBusWidth)
1478 ) u_entropy_src_repcnt_ht (
1479   .clk_i      (clk_i),
1480   .rst_ni     (rst_ni),
1481   .entropy_bit_i (health_test_esbus),
1482   .entropy_bit_vld_i (health_test_esbus_vld),
1483   .rng_bit_en_i (rng_bit_en),
1484   .rng_bit_sel_i (rng_bit_sel),
1485   .clear_i     (health_test_clr),
1486   .active_i    (repcnt_active),
1487   .thresh_i    (repcnt_threshold_oneway),
1488   .test_cnt_o  (repcnt_event_cnt),
1489   .test_fail_pulse_o (repcnt_fail_pulse),
1490   .count_err_o (repcnt_cntr_err)
1491 );
1492
1493 // SEC_CM: CTR.REDUN
1494 entropy_src_cntr_reg #(
1495   .RegWidth(FullRegWidth)
1496 ) u_entropy_src_cntr_reg_repcnt (
1497   .clk_i      (clk_i),
1498   .rst_ni     (rst_ni),
1499   .clear_i    (health_test_clr),
1500   .event_i    (repcnt_fail_pulse),
1501   .step_i     (FullRegWidth'(1)),
1502   .value_o    (repcnt_total_fails),
1503   .err_o      (repcnt_fails_cntr_err)
1504 );
1505
1506 assign hw2reg.repcnt_total_fails.d = repcnt_total_fails;
1507
1508 //-----
1509 // repetitive count symbol test
1510 //-----
1511
1512 // SEC_CM: RNG.BKGN_CHK
1513 entropy_src_repcnts_ht #(
1514   .RegWidth(HalfRegWidth),
1515   .RngBusWidth(RngBusWidth)
1516 ) u_entropy_src_repcnts_ht (
1517   .clk_i      (clk_i),
1518   .rst_ni     (rst_ni),
1519   .entropy_bit_i (health_test_esbus),
1520   .entropy_bit_vld_i (health_test_esbus_vld),
1521   .clear_i     (health_test_clr),
1522   .active_i    (repcnts_active),
1523   .thresh_i    (repcnts_threshold_oneway),
1524   .test_cnt_o  (repcnts_event_cnt),
1525   .test_fail_pulse_o (repcnts_fail_pulse),
1526   .count_err_o (repcnts_cntr_err)
1527 );
1528
1529 // SEC_CM: CTR.REDUN
1530 entropy_src_cntr_reg #(
1531   .RegWidth(FullRegWidth)
1532 ) u_entropy_src_cntr_reg_repcnts (
1533   .clk_i      (clk_i),
1534   .rst_ni     (rst_ni),
1535   .clear_i    (health_test_clr),
1536   .event_i    (repcnts_fail_pulse),
1537   .step_i     (FullRegWidth'(1)),
1538   .value_o    (repcnts_total_fails),
1539   .err_o      (repcnts_fails_cntr_err)
1540 );
1541
1542 assign hw2reg.repcnts_total_fails.d = repcnts_total_fails;
1543
1544 //-----
1545 // Adaptive Proportion Test
1546 //-----
1547
1548 // SEC_CM: RNG.BKGN_CHK
1549 entropy_src_adaptpt_ht #(
1550   .RegWidth(HalfRegWidth),
1551   .RngBusWidth(RngBusWidth),
1552   .RngBusBitSelWidth(RngBusBitSelWidth)
1553 ) u_entropy_src_adaptpt_ht (
1554   .clk_i      (clk_i),
1555   .rst_ni     (rst_ni),
1556   .entropy_bit_i (health_test_esbus),
1557   .entropy_bit_vld_i (health_test_esbus_vld),
1558   .rng_bit_en_i (rng_bit_en),
1559   .rng_bit_sel_i (rng_bit_sel),
1560   .clear_i     (health_test_clr),
1561   .active_i    (adaptpt_active),
1562   .thresh_hi_i (adaptpt_hi_threshold_oneway),
1563   .thresh_lo_i (adaptpt_lo_threshold_oneway),
1564   .window_wrap_pulse_i (health_test_done_pulse),
1565   .threshold_scope_i (threshold_scope),
1566   .test_cnt_hi_o (adaptpt_hi_event_cnt),
1567   .test_cnt_lo_o (adaptpt_lo_event_cnt),
1568   .test_fail_hi_pulse_o (adaptpt_hi_fail_pulse),
1569   .test_fail_lo_pulse_o (adaptpt_lo_fail_pulse),
1570   .count_err_o  (adaptpt_cntr_err)
1571 );
1572
1573 // SEC_CM: CTR.REDUN
1574 entropy_src_cntr_reg #(
1575   .RegWidth(FullRegWidth)
1576 ) u_entropy_src_cntr_reg_adaptpt_hi (
1577   .clk_i      (clk_i),
1578   .rst_ni     (rst_ni),
1579   .clear_i    (health_test_clr),
1580   .event_i    (adaptpt_hi_fail_pulse),
1581   .step_i     (FullRegWidth'(1)),
1582   .value_o    (adaptpt_hi_total_fails),
1583   .err_o      (adaptpt_hi_fails_cntr_err)
1584 );
1585
1586 assign hw2reg.adaptpt_hi_total_fails.d = adaptpt_hi_total_fails;
1587
1588 // SEC_CM: CTR.REDUN
1589 entropy_src_cntr_reg #(
1590   .RegWidth(FullRegWidth)
1591 ) u_entropy_src_cntr_reg_adaptpt_lo (
1592   .clk_i      (clk_i),
1593   .rst_ni     (rst_ni),
1594   .clear_i    (health_test_clr),
1595   .event_i    (adaptpt_lo_fail_pulse),
1596   .step_i     (FullRegWidth'(1)),
1597   .value_o    (adaptpt_lo_total_fails),
1598   .err_o      (adaptpt_lo_fails_cntr_err)
1599 );
1600
1601 assign hw2reg.adaptpt_lo_total_fails.d = adaptpt_lo_total_fails;
1602
1603 //-----
1604 // bucket test
1605 //-----
1606
1607 for (genvar i = 0; i < NumBucketHtInst; i++) begin :
gen_health_test
1608   // SEC_CM: RNG.BKGN_CHK
1609   entropy_src_bucket_ht #(
1610     .RegWidth(HalfRegWidth),
1611     .RngBusWidth(BucketHtDataWidth)
1612   ) u_entropy_src_bucket_ht (
1613     .clk_i      (clk_i),
1614     .rst_ni     (rst_ni),
1615     .entropy_bit_i (health_test_esbus[i*BucketHtDataWidth+BucketHtDataWidth]),
1616     .entropy_bit_vld_i (health_test_esbus_vld),
1617     .clear_i     (health_test_clr),
1618     .active_i    (bucket_active),
1619     .thresh_i    (bucket_threshold_oneway),
1620     .window_wrap_pulse_i (health_test_done_pulse),
1621     .test_cnt_o  (bucket_event_cnt[i]),
1622     .test_fail_pulse_o (bucket_fail_pulse[i]),
1623     .count_err_o (bucket_cntr_err[i])
1624   );
1625 end
1626
1627 // Extract the maximum bin counter value from all NumBucketHtInst groups of BucketHtDataWidth
1628 // channels.
1629 if (NumBucketHtInst > 1) begin : gen_bucket_max_tree
1630   prim_max_tree #(
1631     .NumSrc(NumBucketHtInst),
1632     .Width(HalfRegWidth)
1633   ) u_prim_max_tree_bin_cntr_max (
1634     .clk_i,
1635     .rst_ni,
1636     .values_i (bucket_event_cnt),
1637     .valid_i  ({NumBucketHtInst{1'b1}}),
1638     .max_value_o (bucket_event_cnt_max),
1639     .max_idx_o  (),
1640     .max_valid_o()
1641   );
1642 end else begin : gen_no_bucket_max_tree
1643   assign bucket_event_cnt_max = bucket_event_cnt[0];
1644 end
1645
1646 // Add the failure pulses from all NumBucketHtInst groups.
1647 always_comb begin
1648   bucket_fail_pulse_step = 0;
1649   for (int i = 0; i < NumBucketHtInst; i++) begin
1650     bucket_fail_pulse_step += bucket_fail_pulse[i];
1651   end
1652 end
1653
1654 // SEC_CM: CTR.REDUN
1655 entropy_src_cntr_reg #(
1656   .RegWidth(FullRegWidth)
1657 ) u_entropy_src_cntr_reg_bucket (
1658   .clk_i      (clk_i),
1659   .rst_ni     (rst_ni),
1660   .clear_i    (health_test_clr),
1661   .event_i    (bucket_fail_pulse),
1662   .step_i     (FullReg-
Width'(bucket_fail_pulse_step)),
1663   .value_o    (bucket_total_fails),
1664   .err_o      (bucket_fails_cntr_err)
1665 );
1666
1667 assign hw2reg.bucket_total_fails.d = bucket_total_fails;
1668
1669 //-----
1670 // Markov test
1671 //-----

```



```

1672
1673 // SEC_CM: RNG.BKGN_CHK
1674 entropy_src_markov_ht #(
1675   .RegWidth(HalfRegWidth),
1676   .RngBusWidth(RngBusWidth),
1677   .RngBusBitSelWidth(RngBusBitSelWidth)
1678 ) u_entropy_src_markov_ht (
1679   .clk_i      (clk_i),
1680   .rst_ni     (rst_ni),
1681   .entropy_bit_i (health_test_esbus),
1682   .entropy_bit_vld_i (health_test_esbus_vld),
1683   .rng_bit_en_i (rng_bit_en),
1684   .rng_bit_sel_i (rng_bit_sel),
1685   .clear_i      (health_test_clr),
1686   .active_i      (markov_active),
1687   .thresh_hi_i  (markov_hi_threshold_oneway),
1688   .thresh_lo_i  (markov_lo_threshold_oneway),
1689   .window_wrap_pulse_i (health_test_done_pulse),
1690   .threshold_scope_i (threshold_scope),
1691   .test_cnt_hi_o (markov_hi_event_cnt),
1692   .test_cnt_lo_o (markov_lo_event_cnt),
1693   .test_fail_hi_pulse_o (markov_hi_fail_pulse),
1694   .test_fail_lo_pulse_o (markov_lo_fail_pulse),
1695   .count_err_o   (markov_cntr_err)
1696 );
1697
1698 // SEC_CM: CTR.REDUN
1699 entropy_src_cntr_reg #(
1700   .RegWidth(FullRegWidth)
1701 ) u_entropy_src_cntr_reg_markov_hi (
1702   .clk_i      (clk_i),
1703   .rst_ni     (rst_ni),
1704   .clear_i     (health_test_clr),
1705   .event_i     (markov_hi_fail_pulse),
1706   .step_i      (FullRegWidth'(1)),
1707   .value_o     (markov_hi_total_fails),
1708   .err_o       (markov_hi_fails_cntr_err)
1709 );
1710
1711 assign hw2reg.markov_hi_total_fails.d =
markov_hi_total_fails;
1712
1713 // SEC_CM: CTR.REDUN
1714 entropy_src_cntr_reg #(
1715   .RegWidth(FullRegWidth)
1716 ) u_entropy_src_cntr_reg_markov_lo (
1717   .clk_i      (clk_i),
1718   .rst_ni     (rst_ni),
1719   .clear_i     (health_test_clr),
1720   .event_i     (markov_lo_fail_pulse),
1721   .step_i      (FullRegWidth'(1)),
1722   .value_o     (markov_lo_total_fails),
1723   .err_o       (markov_lo_fails_cntr_err)
1724 );
1725
1726 assign hw2reg.markov_lo_total_fails.d =
markov_lo_total_fails;
1727
1728 //-----
1729 // External health test
1730 //-----
1731
1732 // set outputs to external health test
1733 assign entropy_src_xht_valid_o = health_test_esbus_vld;
1734 assign entropy_src_xht_bits_o = health_test_esbus;
1735 assign entropy_src_xht_bit_sel_o = rng_bit_sel;
1736 assign entropy_src_xht_health_test_window_o =
health_test_window;
1737 assign entropy_src_xht_meta_o.rng_bit_en = rng_bit_en;
1738 assign entropy_src_xht_meta_o.clear = health_test_clr;
1739 assign entropy_src_xht_meta_o.active = extht_active;
1740 assign entropy_src_xht_meta_o.thresh_hi =
extht_hi_threshold_oneway;
1741 assign entropy_src_xht_meta_o.thresh_lo =
extht_lo_threshold_oneway;
1742 assign entropy_src_xht_meta_o.window_wrap_pulse =
health_test_done_pulse;
1743 assign entropy_src_xht_meta_o.threshold_scope =
threshold_scope;
1744 // get inputs from external health test
1745 assign extht_event_cnt_hi = en-
tropy_src_xht_meta_i.test_cnt_hi;
1746 assign extht_event_cnt_lo = en-
tropy_src_xht_meta_i.test_cnt_lo;
1747 assign extht_hi_fail_pulse = en-
tropy_src_xht_meta_i.test_fail_hi_pulse;
1748 assign extht_lo_fail_pulse = en-
tropy_src_xht_meta_i.test_fail_lo_pulse;
1749 assign extht_cont_test = en-
tropy_src_xht_meta_i.continuous_test;
1750
1751 // SEC_CM: CTR.REDUN
1752 entropy_src_cntr_reg #(
1753   .RegWidth(FullRegWidth)
1754 ) u_entropy_src_cntr_reg_extht_hi (
1755   .clk_i      (clk_i),
1756   .rst_ni     (rst_ni),
1757   .clear_i     (health_test_clr),
1758   .event_i     (extht_hi_fail_pulse),
1759   .step_i      (FullRegWidth'(1)),
1760   .value_o     (extht_hi_total_fails),
1761   .err_o       (extht_hi_fails_cntr_err)
1762 );
1763
1764 assign hw2reg.extht_hi_total_fails.d = extht_hi_total_fails;
1765
1766 // SEC_CM: CTR.REDUN
1767 entropy_src_cntr_reg #(
1768   .RegWidth(FullRegWidth)
1769 ) u_entropy_src_cntr_reg_extht_lo (
1770   .clk_i      (clk_i),
1771   .rst_ni     (rst_ni),
1772   .clear_i     (health_test_clr),
1773   .step_i      (FullRegWidth'(1)),
1774   .event_i     (extht_lo_fail_pulse),
1775   .value_o     (extht_lo_total_fails),
1776   .err_o       (extht_lo_fails_cntr_err)
1777 );
1778
1779 assign hw2reg.extht_lo_total_fails.d = extht_lo_total_fails;
1780
1781 //-----
1782 // watermark register
1783 //-----
1784
1785 // Get and resolve HT_WATERMARK_NUM values from register in-
terface.
1786 assign ht_watermark_num_reg_if = ht_water-
mark_num_e'(reg2hw.ht_watermark_num.q);
1787 always_comb begin
1788   unique case (ht_watermark_num_reg_if)
1789     REPCNT_HI,
1790     REPCNTS_HI,
1791     ADAPTP_HI,
1792     ADAPTP_LO,
1793     BUCKET_HI,
1794     MARKOV_HI,
1795     MARKOV_LO,
1796     EXTHT_HI,
1797     EXTHT_LO: ht_watermark_num_d = ht_watermark_num_reg_if;
1798     default: ht_watermark_num_d = REPCNT_HI; // Unsuppor-
ted values are mapped to REPCNT_HI.
1799   endcase
1800 end
1801
1802 // Write the resolved value to the actual register.
1803 always_ff @(posedge clk_i or negedge rst_ni) begin
1804   if (!rst_ni) begin
1805     ht_watermark_num_q <= REPCNT_HI;
1806   end else if (reg2hw.ht_watermark_num.qe) begin
1807     ht_watermark_num_q <= ht_watermark_num_d;
1808   end
1809 end
1810 assign hw2reg.ht_watermark_num.d = ht_watermark_num_q;
1811
1812 // Select the health test for which we want to record the
watermark based on the HT_WATERMARK_NUM
1813 // register.
1814 always_comb begin
1815   unique case (ht_watermark_num_q)
1816     REPCNT_HI: begin
1817       ht_watermark_high = 1'b1;
1818       ht_watermark_event_pre = 1'b1; // continuous
1819       ht_watermark_cnt = repcnt_event_cnt;
1820     end
1821     REPCNTS_HI: begin
1822       ht_watermark_high = 1'b1;
1823       ht_watermark_event_pre = 1'b1; // continuous
1824       ht_watermark_cnt = repcnts_event_cnt;
1825     end
1826     ADAPTP_HI: begin
1827       ht_watermark_high = 1'b1;
1828       ht_watermark_event_pre = health_test_done_pulse;
1829       ht_watermark_cnt = adaptp_hi_event_cnt;
1830     end
1831     ADAPTP_LO: begin
1832       ht_watermark_high = 1'b0;
1833       ht_watermark_event_pre = health_test_done_pulse;
1834       ht_watermark_cnt = adaptp_lo_event_cnt;
1835     end
1836     BUCKET_HI: begin
1837       ht_watermark_high = 1'b1;
1838       ht_watermark_event_pre = health_test_done_pulse;
1839       ht_watermark_cnt = bucket_event_cnt_max;
1840     end
1841     MARKOV_HI: begin
1842       ht_watermark_high = 1'b1;
1843       ht_watermark_event_pre = health_test_done_pulse;
1844       ht_watermark_cnt = markov_hi_event_cnt;
1845     end
1846     MARKOV_LO: begin
1847       ht_watermark_high = 1'b0;
1848       ht_watermark_event_pre = health_test_done_pulse;
1849       ht_watermark_cnt = markov_lo_event_cnt;
1850     end
1851     EXTHT_HI: begin
1852       ht_watermark_high = 1'b1;
1853       ht_watermark_event_pre = extht_cont_test ||
health_test_done_pulse;
1854       ht_watermark_cnt = extht_event_cnt_hi;
1855     end
1856     EXTHT_LO: begin

```

```

1857     ht_watermark_high      = 1'b0;
1858     ht_watermark_event_pre = extht_cont_test ||
health_test_done_pulse;
1859     ht_watermark_cnt        = extht_event_cnt_lo;
1860     end
1861     default: begin // Unsupported values are mapped to REP-
CNT_HI.
1862         ht_watermark_high      = 1'b1;
1863         ht_watermark_event_pre = 1'b1; // continuous
1864         ht_watermark_cnt        = repcnt_event_cnt;
1865     end
1866     endcase
1867     end
1868
1869     // Prevent watermark register updates while the module dis-
abled. Upon enabling, we then clear
1870     // the watermark register before we start recording.
1871     assign ht_watermark_event = ht_watermark_event_pre &&
es_delayed_enable;
1872
1873     entropy_src_watermark_reg #(
1874         .RegWidth(HalfRegWidth),
1875         .ResVal('0)
1876     ) u_entropy_src_ht_watermark_reg (
1877         .clk_i   (clk_i),
1878         .rst_ni  (rst_ni),
1879         .high_i  (ht_watermark_high),
1880         .clear_i (health_test_clr),
1881         .oneway_i(1'b1), // For the watermark register, the one-
way behavior is always enabled. Upon
1882         // enabling, the register is then cleared
according to ht_watermark_high.
1883         .event_i (ht_watermark_event),
1884         .value_i (ht_watermark_cnt),
1885         .value_o (ht_watermark)
1886     );
1887
1888     assign hw2reg.ht_watermark.d = ht_watermark;
1889
1890     //-----
1891     // summary and alert registers
1892     //-----
1893
1894     // We clear the alert counters when clearing the health
tests (upon module enable), and...
1895     assign alert_cntrs_clr = health_test_clr ||
1896     // ...whenever a health test window ends without a fail-
ure (and when the main state machine
1897     // actually allows the clearing).
1898     (health_test_done_pulse && !ht_failed_d &&
alert_cntr_clr_ok);
1899
1900     // SEC_CM: CTR.REDUN
1901     entropy_src_cntr_reg #(
1902         .RegWidth(HalfRegWidth)
1903     ) u_entropy_src_cntr_reg_any_alert_fails (
1904         .clk_i   (clk_i),
1905         .rst_ni  (rst_ni),
1906         .clear_i (alert_cntrs_clr),
1907         .event_i (any_fail_pulse),
1908         .step_i  (HalfRegWidth'1)),
1909         .value_o (any_fail_count),
1910         .err_o   (any_fails_cntr_err)
1911     );
1912
1913     assign any_fail_pulse =
1914     repcnt_fail_pulse ||
1915     repcnts_fail_pulse ||
1916     adaptp_hi_fail_pulse || adaptp_lo_fail_pulse ||
1917     (|bucket_fail_pulse) ||
1918     markov_hi_fail_pulse || markov_lo_fail_pulse ||
1919     extht_hi_fail_pulse || extht_lo_fail_pulse;
1920
1921     // The failure pulses of the window-based health tests are
aligned with the
1922     // health_test_done_pulse. In contrast, the continous health
tests (repcnt and repcnts) can
1923     // signal failures at any point in time and we have to latch
these.
1924     assign ht_failed_d =
1925     // The ht_failed_d/q/qq pulses get consumed together
with ht_done_pulse_d/q/qq. Hence, we
1926     // clear them together.
1927     ht_done_pulse_q ? 1'b0 :
1928     // Latch continuous health test failures which can
happen at any point point in a window.
1929     any_fail_pulse ? 1'b1 :
1930     // Eventually clear when disabling. Note, it may be
possible that the last symbol tested
1931     // before disabling triggers a failure. We want to
record this failure. Thus, the clearing
1932     // has the lowest priority and the delayed enable
signal is used as well.
1933     !(es_enable_fo[6] || es_delayed_enable) ? 1'b0 :
1934     ht_failed_q;
1935
1936
1937     // delay health pulse so that main_sm will

```

```

1938     // get the correct threshold value comparisons
1939     assign ht_done_pulse_d = health_test_done_pulse;
1940
1941     assign hw2reg.alert_summary_fail_counts.d = any_fail_count;
1942
1943     // signal an alert
1944     // SEC_CM: CONFIG.REDUN
1945     assign alert_threshold =
reg2hw.alert_threshold.alert_threshold.q;
1946     assign alert_threshold_inv =
reg2hw.alert_threshold.alert_threshold_inv.q;
1947     assign es_thresh_cfg_alert = (~alert_threshold_inv !=
alert_threshold);
1948
1949     assign alert_threshold_fail =
1950     ((any_fail_count >= ~alert_threshold_inv) &&
(~alert_threshold_inv != '0)) ||
1951     (any_fail_count >= alert_threshold) &&
(alert_threshold != '0);
1952
1953
1954     prim_edge_detector #(
1955         .Width(1),
1956         .ResetValue(0),
1957         .EnSync(0)
1958     ) u_prim_edge_detector_recov_alert (
1959         .clk_i,
1960         .rst_ni,
1961         .d_i(recov_alert_state),
1962         .q_sync_o(),
1963         .q_posedge_pulse_o(recov_alert_o),
1964         .q_negedge_pulse_o()
1965     );
1966
1967     assign recov_alert_state =
1968     es_enable_pfa ||
1969     fips_enable_pfa ||
1970     fips_flag_pfa ||
1971     rng_fips_pfa ||
1972     entropy_data_reg_en_pfa ||
1973     threshold_scope_pfa ||
1974     rng_bit_enable_pfa ||
1975     fw_ov_mode_pfa ||
1976     fw_ov_entropy_insert_pfa ||
1977     fw_ov_sha3_start_pfa ||
1978     es_route_pfa ||
1979     es_type_pfa ||
1980     es_main_sm_alert ||
1981     es_bus_cmp_alert ||
1982     es_thresh_cfg_alert ||
1983     es_fw_ov_wr_alert ||
1984     es_fw_ov_disable_alert ||
1985     postht_entropy_drop_alert;
1986
1987     assign hw2reg.recov_alert_sts.es_main_sm_alert.de =
es_main_sm_alert;
1988     assign hw2reg.recov_alert_sts.es_main_sm_alert.d =
es_main_sm_alert;
1989
1990     assign hw2reg.recov_alert_sts.es_bus_cmp_alert.de =
es_bus_cmp_alert;
1991     assign hw2reg.recov_alert_sts.es_bus_cmp_alert.d =
es_bus_cmp_alert;
1992
1993     assign hw2reg.recov_alert_sts.es_thresh_cfg_alert.de =
es_thresh_cfg_alert;
1994     assign hw2reg.recov_alert_sts.es_thresh_cfg_alert.d =
es_thresh_cfg_alert;
1995
1996     assign hw2reg.recov_alert_sts.es_fw_ov_wr_alert.de =
es_fw_ov_wr_alert;
1997     assign hw2reg.recov_alert_sts.es_fw_ov_wr_alert.d =
es_fw_ov_wr_alert;
1998
1999     assign hw2reg.recov_alert_sts.es_fw_ov_disable_alert.de =
es_fw_ov_disable_alert;
2000     assign hw2reg.recov_alert_sts.es_fw_ov_disable_alert.d =
es_fw_ov_disable_alert;
2001
2002     // repcnt fail counter
2003     // SEC_CM: CTR.REDUN
2004     entropy_src_cntr_reg #(
2005         .RegWidth(EighthRegWidth)
2006     ) u_entropy_src_cntr_reg_repcnt_alert_fails (
2007         .clk_i   (clk_i),
2008         .rst_ni  (rst_ni),
2009         .clear_i (alert_cntrs_clr),
2010         .event_i (repcnt_fail_pulse),
2011         .step_i  (EighthRegWidth'1)),
2012         .value_o (repcnt_fail_count),
2013         .err_o   (repcnt_alert_cntr_err)
2014     );
2015
2016     assign hw2reg.alert_fail_counts.repcnt_fail_count.d = rep-
cnt_fail_count;
2017
2018     // repcnts fail counter
2019     // SEC_CM: CTR.REDUN
2020     entropy_src_cntr_reg #(
2021         .RegWidth(EighthRegWidth)
2022     ) u_entropy_src_cntr_reg_repcnts_alert_fails (

```

```

2023 .clk_i          (clk_i),
2024 .rst_ni        (rst_ni),
2025 .clear_i       (alert_cntrs_clr),
2026 .event_i       (repcnts_fail_pulse),
2027 .step_i        (EighthRegWidth'(1)),
2028 .value_o       (repcnts_fail_count),
2029 .err_o         (repcnts_alert_cntr_err)
2030 );
2031
2032 assign hw2reg.alert_fail_counts.repcnts_fail_count.d = rep-
cnts_fail_count;
2033
2034 // adaptp fail counter hi and lo
2035 // SEC_CM: CTR.REDUN
2036 entropy_src_cntr_reg #(
2037   .RegWidth(EighthRegWidth)
2038 ) u_entropy_src_cntr_reg_adaptp_hi_alert_fails (
2039   .clk_i          (clk_i),
2040   .rst_ni        (rst_ni),
2041   .clear_i       (alert_cntrs_clr),
2042   .event_i       (adaptp_hi_fail_pulse),
2043   .step_i        (EighthRegWidth'(1)),
2044   .value_o       (adaptp_hi_fail_count),
2045   .err_o         (adaptp_hi_alert_cntr_err)
2046 );
2047
2048 assign hw2reg.alert_fail_counts.adaptp_hi_fail_count.d = ad-
aptp_hi_fail_count;
2049
2050 // SEC_CM: CTR.REDUN
2051 entropy_src_cntr_reg #(
2052   .RegWidth(EighthRegWidth)
2053 ) u_entropy_src_cntr_reg_adaptp_lo_alert_fails (
2054   .clk_i          (clk_i),
2055   .rst_ni        (rst_ni),
2056   .clear_i       (alert_cntrs_clr),
2057   .event_i       (adaptp_lo_fail_pulse),
2058   .step_i        (EighthRegWidth'(1)),
2059   .value_o       (adaptp_lo_fail_count),
2060   .err_o         (adaptp_lo_alert_cntr_err)
2061 );
2062
2063 assign hw2reg.alert_fail_counts.adaptp_lo_fail_count.d = ad-
aptp_lo_fail_count;
2064
2065 // bucket fail counter
2066 // SEC_CM: CTR.REDUN
2067 entropy_src_cntr_reg #(
2068   .RegWidth(EighthRegWidth)
2069 ) u_entropy_src_cntr_reg_bucket_alert_fails (
2070   .clk_i          (clk_i),
2071   .rst_ni        (rst_ni),
2072   .clear_i       (alert_cntrs_clr),
2073   .event_i       (bucket_fail_pulse),
2074   .step_i        (EighthRegWidth'(1)),
2075   .value_o       (bucket_fail_count),
2076   .err_o         (bucket_alert_cntr_err)
2077 );
2078
2079 assign hw2reg.alert_fail_counts.bucket_fail_count.d = buck-
et_fail_count;
2080
2081 // markov fail counter hi and lo
2082 // SEC_CM: CTR.REDUN
2083 entropy_src_cntr_reg #(
2084   .RegWidth(EighthRegWidth)
2085 ) u_entropy_src_cntr_reg_markov_hi_alert_fails (
2086   .clk_i          (clk_i),
2087   .rst_ni        (rst_ni),
2088   .clear_i       (alert_cntrs_clr),
2089   .event_i       (markov_hi_fail_pulse),
2090   .step_i        (EighthRegWidth'(1)),
2091   .value_o       (markov_hi_fail_count),
2092   .err_o         (markov_hi_alert_cntr_err)
2093 );
2094
2095 assign hw2reg.alert_fail_counts.markov_hi_fail_count.d = markov-
hi_fail_count;
2096
2097 // SEC_CM: CTR.REDUN
2098 entropy_src_cntr_reg #(
2099   .RegWidth(EighthRegWidth)
2100 ) u_entropy_src_cntr_reg_markov_lo_alert_fails (
2101   .clk_i          (clk_i),
2102   .rst_ni        (rst_ni),
2103   .clear_i       (alert_cntrs_clr),
2104   .event_i       (markov_lo_fail_pulse),
2105   .step_i        (EighthRegWidth'(1)),
2106   .value_o       (markov_lo_fail_count),
2107   .err_o         (markov_lo_alert_cntr_err)
2108 );
2109
2110 assign hw2reg.alert_fail_counts.markov_lo_fail_count.d = markov-
lo_fail_count;
2111
2112 // extht fail counter hi and lo
2113 // SEC_CM: CTR.REDUN
2114 entropy_src_cntr_reg #(
2115   .RegWidth(EighthRegWidth)
2116 ) u_entropy_src_cntr_reg_extht_hi_alert_fails (

```

```

2118 .clk_i          (clk_i),
2119 .rst_ni        (rst_ni),
2120 .clear_i       (alert_cntrs_clr),
2121 .event_i       (extht_hi_fail_pulse),
2122 .step_i        (EighthRegWidth'(1)),
2123 .value_o       (extht_hi_fail_count),
2124 .err_o         (extht_hi_alert_cntr_err)
2125 );
2126
2127 assign hw2reg.extht_fail_counts.extht_hi_fail_count.d =
extht_hi_fail_count;
2128
2129 // SEC_CM: CTR.REDUN
2130 entropy_src_cntr_reg #(
2131   .RegWidth(EighthRegWidth)
2132 ) u_entropy_src_cntr_reg_extht_lo_alert_fails (
2133   .clk_i          (clk_i),
2134   .rst_ni        (rst_ni),
2135   .clear_i       (alert_cntrs_clr),
2136   .event_i       (extht_lo_fail_pulse),
2137   .step_i        (EighthRegWidth'(1)),
2138   .value_o       (extht_lo_fail_count),
2139   .err_o         (extht_lo_alert_cntr_err)
2140 );
2141
2142 assign hw2reg.extht_fail_counts.extht_lo_fail_count.d =
extht_lo_fail_count;
2143
2144 //-----
2145 // Signal a recoverable alert when dropping tested entropy
bits.
2146 //-----
2147
2148 // Post-health test entropy bits can be dropped from the
pipeline in case of e.g. backpressure
2149 // from the conditioner. The conditioner will still use the
amount of bits configured in
2150 // HEALTH_TEST_WINDOW.FIPS_WINDOW to produce the seed, and
the produced seed is okay to use.
2151 // But as the dropped bits are still tested, the effective
test window increases beyond the
2152 // value configured in HEALTH_TEST_WINDOW.FIPS_WINDOW.
2153 // Signaling this condition serves the following purposes:
2154 // 1. When running in Firmware Override: Observe mode, drop-
ping post-health test entropy bits
2155 // may cause the entropy bits observed from the Observe
FIFO to be non-contiguous, causing
2156 // the observed bits to be not usable for validation pur-
poses. Note that Firmware Override:
2157 // Extract & Insert mode is not affected by this.
2158
2159 // 2. It allows the DV environment to know when entropy bits
have been dropped which simplifies
2160 // DV.
2161 // 3. It helps tuning the depth of the distr FIFO which can
be used to absorb the backpressure
2162 // of the conditioner.
2163
2164 // The pop signal of the preceeding esrng FIFO is uncondi-
tional, meaning samples are dropped
2165 // whenever the esbit or postht FIFO is full in single-chan-
nel or multi-channel mode,
2166 // respectively.
2167 assign postht_entropy_drop_alert = sfifo_esrng_not_empty &&
(rng_bit_en ? !pfifo_esbit_not_full : !
pfifo_postht_not_full);
2168
2169 assign hw2reg.recov_alert_sts.postht_entropy_drop_alert.de =
postht_entropy_drop_alert;
2170
2171 assign hw2reg.recov_alert_sts.postht_entropy_drop_alert.d =
postht_entropy_drop_alert;
2172
2173 //-----
2174 // Pack health tested esrng bus into single bit packer FIFO.
2175 //-----
2176
2177 // SEC_CM: CONFIG.MUBI
2178 assign mubi_rng_bit_en = mu-
bi4_t'(reg2hw.conf.rng_bit_enable.q);
2179 assign rng_bit_enable_pfe = mu-
bi4_test_true_strict(mubi_rng_bit_en_fanout[0]);
2180 assign rng_bit_enable_pfa = mu-
bi4_test_invalid(mubi_rng_bit_en_fanout[1]);
2181 assign hw2reg.recov_alert_sts.rng_bit_enable_field_alert.de
= rng_bit_enable_pfa;
2182 assign hw2reg.recov_alert_sts.rng_bit_enable_field_alert.d
= rng_bit_enable_pfa;
2183
2184 prim_mubi4_sync #(
2185   .NumCopies(2),
2186   .AsyncOn(0)
2187 ) u_prim_mubi4_sync_rng_bit_en (
2188   .clk_i,

```



```

2190     .rst_ni,
2191     .mubi_i(mubi_rng_bit_en),
2192     .mubi_o(mubi_rng_bit_en_fanout)
2193 );
2194
2195
2196 assign rng_bit_en = rng_bit_enable_pfe;
2197 assign rng_bit_sel =
reg2hw.conf.rng_bit_sel.q[RngBusBitSelWidth-1:0];
2198
2199 // If RngBusWidth is less than 256 (8-bit), we have unused
bits to
2200 read to avoid linting errors
2201 if (RngBusBitSelWidth < 8) begin : gen_read_unused_bits
2202     logic unused_rng_bit_sel;
2203     assign unused_rng_bit_sel =
^reg2hw.conf.rng_bit_sel.q[7:RngBusBitSelWidth];
2204 end
2205
2206 prim_packer_fifo #(
2207     .InW(1),
2208     .OutW(RngBusWidth),
2209     .ClearOnRead(1'b0)
2210 ) u_prim_packer_fifo_esbit (
2211     .clk_i      (clk_i),
2212     .rst_ni     (rst_ni),
2213     .clr_i      (pfifo_esbit_clr),
2214     .wvalid_i   (pfifo_esbit_push),
2215     .wdata_i    (pfifo_esbit_wdata),
2216     .wready_o   (pfifo_esbit_not_full),
2217     .rvalid_o   (pfifo_esbit_not_empty),
2218     .rdata_o    (pfifo_esbit_rdata),
2219     .rready_i   (pfifo_esbit_pop),
2220     .depth_o    ()
2221 );
2222 // The prim_packer_fifo primitive is constructed to only ac-
cept pushes if there is indeed space
2223
2224 // available. The pop signal of the preceeding esrng FIFO is
unconditional, meaning samples can
2225
2226 // be dropped before the esbit FIFO in case of backpressure.
The samples are however still
2227 // tested.
2228 assign pfifo_esbit_push = rng_bit_en &&
sfifo_esrng_not_empty;
2229 assign pfifo_esbit_clr = ~es_delayed_enable;
2230
2231 assign pfifo_esbit_pop = rng_bit_en && pfifo_esbit_not_empty
&& pfifo_postht_not_full;
2232
2233 always_comb begin
2234     pfifo_esbit_wdata = '0;
2235
2236     for (int i = 0; i < RngBusWidth; i++) begin
2237         if (rng_bit_sel == i) begin
2238             pfifo_esbit_wdata = sfifo_esrng_rdata[i];
2239         end
2240     end
2241 end
2242
2243 //-----
2244 // pack tested entropy into 32 bit packer
2245 //-----
2246
2247 prim_packer_fifo #(
2248     .InW(RngBusWidth),
2249     .OutW(PostHTWidth),
2250     .ClearOnRead(1'b0)
2251 ) u_prim_packer_fifo_postht (
2252     .clk_i      (clk_i),
2253     .rst_ni     (rst_ni),
2254     .clr_i      (pfifo_postht_clr),
2255     .wvalid_i   (pfifo_postht_push),
2256     .wdata_i    (pfifo_postht_wdata),
2257     .wready_o   (pfifo_postht_not_full),
2258     .rvalid_o   (pfifo_postht_not_empty),
2259     .rdata_o    (pfifo_postht_rdata),
2260     .rready_i   (pfifo_postht_pop),
2261     .depth_o    (pfifo_postht_depth)
2262 );
2263
2264 // The prim_packer_fifo primitive is constructed to only ac-
cept pushes if there is indeed space
2265 // available. In case the single-bit mode is enabled, the
pop signal of the preceeding esbit FIFO
2266 // is conditional on the full status of the postht FIFO,
meaning backpressure can be handled. In
2267 // case the single-bit mode is disabled, the pop signal of
the preceeding esrng FIFO is
2268 // unconditional, meaning samples can be dropped before the
esbit in case of backpressure. The
2269 // samples are however still tested.
2270 assign pfifo_postht_push = rng_bit_en ?
pfifo_esbit_not_empty : sfifo_esrng_not_empty;
2271
2272 assign pfifo_postht_wdata = rng_bit_en ? pfifo_esbit_rdata :
sfifo_esrng_rdata;
2273
2274 // For verification purposes, let post-disable data continue

```

```

through to the SHA engine if it has
2273 // made it past the health checks, when in standard (non-
fw_ov) mode. This allows scoreboards
2274 // to use the same data set for computing both the SHA en-
gine outputs and the health-check stats.
2275 //
2276 // In fw_ov mode it is preferable (from a verification
standpoint) to clear all FIFOs whenever
2277 // disabled. Given the lack of handshaking on the fw_ov re-
gister path, this is easier to predict.
2278
2279 // Also, there is no association between SHA data and health
test windows in FW_OV mode, so there
2280 // is no benefit in this mode to clearing the SHA FIFOs at
the same time we clear the HT
2281 // statistics.
2282 assign pfifo_postht_clr = fw_ov_mode_entropy_insert ? !
es_enable_fo[7] : !es_delayed_enable;
2283
2284 // Pop whenever the distribution FIFO is not full. The dis-
tribution FIFO can be sized such that
2285 // it's never going to be full even under pessimistic oper-
ating conditions.
2286 assign pfifo_postht_pop = sfifo_distr_push &
sfifo_distr_not_full;
2287
2288 //-----
2289 // buffer entropy in distribution FIFO
2290 //-----
2291
2292 // The purpose of this FIFO is to buffer postht entropy bits
in case the conditioner cannot
2293 // accept them at the moment, i.e., because it's busy. By
properly sizing this FIFO, it can be
2294 // guaranteed that even under pessimistic operating condi-
tions (see
2295 // entropy_src_rng_max_rate test), entropy bits never need
to be dropped from the hardware
2296 // pipeline.
2297
2298 // SEC_CM: FIFO.CTR.REDUN
2299 prim_fifo_sync #(
2300     .Width(DistrFifoWidth),
2301     .Pass(1),
2302     .Depth(DistrFifoDepth),
2303     .OutputZeroIfEmpty(0),
2304     .Secure(1)
2305 ) u_prim_fifo_sync_distr (
2306     .clk_i      (clk_i),
2307     .rst_ni     (rst_ni),
2308     .clr_i      (sfifo_distr_clr),
2309     .wvalid_i   (sfifo_distr_push),
2310     .wdata_i    (sfifo_distr_wdata),
2311     .wready_o   (sfifo_distr_not_full),
2312     .rvalid_o   (sfifo_distr_not_empty),
2313     .rdata_o    (sfifo_distr_rdata),
2314     .rready_i   (sfifo_distr_pop),
2315     .full_o     (sfifo_distr_full),
2316     .depth_o    (sfifo_distr_depth),
2317     .err_o      (sfifo_distr_int_err)
2318 );
2319
2320 // The prim_fifo_sync primitive is constructed to only ac-
cept pushes if there is indeed space
2321 // available. Backpressure is handled at the sender.
2322 assign sfifo_distr_push = pfifo_postht_not_empty;
2323 assign sfifo_distr_wdata = pfifo_postht_rdata;
2324
2325 assign sfifo_distr_clr = fw_ov_mode_entropy_insert ? !es_en-
able_fo[19] : !es_delayed_enable;
2326
2327 // In firmware override mode with extract & insert enabled,
post-health test entropy bits can
2328
2329 // only move into the observe FIFO. Once the observe FIFO is
full, post-health test entropy is
2330 // just discarded.
2331 assign sfifo_distr_pop = fw_ov_mode_entropy_insert ?
sfifo_distr_not_empty :
// In firmware override mode (ob-
serve only) or during normal
2332 // operation, post-health test en-
tropy bits continue to flow
2333 // through the hardware pipeline.
es_bypass_mode ?
pfifo_bypass_push :
pfifo_precon_push &
pfifo_precon_not_full;
2334
2335 // fifo err
2336 // Note that for the used prim_fifo_sync and prim_pack-
er_fifo primitives it is not an error to
2337 // push to a FIFO that is full. The primitives simply don't
accept the data when full. The
2338 // backpressure needs to be handled at the sender.
2339 assign sfifo_distr_err =
{1'b0,
(sfifo_distr_pop && !sfifo_distr_not_empty),
(sfifo_distr_full && !sfifo_distr_not_empty) ||
sfifo_distr_int_err};

```

```

2344
2345 //-----
2346 // store entropy into a 32 entry deep FIFO
2347 //-----
2348
2349 // SEC_CM: FIFO.CTR.REDUN
2350 prim_fifo_sync #(
2351   .Width(ObserveFifoWidth),
2352   .Pass(0),
2353   .Depth(ObserveFifoDepth),
2354   .OutputZeroIfEmpty(1), // Prevent SVA from firing due un-
known module outputs.
2355   .Secure(1)
2356 ) u_prim_fifo_sync_observe (
2357   .clk_i      (clk_i),
2358   .rst_ni     (rst_ni),
2359   .clr_i      (sfifo_observe_clr),
2360   .wvalid_i   (sfifo_observe_push),
2361   .wdata_i    (sfifo_observe_wdata),
2362   .wready_o   (),
2363   .rvalid_o   (sfifo_observe_not_empty),
2364   .rdata_o    (sfifo_observe_rdata),
2365   .rready_i   (sfifo_observe_pop),
2366   .full_o     (sfifo_observe_full),
2367   .depth_o    (sfifo_observe_depth),
2368   .err_o      (sfifo_observe_int_err)
2369 );
2370
2371 // The Observe fifo is intended to hold kilobits of contiguous
data, yet still gracefully
2372 // drop data when full. This flop gates the observe fifo.
If it ever overflows, no new data is
2373 // allowed until it is empty. Thus if the rate of CSR up-
take almost matches the RNG data rate
2374 // the FIFO avoids unnecessary segmentation, and guarantees
that the remaining RNG data is as
2375 // contiguous as possible.
2376 logic sfifo_observe_gate_d, sfifo_observe_gate_q;
2377
2378 assign sfifo_observe_gate_d = (sfifo_distr_pop &&
sfifo_observe_full) ? 1'b0 :
2379
2380 sfifo_observe_not_empty
2381
2382 ? 1'b1 :
2383 sfifo_observe_gate_q;
2384
2385 always_ff @(posedge clk_i or negedge rst_ni) begin
2386   if (!rst_ni) begin
2387     sfifo_observe_gate_q <= 1'b1;
2388   end else begin
2389     sfifo_observe_gate_q <= sfifo_observe_gate_d;
2390   end
2391 end
2392
2393 assign hw2reg.fw_ov_rd_fifo_overflow.d = !
sfifo_observe_gate_d;
2394 assign hw2reg.fw_ov_rd_fifo_overflow.de = 1'b1;
2395
2396 assign observe_fifo_thresh_met = fw_ov_mode && (ob-
serve_fifo_thresh != '0) &&
2397 (observe_fifo_thresh <= sfifo_observe_depth) &&
es_enable_fo[8];
2398
2399 assign hw2reg.observe_fifo_depth.d = sfifo_observe_depth;
2400
2401 // fifo controls
2402 assign sfifo_observe_push = fw_ov_mode && sfifo_distr_pop &&
(sfifo_observe_gate_q || !
sfifo_observe_not_empty);
2403
2404 assign sfifo_observe_clr = ~es_enable_fo[9];
2405
2406 assign sfifo_observe_wdata = sfifo_distr_rdata;
2407
2408 assign sfifo_observe_pop =
(fw_ov_mode && fw_ov_fifo_rd_pulse);
2409
2410 // fifo err
2411 // Note that for the used prim_fifo_sync and prim_pack-
er_fifo
primitives it is not an error to
2412 // push to a FIFO that is full. The primitives simply don't
accept the data when full. The
2413 // backpressure needs to be handled at the sender.
2414 assign sfifo_observe_err =
{1'b0,
2415  (sfifo_observe_pop && !sfifo_observe_not_empty),
2416  (sfifo_observe_full && !sfifo_observe_not_empty) ||
sfifo_observe_int_err};
2417
2418
2419 //-----
2420 // pack entropy into 64 bit packer
2421 //-----
2422
2423 prim_packer_fifo #(
2424   .InW(ObserveFifoWidth),
2425   .OutW(PreCondWidth),
2426   .ClearOnRead(1'b0)
2427 ) u_prim_packer_fifo_precon (
2428   .clk_i      (clk_i),
2429   .rst_ni     (rst_ni),
2430   .clr_i      (pfifo_precon_clr),
2431   .wvalid_i   (pfifo_precon_push),
2432   .wdata_i    (pfifo_precon_wdata),
2433   .wready_o   (pfifo_precon_not_full),
2434   .rvalid_o   (pfifo_precon_not_empty),
2435   .rdata_o    (pfifo_precon_rdata),
2436   .rready_i   (pfifo_precon_pop),
2437   .depth_o    (pfifo_precon_depth)
2438 );
2439
2440 // When bypassing the hardware conditioning - due to a) dis-
abling FIPS mode or b) routing entropy
2441 // to the ENTROPY_DATA register (ES_ROUTE) and bypassing the
conditioner (ES_TYPE) - nothing is
2442 // going into the conditioner.
2443 assign pfifo_precon_push = es_bypass_mode ? 1'b0 :
2444
2445 // In firmware override mode with
extract & insert enabled, only bits
2446 // inserted by firmware continue
down the pipeline.
2447 fw_ov_mode_entropy_insert ?
2448 fw_ov_fifo_wr_pulse :
2449 // Otherwise post-health test en-
tropy bits continue to flow
2450 // downstream. This includes ob-
serve-only firmware override mode.
2451 sfifo_distr_not_empty;
2452
2453 assign pfifo_precon_wdata = fw_ov_mode_entropy_insert ?
fw_ov_wr_data :
2454 sfifo_distr_rdata;
2455
2456 // For verification purposes, let post-disable data continue
through to the SHA engine if it has
2457 // made it past the health checks, when in standard (non-
fw_ov) mode. This allows scoreboards
2458 // to use the same data set for computing both the SHA en-
gine outputs and the health-check stats.
2459 //
2460 // In fw_ov mode it is preferable (from a verification
standpoint) to clear all FIFOs whenever
2461 // disabled. Given the lack of handshaking on the fw_ov re-
gister path, this is easier to predict.
2462
2463 // Also, there is no association between SHA data and health
test windows in FW_OV mode, so there
2464 // is no benefit in this mode to clearing the SHA FIFOs at
the same time we clear the HT
2465 // statistics.
2466 //
2467 // Corner case: Even in FW_OV mode, if a full SHA word is
ready as the disable comes in,
2468 // let it stay in the FIFO until the SHA engine has picked
it up, as verification has no way
2469 // of knowing if a word will get stalled by SHA backpres-
sure. This is not a problem however
2470 // as the reset is only important for clearing 32-bit half-
SHA-words.
2471 assign pfifo_precon_clr = fw_ov_mode_entropy_insert ?
~es_enable_fo[10] &
~pfifo_precon_not_empty :
2472 ~es_delayed_enable &
~pfifo_precon_not_empty;
2473
2474 assign pfifo_precon_pop = (pfifo_cond_push &&
sha3_msgfifo_ready);
2475
2476 assign es_fw_ov_wr_alert = fw_ov_mode &&
fw_ov_mode_entropy_insert &&
fw_ov_fifo_wr_pulse && fw_ov_wr_fifo_full;
2477
2478 assign es_fw_ov_disable_alert = fw_ov_mode &&
fw_ov_mode_entropy_insert &&
!es_bypass_mode && fw_ov_sha3_disable_pulse &&
fw_ov_wr_fifo_full;
2479
2480 //-----
2481 // entropy conditioner
2482 //-----
2483 // This block takes in either post-health test entropy bits
(when running in FIPS/CC compliant
2484 // mode) or entropy injected by software (when running in
Firmware Override: Extract & Insert
2485 // mode).
2486
2487 // The amount of entropy consumed to generate a 384-bit seed
depends on the mode of operation:
2488 // - In FIPS/CC compliant mode,
HEALTH_TEST_WINDOWS.FIPS_WINDOW x 4 bits (by default 2048 bits)
2489 // are required to produce one 384-bit seed. For the first
seed after start up, the number of
2490 // bits consumed is doubled to align with the required
1024 samples of 4 bits for startup
2491 // health testing.
2492 // For windows which fail a health test, the entropy is
still absorbed by the SHA3 engine
2493 // but no seed is produced. In order for the SHA3 engine
to produce a seed, the last window it

```

```

2493 // absorbed must have passed the health tests.
2494 // - In Firmware Override: Extract & Insert mode, the operation of the SHA3 engine is software
2495 // defined.
2496 //
2497 // Note that the final absorption operation of the SHA3 engine is triggered by the main state
2498 // machine (upon receiving the health-test done pulse). Depending on the backpressure currently
2499 // experienced in the pipeline, the sha3_process pulse needs to be delayed to allow all bits in
2500 // the postht, distr and precon FIFOs to be absorbed first. The SHA3 engine is also triggered
2501 // internally by the padding logic whenever 832 bits (= the rate or block size of SHA3-384) have
2502 // been received.
2503 //
2504 // Note on backpressure handling from the SHA3 engine:
2505 // To avoid inferring a combo loop, the msg_valid_i input (pfifo_cond_push signal) must not
2506 // depend on the msg_ready_o output (sha3_msgfifo_ready). However, we can always push into the
2507 // SHA3 engine as long as the precon FIFO contains valid data. The ready output is just used to
2508 // determine when to pop from the precon FIFO.
2509 //
2510 // The postht FIFO must be completely empty when the health-test done pulse is signaled to the
2511 // main state machine. Note that since the postht FIFO is a packer FIFO, observing the not_empty
2512 // output signal is not sufficient. The only exception is when the pipeline is entirely full.
2513 // logic [PostHTFifoDepthW-1:0] unused_pfifo_postht_depth;
2514 // assign unused_pfifo_postht_depth = pfifo_postht_depth;

2515 `ASSERT(PostHTFifoEmptyWhenHtDonePulse_A, main_sm_done_pulse
|-> pfifo_postht_depth == '0 ||
2516 !pfifo_postht_not_full && sfifo_distr_full)
2517
2518 // Pipeline depth computation
2519 assign pipeline_depth =
2520 PipelineDepthW'(!pfifo_postht_not_full) +
2521 PipelineDepthW'(sfifo_distr_depth) +
2522 PipelineDepthW'(pfifo_precon_depth);
2523
2524 // We only ever push full 64-bit words into the conditioner and for this reason, we just track
2525 // the number of full 64-bit words that still have to be absorbed before triggering the final
2526 // processing. In the worst case, the pipeline is completely full when the health-test done
2527 // pulse is signaled, meaning there are 1 (postht FIFO) + DistrFifoDepth + 2 (precon FIFO)
2528 // number of 32-bit words in the pipeline. As a consequence, this number needs to be divisible
2529 // by two and the distribution FIFO depth needs to be an odd number.
2530 `ASSERT_INIT(DistrFifoDepthOdd_A, DistrFifoDepth % 2 == 1)
2531
2532 logic pipeline_depth_cntr_set;
2533 logic pipeline_depth_cntr_decr_en;
2534 logic [PipelineDepthW-1:0] pipeline_depth_cntr_set_cnt;
2535 logic [PipelineDepthW-1:0] pipeline_depth_cntr;
2536 logic pipeline_depth_cntr_zero;
2537
2538 assign pipeline_depth_cntr_set =
2539 main_sm_done_pulse & ~main_sm_ht_failed &
~fw_ov_mode_entropy_insert;
2540 assign pipeline_depth_cntr_decr_en = pfifo_cond_push & sha3_msgfifo_ready;
2541 // When we set the counter while the current contents of precon FIFO get pushed into the
2542 // conditioner, we have to correct the start value of the counter accordingly.
2543 assign pipeline_depth_cntr_set_cnt =
2544 pipeline_depth_cntr_decr_en ? pipeline_depth - PipelineDepthW'(2) : pipeline_depth;
2545
2546 prim_count #(
2547 .Width(PipelineDepthW),
2548 .PossibleActions(prim_count_pkg::Clr |
2549 prim_count_pkg::Set |
2550 prim_count_pkg::Decr)
2551 ) u_prim_count_pipeline_depth (
2552 .clk_i,
2553 .rst_ni,
2554 .clr_i(!es_delayed_enable),
2555 .set_i(pipeline_depth_cntr_set),
2556 .set_cnt_i(pipeline_depth_cntr_set_cnt),
2557 .incr_en_i('b0),
2558 .decr_en_i(pipeline_depth_cntr_decr_en),
2559 .step_i(PipelineDepthW'(2)),
2560 .commit_i('b1),
2561 .cnt_o(pipeline_depth_cntr),
2562 .cnt_after_commit_o(),
2563 .err_o(pipeline_depth_cntr_err)
2564 );

```

```

2565
2566 assign pipeline_depth_cntr_zero = pipeline_depth_cntr == '0;
2567
2568 // Backpressure handling
2569 assign pfifo_cond_push = pfifo_precon_not_empty && !es_by-pass_mode;
2570 assign pfifo_cond_wdata = pfifo_precon_rdata;
2571
2572 // The SHA3 engine is unmasked - only connect Share 0.
2573 assign msg_data[0] = pfifo_cond_wdata;
2574 assign pfifo_cond_rdata = sha3_state[0][SeedLen-1:0];
2575
2576 // SHA3 hashing engine
2577 sha3 #(
2578 .EnMasking (Sha3EnMasking)
2579 ) u_sha3 (
2580 .clk_i,
2581 .rst_ni,
2582
2583 // MSG_FIFO interface
2584 .msg_valid_i (pfifo_cond_push),
2585 .msg_data_i (msg_data),
2586 .msg_strb_i ({8{pfifo_cond_push}}),
2587 .msg_ready_o (sha3_msgfifo_ready),
2588
2589 // Entropy interface - not using
2590 .rand_valid_i ('b0),
2591 .rand_early_i ('b0),
2592 .rand_data_i ('0),
2593 .rand_aux_i ('0),
2594 .rand_update_o (),
2595 .rand_consumed_o (),
2596
2597 // N, S: Used in cSHAKE mode
2598 .ns_data_i ('0), // ns_prefix,
2599
2600 // Configurations
2601 .mode_i (sha3_pkg::Sha3), // Use SHA3 mode
2602 .strength_i (sha3_pkg::L384), // Use keccak_strength_e of L384
2603
2604 // Controls (CMD register)
2605 .start_i (sha3_start ),
2606 .process_i (sha3_process ),
2607 .run_i ('b0 ), // For software application
2608 .done_i (sha3_done ),
2609
2610 // LC escalation
2611 .lc_escalate_en_i (lc_ctrl_pkg::0ff),
2612
2613 .absorbed_o (sha3_absorbed),
2614 .squeezing_o (sha3_squeezing),
2615
2616 .block_processed_o (sha3_block_processed),
2617
2618 .sha3_fsm_o (sha3_fsm),
2619
2620 .state_valid_o (sha3_state_vld),
2621 .state_o (sha3_state),
2622
2623 // REQ/ACK interface to avoid power spikes
2624 .run_req_o(sha3_block_busy),
2625 .run_ack_i('b1),
2626
2627 .error_o (sha3_err),
2628 .sparse_fsm_error_o (sha3_state_error),
2629 .count_error_o (sha3_count_error),
2630 .keccak_storage_rst_error_o (sha3_rst_storage_err)
2631 );
2632
2633 //-----
2634 // bypass SHA conditioner path
2635 //-----
2636
2637 prim_packer_fifo #(
2638 .InW(PostHTWidth),
2639 .OutW(SeedLen),
2640 .ClearOnRead('b0)
2641 ) u_prim_packer_fifo_bypass (
2642 .clk_i (clk_i),
2643 .rst_ni (rst_ni),
2644 .clr_i (pfifo_bypass_clr),
2645 .wvalid_i (pfifo_bypass_push),
2646 .wdata_i (pfifo_bypass_wdata),
2647 .wready_o (pfifo_bypass_not_full),
2648 .rvalid_o (pfifo_bypass_not_empty),
2649 .rdata_o (pfifo_bypass_rdata),
2650 .rready_i (pfifo_bypass_pop),
2651 .depth_o ()
2652 );
2653
2654 // Unless the hardware conditioning is bypassed - due to a) disabling FIPS mode or b) routing
2655 // entropy to the ENTROPY_DATA register (ES_ROUTE) and bypassing the conditioner (ES_TYPE) -
2656 // nothing is going into the bypass FIFO.
2657 assign pfifo_bypass_push = !es_bypass_mode ? 'b0 :
2658
2659 // In firmware override mode with
2660 extract & insert enabled, only bits

```

```

2659 // inserted by firmware continue
down the pipeline
2660 fw_ov_fifo_wr_pulse :
2661 // Otherwise post-health test en-
2662 // downstream. This includes ob-
2663 // serve-only firmware override mode.
2664 sfifo_distr_not_empty;
2665 assign pfifo_bypass_wdata = fw_ov_mode_entropy_insert ?
2666 fw_ov_wr_data :
2667 sfifo_distr_rdata;
2668 assign pfifo_bypass_clr = !es_enable_fo[11];
2669
2670 // Corner case: If the main state machine encounters an
2671 // alert, drain the
2672 // bypass fifo, to get rid of the seeds and let the HT stats
2673 // continue.
2674 assign pfifo_bypass_pop =
2675 fw_ov_mode_entropy_insert ? pfifo_bypass_not_empty :
2676 bypass_stage_pop;
2677 // mux to select between fips and bypass mode
2678
2679 assign final_es_data = es_bypass_mode ? pfifo_bypass_rdata :
2680 pfifo_cond_rdata;
2681
2682 //-----
2683 // state machine to coordinate fifo flow
2684 //-----
2685 assign main_sm_done_pulse = rng_bit_en ? ht_done_pulse_qq :
2686 ht_done_pulse_q;
2687 assign main_sm_ht_failed = rng_bit_en ? ht_failed_qq :
2688 ht_failed_q;
2689
2690 // SEC_CM: CTR.LOCAL_ESC
2691 // SEC_CM: MAIN_SM.FSM.SPARSE
2692 entropy_src_main_sm
2693 u_entropy_src_main_sm (
2694 .clk_i (clk_i),
2695 .rst_ni (rst_ni),
2696 .enable_i (main_sm_enable),
2697 .fw_ov_ent_insert_i (fw_ov_mode_entropy_insert),
2698 .fw_ov_sha3_start_i (fw_ov_sha3_start_pfe),
2699 .ht_done_pulse_i (main_sm_done_pulse),
2700 .pd_cntr_zero_i (pipeline_depth_cntr_zero),
2701 .ht_fail_pulse_i (main_sm_ht_failed),
2702 .alert_thresh_fail_i (alert_thresh_fail),
2703 .alert_cntr_clr_ok_o (alert_cntr_clr_ok),
2704 .bypass_mode_i (es_bypass_mode),
2705 .bypass_stage_rdy_i (pfifo_bypass_not_empty),
2706 .sha3_state_vld_i (sha3_state_vld),
2707 .main_stage_push_o (main_stage_push_raw),
2708 .bypass_stage_pop_o (bypass_stage_pop),
2709 .boot_phase_done_o (boot_phase_done),
2710 .sha3_start_o (sha3_start_raw),
2711 .sha3_process_o (sha3_process),
2712 .sha3_done_o (sha3_done),
2713 .local_escalate_i (fatal_loc_events),
2714 .main_sm_alert_o (es_main_sm_alert),
2715 .main_sm_idle_o (es_main_sm_idle),
2716 .main_sm_state_o (es_main_sm_state),
2717 .main_sm_err_o (es_main_sm_err)
2718 );
2719
2720 //-----
2721 // Corner case masking of main_sm inputs/outputs
2722 //-----
2723
2724 // When operating in RNG mode the state machine does not re-
2725 spond
2726 // immediately to disable requests if it processing the SHA
2727 output
2728 // (here indicated by the sha3_block_busy signal). The SHA
2729 engine
2730
2731 // will continue to process even if the module is disabled.
2732 These seeds
2733 // that continue to process after the disable signal are re-
2734 ferred to as
2735 // stale.
2736 //
2737 // If the SHA processing were instantaneous, stale seeds
2738 would be discarded
2739
2740 // when the esfinal fifo was cleared on diable. Though since
2741 processing
2742 // can push through a disable pulse, stale seeds need to be
2743 identified,
2744 // and held back from the esfinal FIFO.
2745 //
2746 // There is at most one seed processing at a time so, we
2747 simply need to
2748 // detect when a stale seed has commenced processing, and
2749 mask the following
2750 // main_stage_push signal.
2751
2752

```

```

2736 assign stale_seed_processing = ~es_bypass_mode &
~fw_ov_mode_entropy_insert &
2737 sha3_block_busy & ~es_en-
2738 able_fo[12];
2739 assign sha3_flush_d = stale_seed_processing ? 1'b1 :
2740 main_stage_push_raw ? 1'b0 :
2741 sha3_flush_q;
2742
2743 // If the user incorrectly disables the fw_ov SHA3 pro-
2744 cessing while
2745 // data is in the pipeline, it can potentially scramble two
2746 outputs.
2747 // Thus in addition to triggering a recoverable alert, we
2748 mark the
2749 // following _two_ outputs as corrupted and to not let them
2750 in the
2751 // esfinal FIFO
2752 assign fw_ov_corrupted_d = es_fw_ov_disable_alert ? 2'b11 :
2753 !es_bypass_mode &&
2754 main_stage_push_raw ? {1'b0, fw_ov_corrupted_q[1]} :
2755 fw_ov_corrupted_q;
2756
2757 assign fw_ov_corrupted = (!fw_ov_corrupted_q) & !es-by-
2758 pass_mode;
2759
2760
2761 assign main_stage_push = main_stage_push_raw & !sha3_flush_q
2762 & !fw_ov_corrupted;
2763
2764
2765 // Use the delayed enable signal to keep the Main SM enabled
2766 while Data is in flight, and
2767 // to make sure it receives a delayed disable pulse after
2768 finishing any final SHA processing
2769 // commands
2770 assign main_sm_enable = es_delayed_enable;
2771
2772 // The main SM can also generate redundant start pulses.
2773 After data can be pushed into SHA,
2774 // the SM can be disabled leaving entropy in the SHA
2775 sponge. This is fine, but the SM will
2776 // have no recollection of this previous start pulse. We
2777 track redundant start pulses
2778 // outside the SM and suppress them as needed.
2779 assign sha3_start_mask_d = sha3_start_raw ? 1'b1 :
2780 sha3_process ? 1'b0 :
2781 sha3_start_mask_q;
2782 assign sha3_start = sha3_start_raw & ~sha3_start_mask_q;
2783
2784 //-----
2785 // send processed entropy to final fifo
2786 //-----
2787
2788 // SEC_CM: FIFO.CTR.REDUN
2789 prim_fifo_sync #(
2790 .Width(1+SeedLen),
2791 .Pass(0),
2792 .Depth(EsFifoDepth),
2793 .OutputZeroIfEmpty(0),
2794 .Secure(1)
2795 ) u_prim_fifo_sync_esfinal (
2796 .clk_i (clk_i),
2797 .rst_ni (rst_ni),
2798 .clr_i (sfifo_esfinal_clr),
2799 .wvalid_i (sfifo_esfinal_push),
2800 .wready_o (sfifo_esfinal_not_full),
2801 .wdata_i (sfifo_esfinal_wdata),
2802 .rvalid_o (sfifo_esfinal_not_empty),
2803 .rready_i (sfifo_esfinal_pop),
2804 .rdata_o (sfifo_esfinal_rdata),
2805 .full_o (sfifo_esfinal_full),
2806 .depth_o (sfifo_esfinal_depth),
2807 .err_o (sfifo_esfinal_int_err)
2808 );
2809
2810 // The FIPS flag is fully determined in SW. This has to be
2811 the case since we don't know
2812 // which mode of operation will be validated/certified. An-
2813 other reason is that SW can
2814 // set the threshold values arbitrarily, which on its own
2815 makes the FIPS bit basically
2816 // SW defined.
2817 assign fips_compliance = es_enable_fo[13] && fips_flag_pfe;
2818
2819 // fifo controls
2820
2821 // No backpressure is possible at this point. If the esfinal
2822 FIFO is already full and a new seed
2823 // is pushed, the push is ignored and the seed is lost.
2824 assign sfifo_esfinal_push =
2825 fw_ov_mode_entropy_insert && es_bypass_mode ?
2826 pfifo_bypass_not_empty :
2827 main_stage_push;
2828
2829 assign sfifo_esfinal_clr = !es_enable_fo[14];
2830 assign sfifo_esfinal_wdata = {fips_compli-
2831 ance, final_es_data};
2832 assign sfifo_esfinal_pop = es_route_to_sw ? swread_done :
2833 es_hw_if_fifo_pop;
2834 assign {esfinal_fips_flag, esfinal_data} =

```

```

sfifo_esfinal_rdata;
2813
2814 // fifo err
2815 // Note that for the used prim_fifo_sync and prim_pack-
er_fifo primitives it is not an error to
2816 // push to a FIFO that is full. The primitives simply don't
accept the data when full. The
2817 // backpressure needs to be handled at the sender.
2818 assign sfifo_esfinal_err =
2819     {1'b0,
2820      (sfifo_esfinal_pop && !sfifo_esfinal_not_empty),
2821      (sfifo_esfinal_full && !sfifo_esfinal_not_empty) ||
sfifo_esfinal_int_err};
2822
2823 // drive out hw interface
2824 assign es_hw_if_req = entropy_src_hw_if_i.es_req;
2825 assign entropy_src_hw_if_o.es_ack = es_hw_if_ack;
2826 assign entropy_src_hw_if_o.es_bits = esfinal_data;
2827 assign entropy_src_hw_if_o.es_fips = esfinal_fips_flag;
2828
2829 // SEC_CM: ACK_SM.FSM.SPARSE
2830 entropy_src_ack_sm u_entropy_src_ack_sm (
2831     .clk_i      (clk_i),
2832     .rst_ni     (rst_ni),
2833     .enable_i   (es_enable_fo[15]),
2834     .req_i      (es_hw_if_req),
2835     .ack_o      (es_hw_if_ack),
2836     .fifo_not_empty_i (sfifo_esfinal_not_empty && !
es_route_to_sw),
2837     .local_escalate_i (fatal_loc_events),
2838     .fifo_pop_o    (es_hw_if_fifo_pop),
2839     .ack_sm_err_o  (es_ack_sm_err)
2840 );
2841
2842 //-----
2843 // data path integrity check
2844 // - a countermeasure to detect entropy bus tampering at-
tempts
2845 // - checks to make sure repeated data sets off
2846 //   an alert for sw to handle
2847 //-----
2848
2849 // SEC_CM: ESFINAL_RDATA.BUS.CONSTISTENCY
2850
2851 // capture a copy of the entropy data
2852 assign es_rdata_capt_vld = (sfifo_esfinal_pop &&
sfifo_esfinal_not_empty);
2853
2854 assign es_rdata_capt_d = es_rdata_capt_vld ?
sfifo_esfinal_rdata[63:0] : es_rdata_capt_q;
2855
2856 assign es_rdata_capt_vld_d =
2857     !es_enable_fo[16] ? 1'b0 :
2858     es_rdata_capt_vld ? 1'b1 :
2859     es_rdata_capt_vld_q;
2860
2861 // continuous compare of the entropy data
2862 assign es_bus_cmp_alert = es_rdata_capt_vld &&
es_rdata_capt_vld_q &&
2863     (es_rdata_capt_q == sfifo_esfinal_rdata[63:0]);
2864
2865 //-----
2866 // software es read path via ENTROPY_DATA register
2867 //-----
2868
2869 // Sync and evaluate the OTP input.
2870 prim_mubi8_sync #(
2871     .NumCopies(1),
2872     .AsyncOn(1)
2873 ) u_prim_mubi8_sync_es_fw_read (
2874     .clk_i,
2875     .rst_ni,
2876     .mubi_i(otp_en_entropy_src_fw_read_i),
2877     .mubi_o({en_entropy_src_fw_read})
2878 );
2879
2880 assign efuse_es_sw_reg_en =
prim_mubi_pkg::mubi8_test_true_strict(en_entropy_src_fw_read);
2881
2882 // Is the ENTROPY_DATA register readable?
2883 assign es_data_reg_rd_en = es_enable_fo[17] &&
efuse_es_sw_reg_en && entropy_data_reg_en_pfe;
2884
2885 // Interface the esfinal FIFO and cut its 384-bit output
into 32-bit chunks for software.
2886 // We're done after the last read.
2887
2887 assign swread_done = reg2hw.entropy_data.re && (swread_idx_q
== LastSwRead[SwReadIdxWidth-1:0]);
2888
2889 // Increment the index counter upon reads from software.
2890 assign swread_idx_incr = reg2hw.entropy_data.re;
2891
2892 // Unless the ENTROPY_DATA CSR is readable, keep clearing
the index counter. Clear the counter
2893 // after a full seed has been read.
2894 assign swread_idx_clr = !es_data_reg_rd_en || swread_done;
2895
2896 // Index counter
2897 assign swread_idx_d = !es_enable_fo[18] ?
'0

```

```

2898     swread_idx_clr ?
'0
:
2899     swread_idx_incr ? swread_idx_q +
1'b1 : swread_idx_q;
2900 always_ff @(posedge clk_i or negedge rst_ni) begin :
swread_idx_reg
2901     if (!rst_ni) begin
2902         swread_idx_q <= '0;
2903     end else begin
2904         swread_idx_q <= swread_idx_d;
2905     end
2906 end
2907
2908 // Forward the relevant part of the valid esfinal output or
0 if ES_ROUTE is off.
2909 assign swread_data = sfifo_esfinal_not_empty &&
es_route_to_sw ?
2910     esfinal_data[swread_idx_q * FullRegWidth +: FullReg-
Width] : '0;
2911
2912 // This primitive is used to place a size-only constraint on
the buffers to act as a synthesis
2913 // optimization barrier. This ensures ES_ROUTE and EN-
TROPY_DATA_REG_ENABLE together with the OTP
2914 // input are taken into account in separately.
2915 prim_buf #(
2916     .Width(FullRegWidth)
2917 ) u_prim_buf_swread_data (
2918     .in_i (swread_data),
2919     .out_o(swread_data_buf)
2920 );
2921
2922 // Forward the data to the ENTROPY_DATA CSR or 0 if the CSR
is not readable.
2923 assign hw2reg.entropy_data.d = es_data_reg_rd_en ?
swread_data_buf : '0;
2924
2925 //-----
2926 // unused signals
2927 //-----
2928
2929 assign unused_err_code_test_bit = (|
{err_code_test_bit[27:25],err_code_test_bit[19:4]});
2930
2931 assign unused_sha3_state = ({sha3_state[0]
[sha3_pkg::StateW-1:SeedLen]});
2932
2933 assign unused_entropy_data = ({reg2hw.entropy_data.q});
2934 assign unused_fw_ov_rd_data = ({reg2hw.fw_ov_rd_data.q});
2935 assign unused_sfifo_esrng_not_full = sfifo_esrng_not_full;
2936 assign unused_sfifo_esfinal_not_full =
sfifo_esfinal_not_full;
2937
2938 //-----
2939 // Assertions
2940 //-----
2941 `ifdef INC_ASSERT
2942 `include "prim_macros.svh"
2943
2944 // Assert that we request high quality entropy only when the
rng_fips field of the conf register
2945 // is set to Mubi4True.
2946 `ASSERT(RngFipsOutputHighInFipsMode_A,
prim_mubi_pkg::mubi4_test_true_loose(mubi4_t'(reg2hw.conf.rng_fips.q
)) == rng_fips_o)
2947
2948 // Count number of disables since last reset.
2949 logic [63:0] disable_cnt_d, disable_cnt_q;
2950 always_comb begin
2951     disable_cnt_d = disable_cnt_q;
2952     if (!mubi4_test_true_strict(mubi_mod_en_dly_d) &&
mubi4_test_true_strict(mubi_mod_en_dly_q)) begin
2953         disable_cnt_d += 1;
2954     end
2955 end
2956
2957 // Assert that no entropy gets dropped during FIPS-compliant
operation mode.
2958 //
2959 // The following code, which includes counters, small FSMs,
and assertions, tracks entropy bits
2960
2961 // from the noise source input through Entropy Source to the
hardware interface output and checks
2962 // that in FIPS-compliant mode entropy does not get dropped
unless it should get dropped. The
2963 // code is arranged in the same order as entropy flows
through Entropy Source, so having Entropy
2964 // Source's block diagram ready when reading this code is
highly recommended.
2965
2966 // Delay 'es_delayed_enable' by one clock cycle to track
when Entropy Source actually accepts RNG
2967 // inputs.
2968 logic es_delayed_enable_d, es_delayed_enable_q;
2969 assign es_delayed_enable_d = es_delayed_enable;
2970
2971 // Count number of valid bits from RNG input (RngBusWidth

```



```

wide) while Entropy Source is enabled.
2972 logic [63:0] rng_valid_bit_cnt_d, rng_valid_bit_cnt_q;
2973 assign rng_valid_bit_cnt_d = entropy_src_rng_valid_i &&
es_enable_fo[5] &&
2974 es_delayed_enable_q ?
2975 rng_valid_bit_cnt_q + Rng-
BusWidth :
2976 rng_valid_bit_cnt_q;
2977
2978 // Count number of bits pushed into esrng FIFO (RngBusWidth
wide).
2979 logic [63:0] esrng_push_bit_cnt_d, esrng_push_bit_cnt_q;
2980 assign esrng_push_bit_cnt_d = sfifo_esrng_push &
sfifo_esrng_not_full ?
2981 esrng_push_bit_cnt_q + Rng-
BusWidth :
2982 esrng_push_bit_cnt_q;
2983
2984 // Assert that as many bits got pushed into the esrng FIFO
(destination) as there were valid RNG
2985 // input bits (source). RngBusWidth bits may get lost after
every re-enable; add a margin to
2986 // account for that.
2987 `AS-
SERT_AT_RESET_AND_FINAL(ValidRngBitsPushedIntoEsrngFifo_A,
2988 WITH-
IN_MARGIN(esrng_push_bit_cnt_q, // actual
2989 rng_val-
id_bit_cnt_q, // expected
2990 disable_cnt_q *
RngBusWidth, // allowed less
2991 0)) // allowed more
2992
2993 // Count number of bits pushed into esbit FIFO (1 wide in-
put, RngBusWidth wide output).
2994 logic [63:0] esbit_push_bit_cnt_d, esbit_push_bit_cnt_q;
2995 assign esbit_push_bit_cnt_d = pfifo_esbit_push &
pfifo_esbit_not_full ?
2996 esbit_push_bit_cnt_q + 1 :
2997 esbit_push_bit_cnt_q;
2998
2999 // Count number of bits pushed into postht FIFO (RngBusWidth
wide input, PostHTWidth wide output).
3000 logic [63:0] postht_push_bit_cnt_d, postht_push_bit_cnt_q;
3001 assign postht_push_bit_cnt_d = pfifo_postht_push &
pfifo_postht_not_full ?
3002 postht_push_bit_cnt_q + Rng-
BusWidth :
3003 postht_push_bit_cnt_q;
3004
3005 // Count number of bits pushed into postht FIFO from esrng
FIFO.
3006 logic [63:0] postht_from_esrng_push_bit_cnt_d, pos-
tht_from_esrng_push_bit_cnt_q;
3007 assign postht_from_esrng_push_bit_cnt_d =
3008 pfifo_postht_push & pfifo_postht_not_full &
~rng_bit_en ?
3009 postht_from_esrng_push_bit_cnt_q + RngBusWidth :
3010 postht_from_esrng_push_bit_cnt_q;
3011
3012 // Assert that as many bits got pushed into the esbit FIFO
or the postht FIFO (destinations) as
3013 // into the esrng FIFO (source). The number of bits pushed
into the esbit FIFO has to be
3014 // multiplied by the output width of the esrng FIFO (Rng-
BusWidth) because only 1 bit gets pushed
3015 // into the esbit FIFO for every pop from the esrng FIFO.
Add a margin (allowed less) because:
3016 // - RngBusWidth bits may get lost after every re-enable
3017 // - one entry may just have been pushed into esrng FIFO
when the assertion gets evaluated.
3018 `AS-
SERT_AT_RESET_AND_FINAL(EsrngFifoPushedIntoEsbitOrPosthtFifos_A,
3019 WITH-
IN_MARGIN((esbit_push_bit_cnt_q * RngBusWidth +
3020 postht_from_esrng_push_bit_cnt_q), // actual
3021 es-
rng_push_bit_cnt_q, // expected
3022 (disable_cnt_q +
1) * RngBusWidth, // allowed less
3023 0)) // allowed more
3024
3025 // Count number of bits pushed into postht FIFO from esbit
FIFO.
3026 logic [63:0] postht_from_esbit_push_bit_cnt_d, pos-
tht_from_esbit_push_bit_cnt_q;
3027 assign postht_from_esbit_push_bit_cnt_d =
3028 pfifo_postht_push & pfifo_postht_not_full & rng_bit_en ?
3029 postht_from_esbit_push_bit_cnt_q + RngBusWidth :
3030 postht_from_esbit_push_bit_cnt_q;
3031
3032 // Assert that as many bits got pushed into the postht FIFO
(destination) as into the esbit FIFO
3033 // (source) when the latter was selected as source. Add a
margin (allowed less) because:
3034 // - RngBusWidth bits may get lost after every re-enable

```

```

3035 // - esbit FIFO may be partially full when the assertion
gets evaluated.
3036 `ASSERT_AT_RESET_AND_FINAL(EsbitFifoPushedIntoPosthtFifo_A,
3037 WITH-
IN_MARGIN(postht_from_esbit_push_bit_cnt_q, // actual
3038 es-
bit_push_bit_cnt_q, // expected
3039 (disable_cnt_q +
1) * RngBusWidth - 1, // allowed less
3040 0)) // allowed more
3041
3042 // Assert that as many bits got pushed into the postht FIFO
as got counted from the esrng FIFO or
3043 // the esbit FIFO. This assertion checks more the complete-
ness of the other assertions than the
3044 // design itself.
3045 `AS-
SERT_AT_RESET_AND_FINAL(PosthtFifoPushedFromEsbitOrEsrngFifos_A,
3046 postht_push_bit_cnt_q == (pos-
tht_from_esrng_push_bit_cnt_q +
3047 pos-
tht_from_esbit_push_bit_cnt_q))
3048
3049 // Count number of bits popped from postht FIFO (PostHTWidth
wide output) when bypass mode was
3050 // disabled.
3051 logic [63:0] postht_non_bypass_pop_bit_cnt_d, pos-
tht_non_bypass_pop_bit_cnt_q;
3052 assign postht_non_bypass_pop_bit_cnt_d =
3053 pfifo_postht_pop & pfifo_postht_not_empty & ~es-by-
pass_mode ?
3054 postht_non_bypass_pop_bit_cnt_q + PostHTWidth :
3055 postht_non_bypass_pop_bit_cnt_q;
3056
3057 // Count number of bits pushed into distr FIFO (DistrFifoW-
idth wide input and output).
3058 logic [63:0] distr_non_bypass_push_bit_cnt_d, dis-
tr_non_bypass_push_bit_cnt_q;
3059 assign distr_non_bypass_push_bit_cnt_d =
3060 sfifo_distr_push & sfifo_distr_not_full & ~es-by-
pass_mode ?
3061 distr_non_bypass_push_bit_cnt_q + DistrFifoWidth :
3062 distr_non_bypass_push_bit_cnt_q;
3063
3064 // Assert that as many bits got pushed into the distr FIFO
(destination) as got popped from the
3065 // postht FIFO when bypass mode was disabled (source).
3066 `ASSERT_AT_RESET_AND_FINAL(PosthtFifoPushedIntoDistrFifo_A,
3067 distr_non_bypass_push_bit_cnt_q
== postht_non_bypass_pop_bit_cnt_q)
3068
3069 // Count number of bits popped from distr FIFO (DistrFifoW-
idth wide output) when bypass mode was
3070 // disabled.
3071 logic [63:0] distr_non_bypass_pop_bit_cnt_d, dis-
tr_non_bypass_pop_bit_cnt_q;
3072 assign distr_non_bypass_pop_bit_cnt_d =
3073 sfifo_distr_pop & sfifo_distr_not_empty & ~es-by-
pass_mode ?
3074 distr_non_bypass_pop_bit_cnt_q + DistrFifoWidth :
3075 distr_non_bypass_pop_bit_cnt_q;
3076
3077 // Count number of bits pushed into precon FIFO (ObserveFi-
fowidth wide input, PreCondWidth wide
3078 // output).
3079 logic [63:0] precon_push_bit_cnt_d, precon_push_bit_cnt_q;
3080 assign precon_push_bit_cnt_d = pfifo_precon_push &
pfifo_precon_not_full ?
3081 precon_push_bit_cnt_q + Ob-
serveFifoWidth :
3082 precon_push_bit_cnt_q;
3083
3084 // Assert that as many bits got pushed into the precon FIFO
(destination) as got popped from the
3085 // distr FIFO when bypass mode was disabled (source).
3086 `ASSERT_AT_RESET_AND_FINAL(DistrFifoPushedIntoPreconFifo_A,
3087 precon_push_bit_cnt_q == dis-
tr_non_bypass_pop_bit_cnt_q)
3088
3089 // Track when boot and startup checks are completing.
3090 logic boot_startup_checks_completing;
3091 assign boot_startup_checks_completing =
3092 (u_entropy_src_main_sm.state_q == en-
tropy_src_main_sm_pkg::StartupPass1 &
3093 u_entropy_src_main_sm.ht_done_pulse_i &
3094 ~u_entropy_src_main_sm.ht_fail_pulse_i);
3095
3096 // Track state of boot and startup checks.
3097 typedef enum logic [1:0] {
3098 BscStIncomplete, // checks incomplete
3099 BscStPassed, // checks passed
3100 BscStPushed // entropy from passed checks pushed
3101 } bsc_state_e;
3102 bsc_state_e bsc_state_d, bsc_state_q;
3103 always_comb begin
3104 bsc_state_d = bsc_state_q;
3105 unique case (bsc_state_q)
3106 BscStIncomplete: begin
3107 if (boot_startup_checks_completing) begin

```

```

3108         // Checks have just completed.
3109         bsc_state_d = BscStPassed;
3110     end
3111 end
3112 BscStPassed: begin
3113     if (main_stage_push) begin
3114         // Entropy from passed checks is being pushed.
3115         bsc_state_d = BscStPushed;
3116     end
3117 end
3118 BscStPushed: begin
3119     // Boot and startup checks remained passed and their
entropy pushed until Entropy Source
3120     // gets disabled again (which is handled below).
3121     bsc_state_d = bsc_state_q;
3122 end
3123 default: bsc_state_d = BscStIncomplete;
3124 endcase
3125 // If not enabled, always clear to incomplete.
3126 if (!es_delayed_enable) begin
3127     bsc_state_d = BscStIncomplete;
3128 end
3129 end
3130
3131 // Count number of bits pushed into precon FIFO (ObserveFi-
fowidth wide input, PreCondWidth wide
3132 // output) after boot and startup checks.
3133 logic [63:0] precon_post_startup_push_bit_cnt_d, pre-
con_post_startup_push_bit_cnt_q;
3134 assign precon_post_startup_push_bit_cnt_d =
3135     pfifo_precon_push & pfifo_precon_not_full &
(bsc_state_q != BscStIncomplete) ?
3136     precon_post_startup_push_bit_cnt_q + ObserveFifowidth :
3137     precon_post_startup_push_bit_cnt_q;
3138
3139 // Track when esfinal FIFO gets pushed while bypass mode is
disabled.
3140 logic esfinal_non_bypass_push;
3141 assign esfinal_non_bypass_push = sfifo_esfinal_push &
sfifo_esfinal_not_full & ~es_bypass_mode;
3142
3143 // Count number of bits pushed into esfinal FIFO (SeedLen
bits per push) while bypass mode was
3144 // disabled.
3145 logic [63:0] esfinal_non_bypass_push_cnt_d,
esfinal_non_bypass_push_cnt_q;
3146 assign esfinal_non_bypass_push_cnt_d =
esfinal_non_bypass_push ?
3147     esfinal_non_bypass_push_cnt_q + SeedLen :
3148     esfinal_non_bypass_push_cnt_q;
3149
3150 // Count number of bits pushed into esfinal FIFO (SeedLen
bits per push) after startup checks have
3151 // passed and while bypass mode was disabled.
3152 logic [63:0] esfinal_post_startup_push_bit_cnt_d,
esfinal_post_startup_push_bit_cnt_q;
3153 assign esfinal_post_startup_push_bit_cnt_d =
3154     esfinal_non_bypass_push & (bsc_state_q != BscStIncom-
plete) ?
3155     esfinal_post_startup_push_bit_cnt_q + SeedLen :
3156     esfinal_post_startup_push_bit_cnt_q;
3157
3158 // Assert that all bits pushed into the esfinal FIFO came
from or after passed startup checks.
3159 `ASSERT_AT_RESET_AND_FINAL(EsfinalFifoPushed_A,
3160     esfinal_non_bypass_push_cnt_q ==
esfinal_post_startup_push_bit_cnt_q)
3161
3162 // Track result of health tests after boot and startup
tests.
3163 typedef enum logic [1:0] {
3164     HtStNoResult, // no health test result is currently avail-
able
3165     HtStPassed,   // last health test has passed and entropy
has not propagated from conditioner yet
3166     HtStFailed,   // last health test has failed
3167 } ht_state_e;
3168 ht_state_e ht_state_d, ht_state_q;
3169 always_comb begin
3170     ht_state_d = ht_state_q;
3171     if (bsc_state_q == BscStPushed) begin
3172         if (ht_state_q inside {HtStNoResult, HtStFailed}) begin
3173             if (health_test_done_pulse) begin
3174                 if (!any_fail_pulse && !alert_threshold_fail) begin
3175                     ht_state_d = HtStPassed;
3176                 end else begin
3177                     ht_state_d = HtStFailed;
3178                 end
3179             end
3180         end else if (ht_state_q == HtStPassed) begin
3181             if (main_stage_push_raw) begin
3182                 ht_state_d = HtStNoResult;
3183             end
3184         end else begin
3185             ht_state_d = HtStNoResult;
3186         end
3187         if (ht_state_q == HtStFailed) begin
3188             `ASSERT_I(NoPushAfterFailedHealthTest_A, rst_ni !=
1'b1 || !main_stage_push_raw)

```

```

3189     end
3190 end
3191 // If not enabled, always clear to no result.
3192 if (!es_delayed_enable) begin
3193     ht_state_d = HtStNoResult;
3194 end
3195 end
3196
3197 // Count number of bits that are expected to have gotten
pushed into precon FIFO and into esfinal
3198 // FIFO after boot and startup checks and while bypass mode
was disabled.
3199 logic [63:0] precon_post_startup_exp_push_bit_cnt_d, pre-
con_post_startup_exp_push_bit_cnt_q;
3200 logic [63:0] esfinal_post_startup_exp_push_bit_cnt_d,
esfinal_post_startup_exp_push_bit_cnt_q;
3201 always_comb begin
3202     esfinal_post_startup_exp_push_bit_cnt_d =
esfinal_post_startup_exp_push_bit_cnt_q;
3203     precon_post_startup_exp_push_bit_cnt_d = pre-
con_post_startup_exp_push_bit_cnt_q;
3204     if (bsc_state_q == BscStPassed && bsc_state_d == BscSt-
Pushed) begin
3205         // On the completion of boot and startup checks, SeedLen
bits are expected to be pushed into
3206         // the esfinal FIFO.
3207         esfinal_post_startup_exp_push_bit_cnt_d += SeedLen;
3208     end
3209     if (bsc_state_q != BscStIncomplete &&
health_test_done_pulse) begin
3210         // Once boot and startup checks have completed,
health_test_window bits are
3211         // expected to have gotten pushed into the precon FIFO.
3212         precon_post_startup_exp_push_bit_cnt_d +=
health_test_window;
3213     end
3214     if (ht_state_q == HtStPassed && main_stage_push_raw &&
sfifo_esfinal_not_full) begin
3215         // If none of the health tests failed and the alert
threshold has not been exceeded, SeedLen
3216         // bits are expected to be pushed into the esfinal FIFO
-- unless we expect them to get
3217         // dropped (see above).
3218         esfinal_post_startup_exp_push_bit_cnt_d += SeedLen;
3219     end
3220     // When Entropy Source gets disabled after boot and star-
tup checks have been completed, add the
3221     // number of bits that have been pushed into precon FIFO
since the last conditioner output to
3222     // the expected number of bits.
3223     if ((bsc_state_q != BscStIncomplete) && (bsc_state_d ==
BscStIncomplete)) begin
3224         logic [63:0] diff_;
3225         diff_ = precon_post_startup_push_bit_cnt_q - pre-
con_post_startup_exp_push_bit_cnt_q;
3226         // Assert that the difference is not negative.
3227         `ASSERT_I(PreconPostStartupDiffNonNegative_A,
3228             rst_ni != 1'b1 || (pre-
con_post_startup_push_bit_cnt_q >=
3229                 precon_post_startup_exp_push_bit_cnt_q))
3230     // Assert that the difference is smaller than the number
of bits that would have sufficed to
3231     // get pushed into the conditioner.
3232     `ASSERT_I(PreconPostStartupDiffSmall_A, rst_ni != 1'b1
|| diff_ < health_test_window)
3233     precon_post_startup_exp_push_bit_cnt_d += diff_;
3234 end
3235 end
3236 // This code assumes that `health_test_window` does not
change dynamically; capture that in an
3237 // assertion ensuring it only changes when entropy_src is
not enabled.
3238 `ASSERT(HealthTestWindowStableWhenEnabled_A,
3239     mubi4_test_true_strict(mubi_es_enable) |->
$stable(health_test_window))
3240
3241 // Assert that all bits pushed into the esfinal FIFO after
startup checks were expected.
3242 `ASSERT_AT_RESET_AND_FINAL(EsfinalFifoPushedPostStartup_A,
3243     esfinal_post_startup_push_bit_cnt_q ==
3244     esfinal_post_startup_exp_push_bit_cnt_q)
3245
3246 // Assert that the expected number of bits pushed into the
precon FIFO based on the number of
3247 // outputs of the conditioner matches the actual number of
bits pushed into the precon FIFO after
3248 // startup checks. Add a margin (allowed more) as the simu-
lation may end when bits in the precon
3249 // FIFO have not resulted in conditioner output and Entropy
Source has not been disabled.
3250 logic [63:0] ppspb_allowed_more;
3251 assign ppspb_allowed_more = bsc_state_q != BscStIncomplete ?
health_test_window : '0;
3252 `ASSERT_AT_RESET_AND_FINAL(PreconFifoPushedPostStartup_A,
3253     `WITH-

```

```

IN_MARGIN(precon_post_startup_push_bit_cnt_q, // actual
3254 pre-
con_post_startup_exp_push_bit_cnt_q, // expected
3255
0, // allowed less
3256
ppspb_allowed_more)) // allowed more
3257
3258 always_ff @(posedge clk_i, negedge rst_ni) begin
3259   if (!rst_ni) begin
3260     esbit_push_bit_cnt_q <= '0;
3261     esfinal_non_bypass_push_cnt_q <= '0;
3262     esfinal_post_startup_push_bit_cnt_q <= '0;
3263     esrng_push_bit_cnt_q <= '0;
3264     postht_from_esbit_push_bit_cnt_q <= '0;
3265     postht_from_esrng_push_bit_cnt_q <= '0;
3266     postht_non_bypass_pop_bit_cnt_q <= '0;
3267     postht_push_bit_cnt_q <= '0;
3268     distr_non_bypass_pop_bit_cnt_q <= '0;
3269     distr_non_bypass_push_bit_cnt_q <= '0;
3270     precon_post_startup_push_bit_cnt_q <= '0;
3271     precon_push_bit_cnt_q <= '0;
3272     rng_valid_bit_cnt_q <= '0;
3273   end else if (es_delayed_enable & !
fw_ov_mode_entropy_insert) begin
3274     // All these counters get updated if and only if en-
3275     // tropy_src is enabled and the firmware
3276     // override entropy insertion mode is disabled. Other-
3277     // wise, there are no guarantees on how
3278     // much entropy from the noise source gets dropped due
3279     // to backpressure.
3280     esbit_push_bit_cnt_q <= es-
3281     bit_push_bit_cnt_d;
3282     esfinal_non_bypass_push_cnt_q <=
3283     esfinal_non_bypass_push_cnt_d;
3284     esfinal_post_startup_push_bit_cnt_q <=
3285     esfinal_post_startup_push_bit_cnt_d;
3286     esrng_push_bit_cnt_q <= es-
3287     rng_push_bit_cnt_d;
3288     postht_from_esbit_push_bit_cnt_q <= pos-
3289     tht_from_esbit_push_bit_cnt_d;
3290     postht_from_esrng_push_bit_cnt_q <= pos-
3291     tht_from_esrng_push_bit_cnt_d;
3292     postht_non_bypass_pop_bit_cnt_q <= pos-
3293     tht_non_bypass_pop_bit_cnt_d;
3294     postht_push_bit_cnt_q <= pos-
3295     tht_push_bit_cnt_d;
3296     distr_non_bypass_pop_bit_cnt_q <= dis-
3297     tr_non_bypass_pop_bit_cnt_d;
3298     distr_non_bypass_push_bit_cnt_q <= dis-
3299     tr_non_bypass_push_bit_cnt_d;
3300     precon_post_startup_push_bit_cnt_q <= pre-
3301     con_post_startup_push_bit_cnt_d;
3302     precon_push_bit_cnt_q <= pre-
3303     con_push_bit_cnt_d;
3304     rng_valid_bit_cnt_q <= rng_val-
3305     id_bit_cnt_d;
3306   end
3307 end
3308
3309 always_ff @(posedge clk_i, negedge rst_ni) begin
3310   if (!rst_ni) begin
3311     bsc_state_q <= BscStIncom-
3312     plete;
3313     disable_cnt_q <= '0;
3314     es_delayed_enable_q <= '0;
3315     esfinal_post_startup_exp_push_bit_cnt_q <= '0;
3316     ht_state_q <= HtStNoResult;
3317     precon_post_startup_exp_push_bit_cnt_q <= '0;
3318   end else begin
3319     bsc_state_q <= bsc_state_d;
3320     disable_cnt_q <= dis-
3321     able_cnt_d;
3322     es_delayed_enable_q <=
3323     es_delayed_enable_d;
3324     esfinal_post_startup_exp_push_bit_cnt_q <=
3325     esfinal_post_startup_exp_push_bit_cnt_d;
3326     ht_state_q <= ht_state_d;
3327     precon_post_startup_exp_push_bit_cnt_q <= pre-
3328     con_post_startup_exp_push_bit_cnt_d;
3329   end
3330 end
3331 `endif
3332
3333 endmodule

```

8. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_enable_delay.sv (111 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 // Description: Logic module indicate ongoing activity of
6 // after disablement of entropy_src_core
7 // The entropy_src has a great deal of internal state, and

```

```

when shutting down most of this internal
8 // state should be cleared. (The two notable exceptions are
the health test statistics, which are
9 // only cleared at the next enable, and the SHA3 conditioning
sponge which accumulates unused
10 // entropy until a seed is generated). There are delays as
incoming RNG data is processed, and for
11 // ease of verification we insist all elements of the data
pipeline are cleared consistently. This
12 // means that once an RNG sample enters the pipeline, that
sample should be reflected in the health
13 // tests. The SHA3 conditioner is also assumed to successfully
absorb every 64-bits that enters the
14 // module.
15 //
16 // To achieve this consistency goal the entropy_src delays the
clearing of internal data buffers
17 // and the state machine until:
18 // 1. Any unprocessed data has been counted at the health
checks (regardless of the mode)
19 // 2. Any RNG data bound for the SHA conditioner has been re-
ceived at the conditioner.
20 // 3. Any ongoing SHA processing operations have completed,
and the main FSM has been forced
21 // back to idle.
22 //
23 // This block creates a modified version of the enable pulse
which:
24 // 1. Postpones the disable event until any flowing data has
passed through the RNG, ESBIT, POSTHT
25 // and DISTR FIFOs. If backpressure is encountered at the
Precon FIFO, the stalled data can
26 // be discarded, and so has a maximum time limit of
MaxFifoWait=4 clocks is given for this
27 // check.
28 // 2. Once the disable signal is received, the rising edge
does not occur until:
29 // 2a. One clock after the falling edge OR
30 // 2b. One clock after the SHA engine completes,
31 // Whichever comes later.
32 //
33 module entropy_src_enable_delay import prim_mubi_pkg::*; (
34   input logic clk_i,
35   input logic rst_ni,
36
37   input logic enable_i,
38
39   // Unconsumed FIFO inputs
40   input logic esrng_fifo_not_empty_i,
41   input logic esbit_fifo_not_empty_i,
42   input logic postht_fifo_not_empty_i,
43   input logic distr_fifo_not_empty_i,
44
45   // SHA3 conditioner inputs
46   input logic sha3_block_busy_i,
47   input logic sha3_block_processed_i,
48
49   input logic bypass_mode_i,
50
51   output logic enable_o
52 );
53
54 // Maximum number of cycles to wait for FIFOs to clear out.
55 // Set to 4 to allow one cycle for each FIFO in the
56 pipeline.
57 localparam int MaxFifoWait = 4;
58
59 logic suppress_reenable;
60 logic extend_enable;
61
62 logic data_in_flight;
63 logic [3:0] fifos_not_empty;
64
65 // Flops
66 logic [MaxFifoWait - 1:0] fifo_timer_d, fifo_timer_q;
67 logic sha3_active_post_en_d, sha3_act-
ive_post_en_q;
68 logic sha3_block_processed_q;
69 logic extend_enable_q;
70
71 always_ff @(posedge clk_i or negedge rst_ni) begin
72   if (!rst_ni) begin
73     sha3_active_post_en_q <= 1'b0;
74     fifo_timer_q <= '0;
75     sha3_block_processed_q <= 1'b0;
76     extend_enable_q <= 1'b0;
77   end else begin
78     sha3_active_post_en_q <= sha3_active_post_en_d;
79     fifo_timer_q <= fifo_timer_d;
80     sha3_block_processed_q <= sha3_block_processed_i;
81     extend_enable_q <= extend_enable;
82   end
83 end
84
85 // Output definition
86 assign enable_o = (enable_i & ~suppress_reenable) | ex-
tend_enable;
87 // In flight data monitoring.

```



```

88 // The `fifo_timer` is a small shift register to count out
the maximum number of cycles to wait

89 // for the FIFOs to drain. Since this timer is very small (3
cycles), it is implemented as a shift
90 // register.
91 assign fifo_timer_d = enable_i ? {MaxFifoWait{1'b1}} :
{fifo_timer_q[MaxFifoWait-2:0], 1'b0};
92 assign fifos_not_empty = {esrng_fifo_not_empty_i, es-
bit_fifo_not_empty_i,
93                               !bypass_mode_i & pos-
tnt_fifo_not_empty_i,
94                               !bypass_mode_i & dis-
tr_fifo_not_empty_i};
95 assign data_in_flight = |fifo_timer_q && |fifos_not_empty;
96
97 // Extend the enable by at least one clock to give the FSM
time to receive any last
98 // Health checks.

99 assign extend_enable = ((fifo_timer_q[0] | data_in_flight) &
~enable_i);
100
101 // Pulse to extend from the falling edge of the incoming en-
able pulse until one cycle after the
102 // SHA engine has finished processing the current block.
103 assign sha3_active_post_en_d = sha3_block_busy_i && !en-
able_i ? 1'b1 :
104                               sha3_block_processed_q ?
1'b0 :
105                               sha3_active_post_en_q;
106
107 // Force the output to be low until sha3_active_post_en_q
falls or
108 // for one more cycle after the falling edge of ex-
tend_enable
109 assign suppress_reenable = sha3_active_post_en_q | ex-
tend_enable_q;
110
111 endmodule

```

9. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_field_en.sv (52 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Supports writing a field that enables
6 // a function within a module.
7 // Requirements are that the function will
8 // only be enabled when the field is written
9 // the it is the inverse of the current field
10 // setting. The can only toggle between the
11 // the on value and the off value.

12
13 module entropy_src_field_en #(
14     parameter int FieldW = 4,
15     parameter int FieldEnVal = 'ha
16 ) (
17     input logic clk_i ,
18     input logic rst_ni,
19     input logic wvalid_i,
20     input logic [FieldW-1:0] wdata_i,
21     output logic enable_o
22 );
23
24 localparam logic [FieldW-1:0] FieldEnValBit =
FieldW'(FieldEnVal);
25 localparam logic [FieldW-1:0] FieldEnValBitRevert =
~FieldEnValBit;
26
27 // signal
28 logic field_update;
29
30 // flops
31 logic [FieldW-1:0] field_q, field_d;
32
33 always_ff @(posedge clk_i or negedge rst_ni) begin
34     if (!rst_ni) begin
35         field_q <= FieldEnValBitRevert;
36     end else begin
37         field_q <= field_d;
38     end
39 end
40
41 assign field_update = wvalid_i && (field_q == ~wdata_i) &&
((wdata_i == FieldEnValBit) ||
(wdata_i == FieldEnValBitRevert));
42
43 assign field_d = field_update ? wdata_i : field_q;
44
45 assign enable_o = (field_q == FieldEnValBit);
46
47
48
49
50

```

```

51
52 endmodule

```

10. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_main_sm.sv (287 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: entropy_src main state machine module
6 //
7 // determines when new entropy is ready to be forwarded
8
9 `include "prim_assert.sv"
10
11 module entropy_src_main_sm
12     import entropy_src_main_sm_pkg::*;
13 (
14     input logic clk_i,
15     input logic rst_ni,
16
17     input logic enable_i,
18     input logic fw_ov_ent_insert_i,
19     input logic fw_ov_sha3_start_i,
20     input logic ht_done_pulse_i,
21     input logic pd_cntr_zero_i,
22     input logic ht_fail_pulse_i,
23     input logic alert_thresh_fail_i,
24     output logic alert_cntr_clr_ok_o,
25     input logic bypass_mode_i,
26     input logic bypass_stage_rdy_i,
27     input logic sha3_state_vld_i,
28     output logic main_stage_push_o,
29     output logic bypass_stage_pop_o,
30     output logic boot_phase_done_o,
31     output logic sha3_start_o,
32     output logic sha3_process_o,
33     output prim_mubi_pkg::mubi4_t sha3_done_o,
34     input logic local_escalate_i,
35     output logic main_sm_alert_o,
36     output logic main_sm_idle_o,
37     output logic [StateWidth-1:0] main_sm_state_o,
38     output logic main_sm_err_o
39 );
40
41 // The definition of state_e, the sparse FSM state enum, is
in entropy_src_main_sm_pkg.sv
42 state_e state_d, state_q;
43
44 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
state_e, Idle)
45
46 assign main_sm_state_o = state_q;
47
48 always_comb begin
49     state_d = state_q;
50     alert_cntr_clr_ok_o = 1'b0;
51     main_stage_push_o = 1'b0;
52     bypass_stage_pop_o = 1'b0;
53     boot_phase_done_o = 1'b0;
54     sha3_start_o = 1'b0;
55     sha3_process_o = 1'b0;
56     sha3_done_o = prim_mubi_pkg::MuBi4False;
57     main_sm_alert_o = 1'b0;
58     main_sm_idle_o = 1'b0;
59     main_sm_err_o = 1'b0;
60     unique case (state_q)
61         Idle: begin
62             main_sm_idle_o = 1'b1;
63             if (enable_i) begin
64                 // running fw override mode and in sha3 mode
65                 if (fw_ov_ent_insert_i && !bypass_mode_i) begin
66                     sha3_start_o = 1'b1;
67                     if (fw_ov_sha3_start_i) begin
68                         state_d = FWInsertMsg;
69                     end else begin
70                         state_d = FWInsertStart;
71                     end
72                 // running in bypass_mode and not fw override mode
73             end else if (bypass_mode_i && !fw_ov_ent_insert_i)
begin
74                 state_d = BoothRunning;
75                 // running in bypass_mode and fw override mode
76             end else if (bypass_mode_i && fw_ov_ent_insert_i)
begin
77                 state_d = Idle;
78             end else begin
79                 state_d = StartupHTStart;
80             end
81         end
82         BoothRunning: begin
83             alert_cntr_clr_ok_o = 1'b1;
84             if (!enable_i) begin
85                 state_d = Idle;
86             end else if (ht_done_pulse_i) begin
87                 if (ht_fail_pulse_i) begin
88

```

```

89         if (bypass_stage_rdy_i) begin
90             // Remove failed data
91             bypass_stage_pop_o = 1'b1;
92         end
93         if (alert_thresh_fail_i) begin
94             state_d = AlertState;
95         end else begin
96             state_d = Idle;
97         end
98     end else begin
99         // Window sizes other than 384 bits (the seed
length) are currently not tested nor
100         // supported in bypass or boot-time mode.
101         state_d = BootPostHTChk;
102     end
103 end
104 end
105 BootPostHTChk: begin
106     if (!enable_i) begin
107         state_d = Idle;
108     end else begin
109         if (!bypass_stage_rdy_i) begin
110             end else begin
111                 bypass_stage_pop_o = 1'b1;
112                 main_stage_push_o = 1'b1;
113                 state_d = BootPhaseDone;
114             end
115         end
116     end
117 BootPhaseDone: begin
118     boot_phase_done_o = 1'b1;
119     if (!enable_i) begin
120         state_d = Idle;
121     end
122 // Even when stalled we keep monitoring for alerts and
maintaining alert statistics.
123 // However, we don't signal alerts or clear HT stats
in FW_OV mode.
124     if (!fw_ov_ent_insert_i) begin
125         alert_cntr_clr_ok_o = 1'b1;
126         if (ht_done_pulse_i && alert_thresh_fail_i) begin
127             state_d = AlertState;
128         end
129     end
130 end
131 StartupHTStart: begin
132     if (!enable_i) begin
133         state_d = Idle;
134     end else begin
135         sha3_start_o = 1'b1;
136         state_d = StartupPhase1;
137     end
138 end
139 StartupPhase1: begin
140     if (!enable_i) begin
141         state_d = Idle;
142     end else begin
143         alert_cntr_clr_ok_o = 1'b1;
144         if (ht_done_pulse_i) begin
145             if (ht_fail_pulse_i) begin
146                 state_d = StartupFail1;
147             end else begin
148                 state_d = StartupPass1;
149             end
150         end
151     end
152 end
153 StartupPass1: begin
154     if (!enable_i) begin
155         state_d = Idle;
156     end else begin
157         alert_cntr_clr_ok_o = 1'b1;
158         if (ht_done_pulse_i) begin
159             if (ht_fail_pulse_i) begin
160                 state_d = StartupFail1;
161             end else begin
162                 // We've now passed two consecutive test windows
of the configured window length.
163                 // Next, we're going to compress the collected
entropy to produce a single seed.
164                 state_d = Sha3Process;
165             end
166         end
167     end
168 end
169 StartupFail1: begin
170     if (!enable_i) begin
171         state_d = Idle;
172     end else begin
173         alert_cntr_clr_ok_o = 1'b1;
174         if (ht_done_pulse_i) begin
175             if (ht_fail_pulse_i) begin
176                 // Failed two consecutive tests
177                 state_d = AlertState;
178             end else begin
179                 state_d = StartupPass1;
180             end
181         end
182     end

```

```

183     end
184     ContHTStart: begin
185         if (!enable_i) begin
186             state_d = Idle;
187         end else begin
188             // Send the Start trigger to the SHA3 engine. After
this point, collected raw entropy is
189             // going to be moved into the conditioner in chunks
of 64 bits. The SHA3 engine itself
190             // will hash the received raw entropy whenever
either of the following two conditions are
191             // met:
192             // 1) 13 64-bit chunks have been received (13 x 64 =
832 = the rate or block size of
193             // the employed SHA3-384 engine). This condition
is checked internally to the SHA3
194             // engine.
195             // 2) The total amount of received entropy is equal
to the configured health test window
196             // size. This condition is checked by this FSM
which then triggers the SHA3 engine
197             // using the sha3_process_o signal.
198             sha3_start_o = 1'b1;
199             state_d = ContHTRunning;
200         end
201     end
202     ContHTRunning: begin
203         if (!enable_i) begin
204             state_d = Idle;
205         end else begin
206             alert_cntr_clr_ok_o = 1'b1;
207             if (ht_done_pulse_i) begin
208                 // We've finished testing the current window and
all the collected and tested entropy
209                 // has been forwarded to the SHA3 engine and has
at least partially been absorbed.
210                 if (alert_thresh_fail_i) begin
211                     state_d = AlertState;
212                 end else if (!ht_fail_pulse_i) begin
213                     // Move forward and get the conditioner ready to
finish the absorption process.
214                     state_d = Sha3Process;
215                 end
216             end
217         end
218     end
219     FWInsertStart: begin
220         if (!enable_i) begin
221             state_d = Idle;
222         end else if (fw_ov_sha3_start_i) begin
223             state_d = FWInsertMsg;
224         end
225     end
226     FWInsertMsg: begin
227         if (!enable_i) begin
228             state_d = Idle;
229         end else if (!fw_ov_sha3_start_i) begin
230             state_d = Sha3Process;
231         end
232     end
233     Sha3Process: begin
234         // Wait for words belonging to the current window to
flow into the conditioner before
235         // triggering the final absorption operation of the
SHA3 engine.
236         if (pd_cntr_zero_i) begin
237             sha3_process_o = 1'b1;
238             state_d = Sha3Valid;
239         end
240     end
241     Sha3Valid: begin
242         if (sha3_state_vld_i) begin
243             state_d = Sha3Done;
244         end
245     end
246     Sha3Done: begin
247         if (!enable_i) begin
248             sha3_done_o = prim_mubi_pkg::MuBi4True;
249             state_d = Sha3MsgDone;
250         end else begin
251             // Push the digest produced by the SHA3 engine into
the final FIFO and clear the
252             // internal state of the SHA3 engine to start from
scratch for the next seed.
253             sha3_done_o = prim_mubi_pkg::MuBi4True;
254             main_stage_push_o = 1'b1;
255             state_d = Sha3MsgDone;
256         end
257     end
258     Sha3MsgDone: begin
259         if (!enable_i || fw_ov_ent_insert_i) begin
260             state_d = Idle;
261         end else begin
262             state_d = ContHTStart;
263         end
264     end
265     AlertState: begin
266         main_sm_alert_o = 1'b1;
267         state_d = AlertHang;

```

```

268     end
269     AlertHang: begin
270         if (!enable_i) begin
271             state_d = Idle;
272         end
273     end
274     Error: begin
275         main_sm_err_o = 1'b1;
276     end
277     default: begin
278         state_d = Error;
279         main_sm_err_o = 1'b1;
280     end
281 endcase
282 if (local_escalate_i) begin
283     state_d = Error;
284 end
285 end
286
287 endmodule

```

11. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_main_sm_pkg.sv (54 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // State definitions for entropy_src_main_sm, provided as a
  separate package for use in DV
6
7 package entropy_src_main_sm_pkg;
8
9 // Encoding generated with:
10 // $ ./util/design/parses-fsm-encode.py -d 3 -m 20 -n 9 \
11 //   -s 2359261201 --language=sv
12 //
13 // Hamming distance histogram:
14 //
15 // 0: --
16 // 1: --
17 // 2: --
18 // 3: ||||| (18.42%)
19 // 4: ||||| (29.47%)
20 // 5: ||||| (26.84%)
21 // 6: ||||| (16.32%)
22 // 7: ||| (6.84%)
23 // 8: | (2.11%)
24 // 9: --
25 //
26 // Minimum Hamming distance: 3
27 // Maximum Hamming distance: 8
28 // Minimum Hamming weight: 1
29 // Maximum Hamming weight: 8
30 //
31 localparam int StateWidth = 9;
32 typedef enum logic [StateWidth-1:0] {
33     Idle = 9'b011110101, // idle
34     BootHTRunning = 9'b11101010, // boot mode, wait for
  health test done pulse
35     BootPostHTChk = 9'b101101110, // boot mode, wait for post
  health test packer not empty state
36     BootPhaseDone = 9'b010001110, // boot mode, stay here un-
  til master enable is off
37     StartupHTStart = 9'b000101100, // startup mode, pulse the
  sha3 start input
38     StartupPhase1 = 9'b100000001, // startup mode, look for
  first test pass/fail
39     StartupPass1 = 9'b110100101, // startup mode, look for
  first test pass/fail, done if pass
40     StartupFail1 = 9'b000010111, // startup mode, look for
  second fail, alert if fail
41     ContHTStart = 9'b001000000, // continuous test mode,
  pulse the sha3 start input
42     ContHTRunning = 9'b110100010, // continuous test mode,
  wait for health test done pulse
43     FWInsertStart = 9'b011000011, // fw ov mode, start the
  sha3 block
44     FWInsertMsg = 9'b001011001, // fw ov mode, insert fw
  message into sha3 block
45     Sha3MsgDone = 9'b100001111, // sha3 mode, sha3 state
  cleared, go to continuous check mode
46     Sha3Process = 9'b011111000, // sha3 mode, pulse the
  sha3 process input
47     Sha3Valid = 9'b010111111, // sha3 mode, wait for sha3
  valid indication
48     Sha3Done = 9'b110011000, // sha3 mode, capture sha3
  result, pulse done input
49     AlertState = 9'b111001101, // if some alert condition
  occurs, pulse an alert indication
50     AlertHang = 9'b111111011, // after pulsing alert sig-
  nal, hang here until sw handles
51     Error = 9'b001110011 // illegal state reached
  and hang
52 } state_e;

```

```

53
54 endpackage

```

12. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_markov_ht.sv (168 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: entropy_src Markov health test module
6 //
7
8 module entropy_src_markov_ht #(
9     parameter int RegWidth = 16,
10    parameter int RngBusWidth = 4,
11    parameter int RngBusBitSelWidth = 2
12) (
13    input logic clk_i,
14    input logic rst_ni,
15
16    // ins req interface
17    input logic [RngBusWidth-1:0] entropy_bit_i,
18    input logic entropy_bit_vld_i,
19    input logic rng_bit_en_i,
20    input logic [RngBusBitSelWidth-1:0] rng_bit_sel_i,
21    input logic clear_i,
22    input logic active_i,
23    input logic [RegWidth-1:0] thresh_hi_i,
24    input logic [RegWidth-1:0] thresh_lo_i,
25    input logic window_wrap_pulse_i,
26    input logic threshold_scope_i,
27    output logic [RegWidth-1:0] test_cnt_hi_o,
28    output logic [RegWidth-1:0] test_cnt_lo_o,
29    output logic test_fail_hi_pulse_o,
30    output logic test_fail_lo_pulse_o,
31    output logic count_err_o
32);
33
34 // signals
35 logic [RngBusWidth-1:0] samples_no_match_pulse;
36 logic [RegWidth-1:0] pair_cntr_max;
37 logic [RegWidth-1:0] pair_cntr_min;
38 pair_cntr_min_tmp;
39 logic [RegWidth-1:0] pair_cntr_sum;
40 logic [RngBusWidth-1:0][RegWidth-1:0] pair_cntr;
41 logic [RegWidth-1:0] rng_bit_cnt;
42 logic [RngBusWidth-1:0] pair_cntr_err;
43
44 // flops
45 logic toggle_q, toggle_d;
46 logic [RngBusWidth-1:0] prev_sample_q, prev_sample_d;
47
48 always_ff @(posedge clk_i or negedge rst_ni)
49     if (!rst_ni) begin
50         toggle_q <= '0;
51         prev_sample_q <= '0;
52     end else begin
53         toggle_q <= toggle_d;
54         prev_sample_q <= prev_sample_d;
55     end
56
57 // Markov Test
58 //
59 // Test operation
60 // This test will look at pairs of bit levels per bit-
  stream. A counter for
61 // stream will only count when the pair equals 0b01 or
  0b10.
62
63 for (genvar sh = 0; sh < RngBusWidth; sh = sh+1) begin :
  gen_cntrs
64
65     // bit sampler
66     assign prev_sample_d[sh] = (!active_i || clear_i) ? '0 :
67     window_wrap_pulse_i ? '0 :
68     entropy_bit_vld_i ? en-
69     tropy_bit_i[sh] :
70     prev_sample_q[sh];
71
72     // pair check
73     assign samples_no_match_pulse[sh] = entropy_bit_vld_i &&
  toggle_q &&
74     (prev_sample_q[sh] == !entropy_bit_i[sh]);
75
76     // pair counter
77     prim_count #(
78         .Width(RegWidth)
79     ) u_prim_count_pair_cntr (
80         .clk_i,
81         .rst_ni,
82         .clr_i(window_wrap_pulse_i),
83         .set_i(!active_i || clear_i),
84         .set_cnt_i(RegWidth'(0)),
85         .incr_en_i(samples_no_match_pulse[sh]),
86         .decr_en_i(1'b0),

```

```

87     .step_i(RegWidth'(1)),
88     .commit_i(1'b1),
89     .cnt_o(pair_cntr[sh]),
90     .cnt_after_commit_o(),
91     .err_o(pair_cntr_err[sh])
92 );
93 end : gen_cntrs
94
95 // create a toggle signal to sample pairs with
96 assign toggle_d = (!active_i || clear_i) ? '0 :
97     window_wrap_pulse_i ? '0 :
98     entropy_bit_vld_i ? (!toggle_q) :
99     toggle_q;
100
101 // determine the highest counter pair counter value
102 prim_max_tree #(
103     .NumSrc(RngBusWidth),
104     .Width(RegWidth)
105 ) u_max (
106     .clk_i      (clk_i),
107     .rst_ni     (rst_ni),
108     .values_i   (pair_cntr),
109     .valid_i    ({RngBusWidth{1'b1}}),
110     .max_value_o (pair_cntr_max),
111     .max_idx_o  (),
112     .max_valid_o ()
113 );
114
115 // determine the lowest counter pair counter value
116 // Negate the inputs and outputs of prim_max_tree to find
117 the minimum
118 // For this unsigned application, one's complement negation
119 (i.e. logical inversion) is fine.
120 prim_max_tree #(
121     .NumSrc(RngBusWidth),
122     .Width(RegWidth)
123 ) u_min (
124     .clk_i      (clk_i),
125     .rst_ni     (rst_ni),
126     .values_i   (~pair_cntr),
127     .valid_i    ({RngBusWidth{1'b1}}),
128     .max_value_o (pair_cntr_min_tmp),
129     .max_idx_o  (),
130     .max_valid_o ()
131 );
132 // Invert the output back.
133 assign pair_cntr_min = ~pair_cntr_min_tmp;
134
135 prim_sum_tree #(
136     .NumSrc(RngBusWidth),
137     .InWidth(RegWidth)
138 ) u_sum (
139     .clk_i      (clk_i),
140     .rst_ni     (rst_ni),
141     .values_i   (pair_cntr),
142     .valid_i    ({RngBusWidth{1'b1}}),
143     .sum_value_o (pair_cntr_sum),
144     .sum_valid_o ()
145 );
146 always_comb begin
147     rng_bit_cnt = '0;
148
149     for (int i = 0; i < RngBusWidth; i++) begin
150         if (rng_bit_sel_i == i) begin
151             rng_bit_cnt = pair_cntr[i];
152         end
153     end
154 end
155
156 assign test_cnt_hi_o = rng_bit_en_i ? rng_bit_cnt :
157     threshold_scope_i ? pair_cntr_sum :
158     pair_cntr_max;
159 assign test_cnt_lo_o = rng_bit_en_i ? rng_bit_cnt :
160     threshold_scope_i ? pair_cntr_sum :
161     pair_cntr_min;
162
163 // the pulses will be only one clock in length
164 assign test_fail_hi_pulse_o = active_i && win-
165     dow_wrap_pulse_i && (test_cnt_hi_o > thresh_hi_i);
166 assign test_fail_lo_pulse_o = active_i && win-
167     dow_wrap_pulse_i && (test_cnt_lo_o < thresh_lo_i);
168 assign count_err_o = (!pair_cntr_err);
169 endmodule

```

13. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_pkg.sv (80 lines)

1 // Copyright lowRISC contributors (OpenTitan project).

```

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 details.
4 // SPDX-License-Identifier: Apache-2.0
5 //
6 package entropy_src_pkg;
7
8 //-----
9 // Entropy Interface
10 //-----
11
12 parameter int CSRNG_BUS_WIDTH = 384;
13 parameter int FIPS_BUS_WIDTH = 1;
14 parameter int FIPS_CSRNG_BUS_WIDTH = FIPS_BUS_WIDTH +
15     CSRNG_BUS_WIDTH;
16
17 // For a data width of N, there are 2^N buckets in the buck-
18 et health test, each with its own
19 // counter. To make this health test scale reasonably well
20 with `RngBusWidth`, we limit the bucket
21 // health test to 4 bit. If `RngBusWidth` is larger than 4,
22 the bucket health test gets
23 // instantiated multiple times, once per 4 bit.
24 parameter int BucketHtDataMaxWidth = 4;
25
26 function automatic integer bucket_ht_data_width(integer
27     rng_bus_width);
28     return rng_bus_width >= BucketHtDataMaxWidth ? BucketHt-
29     DataMaxWidth : rng_bus_width;
30 endfunction
31
32 function automatic integer num_bucket_ht_inst(integer
33     rng_bus_width);
34     return prim_util_pkg::ceil_div(rng_bus_width, buck-
35     et_ht_data_width(rng_bus_width));
36 endfunction
37
38 // es entropy i/f
39 typedef struct packed {
40     logic es_ack;
41     logic [CSRNG_BUS_WIDTH-1:0] es_bits;
42     logic [FIPS_BUS_WIDTH-1:0] es_fips;
43 } entropy_src_hw_if_rsp_t;
44
45 typedef struct packed {
46     logic es_req;
47 } entropy_src_hw_if_req_t;
48
49 parameter entropy_src_hw_if_req_t EN-
50 TROPY_SRC_HW_IF_REQ_DEFAULT = '{default: '0};
51 parameter entropy_src_hw_if_rsp_t EN-
52 TROPY_SRC_HW_IF_RSP_DEFAULT = '{default: '0};
53
54 // external health test i/f
55 typedef struct packed {
56     logic rng_bit_en;
57     logic clear;
58     logic active;
59     logic [15:0] thresh_hi;
60     logic [15:0] thresh_lo;
61     logic window_wrap_pulse;
62     logic threshold_scope;
63 } entropy_src_xht_meta_req_t;
64
65 typedef struct packed {
66     logic [15:0] test_cnt_hi;
67     logic [15:0] test_cnt_lo;
68     logic continuous_test;
69     logic test_fail_hi_pulse;
70     logic test_fail_lo_pulse;
71 } entropy_src_xht_meta_rsp_t;
72
73 parameter entropy_src_xht_meta_req_t EN-
74 TROPY_SRC_XHT_META_REQ_DEFAULT = '{default: '0};
75 parameter entropy_src_xht_meta_rsp_t EN-
76 TROPY_SRC_XHT_META_RSP_DEFAULT =
77     '{test_cnt_lo: 16'hffff, default: '0};
78
79 parameter int HT_WATERMARK_NUM_WIDTH = 4;
80 typedef enum logic [HT_WATERMARK_NUM_WIDTH-1:0] {
81     REPCNT_HI = 0,
82     REPCNTS_HI = 1,
83     ADAPT_HI = 2,
84     ADAPT_LO = 3,
85     BUCKET_HI = 4,
86     MARKOV_HI = 5,
87     MARKOV_LO = 6,
88     EXTHI_HI = 7,
89     EXTHI_LO = 8
90 } ht_watermark_num_e;
91
92 endpackage : entropy_src_pkg

```

OpenTitan OTBN -- 빅넘버 코프로세서

What this is. A programmable coprocessor for wide-integer arithmetic, with its own instruction set, register file and modular multiplier.

Why it exists. Because **public-key cryptography needs modular exponentiation on operands of hundreds or thousands of bits**. A general-purpose CPU is too slow; a fixed-function engine cannot follow algorithm changes such as curve replacement or the transition to post-quantum schemes.

Undergraduate review · EECS concepts. Modular arithmetic; Montgomery multiplication (modular reduction without division); constant-time implementation; instruction set design.

Graduate · Practice note. Fixed-function versus programmable is a central IP business decision. **Where the standards move, programmability wins** — which is the situation in cryptography right now.

4 files • 5,083 lines

API — otbn.sv

Table 895. lowRISC_opentitan/hw/ip/otbn/rtl/otbn.sv — module / interface / package (1개)

이름	종류
otbn	module

Table 896. lowRISC_opentitan/hw/ip/otbn/rtl/otbn.sv — parameter (14개)

파라미터	형/폭	기본값
Stub	bit	1'b0
RegFile	regfile_e	RegFileFF
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
RndCnstUrndPrngSeed	urnd_prng_seed_t	RndCnstUrndPrngSeedDefault
RndCnstUrndPerm	urnd_perm_t	RndCnstUrndPermDefault
SecSkipUrndReseedAtStart	bit	1'b0
SecFixMaiOpSeq	bit	1'b0
SecFixMacOpSeq	bit	1'b0
FeatStubMai	bit	1'b0
RndCnstBnMacUrndPerm	bn_mac_urnd_perm_t	RndCnstBnMacUrndPermDefault
RndCnstMaiUrndPerm	mai_urnd_perm_t	RndCnstMaiUrndPermDefault
RndCnstOtbnKey	otp_ctrl_pkg::otbn_key_t	RndCnstOtbnKeyDefault
RndCnstOtbnNonce	otp_ctrl_pkg::otbn_nonce_t	RndCnstOtbnNonceDefault

Table 897. lowRISC_opentitan/hw/ip/otbn/rtl/otbn.sv — localparam (13개)

상수	형/폭	값
ImemSizeByte	int	int'(otbn_reg_pkg::OTBN_IMEM_SIZE)
DmemBusSizeByte	int	int'(otbn_reg_pkg::OTBN_DMEM_SIZE)
DmemSizeByte	int	DmemBusSizeByte + DmemScratchSizeByte
ImemAddrWidth	int	vbits(ImemSizeByte)
DmemBusAddrWidth	int	vbits(DmemBusSizeByte)
DmemAddrWidth	int	vbits(DmemSizeByte)
LcEscalateCopies	int unsigned	2
ImemSizeWords	int	ImemSizeByte / 4
ImemIndexWidth	int	vbits(ImemSizeWords)
DmemSizeWords	int	DmemSizeByte / (WLEN / 8)
DmemIndexWidth	int	vbits(DmemSizeWords)
DmemBusSizeWords	int	DmemBusSizeByte / (WLEN / 8)
DmemBusIndexWidth	int	vbits(DmemBusSizeWords)

Table 898. lowRISC_opentitan/hw/ip/otbn/rtl/otbn.sv — 포트 (28개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
idle_o	output	prim_mubi_pkg::mubi4_t
intr_done_o	output	logic
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
lc_rma_req_i	input	lc_ctrl_pkg::lc_tx_t
lc_rma_ack_o	output	lc_ctrl_pkg::lc_tx_t
ram_cfg_imem_i	input	prim_ram_1p_pkg::ram_1p_cfg_req_t
ram_cfg_dmem_i	input	prim_ram_1p_pkg::ram_1p_cfg_req_t
ram_cfg_imem_o	output	prim_ram_1p_pkg::ram_1p_cfg_rsp_t
ram_cfg_dmem_o	output	prim_ram_1p_pkg::ram_1p_cfg_rsp_t
clk_edn_i	input	
rst_edn_ni	input	
edn_rnd_o	output	edn_pkg::edn_req_t
edn_rnd_i	input	edn_pkg::edn_rsp_t

포트	방향	형/폭
edn_urnd_o	output	edn_pkg::edn_req_t
edn_urnd_i	input	edn_pkg::edn_rsp_t
clk_otp_i	input	
rst_otp_ni	input	
otbn_otp_key_o	output	otp_ctrl_pkg::otbn_otp_key_req_t
otbn_otp_key_i	input	otp_ctrl_pkg::otbn_otp_key_rsp_t
keymgr_key_i	input	keymgr_pkg::otbn_key_req_t
kmac_data_o	output	kmac_pkg::app_req_t
kmac_data_i	input	kmac_pkg::app_rsp_t

Table 899. lowRISC_opentitan/hw/ip/otbn/rtl/otbn.sv — #define / `define 매크로 (1개)

매크로	인자	정의
DEF_FAC_BIT	(NAME)	\

API — otbn_alu_base.sv

Table 900. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_alu_base.sv — module / interface / package (1개)

이름	종류
otbn_alu_base	module

Table 901. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_alu_base.sv — 포트 (6개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
operation_i	input	alu_base_operation_t
comparison_i	input	alu_base_comparison_t
operation_result_o	output	logic [31:0]
comparison_result_o	output	logic

API — otbn_alu_bignum.sv

Table 902. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_alu_bignum.sv — module / interface / package (1개)

이름	종류
otbn_alu_bignum	module

Table 903. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_alu_bignum.sv — localparam (29개)

상수	형/폭	값
NFlagsSrcs	int	5
NNoIntgIspr	int	15

상수	형/폭	값
IsprRndNoIntg	int	0
IsprUrndNoIntg	int	1
IsprMaiCtrlNoIntg	int	2
IsprMaiStatusNoIntg	int	3
IsprFlagsNoIntg	int	4
IsprKeyS0LNoIntg	int	5
IsprKeyS0HNoIntg	int	6
IsprKeyS1LNoIntg	int	7
IsprKeyS1HNoIntg	int	8
IsprKmacStatusNoIntg	int	9
IsprKmacCfgNoIntg	int	10
IsprKmacStrbNoIntg	int	11
IsprInsnCntNoIntg	int	12
IsprUrndStateNoIntg	int	13
IsprUrndStatusNoIntg	int	14
NIntgIspr	int	10
IsprModIntg	int	0
IsprAccIntg	int	1
IsprMaiResS0Intg	int	2
IsprMaiResS1Intg	int	3
IsprMaiIn0S0Intg	int	4
IsprMaiIn0S1Intg	int	5
IsprMaiIn1S0Intg	int	6
IsprMaiIn1S1Intg	int	7
IsprKmacDataS0Intg	int	8
IsprKmacDataS1Intg	int	9
IsprNoIntg	int	10

Table 904. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_alu_bignum.sv — 포트 (82개) — 앞 60개만 표시(전체 82개, 본문 소스 참조)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
operation_i	input	alu_bignum_operation_t
operation_valid_i	input	logic

포트	방향	형/폭
operation_commit_i	input	logic
operation_result_o	output	logic [WLEN-1:0]
selection_flag_o	output	logic
alu_bignum_predec_i	input	alu_bignum_predec_t
ispr_bignum_predec_i	input	ispr_bignum_predec_t
ispr_addr_i	input	ispr_e
ispr_base_wdata_i	input	logic [31:0]
ispr_base_wr_en_i	input	logic [BaseWordsPerWLEN-1:0]
ispr_bignum_wdata_intg_i	input	logic [ExtWLEN-1:0]
ispr_bignum_wr_en_i	input	logic
ispr_flags_wr_i	input	logic [NFlagGroups-1:0]
ispr_wr_commit_i	input	logic
ispr_init_i	input	logic
ispr_rdata_intg_o	output	logic [ExtWLEN-1:0]
ispr_rd_en_i	input	logic
ispr_acc_intg_i	input	logic [ExtWLEN-1:0]
ispr_acc_wr_data_intg_o	output	logic [ExtWLEN-1:0]
ispr_acc_wr_en_o	output	logic
ispr_mod_intg_o	output	logic [ExtWLEN-1:0]
ispr_mai_ctrl_wr_o	output	logic
ispr_mai_ctrl_wdata_o	output	logic [31:0]
ispr_mai_ctrl_rdata_i	input	logic [31:0]
ispr_mai_status_rdata_i	input	logic [31:0]
ispr_mai_in0_s0_wr_o	output	logic
ispr_mai_in0_s0_wdata_o	output	logic [ExtWLEN-1:0]
ispr_mai_in0_s1_wr_o	output	logic
ispr_mai_in0_s1_wdata_o	output	logic [ExtWLEN-1:0]
ispr_mai_in1_s0_wr_o	output	logic
ispr_mai_in1_s0_wdata_o	output	logic [ExtWLEN-1:0]
ispr_mai_in1_s1_wr_o	output	logic
ispr_mai_in1_s1_wdata_o	output	logic [ExtWLEN-1:0]
ispr_mai_res_s0_wr_o	output	logic
ispr_mai_res_s0_wdata_o	output	logic [ExtWLEN-1:0]
ispr_mai_res_s1_wr_o	output	logic
ispr_mai_res_s1_wdata_o	output	logic [ExtWLEN-1:0]

포트	방향	형/폭
ispr_mai_in0_s0_rdata_i	input	logic [ExtWLEN-1:0]
ispr_mai_in0_s1_rdata_i	input	logic [ExtWLEN-1:0]
ispr_mai_in1_s0_rdata_i	input	logic [ExtWLEN-1:0]
ispr_mai_in1_s1_rdata_i	input	logic [ExtWLEN-1:0]
ispr_mai_res_s0_rdata_i	input	logic [ExtWLEN-1:0]
ispr_mai_res_s1_rdata_i	input	logic [ExtWLEN-1:0]
ispr_kmac_status_wr_o	output	logic
ispr_kmac_status_wdata_o	output	logic [31:0]
ispr_kmac_ctrl_wr_o	output	logic
ispr_kmac_ctrl_wdata_o	output	logic [31:0]
ispr_kmac_cfg_wr_o	output	logic
ispr_kmac_cfg_wdata_o	output	logic [31:0]
ispr_kmac_strb_wr_o	output	logic
ispr_kmac_strb_wdata_o	output	logic [31:0]
ispr_kmac_data_s0_wr_o	output	logic
ispr_kmac_data_s0_wdata_o	output	logic [ExtWLEN-1:0]
ispr_kmac_data_s1_wr_o	output	logic
ispr_kmac_data_s1_wdata_o	output	logic [ExtWLEN-1:0]
ispr_kmac_status_rdata_i	input	logic [31:0]
ispr_kmac_cfg_rdata_i	input	logic [31:0]
ispr_kmac_strb_rdata_i	input	logic [31:0]

API — otbn_controller.sv

Table 905. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_controller.sv — module / interface / package (1개)

이름	종류
otbn_controller	module

Table 906. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_controller.sv — parameter (2개)

파라미터	형/폭	기본값
ImemSizeByte	int	4096
DmemSizeByte	int	4096

Table 907. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_controller.sv — localparam (2개)

상수	형/폭	값
ImemAddrWidth	int	prim_util_pkg::vbits(ImemSizeByte)
DmemAddrWidth	int	prim_util_pkg::vbits(DmemSizeByte)

Table 908. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_controller.sv — 포트 (113개) — 앞 60개만 표시(전체 113개, 본문 소스 참조)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
start_i	input	logic
locking_o	output	logic
err_bit_clear_i	input	logic
fatal_escalate_en_i	input	prim_mubi_pkg::mubi4_t
recov_escalate_en_i	input	prim_mubi_pkg::mubi4_t
rma_req_i	input	prim_mubi_pkg::mubi4_t
err_bits_o	output	controller_err_bits_t
recoverable_err_o	output	logic
insn_fetch_req_valid_o	output	logic
insn_fetch_req_valid_raw_o	output	logic
insn_fetch_req_addr_o	output	logic [ImemAddrWidth-1:0]
insn_fetch_resp_clear_o	output	logic
insn_valid_i	input	logic
insn_illegal_i	input	logic
insn_addr_i	input	logic [ImemAddrWidth-1:0]
insn_dec_base_i	input	insn_dec_base_t
insn_dec_bignum_i	input	insn_dec_bignum_t
insn_dec_shared_i	input	insn_dec_shared_t
rf_base_wr_addr_o	output	logic [4:0]
rf_base_wr_en_o	output	logic
rf_base_wr_commit_o	output	logic
rf_base_wr_data_no_intg_o	output	logic [31:0]
rf_base_wr_data_intg_o	output	logic [BaseIntgWidth-1:0]
rf_base_wr_data_intg_sel_o	output	logic
rf_base_rd_addr_a_o	output	logic [4:0]
rf_base_rd_en_a_o	output	logic
rf_base_rd_data_a_intg_i	input	logic [BaseIntgWidth-1:0]
rf_base_rd_addr_b_o	output	logic [4:0]
rf_base_rd_en_b_o	output	logic
rf_base_rd_data_b_intg_i	input	logic [BaseIntgWidth-1:0]
rf_base_rd_commit_o	output	logic

포트	방향	형/폭
rf_base_call_stack_sw_err_i	input	logic
rf_base_call_stack_hw_err_i	input	logic
rf_bignum_wr_addr_o	output	logic [4:0]
rf_bignum_wr_en_o	output	logic [1:0]
rf_bignum_wr_commit_o	output	logic
rf_bignum_wr_data_no_intg_o	output	logic [WLEN-1:0]
rf_bignum_wr_data_intg_o	output	logic [ExtWLEN-1:0]
rf_bignum_wr_data_intg_sel_o	output	logic
rf_bignum_rd_addr_a_o	output	logic [4:0]
rf_bignum_rd_en_a_o	output	logic
rf_bignum_rd_data_a_intg_i	input	logic [ExtWLEN-1:0]
rf_bignum_rd_addr_b_o	output	logic [4:0]
rf_bignum_rd_en_b_o	output	logic
rf_bignum_rd_data_b_intg_i	input	logic [ExtWLEN-1:0]
rf_bignum_intg_err_i	input	logic
rf_bignum_spurious_we_err_i	input	logic
rf_bignum_rd_a_indirect_onehot_o	output	logic [NWdr-1:0]
rf_bignum_rd_b_indirect_onehot_o	output	logic [NWdr-1:0]
rf_bignum_wr_indirect_onehot_o	output	logic [NWdr-1:0]
rf_bignum_indirect_en_o	output	logic
alu_base_operation_o	output	alu_base_operation_t
alu_base_comparison_o	output	alu_base_comparison_t
alu_base_operation_result_i	input	logic [31:0]
alu_base_comparison_result_i	input	logic
alu_bignum_operation_o	output	alu_bignum_operation_t
alu_bignum_operation_valid_o	output	logic
alu_bignum_operation_commit_o	output	logic

1. lowRISC_opentitan/hw/ip/otbn/rtl/otbn.sv (1,654 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6 `include "prim_fifo_assert.svh"
7
8 /**
9  * OpenTitan Big Number Accelerator (OTBN)
10  */
11 module otbn
12     import prim_alert_pkg::*;
13     import otbn_pkg::*;
14     import otbn_reg_pkg::*;
15 #(

```

```

16     parameter bit          Stub          = 1'b0,
17     parameter regfile_e    RegFile       = RegFileFF,
18     parameter logic [NumAlerts-1:0] AlertAsyncOn = {Nu-
mAlerts{1'b1}},
19     // Number of cycles a differential skew is tolerated on the
  alert signal
20     parameter int unsigned AlertSkewCycles = 1,
21
22     // Default seed for URND PRNG
23     parameter urnd_prng_seed_t RndCnstUrndPrngSeed = RndCn-
stUrndPrngSeedDefault,
24     // Compile-time permutation applied to the primary URND out-
  put in otbn_rnd
25     parameter urnd_perm_t RndCnstUrndPerm = RndCnstUrndPermDe-
  fault,
26
27     // Skip URND re-seed at the start of an operation. Useful
  for SCA only.
28     parameter bit SecSkipUrndReseedAtStart = 1'b0,
29     // Masking accelerator interface will not randomize operand
  start indexes.

```

```

30 parameter bit SecFixMaiOpSeq = 1'b0,
31 // BN MAC will not randomize the order in which the vector
elements are processed for vectorized
32 // multiplication instructions.
33 parameter bit SecFixMacOpSeq = 1'b0,
34
35 // Masking accelerator is not present. Useful for resource-
bound targets only.
36 parameter bit FeatStubMai = 1'b0,
37
38 // Compile-time permutation for URND permutation in BN MAC
39 parameter bn_mac_urnd_perm_t RndCnstBnMacUrndPerm = RndCnstBnMacUrndPermDefault,
40
41 // Compile-time permutation for URND permutation in MAI
42 parameter mai_urnd_perm_t RndCnstMaiUrndPerm = RndCnstMaiUrndPermDefault,
43
44 // Default seed and nonce for scrambling
45 parameter otp_ctrl_pkg::otbn_key_t RndCnstOtbnKey = RndCnstOtbnKeyDefault,
46 parameter otp_ctrl_pkg::otbn_nonce_t RndCnstOtbnNonce = RndCnstOtbnNonceDefault
47 ) (
48 input clk_i,
49 input rst_ni,
50
51 input tlul_pkg::tl_h2d_t tl_i,
52 output tlul_pkg::tl_d2h_t tl_o,
53
54 // Inter-module signals
55 output prim_mubi_pkg::mubi4_t idle_o,
56
57 // Interrupts
58 output logic intr_done_o,
59
60 // Alerts
61 input prim_alert_pkg::alert_rx_t [NumAlerts-1:0] alert_rx_i,
62 output prim_alert_pkg::alert_tx_t [NumAlerts-1:0] alert_tx_o,
63
64 // Lifecycle interfaces
65 input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
66
67 input lc_ctrl_pkg::lc_tx_t lc_rma_req_i,
68 output lc_ctrl_pkg::lc_tx_t lc_rma_ack_o,
69
70 // Memory configuration
71 input prim_ram_1p_pkg::ram_1p_cfg_req_t ram_cfg_imem_i,
72 input prim_ram_1p_pkg::ram_1p_cfg_req_t ram_cfg_dmem_i,
73 output prim_ram_1p_pkg::ram_1p_cfg_rsp_t ram_cfg_imem_o,
74 output prim_ram_1p_pkg::ram_1p_cfg_rsp_t ram_cfg_dmem_o,
75
76 // EDN clock and interface
77 input clk_edn_i,
78 input rst_edn_ni,
79 output edn_pkg::edn_req_t edn_rnd_o,
80 input edn_pkg::edn_rsp_t edn_rnd_i,
81
82 output edn_pkg::edn_req_t edn_urnd_o,
83 input edn_pkg::edn_rsp_t edn_urnd_i,
84
85 // Key request to OTP (running on clk_fixed)
86 input clk_otp_i,
87 input rst_otp_ni,
88 output otp_ctrl_pkg::otbn_otp_key_req_t otbn_otp_key_o,
89 input otp_ctrl_pkg::otbn_otp_key_rsp_t otbn_otp_key_i,
90
91 input keymgr_pkg::otbn_key_req_t keymgr_key_i,
92
93 // KMAC application interface.
94 output kmac_pkg::app_req_t kmac_data_o,
95 input kmac_pkg::app_rsp_t kmac_data_i
96 );
97
98 import prim_mubi_pkg::*;
99 import prim_util_pkg::vbits;
100
101 logic rst_n;
102
103 // hold module in reset permanently when stubbing
104 if (Stub) begin : gen_stub_otbn
105 assign rst_n = 1'b0;
106 end else begin : gen_real_otbn
107 assign rst_n = rst_ni;
108 end
109
110 // The OTBN_*_SIZE parameters are auto-generated by regtool
and come from the bus window sizes;
111 // they are given in bytes and must be powers of two.
112 //
113 // DMEM is actually a bit bigger than OTBN_DMEN_SIZE: there
are an extra DmemScratchSizeByte bytes
114 // that aren't accessible over the bus.
115 localparam int ImemSizeByte =
int'(otbn_reg_pkg::OTBN_IMEM_SIZE);
116 localparam int DmemBusSizeByte = int'(otbn_reg_pkg::OTBN_D-
MEM_SIZE);
117 localparam int DmemSizeByte = DmemBusSizeByte + Dmem-
ScratchSizeByte;

```

```

118
119 localparam int ImemAddrWidth = vbits(ImemSizeByte);
120 localparam int DmemBusAddrWidth = vbits(DmemBusSizeByte);
121 localparam int DmemAddrWidth = vbits(DmemSizeByte);
122
123 `ASSERT_INIT(ImemSizePowerOfTwo, 2 ** ImemAddrWidth == Imem-
SizeByte)
124 `ASSERT_INIT(DmemSizePowerOfTwo, 2 ** DmemAddrWidth == Dmem-
SizeByte)
125
126 logic start_d, start_q;
127 logic busy_execute_d, busy_execute_q;
128 logic wfi_pending;
129 logic wfi_resume_d, wfi_resume_q;
130 logic done, done_core, locking, locking_q;
131 logic busy_secure_wipe;
132 logic init_sec_wipe_done_d, init_sec_wipe_done_q;
133 logic illegal_bus_access_d, illegal_bus_access_q;
134 logic missed_gnt_error_d, missed_gnt_error_q;
135 logic dmem_sec_wipe;
136 logic imem_sec_wipe;
137 logic mems_sec_wipe;
138 logic req_sec_wipe_urnd_keys;
139 logic [127:0] dmem_sec_wipe_urnd_key,
imem_sec_wipe_urnd_key;
140
141 logic core_recoverable_err, recoverable_err_d, recover-
able_err_q;
142 mubi4_t core_escalate_en;
143
144 core_err_bits_t core_err_bits;
145 non_core_err_bits_t non_core_err_bits, non_core_err_bits_d,
non_core_err_bits_q;
146 err_bits_t err_bits, err_bits_d, err_bits_q;
147 logic err_bits_en;
148
149 // ERR_BITS register should be cleared due to a write re-
quest from the host processor
150 // when OTBN is not running.
151 logic err_bits_clear;
152
153 logic software_errs_fatal_q, software_errs_fatal_d;
154 logic wfi_enabled_q, wfi_enabled_d;
155 logic urnd_ctrl_enabled_q, urnd_ctrl_enabled_d;
156
157 otbn_reg2hw_t reg2hw;
158 otbn_hw2reg_t hw2reg;
159 status_e status_d, status_q;
160
161 // Bus device windows, as specified in otbn.hjson
162 typedef enum logic {
163 TLWinImem = 1'b0,
164 TLWinDmem = 1'b1
165 } tl_win_e;
166
167 tlul_pkg::tl_h2d_t tl_win_h2d[2];
168 tlul_pkg::tl_d2h_t tl_win_d2h[2];
169
170 // The clock can be gated and some registers can be updated
as long as OTBN isn't currently
171 // running. Other registers can only be updated when OTBN is
in the Idle state (which also implies
172 // we are not locked). In addition, OTBN may not be gated
when it is paused so a RESUME command
173 // can be properly detected.
174 logic is_not_running_d, is_not_running_q;
175 logic otbn_dmem_scramble_key_req_busy,
otbn_imem_scramble_key_req_busy;
176
177 assign is_not_running_d = ~(busy_execute_d,
wfi_pending,
178 otbn_d-
mem_scramble_key_req_busy,
179 otbn_imem_scramble_key_req_busy,
180 busy_secure_wipe);
181
182 always_ff @(posedge clk_i or negedge rst_ni) begin
183 if (!rst_ni) begin
184 // OTBN starts busy, performing the initial secure wipe.
185 is_not_running_q <= 1'b0;
186 end else begin
187 is_not_running_q <= is_not_running_d;
188 end
189 end
190
191 // Inter-module signals
=====
193
194 // Note: This is not the same thing as STATUS == IDLE. For
example, we want to allow clock gating
195 // when locked.
196 prim_mubi4_sender #(
197 .ResetValue(prim_mubi_pkg::MuBi4True)
198 ) u_prim_mubi4_sender (
199 .clk_i,
200 .rst_ni,
201 .mubi_i(mubi4_bool_to_mubi(is_not_running_q)),
202 .mubi_o(idle_o)
203 );

```

```

204
205 // Lifecycle
=====
206 localparam int unsigned LcEscalateCopies = 2;
207 lc_ctrl_pkg::lc_tx_t [LcEscalateCopies-1:0] lc_escalate_en;
208 prim_lc_sync #(
209     .NumCopies(LcEscalateCopies)
210 ) u_lc_escalate_en_sync (
211     .clk_i,
212     .rst_ni,
213     .lc_en_i(lc_escalate_en_i),
214     .lc_en_o(lc_escalate_en)
215 );
216
217 lc_ctrl_pkg::lc_tx_t lc_rma_req;
218 prim_lc_sync #(
219     .NumCopies(1)
220 ) u_lc_rma_req_sync (
221     .clk_i,
222     .rst_ni,
223     .lc_en_i(lc_rma_req_i),
224     .lc_en_o({lc_rma_req})
225 );
226
227 // Internally, OTBN uses MUBI types.
228 mubi4_t mubi_rma_req, mubi_rma_ack;
229 assign mubi_rma_req = lc_ctrl_pkg::lc_to_mubi4(lc_rma_req);
230
231 // When stubbing, forward req to ack.
232 if (Stub) begin : gen_stub_rma_ack
233     assign lc_rma_ack_o = lc_rma_req;
234 end else begin : gen_real_rma_ack
235     assign lc_rma_ack_o = lc_c-
236 trl_pkg::mubi4_to_lc(mubi_rma_ack);
237 end
238
239 // Interrupts
=====
240
241 // Interrupt is set when OTBN is no longer busy. This is the
242 // case when it gets paused or ends
243 // execution. The done interrupt is also set when a direct
244 // paused to locked transition occurs.
245 // This can happen under some faults where the secure wipe
246 // is skipped. These faults raise a fatal
247
248 // alert but for consistency we still set the interrupt. The
249 // faults are:
250 // - An invalid MuBi on lc_escalate_en_i
251 // - Spurious URND reseed ACK
252 assign done = ((is_busy_status(status_q) &
253 -is_busy_status(status_d)) |
254 (status_q == StatusPaused) & (status_d ==
255 StatusLocked)) & init_sec_wipe_done_q;
256
257 prim_intr_hw #(
258     .Width(1)
259 ) u_intr_hw_done (
260     .clk_i,
261     .rst_ni,
262     .event_intr_i (done),
263     .reg2hw_intr_enable_q_i(reg2hw.intr_enable.q),
264     .reg2hw_intr_test_q_i (reg2hw.intr_test.q),
265     .reg2hw_intr_test_qe_i (reg2hw.intr_test.qe),
266     .reg2hw_intr_state_q_i (reg2hw.intr_state.q),
267     .hw2reg_intr_state_de_o(hw2reg.intr_state.de),
268     .hw2reg_intr_state_d_o (hw2reg.intr_state.d),
269     .intr_o (intr_done_o)
270 );
271
272 // Instruction Memory (IMEM)
=====
273 localparam int ImemSizeWords = ImemSizeByte / 4;
274 localparam int ImemIndexWidth = vbits(ImemSizeWords);
275
276 // Access select to IMEM: core (1), or bus (0)
277 logic imem_access_core;
278
279 logic imem_req;
280 logic imem_gnt;
281 logic imem_write;
282 logic imem_wr_collision;
283 logic imem_wpending;
284 logic [ImemIndexWidth-1:0] imem_index;
285 logic [38:0] imem_wdata;
286 logic [38:0] imem_wmask;
287 logic [38:0] imem_rdata;
288 logic imem_rvalid;
289 logic imem_illegal_bus_access;
290 logic imem_missed_gnt;
291
292 logic imem_req_core;
293 logic imem_write_core;
294 logic [ImemIndexWidth-1:0] imem_index_core;
295 logic [38:0] imem_rdata_core;
296 logic imem_rvalid_core;
297
298 logic imem_req_bus;

```

```

293 logic imem_dummy_response_q, imem_dummy_response_d;
294 logic imem_write_bus;
295 logic [ImemIndexWidth-1:0] imem_index_bus;
296 logic [38:0] imem_wdata_bus;
297 logic [38:0] imem_wmask_bus;
298 logic [38:0] imem_rdata_bus, imem_rdata_bus_raw;
299 logic imem_rdata_bus_en_q, imem_rdata_bus_en_d;
300 logic [top_pkg::TL_DBW-1:0] imem_byte_mask_bus;
301 logic imem_rvalid_bus;
302 logic [1:0] imem_error_bus;
303
304 logic imem_bus_intg_violation;
305
306 typedef struct packed {
307     logic imem;
308     logic [14:0] index;
309     logic [31:0] wr_data;
310 } mem_crc_data_in_t;
311
312 logic mem_crc_data_in_valid;
313 mem_crc_data_in_t mem_crc_data_in;
314 logic set_crc;
315 logic [31:0] crc_in, crc_out;
316
317 logic [ImemAddrWidth-1:0] imem_addr_core;
318 assign imem_index_core = imem_addr_core[ImemAddrWidth-1:2];
319
320 logic [1:0] unused_imem_addr_core_wordbits;
321 assign unused_imem_addr_core_wordbits = imem_addr_core[1:0];
322
323 otp_ctrl_pkg::otbn_key_t otbn_imem_scramble_key;
324 otbn_imem_nonce_t otbn_imem_scramble_nonce;
325 logic otbn_imem_scramble_valid;
326 logic un-
327 used_otbn_imem_scramble_key_seed_valid;
328
329 otp_ctrl_pkg::otbn_key_t otbn_dmem_scramble_key;
330 otbn_dmem_nonce_t otbn_dmem_scramble_nonce;
331 logic otbn_dmem_scramble_valid;
332 logic un-
333 used_otbn_dmem_scramble_key_seed_valid;
334
335 logic otbn_scramble_state_error;
336
337 // SEC_CM: SCRAMBLE.KEY.SIDELOAD
338 otbn_scramble_ctrl #(
339     .RndCnstOtbnKey (RndCnstOtbnKey),
340     .RndCnstOtbnNonce(RndCnstOtbnNonce)
341 ) u_otbn_scramble_ctrl (
342     .clk_i,
343     .rst_ni,
344     .clk_otpn_i,
345     .rst_otpn_i,
346     .otbn_otpn_key_o,
347     .otbn_otpn_key_i,
348     .otbn_dmem_scramble_key_o (otbn_d-
349 mem_scramble_key),
350     .otbn_dmem_scramble_nonce_o (otbn_d-
351 mem_scramble_nonce),
352     .otbn_dmem_scramble_valid_o (otbn_d-
353 mem_scramble_valid),
354     .otbn_d-
355 mem_scramble_key_seed_valid_o(unused_otbn_dmem_scramble_key_seed_va
356 lid),
357     .otbn_imem_scramble_key_o
358 (otbn_imem_scramble_key),
359     .otbn_imem_scramble_nonce_o
360 (otbn_imem_scramble_nonce),
361     .otbn_imem_scramble_valid_o
362 (otbn_imem_scramble_valid),
363     .otbn_imem_scramble_key_seed_valid_o(unused_otbn_imem_scr
364 amble_key_seed_valid),
365     .otbn_dmem_scramble_sec_wipe_i (dmem_sec_wipe),
366     .otbn_d-
367 mem_scramble_sec_wipe_key_i(dmem_sec_wipe_urnd_key),
368     .otbn_imem_scramble_sec_wipe_i (imem_sec_wipe),
369     .otbn_imem_scramble_sec_wipe_key_i(imem_sec_wipe_urnd_key)
370 ,
371     .otbn_d-
372 mem_scramble_key_req_busy_o(otbn_dmem_scramble_key_req_busy),
373     .otbn_imem_scramble_key_req_busy_o(otbn_imem_scramble_key-
374 req_busy),
375     .state_error_o(otbn_scramble_state_error)
376 );
377
378 // SEC_CM: MEM.SCRAMBLE
379 prim_ram_1p_scr #(
380     .Width (39),
381     .Depth (ImemSizeWords),
382     .InstDepth (ImemSizeWords),
383     .DataBitsPerMask(39),
384     .EnableParity (0)
385 ) u_imem (

```

```

379 .clk_i,
380 .rst_ni(rst_n),
381
382 .key_valid_i(otbn_imem_scramble_valid),
383 .key_i      (otbn_imem_scramble_key),
384 .nonce_i    (otbn_imem_scramble_nonce),
385
386 .req_i      (imem_req),
387 .gnt_o      (imem_gnt),
388 .write_i    (imem_write),
389 .addr_i     (imem_index),
390 .wdata_i    (imem_wdata),
391 .wmask_i    (imem_wmask),
392 .intg_error_i(locking),
393
394 .rdata_o    (imem_rdata),
395 .rvalid_o   (imem_rvalid),
396 .raddr_o    (),
397 .rerror_o   (),
398 .cfg_i      (ram_cfg_imem_i),
399 .cfg_o      (ram_cfg_imem_o),
400
401 .wr_collision_o (imem_wr_collision),
402 .write_pending_o (imem_wpending),
403
404 .alert_o ()
405 );
406
407 // We should never see a request that doesn't get granted. A
fatal error is raised if this occurs.
408 assign imem_missed_gnt = imem_req & ~imem_gnt;
409
410 // IMEM access from main TL-UL bus
411 logic imem_gnt_bus;
412 // Always grant to bus accesses, when OTBN is running a
dummy response is returned
413 assign imem_gnt_bus = imem_req_bus;
414
415 tlul_adapter_sram #(
416   .SramAw      (ImemIndexWidth),
417   .SramDw      (32),
418   .Outstanding (1),
419   .ByteAccess  (0),
420   .ErrOnRead   (0),
421   .EnableDataIntgPt(1),
422   .SecFifoPtr  (1) // SEC_CM: TLUL_FIFO.CTR.REDUN
423 ) u_tlul_adapter_sram_imem (
424   .clk_i,
425   .rst_ni      (rst_n),
426   .tl_i        (tl_win_h2d[TLWinImem]),
427   .tl_o        (tl_win_d2h[TLWinImem]),
428   .en_ifetch_i (MuBi4False),
429   .req_o       (imem_req_bus),
430   .req_type_o  (),
431   .gnt_i       (imem_gnt_bus),
432   .we_o        (imem_write_bus),
433   .addr_o      (imem_index_bus),
434   .wdata_o     (imem_wdata_bus),
435   .wmask_o     (imem_wmask_bus),
436   .intg_error_o (imem_bus_intg_violation),
437   .user_rsvd_o (),
438   .rdata_i     (imem_rdata_bus),
439   .rvalid_i    (imem_rvalid_bus),
440   .rerror_i    (imem_rerror_bus),
441   .compound_txn_in_progress_o (),
442   .readback_en_i (prim_mubi_pkg::MuBi4False),
443   .readback_error_o (),
444   .wr_collision_i (imem_wr_collision),
445   .write_pending_i (imem_wpending)
446 );
447
448 // IMEM access is granted to the core (not the bus) if the
core is executing, being started, or
449 // is paused by a WFI instruction.
450 assign imem_access_core = busy_execute_q | start_q |
wfi_pending;
451
452 assign imem_req      = imem_access_core ?
imem_req_core      : imem_req_bus;
453 assign imem_write    = imem_access_core ?
imem_write_core    : imem_write_bus;
454 assign imem_index    = imem_access_core ? imem_in-
dex_core          : imem_index_bus;
455 assign imem_wdata    = imem_access_core ?
'imem_wdata_core  : imem_wdata_bus;
456
457 assign imem_illegal_bus_access = imem_req_bus & imem_ac-
cess_core;
458
459 assign imem_dummy_response_d = imem_illegal_bus_access;
460 always_ff @(posedge clk_i or negedge rst_ni) begin
461   if (!rst_ni) begin
462     imem_dummy_response_q <= 1'b0;
463   end else begin
464     imem_dummy_response_q <= imem_dummy_response_d;
465   end
466 end
467
468 // The instruction memory only supports 32b word writes, so

```

```

we hardcoded its
470 // wmask here.
471 //
472 // Since this could cause confusion if the bus tried to do a
partial write
473 // (which wasn't caught in the TLUL adapter for some reas-
on), we assert that
474 // the wmask signal from the bus is indeed '1' when it re-
quests a write. We
475 // don't have the corresponding check for writes from the
core because the
476 // core cannot perform writes (and has no imem_wmask_o
port).
477 assign imem_wmask = imem_access_core ? '1 : imem_wmask_bus;
478 `ASSERT(ImemWmaskBusIsFullWord_A, imem_req_bus &&
imem_write_bus |> imem_wmask_bus == '1)
479
480 // SEC_CM: DATA_REG_SW.SCA
481 // Blank bus read data interface during core operation to
avoid leaking the currently executed
482 // instruction from IMEM through the bus unintentionally.
Also blank when OTBN is paused, is
483 // returning a dummy response (responding to an illegal bus
access), and when OTBN is locked.
484 assign imem_rdata_bus_en_d =
485   ~(busy_execute_d | start_d | wfi_pending) &
~imem_dummy_response_d & ~locking;
486
487 always_ff @(posedge clk_i or negedge rst_ni) begin
488   if (!rst_ni) begin
489     imem_rdata_bus_en_q <= 1'b1;
490   end else begin
491     imem_rdata_bus_en_q <= imem_rdata_bus_en_d;
492   end
493 end
494
495 prim_blanker #(.Width(39)) u_imem_rdata_bus_blanker (
496   .in_i (imem_rdata),
497   .en_i (imem_rdata_bus_en_q),
498   .out_o(imem_rdata_bus_raw)
499 );
500
501 // When OTBN is locked all imem bus reads should return 0.
The blanker produces the 0s, this adds
502 // the appropriate ECC. When OTBN is not locked the output
of the blanker is passed straight
503 // through. Data bits are always left un-modified. A re-
gistered version of `locking` is used for
504 // timing reasons. When a read comes in when `locking` has
just been asserted, `locking_q` will be
505
506 // set the following cycle and the rdata will be forced to 0
with appropriate ECC. When `locking`
507 // is asserted the cycle the rdata is being returned no
locking was occurring when the request
508
509 // came in so it is reasonable to proceed with returning the
supplied integrity.
510 assign imem_rdata_bus =
511   {locking_q ? prim_secdd_pkg::SecddInv3932ZeroEcc :
imem_rdata_bus_raw[38:32],
512   imem_rdata_bus_raw[31:0]};
513
514 `ASSERT(ImemRdataBusDisabledWhenCoreAccess_A, imem_ac-
cess_core |> !imem_rdata_bus_en_q)
515 `ASSERT(ImemRdataBusEnabledWhenIdle_A, status_q ==
StatusIdle |> imem_rdata_bus_en_q)
516 `ASSERT(ImemRdataBusDisabledWhenLocked_A, locking |> !
imem_rdata_bus_en_q)
517 `ASSERT(ImemRdataBusReadAsZeroWhenLocked_A,
518   imem_rvalid_bus & locking |> imem_rdata_bus_raw == '0)
519
520 assign imem_rdata_core = imem_rdata;
521
522 // When an illegal bus access is seen, always return a dummy
response the follow cycle.
523
524 assign imem_rvalid_bus = (~imem_access_core & imem_rvalid) |
imem_dummy_response_q;
525 assign imem_rvalid_core = imem_access_core ? imem_rvalid :
1'b0;
526
527 assign imem_byte_mask_bus = tl_win_h2d[TLWinImem].a_mask;
528
529 // No imem errors reported for bus reads. Integrity is car-
ried through on the bus so integrity
530 // checking on TL responses will pick up any errors.
531 assign imem_rerror_bus = 2'b00;
532
533 // Data Memory (DMEM)
=====
534
535 localparam int DmemSizeWords = DmemSizeByte / (WLEN / 8);
536 localparam int DmemIndexWidth = vbits(DmemSizeWords);
537
538 localparam int DmemBusSizeWords = DmemBusSizeByte / (WLEN /
8);
539 localparam int DmemBusIndexWidth = vbits(DmemBusSizeWords);
540

```



```

538 // Access select to DMEM: core (1), or bus (0)
539 logic dmem_access_core;
540
541 logic dmem_req;
542 logic dmem_gnt;
543 logic dmem_write;
544 logic [DmemIndexWidth-1:0] dmem_index;
545 logic [ExtWLEN-1:0] dmem_wdata;
546 logic [ExtWLEN-1:0] dmem_wmask;
547 logic [ExtWLEN-1:0] dmem_rdata;
548 logic dmem_rvalid;
549 logic [BaseWordsPerWLEN*2-1:0] dmem_error_vec;
550 logic dmem_error;
551 logic dmem_illegal_bus_access;
552 logic dmem_missed_gnt;
553
554 logic dmem_req_core;
555 logic dmem_write_core;
556 logic [DmemIndexWidth-1:0] dmem_index_core;
557 logic [ExtWLEN-1:0] dmem_wdata_core;
558 logic [ExtWLEN-1:0] dmem_wmask_core;
559 logic [BaseWordsPerWLEN-1:0] dmem_rmask_core_q,
dmem_rmask_core_d;
560 logic [ExtWLEN-1:0] dmem_rdata_core;
561 logic dmem_rvalid_core;
562 logic dmem_error_core;
563
564 logic dmem_req_bus;
565 logic dmem_dummy_response_q, dmem_dummy_response_d;
566 logic dmem_write_bus;
567 logic [DmemBusIndexWidth-1:0] dmem_index_bus;
568 logic [ExtWLEN-1:0] dmem_wdata_bus;
569 logic [ExtWLEN-1:0] dmem_wmask_bus;
570 logic [ExtWLEN-1:0] dmem_rdata_bus, dmem_rdata_bus_raw;
571 logic dmem_rdata_bus_en_q, dmem_rdata_bus_en_d;
572 logic [DmemBusAddrWidth-1:0] dmem_addr_bus;
573 logic unused_dmem_addr_bus;
574 logic [31:0] dmem_wdata_narrow_bus;
575 logic [top_pkg::TL_DBW-1:0] dmem_byte_mask_bus;
576 logic dmem_rvalid_bus;
577 logic dmem_wr_collision;
578 logic dmem_wpending;
579 logic [1:0] dmem_error_bus;
580
581 logic dmem_bus_intg_violation;
582
583 logic [DmemAddrWidth-1:0] dmem_addr_core;
584 assign dmem_index_core = dmem_ad-
dr_core[DmemAddrWidth-1:DmemAddrWidth-DmemIndexWidth];
585
586 logic unused_dmem_addr_core_wordbits;
587 assign unused_dmem_addr_core_wordbits = ^dmem_ad-
dr_core[DmemAddrWidth-DmemIndexWidth-1:0];
588
589 // SEC_CM: MEM.SCRAMBLE
590 prim_ram_ip_scr #(
591   .Width      (ExtWLEN),
592   .Depth      (DmemSizeWords),
593   .InstDepth  (DmemSizeWords),
594   .DataBitsPerMask (39),
595   .EnableParity (0),
596   .ReplicateKeyStream(1)
597 ) u_dmem (
598   .clk_i,
599   .rst_ni(rst_n),
600
601   .key_valid_i(otbn_dmem_scramble_valid),
602   .key_i      (otbn_dmem_scramble_key),
603   .nonce_i    (otbn_dmem_scramble_nonce),
604
605   .req_i      (dmem_req),
606   .gnt_o      (dmem_gnt),
607   .write_i    (dmem_write),
608   .addr_i     (dmem_index),
609   .wdata_i    (dmem_wdata),
610   .wmask_i    (dmem_wmask),
611   .intg_error_i(locking),
612
613   .rdata_o    (dmem_rdata),
614   .rvalid_o   (dmem_rvalid),
615   .raddr_o    (),
616   .rerror_o   (),
617   .cfg_i      (ram_cfg_dmem_i),
618   .cfg_o      (ram_cfg_dmem_o),
619
620   .wr_collision_o (dmem_wr_collision),
621   .write_pending_o (dmem_wpending),
622
623   .alert_o ()
624 );
625
626 // We should never see a request that doesn't get granted. A
fatal error is raised if this occurs.
627 assign dmem_missed_gnt = dmem_req & !dmem_gnt;
628
629 always_ff @(posedge clk_i or negedge rst_ni) begin
630   if (!rst_ni) begin
631     dmem_rmask_core_q <= '0;
632   end else begin
633     if (dmem_req_core) begin

```

```

634     dmem_rmask_core_q <= dmem_rmask_core_d;
635   end
636 end
637 end
638
639 // SEC_CM: DATA.MEM.INTEGRITY
640 for (genvar i_word = 0; i_word < BaseWordsPerWLEN; ++i_word)
begin : g_dmem_intg_check
641   logic [1:0] dmem_error_raw;
642
643   // Separate check for dmem read data integrity outside of
'u_dmem' as 'prim_ram_ip_adv' doesn't
644   // have functionality for only integrity checking, just
fully integrated ECC. Integrity bits are
645   // implemented on a 32-bit granule so separate checks are
required for each.
646   prim_secded_inv_39_32_dec u_dmem_intg_check (
647     .data_i    (dmem_rdata[i_word*39+:39]),
648     .data_o    (),
649     .syndrome_o(),
650     .err_o     (dmem_error_raw)
651   );
652
653   // Only report an error where the word was actually ac-
cessed. Otherwise uninitialised memory
654   // that OTBN isn't using will cause false errors.
dmem_error is only reported for reads from
655
656   // OTBN. For Ibex reads integrity checking on TL responses
will serve the same purpose.
657   assign dmem_error_vec[i_word*2 +: 2] = dmem_error_raw &
{2{dmem_rmask_core_q[i_word] & dmem_rvalid & dmem_ac-
cess_core}};
658 end
659
660 // dmem_error_vec is 2 bits wide and is used to report ECC
errors. Bit 1 is set if there's an
661 // uncorrectable error and bit 0 is set if there's a cor-
rectable error. However, we're treating
662 // all errors as fatal, so OR the two signals together.
663 assign dmem_error = |dmem_error_vec;
664
665 // DMEM access from main TL-UL bus
666 logic dmem_gnt_bus;
667 // Always grant to bus accesses, when OTBN is running a
dummy response is returned
668 assign dmem_gnt_bus = dmem_req_bus;
669
670 tlul_adapter_sram #(
671   .SramAw      (DmemBusIndexWidth),
672   .SramDw      (WLEN),
673   .Outstanding (1),
674   .ByteAccess  (0),
675   .ErrOnRead   (0),
676   .EnableDataIntgPt(1),
677   .SecFifoPtr  (1) // SEC_CM: TLUL_FIFO.CTR.REDUN
678 ) u_tlul_adapter_sram_dmem (
679   .clk_i,
680   .rst_ni      (rst_n),
681   .tl_i        (tl_win_h2d[TLWinDmem]),
682   .tl_o        (tl_win_d2h[TLWinDmem]),
683   .en_ifetch_i (MuBi4False),
684   .req_o       (dmem_req_bus),
685   .req_type_o  (),
686   .gnt_i       (dmem_gnt_bus),
687   .we_o        (dmem_write_bus),
688   .addr_o      (dmem_index_bus),
689   .wdata_o     (dmem_wdata_bus),
690   .wmask_o     (dmem_wmask_bus),
691   .intg_error_o (dmem_bus_intg_violation),
692   .user_rsvd_o (),
693   .rdata_i     (dmem_rdata_bus),
694   .rvalid_i    (dmem_rvalid_bus),
695   .rerror_i    (dmem_error_bus),
696   .compound_txn_in_progress_o (),
697   .readback_en_i (prim_mubi_pkg::MuBi4False),
698   .readback_error_o (),
699   .wr_collision_i (dmem_wr_collision),
700   .write_pending_i (dmem_wpending)
701 );
702
703 // Mux core and bus access into dmem. While paused, busy_ex-
ecute_q is low so the DMEM is
704 // accessible to the bus.
705 assign dmem_access_core = busy_execute_q;
706
707 assign dmem_req = dmem_access_core ? dmem_req_core :
dmem_req_bus;
708 assign dmem_write = dmem_access_core ? dmem_write_core :
dmem_write_bus;
709 assign dmem_wmask = dmem_access_core ? dmem_wmask_core :
dmem_wmask_bus;
710 // SEC_CM: DATA.MEM.SW_NOACCESS
711 assign dmem_index = dmem_access_core ? dmem_index_core :
712   {{(DmemIndexWidth-DmemBusIndexWidth){1'b0}}, {dmem_in-
dex_bus}};
713 assign dmem_wdata = dmem_access_core ? dmem_wdata_core :
dmem_wdata_bus;
714
715 assign dmem_illegal_bus_access = dmem_req_bus & dmem_ac-

```



```

cess_core;
716
717 assign dmem_dummy_response_d = dmem_illegal_bus_access;
718 always_ff @(posedge clk_i or negedge rst_ni) begin
719   if (!rst_ni) begin
720     dmem_dummy_response_q <= 1'b0;
721   end else begin
722     dmem_dummy_response_q <= dmem_dummy_response_d;
723   end
724 end
725
726 // SEC_CM: DATA_REG_SW.SCA
727 // Blank bus read data interface during core operation to
avoid leaking DMEM data through the bus
728 // unintentionally. Also blank when OTBN is returning a
dummy response (responding to an illegal
729 // bus access) and when OTBN is locked.
730 assign dmem_rdata_bus_en_d = ~(busy_execute_d | start_d) &
~dmem_dummy_response_d & ~locking;
731
732 always_ff @(posedge clk_i or negedge rst_ni) begin
733   if (!rst_ni) begin
734     dmem_rdata_bus_en_q <= 1'b1;
735   end else begin
736     dmem_rdata_bus_en_q <= dmem_rdata_bus_en_d;
737   end
738 end
739
740 prim_blanker #(Width(ExtWLEN)) u_dmem_rdata_bus_blanker (
741   .in_i (dmem_rdata),
742   .en_i (dmem_rdata_bus_en_q),
743   .out_o(dmem_rdata_bus_raw)
744 );
745
746 // When OTBN is locked all dmem bus reads should return 0.
The blanker produces the 0s, this adds
747 // the appropriate ECC. When OTBN is not locked the output
of the blanker is passed straight
748 // through. Data bits are always left un-modified. A re-
gistered version of `locking` is used for
749 // timing reasons. When a read comes in when `locking` has
just been asserted, `locking_q` will be
750 // timing reasons. When a read comes in when `locking` has
just been asserted, `locking_q` will be
751 // set the following cycle and the rdata will be forced to 0
with appropriate ECC. When `locking`
752 // is asserted the cycle the rdata is being returned no
locking was occurring when the request
753 // came in so it is reasonable to proceed with returning the
supplied integrity.
754 for (genvar i_word = 0; i_word < BaseWordsPerWLEN; ++i_word)
begin : g_dmem_rdata_bus
755   assign dmem_rdata_bus[i_word*39+:39] =
756     {locking_q ? prim_secdd_pkg::SecddInv3932ZeroEcc :
dmem_rdata_bus_raw[i_word*39+32+:7],
757     dmem_rdata_bus_raw[i_word*39+:32]};
758 end
759
760 `ASSERT(DmemRDataBusDisabledWhenCoreAccess_A, dmem_ac-
cess_core |> !dmem_rdata_bus_en_q)
761 `ASSERT(DmemRDataBusEnabledWhenIdle_A, status_q ==
StatusIdle |> dmem_rdata_bus_en_q)
762 `ASSERT(DmemRDataBusDisabledWhenLocked_A, locking |> !
dmem_rdata_bus_en_q)
763 `ASSERT(DmemRDataBusReadAsZeroWhenLocked_A,
dmem_rvalid_bus & locking |> dmem_rdata_bus_raw == '0)
764
765 assign dmem_rdata_core = dmem_rdata;
766
767
768 // When an illegal bus access is seen, always return a dummy
response the follow cycle.
769 assign dmem_rvalid_bus = (~dmem_access_core & dmem_rvalid)
| dmem_dummy_response_q;
770 assign dmem_rvalid_core = dmem_access_core ? dmem_rvalid :
1'b0;
771
772 // No dmem errors reported for bus reads. Integrity is car-
ried through on the bus so integrity
773 // checking on TL responses will pick up any errors.
774 assign dmem_error_bus = 2'b00;
775 assign dmem_error_core = dmem_error;
776
777 assign dmem_addr_bus = tl_win_h2d[TLWinD-
mem].a_address[DmemBusAddrWidth-1:0];
778 assign dmem_wdata_narrow_bus = tl_win_h2d[TLWinD-
mem].a_data[31:0];
779 assign dmem_byte_mask_bus = tl_win_h2d[TLWinDmem].a_mask;
780
781 // Memory Load Integrity
=====
782 // CRC logic below assumes an incoming data bus width of 32
bits and considers
783 // 15 bits of the 32-bit word based addresses.
784 `ASSERT_INIT(TLDWIs32Bit_A, top_pkg::TL_DW == 32)
785 `ASSERT_INIT(ImemAddrTooWideForLoadCrc_A, ImemIndexWidth <=
15)
786 `ASSERT_INIT(DmemBusAddrTooWideForLoadCrc_A, (DmemBusAd-

```

```

drwidth - 2) <= 15)
787
788 // Only advance CRC calculation on full 32-bit writes;
789 assign mem_crc_data_in_valid = ~(dmem_access_core |
imem_access_core) &
790   ((imem_req_bus & (imem_byte_mask_bus == 4'hf)) |
791    (dmem_req_bus & (dmem_byte_mask_bus == 4'hf)));
792
793 assign mem_crc_data_in.wr_data = imem_req_bus ?
imem_wdata_bus[31:0] :
794   dmem_wdata_narrow_bus[31:0];
795
796 // The CRC operates on the 32 bits from the bus. For IMEM we
can take the index. But for DMEM
797 // we must take the relevant part of dmem_addr_bus because
dmem_index_bus indexes 256-bit words.
798 assign mem_crc_data_in.index = imem_req_bus ? {{15 - Ime-
mIndexWidth{1'b0}}, imem_index_bus} :
799   {{15 -
(DmemBusAddrWidth - 2){1'b0}},
800   dmem_ad-
dr_bus[DmemBusAddrWidth-1:2]};
801 assign mem_crc_data_in.imem = imem_req_bus;
802
803 // Only the bits that factor into the dmem index and dmem
word enables are required
804 assign unused_dmem_addr_bus = ^dmem_addr_bus[1:0];
805
806 // SEC_CM: WRITE.MEM.INTEGRITY
807 prim_crc32 #(
808   .BytesPerWord(6)
809   ) u_mem_load_crc32 (
810     .clk_i (clk_i),
811     .rst_ni(rst_ni),
812     .set_crc_i(set_crc),
813     .crc_in_i (crc_in),
814     .data_valid_i(mem_crc_data_in_valid),
815     .data_i (mem_crc_data_in),
816     .crc_out_o (crc_out)
817 );
818
819 assign set_crc = reg2hw.load_checksum.q;
820 assign crc_in = reg2hw.load_checksum.q;
821 assign hw2reg.load_checksum.d = crc_out;
822
823
824 // Registers
=====
825
826 logic reg_bus_intg_violation;
827
828 otbn_reg_top u_reg (
829   .clk_i,
830   .rst_ni (rst_n),
831   .tl_i,
832   .tl_o,
833   .tl_win_o(tl_win_h2d),
834   .tl_win_i(tl_win_d2h),
835   .reg2hw,
836   .hw2reg,
837   .intg_err_o(reg_bus_intg_violation)
838 );
839
840 // SEC_CM: BUS.INTEGRITY
841 // SEC_CM: TLUL_FIFO.CTR.REDUN
842 logic bus_intg_violation;
843 assign bus_intg_violation = (imem_bus_intg_violation |
dmem_bus_intg_violation |
844   reg_bus_intg_violation);
845
846 // CMD register
847 always_comb begin
848
849 // start and resume are flopped to avoid long timing paths
from the TL fabric into OTBN
850 // internals.
851 start_d = 1'b0;
852 wfi_resume_d = 1'b0;
853 dmem_sec_wipe = 1'b0;
854 imem_sec_wipe = 1'b0;
855
856 // Can only start a new command when idle.
857 if (status_q == StatusIdle) begin
858   if (reg2hw.cmd.q) begin
859     unique case (reg2hw.cmd.q)
860       CmdExecute: start_d = 1'b1;
861       CmdSecWipeDmem: dmem_sec_wipe = 1'b1;
862       CmdSecWipeImem: imem_sec_wipe = 1'b1;
863       default: ;
864     endcase
865   end
866 end else if (busy_execute_q || status_q == StatusPaused)
begin
867
868 // OTBN can command a secure wipe of IMEM and DMEM. This
occurs when OTBN encounters a fatal
869 // error.

```

```

870     if (mems_sec_wipe) begin
871         dmem_sec_wipe = 1'b1;
872         imem_sec_wipe = 1'b1;
873     end
874     // Resume execution once the cmd arrives.
875     if (status_q == StatusPaused) begin
876         if (reg2hw.cmd.qe && (reg2hw.cmd.q == CmdResume)) be-
gin
877             wfi_resume_d = 1'b1;
878         end
879     end
880 end
881 end
882
883 assign req_sec_wipe_urnd_keys = dmem_sec_wipe |
imem_sec_wipe;
884
885 assign illegal_bus_access_d = dmem_illegal_bus_access |
imem_illegal_bus_access;
886
887 // It should not be possible to request an imem or dmem ac-
cess without it being granted. Either
888 // a scramble key is present so the request will be granted
or the core is busy obtaining a new
889 // key, so no request can occur (the core won't generate one
whilst awaiting a scrambling key and
890 // the bus requests get an immediate dummy response by-
passing the dmem or imem). A fatal error is
891 // raised if request is seen without a grant.
892 assign missed_gnt_error_d = dmem_missed_gnt |
imem_missed_gnt;
893
894 // Flop `illegal_bus_access_q` and `missed_gnt_error_q` to
break timing paths from the TL
895 // interface into the OTBN core.
896 always_ff @(posedge clk_i or negedge rst_ni) begin
897     if (!rst_ni) begin
898         start_q <= 1'b0;
899         illegal_bus_access_q <= 1'b0;
900         missed_gnt_error_q <= 1'b0;
901     end else begin
902         start_q <= start_d;
903         illegal_bus_access_q <= illegal_bus_access_d;
904         missed_gnt_error_q <= missed_gnt_error_d;
905     end
906 end
907
908 // STATUS register
909 // imem/dmem scramble req can be busy when locked, so use a
priority selection so locked status
910 // always takes priority.
911 //
912 // Note that these signals are all "a cycle early". For ex-
ample, the locking signal gets asserted
913 // combinatorially on the cycle that an error is injected.
The STATUS register change, done
914 // interrupt and any change to the idle signal will be
delayed by 2 cycles.
915 assign status_d = locking ? Status-
Locked :
916     busy_secure_wipe ? Status-
BusySecWipeInt :
917     wfi_pending & ~wfi_resume_q ? Status-
Paused :
918     busy_execute_d ? Status-
BusyExecute :
919     otbn_dmem_scramble_key_req_busy ? Status-
BusySecWipeDmem :
920     otbn_imem_scramble_key_req_busy ? Status-
BusySecWipeImem :
921     StatusIdle;
922 always_ff @(posedge clk_i or negedge rst_ni) begin
923     if (!rst_ni) begin
924         status_q <= StatusBusySecWipeInt;
925     end else begin
926         status_q <= status_d;
927     end
928 end
929
930 assign hw2reg.status.d = status_q;
931 assign hw2reg.status.de = 1'b1;
932
933 // Only certain combinations of the state variable {locking,
busy_execute_d,
934 // otbn_dmem_scramble_key_req_busy,
otbn_imem_scramble_key_req_busy} are possible.
935 //
936 // - Once locking is high, we guarantee never to see a new
execution or the start of a key
937 // rotation. (Assertion: NoStartWhenLocked_A)
938
939 `ASSERT(NoStartWhenLocked_A,
940     locking ==> !($rose(busy_execute_d)) ||
941     $rose(otbn_dmem_scramble_key_req_busy)
||
942     $rose(otbn_imem_scramble_key_req_busy)))

```

```

943
944 // CTRL register
945 assign software_errs_fatal_d =
946     reg2hw.ctrl.software_errs_fatal.qe && (status_q ==
StatusIdle) ?
947     reg2hw.ctrl.software_errs_fatal.q : soft-
ware_errs_fatal_q;
948
949 assign wfi_enabled_d =
950     reg2hw.ctrl.wfi_enabled.qe && (status_q == StatusIdle) ?
951     reg2hw.ctrl.wfi_enabled.q : wfi_enabled_q;
952
953 // The URND control enable bit must be stable during an OTBN
execution because it is used to
954 // control a blanker inside otbn_rnd.sv
955 assign urnd_ctrl_enabled_d =
956     reg2hw.ctrl.urnd_ctrl_enabled.qe && (status_q ==
StatusIdle) ?
957     reg2hw.ctrl.urnd_ctrl_enabled.q : urnd_ctrl_enabled_q;
958
959 always_ff @(posedge clk_i or negedge rst_ni) begin
960     if (!rst_ni) begin
961         software_errs_fatal_q <= 1'b0;
962         wfi_enabled_q <= 1'b0;
963         urnd_ctrl_enabled_q <= 1'b0;
964     end else begin
965         software_errs_fatal_q <= software_errs_fatal_d;
966         wfi_enabled_q <= wfi_enabled_d;
967         urnd_ctrl_enabled_q <= urnd_ctrl_enabled_d;
968     end
969 end
970
971 assign hw2reg.ctrl.software_errs_fatal.d = soft-
ware_errs_fatal_q;
972 assign hw2reg.ctrl.wfi_enabled.d = wfi_enabled_q;
973 assign hw2reg.ctrl.urnd_ctrl_enabled.d = urnd_c-
trl_enabled_q;
974
975 // ERR_BITS register
976 // The error bits for an OTBN operation get stored on the
cycle that done is
977 // asserted. Software is expected to read them out before
starting the next operation.
978
979 assign hw2reg.err_bits.bad_data_addr.d =
err_bits_q.bad_data_addr;
980 assign hw2reg.err_bits.bad_insn_addr.d =
err_bits_q.bad_insn_addr;
981 assign hw2reg.err_bits.call_stack.d = err_bits_q.call_stack;
982 assign hw2reg.err_bits.illegal_insn.d =
err_bits_q.illegal_insn;
983 assign hw2reg.err_bits.loop.d = err_bits_q.loop;
984 assign hw2reg.err_bits.key_invalid.d =
err_bits_q.key_invalid;
985 assign hw2reg.err_bits.rnd_rep_chk_fail.d =
err_bits_q.rnd_rep_chk_fail;
986 assign hw2reg.err_bits.rnd_fips_chk_fail.d =
err_bits_q.rnd_fips_chk_fail;
987 assign hw2reg.err_bits.imem_intg_violation.d =
err_bits_q.imem_intg_violation;
988 assign hw2reg.err_bits.dmem_intg_violation.d =
err_bits_q.dmem_intg_violation;
989 assign hw2reg.err_bits.reg_intg_violation.d =
err_bits_q.reg_intg_violation;
990 assign hw2reg.err_bits.bus_intg_violation.d =
err_bits_q.bus_intg_violation;
991 assign hw2reg.err_bits.bad_internal_state.d =
err_bits_q.bad_internal_state;
992 assign hw2reg.err_bits.illegal_bus_access.d =
err_bits_q.illegal_bus_access;
993 assign hw2reg.err_bits.lifecycle_escalation.d =
err_bits_q.lifecycle_escalation;
994 assign hw2reg.err_bits.fatal_software.d =
err_bits_q.fatal_software;
995 assign hw2reg.err_bits.mai_software_error.d =
err_bits_q.mai_error;
996
997 assign err_bits_clear = reg2hw.err_bits.bad_data_addr.qe &
is_not_running_q;
998 assign err_bits_d = err_bits_clear ? '0 : err_bits;
999 assign err_bits_en = err_bits_clear | done_core;
1000
1001 logic unused_reg2hw_err_bits;
1002
1003 // Majority of reg2hw.err_bits is unused as write values are
ignored, all writes clear the
1004 // register to 0.
1005 assign unused_reg2hw_err_bits =
^ {reg2hw.err_bits.bad_data_addr.q,
1006    reg2hw.err_bits.bad_insn_addr,
1007    reg2hw.err_bits.call_stack,
1008    reg2hw.err_bits.illegal_insn,
1009    reg2hw.err_bits.loop,
1010    reg2hw.err_bits.key_invalid,
1011

```

```

reg2hw.err_bits.rnd_rep_chk_fail,
1012
reg2hw.err_bits.rnd_fips_chk_fail,
1013
reg2hw.err_bits.imem_intg_violation,
1014
reg2hw.err_bits.dmem_intg_violation,
1015
reg2hw.err_bits.reg_intg_violation,
1016
reg2hw.err_bits.bus_intg_violation,
1017
reg2hw.err_bits.bad_internal_state,
1018
reg2hw.err_bits.illegal_bus_access,
1019
reg2hw.err_bits.lifecycle_escalation,
1020
reg2hw.err_bits.fatal_software,
1021
reg2hw.err_bits.mai_software_error};
1022
1023 always_ff @(posedge clk_i or negedge rst_ni) begin
1024     if (!rst_ni) begin
1025         err_bits_q <= '0;
1026     end else if (err_bits_en) begin
1027         err_bits_q <= err_bits_d;
1028     end
1029 end
1030
1031 // Latch the recoverable error signal from the core. This
will be generated as a pulse some time

1032 // during the run (and before secure wipe finishes). Collect
up this bit, clearing on the start or
1033 // end of an operation (start_q / done_core, respectively)
1034 assign recoverable_err_d = (recoverable_err_q | core_recov-
erable_err) & ~(start_q | done_core);
1035 always_ff @(posedge clk_i or negedge rst_ni) begin
1036     if (!rst_ni) begin
1037         recoverable_err_q <= '0;
1038     end else begin
1039         recoverable_err_q <= recoverable_err_d;
1040     end
1041 end
1042
1043 // FATAL_ALERT_CAUSE register. The .de and .d values are
equal for each bit, so that it can only
1044 // be set, not cleared.
1045 `define
DEF_FAC_BIT(NAME)
1046     assign hw2reg.fatal_alert_cause.``NAME``.d =
1'b1;
1047     assign hw2reg.fatal_alert_cause.``NAME``.de =
err_bits.``NAME``;
1048
1049 `DEF_FAC_BIT(fatal_software)
1050 `DEF_FAC_BIT(lifecycle_escalation)
1051 `DEF_FAC_BIT(illegal_bus_access)
1052 `DEF_FAC_BIT(bad_internal_state)
1053 `DEF_FAC_BIT(bus_intg_violation)
1054 `DEF_FAC_BIT(reg_intg_violation)
1055 `DEF_FAC_BIT(dmem_intg_violation)
1056 `DEF_FAC_BIT(imem_intg_violation)
1057
1058 `undef DEF_FAC_BIT
1059
1060 // INSN_CNT register
1061 logic [31:0] insn_cnt;
1062 logic insn_cnt_clear;
1063 logic unused_insn_cnt_q;
1064 assign hw2reg.insn_cnt.d = insn_cnt;
1065 assign insn_cnt_clear = reg2hw.insn_cnt.qe & is_not_run-
ning_q;
1066 // Ignore all write data to insn_cnt. All writes zero the
register.
1067 assign unused_insn_cnt_q = ^reg2hw.insn_cnt.q;
1068
1069 // Alerts
=====
1070
1071 logic [NumAlerts-1:0] alert_test;
1072 assign alert_test[AlertFatal] = reg2hw.alert_test.fatal.q &
reg2hw.alert_test.fatal.qe;
1073 assign alert_test[AlertRecov] = reg2hw.alert_test.recov.q &
reg2hw.alert_test.recov.qe;
1074
1075 logic [NumAlerts-1:0] alerts;
1076 assign alerts[AlertFatal] = {err_bits.fatal_software,
err_bits.lifecycle_escalation,
err_bits.illegal_bus_access,
err_bits.bad_internal_state,
err_bits.bus_intg_violation,
err_bits.reg_intg_violation,
err_bits.dmem_intg_violation,
err_bits.imem_intg_violation};
1077
1078
1079
1080
1081
1082
1083
1084
1085 assign alerts[AlertRecov] = (core_recoverable_err | recover-
able_err_q) & done_core;
1086
1087 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx

```

```

1088     prim_alert_sender #(
1089         .AsyncOn(AlertAsyncOn[i]),
1090         .SkewCycles(AlertSkewCycles),
1091         .IsFatal(i == AlertFatal)
1092     ) u_prim_alert_sender (
1093         .clk_i,
1094         .rst_ni (rst_n),
1095         .alert_test_i (alert_test[i]),
1096         .alert_req_i (alerts[i]),
1097         .alert_ack_o (),
1098         .alert_state_o(),
1099         .alert_rx_i (alert_rx_i[i]),
1100         .alert_tx_o (alert_tx_o[i])
1101     );
1102 end
1103
1104
1105 // EDN Connections
=====
1106 logic edn_rnd_req, edn_rnd_ack;
1107 logic [EdnDataWidth-1:0] edn_rnd_data;
1108 logic edn_rnd_fips, edn_rnd_err;
1109
1110 // This EDN request module synchronizes the data coming from
EDN and stack the 32 bit EDN words
1111 // to achieve an internal entropy width of 256 bit.
1112 prim_edn_req #(
1113     .EnRstChks(1'b1),
1114     .Outwidth(EdnDataWidth),
1115     // SEC_CM: RND.BUS.CONSTENCY
1116     .RepCheck(1'b1)
1117 ) u_prim_edn_req (
1118     .clk_i,
1119     .rst_ni (rst_n),
1120     .req_chk_i (1'b1),
1121     .req_i (edn_rnd_req),
1122     .ack_o (edn_rnd_ack),
1123     .data_o (edn_rnd_data),
1124     .fips_o (edn_rnd_fips),
1125     .err_o (edn_rnd_err),
1126     .clk_edn_i,
1127     .rst_edn_ni,
1128     .edn_o (edn_rnd_o),
1129     .edn_i (edn_rnd_i)
1130 );
1131
1132 edn_pkg::edn_req_t edn_urnd_req;
1133 edn_pkg::edn_rsp_t edn_urnd_rsp;
1134
1135 // Synchronize the data from the EDN network to the main
clock of OTBN.
1136 prim_sync_reqack_data #(
1137     .Width(edn_pkg::ENDPOINT_BUS_WIDTH + 32'd1),
1138     .EnRstChks(1'b1),
1139     .DataSrc2Dst(1'b0),
1140     .DataReg(1'b0)
1141 ) u_prim_sync_reqack_data_urnd (
1142     .clk_src_i (clk_i),
1143     .rst_src_ni (rst_n),
1144     .clk_dst_i (clk_edn_i),
1145     .rst_dst_ni (rst_edn_ni),
1146     .req_chk_i (1'b1),
1147     .src_req_i (edn_urnd_req.edn_req),
1148     .src_ack_o (edn_urnd_rsp.edn_ack),
1149     .dst_req_o (edn_urnd_o.edn_req),
1150     .dst_ack_i (edn_urnd_i.edn_ack),
1151     .data_i ({edn_urnd_i.edn_fips,
edn_urnd_i.edn_bus}),
1152     .data_o ({edn_urnd_rsp.edn_fips, edn_urnd_rsp.edn_bus})
1153 );
1154
1155 // OTBN Core
=====
1156
1157 always_ff @(posedge clk_i or negedge rst_n) begin
1158     if (!rst_n) begin
1159         busy_execute_q <= 1'b0;
1160         wfi_resume_q <= 1'b0;
1161         init_sec_wipe_done_q <= 1'b0;
1162     end else begin
1163         busy_execute_q <= busy_execute_d;
1164         wfi_resume_q <= wfi_resume_d;
1165         init_sec_wipe_done_q <= init_sec_wipe_done_d;
1166     end
1167 end
1168 // OTBN is busy executing once started or resumed until it
is done or gets paused. The
1169 // wfi_pending signal has only effect if we haven't already
received the resume command.
1170 assign busy_execute_d = wfi_resume_d |
1171 ((busy_execute_q | start_d) &
1172 ~done_core | (wfi_pending &
~wfi_resume_q));
1173 assign init_sec_wipe_done_d = init_sec_wipe_done_q |
~wfi_secure_wipe;
1174
1175 otbn_core #(
1176     .RegFile(RegFile),
1177     .DmemSizeByte(DmemSizeByte),
1178     .ImemSizeByte(ImemSizeByte),

```

```

1179 .RndCnstUrndPrngSeed(RndCnstUrndPrngSeed),
1180 .RndCnstUrndPerm(RndCnstUrndPerm),
1181 .SecFixMaiOpSeq(SecFixMaiOpSeq),
1182 .SecFixMacOpSeq(SecFixMacOpSeq),
1183 .FeatStubMai(FeatStubMai),
1184 .SecSkipUrndReseedAtStart(SecSkipUrndReseedAtStart),
1185 .RndCnstBnMacUrndPerm(RndCnstBnMacUrndPerm),
1186 .RndCnstMaiUrndPerm(RndCnstMaiUrndPerm)
1187 ) u_otbn_core (
1188   .clk_i,
1189   .rst_ni                                (rst_n),
1190
1191   .start_i                                (start_q),
1192   .done_o                                (done_core),
1193   .locking_o                              (locking),
1194   .secure_wipe_running_o                 (busy_secure_wipe),
1195
1196   .err_bits_o                             (core_err_bits),
1197   .recoverable_err_o                     (core_recoverable_err),
1198
1199   .imem_req_o                             (imem_req_core),
1200   .imem_addr_o                           (imem_addr_core),
1201   .imem_rdata_i                          (imem_rdata_core),
1202   .imem_rvalid_i                         (imem_rvalid_core),
1203
1204   .dmem_req_o                             (dmem_req_core),
1205   .dmem_write_o                          (dmem_write_core),
1206   .dmem_addr_o                           (dmem_addr_core),
1207   .dmem_wdata_o                          (dmem_wdata_core),
1208   .dmem_wmask_o                          (dmem_wmask_core),
1209   .dmem_rmask_o                          (dmem_rmask_core_d),
1210   .dmem_rdata_i                          (dmem_rdata_core),
1211   .dmem_rvalid_i                         (dmem_rvalid_core),
1212   .dmem_rerror_i                         (dmem_rerror_core),
1213
1214   .edn_rnd_req_o                         (edn_rnd_req),
1215   .edn_rnd_ack_i                         (edn_rnd_ack),
1216   .edn_rnd_data_i                       (edn_rnd_data),
1217   .edn_rnd_fips_i                       (edn_rnd_fips),
1218   .edn_rnd_err_i                        (edn_rnd_err),
1219
1220   .edn_urnd_o                            (edn_urnd_req),
1221   .edn_urnd_i                            (edn_urnd_rsp),
1222
1223   .wfi_enabled_i                         (wfi_enabled_q),
1224   .wfi_pending_o                         (wfi_pending),
1225   .wfi_resume_i                          (wfi_resume_q),
1226
1227   .urnd_ctrl_enabled_i                  (urnd_ctrl_enabled_q),
1228
1229   .insn_cnt_o                            (insn_cnt),
1230   .insn_cnt_clear_i                     (insn_cnt_clear),
1231
1232   .mems_sec_wipe_o                       (mems_sec_wipe),
1233   .dmem_sec_wipe_urnd_key_o              (dmem_sec_wipe_urnd_key),
1234   .imem_sec_wipe_urnd_key_o              (imem_sec_wipe_urnd_key),
1235   .req_sec_wipe_urnd_keys_i              (req_sec_wipe_urnd_keys),
1236
1237   .escalate_en_i                         (core_escalate_en),
1238   .rma_req_i                             (mubi_rma_req),
1239   .rma_ack_o                             (mubi_rma_ack),
1240
1241   .software_errs_fatal_i                 (software_errs_fatal_q),
1242
1243   .sideload_key_shares_i                 (keymgr_key_i.key),
1244   .sideload_key_shares_valid_i           ({keymgr_key_i.valid}),
1245
1246   // The naming kmac_data is just to be consistent with oth-
1247   // er IPs connecting to KMAC. From here
1248   // on use more sensible name.
1249   .kmac_app_req_o(kmac_data_o),
1250   .kmac_app_rsp_i(kmac_data_i)
1251 );
1252
1253 always_ff @(posedge clk_i or negedge rst_n) begin
1254   if (!rst_n) begin
1255     locking_q <= 1'b0;
1256   end else begin
1257     locking_q <= locking;
1258   end
1259 end
1260
1261 // Collect up the error bits that don't come from the core
1262 // itself and latch them so that they'll
1263 // be available when an operation finishes.
1264 assign non_core_err_bits = '{
1265   lifecycle_escalation: lc_c-
1266   trl_pkg::lc_tx_test_true_loose(lc_escalate_en[0]),
1267   illegal_bus_access:   illegal_bus_access_q,
1268   bad_internal_state:   otbn_scramble_state_error |
1269   missed_gnt_error_q,
1270   bus_intg_violation:   bus_intg_violation
1271 };
1272
1273 assign non_core_err_bits_d = non_core_err_bits_q |
1274 non_core_err_bits;
1275
1276 always_ff @(posedge clk_i or negedge rst_ni) begin
1277   if (!rst_ni) begin
1278     non_core_err_bits_q <= '0;
1279   end else begin
1280     non_core_err_bits_q <= non_core_err_bits_d;
1281   end
1282 end
1283
1284 end
1285
1286 // Construct a full set of error bits from the core output
1287 assign err_bits = '{
1288   fatal_software:      core_err_bits.fatal_software,
1289   lifecycle_escalation: core_err_bits.fatal_software,
1290   non_core_err_bits_d.lifecycle_escalation,
1291   illegal_bus_access:  core_err_bits.fatal_software,
1292   non_core_err_bits_d.illegal_bus_access,
1293   bad_internal_state:  |{core_err_bits.bad_internal_state,
1294   non_core_err_bits_d.bad_internal_state},
1295   bus_intg_violation:  core_err_bits.dmem_intg_violation,
1296   non_core_err_bits_d.bus_intg_violation,
1297   reg_intg_violation:  core_err_bits.reg_intg_violation,
1298   dmem_intg_violation: core_err_bits.dmem_intg_violation,
1299   imem_intg_violation: core_err_bits.imem_intg_violation,
1300   rnd_fips_chk_fail:   core_err_bits.rnd_fips_chk_fail,
1301   rnd_rep_chk_fail:    core_err_bits.rnd_rep_chk_fail,
1302   key_invalid:         core_err_bits.key_invalid,
1303   loop:                core_err_bits.loop,
1304   illegal_insn:        core_err_bits.illegal_insn,
1305   call_stack:          core_err_bits.call_stack,
1306   bad_insn_addr:       core_err_bits.bad_insn_addr,
1307   bad_data_addr:       core_err_bits.bad_data_addr,
1308   mai_error:           core_err_bits.mai_error
1309 };
1310
1311 // An error signal going down into the core to show that it
1312 // should locally escalate. In
1313 // accordance with the lc_ctrl spec, all values of the
1314 // lc_escalate_en signal other than the OFF
1315 // value must be interpreted as ON.
1316 assign core_escalate_en = mubi4_bool_to_mubi(
1317   |{non_core_err_bits.illegal_bus_access,
1318   non_core_err_bits.bad_internal_state,
1319   non_core_err_bits.bus_intg_violation,
1320   lc_c-
1321   trl_pkg::lc_tx_test_true_loose(lc_escalate_en[1])});
1322
1323 // The core can never signal a write to IMEM
1324 assign imem_write_core = 1'b0;
1325
1326 // Asserts
1327
1328 =====
1329 for (genvar i = 0; i < LoopStackDepth; ++i) begin :
1330   gen_loop_stack_ctr_asserts
1331   `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT_IN(
1332     LoopStackCntAlertCheck_A,
1333     u_otbn_core.u_otbn_controller.u_otbn_loop_controller.g_loop_counters
1334     [i].u_loop_count,
1335     alert_tx_o[AlertFatal]
1336   )
1337 end
1338
1339 // GPR assertions for secure wipe
1340 // 1. urnd_reseed_err disables the assertion because secure
1341 // wipe finishes with failure and OTBN
1342 // goes to LOCKED state immediately after this error which
1343 // means that it's not guaranteed to have
1344 // secure wiping complete.
1345
1346 // 2. mubi_err_d of start_stop_control disables the internal
1347 // secure wipe related assertion
1348 // because a fatal error affecting internal secure wiping
1349 // could cause an immediate locking
1350 // behaviour in which it's not guaranteed to see a success-
1351 // ful secure wipe.
1352 for (genvar i = 2; i < NGpr; ++i) begin :
1353   gen_sec_wipe_gpr_asserts
1354   // Initial secure wipe needs to initialise all registers
1355   // to nonzero
1356   `ASSERT(InitSecWipeNonZeroBaseRegs_A,
1357     $fell(busy_secure_wipe) |>
1358   )
1359   u_otbn_core.u_otbn_rf_base.gen_rf_base_ff.u_otbn_rf_base_inner.g_rf_
1360   flops[i].rf_reg_q !=
1361   EccZeroWord,
1362   clk_i,
1363   !rst_ni || u_otbn_core.urnd_reseed_err ||
1364   u_otbn_core.u_otbn_start_stop_control.mubi_err_d ||
1365   u_otbn_core.u_otbn_start_stop_control.state_error_d)
1366   // After execution, it's expected to see a change result-
1367   // ing with a nonzero register value
1368   `ASSERT(SecWipeChangedBaseRegs_A,
1369     $rose(busy_secure_wipe) |> ((#[0:$]
1370   )
1371   u_otbn_core.u_otbn_rf_base.gen_rf_base_ff.u_otbn_rf_base_inner.g_rf_
1372   flops[i].rf_reg_q !=
1373   EccZeroWord &&
1374   $changed(
1375   u_otbn_core.u_otbn_rf_base.gen_rf_base_ff.u_otbn_rf_base_inner.g_rf_
1376   flops[i].rf_reg_q))
1377   within ($rose(busy_secure_wipe) #[0:$] $fell(busy_se-
1378   cure_wipe))),
1379   clk_i,
1380   !rst_ni || u_otbn_core.urnd_reseed_err ||

```

```

u_otbn_core.u_otbn_start_stop_control.mubi_err_d ||
1348     u_otbn_core.u_otbn_start_stop_control.state_error_d)
1349 end
1350
1351 // We have several assertions that check that secure wipe
worked properly. However, we've also got
1352 // some tests where we force nets, stopping it from working
properly! That's fine, and the tests
1353 // are checking that some other mechanism catches the prob-
lem. However, we don't want the
1354 // simulation to die with a failed assertion, so we put
everything in a named block which we can
1355 // turn off with $assertoff.
1356 //
1357 // The silly-looking name is to avoid a lint warning.
Verible (correctly) points out that
1358 // SystemVerilog doesn't allow bare begin/end blocks at mod-
ule level. So I cheated and put
1359 // everything in an if(1) block. But this is treated as a
generate block, and our lint rules
1360 // therefore expect its name to start with a "g.".
1361 if (1) begin : g_secure_wipe_assertions
1362 // WDR assertions for secure wipe
1363 // 1. urnd_reseed_err disables the assertion because se-
cure wipe finishes with failure and OTBN
1364 // goes to LOCKED state immediately after this error which
means that it's not guaranteed to
1365 // have secure wiping complete.
1366 // 2. mubi_err_d of start_stop_control disables the in-
ternal secure wipe related assertion
1367 // because a fatal error affecting internal secure wiping
could cause an immediate locking
1368 // behaviour in which it's not guaranteed to see a suc-
cessful secure wipe.
1369 for (genvar i = 0; i < NWdr; ++i) begin :
gen_sec_wipe_wdr_asserts
1370 // Initial secure wipe needs to initialise all registers
to nonzero
1371 `ASSERT(InitSecWipeNonZeroWideRegs_A,
1372     $fell(busy_secure_wipe) |>
1373 )
1374 u_otbn_core.u_otbn_rf_bignum.gen_rf_bignum_ff.u_otbn_rf_bignum_inner
.rf[i] !=
1375     EccWideZeroWord,
1376     clk_i,
1377     !rst_ni || u_otbn_core.urnd_reseed_err ||
1378 u_otbn_core.u_otbn_start_stop_control.mubi_err_d ||
1379 u_otbn_core.u_otbn_start_stop_control.state_error_d)
1380 // After execution, it's expected to see a change res-
ulting with a nonzero register value
1381 `ASSERT(SecWipeChangedWideRegs_A,
1382     $rose(busy_secure_wipe) |> ((#[0:$]
1383 )
1384 u_otbn_core.u_otbn_rf_bignum.gen_rf_bignum_ff.u_otbn_rf_bignum_inner
.rf[i] !=
1385     EccWideZeroWord &&
1386     $changed(
1387 u_otbn_core.u_otbn_rf_bignum.gen_rf_bignum_ff.u_otbn_rf_bignum_inner
.rf[i]))
1388     within ($rose(busy_secure_wipe) #[0:$]
1389 $fell(busy_secure_wipe))),
1390     clk_i, !rst_ni || u_otbn_core.urnd_reseed_err ||
1391 u_otbn_core.u_otbn_start_stop_control.mubi_err_d)
1392 end
1393 // Secure wipe needs to invalidate call and loop stack, ini-
tialize MOD, ACC to nonzero and set
1394 // FLAGS CSR to zero
1395 // 1. urnd_reseed_err disables the assertion because secure
wipe finishes with failure and OTBN
1396 // goes to LOCKED state immediately after this error which
means that it's not guaranteed to have
1397 // secure wiping complete.
1398 // 2. mubi_err_d of start_stop_control disables the secure
wipe related assertions because a
1399 // fatal error affecting internal secure wiping could cause
an immediate locking behaviour
1400 // in which it's not guaranteed to see a successful secure
wipe.
1401 `ASSERT(SecWipeInvalidCallStack_A,
1402     $fell(busy_secure_wipe) |> (!
1403 u_otbn_core.u_otbn_rf_base.u_call_stack.top_valid_o),
1404     clk_i,
1405     !rst_ni || u_otbn_core.urnd_reseed_err ||
1406     u_otbn_core.u_otbn_start_stop_control.mubi_err_d
1407 ||
1408 u_otbn_core.u_otbn_start_stop_control.state_error_d ||
1409 u_otbn_core.u_otbn_start_stop_control.state_error_d)
1410 `ASSERT(SecWipeInvalidLoopStack_A,
1411     $fell(busy_secure_wipe) |>
1412     (!

```

```

u_otbn_core.u_otbn_controller.u_otbn_loop_controller.loop_info_stack
.top_valid_o),
1411     clk_i,
1412     !rst_ni || u_otbn_core.urnd_reseed_err ||
1413     u_otbn_core.u_otbn_start_stop_control.mubi_err_d
1414 ||
1415 u_otbn_core.u_otbn_start_stop_control.state_error_d ||
1416 u_otbn_core.u_otbn_start_stop_control.state_error_d)
1417 `ASSERT(SecWipeNonZeroMod_A,
1418     $fell(busy_secure_wipe) |>
1419 u_otbn_core.u_otbn_alu_bignum.mod_intg_q != EccWideZeroWord,
1420     clk_i,
1421     !rst_ni || u_otbn_core.urnd_reseed_err ||
1422     u_otbn_core.u_otbn_start_stop_control.mubi_err_d
1423 ||
1424 u_otbn_core.u_otbn_start_stop_control.state_error_d ||
1425 u_otbn_core.u_otbn_start_stop_control.state_error_d)
1426 `ASSERT(SecWipeNonZeroACC_A,
1427     $fell(busy_secure_wipe) |>
1428     u_otbn_core.u_otbn_alu_bignum.ispr_acc_intg_i !=
EccWideZeroWord,
1429     clk_i,
1430     !rst_ni || u_otbn_core.urnd_reseed_err ||
1431     u_otbn_core.u_otbn_start_stop_control.mubi_err_d
1432 ||
1433 u_otbn_core.u_otbn_start_stop_control.state_error_d)
1434 `ASSERT(SecWipeNonZeroFlags_A,
1435     $fell(busy_secure_wipe) |> (!
1436 u_otbn_core.u_otbn_alu_bignum.flags_flattened),
1437     clk_i,
1438     !rst_ni || u_otbn_core.urnd_reseed_err ||
1439     u_otbn_core.u_otbn_start_stop_control.mubi_err_d
1440 ||
1441 u_otbn_core.u_otbn_start_stop_control.state_error_d ||
1442 u_otbn_core.u_otbn_start_stop_control.state_error_d)
1443 // Secure wipe of IMEM and DMEM first happens with a key
change from URND (while valid is zero)
1444 `ASSERT(ImemSecWipeRequiresUrndKey_A,
1445     $rose(imem_sec_wipe) |> (otbn_imem_scramble_key ==
1446 $past(imem_sec_wipe_urnd_key)),
1447     clk_i,
1448     !rst_ni || u_otbn_core.urnd_reseed_err ||
1449     u_otbn_core.u_otbn_start_stop_control.mubi_err_d)
1450 `ASSERT(DmemSecWipeRequiresUrndKey_A,
1451     $rose(dmem_sec_wipe) |> (otbn_dmem_scramble_key ==
1452 $past(dmem_sec_wipe_urnd_key)),
1453     clk_i,
1454     !rst_ni || u_otbn_core.urnd_reseed_err ||
1455     u_otbn_core.u_otbn_start_stop_control.mubi_err_d)
1456 // Then it is guaranteed to have a valid key from OTP inter-
face which is different from URND key
1457 `ASSERT(ImemSecWipeRequiresOtpKey_A,
1458     $rose(imem_sec_wipe) #[1] (otbn_imem_scramble_key ==
1459 $past(imem_sec_wipe_urnd_key)) |>
1460     #[0:$] otbn_imem_scramble_valid &&
1461 $changed(otbn_imem_scramble_key),
1462     clk_i,
1463     !rst_ni || u_otbn_core.urnd_reseed_err ||
1464     u_otbn_core.u_otbn_start_stop_control.mubi_err_d)
1465 `ASSERT(DmemSecWipeRequiresOtpKey_A,
1466     $rose(dmem_sec_wipe) #[1] (otbn_dmem_scramble_key ==
1467 $past(dmem_sec_wipe_urnd_key)) |>
1468     #[0:$] otbn_dmem_scramble_valid &&
1469 $changed(otbn_dmem_scramble_key),
1470     clk_i,
1471     !rst_ni || u_otbn_core.urnd_reseed_err ||
1472     u_otbn_core.u_otbn_start_stop_control.mubi_err_d)
1473 // All outputs should be known value after reset
1474 `ASSERT_KNOWN(Tl0ValidKnown_A, tl_o.d_valid)
1475 `ASSERT_KNOWN(Tl0AReadyKnown_A, tl_o.a_ready)
1476 `ASSERT_KNOWN(IdleOKKnown_A, idle_o)
1477 `ASSERT_KNOWN(IntrDoneOKKnown_A, intr_done_o)
1478 `ASSERT_KNOWN(AlertTxOKKnown_A, alert_tx_o)
1479 `ASSERT_KNOWN(LcRmaAckKnown_A, lc_rma_ack_o)
1480 `ASSERT_KNOWN(RamCfgDmemKnown_A, ram_cfg_dmem_o)
1481 `ASSERT_KNOWN(RamCfgImemKnown_A, ram_cfg_imem_o)
1482 `ASSERT_KNOWN(EdnRndOKKnown_A, edn_rnd_o, clk_edn_i, !
rst_edn_ni)
1483 `ASSERT_KNOWN(EdnUrndOKKnown_A, edn_urnd_o, clk_edn_i, !
rst_edn_ni)
1484 `ASSERT_KNOWN(otbnOtpKey0_A, otbn_otp_key_o, clk_otp_i, !
rst_otp_ni)
1485 `ASSERT_KNOWN(ErrBitsKnown_A, err_bits)
1486 // The data part of the request directly originates from
WSRs. These are non resettable flops.
1487 // When a simulation starts, these are still X as only a se-
cure wipe will set a value. We thus
1488 // only check whether the data is known when the valid is

```



```

set.
1483 `ASSERT_KNOWN(KmacReqKnown_A, {kmac_data_o.req_last,
kmac_data_o.req_valid,
1484                                     kmac_data_o.rsp_ready,
kmac_data_o.strb})
1485 `ASSERT_KNOWN_IF(KmacReqDataKnown_A, {kmac_data_o.data_s0,
kmac_data_o.data_s1},
1486                                     kmac_data_o.req_valid)
1487
1488 // Incoming key must be valid (other inputs go via prim mod-
ules that handle the X checks).
1489 `ASSERT_KNOWN(KeyMgrKeyValid_A, keymgr_key_i.valid)
1490
1491 // In locked state, the readable registers INSN_CNT, IMEM,
and DMEM are expected to always read 0

1492 // when accessed from the bus. For INSN_CNT, we use "|>=" so
that the assertion lines up with
1493 // "status.q" (a signal that isn't directly accessible
here).
1494 `ASSERT(LockedInsnCntReadsZero_A, (hw2reg.status.d ==
StatusLocked) |>= insn_cnt == 'd0)
1495 `ASSERT(ExecuteOrLockedImemReadsZero_A,
1496 (hw2reg.status.d inside {StatusBusyExecute, Status-
Locked}) & imem_rvalid_bus
1497 |>= imem_rdata_bus == 'd0)
1498 `ASSERT(ExecuteOrLockedDmemReadsZero_A,
1499 (hw2reg.status.d inside {StatusBusyExecute, Status-
Locked}) & dmem_rvalid_bus
1500 |>= dmem_rdata_bus == 'd0)
1501
1502 // While paused OTBN must not report idle, so the clock
keeps running to observe CMD.RESUME.
1503 `ASSERT(WfiPendingNotIdle_A, wfi_pending |>= (idle_o ==
prim_mubi_pkg::MuBi4False))
1504
1505 // While paused the DMEM is released to the bus until the
resume command arrived.
1506 `ASSERT(PausedDmemReleased_A, (status_q == StatusPaused)
&& !wfi_resume_q |>= !dmem_access_core)
1507
1508 // The paused status and the core's pending WFI cannot drift
apart except when a wipe starts on a
1509 // WFI instruction.
1510 `ASSERT(PausedImpliesWfiPending_A, (status_q == Status-
Paused) && !busy_secure_wipe
1511 |>= wfi_pending)
1512
1513 // OTBN can only be paused if the WFI instruction is en-
abled.
1514 `ASSERT(WfiPendingImpliesEnabled_A, wfi_pending |>= wfi-en-
abled_q)
1515
1516 // CTRL.wfi_enabled is only writable while idle, so it must
not change otherwise.
1517 `ASSERT(WfiEnabledStableUnlessIdle_A, (status_q !=
StatusIdle) |>= $stable(wfi_enabled_q))
1518
1519 // From the cycle the core is told to start to when it is
done, it must always be busy executing,
1520 // locking, or paused -- even if the core is never done. We
use this property to enable blanking
1521 // while the core is executing or locking, and this asser-
tion ensures that there is no gap
1522 // between execution and locking (except a valid pause).
1523 `ASSERT(BusyOrLockingFromStartToDone_A,
1524 $rose(start_q) |>= (busy_execute_d | locking |
wfi_pending) |>= ##[0:$] $rose(done_core))
1525
1526 // Error handling: if we pass an error signal down to the
core then we should also be setting an
1527 // error flag. Note that this uses err_bits, not err_bits_q,
because the latter signal only gets
1528 // asserted when an operation finishes.
1529 `ASSERT(ErrBitIfEscalate_A, mu-
bi4_test_true_loose(core_escalate_en) |>= |err_bits)
1530
1531 // Constraint from package, check here as we cannot have
`ASSERT_INIT in package
1532 `ASSERT_INIT(WsrSizeMatchesParameter_A, $bits(wsr_e) == Ws-
rNumWidth)
1533
1534 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT_IN(
1535 OtnbStartStopFsmCheck_A,
1536 u_otbn_core.u_otbn_start_stop_control.u_state_regs,
1537
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1538 )
1539 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT_IN(
1540 OtnbControllerFsmCheck_A,
1541 u_otbn_core.u_otbn_controller.u_state_regs,
1542
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1543 )
1544 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT_IN(
1545 OtnbScrambleCtrlFsmCheck_A,
1546 u_otbn_scramble_ctrl.u_state_regs,
1547
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1548 )
1549
1550 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT_IN(
1551 OtnbKmacFsmCheck_A,
1552 u_otbn_core.u_otbn_kmac_if.u_state_regs,
1553
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1554 )
1555
1556 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT_IN(
1557 OtnbCallStackWrPtrAlertCheck_A,
1558 u_otbn_core.u_otbn_rf_base.u_call_stack.u_stack_wr_ptr,
1559
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1560 )
1561
1562 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT_IN(
1563 OtnbLoopInfoStackWrPtrAlertCheck_A,
1564
u_otbn_core.u_otbn_controller.u_otbn_loop_controller.loop_info_stack
.u_stack_wr_ptr,
1565
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1566 )
1567
1568 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT_IN(
1569 OtnbKmacMsgSelectorCntAlertCheck_A,
1570 u_otbn_core.u_otbn_kmac_if.u_msg_selector_count,
1571
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1572 )
1573
1574 `ASSERT_ERROR_TRIGGER_ALERT_IN(
1575 OtnbBnMacCycleCountAlertCheck_A,
1576 u_otbn_core.u_otbn_mac_bignum.u_mac_bignum_fsm,
1577
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i,
1578 0, 2, state_err_o)
1579
1580 // Alert assertions for reg_we onehot check
1581 `ASSERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT_IN(
1582 RegWeOnehotCheck_A,
1583 u_reg,
1584
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1585 )
1586
1587 // other onehot checks
1588 `ASSERT_PRIM_ONEHOT_ERROR_TRIGGER_ALERT_IN(
1589 RfBaseOnehotCheck_A,
1590
u_otbn_core.u_otbn_rf_base.gen_rf_base_ff.u_otbn_rf_base_inner.u_pr
im_onehot_check,
1591
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1592 )
1593
1594 `ASSERT_PRIM_ONEHOT_ERROR_TRIGGER_ALERT_IN(
1595 RfBignumOnehotCheck_A,
1596
u_otbn_core.u_otbn_rf_bignum.gen_rf_bignum_ff.u_otbn_rf_bignum_inner
.u_prim_onehot_check,
1597
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1598 )
1599
1600 `ASSERT_PRIM_FIFO_SYNC_ERROR_TRIGGERS_ALERT1_IN(
1601 DmemRspFifo,
1602 u_tlu_adapter_sram_dmem.u_rspfifo,
1603
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1604 )
1605
1606 `ASSERT_PRIM_FIFO_SYNC_ERROR_TRIGGERS_ALERT1_IN(
1607 DmemSramReqFifo,
1608 u_tlu_adapter_sram_dmem.u_sramreqfifo,
1609
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1610 )
1611
1612 `ASSERT_PRIM_FIFO_SYNC_ERROR_TRIGGERS_ALERT1_IN(
1613 ImemRspFifo,
1614 u_tlu_adapter_sram_imem.u_rspfifo,
1615
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1616 )
1617
1618 `ASSERT_PRIM_FIFO_SYNC_ERROR_TRIGGERS_ALERT1_IN(
1619 ImemSramReqFifo,
1620 u_tlu_adapter_sram_imem.u_sramreqfifo,
1621
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1622 )
1623
1624 `ASSERT_PRIM_FIFO_SYNC_ERROR_TRIGGERS_ALERT1_IN(
1625 ImemReqFifo,
1626 u_tlu_adapter_sram_imem.u_reqfifo,
1627
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1628 )
1629
1630 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT_IN(

```

```

1628     MaiMaskFifoWptrCheck_A,
1629 u_otbn_core.gen_mai.u_otbn_mai.u_otbn_mask_accelerator.u_prim_fifo_
sync_mask
1630     .gen_normal_fifo.u_fifo_cnt.gen_secure_ptrs.u_wptr,
1631 gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1632 )
1633     ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT_IN(
1634         MaiMaskFifoRptrCheck_A,
1635 u_otbn_core.gen_mai.u_otbn_mai.u_otbn_mask_accelerator.u_prim_fifo_
sync_mask
1636     .gen_normal_fifo.u_fifo_cnt.gen_secure_ptrs.u_rptr,
1637 gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1638 )
1639     ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT_IN(
1640         OtbmMaiMaskCntAlertCheck_A,
1641 u_otbn_core.gen_mai.u_otbn_mai.u_otbn_mask_accelerator.u_otbn_sec_a
dd_mod.u_prim_count_add_inp,
1642 gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1643 )
1644     ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT_IN(
1645         OtbmMaiInputCntAlertCheck_A,
1646 u_otbn_core.gen_mai.u_otbn_mai.u_prim_count_input_word_select,
1647 gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1648 )
1649     ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT_IN(
1650         OtbmMaiOutputCntAlertCheck_A,
1651 u_otbn_core.gen_mai.u_otbn_mai.u_prim_count_output_word_select,
1652 gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1653 )
1654 endmodule

```

2. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_alu_base.sv (137 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8  * OTBN execute block for the base instruction subset
9  *
10  * This ALU supports the execution of all of OTBN's base in-
struction subset.
11  */
12 module otbn_alu_base
13     import otbn_pkg::*;
14 (
15     // Block is combinatorial; clk/rst are for assertions only.
16     input logic clk_i,
17     input logic rst_ni,
18
19     input alu_base_operation_t operation_i,
20     input alu_base_comparison_t comparison_i,
21
22     output logic [31:0] operation_result_o,
23     output logic      comparison_result_o
24 );
25
26     logic [32:0] adder_op_a, adder_op_b;
27     logic      adder_op_b_negate;
28     logic [32:0] adder_result;
29
30     logic [31:0] and_result;
31     logic [31:0] or_result;
32     logic [31:0] xor_result;
33     logic [31:0] not_result;
34
35     logic      is_equal;
36
37     // Adder //
38     // //
39     // //
40
41 // Adder takes in 33-bit operands. The addition of the input
operands occurs on bits [32:1],
42 // setting addr_op_b_negate will cause a carry-in into bit
1. Combined with an inversion of
43 // operation_i.operand_b this gives a two's-complement nega-
tion (~b + 1)
44
45     assign adder_op_b_negate = operation_i.op == AluOpBaseSub;
46
47     assign adder_op_a = {operation_i.operand_a, 1'b1};
48     assign adder_op_b = adder_op_b_negate ? {-opera-
tion_i.operand_b, 1'b1} :

```

```

49     { opera-
tion_i.operand_b, 1'b0};
50
51     assign adder_result = adder_op_a + adder_op_b;
52
53     // Bit-wise logical ops //
54     // //
55     // //
56
57     assign and_result = operation_i.operand_a & opera-
tion_i.operand_b;
58     assign or_result  = operation_i.operand_a | opera-
tion_i.operand_b;
59     assign xor_result = operation_i.operand_a ^ opera-
tion_i.operand_b;
60     assign not_result = ~operation_i.operand_a;
61
62     // Shifter //
63     // //
64
65     logic [32:0] shift_in;
66     logic [4:0] shift_amt;
67     logic [31:0] operand_a_reverse;
68     logic [32:0] shift_out;
69     logic [31:0] shift_out_reverse;
70
71     for (genvar i = 0; i < 32; i++) begin : g_shifter_reverses
72         assign operand_a_reverse[i] = operation_i.operand_a[31-i];
73         assign shift_out_reverse[i] = shift_out[31-i];
74     end
75
76     assign shift_amt = operation_i.operand_b[4:0];
77     // Shifter performs right arithmetic 33-bit shifts. Force
top bit to 0 to get logical shifting
78     // otherwise replicate top bit of shift_in. Left shifts per-
formed by reversing the input and
79     // output.
80     assign shift_in[31:0] = (operation_i.op == AluOpBaseSll) ?
operand_a_reverse :
81     operation_i.operand_a;
82     assign shift_in[32] = (operation_i.op == AluOpBaseSra) ? op-
eration_i.operand_a[31] : 1'b0;
83
84     logic signed [32:0] shift_in_signed;
85     assign shift_in_signed = signed'(shift_in);
86     assign shift_out = unsigned'(shift_in_signed >>> shift_amt);
87
88     // Output Mux //
89     // //
90     // //
91
92     always_comb begin
93         operation_result_o = adder_result[32:1];
94
95         unique case (operation_i.op)
96             AluOpBaseAnd: operation_result_o = and_result;
97             AluOpBaseOr:  operation_result_o = or_result;
98             AluOpBaseXor: operation_result_o = xor_result;
99             AluOpBaseNot: operation_result_o = not_result;
100             AluOpBaseSra: operation_result_o = shift_out[31:0];
101             AluOpBaseSrl: operation_result_o = shift_out[31:0];
102             AluOpBaseSll: operation_result_o = shift_out_reverse;
103             default: ;
104         endcase
105     end
106
107     // Comparisons //
108     // //
109     // //
110
111     // Dedicated comparator to deal with branches. As only
Equal/Not-Equal is required no need to use
112
113     // the adder (which frees it to compute the branch target in
the same cycle). No point in using
114
115     // existing XOR logic as area added from extra mux to choose
xor operands negates area saving from
116     // avoiding an dedicated comparator.
117     assign is_equal = comparison_i.operand_a == comparis-
on_i.operand_b;
118
119     assign comparison_result_o = (comparison_i.op == Comparis-
onOpBaseEq) ? is_equal : ~is_equal;
120
121     // The bottom bit of adder_result is discarded. It simply
corresponds to the carry in used to
122     // produce twos complement subtraction from an addition.
123     logic unused_adder_result_bit;
124
125     // The top bit of shift_out is discarded. shift_in contains
an extra bit to deal with sign
126     // extension which isn't needed in the shift_out result.
127     logic unused_shift_out_result_bit;
128     assign unused_shift_out_result_bit = shift_out[32];
129
130     assign unused_adder_result_bit = adder_result[0];
131
132     // clk_i, rst_ni are only used by assertions

```

```

132 logic unused_clk;
133 logic unused_rst_n;
134
135 assign unused_clk = clk_i;
136 assign unused_rst_n = rst_ni;
137 endmodule

```

3. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_alu_bignum.sv

(1,612 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6 /**
7  * OTBN alu block for the bignum instruction subset
8  *
9  * This ALU supports all of the 'plain' arithmetic and logic
  bignum instructions and some
10  * vectorized arithmetic instructions. BN.MULQACC and its vec-
  torized counterparts are
11  * implemented in a separate block.
12  *
13  * One barrel shifter and two adders (X and Y) are implemented
  along with the logic operators
14  * (AND, OR, XOR, NOT), a vector transposer and 24-bit to 32-bit
  packing and unpacking logic.
15  *
16  * The adders have 256-bit operands with a carry_in and op-
  tional invert on the second operand. This
17  * can be used to implement subtraction (a - b == a + ~b + 1).
  BN.SUBB/BN.ADDC are implemented by
18  * feeding in the carry flag as carry in rather than a fixed 0 or
  1. These 256-bit operands can
19  * also be interpreted as vectors with 32-bit elements.
20  *
21  * The shifter takes a 512-bit input (to implement BN.RSHI,
  concatenate and right shift) and shifts
22  * right by up to 256-bits. The lower (256-bit) half of the
  input and output can be reversed to
23  * allow left shift implementation. There is no concatenate
  and left shift instruction so reversing
24  * isn't required over the full width.
25  *
26  * The dataflow between the adders and shifter is in the dia-
  gram below. This arrangement allows the
27  * implementation of the pseudo-mod (BN.ADDM(V)/BN.SUBM(V))
  instructions in a single cycle whilst
28  * minimising the critical path. The pseudo-mod instructions
  do not have a shifted input so X can
29  * compute the initial add/sub and Y computes the pseudo-mod
  result. The result is selected based
30  * on the carries of adder X and Y. This selection logic is
  implemented in the modulo result
31  * selector. For all other add/sub operations, Y computes the
  operation with one of the inputs
32  * supplied by the shifter and the other from operand_a. The
  result is then pushed through the
33  * modulo result selector. This allows to save one input to
  the final result selection mux.
34  *
35  * The shifter gets supplied with either operand_a and oper-
  and_b or a dense representation thereof
36  * to implement the packing functionality. In addition, the
  shifter gets a shift amount (shift_amt)
37  * and can use 0 instead of operand_a / its dense representa-
  tion. The shifter concatenates both
38  * inputs together before shifting with operand_a in the upper
  (256-bit) half (A/0, B). This
39  * allows the shifter to pass through operand_b simply by not
  performing a shift.
40  *
41  * The shifter result has no direct connection to the final
  result MUX but the result is pushed
42  * through the logic block with operand_a set to 0 and the lo-
  gic operation set to OR. This saves
43  * one input to the result selection mux.
44  *
45  * The vector packing and unpacking is implemented using the
  shifter's concatenation feature and
46  * some wiring.
47  *
48  * Blanking is employed on the ALU data paths. This holds un-
  used data paths to 0 to reduce side
49  * channel leakage. The lower-case 'b' on the diagram below
  indicates points in the data path that
50  * get blanked. Note that Adder X is never used in isolation,
  it is always combined with Adder Y so
51  * there is no need for blanking between Adder X and Adder Y.
52  *
53  * Due to this blanking only one of the possible results can
  be active at the same time. This
54  * allows to implement the final result selection with one

```

256-bit OR. This is more area as well

55 * as timing efficient compared to a regular 256-bit mux.

```

56 *
57 *
58 *
59 *
60 *
61 *
62 *
63 *
64 *
65 *
66 *
67 *
68 *
69 *
70 *
71 *
72 *
73 *
74 *
75 *
76 *
77 *
78 *
79 *
80 *
81 *
82 *
83 *
84 *
85 *
86 *
87 *
88 *
89 *
90 *
91 *
92 *
93 *
94 *
95 *
96 *
97 *
98 *
99 *
100 *
101 *
102 *
103 *
104 *
105 */
106 module otbn_alu_bignum
107   import otbn_pkg::*;
108 (
109   input logic clk_i,
110   input logic rst_ni,
111
112   input alu_bignum_operation_t operation_i,
113   input logic operation_valid_i,
114   input logic operation_commit_i, // used
  for SVAs only
115   output logic [WLEN-1:0] operation_result_o,
116   output logic selection_flag_o,
117
118   input alu_bignum_predec_t alu_bignum_predec_i,
119   input ispr_bignum_predec_t ispr_bignum_predec_i,
120
121   input ispr_e ispr_addr_i,
122   input logic [31:0] ispr_base_wdata_i,
123   input logic [BaseWordsPerWLEN-1:0] ispr_base_wr_en_i,
124   input logic [ExtWLEN-1:0] ispr_bignum_wdata_intg_i,
125   input logic ispr_bignum_wr_en_i,
126   input logic [NFlagGroups-1:0] ispr_flags_wr_i,
127   input logic ispr_wr_commit_i,
128   input logic ispr_init_i,
129   output logic [ExtWLEN-1:0] ispr_rdata_intg_o,
130   input logic ispr_rd_en_i,

```



```

131
132 input logic [ExtWLEN-1:0] ispr_acc_intg_i,
133 output logic [ExtWLEN-1:0] ispr_acc_wr_data_intg_o,
134 output logic ispr_acc_wr_en_o,
135
136 output logic [ExtWLEN-1:0] ispr_mod_intg_o,
137
138 output logic ispr_mai_ctrl_wr_o,
139 output logic [31:0] ispr_mai_ctrl_wdata_o,
140 input logic [31:0] ispr_mai_ctrl_rdata_i,
141 input logic [31:0] ispr_mai_status_rdata_i,
142 output logic ispr_mai_in0_s0_wr_o,
143 output logic [ExtWLEN-1:0] ispr_mai_in0_s0_wdata_o,
144 output logic ispr_mai_in0_s1_wr_o,
145 output logic [ExtWLEN-1:0] ispr_mai_in0_s1_wdata_o,
146 output logic ispr_mai_in1_s0_wr_o,
147 output logic [ExtWLEN-1:0] ispr_mai_in1_s0_wdata_o,
148 output logic ispr_mai_in1_s1_wr_o,
149 output logic [ExtWLEN-1:0] ispr_mai_in1_s1_wdata_o,
150 output logic ispr_mai_res_s0_wr_o,
151 output logic [ExtWLEN-1:0] ispr_mai_res_s0_wdata_o,
152 output logic ispr_mai_res_s1_wr_o,
153 output logic [ExtWLEN-1:0] ispr_mai_res_s1_wdata_o,
154 input logic [ExtWLEN-1:0] ispr_mai_in0_s0_rdata_i,
155 input logic [ExtWLEN-1:0] ispr_mai_in0_s1_rdata_i,
156 input logic [ExtWLEN-1:0] ispr_mai_in1_s0_rdata_i,
157 input logic [ExtWLEN-1:0] ispr_mai_in1_s1_rdata_i,
158 input logic [ExtWLEN-1:0] ispr_mai_res_s0_rdata_i,
159 input logic [ExtWLEN-1:0] ispr_mai_res_s1_rdata_i,
160
161 output logic ispr_kmac_status_wr_o,
162 output logic [31:0] ispr_k-,
163 output logic ispr_kmac_ctrl_wr_o,
164 output logic [31:0] ispr_kmac_ctrl_wdata_o,
165 output logic ispr_kmac_cfg_wr_o,
166 output logic [31:0] ispr_kmac_cfg_wdata_o,
167 output logic ispr_kmac_strb_wr_o,
168 output logic [31:0] ispr_kmac_strb_wdata_o,
169 output logic ispr_kmac_data_s0_wr_o,
170 output logic [ExtWLEN-1:0] ispr_k-,
171 output logic ispr_kmac_data_s1_wr_o,
172 output logic [ExtWLEN-1:0] ispr_k-,
173 input logic [31:0] ispr_k-,
174 input logic [31:0] ispr_kmac_cfg_rdata_i,
175 input logic [31:0] ispr_kmac_strb_rdata_i,
176 input logic [ExtWLEN-1:0] ispr_k-,
177 input logic [ExtWLEN-1:0] ispr_k-,
178 output logic ispr_kmac_data_s0_rd_o,
179 output logic ispr_kmac_data_s1_rd_o,
180
181 output logic ispr_urnd_ctrl_wr_o,
182 output logic [31:0] ispr_urnd_ctrl_wdata_o,
183 input logic [31:0] ispr_urnd_status_rdata_i,
184 output logic ispr_urnd_state_wr_o,
185 output logic [UrndPartialSeedwidth-1:0] ispr_urnd_state_wdata_o,
186 input logic [WLEN-1:0] ispr_urnd_state_rdata_i,
187
188 output logic reg_intg_violation_err_o,
189
190 input logic sec_wipe_mod_urnd_i,
191 input logic sec_wipe_running_i,
192 output logic sec_wipe_err_o,
193
194 input flags_t mac_operation_flags_i,
195 input flags_t mac_operat-,
196
197 input logic [WLEN-1:0] rnd_data_i,
198 input logic [WLEN-1:0] urnd_data_i,
199 input logic [31:0] insn_cnt_i,
200
201 input logic [1:0][SideloadKeyWidth-1:0] side-
load_key_shares_i,
202
203 output logic alu_predec_error_o,
204 output logic ispr_predec_error_o
205 );
206
207 logic [WLEN-1:0] adder_y_res;
208 logic [NVecProc-1:0] adder_y_carries_out;
209 logic [WLEN-1:0] logical_res;
210
211 //////////////////////////////////////////////////
212 // ISPRs //
213 //////////////////////////////////////////////////
214
215 flags_t flags_d [NFlagGroups];
216 flags_t flags_q [NFlagGroups];
217 logic [NFlagGroups*FlagsWidth-1:0] flags_flattened;

218 flags_t selected_flags;
219 flags_t adder_update_flags;
220 logic ad-
221
222 der_update_flags_en_raw;
223 flags_t logic_update_flags
224 [NFlagGroups];
225 logic lo-
226
227 gic_update_flags_en_raw;
228 flags_t mac_update_flags
229 [NFlagGroups];
230 logic [NFlagGroups-1:0] mac_up-
231
232 date_z_flag_en_blanked;
233 flags_t ispr_update_flags
234 [NFlagGroups];
235
236 logic [NISpr-1:0] expected_ispr_rd_en_onehot;
237 logic [NISpr-1:0] expected_ispr_wr_en_onehot;
238 logic ispr_wr_en;
239
240 logic [NFlagGroups-1:0] expected_flag_group_sel;
241 flags_t expected_flag_sel;
242 logic [NFlagGroups-1:0] expected_flags_keep;
243 logic [NFlagGroups-1:0] expected_flags_adder_update;
244 logic [NFlagGroups-1:0] expected_flags_logic_update;
245 logic [NFlagGroups-1:0] expected_flags_mac_update;
246 logic [NFlagGroups-1:0] expected_flags_ispr_wr;
247
248 //////////////////////////////////////////////////
249 // Flags Selection //
250 //////////////////////////////////////////////////
251
252 always_comb begin
253     expected_flag_group_sel = '0;
254     expected_flag_group_sel[operation_i.flag_group] = '1'b1;
255 end
256 assign expected_flag_sel.C = operation_i.sel_flag == FlagC;
257 assign expected_flag_sel.M = operation_i.sel_flag == FlagM;
258 assign expected_flag_sel.L = operation_i.sel_flag == FlagL;
259 assign expected_flag_sel.Z = operation_i.sel_flag == FlagZ;
260
261 // SEC_CM: DATA_REG_SW.SCA
262 prim_onehot_mux #(
263     .Width(FlagsWidth),
264     .Inputs(NFlagGroups)
265 ) u_flags_q_mux (
266     .clk_i,
267     .rst_ni,
268     .in_i (flags_q),
269     .sel_i(alu_bignum_predec_i.flag_group_sel),
270     .out_o(selected_flags)
271 );
272
273 `ASSERT(BlankingSelectedFlags_A, expected_flag_group_sel ==
'0 |> selected_flags == '0, clk_i,
274     !rst_ni || alu_predec_error_o || !operation_commit_i)
275
276 logic flag_mux_in [FlagsWidth];
277 logic [FlagsWidth-1:0] flag_mux_sel;
278 assign flag_mux_in = '{selected_flags.C,
279     selected_flags.M,
280     selected_flags.L,
281     selected_flags.Z};
282
283 assign flag_mux_sel = {alu_bignum_predec_i.flag_sel.Z,
284     alu_bignum_predec_i.flag_sel.L,
285     alu_bignum_predec_i.flag_sel.M,
286     alu_bignum_predec_i.flag_sel.C};
287
288 // SEC_CM: DATA_REG_SW.SCA
289 prim_onehot_mux #(
290     .Width(1),
291     .Inputs(FlagsWidth)
292 ) u_flag_mux (
293     .clk_i,
294     .rst_ni,
295     .in_i (flag_mux_in),
296     .sel_i(flag_mux_sel),
297     .out_o(selection_flag_o)
298 );
299
300 `ASSERT(BlankingSelectionFlag_A, expected_flag_sel == '0 |>
selection_flag_o == '0, clk_i,
301     !rst_ni || alu_predec_error_o || !operation_commit_i)
302
303 //////////////////////////////////////////////////
304 // Flags Update //
305 //////////////////////////////////////////////////
306
307 // Flags are only updated for regular 256b operations.
308 // Vectorized operations do not update the flags.
309 `ASSERT(VecAndModAndRshOpsNoFlagUpdate_A,
310     operation_i.op inside {AluOpBignumAddv, AluOpBignum-
311     Subv,
312     AluOpBignumAddm, AluOpBignu-
313     mAddvm,
314     AluOpBignumSubm, AluOpBignum-
315     Subvm,
316     AluOpBignumTrn1, AluOp-
317     BignumTrn2,
318     AluOpBignumShv,
319     AluOpBignumPack, AluOpBignu-

```

```

mUnpk,
306             AluOpBignumRshi}
307     |-> !adder_update_flags_en_raw && !lo-
gic_update_flags_en_raw,
308         clk_i, !rst_ni || !operation_commit_i)
309
310 // Note that the flag zeroing triggered by ispr_init_i and
secure wipe is achieved by not
311 // selecting any inputs in the one-hot muxes below. The in-
struction fetch/predecoder stage
312 // is driving the selector inputs accordingly.
313
314 always_comb begin
315     expected_flags_adder_update = '0;
316     expected_flags_logic_update = '0;
317     expected_flags_mac_update = '0;
318
319     expected_flags_adder_update[operation_i.flag_group] = op-
eration_i.alu_flag_en &
320         ad-
der_update_flags_en_raw;
321     expected_flags_logic_update[operation_i.flag_group] = op-
eration_i.alu_flag_en &
322         lo-
gic_update_flags_en_raw;
323     expected_flags_mac_update[operation_i.flag_group] = op-
eration_i.mac_flag_en;
324 end
325 assign expected_flags_ispr_wr = ispr_flags_wr_i;
326
327 assign expected_flags_keep = ~(expected_flags_adder_update |
328     expected_flags_logic_update |
329     expected_flags_mac_update |
330     expected_flags_ispr_wr);
331
332 // Adder operations update all flags.
333 assign adder_update_flags.C = (operation_i.op == AluOpBignu-
mAdd ||
334     operation_i.op == AluOpBignu-
mAddc) ?
335     ad-
der_y_carries_out[NVecProc-1] : ~adder_y_carries_out[NVecProc-1];
336
337 assign adder_update_flags.M = adder_y_res[WLEN-1];
338 assign adder_update_flags.L = adder_y_res[0];
339 assign adder_update_flags.Z = ~|adder_y_res;
340
341 for (genvar i_fg = 0; i_fg < NFlagGroups; i_fg++) begin :
g_update_flag_groups
342
343     // Logical operations only update M, L and Z; C must re-
main at its old value.
344     assign logic_update_flags[i_fg].C = flags_q[i_fg].C;
345     assign logic_update_flags[i_fg].M = logical_res[WLEN-1];
346     assign logic_update_flags[i_fg].L = logical_res[0];
347     assign logic_update_flags[i_fg].Z = ~|logical_res;
348
349     // MAC Flags //
350     // MAC operations don't update C.
351     assign mac_update_flags[i_fg].C = flags_q[i_fg].C;
352
353     // Tie off unused signals.
354     logic unused_mac_operation_flags;
355     assign unused_mac_operation_flags = macopera-
tion_flags_i.C ^ mac_operation_flags_en_i.C;
356
357     // MAC operations update M and L depending on the opera-
tion. The individual enable signals for
358     // M and L are generated from flopped instruction bits
with minimal logic. They are not data
359     // dependent.
360     assign mac_update_flags[i_fg].M = macopera-
tion_flags_en_i.M ?
361         macopera-
tion_flags_i.M : flags_q[i_fg].M;
362     assign mac_update_flags[i_fg].L = macopera-
tion_flags_en_i.L ?
363         macopera-
tion_flags_i.L : flags_q[i_fg].L;
364
365     // MAC operations update Z depending on the operation and
data. For BN.MULQACC.S0, already the
366     // enable signal is data dependent (it depends on the
lower half of the accumulator result). As
367     // a result the enable signal might change back and forth
during instruction execution which may
368     // lead to SCA leakage. There is nothing that can really
be done to avoid this other than
369
370 // pipelining the flag computation which has a performance
impact.
371 //
372
373 // By blanking the enable signal for the other flag group,
we can at least avoid leakage related
374
375 // to the other flag group, i.e., we give the programmer a
way to control where the leakage

```

```

376 // happens.
377 // SEC_CM: DATA_REG_SW.SCA
378 prim_blanker #(.Width(1)) u_mac_z_flag_en_blanker (
379     .in_i (mac_operation_flags_en_i.Z),
380     .en_i (alu_bignum_predec_i.flags_mac_update[i_fg]),
381     .out_o(mac_update_z_flag_en_blanked[i_fg])
382 );
383 assign mac_update_flags[i_fg].Z = mac_up-
date_z_flag_en_blanked[i_fg] ?
384     macopera-
tion_flags_i.Z : flags_q[i_fg].Z;
385
386 // For ISPR writes, we get the full write data from the
base ALU and will select the relevant
387 // parts using the blankers and one-hot muxes below.
388 assign ispr_update_flags[i_fg] =
ispr_base_wdata_i[i_fg*FlagsWidth+:FlagsWidth];
389 end
390
391 localparam int NFlagsSrcs = 5;
392 for (genvar i_fg = 0; i_fg < NFlagGroups; i_fg++) begin :
g_flag_groups
393
394     flags_t             flags_d_mux_in [NFlagsSrcs];
395     logic [NFlagsSrcs-1:0] flags_d_mux_sel;
396     assign flags_d_mux_in = '{ispr_update_flags[i_fg],
397         mac_update_flags[i_fg],
398         logic_update_flags[i_fg],
399         adder_update_flags,
400         flags_q[i_fg]};
401     assign flags_d_mux_sel =
{alu_bignum_predec_i.flags_keep[i_fg],
402         alu_bignum_predec_i.flags_adder_update[i_fg],
403         alu_bignum_predec_i.flags_logic_update[i_fg],
404         alu_bignum_predec_i.flags_mac_update[i_fg],
405         alu_bignum_predec_i.flags_ispr_wr[i_fg]};
406
407     // SEC_CM: DATA_REG_SW.SCA
408     prim_onehot_mux #(
409         .Width(FlagsWidth),
410         .Inputs(NFlagsSrcs)
411     ) u_flags_d_mux (
412         .clk_i,
413         .rst_ni,
414         .in_i (flags_d_mux_in),
415         .sel_i (flags_d_mux_sel),
416         .out_o(flags_d[i_fg])
417     );
418
419     always_ff @(posedge clk_i or negedge rst_ni) begin
420         if (!rst_ni) begin
421             flags_q[i_fg] <= '{Z : 1'b0, L : 1'b0, M : 1'b0, C :
1'b0};
422         end else begin
423             flags_q[i_fg] <= flags_d[i_fg];
424         end
425     end
426
427     assign flags_flattened[i_fg*FlagsWidth+:FlagsWidth] =
flags_q[i_fg];
428 end
429
430 // MOD //
431 // MOD //
432 // MOD //
433
434 logic [ExtWLEN-1:0] mod_intg_q;
435 logic [ExtWLEN-1:0] mod_intg_d;
436 logic [BaseWordsPerWLEN-1:0] mod_ispr_wr_en;
437 logic [BaseWordsPerWLEN-1:0] mod_wr_en;
438
439 logic [ExtWLEN-1:0] ispr_mod_bignum_wdata_intg_blanked;
440
441 // SEC_CM: DATA_REG_SW.SCA
442 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_mod_bignum_wdata_blanker (
443     .in_i (ispr_bignum_wdata_intg_i),
444     .en_i (ispr_bignum_predec_i.ispr_wr_en[IsprMod]),
445     .out_o(ispr_mod_bignum_wdata_intg_blanked)
446 );
447 // If the blanker is enabled, the output will not carry the
correct ECC bits. This is not
448 // a problem because a blanked value should never be written
to the register. If the blanked
449 // value is written to the register nonetheless, an integ-
rity error arises.
450
451 logic [WLEN-1:0] mod_no_intg_d;
452 logic [WLEN-1:0] mod_no_intg_q;
453 logic [WLEN-1:0] mod_no_intg_q_replicated;
454 logic [ExtWLEN-1:0] mod_intg_calc;
455 logic [2*BaseWordsPerWLEN-1:0] mod_intg_err;
456
457 for (genvar i_word = 0; i_word < BaseWordsPerWLEN; i_word++)
begin : g_mod_words
458     prim_secdec_inv_39_32_enc i_secdec_enc (

```

```

458     .data_i(mod_no_intg_d[i_word*32+:32]),
459     .data_o(mod_intg_calc[i_word*39+:39])
460 );
461 prim_secdec_inv_39_32_dec i_secdec_dec (
462     .data_i (mod_intg_q[i_word*39+:39]),
463     .data_o  (* unused because we abort on any integrity
error */),
464     .syndrome_o(* unused */),
465     .err_o      (mod_intg_err[i_word*2+:2])
466 );
467 assign mod_no_intg_q[i_word*32+:32] =
mod_intg_q[i_word*39+:32];
468
469 always_ff @(posedge clk_i) begin
470     if (mod_wr_en[i_word]) begin
471         mod_intg_q[i_word*39+:39] <= mod_intg_d[i_word*39+:
39];
472     end
473 end
474
475 always_comb begin
476     mod_no_intg_d[i_word*32+:32] = '0;
477     unique case (i'b1)
478         // Non-encoded inputs have to be encoded before writ-
ing to the register.
479         sec_wipe_mod_urnd_i: begin
480             // In a secure wipe, `urnd_data_i` is written to the
register before the zero word. The
481             // ECC bits should not matter between the two
writes, but nonetheless we encode
482             // `urnd_data_i` so there is no spurious integrity
error.
483             mod_no_intg_d[i_word*32+:32] =
urnd_data_i[i_word*32+:32];
484             mod_intg_d[i_word*39+:39] =
mod_intg_calc[i_word*39+:39];
485         end
486         // Pre-encoded inputs can directly be written to the
register.
487         default: mod_intg_d[i_word*39+:39] =
ispr_mod_bignum_wdata_intg_blanked[i_word*39+:39];
488     endcase
489
490     unique case (i'b1)
491         ispr_init_i: mod_intg_d[i_word*39+:39] = EccZeroWord;
492         ispr_base_wr_en_i[i_word]: begin
493             mod_no_intg_d[i_word*32+:32] = ispr_base_wdata_i;
494             mod_intg_d[i_word*39+:39] =
mod_intg_calc[i_word*39+:39];
495         end
496         default: ;
497     endcase
498 end
499
500 `ASSERT(MODWrSelOneHot, $onehot0({ispr_init_i,
ispr_base_wr_en_i[i_word]}))
501
502 assign mod_ispr_wr_en[i_word] = (ispr_addr_i == Is-
prMod)
&
503 (ispr_base_wr_en_i[i_word] |
ispr_bignum_wr_en_i) &
504 ispr_wr_commit_i;
505
506 assign mod_wr_en[i_word] = ispr_init_i |
507 mod_ispr_wr_en[i_word] |
508 sec_wipe_mod_urnd_i;
509 end
510
511 // Replicate the modulus value to match the ELEN
512 always_comb begin
513     unique case (alu_bignum_predec_i.alu_elen)
514         AluElen32: mod_no_intg_q_replicated = {(WLEN/32)
{mod_no_intg_q[31:0]}};
515         AluElen256: mod_no_intg_q_replicated = mod_no_intg_q;
516         default: mod_no_intg_q_replicated = mod_no_intg_q;
517     endcase
518 end
519
520 // Output current MOD value towards BN MAC
521 assign ispr_mod_intg_o = mod_intg_q;
522
523 // ACC
524 // ACC
525 // ACC
526
527 assign ispr_acc_wr_en_o =
528 ((ispr_addr_i == IsprAcc) & ispr_bignum_wr_en_i &
ispr_wr_commit_i) | ispr_init_i;
529
530
531 logic [ExtWLEN-1:0] ispr_acc_bignum_wdata_intg_blanked;
532
533 // SEC_CM: DATA_REG_SW.SCA
534 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_acc_bignum_wdata_intg_blanker (
535     .in_i (ispr_bignum_wdata_intg_i),
536     .en_i (ispr_bignum_predec_i.ispr_wr_en[IsprAcc]),
537     .out_o(ispr_acc_bignum_wdata_intg_blanked)
538 );

```

```

539 // If the blanker is enabled, the output will not carry the
correct ECC bits. This is not
540 // a problem because a blanked value should never be used.
If the blanked value is used
541 // nonetheless, an integrity error arises.
542
543 assign ispr_acc_wr_data_intg_o = ispr_init_i ? EccWideZero-
Word
544 :
ispr_acc_bignum_wdata_intg_blanked;
545
546 // MAI Write
547 // MAI Write
548 // MAI Write
549
550 assign ispr_mai_in0_s0_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprMaiIn0S0];
551 // SEC_CM: DATA_REG_SW.SCA
552 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_mai_in0_s0_wdata_blanker (
553     .in_i (ispr_bignum_wdata_intg_i),
554     .en_i (ispr_mai_in0_s0_wr_o),
555     .out_o(ispr_mai_in0_s0_wdata_o)
556 );
557
558 assign ispr_mai_in0_s1_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprMaiIn0S1];
559 // SEC_CM: DATA_REG_SW.SCA
560 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_mai_in0_s1_wdata_blanker (
561     .in_i (ispr_bignum_wdata_intg_i),
562     .en_i (ispr_mai_in0_s1_wr_o),
563     .out_o(ispr_mai_in0_s1_wdata_o)
564 );
565
566 assign ispr_mai_in1_s0_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprMaiIn1S0];
567 // SEC_CM: DATA_REG_SW.SCA
568 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_mai_in1_s0_wdata_blanker (
569     .in_i (ispr_bignum_wdata_intg_i),
570     .en_i (ispr_mai_in1_s0_wr_o),
571     .out_o(ispr_mai_in1_s0_wdata_o)
572 );
573
574 assign ispr_mai_in1_s1_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprMaiIn1S1];
575 // SEC_CM: DATA_REG_SW.SCA
576 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_mai_in1_s1_wdata_blanker (
577     .in_i (ispr_bignum_wdata_intg_i),
578     .en_i (ispr_mai_in1_s1_wr_o),
579     .out_o(ispr_mai_in1_s1_wdata_o)
580 );
581
582 assign ispr_mai_res_s0_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprMaiResS0];
583 // SEC_CM: DATA_REG_SW.SCA
584 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_mai_res_s0_wdata_blanker (
585     .in_i (ispr_bignum_wdata_intg_i),
586     .en_i (ispr_mai_res_s0_wr_o),
587     .out_o(ispr_mai_res_s0_wdata_o)
588 );
589
590 assign ispr_mai_res_s1_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprMaiResS1];
591 // SEC_CM: DATA_REG_SW.SCA
592 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_mai_res_s1_wdata_blanker (
593     .in_i (ispr_bignum_wdata_intg_i),
594     .en_i (ispr_mai_res_s1_wr_o),
595     .out_o(ispr_mai_res_s1_wdata_o)
596 );
597
598 assign ispr_mai_ctrl_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprMaiCtrl];
599 // SEC_CM: DATA_REG_SW.SCA
600 prim_blanker #(.Width(32'd32)) u_ispr_mai_ctrl_wdata_blanker (
601     .in_i (ispr_base_wdata_i),
602     .en_i (ispr_mai_ctrl_wr_o),
603     .out_o(ispr_mai_ctrl_wdata_o)
604 );
605
606 // KMAC Write
607 // KMAC Write
608 // KMAC Write
609
610 assign ispr_kmac_status_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprKmacStatus];
611 // SEC_CM: DATA_REG_SW.SCA
612 prim_blanker #(.Width(32'd32)) u_is-
pr_kmac_status_wdata_blanker (
613     .in_i (ispr_base_wdata_i),
614     .en_i (ispr_kmac_status_wr_o),
615     .out_o(ispr_kmac_status_wdata_o)
616 );
617
618 assign ispr_kmac_ctrl_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprKmacCtrl];

```

```

619 // SEC_CM: DATA_REG_SW.SCA
620 prim_blanker #(.Width(32'd32)) u_is-
pr_kmac_ctrl_wdata_blanker (
621   .in_i (ispr_base_wdata_i),
622   .en_i (ispr_kmac_ctrl_wr_o),
623   .out_o(ispr_kmac_ctrl_wdata_o)
624 );
625
626 assign ispr_kmac_cfg_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprKmacCfg];
627 // SEC_CM: DATA_REG_SW.SCA

628 prim_blanker #(.Width(32'd32)) u_ispr_kmac_cfg_wdata_blanker (
629   .in_i (ispr_base_wdata_i),
630   .en_i (ispr_kmac_cfg_wr_o),
631   .out_o(ispr_kmac_cfg_wdata_o)
632 );
633
634 assign ispr_kmac_strb_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprKmacStrb];
635 // SEC_CM: DATA_REG_SW.SCA
636 prim_blanker #(.Width(32'd32)) u_is-
pr_kmac_strb_wdata_blanker (
637   .in_i (ispr_base_wdata_i),
638   .en_i (ispr_kmac_strb_wr_o),
639   .out_o(ispr_kmac_strb_wdata_o)
640 );
641
642 assign ispr_kmac_data_s0_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprKmacDataS0];
643 // SEC_CM: DATA_REG_SW.SCA
644 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_kmac_data_s0_wdata_blanker (
645   .in_i (ispr_bignum_wdata_intg_i),
646   .en_i (ispr_kmac_data_s0_wr_o),
647   .out_o(ispr_kmac_data_s0_wdata_o)
648 );
649
650 assign ispr_kmac_data_s1_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprKmacDataS1];
651 // SEC_CM: DATA_REG_SW.SCA
652 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_kmac_data_s1_wdata_blanker (
653   .in_i (ispr_bignum_wdata_intg_i),
654   .en_i (ispr_kmac_data_s1_wr_o),
655   .out_o(ispr_kmac_data_s1_wdata_o)
656 );
657
658 assign ispr_kmac_data_s0_rd_o =
ispr_bignum_predec_i.ispr_rd_en[IsprKmacDataS0];
659 assign ispr_kmac_data_s1_rd_o =
ispr_bignum_predec_i.ispr_rd_en[IsprKmacDataS1];
660
661 //////////////////////////////////////////////////
662 // URND Write //
663 //////////////////////////////////////////////////
664 // No blanker is required in front of a register holding
known control values when writing values
665 // from base registers.
666 assign ispr_urnd_ctrl_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprUrndCtrl];
667 assign ispr_urnd_ctrl_wdata_o = ispr_base_wdata_i;
668
669 // We drop the integrity towards the rnd module. SW can
verify the update by reading back the
670 // state.
671 logic [WLEN-1:0] ispr_urnd_state_wdata_no_intg;
672 for (genvar i = 0; i < BaseWordsPerWLEN; i++) begin :
g_urnd_state_drop_intg
673   assign ispr_urnd_state_wdata_no_intg[i * 32 +: 32] =
674     ispr_bignum_wdata_intg_i[i * BaseIntgWidth +: 32];
675 end
676
677 assign ispr_urnd_state_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprUrndState];
678 // SEC_CM: DATA_REG_SW.SCA
679 prim_blanker #(.Width(UrndPartialSeedWidth)) u_is-
pr_urnd_state_wdata_blanker (
680   .in_i
(ispr_urnd_state_wdata_no_intg[UrndPartialSeedWidth-1:0]),
681   .en_i (ispr_urnd_state_wr_o),
682   .out_o(ispr_urnd_state_wdata_o)
683 );
684
685 logic unused_urnd_state_wdata;
686 assign unused_urnd_state_wdata =
^ispr_urnd_state_wdata_no_intg[WLEN-1:UrndPartialSeedWidth];
687
688 //////////////////////////////////////////////////
689 // ISPR Read //
690 //////////////////////////////////////////////////
691
692 // ISPR read data is muxed out in two stages:
693 // 1. Select amongst the ISPRs that have no integrity bits.
The output has integrity calculated
694 // for it.
695 // 2. Select between the ISPRs that have integrity bits and
the result of the first stage.
696
697 // Number of ISPRs that have no integrity protection
698 localparam int NNoIntgIspr = 15;

```

```

699 // IDs for ISPRs without integrity
700 localparam int IsprRndNoIntg = 0;
701 localparam int IsprUrndNoIntg = 1;
702 localparam int IsprMaiCtrlNoIntg = 2;
703 localparam int IsprMaiStatusNoIntg = 3;
704 localparam int IsprFlagsNoIntg = 4;
705 localparam int IsprKeyS0LNoIntg = 5;
706 localparam int IsprKeyS0HNoIntg = 6;
707 localparam int IsprKeyS1LNoIntg = 7;
708 localparam int IsprKeyS1HNoIntg = 8;
709 localparam int IsprKmacStatusNoIntg = 9;
710 localparam int IsprKmacCfgNoIntg = 10;
711 localparam int IsprKmacStrbNoIntg = 11;
712 localparam int IsprInsnCntNoIntg = 12;
713 localparam int IsprUrndStateNoIntg = 13;
714 localparam int IsprUrndStatusNoIntg = 14;

715 // KMAC_CTRL and URND_CTRL always read as '0. Thus we do not
need a mux input.
716
717 logic [NNoIntgIspr-1:0] ispr_rdata_no_intg_mux_sel;
718
719 // Number of ISPRs that have integrity protection
720 localparam int NIntgIspr = 10;
721 // IDs for ISPRs with integrity
722 localparam int IsprModIntg = 0;
723 localparam int IsprAccIntg = 1;
724 localparam int IsprMaiResS0Intg = 2;
725 localparam int IsprMaiResS1Intg = 3;
726 localparam int IsprMaiIn0S0Intg = 4;
727 localparam int IsprMaiIn0S1Intg = 5;
728 localparam int IsprMaiIn1S0Intg = 6;
729 localparam int IsprMaiIn1S1Intg = 7;
730 localparam int IsprKmacDataS0Intg = 8;
731 localparam int IsprKmacDataS1Intg = 9;
732 // ID representing all ISPRs with no integrity
733 localparam int IsprNoIntg = 10;
734
735 logic [NIntgIspr:0] ispr_rdata_intg_mux_sel;
736 logic [ExtWLEN-1:0] ispr_rdata_intg_mux_in [NIntgIspr+1];
737 logic [WLEN-1:0] ispr_rdata_no_intg_mux_in [NNoIntgIspr];
738
739 // First stage
740 assign ispr_rdata_no_intg_mux_in[IsprRndNoIntg] =
rnd_data_i;
741 assign ispr_rdata_no_intg_mux_in[IsprUrndNoIntg] =
urnd_data_i;
742 assign ispr_rdata_no_intg_mux_in[IsprUrndStateNoIntg] =
ispr_urnd_state_rdata_i;
743 assign ispr_rdata_no_intg_mux_in[IsprUrndStatusNoIntg] =
744   {{{(WLEN - 32){1'b0}}, ispr_urnd_status_rdata_i};
745 assign ispr_rdata_no_intg_mux_in[IsprMaiCtrlNoIntg] =
746   {{{(WLEN - 32){1'b0}}, ispr_mai_ctrl_rdata_i};
747 assign ispr_rdata_no_intg_mux_in[IsprMaiStatusNoIntg] =
748   {{{(WLEN - 32){1'b0}}, ispr_mai_status_rdata_i};
749 assign ispr_rdata_no_intg_mux_in[IsprFlagsNoIntg] =
750   {{{(WLEN - (NFlagGroups * FlagsWidth)){1'b0}},
flags_flattened}};
751
752 // SEC_CM: KEY.SIDELOAD
753 assign ispr_rdata_no_intg_mux_in[IsprKeyS0LNoIntg] = side-
load_key_shares_i[0][255:0];
754 assign ispr_rdata_no_intg_mux_in[IsprKeyS0HNoIntg] =
755   {{{(WLEN - (SideLoadKeyWidth - 256)){1'b0}}, side-
load_key_shares_i[0][SideLoadKeyWidth-1:256]}};
756 assign ispr_rdata_no_intg_mux_in[IsprKeyS1LNoIntg] = side-
load_key_shares_i[1][255:0];
757 assign ispr_rdata_no_intg_mux_in[IsprKeyS1HNoIntg] =
758   {{{(WLEN - (SideLoadKeyWidth - 256)){1'b0}}, side-
load_key_shares_i[1][SideLoadKeyWidth-1:256]}};
759 assign ispr_rdata_no_intg_mux_in[IsprInsnCntNoIntg] =
760   {{{(WLEN - 32){1'b0}}, insn_cnt_i}};
761
762 assign ispr_rdata_no_intg_mux_in[IsprKmacStatusNoIntg] =
763   {{{(WLEN - 32){1'b0}}, ispr_kmac_status_rdata_i};
764 assign ispr_rdata_no_intg_mux_in[IsprKmacCfgNoIntg] =
765   {{{(WLEN - 32){1'b0}}, ispr_kmac_cfg_rdata_i};
766 assign ispr_rdata_no_intg_mux_in[IsprKmacStrbNoIntg] =
767   {{{(WLEN - 32){1'b0}}, ispr_kmac_strb_rdata_i};
768
769 assign ispr_rdata_no_intg_mux_sel[IsprRndNoIntg] =
770   ispr_bignum_predec_i.ispr_rd_en[IsprRnd];
771 assign ispr_rdata_no_intg_mux_sel[IsprUrndNoIntg] =
772   ispr_bignum_predec_i.ispr_rd_en[IsprUrnd];
773 assign ispr_rdata_no_intg_mux_sel[IsprUrndStateNoIntg] =
774   ispr_bignum_predec_i.ispr_rd_en[IsprUrndState];
775 assign ispr_rdata_no_intg_mux_sel[IsprUrndStatusNoIntg] =
776   ispr_bignum_predec_i.ispr_rd_en[IsprUrndStatus];
777 assign ispr_rdata_no_intg_mux_sel[IsprMaiCtrlNoIntg] =
778   ispr_bignum_predec_i.ispr_rd_en[IsprMaiCtrl];
779 assign ispr_rdata_no_intg_mux_sel[IsprMaiStatusNoIntg] =
780   ispr_bignum_predec_i.ispr_rd_en[IsprMaiStatus];
781 assign ispr_rdata_no_intg_mux_sel[IsprFlagsNoIntg] =
782   ispr_bignum_predec_i.ispr_rd_en[IsprFlags];
783 assign ispr_rdata_no_intg_mux_sel[IsprKeyS0LNoIntg] =
784   ispr_bignum_predec_i.ispr_rd_en[IsprKeyS0L];
785 assign ispr_rdata_no_intg_mux_sel[IsprKeyS0HNoIntg] =
786   ispr_bignum_predec_i.ispr_rd_en[IsprKeyS0H];
787 assign ispr_rdata_no_intg_mux_sel[IsprKeyS1LNoIntg] =
788   ispr_bignum_predec_i.ispr_rd_en[IsprKeyS1L];
789 assign ispr_rdata_no_intg_mux_sel[IsprKeyS1HNoIntg] =

```



```

789   ispr_bignum_predec_i.ispr_rd_en[IsprKeyS1H];
790   assign ispr_rdata_no_intg_mux_sel[IsprKmacStatusNoIntg] =
791     ispr_bignum_predec_i.ispr_rd_en[IsprKmacStatus];
792   assign ispr_rdata_no_intg_mux_sel[IsprKmacCfgNoIntg] =
793     ispr_bignum_predec_i.ispr_rd_en[IsprKmacCfg];
794   assign ispr_rdata_no_intg_mux_sel[IsprKmacStrbNoIntg] =
795     ispr_bignum_predec_i.ispr_rd_en[IsprKmacStrb];
796   // KMAC_CTRL and URND_CTRL always read as '0'. We use the
onehot MUX to output '0' by not having an
797   // input for these. We thus also don't need to factor in
their read enable signals into the MUX
798   // select signal.
799
800   assign ispr_rdata_no_intg_mux_sel[IsprInsnCntNoIntg] =
801     ispr_bignum_predec_i.ispr_rd_en[IsprInsnCnt];
802
803   logic [WLEN-1:0] ispr_rdata_no_intg;
804   logic [ExtWLEN-1:0] ispr_rdata_intg_calc;
805
806   // SEC_CM: DATA_REG_SW.SCA
807   prim_onehot_mux #(
808     .Width (WLEN),
809     .Inputs (NNoIntgIspr)
810   ) u_ispr_rdata_no_intg_mux (
811     .clk_i,
812     .rst_ni,
813     .in_i (ispr_rdata_no_intg_mux_in),
814     .sel_i (ispr_rdata_no_intg_mux_sel),
815     .out_o (ispr_rdata_no_intg)
816   );
817
818   for (genvar i_word = 0; i_word < BaseWordsPerWLEN; i_word++)
begin : g_rdata_enc
819     prim_secdec_inv_39_32_enc i_secdec_enc (
820       .data_i (ispr_rdata_no_intg[i_word * 32 +: 32]),
821       .data_o (ispr_rdata_intg_calc[i_word * 39 +: 39])
822     );
823   end
824
825   // Second stage
826   assign ispr_rdata_intg_mux_in[IsprModIntg] =
mod_intg_q;
827   assign ispr_rdata_intg_mux_in[IsprAccIntg] =
ispr_acc_intg_i;
828   assign ispr_rdata_intg_mux_in[IsprMaiResS0Intg] =
ispr_mai_res_s0_rdata_i;
829   assign ispr_rdata_intg_mux_in[IsprMaiResS1Intg] =
ispr_mai_res_s1_rdata_i;
830   assign ispr_rdata_intg_mux_in[IsprMaiInS0Intg] =
ispr_mai_in0_s0_rdata_i;
831   assign ispr_rdata_intg_mux_in[IsprMaiInS1Intg] =
ispr_mai_in0_s1_rdata_i;
832   assign ispr_rdata_intg_mux_in[IsprMaiIn1S0Intg] =
ispr_mai_in1_s0_rdata_i;
833   assign ispr_rdata_intg_mux_in[IsprMaiIn1S1Intg] =
ispr_mai_in1_s1_rdata_i;
834   assign ispr_rdata_intg_mux_in[IsprKmacDataS0Intg] = ispr_k-
mac_data_s0_rdata_i;
835   assign ispr_rdata_intg_mux_in[IsprKmacDataS1Intg] = ispr_k-
mac_data_s1_rdata_i;
836   assign ispr_rdata_intg_mux_in[IsprNoIntg] =
ispr_rdata_intg_calc;
837
838   assign ispr_rdata_intg_mux_sel[IsprModIntg] =
ispr_bignum_predec_i.ispr_rd_en[IsprMod];
839   assign ispr_rdata_intg_mux_sel[IsprAccIntg] =
ispr_bignum_predec_i.ispr_rd_en[IsprAcc];
840   assign ispr_rdata_intg_mux_sel[IsprMaiResS0Intg] =
ispr_bignum_predec_i.ispr_rd_en[IsprMaiResS0];
841   assign ispr_rdata_intg_mux_sel[IsprMaiResS1Intg] =
ispr_bignum_predec_i.ispr_rd_en[IsprMaiResS1];
842   assign ispr_rdata_intg_mux_sel[IsprMaiInS0Intg] =
ispr_bignum_predec_i.ispr_rd_en[IsprMaiInS0];
843   assign ispr_rdata_intg_mux_sel[IsprMaiInS1Intg] =
ispr_bignum_predec_i.ispr_rd_en[IsprMaiInS1];
844   assign ispr_rdata_intg_mux_sel[IsprKmacDataS0Intg] =
ispr_bignum_predec_i.ispr_rd_en[IsprKmacDataS0];
845   assign ispr_rdata_intg_mux_sel[IsprKmacDataS1Intg] =
ispr_bignum_predec_i.ispr_rd_en[IsprKmacDataS1];
846   assign ispr_rdata_intg_mux_sel[IsprNoIntg] =
|{ispr_bignum_predec_i.ispr_rd_en[IsprInsnCnt],
ispr_bignum_predec_i.ispr_rd_en[IsprUrndState],
ispr_bignum_predec_i.ispr_rd_en[IsprUrndStatus],
ispr_bignum_predec_i.ispr_rd_en[IsprUrndCtrl],
ispr_bignum_predec_i.ispr_rd_en[IsprKmacStrb],
ispr_bignum_predec_i.ispr_rd_en[IsprKmacCfg],
ispr_bignum_predec_i.ispr_rd_en[IsprKmacCtrl],
ispr_bignum_predec_i.ispr_rd_en[IsprKmacStatus],
ispr_bignum_predec_i.ispr_rd_en[IsprMaiCtrl],
ispr_bignum_predec_i.ispr_rd_en[IsprMaiStatus],
ispr_bignum_predec_i.ispr_rd_en[IsprKeyS1H:IsprKeyS0L],
ispr_bignum_predec_i.ispr_rd_en[IsprUrnd],
ispr_bignum_predec_i.ispr_rd_en[IsprFlags],
ispr_bignum_predec_i.ispr_rd_en[IsprRnd]};
857
858   assign ispr_rdata_intg_mux_sel[IsprNoIntg] =
|{ispr_bignum_predec_i.ispr_rd_en[IsprInsnCnt],
ispr_bignum_predec_i.ispr_rd_en[IsprUrndState],
ispr_bignum_predec_i.ispr_rd_en[IsprUrndStatus],
ispr_bignum_predec_i.ispr_rd_en[IsprUrndCtrl],
ispr_bignum_predec_i.ispr_rd_en[IsprKmacStrb],
ispr_bignum_predec_i.ispr_rd_en[IsprKmacCfg],
ispr_bignum_predec_i.ispr_rd_en[IsprKmacCtrl],
ispr_bignum_predec_i.ispr_rd_en[IsprKmacStatus],
ispr_bignum_predec_i.ispr_rd_en[IsprMaiCtrl],
ispr_bignum_predec_i.ispr_rd_en[IsprMaiStatus],
ispr_bignum_predec_i.ispr_rd_en[IsprKeyS1H:IsprKeyS0L],
ispr_bignum_predec_i.ispr_rd_en[IsprUrnd],
ispr_bignum_predec_i.ispr_rd_en[IsprFlags],
ispr_bignum_predec_i.ispr_rd_en[IsprRnd]};
874

```

```

875   // If we're reading from an ISPR we must be using the
ispr_rdata_intg_mux
876   `ASSERT(IsprRdataIntgMuxSelIfIsprRd_A,
877     |ispr_bignum_predec_i.ispr_rd_en |-> |
ispr_rdata_intg_mux_sel)
878
879   // If we're reading from MOD or ACC we must not take the
read data from the calculated integrity
880   // path
881   `ASSERT(IsprModMustTakeIntg_A,
882     ispr_bignum_predec_i.ispr_rd_en[IsprMod] |-> !
ispr_rdata_intg_mux_sel[IsprNoIntg])
883
884   `ASSERT(IsprAccMustTakeIntg_A,
885     ispr_bignum_predec_i.ispr_rd_en[IsprAcc] |-> !
ispr_rdata_intg_mux_sel[IsprNoIntg])
886
887   prim_onehot_mux #(
888     .Width (ExtWLEN),
889     .Inputs (NIntgIspr+1)
890   ) u_ispr_rdata_intg_mux (
891     .clk_i,
892     .rst_ni,
893     .in_i (ispr_rdata_intg_mux_in),
894     .sel_i (ispr_rdata_intg_mux_sel),
895     .out_o (ispr_rdata_intg_o)
896   );
897
898   // ISPR PreDec Checks //
899   // ISPR PreDec Checks //
900   // ISPR PreDec Checks //
901   // ISPR PreDec Checks //
902   prim_onehot_enc #(
903     .OneHotWidth (NIspr)
904   ) u_expected_ispr_rd_en_enc (
905     .in_i (ispr_addr_i),
906     .en_i (ispr_rd_en_i),
907     .out_o (expected_ispr_rd_en_onehot)
908   );
909
910   assign ispr_wr_en = |{ispr_bignum_wr_en_i,
ispr_base_wr_en_i};
911
912   prim_onehot_enc #(
913     .OneHotWidth (NIspr)
914   ) u_expected_ispr_wr_en_enc (
915     .in_i (ispr_addr_i),
916     .en_i (ispr_wr_en),
917     .out_o (expected_ispr_wr_en_onehot)
918   );
919
920   // SEC_CM: CTRL.REDUN
921   assign ispr_predec_error_o =
|{expected_ispr_rd_en_onehot !=
ispr_bignum_predec_i.ispr_rd_en,
expected_ispr_wr_en_onehot !=
ispr_bignum_predec_i.ispr_wr_en};
924
925   // Shifter & Packing //
926   // Shifter & Packing //
927   // Shifter & Packing //
928
929   logic [8*24-1:0] operand_a_dense;
930   logic [8*24-1:0] operand_b_dense;
931   logic [WLEN-1:0] operand_a_dense_extended;
932   logic [WLEN-1:0] operand_b_dense_extended;
933   logic [WLEN-1:0] shifter_operands_upper [2];
934   logic [WLEN-1:0] shifter_operands_lower [2];
935   logic [WLEN-1:0] shifter_in_upper;
936   logic [WLEN-1:0] shifter_in_lower;
937   logic [WLEN-1:0] shifter_res;
938
939   // Pack both operands into a 8 * 24-bit dense format by
dropping the upper 8 bits per element.
940   for (genvar i = 0; i < 8; i++) begin : gen_packing
941     assign operand_a_dense[i * 24+:24] = opera-
tion_i.operand_a[i * 32+:24];
942     assign operand_b_dense[i * 24+:24] = opera-
tion_i.operand_b[i * 32+:24];
943   end
944
945   // Extend to 256 bits. The lower dense vector is placed at
the upper part such that the
946   // shifting can operate on the concatenated dense vectors.
947   assign operand_a_dense_extended = {64'd0, operand_a_dense};
948   assign operand_b_dense_extended = {operand_b_dense, 64'd0};
949
950   // Using onehot MUXs to select between regular inputs and
dense representations reduces the
951   // overall glitching activity compared to blanking the oper-
ands and using predecoded MUXs.
952   // Note that the packer may not contain any logic!
953
954   assign shifter_operands_upper[AluShiftOpFull] = opera-
tion_i.operand_a;
955   assign shifter_operands_upper[AluShiftOpDense] = oper-
and_a_dense_extended;
956   assign shifter_operands_lower[AluShiftOpFull] = opera-
tion_i.operand_b;
957   assign shifter_operands_lower[AluShiftOpDense] = oper-
and_b_dense_extended;

```

```

958
959 // SEC_CM: DATA_REG_SW.SCA
960 prim_onehot_mux #(
961     .Width (WLEN),
962     .Inputs(2)
963 ) u_shifter_operand_upper_mux (
964     .clk_i,
965     .rst_ni,
966     .in_i (shifter_operands_upper),
967     .sel_i(alu_bignum_predec_i.shift_op_a_sel),
968     .out_o(shifter_in_upper)
969 );
970
971 // SEC_CM: DATA_REG_SW.SCA
972 prim_onehot_mux #(
973     .Width (WLEN),
974     .Inputs(2)
975 ) u_shifter_operand_lower_mux (
976     .clk_i,
977     .rst_ni,
978     .in_i (shifter_operands_lower),
979     .sel_i(alu_bignum_predec_i.shift_op_b_sel),
980     .out_o(shifter_in_lower)
981 );
982
983 otbn_vec_shifter u_vec_shifter (
984     .clk_i,
985     .rst_ni,
986     .shifter_in_upper_i(shifter_in_upper),
987     .shifter_in_lower_i(shifter_in_lower),
988     .shift_direction_i (alu_bignum_predec_i.shift_dir),
989     .shift_amt_i (alu_bignum_predec_i.shift_amt),
990     .vector_mask_i ({8{alu_bignum_predec_i.shift_mask}}),
991     .shifter_res_o (shifter_res)
992 );
993
994 // Unpack the shifted result. Zero extend the 24-bit elements
995 // from the lower 8*24 bits
996 // to 32-bit elements.
997 logic [8*24-1:0] unpack_input_blanked;
998 logic [WLEN-1:0] unpacked_res;
999
1000 // SEC_CM: DATA_REG_SW.SCA
1001 prim_blanker #(.Width(8*24)) u_unpack_input_blanker (
1002     .in_i (shifter_res[8*24-1:0]),
1003     .en_i (alu_bignum_predec_i.unpack_shifter_en),
1004     .out_o(unpack_input_blanked)
1005 );
1006
1007 for (genvar i = 0; i < 8; i++) begin : gen_unpacking
1008     assign unpacked_res[i*32+:32] = {8'b0, unpack_input_blanked[i*24+:24]};
1009 end
1010
1011 // Adders X & Y
1012
1013
1014 logic [WLEN-1:0] adder_x_op_a_blanked, adder_x_op_b_blanked;
1015 logic [WLEN-1:0] adder_x_res;
1016 logic [NVecProc-1:0] adder_x_carries_out;
1017 logic [WLEN-1:0] adder_y_op_a_blanked;
1018 logic [WLEN-1:0] adder_y_op_shifter_res_blanked;
1019 logic [WLEN-1:0] shift_mod_mux_out;
1020 logic [WLEN-1:0] x_res_operand_a_mux_out;
1021 logic adder_y_carry_0_in;
1022
1023 // SEC_CM: DATA_REG_SW.SCA
1024 prim_blanker #(.Width(WLEN)) u_adder_x_op_a_blanked (
1025     .in_i (operation_i.operand_a),
1026     .en_i (alu_bignum_predec_i.adder_x_en),
1027     .out_o(adder_x_op_a_blanked)
1028 );
1029
1030 // SEC_CM: DATA_REG_SW.SCA
1031 prim_blanker #(.Width(WLEN)) u_adder_x_op_b_blanked (
1032     .in_i (operation_i.operand_b),
1033     .en_i (alu_bignum_predec_i.adder_x_en),
1034     .out_o(adder_x_op_b_blanked)
1035 );
1036
1037 otbn_vec_adder #(
1038     .LVLEN(VLEN),
1039     .LVChunkLEN(VChunkLEN)
1040 ) u_vec_adder_x (
1041     .operand_a_i (adder_x_op_a_blanked),
1042     .operand_b_i (adder_x_op_b_blanked),
1043     .oper-
1044 and_b_invert_i(alu_bignum_predec_i.adder_x_op_b_invert),
1045     .carries_in_i
1046 (alu_bignum_predec_i.adder_x_carries_in),
1047     .use_ext_carry_i ({NVec-
1048 Proc{alu_bignum_predec_i.adder_carry_sel}}),
1049     .sum_o (adder_x_res),
1050     .carries_out_o (adder_x_carries_out)
1051 );
1052
1053 // SEC_CM: DATA_REG_SW.SCA
1054 prim_blanker #(.Width(WLEN)) u_adder_y_op_a_blanked (
1055     .in_i (operation_i.operand_a),

```

```

1053     .en_i (alu_bignum_predec_i.adder_y_op_a_en),
1054     .out_o(adder_y_op_a_blanked)
1055 );
1056
1057 assign x_res_operand_a_mux_out =
1058     alu_bignum_predec_i.x_res_operand_a_sel ? adder_x_res :
1059     adder_y_op_a_blanked;
1060
1061 // SEC_CM: DATA_REG_SW.SCA
1062 prim_blanker #(.Width(WLEN)) u_adder_y_op_shifter_blanked (
1063     .in_i (shifter_res),
1064     .en_i (alu_bignum_predec_i.adder_y_op_shifter_en),
1065     .out_o(adder_y_op_shifter_res_blanked)
1066 );
1067
1068 // Despite mod_no_intg_q_replicated is not blanked, a regular
1069 // predecoded MUX is sufficient. The
1070
1071 // reason is that SW can control the value of MOD to prevent
1072 // undesirable SCA leakage due to
1073 // combining MOD and the operand_b of adder Y.
1074 assign shift_mod_mux_out =
1075     alu_bignum_predec_i.shift_mod_sel ? adder_y_op_shifter_res_blanked
1076     :
1077     mod_no_intg_q_replicated;
1078
1079 // The carries_in bits are predecoded except the LSB. It
1080 // cannot be predecoded because its value
1081 // depends on the actual flag value.
1082 otbn_vec_adder #(
1083     .LVLEN(VLEN),
1084     .LVChunkLEN(VChunkLEN)
1085 ) u_vec_adder_y (
1086     .operand_a_i (x_res_operand_a_mux_out),
1087     .operand_b_i (shift_mod_mux_out),
1088     .oper-
1089 and_b_invert_i(alu_bignum_predec_i.adder_y_op_b_invert),
1090     .carries_in_i
1091 ({alu_bignum_predec_i.adder_y_carries_top, adder_y_carry_0_in}),
1092     .use_ext_carry_i ({NVec-
1093 Proc{alu_bignum_predec_i.adder_carry_sel}}),
1094     .sum_o (adder_y_res),
1095     .carries_out_o (adder_y_carries_out)
1096 );
1097
1098 // Modulo Result Selection
1099
1100 logic [WLEN-1:0] arithmetic_result;
1101 logic arithmetic_result_used_adder_y;
1102
1103 otbn_mod_result_selector u_mod_result_selector (
1104     .result_x_i (adder_x_res),
1105     .carries_x_i (adder_x_carries_out),
1106     .result_y_i (adder_y_res),
1107     .carries_y_i (adder_y_carries_out),
1108     .is_subtraction_i(alu_bignum_predec_i.mod_is_subtraction),
1109     // Adder X is exclusively used for modulo instructions
1110     .is_modulo_i (alu_bignum_predec_i.adder_x_en),
1111     .elen_i (alu_bignum_predec_i.alu_elen),
1112     .result_o (arithmetic_result),
1113     .adder_y_used_o (arithmetic_result_used_adder_y)
1114 );
1115
1116 // Shifter, Adders, Logic & Transposer control
1117
1118 logic [NVecProc-1:0] expected_adder_x_carries_in;
1119 logic expected_adder_x_op_b_invert;
1120 logic [NVecProc-2:0] expected_adder_y_carries_top;
1121 logic expected_adder_y_op_b_invert;
1122
1123 logic expected_adder_x_en;
1124 logic expected_x_res_operand_a_sel;
1125 logic expected_adder_y_op_a_en;
1126 logic expected_adder_y_op_shifter_en;
1127
1128 logic [1:0] expected_shift_op_a_sel;
1129 logic [1:0] expected_shift_op_b_sel;
1130 logic expected_shift_en;
1131 logic expected_shift_right;
1132 logic expected_shift_mod_sel;
1133 logic expected_logic_a_en;
1134 logic expected_logic_shifter_en;
1135 logic [3:0] expected_logic_res_sel;
1136
1137 logic expected_mod_is_subtraction;
1138 logic expected_trn_en;
1139 logic expected_trn_is_trn1;
1140 logic expected_unpack_shifter_en;
1141
1142 always_comb begin
1143     // We use vectorized adders which support 8x32b or 256b
1144     // elements stored in a WDR. These adders
1145     // are split into 8 32b processing elements. Thus we have
1146     // a carry for each processing element.
1147     // These carries are predecoded except the LSB carry for
1148     // adder Y as it depends on the flag
1149     // value.
1150     expected_adder_x_carries_in = '0;

```

```

1141 expected_adder_x_op_b_invert = 'b0;
1142 expected_adder_y_carries_top = '0;
1143 adder_y_carry_0_in = 'b0;
1144 expected_adder_y_op_b_invert = 'b0;
1145 adder_update_flags_en_raw = 'b0;
1146 logic_update_flags_en_raw = 'b0;
1147
1148 expected_adder_x_en = 'b0;
1149 expected_x_res_operand_a_sel = 'b0;
1150 expected_adder_y_op_a_en = 'b0;
1151 expected_adder_y_op_shifter_en = 'b0;
1152 expected_shift_op_a_sel = '0;
1153 expected_shift_op_b_sel = '0;
1154 expected_shift_en = 'b0;
1155 expected_shift_right = 'b0;
1156 expected_shift_mod_sel = 'b1;
1157 expected_logic_a_en = 'b0;
1158 expected_logic_shifter_en = 'b0;
1159 expected_logic_res_sel = '0;
1160
1161 expected_mod_is_subtraction = 'b0;
1162 expected_trn_en = 'b0;
1163 expected_trn_is_trn1 = 'b0;
1164
1165 expected_unpack_shifter_en = 'b0;
1166
1167 unique case (operation_i.op)
1168   AluOpBigNumAdd,
1169   AluOpBigNumAddv: begin
1170     // Shifter computes B [>>|<<] shift_amt, no shift for
vectorized variant.
1171     // Y computes A + shifter_res
1172     // X ignored
1173     expected_adder_y_carries_top = '0;
1174     adder_y_carry_0_in = 'b0;
1175     expected_adder_y_op_b_invert = 'b0;
1176
1177     expected_adder_y_op_shifter_en = 'b1;
1178     expected_adder_y_op_a_en = 'b1;
1179     expected_shift_op_b_sel[AluShiftOpFull] = 'b1;
1180     expected_shift_en = 'b1;
1181
1182     if (operation_i.op == AluOpBigNumAdd) begin
1183       adder_update_flags_en_raw = 'b1;
1184       expected_shift_right = operation_i.shift_right;
1185     end
1186   end
1187   AluOpBigNumAddv: begin
1188     // Shifter computes B [>>|<<] shift_amt
1189     // Y computes A + shifter_res + flags.C
1190     // X ignored
1191     expected_adder_y_carries_top = '0;
1192     adder_y_carry_0_in = selected_flags.C;
1193     expected_adder_y_op_b_invert = 'b0;
1194     adder_update_flags_en_raw = 'b1;
1195
1196     expected_adder_y_op_shifter_en = 'b1;
1197     expected_adder_y_op_a_en = 'b1;
1198     expected_shift_op_b_sel[AluShiftOpFull] = 'b1;
1199     expected_shift_en = 'b1;
1200     expected_shift_right = operation_i.shift_right;
1201   end
1202   AluOpBigNumAddm,
1203   AluOpBigNumAddvm: begin
1204     // X computes A + B
1205
1206     // Y computes adder_x_res - mod = adder_x_res + ~mod +
1
1207     // Shifter ignored
1208
1209     // Mod result selector chooses result based on carries
of X and Y (whether mod subtraction
1210     // in Y should be applied or not).
1211     expected_adder_x_carries_in = '0;
1212     expected_adder_x_op_b_invert = 'b0;
1213     expected_adder_y_carries_top = operation_i.op ==
AluOpBigNumAddm ? '0 :
1214
1215     {(NVecProc-1){1'b1}};
1216     adder_y_carry_0_in = 'b1;
1217     expected_adder_y_op_b_invert = 'b1;
1218
1219     expected_adder_x_en = 'b1;
1220     expected_x_res_operand_a_sel = 'b1;
1221     expected_shift_mod_sel = 'b0;
1222   end
1223   AluOpBigNumSub,
1224   AluOpBigNumSubv: begin
1225     // Shifter computes B [>>|<<] shift_amt, no shift for
vectorized variant.
1226     // Y computes A - shifter_res = A + ~shifter_res + 1
1227     // X ignored
1228     expected_adder_y_carries_top = operation_i.op ==
AluOpBigNumSub ? '0 :
1229
1230     {(NVecProc-1){1'b1}};
1231     adder_y_carry_0_in = 'b1;
1232     expected_adder_y_op_b_invert = 'b1;

```

```

1229 expected_adder_y_op_shifter_en = 'b1;
1230
1231 expected_adder_y_op_a_en = 'b1;
1232 expected_shift_op_b_sel[AluShiftOpFull] = 'b1;
1233 expected_shift_en = 'b1;
1234
1235 if (operation_i.op == AluOpBigNumSub) begin
1236   adder_update_flags_en_raw = 'b1;
1237   expected_shift_right = operation_i.shift_right;
1238 end
1239 end
1240 AluOpBigNumSubv: begin
1241   // Shifter computes B [>>|<<] shift_amt
1242   // Y computes A - shifter_res + ~flags.C = A +
~shifter_res + flags.C
1243   // X ignored
1244   expected_adder_y_carries_top = '0;
1245   adder_y_carry_0_in = ~selected_flags.C;
1246   expected_adder_y_op_b_invert = 'b1;
1247   adder_update_flags_en_raw = 'b1;
1248   expected_adder_y_op_shifter_en = 'b1;
1249
1250   expected_adder_y_op_a_en = 'b1;
1251   expected_shift_op_b_sel[AluShiftOpFull] = 'b1;
1252   expected_shift_en = 'b1;
1253   expected_shift_right = operation_i.shift_right;
1254 end
1255 AluOpBigNumSubm,
1256 AluOpBigNumSubvm: begin
1257   // X computes A - B = A + ~B + 1
1258   // Y computes adder_x_res + mod
1259   // Shifter ignored
1260   // Mod result selector chooses result based on carry
of X (whether subtraction in Y should
1261   // be applied or not).
1262   expected_adder_x_carries_in = operation_i.op ==
AluOpBigNumSubm ? NVecProc'(1) :
1263
1264   {(NVecProc){1'b1}};
1265   expected_adder_x_op_b_invert = 'b1;
1266   expected_adder_y_carries_top = '0;
1267   adder_y_carry_0_in = 'b0;
1268   expected_adder_y_op_b_invert = 'b0;
1269
1270   expected_adder_x_en = 'b1;
1271   expected_x_res_operand_a_sel = 'b1;
1272   expected_shift_mod_sel = 'b0;
1273   expected_mod_is_subtraction = 'b1;
1274 end
1275 AluOpBigNumRshi: begin
1276   // Shifter computes {A, B} >> shift_amt
1277   // X, Y ignored
1278   // Feed blanked shifter output (adder_y_op_shifter_res_blanked) to Y to avoid undesired
1279   // leakage in the zero flag computation.
1280   expected_shift_op_a_sel[AluShiftOpFull] = 'b1;
1281   expected_shift_op_b_sel[AluShiftOpFull] = 'b1;
1282   expected_shift_en = 'b1;
1283   expected_shift_right = 'b0;
1284   // The shifter result is pushed through the logic
block to save an input to the result mux.
1285   // logic operation set to OR.
1286   expected_logic_shifter_en = 'b1;
1287   expected_logic_res_sel[AluOpLogicOr] = 'b1;
1288 end
1289 AluOpBigNumXor,
1290 AluOpBigNumOr,
1291 AluOpBigNumAnd,
1292 AluOpBigNumNot: begin
1293   // Shift computes one operand for the logical operation
1294   // X & Y ignored
1295   // Feed blanked shifter output (adder_y_op_shifter_res_blanked) to Y to avoid undesired
1296   // leakage in the zero flag computation.
1297   logic_update_flags_en_raw = 'b1;
1298
1299   expected_shift_op_b_sel[AluShiftOpFull] = 'b1;
1300   expected_shift_en = 'b1;
1301   expected_shift_right = operation_i.shift_right;
1302   expected_logic_a_en = operation_i.op != AluOpBigNumNot;
1303   expected_logic_shifter_en = 'b1;
1304   expected_logic_res_sel[AluOpLogicXor] = operation_i.op == AluOpBigNumXor;
1305   expected_logic_res_sel[AluOpLogicOr] = operation_i.op == AluOpBigNumOr;
1306   expected_logic_res_sel[AluOpLogicAnd] = operation_i.op == AluOpBigNumAnd;
1307   expected_logic_res_sel[AluOpLogicNot] = operation_i.op == AluOpBigNumNot;
1308 end
1309 AluOpBigNumTrn1,
1310 AluOpBigNumTrn2: begin
1311   expected_trn_en = 'b1;
1312   expected_trn_is_trn1 = operation_i.op == AluOp-
BigNumTrn1;
1313 end

```

```

1313     AluOpBignumShv: begin
1314         expected_shift_op_b_sel[AluShiftOpFull] = 'b1;
1315         expected_shift_en = 'b1;
1316         expected_shift_right = opera-
tion_i.shift_right;
1317         // The shifter result is pushed through the logic
block to save an input to the result mux.
1318         // logic operation set to OR.
1319         expected_logic_shifter_en = 'b1;
1320         expected_logic_res_sel[AluOpLogicOr] = 'b1;
1321     end
1322     AluOpBignumPack: begin
1323         expected_shift_op_a_sel[AluShiftOpDense] = 'b1;
1324         expected_shift_op_b_sel[AluShiftOpDense] = 'b1;
1325         expected_shift_en = 'b1;
1326         expected_shift_right = 'b1;
1327         // The packed result (shifter result) is pushed
through the logic block with operand A
1328         // blanked and the logic operation set to OR.
1329         expected_logic_shifter_en = 'b1;
1330         expected_logic_res_sel[AluOpLogicOr] = 'b1;
1331     end
1332     AluOpBignumUnpk: begin
1333         expected_shift_op_a_sel[AluShiftOpFull] = 'b1;
1334         expected_shift_op_b_sel[AluShiftOpFull] = 'b1;
1335         expected_shift_en = 'b1;
1336         expected_shift_right = 'b1;
1337         expected_unpack_shifter_en = 'b1;
1338     end
1339     // No operation, do nothing.
1340     AluOpBignumNone: ;
1341     default: ;
1342 endcase
1343 end
1344
1345     alu_elen_e           expected_alu_elen;
1346     trn_elen_e           expected_trn_elen;
1347     logic                expected_adder_carry_sel;
1348     logic [$clog2(WLEN)-1:0] expected_shift_amt;
1349     logic [VChunkLEN-1:0] expected_shift_mask;
1350     logic [1:0]          expected_shift_dir;
1351
1352     assign expected_alu_elen = operation_i.alu_elen;
1353     assign expected_trn_elen = operation_i.trn_elen;
1354     assign expected_adder_carry_sel = opera-
tion_i.adder_carry_sel;
1355     assign expected_shift_amt = operation_i.shift_amt;
1356     assign expected_shift_mask = operation_i.shift_mask;
1357
1358     assign expected_shift_dir[AluShiftDirLeft] = expec-
ted_shift_en &
1359                                     !expec-
ted_shift_right;
1360     assign expected_shift_dir[AluShiftDirRight] = expec-
ted_shift_en &
1361                                     expec-
ted_shift_right;
1362
1363     // SEC_CM: CTRL.REDUN
1364     assign alu_predec_error_o =
1365         |(expected_adder_x_carries_in !=
alu_bignum_predec_i.adder_x_carries_in,
1366         expected_adder_x_op_b_invert !=
alu_bignum_predec_i.adder_x_op_b_invert,
1367         expected_adder_y_carries_top !=
alu_bignum_predec_i.adder_y_carries_top,
1368         expected_adder_y_op_b_invert !=
alu_bignum_predec_i.adder_y_op_b_invert,
1369         expected_adder_x_en != alu_bignum_predec_i.adder_x_en,
1370         expected_x_res_operand_a_sel !=
alu_bignum_predec_i.x_res_operand_a_sel,
1371         expected_adder_y_op_a_en !=
alu_bignum_predec_i.adder_y_op_a_en,
1372         expected_adder_y_op_shifter_en !=
alu_bignum_predec_i.adder_y_op_shifter_en,
1373         expected_shift_op_a_sel !=
alu_bignum_predec_i.shift_op_a_sel,
1374         expected_shift_op_b_sel !=
alu_bignum_predec_i.shift_op_b_sel,
1375         expected_shift_dir != alu_bignum_predec_i.shift_dir,
1376         expected_shift_amt != alu_bignum_predec_i.shift_amt,
1377         expected_shift_mod_sel !=
alu_bignum_predec_i.shift_mod_sel,
1378         expected_logic_a_en != alu_bignum_predec_i.logic_a_en,
1379         expected_logic_shifter_en !=
alu_bignum_predec_i.logic_shifter_en,
1380         expected_logic_res_sel !=
alu_bignum_predec_i.logic_res_sel,
1381         expected_flag_group_sel !=
alu_bignum_predec_i.flag_group_sel,
1382         expected_flag_sel != alu_bignum_predec_i.flag_sel,
1383         expected_flags_keep != alu_bignum_predec_i.flags_keep,
1384         expected_flags_adder_update !=
alu_bignum_predec_i.flags_adder_update,
1385         expected_flags_logic_update !=
alu_bignum_predec_i.flags_logic_update,
1386         expected_flags_mac_update !=
alu_bignum_predec_i.flags_mac_update,
1387         expected_flags_ispr_wr !=
alu_bignum_predec_i.flags_ispr_wr,
1388         expected_alu_elen != alu_bignum_predec_i.alu_elen,

```

```

1389         expected_trn_elen != alu_bignum_predec_i.trn_elen,
1390         expected_adder_carry_sel !=
alu_bignum_predec_i.adder_carry_sel,
1391         expected_mod_is_subtraction !=
alu_bignum_predec_i.mod_is_subtraction,
1392         expected_shift_mask != alu_bignum_predec_i.shift_mask,
1393         expected_trn_en != alu_bignum_predec_i.trn_en,
1394         expected_trn_is_trn1 != alu_bignum_predec_i.trn_is_trn1,
1395         expected_unpack_shifter_en !=
alu_bignum_predec_i.unpack_shifter_en);
1396
1397     // Logical operations //
1398     // Logical operations //
1399     // Logical operations //
1400
1401     logic [WLEN-1:0] logical_res_mux_in [4];
1402     logic [WLEN-1:0] logical_op_a_blanked;
1403     logic [WLEN-1:0] logical_op_shifter_res_blanked;
1404
1405     // SEC_CM: DATA_REG_SW.SCA
1406     prim_blanker #(.Width(WLEN)) u_logical_op_a_blanker (
1407         .in_i (operation_i.operand_a),
1408         .en_i (alu_bignum_predec_i.logic_a_en),
1409         .out_o(logical_op_a_blanked)
1410     );
1411
1412     // SEC_CM: DATA_REG_SW.SCA
1413     prim_blanker #(.Width(WLEN)) u_logic-
al_op_shifter_res_blanker (
1414         .in_i (shifter_res),
1415         .en_i (alu_bignum_predec_i.logic_shifter_en),
1416         .out_o(logical_op_shifter_res_blanked)
1417     );
1418
1419     assign logical_res_mux_in[AluOpLogicXor] = logic-
al_op_a_blanked ^ logical_op_shifter_res_blanked;
1420     assign logical_res_mux_in[AluOpLogicOr] = logic-
al_op_a_blanked | logical_op_shifter_res_blanked;
1421     assign logical_res_mux_in[AluOpLogicAnd] = logic-
al_op_a_blanked & logical_op_shifter_res_blanked;
1422     assign logical_res_mux_in[AluOpLogicNot] = ~logic-
al_op_shifter_res_blanked;
1423
1424     // SEC_CM: DATA_REG_SW.SCA
1425     prim_onehot_mux #(
1426         .Width (WLEN),
1427         .Inputs(4)
1428     ) u_logical_res_mux (
1429         .clk_i,
1430         .rst_ni,
1431         .in_i (logical_res_mux_in),
1432         .sel_i (alu_bignum_predec_i.logic_res_sel),
1433         .out_o(logical_res)
1434     );
1435
1436     // Vector Transposer //
1437     // Vector Transposer //
1438     // Vector Transposer //
1439
1440     logic [WLEN-1:0] vec_transposer_op_a_blanked;
1441     logic [WLEN-1:0] vec_transposer_op_b_blanked;
1442     logic [WLEN-1:0] vec_transposer_res;
1443
1444     // SEC_CM: DATA_REG_SW.SCA
1445     prim_blanker #(.Width(WLEN)) u_vec_transposer_op_a_blanker (
1446         .in_i (operation_i.operand_a),
1447         .en_i (alu_bignum_predec_i.trn_en),
1448         .out_o(vec_transposer_op_a_blanked)
1449     );
1450
1451     // SEC_CM: DATA_REG_SW.SCA
1452     prim_blanker #(.Width(WLEN)) u_vec_transposer_op_b_blanker (
1453         .in_i (operation_i.operand_b),
1454         .en_i (alu_bignum_predec_i.trn_en),
1455         .out_o(vec_transposer_op_b_blanked)
1456     );
1457
1458     otbn_vec_transposer u_vec_transposer (
1459         .clk_i,
1460         .rst_ni,
1461         .operand_a_i (vec_transposer_op_a_blanked),
1462         .operand_b_i (vec_transposer_op_b_blanked),
1463         .is_trn1_i (alu_bignum_predec_i.trn_is_trn1),
1464         .elen_i (alu_bignum_predec_i.trn_elen),
1465         .result_o (vec_transposer_res)
1466     );
1467
1468     // Output multiplexer //
1469     // Output multiplexer //
1470     // Output multiplexer //
1471     // Due to the blanking only one input to the output multi-
plexer is non zero at the same time.
1472     // This allows to implement the selection with one large OR
instead of a regular mux.
1473     // Note, the shifter result is pushed through the logic
block. Similarly is the non modulo result
1474     // pushed through the modulo result selector.
1475
1475     assign operation_result_o = arithmetic_result | unpacked_res
| logical_res | vec_transposer_res;
1476

```



```

1477 `ASSERT(OnlyOneResultActive_A,
1478     $onehot0({|arithmetic_result,
1479         |unpacked_res,
1480         |logical_res,
1481         |vec_transposer_res})),
1482     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1483
1484 // We need to generate a signal indicating whether we have
used the value of the MOD WSR and thus
1485 // need to check its integrity. See otbn_mod_result_selector
for more details.
1486 logic adder_y_res_used;
1487 always_comb begin
1488     unique case(operation_i.op)
1489         AluOpBignumAdd,
1490         AluOpBignumAddv,
1491         AluOpBignumAddc,
1492         AluOpBignumSub,
1493         AluOpBignumSubv,
1494         AluOpBignumSubb,
1495         AluOpBignumAddm,
1496         AluOpBignumAddvm,
1497         AluOpBignumSubm,
1498         AluOpBignumSubvm: adder_y_res_used = arithmet-
ic_result_used_adder_y;
1499
1500         AluOpBignumRshi,
1501         AluOpBignumXor,
1502         AluOpBignumOr,
1503         AluOpBignumAnd,
1504         AluOpBignumNot,
1505         AluOpBignumTrn1,
1506         AluOpBignumTrn2,
1507         AluOpBignumShv,
1508         AluOpBignumPack,
1509         AluOpBignumUnpk: adder_y_res_used = 1'b0;
1510
1511         // Most arithmetic operations use the adder Y result, so
mark it as used as save default.
1512         default: adder_y_res_used = 1'b1;
1513     endcase
1514 end
1515
1516 // Tie off unused signals.
1517 logic unused_operation_commit;
1518 assign unused_operation_commit = operation_commit_i;
1519
1520 // Determine if `mod_intg_q` is used. The control signals
are only valid if `operation_i.op` is
1521 // not none. If `shift_mod_sel` is low, `mod_intg_q` flows
into `adder_y_op_b` and from there
1522 // into `adder_y_res`. In this case, `mod_intg_q` is used
iff `adder_y_res` flows into
1523 // `operation_result_o`.
1524 logic mod_used;
1525 assign mod_used = operation_valid_i & (operation_i.op !=
AluOpBignumNone) &
1526     !alu_bignum_predec_i.shift_mod_sel & ad-
der_y_res_used;
1527 `ASSERT_KNOWN(ModUsed_A, mod_used)
1528
1529 // Raise a register integrity violation error iff
`mod_intg_q` is used and (at least partially)
1530 // invalid.
1531 assign reg_intg_violation_err_o = mod_used & |
(mod_intg_err);
1532 `ASSERT_KNOWN(RegIntgErrKnown_A, reg_intg_violation_err_o)
1533
1534 // Detect and signal unexpected secure wipe signals.
1535 assign sec_wipe_err_o = sec_wipe_mod_urnd_i &
~sec_wipe_running_i;
1536
1537 // Blanking Assertions
1538 // All blanking assertions are reset with predec_error or
overall error in the whole system
1539 // -indicated by operation_commit_i port- as OTBN does not
guarantee blanking in the case
1540 // of an error.
1541 // adder_x_res related blanking
1542 `ASSERT(BlankingBignumAluXOp_A,
1543     !expected_adder_x_en
1544     |> {adder_x_op_a_blanked, adder_x_op_b_blanked, ad-
der_x_res, adder_x_carries_out} == '0,
1545     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1546
1547 // adder_y_res related blanking
1548 `ASSERT(BlankingBignumAluYOpA_A,
1549     !expected_adder_y_op_a_en |> adder_y_op_a_blanked
== '0,
1550     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1551
1552 `ASSERT(BlankingBignumAluYOpShft_A,
1553     !expected_adder_y_op_shifter_en |> ad-
der_y_op_shifter_res_blanked == '0,
1554     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1555
1556 // Most arithmetic operations use the adder Y result, so
mark it as used as save default.
1557 // not none. If `shift_mod_sel` is low, `mod_intg_q` flows
into `adder_y_op_b` and from there
1558 // into `adder_y_res`. In this case, `mod_intg_q` is used
iff `adder_y_res` flows into
1559 // `operation_result_o`.
1560 logic mod_used;
1561 assign mod_used = operation_valid_i & (operation_i.op !=
AluOpBignumNone) &
1562     !alu_bignum_predec_i.shift_mod_sel & ad-
der_y_res_used;
1563 `ASSERT_KNOWN(ModUsed_A, mod_used)
1564
1565 // Raise a register integrity violation error iff
`mod_intg_q` is used and (at least partially)
1566 // invalid.
1567 assign reg_intg_violation_err_o = mod_used & |
(mod_intg_err);
1568 `ASSERT_KNOWN(RegIntgErrKnown_A, reg_intg_violation_err_o)
1569
1570 // Detect and signal unexpected secure wipe signals.
1571 assign sec_wipe_err_o = sec_wipe_mod_urnd_i &
~sec_wipe_running_i;
1572
1573 // Blanking Assertions
1574 // All blanking assertions are reset with predec_error or
overall error in the whole system
1575 // -indicated by operation_commit_i port- as OTBN does not
guarantee blanking in the case
1576 // of an error.
1577 // adder_x_res related blanking
1578 `ASSERT(BlankingBignumAluXOp_A,
1579     !expected_adder_x_en
1580     |> {adder_x_op_a_blanked, adder_x_op_b_blanked, ad-
der_x_res, adder_x_carries_out} == '0,
1581     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1582
1583 // adder_y_res related blanking
1584 `ASSERT(BlankingBignumAluYOpA_A,
1585     !expected_adder_y_op_a_en |> adder_y_op_a_blanked
== '0,
1586     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1587
1588 `ASSERT(BlankingBignumAluYOpShft_A,
1589     !expected_adder_y_op_shifter_en |> ad-
der_y_op_shifter_res_blanked == '0,
1590     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1591
1592 // Vector transposer related blanking
1593 `ASSERT(BlankingBignumAluVecTrn_A,
1594     !expected_trn_en
1595     |> {vec_transposer_op_a_blanked, vec_trans-
poser_op_b_blanked} == '0,
1596     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1597
1598 // Vector unpack related blanking
1599 `ASSERT(BlankingBignumAluVecUnpk_A,
1600     !expected_unpack_shifter_en |> unpacked_res ==
'0,
1601     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1602
1603 // MOD ISPR Blanking
1604 `ASSERT(BlankingIsprMod_A,
1605     !(|mod_wr_en) |> ispr_mod_bignum_wdata_intg_blanked
== '0,
1606     clk_i, !rst_ni || ispr_predec_error_o || alu_pre-
dec_error_o || !operation_commit_i)
1607
1608 // ACC ISPR Blanking
1609 `ASSERT(BlankingIsprACC_A,
1610     !(|ispr_acc_wr_en) |>
ispr_acc_bignum_wdata_intg_blanked == '0,
1611     clk_i, !rst_ni || ispr_predec_error_o || alu_pre-
dec_error_o || !operation_commit_i)
1612 endmodule

```

```

1555 // Adder Y must be blanked when its result is not used, with
one exception: For `BN.SUBM` and
1556 // `BN.SUBVM` with `a >= b` (thus the result of Adder X has
the carry bit set), the result of

```

```

1557 // Adder Y is not used but it cannot be blanked solely based
on the carry bit.
1558 `ASSERT(BlankingBignumAluYResUsed_A,
1559     !adder_y_res_used &&
1560     !((operation_i.op == AluOpBignumSubm || opera-
tion_i.op == AluOpBignumSubvm)
1561     && !arithmetic_result_used_adder_y)
1562     |> {x_res_operand_a_mux_out, shift_mod_mux_out} ==
'0,
1563     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1564
1565 // shifter_res related blanking
1566 `ASSERT(BlankingBignumAluShftA_A,

```

```

1567     (expected_shift_op_a_sel == '0) |> shifter_in_upper
== '0,
1568     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1569
1570 `ASSERT(BlankingBignumAluShftB_A,

```

```

1571     (expected_shift_op_b_sel == '0) |> shifter_in_lower
== '0,
1572     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1573
1574 `ASSERT(BlankingBignumAluShftRes_A,
1575     (expected_shift_op_a_sel == '0) && (expec-
ted_shift_op_b_sel == '0) |> shifter_res == '0,
1576     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1577
1578 // logical_res related blanking
1579 `ASSERT(BlankingBignumAluLogicOPA_A,
1580     !expected_logic_a_en |> logical_op_a_blanked == '0,
1581     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1582
1583 `ASSERT(BlankingBignumAluLogicShft_A,
1584     !expected_logic_shifter_en |> logic-
al_op_shifter_res_blanked == '0,
1585     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1586
1587 `ASSERT(BlankingBignumAluLogicRes_A,
1588     !expected_logic_a_en || expected_logic_shifter_en)
1589 |> logical_res == '0,
1590     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1591
1592 // Vector transposer related blanking
1593 `ASSERT(BlankingBignumAluVecTrn_A,
1594     !expected_trn_en
1595     |> {vec_transposer_op_a_blanked, vec_trans-
poser_op_b_blanked} == '0,
1596     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1597
1598 // Vector unpack related blanking
1599 `ASSERT(BlankingBignumAluVecUnpk_A,
1600     !expected_unpack_shifter_en |> unpacked_res ==
'0,
1601     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1602
1603 // MOD ISPR Blanking
1604 `ASSERT(BlankingIsprMod_A,

```

```

1604     !(|mod_wr_en) |> ispr_mod_bignum_wdata_intg_blanked
== '0,
1605     clk_i, !rst_ni || ispr_predec_error_o || alu_pre-
dec_error_o || !operation_commit_i)
1606
1607 // ACC ISPR Blanking
1608 `ASSERT(BlankingIsprACC_A,
1609     !(|ispr_acc_wr_en) |>
ispr_acc_bignum_wdata_intg_blanked == '0,
1610     clk_i, !rst_ni || ispr_predec_error_o || alu_pre-
dec_error_o || !operation_commit_i)
1611
1612 endmodule

```

4. lowRISC opentitan/hw/ip/otbn/rtl/otbn_controller.sv (1,680 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6

```

```

7 /**
8  * OTBN Controller
9  */
10 module otbn_controller
11     import otbn_pkg::*;
12     #(
13         // Size of the instruction memory, in bytes
14         parameter int ImemSizeByte = 4096,
15         // Size of the data memory, in bytes
16         parameter int DmemSizeByte = 4096,
17
18         localparam int ImemAddrWidth =
19             prim_util_pkg::vbits(ImemSizeByte),
20         localparam int DmemAddrWidth =
21             prim_util_pkg::vbits(DmemSizeByte)
22     ) (
23         input logic clk_i,
24         input logic rst_ni,
25
26         input logic start_i, // start the processing at address
27         zero
28         output logic locking_o, // Controller is in or is entering
29         the locked state
30         input logic err_bit_clear_i,
31
32         input prim_mubi_pkg::mubi4_t fatal_escalate_en_i,
33         input prim_mubi_pkg::mubi4_t recov_escalate_en_i,
34         input prim_mubi_pkg::mubi4_t rma_req_i,
35         output controller_err_bits_t err_bits_o,
36         output logic recoverable_err_o,
37
38         // Next instruction selection (to instruction fetch)
39         output logic insn_fetch_req_valid_o,
40         output logic insn_fetch_req_valid_raw_o,
41         output logic [ImemAddrWidth-1:0] insn_fetch_req_addr_o,
42         output logic insn_fetch_resp_clear_o,
43
44         // Fetched/decoded instruction
45         input logic insn_valid_i,
46         input logic insn_illegal_i,
47         input logic [ImemAddrWidth-1:0] insn_addr_i,
48
49         // Decoded instruction data
50         input insn_dec_base_t insn_dec_base_i,
51         input insn_dec_bignum_t insn_dec_bignum_i,
52         input insn_dec_shared_t insn_dec_shared_i,
53
54         // Base register file
55         output logic [4:0] rf_base_wr_addr_o,
56         output logic rf_base_wr_en_o,
57         output logic rf_base_wr_commit_o,
58         output logic [31:0] rf_base_wr_data_no_intg_o,
59         output logic [BaseIntgWidth-1:0] rf_base_wr_data_intg_o,
60         output logic rf_base_wr_data_intg_sel_o,
61
62         output logic [4:0] rf_base_rd_addr_a_o,
63         output logic rf_base_rd_en_a_o,
64         input logic [BaseIntgWidth-1:0] rf_base_rd_data_a_intg_i,
65         output logic [4:0] rf_base_rd_addr_b_o,
66         output logic rf_base_rd_en_b_o,
67         input logic [BaseIntgWidth-1:0] rf_base_rd_data_b_intg_i,
68         output logic rf_base_rd_commit_o,
69
70         input logic rf_base_call_stack_sw_err_i,
71         input logic rf_base_call_stack_hw_err_i,
72
73         // Bignum register file (WDRs)
74         output logic [4:0] rf_bignum_wr_addr_o,
75         output logic [1:0] rf_bignum_wr_en_o,
76         output logic rf_bignum_wr_commit_o,
77         output logic [WLEN-1:0] rf_bignum_wr_data_no_intg_o,
78         output logic [ExtWLEN-1:0] rf_bignum_wr_data_intg_o,
79         output logic rf_bignum_wr_data_intg_sel_o,
80
81         output logic [4:0] rf_bignum_rd_addr_a_o,
82         output logic rf_bignum_rd_en_a_o,
83         input logic [ExtWLEN-1:0] rf_bignum_rd_data_a_intg_i,
84
85         output logic [4:0] rf_bignum_rd_addr_b_o,
86         output logic rf_bignum_rd_en_b_o,
87         input logic [ExtWLEN-1:0] rf_bignum_rd_data_b_intg_i,
88
89         input logic rf_bignum_intg_err_i,
90         input logic rf_bignum_spurious_we_err_i,
91
92         output logic [NWdr-1:0] rf_bignum_rd_a_indirect_onehot_o,
93         output logic [NWdr-1:0] rf_bignum_rd_b_indirect_onehot_o,
94         output logic [NWdr-1:0] rf_bignum_wr_indirect_onehot_o,
95         output logic rf_bignum_indirect_en_o,
96
97         // Execution units
98         // Base ALU
99         output alu_base_operation_t alu_base_operation_o,
100         output alu_base_comparison_t alu_base_comparison_o,
101         input logic [31:0] alu_base_operation_result_i,
102         input logic alu_base_comparison_result_i,
103
104         // Bignum ALU
105         output alu_bignum_operation_t alu_bignum_operation_o,
106         output logic alu_bignum_operation_valid_o,
107
108         output logic alu_bignum_operation_commit_o,
109         input logic [WLEN-1:0] alu_bignum_operation_result_i,
110         input logic alu_bignum_operation_valid_i,
111         output logic mac_bignum_en_o,
112         output logic mac_bignum_commit_o,
113
114         // LSU
115         output logic lsu_load_req_o,
116         output logic lsu_store_req_o,
117         output logic insn_subset_e,
118         output logic [DmemAddrWidth-1:0] lsu_addr_o,
119         input logic lsu_addr_en_predec_i,
120
121         output logic [BaseIntgWidth-1:0] lsu_base_wdata_o,
122         output logic [ExtWLEN-1:0] lsu_bignum_wdata_o,
123
124         input logic [BaseIntgWidth-1:0] lsu_base_rdata_i,
125         input logic [ExtWLEN-1:0] lsu_bignum_rdata_i,
126
127         // Internal Special-Purpose Registers (ISPRs)
128         output ispr_e ispr_addr_o,
129         output logic [31:0] ispr_base_wdata_o,
130         output logic [BaseWordsPerWLEN-1:0] ispr_base_wr_en_o,
131         output logic [ExtWLEN-1:0] ispr_bignum_wdata_intg_o,
132         output logic ispr_bignum_wr_en_o,
133         output logic [NFlagGroups-1:0] ispr_flags_wr_o,
134         output logic ispr_wr_commit_o,
135         input logic [ExtWLEN-1:0] ispr_rdata_intg_i,
136         output logic ispr_rd_en_o,
137
138         // MAI
139         input logic mai_software_error_i,
140
141         // RND interface
142         output logic rnd_req_o,
143         output logic rnd_prefetch_req_o,
144         input logic rnd_valid_i,
145
146         input logic urnd_reseed_err_i,
147
148         // WFI
149         output logic wfi_pending_o,
150         input logic wfi_resume_i,
151
152         // Secure Wipe
153         output logic secure_wipe_req_o,
154         input logic secure_wipe_ack_i,
155         input logic sec_wipe_zero_i,
156         input logic secure_wipe_running_i,
157         input logic sec_wipe_err_i,
158
159         input logic state_reset_i,
160         output logic [31:0] insn_cnt_o,
161         input logic insn_cnt_clear_ext_i,
162         input logic insn_cnt_clear_int_i,
163         output logic mems_sec_wipe_o,
164
165         input logic software_errs_fatal_i,
166
167         input logic [1:0] sideload_key_shares_valid_i,
168
169         // Prefetch stage control
170         output logic prefetch_en_o,
171         output logic prefetch_loop_active_o,
172         output logic [31:0] prefetch_loop_iterations_o,
173         output logic [ImemAddrWidth:0] prefetch_loop_end_addr_o,
174         output logic [ImemAddrWidth-1:0] prefetch_loop_jump_addr_o,
175         output logic prefetch_ignore_errs_o,
176
177         // Predecoded control
178         input ctrl_flow_predec_t ctrl_flow_predec_i,
179         input logic [ImemAddrWidth-1:0] ctrl_flow_target_predec_i,
180         output logic predec_error_o,
181     );
182     import prim_mubi_pkg::*;
183
184     otbn_state_e state_q, state_d;
185
186     controller_err_bits_t err_bits_q, err_bits_d;
187
188     // The specific error signals that go into err_bits_d
189     logic fatal_software_err, bad_internal_state_err,
190     reg_intg_violation_err, key_invalid_err;
191     logic illegal_insn_err, bad_data_addr_err,
192     call_stack_sw_err, bad_insn_addr_err;
193
194     logic err;
195     logic internal_err;
196     logic recoverable_err;
197     logic software_err;
198     logic non_insn_addr_software_err;
199     logic fatal_err;
200     logic internal_fatal_err;
201     logic done_complete;

```

```

202 logic executing;
203 logic state_error, state_error_d, state_error_q;
204 logic spurious_secure_wipe_ack_q, spuri-
ous_secure_wipe_ack_d;
205 logic sec_wipe_err_q, sec_wipe_err_d;
206 logic mubi_err_q, mubi_err_d;
207
208 logic          insn_fetch_req_valid_raw;
209 logic [ImemAddrWidth-1:0] insn_fetch_req_addr_last;
210
211 logic stall;
212 logic ispr_stall;
213 logic mem_stall;
214 logic rf_indirect_stall;
215 logic mac_bignum_stall;
216 logic wfi_stall;
217 logic wfi_pending_d, wfi_pending_q;
218 logic jump_or_branch;
219 logic branch_taken;
220 logic insn_executing;
221 logic ld_insn_with_addr_from_call_stack, st_in-
sn_with_addr_from_call_stack;
222 logic [ImemAddrWidth-1:0] branch_target;
223 logic          branch_target_overflow;
224 logic [ImemAddrWidth:0] next_insn_addr_wide;
225 logic [ImemAddrWidth-1:0] next_insn_addr;
226
227 csr_e          csr_addr;
228 logic [$clog2(BaseWordsPerWLEN)-1:0] csr_sub_addr;
229 logic [31:0]   csr_rdata_raw;
230 logic [31:0]   csr_rdata;
231 logic [BaseWordsPerWLEN-1:0]   csr_rdata_mux [32];
232 logic [31:0]   csr_wdata_raw;
233 logic [31:0]   csr_wdata;
234
235 wsr_e          wsr_addr;
236 logic [WLEN-1:0] wsr_wdata;
237
238 ispr_e          ispr_addr_base;
239 logic [$clog2(BaseWordsPerWLEN)-1:0] ispr_word_addr_base;
240 logic [BaseWordsPerWLEN-1:0] ispr_word_sel_base;
241
242 ispr_e          ispr_addr_bignum;
243
244 logic          ispr_wr_insn,
ispr_rd_insn;
245 logic          ispr_wr_base_insn;
246 logic          ispr_wr_bignum_insn;
247 logic          ispr_rd_bignum_insn;
248
249 logic          lsu_load_req_raw;
250 logic          lsu_store_req_raw;
251 logic [DmemAddrWidth-1:0] lsu_addr, lsu_addr_blanked,
lsu_addr_saved_d, lsu_addr_saved_q;
252 logic          lsu_addr_saved_sel;
253 logic          expected_lsu_addr_en;
254
255 logic          expected_call_stack_push, expec-
ted_call_stack_pop;
256 logic          lsu_preddec_error,
branch_target_preddec_error, ctrl_preddec_error;
257
258 logic rnd_req_raw;
259
260 // Register read data with integrity stripped off
261 logic [31:0]   rf_base_rd_data_a_no_intg;
262 logic [31:0]   rf_base_rd_data_b_no_intg;
263 logic [WLEN-1:0] rf_bignum_rd_data_a_no_intg;
264 logic [WLEN-1:0] rf_bignum_rd_data_b_no_intg;
265
266 logic [ExtWLEN-1:0] rf_bignum_rd_data_b_intg_blanked;
267 logic [ExtWLEN-1:0] selection_result;
268
269 logic [1:0] rf_bignum_wr_en_unbuf;
270 logic [4:0] rf_bignum_wr_addr_unbuf;
271 logic [4:0] rf_bignum_rd_addr_a_unbuf;
272 logic          rf_bignum_rd_en_a_unbuf;
273 logic [4:0] rf_bignum_rd_addr_b_unbuf;
274 logic          rf_bignum_rd_en_b_unbuf;
275
276 logic rf_bignum_rd_a_indirect_en;
277 logic rf_bignum_rd_b_indirect_en;
278 logic rf_bignum_wr_indirect_en;
279
280 // Computed increments for indirect register index and
memory address in BN.LID/BN.SID/BN.MOVR
281 // instructions.
282 logic [5:0] rf_base_rd_data_a_inc;
283 logic [5:0] rf_base_rd_data_b_inc;
284 logic [26:0] rf_base_rd_data_a_wlen_word_inc;
285
286 // Read/Write enables for base register file before illegal
instruction encoding are factored in
287 logic rf_base_rd_en_a_raw, rf_base_rd_en_b_raw,
rf_base_wr_en_raw;
288
289 // Output of mux taking the above increments as inputs and
choosing one to write back to base
290
291 // register file with appropriate zero extension and padding
to give a 32-bit result.

```

```

291 logic [31:0]          increment_out;
292
293 // Loop control, used to start a new loop
294 logic          loop_start_req;
295 logic          loop_start_commit;
296 logic          loop_reset;
297 logic [11:0] loop_bodysize;
298 logic [31:0] loop_iterations;
299
300 // Loop generated jumps. The loop controller asks to jump
when execution reaches the end of a loop
301 // body that hasn't completed all of its iterations.
302 logic          loop_jump;
303 logic [ImemAddrWidth-1:0] loop_jump_addr;
304
305 logic [WLEN-1:0] mac_bignum_rf_wr_data;
306
307 logic loop_hw_err, loop_preddec_err;
308 logic csr_illegal_addr, wsr_illegal_addr, ispr_illegal_addr;
309 logic imem_addr_err, loop_sw_err, ispr_err;
310 logic dmem_addr_err_check, dmem_addr_err;
311 logic dmem_addr_unaligned_base, dmem_addr_unaligned_bignum,
dmem_addr_overflow;
312 logic illegal_insn_static;
313 logic key_invalid;
314
315 logic rf_a_indirect_err, rf_b_indirect_err, rf_dindir-
ect_err, rf_indirect_err;
316
317 // If we are doing an indirect access to the bignum register
file, it's possible that the
318 // address that we use for the access is architecturally un-
known. This happens if it came from x1
319 // and we've underflowed the call stack. When this happens,
we want to ignore any read data
320 // integrity errors and spurious write enable errors since
the access to the bignum register file
321 // didn't happen architecturally anyway.
322 logic ignore_rf_bignum_intg_errs;
323 logic rf_bignum_intg_err;
324 logic ignore_rf_bignum_spurious_we_errs;
325 logic rf_bignum_spurious_we_err;
326
327 logic ispr_rdata_intg_err;
328
329 logic [31:0] insn_cnt_d, insn_cnt_q;
330 logic          insn_cnt_clear;
331
332 logic [4:0] insn_bignum_rd_addr_a_q,
insn_bignum_rd_addr_b_q, insn_bignum_wr_addr_q;
333
334 logic          start_secure_wipe;
335 logic          secure_wipe_running_q, secure_wipe_running_d;
336
337 assign secure_wipe_running_d = start_secure_wipe | (se-
cure_wipe_running_q & ~secure_wipe_ack_i);
338
339 always_ff @(posedge clk_i or negedge rst_ni) begin
340   if (!rst_ni) begin
341     secure_wipe_running_q <= 1'b0;
342   end else begin
343     secure_wipe_running_q <= secure_wipe_running_d;
344   end
345 end
346 assign secure_wipe_req_o = start_secure_wipe | se-
cure_wipe_running_q;
347
348 // Spot spurious acks on the secure wipe interface. There is a
an ack at the end of the initial
349 // secure wipe, and as `secure_wipe_running_q` is only high
during secure wipes triggered by this
350 // controller, we have to ignore acks before the initial se-
cure wipe is done. Register this
351 // signal to break a circular path (a secure wipe can be
triggered by a stop, and a spurious
352 // secure wipe ack can trigger a stop).
353 always_ff @(posedge clk_i or negedge rst_ni) begin
354   if (!rst_ni) begin
355     spurious_secure_wipe_ack_q <= 1'b0;
356   end else begin
357     spurious_secure_wipe_ack_q <= spuri-
ous_secure_wipe_ack_d;
358   end
359 end
360 assign spurious_secure_wipe_ack_d = spuri-
ous_secure_wipe_ack_q |
361   (secure_wipe_ack_i
&
362   ~secure_wipe_running_q
&
363   ~se-
cure_wipe_running_i);
364
365 // Detect and latch unexpected secure wipe signals.
366 always_ff @(posedge clk_i or negedge rst_ni) begin
367   if (!rst_ni) begin
368     sec_wipe_err_q <= 1'b0;
369   end else begin
370     sec_wipe_err_q <= sec_wipe_err_d;

```

```

371     end
372   end
373   assign sec_wipe_err_d = sec_wipe_err_q |
374     sec_wipe_err_i |
375     (sec_wipe_zero_i & ~se-
cure_wipe_running_i);
376
377   // Stall a cycle on loads to allow load data writeback to
happen the following cycle. Stall not
378   // required on stores as there is no response to deal with.
379   assign mem_stall = lsu_load_req_raw;
380
381   // Reads to RND must stall until data is available
382   assign ispr_stall = rnd_req_raw & ~rnd_valid_i;
383
384   assign rf_indirect_stall = insn_valid_i &
385     (state_q != OtnbStateStall) &
386     (insn_dec_shared_i.subset == In-
snSubsetBignum) &
387     (insn_dec_bignum_i.rf_a_indirect
|
388     insn_dec_bignum_i.rf_b_indirect
|
389     insn_dec_bignum_i.rf_d_indirect);
390
391   // Stall while a vectorized multicycle MAC calculation is
ongoing
392   assign mac_bignum_stall = insn_valid_i &
393     (insn_dec_shared_i.subset == Insn-
SubsetBignum) &
394     insn_dec_bignum_i.mac_en &
395     (~mac_bignum_operation_valid_i);
396
397   // Stall until the resume command arrives.
398   assign wfi_stall = insn_valid_i &&
insn_dec_shared_i.wfi_insn && !wfi_resume_i;
399   assign wfi_pending_d = wfi_stall;
400
401   // The resume command should only be issued when we are
already waiting.
402   `ASSERT(WfiResumeOnlyIfStalling, !wfi_pending_q |> !wfi_re-
sume_i)
403
404   // The wfi_pending is registered to avoid long paths into
the IMEM/DMEM core/bus mux.
405   always_ff @(posedge clk_i or negedge rst_ni) begin
406     if (!rst_ni) begin
407       wfi_pending_q <= 1'b0;
408     end else begin
409       wfi_pending_q <= wfi_pending_d;
410     end
411   end
412   assign wfi_pending_o = wfi_pending_q;
413
414   assign stall = mem_stall | ispr_stall | rf_indirect_stall |
mac_bignum_stall | wfi_stall;
415
416   // OTBN is done when it was executing something (in state
OtnbStateRun or OtnbStateStall)
417   // and either it executes an ecall or an error occurs. A
pulse on the done signal raises the
418   // 'done' interrupt and also tells the top-level to update
its ERR_BITS status
419   // register. The calculation that ecall triggered done is
factored out as 'done_complete' to
420   // avoid logic loops in the error handling logic.
421   assign done_complete = (insn_valid_i &
insn_dec_shared_i.ecall_insn);
422   assign executing = (state_q == OtnbStateRun) ||
423     (state_q == OtnbStateStall);
424
425   // Set the *locking* output when the next state is the
*locked* state and no secure wipe is
426   // running or there is a URND reseed error. 'locking_o' is
thus set only after the secure wipe
427   // has completed or if it cannot complete due to an URND re-
seed error (in which case
428   // 'secure_wipe_req_o' and 'urnd_reseed_err_i' will remain
high). The condition for secure wipe
429   // running involves 'secure_wipe_running_i', which is high
for the initial secure wipe, and
430   // 'secure_wipe_req_o', which is high for post-execution se-
cure wipes.
431   assign locking_o = (state_d == OtnbStateLocked) & (~(se-
cure_wipe_running_i | secure_wipe_req_o) |
432     urnd_re-
seed_err_i | mubi_err_d);
433
434   assign start_secure_wipe = executing & (done_complete |
err);
435
436   assign jump_or_branch = (insn_valid_i &
437     (insn_dec_shared_i.branch_insn |
insn_dec_shared_i.jump_insn));
438
439   // Branch taken when there is a valid branch instruction and
comparison passes or a valid jump
440   // instruction (which is always taken)
441   assign branch_taken = insn_valid_i &

```

```

442     ((insn_dec_shared_i.branch_insn &
alu_base_comparison_result_i) |
443     insn_dec_shared_i.jump_insn);
444   // Branch target computed by base ALU (PC + imm)
445   assign branch_target =
alu_base_operation_result_i[ImemAddrWidth-1:0];
446   assign branch_target_overflow = |
alu_base_operation_result_i[31:ImemAddrWidth];
447
448   assign next_insn_addr_wide = {1'b0, insn_addr_i} + 'd4;
449   assign next_insn_addr = next_in-
sn_addr_wide[ImemAddrWidth-1:0];
450
451   // Record address for fetch request so it can be retried
when an invalid response is received
452   always_ff @(posedge clk_i) begin
453     if (insn_fetch_req_valid_raw) begin
454       insn_fetch_req_addr_last <= insn_fetch_req_addr_o;
455     end
456   end
457
458   always_comb begin
459     state_d = state_q;
460     // 'insn_fetch_req_valid_raw' is the value
'insn_fetch_req_valid_o' before any errors are
461     // considered.
462     insn_fetch_req_valid_raw = 1'b0;
463     insn_fetch_req_addr_o = '0;
464     insn_fetch_resp_clear_o = 1'b1;
465     prefetch_en_o = 1'b0;
466
467     state_error = 1'b0;
468
469     unique case (state_q)
470       OtnbStateHalt: begin
471         if (start_i) begin
472           state_d = OtnbStateRun;
473
474           insn_fetch_req_addr_o = '0;
475           insn_fetch_req_valid_raw = 1'b1;
476           prefetch_en_o = 1'b1;
477         end
478       end
479       OtnbStateRun: begin
480         insn_fetch_req_valid_raw = 1'b1;
481         prefetch_en_o = 1'b1;
482
483         if (!insn_valid_i) begin
484           insn_fetch_req_addr_o = insn_fetch_req_addr_last;
485         end else if (done_complete) begin
486           state_d = OtnbStateHalt;
487           insn_fetch_req_valid_raw = 1'b0;
488           prefetch_en_o = 1'b0;
489         end else begin
490           if (stall) begin
491             // When stalling don't request a new fetch and
don't clear response either to keep
492             // current instruction.
493             state_d = OtnbStateStall;
494             insn_fetch_req_valid_raw = 1'b0;
495             insn_fetch_resp_clear_o = 1'b0;
496           end else begin
497             if (branch_taken) begin
498               insn_fetch_req_addr_o = branch_target;
499             end else if (loop_jump) begin
500               insn_fetch_req_addr_o = loop_jump_addr;
501             end else begin
502               insn_fetch_req_addr_o = next_insn_addr;
503             end
504           end
505         end
506       end
507       OtnbStateStall: begin
508         prefetch_en_o = 1'b1;
509         // When stalling refetch the same instruction to keep
decode inputs constant
510         if (stall) begin
511           state_d = OtnbStateStall;
512           //insn_fetch_req_addr_o = insn_addr_i;
513           insn_fetch_req_valid_raw = 1'b0;
514           insn_fetch_resp_clear_o = 1'b0;
515         end else begin
516           insn_fetch_req_valid_raw = 1'b1;
517
518           if (loop_jump) begin
519             insn_fetch_req_addr_o = loop_jump_addr;
520           end else begin
521             insn_fetch_req_addr_o = next_insn_addr;
522           end
523
524           state_d = OtnbStateRun;
525         end
526       end
527       OtnbStateLocked: begin
528         insn_fetch_req_valid_raw = 1'b0;
529         state_d = OtnbStateLocked;
530       end
531       default: begin
532         // We should never get here. If we do (e.g. via a ma-
licious glitch), error out immediately.
533         // SEC_CM: CONTROLLER.FSM.LOCAL_ESC

```

```

534     state_d = OtbNStateLocked;
535     state_error = 1'b1;
536   end
537 endcase
538
539 // On any error immediately halt, either going to OtbN-
540 StateLocked or OtbNStateHalt depending on
541 // whether it was a fatal error.
542 if (err) begin
543   insn_fetch_resp_clear_o = 1'b1;
544   if (fatal_err) begin
545     // SEC_CM: CONTROLLER.FSM.GLOBAL_ESC
546     state_d = OtbNStateLocked;
547   end else begin
548     state_d = OtbNStateHalt;
549   end
550 end
551
552 // Regardless of what happens above enforce staying in Ot-
553 nbStateLocked.
554 if (state_q == OtbNStateLocked) begin
555   state_d = OtbNStateLocked;
556 end
557
558 assign state_error_d = state_error | state_error_q;
559
560 prim_flop #(
561   .Width(1),
562   .ResetValue('0)
563 ) u_state_error_flop (
564   .clk_i,
565   .rst_ni,
566   .d_i(state_error_d),
567   .q_o(state_error_q)
568 );
569
570 `ASSERT(InsnAlwaysValidInStall, state_q == OtbNStateStall | -
571 > insn_valid_i)
572
573 // Anything that moves us or keeps us in the stall state
574 // should cause 'stall' to be asserted
575 `ASSERT(StallIfNextStateStall, insn_valid_i & (state_d ==
576 OtbNStateStall) |-> stall)
577
578 // The raw signal is needed by the instruction fetch stage
579 // for generating instruction address
580 // errors (where instruction fetch and prefetch disagree on
581 // address). 'err' will factor this in so
582 // using the qualified signal results in a combinational
583 // loop.
584 assign insn_fetch_req_valid_raw_o =
585 insn_fetch_req_valid_raw;
586 assign insn_fetch_req_valid_o = err ? 1'b0 :
587 insn_fetch_req_valid_raw;
588
589 // Determine if there are any errors related to the Imem
590 // fetch address.
591 always_comb begin
592   imem_addr_err = 1'b0;
593   if (insn_fetch_req_valid_raw) begin
594     if (!insn_fetch_req_addr_o[1:0]) begin
595       // Imem address is unaligned
596       imem_addr_err = 1'b1;
597     end else if (branch_taken) begin
598       imem_addr_err = branch_target_overflow;
599     end else begin
600       imem_addr_err = next_insn_addr_wide[ImemAddrWidth] &
601 insn_valid_i;
602     end
603   end
604 end
605
606 // Signal error if MuBi input signals take on invalid values
607 // as this means something bad is
608 // happening. Register the error signal to break circular
609 // paths (instruction fetch errors factor
610 // into fatal_escalate_en_i, RND errors factor into re-
611 cov_escalate_en_i).
612 assign mubi_err_d = |{mu-
613 bi4_test_invalid(fatal_escalate_en_i),
614 mu-
615 bi4_test_invalid(recov_escalate_en_i),
616 mubi_err_q};
617
618 always_ff @(posedge clk_i or negedge rst_ni) begin
619   if (!rst_ni) begin
620     mubi_err_q <= 1'b0;
621   end else begin
622     mubi_err_q <= mubi_err_d;
623   end
624 end
625
626 // Instruction is illegal based on the static properties of
627 the instruction bits (illegal encoding
628 // or illegal WSR/CSR referenced).
629 assign illegal_insn_static = insn_illegal_i | ispr_err;
630

```

```

631   assign fatal_software_err = software_err & soft-
632 ware_errs_fatal_i;
633   assign bad_internal_state_err = |{state_error_d,
634 loop_hw_err, rf_base_call_stack_hw_err_i,
635 rf_bignum_spurious_we_err, spurious_secure_wipe_ack_q,
636 mubi_err_q};
637   assign reg_intg_violation_err = rf_bignum_intg_err |
638 ispr_rdata_intg_err;
639   assign key_invalid_err = ispr_rd_bignum_insn &
640 insn_valid_i & key_invalid;
641   assign illegal_insn_err = illegal_insn_static |
642 rf_indirect_err;
643   assign call_stack_sw_err =
644 rf_base_call_stack_sw_err_i;
645
646 // Flag a bad data address error if the data memory address
647 is invalid and it does not come from
648 // an empty call stack. The second case cannot be decided
649 as bad data address because the address
650 // on top of the empty call stack may or may not be valid.
651 (Also, in most RTL simulators an empty
652
653 // call stack that has never been pushed contains an unknown
654 value, so this error bit would become
655 // unknown.) Thus, a data memory address coming from an
656 empty call stack raises a call stack
657 // error but never a bad data address error.
658   assign bad_data_addr_err = dmem_addr_err &
659 ~(call_stack_sw_err &
660 ld_in-
661 sn_with_addr_from_call_stack |
662 st_in-
663 sn_with_addr_from_call_stack));
664
665 // Identify load instructions that take the memory address
666 from the call stack.
667   assign ld_insn_with_addr_from_call_stack = insn_val-
668 id_i
669 &
670 insn_dec_shared_i.ld_insn &
671 insn_dec_base_i.rf_ren_a &
672 (insn_dec_base_i.a == 5'd1);
673
674 // Identify store instructions that take the memory address
675 from the call stack.
676   assign st_insn_with_addr_from_call_stack = insn_val-
677 id_i
678 &
679 insn_dec_shared_i.st_insn &
680 insn_dec_base_i.rf_ren_a &
681 (insn_dec_base_i.a == 5'd1);
682
683 // All software errors that aren't bad_insn_addr. Factored
684 into bad_insn_addr so it is only raised
685 // if other software errors haven't occurred. As bad_in-
686 sn_addr relates to the next instruction
687 // begin fetched it cannot occur if the current instruction
688 has seen an error and failed to
689 // execute.
690   assign non_insn_addr_software_err = |{key_invalid_err,
691 loop_sw_err,
692 illegal_insn_err,
693 call_stack_sw_err,
694 bad_data_addr_err,
695 mai_software_error_i};
696
697   assign bad_insn_addr_err = imem_addr_err & ~non_in-
698 sn_addr_software_err;
699
700   assign err_bits_d = '{
701     fatal_software_err,
702     bad_internal_state_err,
703     reg_intg_violation_err,
704     key_invalid_err,
705     loop_sw_err,
706     illegal_insn_err,
707     call_stack_sw_err,
708     bad_data_addr_err,
709     bad_insn_addr_err,
710     mai_error,
711     mai_software_error_i
712   };
713
714   always_ff @(posedge clk_i or negedge rst_ni) begin
715     if (!rst_ni) begin
716       err_bits_q <= '0;
717     end else begin
718       if (err_bit_clear_i && !locking_o) begin
719         err_bits_q <= '0;
720       end else begin
721         err_bits_q <= err_bits_q | err_bits_d;
722       end
723     end
724   end
725
726   assign err_bits_o = err_bits_q;
727

```



```

687 assign software_err = non_insn_addr_software_err | bad_in-
sn_addr_err;
688
689 assign recoverable_err = mu-
bi4_test_true_loose(recov_escalate_en_i);
690
691 assign internal_fatal_err = |{fatal_software_err,
692 bad_internal_state_err,
693 reg_intg_violation_err};
694
695 // In case of an RMA request, just lock up the controller.
This triggers the rotation of the
696 // scrambling keys. The start/stop controller takes care of
initiating the internal secure wipe
697 // and eventually acknowledging the RMA request.
698 assign fatal_err = |{internal_fatal_err,
699 mu-
bi4_test_true_loose(fatal_escalate_en_i),
700 mubi4_test_true_strict(rma_req_i)};
701
702 assign recoverable_err_o = recoverable_err | (software_err &
~software_errs_fatal_i);
703 assign mems_sec_wipe_o = (state_d == OtbNStateLocked) &
(state_q != OtbNStateLocked);
704
705 assign internal_err = software_err | internal_fatal_err;
706 assign err = software_err | recoverable_err |
fatal_err;
707
708 assign prefetch_ignore_errs_o = internal_err;
709
710 // Instructions must not execute if there is an error
711 assign insn_executing = insn_valid_i & ~err;
712
713 `ASSERT(ErrBitSetOnErr,
714 err & (mubi4_test_false_strict(fatal_escalate_en_i) &
715 mubi4_test_false_strict(recov_escalate_en_i) &
716 mubi4_test_false_loose(rma_req_i)) |>
err_bits_o)
717
718 `ASSERT(ErrSetOnFatalErr, fatal_err |> err)
719 `ASSERT(SoftwareErrIfNonInsnAddrSoftwareErr, non_in-
sn_addr_software_err |> software_err)
720
721 `ASSERT(ControllerStateValid,
722 state_q inside {OtbNStateHalt, OtbNStateRun, OtbN-
StateStall, OtbNStateLocked})
723 // Branch only takes effect in OtbNStateRun so must not go
into stall state for branch
724 // instructions.
725 `ASSERT(NoStallOnBranch,
726 insn_valid_i & insn_dec_shared_i.branch_insn |>
state_q != OtbNStateStall)
727
728 // SEC_CM: CONTROLLER.FSM.SPARSE
729 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
otbn_state_e, OtbNStateHalt)
730
731 // SEC_CM: CTRL_FLOW.COUNT
732 // Two explicit clear controls, one comes from external to
otbn_core and the other is generated
733 // internally (by otbn_start_stop_control).
734 assign insn_cnt_clear =
735 (state_q == OtbNStateLocked) | insn_cnt_clear_ext_i |
insn_cnt_clear_int_i;
736
737 always_comb begin
738 if (insn_cnt_clear) begin
739 insn_cnt_d = 32'd0;
740 end else if (insn_executing & ~stall & (insn_cnt_q !=
32'hfffffff)) begin
741 insn_cnt_d = insn_cnt_q + 32'd1;
742 end else begin
743 insn_cnt_d = insn_cnt_q;
744 end
745 end
746
747 always_ff @(posedge clk_i or negedge rst_ni) begin
748 if (!rst_ni) begin
749 insn_cnt_q <= 32'd0;
750 end else begin
751 insn_cnt_q <= insn_cnt_d;
752 end
753 end
754
755 assign insn_cnt_o = insn_cnt_q;
756
757 assign loop_reset = state_reset_i | sec_wipe_zero_i;
758
759 otbn_loop_controller #(
760 .ImemAddrWidth(ImemAddrWidth)
761 ) u_otbn_loop_controller (
762 .clk_i,
763 .rst_ni,
764
765 .state_reset_i(loop_reset),
766
767 .insn_valid_i,
768 .insn_addr_i,
769 .next_insn_addr_i(next_insn_addr),
770

```

```

771 .loop_start_req_i (loop_start_req),
772 .loop_start_commit_i (loop_start_commit),
773 .loop_bodysize_i (loop_bodysize),
774 .loop_iterations_i (loop_iterations),
775 .loop_end_addr_predec_i (ctrl_flow_target_predec_i),
776
777 .loop_jump_o (loop_jump),
778 .loop_jump_addr_o(loop_jump_addr),
779
780 .sw_err_o (loop_sw_err),
781 .hw_err_o (loop_hw_err),
782 .predec_err_o (loop_predec_err),
783
784 .jump_or_branch_i(jump_or_branch),
785 .otbn_stall_i (stall),
786
787 .prefetch_loop_active_o,
788 .prefetch_loop_iterations_o,
789 .prefetch_loop_end_addr_o,
790 .prefetch_loop_jump_addr_o
791 );
792
793 // loop_start_req indicates the instruction wishes to start
a loop, loop_start_commit confirms it
794 // should occur.
795 assign loop_start_req = insn_valid_i &
insn_dec_shared_i.loop_insn;
796 assign loop_start_commit = insn_executing;
797 assign loop_bodysize = insn_dec_base_i.loop_bodysize;
798 assign loop_iterations = insn_dec_base_i.loop_immediate ?
insn_dec_base_i.i :
799
rf_base_rd_data_a_no_intg;
800
801 // Compute increments which can be optionally applied to in-
direct register accesses and memory
802 // addresses in BN.LID/BN.SID/BN.MOVR instructions.
803 assign rf_base_rd_data_a_inc =
rf_base_rd_data_a_no_intg[4:0] + 1'b1;
804 assign rf_base_rd_data_b_inc =
rf_base_rd_data_b_no_intg[4:0] + 1'b1;
805 // We can avoid a full 32-bit adder here because the offset
is 32-bit aligned, so we know the
806 // load/store address will only be valid if
rf_base_rd_data_a_no_intg[4:0] is zero.
807 assign rf_base_rd_data_a_wlen_word_inc =
rf_base_rd_data_a_no_intg[31:5] + 27'h1;
808
809 // Choose increment to write back to base register file,
only one increment can be written as
810 // there is only one write port. Note that where an instruc-
tion is incrementing the indirect
811 // reference to its destination register
(insn_dec_bignum_i.d_inc) that reference is read on the
812 // B read port so the B increment is written back.
813 always_comb begin
814 unique case (1'b1)
815 insn_dec_bignum_i.a_inc: begin
816 increment_out = {26'b0, rf_base_rd_data_a_inc};
817 end
818 insn_dec_bignum_i.b_inc: begin
819 increment_out = {26'b0, rf_base_rd_data_b_inc};
820 end
821 insn_dec_bignum_i.d_inc: begin
822 increment_out = {26'b0, rf_base_rd_data_b_inc};
823 end
824 insn_dec_bignum_i.a_wlen_word_inc: begin
825 increment_out = {rf_base_rd_data_a_wlen_word_inc,
5'b0};
826 end
827 default: begin
828 // Whenever increment_out is written back to the re-
gister file, exactly one of the
829 // increment selector signals is high. To prevent the
automatic inference of latches in
830 // case nothing is written back (rf_wdata_sel != RfWd-
SelIncr) and to save logic, we choose
831 // a valid output as default.
832 increment_out = {26'b0, rf_base_rd_data_a_inc};
833 end
834 endcase
835 end
836
837 // Base RF read/write address, enable and commit control
always_comb begin
838 rf_base_rd_addr_a_o = insn_dec_base_i.a;
839 rf_base_rd_addr_b_o = insn_dec_base_i.b;
840 rf_base_wr_addr_o = insn_dec_base_i.d;
841
842 // Only commit read or write if the instruction is execut-
ing (in particular a read commit pops
843 // the call stack so must not occur where a valid instruc-
tion sees an error and doesn't
844 // execute).
845 rf_base_rd_commit_o = insn_executing;
846 rf_base_wr_commit_o = insn_executing;
847
848 rf_base_rd_en_a_raw = 1'b0;
849 rf_base_rd_en_b_raw = 1'b0;
850 rf_base_wr_en_raw = 1'b0;
851
852

```

```

853   if (insn_valid_i) begin
854     if (insn_dec_shared_i.st_insn) begin
855       // For stores, both base reads happen in the first
cycle of the store instruction. For base
856       // stores this is the same cycle as the request. For
bignum stores this is the cycle before
857       // the request (as the indirect register read to get
the store data occurs the following
858       // cycle).
859       rf_base_rd_en_a_raw = insn_dec_base_i.rf_ren_a &
860       (rf_indirect_stall | (insn_dec_shared_i.subset ==
InsSubsetBase));
861       rf_base_rd_en_b_raw = insn_dec_base_i.rf_ren_b &
862       (rf_indirect_stall | (insn_dec_shared_i.subset ==
InsSubsetBase));
863
864       // Bignum stores can update the base register file
where an increment is used.
865       rf_base_wr_en_raw = (insn_dec_shared_i.subset == In-
snSubsetBignum) &
866       insn_dec_base_i.rf_we
867       & rf_indirect_stall;
868     end else if (insn_dec_shared_i.ld_insn) begin

869       // For loads, both base reads happen in the same cycle
as the request. The address is
870       // required for the request and the indirect destina-
tion register (only used for Bignum
871       // loads) is flopped in ld_insn_bignum_wr_addr_q to
correctly deal with the case where it's
872       // updated by an increment.
873       rf_base_rd_en_a_raw = insn_dec_base_i.rf_ren_a &
lsu_load_req_raw;
874       rf_base_rd_en_b_raw = insn_dec_base_i.rf_ren_b &
lsu_load_req_raw;
875
876       if (insn_dec_shared_i.subset == InsSubsetBignum) be-
gin
877         // Bignum loads can update the base register file
where an increment is used. This must

878         // always happen in the same cycle as the request as
this is where both registers are
879         // read.
880         rf_base_wr_en_raw = insn_dec_base_i.rf_we &
lsu_load_req_raw & rf_indirect_stall;
881       end else begin
882         // For Base loads write the base register file when
the instruction is unstalled (meaning
883         // the load data is available).
884         rf_base_wr_en_raw = insn_dec_base_i.rf_we & ~stall;
885       end
886     end else if (insn_dec_bignum_i.rf_wdata_sel == RfwdSel-
MovSel) begin
887       // For MOVR base register reads occur in the first
cycle of the instruction. The indirect
888       // register read for the bignum data occurs in the
following cycle.
889       rf_base_rd_en_a_raw = insn_dec_base_i.rf_ren_a &
rf_indirect_stall;
890       rf_base_rd_en_b_raw = insn_dec_base_i.rf_ren_b &
rf_indirect_stall;
891       rf_base_wr_en_raw = insn_dec_base_i.rf_we &
rf_indirect_stall;
892     end else begin
893       // For all other instructions the read and write hap-
pen when the instruction is unstalled.
894       rf_base_rd_en_a_raw = insn_dec_base_i.rf_ren_a &
~stall;
895       rf_base_rd_en_b_raw = insn_dec_base_i.rf_ren_b &
~stall;
896       rf_base_wr_en_raw = insn_dec_base_i.rf_we &
~stall;
897     end
898   end
899
900   if (insn_dec_shared_i.subset == InsSubsetBignum) begin
901     unique case (i'b1)
902       insn_dec_bignum_i.a_inc,
903       insn_dec_bignum_i.a_wlen_word_inc: begin
904         rf_base_wr_addr_o = insn_dec_base_i.a;
905       end
906
907       insn_dec_bignum_i.b_inc,
908       insn_dec_bignum_i.d_inc: begin
909         rf_base_wr_addr_o = insn_dec_base_i.b;
910       end
911     default: ;
912   endcase
913 end
914
915 rf_base_rd_en_a_o = rf_base_rd_en_a_raw & ~illeg-
al_insn_static;
916 rf_base_rd_en_b_o = rf_base_rd_en_b_raw & ~illeg-
al_insn_static;
917 rf_base_wr_en_o = rf_base_wr_en_raw & ~illeg-
al_insn_static;
918 end
919
920 // Base ALU Operand A MUX

```

```

921   always_comb begin
922     unique case (insn_dec_base_i.op_a_sel)
923       OpASelRegister: alu_base_operation_o.operand_a =
rf_base_rd_data_a_no_intg;
924       OpASelZero: alu_base_operation_o.operand_a = '0;

925       OpASelCurrPc: alu_base_operation_o.operand_a = {{(32 -
ImemAddrWidth)}1'b0}}, insn_addr_i;
926     default: alu_base_operation_o.operand_a =
rf_base_rd_data_a_no_intg;
927   endcase
928 end
929
930 // Base ALU Operand B MUX
931   always_comb begin
932     unique case (insn_dec_base_i.op_b_sel)
933       OpBSelRegister: alu_base_operation_o.operand_b =
rf_base_rd_data_b_no_intg;
934       OpBSelImmediate: alu_base_operation_o.operand_b =
insn_dec_base_i.i;
935     default: alu_base_operation_o.operand_b =
rf_base_rd_data_b_no_intg;
936   endcase
937 end
938
939   assign alu_base_operation_o.op = insn_dec_base_i.alu_op;
940
941   assign alu_base_comparison_o.operand_a =
rf_base_rd_data_a_no_intg;
942   assign alu_base_comparison_o.operand_b =
rf_base_rd_data_b_no_intg;
943   assign alu_base_comparison_o.op =
insn_dec_base_i.comparison_op;
944
945   assign rf_base_rd_data_a_no_intg =
rf_base_rd_data_a_intg_i[31:0];
946   assign rf_base_rd_data_b_no_intg =
rf_base_rd_data_b_intg_i[31:0];
947
948   logic unused_rf_base_rd_a_intg_bits;
949
950   // TODO(#18266): Implement GPR to ISPR end to end integrity
path (ISPR writes from GPR take data
951   // from base RF port A)
952   assign unused_rf_base_rd_a_intg_bits = |
rf_base_rd_data_a_intg_i[38:32];
953
954   // Base register file write MUX. Depending on the data
source, integrity bits do or don't have to
955   // be appended:
956   // - Data sources that require appending integrity bits go
into `rf_base_wr_data_no_intg_o` and
957   // `rf_base_wr_data_intg_sel_o` is low.
958   // - Data sources that already come with integrity bits go
into `rf_base_wr_data_intg_o` and
959   // `rf_base_wr_data_intg_sel_o` is high.
960   always_comb begin
961     // Default values
962     rf_base_wr_data_no_intg_o = alu_base_operation_result_i;
963     rf_base_wr_data_intg_o = '0;
964     rf_base_wr_data_intg_sel_o = 1'b0;
965
966     unique case (insn_dec_base_i.rf_wdata_sel)
967       RfwdSelEx: begin
968         rf_base_wr_data_no_intg_o =
alu_base_operation_result_i;
969       end
970       RfwdSelNextPc: begin
971         rf_base_wr_data_no_intg_o = {{(32-(ImemAddrWidth+1))
{1'b0}}, next_insn_addr_wide};
972       end
973       RfwdSelIspr: begin
974         rf_base_wr_data_no_intg_o = csr_rdata;
975       end
976       RfwdSelIncr: begin
977         rf_base_wr_data_no_intg_o = increment_out;
978       end
979       RfwdSelLsu: begin
980         rf_base_wr_data_intg_sel_o = 1'b1;
981         rf_base_wr_data_intg_o = lsu_base_rdata_i;
982       end
983     default: ;
984   endcase
985 end
986
987   for (genvar i = 0; i < BaseWordsPerWLEN; ++i) begin :
g_rf_bignum_rd_data
988     assign rf_bignum_rd_data_a_no_intg[i*32+:32] =
rf_bignum_rd_data_a_intg_i[i*39+:32];
989     assign rf_bignum_rd_data_b_no_intg[i*32+:32] =
rf_bignum_rd_data_b_intg_i[i*39+:32];
990   end
991
992   // Bignum RF control signals from the controller aren't ac-
tually used, instead the predecoded

993   // one-hot versions are. The predecoded versions get checked
against the signals produced here.

994   // Buffer them to ensure they don't get optimised away (with a
functionally correct OTBN / if

```

```

995 // there is no FI attack they will always be identical).
996
997 // The rf_bignum_rd_en_stall ensures that the BN RFs data is
only read when it is actually used.
998 // For all instructions except multi-cycle multiplication we
thus must suppress the read if the
999 // core stalls. The multi-cycle multiplications however read
the data also while stalling and
1000 // thus we force the read stall signal to low. This works as
none of these instructions can
1001 // trigger any of the other stall sources (mem_stall,
ispr_stall or rf_indirect_stall). The only
1002 // possible stall reason is the MAC module itself.
1003 logic rf_bignum_rd_en_stall;
1004 assign rf_bignum_rd_en_stall = insn_dec_bignum_i.mac_en ?
1'b0 : stall;
1005
1006 assign rf_bignum_rd_addr_a_unbuf =
insn_dec_bignum_i.rf_a_indirect ? insn_bignum_rd_addr_a_q :
1007
insn_dec_bignum_i.a;
1008
1009 prim_buf #(
1010   .Width(WdrAw)
1011 ) u_rf_bignum_rd_addr_a_buf (
1012   .in_i (rf_bignum_rd_addr_a_unbuf),
1013   .out_o(rf_bignum_rd_addr_a_o)
1014 );
1015
1016 assign rf_bignum_rd_en_a_unbuf = insn_dec_bignum_i.rf_ren_a
&
1017
insn_valid_i
&
1018
~rf_bignum_rd_en_stall;
1019
1020 prim_buf #(
1021   .Width(1)
1022 ) u_rf_bignum_rd_en_a_buf (
1023   .in_i (rf_bignum_rd_en_a_unbuf),
1024   .out_o(rf_bignum_rd_en_a_o)
1025 );
1026
1027 assign rf_bignum_rd_addr_b_unbuf =
insn_dec_bignum_i.rf_b_indirect ? insn_bignum_rd_addr_b_q :
1028
insn_dec_bignum_i.b;
1029
1030 prim_buf #(
1031   .Width(WdrAw)
1032 ) u_rf_bignum_rd_addr_b_buf (
1033   .in_i (rf_bignum_rd_addr_b_unbuf),
1034   .out_o(rf_bignum_rd_addr_b_o)
1035 );
1036
1037 assign rf_bignum_rd_en_b_unbuf = insn_dec_bignum_i.rf_ren_b
&
1038
insn_valid_i
&
1039
~rf_bignum_rd_en_stall;
1040
1041 prim_buf #(
1042   .Width(1)
1043 ) u_rf_bignum_rd_en_b_buf (
1044   .in_i (rf_bignum_rd_en_b_unbuf),
1045   .out_o(rf_bignum_rd_en_b_o)
1046 );
1047
1048 assign alu_bignum_operation_o.operand_a =
rf_bignum_rd_data_a_no_intg;
1049
1050 // Bignum ALU Operand B MUX
1051 always_comb begin
1052   unique case (insn_dec_bignum_i.alu_op_b_sel)
1053     OpBSelRegister: alu_bignum_operation_o.operand_b =
rf_bignum_rd_data_b_no_intg;
1054     OpBSelImmediate: alu_bignum_operation_o.operand_b =
insn_dec_bignum_i.i;
1055     default: alu_bignum_operation_o.operand_b =
rf_bignum_rd_data_b_no_intg;
1056   endcase
1057 end
1058
1059 assign alu_bignum_operation_o.op =
insn_dec_bignum_i.alu_op;
1060 assign alu_bignum_operation_o.alu_elen =
insn_dec_bignum_i.alu_elen;
1061 assign alu_bignum_operation_o.trn_elen =
insn_dec_bignum_i.trn_elen;
1062 assign alu_bignum_operation_o.adder_carry_sel =
insn_dec_bignum_i.alu_adder_carry_sel;
1063 assign alu_bignum_operation_o.shift_right =
insn_dec_bignum_i.alu_shift_right;
1064 assign alu_bignum_operation_o.shift_amt =
insn_dec_bignum_i.alu_shift_amt;
1065 assign alu_bignum_operation_o.shift_mask =

```

```

insn_dec_bignum_i.alu_shift_mask;
1066 assign alu_bignum_operation_o.flag_group =
insn_dec_bignum_i.alu_flag_group;
1067 assign alu_bignum_operation_o.sel_flag =
insn_dec_bignum_i.alu_sel_flag;
1068 assign alu_bignum_operation_o.alu_flag_en =
insn_dec_bignum_i.alu_flag_en & insn_valid_i;
1069 assign alu_bignum_operation_o.mac_flag_en =
insn_dec_bignum_i.mac_flag_en & insn_valid_i;
1070
1071 assign alu_bignum_operation_valid_o = insn_valid_i;
1072 assign alu_bignum_operation_commit_o = insn_executing;
1073
1074 assign mac_bignum_operation_o.operand_a =
rf_bignum_rd_data_a_no_intg;
1075 assign mac_bignum_operation_o.operand_b =
rf_bignum_rd_data_b_no_intg;
1076 assign mac_bignum_operation_o.op_a_qw_sel_raw =
insn_dec_bignum_i.mac_op_a_qw_sel_raw;
1077 assign mac_bignum_operation_o.op_b_elem0_sel_raw =
insn_dec_bignum_i.mac_op_b_elem0_sel_raw;
1078 assign mac_bignum_operation_o.op_b_elem1_sel_raw =
insn_dec_bignum_i.mac_op_b_elem1_sel_raw;
1079 assign mac_bignum_operation_o.wr_hw_sel_upper =
insn_dec_bignum_i.mac_wr_hw_sel_upper;
1080 assign mac_bignum_operation_o.pre_acc_shift_imm =
insn_dec_bignum_i.mac_pre_acc_shift;
1081 assign mac_bignum_operation_o.acc_add_en =
insn_dec_bignum_i.mac_acc_add_en;
1082 assign mac_bignum_operation_o.shift_acc =
insn_dec_bignum_i.mac_shift_out;
1083 assign mac_bignum_operation_o.is_vec =
insn_dec_bignum_i.mac_is_vec;
1084 assign mac_bignum_operation_o.is_mod =
insn_dec_bignum_i.mac_is_mod;
1085 assign mac_bignum_operation_o.is_lane =
insn_dec_bignum_i.mac_is_lane;
1086 assign mac_bignum_operation_o.lane_index =
insn_dec_bignum_i.mac_lane_index;
1087 assign mac_bignum_operation_o.elen =
insn_dec_bignum_i.mac_elen;
1088 assign mac_bignum_operation_o.adder_carry_sel =
insn_dec_bignum_i.mac_adder_carry_sel;
1089
1090 assign mac_bignum_en_o = insn_valid_i &
insn_dec_bignum_i.mac_en;
1091 assign mac_bignum_commit_o = insn_executing;
1092
1093 // Move / Conditional Select. Only select B register data
when a selection instruction is being
1094 // executed and the selection flag isn't set. To avoid un-
desirable SCA leakage between the two
1095 // registers for non-selection instructions, the B register
is blanked except for selection
1096 // instructions.
1097 // Note that blanking both registers is not feasible nor ab-
solutely required because:
1098 // - The flag group selection and flag selection are known
in the predecoder stage but the actual
1099 // flag isn't.
1100 // - Selecting the flag in the predecoder stage using com-
binatorial inputs may lead to SCA leakage
1101 // between the still combinatorial flag groups and flags
within a group which might be
1102 // undesirable as well.
1103
1104 // - When executing a selection instruction, programmers can
expected that there will be some SCA
1105 // leakage between the two options. But it may be much
less expected for such leakage to occur
1106 // for other instructions.
1107 `ASSERT(SelFlagValid, insn_valid_i &
insn_dec_bignum_i.sel_insn |>
1108   insn_dec_bignum_i.alu_sel_flag inside {FlagC, FlagL,
FlagM, FlagZ})
1109
1110 // SEC_CM: DATA_REG_SW.SCA
1111 prim_blanker #(Width(ExtWLEN))
u_rf_bignum_rd_data_b_intg_blanker (
1112   .in_i (rf_bignum_rd_data_b_intg_i),
1113   .en_i (ctrl_flow_predec_i.sel_insn),
1114   .out_o(rf_bignum_rd_data_b_intg_blanked)
1115 );
1116
1117 `ASSERT(BlankingBignumRdDataBSel,
~(insn_valid_i & insn_dec_bignum_i.sel_insn) |>
rf_bignum_rd_data_b_intg_blanked == '0,
1118   clk_i, !rst_ni || ctrl_flow_predec_error || !insn_executing)
1119
1120 assign selection_result =
1121   ~ctrl_flow_predec_i.sel_insn |
alu_bignum_selection_flag_i ? rf_bignum_rd_data_a_intg_i :
1122
rf_bignum_rd_data_b_intg_blanked;
1123
1124 // Bignum Register file write control
1125
1126 always_comb begin
1127   // By default write nothing
1128   rf_bignum_wr_en_unbuf = 2'b00;

```



```

1129
1130 // Only write if valid instruction wants a bignum rf write
and it isn't stalled. If instruction
1131 // doesn't execute (e.g. due to an error detected in the
previous cycle) the write won't
1132 // commit. The write enable is also high during the execu-
tion cycles of multi-cycle
1133 // multiplications. However, the destination register may
not be updated until the instruction
1134 // finishes to avoid overwriting a source register (if
source = destination). As the OTBN
1135 // stalls during these cycles, the write commit signal is
not set and the update will only be
1136 // committed in the last cycle where the OTBN un-stalls.
1137 if (insn_valid_i && insn_dec_bignum_i.rf_we && !rf_indir-
ect_stall) begin
1138   if (insn_dec_bignum_i.mac_en &&
insn_dec_bignum_i.mac_shift_out &&
1139     !insn_dec_bignum_i.mac_is_vec) begin
1140     // Special handling for BN.MULQACC.S0, only enable up-
per or lower half depending on
1141     // mac_wr_hw_sel_upper.
1142     rf_bignum_wr_en_unbuf =
insn_dec_bignum_i.mac_wr_hw_sel_upper ? 2'b10 : 2'b01;
1143   end else begin
1144     // For everything else write both halves immediately.
1145     rf_bignum_wr_en_unbuf = 2'b11;
1146   end
1147 end
1148 end
1149
1150 // Bignum RF control signals from the controller aren't ac-
tually used, instead the predecoded
1151 // one-hot versions are. The predecoded versions get checked
against the signals produced here.
1152 // Buffer them to ensure they don't get optimised away (with
a functionally correct OTBN they will
1153 // always be identical).
1154 prim_buf #(
1155   .width(2)
1156 ) u_bignum_wr_en_buf (
1157   .in_i (rf_bignum_wr_en_unbuf),
1158   .out_o (rf_bignum_wr_en_o)
1159 );
1160
1161 assign rf_bignum_wr_commit_o = |rf_bignum_wr_en_o & insn_ex-
ecuting & !stall;
1162
1163 assign rf_bignum_indirect_en_o = insn_executing & rf_in-
direct_stall;
1164 assign rf_bignum_rd_a_indirect_en = insn_executing &
insn_dec_bignum_i.rf_a_indirect;
1165 assign rf_bignum_rd_b_indirect_en = insn_executing &
insn_dec_bignum_i.rf_b_indirect;
1166 assign rf_bignum_wr_indirect_en = insn_executing &
insn_dec_bignum_i.rf_d_indirect;
1167
1168 prim_onehot_enc #(
1169   .OneHotWidth(NWdr)
1170 ) rf_bignum_rd_a_indirect_onehot_enc (
1171   .in_i (rf_base_rd_data_a_no_intg[4:0]),
1172   .en_i (rf_bignum_rd_a_indirect_en),
1173   .out_o (rf_bignum_rd_a_indirect_onehot_o)
1174 );
1175
1176 prim_onehot_enc #(
1177   .OneHotWidth(NWdr)
1178 ) rf_bignum_rd_b_indirect_onehot_enc (
1179   .in_i (rf_base_rd_data_b_no_intg[4:0]),
1180   .en_i (rf_bignum_rd_b_indirect_en),
1181   .out_o (rf_bignum_rd_b_indirect_onehot_o)
1182 );
1183
1184 prim_onehot_enc #(
1185   .OneHotWidth(NWdr)
1186 ) rf_bignum_wr_indirect_onehot_enc (
1187   .in_i (rf_base_rd_data_b_no_intg[4:0]),
1188   .en_i (rf_bignum_wr_indirect_en),
1189   .out_o (rf_bignum_wr_indirect_onehot_o)
1190 );
1191
1192
1193 // For BN.LID sample the indirect destination register index
in first cycle as an increment might
1194 // change it for the second cycle where the load data is
written to the bignum register file.
1195 always_ff @(posedge clk_i) begin
1196   if (insn_dec_bignum_i.rf_d_indirect) begin
1197     insn_bignum_wr_addr_q <= rf_base_rd_data_b_no_intg[4:0];
1198   end
1199
1200   if (insn_dec_bignum_i.rf_a_indirect) begin
1201     insn_bignum_rd_addr_a_q <=
rf_base_rd_data_a_no_intg[4:0];
1202   end
1203
1204   if (insn_dec_bignum_i.rf_b_indirect) begin

```

```

1205     insn_bignum_rd_addr_b_q <=
rf_base_rd_data_b_no_intg[4:0];
1206   end
1207 end
1208
1209 // Bignum RF control signals from the controller aren't ac-
tually used, instead the predecoded
1210 // one-hot versions are. The predecoded versions get checked
against the signals produced here.
1211 // Buffer them to ensure they don't get optimised away (with
a functionally correct OTBN they will
1212 // always be identical).
1213 assign rf_bignum_wr_addr_unbuf =
insn_dec_bignum_i.rf_d_indirect ? insn_bignum_wr_addr_q :
1214 insn_dec_bignum_i.d;
1215
1216 prim_buf #(
1217   .width(WdrAw)
1218 ) u_rf_bignum_wr_addr_buf (
1219   .in_i (rf_bignum_wr_addr_unbuf),
1220   .out_o (rf_bignum_wr_addr_o)
1221 );
1222
1223 // For the shift-out variant of BN.MULQACC the bottom half
of the MAC result is written to one
1224 // half of a desatination register specified by the instruc-
tion (mac_wr_hw_sel_upper). The bottom
1225 // half of the MAC result must be placed in the appropriate
half of the write data (the RF only
1226 // accepts write data for the top half in the top half of
the write data input). Otherwise
1227 // (shift-out to bottom half and all other BN.MULQACC in-
structions) simply pass the MAC result
1228 // through unchanged as write data.
1229 assign mac_bignum_rf_wr_data[WLEN-1:WLEN/2] =
1230   (insn_dec_bignum_i.mac_wr_hw_sel_upper &&
1231     insn_dec_bignum_i.mac_shift_out &&
1232     !insn_dec_bignum_i.mac_is_vec) ?
mac_bignum_operation_result_i[WLEN/2-1:0] :
1233   mac_bignum_operation_result_i[WLEN-1:WLEN/2];
1234
1235 assign mac_bignum_rf_wr_data[WLEN/2-1:0] =
mac_bignum_operation_result_i[WLEN/2-1:0];
1236
1237 // Bignum register file write MUX. Depending on the data
source, integrity bits do or don't have
1238 // to be appended; see comments on the "Base register file
write MUX" for details.
1239 always_comb begin
1240   // Default values
1241   rf_bignum_wr_data_intg_sel_o = 1'b0;
1242   rf_bignum_wr_data_intg_o = '0;
1243   rf_bignum_wr_data_no_intg_o =
alu_bignum_operation_result_i;
1244
1245   unique case (insn_dec_bignum_i.rf_wdata_sel)
1246     RfwdSelEx: begin
1247       rf_bignum_wr_data_no_intg_o =
alu_bignum_operation_result_i;
1248     end
1249     RfwdSelMac: begin
1250       rf_bignum_wr_data_no_intg_o = mac_bignum_rf_wr_data;
1251     end
1252     RfwdSelIspr: begin
1253       rf_bignum_wr_data_intg_sel_o = 1'b1;
1254       rf_bignum_wr_data_intg_o = ispr_rdata_intg_i;
1255     end
1256     RfwdSelMovSel: begin
1257       rf_bignum_wr_data_intg_sel_o = 1'b1;
1258       rf_bignum_wr_data_intg_o = selection_result;
1259     end
1260     RfwdSelLsu: begin
1261       rf_bignum_wr_data_intg_sel_o = 1'b1;
1262       //SEC_CM: BUS INTEGRITY
1263       rf_bignum_wr_data_intg_o = lsu_bignum_rdata_i;
1264     end
1265     default: ;
1266   endcase
1267 end
1268
1269 assign rf_a_indirect_err =
insn_dec_bignum_i.rf_a_indirect &
1270   (!
rf_base_rd_data_a_no_intg[31:5]) &
1271   ~rf_base_call_stack_sw_err_i &
1272   rf_base_rd_en_a_o;
1273
1274 assign rf_b_indirect_err =
insn_dec_bignum_i.rf_b_indirect &
1275   (!
rf_base_rd_data_b_no_intg[31:5]) &
1276   ~rf_base_call_stack_sw_err_i &
1277   rf_base_rd_en_b_o;
1278

```

```

1279 assign rf_d_indirect_err =
insn_dec_bignum_i.rf_d_indirect &
1280 rf_base_rd_data_b_no_intg[31:5]) &
1281 ~rf_base_call_stack_sw_err_i &
1282 rf_base_rd_en_b_o;
1283
1284 assign rf_indirect_err =
1285 insn_valid_i & (rf_a_indirect_err | rf_b_indirect_err |
rf_d_indirect_err);
1286
1287 // If the source registers are indirectly indexed and there
is a stack error, the source
1288 // register indices were illegal due to a stack pop error.
In this case, ignore bignum RF read
1289 // integrity errors.
1290 assign ignore_rf_bignum_intg_errs =
1291 (insn_dec_bignum_i.rf_a_indirect |
1292 insn_dec_bignum_i.rf_b_indirect) &
1293 rf_base_call_stack_sw_err_i;
1294
1295 assign rf_bignum_intg_err = rf_bignum_intg_err_i & ~ig-
nore_rf_bignum_intg_errs;
1296
1297 // If the destination register is indirectly indexed and
there is a stack error, the destination
1298 // register index was illegal due to a stack pop error. In
this case, ignore bignum RF
1299 // write-enable errors.
1300 assign ignore_rf_bignum_spurious_we_errs =
insn_dec_bignum_i.rf_d_indirect &
1301 rf_base_call_stack_sw_err_i;
1302
1303 assign rf_bignum_spurious_we_err =
rf_bignum_spurious_we_err_i &
1304 ~ig-
nore_rf_bignum_spurious_we_errs;
1305
1306 // CSR/WSR/ISPR handling
1307 // ISPRs (Internal Special Purpose Registers) are the in-
ternal registers. CSRs and WSRs are the
1308 // ISA visible versions of those registers in the base and
bignum ISAs respectively.
1309
1310 assign csr_addr = csr_e'(insn_dec_base_i.i[11:0]);
1311 assign csr_sub_addr =
insn_dec_base_i.i[$clog2(BaseWordsPerWLEN)-1:0];
1312
1313 always_comb begin
1314 ispr_addr_base = IsprMod;
1315 ispr_word_addr_base = '0;
1316 csr_illegal_addr = '1'b0;
1317
1318 unique case (csr_addr)
1319 CsrFlags, CsrFg0, CsrFg1: begin
1320 ispr_addr_base = IsprFlags;
1321 ispr_word_addr_base = '0;
1322 end
1323 CsrMod0, CsrMod1, CsrMod2, CsrMod3, CsrMod4, CsrMod5,
CsrMod6, CsrMod7: begin
1324 ispr_addr_base = IsprMod;
1325 ispr_word_addr_base = csr_sub_addr;
1326 end
1327 CsrRndPrefetch: begin
1328 // Reading from RND_PREFETCH results in 0, there is no
ISPR to read so no address is set.
1329 // The csr_rdata mux logic takes care of producing the
0.
1330 end
1331 CsrRnd: begin
1332 ispr_addr_base = IsprRnd;
1333 ispr_word_addr_base = '0;
1334 end
1335 CsrUrnd: begin
1336 ispr_addr_base = IsprUrnd;
1337 ispr_word_addr_base = '0;
1338 end
1339 CsrUrndCtrl: begin
1340 ispr_addr_base = IsprUrndCtrl;
1341 ispr_word_addr_base = '0;
1342 end
1343 CsrUrndStatus: begin
1344 ispr_addr_base = IsprUrndStatus;
1345 ispr_word_addr_base = '0;
1346 end
1347 CsrKmacStatus: begin
1348 ispr_addr_base = IsprKmacStatus;
1349 ispr_word_addr_base = '0;
1350 end
1351 CsrKmacCtrl: begin
1352 ispr_addr_base = IsprKmacCtrl;
1353 ispr_word_addr_base = '0;
1354 end
1355 CsrKmacCfg: begin

```

```

1356 ispr_addr_base = IsprKmacCfg;
1357 ispr_word_addr_base = '0;
1358 end
1359 CsrKmacStrb: begin
1360 ispr_addr_base = IsprKmacStrb;
1361 ispr_word_addr_base = '0;
1362 end
1363 CsrMaiCtrl: begin
1364 ispr_addr_base = IsprMaiCtrl;
1365 ispr_word_addr_base = '0;
1366 end
1367 CsrMaiStatus: begin
1368 ispr_addr_base = IsprMaiStatus;
1369 ispr_word_addr_base = '0;
1370 end
1371 CsrInsCnt: begin
1372 ispr_addr_base = IsprInsCnt;
1373 ispr_word_addr_base = '0;
1374 end
1375 default: csr_illegal_addr = '1'b1;
1376 endcase
1377 end
1378
1379 for (genvar i_word = 0; i_word < BaseWordsPerWLEN; i_word++)
begin : g_ispr_word_sel_base
1380 assign ispr_word_sel_base[i_word] = ispr_word_addr_base ==
i_word;
1381 end
1382
1383 // Decode wide ISPR read data.
1384 logic [WLEN-1:0] ispr_rdata;
1385 logic [2*BaseWordsPerWLEN-1:0] ispr_rdata_intg_err_wide;
1386 logic [BaseWordsPerWLEN-1:0] ispr_rdata_intg_err_narrow;
1387
1388 for (genvar i_word = 0; i_word < BaseWordsPerWLEN; i_word++)
begin : g_ispr_rdata_dec
1389 prim_secdec_inv_39_32_dec i_secdec_dec (
1390 .data_i (ispr_rdata_intg_i[i_word*39+:39]),
1391 .data_o (* unused because we abort on any integrity
error */),
1392 .syndrome_o (* unused */),
1393 .err_o (ispr_rdata_intg_err_wide[i_word*2+:2])
1394 );
1395 assign ispr_rdata[i_word*32+:32] =
ispr_rdata_intg_i[i_word*39+:32];
1396 assign ispr_rdata_intg_err_narrow[i_word] = |
(ispr_rdata_intg_err_wide[i_word*2+:2]);
1397 end
1398 // Propagate integrity error only if wide ISPR is used.
1399
1400 // Handle ISPR integrity error detection. We've got a bit-
mask of ISPR words that failed their
1401 // integrity check (ispr_rdata_intg_err_narrow), but a
nonzero entry may not be a problem if we
1402 // don't actually use the data.
1403 //
1404 // The situations when the data is actually used are:
1405 //
1406 // (1) This is a bignum instruction that writes back to
the bignum register file by reading an
1407 // ISPR. In this case, we actually pass the data
through with integrity bits, but it
1408 // shouldn't hurt to add fault detection at this
point.
1409 //
1410 // (2) This instruction consumes the data by selecting a
word from an ISPR and then writing it
1411 // back. This happens for things like CSRRS instruc-
tions, where the data flows to the base
1412 // register file through rf_base_wr_data_no_intg_o and
back to the ISPR through
1413 // ispr_base_wdata_o. The word used is given by the
onehot ispr_word_sel_base mask.
1414 //
1415 // In both cases, there's a special case for the
RND_PREFETCH register, which doesn't actually
1416 // have any backing data. It reads as zero with invalid in-
tegrity bits which we want to ignore.
1417 //
1418 // Are we reading all the ISPR data? (case (1) above)
1419 logic all_ispr_words_used;
1420 assign all_ispr_words_used = (insn_dec_bignum_i.rf_wdata_sel
== RfwdSelIspr);
1421
1422 // Are we reading just one word of the ISPR data? (case (2)
above).
1423 logic one_ispr_word_used;
1424 assign one_ispr_word_used = ispr_rd_insn &
(insn_dec_shared_i.subset == InsnSubsetBase);
1425
1426 // A bit-mask giving which ISPR words are being read
1427 logic [BaseWordsPerWLEN-1:0] ispr_read_mask;
1428 assign ispr_read_mask = all_ispr_words_used ? '1 :
one_ispr_word_used ?
1429 ispr_word_sel_base : '0;

```

```

1430
1431 // Use ispr_read_mask to qualify the error bit-mask that
came out of the integrity decoder.
1432 logic [BaseWordsPerWLEN-1:0] ispr_rdata_used_intg_err;
1433 assign ispr_rdata_used_intg_err = ispr_read_mask &
ispr_rdata_intg_err_narrow;
1434
1435 // We only architecturally read the ISPR when there's a non-
stalled instruction. This is also the
1436 // place where we factor in the special RND_PREFETCH beha-
viour. We also need to squash any
1437 // integrity errors if we're reading a sideload key which
isn't currently valid (this will
1438 // generate a key_invalid error, but we shouldn't have any
behaviour that depends on what happens
1439 // to be on the pins)
1440 logic non_prefetch_insn_running;
1441 assign non_prefetch_insn_running = (insn_valid_i & ~stall &
1442                                     (csr_addr != CsrRnd-
Prefetch) & ~key_invalid);
1443
1444 assign ispr_rdata_intg_err = non_prefetch_insn_running & |
(ispr_rdata_used_intg_err);
1445
1446 `ASSERT_KNOWN(IsprRdataIntgErrKnown_A, ispr_rdata_intg_err)
1447
1448 for (genvar i_bit = 0; i_bit < 32; i_bit++) begin :
g_csr_rdata_mux
1449   for (genvar i_word = 0; i_word < BaseWordsPerWLEN; i_word+
+) begin : g_csr_rdata_mux_inner
1450     assign csr_rdata_mux[i_bit][i_word] =
1451       ispr_rdata[i_word*32 + i_bit] &
ispr_word_sel_base[i_word];
1452   end
1453
1454   assign csr_rdata_raw[i_bit] = |csr_rdata_mux[i_bit];
1455 end
1456
1457 // Specialised read data handling for CSR reads where raw
read data needs modification.
1458 always_comb begin
1459   csr_rdata = csr_rdata_raw;
1460
1461   unique case (csr_addr)
1462     // For FG0/FG1 select out appropriate bits from FLAGS
ISPR and pad the rest with zeros.
1463     CsrFg0:   csr_rdata = {28'b0, csr_rdata_raw[3:0]};
1464     CsrFg1:   csr_rdata = {28'b0, csr_rdata_raw[7:4]};
1465     CsrRndPrefetch: csr_rdata = '0;
1466     default: ;
1467   endcase
1468 end
1469
1470 assign csr_wdata_raw = insn_dec_shared_i.ispr_rs_insn ?
csr_rdata | rf_base_rd_data_a_no_intg :
1471 rf_base_rd_data_a_no_intg;
1472
1473 // Specialised write data handling for CSR writes where raw
write data needs modification.
1474 always_comb begin
1475   csr_wdata = csr_wdata_raw;
1476
1477   unique case (csr_addr)
1478     // For FG0/FG1 only modify relevant part of FLAGS ISPR.
1479     CsrFg0:   csr_wdata = {24'b0, csr_rdata_raw[7:4],
csr_wdata_raw[3:0]};
1480     CsrFg1:   csr_wdata = {24'b0, csr_wdata_raw[3:0],
csr_rdata_raw[3:0]};
1481     default: ;
1482   endcase
1483 end
1484
1485 // ISPR RS (read and set) must not be combined with ISPR RD
or WR (read or write). ISPR RD and
1486 // WR (read and write) is allowed.
1487 `ASSERT(NoIsprRorWAndRs, insn_valid_i |->
~(insn_dec_shared_i.ispr_rs_insn &
1488   (insn_dec_shared_i.ispr_rd_insn |
1489     insn_dec_shared_i.ispr_wr_insn)))
1490
1491
1492 assign wsr_addr =
wsr_e'(insn_dec_bignum_i.i[WsrNumWidth-1:0]);
1493
1494 always_comb begin
1495   ispr_addr_bignum = IsprMod;
1496   wsr_illegal_addr = 1'b0;
1497   key_invalid = 1'b0;
1498
1499   unique case (wsr_addr)
1500     WsrMod:   ispr_addr_bignum = IsprMod;
1501     WsrRnd:   ispr_addr_bignum = IsprRnd;
1502     WsrUrnd:  ispr_addr_bignum = IsprUrnd;
1503     WsrAcc:   ispr_addr_bignum = IsprAcc;
1504     WsrKeyS0L: begin
1505       ispr_addr_bignum = IsprKeyS0L;
1506       key_invalid = ~sideload_key_shares_valid_i[0];
1507     end
1508     WsrKeyS0H: begin
1509       ispr_addr_bignum = IsprKeyS0H;
1510       key_invalid = ~sideload_key_shares_valid_i[0];
1511     end
1512     WsrKeyS1L: begin
1513       ispr_addr_bignum = IsprKeyS1L;
1514       key_invalid = ~sideload_key_shares_valid_i[1];
1515     end
1516     WsrKeyS1H: begin
1517       ispr_addr_bignum = IsprKeyS1H;
1518       key_invalid = ~sideload_key_shares_valid_i[1];
1519     end
1520     WsrMaiResS0:   ispr_addr_bignum = IsprMaiResS0;
1521     WsrMaiResS1:   ispr_addr_bignum = IsprMaiResS1;
1522     WsrMaiIn0S0:   ispr_addr_bignum = IsprMaiIn0S0;
1523     WsrMaiIn0S1:   ispr_addr_bignum = IsprMaiIn0S1;
1524     WsrMaiIn1S0:   ispr_addr_bignum = IsprMaiIn1S0;
1525     WsrMaiIn1S1:   ispr_addr_bignum = IsprMaiIn1S1;
1526     WsrKmacDataS0: ispr_addr_bignum = IsprKmacDataS0;
1527     WsrKmacDataS1: ispr_addr_bignum = IsprKmacDataS1;
1528     WsrUrndState:  ispr_addr_bignum = IsprUrndState;
1529     default: wsr_illegal_addr = 1'b1;
1530   endcase
1531 end
1532
1533 assign wsr_wdata = insn_dec_shared_i.ispr_rs_insn ?
ispr_rdata | rf_bignum_rd_data_a_no_intg :
1534 rf_bignum_rd_data_a_no_intg;
1535
1536 assign ispr_illegal_addr = insn_dec_shared_i.subset == Insn-
SubsetBase ? csr_illegal_addr :
1537 wsr_illegal_addr;
1538
1539 assign ispr_err = ispr_illegal_addr & insn_valid_i &
(insn_dec_shared_i.ispr_rd_insn |
1540   insn_dec_shared_i.ispr_wr_insn |
1541   insn_dec_shared_i.ispr_rs_insn);
1542
1543 assign ispr_wr_insn = insn_dec_shared_i.ispr_wr_insn |
insn_dec_shared_i.ispr_rs_insn;
1544 assign ispr_rd_insn = insn_dec_shared_i.ispr_rd_insn |
insn_dec_shared_i.ispr_rs_insn;
1545
1546 assign ispr_flags_wr_o = insn_dec_shared_i.ispr_flags_wr;
1547
1548 // Write to RND_PREFETCH must not produce ISR write
1549 assign ispr_wr_base_insn =
1550   ispr_wr_insn & (insn_dec_shared_i.subset == InsnSubset-
Base) & (csr_addr != CsrRndPrefetch);
1551
1552 assign ispr_wr_bignum_insn = ispr_wr_insn &
(insn_dec_shared_i.subset == InsnSubsetBignum);
1553 assign ispr_rd_bignum_insn = ispr_rd_insn &
(insn_dec_shared_i.subset == InsnSubsetBignum);
1554
1555 assign ispr_addr_o = insn_dec_shared_i.subset == In-
snSubsetBase ? ispr_addr_base :
1556   ispr_addr_bignum;
1557 assign ispr_base_wdata_o = csr_wdata;
1558 assign ispr_base_wr_en_o = {BaseWordsPer-
WLEN(ispr_wr_base_insn & insn_valid_i)} &
1559   ispr_word_sel_base;
1560
1561 for (genvar i_word = 0; i_word < BaseWordsPerWLEN; i_word++)
begin : g_ispr_bignum_wdata_enc
1562   prim_secded_inv_39_32_enc_i.secded_enc (
1563     .data_i(wsr_wdata[i_word*32+:32]),
1564     .data_o(ispr_bignum_wdata_intg_o[i_word*39+:39])
1565   );
1566 end
1567 assign ispr_bignum_wr_en_o = ispr_wr_bignum_insn & insn_val-
id_i;
1568
1569 assign ispr_wr_commit_o = ispr_wr_insn & insn_executing;
1570 assign ispr_rd_en_o = ispr_rd_insn & insn_valid_i &
1571   ~(insn_dec_shared_i.subset == InsnSubsetBase) & (csr_addr
== CsrRndPrefetch));
1572
1573 // For BN.SID the LSU address is computed in the first cycle
by the base ALU. The store request
1574 // itself occurs in the second cycle when the store data is
available (from the indirect register
1575 // read). The calculated address is saved in a flop here so
it's available for use in the second
1576 // cycle.
1577 assign lsu_addr_saved_d =
alu_base_operation_result_i[DmemAddrwidth-1:0];
1578 always_ff @(posedge clk_i) begin
1579   lsu_addr_saved_q <= lsu_addr_saved_d;
1580 end
1581

```

```

1582 //assign expected_lsu_addr_en_predec = insn_valid &
insn_dec_shared_i.ld_insn
1583
1584 // lsu_load_req_raw/lsu_store_req_raw indicate an instruc-
tion wishes to perform a store or a load.
1585 // lsu_load_req_o/lsu_store_req_o factor in whether an in-
struction is actually executing (it may

1586 // be suppressed due an error) and command the load or store
to happen when asserted.
1587 assign lsu_load_req_raw = insn_valid_i &
insn_dec_shared_i.ld_insn & (state_q == 0tbnStateRun);
1588 assign lsu_load_req_o = insn_executing & lsu_load_req_raw;
1589
1590 assign lsu_store_req_raw = insn_valid_i &
insn_dec_shared_i.st_insn & ~rf_indirect_stall;
1591 assign lsu_store_req_o = insn_executing &
lsu_store_req_raw;
1592
1593 assign lsu_req_subset_o = insn_dec_shared_i.subset;
1594
1595 // To simplify blanking logic all two cycle memory opera-
tions (BN.LID, BN.SID, LW) present the
1596 // calculated address in their first cycle and the saved ad-
dress in the second cycle. This results
1597 // in lsu_addr_o remaining stable for the entire instruc-
tion. Only SW is a single cycle
1598 // instruction so it only presents the calculated address.
The stability property is checked by an
1599 // assertion.
1600 assign lsu_addr_saved_sel =
1601   insn_valid_i & ((insn_dec_shared_i.subset == InsnSubset-
Bignum) ||
1602   insn_dec_shared_i.ld_insn
1603   ? ~stall : 1'b0);
1604 assign lsu_addr = lsu_addr_saved_sel ? lsu_ad-
dr_saved_q
1605   :
1606   alu_base_operation_result_i[DmemAddrWidth-1:0];
1607 // SEC_CM: CTRL.REDUN
1608 assign expected_lsu_addr_en =
1609   insn_valid_i & (insn_dec_shared_i.ld_insn |
insn_dec_shared_i.st_insn);
1610
1611 assign lsu_predec_error = expected_lsu_addr_en != lsu_ad-
dr_en_predec_i;
1612
1613 assign expected_call_stack_push =
1614   insn_valid_i & insn_dec_base_i.rf_we & rf_base_wr_addr_o
== 5'd1;
1615
1616 assign expected_call_stack_pop = insn_valid_i &
1617   ((insn_dec_base_i.rf_ren_a
& rf_base_rd_addr_a_o == 5'd1) |
1618   (insn_dec_base_i.rf_ren_b
& rf_base_rd_addr_b_o == 5'd1));
1619
1620 // Check branch target against the precalculated target from
pre-decode. Pre-decode cannot
1621 // calculate the jump target of a JALR as it requires a re-
gister read so this is excluded from the
1622 // check (by looking at the ALU op a selection).
1623 assign branch_target_predec_error =
1624   insn_dec_shared_i.branch_insn
&
1625   insn_dec_shared_i.jump_insn & insn_dec_base_i.op_a_sel !=
OpASelRegister &
1626   (ctrl_flow_target_predec_i != branch_target);
1627
1628 assign ctrl_predec_error =
1629   |(ctrl_flow_predec_i.jump_insn
1630   !=
(insn_dec_shared_i.jump_insn & insn_valid_i),

```

```

1630   ctrl_flow_predec_i.loop_insn
1631   !=
(insn_dec_shared_i.loop_insn & insn_valid_i),
1632   ctrl_flow_predec_i.branch_insn
1633   !=
(insn_dec_shared_i.branch_insn & insn_valid_i),
1634   ctrl_flow_predec_i.sel_insn
1635   !=
(insn_dec_bignum_i.sel_insn & insn_valid_i),
1636   ctrl_flow_predec_i.call_stack_push != expec-
ted_call_stack_push,
1637   ctrl_flow_predec_i.call_stack_pop != expec-
ted_call_stack_pop,
1638   branch_target_predec_error,
1639   loop_predec_err};
1640
1641 assign predec_error_o = lsu_predec_error | ctrl_pre-
dec_error;
1642
1643 // SEC_CM: DATA.REG.SW.SCA
1644 prim_blanker #(.Width(DmemAddrWidth)) u_lsu_addr_blanker (
1645   .in_i (lsu_addr),
1646   .en_i (lsu_addr_en_predec_i),
1647   .out_o(lsu_addr_blanked)
1648 );
1649
1650 // Check stability property described above (see the lsu_ad-
dr_saved_sel signal) holds.
1651 `ASSERT(LsuAddrBlankedStable_A, insn_valid_i & stall & ~err
|=> $stable(lsu_addr_blanked))
1652
1653 assign lsu_addr_o = lsu_addr_blanked;
1654
1655 assign lsu_base_wdata_o = rf_base_rd_data_b_intg_i;
1656 assign lsu_bignum_wdata_o = rf_bignum_rd_data_b_intg_i;
1657
1658 assign dmem_addr_unaligned_bignum =
1659   (lsu_req_subset_o == InsnSubsetBignum) & (lsu_ad-
dr_o[$clog2(WLEN/8)-1:0]);
1660 assign dmem_addr_unaligned_base =
1661   (lsu_req_subset_o == InsnSubsetBase) & (lsu_ad-
dr_o[1:0]);
1662 assign dmem_addr_overflow = |
alu_base_operation_result_i[31:DmemAddrWidth];
1663
1664 // A dmem address is checked the cycle it is available. For
bignum stores this is the first cycle
1665 // where the base register file read occurs, with the store
request occurring the following cycle.
1666 // For all other loads and stores the dmem address is avail-
able the same cycle as the request.
1667 assign dmem_addr_err_check =
1668   (lsu_req_subset_o == InsnSubsetBignum) &
1669   insn_dec_shared_i.st_insn
1670   ? rf_indir-
ect_stall :
1671   lsu_load_req_raw
| lsu_store_req_raw;
1672
1673 assign dmem_addr_err =
1674   insn_valid_i & dmem_addr_err_check & (dmem_ad-
dr_overflow
1675   |
dmem_ad-
dr_unaligned_bignum |
1676   dmem_ad-
dr_unaligned_base);
1677
1678 assign rnd_req_raw = insn_valid_i & ispr_rd_insn & (ispr_ad-
dr_o == IsprRnd);
1679 // Don't factor rnd_rep/fips_err_i into rnd_req_o. This
would lead to a combo loop.
1680 assign rnd_req_o = rnd_req_raw & insn_valid_i & ~(soft-
ware_err | fatal_err);
1681
1682 assign rnd_prefetch_req_o = insn_executing & ispr_wr_insn &
1683   (insn_dec_shared_i.subset == InsnSubsetBase) & (csr_addr
== CsrRndPrefetch);
1684 endmodule

```

SPI 호스트

What this is. An SPI master: clock divider, CPOL/CPHA modes, FIFOs, segmented transfers.

Why it exists. Because it is the archetype of a block that looks trivial and has many variants. Four clock modes, bit order, half- and full-duplex, dual and quad IO, and chip-select timing all differ between peripherals.

Undergraduate review · EECS concepts. Serial communication; clock phase and polarity; setup and hold margins; clock division.

Graduate · Practice note. Most SPI field failures trace to CPOL/CPHA combinations or chip-select timing. A verification plan that does not exercise all four modes will leave one broken.

12 files • 4,482 lines

API — spi_host.sv

Table 909. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host.sv — module / interface / package (1개)

이름	종류
spi_host	module

Table 910. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host.sv — parameter (7개)

파라미터	형/폭	기본값
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
NumCS	int unsigned	1
EnableRacl	bit	1'b0
RaclErrorRsp	bit	EnableRacl
RaclPolicySelWinRXDATA	top_racl_pkg::racl_policy_sel_t	0
RaclPolicySelWinTXDATA	top_racl_pkg::racl_policy_sel_t	0

Table 911. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host.sv — localparam (1개)

상수	형/폭	값
CSW	int	prim_util_pkg::vbits(NumCS)

Table 912. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host.sv — 포트 (20개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t

포트	방향	형/폭
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
cio_sck_o	output	logic
cio_sck_en_o	output	logic
cio_csb_o	output	logic [NumCS-1:0]
cio_csb_en_o	output	logic [NumCS-1:0]
cio_sd_o	output	logic [3:0]
cio_sd_en_o	output	logic [3:0]
cio_sd_i	input	[3:0]
passthrough_i	input	spi_device_pkg::passthrough_req_t
passthrough_o	output	spi_device_pkg::passthrough_rsp_t
lsio_trigger_o	output	logic
intr_error_o	output	logic
intr_spi_event_o	output	logic

API — spi_host_byte_merge.sv

Table 913. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_byte_merge.sv — module / interface / package (1개)

이름	종류
spi_host_byte_merge	module

Table 914. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_byte_merge.sv — 포트 (10개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
byte_i	input	[7:0]
byte_last_i	input	
byte_valid_i	input	
byte_ready_o	output	logic
word_o	output	logic [31:0]
word_valid_o	output	logic
word_ready_i	input	
sw_rst_i	input	

API — spi_host_byte_select.sv**Table 915. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_byte_select.sv — module / interface / package (1개)**

이름	종류
spi_host_byte_select	module

Table 916. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_byte_select.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
word_i	input	[31:0]
word_be_i	input	[3:0]
word_valid_i	input	
word_ready_o	output	logic
byte_o	output	logic [7:0]
byte_valid_o	output	logic
byte_ready_i	input	
flush_i	input	
sw_rst_i	input	

API — spi_host_cmd_pkg.sv**Table 917. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_cmd_pkg.sv — module / interface / package (1개)**

이름	종류
spi_host_cmd_pkg	package

Table 918. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_cmd_pkg.sv — parameter (1개)

파라미터	형/폭	기본값
CmdSize	int	56

API — spi_host_command_queue.sv**Table 919. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_command_queue.sv — module / interface / package (1개)**

이름	종류
spi_host_command_queue	module

Table 920. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_command_queue.sv — parameter (2개)

파라미터	형/폭	기본값
CmdDepth	int	4
NumCS	int unsigned	1

Table 921. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_command_queue.sv — localparam (2개)

상수	형/폭	값
CSW	int	prim_util_pkg::vbits(NumCS)
CmdDepthW	int	prim_util_pkg::vbits(CmdDepth+1)

Table 922. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_command_queue.sv — 포트 (13개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
command_i	input	spi_host_cmd_pkg::command_t
command_csid_i	input	logic [CSW-1:0]
command_valid_i	input	
command_busy_o	output	logic
core_command_o	output	spi_host_cmd_pkg::command_t
core_command_csid_o	output	logic [CSW-1:0]
core_command_valid_o	output	logic
core_command_ready_i	input	
qd_o	output	logic [3:0]
error_busy_o	output	logic
sw_rst_i	input	

API — spi_host_core.sv**Table 923. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_core.sv — module / interface / package (1개)**

이름	종류
spi_host_core	module

Table 924. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_core.sv — parameter (1개)

파라미터	형/폭	기본값
NumCS	int	1

Table 925. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_core.sv — localparam (1개)

상수	형/폭	값
CSW	int	prim_util_pkg::vbits(NumCS)

Table 926. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_core.sv — 포트 (24개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
command_i	input	spi_host_cmd_pkg::command_t

포트	방향	형/폭
command_csid_i	input	logic [CSW-1:0]
command_valid_i	input	
command_ready_o	output	
en_i	input	
tx_data_i	input	[31:0]
tx_be_i	input	[3:0]
tx_valid_i	input	
tx_ready_o	output	logic
tx_byte_select_full_o	output	logic
rx_data_o	output	logic [31:0]
rx_valid_o	output	logic
rx_ready_i	input	
sw_rst_i	input	
sck_o	output	logic
csb_o	output	logic [NumCS-1:0]
sd_o	output	logic [3:0]
sd_en_o	output	logic [3:0]
sd_i	input	[3:0]
rx_stall_o	output	logic
tx_stall_o	output	logic
active_o	output	logic

API — spi_host_data_fifos.sv

Table 927. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_data_fifos.sv — module / interface / package (1개)

이름	종류
spi_host_data_fifos	module

Table 928. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_data_fifos.sv — parameter (3개)

파라미터	형/폭	기본값
TxDepth	int	72
RxDepth	int	64
SwapBytes	logic	0

Table 929. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_data_fifos.sv — localparam (2개)

상수	형/폭	값
RxDepthW	int	prim_util_pkg::vbits(RxDepth+1)

상수	형/폭	값
TxDpthW	int	prim_util_pkg::vbits(TxDpth+1)

Table 930. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_data_fifos.sv — 포트 (28개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tx_data_i	input	[31:0]
tx_be_i	input	[3:0]
tx_valid_i	input	
tx_ready_o	output	logic
tx_watermark_i	input	[7:0]
core_tx_data_o	output	logic [31:0]
core_tx_be_o	output	logic [3:0]
core_tx_valid_o	output	logic
core_tx_ready_i	input	
core_tx_byte_select_full_i	input	
core_rx_data_i	input	[31:0]
core_rx_valid_i	input	
core_rx_ready_o	output	logic
rx_data_o	output	logic [31:0]
rx_valid_o	output	logic
rx_ready_i	input	
rx_watermark_i	input	[7:0]
sw_rst_i	input	
tx_empty_o	output	logic
tx_full_o	output	logic
tx_qd_o	output	logic [7:0]
tx_wm_o	output	logic
rx_empty_o	output	logic
rx_full_o	output	logic
rx_qd_o	output	logic [7:0]
rx_wm_o	output	logic

API — spi_host_fsm.sv**Table 931. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_fsm.sv — module / interface / package (17개)**

이름	종류
spi_host_fsm	module

Table 932. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_fsm.sv — parameter (1개)

파라미터	형/폭	기본값
NumCS	int	1

Table 933. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_fsm.sv — localparam (1개)

상수	형/폭	값
CSW	int	prim_util_pkg::vbits(NumCS)

Table 934. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_fsm.sv — 포트 (24개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
en_i	input	
command_i	input	command_t
command_csid_i	input	logic [CSW-1:0]
command_valid_i	input	
command_ready_o	output	logic
sck_o	output	logic
csb_o	output	logic [NumCS-1:0]
sd_en_o	output	logic [3:0]
last_read_o	output	logic
last_write_o	output	logic
wr_en_o	output	logic
sr_wr_ready_i	input	
rd_en_o	output	logic
sr_rd_ready_i	input	
sample_en_o	output	logic
shift_en_o	output	logic
speed_o	output	logic [1:0]
full_cyc_o	output	logic
rx_stall_o	output	logic
tx_stall_o	output	logic
active_o	output	logic
sw_rst_i	input	

API — spi_host_reg_pkg.sv**Table 935. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_reg_pkg.sv — module / interface / package (1개)**

이름	종류
spi_host_reg_pkg	package

Table 936. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_reg_pkg.sv — parameter (35개)

파라미터	형/폭	기본값
ByteOrder	logic	1
TxDpeth	int	72
RxDpeth	int	64
CmdDepth	int	4
NumAlerts	int	1
BlockAw	int	6
NumRegs	int	12
SPI_HOST_INTR_STATE_OFFSET	logic [BlockAw-1:0]	6'h 0
SPI_HOST_INTR_ENABLE_OFFSET	logic [BlockAw-1:0]	6'h 4
SPI_HOST_INTR_TEST_OFFSET	logic [BlockAw-1:0]	6'h 8
SPI_HOST_ALERT_TEST_OFFSET	logic [BlockAw-1:0]	6'h c
SPI_HOST_CONTROL_OFFSET	logic [BlockAw-1:0]	6'h 10
SPI_HOST_STATUS_OFFSET	logic [BlockAw-1:0]	6'h 14
SPI_HOST_CONFIGOPTS_OFFSET	logic [BlockAw-1:0]	6'h 18
SPI_HOST_CSID_OFFSET	logic [BlockAw-1:0]	6'h 1c
SPI_HOST_COMMAND_OFFSET	logic [BlockAw-1:0]	6'h 20
SPI_HOST_ERROR_ENABLE_OFFSET	logic [BlockAw-1:0]	6'h 2c
SPI_HOST_ERROR_STATUS_OFFSET	logic [BlockAw-1:0]	6'h 30
SPI_HOST_EVENT_ENABLE_OFFSET	logic [BlockAw-1:0]	6'h 34
SPI_HOST_INTR_TEST_RESVAL	logic [1:0]	2'h 0
SPI_HOST_INTR_TEST_ERROR_RESVAL	logic [0:0]	1'h 0
SPI_HOST_INTR_TEST_SPI_EVENT_RESVAL	logic [0:0]	1'h 0
SPI_HOST_ALERT_TEST_RESVAL	logic [0:0]	1'h 0
SPI_HOST_ALERT_TEST_FATAL_FAULT_RESVAL	logic [0:0]	1'h 0
SPI_HOST_COMMAND_RESVAL	logic [24:0]	25'h 0
SPI_HOST_COMMAND_CSAAT_RESVAL	logic [0:0]	1'h 0
SPI_HOST_COMMAND_SPEED_RESVAL	logic [1:0]	2'h 0
SPI_HOST_COMMAND_DIRECTION_RESVAL	logic [1:0]	2'h 0
SPI_HOST_COMMAND_LEN_RESVAL	logic [19:0]	20'h 0

파라미터	형/폭	기본값
SPI_HOST_RXDATA_OFFSET	logic [BlockAw-1:0]	6'h 24
SPI_HOST_RXDATA_SIZE	int unsigned	'h 4
SPI_HOST_RXDATA_IDX	int unsigned	0
SPI_HOST_TXDATA_OFFSET	logic [BlockAw-1:0]	6'h 28
SPI_HOST_TXDATA_SIZE	int unsigned	'h 4
SPI_HOST_TXDATA_IDX	int unsigned	1

API — spi_host_reg_top.sv

Table 937. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_reg_top.sv — module / interface / package (1개)

이름	종류
spi_host_reg_top	module

Table 938. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_reg_top.sv — parameter (2개)

파라미터	형/폭	기본값
EnableRacl	bit	1'b0
RaclErrorRsp	bit	1'b1

Table 939. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_reg_top.sv — localparam (3개)

상수	형/폭	값
AW	int	6
DW	int	32
DBW	int	DW/8

Table 940. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_reg_top.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
reg2hw	output	spi_host_reg_pkg::spi_host_reg2hw_t
hw2reg	input	spi_host_reg_pkg::spi_host_hw2reg_t
rac1_policies_i	input	top_racl_pkg::rac1_policy_vec_t
rac1_error_o	output	top_racl_pkg::rac1_error_log_t
intg_err_o	output	logic

API — spi_host_shift_register.sv

Table 941. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_shift_register.sv — module / interface / package (1개)

이름	종류
spi_host_shift_register	module

Table 942. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_shift_register.sv — 포트 (23개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
wr_en_i	input	
wr_ready_o	output	logic
rd_en_i	input	
rd_ready_o	output	logic
speed_i	input	[1:0]
shift_en_i	input	
sample_en_i	input	
full_cyc_i	input	
last_read_i	input	
last_write_i	input	
tx_data_i	input	[7:0]
tx_valid_i	input	
tx_ready_o	output	
tx_flush_o	output	
rx_data_o	output	logic [7:0]
rx_valid_o	output	logic
rx_ready_i	input	
rx_last_o	output	logic
sd_i	input	[3:0]
sd_o	output	logic [3:0]
sw_rst_i	input	

API — spi_host_window.sv

Table 943. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_window.sv — module / interface / package (1개)

이름	종류
spi_host_window	module

Table 944. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_window.sv — parameter (4개)

파라미터	형/폭	기본값
EnableRac1	bit	1'b0

파라미터	형/폭	기본값
RaClErrorRsp	bit	1'b1
RaClPolicySelWinRXDATA	top_racl_pkg::racl_policy_sel_t	0
RaClPolicySelWinTXDATA	top_racl_pkg::racl_policy_sel_t	0

Table 945. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_window.sv — localparam (4개)

상수	형/폭	값
AW	int	spi_host_reg_pkg::BlockAw
DW	int	32
ByteMaskW	int	DW / 8
RaClPolicySelRangesTXDATA	top_racl_pkg::racl_range_t [0:0]	{

Table 946. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_window.sv — 포트 (14개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
rx_win_i	input	tlul_pkg::tl_h2d_t
rx_win_o	output	tlul_pkg::tl_d2h_t
tx_win_i	input	tlul_pkg::tl_h2d_t
tx_win_o	output	tlul_pkg::tl_d2h_t
tx_data_o	output	logic [31:0]
tx_be_o	output	logic [3:0]
tx_valid_o	output	logic
rx_data_i	input	[31:0]
rx_ready_o	output	logic
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_tx_o	output	top_racl_pkg::racl_error_log_t
racl_error_rx_o	output	top_racl_pkg::racl_error_log_t

1. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host.sv (661 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Serial Peripheral Interface (SPI) Host module.
6 //
7 //
8
9 `include "prim_assert.sv"
10
11 module spi_host
12     import spi_host_reg_pkg::*;
13     #(
14         parameter logic [NumAlerts-1:0]      AlertAsyn-
c0n         = {NumAlerts{1'b1}},
15         // Number of cycles a differential skew is tolerated on the
alert signal
16         parameter int unsigned              AlertSkew-
cycles         = 1,

```

```

17     parameter int unsigned              NumCS         = 1,
18     parameter bit                      EnableR-
acl           = 1'b0,
19     parameter bit                      RaclEr-
rorRsp        = EnableRacl,
20     parameter top_racl_pkg::racl_policy_sel_t RaclPoli-
cySelVec[NumRegs] = '{NumRegs{0}},
21     parameter top_racl_pkg::racl_policy_sel_t RaclPolicySelWin-
RXDATA        = 0,
22     parameter top_racl_pkg::racl_policy_sel_t RaclPolicySel-
WinTXDATA     = 0
23 ) (
24     input          clk_i,
25     input          rst_ni,
26
27     // Register interface
28     input          tlul_pkg::tl_h2d_t tl_i,
29     output         tlul_pkg::tl_d2h_t tl_o,
30
31     // Alerts
32     input prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_rx_i,
33     output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]

```

```

alert_tx_o,
34
35 // RACL interface
36 input top_racl_pkg::racl_policy_vec_t racl_policies_i,
37 output top_racl_pkg::racl_error_log_t racl_error_o,
38
39 // SPI Interface
40 output logic cio_sck_o,
41 output logic cio_sck_en_o,
42 output logic [NumCS-1:0] cio_csb_o,
43 output logic [NumCS-1:0] cio_csb_en_o,
44 output logic [3:0] cio_sd_o,
45 output logic [3:0] cio_sd_en_o,
46 input [3:0] cio_sd_i,
47
48 // Passthrough interface
49 input spi_device_pkg::passthrough_req_t passthrough_i,
50 output spi_device_pkg::passthrough_rsp_t passthrough_o,
51
52 output logic lsio_trigger_o,
53
54 output logic intr_error_o,
55 output logic intr_spi_event_o
56 );
57
58 import spi_host_cmd_pkg::*;
59
60 localparam int CSW = prim_util_pkg::vbits(NumCS);
61
62 spi_host_reg2hw_t reg2hw;
63 spi_host_hw2reg_t hw2reg;
64
65 top_racl_pkg::racl_error_log_t racl_error[3];
66 if (EnableRacl) begin : gen_racl_error_arb
67 // Arbitrate between all simultaneously valid error log
68 requests.
69 prim_racl_error_arb #(
70 .N ( 3 )
71 ) u_prim_err_arb (
72 .clk_i,
73 .rst_ni,
74 .error_log_i ( racl_error ),
75 .error_log_o ( racl_error_o )
76 );
77 end else begin : gen_no_racl_error_arb
78 logic unused_signals;
79 always_comb begin
80 unused_signals = ^{racl_error[0] ^ racl_error[1] ^
81 racl_error[2]};
82 racl_error_o = '0;
83 end
84 end
85
86 tlul_pkg::tl_h2d_t fifo_win_h2d [2];
87 tlul_pkg::tl_d2h_t fifo_win_d2h [2];
88
89 // Register module
90 logic [NumAlerts-1:0] alert_test, alerts;
91 spi_host_reg_top #(
92 .EnableRacl ( EnableRacl ),
93 .RaclErrorRsp ( RaclErrorRsp ),
94 .RaclPolicySelVec ( RaclPolicySelVec )
95 ) u_reg (
96 .clk_i,
97 .rst_ni,
98 .tl_i ( tl_i ),
99 .tl_o ( tl_o ),
100 .tl_win_o ( fifo_win_h2d ),
101 .tl_win_i ( fifo_win_d2h ),
102 .reg2hw,
103 .hw2reg,
104 // SEC_CM: BUS INTEGRITY
105 .intg_err_o ( alerts[0] ),
106 .racl_policies_i ( racl_policies_i ),
107 .racl_error_o ( racl_error[0] )
108 );
109 // Alerts
110 assign alert_test = {
111 reg2hw.alert_test.q &
112 reg2hw.alert_test.qe
113 };
114
115 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
116 prim_alert_sender #(
117 .AsyncOn(AlertAsyncOn[i]),
118 .SkewCycles(AlertSkewCycles),
119 .IsFatal(1'b1)
120 ) u_prim_alert_sender (
121 .clk_i,
122 .rst_ni,
123 .alert_test_i ( alert_test[i] ),
124 .alert_req_i ( alerts[0] ),
125 .alert_ack_o ( ),
126 .alert_state_o ( ),
127 .alert_rx_i ( alert_rx_i[i] ),
128 .alert_tx_o ( alert_tx_o[i] )
129 );
130 end
131 logic sck;
132
133 logic [NumCS-1:0] csb;
134 logic [3:0] sd_out;
135 logic [3:0] sd_en, sd_en_core;
136 logic [3:0] sd_i;
137 logic output_en;
138
139 assign output_en = reg2hw.control.output_en;
140
141 assign sd_en = output_en ? sd_en_core : 4'h0;
142
143 if (NumCS == 1) begin : gen_passthrough_implementation
144 logic passthrough_en;
145 assign passthrough_en = passthrough_i.passthrough_en;
146
147 logic pt_sck;
148 logic pt_sck_en;
149 logic [0:0] pt_csb;
150 logic [0:0] pt_csb_en;
151 logic [3:0] pt_sd_out;
152 logic [3:0] pt_sd_en;
153
154 assign pt_sck = passthrough_i.sck;
155 assign pt_sck_en = passthrough_i.sck_en;
156 assign pt_csb[0] = passthrough_i.csb;
157 assign pt_csb_en[0] = passthrough_i.csb_en;
158 assign pt_sd_out = passthrough_i.s;
159 assign pt_sd_en = passthrough_i.s_en;
160
161 assign cio_sck_o = passthrough_en ? pt_sck : sck;
162 assign cio_sck_en_o = passthrough_en ? pt_sck_en : out-
163 put_en;
164 assign cio_csb_o = passthrough_en ? pt_csb : csb;
165 assign cio_csb_en_o = passthrough_en ? pt_csb_en : out-
166 put_en;
167 assign cio_sd_o = passthrough_en ? pt_sd_out : sd_out;
168 assign cio_sd_en_o = passthrough_en ? pt_sd_en : sd_en;
169
170 end
171 else begin : gen_passthrough_ignore
172 // Passthrough only supported for instances with one CSb
173 line
174 `ASSERT(PassthroughNumCSCompat_A, !
175 passthrough_i.passthrough_en, clk_i, rst_ni)
176
177 assign cio_sck_o = sck;
178 assign cio_sck_en_o = output_en;
179 assign cio_csb_o = csb;
180 assign cio_csb_en_o = {NumCS{output_en}};
181 assign cio_sd_o = sd_out;
182 assign cio_sd_en_o = sd_en;
183
184 logic unused_pt_en;
185 logic unused_pt_sck;
186 logic unused_pt_sck_en;
187 logic unused_pt_csb;
188 logic unused_pt_csb_en;
189 logic [3:0] unused_pt_sd_out;
190 logic [3:0] unused_pt_sd_en;
191
192 assign unused_pt_en = passthrough_i.passthrough_en;
193 assign unused_pt_sck = passthrough_i.sck;
194 assign unused_pt_sck_en = passthrough_i.sck_en;
195 assign unused_pt_csb = passthrough_i.csb;
196 assign unused_pt_csb_en = passthrough_i.csb_en;
197 assign unused_pt_sd_out = passthrough_i.s;
198 assign unused_pt_sd_en = passthrough_i.s_en;
199
200 end : gen_passthrough_ignore
201
202 assign passthrough_o.s = cio_sd_i;
203 assign sd_i = cio_sd_i;
204
205 assign hw2reg.status.byteorder.d = ByteOrder;
206 assign hw2reg.status.byteorder.de = 1'b1;
207
208 logic command_valid;
209 logic core_command_valid;
210 logic command_busy;
211 logic core_command_ready;
212
213 command_t core_command, command;
214 logic [CSW-1:0] core_command_csid, command_csid;
215 logic error_csid_inval;
216 logic error_cmd_inval;
217 logic error_busy;
218 logic test_csid_inval;
219 logic test_dir_inval;
220 logic test_speed_inval;
221
222 assign test_csid_inval = (reg2hw.csid.q >= NumCS);
223
224 always_comb begin
225 test_speed_inval = 1'b1;
226 test_dir_inval = 1'b1;
227 unique case (reg2hw.command.speed.q)
228 Standard: begin
229 test_dir_inval = 1'b0;
230 test_speed_inval = 1'b0;
231 end
232 Dual, Quad: begin
233 test_dir_inval = (reg2hw.command.direction.q == Bid-

```



```

ir));
229     test_speed_inval = 1'b0;
230     end
231     default: begin
232     end
233 endcase
234 end
235
236 always_comb begin
237     command.segment.cmd_rd_en = 1'b0;
238     command.segment.cmd_wr_en = 1'b0;
239     unique case (reg2hw.command.direction.q)
240     RdOnly: begin
241         command.segment.cmd_rd_en = 1'b1;
242     end
243     WrOnly: begin
244         command.segment.cmd_wr_en = 1'b1;
245     end
246     Bidir: begin
247         command.segment.cmd_rd_en = 1'b1;
248         command.segment.cmd_wr_en = 1'b1;
249     end
250     default: begin
251     end
252 endcase
253 end
254
255 assign error_csid_inval = command_valid & ~command_busy &
256     test_csid_inval;
257 assign error_cmd_inval = command_valid & ~command_busy &
258     (test_speed_inval | test_dir_in-
val);
259
260 assign command_csid = (test_csid_inval) ? '0 :
reg2hw.csid.q[CSW-1:0];
261
262 assign command.configopts.clkdiv =
reg2hw.configopts.clkdiv.q;
263 assign command.configopts.csnidle =
reg2hw.configopts.csnidle.q;
264 assign command.configopts.csnlead =
reg2hw.configopts.csnlead.q;
265 assign command.configopts.csntail =
reg2hw.configopts.csntail.q;
266 assign command.configopts.full_cyc =
reg2hw.configopts.fullcyc.q;
267 assign command.configopts.cpha =
reg2hw.configopts.cpha.q;
268 assign command.configopts.cpol =
reg2hw.configopts.cpol.q;
269
270 assign command.segment.len = reg2hw.command.len.q;
271 assign command.segment.csaat = reg2hw.command.csaat.q;
272 assign command.segment.speed = reg2hw.command.speed.q;
273
274
275 logic [3:0] cmd_qes;
276
277 assign cmd_qes = {
278     reg2hw.command.len.qe,
279     reg2hw.command.speed.qe,
280     reg2hw.command.direction.qe,
281     reg2hw.command.csaat.qe
282 };
283
284 // Any qe pin from COMMAND will suffice.
285 assign command_valid = |cmd_qes;
286
287 logic active;
288 logic rx_stall;
289 logic tx_stall;
290
291 assign hw2reg.status.ready.d = ~command_busy;
292 assign hw2reg.status.active.d = active;
293 assign hw2reg.status.rxstall.d = rx_stall;
294 assign hw2reg.status.txstall.d = tx_stall;
295
296 assign hw2reg.status.ready.de = 1'b1;
297 assign hw2reg.status.active.de = 1'b1;
298 assign hw2reg.status.rxstall.de = 1'b1;
299 assign hw2reg.status.txstall.de = 1'b1;
300
301 logic sw_rst;
302
303 logic [3:0] cmd_qd;
304
305 spi_host_command_queue #(
306     .CmdDepth(CmdDepth),
307     .NumCS(NumCS)
308 ) u_cmd_queue (
309     .clk_i,
310     .rst_ni,
311     .command_i (command),
312     .command_csid_i (command_csid),
313     .command_valid_i (command_valid),
314     .command_busy_o (command_busy),
315     .core_command_o (core_command),
316     .core_command_csid_o (core_command_csid),
317     .core_command_valid_o (core_command_valid),
318     .core_command_ready_i (core_command_ready),
319     .error_busy_o (error_busy),

```

```

320     .qd_o (cmd_qd),
321     .sw_rst_i (sw_rst)
322 );
323
324 logic [31:0] tx_data;
325 logic [3:0] tx_be;
326 logic tx_valid;
327 logic tx_ready;
328
329 logic [31:0] rx_data;
330 logic rx_valid;
331 logic rx_ready;
332
333 spi_host_window #(
334     .EnableRacl ( EnableRacl ),
335     .RaclErrorRsp ( RaclErrorRsp ),
336     .RaclPolicySelWinRXDATA ( RaclPolicySelWinRXDATA ),
337     .RaclPolicySelWinTXDATA ( RaclPolicySelWinTXDATA )
338 ) u_window (
339     .clk_i,
340     .rst_ni,
341     .rx_win_i ( fifo_win_h2d[0] ),
342     .rx_win_o ( fifo_win_d2h[0] ),
343     .tx_win_i ( fifo_win_h2d[1] ),
344     .tx_win_o ( fifo_win_d2h[1] ),
345     .tx_data_o ( tx_data ),
346     .tx_be_o ( tx_be ),
347     .tx_valid_o ( tx_valid ),
348     .rx_data_i ( rx_data ),
349     .rx_ready_o ( rx_ready ),
350     .racl_policies_i ( racl_policies_i ),
351     .racl_error_tx_o ( racl_error[1] ),
352     .racl_error_rx_o ( racl_error[2] )
353 );
354
355 logic [31:0] core_tx_data;
356 logic [3:0] core_tx_be;
357 logic core_tx_valid;
358 logic core_tx_ready;
359 logic core_tx_byte_select_full;
360
361 logic [31:0] core_rx_data;
362 logic core_rx_valid;
363 logic core_rx_ready;
364
365 logic [7:0] rx_watermark;
366 logic [7:0] tx_watermark;
367 logic [7:0] rx_qd;
368 logic [7:0] tx_qd;
369
370 logic tx_empty, tx_full, tx_wm;
371 logic rx_empty, rx_full, rx_wm;
372
373 assign rx_watermark = reg2hw.control.rx_watermark.q;
374 assign tx_watermark = reg2hw.control.tx_watermark.q;
375
376 assign hw2reg.status.txqd.d = tx_qd;
377 assign hw2reg.status.rxqd.d = rx_qd;
378 assign hw2reg.status.cmdqd.d = cmd_qd;
379 assign hw2reg.status.txwm.d = tx_wm;
380 assign hw2reg.status.rxwm.d = rx_wm;
381 assign hw2reg.status.rxepty.d = rx_empty;
382 assign hw2reg.status.txempty.d = tx_empty;
383 assign hw2reg.status.rxfull.d = rx_full;
384 assign hw2reg.status.txfull.d = tx_full;
385
386 assign hw2reg.status.txqd.de = 1'b1;
387 assign hw2reg.status.rxqd.de = 1'b1;
388 assign hw2reg.status.cmdqd.de = 1'b1;
389 assign hw2reg.status.txwm.de = 1'b1;
390 assign hw2reg.status.rxwm.de = 1'b1;
391 assign hw2reg.status.rxepty.de = 1'b1;
392 assign hw2reg.status.txempty.de = 1'b1;
393 assign hw2reg.status.rxfull.de = 1'b1;
394 assign hw2reg.status.txfull.de = 1'b1;
395
396 logic error_overflow, error_underflow;
397 logic error_access_inval;
398
399 // Since the DATA FIFOs are essentially directly connected
to SW registers, it is an error if
400 // there is ever a need for flow control.
401 assign error_overflow = tx_valid & ~tx_ready;
402 assign error_underflow = rx_ready & ~rx_valid;
403 logic access_valid;
404 assign error_access_inval = tx_valid & ~access_valid;
405
406 always_comb begin
407     unique case (tx_be)
408     4'b1000,
409     4'b0100,
410     4'b0010,
411     4'b0001,
412     4'b1100,
413     4'b0110,
414     4'b0011,
415     4'b1111: begin
416         access_valid = 1'b1;
417     end
418     default: begin
419         access_valid = 1'b0;

```

```

420     end
421   endcase
422 end
423
424 logic tx_valid_checked;
425 assign tx_valid_checked = tx_valid & ~error_overflow & ~error_access_inval;
426
427 // Note on ByteOrder and ByteSwapping.
428 // ByteOrder == 1 is for Little-Endian transmission (i.e.
LSB first), which is achieved by
429 // default with the prim_packer_fifo implementation. Thus
we have to swap if Big-Endian
430 // transmission is required (i.e. if ByteOrder == 0).
431 spi_host_data_fifos #(
432   .TxDepth(TxDepth),
433   .RxDepth(RxDepth),
434   .SwapBytes(~ByteOrder)
435 ) u_data_fifos (
436   .clk_i,
437   .rst_ni,
438
439   .tx_data_i      (tx_data),
440   .tx_be_i        (tx_be),
441   .tx_valid_i     (tx_valid_checked),
442   .tx_ready_o     (tx_ready),
443   .tx_watermark_i (tx_watermark),
444
445   .core_tx_data_o (core_tx_data),
446   .core_tx_be_o   (core_tx_be),
447   .core_tx_valid_o (core_tx_valid),
448   .core_tx_ready_i (core_tx_ready),
449   .core_tx_byte_select_full_i (core_tx_byte_select_full),
450
451   .core_rx_data_i (core_rx_data),
452   .core_rx_valid_i (core_rx_valid),
453   .core_rx_ready_o (core_rx_ready),
454
455   .rx_data_o      (rx_data),
456   .rx_valid_o     (rx_valid),
457   .rx_ready_i     (rx_ready),
458   .rx_watermark_i (rx_watermark),
459
460   .tx_empty_o     (tx_empty),
461   .tx_full_o      (tx_full),
462   .tx_qd_o        (tx_qd),
463   .tx_wm_o        (tx_wm),
464   .rx_empty_o     (rx_empty),
465   .rx_full_o      (rx_full),
466   .rx_qd_o        (rx_qd),
467   .rx_wm_o        (rx_wm),
468
469   .sw_rst_i       (sw_rst)
470 );
471
472 logic en_sw;
473 logic enb_error;
474 logic en;
475
476 assign en      = en_sw & ~enb_error;
477 assign sw_rst = reg2hw.control.sw_rst.q;
478 assign en_sw  = reg2hw.control.spilen.q;
479
480 spi_host_core #(
481   .NumCS(NumCS)
482 ) u_spi_core (
483   .clk_i,
484   .rst_ni,
485
486   .command_i      (core_command),
487   .command_csid_i (core_command_csid),
488   .command_valid_i (core_command_valid),
489   .command_ready_o (core_command_ready),
490   .en_i           (en),
491   .tx_data_i      (core_tx_data),
492   .tx_be_i        (core_tx_be),
493   .tx_valid_i     (core_tx_valid),
494   .tx_ready_o     (core_tx_ready),
495   .tx_byte_select_full_o (core_tx_byte_select_full),
496   .rx_data_o      (core_rx_data),
497   .rx_valid_o     (core_rx_valid),
498   .rx_ready_i     (core_rx_ready),
499   .sck_o          (sck),
500   .csb_o          (csb),
501   .sd_o           (sd_out),
502   .sd_en_o        (sd_en_core),
503   .sd_i,
504   .rx_stall_o     (rx_stall),
505   .tx_stall_o     (tx_stall),
506   .sw_rst_i       (sw_rst),
507   .active_o       (active)
508 );
509
510 logic event_error;
511
512 logic [5:0] error_vec;
513 logic [5:0] error_mask;
514 logic [5:0] sw_error_status;
515
516 assign error_vec = {
517   error_access_inval,
518   error_csid_inval,
519   error_cmd_inval,
520   error_underflow,
521   error_overflow,
522   error_busy
523 };
524
525 // This mask dictates what classes of error events are to be
escalated to error interrupts.
526 // (Assume here that error interrupts will be enabled)
527 // Software generated errors can be configured to _not_ generate
error events.
528 // Bus errors (such as invalid write access) always generate
error events and therefore
529 // interrupts.
530 assign error_mask = {
531   1'b1, // invalid access: always an error event
532   reg2hw.error_enable.csidinval.q,
533   reg2hw.error_enable.cmdinval.q,
534   reg2hw.error_enable.underflow.q,
535   reg2hw.error_enable.overflow.q,
536   reg2hw.error_enable.cmbusy.q
537 };
538
539 assign hw2reg.error_status.accessinval.d = error_access_inval;
540 assign hw2reg.error_status.csidinval.d = error_csid_inval;
541 assign hw2reg.error_status.cmdinval.d = error_cmd_inval;
542 assign hw2reg.error_status.underflow.d = error_underflow;
543 assign hw2reg.error_status.overflow.d = error_overflow;
544 assign hw2reg.error_status.cmbusy.d = error_busy;
545
546 // Write the status register whenever the corresponding
event occurs.
547 // Only clear them from software.
548 assign hw2reg.error_status.accessinval.de = error_access_inval;
549 assign hw2reg.error_status.csidinval.de = error_csid_inval;
550 assign hw2reg.error_status.cmdinval.de = error_cmd_inval;
551 assign hw2reg.error_status.underflow.de = error_underflow;
552 assign hw2reg.error_status.overflow.de = error_overflow;
553 assign hw2reg.error_status.cmbusy.de = error_busy;
554
555 assign sw_error_status[5] =
reg2hw.error_status.accessinval.q;
556 assign sw_error_status[4] = reg2hw.error_status.csidinval.q;
557 assign sw_error_status[3] = reg2hw.error_status.cmdinval.q;
558 assign sw_error_status[2] = reg2hw.error_status.underflow.q;
559 assign sw_error_status[1] = reg2hw.error_status.overflow.q;
560 assign sw_error_status[0] = reg2hw.error_status.cmbusy.q;
561
562 assign event_error = |(error_vec & error_mask);
563 assign enb_error = |sw_error_status;
564
565 prim_intr_hw #(.Width(1)) intr_hw_error (
566   .clk_i,
567   .rst_ni,
568   .event_intr_i      (event_error),
569   .reg2hw_intr_enable_q_i (reg2hw.intr_enable.error.q),
570   .reg2hw_intr_test_q_i  (reg2hw.intr_test.error.q),
571   .reg2hw_intr_test_qe_i (reg2hw.intr_test.error.qe),
572   .reg2hw_intr_state_q_i (reg2hw.intr_state.error.q),
573   .hw2reg_intr_state_de_o (hw2reg.intr_state.error.de),
574   .hw2reg_intr_state_d_o (hw2reg.intr_state.error.d),
575   .intr_o              (intr_error_o)
576 );
577
578 logic status_spi_event;
579
580 logic status_idle, status_ready, status_tx_wm, status_rx_wm,
status_tx_empty, status_rx_full;
581 logic [5:0] event_vector;
582 logic [5:0] event_mask;
583
584 assign status_idle = ~active;
585 assign status_ready = ~command_busy;
586 assign status_tx_wm = tx_wm;
587 assign status_rx_wm = rx_wm;
588 assign status_tx_empty = tx_empty;
589 assign status_rx_full = rx_full;
590
591 assign event_vector = {
592   status_idle,
593   status_ready,
594   status_tx_wm,
595   status_rx_wm,
596   status_tx_empty,
597   status_rx_full
598 };
599
600 assign event_mask = {
601   reg2hw.event_enable.idle.q,
602   reg2hw.event_enable.ready.q,
603   reg2hw.event_enable.txwm.q,
604   reg2hw.event_enable.rxwm.q,
605   reg2hw.event_enable.txempty.q,
606   reg2hw.event_enable.rxfull.q
607 };

```

```

606   };
607
608   // Qualify interrupt sources individually with dedicated
mask register CSR.EVENT_ENABLE
609   assign status_spi_event = |(event_vector & event_mask);
610
611   // Flop trigger signal to avoid glitches on the output
612   always_ff @(posedge clk_i or negedge rst_ni) begin
613     if (!rst_ni) begin
614       lsio_trigger_o <= 1'b0;
615     end else begin
616       lsio_trigger_o <= tx_wm | rx_wm;
617     end
618   end
619
620   prim_intr_hw #(
621     .Width(1),
622     .IntrT("Status")
623   ) intr_hw_spi_event (
624     .clk_i,
625     .rst_ni,
626     .event_intr_i      (status_spi_event),
627     .reg2hw_intr_enable_q_i (reg2hw_intr_enable_spi_event.q),
628     .reg2hw_intr_test_q_i  (reg2hw_intr_test_spi_event.q),
629     .reg2hw_intr_test_qe_i (reg2hw_intr_test_spi_event.qe),
630     .reg2hw_intr_state_q_i (reg2hw_intr_state_spi_event.q),
631     .hw2reg_intr_state_de_o (hw2reg_intr_state_spi_event.de),
632     .hw2reg_intr_state_do_o (hw2reg_intr_state_spi_event.d),
633     .intr_o              (intr_spi_event_o)
634   );
635
636   `ASSERT_KNOWN(TIDValidKnown0_A, tl_o.d_valid)
637   `ASSERT_KNOWN(TIARReadyKnown0_A, tl_o.a_ready)
638   `ASSERT_KNOWN(AlertKnown0_A, alert_tx_o)
639   `ASSERT_KNOWN(CioSckEnKnown0_A, cio_sck_o)
640   `ASSERT_KNOWN(CioSckEnKnown0_A, cio_sck_en_o)
641   `ASSERT_KNOWN(CioCsbEnKnown0_A, cio_csb_o)
642   `ASSERT_KNOWN(CioCsbEnKnown0_A, cio_csb_en_o)
643   `ASSERT_KNOWN(CioSdEnKnown0_A, cio_sd_o, !
644   passthrough_i.passthrough_en |
645   (passthrough_i.passthrough_en && passthrough_i.csb_en && !
passthrough_i.csb),
646   passthrough_i.sck_en & passthrough_i.sck)
647   `ASSERT_KNOWN(CioSdEnKnown0_A, cio_sd_en_o)
648   `ASSERT_KNOWN(IntrSpiEventKnown0_A, intr_spi_event_o)
649   `ASSERT_KNOWN(IntrErrorKnown0_A, intr_error_o)
650   `ASSERT_KNOWN(LsioTriggerKnown0_A, lsio_trigger_o)
651   `ASSERT_KNOWN(RaclErrorValidKnown0_A, racl_error_o.valid)
652
653   // passthrough_o.s is passed through to spi_device, it may
contain unknown data,
654   // but the unknown data won't be used based on the SPI pro-
655   tocol.
656   // Hence, instead of checking known data, here does a con-
657   nectivity check.
658   `ASSERT(PassthroughConn_A, passthrough_o.s == cio_sd_i)
659   // Alert assertions for reg_we onehot check
660   `AS-
661   SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
u_reg, alert_tx_o[0])
662 endmodule : spi_host

```

2. lowRISC_opentitan/hw/ip/spi_host/rtl/ spi_host_byte_merge.sv (66 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Byte-merge module for collecting words in SPI Host IP
6 //
7
8 module spi_host_byte_merge (
9   input          clk_i,
10  input          rst_ni,
11  input [7:0]    byte_i,
12  input          byte_last_i,
13  input          byte_valid_i,
14  input          byte_ready_o,
15  output logic   word_o,
16  output logic   word_valid_o,
17  input          word_ready_i,
18  input          sw_rst_i
19 );
20
21 logic          clr;
22 logic          byte_valid;
23 logic          byte_ready;
24 logic          last_q;
25 logic          last_d;
26 logic          do_fill;
27 logic          byte_incoming;
28
29 // Don't clear data except on reset;
30 assign clr = sw_rst_i;

```

```

31 assign byte_incoming = byte_valid_i & byte_ready_o;
32
33 assign last_d        = sw_rst_i          ? 1'b0 :
34   byte_incoming && byte_last_i ? 1'b1 :
35   word_valid_o       ? 1'b0 :
36   last_q;
37
38 assign do_fill       = last_q & ~word_valid_o;
39 assign byte_valid    = do_fill | byte_valid_i;
40 assign byte_ready_o  = byte_ready & ~do_fill;
41
42 always_ff @(posedge clk_i or negedge rst_ni) begin
43   if (!rst_ni) begin
44     last_q <= 1'b0;
45   end else begin
46     last_q <= last_d;
47   end
48 end
49
50 prim_packer_fifo #(
51   .InW(8),
52   .OutW(32)
53 ) u_packer (
54   .clk_i,
55   .rst_ni,
56   .clr_i      (clr),
57   .wdata_i    (do_fill ? 8'h0 : byte_i),
58   .wvalid_i   (byte_valid),
59   .wready_o   (byte_ready),
60   .rdata_o    (word_o),
61   .rvalid_o   (word_valid_o),
62   .rready_i   (word_ready_i),
63   .depth_o    ()
64 );
65
66 endmodule : spi_host_byte_merge

```

3. lowRISC_opentitan/hw/ip/spi_host/rtl/ spi_host_byte_select.sv (53 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Byte-select module for dispensing words in SPI Host IP
6 //
7 module spi_host_byte_select (
8   input          clk_i,
9   input          rst_ni,
10  input [31:0]    word_i,
11  input [3:0]     word_be_i,
12  input          word_valid_i,
13  output logic    word_ready_o,
14  output logic [7:0] byte_o,
15  output logic    byte_valid_o,
16  input          byte_ready_i,
17  input          flush_i,
18  input          sw_rst_i
19 );
20
21 logic [35:0]    wdata_be;
22 logic           do_drain;
23 logic           byte_en;
24 logic           byte_valid;
25 logic           byte_ready;
26 logic           clr;
27
28 assign clr      = flush_i | sw_rst_i;
29 assign byte_valid_o = byte_valid & byte_en;
30 assign do_drain  = byte_valid & ~byte_en;
31 assign byte_ready    = byte_ready_i | do_drain;
32
33 for(genvar ii = 0; ii < 4; ii = ii + 1) begin :
gen_map_data_be
34   assign wdata_be[9*ii +: 9] = { word_be_i[ii], word_i[8*ii
+: 8] };
35 end : gen_map_data_be
36
37 prim_packer_fifo #(
38   .InW(36),
39   .OutW(9)
40 ) u_packer (
41   .clk_i,
42   .rst_ni,
43   .clr_i      (clr),
44   .wdata_i    (wdata_be),
45   .wvalid_i   (word_valid_i),
46   .wready_o   (word_ready_o),
47   .rdata_o    ({byte_en, byte_o}),
48   .rvalid_o   (byte_valid),
49   .rready_i   (byte_ready),
50   .depth_o    ()
51 );
52
53 endmodule : spi_host_byte_select

```

4. lowRISC_opentitan/hw/ip/spi_host/rtl/ spi_host_cmd_pkg.sv (51 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Command & Configuration Options structure for SPI HOST.
6 //
7
8 package spi_host_cmd_pkg;
9
10 parameter int CmdSize = 56;
11
12 // For decoding the direction register
13 typedef enum logic [1:0] {
14     Dummy = 2'b00,
15     RdOnly = 2'b01,
16     WrOnly = 2'b10,
17     Bidir = 2'b11
18 } reg_direction_t;
19
20 // For decoding the direction register
21 typedef enum logic [1:0] {
22     Standard = 2'b00,
23     Dual = 2'b01,
24     Quad = 2'b10,
25     RsvdSpd = 2'b11
26 } speed_t;
27
28 typedef struct packed {
29     logic [15:0] clkdiv;
30     logic [3:0] csnsidle;
31     logic [3:0] csnlead;
32     logic [3:0] csntrail;
33     logic full_cyc;
34     logic cpha;
35     logic cpol;
36 } configopts_t;
37
38 typedef struct packed {
39     logic [1:0] speed;
40     logic cmd_wr_en;
41     logic cmd_rd_en;
42     logic [19:0] len;
43     logic csaat;
44 } segment_t;
45
46 typedef struct packed {
47     segment_t segment;
48     configopts_t configopts;
49 } command_t;
50
51 endpackage

```

5. lowRISC_opentitan/hw/ip/spi_host/rtl/ spi_host_command_queue.sv (63 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Queue for SPI_HOST commands
6 //
7
8 module spi_host_command_queue #(
9     parameter int CmdDepth = 4,
10    parameter int unsigned NumCS = 1,
11    localparam int CSW =
    prim_util_pkg::vbits(NumCS)
12 ) (
13     input clk_i,
14     input rst_ni,
15
16     input spi_host_cmd_pkg::command_t command_i,
17     input logic [CSW-1:0] command_csid_i,
18     input command_valid_i,
19     output logic command_busy_o,
20
21     output spi_host_cmd_pkg::command_t core_command_o,
22     output logic [CSW-1:0] core_command_csid_o,
23     output logic core_command_valid_o,
24     input core_command_ready_i,
25
26     output logic [3:0] qd_o,
27
28     output logic error_busy_o,
29
30     input sw_rst_i
31 );
32
33 localparam int CmdDepthW = prim_util_pkg::vbits(CmdDepth+1);
34
35 logic command_ready;

```

```

36
37 assign command_busy_o = ~command_ready;
38 assign error_busy_o = command_valid_i & command_busy_o;
39
40 logic [CmdDepthW-1:0] cmd_depth;
41
42 prim_fifo_sync #(
43     .Width($bits(spi_host_cmd_pkg::command_t) + CSW),
44     .Pass(0),
45     .Depth(CmdDepth)
46 ) cmd_fifo (
47     .clk_i,
48     .rst_ni,
49     .clr_i (sw_rst_i),
50     .wvalid_i (command_valid_i),
51     .wready_o (command_ready),
52     .wdata_i ({command_i, command_csid_i}),
53     .rvalid_o (core_command_valid_o),
54     .rready_i (core_command_ready_i),
55     .rdata_o ({core_command_o, core_command_csid_o}),
56     .full_o (),
57     .depth_o (cmd_depth),
58     .err_o ()
59 );
60
61 assign qd_o = 4'(cmd_depth);
62
63 endmodule : spi_host_command_queue

```

6. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_core.sv (155 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Core Implementation module for Serial Peripheral Interface
  (SPI) Host IP.
6 //
7
8 module spi_host_core #(
9     parameter int NumCS = 1,
10    localparam int CSW = prim_util_pkg::vbits(NumCS)
11 ) (
12     input clk_i,
13     input rst_ni,
14
15     input spi_host_cmd_pkg::command_t command_i,
16     input logic [CSW-1:0] command_csid_i,
17     input command_valid_i,
18     output command_ready_o,
19     input en_i,
20
21     input [31:0] tx_data_i,
22     input [3:0] tx_be_i,
23     input tx_valid_i,
24     output logic tx_ready_o,
25     output logic tx_byte_select_full_o,
26
27     output logic [31:0] rx_data_o,
28     output logic rx_valid_o,
29     input rx_ready_i,
30
31     input sw_rst_i,
32
33     // SPI Interface
34     output logic sck_o,
35     output logic [NumCS-1:0] csb_o,
36     output logic [3:0] sd_o,
37     output logic [3:0] sd_en_o,
38     input [3:0] sd_i,
39
40     // Status bits
41     output logic rx_stall_o,
42     output logic tx_stall_o,
43     output logic active_o
44 );
45
46 logic rx_valid_sr;
47 logic rx_ready_sr;
48 logic rx_last_sr;
49 logic [7:0] rx_data_sr;
50 logic tx_valid_sr;
51 logic tx_ready_sr;
52 logic tx_flush_sr;
53 logic [7:0] tx_data_sr;
54
55 logic wr_en;
56 logic rd_en;
57 logic wr_ready;
58 logic rd_ready;
59
60 logic sample_en;
61 logic shift_en;
62 logic [1:0] speed;
63 logic full_cyc;
64 logic last_read;
65 logic last_write;

```

```

66
67 spi_host_byte_merge u_merge (
68     .clk_i,
69     .rst_ni,
70     .byte_i      (rx_data_sr),
71     .byte_last_i (rx_last_sr),
72     .byte_valid_i (rx_valid_sr),
73     .byte_ready_o (rx_ready_sr),
74     .word_o      (rx_data_o),
75     .word_valid_o (rx_valid_o),
76     .word_ready_i (rx_ready_i),
77     .sw_rst_i
78 );
79
80 spi_host_byte_select u_select (
81     .clk_i,
82     .rst_ni,
83     .word_i      (tx_data_i),
84     .word_be_i   (tx_be_i),
85     .word_valid_i (tx_valid_i),
86     .word_ready_o (tx_ready_o),
87     .byte_o      (tx_data_sr),
88     .byte_valid_o (tx_valid_sr),
89     .byte_ready_i (tx_ready_sr),
90     .flush_i     (tx_flush_sr),
91     .sw_rst_i
92 );
93 // The byte_select module greedily pops data from the TX
FIFO, so
94 // the FIFO is observed as having an effective depth of N+1.
95 // The byte_valid_o signal indicates when the byte_select
contains
96 // a valid word. Pass this on up to the TX FIFO to adjust
the
97 // reported depth/full/empty flags.
98 assign tx_byte_select_full_o = tx_valid_sr;
99
100 spi_host_shift_register u_shift_reg (
101     .clk_i,
102     .rst_ni,
103     .wr_en_i   (wr_en),
104     .wr_ready_o (wr_ready),
105     .rd_en_i   (rd_en),
106     .rd_ready_o (rd_ready),
107     .speed_i   (speed),
108     .shift_en_i (shift_en),
109     .sample_en_i (sample_en),
110     .last_read_i (last_read),
111     .last_write_i (last_write),
112     .full_cyc_i (full_cyc),
113     .tx_data_i   (tx_data_sr),
114     .tx_valid_i   (tx_valid_sr),
115     .tx_ready_o   (tx_ready_sr),
116     .tx_flush_o   (tx_flush_sr),
117     .rx_data_o    (rx_data_sr),
118     .rx_valid_o    (rx_valid_sr),
119     .rx_ready_i    (rx_ready_sr),
120     .rx_last_o     (rx_last_sr),
121     .sw_rst_i,
122     .sd_o          (sd_o),
123     .sd_i          (sd_i)
124 );
125
126 spi_host_fsm #(
127     .NumCS(NumCS)
128 ) u_fsm (
129     .clk_i,
130     .rst_ni,
131     .en_i,
132     .command_i,
133     .command_csid_i,
134     .command_valid_i,
135     .command_ready_o,
136     .sck_o,
137     .csb_o,
138     .sd_en_o,
139     .last_read_o      (last_read),
140     .last_write_o     (last_write),
141     .wr_en_o          (wr_en),
142     .sr_wr_ready_i    (wr_ready),
143     .rd_en_o          (rd_en),
144     .sr_rd_ready_i    (rd_ready),
145     .sample_en_o      (sample_en),
146     .shift_en_o       (shift_en),
147     .speed_o          (speed),
148     .full_cyc_o       (full_cyc),
149     .sw_rst_i,
150     .rx_stall_o,
151     .tx_stall_o,
152     .active_o
153 );
154
155 endmodule : spi_host_core

```

7. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_data_fifos.v (135 lines)

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for

details.

```

3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Module for SPI_HOST RX and TX queues
6 //
7
8 `include "prim_assert.sv"
9
10 module spi_host_data_fifos #(
11     parameter int    TxDepth    = 72,
12     parameter int    RxDepth    = 64,
13     parameter logic   SwapBytes = 0
14 ) (
15     input            clk_i,
16     input            rst_ni,
17
18     input            [31:0] tx_data_i,
19     input            [3:0]  tx_be_i,
20     input            tx_valid_i,
21     output logic      tx_ready_o,
22     input            [7:0]  tx_watermark_i,
23
24     output logic [31:0] core_tx_data_o,
25     output logic [3:0]  core_tx_be_o,
26     output logic      core_tx_valid_o,
27     input            core_tx_ready_i,
28     input            core_tx_byte_select_full_i,
29
30     input            [31:0] core_rx_data_i,
31     input            core_rx_valid_i,
32     output logic      core_rx_ready_o,
33
34     output logic [31:0] rx_data_o,
35     output logic      rx_valid_o,
36     input            rx_ready_i,
37     input            [7:0]  rx_watermark_i,
38
39     input            sw_rst_i,
40
41     output logic      tx_empty_o,
42     output logic      tx_full_o,
43     output logic [7:0] tx_qd_o,
44     output logic      tx_wm_o,
45     output logic      rx_empty_o,
46     output logic      rx_full_o,
47     output logic [7:0] rx_qd_o,
48     output logic      rx_wm_o
49 );
50
51 localparam int RxDepthW = prim_util_pkg::vbits(RxDepth+1);
52 localparam int TxDepthW = prim_util_pkg::vbits(TxDepth+1);
53
54 logic [31:0] tx_data_ordered;
55 logic [3:0]  tx_be_ordered;
56 logic [31:0] rx_data_unordered;
57
58 if (SwapBytes) begin : gen_swap
59     assign tx_data_ordered = { << 8 {tx_data_i} };
60     assign tx_be_ordered   = { << {tx_be_i} };
61     assign rx_data_o       = { << 8 {rx_data_unordered} };
62 end else begin : gen_do_not_swap
63     assign tx_data_ordered = tx_data_i;
64     assign tx_be_ordered   = tx_be_i;
65     assign rx_data_o       = rx_data_unordered;
66 end : gen_do_not_swap
67
68 logic [35:0] tx_data_be;
69 logic [35:0] core_tx_data_be;
70
71 logic [TxDepthW-1:0] tx_depth;
72
73 // The byte_select module greedily pops data from the TX
FIFO, so
74 // the FIFO should be observed as having an effective depth
of N+1.
75 assign tx_qd_o = 8'(tx_depth) + {7'b0,
core_tx_byte_select_full_i};
76
77 assign tx_data_be = { tx_data_ordered, tx_be_ordered };
78 assign { core_tx_data_o, core_tx_be_o } = core_tx_data_be;
79
80 prim_fifo_sync #(
81     .Width(36),
82     .Pass(1),
83     .Depth(TxDepth)
84 ) u_tx_fifo (
85     .clk_i,
86     .rst_ni,
87     .clr_i   (sw_rst_i),
88     .wvalid_i (tx_valid_i),
89     .wready_o (tx_ready_o),
90     .wdata_i  (tx_data_be),
91     .rvalid_o (core_tx_valid_o),
92     .rready_i (core_tx_ready_i),
93     .rdata_o  (core_tx_data_be),
94     .full_o   (),
95     .depth_o  (tx_depth),
96     .err_o    ()
97 );
98
99 logic [RxDepthW-1:0] rx_depth;

```



```

100
101 assign rx_qd_o = 8'(rx_depth);
102
103 prim_fifo_sync #(
104     .Width(32),
105     .Pass(1),
106     .Depth(RxDepth)
107 ) u_rx_fifo (
108     .clk_i,
109     .rst_ni,
110     .clr_i (sw_rst_i),
111     .wvalid_i (core_rx_valid_i),
112     .wready_o (core_rx_ready_o),
113     .wdata_i (core_rx_data_i),
114     .rvalid_o (rx_valid_o),
115     .rready_i (rx_ready_i),
116     .rdata_o (rx_data_unordered),
117     .full_o (),
118     .depth_o (rx_depth),
119     .err_o ()
120 );
121
122 // Create the status outputs
123 // - Include the 1-word storage in byte_select for the TX
datapath
124 assign tx_empty_o = (tx_qd_o == 0);
125 assign rx_empty_o = (rx_qd_o == 0);
126 assign tx_full_o = (tx_qd_o >= 8'(TxDepth) + 1); // inc.
byte_sel
127 assign rx_full_o = (rx_qd_o >= 8'(RxDepth));
128 assign tx_wm_o = (tx_qd_o < tx_watermark_i);
129 assign rx_wm_o = (rx_qd_o >= rx_watermark_i);
130
131 // Due to the addition of the byte_sel as a N+1'th fifo
stage, ensure
132 // we have adequate resolution in our counter (tx_qd_o) to
represent this.
133 `ASSERT_INIT(TxDepth_A, TxDepth < (2**$size(tx_qd_o)) - 1)
134
135 endmodule : spi_host_data_fifos

```

8. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_fsm.sv (626 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Core Implementation module for Serial Peripheral Interface
(SPI) Host IP.
6 //
7
8 module spi_host_fsm
9     import spi_host_cmd_pkg::*;
10
11     #(
12         parameter int NumCS = 1,
13         localparam int CSW = prim_util_pkg::vbits(NumCS)
14     ) (
15         input clk_i,
16         input rst_ni,
17         input en_i,
18         input command_t command_i,
19         input logic [CSW-1:0] command_csid_i,
20         input logic [CSW-1:0] command_valid_i,
21         input logic [CSW-1:0] command_ready_o,
22         output logic [NumCS-1:0] sck_o,
23         output logic [3:0] csb_o,
24         output logic [3:0] sd_en_o,
25         output logic last_read_o,
26         output logic last_write_o,
27         output logic wr_en_o,
28         input sr_wr_ready_i,
29         output logic rd_en_o,
30         input sr_rd_ready_i,
31         output logic sample_en_o,
32         output logic shift_en_o,
33         output logic [1:0] speed_o,
34         output logic full_cyc_o,
35         output logic rx_stall_o,
36         output logic tx_stall_o,
37         output logic active_o,
38         input sw_rst_i
39     );
40
41     logic is_idle;
42     logic [15:0] clkdiv, clkdiv_q;
43     logic [15:0] clk_cntr_q, clk_cntr_d;
44     logic clk_cntr_en;
45
46     logic [CSW-1:0] csid;
47     logic [CSW-1:0] csid_q;
48
49     logic [3:0] csnidle, csntrail, csnlead;
50     logic [3:0] csnidle_q, csntrail_q, csnlead_q;
51     logic full_cyc_q, cpha, cpol;
52     logic full_cyc_q, cpha_q, cpol_q;
53

```

```

54 // Unlike the configopts fields, the segment fields can
change in back-to-back segment operation.
55 // (If a change in only the configopts fields is detected,
the FSM transitions to idle instead).
56 // For that reason, when using the segment fields in back-
to-back operations, we have to bear
57 // in mind context and determine whether it is appropriate
to use the value for the previous
58 // segment or the following segment.
59 // For instance, the cmd_rd_en signal is sometimes used to
push segment bytes into the shift
60 // register, and so in that case it is important to use
cmd_rd_en_q in that sense. The same
61 // signal is consulted to load the bit counter at the /be-
ginning/ of each byte or dummy cycle,
62 // and so in this context it is appropriate to use
cmd_rd_en_d (which refers to the value from
63 // the immediately following segment).
64 logic [1:0] cmd_speed_d, cmd_speed_q;
65 logic cmd_wr_en_d, cmd_wr_en_q;
66 logic cmd_rd_en_d, cmd_rd_en_q;
67 logic [19:0] cmd_len_d, cmd_len_q;
68 logic csat;
69 logic csat_q;
70
71 logic [2:0] bit_cntr_d, bit_cntr_q;
72 logic [19:0] byte_cntr_cpha0_d, byte_cntr_cpha1_d,
byte_cntr_cpha0_q, byte_cntr_cpha1_q;
73 logic [19:0] byte_cntr_early, byte_cntr_late;
74 logic [3:0] wait_cntr_d, wait_cntr_q;
75 logic last_bit, last_byte;
76
77 logic state_changing;
78 logic byte_starting, byte_starting_cpha0,
byte_starting_cpha1;
79 logic bit_shifting, bit_shifting_cpha0,
bit_shifting_cpha1;
80 logic byte_ending, byte_ending_cpha0, byte_end-
ing_cpha1;
81
82 logic sample_en_d, sample_en_q, sample_en_q2;
83
84 logic config_changed;
85 logic fsm_en;
86
87 // new_command: signals a new segment input
88 logic new_command;
89
90 logic csb_single_d;
91 logic [NumCS-1:0] csb_q;
92 logic sck_d, sck_q;
93
94 logic wr_en_internal, rd_en_internal, sample_en_internal,
shift_en_internal;
95
96 logic stall;
97
98 assign stall = rx_stall_o | tx_stall_o;
99
100 // suppress output pulses if stalled.
101 assign wr_en_o = wr_en_internal & ~stall;
102 assign rd_en_o = rd_en_internal & ~stall;
103 assign sample_en_o = sample_en_internal & ~stall;
104 assign shift_en_o = shift_en_internal & ~stall;
105
106 typedef enum logic [2:0] {
107     Idle,
108     WaitLead,
109     InternalClkLow,
110     InternalClkHigh,
111     WaitTrail,
112     WaitIdle,
113     CSBSwitch,
114     IdleCSBActive
115 } spi_host_st_e;
116
117 spi_host_st_e state_q, state_d;
118
119 logic command_ready_int;
120 assign command_ready_o = command_ready_int & ~stall;
121
122
123 assign new_command = command_valid_i && com-
mand_ready_int;
124 assign config_changed = (command_i.configopts.cpol !=
cpol_q) ||
125 (command_i.configopts.cpha !=
cpha_q) ||
126 (command_i.configopts.full_cyc !=
full_cyc_q) ||
127 (command_i.configopts.csnidle !=
csnidle_q) ||
128 (command_i.configopts.csntrail !=
csntrail_q) ||
129 (command_i.configopts.csnlead !=
csnlead_q) ||
130 (command_i.configopts.clkdiv !=
clkdiv_q);
131
132 always_comb begin
133     csid = new_command ? command_csid_i : csid_q;

```

```

134   cpol      = new_command ? command_i.configopts.cpol :
cpol_q;
135   cpha      = new_command ? command_i.configopts.cpha :
cpha_q;
136   full_cyc  = new_command ? command_i.configopts.full_cyc :
full_cyc_q;
137   csnidle   = new_command ? command_i.configopts.csnidle :
csnidle_q;
138   csnlead   = new_command ? command_i.configopts.csnlead :
csnlead_q;
139   csntrail  = new_command ? command_i.configopts.csntrail :
csntrail_q;
140   clkdiv    = new_command ? command_i.configopts.clkdiv :
clkdiv_q;
141   csaat     = new_command ? command_i.segment.csaat :
csaat_q;
142   cmd_len_d = new_command ? command_i.segment.len :
cmd_len_q;
143   cmd_wr_en_d = new_command ? command_i.segment.cmd_wr_en :
cmd_wr_en_q;
144   cmd_rd_en_d = new_command ? command_i.segment.cmd_rd_en :
cmd_rd_en_q;
145   cmd_speed_d = new_command ? command_i.segment.speed :
cmd_speed_q;
146   end
147
148   always_ff @(posedge clk_i or negedge rst_ni) begin
149     if (!rst_ni) begin
150       csid_q    <= {CSW{1'b0}};
151       cpol_q    <= 1'b0;
152       cpha_q    <= 1'b0;
153       full_cyc_q <= 1'b0;
154       csnidle_q <= 4'h0;
155       csnlead_q <= 4'h0;
156       csntrail_q <= 4'h0;
157       clkdiv_q  <= 16'h0;
158       csaat_q   <= 1'b0;
159       cmd_rd_en_q <= 1'b0;
160       cmd_wr_en_q <= 1'b0;
161       cmd_speed_q <= 2'b00;
162       cmd_len_q <= 20'h0;
163     end else begin
164       csid_q    <= (new_command && !stall) ? csid : csid_q;
165       cpol_q    <= (new_command && !stall) ? cpol : cpol_q;
166       cpha_q    <= (new_command && !stall) ? cpha : cpha_q;
167       full_cyc_q <= (new_command && !stall) ? full_cyc :
full_cyc_q;
168       csnidle_q <= (new_command && !stall) ? csnidle :
csnidle_q;
169       csnlead_q <= (new_command && !stall) ? csnlead : csn-
lead_q;
170       csntrail_q <= (new_command && !stall) ? csntrail :
csntrail_q;
171       clkdiv_q  <= (new_command && !stall) ? clkdiv : clk-
div_q;
172       csaat_q   <= (new_command && !stall) ? csaat :
csaat_q;
173       cmd_wr_en_q <= (new_command && !stall) ? cmd_wr_en_d :
cmd_wr_en_q;
174       cmd_rd_en_q <= (new_command && !stall) ? cmd_rd_en_d :
cmd_rd_en_q;
175       cmd_speed_q <= (new_command && !stall) ? cmd_speed_d :
cmd_speed_q;
176       cmd_len_q <= (new_command && !stall) ? cmd_len_d :
cmd_len_q;
177     end
178   end
179
180   assign is_idle    = (state_q == Idle) || (state_q == IdleC-
SBActive);
181
182   assign active_o   = ~is_idle;
183
184   // New commands must always ensure there is the full setup
time available.
185   assign clk_cntr_d = sw_rst_i      ? 16'h0 :
186                       !clk_cntr_en  ? clk_cntr_q :
187                       new_command   ? clkdiv :
188                       is_idle       ? clk_cntr_q :
189                       (clk_cntr_q == 16'h0) ? clkdiv :
190                       clk_cntr_q - 1;
191
192   assign tx_stall_o = wr_en_internal & ~sr_wr_ready_i;
193   assign rx_stall_o = rd_en_internal & ~sr_rd_ready_i;
194   assign clk_cntr_en = en_i;
195   assign fsm_en     = (clk_cntr_en && ((clk_cntr_q == 0) ||
is_idle));
196
197   spi_host_st_e next_state_after_idle;
198   always_comb begin
199     if (command_valid_i) begin
200       if (config_changed) begin
201         next_state_after_idle = CSBSwitch;
202       end else begin
203         next_state_after_idle = WaitLead;
204       end
205     end else begin
206       next_state_after_idle = Idle;
207     end
208   end
209

```

```

210   spi_host_st_e next_state_after_idle_csb_active;
211   logic          command_ready_idle_csb_active;
212   always_comb begin
213     if (command_valid_i) begin
214       if (command_csid_i != csid_q) begin
215         //
216         // Do not acknowledge the command now, as it will
trigger
217
218         // an update of the internal command and configuration
registers.
219         // *Silently* transition to WaitTrail. The command
// will be acknowledged later, at the end of the
WaitIdle state.
220         //
221         next_state_after_idle_csb_active = WaitTrail;
222         // Explicitly *suppress* command_ready
command_ready_idle_csb_active = 1'b0;
223       end else begin
224         if (cpha_q) begin
225           next_state_after_idle_csb_active = InternalClkHigh;
226         end else begin
227           next_state_after_idle_csb_active = InternalClkLow;
228         end
229         command_ready_idle_csb_active = 1'b1;
230       end
231     end else begin
232       next_state_after_idle_csb_active = IdleCSBActive;
233       command_ready_idle_csb_active = 1'b1;
234     end
235   end
236 end
237
238 //
239 // FSM main body: Controls state transitions and com-
mand_ready_o signaling
240 //
241 always_comb begin
242   state_d = state_q;
243   command_ready_int = 1'b0;
244   if (sw_rst_i) begin
245     state_d = Idle;
246   end else if (fsm_en) begin
247     unique case (state_q)
248       Idle: begin
249         // Initial state, wait for commands.
250         command_ready_int = 1'b1;
251         state_d = next_state_after_idle;
252       end
253       WaitLead: begin
254         // Transaction lead: CSB is low, waiting to start
sck pulses.
255         if (wait_cntr_q == 4'h0) begin
256           state_d = InternalClkHigh;
257         end
258       end
259       InternalClkLow: begin
260         // Launch clock and full-cycle sampling clock for
CPHA=0.
261         // Half-cycle sampling clock for CPHA=1.
262         // End-of-transaction clock level for both CPHA=0
and CPHA=1.
263         // One of two active clock states. sck is low when
CPOL=0.
264         if (!last_bit || !last_byte || !cpha_q) begin
265           state_d = InternalClkHigh;
266         end else if (!csaat_q) begin
267           state_d = WaitTrail;
268         end else begin
269           state_d = next_state_after_idle_csb_active;
270           command_ready_int = command_ready_idle_csb_active;
271         end
272       end
273       InternalClkHigh: begin
274         // Half-cycle sampling clock for CPHA=0.
275         // Launch clock for CPHA=1. Note: Full-cycle
sampling is not compatible with CPHA=1.
276         // One of two active clock states. sck is low when
CPOL=0.
277         // Typically often the last state in a command, and
so the next state depends on CSAAT,
278         // and of CSAAT is asserted, the details of the sub-
sequent command.
279         if (!last_bit || !last_byte || cpha_q) begin
280           state_d = InternalClkLow;
281         end
282       end
283       // Check value of csaat for the previously submitted
segment
284       end else if (!csaat_q) begin
285         state_d = WaitTrail;
286       end else begin
287         state_d = next_state_after_idle_csb_active;
288         command_ready_int = command_ready_idle_csb_active;
289       end
290     end
291   end
292   WaitTrail: begin
293     // Prepare to enter CSB high idle state by waiting
csntrail cycles.
294     if (wait_cntr_q == 4'h0) begin
295       state_d = WaitIdle;
296     end
297   end
298 end

```

```

295     WaitIdle: begin
296         // Once CSB is high, wait for the designated number
of cycles before accepting commands.
297         if (wait_cntr_q == 4'h0) begin
298             // ready to accept new command
299             command_ready_int = 1'b1;
300             state_d = next_state_after_idle;
301         end
302     end
303     CSBSwitch: begin
304         // Insert extra idle cycles when swtiching between
CSID, this allows time to switch CPHA,
305         // CPOL and clkdiv settings, as well as guarantee
that the idle delay requirements have
306         // been observed for the new device.
307         if (wait_cntr_q == 4'h0) begin
308             state_d = WaitLead;
309         end else begin
310             state_d = CSBSwitch;
311         end
312     end
313     IdleCSBActive: begin
314         // Wait for new commands, but with CSB held active
low.
315         state_d = next_state_after_idle_csb_active;
316         command_ready_int = command_ready_idle_csb_active;
317     end
318     default: begin
319         command_ready_int = 1'b0;
320         state_d = Idle;
321     end
322 endcase
323 end
324 end
325
326 // All register updates freeze when a stall is detected.
327 // The definition of the stall signal looks ahead to determ-
ine whether a conflict is looming.
328 // Thus stall depends on state_d. Making state_d depend on
stall
329 // would create a circular logic loop, and lint errors.
Therefore stall is applied here, not
330 // in the previous always_comb block;
331 always_ff @(posedge clk_i or negedge rst_ni) begin
332     if (!rst_ni) begin
333         state_q <= Idle;
334         clk_cntr_q <= 16'h0;
335     end else begin
336         state_q <= stall ? state_q : state_d;
337         clk_cntr_q <= stall ? clk_cntr_q : clk_cntr_d;
338     end
339 end
340
341 logic segment_rd_en, segment_rd_en_cpha0, seg-
ment_rd_en_cpha1;
342
343 assign state_changing = (state_q != state_d);
344 assign byte_starting_cpha0 = ~sw_rst_i & state_changing &
((state_d == WaitLead) |
345 (state_d == InternalClkLow &
bit_cntr_q==0));
346 assign bit_shifting_cpha0 = ~sw_rst_i & state_changing &
(state_d == InternalClkLow &
bit_cntr_q != 0);
347 assign byte_ending_cpha0 = ~sw_rst_i & state_changing &
(state_q == InternalClkHigh &
bit_cntr_q == 0);
348
349 assign segment_rd_en_cpha0 = cmd_rd_en_q;
350
351 // We can calculate byte transitions for CPHA=1 by noting
352 // that in this implementation, the sck edges have a 1-1
353 // correspondence with FSM transitions.
354 // New bytes are loaded exactly one state transition behind
the time
355 // when they would be loaded if CPHA=0
356 always_ff @(posedge clk_i or negedge rst_ni) begin
357     if (!rst_ni) begin
358         segment_rd_en_cpha1 <= 1'b0;
359     end else if (state_changing && !stall) begin
360         segment_rd_en_cpha1 <= segment_rd_en_cpha0;
361     end
362 end
363
364 // For CPHA=1, the launch and capture states flip roles, and
data output
365 // does not begin in the WaitIdle state, when CSB first as-
serts. They
366 // otherwise look very similar to the CPHA=0 case.
367 assign byte_starting_cpha1 = ~sw_rst_i & state_changing &
(state_d == InternalClkHigh &
bit_cntr_q==0);
368 assign bit_shifting_cpha1 = ~sw_rst_i & state_changing &
(state_d == InternalClkHigh &
bit_cntr_q != 0);
369 assign byte_ending_cpha1 = ~sw_rst_i & state_changing &
(state_q == InternalClkLow &
bit_cntr_q == 0);
370
371
372

```

```

378 assign byte_starting = (cpha == 1'b0) ? byte_start-
ing_cpha0 :
379 byte_starting_cpha1;
380
381 assign byte_ending = (cpha == 1'b0) ? byte_ending_cpha0 :
byte_ending_cpha1;
382
383 assign bit_shifting = (cpha == 1'b0) ? bit_shifting_cpha0 :
bit_shifting_cpha1;
384
385 assign speed_o = cmd_speed_q;
386
387 assign segment_rd_en = (cpha == 1'b0) ? segment_rd_en_cpha0 :
segment_rd_en_cpha1;
388
389 assign byte_cntr_early = (cpha == 1'b0) ?
byte_cntr_cpha0_d :
390 byte_cntr_cpha1_d;
391
392 assign byte_cntr_late = (cpha == 1'b0) ?
byte_cntr_cpha0_q :
393 byte_cntr_cpha1_q;
394
395 logic [2:0] shift_size;
396 logic [2:0] start_bit;
397
398 always_comb begin
399     if (!cmd_rd_en_d && !cmd_wr_en_d) begin
400         // direction == 0, means to send out
401         // a fixed number of pulses, NOT bytes.
402         // thus "last_bit" is always asserted,
403         // and the number of pulses is counted
404         // by byte_cntr_d.
405         shift_size = 0;
406         start_bit = 3'h0;
407     end else begin
408         unique case (cmd_speed_d)
409             Standard: begin
410                 shift_size = 3'h1;
411                 start_bit = 3'h7;
412             end
413             Dual: begin
414                 shift_size = 3'h2;
415                 start_bit = 3'h6;
416             end
417             Quad: begin
418                 shift_size = 3'h4;
419                 start_bit = 3'h4;
420             end
421             default: begin
422                 // Invalid speed;
423                 shift_size = 3'h1;
424                 start_bit = 3'h1;
425             end
426         endcase
427     end
428 end
429
430 assign bit_cntr_d = sw_rst_i ? 3'h0 :
!fsm_en ? bit_cntr_q :
431 byte_starting ? start_bit :
432 bit_shifting ? bit_cntr_q -
shift_size :
433 bit_cntr_q;
434
435 assign last_bit = (bit_cntr_q == 3'h0);
436
437 // The variable last_byte is only used for updating the FSM
state.
438 // For CPHA=1 operation, either byte_cntr_cpha0_q or
byte_cntr_cpha1_q
439 // can drive the FSM properly. However, we explicitly
choose
440 // byte_cntr_cpha0_q to avoid a combinational logic loop.
441 //
442 always_comb begin
443     if (cpha_q) begin
444         last_byte = (byte_cntr_cpha1_q == 20'h0);
445     end else begin
446         last_byte = (byte_cntr_cpha0_q == 20'h0);
447     end
448 end
449
450 // Note: when updating the byte_cntr in CPHA=0 mode with a
new command value, the length must
451 // be pulled in directly from the command bus, cmd_len_d;
452 assign byte_cntr_cpha0_d = sw_rst_i ? 20'h0 :
!fsm_en ? byte_cntr_cpha0_q :
453 new_command ? cmd_len_d :
454 byte_ending_cpha0 ?
byte_cntr_cpha0_q - 1 :
455 byte_cntr_cpha0_q;
456
457 assign byte_cntr_cpha1_d = sw_rst_i ? 20'h0 :
!fsm_en ?
458 new_command ? cmd_len_d :
459 byte_ending_cpha1 ?
byte_cntr_cpha1_q - 1 :
460 byte_cntr_cpha1_q;
461
462 always_comb begin

```



```

468 if(sw_rst_i) begin
469     wait_cntr_d = 4'b0;
470 end else if (!fsm_en) begin
471     wait_cntr_d = wait_cntr_q;
472 end else if (state_changing) begin
473     unique case (state_d)
474         WaitLead: begin
475             wait_cntr_d = csnlead;
476         end
477         WaitTrail: begin
478             wait_cntr_d = csntrail;
479         end
480         WaitIdle: begin
481             wait_cntr_d = csnidle;
482         end
483         CSBSwitch: begin
484             wait_cntr_d = csnidle;
485         end
486         default: begin
487             // Hold wait_cntr to zero
488             // for states that don't use it
489             wait_cntr_d = 4'b0;
490         end
491     endcase
492 end else if (wait_cntr_q == 0) begin
493     wait_cntr_d = 4'h0;
494 end else begin
495     wait_cntr_d = wait_cntr_q - 1;
496 end
497 end
498
499 always_ff @(posedge clk_i or negedge rst_ni) begin
500     if (!rst_ni) begin
501         bit_cntr_q      <= 3'h0;
502         byte_cntr_cpha0_q <= 20'h0;
503         byte_cntr_cpha1_q <= 20'h0;
504         wait_cntr_q      <= 4'h0;
505     end else begin
506         bit_cntr_q      <= stall ? bit_cntr_q      :
507         byte_cntr_cpha0_q <= stall ? byte_cntr_cpha0_q :
508         byte_cntr_cpha1_q <= stall ? byte_cntr_cpha1_q :
509         wait_cntr_q      <= stall ? wait_cntr_q      :
510     end
511 end
512
513 assign wr_en_internal = byte_starting & cmd_wr_en_d;
514 assign shift_en_internal = bit_shifting;
515
516 assign rd_en_internal = byte_ending & segment_rd_en;
517 assign sample_en_d = byte_starting | shift_en_o;
518 assign full_cyc_o = full_cyc;
519
520 assign last_read_o = (byte_cntr_late == 'h0) & rd_en_o &
521 sr_rd_ready_i;
522
523 assign last_write_o = (byte_cntr_early == 'h0) &
524 wr_en_o & sr_wr_ready_i;
525
526 always_ff @(posedge clk_i or negedge rst_ni) begin
527     if (!rst_ni) begin
528         sample_en_q <= 1'b0;
529         sample_en_q2 <= 1'b0;
530     end else begin
531         sample_en_q <= (fsm_en && !stall) ? sample_en_d :
532         sample_en_q2 <= (fsm_en && !stall) ? sample_en_q :
533         sample_en_q2;
534     end
535 end
536
537 assign sample_en_internal = full_cyc_o ? sample_en_q2 :
538 sample_en_q;
539
540 always_comb begin
541     unique case (state_d)
542         WaitLead, InternalClkLow, InternalClkHigh, IdleCSBAct-
543         ive, WaitTrail:
544             csb_single_d = 1'b0;
545         default:
546             csb_single_d = 1'b1;
547     endcase
548 end
549
550 assign sck_d = cpol ? (state_d != InternalClkHigh) :
551 (state_d == InternalClkHigh);
552
553 assign sck_o = sck_q;
554
555 prim_flop_en u_sck_flop (
556     .clk_i,
557     .rst_ni,
558     .en_i(~stall),
559     .d_i(sck_d),
560     .q_o(sck_q)
561 );
562
563 for (genvar ii = 0; ii < NumCS; ii = ii + 1) begin :

```

```

gen_csb_gen
558 always_ff @(posedge clk_i or negedge rst_ni) begin
559     if (!rst_ni) begin
560         csb_q[ii] <= 1'b1;
561     end else begin
562         csb_q[ii] <= (csid != ii) ? 1'b1 :
563         !stall ? csb_single_d :
564         csb_q[ii];
565     end
566 end
567 end : gen_csb_gen
568
569 // When 'cmd_wr_en_q' deasserts, we need to stretch the en-
570 able used
571 // to drive sd_en_o[0] until the end of the bit period. This
572 is because
573 // 'cmd_wr_en_q' can change up to 1/2 cycle before the end
574 of the period
575 // (depending on cpol/cpha).
576 // The signal 'last_bit' is active in the final period, so
577 use that to start
578
579 // stretching the enable, and 'byte_ending' marks the period
580 end.
581 logic cmd_wr_en_last_bit;
582 always_ff @(posedge clk_i or negedge rst_ni) begin
583     if (!rst_ni) begin
584         cmd_wr_en_last_bit <= 1'b0;
585     end else if (cmd_wr_en_q & last_bit) begin
586         cmd_wr_en_last_bit <= 1'b1;
587     end else if (byte_ending) begin
588         cmd_wr_en_last_bit <= 1'b0;
589     end else begin // Stretching..
590         cmd_wr_en_last_bit <= cmd_wr_en_last_bit;
591     end
592 end
593
594 assign csb_o = csb_q;
595
596 always_comb begin
597     if (&csb_o) begin
598         sd_en_o[3:0] = 4'h0;
599     end else begin
600         unique case (speed_o)
601             Standard: begin
602                 // Observing 'last_bit' ensures we do not deassert
603                 the enable too early
604                 sd_en_o[0] = cmd_wr_en_q | cmd_wr_en_last_bit;
605                 sd_en_o[1] = 1'b0;
606                 sd_en_o[3:2] = 2'b00;
607             end
608             Dual: begin
609                 sd_en_o[1:0] = {2{cmd_wr_en_q}};
610                 sd_en_o[3:2] = 2'b00;
611             end
612             Quad: begin
613                 sd_en_o[3:0] = {4{cmd_wr_en_q}};
614             end
615             default: begin
616                 // invalid speed
617                 sd_en_o[3:0] = 4'h0;
618             end
619         endcase
620     end
621 end
622
623 // Assertions confirming valid user input.
624 //
625 `ASSERT(BidirOnlyInStdMode_A,
626 cmd_speed_d == Standard || !(cmd_rd_en_d &&
627 cmd_wr_en_d),
628 clk_i, rst_ni)
629 `ASSERT(ValidSpeed_A, cmd_speed_d != RsVdSpd, clk_i, rst_ni)
630 `ASSERT(ValidCSID_A, csid < NumCS, clk_i, rst_ni)
631
632 endmodule

```

9. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_reg_pkg.sv (375 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 //
6 // Register Package auto-generated by `reggen` containing data
7 structure
8
9 package spi_host_reg_pkg;
10
11 // Param list
12 parameter logic ByteOrder = 1;
13 parameter int TxDepth = 72;
14 parameter int RxDepth = 64;
15 parameter int CmdDepth = 4;

```

```

14  parameter int NumAlerts = 1;
15
16  // Address widths within the block
17  parameter int BlockAw = 6;
18
19  // Number of registers for every interface
20  parameter int NumRegs = 12;
21
22  // Alert indices
23  typedef enum int {
24      AlertFatalFaultIdx = 0
25  } spi_host_alert_idx_t;
26
27  ///////////////////////////////////////////////////
28  // Typedefs for registers //
29  ///////////////////////////////////////////////////
30
31  typedef struct packed {
32      struct packed {
33          logic q;
34      } spi_event;
35      struct packed {
36          logic q;
37      } error;
38  } spi_host_reg2hw_intr_state_reg_t;
39
40  typedef struct packed {
41      struct packed {
42          logic q;
43      } spi_event;
44      struct packed {
45          logic q;
46      } error;
47  } spi_host_reg2hw_intr_enable_reg_t;
48
49  typedef struct packed {
50      struct packed {
51          logic q;
52          logic qe;
53      } spi_event;
54      struct packed {
55          logic q;
56          logic qe;
57      } error;
58  } spi_host_reg2hw_intr_test_reg_t;
59
60  typedef struct packed {
61      logic q;
62      logic qe;
63  } spi_host_reg2hw_alert_test_reg_t;
64
65  typedef struct packed {
66      struct packed {
67          logic q;
68      } spien;
69      struct packed {
70          logic q;
71      } sw_rst;
72      struct packed {
73          logic q;
74      } output_en;
75      struct packed {
76          logic [7:0] q;
77      } tx_watermark;
78      struct packed {
79          logic [7:0] q;
80      } rx_watermark;
81  } spi_host_reg2hw_control_reg_t;
82
83  typedef struct packed {
84      struct packed {
85          logic q;
86      } cpol;
87      struct packed {
88          logic q;
89      } cpha;
90      struct packed {
91          logic q;
92      } fullcyc;
93      struct packed {
94          logic [3:0] q;
95      } csnlead;
96      struct packed {
97          logic [3:0] q;
98      } csntrail;
99      struct packed {
100         logic [3:0] q;
101     } csnidle;
102     struct packed {
103         logic [15:0] q;
104     } clkdiv;
105 } spi_host_reg2hw_configopts_reg_t;
106
107 typedef struct packed {
108     logic [31:0] q;
109 } spi_host_reg2hw_csid_reg_t;
110
111 typedef struct packed {
112     struct packed {
113         logic [19:0] q;
114         logic qe;
115     } len;
116     struct packed {
117         logic [1:0] q;
118         logic qe;
119     } direction;
120     struct packed {
121         logic [1:0] q;
122         logic qe;
123     } speed;
124     struct packed {
125         logic q;
126         logic qe;
127     } csaat;
128 } spi_host_reg2hw_command_reg_t;
129
130 typedef struct packed {
131     struct packed {
132         logic q;
133     } csidinval;
134     struct packed {
135         logic q;
136     } cmdinval;
137     struct packed {
138         logic q;
139     } underflow;
140     struct packed {
141         logic q;
142     } overflow;
143     struct packed {
144         logic q;
145     } cmdbusy;
146 } spi_host_reg2hw_error_enable_reg_t;
147
148 typedef struct packed {
149     struct packed {
150         logic q;
151     } accessinval;
152     struct packed {
153         logic q;
154     } csidinval;
155     struct packed {
156         logic q;
157     } cmdinval;
158     struct packed {
159         logic q;
160     } underflow;
161     struct packed {
162         logic q;
163     } overflow;
164     struct packed {
165         logic q;
166     } cmdbusy;
167 } spi_host_reg2hw_error_status_reg_t;
168
169 typedef struct packed {
170     struct packed {
171         logic q;
172     } idle;
173     struct packed {
174         logic q;
175     } ready;
176     struct packed {
177         logic q;
178     } txwm;
179     struct packed {
180         logic q;
181     } rxwm;
182     struct packed {
183         logic q;
184     } txempty;
185     struct packed {
186         logic q;
187     } rxfull;
188 } spi_host_reg2hw_event_enable_reg_t;
189
190 typedef struct packed {
191     struct packed {
192         logic d;
193         logic de;
194     } spi_event;
195     struct packed {
196         logic d;
197         logic de;
198     } error;
199 } spi_host_hw2reg_intr_state_reg_t;
200
201 typedef struct packed {
202     struct packed {
203         logic d;
204         logic de;
205     } ready;
206     struct packed {
207         logic d;
208         logic de;
209     } active;
210     struct packed {
211         logic d;
212         logic de;
213     } txfull;
214     struct packed {
215         logic d;

```

```

216     logic      de;
217 } txdempty;
218 struct packed {
219     logic      d;
220     logic      de;
221 } txstall;
222 struct packed {
223     logic      d;
224     logic      de;
225 } txwm;
226 struct packed {
227     logic      d;
228     logic      de;
229 } rxfull;
230 struct packed {
231     logic      d;
232     logic      de;
233 } rxempty;
234 struct packed {
235     logic      d;
236     logic      de;
237 } rxstall;
238 struct packed {
239     logic      d;
240     logic      de;
241 } byteorder;
242 struct packed {
243     logic      d;
244     logic      de;
245 } rxwm;
246 struct packed {
247     logic [3:0] d;
248     logic      de;
249 } cmdqd;
250 struct packed {
251     logic [7:0] d;
252     logic      de;
253 } rxqd;
254 struct packed {
255     logic [7:0] d;
256     logic      de;
257 } txqd;
258 } spi_host_hw2reg_status_reg_t;
259
260 typedef struct packed {
261     struct packed {
262         logic      d;
263         logic      de;
264     } accessinval;
265     struct packed {
266         logic      d;
267         logic      de;
268     } csidinval;
269     struct packed {
270         logic      d;
271         logic      de;
272     } cmdinval;
273     struct packed {
274         logic      d;
275         logic      de;
276     } underflow;
277     struct packed {
278         logic      d;
279         logic      de;
280     } overflow;
281     struct packed {
282         logic      d;
283         logic      de;
284     } cmdbusy;
285 } spi_host_hw2reg_error_status_reg_t;
286
287 // Register -> HW type
288 typedef struct packed {
289     spi_host_reg2hw_intr_state_reg_t intr_state; // [137:136]
290     spi_host_reg2hw_intr_enable_reg_t intr_enable; //
291     spi_host_reg2hw_intr_test_reg_t intr_test; // [133:130]
292     spi_host_reg2hw_alert_test_reg_t alert_test; // [129:128]
293     spi_host_reg2hw_control_reg_t control; // [127:109]
294     spi_host_reg2hw_configopts_reg_t configopts; // [108:78]
295     spi_host_reg2hw_csid_reg_t csid; // [77:46]
296     spi_host_reg2hw_command_reg_t command; // [45:17]
297     spi_host_reg2hw_error_enable_reg_t error_enable; //
298     spi_host_reg2hw_error_status_reg_t error_status; // [11:6]
299     spi_host_reg2hw_event_enable_reg_t event_enable; // [5:0]
300 } spi_host_reg2hw_t;
301
302 // HW -> register type
303 typedef struct packed {
304     spi_host_hw2reg_intr_state_reg_t intr_state; // [60:57]
305     spi_host_hw2reg_status_reg_t status; // [56:12]
306     spi_host_hw2reg_error_status_reg_t error_status; // [11:0]
307 } spi_host_hw2reg_t;
308
309 // Register offsets
310 parameter logic [BlockAw-1:0] SPI_HOST_INTR_STATE_OFFSET =
6'h 0;
311 parameter logic [BlockAw-1:0] SPI_HOST_INTR_ENABLE_OFFSET =
6'h 4;
312 parameter logic [BlockAw-1:0] SPI_HOST_INTR_TEST_OFFSET =

```

```

6'h 8;
313 parameter logic [BlockAw-1:0] SPI_HOST_ALERT_TEST_OFFSET =
6'h c;
314 parameter logic [BlockAw-1:0] SPI_HOST_CONTROL_OFFSET = 6'h
10;
315 parameter logic [BlockAw-1:0] SPI_HOST_STATUS_OFFSET = 6'h
14;
316 parameter logic [BlockAw-1:0] SPI_HOST_CONFIGOPTS_OFFSET =
6'h 18;
317 parameter logic [BlockAw-1:0] SPI_HOST_CSID_OFFSET = 6'h 1c;
318 parameter logic [BlockAw-1:0] SPI_HOST_COMMAND_OFFSET = 6'h
20;
319 parameter logic [BlockAw-1:0] SPI_HOST_ERROR_ENABLE_OFFSET =
6'h 2c;
320 parameter logic [BlockAw-1:0] SPI_HOST_ERROR_STATUS_OFFSET =
6'h 30;
321 parameter logic [BlockAw-1:0] SPI_HOST_EVENT_ENABLE_OFFSET =
6'h 34;
322
323 // Reset values for hwext registers and their fields
324 parameter logic [1:0] SPI_HOST_INTR_TEST_RESVAL = 2'h 0;
325 parameter logic [0:0] SPI_HOST_INTR_TEST_ERROR_RESVAL = 1'h
0;
326 parameter logic [0:0] SPI_HOST_INTR_TEST_SPI_EVENT_RESVAL =
1'h 0;
327 parameter logic [0:0] SPI_HOST_ALERT_TEST_RESVAL = 1'h 0;
328 parameter logic [0:0] SPI_HOST_ALERT_TEST_FATAL_FAULT_RESVAL =
1'h 0;
329 parameter logic [24:0] SPI_HOST_COMMAND_RESVAL = 25'h 0;
330 parameter logic [0:0] SPI_HOST_COMMAND_CSAAT_RESVAL = 1'h 0;
331 parameter logic [1:0] SPI_HOST_COMMAND_SPEED_RESVAL = 2'h 0;
332 parameter logic [1:0] SPI_HOST_COMMAND_DIRECTION_RESVAL =
2'h 0;
333 parameter logic [19:0] SPI_HOST_COMMAND_LEN_RESVAL = 20'h 0;
334
335 // Window parameters
336 parameter logic [BlockAw-1:0] SPI_HOST_RXDATA_OFFSET = 6'h
24;
337 parameter int unsigned SPI_HOST_RXDATA_SIZE = 'h 4;
338 parameter int unsigned SPI_HOST_RXDATA_IDX = 0;
339 parameter logic [BlockAw-1:0] SPI_HOST_TXDATA_OFFSET = 6'h
28;
340 parameter int unsigned SPI_HOST_TXDATA_SIZE = 'h 4;
341 parameter int unsigned SPI_HOST_TXDATA_IDX = 1;
342
343 // Register index
344 typedef enum int {
345     SPI_HOST_INTR_STATE,
346     SPI_HOST_INTR_ENABLE,
347     SPI_HOST_INTR_TEST,
348     SPI_HOST_ALERT_TEST,
349     SPI_HOST_CONTROL,
350     SPI_HOST_STATUS,
351     SPI_HOST_CONFIGOPTS,
352     SPI_HOST_CSID,
353     SPI_HOST_COMMAND,
354     SPI_HOST_ERROR_ENABLE,
355     SPI_HOST_ERROR_STATUS,
356     SPI_HOST_EVENT_ENABLE
357 } spi_host_id_e;
358
359 // Register width information to check illegal writes
360 parameter logic [3:0] SPI_HOST_PERMIT [12] = '{
361     4'b 0001, // index[ 0] SPI_HOST_INTR_STATE
362     4'b 0001, // index[ 1] SPI_HOST_INTR_ENABLE
363     4'b 0001, // index[ 2] SPI_HOST_INTR_TEST
364     4'b 0001, // index[ 3] SPI_HOST_ALERT_TEST
365     4'b 1111, // index[ 4] SPI_HOST_CONTROL
366     4'b 1111, // index[ 5] SPI_HOST_STATUS
367     4'b 1111, // index[ 6] SPI_HOST_CONFIGOPTS
368     4'b 1111, // index[ 7] SPI_HOST_CSID
369     4'b 1111, // index[ 8] SPI_HOST_COMMAND
370     4'b 0001, // index[ 9] SPI_HOST_ERROR_ENABLE
371     4'b 0001, // index[10] SPI_HOST_ERROR_STATUS
372     4'b 0001 // index[11] SPI_HOST_EVENT_ENABLE
373 };
374
375 endpackage

```

10. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_reg_top.v (2,055 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"
8
9 module spi_host_reg_top
10 # (
11     parameter bit      EnableRacl          = 1'b0,
12     parameter bit      RaclErrorRsp       = 1'b1,

```

```

13   parameter top_racl_pkg::racl_policy_sel_t RaclPoli-
cySelVec[spi_host_reg_pkg::NumRegs] =
14   '{spi_host_reg_pkg::NumRegs{0}}
15   ) (
16   input clk_i,
17   input rst_ni,
18   input tlul_pkg::tl_h2d_t tl_i,
19   output tlul_pkg::tl_d2h_t tl_o,
20
21   // Output port for window
22   output tlul_pkg::tl_h2d_t tl_win_o [2],
23   input tlul_pkg::tl_d2h_t tl_win_i [2],
24
25   // To HW
26   output spi_host_reg_pkg::spi_host_reg2hw_t reg2hw, // Write
27   input spi_host_reg_pkg::spi_host_hw2reg_t hw2reg, // Read
28
29   // RACL interface
30   input top_racl_pkg::racl_policy_vec_t racl_policies_i,
31   output top_racl_pkg::racl_error_log_t racl_error_o,
32
33   // Integrity check errors
34   output logic intg_err_o
35 );
36
37 import spi_host_reg_pkg::* ;
38
39 localparam int AW = 6;
40 localparam int DW = 32;
41 localparam int DBW = DW/8; // Byte Width
42
43 // register signals
44 logic reg_we;
45 logic reg_re;
46 logic [AW-1:0] reg_addr;
47 logic [DW-1:0] reg_wdata;
48 logic [DBW-1:0] reg_be;
49 logic [DW-1:0] reg_rdata;
50 logic reg_error;
51
52 logic addrmiss, wr_err;
53
54 logic [DW-1:0] reg_rdata_next;
55 logic reg_busy;
56
57 tlul_pkg::tl_h2d_t tl_reg_h2d;
58 tlul_pkg::tl_d2h_t tl_reg_d2h;
59
60
61 // incoming payload check
62 logic intg_err;
63 tlul_cmd_intg_chk u_chk (
64   .tl_i(tl_i),
65   .err_o(intg_err)
66 );
67
68 // also check for spurious write enables
69 logic reg_we_err;
70 logic [11:0] reg_we_check;
71 prim_reg_we_check #(
72   .OneHotWidth(12)
73 ) u_prim_reg_we_check (
74   .clk_i(clk_i),
75   .rst_ni(rst_ni),
76   .oh_i (reg_we_check),
77   .en_i (reg_we && !addrmiss),
78   .err_o (reg_we_err)
79 );
80
81 logic err_q;
82 always_ff @(posedge clk_i or negedge rst_ni) begin
83   if (!rst_ni) begin
84     err_q <= '0;
85   end else if (intg_err || reg_we_err) begin
86     err_q <= 1'b1;
87   end
88 end
89
90 // integrity error output is permanent and should be used
for alert generation
91 // register errors are transactional
92 assign intg_err_o = err_q | intg_err | reg_we_err;
93
94 // outgoing integrity generation
95 tlul_pkg::tl_d2h_t tl_o_pre;
96 tlul_rsp_intg_gen #(
97   .EnableRspIntgGen(1),
98   .EnableDataIntgGen(1)
99 ) u_rsp_intg_gen (
100   .tl_i(tl_o_pre),
101   .tl_o(tl_o)
102 );
103
104 tlul_pkg::tl_h2d_t tl_socket_h2d [3];
105 tlul_pkg::tl_d2h_t tl_socket_d2h [3];
106
107 logic [1:0] reg_steer;
108
109 // socket_1n connection
110 assign tl_reg_h2d = tl_socket_h2d[2];
111 assign tl_socket_d2h[2] = tl_reg_d2h;
112
113 assign tl_win_o[0] = tl_socket_h2d[0];
114 assign tl_socket_d2h[0] = tl_win_i[0];
115 assign tl_win_o[1] = tl_socket_h2d[1];
116 assign tl_socket_d2h[1] = tl_win_i[1];
117
118 // Create Socket_1n
119 tlul_socket_1n #(
120   .N (3),
121   .HReqPass (1'b1),
122   .HRspPass (1'b1),
123   .DReqPass ({3{1'b1}}),
124   .DRspPass ({3{1'b1}}),
125   .HReqDepth (4'h0),
126   .HRspDepth (4'h0),
127   .DReqDepth ({3{4'h0}}),
128   .DRspDepth ({3{4'h0}}),
129   .ExplicitErrs (1'b0)
130 ) u_socket (
131   .clk_i (clk_i),
132   .rst_ni (rst_ni),
133   .tl_h_i (tl_i),
134   .tl_h_o (tl_o_pre),
135   .tl_d_o (tl_socket_h2d),
136   .tl_d_i (tl_socket_d2h),
137   .dev_select_i (reg_steer)
138 );
139
140 // Create steering logic
141 always_comb begin
142   reg_steer =
143     tl_i.a_address[AW-1:0] inside {[36:39]} ? 2'd0 :
144     tl_i.a_address[AW-1:0] inside {[40:43]} ? 2'd1 :
145     // Default set to register
146     2'd2;
147
148   // Override this in case of an integrity error
149   if (intg_err) begin
150     reg_steer = 2'd2;
151   end
152 end
153
154 tlul_adapter_reg #(
155   .RegAw(AW),
156   .RegDw(DW),
157   .EnableDataIntgGen(0)
158 ) u_reg_if (
159   .clk_i (clk_i),
160   .rst_ni (rst_ni),
161
162   .tl_i (tl_reg_h2d),
163   .tl_o (tl_reg_d2h),
164
165   .en_ifetch_i(prim_mubi_pkg::MuBi4False),
166   .intg_error_o(),
167
168   .we_o (reg_we),
169   .re_o (reg_re),
170   .addr_o (reg_addr),
171   .wdata_o (reg_wdata),
172   .be_o (reg_be),
173   .busy_i (reg_busy),
174   .rdata_i (reg_rdata),
175   // Translate RACL error to TLUL error if enabled
176   .error_i (reg_error | (RaclErrorRsp & racl_error_o.valid))
177 );
178
179 // cdc oversampling signals
180
181 assign reg_rdata = reg_rdata_next ;
182 assign reg_error = addrmiss | wr_err | intg_err;
183
184 // Define SW related signals
185 // Format: <reg>_<field>_{wd|we|qs}
186 // or <reg>_{wd|we|qs} if field == 1 or 0
187 logic intr_state_we;
188 logic intr_state_error_qs;
189 logic intr_state_error_wd;
190 logic intr_state_spi_event_qs;
191 logic intr_enable_we;
192 logic intr_enable_error_qs;
193 logic intr_enable_error_wd;
194 logic intr_enable_spi_event_qs;
195 logic intr_enable_spi_event_wd;
196 logic intr_test_we;
197 logic intr_test_error_wd;
198 logic intr_test_spi_event_wd;
199 logic alert_test_we;
200 logic alert_test_wd;
201 logic control_we;
202 logic [7:0] control_rx_watermark_qs;
203 logic [7:0] control_rx_watermark_wd;
204 logic [7:0] control_tx_watermark_qs;
205 logic [7:0] control_tx_watermark_wd;
206 logic control_output_en_qs;
207 logic control_output_en_wd;
208 logic control_sw_rst_qs;
209 logic control_sw_rst_wd;
210 logic control_spieen_qs;
211 logic control_spieen_wd;
212 logic [7:0] status_txqd_qs;

```

```

213 logic [7:0] status_rxqd_qs;
214 logic [3:0] status_cmdqd_qs;
215 logic status_rxwm_qs;
216 logic status_byteorder_qs;
217 logic status_rxstall_qs;
218 logic status_rxempty_qs;
219 logic status_rxfull_qs;
220 logic status_txwm_qs;
221 logic status_txstall_qs;
222 logic status_txempty_qs;
223 logic status_txfull_qs;
224 logic status_active_qs;
225 logic status_ready_qs;
226 logic configopts_we;
227 logic [15:0] configopts_clkdiv_qs;
228 logic [15:0] configopts_clkdiv_wd;
229 logic [3:0] configopts_csnidle_qs;
230 logic [3:0] configopts_csnidle_wd;
231 logic [3:0] configopts_csntrail_qs;
232 logic [3:0] configopts_csntrail_wd;
233 logic [3:0] configopts_csnlead_qs;
234 logic [3:0] configopts_csnlead_wd;
235 logic configopts_fullcyc_qs;
236 logic configopts_fullcyc_wd;
237 logic configopts_cpha_qs;
238 logic configopts_cpha_wd;
239 logic configopts_cpol_qs;
240 logic configopts_cpol_wd;
241 logic csid_we;
242 logic [31:0] csid_qs;
243 logic [31:0] csid_wd;
244 logic command_we;
245 logic command_csaat_wd;
246 logic [1:0] command_speed_wd;
247 logic [1:0] command_direction_wd;
248 logic [19:0] command_len_wd;
249 logic error_enable_we;
250 logic error_enable_cmdbusy_qs;
251 logic error_enable_cmdbusy_wd;
252 logic error_enable_overflow_qs;
253 logic error_enable_overflow_wd;
254 logic error_enable_underflow_qs;
255 logic error_enable_underflow_wd;
256 logic error_enable_cmdinval_qs;
257 logic error_enable_cmdinval_wd;
258 logic error_enable_csidinval_qs;
259 logic error_enable_csidinval_wd;
260 logic error_status_we;
261 logic error_status_cmdbusy_qs;
262 logic error_status_cmdbusy_wd;
263 logic error_status_overflow_qs;
264 logic error_status_overflow_wd;
265 logic error_status_underflow_qs;
266 logic error_status_underflow_wd;
267 logic error_status_cmdinval_qs;
268 logic error_status_cmdinval_wd;
269 logic error_status_csidinval_qs;
270 logic error_status_csidinval_wd;
271 logic error_status_accessinval_qs;
272 logic error_status_accessinval_wd;
273 logic event_enable_we;
274 logic event_enable_rxfull_qs;
275 logic event_enable_rxfull_wd;
276 logic event_enable_txempty_qs;
277 logic event_enable_txempty_wd;
278 logic event_enable_rxwm_qs;
279 logic event_enable_rxwm_wd;
280 logic event_enable_txwm_qs;
281 logic event_enable_txwm_wd;
282 logic event_enable_ready_qs;
283 logic event_enable_ready_wd;
284 logic event_enable_idle_qs;
285 logic event_enable_idle_wd;
286
287 // Register instances
288 // R[intr_state]: V(False)
289 // F[error]: 0:0
290 prim_subreg #(
291   .DW (1),
292   .SwAccess(prim_subreg_pkg::SwAccessW1C),
293   .RESVAL (1'h0),
294   .Mubi (1'b0)
295 ) u_intr_state_error (
296   .clk_i (clk_i),
297   .rst_ni (rst_ni),
298
299   // from register interface
300   .we (intr_state_we),
301   .wd (intr_state_error_wd),
302
303   // from internal hardware
304   .de (hw2reg.intr_state.error.de),
305   .d (hw2reg.intr_state.error.d),
306
307   // to internal hardware
308   .qe (),
309   .q (reg2hw.intr_state.error.q),
310   .ds (),
311
312   // to register interface (read)
313   .qs (intr_state_error_qs)
314 );
315
316 // F[spi_event]: 1:1
317 prim_subreg #(
318   .DW (1),
319   .SwAccess(prim_subreg_pkg::SwAccessRO),
320   .RESVAL (1'h0),
321   .Mubi (1'b0)
322 ) u_intr_state_spi_event (
323   .clk_i (clk_i),
324   .rst_ni (rst_ni),
325
326   // from register interface
327   .we (1'b0),
328   .wd ('0'),
329
330   // from internal hardware
331   .de (hw2reg.intr_state.spi_event.de),
332   .d (hw2reg.intr_state.spi_event.d),
333
334   // to internal hardware
335   .qe (),
336   .q (reg2hw.intr_state.spi_event.q),
337   .ds (),
338
339   // to register interface (read)
340   .qs (intr_state_spi_event_qs)
341 );
342
343 // R[intr_enable]: V(False)
344 // F[error]: 0:0
345 prim_subreg #(
346   .DW (1),
347   .SwAccess(prim_subreg_pkg::SwAccessRW),
348   .RESVAL (1'h0),
349   .Mubi (1'b0)
350 ) u_intr_enable_error (
351   .clk_i (clk_i),
352   .rst_ni (rst_ni),
353
354   // from register interface
355   .we (intr_enable_we),
356   .wd (intr_enable_error_wd),
357
358   // from internal hardware
359   .de (1'b0),
360   .d ('0'),
361
362   // to internal hardware
363   .qe (),
364   .q (reg2hw.intr_enable.error.q),
365   .ds (),
366
367   // to register interface (read)
368   .qs (intr_enable_error_qs)
369 );
370
371 // F[spi_event]: 1:1
372 prim_subreg #(
373   .DW (1),
374   .SwAccess(prim_subreg_pkg::SwAccessRW),
375   .RESVAL (1'h0),
376   .Mubi (1'b0)
377 ) u_intr_enable_spi_event (
378   .clk_i (clk_i),
379   .rst_ni (rst_ni),
380
381   // from register interface
382   .we (intr_enable_we),
383   .wd (intr_enable_spi_event_wd),
384
385   // from internal hardware
386   .de (1'b0),
387   .d ('0'),
388
389   // to internal hardware
390   .qe (),
391   .q (reg2hw.intr_enable.spi_event.q),
392   .ds (),
393
394   // to register interface (read)
395   .qs (intr_enable_spi_event_qs)
396 );
397
398 // R[intr_test]: V(True)
399 logic intr_test_qe;
400 logic [1:0] intr_test_flds_we;
401 assign intr_test_qe = &intr_test_flds_we;
402 // F[error]: 0:0
403 prim_subreg_ext #(
404   .DW (1)
405 ) u_intr_test_error (
406   .re (1'b0),
407   .we (intr_test_we),
408   .wd (intr_test_error_wd),
409   .d ('0'),
410   .gre (),
411   .qe (intr_test_flds_we[0]),
412   .q (reg2hw.intr_test.error.q),
413
414 );

```

```

415 .ds      (),
416 .qs      ()
417 );
418 assign reg2hw.intr_test.error.qe = intr_test_qe;
419
420 // F[spi_event]: 1:1
421 prim_subreg_ext #(
422   .DW      (1)
423 ) u_intr_test_spi_event (
424   .re      (1'b0),
425   .we      (intr_test_we),
426   .wd      (intr_test_spi_event_wd),
427   .d       ('0'),
428   .gre     (),
429   .qe      (intr_test_flds_we[1]),
430   .q       (reg2hw.intr_test.spi_event.q),
431   .ds      (),
432   .qs      ()
433 );
434 assign reg2hw.intr_test.spi_event.qe = intr_test_qe;
435
436 // R[alert_test]: V(True)
437 logic alert_test_qe;
438 logic [0:0] alert_test_flds_we;
439 assign alert_test_qe = &alert_test_flds_we;
440 prim_subreg_ext #(
441   .DW      (1)
442 ) u_alert_test (
443   .re      (1'b0),
444   .we      (alert_test_we),
445   .wd      (alert_test_wd),
446   .d       ('0'),
447   .gre     (),
448   .qe      (alert_test_flds_we[0]),
449   .q       (reg2hw.alert_test.q),
450   .ds      (),
451   .qs      ()
452 );
453 assign reg2hw.alert_test.qe = alert_test_qe;
454
455 // R[control]: V(False)
456 // F[rx_watermark]: 7:0
457 prim_subreg #(
458   .DW      (8),
459   .SwAccess(prim_subreg_pkg::SwAccessRW),
460   .RESVAL  (8'h7f),
461   .Mubi     (1'b0)
462 ) u_control_rx_watermark (
463   .clk_i    (clk_i),
464   .rst_ni   (rst_ni),
465
466   // from register interface
467   .we      (control_we),
468   .wd      (control_rx_watermark_wd),
469
470   // from internal hardware
471   .de      (1'b0),
472   .d       ('0'),
473
474   // to internal hardware
475   .qe      (),
476   .q       (reg2hw.control.rx_watermark.q),
477   .ds      (),
478
479   // to register interface (read)
480   .qs      (control_rx_watermark_qs)
481 );
482
483 // F[tx_watermark]: 15:8
484 prim_subreg #(
485   .DW      (8),
486   .SwAccess(prim_subreg_pkg::SwAccessRW),
487   .RESVAL  (8'h0),
488   .Mubi     (1'b0)
489 ) u_control_tx_watermark (
490   .clk_i    (clk_i),
491   .rst_ni   (rst_ni),
492
493   // from register interface
494   .we      (control_we),
495   .wd      (control_tx_watermark_wd),
496
497   // from internal hardware
498   .de      (1'b0),
499   .d       ('0'),
500
501   // to internal hardware
502   .qe      (),
503   .q       (reg2hw.control.tx_watermark.q),
504   .ds      (),
505
506   // to register interface (read)
507   .qs      (control_tx_watermark_qs)
508 );
509
510 // F[output_en]: 29:29
511 prim_subreg #(
512   .DW      (1),
513   .SwAccess(prim_subreg_pkg::SwAccessRW),

```

```

516   .RESVAL  (1'h0),
517   .Mubi     (1'b0)
518 ) u_control_output_en (
519   .clk_i    (clk_i),
520   .rst_ni   (rst_ni),
521
522   // from register interface
523   .we      (control_we),
524   .wd      (control_output_en_wd),
525
526   // from internal hardware
527   .de      (1'b0),
528   .d       ('0'),
529
530   // to internal hardware
531   .qe      (),
532   .q       (reg2hw.control.output_en.q),
533   .ds      (),
534
535   // to register interface (read)
536   .qs      (control_output_en_qs)
537 );
538
539 // F[sw_rst]: 30:30
540 prim_subreg #(
541   .DW      (1),
542   .SwAccess(prim_subreg_pkg::SwAccessRW),
543   .RESVAL  (1'h0),
544   .Mubi     (1'b0)
545 ) u_control_sw_rst (
546   .clk_i    (clk_i),
547   .rst_ni   (rst_ni),
548
549   // from register interface
550   .we      (control_we),
551   .wd      (control_sw_rst_wd),
552
553   // from internal hardware
554   .de      (1'b0),
555   .d       ('0'),
556
557   // to internal hardware
558   .qe      (),
559   .q       (reg2hw.control.sw_rst.q),
560   .ds      (),
561
562   // to register interface (read)
563   .qs      (control_sw_rst_qs)
564 );
565
566 // F[spien]: 31:31
567 prim_subreg #(
568   .DW      (1),
569   .SwAccess(prim_subreg_pkg::SwAccessRW),
570   .RESVAL  (1'h0),
571   .Mubi     (1'b0)
572 ) u_control_spie (
573   .clk_i    (clk_i),
574   .rst_ni   (rst_ni),
575
576   // from register interface
577   .we      (control_we),
578   .wd      (control_spie_wd),
579
580   // from internal hardware
581   .de      (1'b0),
582   .d       ('0'),
583
584   // to internal hardware
585   .qe      (),
586   .q       (reg2hw.control.spie.q),
587   .ds      (),
588
589   // to register interface (read)
590   .qs      (control_spie_qs)
591 );
592
593 // R[status]: V(False)
594 // F[txqd]: 7:0
595 prim_subreg #(
596   .DW      (8),
597   .SwAccess(prim_subreg_pkg::SwAccessRO),
598   .RESVAL  (8'h0),
599   .Mubi     (1'b0)
600 ) u_status_txqd (
601   .clk_i    (clk_i),
602   .rst_ni   (rst_ni),
603
604   // from register interface
605   .we      (1'b0),
606   .wd      ('0'),
607
608   // from internal hardware
609   .de      (hw2reg.status.txqd.de),
610   .d       (hw2reg.status.txqd.d),
611
612   // to internal hardware
613   .qe      (),
614   .q       (),
615   .ds      (),
616

```



```

617
618 // to register interface (read)
619 .qs (status_txqd_qs)
620 );
621
622 // F[rxqd]: 15:8
623 prim_subreg #(
624     .DW (8),
625     .SwAccess(prim_subreg_pkg::SwAccessR0),
626     .RESVAL (8'h0),
627     .Mubi (1'b0)
628 ) u_status_rxqd (
629     .clk_i (clk_i),
630     .rst_ni (rst_ni),
631
632     // from register interface
633     .we (1'b0),
634     .wd ('0'),
635
636     // from internal hardware
637     .de (hw2reg.status.rxqd.de),
638     .d (hw2reg.status.rxqd.d),
639
640     // to internal hardware
641     .qe (),
642     .q (),
643     .ds (),
644
645     // to register interface (read)
646     .qs (status_rxqd_qs)
647 );
648
649 // F[cmdqd]: 19:16
650 prim_subreg #(
651     .DW (4),
652     .SwAccess(prim_subreg_pkg::SwAccessR0),
653     .RESVAL (4'h0),
654     .Mubi (1'b0)
655 ) u_status_cmdqd (
656     .clk_i (clk_i),
657     .rst_ni (rst_ni),
658
659     // from register interface
660     .we (1'b0),
661     .wd ('0'),
662
663     // from internal hardware
664     .de (hw2reg.status.cmdqd.de),
665     .d (hw2reg.status.cmdqd.d),
666
667     // to internal hardware
668     .qe (),
669     .q (),
670     .ds (),
671
672     // to register interface (read)
673     .qs (status_cmdqd_qs)
674 );
675
676 // F[rxwm]: 20:20
677 prim_subreg #(
678     .DW (1),
679     .SwAccess(prim_subreg_pkg::SwAccessR0),
680     .RESVAL (1'h0),
681     .Mubi (1'b0)
682 ) u_status_rxwm (
683     .clk_i (clk_i),
684     .rst_ni (rst_ni),
685
686     // from register interface
687     .we (1'b0),
688     .wd ('0'),
689
690     // from internal hardware
691     .de (hw2reg.status.rxwm.de),
692     .d (hw2reg.status.rxwm.d),
693
694     // to internal hardware
695     .qe (),
696     .q (),
697     .ds (),
698
699     // to register interface (read)
700     .qs (status_rxwm_qs)
701 );
702
703 // F[byteorder]: 22:22
704 prim_subreg #(
705     .DW (1),
706     .SwAccess(prim_subreg_pkg::SwAccessR0),
707     .RESVAL (1'h0),
708     .Mubi (1'b0)
709 ) u_status_byteorder (
710     .clk_i (clk_i),
711     .rst_ni (rst_ni),
712
713     // from register interface
714     .we (1'b0),
715     .wd ('0'),
716
717     // from internal hardware

```

```

718     .de (hw2reg.status.byteorder.de),
719     .d (hw2reg.status.byteorder.d),
720
721     // to internal hardware
722     .qe (),
723     .q (),
724     .ds (),
725
726     // to register interface (read)
727     .qs (status_byteorder_qs)
728 );
729
730 // F[rxstall]: 23:23
731 prim_subreg #(
732     .DW (1),
733     .SwAccess(prim_subreg_pkg::SwAccessR0),
734     .RESVAL (1'h0),
735     .Mubi (1'b0)
736 ) u_status_rxstall (
737     .clk_i (clk_i),
738     .rst_ni (rst_ni),
739
740     // from register interface
741     .we (1'b0),
742     .wd ('0'),
743
744     // from internal hardware
745     .de (hw2reg.status.rxstall.de),
746     .d (hw2reg.status.rxstall.d),
747
748     // to internal hardware
749     .qe (),
750     .q (),
751     .ds (),
752
753     // to register interface (read)
754     .qs (status_rxstall_qs)
755 );
756
757 // F[rxempty]: 24:24
758 prim_subreg #(
759     .DW (1),
760     .SwAccess(prim_subreg_pkg::SwAccessR0),
761     .RESVAL (1'h0),
762     .Mubi (1'b0)
763 ) u_status_rxempty (
764     .clk_i (clk_i),
765     .rst_ni (rst_ni),
766
767     // from register interface
768     .we (1'b0),
769     .wd ('0'),
770
771     // from internal hardware
772     .de (hw2reg.status.rxempty.de),
773     .d (hw2reg.status.rxempty.d),
774
775     // to internal hardware
776     .qe (),
777     .q (),
778     .ds (),
779
780     // to register interface (read)
781     .qs (status_rxempty_qs)
782 );
783
784 // F[rxfull]: 25:25
785 prim_subreg #(
786     .DW (1),
787     .SwAccess(prim_subreg_pkg::SwAccessR0),
788     .RESVAL (1'h0),
789     .Mubi (1'b0)
790 ) u_status_rxfull (
791     .clk_i (clk_i),
792     .rst_ni (rst_ni),
793
794     // from register interface
795     .we (1'b0),
796     .wd ('0'),
797
798     // from internal hardware
799     .de (hw2reg.status.rxfull.de),
800     .d (hw2reg.status.rxfull.d),
801
802     // to internal hardware
803     .qe (),
804     .q (),
805     .ds (),
806
807     // to register interface (read)
808     .qs (status_rxfull_qs)
809 );
810
811 // F[txwm]: 26:26
812 prim_subreg #(
813     .DW (1),
814     .SwAccess(prim_subreg_pkg::SwAccessR0),
815     .RESVAL (1'h0),
816     .Mubi (1'b0)
817 ) u_status_txwm (
818     .clk_i (clk_i),

```

```

819 .rst_ni (rst_ni),
820
821 // from register interface
822 .we (1'b0),
823 .wd ('0'),
824
825 // from internal hardware
826 .de (hw2reg.status.txwm.de),
827 .d (hw2reg.status.txwm.d),
828
829 // to internal hardware
830 .qe (),
831 .q (),
832 .ds (),
833
834 // to register interface (read)
835 .qs (status_txwm_qs)
836 );
837
838 // F[txstall]: 27:27
839 prim_subreg #(
840 .DW (1),
841 .SwAccess(prim_subreg_pkg::SwAccessR0),
842 .RESVAL (1'h0),
843 .Mubi (1'b0)
844 ) u_status_txstall (
845 .clk_i (clk_i),
846 .rst_ni (rst_ni),
847
848 // from register interface
849 .we (1'b0),
850 .wd ('0'),
851
852 // from internal hardware
853 .de (hw2reg.status.txstall.de),
854 .d (hw2reg.status.txstall.d),
855
856 // to internal hardware
857 .qe (),
858 .q (),
859 .ds (),
860
861 // to register interface (read)
862 .qs (status_txstall_qs)
863 );
864
865 // F[txempty]: 28:28
866 prim_subreg #(
867 .DW (1),
868 .SwAccess(prim_subreg_pkg::SwAccessR0),
869 .RESVAL (1'h0),
870 .Mubi (1'b0)
871 ) u_status_txempty (
872 .clk_i (clk_i),
873 .rst_ni (rst_ni),
874
875 // from register interface
876 .we (1'b0),
877 .wd ('0'),
878
879 // from internal hardware
880 .de (hw2reg.status.txempty.de),
881 .d (hw2reg.status.txempty.d),
882
883 // to internal hardware
884 .qe (),
885 .q (),
886 .ds (),
887
888 // to register interface (read)
889 .qs (status_txempty_qs)
890 );
891
892 // F[txfull]: 29:29
893 prim_subreg #(
894 .DW (1),
895 .SwAccess(prim_subreg_pkg::SwAccessR0),
896 .RESVAL (1'h0),
897 .Mubi (1'b0)
898 ) u_status_txfull (
899 .clk_i (clk_i),
900 .rst_ni (rst_ni),
901
902 // from register interface
903 .we (1'b0),
904 .wd ('0'),
905
906 // from internal hardware
907 .de (hw2reg.status.txfull.de),
908 .d (hw2reg.status.txfull.d),
909
910 // to internal hardware
911 .qe (),
912 .q (),
913 .ds (),
914
915 // to register interface (read)
916 .qs (status_txfull_qs)
917 );
918
919 // F[active]: 30:30

```

```

920 prim_subreg #(
921 .DW (1),
922 .SwAccess(prim_subreg_pkg::SwAccessR0),
923 .RESVAL (1'h0),
924 .Mubi (1'b0)
925 ) u_status_active (
926 .clk_i (clk_i),
927 .rst_ni (rst_ni),
928
929 // from register interface
930 .we (1'b0),
931 .wd ('0'),
932
933 // from internal hardware
934 .de (hw2reg.status.active.de),
935 .d (hw2reg.status.active.d),
936
937 // to internal hardware
938 .qe (),
939 .q (),
940 .ds (),
941
942 // to register interface (read)
943 .qs (status_active_qs)
944 );
945
946 // F[ready]: 31:31
947 prim_subreg #(
948 .DW (1),
949 .SwAccess(prim_subreg_pkg::SwAccessR0),
950 .RESVAL (1'h0),
951 .Mubi (1'b0)
952 ) u_status_ready (
953 .clk_i (clk_i),
954 .rst_ni (rst_ni),
955
956 // from register interface
957 .we (1'b0),
958 .wd ('0'),
959
960 // from internal hardware
961 .de (hw2reg.status.ready.de),
962 .d (hw2reg.status.ready.d),
963
964 // to internal hardware
965 .qe (),
966 .q (),
967 .ds (),
968
969 // to register interface (read)
970 .qs (status_ready_qs)
971 );
972
973 // R[configopts]: V(False)
974 // F[clkdiv]: 15:0
975 prim_subreg #(
976 .DW (16),
977 .SwAccess(prim_subreg_pkg::SwAccessRW),
978 .RESVAL (16'h0),
979 .Mubi (1'b0)
980 ) u_configopts_clkdiv (
981 .clk_i (clk_i),
982 .rst_ni (rst_ni),
983
984 // from register interface
985 .we (configopts_we),
986 .wd (configopts_clkdiv_wd),
987
988 // from internal hardware
989 .de (1'b0),
990 .d ('0'),
991
992 // to internal hardware
993 .qe (),
994 .q (reg2hw.configopts.clkdiv.q),
995 .ds (),
996
997 // to register interface (read)
998 .qs (configopts_clkdiv_qs)
999 );
1000
1001 // F[csnidle]: 19:16
1002 prim_subreg #(
1003 .DW (4),
1004 .SwAccess(prim_subreg_pkg::SwAccessRW),
1005 .RESVAL (4'h0),
1006 .Mubi (1'b0)
1007 ) u_configopts_csnidle (
1008 .clk_i (clk_i),
1009 .rst_ni (rst_ni),
1010
1011 // from register interface
1012 .we (configopts_we),
1013 .wd (configopts_csnidle_wd),
1014
1015 // from internal hardware
1016 .de (1'b0),
1017 .d ('0'),
1018
1019 // to internal hardware
1020

```



```

1021 .qe      ( ),
1022 .q      (reg2hw.configopts.csndle.q),
1023 .ds      ( ),
1024
1025 // to register interface (read)
1026 .qs      (configopts_csndle_qs)
1027 );
1028
1029 // F[csntrail]: 23:20
1030 prim_subreg #(
1031     .DW      (4),
1032     .SwAccess(prim_subreg_pkg::SwAccessRW),
1033     .RESVAL   (4'h0),
1034     .Mubi     (1'b0)
1035 ) u_configopts_csntrail (
1036     .clk_i    (clk_i),
1037     .rst_ni   (rst_ni),
1038
1039     // from register interface
1040     .we        (configopts_we),
1041     .wd        (configopts_csntrail_wd),
1042
1043     // from internal hardware
1044     .de        (1'b0),
1045     .d          ('0'),
1046
1047     // to internal hardware
1048     .qe        ( ),
1049     .q          (reg2hw.configopts.csntrail.q),
1050     .ds        ( ),
1051
1052     // to register interface (read)
1053     .qs        (configopts_csntrail_qs)
1054 );
1055
1056 // F[csnlead]: 27:24
1057 prim_subreg #(
1058     .DW      (4),
1059     .SwAccess(prim_subreg_pkg::SwAccessRW),
1060     .RESVAL   (4'h0),
1061     .Mubi     (1'b0)
1062 ) u_configopts_csnlead (
1063     .clk_i    (clk_i),
1064     .rst_ni   (rst_ni),
1065
1066     // from register interface
1067     .we        (configopts_we),
1068     .wd        (configopts_csnlead_wd),
1069
1070     // from internal hardware
1071     .de        (1'b0),
1072     .d          ('0'),
1073
1074     // to internal hardware
1075     .qe        ( ),
1076     .q          (reg2hw.configopts.csnlead.q),
1077     .ds        ( ),
1078
1079     // to register interface (read)
1080     .qs        (configopts_csnlead_qs)
1081 );
1082
1083 // F[fullcyc]: 29:29
1084 prim_subreg #(
1085     .DW      (1),
1086     .SwAccess(prim_subreg_pkg::SwAccessRW),
1087     .RESVAL   (1'h0),
1088     .Mubi     (1'b0)
1089 ) u_configopts_fullcyc (
1090     .clk_i    (clk_i),
1091     .rst_ni   (rst_ni),
1092
1093     // from register interface
1094     .we        (configopts_we),
1095     .wd        (configopts_fullcyc_wd),
1096
1097     // from internal hardware
1098     .de        (1'b0),
1099     .d          ('0'),
1100
1101     // to internal hardware
1102     .qe        ( ),
1103     .q          (reg2hw.configopts.fullcyc.q),
1104     .ds        ( ),
1105
1106     // to register interface (read)
1107     .qs        (configopts_fullcyc_qs)
1108 );
1109
1110 // F[cpha]: 30:30
1111 prim_subreg #(
1112     .DW      (1),
1113     .SwAccess(prim_subreg_pkg::SwAccessRW),
1114     .RESVAL   (1'h0),
1115     .Mubi     (1'b0)
1116 ) u_configopts_cpha (
1117     .clk_i    (clk_i),
1118     .rst_ni   (rst_ni),
1119
1120     // from register interface
1121     .we        (configopts_we),
1122
1123     .wd        (configopts_cpha_wd),
1124
1125     // from internal hardware
1126     .de        (1'b0),
1127     .d          ('0'),
1128
1129     // to internal hardware
1130     .qe        ( ),
1131     .q          (reg2hw.configopts.cpha.q),
1132     .ds        ( ),
1133
1134     // to register interface (read)
1135     .qs        (configopts_cpha_qs)
1136 );
1137
1138 // F[cpol]: 31:31
1139 prim_subreg #(
1140     .DW      (1),
1141     .SwAccess(prim_subreg_pkg::SwAccessRW),
1142     .RESVAL   (1'h0),
1143     .Mubi     (1'b0)
1144 ) u_configopts_cpol (
1145     .clk_i    (clk_i),
1146     .rst_ni   (rst_ni),
1147
1148     // from register interface
1149     .we        (configopts_we),
1150     .wd        (configopts_cpol_wd),
1151
1152     // from internal hardware
1153     .de        (1'b0),
1154     .d          ('0'),
1155
1156     // to internal hardware
1157     .qe        ( ),
1158     .q          (reg2hw.configopts.cpol.q),
1159     .ds        ( ),
1160
1161     // to register interface (read)
1162     .qs        (configopts_cpol_qs)
1163 );
1164
1165 // R[csid]: V(False)
1166 prim_subreg #(
1167     .DW      (32),
1168     .SwAccess(prim_subreg_pkg::SwAccessRW),
1169     .RESVAL   (32'h0),
1170     .Mubi     (1'b0)
1171 ) u_csids (
1172     .clk_i    (clk_i),
1173     .rst_ni   (rst_ni),
1174
1175     // from register interface
1176     .we        (csid_we),
1177     .wd        (csid_wd),
1178
1179     // from internal hardware
1180     .de        (1'b0),
1181     .d          ('0'),
1182
1183     // to internal hardware
1184     .qe        ( ),
1185     .q          (reg2hw.csid.q),
1186     .ds        ( ),
1187
1188     // to register interface (read)
1189     .qs        (csid_qs)
1190 );
1191
1192 // R[command]: V(True)
1193 logic command_qe;
1194 logic [3:0] command_flds_we;
1195 assign command_qe = &command_flds_we;
1196 // F[csaat]: 0:0
1197 prim_subreg_ext #(
1198     .DW      (1)
1199 ) u_command_csaat (
1200     .re       (1'b0),
1201     .we        (command_we),
1202     .wd        (command_csaat_wd),
1203     .d          ('0'),
1204     .qre       ( ),
1205     .qe        (command_flds_we[0]),
1206     .q          (reg2hw.command.csaat.q),
1207     .ds        ( ),
1208     .qs        ( ),
1209
1210 );
1211 assign reg2hw.command.csaat.qe = command_qe;
1212
1213 // F[speed]: 2:1
1214 prim_subreg_ext #(
1215     .DW      (2)
1216 ) u_command_speed (
1217     .re       (1'b0),
1218     .we        (command_we),
1219     .wd        (command_speed_wd),
1220     .d          ('0'),
1221     .qre       ( ),
1222     .qe        (command_flds_we[1]),

```

```

1223 .q      (reg2hw.command.speed.q),
1224 .ds    (),
1225 .qs    ()
1226 );
1227 assign reg2hw.command.speed.qe = command_qe;
1228
1229 // F[direction]: 4:3
1230 prim_subreg_ext #(
1231     .DW      (2)
1232 ) u_command_direction (
1233     .re      (1'b0),
1234     .we      (command_we),
1235     .wd      (command_direction_wd),
1236     .d       ('0'),
1237     .qre     (),
1238     .qe      (command_flds_we[2]),
1239     .q       (reg2hw.command.direction.q),
1240     .ds      (),
1241     .qs      ()
1242 );
1243 assign reg2hw.command.direction.qe = command_qe;
1244
1245 // F[len]: 24:5
1246 prim_subreg_ext #(
1247     .DW      (20)
1248 ) u_command_len (
1249     .re      (1'b0),
1250     .we      (command_we),
1251     .wd      (command_len_wd),
1252     .d       ('0'),
1253     .qre     (),
1254     .qe      (command_flds_we[3]),
1255     .q       (reg2hw.command.len.q),
1256     .ds      (),
1257     .qs      ()
1258 );
1259 assign reg2hw.command.len.qe = command_qe;
1260
1261 // R[error_enable]: V(False)
1262 // F[cmdbusy]: 0:0
1263 prim_subreg #(
1264     .DW      (1),
1265     .SwAccess(prim_subreg_pkg::SwAccessRW),
1266     .RESVAL  (1'h1),
1267     .Mubi    (1'b0)
1268 ) u_error_enable_cmdbusy (
1269     .clk_i   (clk_i),
1270     .rst_ni  (rst_ni),
1271
1272     // from register interface
1273     .we      (error_enable_we),
1274     .wd      (error_enable_cmdbusy_wd),
1275
1276     // from internal hardware
1277     .de      (1'b0),
1278     .d       ('0'),
1279
1280     // to internal hardware
1281     .qe      (),
1282     .q       (reg2hw.error_enable.cmdbusy.q),
1283     .ds      (),
1284
1285     // to register interface (read)
1286     .qs      (error_enable_cmdbusy_qs)
1287 );
1288
1289 // F[overflow]: 1:1
1290 prim_subreg #(
1291     .DW      (1),
1292     .SwAccess(prim_subreg_pkg::SwAccessRW),
1293     .RESVAL  (1'h1),
1294     .Mubi    (1'b0)
1295 ) u_error_enable_overflow (
1296     .clk_i   (clk_i),
1297     .rst_ni  (rst_ni),
1298
1299     // from register interface
1300     .we      (error_enable_we),
1301     .wd      (error_enable_overflow_wd),
1302
1303     // from internal hardware
1304     .de      (1'b0),
1305     .d       ('0'),
1306
1307     // to internal hardware
1308     .qe      (),
1309     .q       (reg2hw.error_enable.overflow.q),
1310     .ds      (),
1311
1312     // to register interface (read)
1313     .qs      (error_enable_overflow_qs)
1314 );
1315
1316 // F[underflow]: 2:2
1317 prim_subreg #(
1318     .DW      (1),
1319     .SwAccess(prim_subreg_pkg::SwAccessRW),
1320     .RESVAL  (1'h1),
1321     .Mubi    (1'b0)
1322 ) u_error_enable_underflow (
1323
1324     .clk_i   (clk_i),
1325     .rst_ni  (rst_ni),
1326
1327     // from register interface
1328     .we      (error_enable_we),
1329     .wd      (error_enable_underflow_wd),
1330
1331     // from internal hardware
1332     .de      (1'b0),
1333     .d       ('0'),
1334
1335     // to internal hardware
1336     .qe      (),
1337     .q       (reg2hw.error_enable.underflow.q),
1338     .ds      (),
1339
1340     // to register interface (read)
1341     .qs      (error_enable_underflow_qs)
1342 );
1343
1344 // F[cmdinval]: 3:3
1345 prim_subreg #(
1346     .DW      (1),
1347     .SwAccess(prim_subreg_pkg::SwAccessRW),
1348     .RESVAL  (1'h1),
1349     .Mubi    (1'b0)
1350 ) u_error_enable_cmdinval (
1351     .clk_i   (clk_i),
1352     .rst_ni  (rst_ni),
1353
1354     // from register interface
1355     .we      (error_enable_we),
1356     .wd      (error_enable_cmdinval_wd),
1357
1358     // from internal hardware
1359     .de      (1'b0),
1360     .d       ('0'),
1361
1362     // to internal hardware
1363     .qe      (),
1364     .q       (reg2hw.error_enable.cmdinval.q),
1365     .ds      (),
1366
1367     // to register interface (read)
1368     .qs      (error_enable_cmdinval_qs)
1369 );
1370
1371 // F[csdinval]: 4:4
1372 prim_subreg #(
1373     .DW      (1),
1374     .SwAccess(prim_subreg_pkg::SwAccessRW),
1375     .RESVAL  (1'h1),
1376     .Mubi    (1'b0)
1377 ) u_error_enable_csdinval (
1378     .clk_i   (clk_i),
1379     .rst_ni  (rst_ni),
1380
1381     // from register interface
1382     .we      (error_enable_we),
1383     .wd      (error_enable_csdinval_wd),
1384
1385     // from internal hardware
1386     .de      (1'b0),
1387     .d       ('0'),
1388
1389     // to internal hardware
1390     .qe      (),
1391     .q       (reg2hw.error_enable.csdinval.q),
1392     .ds      (),
1393
1394     // to register interface (read)
1395     .qs      (error_enable_csdinval_qs)
1396 );
1397
1398 // R[error_status]: V(False)
1399 // F[cmdbusy]: 0:0
1400 prim_subreg #(
1401     .DW      (1),
1402     .SwAccess(prim_subreg_pkg::SwAccessW1C),
1403     .RESVAL  (1'h0),
1404     .Mubi    (1'b0)
1405 ) u_error_status_cmdbusy (
1406     .clk_i   (clk_i),
1407     .rst_ni  (rst_ni),
1408
1409     // from register interface
1410     .we      (error_status_we),
1411     .wd      (error_status_cmdbusy_wd),
1412
1413     // from internal hardware
1414     .de      (hw2reg.error_status.cmdbusy.de),
1415     .d       (hw2reg.error_status.cmdbusy.d),
1416
1417     // to internal hardware
1418     .qe      (),
1419     .q       (reg2hw.error_status.cmdbusy.q),
1420     .ds      (),
1421
1422     // to register interface (read)
1423     .qs      (error_status_cmdbusy_qs)
1424 );

```

```

1425 );
1426 // F[overflow]: 1:1
1427 prim_subreg #(
1428     .DW (1),
1429     .SwAccess(prim_subreg_pkg::SwAccessW1C),
1430     .RESVAL (1'h0),
1431     .Mubi (1'b0)
1432 ) u_error_status_overflow (
1433     .clk_i (clk_i),
1434     .rst_ni (rst_ni),
1435
1436     // from register interface
1437     .we (error_status_we),
1438     .wd (error_status_overflow_wd),
1439
1440     // from internal hardware
1441     .de (hw2reg.error_status.overflow.de),
1442     .d (hw2reg.error_status.overflow.d),
1443
1444     // to internal hardware
1445     .qe (),
1446     .q (reg2hw.error_status.overflow.q),
1447     .ds (),
1448
1449     // to register interface (read)
1450     .qs (error_status_overflow_qs)
1451 );
1452
1453 // F[underflow]: 2:2
1454 prim_subreg #(
1455     .DW (1),
1456     .SwAccess(prim_subreg_pkg::SwAccessW1C),
1457     .RESVAL (1'h0),
1458     .Mubi (1'b0)
1459 ) u_error_status_underflow (
1460     .clk_i (clk_i),
1461     .rst_ni (rst_ni),
1462
1463     // from register interface
1464     .we (error_status_we),
1465     .wd (error_status_underflow_wd),
1466
1467     // from internal hardware
1468     .de (hw2reg.error_status.underflow.de),
1469     .d (hw2reg.error_status.underflow.d),
1470
1471     // to internal hardware
1472     .qe (),
1473     .q (reg2hw.error_status.underflow.q),
1474     .ds (),
1475
1476     // to register interface (read)
1477     .qs (error_status_underflow_qs)
1478 );
1479
1480 // F[cmdinval]: 3:3
1481 prim_subreg #(
1482     .DW (1),
1483     .SwAccess(prim_subreg_pkg::SwAccessW1C),
1484     .RESVAL (1'h0),
1485     .Mubi (1'b0)
1486 ) u_error_status_cmdinval (
1487     .clk_i (clk_i),
1488     .rst_ni (rst_ni),
1489
1490     // from register interface
1491     .we (error_status_we),
1492     .wd (error_status_cmdinval_wd),
1493
1494     // from internal hardware
1495     .de (hw2reg.error_status.cmdinval.de),
1496     .d (hw2reg.error_status.cmdinval.d),
1497
1498     // to internal hardware
1499     .qe (),
1500     .q (reg2hw.error_status.cmdinval.q),
1501     .ds (),
1502
1503     // to register interface (read)
1504     .qs (error_status_cmdinval_qs)
1505 );
1506
1507 // F[csidinval]: 4:4
1508 prim_subreg #(
1509     .DW (1),
1510     .SwAccess(prim_subreg_pkg::SwAccessW1C),
1511     .RESVAL (1'h0),
1512     .Mubi (1'b0)
1513 ) u_error_status_csidinval (
1514     .clk_i (clk_i),
1515     .rst_ni (rst_ni),
1516
1517     // from register interface
1518     .we (error_status_we),
1519     .wd (error_status_csidinval_wd),
1520
1521     // from internal hardware
1522     .de (hw2reg.error_status.csidinval.de),
1523     .d (hw2reg.error_status.csidinval.d),
1524
1525     // to internal hardware
1526     .qe (),
1527     .q (reg2hw.error_status.csidinval.q),
1528     .ds (),
1529
1530     // to register interface (read)
1531     .qs (error_status_csidinval_qs)
1532 );
1533
1534 // F[accessinval]: 5:5
1535 prim_subreg #(
1536     .DW (1),
1537     .SwAccess(prim_subreg_pkg::SwAccessW1C),
1538     .RESVAL (1'h0),
1539     .Mubi (1'b0)
1540 ) u_error_status_accessinval (
1541     .clk_i (clk_i),
1542     .rst_ni (rst_ni),
1543
1544     // from register interface
1545     .we (error_status_we),
1546     .wd (error_status_accessinval_wd),
1547
1548     // from internal hardware
1549     .de (hw2reg.error_status.accessinval.de),
1550     .d (hw2reg.error_status.accessinval.d),
1551
1552     // to internal hardware
1553     .qe (),
1554     .q (reg2hw.error_status.accessinval.q),
1555     .ds (),
1556
1557     // to register interface (read)
1558     .qs (error_status_accessinval_qs)
1559 );
1560
1561 // R[event_enable]: V(False)
1562 // F[rxfull]: 0:0
1563 prim_subreg #(
1564     .DW (1),
1565     .SwAccess(prim_subreg_pkg::SwAccessRW),
1566     .RESVAL (1'h0),
1567     .Mubi (1'b0)
1568 ) u_event_enable_rxfull (
1569     .clk_i (clk_i),
1570     .rst_ni (rst_ni),
1571
1572     // from register interface
1573     .we (event_enable_we),
1574     .wd (event_enable_rxfull_wd),
1575
1576     // from internal hardware
1577     .de (1'b0),
1578     .d ('0'),
1579
1580     // to internal hardware
1581     .qe (),
1582     .q (reg2hw.event_enable.rxfull.q),
1583     .ds (),
1584
1585     // to register interface (read)
1586     .qs (event_enable_rxfull_qs)
1587 );
1588
1589 // F[tEMPTY]: 1:1
1590 prim_subreg #(
1591     .DW (1),
1592     .SwAccess(prim_subreg_pkg::SwAccessRW),
1593     .RESVAL (1'h0),
1594     .Mubi (1'b0)
1595 ) u_event_enable_txempty (
1596     .clk_i (clk_i),
1597     .rst_ni (rst_ni),
1598
1599     // from register interface
1600     .we (event_enable_we),
1601     .wd (event_enable_txempty_wd),
1602
1603     // from internal hardware
1604     .de (1'b0),
1605     .d ('0'),
1606
1607     // to internal hardware
1608     .qe (),
1609     .q (reg2hw.event_enable.txempty.q),
1610     .ds (),
1611
1612     // to register interface (read)
1613     .qs (event_enable_txempty_qs)
1614 );
1615
1616 // F[rxwm]: 2:2
1617 prim_subreg #(
1618     .DW (1),
1619     .SwAccess(prim_subreg_pkg::SwAccessRW),
1620     .RESVAL (1'h0),
1621     .Mubi (1'b0)
1622 ) u_event_enable_rxwm (
1623     .clk_i (clk_i),
1624     .rst_ni (rst_ni),

```

```

1627
1628 // from register interface
1629 .we (event_enable_we),
1630 .wd (event_enable_rxwm_wd),
1631
1632 // from internal hardware
1633 .de (1'b0),
1634 .d ('0'),
1635
1636 // to internal hardware
1637 .qe (),
1638 .q (reg2hw.event_enable.rxwm.q),
1639 .ds (),
1640
1641 // to register interface (read)
1642 .qs (event_enable_rxwm_qs)
1643 );
1644
1645 // F[txwm]: 3:3
1646 prim_subreg #(
1647 .DW (1),
1648 .SwAccess(prim_subreg_pkg::SwAccessRW),
1649 .RESVAL (1'h0),
1650 .Mubi (1'b0)
1651 ) u_event_enable_txwm (
1652 .clk_i (clk_i),
1653 .rst_ni (rst_ni),
1654
1655 // from register interface
1656 .we (event_enable_we),
1657 .wd (event_enable_txwm_wd),
1658
1659 // from internal hardware
1660 .de (1'b0),
1661 .d ('0'),
1662
1663 // to internal hardware
1664 .qe (),
1665 .q (reg2hw.event_enable.txwm.q),
1666 .ds (),
1667
1668 // to register interface (read)
1669 .qs (event_enable_txwm_qs)
1670 );
1671
1672 // F[ready]: 4:4
1673 prim_subreg #(
1674 .DW (1),
1675 .SwAccess(prim_subreg_pkg::SwAccessRW),
1676 .RESVAL (1'h0),
1677 .Mubi (1'b0)
1678 ) u_event_enable_ready (
1679 .clk_i (clk_i),
1680 .rst_ni (rst_ni),
1681
1682 // from register interface
1683 .we (event_enable_we),
1684 .wd (event_enable_ready_wd),
1685
1686 // from internal hardware
1687 .de (1'b0),
1688 .d ('0'),
1689
1690 // to internal hardware
1691 .qe (),
1692 .q (reg2hw.event_enable.ready.q),
1693 .ds (),
1694
1695 // to register interface (read)
1696 .qs (event_enable_ready_qs)
1697 );
1698
1699 // F[idle]: 5:5
1700 prim_subreg #(
1701 .DW (1),
1702 .SwAccess(prim_subreg_pkg::SwAccessRW),
1703 .RESVAL (1'h0),
1704 .Mubi (1'b0)
1705 ) u_event_enable_idle (
1706 .clk_i (clk_i),
1707 .rst_ni (rst_ni),
1708
1709 // from register interface
1710 .we (event_enable_we),
1711 .wd (event_enable_idle_wd),
1712
1713 // from internal hardware
1714 .de (1'b0),
1715 .d ('0'),
1716
1717 // to internal hardware
1718 .qe (),
1719 .q (reg2hw.event_enable.idle.q),
1720 .ds (),
1721
1722 // to register interface (read)
1723 .qs (event_enable_idle_qs)
1724 );
1725
1726
1727

```

```

1728 logic [11:0] addr_hit;
1729 top_racl_pkg::racl_role_vec_t racl_role_vec;
1730 top_racl_pkg::racl_role_t racl_role;
1731
1732 logic [11:0] racl_addr_hit_read;
1733 logic [11:0] racl_addr_hit_write;
1734
1735 if (EnableRacl) begin : gen_racl_role_logic
1736 // Retrieve RACL role from user bits and one-hot encode
that for the comparison bitmap
1737 assign racl_role =
top_racl_pkg::tlul_extract_racl_role_bits(tl_i.a.user.rsvd);
1738
1739 prim_onehot_enc #(
1740 .OneHotWidth( $bits(top_racl_pkg::racl_role_vec_t) )
1741 ) u_racl_role_encode (
1742 .in_i ( racl_role ),
1743 .en_i ( 1'b1 ),
1744 .out_o( racl_role_vec )
1745 );
1746 end else begin : gen_no_racl_role_logic
1747 assign racl_role = '0;
1748 assign racl_role_vec = '0;
1749 end
1750
1751 always_comb begin
1752 racl_addr_hit_read = '0;
1753 racl_addr_hit_write = '0;
1754 addr_hit[ 0] = (reg_addr == SPI_HOST_INTR_STATE_OFFSET);
1755 addr_hit[ 1] = (reg_addr == SPI_HOST_INTR_ENABLE_OFFSET);
1756 addr_hit[ 2] = (reg_addr == SPI_HOST_INTR_TEST_OFFSET);
1757 addr_hit[ 3] = (reg_addr == SPI_HOST_ALERT_TEST_OFFSET);
1758 addr_hit[ 4] = (reg_addr == SPI_HOST_CONTROL_OFFSET);
1759 addr_hit[ 5] = (reg_addr == SPI_HOST_STATUS_OFFSET);
1760 addr_hit[ 6] = (reg_addr == SPI_HOST_CONFIGOPTS_OFFSET);
1761 addr_hit[ 7] = (reg_addr == SPI_HOST_CSID_OFFSET);
1762 addr_hit[ 8] = (reg_addr == SPI_HOST_COMMAND_OFFSET);
1763 addr_hit[ 9] = (reg_addr == SPI_HOST_ERROR_ENABLE_OFFSET);
1764 addr_hit[10] = (reg_addr == SPI_HOST_ERROR_STATUS_OFFSET);
1765 addr_hit[11] = (reg_addr == SPI_HOST_EVENT_ENABLE_OFFSET);
1766
1767 if (EnableRacl) begin : gen_racl_hit
1768 for (int unsigned slice_idx = 0; slice_idx < 12;
slice_idx++) begin
1769 racl_addr_hit_read[slice_idx] =
1770 addr_hit[slice_idx] & (
(racl_policies_i[RaclPolicySelVec[slice_idx]].read_perm
& racl_role_vec));
1771
1772 racl_addr_hit_write[slice_idx] =
1773 addr_hit[slice_idx] & (
(racl_policies_i[RaclPolicySelVec[slice_idx]].write_perm
& racl_role_vec));
1774
1775 end
1776 end else begin : gen_no_racl
1777 racl_addr_hit_read = addr_hit;
1778 racl_addr_hit_write = addr_hit;
1779 end
1780 end
1781
1782 assign addrmiss = (reg_re || reg_we) ? ~|addr_hit : 1'b0 ;
1783 // A valid address hit, access, but failed the RACL check
1784 assign racl_error_o.valid = |addr_hit & ((reg_re & ~|
racl_addr_hit_read) |
1785 (reg_we & ~|
racl_addr_hit_write));
1786 assign racl_error_o.request_address =
top_pkg::TL_AW'(reg_addr);
1787 assign racl_error_o.racl_role = racl_role;
1788 assign racl_error_o.overflow = 1'b0;
1789
1790 if (EnableRacl) begin : gen_racl_log
1791 assign racl_error_o.ctn_uid =
top_racl_pkg::tlul_extract_ctn_uid_bits(tl_i.a.user.rsvd);
1792 assign racl_error_o.read_access = tl_i.a.opcode ==
tlul_pkg::Get;
1793 end else begin : gen_no_racl_log
1794 assign racl_error_o.ctn_uid = '0;
1795 assign racl_error_o.read_access = 1'b0;
1796 end
1797
1798 // Check sub-word write is permitted
1799 always_comb begin
1800 wr_err = (reg_we &
1801 ((racl_addr_hit_write[ 0] & (|
(SPI_HOST_PERMIT[ 0] & ~reg_be))) |
1802 (racl_addr_hit_write[ 1] & (|
(SPI_HOST_PERMIT[ 1] & ~reg_be))) |
1803 (racl_addr_hit_write[ 2] & (|
(SPI_HOST_PERMIT[ 2] & ~reg_be))) |
1804 (racl_addr_hit_write[ 3] & (|
(SPI_HOST_PERMIT[ 3] & ~reg_be))) |
1805 (racl_addr_hit_write[ 4] & (|
(SPI_HOST_PERMIT[ 4] & ~reg_be))) |
1806 (racl_addr_hit_write[ 5] & (|
(SPI_HOST_PERMIT[ 5] & ~reg_be))) |
1807 (racl_addr_hit_write[ 6] & (|
(SPI_HOST_PERMIT[ 6] & ~reg_be))) |
1808 (racl_addr_hit_write[ 7] & (|
(SPI_HOST_PERMIT[ 7] & ~reg_be))) |
1809 (racl_addr_hit_write[ 8] & (|
(SPI_HOST_PERMIT[ 8] & ~reg_be)))))) |

```

```

1810      (rac1_addr_hit_write[ 9] & (!
(SPI_HOST_PERMIT[ 9] & ~reg_be))) |
1811      (rac1_addr_hit_write[10] & (!
(SPI_HOST_PERMIT[10] & ~reg_be))) |
1812      (rac1_addr_hit_write[11] & (!
(SPI_HOST_PERMIT[11] & ~reg_be)))));
1813  end
1814
1815  // Generate write-enables
1816  assign intr_state_we = rac1_addr_hit_write[0] & reg_we & !
reg_error;
1817
1818  assign intr_state_error_wd = reg_wdata[0];
1819  assign intr_enable_we = rac1_addr_hit_write[1] & reg_we & !
reg_error;
1820
1821  assign intr_enable_error_wd = reg_wdata[0];
1822
1823  assign intr_enable_spi_event_wd = reg_wdata[1];
1824  assign intr_test_we = rac1_addr_hit_write[2] & reg_we & !
reg_error;
1825
1826  assign intr_test_error_wd = reg_wdata[0];
1827
1828  assign intr_test_spi_event_wd = reg_wdata[1];
1829  assign alert_test_we = rac1_addr_hit_write[3] & reg_we & !
reg_error;
1830
1831  assign alert_test_wd = reg_wdata[0];
1832  assign control_we = rac1_addr_hit_write[4] & reg_we & !
reg_error;
1833
1834  assign control_rx_watermark_wd = reg_wdata[7:0];
1835
1836  assign control_tx_watermark_wd = reg_wdata[15:8];
1837
1838  assign control_output_en_wd = reg_wdata[29];
1839
1840  assign control_sw_rst_wd = reg_wdata[30];
1841
1842  assign control_spilen_wd = reg_wdata[31];
1843  assign configopts_we = rac1_addr_hit_write[6] & reg_we & !
reg_error;
1844
1845  assign configopts_clkdiv_wd = reg_wdata[15:0];
1846
1847  assign configopts_csnidle_wd = reg_wdata[19:16];
1848
1849  assign configopts_csntail_wd = reg_wdata[23:20];
1850
1851  assign configopts_csnlead_wd = reg_wdata[27:24];
1852
1853  assign configopts_fullcyc_wd = reg_wdata[29];
1854
1855  assign configopts_cpha_wd = reg_wdata[30];
1856
1857  assign configopts_cpol_wd = reg_wdata[31];
1858  assign csid_we = rac1_addr_hit_write[7] & reg_we & !reg_er-
ror;
1859
1860  assign csid_wd = reg_wdata[31:0];
1861  assign command_we = rac1_addr_hit_write[8] & reg_we & !
reg_error;
1862
1863  assign command_csaat_wd = reg_wdata[0];
1864
1865  assign command_speed_wd = reg_wdata[2:1];
1866
1867  assign command_direction_wd = reg_wdata[4:3];
1868
1869  assign command_len_wd = reg_wdata[24:5];
1870  assign error_enable_we = rac1_addr_hit_write[9] & reg_we & !
reg_error;
1871
1872  assign error_enable_cmdbusy_wd = reg_wdata[0];
1873
1874  assign error_enable_overflow_wd = reg_wdata[1];
1875
1876  assign error_enable_underflow_wd = reg_wdata[2];
1877
1878  assign error_enable_cmdinval_wd = reg_wdata[3];
1879
1880  assign error_enable_csidinval_wd = reg_wdata[4];
1881  assign error_status_we = rac1_addr_hit_write[10] & reg_we
& !reg_error;
1882
1883  assign error_status_cmdbusy_wd = reg_wdata[0];
1884
1885  assign error_status_overflow_wd = reg_wdata[1];
1886
1887  assign error_status_underflow_wd = reg_wdata[2];
1888
1889  assign error_status_cmdinval_wd = reg_wdata[3];
1890
1891  assign error_status_csidinval_wd = reg_wdata[4];
1892
1893  assign error_status_accessinval_wd = reg_wdata[5];
1894  assign event_enable_we = rac1_addr_hit_write[11] & reg_we
& !reg_error;
1895
1896  assign event_enable_rxfull_wd = reg_wdata[0];

```

```

1897  assign event_enable_txempty_wd = reg_wdata[1];
1898
1899  assign event_enable_rxwm_wd = reg_wdata[2];
1900
1901  assign event_enable_txwm_wd = reg_wdata[3];
1902
1903  assign event_enable_ready_wd = reg_wdata[4];
1904
1905  assign event_enable_idle_wd = reg_wdata[5];
1906
1907  // Assign write-enables to checker logic vector.
1908  always_comb begin
1909    reg_we_check[0] = intr_state_we;
1910    reg_we_check[1] = intr_enable_we;
1911    reg_we_check[2] = intr_test_we;
1912    reg_we_check[3] = alert_test_we;
1913    reg_we_check[4] = control_we;
1914    reg_we_check[5] = 1'b0;
1915    reg_we_check[6] = configopts_we;
1916    reg_we_check[7] = csid_we;
1917    reg_we_check[8] = command_we;
1918    reg_we_check[9] = error_enable_we;
1919    reg_we_check[10] = error_status_we;
1920    reg_we_check[11] = event_enable_we;
1921  end
1922
1923  // Read data return
1924  always_comb begin
1925    reg_rdata_next = '0;
1926    unique case (1'b1)
1927      rac1_addr_hit_read[0]: begin
1928        reg_rdata_next[0] = intr_state_error_qs;
1929        reg_rdata_next[1] = intr_state_spi_event_qs;
1930      end
1931
1932      rac1_addr_hit_read[1]: begin
1933        reg_rdata_next[0] = intr_enable_error_qs;
1934        reg_rdata_next[1] = intr_enable_spi_event_qs;
1935      end
1936
1937      rac1_addr_hit_read[2]: begin
1938        reg_rdata_next[0] = '0;
1939        reg_rdata_next[1] = '0;
1940      end
1941
1942      rac1_addr_hit_read[3]: begin
1943        reg_rdata_next[0] = '0;
1944      end
1945
1946      rac1_addr_hit_read[4]: begin
1947        reg_rdata_next[7:0] = control_rx_watermark_qs;
1948        reg_rdata_next[15:8] = control_tx_watermark_qs;
1949        reg_rdata_next[29] = control_output_en_qs;
1950        reg_rdata_next[30] = control_sw_rst_qs;
1951        reg_rdata_next[31] = control_spilen_qs;
1952      end
1953
1954      rac1_addr_hit_read[5]: begin
1955        reg_rdata_next[7:0] = status_txqd_qs;
1956        reg_rdata_next[15:8] = status_rxqd_qs;
1957        reg_rdata_next[19:16] = status_cmdqd_qs;
1958        reg_rdata_next[20] = status_rxwm_qs;
1959        reg_rdata_next[22] = status_byteorder_qs;
1960        reg_rdata_next[23] = status_rxstall_qs;
1961        reg_rdata_next[24] = status_rxempty_qs;
1962        reg_rdata_next[25] = status_rxfull_qs;
1963        reg_rdata_next[26] = status_txwm_qs;
1964        reg_rdata_next[27] = status_txstall_qs;
1965        reg_rdata_next[28] = status_txempty_qs;
1966        reg_rdata_next[29] = status_txfull_qs;
1967        reg_rdata_next[30] = status_active_qs;
1968        reg_rdata_next[31] = status_ready_qs;
1969      end
1970
1971      rac1_addr_hit_read[6]: begin
1972        reg_rdata_next[15:0] = configopts_clkdiv_qs;
1973        reg_rdata_next[19:16] = configopts_csnidle_qs;
1974        reg_rdata_next[23:20] = configopts_csntail_qs;
1975        reg_rdata_next[27:24] = configopts_csnlead_qs;
1976        reg_rdata_next[29] = configopts_fullcyc_qs;
1977        reg_rdata_next[30] = configopts_cpha_qs;
1978        reg_rdata_next[31] = configopts_cpol_qs;
1979      end
1980
1981      rac1_addr_hit_read[7]: begin
1982        reg_rdata_next[31:0] = csid_qs;
1983      end
1984
1985      rac1_addr_hit_read[8]: begin
1986        reg_rdata_next[0] = '0;
1987        reg_rdata_next[2:1] = '0;
1988        reg_rdata_next[4:3] = '0;
1989        reg_rdata_next[24:5] = '0;
1990      end
1991
1992      rac1_addr_hit_read[9]: begin
1993        reg_rdata_next[0] = error_enable_cmdbusy_qs;
1994        reg_rdata_next[1] = error_enable_overflow_qs;
1995        reg_rdata_next[2] = error_enable_underflow_qs;
1996        reg_rdata_next[3] = error_enable_cmdinval_qs;
1997

```

```

1998     reg_rdata_next[4] = error_enable_csinvalid_qs;
1999 end
2000
2001     rac1_addr_hit_read[10]: begin
2002         reg_rdata_next[0] = error_status_cmdbusy_qs;
2003         reg_rdata_next[1] = error_status_overflow_qs;
2004         reg_rdata_next[2] = error_status_underflow_qs;
2005         reg_rdata_next[3] = error_status_cmdinvalid_qs;
2006         reg_rdata_next[4] = error_status_csinvalid_qs;
2007         reg_rdata_next[5] = error_status_accessinvalid_qs;
2008     end
2009
2010     rac1_addr_hit_read[11]: begin
2011         reg_rdata_next[0] = event_enable_rxfull_qs;
2012         reg_rdata_next[1] = event_enable_txempty_qs;
2013         reg_rdata_next[2] = event_enable_rxwm_qs;
2014         reg_rdata_next[3] = event_enable_txwm_qs;
2015         reg_rdata_next[4] = event_enable_ready_qs;
2016         reg_rdata_next[5] = event_enable_idle_qs;
2017     end
2018
2019     default: begin
2020         reg_rdata_next = '1';
2021     end
2022 endcase
2023 end
2024
2025 // shadow busy
2026 logic shadow_busy;
2027 assign shadow_busy = 1'b0;
2028
2029 // register busy
2030 assign reg_busy = shadow_busy;
2031
2032 // Unused signal tieoff
2033
2034 // wdata / byte enable are not always fully used
2035 // add a blanket unused statement to handle lint waivers
2036 logic unused_wdata;
2037 logic unused_be;
2038 assign unused_wdata = ^reg_wdata;
2039 assign unused_be = ^reg_be;
2040 logic unused_policy_sel;
2041 assign unused_policy_sel = ^rac1_policies_i;
2042
2043 // Assertions for Register Interface
2044 `ASSERT_PULSE(wePulse, reg_we, clk_i, !rst_ni)
2045 `ASSERT_PULSE(rePulse, reg_re, clk_i, !rst_ni)
2046
2047 `ASSERT(reAfterRv, $rose(reg_re || reg_we) | =>
tlo_pre_d_valid, clk_i, !rst_ni)
2048
2049 `ASSERT(en2addrHit, (reg_we || reg_re) | => $one-
hot0(addr_hit), clk_i, !rst_ni)
2050
2051 // this is formulated as an assumption such that the FPV
testbenches do disprove this
2052 // property by mistake
2053 `ASSUME(reqParity, tl_reg_h2d.a_valid | =>
tl_reg_h2d.a_user.chk_en == tlul_pkg::CheckDis)
2054
2055 endmodule

```

11. lowRISC_opentitan/hw/ip/spi_host/rtl/ spi_host_shift_register.sv (115 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Shift Register for Serial Peripheral Interface (SPI) Host
IP.
6 //
7
8 `include "prim_assert.sv"
9
10 module spi_host_shift_register (
11     input          clk_i,
12     input          rst_ni,
13     input          wr_en_i,
14     output logic   wr_ready_o,
15     input          rd_en_i,
16     output logic   rd_ready_o,
17
18     input          [1:0] speed_i,
19     input          shift_en_i,
20     input          sample_en_i,
21     input          full_cyc_i,
22     input          last_read_i,
23     input          last_write_i,
24
25     input          [7:0] tx_data_i,
26     input          tx_valid_i,
27     output         tx_ready_o,
28     output         tx_flush_o,
29
30     output logic   [7:0] rx_data_o,
31     output logic   rx_valid_o,

```

```

32     input          rx_ready_i,
33     output logic   rx_last_o,
34
35     input          [3:0] sd_i,
36     output logic   [3:0] sd_o,
37
38     input          sw_rst_i
39 );
40 import spi_host_cmd_pkg::*;
41
42     logic          [7:0] sr_q;
43     logic          [7:0] sr_d;
44     logic          [3:0] sd_i_q, sd_i_d;
45     logic          [3:0] next_bits;
46     logic          [7:0] sr_shifted;
47
48 // 9-bit buffer to hold shift register data
49 // and the last_read bit to indicate whether
50 // it is the last in a series.
51     logic          [8:0] rx_buf_q;
52     logic          [8:0] rx_buf_d;
53     logic          rx_buf_valid_q;
54     logic          rx_buf_valid_d;
55
56 `ASSERT(SpeedValid, speed_i != RsvdSpd, clk_i, rst_ni)
57
58     assign next_bits = full_cyc_i ? sd_i : sd_i_q;
59     assign sr_shifted = (speed_i == Standard) ? {sr_q[6:0],
next_bits[1]} :
60                                     (speed_i == Dual) ? {sr_q[5:0],
next_bits[1:0]} :
61                                     (speed_i == Quad) ? {sr_q[3:0],
next_bits[3:0]} :
62                                     8'h00;
63
64     assign sd_o = (speed_i == Standard) ? {3'b000,
sr_q[7]} :
65                                     (speed_i == Dual) ? {2'b00,
sr_q[7:6]} :
66                                     (speed_i == Quad) ? {sr_q[7:4] :
4'h0;
67
68 // Buffer the rx_data outputs to simplify three-way flow
control
69 // between fsm, shift reg and byte_merge.
70     assign rd_ready_o = ~rx_buf_valid_q | (rx_val-
id_o & rx_ready_i);
71     assign rx_valid_o = rx_buf_valid_q;
72
73     assign rx_buf_d = sw_rst_i ?
9'h0 :
74                                     (rd_en_i && rd_ready_o) ?
{last_read_i, sr_shifted} :
75                                     rx_buf_q;
76
77     assign rx_buf_valid_d = sw_rst_i ?
1'b0 :
78                                     (rd_en_i && rd_ready_o) ?
1'b1 :
79                                     (rx_valid_o & rx_ready_i) ?
1'b0 :
80                                     rx_buf_valid_q;
81
82     assign sd_i_d = sw_rst_i ? 4'b0 :
83                                     sample_en_i ? sd_i :
84                                     sd_i_q;
85
86     assign {rx_last_o, rx_data_o} = rx_buf_q;
87
88 // tx_data cannot be buffered, because of the
89 // need to transmit tx_flush as we fetch the
90 // last-needed byte.
91     assign wr_ready_o = tx_valid_i;
92     assign tx_ready_o = wr_en_i;
93     assign tx_flush_o = last_write_i;
94
95     assign sr_d = sw_rst_i ? 8'h0 :
96                                     (wr_en_i & wr_ready_o) ? tx_data_i :
97                                     shift_en_i ? sr_shifted :
98                                     sr_q;
99
100     always_ff @(posedge clk_i or negedge rst_ni) begin
101         if (!rst_ni) begin
102             sd_i_q <= 4'h0;
103             sr_q <= 8'h0;
104             rx_buf_valid_q <= 1'b0;
105             rx_buf_q <= 9'h0;
106         end else begin
107             sd_i_q <= sd_i_d;
108             sr_q <= sr_d;
109             rx_buf_valid_q <= rx_buf_valid_d;
110             rx_buf_q <= rx_buf_d;
111         end
112     end
113 end
114
115 endmodule : spi_host_shift_register

```


12. lowRISC_opentitan/hw/ip/spi_host/rtl/ spi_host_window.sv (127 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Module to manage TX & RX FIFO windows for Serial Peripheral In-
  terface (SPI) host IP.
6 //
7
8 `include "prim_assert.sv"
9
10 module spi_host_window
11 #(
12     parameter bit EnableR-
13     parameter bit RACLer-
14     parameter top_racl_pkg::racl_policy_sel_t RaclPolicySelWin-
15     RXDATA = 0,
16     parameter top_racl_pkg::racl_policy_sel_t RaclPolicySel-
17     WinTXDATA = 0
18 ) (
19     input clk_i,
20     input rst_ni,
21     input tlul_pkg::tl_h2d_t rx_win_i,
22     output tlul_pkg::tl_d2h_t rx_win_o,
23     input tlul_pkg::tl_h2d_t tx_win_i,
24     output tlul_pkg::tl_d2h_t tx_win_o,
25     output logic [31:0] tx_data_o,
26     output logic [3:0] tx_be_o,
27     output logic tx_valid_o,
28     input [31:0] rx_data_i,
29     output logic rx_ready_o,
30     // RACL interface
31     input top_racl_pkg::racl_policy_vec_t racl_policies_i,
32     output top_racl_pkg::racl_error_log_t racl_error_tx_o,
33     output top_racl_pkg::racl_error_log_t racl_error_rx_o
34 );
35
36 localparam int AW = spi_host_reg_pkg::BlockAw;
37 localparam int DW = 32;
38 localparam int ByteMaskW = DW / 8;
39 localparam top_racl_pkg::racl_range_t [0:0] RaclPoli-
40 cySelRangesTXDATA = '{
41     '{
42         base: {top_pkg::TL_AW{1'b0}},
43         limit: {top_pkg::TL_AW{1'b1}},
44         policy_sel:
45         top_racl_pkg::racl_policy_sel_t'(RaclPolicySelWinTXDATA),
46         enable: 1'b1
47     }
48 };
49
50 logic rx_we;
51
52 // Only support reads from the data RX fifo window
53 logic rx_access_error;
54 assign rx_access_error = rx_we;
55
56 tlul_adapter_reg_racl #(
57     .RegAw ( AW ),
58     .RegDw ( DW ),
59     .EnableDataIntgGen ( 0 ),
60     .EnableRacl ( EnableRacl ),
61     .RaclErrorRsp ( RaclErrorRsp ),
62     .RaclPolicySelVec ( RaclPolicySelWinRXDATA ),
63 ) u_adapter_rx (
64     .clk_i,
65     .rst_ni,
66     .tl_i ( rx_win_i ),
67     .tl_o ( rx_win_o ),
68     .en_ifetch_i ( prim_mubi_pkg::MuBi4False ),
69     .intg_error_o ( ),
70     .racl_policies_i ( racl_policies_i ),
71     .racl_error_o ( racl_error_rx_o ),
72     .we_o ( rx_we ),
73     .re_o ( rx_ready_o ),
74     .addr_o ( ),
75     .wdata_o ( ),
76     .be_o ( ),
77     .rdata_i ( rx_data_i ),
78     .error_i ( rx_access_error ),
79     .busy_i ( '0'
80 );
81
82 // translate bitmask to byte mask
83 logic [DW-1:0] bit_mask;
84 for (genvar i = 0; i < ByteMaskW; i++) begin : gen_byte_mask
85     assign tx_be_o[i] = |bit_mask[i*8 +: 8];
86 end
87
88 // all the bits of particular byte must be the same
89 `ASSERT(BitMaskCheck_A, (|bit_mask[i*8 +: 8] == 1'b0) ||
90 (&bit_mask[i*8 +: 8] == 1'b1))
91
92 end
93
94 // Only support writes to the data TX fifo window
95 tlul_adapter_sram_racl #(
96     .SramAw(AW),
97     .SramDw(DW),
98     .Outstanding(1),
99     .ByteAccess(1),
100     .ErrOnWrite(0),
101     .ErrOnRead(1),
102     .EnableRacl(EnableRacl),
103     .RaclErrorRsp(RaclErrorRsp),
104     .RaclPolicySelNumRanges(1)
105 ) u_adapter_tx (
106     .clk_i,
107     .rst_ni,
108     .tl_i(tx_win_i),
109     .tl_o(tx_win_o),
110     .en_ifetch_i(prim_mubi_pkg::MuBi4False),
111     .req_o(tx_valid_o),
112     .req_type_o(),
113     .gnt_i(1'b1),
114     .we_o(),
115     .addr_o(),
116     .wdata_o(tx_data_o),
117     .wmask_o(bit_mask),
118     .intg_error_o(),
119     .user_rsvd_o(),
120     .rdata_i('0),
121     .rvalid_i('0),
122     .error_i('0),
123     .compound_txn_in_progress_o(),
124     .readback_en_i(prim_mubi_pkg::MuBi4False),
125     .readback_error_o(),
126     .wr_collision_i(1'b0),
127     .write_pending_i(1'b0),
128     .racl_policies_i ( racl_policies_i ),
129     .racl_error_o ( racl_error_tx_o ),
130     .racl_policy_sel_ranges_i(RaclPolicySelRangesTXDATA)
131 );
132
133 endmodule : spi_host_window

```

I2C 컨트롤러

What this is. An I2C controller supporting clock stretching, arbitration and multi-master operation.

Why it exists. Because I2C is an open-drain bidirectional bus, which makes it unusual from a digital design viewpoint: targets may stretch the clock, and simultaneous masters must arbitrate. Neither behaviour fits a simple shift register.

Undergraduate review · EECS concepts. Open-drain / wired-AND signalling; bus arbitration; input filtering and debouncing.

Graduate · Practice note. Clock stretching is frequently unimplemented, and that omission is a recurring field failure. Check for it in any I2C IP you evaluate, and state it explicitly in any you sell.

8 files • 3,857 lines

API — i2c.sv

Table 947. lowRISC_opentitan/hw/ip/i2c/rtl/i2c.sv — module / interface / package (1개)

이름	종류
i2c	module

Table 948. lowRISC_opentitan/hw/ip/i2c/rtl/i2c.sv — parameter (5개)

파라미터	형/폭	기본값
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
InputDelayCycles	int unsigned	0
EnableRacl	bit	1'b0
RaclErrorRsp	bit	EnableRacl

Table 949. lowRISC_opentitan/hw/ip/i2c/rtl/i2c.sv — 포트 (32개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
ram_cfg_i	input	prim_ram_1p_pkg::ram_1p_cfg_req_t
ram_cfg_o	output	prim_ram_1p_pkg::ram_1p_cfg_rsp_t
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t

포트	방향	형/폭
cio_scl_i	input	
cio_scl_o	output	logic
cio_scl_en_o	output	logic
cio_sda_i	input	
cio_sda_o	output	logic
cio_sda_en_o	output	logic
lsio_trigger_o	output	logic
intr_fmt_threshold_o	output	logic
intr_rx_threshold_o	output	logic
intr_acq_threshold_o	output	logic
intr_rx_overflow_o	output	logic
intr_controller_halt_o	output	logic
intr_scl_interference_o	output	logic
intr_sda_interference_o	output	logic
intr_stretch_timeout_o	output	logic
intr_sda_unstable_o	output	logic
intr_cmd_complete_o	output	logic
intr_tx_stretch_o	output	logic
intr_tx_threshold_o	output	logic
intr_acq_stretch_o	output	logic
intr_unexp_stop_o	output	logic
intr_host_timeout_o	output	logic

API — i2c_bus_monitor.sv

Table 950. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_bus_monitor.sv — module / interface / package (1개)

이름	종류
i2c_bus_monitor	module

Table 951. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_bus_monitor.sv — 포트 (18개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
scl_i	input	
sda_i	input	
controller_enable_i	input	
multi_controller_enable_i	input	

포트	방향	형/폭
target_enable_i	input	
target_idle_i	input	
thd_dat_i	input	[12:0]
t_buf_i	input	[12:0]
bus_active_timeout_i	input	[29:0]
bus_active_timeout_en_i	input	
bus_inactive_timeout_i	input	[19:0]
bus_free_o	output	logic
start_detect_o	output	
stop_detect_o	output	
event_bus_active_timeout_o	output	
event_host_timeout_o	output	

API — i2c_controller_fsm.sv

Table 952. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_controller_fsm.sv — module / interface / package (1개)

이름	종류
i2c_controller_fsm	module

Table 953. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_controller_fsm.sv — parameter (1개)

파라미터	형/폭	기본값
FifoDepth	int	64

Table 954. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_controller_fsm.sv — localparam (1개)

상수	형/폭	값
FifoDepthWidth	int	\$clog2(FifoDepth+1)

Table 955. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_controller_fsm.sv — 포트 (44개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
scl_i	input	
scl_o	output	logic
sda_i	input	
sda_o	output	logic
bus_free_i	input	
transmitting_o	output	logic
host_enable_i	input	

포트	방향	형/폭
halt_controller_i	input	
fmt_fifo_rvalid_i	input	
fmt_fifo_depth_i	input	[FifoDepthWidth-1:0]
fmt_fifo_rready_o	output	logic
fmt_byte_i	input	[7:0]
fmt_flag_start_before_i	input	
fmt_flag_stop_after_i	input	
fmt_flag_read_bytes_i	input	
fmt_flag_read_continue_i	input	
fmt_flag_nak_ok_i	input	
unhandled_unexp_nak_i	input	
unhandled_nak_timeout_i	input	
rx_fifo_wvalid_o	output	logic
rx_fifo_wdata_o	output	logic [RX_FIFO_WIDTH-1:0]
host_idle_o	output	logic
thigh_i	input	[12:0]
tlow_i	input	[12:0]
t_r_i	input	[12:0]
t_f_i	input	[12:0]
thd_sta_i	input	[12:0]
tsu_sta_i	input	[12:0]
tsu_sto_i	input	[12:0]
thd_dat_i	input	[12:0]
sda_interference_i	input	
stretch_timeout_i	input	[29:0]
timeout_enable_i	input	
host_nack_handler_timeout_i	input	[30:0]
host_nack_handler_timeout_en_i	input	
event_nak_o	output	logic
event_unhandled_nak_timeout_o	output	logic
event_arbitration_lost_o	output	logic
event_scl_interference_o	output	logic
event_stretch_timeout_o	output	logic
event_sda_unstable_o	output	logic
complete	output	logic event_cmd_complete_o // Command is

API — i2c_core.sv**Table 956. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_core.sv — module / interface / package (1개)**

이름	종류
i2c_core	module

Table 957. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_core.sv — parameter (1개)

파라미터	형/폭	기본값
InputDelayCycles	int unsigned	0

Table 958. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_core.sv — localparam (4개)

상수	형/폭	값
FifoDepthW	int unsigned	\$clog2(FifoDepth + 1)
AcqFifoDepthW	int unsigned	\$clog2(AcqFifoDepth+1)
MaxFifoDepthW	int unsigned	12
RoundTripCycles	int unsigned	InputDelayCycles + 2 + 1

Table 959. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_core.sv — 포트 (26개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
ram_cfg_i	input	prim_ram_1p_pkg::ram_1p_cfg_req_t
ram_cfg_o	output	prim_ram_1p_pkg::ram_1p_cfg_rsp_t
reg2hw	input	i2c_reg_pkg::i2c_reg2hw_t
hw2reg	output	i2c_reg_pkg::i2c_hw2reg_t
scl_i	input	
scl_o	output	logic
sda_i	input	
sda_o	output	logic
lsio_trigger_o	output	logic
intr_fmt_threshold_o	output	logic
intr_rx_threshold_o	output	logic
intr_acq_threshold_o	output	logic
intr_rx_overflow_o	output	logic
intr_controller_halt_o	output	logic
intr_scl_interference_o	output	logic
intr_sda_interference_o	output	logic
intr_stretch_timeout_o	output	logic
intr_sda_unstable_o	output	logic

포트	방향	형/폭
intr_cmd_complete_o	output	logic
intr_tx_stretch_o	output	logic
intr_tx_threshold_o	output	logic
intr_acq_stretch_o	output	logic
intr_unexp_stop_o	output	logic
intr_host_timeout_o	output	logic

API — i2c_fifo_sync_sram_adapter.sv

Table 960. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_fifo_sync_sram_adapter.sv — module / interface / package (1개)

이름	종류
i2c_fifo_sync_sram_adapter	module

Table 961. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_fifo_sync_sram_adapter.sv — parameter (4개)

파라미터	형/폭	기본값
Width	int unsigned	1
Depth	int unsigned	3
SramAw	int unsigned	1
SramBaseAddr	logic [SramAw-1:0]	'0

Table 962. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_fifo_sync_sram_adapter.sv — localparam (9개)

상수	형/폭	값
DepthW	int unsigned	prim_util_pkg::vbits(Depth + 1)
InpBufDepth	int unsigned	2
OupBufDepth	int unsigned	2
SramFifoDepth	int	Depth - OupBufDepth
InpBufDepthW	int unsigned	prim_util_pkg::vbits(InpBufDepth + 1)
OupBufDepthW	int unsigned	prim_util_pkg::vbits(OupBufDepth + 1)
SramPtrW	int unsigned	prim_util_pkg::vbits(SramFifoDepth)
SramDepthW	int unsigned	prim_util_pkg::vbits(SramFifoDepth + 1)
SramAddrLeadingZeros	int unsigned	SramAw - SramPtrW

Table 963. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_fifo_sync_sram_adapter.sv — 포트 (19개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
clr_i	input	logic

포트	방향	형/폭
fifo_wvalid_i	input	logic
fifo_wready_o	output	logic
fifo_wdata_i	input	logic [Width-1:0]
fifo_rvalid_o	output	logic
fifo_rready_i	input	logic
fifo_rdata_o	output	logic [Width-1:0]
fifo_depth_o	output	logic [DepthW-1:0]
sram_req_o	output	logic
sram_gnt_i	input	logic
sram_write_o	output	logic
sram_addr_o	output	logic [SramAw-1:0]
sram_wdata_o	output	logic [Width-1:0]
sram_wmask_o	output	logic [Width-1:0]
sram_rdata_i	input	logic [Width-1:0]
sram_rvalid_i	input	logic
err_o	output	logic

API — i2c_fifos.sv

Table 964. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_fifos.sv — module / interface / package (17개)

이름	종류
i2c_fifos	module

Table 965. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_fifos.sv — localparam (12개)

상수	형/폭	값
FifoDepthW	int unsigned	$\$clog2(\text{FifoDepth} + 1)$
AcqFifoDepthW	int unsigned	$\$clog2(\text{AcqFifoDepth} + 1)$
RamDepth	int unsigned	464
RamAw	int unsigned	$\text{prim_util_pkg::vbits}(\text{RamDepth})$
RamWidth	int unsigned	13
FmtFifoRamBaseAddr	logic [RamAw-1:0]	$\text{RamAw}'(0)$
RxFifoRamBaseAddr	logic [RamAw-1:0]	$\text{RamAw}'(1 * \text{FifoDepth})$
TxFifoRamBaseAddr	logic [RamAw-1:0]	$\text{RamAw}'(2 * \text{FifoDepth})$
AcqFifoRamBaseAddr	logic [RamAw-1:0]	$\text{RamAw}'(3 * \text{FifoDepth})$
RamArbN	int unsigned	4
RamArbIdxW	int unsigned	$\text{prim_util_pkg::vbits}(\text{RamArbN})$
RamArbDw	int unsigned	$1 + \text{RamAw} + \text{RamWidth}$

Table 966. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_fifos.sv — 포트 (36개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
ram_cfg_i	input	prim_ram_1p_pkg::ram_1p_cfg_req_t
ram_cfg_o	output	prim_ram_1p_pkg::ram_1p_cfg_rsp_t
fmt_fifo_clr_i	input	logic
fmt_fifo_depth_o	output	logic [FifoDepthW-1:0]
fmt_fifo_wvalid_i	input	logic
fmt_fifo_wready_o	output	logic
fmt_fifo_wdata_i	input	logic [FMT_FIFO_WIDTH-1:0]
fmt_fifo_rvalid_o	output	logic
fmt_fifo_rready_i	input	logic
fmt_fifo_rdata_o	output	logic [FMT_FIFO_WIDTH-1:0]
rx_fifo_clr_i	input	logic
rx_fifo_depth_o	output	logic [FifoDepthW-1:0]
rx_fifo_wvalid_i	input	logic
rx_fifo_wready_o	output	logic
rx_fifo_wdata_i	input	logic [RX_FIFO_WIDTH-1:0]
rx_fifo_rvalid_o	output	logic
rx_fifo_rready_i	input	logic
rx_fifo_rdata_o	output	logic [RX_FIFO_WIDTH-1:0]
tx_fifo_clr_i	input	logic
tx_fifo_depth_o	output	logic [FifoDepthW-1:0]
tx_fifo_wvalid_i	input	logic
tx_fifo_wready_o	output	logic
tx_fifo_wdata_i	input	logic [TX_FIFO_WIDTH-1:0]
tx_fifo_rvalid_o	output	logic
tx_fifo_rready_i	input	logic
tx_fifo_rdata_o	output	logic [TX_FIFO_WIDTH-1:0]
acq_fifo_clr_i	input	logic
acq_fifo_depth_o	output	logic [AcqFifoDepthW-1:0]
acq_fifo_wvalid_i	input	logic
acq_fifo_wready_o	output	logic
acq_fifo_wdata_i	input	logic [ACQ_FIFO_WIDTH-1:0]
acq_fifo_rvalid_o	output	logic

포트	방향	형/폭
acq_fifo_rready_i	input	logic
acq_fifo_rdata_o	output	logic [ACQ_FIFO_WIDTH-1:0]

API — i2c_pkg.sv

Table 967. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_pkg.sv — module / interface / package (1개)

이름	종류
i2c_pkg	package

Table 968. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_pkg.sv — parameter (5개)

파라미터	형/폭	기본값
I2C_ACQ_BYTE_ID_WIDTH	int unsigned	3
FMT_FIFO_WIDTH	int unsigned	8 + 5
RX_FIFO_WIDTH	int unsigned	8
TX_FIFO_WIDTH	int unsigned	8
ACQ_FIFO_WIDTH	int unsigned	I2C_ACQ_BYTE_ID_WIDTH + 8

API — i2c_reg_pkg.sv

Table 969. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_reg_pkg.sv — module / interface / package (1개)

이름	종류
i2c_reg_pkg	package

Table 970. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_reg_pkg.sv — parameter (69개) — 앞 60개만 표시(전체 69개, 본문 소스 참조)

파라미터	형/폭	기본값
FifoDepth	int	64
AcqFifoDepth	int	268
NumAlerts	int	1
BlockAw	int	7
NumRegs	int	32
I2C_INTR_STATE_OFFSET	logic [BlockAw-1:0]	7'h 0
I2C_INTR_ENABLE_OFFSET	logic [BlockAw-1:0]	7'h 4
I2C_INTR_TEST_OFFSET	logic [BlockAw-1:0]	7'h 8
I2C_ALERT_TEST_OFFSET	logic [BlockAw-1:0]	7'h c
I2C_CTRL_OFFSET	logic [BlockAw-1:0]	7'h 10
I2C_STATUS_OFFSET	logic [BlockAw-1:0]	7'h 14
I2C_RDATA_OFFSET	logic [BlockAw-1:0]	7'h 18
I2C_FDATA_OFFSET	logic [BlockAw-1:0]	7'h 1c

파라미터	형/폭	기본값
I2C_FIFO_CTRL_OFFSET	logic [BlockAw-1:0]	7'h 20
I2C_HOST_FIFO_CONFIG_OFFSET	logic [BlockAw-1:0]	7'h 24
I2C_TARGET_FIFO_CONFIG_OFFSET	logic [BlockAw-1:0]	7'h 28
I2C_HOST_FIFO_STATUS_OFFSET	logic [BlockAw-1:0]	7'h 2c
I2C_TARGET_FIFO_STATUS_OFFSET	logic [BlockAw-1:0]	7'h 30
I2C_OVRD_OFFSET	logic [BlockAw-1:0]	7'h 34
I2C_VAL_OFFSET	logic [BlockAw-1:0]	7'h 38
I2C_TIMING0_OFFSET	logic [BlockAw-1:0]	7'h 3c
I2C_TIMING1_OFFSET	logic [BlockAw-1:0]	7'h 40
I2C_TIMING2_OFFSET	logic [BlockAw-1:0]	7'h 44
I2C_TIMING3_OFFSET	logic [BlockAw-1:0]	7'h 48
I2C_TIMING4_OFFSET	logic [BlockAw-1:0]	7'h 4c
I2C_TIMEOUT_CTRL_OFFSET	logic [BlockAw-1:0]	7'h 50
I2C_TARGET_ID_OFFSET	logic [BlockAw-1:0]	7'h 54
I2C_ACQDATA_OFFSET	logic [BlockAw-1:0]	7'h 58
I2C_TXDATA_OFFSET	logic [BlockAw-1:0]	7'h 5c
I2C_HOST_TIMEOUT_CTRL_OFFSET	logic [BlockAw-1:0]	7'h 60
I2C_TARGET_TIMEOUT_CTRL_OFFSET	logic [BlockAw-1:0]	7'h 64
I2C_TARGET_NACK_COUNT_OFFSET	logic [BlockAw-1:0]	7'h 68
I2C_TARGET_ACK_CTRL_OFFSET	logic [BlockAw-1:0]	7'h 6c
I2C_ACQ_FIFO_NEXT_DATA_OFFSET	logic [BlockAw-1:0]	7'h 70
I2C_HOST_NACK_HANDLER_TIMEOUT_OFFSET	logic [BlockAw-1:0]	7'h 74
I2C_CONTROLLER_EVENTS_OFFSET	logic [BlockAw-1:0]	7'h 78
I2C_TARGET_EVENTS_OFFSET	logic [BlockAw-1:0]	7'h 7c
I2C_INTR_TEST_RESVAL	logic [14:0]	15'h 0
I2C_INTR_TEST_FMT_THRESHOLD_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_RX_THRESHOLD_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_ACQ_THRESHOLD_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_RX_OVERFLOW_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_CONTROLLER_HALT_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_SCL_INTERFERENCE_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_SDA_INTERFERENCE_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_STRETCH_TIMEOUT_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_SDA_UNSTABLE_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_CMD_COMPLETE_RESVAL	logic [0:0]	1'h 0

파라미터	형/폭	기본값
I2C_INTR_TEST_TX_STRETCH_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_TX_THRESHOLD_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_ACQ_STRETCH_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_UNEXP_STOP_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_HOST_TIMEOUT_RESVAL	logic [0:0]	1'h 0
I2C_ALERT_TEST_RESVAL	logic [0:0]	1'h 0
I2C_ALERT_TEST_FATAL_FAULT_RESVAL	logic [0:0]	1'h 0
I2C_STATUS_RESVAL	logic [10:0]	11'h 33c
I2C_STATUS_FMEMPTY_RESVAL	logic [0:0]	1'h 1
I2C_STATUS_HOSTIDLE_RESVAL	logic [0:0]	1'h 1
I2C_STATUS_TARGETIDLE_RESVAL	logic [0:0]	1'h 1
I2C_STATUS_RXEMPTY_RESVAL	logic [0:0]	1'h 1

1. lowRISC_opentitan/hw/ip/i2c/rtl/i2c.sv (184 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: I2C top level wrapper file
6
7 `include "prim_assert.sv"
8
9 module i2c
10   import i2c_reg_pkg::*;
11   #(
12     parameter logic [NumAlerts-1:0]      AlertAsyn-
13       = {NumAlerts{1'b1}},
14     // Number of cycles a differential skew is tolerated on the
15     alert signal
16     parameter int unsigned                AlertSkew-
17       = 1,
18     parameter int unsigned                InputDelay-
19       = 0,
20     parameter bit                         EnableR-
21       = 1'b0,
22     parameter bit                         RaclEr-
23       = EnableRacl,
24     parameter top_racl_pkg::racl_policy_sel_t RaclPoli-
25     cySelVec[NumRegs] = '{NumRegs{0}}
26   ) (
27     input          clk_i,
28     input          rst_ni,
29     input  prim_ram_1p_pkg::ram_1p_cfg_req_t ram_cfg_i,
30     output prim_ram_1p_pkg::ram_1p_cfg_rsp_t ram_cfg_o,
31
32     // Bus Interface
33     input  tlul_pkg::tl_h2d_t tl_i,
34     output tlul_pkg::tl_d2h_t tl_o,
35
36     // Alerts
37     input  prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
38     alert_rx_i,
39     output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
40     alert_tx_o,
41
42     // RACL interface
43     input  top_racl_pkg::racl_policy_vec_t racl_policies_i,
44     output top_racl_pkg::racl_error_log_t  racl_error_o,
45
46     // Generic IO
47     input          cio_scl_i,
48     output logic   cio_scl_o,
49     output logic   cio_scl_en_o,
50     input          cio_sda_i,
51     output logic   cio_sda_o,
52     output logic   cio_sda_en_o,
53
54     output logic   lsio_trigger_o,
55
56     // Interrupts
57     output logic   intr_fmt_threshold_o,
58
59     output logic   intr_rx_threshold_o,
60     output logic   intr_acq_threshold_o,
61     output logic   intr_rx_overflow_o,
62     output logic   intr_controller_halt_o,
63     output logic   intr_scl_interference_o,
64     output logic   intr_sda_interference_o,
65     output logic   intr_stretch_timeout_o,
66     output logic   intr_sda_unstable_o,
67     output logic   intr_cmd_complete_o,
68     output logic   intr_tx_stretch_o,
69     output logic   intr_tx_threshold_o,
70     output logic   intr_acq_stretch_o,
71     output logic   intr_unexp_stop_o,
72     output logic   intr_host_timeout_o
73   );
74
75   i2c_reg2hw_t reg2hw;
76   i2c_hw2reg_t hw2reg;
77
78   logic [NumAlerts-1:0] alert_test, alerts;
79
80   i2c_reg_top #(
81     .EnableRacl(EnableRacl),
82     .RaclErrorRsp(RaclErrorRsp),
83     .RaclPolicySelVec(RaclPolicySelVec)
84   ) u_reg (
85     .clk_i,
86     .rst_ni,
87     .tl_i,
88     .tl_o,
89     .reg2hw,
90     .hw2reg,
91     .racl_policies_i,
92     .racl_error_o,
93     // SEC_CM: BUS.INTEGRITY
94     .intg_err_o(alerts[0])
95   );
96
97   assign alert_test = {
98     reg2hw.alert_test.q &
99     reg2hw.alert_test.qe
100   };
101
102   for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
103     prim_alert_sender #(
104       .AsyncOn(AlertAsyncOn[i]),
105       .SkewCycles(AlertSkewCycles),
106       .IsFatal(1'b1)
107     ) u_prim_alert_sender (
108       .clk_i,
109       .rst_ni,
110       .alert_test_i ( alert_test[i] ),
111       .alert_req_i   ( alerts[i] ),
112       .alert_ack_o   (          ),
113       .alert_state_o (          ),
114       .alert_rx_i    ( alert_rx_i[i] ),
115       .alert_tx_o    ( alert_tx_o[i] )
116     );
117   end
118
119   logic scl_int;
120   logic sda_int;

```

```

112 i2c_core #(
113     .InputDelayCycles(InputDelayCycles)
114 ) i2c_core (
115     .clk_i,
116     .rst_ni,
117     .ram_cfg_i,
118     .ram_cfg_o,
119
120     .reg2hw,
121     .hw2reg,
122
123     .scl_i(cio_scl_i),
124     .scl_o(scl_int),
125     .sda_i(cio_sda_i),
126     .sda_o(sda_int),
127
128     .lsio_trigger_o,
129
130     .intr_fmt_threshold_o,
131     .intr_rx_threshold_o,
132     .intr_acq_threshold_o,
133     .intr_rx_overflow_o,
134     .intr_controller_halt_o,
135     .intr_scl_interference_o,
136     .intr_sda_interference_o,
137     .intr_stretch_timeout_o,
138     .intr_sda_unstable_o,
139     .intr_cmd_complete_o,
140     .intr_tx_stretch_o,
141     .intr_tx_threshold_o,
142     .intr_acq_stretch_o,
143     .intr_unexp_stop_o,
144     .intr_host_timeout_o
145 );
146
147 // For I2C, in standard, fast and fast-plus modes, outputs
simulated as open-drain outputs.
148 // Asserting scl or sda high should be equivalent to a tri-
state output.
149 // The output, when enabled, should only assert low.
150
151 assign cio_scl_o = 1'b0;
152 assign cio_sda_o = 1'b0;
153
154 assign cio_scl_en_o = ~scl_int;
155 assign cio_sda_en_o = ~sda_int;
156
157 `ASSERT_KNOWN(TIDValidKnown0_A, tl_o.d_valid)
158 `ASSERT_KNOWN(TIAReadyKnown0_A, tl_o.a_ready)
159 `ASSERT_KNOWN(AlertKnown0_A, alert_tx_o)
160 `ASSERT_KNOWN(CioSclKnown0_A, cio_scl_o)
161 `ASSERT_KNOWN(CioSclEnKnown0_A, cio_scl_en_o)
162 `ASSERT_KNOWN(CioSdaKnown0_A, cio_sda_o)
163 `ASSERT_KNOWN(CioSdaEnKnown0_A, cio_sda_en_o)
164 `ASSERT_KNOWN(IntrFmtWtmkKnown0_A, intr_fmt_threshold_o)
165 `ASSERT_KNOWN(IntrRxWtmkKnown0_A, intr_rx_threshold_o)
166 `ASSERT_KNOWN(IntrAcqWtmkKnown0_A, intr_acq_threshold_o)
167 `ASSERT_KNOWN(IntrRxOflwKnown0_A, intr_rx_overflow_o)
168 `ASSERT_KNOWN(IntrControllerHaltKnown0_A, intr_control-
ler_halt_o)
169 `ASSERT_KNOWN(IntrSclInterfKnown0_A, intr_scl_interfer-
ence_o)
170 `ASSERT_KNOWN(IntrSdaInterfKnown0_A, intr_sda_interfer-
ence_o)
171 `ASSERT_KNOWN(IntrStretchTimeoutKnown0_A,
intr_stretch_timeout_o)
172 `ASSERT_KNOWN(IntrSdaUnstableKnown0_A, intr_sda_unstable_o)
173 `ASSERT_KNOWN(IntrCommandCompleteKnown0_A, intr_cmd_com-
plete_o)
174 `ASSERT_KNOWN(IntrTxStretchKnown0_A, intr_tx_stretch_o)
175 `ASSERT_KNOWN(IntrTxWtmkKnown0_A, intr_tx_threshold_o)
176 `ASSERT_KNOWN(IntrAcqStretchKnown0_A, intr_acq_stretch_o)
177 `ASSERT_KNOWN(IntrUnexpStopKnown0_A, intr_unexp_stop_o)
178 `ASSERT_KNOWN(IntrHostTimeoutKnown0_A, intr_host_timeout_o)
179 `ASSERT_KNOWN(LsioTriggerKnown0_A, lsio_trigger_o)
180 `ASSERT_KNOWN(RaclErrorValidKnown0_A, racl_error_o.valid)
181
182 // Alert assertions for reg_we onehot check
183 `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
u_reg, alert_tx_o[0])
184 endmodule

```

2. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_bus_monitor.sv (306 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: I2C bus idle and timeout monitoring, plus
Start/Stop
6 // detection.
7
8 module i2c_bus_monitor import i2c_pkg::*;
9 (
10     input          clk_i,
11     input          rst_ni,

```

```

12
13     input          scl_i,
14     input          sda_i,
15
16     input          controller_enable_i,
17     input          multi_controller_enable_i,
18     input          target_enable_i,
19     input          target_idle_i,
20     input [12:0]   thd_dat_i,          //
Data hold time(< 200 ns, < thd_sta)
21     input [12:0]   t_buf_i,          //
Bus free time (< 5 us)
22     input [29:0]   bus_active_timeout_i, //
SCL held low (~25 ms)
23     input          bus_active_timeout_en_i,
24     input [19:0]   bus_inactive_timeout_i, //
SCL held high (~50 us)
25
26     output logic   bus_free_o,
27     output         start_detect_o,
28     output         stop_detect_o,
29
30     output          event_bus_active_timeout_o,
31     output          event_host_timeout_o,
32
33 );
34
35 // Only activate this monitor if at least one of the modules is
enabled.
36 logic monitor_enable, monitor_enable_q;
37 assign monitor_enable = controller_enable_i | tar-
get_enable_i | multi_controller_enable_i;
38
39 always_ff @(posedge clk_i or negedge rst_ni) begin
40     if (!rst_ni) begin
41         monitor_enable_q <= 1'b0;
42     end else begin
43         monitor_enable_q <= monitor_enable;
44     end
45 end
46
47 // SDA and SCL at the previous clock edge
48 logic scl_i_q, sda_i_q;
49 always_ff @(posedge clk_i or negedge rst_ni) begin :
bus_prev
50     if (!rst_ni) begin
51         scl_i_q <= 1'b1;
52         sda_i_q <= 1'b1;
53     end else begin
54         scl_i_q <= scl_i;
55         sda_i_q <= sda_i;
56     end
57 end
58
59 // Start and Stop detection
60 //
61 // To resolve ambiguity with early SDA arrival, reject con-
trol symbols when
62 // SCL goes low too soon. The hold time for ordinary data/
ACK bits is too
63 // short to reliably see SCL change before SDA. Use the hold
time for
64 // control signals to ensure a Start or Stop symbol was ac-
tually received.
65 // Requirements: thd_dat + 1 < thd_sta
66 // The extra (+1) here is to account for a late SDA ar-
rival due to CDC
67 // skew.
68 //
69 // Note that this counter combines Start and Stop detection
into one
70 // counter. A controller-only reset scenario could end up
with a Stop
71 // following shortly after a Start, with the requisite setup
time not
72 // observed.
73 logic start_det_trigger, start_det_pending;
74 logic start_det; // indicates start or repeated
start is detected on the bus
75 logic stop_det_trigger, stop_det_pending;
76 logic stop_det; // indicates stop is detected on
the bus
77
78 // Stop / Start detection counter
79 logic [13:0] ctrl_det_count;
80
81 always_ff @(posedge clk_i or negedge rst_ni) begin
82     if (!rst_ni) begin
83         ctrl_det_count <= '0;
84     end else if (start_det_trigger || stop_det_trigger) begin
85         ctrl_det_count <= 14'd1;
86     end else if (start_det_pending || stop_det_pending) begin
87         ctrl_det_count <= ctrl_det_count + 1'b1;
88     end
89 end
90
91 always_ff @(posedge clk_i or negedge rst_ni) begin

```

```

92   if (!rst_ni) begin
93     start_det_pending <= 1'b0;
94   end else if (start_det_trigger) begin
95     start_det_pending <= 1'b1;
96   end else if (!monitor_enable || !scl_i || start_det ||
stop_det_trigger) begin
97     start_det_pending <= 1'b0;
98   end
99 end
100
101 always_ff @(posedge clk_i or negedge rst_ni) begin
102   if (!rst_ni) begin
103     stop_det_pending <= 1'b0;
104   end else if (stop_det_trigger) begin
105     stop_det_pending <= 1'b1;
106   end else if (!monitor_enable || !scl_i || stop_det ||
start_det_trigger) begin
107     stop_det_pending <= 1'b0;
108   end
109 end
110
111 // (Repeated) Start condition detection by target
112 assign start_det_trigger = monitor_enable && (scl_i_q &&
scl_i) & (!sda_i_q && !sda_i);
113 assign start_det = monitor_enable && start_det_pending &&
(ctrl_det_count >= 14'(thd_dat_i));
114 assign start_detect_o = start_det;
115
116 // Stop condition detection by target
117 assign stop_det_trigger = monitor_enable && (scl_i_q &&
scl_i) & (!sda_i_q && sda_i);
118 assign stop_det = monitor_enable && stop_det_pending &&
(ctrl_det_count >= 14'(thd_dat_i));
119 assign stop_detect_o = stop_det;
120
121 //
122 // Bus timeout logic
123 //
124
125 // Detection of bus in a released state.
126 logic bus_idling;
127 assign bus_idling = scl_i && (sda_i == sda_i_q);
128
129 logic [29:0] bus_release_cnt, bus_release_cnt_sel;
130 logic bus_release_cnt_load, bus_release_cnt_dec;
131
132 // bus_inactive_timeout_det is only high for the case where
the bus release
133 // counter reaches zero for bus idling, not the wait after a
Stop condition.
134 logic bus_inactive_timeout_det;
135
136 logic bus_inactive_timeout_en;
137 assign bus_inactive_timeout_en = (bus_inactive_timeout_i >
'0);
138
139 // bus_active_timeout latches high when SCL has been held
low for too long.
140
141 // This can be done intentionally or unintentionally, as the
response for
142
143 // target devices should be to abort the current transaction
and release any
144 // hold on the bus. Note that the bus doesn't immediately
transition to
145 // "free" status, since the controller can continue holding
SCL low for some
146 // time.
147 logic bus_active_timeout_det_d, bus_active_timeout_det_q;
148 always_ff @(posedge clk_i or negedge rst_ni) begin
149   if (!rst_ni) begin
150     bus_active_timeout_det_q <= 1'b0;
151   end else begin
152     bus_active_timeout_det_q <= bus_active_timeout_det_d;
153   end
154 end
155 // Bus release counter.
156 // Together with the FSM below, this counter detects bus
timeouts and the end
157 // of the bus free time following a Stop condition.
158 always_ff @ (posedge clk_i or negedge rst_ni) begin :
bus_idle
159   if (!rst_ni) begin
160     bus_release_cnt <= '0;
161   end else if (monitor_enable && !monitor_enable_q) begin
162     // The rising edge of monitor enable resets the counter
for
163     // multi-controller mode.
164     if (multi_controller_enable_i) begin
165       // For the multi-controller case, wait until the bus
isn't busy before
166       // transmitting.
167       bus_release_cnt <= 30'(bus_inactive_timeout_i);
168     end
169   end else if (bus_release_cnt_load) begin
170     bus_release_cnt <= bus_release_cnt_sel;
171   end else if (bus_release_cnt_dec && (bus_release_cnt !=
'0)) begin

```

```

171     bus_release_cnt <= bus_release_cnt - 1'b1;
172   end
173 end
174
175 typedef enum logic [1:0] {
176   // Bus is currently free. Can transmit.
177   StBusFree,
178   // Bus is busy and not held with SCL high.
179   StBusBusyLow,
180   // Bus is currently busy, but SCL is held high.
181   StBusBusyHigh,
182   // Bus is currently busy, but saw a Stop. Count down to
Bus Free.
183   StBusBusyStop
184 } bus_state_e;
185
186 bus_state_e state_q, state_d;
187
188 always_comb begin
189   state_d = state_q;
190   bus_release_cnt_load = 1'b0;
191   bus_release_cnt_sel = 30'(t_buf_i);
192   bus_release_cnt_dec = 1'b0;
193   bus_inactive_timeout_det = 1'b0;
194   bus_active_timeout_det_d = bus_active_timeout_det_q;
195
196   unique case (state_q)
197     StBusFree: begin
198       bus_active_timeout_det_d = 1'b0;
199
200       if (!scl_i || !sda_i) begin
201         state_d = StBusBusyLow;
202         bus_release_cnt_load = 1'b1;
203         bus_release_cnt_sel = bus_active_timeout_i;
204       end
205     end
206
207     StBusBusyLow: begin
208       bus_release_cnt_dec = !scl_i;
209
210       if (stop_det) begin
211         state_d = StBusBusyStop;
212         bus_release_cnt_load = 1'b1;
213         bus_release_cnt_sel = 30'(t_buf_i);
214       end else if (bus_idling && bus_inactive_timeout_en)
begin
215         state_d = StBusBusyHigh;
216         bus_release_cnt_load = 1'b1;
217         bus_release_cnt_sel = 30'(bus_inactive_timeout_i);
218       end else if (scl_i) begin
219         bus_release_cnt_load = 1'b1;
220         bus_release_cnt_sel = bus_active_timeout_i;
221       end if (bus_active_timeout_det_q) begin
222         // SCL was released due to the bus timeout, so go
to BusFree.
223         state_d = StBusFree;
224       end
225     end else if (bus_release_cnt == 30'd1) begin
226       // The active timeout occurs when SCL has been held
continuously low
227       // for too long. Both the controller and target
should respond to
228       // this timeout and release the bus. We don't con-
sider the bus free
229       // yet, though: SCL must be released first.
230       bus_active_timeout_det_d = bus_active_timeout_en_i;
231     end
232   end
233
234   StBusBusyHigh: begin
235     bus_release_cnt_dec = 1'b1;
236
237     if (stop_det) begin
238       state_d = StBusBusyStop;
239       bus_release_cnt_load = 1'b1;
240       bus_release_cnt_sel = 30'(t_buf_i);
241     end else if (!bus_idling) begin
242       state_d = StBusBusyLow;
243       bus_release_cnt_load = 1'b1;
244       bus_release_cnt_sel = bus_active_timeout_i;
245     end else if (bus_release_cnt == 30'd1) begin
246       // The host_timeout interrupt occurs regardless of
which value of
247       // SDA was present, but only transition to StBusFree
if we entered
248       // this state with SDA high. If SDA is low, a change
to SCL will
249       // cause a transition back to StBusBusyLow. If SDA
changes from low
250       // to high, we get a Stop condition and transition
to StBusBusyStop.
251       bus_inactive_timeout_det = bus_inactive_timeout_en;
252       if (sda_i) begin
253         state_d = StBusFree;
254       end
255     end
256   end
257
258   StBusBusyStop: begin

```

```

259     bus_release_cnt_dec = 1'b1;
260
261     if (!scl_i || !sda_i) begin
262         state_d = StBusBusyLow;
263     end else if (bus_release_cnt == 30'd1) begin
264         state_d = StBusFree;
265     end
266 end
267
268 default: begin
269     state_d = StBusFree;
270 end
271 endcase
272 end
273
274 always_ff @(posedge clk_i or negedge rst_ni) begin
275     if (!rst_ni) begin
276         state_q <= StBusFree;
277     end else if (!monitor_enable) begin
278         state_q <= StBusFree;
279     end else if (monitor_enable && !monitor_enable_q) begin
280         if (multi_controller_enable_i) begin
281             // For the multi-controller case, wait until the bus
282             // isn't busy before
283             // transmitting.
284             state_q <= StBusBusyHigh;
285         end else begin
286             state_q <= StBusFree;
287         end
288     end else begin
289         state_q <= state_d;
290     end
291 end
292
293 always_comb begin
294     if (multi_controller_enable_i) begin
295         bus_free_o = (state_q == StBusFree);
296     end else begin
297         // For single-controller cases, the bus is only "busy"
298         // while waiting for the "bus free" time
299         // after a Stop condition. In other words, that is the
300         // only time our controller can't
301         // continue to the next transaction.
302         bus_free_o = (state_q != StBusBusyStop);
303     end
304 end
305
306 assign event_bus_active_timeout_o = bus_active_timeout_det_d
307 && !bus_active_timeout_det_q;
308 assign event_host_timeout_o = !target_idle_i && bus_inactive_timeout_det;
309
310 endmodule

```

3. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_controller_fsm.v (995 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 // Description: I2C finite state machine
6
7 `include "prim_assert.sv"
8
9 module i2c_controller_fsm import i2c_pkg::*;
10 #(
11     parameter int FifoDepth = 64,
12     localparam int FifoDepthWidth = $clog2(FifoDepth+1)
13 ) (
14     input          clk_i, // clock
15     input          rst_ni, // active low reset
16
17     input          scl_i, // serial clock input from i2c bus
18     output logic   scl_o, // serial clock output to i2c bus
19     input          sda_i, // serial data input from i2c bus
20     output logic   sda_o, // serial data output to i2c bus
21     input          bus_free_i, // High if the bus is free for
22     // a new transmission
23     output logic   transmitting_o, // Controller is currently
24     // transmitting SDA
25
26     input          host_enable_i, // enable host functionality
27     input          halt_controller_i, // Halt the controller FSM
28
29     in idle
30
31     input          fmt_fifo_rvalid_i, // indicates
32     // there is valid data in fmt_fifo
33     input [FifoDepthWidth-1:0] fmt_fifo_depth_i, //
34     // fmt_fifo_depth
35     output logic   fmt_fifo_rready_o, // populates
36     // fmt_fifo
37     input [7:0]    fmt_byte_i, // byte in
38     // fmt_fifo to be sent to target
39
40     input          fmt_flag_start_before_i, // issue
41     // start before sending byte

```

```

32     input          fmt_flag_stop_after_i, // issue
33     // stop after sending byte
34     input          fmt_flag_read_bytes_i, // in-
35     // dicates byte is an number of reads
36     input          fmt_flag_read_continue_i, // host
37     // to send Ack to final byte read
38     input          fmt_flag_nak_ok_i, // no
39     // Ack is expected
40     input          unhandled_unexp_nak_i,
41     // input          unhandled_nak_timeout_i, // NACK
42     // handler timeout event not cleared
43
44     output logic   rx_fifo_wvalid_o, // high
45     // if there is valid data in rx_fifo
46     output logic [RX_FIFO_WIDTH-1:0] rx_fifo_wdata_o, // byte
47     // in rx_fifo read from target
48
49     output logic   host_idle_o, // indicates the host
50     // is idle
51
52     input [12:0]   thigh_i, // high period of the SCL in clock
53     // units
54     input [12:0]   tlow_i, // low period of the SCL in clock
55     // units
56
57     input [12:0]   t_r_i, // rise time of both SDA and SCL in
58     // clock units
59
60     input [12:0]   t_f_i, // fall time of both SDA and SCL in
61     // clock units
62     input [12:0]   thd_sta_i, // hold time for (repeated) START
63     // in clock units
64
65     input [12:0]   tsu_sta_i, // setup time for repeated START in
66     // clock units
67     input [12:0]   tsu_sto_i, // setup time for STOP in clock
68     // units
69     input [12:0]   thd_dat_i, // data hold time in clock units
70     // sda_interference_i, // High when SCL is high
71     // and SDA doesn't match while transmitting
72     input [20:0]   stretch_timeout_i, // max time target connec-
73     // ted to this host may stretch the clock
74     input          timeout_enable_i, // assert if target
75     // stretches clock past max
76     input [30:0]   host_nack_handler_timeout_i, // Timeout
77     // threshold for unhandled Host-Mode 'nak' irq.
78     input          host_nack_handler_timeout_en_i,
79
80     output logic   event_nak_o, // target didn't Ack
81     // when expected
82     output logic   event_unhandled_nak_timeout_o, // SW didn't
83     // handle the NACK in time
84     output logic   event_arbitration_lost_o, // Lost arbitration
85     // after beginning a transaction
86     output logic   event_scl_interference_o, // other device for-
87     // cing SCL low
88     output logic   event_stretch_timeout_o, // target stretches
89     // clock past max time
90     output logic   event_sda_unstable_o, // SDA is not con-
91     // stant during SCL pulse
92     output logic   event_cmd_complete_o, // Command is com-
93     // plete
94 );
95
96 // I2C bus clock timing variables
97 logic [15:0] tcount_q; // current counter for setting
98 delays
99 logic [15:0] tcount_d; // next counter for setting
100 delays
101 logic load_tcount; // indicates counter must be
102 loaded
103 logic [30:0] stretch_idle_cnt; // counter for clock being
104 stretched by target
105 // or clock idle by host.
106 logic [30:0] unhandled_nak_cnt; // In Host-mode, count
107 cycles where the FSM is halted awaiting
108
109 // the nak irq to be handled by
110 SW.
111 logic incr_nak_cnt;
112
113 // (IP in HOST-Mode) This bit is active when the FSM is in a
114 state where a TARGET might
115
116 // be trying to stretch the clock, preventing the controller
117 FSM from continuing.
118 logic stretch_en;
119
120 // Bit and byte counter variables
121 logic [2:0] bit_index; // bit being transmitted to or
122 read from the bus
123 logic bit_decr; // indicates bit_index must be
124 decremented by 1
125 logic bit_clr; // indicates bit_index must be
126 reset to 7
127 logic [8:0] byte_num; // number of bytes to read
128 logic [8:0] byte_index; // byte being read from the bus
129 logic byte_decr; // indicates byte_index must be
130 decremented by 1

```

```

88 logic      byte_clr;      // indicates byte_index must be
reset to byte_num
89
90 // Other internal variables
91 logic      scl_d;          // scl internal
92 logic      sda_d;          // data internal
93 logic      scl_i_q;        // scl_i delayed by one clock
94 logic      sda_i_q;        // sda_i delayed by one clock
95 logic [7:0] read_byte;    // register for reads from tar-
get
96 logic      read_byte_clr;  // clear read_byte contents
97 logic      shift_data_en;  // indicates data must be shif-
ted in from the bus
98 logic      trans_started; // indicates a transaction has
started
99 logic      pend_restart;  // there is a pending restart
waiting to be processed
100 logic      req_restart;   // request restart
101 logic      log_start;     // indicates start is been is-
sued
102 logic      log_stop;      // indicates stop is been issued
103 logic      auto_stop_d, auto_stop_q; // Tracks whether a
Stop symbol was autonomously generated.
104 logic      ctrl_symbol_failed; // If SCL pulls low before
the controller can issue Start/Stop
105
106 // Clock counter implementation
107 typedef enum logic [3:0] {
108     tSetupStart,
109     tHoldStart,
110     tClockStart,
111     tClockLow,
112     tClockPulse,
113     tClockHigh,
114     tHoldBit,
115     tClockStop,
116     tSetupStop,
117     tNoDelay
118 } tcount_sel_e;
119
120 tcount_sel_e tcount_sel;
121
122 always_comb begin : counter_functions
123     tcount_d = tcount_q;
124     if (load_tcount) begin
125         unique case (tcount_sel)
126             tSetupStart : tcount_d = 16'(t_r_i) + 16'(tsu_sta_i);
127             tHoldStart  : tcount_d = 16'(t_f_i) + 16'(thd_sta_i);
128             tClockStart : tcount_d = 16'(thd_dat_i);
129             tClockLow   : tcount_d = 16'(tlow_i) - 16'(thd_dat_i);
130             tClockPulse : tcount_d = 16'(t_r_i) + 16'(thigh_i);
131             tClockHigh  : tcount_d = 16'(thigh_i);
132             tHoldBit    : tcount_d = 16'(t_f_i) + 16'(thd_dat_i);
133             tClockStop  : tcount_d = 16'(t_f_i) + 16'(tlow_i) -
16'(thd_dat_i);
134             tSetupStop  : tcount_d = 16'(t_r_i) + 16'(tsu_sto_i);
135             tNoDelay    : tcount_d = 16'h0001;
136             default     : tcount_d = 16'h0001;
137         endcase
138     end else if (
139         host_enable_i ||
140         // If we disable Host-Mode mid-txn, keep counting until
the end of
141         // byte, at which point we create a STOP condition then
return to Idle.
142         (!host_idle_o && !host_enable_i)) begin
143         tcount_d = tcount_q - 1'b1;
144     end
145 end
146
147 always_ff @ (posedge clk_i or negedge rst_ni) begin :
clk_counter
148     if (!rst_ni) begin
149         tcount_q <= '1;
150     end else begin
151         tcount_q <= tcount_d;
152     end
153 end
154
155 // Clock stretching/idle detection when i2c_ctrl.
156 // When in host mode, this is a stretch count for how long
an external target
157 // has stretched the clock.
158 // When in target mode, this is an idle count for how long
an external host
159 // has kept the clock idle after a START indication.
160 always_ff @ (posedge clk_i or negedge rst_ni) begin :
clk_stretch
161     if (!rst_ni) begin
162         stretch_idle_cnt <= '0;
163     end else if (stretch_en && !scl_i) begin
164         // HOST-mode count of clock stretching
165         stretch_idle_cnt <= stretch_idle_cnt + 1'b1;
166     end else begin
167         stretch_idle_cnt <= '0;
168     end
169 end
170
171 // The TARGET can stretch the clock during any time that the
host drives SCL to 0.

```

```

172 // However, we (the HOST) cannot know it is being stretched
until we release SCL,
173 // usually trying to create the next clock pulse.
174 // There is a minimum 3-cycle round trip (1-cycle output
flop, 2-cycle input synchronizer),
175 // between releasing the clock and observing the effect of
releasing the clock on
176 // the inputs. However, this is really '1 + t_r + 2' as the
bus also needs to slew to '1'
177 // before can observe it. Even if the TARGET is not stretch-
ing the clock, we cannot
178 // confirm it until at-least this amount of time has
elapsed.
179 //
180 // 'stretch_predict_cnt_expired' becomes active once we have
observed (4 + t_r) cycles of
181 // delay, and if !scl_i at this point we know that the TAR-
GET is stretching the clock.
182 // > This implementation requires 'thigh >= 4' to guarantee
we don't miss stretching.
183 logic [30:0] stretch_cnt_threshold;
184 assign stretch_cnt_threshold = 31'd2 + 31'(t_r_i);
185
186 logic stretch_predict_cnt_expired;
187 always_ff @ (posedge clk_i or negedge rst_ni) begin
188     if (!rst_ni) begin
189         stretch_predict_cnt_expired <= 1'b0;
190     end else begin
191         if (stretch_idle_cnt == stretch_cnt_threshold) begin
192             stretch_predict_cnt_expired <= 1'b1;
193         end else if (!stretch_en) begin
194             stretch_predict_cnt_expired <= 1'b0;
195         end
196     end
197 end
198
199 // While the FSM is halted due to an unhandled 'nak' irq in
Host-Mode, this counter can
200 // be used to trigger a timeout which disables Host-Mode and
creates a STOP condition to
201 // end the transaction.
202 logic unhandled_nak_cnt_expired;
203 always_ff @ (posedge clk_i or negedge rst_ni) begin : un-
handled_nak_cnt_b
204     if (!rst_ni) begin
205         unhandled_nak_cnt <= '0;
206         unhandled_nak_cnt_expired <= 1'b0;
207     end else if (incr_nak_cnt) begin
208         // Increment the counter while the FSM is halted in
Idle.
209         unhandled_nak_cnt <= unhandled_nak_cnt + 1'b1;
210         if (unhandled_nak_cnt > host_nack_handler_timeout_i) be-
gin
211             unhandled_nak_cnt_expired <= 1'b1;
212         end
213     end else begin
214         unhandled_nak_cnt <= '0;
215         unhandled_nak_cnt_expired <= 1'b0;
216     end
217 end
218
219 assign event_unhandled_nak_timeout_o = un-
handled_nak_cnt_expired;
220
221 // Bit index implementation
222 always_ff @ (posedge clk_i or negedge rst_ni) begin :
bit_counter
223     if (!rst_ni) begin
224         bit_index <= 3'd7;
225     end else if (bit_clr) begin
226         bit_index <= 3'd7;
227     end else if (bit_decr) begin
228         bit_index <= bit_index - 1'b1;
229     end else begin
230         bit_index <= bit_index;
231     end
232 end
233
234 // Deserializer for a byte read from the bus
235 always_ff @ (posedge clk_i or negedge rst_ni) begin :
read_register
236     if (!rst_ni) begin
237         read_byte <= 8'h00;
238     end else if (read_byte_clr) begin
239         read_byte <= 8'h00;
240     end else if (shift_data_en) begin
241         read_byte[7:0] <= {read_byte[6:0], sda_i}; // MSB goes
in first
242     end
243 end
244
245 // Number of bytes to read
246 always_comb begin : byte_number
247     if (!fmt_flag_read_bytes_i) byte_num = 9'd0;
248     else if (fmt_byte_i == '0) byte_num = 9'd256;
249     else byte_num = 9'(fmt_byte_i);
250 end
251
252 // Byte index implementation

```



```

253 always_ff @(posedge clk_i or negedge rst_ni) begin :
byte_counter
254 if (!rst_ni) begin
255   byte_index <= '0;
256 end else if (byte_clr) begin
257   byte_index <= byte_num;
258 end else if (byte_decr) begin
259   byte_index <= byte_index - 1'b1;
260 end else begin
261   byte_index <= byte_index;
262 end
263 end
264
265 // SDA and SCL at the previous clock edge
266 always_ff @(posedge clk_i or negedge rst_ni) begin :
bus_prev
267 if (!rst_ni) begin
268   scl_i_q <= 1'b1;
269   sda_i_q <= 1'b1;
270 end else begin
271   scl_i_q <= scl_i;
272   sda_i_q <= sda_i;
273 end
274 end
275
276 // SCL going low early is just clock synchronization. SCL
switching so fast that the controller
277 // can't even sense its outputs is cause for a bus error.
278 assign event_arbitration_lost_o = event_sda_unstable_o ||
sda_interference_i ||
279                               ctrl_symbol_failed;
280
281 // Registers whether a transaction start has been observed.
282 // A transaction start does not include a "restart", but
rather
283 // the first start after enabling i2c, or a start observed
after a
284 // stop.
285 always_ff @(posedge clk_i or negedge rst_ni) begin
286   if (!rst_ni) begin
287     trans_started <= '0;
288   end else if (trans_started && (!host_enable_i ||
event_arbitration_lost_o)) begin
289     trans_started <= '0;
290   end else if (log_start) begin
291     trans_started <= 1'b1;
292   end else if (log_stop) begin
293     trans_started <= 1'b0;
294   end
295 end
296
297 always_ff @(posedge clk_i or negedge rst_ni) begin
298   if (!rst_ni) begin
299     pend_restart <= '0;
300   end else if (pend_restart && !host_enable_i) begin
301     pend_restart <= '0;
302   end else if (req_restart) begin
303     pend_restart <= 1'b1;
304   end else if (log_start) begin
305     pend_restart <= '0;
306   end
307 end
308
309 always_ff @(posedge clk_i or negedge rst_ni) begin
310   if (!rst_ni) begin
311     auto_stop_q <= 1'b0;
312   end else begin
313     auto_stop_q <= auto_stop_d;
314   end
315 end
316
317 // State definitions
318 typedef enum logic [4:0] {
319   Idle,
320   // Host function states
321   // Host function starts a transaction
322   SetupStart, HoldStart, ClockStart,
323   // Host function stops a transaction
324   SetupStop, HoldStop, ClockStop,
325   // Host function transmits a bit to the external target
326   ClockLow, ClockPulse, HoldBit,
327   // Host function receives an ack from the external target
328   ClockLowAck, ClockPulseAck, HoldDevAck,
329   // Host function reads a bit from the external target
330   ReadClockLow, ReadClockPulse, ReadHoldBit,
331   // Host function transmits an ack to the external target
332   HostClockLowAck, HostClockPulseAck, HostHoldBitAck
333 } state_e;
334
335 state_e state_q, state_d;
336
337
338 // Increment the NACK timeout count if the controller is
halted in Idle and
339 // the timeout hasn't yet occurred.
340
341 assign incr_nak_cnt = unhandled_unexp_nak_i && host_enable_i
&& (state_q == Idle) &&

```

```

342 host_nack_handler_timeout_en_i && !un-
handled_nak_timeout_i;
343
344 // Outputs for each state
345 always_comb begin : state_outputs
346   host_idle_o = 1'b1;
347   sda_d = 1'b1;
348   scl_d = 1'b1;
349   transmitting_o = 1'b0;
350   log_start = 1'b0;
351   log_stop = 1'b0;
352   fmt_fifo_rready_o = 1'b0;
353   rx_fifo_wvalid_o = 1'b0;
354   rx_fifo_wdata_o = RX_FIFO_WIDTH'(0);
355   event_nak_o = 1'b0;
356   event_scl_interference_o = 1'b0;
357   ctrl_symbol_failed = 1'b0;
358   event_sda_unstable_o = 1'b0;
359   event_cmd_complete_o = 1'b0;
360   stretch_en = 1'b0;
361   unique case (state_q)
362     // Idle: initial state, SDA is released (high), SCL is
released if the
363     // bus is idle. Otherwise, if no STOP condition has been
sent yet,
364     // continue pulling SCL low in host mode.
365     Idle : begin
366       sda_d = 1'b1;
367       if (trans_started) begin
368         host_idle_o = 1'b0;
369         scl_d = 1'b0;
370       end else begin
371         host_idle_o = 1'b1;
372         scl_d = 1'b1;
373       end
374     end
375     // HOST MODE //
376     // SetupStart: SDA and SCL are released
377     SetupStart : begin
378       host_idle_o = 1'b0;
379       sda_d = 1'b1;
380       scl_d = 1'b1;
381       transmitting_o = 1'b1;
382       // If this is a restart, SCL was last low, and a tar-
get could be stretching the clock.
383       stretch_en = trans_started;
384       if (trans_started && !scl_i && scl_i_q) begin
385         // If this is a repeated Start, an early clock pre-
vents issuing the symbol. If it's not
386         // a repeated start, the FSM will just go back to
Idle and wait for the bus to go free
387         // again.
388         ctrl_symbol_failed = 1'b1;
389       end else if (tcount_q == 16'd1) begin
390         log_start = 1'b1;
391         event_cmd_complete_o = pend_restart;
392       end
393     end
394     // HoldStart: SDA is pulled low, SCL is released
395     HoldStart : begin
396       host_idle_o = 1'b0;
397       sda_d = 1'b0;
398       scl_d = 1'b1;
399       transmitting_o = 1'b1;
400       if (scl_i_q && !scl_i) event_scl_interference_o =
1'b1;
401     end
402     // ClockStart: SCL is pulled low, SDA stays low
403     ClockStart : begin
404       host_idle_o = 1'b0;
405       sda_d = 1'b0;
406       scl_d = 1'b0;
407       transmitting_o = 1'b1;
408     end
409     ClockLow : begin
410       host_idle_o = 1'b0;
411       if (pend_restart) begin
412         sda_d = 1'b1;
413       end else begin
414         sda_d = fmt_byte_i[bit_index];
415       end
416       scl_d = 1'b0;
417       transmitting_o = 1'b1;
418     end
419     // ClockPulse: SCL is released, SDA keeps the indexed
bit value
420     ClockPulse : begin
421       host_idle_o = 1'b0;
422       sda_d = fmt_byte_i[bit_index];
423       scl_d = 1'b1;
424       transmitting_o = 1'b1;
425       stretch_en = 1'b1;
426       if (scl_i_q && !scl_i) event_scl_interference_o =
1'b1;
427     end
428     if (scl_i_q && scl_i && (sda_i_q != sda_i)) begin
429       // Unexpected Stop / Start

```

```

435     event_sda_unstable_o = '1'b1;
436 end
437 end
438 // HoldBit: SCL is pulled low
439 HoldBit : begin
440     host_idle_o = '1'b0;
441     sda_d = fmt_byte_i[bit_index];
442     scl_d = '1'b0;
443     transmitting_o = '1'b1;
444 end
445 // ClockLowAck: SCL pulled low, SDA is released
446 ClockLowAck : begin
447     host_idle_o = '1'b0;
448     sda_d = '1'b1;
449     scl_d = '1'b0;
450 end
451 // ClockPulseAck: SCL is released
452 ClockPulseAck : begin
453     host_idle_o = '1'b0;
454     sda_d = '1'b1;
455     scl_d = '1'b1;
456     if (!scli_q && scl_i && sda_i && !fmt_flag_nak_ok_i)
event_nak_o = '1'b1;
457     stretch_en = '1'b1;
458     if (scli_q && !scl_i) event_scl_interference_o =
'1'b1;
459     if (scli_q && scl_i && (sda_i_q != sda_i)) begin
460         // Unexpected Stop / Start
461         event_sda_unstable_o = '1'b1;
462     end
463 end
464 // HoldDevAck: SCL is pulled low
465 HoldDevAck : begin
466     host_idle_o = '1'b0;
467     sda_d = '1'b1;
468     scl_d = '1'b0;
469 end
470 // ReadClockLow: SCL is pulled low, SDA is released
471 ReadClockLow : begin
472     host_idle_o = '1'b0;
473     sda_d = '1'b1;
474     scl_d = '1'b0;
475 end
476 // ReadClockPulse: SCL is released, the indexed bit
value is read off SDA
477 ReadClockPulse : begin
478     host_idle_o = '1'b0;
479     scl_d = '1'b1;
480     stretch_en = '1'b1;
481     if (scli_q && !scl_i) event_scl_interference_o =
'1'b1;
482     if (scli_q && scl_i && (sda_i_q != sda_i)) begin
483         // Unexpected Stop / Start
484         event_sda_unstable_o = '1'b1;
485     end
486 end
487 // ReadHoldBit: SCL is pulled low
488 ReadHoldBit : begin
489     host_idle_o = '1'b0;
490     scl_d = '1'b0;
491     if (bit_index == '0 && tcount_q == 16'd1) begin
492         rx_fifo_wvalid_o = '1'b1; // assert that rx_fifo
has valid data
493         rx_fifo_wdata_o = read_byte; // transfer read data
to rx_fifo
494     end
495 end
496 // HostClockLowAck: SCL pulled low, SDA is conditional
497 HostClockLowAck : begin
498     host_idle_o = '1'b0;
499     scl_d = '1'b0;
500     transmitting_o = '1'b1;
501
502 // If it is the last byte of a read, send a NAK before
the stop.
503 // Otherwise send the ack.
504 if (fmt_flag_read_continue_i) sda_d = '1'b0;
505 else if (byte_index == 9'd1) sda_d = '1'b1;
506 else sda_d = '1'b0;
507 end
508 // HostClockPulseAck: SCL is released
509 HostClockPulseAck : begin
510     host_idle_o = '1'b0;
511     if (fmt_flag_read_continue_i) sda_d = '1'b0;
512     else if (byte_index == 9'd1) sda_d = '1'b1;
513     else sda_d = '1'b0;
514     scl_d = '1'b1;
515     transmitting_o = '1'b1;
516     stretch_en = '1'b1;
517     if (scli_q && !scl_i) event_scl_interference_o =
'1'b1;
518     if (scli_q && scl_i && (sda_i_q != sda_i)) begin
519         // Unexpected Stop / Start
520         event_sda_unstable_o = '1'b1;
521     end
522 end
523 // HostHoldBitAck: SCL is pulled low
524 HostHoldBitAck : begin
525     host_idle_o = '1'b0;

```

```

526     if (fmt_flag_read_continue_i) sda_d = '1'b0;
527     else if (byte_index == 9'd1) sda_d = '1'b1;
528     else sda_d = '1'b0;
529     scl_d = '1'b0;
530     transmitting_o = '1'b1;
531 end
532 // ClockStop: SCL is pulled low, SDA stays low
533 ClockStop : begin
534     host_idle_o = '1'b0;
535     sda_d = '1'b0;
536     scl_d = '1'b0;
537     transmitting_o = '1'b1;
538 end
539 // SetupStop: SDA is pulled low, SCL is released
540 SetupStop : begin
541     host_idle_o = '1'b0;
542     sda_d = '1'b0;
543     scl_d = '1'b1;
544     transmitting_o = '1'b1;
545     stretch_en = '1'b1;
546     if (!scli_q && scl_i_q) begin
547         // Failed to issue Stop before some other device
could pull SCL low.
548         ctrl_symbol_failed = '1'b1;
549     end
550 end
551 // HoldStop: SDA and SCL are released
552 HoldStop : begin
553     host_idle_o = '1'b0;
554     sda_d = '1'b1;
555     scl_d = '1'b1;
556     event_cmd_complete_o = '1'b1;
557     if (!sda_i && !scl_i) begin
558         // Failed to issue Stop before some other device
could pull SCL low.
559         ctrl_symbol_failed = '1'b1;
560     end else if (sda_i) begin
561         log_stop = '1'b1;
562     end
563 end
564 // Active: continue while keeping SCL low
565 Active : begin
566     host_idle_o = '1'b0;
567
568 // If this is a transaction start, do not drive scl
low
569 // since in the next state we will drive it high to
initiate
570 // the start bit.
571 // If this is a restart, continue driving the clock
low.
572     scl_d = fmt_flag_start_before_i && !trans_started;
573 end
574 // PopFmtFifo: populate fmt_fifo
575 PopFmtFifo : begin
576     host_idle_o = '1'b0;
577     if (fmt_flag_stop_after_i) scl_d = '1'b1;
578     else scl_d = '1'b0;
579     fmt_fifo_rready_o = '1'b1;
580 end
581
582 // default
583 default : begin
584     host_idle_o = '1'b1;
585     sda_d = '1'b1;
586     scl_d = '1'b1;
587     transmitting_o = '1'b0;
588     log_start = '1'b0;
589     log_stop = '1'b0;
590     event_scl_interference_o = '1'b0;
591     ctrl_symbol_failed = '1'b0;
592     fmt_fifo_rready_o = '1'b0;
593     rx_fifo_wvalid_o = '1'b0;
594     rx_fifo_wdata_o = RX_FIFO_WIDTH'(0);
595     event_nak_o = '1'b0;
596     event_sda_unstable_o = '1'b0;
597     event_cmd_complete_o = '1'b0;
598 end
599 endcase // unique case (state_q)
600 end
601
602 // Conditional state transition
603 always_comb begin : state_functions
604     state_d = state_q;
605     load_tcount = '1'b0;
606     tcount_sel = tNoDelay;
607     bit_decr = '1'b0;
608     bit_clr = '1'b0;
609     byte_decr = '1'b0;
610     byte_clr = '1'b0;
611     read_byte_clr = '1'b0;
612     shift_data_en = '1'b0;
613     req_restart = '1'b0;
614     auto_stop_d = auto_stop_q;
615
616     unique case (state_q)
617         // Idle: initial state, SDA is released (high), and SCL
is released if there is no ongoing
618         // transaction.
619         Idle : begin
620             if (host_enable_i) begin

```



```

621         if (unhandled_unexp_nak_i || unhandled_nak_timeout_i
|| halt_controller_i) begin
622             // If we are awaiting software to handle an unex-
pected NACK, halt the FSM here.
623             // The current transaction does not end, and SCL
remains in its current state.
624             // Software typically should handle an unexpected
NACK by either disabling the
625             // controller (causing host_enable_i to fall) or
by the following sequence:
626             // 1. Clear and/or populate the FMT FIFO.
627             // 2. Clear CONTROLLER_EVENTS.NACK
628             // Note that if the timeout feature is enabled,
the controller will be forced to
629             // issue a Stop if software takes too long to ad-
dress the NACK. A short timeout
630             // could also be used to automatically issue a
Stop whenever an unexpected NACK
631             // occurs.
632             // Note that we may also halt here on a bus
timeout or if arbitration was lost, so
633             // software may fix up the FIFOs before beginning
a new transaction.
634             if (trans_started && unhandled_nak_cnt_expired)
begin
635                 // If our timeout counter expires, generate a
STOP condition automatically.
636                 auto_stop_d = 1'b1;
637                 state_d = ClockStop;
638                 load_tcount = 1'b1;
639                 tcount_sel = tClockStop;
640             end
641             end else if (fmt_fifo_rvalid_i) begin
642                 if (trans_started || bus_free_i) begin
643                     state_d = Active;
644                 end
645             end
646             end else if (trans_started && !host_enable_i) begin
647                 auto_stop_d = 1'b1;
648                 state_d = ClockStop;
649                 load_tcount = 1'b1;
650                 tcount_sel = tClockStop;
651             end
652         end
653         //////////////////////////////////////////////////
654         // HOST MODE //
655         //////////////////////////////////////////////////
656         // SetupStart: SDA and SCL are released
657         SetupStart : begin
658             if (!trans_started && !scl_i) begin
659                 // This was the start of a transaction, but another
device beat us to access. Go back to
660                 // Idle, and wait for the next turn.
661                 state_d = Idle;
662             end else if (trans_started && !scl_i && !scl_i_q &&
stretch_predict_cnt_expired) begin
663                 // Saw stretching. Remain in this state and don't
count down until we see SCL high.
664                 state_d = SetupStart;
665                 load_tcount = 1'b1;
666                 // This double-counts the rise time, unfortunately.
667                 tcount_sel = tSetupStart;
668             end else if (trans_started && !scl_i && scl_i_q) begin
669                 // Failed to issue repeated Start. Effectively lost
arbitration.
670                 state_d = Idle;
671             end else if (tcount_q == 16'd1) begin
672                 state_d = HoldStart;
673                 load_tcount = 1'b1;
674                 tcount_sel = tHoldStart;
675             end
676         end
677         // HoldStart: SDA is pulled low, SCL is released
678         HoldStart : begin
679             if (tcount_q == 16'd1 || (!scl_i && scl_i_q)) begin
680                 state_d = ClockStart;
681                 load_tcount = 1'b1;
682                 tcount_sel = tClockStart;
683             end
684         end
685         // ClockStart: SCL is pulled low, SDA stays low
686         ClockStart : begin
687             if (tcount_q == 16'd1) begin
688                 state_d = ClockLow;
689                 load_tcount = 1'b1;
690                 tcount_sel = tClockLow;
691             end
692         end
693         // ClockLow: SCL stays low, shift indexed bit onto SDA
694         ClockLow : begin
695             if (tcount_q == 16'd1) begin
696                 load_tcount = 1'b1;
697                 if (pend_restart) begin
698                     state_d = SetupStart;
699                     tcount_sel = tSetupStart;
700                 end else begin
701                     state_d = ClockPulse;
702                     tcount_sel = tClockPulse;
703                 end
704             end

```

```

705         end
706     end
707 end
708 // ClockPulse: SCL is released, SDA keeps the indexed
bit value
709 ClockPulse : begin
710     if (!scl_i && !scl_i_q && stretch_predict_cnt_expired)
begin
711         // Saw stretching. Remain in this state and don't
count down until we see SCL high.
712         load_tcount = 1'b1;
713         tcount_sel = tClockHigh;
714     end else if (scl_i_q && scl_i && (sda_i_q != sda_i))
begin
715         // Unexpected Stop / Start
716         state_d = Idle;
717     end else if (tcount_q == 16'd1 || (!scl_i && scl_i_q))
begin
718         // Transition either when we finish counting our
high period or
719         // another controller pulls clock low.
720         state_d = HoldBit;
721         load_tcount = 1'b1;
722         tcount_sel = tHoldBit;
723     end
724 end
725 // HoldBit: SCL is pulled low
726 HoldBit : begin
727     if (tcount_q == 16'd1) begin
728         load_tcount = 1'b1;
729         tcount_sel = tClockLow;
730         if (bit_index == '0) begin
731             state_d = ClockLowAck;
732             bit_clr = 1'b1;
733         end else begin
734             state_d = ClockLow;
735             bit_decr = 1'b1;
736         end
737     end
738 end
739 // ClockLowAck: Target is allowed to drive ack back
740 // to host (dut)
741 ClockLowAck : begin
742     if (tcount_q == 16'd1) begin
743         state_d = ClockPulseAck;
744         load_tcount = 1'b1;
745         tcount_sel = tClockPulse;
746     end
747 end
748 // ClockPulseAck: SCL is released
749 ClockPulseAck : begin
750     if (!scl_i && !scl_i_q && stretch_predict_cnt_expired)
begin
751         // Saw stretching. Remain in this state and don't
count down until we see SCL high.
752         load_tcount = 1'b1;
753         tcount_sel = tClockHigh;
754     end else if (scl_i_q && scl_i && (sda_i_q != sda_i))
begin
755         // Unexpected Stop / Start
756         state_d = Idle;
757     end else begin
758         if (tcount_q == 16'd1 || (!scl_i && scl_i_q)) begin
759             state_d = HoldDevAck;
760             load_tcount = 1'b1;
761             tcount_sel = tHoldBit;
762         end
763     end
764 end
765 // HoldDevAck: SCL is pulled low
766 HoldDevAck : begin
767     if (tcount_q == 16'd1) begin
768         if (fmt_flag_stop_after_i) begin
769             state_d = ClockStop;
770             load_tcount = 1'b1;
771             tcount_sel = tClockStop;
772         end else begin
773             state_d = PopFmtFifo;
774             load_tcount = 1'b1;
775             tcount_sel = tNoDelay;
776         end
777     end
778 end
779 // ReadClockLow: SCL is pulled low, SDA is released
780 ReadClockLow : begin
781     if (tcount_q == 16'd1) begin
782         state_d = ReadClockPulse;
783         load_tcount = 1'b1;
784         tcount_sel = tClockPulse;
785     end
786 end
787 // ReadClockPulse: SCL is released, the indexed bit
value is read off SDA
788 ReadClockPulse : begin
789     if (!scl_i && !scl_i_q && stretch_predict_cnt_expired)
begin
790         // Saw stretching. Remain in this state and don't

```

```

count down until we see SCL high.
791     load_tcount = 1'b1;
792     tcount_sel = tClockHigh;
793     end else if (scl_i_q && scl_i && (sda_i_q != sda_i))
begin
794         // Unexpected Stop / Start
795         state_d = Idle;

796     end else if (tcount_q == 16'd1 || (!scl_i && scl_i_q))
begin
797         state_d = ReadHoldBit;
798         load_tcount = 1'b1;
799         tcount_sel = tHoldBit;

800     shift_data_en = 1'b1; // SDA is sampled on the final
clk_i cycle of the SCL pulse.
801     end
802 end
803 // ReadHoldBit: SCL is pulled low
804 ReadHoldBit : begin
805     if (tcount_q == 16'd1) begin
806         load_tcount = 1'b1;
807         tcount_sel = tClockLow;
808         if (bit_index == '0) begin
809             state_d = HostClockLowAck;
810             bit_clr = 1'b1;
811             read_byte_clr = 1'b1;
812         end else begin
813             state_d = ReadClockLow;
814             bit_decr = 1'b1;
815         end
816     end
817 end
818 // HostClockLowAck: SCL is pulled low, SDA is condition-
al based on
819 // byte position
820 HostClockLowAck : begin
821     if (tcount_q == 16'd1) begin
822         state_d = HostClockPulseAck;
823         load_tcount = 1'b1;
824         tcount_sel = tClockPulse;
825     end
826 end
827 // HostClockPulseAck: SCL is released
828 HostClockPulseAck : begin

829     if (!scl_i && !scl_i_q && stretch_predict_cnt_expired)
begin
830         // Saw stretching. Remain in this state and don't
count down until we see SCL high.
831         load_tcount = 1'b1;
832         tcount_sel = tClockHigh;
833     end else if (scl_i_q && scl_i && (sda_i_q != sda_i))
begin
834         // Unexpected Stop / Start
835         state_d = Idle;

836     end else if (tcount_q == 16'd1 || (!scl_i && scl_i_q))
begin
837         state_d = HostHoldBitAck;
838         load_tcount = 1'b1;
839         tcount_sel = tHoldBit;
840     end
841 end
842 // HostHoldBitAck: SCL is pulled low
843 HostHoldBitAck : begin
844     if (tcount_q == 16'd1) begin
845         if (byte_index == 9'd1) begin
846             if (fmt_flag_stop_after_i) begin
847                 state_d = ClockStop;
848                 load_tcount = 1'b1;
849                 tcount_sel = tClockStop;
850             end else begin
851                 state_d = PopFmtFifo;
852                 load_tcount = 1'b1;
853                 tcount_sel = tNoDelay;
854             end
855         end else begin
856             state_d = ReadClockLow;
857             load_tcount = 1'b1;
858             tcount_sel = tClockLow;
859             byte_decr = 1'b1;
860         end
861     end
862 end
863 // ClockStop: SCL is pulled low, SDA stays low
864 ClockStop : begin
865     if (tcount_q == 16'd1) begin
866         state_d = SetupStop;
867         load_tcount = 1'b1;
868         tcount_sel = tSetupStop;
869     end
870 end
871 // SetupStop: SDA is pulled low, SCL is released
872 SetupStop : begin

873     if (!scl_i && !scl_i_q && stretch_predict_cnt_expired)
begin
874         // Saw stretching. Remain in this state and don't
count down until we see SCL high.
875         load_tcount = 1'b1;

```

```

876         tcount_sel = tSetupStop;
877     end else if (!scl_i && scl_i_q) begin
878         // Failed to issue Stop before some other device
could pull SCL low.
879         state_d = Idle;
880     end else if (tcount_q == 16'd1) begin
881         state_d = HoldStop;
882     end
883 end
884 // HoldStop: SDA and SCL are released
885 HoldStop : begin
886     if (!sda_i && !scl_i) begin
887         // Failed to issue Stop before some other device
could pull SCL low.
888         state_d = Idle;
889         auto_stop_d = 1'b0;
890     end else if (sda_i) begin
891         auto_stop_d = 1'b0;
892     end if (auto_stop_q) begin
893         // If this Stop symbol was generated automatic-
ally, go back to Idle.
894         state_d = Idle;
895         load_tcount = 1'b1;
896         tcount_sel = tNoDelay;
897     end else begin
898         state_d = PopFmtFifo;
899         load_tcount = 1'b1;
900         tcount_sel = tNoDelay;
901     end
902 end
903 end
904 // Active: continue while keeping SCL low
905 Active : begin
906     if (fmt_flag_read_bytes_i) begin
907         byte_clr = 1'b1;
908         state_d = ReadClockLow;
909         load_tcount = 1'b1;
910         tcount_sel = tClockLow;
911     end else if (fmt_flag_start_before_i && !
trans_started) begin
912         state_d = SetupStart;
913         load_tcount = 1'b1;
914         tcount_sel = tSetupStart;
915     end else begin
916         state_d = ClockLow;
917         load_tcount = 1'b1;
918         req_restart = fmt_flag_start_before_i;
919         tcount_sel = tClockLow;
920     end
921 end
922 // PopFmtFifo: pop fmt_fifo item
923 PopFmtFifo : begin
924     if (!host_enable_i && trans_started) begin
925         auto_stop_d = 1'b1;
926         state_d = ClockStop;
927         load_tcount = 1'b1;
928         tcount_sel = tClockStop;
929     end else if (!host_enable_i || (fmt_fifo_depth_i ==
7'h1) ||
930         unhandled_unexp_nak_i || !trans_started)
begin
931         state_d = Idle;
932         load_tcount = 1'b1;
933         tcount_sel = tNoDelay;
934     end else begin
935         state_d = Active;
936         load_tcount = 1'b1;
937         tcount_sel = tNoDelay;
938     end
939 end
940 // default
941 default : begin
942     state_d = Idle;
943     load_tcount = 1'b0;
944     tcount_sel = tNoDelay;
945     bit_decr = 1'b0;
946     bit_clr = 1'b0;
947     byte_decr = 1'b0;
948     read_byte_clr = 1'b0;
949     shift_data_en = 1'b0;
950     auto_stop_d = 1'b0;
951 end
952 endcase // unique case (state_q)
953
954 if (trans_started && (sda_interference_i || ctrl_sym-
bol_failed)) begin
955     state_d = Idle;
956 end
957 end
958 // Synchronous state transition
959 always_ff @(posedge clk_i or negedge rst_ni) begin :
state_transition
960     if (!rst_ni) begin
961         state_q <= Idle;
962     end else begin
963         state_q <= state_d;
964     end
965 end
966 end

```

```

969
970 assign scl_o = scl_d;
971 assign sda_o = sda_d;
972
973 // Target stretched clock beyond timeout
974 assign event_stretch_timeout_o = stretch_en &&
timeout_enable_i &&
975 (stretch_idle_cnt >
31'(stretch_timeout_i));
976
977 // Make sure we never attempt to send a single cycle glitch
978 `ASSERT(SclOutputGlitch_A, $rose(scl_o) |-> ##1 scl_o)
979
980 // TODO: Handle the assertion below
981 // // I2C bus outputs
982 // always_ff @(posedge clk_i or negedge rst_ni) begin
983 //   if (!rst_ni) begin
984 //     scl_q <= 'b1;
985 //     sda_q <= 'b1;
986 //   end else begin
987 //     scl_q <= scl_d;
988 //     sda_q <= sda_d;
989 //   end
990 // end
991 //
992 // // Check that we don't change SCL and SDA in the same
clock cycle in host mode.
993 // `ASSERT(SclSdaChangeNotSimultaneous_A, !(host_enable_i &&
(scl_d != scl_q) && (sda_d != sda_q)))
994
995 endmodule

```

4. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_core.sv (891 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: I2C core module
6
7 `include "prim_assert.sv"
8
9 module i2c_core import i2c_pkg::*;
10 #(
11   parameter int unsigned InputDelayCycles = 0
12 ) (
13   input          clk_i,
14   input          rst_ni,
15   input  prim_ram_1p_pkg::ram_1p_cfg_req_t ram_cfg_i,
16   output prim_ram_1p_pkg::ram_1p_cfg_rsp_t ram_cfg_o,
17
18   input i2c_reg_pkg::i2c_reg2hw_t reg2hw,
19   output i2c_reg_pkg::i2c_hw2reg_t hw2reg,
20
21   input          scl_i,
22   output logic   scl_o,
23   input          sda_i,
24   output logic   sda_o,
25
26   output logic   lsio_trigger_o,
27
28   output logic   intr_fmt_threshold_o,
29   output logic   intr_rx_threshold_o,
30   output logic   intr_acq_threshold_o,
31   output logic   intr_rx_overflow_o,
32   output logic   intr_control-
ler_halt_o,
33   output logic   intr_scl_interfer-
ence_o,
34   output logic   intr_sda_interfer-
ence_o,
35   output logic   intr_stretch_timeout_o,
36   output logic   intr_sda_un-
stable_o,
37   output logic   intr_cmd_com-
plete_o,
38   output logic   intr_tx_stretch_o,
39   output logic   intr_tx_threshold_o,
40   output logic   intr_acq_stretch_o,
41   output logic   intr_unexp_stop_o,
42   output logic   intr_host_timeout_o
43 );
44
45 import i2c_reg_pkg::FifoDepth;
46 import i2c_reg_pkg::AcqFifoDepth;
47
48 // Number of bits required to represent the FIFO level/
depth.
49 localparam int unsigned FifoDepthW = $clog2(FifoDepth + 1);
50 localparam int unsigned AcqFifoDepthW = $clog2(AcqFifoDepth+1);
51
52 // Maximum number of bits required to represent the level/

```

```

depth of any FIFO.
53 localparam int unsigned MaxFifoDepthW = 12;
54
55 // Round-trip delay for outputs to appear on the inputs, not
including rise
56 // time. This is the input delay external to this IP, plus
the output flop,
57 // plus the 2-flop synchronizer on the input. The total
value here
58 // represents the minimum allowed high and low times for
SCL.
59 localparam int unsigned RoundTripCycles = InputDelayCycles + 2
+ 1;
60
61 logic [12:0] thigh;
62 logic [12:0] tlow;
63 logic [12:0] t_r;
64 logic [12:0] t_f;
65 logic [12:0] thd_sta;
66 logic [12:0] tsu_sta;
67 logic [12:0] tsu_sto;
68 logic [12:0] tsu_dat;
69 logic [12:0] thd_dat;
70 logic [12:0] t_buf;
71 logic [29:0] bus_active_timeout;
72 logic       stretch_timeout_enable;
73 logic       bus_timeout_enable;
74 logic [19:0] host_timeout;
75 logic [30:0] nack_timeout;
76 logic       nack_timeout_en;
77 logic [30:0] host_nack_handler_timeout;
78 logic       host_nack_handler_timeout_en;
79
80 logic scl_sync;
81 logic sda_sync;
82 logic scl_out_controller_fsm, sda_out_controller_fsm;
83 logic scl_out_target_fsm, sda_out_target_fsm;
84 logic scl_out_fsm;
85 logic sda_out_fsm;
86 logic controller_transmitting;
87 logic target_transmitting;
88
89 // bus_event_detect goes low after any drive change from
this IP, and it
90 // returns to high once enough time has passed for the out-
put change to
91 // reach the FSMs. This is used to qualify detection of un-
expected bus events,
92 // where some other I2C controller or target is driving SCL
or SDA
93 // simultaneously.
94 logic bus_event_detect;
95 logic [10:0] bus_event_detect_cnt;
96 logic sda_released_but_low;
97 logic controller_sda_interference;
98 logic target_arbitration_lost;
99
100 logic bus_free;
101 logic start_detect;
102 logic stop_detect;
103
104 logic event_rx_overflow;
105 logic status_controller_halt;
106 logic event_nak;
107 logic event_unhandled_nak_timeout;
108 logic event_controller_arbitration_lost;
109 logic event_scl_interference;
110 logic event_sda_interference;
111 logic event_bus_active_timeout;
112 logic event_stretch_timeout;
113 logic event_sda_unstable;
114 logic event_read_cmd_received;
115 logic event_cmd_complete;
116 logic event_controller_cmd_complete,
event_target_cmd_complete;
117 logic event_target_nack;
118 logic event_tx_arbitration_lost;
119 logic event_tx_bus_timeout;
120 logic event_tx_stretch;
121 logic event_acq_stretch;
122 logic event_unexp_stop;
123 logic event_host_timeout;
124
125 logic unhandled_unexp_nak;
126 logic unhandled_tx_stretch_event;
127 logic target_ack_ctrl_stretching;
128 logic target_ack_ctrl_sw_nack;
129
130 logic [15:0] scl_rx_val;
131 logic [15:0] sda_rx_val;
132
133 logic override;
134
135 logic       fmt_fifo_wvalid;
136 logic       fmt_fifo_wready;
137 logic [FMT_FIFO_WIDTH-1:0] fmt_fifo_wdata;
138 logic [FifoDepthW-1:0]   fmt_fifo_depth;
139 logic       fmt_fifo_rvalid;
140 logic       fmt_fifo_rready;

```

```

141 logic [FMT_FIFO_WIDTH-1:0] fmt_fifo_rdata;
142 logic [7:0] fmt_byte;
143 logic fmt_flag_start_before;
144 logic fmt_flag_stop_after;
145 logic fmt_flag_read_bytes;
146 logic fmt_flag_read_continue;
147 logic fmt_flag_nak_ok;
148
149 logic i2c_fifo_rxrst;
150 logic i2c_fifo_fmtrst;
151 logic [MaxFifoDepthW-1:0] i2c_fifo_rx_thresh;
152 logic [MaxFifoDepthW-1:0] i2c_fifo_fmt_thresh;
153
154 logic rx_fifo_wvalid;
155 logic rx_fifo_wready;
156 logic [RX_FIFO_WIDTH-1:0] rx_fifo_wdata;
157 logic [FifoDepthW-1:0] rx_fifo_depth;
158 logic rx_fifo_rvalid;
159 logic rx_fifo_rready;
160 logic [RX_FIFO_WIDTH-1:0] rx_fifo_rdata;
161
162 // FMT FIFO level below programmed threshold?
163 logic fmt_lt_threshold;
164 // Rx FIFO level above programmed threshold?
165 logic rx_gt_threshold;
166
167 logic tx_fifo_wvalid;
168 logic tx_fifo_wready;
169 logic [TX_FIFO_WIDTH-1:0] tx_fifo_wdata;
170 logic [FifoDepthW-1:0] tx_fifo_depth;
171 logic tx_fifo_rvalid;
172 logic tx_fifo_rready;
173 logic [TX_FIFO_WIDTH-1:0] tx_fifo_rdata;
174
175 logic acq_fifo_wvalid;
176 logic [ACQ_FIFO_WIDTH-1:0] acq_fifo_wdata;
177 logic [AcqFifoDepthW-1:0] acq_fifo_depth;
178 logic acq_fifo_full;
179 logic acq_fifo_rvalid;
180 logic acq_fifo_rready;
181 logic [ACQ_FIFO_WIDTH-1:0] acq_fifo_rdata;
182
183 logic i2c_fifo_txrst;
184 logic i2c_fifo_acqrst;
185 logic [MaxFifoDepthW-1:0] i2c_fifo_tx_thresh;
186 logic [MaxFifoDepthW-1:0] i2c_fifo_acq_thresh;
187
188 // Tx FIFO level below programmed threshold?
189 logic tx_lt_threshold;
190 // ACQ FIFO level above programmed threshold?
191 logic acq_gt_threshold;
192
193 logic host_idle;
194 logic target_idle;
195
196 logic host_enable;
197 logic target_enable;
198 logic line_loopback;
199 logic target_loopback;
200
201 logic [6:0] target_address0;
202 logic [6:0] target_mask0;
203 logic [6:0] target_address1;
204 logic [6:0] target_mask1;
205
206 // Unused parts of exposed bits
207 logic unused_rx_thr_qe;
208 logic unused_fmt_thr_qe;
209 logic unused_tx_thr_qe;
210 logic unused_acq_thr_qe;
211 logic [7:0] unused_rx_fifo_rdata_q;
212 logic [7:0] unused_acq_fifo_adata_q;
213 logic [ACQ_FIFO_WIDTH-9:0] unused_acq_fifo_signal_q;
214 logic unused_alert_test_qe;
215 logic unused_alert_test_q;
216
217 assign hw2reg.status.fmtfull.d = ~fmt_fifo_wready;
218 assign hw2reg.status.rxfull.d = ~rx_fifo_wready;
219 assign hw2reg.status.fmtempty.d = ~fmt_fifo_rvalid;
220 assign hw2reg.status.hostidle.d = host_idle;
221 assign hw2reg.status.targetidle.d = target_idle;
222 assign hw2reg.status.rxempty.d = ~rx_fifo_rvalid;
223 assign hw2reg.status.ack_ctrl_stretch.d = target_ack_ctrl_stretching;
224
225 assign hw2reg.rdata.d = rx_fifo_rdata;
226 assign hw2reg.host_fifo_status.fmtlvl.d = MaxFifoDepthW'(fmt_fifo_depth);
227 assign hw2reg.host_fifo_status.rxlvl.d = MaxFifoDepthW'(rx_fifo_depth);
228 assign hw2reg.val.scl_rx.d = scl_rx_val;
229 assign hw2reg.val.sda_rx.d = sda_rx_val;
230
231 assign hw2reg.status.txfull.d = ~tx_fifo_wready;
232 assign hw2reg.status.acqfull.d = acq_fifo_full;
233 assign hw2reg.status.txempty.d = ~tx_fifo_rvalid;
234 assign hw2reg.status.acqempty.d = ~acq_fifo_rready;
235 assign hw2reg.target_fifo_status.txlvl.d = MaxFifoDepthW'(tx_fifo_depth);
236 assign hw2reg.target_fifo_status.acqlvl.d = MaxFifoDepthW'(acq_fifo_depth);
237 assign hw2reg.acqdata.abyte.d = acq_fifo_rdata[7:0];
238 assign hw2reg.acqdata.signal.d = acq_fifo_rdata[ACQ_FIFO_WIDTH-1:8];
239
240 // Add one to the target NACK count if this target has sent a NACK and if
241 // counter has not saturated.
242 assign hw2reg.target_nack_count.de = event_target_nack && (reg2hw.target_nack_count.q < 8'hFF);
243 assign hw2reg.target_nack_count.d = reg2hw.target_nack_count.q + 1;
244
245 assign override = reg2hw.ovrd.txovrden;
246
247 assign scl_o = override ? reg2hw.ovrd.sclval : scl_out_fsm;
248 assign sda_o = override ? reg2hw.ovrd.sdaval : sda_out_fsm;
249
250 assign host_enable = reg2hw.ctrl.enablehost.q;
251 assign target_enable = reg2hw.ctrl.enabletarget.q;
252 assign line_loopback = reg2hw.ctrl.llpbk.q;
253
254 assign event_cmd_complete = event_controller_cmd_complete | event_target_cmd_complete;
255
256 // Target loopback simply plays back whatever is received from the external host
257 // back to it.
258 assign target_loopback = target_enable & line_loopback;
259
260 assign target_address0 = reg2hw.target_id.address0.q;
261 assign target_mask0 = reg2hw.target_id.mask0.q;
262 assign target_address1 = reg2hw.target_id.address1.q;
263 assign target_mask1 = reg2hw.target_id.mask1.q;
264
265 // Flop I2C bus outputs
266 always_ff @(posedge clk_i or negedge rst_ni) begin
267   if (!rst_ni) begin
268     scl_out_fsm <= 1'b1;
269     sda_out_fsm <= 1'b1;
270   end else begin
271     // Drive 0 if any FSM requests it.
272     scl_out_fsm <= scl_out_controller_fsm & scl_out_target_fsm;
273     sda_out_fsm <= sda_out_controller_fsm & sda_out_target_fsm;
274   end
275 end
276
277 // Sample scl_i and sda_i at system clock
278 always_ff @(posedge clk_i or negedge rst_ni) begin : rx_oversampling
279   if (!rst_ni) begin
280     scl_rx_val <= 16'h0;
281     sda_rx_val <= 16'h0;
282   end else begin
283     scl_rx_val <= {scl_rx_val[14:0], scl_sync};
284     sda_rx_val <= {sda_rx_val[14:0], sda_sync};
285   end
286 end
287
288 assign thigh = reg2hw.timing0.thigh.q;
289 assign tlow = reg2hw.timing0.tlow.q;
290 assign t_r = 13'(reg2hw.timing1.t_r.q);
291 assign t_f = 13'(reg2hw.timing1.t_f.q);
292 assign tsu_sta = reg2hw.timing2.tsu_sta.q;
293 assign thd_sta = reg2hw.timing2.thd_sta.q;
294 assign tsu_dat = 13'(reg2hw.timing3.tsu_dat.q);
295 assign thd_dat = reg2hw.timing3.thd_dat.q;
296 assign tsu_sto = reg2hw.timing4.tsu_sto.q;
297 assign t_buf = reg2hw.timing4.t_buf.q;
298 assign bus_active_timeout = reg2hw.timeout_ctrl.val.q;
299 assign stretch_timeout_enable = reg2hw.timeout_ctrl.en.q && (reg2hw.timeout_ctrl.mode.q == StretchTimeoutMode);
300
301 assign bus_timeout_enable = reg2hw.timeout_ctrl.en.q && (reg2hw.timeout_ctrl.mode.q == BusTimeoutMode);
302
303 assign host_timeout = reg2hw.host_timeout_ctrl.q;
304 assign nack_timeout = reg2hw.target_timeout_ctrl.val.q;
305 assign nack_timeout_en = reg2hw.target_timeout_ctrl.en.q;
306 assign host_nack_handler_timeout = reg2hw.host_nack_handler_timeout.val.q;
307 assign host_nack_handler_timeout_en = reg2hw.host_nack_handler_timeout.en.q;
308 assign target_ack_ctrl_sw_nack = reg2hw.target_ack_ctrl.nack.qe &

```

```

309 reg2hw.target_ack_ctrl.nack.q;
310
311 assign i2c_fifo_rxrst = reg2hw.fifo_ctrl.rxrst.q &
reg2hw.fifo_ctrl.rxrst.qe;
312 assign i2c_fifo_fmtrst = reg2hw.fifo_ctrl.fmtrst.q &
reg2hw.fifo_ctrl.fmtrst.qe;
313 assign i2c_fifo_rx_thresh =
reg2hw.host_fifo_config.rx_thresh.q;
314 assign i2c_fifo_fmt_thresh =
reg2hw.host_fifo_config.fmt_thresh.q;
315
316 assign i2c_fifo_txrst = reg2hw.fifo_ctrl.txrst.q &
reg2hw.fifo_ctrl.txrst.qe;
317 assign i2c_fifo_acqrst = reg2hw.fifo_ctrl.acqrst.q &
reg2hw.fifo_ctrl.acqrst.qe;
318 assign i2c_fifo_tx_thresh =
reg2hw.target_fifo_config.tx_thresh.q;
319 assign i2c_fifo_acq_thresh =
reg2hw.target_fifo_config.acq_thresh.q;
320
321 // FMT FIFO level below programmed threshold?
322 assign fmt_lt_threshold = (MaxFifoDepthW'(fmt_fifo_depth) <
i2c_fifo_fmt_thresh);
323 // Rx FIFO level above programmed threshold?
324 assign rx_gt_threshold = (MaxFifoDepthW'(rx_fifo_depth) >
i2c_fifo_rx_thresh);
325 // Tx FIFO level below programmed threshold?
326 assign tx_lt_threshold = (MaxFifoDepthW'(tx_fifo_depth) <
i2c_fifo_tx_thresh);
327 // ACQ FIFO level above programmed threshold?
328 assign acq_gt_threshold = (MaxFifoDepthW'(acq_fifo_depth) >
i2c_fifo_acq_thresh);
329
330 assign lsio_trigger_o = rx_gt_threshold;
331 assign event_rx_overflow = rx_fifo_wvalid & ~rx_fifo_wready;
332 assign event_acq_stretch = acq_fifo_full || tar-
get_ack_ctrl.stretching;
333
334 // The fifo write enable is controlled by fbyte, start,
stop, read, rcont,
335 // and nakok field qe bits.
336 // When all qe bits are asserted, fdata is injected into the
fifo.
337 assign fmt_fifo_wvalid = reg2hw.fdata.fbyte.qe &
reg2hw.fdata.start.qe &
338 reg2hw.fdata.stop.qe &
339 reg2hw.fdata.readb.qe &
340 reg2hw.fdata.rcont.qe &
341 reg2hw.fdata.nakok.qe;
342
343 assign fmt_fifo_wdata[7:0] = reg2hw.fdata.fbyte.q;
344 assign fmt_fifo_wdata[8] = reg2hw.fdata.start.q;
345 assign fmt_fifo_wdata[9] = reg2hw.fdata.stop.q;
346 assign fmt_fifo_wdata[10] = reg2hw.fdata.readb.q;
347 assign fmt_fifo_wdata[11] = reg2hw.fdata.rcont.q;
348 assign fmt_fifo_wdata[12] = reg2hw.fdata.nakok.q;
349
350 assign fmt_byte = fmt_fifo_rvalid ?
fmt_fifo_rdata[7:0] : '0;
351 assign fmt_flag_start_before = fmt_fifo_rvalid ?
fmt_fifo_rdata[8] : '0;
352 assign fmt_flag_stop_after = fmt_fifo_rvalid ?
fmt_fifo_rdata[9] : '0;
353 assign fmt_flag_read_bytes = fmt_fifo_rvalid ?
fmt_fifo_rdata[10] : '0;
354 assign fmt_flag_read_continue = fmt_fifo_rvalid ?
fmt_fifo_rdata[11] : '0;
355 assign fmt_flag_nak_ok = fmt_fifo_rvalid ?
fmt_fifo_rdata[12] : '0;
356
357 // Operating this HWIP as a controller-transmitter, the ad-
dressed Target device
358 // may NACK our bytes. In Byte-Formatted Programming Mode,
each FDATA format indicator
359 // can set the 'NAKOK' bit to ignore the Target's NACK and
proceed to the next item in
360 // the FMTFIFO. If 'NAKOK' is not set, the 'controller_halt'
interrupt is asserted, and the FSM
361 // halts until software intervenes. In addition, the 'con-
troller_events.nack' status bit is set.
362 // To acknowledge the 'NACK', software should clear the
status bit by writing 1 to it in the
363 // CONTROLLER_EVENTS register.
364 assign unhandled_unexp_nak =
reg2hw.controller_events.nack.q;
365
366 assign unused_rx_thr_qe =
reg2hw.host_fifo_config.rx_thresh.qe;
367 assign unused_fmt_thr_qe =
reg2hw.host_fifo_config.fmt_thresh.qe;
368 assign unused_tx_thr_qe =
reg2hw.target_fifo_config.tx_thresh.qe;
369 assign unused_acq_thr_qe =
reg2hw.target_fifo_config.acq_thresh.qe;
370 assign unused_rx_fifo_rdata_q = reg2hw.rdata.q;
371 assign unused_acq_fifo_adata_q = reg2hw.acqdata.abyte.q;
372 assign unused_acq_fifo_signal_q = reg2hw.acqdata.signal.q;
373 assign unused_alert_test_qe = reg2hw.alert_test.qe;
374 assign unused_alert_test_q = reg2hw.alert_test.q;

```

```

375 i2c_fifos u_fifos (
376 .clk_i,
377 .rst_ni,
378 .ram_cfg_i,
379 .ram_cfg_o,
380
381 .fmt_fifo_clr_i (i2c_fifo_fmtrst),
382 .fmt_fifo_depth_o (fmt_fifo_depth),
383 .fmt_fifo_wvalid_i (fmt_fifo_wvalid),
384 .fmt_fifo_wready_o (fmt_fifo_wready),
385 .fmt_fifo_wdata_i (fmt_fifo_wdata),
386 .fmt_fifo_rvalid_o (fmt_fifo_rvalid),
387 .fmt_fifo_rready_i (fmt_fifo_rready),
388 .fmt_fifo_rdata_o (fmt_fifo_rdata),
389
390
391 .rx_fifo_clr_i (i2c_fifo_rxrst),
392 .rx_fifo_depth_o (rx_fifo_depth),
393 .rx_fifo_wvalid_i (rx_fifo_wvalid),
394 .rx_fifo_wready_o (rx_fifo_wready),
395 .rx_fifo_wdata_i (rx_fifo_wdata),
396 .rx_fifo_rvalid_o (rx_fifo_rvalid),
397 .rx_fifo_rready_i (rx_fifo_rready),
398 .rx_fifo_rdata_o (rx_fifo_rdata),
399
400 .tx_fifo_clr_i (i2c_fifo_txrst),
401 .tx_fifo_depth_o (tx_fifo_depth),
402 .tx_fifo_wvalid_i (tx_fifo_wvalid),
403 .tx_fifo_wready_o (tx_fifo_wready),
404 .tx_fifo_wdata_i (tx_fifo_wdata),
405 .tx_fifo_rvalid_o (tx_fifo_rvalid),
406 .tx_fifo_rready_i (tx_fifo_rready),
407 .tx_fifo_rdata_o (tx_fifo_rdata),
408
409 .acq_fifo_clr_i (i2c_fifo_acqrst),
410 .acq_fifo_depth_o (acq_fifo_depth),
411 .acq_fifo_wvalid_i (acq_fifo_wvalid),
412 .acq_fifo_wready_o (),
413 .acq_fifo_wdata_i (acq_fifo_wdata),
414 .acq_fifo_rvalid_o (acq_fifo_rvalid),
415 .acq_fifo_rready_i (acq_fifo_rready),
416 .acq_fifo_rdata_o (acq_fifo_rdata)
417 );
418
419 assign rx_fifo_rready = reg2hw.rdata.re;
420
421 // Need to add a valid qualification to write only payload
bytes
422 logic valid_target_lb_wr;
423 i2c_acq_byte_id_e acq_type;
424 assign acq_type =
i2c_acq_byte_id_e'(acq_fifo_rdata[ACQ_FIFO_WIDTH-1:8]);
425
426 assign valid_target_lb_wr = target_enable & (acq_type == Ac-
qData);
427
428 // only write into tx fifo if it's payload
429 assign tx_fifo_wvalid = target_loopback ? acq_fifo_rvalid &
valid_target_lb_wr : reg2hw.txdata.qe;
430 assign tx_fifo_wdata = target_loopback ?
acq_fifo_rdata[7:0] : reg2hw.txdata.q;
431
432 // During line loopback, pop from acquisition fifo only when
there is space in
433 // the tx_fifo. We are also allowed to pop even if there is
no space if th acq entry
434 // is not data payload.
435 assign acq_fifo_rready = (reg2hw.acqdata.abyte.re &
reg2hw.acqdata.signal.re) |
436 (target_loopback & (tx_fifo_wready
| (acq_type != AcqData)));
437
438 // sync the incoming SCL and SDA signals
439 prim_flop_2sync #(
440 .Width(1),
441 .ResetValue(1'b1)
442 ) u_i2c_sync_scl (
443 .clk_i,
444 .rst_ni,
445 .d_i (scl_i),
446 .q_o (scl_sync)
447 );
448
449 prim_flop_2sync #(
450 .Width(1),
451 .ResetValue(1'b1)
452 ) u_i2c_sync_sda (
453 .clk_i,
454 .rst_ni,
455 .d_i (sda_i),
456 .q_o (sda_sync)
457 );
458
459 // Various bus collision events are detected while SCL is
high.
460 logic sda_fsm, sda_fsm_q;
461 logic scl_fsm, scl_fsm_q;
462 assign sda_fsm = sda_out_controller_fsm & sda_out_tar-
get_fsm;

```



```

463 assign scl_fsm = scl_out_controller_fsm & scl_out_tar-
get_fsm;
464
465 always_ff @(posedge clk_i or negedge rst_ni) begin
466   if (!rst_ni) begin
467     sda_fsm_q <= 1'b1;
468     scl_fsm_q <= 1'b1;
469   end else begin
470     sda_fsm_q <= sda_fsm;
471     scl_fsm_q <= scl_fsm;
472   end
473 end
474
475 always_ff @(posedge clk_i or negedge rst_ni) begin
476   if (!rst_ni) begin
477     bus_event_detect_cnt <= '1;
478   end else if ((scl_fsm != scl_fsm_q) || (sda_fsm !=
sda_fsm_q)) begin
479     // Wait for the round-trip time on SCL changes or on SDA
changes. The
480     // latter handles Start and Stop conditions, so changes
while SCL is high
481     // are allowed to propagate. The rise time is used here
because it
482     // should be the longer value. The (-1) term here is to
account for the
483     // delay in the counter.
484     // Note that there are limits to this method of detect-
ing arbitration.
485     // A separate, buggy device that drives clock low faster
than the counter
486     // can expire would not trigger loss of arbitration.
487     bus_event_detect_cnt <= reg2hw.timing1.t_r_q +
10'(RoundTripCycles - 1);
488   end else if (bus_event_detect_cnt != '0) begin
489     bus_event_detect_cnt <= bus_event_detect_cnt - 1'b1;
490   end
491 end
492 assign bus_event_detect = (bus_event_detect_cnt == '0);
493 assign sda_released_but_low = bus_event_detect && scl_sync
&& (sda_fsm_q != sda_sync);
494 // What about unexpected start / stop on the bits that are
read?
495 assign controller_sda_interference = controller_transmitting
&& sda_released_but_low;
496 assign target_arbitration_lost = target_transmitting &&
sda_released_but_low;
497
498 assign event_sda_interference = controller_sda_interference;
499
500 // The bus monitor detects starts, stops, and bus timeouts.
It also reports
501 // when the bus is free for the controller to transmit.
502 i2c_bus_monitor u_i2c_bus_monitor (
503   .clk_i,
504   .rst_ni,
505   .scl_i
506     (scl_sync),
507   .sda_i
508     (sda_sync),
509   .controller_enable_i
510     (host_enable),
511   .multi_controller_enable_i
512     (reg2hw.ctrl.multi_controller_monitor_en.q),
513   .target_enable_i
514     (target_enable),
515   .target_idle_i
516     (target_idle),
517   .thd_dat_i
518     (thd_dat),
519   .t_buf_i
520     (t_buf),
521   .bus_active_timeout_i
522     (bus_active_timeout),
523   .bus_active_timeout_en_i
524     (bus_timeout_enable),
525   .bus_inactive_timeout_i
526     (host_timeout),
527   .bus_free_o
528     (bus_free),
529   .start_detect_o
530     (start_detect),
531   .stop_detect_o
532     (stop_detect),
533   .event_bus_active_timeout_o
534     (event_bus_active_timeout),
535   .event_host_timeout_o
536     (event_host_timeout)
537 );
538
539 i2c_controller_fsm #(
540   .FifoDepth(FifoDepth)
541 ) u_i2c_controller_fsm (
542   .clk_i,
543   .rst_ni,
544   .scl_i
545     (scl_sync),
546   .scl_o
547     (scl_out_controller_fsm),
548   .sda_i
549     (sda_sync),
550   .sda_o
551     (sda_out_controller_fsm),
552   .bus_free_i
553     (bus_free),
554   .transmitting_o
555     (controller_transmitting),
556   .host_enable_i
557     (host_enable),
558   .halt_controller_i
559     (status_controller_halt),
560   .fmt_fifo_rvalid_i
561     (fmt_fifo_rvalid),
562   .fmt_fifo_depth_i
563     (fmt_fifo_depth),

```

```

564   .fmt_fifo_rready_o
565     (fmt_fifo_rready),
566   .fmt_byte_i
567     (fmt_byte),
568   .fmt_flag_start_before_i
569     (fmt_flag_start_before),
570   .fmt_flag_stop_after_i
571     (fmt_flag_stop_after),
572   .fmt_flag_read_bytes_i
573     (fmt_flag_read_bytes),
574   .fmt_flag_read_continue_i
575     (fmt_flag_read_continue),
576   .fmt_flag_nak_ok_i
577     (fmt_flag_nak_ok),
578   .unhandled_unexp_nak_i
579     (unhandled_unexp_nak),
580   .unhandled_nak_timeout_i
581     (reg2hw.controller_events.unhandled_nak_timeout.q),
582   .rx_fifo_wvalid_o
583     (rx_fifo_wvalid),
584   .rx_fifo_wdata_o
585     (rx_fifo_wdata),
586   .host_idle_o
587     (host_idle),
588   .thigh_i
589     (thigh),
590   .tlow_i
591     (tlow),
592   .t_r_i
593     (t_r),
594   .t_f_i
595     (t_f),
596   .thd_sta_i
597     (thd_sta),
598   .tsu_sta_i
599     (tsu_sta),
600   .tsu_sto_i
601     (tsu_sto),
602   .thd_dat_i
603     (thd_dat),
604   .sda_interference_i
605     (control-
ler_sda_interference),
606   .stretch_timeout_i
607     (bus_active_timeout),
608   .timeout_enable_i
609     (stretch_timeout_enable),
610   .host_nack_handler_timeout_i
611     (host_n-
ack_handler_timeout),
612   .host_nack_handler_timeout_en_i
613     (host_n-
ack_handler_timeout_en),
614   .event_nak_o
615     (event_nak),
616   .event_unhandled_nak_timeout_o
617     (event_unhandled_nak_timeout),
618   .event_arbitration_lost_o
619     (event_controller_arbitration_lost),
620   .event_scl_interference_o
621     (event_scl_interference),
622   .event_stretch_timeout_o
623     (event_stretch_timeout),
624   .event_sda_unstable_o
625     (event_sda_unstable),
626   .event_cmd_complete_o
627     (event_controller_cmd_complete)
628 );
629
630 i2c_target_fsm #(
631   .AcqFifoDepth(AcqFifoDepth)
632 ) u_i2c_target_fsm (
633   .clk_i,
634   .rst_ni,
635   .scl_i
636     (scl_sync),
637   .scl_o
638     (scl_out_target_fsm),
639   .sda_i
640     (sda_sync),
641   .sda_o
642     (sda_out_target_fsm),
643   .start_detect_i
644     (start_detect),
645   .stop_detect_i
646     (stop_detect),
647   .transmitting_o
648     (target_transmitting),
649   .target_enable_i
650     (target_enable),
651   .tx_fifo_rvalid_i
652     (tx_fifo_rvalid),
653   .tx_fifo_rready_o
654     (tx_fifo_rready),
655   .tx_fifo_rdata_i
656     (tx_fifo_rdata),
657   .acq_fifo_wvalid_o
658     (acq_fifo_wvalid),
659   .acq_fifo_wdata_o
660     (acq_fifo_wdata),
661   .acq_fifo_rdata_i
662     (acq_fifo_rdata),
663   .acq_fifo_full_o
664     (acq_fifo_full),
665   .acq_fifo_depth_i
666     (acq_fifo_depth),
667   .target_idle_o
668     (target_idle),
669   .t_r_i
670     (t_r),
671   .tsu_dat_i
672     (tsu_dat),
673   .thd_dat_i
674     (thd_dat),
675   .nak_timeout_i
676     (nak_timeout),
677   .nak_timeout_en_i
678     (nak_timeout_en),
679   .nak_addr_after_timeout_i
680     (reg2hw.ctrl.nak_addr_after_timeout.q),
681   .arbitration_lost_i
682     (target_arbitration_lost),
683   .bus_timeout_i
684     (event_bus_active_timeout),
685   .unhandled_tx_stretch_event_i
686     (un-
handled_tx_stretch_event),
687   .ack_ctrl_mode_i
688     (reg2hw.ctrl.ack_ctrl_en.q),
689   .auto_ack_cnt_o
690     (hw2reg.target_ack_ctrl.nbytes.d),
691   .auto_ack_load_i
692     (reg2hw.target_ack_ctrl.nbytes.qe),
693   .auto_ack_load_value_i
694     (reg2hw.target_ack_ctrl.nbytes.q),
695   .sw_nack_i
696     (target_ack_ctrl_sw_nack),
697   .ack_ctrl_stretching_o
698     (tar-
get_ack_ctrl_stretching),
699   .acq_fifo_next_data_o
700     (hw2reg.acq_fifo_next_data.d),
701   .target_address0_i
702     (target_address0),
703   .target_mask0_i
704     (target_mask0),
705   .target_address1_i
706     (target_address1),

```

```

630 .target_mask1_i          (target_mask1),
631 .event_target_nack_o      (event_target_nack),
632 .event_read_cmd_received_o (event_read_cmd_received),
633 .event_cmd_complete_o
(event_target_cmd_complete),
634 .event_tx_stretch_o      (event_tx_stretch),
635 .event_unexp_stop_o      (event_unexp_stop),
636 .event_tx_arbitration_lost_o
(event_tx_arbitration_lost),
637 .event_tx_bus_timeout_o  (event_tx_bus_timeout)
638 );
639
640 prim_intr_hw #(
641   .Width(1),
642   .IntrT("Status")
643 ) intr_hw_fmt_threshold (
644   .clk_i,
645   .rst_ni,
646   .event_intr_i          (fmt_lt_threshold),
647   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_fmt_threshold.q),
648   .reg2hw_intr_test_q_i
(reg2hw_intr_test_fmt_threshold.q),
649   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_fmt_threshold.qe),
650   .reg2hw_intr_state_q_i
(reg2hw_intr_state_fmt_threshold.q),
651   .hw2reg_intr_state_de_o
(hw2reg_intr_state_fmt_threshold.de),
652   .hw2reg_intr_state_d_o
(hw2reg_intr_state_fmt_threshold.d),
653   .intr_o                (intr_fmt_threshold_o)
654 );
655
656 prim_intr_hw #(
657   .Width(1),
658   .IntrT("Status")
659 ) intr_hw_rx_threshold (
660   .clk_i,
661   .rst_ni,
662   .event_intr_i          (rx_gt_threshold),
663   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_rx_threshold.q),
664   .reg2hw_intr_test_q_i
(reg2hw_intr_test_rx_threshold.q),
665   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_rx_threshold.qe),
666   .reg2hw_intr_state_q_i
(reg2hw_intr_state_rx_threshold.q),
667   .hw2reg_intr_state_de_o
(hw2reg_intr_state_rx_threshold.de),
668   .hw2reg_intr_state_d_o
(hw2reg_intr_state_rx_threshold.d),
669   .intr_o                (intr_rx_threshold_o)
670 );
671
672 prim_intr_hw #(
673   .Width(1),
674   .IntrT("Status")
675 ) intr_hw_acq_threshold (
676   .clk_i,
677   .rst_ni,
678   .event_intr_i          (acq_gt_threshold),
679   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_acq_threshold.q),
680   .reg2hw_intr_test_q_i
(reg2hw_intr_test_acq_threshold.q),
681   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_acq_threshold.qe),
682   .reg2hw_intr_state_q_i
(reg2hw_intr_state_acq_threshold.q),
683   .hw2reg_intr_state_de_o
(hw2reg_intr_state_acq_threshold.de),
684   .hw2reg_intr_state_d_o
(hw2reg_intr_state_acq_threshold.d),
685   .intr_o                (intr_acq_threshold_o)
686 );
687
688 prim_intr_hw #(Width(1)) intr_hw_rx_overflow (
689   .clk_i,
690   .rst_ni,
691   .event_intr_i          (event_rx_overflow),
692   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_rx_overflow.q),
693   .reg2hw_intr_test_q_i
(reg2hw_intr_test_rx_overflow.q),
694   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_rx_overflow.qe),
695   .reg2hw_intr_state_q_i
(reg2hw_intr_state_rx_overflow.q),
696   .hw2reg_intr_state_de_o
(hw2reg_intr_state_rx_overflow.de),
697   .hw2reg_intr_state_d_o
(hw2reg_intr_state_rx_overflow.d),
698   .intr_o                (intr_rx_overflow_o)
699 );
700
701 prim_intr_hw #(
702   .Width(1),
703   .IntrT("Status")
704 ) intr_hw_controller_halt (
705   .clk_i,
706   .rst_ni,
707   .event_intr_i          (status_controller_halt),
708   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_controller_halt.q),

```

```

709   .reg2hw_intr_test_q_i
(reg2hw_intr_test_controller_halt.q),
710   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_controller_halt.qe),
711   .reg2hw_intr_state_q_i
(reg2hw_intr_state_controller_halt.q),
712   .hw2reg_intr_state_de_o
(hw2reg_intr_state_controller_halt.de),
713   .hw2reg_intr_state_d_o
(hw2reg_intr_state_controller_halt.d),
714   .intr_o                (intr_controller_halt_o)
715 );
716
717 prim_intr_hw #(Width(1)) intr_hw_scl_interference (
718   .clk_i,
719   .rst_ni,
720   .event_intr_i          (event_scl_interference),
721   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_scl_interference.q),
722   .reg2hw_intr_test_q_i
(reg2hw_intr_test_scl_interference.q),
723   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_scl_interference.qe),
724   .reg2hw_intr_state_q_i
(reg2hw_intr_state_scl_interference.q),
725   .hw2reg_intr_state_de_o
(hw2reg_intr_state_scl_interference.de),
726   .hw2reg_intr_state_d_o
(hw2reg_intr_state_scl_interference.d),
727   .intr_o                (intr_scl_interference_o)
728 );
729
730 prim_intr_hw #(Width(1)) intr_hw_sda_interference (
731   .clk_i,
732   .rst_ni,
733   .event_intr_i          (event_sda_interference),
734   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_sda_interference.q),
735   .reg2hw_intr_test_q_i
(reg2hw_intr_test_sda_interference.q),
736   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_sda_interference.qe),
737   .reg2hw_intr_state_q_i
(reg2hw_intr_state_sda_interference.q),
738   .hw2reg_intr_state_de_o
(hw2reg_intr_state_sda_interference.de),
739   .hw2reg_intr_state_d_o
(hw2reg_intr_state_sda_interference.d),
740   .intr_o                (intr_sda_interference_o)
741 );
742
743 prim_intr_hw #(Width(1)) intr_hw_stretch_timeout (
744   .clk_i,
745   .rst_ni,
746   .event_intr_i          (event_stretch_timeout),
747   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_stretch_timeout.q),
748   .reg2hw_intr_test_q_i
(reg2hw_intr_test_stretch_timeout.q),
749   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_stretch_timeout.qe),
750   .reg2hw_intr_state_q_i
(reg2hw_intr_state_stretch_timeout.q),
751   .hw2reg_intr_state_de_o
(hw2reg_intr_state_stretch_timeout.de),
752   .hw2reg_intr_state_d_o
(hw2reg_intr_state_stretch_timeout.d),
753   .intr_o                (intr_stretch_timeout_o)
754 );
755
756 prim_intr_hw #(Width(1)) intr_hw_sda_unstable (
757   .clk_i,
758   .rst_ni,
759   .event_intr_i          (event_sda_unstable),
760   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_sda_unstable.q),
761   .reg2hw_intr_test_q_i
(reg2hw_intr_test_sda_unstable.q),
762   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_sda_unstable.qe),
763   .reg2hw_intr_state_q_i
(reg2hw_intr_state_sda_unstable.q),
764   .hw2reg_intr_state_de_o
(hw2reg_intr_state_sda_unstable.de),
765   .hw2reg_intr_state_d_o
(hw2reg_intr_state_sda_unstable.d),
766   .intr_o                (intr_sda_unstable_o)
767 );
768
769 prim_intr_hw #(Width(1)) intr_hw_cmd_complete (
770   .clk_i,
771   .rst_ni,
772   .event_intr_i          (event_cmd_complete),
773   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_cmd_complete.q),
774   .reg2hw_intr_test_q_i
(reg2hw_intr_test_cmd_complete.q),
775   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_cmd_complete.qe),
776   .reg2hw_intr_state_q_i
(reg2hw_intr_state_cmd_complete.q),
777   .hw2reg_intr_state_de_o
(hw2reg_intr_state_cmd_complete.de),

```

```

778     .hw2reg_intr_state_d_o
(hw2reg.intr_state.cmd_complete.d),
779     .intr_o             (intr_cmd_complete_o)
780 );
781
782 prim_intr_hw #(
783     .Width(1),
784     .IntrT("Status")
785 ) intr_hw_tx_stretch (
786     .clk_i,
787     .rst_ni,
788     .event_intr_i      (event_tx_stretch),
789     .reg2hw_intr_enable_q_i (reg2hw.intr_enable.tx_stretch.q),
790     .reg2hw_intr_test_q_i  (reg2hw.intr_test.tx_stretch.q),
791     .reg2hw_intr_test_qe_i (reg2hw.intr_test.tx_stretch.qe),
792     .reg2hw_intr_state_q_i (reg2hw.intr_state.tx_stretch.q),
793     .hw2reg_intr_state_de_o (hw2reg.intr_state.tx_stretch.de),
794     .hw2reg_intr_state_d_o (hw2reg.intr_state.tx_stretch.d),
795     .intr_o             (intr_tx_stretch_o)
796 );
797
798 prim_intr_hw #(
799     .Width(1),
800     .IntrT("Status")
801 ) intr_hw_tx_threshold (
802     .clk_i,
803     .rst_ni,
804     .event_intr_i      (tx_lt_threshold),
805     .reg2hw_intr_enable_q_i (reg2hw.intr_enable.tx_threshold.q),
806     .reg2hw_intr_test_q_i  (reg2hw.intr_test.tx_threshold.q),
807     .reg2hw_intr_test_qe_i (reg2hw.intr_test.tx_threshold.qe),
808     .reg2hw_intr_state_q_i (reg2hw.intr_state.tx_threshold.q),
809     .hw2reg_intr_state_de_o (hw2reg.intr_state.tx_threshold.de),
810     .hw2reg_intr_state_d_o (hw2reg.intr_state.tx_threshold.d),
811     .intr_o             (intr_tx_threshold_o)
812 );
813
814 prim_intr_hw #(
815     .Width(1),
816     .IntrT("Status")
817 ) intr_hw_acq_overflow (
818     .clk_i,
819     .rst_ni,
820     .event_intr_i      (event_acq_stretch),
821     .reg2hw_intr_enable_q_i (reg2hw.intr_enable.acq_stretch.q),
822     .reg2hw_intr_test_q_i  (reg2hw.intr_test.acq_stretch.q),
823     .reg2hw_intr_test_qe_i (reg2hw.intr_test.acq_stretch.qe),
824     .reg2hw_intr_state_q_i (reg2hw.intr_state.acq_stretch.q),
825     .hw2reg_intr_state_de_o (hw2reg.intr_state.acq_stretch.de),
826     .hw2reg_intr_state_d_o (hw2reg.intr_state.acq_stretch.d),
827     .intr_o             (intr_acq_stretch_o)
828 );
829
830 prim_intr_hw #(.Width(1)) intr_hw_unexp_stop (
831     .clk_i,
832     .rst_ni,
833     .event_intr_i      (event_unexp_stop),
834     .reg2hw_intr_enable_q_i (reg2hw.intr_enable.unexp_stop.q),
835     .reg2hw_intr_test_q_i  (reg2hw.intr_test.unexp_stop.q),
836     .reg2hw_intr_test_qe_i (reg2hw.intr_test.unexp_stop.qe),
837     .reg2hw_intr_state_q_i (reg2hw.intr_state.unexp_stop.q),
838     .hw2reg_intr_state_de_o (hw2reg.intr_state.unexp_stop.de),
839     .hw2reg_intr_state_d_o (hw2reg.intr_state.unexp_stop.d),
840     .intr_o             (intr_unexp_stop_o)
841 );
842
843 prim_intr_hw #(.Width(1)) intr_hw_host_timeout (
844     .clk_i,
845     .rst_ni,
846     .event_intr_i      (event_host_timeout),
847     .reg2hw_intr_enable_q_i (reg2hw.intr_enable.host_timeout.q),
848     .reg2hw_intr_test_q_i  (reg2hw.intr_test.host_timeout.q),
849     .reg2hw_intr_test_qe_i (reg2hw.intr_test.host_timeout.qe),
850     .reg2hw_intr_state_q_i (reg2hw.intr_state.host_timeout.q),
851     .hw2reg_intr_state_de_o (hw2reg.intr_state.host_timeout.de),
852     .hw2reg_intr_state_d_o (hw2reg.intr_state.host_timeout.d),
853     .intr_o             (intr_host_timeout_o)
854 );
855
856 assign hw2reg.controller_events.nack.d = 1'b1;
857 assign hw2reg.controller_events.nack.de = event_nak;
858 assign hw2reg.controller_events.unhandled_nack_timeout.d =
1'b1;
859 assign hw2reg.controller_events.unhandled_nack_timeout.de =
event_unhandled_nak_timeout;
860 assign hw2reg.controller_events.bus_timeout.d = 1'b1;
861 assign hw2reg.controller_events.bus_timeout.de =
event_bus_active_timeout & !host_idle;
862 assign hw2reg.controller_events.arbitration_lost.d = 1'b1;

```

```

863 assign hw2reg.controller_events.arbitration_lost.de =
event_controller_arbitration_lost;
864 assign status_controller_halt = | {
865     reg2hw.controller_events.nack.q,
866     reg2hw.controller_events.unhandled_nack_timeout.q,
867     reg2hw.controller_events.bus_timeout.q,
868     reg2hw.controller_events.arbitration_lost.q
869 };
870
871 assign hw2reg.target_events.tx_pending.d = 1'b1;
872 assign hw2reg.target_events.tx_pending.de =
event_read_cmd_received &&
873 reg2hw.ctrl.tx_stretch_ctrl_en.q;
874 assign hw2reg.target_events.bus_timeout.d = 1'b1;
875 assign hw2reg.target_events.bus_timeout.de =
event_tx_bus_timeout;
876 assign hw2reg.target_events.arbitration_lost.d = 1'b1;
877 assign hw2reg.target_events.arbitration_lost.de =
event_tx_arbitration_lost;
878 assign unhandled_tx_stretch_event = | {
879     reg2hw.target_events.tx_pending.q,
880     reg2hw.target_events.bus_timeout.q,
881     reg2hw.target_events.arbitration_lost.q
882 };
883
884 ///////////////////////////////////////////////////////////////////
885 // ASSERTIONS //
886 ///////////////////////////////////////////////////////////////////
887
888 `ASSERT_INIT(FifoDepthValid_A, FifoDepth > 0 && FifoDepthW
<= MaxFifoDepthW)
889 `ASSERT_INIT(AcqFifoDepthValid_A, AcqFifoDepth > 0 && AcqFi
foDepthW <= MaxFifoDepthW)
890
891 endmodule

```

5. lowRISC_opentitan/hw/ipl/i2c/rtl/i2c_fifo_sync_sram_adapter.sv (268 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // This adapter module allows using a single-port SRAM like a
FIFO. It has been written for I2C,
6 // but it could later be generalized into a prim.
7
8 `include "prim_assert.sv"
9
10 module i2c_fifo_sync_sram_adapter #(
11     // Width of each entry (in bits)
12     parameter int unsigned Width = 1,
13     // Total depth of the FIFO (in number of entries), including
the output buffer in this module.
14     // Must be at least 3.
15     parameter int unsigned Depth = 3,
16     // Address width of the SRAM
17     parameter int unsigned SramAw = 1,
18     // Base address of the FIFO in the SRAM
19     parameter logic [SramAw-1:0] SramBaseAddr = '0,
20     // Derived parameters; don't override!
21     localparam int unsigned DepthW = prim_util_pkg::vbits(Depth
+ 1)
22 ) (
23     input logic          clk_i,
24     input logic          rst_ni,
25     input logic          clr_i,
26
27     // FIFO interface
28     input logic          fifo_wvalid_i,
29     output logic         fifo_wready_o,
30     input logic [Width-1:0] fifo_wdata_i,
31     output logic         fifo_rvalid_o,
32     input logic          fifo_rready_i,
33     output logic [Width-1:0] fifo_rdata_o,
34     output logic [DepthW-1:0] fifo_depth_o,
35
36     // SRAM interface. When this module reads from the SRAM, it
expects the read data in the next
37     // cycle.
38     output logic         sram_req_o,
39     input logic          sram_gnt_i,
40     output logic         sram_write_o,
41     output logic [SramAw-1:0] sram_addr_o,
42     output logic [Width-1:0] sram_wdata_o,
43     output logic [Width-1:0] sram_wmask_o,
44     input logic [Width-1:0] sram_rdata_i,
45     input logic          sram_rvalid_i,
46
47     output logic         err_o
48 );
49
50 // Depth of the input buffer.
51 localparam int unsigned InpBufDepth = 2;
52

```



```

53 // Depth of the output buffer. To support a read in the
cycle after N consecutive cycles in which
54 // this module simultaneously got written and read, the
depth needs to be N + 1 (one entry extra
55 // to cover the single-cycle latency of SRAM reads). We
currently support this for 1 cycle.
56 localparam int unsigned OupBufDepth = 2;
57
58 // Depth of the SRAM-stored FIFO; simply the entries that
don't fit into the output buffer.
59 localparam int SramFifoDepth = Depth - OupBufDepth;
60
61 // While it would theoretically be possible that the output
buffer is as deep as the entire FIFO,
62 // the current implementation doesn't support this.
63 `ASSERT_INIT(MinimalSramFifoDepth_A, SramFifoDepth >= 1)
64
65 localparam int unsigned InpBufDepthW =
prim_util_pkg::vbits(InpBufDepth + 1);
66 localparam int unsigned OupBufDepthW =
prim_util_pkg::vbits(OupBufDepth + 1);
67 localparam int unsigned SramPtrW =
prim_util_pkg::vbits(SramFifoDepth);
68 localparam int unsigned SramDepthW =
prim_util_pkg::vbits(SramFifoDepth + 1);
69
70 `ASSERT_INIT(MinimalSramAw_A, SramAw >= SramPtrW)
71
72 localparam int unsigned SramAddrLeadingZeros = SramAw -
SramPtrW;
73
74 // Input buffer
75 logic                               inp_buf_wvalid, inp_buf_wready,
76                               inp_buf_rvalid, inp_buf_rready,
77                               inp_buf_err;
78 logic [Width-1:0]               inp_buf_rdata;
79 logic [InpBufDepthW-1:0] inp_buf_depth;
80 prim_fifo_sync #(
81   .Width(Width),
82   .Pass (1'b1),
83   .Depth(InpBufDepth)
84 ) u_inp_buf (
85   .clk_i,
86   .rst_ni,
87   .clr_i,
88   .wvalid_i(inp_buf_wvalid),
89   .wready_o(inp_buf_wready),
90   .wdata_i (fifo_wdata_i),
91   .rvalid_o(inp_buf_rvalid),
92   .rready_i(inp_buf_rready),
93   .rdata_o (inp_buf_rdata),
94   .full_o  (* unused *),
95   .depth_o (inp_buf_depth),
96   .err_o   (inp_buf_err)
97 );
98
99 // Output buffer
100 logic                               oup_buf_wvalid, oup_buf_wready,
oup_buf_full, oup_buf_almost_full,
101                               oup_buf_err;
102 logic [Width-1:0]               oup_buf_wdata;
103 logic [OupBufDepthW-1:0] oup_buf_depth;
104 prim_fifo_sync #(
105   .Width(Width),
106   .Pass (1'b1),
107   .Depth(OupBufDepth)
108 ) u_oup_buf (
109   .clk_i,
110   .rst_ni,
111   .clr_i,
112   .wvalid_i(oup_buf_wvalid),
113   .wready_o(oup_buf_wready),
114   .wdata_i (oup_buf_wdata),
115   .rvalid_o(fifo_rvalid_o),
116   .rready_i(fifo_rready_i),
117   .rdata_o (fifo_rdata_o),
118   .full_o  (oup_buf_full),
119   .depth_o (oup_buf_depth),
120   .err_o   (oup_buf_err)
121 );
122 assign inp_buf_wvalid = fifo_wvalid_i && fifo_wready_o;
123 assign oup_buf_almost_full = oup_buf_depth >= OupBuf-
DepthW'(OupBufDepth - 1);
124
125 // Signal whether we access the SRAM in this cycle
126 logic sram_access;
127 assign sram_access = sram_req_o && sram_gnt_i;
128
129 // SRAM read and write addresses
130 logic [SramAw-1:0] sram_wr_addr, sram_rd_addr;
131 logic [SramPtrW-1:0] sram_wr_ptr, sram_rd_ptr;
132 logic [SramDepthW-1:0] sram_depth;
133 logic sram_incr_wr_ptr, sram_incr_rd_ptr;
134 logic sram_full, sram_empty;
135 logic sram_ptrs_err;
136 prim_fifo_sync_cnt #(
137   .Depth (SramFifoDepth),
138   .Secure(1'b0)
139 ) u_sram_ptrs (
140   .clk_i,
141   .rst_ni,

```

```

142   .clr_i,
143   .incr_wptr_i(sram_incr_wr_ptr),
144   .incr_rptr_i(sram_incr_rd_ptr),
145   .wptr_o      (sram_wr_ptr),
146   .rptr_o      (sram_rd_ptr),
147   .full_o      (sram_full),
148   .empty_o     (sram_empty),
149   .depth_o     (sram_depth),
150   .err_o       (sram_ptrs_err)
151 );
152 assign sram_incr_wr_ptr = sram_access && sram_write_o;
153 assign sram_incr_rd_ptr = sram_access && !sram_write_o;
154 if (SramAddrLeadingZeros > 0) begin :
gen_zero_extend_sram_addrs
155   assign sram_wr_addr = {{SramAddrLeadingZeros{1'b0}},
sram_wr_ptr} + SramBaseAddr;
156   assign sram_rd_addr = {{SramAddrLeadingZeros{1'b0}},
sram_rd_ptr} + SramBaseAddr;
157 end else begin : gen_no_zero_extend_sram_addrs
158   assign sram_wr_addr = sram_wr_ptr + SramBaseAddr;
159   assign sram_rd_addr = sram_rd_ptr + SramBaseAddr;
160 end
161
162 // FF to remember whether we read from the SRAM in the pre-
vious clock cycle.
163 logic sram_read_in_prev_cyc_d, sram_read_in_prev_cyc_q;
164 assign sram_read_in_prev_cyc_d = clr_i ? 1'b0 : sram_in-
cr_rd_ptr;
165 always_ff @(posedge clk_i or negedge rst_ni) begin
166   if (!rst_ni) begin
167     sram_read_in_prev_cyc_q <= 1'b0;
168   end else begin
169     sram_read_in_prev_cyc_q <= sram_read_in_prev_cyc_d;
170   end
171 end
172
173 // Control logic for FIFO interface wready, output buffer
writes, and SRAM requests
174 logic state_err;
175 always_comb begin
176   inp_buf_rready = 1'b0;
177   oup_buf_wvalid = 1'b0;
178   oup_buf_wdata = inp_buf_rdata;
179   sram_req_o = 1'b0;
180   sram_write_o = 1'b0;
181   sram_addr_o = sram_wr_addr;
182   state_err = 1'b0;
183
184   // If the SRAM was read in the previous cycle, write the
read data to the output buffer.
185   if (sram_read_in_prev_cyc_q) begin
186     oup_buf_wvalid = 1'b1;
187     oup_buf_wdata = sram_rdata_i;
188     // The output buffer must be ready; otherwise we are in
an erroneous state.
189     state_err = !oup_buf_wready;
190   end
191
192   // If the SRAM is not empty, data from the input buffer
must flow via the SRAM.
193   if (!sram_empty) begin
194
195     // Prioritize refilling of the output buffer: if the
output buffer is not being filled to the
196     // last slot in the current cycle or it is being read in
the current cycle, read from the SRAM
197     // so the output buffer can be written in the next
cycle.
198     if (!(oup_buf_almost_full && oup_buf_wvalid) ||
oup_buf_full)
199       || (fifo_rvalid_o && fifo_rready_i)) begin
200       sram_req_o = 1'b1;
201       sram_write_o = 1'b0;
202       sram_addr_o = sram_rd_addr;
203
204       // If the output buffer is full (and not being read),
drain the input buffer into the SRAM.
205       end else begin
206         sram_req_o = !sram_full && inp_buf_rvalid;
207         sram_write_o = 1'b1;
208         sram_addr_o = sram_wr_addr;
209         inp_buf_rready = !sram_full && sram_gnt_i;
210       end
211
212       // If the SRAM is empty, data must flow from the input
buffer to the output buffer if the output
213       // buffer is ready and is not receiving data read from the
SRAM in the previous cycle
214       end else if (oup_buf_wready && !sram_read_in_prev_cyc_q)
begin
215         oup_buf_wvalid = inp_buf_rvalid;
216         oup_buf_wdata = inp_buf_rdata;
217         inp_buf_rready = oup_buf_wready;
218
219         // Otherwise the SRAM is empty but the output buffer can-
not take the data from the input buffer,
220         // so drain the input buffer into the SRAM.
221         end else begin
222           sram_req_o = !sram_full && inp_buf_rvalid;

```

```

223     sram_write_o = 'b1';
224     sram_addr_o = sram_wr_addr;
225     inp_buf_rready = !sram_full && sram_gnt_i;
226 end
227 end
228
229 // Error output is high if any of the internal errors is
high
230 assign err_o = {|inp_buf_err, oup_buf_err, sram_ptr_err,
state_err};
231
232 // The SRAM and the output buffer together form the entire
architectural capacity of the FIFO.
233 // Thus, when both are full, the FIFO is no longer ready to
receive writes even though the input
234 // buffer could store one additional entry. This ensures
that in the cycle after an entry has
235 // been read from the full FIFO, the next entry can be writ-
ten to the FIFO.
236 // (It may be possible that all input buffer slots except
one can be counted to the architectural
237 // capacity of the FIFO, but this is a relatively small op-
timization left for future work.)
238 assign fifo_wready_o = inp_buf_wready && !(sram_full &&
oup_buf_full);
239
240 // The current depth of the FIFO represented by this module
is the depth of all buffers plus the
241 // FIFO in the SRAM plus one if there's an entry in trans-
ition between SRAM and output buffer.
242 assign fifo_depth_o = DepthW'(inp_buf_depth) +
DepthW'(sram_depth) + DepthW'(oup_buf_depth) +
243     DepthW'(sram_read_in_prev_cyc_q);
244
245 // SRAM write data always comes from the input buffer.
assign sram_wdata_o = inp_buf_rdata;
246
247 assign sram_wmask_o = '1;
248
249 // `sram_rvalid_i` is only used for assertions.
logic unused_sram_rvalid;
250
251 assign unused_sram_rvalid = sram_rvalid_i;
252
253 // When we read from the SRAM in the previous cycle, we ex-
pect valid read data in this cycle.
254 `ASSERT(SramValidAfterRead_A, sram_read_in_prev_cyc_q |>
sram_rvalid_i)
255
256 // When we read from the SRAM in the previous cycle, the
output buffer must be ready to store data
257 // in this cycle.
258 `ASSERT(OupBufWreadyAfterSramRead_A, sram_read_in_prev_cyc_q
|> oup_buf_wready)
259
260 // We must never write the SRAM when it is full.
261 `ASSERT(NoSramWriteWhenFull_A, sram_full |> !sram_in-
cr_wr_ptr)
262
263 // We must never read from the SRAM when it is empty.
264 `ASSERT(NoSramReadWhenEmpty_A, sram_empty |> !sram_in-
cr_rd_ptr)
265
266 // We must never be in an erroneous state (impossible
without functional defects).
267 `ASSERT(NoErr_A, !err_o)
268 endmodule

```

6. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_fifo.sv (322 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 module i2c_fifo
8 import i2c_pkg::*;
9 import i2c_reg_pkg::FifoDepth;
10 import i2c_reg_pkg::AcqFifoDepth;
11 #(
12     localparam int unsigned FifoDepthW = $clog2(FifoDepth + 1),
13     localparam int unsigned AcqFifoDepthW = $clog2(AcqFifoDepth
+ 1)
14 ) (
15     input logic clk_i,
16     input logic rst_ni,
17     input prim_ram_1p_pkg::ram_1p_cfg_req_t ram_cfg_i,
18     output prim_ram_1p_pkg::ram_1p_cfg_rsp_t ram_cfg_o,
19
20     input logic fmt_fifo_clr_i,
21     output logic [FifoDepthW-1:0] fmt_fifo_depth_o,
22     // FMT FIFO: writes controlled by CSR
23     input logic fmt_fifo_wvalid_i,
24     output logic fmt_fifo_wready_o,
25     input logic [FMT_FIFO_WIDTH-1:0] fmt_fifo_wdata_i,
26     // FMT FIFO: reads controlled by FSM
27     output logic fmt_fifo_rvalid_o,
28     input logic fmt_fifo_rready_i,

```

```

29     output logic [FMT_FIFO_WIDTH-1:0] fmt_fifo_rdata_o,
30
31     input logic rx_fifo_clr_i,
32     output logic [FifoDepthW-1:0] rx_fifo_depth_o,
33     // RX FIFO: writes controlled by FSM
34     input logic rx_fifo_wvalid_i,
35     output logic rx_fifo_wready_o,
36     input logic [RX_FIFO_WIDTH-1:0] rx_fifo_wdata_i,
37     // RX FIFO: reads controlled by CSR
38     output logic rx_fifo_rvalid_o,
39     input logic rx_fifo_rready_i,
40     output logic [RX_FIFO_WIDTH-1:0] rx_fifo_rdata_o,
41
42     input logic tx_fifo_clr_i,
43     output logic [FifoDepthW-1:0] tx_fifo_depth_o,
44     // TX FIFO: writes controlled by CSR
45     input logic tx_fifo_wvalid_i,
46     output logic tx_fifo_wready_o,
47     input logic [TX_FIFO_WIDTH-1:0] tx_fifo_wdata_i,
48     // TX FIFO: reads controlled by FSM
49     output logic tx_fifo_rvalid_o,
50     input logic tx_fifo_rready_i,
51     output logic [TX_FIFO_WIDTH-1:0] tx_fifo_rdata_o,
52
53     input logic acq_fifo_clr_i,
54     output logic [AcqFifoDepthW-1:0] acq_fifo_depth_o,
55     // ACQ FIFO: writes controlled by FSM
56     input logic acq_fifo_wvalid_i,
57     output logic acq_fifo_wready_o,
58     input logic [ACQ_FIFO_WIDTH-1:0] acq_fifo_wdata_i,
59     // ACQ FIFO: reads controlled by CSR
60     output logic acq_fifo_rvalid_o,
61     input logic acq_fifo_rready_i,
62     output logic [ACQ_FIFO_WIDTH-1:0] acq_fifo_rdata_o
63 );
64
65 // RAM synthesis parameters
66 localparam int unsigned RamDepth = 464;
67
68 `ASSERT_INIT(RamDepthSuffices_A, RamDepth >= 3 * FifoDepth +
AcqFifoDepth)
69 localparam int unsigned RamAw =
prim_util_pkg::vbits(RamDepth);
70 localparam int unsigned RamWidth = 13;
71 `ASSERT_INIT(RamWidthSuffices_A,
72     RamWidth >= FMT_FIFO_WIDTH &&
73     RamWidth >= RX_FIFO_WIDTH &&
74     RamWidth >= TX_FIFO_WIDTH &&
75     RamWidth >= ACQ_FIFO_WIDTH)
76
77 // RAM address map
78 localparam logic [RamAw-1:0] FmtFifoRamBaseAddr = RamAw'(0);
79 localparam logic [RamAw-1:0] RxFifoRamBaseAddr = RamAw'(1 *
FifoDepth);
80 localparam logic [RamAw-1:0] TxFifoRamBaseAddr = RamAw'(2 *
FifoDepth);
81 localparam logic [RamAw-1:0] AcqFifoRamBaseAddr = RamAw'(3 *
FifoDepth);
82
83 // RAM read data, which is a common signal to all adapters
84 logic [RamWidth-1:0] ram_rdata;
85
86 // RAM adapter for FMT FIFO
87 logic [RamWidth-1:0] fmt_fifo_wdata, fmt_fifo_rdata,
fmt_ram_wdata;
88 logic fmt_ram_req, fmt_ram_gnt, fmt_ram_write, fmt_ram_rval-
id;
89 logic [RamAw-1:0] fmt_ram_addr;
90 i2c_fifo_sync_sram_adapter #(
91     .Width (RamWidth),
92     .Depth (FifoDepth),
93     .SramAw (RamAw),
94     .SramBaseAddr (FmtFifoRamBaseAddr)
95 ) u_fmt_fifo_sram_adapter (
96     .clk_i,
97     .rst_ni,
98     .clr_i (fmt_fifo_clr_i),
99     .fifo_wvalid_i (fmt_fifo_wvalid_i),
100     .fifo_wready_o (fmt_fifo_wready_o),
101     .fifo_wdata_i (fmt_fifo_wdata),
102     .fifo_rvalid_o (fmt_fifo_rvalid_o),
103     .fifo_rready_i (fmt_fifo_rready_i),
104     .fifo_rdata_o (fmt_fifo_rdata),
105     .fifo_depth_o (fmt_fifo_depth_o),
106     .sram_req_o (fmt_ram_req),
107     .sram_gnt_i (fmt_ram_gnt),
108     .sram_write_o (fmt_ram_write),
109     .sram_addr_o (fmt_ram_addr),
110     .sram_wdata_o (fmt_ram_wdata),
111     .sram_wmask_o (* unused *),
112     .sram_rdata_i (ram_rdata),
113     .sram_rvalid_i (fmt_ram_rvalid),
114     .err_o (* unused *)
115 );
116 assign fmt_fifo_wdata = fmt_fifo_wdata_i;
117 assign fmt_fifo_rdata_o =
fmt_fifo_rdata[FMT_FIFO_WIDTH-1:0];
118

```

```

118 // RAM adapter for RX FIFO
119 logic [RamWidth-1:0] rx_fifo_wdata, rx_fifo_rdata,
rx_fifo_wdata;
120 logic rx_ram_req, rx_ram_gnt, rx_ram_write, rx_ram_rvalid;
121 logic [RamAw-1:0] rx_ram_addr;
122 i2c_fifo_sync_sram_adapter #(
123     .Width      (RamWidth),
124     .Depth      (FifoDepth),
125     .SramAw      (RamAw),
126     .SramBaseAddr(RxFifoRamBaseAddr)
127 ) u_rx_fifo_sram_adapter (
128     .clk_i,
129     .rst_ni,
130     .clr_i      (rx_fifo_clr_i),
131     .fifo_wvalid_i(rx_fifo_wvalid_i),
132     .fifo_wready_o(rx_fifo_wready_o),
133     .fifo_wdata_i (rx_fifo_wdata),
134     .fifo_rvalid_o(rx_fifo_rvalid_o),
135     .fifo_rready_i(rx_fifo_rready_i),
136     .fifo_rdata_o (rx_fifo_rdata),
137     .fifo_depth_o (rx_fifo_depth_o),
138     .sram_req_o   (rx_ram_req),
139     .sram_gnt_i   (rx_ram_gnt),
140     .sram_write_o (rx_ram_write),
141     .sram_addr_o  (rx_ram_addr),
142     .sram_wdata_o (rx_fifo_wdata),
143     .sram_wmask_o (* unused */,
144     .sram_rdata_i (ram_rdata),
145     .sram_rvalid_i(rx_fifo_rvalid),
146     .err_o       (* unused */)
147 );
148 assign rx_fifo_wdata = {{{(RamWidth - RX_FIFO_WIDTH){1'b0}}},
rx_fifo_wdata_i};
149 assign rx_fifo_rdata_o = rx_fifo_rdata[RX_FIFO_WIDTH-1:0];
150 logic unused_rx_fifo_rdata;
151 assign unused_rx_fifo_rdata = ^rx_fifo_rdata[(RamWidth-1):
(RX_FIFO_WIDTH-1)];
152
153 // RAM adapter for TX FIFO
154 logic [RamWidth-1:0] tx_fifo_wdata, tx_fifo_rdata,
tx_fifo_wdata;
155 logic tx_ram_req, tx_ram_gnt, tx_ram_write, tx_ram_rvalid;
156 logic [RamAw-1:0] tx_ram_addr;
157 i2c_fifo_sync_sram_adapter #(
158     .Width      (RamWidth),
159     .Depth      (FifoDepth),
160     .SramAw      (RamAw),
161     .SramBaseAddr(TxFifoRamBaseAddr)
162 ) u_tx_fifo_sram_adapter (
163     .clk_i,
164     .rst_ni,
165     .clr_i      (tx_fifo_clr_i),
166     .fifo_wvalid_i(tx_fifo_wvalid_i),
167     .fifo_wready_o(tx_fifo_wready_o),
168     .fifo_wdata_i (tx_fifo_wdata),
169     .fifo_rvalid_o(tx_fifo_rvalid_o),
170     .fifo_rready_i(tx_fifo_rready_i),
171     .fifo_rdata_o (tx_fifo_rdata),
172     .fifo_depth_o (tx_fifo_depth_o),
173     .sram_req_o   (tx_ram_req),
174     .sram_gnt_i   (tx_ram_gnt),
175     .sram_write_o (tx_ram_write),
176     .sram_addr_o  (tx_ram_addr),
177     .sram_wdata_o (tx_fifo_wdata),
178     .sram_wmask_o (* unused */,
179     .sram_rdata_i (ram_rdata),
180     .sram_rvalid_i(tx_fifo_rvalid),
181     .err_o       (* unused */)
182 );
183 assign tx_fifo_wdata = {{{(RamWidth - TX_FIFO_WIDTH){1'b0}}},
tx_fifo_wdata_i};
184 assign tx_fifo_rdata_o = tx_fifo_rdata[TX_FIFO_WIDTH-1:0];
185 logic unused_tx_fifo_rdata;
186 assign unused_tx_fifo_rdata = ^tx_fifo_rdata[(RamWidth-1):
(TX_FIFO_WIDTH-1)];
187
188 // RAM adapter for ACQ FIFO
189 logic [RamWidth-1:0] acq_fifo_wdata, acq_fifo_rdata,
acq_fifo_wdata;
190 logic acq_ram_req, acq_ram_gnt, acq_ram_write, acq_ram_rval-
id;
191 logic [RamAw-1:0] acq_ram_addr;
192 i2c_fifo_sync_sram_adapter #(
193     .Width      (RamWidth),
194     .Depth      (AcqFifoDepth),
195     .SramAw      (RamAw),
196     .SramBaseAddr(AcqFifoRamBaseAddr)
197 ) u_acq_fifo_sram_adapter (
198     .clk_i,
199     .rst_ni,
200     .clr_i      (acq_fifo_clr_i),
201     .fifo_wvalid_i(acq_fifo_wvalid_i),
202     .fifo_wready_o(acq_fifo_wready_o),
203     .fifo_wdata_i (acq_fifo_wdata),
204     .fifo_rvalid_o(acq_fifo_rvalid_o),
205     .fifo_rready_i(acq_fifo_rready_i),
206     .fifo_rdata_o (acq_fifo_rdata),
207     .fifo_depth_o (acq_fifo_depth_o),
208     .sram_req_o   (acq_ram_req),
209     .sram_gnt_i   (acq_ram_gnt),
210     .sram_write_o (acq_ram_write),
211     .sram_addr_o  (acq_ram_addr),
212     .sram_wdata_o (acq_fifo_wdata),
213     .sram_wmask_o (* unused */,
214     .sram_rdata_i (ram_rdata),
215     .sram_rvalid_i(acq_fifo_rvalid),
216     .err_o       (* unused */)
217 );
218 assign acq_fifo_wdata = {{{(RamWidth - ACQ_FIFO_WIDTH)
{1'b0}}}, acq_fifo_wdata_i};
219 assign acq_fifo_rdata_o =
acq_fifo_rdata[ACQ_FIFO_WIDTH-1:0];
220 logic unused_acq_fifo_rdata;
221 assign unused_acq_fifo_rdata = ^acq_fifo_rdata[(RamWidth-1):
(ACQ_FIFO_WIDTH-1)];
222
223 // RAM signals
224 logic ram_req, ram_write, ram_rvalid;
225 logic [RamAw-1:0] ram_addr;
226 logic [RamWidth-1:0] ram_wdata;
227
228 // RAM arbiter
229 localparam int unsigned RamArbN = 4;
230 localparam int unsigned RamArbIdxW =
prim_util_pkg::vbits(RamArbN);
231 // Data for arbiter consists of write bit, address, and
write data
232 localparam int unsigned RamArbDw = 1 + RamAw + RamWidth;
233 logic [RamArbN-1:0] ram_arb_req, ram_arb_gnt;
234 logic [RamArbIdxW-1:0] ram_arb_idx;
235 logic [RamArbDw-1:0] ram_arb_inp_data[RamArbN];
236 logic [RamArbDw-1:0] ram_arb_oup_data;
237 prim_arbiter_tree #(
238     .N      (RamArbN),
239     .DW      (RamArbDw),
240     .EnDataPort(1)
241 ) u_ram_arbiter (
242     .clk_i,
243     .rst_ni,
244     .req_chk_i(1'b1),
245     .req_i      (ram_arb_req),
246     .data_i      (ram_arb_inp_data),
247     .gnt_o      (ram_arb_gnt),
248     .idx_o      (ram_arb_idx),
249     .valid_o     (ram_req),
250     .data_o      (ram_arb_oup_data),
251     .ready_i     (1'b1)
252 );
253 assign ram_arb_req[0] = fmt_ram_req;
254 assign ram_arb_inp_data[0] = {fmt_ram_write, fmt_ram_addr,
fmt_ram_wdata};
255 assign fmt_ram_gnt = ram_arb_gnt[0];
256 assign ram_arb_req[1] = rx_ram_req;
257 assign ram_arb_inp_data[1] = {rx_ram_write, rx_ram_addr,
rx_ram_wdata};
258 assign rx_ram_gnt = ram_arb_gnt[1];
259 assign ram_arb_req[2] = tx_ram_req;
260 assign ram_arb_inp_data[2] = {tx_ram_write, tx_ram_addr,
tx_ram_wdata};
261 assign tx_ram_gnt = ram_arb_gnt[2];
262 assign ram_arb_req[3] = acq_ram_req;
263 assign ram_arb_inp_data[3] = {acq_ram_write, acq_ram_addr,
acq_ram_wdata};
264 assign acq_ram_gnt = ram_arb_gnt[3];
265
266 // Demux `ram_rvalid` based on arbiter index one cycle
earlier.
267 logic [RamArbIdxW-1:0] ram_arb_idx_q;
268 always_ff @(posedge clk_i or negedge rst_ni) begin
269     if (!rst_ni) begin
270         ram_arb_idx_q <= '0;
271     end else begin
272         ram_arb_idx_q <= ram_arb_idx;
273     end
274 end
275 assign fmt_ram_rvalid = (ram_arb_idx_q == un-
signed'(RamArbIdxW'(0))) && ram_rvalid;
276 assign rx_ram_rvalid = (ram_arb_idx_q == un-
signed'(RamArbIdxW'(1))) && ram_rvalid;
277 assign tx_ram_rvalid = (ram_arb_idx_q == un-
signed'(RamArbIdxW'(2))) && ram_rvalid;
278 assign acq_ram_rvalid = (ram_arb_idx_q == un-
signed'(RamArbIdxW'(3))) && ram_rvalid;
279
280 // RAM instance
281 prim_ram_1p_adv #(
282     .Depth      (RamDepth),
283     .Width      (RamWidth),
284     .DataBitsPerMask (RamWidth),
285     .EnableECC      (1'b0),
286     .EnableParity    (1'b0),
287     .EnableInputPipeline(1'b0),
288     .EnableOutputPipeline(1'b0)
289 ) u_ram_1p (
290     .clk_i,
291     .rst_ni,
292     .req_i      (ram_req),
293     .write_i     (ram_write),
294     .addr_i      (ram_addr),
295     .wdata_i     (ram_wdata),

```

```

298 .wmask_i ('1),
299 .rdata_o (ram_rdata),
300 .rvalid_o (ram_rvalid),
301 .rerror_o (* unused *),
302 .cfg_i (ram_cfg_i),
303 .cfg_o (ram_cfg_o),
304 .alert_o (* unused *)
305 );
306 assign {ram_write, ram_addr, ram_wdata} = ram_arb_oup_data;
307
308 // For the FMT, ACQ, and TX FIFOs, we assume that a write
request stays stable until it has been
309 // granted. Put differently, we assume this module for
those FIFOs will never see a write that
310 // would be dropped (e.g., because the FIFO is full). This
does not hold for the RX FIFO, for

311 // which 'i2c_core' will register an overflow event if there
is a wvalid without a wready.
312 `ASSUME(FmtWriteStableBeforeHandshake_A,
313         fmt_fifo_wvalid_i && !fmt_fifo_wready_o
314         |> $stable(fmt_fifo_wvalid_i) &&
$stable(fmt_fifo_wdata_i))
315 `ASSUME(AcqWriteStableBeforeHandshake_A,
316         acq_fifo_wvalid_i && !acq_fifo_wready_o
317         |> $stable(acq_fifo_wvalid_i) &&
$stable(acq_fifo_wdata_i))
318 `ASSUME(TxWriteStableBeforeHandshake_A,
319         tx_fifo_wvalid_i && !tx_fifo_wready_o
320         |> $stable(tx_fifo_wvalid_i) &&
$stable(tx_fifo_wdata_i))
321
322 endmodule

```

7. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_pkg.sv (62 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // i2c package
6 //
7 package i2c_pkg;
8     parameter int unsigned I2C_ACQ_BYTE_ID_WIDTH = 3;
9
10 // TODO(#22028) encode this more efficiently in the ACQ
FIFO. Each entry in the
11 // ACQ FIFO does not need to contain both an 8 bit data
field and a 3 bit
12 // identifier. We should have the ACQ FIFO be 9 bits wide
where the MSB
13 // indicates whether it is a data byte or a control byte.
This way we can
14 // add more values to this enum without having to widen the
ACQ FIFO width.
15 typedef enum logic [I2C_ACQ_BYTE_ID_WIDTH-1:0] {
16     AcqData      = 3'b000,
17     AcqStart     = 3'b001,
18     AcqStop      = 3'b010,
19     AcqRestart   = 3'b011,
20     // AcqNack means one of two things:
21     // 1. We received a read request to our address, but had
to NACK the
22     // address because our ACQ FIFO is full.
23     // 2. We received too many bytes in a write request and
had to NACK a data
24     // byte. The NACK'ed data byte is still in the data field
for inspection.
25     AcqNack      = 3'b100,
26
27     // AcqNackStart means that we were addressed on this item,
but we timed
28     // out after stretching.
29     AcqNackStart = 3'b101,
30     // AcqNackStop means that we were addressed during the
transaction, but we
31     // timed out after stretching and received the Stop to end
the
32     // transaction.
33     AcqNackStop  = 3'b110
34 } i2c_acq_byte_id_e;
35
36 typedef enum logic {
37     StretchTimeoutMode = 1'b0,
38     BusTimeoutMode     = 1'b1
39 } i2c_timeout_mode_e;
40
41 // Width of each entry in the FMT FIFO with enough space for an
8-bit data
42 // byte and 5 flags.
43 parameter int unsigned FMT_FIFO_WIDTH = 8 + 5;
44
45 // Width of each entry in the RX and TX FIFO: just an 8-bit
data byte.
46 parameter int unsigned RX_FIFO_WIDTH = 8;
47 parameter int unsigned TX_FIFO_WIDTH = 8;

```

```

47
48 // Width of each entry in the ACQ FIFO with enough space for an
8-bit data
49 // byte and an identifier defined by i2c_acq_byte_id_e.
50 parameter int unsigned ACQ_FIFO_WIDTH =
I2C_ACQ_BYTE_ID_WIDTH + 8;
51
52 typedef enum bit {
53     WRITE = 1'b0,
54     READ  = 1'b1
55 } rw_e;
56
57 typedef enum bit {
58     ACK = 1'b0,
59     NACK = 1'b1
60 } acknack_e;
61
62 endpackage : i2c_pkg

```

8. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_reg_pkg.sv (829 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Package auto-generated by `reggen` containing data
structure
6
7 package i2c_reg_pkg;
8
9 // Param list
10 parameter int FifoDepth = 64;
11 parameter int AcqFifoDepth = 268;
12 parameter int NumAlerts = 1;
13
14 // Address widths within the block
15 parameter int BlockAw = 7;
16
17 // Number of registers for every interface
18 parameter int NumRegs = 32;
19
20 // Alert indices
21 typedef enum int {
22     AlertFatalFaultIdx = 0
23 } i2c_alert_idx_t;
24
25 ///////////////////////////////////////////////////
26 // Typedefs for registers //
27 ///////////////////////////////////////////////////
28
29 typedef struct packed {
30     struct packed {
31         logic q;
32     } host_timeout;
33     struct packed {
34         logic q;
35     } unexp_stop;
36     struct packed {
37         logic q;
38     } acq_stretch;
39     struct packed {
40         logic q;
41     } tx_threshold;
42     struct packed {
43         logic q;
44     } tx_stretch;
45     struct packed {
46         logic q;
47     } cmd_complete;
48     struct packed {
49         logic q;
50     } sda_unstable;
51     struct packed {
52         logic q;
53     } stretch_timeout;
54     struct packed {
55         logic q;
56     } sda_interference;
57     struct packed {
58         logic q;
59     } scl_interference;
60     struct packed {
61         logic q;
62     } controller_halt;
63     struct packed {
64         logic q;
65     } rx_overflow;
66     struct packed {
67         logic q;
68     } acq_threshold;
69     struct packed {
70         logic q;
71     } rx_threshold;
72     struct packed {
73         logic q;
74     } fmt_threshold;

```

```

75 } i2c_reg2hw_intr_state_reg_t;
76
77 typedef struct packed {
78     struct packed {
79         logic q;
80     } host_timeout;
81     struct packed {
82         logic q;
83     } unexp_stop;
84     struct packed {
85         logic q;
86     } acq_stretch;
87     struct packed {
88         logic q;
89     } tx_threshold;
90     struct packed {
91         logic q;
92     } tx_stretch;
93     struct packed {
94         logic q;
95     } cmd_complete;
96     struct packed {
97         logic q;
98     } sda_unstable;
99     struct packed {
100         logic q;
101     } stretch_timeout;
102     struct packed {
103         logic q;
104     } sda_interference;
105     struct packed {
106         logic q;
107     } scl_interference;
108     struct packed {
109         logic q;
110     } controller_halt;
111     struct packed {
112         logic q;
113     } rx_overflow;
114     struct packed {
115         logic q;
116     } acq_threshold;
117     struct packed {
118         logic q;
119     } rx_threshold;
120     struct packed {
121         logic q;
122     } fmt_threshold;
123 } i2c_reg2hw_intr_enable_reg_t;
124
125 typedef struct packed {
126     struct packed {
127         logic q;
128         logic qe;
129     } host_timeout;
130     struct packed {
131         logic q;
132         logic qe;
133     } unexp_stop;
134     struct packed {
135         logic q;
136         logic qe;
137     } acq_stretch;
138     struct packed {
139         logic q;
140         logic qe;
141     } tx_threshold;
142     struct packed {
143         logic q;
144         logic qe;
145     } tx_stretch;
146     struct packed {
147         logic q;
148         logic qe;
149     } cmd_complete;
150     struct packed {
151         logic q;
152         logic qe;
153     } sda_unstable;
154     struct packed {
155         logic q;
156         logic qe;
157     } stretch_timeout;
158     struct packed {
159         logic q;
160         logic qe;
161     } sda_interference;
162     struct packed {
163         logic q;
164         logic qe;
165     } scl_interference;
166     struct packed {
167         logic q;
168         logic qe;
169     } controller_halt;
170     struct packed {
171         logic q;
172         logic qe;
173     } rx_overflow;
174     struct packed {
175         logic q;
176         logic qe;
177     } acq_threshold;
178     struct packed {
179         logic q;
180         logic qe;
181     } rx_threshold;
182     struct packed {
183         logic q;
184         logic qe;
185     } fmt_threshold;
186 } i2c_reg2hw_intr_test_reg_t;
187
188 typedef struct packed {
189     logic q;
190     logic qe;
191 } i2c_reg2hw_alert_test_reg_t;
192
193 typedef struct packed {
194     struct packed {
195         logic q;
196     } tx_stretch_ctrl_en;
197     struct packed {
198         logic q;
199     } multi_controller_monitor_en;
200     struct packed {
201         logic q;
202     } ack_ctrl_en;
203     struct packed {
204         logic q;
205     } nack_addr_after_timeout;
206     struct packed {
207         logic q;
208     } llpbk;
209     struct packed {
210         logic q;
211     } enabletarget;
212     struct packed {
213         logic q;
214     } enablehost;
215 } i2c_reg2hw_ctrl_reg_t;
216
217 typedef struct packed {
218     logic [7:0] q;
219     logic re;
220 } i2c_reg2hw_rdata_reg_t;
221
222 typedef struct packed {
223     struct packed {
224         logic q;
225         logic qe;
226     } nakok;
227     struct packed {
228         logic q;
229         logic qe;
230     } rcont;
231     struct packed {
232         logic q;
233         logic qe;
234     } readb;
235     struct packed {
236         logic q;
237         logic qe;
238     } stop;
239     struct packed {
240         logic q;
241         logic qe;
242     } start;
243     struct packed {
244         logic [7:0] q;
245         logic qe;
246     } fbyte;
247 } i2c_reg2hw_fdata_reg_t;
248
249 typedef struct packed {
250     struct packed {
251         logic q;
252         logic qe;
253     } txrst;
254     struct packed {
255         logic q;
256         logic qe;
257     } acqrst;
258     struct packed {
259         logic q;
260         logic qe;
261     } fmtrst;
262     struct packed {
263         logic q;
264         logic qe;
265     } rxrst;
266 } i2c_reg2hw_fifo_ctrl_reg_t;
267
268 typedef struct packed {
269     struct packed {
270         logic [11:0] q;
271         logic qe;
272     } fmt_thresh;
273     struct packed {
274         logic [11:0] q;
275         logic qe;
276     } rx_thresh;

```

```

277 } i2c_reg2hw_host_fifo_config_reg_t;
278
279 typedef struct packed {
280     struct packed {
281         logic [11:0] q;
282         logic       qe;
283     } acq_thresh;
284     struct packed {
285         logic [11:0] q;
286         logic       qe;
287     } tx_thresh;
288 } i2c_reg2hw_target_fifo_config_reg_t;
289
290 typedef struct packed {
291     struct packed {
292         logic       q;
293     } sdaval;
294     struct packed {
295         logic       q;
296     } sclval;
297     struct packed {
298         logic       q;
299     } txovrden;
300 } i2c_reg2hw_ovrd_reg_t;
301
302 typedef struct packed {
303     struct packed {
304         logic [12:0] q;
305     } tlow;
306     struct packed {
307         logic [12:0] q;
308     } thigh;
309 } i2c_reg2hw_timing0_reg_t;
310
311 typedef struct packed {
312     struct packed {
313         logic [8:0] q;
314     } t_f;
315     struct packed {
316         logic [9:0] q;
317     } t_r;
318 } i2c_reg2hw_timing1_reg_t;
319
320 typedef struct packed {
321     struct packed {
322         logic [12:0] q;
323     } thd_sta;
324     struct packed {
325         logic [12:0] q;
326     } tsu_sta;
327 } i2c_reg2hw_timing2_reg_t;
328
329 typedef struct packed {
330     struct packed {
331         logic [12:0] q;
332     } thd_dat;
333     struct packed {
334         logic [8:0] q;
335     } tsu_dat;
336 } i2c_reg2hw_timing3_reg_t;
337
338 typedef struct packed {
339     struct packed {
340         logic [12:0] q;
341     } t_buf;
342     struct packed {
343         logic [12:0] q;
344     } tsu_sto;
345 } i2c_reg2hw_timing4_reg_t;
346
347 typedef struct packed {
348     struct packed {
349         logic       q;
350     } en;
351     struct packed {
352         logic       q;
353     } mode;
354     struct packed {
355         logic [29:0] q;
356     } val;
357 } i2c_reg2hw_timeout_ctrl_reg_t;
358
359 typedef struct packed {
360     struct packed {
361         logic [6:0] q;
362     } mask1;
363     struct packed {
364         logic [6:0] q;
365     } address1;
366     struct packed {
367         logic [6:0] q;
368     } mask0;
369     struct packed {
370         logic [6:0] q;
371     } address0;
372 } i2c_reg2hw_target_id_reg_t;
373
374 typedef struct packed {
375     struct packed {
376         logic [2:0] q;
377         logic       re;
378     } signal;
379     struct packed {
380         logic [7:0] q;
381         logic       re;
382     } abyte;
383 } i2c_reg2hw_acqdata_reg_t;
384
385 typedef struct packed {
386     logic [7:0] q;
387     logic       qe;
388 } i2c_reg2hw_txdata_reg_t;
389
390 typedef struct packed {
391     logic [19:0] q;
392 } i2c_reg2hw_host_timeout_ctrl_reg_t;
393
394 typedef struct packed {
395     struct packed {
396         logic       q;
397     } en;
398     struct packed {
399         logic [30:0] q;
400     } val;
401 } i2c_reg2hw_target_timeout_ctrl_reg_t;
402
403 typedef struct packed {
404     logic [7:0] q;
405 } i2c_reg2hw_target_nack_count_reg_t;
406
407 typedef struct packed {
408     struct packed {
409         logic       q;
410         logic       qe;
411     } nack;
412     struct packed {
413         logic [8:0] q;
414         logic       qe;
415     } nbytes;
416 } i2c_reg2hw_target_ack_ctrl_reg_t;
417
418 typedef struct packed {
419     struct packed {
420         logic       q;
421     } en;
422     struct packed {
423         logic [30:0] q;
424     } val;
425 } i2c_reg2hw_host_nack_handler_timeout_reg_t;
426
427 typedef struct packed {
428     struct packed {
429         logic       q;
430     } arbitration_lost;
431     struct packed {
432         logic       q;
433     } bus_timeout;
434     struct packed {
435         logic       q;
436     } unhandled_nack_timeout;
437     struct packed {
438         logic       q;
439     } nack;
440 } i2c_reg2hw_controller_events_reg_t;
441
442 typedef struct packed {
443     struct packed {
444         logic       q;
445     } arbitration_lost;
446     struct packed {
447         logic       q;
448     } bus_timeout;
449     struct packed {
450         logic       q;
451     } tx_pending;
452 } i2c_reg2hw_target_events_reg_t;
453
454 typedef struct packed {
455     struct packed {
456         logic       d;
457         logic       de;
458     } host_timeout;
459     struct packed {
460         logic       d;
461         logic       de;
462     } unexp_stop;
463     struct packed {
464         logic       d;
465         logic       de;
466     } acq_stretch;
467     struct packed {
468         logic       d;
469         logic       de;
470     } tx_threshold;
471     struct packed {
472         logic       d;
473         logic       de;
474     } tx_stretch;
475     struct packed {
476         logic       d;
477         logic       de;
478     } cmd_complete;

```



```

479 struct packed {
480     logic d;
481     logic de;
482 } sda_unstable;
483 struct packed {
484     logic d;
485     logic de;
486 } stretch_timeout;
487 struct packed {
488     logic d;
489     logic de;
490 } sda_interference;
491 struct packed {
492     logic d;
493     logic de;
494 } scl_interference;
495 struct packed {
496     logic d;
497     logic de;
498 } controller_halt;
499 struct packed {
500     logic d;
501     logic de;
502 } rx_overflow;
503 struct packed {
504     logic d;
505     logic de;
506 } acq_threshold;
507 struct packed {
508     logic d;
509     logic de;
510 } rx_threshold;
511 struct packed {
512     logic d;
513     logic de;
514 } fmt_threshold;
515 } i2c_hw2reg_intr_state_reg_t;
516
517 typedef struct packed {
518     struct packed {
519         logic d;
520     } ack_ctrl_stretch;
521     struct packed {
522         logic d;
523     } acqempty;
524     struct packed {
525         logic d;
526     } txempty;
527     struct packed {
528         logic d;
529     } acqfull;
530     struct packed {
531         logic d;
532     } txfull;
533     struct packed {
534         logic d;
535     } rxempty;
536     struct packed {
537         logic d;
538     } targetidle;
539     struct packed {
540         logic d;
541     } hostidle;
542     struct packed {
543         logic d;
544     } fmtempty;
545     struct packed {
546         logic d;
547     } rxfull;
548     struct packed {
549         logic d;
550     } fmtfull;
551 } i2c_hw2reg_status_reg_t;
552
553 typedef struct packed {
554     logic [7:0] d;
555 } i2c_hw2reg_rdata_reg_t;
556
557 typedef struct packed {
558     struct packed {
559         logic [11:0] d;
560     } rxlvl;
561     struct packed {
562         logic [11:0] d;
563     } fmtrlvl;
564 } i2c_hw2reg_host_fifo_status_reg_t;
565
566 typedef struct packed {
567     struct packed {
568         logic [11:0] d;
569     } acqlvl;
570     struct packed {
571         logic [11:0] d;
572     } txlvl;
573 } i2c_hw2reg_target_fifo_status_reg_t;
574
575 typedef struct packed {
576     struct packed {
577         logic [15:0] d;
578     } sda_rx;
579     struct packed {

```

```

580         logic [15:0] d;
581     } scl_rx;
582 } i2c_hw2reg_val_reg_t;
583
584 typedef struct packed {
585     struct packed {
586         logic [2:0] d;
587     } signal;
588     struct packed {
589         logic [7:0] d;
590     } abyte;
591 } i2c_hw2reg_acqdata_reg_t;
592
593 typedef struct packed {
594     logic [7:0] d;
595     logic de;
596 } i2c_hw2reg_target_nack_count_reg_t;
597
598 typedef struct packed {
599     struct packed {
600         logic [8:0] d;
601     } nbytes;
602 } i2c_hw2reg_target_ack_ctrl_reg_t;
603
604 typedef struct packed {
605     logic [7:0] d;
606 } i2c_hw2reg_acq_fifo_next_data_reg_t;
607
608 typedef struct packed {
609     struct packed {
610         logic d;
611         logic de;
612     } arbitration_lost;
613     struct packed {
614         logic d;
615         logic de;
616     } bus_timeout;
617     struct packed {
618         logic d;
619         logic de;
620     } unhandled_nack_timeout;
621     struct packed {
622         logic d;
623         logic de;
624     } nack;
625 } i2c_hw2reg_controller_events_reg_t;
626
627 typedef struct packed {
628     struct packed {
629         logic d;
630         logic de;
631     } arbitration_lost;
632     struct packed {
633         logic d;
634         logic de;
635     } bus_timeout;
636     struct packed {
637         logic d;
638         logic de;
639     } tx_pending;
640 } i2c_hw2reg_target_events_reg_t;
641
642 // Register -> HW type
643 typedef struct packed {
644     i2c_reg2hw_intr_state_reg_t intr_state; // [471:457]
645     i2c_reg2hw_intr_enable_reg_t intr_enable; // [456:442]
646     i2c_reg2hw_intr_test_reg_t intr_test; // [441:412]
647     i2c_reg2hw_alert_test_reg_t alert_test; // [411:410]
648     i2c_reg2hw_ctrl_reg_t ctrl; // [409:403]
649     i2c_reg2hw_rdata_reg_t rdata; // [402:394]
650     i2c_reg2hw_fdata_reg_t fdata; // [393:375]
651     i2c_reg2hw_fifo_ctrl_reg_t fifo_ctrl; // [374:367]
652     i2c_reg2hw_host_fifo_config_reg_t host_fifo_config; //
[366:341]
653     i2c_reg2hw_target_fifo_config_reg_t target_fifo_config; //
[340:315]
654     i2c_reg2hw_ovrd_reg_t ovrd; // [314:312]
655     i2c_reg2hw_timing0_reg_t timing0; // [311:286]
656     i2c_reg2hw_timing1_reg_t timing1; // [285:267]
657     i2c_reg2hw_timing2_reg_t timing2; // [266:241]
658     i2c_reg2hw_timing3_reg_t timing3; // [240:219]
659     i2c_reg2hw_timing4_reg_t timing4; // [218:193]
660     i2c_reg2hw_timeout_ctrl_reg_t timeout_ctrl; // [192:161]
661     i2c_reg2hw_target_id_reg_t target_id; // [160:133]
662     i2c_reg2hw_acqdata_reg_t acqdata; // [132:120]
663     i2c_reg2hw_txdata_reg_t txdata; // [119:111]
664     i2c_reg2hw_host_timeout_ctrl_reg_t host_timeout_ctrl; //
[110:91]
665     i2c_reg2hw_target_timeout_ctrl_reg_t tar-
get_timeout_ctrl; // [90:59]
666     i2c_reg2hw_target_nack_count_reg_t target_nack_count; //
[58:51]
667     i2c_reg2hw_target_ack_ctrl_reg_t target_ack_ctrl; //
[50:39]
668     i2c_reg2hw_host_nack_handler_timeout_reg_t host_n-
ack_handler_timeout; // [38:7]
669     i2c_reg2hw_controller_events_reg_t controller_events; //
[6:3]
670     i2c_reg2hw_target_events_reg_t target_events; // [2:0]
671 } i2c_reg2hw_t;

```

```

672 // HW -> register type
673 typedef struct packed {
674     i2c_hw2reg_intr_state_reg_t intr_state; // [179:150]
675     i2c_hw2reg_status_reg_t status; // [149:139]
676     i2c_hw2reg_rdata_reg_t rdata; // [138:131]
677     i2c_hw2reg_host_fifo_status_reg_t host_fifo_status; //
[130:107]
678
679     i2c_hw2reg_target_fifo_status_reg_t target_fifo_status; //
[106:83]
680     i2c_hw2reg_val_reg_t val; // [82:51]
681     i2c_hw2reg_acqdata_reg_t acqdata; // [50:40]
682     i2c_hw2reg_target_nack_count_reg_t target_nack_count; //
[39:31]
683     i2c_hw2reg_target_ack_ctrl_reg_t target_ack_ctrl; //
[30:22]
684
685     i2c_hw2reg_acq_fifo_next_data_reg_t acq_fifo_next_data; //
[21:14]
686     i2c_hw2reg_controller_events_reg_t controller_events; //
[13:6]
687     i2c_hw2reg_target_events_reg_t target_events; // [5:0]
688 } i2c_hw2reg_t;
689
690 // Register offsets
691 parameter logic [BlockAw-1:0] I2C_INTR_STATE_OFFSET = 7'h 0;
692 parameter logic [BlockAw-1:0] I2C_INTR_ENABLE_OFFSET = 7'h
4;
693 parameter logic [BlockAw-1:0] I2C_INTR_TEST_OFFSET = 7'h 8;
694 parameter logic [BlockAw-1:0] I2C_ALERT_TEST_OFFSET = 7'h c;
695 parameter logic [BlockAw-1:0] I2C_CTRL_OFFSET = 7'h 10;
696 parameter logic [BlockAw-1:0] I2C_STATUS_OFFSET = 7'h 14;
697 parameter logic [BlockAw-1:0] I2C_RDATA_OFFSET = 7'h 18;
698 parameter logic [BlockAw-1:0] I2C_FDATA_OFFSET = 7'h 1c;
699 parameter logic [BlockAw-1:0] I2C_FIFO_CTRL_OFFSET = 7'h 20;
700 parameter logic [BlockAw-1:0] I2C_HOST_FIFO_CONFIG_OFFSET =
7'h 24;
701 parameter logic [BlockAw-1:0] I2C_TARGET_FIFO_CONFIG_OFFSET
= 7'h 28;
702 parameter logic [BlockAw-1:0] I2C_HOST_FIFO_STATUS_OFFSET =
7'h 2c;
703 parameter logic [BlockAw-1:0] I2C_TARGET_FIFO_STATUS_OFFSET
= 7'h 30;
704 parameter logic [BlockAw-1:0] I2C_OVRD_OFFSET = 7'h 34;
705 parameter logic [BlockAw-1:0] I2C_VAL_OFFSET = 7'h 38;
706 parameter logic [BlockAw-1:0] I2C_TIMING0_OFFSET = 7'h 3c;
707 parameter logic [BlockAw-1:0] I2C_TIMING1_OFFSET = 7'h 40;
708 parameter logic [BlockAw-1:0] I2C_TIMING2_OFFSET = 7'h 44;
709 parameter logic [BlockAw-1:0] I2C_TIMING3_OFFSET = 7'h 48;
710 parameter logic [BlockAw-1:0] I2C_TIMING4_OFFSET = 7'h 4c;
711 parameter logic [BlockAw-1:0] I2C_TIMEOUT_CTRL_OFFSET = 7'h
50;
712 parameter logic [BlockAw-1:0] I2C_TARGET_ID_OFFSET = 7'h 54;
713 parameter logic [BlockAw-1:0] I2C_ACQDATA_OFFSET = 7'h 58;
714 parameter logic [BlockAw-1:0] I2C_TXDATA_OFFSET = 7'h 5c;
715 parameter logic [BlockAw-1:0] I2C_HOST_TIMEOUT_CTRL_OFFSET =
7'h 60;
716 parameter logic [BlockAw-1:0] I2C_TARGET_TIMEOUT_CTRL_OFFSET =
7'h 64;
717 parameter logic [BlockAw-1:0] I2C_TARGET_NACK_COUNT_OFFSET =
7'h 68;
718 parameter logic [BlockAw-1:0] I2C_TARGET_ACK_CTRL_OFFSET =
7'h 6c;
719 parameter logic [BlockAw-1:0] I2C_ACQ_FIFO_NEXT_DATA_OFFSET
= 7'h 70;
720 parameter logic [BlockAw-1:0] I2C_HOST_NACK_HANDLER_TIMEOUT_OFFSET
= 7'h 74;
721 parameter logic [BlockAw-1:0] I2C_CONTROLLER_EVENTS_OFFSET =
7'h 78;
722 parameter logic [BlockAw-1:0] I2C_TARGET_EVENTS_OFFSET = 7'h
7c;
723 // Reset values for hwext registers and their fields
724 parameter logic [14:0] I2C_INTR_TEST_RESVAL = 15'h 0;
725 parameter logic [0:0] I2C_INTR_TEST_FMT_THRESHOLD_RESVAL =
1'h 0;
726 parameter logic [0:0] I2C_INTR_TEST_RX_THRESHOLD_RESVAL =
1'h 0;
727 parameter logic [0:0] I2C_INTR_TEST_ACQ_THRESHOLD_RESVAL =
1'h 0;
728 parameter logic [0:0] I2C_INTR_TEST_RX_OVERFLOW_RESVAL = 1'h
0;
729 parameter logic [0:0] I2C_INTR_TEST_CONTROLLER_HALT_RESVAL =
1'h 0;
730 parameter logic [0:0] I2C_INTR_TEST_SCL_INTERFERENCE_RESVAL
= 1'h 0;
731 parameter logic [0:0] I2C_INTR_TEST_SDA_INTERFERENCE_RESVAL
= 1'h 0;
732 parameter logic [0:0] I2C_INTR_TEST_STRETCH_TIMEOUT_RESVAL =
1'h 0;
733 parameter logic [0:0] I2C_INTR_TEST_SDA_UNSTABLE_RESVAL =
1'h 0;

```

```

734 parameter logic [0:0] I2C_INTR_TEST_CMD_COMPLETE_RESVAL =
1'h 0;
735 parameter logic [0:0] I2C_INTR_TEST_TX_STRETCH_RESVAL = 1'h
0;
736 parameter logic [0:0] I2C_INTR_TEST_TX_THRESHOLD_RESVAL =
1'h 0;
737 parameter logic [0:0] I2C_INTR_TEST_ACQ_STRETCH_RESVAL = 1'h
0;
738 parameter logic [0:0] I2C_INTR_TEST_UNEXP_STOP_RESVAL = 1'h
0;
739 parameter logic [0:0] I2C_INTR_TEST_HOST_TIMEOUT_RESVAL =
1'h 0;
740 parameter logic [0:0] I2C_ALERT_TEST_RESVAL = 1'h 0;
741 parameter logic [0:0] I2C_ALERT_TEST_FATAL_FAULT_RESVAL =
1'h 0;
742 parameter logic [10:0] I2C_STATUS_RESVAL = 11'h 33c;
743 parameter logic [0:0] I2C_STATUS_FMTEMPTY_RESVAL = 1'h 1;
744 parameter logic [0:0] I2C_STATUS_HOSTIDLE_RESVAL = 1'h 1;
745 parameter logic [0:0] I2C_STATUS_TARGETIDLE_RESVAL = 1'h 1;
746 parameter logic [0:0] I2C_STATUS_RXEMPTY_RESVAL = 1'h 1;
747 parameter logic [0:0] I2C_STATUS_TXEMPTY_RESVAL = 1'h 1;
748 parameter logic [0:0] I2C_STATUS_ACQEMPTY_RESVAL = 1'h 1;
749 parameter logic [7:0] I2C_RDATA_RESVAL = 8'h 0;
750 parameter logic [27:0] I2C_HOST_FIFO_STATUS_RESVAL = 28'h 0;
751 parameter logic [27:0] I2C_TARGET_FIFO_STATUS_RESVAL = 28'h
0;
752 parameter logic [31:0] I2C_VAL_RESVAL = 32'h 0;
753 parameter logic [10:0] I2C_ACQDATA_RESVAL = 11'h 0;
754 parameter logic [31:0] I2C_TARGET_ACK_CTRL_RESVAL = 32'h 0;
755 parameter logic [7:0] I2C_ACQ_FIFO_NEXT_DATA_RESVAL = 8'h 0;
756
757 // Register index
758 typedef enum int {
759     I2C_INTR_STATE,
760     I2C_INTR_ENABLE,
761     I2C_INTR_TEST,
762     I2C_ALERT_TEST,
763     I2C_CTRL,
764     I2C_STATUS,
765     I2C_RDATA,
766     I2C_FDATA,
767     I2C_FIFO_CTRL,
768     I2C_HOST_FIFO_CONFIG,
769     I2C_TARGET_FIFO_CONFIG,
770     I2C_HOST_FIFO_STATUS,
771     I2C_TARGET_FIFO_STATUS,
772     I2C_OVRD,
773     I2C_VAL,
774     I2C_TIMING0,
775     I2C_TIMING1,
776     I2C_TIMING2,
777     I2C_TIMING3,
778     I2C_TIMING4,
779     I2C_TIMEOUT_CTRL,
780     I2C_TARGET_ID,
781     I2C_ACQDATA,
782     I2C_TXDATA,
783     I2C_HOST_TIMEOUT_CTRL,
784     I2C_TARGET_TIMEOUT_CTRL,
785     I2C_TARGET_NACK_COUNT,
786     I2C_TARGET_ACK_CTRL,
787     I2C_ACQ_FIFO_NEXT_DATA,
788     I2C_HOST_NACK_HANDLER_TIMEOUT,
789     I2C_CONTROLLER_EVENTS,
790     I2C_TARGET_EVENTS
791 } i2c_id_e;
792
793 // Register width information to check illegal writes
794 parameter logic [3:0] I2C_PERMIT [32] = '{
795     4'b 0011, // index[ 0] I2C_INTR_STATE
796     4'b 0011, // index[ 1] I2C_INTR_ENABLE
797     4'b 0011, // index[ 2] I2C_INTR_TEST
798     4'b 0001, // index[ 3] I2C_ALERT_TEST
799     4'b 0001, // index[ 4] I2C_CTRL
800     4'b 0011, // index[ 5] I2C_STATUS
801     4'b 0001, // index[ 6] I2C_RDATA
802     4'b 0011, // index[ 7] I2C_FDATA
803     4'b 0011, // index[ 8] I2C_FIFO_CTRL
804     4'b 1111, // index[ 9] I2C_HOST_FIFO_CONFIG
805     4'b 1111, // index[10] I2C_TARGET_FIFO_CONFIG
806     4'b 1111, // index[11] I2C_HOST_FIFO_STATUS
807     4'b 1111, // index[12] I2C_TARGET_FIFO_STATUS
808     4'b 0001, // index[13] I2C_OVRD
809     4'b 1111, // index[14] I2C_VAL
810     4'b 1111, // index[15] I2C_TIMING0
811     4'b 1111, // index[16] I2C_TIMING1
812     4'b 1111, // index[17] I2C_TIMING2
813     4'b 1111, // index[18] I2C_TIMING3
814     4'b 1111, // index[19] I2C_TIMING4
815     4'b 1111, // index[20] I2C_TIMEOUT_CTRL
816     4'b 1111, // index[21] I2C_TARGET_ID
817     4'b 0011, // index[22] I2C_ACQDATA
818     4'b 0001, // index[23] I2C_TXDATA
819     4'b 0111, // index[24] I2C_HOST_TIMEOUT_CTRL
820     4'b 1111, // index[25] I2C_TARGET_TIMEOUT_CTRL
821     4'b 0001, // index[26] I2C_TARGET_NACK_COUNT
822     4'b 1111, // index[27] I2C_TARGET_ACK_CTRL
823     4'b 0001, // index[28] I2C_ACQ_FIFO_NEXT_DATA
824     4'b 1111, // index[29] I2C_HOST_NACK_HANDLER_TIMEOUT
825     4'b 0001, // index[30] I2C_CONTROLLER_EVENTS

```



```
826     4'b 0001 // index[31] I2C_TARGET_EVENTS  
827 };
```

```
828  
829 endpackage
```

UART

What this is. An asynchronous serial transceiver with baud generation, oversampled receive, FIFOs and error flags.

Why it exists. Because it is the bring-up lifeline. When a chip does not boot, the UART is often the only visible output, so it must be the simplest and most robust block on the die.

Undergraduate review · EECS concepts. Asynchronous communication; oversampling and majority voting; tolerance to clock frequency error (typically $\pm 2\%$); framing errors.

Graduate · Practice note. Compute and publish the baud-rate error for the intended system clocks. If the ratio is not close to integral, accumulated error drops bits near the end of a frame. A table of achievable baud rates and their errors is a mark of a well-documented UART.

6 files • 3,190 lines

API — uart.sv

Table 971. lowRISC_opentitan/hw/ip/uart/rtl/uart.sv — module / interface / package (1개)

이름	종류
uart	module

Table 972. lowRISC_opentitan/hw/ip/uart/rtl/uart.sv — parameter (4개)

파라미터	형/폭	기본값
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
EnableRacl	bit	1'b0
RaclErrorRsp	bit	EnableRacl

Table 973. lowRISC_opentitan/hw/ip/uart/rtl/uart.sv — 포트 (21개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
lsio_trigger_o	output	logic
cio_rx_i	input	

포트	방향	형/폭
cio_tx_o	output	logic
cio_tx_en_o	output	logic
intr_tx_watermark_o	output	logic
intr_tx_empty_o	output	logic
intr_rx_watermark_o	output	logic
intr_tx_done_o	output	logic
intr_rx_overflow_o	output	logic
intr_rx_frame_err_o	output	logic
intr_rx_break_err_o	output	logic
intr_rx_timeout_o	output	logic
intr_rx_parity_err_o	output	logic

API — uart_core.sv

Table 974. lowRISC_opentitan/hw/ip/uart/rtl/uart_core.sv — module / interface / package (1개)

이름	종류
uart_core	module

Table 975. lowRISC_opentitan/hw/ip/uart/rtl/uart_core.sv — localparam (3개)

상수	형/폭	값
NcoWidth	int	\$bits(reg2hw.ctrl.nco.q)
TxFifoDepthW	int	\$clog2(TxFifoDepth)+1
RxFifoDepthW	int	\$clog2(RxFifoDepth)+1

Table 976. lowRISC_opentitan/hw/ip/uart/rtl/uart_core.sv — 포트 (16개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
reg2hw	input	uart_reg_pkg::uart_reg2hw_t
hw2reg	output	uart_reg_pkg::uart_hw2reg_t
rx	input	
tx	output	logic
lsio_trigger_o	output	logic
intr_tx_watermark_o	output	logic
intr_tx_empty_o	output	logic
intr_rx_watermark_o	output	logic
intr_tx_done_o	output	logic

포트	방향	형/폭
intr_rx_overflow_o	output	logic
intr_rx_frame_err_o	output	logic
intr_rx_break_err_o	output	logic
intr_rx_timeout_o	output	logic
intr_rx_parity_err_o	output	logic

API — uart_reg_pkg.sv

Table 977. lowRISC_opentitan/hw/ip/uart/rtl/uart_reg_pkg.sv — module / interface / package (1개)

이름	종류
uart_reg_pkg	package

Table 978. lowRISC_opentitan/hw/ip/uart/rtl/uart_reg_pkg.sv — parameter (38개)

파라미터	형/폭	기본값
RxFifoDepth	int	64
TxFifoDepth	int	32
NumAlerts	int	1
BlockAw	int	6
NumRegs	int	13
UART_INTR_STATE_OFFSET	logic [BlockAw-1:0]	6'h 0
UART_INTR_ENABLE_OFFSET	logic [BlockAw-1:0]	6'h 4
UART_INTR_TEST_OFFSET	logic [BlockAw-1:0]	6'h 8
UART_ALERT_TEST_OFFSET	logic [BlockAw-1:0]	6'h c
UART_CTRL_OFFSET	logic [BlockAw-1:0]	6'h 10
UART_STATUS_OFFSET	logic [BlockAw-1:0]	6'h 14
UART_RDATA_OFFSET	logic [BlockAw-1:0]	6'h 18
UART_WDATA_OFFSET	logic [BlockAw-1:0]	6'h 1c
UART_FIFO_CTRL_OFFSET	logic [BlockAw-1:0]	6'h 20
UART_FIFO_STATUS_OFFSET	logic [BlockAw-1:0]	6'h 24
UART_OVRD_OFFSET	logic [BlockAw-1:0]	6'h 28
UART_VAL_OFFSET	logic [BlockAw-1:0]	6'h 2c
UART_TIMEOUT_CTRL_OFFSET	logic [BlockAw-1:0]	6'h 30
UART_INTR_TEST_RESVAL	logic [8:0]	9'h 0
UART_INTR_TEST_TX_WATERMARK_RESVAL	logic [0:0]	1'h 0
UART_INTR_TEST_RX_WATERMARK_RESVAL	logic [0:0]	1'h 0
UART_INTR_TEST_TX_DONE_RESVAL	logic [0:0]	1'h 0
UART_INTR_TEST_RX_OVERFLOW_RESVAL	logic [0:0]	1'h 0

파라미터	형/폭	기본값
UART_INTR_TEST_RX_FRAME_ERR_RESVAL	logic [0:0]	1'h 0
UART_INTR_TEST_RX_BREAK_ERR_RESVAL	logic [0:0]	1'h 0
UART_INTR_TEST_RX_TIMEOUT_RESVAL	logic [0:0]	1'h 0
UART_INTR_TEST_RX_PARITY_ERR_RESVAL	logic [0:0]	1'h 0
UART_INTR_TEST_TX_EMPTY_RESVAL	logic [0:0]	1'h 0
UART_ALERT_TEST_RESVAL	logic [0:0]	1'h 0
UART_ALERT_TEST_FATAL_FAULT_RESVAL	logic [0:0]	1'h 0
UART_STATUS_RESVAL	logic [5:0]	6'h 3c
UART_STATUS_TXEMPTY_RESVAL	logic [0:0]	1'h 1
UART_STATUS_TXIDLE_RESVAL	logic [0:0]	1'h 1
UART_STATUS_RXIDLE_RESVAL	logic [0:0]	1'h 1
UART_STATUS_RXEMPTY_RESVAL	logic [0:0]	1'h 1
UART_RDATA_RESVAL	logic [7:0]	8'h 0
UART_FIFO_STATUS_RESVAL	logic [23:0]	24'h 0
UART_VAL_RESVAL	logic [15:0]	16'h 0

API — uart_reg_top.sv

Table 979. lowRISC_opentitan/hw/ip/uart/rtl/uart_reg_top.sv — module / interface / package (1개)

이름	종류
uart_reg_top	module

Table 980. lowRISC_opentitan/hw/ip/uart/rtl/uart_reg_top.sv — parameter (2개)

파라미터	형/폭	기본값
EnableRacL	bit	1'b0
RacLErrorRsp	bit	1'b1

Table 981. lowRISC_opentitan/hw/ip/uart/rtl/uart_reg_top.sv — localparam (3개)

상수	형/폭	값
AW	int	6
DW	int	32
DBW	int	DW/8

Table 982. lowRISC_opentitan/hw/ip/uart/rtl/uart_reg_top.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t

포트	방향	형/폭
tl_o	output	tlul_pkg::tl_d2h_t
reg2hw	output	uart_reg_pkg::uart_reg2hw_t
hw2reg	input	uart_reg_pkg::uart_hw2reg_t
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
intg_err_o	output	logic

API — uart_rx.sv

Table 983. lowRISC_opentitan/hw/ip/uart/rtl/uart_rx.sv — module / interface / package (1개)

이름	종류
uart_rx	module

Table 984. lowRISC_opentitan/hw/ip/uart/rtl/uart_rx.sv — 포트 (13개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
rx_enable	input	
tick_baud_x16	input	
parity_enable	input	
parity_odd	input	
tick_baud	output	logic
rx_valid	output	logic
rx_data	output	[7:0]
idle	output	logic
frame_err	output	
rx_parity_err	output	
rx	input	

API — uart_tx.sv

Table 985. lowRISC_opentitan/hw/ip/uart/rtl/uart_tx.sv — module / interface / package (1개)

이름	종류
uart_tx	module

Table 986. lowRISC_opentitan/hw/ip/uart/rtl/uart_tx.sv — 포트 (10개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	

포트	방향	형/폭
tx_enable	input	
tick_baud_x16	input	
parity_enable	input	logic
wr	input	
wr_parity	input	logic
wr_data	input	[7:0]
idle	output	
tx	output	logic

1. lowRISC_opentitan/hw/ip/uart/rtl/uart.sv (144 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: UART top level wrapper file
6
7 `include "prim_assert.sv"
8
9 module uart
10     import uart_reg_pkg::*;
11
12     #(
13         parameter logic [NumAlerts-1:0]      AlertAsyn-
14         cOn = {NumAlerts{1'b1}},
15         // Number of cycles a differential skew is tolerated on the
16         alert signal
17         parameter int unsigned                AlertSkew-
18         cycles = 1,
19         parameter bit                         EnableR-
20         acl = 1'b0,
21         parameter bit                         RaclEr-
22         rorRsp = EnableRacl,
23         parameter top_racl_pkg::racl_policy_sel_t RaclPoli-
24         cySelVec[NumRegs] = '{NumRegs{0}}
25     ) (
26         input          clk_i,
27         input          rst_ni,
28
29         // Bus Interface
30         input  tlul_pkg::tl_h2d_t tl_i,
31         output tlul_pkg::tl_d2h_t tl_o,
32
33         // Alerts
34         input  prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
35         alert_rx_i,
36         output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
37         alert_tx_o,
38
39         // RACL interface
40         input  top_racl_pkg::racl_policy_vec_t racl_policies_i,
41         output top_racl_pkg::racl_error_log_t  racl_error_o,
42
43         output logic      lsio_trigger_o,
44
45         // Generic IO
46         input  cio_rx_i,
47         output logic cio_tx_o,
48         output logic cio_tx_en_o,
49
50         // Interrupts
51         output logic intr_tx_watermark_o ,
52         output logic intr_tx_empty_o ,
53         output logic intr_rx_watermark_o ,
54         output logic intr_tx_done_o ,
55         output logic intr_rx_overflow_o ,
56         output logic intr_rx_frame_err_o ,
57         output logic intr_rx_break_err_o ,
58         output logic intr_rx_timeout_o ,
59         output logic intr_rx_parity_err_o
60     );
61
62     logic [NumAlerts-1:0] alert_test, alerts;
63     uart_reg2hw_t reg2hw;
64     uart_hw2reg_t hw2reg;
65
66     uart_reg_top #(
67         .EnableRacl(EnableRacl),
68         .RaclErrorRsp(RaclErrorRsp),
69         .RaclPolicySelVec(RaclPolicySelVec)
70     ) u_reg (

```

```

62     .clk_i,
63     .rst_ni,
64     .tl_i,
65     .tl_o,
66     .reg2hw,
67     .hw2reg,
68     .racl_policies_i,
69     .racl_error_o,
70     // SEC_CM: BUS_INTEGRITY
71     .intg_err_o (alerts[0])
72 );
73
74 uart_core uart_core (
75     .clk_i,
76     .rst_ni,
77     .reg2hw,
78     .hw2reg,
79
80     .rx      (cio_rx_i  ),
81     .tx      (cio_tx_o  ),
82
83     .lsio_trigger_o,
84
85     .intr_tx_watermark_o,
86     .intr_tx_empty_o,
87     .intr_rx_watermark_o,
88     .intr_tx_done_o,
89     .intr_rx_overflow_o,
90     .intr_rx_frame_err_o,
91     .intr_rx_break_err_o,
92     .intr_rx_timeout_o,
93     .intr_rx_parity_err_o
94 );
95
96 // Alerts
97 assign alert_test = {
98     reg2hw.alert_test.q &
99     reg2hw.alert_test.qe
100 };
101
102 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
103     prim_alert_sender #(
104         .AsyncOn(AlertAsyncOn[i]),
105         .SkewCycles(AlertSkewCycles),
106         .IsFatal(1'b1)
107     ) u_prim_alert_sender (
108         .clk_i,
109         .rst_ni,
110         .alert_test_i ( alert_test[i] ),
111         .alert_req_i  ( alerts[0]      ),
112         .alert_ack_o  (                ),
113         .alert_state_o (                ),
114         .alert_rx_i   ( alert_rx_i[i] ),
115         .alert_tx_o   ( alert_tx_o[i] )
116     );
117 end
118
119 // always enable the driving out of TX
120 assign cio_tx_en_o = 1'b1;
121
122 // Assert Known for outputs
123 `ASSERT(TxEnIsOne_A, cio_tx_en_o == 1'b1)
124 `ASSERT_KNOWN(TxKnown_A, cio_tx_o, clk_i, !rst_ni || !
125 cio_tx_en_o)
126
127 // Assert Known for alerts
128 `ASSERT_KNOWN(AlertsKnown_A, alert_tx_o)
129
130 // Assert Known for interrupts
131 `ASSERT_KNOWN(TxWatermarkKnown_A, intr_tx_watermark_o)
132 `ASSERT_KNOWN(TxEmptyKnown_A, intr_tx_empty_o)
133 `ASSERT_KNOWN(RxWatermarkKnown_A, intr_rx_watermark_o)
134 `ASSERT_KNOWN(TxDoneKnown_A, intr_tx_done_o)
135 `ASSERT_KNOWN(RxOverflowKnown_A, intr_rx_overflow_o)

```

```

135 `ASSERT_KNOWN(RxFrameErrKnown_A, intr_rx_frame_err_o)
136 `ASSERT_KNOWN(RxBreakErrKnown_A, intr_rx_break_err_o)
137 `ASSERT_KNOWN(RxTimeoutKnown_A, intr_rx_timeout_o)
138 `ASSERT_KNOWN(RxParityErrKnown_A, intr_rx_parity_err_o)
139 `ASSERT_KNOWN(LsioTriggerKnown_A, lsio_trigger_o)
140 `ASSERT_KNOWN(RacLErrKnown_A, rac_l_error_o.valid)
141
142 // Alert assertions for reg_we onehot check
143 `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
u_reg, alert_tx_o[0])
144 endmodule

```

2. lowRISC_opentitan/hw/ip/uart/rtl/uart_core.sv (525 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: UART core module
6 //
7
8 `include "prim_assert.sv"
9
10 module uart_core (
11     input                clk_i,
12     input                rst_ni,
13
14     input  uart_reg_pkg::uart_reg2hw_t reg2hw,
15     output uart_reg_pkg::uart_hw2reg_t hw2reg,
16
17     input                rx,
18     output logic         tx,
19
20     output logic         lsio_trigger_o,
21
22     output logic         intr_tx_watermark_o,
23     output logic         intr_tx_empty_o,
24     output logic         intr_rx_watermark_o,
25     output logic         intr_tx_done_o,
26     output logic         intr_rx_overflow_o,
27     output logic         intr_rx_frame_err_o,
28     output logic         intr_rx_break_err_o,
29     output logic         intr_rx_timeout_o,
30     output logic         intr_rx_parity_err_o
31 );
32
33 import uart_reg_pkg::*;
34
35 localparam int NcoWidth = $bits(reg2hw.ctrl.nco.q);
36 localparam int TxFifoDepthW = $clog2(TxFifoDepth)+1;
37 localparam int RxFifoDepthW = $clog2(RxFifoDepth)+1;
38
39 // The design does not support FIFOs deeper than 255 elements with the current CSR layout.
40 `ASSERT_INIT(TxFifoDepth_A, TxFifoDepth < 256)
41 `ASSERT_INIT(RxFifoDepth_A, RxFifoDepth < 256)
42
43 logic [15:0] rx_val_q;
44 logic [7:0]  uart_rdata;
45 logic        tick_baud_x16, rx_tick_baud;
46
47 logic [TxFifoDepthW-1:0] tx_fifo_depth;
48 logic [RxFifoDepthW-1:0] rx_fifo_depth;
49 logic [RxFifoDepthW-1:0] rx_fifo_depth_prev_q;
50
51 logic [23:0] rx_timeout_count_d, rx_timeout_count_q,
uart_rxtx_val;
52 logic        rx_fifo_depth_changed, uart_rxtx_en;
53 logic        tx_enable, rx_enable;
54 logic        sys_loopback, line_loopback, rxnf_enable;
55 logic        uart_fifo_rrst, uart_fifo_txrst;
56 logic [2:0]  uart_fifo_rxilvl;
57 logic [2:0]  uart_fifo_txilvl;
58 logic        ovr_d_tx_en, ovr_d_tx_val;
59 logic [7:0]  tx_fifo_data;
60 logic        tx_fifo_rready, tx_fifo_rvalid;
61 logic        tx_fifo_wready, tx_uart_idle;
62 logic        tx_out;
63 logic        tx_out_q;
64 logic [7:0]  rx_fifo_data;
65 logic        rx_valid, rx_fifo_wvalid, rx_fifo_rvalid;
66 logic        rx_fifo_wready, rx_uart_idle;
67 logic        rx_sync;
68 logic        rx_in;
69 logic        break_err;
70 logic [4:0]  allzero_cnt_d, allzero_cnt_q;
71 logic        allzero_err, not_allzero_char;
72 logic        event_tx_watermark, event_tx_empty,
event_rx_watermark, event_tx_done;
73 logic        event_rx_overflow, event_rx_frame_err,
event_rx_break_err, event_rx_timeout;
74 logic        event_rx_parity_err;
75 logic        tx_uart_idle_q;
76
77 assign tx_enable = reg2hw.ctrl.tx.q;
78 assign rx_enable = reg2hw.ctrl.rx.q;
79 assign rxnf_enable = reg2hw.ctrl.nf.q;

```

```

80 assign sys_loopback = reg2hw.ctrl.slpbk.q;
81 assign line_loopback = reg2hw.ctrl.llpbk.q;
82
83 assign uart_fifo_rrst = reg2hw.fifo_ctrl.rrst.q &
reg2hw.fifo_ctrl.rrst.q;
84 assign uart_fifo_txrst = reg2hw.fifo_ctrl.txrst.q &
reg2hw.fifo_ctrl.txrst.q;
85 assign uart_fifo_rxilvl = reg2hw.fifo_ctrl.rxilvl.q;
86 assign uart_fifo_txilvl = reg2hw.fifo_ctrl.txilvl.q;
87
88 assign ovr_d_tx_en = reg2hw.ovrd.txen.q;
89 assign ovr_d_tx_val = reg2hw.ovrd.txval.q;
90
91 typedef enum logic {
92     BRK_CHK,
93     BRK_WAIT
94 } break_st_e ;
95
96 break_st_e break_st_q;
97
98 assign not_allzero_char = rx_valid & (~event_rx_frame_err |
(rx_fifo_data != 8'h0));
99 assign allzero_err = event_rx_frame_err & (rx_fifo_data ==
8'h0);
100
101
102 assign allzero_cnt_d = (break_st_q == BRK_WAIT ||
not_allzero_char) ? 5'h0 :
103
104 //allzero_cnt_q[4] never be 1b
without break_st_q as BRK_WAIT
105 //allzero_cnt_q[4] ? allzero_cnt_q :
106 allzero_err ? allzero_cnt_q + 5'd1 :
107 allzero_cnt_q;
108
109 always_ff @(posedge clk_i or negedge rst_ni) begin
110     if (!rst_ni) allzero_cnt_q <= '0;
111     else if (rx_enable) allzero_cnt_q <= allzero_cnt_d;
112 end
113
114 // break_err edges in same cycle as event_rx_frame_err
edges ; that way the
115 // reset-on-read works the same way for break and frame er-
ror interrupts.
116
117 always_comb begin
118     unique case (reg2hw.ctrl.rxbvl.q)
119         2'h0: break_err = allzero_cnt_d >= 5'd2;
120         2'h1: break_err = allzero_cnt_d >= 5'd4;
121         2'h2: break_err = allzero_cnt_d >= 5'd8;
122         default: break_err = allzero_cnt_d >= 5'd16;
123     endcase
124 end
125
126 always_ff @(posedge clk_i or negedge rst_ni) begin
127     if (!rst_ni) break_st_q <= BRK_CHK;
128     else begin
129         unique case (break_st_q)
130             BRK_CHK: begin
131                 if (event_rx_break_err) break_st_q <= BRK_WAIT;
132             end
133             BRK_WAIT: begin
134                 if (rx_in) break_st_q <= BRK_CHK;
135             end
136             default: begin
137                 break_st_q <= BRK_CHK;
138             end
139         endcase
140     end
141 end
142
143 assign hw2reg.val.d = rx_val_q;
144
145 assign hw2reg.rdata.d = uart_rdata;
146
147 assign hw2reg.status.rxempty.d = ~rx_fifo_rvalid;
148 assign hw2reg.status.rxidle.d = rx_uart_idle;
149 assign hw2reg.status.txidle.d = tx_uart_idle &
~tx_fifo_rvalid;
150 assign hw2reg.status.txempty.d = ~tx_fifo_rvalid;
151 assign hw2reg.status.rxfull.d = ~rx_fifo_wready;
152 assign hw2reg.status.txfull.d = ~tx_fifo_wready;
153
154 assign hw2reg.fifo_status.txlvl.d = 8'(tx_fifo_depth);
155 assign hw2reg.fifo_status.rxlvl.d = 8'(rx_fifo_depth);
156
157 // resets are self-clearing, so need to update FIFO_CTRL
158 assign hw2reg.fifo_ctrl.rxilvl.d = 1'b0;
159 assign hw2reg.fifo_ctrl.txilvl.d = 3'h0;
160 assign hw2reg.fifo_ctrl.txilvl.d = 1'b0;
161 assign hw2reg.fifo_ctrl.txilvl.d = 3'h0;
162
163 //
164 // NCO 16x Baud Generator
165 // output clock rate is:
166 // Fin * (NCO/2**NcoWidth)
167 logic [NcoWidth:0] nco_sum_q; // extra bit to get the
carry
168
169 always_ff @(posedge clk_i or negedge rst_ni) begin
170     if (!rst_ni) begin

```



```

171     nco_sum_q <= 17'h0;
172 end else if (tx_enable || rx_enable) begin
173     nco_sum_q <= {1'b0, nco_sum_q[NcoWidth-1:0]} +
{1'b0, reg2hw.ctrl.nco.q[NcoWidth-1:0]};
174 end
175 end
176
177 assign tick_baud_x16 = nco_sum_q[16];
178
179 ///////////////
180 // TX Logic //
181 ///////////////
182
183 assign tx_fifo_rready = tx_uart_idle & tx_fifo_rvalid &
tx_enable;
184
185 prim_fifo_sync #(
186     .Width      (8),
187     .Pass       (1'b0),
188     .Depth      (TxFifoDepth)
189 ) u_uart_txfifo (
190     .clk_i,
191     .rst_ni,
192     .clr_i      (uart_fifo_txrst),
193     .wvalid_i   (reg2hw.wdata.qe),
194     .wready_o   (tx_fifo_wready),
195     .wdata_i    (reg2hw.wdata.q),
196     .depth_o    (tx_fifo_depth),
197     .full_o     (),
198     .rvalid_o   (tx_fifo_rvalid),
199     .rready_i   (tx_fifo_rready),
200     .rdata_o    (tx_fifo_data),
201     .err_o      ()
202 );
203
204 uart_tx uart_tx (
205     .clk_i,
206     .rst_ni,
207     .tx_enable,
208     .tick_baud_x16,
209     .parity_enable (reg2hw.ctrl.parity_en.q),
210     .wr            (tx_fifo_rready),
211     .wr_parity     ((^tx_fifo_data) ^
reg2hw.ctrl.parity_odd.q),
212     .wr_data       (tx_fifo_data),
213     .idle          (tx_uart_idle),
214     .tx            (tx_out)
215 );
216
217 assign tx = line_loopback ? rx : tx_out_q ;
218 always_ff @(posedge clk_i or negedge rst_ni) begin
219     if (!rst_ni) begin
220         tx_out_q <= 1'b1;
221     end else if (ovrd_tx_en) begin
222         tx_out_q <= ovrd_tx_val ;
223     end else if (sys_loopback) begin
224         tx_out_q <= 1'b1;
225     end else begin
226         tx_out_q <= tx_out;
227     end
228 end
229
230 ///////////////
231 // RX Logic //
232 ///////////////
233
234 // sync the incoming data
235 prim_flop_2sync #(
236     .Width(1),
237     .ResetValue(1'b1)
238 ) sync_rx (
239     .clk_i,
240     .rst_ni,
241     .d_i(rx),
242     .q_o(rx_sync)
243 );
244
245 // Based on: en.wikipedia.org/wiki/Repetition_code mentions
the use of a majority filter
246 // in UART to ignore brief noise spikes
247 logic rx_sync_q1, rx_sync_q2, rx_in_mx, rx_in_maj;
248
249 always_ff @(posedge clk_i or negedge rst_ni) begin
250     if (!rst_ni) begin
251         rx_sync_q1 <= 1'b1;
252         rx_sync_q2 <= 1'b1;
253     end else begin
254         rx_sync_q1 <= rx_sync;
255         rx_sync_q2 <= rx_sync_q1;
256     end
257 end
258
259 assign rx_in_maj = (rx_sync & rx_sync_q1) |
260                  (rx_sync & rx_sync_q2) |
261                  (rx_sync_q1 & rx_sync_q2);
262 assign rx_in_mx  = rxnf_enable ? rx_in_maj : rx_sync;
263
264 assign rx_in = sys_loopback ? tx_out :
265               line_loopback ? 1'b1 :
266               rx_in_mx;
267

```

```

268 uart_rx uart_rx (
269     .clk_i,
270     .rst_ni,
271     .rx_enable,
272     .tick_baud_x16,
273     .parity_enable (reg2hw.ctrl.parity_en.q),
274     .parity_odd    (reg2hw.ctrl.parity_odd.q),
275     .tick_baud     (rx_tick_baud),
276     .rx_valid,
277     .rx_data       (rx_fifo_data),
278     .idle          (rx_uart_idle),
279     .frame_err     (event_rx_frame_err),
280     .rx            (rx_in),
281     .rx_parity_err (event_rx_parity_err)
282 );
283
284 assign rx_fifo_wvalid = rx_valid & ~event_rx_frame_err &
~event_rx_parity_err;
285
286 prim_fifo_sync #(
287     .Width      (8),
288     .Pass       (1'b0),
289     .Depth      (RxFifoDepth)
290 ) u_uart_rxfifo (
291     .clk_i,
292     .rst_ni,
293     .clr_i      (uart_fifo_rxrst),
294     .wvalid_i   (rx_fifo_wvalid),
295     .wready_o   (rx_fifo_wready),
296     .wdata_i    (rx_fifo_data),
297     .depth_o    (rx_fifo_depth),
298     .full_o     (),
299     .rvalid_o   (rx_fifo_rvalid),
300     .rready_i   (reg2hw.rdata.re),
301     .rdata_o    (uart_rdata),
302     .err_o      ()
303 );
304
305 always_ff @(posedge clk_i or negedge rst_ni) begin
306     if (!rst_ni) rx_val_q <= 16'h0;
307     else if (tick_baud_x16) rx_val_q <= {rx_val_q[14:0],
rx_in};
308 end
309
310 ///////////////
311 // Interrupt & Status //
312 ///////////////
313
314 logic [TxFifoDepthW-1:0] tx_watermark_thresh;
315 always_comb begin
316     // Create power of two thresholds.
317     // The threshold saturates at half the FIFO depth.
318     if (uart_fifo_txilvl >= (TxFifoDepthW-2)) begin
319         tx_watermark_thresh = TxFifoDepthW'(TxFifoDepth/2);
320     end else begin
321         tx_watermark_thresh = 1'b1 << uart_fifo_txilvl;
322     end
323     event_tx_watermark = tx_fifo_depth < tx_watermark_thresh;
324 end
325
326 assign event_tx_empty = tx_fifo_depth == '0;
327
328 assign event_tx_done = ~tx_fifo_rvalid & ~tx_uart_idle_q &
tx_uart_idle;
329
330 always_ff @(posedge clk_i or negedge rst_ni) begin
331     if (!rst_ni) begin
332         tx_uart_idle_q <= 1'b1;
333     end else begin
334         tx_uart_idle_q <= tx_uart_idle;
335     end
336 end
337
338 logic [RxFifoDepthW-1:0] rx_watermark_thresh;
339 always_comb begin
340     // Create power of two thresholds.
341     if (uart_fifo_rxilvl > (RxFifoDepthW-1)) begin
342         // This results in the comparison always returning 0 be-
low because RxFifoDepth can
343         // encode depths up to 2*RxFifoDepth-1.
344         rx_watermark_thresh = {RxFifoDepthW{1'b1}};
345     end else if (uart_fifo_rxilvl == (RxFifoDepthW-1)) begin
346         // The maximum valid threshold threshold is an exception
and saturates at RxFifoDepth-2.
347         rx_watermark_thresh = RxFifoDepthW'(RxFifoDepth-2);
348     end else begin
349         rx_watermark_thresh = 1'b1 << uart_fifo_rxilvl;
350     end
351     event_rx_watermark = rx_fifo_depth >= rx_watermark_thresh;
352 end
353
354 // Flop trigger signal to avoid glitches on the output
355 always_ff @(posedge clk_i or negedge rst_ni) begin
356     if (!rst_ni) begin
357         lsio_trigger_o <= 1'b0;
358     end else begin
359         lsio_trigger_o <= event_tx_watermark |
event_rx_watermark;
360     end
361 end

```

```

362
363 // rx timeout interrupt
364 assign uart_rxtx_en = reg2hw.timeout_ctrl.en.q;
365 assign uart_rxtx_val = reg2hw.timeout_ctrl.val.q;
366
367 assign rx_fifo_depth_changed = (rx_fifo_depth !=
rx_fifo_depth_prev_q);
368
369 assign rx_timeout_count_d =
370 // don't count if timeout feature not enabled ;
371 // will never reach timeout val + lower power
372 (uart_rxtx_en == 1'b0) ? 24'd0 :
373 // reset count if timeout interrupt is set
374 event_rx_timeout ? 24'd0 :

375 // reset count upon change in fifo level: covers
both read and receiving a new byte
376 rx_fifo_depth_changed ? 24'd0 :
377 // reset count if no bytes are pending
378 (rx_fifo_depth == '0) ? 24'd0 :
379 // stop the count at timeout value (this will
set the interrupt)
380 // Removed below line as when the timeout
reaches the value,
381 // event occurred, and timeout value reset to
0h.
382 //(rx_timeout_count_q == uart_rxtx_val) ?
rx_timeout_count_q :
383 // increment if at rx baud tick
384 rx_tick_baud ?
(rx_timeout_count_q + 24'd1) :
385 rx_timeout_count_q;
386
387 assign event_rx_timeout = (rx_timeout_count_q ==
uart_rxtx_val) & uart_rxtx_en;
388
389 always_ff @(posedge clk_i or negedge rst_ni) begin
390 if (!rst_ni) begin
391 rx_timeout_count_q <= 24'd0;
392 rx_fifo_depth_prev_q <= '0;
393 end else begin
394 rx_timeout_count_q <= rx_timeout_count_d;
395 rx_fifo_depth_prev_q <= rx_fifo_depth;
396 end
397 end
398
399 assign event_rx_overflow = rx_fifo_wvalid &
~rx_fifo_wready;
400 assign event_rx_break_err = break_err & (break_st_q ==
BRK_CHK);
401
402 // instantiate interrupt hardware primitives
403
404 prim_intr_hw #(.Width(1), .IntrT("Status")) intr_hw_tx_wa-
termark (
405 .clk_i,
406 .rst_ni,
407 .event_intr_i (event_tx_watermark),
408 .reg2hw_intr_enable_q_i
(reg2hw.intr_enable.tx_watermark.q),
409 .reg2hw_intr_test_q_i (reg2hw.intr_test.tx_watermark.q),
410 .reg2hw_intr_test_qe_i
(reg2hw.intr_test.tx_watermark.qe),
411 .reg2hw_intr_state_q_i
(reg2hw.intr_state.tx_watermark.q),
412 .hw2reg_intr_state_de_o
(hw2reg.intr_state.tx_watermark.de),
413 .hw2reg_intr_state_d_o
(hw2reg.intr_state.tx_watermark.d),
414 .intr_o (intr_tx_watermark_o)
415 );
416
417 prim_intr_hw #(.Width(1), .IntrT("Status")) intr_hw_tx_empty (
418 .clk_i,
419 .rst_ni,
420 .event_intr_i (event_tx_empty),
421 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.tx_empty.q),
422 .reg2hw_intr_test_q_i (reg2hw.intr_test.tx_empty.q),
423 .reg2hw_intr_test_qe_i (reg2hw.intr_test.tx_empty.qe),
424 .reg2hw_intr_state_q_i (reg2hw.intr_state.tx_empty.q),
425 .hw2reg_intr_state_de_o (hw2reg.intr_state.tx_empty.de),
426 .hw2reg_intr_state_d_o (hw2reg.intr_state.tx_empty.d),
427 .intr_o (intr_tx_empty_o)
428 );
429
430 prim_intr_hw #(.Width(1), .IntrT("Status")) intr_hw_rx_wa-
termark (
431 .clk_i,
432 .rst_ni,
433 .event_intr_i (event_rx_watermark),
434 .reg2hw_intr_enable_q_i
(reg2hw.intr_enable.rx_watermark.q),
435 .reg2hw_intr_test_q_i (reg2hw.intr_test.rx_watermark.q),
436 .reg2hw_intr_test_qe_i
(reg2hw.intr_test.rx_watermark.qe),
437 .reg2hw_intr_state_q_i
(reg2hw.intr_state.rx_watermark.q),
438 .hw2reg_intr_state_de_o
(hw2reg.intr_state.rx_watermark.de),
439 .hw2reg_intr_state_d_o
(hw2reg.intr_state.rx_watermark.d),
440 .intr_o (intr_rx_watermark_o)
441 );
442
443 prim_intr_hw #(.Width(1)) intr_hw_tx_done (
444 .clk_i,
445 .rst_ni,
446 .event_intr_i (event_tx_done),
447 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.tx_done.q),
448 .reg2hw_intr_test_q_i (reg2hw.intr_test.tx_done.q),
449 .reg2hw_intr_test_qe_i (reg2hw.intr_test.tx_done.qe),
450 .reg2hw_intr_state_q_i (reg2hw.intr_state.tx_done.q),
451 .hw2reg_intr_state_de_o (hw2reg.intr_state.tx_done.de),
452 .hw2reg_intr_state_d_o (hw2reg.intr_state.tx_done.d),
453 .intr_o (intr_tx_done_o)
454 );
455
456 prim_intr_hw #(.Width(1)) intr_hw_rx_overflow (
457 .clk_i,
458 .rst_ni,
459 .event_intr_i (event_rx_overflow),
460 .reg2hw_intr_enable_q_i
(reg2hw.intr_enable.rx_overflow.q),
461 .reg2hw_intr_test_q_i (reg2hw.intr_test.rx_overflow.q),
462 .reg2hw_intr_test_qe_i (reg2hw.intr_test.rx_overflow.qe),
463 .reg2hw_intr_state_q_i (reg2hw.intr_state.rx_overflow.q),
464 .hw2reg_intr_state_de_o
(hw2reg.intr_state.rx_overflow.de),
465 .hw2reg_intr_state_d_o (hw2reg.intr_state.rx_overflow.d),
466 .intr_o (intr_rx_overflow_o)
467 );
468
469 prim_intr_hw #(.Width(1)) intr_hw_rx_frame_err (
470 .clk_i,
471 .rst_ni,
472 .event_intr_i (event_rx_frame_err),
473 .reg2hw_intr_enable_q_i
(reg2hw.intr_enable.rx_frame_err.q),
474 .reg2hw_intr_test_q_i (reg2hw.intr_test.rx_frame_err.q),
475 .reg2hw_intr_test_qe_i
(reg2hw.intr_test.rx_frame_err.qe),
476 .reg2hw_intr_state_q_i
(reg2hw.intr_state.rx_frame_err.q),
477 .hw2reg_intr_state_de_o
(hw2reg.intr_state.rx_frame_err.de),
478 .hw2reg_intr_state_d_o
(hw2reg.intr_state.rx_frame_err.d),
479 .intr_o (intr_rx_frame_err_o)
480 );
481
482 prim_intr_hw #(.Width(1)) intr_hw_rx_break_err (
483 .clk_i,
484 .rst_ni,
485 .event_intr_i (event_rx_break_err),
486 .reg2hw_intr_enable_q_i
(reg2hw.intr_enable.rx_break_err.q),
487 .reg2hw_intr_test_q_i (reg2hw.intr_test.rx_break_err.q),
488 .reg2hw_intr_test_qe_i
(reg2hw.intr_test.rx_break_err.qe),
489 .reg2hw_intr_state_q_i
(reg2hw.intr_state.rx_break_err.q),
490 .hw2reg_intr_state_de_o
(hw2reg.intr_state.rx_break_err.de),
491 .hw2reg_intr_state_d_o
(hw2reg.intr_state.rx_break_err.d),
492 .intr_o (intr_rx_break_err_o)
493 );
494
495 prim_intr_hw #(.Width(1)) intr_hw_rx_timeout (
496 .clk_i,
497 .rst_ni,
498 .event_intr_i (event_rx_timeout),
499 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.rx_timeout.q),
500 .reg2hw_intr_test_q_i (reg2hw.intr_test.rx_timeout.q),
501 .reg2hw_intr_test_qe_i (reg2hw.intr_test.rx_timeout.qe),
502 .reg2hw_intr_state_q_i (reg2hw.intr_state.rx_timeout.q),
503 .hw2reg_intr_state_de_o (hw2reg.intr_state.rx_timeout.de),
504 .hw2reg_intr_state_d_o (hw2reg.intr_state.rx_timeout.d),
505 .intr_o (intr_rx_timeout_o)
506 );
507
508 prim_intr_hw #(.Width(1)) intr_hw_rx_parity_err (
509 .clk_i,
510 .rst_ni,
511 .event_intr_i (event_rx_parity_err),
512 .reg2hw_intr_enable_q_i
(reg2hw.intr_enable.rx_parity_err.q),
513 .reg2hw_intr_test_q_i
(reg2hw.intr_test.rx_parity_err.q),
514 .reg2hw_intr_test_qe_i
(reg2hw.intr_test.rx_parity_err.qe),
515 .reg2hw_intr_state_q_i
(reg2hw.intr_state.rx_parity_err.q),
516 .hw2reg_intr_state_de_o
(hw2reg.intr_state.rx_parity_err.de),
517 .hw2reg_intr_state_d_o
(hw2reg.intr_state.rx_parity_err.d),
518 .intr_o (intr_rx_parity_err_o)
519 );
520
521 // unused registers

```

```

522 logic unused_reg;
523 assign unused_reg = ^reg2hw.alert_test;
524
525 endmodule

```

3. lowRISC_opentitan/hw/ip/uart/rtl/uart_reg_pkg.sv (421 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Package auto-generated by `reggen` containing data
  structure
6
7 package uart_reg_pkg;
8
9 // Param list
10 parameter int RxFifoDepth = 64;
11 parameter int TxFifoDepth = 32;
12 parameter int NumAlerts = 1;
13
14 // Address widths within the block
15 parameter int BlockAw = 6;
16
17 // Number of registers for every interface
18 parameter int NumRegs = 13;
19
20 // Alert indices
21 typedef enum int {
22     AlertFatalFaultIdx = 0
23 } uart_alert_idx_t;
24
25 ///////////////////////////////////////////////////
26 // Typedefs for registers //
27 ///////////////////////////////////////////////////
28
29 typedef struct packed {
30     struct packed {
31         logic q;
32     } tx_empty;
33     struct packed {
34         logic q;
35     } rx_parity_err;
36     struct packed {
37         logic q;
38     } rx_timeout;
39     struct packed {
40         logic q;
41     } rx_break_err;
42     struct packed {
43         logic q;
44     } rx_frame_err;
45     struct packed {
46         logic q;
47     } rx_overflow;
48     struct packed {
49         logic q;
50     } tx_done;
51     struct packed {
52         logic q;
53     } rx_watermark;
54     struct packed {
55         logic q;
56     } tx_watermark;
57 } uart_reg2hw_intr_state_reg_t;
58
59 typedef struct packed {
60     struct packed {
61         logic q;
62     } tx_empty;
63     struct packed {
64         logic q;
65     } rx_parity_err;
66     struct packed {
67         logic q;
68     } rx_timeout;
69     struct packed {
70         logic q;
71     } rx_break_err;
72     struct packed {
73         logic q;
74     } rx_frame_err;
75     struct packed {
76         logic q;
77     } rx_overflow;
78     struct packed {
79         logic q;
80     } tx_done;
81     struct packed {
82         logic q;
83     } rx_watermark;
84     struct packed {
85         logic q;
86     } tx_watermark;
87 } uart_reg2hw_intr_enable_reg_t;
88
89 typedef struct packed {

```

```

90     struct packed {
91         logic q;
92         logic qe;
93     } tx_empty;
94     struct packed {
95         logic q;
96         logic qe;
97     } rx_parity_err;
98     struct packed {
99         logic q;
100        logic qe;
101    } rx_timeout;
102    struct packed {
103        logic q;
104        logic qe;
105    } rx_break_err;
106    struct packed {
107        logic q;
108        logic qe;
109    } rx_frame_err;
110    struct packed {
111        logic q;
112        logic qe;
113    } rx_overflow;
114    struct packed {
115        logic q;
116        logic qe;
117    } tx_done;
118    struct packed {
119        logic q;
120        logic qe;
121    } rx_watermark;
122    struct packed {
123        logic q;
124        logic qe;
125    } tx_watermark;
126 } uart_reg2hw_intr_test_reg_t;
127
128 typedef struct packed {
129     logic q;
130     logic qe;
131 } uart_reg2hw_alert_test_reg_t;
132
133 typedef struct packed {
134     struct packed {
135         logic [15:0] q;
136     } nco;
137     struct packed {
138         logic [1:0] q;
139     } rxblvl;
140     struct packed {
141         logic q;
142     } parity_odd;
143     struct packed {
144         logic q;
145     } parity_en;
146     struct packed {
147         logic q;
148     } llpbk;
149     struct packed {
150         logic q;
151     } slpbk;
152     struct packed {
153         logic q;
154     } nf;
155     struct packed {
156         logic q;
157     } rx;
158     struct packed {
159         logic q;
160     } tx;
161 } uart_reg2hw_ctrl_reg_t;
162
163 typedef struct packed {
164     struct packed {
165         logic q;
166         logic re;
167     } rxempty;
168     struct packed {
169         logic q;
170         logic re;
171     } rxidle;
172     struct packed {
173         logic q;
174         logic re;
175     } txidle;
176     struct packed {
177         logic q;
178         logic re;
179     } txempty;
180     struct packed {
181         logic q;
182         logic re;
183     } rxfull;
184     struct packed {
185         logic q;
186         logic re;
187     } txfull;
188 } uart_reg2hw_status_reg_t;
189
190 typedef struct packed {

```

```

191 logic [7:0] q;
192 logic re;
193 } uart_reg2hw_rdata_reg_t;
194
195 typedef struct packed {
196     logic [7:0] q;
197     logic qe;
198 } uart_reg2hw_wdata_reg_t;
199
200 typedef struct packed {
201     struct packed {
202         logic [2:0] q;
203         logic qe;
204     } txilvl;
205     struct packed {
206         logic [2:0] q;
207         logic qe;
208     } rxilvl;
209     struct packed {
210         logic q;
211         logic qe;
212     } txrst;
213     struct packed {
214         logic q;
215         logic qe;
216     } rxrst;
217 } uart_reg2hw_fifo_ctrl_reg_t;
218
219 typedef struct packed {
220     struct packed {
221         logic q;
222     } txval;
223     struct packed {
224         logic q;
225     } txen;
226 } uart_reg2hw_ovrd_reg_t;
227
228 typedef struct packed {
229     struct packed {
230         logic q;
231     } en;
232     struct packed {
233         logic [23:0] q;
234     } val;
235 } uart_reg2hw_timeout_ctrl_reg_t;
236
237 typedef struct packed {
238     struct packed {
239         logic d;
240         logic de;
241     } tx_empty;
242     struct packed {
243         logic d;
244         logic de;
245     } rx_parity_err;
246     struct packed {
247         logic d;
248         logic de;
249     } rx_timeout;
250     struct packed {
251         logic d;
252         logic de;
253     } rx_break_err;
254     struct packed {
255         logic d;
256         logic de;
257     } rx_frame_err;
258     struct packed {
259         logic d;
260         logic de;
261     } rx_overflow;
262     struct packed {
263         logic d;
264         logic de;
265     } tx_done;
266     struct packed {
267         logic d;
268         logic de;
269     } rx_watermark;
270     struct packed {
271         logic d;
272         logic de;
273     } tx_watermark;
274 } uart_hw2reg_intr_state_reg_t;
275
276 typedef struct packed {
277     struct packed {
278         logic d;
279     } rxempty;
280     struct packed {
281         logic d;
282     } rxidle;
283     struct packed {
284         logic d;
285     } txidle;
286     struct packed {
287         logic d;
288     } txempty;
289     struct packed {
290         logic d;
291     } rxfull;
292
293     struct packed {
294         logic d;
295     } txfull;
296 } uart_hw2reg_status_reg_t;
297
298 typedef struct packed {
299     logic [7:0] d;
300 } uart_hw2reg_rdata_reg_t;
301
302 typedef struct packed {
303     struct packed {
304         logic [2:0] d;
305         logic de;
306     } txilvl;
307     struct packed {
308         logic [2:0] d;
309         logic de;
310     } rxilvl;
311 } uart_hw2reg_fifo_ctrl_reg_t;
312
313 typedef struct packed {
314     struct packed {
315         logic [7:0] d;
316     } rxilvl;
317     struct packed {
318         logic [7:0] d;
319     } txilvl;
320 } uart_hw2reg_fifo_status_reg_t;
321
322 typedef struct packed {
323     logic [15:0] d;
324 } uart_hw2reg_val_reg_t;
325
326 // Register -> HW type
327 typedef struct packed {
328     uart_reg2hw_intr_state_reg_t intr_state; // [131:123]
329     uart_reg2hw_intr_enable_reg_t intr_enable; // [122:114]
330     uart_reg2hw_intr_test_reg_t intr_test; // [113:96]
331     uart_reg2hw_alert_test_reg_t alert_test; // [95:94]
332     uart_reg2hw_ctrl_reg_t ctrl; // [93:69]
333     uart_reg2hw_status_reg_t status; // [68:57]
334     uart_reg2hw_rdata_reg_t rdata; // [56:48]
335     uart_reg2hw_wdata_reg_t wdata; // [47:39]
336     uart_reg2hw_fifo_ctrl_reg_t fifo_ctrl; // [38:27]
337     uart_reg2hw_ovrd_reg_t ovrd; // [26:25]
338     uart_reg2hw_timeout_ctrl_reg_t timeout_ctrl; // [24:0]
339 } uart_reg2hw_t;
340
341 // HW -> register type
342 typedef struct packed {
343     uart_hw2reg_intr_state_reg_t intr_state; // [71:54]
344     uart_hw2reg_status_reg_t status; // [53:48]
345     uart_hw2reg_rdata_reg_t rdata; // [47:40]
346     uart_hw2reg_fifo_ctrl_reg_t fifo_ctrl; // [39:32]
347     uart_hw2reg_fifo_status_reg_t fifo_status; // [31:16]
348     uart_hw2reg_val_reg_t val; // [15:0]
349 } uart_hw2reg_t;
350
351 // Register offsets
352 parameter logic [BlockAw-1:0] UART_INTR_STATE_OFFSET = 6'h 0;
353 parameter logic [BlockAw-1:0] UART_INTR_ENABLE_OFFSET = 6'h 4;
354 parameter logic [BlockAw-1:0] UART_INTR_TEST_OFFSET = 6'h 8;
355 parameter logic [BlockAw-1:0] UART_ALERT_TEST_OFFSET = 6'h c;
356 parameter logic [BlockAw-1:0] UART_CTRL_OFFSET = 6'h 10;
357 parameter logic [BlockAw-1:0] UART_STATUS_OFFSET = 6'h 14;
358 parameter logic [BlockAw-1:0] UART_RDATA_OFFSET = 6'h 18;
359 parameter logic [BlockAw-1:0] UART_WDATA_OFFSET = 6'h 1c;
360 parameter logic [BlockAw-1:0] UART_FIFO_CTRL_OFFSET = 6'h 20;
361 parameter logic [BlockAw-1:0] UART_FIFO_STATUS_OFFSET = 6'h 24;
362 parameter logic [BlockAw-1:0] UART_OVRD_OFFSET = 6'h 28;
363 parameter logic [BlockAw-1:0] UART_VAL_OFFSET = 6'h 2c;
364
365 parameter logic [BlockAw-1:0] UART_TIMEOUT_CTRL_OFFSET = 6'h 30;
366
367 // Reset values for hwext registers and their fields
368 parameter logic [8:0] UART_INTR_TEST_RESVAL = 9'h 0;
369 parameter logic [0:0] UART_INTR_TEST_TX_WATERMARK_RESVAL = 1'h 0;
370 parameter logic [0:0] UART_INTR_TEST_RX_WATERMARK_RESVAL = 1'h 0;
371 parameter logic [0:0] UART_INTR_TEST_TX_DONE_RESVAL = 1'h 0;
372 parameter logic [0:0] UART_INTR_TEST_RX_OVERFLOW_RESVAL = 1'h 0;
373 parameter logic [0:0] UART_INTR_TEST_RX_FRAME_ERR_RESVAL = 1'h 0;
374 parameter logic [0:0] UART_INTR_TEST_RX_BREAK_ERR_RESVAL = 1'h 0;
375 parameter logic [0:0] UART_INTR_TEST_RX_TIMEOUT_RESVAL = 1'h 0;
376 parameter logic [0:0] UART_INTR_TEST_RX_PARITY_ERR_RESVAL = 1'h 0;
377 parameter logic [0:0] UART_INTR_TEST_TX_EMPTY_RESVAL = 1'h 0;
378 parameter logic [0:0] UART_ALERT_TEST_RESVAL = 1'h 0;

```

```

377 parameter logic [0:0] UART_ALERT_TEST_FATAL_FAULT_RESVAL =
1'h 0;
378 parameter logic [5:0] UART_STATUS_RESVAL = 6'h 3c;
379 parameter logic [0:0] UART_STATUS_TXEMPTY_RESVAL = 1'h 1;
380 parameter logic [0:0] UART_STATUS_TXIDLE_RESVAL = 1'h 1;
381 parameter logic [0:0] UART_STATUS_RXIDLE_RESVAL = 1'h 1;
382 parameter logic [0:0] UART_STATUS_RXEMPTY_RESVAL = 1'h 1;
383 parameter logic [7:0] UART_RDATA_RESVAL = 8'h 0;
384 parameter logic [23:0] UART_FIFO_STATUS_RESVAL = 24'h 0;
385 parameter logic [15:0] UART_VAL_RESVAL = 16'h 0;
386
387 // Register index
388 typedef enum int {
389     UART_INTR_STATE,
390     UART_INTR_ENABLE,
391     UART_INTR_TEST,
392     UART_ALERT_TEST,
393     UART_CTRL,
394     UART_STATUS,
395     UART_RDATA,
396     UART_WDATA,
397     UART_FIFO_CTRL,
398     UART_FIFO_STATUS,
399     UART_OVRD,
400     UART_VAL,
401     UART_TIMEOUT_CTRL
402 } uart_id_e;
403
404 // Register width information to check illegal writes
405 parameter logic [3:0] UART_PERMIT [13] = '{
406     4'b 0011, // index[ 0] UART_INTR_STATE
407     4'b 0011, // index[ 1] UART_INTR_ENABLE
408     4'b 0011, // index[ 2] UART_INTR_TEST
409     4'b 0001, // index[ 3] UART_ALERT_TEST
410     4'b 1111, // index[ 4] UART_CTRL
411     4'b 0001, // index[ 5] UART_STATUS
412     4'b 0001, // index[ 6] UART_RDATA
413     4'b 0001, // index[ 7] UART_WDATA
414     4'b 0001, // index[ 8] UART_FIFO_CTRL
415     4'b 0111, // index[ 9] UART_FIFO_STATUS
416     4'b 0001, // index[10] UART_OVRD
417     4'b 0011, // index[11] UART_VAL
418     4'b 1111 // index[12] UART_TIMEOUT_CTRL
419 };
420
421 endpackage

```

4. lowRISC_opentitan/hw/ip/uart/rtl/uart_reg_top.sv (1,916

lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"
8
9 module uart_reg_top
10 # (
11     parameter bit      EnableRacl      = 1'b0,
12     parameter bit      RclErrorRsp     = 1'b1,
13     parameter top_racl_pkg::racl_policy_sel_t RaclPoli-
cySelVec[uart_reg_pkg::NumRegs] =
14     '{uart_reg_pkg::NumRegs{0}}
15 ) (
16     input clk_i,
17     input rst_ni,
18     input tlul_pkg::tl_h2d_t tl_i,
19     output tlul_pkg::tl_d2h_t tl_o,
20     // To HW
21     output uart_reg_pkg::uart_reg2hw_t reg2hw, // Write
22     input  uart_reg_pkg::uart_hw2reg_t hw2reg, // Read
23
24     // RACL interface
25     input top_racl_pkg::racl_policy_vec_t racl_policies_i,
26     output top_racl_pkg::racl_error_log_t  racl_error_o,
27
28     // Integrity check errors
29     output logic intg_err_o
30 );
31
32 import uart_reg_pkg::* ;
33
34 localparam int AW = 6;
35 localparam int DW = 32;
36 localparam int DBW = DW/8; // Byte Width
37
38 // register signals
39 logic reg_we;
40 logic reg_re;
41 logic [AW-1:0] reg_addr;
42 logic [DW-1:0] reg_wdata;
43 logic [DBW-1:0] reg_be;
44 logic [DW-1:0] reg_rdata;
45 logic reg_error;
46
47 logic addrmiss, wr_err;

```

```

48
49 logic [DW-1:0] reg_rdata_next;
50 logic reg_busy;
51
52 tlul_pkg::tl_h2d_t tl_reg_h2d;
53 tlul_pkg::tl_d2h_t tl_reg_d2h;
54
55
56 // incoming payload check
57 logic intg_err;
58 tlul_cmd_intg_chk u_chk (
59     .tl_i(tl_i),
60     .err_o(intg_err)
61 );
62
63 // also check for spurious write enables
64 logic reg_we_err;
65 logic [12:0] reg_we_check;
66 prim_reg_we_check #(
67     .OneHotWidth(13)
68 ) u_prim_reg_we_check (
69     .clk_i(clk_i),
70     .rst_ni(rst_ni),
71     .oh_i (reg_we_check),
72     .en_i (reg_we && !addrmiss),
73     .err_o (reg_we_err)
74 );
75
76 logic err_q;
77 always_ff @(posedge clk_i or negedge rst_ni) begin
78     if (!rst_ni) begin
79         err_q <= '0;
80     end else if (intg_err || reg_we_err) begin
81         err_q <= 1'b1;
82     end
83 end
84
85 // integrity error output is permanent and should be used
  for alert generation
86 // register errors are transactional
87 assign intg_err_o = err_q | intg_err | reg_we_err;
88
89 // outgoing integrity generation
90 tlul_pkg::tl_d2h_t tl_o_pre;
91 tlul_rsp_intg_gen #(
92     .EnableRspIntgGen(1),
93     .EnableDataIntgGen(1)
94 ) u_rsp_intg_gen (
95     .tl_i(tl_o_pre),
96     .tl_o(tl_o)
97 );
98
99 assign tl_reg_h2d = tl_i;
100 assign tl_o_pre = tl_reg_d2h;
101
102 tlul_adapter_reg #(
103     .RegAw(AW),
104     .RegDw(DW),
105     .EnableDataIntgGen(0)
106 ) u_reg_if (
107     .clk_i (clk_i),
108     .rst_ni (rst_ni),
109
110     .tl_i (tl_reg_h2d),
111     .tl_o (tl_reg_d2h),
112
113     .en_ifetch_i(prim_mubi_pkg::MuBi4False),
114     .intg_error_o(),
115
116     .we_o (reg_we),
117     .re_o (reg_re),
118     .addr_o (reg_addr),
119     .wdata_o (reg_wdata),
120     .be_o (reg_be),
121     .busy_i (reg_busy),
122     .rdata_i (reg_rdata),
123     // Translate RACL error to TLUL error if enabled
124     .error_i (reg_error | (RaclErrorRsp & racl_error_o.valid))
125 );
126
127 // cdc oversampling signals
128
129 assign reg_rdata = reg_rdata_next ;
130 assign reg_error = addrmiss | wr_err | intg_err;
131
132 // Define SW related signals
133 // Format: <reg>_<field>_<wd|we|qs>
134 // or <reg>_<wd|we|qs> if field == 1 or 0
135 logic intr_state_we;
136 logic intr_state_tx_watermark_qs;
137 logic intr_state_rx_watermark_qs;
138 logic intr_state_tx_done_qs;
139 logic intr_state_tx_done_wd;
140 logic intr_state_rx_overflow_qs;
141 logic intr_state_rx_overflow_wd;
142 logic intr_state_rx_frame_err_qs;
143 logic intr_state_rx_frame_err_wd;
144 logic intr_state_rx_break_err_qs;
145 logic intr_state_rx_break_err_wd;
146 logic intr_state_rx_timeout_qs;
147 logic intr_state_rx_timeout_wd;

```

```

148 logic intr_state_rx_parity_err_qs;
149 logic intr_state_rx_parity_err_wd;
150 logic intr_state_tx_empty_qs;
151 logic intr_enable_we;
152 logic intr_enable_tx_watermark_qs;
153 logic intr_enable_tx_watermark_wd;
154 logic intr_enable_rx_watermark_qs;
155 logic intr_enable_rx_watermark_wd;
156 logic intr_enable_tx_done_qs;
157 logic intr_enable_tx_done_wd;
158 logic intr_enable_rx_overflow_qs;
159 logic intr_enable_rx_overflow_wd;
160 logic intr_enable_rx_frame_err_qs;
161 logic intr_enable_rx_frame_err_wd;
162 logic intr_enable_rx_break_err_qs;
163 logic intr_enable_rx_break_err_wd;
164 logic intr_enable_rx_timeout_qs;
165 logic intr_enable_rx_timeout_wd;
166 logic intr_enable_rx_parity_err_qs;
167 logic intr_enable_rx_parity_err_wd;
168 logic intr_enable_tx_empty_qs;
169 logic intr_enable_tx_empty_wd;
170 logic intr_test_we;
171 logic intr_test_tx_watermark_wd;
172 logic intr_test_rx_watermark_wd;
173 logic intr_test_tx_done_wd;
174 logic intr_test_rx_overflow_wd;
175 logic intr_test_rx_frame_err_wd;
176 logic intr_test_rx_break_err_wd;
177 logic intr_test_rx_timeout_wd;
178 logic intr_test_rx_parity_err_wd;
179 logic intr_test_tx_empty_wd;
180 logic alert_test_we;
181 logic alert_test_wd;
182 logic ctrl_we;
183 logic ctrl_tx_qs;
184 logic ctrl_tx_wd;
185 logic ctrl_rx_qs;
186 logic ctrl_rx_wd;
187 logic ctrl_nf_qs;
188 logic ctrl_nf_wd;
189 logic ctrl_slpbk_qs;
190 logic ctrl_slpbk_wd;
191 logic ctrl_llpbk_qs;
192 logic ctrl_llpbk_wd;
193 logic ctrl_parity_en_qs;
194 logic ctrl_parity_en_wd;
195 logic ctrl_parity_odd_qs;
196 logic ctrl_parity_odd_wd;
197 logic [1:0] ctrl_rxbvlv_qs;
198 logic [1:0] ctrl_rxbvlv_wd;
199 logic [15:0] ctrl_nco_qs;
200 logic [15:0] ctrl_nco_wd;
201 logic status_re;
202 logic status_txfull_qs;
203 logic status_rxfull_qs;
204 logic status_txempty_qs;
205 logic status_txidle_qs;
206 logic status_rxidle_qs;
207 logic status_rxempty_qs;
208 logic rdata_re;
209 logic [7:0] rdata_qs;
210 logic wdata_we;
211 logic [7:0] wdata_wd;
212 logic fifo_ctrl_we;
213 logic fifo_ctrl_rxrst_wd;
214 logic fifo_ctrl_txrst_wd;
215 logic [2:0] fifo_ctrl_rxilvl_qs;
216 logic [2:0] fifo_ctrl_rxilvl_wd;
217 logic [2:0] fifo_ctrl_txilvl_qs;
218 logic [2:0] fifo_ctrl_txilvl_wd;
219 logic fifo_status_re;
220 logic [7:0] fifo_status_txlvl_qs;
221 logic [7:0] fifo_status_rxlvl_qs;
222 logic ovrd_we;
223 logic ovrd_txen_qs;
224 logic ovrd_txen_wd;
225 logic ovrd_txval_qs;
226 logic ovrd_txval_wd;
227 logic val_re;
228 logic [15:0] val_qs;
229 logic timeout_ctrl_we;
230 logic [23:0] timeout_ctrl_val_qs;
231 logic [23:0] timeout_ctrl_val_wd;
232 logic timeout_ctrl_en_qs;
233 logic timeout_ctrl_en_wd;
234
235 // Register instances
236 // R[intr_state]: V(False)
237 // F[tx_watermark]: 0:0
238 prim_subreg #(
239     .DW (1),
240     .SWAccess(prim_subreg_pkg::SWAccessR0),
241     .RESVAL (1'h1),
242     .Mubi (1'b0)
243 ) u_intr_state_tx_watermark (
244     .clk_i (clk_i),
245     .rst_ni (rst_ni),
246
247     // from register interface
248     .we (1'b0),
249     .wd ('0'),
250
251     // from internal hardware
252     .de (hw2reg.intr_state.tx_watermark.de),
253     .d (hw2reg.intr_state.tx_watermark.d),
254
255     // to internal hardware
256     .qe (),
257     .q (reg2hw.intr_state.tx_watermark.q),
258     .ds (),
259
260     // to register interface (read)
261     .qs (intr_state_tx_watermark_qs)
262 );
263
264 // F[rx_watermark]: 1:1
265 prim_subreg #(
266     .DW (1),
267     .SWAccess(prim_subreg_pkg::SWAccessR0),
268     .RESVAL (1'h0),
269     .Mubi (1'b0)
270 ) u_intr_state_rx_watermark (
271     .clk_i (clk_i),
272     .rst_ni (rst_ni),
273
274     // from register interface
275     .we (1'b0),
276     .wd ('0'),
277
278     // from internal hardware
279     .de (hw2reg.intr_state.rx_watermark.de),
280     .d (hw2reg.intr_state.rx_watermark.d),
281
282     // to internal hardware
283     .qe (),
284     .q (reg2hw.intr_state.rx_watermark.q),
285     .ds (),
286
287     // to register interface (read)
288     .qs (intr_state_rx_watermark_qs)
289 );
290
291 // F[tx_done]: 2:2
292 prim_subreg #(
293     .DW (1),
294     .SWAccess(prim_subreg_pkg::SWAccessW1C),
295     .RESVAL (1'h0),
296     .Mubi (1'b0)
297 ) u_intr_state_tx_done (
298     .clk_i (clk_i),
299     .rst_ni (rst_ni),
300
301     // from register interface
302     .we (intr_state_we),
303     .wd (intr_state_tx_done_wd),
304
305     // from internal hardware
306     .de (hw2reg.intr_state.tx_done.de),
307     .d (hw2reg.intr_state.tx_done.d),
308
309     // to internal hardware
310     .qe (),
311     .q (reg2hw.intr_state.tx_done.q),
312     .ds (),
313
314     // to register interface (read)
315     .qs (intr_state_tx_done_qs)
316 );
317
318 // F[rx_overflow]: 3:3
319 prim_subreg #(
320     .DW (1),
321     .SWAccess(prim_subreg_pkg::SWAccessW1C),
322     .RESVAL (1'h0),
323     .Mubi (1'b0)
324 ) u_intr_state_rx_overflow (
325     .clk_i (clk_i),
326     .rst_ni (rst_ni),
327
328     // from register interface
329     .we (intr_state_we),
330     .wd (intr_state_rx_overflow_wd),
331
332     // from internal hardware
333     .de (hw2reg.intr_state.rx_overflow.de),
334     .d (hw2reg.intr_state.rx_overflow.d),
335
336     // to internal hardware
337     .qe (),
338     .q (reg2hw.intr_state.rx_overflow.q),
339     .ds (),
340
341     // to register interface (read)
342     .qs (intr_state_rx_overflow_qs)
343 );
344
345 // F[rx_frame_err]: 4:4
346 prim_subreg #(
347     .DW (1),
348     .SWAccess(prim_subreg_pkg::SWAccessW1C),
349     .RESVAL (1'h0),

```



```

350     .Mubi    (1'b0)
351 ) u_intr_state_rx_frame_err (
352     .clk_i    (clk_i),
353     .rst_ni   (rst_ni),
354
355     // from register interface
356     .we        (intr_state_we),
357     .wd        (intr_state_rx_frame_err_wd),
358
359     // from internal hardware
360     .de        (hw2reg.intr_state.rx_frame_err.de),
361     .d         (hw2reg.intr_state.rx_frame_err.d),
362
363     // to internal hardware
364     .qe        (),
365     .q         (reg2hw.intr_state.rx_frame_err.q),
366     .ds        (),
367
368     // to register interface (read)
369     .qs        (intr_state_rx_frame_err_qs)
370 );
371
372 // F[rx_break_err]: 5:5
373 prim_subreg #(
374     .DW        (1),
375     .SwAccess(prim_subreg_pkg::SwAccessW1C),
376     .RESVAL    (1'h0),
377     .Mubi      (1'b0)
378 ) u_intr_state_rx_break_err (
379     .clk_i    (clk_i),
380     .rst_ni   (rst_ni),
381
382     // from register interface
383     .we        (intr_state_we),
384     .wd        (intr_state_rx_break_err_wd),
385
386     // from internal hardware
387     .de        (hw2reg.intr_state.rx_break_err.de),
388     .d         (hw2reg.intr_state.rx_break_err.d),
389
390     // to internal hardware
391     .qe        (),
392     .q         (reg2hw.intr_state.rx_break_err.q),
393     .ds        (),
394
395     // to register interface (read)
396     .qs        (intr_state_rx_break_err_qs)
397 );
398
399 // F[rx_timeout]: 6:6
400 prim_subreg #(
401     .DW        (1),
402     .SwAccess(prim_subreg_pkg::SwAccessW1C),
403     .RESVAL    (1'h0),
404     .Mubi      (1'b0)
405 ) u_intr_state_rx_timeout (
406     .clk_i    (clk_i),
407     .rst_ni   (rst_ni),
408
409     // from register interface
410     .we        (intr_state_we),
411     .wd        (intr_state_rx_timeout_wd),
412
413     // from internal hardware
414     .de        (hw2reg.intr_state.rx_timeout.de),
415     .d         (hw2reg.intr_state.rx_timeout.d),
416
417     // to internal hardware
418     .qe        (),
419     .q         (reg2hw.intr_state.rx_timeout.q),
420     .ds        (),
421
422     // to register interface (read)
423     .qs        (intr_state_rx_timeout_qs)
424 );
425
426 // F[rx_parity_err]: 7:7
427 prim_subreg #(
428     .DW        (1),
429     .SwAccess(prim_subreg_pkg::SwAccessW1C),
430     .RESVAL    (1'h0),
431     .Mubi      (1'b0)
432 ) u_intr_state_rx_parity_err (
433     .clk_i    (clk_i),
434     .rst_ni   (rst_ni),
435
436     // from register interface
437     .we        (intr_state_we),
438     .wd        (intr_state_rx_parity_err_wd),
439
440     // from internal hardware
441     .de        (hw2reg.intr_state.rx_parity_err.de),
442     .d         (hw2reg.intr_state.rx_parity_err.d),
443
444     // to internal hardware
445     .qe        (),
446     .q         (reg2hw.intr_state.rx_parity_err.q),
447     .ds        (),
448
449     // to register interface (read)
450     .qs        (intr_state_rx_parity_err_qs)
451 );
452
453 // F[tx_empty]: 8:8
454 prim_subreg #(
455     .DW        (1),
456     .SwAccess(prim_subreg_pkg::SwAccessR0),
457     .RESVAL    (1'h1),
458     .Mubi      (1'b0)
459 ) u_intr_state_tx_empty (
460     .clk_i    (clk_i),
461     .rst_ni   (rst_ni),
462
463     // from register interface
464     .we        (1'b0),
465     .wd        ('0'),
466
467     // from internal hardware
468     .de        (hw2reg.intr_state.tx_empty.de),
469     .d         (hw2reg.intr_state.tx_empty.d),
470
471     // to internal hardware
472     .qe        (),
473     .q         (reg2hw.intr_state.tx_empty.q),
474     .ds        (),
475
476     // to register interface (read)
477     .qs        (intr_state_tx_empty_qs)
478 );
479
480 // R[intr_enable]: V(False)
481 // F[tx_watermark]: 0:0
482 prim_subreg #(
483     .DW        (1),
484     .SwAccess(prim_subreg_pkg::SwAccessRW),
485     .RESVAL    (1'h0),
486     .Mubi      (1'b0)
487 ) u_intr_enable_tx_watermark (
488     .clk_i    (clk_i),
489     .rst_ni   (rst_ni),
490
491     // from register interface
492     .we        (intr_enable_we),
493     .wd        (intr_enable_tx_watermark_wd),
494
495     // from internal hardware
496     .de        (1'b0),
497     .d         ('0'),
498
499     // to internal hardware
500     .qe        (),
501     .q         (reg2hw.intr_enable.tx_watermark.q),
502     .ds        (),
503
504     // to register interface (read)
505     .qs        (intr_enable_tx_watermark_qs)
506 );
507
508 // F[rx_watermark]: 1:1
509 prim_subreg #(
510     .DW        (1),
511     .SwAccess(prim_subreg_pkg::SwAccessRW),
512     .RESVAL    (1'h0),
513     .Mubi      (1'b0)
514 ) u_intr_enable_rx_watermark (
515     .clk_i    (clk_i),
516     .rst_ni   (rst_ni),
517
518     // from register interface
519     .we        (intr_enable_we),
520     .wd        (intr_enable_rx_watermark_wd),
521
522     // from internal hardware
523     .de        (1'b0),
524     .d         ('0'),
525
526     // to internal hardware
527     .qe        (),
528     .q         (reg2hw.intr_enable.rx_watermark.q),
529     .ds        (),
530
531     // to register interface (read)
532     .qs        (intr_enable_rx_watermark_qs)
533 );
534
535 // F[tx_done]: 2:2
536 prim_subreg #(
537     .DW        (1),
538     .SwAccess(prim_subreg_pkg::SwAccessRW),
539     .RESVAL    (1'h0),
540     .Mubi      (1'b0)
541 ) u_intr_enable_tx_done (
542     .clk_i    (clk_i),
543     .rst_ni   (rst_ni),
544
545     // from register interface
546     .we        (intr_enable_we),
547     .wd        (intr_enable_tx_done_wd),
548
549     // from internal hardware
550     .de        (1'b0),
551

```

```

552 .d ('0),
553
554 // to internal hardware
555 .qe (),
556 .q (reg2hw.intr_enable.tx_done.q),
557 .ds (),
558
559 // to register interface (read)
560 .qs (intr_enable_tx_done_qs)
561 );
562
563 // F[rx_overflow]: 3:3
564 prim_subreg #(
565 .DW (1),
566 .SwAccess(prim_subreg_pkg::SwAccessRW),
567 .RESVAL (1'h0),
568 .Mubi (1'b0)
569 ) u_intr_enable_rx_overflow (
570 .clk_i (clk_i),
571 .rst_ni (rst_ni),
572
573 // from register interface
574 .we (intr_enable_we),
575 .wd (intr_enable_rx_overflow_wd),
576
577 // from internal hardware
578 .de (1'b0),
579 .d ('0),
580
581 // to internal hardware
582 .qe (),
583 .q (reg2hw.intr_enable.rx_overflow.q),
584 .ds (),
585
586 // to register interface (read)
587 .qs (intr_enable_rx_overflow_qs)
588 );
589
590 // F[rx_frame_err]: 4:4
591 prim_subreg #(
592 .DW (1),
593 .SwAccess(prim_subreg_pkg::SwAccessRW),
594 .RESVAL (1'h0),
595 .Mubi (1'b0)
596 ) u_intr_enable_rx_frame_err (
597 .clk_i (clk_i),
598 .rst_ni (rst_ni),
599
600 // from register interface
601 .we (intr_enable_we),
602 .wd (intr_enable_rx_frame_err_wd),
603
604 // from internal hardware
605 .de (1'b0),
606 .d ('0),
607
608 // to internal hardware
609 .qe (),
610 .q (reg2hw.intr_enable.rx_frame_err.q),
611 .ds (),
612
613 // to register interface (read)
614 .qs (intr_enable_rx_frame_err_qs)
615 );
616
617 // F[rx_break_err]: 5:5
618 prim_subreg #(
619 .DW (1),
620 .SwAccess(prim_subreg_pkg::SwAccessRW),
621 .RESVAL (1'h0),
622 .Mubi (1'b0)
623 ) u_intr_enable_rx_break_err (
624 .clk_i (clk_i),
625 .rst_ni (rst_ni),
626
627 // from register interface
628 .we (intr_enable_we),
629 .wd (intr_enable_rx_break_err_wd),
630
631 // from internal hardware
632 .de (1'b0),
633 .d ('0),
634
635 // to internal hardware
636 .qe (),
637 .q (reg2hw.intr_enable.rx_break_err.q),
638 .ds (),
639
640 // to register interface (read)
641 .qs (intr_enable_rx_break_err_qs)
642 );
643
644 // F[rx_timeout]: 6:6
645 prim_subreg #(
646 .DW (1),
647 .SwAccess(prim_subreg_pkg::SwAccessRW),
648 .RESVAL (1'h0),
649 .Mubi (1'b0)
650 ) u_intr_enable_rx_timeout (
651 .clk_i (clk_i),
652 .rst_ni (rst_ni),
653
654 // from register interface
655 .we (intr_enable_we),
656 .wd (intr_enable_rx_timeout_wd),
657
658 // from internal hardware
659 .de (1'b0),
660 .d ('0),
661
662 // to internal hardware
663 .qe (),
664 .q (reg2hw.intr_enable.rx_timeout.q),
665 .ds (),
666
667 // to register interface (read)
668 .qs (intr_enable_rx_timeout_qs)
669 );
670
671 // F[rx_parity_err]: 7:7
672 prim_subreg #(
673 .DW (1),
674 .SwAccess(prim_subreg_pkg::SwAccessRW),
675 .RESVAL (1'h0),
676 .Mubi (1'b0)
677 ) u_intr_enable_rx_parity_err (
678 .clk_i (clk_i),
679 .rst_ni (rst_ni),
680
681 // from register interface
682 .we (intr_enable_we),
683 .wd (intr_enable_rx_parity_err_wd),
684
685 // from internal hardware
686 .de (1'b0),
687 .d ('0),
688
689 // to internal hardware
690 .qe (),
691 .q (reg2hw.intr_enable.rx_parity_err.q),
692 .ds (),
693
694 // to register interface (read)
695 .qs (intr_enable_rx_parity_err_qs)
696 );
697
698 // F[tx_empty]: 8:8
699 prim_subreg #(
700 .DW (1),
701 .SwAccess(prim_subreg_pkg::SwAccessRW),
702 .RESVAL (1'h0),
703 .Mubi (1'b0)
704 ) u_intr_enable_tx_empty (
705 .clk_i (clk_i),
706 .rst_ni (rst_ni),
707
708 // from register interface
709 .we (intr_enable_we),
710 .wd (intr_enable_tx_empty_wd),
711
712 // from internal hardware
713 .de (1'b0),
714 .d ('0),
715
716 // to internal hardware
717 .qe (),
718 .q (reg2hw.intr_enable.tx_empty.q),
719 .ds (),
720
721 // to register interface (read)
722 .qs (intr_enable_tx_empty_qs)
723 );
724
725
726 // R[intr_test]: V(True)
727 logic intr_test_qe;
728 logic [8:0] intr_test_flds_we;
729 assign intr_test_qe = &intr_test_flds_we;
730 // F[tx_watermark]: 0:0
731 prim_subreg_ext #(
732 .DW (1)
733 ) u_intr_test_tx_watermark (
734 .re (1'b0),
735 .we (intr_test_we),
736 .wd (intr_test_tx_watermark_wd),
737 .d ('0),
738 .qre (),
739 .qe (intr_test_flds_we[0]),
740 .q (reg2hw.intr_test.tx_watermark.q),
741 .ds (),
742 .qs ()
743 );
744 assign reg2hw.intr_test.tx_watermark.qe = intr_test_qe;
745
746 // F[rx_watermark]: 1:1
747 prim_subreg_ext #(
748 .DW (1)
749 ) u_intr_test_rx_watermark (
750 .re (1'b0),
751 .we (intr_test_we),
752 .wd (intr_test_rx_watermark_wd),
753 .d ('0),

```



```

754 .qre ( ),
755 .qe (intr_test_flds_we[1]),
756 .q (reg2hw.intr_test.rx_watermark.q),
757 .ds ( ),
758 .qs ( )
759 );
760 assign reg2hw.intr_test.rx_watermark.qe = intr_test_qe;
761
762 // F[tx_done]: 2:2
763 prim_subreg_ext #(
764 .DW (1)
765 ) u_intr_test_tx_done (
766 .re (1'b0),
767 .we (intr_test_we),
768 .wd (intr_test_tx_done_wd),
769 .d ('0'),
770 .qre ( ),
771 .qe (intr_test_flds_we[2]),
772 .q (reg2hw.intr_test.tx_done.q),
773 .ds ( ),
774 .qs ( )
775 );
776 assign reg2hw.intr_test.tx_done.qe = intr_test_qe;
777
778 // F[rx_overflow]: 3:3
779 prim_subreg_ext #(
780 .DW (1)
781 ) u_intr_test_rx_overflow (
782 .re (1'b0),
783 .we (intr_test_we),
784 .wd (intr_test_rx_overflow_wd),
785 .d ('0'),
786 .qre ( ),
787 .qe (intr_test_flds_we[3]),
788 .q (reg2hw.intr_test.rx_overflow.q),
789 .ds ( ),
790 .qs ( )
791 );
792 assign reg2hw.intr_test.rx_overflow.qe = intr_test_qe;
793
794 // F[rx_frame_err]: 4:4
795 prim_subreg_ext #(
796 .DW (1)
797 ) u_intr_test_rx_frame_err (
798 .re (1'b0),
799 .we (intr_test_we),
800 .wd (intr_test_rx_frame_err_wd),
801 .d ('0'),
802 .qre ( ),
803 .qe (intr_test_flds_we[4]),
804 .q (reg2hw.intr_test.rx_frame_err.q),
805 .ds ( ),
806 .qs ( )
807 );
808 assign reg2hw.intr_test.rx_frame_err.qe = intr_test_qe;
809
810 // F[rx_break_err]: 5:5
811 prim_subreg_ext #(
812 .DW (1)
813 ) u_intr_test_rx_break_err (
814 .re (1'b0),
815 .we (intr_test_we),
816 .wd (intr_test_rx_break_err_wd),
817 .d ('0'),
818 .qre ( ),
819 .qe (intr_test_flds_we[5]),
820 .q (reg2hw.intr_test.rx_break_err.q),
821 .ds ( ),
822 .qs ( )
823 );
824 assign reg2hw.intr_test.rx_break_err.qe = intr_test_qe;
825
826 // F[rx_timeout]: 6:6
827 prim_subreg_ext #(
828 .DW (1)
829 ) u_intr_test_rx_timeout (
830 .re (1'b0),
831 .we (intr_test_we),
832 .wd (intr_test_rx_timeout_wd),
833 .d ('0'),
834 .qre ( ),
835 .qe (intr_test_flds_we[6]),
836 .q (reg2hw.intr_test.rx_timeout.q),
837 .ds ( ),
838 .qs ( )
839 );
840 assign reg2hw.intr_test.rx_timeout.qe = intr_test_qe;
841
842 // F[rx_parity_err]: 7:7
843 prim_subreg_ext #(
844 .DW (1)
845 ) u_intr_test_rx_parity_err (
846 .re (1'b0),
847 .we (intr_test_we),
848 .wd (intr_test_rx_parity_err_wd),
849 .d ('0'),
850 .qre ( ),
851 .qe (intr_test_flds_we[7]),
852 .q (reg2hw.intr_test.rx_parity_err.q),
853 .ds ( ),
854 .qs ( )

```

```

855 );
856 assign reg2hw.intr_test.rx_parity_err.qe = intr_test_qe;
857
858 // F[tx_empty]: 8:8
859 prim_subreg_ext #(
860 .DW (1)
861 ) u_intr_test_tx_empty (
862 .re (1'b0),
863 .we (intr_test_we),
864 .wd (intr_test_tx_empty_wd),
865 .d ('0'),
866 .qre ( ),
867 .qe (intr_test_flds_we[8]),
868 .q (reg2hw.intr_test.tx_empty.q),
869 .ds ( ),
870 .qs ( )
871 );
872 assign reg2hw.intr_test.tx_empty.qe = intr_test_qe;
873
874 // R[alert_test]: V(True)
875 logic alert_test_qe;
876 logic [0:0] alert_test_flds_we;
877 assign alert_test_qe = &alert_test_flds_we;
878 prim_subreg_ext #(
879 .DW (1)
880 ) u_alert_test (
881 .re (1'b0),
882 .we (alert_test_we),
883 .wd (alert_test_wd),
884 .d ('0'),
885 .qre ( ),
886 .qe (alert_test_flds_we[0]),
887 .q (reg2hw.alert_test.q),
888 .ds ( ),
889 .qs ( )
890 );
891 assign reg2hw.alert_test.qe = alert_test_qe;
892
893 // R[ctrl]: V(False)
894 // F[tx]: 0:0
895 prim_subreg #(
896 .DW (1),
897 .SWAccess(prim_subreg_pkg::SWAccessRW),
898 .RESVAL (1'h0),
899 .Mubi (1'b0)
900 ) u_ctrl_tx (
901 .clk_i (clk_i),
902 .rst_ni (rst_ni),
903
904 // from register interface
905 .we (ctrl_we),
906 .wd (ctrl_tx_wd),
907
908 // from internal hardware
909 .de (1'b0),
910 .d ('0'),
911
912 // to internal hardware
913 .qe ( ),
914 .q (reg2hw.ctrl.tx.q),
915 .ds ( ),
916
917 // to register interface (read)
918 .qs (ctrl_tx_qs)
919 );
920
921 // F[rx]: 1:1
922 prim_subreg #(
923 .DW (1),
924 .SWAccess(prim_subreg_pkg::SWAccessRW),
925 .RESVAL (1'h0),
926 .Mubi (1'b0)
927 ) u_ctrl_rx (
928 .clk_i (clk_i),
929 .rst_ni (rst_ni),
930
931 // from register interface
932 .we (ctrl_we),
933 .wd (ctrl_rx_wd),
934
935 // from internal hardware
936 .de (1'b0),
937 .d ('0'),
938
939 // to internal hardware
940 .qe ( ),
941 .q (reg2hw.ctrl.rx.q),
942 .ds ( ),
943
944 // to register interface (read)
945 .qs (ctrl_rx_qs)
946 );
947
948 // F[nf]: 2:2
949 prim_subreg #(
950 .DW (1),
951 .SWAccess(prim_subreg_pkg::SWAccessRW),
952 .RESVAL (1'h0),
953 .Mubi (1'b0)

```

```

956 ) u_ctrl_nf (
957   .clk_i  (clk_i),
958   .rst_ni (rst_ni),
959
960   // from register interface
961   .we      (ctrl_we),
962   .wd      (ctrl_nf_wd),
963
964   // from internal hardware
965   .de      (1'b0),
966   .d        ('0'),
967
968   // to internal hardware
969   .qe      (),
970   .q        (reg2hw.ctrl_nf.q),
971   .ds      (),
972
973   // to register interface (read)
974   .qs      (ctrl_nf_qs)
975 );
976
977 // F[slpbk]: 4:4
978 prim_subreg #(
979   .DW      (1),
980   .SwAccess(prim_subreg_pkg::SwAccessRW),
981   .RESVAL  (1'h0),
982   .Mubi    (1'b0)
983 ) u_ctrl_slpbk (
984   .clk_i  (clk_i),
985   .rst_ni (rst_ni),
986
987   // from register interface
988   .we      (ctrl_we),
989   .wd      (ctrl_slpbk_wd),
990
991   // from internal hardware
992   .de      (1'b0),
993   .d        ('0'),
994
995   // to internal hardware
996   .qe      (),
997   .q        (reg2hw.ctrl_slpbk.q),
998   .ds      (),
999
1000  // to register interface (read)
1001  .qs      (ctrl_slpbk_qs)
1002 );
1003
1004 // F[llpbk]: 5:5
1005 prim_subreg #(
1006   .DW      (1),
1007   .SwAccess(prim_subreg_pkg::SwAccessRW),
1008   .RESVAL  (1'h0),
1009   .Mubi    (1'b0)
1010 ) u_ctrl_llpbk (
1011   .clk_i  (clk_i),
1012   .rst_ni (rst_ni),
1013
1014   // from register interface
1015   .we      (ctrl_we),
1016   .wd      (ctrl_llpbk_wd),
1017
1018   // from internal hardware
1019   .de      (1'b0),
1020   .d        ('0'),
1021
1022   // to internal hardware
1023   .qe      (),
1024   .q        (reg2hw.ctrl_llpbk.q),
1025   .ds      (),
1026
1027   // to register interface (read)
1028   .qs      (ctrl_llpbk_qs)
1029 );
1030
1031 // F[parity_en]: 6:6
1032 prim_subreg #(
1033   .DW      (1),
1034   .SwAccess(prim_subreg_pkg::SwAccessRW),
1035   .RESVAL  (1'h0),
1036   .Mubi    (1'b0)
1037 ) u_ctrl_parity_en (
1038   .clk_i  (clk_i),
1039   .rst_ni (rst_ni),
1040
1041   // from register interface
1042   .we      (ctrl_we),
1043   .wd      (ctrl_parity_en_wd),
1044
1045   // from internal hardware
1046   .de      (1'b0),
1047   .d        ('0'),
1048
1049   // to internal hardware
1050   .qe      (),
1051   .q        (reg2hw.ctrl_parity_en.q),
1052   .ds      (),
1053
1054   // to register interface (read)
1055   .qs      (ctrl_parity_en_qs)
1056 );

```

```

1057
1058 // F[parity_odd]: 7:7
1059 prim_subreg #(
1060   .DW      (1),
1061   .SwAccess(prim_subreg_pkg::SwAccessRW),
1062   .RESVAL  (1'h0),
1063   .Mubi    (1'b0)
1064 ) u_ctrl_parity_odd (
1065   .clk_i  (clk_i),
1066   .rst_ni (rst_ni),
1067
1068   // from register interface
1069   .we      (ctrl_we),
1070   .wd      (ctrl_parity_odd_wd),
1071
1072   // from internal hardware
1073   .de      (1'b0),
1074   .d        ('0'),
1075
1076   // to internal hardware
1077   .qe      (),
1078   .q        (reg2hw.ctrl_parity_odd.q),
1079   .ds      (),
1080
1081   // to register interface (read)
1082   .qs      (ctrl_parity_odd_qs)
1083 );
1084
1085 // F[rxblvl]: 9:8
1086 prim_subreg #(
1087   .DW      (2),
1088   .SwAccess(prim_subreg_pkg::SwAccessRW),
1089   .RESVAL  (2'h0),
1090   .Mubi    (1'b0)
1091 ) u_ctrl_rxblvl (
1092   .clk_i  (clk_i),
1093   .rst_ni (rst_ni),
1094
1095   // from register interface
1096   .we      (ctrl_we),
1097   .wd      (ctrl_rxblvl_wd),
1098
1099   // from internal hardware
1100   .de      (1'b0),
1101   .d        ('0'),
1102
1103   // to internal hardware
1104   .qe      (),
1105   .q        (reg2hw.ctrl_rxblvl.q),
1106   .ds      (),
1107
1108   // to register interface (read)
1109   .qs      (ctrl_rxblvl_qs)
1110 );
1111
1112 // F[nco]: 31:16
1113 prim_subreg #(
1114   .DW      (16),
1115   .SwAccess(prim_subreg_pkg::SwAccessRW),
1116   .RESVAL  (16'h0),
1117   .Mubi    (1'b0)
1118 ) u_ctrl_nco (
1119   .clk_i  (clk_i),
1120   .rst_ni (rst_ni),
1121
1122   // from register interface
1123   .we      (ctrl_we),
1124   .wd      (ctrl_nco_wd),
1125
1126   // from internal hardware
1127   .de      (1'b0),
1128   .d        ('0'),
1129
1130   // to internal hardware
1131   .qe      (),
1132   .q        (reg2hw.ctrl_nco.q),
1133   .ds      (),
1134
1135   // to register interface (read)
1136   .qs      (ctrl_nco_qs)
1137 );
1138
1139 // R[status]: V(True)
1140 // F[txfull]: 0:0
1141 prim_subreg_ext #(
1142   .DW      (1)
1143 ) u_status_txfull (
1144   .re      (status_re),
1145   .we      (1'b0),
1146   .wd      ('0'),
1147   .d        (hw2reg.status_txfull.d),
1148   .qre     (reg2hw.status_txfull.re),
1149   .qe      (),
1150   .q        (reg2hw.status_txfull.q),
1151   .ds      (),
1152   .qs      (status_txfull_qs)
1153 );
1154
1155 // F[rxfull]: 1:1
1156 prim_subreg_ext #(

```

```

1158 .DW (1)
1159 ) u_status_rxfull (
1160 .re (status_re),
1161 .we (1'b0),
1162 .wd ('0'),
1163 .d (hw2reg.status.rxfull.d),
1164 .qre (reg2hw.status.rxfull.re),
1165 .qe (),
1166 .q (reg2hw.status.rxfull.q),
1167 .ds (),
1168 .qs (status_rxfull_qs)
1169 );
1170
1171 // F[txempty]: 2:2
1172 prim_subreg_ext #(
1173 .DW (1)
1174 ) u_status_txempty (
1175 .re (status_re),
1176 .we (1'b0),
1177 .wd ('0'),
1178 .d (hw2reg.status.txempty.d),
1179 .qre (reg2hw.status.txempty.re),
1180 .qe (),
1181 .q (reg2hw.status.txempty.q),
1182 .ds (),
1183 .qs (status_txempty_qs)
1184 );
1185
1186 // F[txidle]: 3:3
1187 prim_subreg_ext #(
1188 .DW (1)
1189 ) u_status_txidle (
1190 .re (status_re),
1191 .we (1'b0),
1192 .wd ('0'),
1193 .d (hw2reg.status.txidle.d),
1194 .qre (reg2hw.status.txidle.re),
1195 .qe (),
1196 .q (reg2hw.status.txidle.q),
1197 .ds (),
1198 .qs (status_txidle_qs)
1199 );
1200
1201 // F[rxidle]: 4:4
1202 prim_subreg_ext #(
1203 .DW (1)
1204 ) u_status_rxidle (
1205 .re (status_re),
1206 .we (1'b0),
1207 .wd ('0'),
1208 .d (hw2reg.status.rxidle.d),
1209 .qre (reg2hw.status.rxidle.re),
1210 .qe (),
1211 .q (reg2hw.status.rxidle.q),
1212 .ds (),
1213 .qs (status_rxidle_qs)
1214 );
1215
1216 // F[rxempty]: 5:5
1217 prim_subreg_ext #(
1218 .DW (1)
1219 ) u_status_rxempty (
1220 .re (status_re),
1221 .we (1'b0),
1222 .wd ('0'),
1223 .d (hw2reg.status.rxempty.d),
1224 .qre (reg2hw.status.rxempty.re),
1225 .qe (),
1226 .q (reg2hw.status.rxempty.q),
1227 .ds (),
1228 .qs (status_rxempty_qs)
1229 );
1230
1231 // R[rdata]: V(True)
1232 prim_subreg_ext #(
1233 .DW (8)
1234 ) u_rdata (
1235 .re (rdata_re),
1236 .we (1'b0),
1237 .wd ('0'),
1238 .d (hw2reg.rdata.d),
1239 .qre (reg2hw.rdata.re),
1240 .qe (),
1241 .q (reg2hw.rdata.q),
1242 .ds (),
1243 .qs (rdata_qs)
1244 );
1245
1246 // R[wdata]: V(False)
1247 logic wdata_qe;
1248 logic [0:0] wdata_flds_we;
1249 prim_flop #(
1250 .Width(1),
1251 .ResetValue(0)
1252 ) u_wdata0_qe (
1253 .clk_i (clk_i),
1254 .rst_ni (rst_ni),
1255 .d_i (&wdata_flds_we),
1256 .q_o (wdata_qe)
1257 );
1258
1259 );
1260 prim_subreg #(
1261 .DW (8),
1262 .SwAccess(prim_subreg_pkg::SwAccessW0),
1263 .RESVAL (8'h0),
1264 .Mubi (1'b0)
1265 ) u_wdata (
1266 .clk_i (clk_i),
1267 .rst_ni (rst_ni),
1268
1269 // from register interface
1270 .we (wdata_we),
1271 .wd (wdata_wd),
1272
1273 // from internal hardware
1274 .de (1'b0),
1275 .d ('0'),
1276
1277 // to internal hardware
1278 .qe (wdata_flds_we[0]),
1279 .q (reg2hw.wdata.q),
1280 .ds (),
1281
1282 // to register interface (read)
1283 .qs ()
1284 );
1285 assign reg2hw.wdata.qe = wdata_qe;
1286
1287 // R[fifo_ctrl]: V(False)
1288 logic fifo_ctrl_qe;
1289 logic [3:0] fifo_ctrl_flds_we;
1290 prim_flop #(
1291 .Width(1),
1292 .ResetValue(0)
1293 ) u_fifo_ctrl0_qe (
1294 .clk_i (clk_i),
1295 .rst_ni (rst_ni),
1296 .d_i (&fifo_ctrl_flds_we),
1297 .q_o (fifo_ctrl_qe)
1298 );
1299 // F[rxrst]: 0:0
1300 prim_subreg #(
1301 .DW (1),
1302 .SwAccess(prim_subreg_pkg::SwAccessW0),
1303 .RESVAL (1'h0),
1304 .Mubi (1'b0)
1305 ) u_fifo_ctrl_rxrst (
1306 .clk_i (clk_i),
1307 .rst_ni (rst_ni),
1308
1309 // from register interface
1310 .we (fifo_ctrl_we),
1311 .wd (fifo_ctrl_rxrst_wd),
1312
1313 // from internal hardware
1314 .de (1'b0),
1315 .d ('0'),
1316
1317 // to internal hardware
1318 .qe (fifo_ctrl_flds_we[0]),
1319 .q (reg2hw.fifo_ctrl.rxrst.q),
1320 .ds (),
1321
1322 // to register interface (read)
1323 .qs ()
1324 );
1325 assign reg2hw.fifo_ctrl.rxrst.qe = fifo_ctrl_qe;
1326
1327 // F[txrst]: 1:1
1328 prim_subreg #(
1329 .DW (1),
1330 .SwAccess(prim_subreg_pkg::SwAccessW0),
1331 .RESVAL (1'h0),
1332 .Mubi (1'b0)
1333 ) u_fifo_ctrl_txrst (
1334 .clk_i (clk_i),
1335 .rst_ni (rst_ni),
1336
1337 // from register interface
1338 .we (fifo_ctrl_we),
1339 .wd (fifo_ctrl_txrst_wd),
1340
1341 // from internal hardware
1342 .de (1'b0),
1343 .d ('0'),
1344
1345 // to internal hardware
1346 .qe (fifo_ctrl_flds_we[1]),
1347 .q (reg2hw.fifo_ctrl.txrst.q),
1348 .ds (),
1349
1350 // to register interface (read)
1351 .qs ()
1352 );
1353 assign reg2hw.fifo_ctrl.txrst.qe = fifo_ctrl_qe;
1354
1355 // F[rxilvl]: 4:2
1356 prim_subreg #(
1357 .DW (3),
1358 .SwAccess(prim_subreg_pkg::SwAccessRW),

```

```

1360 .RESVAL (3'h0),
1361 .Mubi (1'b0)
1362 ) u_fifo_ctrl_rxilvl (
1363 .clk_i (clk_i),
1364 .rst_ni (rst_ni),
1365
1366 // from register interface
1367 .we (fifo_ctrl_we),
1368 .wd (fifo_ctrl_rxilvl_wd),
1369
1370 // from internal hardware
1371 .de (hw2reg.fifo_ctrl_rxilvl.de),
1372 .d (hw2reg.fifo_ctrl_rxilvl.d),
1373
1374 // to internal hardware
1375 .qe (fifo_ctrl_flds_we[2]),
1376 .q (reg2hw.fifo_ctrl_rxilvl.q),
1377 .ds (()),
1378
1379 // to register interface (read)
1380 .qs (fifo_ctrl_rxilvl_qs)
1381 );
1382 assign reg2hw.fifo_ctrl_rxilvl.qe = fifo_ctrl_qe;
1383
1384 // F[txilvl]: 7:5
1385 prim_subreg #(
1386 .DW (3),
1387 .SwAccess(prim_subreg_pkg::SwAccessRW),
1388 .RESVAL (3'h0),
1389 .Mubi (1'b0)
1390 ) u_fifo_ctrl_txilvl (
1391 .clk_i (clk_i),
1392 .rst_ni (rst_ni),
1393
1394 // from register interface
1395 .we (fifo_ctrl_we),
1396 .wd (fifo_ctrl_txilvl_wd),
1397
1398 // from internal hardware
1399 .de (hw2reg.fifo_ctrl_txilvl.de),
1400 .d (hw2reg.fifo_ctrl_txilvl.d),
1401
1402 // to internal hardware
1403 .qe (fifo_ctrl_flds_we[3]),
1404 .q (reg2hw.fifo_ctrl_txilvl.q),
1405 .ds (()),
1406
1407 // to register interface (read)
1408 .qs (fifo_ctrl_txilvl_qs)
1409 );
1410 assign reg2hw.fifo_ctrl_txilvl.qe = fifo_ctrl_qe;
1411
1412 // R[fifo_status]: V(True)
1413 // F[txilvl]: 7:0
1414 prim_subreg_ext #(
1415 .DW (8)
1416 ) u_fifo_status_txilvl (
1417 .re (fifo_status_re),
1418 .we (1'b0),
1419 .wd (1'b0),
1420 .d (hw2reg.fifo_status_txilvl.d),
1421 .qre (()),
1422 .qe (()),
1423 .q (()),
1424 .ds (()),
1425 .qs (fifo_status_txilvl_qs)
1426 );
1427
1428 // F[rxilvl]: 23:16
1429 prim_subreg_ext #(
1430 .DW (8)
1431 ) u_fifo_status_rxilvl (
1432 .re (fifo_status_re),
1433 .we (1'b0),
1434 .wd (1'b0),
1435 .d (hw2reg.fifo_status_rxilvl.d),
1436 .qre (()),
1437 .qe (()),
1438 .q (()),
1439 .ds (()),
1440 .qs (fifo_status_rxilvl_qs)
1441 );
1442
1443 // R[ovrd]: V(False)
1444 // F[txen]: 0:0
1445 prim_subreg #(
1446 .DW (1),
1447 .SwAccess(prim_subreg_pkg::SwAccessRW),
1448 .RESVAL (1'h0),
1449 .Mubi (1'b0)
1450 ) u_ovrd_txen (
1451 .clk_i (clk_i),
1452 .rst_ni (rst_ni),
1453
1454 // from register interface
1455 .we (ovrd_we),
1456 .wd (ovrd_txen_wd),
1457
1458 // from internal hardware

```

```

1461 .de (1'b0),
1462 .d (1'b0),
1463
1464 // to internal hardware
1465 .qe (()),
1466 .q (reg2hw.ovrd_txen.q),
1467 .ds (()),
1468
1469 // to register interface (read)
1470 .qs (ovrd_txen_qs)
1471 );
1472
1473 // F[txval]: 1:1
1474 prim_subreg #(
1475 .DW (1),
1476 .SwAccess(prim_subreg_pkg::SwAccessRW),
1477 .RESVAL (1'h0),
1478 .Mubi (1'b0)
1479 ) u_ovrd_txval (
1480 .clk_i (clk_i),
1481 .rst_ni (rst_ni),
1482
1483 // from register interface
1484 .we (ovrd_we),
1485 .wd (ovrd_txval_wd),
1486
1487 // from internal hardware
1488 .de (1'b0),
1489 .d (1'b0),
1490
1491 // to internal hardware
1492 .qe (()),
1493 .q (reg2hw.ovrd_txval.q),
1494 .ds (()),
1495
1496 // to register interface (read)
1497 .qs (ovrd_txval_qs)
1498 );
1499
1500 // R[val]: V(True)
1501 prim_subreg_ext #(
1502 .DW (16)
1503 ) u_val (
1504 .re (val_re),
1505 .we (1'b0),
1506 .wd (1'b0),
1507 .d (hw2reg.val.d),
1508 .qre (()),
1509 .qe (()),
1510 .q (()),
1511 .ds (()),
1512 .qs (val_qs)
1513 );
1514
1515 // R[timeout_ctrl]: V(False)
1516 // F[val]: 23:0
1517 prim_subreg #(
1518 .DW (24),
1519 .SwAccess(prim_subreg_pkg::SwAccessRW),
1520 .RESVAL (24'h0),
1521 .Mubi (1'b0)
1522 ) u_timeout_ctrl_val (
1523 .clk_i (clk_i),
1524 .rst_ni (rst_ni),
1525
1526 // from register interface
1527 .we (timeout_ctrl_we),
1528 .wd (timeout_ctrl_val_wd),
1529
1530 // from internal hardware
1531 .de (1'b0),
1532 .d (1'b0),
1533
1534 // to internal hardware
1535 .qe (()),
1536 .q (reg2hw.timeout_ctrl_val.q),
1537 .ds (()),
1538
1539 // to register interface (read)
1540 .qs (timeout_ctrl_val_qs)
1541 );
1542
1543 // F[en]: 31:31
1544 prim_subreg #(
1545 .DW (1),
1546 .SwAccess(prim_subreg_pkg::SwAccessRW),
1547 .RESVAL (1'h0),
1548 .Mubi (1'b0)
1549 ) u_timeout_ctrl_en (
1550 .clk_i (clk_i),
1551 .rst_ni (rst_ni),
1552
1553 // from register interface
1554 .we (timeout_ctrl_we),
1555 .wd (timeout_ctrl_en_wd),
1556
1557 // from internal hardware
1558 .de (1'b0),
1559 .d (1'b0),

```

```

1562
1563 // to internal hardware
1564 .qe ( ),
1565 .q (reg2hw.timeout_ctrl.en.q),
1566 .ds ( ),
1567
1568 // to register interface (read)
1569 .qs (timeout_ctrl.en_qs)
1570 );
1571
1572
1573
1574 logic [12:0] addr_hit;
1575 top_racl_pkg::racl_role_vec_t racl_role_vec;
1576 top_racl_pkg::racl_role_t racl_role;
1577
1578 logic [12:0] racl_addr_hit_read;
1579 logic [12:0] racl_addr_hit_write;
1580
1581 if (EnableRacl) begin : gen_racl_role_logic
1582 // Retrieve RACL role from user bits and one-hot encode
1583 that for the comparison bitmap
1584 assign racl_role =
1585 top_racl_pkg::tlul_extract_racl_role_bits(tl_i.a.user.rsvd);
1586
1587 prim_onehot_enc #(
1588 .OneHotWidth( $bits(top_racl_pkg::racl_role_vec_t) )
1589 ) u_racl_role_encode (
1590 .in_i ( racl_role ),
1591 .en_i ( 1'b1 ),
1592 .out_o( racl_role_vec )
1593 );
1594 end else begin : gen_no_racl_role_logic
1595 assign racl_role = '0;
1596 assign racl_role_vec = '0;
1597 end
1598
1599 always_comb begin
1600 racl_addr_hit_read = '0;
1601 racl_addr_hit_write = '0;
1602 addr_hit[ 0] = (reg_addr == UART_INTR_STATE_OFFSET);
1603 addr_hit[ 1] = (reg_addr == UART_INTR_ENABLE_OFFSET);
1604 addr_hit[ 2] = (reg_addr == UART_INTR_TEST_OFFSET);
1605 addr_hit[ 3] = (reg_addr == UART_ALERT_TEST_OFFSET);
1606 addr_hit[ 4] = (reg_addr == UART_CTRL_OFFSET);
1607 addr_hit[ 5] = (reg_addr == UART_STATUS_OFFSET);
1608 addr_hit[ 6] = (reg_addr == UART_RDATA_OFFSET);
1609 addr_hit[ 7] = (reg_addr == UART_WDATA_OFFSET);
1610 addr_hit[ 8] = (reg_addr == UART_FIFO_CTRL_OFFSET);
1611 addr_hit[ 9] = (reg_addr == UART_FIFO_STATUS_OFFSET);
1612 addr_hit[10] = (reg_addr == UART_OVRD_OFFSET);
1613 addr_hit[11] = (reg_addr == UART_VAL_OFFSET);
1614 addr_hit[12] = (reg_addr == UART_TIMEOUT_CTRL_OFFSET);
1615
1616 if (EnableRacl) begin : gen_racl_hit
1617 for (int unsigned slice_idx = 0; slice_idx < 13;
1618 slice_idx++) begin
1619 racl_addr_hit_read[slice_idx] =
1620 addr_hit[slice_idx] & (|
1621 (racl_policies_i[RaclPolicySelVec[slice_idx]].read_perm
1622 & racl_role_vec));
1623 racl_addr_hit_write[slice_idx] =
1624 addr_hit[slice_idx] & (|
1625 (racl_policies_i[RaclPolicySelVec[slice_idx]].write_perm
1626 & racl_role_vec));
1627 end
1628 end else begin : gen_no_racl
1629 racl_addr_hit_read = addr_hit;
1630 racl_addr_hit_write = addr_hit;
1631 end
1632 end
1633
1634 assign addrmiss = (reg_re || reg_we) ? ~|addr_hit : 1'b0 ;
1635 // A valid address hit, access, but failed the RACL check
1636 assign racl_error_o.valid = |addr_hit & ((reg_re & ~|
1637 racl_addr_hit_read) |
1638 (reg_we & ~|
1639 racl_addr_hit_write));
1640 assign racl_error_o.request_address =
1641 top_pkg::TL_AW'(reg_addr);
1642 assign racl_error_o.racl_role = racl_role;
1643 assign racl_error_o.overflow = 1'b0;
1644
1645 if (EnableRacl) begin : gen_racl_log
1646 assign racl_error_o.ctn_uid =
1647 top_racl_pkg::tlul_extract_ctn_uid_bits(tl_i.a.user.rsvd);
1648 assign racl_error_o.read_access = tl_i.a.opcode ==
1649 tlul_pkg::Get;
1650 end else begin : gen_no_racl_log
1651 assign racl_error_o.ctn_uid = '0;
1652 assign racl_error_o.read_access = 1'b0;
1653 end
1654
1655 // Check sub-word write is permitted
1656 always_comb begin
1657 wr_err = (reg_we &
1658 ((racl_addr_hit_write[ 0] & (|(UART_PERMIT[ 0] &
1659 ~reg_be))) |

```

```

~reg_be))) |
1660 (racl_addr_hit_write[ 2] & (|(UART_PERMIT[ 2] &
~reg_be))) |
1661 (racl_addr_hit_write[ 3] & (|(UART_PERMIT[ 3] &
~reg_be))) |
1662 (racl_addr_hit_write[ 4] & (|(UART_PERMIT[ 4] &
~reg_be))) |
1663 (racl_addr_hit_write[ 5] & (|(UART_PERMIT[ 5] &
~reg_be))) |
1664 (racl_addr_hit_write[ 6] & (|(UART_PERMIT[ 6] &
~reg_be))) |
1665 (racl_addr_hit_write[ 7] & (|(UART_PERMIT[ 7] &
~reg_be))) |
1666 (racl_addr_hit_write[ 8] & (|(UART_PERMIT[ 8] &
~reg_be))) |
1667 (racl_addr_hit_write[ 9] & (|(UART_PERMIT[ 9] &
~reg_be))) |
1668 (racl_addr_hit_write[10] & (|(UART_PERMIT[10] &
~reg_be))) |
1669 (racl_addr_hit_write[11] & (|(UART_PERMIT[11] &
~reg_be))) |
1670 (racl_addr_hit_write[12] & (|(UART_PERMIT[12] &
~reg_be)))));
1671 end
1672 // Generate write-enables
1673 assign intr_state_we = racl_addr_hit_write[0] & reg_we & !
reg_error;
1674 assign intr_state_tx_done_wd = reg_wdata[2];
1675 assign intr_state_rx_overflow_wd = reg_wdata[3];
1676 assign intr_state_rx_frame_err_wd = reg_wdata[4];
1677 assign intr_state_rx_break_err_wd = reg_wdata[5];
1678 assign intr_state_rx_timeout_wd = reg_wdata[6];
1679 assign intr_state_rx_parity_err_wd = reg_wdata[7];
1680 assign intr_enable_we = racl_addr_hit_write[1] & reg_we & !
reg_error;
1681 assign intr_enable_tx_watermark_wd = reg_wdata[0];
1682 assign intr_enable_rx_watermark_wd = reg_wdata[1];
1683 assign intr_enable_tx_done_wd = reg_wdata[2];
1684 assign intr_enable_rx_overflow_wd = reg_wdata[3];
1685 assign intr_enable_rx_frame_err_wd = reg_wdata[4];
1686 assign intr_enable_rx_break_err_wd = reg_wdata[5];
1687 assign intr_enable_rx_timeout_wd = reg_wdata[6];
1688 assign intr_enable_rx_parity_err_wd = reg_wdata[7];
1689 assign intr_enable_tx_empty_wd = reg_wdata[8];
1690 assign intr_test_we = racl_addr_hit_write[2] & reg_we & !
reg_error;
1691 assign intr_test_tx_watermark_wd = reg_wdata[0];
1692 assign intr_test_rx_watermark_wd = reg_wdata[1];
1693 assign intr_test_tx_done_wd = reg_wdata[2];
1694 assign intr_test_rx_overflow_wd = reg_wdata[3];
1695 assign intr_test_rx_frame_err_wd = reg_wdata[4];
1696 assign intr_test_rx_break_err_wd = reg_wdata[5];
1697 assign intr_test_rx_timeout_wd = reg_wdata[6];
1698 assign intr_test_rx_parity_err_wd = reg_wdata[7];
1699 assign intr_test_tx_empty_wd = reg_wdata[8];
1700 assign alert_test_we = racl_addr_hit_write[3] & reg_we & !
reg_error;
1701 assign alert_test_wd = reg_wdata[0];
1702 assign ctrl_we = racl_addr_hit_write[4] & reg_we & !reg_er-
ror;
1703 assign ctrl_tx_wd = reg_wdata[0];
1704 assign ctrl_rx_wd = reg_wdata[1];

```

```

1723
1724 assign ctrl_nf_wd = reg_wdata[2];
1725
1726 assign ctrl_slpbk_wd = reg_wdata[4];
1727
1728 assign ctrl_llpbk_wd = reg_wdata[5];
1729
1730 assign ctrl_parity_en_wd = reg_wdata[6];
1731
1732 assign ctrl_parity_odd_wd = reg_wdata[7];
1733
1734 assign ctrl_rxbvl_wd = reg_wdata[9:8];
1735
1736 assign ctrl_nco_wd = reg_wdata[31:16];
1737 assign status_re = racl_addr_hit_read[5] & reg_re & !reg_er-
ror;
1738 assign rdata_re = racl_addr_hit_read[6] & reg_re & !reg_er-
ror;
1739 assign wdata_we = racl_addr_hit_write[7] & reg_we & !reg_er-
ror;
1740
1741 assign wdata_wd = reg_wdata[7:0];
1742 assign fifo_ctrl_we = racl_addr_hit_write[8] & reg_we & !
reg_error;
1743
1744 assign fifo_ctrl_rxrst_wd = reg_wdata[0];
1745
1746 assign fifo_ctrl_txrst_wd = reg_wdata[1];
1747
1748 assign fifo_ctrl_rxilvl_wd = reg_wdata[4:2];
1749
1750 assign fifo_ctrl_txilvl_wd = reg_wdata[7:5];
1751 assign fifo_status_re = racl_addr_hit_read[9] & reg_re & !
reg_error;
1752 assign ovrd_we = racl_addr_hit_write[10] & reg_we & !reg_er-
ror;
1753
1754 assign ovrd_txen_wd = reg_wdata[0];
1755
1756 assign ovrd_txval_wd = reg_wdata[1];
1757 assign val_re = racl_addr_hit_read[11] & reg_re & !reg_er-
ror;
1758 assign timeout_ctrl_we = racl_addr_hit_write[12] & reg_we
& !reg_error;
1759
1760 assign timeout_ctrl_val_wd = reg_wdata[23:0];
1761
1762 assign timeout_ctrl_en_wd = reg_wdata[31];
1763
1764 // Assign write-enables to checker logic vector.
1765 always_comb begin
1766   reg_we_check[0] = intr_state_we;
1767   reg_we_check[1] = intr_enable_we;
1768   reg_we_check[2] = intr_test_we;
1769   reg_we_check[3] = alert_test_we;
1770   reg_we_check[4] = ctrl_we;
1771   reg_we_check[5] = 1'b0;
1772   reg_we_check[6] = 1'b0;
1773   reg_we_check[7] = wdata_we;
1774   reg_we_check[8] = fifo_ctrl_we;
1775   reg_we_check[9] = 1'b0;
1776   reg_we_check[10] = ovrd_we;
1777   reg_we_check[11] = 1'b0;
1778   reg_we_check[12] = timeout_ctrl_we;
1779 end
1780
1781 // Read data return
1782 always_comb begin
1783   reg_rdata_next = '0;
1784   unique case (1'b1)
1785     racl_addr_hit_read[0]: begin
1786       reg_rdata_next[0] = intr_state_tx_watermark_qs;
1787       reg_rdata_next[1] = intr_state_rx_watermark_qs;
1788       reg_rdata_next[2] = intr_state_tx_done_qs;
1789       reg_rdata_next[3] = intr_state_rx_overflow_qs;
1790       reg_rdata_next[4] = intr_state_rx_frame_err_qs;
1791       reg_rdata_next[5] = intr_state_rx_break_err_qs;
1792       reg_rdata_next[6] = intr_state_rx_timeout_qs;
1793       reg_rdata_next[7] = intr_state_rx_parity_err_qs;
1794       reg_rdata_next[8] = intr_state_tx_empty_qs;
1795     end
1796     racl_addr_hit_read[1]: begin
1797       reg_rdata_next[0] = intr_enable_tx_watermark_qs;
1798       reg_rdata_next[1] = intr_enable_rx_watermark_qs;
1799       reg_rdata_next[2] = intr_enable_tx_done_qs;
1800       reg_rdata_next[3] = intr_enable_rx_overflow_qs;
1801       reg_rdata_next[4] = intr_enable_rx_frame_err_qs;
1802       reg_rdata_next[5] = intr_enable_rx_break_err_qs;
1803       reg_rdata_next[6] = intr_enable_rx_timeout_qs;
1804       reg_rdata_next[7] = intr_enable_rx_parity_err_qs;
1805       reg_rdata_next[8] = intr_enable_tx_empty_qs;
1806     end
1807   end
1808
1809   racl_addr_hit_read[2]: begin
1810     reg_rdata_next[0] = '0;
1811     reg_rdata_next[1] = '0;
1812     reg_rdata_next[2] = '0;
1813     reg_rdata_next[3] = '0;
1814     reg_rdata_next[4] = '0;
1815     reg_rdata_next[5] = '0;
1816     reg_rdata_next[6] = '0;
1817     reg_rdata_next[7] = '0;
1818     reg_rdata_next[8] = '0;
1819   end
1820
1821   racl_addr_hit_read[3]: begin
1822     reg_rdata_next[0] = '0;
1823   end
1824
1825   racl_addr_hit_read[4]: begin
1826     reg_rdata_next[0] = ctrl_tx_qs;
1827     reg_rdata_next[1] = ctrl_rx_qs;
1828     reg_rdata_next[2] = ctrl_nf_qs;
1829     reg_rdata_next[4] = ctrl_slpbk_qs;
1830     reg_rdata_next[5] = ctrl_llpbk_qs;
1831     reg_rdata_next[6] = ctrl_parity_en_qs;
1832     reg_rdata_next[7] = ctrl_parity_odd_qs;
1833     reg_rdata_next[9:8] = ctrl_rxbvl_qs;
1834     reg_rdata_next[31:16] = ctrl_nco_qs;
1835   end
1836
1837   racl_addr_hit_read[5]: begin
1838     reg_rdata_next[0] = status_txfull_qs;
1839     reg_rdata_next[1] = status_rxfull_qs;
1840     reg_rdata_next[2] = status_txempty_qs;
1841     reg_rdata_next[3] = status_txidle_qs;
1842     reg_rdata_next[4] = status_rxidle_qs;
1843     reg_rdata_next[5] = status_rxempty_qs;
1844   end
1845
1846   racl_addr_hit_read[6]: begin
1847     reg_rdata_next[7:0] = rdata_qs;
1848   end
1849
1850   racl_addr_hit_read[7]: begin
1851     reg_rdata_next[7:0] = '0;
1852   end
1853
1854   racl_addr_hit_read[8]: begin
1855     reg_rdata_next[0] = '0;
1856     reg_rdata_next[1] = '0;
1857     reg_rdata_next[4:2] = fifo_ctrl_rxilvl_qs;
1858     reg_rdata_next[7:5] = fifo_ctrl_txilvl_qs;
1859   end
1860
1861   racl_addr_hit_read[9]: begin
1862     reg_rdata_next[7:0] = fifo_status_txilvl_qs;
1863     reg_rdata_next[23:16] = fifo_status_rxilvl_qs;
1864   end
1865
1866   racl_addr_hit_read[10]: begin
1867     reg_rdata_next[0] = ovrd_txen_qs;
1868     reg_rdata_next[1] = ovrd_txval_qs;
1869   end
1870
1871   racl_addr_hit_read[11]: begin
1872     reg_rdata_next[15:0] = val_qs;
1873   end
1874
1875   racl_addr_hit_read[12]: begin
1876     reg_rdata_next[23:0] = timeout_ctrl_val_qs;
1877     reg_rdata_next[31] = timeout_ctrl_en_qs;
1878   end
1879
1880   default: begin
1881     reg_rdata_next = '1;
1882   end
1883 endcase
1884 end
1885
1886 // shadow busy
1887 logic shadow_busy;
1888 assign shadow_busy = 1'b0;
1889
1890 // register busy
1891 assign reg_busy = shadow_busy;
1892
1893 // Unused signal tieoff
1894
1895 // wdata / byte enable are not always fully used
1896 // add a blanket unused statement to handle lint waivers
1897 logic unused_wdata;
1898 assign unused_wdata = ^reg_wdata;
1899 logic unused_be;
1900 assign unused_be = ^reg_be;
1901 logic unused_policy_sel;
1902 assign unused_policy_sel = ^racl_policies_i;
1903
1904 // Assertions for Register Interface
1905 `ASSERT_PULSE(wePulse, reg_we, clk_i, !rst_ni)
1906 `ASSERT_PULSE(rePulse, reg_re, clk_i, !rst_ni)
1907
1908 `ASSERT(reAfterRv, $rose(reg_re || reg_we) |>
t1_o_pre.d_valid, clk_i, !rst_ni)
1909
1910 `ASSERT(en2addrHit, (reg_we || reg_re) |> $one-
hot0(addr_hit), clk_i, !rst_ni)
1911
1912 // this is formulated as an assumption such that the FPV
testbenches do disprove this
1913 // property by mistake

```



```

1914 //`ASSUME(reqParity, tl_reg_h2d.a_valid |->
tl_reg_h2d.a_user.chk_en == tlul_pkg::CheckDis)
1915
1916 endmodule

```

5. lowRISC_opentitan/hw/ip/uart/rtl/uart_rx.sv (105 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: UART Receive Module
6 //
7
8 module uart_rx (
9     input          clk_i,
10    input          rst_ni,
11
12    input          rx_enable,
13    input          tick_baud_x16,
14    input          parity_enable,
15    input          parity_odd,
16
17    output logic    tick_baud,
18    output logic    rx_valid,
19    output [7:0]    rx_data,
20    output logic    idle,
21    output          frame_err,
22    output          rx_parity_err,
23
24    input          rx
25 );
26
27 logic [10:0] sreg_q, sreg_d;
28 logic [3:0] bit_cnt_q, bit_cnt_d;
29 logic [3:0] baud_div_q, baud_div_d;
30 logic [3:0] tick_baud_q, tick_baud_d;
31 logic [3:0] idle_q, idle_d;
32
33 assign tick_baud = tick_baud_q;
34 assign idle = idle_q;
35
36 always_ff @(posedge clk_i or negedge rst_ni) begin
37     if (!rst_ni) begin
38         sreg_q <= 11'h0;
39         bit_cnt_q <= 4'h0;
40         baud_div_q <= 4'h0;
41         tick_baud_q <= 1'b0;
42         idle_q <= 1'b1;
43     end else begin
44         sreg_q <= sreg_d;
45         bit_cnt_q <= bit_cnt_d;
46         baud_div_q <= baud_div_d;
47         tick_baud_q <= tick_baud_d;
48         idle_q <= idle_d;
49     end
50 end
51
52 always_comb begin
53     if (!rx_enable) begin
54         sreg_d = 11'h0;
55         bit_cnt_d = 4'h0;
56         baud_div_d = 4'h0;
57         tick_baud_d = 1'b0;
58         idle_d = 1'b1;
59     end else begin
60         tick_baud_d = 1'b0;
61         sreg_d = sreg_q;
62         bit_cnt_d = bit_cnt_q;
63         baud_div_d = baud_div_q;
64         idle_d = idle_q;
65         if (tick_baud_x16) begin
66             {tick_baud_d, baud_div_d} = {1'b0, baud_div_q} + 5'h1;
67         end
68     end
69
70     if (idle_q && !rx) begin
71         // start of char, sample in the middle of the bit time
72         baud_div_d = 4'd8;
73         tick_baud_d = 1'b0;
74         bit_cnt_d = (parity_enable ? 4'd11 : 4'd10);
75         sreg_d = 11'h0;
76         idle_d = 1'b0;
77     end else if (!idle_q && tick_baud_q) begin
78         if ((bit_cnt_q == (parity_enable ? 4'd11 : 4'd10)) &&
79 rx) begin
80             // must have been a glitch on the input, start bit
81             // in the middle of the bit time, abort
82             idle_d = 1'b1;
83             bit_cnt_d = 4'h0;
84         end else begin
85             sreg_d = {rx, sreg_q[10:1]};
86             bit_cnt_d = bit_cnt_q - 4'h1;
87             idle_d = (bit_cnt_q == 4'h1);
88         end
89     end
90 end

```

```

90 end
91
92 always_ff @(posedge clk_i or negedge rst_ni) begin
93     if (!rst_ni) rx_valid_q <= 1'b0;
94     else rx_valid_q <= tick_baud_q & (bit_cnt_q ==
4'h1);
95
96 end
97
98 assign rx_valid = rx_valid_q;
99 assign rx_data = parity_enable ? sreg_q[8:1] :
sreg_q[9:2];
100 // (rx_parity = sreg_q[9])
101 assign frame_err = rx_valid_q & ~sreg_q[10];
102 assign rx_parity_err = parity_enable & rx_valid_q &
(^{sreg_q[9:1], parity_odd});
103
104
105 endmodule

```

6. lowRISC_opentitan/hw/ip/uart/rtl/uart_tx.sv (79 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: UART Transmit Module
6 //
7
8 module uart_tx (
9     input          clk_i,
10    input          rst_ni,
11
12    input          tx_enable,
13    input          tick_baud_x16,
14    input logic     parity_enable,
15
16    input          wr,
17    input logic     wr_parity,
18    input [7:0]     wr_data,
19    output          idle,
20
21    output logic     tx
22 );
23
24 logic [3:0] baud_div_q;
25 logic [3:0] tick_baud_q;
26
27 logic [3:0] bit_cnt_q, bit_cnt_d;
28 logic [10:0] sreg_q, sreg_d;
29 logic [3:0] tx_q, tx_d;
30
31 assign tx = tx_q;
32
33 always_ff @(posedge clk_i or negedge rst_ni) begin
34     if (!rst_ni) begin
35         baud_div_q <= 4'h0;
36         tick_baud_q <= 1'b0;
37     end else if (tick_baud_x16) begin
38         {tick_baud_q, baud_div_q} <= {1'b0, baud_div_q} + 5'h1;
39     end else begin
40         tick_baud_q <= 1'b0;
41     end
42 end
43
44 always_ff @(posedge clk_i or negedge rst_ni) begin
45     if (!rst_ni) begin
46         bit_cnt_q <= 4'h0;
47         sreg_q <= 11'h7ff;
48         tx_q <= 1'b1;
49     end else begin
50         bit_cnt_q <= bit_cnt_d;
51         sreg_q <= sreg_d;
52         tx_q <= tx_d;
53     end
54 end
55
56 always_comb begin
57     if (!tx_enable) begin
58         bit_cnt_d = 4'h0;
59         sreg_d = 11'h7ff;
60         tx_d = 1'b1;
61     end else begin
62         bit_cnt_d = bit_cnt_q;
63         sreg_d = sreg_q;
64         tx_d = tx_q;
65         if (wr) begin
66             sreg_d = {1'b1, (parity_enable ? wr_parity : 1'b1),
67 wr_data, 1'b0};
68             bit_cnt_d = (parity_enable ? 4'd11 : 4'd10);
69         end else if (tick_baud_q && (bit_cnt_q != 4'h0)) begin
70             sreg_d = {1'b1, sreg_q[10:1]};
71             tx_d = sreg_q[0];
72             bit_cnt_d = bit_cnt_q - 4'h1;
73         end
74     end
75 end

```

```
76  
77 assign idle = (tx_enable) ? (bit_cnt_q == 4'h0) : 1'b1;  
78  
79 endmodule
```


USB 디바이스

What this is. A full-speed USB device controller: endpoints, packet handling, bus state machine.

Why it exists. Because it illustrates a deep protocol stack. Above the differential signalling sit packets, transactions, transfers and enumeration, and deciding which layers are hardware and which are firmware is the principal architectural choice.

Undergraduate review · EECS concepts. Layered protocols; bit stuffing; NRZI encoding; clock recovery from SOF; CRC.

Graduate · Practice note. State the hardware/firmware split in the datasheet. Without it, integrators assume the hardware performs enumeration and discover otherwise late.

13 files • 4,805 lines

API — usb_consts_pkg.sv

Table 987. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_consts_pkg.sv — module / interface / package (1개)

이름	종류
usb_consts_pkg	package

API — usb_fs_nb_in_pe.sv

Table 988. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_in_pe.sv — module / interface / package (1개)

이름	종류
usb_fs_nb_in_pe	module

Table 989. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_in_pe.sv — parameter (3개)

파라미터	형/폭	기본값
NumInEps	logic [4:0]	12
MaxInPktSizeByte	int unsigned	32
AckTimeoutCnt	int unsigned	$((17 * 4) + 6) - 1$

Table 990. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_in_pe.sv — localparam (3개)

상수	형/폭	값
InEpW	int unsigned	$\$ \log_2(\text{NumInEps})$
PktW	int unsigned	$\$ \log_2(\text{MaxInPktSizeByte})$
AckTimeoutCntW	int unsigned	$\$ \log_2(\text{AckTimeoutCnt})$ // derived parameter

Table 991. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_in_pe.sv — 포트 (38개)

포트	방향	형/폭
clk_48mhz_i	input	logic
rst_ni	input	logic

포트	방향	형/폭
link_reset_i	input	logic
link_active_i	input	logic
dev_addr_i	input	logic [6:0]
in_xact_starting_o	output	logic
in_xact_start_ep_o	output	logic [3:0]
in_ep_current_o	output	logic [3:0]
in_ep_rollback_o	output	logic
in_ep_xact_end_o	output	logic
in_ep_get_addr_o	output	logic [PktW - 1:0]
in_ep_data_get_o	output	logic
in_ep_newpkt_o	output	logic
in_ep_enabled_i	input	logic [NumInEps-1:0]
in_ep_stall_i	input	logic [NumInEps-1:0]
in_ep_has_data_i	input	logic [NumInEps-1:0]
in_ep_data_i	input	logic [7:0]
in_ep_data_done_i	input	logic [NumInEps-1:0]
in_ep_iso_i	input	logic [NumInEps-1:0]
in_data_toggle_o	output	logic [NumInEps-1:0]
in_datatog_we_i	input	logic
in_datatog_status_i	input	logic [NumInEps-1:0]
in_datatog_mask_i	input	logic [NumInEps-1:0]
rx_pkt_start_i	input	logic
rx_pkt_end_i	input	logic
rx_pkt_valid_i	input	logic
rx_pid_i	input	logic [3:0]
rx_addr_i	input	logic [6:0]
rx_endp_i	input	logic [3:0]
tx_pkt_start_o	output	logic
tx_pkt_end_i	input	logic
tx_pid_o	output	logic [3:0]
tx_data_avail_o	output	logic
tx_data_get_i	input	logic
tx_data_o	output	logic [7:0]
event_timeout_in_o	output	logic
event_nak_in_o	output	logic

포트	방향	형/폭
event_nodata_in_o	output	logic

API — usb_fs_nb_out_pe.sv

Table 992. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_out_pe.sv — module / interface / package (1개)

이름	종류
usb_fs_nb_out_pe	module

Table 993. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_out_pe.sv — parameter (3개)

파라미터	형/폭	기본값
NumOutEps	logic [4:0]	2
MaxOutPktSizeByte	int unsigned	32
AckTimeoutCnt	int unsigned	17 * 4

Table 994. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_out_pe.sv — localparam (3개)

상수	형/폭	값
OutEpW	int unsigned	\$clog2(NumOutEps)
PktW	int unsigned	\$clog2(MaxOutPktSizeByte)
AckTimeoutCntW	int unsigned	\$clog2(AckTimeoutCnt) // derived parameter

Table 995. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_out_pe.sv — 포트 (33개)

포트	방향	형/폭
clk_48mhz_i	input	logic
rst_ni	input	logic
link_reset_i	input	logic
dev_addr_i	input	logic [6:0]
out_ep_current_o	output	logic [3:0]
out_ep_data_put_o	output	logic
out_ep_put_addr_o	output	logic [PktW - 1:0]
out_ep_data_o	output	logic [7:0]
out_ep_newpkt_o	output	logic
out_ep_acked_o	output	logic
out_ep_rollback_o	output	logic
out_ep_setup_o	output	logic [NumOutEps-1:0]
out_ep_enabled_i	input	logic [NumOutEps-1:0]
out_ep_control_i	input	logic [NumOutEps-1:0]
out_ep_full_i	input	logic [NumOutEps-1:0]
out_ep_stall_i	input	logic [NumOutEps-1:0]

포트	방향	형/폭
out_ep_iso_i	input	logic [NumOutEps-1:0]
out_data_toggle_o	output	logic [NumOutEps-1:0]
out_datatog_we_i	input	logic
out_datatog_status_i	input	logic [NumOutEps-1:0]
out_datatog_mask_i	input	logic [NumOutEps-1:0]
rx_pkt_start_i	input	logic
rx_pkt_end_i	input	logic
rx_pkt_valid_i	input	logic
rx_pid_i	input	logic [3:0]
rx_addr_i	input	logic [6:0]
rx_endp_i	input	logic [3:0]
rx_data_put_i	input	logic
rx_data_i	input	logic [7:0]
tx_pkt_start_o	output	logic
tx_pkt_end_i	input	logic
tx_pid_o	output	logic [3:0]
event_datatog_out_o	output	logic

API — usb_fs_nb_pe.sv

Table 996. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_pe.sv — module / interface / package (1개)

이름	종류
usb_fs_nb_pe	module

Table 997. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_pe.sv — parameter (3개)

파라미터	형/폭	기본값
NumOutEps	int unsigned	2
NumInEps	int unsigned	2
MaxPktSizeByte	int unsigned	32

Table 998. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_pe.sv — localparam (1개)

상수	형/폭	값
PktW	int unsigned	\$clog2(MaxPktSizeByte) // derived parameter

Table 999. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_pe.sv — 포트 (66개) — 앞 60개만 표시(전체 66개, 본문 소스 참조)

포트	방향	형/폭
clk_48mhz_i	input	logic

포트	방향	형/폭
rst_ni	input	logic
link_reset_i	input	logic
link_active_i	input	logic
dev_addr_i	input	logic [6:0]
cfg_eop_single_bit_i	input	logic
cfg_use_diff_rcvr_i	input	logic
cfg_pinflip_i	input	logic
tx_osc_test_mode_i	input	logic
out_data_toggle_o	output	logic [NumOutEps-1:0]
out_datatog_we_i	input	logic
out_datatog_status_i	input	logic [NumOutEps-1:0]
out_datatog_mask_i	input	logic [NumOutEps-1:0]
in_data_toggle_o	output	logic [NumInEps-1:0]
in_datatog_we_i	input	logic
in_datatog_status_i	input	logic [NumInEps-1:0]
in_datatog_mask_i	input	logic [NumInEps-1:0]
diff_rx_ok_i	input	logic
out_ep_current_o	output	logic [3:0]
out_ep_data_put_o	output	logic
out_ep_put_addr_o	output	logic [PktW - 1:0]
out_ep_data_o	output	logic [7:0]
out_ep_newpkt_o	output	logic
out_ep_acked_o	output	logic
out_ep_rollback_o	output	logic
out_ep_setup_o	output	logic [NumOutEps-1:0]
out_ep_enabled_i	input	logic [NumOutEps-1:0]
out_ep_control_i	input	logic [NumOutEps-1:0]
out_ep_full_i	input	logic [NumOutEps-1:0]
out_ep_stall_i	input	logic [NumOutEps-1:0]
out_ep_iso_i	input	logic [NumOutEps-1:0]
in_xact_starting_o	output	logic
in_xact_start_ep_o	output	logic [3:0]
in_ep_current_o	output	logic [3:0]
in_ep_rollback_o	output	logic
in_ep_xact_end_o	output	logic

포트	방향	형/폭
in_ep_get_addr_o	output	logic [PktW - 1:0]
in_ep_data_get_o	output	logic
in_ep_newpkt_o	output	logic
in_ep_enabled_i	input	logic [NumInEps-1:0]
in_ep_stall_i	input	logic [NumInEps-1:0]
in_ep_has_data_i	input	logic [NumInEps-1:0]
in_ep_data_i	input	logic [7:0]
in_ep_data_done_i	input	logic [NumInEps-1:0]
in_ep_iso_i	input	logic [NumInEps-1:0]
sof_detected_o	output	logic
sof_valid_o	output	logic
frame_index_o	output	logic [10:0]
rx_idle_det_o	output	logic
rx_j_det_o	output	logic
rx_crc5_err_o	output	logic
rx_crc16_err_o	output	logic
rx_pid_err_o	output	logic
rx_bitstuff_err_o	output	logic
usb_d_i	input	logic
usb_dp_i	input	logic
usb_dn_i	input	logic
usb_d_o	output	logic
usb_se0_o	output	logic
usb_dp_o	output	logic

API — usb_fs_rx.sv

Table 1000. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_rx.sv — module / interface / package (1개)

이름	종류
usb_fs_rx	module

Table 1001. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_rx.sv — localparam (4개)

상수	형/폭	값
DT	logic [2:0]	3'b100
DJ	logic [2:0]	3'b010
DK	logic [2:0]	3'b001
SE0	logic [2:0]	3'b000

Table 1002. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_rx.sv — 포트 (28개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
link_reset_i	input	logic
cfg_eop_single_bit_i	input	logic
cfg_use_diff_rcvr_i	input	logic
cfg_pinflip_i	input	logic
diff_rx_ok_i	input	logic
usb_d_i	input	logic
usb_dp_i	input	logic
usb_dn_i	input	logic
tx_en_i	input	logic
bit_strobe_o	output	logic
pkt_start_o	output	logic
pkt_end_o	output	logic
pid_o	output	logic [3:0]
addr_o	output	logic [6:0]
endp_o	output	logic [3:0]
frame_num_o	output	logic [10:0]
rx_data_put_o	output	logic
rx_data_o	output	logic [7:0]
valid_pid_o	output	logic
valid_packet_o	output	logic
rx_idle_det_o	output	logic
rx_j_det_o	output	logic
crc5_error_o	output	logic
crc16_error_o	output	logic
pid_error_o	output	logic
bitstuff_error_o	output	logic

API — usb_fs_tx.sv**Table 1003. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_tx.sv — module / interface / package (1개)**

이름	종류
usb_fs_tx	module

Table 1004. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_tx.sv — 포트 (17개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
link_reset_i	input	logic
cfg_pinflip_i	input	logic
tx_osc_test_mode_i	input	logic
bit_strobe_i	input	logic
usb_oe_o	output	logic
usb_d_o	output	logic
usb_se0_o	output	logic
usb_dp_o	output	logic
usb_dn_o	output	logic
pkt_start_i	input	logic
pkt_end_o	output	logic
pid_i	input	logic [3:0]
tx_data_avail_i	input	logic
tx_data_get_o	output	logic
tx_data_i	input	logic [7:0]

Table 1005. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_tx.sv — typedef / using (2개)

새 이름	원래 형
state_e	enum logic [2:0] {Idle, Sync, Pid, DataOrCrc160, Crc161, Eop, OscTest}
out_state_e	enum logic [1:0] {OsIdle, OsWaitByte, OsTransmit}

API — usb_fs_tx_mux.sv

Table 1006. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_tx_mux.sv — module / interface / package (1개)

이름	종류
usb_fs_tx_mux	module

Table 1007. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_tx_mux.sv — 포트 (6개)

포트	방향	형/폭
in_tx_pkt_start_i	input	logic
in_tx_pid_i	input	logic [3:0]
out_tx_pkt_start_i	input	logic
out_tx_pid_i	input	logic [3:0]
tx_pkt_start_o	output	logic
tx_pid_o	output	logic [3:0]

API — usbdev.sv**Table 1008. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev.sv — module / interface / package (1개)**

이름	종류
usbdev	module

Table 1009. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev.sv — parameter (4개)

파라미터	형/폭	기본값
Stub	bit	1'b0
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
RcvrWakeTimeUs	int unsigned	1

Table 1010. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev.sv — localparam (15개)

상수	형/폭	값
SramDw	int	32
SramDepth	int	512
MaxPktSizeByte	int	64
SramAw	int	$\$clog2(SramDepth)$
SizeWidth	int	$\$clog2(MaxPktSizeByte)$
NBuf	int	$(SramDepth * SramDw) / (MaxPktSizeByte * 8)$
NBufWidth	int	$\$clog2(NBuf)$
AVFifoWidth	int	NBufWidth
AVSetupFifoDepth	int	4
AVOutFifoDepth	int	8
RXFifoWidth	int	$NBufWidth + (1 + SizeWidth) + 4 + 1$
RXFifoDepth	int	8
RXFifoDepthw	int	$prim_util_pkg::vbits(RXFifoDepth+1)$
AlertIsFatal	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
RcvrWakeTimeWidth	int	$vbits(RcvrWakeTimeUs + 2)$

Table 1011. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev.sv — 포트 (50개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
clk_aon_i	input	logic
rst_aon_ni	input	logic
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t

포트	방향	형/폭
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
cio_usb_dp_i	input	logic
cio_usb_dn_i	input	logic
usb_rx_d_i	input	logic
cio_usb_dp_o	output	logic
cio_usb_dp_en_o	output	logic
cio_usb_dn_o	output	logic
cio_usb_dn_en_o	output	logic
usb_tx_se0_o	output	logic
usb_tx_d_o	output	logic
cio_sense_i	input	logic
usb_dp_pullup_o	output	logic
usb_dn_pullup_o	output	logic
usb_rx_enable_o	output	logic
usb_tx_use_d_se0_o	output	logic
usb_aon_suspend_req_o	output	logic
usb_aon_wake_ack_o	output	logic
usb_aon_bus_reset_i	input	logic
usb_aon_sense_lost_i	input	logic
usb_aon_bus_not_idle_i	input	logic
usb_aon_wake_detect_active_i	input	logic
usb_ref_val_o	output	logic
usb_ref_pulse_o	output	logic
ram_cfg_i	input	prim_ram_1p_pkg::ram_1p_cfg_req_t
ram_cfg_o	output	prim_ram_1p_pkg::ram_1p_cfg_rsp_t
intr_pkt_received_o	output	logic
intr_pkt_sent_o	output	logic
intr_powered_o	output	logic
intr_disconnected_o	output	logic
intr_host_lost_o	output	logic
intr_link_reset_o	output	logic
intr_link_suspend_o	output	logic
intr_link_resume_o	output	logic
intr_av_out_empty_o	output	logic

포트	방향	형/폭
intr_rx_full_o	output	logic
intr_av_overflow_o	output	logic
intr_link_in_err_o	output	logic
intr_link_out_err_o	output	logic
intr_rx_crc_err_o	output	logic
intr_rx_pid_err_o	output	logic
intr_rx_bitstuff_err_o	output	logic
intr_frame_o	output	logic
intr_av_setup_empty_o	output	logic

API — usbdev_aon_wake.sv

Table 1012. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_aon_wake.sv — module / interface / package (1개)

이름	종류
usbdev_aon_wake	module

Table 1013. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_aon_wake.sv — 포트 (16개)

포트	방향	형/폭
clk_aon_i	input	logic
rst_aon_ni	input	logic
usb_dp_i	input	logic
usb_dn_i	input	logic
usb_sense_i	input	logic
usbdev_dppullup_en_i	input	logic
usbdev_dnpullup_en_i	input	logic
wake_ack_aon_i	input	logic
suspend_req_aon_i	input	logic
usb_dppullup_en_o	output	logic
usb_dnpullup_en_o	output	logic
wake_req_aon_o	output	logic
bus_not_idle_aon_o	output	logic
bus_reset_aon_o	output	logic
sense_lost_aon_o	output	logic
wake_detect_active_aon_o	output	logic

API — usbdev_counter.sv

Table 1014. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_counter.sv — module / interface / package (1개)

이름	종류
usbdev_counter	module

Table 1015. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_counter.sv — parameter (3개)

파라미터	형/폭	기본값
NEndpoints	int unsigned	12
NEvents	int unsigned	1
Width	int unsigned	8

Table 1016. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_counter.sv — localparam (1개)

상수	형/폭	값
EpW	int unsigned	prim_util_pkg::vbits(NEndpoints) // derived parameter

Table 1017. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_counter.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
reset_i	input	logic
event_i	input	logic [NEvents-1:0]
ep_i	input	logic [EpW-1:0]
ev_qe_i	input	logic
ev_i	input	logic [NEvents-1:0]
ev_o	output	logic [NEvents-1:0]
endp_qe_i	input	logic
endpoints_i	input	logic [NEndpoints-1:0]
endpoints_o	output	logic [NEndpoints-1:0]

API — usbdev_iomux.sv**Table 1018. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_iomux.sv — module / interface / package (1개)**

이름	종류
usbdev_iomux	module

Table 1019. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_iomux.sv — 포트 (28개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
hw2reg_sense_o	output	usbdev_hw2reg_phy_pins_sense_reg_t
reg2hw_drive_i	input	usbdev_reg2hw_phy_pins_drive_reg_t
usb_rx_d_i	input	logic

포트	방향	형/폭
usb_rx_dp_i	input	logic
usb_rx_dn_i	input	logic
cio_usb_sense_i	input	logic
usb_tx_d_o	output	logic
usb_tx_se0_o	output	logic
usb_tx_dp_o	output	logic
usb_tx_dn_o	output	logic
usb_tx_oe_o	output	logic
usb_dp_pullup_en_o	output	logic
usb_dn_pullup_en_o	output	logic
usb_rx_enable_o	output	logic
usb_rx_d_o	output	logic
usb_rx_dp_o	output	logic
usb_rx_dn_o	output	logic
usb_tx_d_i	input	logic
usb_tx_se0_i	input	logic
usb_tx_dp_i	input	logic
usb_tx_dn_i	input	logic
usb_tx_oe_i	input	logic
usb_dp_pullup_en_i	input	logic
usb_dn_pullup_en_i	input	logic
usb_rx_enable_i	input	logic
usb_pwr_sense_o	output	logic

API — usbdev_linkstate.sv

Table 1020. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_linkstate.sv — module / interface / package (1개)

이름	종류
usbdev_linkstate	module

Table 1021. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_linkstate.sv — localparam (3개)

상수	형/폭	값
SUSPEND_TIMEOUT	logic [11:0]	12'd3000
RESET_TIMEOUT	logic [2:0]	3'd3
SOF_TIMEOUT	logic [9:0]	10'd1005

Table 1022. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_linkstate.sv — 포트 (21개)

포트	방향	형/폭
clk_48mhz_i	input	logic
rst_ni	input	logic
us_tick_i	input	logic
usb_sense_i	input	logic
usb_dp_i	input	logic
usb_dn_i	input	logic
usb_oe_i	input	logic
usb_pullup_en_i	input	logic
rx_idle_det_i	input	logic
rx_j_det_i	input	logic
sof_detected_i	input	logic
resume_link_active_i	input	logic
link_disconnect_o	output	logic
link_powered_o	output	logic
link_reset_o	output	logic
link_active_o	output	logic
link_suspend_o	output	logic
link_resume_o	output	logic
host_lost_o	output	logic
sof_missed_o	output	logic
link_state_o	output	logic [2:0]

API — usbdev_pkg.sv

Table 1023. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_pkg.sv — module / interface / package (17#)

이름	종류
usbdev_pkg	package

```

1. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_consts_pkg.sv (66
lines)
1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 package usb_consts_pkg;
6
7 // Definitions for USB Packet Identifier (called PID in the
  spec)
8
9 // Bottom two bits of the PID are type
10 typedef enum logic [1:0] {
11     UsbPidTypeSpecial = 2'b00,
12     UsbPidTypeToken   = 2'b01,
13     UsbPidTypeHandshake = 2'b10,
14     UsbPidTypeData    = 2'b11
15 } usb_pid_type_e;
16
17 // Direct from USB 2.0 Spec Table 8-1

```

```

18 typedef enum logic [3:0] {
19     // TOKEN
20     UsbPidOut   = 4'b0001,
21     UsbPidIn    = 4'b1001,
22     UsbPidSof   = 4'b0101,
23     UsbPidSetup = 4'b1101,
24     // DATA
25     UsbPidData0 = 4'b0011,
26     UsbPidData1 = 4'b1011,
27     UsbPidData2 = 4'b0111, // Only valid for High Speed
28     UsbPidMData = 4'b1111, // Only valid for High Speed
29     // HANDSHAKE
30     UsbPidAck   = 4'b0010,
31     UsbPidNak   = 4'b1010,
32     UsbPidStall = 4'b1110,
33     UsbPidNyet  = 4'b0110,
34     // SPECIAL
35     UsbPidPre   = 4'b1100
36 } usb_pid_e;
37
38 // USB SETUP packet commands (USB2.0 Table 9-4)
39 typedef enum logic [7:0] {
40     SetupGetStatus      = 8'd0,
41     SetupClearFeature   = 8'd1,
42

```

```

43 SetupSetFeature      = 8'd3,
44 SetupSetAddress      = 8'd5,
45 SetupGetDescriptor   = 8'd6,
46 SetupSetDescriptor   = 8'd7,
47 SetupGetConfiguration = 8'd8,
48 SetupSetConfiguration = 8'd9,
49 SetupGetInterface    = 8'd10,
50 SetupSetInterface     = 8'd11,
51 SetupSynchFrame      = 8'd12
52 } usb_setup_request_e;
53
54 // USB Descriptor Types (USB2.0 Table 9-5)
55 typedef enum logic [7:0] {
56     DscrTypeDevice      = 8'd1,
57     DscrTypeConfiguration = 8'd2,
58     DscrTypeString      = 8'd3,
59     DscrTypeInterface    = 8'd4,
60     DscrTypeEndpoint     = 8'd5,
61     DscrTypeDevQual      = 8'd6,
62     DscrTypeOthrSpd      = 8'd7,
63     DscrTypeIntPwr       = 8'd8
64 } usb_dscr_type_e;
65
66 endpackage

```

2. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_in_pe.sv (431 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 // Copyright Luke Valenty (TinyFPGA project, https://git-
  hub.com/tinyfpga/TinyFPGA-Bootloader).
5
6 // USB Full Speed Non-Buffered Protocol Engine for IN end-
  points
7 //

8 // Decode IN requests from host and respond with data if there is
  any
9 //
10 // Based on usb_fs_in_pe.v from the TinyFPGA-Bootloader pro-
  ject but
11 // this version contains no packet buffers
12
13 `include "prim_assert.sv"
14
15 module usb_fs_nb_in_pe #(
16     parameter logic [4:0] NumInEps = 12,
17     parameter int unsigned MaxInPktSizeByte = 32,
18     // Targeting 17 bit times for the timeout. The counter runs
  on the 48 MHz
19     // clock, the SOP delay to rx_pkt_start_i is 1.5 bit times
  (synchronizer flops and K detection),
20     // and the EOP delay to tx_pkt_end_i is essentially 0. This
  timeout originates from Section
21     // 7.1.19.1 of the USB 2.0 specification.
22     parameter int unsigned AckTimeoutCnt = ((17 * 4) + 6) -
  1, // -1 because [N-1:0] counted

23     localparam int unsigned InEpW = $clog2(NumInEps), // derived
  parameter
24     localparam int unsigned PktW = $clog2(MaxInPktSizeByte), //
  derived parameter
25     localparam int unsigned AckTimeoutCntW = $clog-
  2(AckTimeoutCnt) // derived parameter
26 ) (
27     input logic          clk_48mhz_i,
28     input logic          rst_ni,
29     input logic          link_reset_i,
30     input logic          link_active_i,
31     input logic [6:0]    dev_addr_i,
32
33     // Transaction starting
34     // of an IN transaction; capture details
35     output logic [3:0]    in_xact_start_ep_o, // End-
  point starting an IN transaction
36
37     // endpoint interface
38     output logic [3:0]    in_ep_current_o, // Other sig-
  nals addressed to this ep
39     output logic          in_ep_rollback_o, // Bad ter-
  mination, rollback transaction
40     output logic          in_ep_xact_end_o, // good ter-
  mination, transaction complete
41     output logic [PktW - 1:0] in_ep_get_addr_o, // Offset
  requested (0..pktlen)
42     output logic          in_ep_data_get_o, // Accept
  data (get_addr advances too)
43     output logic          in_ep_newpkt_o, // New IN
  packet starting (updates in_ep_current_o)
44     input logic [NumInEps-1:0] in_ep_enabled_i, // Endpoint

```

```

  is enabled and produces handshakes
50     input logic [NumInEps-1:0] in_ep_stall_i, // Endpoint in
  a stall state
51     input logic [NumInEps-1:0] in_ep_has_data_i, // Endpoint
  has data to supply
52     input logic [7:0]          in_ep_data_i, // Data for cur-
  rent get_addr
53     input logic [NumInEps-1:0] in_ep_data_done_i, // Set when
  out of data
54     input logic [NumInEps-1:0] in_ep_iso_i, // Configure end-
  point in isochronous mode
55     output logic [NumInEps-1:0] in_data_toggle_o, // Current
  state of IN data toggles
56     input logic          in_datatog_we_i, // IN data
  toggles write strobe from software
57     input logic [NumInEps-1:0] in_datatog_status_i, // New
  state of selected IN data toggles
58     input logic [NumInEps-1:0] in_datatog_mask_i, // Which IN EP
  data toggles to modify
59
60     // rx path
61     // tx path
62     // Strobed on reception of packet.
63     input logic          rx_pkt_start_i,
64     input logic          rx_pkt_end_i,
65     input logic          rx_pkt_valid_i,
66
67     // Most recent packet received.
68     input logic [3:0]    rx_pid_i,
69     input logic [6:0]    rx_addr_i,
70     input logic [3:0]    rx_endp_i,
71
72     // tx path
73     // Strobe to send new packet.
74     output logic          tx_pkt_start_o,
75     input logic          tx_pkt_end_i,
76     // Packet type to send
77     output logic [3:0]    tx_pid_o,
78
79     // Data payload to send if any
80     output logic          tx_data_avail_o,
81     input logic          tx_data_get_i,
82     output logic [7:0]    tx_data_o,
83
84     // event counters
85     output logic          event_timeout_in_o,
86     output logic          event_nak_in_o,
87     output logic          event_nodata_in_o
88 );
89
90 // in transaction state machine
91
92 import usb_consts_pkg::*;
93
94 typedef enum logic [2:0] {
95     StIdle,
96     StRcvdIn,
97     StSendData,
98     StWaitTxEnd,
99     StWaitAckStart,
100     StWaitAck
101 } state_in_e;
102
103 state_in_e in_xact_state;
104 state_in_e in_xact_state_next;
105
106 logic in_xact_end;
107
108 assign in_ep_xact_end_o = in_xact_end;
109
110 // data toggle state
111 logic [NumInEps-1:0] data_toggle_q, data_toggle_d;
112
113 // endpoint data buffer
114 logic token_received, setup_token_received,
  in_token_received, ack_received, nak_received;
115 logic more_data_to_send;
116 logic ep_in_hw, ep_active;
117 logic [3:0] in_ep_current_d;
118
119 // More syntax so can compare with enum
120 usb_pid_type_e rx_pid_type;
121 usb_pid_e rx_pid;
122 assign rx_pid_type = usb_pid_type_e'(rx_pid_i[1:0]);
123 assign rx_pid = usb_pid_e'(rx_pid_i);
124
125 assign token_received =

```

```

136 rx_pkt_end_i &&
137 rx_pkt_valid_i &&
138 rx_pid_type == UsbPidTypeToken &&
139 rx_addr_i == dev_addr_i;
140
141 assign setup_token_received =
142 token_received &&
143 rx_pid == UsbPidSetup;
144
145 assign in_token_received =
146 token_received &&
147 rx_pid == UsbPidIn;
148
149 assign ack_received =
150 rx_pkt_end_i &&
151 rx_pkt_valid_i &&
152 rx_pid == UsbPidAck;
153
154 assign nak_received =
155 rx_pkt_end_i &&
156 rx_pkt_valid_i &&
157 rx_pid == UsbPidNak;
158
159 // Is the specified endpoint actually implemented in hard-
ware?
160 assign ep_in_hw = {1'b0, rx_endp_i} < NumInEps;
161 assign in_ep_current_d = ep_in_hw ? rx_endp_i : '0;
162
163 // Make widths work - in_ep_current_d/in_ep_current_o only
hold implemented endpoint IDs.
164 // These signals can be used to index signals of NumInEps
width.
165 // They are only valid if ep_active is set, i.e., if the
specified endpoint is implemented.
166 logic [InEpW-1:0] in_ep_index;
167 logic [InEpW-1:0] in_ep_index_d;
168 assign in_ep_index = in_ep_current_o[0+:InEpW];
169 assign in_ep_index_d = in_ep_current_d[0+:InEpW];
170
171 // Is the endpoint active?
172 assign ep_active = in_ep_enabled_i[in_ep_index_d] &
ep_in_hw;
173
174 // Retain whether the current IN transmission has associated
data; an Isochronous transaction
175 // for which there was no packet data available must not
raise a 'pkt_sent' interrupt.
176 logic has_data_q;
177
178 assign more_data_to_send = has_data_q &
~in_ep_data_done_i[in_ep_index];
179
180 assign tx_data_avail_o = logic'(in_xact_state == StSendData) &
more_data_to_send;
181
182 // Transaction is starting on this IN endpoint; capture the
packet details.
183 // Transaction is starting on this IN endpoint; capture the
packet details.
184 // Transaction is starting on this IN endpoint; capture the
packet details.
185 logic in_starting;
186 assign in_starting = (in_xact_state == StIdle || in_x-
act_state == StWaitAck) && in_token_received;
187
188 assign in_xact_starting_o = in_starting & ep_active;
189 assign in_xact_start_ep_o = in_ep_current_d;
190
191 // in transaction state machine
192 // in transaction state machine
193 // in transaction state machine
194
195 logic [AckTimeoutCntW-1:0] timeout_cntdown_d,
timeout_cntdown_q;
196 logic rollback_in_xact;
197
198 always_comb begin
199 in_xact_state_next = in_xact_state;
200 in_xact_end = 1'b0;
201 tx_pkt_start_o = 1'b0;
202 tx_pid_o = 4'b0000;
203 rollback_in_xact = 1'b0;
204 timeout_cntdown_d = AckTimeoutCnt[AckTimeoutCntW-1:0];
205 unique case (in_xact_state)
206 StIdle: begin
207 if (ep_active && in_token_received) begin
208 in_xact_state_next = StRcvdIn;
209 end else begin
210 // Ignore tokens to inactive endpoints. Send no re-
sponse.
211 in_xact_state_next = StIdle;
212 end
213 end
214
215 StRcvdIn: begin
216 tx_pkt_start_o = 1'b1; // Need to transmit NAK/STALL
or DATA

```

```

217
218 if (in_ep_iso_i[in_ep_index]) begin
219 // ISO endpoint
220 // We always need to transmit. When no data is
available, we send
221 // a zero-length packet.
222 // DATA0 always for full-speed isochronous endpoints
223 in_xact_state_next = StSendData;
224 tx_pid_o = {UsbPidData0};
225 end else if (in_ep_stall_i[in_ep_index]) begin
226 in_xact_state_next = StIdle;
227 tx_pid_o = {UsbPidStall}; // STALL
228 end else if (has_data_q) begin
229 in_xact_state_next = StSendData;
230 tx_pid_o = {data_toggle_q[in_ep_index], 1'b0, {Us-
bPidTypeData}}; // DATA0/1
231 end else begin
232 in_xact_state_next = StIdle;
233 tx_pid_o = {UsbPidNak}; // NAK
234 end
235 end
236
237 StSendData: begin
238 // Use &in_ep_get_addr so width can vary, looking for
all ones
239 if ((!more_data_to_send) || (&in_ep_get_addr_o) &&
tx_data_get_i) begin
240 if (in_ep_iso_i[in_ep_index]) begin
241 in_xact_state_next = StIdle; // no ACK for ISO EPs
242 in_xact_end = has_data_q;
243 end else begin
244 if (tx_pkt_end_i) begin
245 in_xact_state_next = StWaitAckStart;
246 end else begin
247 in_xact_state_next = StWaitTxEnd;
248 end
249 end
250 end else begin
251 in_xact_state_next = StSendData;
252 end
253 end
254
255 StWaitTxEnd: begin
256 if (tx_pkt_end_i) begin
257 in_xact_state_next = StWaitAckStart;
258 end
259 end
260
261 StWaitAckStart: begin
262 // The spec says we have up to 18 bit times to wait
for the host
263 // response. If it doesn't arrive in time, we must in-
validate the
264 // transaction.
265 timeout_cntdown_d = timeout_cntdown_q - 1'b1;
266
267 if (rx_pkt_start_i) begin
268 in_xact_state_next = StWaitAck;
269 end else if (timeout_cntdown_q == '0) begin
270 in_xact_state_next = StIdle;
271 rollback_in_xact = 1'b1;
272 end else begin
273 in_xact_state_next = StWaitAckStart;
274 end
275 end
276
277 StWaitAck: begin
278 if (ack_received) begin
279 in_xact_state_next = StIdle;
280 in_xact_end = 1'b1;
281 end else if (in_token_received) begin
282 // Handshake response is missing.
283 in_xact_state_next = ep_active ? StRcvdIn : StIdle;
284 rollback_in_xact = 1'b1;
285 end else if (rx_pkt_end_i) begin
286 // Includes NAK
287 in_xact_state_next = StIdle;
288 rollback_in_xact = 1'b1;
289 end else begin
290 in_xact_state_next = StWaitAck;
291 end
292 end
293
294 default: in_xact_state_next = StIdle;
295 endcase
296 end
297
298 `ASSERT(InXactStateValid_A,
299 in_xact_state inside {StIdle, StRcvdIn, StSendData,
StWaitTxEnd, StWaitAckStart, StWaitAck},
300 clk_48mhz_i)
301
302 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
303 if (!rst_ni) begin
304 timeout_cntdown_q <= AckTimeoutCnt[AckTimeoutCntW-1:0];
305 end else begin
306 timeout_cntdown_q <= timeout_cntdown_d;
307 end
308 end
309
310 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin

```



```

311 if (!rst_ni) begin
312   tx_data_o <= '0';
313 end else begin
314   tx_data_o <= in_ep_data_i;
315 end
316 end
317
318 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
319   if (!rst_ni) begin
320     in_xact_state <= StIdle;
321     in_ep_rollback_o <= 1'b0;
322   end else if (link_reset_i || !link_active_i) begin
323     in_xact_state <= StIdle;
324     in_ep_rollback_o <= 1'b0;
325   end else begin
326     in_xact_state <= in_xact_state_next;
327     in_ep_rollback_o <= rollback_in_xact;
328   end
329 end
330
331 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
332   if (!rst_ni) begin
333     in_ep_get_addr_o <= '0';
334   end else begin
335     if (in_xact_state == StIdle) begin
336       in_ep_get_addr_o <= '0';
337     end else if ((in_xact_state == StSendData) &&
tx_data_get_i) begin
338       in_ep_get_addr_o <= in_ep_get_addr_o + 1'b1;
339     end
340   end
341 end
342
343 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
344   if (!rst_ni) begin
345     in_ep_newpkt_o <= 1'b0;
346     in_ep_current_o <= '0';
347     has_data_q <= 1'b0;
348   end else begin
349     if (in_token_received) begin
350       in_ep_current_o <= in_ep_current_d;
351       in_ep_newpkt_o <= 1'b1;
352       has_data_q <= in_ep_has_data_i[in_ep_index_d];
353     end else begin
354       in_ep_newpkt_o <= 1'b0;
355     end
356   end
357 end
358
359 // Updating of data toggles
360 always_comb begin : proc_data_toggle_d
361   data_toggle_d = data_toggle_q;
362
363   if (setup_token_received && ep_active) begin
364     data_toggle_d[in_ep_index_d] = 1'b1;
365   end else if ((in_xact_state == StWaitAck) && ack_received)
begin
366     data_toggle_d[in_ep_index] =
~data_toggle_q[in_ep_index];
367   end
368   // Selective modification by software
369   if (in_datatog_we_i) begin
370     data_toggle_d = (data_toggle_d & ~in_datatog_mask_i) |
371       (in_datatog_status_i & in_data-
togg_mask_i);
372   end
373 end
374 // Supply current data toggles to register interface
375 assign in_data_toggle_o = data_toggle_q;
376
377 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
378   if (!rst_ni) begin
379     data_toggle_q <= '0'; // Clear for all endpoints
380   end else if (link_reset_i) begin
381     data_toggle_q <= '0'; // Clear for all endpoints
382   end else begin
383     data_toggle_q <= data_toggle_d;
384   end
385 end
386
387 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
388   if (!rst_ni) begin
389     in_ep_data_get_o <= 1'b0;
390   end else begin
391     if ((in_xact_state == StSendData) && tx_data_get_i) be-
gin
392       in_ep_data_get_o <= 1'b1;
393     end else begin
394       in_ep_data_get_o <= 1'b0;
395     end
396   end
397 end
398
399 ///////////////////////////////////////////////////
400 // Count non-Isynchronous IN transactions not receiving a
handshake response.
401 //
402 // - no handshake response within the timeout period
403 // - packet reception started within the timeout period but

```

```

it was not ACK/NAK.
404 ///////////////////////////////////////////////////
405 assign event_timeout_in_o = (in_xact_state == StWaitAckStart
||
406   (in_xact_state == StWaitAck &&
rx_pkt_end_i && !nak_received))
407   & rollback_in_xact;
408
409 ///////////////////////////////////////////////////
410 // Count IN transactions that are actively NAKed by the
host.
411 ///////////////////////////////////////////////////
412 assign event_nak_in_o = (in_xact_state == StWaitAck) &&
nak_received;
413
414 ///////////////////////////////////////////////////
415 // Count the number of IN requests for which data is not
available, including
416 // endpoints which are supported but not presently enabled/
configured.
417 // (These may be ignored using the event counter configura-
tion if required.)
418 ///////////////////////////////////////////////////
419 logic event_nodata_in_q;
420 // This event signal must be delayed by a single cycle so
that 'in_ep_current_o' becomes valid.
421 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
422   if (!rst_ni) event_nodata_in_q <= 1'b0;
423   else event_nodata_in_q <= in_starting & (ep_in_hw ? !
in_ep_has_data_i[in_ep_index_d] : 1'b0);
424 end
425 assign event_nodata_in_o = event_nodata_in_q;
426
427 ///////////////////////////////////////////////////
428 // Assertions //
429 ///////////////////////////////////////////////////
430
431 endmodule

```

3. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_out_pe.sv (443 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 // Copyright Luke Valenty (TinyFPGA project, https://git-
hub.com/tinyfpga/TinyFPGA-Bootloader).
5
6 // USB Full Speed Non-Buffered Protocol Engine for OUT/SETUP
endpoints
7 //
8 // Decode OUT/SETUP requests from host and accept data unless
buffers are full
9 // (SETUP is a special form of OUT and starts a transaction
sequence)
10 //
11 // Based on usb_fs_out_pe.v from the TinyFPGA-Bootloader pro-
ject but
12 // this version contains no packet buffers
13
14 `include "prim_assert.sv"
15
16 module usb_fs_nb_out_pe #(
17   parameter logic [4:0] NumOutEps = 2,
18   parameter int unsigned MaxOutPktSizeByte = 32,
19   // Targeting 17 bit times for the timeout. The counter runs
on the 48 MHz
20   // clock. The SOP delay to rx_pkt_start_i is 3 bit times,
but the two
21   // packets go in the same direction, so this is negated.
This timeout
22   // originates from Section 7.1.19.1 of the USB 2.0 specific-
ation.
23   parameter int unsigned AckTimeoutCnt = 17 * 4,
24   localparam int unsigned OutEpW = $clog2(NumOutEps), // de-
rived parameter
25   localparam int unsigned PktW = $clog2(MaxOutPktSizeByte), //
derived parameter
26   localparam int unsigned AckTimeoutCntW = $clog2(AckTimeoutCnt) // derived parameter
27 ) (
28   input logic clk_48mhz_i,
29   input logic rst_ni,
30   input logic link_reset_i,
31   input logic [6:0] dev_addr_i,
32
33   ///////////////////////////////////////////////////
34   // endpoint interface //
35   ///////////////////////////////////////////////////
36   output logic [3:0] out_ep_current_o, // Other
signals address to this ep,

```

```

37 // stable
for several cycles

38 output logic out_ep_data_put_o, // put the
data (put addr advances after)
39 output logic [PKTW - 1:0] out_ep_put_addr_o, // Offset
to put data (0..pktlen)
40 output logic [7:0] out_ep_data_o,
41 output logic out_ep_newpkt_o, // new pack-
et, current was set
42 output logic out_ep_acked_o, // good ter-
mination, device has acked
43 output logic out_ep_rollback_o, // bad
termination, discard data
44 output logic [NumOutEps-1:0] out_ep_setup_o,

45 input logic [NumOutEps-1:0] out_ep_enabled_i, // Endpoint is
enabled and produces handshakes
46 input logic [NumOutEps-1:0] out_ep_control_i, // Is a
control endpoint: Accepts SETUP packets
47 input logic [NumOutEps-1:0] out_ep_full_i, // Cannot ac-
cept data
48 input logic [NumOutEps-1:0] out_ep_stall_i, // Stalled
49 input logic [NumOutEps-1:0] out_ep_iso_i, // Configure
endpoint in isochronous mode
50

51 output logic [NumOutEps-1:0] out_data_toggle_o, // Current
state of OUT data toggles

52 input logic out_datatog_we_i, // OUT data
toggles write strobe from software
53 input logic [NumOutEps-1:0] out_datatog_status_i, // New
state of selected OUT data toggles
54 input logic [NumOutEps-1:0] out_datatog_mask_i, // Which
OUT EP data toggles to modify
55
56 // rx path //
57 // rx path //
58 // rx path //
59
60 // Strobed on reception of packet.
61 input logic rx_pkt_start_i,
62 input logic rx_pkt_end_i,
63 input logic rx_pkt_valid_i,
64
65 // Most recent packet received.
66 input logic [3:0] rx_pid_i,
67 input logic [6:0] rx_addr_i,
68 input logic [3:0] rx_endp_i,
69
70 // rx_data is pushed into endpoint controller.
71 input logic rx_data_put_i,
72 input logic [7:0] rx_data_i,
73
74
75 // tx path //
76 // tx path //
77 // tx path //
78
79 // Strobe to send new packet.
80 output logic tx_pkt_start_o,
81 input logic tx_pkt_end_i,
82 output logic [3:0] tx_pid_o,
83
84 // event counters
85 output logic event_datatog_out_o
86 );
87
88 // suppress warnings
89 logic unused_1;
90 assign unused_1 = tx_pkt_end_i;
91
92 // out transaction state machine //
93 // out transaction state machine //
94 // out transaction state machine //
95 import usb_consts_pkg::*;
96
97 typedef enum logic [2:0] {
98     StIdle,
99     StRcvdOut,
100     StRcvdDataStart,
101     StRcvdDataEnd,
102     StRcvdIsoDataEnd
103 } state_out_e;
104
105 state_out_e out_xact_state;
106 state_out_e out_xact_state_next;
107
108 logic out_xact_start;
109 logic new_pkt_end;
110 logic rollback_data;
111
112 // set when the endpoint buffer is unable to receive the out
transaction
113 logic nak_out_transaction;
114
115 // data toggle state
116 logic [NumOutEps - 1:0] data_toggle_q, data_toggle_d;
117

```

```

118 // Decode the rx token
119 logic token_received, out_token_received,
setup_token_received;
120 logic invalid_packet_received, data_packet_received,
non_data_packet_received;
121 logic bad_data_toggle;
122 logic ep_in_hw, ep_active, ep_is_control;
123 logic [3:0] out_ep_current_d;
124
125 // 1: If the current transaction is a SETUP, 0: OUT
126 logic current_xact_setup_q;
127
128 // More syntax so can compare with enum
129 usb_pid_type_e rx_pid_type;
130 usb_pid_e rx_pid;
131 assign rx_pid_type = usb_pid_type_e'(rx_pid_i[1:0]);
132 assign rx_pid = usb_pid_e'(rx_pid_i);
133
134 assign token_received =
135     rx_pkt_end_i &&
136     rx_pkt_valid_i &&
137     rx_pid_type == UsbPidTypeToken &&
138     rx_addr_i == dev_addr_i;
139
140 assign out_token_received =
141     token_received &&
142     rx_pid == UsbPidOut;
143
144 assign setup_token_received =
145     token_received &&
146     rx_pid == UsbPidSetup;
147
148 assign invalid_packet_received =
149     rx_pkt_end_i &&
150     !rx_pkt_valid_i;
151
152 assign data_packet_received =
153     rx_pkt_end_i &&
154     rx_pkt_valid_i &&
155     ((rx_pid == UsbPidData0) || (rx_pid == UsbPidData1));
156
157 assign non_data_packet_received =
158     rx_pkt_end_i &&
159     rx_pkt_valid_i &&
160     !((rx_pid == UsbPidData0) || (rx_pid == UsbPidData1));
161
162 // Is the specified endpoint actually implemented in hard-
ware?
163 assign ep_in_hw = {1'b0, rx_endp_i} < NumOutEps;
164 assign out_ep_current_d = ep_in_hw ? rx_endp_i : '0;
165
166 // Make widths work - out_ep_current_d/out_ep_current_o only
hold implemented endpoint IDs.
167 // These signals can be used to index signals of NumOutEps
width.
168 // They are only valid if ep_in_hw is set, i.e., if the spe-
cified endpoint is implemented.
169 logic [OutEpW-1:0] out_ep_index;
170 logic [OutEpW-1:0] out_ep_index_d;
171 assign out_ep_index = out_ep_current_o[0+: OutEpW];
172 assign out_ep_index_d = out_ep_current_d[0+: OutEpW];
173
174 // Is the endpoint active?
175 assign ep_active = out_ep_enabled_i[out_ep_index_d] &
ep_in_hw;
176 assign ep_is_control = out_ep_control_i[out_ep_index_d];
177
178 assign bad_data_toggle =
179     data_packet_received &&
180     ep_active &&
181     rx_pid_i[3] != data_toggle_q[out_ep_index_d];
182
183 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
184     if (!rst_ni) begin
185         out_ep_setup_o <= '0;
186     end else begin
187         if (setup_token_received && ep_active) begin
188             out_ep_setup_o[out_ep_index_d] <= 1'b1;
189         end else if (out_token_received && ep_active) begin
190             out_ep_setup_o[out_ep_index_d] <= 1'b0;
191         end
192     end
193 end
194
195 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
196     if (!rst_ni) begin
197         out_ep_data_o <= 0;
198     end else begin
199         if (rx_data_put_i) begin
200             out_ep_data_o <= rx_data_i;
201         end
202     end
203 end
204
205 // out transaction state machine //
206 // out transaction state machine //
207 // out transaction state machine //
208
209 logic [AckTimeoutCntW-1:0] timeout_cntdown_d,
timeout_cntdown_q;

```

```

210
211 always_comb begin
212     out_ep_acked_o = 1'b0;
213     out_xact_start = 1'b0;
214     out_xact_state_next = out_xact_state;
215     tx_pkt_start_o = 1'b0;
216     tx_pid_o = 4'b0000;
217     new_pkt_end = 1'b0;
218     rollback_data = 1'b0;
219     timeout_cntdown_d = AckTimeoutCnt[AckTimeoutCntW-1:0];
220
221     unique case (out_xact_state)
222         StIdle: begin
223             // For unimplemented EPs, transactions are ignored.
224
225             // SETUP transactions are also ignored for non-control
226             EPs.
227             if (ep_active && (out_token_received ||
228             (setup_token_received && ep_is_control))) begin
229                 out_xact_state_next = StRcvdOut;
230                 out_xact_start = 1'b1;
231             end else begin
232                 out_xact_state_next = StIdle;
233             end
234         end
235         StRcvdOut: begin
236             // The spec says we have up to 18 bit times to wait
237             for the host's
238             // data packet. If it doesn't arrive in time, we must
239             invalidate the
240             // transaction.
241             timeout_cntdown_d = timeout_cntdown_q - 1'b1;
242
243             if (rx_pkt_start_i) begin
244                 out_xact_state_next = StRcvdDataStart;
245             end else if (timeout_cntdown_q == '0) begin
246                 out_xact_state_next = StIdle;
247             end else begin
248                 out_xact_state_next = StRcvdOut;
249             end
250         end
251         StRcvdDataStart: begin
252             if (!ep_is_control && out_ep_iso_i[out_ep_index] &&
253             data_packet_received) begin
254                 // ISO EP: Don't send a handshake, ignore toggle.
255                 // But ignore iso bit for endpoints marked control.
256                 This is
257                 // a configuration error.
258                 out_xact_state_next = StRcvdIsoDataEnd;
259             end else if (bad_data_toggle && !
260             out_ep_stall_i[out_ep_index]) begin
261                 // The DATA toggle was wrong (skipped when this EP
262                 is stalled)
263                 // Note: bad_data_toggle is meaningless for unimple-
264                 mented EPs.
265                 out_xact_state_next = StIdle;
266                 rollback_data = 1'b1;
267                 tx_pkt_start_o = 1'b1;
268                 tx_pid_o = {UsbPidAck}; // ACK by spec because this
269                 is most likely previous ACK was lost
270             end else if (invalid_packet_received ||
271             non_data_packet_received) begin
272                 // in these cases eg bad CRC, send no response (not
273                 a NAK)
274                 out_xact_state_next = StIdle;
275                 rollback_data = 1'b1;
276             end else if (data_packet_received) begin
277                 out_xact_state_next = StRcvdDataEnd;
278             end else begin
279                 out_xact_state_next = StRcvdDataStart;
280             end
281         end
282         StRcvdDataEnd: begin
283             out_xact_state_next = StIdle;
284             tx_pkt_start_o = 1'b1;
285
286             if (current_xact_setup_q) begin
287                 // SETUP transactions end here
288                 if (nak_out_transaction |
289                 out_ep_full_i[out_ep_index]) begin
290                     // SETUP transactions that fail to be received are
291                     dropped without a response.
292                     tx_pkt_start_o = 1'b0;
293                     rollback_data = 1'b1;
294                 end else begin
295                     tx_pid_o = {UsbPidAck};
296                     new_pkt_end = 1'b1;
297                     out_ep_acked_o = 1'b1;
298                 end
299             end else begin
300                 // Non-isochronous OUT transactions end here
301                 if (out_ep_stall_i[out_ep_index]) begin
302                     tx_pid_o = {UsbPidStall}; // STALL
303                     rollback_data = 1'b1; // Packet not accepted
304                 end else if (nak_out_transaction |
305                 out_ep_full_i[out_ep_index]) begin
306                     tx_pid_o = {UsbPidNak}; // NAK -- the endpoint

```

```

could not accept the data at the moment
307     rollback_data = 1'b1;
308     end else begin
309         tx_pid_o = {UsbPidAck}; // ACK
310         new_pkt_end = 1'b1;
311         out_ep_acked_o = 1'b1;
312     end
313 end
314
315 StRcvdIsoDataEnd: begin
316     // Isochronous OUT transactions end here
317     out_xact_state_next = StIdle;
318 end
319
320 if (nak_out_transaction | out_ep_full_i[out_ep_index])
321 begin
322     // We got a valid packet, but can't store it (error
323     that the software must resolve)
324     rollback_data = 1'b1;
325     end else begin
326         // We got a valid packet, but we don't send an ACK
327         on the bus
328         new_pkt_end = 1'b1;
329         out_ep_acked_o = 1'b1;
330     end
331 end
332
333 default: out_xact_state_next = StIdle;
334 endcase
335 end
336
337 `ASSERT(OutXactStateValid_A,
338
339 out_xact_state inside {StIdle, StRcvdOut, StRcvdDataStart,
340 StRcvdDataEnd, StRcvdIsoDataEnd},
341 clk_48mhz_i)
342
343 // could flop this if needed
344 assign out_ep_rollback_o = rollback_data;
345
346 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
347     if (!rst_ni) begin
348         timeout_cntdown_q <= AckTimeoutCnt[AckTimeoutCntW-1:0];
349     end else begin
350         timeout_cntdown_q <= timeout_cntdown_d;
351     end
352 end
353
354 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
355     if (!rst_ni) begin
356         out_xact_state <= StIdle;
357     end else begin
358         out_xact_state <= link_reset_i ? StIdle : out_x-
359         act_state_next;
360     end
361 end
362
363 // Updating of data toggles
364 always_comb begin : proc_data_toggle_d
365     data_toggle_d = data_toggle_q;
366
367     if (setup_token_received && ep_active) begin
368         data_toggle_d[out_ep_index_d] = 1'b0;
369     end else if (new_pkt_end) begin
370         data_toggle_d[out_ep_index] =
371         ~data_toggle_q[out_ep_index];
372     end
373     // Selective modification by software
374     if (out_datatog_we_i) begin
375         data_toggle_d = (data_toggle_d & ~out_datatog_mask_i) |
376         (out_datatog_status_i & out_data-
377         tog_mask_i);
378     end
379 end
380
381 // Supply current data toggles to register interface
382 assign out_data_toggle_o = data_toggle_q;
383
384 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
385     if (!rst_ni) begin
386         data_toggle_q <= '0; // Clear for all endpoints
387     end else if (link_reset_i) begin
388         data_toggle_q <= '0; // Clear for all endpoints
389     end else begin
390         data_toggle_q <= data_toggle_d;
391     end
392 end
393
394 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
395     if (!rst_ni) begin
396         out_ep_newpkt_o <= 1'b0;
397         out_ep_current_o <= '0;
398         current_xact_setup_q <= 1'b0;
399     end else begin
400         if (out_xact_start) begin
401             out_ep_newpkt_o <= 1'b1;
402             out_ep_current_o <= out_ep_current_d;
403             current_xact_setup_q <= setup_token_received;
404         end else begin
405             out_ep_newpkt_o <= 1'b0;
406         end
407     end

```

```

385     end
386   end
387
388   // put data strobe follows the rx strobe (which will latch
the data)
389   always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
390     if (!rst_ni) begin
391       out_ep_data_put_o <= 1'b0;
392     end else begin
393       out_ep_data_put_o <= ((out_xact_state == StRcvdData-
Start) && rx_data_put_i);
394     end
395   end
396
397   // nak an OUT if any data comes in with the endpoint full
398
399   // Note that if there is a full size packet buffer this will
only be all or nothing
400   // but in the case there was a FIFO with less than a max
packet size free you
401   // could get lucky and the packet received be small and fit
with no need to NAK
402   always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
403     if (!rst_ni) begin
404       nak_out_transaction <= 1'b0;
405     end else begin
406       if ((out_xact_state == StIdle) || (out_xact_state ==
StRcvdOut)) begin
407         nak_out_transaction <= 1'b0;
408       end else if (out_ep_data_put_o &&
out_ep_full_i[out_ep_index]) begin
409         nak_out_transaction <= 1'b1;
410       end
411     end
412
413     // address increment whenever there is a data put unless
414     // -- already going to NAK transaction and replay things
415     // -- the address is at max packet size
416     // NOTE if more than max packet size received then data is
lost
417     logic increment_addr;
418     assign increment_addr = !nak_out_transaction &&
(~&out_ep_put_addr_o) && out_ep_data_put_o;
419
420     always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
421       if (!rst_ni) begin
422         out_ep_put_addr_o <= '0;
423       end else begin
424         if (out_xact_state == StRcvdOut) begin
425           out_ep_put_addr_o <= '0;
426         end else if ((out_xact_state == StRcvdDataStart) && in-
crement_addr) begin
427           out_ep_put_addr_o <= out_ep_put_addr_o + 1;
428         end
429       end
430     end
431
432     ///////////////////////////////////////////////////////////////////
433     // Count OUT transactions that have been ACKed and dropped
because of unexpected Data Toggles.
434     ///////////////////////////////////////////////////////////////////
435     assign event_datatog_out_o = (out_xact_state == StRcvdData-
Start) &&
436                                     (ep_is_control || !
out_ep_iso_i[out_ep_index]) &&
437                                     !out_ep_stall_i[out_ep_index]
&& bad_data_toggle;
438
439     ///////////////////////////////////////////////////////////////////
440     // Assertions //
441     ///////////////////////////////////////////////////////////////////
442
443   endmodule

```

4. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_pe.sv (353 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 // Copyright Luke Valenty (TinyFPGA project, https://git-
hub.com/tinyfpga/TinyFPGA-Bootloader).
5
6 // USB Full Speed Non-Buffered Protocol Engine
7 //
8 // Decodes low level USB link protocol from external
usb_fs_tx/rx
9 // Provides upper level with
10 // - OUT endpoint interface (host to device) which also covers
SETUP
11 // - IN endpoint interface (device to host)
12 // - Start of Frame interface
13 //
14 // Based on usb_fs_pe.v from the TinyFPGA-Bootloader project

```

```

but
15 // this version contains no packet buffers
16
17 `include "prim_assert.sv"
18
19 module usb_fs_nb_pe #(
20   // Currently only accepts NumOutEps == NumInEps
21   parameter int unsigned NumOutEps = 2,
22   parameter int unsigned NumInEps = 2,
23   parameter int unsigned MaxPktSizeByte = 32,
24   localparam int unsigned PktW = $clog2(MaxPktSizeByte) // de-
rived parameter
25 ) (
26   input logic clk_48mhz_i,
27   input logic rst_ni, // Async. re-
set, active low
28   input logic link_reset_i, // USB reset,
sync to 48 MHz, active high
29   input logic link_active_i, // Device is
in Default/Addressed/Configured state
30   // and may re-
spond to transactions
31   input logic [6:0] dev_addr_i,
32
33   input logic cfg_eop_single_bit_i, // 1:
detect a single SE0 bit as EOP
34   input logic cfg_use_diff_rcvr_i, // 1:
use usb_d_i from a differential receiver
35   input logic cfg_pinflip_i, // 1: USB-side D+
and D- pins are flipped.
36   // Change val-
ues in logic to accommodate.
37   input logic tx_osc_test_mode_i, // Oscil-
lator test mode (constantly output JK)
38   output logic [NumOutEps-1:0] out_data_toggle_o, // Current
state of OUT data toggles
39   input logic out_datatog_we_i, // OUT data
toggles write strobe from software
40   input logic [NumOutEps-1:0] out_datatog_status_i, // New
state of selected OUT data toggles
41   input logic [NumOutEps-1:0] out_datatog_mask_i, // Which
OUT EP data toggles to modify
42   output logic [NumInEps-1:0] in_data_toggle_o, // Current
state of IN data toggles
43   input logic in_datatog_we_i, // IN data
toggles write strobe from software
44   input logic [NumInEps-1:0] in_datatog_status_i, // New
state of selected IN data toggles
45   input logic [NumInEps-1:0] in_datatog_mask_i, // Which
IN EP data toggles to modify
46   input logic diff_rx_ok_i, // 1: received
differential data symbols are valid.
47   // Set low if K
and J symbols might be invalid, such
48   // as when an
external differential receiver is
49   // powering on.
50
51   ///////////////////////////////////////////////////////////////////
52   // USB Endpoint Interface //
53   ///////////////////////////////////////////////////////////////////
54   ///////////////////////////////////////////////////////////////////
55   // global endpoint interface //
56   ///////////////////////////////////////////////////////////////////
57
58   // out endpoint interfaces
59   output logic [3:0] out_ep_current_o, // Other
signals address to this ep
60
61   output logic out_ep_data_put_o, // put the
data (put addr advances after)
62   output logic [PktW - 1:0] out_ep_put_addr_o, // Offset
to put data (0..pktlen)
63   output logic [7:0] out_ep_data_o,
out_ep_newpkt_o, // New OUT
pkt start (with in_ep_current_o update)
64   output logic out_ep_acked_o, // good ter-
mination, device has acked
65   output logic out_ep_rollback_o, // bad
termination, discard data
66   output logic [NumOutEps-1:0] out_ep_setup_o,
67   input logic [NumOutEps-1:0] out_ep_enabled_i, // Endpoint is
enabled and produces handshakes
68   input logic [NumOutEps-1:0] out_ep_control_i, // Is a
control endpoint: Accepts SETUP packets
69   input logic [NumOutEps-1:0] out_ep_full_i, // Cannot ac-
cept data
70   input logic [NumOutEps-1:0] out_ep_stall_i, // Stalled
71   input logic [NumOutEps-1:0] out_ep_iso_i, // Configure
endpoint in isochronous mode
72
73   // in endpoint interfaces
74   output logic in_xact_starting_o, // Start
of an IN transaction
75   output logic [3:0] in_xact_start_ep_o, // Cap-
ture IN packet details

```

```

76
77 output logic [3:0] in_ep_current_o, // Other
signals addressed to this ep
78 output logic in_ep_rollback_o, // Bad ter-
mination, rollback transaction
79 output logic in_ep_xact_end_o, // good
termination, transaction complete
80 output logic [PktW - 1:0] in_ep_get_addr_o, // Offset
requested (0..pktlen)
81 output logic in_ep_data_get_o, // Accept
data (get_addr advances too)

82 output logic in_ep_newpkt_o, // New IN pkt
start (with in_ep_current_o update)
83 input logic [NumInEps-1:0] in_ep_enabled_i, // Endpoint
is enabled and produces handshakes

84 input logic [NumInEps-1:0] in_ep_stall_i, // Endpoint in a
stall state

85 input logic [NumInEps-1:0] in_ep_has_data_i, // Endpoint
has data to supply
86 input logic [7:0] in_ep_data_i, // Data for
current get_addr
87 input logic [NumInEps-1:0] in_ep_data_done_i, // Set
when out of data
88 input logic [NumInEps-1:0] in_ep_iso_i, // Configure en-
dpoint in isochronous mode
89
90 // sof interface
91 output logic sof_detected_o, // SOF detec-
ted, PID valid
92 output logic sof_valid_o, // SOF packet
valid; frame number may be trusted
93 output logic [10:0] frame_index_o,
94
95 // RX line status
96 output logic rx_idle_det_o,
97 output logic rx_j_det_o,
98
99 // RX errors
100 output logic rx_crc5_err_o,
101 output logic rx_crc16_err_o,
102 output logic rx_pid_err_o,
103 output logic rx_bitstuff_err_o,
104
105 ///////////////////////////////////////////////////
106 // USB RX Interface (synchronous) //
107 ///////////////////////////////////////////////////
108 input logic usb_d_i,
109 input logic usb_dp_i,
110 input logic usb_dn_i,
111
112 ///////////////////////////////////////////////////
113 // USB TX Interface (synchronous) //
114 ///////////////////////////////////////////////////
115 output logic usb_d_o,
116 output logic usb_se0_o,
117 output logic usb_dp_o,
118 output logic usb_dn_o,
119 output logic usb_oe_o,
120
121 // event counters
122 output logic event_datatog_out_o,
123 output logic event_timeout_in_o,
124 output logic event_nak_in_o,
125 output logic event_nodata_in_o
126 );
127
128 import usb_consts_pkg::*;
129

130 // The code below assumes the number of OUT endpoints and IN
endpoints are
131 // interchangeable. Require them to be equal.
132 `ASSERT_INIT(NumOutEpsEqualsNumInEps_A, NumOutEps == Nu-
mInEps)
133
134 // rx interface
135 logic bit_strobe;
136 logic rx_pkt_start;
137 logic rx_pkt_end;
138 logic [3:0] rx_pid;
139 logic [6:0] rx_addr;
140 logic [3:0] rx_endp;
141 logic [10:0] rx_frame_num;
142 logic rx_data_put;
143 logic [7:0] rx_data;
144 logic rx_pid_valid;
145 logic rx_pkt_valid;
146
147 // tx mux
148 logic in_tx_pkt_start;
149 logic [3:0] in_tx_pid;
150 logic out_tx_pkt_start;
151 logic [3:0] out_tx_pid;
152
153 // tx interface
154 logic tx_pkt_start;
155 logic tx_pkt_end;
156 logic [3:0] tx_pid;

157 logic tx_data_avail;
158 logic tx_data_get;
159 logic [7:0] tx_data;
160
161 logic usb_oe;
162
163 // Start Of Frame handling
164 //
165 // - SOF is detected and may be used for timing/synchroniza-
tion purposes if the PID is valid.
166 assign sof_detected_o = rx_pkt_end & rx_pid_valid &
(usb_pid_e'(rx_pid) == UsbPidSof);
167 // - The SOF packet is considered valid iff the packet is
also free of bit stuffing violations
168 // and the CRC5 is correct.
169 assign sof_valid_o = rx_pkt_end & rx_pkt_valid &
(usb_pid_e'(rx_pid) == UsbPidSof);
170 // - The frame number shall be used only if the SOF packet
is valid.
171 assign frame_index_o = rx_frame_num;
172 assign usb_oe_o = usb_oe;
173
174 // IN ep type configuration
175 logic [NumInEps-1:0] in_ep_iso_not_control;
176 assign in_ep_iso_not_control = in_ep_iso_i & ~out_ep_con-
trol_i;
177
178 usb_fs_nb_in_pe #(
179 .NumInEps (NumInEps),
180 .MaxInPktSizeByte (MaxPktSizeByte)
181 ) u_usb_fs_nb_in_pe (
182 .clk_48mhz_i (clk_48mhz_i),
183 .rst_ni (rst_ni),
184 .link_reset_i (link_reset_i),
185 .link_active_i (link_active_i),
186 .dev_addr_i (dev_addr_i),
187
188 // transaction starting
189 .in_xact_starting_o (in_xact_starting_o),
190 .in_xact_start_ep_o (in_xact_start_ep_o),
191
192 // endpoint interface
193 .in_ep_current_o (in_ep_current_o),
194 .in_ep_rollback_o (in_ep_rollback_o),
195 .in_ep_xact_end_o (in_ep_xact_end_o),
196 .in_ep_get_addr_o (in_ep_get_addr_o),
197 .in_ep_data_get_o (in_ep_data_get_o),
198 .in_ep_newpkt_o (in_ep_newpkt_o),
199 .in_ep_enabled_i (in_ep_enabled_i),
200 .in_ep_stall_i (in_ep_stall_i),
201 .in_ep_has_data_i (in_ep_has_data_i),
202 .in_ep_data_i (in_ep_data_i),
203 .in_ep_data_done_i (in_ep_data_done_i),
204 .in_ep_iso_i (in_ep_iso_not_control),
205
206 .in_data_toggle_o (in_data_toggle_o),
207 .in_datatog_we_i (in_datatog_we_i),
208 .in_datatog_status_i (in_datatog_status_i),
209 .in_datatog_mask_i (in_datatog_mask_i),
210
211 // rx path
212 .rx_pkt_start_i (rx_pkt_start),
213 .rx_pkt_end_i (rx_pkt_end),
214 .rx_pkt_valid_i (rx_pkt_valid),
215 .rx_pid_i (rx_pid),
216 .rx_addr_i (rx_addr),
217 .rx_endp_i (rx_endp),
218
219 // tx path
220 .tx_pkt_start_o (in_tx_pkt_start),
221 .tx_pkt_end_i (tx_pkt_end),
222 .tx_pid_o (in_tx_pid),
223 .tx_data_avail_o (tx_data_avail),
224 .tx_data_get_i (tx_data_get),
225 .tx_data_o (tx_data),
226
227 // event counters
228 .event_timeout_in_o (event_timeout_in_o),
229 .event_nak_in_o (event_nak_in_o),
230 .event_nodata_in_o (event_nodata_in_o)
231 );
232
233 usb_fs_nb_out_pe #(
234 .NumOutEps (NumOutEps),
235 .MaxOutPktSizeByte (MaxPktSizeByte)
236 ) u_usb_fs_nb_out_pe (
237 .clk_48mhz_i (clk_48mhz_i),
238 .rst_ni (rst_ni),
239 .link_reset_i (link_reset_i),
240 .dev_addr_i (dev_addr_i),
241
242 // endpoint interface
243 .out_ep_current_o (out_ep_current_o),
244 .out_ep_data_put_o (out_ep_data_put_o),
245 .out_ep_put_addr_o (out_ep_put_addr_o),
246 .out_ep_data_o (out_ep_data_o),
247 .out_ep_newpkt_o (out_ep_newpkt_o),
248 .out_ep_acked_o (out_ep_acked_o),
249 .out_ep_rollback_o (out_ep_rollback_o),
250 .out_ep_setup_o (out_ep_setup_o),
251 .out_ep_enabled_i (out_ep_enabled_i),

```



```

252 .out_ep_control_i      (out_ep_control_i),
253 .out_ep_full_i        (out_ep_full_i),
254 .out_ep_stall_i       (out_ep_stall_i),
255 .out_ep_iso_i         (out_ep_iso_i),
256
257 .out_data_toggle_o    (out_data_toggle_o),
258 .out_datatog_we_i     (out_datatog_we_i),
259 .out_datatog_status_i (out_datatog_status_i),
260 .out_datatog_mask_i   (out_datatog_mask_i),
261
262 // rx path
263 .rx_pkt_start_i       (rx_pkt_start),
264 .rx_pkt_end_i         (rx_pkt_end),
265 .rx_pkt_valid_i       (rx_pkt_valid),
266 .rx_pid_i             (rx_pid),
267 .rx_addr_i           (rx_addr),
268 .rx_endp_i           (rx_endp),
269 .rx_data_put_i        (rx_data_put),
270 .rx_data_i            (rx_data),
271
272 // tx path
273 .tx_pkt_start_o       (out_tx_pkt_start),
274 .tx_pkt_end_i         (tx_pkt_end),
275 .tx_pid_o             (out_tx_pid),
276
277 // event counters
278 .event_datatog_out_o  (event_datatog_out_o)
279 );
280
281 usb_fs_rx u_usb_fs_rx (
282     .clk_i            (clk_48mhz_i),
283     .rst_ni           (rst_ni),
284     .link_reset_i     (link_reset_i),
285     .cfg_eop_single_bit_i (cfg_eop_single_bit_i),
286     .cfg_use_diff_rcvr_i (cfg_use_diff_rcvr_i),
287     .cfg_pinflip_i     (cfg_pinflip_i),
288     .diff_rx_ok_i     (diff_rx_ok_i),
289     .usb_d_i          (usb_d_i),
290     .usb_dp_i         (usb_dp_i),
291     .usb_dn_i         (usb_dn_i),
292     .tx_en_i          (usb_oe),
293     .bit_strobe_o     (bit_strobe),
294     .pkt_start_o      (rx_pkt_start),
295     .pkt_end_o        (rx_pkt_end),
296     .pid_o            (rx_pid),
297     .addr_o           (rx_addr),
298     .endp_o           (rx_endp),
299     .frame_num_o      (rx_frame_num),
300     .rx_data_put_o    (rx_data_put),
301     .rx_data_o        (rx_data),
302     .valid_pid_o      (rx_pid_valid),
303     .valid_packet_o   (rx_pkt_valid),
304     .rx_idle_det_o    (rx_idle_det_o),
305     .rx_j_det_o       (rx_j_det_o),
306     .crc5_error_o     (rx_crc5_err_o),
307     .crc16_error_o    (rx_crc16_err_o),
308     .pid_error_o      (rx_pid_err_o),
309     .bitstuff_error_o (rx_bitstuff_err_o)
310 );
311
312 usb_fs_tx_mux u_usb_fs_tx_mux (
313     // interface to IN Protocol Engine
314     .in_tx_pkt_start_i (in_tx_pkt_start),
315     .in_tx_pid_i       (in_tx_pid),
316
317     // interface to OUT Protocol Engine
318     .out_tx_pkt_start_i (out_tx_pkt_start),
319     .out_tx_pid_i       (out_tx_pid),
320
321     // interface to tx module
322     .tx_pkt_start_o    (tx_pkt_start),
323     .tx_pid_o          (tx_pid)
324 );
325
326 usb_fs_tx u_usb_fs_tx (
327     .clk_i            (clk_48mhz_i),
328     .rst_ni           (rst_ni),
329     .link_reset_i     (link_reset_i),
330     .cfg_pinflip_i     (cfg_pinflip_i),
331     .tx_osc_test_mode_i (tx_osc_test_mode_i),
332     .bit_strobe_i     (bit_strobe),
333     .usb_d_o          (usb_d_o),
334     .usb_se0_o        (usb_se0_o),
335     .usb_dp_o         (usb_dp_o),
336     .usb_dn_o         (usb_dn_o),
337     .usb_oe_o         (usb_oe),
338     .pkt_start_i      (tx_pkt_start),
339     .pkt_end_o        (tx_pkt_end),
340     .pid_i            (tx_pid),
341     .tx_data_avail_i  (tx_data_avail),
342     .tx_data_get_o    (tx_data_get),
343     .tx_data_i        (tx_data)
344 );
345
346 ///////////////////////////////////////////////////////////////////
347 // Assertions //
348 ///////////////////////////////////////////////////////////////////
349 `ASSERT_INIT(ParamNumEpsOutAndInEqual, NumOutEps == NumInEps)
350 `ASSERT_INIT(ParamNumOutEpsValid, (NumOutEps > 0) && (NumOutEps <= 16))

```

```

351 `ASSERT_INIT(ParamNumInEpsValid, (NumInEps > 0) && (NumInEps <= 16))
352 `ASSERT_INIT(ParamMaxPktSizeByteValid, (MaxPktSizeByte >= 8)
353 && (MaxPktSizeByte <= 64))
354 endmodule

```

5. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_rx.sv (673 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 // Copyright Luke Valenty (TinyFPGA project, https://git-
  hub.com/tinyfpga/TinyFPGA-Bootloader).
5
6 module usb_fs_rx (
7     // A 48MHz clock is required to recover the clock from the
  incoming data.
8     input logic clk_i,
9     input logic rst_ni,
10    input logic link_reset_i,
11
12    // configuration
13    input logic cfg_eop_single_bit_i,
14    input logic cfg_use_diff_rcvr_i,
15    input logic cfg_pinflip_i,
16    input logic diff_rx_ok_i,
17
18    // USB data+ and data- lines (synchronous)
19    input logic usb_d_i,
20    input logic usb_dp_i,
21    input logic usb_dn_i,
22
23    // Transmit enable disables the receiver
24    input logic tx_en_i,
25
26    // pulse on every bit transition.
27    output logic bit_strobe_o,
28
29    // Pulse on detecting a start of packet symbol (i.e. the be-
  ginning of sync)
30    output logic pkt_start_o,
31
32    // Pulse on end of current packet.
33    output logic pkt_end_o,
34
35    // Most recent packet decoded.
36    output logic [3:0] pid_o,
37    output logic [6:0] addr_o,
38    output logic [3:0] endp_o,
39    output logic [10:0] frame_num_o,
40
41    // Pulse on valid data on rx_data. Will not go high for
  token or handshake
42    // packets.
43    output logic rx_data_put_o,
44    output logic [7:0] rx_data_o,
45
46    // Most recent packet passes PID check
47    output logic valid_pid_o,
48    // Most recent packet passes PID, length and CRC checks
49    output logic valid_packet_o,
50
51    // line status for the status detection (actual rx bits
  after clock recovery)
52    output logic rx_idle_det_o,
53    output logic rx_j_det_o,
54
55    // Error detection
56    output logic crc5_error_o,
57    output logic crc16_error_o,
58    output logic pid_error_o,
59    output logic bitstuff_error_o
60 );
61
62 import usb_consts_pkg::*;
63
64 logic [6:0] bitstuff_history_q, bitstuff_history_d;
65 logic bitstuff_error;
66 logic bitstuff_error_q, bitstuff_error_d;
67
68 ///////////////////////////////////////////////////////////////////
69 // usb receive path //
70 ///////////////////////////////////////////////////////////////////
71 // Adjust inputs when D+/D- are flipped on the USB side
72 logic usb_dp_flipped, usb_dn_flipped, usb_d_flipped;
73 assign usb_dp_flipped = cfg_pinflip_i ? usb_dn_i : usb_dp_i;
74 assign usb_dn_flipped = cfg_pinflip_i ? usb_dp_i : usb_dn_i;
75 assign usb_d_flipped = usb_d_i ^ cfg_pinflip_i;
76
77 ///////////////////////////////////////////////////////////////////
78 // line state recovery state machine //
79 ///////////////////////////////////////////////////////////////////
80
81 // If the receive path is set NOT to use an external differ-
  ential receiver:

```

```

82 // This block samples data purely from the usb_dp_i/usb_dn_i
pair.
83 // There is a chance that one of the signals in the pair
will appear to have
84 // changed to the new state while the other is still in the
old state. The
85 // following state machine detects transitions and waits an
extra sampling clock
86 // before decoding the state on the dp/dn pair. This trans-
ition period will

87 // only ever last for one clock as long as there is no noise on
the line.

88 // If there is enough noise on the line then the data may be
corrupted and the
89 // packet will fail the data integrity checks.
90
91 // If the receive path uses an external differential receiv-
er:
92 // This block uses the usb_d_i input to detect K and J sym-
bols.
93 // The individual signals of the differential pair must
still be connected
94 // to this block to detect SE0.
95 // Note that the spec warns in section 7.1.4.1:
96 // Both D+ and D- may temporarily be less than VIH (min)
during differential

97 // signal transitions. This period can be up to 14 ns (TFST)
for full-speed
98 // transitions and up to 210 ns (TLST) for low-speed trans-
itions. Logic in the
99 // receiver must ensure that this is not interpreted as an
SE0.
100 // Since the 48MHz sample clock is 20.833ns period we will
either miss this or
101 // sample it only once, so it will be covered by
line_state=DT and the next
102 // sample will not be SE0 unless this was a real SE0 trans-
ition
103 // Note: if it is a real SE0 the usb_d_i input could be do-
ing anything.
104
105 logic [2:0] line_state_qq, line_state_q, line_state_d;
106 logic [2:0] diff_state_q, diff_state_d;
107 logic [2:0] line_state_rx;
108 logic
109     use_se;
110
111 localparam logic [2:0] DT = 3'b100; // transition state
112 localparam logic [2:0] DJ = 3'b010; // J - idle line state
113 localparam logic [2:0] DK = 3'b001; // K - inverse of J

114 localparam logic [2:0] SE0 = 3'b000; // single-ended 0 - end
of packet or detached
115 // localparam logic [2:0] SE1 = 3'b011; // single-ended 1 -
illegal
116
117 // Mute the input if we're transmitting
118 // dpair is the usb_dp_i/usb_dn_i pair, used in both modes.
With
119 // an external differential receiver, it is only used for
detecting SE0 and
120 // transitions. Without an external differential receiver
driving the
121 // usb_d_i input, it is used for all symbols.
122 // ddiff is the decoded data input from an external differ-
ential receiver,
123 // if available, and it is only for K and J symbols, plus
transition detection.
124 logic [1:0] dpair, ddiff;
125 always_comb begin : proc_dpair_mute
126     if (tx_en_i) begin
127         dpair = DJ[1:0]; // J
128         ddiff = DJ[1:0]; // J
129     end else begin
130         dpair = {usb_dp_flipped, usb_dn_flipped};
131         ddiff = usb_d_flipped ? DJ[1:0] : DK[1:0]; // equiv to
{usb_d_i, ~usb_d_i}
132     end
133 end
134
135 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_line_state_q
136     if (!rst_ni) begin
137         line_state_q <= SE0;
138         line_state_qq <= SE0;
139         diff_state_q <= SE0;
140     end else begin
141         if (link_reset_i) begin
142             line_state_q <= SE0;
143             line_state_qq <= SE0;
144             diff_state_q <= SE0;
145         end else begin
146             line_state_q <= line_state_d;
147             line_state_qq <= line_state_q;
148             diff_state_q <= diff_state_d;
149         end
150     end
151 end

```

```

152 always_comb begin : proc_line_state_d
153     // Default assignment
154     line_state_d = line_state_q;
155
156     if (line_state_q == DT) begin
157         // if we are in a transition state, then we can sample
the pair and
158         // move to the next corresponding line state
159         line_state_d = {1'b0, dpair};
160     end else begin
161         // if we are in a valid line state and the value of the
pair changes,
162         // then we need to move to the transition state
163         if (dpair != line_state_q[1:0]) begin
164             line_state_d = DT;
165         end
166     end
167 end
168
169 always_comb begin : proc_diff_state_d
170     // Default assignment
171     diff_state_d = diff_state_q;
172
173     if (diff_state_q == DT) begin
174         // if we are in a transition state, then we can sample
the diff input and
175         // move to the next corresponding line state
176         diff_state_d = {1'b0, ddiff};
177     end else begin
178         // if we are in a valid line state and the value of the
diff input changes,
179         // then we need to move to the transition state
180         if (ddiff != diff_state_q[1:0]) begin
181             diff_state_d = DT;
182         end
183     end
184 end
185
186 // The received line state depends on how the receiver is
configured:
187 // NOT using a differential receiver: it is just the
line_state_q that was captured
188 // Using a differential receiver: recovered from the differ-
ential receiver (diff_state_q)
189 // unless the diff pair in-
dicate SE0 when the differential
190 // receiver could produce any
value
191 // Transition where the dp/dn pair happen to see SE0 will
look like (driven by diff DT)
192 // line_state D? DT D?...
193 // diff_state Dx DT Dy (expect Dy to be inverse
of Dx since diff changed)
194 // Transition to SE0 when usb_d_i changes will look like:
195 // line_state DT D? D? D? DT SE0 SE0... (DT is the first
sample at SE0)
196 // diff_state DT Dx Dx Dx DT ?? ??... (diff saw trans-
ition as line went SE0)
197
198 // --> out DT Dx Dx Dx DT SE0 SE0 (if no transition
then DT would be Dx and n=3)
199
200 // bit_phase n 0 1 2 3 0 1 (n=3 unless there
was a clock resync)
201 // Transition to SE0 when usb_d_i does not change will look
like:
202 // line_state DT D? D? D? DT SE0 SE0... (DT is the first
sample at SE0)
203 // diff_state DT Dx Dx Dx Dx ?? ??... (diff no trans-
ition as line went SE0)
204
205 // --> out DT Dx Dx Dx Dx SE0 SE0 (if no transition
then DT would be Dx and n=3)
206
207 // bit_phase n 0 1 2 3 0 1 (n=3 unless there
was a clock resync)
208 // Transition to SE0 when usb_d_i does not change and clock
resync earlier:
209 // line_state DT D? D? DT SE0 SE0 SE0... (DT is the first
sample at SE0, should resync clock)
210 // diff_state DT Dx Dx Dx Dx ?? ??... (diff no trans-
ition as line went SE0)
211 // --> out DT Dx Dx Dx SE0 SE0 SE0 (if no trans-
ition then DT would be Dx and n=3)
212 // bit_phase n 0 1 2 3 0 1 (n=3 unless
there was a clock resync)
213 // On transition back from SE0 want to generate a DT to re-
sync the clock
214 // since SE0 could have gone on a while no idea what
bit_phase is
215 // line_state SE0 SE0 DT D? D? D?

```

```

220 // diff_state ?? ?? ?? Dx Dx Dx
221 // --> out SE0 SE0 DT Dx Dx Dx
222 // bit_phase ? ? ? 0 1 2
223
224 assign use_se = (line_state_q == SE0) || ((line_state_q ==
DT) && (line_state_q == SE0));
225 assign line_state_rx = cfg_use_diff_rcvr_i ? (use_se ?
line_state_q : diff_state_q) :
line_state_q;
226
227 // clock recovery
228 // clock recovery
229 // clock recovery
230 // clock recovery
231
232 // the DT state from the line state recovery state machine
is used to align to
233 // transmit clock. the line state is sampled in the middle
of the bit time.
234
235 // example of signal relationships
236 // -----
237 // line_state DT DJ DJ DJ DT DK DK DK DK DK
DK DT DJ DJ DJ
238 // line_state_valid -----
239 // bit_phase 0 0 1 2 3 0 1 2 3 0
1 2 0 1 2
240
241
242 logic [1:0] bit_phase_q, bit_phase_d;
243 logic line_state_valid;
244
245 assign line_state_valid = (bit_phase_q == 2'd1);
246 assign bit_strobe_o = (bit_phase_q == 2'd2);
247
248 // keep track of phase within each bit
249 assign bit_phase_d = (line_state_rx == DT) ? 0 : bit_phase_q +
1;
250
251 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_bit_phase_q
252 if (!rst_ni) begin
253 bit_phase_q <= 0;
254 end else begin
255 if (link_reset_i) begin
256 bit_phase_q <= 0;
257 end else begin
258 bit_phase_q <= bit_phase_d;
259 end
260 end
261 end
262
263
264 // packet detection
265 // packet detection
266 // packet detection
267
268 // usb uses a sync to denote the beginning of a packet's PID
and two
269 // single-ended-0 to denote the end of a packet. this state
machine
270 // recognizes the beginning and end of packets for sub-
sequent layers to
271 // process.
272
273 logic [11:0] line_history_q, line_history_d;
274 logic packet_valid_q, packet_valid_d;
275 logic see_sop, see_eop, see_preamble, packet_start, pack-
et_end;
276 logic in_packet_d, in_packet_q;
277
278 // A bit of a misnomer: packet_start pulses when the PID be-
gins, not SOP.
279 assign packet_start = packet_valid_d & ~packet_valid_q;
280 assign packet_end = ~packet_valid_d & packet_valid_q;
281
282 // EOP detection is configurable for 1/2 bit periods of SE0.
283 // The standard (Table 7-7) mandates min = 82 ns = 1 bit
period.
284 // We also trigger an EOP on seeing a bitstuff error or a
PRE PID.
285 assign see_eop = (cfg_eop_single_bit_i && line_his-
tory_q[1:0] == 2'b00);
286
287 // SOP is the transition from idle (J) to K
288
289 assign see_sop = (line_history_q[3:0] == 4'b1001) & ~tx_en_i &
~in_packet_q;
290 assign in_packet_d = see_eop ? 1'b0 :
see_sop ? 1'b1 : in_packet_q;
291
292 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_reg_in_packet

```

```

294 if (!rst_ni) begin
295 in_packet_q <= 1'b0;
296 end else begin
297 in_packet_q <= in_packet_d;
298 end
299 end
300
301 always_comb begin : proc_packet_valid_d
302 if (line_state_valid) begin
303 // If the differential K and J symbols are not valid,
reject the
304 // containing packet as invalid.
305 if (~diff_rx_ok_i) begin
306 packet_valid_d = 0;
307 // check for packet start: KJKJKK, we use the last 6
bits
308 end else if (!packet_valid_q && line_history_q[11:0] ==
12'b011001100101) begin
309 packet_valid_d = 1;
310 end
311
312 // check for packet end: SE0 SE0
313 else if (packet_valid_q && see_eop) begin
314 packet_valid_d = 0;
315
316 end else begin
317 packet_valid_d = packet_valid_q;
318 end
319 end else begin
320 packet_valid_d = packet_valid_q;
321 end
322 end
323
324 // keep a history of the last two states on the line
325 assign line_history_d = line_state_valid ? {line_his-
tory_q[9:0], line_state_rx[1:0]} :
line_history_q;
326
327 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_reg_pkt_line
328 if (!rst_ni) begin
329 packet_valid_q <= 0;
330 line_history_q <= 12'b0101010101010; // all K
331 end else begin
332 if (link_reset_i) begin
333 packet_valid_q <= 0;
334 line_history_q <= 12'b0101010101010; // all K
335 end else begin
336 packet_valid_q <= packet_valid_d;
337 line_history_q <= line_history_d;
338 end
339 end
340 end
341 end
342
343 // mask out idle detection when transmitting (because rx may
be forced to
344 // J and look like an idle symbol)
345 logic rx_idle_det_d, rx_idle_det_q;
346 assign rx_idle_det_d = (~tx_en_i & line_state_valid) ?
(line_state_q == DJ) : rx_idle_det_q;
347 assign rx_idle_det_o = rx_idle_det_q;
348
349 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_reg_idle_det
350 if (!rst_ni) begin
351 rx_idle_det_q <= 1'b0;
352 end else begin
353 rx_idle_det_q <= rx_idle_det_d;
354 end
355 end
356
357 // Used for seeing a J after the completion of resume sig-
naling
358 assign rx_j_det_o = diff_rx_ok_i & ~tx_en_i & (line_his-
tory_q[1:0] == 2'b10);
359
360 // NRZI decode
361 // NRZI decode
362
363
364 // in order to ensure there are enough bit transitions for a
receiver to recover
365 // the clock usb uses NRZI encoding.
366
367 // https://en.wikipedia.org/wiki/Non-return-to-zero
368
369 logic dvalid_raw;
370 logic din;
371
372 always_comb begin
373 unique case (line_history_q[3:0])
374 4'b0101 : din = 1;
375 4'b0110 : din = 0;
376 4'b1001 : din = 0;
377 4'b1010 : din = 1;
378 default : din = 0;
379 endcase
380
381 if (packet_valid_q && line_state_valid) begin

```



```

382     unique case (line_history_q[3:0])
383     4'b0101 : dvalid_raw = 1;
384     4'b0110 : dvalid_raw = 1;
385     4'b1001 : dvalid_raw = 1;
386     4'b1010 : dvalid_raw = 1;
387     default : dvalid_raw = 0;
388   endcase
389   end else begin
390     dvalid_raw = 0;
391   end
392 end
393
394 ///////////////////////////////////////////////////////////////////
395 // Undo bit stuffing and detect bit stuffing errors //
396 ///////////////////////////////////////////////////////////////////
397
398 always_comb begin : proc_bitstuff_history_d
399   if (packet_end) begin
400     bitstuff_history_d = '0;
401   end else if (dvalid_raw) begin
402     bitstuff_history_d = {bitstuff_history_q[5:0], din};
403   end else begin
404     bitstuff_history_d = bitstuff_history_q;
405   end
406 end
407
408 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_bitstuff_history_q
409   if (!rst_ni) begin
410     bitstuff_history_q <= 0;
411   end else begin
412     if (link_reset_i) begin
413       bitstuff_history_q <= 0;
414     end else begin
415       bitstuff_history_q <= bitstuff_history_d;
416     end
417   end
418 end
419
420 logic dvalid;
421 assign dvalid = dvalid_raw && !(bitstuff_history_q[5:0] ==
6'b111111);
422
423 // 7 consecutive ones should not be seen on the bus
424 // USB spec, 7.1.9.1: "If the receiver sees seven
425 // consecutive ones anywhere in the packet, then a bit
stuffing error
426 // has occurred and the packet should be ignored."
427 assign bitstuff_error = bitstuff_history_q == 7'b1111111;
428
429 // remember the bitstuff errors
430 always_comb begin : proc_bitstuff_error_d
431   bitstuff_error_d = bitstuff_error_q;
432   if (packet_start) begin
433     bitstuff_error_d = 0;
434   end else if (bitstuff_error && dvalid_raw) begin
435     bitstuff_error_d = 1;
436   end
437 end
438
439 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_bitstuff_error_q
440   if (!rst_ni) begin
441     bitstuff_error_q <= 0;
442   end else begin
443     bitstuff_error_q <= bitstuff_error_d;
444   end
445 end
446
447 assign bitstuff_error_o = bitstuff_error_q && packet_end;
448
449
450 ///////////////////////////////////////////////////////////////////
451 // save and check pid //
452 ///////////////////////////////////////////////////////////////////
453
454 // shift in the entire 8-bit pid with an additional 9th bit
used as a sentinel.
455
456 logic [8:0] full_pid_q, full_pid_d;
457 logic pid_valid, pid_complete;
458
459 assign pid_valid = full_pid_q[4:1] == ~full_pid_q[8:5];
460 assign pid_complete = full_pid_q[0];
461
462 always_comb begin : proc_full_pid_d
463   if (dvalid && !pid_complete) begin
464     full_pid_d = {din, full_pid_q[8:1]};
465   end else if (packet_start) begin
466     full_pid_d = 9'b100000000;
467   end else begin
468     full_pid_d = full_pid_q;
469   end
470 end
471
472 ///////////////////////////////////////////////////////////////////
473 // check crc5 //
474 ///////////////////////////////////////////////////////////////////
475 logic [4:0] crc5_q, crc5_d;
476 logic crc5_valid, crc5_invert;
477 assign crc5_valid = crc5_q == 5'b01100;

```

```

478 assign crc5_invert = din ^ crc5_q[4];
479
480 always_comb begin
481   crc5_d = crc5_q; // default value
482
483   if (packet_start) begin
484     crc5_d = 5'b11111;
485   end
486
487   if (dvalid && pid_complete) begin
488     crc5_d = {crc5_q[3:0], 1'b0} ^ ({5{crc5_invert}} &
5'b00101);
489   end
490 end
491
492
493 ///////////////////////////////////////////////////////////////////
494 // check crc16 //
495 ///////////////////////////////////////////////////////////////////
496 logic [15:0] crc16_q, crc16_d;
497 logic crc16_valid, crc16_invert;
498
499 assign crc16_valid = crc16_q == 16'b100000000000101;
500 assign crc16_invert = din ^ crc16_q[15];
501
502 always_comb begin
503   crc16_d = crc16_q; // default value
504
505   if (packet_start) begin
506     crc16_d = 16'b111111111111111;
507   end
508
509   if (dvalid && pid_complete) begin
510     crc16_d = {crc16_q[14:0], 1'b0} ^ ({16{crc16_invert}} &
16'b100000000000101);
511   end
512 end
513
514
515 ///////////////////////////////////////////////////////////////////
516 // output control signals //
517 ///////////////////////////////////////////////////////////////////
518 logic valid_token_len, valid_data_len, valid_handshake_len;
519
520 logic pkt_is_token, pkt_is_data, pkt_is_handshake;
521 assign pkt_is_token = full_pid_q[2:1] == 2'b01;
522 assign pkt_is_data = full_pid_q[2:1] == 2'b11;
523 assign pkt_is_handshake = full_pid_q[2:1] == 2'b10;
524
525 assign see_preamble = packet_valid_q & pid_valid & pid_com-
plete &&
526   (usb_pid_e'(full_pid_q[4:1]) == Us-
bPidPre);
527
528 assign valid_pid_o = pid_valid;
529
530 assign valid_packet_o = pid_valid && !bitstuff_error_q &&
((pkt_is_handshake && valid_handshake_len) ||
531  (pkt_is_data && valid_data_len && crc16_valid) ||
532  (pkt_is_token && valid_token_len && crc5_valid));
533
534
535 // Detect CRC errors
536 assign crc5_error_o = pkt_is_token & packet_end & !
crc5_valid;
537 assign crc16_error_o = pkt_is_data & packet_end & !
crc16_valid;
538
539 // Detect PID errors
540 assign pid_error_o = !pid_valid && packet_end;
541
542 logic [11:0] token_payload_q, token_payload_d;
543 logic token_payload_done;
544
545 assign token_payload_done = token_payload_q[0];
546
547 logic [6:0] addr_q, addr_d;
548 logic [3:0] endp_q, endp_d;
549 logic [10:0] frame_num_q, frame_num_d;
550
551 always_comb begin
552   token_payload_d = token_payload_q; // default
553
554   if (packet_start) begin
555     token_payload_d = 12'b100000000000;
556   end
557
558   if (dvalid && pid_complete && pkt_is_token && !
token_payload_done) begin
559     token_payload_d = {din, token_payload_q[11:1]};
560   end
561 end
562
563 always_comb begin
564   // defaults
565   addr_d = addr_q;
566   endp_d = endp_q;
567   frame_num_d = frame_num_q;
568
569   if (token_payload_done && pkt_is_token) begin
570     addr_d = token_payload_q[7:1];
571     endp_d = token_payload_q[11:8];

```

```

572     frame_num_d = token_payload_q[11:1];
573 end
574 end
575
576 assign addr_o      = addr_q;
577 assign endp_o      = endp_q;
578 assign frame_num_o = frame_num_q;
579 assign pid_o       = full_pid_q[4:1];
580
581 assign pkt_start_o = see_sop;
582 assign pkt_end_o   = packet_end;
583
584
585 ///////////////////////////////////////////////////
586 // deserialize and output data //
587 ///////////////////////////////////////////////////
588 logic [8:0] rx_data_buffer_q, rx_data_buffer_d;
589 logic rx_data_buffer_full;
590
591 assign rx_data_buffer_full = rx_data_buffer_q[0];
592 assign rx_data_put_o      = rx_data_buffer_full;
593 assign rx_data_o          = rx_data_buffer_q[8:1];
594
595 always_comb begin
596     rx_data_buffer_d = rx_data_buffer_q; // default
597
598     if (packet_start || rx_data_buffer_full) begin
599         rx_data_buffer_d = 9'b100000000;
600     end
601
602     if (dvalid && pid_complete && pkt_is_data) begin
603         rx_data_buffer_d = {din, rx_data_buffer_q[8:1]};
604     end
605 end
606
607 // Counting packet length in bits; we need to know whether
the length is byte-aligned,
608 // and whether it's equal to 16 (token packet) or at least
16 (data packet; CRC16 required).
609 logic rx_data_len16_q, rx_data_len16_d;
610 logic [3:0] rx_data_len_q, rx_data_len_d;
611
612 // Valid packet length indications.
613 assign valid_token_len = rx_data_len16_q & ~|
rx_data_len_q;
614 assign valid_data_len = rx_data_len16_q & ~|
rx_data_len_q[2:0]; // Must be byte-aligned.
615 assign valid_handshake_len = ~|{rx_data_len16_q,
rx_data_len_q};
616
617 always_comb begin
618     rx_data_len16_d = rx_data_len16_q;
619     rx_data_len_d   = rx_data_len_q;
620
621     if (packet_start) begin
622         rx_data_len16_d = 1'b0;
623         rx_data_len_d   = 4'h0;
624     end
625     if (dvalid && pid_complete) begin
626         rx_data_len16_d = rx_data_len16_q | &rx_data_len_q;
627         rx_data_len_d   = rx_data_len_q + 4'h1;
628     end
629 end
630
631 ///////////////////////////////////////////////////
632 // Registers //
633 ///////////////////////////////////////////////////
634 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_gp_regs
635     if (!rst_ni) begin
636         full_pid_q      <= 0;
637         crc16_q         <= 0;
638         crc5_q          <= 0;
639         token_payload_q <= 0;
640         addr_q          <= 0;
641         endp_q          <= 0;
642         frame_num_q     <= 0;
643         rx_data_buffer_q <= 0;
644         rx_data_len16_q <= 0;
645         rx_data_len_q   <= 0;
646     end else begin
647         if (link_reset_i) begin
648             full_pid_q      <= 0;
649             crc16_q         <= 0;
650             crc5_q          <= 0;
651             token_payload_q <= 0;
652             addr_q          <= 0;
653             endp_q          <= 0;
654             frame_num_q     <= 0;
655             rx_data_buffer_q <= 0;
656             rx_data_len16_q <= 0;
657             rx_data_len_q   <= 0;
658         end else begin
659             full_pid_q      <= full_pid_d;
660             crc16_q         <= crc16_d;
661             crc5_q          <= crc5_d;
662             token_payload_q <= token_payload_d;
663             addr_q          <= addr_d;
664             endp_q          <= endp_d;
665             frame_num_q     <= frame_num_d;
666             rx_data_buffer_q <= rx_data_buffer_d;

```

```

667     rx_data_len16_q      <= rx_data_len16_d;
668     rx_data_len_q       <= rx_data_len_d;
669 end
670 end
671 end
672
673 endmodule // usb_fs_rx

```

6. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_tx.sv (508 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 // Copyright Luke Valenty (TinyFPGA project, https://git-
hub.com/tinyfpga/TinyFPGA-Bootloader).
5
6 module usb_fs_tx (
7     // A 48MHz clock is required to receive USB data at 12MHz
8     // it's simpler to just use 48MHz everywhere
9     input logic clk_i,
10    input logic rst_ni, // async reset
11    input logic link_reset_i, // USB reset, sync to 48 MHz,
active high
12    input logic cfg_pinflip_i,
13
14    // Oscillator test mode (constantly output JK)
15    input logic tx_osc_test_mode_i,
16
17    // bit strobe from rx to align with senders clock
18    input logic bit_strobe_i,
19
20    // output enable to take ownership of bus and data out
21    output logic usb_oe_o,
22    output logic usb_d_o,
23    output logic usb_se0_o,
24    output logic usb_dp_o,
25    output logic usb_dn_o,
26
27    // pulse to initiate new packet transmission
28    input logic pkt_start_i,
29    output logic pkt_end_o,
30
31    // pid_i to send
32    input logic [3:0] pid_i,
33
34    // tx logic pulls data until there is nothing available
35    input logic tx_data_avail_i,
36    output logic tx_data_get_o,
37    input logic [7:0] tx_data_i
38);
39
40
41 typedef enum logic [2:0] {Idle, Sync, Pid, Data0rcrc160,
Crc161, Eop, OscTest} state_e;
42 typedef enum logic [1:0] {0sIdle, 0sWaitByte, 0sTransmit}
out_state_e;
43
44 ///////////////////////////////////////////////////
45 // Signal Declarations //
46 ///////////////////////////////////////////////////
47
48 logic [3:0] pid_q, pid_d;
49 logic bitstuff;
50 logic bitstuff_q;
51 logic bitstuff_q2;
52 logic bitstuff_q3;
53 logic bitstuff_q4;
54
55 logic [5:0] bit_history;
56
57 state_e state_d, state_q;
58 out_state_e out_state_d, out_state_q;
59
60
61 logic [7:0] data_shift_reg_q, data_shift_reg_d;
62 logic [7:0] oe_shift_reg_q, oe_shift_reg_d;
63 logic [7:0] se0_shift_reg_q, se0_shift_reg_d;
64 logic data_payload_q, data_payload_d;
65 logic tx_data_get_q, tx_data_get_d;
66 logic byte_strobe_q, byte_strobe_d;
67 logic [4:0] bit_history_d, bit_history_q;
68 logic [2:0] bit_count_d, bit_count_q;
69
70 logic [15:0] crc16_d, crc16_q;
71
72 logic oe_q, oe_d;
73 logic usb_d_q, usb_d_d;
74 logic usb_se0_q, usb_se0_d;
75 logic [2:0] dp_eop_q, dp_eop_d;
76
77 logic test_mode_start;
78 logic serial_tx_data;
79 logic serial_tx_oe;
80 logic serial_tx_se0;
81 logic crc16_invert;
82 logic pkt_end;
83 logic out_nrzi_en;
84

```

```

85 // save packet parameters at pkt_start_i
86 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_pid
87   if (!rst_ni) begin
88     pid_q <= 0;
89   end else begin
90     if (link_reset_i) begin
91       pid_q <= 0;
92     end else begin
93       pid_q <= pid_d;
94     end
95   end
96 end
97
98 assign pid_d = pkt_start_i ? pid_i : pid_q;
99
100
101 assign serial_tx_data = data_shift_reg_q[0];
102 assign serial_tx_oe = oe_shift_reg_q[0];
103 assign serial_tx_se0 = se0_shift_reg_q[0];
104
105
106 // serialize sync, pid_i, data payload, and crc16
107 assign bit_history = {serial_tx_data, bit_history_q};
108 assign bitstuff    = bit_history == 6'b111111;
109
110 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_bitstuff
111   if (!rst_ni) begin
112     bitstuff_q <= 0;
113     bitstuff_q2 <= 0;
114     bitstuff_q3 <= 0;
115     bitstuff_q4 <= 0;
116   end else begin
117     if (link_reset_i) begin
118       bitstuff_q <= 0;
119       bitstuff_q2 <= 0;
120       bitstuff_q3 <= 0;
121       bitstuff_q4 <= 0;
122     end else begin
123       bitstuff_q <= bitstuff;
124       bitstuff_q2 <= bitstuff_q;
125       bitstuff_q3 <= bitstuff_q2;
126       bitstuff_q4 <= bitstuff_q3;
127     end
128   end
129 end
130
131 assign pkt_end    = bit_strobe_i && se0_shift_reg_q[1:0] ==
2'b01;
132 assign pkt_end_o = pkt_end;
133
134
135 // FSM //
136 // FSM //
137 // FSM //
138 always_comb begin : proc_fsm
139   // Default assignments
140   state_d = state_q;
141   data_shift_reg_d = data_shift_reg_q;
142   oe_shift_reg_d = oe_shift_reg_q;
143   se0_shift_reg_d = se0_shift_reg_q;
144   data_payload_d = data_payload_q;
145   tx_data_get_d = tx_data_get_q;
146   bit_history_d = bit_history_q;
147   bit_count_d = bit_count_q;
148   test_mode_start = 0;
149
150   unique case (state_q)
151     Idle : begin
152       if (tx_osc_test_mode_i) begin
153         state_d = OscTest;
154         test_mode_start = 1;
155       end else if (pkt_start_i) begin
156         state_d = Sync;
157       end
158     end
159
160     Sync : begin
161       if (byte_strobe_q) begin
162         state_d = Pid;
163         data_shift_reg_d = 8'b10000000;
164         oe_shift_reg_d = 8'b11111111;
165         se0_shift_reg_d = 8'b00000000;
166       end
167     end
168
169     Pid : begin
170       if (byte_strobe_q) begin
171         if (pid_q[1:0] == 2'b11) begin
172           state_d = Data0rCrc160;
173         end else begin
174           state_d = Eop;
175         end
176
177         data_shift_reg_d = {~pid_q, pid_q};
178         oe_shift_reg_d = 8'b11111111;
179         se0_shift_reg_d = 8'b00000000;
180       end
181     end
182

```

```

183   Data0rCrc160 : begin
184     if (byte_strobe_q) begin
185       if (tx_data_avail_i) begin
186         state_d = Data0rCrc160;
187         data_payload_d = 1;
188         tx_data_get_d = 1;
189         data_shift_reg_d = tx_data_i;
190         oe_shift_reg_d = 8'b11111111;
191         se0_shift_reg_d = 8'b00000000;
192       end else begin
193         state_d = Crc161;
194         data_payload_d = 0;
195         tx_data_get_d = 0;
196         data_shift_reg_d = ~(crc16_q[8], crc16_q[9],
183         crc16_q[10], crc16_q[11],
184         crc16_q[12], crc16_q[13],
185         crc16_q[14], crc16_q[15]);
197         oe_shift_reg_d = 8'b11111111;
198         se0_shift_reg_d = 8'b00000000;
199       end
200     end else begin
201       tx_data_get_d = 0;
202     end
203   end
204 end
205
206   Crc161 : begin
207     if (byte_strobe_q) begin
208       state_d = Eop;
209       data_shift_reg_d = ~(crc16_q[0], crc16_q[1],
183         crc16_q[2], crc16_q[3],
184         crc16_q[4], crc16_q[5],
185         crc16_q[6], crc16_q[7]);
210       oe_shift_reg_d = 8'b11111111;
211       se0_shift_reg_d = 8'b00000000;
212     end
213   end
214 end
215
216   Eop : begin
217     if (byte_strobe_q) begin
218       state_d = Idle;
219       oe_shift_reg_d = 8'b00000011;
220       se0_shift_reg_d = 8'b00000011;
221     end
222   end
223 end
224
225   OscTest: begin
226     // Oscillator test mode: toggle constantly
227     if (!tx_osc_test_mode_i && byte_strobe_q) begin
228       oe_shift_reg_d = 8'b00000000;
229       state_d = Idle;
230     end else if (byte_strobe_q) begin
231       data_shift_reg_d = 8'b00000000;
232       oe_shift_reg_d = 8'b11111111;
233       se0_shift_reg_d = 8'b00000000;
234     end
235   end
236 end
237
238   default: state_d = Idle;
239 endcase
240
241 // Logic closely coupled to the FSM
242 if (pkt_start_i) begin
243   // We need to have an inter-packet delay between
244   // 2 and 6.5 bit times (see USB 2.0 spec / 7.1.18.1)
245   // The latency in the rest of the system is approxi-
246   ately (measured)
247   // 3.68 bit-times, so we only introduce 1 bit-time here
248   bit_count_d = 7; // 8-7 = 1
249   bit_history_d = 0;
250 end else if (bit_strobe_i) begin
251   // bitstuff
252   if (bitstuff /* && !serial_tx_se0 */) begin
253     bit_history_d = bit_history[5:1];
254     data_shift_reg_d[0] = 0;
255   end
256
257   // normal deserialize
258   end else begin
259     bit_count_d = bit_count_q + 1;
260
261     data_shift_reg_d = (data_shift_reg_q >> 1);
262     oe_shift_reg_d = (oe_shift_reg_q >> 1);
263     se0_shift_reg_d = (se0_shift_reg_q >> 1);
264
265     bit_history_d = bit_history[5:1];
266   end
267 end
268
269 `ASSERT(StateValid_A, state_q inside {Idle, Sync, Pid,
Data0rCrc160, Crc161, Eop, OscTest})
270
271 always_comb begin : proc_byte_str
272   if (bit_strobe_i && !bitstuff && !pkt_start_i) begin
273     byte_strobe_d = (bit_count_q == 3'b000);
274   end else begin
275     byte_strobe_d = 0;
276   end
277 end

```

```

278 assign tx_data_get_o = tx_data_get_q;
279
280 // calculate crc16
281 assign crc16_invert = serial_tx_data ^ crc16_q[15];
282
283 always_comb begin : proc_crc16
284   crc16_d = crc16_q; // default assignment
285
286   if (pkt_start_i) begin
287     crc16_d = 16'b1111111111111111;
288   end
289
290   if (bit_strobe_i && data_payload_q && !bitstuff_q4 && !
pkt_start_i) begin
291     crc16_d = {crc16_q[14:0], 1'b0} ^ ({16{crc16_invert}} &
16'b10000000000000101);
292   end
293 end
294
295 //////////////////////////////////////////////////
296 // Regular Registers //
297 //////////////////////////////////////////////////
298
299 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_reg
300   if (!rst_ni) begin
301     state_q <= Idle;
302     data_payload_q <= 0;
303     data_shift_reg_q <= 0;
304     oe_shift_reg_q <= 0;
305     se0_shift_reg_q <= 0;
306     tx_data_get_q <= 0;
307     byte_strobe_q <= 0;
308     bit_history_q <= 0;
309     bit_count_q <= 0;
310     crc16_q <= 0;
311   end else begin
312     if (link_reset_i) begin
313       state_q <= Idle;
314       data_payload_q <= 0;
315       data_shift_reg_q <= 0;
316       oe_shift_reg_q <= 0;
317       se0_shift_reg_q <= 0;
318       tx_data_get_q <= 0;
319       byte_strobe_q <= 0;
320       bit_history_q <= 0;
321       bit_count_q <= 0;
322       crc16_q <= 0;
323     end else begin
324       state_q <= state_d;
325       data_payload_q <= data_payload_d;
326       data_shift_reg_q <= data_shift_reg_d;
327       oe_shift_reg_q <= oe_shift_reg_d;
328       se0_shift_reg_q <= se0_shift_reg_d;
329       tx_data_get_q <= tx_data_get_d;
330       byte_strobe_q <= byte_strobe_d;
331       bit_history_q <= bit_history_d;
332       bit_count_q <= bit_count_d;
333       crc16_q <= crc16_d;
334     end
335   end
336 end
337
338 //////////////////////////////////////////////////
339 // nrzi and differential driving //
340 //////////////////////////////////////////////////
341
342 // Output FSM
343 always_comb begin : proc_out_fsm
344   out_state_d = out_state_q;
345   out_nrzi_en = 1'b0;
346
347   unique case (out_state_q)
348     OsIdle: begin
349       if (pkt_start_i || test_mode_start) begin
350         out_state_d = OsWaitByte;
351       end
352     end
353
354     OsWaitByte: begin
355       if (byte_strobe_q) begin
356         out_state_d = OsTransmit;
357       end
358     end
359
360     OsTransmit: begin
361       out_nrzi_en = 1'b1;
362       if ((bit_strobe_i && !serial_tx_oe)) begin
363         out_state_d = OsIdle;
364       end
365     end
366
367     default : out_state_d = OsIdle;
368   endcase
369 end
370
371 `ASSERT(OutStateValid_A, out_state_q inside {OsIdle, OsWait-
Byte, OsTransmit})
372
373 always_comb begin : proc_diff
374   usb_d_d = usb_d_q;

```

```

375   usb_se0_d = usb_se0_q;
376   oe_d = oe_q;
377   dp_eop_d = dp_eop_q;
378
379   if (pkt_start_i) begin
380     usb_d_d = 1; // J -> first bit will be K (start of sync)
381     dp_eop_d = 3'b100; // Eop: {SE0, SE0, J}
382   end else if (bit_strobe_i && out_nrzi_en) begin
383     oe_d = serial_tx_oe;
384
385     if (serial_tx_se0) begin
386       // Eop
387       dp_eop_d = dp_eop_q >> 1;
388
389       if (dp_eop_q[0]) begin
390         // last bit of Eop: J
391         usb_d_d = 1;
392         usb_se0_d = 0;
393       end else begin
394         // first two bits of Eop: SE0
395         usb_se0_d = 1;
396       end
397     end
398
399     end else if (serial_tx_data) begin
400       // value should stay the same, do nothing
401     end else begin
402       usb_d_d = !usb_d_q;
403     end
404   end
405
406   // Set to J state when OE=0 to avoid
407   // glitches
408   if (!oe_d) begin
409     usb_d_d = 1;
410   end
411 end
412
413 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_diff_reg
414   if (!rst_ni) begin
415     dp_eop_q <= 0;
416     out_state_q <= OsIdle;
417   end else begin
418     if (link_reset_i) begin
419       dp_eop_q <= 0;
420       out_state_q <= OsIdle;
421     end else begin
422       dp_eop_q <= dp_eop_d;
423       out_state_q <= out_state_d;
424     end
425   end
426 end
427
428 prim_flop u_oe_flop (
429   .clk_i,
430   .rst_ni,
431   .d_i(link_reset_i ? 1'b0 : oe_d),
432   .q_o(oe_q)
433 );
434
435 prim_flop #(
436   .ResetValue(1) // J state = idle state
437 ) u_usb_d_flop (
438   .clk_i,
439   .rst_ni,
440   .d_i(link_reset_i ? 1'b1 : usb_d_d),
441   .q_o(usb_d_q)
442 );
443
444 prim_flop u_usb_se0_flop (
445   .clk_i,
446   .rst_ni,
447   .d_i(link_reset_i ? 1'b0 : usb_se0_d),
448   .q_o(usb_se0_q)
449 );
450
451 // Handle the D+ / D- pin flip on the USB side, and provide
both the
452 // dp/dn and d/se0 interfaces, for compatibility with mul-
tiple
453 // driver types.
454 logic usb_se0_flipped, usb_dp_flipped, usb_dn_flipped;
455
456 always_comb begin
457   if (link_reset_i) begin
458     usb_se0_flipped = 1'b0;
459     usb_dp_flipped = !cfg_pinflip_i;
460     usb_dn_flipped = !cfg_pinflip_i;
461   end else begin
462     usb_se0_flipped = usb_se0_d;
463
464     usb_dp_flipped = (cfg_pinflip_i ? ~usb_d_d : usb_d_d) &
~usb_se0_d;
465
466     usb_dn_flipped = (cfg_pinflip_i ? usb_d_d : ~usb_d_d) &
~usb_se0_d;
467   end
468 end

```

```

469 // Use registered outputs for the I/Os
470 prim_flop #(
471   .ResetValue(1) // J state = idle state
472 ) u_usb_d_o_flop (
473   .clk_i,
474   .rst_ni,
475   .d_i(usb_dp_flipped), // Note: single-ended 'D' output
mirrors D+
476   .q_o(usb_d_o)
477 );
478
479 prim_flop #(
480   .ResetValue(0) // J state = idle state
481 ) u_usb_se0_o_flop (
482   .clk_i,
483   .rst_ni,
484   .d_i(usb_se0_flipped),
485   .q_o(usb_se0_o)
486 );
487
488 prim_flop #(
489   .ResetValue(1) // J state = idle state
490 ) u_usb_dp_o_flop (
491   .clk_i,
492   .rst_ni,
493   .d_i(usb_dp_flipped),
494   .q_o(usb_dp_o)
495 );
496
497 prim_flop #(
498   .ResetValue(0) // J state = idle state
499 ) u_usb_dn_o_flop (
500   .clk_i,
501   .rst_ni,
502   .d_i(usb_dn_flipped),
503   .q_o(usb_dn_o)
504 );
505
506 assign usb_oe_o = oe_q;
507
508 endmodule

```

7. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_tx_mux.sv (23 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 // Copyright Luke Valenty (TinyFPGA project, https://git-
  hub.com/tinyfpga/TinyFPGA-Bootloader).
5
6 module usb_fs_tx_mux (
7   // interface to IN Protocol Engine
8   input logic in_tx_pkt_start_i,
9   input logic [3:0] in_tx_pid_i,
10
11   // interface to OUT Protocol Engine
12   input logic out_tx_pkt_start_i,
13   input logic [3:0] out_tx_pid_i,
14
15   // interface to tx module
16   output logic tx_pkt_start_o,
17   output logic [3:0] tx_pid_o
18 );
19
20 assign tx_pkt_start_o = in_tx_pkt_start_i |
  out_tx_pkt_start_i;
21 assign tx_pid_o = out_tx_pkt_start_i ? out_tx_pid_i :
  in_tx_pid_i;
22
23 endmodule

```

8. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev.sv (1,470 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // USB Full Speed Device Interface (usbdev).
6
7 `include "prim_assert.sv"
8
9 module usbdev
10   import usbdev_pkg::*;
11   import usbdev_reg_pkg::*;
12   import prim_util_pkg::vbits;
13 # (
14   parameter bit Stub = 1'b0,
15   parameter logic [NumAlerts-1:0] AlertAsyncOn = {Nu-
  mAlerts{1'b1}},
16   // Number of cycles of differential skew tolerated on the
  alert signal.
17   parameter int unsigned AlertSkewCycles = 1,
18   // Max time (in microseconds) from rx_enable_o high to the
19   // external differential receiver outputting valid data

```

```

(when
20   // configured to use one).
21   parameter int unsigned RcvrWakeTimeUs = 1
22 ) (
23   input logic clk_i,
24   input logic rst_ni,
25   input logic clk_aon_i,
26   input logic rst_aon_ni,
27
28   // Register interface
29   input tlul_pkg::tl_h2d_t tl_i,
30   output tlul_pkg::tl_d2h_t tl_o,
31
32   // Alerts
33   input prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
  alert_rx_i,
34   output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
  alert_tx_o,
35
36   // Data inputs
37   input logic cio_usb_dp_i, // differential P, can be
  used in single-ended mode to detect SE0
38   input logic cio_usb_dn_i, // differential N, can be
  used in single-ended mode to detect SE0
39   input logic usb_rx_d_i, // single-ended input from
  the differential receiver
40
41   // Data outputs
42   output logic cio_usb_dp_o,
43   output logic cio_usb_dp_en_o,
44   output logic cio_usb_dn_o,
45   output logic cio_usb_dn_en_o,
46   output logic usb_tx_se0_o, // single-ended zero output
47   output logic usb_tx_d_o,
48
49   // Non-data I/O
50   input logic cio_sense_i,
51   output logic usb_dp_pullup_o,
52   output logic usb_dn_pullup_o,
53   output logic usb_rx_enable_o,
54   output logic usb_tx_use_d_se0_o,
55
56   // Direct pinmux aon detect connections
57   output logic usb_aon_suspend_req_o,
58   output logic usb_aon_wake_ack_o,
59
60   // Events and state from wakeup module
61   input logic usb_aon_bus_reset_i,
62   input logic usb_aon_sense_lost_i,
63   input logic usb_aon_bus_not_idle_i,
64   input logic usb_aon_wake_detect_active_i,
65
66   // SOF reference for clock calibration
67   output logic usb_ref_val_o,
68   output logic usb_ref_pulse_o,
69
70   // memory configuration
71   input prim_ram_ip_pkg::ram_ip_cfg_req_t ram_cfg_i,
72   output prim_ram_ip_pkg::ram_ip_cfg_rsp_t ram_cfg_o,
73
74   // Interrupts
75   output logic intr_pkt_received_o, // Packet received
76   output logic intr_pkt_sent_o, // Packet sent
77   output logic intr_powered_o,
78   output logic intr_disconnected_o,
79   output logic intr_host_lost_o,
80   output logic intr_link_reset_o,
81   output logic intr_link_suspend_o,
82   output logic intr_link_resume_o,
83   output logic intr_av_out_empty_o,
84   output logic intr_rx_full_o,
85   output logic intr_av_overflow_o,
86   output logic intr_link_in_err_o,
87   output logic intr_link_out_err_o,
88   output logic intr_rx_crc_err_o,
89   output logic intr_rx_pid_err_o,
90   output logic intr_rx_bitstuff_err_o,
91   output logic intr_frame_o,
92   output logic intr_av_setup_empty_o
93 );
94
95 // Could make SramDepth, MaxPktSizeByte, AVSetupFifoDepth,
  AVOutFifoDepth and RXFifoDepth
96 // module parameters but may need to fix register def for
  the first two
97 localparam int SramDw = 32; // Places packing bytes to SRAM
  assume this
98 localparam int SramDepth = 512; // 2kB, SRAM width is DW
99 localparam int MaxPktSizeByte = 64;
100
101 localparam int SramAw = $clog2(SramDepth);
102 localparam int SizeWidth = $clog2(MaxPktSizeByte);
103
104 localparam int NBuf = (SramDepth * SramDw) / (MaxPktSizeByte *
  8);
105
106 localparam int NBufWidth = $clog2(NBuf);
107
108 // AV SETUP and AV OUT fifos just store buffer numbers
109 localparam int AVFifoWidth = NBufWidth;
110 localparam int AVSetupFifoDepth = 4;
111 localparam int AVOutFifoDepth = 8;

```



```

110
111 // RX fifo stores          buf# + size(0-MaxPktSize-
Byte) + EP# + Type
112 localparam int RXFifoWidth = NBufWidth +
(1+SizeWidth) + 4 + 1;
113 localparam int RXFifoDepth = 8;
114 // derived parameter
115 localparam int RXFifoDepthW =
prim_util_pkg::vbits(RXFifoDepth+1);
116
117 usbdev_reg2hw_t reg2hw, reg2hw_regtop;
118 usbdev_hw2reg_t hw2reg, hw2reg_regtop;
119
120 logic rst_n;
121 if (Stub) begin : gen_stubbed_reset
122     assign rst_n = '0;
123 end else begin : gen_no_stubbed_reset
124     assign rst_n = rst_n1;
125 end
126
127 tlul_pkg::tl_h2d_t tl_sram_h2d;
128 tlul_pkg::tl_d2h_t tl_sram_d2h;
129
130 // Software access to the Packet Buffer RAM
131 logic sw_mem_a_req;
132 logic sw_mem_a_gnt;
133 logic sw_mem_a_write;
134 logic [SramAw-1:0] sw_mem_a_addr;
135 logic [SramDw-1:0] sw_mem_a_wdata;
136 logic sw_mem_a_rvalid;
137 logic [SramDw-1:0] sw_mem_a_rdata;
138 logic [1:0] sw_mem_a_rerror;
139
140 // usbdev hardware access to the Packet Buffer RAM
141 logic usb_mem_b_req;
142 logic usb_mem_b_write;
143 logic [SramAw-1:0] usb_mem_b_addr;
144 logic [SramDw-1:0] usb_mem_b_wdata;
145 logic [SramDw-1:0] usb_mem_b_rdata;
146
147 logic clr_devaddr;
148
149 logic event_av_setup_empty, event_av_out_empty,
event_av_overflow, event_rx_full;
150 logic link_reset, link_suspend;
151 logic host_lost, link_disconnect, link_powered;
152 logic event_link_reset, event_link_suspend,
event_link_resume;
153 logic event_host_lost, event_disconnect,
event_powered;
154 logic event_rx_crc5_err, event_rx_crc16_err;
155 logic event_rx_crc_err, event_rx_pid_err;
156 logic event_rx_bitstuff_err;
157 logic event_in_err;
158 logic event_out_err;
159 logic event_frame, event_sof;
160 logic link_active;
161
162 // Diagnostic visibility of OUT-side exceptional events
163 logic event_ign_avsetup, event_drop_avout,
event_drop_rx, event_datatog_out;
164 // Diagnostic visibility of IN-side exceptional events
165 logic event_timeout_in, event_nak_in,
event_nodata_in;
166 // Interrupt to software reports both types of CRC error;
they are separated only for the
167 // purpose of diagnostic event counting.
168 assign event_rx_crc_err = event_rx_crc5_err |
event_rx_crc16_err;
169
170 logic [10:0] frame;
171 logic [2:0] link_state;
172 logic connect_en;
173 logic resume_link_active;
174
175 // Current state of OUT data toggles
176 logic [NEndpoints-1:0] out_data_toggle;
177 // Write strobe from register interface
178 logic out_datatog_we;
179 // Write data from register interface
180 logic [NEndpoints-1:0] out_datatog_status;
181 logic [NEndpoints-1:0] out_datatog_mask;
182
183 // Current state of IN data toggles
184 logic [NEndpoints-1:0] in_data_toggle;
185 // Write strobe from register interface
186 logic in_datatog_we;
187 // Write data from register interface
188 logic [NEndpoints-1:0] in_datatog_status;
189 logic [NEndpoints-1:0] in_datatog_mask;
190
191 ///////////////////////////////////////////////////
192 // USB RX after CDC & muxing //
193 ///////////////////////////////////////////////////
194 logic usb_rx_d;
195 logic usb_rx_dp;
196 logic usb_rx_dn;
197 ///////////////////////////////////////////////////
198 // USB TX after CDC & muxing //
199 ///////////////////////////////////////////////////
200 logic usb_tx_d;
201 logic usb_tx_se0;
202 logic usb_tx_dp;
203 logic usb_tx_dn;
204 logic usb_tx_oe;
205 ///////////////////////////////////////////////////
206 // USB control pins after CDC //
207 ///////////////////////////////////////////////////
208 logic usb_pwr_sense;
209 logic usb_pullup_en;
210 logic usb_dp_pullup_en;
211 logic usb_dn_pullup_en;
212
213 ///////////////////////////////////////////////////
214 // Microsecond timing reference //
215 ///////////////////////////////////////////////////
216 // us_tick ticks for one cycle every us, and it is based off a
free-running
217 // counter.
218 logic [5:0] ns_cnt;
219 logic us_tick;
220
221 assign us_tick = (ns_cnt == 6'd47);
222 always_ff @(posedge clk_i or negedge rst_n) begin
223     if (!rst_n) begin
224         ns_cnt <= '0;
225     end else begin
226         if (us_tick) begin
227             ns_cnt <= '0;
228         end else begin
229             ns_cnt <= ns_cnt + 1'b1;
230         end
231     end
232 end
233
234 ///////////////////////////////////////////////////
235 // Receive interface fifos //
236 ///////////////////////////////////////////////////
237
238 logic avsetup_fifo_rst;
239 logic avsetup_fifo_wready;
240 logic avout_fifo_rst;
241 logic avout_fifo_wready;
242 logic event_pkt_received;
243 logic avsetup_rvalid, avsetup_rready;
244 logic avout_rvalid, avout_rready;
245 logic rx_fifo_rst;
246 logic rx_wvalid, rx_wready;
247 logic rx_wready_setup, rx_wready_out;
248 logic rx_fifo_rvalid;
249 logic rx_fifo_re;
250
251 logic [AVFifoWidth - 1:0] avsetup_rdata;
252 logic [AVFifoWidth - 1:0] avout_rdata;
253 logic [RXFifoWidth - 1:0] rx_wdata, rx_rdata;
254
255 logic [NEndpoints-1:0] clear_rxenable_out;
256
257 // Software reset signals
258 assign avsetup_fifo_rst = reg2hw.fifo_ctrl.avsetup_rst.qe &
reg2hw.fifo_ctrl.avsetup_rst.q;
259 assign avout_fifo_rst = reg2hw.fifo_ctrl.avout_rst.qe &
reg2hw.fifo_ctrl.avout_rst.q;
260 assign rx_fifo_rst = reg2hw.fifo_ctrl.rx_rst.qe &
reg2hw.fifo_ctrl.rx_rst.q;
261
262 // Separate 'FIFO empty' interrupts for the OUT and SETUP
FIFOs because each interrupt cannot be
263 // cleared without writing a buffer into the FIFO
264 assign event_av_setup_empty = connect_en & ~avsetup_rvalid;
265 assign event_av_out_empty = connect_en & ~avout_rvalid;
266 // A single 'overflow' interrupt suffices since this indic-
ates a programming error
267 assign event_av_overflow = (reg2hw.avsetupbuffer.qe &
(~avsetup_fifo_wready))
268 | (reg2hw.avoutbuffer.qe &
(~avout_fifo_wready));
269 assign hw2reg.usbstat.rx_empty.d = connect_en &
~rx_fifo_rvalid;
270
271 // Available SETUP Buffer FIFO
272 prim_fifo_sync #(
273     .Width(AVFifoWidth),
274     .Pass(1'b0),
275     .Depth(AVSetupFifoDepth),
276     .OutputZeroIfEmpty(1'b0)
277 ) usbdev_avsetupfifo (
278     .clk_i,
279     .rst_ni (rst_n),
280     .clr_i (avsetup_fifo_rst),
281
282     .wvalid_i (reg2hw.avsetupbuffer.qe),
283     .wready_o (avsetup_fifo_wready),
284     .wdata_i (reg2hw.avsetupbuffer.q),
285
286     .rvalid_o (avsetup_rvalid),
287     .rready_i (avsetup_rready),
288     .rdata_o (avsetup_rdata),
289     .full_o (hw2reg.usbstat.av_setup_full.d),
290     .depth_o (hw2reg.usbstat.av_setup_depth.d),

```

```

291 .err_o      ()
292 );
293
294 // Available OUT Buffer FIFO
295 prim_fifo_sync #(
296   .Width(AVOutFifoWidth),
297   .Pass(1'b0),
298   .Depth(AVOutFifoDepth),
299   .OutputZeroIfEmpty(1'b0)
300 ) usbdev_avoutfifo (
301   .clk_i,
302   .rst_ni   (rst_n),
303   .clr_i    (avout_fifo_rst),
304
305   .wvalid_i (reg2hw.avoutbuffer.qe),
306   .wready_o (avout_fifo_wready),
307   .wdata_i  (reg2hw.avoutbuffer.q),
308
309   .rvalid_o (avout_rvalid),
310   .rready_i (avout_rready),
311   .rdata_o  (avout_rdata),
312   .full_o   (hw2reg.usbstat.av_out_full.d),
313   .depth_o  (hw2reg.usbstat.av_out_depth.d),
314   .err_o    ()
315 );
316
317 assign rx_fifo_re = reg2hw.rxfifo.ep.re |
reg2hw.rxfifo.setup.re |
318 reg2hw.rxfifo.size.re |
reg2hw.rxfifo.buffer.re;
319
320 // The number of used entries in the Received Buffer FIFO is
presented to the software
321 logic [RXFifoDepthW-1:0] rx_depth;
322 assign hw2reg.usbstat.rx_depth.d = rx_depth;
323
324 // We can always accept a SETUP packet if the Received Buf-
fer FIFO is not full...
325 assign rx_wready_setup = rx_wready;
326 // ... but regular OUT packets are not permitted to use the
final entry; still qualified
327 // with 'rx_wready' for when reset is asserted.
328 assign rx_wready_out = rx_wready & (rx_depth < RXFi-
foDepthW'(RXFifoDepth - 1));
329
330 // Received Buffer FIFO
331 prim_fifo_sync #(
332   .Width(RXFifoWidth),
333   .Pass(1'b0),
334   .Depth(RXFifoDepth),
335   .OutputZeroIfEmpty(1'b1)
336 ) usbdev_rxfifo (
337   .clk_i,
338   .rst_ni   (rst_n),
339   .clr_i    (rx_fifo_rst),
340
341   .wvalid_i (rx_wvalid),
342   .wready_o (rx_wready),
343   .wdata_i  (rx_wdata),
344
345   .rvalid_o (rx_fifo_rvalid),
346   .rready_i (rx_fifo_re),
347   .rdata_o  (rx_rdata),
348   .full_o   (event_rx_full),
349   .depth_o  (rx_depth),
350   .err_o    ()
351 );
352
353 assign hw2reg.rxfifo.ep.d = rx_rdata[16:13];
354 assign hw2reg.rxfifo.setup.d = rx_rdata[12];
355 assign hw2reg.rxfifo.size.d = rx_rdata[11:5];
356 assign hw2reg.rxfifo.buffer.d = rx_rdata[4:0];
357 assign event_pkt_received = rx_fifo_rvalid;
358
359 // The rxfifo register is hrw, but we just need the read en-
ables.
360 logic [16:0] unused_rxfifo_q;
361 assign unused_rxfifo_q = {reg2hw.rxfifo.ep.q,
reg2hw.rxfifo.setup.q,
362 reg2hw.rxfifo.size.q,
reg2hw.rxfifo.buffer.q};
363
364 ///////////////////////////////////////////////////////////////////
365 // IN (Transmit) interface config //
366 ///////////////////////////////////////////////////////////////////
367 logic [NBufWidth-1:0] in_buf [NEndpoints];
368 logic [SizeWidth:0] in_size [NEndpoints];
369 logic [3:0] in_endpoint;
370 logic in_endpoint_val;
371 logic [NEndpoints-1:0] in_rdy;
372 logic [NEndpoints-1:0] clear_rdybit, set_sentbit, up-
date_pend, set_sending;
373 logic setup_received, in_ep_xact_end;
374 logic [NEndpoints-1:0] ep_out_iso, ep_in_iso;
375 logic [NEndpoints-1:0] enable_out, enable_setup,
in_ep_stall, out_ep_stall;
376 logic [NEndpoints-1:0] ep_set_nak_on_out;
377 logic [NEndpoints-1:0] ep_in_enable, ep_out_enable;
378 logic [3:0] out_endpoint;
379 logic out_endpoint_val;

380 logic use_diff_rcvr, usb_diff_rx_ok;
381
382 // Endpoint enables
383 always_comb begin : proc_map_ep_enable
384   for (int i = 0; i < NEndpoints; i++) begin
385     ep_in_enable[i] = reg2hw.ep_in_enable[i].q;
386     ep_out_enable[i] = reg2hw.ep_out_enable[i].q;
387   end
388 end
389
390 // RX enables
391 logic [NEndpoints-1:0] rxenable_out;
392 always_comb begin : proc_map_rxenable
393   for (int i = 0; i < NEndpoints; i++) begin
394     enable_setup[i] = reg2hw.rxenable_setup[i].q;
395     enable_out[i] = rxenable_out[i];
396     ep_set_nak_on_out[i] = reg2hw.set_nak_out[i].q;
397   end
398 end
399
400 // STALL for both directions
401 always_comb begin : proc_map_stall
402   for (int i = 0; i < NEndpoints; i++) begin
403     in_ep_stall[i] = reg2hw.in_stall[i];
404     out_ep_stall[i] = reg2hw.out_stall[i];
405   end
406 end
407
408 always_comb begin : proc_map_iso
409   for (int i = 0; i < NEndpoints; i++) begin
410     ep_out_iso[i] = reg2hw.out_iso[i].q;
411     ep_in_iso[i] = reg2hw.in_iso[i].q;
412   end
413 end
414
415 always_comb begin : proc_map_buf_size
416   for (int i = 0; i < NEndpoints; i++) begin
417     in_buf[i] = reg2hw.configin[i].buffer.q;
418     in_size[i] = reg2hw.configin[i].size.q;
419   end
420 end
421
422 always_comb begin : proc_map_rdy_reg2hw
423   for (int i = 0; i < NEndpoints; i++) begin
424     in_rdy[i] = reg2hw.configin[i].rdy.q;
425   end
426 end
427
428 // Captured properties of current IN buffer, maintained
throughout packet collection as
429 // protection against change during packet retraction by FW.
430 logic [NBufWidth-1:0] in_buf_q, in_buf_d;
431 logic [SizeWidth:0] in_size_q, in_size_d;
432 logic in_xact_starting;
433 logic [3:0] in_xact_start_ep;
434
435 always_ff @(posedge clk_i or negedge rst_n) begin
436   if (!rst_n) begin
437     in_buf_q <= '0;
438     in_size_q <= '0;
439   end else begin
440     in_buf_q <= in_buf_d;
441     in_size_q <= in_size_d;
442   end
443 end
444 assign in_buf_d = in_xact_starting ? in_buf[in_x-
act_start_ep] : in_buf_q;
445 assign in_size_d = in_xact_starting ? in_s-
ize[in_xact_start_ep] : in_size_q;
446
447 // OUT data toggles are maintained with the packet engine
but may be set or
448 // cleared by software
449 assign out_datatog_we = reg2hw.out_data_toggle.status.qe &
reg2hw.out_data_toggle.mask.qe;
450 assign out_datatog_status = reg2hw.out_data_toggle.status.q;
451 assign out_datatog_mask = reg2hw.out_data_toggle.mask.q;
452 // Software may read them at any time
453 assign hw2reg.out_data_toggle.status.d = out_data_toggle;
454 assign hw2reg.out_data_toggle.mask.d = {NEndpoints{1'b0}};
455
456 // IN data toggles are maintained with the packet engine but
may be set or
457 // cleared by software
458 assign in_datatog_we = reg2hw.in_data_toggle.status.qe &
reg2hw.in_data_toggle.mask.qe;
459 assign in_datatog_status = reg2hw.in_data_toggle.status.q;
460 assign in_datatog_mask = reg2hw.in_data_toggle.mask.q;
461 // Software may read them at any time
462 assign hw2reg.in_data_toggle.status.d = in_data_toggle;
463 assign hw2reg.in_data_toggle.mask.d = {NEndpoints{1'b0}};
464
465 always_comb begin
466   set_sentbit = '0;
467   if (in_ep_xact_end && in_endpoint_val) begin
468     set_sentbit[in_endpoint] = 1'b1;
469   end
470 end
471
472 always_comb begin : proc_map_sent

```

```

473   for (int i = 0; i < NEndpoints; i++) begin
474       hw2reg.in_sent[i].de = set_sentbit[i];
475       hw2reg.in_sent[i].d = 1'b1;
476   end
477 end
478
479 // This must be held level for the interrupt, so no sent
packets are missed.
480 logic sent_event_pending;
481 always_comb begin
482     sent_event_pending = 1'b0;
483     for (int i = 0; i < NEndpoints; i++) begin
484         sent_event_pending |= reg2hw.in_sent[i].q;
485     end
486 end
487
488 // rxenable_out register must be implemented as 'hwext' so
that we can treat each endpoint
489 // individually and prevent sw/hw races from inadvertently
re-enabling an endpoint that was
490 // recently disabled by the hardware.
491 logic [NEndpoints-1:0] rxenable_out_we;
492 assign rxenable_out_we = {NEndpoints}{reg2hw.rxenable_out.out.qe &
reg2hw.rxenable_out.preserve.qe} &
493 ~reg2hw.rxenable_out.preserve.q;
494 always_ff @(posedge clk_i or negedge rst_ni) begin
495     if (!rst_ni) begin
496         rxenable_out <= 'b0;
497     end else begin
498         for (int unsigned i = 0; i < NEndpoints; i++) begin
499             if (rxenable_out_we[i]) begin
500                 // Software wins for backwards compatibility; an
earlier implementation employed the
501                 // module 'prim_subreg_arb' which exhibits this be-
havior.
502                 rxenable_out[i] <= reg2hw.rxenable_out.out.q[i];
503             end else if (clear_rxenable_out[i]) begin
504                 rxenable_out[i] <= 1'b0;
505             end
506         end
507     end
508 end
509 end
510 assign hw2reg.rxenable_out.out.d = rxenable_out;
511
512 // Clear of rxenable_out bit
513 // If so configured, for every received transaction on a
given endpoint, clear
514 // the rxenable_out bit. In this configuration, hardware de-
faults to NAKing
515 // any subsequent transaction, so software has time to de-
cide the next
516 // response.
517 always_comb begin
518     clear_rxenable_out = '0;
519     if (rx_wvalid && out_endpoint_val) begin
520         clear_rxenable_out[out_endpoint] =
ep_set_nak_on_out[out_endpoint];
521     end
522 end
523
524 always_comb begin
525     clear_rdybit = '0;
526     update_pend = '0;
527     if (event_link_reset) begin
528         clear_rdybit = {NEndpoints}{1'b1};
529         update_pend = {NEndpoints}{1'b1};
530     end else begin
531         if (setup_received & out_endpoint_val) begin
532             // Clear pending when a SETUP is received
533             clear_rdybit[out_endpoint] = 1'b1;
534             update_pend[out_endpoint] = 1'b1;
535         end else if (in_ep_xact_end & in_endpoint_val) begin
536             // Clear rdy and sending when an IN transmission was
successful
537             clear_rdybit[in_endpoint] = 1'b1;
538         end
539     end
540 end
541
542 // IN transaction starting on any endpoint?
543 always_comb begin
544     set_sending = '0;
545     if (in_xact_starting) set_sending[in_xact_start_ep] =
1'b1;
546 end
547
548 // Clearing of rdy bit in response to successful IN packet
transmission or packet cancellation
549 // through link reset or SETUP packet reception.
550 always_comb begin : proc_map_rdy_hw2reg
551     for (int i = 0; i < NEndpoints; i++) begin
552         hw2reg.configin[i].rdy.de = clear_rdybit[i];
553         hw2reg.configin[i].rdy.d = 1'b0;
554     end
555 end
556
557 // Update the pending bit by copying the ready bit that is
about to clear
558 always_comb begin : proc_map_pend

```

```

559   for (int i = 0; i < NEndpoints; i++) begin
560       hw2reg.configin[i].pend.de = update_pend[i];
561       hw2reg.configin[i].pend.d = reg2hw.configin[i].rdy.q |
reg2hw.configin[i].pend.q;
562   end
563 end
564
565 // Update the sending bit to mark that collection of the
packet by the USB host has been
566 // attempted and FW shall not attempt retraction of the
packet.
567 always_comb begin : proc_map_sending
568     for (int i = 0; i < NEndpoints; i++) begin
569         hw2reg.configin[i].sending.de = set_sending[i] |
set_sentbit[i] | update_pend[i];
570         hw2reg.configin[i].sending.d = ~set_sentbit[i] & ~up-
date_pend[i];
571     end
572 end
573
574 ////////////////////////////////////////////////////
575 // USB interface -- everything is in USB clock domain //
576 ////////////////////////////////////////////////////
577 logic cfg_pinflip;
578 assign cfg_pinflip = reg2hw.phy_config.pinflip.q;
579 assign usb_dp_pullup_en = cfg_pinflip ? 1'b0 : usb_pul-
lup_en;
580 assign usb_dn_pullup_en = !cfg_pinflip ? 1'b0 : usb_pul-
lup_en;
581
582
583 usbdev_usbif #(
584     .NEndpoints (NEndpoints),
585     .AVFifoWidth (AVFifoWidth),
586     .RXFifoWidth (RXFifoWidth),
587     .MaxPktSizeByte (MaxPktSizeByte),
588     .NBuf (NBuf),
589     .SramAw (SramAw)
590 ) usbdev_impl (
591     .clk_48mhz_i (clk_i),
592     .rst_ni (rst_n),
593
594     // Pins
595     .usb_d_i (usb_rx_d),
596     .usb_dp_i (usb_rx_dp),
597     .usb_dn_i (usb_rx_dn),
598     .usb_oe_o (usb_tx_oe),
599     .usb_d_o (usb_tx_d),
600     .usb_se0_o (usb_tx_se0),
601     .usb_dp_o (usb_tx_dp),
602     .usb_dn_o (usb_tx_dn),
603     .usb_sense_i (usb_pwr_sense),
604     .usb_pullup_en_o (usb_pullup_en),
605
606     // receive side
607     .rx_setup_i (enable_setup),
608     .rx_out_i (enable_out),
609     .rx_stall_i (out_ep_stall),
610     .avsetup_rvalid_i (avsetup_rvalid),
611     .avsetup_rready_o (avsetup_rready),
612     .avsetup_rdata_i (avsetup_rdata),
613     .avout_rvalid_i (avout_rvalid),
614     .avout_rready_o (avout_rready),
615     .avout_rdata_i (avout_rdata),
616
617     .rx_wvalid_o (rx_wvalid),
618     .rx_wready_setup_i (rx_wready_setup),
619     .rx_wready_out_i (rx_wready_out),
620     .rx_wdata_o (rx_wdata),
621     .setup_received_o (setup_received),
622     .out_endpoint_o (out_endpoint), // will be stable
for several cycles
623     .out_endpoint_val_o (out_endpoint_val),
624
625     // transmit side
626     .in_xact_starting_o (in_xact_starting),
627     .in_xact_start_ep_o (in_xact_start_ep),
628     .in_buf_i (in_buf_q),
629     .in_size_i (in_size_q),
630     .in_stall_i (in_ep_stall),
631     .in_rdy_i (in_rdy),
632     .in_ep_xact_end_o (in_ep_xact_end),
633     .in_endpoint_o (in_endpoint),
634     .in_endpoint_val_o (in_endpoint_val),
635
636     // memory
637     .mem_req_o (usb_mem_b_req),
638     .mem_write_o (usb_mem_b_write),
639     .mem_addr_o (usb_mem_b_addr),
640     .mem_wdata_o (usb_mem_b_wdata),
641     .mem_rdata_i (usb_mem_b_rdata),
642
643     // time reference
644     .us_tick_i (us_tick),
645
646     // control
647     .connect_en_i (connect_en),
648     .devaddr_i (reg2hw.usbctrl.device_address.q),
649     .clr_devaddr_o (clr_devaddr),
650     .in_ep_enabled_i (ep_in_enable),
651     .out_ep_enabled_i (ep_out_enable),

```



```

652 .out_ep_iso_i      (ep_out_iso),
653 .in_ep_iso_i       (ep_in_iso),
654 .diff_rx_ok_i      (usb_diff_rx_ok),
655 .cfg_eop_single_bit_i
(reg2hw.phy_config.eop_single_bit.q), // cdc ok: quasi-static
656 .tx_osc_test_mode_i
(reg2hw.phy_config.tx_osc_test_mode.q), // cdc ok: quasi-static
657 .cfg_use_diff_rcvr_i (usb_rx_enable_o),
658 .cfg_pinflip_i       (cfg_pinflip),
659 .out_data_toggle_o    (out_data_toggle),
660 .out_datatog_we_i     (out_datatog_we),
661 .out_datatog_status_i (out_datatog_status),
662 .out_datatog_mask_i   (out_datatog_mask),
663 .in_data_toggle_o     (in_data_toggle),
664 .in_datatog_we_i      (in_datatog_we),
665 .in_datatog_status_i  (in_datatog_status),
666 .in_datatog_mask_i    (in_datatog_mask),
667
668 .resume_link_active_i (resume_link_active),
669
670 // status
671 .frame_o               (frame),
672 .frame_start_o         (event_frame),
673 .sof_detected_o        (event_sof),
674 .link_state_o          (link_state),
675 .link_disconnect_o     (link_disconnect),
676 .link_powered_o        (link_powered),
677 .link_reset_o          (link_reset),
678 .link_active_o         (link_active),
679 .link_suspend_o        (link_suspend),
680 .link_resume_o         (event_link_resume),
681 .host_lost_o           (host_lost),
682 .link_in_err_o         (event_in_err),
683 .link_out_err_o        (event_out_err),
684 .rx_crc5_err_o         (event_rx_crc5_err),
685 .rx_crc16_err_o        (event_rx_crc16_err),
686 .rx_pid_err_o          (event_rx_pid_err),
687 .rx_bitstuff_err_o     (event_rx_bitstuff_err),
688
689 // event counters
690 .event_ign_avsetup_o    (event_ign_avsetup),
691 .event_drop_avout_o     (event_drop_avout),
692 .event_drop_rx_o       (event_drop_rx),
693 .event_datatog_out_o    (event_datatog_out),
694 .event_timeout_in_o     (event_timeout_in),
695 .event_nak_in_o         (event_nak_in),
696 .event_nodata_in_o      (event_nodata_in)
697 );
698
699 ///////////////////////////////////////////////////
700 // Control signal / status CDC //
701 ///////////////////////////////////////////////////
702
703 assign hw2reg.usbstat.link_state.d = link_state;
704 assign hw2reg.usbstat.frame.d = frame;
705
706 assign connect_en = reg2hw.usbctrl.enable.q;
707 assign resume_link_active =
reg2hw.usbctrl.resume_link_active.qe &
reg2hw.usbctrl.resume_link_active.q;
708
709 // Just want a pulse to ensure only one interrupt for an
event
710 prim_edge_detector #(
711 .Width(5),
712 .EnSync(1'b0)
713 ) gen_event (
714 .clk_i,
715 .rst_ni      (rst_n),
716 .d_i         ({link_disconnect, link_reset, link_suspend,
717               host_lost, link_powered}),
718 .q_sync_o    (),
719 .q_posedge_pulse_o({event_disconnect, event_link_reset,
event_link_suspend,
720                     event_host_lost, event_powered}),
721 .q_negedge_pulse_o()
722 );
723
724
725 assign hw2reg.usbstat.host_lost.d = host_lost;
726
727 // resets etc cause the device address to clear
728 assign hw2reg.usbctrl.device_address.de = clr_devaddr;
729 assign hw2reg.usbctrl.device_address.d = '0;
730
731 // Clear the stall flag when a SETUP is received
732
733 always_comb begin : proc_stall_tieoff
734   for (int i = 0; i < NEndpoints; i++) begin
735     hw2reg.in_stall[i].d = 1'b0;
736     hw2reg.out_stall[i].d = 1'b0;
737     if (setup_received && out_endpoint_val && out_endpoint
== 4'(unsigned'(i))) begin
738       hw2reg.out_stall[i].de = 1'b1;
739       hw2reg.in_stall[i].de = 1'b1;
740     end else begin
741       hw2reg.out_stall[i].de = 1'b0;
742       hw2reg.in_stall[i].de = 1'b0;
743     end
744   end
end

745 end
746
747 if (Stub) begin : gen_stubbed_memory
748   // Stub this window off with an error responder if
stubbed.
749   tlul_err_resp u_tlul_err_resp (
750     .clk_i,
751     .rst_ni,
752     .tl_h_i(tl_sram_h2d),
753     .tl_h_o(tl_sram_d2h)
754   );
755
756   // Tie off unused signals
757   assign sw_mem_a_req = '0;
758   assign sw_mem_a_gnt = '0;
759   assign sw_mem_a_write = '0;
760   assign sw_mem_a_addr = '0;
761   assign sw_mem_a_wdata = '0;
762   assign sw_mem_a_rvalid = '0;
763   assign sw_mem_a_rdata = '0;
764   assign sw_mem_a_rerror = '0;
765
766   assign usb_mem_b_rdata = '0;
767   assign ram_cfg_o =
prim_ram_1p_pkg::RAM_1P_CFG_RSP_DEFAULT;
768
769   logic unused_usb_mem_b_sigs;
770   assign unused_usb_mem_b_sigs = ^{
771     ram_cfg_i,
772     usb_mem_b_req,
773     usb_mem_b_write,
774     usb_mem_b_addr,
775     usb_mem_b_wdata,
776     usb_mem_b_rdata
777   };
778   end else begin : gen_no_stubbed_memory
779     // TL-UL to SRAM adapter
780     tlul_adapter_sram #(
781       .SramAw(SramAw),
782       .ByteAccess(0)
783     ) u_tlul2sram (
784       .clk_i,
785       .rst_ni,
786
787       .tl_i      (tl_sram_h2d),
788       .tl_o      (tl_sram_d2h),
789       .en_ifetch_i (prim_mubi_pkg::MuBi4False),
790       .req_o      (sw_mem_a_req),
791       .req_type_o (),
792       .gnt_i      (sw_mem_a_gnt),
793       .we_o       (sw_mem_a_write),
794       .addr_o     (sw_mem_a_addr),
795       .wdata_o    (sw_mem_a_wdata),
796       .wmask_o    (), // Not used
797       .intg_error_o (),
798       .user_rsvd_o (),
799       .rdata_i    (sw_mem_a_rdata),
800       .rvalid_i   (sw_mem_a_rvalid),
801       .rerror_i   (sw_mem_a_rerror),
802       .compound_txn_in_progress_o (),
803       .readback_en_i (prim_mubi_pkg::MuBi4False),
804       .readback_error_o (),
805       .wr_collision_i (1'b0),
806       .write_pending_i (1'b0)
807     );
808
809     // Single Port RAM implementation, which will award the
`usb` port absolute priority and
810     // delay `sw` access by in the event of a collision.
811     //
812     // In practice the `usb` access to memory is sporadic (4x
oversampling of bits, and read/write
813     // operations transfer 32 bits so on average the probability
of collision is just 1/128 even
814     // during active USB traffic, if the TL-UL interface were
active on every cycle).
815
816     // usb access has absolute priority, followed by any deferred
write, and then any sw access.
817     logic mem_req;
818     logic mem_write;
819     logic [SramAw-1:0] mem_addr;
820     logic [SramDw-1:0] mem_wdata;
821     assign mem_req = usb_mem_b_req | sw_mem_a_req;
822     assign mem_write = usb_mem_b_req ? usb_mem_b_write :
sw_mem_a_write;
823     assign mem_addr = usb_mem_b_req ? usb_mem_b_addr :
sw_mem_a_addr;
824     assign mem_wdata = usb_mem_b_req ? usb_mem_b_wdata :
sw_mem_a_wdata;
825
826     logic mem_rvalid;
827     logic [SramDw-1:0] mem_rdata;
828     logic [1:0] mem_rerror;
829     logic mem_rsteering;
830
831     // Always grant when no `usb` request.
832     assign sw_mem_a_gnt = !usb_mem_b_req;
833
834     // `usb` relies upon its read data remaining static after
read.

```

```

835 logic mem_b_read_q;
836 logic [SramDw-1:0] mem_b_rdata_q;
837
838 // Remember granted read accesses.
839 // NOTE: No pipelining within the RAM model.
840 always_ff @(posedge clk_i or negedge rst_ni) begin
841   if (!rst_ni) begin
842     mem_rsteering <= 1'b0;
843     mem_b_read_q <= 1'b0;
844     mem_b_rdata_q <= {SramDw{1'b0}};
845   end else begin
846     mem_rsteering <= usb_mem_b_req;
847     mem_b_read_q <= usb_mem_b_req & !usb_mem_b_write;
848     // Capture the `usb` read data.
849     if (mem_b_read_q)
850       mem_b_rdata_q <= mem_rdata;
851   end
852 end
853
854 // Read responses.
855 assign sw_mem_a_rvalid = mem_rvalid & !mem_rsteering;
856 assign sw_mem_a_rerror = {2{sw_mem_a_rvalid}} & mem_rer-
ror;
857
858 // We may safely return the read data to both (no security
859 // implications), but `usb` rdata
860 // must be held static after the read, and be unaffected
861 // by `sw` reads.
862 assign sw_mem_a_rdata = mem_rdata;
863 assign usb_mem_b_rdata = mem_b_read_q ? mem_rdata :
mem_b_rdata_q;
864
865 // SRAM Wrapper
866 prim_ram_1p_adv #(
867   .Depth (SramDepth),
868   .Width (SramDw), // 32 x 512 --> 2kB
869   .DataBitsPerMask(8),
870   .EnableECC (0), // No Protection
871   .EnableParity (0),
872   .EnableInputPipeline (0),
873   .EnableOutputPipeline(0)
874 ) u_memory_1p (
875   .clk_i,
876   .rst_ni,
877   .req_i (mem_req),
878   .write_i (mem_write),
879   .addr_i (mem_addr),
880   .wdata_i (mem_wdata),
881   .wmask_i ({SramDw{1'b1}}),
882   .rdata_o (mem_rdata),
883   .rvalid_o (mem_rvalid),
884   .rerror_o (mem_rerror),
885   .cfg_i (ram_cfg_i),
886   .cfg_o (ram_cfg_o),
887   .alert_o ()
888 );
889 end : gen_no_stubbed_memory
890
891 logic [NumAlerts-1:0] alert_test, alerts;
892
893 // Register module
894 usbdev_reg_top u_reg (
895   .clk_i,
896   .rst_ni, // this reset is not stubbed off so that the re-
gisters are still accessible.
897   .clk_aon_i,
898   .rst_aon_ni, // this reset is not stubbed off so that the
registers are still accessible.
899   .tl_i (tl_i),
900   .tl_o (tl_o),
901   .tl_win_o (tl_sram_h2d),
902   .tl_win_i (tl_sram_d2h),
903   .reg2hw(reg2hw_regtop),
904   .hw2reg(hw2reg_regtop),
905   // SEC_CM: BUS INTEGRITY
906   .intg_err_o (alerts[0])
907 );
908
909 // Stub off all register connections to reg_top.
910 if (Stub) begin : gen_stubbed
911   logic unused_sigs;
912   assign reg2hw = '0;
913   assign hw2reg_regtop = '0;
914   assign unused_sigs = ^{reg2hw_regtop, hw2reg};
915 end else begin : gen_not_stubbed
916   assign reg2hw = reg2hw_regtop;
917   assign hw2reg_regtop = hw2reg;
918 end
919
920 // Alerts
921 assign alert_test = {
922   reg2hw.alert_test.q &
923   reg2hw.alert_test.qe
924 };

```

```

929 // Alerts not stubbed off because registers and T-L access
930 // still present.
931 localparam logic [NumAlerts-1:0] AlertIsFatal = {Nu-
mAlerts{1'b1}};
932 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
933   prim_alert_sender #(
934     .AsyncOn(AlertAsyncOn[i]),
935     .SkewCycles(AlertSkewCycles),
936     .IsFatal(AlertIsFatal[i])
937   ) u_prim_alert_sender (
938     .clk_i,
939     .rst_ni, // this reset is not stubbed off so that the
pings still work.
940     .alert_test_i ( alert_test[i] ),
941     .alert_req_i ( alerts[i] ),
942     .alert_ack_o ( ),
943     .alert_state_o ( ),
944     .alert_rx_i ( alert_rx_i[i] ),
945     .alert_tx_o ( alert_tx_o[i] )
946   );
947 end
948
949 // Interrupts
950 prim_intr_hw #(.Width(1), .IntrT("Status")) intr_hw_pkt_re-
ceived (
951   .clk_i,
952   .rst_ni, // not stubbed off so that the interrupt regs
still work.
953   .event_intr_i (event_pkt_received),
954   .reg2hw_intr_enable_q_i (reg2hw_intr_enable.pkt_received.q),
955   .reg2hw_intr_test_q_i (reg2hw_intr_test.pkt_received.q),
956   .reg2hw_intr_test_qe_i (reg2hw_intr_test.pkt_received.qe),
957   .reg2hw_intr_state_q_i (reg2hw_intr_state.pkt_received.q),
958   .hw2reg_intr_state_de_o (hw2reg_intr_state.pkt_received.de),
959   .hw2reg_intr_state_d_o (hw2reg_intr_state.pkt_received.d),
960   .intr_o (intr_pkt_received_o)
961 );
962
963 prim_intr_hw #(.Width(1), .IntrT("Status")) intr_hw_pkt_sent (
964   .clk_i,
965   .rst_ni, // not stubbed off so that the interrupt regs
still work.
966   .event_intr_i (sent_event_pending),
967   .reg2hw_intr_enable_q_i (reg2hw_intr_enable.pkt_sent.q),
968   .reg2hw_intr_test_q_i (reg2hw_intr_test.pkt_sent.q),
969   .reg2hw_intr_test_qe_i (reg2hw_intr_test.pkt_sent.qe),
970   .reg2hw_intr_state_q_i (reg2hw_intr_state.pkt_sent.q),
971   .hw2reg_intr_state_de_o (hw2reg_intr_state.pkt_sent.de),
972   .hw2reg_intr_state_d_o (hw2reg_intr_state.pkt_sent.d),
973   .intr_o (intr_pkt_sent_o)
974 );
975
976 prim_intr_hw #(.Width(1)) intr_disconnected (
977   .clk_i,
978   .rst_ni, // not stubbed off so that the interrupt regs
still work.
979   .event_intr_i (event_disconnect),
980   .reg2hw_intr_enable_q_i (reg2hw_intr_enable.disconnected.q),
981   .reg2hw_intr_test_q_i (reg2hw_intr_test.disconnected.q),
982   .reg2hw_intr_test_qe_i (reg2hw_intr_test.disconnected.qe),
983   .reg2hw_intr_state_q_i (reg2hw_intr_state.disconnected.q),
984   .hw2reg_intr_state_de_o (hw2reg_intr_state.disconnected.de),
985   .hw2reg_intr_state_d_o (hw2reg_intr_state.disconnected.d),
986   .intr_o (intr_disconnected_o)
987 );
988
989 prim_intr_hw #(.Width(1)) intr_powered (
990   .clk_i,
991   .rst_ni, // not stubbed off so that the interrupt regs
still work.
992   .event_intr_i (event_powered),
993   .reg2hw_intr_enable_q_i (reg2hw_intr_enable.powered.q),
994   .reg2hw_intr_test_q_i (reg2hw_intr_test.powered.q),
995   .reg2hw_intr_test_qe_i (reg2hw_intr_test.powered.qe),
996   .reg2hw_intr_state_q_i (reg2hw_intr_state.powered.q),
997   .hw2reg_intr_state_de_o (hw2reg_intr_state.powered.de),
998   .hw2reg_intr_state_d_o (hw2reg_intr_state.powered.d),
999   .intr_o (intr_powered_o)
1000 );
1001
1002 prim_intr_hw #(.Width(1)) intr_host_lost (
1003   .clk_i,
1004   .rst_ni, // not stubbed off so that the interrupt regs
still work.
1005   .event_intr_i (event_host_lost),
1006   .reg2hw_intr_enable_q_i (reg2hw_intr_enable.host_lost.q),
1007   .reg2hw_intr_test_q_i (reg2hw_intr_test.host_lost.q),
1008   .reg2hw_intr_test_qe_i (reg2hw_intr_test.host_lost.qe),
1009   .reg2hw_intr_state_q_i (reg2hw_intr_state.host_lost.q),
1010   .hw2reg_intr_state_de_o (hw2reg_intr_state.host_lost.de),

```

```

1010 .hw2reg_intr_state_d_o (hw2reg.intr_state.host_lost.d),
1011 .intr_o (intr_host_lost_o)
1012 );
1013
1014 prim_intr_hw #(.Width(1)) intr_link_reset (
1015 .clk_i,
1016 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1017 .event_intr_i (event_link_reset),
1018 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.link_reset.q),
1019 .reg2hw_intr_test_q_i (reg2hw.intr_test.link_reset.q),
1020 .reg2hw_intr_test_qe_i (reg2hw.intr_test.link_reset.qe),
1021 .reg2hw_intr_state_q_i (reg2hw.intr_state.link_reset.q),
1022 .hw2reg_intr_state_de_o (hw2reg.intr_state.link_reset.de),
1023 .hw2reg_intr_state_d_o (hw2reg.intr_state.link_reset.d),
1024 .intr_o (intr_link_reset_o)
1025 );
1026
1027 prim_intr_hw #(.Width(1)) intr_link_suspend (
1028 .clk_i,
1029 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1030 .event_intr_i (event_link_suspend),
1031 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.link_suspend.q),
1032 .reg2hw_intr_test_q_i (reg2hw.intr_test.link_suspend.q),
1033 .reg2hw_intr_test_qe_i (reg2hw.intr_test.link_suspend.qe),
1034 .reg2hw_intr_state_q_i (reg2hw.intr_state.link_suspend.q),
1035 .hw2reg_intr_state_de_o (hw2reg.intr_state.link_suspend.de),
1036 .hw2reg_intr_state_d_o (hw2reg.intr_state.link_suspend.d),
1037 .intr_o (intr_link_suspend_o)
1038 );
1039
1040 prim_intr_hw #(.Width(1)) intr_link_resume (
1041 .clk_i,
1042 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1043 .event_intr_i (event_link_resume),
1044 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.link_resume.q),
1045 .reg2hw_intr_test_q_i (reg2hw.intr_test.link_resume.q),
1046 .reg2hw_intr_test_qe_i (reg2hw.intr_test.link_resume.qe),
1047 .reg2hw_intr_state_q_i (reg2hw.intr_state.link_resume.q),
1048 .hw2reg_intr_state_de_o (hw2reg.intr_state.link_resume.de),
1049 .hw2reg_intr_state_d_o (hw2reg.intr_state.link_resume.d),
1050 .intr_o (intr_link_resume_o)
1051 );
1052
1053 prim_intr_hw #(.Width(1), .IntrT("Status"))
intr_av_out_empty (
1054 .clk_i,
1055 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1056 .event_intr_i (event_av_out_empty),
1057 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.av_out_empty.q),
1058 .reg2hw_intr_test_q_i (reg2hw.intr_test.av_out_empty.q),
1059 .reg2hw_intr_test_qe_i (reg2hw.intr_test.av_out_empty.qe),
1060 .reg2hw_intr_state_q_i (reg2hw.intr_state.av_out_empty.q),
1061 .hw2reg_intr_state_de_o (hw2reg.intr_state.av_out_empty.de),
1062 .hw2reg_intr_state_d_o (hw2reg.intr_state.av_out_empty.d),
1063 .intr_o (intr_av_out_empty_o)
1064 );
1065
1066 prim_intr_hw #(.Width(1), .IntrT("Status")) intr_rx_full (
1067 .clk_i,
1068 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1069 .event_intr_i (event_rx_full),
1070 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.rx_full.q),
1071 .reg2hw_intr_test_q_i (reg2hw.intr_test.rx_full.q),
1072 .reg2hw_intr_test_qe_i (reg2hw.intr_test.rx_full.qe),
1073 .reg2hw_intr_state_q_i (reg2hw.intr_state.rx_full.q),
1074 .hw2reg_intr_state_de_o (hw2reg.intr_state.rx_full.de),
1075 .hw2reg_intr_state_d_o (hw2reg.intr_state.rx_full.d),
1076 .intr_o (intr_rx_full_o)
1077 );
1078
1079 prim_intr_hw #(.Width(1)) intr_av_overflow (
1080 .clk_i,
1081 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1082 .event_intr_i (event_av_overflow),
1083 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.av_overflow.q),
1084 .reg2hw_intr_test_q_i (reg2hw.intr_test.av_overflow.q),
1085 .reg2hw_intr_test_qe_i (reg2hw.intr_test.av_overflow.qe),
1086 .reg2hw_intr_state_q_i (reg2hw.intr_state.av_overflow.q),
1087 .hw2reg_intr_state_de_o (hw2reg.intr_state.av_overflow.de),
1088 .hw2reg_intr_state_d_o (hw2reg.intr_state.av_overflow.d),
1089 .intr_o (intr_av_overflow_o)
1090 );
1091
1092 prim_intr_hw #(.Width(1)) intr_link_in_err (
1093 .clk_i,
1094 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1095 .event_intr_i (event_in_err),
1096 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.link_in_err.q),
1097 .reg2hw_intr_test_q_i (reg2hw.intr_test.link_in_err.q),
1098 .reg2hw_intr_test_qe_i (reg2hw.intr_test.link_in_err.qe),
1099 .reg2hw_intr_state_q_i (reg2hw.intr_state.link_in_err.q),
1100 .hw2reg_intr_state_de_o (hw2reg.intr_state.link_in_err.de),
1101 .hw2reg_intr_state_d_o (hw2reg.intr_state.link_in_err.d),
1102 .intr_o (intr_link_in_err_o)
1103 );
1104
1105 prim_intr_hw #(.Width(1)) intr_link_out_err (
1106 .clk_i,
1107 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1108 .event_intr_i (event_out_err),
1109 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.link_out_err.q),
1110 .reg2hw_intr_test_q_i (reg2hw.intr_test.link_out_err.q),
1111 .reg2hw_intr_test_qe_i (reg2hw.intr_test.link_out_err.qe),
1112 .reg2hw_intr_state_q_i (reg2hw.intr_state.link_out_err.q),
1113 .hw2reg_intr_state_de_o (hw2reg.intr_state.link_out_err.de),
1114 .hw2reg_intr_state_d_o (hw2reg.intr_state.link_out_err.d),
1115 .intr_o (intr_link_out_err_o)
1116 );
1117
1118 prim_intr_hw #(.Width(1)) intr_rx_crc_err (
1119 .clk_i,
1120 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1121 .event_intr_i (event_rx_crc_err),
1122 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.rx_crc_err.q),
1123 .reg2hw_intr_test_q_i (reg2hw.intr_test.rx_crc_err.q),
1124 .reg2hw_intr_test_qe_i (reg2hw.intr_test.rx_crc_err.qe),
1125 .reg2hw_intr_state_q_i (reg2hw.intr_state.rx_crc_err.q),
1126 .hw2reg_intr_state_de_o (hw2reg.intr_state.rx_crc_err.de),
1127 .hw2reg_intr_state_d_o (hw2reg.intr_state.rx_crc_err.d),
1128 .intr_o (intr_rx_crc_err_o)
1129 );
1130
1131 prim_intr_hw #(.Width(1)) intr_rx_pid_err (
1132 .clk_i,
1133 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1134 .event_intr_i (event_rx_pid_err),
1135 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.rx_pid_err.q),
1136 .reg2hw_intr_test_q_i (reg2hw.intr_test.rx_pid_err.q),
1137 .reg2hw_intr_test_qe_i (reg2hw.intr_test.rx_pid_err.qe),
1138 .reg2hw_intr_state_q_i (reg2hw.intr_state.rx_pid_err.q),
1139 .hw2reg_intr_state_de_o (hw2reg.intr_state.rx_pid_err.de),
1140 .hw2reg_intr_state_d_o (hw2reg.intr_state.rx_pid_err.d),
1141 .intr_o (intr_rx_pid_err_o)
1142 );
1143
1144 prim_intr_hw #(.Width(1)) intr_rx_bitstuff_err (
1145 .clk_i,
1146 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1147 .event_intr_i (event_rx_bitstuff_err),
1148 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.rx_bitstuff_err.q),
1149 .reg2hw_intr_test_q_i (reg2hw.intr_test.rx_bitstuff_err.q),
1150 .reg2hw_intr_test_qe_i (reg2hw.intr_test.rx_bitstuff_err.qe),
1151 .reg2hw_intr_state_q_i (reg2hw.intr_state.rx_bitstuff_err.q),
1152 .hw2reg_intr_state_de_o (hw2reg.intr_state.rx_bitstuff_err.de),
1153 .hw2reg_intr_state_d_o (hw2reg.intr_state.rx_bitstuff_err.d),
1154 .intr_o (intr_rx_bitstuff_err_o)
1155 );
1156
1157 prim_intr_hw #(.Width(1)) intr_frame (
1158 .clk_i,
1159 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1160 .event_intr_i (event_frame),
1161 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.frame.q),
1162 .reg2hw_intr_test_q_i (reg2hw.intr_test.frame.q),
1163 .reg2hw_intr_test_qe_i (reg2hw.intr_test.frame.qe),
1164 .reg2hw_intr_state_q_i (reg2hw.intr_state.frame.q),
1165 .hw2reg_intr_state_de_o (hw2reg.intr_state.frame.de),
1166 .hw2reg_intr_state_d_o (hw2reg.intr_state.frame.d),
1167 .intr_o (intr_frame_o)
1168 );
1169
1170 prim_intr_hw #(.Width(1), .IntrT("Status"))
intr_av_setup_empty (

```

```

1171 .clk_i,
1172 .rst_n1, // not stubbed off so that the interrupt regs
still work.
1173 .event_intr_i (event_av_setup_empty),
1174 .reg2hw_intr_enable_q_i
(reg2hw.intr_enable.av_setup_empty.q),
1175 .reg2hw_intr_test_q_i
(reg2hw.intr_test.av_setup_empty.q),
1176 .reg2hw_intr_test_qe_i
(reg2hw.intr_test.av_setup_empty.qe),
1177 .reg2hw_intr_state_q_i
(reg2hw.intr_state.av_setup_empty.q),
1178 .hw2reg_intr_state_de_o
(hw2reg.intr_state.av_setup_empty.de),
1179 .hw2reg_intr_state_d_o
(hw2reg.intr_state.av_setup_empty.d),
1180 .intr_o (intr_av_setup_empty_o)
1181 );
1182
1183 ///////////////////////////////////////////////////
1184 // USB IO Muxing //
1185 ///////////////////////////////////////////////////
1186 logic cio_usb_oe;
1187 logic usb_rx_enable;
1188 assign cio_usb_dp_en_o = cio_usb_oe;
1189 assign cio_usb_dn_en_o = cio_usb_oe;
1190 assign usb_tx_use_d_se0_o =
reg2hw.phy_config.tx_use_d_se0.q; // cdc ok: quasi-static
assign hw2reg.usbstat.sense.d = usb_pwr_sense;
1191
1192 usbdev_iomux i_usbdev_iomux (
1193 .clk_i,
1194 .rst_n1 (rst_n),
1195
1196 // Register interface
1197 .hw2reg_sense_o (hw2reg.phy_pins_sense),
1198 .reg2hw_drive_i (reg2hw.phy_pins_drive),
1199
1200 // Chip IO
1201 .usb_rx_d_i (usb_rx_d_i),
1202 .usb_rx_dp_i (cio_usb_dp_i),
1203 .usb_rx_dn_i (cio_usb_dn_i),
1204 .cio_usb_sense_i (cio_sense_i),
1205 .usb_tx_d_o (usb_tx_d_o),
1206 .usb_tx_se0_o (usb_tx_se0_o),
1207 .usb_tx_dp_o (cio_usb_dp_o),
1208 .usb_tx_dn_o (cio_usb_dn_o),
1209 .usb_tx_oe_o (cio_usb_oe),
1210 .usb_dp_pullup_en_o (usb_dp_pullup_o),
1211 .usb_dn_pullup_en_o (usb_dn_pullup_o),
1212 .usb_rx_enable_o (usb_rx_enable_o),
1213
1214 // Internal interface
1215 .usb_rx_d_o (usb_rx_d),
1216 .usb_rx_dp_o (usb_rx_dp),
1217 .usb_rx_dn_o (usb_rx_dn),
1218 .usb_tx_d_i (usb_tx_d),
1219 .usb_tx_se0_i (usb_tx_se0),
1220 .usb_tx_dp_i (usb_tx_dp),
1221 .usb_tx_dn_i (usb_tx_dn),
1222 .usb_tx_oe_i (usb_tx_oe),
1223 .usb_pwr_sense_o (usb_pwr_sense),
1224 .usb_dp_pullup_en_i (usb_dp_pullup_en),
1225 .usb_dn_pullup_en_i (usb_dn_pullup_en),
1226 .usb_rx_enable_i (usb_rx_enable)
1227 );
1228
1229 // Differential receiver enable
1230 assign use_diff_rcvr = reg2hw.phy_config.use_diff_rcvr.q;
1231
1232 // enable rx only when the single-ended input is enabled and
the device is
1233 // not suspended (unless it is forced on in the I/O mux).
1234 assign usb_rx_enable = use_diff_rcvr & ~link_suspend;
1235
1236 // Symbols from the differential receiver are invalid until
it has finished
1237 // waking up / powering on
1238
1239 // Add 1 to the specified time to account for uncertainty in
the
1240 // free-running counter for us_tick.
1241 localparam int RcvrWakeTimeWidth = vbits(RcvrWakeTimeUs +
2);
1242 logic [RcvrWakeTimeWidth-1:0] usb_rcvr_ok_counter_d, usb_r-
cvr_ok_counter_q;
1243 assign usb_diff_rx_ok = (usb_rcvr_ok_counter_q == '0);
1244 always_comb begin
1245 // When don't need to use a differential receiver, RX is
always ready
1246 usb_rcvr_ok_counter_d = '0;
1247 if (use_diff_rcvr & !usb_rx_enable_o) begin
1248 usb_rcvr_ok_counter_d = RcvrWake-
TimeWidth'(RcvrWakeTimeUs + 1);
1249 end else if (us_tick && (usb_rcvr_ok_counter_q > '0)) be-
gin
1250 usb_rcvr_ok_counter_d = usb_rcvr_ok_counter_q - 1;
1251 end
1252 end
1253
1254 always_ff @(posedge clk_i or negedge rst_n) begin
1255 if (!rst_n) begin
1256 usb_rcvr_ok_counter_q <= RcvrWake-
TimeWidth'(RcvrWakeTimeUs + 1);
1257 end else begin
1258 usb_rcvr_ok_counter_q <= usb_rcvr_ok_counter_d;
1259 end
1260 end
1261
1262 ///////////////////////////////////////////////////
1263 // SOF Reference for Clock Calibration //
1264 ///////////////////////////////////////////////////
1265
1266 logic usb_ref_val_d, usb_ref_val_q;
1267 logic usb_ref_disable;
1268 assign usb_ref_disable =
reg2hw.phy_config.usb_ref_disable.q;
1269
1270 // Directly forward the pulse unless disabled.
1271 assign usb_ref_pulse_o = usb_ref_disable ? 1'b0 : event_sof;
1272
1273 // The first pulse is always ignored, but causes the valid
to be asserted.
1274 // The valid signal is deasserted when:
1275 // - The link is no longer active.
1276 // - The host is lost (no SOF for 4ms).
1277 // - The reference generation is disabled.
1278 assign usb_ref_val_d =
usb_ref_pulse_o ? 1'b1 :
(!link_active || host_lost || usb_ref_disable) ? 1'b0 :
usb_ref_val_q;
1279
1280 always_ff @(posedge clk_i or negedge rst_n) begin
1281 if (!rst_n) begin
1282 usb_ref_val_q <= 1'b0;
1283 end else begin
1284 usb_ref_val_q <= usb_ref_val_d;
1285 end
1286 end
1287
1288 assign usb_ref_val_o = usb_ref_val_q;
1289
1290 ///////////////////////////////////////////////////
1291 // USB aon detector signaling //
1292 ///////////////////////////////////////////////////
1293 assign usb_aon_suspend_req_o =
reg2hw.wake_control.suspend_req.qe &
1294 reg2hw.wake_control.suspend_req.q;
1295
1296 assign usb_aon_wake_ack_o = reg2hw.wake_control.wake_ack.qe
&
1297 reg2hw.wake_control.wake_ack.q;
1298
1299 ///////////////////////////////////////////////////
1300 // capture async event and debug info //
1301 ///////////////////////////////////////////////////
1302
1303 assign hw2reg.wake_events.module_active.de = 1'b1;
1304 assign hw2reg.wake_events.module_active.d =
usb_aon_wake_detect_active_i;
1305 assign hw2reg.wake_events.bus_not_idle.de = 1'b1;
1306 assign hw2reg.wake_events.bus_not_idle.d =
usb_aon_bus_not_idle_i;
1307 assign hw2reg.wake_events.disconnected.de = 1'b1;
1308 assign hw2reg.wake_events.disconnected.d =
usb_aon_sense_lost_i;
1309 assign hw2reg.wake_events.bus_reset.de = 1'b1;
1310 assign hw2reg.wake_events.bus_reset.d = usb_aon_bus_reset_i;
1311
1312 ///////////////////////////////////////////////////
1313 // Diagnostic/performance counters //
1314 ///////////////////////////////////////////////////
1315
1316 // SW write strobes for the event enables of the counters.
1317 logic ctr_out_ev_qe;
1318 logic ctr_in_ev_qe;
1319 logic ctr_errors_ev_qe;
1320 assign ctr_out_ev_qe = &{reg2hw.count_out.ign_avsetup.qe,
reg2hw.count_out.drop_avout.qe,
reg2hw.count_out.drop_rx.qe,
reg2hw.count_out.datatog_out.qe};
1321
1322 assign ctr_in_ev_qe = &{reg2hw.count_in.timeout.qe,
reg2hw.count_in.nak.qe,
reg2hw.count_in.nodata.qe};
1323
1324 assign ctr_errors_ev_qe = &{reg2hw.count_errors.crc5.qe,
reg2hw.count_errors.crc16.qe,
reg2hw.count_errors.bitstuff.qe,
1325 reg2hw.count_errors.pid_invalid.qe};
1326
1327 // Counters use 'rst_n' and remain at zero in Stubbed imple-
mentation
1328 usbdev_counter #(NEndpoints(NEndpoints), .NEvents(4))
u_ctr_out(
1329 .clk_i (clk_i),
1330 .rst_n1 (rst_n),
1331 .reset_i (reg2hw.count_out.rst.qe &
reg2hw.count_out.rst.q),
1332 .event_i ({event_ign_avsetup,
event_drop_avout,
event_drop_rx,

```

```

1340         event_datatog_out}},
1341     .ep_i         (out_endpoint),
1342     // Set of events being counted.
1343     .ev_qe_i      (ctr_out_ev_qe),
1344     .ev_i         ({reg2hw.count_out.ign_avsetup.q,
1345                     reg2hw.count_out.drop_avout.q,
1346                     reg2hw.count_out.drop_rx.q,
1347                     reg2hw.count_out.datatog_out.q}),
1348     .ev_o         ({hw2reg.count_out.ign_avsetup.d,
1349                     hw2reg.count_out.drop_avout.d,
1350                     hw2reg.count_out.drop_rx.d,
1351                     hw2reg.count_out.datatog_out.d}),
1352     // Endpoints being monitored.
1353     .endp_qe_i    (reg2hw.count_out.endpoints.qe),
1354     .endpoints_i  (reg2hw.count_out.endpoints.q),
1355     .endpoints_o  (hw2reg.count_out.endpoints.d),
1356     .count_o     (hw2reg.count_out.count.d)
1357 );
1358
1359 usbdev_counter #(.NEndpoints(NEndpoints), .NEvents(3))
u_ctr_in(
1360     .clk_i        (clk_i),
1361     .rst_ni       (rst_n),
1362     .reset_i      (reg2hw.count_in.rst.qe &
reg2hw.count_in.rst.q),
1363     .event_i      ({event_timeout_in,
1364                     event_nak_in,
1365                     event_nodata_in}),
1366     .ep_i         (in_endpoint),
1367     // Set of events being counted.
1368     .ev_qe_i      (ctr_in_ev_qe),
1369     .ev_i         ({reg2hw.count_in.timeout.q,
1370                     reg2hw.count_in.nak.q,
1371                     reg2hw.count_in.nodata.q}),
1372     .ev_o         ({hw2reg.count_in.timeout.d,
1373                     hw2reg.count_in.nak.d,
1374                     hw2reg.count_in.nodata.d}),
1375     // Endpoints being monitored.
1376     .endp_qe_i    (reg2hw.count_in.endpoints.qe),
1377     .endpoints_i  (reg2hw.count_in.endpoints.q),
1378     .endpoints_o  (hw2reg.count_in.endpoints.d),
1379     .count_o     (hw2reg.count_in.count.d)
1380 );
1381
1382 usbdev_counter #(.NEndpoints(NEndpoints), .NEvents(1))
u_ctr_nodata_in(
1383     .clk_i        (clk_i),
1384     .rst_ni       (rst_n),
1385     .reset_i      (reg2hw.count_nodata_in.rst.qe &
reg2hw.count_nodata_in.rst.q),
1386     .event_i      (event_nodata_in),
1387     .ep_i         (in_endpoint),
1388     // Single event for this counter, so enables not required.
1389     .ev_qe_i      (1'b1),
1390     .ev_i         (1'b1),
1391     .ev_o         (), // not used
1392     // Endpoints being monitored.
1393     .endp_qe_i    (reg2hw.count_nodata_in.endpoints.qe),
1394     .endpoints_i  (reg2hw.count_nodata_in.endpoints.q),
1395     .endpoints_o  (hw2reg.count_nodata_in.endpoints.d),
1396     .count_o     (hw2reg.count_nodata_in.count.d)
1397 );
1398
1399 usbdev_counter #(.NEndpoints(1), .NEvents(4)) u_ctr_errors(
1400     .clk_i        (clk_i),
1401     .rst_ni       (rst_n),
1402     .reset_i      (reg2hw.count_errors.rst.qe &
reg2hw.count_errors.rst.q),
1403     .event_i      ({event_rx_crc5_err,
1404                     event_rx_crc16_err,
1405                     event_rx_bitstuff_err,
1406                     event_rx_pid_err}),
1407     .ep_i         (1'b0),
1408     // Set of events being counted.
1409     .ev_qe_i      (ctr_errors_ev_qe),
1410     .ev_i         ({reg2hw.count_errors.crc5.q,
1411                     reg2hw.count_errors.crc16.q,
1412                     reg2hw.count_errors.bitstuff.q,
1413                     reg2hw.count_errors.pid_invalid.q}),
1414     .ev_o         ({hw2reg.count_errors.crc5.d,
1415                     hw2reg.count_errors.crc16.d,
1416                     hw2reg.count_errors.bitstuff.d,
1417                     hw2reg.count_errors.pid_invalid.d}),
1418     // This events are not reliably associated with specific
endpoints, so no endpoint enables.
1419     .endp_qe_i    (1'b1),
1420     .endpoints_i  (1'b1),
1421     .endpoints_o  (), // not used
1422     .count_o     (hw2reg.count_errors.count.d)
1423 );
1424
1425 ///////////////////////////////////////////////////////////////////
1426 // Xprop assertions on outputs //
1427 ///////////////////////////////////////////////////////////////////
1428
1429 `ASSERT_KNOWN(TL0DValidKnown_A, tl_o.d_valid)
1430 `ASSERT_KNOWN(TL0AReadyKnown_A, tl_o.a_ready)
1431 // These pins are not necessarily associated with any clock
but it probably makes most sense to
1432 // check them on the fastest clock.
1433 `ASSERT_KNOWN(USBTxDKnown_A, usb_tx_d_o)

```

```

1434 `ASSERT_KNOWN(CIODpKnown_A, cio_usb_dp_o)
1435 `ASSERT_KNOWN(CIODpEnKnown_A, cio_usb_dp_en_o)
1436 `ASSERT_KNOWN(CIODnKnown_A, cio_usb_dn_o)
1437 `ASSERT_KNOWN(CIODnEnKnown_A, cio_usb_dn_en_o)
1438 `ASSERT_KNOWN(USBTxSe0Known_A, usb_tx_se0_o)
1439 `ASSERT_KNOWN(USBDpPUKnown_A, usb_dp_pullup_o)
1440 `ASSERT_KNOWN(USBDnPUKnown_A, usb_dn_pullup_o)
1441 `ASSERT_KNOWN(USBRxEnableKnown_A, usb_rx_enable_o)
1442 `ASSERT_KNOWN(USBAonSuspendReqKnown_A, usb_aon_suspend_req_o)
1443 `ASSERT_KNOWN(USBAonWakeAckKnown_A, usb_aon_wake_ack_o)
1444 `ASSERT_KNOWN(USBRefValKnown_A, usb_ref_val_o, clk_i, !
rst_ni)
1445 `ASSERT_KNOWN(USBRefPulseKnown_A, usb_ref_pulse_o, clk_i, !
rst_ni)
1446 // Assert Known for alerts
1447 `ASSERT_KNOWN(AlertsKnown_A, alert_tx_o)
1448 //Interrupt signals
1449 `ASSERT_KNOWN(USBIntrPktRcvdKnown_A, intr_pkt_received_o)
1450 `ASSERT_KNOWN(USBIntrPktSentKnown_A, intr_pkt_sent_o)
1451 `ASSERT_KNOWN(USBIntrPwrKnown_A, intr_powered_o)
1452 `ASSERT_KNOWN(USBIntrDisConKnown_A, intr_disconnected_o)
1453 `ASSERT_KNOWN(USBIntrHostLostKnown_A, intr_host_lost_o)
1454 `ASSERT_KNOWN(USBIntrLinkRstKnown_A, intr_link_reset_o)
1455 `ASSERT_KNOWN(USBIntrLinkSusKnown_A, intr_link_suspend_o)
1456 `ASSERT_KNOWN(USBIntrLinkResKnown_A, intr_link_resume_o)
1457 `ASSERT_KNOWN(USBIntrAvOutEmptyKnown_A, intr_av_out_empty_o)
1458 `ASSERT_KNOWN(USBIntrRxFullKnown_A, intr_rx_full_o)
1459 `ASSERT_KNOWN(USBIntrAvOverflowKnown_A, intr_av_overflow_o)
1460 `ASSERT_KNOWN(USBIntrLinkInErrKnown_A, intr_link_in_err_o)
1461 `ASSERT_KNOWN(USBIntrLinkOutErrKnown_A, intr_link_out_err_o)
1462 `ASSERT_KNOWN(USBIntrRxCrErrKnown_A, intr_rx_crc_err_o)
1463 `ASSERT_KNOWN(USBIntrRxPidErrKnown_A, intr_rx_pid_err_o)
1464 `ASSERT_KNOWN(USBIntrRxBitstuffErrKnown_A, intr_rx_bitstuff_err_o)
1465 `ASSERT_KNOWN(USBIntrFrameKnown_A, intr_frame_o)
1466 `ASSERT_KNOWN(USBIntrAvSetupEmptyKnown_A,
intr_av_setup_empty_o)
1467
1468 // Alert assertions for reg_we onehot check
1469 `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
u_reg, alert_tx_o[0])
1470 endmodule

```

9. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_aon_wake.sv (206 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Always On USB wake detect
6 //
7
8 `include "prim_assert.sv"
9
10 module usbdev_aon_wake import usbdev_pkg::*;
11     input logic clk_aon_i,
12     input logic rst_aon_ni,
13
14     // These come from the chip pin
15     input logic usb_dp_i,
16     input logic usb_dn_i,
17     input logic usb_sense_i,
18
19     // These come from the IP
20     input logic usbdev_dppullup_en_i,
21     input logic usbdev_dnpullup_en_i,
22
23     // Register signals from the IP, which must already be syn-
chronized to the AON domain.
24     input logic wake_ack_aon_i,
25     input logic suspend_req_aon_i,
26
27     // The I/Os that need to be maintained in low-power mode
28     output logic usb_dppullup_en_o,
29     output logic usb_dnpullup_en_o,
30
31     // wake/powerup request
32     output logic wake_req_aon_o,
33
34     // Event signals that indicate what happened while monitor-
ing
35     output logic bus_not_idle_aon_o,
36     output logic bus_reset_aon_o,
37     output logic sense_lost_aon_o,
38
39     // state debug information
40     output logic wake_detect_active_aon_o
41 );
42
43 // Whether this AON wake detector is active and controlling
the pull-ups.
44     logic wake_detect_active_d, wake_detect_active_q;
45
46 // Detect whether the USB has moved from an idle state.
47     logic not_idle_async;

```



```

48 logic event_not_idle;
49
50 // In suspend it is the device pullup that sets the line
state
51 // so if the input value differs then the host is doing
something
52 // This covers both host generated wake (J->K) and host gen-
erated reset (J->SE0)
53 // Use of the pullups takes care of pinflipping
54 assign not_idle_async = (usb_dp_i != usb_dppullup_en_o) |
55 (usb_dn_i != usb_dnpullup_en_o);
56
57 // aon clock is ~200kHz so 4 cycle filter is about 20us
58 // as well as noise debounce this gives the main IP time to
detect resume if it didn't turn off
59 prim_filter #(
60 .AsyncOn(1), // Instantiate 2-stage synchronizer
61 .Cycles(4)
62 ) filter_activity (
63 .clk_i (clk_aon_i),
64 .rst_ni (rst_aon_ni),
65 .enable_i (1'b1),
66 .filter_i (not_idle_async),
67 .filter_o (event_not_idle)
68 );
69
70 // Detect bus reset and VBUS removal events.
71 // Hold the detectors in reset when this module is in the
idle state, to
72 // avoid sampling / hysteresis issues that carry over when
the link is
73 // active.
74 logic se0_async, sense_lost_async;
75 logic event_bus_reset, event_sense_lost;
76 logic bus_not_idle_d, bus_not_idle_q;
77 logic bus_reset_d, bus_reset_q;
78 logic sense_lost_d, sense_lost_q;
79
80 assign se0_async = ~usb_dp_i & ~usb_dn_i;
81 assign sense_lost_async = ~usb_sense_i;
82
83 prim_filter #(
84 .AsyncOn(1),
85 .Cycles(3)
86 ) filter_bus_reset (
87 .clk_i (clk_aon_i),
88 .rst_ni (rst_aon_ni),
89 .enable_i (1'b1),
90 .filter_i (se0_async),
91 .filter_o (event_bus_reset)
92 );
93
94 prim_filter #(
95 .AsyncOn(1),
96 .Cycles(3)
97 ) filter_sense (
98 .clk_i (clk_aon_i),
99 .rst_ni (rst_aon_ni),
100 .enable_i (1'b1),
101 .filter_i (sense_lost_async),
102 .filter_o (event_sense_lost)
103 );
104
105 // USB has become non-idle while monitoring.
106 assign bus_not_idle_d = (event_not_idle | bus_not_idle_q) &
wake_detect_active_q;
107 // USB has issued a Bus Reset condition while monitoring.
108 assign bus_reset_d = (event_bus_reset | bus_reset_q) &
wake_detect_active_q;
109 // USB SENSE signal has been lost (Disconnection) while mon-
itoring.
110 assign sense_lost_d = (event_sense_lost | sense_lost_q) &
wake_detect_active_q;
111
112 // Detected events
113 assign bus_not_idle_aon_o = bus_not_idle_q;
114 assign bus_reset_aon_o = bus_reset_q;
115 assign sense_lost_aon_o = sense_lost_q;
116
117 logic wake_req_d, wake_req_q;
118 assign wake_req_d = wake_detect_active_q &
(event_not_idle | event_bus_reset |
event_sense_lost | wake_req_q);
119 assign wake_req_aon_o = wake_req_q;
120
121
122 always_ff @(posedge clk_aon_i or negedge rst_aon_ni) begin :
proc_reg_events
123 if (!rst_aon_ni) begin
124 bus_not_idle_q <= 1'b0;
125 bus_reset_q <= 1'b0;
126 sense_lost_q <= 1'b0;
127 wake_req_q <= 1'b0;
128 end else begin
129 bus_not_idle_q <= bus_not_idle_d;
130 bus_reset_q <= bus_reset_d;
131 sense_lost_q <= sense_lost_d;
132 wake_req_q <= wake_req_d;
133 end
134 end
135

```

```

136 always_comb begin : proc_awk_fsm
137 wake_detect_active_d = wake_detect_active_q;
138
139 unique case (wake_detect_active_q)
140 // No aon suspend entry has been requested or detected
141 1'b0: begin
142 if (suspend_req_aon_i) begin
143 wake_detect_active_d = 1'b1;
144 end
145 end
146
147 // The USB IP has passed control to this module for
handling the suspend
148 // request. wake_ack_i must be asserted to bring control
back to the USB
149 // IP.
150 1'b1: begin
151 if (wake_ack_aon_i) begin
152 wake_detect_active_d = 1'b0;
153 end
154 end
155 default : wake_detect_active_d = 1'b0;
156 endcase
157 end
158
159 `ASSERT_KNOWN(WakeDetectActiveAonKnown_A, wake_de-
tect_active_aon_o,
160 clk_aon_i, !rst_aon_ni)
161
162 always_ff @(posedge clk_aon_i or negedge rst_aon_ni) begin :
proc_reg_awk
163 if (!rst_aon_ni) begin
164 wake_detect_active_q <= 1'b0;
165 end else begin
166 wake_detect_active_q <= wake_detect_active_d;
167 end
168 end
169
170 assign wake_detect_active_aon_o = wake_detect_active_q;
171
172 // Control the pullup enable outputs from the AON module
when it's active
173 logic usbdev_dppullup_en_aon, usbdev_dnpullup_en_aon;
174 logic aon_dppullup_en_d, aon_dppullup_en_q;
175 logic aon_dnpullup_en_d, aon_dnpullup_en_q;
176
177 prim_flop_2sync #(
178 .Width(2)
179 ) u_pullup_en_cdc (
180 .clk_i(clk_aon_i),
181 .rst_ni(rst_aon_ni),
182 .d_i({usbdev_dppullup_en_i, usbdev_dnpullup_en_i}),
183 .q_o({usbdev_dppullup_en_aon, usbdev_dnpullup_en_aon})
184 );
185
186 assign aon_dppullup_en_d = wake_detect_active_q ? aon_dppul-
lup_en_q
187 : usb-
dev_dppullup_en_aon;
188 assign aon_dnpullup_en_d = wake_detect_active_q ? aon_dnpul-
lup_en_q
189 : usb-
dev_dnpullup_en_aon;
190
191 always_ff @(posedge clk_aon_i or negedge rst_aon_ni) begin :
proc_reg_pullup_en
192 if (!rst_aon_ni) begin
193 aon_dppullup_en_q <= 1'b0;
194 aon_dnpullup_en_q <= 1'b0;
195 end else begin
196 aon_dppullup_en_q <= aon_dppullup_en_d;
197 aon_dnpullup_en_q <= aon_dnpullup_en_d;
198 end
199 end
200
201 assign usb_dppullup_en_o = wake_detect_active_q ? aon_dppul-
lup_en_q
202 : usb-
dev_dppullup_en_i;
203 assign usb_dnpullup_en_o = wake_detect_active_q ? aon_dnpul-
lup_en_q
204 : usb-
dev_dnpullup_en_i;
205
206 endmodule

```

10. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_counter.sv

(88 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // USB device event counter
6 //

```

```

7 // At every rising edge within 'event_i' the counter samples
the current endpoint as indicated by

8 // 'ep_i' and increments the count if the endpoint in question is
enabled at that moment.
9 //
10 // This means the set of endpoints for which events are coun-
ted may be changed over time by
11 // software, whilst the USB device is active, multiplexing a
single counter amongst multiple
12 // sets of endpoints.
13 //

14 // The counter value saturates at its maximum and may be reset by
software at any point.
15
16 module usbdev_counter
17 #(
18     parameter int unsigned NEndpoints = 12, // Number of end-
points supported.
19     parameter int unsigned NEvents = 1, // Number of events be-
ing counted.
20     parameter int unsigned Width = 8, // Counter width, in
bits.
21     localparam int unsigned EpW =
prim_util_pkg::vbits(NEndpoints) // derived parameter
22 ) (
23     input logic clk_i,
24     input logic rst_ni,
25     input logic reset_i, // Software reset
26     input logic [NEvents-1:0] event_i, // Event pulse
27     input logic [EpW-1:0] ep_i, // Endpoint on which
event occurred.
28     // Set of events being counted.
29     input logic ev_qe_i, // SW write to the
event enables.
30     input logic [NEvents-1:0] ev_i, // Per-event enable/dis-
able.
31     output logic [NEvents-1:0] ev_o, // Current event en-
ables.
32     // Endpoints being monitored.
33     input logic endp_qe_i, // SW write to the
endpoint enables.
34     input logic [NEndpoints-1:0] endpoints_i, // Per-endpoint
enable/disable.
35     output logic [NEndpoints-1:0] endpoints_o, // Current end-
point enables.
36     output logic [Width-1:0] count_o // Current value of
counter.
37 );
38
39 // Current per-endpoint and per-event enables.
40 logic [NEndpoints-1:0] endpoints;
41 logic [NEvents-1:0] ev_enables;
42 always_ff @(posedge clk_i or negedge rst_ni) begin
43     if (!rst_ni) begin
44         endpoints <= NEndpoints'(0);
45         ev_enables <= NEvents'(0);
46     end else begin
47         // Software writes to set the current enables.
48         if (endp_qe_i) endpoints <= endpoints_i;
49         if (ev_qe_i) ev_enables <= ev_i;
50     end
51 end
52
53 // Respond to events on this endpoint?
54 logic ep_enabled;
55 if (NEndpoints > 1) begin : gen_multi
56     assign ep_enabled = (ep_i < EpW'(NEndpoints)) ? end-
points[ep_i] : 1'b0;
57 end else begin : gen_single
58     logic unused_ep;
59     assign unused_ep = ^ep_i; // Endpoint number not required
60     assign ep_enabled = endpoints[0];
61 end
62
63 // Saturating event counter.
64 logic [Width-1:0] count;
65 logic [NEvents-1:0] event_q;
66
67 always_ff @(posedge clk_i or negedge rst_ni) begin
68     if (!rst_ni) begin
69         count <= Width'(0);
70         event_q <= NEvents'(0);
71     end else if (reset_i) begin
72         count <= Width'(0);
73     end
74 end
75
76 // Do not modify 'event_q' here because the event inputs
may still be asserted.
77 end else begin
78     // Positive-edge triggered, saturating count of events.
79     if (!(ev_enables & event_i & ~event_q) & ep_enabled)
count <= count + ~count;
80     // Retain previous state of event input signals.
81     event_q <= event_i;
82 end
83

```

```

84 assign ev_o = ev_enables;
85 assign endpoints_o = endpoints;
86 assign count_o = count;
87
88 endmodule

```

11. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_iomux.sv (147 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // USB IO Mux
6 //
7 // Muxes the USB IO signals between the override CSRs and the
USB engine. The
8 // incoming signals are also synchronized to the corresponding
clock domain.
9
10 module usbdev_iomux
11 import usbdev_reg_pkg::*;
12 (
13     input logic clk_i,
14     input logic rst_ni,
15
16     // Register interface
17     output usbdev_hw2reg_phy_pins_sense_reg_t hw2reg_sense_o,
18     input usbdev_reg2hw_phy_pins_drive_reg_t reg2hw_drive_i,
19
20     // External USB Inputs (async)
21     input logic usb_rx_d_i,
22     input logic usb_rx_dp_i,
23     input logic usb_rx_dn_i,
24     input logic cio_usb_sense_i,
25
26     // External USB outputs (usb clk)
27     output logic usb_tx_d_o,
28     output logic usb_tx_se0_o,
29     output logic usb_tx_dp_o,
30     output logic usb_tx_dn_o,
31     output logic usb_tx_oe_o,
32
33     output logic usb_dp_pullup_en_o,
34     output logic usb_dn_pullup_en_o,
35     output logic usb_rx_enable_o,
36
37     // Internal USB Interface (usb clk)
38     output logic usb_rx_d_o,
39     output logic usb_rx_dp_o,
40     output logic usb_rx_dn_o,
41     input logic usb_tx_d_i,
42     input logic usb_tx_se0_i,
43     input logic usb_tx_dp_i,
44     input logic usb_tx_dn_i,
45     input logic usb_tx_oe_i,
46     input logic usb_dp_pullup_en_i,
47     input logic usb_dn_pullup_en_i,
48     input logic usb_rx_enable_i,
49     output logic usb_pwr_sense_o
50 );
51
52 // CDCs
53 //
54 //
55
56 // USB input pins
57 prim_flop_2sync #(
58     .width(4)
59 ) cdc_io_to_usb (
60     .clk_i,
61     .rst_ni,
62     .d_i ({usb_rx_dp_i,
63            usb_rx_dn_i,
64            usb_rx_d_i,
65            cio_usb_sense_i}),
66     .q_o ({usb_rx_dp_o,
67            usb_rx_dn_o,
68            usb_rx_d_o,
69            usb_pwr_sense_o}),
70 );
71
72 // USB pins sense
73 assign hw2reg_sense_o.rx_dp_i.d = usb_rx_dp_o;
74 assign hw2reg_sense_o.rx_dn_i.d = usb_rx_dn_o;
75 assign hw2reg_sense_o.rx_d_i.d = usb_rx_d_o;
76 assign hw2reg_sense_o.pwr_sense.d = usb_pwr_sense_o;
77 assign hw2reg_sense_o.tx_dp_o.d = usb_tx_dp_i;
78 assign hw2reg_sense_o.tx_dn_o.d = usb_tx_dn_i;
79 assign hw2reg_sense_o.tx_d_o.d = usb_tx_d_i;
80 assign hw2reg_sense_o.tx_se0_o.d = usb_tx_se0_i;
81 assign hw2reg_sense_o.tx_oe_o.d = usb_tx_oe_i;
82
83
84 //
85 // USB output pin mux //
86 //

```

```

87 always_comb begin : proc_drive_out
88   if (reg2hw_drive_i.en.q) begin
89     // Override from registers
90     usb_dp_pullup_en_o = reg2hw_drive_i.dp_pullup_en_o.q;
91     usb_dn_pullup_en_o = reg2hw_drive_i.dn_pullup_en_o.q;
92     usb_rx_enable_o = reg2hw_drive_i.rx_enable_o.q;
93   end else begin
94     // Signals from the peripheral core
95     usb_dp_pullup_en_o = usb_dp_pullup_en_i;
96     usb_dn_pullup_en_o = usb_dn_pullup_en_i;
97     usb_rx_enable_o = usb_rx_enable_i;
98   end
99 end
100

101 // Use explicit muxes for the critical output signals, we do
this
102 // to avoid glitches from synthesized logic on these sig-
nals.

103 // Clock muxes should be used here to achieve the best match
between

104 // rising and falling edges on an ASIC. This mismatch on the
data line
105 // degrades performance in the JK-KJ jitter test.
106 prim_clock_mux2 #(
107   .NoFpgaBufG(1)
108 ) i_mux_tx_dp (
109   .clk0_i (usb_tx_dp_i),
110   .clk1_i (reg2hw_drive_i.dp_o.q),
111   .sel_i (reg2hw_drive_i.en.q),
112   .clk_o (usb_tx_dp_o)
113 );
114 prim_clock_mux2 #(
115   .NoFpgaBufG(1)
116 ) i_mux_tx_dn (
117   .clk0_i (usb_tx_dn_i),
118   .clk1_i (reg2hw_drive_i.dn_o.q),
119   .sel_i (reg2hw_drive_i.en.q),
120   .clk_o (usb_tx_dn_o)
121 );
122 prim_clock_mux2 #(
123   .NoFpgaBufG(1)
124 ) i_mux_tx_d (
125   .clk0_i (usb_tx_d_i),
126   .clk1_i (reg2hw_drive_i.d_o.q),
127   .sel_i (reg2hw_drive_i.en.q),
128   .clk_o (usb_tx_d_o)
129 );
130 prim_clock_mux2 #(
131   .NoFpgaBufG(1)
132 ) i_mux_tx_se0 (
133   .clk0_i (usb_tx_se0_i),
134   .clk1_i (reg2hw_drive_i.se0_o.q),
135   .sel_i (reg2hw_drive_i.en.q),
136   .clk_o (usb_tx_se0_o)
137 );
138 prim_clock_mux2 #(
139   .NoFpgaBufG(1)
140 ) i_mux_tx_oe (
141   .clk0_i (usb_tx_oe_i),
142   .clk1_i (reg2hw_drive_i.oe_o.q),
143   .sel_i (reg2hw_drive_i.en.q),
144   .clk_o (usb_tx_oe_o)
145 );
146
147 endmodule

```

12. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_linkstate.sv (390 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Link state detection
6 //
7
8 `include "prim_assert.sv"
9
10 module usbdev_linkstate (
11   input logic clk_48mhz_i,
12   input logic rst_ni,
13   input logic us_tick_i,
14   input logic usb_sense_i,
15   input logic usb_dp_i,
16   input logic usb_dn_i,
17   input logic usb_oe_i,
18   input logic usb_pullup_en_i,
19   input logic rx_idle_det_i,
20   input logic rx_j_det_i,
21   input logic sof_detected_i,
22   input logic resume_link_active_i, // pulse
23
24   output logic link_disconnect_o, // level
25   output logic link_powered_o, // level
26   output logic link_reset_o, // level

```

```

27   output logic link_active_o, // level
28   output logic link_suspend_o, // level
29   output logic link_resume_o, // pulse
30   output logic host_lost_o, // level
31   output logic sof_missed_o, // pulse
32
33   output logic [2:0] link_state_o
34 );
35
36 // Suspend signaling is 3ms of J by spec.
37 localparam logic [11:0] SUSPEND_TIMEOUT = 12'd3000;
38 // Reset is 2.5us - 10ms of SE0 by spec, though care should
be taken to not
39 // confuse the 2 *low-speed* bit times (1.33us) of SE0 that
terminate resume
40 // signaling. Use 3us here.
41 localparam logic [2:0] RESET_TIMEOUT = 3'd3;
42
43 // Consider an SOF lost after 1.005 ms. The extra 5 us helps
accommodate
44 // the worst case frequency difference between the host and
device, due to a
45 // +/- 2500 ppm range around 12 MHz.
46 localparam logic [9:0] SOF_TIMEOUT = 10'd1005;
47
48 typedef enum logic [2:0] {
49   // No power and/or no pull-up connected state
50   LinkDisconnected = 0,
51   // Powered / connected states
52   LinkPowered = 1,
53   LinkPoweredSuspended = 2,
54   // Active states
55   LinkActiveNoSOF = 5,
56   LinkActive = 3,
57   LinkSuspended = 4,
58   LinkResuming = 6
59 } link_state_e;
60
61 typedef enum logic [1:0] {
62   NoRst,
63   RstCnt,
64   RstPend
65 } link_rst_state_e;
66
67 typedef enum logic [1:0] {
68   Active,
69   InactCnt,
70   InactPend
71 } link_inac_state_e;
72
73 link_state_e link_state_d, link_state_q;
74 logic see_pwr_sense;
75
76 // Reset FSM
77 logic [2:0] link_rst_timer_d, link_rst_timer_q;
78 link_rst_state_e link_rst_state_d, link_rst_state_q;
79 logic link_reset; // reset detected (level)
80
81 // Link inactivity detection
82 logic monitor_inac; // monitor link inactivity
83 link_inac_state_e link_inac_timer_d, link_inac_timer_q;
84 link_inac_state_e link_inac_state_d, link_inac_state_q;
85
86 // Events that are not triggered by a timeout
87 logic ev_bus_active;
88
89 // Events that are triggered by timeout
90 logic ev_bus_inactive, ev_reset;
91
92 assign link_disconnect_o = (link_state_q == LinkDisconnec-
ted);
93 assign link_powered_o = see_pwr_sense;
94 assign link_suspend_o = (link_state_q == LinkSuspended ||
link_state_q == LinkPoweredSuspended);
95 assign link_active_o = (link_state_q == LinkActive) ||
(link_state_q == LinkActiveNoSOF);
96 // Link state is stable, so we can output it to the register
assign link_state_o = link_state_q;
101
102 // If the PHY reflects the line state on rx pins when the
device is driving
103 // then the usb_oe_i check isn't needed here. But it seems
best to do the check
104 // to be robust in the face of different PHY designs.
105 assign line_se0_raw = (usb_dn_i == 1'b0) & (usb_dp_i ==
1'b0) & (usb_oe_i == 1'b0);
106
107 // four ticks is a bit time
108 // Could completely filter out 2-cycle EOP SE0 here but
// does not seem needed
109 prim_filter #(
110   .AsyncOn(0), // No synchronizer required
111   .Cycles(6)
112 ) filter_se0 (
113   .clk_i (clk_48mhz_i),
114   .rst_ni (rst_ni),
115   .enable_i (1'b1),
116   .filter_i (line_se0_raw),
117   .filter_o (see_se0)
118

```



```

119 );
120
121 prim_filter #(
122     .AsyncOn(0), // No synchronizer required
123     .Cycles(6)
124 ) filter_pwr_sense (
125     .clk_i    (clk_48mhz_i),
126     .rst_ni   (rst_ni),
127     .enable_i (1'b1),
128     .filter_i (usb_sense_i),
129     .filter_o (see_pwr_sense)
130 );
131
132 // Simple events
133 assign ev_bus_active = !rx_idle_det_i;
134
135 assign monitor_inac = see_pwr_sense ? ((link_state_q ==
LinkPowered) | link_active_o) :
136     1'b0;
137
138 always_comb begin
139     link_state_d = link_state_q;
140     link_resume_o = 0;
141
142     // If VBUS ever goes away the link has disconnected (like-
wise if the
143     // pull-up goes away / user requested disconnection)
144     if (!see_pwr_sense || !usb_pullup_en_i) begin
145         link_state_d = LinkDisconnected;
146     end else begin
147         unique case (link_state_q)
148             // No USB supply detected (USB spec: Attached)
149             LinkDisconnected: begin
150                 if (see_pwr_sense & usb_pullup_en_i) begin
151                     link_state_d = LinkPowered;
152                 end
153             end
154
155             LinkPowered: begin
156                 if (ev_reset) begin
157                     link_state_d = LinkActiveNoSOF;
158                 end else if (resume_link_active_i) begin
159                     // Software-directed jump to resume from LinkSus-
pend, in case
160                     // this module was previously powered down.
161                     link_state_d = LinkResuming;
162                 end else if (ev_bus_inactive) begin
163                     link_state_d = LinkPoweredSuspended;
164                 end
165             end
166
167             LinkPoweredSuspended: begin
168                 if (ev_reset) begin
169                     link_state_d = LinkActiveNoSOF;
170                 end else if (ev_bus_active) begin
171                     link_resume_o = 1;
172                     link_state_d = LinkPowered;
173                 end
174             end
175
176             // An event occurred that brought the link out of
LinkSuspended, but
177             // the end-of-resume signaling may not have occurred
yet.
178             // Park here before starting to count towards not see-
ing SOF. Wait for
179             // the end of resume signaling before expecting SOF.
The host will
180             // return the link to idle after a low-speed EOP. In-
stead of trying to
181             // capture the termination of resume signaling dir-
ectly, wait for
182             // any J / idle symbol (or a bus reset).
183             LinkResuming: begin
184                 if (rx_j_det_i | ev_reset) begin
185                     link_resume_o = 1;
186                     link_state_d = LinkActiveNoSOF;
187                 end
188             end
189
190             // Active but not yet seen a frame
191             // One reason for getting stuck here is the host
thinks it is a LS link
192             // which could happen if the flipped bit does not
match the actual pins
193             // Another is the SI is bad so good data is not re-
covered from the link
194             LinkActiveNoSOF: begin
195                 if (ev_bus_inactive) begin
196                     link_state_d = LinkSuspended;
197                 end else if (sof_detected_i) begin
198                     link_state_d = LinkActive;
199                 end
200             end
201
202             // Active (USB spec: Default / Address / Configured)
LinkActive: begin
203                 if (ev_bus_inactive) begin
204                     link_state_d = LinkSuspended;
205                 end else if (ev_reset) begin
206                     link_state_d = LinkActiveNoSOF;
207

```

```

208         end
209     end
210
211     LinkSuspended: begin
212         if (ev_reset) begin
213             link_resume_o = 1;
214             link_state_d = LinkActiveNoSOF;
215         end else if (ev_bus_active) begin
216             link_state_d = LinkResuming;
217         end
218     end
219
220     default: begin
221         link_state_d = LinkDisconnected;
222     end
223 endcase // case (link_state_q)
224 end
225
226 `ASSERT(LinkStateValid_A, link_state_q inside {LinkDiscon-
nected, LinkPowered,
227     LinkPoweredSuspended, LinkResuming, LinkActiveNoSOF,
LinkActive, LinkSuspended}, clk_48mhz_i)
228
229 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
230     if (!rst_ni) begin
231         link_state_q <= LinkDisconnected;
232     end else begin
233         link_state_q <= link_state_d;
234     end
235 end
236
237 // Reset detection //
238 // Here we clean up the SE0 signal and generate a single
ev_reset at
239 // the end of a valid reset
240
241 always_comb begin : proc_rst_fsm
242     link_rst_state_d = link_rst_state_q;
243     link_rst_timer_d = link_rst_timer_q;
244     ev_reset = 1'b0;
245     link_reset = 1'b0;
246
247     unique case (link_rst_state_q)
248         // No reset signal detected
249         NoRst: begin
250             if (see_se0) begin
251                 link_rst_state_d = RstCnt;
252                 link_rst_timer_d = 0;
253             end
254         end
255
256         // Reset signal detected -> counting
257         RstCnt: begin
258             if (!see_se0) begin
259                 link_rst_state_d = NoRst;
260             end else begin
261                 if (us_tick_i) begin
262                     if (link_rst_timer_q == RESET_TIMEOUT) begin
263                         link_rst_state_d = RstPend;
264                     end else begin
265                         link_rst_timer_d = link_rst_timer_q + 1;
266                     end
267                 end
268             end
269         end
270
271         // Detected reset -> wait for falling edge
272         RstPend: begin
273             if (!see_se0) begin
274                 link_rst_state_d = NoRst;
275                 ev_reset = 1'b1;
276             end
277             link_reset = 1'b1;
278         end
279
280         default : link_rst_state_d = NoRst;
281     endcase
282 end
283
284 `ASSERT(LinkRstStateValid_A, link_rst_state_q inside {NoRst,
RstCnt, RstPend}, clk_48mhz_i)
285
286 assign link_reset_o = link_reset;
287
288 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin :
proc_reg_rst
289     if (!rst_ni) begin
290         link_rst_state_q <= NoRst;
291         link_rst_timer_q <= 0;
292     end else begin
293         link_rst_state_q <= link_rst_state_d;
294         link_rst_timer_q <= link_rst_timer_d;
295     end
296 end
297
298 // Idle detection //
299

```

```

303 ///////////////////////////////////////////////////
304 // Here we clean up the idle signal and generate a single
ev_bus_inactive
305 // after the timer expires
306 always_comb begin : proc_idle_det
307     link_inac_state_d = link_inac_state_q;
308     link_inac_timer_d = link_inac_timer_q;
309     ev_bus_inactive = 0;
310
311     unique case (link_inac_state_q)
312         // Active or disabled
313         Active: begin
314             link_inac_timer_d = 0;
315             if (!ev_bus_active && monitor_inac) begin
316                 link_inac_state_d = InactCnt;
317             end
318         end
319
320         // Got an inactivity signal -> count duration
321         InactCnt: begin
322             if (ev_bus_active || !monitor_inac) begin
323                 link_inac_state_d = Active;
324             end else if (us_tick_i) begin
325                 if (link_inac_timer_q == SUSPEND_TIMEOUT) begin
326                     link_inac_state_d = InactPend;
327                     ev_bus_inactive = 1;
328                 end else begin
329                     link_inac_timer_d = link_inac_timer_q + 1;
330                 end
331             end
332         end
333
334         // Counter expired & event sent, wait here
335         InactPend: begin
336             if (ev_bus_active || !monitor_inac) begin
337                 link_inac_state_d = Active;
338             end
339         end
340
341         default : link_inac_state_d = Active;
342     endcase
343 end
344
345 `ASSERT(LincInacStateValid_A, link_inac_state_q inside {Active, InactCnt, InactPend}, clk_48mhz_i)
346
347 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin :
proc_reg_idle_det
348     if (!rst_ni) begin
349         link_inac_state_q <= Active;
350         link_inac_timer_q <= 0;
351     end else begin
352         link_inac_state_q <= link_inac_state_d;
353         link_inac_timer_q <= link_inac_timer_d;
354     end
355 end

```

```

356
357 ///////////////////////////////////////////////////
358 // Host loss and missing sof detection //
359 ///////////////////////////////////////////////////
360 // sof_missed if no SOF was observed in 1.005ms and the link
is active
361 // host_lost if 4 frames have gone by without observing a
SOF
362 logic [2:0] missed_sof_count;
363 logic [9:0] missing_sof_timer;
364
365 assign host_lost_o = missed_sof_count[2];
366 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
367     if (!rst_ni) begin
368         missed_sof_count <= '0;
369     end else begin
370         if (sof_detected_i || !link_active_o || link_reset) be-
gin
371             missed_sof_count <= '0;
372         end else if (sof_missed_o && !host_lost_o) begin
373             missed_sof_count <= missed_sof_count + 1;
374         end
375     end
376 end
377
378 assign sof_missed_o = (missing_sof_timer == SOF_TIMEOUT);
379 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
380     if (!rst_ni) begin
381         missing_sof_timer <= '0;
382     end else begin
383         if (sof_missed_o || sof_detected_i || !link_active_o ||
link_reset) begin
384             missing_sof_timer <= '0;
385         end else if (us_tick_i) begin
386             missing_sof_timer <= missing_sof_timer + 1;
387         end
388     end
389 end
390 endmodule

```

13. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_pkg.sv (7 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 package usbdev_pkg;
6
7 endpackage : usbdev_pkg

```

RISC-V 타이머

What this is. The mtime/mtimecmp registers and timer interrupt generation.

Why it exists. Because the privileged specification defines its behaviour exactly, leaving almost no implementation freedom. Verification reduces to conformance.

Undergraduate review · EECS concepts. Timers and counters; interrupt controllers; atomicity of wide counter reads.

Graduate · Practice note. Reading a 64-bit counter over a 32-bit bus can tear if the value rolls over between the two halves. Either the read order is specified or a shadow register is required — a classic and easily missed defect.

4 files • 1,088 lines

API — rv_timer.sv

Table 1024. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer.sv — module / interface / package (1개)

이름	종류
rv_timer	module

Table 1025. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer.sv — parameter (4개)

파라미터	형/폭	기본값
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
EnableRacl	bit	1'b0
RaclErrorRsp	bit	EnableRacl

Table 1026. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
intr_timer_expired_hart0_timer0_o	output	logic

API — rv_timer_reg_pkg.sv**Table 1027. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer_reg_pkg.sv — module / interface / package (1개)**

이름	종류
rv_timer_reg_pkg	package

Table 1028. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer_reg_pkg.sv — parameter (18개)

파라미터	형/폭	기본값
N_HARTS	int	1
N_TIMERS	int	1
NumAlerts	int	1
BlockAw	int	9
NumRegs	int	10
RV_TIMER_ALERT_TEST_OFFSET	logic [BlockAw-1:0]	9'h 0
RV_TIMER_CTRL_OFFSET	logic [BlockAw-1:0]	9'h 4
RV_TIMER_INTR_ENABLE0_OFFSET	logic [BlockAw-1:0]	9'h 100
RV_TIMER_INTR_STATE0_OFFSET	logic [BlockAw-1:0]	9'h 104
RV_TIMER_INTR_TEST0_OFFSET	logic [BlockAw-1:0]	9'h 108
RV_TIMER_CFG0_OFFSET	logic [BlockAw-1:0]	9'h 10c
RV_TIMER_TIMER_V_LOWER0_OFFSET	logic [BlockAw-1:0]	9'h 110
RV_TIMER_TIMER_V_UPPER0_OFFSET	logic [BlockAw-1:0]	9'h 114
RV_TIMER_COMPARE_LOWER0_0_OFFSET	logic [BlockAw-1:0]	9'h 118
RV_TIMER_COMPARE_UPPER0_0_OFFSET	logic [BlockAw-1:0]	9'h 11c
RV_TIMER_ALERT_TEST_RESVAL	logic [0:0]	1'h 0
RV_TIMER_ALERT_TEST_FATAL_FAULT_RESVAL	logic [0:0]	1'h 0
RV_TIMER_INTR_TEST0_RESVAL	logic [0:0]	1'h 0

API — rv_timer_reg_top.sv**Table 1029. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer_reg_top.sv — module / interface / package (1개)**

이름	종류
rv_timer_reg_top	module

Table 1030. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer_reg_top.sv — parameter (2개)

파라미터	형/폭	기본값
EnableRacl	bit	1'b0
RaclErrorRsp	bit	1'b1

Table 1031. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer_reg_top.sv — localparam (3개)

상수	형/폭	값
AW	int	9
DW	int	32
DBW	int	DW/8

Table 1032. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer_reg_top.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
reg2hw	output	rv_timer_reg_pkg::rv_timer_reg2hw_t
hw2reg	input	rv_timer_reg_pkg::rv_timer_hw2reg_t
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
intg_err_o	output	logic

API — timer_core.sv

Table 1033. lowRISC_opentitan/hw/ip/rv_timer/rtl/timer_core.sv — module / interface / package (1개)

이름	종류
timer_core	module

Table 1034. lowRISC_opentitan/hw/ip/rv_timer/rtl/timer_core.sv — parameter (1개)

파라미터	형/폭	기본값
N	int	1

Table 1035. lowRISC_opentitan/hw/ip/rv_timer/rtl/timer_core.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
active	input	
prescaler	input	[11:0]
step	input	[7:0]
tick	output	logic
mtime_d	output	logic [63:0]
mtime	input	[63:0]
intr	output	logic [N-1:0]

1. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer.sv (181 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5
6 `include "prim_assert.sv"
7
8 module rv_timer import rv_timer_reg_pkg::*;
9 #(
10     parameter logic [NumAlerts-1:0] AlertAsyn-
c0n    = {NumAlerts{1'b1}},
11     // Number of cycles a differential skew is tolerated on the
  alert signal
12     parameter int unsigned AlertSkew-
cycles    = 1,
13     parameter bit EnableR-
acl        = 1'b0,
14     parameter bit RactEr-
rorRsp     = EnableRacl,
15     parameter top_racl_pkg::racl_policy_sel_t RactPol-
cySelVec[NumRegs] = '{NumRegs{0}}
16 ) (
17     input clk_i,
18     input rst_ni,
19
20     input tlul_pkg::tl_h2d_t tl_i,
21     output tlul_pkg::tl_d2h_t tl_o,
22
23     input prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
  alert_rx_i,
24     output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
  alert_tx_o,
25
26     // RACL interface
27     input top_racl_pkg::racl_policy_vec_t racl_policies_i,
28     output top_racl_pkg::racl_error_log_t racl_error_o,
29
30     output logic intr_timer_expired_hart0_timer0_o
  );
31
32 rv_timer_reg2hw_t reg2hw;
33 rv_timer_hw2reg_t hw2reg;
34
35 logic [N_HARTS-1:0] active;
36
37 logic [11:0] prescaler [N_HARTS];
38 logic [7:0] step [N_HARTS];
39
40 logic [N_HARTS-1:0] tick;
41
42 logic [63:0] mtime_d [N_HARTS];
43 logic [63:0] mtime [N_HARTS];
44 logic [63:0] mtimecmp [N_HARTS][N_TIMERS]; // Only [harts]
  [0] is connected to mtimecmp CSRs
45 logic mtimecmp_update [N_HARTS][N_TIMERS];
46
47 logic [N_HARTS*N_TIMERS-1:0] intr_timer_set;
48 logic [N_HARTS*N_TIMERS-1:0] intr_timer_en;
49 logic [N_HARTS*N_TIMERS-1:0] intr_timer_test_q;
50 logic [N_HARTS-1:0] intr_timer_test_qe;
51 logic [N_HARTS*N_TIMERS-1:0] intr_timer_state_q;
52 logic [N_HARTS-1:0] intr_timer_state_de;
53 logic [N_HARTS*N_TIMERS-1:0] intr_timer_state_d;
54
55 logic [N_HARTS*N_TIMERS-1:0] intr_out;
56
57 // Connecting register interface to the signal //
58 // be automated. For the moment, it must
59 // be connected manually.
60 assign active[0] = reg2hw.ctrl[0].q;
61 assign prescaler = '{reg2hw.cfg0.prescale.q};
62 assign step = '{reg2hw.cfg0.step.q};
63
64 assign hw2reg.timer_v_upper0.de = tick[0];
65 assign hw2reg.timer_v_lower0.de = tick[0];
66 assign hw2reg.timer_v_upper0.d = mtime_d[0][63:32];
67 assign hw2reg.timer_v_lower0.d = mtime_d[0][31:0];
68 assign mtime[0] = {reg2hw.timer_v_upper0.q,
  reg2hw.timer_v_lower0.q};
69 assign mtimecmp =
  '{reg2hw.compare_upper0.q, reg2hw.compare_lower0.q}};
70 assign mtimecmp_update[0][0] = reg2hw.compare_upper0.qe |
  reg2hw.compare_lower0.qe;
71
72 assign intr_timer_expired_hart0_timer0_o = intr_out[0];
73 assign intr_timer_en = reg2hw.intr_enable[0].q;
74 assign intr_timer_state_q = reg2hw.intr_state[0].q;
75 assign intr_timer_test_q = reg2hw.intr_test[0].q;
76 assign intr_timer_test_qe = reg2hw.intr_test[0].qe;
77 assign hw2reg.intr_state[0].de = intr_timer_state_de |
  mtimecmp_update[0][0];
78 assign hw2reg.intr_state[0].d = intr_timer_state_d &
  ~mtimecmp_update[0][0];
79
80 for (genvar h = 0 ; h < N_HARTS ; h++) begin : gen_harts
81     prim_intr_hw #(
82         .Width(N_TIMERS)
83     ) u_intr_hw (
84         .clk_i,
85         .rst_ni,
86         .event_intr_i (intr_timer_set),
87         .reg2hw_intr_enable_q_i
  (intr_timer_en[h*N_TIMERS+:N_TIMERS]),
88         .reg2hw_intr_test_q_i
  (intr_timer_test_q[h*N_TIMERS+:N_TIMERS]),
89         .reg2hw_intr_test_qe_i (intr_timer_test_qe[h]),
90         .reg2hw_intr_state_q_i
  (intr_timer_state_q[h*N_TIMERS+:N_TIMERS]),
91         .hw2reg_intr_state_de_o (intr_timer_state_de),
92         .hw2reg_intr_state_d_o
  (intr_timer_state_d[h*N_TIMERS+:N_TIMERS]),
93         .intr_o (intr_out[h*N_TIMERS+:N_TIMERS])
  );
94
95     timer_core #(
96         .N (N_TIMERS)
97     ) u_core (
98         .clk_i,
99         .rst_ni,
100
101         .active (active[h]),
102         .prescaler (prescaler[h]),
103         .step (step[h]),
104
105         .tick (tick[h]),
106
107         .mtime_d (mtime_d[h]),
108         .mtime (mtime[h]),
109         .mtimecmp (mtimecmp[h]),
110
111         .intr (intr_timer_set[h*N_TIMERS+:N_TIMERS])
  );
112 end : gen_harts
113
114 // Register module
115 logic [NumAlerts-1:0] alert_test, alerts;
116 rv_timer_reg_top #(
117     .EnableRacl(EnableRacl),
118     .RactErrorRsp(RactErrorRsp),
119     .RactPolicySelVec(RactPolicySelVec)
120 ) u_reg (
121     .clk_i,
122     .rst_ni,
123
124     .tl_i,
125     .tl_o,
126
127     .reg2hw,
128     .hw2reg,
129
130     .racl_policies_i,
131     .racl_error_o,
132
133     // SEC_CM: BUS INTEGRITY
134     .intg_err_o (alerts[0])
  );
135
136 // Alerts
137 assign alert_test = {
138     reg2hw.alert_test.q &
139     reg2hw.alert_test.qe
140 };
141
142 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
143     prim_alert_sender #(
144         .AsyncOn(AlertAsyncOn[i]),
145         .SkewCycles(AlertSkewCycles),
146         .IsFatal(1'b1)
147     ) u_prim_alert_sender (
148         .clk_i,
149         .rst_ni,
150         .alert_test_i ( alert_test[i] ),
151         .alert_req_i ( alerts[i] ),
152         .alert_ack_o ( ),
153         .alert_state_o ( ),
154         .alert_rx_i ( alert_rx_i[i] ),
155         .alert_tx_o ( alert_tx_o[i] )
156     );
157 end
158
159 // Assertions //
160 //////////////////////////////////
161 `ASSERT_KNOWN(Tl0ValidKnown, tl_o.d_valid)
162 `ASSERT_KNOWN(Tl0AReadyKnown, tl_o.a_ready)
163 `ASSERT_KNOWN(AlertsKnown_A, alert_tx_o)
164 `ASSERT_KNOWN_IF(RactError0Known_A, racl_error_o, racl_er-
  ror_o.valid)

```

```

177 `ASSERT_KNOWN(IntrTimerExpiredHart0Timer0Known,
intr_timer_expired_hart0_timer0_0)
178
179 // Alert assertions for reg_we onehot check
180 `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
u_reg, alert_tx_o[0])
181 endmodule

```

2. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer_reg_pkg.sv

(159 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Package auto-generated by `reggen` containing data
structure
6
7 package rv_timer_reg_pkg;
8
9 // Param list
10 parameter int N_HARTS = 1;
11 parameter int N_TIMERS = 1;
12 parameter int NumAlerts = 1;
13
14 // Address widths within the block
15 parameter int BlockAw = 9;
16
17 // Number of registers for every interface
18 parameter int NumRegs = 10;
19
20 // Alert indices
21 typedef enum int {
22     AlertFatalFaultIdx = 0
23 } rv_timer_alert_idx_t;
24
25 ///////////////////////////////////////////////////
26 // Typedefs for registers //
27 ///////////////////////////////////////////////////
28
29 typedef struct packed {
30     logic q;
31     logic qe;
32 } rv_timer_reg2hw_alert_test_reg_t;
33
34 typedef struct packed {
35     logic q;
36 } rv_timer_reg2hw_ctrl_mreg_t;
37
38 typedef struct packed {
39     logic q;
40 } rv_timer_reg2hw_intr_enable0_mreg_t;
41
42 typedef struct packed {
43     logic q;
44 } rv_timer_reg2hw_intr_state0_mreg_t;
45
46 typedef struct packed {
47     logic q;
48     logic qe;
49 } rv_timer_reg2hw_intr_test0_mreg_t;
50
51 typedef struct packed {
52     struct packed {
53         logic [7:0] q;
54     } step;
55     struct packed {
56         logic [11:0] q;
57     } prescale;
58 } rv_timer_reg2hw_cfg0_reg_t;
59
60 typedef struct packed {
61     logic [31:0] q;
62 } rv_timer_reg2hw_timer_v_lower0_reg_t;
63
64 typedef struct packed {
65     logic [31:0] q;
66 } rv_timer_reg2hw_timer_v_upper0_reg_t;
67
68 typedef struct packed {
69     logic [31:0] q;
70     logic qe;
71 } rv_timer_reg2hw_compare_lower0_0_reg_t;
72
73 typedef struct packed {
74     logic [31:0] q;
75     logic qe;
76 } rv_timer_reg2hw_compare_upper0_0_reg_t;
77
78 typedef struct packed {
79     logic d;
80     logic de;
81 } rv_timer_hw2reg_intr_state0_mreg_t;
82
83 typedef struct packed {
84     logic [31:0] d;

```

```

85     logic de;
86 } rv_timer_hw2reg_timer_v_lower0_reg_t;
87
88 typedef struct packed {
89     logic [31:0] d;
90     logic de;
91 } rv_timer_hw2reg_timer_v_upper0_reg_t;
92
93 // Register -> HW type
94 typedef struct packed {
95     rv_timer_reg2hw_alert_test_reg_t alert_test; // [156:155]
96     rv_timer_reg2hw_ctrl_mreg_t [0:0] ctrl; // [154:154]
97
98     rv_timer_reg2hw_intr_enable0_mreg_t [0:0] intr_enable0; //
[153:153]
99     rv_timer_reg2hw_intr_state0_mreg_t [0:0] intr_state0; //
[152:152]
100     rv_timer_reg2hw_intr_test0_mreg_t [0:0] intr_test0; //
[151:150]
101     rv_timer_reg2hw_cfg0_reg_t cfg0; // [149:130]
102     rv_timer_reg2hw_timer_v_lower0_reg_t timer_v_lower0; //
[129:98]
103     rv_timer_reg2hw_timer_v_upper0_reg_t timer_v_upper0; //
[97:66]
104     rv_timer_reg2hw_compare_lower0_0_reg_t com-
pare_lower0_0; // [65:33]
105     rv_timer_reg2hw_compare_upper0_0_reg_t com-
pare_upper0_0; // [32:0]
106 } rv_timer_reg2hw_t;
107
108 // HW -> register type
109 typedef struct packed {
110     rv_timer_hw2reg_intr_state0_mreg_t [0:0] intr_state0; //
[67:66]
111     rv_timer_hw2reg_timer_v_lower0_reg_t timer_v_lower0; //
[65:33]
112     rv_timer_hw2reg_timer_v_upper0_reg_t timer_v_upper0; //
[32:0]
113 } rv_timer_hw2reg_t;
114
115 // Register offsets
116 parameter logic [BlockAw-1:0] RV_TIMER_ALERT_TEST_OFFSET =
9'h 0;
117 parameter logic [BlockAw-1:0] RV_TIMER_CTRL_OFFSET = 9'h 4;
118 parameter logic [BlockAw-1:0] RV_TIMER_INTR_ENABLE0_OFFSET =
9'h 100;
119 parameter logic [BlockAw-1:0] RV_TIMER_INTR_STATE0_OFFSET =
9'h 104;
120 parameter logic [BlockAw-1:0] RV_TIMER_INTR_TEST0_OFFSET =
9'h 108;
121 parameter logic [BlockAw-1:0] RV_TIMER_CFG0_OFFSET = 9'h
100;
122 parameter logic [BlockAw-1:0] RV_TIMER_TIMER_V_LOWER0_OFFSET =
9'h 110;
123 parameter logic [BlockAw-1:0] RV_TIMER_TIMER_V_UPPER0_OFFSET =
9'h 114;
124 parameter logic [BlockAw-1:0]
RV_TIMER_COMPARE_LOWER0_0_OFFSET = 9'h 118;
125 parameter logic [BlockAw-1:0]
RV_TIMER_COMPARE_UPPER0_0_OFFSET = 9'h 11c;
126
127 // Reset values for hwext registers and their fields
128 parameter logic [0:0] RV_TIMER_ALERT_TEST_RESVAL = 1'h 0;
129 parameter logic [0:0] RV_TIMER_INTR_TEST0_RESVAL = 1'h 0;
130
131 // Register index
132 typedef enum int {
133     RV_TIMER_ALERT_TEST,
134     RV_TIMER_CTRL,
135     RV_TIMER_INTR_ENABLE0,
136     RV_TIMER_INTR_STATE0,
137     RV_TIMER_INTR_TEST0,
138     RV_TIMER_CFG0,
139     RV_TIMER_TIMER_V_LOWER0,
140     RV_TIMER_TIMER_V_UPPER0,
141     RV_TIMER_COMPARE_LOWER0_0,
142     RV_TIMER_COMPARE_UPPER0_0
143 } rv_timer_id_e;
144
145 // Register width information to check illegal writes
146 parameter logic [3:0] RV_TIMER_PERMIT [10] = '{
147     4'b 0001, // index[0] RV_TIMER_ALERT_TEST
148     4'b 0001, // index[1] RV_TIMER_CTRL
149     4'b 0001, // index[2] RV_TIMER_INTR_ENABLE0
150     4'b 0001, // index[3] RV_TIMER_INTR_STATE0
151     4'b 0001, // index[4] RV_TIMER_INTR_TEST0
152     4'b 0111, // index[5] RV_TIMER_CFG0
153     4'b 1111, // index[6] RV_TIMER_TIMER_V_LOWER0
154     4'b 1111, // index[7] RV_TIMER_TIMER_V_UPPER0
155     4'b 1111, // index[8] RV_TIMER_COMPARE_LOWER0_0
156     4'b 1111 // index[9] RV_TIMER_COMPARE_UPPER0_0
157 };
158
159 endpackage

```

3. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer_reg_top.sv (700 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"
8
9 module rv_timer_reg_top
10 # (
11     parameter bit          EnableRacl          = 1'b0,
12     parameter bit          RaclErrorRsp        = 1'b1,
13     parameter top_racl_pkg::racl_policy_sel_t  RaclPoli-
cySelVec[rv_timer_reg_pkg::NumRegs] =
14     '{rv_timer_reg_pkg::NumRegs{0}}
15 ) (
16     input clk_i,
17     input rst_ni,
18     input tlul_pkg::tl_h2d_t tl_i,
19     output tlul_pkg::tl_d2h_t tl_o,
20     // To HW
21     output rv_timer_reg_pkg::rv_timer_reg2hw_t reg2hw, // Write
22     input  rv_timer_reg_pkg::rv_timer_hw2reg_t hw2reg, // Read
23
24     // RACL interface
25     input top_racl_pkg::racl_policy_vec_t racl_policies_i,
26     output top_racl_pkg::racl_error_log_t  racl_error_o,
27
28     // Integrity check errors
29     output logic intg_err_o
30 );
31
32 import rv_timer_reg_pkg::*;
33
34 localparam int AW = 9;
35 localparam int DW = 32;
36 localparam int DBW = DW/8; // Byte Width
37
38 // register signals
39 logic reg_we;
40 logic reg_re;
41 logic [AW-1:0] reg_addr;
42 logic [DW-1:0] reg_wdata;
43 logic [DBW-1:0] reg_be;
44 logic [DW-1:0] reg_rdata;
45 logic reg_error;
46
47 logic addrmiss, wr_err;
48
49 logic [DW-1:0] reg_rdata_next;
50 logic reg_busy;
51
52 tlul_pkg::tl_h2d_t tl_reg_h2d;
53 tlul_pkg::tl_d2h_t tl_reg_d2h;
54
55 // incoming payload check
56 logic intg_err;
57 tlul_cmd_intg_chk u_chk (
58     .tl_i(tl_i),
59     .err_o(intg_err)
60 );
61
62 // also check for spurious write enables
63 logic reg_we_err;
64 logic [9:0] reg_we_check;
65 prim_reg_we_check #(
66     .OneHotWidth(10)
67 ) u_prim_reg_we_check (
68     .clk_i(clk_i),
69     .rst_ni(rst_ni),
70     .oh_i (reg_we_check),
71     .en_i (reg_we && !addrmiss),
72     .err_o (reg_we_err)
73 );
74
75 logic err_q;
76 always_ff @(posedge clk_i or negedge rst_ni) begin
77     if (!rst_ni) begin
78         err_q <= '0;
79     end else if (intg_err || reg_we_err) begin
80         err_q <= 1'b1;
81     end
82 end
83
84 // integrity error output is permanent and should be used
  for alert generation
85 // register errors are transactional
86 assign intg_err_o = err_q | intg_err | reg_we_err;
87
88 // outgoing integrity generation
89 tlul_pkg::tl_d2h_t tl_o_pre;
90 tlul_rsp_intg_gen #(
91     .EnableRspIntgGen(1),
92
93     .EnableDataIntgGen(1)
94 ) u_rsp_intg_gen (
95     .tl_i(tl_o_pre),
96     .tl_o(tl_o)
97 );
98
99 assign tl_reg_h2d = tl_i;
100 assign tl_o_pre = tl_reg_d2h;
101
102 tlul_adapter_reg #(
103     .RegAw(AW),
104     .RegDw(DW),
105     .EnableDataIntgGen(0)
106 ) u_reg_if (
107     .clk_i (clk_i),
108     .rst_ni (rst_ni),
109
110     .tl_i (tl_reg_h2d),
111     .tl_o (tl_reg_d2h),
112
113     .en_ifetch_i(prim_mubi_pkg::MuBi4False),
114     .intg_error_o(),
115
116     .we_o (reg_we),
117     .re_o (reg_re),
118     .addr_o (reg_addr),
119     .wdata_o (reg_wdata),
120     .be_o (reg_be),
121     .busy_i (reg_busy),
122     .rdata_i (reg_rdata),
123     // Translate RACL error to TLUL error if enabled
124     .error_i (reg_error | (RaclErrorRsp & racl_error_o.valid))
125 );
126
127 // cdc oversampling signals
128
129 assign reg_rdata = reg_rdata_next;
130 assign reg_error = addrmiss | wr_err | intg_err;
131
132 // Define SW related signals
133 // Format: <reg>_<field>_<wd|we|qs>
134 // or <reg>_<wd|we|qs> if field == 1 or 0
135 logic alert_test_we;
136 logic alert_test_wd;
137 logic ctrl_we;
138 logic ctrl_qs;
139 logic ctrl_wd;
140 logic intr_enable0_we;
141 logic intr_enable0_qs;
142 logic intr_enable0_wd;
143 logic intr_state0_we;
144 logic intr_state0_qs;
145 logic intr_state0_wd;
146 logic intr_test0_we;
147 logic intr_test0_wd;
148 logic cfg0_we;
149 logic [11:0] cfg0_prescale_qs;
150 logic [11:0] cfg0_prescale_wd;
151 logic [7:0] cfg0_step_qs;
152 logic [7:0] cfg0_step_wd;
153 logic timer_v_lower0_we;
154 logic [31:0] timer_v_lower0_qs;
155 logic [31:0] timer_v_lower0_wd;
156 logic timer_v_upper0_we;
157 logic [31:0] timer_v_upper0_qs;
158 logic [31:0] timer_v_upper0_wd;
159 logic compare_lower0_0_we;
160 logic [31:0] compare_lower0_0_qs;
161 logic [31:0] compare_lower0_0_wd;
162 logic compare_upper0_0_we;
163 logic [31:0] compare_upper0_0_qs;
164 logic [31:0] compare_upper0_0_wd;
165
166 // Register instances
167 // R[alert_test]: V(True)
168 logic alert_test_qe;
169 logic [0:0] alert_test_flds_we;
170 assign alert_test_qe = &alert_test_flds_we;
171 prim_subreg_ext #(
172     .DW (1)
173 ) u_alert_test (
174     .re (1'b0),
175     .we (alert_test_we),
176     .wd (alert_test_wd),
177     .d ('0),
178     .qre (),
179     .qe (alert_test_flds_we[0]),
180     .q (reg2hw.alert_test.q),
181     .ds (),
182     .qs ()
183 );
184 assign reg2hw.alert_test.qe = alert_test_qe;
185
186 // Subregister 0 of Multireg ctrl
187 // R[ctrl]: V(False)
188 prim_subreg #(
189     .DW (1),
190     .SwAccess(prim_subreg_pkg::SwAccessRW),
191     .RESVAL (1'h0),
192     .Mubi (1'b0)
193

```



```

194 ) u_ctrl (
195   .clk_i   (clk_i),
196   .rst_ni  (rst_ni),
197
198   // from register interface
199   .we      (ctrl_we),
200   .wd      (ctrl_wd),
201
202   // from internal hardware
203   .de      (1'b0),
204   .d        ('0'),
205
206   // to internal hardware
207   .qe      (),
208   .q        (reg2hw.ctrl[0].q),
209   .ds      (),
210
211   // to register interface (read)
212   .qs      (ctrl_qs)
213 );
214
215 // Subregister 0 of Multireg intr_enable0
216 // R[intr_enable0]: V(False)
217 prim_subreg #(
218   .DW      (1),
219   .SwAccess(prim_subreg_pkg::SwAccessRW),
220   .RESVAL  (1'h0),
221   .Mubi    (1'b0)
222 ) u_intr_enable0 (
223   .clk_i   (clk_i),
224   .rst_ni  (rst_ni),
225
226   // from register interface
227   .we      (intr_enable0_we),
228   .wd      (intr_enable0_wd),
229
230   // from internal hardware
231   .de      (1'b0),
232   .d        ('0'),
233
234   // to internal hardware
235   .qe      (),
236   .q        (reg2hw.intr_enable0[0].q),
237   .ds      (),
238
239   // to register interface (read)
240   .qs      (intr_enable0_qs)
241 );
242
243 // Subregister 0 of Multireg intr_state0
244 // R[intr_state0]: V(False)
245 prim_subreg #(
246   .DW      (1),
247   .SwAccess(prim_subreg_pkg::SwAccessW1C),
248   .RESVAL  (1'h0),
249   .Mubi    (1'b0)
250 ) u_intr_state0 (
251   .clk_i   (clk_i),
252   .rst_ni  (rst_ni),
253
254   // from register interface
255   .we      (intr_state0_we),
256   .wd      (intr_state0_wd),
257
258   // from internal hardware
259   .de      (hw2reg.intr_state0[0].de),
260   .d        (hw2reg.intr_state0[0].d),
261
262   // to internal hardware
263   .qe      (),
264   .q        (reg2hw.intr_state0[0].q),
265   .ds      (),
266
267   // to register interface (read)
268   .qs      (intr_state0_qs)
269 );
270
271 // Subregister 0 of Multireg intr_test0
272 // R[intr_test0]: V(True)
273 logic intr_test0_qe;
274 logic [0:0] intr_test0_flds_we;
275 assign intr_test0_qe = &intr_test0_flds_we;
276 prim_subreg_ext #(
277   .DW      (1)
278 ) u_intr_test0 (
279   .re      (1'b0),
280   .we      (intr_test0_we),
281   .wd      (intr_test0_wd),
282   .d        ('0'),
283   .qre     (),
284   .qe      (intr_test0_flds_we[0]),
285   .q        (reg2hw.intr_test0[0].q),
286   .ds      (),
287   .qs      ()
288 );
289 assign reg2hw.intr_test0[0].qe = intr_test0_qe;
290
291

```

```

295 // R[cfg0]: V(False)
296 // F[prescale]: 11:0
297 prim_subreg #(
298   .DW      (12),
299   .SwAccess(prim_subreg_pkg::SwAccessRW),
300   .RESVAL  (12'h0),
301   .Mubi    (1'b0)
302 ) u_cfg0_prescale (
303   .clk_i   (clk_i),
304   .rst_ni  (rst_ni),
305
306   // from register interface
307   .we      (cfg0_we),
308   .wd      (cfg0_prescale_wd),
309
310   // from internal hardware
311   .de      (1'b0),
312   .d        ('0'),
313
314   // to internal hardware
315   .qe      (),
316   .q        (reg2hw.cfg0.prescale.q),
317   .ds      (),
318
319   // to register interface (read)
320   .qs      (cfg0_prescale_qs)
321 );
322
323 // F[step]: 23:16
324 prim_subreg #(
325   .DW      (8),
326   .SwAccess(prim_subreg_pkg::SwAccessRW),
327   .RESVAL  (8'h1),
328   .Mubi    (1'b0)
329 ) u_cfg0_step (
330   .clk_i   (clk_i),
331   .rst_ni  (rst_ni),
332
333   // from register interface
334   .we      (cfg0_we),
335   .wd      (cfg0_step_wd),
336
337   // from internal hardware
338   .de      (1'b0),
339   .d        ('0'),
340
341   // to internal hardware
342   .qe      (),
343   .q        (reg2hw.cfg0.step.q),
344   .ds      (),
345
346   // to register interface (read)
347   .qs      (cfg0_step_qs)
348 );
349
350 // R[timer_v_lower0]: V(False)
351 prim_subreg #(
352   .DW      (32),
353   .SwAccess(prim_subreg_pkg::SwAccessRW),
354   .RESVAL  (32'h0),
355   .Mubi    (1'b0)
356 ) u_timer_v_lower0 (
357   .clk_i   (clk_i),
358   .rst_ni  (rst_ni),
359
360   // from register interface
361   .we      (timer_v_lower0_we),
362   .wd      (timer_v_lower0_wd),
363
364   // from internal hardware
365   .de      (hw2reg.timer_v_lower0.de),
366   .d        (hw2reg.timer_v_lower0.d),
367
368   // to internal hardware
369   .qe      (),
370   .q        (reg2hw.timer_v_lower0.q),
371   .ds      (),
372
373   // to register interface (read)
374   .qs      (timer_v_lower0_qs)
375 );
376
377 // R[timer_v_upper0]: V(False)
378 prim_subreg #(
379   .DW      (32),
380   .SwAccess(prim_subreg_pkg::SwAccessRW),
381   .RESVAL  (32'h0),
382   .Mubi    (1'b0)
383 ) u_timer_v_upper0 (
384   .clk_i   (clk_i),
385   .rst_ni  (rst_ni),
386
387   // from register interface
388   .we      (timer_v_upper0_we),
389   .wd      (timer_v_upper0_wd),
390
391   // from internal hardware
392   .de      (hw2reg.timer_v_upper0.de),
393   .d        (hw2reg.timer_v_upper0.d),
394
395

```

```

396
397 // to internal hardware
398 .qe ( ),
399 .q (reg2hw.timer_v_upper0.q),
400 .ds ( ),
401
402 // to register interface (read)
403 .qs (timer_v_upper0_qs)
404 );
405
406 // R[compare_lower0_0]: V(False)
407 logic compare_lower0_0_qe;
408 logic [0:0] compare_lower0_0_flds_we;
409 prim_flop #(
410     .Width(1),
411     .ResetValue(0)
412 ) u_compare_lower0_00_qe (
413     .clk_i(clk_i),
414     .rst_ni(rst_ni),
415     .d_i(&compare_lower0_0_flds_we),
416     .q_o(compare_lower0_0_qe)
417 );
418
419 prim_subreg #(
420     .DW (32),
421     .SwAccess(prim_subreg_pkg::SwAccessRW),
422     .RESVAL (32'hffffffff),
423     .Mubi (1'b0)
424 ) u_compare_lower0_0 (
425     .clk_i (clk_i),
426     .rst_ni (rst_ni),
427
428     // from register interface
429     .we (compare_lower0_0_we),
430     .wd (compare_lower0_0_wd),
431
432     // from internal hardware
433     .de (1'b0),
434     .d ('0'),
435
436     // to internal hardware
437     .qe (compare_lower0_0_flds_we[0]),
438     .q (reg2hw.compare_lower0_0.q),
439     .ds ( ),
440
441     // to register interface (read)
442     .qs (compare_lower0_0_qs)
443 );
444 assign reg2hw.compare_lower0_0.qe = compare_lower0_0_qe;
445
446 // R[compare_upper0_0]: V(False)
447 logic compare_upper0_0_qe;
448 logic [0:0] compare_upper0_0_flds_we;
449 prim_flop #(
450     .Width(1),
451     .ResetValue(0)
452 ) u_compare_upper0_00_qe (
453     .clk_i(clk_i),
454     .rst_ni(rst_ni),
455     .d_i(&compare_upper0_0_flds_we),
456     .q_o(compare_upper0_0_qe)
457 );
458
459 prim_subreg #(
460     .DW (32),
461     .SwAccess(prim_subreg_pkg::SwAccessRW),
462     .RESVAL (32'hffffffff),
463     .Mubi (1'b0)
464 ) u_compare_upper0_0 (
465     .clk_i (clk_i),
466     .rst_ni (rst_ni),
467
468     // from register interface
469     .we (compare_upper0_0_we),
470     .wd (compare_upper0_0_wd),
471
472     // from internal hardware
473     .de (1'b0),
474     .d ('0'),
475
476     // to internal hardware
477     .qe (compare_upper0_0_flds_we[0]),
478     .q (reg2hw.compare_upper0_0.q),
479     .ds ( ),
480
481     // to register interface (read)
482     .qs (compare_upper0_0_qs)
483 );
484 assign reg2hw.compare_upper0_0.qe = compare_upper0_0_qe;
485
486
487
488 logic [9:0] addr_hit;
489 top_racl_pkg::racl_role_vec_t racl_role_vec;
490 top_racl_pkg::racl_role_t racl_role;
491
492 logic [9:0] racl_addr_hit_read;
493 logic [9:0] racl_addr_hit_write;
494
495 if (EnableRacl) begin : gen_racl_role_logic
496     // Retrieve RACL role from user bits and one-hot encode

```

```

that for the comparison bitmap
497     assign racl_role =
top_racl_pkg::tlul_extract_racl_role_bits(tl_i.a_user.rsvd);
498
499     prim_onehot_enc #(
500         .OneHotWidth( $bits(top_racl_pkg::racl_role_vec_t) )
501     ) u_racl_role_encode (
502         .in_i ( racl_role ),
503         .en_i ( 1'b1 ),
504         .out_o( racl_role_vec )
505     );
506 end else begin : gen_no_racl_role_logic
507     assign racl_role = '0;
508     assign racl_role_vec = '0;
509 end
510
511 always_comb begin
512     racl_addr_hit_read = '0;
513     racl_addr_hit_write = '0;
514     addr_hit[0] = (reg_addr == RV_TIMER_ALERT_TEST_OFFSET);
515     addr_hit[1] = (reg_addr == RV_TIMER_CTRL_OFFSET);
516     addr_hit[2] = (reg_addr == RV_TIMER_INTR_ENABLE0_OFFSET);
517     addr_hit[3] = (reg_addr == RV_TIMER_INTR_STATE0_OFFSET);
518     addr_hit[4] = (reg_addr == RV_TIMER_INTR_TEST0_OFFSET);
519     addr_hit[5] = (reg_addr == RV_TIMER_CFG0_OFFSET);
520     addr_hit[6] = (reg_addr ==
RV_TIMER_TIMER_V_LOWER0_OFFSET);
521     addr_hit[7] = (reg_addr ==
RV_TIMER_TIMER_V_UPPER0_OFFSET);
522     addr_hit[8] = (reg_addr ==
RV_TIMER_COMPARE_LOWER0_0_OFFSET);
523     addr_hit[9] = (reg_addr ==
RV_TIMER_COMPARE_UPPER0_0_OFFSET);
524
525     if (EnableRacl) begin : gen_racl_hit
526         for (int unsigned slice_idx = 0; slice_idx < 10;
slice_idx++) begin
527             racl_addr_hit_read[slice_idx] =
528                 addr_hit[slice_idx] & (|
(racl_policies_i[RaclPolicySelVec[slice_idx]].read_perm
& racl_role_vec));
529             racl_addr_hit_write[slice_idx] =
530                 addr_hit[slice_idx] & (|
(racl_policies_i[RaclPolicySelVec[slice_idx]].write_perm
& racl_role_vec));
531         end
532     end else begin : gen_no_racl
533         racl_addr_hit_read = addr_hit;
534         racl_addr_hit_write = addr_hit;
535     end
536 end
537
538 assign addrmiss = (reg_re || reg_we) ? ~|addr_hit : 1'b0 ;
539 // A valid address hit, access, but failed the RACL check
540 assign racl_error_o.valid = |addr_hit & ((reg_re & ~|
racl_addr_hit_read) |
541                                     (reg_we & ~|
racl_addr_hit_write));
542 assign racl_error_o.request_address =
top_pkg::TL_AW'(reg_addr);
543 assign racl_error_o.racl_role = racl_role;
544 assign racl_error_o.overflow = 1'b0;
545
546 if (EnableRacl) begin : gen_racl_log
547     assign racl_error_o.ctn_uid =
top_racl_pkg::tlul_extract_ctn_uid_bits(tl_i.a_user.rsvd);
548 assign racl_error_o.read_access = tl_i.a_opcode ==
tlul_pkg::Get;
549 end else begin : gen_no_racl_log
550     assign racl_error_o.ctn_uid = '0;
551     assign racl_error_o.read_access = 1'b0;
552 end
553
554 // Check sub-word write is permitted
555 always_comb begin
556     wr_err = (reg_we &
557
558         ((racl_addr_hit_write[0] & (|(RV_TIMER_PERMIT[0] &
~reg_be))) |
559
560         (racl_addr_hit_write[1] & (|(RV_TIMER_PERMIT[1] &
~reg_be))) |
561
562         (racl_addr_hit_write[2] & (|(RV_TIMER_PERMIT[2] &
~reg_be))) |
563
564         (racl_addr_hit_write[3] & (|(RV_TIMER_PERMIT[3] &
~reg_be))) |
565
566         (racl_addr_hit_write[4] & (|(RV_TIMER_PERMIT[4] &
~reg_be))) |
567
568         (racl_addr_hit_write[5] & (|(RV_TIMER_PERMIT[5] &
~reg_be))) |
569
570         (racl_addr_hit_write[6] & (|(RV_TIMER_PERMIT[6] &
~reg_be))) |
571
572         (racl_addr_hit_write[7] & (|(RV_TIMER_PERMIT[7] &
~reg_be))) |

```

```

567         (rac1_addr_hit_write[8] & (!(RV_TIMER_PERMIT[8] &
~reg_be)))) |
568         (rac1_addr_hit_write[9] & (!(RV_TIMER_PERMIT[9] &
~reg_be)))));
569     end
570 // Generate write-enables
571 assign alert_test_we = rac1_addr_hit_write[0] & reg_we & !
reg_error;
572 assign alert_test_wd = reg_wdata[0];
573 assign ctrl_we = rac1_addr_hit_write[1] & reg_we & !reg_er-
ror;
574 assign ctrl_wd = reg_wdata[0];
575 assign intr_enable0_we = rac1_addr_hit_write[2] & reg_we & !
reg_error;
576 assign intr_enable0_wd = reg_wdata[0];
577 assign intr_state0_we = rac1_addr_hit_write[3] & reg_we & !
reg_error;
578 assign intr_state0_wd = reg_wdata[0];
579 assign intr_test0_we = rac1_addr_hit_write[4] & reg_we & !
reg_error;
580 assign intr_test0_wd = reg_wdata[0];
581 assign cfg0_we = rac1_addr_hit_write[5] & reg_we & !reg_er-
ror;
582 assign cfg0_prescale_wd = reg_wdata[11:0];
583 assign cfg0_step_wd = reg_wdata[23:16];
584 assign timer_v_lower0_we = rac1_addr_hit_write[6] & reg_we
& !reg_error;
585 assign timer_v_lower0_wd = reg_wdata[31:0];
586 assign timer_v_upper0_we = rac1_addr_hit_write[7] & reg_we
& !reg_error;
587 assign timer_v_upper0_wd = reg_wdata[31:0];
588
589 assign compare_lower0_0_we = rac1_addr_hit_write[8] & reg_we
& !reg_error;
590 assign compare_lower0_0_wd = reg_wdata[31:0];
591
592 assign compare_upper0_0_we = rac1_addr_hit_write[9] & reg_we
& !reg_error;
593 assign compare_upper0_0_wd = reg_wdata[31:0];
594
595 // Assign write-enables to checker logic vector.
596 always_comb begin
597     reg_we_check[0] = alert_test_we;
598     reg_we_check[1] = ctrl_we;
599     reg_we_check[2] = intr_enable0_we;
600     reg_we_check[3] = intr_state0_we;
601     reg_we_check[4] = intr_test0_we;
602     reg_we_check[5] = cfg0_we;
603     reg_we_check[6] = timer_v_lower0_we;
604     reg_we_check[7] = timer_v_upper0_we;
605     reg_we_check[8] = compare_lower0_0_we;
606     reg_we_check[9] = compare_upper0_0_we;
607 end
608
609 // Read data return
610 always_comb begin
611     reg_rdata_next = '0;
612     unique case (1'b1)
613         rac1_addr_hit_read[0]: begin
614             reg_rdata_next[0] = '0;
615         end
616         rac1_addr_hit_read[1]: begin
617             reg_rdata_next[0] = ctrl_qs;
618         end
619         rac1_addr_hit_read[2]: begin
620             reg_rdata_next[0] = intr_enable0_qs;
621         end
622         rac1_addr_hit_read[3]: begin
623             reg_rdata_next[0] = intr_state0_qs;
624         end
625         rac1_addr_hit_read[4]: begin
626             reg_rdata_next[0] = '0;
627         end
628         rac1_addr_hit_read[5]: begin
629             reg_rdata_next[11:0] = cfg0_prescale_qs;
630             reg_rdata_next[23:16] = cfg0_step_qs;
631         end
632         rac1_addr_hit_read[6]: begin
633             reg_rdata_next[31:0] = timer_v_lower0_qs;
634         end
635         rac1_addr_hit_read[7]: begin

```

```

636         reg_rdata_next[31:0] = timer_v_upper0_qs;
637     end
638     rac1_addr_hit_read[8]: begin
639         reg_rdata_next[31:0] = compare_lower0_0_qs;
640     end
641     rac1_addr_hit_read[9]: begin
642         reg_rdata_next[31:0] = compare_upper0_0_qs;
643     end
644     default: begin
645         reg_rdata_next = '1;
646     end
647 endcase
648 end
649 // shadow busy
650 logic shadow_busy;
651 assign shadow_busy = 1'b0;
652 // register busy
653 assign reg_busy = shadow_busy;
654 // Unused signal tieoff
655 // wdata / byte enable are not always fully used
656 // add a blanket unused statement to handle lint waivers
657 logic unused_wdata;
658 logic unused_be;
659 assign unused_wdata = ^reg_wdata;
660 assign unused_be = ^reg_be;
661 logic unused_policy_sel;
662 assign unused_policy_sel = ^rac1_policies_i;
663 // Assertions for Register Interface
664 `ASSERT_PULSE(wePulse, reg_we, clk_i, !rst_ni)
665 `ASSERT_PULSE(rePulse, reg_re, clk_i, !rst_ni)
666 `ASSERT(reAfterRv, $rose(reg_re || reg_we) |>=
tl_o_pre.d_valid, clk_i, !rst_ni)
667 `ASSERT(en2addrHit, (reg_we || reg_re) |>= $one-
hot0(addr_hit), clk_i, !rst_ni)
668 // this is formulated as an assumption such that the FPV
testbenches do disprove this
669 // property by mistake
670 `ASSUME(reqParity, tl_reg_h2d.a_valid |>=
tl_reg_h2d.a_user.chk_en == tlul_pkg::CheckDis)
671
672 endmodule

```

4. lowRISC_opentitan/hw/ip/rv_timer/rtl/timer_core.sv (48 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5
6 module timer_core #(
7     parameter int N = 1
8 ) (
9     input clk_i,
10    input rst_ni,
11
12    input active,
13    input [11:0] prescaler,
14    input [7:0] step,
15
16    output logic tick,
17    output logic [63:0] mtime_d,
18    input [63:0] mtime,
19    input [63:0] mtimecmp [N],
20
21    output logic [N-1:0] intr
22 );
23
24 logic [11:0] tick_count;
25
26 always_ff @(posedge clk_i or negedge rst_ni) begin : gener-
ate_tick
27     if (!rst_ni) begin
28         tick_count <= 12'h0;
29     end else if (!active) begin
30         tick_count <= 12'h0;
31     end else if (tick_count == prescaler) begin
32         tick_count <= 12'h0;
33     end else begin
34         tick_count <= tick_count + 1'b1;
35     end
36 end
37
38 assign tick = active & (tick_count >= prescaler);
39
40 assign mtime_d = mtime + 64'(step);
41
42

```

```
43 // interrupt is generated if mtime is greater than or equal  
to mtimecmp  
44 for (genvar t = 0 ; t < N ; t++) begin : gen_intr  
45     assign intr[t] = active & (mtime >= mtimecmp[t]);  
46 end  
47  
48 endmodule : timer_core
```

RISC-V 디버그 모듈

What this is. A RISC-V debug module with JTAG transport and core control.

Why it exists. Because it is the only window into a chip that will not boot. It must therefore operate while the core is halted, which usually places it in a separate clock and reset domain.

Undergraduate review · EECS concepts. JTAG TAP state machine (IEEE 1149.1); boundary scan; debug transport; clock domain crossing.

Graduate · Practice note. Debug access crosses the security boundary. Production devices lock it according to life-cycle state. **The tension between debuggability and security is a design issue in every secure chip**, and the resolution must be specified, not improvised.

7 files • 2,548 lines

API — jtag_pkg.sv

Table 1036. lowRISC_opentitan/hw/ip/rv_dm/rtl/jtag_pkg.sv — module / interface / package (1개)

이름	종류
jtag_pkg	package

Table 1037. lowRISC_opentitan/hw/ip/rv_dm/rtl/jtag_pkg.sv — parameter (2개)

파라미터	형/폭	기본값
JTAG_REQ_DEFAULT	jtag_req_t	'0
JTAG_RSP_DEFAULT	jtag_rsp_t	'0

API — rv_dm.sv

Table 1038. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm.sv — module / interface / package (1개)

이름	종류
rv_dm	module

Table 1039. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm.sv — parameter (8개)

파라미터	형/폭	기본값
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
IdcodeValue	logic [31:0]	32'h 0000_0001
UseDmiInterface	bit	1'b0
SecVolatileRawUnlockEn	bit	0
TlulHostUserRsvdBits	logic [tlul_pkg::RsvdWidth-1:0]	0
EnableRac1	bit	1'b0

파라미터	형/폭	기본값
RacLErrorRsp	bit	EnableRacI

Table 1040. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm.sv — localparam (6개)

상수	형/폭	값
DebugHartInfo	dm::hartinfo_t	{
BusWidth	int	32
SelectableHarts	logic [NrHarts-1:0]	{NrHarts{1'b1}}
LcEnDebugReqVal	int	4 - 1
LcEnResetReqVal	int	LcEnDebugReqVal + NrHarts
RvDmLcEnSize	int	\$clog2(LcEnResetReqVal + 2)

Table 1041. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm.sv — 포트 (35개)

포트	방향	형/폭
clk_i	input	logic
clk_lc_i	input	logic
rst_ni	input	logic
rst_lc_ni	input	logic
next_dm_addr_i	input	logic [31:0]
lc_hw_debug_clr_i	input	lc_ctrl_pkg::lc_tx_t
lc_hw_debug_en_i	input	lc_ctrl_pkg::lc_tx_t
lc_dft_en_i	input	lc_ctrl_pkg::lc_tx_t
pinmux_hw_debug_en_i	input	lc_ctrl_pkg::lc_tx_t
lc_check_byp_en_i	input	lc_ctrl_pkg::lc_tx_t
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
lc_init_done_i	input	lc_ctrl_pkg::lc_tx_t
strap_en_i	input	
strap_en_override_i	input	
otp_dis_rv_dm_late_debug_i	input	prim_mubi_pkg::mubi8_t
scanmode_i	input	prim_mubi_pkg::mubi4_t
scan_rst_ni	input	
ndmreset_req_o	output	logic
dmactive_o	output	logic
debug_req_o	output	logic [NrHarts-1:0]
unavailable_i	input	logic [NrHarts-1:0]
regs_tl_d_i	input	tlul_pkg::tl_h2d_t
regs_tl_d_o	output	tlul_pkg::tl_d2h_t

포트	방향	형/폭
mem_tl_d_i	input	tlul_pkg::tl_h2d_t
mem_tl_d_o	output	tlul_pkg::tl_d2h_t
sba_tl_h_o	output	tlul_pkg::tl_h2d_t
sba_tl_h_i	input	tlul_pkg::tl_d2h_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
jtag_i	input	jtag_pkg::jtag_req_t
jtag_o	output	jtag_pkg::jtag_rsp_t
dbg_tl_d_i	input	tlul_pkg::tl_h2d_t
dbg_tl_d_o	output	tlul_pkg::tl_d2h_t

API — rv_dm_dbg_reg_top.sv

Table 1042. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_dbg_reg_top.sv — module / interface / package (1개)

이름	종류
rv_dm_dbg_reg_top	module

Table 1043. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_dbg_reg_top.sv — 포트 (7개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
tl_win_o	output	tlul_pkg::tl_h2d_t
tl_win_i	input	tlul_pkg::tl_d2h_t
intg_err_o	output	logic

API — rv_dm_dmi_gate.sv

Table 1044. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_dmi_gate.sv — module / interface / package (1개)

이름	종류
rv_dm_dmi_gate	module

Table 1045. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_dmi_gate.sv — parameter (1개)

파라미터	형/폭	기본값
SecVolatileRawUnlockEn	bit	0

Table 1046. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_dmi_gate.sv — 포트 (17개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
strap_en_override_i	input	
strap_en_i	input	
lc_hw_debug_clr_i	input	lc_ctrl_pkg::lc_tx_t
lc_hw_debug_en_i	input	lc_ctrl_pkg::lc_tx_t
lc_check_byp_en_i	input	lc_ctrl_pkg::lc_tx_t
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
dbg_tl_h2d_win_i	input	tlul_pkg::tl_h2d_t
dbg_tl_d2h_win_o	output	tlul_pkg::tl_d2h_t
dmi_req_valid_o	output	
dmi_req_ready_i	input	
dmi_req_o	output	dm::dmi_req_t
dmi_rsp_valid_i	input	
dmi_rsp_ready_o	output	
dmi_rsp_i	input	dm::dmi_resp_t
intg_error_o	output	

API — rv_dm_pkg.sv**Table 1047. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_pkg.sv — module / interface / package (1개)**

이름	종류
rv_dm_pkg	package

Table 1048. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_pkg.sv — parameter (1개)

파라미터	형/폭	기본값
NEXT_DM_ADDR_DEFAULT	next_dm_addr_t	'0

Table 1049. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_pkg.sv — typedef / using (1개)

새 이름	원래 형
next_dm_addr_t	logic [31:0]

API — rv_dm_reg_pkg.sv**Table 1050. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_reg_pkg.sv — module / interface / package (1개)**

이름	종류
rv_dm_reg_pkg	package

Table 1051. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_reg_pkg.sv — parameter (300개) — 앞 60개만 표시(전체 300개, 본문 소스 참조)

파라미터	형/폭	기본값
NrHarts	int	1
NumAlerts	int	1
RegsAw	int	4
MemAw	int	12
DbgAw	int	9
NumRegsRegs	int	3
NumRegsMem	int	281
NumRegsDbg	int	0
RV_DM_ALERT_TEST_OFFSET	logic [RegsAw-1:0]	4'h 0
RV_DM_LATE_DEBUG_ENABLE_REGWEN_OFFSET	logic [RegsAw-1:0]	4'h 4
RV_DM_LATE_DEBUG_ENABLE_OFFSET	logic [RegsAw-1:0]	4'h 8
RV_DM_ALERT_TEST_RESVAL	logic [0:0]	1'h 0
RV_DM_ALERT_TEST_FATAL_FAULT_RESVAL	logic [0:0]	1'h 0
RV_DM_HALTED_OFFSET	logic [MemAw-1:0]	12'h 100
RV_DM_GOING_OFFSET	logic [MemAw-1:0]	12'h 108
RV_DM_RESUMING_OFFSET	logic [MemAw-1:0]	12'h 110
RV_DM_EXCEPTION_OFFSET	logic [MemAw-1:0]	12'h 118
RV_DM_WHERETO_OFFSET	logic [MemAw-1:0]	12'h 300
RV_DM_ABSTRACTCMD_0_OFFSET	logic [MemAw-1:0]	12'h 338
RV_DM_ABSTRACTCMD_1_OFFSET	logic [MemAw-1:0]	12'h 33c
RV_DM_ABSTRACTCMD_2_OFFSET	logic [MemAw-1:0]	12'h 340
RV_DM_ABSTRACTCMD_3_OFFSET	logic [MemAw-1:0]	12'h 344
RV_DM_ABSTRACTCMD_4_OFFSET	logic [MemAw-1:0]	12'h 348
RV_DM_ABSTRACTCMD_5_OFFSET	logic [MemAw-1:0]	12'h 34c
RV_DM_ABSTRACTCMD_6_OFFSET	logic [MemAw-1:0]	12'h 350
RV_DM_ABSTRACTCMD_7_OFFSET	logic [MemAw-1:0]	12'h 354
RV_DM_ABSTRACTCMD_8_OFFSET	logic [MemAw-1:0]	12'h 358
RV_DM_ABSTRACTCMD_9_OFFSET	logic [MemAw-1:0]	12'h 35c
RV_DM_PROGRAM_BUFFER_0_OFFSET	logic [MemAw-1:0]	12'h 360
RV_DM_PROGRAM_BUFFER_1_OFFSET	logic [MemAw-1:0]	12'h 364
RV_DM_PROGRAM_BUFFER_2_OFFSET	logic [MemAw-1:0]	12'h 368
RV_DM_PROGRAM_BUFFER_3_OFFSET	logic [MemAw-1:0]	12'h 36c
RV_DM_PROGRAM_BUFFER_4_OFFSET	logic [MemAw-1:0]	12'h 370

파라미터	형/폭	기본값
RV_DM_PROGRAM_BUFFER_5_OFFSET	logic [MemAw-1:0]	12'h 374
RV_DM_PROGRAM_BUFFER_6_OFFSET	logic [MemAw-1:0]	12'h 378
RV_DM_PROGRAM_BUFFER_7_OFFSET	logic [MemAw-1:0]	12'h 37c
RV_DM_DATAADDR_0_OFFSET	logic [MemAw-1:0]	12'h 380
RV_DM_DATAADDR_1_OFFSET	logic [MemAw-1:0]	12'h 384
RV_DM_FLAGS_0_OFFSET	logic [MemAw-1:0]	12'h 400
RV_DM_FLAGS_1_OFFSET	logic [MemAw-1:0]	12'h 404
RV_DM_FLAGS_2_OFFSET	logic [MemAw-1:0]	12'h 408
RV_DM_FLAGS_3_OFFSET	logic [MemAw-1:0]	12'h 40c
RV_DM_FLAGS_4_OFFSET	logic [MemAw-1:0]	12'h 410
RV_DM_FLAGS_5_OFFSET	logic [MemAw-1:0]	12'h 414
RV_DM_FLAGS_6_OFFSET	logic [MemAw-1:0]	12'h 418
RV_DM_FLAGS_7_OFFSET	logic [MemAw-1:0]	12'h 41c
RV_DM_FLAGS_8_OFFSET	logic [MemAw-1:0]	12'h 420
RV_DM_FLAGS_9_OFFSET	logic [MemAw-1:0]	12'h 424
RV_DM_FLAGS_10_OFFSET	logic [MemAw-1:0]	12'h 428
RV_DM_FLAGS_11_OFFSET	logic [MemAw-1:0]	12'h 42c
RV_DM_FLAGS_12_OFFSET	logic [MemAw-1:0]	12'h 430
RV_DM_FLAGS_13_OFFSET	logic [MemAw-1:0]	12'h 434
RV_DM_FLAGS_14_OFFSET	logic [MemAw-1:0]	12'h 438
RV_DM_FLAGS_15_OFFSET	logic [MemAw-1:0]	12'h 43c
RV_DM_FLAGS_16_OFFSET	logic [MemAw-1:0]	12'h 440
RV_DM_FLAGS_17_OFFSET	logic [MemAw-1:0]	12'h 444
RV_DM_FLAGS_18_OFFSET	logic [MemAw-1:0]	12'h 448
RV_DM_FLAGS_19_OFFSET	logic [MemAw-1:0]	12'h 44c
RV_DM_FLAGS_20_OFFSET	logic [MemAw-1:0]	12'h 450
RV_DM_FLAGS_21_OFFSET	logic [MemAw-1:0]	12'h 454

API — rv_dm_regs_reg_top.sv

Table 1052. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_regs_reg_top.sv — module / interface / package (1개)

이름	종류
rv_dm_regs_reg_top	module

Table 1053. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_regs_reg_top.sv — parameter (2개)

파라미터	형/폭	기본값
EnableRac1	bit	1'b0

파라미터	형/폭	기본값
RacLErrorRsp	bit	1'b1

Table 1054. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_regs_reg_top.sv — localparam (3개)

상수	형/폭	값
AW	int	4
DW	int	32
DBW	int	DW/8

Table 1055. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_regs_reg_top.sv — 포트 (8개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
reg2hw	output	rv_dm_reg_pkg::rv_dm_regs_reg2hw_t
racL_policies_i	input	top_racL_pkg::racL_policy_vec_t
racL_error_o	output	top_racL_pkg::racL_error_log_t
intg_err_o	output	logic

1. lowRISC_opentitan/hw/ip/rv_dm/rtl/jtag_pkg.sv (24 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 //
6 package jtag_pkg;
7
8   typedef struct packed {
9     logic tck;
10    logic tms;
11    logic trst_n;
12    logic tdi;
13  } jtag_req_t;
14
15  parameter jtag_req_t JTAG_REQ_DEFAULT = '0;
16
17  typedef struct packed {
18    logic tdo;
19    logic tdo_oe;
20  } jtag_rsp_t;
21
22  parameter jtag_rsp_t JTAG_RSP_DEFAULT = '0;
23
24 endpackage : jtag_pkg

```

2. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm.sv (783 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Top-level debug module (DM)
6 //
7 // This module implements the RISC-V debug specification ver-
  sion 0.13,
8 //
9 // This toplevel wraps the PULP debug module available from
10 // https://github.com/pulp-platform/riscv-dbg to match the
  needs of
11 // the TL-UL-based lowRISC chip design.
12
13 `include "prim_assert.sv"

```

```

14
15 module rv_dm
16   import rv_dm_reg_pkg::*;
17   #(
18     parameter logic [NumAlerts-1:0] AlertAsyn-
19       cOn = {NumAlerts{1'b1}},
20     // Number of cycles a differential skew is tolerated on the
21     alert signal
22     parameter int unsigned AlertSkew-
23       Cycles = 1,
24     parameter logic [31:0] Idcode-
25       Value = 32'h 0000_0001,
26     parameter bit UseDmiInter-
27       face = 1'b0,
28     parameter bit SecVolatileRawUn-
29       lockEn = 0,
30     parameter logic [tlul_pkg::RsvdWidth-1:0] TlulHostUserRsvd-
31       Bits = 0,
32     parameter bit EnableR-
33       acl = 1'b0,
34     parameter bit RacLer-
35       rorRsp = EnableRacL,
36     parameter top_racL_pkg::racL_policy_sel_t RacLPoli-
37       cySelVecRegs[NumRegsRegs] = '{NumRegsRegs{0}}
38   ) (
39     input logic clk_i, // clock
40     input logic clk_lc_i, // only declared
41     here so that the topgen
42
43     // tooling connects
44     the correct clk/rst domains.
45     input logic rst_ni, // asynchronous re-
46     set active low, connect PoR
47     // here, not the
48     system reset
49     input logic rst_lc_ni, // asynchronous re-
50     set active low, connect the lc
51
52     // reset here. this is
53     only used for NDM reset tracking.
54     input logic [31:0] next_dm_addr_i, // static word
55     address of the next debug module.
56     // SEC_CM: LC_HW_DEBUG_CLR.INTERSIG.MUBI
57     input lc_ctrl_pkg::lc_tx_t lc_hw_debug_clr_i,
58     // SEC_CM: LC_HW_DEBUG_EN.INTERSIG.MUBI
59     // HW Debug lifecycle enable signal (live version from the
60     life cycle controller)
61     input lc_ctrl_pkg::lc_tx_t lc_hw_debug_en_i,
62     // SEC_CM: LC_DFT_EN.INTERSIG.MUBI
63     // HW DFT lifecycle enable signal (live version from the

```

```

life cycle controller)
44 input lc_ctrl_pkg::lc_tx_t lc_dft_en_i,
45 // HW Debug lifecycle enable signal (latched version from
pinmux,
only used for JTAG/TAP gating)
46 input lc_ctrl_pkg::lc_tx_t pinmux_hw_debug_en_i,
47 input lc_ctrl_pkg::lc_tx_t lc_check_byp_en_i,
48 input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
49 // Lifecycle init done indicator used to clear NDM latch
after LC re-initialization
50 input lc_ctrl_pkg::lc_tx_t lc_init_done_i,
51 input strap_en_i,
52 input strap_en_override_i,
53 // SEC_CM: OTP_DIS_RV_DM_LATE_DEBUG.INTERSIG.MUBI
54 // Late debug enable disablement signal coming from the OTP
HW_CFG1
partition.
55 input prim_mubi_pkg::mubi8_t otp_dis_rv_dm_late_debug_i,
56 input prim_mubi_pkg::mubi4_t scanmode_i,
57 input scan_rst_ni,
58 output logic ndmreset_req_o, // non-debug
module reset
59 output logic dmactive_o, // debug module is
active
60 output logic [NrHarts-1:0] debug_req_o, // async debug re-
quest
61 input logic [NrHarts-1:0] unavailable_i, // communicate
whether
the hart is unavailable
62 // (e.g.: power
down)
63
64 // bus device for composable CSR access
65 input tlul_pkg::tl_h2d_t regs_tl_d_i,
66 output tlul_pkg::tl_d2h_t regs_tl_d_o,
67
68 // bus device with debug memory, for an execution based
technique
69 input tlul_pkg::tl_h2d_t mem_tl_d_i,
70 output tlul_pkg::tl_d2h_t mem_tl_d_o,
71
72 // bus host, for system bus accesses
73 output tlul_pkg::tl_h2d_t sba_tl_h_o,
74 input tlul_pkg::tl_d2h_t sba_tl_h_i,
75
76 // Alerts
77 input prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_rx_i,
78 output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
alert_tx_o,
79
80 // RACL interface
81 input top_racl_pkg::racl_policy_vec_t racl_policies_i,
82 output top_racl_pkg::racl_error_log_t racl_error_o,
83
84 // JTAG TAP
85 input jtag_pkg::jtag_req_t jtag_i,
86 output jtag_pkg::jtag_rsp_t jtag_o,
87 // TL-UL-based DMI
88 input tlul_pkg::tl_h2d_t dbg_tl_d_i,
89 output tlul_pkg::tl_d2h_t dbg_tl_d_o
90 );
91
92 ///////////////////////////////////////////////////
93 // Parameter Definitions //
94 ///////////////////////////////////////////////////
95
96 import prim_mubi_pkg::mubi4_test_true_strict;
97 import prim_mubi_pkg::mubi8_test_true_strict;
98 import prim_mubi_pkg::mubi32_test_true_strict;
99 import lc_ctrl_pkg::lc_tx_test_true_strict;
100
101 `ASSERT_INIT(paramCheckNrHarts, NrHarts > 0)
102
103 // static debug hartinfo
104 localparam dm::hartinfo_t DebugHartInfo = '{
105   zero1: 0,
106   nscratch: 2, // Debug module needs at least two scratch
regs
107   zero0: 0,
108   dataaccess: 1'b1, // data registers are memory mapped in
the debugger
109   datasize: dm::DataCount,
110   dataaddr: dm::DataAddr
111 };
112
113 dm::hartinfo_t [NrHarts-1:0] hartinfo;
114 for (genvar i = 0; i < NrHarts; i++) begin :
gen_dm_hart_ctrl
115   assign hartinfo[i] = DebugHartInfo;
116 end
117
118 // Currently only 32 bit busses are supported by our TL-UL
IP
119 localparam int BusWidth = 32;
120 // all harts have contiguous IDs
121 localparam logic [NrHarts-1:0] SelectableHarts =
{NrHarts{1'b1}};
122
123 ///////////////////////////////////////////////////
124 // CSR Nodes //
125 ///////////////////////////////////////////////////
126
127 tlul_pkg::tl_h2d_t mem_tl_win_h2d;

```

```

128 tlul_pkg::tl_d2h_t mem_tl_win_d2h;
129 rv_dm_reg_pkg::rv_dm_regs_reg2hw_t regs_reg2hw;
130 logic regs_intg_error, rom_intg_error, dmi_intg_error,
dbg_intg_error;
131 logic sba_gate_intg_error, rom_gate_intg_error;
132
133 rv_dm_regs_reg_top #(
134   .EnableRacl(EnableRacl),
135   .RaclErrorRsp(RaclErrorRsp),
136   .RaclPolicySelVec(RaclPolicySelVecRegs)
137 ) u_reg_regs (
138   .clk_i,
139   .rst_ni,
140   .tl_i (regs_tl_d_i ),
141   .tl_o (regs_tl_d_o ),
142   .reg2hw (regs_reg2hw ),
143   .racl_policies_i,
144   .racl_error_o,
145   // SEC_CM: BUS.INTEGRITY
146   .intg_err_o(regs_intg_error)
147 );
148
149 // We are not instantiating the generated rv_dm_mem_reg_top,
since the registers are actually
150 // implemented inside the vendored-in rv_dm module from the
PULP project.
151 assign mem_tl_win_h2d = mem_tl_d_i;
152 assign mem_tl_d_o = mem_tl_win_d2h;
153
154 // Alerts
155 logic [NumAlerts-1:0] alert_test, alerts;
156
157 assign alerts[0] = regs_intg_error | rom_intg_error |
dmi_intg_error | dbg_intg_error |
158   sba_gate_intg_error | rom_g-
ate_intg_error;
159
160 assign alert_test = {
161   regs_reg2hw.alert_test.q &
162   regs_reg2hw.alert_test.qe
163 };
164
165 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
166   prim_alert_sender #(
167     .AsyncOn(AlertAsyncOn[i]),
168     .SkewCycles(AlertSkewCycles),
169     .IsFatal(1'b1)
170   ) u_prim_alert_sender (
171     .clk_i,
172     .rst_ni,
173     .alert_test_i ( alert_test[i] ),
174     .alert_req_i ( alerts[i] ),
175     .alert_ack_o ( ),
176     .alert_state_o ( ),
177     .alert_rx_i ( alert_rx_i[i] ),
178     .alert_tx_o ( alert_tx_o[i] )
179   );
180 end
181
182 // Decode multibit scanmode enable
183 logic testmode;
184 assign testmode = mubi4_test_true_strict(scanmode_i);
185
186 ///////////////////////////////////////////////////
187 // Life Cycle Gating //
188 ///////////////////////////////////////////////////
189
190 // Debug enable gating.
191 localparam int LcEnDebugReqVal = 4 - 1;
192 localparam int LcEnResetReqVal = LcEnDebugReqVal + NrHarts;
193 // +1 to get number of bits and another +1 because LcEnLast-
Pos is one more than LcEnResetReq.
194 localparam int RvDmLcEnSize = $clog2(LcEnResetReqVal +
2);
195 typedef enum logic [RvDmLcEnSize-1:0] {
196   LcEnFetch,
197   LcEnRom,
198   LcEnSba,
199   // LcEnDebugReq[NrHarts], <= this unfortunately does not
work - SV-LRM mandates the use of
200   // integral numbers. Parameters are not allowed in this
context.
201   LcEnDebugReq,
202   // The above literal accommodates NrHarts number of debug
requests - so we number the next
203   // literal accordingly.
204   LcEnResetReq = RvDmLcEnSize'(LcEnResetReqVal),
205   // LcEnLastPos must immediately follow LcEnResetReq to
calculate RvDmLcEnSize.
206   LcEnLastPos
207 } rv_dm_lc_en_e;
208 // These must be equal so that the difference between
LcEnResetReq and LcEnDebugReq is NrHarts.
209 `ASSERT(RvDmLcEnDebugVal_A, int'(LcEnDebugReq) == LcEnDebu-
gReqVal)
210
211 lc_ctrl_pkg::lc_tx_t lc_hw_debug_en;
212 prim_lc_sync #(
213   .NumCopies(1)
214 ) u_prim_lc_sync_lc_hw_debug_en (

```

```

215     .clk_i,
216     .rst_ni,
217     .lc_en_i(lc_hw_debug_en_i),
218     .lc_en_o({lc_hw_debug_en}))
219 );
220
221 lc_ctrl_pkg::lc_tx_t lc_dft_en;
222 prim_lc_sync #(
223     .NumCopies(1)
224 ) u_prim_lc_sync_lc_dft_en (
225     .clk_i,
226     .rst_ni,
227     .lc_en_i(lc_dft_en_i),
228     .lc_en_o({lc_dft_en}))
229 );
230
231 prim_mubi_pkg::mubi8_t [lc_ctrl_pkg::TxWidth-1:0]
otp_dis_rv_dm_late_debug;
232 prim_mubi8_sync #(
233     .NumCopies (lc_ctrl_pkg::TxWidth)
234 ) u_prim_mubi8_sync_otp_dis_rv_dm_late_debug (
235     .clk_i,
236     .rst_ni,
237     .mubi_i(otp_dis_rv_dm_late_debug_i),
238     .mubi_o(otp_dis_rv_dm_late_debug)
239 );
240
241 prim_mubi_pkg::mubi32_t [lc_ctrl_pkg::TxWidth-1:0] late_de-
bug_enable;
242 prim_mubi32_sync #(
243     .NumCopies (lc_ctrl_pkg::TxWidth),
244     .AsyncOn(0) // No synchronization required since the input
signal is already synchronous.
245 ) u_prim_mubi32_sync_late_debug_enable (
246     .clk_i,
247     .rst_ni,
248     .mubi_i(prim_mubi_pkg::mubi32_t'(regs_reg2hw.late_debug_e-
nable)),
249     .mubi_o(late_debug_enable)
250 );
251
252 // SEC_CM: DM_EN.CTRL.LC_GATED

253 // This implements a hardened MuBi multiplexor circuit where
each output bitlane has its own
254 // associated comparators for the enablement condition.
255 logic [lc_ctrl_pkg::TxWidth-1:0] lc_hw_debug_en_raw;
256 logic [lc_ctrl_pkg::TxWidth-1:0] lc_dft_en_raw;
257 logic [lc_ctrl_pkg::TxWidth-1:0] lc_hw_debug_en_gated_raw;
258 assign lc_hw_debug_en_raw = lc_hw_debug_en;
259 assign lc_dft_en_raw = lc_dft_en;
260 for (genvar k = 0; k < lc_ctrl_pkg::TxWidth; k++) begin :
gen_mubi_mux
261     assign lc_hw_debug_en_gated_raw[k] = (mu-
bi8_test_true_strict(otp_dis_rv_dm_late_debug[k]) ||
262         mu-
bi32_test_true_strict(late_debug_enable[k])) ?
263         lc_hw_de-
bug_en_raw[k] :
264         lc_dft_en_raw[k];
265     end
266

267 // The lc_hw_debug_en_gated signal modulates gating logic on
the bus-side of the RV_DM.

268 // The pinmux_hw_debug_en signal on the other hand modulates
the TAP side of the RV_DM.
269 // In order for the RV_DM to remain response during a NDM
reset request, the TAP side
270 // is not further modulated with the LATE_DEBUG_ENABLE CSR.
271 lc_ctrl_pkg::lc_tx_t [LcEnLastPos-1:0] lc_hw_debug_en_gated;
272 prim_lc_sync #(
273     .NumCopies(int'(LcEnLastPos)),
274     .AsyncOn(0) // No synchronization required since the input
signal is already synchronous.
275 ) u_lc_en_sync_copies (
276     .clk_i,
277     .rst_ni,
278     .lc_en_i(lc_ctrl_pkg::lc_tx_t'(lc_hw_debug_en_gated_raw)),
279     .lc_en_o(lc_hw_debug_en_gated)
280 );
281
282 // Keep debug gating asserted during an active NDM reset by
latching the
283 // current LC debug enable and ORing it into the gating
while the reset is
284 // pending. Clear the latch once LC initialization has com-
pleted again.
285 lc_ctrl_pkg::lc_tx_t lc_hw_debug_en_latched_q;
286 lc_ctrl_pkg::lc_tx_t lc_init_done_sync;
287 prim_lc_sync #(
288     .NumCopies(1)
289 ) u_lc_init_done_sync (
290     .clk_i,
291     .rst_ni,
292     .lc_en_i(lc_init_done_i),
293     .lc_en_o({lc_init_done_sync})
294 );

```

```

295
296 // Version of lc_hw_debug_en_gated with NDM latch applied
(defined below)
297 lc_ctrl_pkg::lc_tx_t [LcEnLastPos-1:0] lc_hw_de-
bug_en_gated_ndm;
298
299 dm::dmi_req_t dmi_req;
300 dm::dmi_resp_t dmi_rsp;
301 logic dmi_req_valid, dmi_req_ready;
302 logic dmi_rsp_valid, dmi_rsp_ready;
303 logic dmi_rst_n;
304
305 //////////////////////////////////////////////////
306 // NDM Reset Tracking //
307 //////////////////////////////////////////////////
308
309 logic reset_req_en;
310 logic ndmreset_req, ndmreset_ack;
311 logic ndmreset_req_qual;
312 // SEC_CM: DM_EN.CTRL.LC_GATED
313 assign reset_req_en =
lc_tx_test_true_strict(lc_hw_debug_en_gated_ndm[LcEnResetReq]);
314 assign ndmreset_req_o = ndmreset_req_qual & reset_req_en;
315
316 // Sample the processor reset to detect lc reset assertion.
317 logic lc_rst_asserted_async;
318 prim_flop_2sync #(
319     .Width(1),
320     .ResetValue(1) // Resets to 1 to indicate assertion.
321 ) u_prim_flop_2sync_lc_rst_assert (
322     .clk_i, // Use RV_DM clock
323     .rst_ni(rst_lc_ni), // Use LC reset here that resets the
entire system except the RV_DM.
324     .d_i(1'b0), // Set to 0 to indicate deassertion.
325     .q_o(lc_rst_asserted_async)
326 );
327

328 // Note that the output of the above flops can be metastable
at reset assertion, since the reset
329 // signal is coming from a different clock domain and has
not been synchronized with clk_i.
330 logic lc_rst_asserted;
331 prim_flop_2sync #(
332     .Width(1)
333 ) u_prim_flop_2sync_lc_rst_sync (
334     .clk_i,
335     .rst_ni,
336     .d_i(lc_rst_asserted_async),
337     .q_o(lc_rst_asserted)
338 );
339
340 // The acknowledgement pulse sets the dm-
status.allhavereset / dmstatus.anyhavereset registers in
341 // RV_DM. It should only be asserted once an NDM reset re-
quest has been fully completed.
342 logic ndmreset_pending_q;
343 logic lc_rst_pending_q;
344 always_ff @(posedge clk_i or negedge rst_ni) begin :
p_ndm_reset
345     if (!rst_ni) begin
346         ndmreset_pending_q <= 1'b0;
347         lc_rst_pending_q <= 1'b0;
348         lc_hw_debug_en_latched_q <= lc_ctrl_pkg::Off;
349     end else begin
350         // Only set this if there was no previous pending NDM
request.
351         if (ndmreset_req && !ndmreset_pending_q) begin
352             ndmreset_pending_q <= 1'b1;
353
354 // Latch current LC debug enable value at the start of
NDM reset
355         lc_hw_debug_en_latched_q <= lc_hw_debug_en;
356         end else if (ndmreset_ack && ndmreset_pending_q) begin
357             ndmreset_pending_q <= 1'b0;
358         end
359         // We only track lc resets that are asserted during an
active ndm reset request..
360         if (ndmreset_pending_q && lc_rst_asserted) begin
361             lc_rst_pending_q <= 1'b1;
362         end else if (ndmreset_ack && lc_rst_pending_q) begin
363             lc_rst_pending_q <= 1'b0;
364         end
365         // Clear the latch after lifecycle initialization com-
pletes
366         if (lc_c-
trl_pkg::lc_tx_test_true_strict(lc_init_done_sync)) begin
367             lc_hw_debug_en_latched_q <= lc_ctrl_pkg::Off;
368         end
369     end
370
371 // In order to ACK the following conditions must be met
372 // 1) an NDM reset request was asserted and is pending
373 // 2) a lc reset was asserted after the NDM reset request
374 // 3) the NDM reset request was deasserted
375 // 4) the NDM lc request was deasserted
376 // 5) the debug module has been ungated for operation (de-
pending on LC state, OTP config and CSR)
377 assign ndmreset_ack = ndmreset_pending_q &&
lc_rst_pending_q &&

```

```

379             !ndmreset_req &&
380             !lc_rst_asserted &&
381             reset_req_en;
382
383 // Apply the latched LC value only while an NDM reset is
pending.
384 for (genvar k = 0; k < int'(LcEnLastPos); k++) begin :
gen_ndm_gated
385     assign lc_hw_debug_en_gated_ndm[k] = (ndmre-
set_pending_q) ?
386         lc_ctrl_pkg::lc_tx_or_hi(lc_hw_debug_en_gated[k],
lc_hw_debug_en_latched_q) :
387         lc_hw_debug_en_gated[k];
388 end
389
390 //////////////////////////////////////////////////
391 // System Bus Access Port (TL-UL Host) //
392 //////////////////////////////////////////////////
393
394 logic                host_req;
395 logic [BusWidth-1:0] host_add;
396 logic                host_we;
397 logic [BusWidth-1:0] host_wdata;
398 logic [BusWidth/8-1:0] host_be;
399 logic                host_gnt;
400 logic                host_r_valid;
401 logic [BusWidth-1:0] host_r_rdata;
402 logic                host_r_err;
403 logic                host_r_other_err;
404
405 // SEC_CM: DM_EN.CTRL.LC_GATED
406 // SEC_CM: SBA_TL_LC_GATE.FSM.SPARSE
407 tlul_pkg::tl_h2d_t sba_tl_h_o_int;
408 tlul_pkg::tl_d2h_t sba_tl_h_i_int;
409 tlul_lc_gate #(
410     .NumGatesPerDirection(2)
411 ) u_tlul_lc_gate_sba (
412     .clk_i,
413     .rst_ni,
414     .tl_h2d_i(sba_tl_h_o_int),
415     .tl_d2h_o(sba_tl_h_i_int),
416     .tl_h2d_o(sba_tl_h_o),
417     .tl_d2h_i(sba_tl_h_i),
418     .lc_en_i (lc_hw_debug_en_gated_ndm[LcEnSba]),
419     .err_o   (sba_gate_intg_error),
420     .flush_req_i('0),
421     .flush_ack_o(),
422     .resp_pending_o()
423 );
424
425 tlul_adapter_host #(
426     .MAX_REQS(1),
427     .EnableDataIntgGen(1),
428     .EnableRspDataIntgCheck(1)
429 ) tl_adapter_host_sba (
430     .clk_i,
431     .rst_ni,
432     .req_i   (host_req),
433     .instr_type_i (prim_mubi_pkg::MuBi4False),
434     .gnt_o   (host_gnt),
435     .addr_i   (host_add),
436     .we_i     (host_we),
437     .wdata_i   (host_wdata),
438     .wdata_intg_i ('0),
439     .be_i     (host_be),
440     .user_rsvd_i (TlulHostUserRsvdBits),
441     .valid_o   (host_r_valid),
442     .rdata_o   (host_r_rdata),
443     .rdata_intg_o (),
444     .err_o     (host_r_err),
445     // Note: This bus integrity error is not connected to the
alert due to a few reasons:
446     // 1) the SBA module is not active in production life
cycle states.
447     // 2) there is value in being able to accept incoming
transactions with integrity
448 // errors during test / debug life cycle states so that
the system can be debugged
449 // without triggering alerts.
450 // 3) the error condition is hooked up to an error CSR
that can be read out by the debugger
451 // via JTAG or DMI so that bus integrity errors can be
told apart from regular bus errors.
452     .intg_err_o (host_r_other_err),
453     .tl_o       (sba_tl_h_o_int),
454     .tl_i       (sba_tl_h_i_int)
455 );
456
457 //////////////////////////////////////////////////
458 // Debug Memory Port (TL-UL Device) //
459 //////////////////////////////////////////////////
460
461 logic                device_req;
462 logic                device_we;
463 logic                device_re;
464 logic [BusWidth/8-1:0] device_be;
465 logic [BusWidth-1:0] device_wdata;
466 logic [BusWidth-1:0] device_rdata;
467 logic                device_err;
468

```

```

469 logic [BusWidth-1:0] device_addr_aligned;
470 logic [MemAw-1:0] device_addr;
471
472 assign device_addr_aligned = BusWidth'(device_addr);
473
474 logic [NrHarts-1:0] debug_req_en;
475 logic [NrHarts-1:0] debug_req;
476 for (genvar i = 0; i < NrHarts; i++) begin : gen_de-
bug_req_hart
477     // SEC_CM: DM_EN.CTRL.LC_GATED
478     assign debug_req_en[i] =
lc_tx_test_true_strict(lc_hw_debug_en_gated_ndm[LcEnDebugReq + i]);
479 end
480 assign debug_req_o = debug_req & debug_req_en;
481
482 if (UseDmiInterface) begin : gen_dmi_gating
483     //////////////////////////////////////////////////
484     // TL-UL-based Debug Module Interface (DMI) //
485     //////////////////////////////////////////////////
486
487 // If DMIDirectTAP is defined, a bound-in DPI module re-
places the TAP that's defined
488 // within the ifndef block
489 `ifndef DMIDirectTAP
490     tlul_pkg::tl_h2d_t dbg_tl_h2d_win;
491     tlul_pkg::tl_d2h_t dbg_tl_d2h_win;
492
493     rv_dm_dbg_reg_top u_rv_dm_dbg_reg_top (
494         .clk_i,
495         .rst_ni,
496         .tl_i   (dbg_tl_d_i),
497         .tl_o   (dbg_tl_d_o),
498         .tl_win_o (dbg_tl_h2d_win),
499         .tl_win_i (dbg_tl_d2h_win),
500         .intg_err_o(dbg_intg_error)
501     );
502
503     rv_dm_dmi_gate #(
504         .SecVolatileRawUnlockEn(SecVolatileRawUnlockEn)
505     ) u_rv_dm_dmi_gate (
506         .clk_i,
507         .rst_ni,
508         .strap_en_override_i,
509         .strap_en_i,
510         .lc_hw_debug_clr_i,
511         .lc_hw_debug_en_i,
512         .lc_check_byp_en_i,
513         .lc_escalate_en_i,
514         .dbg_tl_h2d_win_i ( dbg_tl_h2d_win ),
515         .dbg_tl_d2h_win_o ( dbg_tl_d2h_win ),
516         .dmi_req_valid_o ( dmi_req_valid ),
517         .dmi_req_ready_i ( dmi_req_ready ),
518         .dmi_req_o       ( dmi_req ),
519         .dmi_rsp_valid_i ( dmi_rsp_valid ),
520         .dmi_rsp_ready_o ( dmi_rsp_ready ),
521         .dmi_rsp_i       ( dmi_rsp ),
522
523         // Integrity error
524         .intg_error_o(                dmi_intg_error)
525     );
526
527 // This only clears the DMI FIFO inside the dm_csrs imple-
mentation.
528 // Since the JTAG DTM used in this system can always drain
this FIFO,
529 // no additional reset request should be needed in order
to clear it.
530     assign dmi_rst_n = rst_ni;
531 `else
532     assign dmi_intg_error = 1'b0;
533     assign dbg_tl_d_o = tlul_pkg::TL_D2H_DEFAULT;
534 `endif
535
536 // Tied-off signals from the JTAG interface and read unused
signals
537 assign jtag_o = '0;
538 logic unused_signals;
539 assign unused_signals = ^{jtag_i, scan_rst_ni, pin-
mux_hw_debug_en_i};
540 end else begin : gen_jtag_gating
541     // Gating of JTAG signals
542     jtag_pkg::jtag_req_t jtag_in_int;
543     jtag_pkg::jtag_rsp_t jtag_out_int;
544
545     typedef enum logic [1:0] {
546         PmEnDmiReq,
547         PmEnJtagIn,
548         PmEnJtagOut,
549         PmEnLastPos
550     } rv_dm_pm_en_e;
551
552     lc_ctrl_pkg::lc_tx_t [PmEnLastPos-1:0] pinmux_hw_debug_en;
553     prim_lc_sync #(
554         .NumCopies(int'(PmEnLastPos))
555     ) u_pm_en_sync (
556         .clk_i,
557         .rst_ni,
558         .lc_en_i(pinmux_hw_debug_en_i),
559         .lc_en_o(pinmux_hw_debug_en)
560     )

```



```

560 );
561
562 assign jtag_in_int =
(lc_tx_test_true_strict(pinmux_hw_debug_en[PmEnJtagIn])) ? jtag_i :
'0';
563 assign jtag_o =
(lc_tx_test_true_strict(pinmux_hw_debug_en[PmEnJtagOut])) ?
jtag_out_int : '0';
564
565 logic dmi_en;
566
567 // SEC_CM: DM_EN.CTRL.LC_GATED
568 assign dmi_en =
lc_tx_test_true_strict(pinmux_hw_debug_en[PmEnDmiReq]);
569
570 // If DMIDirectTAP is not defined, a bound-in DPI module
replaces the TAP and TL-UL DMI that
571 // is defined within the ifndef block
572 `ifndef DMIDirectTAP
573 logic tck_muxed;
574 logic trst_n_muxed;
575 logic dmi_req_valid_raw, dmi_rsp_ready_raw;
576
577 prim_clock_mux2 #(
578   .NoFpgaBufG(1'b1)
579 ) u_prim_clock_mux2 (
580   .clk0_i(jtag_in_int.tck),
581   .clk1_i(clk_i),
582   .sel_i(testmode),
583   .clk_o(tck_muxed)
584 );
585
586 prim_clock_mux2 #(
587   .NoFpgaBufG(1'b1)
588 ) u_prim_rst_n_mux2 (
589   .clk0_i(jtag_in_int.trst_n),
590   .clk1_i(scan_rst_ni),
591   .sel_i(testmode),
592   .clk_o(trst_n_muxed)
593 );
594
595 // JTAG TAP
596 dmi_jtag #(
597   .IdcodeValue (IdcodeValue),
598   .NumDmiWordAbits(7)
599 ) dap (
600   .clk_i (clk_i),
601   .rst_ni (rst_ni),
602   .testmode_i (testmode),
603   .test_rst_ni (scan_rst_ni),
604
605   .dmi_rst_no (dmi_rst_n),
606   .dmi_req_o (dmi_req),
607   .dmi_req_valid_o (dmi_req_valid_raw),
608   .dmi_req_ready_i (dmi_req_ready & dmi_en),
609
610   .dmi_resp_i (dmi_rsp),
611   .dmi_resp_ready_o (dmi_rsp_ready_raw),
612   .dmi_resp_valid_i (dmi_rsp_valid & dmi_en),
613
614   //JTAG
615   .tck_i (tck_muxed),
616   .tms_i (jtag_in_int.tms),
617   .trst_ni (trst_n_muxed),
618   .td_i (jtag_in_int.tdi),
619   .td_o (jtag_out_int.tdo),
620   .tdo_oe_o (jtag_out_int.tdo_oe)
621 );
622
623 // Gated DMI signals
624 assign dmi_req_valid = dmi_req_valid_raw & dmi_en;
625 assign dmi_rsp_ready = dmi_rsp_ready_raw & dmi_en;
626 `endif
627 // Tied-off and ignore signals from the DMI interface
628 assign dmi_intg_error = 1'b0;
629 assign dbg_intg_error = 1'b0;
630 assign dbg_tl_d_o = tlul_pkg::TL_D2H_DEFAULT;
631
632 logic unused_signals;
633 assign unused_signals = ^(dbg_tl_d_i,
634   lc_check_byp_en_i,
635   lc_hw_debug_clr_i,
636   lc_escalate_en_i,
637   strap_en_i,
638   strap_en_override_i);
639
640 end
641
642 // SEC_CM: DM_EN.CTRL.LC_GATED
643 // SEC_CM: MEM_TL_LC_GATE.FSM.SPARSE
644 tlul_pkg::tl_h2d_t mem_tl_win_h2d_gated;
645 tlul_pkg::tl_d2h_t mem_tl_win_d2h_gated;
646 tlul_lc_gate #(
647   .NumGatesPerDirection(2)
648 ) u_tlul_lc_gate_rom (
649   .clk_i,
650   .rst_ni,
651   .tl_h2d_i(mem_tl_win_h2d),
652   .tl_d2h_o(mem_tl_win_d2h),
653   .tl_h2d_o(mem_tl_win_h2d_gated),
654   .tl_d2h_i(mem_tl_win_d2h_gated),
655   .flush_req_i(ndmreset_req),

```

```

655   .flush_ack_o(ndmreset_req_qual),
656   .resp_pending_o(),
657   .lc_en_i (lc_hw_debug_en_gated_ndm[LcEnRom]),
658   .err_o (rom_gate_intg_error)
659 );
660
661 prim_mubi_pkg::mubi4_t en_ifetch;
662 // SEC_CM: DM_EN.CTRL.LC_GATED, EXEC.CTRL.MUBI
663 assign en_ifetch = lc_c-
trl_pkg::lc_to_mubi4(lc_hw_debug_en_gated_ndm[LcEnFetch]);
664
665 tlul_adapter_reg #(
666   .CmdIntgCheck (1),
667   .EnableRspIntgGen (1),
668   .EnableDataIntgGen(1),
669   .RegAw (MemAw),
670   .RegDw (BusWidth),
671   .AccessLatency (1)
672 ) i_tlul_adapter_reg (
673   .clk_i,
674   .rst_ni,
675   .tl_i (mem_tl_win_h2d_gated),
676   .tl_o (mem_tl_win_d2h_gated),
677   // SEC_CM: EXEC.CTRL.MUBI
678   .en_ifetch_i(en_ifetch),
679   // SEC_CM: BUS.INTEGRITY
680   .intg_error_o(rom_intg_error),
681   .re_o (device_re),
682   .we_o (device_we),
683   .addr_o (device_addr),
684   .wdata_o (device_wdata),
685   .be_o (device_be),
686   .busy_i (1'b0),
687   .rdata_i (device_rdata),
688   .error_i (device_err)
689 );
690
691 assign device_req = device_we || device_re;
692
693 ///////////////////////////////////////////////////////////////////
694 // Debug Module Instance //
695 ///////////////////////////////////////////////////////////////////
696
697 dm_top #(
698   .NrHarts (NrHarts),
699   .BusWidth (BusWidth),
700   .SelectableHarts(SelectableHarts),
701   // The debug module provides a simplified ROM for systems
that map the debug ROM to offset 0x0
702   // on the system bus. In that case, only one scratch re-
gister has to be implemented in the core.
703   // However, we require that the DM can be placed at arbit-
rary offsets in the system, which
704   // requires the generalized debug ROM implementation and
two scratch registers. We hence set
705   // this parameter to a non-zero value (inside dm_mem, this
just feeds into a comparison with 0).
706   .DmBaseAddress (1)
707 ) u_dm_top (
708   .clk_i,
709   .rst_ni,
710   .next_dm_addr_i,
711   .testmode_i (testmode),
712   .ndmreset_o (ndmreset_req),
713   .ndmreset_ack_i (ndmreset_ack),
714   .dmactive_o,
715   .debug_req_o (debug_req),
716   .unavailable_i,
717   .hartinfo_i (hartinfo),
718   .slave_req_i (device_req),
719   .slave_we_i (device_we),
720   .slave_addr_i (device_addr_aligned),
721   .slave_be_i (device_be),
722   .slave_wdata_i (device_wdata),
723   .slave_rdata_o (device_rdata),
724   .slave_err_o (device_err),
725   .master_req_o (host_req),
726   .master_add_o (host_add),
727   .master_we_o (host_we),
728   .master_wdata_o (host_wdata),
729   .master_be_o (host_be),
730   .master_gnt_i (host_gnt),
731   .master_r_valid_i (host_r_valid),
732   .master_r_err_i (host_r_err),
733   .master_r_other_err_i (host_r_other_err),
734   .master_r_rdata_i (host_r_rdata),
735   .dmi_rst_ni (dmi_rst_n),
736   .dmi_req_valid_i (dmi_req_valid),
737   .dmi_req_ready_o (dmi_req_ready),
738   .dmi_req_i (dmi_req),
739   .dmi_resp_valid_o (dmi_rsp_valid),
740   .dmi_resp_ready_i (dmi_rsp_ready),
741   .dmi_resp_o (dmi_rsp)
742 );
743
744 ///////////////////////////////////////////////////////////////////
745 // Assertions //
746 ///////////////////////////////////////////////////////////////////
747
748 `ASSERT_KNOWN(TlRegsDValidKnown_A, regs_tl_d_o.d_valid)

```

```

749 `ASSERT_KNOWN(TlRegsAReadyKnown_A, regs_tl_d_o.a_ready)
750
751 `ASSERT_KNOWN(TlMemDValidKnown_A, mem_tl_d_o.d_valid)
752 `ASSERT_KNOWN(TlMemAReadyKnown_A, mem_tl_d_o.a_ready)
753
754 `ASSERT_KNOWN(TlSbaAValidKnown_A, sba_tl_h_o.a_valid)
755 `ASSERT_KNOWN(TlSbaDReadyKnown_A, sba_tl_h_o.d_ready)
756
757 `ASSERT_KNOWN(TlDmiDValidKnown_A, dbg_tl_d_o.d_valid)
758 `ASSERT_KNOWN(TlDmiAReadyKnown_A, dbg_tl_d_o.a_ready)
759
760 `ASSERT_KNOWN(NdmresetOKnown_A, ndmreset_req_o)
761 `ASSERT_KNOWN(DmactiveOKnown_A, dmactive_o)
762 `ASSERT_KNOWN(DebugReqOKnown_A, debug_req_o)
763 `ASSERT_KNOWN_IF(RaclErrorOKnown_A, racl_error_o, racl_er-
ror_o.valid)
764
765 if (UseDmiInterface) begin : gen_dmi_assertions
766   `ASSERT(DmiRspOneCycleAfterReq_A, dmi_req_valid |>=
dmi_rsp_valid)
767   `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(DbgTlLcGateFsm_A,
768     gen_dmi_gat-
ing.u_rv_dm_dmi_gate.u_tlul_lc_gate.dbg.u_state_regs, alert_tx_o[0])
769   end else begin : gen_jtag_assertions
770     // JTAG TDO is driven by an inverted TCK in
dmi_jtag_tap.sv
771     `ASSERT_KNOWN(JtagRspOTdoKnown_A, jtag_o.tdo, !
jtag_i.tck, !jtag_i.trst_n)
772     `ASSERT_KNOWN(JtagRspOTdoOeKnown_A, jtag_o.tdo_oe, !
jtag_i.tck, !jtag_i.trst_n)
773   end
774
775   `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(SbaTlLcGateFsm_A,
776     u_tlul_lc_gate.sba.u_state_regs, alert_tx_o[0])
777
778   `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(RomTlLcGateFsm_A,
779     u_tlul_lc_gate.rom.u_state_regs, alert_tx_o[0])
780
781   // Alert assertions for reg_we onehot check
782   `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegswOnehotCheck_A,
u_reg_regs, alert_tx_o[0])
783 endmodule

```

3. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_dbg_reg_top.sv (60 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"
8
9 module rv_dm_dbg_reg_top (
10   input clk_i,
11   input rst_ni,
12   input tlul_pkg::tl_h2d_t tl_i,
13   output tlul_pkg::tl_d2h_t tl_o,
14
15   // Output port for window
16   output tlul_pkg::tl_h2d_t tl_win_o,
17   input tlul_pkg::tl_d2h_t tl_win_i,
18
19   // To HW
20
21   // Integrity check errors
22   output logic intg_err_o
23 );
24
25 import rv_dm_reg_pkg::* ;
26
27
28 // Add an unloaded flop to make use of clock / reset
29 // This is done to specifically address lint complaints of
unused clocks/resets
30 // Since the flop is unloaded it will be removed during syn-
thesis
31 logic unused_reg;
32 always_ff @(posedge clk_i or negedge rst_ni) begin
33   if (!rst_ni) begin
34     unused_reg <= '0;
35   end else begin
36     unused_reg <= tl_i.a_valid;
37   end
38 end
39
40
41
42 // Since there are no registers in this block, commands are
routed through to windows which
43 // can report their own integrity errors.
44 assign intg_err_o = 1'b0;
45
46 // outgoing integrity generation
47 tlul_pkg::tl_d2h_t tl_o_pre;
48 tlul_rsp_intg_gen #(

```

```

49   .EnableRspIntgGen(1),
50   .EnableDataIntgGen(0)
51 ) u_rsp_intg_gen (
52   .tl_i(tl_o_pre),
53   .tl_o(tl_o)
54 );
55
56 assign tl_win_o = tl_i;
57 assign tl_o_pre = tl_win_i;
58
59 // Unused signal tieoff
60 endmodule

```

4. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_dmi_gate.sv (304 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 // DMI gate for the RISC-V Debug Module
6 //
7 // This module latches the power-on strap for DM enablement
from lc_ctrl and
8 // gates the DMI if debugging is not enabled. It also monitors for
lc_ctrl
9 // events that would invalidate the strap.
10
11 `include "prim_assert.sv"
12
13 module rv_dm_dmi_gate
14   import lc_ctrl_pkg::*;
15   #(
16     parameter bit SecVolatileRawUnlockEn = 0
17   ) (
18     input clk_i,
19     input rst_ni,
20     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION START
21
22     // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO MITIG-
ATE
23     // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DISABLED VIA
24     // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUCTION
DEVICES.
25     //
26     // Strap sampling override that is only used when SecVolat-
ileRawUnlockEn = 1, Otherwise this input
27     // is unused. This needs to be synchronized since it is com-
ing from a different clock domain.
28     // This signal goes from 0 -> 1 and then stays high, since
we only ever re-sample once. The
29     // synchronization logic can therefore just detect the edge
to create the sampling pulse
30     // internally.
31     input strap_en_override_i,
32     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
33
34     input strap_en_i,
35     // LC signals for DMI qualification
36     input lc_ctrl_pkg::lc_tx_t lc_hw_debug_clr_i,
37     input lc_ctrl_pkg::lc_tx_t lc_hw_debug_en_i,
38     input lc_ctrl_pkg::lc_tx_t lc_check_byp_en_i,
39     input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
40
41     // TL-UL-based DMI
42     input tlul_pkg::tl_h2d_t dbg_tl_h2d_win_i,
43     output tlul_pkg::tl_d2h_t dbg_tl_d2h_win_o,
44
45     // Gated Rocket Legacy DMI
46     output dmi_req_valid_o,
47     input dmi_req_ready_i,
48     output dm::dmi_req_t dmi_req_o,
49     input dmi_rsp_valid_i,
50     output dmi_rsp_ready_o,
51     input dm::dmi_resp_t dmi_rsp_i,
52
53     // Integrity error
54     output intg_error_o
55 );
56
57 // Life cycle signal synchronizers //
58 // -----
59 lc_tx_t [0:0] lc_hw_debug_clr;
60 prim_lc_sync #(
61   .NumCopies(1)
62 ) u_prim_lc_sync_lc_hw_debug_clr (
63   .clk_i,
64   .rst_ni,
65   .lc_en_i(lc_hw_debug_clr_i),
66   .lc_en_o(lc_hw_debug_clr)
67 );
68 lc_tx_t [0:0] lc_hw_debug_en;
69 prim_lc_sync #(
70   .NumCopies(1)
71 ) u_prim_lc_sync_lc_hw_debug_en (

```



```

71   .clk_i,
72   .rst_ni,
73   .lc_en_i(lc_hw_debug_en_i),
74   .lc_en_o(lc_hw_debug_en)
75 );
76 lc_tx_t [0:0] lc_check_byp_en;
77 prim_lc_sync #(
78   .NumCopies(1)
79 ) u_prim_lc_sync_lc_check_byp_en (
80   .clk_i,
81   .rst_ni,
82   .lc_en_i(lc_check_byp_en_i),
83   .lc_en_o(lc_check_byp_en)
84 );
85 lc_tx_t [0:0] lc_escalate_en;
86 prim_lc_sync #(
87   .NumCopies(1)
88 ) u_prim_lc_sync_lc_escalate_en (
89   .clk_i,
90   .rst_ni,
91   .lc_en_i(lc_escalate_en_i),
92   .lc_en_o(lc_escalate_en)
93 );
94
95 ///////////////////////////////////////////////////
96 // Strap Sampling Logic //
97 ///////////////////////////////////////////////////
98
99 logic strap_en;
100 if (SecVolatileRawUnlockEn) begin : gen_strap_override
101   logic strap_en_override_d, strap_en_override_q;
102
103   prim_flop_2sync #(
104     .Width(1),
105     .ResetValue(0)
106   ) u_prim_flop_2sync (
107     .clk_i,
108     .rst_ni,
109     .d_i(strap_en_override_i),
110     .q_o(strap_en_override_d)
111   );
112
113   always_ff @(posedge clk_i or negedge rst_ni) begin :
p_strap_override_reg
114     if(!rst_ni) begin
115       strap_en_override_q <= 1'b0;
116     end else begin
117       strap_en_override_q <= strap_en_override_d;
118     end
119   end
120
121   // Detect a posedge on the override signal and reflect it
for just a cycle on strap_en
122   // (ignoring the fact that strap_en_override_i will stay
high afterwards).
123   assign strap_en = strap_en_i || (strap_en_override_d && !
strap_en_override_q);
124
125   // Once strap_en_override_i is asserted, it will always
stay high
126   `ASSUME(LcCtrlStrapSampleOverrideOnce_A,
strap_en_override_i => strap_en_override_i)
127
128   end else begin : gen_no_strap_override
129     logic unused_strap_en_override;
130     assign unused_strap_en_override = strap_en_override_i;
131     assign strap_en = strap_en_i;
132   end
133
134   //////////////////////////////////////
135   // LC_HW_DEBUG_EN Latching //
136   //////////////////////////////////////
137
138   // In order to keep a RV_DM JTAG debug session alive during
NDM reset, we need to memorize the
139   // state of lc_hw_debug_en, since OTP/LC will be reset as
part of NDM reset (the only parts not
140   // reset are in the pwr/rst/clkmgrs, RV_DM and this strap
sampling module). We sample the life
141   // cycle signal state when the strap sampling pulse is as-
serted but the PWRMGR. This pulse is
142   // asserted once during boot (and not after an NDM reset).
143   //
144   // Note that we make sure to invalidate the sampled
lc_hw_debug_en whenever lc_check_byp_en or
145   // lc_escalate_en are not OFF. lc_escalate_en is asserted as
part of an escalation, and
146   // lc_check_byp_en is asserted whenever a life cycle trans-
ition is initiated (it causes the OTP
147   // controller to skip background checks on the life cycle
partition as it undergoes
148   // modification). This makes sure that the sampled value
here does not survive a life cycle
149   // transition.
150   //
151   // Finally, note that there is secondary gating on the RV_DM
and DFT TAPS that is always consuming
152   // live lc_hw_debug_en and lc_dft_en signals for added pro-
tection.

```

```

153
154   // Convert the strap enable pulse to a mubi signal and mask
lc_hw_debug_en with it.
155   lc_tx_t lc_strap_en, lc_hw_debug_en_masked;
156   assign lc_strap_en = lc_tx_bool_to_lc_tx(strap_en);
157   assign lc_hw_debug_en_masked = lc_tx_and_hi(lc_strap_en,
lc_hw_debug_en[0]);
158
159   // Output ON if
160   // - If the strap sampling pulse is asserted and lc_hw-de-
bug_en is ON
161   // - If the strap_hw_debug_en_q is already set to ON (this
is the latching feedback loop)
162   // Note: make sure we use a hardened, rectifying OR function
since otherwise two non-strict
163   // values may produce a strict ON value.
164   lc_tx_t hw_debug_en_set, strap_hw_debug_en_q;
165   prim_lc_or_hardenen #(
166     .ActVal(On)
167   ) u_prim_lc_or_hardenen (
168     .clk_i,
169     .rst_ni,
170     .lc_en_a_i(lc_hw_debug_en_masked),
171     .lc_en_b_i(strap_hw_debug_en_q),
172     .lc_en_o (hw_debug_en_set)
173   );
174
175   // Output ON if lc_check_byp_en AND lc_escalate_en are
strictly OFF; otherwise output a non-ON
176   // value (which may be different from OFF).
177   lc_tx_t neither_check_byp_nor_escalate;
178   assign neither_check_byp_nor_escalate =
lc_tx_inv(lc_tx_and_lo(lc_check_byp_en[0],
lc_escalate_en[0]));
179
180   // Output ON if the signal above is strictly ON AND
lc_hw_debug_clr is strictly OFF; otherwise
181   // output a non-ON value (which may be different from OFF).
182   lc_tx_t hw_debug_en_gating;
183   assign hw_debug_en_gating =
lc_tx_and_hi(neither_check_byp_nor_escalate,
lc_tx_inv(lc_hw_de-
bug_clr[0]));
184
185   // Gate the hw_debug_en_set signal and feed it into the
latching flop.
186   lc_tx_t strap_hw_debug_en_d;
187   assign strap_hw_debug_en_d = lc_tx_and_hi(hw_debug_en_set,
hw_debug_en_gating);
188
189   prim_lc_sender u_prim_lc_sender_strap_hw_debug_en (
190     .clk_i,
191     .rst_ni,
192     .lc_en_i(strap_hw_debug_en_d),
193     .lc_en_o(strap_hw_debug_en_q)
194   );
195
196   typedef enum logic [1:0] {
197     StEnDmiTlulAdapter,
198     StEnDmiTlulGate,
199     StEnDmiTlulLast
200   } strap_hw_debug_en_e;
201
202   lc_tx_t [StEnDmiTlulLast-1:0] strap_hw_debug_en;
203   prim_lc_sync #(
204     .NumCopies(int'(StEnDmiTlulLast)),
205     .AsyncOn(0) // no sync needed
206   ) u_prim_lc_sync_strap_hw_debug_en (
207     .clk_i,
208     .rst_ni,
209     .lc_en_i(strap_hw_debug_en_q),
210     .lc_en_o(strap_hw_debug_en)
211   );
212
213   // SEC_CM: DM_EN.CTRL.LC_GATED
214   logic dmi_en;
215   assign dmi_en =
lc_tx_test_true_strict(strap_hw_debug_en[StEnDmiTlulAdapter]);
216
217   //////////////////////////////////////
218   // TL-UL to Rocket Legacy DMI //
219   //////////////////////////////////////
220
221   logic dbg_gate_intg_error, dmi_intg_error;
222   assign intg_error_o = dbg_gate_intg_error | dmi_intg_error;
223
224   logic tlul_resp_pending;
225
226   tlul_pkg::tl_h2d_t dbg_tl_h2d_gated;
227   tlul_pkg::tl_d2h_t dbg_tl_d2h_gated;
228   tlul_lc_gate #(
229     // If the DMI side is gated, the spec requires the DM to
return a valid all-zero response.
230     .ReturnBlankResp(1)
231   ) u_tlul_lc_gate_dbg (
232     .clk_i,
233     .rst_ni,
234     .tl_h2d_i (dbg_tl_h2d_win_i),

```

```

237 .tl_d2h_o      (dbg_tl_d2h_win_o),
238 .tl_h2d_o      (dbg_tl_h2d_gated),
239 .tl_d2h_i      (dbg_tl_d2h_gated),

240 // The DMI side should not be flushed upon NDM-reset since
we keep the
241 // debugger side of the connection stable during an NDM-
reset request.
242 .flush_req_i    (1'b0),
243 .flush_ack_o    (),
244 .resp_pending_o (tlul_resp_pending),
245 .lc_en_i        (strap_hw_debug_en[StEnDmiTlulLcGate]),
246 .err_o          (dbg_gate_intg_error)
247 );
248
249 logic dmi_req_ready, dmi_rsp_valid;
250 tlul_adapter_dmi u_tlul_adapter_dmi (
251   .clk_i,
252   .rst_ni,
253   .tl_h2d_i      (dbg_tl_h2d_gated),
254   .tl_d2h_o      (dbg_tl_d2h_gated),
255   .intg_error_o  (dmi_intg_error),
256   .dmi_req_valid_o (dmi_req_valid_o),
257   .dmi_req_ready_i (dmi_req_ready),
258   .dmi_req_o      (dmi_req_o),
259   .dmi_resp_valid_i (dmi_rsp_valid),
260   .dmi_resp_ready_o (dmi_rsp_ready_o),
261   .dmi_resp_i     (dmi_rsp_i)
262 );
263
264 // Connect the TL-UL adapter to the DMI handshake if the DMI
is enabled. Let responses also
265 // propagate from the DMI to TL-UL if any responses are
pending.
266 assign dmi_req_ready = dmi_req_ready_i & dmi_en;
267 assign dmi_rsp_valid = dmi_rsp_valid_i & (dmi_en |
tlul_resp_pending);
268
269 // Assertions //
270 // Assertions //
271 // Assertions //
272
273 // Check that we can correctly latch upon strap_en_i
274 `ASSERT(LcHwDebugEnSet_A,
275   (lc_tx_test_true_strict(lc_hw_debug_en[0]) ||
lc_tx_test_true_strict(strap_hw_debug_en_q)) &&
lc_tx_test_false_strict(lc_hw_debug_clr[0]) &&
lc_tx_test_false_strict(lc_check_byp_en[0]) &&
lc_tx_test_false_strict(lc_escalate_en[0]) &&
strap_en_i
|=>
lc_tx_test_true_strict(strap_hw_debug_en_q))
276
277
278
279
280
281
282
283 // Check that latching ON can only occur if lc_hw_debug_en_i
is set.
284 `ASSERT(LcHwDebugEnSetRev0_A,
285   lc_tx_test_false_loose(strap_hw_debug_en_q) ##1
lc_tx_test_true_strict(strap_hw_debug_en_q)
|=>
$past(lc_tx_test_true_strict(lc_hw_debug_en[0])))
286
287
288
289 // Check that latching ON can only occur if strap_en is set.
290 `ASSERT(LcHwDebugEnSetRev1_A,
291   lc_tx_test_false_loose(strap_hw_debug_en_q) ##1
lc_tx_test_true_strict(strap_hw_debug_en_q)
|=>
$past(strap_en))
292
293
294
295 // Check that any non-OFF value on lc_check_byp_en_i and
296 // lc_escalate_en_i clears the latched value.
297 `ASSERT(LcHwDebugEnClear_A,
298   lc_tx_test_true_loose(lc_check_byp_en[0]) ||
lc_tx_test_true_loose(lc_escalate_en[0]) ||
lc_tx_test_true_loose(lc_hw_debug_clr[0])
|=>
lc_tx_test_false_loose(strap_hw_debug_en_q))
299
300
301
302
303
304 endmodule

```

5. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_pkg.sv (12 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5
6 package rv_dm_pkg;
7
8   typedef logic [31:0] next_dm_addr_t;
9
10   parameter next_dm_addr_t NEXT_DM_ADDR_DEFAULT = '0;
11
12 endpackage : rv_dm_pkg

```

6. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_reg_pkg.sv (1,002 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //

5 // Register Package auto-generated by `reggen` containing data
structure
6
7 package rv_dm_reg_pkg;
8
9 // Param list
10 parameter int NrHarts = 1;
11 parameter int NumAlerts = 1;
12
13 // Address widths within the block
14 parameter int RegsAw = 4;
15 parameter int MemAw = 12;
16 parameter int DbgAw = 9;
17
18 // Number of registers for every interface
19 parameter int NumRegsRegs = 3;
20 parameter int NumRegsMem = 281;
21 parameter int NumRegsDbg = 0;
22
23 // Alert indices
24 typedef enum int {
25   AlertFatalFaultIdx = 0
26 } rv_dm_alert_idx_t;
27
28 // Typedefs for registers for regs interface //
29 // Typedefs for registers for regs interface //
30 // Typedefs for registers for regs interface //
31
32 typedef struct packed {
33   logic q;
34   logic qe;
35 } rv_dm_reg2hw_alert_test_reg_t;
36
37 typedef struct packed {
38   logic [31:0] q;
39 } rv_dm_reg2hw_late_debug_enable_reg_t;
40
41 // Register -> HW type for regs interface
42 typedef struct packed {
43   rv_dm_reg2hw_alert_test_reg_t alert_test; // [33:32]
44
45   rv_dm_reg2hw_late_debug_enable_reg_t late_debug_enable; //
[31:0]
46 } rv_dm_regs_reg2hw_t;
47
48 // Register offsets for regs interface
49 parameter logic [RegsAw-1:0] RV_DM_ALERT_TEST_OFFSET = 4'h
0;
50 parameter logic [RegsAw-1:0] RV_DM_LATE_DEBUG_ENABLE_OFFSET
= 4'h 8;
51
52 // Reset values for hwext registers and their fields for
regs interface
53 parameter logic [0:0] RV_DM_ALERT_TEST_RESVAL = 1'h 0;
54 parameter logic [0:0] RV_DM_ALERT_TEST_FATAL_FAULT_RESVAL =
1'h 0;
55
56 // Register index for regs interface
57 typedef enum int {
58   RV_DM_ALERT_TEST,
59   RV_DM_LATE_DEBUG_ENABLE_REGWEN,
60   RV_DM_LATE_DEBUG_ENABLE
61 } rv_dm_regs_id_e;
62
63 // Register width information to check illegal writes for
regs interface
64 parameter logic [3:0] RV_DM_REGS_PERMIT [3] = '{
65   4'b 0001, // index[0] RV_DM_ALERT_TEST
66   4'b 0001, // index[1] RV_DM_LATE_DEBUG_ENABLE_REGWEN
67   4'b 1111 // index[2] RV_DM_LATE_DEBUG_ENABLE
68 };
69
70 // Typedefs for registers for mem interface //
71 // Typedefs for registers for mem interface //
72 // Typedefs for registers for mem interface //
73
74 typedef struct packed {
75   logic q;
76 } rv_dm_reg2hw_halted_reg_t;
77
78 typedef struct packed {
79   logic q;
80 } rv_dm_reg2hw_going_reg_t;
81
82 typedef struct packed {
83   logic q;
84 } rv_dm_reg2hw_resuming_reg_t;

```

```

85
86 typedef struct packed {
87     logic q;
88 } rv_dm_reg2hw_exception_reg_t;
89
90 typedef struct packed {
91     logic [31:0] q;
92 } rv_dm_reg2hw_where_to_reg_t;
93
94 typedef struct packed {
95     logic [31:0] q;
96 } rv_dm_reg2hw_abstractcmd_mreg_t;
97
98 typedef struct packed {
99     logic [31:0] q;
100 } rv_dm_reg2hw_program_buffer_mreg_t;
101
102 typedef struct packed {
103     logic [31:0] q;
104 } rv_dm_reg2hw_dataaddr_mreg_t;
105
106 typedef struct packed {
107     logic [31:0] q;
108 } rv_dm_reg2hw_flags_mreg_t;
109
110 typedef struct packed {
111     logic d;
112     logic de;
113 } rv_dm_hw2reg_halted_reg_t;
114
115 typedef struct packed {
116     logic [31:0] d;
117     logic de;
118 } rv_dm_hw2reg_where_to_reg_t;
119
120 // Register -> HW type for mem interface
121 typedef struct packed {
122     rv_dm_reg2hw_halted_reg_t halted; // [8867:8867]
123     rv_dm_reg2hw_going_reg_t going; // [8866:8866]
124     rv_dm_reg2hw_resuming_reg_t resuming; // [8865:8865]
125     rv_dm_reg2hw_exception_reg_t exception; // [8864:8864]
126     rv_dm_reg2hw_where_to_reg_t where_to; // [8863:8832]
127     rv_dm_reg2hw_abstractcmd_mreg_t [9:0] abstractcmd; //
[8831:8512]
128     rv_dm_reg2hw_program_buffer_mreg_t [7:0] pro-
gram_buffer; // [8511:8256]
129     rv_dm_reg2hw_dataaddr_mreg_t [1:0] dataaddr; //
[8255:8192]
130     rv_dm_reg2hw_flags_mreg_t [255:0] flags; // [8191:0]
131 } rv_dm_mem_reg2hw_t;
132
133 // HW -> register type for mem interface
134 typedef struct packed {
135     rv_dm_hw2reg_halted_reg_t halted; // [34:33]
136     rv_dm_hw2reg_where_to_reg_t where_to; // [32:0]
137 } rv_dm_mem_hw2reg_t;
138
139 // Register offsets for mem interface
140 parameter logic [MemAw-1:0] RV_DM_HALTED_OFFSET = 12'h 100;
141 parameter logic [MemAw-1:0] RV_DM_GOING_OFFSET = 12'h 108;
142 parameter logic [MemAw-1:0] RV_DM_RESUMING_OFFSET = 12'h
110;
143 parameter logic [MemAw-1:0] RV_DM_EXCEPTION_OFFSET = 12'h
118;
144 parameter logic [MemAw-1:0] RV_DM_WHERETO_OFFSET = 12'h 300;
145 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_0_OFFSET =
12'h 338;
146 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_1_OFFSET =
12'h 33c;
147 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_2_OFFSET =
12'h 340;
148 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_3_OFFSET =
12'h 344;
149 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_4_OFFSET =
12'h 348;
150 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_5_OFFSET =
12'h 34c;
151 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_6_OFFSET =
12'h 350;
152 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_7_OFFSET =
12'h 354;
153 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_8_OFFSET =
12'h 358;
154 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_9_OFFSET =
12'h 35c;
155 parameter logic [MemAw-1:0] RV_DM_PROGRAM_BUFFER_0_OFFSET =
12'h 360;
156 parameter logic [MemAw-1:0] RV_DM_PROGRAM_BUFFER_1_OFFSET =
12'h 364;
157 parameter logic [MemAw-1:0] RV_DM_PROGRAM_BUFFER_2_OFFSET =
12'h 368;
158 parameter logic [MemAw-1:0] RV_DM_PROGRAM_BUFFER_3_OFFSET =
12'h 36c;
159 parameter logic [MemAw-1:0] RV_DM_PROGRAM_BUFFER_4_OFFSET =
12'h 370;
160 parameter logic [MemAw-1:0] RV_DM_PROGRAM_BUFFER_5_OFFSET =
12'h 374;
161 parameter logic [MemAw-1:0] RV_DM_PROGRAM_BUFFER_6_OFFSET =
12'h 378;
162 parameter logic [MemAw-1:0] RV_DM_PROGRAM_BUFFER_7_OFFSET =
12'h 37c;
163
164 parameter logic [MemAw-1:0] RV_DM_DATAADDR_0_OFFSET = 12'h
380;
165 parameter logic [MemAw-1:0] RV_DM_DATAADDR_1_OFFSET = 12'h
384;
166 parameter logic [MemAw-1:0] RV_DM_FLAGS_0_OFFSET = 12'h 400;
167 parameter logic [MemAw-1:0] RV_DM_FLAGS_1_OFFSET = 12'h 404;
168 parameter logic [MemAw-1:0] RV_DM_FLAGS_2_OFFSET = 12'h 408;
169 parameter logic [MemAw-1:0] RV_DM_FLAGS_3_OFFSET = 12'h 40c;
170 parameter logic [MemAw-1:0] RV_DM_FLAGS_4_OFFSET = 12'h 410;
171 parameter logic [MemAw-1:0] RV_DM_FLAGS_5_OFFSET = 12'h 414;
172 parameter logic [MemAw-1:0] RV_DM_FLAGS_6_OFFSET = 12'h 418;
173 parameter logic [MemAw-1:0] RV_DM_FLAGS_7_OFFSET = 12'h 41c;
174 parameter logic [MemAw-1:0] RV_DM_FLAGS_8_OFFSET = 12'h 420;
175 parameter logic [MemAw-1:0] RV_DM_FLAGS_9_OFFSET = 12'h 424;
176 parameter logic [MemAw-1:0] RV_DM_FLAGS_10_OFFSET = 12'h
428;
177 parameter logic [MemAw-1:0] RV_DM_FLAGS_11_OFFSET = 12'h
42c;
178 parameter logic [MemAw-1:0] RV_DM_FLAGS_12_OFFSET = 12'h
430;
179 parameter logic [MemAw-1:0] RV_DM_FLAGS_13_OFFSET = 12'h
434;
180 parameter logic [MemAw-1:0] RV_DM_FLAGS_14_OFFSET = 12'h
438;
181 parameter logic [MemAw-1:0] RV_DM_FLAGS_15_OFFSET = 12'h
43c;
182 parameter logic [MemAw-1:0] RV_DM_FLAGS_16_OFFSET = 12'h
440;
183 parameter logic [MemAw-1:0] RV_DM_FLAGS_17_OFFSET = 12'h
444;
184 parameter logic [MemAw-1:0] RV_DM_FLAGS_18_OFFSET = 12'h
448;
185 parameter logic [MemAw-1:0] RV_DM_FLAGS_19_OFFSET = 12'h
44c;
186 parameter logic [MemAw-1:0] RV_DM_FLAGS_20_OFFSET = 12'h
450;
187 parameter logic [MemAw-1:0] RV_DM_FLAGS_21_OFFSET = 12'h
454;
188 parameter logic [MemAw-1:0] RV_DM_FLAGS_22_OFFSET = 12'h
458;
189 parameter logic [MemAw-1:0] RV_DM_FLAGS_23_OFFSET = 12'h
45c;
190 parameter logic [MemAw-1:0] RV_DM_FLAGS_24_OFFSET = 12'h
460;
191 parameter logic [MemAw-1:0] RV_DM_FLAGS_25_OFFSET = 12'h
464;
192 parameter logic [MemAw-1:0] RV_DM_FLAGS_26_OFFSET = 12'h
468;
193 parameter logic [MemAw-1:0] RV_DM_FLAGS_27_OFFSET = 12'h
46c;
194 parameter logic [MemAw-1:0] RV_DM_FLAGS_28_OFFSET = 12'h
470;
195 parameter logic [MemAw-1:0] RV_DM_FLAGS_29_OFFSET = 12'h
474;
196 parameter logic [MemAw-1:0] RV_DM_FLAGS_30_OFFSET = 12'h
478;
197 parameter logic [MemAw-1:0] RV_DM_FLAGS_31_OFFSET = 12'h
47c;
198 parameter logic [MemAw-1:0] RV_DM_FLAGS_32_OFFSET = 12'h
480;
199 parameter logic [MemAw-1:0] RV_DM_FLAGS_33_OFFSET = 12'h
484;
200 parameter logic [MemAw-1:0] RV_DM_FLAGS_34_OFFSET = 12'h
488;
201 parameter logic [MemAw-1:0] RV_DM_FLAGS_35_OFFSET = 12'h
48c;
202 parameter logic [MemAw-1:0] RV_DM_FLAGS_36_OFFSET = 12'h
490;
203 parameter logic [MemAw-1:0] RV_DM_FLAGS_37_OFFSET = 12'h
494;
204 parameter logic [MemAw-1:0] RV_DM_FLAGS_38_OFFSET = 12'h
498;
205 parameter logic [MemAw-1:0] RV_DM_FLAGS_39_OFFSET = 12'h
49c;
206 parameter logic [MemAw-1:0] RV_DM_FLAGS_40_OFFSET = 12'h
4a0;
207 parameter logic [MemAw-1:0] RV_DM_FLAGS_41_OFFSET = 12'h
4a4;
208 parameter logic [MemAw-1:0] RV_DM_FLAGS_42_OFFSET = 12'h
4a8;
209 parameter logic [MemAw-1:0] RV_DM_FLAGS_43_OFFSET = 12'h
4ac;
210 parameter logic [MemAw-1:0] RV_DM_FLAGS_44_OFFSET = 12'h
4b0;
211 parameter logic [MemAw-1:0] RV_DM_FLAGS_45_OFFSET = 12'h
4b4;
212 parameter logic [MemAw-1:0] RV_DM_FLAGS_46_OFFSET = 12'h
4b8;
213 parameter logic [MemAw-1:0] RV_DM_FLAGS_47_OFFSET = 12'h
4bc;
214 parameter logic [MemAw-1:0] RV_DM_FLAGS_48_OFFSET = 12'h
4c0;
215 parameter logic [MemAw-1:0] RV_DM_FLAGS_49_OFFSET = 12'h
4c4;
216 parameter logic [MemAw-1:0] RV_DM_FLAGS_50_OFFSET = 12'h
4c8;
217 parameter logic [MemAw-1:0] RV_DM_FLAGS_51_OFFSET = 12'h
4cc;
218 parameter logic [MemAw-1:0] RV_DM_FLAGS_52_OFFSET = 12'h
4d0;
219 parameter logic [MemAw-1:0] RV_DM_FLAGS_53_OFFSET = 12'h

```


[illegible]

```

7fc;
421
422 // Window parameters for mem interface
423 parameter logic [MemAw-1:0] RV_DM_ROM_OFFSET = 12'h 800;
424 parameter int unsigned RV_DM_ROM_SIZE = 'h 800;
425 parameter int unsigned RV_DM_ROM_IDX = 0;
426
427 // Register index for mem interface
428 typedef enum int {
429     RV_DM_HALTED,
430     RV_DM_GOING,
431     RV_DM_RESUMING,
432     RV_DM_EXCEPTION,
433     RV_DM_WHERETO,
434     RV_DM_ABSTRACTCMD_0,
435     RV_DM_ABSTRACTCMD_1,
436     RV_DM_ABSTRACTCMD_2,
437     RV_DM_ABSTRACTCMD_3,
438     RV_DM_ABSTRACTCMD_4,
439     RV_DM_ABSTRACTCMD_5,
440     RV_DM_ABSTRACTCMD_6,
441     RV_DM_ABSTRACTCMD_7,
442     RV_DM_ABSTRACTCMD_8,
443     RV_DM_ABSTRACTCMD_9,
444     RV_DM_PROGRAM_BUFFER_0,
445     RV_DM_PROGRAM_BUFFER_1,
446     RV_DM_PROGRAM_BUFFER_2,
447     RV_DM_PROGRAM_BUFFER_3,
448     RV_DM_PROGRAM_BUFFER_4,
449     RV_DM_PROGRAM_BUFFER_5,
450     RV_DM_PROGRAM_BUFFER_6,
451     RV_DM_PROGRAM_BUFFER_7,
452     RV_DM_DATAADDR_0,
453     RV_DM_DATAADDR_1,
454     RV_DM_FLAGS_0,
455     RV_DM_FLAGS_1,
456     RV_DM_FLAGS_2,
457     RV_DM_FLAGS_3,
458     RV_DM_FLAGS_4,
459     RV_DM_FLAGS_5,
460     RV_DM_FLAGS_6,
461     RV_DM_FLAGS_7,
462     RV_DM_FLAGS_8,
463     RV_DM_FLAGS_9,
464     RV_DM_FLAGS_10,
465     RV_DM_FLAGS_11,
466     RV_DM_FLAGS_12,
467     RV_DM_FLAGS_13,
468     RV_DM_FLAGS_14,
469     RV_DM_FLAGS_15,
470     RV_DM_FLAGS_16,
471     RV_DM_FLAGS_17,
472     RV_DM_FLAGS_18,
473     RV_DM_FLAGS_19,
474     RV_DM_FLAGS_20,
475     RV_DM_FLAGS_21,
476     RV_DM_FLAGS_22,
477     RV_DM_FLAGS_23,
478     RV_DM_FLAGS_24,
479     RV_DM_FLAGS_25,
480     RV_DM_FLAGS_26,
481     RV_DM_FLAGS_27,
482     RV_DM_FLAGS_28,
483     RV_DM_FLAGS_29,
484     RV_DM_FLAGS_30,
485     RV_DM_FLAGS_31,
486     RV_DM_FLAGS_32,
487     RV_DM_FLAGS_33,
488     RV_DM_FLAGS_34,
489     RV_DM_FLAGS_35,
490     RV_DM_FLAGS_36,
491     RV_DM_FLAGS_37,
492     RV_DM_FLAGS_38,
493     RV_DM_FLAGS_39,
494     RV_DM_FLAGS_40,
495     RV_DM_FLAGS_41,
496     RV_DM_FLAGS_42,
497     RV_DM_FLAGS_43,
498     RV_DM_FLAGS_44,
499     RV_DM_FLAGS_45,
500     RV_DM_FLAGS_46,
501     RV_DM_FLAGS_47,
502     RV_DM_FLAGS_48,
503     RV_DM_FLAGS_49,
504     RV_DM_FLAGS_50,
505     RV_DM_FLAGS_51,
506     RV_DM_FLAGS_52,
507     RV_DM_FLAGS_53,
508     RV_DM_FLAGS_54,
509     RV_DM_FLAGS_55,
510     RV_DM_FLAGS_56,
511     RV_DM_FLAGS_57,
512     RV_DM_FLAGS_58,
513     RV_DM_FLAGS_59,
514     RV_DM_FLAGS_60,
515     RV_DM_FLAGS_61,
516     RV_DM_FLAGS_62,
517     RV_DM_FLAGS_63,
518     RV_DM_FLAGS_64,
519     RV_DM_FLAGS_65,
520     RV_DM_FLAGS_66,
521     RV_DM_FLAGS_67,
522     RV_DM_FLAGS_68,
523     RV_DM_FLAGS_69,
524     RV_DM_FLAGS_70,
525     RV_DM_FLAGS_71,
526     RV_DM_FLAGS_72,
527     RV_DM_FLAGS_73,
528     RV_DM_FLAGS_74,
529     RV_DM_FLAGS_75,
530     RV_DM_FLAGS_76,
531     RV_DM_FLAGS_77,
532     RV_DM_FLAGS_78,
533     RV_DM_FLAGS_79,
534     RV_DM_FLAGS_80,
535     RV_DM_FLAGS_81,
536     RV_DM_FLAGS_82,
537     RV_DM_FLAGS_83,
538     RV_DM_FLAGS_84,
539     RV_DM_FLAGS_85,
540     RV_DM_FLAGS_86,
541     RV_DM_FLAGS_87,
542     RV_DM_FLAGS_88,
543     RV_DM_FLAGS_89,
544     RV_DM_FLAGS_90,
545     RV_DM_FLAGS_91,
546     RV_DM_FLAGS_92,
547     RV_DM_FLAGS_93,
548     RV_DM_FLAGS_94,
549     RV_DM_FLAGS_95,
550     RV_DM_FLAGS_96,
551     RV_DM_FLAGS_97,
552     RV_DM_FLAGS_98,
553     RV_DM_FLAGS_99,
554     RV_DM_FLAGS_100,
555     RV_DM_FLAGS_101,
556     RV_DM_FLAGS_102,
557     RV_DM_FLAGS_103,
558     RV_DM_FLAGS_104,
559     RV_DM_FLAGS_105,
560     RV_DM_FLAGS_106,
561     RV_DM_FLAGS_107,
562     RV_DM_FLAGS_108,
563     RV_DM_FLAGS_109,
564     RV_DM_FLAGS_110,
565     RV_DM_FLAGS_111,
566     RV_DM_FLAGS_112,
567     RV_DM_FLAGS_113,
568     RV_DM_FLAGS_114,
569     RV_DM_FLAGS_115,
570     RV_DM_FLAGS_116,
571     RV_DM_FLAGS_117,
572     RV_DM_FLAGS_118,
573     RV_DM_FLAGS_119,
574     RV_DM_FLAGS_120,
575     RV_DM_FLAGS_121,
576     RV_DM_FLAGS_122,
577     RV_DM_FLAGS_123,
578     RV_DM_FLAGS_124,
579     RV_DM_FLAGS_125,
580     RV_DM_FLAGS_126,
581     RV_DM_FLAGS_127,
582     RV_DM_FLAGS_128,
583     RV_DM_FLAGS_129,
584     RV_DM_FLAGS_130,
585     RV_DM_FLAGS_131,
586     RV_DM_FLAGS_132,
587     RV_DM_FLAGS_133,
588     RV_DM_FLAGS_134,
589     RV_DM_FLAGS_135,
590     RV_DM_FLAGS_136,
591     RV_DM_FLAGS_137,
592     RV_DM_FLAGS_138,
593     RV_DM_FLAGS_139,
594     RV_DM_FLAGS_140,
595     RV_DM_FLAGS_141,
596     RV_DM_FLAGS_142,
597     RV_DM_FLAGS_143,
598     RV_DM_FLAGS_144,
599     RV_DM_FLAGS_145,
600     RV_DM_FLAGS_146,
601     RV_DM_FLAGS_147,
602     RV_DM_FLAGS_148,
603     RV_DM_FLAGS_149,
604     RV_DM_FLAGS_150,
605     RV_DM_FLAGS_151,
606     RV_DM_FLAGS_152,
607     RV_DM_FLAGS_153,
608     RV_DM_FLAGS_154,
609     RV_DM_FLAGS_155,
610     RV_DM_FLAGS_156,
611     RV_DM_FLAGS_157,
612     RV_DM_FLAGS_158,
613     RV_DM_FLAGS_159,
614     RV_DM_FLAGS_160,
615     RV_DM_FLAGS_161,
616     RV_DM_FLAGS_162,
617     RV_DM_FLAGS_163,
618     RV_DM_FLAGS_164,
619     RV_DM_FLAGS_165,
620     RV_DM_FLAGS_166,
621     RV_DM_FLAGS_167,

```

```

622 RV_DM_FLAGS_168,
623 RV_DM_FLAGS_169,
624 RV_DM_FLAGS_170,
625 RV_DM_FLAGS_171,
626 RV_DM_FLAGS_172,
627 RV_DM_FLAGS_173,
628 RV_DM_FLAGS_174,
629 RV_DM_FLAGS_175,
630 RV_DM_FLAGS_176,
631 RV_DM_FLAGS_177,
632 RV_DM_FLAGS_178,
633 RV_DM_FLAGS_179,
634 RV_DM_FLAGS_180,
635 RV_DM_FLAGS_181,
636 RV_DM_FLAGS_182,
637 RV_DM_FLAGS_183,
638 RV_DM_FLAGS_184,
639 RV_DM_FLAGS_185,
640 RV_DM_FLAGS_186,
641 RV_DM_FLAGS_187,
642 RV_DM_FLAGS_188,
643 RV_DM_FLAGS_189,
644 RV_DM_FLAGS_190,
645 RV_DM_FLAGS_191,
646 RV_DM_FLAGS_192,
647 RV_DM_FLAGS_193,
648 RV_DM_FLAGS_194,
649 RV_DM_FLAGS_195,
650 RV_DM_FLAGS_196,
651 RV_DM_FLAGS_197,
652 RV_DM_FLAGS_198,
653 RV_DM_FLAGS_199,
654 RV_DM_FLAGS_200,
655 RV_DM_FLAGS_201,
656 RV_DM_FLAGS_202,
657 RV_DM_FLAGS_203,
658 RV_DM_FLAGS_204,
659 RV_DM_FLAGS_205,
660 RV_DM_FLAGS_206,
661 RV_DM_FLAGS_207,
662 RV_DM_FLAGS_208,
663 RV_DM_FLAGS_209,
664 RV_DM_FLAGS_210,
665 RV_DM_FLAGS_211,
666 RV_DM_FLAGS_212,
667 RV_DM_FLAGS_213,
668 RV_DM_FLAGS_214,
669 RV_DM_FLAGS_215,
670 RV_DM_FLAGS_216,
671 RV_DM_FLAGS_217,
672 RV_DM_FLAGS_218,
673 RV_DM_FLAGS_219,
674 RV_DM_FLAGS_220,
675 RV_DM_FLAGS_221,
676 RV_DM_FLAGS_222,
677 RV_DM_FLAGS_223,
678 RV_DM_FLAGS_224,
679 RV_DM_FLAGS_225,
680 RV_DM_FLAGS_226,
681 RV_DM_FLAGS_227,
682 RV_DM_FLAGS_228,
683 RV_DM_FLAGS_229,
684 RV_DM_FLAGS_230,
685 RV_DM_FLAGS_231,
686 RV_DM_FLAGS_232,
687 RV_DM_FLAGS_233,
688 RV_DM_FLAGS_234,
689 RV_DM_FLAGS_235,
690 RV_DM_FLAGS_236,
691 RV_DM_FLAGS_237,
692 RV_DM_FLAGS_238,
693 RV_DM_FLAGS_239,
694 RV_DM_FLAGS_240,
695 RV_DM_FLAGS_241,
696 RV_DM_FLAGS_242,
697 RV_DM_FLAGS_243,
698 RV_DM_FLAGS_244,
699 RV_DM_FLAGS_245,
700 RV_DM_FLAGS_246,
701 RV_DM_FLAGS_247,
702 RV_DM_FLAGS_248,
703 RV_DM_FLAGS_249,
704 RV_DM_FLAGS_250,
705 RV_DM_FLAGS_251,
706 RV_DM_FLAGS_252,
707 RV_DM_FLAGS_253,
708 RV_DM_FLAGS_254,
709 RV_DM_FLAGS_255
710 } rv_dm_mem_id_e;
711
712 // Register width information to check illegal writes for
mem interface
713 parameter logic [3:0] RV_DM_MEM_PERMIT [281] = '{
714     4'b 0001, // index[ 0] RV_DM_HALTED
715     4'b 0001, // index[ 1] RV_DM_GOING
716     4'b 0001, // index[ 2] RV_DM_RESUMING
717     4'b 0001, // index[ 3] RV_DM_EXCEPTION
718     4'b 1111, // index[ 4] RV_DM_WHERETO
719     4'b 1111, // index[ 5] RV_DM_ABSTRACTCMD_0
720     4'b 1111, // index[ 6] RV_DM_ABSTRACTCMD_1
721     4'b 1111, // index[ 7] RV_DM_ABSTRACTCMD_2
722     4'b 1111, // index[ 8] RV_DM_ABSTRACTCMD_3
723     4'b 1111, // index[ 9] RV_DM_ABSTRACTCMD_4
724     4'b 1111, // index[10] RV_DM_ABSTRACTCMD_5
725     4'b 1111, // index[11] RV_DM_ABSTRACTCMD_6
726     4'b 1111, // index[12] RV_DM_ABSTRACTCMD_7
727     4'b 1111, // index[13] RV_DM_ABSTRACTCMD_8
728     4'b 1111, // index[14] RV_DM_ABSTRACTCMD_9
729     4'b 1111, // index[15] RV_DM_PROGRAM_BUFFER_0
730     4'b 1111, // index[16] RV_DM_PROGRAM_BUFFER_1
731     4'b 1111, // index[17] RV_DM_PROGRAM_BUFFER_2
732     4'b 1111, // index[18] RV_DM_PROGRAM_BUFFER_3
733     4'b 1111, // index[19] RV_DM_PROGRAM_BUFFER_4
734     4'b 1111, // index[20] RV_DM_PROGRAM_BUFFER_5
735     4'b 1111, // index[21] RV_DM_PROGRAM_BUFFER_6
736     4'b 1111, // index[22] RV_DM_PROGRAM_BUFFER_7
737     4'b 1111, // index[23] RV_DM_DATAADDR_0
738     4'b 1111, // index[24] RV_DM_DATAADDR_1
739     4'b 1111, // index[25] RV_DM_FLAGS_0
740     4'b 1111, // index[26] RV_DM_FLAGS_1
741     4'b 1111, // index[27] RV_DM_FLAGS_2
742     4'b 1111, // index[28] RV_DM_FLAGS_3
743     4'b 1111, // index[29] RV_DM_FLAGS_4
744     4'b 1111, // index[30] RV_DM_FLAGS_5
745     4'b 1111, // index[31] RV_DM_FLAGS_6
746     4'b 1111, // index[32] RV_DM_FLAGS_7
747     4'b 1111, // index[33] RV_DM_FLAGS_8
748     4'b 1111, // index[34] RV_DM_FLAGS_9
749     4'b 1111, // index[35] RV_DM_FLAGS_10
750     4'b 1111, // index[36] RV_DM_FLAGS_11
751     4'b 1111, // index[37] RV_DM_FLAGS_12
752     4'b 1111, // index[38] RV_DM_FLAGS_13
753     4'b 1111, // index[39] RV_DM_FLAGS_14
754     4'b 1111, // index[40] RV_DM_FLAGS_15
755     4'b 1111, // index[41] RV_DM_FLAGS_16
756     4'b 1111, // index[42] RV_DM_FLAGS_17
757     4'b 1111, // index[43] RV_DM_FLAGS_18
758     4'b 1111, // index[44] RV_DM_FLAGS_19
759     4'b 1111, // index[45] RV_DM_FLAGS_20
760     4'b 1111, // index[46] RV_DM_FLAGS_21
761     4'b 1111, // index[47] RV_DM_FLAGS_22
762     4'b 1111, // index[48] RV_DM_FLAGS_23
763     4'b 1111, // index[49] RV_DM_FLAGS_24
764     4'b 1111, // index[50] RV_DM_FLAGS_25
765     4'b 1111, // index[51] RV_DM_FLAGS_26
766     4'b 1111, // index[52] RV_DM_FLAGS_27
767     4'b 1111, // index[53] RV_DM_FLAGS_28
768     4'b 1111, // index[54] RV_DM_FLAGS_29
769     4'b 1111, // index[55] RV_DM_FLAGS_30
770     4'b 1111, // index[56] RV_DM_FLAGS_31
771     4'b 1111, // index[57] RV_DM_FLAGS_32
772     4'b 1111, // index[58] RV_DM_FLAGS_33
773     4'b 1111, // index[59] RV_DM_FLAGS_34
774     4'b 1111, // index[60] RV_DM_FLAGS_35
775     4'b 1111, // index[61] RV_DM_FLAGS_36
776     4'b 1111, // index[62] RV_DM_FLAGS_37
777     4'b 1111, // index[63] RV_DM_FLAGS_38
778     4'b 1111, // index[64] RV_DM_FLAGS_39
779     4'b 1111, // index[65] RV_DM_FLAGS_40
780     4'b 1111, // index[66] RV_DM_FLAGS_41
781     4'b 1111, // index[67] RV_DM_FLAGS_42
782     4'b 1111, // index[68] RV_DM_FLAGS_43
783     4'b 1111, // index[69] RV_DM_FLAGS_44
784     4'b 1111, // index[70] RV_DM_FLAGS_45
785     4'b 1111, // index[71] RV_DM_FLAGS_46
786     4'b 1111, // index[72] RV_DM_FLAGS_47
787     4'b 1111, // index[73] RV_DM_FLAGS_48
788     4'b 1111, // index[74] RV_DM_FLAGS_49
789     4'b 1111, // index[75] RV_DM_FLAGS_50
790     4'b 1111, // index[76] RV_DM_FLAGS_51
791     4'b 1111, // index[77] RV_DM_FLAGS_52
792     4'b 1111, // index[78] RV_DM_FLAGS_53
793     4'b 1111, // index[79] RV_DM_FLAGS_54
794     4'b 1111, // index[80] RV_DM_FLAGS_55
795     4'b 1111, // index[81] RV_DM_FLAGS_56
796     4'b 1111, // index[82] RV_DM_FLAGS_57
797     4'b 1111, // index[83] RV_DM_FLAGS_58
798     4'b 1111, // index[84] RV_DM_FLAGS_59
799     4'b 1111, // index[85] RV_DM_FLAGS_60
800     4'b 1111, // index[86] RV_DM_FLAGS_61
801     4'b 1111, // index[87] RV_DM_FLAGS_62
802     4'b 1111, // index[88] RV_DM_FLAGS_63
803     4'b 1111, // index[89] RV_DM_FLAGS_64
804     4'b 1111, // index[90] RV_DM_FLAGS_65
805     4'b 1111, // index[91] RV_DM_FLAGS_66
806     4'b 1111, // index[92] RV_DM_FLAGS_67
807     4'b 1111, // index[93] RV_DM_FLAGS_68
808     4'b 1111, // index[94] RV_DM_FLAGS_69
809     4'b 1111, // index[95] RV_DM_FLAGS_70
810     4'b 1111, // index[96] RV_DM_FLAGS_71
811     4'b 1111, // index[97] RV_DM_FLAGS_72
812     4'b 1111, // index[98] RV_DM_FLAGS_73
813     4'b 1111, // index[99] RV_DM_FLAGS_74
814     4'b 1111, // index[100] RV_DM_FLAGS_75
815     4'b 1111, // index[101] RV_DM_FLAGS_76
816     4'b 1111, // index[102] RV_DM_FLAGS_77
817     4'b 1111, // index[103] RV_DM_FLAGS_78
818     4'b 1111, // index[104] RV_DM_FLAGS_79
819     4'b 1111, // index[105] RV_DM_FLAGS_80
820     4'b 1111, // index[106] RV_DM_FLAGS_81
821     4'b 1111, // index[107] RV_DM_FLAGS_82
822     4'b 1111, // index[108] RV_DM_FLAGS_83

```

```

823 4'b 1111, // index[109] RV_DM_FLAGS_84
824 4'b 1111, // index[110] RV_DM_FLAGS_85
825 4'b 1111, // index[111] RV_DM_FLAGS_86
826 4'b 1111, // index[112] RV_DM_FLAGS_87
827 4'b 1111, // index[113] RV_DM_FLAGS_88
828 4'b 1111, // index[114] RV_DM_FLAGS_89
829 4'b 1111, // index[115] RV_DM_FLAGS_90
830 4'b 1111, // index[116] RV_DM_FLAGS_91
831 4'b 1111, // index[117] RV_DM_FLAGS_92
832 4'b 1111, // index[118] RV_DM_FLAGS_93
833 4'b 1111, // index[119] RV_DM_FLAGS_94
834 4'b 1111, // index[120] RV_DM_FLAGS_95
835 4'b 1111, // index[121] RV_DM_FLAGS_96
836 4'b 1111, // index[122] RV_DM_FLAGS_97
837 4'b 1111, // index[123] RV_DM_FLAGS_98
838 4'b 1111, // index[124] RV_DM_FLAGS_99
839 4'b 1111, // index[125] RV_DM_FLAGS_100
840 4'b 1111, // index[126] RV_DM_FLAGS_101
841 4'b 1111, // index[127] RV_DM_FLAGS_102
842 4'b 1111, // index[128] RV_DM_FLAGS_103
843 4'b 1111, // index[129] RV_DM_FLAGS_104
844 4'b 1111, // index[130] RV_DM_FLAGS_105
845 4'b 1111, // index[131] RV_DM_FLAGS_106
846 4'b 1111, // index[132] RV_DM_FLAGS_107
847 4'b 1111, // index[133] RV_DM_FLAGS_108
848 4'b 1111, // index[134] RV_DM_FLAGS_109
849 4'b 1111, // index[135] RV_DM_FLAGS_110
850 4'b 1111, // index[136] RV_DM_FLAGS_111
851 4'b 1111, // index[137] RV_DM_FLAGS_112
852 4'b 1111, // index[138] RV_DM_FLAGS_113
853 4'b 1111, // index[139] RV_DM_FLAGS_114
854 4'b 1111, // index[140] RV_DM_FLAGS_115
855 4'b 1111, // index[141] RV_DM_FLAGS_116
856 4'b 1111, // index[142] RV_DM_FLAGS_117
857 4'b 1111, // index[143] RV_DM_FLAGS_118
858 4'b 1111, // index[144] RV_DM_FLAGS_119
859 4'b 1111, // index[145] RV_DM_FLAGS_120
860 4'b 1111, // index[146] RV_DM_FLAGS_121
861 4'b 1111, // index[147] RV_DM_FLAGS_122
862 4'b 1111, // index[148] RV_DM_FLAGS_123
863 4'b 1111, // index[149] RV_DM_FLAGS_124
864 4'b 1111, // index[150] RV_DM_FLAGS_125
865 4'b 1111, // index[151] RV_DM_FLAGS_126
866 4'b 1111, // index[152] RV_DM_FLAGS_127
867 4'b 1111, // index[153] RV_DM_FLAGS_128
868 4'b 1111, // index[154] RV_DM_FLAGS_129
869 4'b 1111, // index[155] RV_DM_FLAGS_130
870 4'b 1111, // index[156] RV_DM_FLAGS_131
871 4'b 1111, // index[157] RV_DM_FLAGS_132
872 4'b 1111, // index[158] RV_DM_FLAGS_133
873 4'b 1111, // index[159] RV_DM_FLAGS_134
874 4'b 1111, // index[160] RV_DM_FLAGS_135
875 4'b 1111, // index[161] RV_DM_FLAGS_136
876 4'b 1111, // index[162] RV_DM_FLAGS_137
877 4'b 1111, // index[163] RV_DM_FLAGS_138
878 4'b 1111, // index[164] RV_DM_FLAGS_139
879 4'b 1111, // index[165] RV_DM_FLAGS_140
880 4'b 1111, // index[166] RV_DM_FLAGS_141
881 4'b 1111, // index[167] RV_DM_FLAGS_142
882 4'b 1111, // index[168] RV_DM_FLAGS_143
883 4'b 1111, // index[169] RV_DM_FLAGS_144
884 4'b 1111, // index[170] RV_DM_FLAGS_145
885 4'b 1111, // index[171] RV_DM_FLAGS_146
886 4'b 1111, // index[172] RV_DM_FLAGS_147
887 4'b 1111, // index[173] RV_DM_FLAGS_148
888 4'b 1111, // index[174] RV_DM_FLAGS_149
889 4'b 1111, // index[175] RV_DM_FLAGS_150
890 4'b 1111, // index[176] RV_DM_FLAGS_151
891 4'b 1111, // index[177] RV_DM_FLAGS_152
892 4'b 1111, // index[178] RV_DM_FLAGS_153
893 4'b 1111, // index[179] RV_DM_FLAGS_154
894 4'b 1111, // index[180] RV_DM_FLAGS_155
895 4'b 1111, // index[181] RV_DM_FLAGS_156
896 4'b 1111, // index[182] RV_DM_FLAGS_157
897 4'b 1111, // index[183] RV_DM_FLAGS_158
898 4'b 1111, // index[184] RV_DM_FLAGS_159
899 4'b 1111, // index[185] RV_DM_FLAGS_160
900 4'b 1111, // index[186] RV_DM_FLAGS_161
901 4'b 1111, // index[187] RV_DM_FLAGS_162
902 4'b 1111, // index[188] RV_DM_FLAGS_163
903 4'b 1111, // index[189] RV_DM_FLAGS_164
904 4'b 1111, // index[190] RV_DM_FLAGS_165
905 4'b 1111, // index[191] RV_DM_FLAGS_166
906 4'b 1111, // index[192] RV_DM_FLAGS_167
907 4'b 1111, // index[193] RV_DM_FLAGS_168
908 4'b 1111, // index[194] RV_DM_FLAGS_169
909 4'b 1111, // index[195] RV_DM_FLAGS_170
910 4'b 1111, // index[196] RV_DM_FLAGS_171
911 4'b 1111, // index[197] RV_DM_FLAGS_172
912 4'b 1111, // index[198] RV_DM_FLAGS_173
913 4'b 1111, // index[199] RV_DM_FLAGS_174
914 4'b 1111, // index[200] RV_DM_FLAGS_175
915 4'b 1111, // index[201] RV_DM_FLAGS_176
916 4'b 1111, // index[202] RV_DM_FLAGS_177
917 4'b 1111, // index[203] RV_DM_FLAGS_178
918 4'b 1111, // index[204] RV_DM_FLAGS_179
919 4'b 1111, // index[205] RV_DM_FLAGS_180
920 4'b 1111, // index[206] RV_DM_FLAGS_181
921 4'b 1111, // index[207] RV_DM_FLAGS_182
922 4'b 1111, // index[208] RV_DM_FLAGS_183
923 4'b 1111, // index[209] RV_DM_FLAGS_184

```

```

924 4'b 1111, // index[210] RV_DM_FLAGS_185
925 4'b 1111, // index[211] RV_DM_FLAGS_186
926 4'b 1111, // index[212] RV_DM_FLAGS_187
927 4'b 1111, // index[213] RV_DM_FLAGS_188
928 4'b 1111, // index[214] RV_DM_FLAGS_189
929 4'b 1111, // index[215] RV_DM_FLAGS_190
930 4'b 1111, // index[216] RV_DM_FLAGS_191
931 4'b 1111, // index[217] RV_DM_FLAGS_192
932 4'b 1111, // index[218] RV_DM_FLAGS_193
933 4'b 1111, // index[219] RV_DM_FLAGS_194
934 4'b 1111, // index[220] RV_DM_FLAGS_195
935 4'b 1111, // index[221] RV_DM_FLAGS_196
936 4'b 1111, // index[222] RV_DM_FLAGS_197
937 4'b 1111, // index[223] RV_DM_FLAGS_198
938 4'b 1111, // index[224] RV_DM_FLAGS_199
939 4'b 1111, // index[225] RV_DM_FLAGS_200
940 4'b 1111, // index[226] RV_DM_FLAGS_201
941 4'b 1111, // index[227] RV_DM_FLAGS_202
942 4'b 1111, // index[228] RV_DM_FLAGS_203
943 4'b 1111, // index[229] RV_DM_FLAGS_204
944 4'b 1111, // index[230] RV_DM_FLAGS_205
945 4'b 1111, // index[231] RV_DM_FLAGS_206
946 4'b 1111, // index[232] RV_DM_FLAGS_207
947 4'b 1111, // index[233] RV_DM_FLAGS_208
948 4'b 1111, // index[234] RV_DM_FLAGS_209
949 4'b 1111, // index[235] RV_DM_FLAGS_210
950 4'b 1111, // index[236] RV_DM_FLAGS_211
951 4'b 1111, // index[237] RV_DM_FLAGS_212
952 4'b 1111, // index[238] RV_DM_FLAGS_213
953 4'b 1111, // index[239] RV_DM_FLAGS_214
954 4'b 1111, // index[240] RV_DM_FLAGS_215
955 4'b 1111, // index[241] RV_DM_FLAGS_216
956 4'b 1111, // index[242] RV_DM_FLAGS_217
957 4'b 1111, // index[243] RV_DM_FLAGS_218
958 4'b 1111, // index[244] RV_DM_FLAGS_219
959 4'b 1111, // index[245] RV_DM_FLAGS_220
960 4'b 1111, // index[246] RV_DM_FLAGS_221
961 4'b 1111, // index[247] RV_DM_FLAGS_222
962 4'b 1111, // index[248] RV_DM_FLAGS_223
963 4'b 1111, // index[249] RV_DM_FLAGS_224
964 4'b 1111, // index[250] RV_DM_FLAGS_225
965 4'b 1111, // index[251] RV_DM_FLAGS_226
966 4'b 1111, // index[252] RV_DM_FLAGS_227
967 4'b 1111, // index[253] RV_DM_FLAGS_228
968 4'b 1111, // index[254] RV_DM_FLAGS_229
969 4'b 1111, // index[255] RV_DM_FLAGS_230
970 4'b 1111, // index[256] RV_DM_FLAGS_231
971 4'b 1111, // index[257] RV_DM_FLAGS_232
972 4'b 1111, // index[258] RV_DM_FLAGS_233
973 4'b 1111, // index[259] RV_DM_FLAGS_234
974 4'b 1111, // index[260] RV_DM_FLAGS_235
975 4'b 1111, // index[261] RV_DM_FLAGS_236
976 4'b 1111, // index[262] RV_DM_FLAGS_237
977 4'b 1111, // index[263] RV_DM_FLAGS_238
978 4'b 1111, // index[264] RV_DM_FLAGS_239
979 4'b 1111, // index[265] RV_DM_FLAGS_240
980 4'b 1111, // index[266] RV_DM_FLAGS_241
981 4'b 1111, // index[267] RV_DM_FLAGS_242
982 4'b 1111, // index[268] RV_DM_FLAGS_243
983 4'b 1111, // index[269] RV_DM_FLAGS_244
984 4'b 1111, // index[270] RV_DM_FLAGS_245
985 4'b 1111, // index[271] RV_DM_FLAGS_246
986 4'b 1111, // index[272] RV_DM_FLAGS_247
987 4'b 1111, // index[273] RV_DM_FLAGS_248
988 4'b 1111, // index[274] RV_DM_FLAGS_249
989 4'b 1111, // index[275] RV_DM_FLAGS_250
990 4'b 1111, // index[276] RV_DM_FLAGS_251
991 4'b 1111, // index[277] RV_DM_FLAGS_252
992 4'b 1111, // index[278] RV_DM_FLAGS_253
993 4'b 1111, // index[279] RV_DM_FLAGS_254
994 4'b 1111, // index[280] RV_DM_FLAGS_255
995 };
996
997 // Window parameters for dbg interface
998 parameter logic [DbgAw-1:0] RV_DM_DBG_OFFSET = 9'h 0;
999 parameter int unsigned RV_DM_DBG_SIZE = 'h 200;
1000 parameter int unsigned RV_DM_DBG_IDX = 0;
1001
1002 endpackage

```

7. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_regs_reg_top.sv (363 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"
8
9 module rv_dm_regs_reg_top
10 # (
11     parameter bit EnableRacl = 1'b0,
12     parameter bit RaclErrorRsp = 1'b1,
13     parameter top_racl_pkg::racl_policy_sel_t RaclPoli-
  cySelVec[rv_dm_reg_pkg::NumRegsRegs] =
14         '{rv_dm_reg_pkg::NumRegsRegs{0}}

```



```

15 ) (
16 input clk_i,
17 input rst_ni,
18 input tlul_pkg::tl_h2d_t tl_i,
19 output tlul_pkg::tl_d2h_t tl_o,
20 // To HW
21 output rv_dm_reg_pkg::rv_dm_regs_reg2hw_t reg2hw, // Write
22
23 // RACL interface
24 input top_racl_pkg::racl_policy_vec_t racl_policies_i,
25 output top_racl_pkg::racl_error_log_t racl_error_o,
26
27 // Integrity check errors
28 output logic intg_err_o
29 );
30
31 import rv_dm_reg_pkg::* ;
32
33 localparam int AW = 4;
34 localparam int DW = 32;
35 localparam int DBW = DW/8; // Byte Width
36
37 // register signals
38 logic reg_we;
39 logic reg_re;
40 logic [AW-1:0] reg_addr;
41 logic [DW-1:0] reg_wdata;
42 logic [DBW-1:0] reg_be;
43 logic [DW-1:0] reg_rdata;
44 logic reg_error;
45
46 logic addrmiss, wr_err;
47
48 logic [DW-1:0] reg_rdata_next;
49 logic reg_busy;
50
51 tlul_pkg::tl_h2d_t tl_reg_h2d;
52 tlul_pkg::tl_d2h_t tl_reg_d2h;
53
54 // incoming payload check
55 logic intg_err;
56 tlul_cmd_intg_chk u_chk (
57     .tl_i(tl_i),
58     .err_o(intg_err)
59 );
60
61 // also check for spurious write enables
62 logic reg_we_err;
63 logic [2:0] reg_we_check;
64 prim_reg_we_check #(
65     .OneHotWidth(3)
66 ) u_prim_reg_we_check (
67     .clk_i(clk_i),
68     .rst_ni(rst_ni),
69     .oh_i (reg_we_check),
70     .en_i (reg_we && !addrmiss),
71     .err_o (reg_we_err)
72 );
73
74 logic err_q;
75 always_ff @(posedge clk_i or negedge rst_ni) begin
76     if (!rst_ni) begin
77         err_q <= '0;
78     end else if (intg_err || reg_we_err) begin
79         err_q <= 1'b1;
80     end
81 end
82
83 // integrity error output is permanent and should be used
84 for alert generation
85 // register errors are transactional
86 assign intg_err_o = err_q | intg_err | reg_we_err;
87
88 // outgoing integrity generation
89 tlul_pkg::tl_d2h_t tl_o_pre;
90 tlul_rsp_intg_gen #(
91     .EnableRspIntgGen(1),
92     .EnableDataIntgGen(1)
93 ) u_rsp_intg_gen (
94     .tl_i(tl_o_pre),
95     .tl_o(tl_o)
96 );
97
98 assign tl_reg_h2d = tl_i;
99 assign tl_o_pre = tl_reg_d2h;
100
101 tlul_adapter_reg #(
102     .RegAw(AW),
103     .RegDw(DW),
104     .EnableDataIntgGen(0)
105 ) u_reg_if (
106     .clk_i (clk_i),
107     .rst_ni (rst_ni),
108
109     .tl_i (tl_reg_h2d),
110     .tl_o (tl_reg_d2h),
111
112     .en_ifetch_i(prim_mubi_pkg::MuBi4False),
113     .intg_error_o(),
114

```

```

115     .we_o (reg_we),
116     .re_o (reg_re),
117     .addr_o (reg_addr),
118     .wdata_o (reg_wdata),
119     .be_o (reg_be),
120     .busy_i (reg_busy),
121     .rdata_i (reg_rdata),
122     // Translate RACL error to TLUL error if enabled
123     .error_i (reg_error | (RaclErrorRsp & racl_error_o.valid))
124 );
125
126 // cdc oversampling signals
127
128 assign reg_rdata = reg_rdata_next ;
129 assign reg_error = addrmiss | wr_err | intg_err;
130
131 // Define SW related signals
132 // Format: <reg>_<field>_<wd|we|qs>
133 // or <reg>_<wd|we|qs> if field == 1 or 0
134 logic alert_test_we;
135 logic alert_test_wd;
136 logic late_debug_enable_regwen_we;
137 logic late_debug_enable_regwen_qs;
138 logic late_debug_enable_regwen_wd;
139 logic late_debug_enable_we;
140 logic [31:0] late_debug_enable_qs;
141 logic [31:0] late_debug_enable_wd;
142
143 // Register instances
144 // R[alert_test]: V(True)
145 logic alert_test_qe;
146 logic [0:0] alert_test_flds_we;
147 assign alert_test_qe = &alert_test_flds_we;
148 prim_subreg_ext #(
149     .DW (1)
150 ) u_alert_test (
151     .re (1'b0),
152     .we (alert_test_we),
153     .wd (alert_test_wd),
154     .d ('0),
155     .qre (),
156     .qe (alert_test_flds_we[0]),
157     .q (reg2hw.alert_test.q),
158     .ds (),
159     .qs ()
160 );
161 assign reg2hw.alert_test.qe = alert_test_qe;
162
163 // R[late_debug_enable_regwen]: V(False)
164 prim_subreg #(
165     .DW (1),
166     .SwAccess(prim_subreg_pkg::SwAccessW0C),
167     .RESVAL (1'h1),
168     .Mubi (1'b0)
169 ) u_late_debug_enable_regwen (
170     .clk_i (clk_i),
171     .rst_ni (rst_ni),
172
173 // from register interface
174     .we (late_debug_enable_regwen_we),
175     .wd (late_debug_enable_regwen_wd),
176
177 // from internal hardware
178     .de (1'b0),
179     .d ('0),
180
181 // to internal hardware
182     .qe (),
183     .q (),
184     .ds (),
185
186 // to register interface (read)
187     .qs (late_debug_enable_regwen_qs)
188 );
189
190 // R[late_debug_enable]: V(False)
191 // Create REGWEN-gated WE signal
192 logic late_debug_enable_gated_we;
193 assign late_debug_enable_gated_we = late_debug_enable_we &
194 late_debug_enable_regwen_qs;
195 prim_subreg #(
196     .DW (32),
197     .SwAccess(prim_subreg_pkg::SwAccessRW),
198     .RESVAL (32'h69696969),
199     .Mubi (1'b1)
200 ) u_late_debug_enable (
201     .clk_i (clk_i),
202     .rst_ni (rst_ni),
203
204 // from register interface
205     .we (late_debug_enable_gated_we),
206     .wd (late_debug_enable_wd),
207
208 // from internal hardware
209     .de (1'b0),
210     .d ('0),
211
212 // to internal hardware
213     .qe (),
214

```

```

215 .q      (reg2hw.late_debug_enable.q),
216 .ds      (,
217
218 // to register interface (read)
219 .qs      (late_debug_enable_qs)
220 );
221
222
223
224 logic [2:0] addr_hit;
225 top_racl_pkg::racL_role_vec_t racL_role_vec;
226 top_racl_pkg::racL_role_t racL_role;
227
228 logic [2:0] racL_addr_hit_read;
229 logic [2:0] racL_addr_hit_write;
230
231 if (EnableRacl) begin : gen_racl_role_logic
232 // Retrieve RACL role from user bits and one-hot encode
that for the comparison bitmap
233 assign racL_role =
top_racl_pkg::tlul_extract_racl_role_bits(tl_i.a_user.rsvd);
234
235 prim_onehot_enc #(
236 .OneHotWidth( $bits(top_racl_pkg::racL_role_vec_t) )
237 ) u_racl_role_encode (
238 .in_i ( racL_role ),
239 .en_i ( 1'b1 ),
240 .out_o( racL_role_vec )
241 );
242 end else begin : gen_no_racl_role_logic
243 assign racL_role = '0;
244 assign racL_role_vec = '0;
245 end
246
247 always_comb begin
248 racL_addr_hit_read = '0;
249 racL_addr_hit_write = '0;
250 addr_hit[0] = (reg_addr == RV_DM_ALERT_TEST_OFFSET);
251 addr_hit[1] = (reg_addr ==
RV_DM_LATE_DEBUG_ENABLE_REGWEN_OFFSET);
252 addr_hit[2] = (reg_addr ==
RV_DM_LATE_DEBUG_ENABLE_OFFSET);
253
254 if (EnableRacl) begin : gen_racl_hit
255 for (int unsigned slice_idx = 0; slice_idx < 3;
slice_idx++) begin
256 racL_addr_hit_read[slice_idx] =
257 addr_hit[slice_idx] & (|
(racL_policies_i[RaclPolicySelVec[slice_idx]].read_perm
& racL_role_vec));
258
259 racL_addr_hit_write[slice_idx] =
260 addr_hit[slice_idx] & (|
(racL_policies_i[RaclPolicySelVec[slice_idx]].write_perm
& racL_role_vec));
261
262 end
263 end else begin : gen_no_racl
264 racL_addr_hit_read = addr_hit;
265 racL_addr_hit_write = addr_hit;
266 end
267 end
268
269 assign addrmiss = (reg_re || reg_we) ? ~|addr_hit : 1'b0 ;
270 // A valid address hit, access, but failed the RACL check
271 assign racL_error_o.valid = |addr_hit & ((reg_re & ~|
racL_addr_hit_read) |
272 (reg_we & ~|
racL_addr_hit_write));
273 assign racL_error_o.request_address =
top_pkg::TL_AW'(reg_addr);
274 assign racL_error_o.ctn_uid = racL_role;
275 assign racL_error_o.overflow = 1'b0;
276
277 if (EnableRacl) begin : gen_racl_log
278 assign racL_error_o.ctn_uid =
top_racl_pkg::tlul_extract_ctn_uid_bits(tl_i.a_user.rsvd);
279 assign racL_error_o.read_access = tl_i.a_opcode ==
tlul_pkg::Get;
280 end else begin : gen_no_racl_log
281 assign racL_error_o.ctn_uid = '0;
282 assign racL_error_o.read_access = 1'b0;
283 end
284
285 // Check sub-word write is permitted
286 always_comb begin
287 wr_err = (reg_we &
288 ((racL_addr_hit_write[0] & (|

```

```

(RV_DM_REGS_PERMIT[0] & ~reg_be))) |
289 (racL_addr_hit_write[1] & (|
(RV_DM_REGS_PERMIT[1] & ~reg_be))) |
290 (racL_addr_hit_write[2] & (|
(RV_DM_REGS_PERMIT[2] & ~reg_be)))));
291 end
292
293 // Generate write-enables
294 assign alert_test_we = racL_addr_hit_write[0] & reg_we & !
reg_error;
295
296 assign alert_test_wd = reg_wdata[0];
297 assign late_debug_enable_regwen_we = racL_addr_hit_write[1]
& reg_we & !reg_error;
298
299 assign late_debug_enable_regwen_wd = reg_wdata[0];
300 assign late_debug_enable_we = racL_addr_hit_write[2] &
reg_we & !reg_error;
301
302 assign late_debug_enable_wd = reg_wdata[31:0];
303
304 // Assign write-enables to checker logic vector.
305 always_comb begin
306 reg_we_check[0] = alert_test_we;
307 reg_we_check[1] = late_debug_enable_regwen_we;
308 reg_we_check[2] = late_debug_enable_gated_we;
309 end
310
311 // Read data return
312 always_comb begin
313 reg_rdata_next = '0;
314 unique case (1'b1)
315 racL_addr_hit_read[0]: begin
316 reg_rdata_next[0] = '0;
317 end
318
319 racL_addr_hit_read[1]: begin
320 reg_rdata_next[0] = late_debug_enable_regwen_qs;
321 end
322
323 racL_addr_hit_read[2]: begin
324 reg_rdata_next[31:0] = late_debug_enable_qs;
325 end
326
327 default: begin
328 reg_rdata_next = '1;
329 end
330 endcase
331 end
332
333 // shadow busy
334 logic shadow_busy;
335 assign shadow_busy = 1'b0;
336
337 // register busy
338 assign reg_busy = shadow_busy;
339
340 // Unused signal tieoff
341
342 // wdata / byte enable are not always fully used
343 // add a blanket unused statement to handle lint waivers
344 logic unused_wdata;
345 logic unused_be;
346 assign unused_wdata = ^reg_wdata;
347 assign unused_be = ^reg_be;
348 logic unused_policy_sel;
349 assign unused_policy_sel = ^racL_policies_i;
350
351 // Assertions for Register Interface
352 `ASSERT_PULSE(wePulse, reg_we, clk_i, !rst_ni)
353 `ASSERT_PULSE(rePulse, reg_re, clk_i, !rst_ni)
354
355 `ASSERT(reAfterRv, $rose(reg_re || reg_we) |>
tl_o_pre.d_valid, clk_i, !rst_ni)
356
357 `ASSERT(en2addrHit, (reg_we || reg_re) |> $one-
hot0(addr_hit), clk_i, !rst_ni)
358
359 // this is formulated as an assumption such that the FPV
testbenches do disprove this
360 // property by mistake
361 //`ASSUME(reqParity, tl_reg_h2d.a_valid |>
tl_reg_h2d.a_user.chk_en == tlul_pkg::CheckDis)
362
363 endmodule

```

TL-UL 버스 구조

What this is. TileLink Uncached Lightweight adapters, sockets, FIFOs and error responders.

Why it exists. Because a common bus is what makes IP reuse practical. One interface convention permits shared register generators, verification IP and address decoders across every peripheral.

Undergraduate review · EECS concepts. Bus protocols; handshaking; address decoding; arbitration; pipelining and throughput.

Graduate · Practice note. Bus adapters are not free. For a small peripheral the adapter can exceed the function in area. **Resource reports should state whether the adapter is included.**

28 files • 4,683 lines

API — sram2tlul.sv

Table 1056. lowRISC_opentitan/hw/ip/tlul/rtl/sram2tlul.sv — module / interface / package (1개)

이름	종류
sram2tlul	module

Table 1057. lowRISC_opentitan/hw/ip/tlul/rtl/sram2tlul.sv — parameter (3개)

파라미터	형/폭	기본값
SramAw	int	12
SramDw	int	32
TlBaseAddr	logic [top_pkg::TL_AW-1:0]	'h0 // Base address of SRAM request

Table 1058. lowRISC_opentitan/hw/ip/tlul/rtl/sram2tlul.sv — localparam (1개)

상수	형/폭	값
SRAM_DWB	int unsigned	\$clog2(SramDw/8)

Table 1059. lowRISC_opentitan/hw/ip/tlul/rtl/sram2tlul.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_o	output	tlul_pkg::tl_h2d_t
tl_i	input	tlul_pkg::tl_d2h_t
mem_req_i	input	
mem_write_i	input	
mem_addr_i	input	[SramAw-1:0]
mem_wdata_i	input	[SramDw-1:0]

포트	방향	형/폭
mem_rvalid_o	output	logic
mem_rdata_o	output	logic [SramDw-1:0]
mem_error_o	output	logic [1:0]

API — tlul_adapter_dmi.sv

Table 1060. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_dmi.sv — module / interface / package (1개)

이름	종류
tlul_adapter_dmi	module

Table 1061. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_dmi.sv — parameter (3개)

파라미터	형/폭	기본값
CmdIntgCheck	bit	1
EnableRspIntgGen	bit	1
EnableDataIntgGen	bit	1 // 1: Generate response data integrity

Table 1062. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_dmi.sv — localparam (2개)

상수	형/폭	값
IdW	int	\$bits(tl_h2d_i.a_source)
SzW	int	\$bits(tl_h2d_i.a_size)

Table 1063. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_dmi.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_h2d_i	input	tl_h2d_t
tl_d2h_o	output	tl_d2h_t
intg_error_o	output	logic
dmi_req_valid_o	output	logic
dmi_req_ready_i	input	logic
dmi_req_o	output	dm::dmi_req_t
dmi_resp_valid_i	input	logic
dmi_resp_ready_o	output	logic
dmi_resp_i	input	dm::dmi_resp_t

API — tlul_adapter_host.sv

Table 1064. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_host.sv — module / interface / package (1개)

이름	종류
tlul_adapter_host	module

Table 1065. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_host.sv — parameter (3개)

파라미터	형/폭	기본값
MAX_REQS	int unsigned	2
EnableDataIntgGen	bit	0
EnableRspDataIntgCheck	bit	0

Table 1066. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_host.sv — localparam (6개)

상수	형/폭	값
WordSize	int unsigned	$\$clog2(top_pkg::TL_DBW)$
ReqNumW	int	$\$clog2(MAX_REQS)$
MaxSource	int unsigned	$MAX_REQS - 1$
ReqNumOne	logic [ReqNumW-1:0]	$ReqNumW'(1'b1)$
OutstandingReqCntW	int	$(MAX_REQS == 2 ** \$clog2(MAX_REQS)) ? \$clog2(MAX_REQS) + 1 : ...$
OutstandingReqCntOne	logic [OutstandingReqCntW-1:0]	$OutstandingReqCntW'(1'b1)$

Table 1067. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_host.sv — 포트 (18개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
req_i	input	
gnt_o	output	logic
addr_i	input	logic [top_pkg::TL_AW-1:0]
we_i	input	logic
wdata_i	input	logic [top_pkg::TL_DW-1:0]
wdata_intg_i	input	logic [DataIntgWidth-1:0]
be_i	input	logic [top_pkg::TL_DBW-1:0]
instr_type_i	input	mubi4_t
user_rsvd_i	input	logic [RsvdWidth-1:0]
valid_o	output	logic
rdata_o	output	logic [top_pkg::TL_DW-1:0]
rdata_intg_o	output	logic [DataIntgWidth-1:0]
err_o	output	logic
intg_err_o	output	logic

포트	방향	형/폭
tl_o	output	tl_h2d_t
tl_i	input	tl_d2h_t

API — tlul_adapter_racl.sv

Table 1068. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_racl.sv — module / interface / package (1개)

이름	종류
tlul_adapter_racl	module

Table 1069. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_racl.sv — parameter (3개)

파라미터	형/폭	기본값
EnableRacl	bit	0
RaclErrorRsp	bit	EnableRacl
RaclPolicySelNumRanges	int	1 // Number of ranges with a RACL policy

Table 1070. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_racl.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_h2d_i	input	tlul_pkg::tl_h2d_t
tl_d2h_o	output	tlul_pkg::tl_d2h_t
tl_filtered_h2d_o	output	tlul_pkg::tl_h2d_t
tl_filtered_d2h_i	input	tlul_pkg::tl_d2h_t
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
racl_policy_sel_ranges_i	input	top_racl_pkg::racl_range_t [RaclPolicySelNumRanges...

API — tlul_adapter_reg.sv

Table 1071. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg.sv — module / interface / package (1개)

이름	종류
tlul_adapter_reg	module

Table 1072. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg.sv — parameter (6개)

파라미터	형/폭	기본값
CmdIntgCheck	bit	0
EnableRspIntgGen	bit	0
EnableDataIntgGen	bit	0
RegAw	int	8

파라미터	형/폭	기본값
RegDw	int	32
AccessLatency	int	0

Table 1073. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg.sv — localparam (3개)

상수	형/폭	값
RegBw	int	RegDw/8
IW	int	\$bits(tl_i.a_source)
SZW	int	\$bits(tl_i.a_size)

Table 1074. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg.sv — 포트 (14개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tl_h2d_t
tl_o	output	tl_d2h_t
en_ifetch_i	input	mubi4_t
intg_error_o	output	logic
re_o	output	logic
we_o	output	logic
addr_o	output	logic [RegAw-1:0]
wdata_o	output	logic [RegDw-1:0]
be_o	output	logic [RegBw-1:0]
busy_i	input	
rdata_i	input	[RegDw-1:0]
error_i	input	

API — tlul_adapter_reg_racl.sv

Table 1075. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg_racl.sv — module / interface / package (1개)

이름	종류
tlul_adapter_reg_racl	module

Table 1076. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg_racl.sv — parameter (9개)

파라미터	형/폭	기본값
CmdIntgCheck	bit	0
EnableRspIntgGen	bit	0
EnableDataIntgGen	bit	0

파라미터	형/폭	기본값
RegAw	int	8
RegDw	int	32
AccessLatency	int	0
EnableRacl	bit	0
RaclErrorRsp	bit	EnableRacl
RaclPolicySelVec	top_racl_pkg::racl_policy_sel_t	0

Table 1077. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg_racl.sv — localparam (1개)

상수	형/폭	값
RegBw	int	RegDw/8

Table 1078. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg_racl.sv — 포트 (16개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tl_h2d_t
tl_o	output	tl_d2h_t
en_ifetch_i	input	mubi4_t
intg_error_o	output	logic
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
re_o	output	logic
we_o	output	logic
addr_o	output	logic [RegAw-1:0]
wdata_o	output	logic [RegDw-1:0]
be_o	output	logic [RegBw-1:0]
busy_i	input	
rdata_i	input	[RegDw-1:0]
error_i	input	

API — tlul_adapter_shim.sv

Table 1079. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_shim.sv — module / interface / package (1개)

이름	종류
tlul_adapter_vh	module

Table 1080. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_shim.sv — parameter (8개)

파라미터	형/폭	기본값
ADDR_WIDTH	int	top_pkg::TL_AW
DATA_WIDTH	int	top_pkg::TL_DW
MASK_WIDTH	int	DATA_WIDTH >> 3
USER_WIDTH	int	32
ID_WIDTH	int	top_pkg::TL_AIW
VH_REGISTER_ADDRESS_OFFSET	bit [ADDR_WIDTH-1:0]	32'h0000_0100
EnableDataIntgGen	bit	1
EnableRspDataIntgCheck	bit	1

Table 1081. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_shim.sv — 포트 (28개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_o	output	tl_h2d_t
tl_i	input	tl_d2h_t
dv_i	input	logic
hld_o	output	logic
addr_i	input	logic [ADDR_WIDTH-1:0]
write_i	input	logic
wdata_i	input	logic [DATA_WIDTH-1:0]
wstrb_i	input	logic [MASK_WIDTH-1:0]
size_i	input	logic [2:0]
rdata_o	output	logic [DATA_WIDTH-1:0]
error_o	output	logic
last_i	input	logic
user_i	input	logic [USER_WIDTH-1:0]
id_i	input	logic [ID_WIDTH-1:0]
int_dv_o	output	logic
int_hld_i	input	logic
int_addr_o	output	logic [ADDR_WIDTH-1:0]
int_write_o	output	logic
int_wdata_o	output	logic [DATA_WIDTH-1:0]
int_wstrb_o	output	logic [MASK_WIDTH-1:0]
int_size_o	output	logic [2:0]
int_rdata_i	input	logic [DATA_WIDTH-1:0]

포트	방향	형/폭
int_error_i	input	logic
int_last_o	output	logic
int_user_o	output	logic [USER_WIDTH-1:0]
int_id_o	output	logic [ID_WIDTH-1:0]

API — tlul_adapter_sram.sv

Table 1082. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram.sv — module / interface / package (17개)

이름	종류
tlul_adapter_sram	module

Table 1083. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram.sv — parameter (15개)

파라미터	형/폭	기본값
SramAw	int	12
SramDw	int	32
SramDepth	int	2**SramAw
Outstanding	int	1
SramBusBankAW	int	12
ByteAccess	bit	1
ErrOnWrite	bit	0
ErrOnRead	bit	0
CmdIntgCheck	bit	0
EnableRspIntgGen	bit	0
EnableDataIntgGen	bit	0
EnableDataIntgPt	bit	0
SecFifoPtr	bit	0
EnableReadback	bit	0
DataXorAddr	bit	0

Table 1084. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram.sv — localparam (11개)

상수	형/폭	값
WidthMult	int	SramDw / top_pkg::TL_DW
IntgWidth	int	tlul_pkg::DataIntgWidth * WidthMult
DataOutW	int	EnableDataIntgPt ? SramDw + IntgWidth : SramDw
SramByte	int	SramDw/8
DataBitWidth	int	prim_util_pkg::vbits(SramByte)
WoffsetWidth	int	(SramByte == top_pkg::TL_DBW) ? 1 :

상수	형/폭	값
SramReqWidth	int	\$bits(sram_req_t)
SramReqFifoWidth	int	SramReqWidth + (DataXorAddr ? SramBusBankAW : 0)
ReqFifoWidth	int	\$bits(req_t)
RspFifoWidth	int	\$bits(rsp_t)
DataWidth	int	EnableDataIntgPt ? top_pkg::TL_DW + DataIntgWidth : top_pkg::...

Table 1085. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram.sv — 포트 (22개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tl_h2d_t
tl_o	output	tl_d2h_t
en_ifetch_i	input	mubi4_t
req_o	output	logic
req_type_o	output	mubi4_t
gnt_i	input	
we_o	output	logic
addr_o	output	logic [SramAw-1:0]
wdata_o	output	logic [DataOutW-1:0]
wmask_o	output	logic [DataOutW-1:0]
intg_error_o	output	logic
user_rsvd_o	output	logic [RsvdWidth-1:0]
rdata_i	input	[DataOutW-1:0]
rvalid_i	input	
rerror_i	input	[1:0]
compound_txn_in_progress_o	output	logic
readback_en_i	input	mubi4_t
readback_error_o	output	logic
wr_collision_i	input	logic
write_pending_i	input	logic

API — tlul_adapter_sram_racl.sv

Table 1086. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram_racl.sv — module / interface / package (1개)

이름	종류
tlul_adapter_sram_racl	module

Table 1087. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram_racl.sv — parameter (18개)

파라미터	형/폭	기본값
SramAw	int	12
SramDw	int	32
SramDepth	int	2**SramAw
Outstanding	int	1
SramBusBankAW	int	12
ByteAccess	bit	1
ErrOnWrite	bit	0
ErrOnRead	bit	0
CmdIntgCheck	bit	0
EnableRspIntgGen	bit	0
EnableDataIntgGen	bit	0
EnableDataIntgPt	bit	0
SecFifoPtr	bit	0
EnableReadback	bit	0
DataXorAddr	bit	0
EnableRacl	bit	0
RaclErrorRsp	bit	EnableRacl
RaclPolicySelNumRanges	int	1

Table 1088. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram_racl.sv — localparam (3개)

상수	형/폭	값
WidthMult	int	SramDw / top_pkg::TL_DW
IntgWidth	int	tlul_pkg::DataIntgWidth * WidthMult
DataOutW	int	EnableDataIntgPt ? SramDw + IntgWidth : SramDw

Table 1089. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram_racl.sv — 포트 (25개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tl_h2d_t
tl_o	output	tl_d2h_t
en_ifetch_i	input	mubi4_t
req_o	output	logic
req_type_o	output	mubi4_t
gnt_i	input	

포트	방향	형/폭
we_o	output	logic
addr_o	output	logic [SramAw-1:0]
wdata_o	output	logic [DataOutW-1:0]
wmask_o	output	logic [DataOutW-1:0]
intg_error_o	output	logic
user_rsvd_o	output	logic [RsvdWidth-1:0]
rdata_i	input	[DataOutW-1:0]
rvalid_i	input	
rerror_i	input	[1:0]
compound_txn_in_progress_o	output	logic
readback_en_i	input	mubi4_t
readback_error_o	output	logic
wr_collision_i	input	logic
write_pending_i	input	logic
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
racl_policy_sel_ranges_i	input	top_racl_pkg::racl_range_t [RaclPolicySelNumRanges...

API — tlul_adapter_vh.sv

Table 1090. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_vh.sv — module / interface / package (1개)

이름	종류
tlul_adapter_vh	module

Table 1091. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_vh.sv — parameter (8개)

파라미터	형/폭	기본값
ADDR_WIDTH	int	top_pkg::TL_AW
DATA_WIDTH	int	top_pkg::TL_DW
MASK_WIDTH	int	DATA_WIDTH >> 3
USER_WIDTH	int	32
ID_WIDTH	int	top_pkg::TL_AIW
VH_REGISTER_ADDRESS_OFFSET	bit [ADDR_WIDTH-1:0]	32'h0000_0100
EnableDataIntgGen	bit	1
EnableRspDataIntgCheck	bit	1

Table 1092. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_vh.sv — 포트 (28개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_o	output	tl_h2d_t
tl_i	input	tl_d2h_t
dv_i	input	logic
hld_o	output	logic
addr_i	input	logic [ADDR_WIDTH-1:0]
write_i	input	logic
wdata_i	input	logic [DATA_WIDTH-1:0]
wstrb_i	input	logic [MASK_WIDTH-1:0]
size_i	input	logic [2:0]
rdata_o	output	logic [DATA_WIDTH-1:0]
error_o	output	logic
last_i	input	logic
user_i	input	logic [USER_WIDTH-1:0]
id_i	input	logic [ID_WIDTH-1:0]
int_dv_o	output	logic
int_hld_i	input	logic
int_addr_o	output	logic [ADDR_WIDTH-1:0]
int_write_o	output	logic
int_wdata_o	output	logic [DATA_WIDTH-1:0]
int_wstrb_o	output	logic [MASK_WIDTH-1:0]
int_size_o	output	logic [2:0]
int_rdata_i	input	logic [DATA_WIDTH-1:0]
int_error_i	input	logic
int_last_o	output	logic
int_user_o	output	logic [USER_WIDTH-1:0]
int_id_o	output	logic [ID_WIDTH-1:0]

API — tlul_assert.sv

Table 1093. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_assert.sv — module / interface / package (1개)

이름	종류
tlul_assert	module

Table 1094. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_assert.sv — parameter (1개)

파라미터	형/폭	기본값
EndpointType		"Device" // can be either "Host" or "Device"

Table 1095. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_assert.sv — 포트 (4개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
h2d	input	tlul_pkg::tl_h2d_t
d2h	input	tlul_pkg::tl_d2h_t

Table 1096. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_assert.sv — #define / `define 매크로 (3개)

매크로	인자	정의
TLUL_COVER	(SEQ)	`COVER(`SEQ``_C, ``SEQ``_S, !clk_i, !rst_ni disable_sva)
TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED	(NAME)	\
TLUL_D_CHAN_CONTENT_CHANGED_WO_ACCEPTED	(NAME)	\

API — tlul_assert_multiple.sv**Table 1097. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_assert_multiple.sv — module / interface / package (1개)**

이름	종류
tlul_assert_multiple	module

Table 1098. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_assert_multiple.sv — parameter (2개)

파라미터	형/폭	기본값
N	int unsigned	2
EndpointType		"Device" // can be "Device" or "Host"

Table 1099. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_assert_multiple.sv — 포트 (2개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	

API — tlul_cmd_intg_chk.sv**Table 1100. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_cmd_intg_chk.sv — module / interface / package (1개)**

이름	종류
tlul_cmd_intg_chk	module

Table 1101. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_cmd_intg_chk.sv — 포트 (2개)

포트	방향	형/폭
tl_i	input	tl_h2d_t

포트	방향	형/폭
err_o	output	logic

API — tlul_cmd_intg_gen.sv

Table 1102. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_cmd_intg_gen.sv — module / interface / package (1개)

이름	종류
tlul_cmd_intg_gen	module

Table 1103. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_cmd_intg_gen.sv — parameter (1개)

파라미터	형/폭	기본값
EnableDataIntgGen	bit	1'b1

Table 1104. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_cmd_intg_gen.sv — 포트 (2개)

포트	방향	형/폭
tl_i	input	tl_h2d_t
tl_o	output	tl_h2d_t

API — tlul_data_integ_dec.sv

Table 1105. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_data_integ_dec.sv — module / interface / package (1개)

이름	종류
tlul_data_integ_dec	module

Table 1106. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_data_integ_dec.sv — 포트 (2개)

포트	방향	형/폭
data_intg_i	input	[DataMaxWidth+DataIntgWidth-1:0]
data_err_o	output	logic

API — tlul_data_integ_enc.sv

Table 1107. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_data_integ_enc.sv — module / interface / package (1개)

이름	종류
tlul_data_integ_enc	module

Table 1108. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_data_integ_enc.sv — 포트 (2개)

포트	방향	형/폭
data_i	input	[DataMaxWidth-1:0]
data_intg_o	output	logic [DataMaxWidth+DataIntgWidth-1:0]

API — tlul_err.sv

Table 1109. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_err.sv — module / interface / package (1개)

이름	종류
tlul_err	module

Table 1110. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_err.sv — localparam (6개)

상수	형/폭	값
IW	int	\$bits(tl_i.a_source)
SZW	int	\$bits(tl_i.a_size)
DW	int	\$bits(tl_i.a_data)
MW	int	\$bits(tl_i.a_mask)
SubAW	int	\$clog2(DW/8)
MaskOne	bit [MW-1:0]	1

Table 1111. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_err.sv — 포트 (4개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tl_h2d_t
err_o	output	logic

API — tlul_err_resp.sv

Table 1112. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_err_resp.sv — module / interface / package (1개)

이름	종류
tlul_err_resp	module

Table 1113. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_err_resp.sv — parameter (1개)

파라미터	형/폭	기본값
ReturnBlankResp	bit	0

Table 1114. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_err_resp.sv — 포트 (4개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_h_i	input	tlul_pkg::tl_h2d_t
tl_h_o	output	tlul_pkg::tl_d2h_t

API — tlul_fifo_async.sv

Table 1115. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_async.sv — module / interface / package (1개)

이름	종류
tlul_fifo_async	module

Table 1116. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_async.sv — parameter (2개)

파라미터	형/폭	기본값
ReqDepth	int unsigned	4
RspDepth	int unsigned	4

Table 1117. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_async.sv — localparam (2개)

상수	형/폭	값
REQFIFO_WIDTH	int unsigned	\$bits(tlul_pkg::tl_h2d_t)-2
RSPFIFO_WIDTH	int unsigned	\$bits(tlul_pkg::tl_d2h_t) -2

Table 1118. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_async.sv — 포트 (8개)

포트	방향	형/폭
clk_h_i	input	
rst_h_ni	input	
clk_d_i	input	
rst_d_ni	input	
tl_h_i	input	tlul_pkg::tl_h2d_t
tl_h_o	output	tlul_pkg::tl_d2h_t
tl_d_o	output	tlul_pkg::tl_h2d_t
tl_d_i	input	tlul_pkg::tl_d2h_t

API — tlul_fifo_sync.sv**Table 1119. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_sync.sv — module / interface / package (1개)**

이름	종류
tlul_fifo_sync	module

Table 1120. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_sync.sv — parameter (6개)

파라미터	형/폭	기본값
ReqPass	bit	1'b1
RspPass	bit	1'b1
ReqDepth	int unsigned	2
RspDepth	int unsigned	2
SpareReqW	int unsigned	1
SpareRspW	int unsigned	1

Table 1121. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_sync.sv — localparam (6개)

상수	형/폭	값
REQFIFO_INTG_WIDTH	int unsigned	tlul_pkg::H2DCmdIntgWidth + tlul_pkg::DataIntgWidth
REQFIFO_WIDTH	int unsigned	\$bits(tlul_pkg::tl_h2d_t) -2 + SpareReqW -

상수	형/폭	값
NumBufferBitsReqFifoIntg	int	\$bits({
RSPFIFO_INTG_WIDTH	int unsigned	\$bits(tlul_pkg::tl_d_user_t)
RSPFIFO_WIDTH	int unsigned	\$bits(tlul_pkg::tl_d2h_t) - 2 + SpareRspW -
NumBufferBitsRspFifoIntg	int	\$bits({

Table 1122. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_sync.sv — 포트 (10개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_h_i	input	tlul_pkg::tl_h2d_t
tl_h_o	output	tlul_pkg::tl_d2h_t
tl_d_o	output	tlul_pkg::tl_h2d_t
tl_d_i	input	tlul_pkg::tl_d2h_t
spare_req_i	input	[SpareReqW-1:0]
spare_req_o	output	[SpareReqW-1:0]
spare_rsp_i	input	[SpareRspW-1:0]
spare_rsp_o	output	[SpareRspW-1:0]

API — tlul_jtag_dtm.sv

Table 1123. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_jtag_dtm.sv — module / interface / package (1개)

이름	종류
tlul_jtag_dtm	module

Table 1124. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_jtag_dtm.sv — parameter (2개)

파라미터	형/폭	기본값
IdcodeValue	logic [31:0]	32'h00000001
NumDmiByteAbits	int unsigned	18

Table 1125. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_jtag_dtm.sv — 포트 (8개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
jtag_i	input	jtag_pkg::jtag_req_t
jtag_o	output	jtag_pkg::jtag_rsp_t
scan_rst_ni	input	
scanmode_i	input	prim_mubi_pkg::mubi4_t
tl_h2d_o	output	tlul_pkg::tl_h2d_t

포트	방향	형/폭
tl_d2h_i	input	tlul_pkg::tl_d2h_t

API — tlul_lc_gate.sv

Table 1126. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_lc_gate.sv — module / interface / package (1개)

이름	종류
tlul_lc_gate	module

Table 1127. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_lc_gate.sv — parameter (3개)

파라미터	형/폭	기본값
NumGatesPerDirection	int	2
ReturnBlankResp	bit	0
Outstanding	int unsigned	3

Table 1128. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_lc_gate.sv — localparam (1개)

상수	형/폭	값
StateWidth	int	9

Table 1129. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_lc_gate.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_h2d_i	input	tl_h2d_t
tl_d2h_o	output	tl_d2h_t
tl_h2d_o	output	tl_h2d_t
tl_d2h_i	input	tl_d2h_t
flush_req_i	input	
flush_ack_o	output	logic
resp_pending_o	output	logic
lc_en_i	input	lc_tx_t
err_o	output	logic

API — tlul_pkg.sv

Table 1130. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_pkg.sv — module / interface / package (1개)

이름	종류
tlul_pkg	package

Table 1131. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_pkg.sv — parameter (15개)

파라미터	형/폭	기본값
ArbiterImpl		"PPC"
H2DCmdMaxWidth	int	57
H2DCmdIntgWidth	int	7
H2DCmdFullWidth	int	H2DCmdMaxWidth + H2DCmdIntgWidth
D2HRspMaxWidth	int	57
D2HRspIntgWidth	int	7
D2HRspFullWidth	int	D2HRspMaxWidth + D2HRspIntgWidth
DataMaxWidth	int	32
DataIntgWidth	int	7
DataFullWidth	int	DataMaxWidth + DataIntgWidth
RsvdWidth	int	top_pkg::TL_AUW - prim_mubi_pkg::MuBi4Width -
DataWhenInstrError	logic [top_pkg::TL_DW-1:0]	'0
DataWhenError	logic [top_pkg::TL_DW-1:0]	{top_pkg::TL_DW{1'b1}}
TL_A_USER_DEFAULT	tl_a_user_t	'{
TL_D_USER_DEFAULT	tl_d_user_t	'{

Table 1132. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_pkg.sv — localparam (3개)

상수	형/폭	값
BlankedAData	logic [top_pkg::TL_DW-1:0]	{top_pkg::TL_DW{1'b1}}
TL_H2D_DEFAULT	tl_h2d_t	'{
TL_D2H_DEFAULT	tl_d2h_t	'{

API — tlul_request_loopback.sv

Table 1133. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_request_loopback.sv — module / interface / package (1개)

이름	종류
tlul_request_loopback	module

Table 1134. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_request_loopback.sv — parameter (1개)

파라미터	형/폭	기본값
ErrorRsp	bit	1 // 1: Return TLUL error on on squash

Table 1135. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_request_loopback.sv — 포트 (7개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
squash_req_i	input	logic

포트	방향	형/폭
tl_h2d_i	input	tlul_pkg::tl_h2d_t
tl_d2h_o	output	tlul_pkg::tl_d2h_t
tl_h2d_o	output	tlul_pkg::tl_h2d_t
target	input	tlul_pkg::tl_d2h_t tl_d2h_i // Response from the

API — tlul_rsp_intg_chk.sv

Table 1136. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_rsp_intg_chk.sv — module / interface / package (1개)

이름	종류
tlul_rsp_intg_chk	module

Table 1137. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_rsp_intg_chk.sv — parameter (1개)

파라미터	형/폭	기본값
EnableRspDataIntgCheck	bit	0

Table 1138. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_rsp_intg_chk.sv — 포트 (2개)

포트	방향	형/폭
tl_i	input	tl_d2h_t
err_o	output	logic

API — tlul_rsp_intg_gen.sv

Table 1139. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_rsp_intg_gen.sv — module / interface / package (1개)

이름	종류
tlul_rsp_intg_gen	module

Table 1140. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_rsp_intg_gen.sv — parameter (4개)

파라미터	형/폭	기본값
EnableRspIntgGen	bit	1'b1
EnableDataIntgGen	bit	1'b1
UserInIsZero	bit	1'b0
RspIntgInIsZero	bit	UserInIsZero

Table 1141. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_rsp_intg_gen.sv — 포트 (2개)

포트	방향	형/폭
tl_i	input	tl_d2h_t
tl_o	output	tl_d2h_t

API — tlul_socket_1n.sv

Table 1142. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_1n.sv — module / interface / package (1개)

이름	종류
tlul_socket_1n	module

Table 1143. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_1n.sv — parameter (8개)

파라미터	형/폭	기본값
N	int unsigned	4
HReqPass	bit	1'b1
HRspPass	bit	1'b1
DReqPass	bit [N-1:0]	{N{1'b1}}
DRspPass	bit [N-1:0]	{N{1'b1}}
HReqDepth	bit [3:0]	4'h1
HRspDepth	bit [3:0]	4'h1
ExplicitErrs	bit	1'b1

Table 1144. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_1n.sv — localparam (3개)

상수	형/폭	값
NWD	int unsigned	\$clog2(ExplicitErrs ? N+1 : N)
MaxOutstanding	int	2**top_pkg::TL_AIW
OutstandingW	int	\$clog2(MaxOutstanding+1)

Table 1145. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_1n.sv — 포트 (5개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_h_i	input	tlul_pkg::tl_h2d_t
tl_h_o	output	tlul_pkg::tl_d2h_t
dev_select_i	input	[NWD-1:0]

API — tlul_socket_m1.sv**Table 1146. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_m1.sv — module / interface / package (17개)**

이름	종류
tlul_socket_m1	module

Table 1147. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_m1.sv — parameter (7개)

파라미터	형/폭	기본값
M	int unsigned	4
HReqPass	bit [M-1:0]	{M{1'b1}}
HRspPass	bit [M-1:0]	{M{1'b1}}
DReqPass	bit	1'b1

파라미터	형/폭	기본값
DRspPass	bit	1'b1
DReqDepth	bit [3:0]	4'h1
DRspDepth	bit [3:0]	4'h1

Table 1148. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_m1.sv — localparam (2개)

상수	형/폭	값
IDW	int unsigned	top_pkg::TL_AIW
STIDW	int unsigned	\$clog2(M)

Table 1149. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_m1.sv — 포트 (4개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_d_o	output	tlul_pkg::tl_h2d_t
tl_d_i	input	tlul_pkg::tl_d2h_t

1. lowRISC_opentitan/hw/ip/tlul/rtl/sram2tlul.sv (60 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // SRAM interface to TL-UL converter
6 // Current version only supports if TL-UL width and SRAM
  width are same
7 // If SRAM interface requests more than MaxOutstanding
  cap, it generates
8 // error in simulation but not in Silicon.
9
10 `include "prim_assert.sv"
11
12 module sram2tlul #(
13     parameter int          SramAw = 12,
14     parameter int          SramDw = 32,
15     parameter logic [top_pkg::TL_AW-1:0] TlBaseAddr = 'h0 //
  Base address of SRAM request
16 ) (
17     input clk_i,
18     input rst_ni,
19
20     output tlul_pkg::tl_h2d_t tl_o,
21     input  tlul_pkg::tl_d2h_t tl_i,
22
23     // SRAM
24     input          mem_req_i,
25     input          mem_write_i,
26     input [SramAw-1:0] mem_addr_i,
27     input [SramDw-1:0] mem_wdata_i,
28     output logic    mem_rvalid_o,
29     output logic [SramDw-1:0] mem_rdata_o,
30     output logic    [1:0] mem_error_o
31 );
32
33 import tlul_pkg::*;
34
35 `ASSERT_INIT(wrongSramDw, SramDw == top_pkg::TL_DW)
36
37 localparam int unsigned SRAM_DWB = $clog2(SramDw/8);
38
39 assign tl_o.a_valid = mem_req_i;
40 assign tl_o.a_opcode = (mem_write_i) ? PutFullData : Get;
41 assign tl_o.a_param = '0;
42 assign tl_o.a_size = top_pkg::TL_SZW'(SRAM_DWB); // Max
  Size always
43 assign tl_o.a_source = '0;
44 assign tl_o.a_address = TlBaseAddr |
45     {{{(top_pkg::TL_AW-SramAw-SRAM_DWB)
46     {1'b0}}, mem_addr_i, {(SRAM_DWB){1'b0}}}};
47 assign tl_o.a_data = mem_wdata_i;
48 assign tl_o.a_user = '0;

```

```

49
50 assign tl_o.d_ready = 1'b1;
51
52 assign mem_rvalid_o = tl_i.d_valid && (tl_i.d_opcode ==
  AccessAckData);
53 assign mem_rdata_o = tl_i.d_data;
54 assign mem_error_o = {2{tl_i.d_error}};
55
56 // below assertion fails when TL-UL doesn't accept request
  in a cycle,
57 // which is currently not supported by sram2tlul
58 `ASSERT(validNotReady, tl_o.a_valid |-> tl_i.a_ready)
59
60 endmodule

```

2. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_dmi.sv (183 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Tile-Link UL adapter for RISC-V debug module from the PULP
  project.
6 //
7 // Note that this module can only handle one outstanding re-
  quest at a time.
8
9 `include "prim_assert.sv"
10
11 module tlul_adapter_dmi
12 import tlul_pkg::*;
13 #(
14     parameter bit CmdIntgCheck = 1, // 1: Enable command
  integrity check
15     parameter bit EnableRspIntgGen = 1, // 1: Generate re-
  sponse integrity
16     parameter bit EnableDataIntgGen = 1 // 1: Generate re-
  sponse data integrity
17 ) (
18     input clk_i,
19     input rst_ni,
20
21     // TL-UL interface
22     input tl_h2d_t tl_h2d_i,
23     output tl_d2h_t tl_d2h_o,
24
25     // control interface
26     output logic intg_error_o,
27
28     output logic dmi_req_valid_o,
29     input logic dmi_req_ready_i,
30     output dm::dmi_req_t dmi_req_o,
31
32     input logic dmi_resp_valid_i,

```



```

33 output logic dmi_resp_ready_o,
34 input dm::dmi_resp_t dmi_resp_i
35 );
36
37 localparam int IdW = $bits(tl_h2d_i.a_source);
38 localparam int SzW = $bits(tl_h2d_i.a_size);
39
40 logic a_ack, d_ack; // Indicates whether the corresponding
valid/ready handshake completes.
41 logic outstanding_q; // Indicates whether there is a pending
request.
42 assign a_ack = tl_h2d_i.a_valid & tl_d2h_o.a_ready;
43 assign d_ack = tl_d2h_o.d_valid & tl_h2d_i.d_ready;
44 always_ff @(posedge clk_i or negedge rst_ni) begin
45 if (!rst_ni) outstanding_q <= 1'b0;
46 else if (a_ack) outstanding_q <= 1'b1;
47 else if (d_ack) outstanding_q <= 1'b0;
48 end
49
50 // Decode Requests
51 logic rd_req, wr_req;
52 assign wr_req = tl_h2d_i.a_valid && ((tl_h2d_i.a_opcode ==
PutFullData) ||
53 (tl_h2d_i.a_opcode ==
PutPartialData));
54 assign rd_req = tl_h2d_i.a_valid && (tl_h2d_i.a_opcode ==
Get);
55
56 // If we detect an error, the response will be returned im-
mediately in the next cycle
57 // and we do not send a request out to the DMI.
58 logic error_d, error_q;
59 assign dmi_req_valid_o = (wr_req || rd_req) && !error_d;
60 // The DMI response can be accepted if there is a pending
request without error.
61 assign dmi_resp_ready_o = outstanding_q && !error_q;
62
63 // We expect a word-aligned address here, otherwise an error is
returned (see further below).
64 assign dmi_req_o.addr = {2'b00, tl_h2d_i.a.ad-
dress[top_pkg::TL_AW-1:2]};
65 assign dmi_req_o.data = tl_h2d_i.a.data;
66 assign dmi_req_o.op = (wr_req) ? dm::DTM_WRITE :
67 (rd_req) ? dm::DTM_READ :
dm::DTM_NOP;
68
69 logic [IdW-1:0] reqid_q;
70 logic [SzW-1:0] reqsz_q;
71 tl_d_op_e rspop_q;
72 logic wr_req_q, rd_req_q;
73 always_ff @(posedge clk_i or negedge rst_ni) begin
74 if (!rst_ni) begin
75 reqid_q <= '0;
76 reqsz_q <= '0;
77 rspop_q <= AccessAck;
78 error_q <= 1'b0;
79 wr_req_q <= 1'b0;
80 rd_req_q <= 1'b0;
81 end else if (a_ack) begin
82 reqid_q <= tl_h2d_i.a.source;
83 reqsz_q <= tl_h2d_i.a.size;
84 // Return AccessAckData regardless of error
85 rspop_q <= (rd_req) ? AccessAckData : AccessAck;
86 rd_req_q <= rd_req;
87 wr_req_q <= wr_req;
88 error_q <= error_d;
89 end
90 end
91
92 // Send back a bus error if the dm_csrs module responded
with an error.
93 logic error_resp;
94 assign error_resp = error_q || (dmi_resp_i.resp ==
dm::DTM_ERR);
95
96 tlul_pkg::tl_d2h_t tl_d2h_o_pre;
97 assign tl_d2h_o_pre = '{
98 // We are ready to accept a request if there is no out-
standing one and if the DMI is ready.
99 a_ready: dmi_req_ready_i && !outstanding_q,
100 // A data response is valid if the DMI response is valid
and we've got an outstanding request.
101 // If the request resulted in an internal error, the error
response is returned without a DMI
102 // response, since the request to the DMI has been sup-
pressed by the error.
103 d_valid: (dmi_resp_valid_i || error_q) && outstanding_q,
104 d_opcode: rspop_q,
105 d_param: '0,
106 d_size: reqsz_q,
107 d_source: reqid_q,
108 d_sink: '0,
109 // Blank data upon error in the same way as other TL-UL
adapters would in OpenTitan.
110 d_data: (rd_req_q && !(error_resp || wr_req_q)) ?
dmi_resp_i.data : {top_pkg::TL_DW{1'b1}},
111 d_user: '0,
112 d_error: error_resp

```

```

113 };
114
115 // outgoing integrity generation
116 tlul_rsp_intg_gen #(
117 .EnableRspIntgGen(EnableRspIntgGen),
118 .EnableDataIntgGen(EnableDataIntgGen),
119 .UserInIsZero(1'b1)
120 ) u_rsp_intg_gen (
121 .tl_i(tl_d2h_o_pre),
122 .tl_o(tl_d2h_o)
123 );
124
125 logic intg_error;
126 if (CmdIntgCheck) begin : gen_cmd_intg_check
127 logic intg_error_q;
128 tlul_cmd_intg_chk u_cmd_intg_chk (
129 .tl_i(tl_h2d_i),
130 .err_o(intg_error)
131 );
132 // permanently latch integrity error until reset
133 always_ff @(posedge clk_i or negedge rst_ni) begin
134 if (!rst_ni) begin
135 intg_error_q <= 1'b0;
136 end else if (intg_error) begin
137 intg_error_q <= 1'b1;
138 end
139 end
140 assign intg_error_o = intg_error_q;
141 end else begin : gen_no_cmd_intg_check
142 assign intg_error = 1'b0;
143 assign intg_error_o = 1'b0;
144 end
145
146 // Error Handling //
147 // Error Handling //
148 // Error Handling //
149
150 logic addr_align_err; // Size and alignment
151 logic be_err; // Byte enable error.
152 logic malformed_meta_err; // User signal format error or un-
supported
153 logic tl_err; // Common TL-UL error checker
154 assign error_d = addr_align_err ||
155 be_err ||
156 malformed_meta_err ||
157 tl_err ||
158 intg_error;
159
160 // addr_align_err
161 // Raised if addr isn't aligned with the size
162 // Read size error is checked in tlul_assert.sv
163 // Here is it added due to the limitation of register in-
terface.
164 // Only word-align is accepted based on comportability
spec.
165 assign addr_align_err = (wr_req) && |tl_h2d_i.a.ad-
dress[1:0];
166
167 // The DMI does not support a non-all-ones byte enable.
168 assign be_err = (wr_req) && !(tl_h2d_i.a.mask);
169
170 // Don't allow unsupported values.
171 assign malformed_meta_err = tl_a_user_chk(tl_h2d_i.a.user);
172
173 // tl_err : separate checker
174 tlul_err u_err (
175 .clk_i,
176 .rst_ni,
177 .tl_i(tl_h2d_i),
178 .err_o(tl_err)
179 );
180
181 `ASSERT_INIT(DwMatchedWithDm_A, top_pkg::TL_DW ==
$bits(dmi_req_o.data))
182
183 endmodule : tlul_adapter_dmi

```

3. lowRISC_opentitan/hwip/tlul/rtl/tlul_adapter_host.sv (190 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // tlul_adapter (Host adapter) converts basic req/grant/rvalid
into TL-UL interface. If
6 // MAX_REQS == 1 it is purely combinational logic. If MAX_REQS > 1
flops are required.
7 //
8 // The host driving the adapter is responsible for ensuring it
doesn't have more requests in flight
9 // than the specified MAX_REQS.
10 //
11 // The outgoing address is always word aligned. The access
size is always the word size (as

```

```

12 // specified by TL_DW). For write accesses that occupy all
lanes the operation is PutFullData,
13 // otherwise it is PutPartialData, mask is generated from
be_i. For reads all lanes are enabled as
14 // required by TL-UL (every bit in mask set).
15 //

16 // When MAX_REQS > 1 tlul_adapter_host does not do anything to
return responses from the TL-UL
17 // interface which could return them out of order. It is the
host's responsibility to either only

18 // have outstanding requests to an address space it knows will
return responses in order or to not
19 // care about out of order responses (note that if read data
is returned out of order there is no
20 // way to determine this).
21
22 `include "prim_assert.sv"
23
24 module tlul_adapter_host
25 import tlul_pkg::*;
26 import prim_mubi_pkg::mubi4_t;
27 #(
28     parameter int unsigned MAX_REQS = 2,
29     parameter bit EnableDataIntgGen = 0,
30     parameter bit EnableRspDataIntgCheck = 0
31 ) (
32     input clk_i,
33     input rst_ni,
34
35     input req_i,
36     output logic gnt_o,
37     input logic [top_pkg::TL_AW-1:0] addr_i,
38     input logic we_i,
39     input logic [top_pkg::TL_DW-1:0] wdata_i,
40     input logic [DataIntgWidth-1:0] wdata_intg_i,
41     input logic [top_pkg::TL_DBW-1:0] be_i,
42     input mubi4_t instr_type_i,
43     input logic [RsvdWidth-1:0] user_rsvd_i,
44
45     output logic valid_o,
46     output logic [top_pkg::TL_DW-1:0] rdata_o,
47     output logic [DataIntgWidth-1:0] rdata_intg_o,
48     output logic err_o,
49     output logic intg_err_o,
50
51     output tl_h2d_t tl_o,
52     input tl_d2h_t tl_i
53 );
54 localparam int unsigned WordSize = $clog2(top_pkg::TL_DBW);
55
56 logic [top_pkg::TL_AIW-1:0] tl_source;
57 logic [top_pkg::TL_DBW-1:0] tl_be;
58 tl_h2d_t tl_out;
59
60 if (MAX_REQS == 1) begin : g_single_req
61     assign tl_source = '0;
62 end else begin : g_multiple_reqs
63     localparam int ReqNumW = $clog2(MAX_REQS);
64     localparam int unsigned MaxSource = MAX_REQS - 1;
65     localparam logic [ReqNumW-1:0] ReqNumOne = ReqNumW'(1'b1);
66
67     logic [ReqNumW-1:0] source_d;
68     logic [ReqNumW-1:0] source_q;
69
70     always_ff @(posedge clk_i or negedge rst_ni) begin
71         if (!rst_ni) begin
72             source_q <= '0;
73         end else begin
74             source_q <= source_d;
75         end
76     end
77
78     always_comb begin
79         source_d = source_q;
80
81         if (req_i && gnt_o) begin
82             if (source_q == MaxSource[ReqNumW-1:0]) begin
83                 source_d = '0;
84             end else begin
85                 source_d = source_q + ReqNumOne;
86             end
87         end
88     end
89
90     assign tl_source = top_pkg::TL_AIW'(source_q);
91 end
92
93 // For TL-UL Get opcode all active bytes must have their
mask bit set, so all reads get all tl_be
94 // bits set. For writes the supplied be_i is used as the
mask.
95 assign tl_be = ~we_i ? {top_pkg::TL_DBW{1'b1}} : be_i;
96
97 assign tl_out = '{
98     a_valid: req_i,
99     a_opcode: (~we_i) ? Get :
100             (&be_i) ? PutFullData :
101             PutPartialData,
102     a_param: 3'h0,

```

```

103     a_size: top_pkg::TL_SZW'(WordSize),
104     a_mask: tl_be,
105     a_source: tl_source,
106     a_address: {addr_i[31:WordSize], {WordSize{1'b0}}},
107     a_data: wdata_i,
108     a_user: '{default: '0, data_intg: wdata_intg_i, in-
str_type: instr_type_i, rsvd: user_rsvd_i},
109     d_ready: 1'b1
110 };
111
112 tlul_cmd_intg_gen #(EnableDataIntgGen (EnableDataIntgGen))
u_cmd_intg_gen (
113     .tl_i(tl_i),
114     .tl_o(tl_o)
115 );
116
117 assign gnt_o = tl_i.a_ready;
118
119 assign valid_o = tl_i.d_valid;
120 assign rdata_o = tl_i.d_data;
121 assign rdata_intg_o = tl_i.d_user.data_intg;
122
123 logic intg_err;
124 tlul_rsp_intg_chk #(
125     .EnableRspDataIntgCheck(EnableRspDataIntgCheck)
126 ) u_rsp_chk (
127     .tl_i,
128     .err_o(intg_err)
129 );
130
131 logic intg_err_q;
132 always_ff @(posedge clk_i or negedge rst_ni) begin
133     if (!rst_ni) begin
134         intg_err_q <= '0;
135     end else if (intg_err) begin
136         intg_err_q <= 1'b1;
137     end
138 end
139
140 // err_o is transactional. This allows the host to continue
141 // debug without receiving an endless stream of errors.
142 assign err_o = tl_i.d_error | intg_err;
143
144 // intg_err_o is permanent once detected, and should be used
145 // to trigger alerts
146 assign intg_err_o = intg_err_q | intg_err;
147
148 // Addresses are assumed to be word-aligned, and the bottom
bits are ignored
149 logic unused_addr_bottom_bits;
150 assign unused_addr_bottom_bits = ^addr_i[WordSize-1:0];
151
152 // Explicitly ignore unused fields of tl_i
153 logic unused_tl_i_fields;
154 assign unused_tl_i_fields = ^{tl_i.d_opcode, tl_i.d_param,
tl_i.d_size, tl_i.d_source,
tl_i.d_sink,
tl_i.d_user};
155
156
157 `ifdef INC_ASSERT
158 //VCS coverage off
159 // pragma coverage off
160 localparam int OutstandingReqCntW =
161     (MAX_REQS == 2 ** $clog2(MAX_REQS)) ? $clog2(MAX_REQS) +
1 : $clog2(MAX_REQS);
162 localparam logic [OutstandingReqCntW-1:0] OutstandingRe-
qCntOne = OutstandingReqCntW'(1'b1);
163
164 logic [OutstandingReqCntW-1:0] outstanding_reqs_q;
165 logic [OutstandingReqCntW-1:0] outstanding_reqs_d;
166
167 always_comb begin
168     outstanding_reqs_d = outstanding_reqs_q;
169
170     if ((req_i && gnt_o) && !valid_o) begin
171         outstanding_reqs_d = outstanding_reqs_q + OutstandingRe-
qCntOne;
172     end else if (!(req_i && gnt_o) && valid_o) begin
173         outstanding_reqs_d = outstanding_reqs_q - OutstandingRe-
qCntOne;
174     end
175 end
176
177 always_ff @(posedge clk_i or negedge rst_ni) begin
178     if (!rst_ni) begin
179         outstanding_reqs_q <= '0;
180     end else begin
181         outstanding_reqs_q <= outstanding_reqs_d;
182     end
183 end
184
185 //VCS coverage on
186 // pragma coverage on
187
188 `ASSERT(DontExceedMaxReqs, req_i |> outstanding_reqs_d <=
MAX_REQS)
189 `endif
190 endmodule

```

4. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_racl.v (114 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8  * Tile-Link UL adapter with RACL support
9  */
10 * - It allows filtering TLUL requests based on their range
  using an allow-list approach. A request
11 * is allowed if its address matches a racl range which al-
  lows access for the racl role associated
12 * with this request.
13 */
14 module tlul_adapter_racl
15   import tlul_pkg::*;
16   import prim_mubi_pkg::mubi4_t;
17   #(
18     parameter bit EnableRacl              = 0,          // 1: En-
  able RACL checks on access
19     parameter bit RaclErrorRsp           = EnableRacl, // 1: Re-
  turn TLUL error on RACL errors

20     parameter int RaclPolicySelNumRanges = 1            // Number of
  ranges with a RACL policy
21   ) (
22     in-
23       clk_i,
24       rst_ni,
25       tlul_pkg::tl_h2d_t tl_h2d_i,
26       tlul_pkg::tl_d2h_t tl_d2h_o,
27       tlul_pkg::tl_h2d_t tl_filtered_h2d_o,
28       tlul_pkg::tl_d2h_t tl_filtered_d2h_i,
29       // RACL interface
30       input top_racl_pkg::racl_policy_vec_t
  racl_policies_i,
31       output top_racl_pkg::racl_error_log_t
  racl_er-
  ror_o,
32       input top_racl_pkg::racl_range_t [RaclPolicySelNum-
  Ranges-1:0] racl_policy_sel_ranges_i
33   );
34   if (EnableRacl) begin : gen_racl_role_logic
35     // Retrieve RACL role from user bits and one-hot encode
  that for the comparison bitmap
36     top_racl_pkg::racl_role_t racl_role;
37     assign racl_role =
  top_racl_pkg::tlul_extract_racl_role_bits(tl_h2d_i.a_user.rsvd);
38
39     top_racl_pkg::racl_role_vec_t racl_role_vec;
40     prim_onehot_enc #(
41       .OneHotWidth( $bits(top_racl_pkg::racl_role_vec_t) )
42     ) u_racl_role_encode (
43       .in_i ( racl_role ),
44       .en_i ( 1'b1 ),
45       .out_o ( racl_role_vec )
46     );
47
48     logic rd_req, wr_req, racl_read_allowed,
  racl_write_allowed, racl_error;
49     logic [RaclPolicySelNumRanges-1:0] range_read_allowed;
50     logic [RaclPolicySelNumRanges-1:0] range_write_allowed;
51
52     for (genvar r = 0; r < RaclPolicySelNumRanges; r++) be-
  gin : gen_racl_range_check
53       top_racl_pkg::racl_range_t range;
54       top_racl_pkg::racl_policy_t policy;
55       logic range_match;
56       assign range = racl_policy_sel_ranges_i[r];
57       assign policy = racl_policies_i[range.policy_sel];
58       // Check if the address is within range
59       assign range_match = range.enable
  & tl_h2d_i.a_address >= range.base
  & tl_h2d_i.a_address <=
  range.limit;
60       // If address matches, lookup permissions for policy
  defined for this range
61       assign range_read_allowed[r] = range_match & |
  (policy.read_perm & racl_role_vec);
62       assign range_write_allowed[r] = range_match & |
  (policy.write_perm & racl_role_vec);
63     end
64
65     assign racl_read_allowed = |range_read_allowed;

```

```

68     assign racl_write_allowed = |range_write_allowed;
69
70     assign rd_req      = tl_h2d_i.a_valid &
  (tl_h2d_i.a_opcode == tlul_pkg::Get);
71     assign wr_req      = tl_h2d_i.a_valid &
  (tl_h2d_i.a_opcode == tlul_pkg::PutFullData |
72   tl_h2d_i.a_opcode == tlul_pkg::PutPartialData);
73     assign racl_error  = (rd_req & ~racl_read_allowed)
  | (wr_req & ~racl_write_allowed);
74     assign racl_error_o.valid = racl_error & tl_d2h_o.a_ready;
75
76     tlul_request_loopback #(
77       .ErrorRsp ( RaclErrorRsp )
78     ) u_loopback (
79       .clk_i      ( clk_i ),
80       .rst_ni     ( rst_ni ),
81       .squash_req_i ( racl_error ),
82       .tl_h2d_i   ( tl_h2d_i ),
83       .tl_d2h_o   ( tl_d2h_o ),
84       .tl_h2d_o   ( tl_filtered_h2d_o ),
85       .tl_d2h_i   ( tl_filtered_d2h_i )
86     );
87
88     // Collect RACL error information
89     assign racl_error_o.overflow = 1'b0;
90     assign racl_error_o.read_access = tl_h2d_i.a_opcode ==
  tlul_pkg::Get;
91     assign racl_error_o.racl_role = racl_role;
92     assign racl_error_o.ctn_uid =
  top_racl_pkg::tlul_extract_ctn_uid_bits(tl_h2d_i.a_user.rsvd);
93     assign racl_error_o.request_address = tl_h2d_i.a_address;
94     end else begin : gen_no_racl_role_logic
95       // Pass through and default assignments
96       assign tl_filtered_h2d_o = tl_h2d_i;
97       assign tl_d2h_o = tl_filtered_d2h_i;
98       assign racl_error_o = '0;
99
100      // Some signals are not read when RACL is disabled
101      logic unused_signals;
102      assign unused_signals = ^racl_policy_sel_ranges_i;
103    end
104
105    // Not all RACL policies are used, even if RACL is enabled
106    logic unused_policy_sel;
107    assign unused_policy_sel = ^racl_policies_i;
108
109    `ASSERT(RaclAdapterNumRanges, EnableRacl |-> RaclPolicySel-
  NumRanges > 0)
110
111    // Ensure that RACL signals are not undefined
112    `ASSERT_KNOWN(RaclAdapterErrorKnown_A, racl_error_o.valid)
113
114  endmodule

```

5. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg.v (307 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 // Tile-Link UL to register interface adapter
8
9 // This module acts as a device, addressed through the TL-UL
  bus. It receives Get, PutPartialData
10 // and PutFullData messages on the A channel. These are read
  or write requests, which are passed to
11 // the register interface on the same cycle. The response from
  the register interface is sent back
12 // to the host as a message on the D channel on the next
  cycle.
13
14 // A module instance performs several checks on A-channel mes-
  sages and only forwards a request to
15 // the register interface if they all pass. These checks are:
16
17 // - A_OPCODE must be Get, PutPartialData or PutFullData.
18
19 // - A_SIZE must be 0, 1 or 2 (giving a 1-, 2- or 4-byte op-
  eration, respectively).
20
21 // - A_ADDRESS must be naturally aligned for the operation
  byte width signaled in A_SIZE.
22
23 // - A_MASK must only be true for active byte-lanes. The pos-
  ition of the first of these active
24 // lanes is calculated from the address, modulo the width
  of the TL-UL data bus.
25
26 // - If the A_USER.INSTR_TYPE value is Mubi4True, A_OPCODE
  must be Get.
27
28 // - Either the A_USER.INSTR_TYPE value must be a value other

```

```

than MuBi4True or the en_ifetch_i
29 // port must be MuBi4True.
30 //
31 // - If the CmdIntgCheck parameter is true then the command
integrity check must pass.
32 //
33 //
34 // ** Parameters
35 //
36 // CmdIntgCheck: If this parameter is true then there is a
"command integrity check". This uses a
37 // tlul_cmd_intg_chk instance to compare integ-
rity checksums from requests on the A
38 // channel. If the integrity checksums don't
match, the adapter doesn't pass the
39 // request to the register interface through
re_o/we_o and the response on the D
40 // channel will then have its D_ERROR flag set.
41 //
42 // EnableRspIntgGen: OpenTitan sends response integrity with
messages on the TL-UL bus. If this
43 // parameter is true, the module will per-
form calculations to send a correct
44 // integrity value in the response's
A_USER.RSP_INTG field.
45 //
46 // EnableDataIntgGen: This parameter is analogous to EnableR-
spIntgGen but controls whether the
47 // module performs calculations to send a
correct integrity value in the
48 // response's A_USER.DATA_INTG field.
49 //
50 // RegAw: The number of bits used for addresses in the re-
gister interface. Higher bits of the
51 // A_ADDRESS field in an A channel message are ignored.
52 //
53 // RegDw: The number of bits used for data words in the re-
gister interface. This is assumed to match
54 // the data width of the TL-UL bus (see the Matched-
Width_A assertion).
55 //
56 // AccessLatency: This adapter always returns TileLink re-
sponses after one cycle, but this parameter
57 // allows it to handle different timings on the
register interface. If AccessLatency
58 // is zero then the adapter expects a combinat-
orial response on the register
59 // interface and flops the result to break
paths and add a cycle delay. If
60 // AccessLatency is one then the adapter ex-
pects a response on the register interface
61 // in the cycle after the re_o/we_o signal goes
high, but then forwards it
62 // combinatorially to the TileLink interface.
Only values 0 and 1 are allowed for
63 // this parameter.
64 //
65 //
66 // ** Ports
67 //
68 // clk_i/rst_ni: Clock and reset.
69 // tl_i/tl_o: Connections to the TL-UL bus.
70 // en_ifetch_i: This mubi4_t port controls whether to allow
read requests from Get messages whose
71 // A_USER.INSTR_TYPE is MuBi4True (considered to be
processor "fetch" requests).
72 // intg_error_o: This port is high when the adapter has detec-
ted a command integrity error. That can
73 // only happen if the CmdIntgCheck parameter is
set.
74 // re_o/we_o: The read-enable and write-enable ports in the
register interface.
75 // addr_o: The address to read or write through the re-
gister interface.
76 // wdata_o: The value to write through the register in-
terface if we_o is true.
77 // be_o: A byte-enable signal, which is true for each
byte-lane where the TL-UL operation's
78 // mask is true.
79 // busy_i: If this input is high then the A channel in-
terface is stalled (by dropping
80 // A_READY), which means that the adapter will
not accept any new operations.
81 // rdata_i: Read data being returned for a read request
through the register interface.
82 // error_i: An error flag in the register interface. This
signals that an error has been seen,
83 // which can be seen in the D channel response
by D_ERROR being high. The D_DATA
84 // response is also set to '1, squashing any
data that might have been returned
85 // through rdata_i.
86 //
87 // ICEBOX(#15822): Note that due to some modules with special
needs (like the vendored-in RV_DM),
88 // this module has been extended so that it

```

```

supports use cases outside of the
89 // generated reg_top module. This makes this
adapter and its parameterization
90 // options a bit heavy.
91 //
92 // We should in the future come back to this
and refactor / align the module and its
93 // parameterization needs.
94 //
95 module tlul_adapter_reg
96 import tlul_pkg::*;
97 import prim_mubi_pkg::mubi4_t;
98 #(
99 parameter bit CmdIntgCheck = 0, // 1: Enable command
integrity check
100 parameter bit EnableRspIntgGen = 0, // 1: Generate re-
sponse integrity
101 parameter bit EnableDataIntgGen = 0, // 1: Generate re-
sponse data integrity
102 parameter int RegAw = 8, // Width of register
address
103 parameter int RegDw = 32, // Shall be matched
with TL_DW
104 parameter int AccessLatency = 0, // 0: same cycle, 1:
next cycle
105 localparam int RegBw = RegDw/8
106 ) (
107 input clk_i,
108 input rst_ni,
109 // TL-UL interface
110 input tl_h2d_t tl_i,
111 output tl_d2h_t tl_o,
112 // control interface
113 input mubi4_t en_ifetch_i,
114 output logic intg_error_o,
115 // Register interface
116 output logic re_o,
117 output logic we_o,
118 output logic [RegAw-1:0] addr_o,
119 output logic [RegDw-1:0] wdata_o,
120 output logic [RegBw-1:0] be_o,
121 input busy_i,
122 // The following two signals are expected
123 // to be returned in AccessLatency cycles.
124 input [RegDw-1:0] rdata_i,
125 // This can be a write or read error.
126 input error_i
127 );
128
129 `ASSERT_INIT(AllowedLatency_A, AccessLatency inside {0, 1})
130
131 localparam int IW = $bits(tl_i.a_source);
132 localparam int SZW = $bits(tl_i.a_size);
133
134 logic outstanding_q; // Indicates current request is
pending
135 logic a_ack, d_ack;
136 logic [RegDw-1:0] rdata, rdata_q;
137
138 logic
139 error_q, error, err_internal, instr_error,
intg_error;
140
141 logic addr_align_err; // Size and alignment
142 logic tl_err; // Common TL-UL error checker
143
144 logic [IW-1:0] reqid_q;
145 logic [SZW-1:0] reqsz_q;
146 tl_d_op_e rspop_q;
147
148 logic rd_req, wr_req;
149
150 assign a_ack = tl_i.a_valid & tl_o.a_ready;
151 assign d_ack = tl_o.d_valid & tl_i.d_ready;
152 // Request signal
153 assign wr_req = a_ack & ((tl_i.a_opcode == PutFullData) |
(tl_i.a_opcode == PutPartialData));
154 assign rd_req = a_ack & (tl_i.a_opcode == Get);
155
156 assign we_o = wr_req & ~err_internal;
157 assign re_o = rd_req & ~err_internal;
158 assign wdata_o = tl_i.a_data;
159 assign be_o = tl_i.a_mask;
160
161 if (RegAw <= 2) begin : gen_only_one_reg
162 assign addr_o = '0;
163 end else begin : gen_more_regs
164 assign addr_o = {tl_i.a_address[RegAw-1:2], 2'b00}; //
generate always word-align
165 end
166
167 always_ff @(posedge clk_i or negedge rst_ni) begin
168 outstanding_q <= 1'b0;
169 else if (a_ack) outstanding_q <= 1'b1;
170 else if (d_ack) outstanding_q <= 1'b0;
171 end
172
173 always_ff @(posedge clk_i or negedge rst_ni) begin

```

```

176   if (!rst_ni) begin
177       reqid_q <= '0;
178       reqsz_q <= '0;
179       rspop_q <= AccessAck;
180   end else if (a_ack) begin
181       reqid_q <= tl_i.a_source;
182       reqsz_q <= tl_i.a_size;
183       // Return AccessAckData regardless of error
184       rspop_q <= (rd_req) ? AccessAckData : AccessAck ;
185   end
186 end
187
188 if (AccessLatency == 1) begin : gen_access_latency1
189     logic a_ack_q, err_internal_q, wr_req_q;
190     always_ff @(posedge clk_i or negedge rst_ni) begin
191         if (!rst_ni) begin
192             a_ack_q <= 1'b0;
193             err_internal_q <= 1'b0;
194             wr_req_q <= 1'b0;
195             rdata_q <= '0;
196             error_q <= 1'b0;
197         end else begin
198             a_ack_q <= a_ack;
199             err_internal_q <= err_internal;
200             wr_req_q <= wr_req;
201
202             rdata_q <= rdata;
203             error_q <= error;
204         end
205     end
206     always_comb begin
207         if (a_ack_q) begin
208             rdata = (error_i || err_internal_q || wr_req_q) ? '1 :
rdata_i;
209             error = error_i || err_internal_q;
210         end else begin
211             rdata = rdata_q;
212             error = error_q;
213         end
214     end
215 end else begin : gen_access_latency0
216     always_ff @(posedge clk_i or negedge rst_ni) begin
217         if (!rst_ni) begin
218             rdata_q <= '0;
219             error_q <= 1'b0;
220         end else if (a_ack) begin
221             rdata_q <= (error_i || err_internal || wr_req) ? '1 :
rdata_i;
222             error_q <= error_i || err_internal;
223         end
224     end
225     assign rdata = rdata_q;
226     assign error = error_q;
227 end
228
229 tlul_pkg::tl_d2h_t tl_o_pre;
230 assign tl_o_pre = '{
231     a_ready: ~(outstanding_q | busy_i),
232     d_valid: outstanding_q,
233     d_opcode: rspop_q,
234     d_param: '0,
235     d_size: reqsz_q,
236     d_source: reqid_q,
237     d_sink: '0,
238     d_data: rdata,
239     d_user: '0,
240     d_error: error
241 };
242
243 // outgoing integrity generation
244 tlul_rsp_intg_gen #(
245     .EnableRspIntgGen(EnableRspIntgGen),
246     .EnableDataIntgGen(EnableDataIntgGen),
247     .UserInIsZero(1'b1)
248 ) u_rsp_intg_gen (
249     .tl_i(tl_o_pre),
250     .tl_o(tl_o)
251 );
252
253 if (CmdIntgCheck) begin : gen_cmd_intg_check
254     logic intg_error_q;
255     tlul_cmd_intg_chk u_cmd_intg_chk (
256         .tl_i(tl_i),
257         .err_o(intg_error)
258     );
259     // permanently latch integrity error until reset
260     always_ff @(posedge clk_i or negedge rst_ni) begin
261         if (!rst_ni) begin
262             intg_error_q <= 1'b0;
263         end else if (intg_error) begin
264             intg_error_q <= 1'b1;
265         end
266     end
267     assign intg_error_o = intg_error_q;
268 end else begin : gen_no_cmd_intg_check
269     assign intg_error = 1'b0;
270     assign intg_error_o = 1'b0;
271 end
272
273 //////////////////////////////////////////////////

```

```

274 // Error Handling //
275 //////////////////////////////////////////////////
276
277 // An instruction type transaction is only valid if
en_ifetch is enabled
278 assign instr_error =
prim_mubi_pkg::mubi4_test_true_strict(tl_i.a_user.instr_type) &
279 prim_mubi_pkg::mubi4_test_false_loose(en_ifetch_i);
280
281 assign err_internal = addr_align_err | tl_err | instr_error
| intg_error;
282
283 // addr_align_err
284 // Raised if addr isn't aligned with the size
285 // Read size error is checked in tlul_assert.sv
286 // Here is it added due to the limitation of register in-
terface.
287 always_comb begin
288     if (wr_req) begin
289         // Only word-align is accepted based on comportsability
spec
290         addr_align_err = |tl_i.a_address[1:0];
291     end else begin
292         // No request
293         addr_align_err = 1'b0;
294     end
295 end
296
297 // tl_err : separate checker
298 tlul_err u_err (
299     .clk_i,
300     .rst_ni,
301     .tl_i,
302     .err_o (tl_err)
303 );
304
305 `ASSERT_INIT(MatchedWidth_A, RegDw == top_pkg::TL_DW)
306
307 endmodule

```

6. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg_racl.sv (137 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8  * Tile-Link UL adapter for Register interface with RACL pro-
tection
9  */
10
11 module tlul_adapter_reg_racl
12     import tlul_pkg::*;
13     import prim_mubi_pkg::mubi4_t;
14 #(
15     parameter bit CmdIntgCheck      = 0,          // 1: Enable
command integrity check
16     parameter bit EnableRspIntgGen  = 0,          // 1: Gener-
ate response integrity
17     parameter bit EnableDataIntgGen = 0,          // 1: Gener-
ate response data integrity
18     parameter int RegAw              = 8,          // Width of
register address
19     parameter int RegDw              = 32,          // Shall be
matched with TL_DW
20     parameter int AccessLatency     = 0,          // 0: same
cycle, 1: next cycle
21     parameter bit EnableRacl        = 0,          // 1: Enable
RACL checks on access
22     parameter bit RaclErrorRsp      = EnableRacl, // 1: Return
TLUL error on RACL errors
23     parameter top_racl_pkg::racl_policy_sel_t RaclPolicySelVec =
0, // RACL policy for this reg
24
25     // adapter
26     localparam int RegBw              = RegDw/8
27 ) (
28     input clk_i,
29     input rst_ni,
30
31     // TL-UL interface
32     input tl_h2d_t tl_i,
33     output tl_d2h_t tl_o,
34
35     // control interface
36     input mubi4_t en_ifetch_i,
37     output logic intg_error_o,
38
39     // RACL interface

```



```

39 input top_racl_pkg::racl_policy_vec_t racl_policies_i,
40 output top_racl_pkg::racl_error_log_t racl_error_o,
41
42 // Register interface
43 output logic re_o,
44 output logic we_o,
45 output logic [RegAw-1:0] addr_o,
46 output logic [RegDw-1:0] wdata_o,
47 output logic [RegBw-1:0] be_o,
48 input busy_i,
49 // The following two signals are expected
50 // to be returned in AccessLatency cycles.
51 input [RegDw-1:0] rdata_i,
52 // This can be a write or read error.
53 input error_i
54 );
55 tl_h2d_t tl_h2d_filtered;
56 tl_d2h_t tl_d2h_filtered;
57
58 if (EnableRacl) begin : gen_racl_role_logic
59 // Retrieve RACL role from user bits and one-hot encode
60 that for the comparison bitmap
61 top_racl_pkg::racl_role_t racl_role;
62 assign racl_role =
63 top_racl_pkg::tlul_extract_racl_role_bits(tl_i.a_user.rsvd);
64
65 top_racl_pkg::racl_role_vec_t racl_role_vec;
66 prim_onehot_enc #(
67 .OneHotWidth( $bits(top_racl_pkg::racl_role_vec_t) )
68 ) u_racl_role_encode (
69 .in_i ( racl_role ),
70 .en_i ( 1'b1 ),
71 .out_o( racl_role_vec );
72 );
73
74 logic rd_req, wr_req, racl_read_allowed,
75 racl_write_allowed, racl_error;
76 assign rd_req = tl_i.a_valid & (tl_i.a_opcode
77 == tlul_pkg::Get);
78 assign wr_req = tl_i.a_valid & (tl_i.a_opcode
79 == tlul_pkg::PutFullData |
80 == tlul_pkg::PutPartialData);
81
82 assign racl_read_allowed = (|
83 (racl_policies_i[RaclPolicySelVec].read_perm & racl_role_vec));
84 assign racl_write_allowed = (|
85 (racl_policies_i[RaclPolicySelVec].write_perm & racl_role_vec));
86 assign racl_error = (rd_req & ~racl_read_allowed)
87 | (wr_req & ~racl_write_allowed);
88 assign racl_error_o.valid = racl_error & tl_o.a_ready;
89
90 tlul_request_loopback #(
91 .ErrorRsp(RaclErrorRsp)
92 ) u_loopback (
93 .clk_i,
94 .rst_ni,
95 .squash_req_i ( racl_error ),
96 .tl_h2d_i ( tl_i ),
97 .tl_d2h_o ( tl_o ),
98 .tl_h2d_o ( tl_h2d_filtered ),
99 .tl_d2h_i ( tl_d2h_filtered );
100 );
101
102 // Collect RACL error information
103 assign racl_error_o.overflow = 1'b0;
104 assign racl_error_o.read_access = tl_i.a_opcode ==
105 tlul_pkg::Get;
106 assign racl_error_o.racl_role = racl_role;
107 assign racl_error_o.ctn_uid =
108 top_racl_pkg::tlul_extract_ctn_uid_bits(tl_i.a_user.rsvd);
109 assign racl_error_o.request_address = tl_i.a.address;
110 end else begin : gen_no_racl_role_logic
111 // Pass through and default assignments
112 assign tl_h2d_filtered = tl_i;
113 assign tl_o = tl_d2h_filtered;
114 assign racl_error_o = '0;
115 end
116
117 tlul_adapter_reg #(
118 .CmdIntgCheck (CmdIntgCheck),
119 .EnableRspIntgGen (EnableRspIntgGen),
120 .EnableDataIntgGen (EnableDataIntgGen),
121 .RegAw (RegAw),
122 .RegDw (RegDw),
123 .AccessLatency (AccessLatency)
124 ) tlul_adapter_reg (
125 .clk_i,
126 .rst_ni,
127 .tl_i(tl_h2d_filtered),
128 .tl_o(tl_d2h_filtered),
129 .en_ifetch_i,
130 .intg_error_o,
131 .re_o,
132 .we_o,
133 .addr_o,
134 .wdata_o,
135 .be_o,
136 .busy_i,
137 .rdata_i,
138 .error_i
139 );

```

```

129
130 // Not all RACL policies are used, even if RACL is enabled
131 logic unused_policy;
132 assign unused_policy = ^racl_policies_i;
133
134 // Ensure that RACL signals are not undefined
135 `ASSERT_KNOWN(RaclAdapterRegErrorKnown_A, racl_er-
136 ror_o.valid)
137
138 endmodule

```

7. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_shim.sv (149 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5
6 `include "prim_assert.sv"
7
8 module tlul_adapter_vh
9 import tlul_pkg::*;
10 import prim_mubi_pkg::mubi4_t;
11
12 #(
13 parameter int ADDR_WIDTH = top_pkg::TL_AW,
14 parameter int DATA_WIDTH = top_pkg::TL_DW,
15 parameter int MASK_WIDTH = DATA_WIDTH >> 3,
16 parameter int USER_WIDTH = 32,
17 parameter int ID_WIDTH = top_pkg::TL_AIW,
18
19 parameter bit [ADDR_WIDTH-1:0] VH_REGISTER_ADDRESS_OFFSET =
20 32'h0000_0100,
21
22 parameter bit EnableDataIntgGen = 1,
23 parameter bit EnableRspDataIntgCheck = 1
24 ) (
25 input clk_i,
26 input rst_ni,
27
28 output tl_h2d_t tl_o,
29 input tl_d2h_t tl_i,
30
31 // Valid-Hold device interface (VH to TLUL).
32 input logic dv_i,
33 output logic hld_o,
34 input logic [ADDR_WIDTH-1:0] addr_i,
35 input logic [DATA_WIDTH-1:0] wdata_i,
36 input logic [MASK_WIDTH-1:0] wstrb_i,
37 input logic [2:0] size_i,
38 output logic [DATA_WIDTH-1:0] rdata_o,
39 output logic error_o,
40 input logic last_i,
41 input logic [USER_WIDTH-1:0] user_i,
42 input logic [ID_WIDTH-1:0] id_i,
43
44 // Valid-Hold host interface (VH to internal registers). The
45 // signals from the VH device interface
46 // are routed to the VH host interface for every internal
47 // access, see the 'internal_access' signal.
48 output logic int_dv_o,
49 input logic int_hld_i,
50 output logic [ADDR_WIDTH-1:0] int_addr_o,
51 output logic int_write_o,
52 output logic [DATA_WIDTH-1:0] int_wdata_o,
53 output logic [MASK_WIDTH-1:0] int_wstrb_o,
54 output logic [2:0] int_size_o,
55 input logic [DATA_WIDTH-1:0] int_rdata_i,
56 input logic int_error_i,
57 output logic int_last_o,
58 output logic [USER_WIDTH-1:0] int_user_o,
59 output logic [ID_WIDTH-1:0] int_id_o,
60 );
61
62 // The adapter distinguishes between two types of VH ac-
63 cesses: those going to an internal register
64 // file and those targeting OpenTitan registers, with the
65 // latter ones being translated into TLUL
66 // requests while the other requests are relayed to the in-
67 // ternal register file.
68 logic internal_access;
69 assign internal_access = addr_i >= VH_RE-
70 GISTER_ADDRESS_OFFSET;
71
72 // Differentiate between two levels of acknowledgements:
73
74 // - `req_ack`: A host-to-device request is acknowledged by
75 // the device, meaning that the response
76 // is pending.
77 // - `resp_ack`: The device-to-host response is acknow-
78 // ledged.
79 logic pending_d, pending_q;
80 logic req_ack, resp_ack;
81
82 assign req_ack = dv_i & tl_i.a_ready & tl_o.a_valid & ~in-
83 ternal_access;
84 assign resp_ack = dv_i & tl_o.d_ready & tl_i.d_valid & ~in-

```

```

ternal_access;
73
74 always_comb begin
75     pending_d = pending_q;
76     if (req_ack) begin
77         pending_d = 1'b1;
78     end else if (resp_ack) begin
79         pending_d = 1'b0;
80     end
81 end
82
83 always_ff @(posedge clk_i or negedge rst_ni) begin
84     if (!rst_ni) begin
85         pending_q <= 1'b0;
86     end else begin
87         pending_q <= pending_d;
88     end
89 end
90
91 // Only make a new request (through `a_valid`) to the device
92 // when none are pending. If the
93 // integrity checks are enabled, the `a_user` fields will be
94 // set by the corresponding module (see
95 // `tlul_cmd_intg_gen`).
96 tlul_pkg::tl_h2d_t tlo_pre;
97 assign tlo_pre = '{
98     a_valid: dv_i & ~pending_q & ~internal_access,
99     a_opcode: ~write_i ? Get : (&wstrb_i ? PutFullData : Put-
PartialData),
100     a_size: top_pkg::TL_SZW'(size_i),
101     a_mask: wstrb_i,
102     a_source: id_i,
103     a_address: addr_i,
104     a_data: wdata_i,
105     a_user: '{default: '0, instr_type:
prim_mubi_pkg::MuB14False},
106     d_ready: 1'b1,
107     // unused
108     a_param: 3'h0
109 };
110 // Accesses to device registers are acknowledged with the
111 // TLUL d-channel handshake.
112
113 assign hld_o = !pending_q && internal_access && !int_hld_i ?
1'b0 :
114
115     pending_q && tlo.d_ready && tli.d_valid ?
1'b0 : 1'b1;
116
117 assign rdata_o = internal_access ? int_rdata_i :
tli.d_data;
118
119 // Route signals to the VH host interface.
120 assign int_dv_o = dv_i & internal_access;
121 assign int_addr_o = addr_i;
122 assign int_write_o = write_i;
123 assign int_wdata_o = wdata_i;
124 assign int_wstrb_o = wstrb_i;
125 assign int_size_o = size_i;
126 assign int_last_o = last_i;
127 assign int_user_o = user_i;
128 assign int_id_o = id_i;
129
130 tlul_cmd_intg_gen #(
131     .EnableDataIntgGen (EnableDataIntgGen)
132 ) u_cmd_intg_gen (
133     .tli ( tlo_pre ),
134     .tlo ( tlo )
135 );
136
137 logic intg_err_chk;
138 tlul_rsp_intg_chk #(
139     .EnableRspDataIntgCheck(EnableRspDataIntgCheck)
140 ) u_rsp_chk (
141     .tli ( tli ),
142     .err_o ( intg_err_chk )
143 );
144
145 assign error_o = tli.d_error | int_error_i | intg_err_chk;
146
147 // Make sure the VH parameters are compatible with TLUL
148 datapath widths.
149 `ASSERT_INIT(TlInvalidAddrWidth_A, ADDR_WIDTH ==
top_pkg::TL_AW)
150 `ASSERT_INIT(TlInvalidDataWidth_A, DATA_WIDTH ==
top_pkg::TL_DW)
151 `ASSERT_INIT(TlInvalidMaskWidth_A, MASK_WIDTH ==
top_pkg::TL_DBW)
152 `ASSERT_INIT(TlInvalidUserWidth_A, ID_WIDTH ==
top_pkg::TL_AIW)
153
154 endmodule

```

8. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram.sv (688 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 `include "prim_assert.sv"
6
7 /**
8  * Tile-Link UL adapter for SRAM-like devices
9  */
10 * - Intentionally omitted BaseAddr in case of multiple memory
11 * maps are used in a SoC,
12 * it means that aliasing can happen if target device size
13 * in TL-UL crossbar is bigger
14 * than SRAM size
15 * - At most one of EnableDataIntgGen / EnableDataIntgPt can
16 * be enabled. However it
17 * possible for both to be disabled.
18 * A module can neither generate an integrity response nor
19 * pass through any pre-existing
20 * integrity. This might be the case for non-security crit-
21 * ical memories where there is
22 * no stored integrity AND another entity upstream is
23 * already generating returning integrity.
24 * There is however no case where EnableDataIntgGen and En-
25 * ableDataIntgPt are both true.
26 */
27 module tlul_adapter_sram
28     import tlul_pkg::*;
29     import prim_mubi_pkg::mubi4_t;
30     #(
31         parameter int SramAw          = 12,
32         parameter int SramDw          = 32, // Must be multiple of
33         // the TL width
34         parameter int SramDepth       = 2**SramAw, // Must be <=
35         2**SramAw
36         parameter int Outstanding     = 1, // Only one request is
37         // accepted
38         parameter int SramBusBankAW  = 12, // SRAM bus address
39         // width of the SRAM bank. Only used
40         // when DataXorAddr=1.
41         parameter bit ByteAccess      = 1, // 1: Enables sub-word
42         // write transactions. Note that this
43         // results in read-
44         // modify-write operations for integrity
45         // re-generation if
46         // EnableDataIntgPt is set to 1.
47         parameter bit ErrOnWrite      = 0, // 1: Writes not al-
48         // lowed, automatically error
49         parameter bit ErrOnRead       = 0, // 1: Reads not al-
50         // lowed, automatically error
51         parameter bit CmdIntgCheck    = 0, // 1: Enable command
52         // integrity check
53         parameter bit EnableRspIntgGen = 0, // 1: Generate re-
54         // sponse integrity
55         parameter bit EnableDataIntgGen = 0, // 1: Generate re-
56         // sponse data integrity
57         parameter bit EnableDataIntgPt = 0, // 1: Passthrough com-
58         // mand/response data integrity
59         parameter bit SecFifoPtr       = 0, // 1: Duplicated fifo
60         // pointers
61         parameter bit EnableReadback  = 0, // 1: Readback and
62         // check written/read data.
63         parameter bit DataXorAddr     = 0, // 1: XOR data and ad-
64         // dress for address protection
65         localparam int WidthMult      = SramDw / top_pkg::TL_DW,
66         localparam int IntgWidth      = tlul_pkg::DataIntgWidth *
67         WidthMult,
68         localparam int DataOutW       = EnableDataIntgPt ? SramDw
69         + IntgWidth : SramDw
70     ) (
71         input  clk_i,
72         input  rst_ni,
73
74         // TL-UL interface
75         input  tl_h2d_t      tli,
76         output tl_d2h_t      tlo,
77
78         // control interface
79         input  mubi4_t en_ifetch_i,
80
81         // SRAM interface
82         output logic         req_o,
83         output mubi4_t       req_type_o,
84         input               gnt_i,
85         output logic         we_o,
86         output logic [SramAw-1:0] addr_o,
87         output logic [DataOutW-1:0] wdata_o,
88         output logic [DataOutW-1:0] wmask_o,
89         output logic         intg_error_o,

```

```

65 output logic [RsvdWidth-1:0] user_rsvd_o,
66 input [DataOutW-1:0] rdata_i,
67 input rvalid_i,
68 input [1:0] error_i, // 2 bit error [1]:
Uncorrectable, [0]: Correctable
69 output logic compound_txn_in_progress_o,
70 input mubi4_t readback_en_i,
71 output logic readback_error_o,
72 input logic wr_collision_i,
73 input logic write_pending_i
74 );
75
76 localparam int SramByte = SramDw/8;
77 localparam int DataBitWidth =
prim_util_pkg::vbits(SramByte);
78 localparam int WoffsetWidth = (SramByte ==
top_pkg::TL_DBW) ? 1 :
79 DataBitWidth -
prim_util_pkg::vbits(top_pkg::TL_DBW);
80
81 logic error_det; // Internal protocol error checker
82 logic error_internal; // Internal protocol error checker
83 logic wr_attr_error;
84 logic instr_error;
85 logic wr_vld_error;
86 logic rd_vld_error;
87 logic rsp_fifo_error;
88 logic sramreqfifo_error;
89 logic reqfifo_error;
90 logic intg_error;
91 logic tlul_error;
92 logic readback_error;
93 logic sram_byte_readback_error;
94 logic addr_miss_error;
95
96 // readback check
97 logic readback_error_q;
98 if (EnableReadback) begin : gen_cmd_readback_check
99 assign readback_error = sram_byte_readback_error;
100 // permanently latch readback error until reset
101 always_ff @(posedge clk_i or negedge rst_ni) begin
102 if (!rst_ni) begin
103 readback_error_q <= '0;
104 end else if (readback_error) begin
105 readback_error_q <= '1'b1;
106 end
107 end
108 end else begin : gen_no_readback_check
109 logic unused_sram_byte_readback_error;
110 assign unused_sram_byte_readback_error =
sram_byte_readback_error;
111 assign readback_error = '0;
112 assign readback_error_q = '0;
113 end
114
115 // readback error output is permanent and should be used for
alert generation
116 // or other downstream effects
117 assign readback_error_o = readback_error | readback_error_q;
118
119 // integrity check
120 if (CmdIntgCheck) begin : gen_cmd_intg_check
121 tlul_cmd_intg_chk u_cmd_intg_chk (
122 .tl_i(tl_i),
123 .err_o(intg_error)
124 );
125 end else begin : gen_no_cmd_intg_check
126 assign intg_error = '0;
127 end
128
129 // permanently latch integrity error until reset
130 logic intg_error_q;
131 always_ff @(posedge clk_i or negedge rst_ni) begin
132 if (!rst_ni) begin
133 intg_error_q <= '0;
134 end else if (intg_error || rsp_fifo_error || sram-
reqfifo_error || reqfifo_error) begin
135 intg_error_q <= '1'b1;
136 end
137 end
138
139 // integrity error output is permanent and should be used
for alert generation
140 // or other downstream effects
141 assign intg_error_o = intg_error | rsp_fifo_error | sram-
reqfifo_error |
142 reqfifo_error | intg_error_q;
143
144 // wr_attr_error is true if this is a PUT with an unsuppor-
ted request size or mask. This is only
145 // possible if ByteAccess is not allowed.
146 assign wr_attr_error = (ByteAccess == 0) &&
147 (tl_i.a_opcode == PutFullData ||
148 (tl_i.a_mask != '1 || tl_i.a_size !=
2'h2);
149
150 // An instruction type transaction is only valid if
en_ifetch is enabled
151 // If the instruction type is completely invalid, also con-

```

```

sidered an instruction error
152 assign instr_error =
prim_mubi_pkg::mubi4_test_invalid(tl_i.a_user.instr_type) |
153
154 (prim_mubi_pkg::mubi4_test_true_strict(tl_i.a_user.instr_type) &
155 prim_mubi_pkg::mubi4_test_false_loose(en_ifetch_i));
156
157 if (ErrOnWrite == 1) begin : gen_no_writes
158 assign wr_vld_error = tl_i.a_opcode != Get;
159 end else begin : gen_writes_allowed
160 assign wr_vld_error = '1'b0;
161 end
162
163 if (ErrOnRead == 1) begin: gen_no_reads
164 assign rd_vld_error = tl_i.a_opcode == Get;
165 end else begin : gen_reads_allowed
166 assign rd_vld_error = '1'b0;
167 end
168
169 if (2**SramAw == SramDepth) begin : gen_no_addr_chk
170 // The depth of the memory is a power of two. Here, we
only ever get to see addresses that hit
171 // in the memory.
172 assign addr_miss_error = '1'b0;
173 end else begin : gen_addr_chk
174 // The depth of the memory is not a power of two. We have
to signal an error if the address
175 // hits the unmapped range.
176 assign addr_miss_error = tl_i.a_address[DataBitWidth +:
SramAw] >= SramDepth;
177 end
178
179 // tlul protocol check
180 tlul_err u_err (
181 .clk_i,
182 .rst_ni,
183 .tl_i(tl_i),
184 .err_o(tlul_error)
185 );
186
187 // error return is transactional and thus does not used the
"latched" intg_err signal
188 assign error_det = wr_attr_error | wr_vld_error | rd_vld_er-
ror | instr_error |
189 tlul_error | intg_error |
addr_miss_error;
190
191 // from sram_byte to adapter logic
192 tl_h2d_t tl_i_int;
193 // from adapter logic to sram_byte
194 tl_d2h_t tl_o_int;
195 // from sram_byte to rsp_gen
196 tl_d2h_t tl_out;
197
198 // not all parts of tl_i_int are used
199 logic unused_tl_i_int;
200 assign unused_tl_i_int = ^tl_i_int;
201
202 tlul_rsp_intg_gen #(
203 .EnableRspIntgGen(EnableRspIntgGen),
204 .EnableDataIntgGen(EnableDataIntgGen),
205 .RspIntgInIsZero('1'b1)
206 ) u_rsp_gen (
207 .tl_i(tl_out),
208 .tl_o
209 );
210
211 // byte handling for integrity
212 tlul_sram_byte #(
213 .EnableIntg(ByteAccess & EnableDataIntgPt & !ErrOnWrite),
214 .Outstanding(Outstanding),
215 .EnableReadback(EnableReadback)
216 ) u_sram_byte (
217 .clk_i,
218 .rst_ni,
219 .tl_i,
220 .tl_o(tl_out),
221 .tl_sram_o(tl_i_int),
222 .tl_sram_i(tl_o_int),
223 .error_i(error_det),
224 .error_o(error_internal),
225 .alert_o(sram_byte_readback_error),
226 .compound_txn_in_progress_o,
227 .readback_en_i,
228 .wr_collision_i,
229 .write_pending_i
230 );
231
232 typedef struct packed {
233 logic [top_pkg::TL_DBW-1:0] mask ; // Byte mask within the
TL-UL word
234 logic [WoffsetWidth-1:0] woffset ; // Offset of the TL-
UL word within the SRAM word
235 } sram_req_t ;
236
237 typedef struct packed {
238 logic [SramBusBankAW-1:0] addr; // Address of the request
going to the memory.
239 } sram_req_addr_t ;

```



```

239
240 typedef struct packed {
241     logic                                is_read ;
242     logic                                error ;
243     prim_mubi_pkg::mubi4_t               instr_type;
244     logic [top_pkg::TL_SZW-1:0]         size ;
245     logic [top_pkg::TL_AIW-1:0]         source ;
246 } req_t ;
247
248 typedef struct packed {
249     logic [top_pkg::TL_DW-1:0] data ;
250     logic [DataIntgWidth-1:0] data_intg ;
251     logic                                error ;
252 } rsp_t ;
253
254 localparam int SramReqWidth = $bits(sram_req_t);
255 localparam int SramReqFifoWidth = SramReqWidth + (DataXorAd-
dr ? SramBusBankAW : 0);
256 localparam int ReqFifoWidth = $bits(req_t) ;
257 localparam int RspFifoWidth = $bits(rsp_t) ;
258
259 // FIFO signal in case OutStand is greater than 1
260 // If request is latched, {write, source} is pushed to req
fifo.
261 // Req fifo is popped when D channel is acknowledged (v & r)

262 // D channel valid is asserted if it is write request or rsp
fifo not empty if read.
263 logic reqfifo_wvalid, reqfifo_wready;
264 logic reqfifo_rvalid, reqfifo_rready;
265 req_t reqfifo_wdata, reqfifo_rdata;
266
267 logic sramreqfifo_wvalid, sramreqfifo_wready;
268 logic sramreqfifo_rready;
269
270 // An item in u_sramreqfifo is the request itself, together
(if DataXorAddr is nonzero) with some
271 // bits of the request address. These values are in
sram_req_*data and sram_addr_*data, which get
272 // combined to fifo items in sramreqfifo_*data.
273 sram_req_t sram_req_wdata, sram_req_rdata;
274 logic [SramBusBankAW-1:0] sram_addr_wdata, sram_ad-
dr_rdata;
275 logic [SramReqFifoWidth-1:0] sramreqfifo_wdata, sram-
reqfifo_rdata;
276
277 if (DataXorAddr) begin : gen_combine_with_addr
278     assign sramreqfifo_wdata = {sram_addr_wdata,
sram_req_wdata};
279     assign {sram_addr_rdata, sram_req_rdata} = sram-
reqfifo_rdata;
280 end else begin : gen_combine_without_addr
281     assign sramreqfifo_wdata = sram_req_wdata;
282     assign sram_addr_rdata = '0;
283     assign sram_req_rdata = sramreqfifo_rdata;
284 end
285
286 logic rspfifo_wvalid, rspfifo_wready;
287 logic rspfifo_rvalid, rspfifo_rready;
288 rsp_t rspfifo_wdata, rspfifo_rdata;
289
290 logic a_ack, d_ack, sram_ack;
291 assign a_ack = tl_i_int.a_valid & tl_o_int.a_ready ;
292 assign d_ack = tl_o_int.d_valid & tl_i_int.d_ready ;
293 assign sram_ack = req_o & gnt_i ;
294
295 // Valid handling
296 logic d_valid, d_error;
297 always_comb begin
298     d_valid = 1'b0;
299
300     if (reqfifo_rvalid) begin
301         if (reqfifo_rdata.error) begin
302             // Return error response. Assume no request went out
to SRAM
303             d_valid = 1'b1;
304         end else if (reqfifo_rdata.is_read) begin
305             d_valid = rspfifo_rvalid;
306         end else begin
307             // Write without error
308             d_valid = 1'b1;
309         end
310     end else begin
311         d_valid = 1'b0;
312     end
313 end
314
315
316
317 always_comb begin
318     d_error = 1'b0;
319
320     if (reqfifo_rvalid) begin
321         if (reqfifo_rdata.is_read) begin
322             d_error = rspfifo_rdata.error | reqfifo_rdata.error;
323         end else begin
324             d_error = reqfifo_rdata.error;
325         end
326     end else begin
327         d_error = 1'b0;
328     end

```

```

329 end
330
331 logic vld_rd_rsp;
332 assign vld_rd_rsp = d_valid & rspfifo_rvalid &
reqfifo_rdata.is_read;
333 // If the response data is not valid, we set it to an illeg-
al blanking value which is determined
334 // by whether the current transaction is an instruction
fetch or a regular read operation.
335 logic [top_pkg::TL_DW-1:0] error_blanking_data;
336 assign error_blanking_data =
(prim_mubi_pkg::mubi4_test_true_strict(reqfifo_rdata.instr_type)) ?
DataWhenInstrError :
DataWhenError;
337
338 // Since DataWhenInstrError and DataWhenError can be arbit-
rary parameters
339 // we statically calculate the correct integrity values for
these parameters here so that
340 // they do not have to be supplied externally.
341 logic [top_pkg::TL_DW-1:0] unused_instr, unused_data;
342 logic [DataIntgWidth-1:0] error_instr_integ, er-
ror_data_integ;
343
344 tlul_data_integ_enc u_tlul_data_integ_enc_instr (
345     .data_i(DataMaxWidth'(DataWhenInstrError)),
346     .data_intg_o({error_instr_integ, unused_instr})
347 );
348 tlul_data_integ_enc u_tlul_data_integ_enc_data (
349     .data_i(DataMaxWidth'(DataWhenError)),
350     .data_intg_o({error_data_integ, unused_data})
351 );
352
353 logic [DataIntgWidth-1:0] error_blanking_integ;
354 assign error_blanking_integ =
(prim_mubi_pkg::mubi4_test_true_strict(reqfifo_rdata.instr_type)) ?
error_instr_integ :
error_data_integ;
355
356 logic [top_pkg::TL_DW-1:0] d_data;
357 assign d_data = (vld_rd_rsp & ~d_error) ? rsp-
fifo_rdata.data // valid read
: er-
ror_blanking_data; // write or TL-UL error
358
359 // If this a write response with data fields set to 0, we
have to set all ECC bits correctly
360 // since we are using an inverted Hsiao code.
361 logic [DataIntgWidth-1:0] data_intg;
362 assign data_intg = (reqfifo_rdata.error) ? er-
ror_blanking_integ // TL-UL error
: (vld_rd_rsp) // valid read
363
364 prim_secdec_pkg::SecdecInv3932ZeroEcc; // valid write
365
366 // When an error is seen on an incoming transaction it gets
an immediate response without
367 // performing an SRAM request. It may be the transaction re-
ceives a ready the first cycle it is
368 // seen, but if not we force a ready the following cycle.
This avoids factoring the error
369 // calculation into the outgoing ready preventing a feed-
through path from the incoming tilelink
370 // signals to the outgoing tilelink signals.
371 logic missed_err_gnt_d, missed_err_gnt_q;
372
373 // Track whether we've seen an incoming transaction with an
error that didn't get a ready
374 assign missed_err_gnt_d = error_internal & tl_i_int.a_valid
& ~tl_o_int.a_ready;
375
376 always_ff @(posedge clk_i or negedge rst_ni) begin
377     if (!rst_ni) begin
378         missed_err_gnt_q <= 1'b0;
379     end else begin
380         missed_err_gnt_q <= missed_err_gnt_d;
381     end
382 end
383
384 assign tl_o_int = '{
385     d_valid : d_valid ,
386     d_opcode : (d_valid && !reqfifo_rdata.is_read) ? Acces-
sAck : AccessAckData,
387     d_param : '0,
388     d_size : (d_valid) ? reqfifo_rdata.size : '0,
389     d_source : (d_valid) ? reqfifo_rdata.source : '0,
390     d_sink : 1'b0,
391     d_data : d_data,
392     d_user : '{default: '0, data_intg: data_intg},
393     d_error : d_valid && d_error,
394
395     a_ready : (gnt_i | missed_err_gnt_q) & reqfifo_wready &
sramreqfifo_wready
396 };
397
398 // a_ready depends on the FIFO full condition and grant from
SRAM (or SRAM arbiter)
399 // assemble response, including read response, write re-
sponse, and error for unsupported stuff
400
401

```

```

404 // Output to SRAM:
405 // Generate request only when no internal error occurs.
If error occurs, the request should be
406 // dropped and returned error response to the host. So,
error to be pushed to reqfifo.
407 // In this case, it is assumed the request is granted
(may cause ordering issue later?)
408 assign req_o = t_l_i_int.a_valid & reqfifo_wready &
~error_internal;
409 assign req_type_o = t_l_i_int.a_user.instr_type;
410 assign we_o = t_l_i_int.a_valid & (t_l_i_int.a_opcode
inside {PutFullData, PutPartialData});
411 assign addr_o = (t_l_i_int.a_valid) ? t_l_i_int.a_ad-
dress[DataBitWidth+:SramAw] : '0;
412 assign user_rsvd_o = (t_l_i_int.a_valid) ?
t_l_i_int.a_user.rsvd : '0;
413
414 // Support SRAMs wider than the TL-UL word width by mapping
the parts of the
415 // TL-UL address which are more fine-granular than the SRAM
width to the
416 // SRAM write mask.
417 logic [WoffsetWidth-1:0] woffset;
418 if (top_pkg::TL_DW != SramDw) begin : gen_wordwidthadapt
419 assign woffset = t_l_i_int.a_ad-
dress[DataBitWidth-1:prim_util_pkg::vbits(top_pkg::TL_DBW)];
420 end else begin : gen_no_wordwidthadapt
421 assign woffset = '0;
422 end
423
424 // The size of the data/wmask depends on whether passthrough
integrity is enabled.
425 // If passthrough integrity is enabled, the data is concat-
enated with the integrity passed through
426 // the user bits. Otherwise, it is the data only.

427 localparam int DataWidth = EnableDataIntgPt ? top_pkg::TL_DW +
DataIntgWidth : top_pkg::TL_DW;
428
429 // Final combined wmask / wdata
430 logic [WidthMult-1:0][DataWidth-1:0] wmask_combined;
431 logic [WidthMult-1:0][DataWidth-1:0] wdata_combined;
432
433 // Original tlul portion
434 logic [WidthMult-1:0][top_pkg::TL_DW-1:0] wmask_int;
435 logic [WidthMult-1:0][top_pkg::TL_DW-1:0] wdata_int;
436
437 // Integrity portion
438 logic [WidthMult-1:0][DataIntgWidth-1:0] wmask_intg;
439 logic [WidthMult-1:0][DataIntgWidth-1:0] wdata_intg;
440
441 always_comb begin
442 wmask_int = '0;
443 wdata_int = '0;
444
445 if (t_l_i_int.a_valid) begin
446 for (int i = 0 ; i < top_pkg::TL_DW/8 ; i++) begin
447 wmask_int[woffset][8*i+:8] =
{8{t_l_i_int.a_mask[i]}};
448 wdata_int[woffset][8*i+:8] = (t_l_i_int.a_mask[i] &&
we_o) ? t_l_i_int.a_data[8*i+:8] : '0;
449 end
450 end
451 end
452
453 always_comb begin
454 wmask_intg = '0;
455 wdata_intg = '0;
456
457 if (t_l_i_int.a_valid) begin
458 wmask_intg[woffset] = {DataIntgWidth{1'b1}};
459 wdata_intg[woffset] = t_l_i_int.a_user.data_intg;
460 end
461 end
462
463 for (genvar i = 0 ; i < WidthMult ; i++) begin :
gen_write_output
464 if (EnableDataIntgPt) begin : gen_combined_output
465 assign wmask_combined[i] = {wmask_intg[i],
wmask_int[i]};
466 assign wdata_combined[i] = {wdata_intg[i],
wdata_int[i]};
467 end else begin : gen_ft_output
468 logic unused_w;
469 assign wmask_combined[i] = wmask_int[i];
470 assign wdata_combined[i] = wdata_int[i];
471 assign unused_w = |wmask_intg & |wdata_intg;
472 end
473 end
474
475 assign wmask_o = wmask_combined;
476 assign wdata_o = wdata_combined;
477
478 assign reqfifo_wvalid = a_ack ; // Push to FIFO only when
granted
479 assign reqfifo_wdata = '{
480 is_read: t_l_i_int.a_opcode == Get,
481 error: error_internal,
482 instr_type: t_l_i_int.a_user.instr_type,
483 size: t_l_i_int.a_size,

```

```

484 source: t_l_i_int.a_source
485 }; // Store the request only. Doesn't have to store data
486 assign reqfifo_rready = d_ack ;
487
488 // push together with ReqFIFO, pop upon returning read
489 assign sram_req_wdata = '{
490 mask : t_l_i_int.a_mask,
491 woffset : woffset
492 };
493 assign sramreqfifo_wvalid = sram_ack & ~we_o;
494 assign sramreqfifo_rready = rspfifo_wvalid;
495
496 assign rspfifo_wvalid = rvalid_i & reqfifo_rvalid;
497
498 assign sram_addr_wdata = t_l_i_int.a_ad-
dress[DataBitWidth+:SramBusBankAw];
499
500 // Make sure only requested bytes are forwarded
501 logic [WidthMult-1:0][DataWidth-1:0] rdata_resaped;
502 logic [DataWidth-1:0] rdata_tlword;
503
504 // This just changes the array format so that the correct
word can be selected by indexing.
505 assign rdata_resaped = rdata_i;
506
507 if (EnableDataIntgPt) begin : gen_no_rmask
508 always_comb begin
509 // If the read mask is set to zero, all read data is
zeroed out by the mask.
510 // We have to set the ECC bits accordingly since we are
using an inverted Hsiao code.
511 rdata_tlword = prim_secded_pkg::SecdedInv3932ZeroWord;
512 // Otherwise, if at least one mask bit is nonzero, we
are passing through the integrity.
513 // In that case we need to feed back the entire word
since otherwise the integrity
514 // will not calculate correctly.
515 if (!sram_req_rdata.mask) begin
516 // Select correct word.
517 if (DataXorAddr) begin : gen_data_xor_addr
518 // When DataXorAddr is enabled, on a read, the ad-
dress is XORed with the data fetched from
519 // the memory in the underlying memory controller
(e.g., flash controller). At this point,
520 // the address is again removed. If the address in
the read transaction has been modified,
521 // e.g., due to a fault, rdata now contains faulty
data, which is detected by the
522 // integrity mechanism.
523 rdata_tlword = {
524 rdata_resaped[sram_req_rdata.woffset][DataW-
idth-1:top_pkg::TL_DW],
525 rdata_resaped[sram_req_rdata.woffset]
[top_pkg::TL_DW-1:0] ^
526 {{{(top_pkg::TL_DW-SramBusBankAw){1'b0}}},
sram_addr_rdata}
527 };
528 end else begin: gen_no_data_xor_addr
529 rdata_tlword =
rdata_resaped[sram_req_rdata.woffset];
530 end
531 end
532 end
533 end else begin : gen_rmask
534 logic [DataWidth-1:0] rmask;
535 always_comb begin
536 rmask = '0;
537 for (int i = 0 ; i < top_pkg::TL_DW/8 ; i++) begin
538 rmask[8*i+:8] = {8{sram_req_rdata.mask[i]}};
539 end
540 end
541 // Select correct word and mask it.
542 assign rdata_tlword =
rdata_resaped[sram_req_rdata.woffset] & rmask;
543 end
544
545 assign rspfifo_wdata = '{
546 data : rdata_tlword[top_pkg::TL_DW-1:0],
547 data_intg : EnableDataIntgPt ? rdata_tlword[DataWidth-1 -:
DataIntgWidth] : '0,
548 error : error_i[1] // Only care for Uncorrectable er-
ror
549 };
550 assign rspfifo_rready = reqfifo_rdata.is_read &
~reqfifo_rdata.error & reqfifo_rready;
551
552 // This module only cares about uncorrectable errors.
553 logic unused_error;
554 assign unused_error = error_i[0];
555
556 // FIFO instance: REQ, RSP
557
558 // ReqFIFO is to store the Access type to match to the Re-
sponse data.
559 // For instance, SRAM accepts the write request but
doesn't return the
560 // acknowledge. In this case, it may be hard to determine
when the D
561 // response for the write data should send out if reads/

```

```

writes are
562 // interleaved. So, to make it in-order (even TL-UL al-
lows out-of-order
563 // responses), storing the request is necessary. And if
the read entry
564 // is write op, it is safe to return the response right
away. If it is
565 // read request, then D response is waiting until read
data arrives.
566 prim_fifo_sync #(
567   .Width      (ReqFifoWidth),
568   .Pass       (1'b0),
569   .Depth      (Outstanding),
570   .NeverClears (1'b1),
571   .Secure     (SecFifoPtr)
572 ) u_reqfifo (
573   .clk_i,
574   .rst_ni,
575   .clr_i (1'b0),
576   .wvalid_i(reqfifo_wvalid),
577   .wready_o(reqfifo_wready),
578   .wdata_i (reqfifo_wdata),
579   .rvalid_o(reqfifo_rvalid),
580   .rready_i(reqfifo_rready),
581   .rdata_o (reqfifo_rdata),
582   .full_o (),
583   .depth_o (),
584   .err_o (reqfifo_error)
585 );
586
587 // sramreqfifo:
588 // While the ReqFIFO holds the request until it is sent
back via TL-UL, the
589 // sramreqfifo only needs to hold the mask and word off-
set until the read
590 // data returns from memory.
591 prim_fifo_sync #(
592   .Width      (SramReqFifoWidth),
593   .Pass       (1'b0),
594   .Depth      (Outstanding),
595   .NeverClears (1'b1),
596   .Secure     (SecFifoPtr),
597   .OutputZeroIfEmpty (1)
598 ) u_sramreqfifo (
599   .clk_i,
600   .rst_ni,
601   .clr_i (1'b0),
602   .wvalid_i(sramreqfifo_wvalid),
603   .wready_o(sramreqfifo_wready),
604   .wdata_i (sramreqfifo_wdata),
605   .rvalid_o(),
606   .rready_i(sramreqfifo_rready),
607   .rdata_o (sramreqfifo_rdata),
608   .full_o (),
609   .depth_o (),
610   .err_o (sramreqfifo_error)
611 );
612
613 if (!DataXorAddr) begin : gen_no_data_xor_addr_fifo
614 // If u_sramreqfifo doesn't contain any address data,
nothing will be reading sram_addr_wdata or
615 // sram_addr_rdata. Tie them off with an unused signal.
616 logic unused_sram_addresses;
617 assign unused_sram_addresses = ^{sram_addr_wdata, sram_ad-
dr_rdata};
618 end
619
620 // Rationale having #Outstanding depth in response FIFO.
621 // In normal case, if the host or the crossbar accepts
the response data,
622 // response FIFO isn't needed. But if in any case it has
a chance to be
623 // back pressured, the response FIFO should store the re-
turned data not to
624 // lose the data from the SRAM interface. Remember, SRAM
interface doesn't
625 // have back-pressure signal such as read_ready.
626 prim_fifo_sync #(
627   .Width      (RspFifoWidth),
628   .Pass       (1'b1),
629   .Depth      (Outstanding),
630   .NeverClears (1'b1),
631   .Secure     (SecFifoPtr)
632 ) u_rspfifo (
633   .clk_i,
634   .rst_ni,
635   .clr_i (1'b0),
636   .wvalid_i(rspfifo_wvalid),
637   .wready_o(rspfifo_wready),
638   .wdata_i (rspfifo_wdata),
639   .rvalid_o(rspfifo_rvalid),
640   .rready_i(rspfifo_rready),
641   .rdata_o (rspfifo_rdata),
642   .full_o (),
643   .depth_o (),
644   .err_o (rsp_fifo_error)
645 );
646
647 // below assertion fails when SRAM rvalid is asserted even
though ReqFifo is empty
648 `ASSERT(rvalidHighReqFifoEmpty, rvalid_i |>- reqfifo_rvalid)

```

```

649
650 // below assertion fails when outstanding value is too small
(SRAM rvalid is asserted
651 // even though the RspFifo is full)
652 `ASSERT(rvalidHighWhenRspFifoFull, rvalid_i |>- rsp-
fifo_wready)
653
654 // If both ErrOnWrite and ErrOnRead are set, this block is
useless
655 `ASSERT_INIT(adapterNoReadOrWrite, (ErrOnWrite & ErrOnRead)
== 0)
656
657 `ASSERT_INIT(SramDwHasByteGranularity_A, SramDw % 8 == 0)
658 `ASSERT_INIT(SramDwIsMultipleOfTlulWidth_A, SramDw %
top_pkg::TL_DW == 0)
659 // Either the memory has a power-of-two depth and the ad-
dress width perfectly matches, or the
660 // depth is not a power of two but the address width is
still minimal.
661 `ASSERT_INIT(SramAwCorrectlySizedForDepth_A,
662   ((SramDepth & (SramDepth - 1)) == 0) ? (2**SramAw ==
SramDepth) :
663   ($clog2(SramDepth) == SramAw))
664
665 // These parameter options cannot both be true at the same
time
666 `ASSERT_INIT(DataIntgOptions_A, ~(EnableDataIntgGen & En-
ableDataIntgPt))
667
668 // Make sure that outputs are defined (a special case for
tl_o is explained separately below)
669 `ASSERT_KNOWN(ReqOutKnown_A, req_o )
670 `ASSERT_KNOWN(WeOutKnown_A, we_o )
671 `ASSERT_KNOWN(AddrOutKnown_A, addr_o )
672 `ASSERT_KNOWN(WdataOutKnown_A, wdata_o )
673 `ASSERT_KNOWN(WmaskOutKnown_A, wmask_o )
674
675 // We'd like to claim that the payload of the TL output is
known, but this isn't necessarily true!
676 // This block is just an adapter that converts from an SRAM
interface to a TL interface. To make
677 // the assertion true, we need to weaken it to say that that
tl_o is only X if the SRAM supplied
678 // that X.
679 //
680 // This is a bit tricky to track because SRAM responses get
stored in u_rspfifo. Assuming that the
681 // FIFO doesn't manufacture X's (an assertion in
prim_fifo_sync), the only stage of the path
682 // needed in this file is the following:
683 `ASSERT_KNOWN(TlOutValidKnown_A, tl_o.d_valid)
684 `ASSERT(TlOutKnownIfFifoKnown_A, !$isunknown(rspfifo_rdata)
-> !$isunknown(tl_o))
685
686 // The definition of d_valid leads to the assertion below.
687 `ASSERT(DValidNeedsReqFifoRvalid_A, d_valid ->
reqfifo_rvalid)
688 endmodule

```

9. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram_racl.sv (146 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8  * Tile-Link UL adapter for SRAM-like devices with RACL sup-
port
9  *
10  * - Intentionally omitted BaseAddr in case of multiple memory
maps are used in a SoC,
11  *   it means that aliasing can happen if target device size
in TL-UL crossbar is bigger
12  *   than SRAM size
13  * - At most one of EnableDataIntgGen / EnableDataIntgPt can
be enabled. However it
14  *   possible for both to be disabled.
15  *   A module can neither generate an integrity response nor
pass through any pre-existing
16  *   integrity. This might be the case for non-security crit-
ical memories where there is
17  *   no stored integrity AND another entity upstream is
already generating returning integrity.
18  *   There is however no case where EnableDataIntgGen and En-
ableDataIntgPt are both true.
19  */
20 module tlul_adapter_sram_racl
21 import tlul_pkg::*;
22 import prim_mubi_pkg::mubi4_t;
23 #(
24   parameter int SramAw = 12,

```

```

25 parameter int SramDw          = 32,          // Must be
multiple of the TL width
26 parameter int SramDepth       = 2**SramAw,    // Must be <=
2**SramAw
27 parameter int Outstanding     = 1,          // Only one
request is accepted
28 parameter int SramBusBankAW   = 12,          // SRAM bus
address width of the SRAM bank. Only
29                                     // used when
DataXorAddr=1.
30 parameter bit ByteAccess      = 1,          // 1: Enables
sub-word write transactions. Note that
31                                     // this
results in read-modify-write operations
32                                     // for in-
tegrity re-generation if
33                                     // En-
ableDataIntgPt is set to 1.
34 parameter bit ErrOnWrite      = 0,          // 1: Writes
not allowed, automatically error
35 parameter bit ErrOnRead      = 0,          // 1: Reads
not allowed, automatically error
36 parameter bit CmdIntgCheck    = 0,          // 1: Enable
command integrity check

37 parameter bit EnableRspIntgGen = 0,          // 1: Generate
response integrity

38 parameter bit EnableDataIntgGen = 0,          // 1: Generate
response data integrity
39 parameter bit EnableDataIntgPt = 0,          // 1:
Passthrough command/response data integrity
40 parameter bit SecFifoPtr      = 0,          // 1: Duplic-
ated fifo pointers

41 parameter bit EnableReadback  = 0,          // 1: Readback
and check written/read data.

42 parameter bit DataXorAddr     = 0,          // 1: XOR data
and address for address protection
43 parameter bit EnableRacl      = 0,          // 1: Enable
RACL checks on access
44 parameter bit RclErrorRsp     = EnableRacl, // 1: Return
TLUL error on RACL errors
45 parameter int RclPolicySelNumRanges = 1,    // Number of
ranges with a RACL policy
46 localparam int WidthMult      = SramDw / top_pkg::TL_DW,
47 localparam int IntgWidth      = tlul_pkg::DataIntgWidth *
WidthMult,
48 localparam int DataOutW       = EnableDataIntgPt ? SramDw
+ IntgWidth : SramDw
49 ) (
50 input  clk_i,
51 input  rst_ni,
52
53 // TL-UL interface
54 input  tl_h2d_t      tl_i,
55 output tl_d2h_t      tl_o,
56
57 // control interface
58 input  mubi4_t en_ifetch_i,
59
60 // SRAM interface
61 output logic        req_o,
62 output mubi4_t      req_type_o,
63 input              gnt_i,
64 output logic        we_o,
65 output logic [SramAw-1:0] addr_o,
66 output logic [DataOutW-1:0] wdata_o,
67 output logic [DataOutW-1:0] wmask_o,
68 output logic        intg_error_o,
69 output logic [RsvdWidth-1:0] user_rsvd_o,
70 input              [DataOutW-1:0] rdata_i,
71 input              rvalid_i,
72 input              [1:0] error_i, // 2 bit error [1:
Uncorrectable, [0]: Correctable
73 output logic        compound_txn_in_progress_o,
74 input  mubi4_t      readback_en_i,
75 output logic        readback_error_o,
76 input  logic        wr_collision_i,
77 input  logic        write_pending_i,
78 // RACL interface
79 input  top_racl_pkg::racl_policy_vec_t racl_policies_i,
80 output top_racl_pkg::racl_error_log_t racl_error_o,
81 input  top_racl_pkg::racl_range_t [RaclPolicySelNum-
Ranges-1:0] racl_policy_sel_ranges_i
82 );
83 tl_h2d_t tl_h2d_filtered;
84 tl_d2h_t tl_d2h_filtered;
85
86 tlul_adapter_racl #(
87     .EnableRacl      ( EnableRacl ),
88     .RaclErrorRsp    ( RclErrorRsp ),
89     .RaclPolicySelNumRanges ( RaclPolicySelNumRanges )
90 ) tlul_adapter_racl(
91     .clk_i      ( clk_i ),
92     .rst_ni     ( rst_ni ),
93     .tl_h2d_i   ( tl_i ),
94     .tl_d2h_o   ( tl_o ),
95     .tl_filtered_h2d_o ( tl_h2d_filtered ),
96     .tl_filtered_d2h_i ( tl_d2h_filtered ),

```

```

97     .racl_policies_i      ( racl_policies_i ),
98     .racl_error_o         ( racl_error_o ),
99     .racl_policy_sel_ranges_i ( racl_policy_sel_ranges_i )
100 );
101
102 tlul_adapter_sram #(
103     .SramAw      ( SramAw ),
104     .SramDw      ( SramDw ),
105     .SramDepth   ( SramDepth ),
106     .Outstanding ( Outstanding ),
107     .SramBusBankAW ( SramBusBankAW ),
108     .ByteAccess  ( ByteAccess ),
109     .ErrOnWrite  ( ErrOnWrite ),
110     .ErrOnRead   ( ErrOnRead ),
111     .CmdIntgCheck ( CmdIntgCheck ),
112     .EnableRspIntgGen ( EnableRspIntgGen ),
113     .EnableDataIntgGen ( EnableDataIntgGen ),
114     .EnableDataIntgPt ( EnableDataIntgPt ),
115     .SecFifoPtr   ( SecFifoPtr ),
116     .EnableReadback ( EnableReadback ),
117     .DataXorAddr  ( DataXorAddr )
118 ) tlul_adapter_sram (
119     .clk_i,
120     .rst_ni,
121     .tl_i(tl_h2d_filtered),
122     .tl_o(tl_d2h_filtered),
123     .en_ifetch_i,
124     .req_o,
125     .req_type_o,
126     .gnt_i,
127     .we_o,
128     .addr_o,
129     .wdata_o,
130     .wmask_o,
131     .intg_error_o,
132     .user_rsvd_o,
133     .rdata_i,
134     .rvalid_i,
135     .error_i,
136     .compound_txn_in_progress_o,
137     .readback_en_i,
138     .readback_error_o,
139     .wr_collision_i,
140     .write_pending_i
141 );
142
143 // Ensure that RACL signals are not undefined
144 `ASSERT_KNOWN(RaclAdapterSramErrorKnown_A, racl_er-
ror_o.valid)
145
146 endmodule

```

10. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_vh.sv (149 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 module tlul_adapter_vh
8 import tlul_pkg::*;
9 import prim_mubi_pkg::mubi4_t;
10 #(
11     parameter int ADDR_WIDTH = top_pkg::TL_AW,
12     parameter int DATA_WIDTH = top_pkg::TL_DW,
13     parameter int MASK_WIDTH = DATA_WIDTH >> 3,
14     parameter int USER_WIDTH = 32,
15     parameter int ID_WIDTH   = top_pkg::TL_AIW,
16
17     parameter bit [ADDR_WIDTH-1:0] VH_REGISTER_ADDRESS_OFFSET =
32'h0000_0100,
18
19     parameter bit EnableDataIntgGen = 1,
20     parameter bit EnableRspDataIntgCheck = 1
21 ) (
22     input          clk_i,
23     input          rst_ni,
24
25     output         tl_h2d_t tl_o,
26     input          tl_d2h_t tl_i,
27
28     // Valid-Hold device interface (VH to TLUL).
29     input logic    dv_i,
30     output logic   hld_o,
31     input logic [ADDR_WIDTH-1:0] addr_i,
32     input logic    write_i,
33     input logic [DATA_WIDTH-1:0] wdata_i,
34     input logic [MASK_WIDTH-1:0] wstrb_i,
35     input logic [2:0] size_i,
36     output logic [DATA_WIDTH-1:0] rdata_o,
37     output logic    error_o,
38     input logic    last_i,
39     input logic [USER_WIDTH-1:0] user_i,
40     input logic [ID_WIDTH-1:0] id_i,
41

```

```

42 // Valid-Hold host interface (VH to internal registers). The
signals from the VH device interface
43 // are routed to the VH host interface for every internal
access, see the 'internal_access' signal.
44 output logic int_dv_o,
45 input logic int_hld_i,
46 output logic [ADDR_WIDTH-1:0] int_addr_o,
47 output logic int_write_o,
48 output logic [DATA_WIDTH-1:0] int_wdata_o,
49 output logic [MASK_WIDTH-1:0] int_wstrb_o,
50 output logic [2:0] int_size_o,
51 input logic [DATA_WIDTH-1:0] int_rdata_i,
52 input logic int_error_i,
53 output logic int_last_o,
54 output logic [USER_WIDTH-1:0] int_user_o,
55 output logic [ID_WIDTH-1:0] int_id_o
56 );
57
58 // The adapter distinguishes between two types of VH ac-
cesses: those going to an internal register
59 // file and those targeting OpenTitan registers, with the
latter ones being translated into TLUL
60 // requests while the other requests are relayed to the in-
ternal register file.
61 logic internal_access;
62 assign internal_access = addr_i >= VH_RE-
GISTER_ADDRESS_OFFSET;
63
64 // Differentiate between two levels of acknowledgements:
65 // - 'req_ack': A host-to-device request is acknowledged by
the device, meaning that the response
66 // is pending.
67 // - 'resp_ack': The device-to-host response is acknow-
ledged.
68 logic pending_d, pending_q;
69 logic req_ack, resp_ack;
70
71 assign req_ack = dv_i & t_l_i.a_ready & t_l_o.a_valid & ~in-
ternal_access;
72 assign resp_ack = dv_i & t_l_o.d_ready & t_l_i.d_valid & ~in-
ternal_access;
73
74 always_comb begin
75 pending_d = pending_q;
76 if (req_ack) begin
77 pending_d = 1'b1;
78 end else if (resp_ack) begin
79 pending_d = 1'b0;
80 end
81 end
82
83 always_ff @(posedge clk_i or negedge rst_ni) begin
84 if (!rst_ni) begin
85 pending_q <= 1'b0;
86 end else begin
87 pending_q <= pending_d;
88 end
89 end
90
91 // Only make a new request (through 'a_valid') to the device
when none are pending. If the
92 // integrity checks are enabled, the 'a_user' fields will be
set by the corresponding module (see
93 // 'tlul_cmd_intg_gen').
94 tlul_pkg::tl_h2d_t t_l_o_pre;
95 assign t_l_o_pre = '{
96 a_valid: dv_i & ~pending_q & ~internal_access,
97 a_opcode: ~write_i ? Get : (&wstrb_i ? PutFullData : Put-
PartialData),
98 a_size: top_pkg::TL_SZW'(size_i),
99 a_mask: wstrb_i,
100 a_source: id_i,
101 a_address: addr_i,
102 a_data: wdata_i,
103 a_user: '{default: '0, instr_type:
prim_mubi_pkg::MuB14False},
104 d_ready: 1'b1,
105 // unused
106 a_param: 3'h0
107 };
108
109 // Accesses to device registers are acknowledged with the
TLUL d-channel handshake.
110 assign hld_o = !pending_q && internal_access && !int_hld_i ?
1'b0 :
111 pending_q && t_l_o.d_ready && t_l_i.d_valid ?
1'b0 : 1'b1;
112
113 assign rdata_o = internal_access ? int_rdata_i :
t_l_i.d_data;
114
115 // Route signals to the VH host interface.
116 assign int_dv_o = dv_i & internal_access;
117 assign int_addr_o = addr_i;
118 assign int_write_o = write_i;
119 assign int_wdata_o = wdata_i;

```

```

120 assign int_wstrb_o = wstrb_i;
121 assign int_size_o = size_i;
122 assign int_last_o = last_i;
123 assign int_user_o = user_i;
124 assign int_id_o = id_i;
125
126 tlul_cmd_intg_gen #(
127 .EnableDataIntgGen (EnableDataIntgGen)
128 ) u_cmd_intg_gen (
129 .t_l_i ( t_l_o_pre ),
130 .t_l_o ( t_l_o )
131 );
132
133 logic intg_err_chk;
134 tlul_rsp_intg_chk #(
135 .EnableRspDataIntgCheck(EnableRspDataIntgCheck)
136 ) u_rsp_chk (
137 .t_l_i ( t_l_i ),
138 .err_o ( intg_err_chk )
139 );
140
141 assign error_o = t_l_i.d_error | int_error_i | intg_err_chk;
142
143 // Make sure the VH parameters are compatible with TLUL
datapath widths.
144 `ASSERT_INIT(TlInvalidAddrWidth_A, ADDR_WIDTH ==
top_pkg::TL_AW)
145 `ASSERT_INIT(TlInvalidDataWidth_A, DATA_WIDTH ==
top_pkg::TL_DW)
146 `ASSERT_INIT(TlInvalidMaskWidth_A, MASK_WIDTH ==
top_pkg::TL_DBW)
147 `ASSERT_INIT(TlInvalidUserWidth_A, ID_WIDTH ==
top_pkg::TL_AIW)
148
149 endmodule

```

11. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_assert.sv (502 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 // Protocol checker for TL-UL ports using assertions. Supports in-
terface-widths
6 // up to 64bit.
7
8 `include "prim_assert.sv"
9
10 module tlul_assert #(
11 parameter EndpointType = "Device" // can be either "Host" or
"Device"
12 ) (
13 input clk_i,
14 input rst_ni,
15
16 // tile link ports
17 input tlul_pkg::tl_h2d_t h2d,
18 input tlul_pkg::tl_d2h_t d2h
19 );
20
21 `ifndef VERILATOR
22 `ifndef SYNTHESIS
23
24 `ifdef UVM
25 import uvm_pkg::*;
26 `include "uvm_macros.svh"
27 `endif
28 import tlul_pkg::*;
29 import top_pkg::*;
30
31 // To test TL-UL error cases in UVM tests, pass the
tlul_assert_en argument in the UVM config db,
32 // which will cause disable_sva to be set to 1. This dis-
ables some assertions about the
33 // well-formedness of the TL input.
34 bit disable_sva;
35
36 default disable iff disable_sva || !rst_ni;
37 default clocking @(posedge clk_i); endclocking
38
39 ///////////////////////////////////////////////////
40 // check requests and responses //
41 ///////////////////////////////////////////////////
42
43 // There are up to 2**TL_AIW possible source-IDs. Below ar-
ray "pend_req" has one entry
44 // for each source-ID. Each entry has the following fields:
45 // - pend : is set to 1 to indicate up to 1 pending re-
quest for the source ID
46 // - opcode : "Get" requires "AccessAckData" response,
"Put" require "AccessAck"
47 // - size : d_size of response must match a_size of re-
quest
48 // - mask : is used to allow X for bytes whose mask bit
is 0
49 typedef struct packed {

```



```

50 bit pend; // set to 1 to indicate
a pending request
51 tl_a_op_e opcode;
52 logic [top_pkg::TL_SZW-1:0] size;
53 logic [top_pkg::TL_DBW-1:0] mask;
54 } pend_req_t;
55
56 pend_req_t [2**TL_AIW-1:0] pend_req;
57
58 // Current request
59 // Current request
60 // Current request
61 pend_req_t curr_req;
62 always_comb begin
63 // Indicates whether a request is being accepted.
64 curr_req.pend = h2d.a_valid & d2h.a_ready;
65 // Properties of current request.
66 curr_req.opcode = h2d.a_opcode;
67 curr_req.size = h2d.a_size;
68 curr_req.mask = h2d.a_mask;
69 end
70
71 // Forward the current request for checking?
72 // This supports combinational responses. Only defined when
'd_valid' asserted, i.e. it may
73 // be used unqualified in the 'consequent' of an assertion,
where the 'antecedent' tests
74 // assertion of 'd_valid'.
75 logic curr_fwd;
76 assign curr_fwd = curr_req.pend & (d2h.d_source ==
h2d.a_source);
77
78 logic [7:0] a_mask, d_mask;
79 logic [63:0] a_data, d_data;
80 assign a_mask = 8'(h2d.a_mask);
81 assign a_data = 64'(h2d.a_data);
82 assign d_mask = 8'(curr_fwd ? curr_req.mask :
pend_req[d2h.d_source].mask);
83 assign d_data = 64'(d2h.d_data);
84
85 // keep track of pending requests
86 // keep track of pending requests
87 // keep track of pending requests
88
89 always_ff @(posedge clk_i or negedge rst_ni) begin
90 if (!rst_ni) begin
91 pend_req <= '0;
92 end else begin
93 // Update pend_req if there is an A channel transaction
on this cycle.
94 //
95 // Note that we can safely do this if there was a com-
binatorial response. In that case, it
96 // will be removed again just below.
97 if (curr_req.pend) begin
98 pend_req[h2d.a_source] <= curr_req;
99 end
100
101 // If there is a D channel transaction on this cycle,
clear any pending request for the
102 // associated d_source.
103 if (d2h.d_valid && h2d.d_ready) begin
104 pend_req[d2h.d_source] <= '0;
105 end
106 end
107 end
108
109 // define sequences for request checks
110 // define sequences for request checks
111 // define sequences for request checks
112
113 sequence h2d_pre_S;
114 h2d.a_valid;
115 endsequence
116
117 // a_opcode: only 3 opcodes are legal for requests
118 sequence legalAOpcode_S;
119 (h2d.a_opcode == PutFullData) ||
120 (h2d.a_opcode == Get) ||
121 (h2d.a_opcode == PutPartialData);
122 endsequence
123
124 // a_param is reserved
125 sequence legalAParam_S;
126 h2d.a_param == '0;
127 endsequence
128
129 // a_size: Size shouldn't be greater than the bus width in
TL-UL (not in TL-UH)
130 // This assertion can be covered by below
131 // (a_size must less than or equal to ones of
a_mask)
132
133 // a_size: 2**a_size must greater than or equal to $count-
ones(a_mask) for PutPartial and Get
134 sequence sizeGTEMask_S;
135 (h2d.a_opcode == PutFullData) || ((1 << h2d.a_size) >=
$countones(h2d.a_mask));
136 endsequence
137
138 // a_size: 2**a_size must equal to $countones(a_mask) for

```

```

PutFull
139 sequence sizeMatchesMask_S;
140 (h2d.a_opcode inside {PutPartialData, Get}) ||
141 ((1 << h2d.a_size) == $countones(h2d.a_mask));
142 endsequence
143
144 // a_source: there should be no more than one pending re-
quest per each source ID
145 sequence pendingReqPerSrc_S;
146 pend_req[h2d.a_source].pend == 0;
147 endsequence
148
149 // a_address must be aligned to a_size: a_address & ((1 <<
a_size) - 1) == 0
150 sequence addrSizeAligned_S;
151 (h2d.a_address & ((1 << h2d.a_size)-1)) == '0;
152 endsequence
153
154 // a_mask must be contiguous for Get and PutFullData re-
quests
155 // the spec talks about "naturally aligned". Does this
mean that bit [0] of
156 // mask is always 1? If that's true, then below code
could be much simpler.
157 // However, the spec shows a timing diagram where bit 0
of mask is 0.
158 sequence contigMask_pre_S;
159 h2d.a_opcode != PutPartialData;
160 endsequence
161
162 sequence contigMask_S;
163 $countones(h2d.a_mask ^
{h2d.a_mask[$bits(h2d.a_mask)-2:0], 1'b0}) <= 2;
164 endsequence
165
166 // a_data must be known for opcode == Put*(depending on mask
bits)
167 sequence aDataKnown_pre_S;
168 (h2d.a_opcode != Get);
169 endsequence
170
171 sequence aDataKnown_S;
172 // no check if this lane mask is inactive
173 ((!a_mask[0]) || (a_mask[0] && !$isunknown(a_data[8*0 +
8])) &&
174 ((!a_mask[1]) || (a_mask[1] && !$isunknown(a_data[8*1 +
8])) &&
175 ((!a_mask[2]) || (a_mask[2] && !$isunknown(a_data[8*2 +
8])) &&
176 ((!a_mask[3]) || (a_mask[3] && !$isunknown(a_data[8*3 +
8])) &&
177 ((!a_mask[4]) || (a_mask[4] && !$isunknown(a_data[8*4 +
8])) &&
178 ((!a_mask[5]) || (a_mask[5] && !$isunknown(a_data[8*5 +
8])) &&
179 ((!a_mask[6]) || (a_mask[6] && !$isunknown(a_data[8*6 +
8])) &&
180 ((!a_mask[7]) || (a_mask[7] && !$isunknown(a_data[8*7 +
8])));
181 endsequence
182
183 // define sequences for request checks
184 // define sequences for request checks
185 // define sequences for request checks
186
187 sequence d2h_pre_S;
188 d2h.d_valid;
189 endsequence
190
191 // d_opcode: if request was Get, then response must be Ac-
cessAckData
192 sequence respOpcode_S;
193 d2h.d_opcode == ((curr_fwd ? curr_req.opcode :
pend_req[d2h.d_source].opcode) == Get) ?
AccessAckData : AccessAck;
194 endsequence
195
196 // d_param is reserved
197 sequence legalDParam_S;
198 d2h.d_param == '0;
199 endsequence
200
201 // d_size must equal the a_size of the corresponding request
202 sequence respSzEqReqSz_S;
203 d2h.d_size == (curr_fwd ? curr_req.size :
pend_req[d2h.d_source].size);
204 endsequence
205
206 // d_source: each non-combinational response should have a
pending request using same source ID
207 sequence respMustHaveReq_S;
208 curr_fwd | pend_req[d2h.d_source].pend;
209 endsequence
210
211 // d_data must be known for AccessAckData (depending on mask
bits)
212 sequence dDataKnown_pre_S;
213 d2h.d_opcode == AccessAckData;
214 endsequence
215
216

```

```

217 sequence dDataKnown_S;
218 // no check if this lane mask is inactive
219 ((!d_mask[0]) || (d_mask[0] && !$isunknown(d_data[8*0 +
8]))) &&
220 ((!d_mask[1]) || (d_mask[1] && !$isunknown(d_data[8*1 +
8]))) &&
221 ((!d_mask[2]) || (d_mask[2] && !$isunknown(d_data[8*2 +
8]))) &&
222 ((!d_mask[3]) || (d_mask[3] && !$isunknown(d_data[8*3 +
8]))) &&
223 ((!d_mask[4]) || (d_mask[4] && !$isunknown(d_data[8*4 +
8]))) &&
224 ((!d_mask[5]) || (d_mask[5] && !$isunknown(d_data[8*5 +
8]))) &&
225 ((!d_mask[6]) || (d_mask[6] && !$isunknown(d_data[8*6 +
8]))) &&
226 ((!d_mask[7]) || (d_mask[7] && !$isunknown(d_data[8*7 +
8])));
227 endsequence
228 ///////////////////////////////////////////////////
229 // define sequences for d_error checks //
230 ///////////////////////////////////////////////////
231
232
233 sequence d_error_pre_S;
234 h2d.a_valid && d2h.a_ready;
235 endsequence
236
237 sequence legalA0pcodeErr_S;
238 !(h2d.a_opcode inside {PutFullData, Get, PutPartialData});
239 endsequence
240
241 sequence sizeGTEMaskErr_S;
242 (1 << h2d.a_size) < $countones(h2d.a_mask);
243 endsequence
244
245 sequence sizeMatchesMaskErr_S;
246 (h2d.a_opcode == PutFullData) && ((1 << h2d.a_size) !=
$countones(h2d.a_mask));
247 endsequence
248
249 sequence addrSizeAlignedErr_S;
250 (h2d.a_address & ((1 << h2d.a_size)-1)) != '0;
251 endsequence
252
253 ///////////////////////////////////////////////////
254 // assemble properties and check //
255 ///////////////////////////////////////////////////
256
257 // note: use negedge clk to avoid possible race conditions
258
259 // in this case all signals coming from the device side have
an assumed property
259 if (EndpointType == "Host") begin : gen_host
260 // h2d
261 if (1) begin : gen_h2d
262 legalA0pcode_A:
263 assert property (h2d_pre_S |-> legalA0pcode_S)
264 else `ASSERT_ERROR(legalA0pcode_A)
265 legalAParam_A:
266 assert property (h2d_pre_S |-> legalAParam_S)
267 else `ASSERT_ERROR(legalAParam_A)
268 sizeGTEMask_A:
269 assert property (h2d_pre_S |-> sizeGTEMask_S)
270 else `ASSERT_ERROR(sizeGTEMask_A)
271 sizeMatchesMask_A:
272 assert property (h2d_pre_S |-> sizeMatchesMask_S)
273 else `ASSERT_ERROR(sizeMatchesMask_A)
274 pendingReqPerSrc_A:
275 assert property (h2d_pre_S |-> pendingReqPerSrc_S)
276 else `ASSERT_ERROR(pendingReqPerSrc_A)
277 addrSizeAligned_A:
278 assert property (h2d_pre_S |-> addrSizeAligned_S)
279 else `ASSERT_ERROR(addrSizeAligned_A)
280 contigMask_A:
281 assert property (h2d_pre_S and contigMask_pre_S |->
contigMask_S)
282 else `ASSERT_ERROR(contigMask_A)
283 aDataKnown_A:
284 assert property (h2d_pre_S and aDataKnown_pre_S |->
aDataKnown_S)
285 else `ASSERT_ERROR(aDataKnown_A)
286 end
287 // d2h
288 if (1) begin : gen_d2h
289 resp0pcode_M:
290 assume property (d2h_pre_S |-> resp0pcode_S)
291 else `ASSERT_ERROR(resp0pcode_M)
292 legalDParam_M:
293 assume property (d2h_pre_S |-> legalDParam_S)
294 else `ASSERT_ERROR(legalDParam_M)
295 respSzEqReqSz_M:
296 assume property (d2h_pre_S |-> respSzEqReqSz_S)
297 else `ASSERT_ERROR(respSzEqReqSz_M)
298 respMustHaveReq_M:
299 assume property (d2h_pre_S |-> respMustHaveReq_S)
300 else `ASSERT_ERROR(respMustHaveReq_M)
301 dDataKnown_M:
302 assume property (d2h_pre_S and dDataKnown_pre_S |->
dDataKnown_S)
303 else `ASSERT_ERROR(dDataKnown_M)

```

```

304 end
305 // in this case all signals coming from the host side have
an assumed property
306 end else if (EndpointType == "Device") begin : gen_device
307 // h2d
308 if (1) begin : gen_h2d
309 legalAParam_M:
310 assume property (h2d_pre_S |-> legalAParam_S)
311 else `ASSERT_ERROR(legalAParam_M)
312 pendingReqPerSrc_M:
313 assume property (h2d_pre_S |-> pendingReqPerSrc_S)
314 else `ASSERT_ERROR(pendingReqPerSrc_M)
315 aDataKnown_M:
316 assume property (h2d_pre_S and aDataKnown_pre_S |->
aDataKnown_S)
317 else `ASSERT_ERROR(aDataKnown_M)
318 contigMask_M:
319 assume property (h2d_pre_S and contigMask_pre_S |->
contigMask_S)
320 else `ASSERT_ERROR(contigMask_M)
321 end
322 // d2h
323 if (1) begin : gen_d2h
324 resp0pcode_A:
325 assert property (d2h_pre_S |-> resp0pcode_S)
326 else `ASSERT_ERROR(resp0pcode_A)
327 legalDParam_A:
328 assert property (d2h_pre_S |-> legalDParam_S)
329 else `ASSERT_ERROR(legalDParam_A)
330 respSzEqReqSz_A:
331 assert property (d2h_pre_S |-> respSzEqReqSz_S)
332 else `ASSERT_ERROR(respSzEqReqSz_A)
333 respMustHaveReq_A:
334 assert property (d2h_pre_S |-> respMustHaveReq_S)
335 else `ASSERT_ERROR(respMustHaveReq_A)
336 dDataKnown_A:
337 assert property (d2h_pre_S and dDataKnown_pre_S |->
dDataKnown_S)
338 else `ASSERT_ERROR(dDataKnown_A)
339
340 // d2h error cases
341 legalA0pcodeErr_A:
342 assert property (d_error_pre_S and legalA0pcodeErr_S |
=>
343 s_eventually (d2h.d_valid &&
d2h.d_error))
344 else `ASSERT_ERROR(legalA0pcodeErr_A)
345 sizeGTEMaskErr_A:
346 assert property (d_error_pre_S and sizeGTEMaskErr_S |
=>
347 s_eventually (d2h.d_valid &&
d2h.d_error))
348 else `ASSERT_ERROR(sizeGTEMaskErr_A)
349 sizeMatchesMaskErr_A:
350 assert property (d_error_pre_S and sizeMatchesMask-
Err_S |>=>
351 s_eventually (d2h.d_valid &&
d2h.d_error))
352 else `ASSERT_ERROR(sizeMatchesMaskErr_A)
353 addrSizeAlignedErr_A:
354 assert property (d_error_pre_S and addrSize-
AlignedErr_S |>=>
355 s_eventually (d2h.d_valid &&
d2h.d_error))
356 else `ASSERT_ERROR(addrSizeAlignedErr_A)
357 end
358 end else begin : gen_unknown
359 initial begin : p_unknown
360 `ASSERT_I(unknownConfig_A, 0 == 1)
361 end
362 end
363
364 initial begin : p_dbw
365 // widths up to 64bit / 8 Byte are supported
366 `ASSERT_I(TlDbw_A, TL_DBW <= 8)
367 end
368
369 // make sure all "pending" bits are 0 at the end of the sim
370 for (genvar ii = 0; ii < 2*TL_AIW; ii++) begin :
gen_assert_final
371 `ASSERT_FINAL(noOutstandingReqsAtEndOfSim_A,
(pend_req[ii].pend == 0))
372 end
373
374 ///////////////////////////////////////////////////
375 // additional checks for X values //
376 ///////////////////////////////////////////////////
377
378 // a_* should be known when a_valid == 1 (a_opcode and
a_param are already covered above)
379 // This also covers ASSERT_KNOWN of a_valid
380 `ASSERT_KNOWN_IF(aKnown_A, {h2d.a_size, h2d.a_source,
h2d.a_address, h2d.a_mask, h2d.a_user},
381 h2d.a_valid)
382
383 // d_* should be known when d_valid == 1 (d_opcode, d_param,
d_size already covered above)
384 // This also covers ASSERT_KNOWN of d_valid
385 `ASSERT_KNOWN_IF(dKnown_A, {d2h.d_source, d2h.d_sink,
d2h.d_error, d2h.d_user}, d2h.d_valid)

```

```

386
387 // make sure ready is not X after reset
388 `ASSERT_KNOWN(aReadyKnown_A, d2h.a_ready)
389 `ASSERT_KNOWN(dReadyKnown_A, h2d.d_ready)
390
391 ///////////////////////////////////////////////////
392 // SVA coverage //
393 ///////////////////////////////////////////////////
394 `define TLUL_COVER(SEQ) `COVER(`SEQ``_C, `SEQ``_S, !
clk_i, !rst_ni || disable_sva)
395
396 // host sends back2back requests
397 sequence b2bReq_S;
398 h2d.a_valid && d2h.a_ready ##1 h2d.a_valid;
399 endsequence
400
401 // device sends back2back responses
402 sequence b2bRsp_S;
403 d2h.d_valid && h2d.d_ready ##1 d2h.d_valid;
404 endsequence
405
406 // host sends back2back requests with same address
407 // UVM RAL can't issue this scenario, add this cover to make
sure it's tested in some other seq
408 sequence b2bReqWithSameAddr_S;
409 bit [top_pkg::TL_AW-1:0] pre_addr;
410 (h2d.a_valid && d2h.a_ready, pre_addr = h2d.a_address)
411 ##1 h2d.a_valid && pre_addr == h2d.a_address;
412 endsequence
413
414 // a_valid is dropped without a_ready
415 sequence aValidNotAccepted_S;
416 h2d.a_valid && !d2h.a_ready ##1 !h2d.a_valid;
417 endsequence
418
419 // d_valid is dropped without a_ready
420 sequence dValidNotAccepted_S;
421 d2h.d_valid && !h2d.d_ready ##1 !d2h.d_valid;
422 endsequence
423
424 // host uses same source for back2back items
425 sequence b2bSameSource_S;
426 bit [top_pkg::TL_AW-1:0] pre_source;
427 (h2d.a_valid && d2h.a_ready, pre_source = h2d.a_source)
##1 h2d.a_valid[->1]
428 ##0 pre_source == h2d.a_source;
429 endsequence
430
431 // This macro defines a sequence where a value is provided
for the given signal on the A channel
432 // which isn't accepted and then a different value is
provided on the next cycle (without a_valid
433 // dropping).
434 //
435 // It also defines a coverpoint for the sequence.
436 `define TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED(NAME)
\
437 sequence a_`NAME``ChangedNotAccepted_S;
\
438 h2d.a_valid && !d2h.a_ready ##1
\
439 h2d.a_valid && $changed(h2d.a_`NAME``);
\
440 endsequence
\
441 `TLUL_COVER(a_`NAME``ChangedNotAccepted)
442
443 // This macro is like the previous one
(TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED), but applies to
444 // the D channel.
445 `define TLUL_D_CHAN_CONTENT_CHANGED_WO_ACCEPTED(NAME)
\
446 sequence d_`NAME``ChangedNotAccepted_S;
\
447 d2h.d_valid && !h2d.d_ready ##1
\
448 d2h.d_valid && $changed(d2h.d_`NAME``);
\
449 endsequence
\
450 `TLUL_COVER(d_`NAME``ChangedNotAccepted)
451
452 if (EndpointType == "Host") begin : gen_host_cov // DUT is
host
453 `TLUL_COVER(b2bRsp)
454 `TLUL_COVER(dValidNotAccepted)
455 `TLUL_D_CHAN_CONTENT_CHANGED_WO_ACCEPTED(data)
456 `TLUL_D_CHAN_CONTENT_CHANGED_WO_ACCEPTED(opcode)
457 `TLUL_D_CHAN_CONTENT_CHANGED_WO_ACCEPTED(size)
458 `TLUL_D_CHAN_CONTENT_CHANGED_WO_ACCEPTED(source)
459 `TLUL_D_CHAN_CONTENT_CHANGED_WO_ACCEPTED(sink)
460 `TLUL_D_CHAN_CONTENT_CHANGED_WO_ACCEPTED(error)
461 end else if (EndpointType == "Device") begin :
gen_device_cov // DUT is device
462 `TLUL_COVER(b2bReq)
463 `TLUL_COVER(b2bReqWithSameAddr)
464 `TLUL_COVER(aValidNotAccepted)
465 `TLUL_COVER(b2bSameSource)
466 `TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED(address)
467 `TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED(data)

```

```

468 `TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED(opcode)
469 `TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED(size)
470 `TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED(source)
471 `TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED(mask)
472 end else begin : gen_unknown_cov
473 initial begin : p_unknown_cov
474 `ASSERT_I(unknownConfig_A, 0 == 1)
475 end
476 end
477
478 `ifdef UVM
479 initial forever begin
480 automatic string hier_path = $formatf("%m");
481 bit tlul_assert_en;
482 uvm_config_db#(bit)::wait_modified(null, hier_path,
"tlul_assert_en");
483 if (!uvm_config_db#(bit)::get(null, hier_path,
"tlul_assert_en", tlul_assert_en)) begin
484 uvm_fatal(hier_path, "Can't find tlul_assert_en")
485 end
486 disable_sva = !tlul_assert_en;
487 end
488 `else
489 // Set default values for the disable*_sva signals (not
disabling the assertions)
490 always_ff @(posedge clk_i or negedge rst_ni) begin
491 if (!rst_ni) begin
492 disable_sva <= 0;
493 end
494 end
495 `endif
496
497 `undef TLUL_COVER
498 `undef TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED
499 `undef TLUL_D_CHAN_CONTENT_CHANGED_WO_ACCEPTED
500 `endif
501 `endif
502 endmodule : tlul_assert

```

12. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_assert_multiple.sv (31 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 // Protocol checker for multiple TL-UL ports
6
7 module tlul_assert_multiple #(
8 parameter int unsigned N = 2,
9 parameter EndpointType = "Device" // can be "Device" or
"Host"
10 ) (
11 input clk_i,
12 input rst_ni,
13
14 // tile link ports
15 input tlul_pkg::tl_h2d_t h2d [N],
16 input tlul_pkg::tl_d2h_t d2h [N]
17 );
18
19 // instantiate N tlul_assert modules
20 for (genvar ii = 0; ii < N; ii++) begin : gen_assert
21 tlul_assert #(
22 .EndpointType(EndpointType)
23 ) tlul_assert (
24 .clk_i,
25 .rst_ni,
26 // TL-UL ports
27 .h2d (h2d[ii]),
28 .d2h (d2h[ii])
29 );
30 end
31 endmodule

```

13. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_cmd_intg_chk.sv (53 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8 * Tile-Link UL command integrity check
9 */
10
11 module tlul_cmd_intg_chk import tlul_pkg::*; (
12 // TL-UL interface
13 input tl_h2d_t tl_i,
14
15 // error output
16 output logic err_o
17 );

```



```

18 logic [1:0] err;
19 logic data_err;
20 tl_h2d_cmd_intg_t cmd;
21 assign cmd = extract_h2d_cmd_intg(tl_i);
22
23
24 prim_secded_inv_64_57_dec u_chk (
25   .data_i({tl_i.a_user.cmd_intg, H2DCmdMaxWidth'(cmd)}),
26   .data_o(),
27   .syndrome_o(),
28   .err_o(err)
29 );
30
31 tlul_data_integ_dec u_tlul_data_integ_dec (
32   .data_intg_i({tl_i.a_user.data_intg, DataMax-
Width'(tl_i.a_data)}),
33   .data_err_o(data_err)
34 );
35
36 // error output is transactional, it is up to the instanti-
ating module
37 // to determine if a permanent latch is feasible
38 // [LOWRISC] err and data_err is unknown when a_valid is
low, so we can't cover
39 // the condition coverage - (!err | (!data_err)) == 0/1,
when a_valid = 0, which is
40 // fine as driving unknown is better. `err_o` is used as a
condition in other places,
41 // which needs to be covered with 0 and 1, so it's OK to
disable the entire coverage.
42 //VCS coverage off
43 // pragma coverage off
44 assign err_o = tl_i.a_valid & (!err | (!data_err));
45 //VCS coverage on
46 // pragma coverage on
47
48 logic unused_tl;
49 assign unused_tl = |tl_i;
50
51 `ASSERT_INIT(PayLoadWidthCheck, $bits(tl_h2d_cmd_intg_t) <=
H2DCmdMaxWidth)
52
53 endmodule // tlul_payload_chk

```

14. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_cmd_intg_gen.sv (58 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8  * Tile-Link UL command integrity generator
9  */
10
11 module tlul_cmd_intg_gen import tlul_pkg::*; #(
12   parameter bit EnableDataIntgGen = 1'b1
13 ) (
14   // TL-UL interface
15   input  tl_h2d_t tl_i,
16   output tl_h2d_t tl_o
17 );
18
19   tl_h2d_cmd_intg_t cmd;
20   assign cmd = extract_h2d_cmd_intg(tl_i);
21   logic [H2DCmdMaxWidth-1:0] unused_cmd_payload;
22
23   logic [H2DCmdIntgWidth-1:0] cmd_intg;
24   prim_secded_inv_64_57_enc u_cmd_gen (
25     .data_i(H2DCmdMaxWidth'(cmd)),
26     .data_o({cmd_intg, unused_cmd_payload})
27 );
28
29   logic [top_pkg::TL_DW-1:0] data_final;
30   logic [DataIntgWidth-1:0] data_intg;
31
32   if (EnableDataIntgGen) begin : gen_data_intg
33     assign data_final = tl_i.a_data;
34
35     logic [DataMaxWidth-1:0] unused_data;
36     prim_secded_inv_39_32_enc u_data_gen (
37       .data_i(DataMaxWidth'(data_final)),
38       .data_o({data_intg, unused_data})
39 );
40   end else begin : gen_passthrough_data_intg
41     assign data_final = tl_i.a_data;
42     assign data_intg = tl_i.a_user.data_intg;
43   end
44
45   always_comb begin
46     tl_o = tl_i;
47     tl_o.a_data = data_final;
48     tl_o.a_user.cmd_intg = cmd_intg;
49     tl_o.a_user.data_intg = data_intg;
50   end
51

```

```

52
53   logic unused_tl;
54   assign unused_tl = ^tl_i;
55
56   `ASSERT_INIT(PayMaxWidthCheck_A, $bits(tl_h2d_cmd_intg_t) <=
H2DCmdMaxWidth)
57
58 endmodule : tlul_cmd_intg_gen

```

15. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_data_integ_dec.sv (27 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8  * Data integrity decoder for bus integrity scheme
9  */
10
11 module tlul_data_integ_dec import tlul_pkg::*; (
12   // TL-UL interface
13   input  [DataMaxWidth+DataIntgWidth-1:0] data_intg_i,
14   output logic [DataMaxWidth+DataIntgWidth-1:0] data_err_o
15 );
16
17   logic [1:0] data_err;
18   prim_secded_inv_39_32_dec u_data_chk (
19     .data_i(data_intg_i),
20     .data_o(),
21     .syndrome_o(),
22     .err_o(data_err)
23 );
24
25   assign data_err_o = |data_err;
26
27 endmodule : tlul_data_integ_dec

```

16. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_data_integ_enc.sv (22 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8  * Data integrity encoder for bus integrity scheme
9  */
10
11 module tlul_data_integ_enc import tlul_pkg::*; (
12   // TL-UL interface
13   input  [DataMaxWidth-1:0] data_i,
14   output logic [DataMaxWidth+DataIntgWidth-1:0] data_intg_o
15 );
16
17   prim_secded_inv_39_32_enc u_data_gen (
18     .data_i,
19     .data_o(data_intg_o)
20 );
21
22 endmodule : tlul_data_integ_enc

```

17. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_err.sv (99 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5
6 `include "prim_assert.sv"
7
8 module tlul_err import tlul_pkg::*; (
9   input clk_i,
10  input rst_ni,
11
12  input tl_h2d_t tl_i,
13
14  output logic err_o
15 );
16
17   localparam int IW = $bits(tl_i.a_source);
18   localparam int SZW = $bits(tl_i.a_size);
19   localparam int DW = $bits(tl_i.a_data);
20   localparam int MW = $bits(tl_i.a_mask);
21   localparam int SubAW = $clog2(DW/8);
22
23   logic opcode_allowed, a_config_allowed;

```

```

24
25 logic op_full, op_partial, op_get;
26 assign op_full = (tl_i.a_opcode == PutFullData);
27 assign op_partial = (tl_i.a_opcode == PutPartialData);
28 assign op_get = (tl_i.a_opcode == Get);
29
30 // An instruction type transaction cannot be write
31 logic instr_wr_err;
32 assign instr_wr_err =
prim_mubi_pkg::mubi4_test_true_strict(tl_i.a_user.instr_type) &
33 (op_full | op_partial);
34
35 logic instr_type_err;
36 assign instr_type_err =
prim_mubi_pkg::mubi4_test_invalid(tl_i.a_user.instr_type);
37
38 // Anything that doesn't fall into the permitted category,
it raises an error
39 assign err_o = ~(opcode_allowed & a_config_allowed) | instr_wr_err | instr_type_err;
40
41 // opcode check
42 assign opcode_allowed = (tl_i.a_opcode == PutFullData)
43 | (tl_i.a_opcode == PutPartialData)
44 | (tl_i.a_opcode == Get);
45
46 // a channel configuration check
47 logic addr_sz_chk; // address and size alignment check
48 logic mask_chk; // inactive lane a_mask check
49 logic fulldata_chk; // PutFullData should have size match
to mask
50
51 localparam bit [MW-1:0] MaskOne = 1;
52 logic [MW-1:0] mask;
53
54 assign mask = MaskOne << tl_i.a_address[SubAW-1:0];
55
56 always_comb begin
57 addr_sz_chk = 1'b0;
58 mask_chk = 1'b0;
59 fulldata_chk = 1'b0; // Only valid when opcode is Put-
FullData
60
61 if (tl_i.a_valid) begin
62 unique case (tl_i.a_size)
63 'h0: begin // 1 Byte
64 addr_sz_chk = 1'b1;
65 mask_chk = ~(tl_i.a_mask & ~mask);
66 fulldata_chk = |(tl_i.a_mask & mask);
67 end
68
69 'h1: begin // 2 Byte
70 addr_sz_chk = ~tl_i.a_address[0];
71 // check inactive lanes if lower 2B, check
a_mask[3:2], if uppper 2B, a_mask[1:0]
72 mask_chk = (tl_i.a_address[1]) ? ~(tl_i.a_mask
& 4'b0011)
73 : ~(tl_i.a_mask & 4'b1100);
74 fulldata_chk = (tl_i.a_address[1]) ?
&tl_i.a_mask[3:2] : &tl_i.a_mask[1:0];
75 end
76
77 'h2: begin // 4 Byte
78 addr_sz_chk = ~tl_i.a_address[SubAW-1:0];
79 mask_chk = 1'b1;
80 fulldata_chk = &tl_i.a_mask[3:0];
81 end
82
83 default: begin // else
84 addr_sz_chk = 1'b0;
85 mask_chk = 1'b0;
86 fulldata_chk = 1'b0;
87 end
88 endcase
89 end
90 end
91
92 assign a_config_allowed = addr_sz_chk
93 & mask_chk
94 & (op_get | op_partial |
fulldata_chk);
95
96 // Only 32 bit data width for current tlul_err
97 `ASSERT_INIT(dataWidthOnly32_A, DW == 32)
98
99 endmodule

```

18. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_err_resp.sv (75 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // TL-UL error responder module, used by tlul_socket_1n to
help response
6 // to requests to no correct address space. Responses are al-
ways one cycle
7 // after request with no stalling unless response is stuck on

```

the way out.

```

8
9 module tlul_err_resp #(
10 // By default, we return a proper bus error. In some cases,
we need to return a blank all-zero
11 // response without setting the error bit, and for those
cases ReturnBlankResp can be set to 1.
12 parameter bit ReturnBlankResp = 0
13 ) (
14 input clk_i,
15 input rst_ni,
16 input tlul_pkg::tl_h2d_t tl_h_i,
17 output tlul_pkg::tl_d2h_t tl_h_o
18 );
19 import tlul_pkg::*;
20 import prim_mubi_pkg::*;
21
22 tl_a_op_e err_opcode;
23 logic [$bits(tl_h_i.a_source)-1:0] err_source;
24 logic [$bits(tl_h_i.a_size)-1:0] err_size;
25 logic err_rsp_pending;
26 mubi4_t err_instr_type;
27 tlul_pkg::tl_d2h_t tl_h_o_int;
28
29 tlul_rsp_intg_gen #(
30 .EnableRspIntgGen(1),
31 .EnableDataIntgGen(1)
32 ) u_intg_gen (
33 .tl_i(tl_h_o_int),
34 .tl_o(tl_h_o)
35 );
36
37 always_ff @(posedge clk_i or negedge rst_ni) begin
38 if (!rst_ni) begin
39 err_rsp_pending <= 1'b0;
40 err_source <= {top_pkg::TL_AIW{1'b0}};
41 err_opcode <= Get;
42 err_size <= '0;
43 err_instr_type <= MUBi4False;
44 end else if (err_rsp_pending && tl_h_i.d_ready) begin
45 err_rsp_pending <= 1'b0;
46 end else if (tl_h_i.a_valid && tl_h_o_int.a_ready) begin
47 err_rsp_pending <= 1'b1;
48 err_source <= tl_h_i.a_source;
49 err_opcode <= tl_h_i.a_opcode;
50 err_size <= tl_h_i.a_size;
51 err_instr_type <= tl_h_i.a_user.instr_type;
52 end
53 end
54
55 assign tl_h_o_int.a_ready = ~err_rsp_pending;
56 assign tl_h_o_int.d_valid = err_rsp_pending;
57 if (ReturnBlankResp) begin : gen_zero_resp
58 assign tl_h_o_int.d_data = '0;
59 end else begin : gen_err_resp
60 assign tl_h_o_int.d_data = (mu-
bi4_test_true_strict(err_instr_type)) ? DataWhenInstrError :
61
62 DataWhenError;
63 end
64 assign tl_h_o_int.d_source = err_source;
65 assign tl_h_o_int.d_sink = '0;
66 assign tl_h_o_int.d_param = '0;
67 assign tl_h_o_int.d_size = err_size;
68 assign tl_h_o_int.d_opcode = (err_opcode == Get) ? Acces-
SackData : AccessAck;
69 assign tl_h_o_int.d_user = '0;
70 assign tl_h_o_int.d_error = ~ReturnBlankResp;
71
72 // Waive unused bits of tl_h_i
73 logic unused_tl_h;
74 assign unused_tl_h = ^{tl_h_i, err_instr_type};
75 endmodule

```

19. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_async.sv (98 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Asynchronous TL-UL FIFO, used to cross asynchronous clock
domains.
6 // This instantiates two FIFOs, one for the request side,
7 // and one for the response side.
8
9 `include "prim_assert.sv"
10
11 module tlul_fifo_async #(
12 parameter int unsigned ReqDepth = 4,
13 parameter int unsigned RspDepth = 4
14 ) (
15 input clk_h_i,
16 input rst_h_ni,
17 input clk_d_i,
18 input rst_d_ni,
19 input tlul_pkg::tl_h2d_t tl_h_i,

```

```

20 output tlul_pkg::tl_d2h_t  tl_h_o,
21 output tlul_pkg::tl_h2d_t  tl_d_o,
22 input  tlul_pkg::tl_d2h_t  tl_d_i
23 );
24
25 // Put everything on the request side into one FIFO
26 localparam int unsigned REQ_FIFO_WIDTH =
$bits(tlul_pkg::tl_h2d_t)-2;
27
28 prim_fifo_async #(
29     .Width(REQ_FIFO_WIDTH),
30     .Depth(ReqDepth),
31     .OutputZeroIfInvalid(1)
32 ) reqfifo (
33     .clk_wr_i      (clk_h_i),
34     .rst_wr_ni     (rst_h_ni),
35     .clk_rd_i      (clk_d_i),
36     .rst_rd_ni     (rst_d_ni),
37     .wvalid_i      (tl_h_i.a_valid),
38     .wready_o      (tl_h_o.a_ready),
39     .wdata_i       ({tl_h_i.a_opcode ,
40                     tl_h_i.a_param ,
41                     tl_h_i.a_size ,
42                     tl_h_i.a_source ,
43                     tl_h_i.a_address,
44                     tl_h_i.a_mask ,
45                     tl_h_i.a_data ,
46                     tl_h_i.a_user  })),
47     .rvalid_o      (tl_d_o.a_valid),
48     .rready_i      (tl_d_i.a_ready),
49     .rdata_o       ({tl_d_o.a_opcode ,
50                     tl_d_o.a_param ,
51                     tl_d_o.a_size ,
52                     tl_d_o.a_source ,
53                     tl_d_o.a_address,
54                     tl_d_o.a_mask ,
55                     tl_d_o.a_data ,
56                     tl_d_o.a_user  })),
57     .wdepth_o      (),
58     .rdepth_o      ()
59 );
60
61 // Put everything on the response side into the other FIFO
62
63 localparam int unsigned RSP_FIFO_WIDTH =
$bits(tlul_pkg::tl_d2h_t) -2;
64
65 prim_fifo_async #(
66     .Width(RSP_FIFO_WIDTH),
67     .Depth(RspDepth),
68     .OutputZeroIfInvalid(1)
69 ) rspfifo (
70     .clk_wr_i      (clk_d_i),
71     .rst_wr_ni     (rst_d_ni),
72     .clk_rd_i      (clk_h_i),
73     .rst_rd_ni     (rst_h_ni),
74     .wvalid_i      (tl_d_i.d_valid),
75     .wready_o      (tl_d_o.d_ready),
76     .wdata_i       ({tl_d_i.d_opcode,
77                     tl_d_i.d_param ,
78                     tl_d_i.d_size ,
79                     tl_d_i.d_source,
80                     tl_d_i.d_sink ,
81                     tl_d_i.d_data ,
82                     tl_d_i.d_user ,
83                     tl_d_i.d_error })),
84     .rvalid_o      (tl_h_o.d_valid),
85     .rready_i      (tl_h_i.d_ready),
86     .rdata_o       ({tl_h_o.d_opcode,
87                     tl_h_o.d_param ,
88                     tl_h_o.d_size ,
89                     tl_h_o.d_source,
90                     tl_h_o.d_sink ,
91                     tl_h_o.d_data ,
92                     tl_h_o.d_user ,
93                     tl_h_o.d_error })),
94     .wdepth_o      (),
95     .rdepth_o      ()
96 );
97
98 endmodule

```

20. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_sync.sv (191 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Synchronous TL-UL FIFO, used to add elasticity (the ability for
6 // transactions to stall on one side without affecting the
  other side)
7 // to a TL-UL bus. This instantiates two pairs (data + integ-
  rity) of FIFOs:
8 // one pair for the request side, and another pair for the re-
  sponse side.
9
10 module tlul_fifo_sync #(

```

```

11     parameter bit          ReqPass = 1'b1,
12     parameter bit          RspPass = 1'b1,
13     parameter int unsigned ReqDepth = 2,
14     parameter int unsigned RspDepth = 2,
15     parameter int unsigned SpareReqW = 1,
16     parameter int unsigned SpareRspW = 1
17 ) (
18     input                  clk_i,
19     input                  rst_ni,
20     input  tlul_pkg::tl_h2d_t  tl_h_i,
21     output tlul_pkg::tl_d2h_t  tl_h_o,
22     output tlul_pkg::tl_h2d_t  tl_d_o,
23     input  tlul_pkg::tl_d2h_t  tl_d_i,
24     input  [SpareReqW-1:0]     spare_req_i,
25     output [SpareReqW-1:0]     spare_req_o,
26     input  [SpareRspW-1:0]     spare_rsp_i,
27     output [SpareRspW-1:0]     spare_rsp_o
28 );
29
30 // Put everything on the request side into two FIFOs.
31 // The first FIFO holds all data except the integrity bits
  of the `a_user` field. The second FIFO
32 // only holds the command and data integrity bits of the
  `a_user` field. This is a FI
33
34 // Countermeasure, as a fault into one of the two FIFOs will
  trigger an integrity error once the
35 // integrity bits are checked.
36 localparam int unsigned REQ_FIFO_INTG_WIDTH = tlul_pkg::H2D-
  CmdIntgWidth + tlul_pkg::DataIntgWidth;
37 localparam int unsigned REQ_FIFO_WIDTH =
$bits(tlul_pkg::tl_h2d_t) -2 + SpareReqW -
38 REQ_FIFO_INTG_WIDTH;
39
40 prim_fifo_sync
  #(.Width(REQ_FIFO_WIDTH), .Pass(ReqPass), .Depth(ReqDepth)) reqfifo (
41     .clk_i,
42     .rst_ni,
43     .clr_i      (1'b0),
44     .wvalid_i   (tl_h_i.a_valid),
45     .wready_o   (tl_h_o.a_ready),
46     .wdata_i    ({tl_h_i.a_opcode,
47                   tl_h_i.a_param,
48                   tl_h_i.a_size,
49                   tl_h_i.a_source,
50                   tl_h_i.a_address,
51                   tl_h_i.a_mask,
52                   tl_h_i.a_data,
53                   tl_h_i.a_user.rsvd,
54                   tl_h_i.a_user.instr_type,
55                   spare_req_i}),
56     .rvalid_o   (tl_d_o.a_valid),
57     .rready_i   (tl_d_i.a_ready),
58     .rdata_o    ({tl_d_o.a_opcode,
59                   tl_d_o.a_param,
60                   tl_d_o.a_size,
61                   tl_d_o.a_source,
62                   tl_d_o.a_address,
63                   tl_d_o.a_mask,
64                   tl_d_o.a_data,
65                   tl_d_o.a_user.rsvd,
66                   tl_d_o.a_user.instr_type,
67                   spare_req_o}),
68     .full_o     (),
69     .depth_o    (),
70     .err_o      ());
71
72 // Buffer the inputs of the FIFO holding the integrity to
  avoid synthesis optimizations.
73 localparam int NumBufferBitsReqFifoIntg = $bits({
74     tl_h_i.a_valid,
75     tl_d_i.a_ready
76 });
77
78 logic [NumBufferBitsReqFifoIntg-1:0] buf_reqfifo_intg_in,
  buf_reqfifo_intg_out;
79
80 logic reqfifo_intg_a_valid_buf;
81 logic reqfifo_intg_a_ready_buf;
82
83 assign buf_reqfifo_intg_in = {
84     tl_h_i.a_valid,
85     tl_d_i.a_ready
86 };
87
88 assign {
89     reqfifo_intg_a_valid_buf,
90     reqfifo_intg_a_ready_buf
91 } = buf_reqfifo_intg_out;
92
93 prim_buf #(
94     .Width(NumBufferBitsReqFifoIntg)
95 ) u_reqfifo_intg_prim_buf (
96     .in_i(buf_reqfifo_intg_in),
97     .out_o(buf_reqfifo_intg_out)
98 );
99
100 prim_fifo_sync
  #(.Width(REQ_FIFO_INTG_WIDTH), .Pass(ReqPass), .Depth(ReqDepth))
  reqfifo_intg (
101     .clk_i,

```

```

100 .rst_ni,
101 .clr_i      (1'b0),
102 .wvalid_i   (reqfifo_intg_a_valid_buf),
103 .wready_o   (),
104 .wdata_i     ({tl_h_i.a_user.cmd_intg,
105                tl_h_i.a_user.data_intg}),
106 .rvalid_o   (),
107 .rready_i   (reqfifo_intg_a_ready_buf),
108 .rdata_o     ({tl_d_o.a_user.cmd_intg,
109                tl_d_o.a_user.data_intg}),
110 .full_o      (),
111 .depth_o     (),
112 .err_o       ();
113
114 // Put everything on the response side into the other two
FIFOs.
115 // The integrity bits and all other bits are separated into
two different FIFOs for the same
116 // reason as above.
117 localparam int unsigned RSPFIFO_INTG_WIDTH =
$bits(tlul_pkg::tl_d_user_t);
118 localparam int unsigned RSPFIFO_WIDTH =
$bits(tlul_pkg::tl_d2h_t) - 2 + SpareRspW -
119
RSP-
FIFO_INTG_WIDTH;
120
121 prim_fifo_sync
#(.Width(RSPFIFO_WIDTH), .Pass(RspPass), .Depth(RspDepth)) rspfifo (
122 .clk_i,
123 .rst_ni,
124 .clr_i      (1'b0),
125 .wvalid_i   (tl_d_i.d_valid),
126 .wready_o   (tl_d_o.d_ready),
127 .wdata_i     ({tl_d_i.d_opcode,
128                tl_d_i.d_param,
129                tl_d_i.d_size,
130                tl_d_i.d_source,
131                tl_d_i.d_sink,
132                (tl_d_i.d_opcode == tlul_pkg::AccessAck-
Data) ? tl_d_i.d_data :
133
{top_pkg::TL_DW{1'b0}}),
134                tl_d_i.d_error,
135                spare_rsp_i}),
136 .rvalid_o   (tl_h_o.d_valid),
137 .rready_i   (tl_h_i.d_ready),
138 .rdata_o     ({tl_h_o.d_opcode,
139                tl_h_o.d_param,
140                tl_h_o.d_size,
141                tl_h_o.d_source,
142                tl_h_o.d_sink,
143                tl_h_o.d_data,
144                tl_h_o.d_error,
145                spare_rsp_o}),
146 .full_o      (),
147 .depth_o     (),
148 .err_o       ();
149
150 // Buffer the inputs of the FIFO holding the integrity to
avoid synthesis optimizations.
151 localparam int NumBufferBitsRspFifoIntg = $bits({
152 tl_d_i.d_valid,
153 tl_h_i.d_ready
154 });
155
156 logic [NumBufferBitsRspFifoIntg-1:0] buf_rspfifo_intg_in,
buf_rspfifo_intg_out;
157 logic rspfifo_intg_d_valid_buf;
158 logic rspfifo_intg_d_ready_buf;
159
160 assign buf_rspfifo_intg_in = {
161 tl_d_i.d_valid,
162 tl_h_i.d_ready
163 };
164
165 assign {
166 rspfifo_intg_d_valid_buf,
167 rspfifo_intg_d_ready_buf
168 } = buf_rspfifo_intg_out;
169
170 prim_buf #(
171 .Width(NumBufferBitsRspFifoIntg)
172 ) u_rspfifo_intg_prim_buf (
173 .in_i(buf_rspfifo_intg_in),
174 .out_o(buf_rspfifo_intg_out)
175 );
176
177 prim_fifo_sync
#(.Width(RSPFIFO_INTG_WIDTH), .Pass(RspPass), .Depth(RspDepth)) rsp-
fifo_intg (
178 .clk_i,
179 .rst_ni,
180 .clr_i      (1'b0),
181 .wvalid_i   (rspfifo_intg_d_valid_buf),
182 .wready_o   (),
183 .wdata_i     (tl_d_i.d_user),
184 .rvalid_o   (),
185 .rready_i   (rspfifo_intg_d_ready_buf),
186 .rdata_o     (tl_h_o.d_user),
187 .full_o      (),

```

```

188 .depth_o     (),
189 .err_o       ();
190
191 endmodule

```

21. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_jtag_dtm.sv (150 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // JTAG DTM to TL-UL Converter.
6 // This implements a JTAG DTM according to the RISC-V external de-
bug v0.13 specification and
7 // uses the TL-UL protocol to transport read/write operations
to the RISC-V debug module
8 // (i.e. the DMI is implemented with the TL-UL protocol):
9 // https://github.com/riscv/riscv-debug-spec/blob/release/
riscv-debug-release.pdf
10 //
11
12 module tlul_jtag_dtm #(
13 // Idcode value for the JTAG.
14 parameter logic [31:0] IdcodeValue = 32'h00000001,
15 parameter int unsigned NumDmiByteAbits = 18
16 ) (
17 input clk_i,
18 input rst_ni,
19 input jtag_pkg::jtag_req_t jtag_i,
20 output jtag_pkg::jtag_rsp_t jtag_o,
21 // This bypasses the clock inverter inside the JTAG TAP for
scanmode.
22 input scanmode,
23 input prim_mubi_pkg::mubi4_t scan-
mode_i,
24 // TL-UL-based DMI
25 output tlul_pkg::tl_h2d_t tl_h2d_o,
26 input tlul_pkg::tl_d2h_t tl_d2h_i,
27 );
28
29 dm::dmi_req_t dmi_req;
30 logic dmi_req_valid;
31 logic dmi_req_ready;
32 dm::dmi_resp_t dmi_resp;
33 logic dmi_resp_ready;
34 logic dmi_resp_valid;
35
36 logic scanmode;
37 prim_mubi4_dec u_prim_mubi4_dec (
38 .mubi_i(scanmode_i),
39 .mubi_dec_o(scanmode)
40 );
41
42 logic tck_muxed;
43 logic trst_n_muxed;
44 prim_clock_mux2 #(
45 .NoFpgaBufG(1'b1)
46 ) u_prim_clock_mux2 (
47 .clk0_i(jtag_i.tck),
48 .clk1_i(clk_i),
49 .sel_i (scanmode),
50 .clk_o (tck_muxed)
51 );
52
53 prim_clock_mux2 #(
54 .NoFpgaBufG(1'b1)
55 ) u_prim_rst_n_mux2 (
56 .clk0_i(jtag_i.trst_n),
57 .clk1_i(scan_rst_ni),
58 .sel_i (scanmode),
59 .clk_o (trst_n_muxed)
60 );
61
62 // TODO: At the moment, this uses the JTAG DTM implementa-
tion from the PULP project.
63 // This module should be refactored in the future to support
TL-UL directly, instead
64 // of a custom DMI protocol which necessitates another
translation layer to TL-UL.
65 logic req_ready;
66 assign req_ready = dmi_req_ready && dmi_resp_ready;
67
68 dmi_jtag #(
69 .IdcodeValue(IdcodeValue),
70 .NumDmiWordAbits(NumDmiByteAbits - 2)
71 ) u_dmi_jtag (
72 .clk_i,
73 .rst_ni,
74 .testmode_i ( scanmode ),
75 .test_rst_ni ( scan_rst_ni ),
76 .dmi_rst_no ( ), // unused
77 .dmi_req_o ( dmi_req ),
78 .dmi_req_valid_o ( dmi_req_valid ),
79 // unless there is room for response, stall
80 .dmi_req_ready_i ( req_ready ),

```

```

81 .dmi_resp_i      ( dmi_resp      ),
82 .dmi_resp_ready_o ( dmi_resp_ready ),
83 .dmi_resp_valid_i ( dmi_resp_valid ),
84 .tck_i           ( tck_muxed     ),
85 .tms_i           ( jtag_i.tms     ),
86 .trst_ni         ( trst_n_muxed   ),
87 .td_i            ( jtag_i.tdi      ),
88 .td_o            ( jtag_o.tdo      ),
89 .tdo_oe_o        ( jtag_o.tdo_oe   )
90 );
91
92 // Outstanding request handling.
93 logic pending_req_q;
94 always_ff @(posedge clk_i or negedge rst_ni) begin : p_pending
95     if (!rst_ni) begin
96         pending_req_q <= 1'b0;
97     end else begin
98         // New request
99         if (dmi_req_ready && dmi_req_valid && (!pending_req_q))
100             pending_req_q <= 1'b1;
101     end else if (dmi_resp_valid) begin
102         pending_req_q <= 1'b0;
103     end
104 end
105
106 // DMI to TL-UL transducing
107 logic dmi_error;
108 tlul_adapter_host #(
109     // Only make one request at a time.
110     .MAX_REQS(1),
111     .EnableDataIntgGen(1)
112 ) u_tap_tlul_host (
113     .clk_i,
114     .rst_ni,
115     // Do not make more than one request at a time
116     .req_i      ( dmi_req_valid && !pending_req_q ),
117     .gnt_o      ( dmi_req_ready ),
118     .addr_i     ( top_pkg::TL_AW'({dmi_req.addr, 2'b00}) ),
119     .we_i       ( dmi_req.op == dm::DTM_WRITE ),
120     .wdata_i    ( dmi_req.data ),
121     .wdata_intg_i ('0'),
122     .be_i       ( {top_pkg::TL_DBW{1'b1}} ),
123     .user_rsvd_i ('0'),
124     .instr_type_i ( prim_mubi_pkg::MuBi4False ),
125     .valid_o     ( dmi_resp_valid ),
126     .rdata_o     ( dmi_resp.data ),
127     .rdata_intg_o ( ),
128     .err_o       ( dmi_error ),
129     .intg_err_o   ( ),
130     .tl_o        ( tl_h2d_o ),
131     .tl_i        ( tl_d2h_i )
132 );
133
134 // We use the user channel to transmit the DMI response
135 type.
136 assign dmi_resp.resp = (dmi_error) ? dm::DTM_ERR :
dm::DTM_SUCCESS;
137
138 // These signals are unused
139 logic unused_sigs;
140 assign unused_sigs = ^{
141     dmi_req.addr[31:30],
142     tl_d2h_i.d_opcode,
143     tl_d2h_i.d_param,
144     tl_d2h_i.d_size,
145     tl_d2h_i.d_source,
146     tl_d2h_i.d_sink,
147     tl_d2h_i.d_error
148 };
149
150 endmodule : tlul_jtag_dtm

```

22. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_lc_gate.sv (267 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Life cycle gating module for TL-UL protocol.
6 // Transactions are passed through when lc_en_i == ON.
7 // In all other cases (lc_en_i != ON) incoming transactions
return a bus error.
8 //
9 // Note that the lc_en_i should be synchronized and buffered
outside of this module using
10 // an instance of prim_lc_sync.
11
12 `include "prim_assert.sv"
13
14 module tlul_lc_gate
15     import tlul_pkg::*;
16     import lc_ctrl_pkg::*;
17 #()
18 // Number of LC gating muxes in each direction.
19 // It is recommended to set this parameter to 2, which res-

```

```

ults
20 // in a total of 4 gating muxes.
21 parameter int NumGatesPerDirection = 2,
22 // By default we return a TL-UL bus error response if the
bus is gated. However, in some special
23 // cases we need to be able to return valid, all-zero re-
sponses instead (e.g. for the RV_DM). In
24 // those cases, ReturnBlankResp can be set to 1.
25 parameter bit ReturnBlankResp = 0,
26 // The maximum number of outstanding TL-UL requests at the
output
27 parameter int unsigned Outstanding = 3
28 ) (
29     input clk_i,
30     input rst_ni,
31
32     // To host
33     input tl_h2d_t tl_h2d_i,
34     output tl_d2h_t tl_d2h_o,
35
36     // To device
37     output tl_h2d_t tl_h2d_o,
38     input tl_d2h_t tl_d2h_i,
39
40     // Flush control signaling
41     input flush_req_i,
42     output logic flush_ack_o,
43
44     // Indicates whether there are pending responses on the
device side.
45     output logic resp_pending_o,
46
47     // LC control signal
48     input lc_tx_t lc_en_i,
49     output logic err_o
50 );
51
52 // Access Gates //
53 // LC control signal
54 lc_tx_t err_en;
55 lc_tx_t [NumGatesPerDirection-1:0] err_en_buf;
56
57 prim_lc_sync #(
58     .NumCopies(NumGatesPerDirection),
59     .AsyncOn(0)
60 ) u_err_en_sync (
61     .clk_i,
62     .rst_ni,
63     .lc_en_i(err_en),
64     .lc_en_o(err_en_buf)
65 );
66
67 tl_h2d_t tl_h2d_int [NumGatesPerDirection+1];
68 tl_d2h_t tl_d2h_int [NumGatesPerDirection+1];
69 for (genvar k = 0; k < NumGatesPerDirection; k++) begin :
gen_lc_gating_muxes
70     // H -> D path.
71     prim_blanker #(
72         .Width($bits(tl_h2d_t))
73     ) u_prim_blanker_h2d (
74         .in_i(tl_h2d_int[k]),
75         .en_i(lc_tx_test_false_strict(err_en_buf[k])),
76         .out_o(tl_h2d_int[k+1])
77     );
78
79     // D -> H path.
80     prim_blanker #(
81         .Width($bits(tl_d2h_t))
82     ) u_prim_blanker_d2h (
83         .in_i(tl_d2h_int[k+1]),
84         .en_i(lc_tx_test_false_strict(err_en_buf[k])),
85         .out_o(tl_d2h_int[k])
86     );
87 end
88
89 // Assign signals on the device side.
90 assign tl_h2d_o = tl_h2d_int[NumGatesPerDirection];
91 assign tl_d2h_int[NumGatesPerDirection] = tl_d2h_i;
92
93 // Host Side Interposing //
94 // Encoding generated with:
95 // $ ./util/design/sparse-fsm-encode.py -d 5 -m 4 -n 8 \
96 // -s 3379253306 --language=sv
97 //
98 // Hamming distance histogram:
99 //
100 // 0: --
101 // 1: --
102 // 2: --
103 // 3: --
104 // 4: --
105 // 5: ||||| (66.67%)
106 // 6: ||||| (33.33%)
107 // 7: --
108 // 8: --
109 //

```

```

115 // Minimum Hamming distance: 5
116 // Maximum Hamming distance: 6
117 // Minimum Hamming weight: 3
118 // Maximum Hamming weight: 5
119 //
120 // Encoding generated with:
121 // $ ./util/design/sparse-fsm-encode.py -d 5 -m 5 -n 9 \
122 // -s 686407169 --language=sv
123 //
124 // Hamming distance histogram:
125 //
126 // 0: --
127 // 1: --
128 // 2: --
129 // 3: --
130 // 4: --
131 // 5: ||||| (60.00%)
132 // 6: ||||| (40.00%)
133 // 7: --
134 // 8: --
135 // 9: --
136 //
137 // Minimum Hamming distance: 5
138 // Maximum Hamming distance: 6
139 // Minimum Hamming weight: 3
140 // Maximum Hamming weight: 6
141 //
142 localparam int StateWidth = 9;
143 typedef enum logic [StateWidth-1:0] {
144     StActive = 9'b100100001,
145     StOutstanding = 9'b011100111,
146     StFlush = 9'b001001100,
147     StError = 9'b01011010,
148     StErrorOutstanding = 9'b100010110
149 } state_e;
150
151 state_e state_d, state_q;
152 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
state_e, StError)
153
154 logic [prim_util_pkg::vbits(Outstanding+1)-1:0] outstand-
ing_txn;
155 logic a_ack;
156 logic d_ack;
157 assign a_ack = tl_h2d_i.a_valid & tl_d2h_o.a_ready;
158 assign d_ack = tl_h2d_i.d_ready & tl_d2h_o.d_valid;
159
160 always_ff @(posedge clk_i or negedge rst_ni) begin
161     if (!rst_ni) begin
162         outstanding_txn <= '0;
163     end else if (a_ack && !d_ack) begin
164         outstanding_txn <= outstanding_txn + 1'b1;
165     end else if (d_ack && !a_ack) begin
166         outstanding_txn <= outstanding_txn - 1'b1;
167     end
168 end
169
170 logic block_cmd;
171 always_comb begin
172     block_cmd = '0;
173     state_d = state_q;
174     err_en = Off;
175     err_o = '0;
176     flush_ack_o = '0;
177     resp_pending_o = 1'b0;
178
179     unique case (state_q)
180         StActive: begin
181             if (lc_tx_test_false_loose(lc_en_i) || flush_req_i)
begin
182                 state_d = StOutstanding;
183             end
184             if (outstanding_txn != '0) begin
185                 resp_pending_o = 1'b1;
186             end
187         end
188
189         StOutstanding: begin
190             block_cmd = 1'b1;
191             if (outstanding_txn == '0) begin
192                 state_d = lc_tx_test_false_loose(lc_en_i) ? StEr-
ror : StFlush;
193             end else begin
194                 resp_pending_o = 1'b1;
195             end
196         end
197
198         StFlush: begin
199             block_cmd = 1'b1;
200             flush_ack_o = 1'b1;
201             if (lc_tx_test_false_loose(lc_en_i)) begin
202                 state_d = StError;
203             end else if (!flush_req_i) begin
204                 state_d = StActive;
205             end
206         end
207
208         StError: begin
209             err_en = On;
210             if (lc_tx_test_true_strict(lc_en_i)) begin
211                 state_d = StErrorOutstanding;

```

```

212     end
213 end
214
215 StErrorOutstanding: begin
216     err_en = On;
217     block_cmd = 1'b1;
218     if (outstanding_txn == '0) begin
219         state_d = StActive;
220     end
221 end
222
223 default: begin
224     err_o = 1'b1;
225     err_en = On;
226 end
227
228 endcase // unique case (state_q)
229 end
230
231
232 // At the host side, we interpose the ready / valid signals
so that we can return a bus error
233 // in case the lc signal is not set to ON. Note that this
logic does not have to be duplicated
234 // since erroring back is considered a convenience feature
so that the bus does not lock up.
235 tl_h2d_t tl_h2d_error;
236 tl_d2h_t tl_d2h_error;
237 always_comb begin
238     tl_h2d_int[0] = tl_h2d_i;
239     tl_d2h_o = tl_d2h_int[0];
240     tl_h2d_error = '0;
241
242     if (lc_tx_test_true_loose(err_en)) begin
243         tl_h2d_error = tl_h2d_i;
244         tl_d2h_o = tl_d2h_error;
245     end
246
247     if (block_cmd) begin
248         tl_d2h_o.a_ready = 1'b0;
249         tl_h2d_int[0].a_valid = 1'b0;
250         tl_h2d_error.a_valid = 1'b0;
251     end
252 end
253
254 tlul_err_resp #(
255     .ReturnBlankResp(ReturnBlankResp)
256 ) u_tlul_err_resp (
257     .clk_i,
258     .rst_ni,
259     .tl_h_i(tl_h2d_error),
260     .tl_h_o(tl_d2h_error)
261 );
262
263 // Add assertion
264 `ASSERT(OutStanding0vfl_A, (outstanding_txn == Outstanding)
|-> ~a_ack)
265 `ASSERT(SizeOutstandingTxn_A, (2 ** $bits(outstanding_txn))
- 1 >= Outstanding)
266
267 endmodule : tlul_lc_gate

```

23. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_pkg.sv (209 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5
6 package tlul_pkg;
7
8 // this can be either PPC or BINTREE
9 // there is no functional difference, but timing and area
behavior is different
10 // between the two instances. PPC can result in smaller im-
plementations when timing
11 // is not critical, whereas BINTREE is favorable when timing
pressure is high (but this
12 // may also result in a larger implementation). on FPGA tar-
gets, BINTREE is favorable
13 // both in terms of area and timing.
14 parameter ArbiterImpl = "PPC";
15
16 typedef enum logic [2:0] {
17     PutFullData = 3'h 0,
18     PutPartialData = 3'h 1,
19     Get = 3'h 4
20 } tl_a_op_e;
21
22 typedef enum logic [2:0] {
23     AccessAck = 3'h 0,
24     AccessAckData = 3'h 1
25 } tl_d_op_e;
26
27 parameter int H2DCmdMaxWidth = 57;
28 parameter int H2DCmdIntgWidth = 7;
29 parameter int H2DCmdFullWidth = H2DCmdMaxWidth + H2DCmdIntg-

```



```

Width;
30 parameter int D2HRspMaxWidth = 57;
31 parameter int D2HRspIntgWidth = 7;
32 parameter int D2HRspFullWidth = D2HRspMaxWidth + D2HRspIntg-
Width;
33 parameter int DataMaxWidth = 32;
34 parameter int DataIntgWidth = 7;
35 parameter int DataFullWidth = DataMaxWidth + DataIntg-
Width;
36 parameter int RsvdWidth = top_pkg::TL_AUW -
prim_mubi_pkg::MuB14Width -
37 H2DCmdIntgWidth - DataIntg-
Width;
38
39 // Data that is returned upon an a TL-UL error belonging to
an instruction fetch.

40 // Note that this data will be returned with the correct bus
integrity value.
41 parameter logic [top_pkg::TL_DW-1:0] DataWhenInstrError =
'0;

42 // Data that is returned upon an a TL-UL error not belonging to
an instruction fetch.

43 // Note that this data will be returned with the correct bus
integrity value.
44 parameter logic [top_pkg::TL_DW-1:0] DataWhenError =
{top_pkg::TL_DW{1'b1}};
45
46 typedef struct packed {
47 logic [RsvdWidth-1:0] rsvd;
48 prim_mubi_pkg::mubi4_t instr_type;
49 logic [H2DCmdIntgWidth-1:0] cmd_intg;
50 logic [DataIntgWidth-1:0] data_intg;
51 } tl_a_user_t;
52
53 parameter tl_a_user_t TL_A_USER_DEFAULT = '{
54 rsvd: '0,
55 instr_type: prim_mubi_pkg::MuB14False,
56 cmd_intg: {H2DCmdIntgWidth{1'b1}},
57 data_intg: {DataIntgWidth{1'b1}}
58 };
59
60 typedef struct packed {
61 prim_mubi_pkg::mubi4_t instr_type;
62 logic [top_pkg::TL_AW-1:0] addr;
63 tl_a_op_e opcode;
64 logic [top_pkg::TL_DBW-1:0] mask;
65 } tl_h2d_cmd_intg_t;
66
67 typedef struct packed {
68 logic a_valid;
69 tl_a_op_e a_opcode;
70 logic [2:0] a_param;
71 logic [top_pkg::TL_SZW-1:0] a_size;
72 logic [top_pkg::TL_AIW-1:0] a_source;
73 logic [top_pkg::TL_AW-1:0] a_address;
74 logic [top_pkg::TL_DBW-1:0] a_mask;
75 logic [top_pkg::TL_DW-1:0] a_data;
76 tl_a_user_t a_user;
77
78 logic d_ready;
79 } tl_h2d_t;
80
81 // The choice of all 1's as the blanked value is deliberate.
82 // It is assumed that most security features of the design
are opt-in instead
83 // of opt-out.
84 // Given the opt-in nature, if a 0 were to propagate, the
feature
would be turned
85 // off. Whereas if a 1 were to propagate, it would either
stay on
or be turned on.
86 // There is however no perfect value for this purpose.
87 localparam logic [top_pkg::TL_DW-1:0] BlankedAData =
{top_pkg::TL_DW{1'b1}};
88
89 localparam tl_h2d_t TL_H2D_DEFAULT = '{
90 d_ready: 1'b1,
91 a_opcode: tl_a_op_e('0),
92 a_user: TL_A_USER_DEFAULT,
93 a_data: BlankedAData,
94 default: '0
95 };
96
97 typedef struct packed {
98 logic [D2HRspIntgWidth-1:0] rsp_intg;
99 logic [DataIntgWidth-1:0] data_intg;
100 } tl_d_user_t;
101
102 parameter tl_d_user_t TL_D_USER_DEFAULT = '{
103 rsp_intg: {D2HRspIntgWidth{1'b1}},
104 data_intg: {DataIntgWidth{1'b1}}
105 };
106
107 typedef struct packed {
108 logic d_valid;
109 tl_d_op_e d_opcode;
110 logic [2:0] d_param;
111 logic [top_pkg::TL_SZW-1:0] d_size; // Bouncing back
a_size

```

```

112 logic [top_pkg::TL_AIW-1:0] d_source;
113 logic [top_pkg::TL_DIW-1:0] d_sink;
114 logic [top_pkg::TL_DW-1:0] d_data;
115 tl_d_user_t d_user;
116 logic d_error;
117
118 logic a_ready;
119
120 } tl_d2h_t;
121
122 typedef struct packed {
123 tl_d_op_e opcode;
124 logic [top_pkg::TL_SZW-1:0] size;
125 // Temporarily removed because source changes throughout
the fabric
126 // and thus cannot be used for end-to-end checking.

127 // A different PR will propose a work-around (a hoaky one)
to see if
128 // it gets the job done.
129 //logic [top_pkg::TL_AIW-1:0] source;
130 logic error;
131 } tl_d2h_rsp_intg_t;
132
133 localparam tl_d2h_t TL_D2H_DEFAULT = '{
134 a_ready: 1'b1,
135 d_opcode: tl_d_op_e('0),
136 d_user: TL_D_USER_DEFAULT,
137 default: '0
138 };
139
140 // Check user for unsupported values
141 function automatic logic tl_a_user_chk(tl_a_user_t user);
142 logic malformed_err;
143 logic unused_user;
144 unused_user = user;
145 malformed_err =
prim_mubi_pkg::mubi4_test_invalid(user.instr_type);
146 return malformed_err;
147 endfunction // tl_a_user_chk
148
149 // extract variables used for command checking
150 function automatic tl_h2d_cmd_intg_t ex-
tract_h2d_cmd_intg(tl_h2d_t tl);
151 tl_h2d_cmd_intg_t payload;
152 logic unused_tlul;
153 unused_tlul = ^tl;
154 payload.addr = tl.a_address;
155 payload.opcode = tl.a_opcode;
156 payload.mask = tl.a_mask;
157 payload.instr_type = tl.a_user.instr_type;
158 return payload;
159 endfunction // extract_h2d_payload
160
161 // extract variables used for response checking
162 function automatic tl_d2h_rsp_intg_t ex-
tract_d2h_rsp_intg(tl_d2h_t tl);
163 tl_d2h_rsp_intg_t payload;
164 logic unused_tlul;
165 unused_tlul = ^tl;
166 payload.opcode = tl.d_opcode;
167 payload.size = tl.d_size;
168 //payload.source = tl.d_source;
169 payload.error = tl.d_error;
170 return payload;
171 endfunction // extract_d2h_rsp_intg
172
173 // calculate ecc for command checking
174 function automatic logic [H2DCmdIntgWidth-1:0]
get_cmd_intg(tl_h2d_t tl);
175 logic [H2DCmdIntgWidth-1:0] cmd_intg;
176 logic [H2DCmdMaxWidth-1:0] unused_cmd_payload;
177 tl_h2d_cmd_intg_t cmd;
178 cmd = extract_h2d_cmd_intg(tl);
179 {cmd_intg, unused_cmd_payload} =
180
181 prim_secdec_pkg::prim_secdec_inv_64_57_enc(H2DCmdMaxWidth'(cmd));
182 return cmd_intg;
183 endfunction // get_cmd_intg
184
185 // calculate ecc for data checking
186 function automatic logic [DataIntgWidth-1:0]
get_data_intg(logic [top_pkg::TL_DW-1:0] data);
187 logic [DataIntgWidth-1:0] data_intg;
188 logic [top_pkg::TL_DW-1:0] unused_data;
189 logic [DataIntgWidth + top_pkg::TL_DW - 1 : 0] enc_data;
190 enc_data =
prim_secdec_pkg::prim_secdec_inv_39_32_enc(data);
191 data_intg = enc_data[DataIntgWidth + top_pkg::TL_DW - 1 :
top_pkg::TL_DW];
192 unused_data = enc_data[top_pkg::TL_DW - 1 : 0];
193 return data_intg;
194 endfunction // get_data_intg
195
196 // return inverted integrity for command payload
197 function automatic logic [H2DCmdIntgWidth-1:0]
get_bad_cmd_intg(tl_h2d_t tl);
198 logic [H2DCmdIntgWidth-1:0] cmd_intg;
199 cmd_intg = get_cmd_intg(tl);
200 return ~cmd_intg;
201 endfunction // get_bad_cmd_intg

```

```

201
202 // return inverted integrity for data payload
203 function automatic logic [H2CmdIntgWidth-1:0]
get_bad_data_intg(logic [top_pkg::TL_DW-1:0] data);
204     logic [H2CmdIntgWidth-1:0] data_intg;
205     data_intg = get_data_intg(data);
206     return ~data_intg;
207 endfunction // get_bad_data_intg
208
209 endpackage

```

24. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_request_loopback.sv (122 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 module tlul_request_loopback
6     import tlul_pkg::*;
7 # (
8     parameter bit ErrorRsp = 1 // 1: Return TLUL error on on
squash
9 ) (
10     input logic clk_i,
11     input logic rst_ni,
12     input logic squash_req_i, // Incoming request
13     input tlul_pkg::tl_h2d_t tl_h2d_i, // Response
14     output tlul_pkg::tl_d2h_t tl_d2h_o, // Feed through re-
quest if not squashed to the target
15     output tlul_pkg::tl_h2d_t tl_h2d_o, // Response from the
target
16     input tlul_pkg::tl_d2h_t tl_d2h_i
17 );
18
19 // A valid request is looped back if the squash_req_i is as-
serted in the same cycle
20 // Regardless of the request being squashed or not, the re-
quest payload is NOT modified by this
21 // module
22 logic loopback_request;
23 assign loopback_request = tl_h2d_i.a_valid & squash_req_i;
24
25 tlul_pkg::tl_h2d_t tl_muxed_h2d [2];
26 tlul_pkg::tl_d2h_t tl_muxed_d2h [2];
27 tlul_pkg::tl_h2d_t tl_error_h2d;
28 tlul_pkg::tl_d2h_t tl_error_d2h;
29
30 // Device port is Idx0
31 assign tl_h2d_o = tl_muxed_h2d[0];
32 assign tl_muxed_d2h[0] = tl_d2h_i;
33
34 // Error port is Idx1
35 assign tl_error_h2d = tl_muxed_h2d[1];
36 assign tl_muxed_d2h[1] = tl_error_d2h;
37
38 // Use a tlul_socket_in to mux between the genuine response
and the error response. The
39 // tlul_socket_in keeps track of outstanding transactions
and ensures that all responses
40 // happen in the correct order.
41 tlul_socket_in #(
42     .N(2),
43     .ExplicitErrs(0)
44 ) u_socket (
45     .clk_i,
46     .rst_ni,
47     .tl_h_i (tl_h2d_i),
48     .tl_h_o (tl_d2h_o),
49     .tl_d_o (tl_muxed_h2d),
50     .tl_d_i (tl_muxed_d2h),
51     .dev_select_i (loopback_request)
52 );
53
54 // Assemble the All-Zero response with error set if the re-
quest is squashed
55 tlul_pkg::tl_d2h_t tl_razwi_rsp_pre_intg;
56 logic loopback_valid_q;
57
58 prim_flop #(
59     .Width(1)
60 ) u_loopback_valid (
61     .clk_i (clk_i),
62     .rst_ni (rst_ni),
63     .d_i (tl_error_h2d.a_valid),
64     .q_o (loopback_valid_q)
65 );
66
67 logic [$bits(tlul_pkg::tl_d_op_e)-1:0] d_opcode_q;
68 prim_flop #(
69     .Width($bits(tlul_pkg::tl_d_op_e)),
70     .ResetValue({AccessAck})
71 ) u_rsp_opcode (
72     .clk_i
73 ),
74 ( rst_ni
75 ),

```

```

74     .d_i ( (tl_error_h2d.a_opcode == Get) ? AccessAckData :
AccessAck ),
75     .q_o (d_op-
76 )
77 assign tl_razwi_rsp_pre_intg.d_opcode =
tlul_pkg::tl_d_op_e'(d_opcode_q);
78
79 prim_flop #(
80     .Width(top_pkg::TL_SZW)
81 ) u_rsp_size (
82     .clk_i (clk_i),
83     .rst_ni (rst_ni),
84     .d_i (tl_error_h2d.a_size),
85     .q_o (tl_razwi_rsp_pre_intg.d_size)
86 );
87
88 prim_flop #(
89     .Width(top_pkg::TL_AIW)
90 ) u_rsp_source (
91     .clk_i (clk_i),
92     .rst_ni (rst_ni),
93     .d_i (tl_error_h2d.a_source),
94     .q_o (tl_razwi_rsp_pre_intg.d_source)
95 );
96
97 // Assemble the RAZWI (Return as Zero, Write Ignored) re-
sponse
98 assign tl_razwi_rsp_pre_intg.d_valid = loopback_valid_q;
99 assign tl_razwi_rsp_pre_intg.a_ready = 1'b1;
100
101 assign tl_razwi_rsp_pre_intg.d_param = '0;
102 assign tl_razwi_rsp_pre_intg.d_sink = '0;
103 assign tl_razwi_rsp_pre_intg.d_data = '0;
104 assign tl_razwi_rsp_pre_intg.d_user = '0;
105 assign tl_razwi_rsp_pre_intg.d_error = ErrorRsp;
106
107 // Compute integrity bits from the manually assembled RAZWI
response
108 tlul_rsp_intg_gen gen_intg_razwi_rsp (
109     .tl_i (tl_razwi_rsp_pre_intg),
110     .tl_o (tl_error_d2h)
111 );
112
113 // Not all TLUL signals are read from the request muxed to
the error leg
114 logic unused_tlul_signals;
115 assign unused_tlul_signals = ^{tl_error_h2d.a_param,
tl_error_h2d.a_address,
tl_error_h2d.a_mask,
tl_error_h2d.a_data,
tl_error_h2d.a_user,
tl_error_h2d.d_ready};
116
117 endmodule

```

25. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_rsp_intg_chk.sv (54 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8  * Tile-Link UL response integrity check
9  */
10
11 module tlul_rsp_intg_chk import tlul_pkg::*; # (
12     parameter bit EnableRspDataIntgCheck = 0
13 ) (
14     // TL-UL interface
15     input tl_d2h_t tl_i,
16
17     // error output
18     output logic err_o
19 );
20
21 logic [1:0] rsp_err;
22 tl_d2h_rsp_intg_t rsp;
23 assign rsp = extract_d2h_rsp_intg(tl_i);
24
25 prim_secded_inv_64_57_dec u_chk (
26     .data_i({tl_i.d_user.rsp_intg, D2HRspMaxWidth'(rsp)}),
27     .data_o(),
28     .syndrome_o(),
29     .err_o(rsp_err)
30 );
31
32 logic rsp_data_err;
33 if (EnableRspDataIntgCheck) begin : gen_rsp_data_intg_check
34     tlul_data_integ_dec u_tlul_data_integ_dec (
35         .data_intg_i({tl_i.d_user.data_intg, DataMax-
width'(tl_i.d_data)}),
36         .data_err_o(rsp_data_err)
37     );
38 end else begin : gen_no_rsp_data_intg_check

```



```

39     assign rsp_data_err = 1'b0;
40 end
41
42 // error is not permanently latched as rsp_intg_chk is typ-
ically
43 // used near the host.
44 // if the error is permanent, it would imply the host could
forever
45 // receive bus errors and lose all ability to debug.
46 // It should be up to the host to determine the permanence
of this
47 error.
48 assign err_o = t_l_i.d_valid & (!rsp_err | rsp_data_err);
49
50 logic unused_tl;
51 assign unused_tl = !t_l_i;
52
53 `ASSERT_INIT(PayLoadWidthCheck, $bits(t_l_d2h_rsp_intg_t) <=
D2HRspMaxWidth)
54 endmodule // tlul_rsp_intg_chk

```

26. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_rsp_intg_gen.sv (87 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /*
8  *
9  * Tile-Link UL response integrity generator
10
11 This generates integrity bits that get stored in the rsp_intg
and data_intg fields of t_l_o.d_user.
12
13 If EnableRspIntgGen is true then the rsp_intg field is gener-
ated from the opcode, d_size and
14 d_error fields of the response (extracted with tlul_pkg::ex-
tract_d2h_rsp_intg). If it is false then
15 the rsp_intg field either comes from the same field in the
t_l_i input (if UserInIsZero is false) or
16 is wired to zero (if UserInIsZero is true).
17
18 If EnableDataIntgGen is true then the data_intg field is gen-
erated from the d_data field of the
19 response. If it is false then the rsp_intg field either comes
from the same field in the t_l_i input
20 (if UserInIsZero is false) or is wired to zero (if UserIn-
IsZero is true).
21
22 */
23
24 module tlul_rsp_intg_gen import tlul_pkg::*; #(
25     parameter bit EnableRspIntgGen = 1'b1,
26     parameter bit EnableDataIntgGen = 1'b1,
27     parameter bit UserInIsZero = 1'b0,
28     parameter bit RspIntgInIsZero = UserInIsZero
29 ) (
30     // TL-UL interface
31     input t_l_d2h_t t_l_i,
32     output t_l_d2h_t t_l_o
33 );
34
35 logic [D2HRspIntgWidth-1:0] rsp_intg;
36 if (EnableRspIntgGen) begin : gen_rsp_intg
37     t_l_d2h_rsp_intg_t rsp;
38     logic [D2HRspMaxWidth-1:0] unused_payload;
39
40     assign rsp = extract_d2h_rsp_intg(t_l_i);
41
42     prim_secded_inv_64_57_enc u_rsp_gen (
43         .data_i(D2HRspMaxWidth'(rsp)),
44         .data_o({rsp_intg, unused_payload})
45     );
46 end else if (RspIntgInIsZero) begin : gen_zero_rsp_intg
47     assign rsp_intg = 0;
48 end else begin : gen_passthrough_rsp_intg
49     assign rsp_intg = t_l_i.d_user.rsp_intg;
50 end
51
52 logic [DataIntgWidth-1:0] data_intg;
53 if (EnableDataIntgGen) begin : gen_data_intg
54     logic [DataMaxWidth-1:0] unused_data;
55     tlul_data_integ_enc u_tlul_data_integ_enc (
56         .data_i(DataMaxWidth'(t_l_i.d_data)),
57         .data_intg_o({data_intg, unused_data})
58     );
59 end else if (UserInIsZero) begin : gen_zero_data_intg
60     assign data_intg = 0;
61 end else begin : gen_passthrough_data_intg
62     assign data_intg = t_l_i.d_user.data_intg;
63 end
64
65 always_comb begin

```

```

66     t_l_o = t_l_i;
67     t_l_o.d_user.rsp_intg = rsp_intg;
68     t_l_o.d_user.data_intg = data_intg;
69 end
70
71 logic unused_tl;
72 assign unused_tl = ^t_l_i;
73
74
75 `ASSERT_INIT(PayLoadWidthCheck, $bits(t_l_d2h_rsp_intg_t) <=
D2HRspMaxWidth)
76 `ASSERT_INIT(DataWidthCheck_A, $bits(t_l_i.d_data) <=
DataMaxWidth)
77
78 // the code below is not meant to be synthesized,
79 // but it is intended to be used in simulation and FPV
80 `ifndef SYNTHESIS
81     always @(t_l_i) begin
82         `ASSERT_I(RspZero_A, t_l_i.d_valid & RspIntgInIsZero -> ~|
t_l_i.d_user.rsp_intg)
83         `ASSERT_I(UserZero_A, t_l_i.d_valid & UserInIsZero -> ~|
t_l_i.d_user)
84     end
85 `endif
86
87 endmodule // tlul_rsp_intg_gen

```

27. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_1n.sv (255 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 // TL-UL socket 1:N module
6
7 // configuration settings
8     device_count: 4
9
10 // Verilog parameters
11 // HReqPass: if 1 then host requests can pass through
on empty fifo,
12 // default 1
13 // HRspPass: if 1 then host responses can pass through
on empty fifo,
14 // default 1
15 // DReqPass: (one per device_count) if 1 then device i
requests can
16 // pass through on empty fifo, default 1
17 // DRspPass: (one per device_count) if 1 then device i
responses can
18 // pass through on empty fifo, default 1
19 // HReqDepth: Depth of host request FIFO, default 2
20 // HRspDepth: Depth of host response FIFO, default 2
21 // DReqDepth: (one per device_count) Depth of device i
request FIFO,
22 // default 2
23 // DRspDepth: (one per device_count) Depth of device i
response FIFO,
24 // default 2
25
26 // ExplicitErrs: This module always returns a request error if
dev_select_i
27 // is greater than N-1. If ExplicitErrs is
set then the width
28 // of the dev_select_i signal will be chosen
to make sure that
29 // this is possible. This only makes a dif-
ference if N is a
30 // power of 2.
31
32 // Requests must stall to one device until all responses from
other devices
33 // have returned. Need to keep a counter of all outstanding
requests and
34 // wait until that counter is zero before switching devices.
35
36 // This module will return a request error if the input value
of 'dev_select_i'
37 // is not within the range 0..N-1. Thus the instantiator of
the socket
38 // can indicate error by any illegal value of dev_select_i.
4'b1111 is
39 // recommended for visibility
40
41 // The maximum value of N is 63
42
43 `include "prim_assert.sv"
44
45 module tlul_socket_1n #(
46     parameter int unsigned N = 4,
47     parameter bit HReqPass = 1'b1,
48     parameter bit HRspPass = 1'b1,
49     parameter bit [N-1:0] DReqPass = {N{1'b1}},
50     parameter bit [N-1:0] DRspPass = {N{1'b1}},
51     parameter bit [3:0] HReqDepth = 4'h1,
52     parameter bit [3:0] HRspDepth = 4'h1,
53     parameter bit [N*4-1:0] DReqDepth = {N{4'h1}},

```

```

53 parameter bit [N*4-1:0] DRspDepth    = {N{4'h1}},
54 parameter bit          ExplicitErrs = 1'b1,
55
56 // The width of dev_select_i. We must be able to select any
of the N devices
57 // (i.e. values 0..N-1). If ExplicitErrs is set, we also
need to be able to
58 // represent N.
59 localparam int unsigned NWD = $clog2(ExplicitErrs ? N+1 : N)
60 ) (
61   input          clk_i,
62   input          rst_ni,
63   input  tlul_pkg::tl_h2d_t  tl_h_i,
64   output tlul_pkg::tl_d2h_t  tl_h_o,
65   output tlul_pkg::tl_h2d_t  tl_d_o  [N],
66   input  tlul_pkg::tl_d2h_t  tl_d_i  [N],
67   input  [NWD-1:0]          dev_select_i
68 );
69
70 `ASSERT_INIT(maxN, N < 64)
71
72 // Since our steering is done after potential FIFOing, we
need to
73 // shove our device select bits into spare bits of reqfifo
74
75 // instantiate the host fifo, create intermediate bus 't'
76
77 // FIFO'd version of device select
78 logic [NWD-1:0] dev_select_t;
79
80 tlul_pkg::tl_h2d_t  tl_t_o;
81 tlul_pkg::tl_d2h_t  tl_t_i;
82
83 tlul_fifo_sync #(
84   .ReqPass(HReqPass),
85   .RspPass(HRspPass),
86   .ReqDepth(HReqDepth),
87   .RspDepth(HRspDepth),
88   .SpareReqW(NWD)
89 ) fifo_h (
90   .clk_i,
91   .rst_ni,
92   .tl_h_i,
93   .tl_h_o,
94   .tl_d_o  (tl_t_o),
95   .tl_d_i  (tl_t_i),
96   .spare_req_i (dev_select_i),
97   .spare_req_o (dev_select_t),
98   .spare_rsp_i (1'b0),
99   .spare_rsp_o ());
100
101
102 // We need to keep track of how many requests are outstanding,
103 // and to which device. New requests are compared to this
and
104 // stall until that number is zero.
105 localparam int MaxOutstanding = 2**top_pkg::TL_AIW; // Up to
256 outstanding
106 localparam int OutstandingW = $clog2(MaxOutstanding+1);
107 logic [OutstandingW-1:0] num_req_outstanding;
108 logic [NWD-1:0]          dev_select_outstanding;
109 logic                   hold_all_requests;
110 logic                   accept_t_req;
111
112 assign accept_t_req = tl_t_o.a_valid & tl_t_i.a_ready;
113 assign accept_t_rsp = tl_t_i.d_valid & tl_t_o.d_ready;
114
115 always_ff @(posedge clk_i or negedge rst_ni) begin
116   if (!rst_ni) begin
117     num_req_outstanding <= '0;
118     dev_select_outstanding <= '0;
119   end else if (accept_t_req) begin
120     if (!accept_t_rsp) begin
121       num_req_outstanding <= num_req_outstanding + 1'b1;
122     end
123     dev_select_outstanding <= dev_select_t;
124   end else if (accept_t_rsp) begin
125     num_req_outstanding <= num_req_outstanding - 1'b1;
126   end
127 end
128
129 `ASSERT(NotOverflowed_A,
130   accept_t_req && !accept_t_rsp -> num_req_outstanding
<= MaxOutstanding)
131
132 assign hold_all_requests =
133   (num_req_outstanding != '0) &
134   (dev_select_t != dev_select_outstanding);
135
136 // Make N copies of 't' request side with modified reqvalid,
call
137 // them 'u[0]' .. 'u[n-1]'.
138
139 tlul_pkg::tl_h2d_t  tl_u_o [N+1];
140 tlul_pkg::tl_d2h_t  tl_u_i [N+1];
141
142 // ensure that when a device is not selected, both command

```

```

143 // data integrity can never match
144 tlul_pkg::tl_a_user_t blanked_auser;
145 assign blanked_auser = '{
146   rsvd: tl_t_o.a_user.rsvd,
147   instr_type: tl_t_o.a_user.instr_type,
148   cmd_intg: tlul_pkg::get_bad_cmd_intg(tl_t_o),
149   data_intg:
tlul_pkg::get_bad_data_intg(tlul_pkg::BlankedAData)
150 };
151
152 // if a host is not selected, or if requests are held off,
blank the bus
153 for (genvar i = 0 ; i < N ; i++) begin : gen_u_o
154   logic dev_select;
155   assign dev_select = dev_select_t == NWD'(i) &
~hold_all_requests;
156
157   assign tl_u_o[i].a_valid  = tl_t_o.a_valid & dev_select;
158   assign tl_u_o[i].a_opcode = tl_t_o.a_opcode;
159   assign tl_u_o[i].a_param  = tl_t_o.a_param;
160   assign tl_u_o[i].a_size   = tl_t_o.a_size;
161   assign tl_u_o[i].a_source = tl_t_o.a_source;
162   assign tl_u_o[i].a_address = tl_t_o.a_address;
163   assign tl_u_o[i].a_mask   = tl_t_o.a_mask;
164   assign tl_u_o[i].a_data   = dev_select ?
165     tl_t_o.a_data :
166     tlul_pkg::BlankedAData;
167   assign tl_u_o[i].a_user   = dev_select ?
168     tl_t_o.a_user :
169     blanked_auser;
170
171   assign tl_u_o[i].d_ready  = tl_t_o.d_ready;
172 end
173
174
175 tlul_pkg::tl_d2h_t tl_t_p ;
176
177 // for the returning reqready, only look at the device we're
addressing
178 logic hfifo_reqready;
179 always_comb begin
180   hfifo_reqready = tl_u_i[N].a_ready; // default to error
181   for (int idx = 0 ; idx < N ; idx++) begin
182     //if (dev_select_outstanding == NWD'(idx))
hfifo_reqready = tl_u_i[idx].a_ready;
183     if (dev_select_t == NWD'(idx)) hfifo_reqready =
tl_u_i[idx].a_ready;
184   end
185   if (hold_all_requests) hfifo_reqready = 1'b0;
186 end
187 // Adding a_valid as a qualifier. This prevents the a_ready
from having unknown value
188 // when the address is unknown and the Host TL-UL FIFO is
bypass mode.
189 assign tl_t_i.a_ready = tl_t_o.a_valid & hfifo_reqready;
190
191 always_comb begin
192   tl_t_p = tl_u_i[N];
193   for (int idx = 0 ; idx < N ; idx++) begin
194     if (dev_select_outstanding == NWD'(idx)) tl_t_p =
tl_u_i[idx];
195   end
196 end
197 assign tl_t_i.d_valid  = tl_t_p.d_valid ;
198 assign tl_t_i.d_opcode = tl_t_p.d_opcode;
199 assign tl_t_i.d_param  = tl_t_p.d_param ;
200 assign tl_t_i.d_size   = tl_t_p.d_size ;
201 assign tl_t_i.d_source = tl_t_p.d_source;
202 assign tl_t_i.d_sink   = tl_t_p.d_sink ;
203 assign tl_t_i.d_data   = tl_t_p.d_data ;
204 assign tl_t_i.d_user   = tl_t_p.d_user ;
205 assign tl_t_i.d_error  = tl_t_p.d_error ;
206
207 // Instantiate all the device FIFOs
208 for (genvar i = 0 ; i < N ; i++) begin : gen_dfifo
209   tlul_fifo_sync #(
210     .ReqPass(DReqPass[i]),
211     .RspPass(DRspPass[i]),
212     .ReqDepth(DReqDepth[i*4+4]),
213     .RspDepth(DRspDepth[i*4+4])
214   ) fifo_d (
215     .clk_i,
216     .rst_ni,
217     .tl_h_i  (tl_u_o[i]),
218     .tl_h_o  (tl_u_i[i]),
219     .tl_d_o  (tl_d_o[i]),
220     .tl_d_i  (tl_d_i[i]),
221     .spare_req_i (1'b0),
222     .spare_req_o ( ),
223     .spare_rsp_i (1'b0),
224     .spare_rsp_o ());
225 end
226
227 // Instantiate the error responder. It's only needed if a
value greater than
228 // N-1 is actually representable in NWD bits.
229 if ($clog2(N+1) <= NWD) begin : gen_err_rsp
230   assign tl_u_o[N].d_ready  = tl_t_o.d_ready;
231   assign tl_u_o[N].a_valid  = tl_t_o.a_valid &
232     (dev_select_t >= NWD'(N)) &

```

```

233             ~hold_all_requests;
234     assign tl_u_o[N].a_opcode = tl_t_o.a_opcode;
235     assign tl_u_o[N].a_param = tl_t_o.a_param;
236     assign tl_u_o[N].a_size = tl_t_o.a_size;
237     assign tl_u_o[N].a_source = tl_t_o.a_source;
238     assign tl_u_o[N].a_address = tl_t_o.a_address;
239     assign tl_u_o[N].a_mask = tl_t_o.a_mask;
240     assign tl_u_o[N].a_data = tl_t_o.a_data;
241     assign tl_u_o[N].a_user = tl_t_o.a_user;
242     tlul_err_resp err_resp (
243         .clk_i,
244         .rst_ni,
245         .tl_h_i (tl_u_o[N]),
246         .tl_h_o (tl_u_i[N])
247     );
248 end else begin : gen_no_err_resp // block: gen_err_resp
249     assign tl_u_o[N] = '0;
250     assign tl_u_i[N] = '0;
251     logic unused_sig;
252     assign unused_sig = ^tl_u_o[N];
253 end
254
255 endmodule

```

28. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_m1.sv (260 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // TL-UL socket M:1 module
6 //
7 // Verilog parameters
8 // M: Number of host ports.
9 // HReqPass: M bit array to allow requests to pass
  through the host i
10 // FIFO with no clock delay if the request
  FIFO is empty. If
11 // 1'b0, at least one clock cycle of latency
  is created.
12 // Default is 1'b1.
13 // HRspPass: Same as HReqPass but for host response
  FIFO.
14 // HReqDepth: Mx4 bit array. bit[i*4+:4] is depth of
  host i request FIFO.
15 // Depth of zero is allowed if ReqPass is
  true. A maximum value
16 // of 16 is allowed, default is 2.
17 // HRspDepth: Same as HReqDepth but for host response
  FIFO.
18 // DReqPass: Same as HReqPass but for device request
  FIFO.
19 // DRspPass: Same as HReqPass but for device response
  FIFO.
20 // DReqDepth: Same as HReqDepth but for device request
  FIFO.
21 // DRspDepth: Same as HReqDepth but for device response
  FIFO.
22
23 `include "prim_assert.sv"
24
25 module tlul_socket_m1 #(
26     parameter int unsigned M = 4,
27     parameter bit [M-1:0] HReqPass = {M{1'b1}},
28     parameter bit [M-1:0] HRspPass = {M{1'b1}},
29     parameter bit [M*4-1:0] HReqDepth = {M{4'h1}},
30     parameter bit [M*4-1:0] HRspDepth = {M{4'h1}},
31     parameter bit DReqPass = 1'b1,
32     parameter bit DRspPass = 1'b1,
33     parameter bit [3:0] DReqDepth = 4'h1,
34     parameter bit [3:0] DRspDepth = 4'h1
35 ) (
36     input clk_i,
37     input rst_ni,
38
39     input tlul_pkg::tl_h2d_t tl_h_i [M],
40     output tlul_pkg::tl_d2h_t tl_h_o [M],
41
42     output tlul_pkg::tl_h2d_t tl_d_o,
43     input tlul_pkg::tl_d2h_t tl_d_i
44 );
45
46 `ASSERT_INIT(maxM, M < 16)
47
48 // Signals
49 //
50 // tl_h_i/o[0] | tl_h_i/o[1] | ... | tl_h_i/o[M-1]
51 // | | | | |
52 // | | | | |
53 // u_hostfifo[0] u_hostfifo[1] u_hostfifo[M-1]
54 // | | | | |
55 // | | | | |
56 // hreq_fifo_o(i) / hrsp_fifo_i(i)
57 // |-----|
58 // | request/grant/req_data |
59 // | | | | |
60 // | PRIM_ARBITER |

```

```

61 // | arb_valid / arb_ready / arb_data |
62 // |-----|
63 // |
64 // | dreq_fifo_i / drsp_fifo_o
65 // |
66 // | u_devicefifo
67 // |
68 // | tl_d_o/i
69 //
70 // Required ID width to distinguish between host ports
71 // Used in response steering
72 localparam int unsigned IDW = top_pkg::TL_AIW;
73 localparam int unsigned STIDW = $clog2(M);
74
75 tlul_pkg::tl_h2d_t hreq_fifo_o [M];
76 tlul_pkg::tl_d2h_t hrsp_fifo_i [M];
77
78 logic [M-1:0] hrequest;
79 logic [M-1:0] hgrant;
80
81 tlul_pkg::tl_h2d_t dreq_fifo_i;
82 tlul_pkg::tl_d2h_t drsp_fifo_o;
83
84 logic arb_valid;
85 logic arb_ready;
86 tlul_pkg::tl_h2d_t arb_data;
87
88 // Host Req/Rsp FIFO
89 for (genvar i = 0; i < M; i++) begin : gen_host_fifo
90     tlul_pkg::tl_h2d_t hreq_fifo_i;
91
92     // ID Shifting
93     logic [STIDW-1:0] reqid_sub;
94     logic [IDW-1:0] shifted_id;
95     assign reqid_sub = i; // can cause conversion error?
96     assign shifted_id = {
97         tl_h_i[i].a_source[0+: (IDW-STIDW)],
98         reqid_sub
99     };
100
101     `ASSERT(idInRange, tl_h_i[i].a_valid |->
  tl_h_i[i].a_source[IDW-1 -:STIDW] == '0)
102
103     // assign not connected bits to nc_* signal to make lint
  happy
104     logic [IDW-1 : IDW-STIDW] unused_tl_h_source;
105     assign unused_tl_h_source = tl_h_i[i].a_source[IDW-1 -:
  STIDW];
106
107     // Put shifted ID
108     assign hreq_fifo_i = '{
109         a_valid: tl_h_i[i].a_valid,
110         a_opcode: tl_h_i[i].a_opcode,
111         a_param: tl_h_i[i].a_param,
112         a_size: tl_h_i[i].a_size,
113         a_source: shifted_id,
114         a_address: tl_h_i[i].a_address,
115         a_mask: tl_h_i[i].a_mask,
116         a_data: tl_h_i[i].a_data,
117         a_user: tl_h_i[i].a_user,
118         d_ready: tl_h_i[i].d_ready
119     };
120
121     tlul_fifo_sync #(
122         .ReqPass (HReqPass[i]),
123         .RspPass (HRspPass[i]),
124         .ReqDepth (HReqDepth[i*4+:4]),
125         .RspDepth (HRspDepth[i*4+:4]),
126         .SpareReqW (1)
127     ) u_hostfifo (
128         .clk_i,
129         .rst_ni,
130         .tl_h_i (hreq_fifo_i),
131         .tl_h_o (tl_h_o[i]),
132         .tl_d_o (hreq_fifo_o[i]),
133         .tl_d_i (hrsp_fifo_i[i]),
134         .spare_req_i (1'b0),
135         .spare_req_o (),
136         .spare_rsp_i (1'b0),
137         .spare_rsp_o ()
138     );
139 end
140
141 // Device Req/Rsp FIFO
142 tlul_fifo_sync #(
143     .ReqPass (DReqPass),
144     .RspPass (DRspPass),
145     .ReqDepth (DReqDepth),
146     .RspDepth (DRspDepth),
147     .SpareReqW (1)
148 ) u_devicefifo (
149     .clk_i,
150     .rst_ni,
151     .tl_h_i (dreq_fifo_i),
152     .tl_h_o (drsp_fifo_o),
153     .tl_d_o (tl_d_o),
154     .tl_d_i (tl_d_i),
155     .spare_req_i (1'b0),
156     .spare_req_o (),
157     .spare_rsp_i (1'b0),
158     .spare_rsp_o ()

```

```

159 );
160
161 // Request Arbiter
162 for (genvar i = 0 ; i < M ; i++) begin : gen_arbreqgnt
163     assign hrequest[i] = hreq_fifo_o[i].a_valid;
164 end
165
166 assign arb_ready = drsp_fifo_o.a_ready;
167
168 if (tlul_pkg::ArbiterImpl == "PPC") begin : gen_arb_ppc
169     prim_arbiter_ppc #(
170         .N          (M),
171         .DW          ($bits(tlul_pkg::tl_h2d_t))
172     ) u_reqarb (
173         .clk_i,
174         .rst_ni,
175         .req_chk_i ( 1'b0 ), // TL-UL allows dropping
valid without ready. See #3354.
176         .req_i      ( hrequest ),
177         .data_i      ( hreq_fifo_o ),
178         .gnt_o       ( hgrant ),
179         .idx_o       ( ),
180         .valid_o      ( arb_valid ),
181         .data_o       ( arb_data ),
182         .ready_i      ( arb_ready );
183     end else if (tlul_pkg::ArbiterImpl == "BINTREE") begin :
gen_tree_arb
184         prim_arbiter_tree #(
185             .N          (M),
186             .DW          ($bits(tlul_pkg::tl_h2d_t))
187         ) u_reqarb (
188             .clk_i,
189             .rst_ni,
190             .req_chk_i ( 1'b0 ), // TL-UL allows dropping
valid without ready. See #3354.
191             .req_i      ( hrequest ),
192             .data_i      ( hreq_fifo_o ),
193             .gnt_o       ( hgrant ),
194             .idx_o       ( ),
195             .valid_o      ( arb_valid ),
196             .data_o       ( arb_data ),
197             .ready_i      ( arb_ready );
198         end else begin : gen_unknown
199             `ASSERT_INIT(UnknownArbImpl_A, 0)
200         end
201     end
202
203     logic [ M-1:0] hfifo_rspvalid;
204     logic [ M-1:0] dfifo_rspready;
205     logic [IDW-1:0] hfifo_rspid;
206     logic dfifo_rspready_merged;
207
208     // arb_data --> dreq_fifo_i
209     // dreq_fifo_i.hd_rspready <= dfifo_rspready
210
211

```

```

212 assign dfifo_rspready_merged = dfifo_rspready;
213 assign dreq_fifo_i = '{
214     a_valid:    arb_valid,
215     a_opcode:   arb_data.a_opcode,
216     a_param:    arb_data.a_param,
217     a_size:     arb_data.a_size,
218     a_source:   arb_data.a_source,
219     a_address:  arb_data.a_address,
220     a_mask:     arb_data.a_mask,
221     a_data:     arb_data.a_data,
222     a_user:     arb_data.a_user,
223
224     d_ready:    dfifo_rspready_merged
225 };
226
227 // Response ID steering
228 // drsp_fifo_o --> hrsp_fifo_i[i]
229
230 // Response ID shifting before put into host fifo
231 assign hfifo_rspid = {
232     {STIDW{1'b0}},
233     drsp_fifo_o.d_source[IDW-1:STIDW]
234 };
235 for (genvar i = 0 ; i < M ; i++) begin : gen_idrouting
236     assign hfifo_rspvalid[i] = drsp_fifo_o.d_valid &
237                                     (drsp_fifo_o.d_source[0+:STIDW]
== i);
238     assign dfifo_rspready[i] =
hreq_fifo_o[i].d_ready &
239                                     (drsp_fifo_o.d_source[0+:STIDW]
== i) &
240                                     drsp_fifo_o.d_valid;
241
242     assign hrsp_fifo_i[i] = '{
243         d_valid:    hfifo_rspvalid[i],
244         d_opcode:   drsp_fifo_o.d_opcode,
245         d_param:    drsp_fifo_o.d_param,
246         d_size:     drsp_fifo_o.d_size,
247         d_source:   hfifo_rspid,
248         d_sink:     drsp_fifo_o.d_sink,
249         d_data:     drsp_fifo_o.d_data,
250         d_user:     drsp_fifo_o.d_user,
251         d_error:    drsp_fifo_o.d_error,
252         a_ready:    hgrant[i]
253     };
254 end
255
256 // this assertion fails when rspid[0+:STIDW] not in [0..M-1]
257 `ASSERT(rspidInRange, drsp_fifo_o.d_valid |->
258     drsp_fifo_o.d_source[0+:STIDW] >= 0 &&
drsp_fifo_o.d_source[0+:STIDW] < M)
259
260 endmodule

```

SRAM 컨트롤러(스크램블)

What this is. An SRAM controller that scrambles addresses and data and adds ECC.

Why it exists. Because secrets in memory can be read physically. Decapsulating a die and probing SRAM cells is a practised attack, so contents are scrambled; independently, soft errors require ECC.

Undergraduate review · EECS concepts. Lightweight block ciphers (PRINCE and similar); SECDED Hamming codes; soft-error rate and FIT units.

Graduate · Practice note. ECC adds a cycle of latency. Confirm early that the pipeline can absorb it; discovering this after timing closure forces a redesign.

5 files • 1,933 lines

API — sram_ctrl.sv

Table 1150. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl.sv — module / interface / package (1개)

이름	종류
sram_ctrl	module

Table 1151. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl.sv — parameter (17개)

파라미터	형/폭	기본값
MemSizeRam	int	32'h1000
InstSize	int	MemSizeRam
NumRamInst	int	1
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
InstrExec	bit	1
NumPrinceRoundsHalf	int	3
NumAddrScrRounds	int	2
Outstanding	int	2
EccCorrection	bit	0
EnableRacL	bit	1'b0
RacLErrorRsp	bit	EnableRacL
RacLPolicySelNumRangesRam	int unsigned	1
RndCnstSramKey	otp_ctrl_pkg::sram_key_t	RndCnstSramKeyDefault
RndCnstSramNonce	otp_ctrl_pkg::sram_nonce_t	RndCnstSramNonceDefault
RndCnstLfsrSeed	lfsr_seed_t	RndCnstLfsrSeedDefault
RndCnstLfsrPerm	lfsr_perm_t	RndCnstLfsrPermDefault

Table 1152. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl.sv — localparam (4개)

상수	형/폭	값
Depth	int unsigned	MemSizeRam >> 2
InstDepth	int unsigned	InstSize >> 2
AddrWidth	int unsigned	prim_util_pkg::vbits(Depth)
TotalAnchorWidth	int	\$bits(otp_ctrl_pkg::sram_key_t) +

Table 1153. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl.sv — 포트 (21개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
clk_otp_i	input	logic
rst_otp_ni	input	logic
ram_tl_i	input	tlul_pkg::tl_h2d_t
ram_tl_o	output	tlul_pkg::tl_d2h_t
regs_tl_i	input	tlul_pkg::tl_h2d_t
regs_tl_o	output	tlul_pkg::tl_d2h_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
racl_policy_sel_ranges_ram_i	input	top_racl_pkg::racl_range_t [RaclPolicySelNumRanges...
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
lc_hw_debug_en_i	input	lc_ctrl_pkg::lc_tx_t
otp_en_sram_ifetch_i	input	prim_mubi_pkg::mubi8_t
sram_otp_key_o	output	otp_ctrl_pkg::sram_otp_key_req_t
sram_otp_key_i	input	otp_ctrl_pkg::sram_otp_key_rsp_t
ram_cfg_i	input	prim_ram_1p_pkg::ram_1p_cfg_req_t [NumRamInst-1:0]
ram_cfg_o	output	prim_ram_1p_pkg::ram_1p_cfg_rsp_t [NumRamInst-1:0]
sram_rerror_o	output	sram_ctrl_pkg::sram_error_t

API — sram_ctrl_pkg.sv**Table 1154. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_pkg.sv — module / interface / package (1개)**

이름	종류
sram_ctrl_pkg	package

Table 1155. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_pkg.sv — parameter (8개)

파라미터	형/폭	기본값
DataWidth	int	32 + tlul_pkg::DataIntgWidth
NonceWidth	int	64
RndCnstSramKeyDefault	otp_ctrl_pkg::sram_key_t	128'hbecda03b34bc0418a30a33861a610f71
RndCnstSramNonceDefault	otp_ctrl_pkg::sram_nonce_t	128'h22f296f8f95efb84a75cd435a5541e9f
LfsrWidth	int	64
RndCnstLfsrSeedDefault	lfsr_seed_t	64'hb496209a_10a81ea5
RndCnstLfsrPermDefault	lfsr_perm_t	{
LfsrOutWidth	int	32

Table 1156. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_pkg.sv — typedef / using (2개)

새 이름	원래 형
lfsr_seed_t	logic [LfsrWidth-1:0]
lfsr_perm_t	logic [LfsrWidth-1:0][\$clog2(LfsrWidth)-1:0]

API — sram_ctrl_ram_reg_top.sv

Table 1157. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_ram_reg_top.sv — module / interface / package (1개)

이름	종류
sram_ctrl_ram_reg_top	module

Table 1158. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_ram_reg_top.sv — parameter (2개)

파라미터	형/폭	기본값
EnableRacl	bit	1'b0
RaclErrorRsp	bit	1'b1

Table 1159. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_ram_reg_top.sv — 포트 (7개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
rac1_policies_i	input	top_rac1_pkg::rac1_policy_vec_t
rac1_error_o	output	top_rac1_pkg::rac1_error_log_t
intg_err_o	output	logic

API — sram_ctrl_reg_pkg.sv

Table 1160. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_reg_pkg.sv — module / interface / package (1개)

이름	종류
sram_ctrl_reg_pkg	package

Table 1161. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_reg_pkg.sv — parameter (14개)

파라미터	형/폭	기본값
NumAlerts	int	1
RegsAw	int	6
NumRegsRegs	int	9
SRAM_CTRL_ALERT_TEST_OFFSET	logic [RegsAw-1:0]	6'h 0
SRAM_CTRL_STATUS_OFFSET	logic [RegsAw-1:0]	6'h 4
SRAM_CTRL_EXEC_REGWEN_OFFSET	logic [RegsAw-1:0]	6'h 8
SRAM_CTRL_EXEC_OFFSET	logic [RegsAw-1:0]	6'h c
SRAM_CTRL_CTRL_REGWEN_OFFSET	logic [RegsAw-1:0]	6'h 10
SRAM_CTRL_CTRL_OFFSET	logic [RegsAw-1:0]	6'h 14
SRAM_CTRL_SCR_KEY_ROTATED_OFFSET	logic [RegsAw-1:0]	6'h 18
SRAM_CTRL_READBACK_REGWEN_OFFSET	logic [RegsAw-1:0]	6'h 1c
SRAM_CTRL_READBACK_OFFSET	logic [RegsAw-1:0]	6'h 20
SRAM_CTRL_ALERT_TEST_RESVAL	logic [0:0]	1'h 0
SRAM_CTRL_ALERT_TEST_FATAL_ERROR_RESVAL	logic [0:0]	1'h 0

API — sram_ctrl_regs_reg_top.sv**Table 1162. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_regs_reg_top.sv — module / interface / package (1개)**

이름	종류
sram_ctrl_regs_reg_top	module

Table 1163. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_regs_reg_top.sv — parameter (2개)

파라미터	형/폭	기본값
EnableRacl	bit	1'b0
RaclErrorRsp	bit	1'b1

Table 1164. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_regs_reg_top.sv — localparam (3개)

상수	형/폭	값
AW	int	6
DW	int	32
DBW	int	DW/8

Table 1165. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_regs_reg_top.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
reg2hw	output	sram_ctrl_reg_pkg::sram_ctrl_regs_reg2hw_t
hw2reg	input	sram_ctrl_reg_pkg::sram_ctrl_regs_hw2reg_t
rac_l_policies_i	input	top_rac_l_pkg::rac_l_policy_vec_t
rac_l_error_o	output	top_rac_l_pkg::rac_l_error_log_t
intg_err_o	output	logic

1. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl.sv (799 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // SRAM controller.
6 //
7
8 `include "prim_assert.sv"
9
10 module sram_ctrl
11     import sram_ctrl_pkg::*;
12     import sram_ctrl_reg_pkg::*;
13 #()
14     // Number of words stored in the SRAM.
15     parameter int MemSizeRam =
32'h1000,
16     parameter int InstSize =
MemSizeRam,
17     parameter int NumRamInst =
1,
18     // Enable asynchronous transitions on alerts.
19     parameter logic [NumAlerts-1:0] AlertAsyncOn =
{NumAlerts{1'b1}},
20     // Number of cycles a differential skew is tolerated on the
  alert signal
21     parameter int unsigned AlertSkewCycles =
1,
22     // Enables the execute from SRAM feature.
23     parameter bit InstrExec =
1,
24     // Number of PRINCE half rounds for the SRAM scrambling fea-
  ture, can be [1..5].
25     // Note that this needs to be low-latency, hence we have to
  keep the amount of cipher rounds low.
26     // PRINCE has 5 half rounds in its original form, which cor-
  responds to 2*5 + 1 effective rounds.
27     // Setting this to 3 lowers this to approximately 7 effect-
  ive rounds.
28     parameter int NumPrinceRoundsHalf =
3,
29     // Number of address scrambling rounds. Setting this to 0
  disables address scrambling.
30     parameter int NumAddrScrRounds =
2,
31     // Number of outstanding TLUL transfers
32     parameter int Outstanding =
2,
33     // Enable single-bit error correction and error logging
34     parameter bit EccCorrection =
0,
35     // RACL configuration
36     parameter bit EnableRacl =
1'b0,
37     parameter bit RacLErrorRsp = En-
ableRacl,
38     parameter top_rac_l_pkg::rac_l_policy_sel_t RacLPoli-
  cySelVecRegs[NumRegsRegs] = '{NumRegsRegs{0}},
39     parameter int unsigned RacLPolicySelNum-
  RangesRam = 1,
40     // Random netlist constants
41     parameter otp_ctrl_pkg::sram_key_t RndCnstSramKey =
RndCnstSramKeyDefault,
42     parameter otp_ctrl_pkg::sram_nonce_t RndCnstSramNonce =
RndCnstSramNonceDefault,

```

```

43     parameter lfsr_seed_t RndCnstLfsrSeed =
RndCnstLfsrSeedDefault,
44     parameter lfsr_perm_t RndCnstLfsrPerm =
RndCnstLfsrPermDefault
45 ) (
46     // SRAM Clock
47     input logic clk_i,
48     input logic rst_ni,
49     // OTP Clock (for key interface)
50     input logic clk_otp_i,
51     input logic rst_otp_ni,
52     // Bus Interface (device) for SRAM
53     input tlul_pkg::tl_h2d_t ram_tl_i,
54     output tlul_pkg::tl_d2h_t ram_tl_o,
55     // Bus Interface (device) for CSRs
56     input tlul_pkg::tl_h2d_t regs_tl_i,
57     output tlul_pkg::tl_d2h_t regs_tl_o,
58     // Alert outputs.
59     input prim_alert_pkg::alert_rx_t [NumAlerts-1:0] alert_rx_i,
60     output prim_alert_pkg::alert_tx_t [NumAlerts-1:0] alert_tx_o,
61     // RACL interface
62     input top_rac_l_pkg::rac_l_policy_vec_t rac_l_policies_i,
63     output top_rac_l_pkg::rac_l_error_log_t rac_l_error_o,
64     input top_rac_l_pkg::rac_l_range_t [RacLPolicySelNumRanges-
  Ram-1:0] rac_l_policy_sel_ranges_ram_i,
65     // Life-cycle escalation input (scraps the scrambling keys)
66     // SEC_CM: LC_ESCALATE_EN.INTERSIG.MUBI
67     input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
68     // SEC_CM: LC_HW_DEBUG_EN.INTERSIG.MUBI
69     input lc_ctrl_pkg::lc_tx_t lc_hw_debug_en_i,
70     // Otp configuration for sram execution
71     // SEC_CM: EXEC.INTERSIG.MUBI
72     input prim_mubi_pkg::mubi8_t otp_en_sram_ifetch_i,
73     // Key request to OTP (running on clk_fixed)
74     // SEC_CM: SCRAMBLE.KEY.SIDELOAD
75     output otp_ctrl_pkg::sram_otp_key_req_t sram_otp_key_o,
76     input otp_ctrl_pkg::sram_otp_key_rsp_t sram_otp_key_i,
77     // config
78     input prim_ram_1p_pkg::ram_1p_cfg_req_t [NumRamInst-1:0] ram_cfg_i,
79     output prim_ram_1p_pkg::ram_1p_cfg_rsp_t [NumRamInst-1:0] ram_cfg_o,
80     // Error record
81     output sram_ctrl_pkg::sram_error_t sram_error_o
82 );
83
84 import lc_ctrl_pkg::lc_tx_t;
85 import lc_ctrl_pkg::lc_tx_test_true_loose;
86 import lc_ctrl_pkg::lc_tx_bool_to_lc_tx;
87 import lc_ctrl_pkg::lc_tx_or_hi;
88 import lc_ctrl_pkg::lc_tx_inv;
89 import lc_ctrl_pkg::lc_to_mubi4;
90 import prim_mubi_pkg::mubi4_t;

```

```

91 import prim_mubi_pkg::mubi8_t;
92 import prim_mubi_pkg::MuBi4True;
93 import prim_mubi_pkg::MuBi4False;
94 import prim_mubi_pkg::mubi8_test_true_strict;
95
96 // The memory can have a non-power-of-2 size (checked inside
97 prim_ram_1p_scr) but the size needs
98 // to be divisible by 4.
99 `ASSERT_INIT(MemSizeRamDivisibleBy4_A, MemSizeRam % 4 == 0)
100
101 // This is later on pruned to the correct width at the SRAM
102 wrapper interface.
103 localparam int unsigned Depth = MemSizeRam >> 2;
104 localparam int unsigned InstDepth = InstSize >> 2;
105 localparam int unsigned AddrWidth =
106 prim_util_pkg::vbits(Depth);
107
108 `ASSERT_INIT(NumRamInstSameAsComputed_A,
109 NumRamInst ==
110 prim_util_pkg::ceil_div(MemSizeRam, InstSize))
111
112 `ASSERT_INIT(NonceWidthsLessThanSource_A, NonceWidth + Lfsr-
113 Width <= otp_ctrl_pkg::SramNonceWidth)
114
115 top_racl_pkg::racl_error_log_t racl_error[2];
116 if (EnableRacl) begin : gen_racl_error_arb
117 // Arbitrate between all simultaneously valid error log
118 requests.
119 prim_racl_error_arb #(
120 .N ( 2 )
121 ) u_prim_err_arb (
122 .clk_i,
123 .rst_ni,
124 .error_log_i ( racl_error ),
125 .error_log_o ( racl_error_o ),
126 );
127 end else begin : gen_no_racl_error_arb
128 logic unused_signals;
129 assign unused_signals = ^{racl_error[0] ^ racl_error[1]};
130 assign racl_error_o = '0;
131 end
132
133 // Anchor incoming seeds and constants
134 ///////////////////////////////////////////////////
135 localparam int TotalAnchorWidth = $bits(otp-c-
136 trl_pkg::sram_key_t) +
137 $bits(otp-c-
138 trl_pkg::sram_nonce_t);
139
140 otp_ctrl_pkg::sram_key_t cnst_sram_key;
141 otp_ctrl_pkg::sram_nonce_t cnst_sram_nonce;
142
143 prim_sec_anchor_buf #(
144 .Width(TotalAnchorWidth)
145 ) u_seed_anchor (
146 .in_i({RndCnstSramKey,
147 RndCnstSramNonce}),
148 .out_o({cnst_sram_key,
149 cnst_sram_nonce})
150 );
151
152 ///////////////////////////////////////////////////
153 // CSR Node and Mapping //
154 ///////////////////////////////////////////////////
155 // We've got two bus interfaces + an lc_gate in this module,
156 // hence three integ failure sources.
157 logic [2:0] bus_integ_error;
158
159 sram_ctrl_regs_reg2hw_t reg2hw;
160 sram_ctrl_regs_hw2reg_t hw2reg;
161
162 // SEC_CM: CTRL.CONFIG.REGWEN
163 // SEC_CM: EXEC.CONFIG.REGWEN
164 // SEC_CM: READBACK.CONFIG.REGWEN
165 sram_ctrl_regs_reg_top #(
166 .EnableRacl ( EnableRacl ),
167 .RaclErrorRsp ( RaclErrorRsp ),
168 .RaclPolicySelVec ( RaclPolicySelVecRegs )
169 ) u_reg_regs (
170 .clk_i,
171 .rst_ni,
172 .tl_i ( regs_tl_i ),
173 .tl_o ( regs_tl_o ),
174 .reg2hw,
175 .hw2reg,
176 // RACL interface
177 .racl_policies_i ( racl_policies_i ),
178 .racl_error_o ( racl_error[0] ),
179 // SEC_CM: BUS.INTEGRITY
180 .integ_err_o ( bus_integ_error[0] )
181 );
182
183 // Key and attribute outputs to scrambling device
184 logic [otp_ctrl_pkg::SramKeyWidth-1:0] key_d, key_q;
185 logic [otp_ctrl_pkg::SramNonceWidth-1:0] nonce_d, nonce_q;
186
187 // tie-off unused nonce bits

```

```

183 if (otp_ctrl_pkg::SramNonceWidth > LfsrWidth + NonceWidth)
184 begin : gen_nonce_tieoff
185 logic unused_nonce;
186 assign unused_nonce =
187 ^nonce_q[otp_ctrl_pkg::SramNonceWidth-1:LfsrWidth + NonceWidth];
188 end
189
190 // Alert Sender //
191 ///////////////////////////////////////////////////
192 logic alert_test;
193 assign alert_test = reg2hw.alert_test.q &
194 reg2hw.alert_test.qe;
195
196 assign hw2reg.status.bus_integ_error.d = 1'b1;
197 assign hw2reg.status.bus_integ_error.de = |bus_integ_error;
198
199 logic init_error;
200 assign hw2reg.status.init_error.d = 1'b1;
201 assign hw2reg.status.init_error.de = init_error;
202
203 logic readback_error;
204 assign hw2reg.status.readback_error.d = 1'b1;
205 assign hw2reg.status.readback_error.de = readback_error;
206
207 logic sram_alert;
208 assign hw2reg.status.sram_alert.d = 1'b1;
209 assign hw2reg.status.sram_alert.de = sram_alert;
210
211 logic alert_req;
212 assign alert_req = (|bus_integ_error) | init_error | read-
213 back_error | sram_alert;
214
215 prim_alert_sender #(
216 .AsyncOn(AlertAsyncOn[0]),
217 .SkewCycles(AlertSkewCycles),
218 .IsFatal(1)
219 ) u_prim_alert_sender_parity (
220 .clk_i,
221 .rst_ni,
222 .alert_test_i ( alert_test ),
223 .alert_req_i ( alert_req ),
224 .alert_ack_o ( ),
225 .alert_state_o ( ),
226 .alert_rx_i ( alert_rx_i[0] ),
227 .alert_tx_o ( alert_tx_o[0] )
228 );
229
230 // Escalation Triggers //
231 ///////////////////////////////////////////////////
232 lc_tx_t [1:0] escalate_en;
233 prim_lc_sync #(
234 .NumCopies (2)
235 ) u_prim_lc_sync (
236 .clk_i,
237 .rst_ni,
238 .lc_en_i (lc_escalate_en_i),
239 .lc_en_o (escalate_en)
240 );
241
242 // SEC_CM: KEY.GLOBAL_ESC
243 logic escalate;
244 assign escalate = lc_tx_test_true_loose(escalate_en[0]);
245 assign hw2reg.status.escalated.d = 1'b1;
246 assign hw2reg.status.escalated.de = escalate;
247
248 // SEC_CM: KEY.LOCAL_ESC
249 // Aggregate external and internal escalation sources.
250 // This is used in countermeasures further below (key reset
251 and transaction blocking).
252 logic local_esc, local_esc_reg;
253
254 // This signal only aggregates registered escalation signals
255 and is used for transaction
256 // blocking further below, which is on a timing-critical
257 path.
258 assign local_esc_reg = reg2hw.status.escalated.q |
259 reg2hw.status.init_error.q |
260 reg2hw.status.bus_integ_error.q |
261 reg2hw.status.sram_alert.q |
262 reg2hw.status.readback_error.q;
263
264 // This signal aggregates all escalation trigger signals,
265 including the ones that are generated
266 // in the same cycle such as init_error, sram alert, and
267 bus_integ_error. It is used for
268 // countermeasures that are not on the critical path (such
269 as clearing the scrambling keys).
270 assign local_esc = escalate |
271 init_error |
272 (|bus_integ_error) |
273 sram_alert |
274 readback_error |
275 local_esc_reg;
276
277 // Convert registered, local escalation sources to a multib-
278 it signal and combine this with
279
280 // the incoming escalation enable signal before feeding into

```

```

the TL-UL gate further below.
271 lc_tx_t lc_tlul_gate_en;
272 assign lc_tlul_gate_en = lc_tx_inv(lc_tx_or_hi(escal-
ate_en[1]),
273
lc_tx_bool_to_lc_tx(local_esc_reg));
274 //////////////////////////////////////////////////
275 // HW Initialization //
276 //////////////////////////////////////////////////
277
278 // A write to the init register reloads the LFSR seed, re-
sets the init counter and
279 // sets init_q to flag a pending initialization request.
280 logic init_trig, init_q;
281 assign init_trig = reg2hw.ctrl.init.q &&
282 reg2hw.ctrl.init.qe &&
283 init_q; // Ignore new requests while
memory init is already pending.
284
285 logic init_d, init_done;
286 assign init_d = (init_done) ? 1'b0 :
287 (init_trig) ? 1'b1 : init_q;
288
289 always_ff @(posedge clk_i or negedge rst_ni) begin : p_in-
it_reg
290 if(!rst_ni) begin
291 init_q <= 1'b0;
292 end else begin
293 init_q <= init_d;
294 end
295 end
296
297 // This waits until the scrambling keys are actually valid
(this allows the SW to trigger
298 // key renewal and initialization at the same time).
299 logic init_req;
300 logic [AddrWidth-1:0] init_cnt;
301 logic key_req_pending_d, key_req_pending_q;
302 assign init_req = init_q & ~key_req_pending_q;
303 assign init_done = (init_cnt == AddrWidth'(Depth - 1)) &
init_req;
304
305 // We employ two redundant counters to guard against FI at-
tacks.
306 // If any of the two is glitched and the two counter states
do not agree,
307 // we trigger an alert.
308 // SEC_CM: INIT.CTR.REDUN
309 prim_count #(
310 .Width(AddrWidth)
311 ) u_prim_count (
312 .clk_i,
313 .rst_ni,
314 .clr_i(init_trig),
315 .set_i(1'b0),
316 .set_cnt_i('0),
317 .incr_en_i(init_req),
318 .decr_en_i(1'b0),
319 .step_i(AddrWidth'(1)),
320 .commit_i(1'b1),
321 .cnt_o(init_cnt),
322 .cnt_after_commit_o(),
323 .err_o(init_error)
324 );
325
326 // Clear this bit on local escalation.
327 assign hw2reg.status.init_done.d = init_done & ~init_trig &
~local_esc;
328 assign hw2reg.status.init_done.de = init_done | init_trig |
local_esc;
329
330 //////////////////////////////////////////////////
331 // Scrambling Key Request //
332 //////////////////////////////////////////////////
333
334 // The scrambling key and nonce have to be requested from
the OTP controller via a req/ack
335 // protocol. Since the OTP controller works in a different
clock domain, we have to synchronize
336 // the req/ack protocol as described in more details in the
OTP controller documentation.
337 //
338 // This is specialised for different tops that use it but
the req/ack protocol is the same in each
339 // case. For one example, see
340 //
341 // https://opentitan.org/book/hw/top_earlgrey/
342 // ip_autogen/otp_ctrl/doc/interfaces.html#interfaces-to-
sram-and-otbn-scramblers
343 logic key_req, key_ack;
344 assign key_req = reg2hw.ctrl.renew_scr_key.q &&
345 reg2hw.ctrl.renew_scr_key.qe &&
346 !key_req_pending_q && // Ignore new re-
quests while a request is already pending.
347 !init_q; // Ignore new requests while
memory init is already pending.
348
349 assign key_req_pending_d = (key_req) ? 1'b1 :
350 (key_ack) ? 1'b0 : key_req_pend-
ing_q;

```

```

351
352 // Clear this bit on local escalation.
353 assign hw2reg.status.scr_key_valid.d = key_ack & ~key_req
& ~local_esc;
354
355 assign hw2reg.status.scr_key_valid.de = key_req | key_ack |
local_esc;
356
357 // As opposed to scr_key_valid, SW is responsible for clear-
ing this register.
358 // It is not automatically cleared by HW, except when escal-
ating.
359 assign hw2reg.scr_key_rotated.d = (key_ack & ~local_esc) ?
MuBi4True : MuBi4False;
360 assign hw2reg.scr_key_rotated.de = key_ack | local_esc;
361
362 // Clear this bit on local escalation.
363 logic key_seed_valid;
364 assign hw2reg.status.scr_key_seed_valid.d = key_seed_valid
& ~local_esc;
365 assign hw2reg.status.scr_key_seed_valid.de = key_ack | loc-
al_esc;
366
367 always_ff @(posedge clk_i or negedge rst_ni) begin : p_regs
368 if (!rst_ni) begin
369 key_req_pending_q <= 1'b0;
370 // reset case does not use buffered values as the
// reset value will be directly encoded into flop types
371 key_q <= RndCnstSramKey;
372 nonce_q <= RndCnstSramNonce;
373 end else begin
374 key_req_pending_q <= key_req_pending_d;
375 if (key_ack) begin
376 key_q <= key_d;
377 nonce_q <= nonce_d;
378 end
379 // This scraps the keys.
380 // SEC_CM: KEY.GLOBAL_ESC
381 // SEC_CM: KEY.LOCAL_ESC
382 if (local_esc) begin
383 key_q <= cnst_sram_key;
384 nonce_q <= cnst_sram_nonce;
385 end
386 end
387 end
388
389 prim_sync_reqack_data #(
390 .Width($bits(otp_ctrl_pkg::sram_otp_key_rsp_t)-1),
391 .DataSrc2Dst(1'b0)
392 ) u_prim_sync_reqack_data (
393 .clk_src_i ( clk_i ),
394 .rst_src_ni ( rst_ni ),
395 .clk_dst_i ( clk_otp_i ),
396 .rst_dst_ni ( rst_otp_ni ),
397 .req_chk_i ( 1'b1 ),
398 .src_req_i ( key_req_pending_q ),
399 .src_ack_o ( key_ack ),
400 .dst_req_o ( sram_otp_key_o.req ),
401 .dst_ack_i ( sram_otp_key_i.ack ),
402 .data_i ( {sram_otp_key_i.key,
sram_otp_key_i.nonce,
sram_otp_key_i.seed_valid} ),
403
404 .data_o ( {key_d,
nonce_d,
key_seed_valid} )
405 );
406
407 logic unused_csr_sigs;
408 assign unused_csr_sigs = ^{reg2hw.status.init_done.q,
reg2hw.status.scr_key_seed_valid.q};
409
410 //////////////////////////////////////////////////
411 // SRAM Execution //
412 //////////////////////////////////////////////////
413
414 mubi4_t en_ifetch;
415 if (InstrExec) begin : gen_instr_ctrl
416 lc_tx_t lc_hw_debug_en;
417 prim_lc_sync #(
418 .NumCopies (1)
419 ) u_prim_lc_sync_hw_debug_en (
420 .clk_i,
421 .rst_ni,
422 .lc_en_i (lc_hw_debug_en_i),
423 .lc_en_o ({lc_hw_debug_en})
424 );
425
426 mubi8_t otp_en_sram_ifetch;
427 prim_mubi8_sync #(
428 .NumCopies (1)
429 ) u_prim_mubi8_sync_otp_en_sram_ifetch (
430 .clk_i,
431 .rst_ni,
432 .mubi_i(otp_en_sram_ifetch_i),
433 .mubi_o({otp_en_sram_ifetch})
434 );
435
436 mubi4_t lc_ifetch_en;
437 mubi4_t reg_ifetch_en;
438 // SEC_CM: INSTR.BUS.LC_GATED

```

```

443 assign lc_ifetch_en = lc_to_mubi4(lc_hw_debug_en);
444 // SEC_CM: EXEC.CONFIG.MUBI
445 assign reg_ifetch_en = mubi4_t'(reg2hw.exec.q);
446 // SEC_CM: EXEC.INTERSIG.MUBI
447 assign en_ifetch = (mu-
bi8_test_true_strict(otp_en_sram_ifetch)) ? reg_ifetch_en :
448 lc_ifetch_en;
449 end else begin : gen_tieoff
450 assign en_ifetch = MUBI4False;
451
452 // tie off unused signals
453 logic unused_sigs;
454 assign unused_sigs = ^{lc_hw_debug_en_i,
455 reg2hw.exec.q,
456 otp_en_sram_ifetch_i};
457 end
458
459 ///////////////////////////////////////////////////
460 // Initialization LFSR //
461 ///////////////////////////////////////////////////
462
463 logic [LfsrOutWidth-1:0] lfsr_out;
464 prim_lfsr #(
465 .LfsrDw ( LfsrWidth ),
466 .EntropyDw ( LfsrWidth ),
467 .StateOutDw ( LfsrOutWidth ),
468 .DefaultSeed ( RndCnstLfsrSeed ),
469 .StatePermEn ( 1'b1 ),
470 .StatePerm ( RndCnstLfsrPerm )
471 ) u_lfsr (
472 .clk_i,
473 .rst_ni,
474 .lfsr_en_i(init_req),
475 .seed_en_i(init_trig),
476 .seed_i(nonce_q[NonceWidth+: LfsrWidth]),
477 .entropy_i('0),
478 .state_o(lfsr_out)
479 );
480
481 // Compute the correct integrity alongside for the pseudo-
random initialization values.
482 logic [DataWidth - 1 :0] lfsr_out_integ;
483 tlul_data_integ_enc u_tlul_data_integ_enc (
484 .data_i(lfsr_out),
485 .data_intg_o(lfsr_out_integ)
486 );
487
488 ///////////////////////////////////////////////////
489 // SRAM TL-UL Access Gate //
490 ///////////////////////////////////////////////////
491
492 logic tl_gate_resp_pending;
493 tlul_pkg::tl_h2d_t ram_tl_in_gated;
494 tlul_pkg::tl_d2h_t ram_tl_out_gated;
495
496 // SEC_CM: RAM_TL_LC_GATE.FSM.SPARSE
497 tlul_lc_gate #(
498 .NumGatesPerDirection(2),
499 .Outstanding(Outstanding)
500 ) u_tlul_lc_gate (
501 .clk_i,
502 .rst_ni,
503 .tl_h2d_i(ram_tl_i),
504 .tl_d2h_o(ram_tl_o),
505 .tl_h2d_o(ram_tl_in_gated),
506 .tl_d2h_i(ram_tl_out_gated),
507 .flush_req_i('0),
508 .flush_ack_o(),
509 .resp_pending_o(tl_gate_resp_pending),
510 .lc_en_i (lc_tlul_gate_en),
511 .err_o (bus_integ_error[2])
512 );
513
514 ///////////////////////////////////////////////////
515 // SRAM with scrambling device //
516 ///////////////////////////////////////////////////
517
518 logic tlul_req, tlul_gnt, tlul_we;
519 logic [AddrWidth-1:0] tlul_addr;
520 logic [DataWidth-1:0] tlul_wdata, tlul_wmask;
521
522 logic sram_intg_error, sram_req, sram_gnt, sram_we, sram_r-
valid, sram_rvalid_scr;
523 logic [1:0] sram_error, sram_error_scr;
524 logic [AddrWidth-1:0] sram_addr, sram_error_addr_scr;
525 logic [DataWidth-1:0] sram_wdata, sram_wmask, sram_rdata,
sram_rdata_scr;
526 logic sram_wpending, sram_wr_collision;
527
528 logic sram_compound_txn_in_progress;
529
530 // // SEC_CM: MEM.READBACK
531 mubi4_t reg_readback_en;
532 assign reg_readback_en = mubi4_t'(reg2hw.readback.q);
533
534 tlul_adapter_sram_racl #(
535 .SramAw(AddrWidth),
536 .SramDw(DataWidth - tlul_pkg::DataIntgWidth),

```

```

538 .SramDepth(Depth),
539 .Outstanding(Outstanding),
540 .ByteAccess(1),
541 .CmdIntgCheck(1),
542 .EnableRspIntgGen(1),
543 .EnableDataIntgGen(0),
544 .EnableDataIntgPt(1), // SEC_CM: MEM.INTEGRITY
545 .SecFifoPtr (1), // SEC_CM: TLUL_FIFO.CTR.REDUN
546 .EnableReadback (1), // SEC_CM: MEM.READBACK
547 .EnableRacl(EnableRacl),
548 .RaclErrorRsp(RaclErrorRsp),
549 .RaclPolicySelNumRanges(RaclPolicySelNumRangesRam)
550 ) u_tlul_adapter_sram_racl (
551 .clk_i,
552 .rst_ni,
553 .tl_i (ram_tl_in_gated),
554 .tl_o (ram_tl_out_gated),
555 .en_ifetch_i (en_ifetch),
556 .req_o (tlul_req),
557 .req_type_o (),
558 .gnt_i (tlul_gnt),
559 .we_o (tlul_we),
560 .addr_o (tlul_addr),
561 .wdata_o (tlul_wdata),
562 .wmask_o (tlul_wmask),
563 // SEC_CM: BUS.INTEGRITY
564 .intg_error_o (bus_integ_error[1]),
565 .user_rsvd_o (),
566 .rdata_i (sram_rdata),
567 .rvalid_i (sram_rvalid),
568 .error_i (sram_error),
569 .compound_txn_in_progress_o (sram_com-
pound_txn_in_progress),
570 .readback_en_i (reg_readback_en),
571 .readback_error_o (readback_error),
572 .wr_collision_i (sram_wr_collision),
573 .write_pending_i (sram_wpending),
574 // RACL interface
575 .racl_policies_i (racl_policies_i),
576 .racl_error_o (racl_error[1]),
577 .racl_policy_sel_ranges_i (racl_policy_sel_ranges_ram_i)
578 );
579
580 logic key_valid;
581
582 // Interposing mux logic for initialization with pseudo ran-
dom data.
583 assign sram_req = tlul_req | init_req;
584 // This grant signal acts more like a ready internally in
tlul_adapter_sram. In particular it's
585 // fine to assert it when tlul_gnt is low (it has no ef-
fect). So here tlul_gnt is asserted when
586 // a request from tlul_req will be granted regardless of
whether a request exists. This is done
587 // for timing reasons so that the output TL ready isn't com-
binatorially connected to the incoming
588 // TL valid. In particular we must not use `sram_gnt` in the
expression to avoid this.
589 assign tlul_gnt = key_valid & ~init_req;
590 assign sram_we = tlul_we | init_req;
591 assign sram_intg_error = |bus_integ_error[2:1] & ~init_req;
592 assign sram_addr = (init_req) ? init_cnt :
tlul_addr;
593 assign sram_wdata = (init_req) ? lfsr_out_integ :
tlul_wdata;
594 assign sram_wmask = (init_req) ? {DataWidth{1'b1}} :
tlul_wmask;
595
596 if (EccCorrection) begin : gen_ecc_correction
597 // Detect ECC errors and decode data. If data is correct-
able, data is genuine and we
598 // can use it further and re-encode
599 logic [31:0] dec_data;
600 logic [1:0] ecc_error;
601 prim_secdec_inv_39_32_dec u_dec (
602 .data_i (sram_rdata_scr),
603 .data_o (dec_data),
604 .syndrome_o (),
605 .err_o (ecc_error)
606 );
607
608 logic [DataWidth-1:0] ecc_enc_data;
609 prim_secdec_inv_39_32_enc u_enc (
610 .data_i(dec_data),
611 .data_o(ecc_enc_data)
612 );
613
614 logic uncorrectable_error_q;
615 prim_flop #(
616 .width(1)
617 ) u_flop_uncorr_error (
618 .clk_i,
619 .rst_ni,
620 .d_i(ecc_error[1]),
621 .q_o(uncorrectable_error_q)
622 );
623
624 // Correctable errors are corrected.
625 // Uncorrectable errors are passed through to the re-
quester.

```

```

626 assign sram_error[0] = 1'b0;
627 assign sram_error[1] = uncorrectable_error_q;
628
629 // Error log if any error happened
630 assign sram_error_o.valid = sram_rvalid_scr & |
ecc_error;
631 assign sram_error_o.correctable = sram_rvalid_scr &
~ecc_error[1];
632
633 // Translate word address to byte address and fill remain-
ing bits with 0
634 always_comb begin
635     sram_error_o.address = '0;
636     sram_error_o.address[2+:AddrWidth] = sram_rer-
ror_addr_scr;
637 end
638
639 prim_flop #(
640     .Width(1)
641 ) u_flop_rvalid (
642     .clk_i,
643     .rst_ni,
644     .d_i(sram_rvalid_scr),
645     .q_o(sram_rvalid)
646 );
647
648 prim_flop #(
649     .Width(DataWidth)
650 ) u_flop_enc_data (
651     .clk_i,
652     .rst_ni,
653     .d_i(ecc_enc_data),
654     .q_o(sram_rdata)
655 );
656
657 // We don't use the read error from the prim, we re-com-
pute it here
658 logic unused_error;
659 assign unused_error = ^sram_error_scr;
660 end else begin : gen_no_ecc_correction
661     assign sram_rdata = sram_rdata_scr;
662     assign sram_rerror = sram_rerror_scr;
663     assign sram_rvalid = sram_rvalid_scr;
664     // ECC errors are not detected (and thus not reported
either) in this configuration.
665     assign sram_error_o = '0;
666
667     // Error address not used here
668     logic unused_error_addr;
669     assign unused_error_addr = ^sram_error_addr_scr;
670 end
671
672 // The SRAM scrambling wrapper will not accept any transac-
tions while the
673 // key req is pending or if we have escalated. Note that
we're not using
674 // the scr_key_valid CSR here, such that the SRAM can be
used right after
675 // reset, where the keys are reset to the default netlist
constant.
676 //
677 // If we have escalated, but there is a pending request in
the TL gate, we may have a pending
678 // read-modify-write transaction or readback in the SRAM ad-
apter. In that case we force key_valid
679 // high to enable that to complete so it returns a response,
the TL gate won't accept any new
680 // transactions and the SRAM keys have been clobbered
already.
681 assign key_valid =
682     (key_req_pending_q) ? 1'b0 :
683     (reg2hw.status.escalated.q) ? (tl_gate_resp_pending &
sram_compound_txn_in_progress) : 1'b1;
684
685 // SEC_CM: MEM.SCRAMBLE, ADDR.SCRAMBLE, PRIM_RAM.CTRL.MUBI
686 prim_ram_1p_scr #(
687     .Width(DataWidth),
688     .Depth(Depth),
689     .InstDepth(InstDepth),
690     .EnableParity(0),
691     .DataBitsPerMask(DataWidth),
692     .NumPrinceRoundsHalf(NumPrinceRoundsHalf),
693     .NumAddrScrRounds(NumAddrScrRounds)
694 ) u_prim_ram_1p_scr (
695     .clk_i,
696     .rst_ni,
697
698     .key_valid_i (key_valid),
699     .key_i (key_q),
700     .nonce_i (nonce_q[NonceWidth-1:0]),
701
702     .req_i (sram_req),
703     .intg_error_i (sram_intg_error),
704     .gnt_o (sram_gnt),
705     .write_i (sram_we),
706     .addr_i (sram_addr),
707     .wdata_i (sram_wdata),
708     .wmask_i (sram_wmask),
709     .rdata_o (sram_rdata_scr),
710     .rvalid_o (sram_rvalid_scr),

```

```

711     .error_o (sram_error_scr),
712     .raddr_o (sram_error_addr_scr),
713     .cfg_i (ram_cfg_i),
714     .cfg_o (ram_cfg_o),
715     .wr_collision_o (sram_wr_collision),
716     .write_pending_o (sram_wpending),
717     .alert_o (sram_alert)
718 );
719
720 logic unused_sram_gnt;
721 // Ignore sram_gnt signal to avoid creating a bad timing
path, see comment on 'tlul_gnt' above for
722 // more details.
723 assign unused_sram_gnt = sram_gnt;
724
725 ///////////////////////////////////////////////////////////////////
726 // Assertions //
727 ///////////////////////////////////////////////////////////////////
728
729 `ASSERT_KNOWN(RegsTlOutKnown_A, regs_tl_o)
730 `ASSERT_KNOWN(RamTlOutKnown_A, ram_tl_o.d_valid)
731 `ASSERT_KNOWN_IF(RamTlOutPayloadKnown_A, ram_tl_o,
ram_tl_o.d_valid)
732 `ASSERT_KNOWN(AlertOutKnown_A, alert_tx_o)
733 `ASSERT_KNOWN(SramOtpKeyKnown_A, sram_otp_key_o)
734 `ASSERT_KNOWN(RaclErrorValidKnown_A, racl_error_o.valid)
735 `ASSERT_KNOWN(SramRerrorKnown_A, sram_rerror_o)
736
737 // Alert assertions for redundant counters.
738 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntCheck_A,
u_prim_count, alert_tx_o[0])
739
740 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(LcGateFsmCheck_A,
u_tlul_lc_gate.u_state_regs, alert_tx_o[0])
741
742 // Alert assertions for reg_we onehot check.
743 `AS-
744 SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
u_reg_regs, alert_tx_o[0])
745
746 // Alert assertions for redundant counters.
747 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(RspFifoWptrCheck_A,
748 u_tlul_adapter_sram_racl.tlul_adapter_sram.u_rspfifo.gen_normal_fifo
.u_fifo_cnt
749     .gen_secure_ptrs.u_wptr,
750     alert_tx_o[0])
751 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(RspFifoRptrCheck_A,
752 u_tlul_adapter_sram_racl.tlul_adapter_sram.u_rspfifo.gen_normal_fifo
.u_fifo_cnt
753     .gen_secure_ptrs.u_rptr,
754     alert_tx_o[0])
755 `AS-
756 SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(SramReqFifoWptrCheck_A,
757 u_tlul_adapter_sram_racl.tlul_adapter_sram.u_sramreqfifo.gen_normal
fifo.u_fifo_cnt
758     .gen_secure_ptrs.u_wptr,
759     alert_tx_o[0])
760 `AS-
761 SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(SramReqFifoRptrCheck_A,
762 u_tlul_adapter_sram_racl.tlul_adapter_sram.u_sramreqfifo.gen_normal
fifo.u_fifo_cnt
763     .gen_secure_ptrs.u_rptr,
764     alert_tx_o[0])
765 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(ReqFifoWptrCheck_A,
766 u_tlul_adapter_sram_racl.tlul_adapter_sram.u_reqfifo.gen_normal_fifo
.u_fifo_cnt
767     .gen_secure_ptrs.u_wptr,
768     alert_tx_o[0])
769 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(ReqFifoRptrCheck_A,
770 u_tlul_adapter_sram_racl.tlul_adapter_sram.u_reqfifo.gen_normal_fifo
.u_fifo_cnt
771     .gen_secure_ptrs.u_rptr,
772     alert_tx_o[0])
773 // 'tlul_gnt' doesn't factor in 'sram_gnt' for timing reas-
ons. This assertions checks that
774 // 'tlul_gnt' is the same as 'sram_gnt' when there's an act-
ive 'tlul_req' that isn't being ignored
775 // because the SRAM is initializing.
776 `ASSERT(TlulGntIsCorrect_A, tlul_req |-> (sram_gnt &
~init_req) == tlul_gnt)
777
778 `ifdef FI_SIM_Z01X
779 // Check if there are any TL-UL integrity errors caused by
faults that Z01X has introduced.
780 // Specific to fault injection simulation as Z01X expects
that those strobing points are
781 // available in the design.
782 wire ram_tl_intg_err;
783 tlul_rsp_intg_chk #(
784     .EnableRspDataIntgCheck(1)
785 ) u_rsp_chk_ram (
786     .tl_i (ram_tl_o),
787     .err_o (ram_tl_intg_err)
788 );

```



```

789
790 wire regs_tl_intg_err;
791 tlul_rsp_intg_chk #(
792   .EnableRspDataIntgCheck(1)
793 ) u_rsp_chk_regs (
794   .tl_i (regs_tl_o),
795   .err_o(regs_tl_intg_err)
796 );
797 `endif
798
799 endmodule : sram_ctrl

```

2. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_pkg.sv (50 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5
6 package sram_ctrl_pkg;
7
8 // Parameters //
9 // Parameters //
10 // Parameters //
11
12 // The width of this RAM is currently restricted to 39
  (32bit data + 7bit integrity).
13 parameter int DataWidth = 32 + tlul_pkg::DataIntgWidth;
14 parameter int NonceWidth = 64;
15
16 // RndCnst //
17 // RndCnst //
18 // RndCnst //
19
20 parameter otp_ctrl_pkg::sram_key_t RndCnstSramKeyDefault =
  128'hbecda03b34bc0418a30a33861a610f71;
21
22 parameter otp_ctrl_pkg::sram_nonce_t RndCnstSramNonceDefault =
  128'h22f296f8f95efb84a75cd435a5541e9f;
23
24 // These LFSR parameters have been generated with
25 // $ ./util/design/gen-lfsr-seed.py --width 64 --seed
  3296833456 --prefix ""
26
27 parameter int LfsrWidth = 64;
28 typedef logic [LfsrWidth-1:0] lfsr_seed_t;
29 typedef logic [LfsrWidth-1:0][$clog2(LfsrWidth)-1:0]
  lfsr_perm_t;
30 parameter lfsr_seed_t RndCnstLfsrSeedDefault =
  64'hb496209a_10a81ea5;
31 parameter lfsr_perm_t RndCnstLfsrPermDefault = {
32   128'hf7963515_f8af8e60_fbfec4c0_f1edd9e2,
33
34   256'h41e1c6d4_273d5046_2da7165d_1c1db882_693146c2_a33aa048_43762bed_
    0ecabea5
35 };
36 // The LFSR has an internal state width of 64 bits but we
  just use the lowest 32 bits of the
37 // permuted state for initializing the RAM.
38 parameter int LfsrOutWidth = 32;
39
40 // Type definitions //
41 // Type definitions //
42 // Type definitions //
43
44 typedef struct packed {
45   logic valid;
46   logic correctable;
47   logic [top_pkg::TL_AW-1:0] address;
48 } sram_error_t;
49
50 endpackage : sram_ctrl_pkg

```

3. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_ram_reg_top.sv (54 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"
8
9 module sram_ctrl_ram_reg_top
10 # (
11   parameter bit EnableRacl = 1'b0,
12   parameter bit RclErrorRsp = 1'b1,
13   parameter top_racl_pkg::racl_policy_sel_t RaclPoli-
  cySelVec[sram_ctrl_reg_pkg::NumRegsRam] =
14   '{sram_ctrl_reg_pkg::NumRegsRam{0}}
15 ) (
16   input clk_i,
17   input rst_ni,

```

```

18   input tlul_pkg::tl_h2d_t tl_i,
19   output tlul_pkg::tl_d2h_t tl_o,
20   // To HW
21
22   // RACL interface
23   input top_racl_pkg::racl_policy_vec_t racl_policies_i,
24   output top_racl_pkg::racl_error_log_t racl_error_o,
25
26   // Integrity check errors
27   output logic intg_err_o
28 );
29
30 import sram_ctrl_reg_pkg::* ;
31
32
33 // Since there are no registers in this block, commands are
  routed through to windows which
34 // can report their own integrity errors.
35 assign intg_err_o = 1'b0;
36
37 // outgoing integrity generation
38 tlul_pkg::tl_d2h_t tl_o_pre;
39 tlul_rsp_intg_gen #(
40   .EnableRspIntgGen(1),
41   .EnableDataIntgGen(1)
42 ) u_rsp_intg_gen (
43   .tl_i(tl_o_pre),
44   .tl_o(tl_o)
45 );
46
47 assign tl_reg_h2d = tl_i;
48 assign tl_o_pre = tl_reg_d2h;
49
50 // Unused signal tieoff
51 logic unused_policy_sel;
52 assign unused_policy_sel = ^racl_policies_i;
53
54 endmodule

```

4. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_reg_pkg.sv (171 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Package auto-generated by `reggen` containing data
  structure
6
7 package sram_ctrl_reg_pkg;
8
9 // Param list
10 parameter int NumAlerts = 1;
11
12 // Address widths within the block
13 parameter int RegsAw = 6;
14
15 // Number of registers for every interface
16 parameter int NumRegsRegs = 9;
17
18 // Alert indices
19 typedef enum int {
20   AlertFatalErrorIdx = 0
21 } sram_ctrl_alert_idx_t;
22
23 // Typedefs for registers for regs interface //
24 // Typedefs for registers for regs interface //
25 // Typedefs for registers for regs interface //
26
27 typedef struct packed {
28   logic q;
29   logic qe;
30 } sram_ctrl_reg2hw_alert_test_reg_t;
31
32 typedef struct packed {
33   struct packed {
34     logic q;
35   } sram_alert;
36   struct packed {
37     logic q;
38   } readback_error;
39   struct packed {
40     logic q;
41   } init_done;
42   struct packed {
43     logic q;
44   } scr_key_seed_valid;
45   struct packed {
46     logic q;
47   } escalated;
48   struct packed {
49     logic q;
50   } init_error;
51   struct packed {
52     logic q;
53   } bus_integ_error;
54 } sram_ctrl_reg2hw_status_reg_t;

```

```

55
56 typedef struct packed {
57     logic [3:0] q;
58 } sram_ctrl_reg2hw_exec_reg_t;
59
60 typedef struct packed {
61     struct packed {
62         logic q;
63         logic qe;
64     } init;
65     struct packed {
66         logic q;
67         logic qe;
68     } renew_scr_key;
69 } sram_ctrl_reg2hw_ctrl_reg_t;
70
71 typedef struct packed {
72     logic [3:0] q;
73 } sram_ctrl_reg2hw_readback_reg_t;
74
75 typedef struct packed {
76     struct packed {
77         logic d;
78         logic de;
79     } sram_alert;
80     struct packed {
81         logic d;
82         logic de;
83     } readback_error;
84     struct packed {
85         logic d;
86         logic de;
87     } init_done;
88     struct packed {
89         logic d;
90         logic de;
91     } scr_key_seed_valid;
92     struct packed {
93         logic d;
94         logic de;
95     } scr_key_valid;
96     struct packed {
97         logic d;
98         logic de;
99     } escalated;
100    struct packed {
101        logic d;
102        logic de;
103    } init_error;
104    struct packed {
105        logic d;
106        logic de;
107    } bus_integ_error;
108 } sram_ctrl_hw2reg_status_reg_t;
109
110 typedef struct packed {
111     logic [3:0] d;
112     logic de;
113 } sram_ctrl_hw2reg_scr_key_rotated_reg_t;
114
115 // Register -> HW type for regs interface
116 typedef struct packed {
117     sram_ctrl_reg2hw_alert_test_reg_t alert_test; // [20:19]
118     sram_ctrl_reg2hw_status_reg_t status; // [18:12]
119     sram_ctrl_reg2hw_exec_reg_t exec; // [11:8]
120     sram_ctrl_reg2hw_ctrl_reg_t ctrl; // [7:4]
121     sram_ctrl_reg2hw_readback_reg_t readback; // [3:0]
122 } sram_ctrl_regs_reg2hw_t;
123
124 // HW -> register type for regs interface
125 typedef struct packed {
126     sram_ctrl_hw2reg_status_reg_t status; // [20:5]
127
128     sram_ctrl_hw2reg_scr_key_rotated_reg_t scr_key_rotated; // [4:0]
129 } sram_ctrl_regs_hw2reg_t;
130
131 // Register offsets for regs interface
132 parameter logic [RegsAw-1:0] SRAM_CTRL_ALERT_TEST_OFFSET = 6'h 0;
133 parameter logic [RegsAw-1:0] SRAM_CTRL_STATUS_OFFSET = 6'h 4;
134 parameter logic [RegsAw-1:0] SRAM_CTRL_EXEC_REGWEN_OFFSET = 6'h 8;
135 parameter logic [RegsAw-1:0] SRAM_CTRL_EXEC_OFFSET = 6'h c;
136 parameter logic [RegsAw-1:0] SRAM_CTRL_CTRL_REGWEN_OFFSET = 6'h 10;
137 parameter logic [RegsAw-1:0] SRAM_CTRL_CTRL_OFFSET = 6'h 14;
138 parameter logic [RegsAw-1:0] SRAM_CTRL_SCR_KEY_ROTATED_OFFSET = 6'h 18;
139 parameter logic [RegsAw-1:0] SRAM_CTRL_READBACK_REGWEN_OFFSET = 6'h 1c;
140
141 parameter logic [RegsAw-1:0] SRAM_CTRL_READBACK_OFFSET = 6'h 20;
142
143 // Reset values for hwext registers and their fields for
144 regs interface
145 parameter logic [0:0] SRAM_CTRL_ALERT_TEST_RESVAL = 1'h 0;
146 parameter logic [0:0] SRAM_CTRL-
147     TRL_ALERT_TEST_FATAL_ERROR_RESVAL = 1'h 0;

```

```

144
145 // Register index for regs interface
146 typedef enum int {
147     SRAM_CTRL_ALERT_TEST,
148     SRAM_CTRL_STATUS,
149     SRAM_CTRL_EXEC_REGWEN,
150     SRAM_CTRL_EXEC,
151     SRAM_CTRL_CTRL_REGWEN,
152     SRAM_CTRL_CTRL,
153     SRAM_CTRL_SCR_KEY_ROTATED,
154     SRAM_CTRL_READBACK_REGWEN,
155     SRAM_CTRL_READBACK,
156 } sram_ctrl_regs_id_e;
157
158 // Register width information to check illegal writes for
159 regs interface
160 parameter logic [3:0] SRAM_CTRL_REGS_PERMIT [9] = '{
161     4'b 0001, // index[0] SRAM_CTRL_ALERT_TEST
162     4'b 0001, // index[1] SRAM_CTRL_STATUS
163     4'b 0001, // index[2] SRAM_CTRL_EXEC_REGWEN
164     4'b 0001, // index[3] SRAM_CTRL_EXEC
165     4'b 0001, // index[4] SRAM_CTRL_CTRL_REGWEN
166     4'b 0001, // index[5] SRAM_CTRL_CTRL
167     4'b 0001, // index[6] SRAM_CTRL_SCR_KEY_ROTATED
168     4'b 0001, // index[7] SRAM_CTRL_READBACK_REGWEN
169     4'b 0001 // index[8] SRAM_CTRL_READBACK
170 };
171 endpackage

```

5. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_regs_reg_top.sv (859 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPD-License-Identifier: Apache-2.0
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"
8
9 module sram_ctrl_regs_reg_top
10 # (
11     parameter bit EnableRacl = 1'b0,
12     parameter bit RaclErrorRsp = 1'b1,
13     parameter top_racl_pkg::racl_policy_sel_t RaclPolicySelVec[sram_ctrl_reg_pkg::NumRegsRegs] =
14         '{sram_ctrl_reg_pkg::NumRegsRegs{0}}
15 ) (
16     input clk_i,
17     input rst_ni,
18     input tlul_pkg::tl_h2d_t tl_i,
19     output tlul_pkg::tl_d2h_t tl_o,
20     // To HW
21
22     output sram_ctrl_reg_pkg::sram_ctrl_regs_reg2hw_t reg2hw, // Write
23
24     input sram_ctrl_reg_pkg::sram_ctrl_regs_hw2reg_t hw2reg, // Read
25
26     // RACL interface
27     input top_racl_pkg::racl_policy_vec_t racl_policies_i,
28     output top_racl_pkg::racl_error_log_t racl_error_o,
29
30     // Integrity check errors
31     output logic intg_err_o
32 );
33
34 import sram_ctrl_reg_pkg::* ;
35
36 localparam int AW = 6;
37 localparam int DW = 32;
38 localparam int DBW = DW/8; // Byte Width
39
40 // register signals
41 logic reg_we;
42 logic reg_re;
43 logic [AW-1:0] reg_addr;
44 logic [DW-1:0] reg_wdata;
45 logic [DBW-1:0] reg_be;
46 logic [DW-1:0] reg_rdata;
47 logic reg_error;
48
49 logic addrmiss, wr_err;
50
51 logic [DW-1:0] reg_rdata_next;
52 logic reg_busy;
53
54 tlul_pkg::tl_h2d_t tl_reg_h2d;
55 tlul_pkg::tl_d2h_t tl_reg_d2h;
56
57 // incoming payload check
58 logic intg_err;
59 tlul_cmd_intg_chk u_chk (
60     .tl_i(tl_i),
61     .err_o(intg_err)

```

```

61 );
62
63 // also check for spurious write enables
64 logic reg_we_err;
65 logic [8:0] reg_we_check;
66 prim_reg_we_check #(
67     .OneHotWidth(9)
68 ) u_prim_reg_we_check (
69     .clk_i(clk_i),
70     .rst_ni(rst_ni),
71     .oh_i (reg_we_check),
72     .en_i (reg_we && !addrmiss),
73     .err_o (reg_we_err)
74 );
75
76 logic err_q;
77 always_ff @(posedge clk_i or negedge rst_ni) begin
78     if (!rst_ni) begin
79         err_q <= '0;
80     end else if (intg_err || reg_we_err) begin
81         err_q <= 1'b1;
82     end
83 end
84
85 // integrity error output is permanent and should be used
for alert generation
86 // register errors are transactional
87 assign intg_err_o = err_q | intg_err | reg_we_err;
88
89 // outgoing integrity generation
90 tlul_pkg::tl_d2h_t tl_o_pre;
91 tlul_rsp_intg_gen #(
92     .EnableRspIntgGen(1),
93     .EnableDataIntgGen(1)
94 ) u_rsp_intg_gen (
95     .tl_i(tl_o_pre),
96     .tl_o(tl_o)
97 );
98
99 assign tl_reg_h2d = tl_i;
100 assign tl_o_pre = tl_reg_d2h;
101
102 tlul_adapter_reg #(
103     .RegAw(AW),
104     .RegDw(DW),
105     .EnableDataIntgGen(0)
106 ) u_reg_if (
107     .clk_i (clk_i),
108     .rst_ni (rst_ni),
109
110     .tl_i (tl_reg_h2d),
111     .tl_o (tl_reg_d2h),
112
113     .en_ifetch_i(prim_mubi_pkg::MuBi4False),
114     .intg_error_o(),
115
116     .we_o (reg_we),
117     .re_o (reg_re),
118     .addr_o (reg_addr),
119     .wdata_o (reg_wdata),
120     .be_o (reg_be),
121     .busy_i (reg_busy),
122     .rdata_i (reg_rdata),
123     // Translate RACL error to TLUL error if enabled
124     .error_i (reg_error | (RaclErrorRsp & racl_error_o.valid))
125 );
126
127 // cdc oversampling signals
128
129 assign reg_rdata = reg_rdata_next ;
130 assign reg_error = addrmiss | wr_err | intg_err;
131
132 // Define SW related signals
133 // Format: <reg>_<field>_{wd|we|qs}
134 // or <reg>_{wd|we|qs} if field == 1 or 0
135 logic alert_test_we;
136 logic alert_test_wd;
137 logic status_bus_integ_error_qs;
138 logic status_init_error_qs;
139 logic status_escalated_qs;
140 logic status_scr_key_valid_qs;
141 logic status_scr_key_seed_valid_qs;
142 logic status_init_done_qs;
143 logic status_readback_error_qs;
144 logic status_sram_alert_qs;
145 logic exec_regwen_we;
146 logic exec_regwen_qs;
147 logic exec_regwen_wd;
148 logic exec_we;
149 logic [3:0] exec_qs;
150 logic [3:0] exec_wd;
151 logic ctrl_regwen_we;
152 logic ctrl_regwen_qs;
153 logic ctrl_regwen_wd;
154 logic ctrl_we;
155 logic ctrl_renew_scr_key_wd;
156 logic ctrl_init_wd;
157 logic scr_key_rotated_we;
158 logic [3:0] scr_key_rotated_qs;
159 logic [3:0] scr_key_rotated_wd;
160 logic readback_regwen_we;

161 logic readback_regwen_qs;
162 logic readback_regwen_wd;
163 logic readback_we;
164 logic [3:0] readback_qs;
165 logic [3:0] readback_wd;
166
167 // Register instances
168 // R[alert_test]: V(True)
169 logic alert_test_qe;
170 logic [0:0] alert_test_flds_we;
171 assign alert_test_qe = &alert_test_flds_we;
172 prim_subreg_ext #(
173     .DW (1)
174 ) u_alert_test (
175     .re (1'b0),
176     .we (alert_test_we),
177     .wd (alert_test_wd),
178     .d ('0),
179     .qre (),
180     .qe (alert_test_flds_we[0]),
181     .q (reg2hw.alert_test.q),
182     .ds (),
183     .qs ()
184 );
185 assign reg2hw.alert_test.qe = alert_test_qe;
186
187 // R[status]: V(False)
188 // F[bus_integ_error]: 0:0
189 prim_subreg #(
190     .DW (1),
191     .SwAccess(prim_subreg_pkg::SwAccessR0),
192     .RESVAL (1'h0),
193     .Mubi (1'b0)
194 ) u_status_bus_integ_error (
195     .clk_i (clk_i),
196     .rst_ni (rst_ni),
197
198     // from register interface
199     .we (1'b0),
200     .wd ('0),
201
202     // from internal hardware
203     .de (hw2reg.status.bus_integ_error.de),
204     .d (hw2reg.status.bus_integ_error.d),
205
206     // to internal hardware
207     .qe (),
208     .q (reg2hw.status.bus_integ_error.q),
209     .ds (),
210
211     // to register interface (read)
212     .qs (status_bus_integ_error_qs)
213 );
214
215 // F[init_error]: 1:1
216 prim_subreg #(
217     .DW (1),
218     .SwAccess(prim_subreg_pkg::SwAccessR0),
219     .RESVAL (1'h0),
220     .Mubi (1'b0)
221 ) u_status_init_error (
222     .clk_i (clk_i),
223     .rst_ni (rst_ni),
224
225     // from register interface
226     .we (1'b0),
227     .wd ('0),
228
229     // from internal hardware
230     .de (hw2reg.status.init_error.de),
231     .d (hw2reg.status.init_error.d),
232
233     // to internal hardware
234     .qe (),
235     .q (reg2hw.status.init_error.q),
236     .ds (),
237
238     // to register interface (read)
239     .qs (status_init_error_qs)
240 );
241
242 // F[escalated]: 2:2
243 prim_subreg #(
244     .DW (1),
245     .SwAccess(prim_subreg_pkg::SwAccessR0),
246     .RESVAL (1'h0),
247     .Mubi (1'b0)
248 ) u_status_escalated (
249     .clk_i (clk_i),
250     .rst_ni (rst_ni),
251
252     // from register interface
253     .we (1'b0),
254     .wd ('0),
255
256     // from internal hardware
257     .de (hw2reg.status.escalated.de),
258     .d (hw2reg.status.escalated.d),
259
260     // to internal hardware

```



```

262 .qe      ( ),
263 .q       (reg2hw.status.escalated.q),
264 .ds      ( ),
265
266 // to register interface (read)
267 .qs      (status_escalated_qs)
268 );
269
270 // F[scr_key_valid]: 3:3
271 prim_subreg #(
272   .DW      (1),
273   .SwAccess(prim_subreg_pkg::SwAccessR0),
274   .RESVAL  (1'h0),
275   .Mubi    (1'b0)
276 ) u_status_scr_key_valid (
277   .clk_i   (clk_i),
278   .rst_ni  (rst_ni),
279
280 // from register interface
281   .we      (1'b0),
282   .wd      ('0'),
283
284 // from internal hardware
285   .de      (hw2reg.status.scr_key_valid.de),
286   .d       (hw2reg.status.scr_key_valid.d),
287
288 // to internal hardware
289   .qe      ( ),
290   .q       ( ),
291   .ds      ( ),
292
293 // to register interface (read)
294   .qs      (status_scr_key_valid_qs)
295 );
296
297 // F[scr_key_seed_valid]: 4:4
298 prim_subreg #(
299   .DW      (1),
300   .SwAccess(prim_subreg_pkg::SwAccessR0),
301   .RESVAL  (1'h0),
302   .Mubi    (1'b0)
303 ) u_status_scr_key_seed_valid (
304   .clk_i   (clk_i),
305   .rst_ni  (rst_ni),
306
307 // from register interface
308   .we      (1'b0),
309   .wd      ('0'),
310
311 // from internal hardware
312   .de      (hw2reg.status.scr_key_seed_valid.de),
313   .d       (hw2reg.status.scr_key_seed_valid.d),
314
315 // to internal hardware
316   .qe      ( ),
317   .q       (reg2hw.status.scr_key_seed_valid.q),
318   .ds      ( ),
319
320 // to register interface (read)
321   .qs      (status_scr_key_seed_valid_qs)
322 );
323
324 // F[init_done]: 5:5
325 prim_subreg #(
326   .DW      (1),
327   .SwAccess(prim_subreg_pkg::SwAccessR0),
328   .RESVAL  (1'h0),
329   .Mubi    (1'b0)
330 ) u_status_init_done (
331   .clk_i   (clk_i),
332   .rst_ni  (rst_ni),
333
334 // from register interface
335   .we      (1'b0),
336   .wd      ('0'),
337
338 // from internal hardware
339   .de      (hw2reg.status.init_done.de),
340   .d       (hw2reg.status.init_done.d),
341
342 // to internal hardware
343   .qe      ( ),
344   .q       (reg2hw.status.init_done.q),
345   .ds      ( ),
346
347 // to register interface (read)
348   .qs      (status_init_done_qs)
349 );
350
351 // F[readback_error]: 6:6
352 prim_subreg #(
353   .DW      (1),
354   .SwAccess(prim_subreg_pkg::SwAccessR0),
355   .RESVAL  (1'h0),
356   .Mubi    (1'b0)
357 ) u_status_readback_error (
358   .clk_i   (clk_i),
359   .rst_ni  (rst_ni),
360
361 // from register interface
362   .we      (1'b0),
363
364   .wd      ('0'),
365
366 // from internal hardware
367   .de      (hw2reg.status.readback_error.de),
368   .d       (hw2reg.status.readback_error.d),
369
370 // to internal hardware
371   .qe      ( ),
372   .q       (reg2hw.status.readback_error.q),
373   .ds      ( ),
374
375 // to register interface (read)
376   .qs      (status_readback_error_qs)
377 );
378
379 // F[sram_alert]: 7:7
380 prim_subreg #(
381   .DW      (1),
382   .SwAccess(prim_subreg_pkg::SwAccessR0),
383   .RESVAL  (1'h0),
384   .Mubi    (1'b0)
385 ) u_status_sram_alert (
386   .clk_i   (clk_i),
387   .rst_ni  (rst_ni),
388
389 // from register interface
390   .we      (1'b0),
391   .wd      ('0'),
392
393 // from internal hardware
394   .de      (hw2reg.status.sram_alert.de),
395   .d       (hw2reg.status.sram_alert.d),
396
397 // to internal hardware
398   .qe      ( ),
399   .q       (reg2hw.status.sram_alert.q),
400   .ds      ( ),
401
402 // to register interface (read)
403   .qs      (status_sram_alert_qs)
404 );
405
406 // R[exec_regwen]: V(False)
407 prim_subreg #(
408   .DW      (1),
409   .SwAccess(prim_subreg_pkg::SwAccessW0C),
410   .RESVAL  (1'h1),
411   .Mubi    (1'b0)
412 ) u_exec_regwen (
413   .clk_i   (clk_i),
414   .rst_ni  (rst_ni),
415
416 // from register interface
417   .we      (exec_regwen_we),
418   .wd      (exec_regwen_wd),
419
420 // from internal hardware
421   .de      (1'b0),
422   .d       ('0'),
423
424 // to internal hardware
425   .qe      ( ),
426   .q       ( ),
427   .ds      ( ),
428
429 // to register interface (read)
430   .qs      (exec_regwen_qs)
431 );
432
433 // R[exec]: V(False)
434 // Create REGWEN-gated WE signal
435 logic exec_gated_we;
436 assign exec_gated_we = exec_we & exec_regwen_qs;
437 prim_subreg #(
438   .DW      (4),
439   .SwAccess(prim_subreg_pkg::SwAccessRW),
440   .RESVAL  (4'h9),
441   .Mubi    (1'b1)
442 ) u_exec (
443   .clk_i   (clk_i),
444   .rst_ni  (rst_ni),
445
446 // from register interface
447   .we      (exec_gated_we),
448   .wd      (exec_wd),
449
450 // from internal hardware
451   .de      (1'b0),
452   .d       ('0'),
453
454 // to internal hardware
455   .qe      ( ),
456   .q       (reg2hw.exec.q),
457   .ds      ( ),
458
459 // to register interface (read)
460   .qs      (exec_qs)
461 );
462
463

```

```

464
465 // R[ctrl_regwen]: V(False)
466 prim_subreg #(
467     .DW      (1),
468     .SwAccess(prim_subreg_pkg::SwAccessW0C),
469     .RESVAL   (1'h1),
470     .Mubi     (1'b0)
471 ) u_ctrl_regwen (
472     .clk_i   (clk_i),
473     .rst_ni  (rst_ni),
474
475     // from register interface
476     .we      (ctrl_regwen_we),
477     .wd      (ctrl_regwen_wd),
478
479     // from internal hardware
480     .de      (1'b0),
481     .d       ('0'),
482
483     // to internal hardware
484     .qe      (),
485     .q       (),
486     .ds      (),
487
488     // to register interface (read)
489     .qs      (ctrl_regwen_qs)
490 );
491
492 // R[ctrl]: V(False)
493 logic ctrl_qe;
494 logic [1:0] ctrl_flds_we;
495 prim_flop #(
496     .Width(1),
497     .ResetValue(0)
498 ) u_ctrl0_qe (
499     .clk_i(clk_i),
500     .rst_ni(rst_ni),
501     .d_i(&ctrl_flds_we),
502     .q_o(ctrl_qe)
503 );
504 // Create REGWEN-gated WE signal
505 logic ctrl_gated_we;
506 assign ctrl_gated_we = ctrl_we & ctrl_regwen_qs;
507 // F[renew_scr_key]: 0:0
508 prim_subreg #(
509     .DW      (1),
510     .SwAccess(prim_subreg_pkg::SwAccessW0),
511     .RESVAL   (1'h0),
512     .Mubi     (1'b0)
513 ) u_ctrl_renew_scr_key (
514     .clk_i   (clk_i),
515     .rst_ni  (rst_ni),
516
517     // from register interface
518     .we      (ctrl_gated_we),
519     .wd      (ctrl_renew_scr_key_wd),
520
521     // from internal hardware
522     .de      (1'b0),
523     .d       ('0'),
524
525     // to internal hardware
526     .qe      (ctrl_flds_we[0]),
527     .q       (reg2hw.ctrl.renew_scr_key.q),
528     .ds      (),
529
530     // to register interface (read)
531     .qs      ()
532 );
533 assign reg2hw.ctrl.renew_scr_key.qe = ctrl_qe;
534
535 // F[init]: 1:1
536 prim_subreg #(
537     .DW      (1),
538     .SwAccess(prim_subreg_pkg::SwAccessW0),
539     .RESVAL   (1'h0),
540     .Mubi     (1'b0)
541 ) u_ctrl_init (
542     .clk_i   (clk_i),
543     .rst_ni  (rst_ni),
544
545     // from register interface
546     .we      (ctrl_gated_we),
547     .wd      (ctrl_init_wd),
548
549     // from internal hardware
550     .de      (1'b0),
551     .d       ('0'),
552
553     // to internal hardware
554     .qe      (ctrl_flds_we[1]),
555     .q       (reg2hw.ctrl.init.q),
556     .ds      (),
557
558     // to register interface (read)
559     .qs      ()
560 );
561 assign reg2hw.ctrl.init.qe = ctrl_qe;
562
563
564
565 // R[scr_key_rotated]: V(False)
566 prim_subreg #(
567     .DW      (4),
568     .SwAccess(prim_subreg_pkg::SwAccessW1C),
569     .RESVAL   (4'h9),
570     .Mubi     (1'b1)
571 ) u_scr_key_rotated (
572     .clk_i   (clk_i),
573     .rst_ni  (rst_ni),
574
575     // from register interface
576     .we      (scr_key_rotated_we),
577     .wd      (scr_key_rotated_wd),
578
579     // from internal hardware
580     .de      (hw2reg.scr_key_rotated.de),
581     .d       (hw2reg.scr_key_rotated.d),
582
583     // to internal hardware
584     .qe      (),
585     .q       (),
586     .ds      (),
587
588     // to register interface (read)
589     .qs      (scr_key_rotated_qs)
590 );
591
592 // R[readback_regwen]: V(False)
593 prim_subreg #(
594     .DW      (1),
595     .SwAccess(prim_subreg_pkg::SwAccessW0C),
596     .RESVAL   (1'h1),
597     .Mubi     (1'b0)
598 ) u_readback_regwen (
599     .clk_i   (clk_i),
600     .rst_ni  (rst_ni),
601
602     // from register interface
603     .we      (readback_regwen_we),
604     .wd      (readback_regwen_wd),
605
606     // from internal hardware
607     .de      (1'b0),
608     .d       ('0'),
609
610     // to internal hardware
611     .qe      (),
612     .q       (),
613     .ds      (),
614
615     // to register interface (read)
616     .qs      (readback_regwen_qs)
617 );
618
619 // R[readback]: V(False)
620 // Create REGWEN-gated WE signal
621 logic readback_gated_we;
622 assign readback_gated_we = readback_we & readback_regwen_qs;
623 prim_subreg #(
624     .DW      (4),
625     .SwAccess(prim_subreg_pkg::SwAccessRW),
626     .RESVAL   (4'h9),
627     .Mubi     (1'b1)
628 ) u_readback (
629     .clk_i   (clk_i),
630     .rst_ni  (rst_ni),
631
632     // from register interface
633     .we      (readback_gated_we),
634     .wd      (readback_wd),
635
636     // from internal hardware
637     .de      (1'b0),
638     .d       ('0'),
639
640     // to internal hardware
641     .qe      (),
642     .q       (reg2hw.readback.q),
643     .ds      (),
644
645     // to register interface (read)
646     .qs      (readback_qs)
647 );
648
649 logic [8:0] addr_hit;
650 top_racl_pkg::racl_role_vec_t racl_role_vec;
651 top_racl_pkg::racl_role_t racl_role;
652
653 logic [8:0] racl_addr_hit_read;
654 logic [8:0] racl_addr_hit_write;
655
656 if (EnableRacl) begin : gen_racl_role_logic
657     // Retrieve RACL role from user bits and one-hot encode
658     that for the comparison bitmap
659     assign racl_role =
660     top_racl_pkg::tlul_extract_racl_role_bits(tl_i.a_user.rsvd);
661
662
663

```

```

664 prim_onehot_enc #(
665   .OneHotWidth( $bits(top_racl_pkg::racl_role_vec_t) )
666 ) u_racl_role_encode (
667   .in_i ( racl_role ),
668   .en_i ( 1'b1 ),
669   .out_o( racl_role_vec )
670 );
671 end else begin : gen_no_racl_role_logic
672   assign racl_role = '0;
673   assign racl_role_vec = '0;
674 end
675
676 always_comb begin
677   racl_addr_hit_read = '0;
678   racl_addr_hit_write = '0;
679   addr_hit[0] = (reg_addr == SRAM_CTRL_ALERT_TEST_OFFSET);
680   addr_hit[1] = (reg_addr == SRAM_CTRL_STATUS_OFFSET);
681   addr_hit[2] = (reg_addr == SRAM_CTRL_EXEC_REGWEN_OFFSET);
682   addr_hit[3] = (reg_addr == SRAM_CTRL_EXEC_OFFSET);
683   addr_hit[4] = (reg_addr == SRAM_CTRL_CTRL_REGWEN_OFFSET);
684   addr_hit[5] = (reg_addr == SRAM_CTRL_CTRL_OFFSET);
685   addr_hit[6] = (reg_addr == SRAM_C-
TRL_SCR_KEY_ROTATED_OFFSET);
686   addr_hit[7] = (reg_addr == SRAM_C-
TRL_READBACK_REGWEN_OFFSET);
687   addr_hit[8] = (reg_addr == SRAM_CTRL_READBACK_OFFSET);
688
689   if (EnableRacl) begin : gen_racl_hit
690     for (int unsigned slice_idx = 0; slice_idx < 9;
slice_idx++) begin
691       racl_addr_hit_read[slice_idx] =
692         addr_hit[slice_idx] & (|
(racl_policies_i[RaclPolicySelVec[slice_idx]].read_perm
& racl_role_vec));
693       racl_addr_hit_write[slice_idx] =
694         addr_hit[slice_idx] & (|
(racl_policies_i[RaclPolicySelVec[slice_idx]].write_perm
& racl_role_vec));
695     end
696   end else begin : gen_no_racl
697     racl_addr_hit_read = addr_hit;
698     racl_addr_hit_write = addr_hit;
699   end
700
701   end
702
703   assign addrmiss = (reg_re || reg_we) ? ~|addr_hit : 1'b0 ;
704   // A valid address hit, access, but failed the RACL check
705   assign racl_error_o.valid = |addr_hit & ((reg_re & ~|
racl_addr_hit_read) |
706   (reg_we & ~|
racl_addr_hit_write));
707   assign racl_error_o.request_address =
top_pkg::TL_AW'(reg_addr);
708   assign racl_error_o.racl_role = racl_role;
709   assign racl_error_o.overflow = 1'b0;
710
711   if (EnableRacl) begin : gen_racl_log
712     assign racl_error_o.ctn_uid =
top_pkg::tlul_extract_ctn_uid_bits(tl_i.a.user.rsvd);
713     assign racl_error_o.read_access = tl_i.a.opcode ==
tlul_pkg::Get;
714   end else begin : gen_no_racl_log
715     assign racl_error_o.ctn_uid = '0;
716     assign racl_error_o.read_access = 1'b0;
717   end
718
719   // Check sub-word write is permitted
720   always_comb begin
721     wr_err = (reg_we &
722       ((racl_addr_hit_write[0] & (|(SRAM_C-
TRL_REGS_PERMIT[0] & ~reg_be))) |
723       (racl_addr_hit_write[1] & (|(SRAM_C-
TRL_REGS_PERMIT[1] & ~reg_be))) |
724       (racl_addr_hit_write[2] & (|(SRAM_C-
TRL_REGS_PERMIT[2] & ~reg_be))) |
725       (racl_addr_hit_write[3] & (|(SRAM_C-
TRL_REGS_PERMIT[3] & ~reg_be))) |
726       (racl_addr_hit_write[4] & (|(SRAM_C-
TRL_REGS_PERMIT[4] & ~reg_be))) |
727       (racl_addr_hit_write[5] & (|(SRAM_C-
TRL_REGS_PERMIT[5] & ~reg_be))) |
728       (racl_addr_hit_write[6] & (|(SRAM_C-
TRL_REGS_PERMIT[6] & ~reg_be))) |
729       (racl_addr_hit_write[7] & (|(SRAM_C-
TRL_REGS_PERMIT[7] & ~reg_be))) |
730       (racl_addr_hit_write[8] & (|(SRAM_C-
TRL_REGS_PERMIT[8] & ~reg_be)))));
731   end
732
733   // Generate write-enables
734   assign alert_test_we = racl_addr_hit_write[0] & reg_we & !
reg_error;
735   assign alert_test_wd = reg_wdata[0];
736   assign exec_regwen_we = racl_addr_hit_write[2] & reg_we & !
reg_error;
737   assign exec_regwen_wd = reg_wdata[0];
738   assign exec_we = racl_addr_hit_write[3] & reg_we & !reg_er-
ror;
739
740   assign exec_regwen_wd = reg_wdata[0];
741   assign exec_we = racl_addr_hit_write[3] & reg_we & !reg_er-
ror;
742

```

```

743   assign exec_wd = reg_wdata[3:0];
744   assign ctrl_regwen_we = racl_addr_hit_write[4] & reg_we & !
reg_error;
745   assign ctrl_regwen_wd = reg_wdata[0];
746   assign ctrl_we = racl_addr_hit_write[5] & reg_we & !reg_er-
ror;
747   assign ctrl_renew_scr_key_wd = reg_wdata[0];
748   assign ctrl_init_wd = reg_wdata[1];
749   assign scr_key_rotated_we = racl_addr_hit_write[6] & reg_we
& !reg_error;
750   assign scr_key_rotated_wd = reg_wdata[3:0];
751   assign readback_regwen_we = racl_addr_hit_write[7] & reg_we
& !reg_error;
752   assign readback_regwen_wd = reg_wdata[0];
753   assign readback_we = racl_addr_hit_write[8] & reg_we & !
reg_error;
754   assign readback_wd = reg_wdata[3:0];
755
756   // Assign write-enables to checker logic vector.
757   always_comb begin
758     reg_we_check[0] = alert_test_we;
759     reg_we_check[1] = 1'b0;
760     reg_we_check[2] = exec_regwen_we;
761     reg_we_check[3] = exec_gated_we;
762     reg_we_check[4] = ctrl_regwen_we;
763     reg_we_check[5] = ctrl_gated_we;
764     reg_we_check[6] = scr_key_rotated_we;
765     reg_we_check[7] = readback_regwen_we;
766     reg_we_check[8] = readback_gated_we;
767   end
768
769   // Read data return
770   always_comb begin
771     reg_rdata_next = '0;
772     unique case (1'b1)
773       racl_addr_hit_read[0]: begin
774         reg_rdata_next[0] = '0;
775       end
776       racl_addr_hit_read[1]: begin
777         reg_rdata_next[0] = status_bus_integ_error_qs;
778         reg_rdata_next[1] = status_init_error_qs;
779         reg_rdata_next[2] = status_escalated_qs;
780         reg_rdata_next[3] = status_scr_key_valid_qs;
781         reg_rdata_next[4] = status_scr_key_seed_valid_qs;
782         reg_rdata_next[5] = status_init_done_qs;
783         reg_rdata_next[6] = status_readback_error_qs;
784         reg_rdata_next[7] = status_sram_alert_qs;
785       end
786       racl_addr_hit_read[2]: begin
787         reg_rdata_next[0] = exec_regwen_qs;
788       end
789       racl_addr_hit_read[3]: begin
790         reg_rdata_next[3:0] = exec_qs;
791       end
792       racl_addr_hit_read[4]: begin
793         reg_rdata_next[0] = ctrl_regwen_qs;
794       end
795       racl_addr_hit_read[5]: begin
796         reg_rdata_next[0] = '0;
797         reg_rdata_next[1] = '0;
798       end
799       racl_addr_hit_read[6]: begin
800         reg_rdata_next[3:0] = scr_key_rotated_qs;
801       end
802       racl_addr_hit_read[7]: begin
803         reg_rdata_next[0] = readback_regwen_qs;
804       end
805       racl_addr_hit_read[8]: begin
806         reg_rdata_next[3:0] = readback_qs;
807       end
808       default: begin
809         reg_rdata_next = '1;
810       end
811     endcase
812   end
813
814   // shadow busy
815   logic shadow_busy;
816   assign shadow_busy = 1'b0;
817
818   // register busy
819   assign reg_busy = shadow_busy;
820
821   // Unused signal tieoff
822
823   // wdata / byte enable are not always fully used

```

```

839 // add a blanket unused statement to handle lint waivers
840 logic unused_wdata;
841 logic unused_be;
842 assign unused_wdata = ^reg_wdata;
843 assign unused_be = ^reg_be;
844 logic unused_policy_sel;
845 assign unused_policy_sel = ^racl_policies_i;
846
847 // Assertions for Register Interface
848 `ASSERT_PULSE(wePulse, reg_we, clk_i, !rst_ni)
849 `ASSERT_PULSE(rePulse, reg_re, clk_i, !rst_ni)
850
851 `ASSERT(reAfterRv, $rose(reg_re || reg_we) |>

```

```

tl_o_pre.d_valid, clk_i, !rst_ni)
852
853 `ASSERT(en2addrHit, (reg_we || reg_re) |> $one-
hot0(addr_hit), clk_i, !rst_ni)
854
855 // this is formulated as an assumption such that the FPV
testbenches do disprove this
856 // property by mistake
857 `ASSUME(reqParity, tl_reg_h2d.a_valid |>
tl_reg_h2d.a_user.chk_en == tlul_pkg::CheckDis)
858
859 endmodule

```

OTP 컨트롤러

What this is. A controller for one-time-programmable memory.

Why it exists. Because some state must be unerasable: device-unique keys, life-cycle state, trim values. Since writes are irreversible, the programming procedure is extremely conservative.

Undergraduate review · EECS concepts. Non-volatile memory physics (eFuse, antifuse); error correction; idempotent writes.

Graduate · Practice note. A defect here produces permanently unusable devices. Verify this block more conservatively than any other.

1 files • 201 lines

API — otp_ctrl_pkg.sv

Table 1166. lowRISC_opentitan/hw/ip/otp_ctrl/rtl/otp_ctrl_pkg.sv — module / interface / package (1개)

이름	종류
otp_ctrl_pkg	package

Table 1167. lowRISC_opentitan/hw/ip/otp_ctrl/rtl/otp_ctrl_pkg.sv — parameter (15개)

파라미터	형/폭	기본값
DeviceIdWidth	int	256
ManufStateWidth	int	256
ScramblKeyWidth	int	128
ScramblBlockWidth	int	64
OTP_LC_DATA_DEFAULT	otp_lc_data_t	{
NvmKeySeedWidth	int	256
SramKeySeedWidth	int	128
NvmKeyWidth	int	128
SramKeyWidth	int	128
SramNonceWidth	int	128
OtbnKeyWidth	int	128
OtbnNonceWidth	int	64
OTP_KEYMGR_KEY_DEFAULT	otp_keymgr_key_t	{
NVM_OTP_KEY_RSP_DEFAULT	nvm_otp_key_rsp_t	{
SRAM_OTP_KEY_RSP_DEFAULT	sram_otp_key_rsp_t	{

Table 1168. lowRISC_opentitan/hw/ip/otp_ctrl/rtl/otp_ctrl_pkg.sv — localparam (3개)

상수	형/폭	값
OtbnNonceSel	int	OtbnNonceWidth / ScramblBlockWidth
NvmNonceSel	int	NvmKeyWidth / ScramblBlockWidth
SramNonceSel	int	SramNonceWidth / ScramblBlockWidth

Table 1169. lowRISC_opentitan/hw/ip/otp_ctrl/rtl/otp_ctrl_pkg.sv — typedef / using (6개)

새 이름	원래 형
otp_device_id_t	logic [DeviceIdWidth-1:0]
otp_manuf_state_t	logic [ManufStateWidth-1:0]
sram_key_t	logic [SramKeyWidth-1:0]
sram_nonce_t	logic [SramNonceWidth-1:0]
otbn_key_t	logic [OtbnKeyWidth-1:0]
otbn_nonce_t	logic [OtbnNonceWidth-1:0]

1. lowRISC_opentitan/hw/ip/otp_ctrl/rtl/otp_ctrl_pkg.sv (201 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // This package can be imported by generic IPs:
6 // - It does not import otp_ctrl_reg_pkg, which is generated
  and top-specific.
7
8 `include "prim_assert.sv"
9
10 package otp_ctrl_pkg;
11
12 ///////////////////////////////////////////////////////////////////
13 // General Parameters //
14 ///////////////////////////////////////////////////////////////////
15
16 parameter int DeviceIdWidth = 256;
17 typedef logic [DeviceIdWidth-1:0] otp_device_id_t;
18 // Device ID defined in the keymgr_dpe package must match
19 `ASSERT_STATIC_IN_PACKAGE(DeviceIdWidth_A,
20                           DeviceIdWidth == keymgr_dpe_pkg::DeviceIdWidth)
21
22 parameter int ManufStateWidth = 256;
23 typedef logic [ManufStateWidth-1:0] otp_manuf_state_t;
24
25 ///////////////////////////////////////////////////////////////////
26 // Typedefs for OTP Scrambling //
27 ///////////////////////////////////////////////////////////////////
28
29 parameter int ScramblKeyWidth = 128;
30 parameter int ScramblBlockWidth = 64;
31
32 ///////////////////////////////////////////////////////////////////
33 // Typedefs for LC Interface //
34 ///////////////////////////////////////////////////////////////////
35
36 // The tokens below are all hash post-images
37 typedef struct packed {
38     logic                                valid;
39     logic                                error;
40     // Use lc_state_t and lc_cnt_t here as very wide enumerations
41     // (> 64 bits)
42     // are not supported for virtual interfaces by Xcelium yet
43     // https://github.com/lowRISC/opentitan/issues/8884 (Cadenance issue: cds_46570160)
44
45     // The enumeration types lc_state_e and lc_cnt_e are still ok
46     // in other circumstances
47     lc_ctrl_state_pkg::lc_state_t        state;
48     lc_ctrl_state_pkg::lc_cnt_t          count;
49     // This is set to "On" if the partition containing the
50     // root secrets have been locked. In that case, the device
51     // is considered "personalized".
52     lc_ctrl_pkg::lc_tx_t                 secrets_valid;
53     // This is set to "On" if the partition containing the
54     // test tokens has been locked.
55     lc_ctrl_pkg::lc_tx_t                 test_tokens_valid;
56
57     lc_ctrl_state_pkg::lc_token_t        test_unlock_token;
58     lc_ctrl_state_pkg::lc_token_t        test_exit_token;
59     // This is set to "On" if the partition containing the
60     // rma token has been locked.
61     lc_ctrl_pkg::lc_tx_t                 rma_token_valid;
62     lc_ctrl_state_pkg::lc_token_t        rma_token;
63 } otp_lc_data_t;
64
65 // Default for dangling connection.
66 // Note that we put the life cycle into
67 // TEST_UNLOCKED0 by default such that top levels without
68 // the OTP controller can still function.
69 parameter otp_lc_data_t OTP_LC_DATA_DEFAULT = '{
70     valid: 1'b1,
71     error: 1'b0,
72     state: lc_ctrl_state_pkg::LcStTestUnlocked0,
73     count: lc_ctrl_state_pkg::LcCnt1,
74     secrets_valid: lc_ctrl_pkg::Off,
75     test_tokens_valid: lc_ctrl_pkg::Off,
76     test_unlock_token: '0,
77     test_exit_token: '0,
78     rma_token_valid: lc_ctrl_pkg::Off,
79     rma_token: '0
80 };
81
82 typedef struct packed {
83     logic req;
84     lc_ctrl_state_pkg::lc_state_e state;
85     lc_ctrl_state_pkg::lc_cnt_e count;
86 } lc_otp_program_req_t;
87
88 typedef struct packed {
89     logic err;
90     logic ack;
91 } lc_otp_program_rsp_t;
92
93 ///////////////////////////////////////////////////////////////////
94 // Typedefs for Key Broadcast //
95 ///////////////////////////////////////////////////////////////////
96
97 parameter int NvmKeySeedWidth = 256;
98 parameter int SramKeySeedWidth = 128;
99 parameter int NvmKeyWidth = 128;
100 parameter int SramKeyWidth = 128;
101 parameter int SramNonceWidth = 128;
102 parameter int OtbnKeyWidth = 128;
103 parameter int OtbnNonceWidth = 64;
104
105 typedef logic [SramKeyWidth-1:0] sram_key_t;
106 typedef logic [SramNonceWidth-1:0] sram_nonce_t;
107 typedef logic [OtbnKeyWidth-1:0] otbn_key_t;
108 typedef logic [OtbnNonceWidth-1:0] otbn_nonce_t;
109
110 localparam int OtbnNonceSel = OtbnNonceWidth / ScramblBlockWidth;
111 localparam int NvmNonceSel = NvmKeyWidth / ScramblBlockWidth;
112 localparam int SramNonceSel = SramNonceWidth / ScramblBlockWidth;
113
114 typedef struct packed {
115     logic [keymgr_dpe_pkg::KeyMgrKeyWidth-1:0] creat-
116     or_root_key_share0;
117     logic                                         creat-
118     or_root_key_share0_valid;
119 }

```

```

113 logic [keymgr_dpe_pkg::KeyMgrKeyWidth-1:0] creat-
or_root_key_share1;
114 logic creat-
or_root_key_share1_valid;
115 logic [keymgr_dpe_pkg::KeyMgrKeyWidth-1:0] creator_seed;
116 logic creat-
or_seed_valid;
117 logic [keymgr_dpe_pkg::KeyMgrKeyWidth-1:0] owner_seed;
118 logic own-
er_seed_valid;
119 } otp_keymgr_key_t;
120
121 parameter otp_keymgr_key_t OTP_KEYMGR_KEY_DEFAULT = '{
122 creator_root_key_share0: 256'hfeb7ea7ee90093cf4aff-
d9aaa2d6c0ec446cfd5f2d5a0bfd7e2d93edc63a102,
123 creator_root_key_share0_valid: 1'b1,
124 creator_root_key_share1:
256'h56d24a00181de99e0f690b447a8dde2a1ffb8b-
c306707107aa6e2410f15cfc37,
125 creator_root_key_share1_valid: 1'b1,
126 creator_seed: 256'hc7c50b38655cc87f821e5b07fed-
85d2c07e222a9e00bef308b3eccba0ba406fa,
127 creator_seed_valid: 1'b1,
128 owner_seed:
256'hf5052c0f14782d8b066be9f49c0b2000d3643ff3723ea7d-
b972f69cd3e2e3e68,
129 owner_seed_valid: 1'b1
130 };
131
132 typedef struct packed {
133 logic data_req; // Requests static key for data scram-
bling.
134 logic addr_req; // Requests static key for address scram-
bling.
135 } nvm_otp_key_req_t;
136
137 typedef struct packed {
138 logic req; // Requests ephemeral scrambling key and nonce.
139 } sram_otp_key_req_t;
140
141 typedef struct packed {
142 logic req; // Requests ephemeral scrambling key and nonce.
143 } otbn_otp_key_req_t;
144
145 typedef struct packed {
146 logic data_ack; // Ack for data key.
147 logic addr_ack; // Ack for address key.
148 logic [NvmKeyWidth-1:0] key; // 128bit static scram-
bling key.
149 logic [NvmKeyWidth-1:0] rand_key;
150 logic seed_valid; // Set to 1 if the key
seed has been provisioned and is
151 // valid.
152 } nvm_otp_key_rsp_t;

```

```

153
154 // Default for dangling connection
155 parameter nvm_otp_key_rsp_t NVM_OTP_KEY_RSP_DEFAULT = '{
156 data_ack: 1'b1,
157 addr_ack: 1'b1,
158 key: '0,
159 rand_key: '0,
160 seed_valid: 1'b1
161 };
162
163 typedef struct packed {
164 logic ack; // Ack for key.
165 sram_key_t key; // 128bit ephemeral scrambling
key.
166 sram_nonce_t nonce; // 128bit nonce.
167 logic seed_valid; // Set to 1 if the key seed has
been provisioned and is valid.
168 } sram_otp_key_rsp_t;
169
170 // Default for dangling connection
171 parameter sram_otp_key_rsp_t SRAM_OTP_KEY_RSP_DEFAULT = '{
172 ack: 1'b1,
173 key: '0,
174 nonce: '0,
175 seed_valid: 1'b1
176 };
177
178 typedef struct packed {
179 logic ack; // Ack for key.
180 otbn_key_t key; // 128bit ephemeral scrambling
key.
181 otbn_nonce_t nonce; // 256bit nonce.
182 logic seed_valid; // Set to 1 if the key seed has
been provisioned and is valid.
183 } otbn_otp_key_rsp_t;
184
185 ////////////////////////////////////////////////////
186 // Power/Reset Ctrl Interface //
187 ////////////////////////////////////////////////////
188
189 typedef struct packed {
190 logic init;
191 } pwr_otp_init_req_t;
192
193 typedef struct packed {
194 logic done;
195 } pwr_otp_init_rsp_t;
196
197 typedef struct packed {
198 logic idle;
199 } otp_pwr_state_t;
200
201 endpackage : otp_ctrl_pkg

```

키 관리자

What this is. Derives purpose-specific keys from a root key and gates access by life-cycle state.

Why it exists. To keep key material out of software entirely. Software requests an operation with a named key; the value travels only over hardware paths to the crypto engines.

Undergraduate review · EECS concepts. Key derivation functions; chain of trust; least privilege; hardware isolation.

Graduate · Practice note. Whether the hardware key path is genuinely isolated can only be established by reading the RTL: debug paths, scan chains and test modes are the usual leaks. **A scan chain can read every flip-flop**, which is why production parts lock scan.

10 files • 3,528 lines

API — keymgr.sv

Table 1170. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr.sv — module / interface / package (1개)

이름	종류
keymgr	module

Table 1171. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr.sv — parameter (18개)

파라미터	형/폭	기본값
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
UseOtpSeedsInsteadOfFlash	bit	1'b0
KmacEnMasking	bit	1'b1
RndCnstLfsrSeed	lfsr_seed_t	RndCnstLfsrSeedDefault
RndCnstLfsrPerm	lfsr_perm_t	RndCnstLfsrPermDefault
RndCnstRandPerm	rand_perm_t	RndCnstRandPermDefault
RndCnstRevisionSeed	seed_t	RndCnstRevisionSeedDefault
RndCnstCreatorIdentitySeed	seed_t	RndCnstCreatorIdentitySeedDefault
RndCnstOwnerIntIdentitySeed	seed_t	RndCnstOwnerIntIdentitySeedDefault
RndCnstOwnerIdentitySeed	seed_t	RndCnstOwnerIdentitySeedDefault
RndCnstSoftOutputSeed	seed_t	RndCnstSoftOutputSeedDefault
RndCnstHardOutputSeed	seed_t	RndCnstHardOutputSeedDefault
RndCnstNoneSeed	seed_t	RndCnstNoneSeedDefault
RndCnstAesSeed	seed_t	RndCnstAesSeedDefault
RndCnstOtbnSeed	seed_t	RndCnstOtbnSeedDefault
RndCnstKmacSeed	seed_t	RndCnstKmacSeedDefault

파라미터	형/폭	기본값
RndCnstCdi	seed_t	RndCnstCdiDefault

Table 1172. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr.sv — localparam (6개)

상수	형/폭	값
NumRomDigestInputs	int unsigned	1
TotalSeedWidth	int	KeyWidth * 10
RndConstSeed	logic [TotalSeedWidth-1:0]	{RndCnstRevisionSeed
AdvLfsrcopies	int	AdvDataWidth / 32
IdLfsrcopies	int	IdDataWidth / 32
GenLfsrcopies	int	GenDataWidth / 32

Table 1173. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr.sv — 포트 (24개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
rst_shadowed_ni	input	
clk_edn_i	input	
rst_edn_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
aes_key_o	output	hw_key_req_t
kmac_key_o	output	hw_key_req_t
otbn_key_o	output	otbn_key_req_t
kmac_data_o	output	kmac_pkg::app_req_t
kmac_data_i	input	kmac_pkg::app_rsp_t
kmac_en_masking_i	input	
lc_keymgr_en_i	input	lc_ctrl_pkg::lc_tx_t
lc_keymgr_div_i	input	lc_ctrl_pkg::lc_keymgr_div_t
otp_key_i	input	otp_ctrl_pkg::otp_keymgr_key_t
otp_device_id_i	input	otp_ctrl_pkg::otp_device_id_t
flash_i	input	flash_ctrl_pkg::keymgr_flash_t
edn_o	output	edn_pkg::edn_req_t
edn_i	input	edn_pkg::edn_rsp_t
rom_digest_i	input	rom_ctrl_pkg::keymgr_data_t
intr_op_done_o	output	logic
alert_rx_i	input	prim_alert_pkg::alert_rx_t [keymgr_reg_pkg::NumAle...

포트	방향	형/폭
alert_tx_o	output	prim_alert_pkg::alert_tx_t[keymgr_reg_pkg::NumAle...

API — keymgr_cfg_en.sv

Table 1174. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_cfg_en.sv — module / interface / package (1개)

이름	종류
keymgr_cfg_en	module

Table 1175. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_cfg_en.sv — parameter (1개)

파라미터	형/폭	기본값
NonInitClr	bit	1'b1

Table 1176. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_cfg_en.sv — 포트 (7개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
init_i	input	
en_i	input	
set_i	input	
clr_i	input	
out_o	output	logic

API — keymgr_ctrl.sv

Table 1177. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_ctrl.sv — module / interface / package (1개)

이름	종류
keymgr_ctrl	module

Table 1178. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_ctrl.sv — parameter (1개)

파라미터	형/폭	기본값
KmacEnMasking	bit	1'b1

Table 1179. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_ctrl.sv — localparam (9개)

상수	형/폭	값
EntropyWidth	int	LfsrWidth / 2
EntropyRounds	int	KeyWidth / EntropyWidth
EntropyRndWidth	int	prim_util_pkg::vbits(EntropyRounds)
CntWidth	int	EntropyRounds > CDIs ? EntropyRndWidth : CdiWidth
EccDataWidth	int	64
EccWidth	int	8

상수	형/폭	값
EccWords	int	KeyWidth / EccDataWidth
TotalEccWords	int	EccWords * Shares * CDIs
StateWidth	int	10

Table 1180. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_ctrl.sv — 포트 (44개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
en_i	input	
regfile_intg_err_i	input	
shadowed_update_err_i	input	
shadowed_storage_err_i	input	
reseed_cnt_err_i	input	
sideload_sel_err_i	input	
sideload_fsm_err_i	input	
op_start_i	input	
op_i	input	keymgr_ops_e
op_cdi_sel_i	input	[CdiWidth-1:0]
op_done_o	output	logic
status_o	output	keymgr_op_status_e
error_o	output	logic [ErrLastPos-1:0]
fault_o	output	logic [FaultLastPos-1:0]
data_hw_en_o	output	logic
data_sw_en_o	output	logic
data_valid_o	output	logic
wipe_key_o	output	logic
working_state_o	output	keymgr_working_state_e
sw_binding_unlock_o	output	logic
init_o	output	logic
root_key_i	input	otp_ctrl_pkg::otp_keymgr_key_t
hw_sel_o	output	prim_mubi_pkg::mubi4_t
stage_sel_o	output	keymgr_stage_e
invalid_stage_sel_o	output	logic
cdi_sel_o	output	logic [CdiWidth-1:0]
adv_en_o	output	logic

포트	방향	형/폭
id_en_o	output	logic
gen_en_o	output	logic
key_o	output	hw_key_req_t
kmac_done_i	input	
kmac_input_invalid_i	input	
kmac_fsm_err_i	input	
kmac_op_err_i	input	
kmac_done_err_i	input	
kmac_cmd_err_i	input	
kmac_data_i	input	[Shares-1:0][KeyWidth-1:0]
entropy_i	input	[Shares-1:0][RandWidth-1:0]
prng_reseed_done_i	input	
prng_reseed_ack_i	input	
prng_reseed_req_o	output	logic
prng_en_o	output	logic

API — keymgr_data_en_state.sv

Table 1181. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_data_en_state.sv — module / interface / package (1개)

이름	종류
keymgr_data_en_state	module

Table 1182. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_data_en_state.sv — localparam (1개)

상수	형/폭	값
DataStateWidth	int	10

Table 1183. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_data_en_state.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
hw_sel_i	input	prim_mubi_pkg::mubi4_t
adv_en_i	input	
id_en_i	input	
gen_en_i	input	
op_done_i	input	
op_start_i	input	
data_hw_en_o	output	logic

포트	방향	형/폭
data_sw_en_o	output	logic
fsm_err_o	output	logic

API — keymgr_err.sv

Table 1184. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_err.sv — module / interface / package (1개)

이름	종류
keymgr_err	module

Table 1185. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_err.sv — 포트 (32개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
invalid_op_i	input	
disabled_i	input	
invalid_i	input	
kmac_input_invalid_i	input	
shadowed_update_err_i	input	
kmac_op_err_i	input	
invalid_kmac_out_i	input	
sideload_sel_err_i	input	
kmac_cmd_err_i	input	
kmac_fsm_err_i	input	
kmac_done_err_i	input	
regfile_intg_err_i	input	
shadowed_storage_err_i	input	
ctrl_fsm_err_i	input	
data_fsm_err_i	input	
op_fsm_err_i	input	
ecc_err_i	input	
state_change_err_i	input	
op_state_cmd_err_i	input	
cnt_err_i	input	
reseed_cnt_err_i	input	
sideload_fsm_err_i	input	
op_update_i	input	
op_done_i	input	

포트	방향	형/폭
sync_err_o	output	logic [SyncErrLastIdx-1:0]
async_err_o	output	logic [AsyncErrLastIdx-1:0]
sync_fault_o	output	logic [SyncFaultLastIdx-1:0]
async_fault_o	output	logic [AsyncFaultLastIdx-1:0]
error_o	output	logic [ErrLastPos-1:0]
fault_o	output	logic [FaultLastPos-1:0]

API — keymgr_input_checks.sv

Table 1186. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_input_checks.sv — module / interface / package (1개)

이름	종류
keymgr_input_checks	module

Table 1187. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_input_checks.sv — parameter (2개)

파라미터	형/폭	기본값
KmacEnMasking	bit	1'b1
NumRomDigestInputs	int unsigned	1

Table 1188. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_input_checks.sv — localparam (1개)

상수	형/폭	값
KeyShares	int	KmacEnMasking ? Shares : 1

Table 1189. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_input_checks.sv — 포트 (15개)

포트	방향	형/폭
rom_digest_i	input	rom_ctrl_pkg::keymgr_data_t [NumRomDigestInputs-1:...
cur_max_key_version_i	input	[KeyVersionWidth-1:0]
key_i	input	hw_key_req_t
key_version_i	input	[31:0]
creator_seed_i	input	[KeyWidth-1:0]
owner_seed_i	input	[KeyWidth-1:0]
devid_i	input	[DevidWidth-1:0]
health_state_i	input	[HealthStateWidth-1:0]
creator_seed_vld_o	output	logic
owner_seed_vld_o	output	logic
devid_vld_o	output	logic
health_state_vld_o	output	logic
key_version_vld_o	output	logic

포트	방향	형/폭
key_vld_o	output	logic
rom_digest_vld_o	output	logic

API — keymgr_kmac_if.sv

Table 1190. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_kmac_if.sv — module / interface / package (1개)

이름	종류
keymgr_kmac_if	module

Table 1191. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_kmac_if.sv — parameter (2개)

파라미터	형/폭	기본값
RndCnstRandPerm	rand_perm_t	RndCnstRandPermDefault
MaxAdvDataWidth	int	AdvDataWidth

Table 1192. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_kmac_if.sv — localparam (21개)

상수	형/폭	값
StateWidth	int	10
AdvRem	int	MaxAdvDataWidth % KmacDataIfWidth
IdRem	int	IdDataWidth % KmacDataIfWidth
GenRem	int	GenDataWidth % KmacDataIfWidth
AdvRounds	int	(MaxAdvDataWidth + KmacDataIfWidth - 1) / KmacDataIfWidth
IdRounds	int	(IdDataWidth + KmacDataIfWidth - 1) / KmacDataIfWidth
GenRounds	int	(GenDataWidth + KmacDataIfWidth - 1) / KmacDataIfWidth
MaxRounds	int	KDFMaxWidth / KmacDataIfWidth
CntWidth	int	\$clog2(MaxRounds)
IfBytes	int	KmacDataIfWidth / 8
DecoyCopies	int	KmacDataIfWidth / RandWidth
DecoyOutputCopies	int	(kmac_pkg::AppDigestW / RandWidth)
LastAdvRoundInt	int unsigned	AdvRounds - 1
LastIdRoundInt	int unsigned	IdRounds - 1
LastGenRoundInt	int unsigned	GenRounds - 1
LastAdvRound	bit [CntWidth-1:0]	LastAdvRoundInt[CntWidth-1:0]
LastIdRound	bit [CntWidth-1:0]	LastIdRoundInt[CntWidth-1:0]
LastGenRound	bit [CntWidth-1:0]	LastGenRoundInt[CntWidth-1:0]
AdvByteMask	logic [IfBytes-1:0]	(AdvRem > 0) ? (2**(AdvRem/8)-1) : {IfBytes{1'b1}}
IdByteMask	logic [IfBytes-1:0]	(IdRem > 0) ? (2**(IdRem/8)-1) : {IfBytes{1'b1}}
GenByteMask	logic [IfBytes-1:0]	(GenRem > 0) ? (2**(GenRem/8)-1) : {IfBytes{1'b1}}

Table 1193. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_kmac_if.sv — 포트 (20개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
adv_data_i	input	[MaxAdvDataWidth-1:0]
id_data_i	input	[IdDataWidth-1:0]
gen_data_i	input	[GenDataWidth-1:0]
inputs_invalid_i	input	[3:0]
inputs_invalid_o	output	logic
adv_en_i	input	
id_en_i	input	
gen_en_i	input	
done_o	output	logic
data_o	output	logic [Shares-1:0][kmac_pkg::AppDigestW-1:0]
kmac_data_o	output	kmac_pkg::app_req_t
kmac_data_i	input	kmac_pkg::app_rsp_t
prng_en_o	output	logic
entropy_i	input	[Shares-1:0][RandWidth-1:0]
fsm_error_o	output	logic
kmac_error_o	output	logic
kmac_done_error_o	output	logic
cmd_error_o	output	logic

API — keymgr_op_state_ctrl.sv**Table 1194. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_op_state_ctrl.sv — module / interface / package (1개)**

이름	종류
keymgr_op_state_ctrl	module

Table 1195. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_op_state_ctrl.sv — localparam (1개)

상수	형/폭	값
OpStateWidth	int	8

Table 1196. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_op_state_ctrl.sv — 포트 (15개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
adv_req_i	input	

포트	방향	형/폭
dis_req_i	input	
id_req_i	input	
gen_req_i	input	
cnt_i	input	[CdiWidth-1:0]
op_ack_o	output	logic
op_busy_o	output	logic
op_update_o	output	logic
kmac_done_i	input	
adv_en_o	output	logic
id_en_o	output	logic
gen_en_o	output	logic
op_fsm_err_o	output	logic

API — keymgr_pkg.sv

Table 1197. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_pkg.sv — module / interface / package (1개)

이름	종류
keymgr_pkg	package

Table 1198. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_pkg.sv — parameter (37개)

파라미터	형/폭	기본값
KeyWidth	int	256
CDIs	int	2
CdiWidth	int	prim_util_pkg::vbits(CDIs)
OtbnKeyWidth	int	384
DigestWidth	int	128
KmacDataIfWidth	int	64
KeyMgrStages	int	3
SwBindingWidth	int	32 * keymgr_reg_pkg::NumSwBindingReg
SaltWidth	int	32 * keymgr_reg_pkg::NumSaltReg
Shares	int	2
KeyVersionWidth	int	32
HealthStateWidth	int	128
DevIdWidth	int	256
MaxWidth	int	256
RndCnstRevisionSeedDefault	seed_t	256'h69802e51_bacf8874_e650d692_e3d8a646_2d3f158f_0bf7961d_d...

파라미터	형/폭	기본값
RndCnstCreatorIdentity-SeedDefault	seed_t	256'h344bff84_d3edf83e_aa565c5e_6c2d22f2_75283734_15cf6d83_3...
RndCnstOwnerIntIdentity-SeedDefault	seed_t	256'h32a37bf9_2627382b_47180548_1fc4b847_a7eba1ce_c13e80ab_0...
RndCnstOwnerIdentity-SeedDefault	seed_t	256'hc1ba416c_822ad676_e4f54f36_33d8dc7e_00ccfb8b_92209f1b_d...
RndCnstSoftOutputSeedDefault	seed_t	256'h23cbe85e_62e39992_4ab8d6a6_acbf12f0_0ca6f488_63eaa428_f...
RndCnstHardOutputSeedDefault	seed_t	256'h877ef0b9_aceaefc5_693b1aa7_e43a7f3d_ee7d63b6_4e73e182_b...
RndCnstNoneSeedDefault	seed_t	256'h6013ba4d_8d04fe4f_80cb0472_536c9679_08b82f1d_98a4e405_c...
RndCnstAesSeedDefault	seed_t	256'he16a8fa9_5b613cd5_fb9ad23f_bd8347e1_64e45dac_5d08a41b_e...
RndCnstKmacSeedDefault	seed_t	256'hc57f4c0b_b308e83f_3fc4bc63_d87dd67d_9071dc1c_e19484c8_3...
RndCnstOtbnSeedDefault	seed_t	256'hcb-cb4d2d_0abeb81b_ca7451ae_d1e2479d_ba13530a_d046b945_6...
RndCnstCdiDefault	seed_t	256'h54180905_d14c1d2f_2dda1522_f332bc0e_fcd6b92f_f0f9db75_3...
LfsrWidth	int	64
RndCnstLfsrSeedDefault	lfsr_seed_t	64'h22d326255bd24320
RndCnstLfsrPermDefault	lfsr_perm_t	{
RandWidth	int	LfsrWidth / 2
RndCnstRandPermDefault	rand_perm_t	{
AdvDataWidth	int	SwBindingWidth + 2*KeyWidth + DevIdWidth + HealthStateWidth
IdDataWidth	int	KeyWidth
GenDataWidth	int	32 + SaltWidth + KeyWidth*2
StageWidth	int	\$clog2(KeyMgrStages)
KDFMaxWidth	int	1984
HW_KEY_REQ_DEFAULT	hw_key_req_t	{
OTBN_KEY_REQ_DEFAULT	otbn_key_req_t	{

Table 1199. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_pkg.sv — typedef / using (4개)

새 이름	원래 형
seed_t	logic [KeyWidth-1:0]
lfsr_seed_t	logic [LfsrWidth-1:0]
lfsr_perm_t	logic [LfsrWidth-1:0][\$clog2(LfsrWidth)-1:0]
rand_perm_t	logic [RandWidth-1:0][\$clog2(RandWidth)-1:0]

API — keymgr_reg_pkg.sv

Table 1200. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_reg_pkg.sv — module / interface / package (1개)

이름	종류
keymgr_reg_pkg	package

Table 1201. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_reg_pkg.sv — parameter (79개) — 앞 60개만 표시(전체 79개, 본문 소스 참조)

파라미터	형/폭	기본값
NumSaltReg	int	8
NumSwBindingReg	int	8
NumOutReg	int	8
NumKeyVersion	int	1
NumAlerts	int	2
BlockAw	int	8
NumRegs	int	63
KEYMGR_INTR_STATE_OFFSET	logic [BlockAw-1:0]	8'h 0
KEYMGR_INTR_ENABLE_OFFSET	logic [BlockAw-1:0]	8'h 4
KEYMGR_INTR_TEST_OFFSET	logic [BlockAw-1:0]	8'h 8
KEYMGR_ALERT_TEST_OFFSET	logic [BlockAw-1:0]	8'h c
KEYMGR_CFG_REGWEN_OFFSET	logic [BlockAw-1:0]	8'h 10
KEYMGR_START_OFFSET	logic [BlockAw-1:0]	8'h 14
KEYMGR_CONTROL_SHADOWED_OFFSET	logic [BlockAw-1:0]	8'h 18
KEYMGR_SIDELOAD_CLEAR_OFFSET	logic [BlockAw-1:0]	8'h 1c
KEYMGR_RESEED_INTERVAL_REGWEN_OFFSET	logic [BlockAw-1:0]	8'h 20
KEYMGR_RESEED_INTERVAL_SHADOWED_OFFSET	logic [BlockAw-1:0]	8'h 24
KEYMGR_SW_BINDING_REGWEN_OFFSET	logic [BlockAw-1:0]	8'h 28
KEYMGR_SEALING_SW_BINDING_0_OFFSET	logic [BlockAw-1:0]	8'h 2c
KEYMGR_SEALING_SW_BINDING_1_OFFSET	logic [BlockAw-1:0]	8'h 30
KEYMGR_SEALING_SW_BINDING_2_OFFSET	logic [BlockAw-1:0]	8'h 34
KEYMGR_SEALING_SW_BINDING_3_OFFSET	logic [BlockAw-1:0]	8'h 38
KEYMGR_SEALING_SW_BINDING_4_OFFSET	logic [BlockAw-1:0]	8'h 3c
KEYMGR_SEALING_SW_BINDING_5_OFFSET	logic [BlockAw-1:0]	8'h 40
KEYMGR_SEALING_SW_BINDING_6_OFFSET	logic [BlockAw-1:0]	8'h 44
KEYMGR_SEALING_SW_BINDING_7_OFFSET	logic [BlockAw-1:0]	8'h 48
KEYMGR_ATTEST_SW_BINDING_0_OFFSET	logic [BlockAw-1:0]	8'h 4c
KEYMGR_ATTEST_SW_BINDING_1_OFFSET	logic [BlockAw-1:0]	8'h 50
KEYMGR_ATTEST_SW_BINDING_2_OFFSET	logic [BlockAw-1:0]	8'h 54
KEYMGR_ATTEST_SW_BINDING_3_OFFSET	logic [BlockAw-1:0]	8'h 58
KEYMGR_ATTEST_SW_BINDING_4_OFFSET	logic [BlockAw-1:0]	8'h 5c

파라미터	형/폭	기본값
KEYMGR_ATTEST_SW_BINDING_5_OFFSET	logic [BlockAw-1:0]	8'h 60
KEYMGR_ATTEST_SW_BINDING_6_OFFSET	logic [BlockAw-1:0]	8'h 64
KEYMGR_ATTEST_SW_BINDING_7_OFFSET	logic [BlockAw-1:0]	8'h 68
KEYMGR_SALT_0_OFFSET	logic [BlockAw-1:0]	8'h 6c
KEYMGR_SALT_1_OFFSET	logic [BlockAw-1:0]	8'h 70
KEYMGR_SALT_2_OFFSET	logic [BlockAw-1:0]	8'h 74
KEYMGR_SALT_3_OFFSET	logic [BlockAw-1:0]	8'h 78
KEYMGR_SALT_4_OFFSET	logic [BlockAw-1:0]	8'h 7c
KEYMGR_SALT_5_OFFSET	logic [BlockAw-1:0]	8'h 80
KEYMGR_SALT_6_OFFSET	logic [BlockAw-1:0]	8'h 84
KEYMGR_SALT_7_OFFSET	logic [BlockAw-1:0]	8'h 88
KEYMGR_KEY_VERSION_OFFSET	logic [BlockAw-1:0]	8'h 8c
KEYMGR_MAX_CREATOR_KEY_VER_REGWEN_OFFSET	logic [BlockAw-1:0]	8'h 90
KEYMGR_MAX_CREATOR_KEY_VER_SHADOWED_OFFSET	logic [BlockAw-1:0]	8'h 94
KEYMGR_MAX_OWNER_INT_KEY_VER_REGWEN_OFFSET	logic [BlockAw-1:0]	8'h 98
KEYMGR_MAX_OWNER_INT_KEY_VER_SHADOWED_OFFSET	logic [BlockAw-1:0]	8'h 9c
KEYMGR_MAX_OWNER_KEY_VER_REGWEN_OFFSET	logic [BlockAw-1:0]	8'h a0
KEYMGR_MAX_OWNER_KEY_VER_SHADOWED_OFFSET	logic [BlockAw-1:0]	8'h a4
KEYMGR_SW_SHARE0_OUTPUT_0_OFFSET	logic [BlockAw-1:0]	8'h a8
KEYMGR_SW_SHARE0_OUTPUT_1_OFFSET	logic [BlockAw-1:0]	8'h ac
KEYMGR_SW_SHARE0_OUTPUT_2_OFFSET	logic [BlockAw-1:0]	8'h b0
KEYMGR_SW_SHARE0_OUTPUT_3_OFFSET	logic [BlockAw-1:0]	8'h b4
KEYMGR_SW_SHARE0_OUTPUT_4_OFFSET	logic [BlockAw-1:0]	8'h b8
KEYMGR_SW_SHARE0_OUTPUT_5_OFFSET	logic [BlockAw-1:0]	8'h bc
KEYMGR_SW_SHARE0_OUTPUT_6_OFFSET	logic [BlockAw-1:0]	8'h c0
KEYMGR_SW_SHARE0_OUTPUT_7_OFFSET	logic [BlockAw-1:0]	8'h c4
KEYMGR_SW_SHARE1_OUTPUT_0_OFFSET	logic [BlockAw-1:0]	8'h c8
KEYMGR_SW_SHARE1_OUTPUT_1_OFFSET	logic [BlockAw-1:0]	8'h cc
KEYMGR_SW_SHARE1_OUTPUT_2_OFFSET	logic [BlockAw-1:0]	8'h d0

1. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr.sv (814 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Key manager top level
6 //
7
8 `include "prim_assert.sv"
9
10 module keymgr

```

```

11 import keymgr_pkg::*;
12 import keymgr_reg_pkg::*;
13 #(
14     parameter logic [NumAlerts-1:0] AlertAsyncOn = {Num-
mAlerts{1'b1}},
15     // Number of cycles a differential skew is tolerated on the
alert signal
16     parameter int unsigned AlertSkewCycles = 1,
17     // In case this is set to true, the keymgr will ignore the
creator / owner seeds
18     // on the flash_i port and use the seeds provided in
otp_key_i instead.
19     parameter bit UseOtpSeedsInsteadOfFlash = 1'b0,
20     parameter bit KmacEnMasking = 1'b1,
21     parameter lfsr_seed_t RndCnstLfsrSeed = RndCnstLfsr-

```

```

SeedDefault,
22 parameter lfsr_perm_t RndCnstLfsrPerm = RndCnstLfs-
rPermDefault,
23 parameter rand_perm_t RndCnstRandPerm = RndCnstRand-
PermDefault,
24 parameter seed_t RndCnstRevisionSeed = RndCnstRevi-
sionSeedDefault,
25 parameter seed_t RndCnstCreatorIdentitySeed = RndCnstCreat-
orIdentitySeedDefault,
26 parameter seed_t RndCnstOwnerIntIdentitySeed = RndCnstOwner-
IntIdentitySeedDefault,
27 parameter seed_t RndCnstOwnerIdentitySeed = RndCnstOwn-
erIdentitySeedDefault,
28 parameter seed_t RndCnstSoftOutputSeed = RndCnst-
SoftOutputSeedDefault,
29 parameter seed_t RndCnstHardOutputSeed = RndCnst-
stHardOutputSeedDefault,
30 parameter seed_t RndCnstNoneSeed = RndCnstNone-
SeedDefault,
31 parameter seed_t RndCnstAesSeed = RndCnstAesS-
eedDefault,
32 parameter seed_t RndCnstOtbnSeed = RndCnstOtbn-
SeedDefault,
33 parameter seed_t RndCnstKmacSeed = RndCnstKmac-
SeedDefault,
34 parameter seed_t RndCnstCdi = RndCnstCdiDe-
fault
35 ) (
36 input clk_i,
37 input rst_ni,
38 input rst_shadowed_ni,
39 input clk_edn_i,
40 input rst_edn_i,
41
42 // Bus Interface
43 input tlul_pkg::tl_h2d_t tl_i,
44 output tlul_pkg::tl_d2h_t tl_o,
45
46 // key interface to crypto modules
47 output hw_key_req_t aes_key_o,
48 output hw_key_req_t kmac_key_o,
49 output otbn_key_req_t otbn_key_o,
50
51 // data interface to/from crypto modules
52 output kmac_pkg::app_req_t kmac_data_o,
53 input kmac_pkg::app_rsp_t kmac_data_i,
54
55 // whether kmac is masked
56 // Note this input is not driving ANY logic directly. In-
stead it is only used
57 // as part of assertions. This is done because if boundary
optimization were
58 // ever disabled, it would provide a VERY obvious location
for attacks.
59 input kmac_en_masking_i,
60
61 // the following signals should eventually be wrapped into
structs
from other modules
62 // SEC_CM: LC_CTRL.INTERSIG.MUBI
63 input lc_ctrl_pkg::lc_tx_t lc_keymgr_en_i,
64 input lc_ctrl_pkg::lc_keymgr_div_t lc_keymgr_div_i,
65 input otp_ctrl_pkg::otp_keymgr_key_t otp_key_i,
66 input otp_ctrl_pkg::otp_device_id_t otp_device_id_i,
67 input flash_ctrl_pkg::keymgr_flash_t flash_i,
68
69 // connection to edn
70 output edn_pkg::edn_req_t edn_o,
71 input edn_pkg::edn_rsp_t edn_i,
72
73 // connection to rom_ctrl
74 input rom_ctrl_pkg::keymgr_data_t rom_digest_i,
75
76 // interrupts and alerts
77 output logic intr_op_done_o,
78 input prim_alert_pkg::alert_rx_t [keym-
gr_reg_pkg::NumAlerts-1:0] alert_rx_i,
79 output prim_alert_pkg::alert_tx_t [keym-
gr_reg_pkg::NumAlerts-1:0] alert_tx_o
80 );
81
82 `ASSERT_INIT(AdvDataWidth_A, AdvDataWidth <= KDFMaxWidth)
83 `ASSERT_INIT(IdDataWidth_A, IdDataWidth <= KDFMaxWidth)
84 `ASSERT_INIT(GenDataWidth_A, GenDataWidth <= KDFMaxWidth)
85 `ASSERT_INIT(MaxWidthDivisible_A, KDFMaxWidth %
KmacDataIfWidth == 0)
86 `ASSERT_INIT(OutputKeyDiff_A, RndCnstHardOutputSeed != Rnd-
CnstSoftOutputSeed)
87
88 import prim_mubi_pkg::mubi4_test_true_strict;
89 import prim_mubi_pkg::mubi4_test_false_strict;
90 import lc_ctrl_pkg::lc_tx_test_true_strict;
91 import lc_ctrl_pkg::lc_tx_t;
92
93 localparam int unsigned NumRomDigestInputs = 1;
94
95 //////////////////////////////////////////////////
96 // Anchor incoming seeds and constants
97 //////////////////////////////////////////////////
98 localparam int TotalSeedWidth = KeyWidth * 10;
99 seed_t revision_seed;
100 seed_t creator_identity_seed;

```

```

101 seed_t owner_int_identity_seed;
102 seed_t owner_identity_seed;
103 seed_t soft_output_seed;
104 seed_t hard_output_seed;
105 seed_t aes_seed;
106 seed_t otbn_seed;
107 seed_t kmac_seed;
108 seed_t none_seed;
109
110 localparam logic [TotalSeedWidth-1:0] RndConstSeed = {RndCn-
stRevisionSeed,
111 RndCn-
stCreatorIdentitySeed,
112 RndCn-
stOwnerIntIdentitySeed,
113 RndCn-
stOwnerIdentitySeed,
114 RndCn-
stSoftOutputSeed,
115 RndCn-
stHardOutputSeed,
116 RndCn-
stAesSeed,
117 RndCn-
stOtbnSeed,
118 RndCn-
stKmacSeed,
119 RndCn-
stNoneSeed};
120
121 prim_sec_anchor_const #(
122 .Width(TotalSeedWidth),
123 .ConstVal(RndConstSeed)
124 ) u_seed_anchor_const (
125 .out_o({revision_seed,
126 creator_identity_seed,
127 owner_int_identity_seed,
128 owner_identity_seed,
129 soft_output_seed,
130 hard_output_seed,
131 aes_seed,
132 otbn_seed,
133 kmac_seed,
134 none_seed})
135 );
136
137 // Register module
138 keymgr_reg2hw_t reg2hw;
139 keymgr_hw2reg_t hw2reg;
140
141 logic regfile_intg_err;
142 logic shadowed_storage_err;
143 logic shadowed_update_err;
144 // SEC_CM: BUS INTEGRITY
145 // SEC_CM: CONFIG.SHADOW
146 // SEC_CM: OP.CONFIG.REGWEN, RESEED.CONFIG.REGWEN, SW_BIND-
ING.CONFIG.REGWEN
147 // SEC_CM: MAX_KEY_VER.CONFIG.REGWEN
148 keymgr_reg_top u_reg (
149 .clk_i,
150 .rst_ni,
151 .rst_shadowed_ni,
152 .tl_i,
153 .tl_o,
154 .reg2hw,
155 .hw2reg,
156 .shadowed_storage_err_o (shadowed_storage_err),
157 .shadowed_update_err_o (shadowed_update_err),
158 .intg_err_o (regfile_intg_err)
159 );
160
161 //////////////////////////////////////////////////
162 // Synchronize lc_ctrl control inputs
163 // Data inputs are not synchronized and assumed quasi-stat-
ic
164 //////////////////////////////////////////////////
165 lc_tx_t [KeyMgrEnLast-1:0] lc_keymgr_en;
166
167 prim_lc_sync #(
168 .NumCopies(int'(KeyMgrEnLast))
169 ) u_lc_keymgr_en_sync (
170 .clk_i,
171 .rst_ni,
172 .lc_en_i(lc_keymgr_en_i),
173 .lc_en_o(lc_keymgr_en)
174 );
175
176 //////////////////////////////////////////////////
177 // LFSR
178 //////////////////////////////////////////////////
179
180 // A fairly large lfsr is used here as entropy in multiple
places.
181 // - populate the default working state
182 // - generate random inputs when a bad input is selected
183 //
184 // The first case is sensitive, and thus the working state
is constructed
185 // through multiple rounds of the Lfsr
186 // The second case is less sensitive and is applied dir-

```

```

ectly. If the inputs
188 // have more bits than the lfsr output, the lfsr value is
simply replicated
189
190 logic lfsr_en;
191 logic seed_en;
192 logic [LfsrWidth-1:0] seed;
193 logic reseed_req;
194 logic reseed_ack;
195 logic reseed_done;
196 logic reseed_cnt_err;
197
198 keymgr_reseed_ctrl u_reseed_ctrl (
199   .clk_i,
200   .rst_ni,
201   .clk_edn_i,
202   .rst_edn_ni,
203   .reseed_req_i(reseed_req),
204   .reseed_ack_o(reseed_ack),
205   .reseed_done_o(reseed_done),
206   .reseed_interval_i(reg2hw.reseed_interval_shadowed.q),
207   .edn_o,
208   .edn_i,
209   .lfsr_en_i(lfsr_en),
210   .seed_en_o(seed_en),
211   .seed_o(seed),
212   .cnt_err_o(reseed_cnt_err)
213 );
214
215 logic [63:0] lfsr;
216 logic ctrl_lfsr_en, data_lfsr_en, sideload_lfsr_en;
217 assign lfsr_en = ctrl_lfsr_en | data_lfsr_en | side-
load_lfsr_en;
218
219 prim_lfsr #(
220   .LfsrDw(LfsrWidth),
221   .StateOutDw(LfsrWidth),
222   .DefaultSeed(RndCnstLfsrSeed),
223   .StatePermEn(1'b1),
224   .StatePerm(RndCnstLfsrPerm),
225   .NonLinearOut(1'b1)
226 ) u_lfsr (
227   .clk_i,
228   .rst_ni,
229   .lfsr_en_i(lfsr_en),
230   .seed_en_i(seed_en),
231   .seed_i(seed),
232   .entropy_i('0),
233   .state_o(lfsr)
234 );
235 `ASSERT_INIT(LfsrWidth_A, LfsrWidth == 64)
236
237
238 logic [Shares-1:0][RandWidth-1:0] ctrl_rand;
239 logic [Shares-1:0][RandWidth-1:0] data_rand;
240
241 assign ctrl_rand[0] = lfsr[63:32];
242 assign ctrl_rand[1] = perm_data(lfsr[31:0], RndCnstRand-
Perm);
243
244 assign data_rand[0] = lfsr[31:0];
245 assign data_rand[1] = perm_data(lfsr[63:32], RndCnstRand-
Perm);
246
247 //////////////////////////////////////////////////
248 // Key Manager Control
249 //////////////////////////////////////////////////
250
251 keymgr_stage_e stage_sel;
252 logic invalid_stage_sel;
253 prim_mubi_pkg::mubi4_t hw_key_sel;
254 logic adv_en, id_en, gen_en;
255 logic wipe_key;
256 hw_key_req_t kmac_key;
257 logic op_done;
258 logic init;
259 logic data_valid;
260 logic data_hw_en;
261 logic data_sw_en;
262 logic kmac_done;
263 logic kmac_input_invalid;
264 logic kmac_cmd_err;
265 logic kmac_fsm_err;
266 logic kmac_op_err;
267 logic kmac_done_err;
268 logic [Shares-1:0][kmac_pkg::AppDigestW-1:0] kmac_data;
269 logic [Shares-1:0][KeyWidth-1:0] kmac_data_truncated;
270 logic [ErrLastPos-1:0] err_code;
271 logic [FaultLastPos-1:0] fault_code;
272 logic sw_binding_unlock;
273 logic [CdiWidth-1:0] cdi_sel;
274 logic sideload_fsm_err;
275 logic sideload_sel_err;
276
277 for (genvar i = 0; i < Shares; i++) begin : gen_trun-
cate_data
278   assign kmac_data_truncated[i] = kmac_data[i][Key-
Width-1:0];
279 end
280
281 logic op_start;

```

```

282 assign op_start = reg2hw.start.q;
283 keymgr_ctrl #(
284   .KmacEnMasking(KmacEnMasking)
285 ) u_ctrl (
286   .clk_i,
287   .rst_ni,
288   .en_i(lc_tx_test_true_strict(lc_keymgr_en[KeyMgrEnCtrl])),
289   .regfile_intg_err_i(regfile_intg_err),
290   .shadowed_update_err_i(shadowed_update_err),
291   .shadowed_storage_err_i(shadowed_storage_err),
292   .reseed_cnt_err_i(reseed_cnt_err),
293   .sideload_sel_err_i(sideload_sel_err),
294   .sideload_fsm_err_i(sideload_fsm_err),
295   .prng_reseed_req_o(reseed_req),
296   .prng_reseed_ack_i(reseed_ack),
297   .prng_reseed_done_i(reseed_done),
298   .prng_en_o(ctrl_lfsr_en),
299   .entropy_i(ctrl_rand),
300   .op_i(keymgr_ops_e'(reg2hw.control_shadowed.operation.q)),
301   .op_start_i(op_start),
302   .op_cdi_sel_i(reg2hw.control_shadowed.cdi_sel.q),
303   .op_done_o(op_done),
304   .init_o(init),
305   .sw_binding_unlock_o(sw_binding_unlock),
306   .status_o(hw2reg.op_status.d),
307   .fault_o(fault_code),
308   .error_o(err_code),
309   .data_hw_en_o(data_hw_en),
310   .data_sw_en_o(data_sw_en),
311   .data_valid_o(data_valid),
312   .working_state_o(hw2reg.working_state.d),
313   .root_key_i(otp_key_i),
314   .hw_sel_o(hw_key_sel),
315   .stage_sel_o(stage_sel),
316   .invalid_stage_sel_o(invalid_stage_sel),
317   .cdi_sel_o(cdi_sel),
318   .wipe_key_o(wipe_key),
319   .adv_en_o(adv_en),
320   .id_en_o(id_en),
321   .gen_en_o(gen_en),
322   .key_o(kmac_key),
323   .kmac_done_i(kmac_done),
324   .kmac_input_invalid_i(kmac_input_invalid),
325   .kmac_fsm_err_i(kmac_fsm_err),
326   .kmac_op_err_i(kmac_op_err),
327   .kmac_done_err_i(kmac_done_err),
328   .kmac_cmd_err_i(kmac_cmd_err),
329   .kmac_data_i(kmac_data_truncated)
330 );
331
332 assign hw2reg.start.d = '0;
333 assign hw2reg.start.de = op_done;
334 // as long as operation is ongoing, capture status
335 assign hw2reg.op_status.de = op_start;
336
337 // working state is always visible
338 assign hw2reg.working_state.de = 1'b1;
339
340 logic cfg_regwen;
341
342 // key manager registers cannot be changed once an operation
starts
343 keymgr_cfg_en u_cfgen (
344   .clk_i,
345   .rst_ni,
346   .init_i(1'b1), // cfg_regwen does not care about init
347   .en_i(lc_tx_test_true_strict(lc_keymgr_en[KeyMgrEnCfgEn]))
348 ,
349   .set_i(op_start & op_done),
350   .clr_i(op_start),
351   .out_o(cfg_regwen)
352 );
353
354 assign hw2reg.cfg_regwen.d = cfg_regwen;
355
356 logic sw_binding_clr;
357 logic sw_binding_regwen;
358
359 // this is w0c
360 assign sw_binding_clr = reg2hw.sw_binding_regwen.qe &
~reg2hw.sw_binding_regwen.q;
361
362 // software clears the enable
363 // hardware restores it upon successful advance
364 keymgr_cfg_en #(
365   .NonInitClr(1'b1) // clear has an effect regardless of
init state
366 ) u_sw_binding_regwen (
367   .clk_i,
368   .rst_ni,
369   .init_i(init),
370   .en_i(lc_tx_test_true_strict(lc_keymgr_en[KeyMgrEnSwBindi-
ngEn])),
371   .set_i(sw_binding_unlock),
372   .clr_i(sw_binding_clr),
373   .out_o(sw_binding_regwen)
374 );
375
376 assign hw2reg.sw_binding_regwen.d = sw_binding_regwen &

```

```

cfg_regwen;
377
378 //////////////////////////////////////////////////
379 // Key Manager Input Construction
380 //////////////////////////////////////////////////
381
382 // The various arrays of inputs for each operation
383 logic rom_digest_vld;
384 logic [2**StageWidth-1:0][AdvDataWidth-1:0] adv_matrix;
385 logic [2**StageWidth-1:0] adv_dvalid;
386 logic [2**StageWidth-1:0][IdDataWidth-1:0] id_matrix;
387 logic [GenDataWidth-1:0] gen_in;
388
389 // The max key version for each stage
390 logic [2**StageWidth-1:0][31:0] max_key_versions;
391
392 // Number of times the lfsr output fits into the inputs
393 localparam int AdvLfsrCopies = AdvDataWidth / 32;
394 localparam int IdLfsrCopies = IdDataWidth / 32;
395 localparam int GenLfsrCopies = GenDataWidth / 32;
396
397 // input checking
398 logic creator_seed_vld;
399 logic owner_seed_vld;
400 logic devid_vld;
401 logic health_state_vld;
402 logic key_version_vld;
403
404 // software binding
405 logic [SwBindingWidth-1:0] sw_binding;
406 assign sw_binding = (cdi_sel == 0) ?
reg2hw.sealing_sw_binding :
407 (cdi_sel == 1) ?
reg2hw.attest_sw_binding : RndCnstCdi;
408
409 // Advance state operation input construction
410 for (genvar i = KeyMgrStages; i < 2**StageWidth; i++) be-
gin : gen_adv_matrix_fill
411 assign adv_matrix[i] = {AdvLfsrCopies{data_rand[0]}};
412 assign adv_dvalid[i] = '1'b1;
413 end
414
415 // Advance to creator_root_key
416 // The values coming from otp_ctrl / lc_ctrl are treat as
quasi-static for CDC purposes
417 logic [KeyWidth-1:0] creator_seed;
418 logic unused_creator_seed;
419 if (UseOtpSeedsInsteadOfFlash) begin : gen_otp_creator_seed
420 assign unused_creator_seed =
^({flash_i.seeds[flash_ctrl_pkg::CreatorSeedIdx],
421 otp_key_i.creat-
or_seed_vld});
422 assign creator_seed = otp_key_i.creator_seed;
423 end else begin : gen_flash_creator_seed
424 assign unused_creator_seed = ^{otp_key_i.creator_seed,
425 otp_key_i.creat-
or_seed_vld};
426 assign creator_seed =
flash_i.seeds[flash_ctrl_pkg::CreatorSeedIdx];
427 end
428 // TODO(opentitan-integrated/issues/251):
429 // replace below code with commented code once SW and DV
model can handle multiple
430 // // ROM_CTRL digests.
431 // logic [KeyWidth*NumRomDigestInputs-1:0] rom_digests;
432 // always_comb begin
433 // rom_digests = '0;
434 // for (int k = 0; k < NumRomDigestInputs; k++) begin
435 // rom_digests[KeyWidth*k +: KeyWidth] = rom_di-
gest_i[k].data;
436 // end
437 // end
438 // assign adv_matrix[Creator] = AdvDataWidth'({sw_binding,
439 //
otp_device_id_i,
440 // lc_keym-
gr_div_i,
441 // rom_digests,
442 // revi-
sion_seed});
443 assign adv_matrix[Creator] = AdvDataWidth'({sw_binding,
444 otp_device_id_i,
445 lc_keymgr_div_i,
446 rom_di-
gest_i.data,
447 revision_seed});
448
449 assign adv_dvalid[Creator] = creator_seed_vld &
450 devid_vld &
451 health_state_vld &
452 rom_digest_vld;
453
454 // Advance to owner_intermediate_key
455 logic [KeyWidth-1:0] owner_seed;
456 logic unused_owner_seed;
457 if (UseOtpSeedsInsteadOfFlash) begin : gen_otp_owner_seed
458 assign unused_owner_seed =
^({flash_i.seeds[flash_ctrl_pkg::OwnerSeedIdx],
459 otp_key_i.owner_seed_vld});
460 assign owner_seed = otp_key_i.owner_seed;
461 end else begin : gen_flash_owner_seed
462 assign unused_owner_seed = ^{otp_key_i.owner_seed,
463 otp_key_i.owner_seed_vld};
464 assign owner_seed =
flash_i.seeds[flash_ctrl_pkg::OwnerSeedIdx];
465 end
466
467 assign adv_dvalid[OwnerInt] = owner_seed_vld;
468
469 // Advance to owner_key
470 assign adv_matrix[Owner] = AdvDataWidth'({sw_binding, own-
er_seed});
471 assign adv_dvalid[Owner] = '1'b1;
472
473 // Generate Identity operation input construction
474 for (genvar i = KeyMgrStages; i < 2**StageWidth; i++) be-
gin : gen_id_matrix_fill
475 assign id_matrix[i] = {IdLfsrCopies{data_rand[0]}};
476 end
477
478 assign id_matrix[Creator] = creator_identity_seed;
479 assign id_matrix[OwnerInt] = owner_int_identity_seed;
480 assign id_matrix[Owner] = owner_identity_seed;
481
482 // Generate output operation input construction
483 logic [KeyWidth-1:0] output_key;
484 keymgr_key_dest_e dest_sel;
485 logic [KeyWidth-1:0] dest_seed;
486
487 assign dest_sel = keym-
gr_key_dest_e'(reg2hw.control_shadowed.dest_sel.q);
488 assign dest_seed = dest_sel == Aes ? aes_seed :
489 dest_sel == Kmac ? kmac_seed :
490 dest_sel == Otbn ? otbn_seed :
none_seed;
491
492 assign output_key = mubi4_test_true_strict(hw_key_sel) ?
hard_output_seed :
493 soft_output_seed;
494 assign gen_in = invalid_stage_sel ? {GenLfsrCop-
ies{lfsr[31:0]}} : {reg2hw.key_version,
495
reg2hw.salt,
496
dest_seed,
497
output_key};
498
499 // Advance state operation input construction
500 for (genvar i = KeyMgrStages; i < 2**StageWidth; i++) be-
gin : gen_key_version_fill
501 assign max_key_versions[i] = '0;
502 end
503
504 assign max_key_versions[Creator] =
reg2hw.max_creator_key_ver_shadowed.q;
505 assign max_key_versions[OwnerInt] =
reg2hw.max_owner_int_key_ver_shadowed.q;
506 assign max_key_versions[Owner] =
reg2hw.max_owner_key_ver_shadowed.q;
507
508 logic [KeyVersionWidth-1:0] cur_max_key_version;
509 assign cur_max_key_version = max_key_versions[stage_sel];
510
511 // General module for checking inputs
512 logic key_vld;
513 // SEC_CM: CONSTANTS.CONSISTENCY
514 // SEC_CM: INTERSIG.CONSISTENCY
515 keymgr_input_checks #(
516 .KmacEnMasking(KmacEnMasking),
517 .NumRomDigestInputs(NumRomDigestInputs)
518 ) u_checks (
519 .rom_digest_i,
520 .cur_max_key_version_i(cur_max_key_version),
521 .key_version_i(reg2hw.key_version),
522 .creator_seed_i(creator_seed),
523 .owner_seed_i(owner_seed),
524 .key_i(kmac_key_o),
525 .devid_i(otp_device_id_i),
526 .health_state_i(HealthStateWidth'(lc_keymgr_div_i)),
527 .creator_seed_vld_o(creator_seed_vld),
528 .owner_seed_vld_o(owner_seed_vld),
529 .devid_vld_o(devid_vld),
530 .health_state_vld_o(health_state_vld),
531 .key_version_vld_o(key_version_vld),
532 .key_vld_o(key_vld),
533 .rom_digest_vld_o(rom_digest_vld)
534 );
535
536 assign hw2reg.debug.invalid_creator_seed.d = '1'b1;
537 assign hw2reg.debug.invalid_owner_seed.d = '1'b1;
538 assign hw2reg.debug.invalid_dev_id.d = '1'b1;
539 assign hw2reg.debug.invalid_health_state.d = '1'b1;
540 assign hw2reg.debug.invalid_key_version.d = '1'b1;
541 assign hw2reg.debug.invalid_key.d = '1'b1;
542 assign hw2reg.debug.invalid_digest.d = '1'b1;
543
544 logic valid_op;

```



```

545 assign valid_op = adv_en | id_en | gen_en;
546 assign hw2reg.debug.invalid_creator_seed.de = adv_en &
(stage_sel == Creator) & ~creator_seed_vld;
547 assign hw2reg.debug.invalid_owner_seed.de = adv_en &
(stage_sel == OwnerInt) & ~owner_seed_vld;
548 assign hw2reg.debug.invalid_dev_id.de = adv_en & (stage_sel
== Creator) & ~dev_id_vld;
549 assign hw2reg.debug.invalid_health_state.de = adv_en &
(stage_sel == Creator) & ~health_state_vld;
550 assign hw2reg.debug.invalid_key_version.de = gen_en &
~key_version_vld;
551 assign hw2reg.debug.invalid_key.de = valid_op & ~key_vld;
552 assign hw2reg.debug.invalid_digest.de = adv_en & (stage_sel
== Creator) & ~rom_digest_vld;
553
554 ///////////////////////////////////////////////////
555 // KMAC Control
556 ///////////////////////////////////////////////////
557
558 logic [3:0] invalid_data;
559 assign invalid_data[0pAdvance] = ~key_vld | ~adv_dval-
id[stage_sel];
560 assign invalid_data[0pGenId] = ~key_vld;
561 assign invalid_data[0pGenSwOut] = ~key_vld | ~key_ver-
sion_vld;
562 assign invalid_data[0pGenHwOut] = ~key_vld | ~key_ver-
sion_vld;
563
564 keymgr_kmac_if #(
565   .RndCnstRandPerm(RndCnstRandPerm)
566 ) u_kmac_if (
567   .clk_i,
568   .rst_ni,
569   .prng_en_o(data_lfsr_en),
570   .adv_data_i(adv_matrix[stage_sel]),
571   .id_data_i(id_matrix[stage_sel]),
572   .gen_data_i(gen_in),
573   .inputs_invalid_i(invalid_data),
574   .inputs_invalid_o(kmac_input_invalid),
575   .adv_en_i(adv_en),
576   .id_en_i(id_en),
577   .gen_en_i(gen_en),
578   .done_o(kmac_done),
579   .data_o(kmac_data),
580   .kmac_data_o,
581   .kmac_data_i,
582   .entropy_i(data_rand),
583   .fsm_error_o(kmac_fsm_err),
584   .kmac_error_o(kmac_op_err),
585   .kmac_done_error_o(kmac_done_err),
586   .cmd_error_o(kmac_cmd_err)
587 );
588
589
590 ///////////////////////////////////////////////////
591 // Side load key storage
592 ///////////////////////////////////////////////////
593 // SEC_CM: HW.KEY.SW.NOACCESS
594 keymgr_sideload_key_ctrl u_sideload_ctrl (
595   .clk_i,
596   .rst_ni,
597   .init_i(init),
598   .entropy_i(data_rand),
599   .clr_key_i(keym-
gr_sideload_clr_e'(reg2hw_sideload_clear.q)),
600   .wipe_key_i(wipe_key),
601   .dest_sel_i(dest_sel),
602   .hw_key_sel_i(hw_key_sel),
603   // SEC_CM: OUTPUT_KEYS.CTRL.REDUN
604   .data_en_i(data_hw_en),
605   .data_valid_i(data_valid),
606   .key_i(kmac_key),
607   .data_i(kmac_data),
608   .prng_en_o(sideload_lfsr_en),
609   .aes_key_o,
610   .otbn_key_o,
611   .kmac_key_o,
612   .sideload_sel_err_o(sideload_sel_err),
613   .fsm_err_o(sideload_fsm_err)
614 );
615
616 for (genvar i = 0; i < 8; i++) begin : gen_sw_assigns
617
618   prim_mubi_pkg::mubi4_t [1:0] hw_key_sel_buf;
619   prim_mubi4_sync #(
620     .NumCopies(2),
621     .AsyncOn(0)
622   ) u_mubi_buf (
623     .clk_i,
624     .rst_ni,
625     .mubi_i(hw_key_sel),
626     .mubi_o(hw_key_sel_buf)
627   );
628
629   // SEC_CM: OUTPUT_KEYS.CTRL.REDUN
630   prim_sec_anchor_buf #(
631     .Width(32)
632   ) u_prim_buf_share0_d (
633     .in_i(~data_sw_en | wipe_key ? data_rand[0] :
kmac_data[0][i*32 +: 32]),
634     .out_o(hw2reg.sw_share0_output[i].d)

```

```

635   );
636
637   prim_sec_anchor_buf #(
638     .Width(32)
639   ) u_prim_buf_share1_d (
640     .in_i(~data_sw_en | wipe_key ? data_rand[1] :
kmac_data[1][i*32 +: 32]),
641     .out_o(hw2reg.sw_share1_output[i].d)
642   );
643
644   prim_sec_anchor_buf #(
645     .Width(1)
646   ) u_prim_buf_share0_de (
647     .in_i(wipe_key | data_valid & mu-
bi4_test_false_strict(hw_key_sel_buf[0])),
648     .out_o(hw2reg.sw_share0_output[i].de)
649   );
650
651   prim_sec_anchor_buf #(
652     .Width(1)
653   ) u_prim_buf_share1_de (
654     .in_i(wipe_key | data_valid & mu-
bi4_test_false_strict(hw_key_sel_buf[1])),
655     .out_o(hw2reg.sw_share1_output[i].de)
656   );
657 end
658
659 ///////////////////////////////////////////////////
660 // Alerts and Interrupts
661 ///////////////////////////////////////////////////
662
663 prim_intr_hw #(Width(1)) u_intr_op_done (
664   .clk_i,
665   .rst_ni,
666   .event_intr_i (op_done),
667   .reg2hw_intr_enable_q_i (reg2hw_intr_enable.q),
668   .reg2hw_intr_test_q_i (reg2hw_intr_test.q),
669   .reg2hw_intr_test_qe_i (reg2hw_intr_test.qe),
670   .reg2hw_intr_state_q_i (reg2hw_intr_state.q),
671   .hw2reg_intr_state_de_o (hw2reg_intr_state.de),
672   .hw2reg_intr_state_d_o (hw2reg_intr_state.d),
673   .intr_o (intr_op_done_o)
674 );
675
676 assign hw2reg.err_code.invalid_op.d = 1'b1;
677 assign hw2reg.err_code.invalid_kmac_input.d = 1'b1;
678 assign hw2reg.err_code.invalid_shadow_update.d = 1'b1;
679 assign hw2reg.err_code.invalid_op.de =
err_code[ErrInvalidOp];
680 assign hw2reg.err_code.invalid_kmac_input.de =
err_code[ErrInvalidIn];
681 assign hw2reg.err_code.invalid_shadow_update.de =
err_code[ErrShadowUpdate];
682
683 assign hw2reg.fault_status.cmd.de =
fault_code[FaultKmacCmd];
684 assign hw2reg.fault_status.kmac_fsm.de =
fault_code[FaultKmacFsm];
685 assign hw2reg.fault_status.kmac_op.de =
fault_code[FaultKmacOp];
686 assign hw2reg.fault_status.kmac_done.de =
fault_code[FaultKmacDone];
687 assign hw2reg.fault_status.kmac_out.de =
fault_code[FaultKmacOut];
688 assign hw2reg.fault_status.regfile_intg.de =
fault_code[FaultRegIntg];
689 assign hw2reg.fault_status.shadow.de =
fault_code[FaultShadow];
690 assign hw2reg.fault_status.ctrl_fsm_intg.de =
fault_code[FaultCtrlFsm];
691 assign hw2reg.fault_status.ctrl_fsm_chk.de =
fault_code[FaultCtrlFsmChk];
692 assign hw2reg.fault_status.ctrl_fsm_cnt.de =
fault_code[FaultCtrlCnt];
693 assign hw2reg.fault_status.reseed_cnt.de =
fault_code[FaultReseedCnt];
694 assign hw2reg.fault_status.side_ctrl_fsm.de =
fault_code[FaultSideFsm];
695 assign hw2reg.fault_status.side_ctrl_sel.de =
fault_code[FaultSideSel];
696 assign hw2reg.fault_status.key_ecc.de =
fault_code[FaultKeyEcc];
697 assign hw2reg.fault_status.cmd.d = 1'b1;
698 assign hw2reg.fault_status.kmac_fsm.d = 1'b1;
699 assign hw2reg.fault_status.kmac_done.d = 1'b1;
700 assign hw2reg.fault_status.kmac_op.d = 1'b1;
701 assign hw2reg.fault_status.kmac_out.d = 1'b1;
702 assign hw2reg.fault_status.regfile_intg.d = 1'b1;
703 assign hw2reg.fault_status.shadow.d = 1'b1;
704 assign hw2reg.fault_status.ctrl_fsm_intg.d = 1'b1;
705 assign hw2reg.fault_status.ctrl_fsm_chk.d = 1'b1;
706 assign hw2reg.fault_status.ctrl_fsm_cnt.d = 1'b1;
707 assign hw2reg.fault_status.reseed_cnt.d = 1'b1;
708 assign hw2reg.fault_status.side_ctrl_fsm.d = 1'b1;
709 assign hw2reg.fault_status.side_ctrl_sel.d = 1'b1;
710 assign hw2reg.fault_status.key_ecc.d = 1'b1;
711
712 // There are two types of alerts
713 // - alerts for hardware errors, these could not have been
generated by software.
714 // - alerts for errors that may have been generated by soft-

```



```

ware.
715
716 logic fault_errs, fault_err_req_q, fault_err_req_d,
fault_err_ack;
717 logic op_errs, op_err_req_q, op_err_req_d, op_err_ack;
718
719 // Fault status can happen independently of any operation
720 assign fault_errs = |reg2hw.fault_status;
721
722 assign fault_err_req_d = fault_errs ? 1'b1 :
723 fault_err_ack ? 1'b0 :
fault_err_req_q;
724
725 assign op_errs = |err_code;
726 assign op_err_req_d = op_errs ? 1'b1 :
727 op_err_ack ? 1'b0 : op_err_req_q;
728
729 always_ff @(posedge clk_i or negedge rst_ni) begin
730 if (!rst_ni) begin
731 fault_err_req_q <= '0;
732 op_err_req_q <= '0;
733 end else begin
734 fault_err_req_q <= fault_err_req_d;
735 op_err_req_q <= op_err_req_d;
736 end
737 end
738
739 logic fault_alert_test;
740 assign fault_alert_test =
reg2hw.alert_test.fatal_fault_err.q &
741
reg2hw.alert_test.fatal_fault_err.qe;
742 prim_alert_sender #(
743 .AsyncOn(AlertAsyncOn[1]),
744 .SkewCycles(AlertSkewCycles),
745 .IsFatal(1)
746 ) u_fault_alert (
747 .clk_i,
748 .rst_ni,
749 .alert_test_i(fault_alert_test),
750 .alert_req_i(fault_err_req_q),
751 .alert_ack_o(fault_err_ack),
752 .alert_state_o(),
753 .alert_rx_i(alert_rx_i[1]),
754 .alert_tx_o(alert_tx_o[1])
755 );
756
757 logic op_err_alert_test;
758 assign op_err_alert_test =
reg2hw.alert_test.recov_operation_err.q &
759
reg2hw.alert_test.recov_operation_err.qe;
760 prim_alert_sender #(
761 .AsyncOn(AlertAsyncOn[0]),
762 .SkewCycles(AlertSkewCycles),
763 .IsFatal(0)
764 ) u_op_err_alert (
765 .clk_i,
766 .rst_ni,
767 .alert_test_i(op_err_alert_test),
768 .alert_req_i(op_err_req_q),
769 .alert_ack_o(op_err_ack),
770 .alert_state_o(),
771 .alert_rx_i(alert_rx_i[0]),
772 .alert_tx_o(alert_tx_o[0])
773 );
774
775 // known asserts
776 `ASSERT_KNOWN(TlDValidKnown0_A, tl_o.d_valid)
777 `ASSERT_KNOWN(TlAReadyKnown0_A, tl_o.a_ready)
778 `ASSERT_KNOWN(IntrKnown0_A, intr_op_done_o)
779 `ASSERT_KNOWN(AlertKnown0_A, alert_tx_o)
780
781 `ASSERT_KNOWN(AesKeyKnown0_A, aes_key_o)
782 `ASSERT_KNOWN(KmacKeyKnown0_A, kmac_key_o)
783 `ASSERT_KNOWN(OtbnKeyKnown0_A, otbn_key_o)
784 `ASSERT_KNOWN(KmacDataKnown0_A, kmac_data_o)
785
786
787 // kmac parameter consistency
788 // Both modules must be consistent with regards to masking
assumptions
789 logic unused_kmac_en_masking;
790 assign unused_kmac_en_masking = kmac_en_masking_i;
791
792 `ASSERT_INIT_NET(KmacMaskCheck_A, KmacEnMasking ==
kmac_en_masking_i)
793
794 // Ensure all parameters are consistent
795 `ASSERT_INIT(FaultCntMatch_A, FaultLastPos == AsyncFault-
LastIdx + SyncFaultLastIdx)
796 `ASSERT_INIT(ErrCntMatch_A, ErrLastPos == AsyncErrLastIdx +
SyncErrLastIdx)
797 `ASSERT_INIT(StageMatch_A, KeyMgrStages == Disable)
798
799 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CtrlCntAlertCheck_A,
u_ctrl.u_cnt, alert_tx_o[1])
800 `AS-
SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(KmacIfCntAlertCheck_A, u_k-
mac_if.u_cnt, alert_tx_o[1])
801 `AS-

```

```

SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(ReseedCtrlCntAlertCheck_A, u_re-
seed_ctrl.u_reseed_cnt,
802 alert_tx_o[1])
803 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlMainFsmCheck_A,
u_ctrl.u_state_regs, alert_tx_o[1])
804 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlDataFsmCheck_A,
805 u_ctrl.u_data_en.u_state_regs, alert_tx_o[1])
806 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlOpFsmCheck_A,
807 u_ctrl.u_op_state.u_state_regs, alert_tx_o[1])
808 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(KmacIfFsmCheck_A, u_k-
mac_if.u_state_regs, alert_tx_o[1])
809 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(SideLoadCtrlFsmCheck_A,
810 u_side_load_ctrl.u_state_regs, alert_tx_o[1])
811
812 // Alert assertions for reg_we onehot check
813 `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
u_reg, alert_tx_o[1])
814 endmodule // keymgr

```

2. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_cfg_en.sv (63 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Key manager CFGEN
6 // TBD This should be enhanced in the future to contain a
shadow copy
7
8 `include "prim_assert.sv"
9
10 module keymgr_cfg_en #(
11 // controls whether clear has an effect on output value dur-
ing non-init
12 parameter bit NonInitClr = 1'b1
13 ) (
14 input clk_i,
15 input rst_ni,
16 input init_i,
17 input en_i,
18 input set_i,
19 input clr_i,
20 output logic out_o
21 );
22
23 logic out_q;
24 logic init_q;
25
26 logic vld_clr;
27 logic vld_set;
28 logic vld_dis;
29
30 assign vld_clr = init_q && clr_i;
31 assign vld_set = init_q && set_i;
32 assign vld_dis = init_q && !en_i;
33
34 // the same cycle where clear is asserted should already
block future
35 // configuration
36 logic out_clr;
37 assign out_clr = NonInitClr ? clr_i : vld_clr;
38 assign out_o = ~out_clr & out_q & en_i;
39
40 always_ff @(posedge clk_i or negedge rst_ni) begin
41 if (!rst_ni) begin
42 init_q <= '0;
43 end else if (init_q && !en_i) begin
44 init_q <= '0;
45 end else if (init_i && en_i) begin
46 init_q <= 1'b1;
47 end
48 end
49
50 // clearing the configure enable always has higher priority
than setting
51 always_ff @(posedge clk_i or negedge rst_ni) begin
52 if (!rst_ni) begin
53 out_q <= 1'b1;
54 end else if (vld_dis) begin
55 out_q <= 1'b0;
56 end else if (vld_set) begin
57 out_q <= 1'b1;
58 end else if (out_clr) begin
59 out_q <= 1'b0;
60 end
61 end
62
63 endmodule // keymgr_cfg_en

```

3. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_ctrl.sv (897 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for

```

```

details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Key manager top level
6 //
7
8 `include "prim_assert.sv"
9
10 module keymgr_ctrl
11     import keymgr_pkg::*;
12     import keymgr_reg_pkg::*;
13     #(
14         parameter bit KmacEnMasking = 1'b1
15     ) (
16         input clk_i,
17         input rst_ni,
18
19         // lifecycle enforcement
20         // SEC_CM: CTRL.FSM.GLOBAL_ESC
21         input en_i,
22
23         // faults that can occur outside of operations
24         input regfile_intg_err_i,
25         input shadowed_update_err_i,
26         input shadowed_storage_err_i,
27         input reseed_cnt_err_i,
28         input sideload_sel_err_i,
29         input sideload_fsm_err_i,
30
31         // Software interface
32         input op_start_i,
33         input keymgr_ops_e op_i,
34         input [CdiWidth-1:0] op_cdi_sel_i,
35         output logic op_done_o,
36         output keymgr_op_status_e status_o,
37         output logic [ErrLastPos-1:0] error_o,
38         output logic [FaultLastPos-1:0] fault_o,
39         output logic data_hw_en_o,
40         output logic data_sw_en_o,
41         output logic data_valid_o,
42         output logic wipe_key_o,
43         output keymgr_working_state_e working_state_o,
44         output logic sw_binding_unlock_o,
45         output logic init_o,
46
47         // Data input
48         input otp_ctrl_pkg::otp_keymgr_key_t root_key_i,
49         output prim_mubi_pkg::mubi4_t hw_sel_o,
50         output keymgr_stage_e stage_sel_o,
51         output logic invalid_stage_sel_o,
52         output logic [CdiWidth-1:0] cdi_sel_o,
53
54         // KMAC ctrl interface
55         output logic adv_en_o,
56         output logic id_en_o,
57         output logic gen_en_o,
58         output hw_key_req_t key_o,
59         input kmac_done_i,
60         input kmac_input_invalid_i, // asserted when selected data
        fails criteria check
61         input kmac_fsm_err_i, // asserted when kmac fsm reaches un-
        expected state
62         input kmac_op_err_i, // asserted when kmac itself reports
        an error
63         input kmac_done_err_i, // asserted when kmac unexpectedly
        toggles done
64         input kmac_cmd_err_i, // asserted when more than one command
        given to kmac
65         input [Shares-1:0][KeyWidth-1:0] kmac_data_i,
66
67         // prng control interface
68         input [Shares-1:0][RandWidth-1:0] entropy_i,
69         input prng_reseed_done_i,
70         input prng_reseed_ack_i,
71         output logic prng_reseed_req_o,
72         output logic prng_en_o
73     );
74
75     localparam int EntropyWidth = LfsrWidth / 2;
76     localparam int EntropyRounds = KeyWidth / EntropyWidth;
77     localparam int EntropyRndWidth =
        prim_util_pkg::vbits(EntropyRounds);
78     localparam int CntWidth = EntropyRounds > CDIs ? EntropyRnd-
        Width : CdiWidth;
79     localparam int EccDataWidth = 64;
80     localparam int EccWidth = 8;
81     localparam int EccWords = KeyWidth / EccDataWidth;
82     localparam int TotalEccWords = EccWords * Shares * CDIs;
83
84
85     // Enumeration for working state
86     // Encoding generated with:
87     // $ ./util/design/sparse-fsm-encode.py -d 5 -m 11 -n 10 \
88     // -s 4101887575 --language=sv
89     //
90     // Hamming distance histogram
91     //
92     // 0: --
93     // 1: --
94     // 2: --
95     // 3: --
96     // 4: --
97     // 5: ||||| (54.55%)
98     // 6: ||||| (45.45%)
99     // 7: --
100    // 8: --
101    // 9: --
102    // 10: --
103
104    // Minimum Hamming distance: 5
105    // Maximum Hamming distance: 6
106    // Minimum Hamming weight: 2
107    // Maximum Hamming weight: 8
108
109    localparam int StateWidth = 10;
110    typedef enum logic [StateWidth-1:0] {
111        StCtrlReset = 10'b101100001,
112        StCtrlEntropyReSeed = 10'b1110010010,
113        StCtrlRandom = 10'b0011110100,
114        StCtrlRootKey = 10'b0110101111,
115        StCtrlInit = 10'b0100000100,
116        StCtrlCreatorRootKey = 10'b1000011101,
117        StCtrlOwnerIntKey = 10'b0001001010,
118        StCtrlOwnerKey = 10'b1101111110,
119        StCtrlDisabled = 10'b1010101000,
120        StCtrlWipe = 10'b0000110011,
121        StCtrlInvalid = 10'b1011000111
122    } state_e;
123    state_e state_q, state_d;
124
125    // A variable that represents differentiates states before
    root key and after root key.
126    logic initialized;
127
128    // There are two versions of the key state, one for sealing
    one for attestation
129    // Among each version, there are multiple shares
130    // Each share is a fixed multiple of the entropy width
131    logic [CDIs-1:0][Shares-1:0][EntropyRounds-1:0][Entropy-
        Width-1:0] key_state_d;
132    logic [CDIs-1:0][Shares-1:0][EccWords-1:0][EccDataWidth-1:0]
        key_state_ecc_words_d;
133    logic [CDIs-1:0][Shares-1:0][EccWords-1:0][EccDataWidth-1:0]
        key_state_q;
134    logic [CDIs-1:0][Shares-1:0][EccWords-1:0][EccWidth-1:0]
        key_state_ecc_q;
135    logic [CntWidth-1:0] cnt;
136    logic [CdiWidth-1:0] cdi_cnt;
137
138    // error conditions
139    logic invalid_kmac_out;
140    logic invalid_op;
141    logic cnt_err;
142    // states fall out of sparsely encoded range
143    logic state_intg_err_q, state_intg_err_d;
144
145    // General operation decode
146    //
147    //
148
149    logic adv_op, dis_op, gen_id_op, gen_sw_op, gen_hw_op,
    gen_op;
150    assign adv_op = (op_i == OpAdvance);
151    assign gen_id_op = (op_i == OpGenId);
152    assign gen_sw_op = (op_i == OpGenSwOut);
153    assign gen_hw_op = (op_i == OpGenHwOut);
154    assign dis_op = ~(op_i inside {OpAdvance, OpGenId, Op-
        GenSwOut, OpGenHwOut});
155    assign gen_op = (gen_id_op | gen_sw_op | gen_hw_op);
156
157    // interaction between software and main fsm
158    // disable is treated like an advanced call
159    logic advance_sel;
160    logic disable_sel;
161    logic gen_out_hw_sel;
162
163    assign advance_sel = op_start_i & adv_op & en_i;
164    assign gen_out_hw_sel = op_start_i & gen_hw_op & en_i;
165
166    // disable is selected whenever a normal operation is not
    set
167
168    assign disable_sel = (op_start_i & dis_op) | !en_i;
169
170    // interaction between main control fsm and operation fsm
171    // req/ack interface with op handling fsm
172    logic op_req;
173    logic op_ack;
174    logic op_update;
175    logic op_busy;
176    logic disabled;
177    logic invalid;
178
179    logic adv_req, dis_req, id_req, gen_req;

```

```

185 assign adv_req = op_req & adv_op;
186 assign dis_req = op_req & dis_op;
187 assign id_req = op_req & gen_id_op;
188 assign gen_req = op_req & (gen_sw_op | gen_hw_op);
189
190 ///////////////////////////////////////////////////
191 // interaction between operation fsm and software
192 ///////////////////////////////////////////////////
193 // categories of keymgr errors
194 logic [SyncErrLastIdx-1:0] sync_err;
195 logic [SyncFaultLastIdx-1:0] sync_fault;
196 logic [AsyncFaultLastIdx-1:0] async_fault;
197
198 logic op_err;
199 logic op_fault_err;
200
201 // unlock sw binding configuration whenever an advance call
is made without errors
202 assign sw_binding_unlock_o = adv_req & op_ack & ~(op_err |
op_fault_err);
203
204 // error definition
205 // check incoming kmac data validity
206 // Only check during the periods when there is actual kmac
output
207 assign invalid_kmac_out = (op_update | op_ack) &
208 (~valid_data_chk(kmac_data_i[0]) |
209 (~valid_data_chk(kmac_data_i[1]) &
KmacEnMasking));
210
211 // async errors have nothing to do with the operation and
thus should not
212 // impact operation results.
213 assign op_err = |sync_err;
214
215 assign op_fault_err = |{sync_fault, async_fault};
216
217 ///////////////////////////////////////////////////
218 // key update controls
219 ///////////////////////////////////////////////////
220
221 // update select can come from both main and operation fsm's
222 keymgr_key_update_e update_sel, op_update_sel;
223
224 // req from main control fsm to key update controls
225 logic wipe_req;
226 logic random_req;
227 logic random_ack;
228
229 // wipe and initialize take precedence
230 assign update_sel = wipe_req ? KeyUpdateWipe :
231 random_req ? KeyUpdateRandom :
232 init_o ? KeyUpdateRoot :
op_update_sel;
233
234 ///////////////////////////////////////////////////
235 // interaction between main fsm and prng
236 ///////////////////////////////////////////////////
237
238 // Upon entering StCtrlDisabled or StCtrlInvalid, the PRNG
is kept advancing until it has been
239 // reseeded twice (through the reseeding mechanism inside
keymgr_reseed_ctrl.sv).
240 logic [1:0] prng_en_dis_inv_d, prng_en_dis_inv_q;
241 logic prng_en_dis_inv_set;
242
243 assign prng_en_dis_inv_d =
244 prng_en_dis_inv_set ? 2'b11 :
245 prng_reseed_done_i ? {1'b0, prng_en_dis_inv_q[1]} :
prng_en_dis_inv_q;
246
247 always_ff @(posedge clk_i or negedge rst_ni) begin
248 if (!rst_ni) begin
249 prng_en_dis_inv_q <= '0;
250 end else begin
251 prng_en_dis_inv_q <= prng_en_dis_inv_d;
252 end
253 end
254
255 assign prng_en_o = random_req | wipe_req |
prng_en_dis_inv_q[0];
256
257 ///////////////////////////////////////////////////
258 // Main Control FSM
259 ///////////////////////////////////////////////////
260 // SEC_CM: CTRL.FSM.SPARSE
261 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
state_e, StCtrlReset)
262
263 always_ff @(posedge clk_i or negedge rst_ni) begin
264 if (!rst_ni) begin
265 state_intg_err_q <= '0;
266 end else begin
267 state_intg_err_q <= state_intg_err_d;
268 end
269 end
270
271 // prevents unknowns from reaching the outside world.
272 // - whatever operation causes the input data select to be

```

```

disabled should not expose the key
273 // state.
274 // - when there are no operations, the key state also should
be exposed.
275 assign key_o.valid = op_req;
276
277 assign cdi_sel_o = advance_sel ? cdi_cnt : op_cdi_sel_i;
278
279 assign invalid_stage_sel_o = ~(stage_sel_o inside {Creator,
OwnerInt, Owner});
280 for (genvar i = 0; i < Shares; i++) begin :
gen_key_out_assign
281 assign key_o.key[i] = invalid_stage_sel_o ?
282 {EntropyRounds(entropy_i[i])} :
283 key_state_q[cdi_sel_o][i];
284 end
285
286
287 //SEC_CM: CTRL.KEY.INTEGRITY
288 assign key_state_ecc_words_d = key_state_d;
289 always_ff @(posedge clk_i or negedge rst_ni) begin
290 if (!rst_ni) begin
291 key_state_q <= '0;
292 key_state_ecc_q <= {TotalEc-
cWords{prim_secdded_pkg::SecddedInv7264ZeroEcc}};
293 end else begin
294 for (int i = 0; i < CDIs; i++) begin
295 for (int j = 0; j < Shares; j++) begin
296 for (int k = 0; k < EccWords; k++) begin
297 {key_state_ecc_q[i][j][k], key_state_q[i][j][k]}
<=
298 prim_secdded_pkg::prim_secdded_inv_72_64_enc(key_state_ecc_words_d[i]
[j][k]);
299 end
300 end
301 end
302 end
303 end
304
305 logic [CDIs-1:0][Shares-1:0][EccWords-1:0] ecc_errs;
306 for (genvar i = 0; i < CDIs; i++) begin : gen_ecc_loop_cdi
307 for (genvar j = 0; j < Shares; j++) begin :
gen_ecc_loop_shares
308 for (genvar k = 0; k < EccWords; k++) begin :
gen_ecc_loop_words
309 logic [1:0] errs;
310 prim_secdded_inv_72_64_dec u_dec (
311 .data_i({key_state_ecc_q[i][j][k], key_state_q[i][j]
[k]}),
312 .data_o(),
313 .syndrome_o(),
314 .err_o(errs)
315 );
316 assign ecc_errs[i][j][k] = |errs;
317 end
318 end
319 end
320
321 // These are consumed one level above in keymgr.sv
322 logic unused_otp_sigs;
323 assign unused_otp_sigs = ^{root_key_i.creator_seed,
root_key_i.creator_seed_valid,
root_key_i.owner_seed,
root_key_i.owner_seed_valid};
324
325 // root key valid sync
326 logic root_key_valid_q;
327
328 prim_flop_2sync # (
329 .Width(1)
330 ) u_key_valid_sync (
331 .clk_i,
332 .rst_ni,
333 // Both valid signals are flopped in OTP_CTRL, and they
only ever
334 transition from 0 -> 1.
335 // It is hence ok to AND them here before the synchron-
izer, since we don't expect this
336 // to create glitches.
337 .d_i(root_key_i.creator_root_key_share0_valid &&
root_key_i.creator_root_key_share1_valid),
338 .q_o(root_key_valid_q)
339 );
340
341 // Do not let the count toggle unless an advance operation
is
342 // selected
343 assign cdi_cnt = op_req ? cnt[CDiWidth-1:0] : '0;
344
345 always_comb begin
346 key_state_d = key_state_q;
347 data_valid_o = 1'b0;
348 wipe_key_o = 1'b0;
349
350 // if a wipe request arrives, immediately destroy the
// keys regardless of current state
351 unique case (update_sel)
352 KeyUpdateRandom: begin
353 for (int i = 0; i < CDIs; i++) begin
354 for (int j = 0; j < Shares; j++) begin

```

```

359          // Load each share with the same randomness so we
can
360          // later simply XOR root key on them
361          key_state_d[i][j][cnt[EntropyRndWidth-1:0]] = en-
tropy_i[i];
362      end
363  end
364  end
365
366  KeyUpdateRoot: begin
367      if (root_key_valid_q) begin
368          for (int i = 0; i < CDIs; i++) begin
369              if (KmacEnMasking) begin : gen_two_share_key
370                  key_state_d[i][0] ^= root_key_i.creat-
or_root_key_share0;
371                  key_state_d[i][1] ^= root_key_i.creat-
or_root_key_share1;
372              end else begin : gen_one_share_key
373                  key_state_d[i][0] = root_key_i.creat-
or_root_key_share0 ^
374                      root_key_i.creat-
or_root_key_share1;
375                  key_state_d[i][1] = '0;
376              end
377          end
378      end else begin
379          // if root key is not valid, load and invalid value
380          for (int i = 0; i < CDIs; i++) begin
381              key_state_d[i][0] = '0;
382              key_state_d[i][1] = '{default: '1;
383          end
384      end
385  end
386
387  KeyUpdateKmac: begin
388      data_valid_o = gen_op;
389      key_state_d[cdi_sel_o] = (adv_op || dis_op) ?
kmac_data_i : key_state_q[cdi_sel_o];
390  end
391
392  KeyUpdateWipe: begin
393      wipe_key_o = 'b1;
394      for (int i = 0; i < CDIs; i++) begin
395          for (int j = 0; j < Shares; j++) begin
396              key_state_d[i][j] = {EntropyRounds[entropy_i[j]]};
397          end
398      end
399  end
400
401  default;;
402  endcase // unique case (update_sel)
403  end
404
405  // SEC_CM: CTRL.CTR.REDUN
406  prim_count #(
407      .Width(CntWidth)
408  ) u_cnt (
409      .clk_i,
410      .rst_ni,
411      .clr_i(op_ack | random_ack),
412      .set_i('0),
413      .set_cnt_i('0),
414      .incr_en_i(op_update | random_req),
415      .decr_en_i(1'b0),
416      .step_i(CntWidth'(1'b1)),
417      .commit_i(1'b1),
418      .cnt_o(cnt),
419      .cnt_after_commit_o(),
420      .err_o(cnt_err)
421  );
422
423  prim_mubi4_sender u_hw_sel (
424      .clk_i,
425      .rst_ni,
426      .mubi_i
427      (prim_mubi_pkg:mubi4_bool_to_mubi(gen_out_hw_sel)),
428      .mubi_o (hw_sel_o)
429  );
430
431  // when in a state that accepts commands, look at op_ack for
completion
432  // when in a state that does not accept commands, wait for
other triggers.
433  assign op_done_o = op_req ? op_ack :
434      (init_o | invalid_op);
435
436  // There are 3 possibilities
437  // advance to next state (software command)
438  // advance to disabled state (software command)
439  // advance to invalid state (detected fault)
440  logic adv_state;
441  logic dis_state;
442  logic inv_state;
443  assign adv_state = op_ack & adv_req & ~op_err;
444  assign dis_state = op_ack & dis_req;
445
446  // SEC_CM: CTRL.FSM.LOCAL_ESC
447  // begin invalidation when faults are observed.

```

```

449  // sync faults only invalidate on transaction boundaries
450  // async faults begin invalidating immediately
451  assign inv_state = |fault_o;
452
453  always_comb begin
454      // persistent data
455      state_d = state_q;
456
457      // request to op handling
458      op_req = 1'b0;
459      random_req = 1'b0;
460      random_ack = 1'b0;
461
462      // request to key updates
463      wipe_req = 1'b0;
464
465      // invalid operation issued
466      invalid_op = '0;
467
468      // data update and select signals
469      stage_sel_o = Disable;
470
471      // indication that state is disabled
472      disabled = 1'b0;
473
474      // indication that state is invalid
475      invalid = 1'b0;
476
477      // Don't request final PRNG updating and reseeding.
478      prng_en_dis_inv_set = 1'b0;
479
480      // Request PRNG reseeding.
481      prng_reseed_req_o = 1'b0;
482
483      // initialization complete
484      init_o = 1'b0;
485
486      // Most states are initialized, mark the exceptions
487      initialized = 1'b1;
488
489      // if state is ever faulted, hold on to this indication
490      // until reset.
491      state_intg_err_d = state_intg_err_q;
492
493  unique case (state_q)
494      // Only advance can be called from reset state
495      StCtrlReset: begin
496          initialized = 1'b0;
497
498          // always use random data for advance, since out of
reset state
499          // the key state will be randomized.
500          stage_sel_o = Disable;
501
502          // key state is updated when it is an advance call
503          // all other operations are invalid, including disable
504          invalid_op = op_start_i & ~advance_sel;
505
506          // if there was a structural fault before anything
began, wipe immediately
507          if (inv_state) begin
508              state_d = StCtrlWipe;
509          end else if (advance_sel) begin
510              state_d = StCtrlEntropyReseed;
511          end
512      end
513
514      // reseed entropy
515      StCtrlEntropyReseed: begin
516          initialized = 1'b0;
517          prng_reseed_req_o = 1'b1;
518
519          if (prng_reseed_ack_i) begin
520              state_d = StCtrlRandom;
521          end
522      end
523
524      // This state does not accept any command.
525      StCtrlRandom: begin
526          initialized = 1'b0;
527          random_req = 1'b1;
528
529      // when mask population is complete, xor the root_key
into the zero share
530      // if in the future the root key is updated to 2
shares, it will directly overwrite
531      // the values here
532      if (int'(cnt) == EntropyRounds-1) begin
533          random_ack = 1'b1;
534          state_d = StCtrlRootKey;
535      end
536
537      // load the root key.
538      StCtrlRootKey: begin
539          init_o = 1'b1;
540          initialized = 1'b1;
541          state_d = (en_i && root_key_valid_q) ? StCtrlInit :
StCtrlWipe;
542      end
543  end
544

```

```

545 // Beginning from the Init state, operations are accep-
ted.
546 // Only valid operation is advance state. If invalid
command received,
547 // random data is selected for operation and no persist-
ent state is changed.
548 StCtrlInit: begin
549   op_req = op_start_i;
550
551 // when advancing select creator data, otherwise use
random input
552   stage_sel_o = advance_sel ? Creator : Disable;
553   invalid_op = op_start_i & ~(advance_sel | dis-
able_sel);
554
555   if (!en_i || inv_state) begin
556     state_d = StCtrlWipe;
557   end else if (dis_state) begin
558     state_d = StCtrlDisabled;
559     prng_en_dis_inv_set = 1'b1;
560   end else if (adv_state) begin
561     state_d = StCtrlCreatorRootKey;
562   end
563 end
564
565 // all commands are valid during this stage
StCtrlCreatorRootKey: begin
566   op_req = op_start_i;
567
568 // when generating, select creator data input
569 // when advancing, select owner intermediate key as
target
570   // when disabling, select random data input
571   stage_sel_o = disable_sel ? Disable :
572     advance_sel ? OwnerInt : Creator;
573
574   if (!en_i || inv_state) begin
575     state_d = StCtrlWipe;
576   end else if (dis_state) begin
577     state_d = StCtrlDisabled;
578     prng_en_dis_inv_set = 1'b1;
579   end else if (adv_state) begin
580     state_d = StCtrlOwnerIntKey;
581   end
582 end
583
584 // all commands are valid during this stage
StCtrlOwnerIntKey: begin
585   op_req = op_start_i;
586
587 // when generating, select owner intermediate data in-
put
588   // when advancing, select owner as target
589   // when disabling, select random data input
590   stage_sel_o = disable_sel ? Disable :
591     advance_sel ? Owner : OwnerInt;
592
593   if (!en_i || inv_state) begin
594     state_d = StCtrlWipe;
595   end else if (dis_state) begin
596     state_d = StCtrlDisabled;
597     prng_en_dis_inv_set = 1'b1;
598   end else if (adv_state) begin
599     state_d = StCtrlOwnerKey;
600   end
601 end
602
603 // all commands are valid during this stage
// however advance goes directly to disabled state
StCtrlOwnerKey: begin
604   op_req = op_start_i;
605
606 // when generating, select owner data input
607 // when advancing, select disable as target
608 // when disabling, select random data input
609   stage_sel_o = disable_sel | advance_sel ? Disable :
Owner;
610
611   if (!en_i || inv_state) begin
612     state_d = StCtrlWipe;
613   end else if (adv_state || dis_state) begin
614     state_d = StCtrlDisabled;
615     prng_en_dis_inv_set = 1'b1;
616   end
617 end
618
619 // The wipe state immediately clears out the key state,
but waits for any ongoing
620 // transaction to finish before going to disabled state.
621 // Unlike the random state, this is an immediate shut-
down request, so all parts of the
622 // key are wiped.
623 StCtrlWipe: begin
624   wipe_req = 1'b1;
625
626 // if there was already an operation ongoing, maintain
the request until completion
627   op_req = op_busy;
628   invalid_op = op_start_i;
629
630 // If the enable is dropped during the middle of a

```

```

transaction, we clear and wait for that
634 // transaction to gracefully complete (if it can).
635 // There are two scenarios:
636 // 1. the operation completed right when we started
wiping, in which case the done would
637 // clear the start.
638 // 2. the operation completed before we started wip-
ing, or there was never an operation to
639 // begin with (op_start_i == 0), in this case,
don't wait and immediately transition
640   if (!op_start_i) begin
641     state_d = StCtrlInvalid;
642     prng_en_dis_inv_set = 1'b1;
643   end
644 end
645
646 // StCtrlDisabled and StCtrlInvalid are almost function-
ally equivalent
647 // The only difference is that Disabled is entered
through software invocation,
648 // while Invalid is entered through life cycle disable
or operational fault.
649 //
650 // Both states continue to kick off random transactions
651 // All transactions are treated as invalid despite com-
pleting
652 StCtrlDisabled: begin
653   op_req = op_start_i;
654   disabled = 1'b1;
655
656   if (!en_i || inv_state) begin
657     state_d = StCtrlWipe;
658   end
659 end
660
661 StCtrlInvalid: begin
662   invalid_op = op_start_i;
663   invalid = 1'b1;
664 end
665
666 // latch the fault indication and start to wipe the key
manager
667   default: begin
668     state_intg_err_d = 1'b1;
669     state_d = StCtrlWipe;
670   end
671
672   endcase // unique case (state_q)
end // always_comb
673
674 // Current working state provided for software read
675 // Certain states are collapsed for simplicity
676 keymgr_working_state_e last_working_st;
677 logic update_en;
678
679 always_ff @(posedge clk_i or negedge rst_ni) begin
680   if (!rst_ni) begin
681     last_working_st <= StReset;
682   end else if (update_en) begin
683     last_working_st <= working_state_o;
684   end
685 end
686
687 always_comb begin
688   update_en = 1'b1;
689   working_state_o = StInvalid;
690
691   unique case (state_q)
692     StCtrlReset, StCtrlEntropyReSeed, StCtrlRandom:
693       working_state_o = StReset;
694
695     StCtrlRootKey, StCtrlInit:
696       working_state_o = StInit;
697
698     StCtrlCreatorRootKey:
699       working_state_o = StCreatorRootKey;
700
701     StCtrlOwnerIntKey:
702       working_state_o = StOwnerIntKey;
703
704     StCtrlOwnerKey:
705       working_state_o = StOwnerKey;
706
707     StCtrlDisabled:
708       working_state_o = StDisabled;
709
710     StCtrlWipe: begin
711       update_en = 1'b0;
712       working_state_o = last_working_st;
713     end
714
715     StCtrlInvalid:
716       working_state_o = StInvalid;
717
718     default:
719       working_state_o = StInvalid;
720   endcase // unique case (state_q)
end
721
722 always_comb begin
723   status_o = OpIdle;
724
725

```



```

726   if (op_done_o) begin
727     // It is possible for an operation to finish the same
cycle en_i goes low.
728     // The main fsm handling is one cycle behind, but still
report operation
729     // fail.
730     status_o = {error_o, fault_o} ? OpDoneFail : OpDoneSuc-
cess;
731   end else if (op_start_i) begin
732     status_o = OpWip;
733   end
734 end
735
736 //////////////////////////////////////////////////
737 // Operation state, handle advance and generate
738 //////////////////////////////////////////////////
739
740 logic op_fsm_err;
741 keymgr_op_state_ctrl u_op_state (
742   .clk_i,
743   .rst_ni,
744   .adv_req_i(adv_req),
745   .dis_req_i(dis_req),
746   .id_req_i(id_req),
747   .gen_req_i(gen_req),
748   .cnt_i(cdi_cnt),
749   .op_ack_o(op_ack),
750   .op_busy_o(op_busy),
751   .op_update_o(op_update),
752   .kmac_done_i,
753   .adv_en_o,
754   .id_en_o,
755   .gen_en_o,
756   .op_fsm_err_o(op_fsm_err)
757 );
758
759 // operational state cross check. The state value must be
consistent with
760 // the input operations.
761 logic op_state_cmd_err;
762 assign op_state_cmd_err = (adv_en_o & ~(advance_sel | dis-
able_sel)) |
763   (gen_en_o & ~gen_op);
764
765 // operations fsm update precedence
766 // when in invalid state, always update.
767 // when in disabled state, always update unless a fault is
encountered.
768
769 assign op_update_sel = (op_ack | op_update) & invalid ?
KeyUpdateKmac :
770   (op_ack | op_update) & op_fault_err ?
KeyUpdateWipe :
771   (op_ack | op_update) & disabled ?
KeyUpdateKmac :
772   (op_ack | op_update) & op_err ?
KeyUpdateIdle :
773   (op_ack | op_update) ?
KeyUpdateKmac : KeyUpdateIdle;
774
775 //////////////////////////////////////////////////
776 // Suppress kmac return data
777 //////////////////////////////////////////////////
778
779 logic data_fsm_err;
780 keymgr_data_en_state u_data_en (
781   .clk_i,
782   .rst_ni,
783   .hw_sel_i(hw_sel_o),
784   .adv_en_i(adv_en_o),
785   .id_en_i(id_en_o),
786   .gen_en_i(gen_en_o),
787   .op_done_i(op_done_o),
788   .op_start_i,
789   .data_hw_en_o,
790   .data_sw_en_o,
791   .fsm_err_o(data_fsm_err)
792 );
793
794 //////////////////////////////////////////////////
795 // Cross-checks, errors and faults
796 //////////////////////////////////////////////////
797
798 logic vld_state_change_d, vld_state_change_q;
799 assign vld_state_change_d = (state_d != state_q) &
800   (state_d inside {StCtrlRootKey,
801     StCtrlCreator-
RootKey,
802     StCtrlOwner-
IntKey,
803     StCtrlOwner-
Key});
804
805 // capture for cross check in following cycle
806 always_ff @(posedge clk_i or negedge rst_ni) begin

```

```

808   if (!rst_ni) begin
809     vld_state_change_q <= '0;
810   end else begin
811     vld_state_change_q <= vld_state_change_d;
812   end
813 end
814
815 // state cross check
816 // if the state advanced, ensure that it was due to an ad-
vanced operation
817 logic state_change_err;
818 assign state_change_err = vld_state_change_q & !adv_op;
819
820 keymgr_err_u_err (
821   .clk_i,
822   .rst_ni,
823   .invalid_op_i(invalid_op),
824   .disabled_i(disabled | (initialized & ~en_i)),
825   .invalid_i(invalid),
826   .kmac_input_invalid_i,
827   .shadowed_update_err_i,
828   .kmac_op_err_i,
829   .invalid_kmac_out_i(invalid_kmac_out),
830   .sideload_sel_err_i,
831   .kmac_cmd_err_i,
832   .kmac_fsm_err_i,
833   .kmac_done_err_i,
834   .regfile_intg_err_i,
835   .shadowed_storage_err_i,
836   .ctrl_fsm_err_i(state_intg_err_q | state_intg_err_d),
837   .data_fsm_err_i(data_fsm_err),
838   .op_fsm_err_i(op_fsm_err),
839   .ecc_err_i(jecc_errs),
840   .state_change_err_i(state_change_err),
841   .op_state_cmd_err_i(op_state_cmd_err),
842   .cnt_err_i(cnt_err),
843   .reseed_cnt_err_i,
844   .sideload_fsm_err_i,
845
846   .op_update_i(op_update),
847   .op_done_i(op_done_o),
848
849   .sync_err_o(sync_err),
850   .async_err_o(),
851   .sync_fault_o(sync_fault),
852   .async_fault_o(async_fault),
853   .error_o,
854   .fault_o
855 );
856
857 //////////////////////////////////////////////////
858 // Assertions
859 //////////////////////////////////////////////////
860
861 // This assertion will not work if fault_status ever takes
on metafields such as
862 // qe / re etc.
863 `ASSERT_INIT(SameErrCnt_A, $bits(keym-
gr_reg2hw_fault_status_reg_t) ==
864   (SyncFaultLastIdx + AsyncFault-
LastIdx))
865
866 // stage select should always be Disable whenever it is not
enabled
867 `ASSERT(StageDisableSel_A, !en_i |-> stage_sel_o == Disable)
868
869 // Unless it is a legal command, only select disable
870 `ASSERT(InitLegalCommands_A, op_start_i & en_i & state_q in-
side {StCtrlInit} &
871   !(op_i inside {OpAdvance}) |->
stage_sel_o == Disable)
872
873 // All commands are legal, so select disable only if opera-
tion is disable
874 `ASSERT(GeneralLegalCommands_A, op_start_i & en_i & state_q in-
side {StCtrlCreat-
orRootKey, StCtrlOwnerIntKey} &
875   (op_i inside {OpDisable}) |->
> stage_sel_o == Disable)
876
877 `ASSERT(OwnerLegalCommands_A, op_start_i & en_i & state_q
inside {StCtrlOwnerKey} &
878   (op_i inside {OpAdvance, Op-
Disable}) |-> stage_sel_o == Disable)
879
880 // load_key should not be high if there is no ongoing opera-
tion
881 `ASSERT(LoadKey_A, key_o.valid |-> op_start_i)
882
883 // The count value should always be 0 when a transaction
start
884 `ASSERT(CntZero_A, $rose(op_start_i) |-> cnt == '0)
885
886 // Whenever a transaction completes, data_en must return to
0 on the next cycle
887 `ASSERT(DataEndDis_A, op_start_i & op_done_o |>
~data_hw_en_o && ~data_sw_en_o)
888
889 // Whenever data enable asserts, it must be the case that
there was a generate or
890 // id operation

```

```

892 `ASSERT(DataEn_A, data_hw_en_o | data_sw_en_o |-> (id_en_o |
gen_en_o) & ~adv_en_o)
893
894 // Check that the FSM is linear and does not contain any
loops
895 `ASSERT_FPV_LINEAR_FSM(SecCmCFILinear_A, state_q, state_e)
896
897 endmodule

```

4. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_data_en_state.sv (136 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Key manager data enable generation
6 // This is a redundant alternative to data_valid
7
8 `include "prim_assert.sv"
9
10 module keymgr_data_en_state
11 import keymgr_pkg::*;
12 import keymgr_reg_pkg::*;
13 (
14 input clk_i,
15 input rst_ni,
16 input prim_mubi_pkg::mubi4_t hw_sel_i,
17 input adv_en_i,
18 input id_en_i,
19 input gen_en_i,
20 input op_done_i,
21 input op_start_i,
22 output logic data_hw_en_o,
23 output logic data_sw_en_o,
24 output logic fsm_err_o
25 );
26
27 import prim_mubi_pkg::mubi4_test_true_strict;
28 import prim_mubi_pkg::mubi4_test_true_loose;
29 import prim_mubi_pkg::mubi4_test_false_strict;
30 import prim_mubi_pkg::mubi4_test_false_loose;
31
32 // This is a separate data path from the FSM used to control
the data_en outputs
33 // Encoding generated with:
34 // $ ./util/design/sparse-fsm-encode.py -d 5 -m 6 -n 10 \
35 // -s 2015444891 --language=sv
36 //
37 // Hamming distance histogram:
38 //
39 // 0: --
40 // 1: --
41 // 2: --
42 // 3: --
43 // 4: --
44 // 5: ||||| (33.33%)
45 // 6: ||||| (40.00%)
46 // 7: ||||| (26.67%)
47 // 8: --
48 // 9: --
49 // 10: --
50 //
51 // Minimum Hamming distance: 5
52 // Maximum Hamming distance: 7
53 // Minimum Hamming weight: 2
54 // Maximum Hamming weight: 7
55 //
56 localparam int DataStateWidth = 10;
57 typedef enum logic [DataStateWidth-1:0] {
58 StCtrlDataIdle = 10'b1000010000,
59 StCtrlDataHwEn = 10'b0001100100,
60 StCtrlDataSwEn = 10'b1110101110,
61 StCtrlDataDis = 10'b0010011111,
62 StCtrlDataWait = 10'b0111110011,
63 StCtrlDataInvalid = 10'b1111001001
64 } state_e;
65
66 state_e state_d, state_q;
67
68 // SEC_CM: DATA.FSM.SPARSE
69 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
state_e, StCtrlDataIdle)
70
71 // The below control path is used for modulating the
datapath to sideload and sw keys.
72 // This path is separate from the data_valid_o path, thus
creating two separate attack points.
73 // The data is only enabled when a non-advance operation is
invoked.
74 // When an advance operation is called, the data is dis-
abled. It will stay disabled until an
75 // entire completion sequence is seen (op_done_o assert ->
start_i de-assertion).
76 // When a generate operation is called, the data is en-
abled. However, any indication of this

```

```

77 // supposedly being an advance call will force the path to
disable again.
78 always_comb begin
79 state_d = state_q;
80 fsm_err_o = 1'b0;
81 data_hw_en_o = 1'b0;
82 data_sw_en_o = 1'b0;
83 unique case (state_q)
84
85 StCtrlDataIdle: begin
86 if (adv_en_i) begin
87 state_d = StCtrlDataDis;
88 end else if ((id_en_i || gen_en_i) && mu-
bi4_test_true_strict(hw_sel_i)) begin
89 state_d = StCtrlDataHwEn;
90 end else if ((id_en_i || gen_en_i) && mu-
bi4_test_false_strict(hw_sel_i)) begin
91 state_d = StCtrlDataSwEn;
92 end else if (id_en_i || gen_en_i) begin
93 state_d = StCtrlDataDis;
94 end
95 end
96
97 StCtrlDataHwEn: begin
98 data_hw_en_o = 1'b1;
99 if (op_done_i) begin
100 state_d = StCtrlDataWait;
101 end else if (adv_en_i || mu-
bi4_test_false_loose(hw_sel_i)) begin
102 state_d = StCtrlDataDis;
103 end
104 end
105
106 StCtrlDataSwEn: begin
107 data_sw_en_o = 1'b1;
108 if (op_done_i) begin
109 state_d = StCtrlDataWait;
110 end else if (adv_en_i || mu-
bi4_test_true_loose(hw_sel_i)) begin
111 state_d = StCtrlDataDis;
112 end
113 end
114
115 StCtrlDataDis: begin
116 if (op_done_i) begin
117 state_d = StCtrlDataWait;
118 end
119 end
120
121 StCtrlDataWait: begin
122 if (!op_start_i) begin
123 state_d = StCtrlDataIdle;
124 end
125 end
126
127 default: begin
128 fsm_err_o = 1'b1;
129 end
130
131 endcase // unique case (state_q)
132 end
133
134
135
136 endmodule // keymgr_data_en_state

```

5. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_err.sv (153 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Key manager error and fault collection
6 //
7
8 `include "prim_assert.sv"
9
10 module keymgr_err
11 import keymgr_pkg::*;
12 import keymgr_reg_pkg::*;
13 (
14 input clk_i,
15 input rst_ni,
16
17 input invalid_op_i,
18 input disabled_i,
19 input invalid_i,
20 input kmac_input_invalid_i,
21 input shadowed_update_err_i,
22 input kmac_op_err_i,
23 input invalid_kmac_out_i,
24 input sideload_sel_err_i,
25 input kmac_cmd_err_i,
26 input kmac_fsm_err_i,
27 input kmac_done_err_i,
28 input regfile_intg_err_i,
29 input shadowed_storage_err_i,

```

```

30 input ctrl_fsm_err_i,
31 input data_fsm_err_i,
32 input op_fsm_err_i,
33 input ecc_err_i,
34 input state_change_err_i,
35 input op_state_cmd_err_i,
36 input cnt_err_i,
37 input reseed_cnt_err_i,
38 input sideload_fsm_err_i,
39
40 input op_update_i,
41 input op_done_i,
42
43 // The following outputs are very similar, but have slightly
different timing for
44 // for CDIs on sync errors/faults.
45 // Advance operations must go through for all CDIs.
46 // The sync_err/fault outputs register when any CDI com-
pletes and helps with
47 // the appropriate behavior on key state change.
48 // The sync_error_o/fault_o outputs on the other hand only
output when the entire
49 // operation is complete, which could be multiple CDIs.
50 output logic [SyncErrLastIdx-1:0] sync_err_o,
51 output logic [AsyncErrLastIdx-1:0] async_err_o,
52 output logic [SyncFaultLastIdx-1:0] sync_fault_o,
53 output logic [AsyncFaultLastIdx-1:0] async_fault_o,
54 output logic [ErrLastPos-1:0] error_o,
55 output logic [FaultLastPos-1:0] fault_o
56 );
57
58 // Advance calls are made up of multiple rounds of kmac op-
erations.
59 // Any sync error that occurs is treated as an error of the
entire call.
60 // Therefore sync errors that happen before the end of the
call must be
61 // latched.
62 logic[SyncErrLastIdx-1:0] sync_err_q, sync_err_d;
63 logic[SyncFaultLastIdx-1:0] sync_fault_q, sync_fault_d;
64
65 logic err_vld;
66 assign err_vld = op_update_i | op_done_i;
67
68 // sync errors
69 // When an operation encounters a fault, the operation is
always rejected as the FSM
70 // transitions to wipe. When an operation is ongoing and en
drops, it is also rejected.
71 assign sync_err_d[SyncErrInvalidOp] = err_vld & (inval-
id_op_i |
72                                     disabled_i
73 |
74                                     invalid_i |
75                                     (|
76                                     fault_o));
77 assign sync_err_d[SyncErrInvalidIn] = err_vld & kmac_in-
put_invalid_i;
78
79 always_ff @(posedge clk_i or negedge rst_ni) begin
80   if (!rst_ni) begin
81     sync_err_q <= '0;
82   end else if (op_done_i) begin
83     sync_err_q <= '0;
84   end else if (op_update_i) begin
85     sync_err_q <= sync_err_d;
86   end
87 end
88 assign sync_err_o = sync_err_q | sync_err_d;
89
90 // async errors
91 assign async_err_o[AsyncErrShadowUpdate] = shad-
owed_update_err_i;
92
93 // sync faults
94 assign sync_fault_d[SyncFaultKmacOp] = err_vld &
kmac_op_err_i;
95 assign sync_fault_d[SyncFaultKmacOut] = err_vld & inval-
id_kmac_out_i;
96 assign sync_fault_d[SyncFaultSideSel] = err_vld & side-
load_sel_err_i;
97
98 always_ff @(posedge clk_i or negedge rst_ni) begin
99   if (!rst_ni) begin
100     sync_fault_q <= '0;
101   end else if (op_done_i) begin
102     sync_fault_q <= '0;
103   end else if (op_update_i) begin
104     sync_fault_q <= sync_fault_d;
105   end
106 end
107 assign sync_fault_o = sync_fault_q | sync_fault_d;
108
109 // async faults
110 logic [AsyncFaultLastIdx-1:0] async_fault_q, async_fault_d;
111 always_ff @(posedge clk_i or negedge rst_ni) begin
112   if (!rst_ni) begin
113     async_fault_q <= '0;
114   end else begin
115     async_fault_q <= async_fault_o;
116   end
117 end
118
119 end
120
121 assign async_fault_o = async_fault_q | async_fault_d;
122 assign async_fault_d[AsyncFaultKmacCmd] = kmac_cmd_err_i;
123 assign async_fault_d[AsyncFaultKmacFsm] = kmac_fsm_err_i;
124 assign async_fault_d[AsyncFaultKmacDone] = kmac_done_err_i;
125 assign async_fault_d[AsyncFaultRegIntg] = reg-
file_intg_err_i;
126 assign async_fault_d[AsyncFaultShadow] = shad-
owed_storage_err_i;
127 assign async_fault_d[AsyncFaultFsmIntg] = ctrl_fsm_err_i |
data_fsm_err_i | op_fsm_err_i;
128 assign async_fault_d[AsyncFaultKeyEcc] = ecc_err_i;
129
130 // SEC_CM: CTRL.FSM.CONSTISTENCY
131 assign async_fault_d[AsyncFaultFsmChk] =
state_change_err_i | op_state_cmd_err_i;
132 assign async_fault_d[AsyncFaultCntErr] = cnt_err_i;
133 assign async_fault_d[AsyncFaultRCntErr] = reseed_cnt_err_i;
134 assign async_fault_d[AsyncFaultSideErr] = side-
load_fsm_err_i;
135
136 // certain errors/faults can only happen when there's an ac-
tual kmac transaction,
137 // others can happen with or without.
138 assign error_o[ErrInvalidOp] = op_done_i &
sync_err_o[SyncErrInvalidOp];
139 assign error_o[ErrInvalidIn] = op_done_i &
sync_err_o[SyncErrInvalidIn];
140 assign error_o[ErrShadowUpdate] =
async_err_o[AsyncErrShadowUpdate];
141
142 // output to fault code register
143 assign fault_o[FaultKmacOp] = op_done_i &
sync_fault_o[SyncFaultKmacOp];
144 assign fault_o[FaultKmacOut] = op_done_i &
sync_fault_o[SyncFaultKmacOut];
145 assign fault_o[FaultSideSel] = op_done_i &
sync_fault_o[SyncFaultSideSel];
146 assign fault_o[FaultKmacCmd] =
async_fault_o[AsyncFaultKmacCmd];
147 assign fault_o[FaultKmacFsm] =
async_fault_o[AsyncFaultKmacFsm];
148 assign fault_o[FaultKmacDone] =
async_fault_o[AsyncFaultKmacDone];
149 assign fault_o[FaultRegIntg] =
async_fault_o[AsyncFaultRegIntg];
150 assign fault_o[FaultShadow] =
async_fault_o[AsyncFaultShadow];
151 assign fault_o[FaultCtrlFsm] =
async_fault_o[AsyncFaultFsmIntg];
152 assign fault_o[FaultCtrlFsmChk] =
async_fault_o[AsyncFaultFsmChk];
153 assign fault_o[FaultCtrlCnt] =
async_fault_o[AsyncFaultCntErr];
154 assign fault_o[FaultReseedCnt] =
async_fault_o[AsyncFaultRCntErr];
155 assign fault_o[FaultSideFsm] =
async_fault_o[AsyncFaultSideErr];
156 assign fault_o[FaultKeyEcc] =
async_fault_o[AsyncFaultKeyEcc];
157
158 endmodule // keymgr_err

```

6. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_input_checks.sv (121 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Key manager input checks
6 // Checks input data for errors
7
8 `include "prim_assert.sv"
9
10 // We should also check for input validity
11 module keymgr_input_checks import keymgr_pkg::*; #(
12   parameter bit KmacEnMasking = 1'b1,
13   parameter int unsigned NumRomDigestInputs = 1
14 ) (
15   input rom_ctrl_pkg::keymgr_data_t [NumRomDigestInputs-1:0]
rom_digest_i,
16   input [KeyVersionWidth-1:0] cur_max_key_version_i,
17   input hw_key_req_t key_i,
18   input [31:0] key_version_i,
19   input [KeyWidth-1:0] creator_seed_i,
20   input [KeyWidth-1:0] owner_seed_i,
21   input [DevIdWidth-1:0] devid_i,
22   input [HealthStateWidth-1:0] health_state_i,
23   output logic creator_seed_vld_o,
24   output logic owner_seed_vld_o,
25   output logic devid_vld_o,
26   output logic health_state_vld_o,
27   output logic key_version_vld_o,
28   output logic key_vld_o,
29   output logic rom_digest_vld_o

```



```

30 );
31 // checks for all 0's or all 1's of value
32 function automatic logic valid_chk (logic [MaxWidth-1:0]
value);
33     return |value & ~&value;
34 endfunction // valid_chk
35
36 // key version must be smaller than or equal to max version
37 assign key_version_vld_o = key_version_i <= cur_max_key_ver-
sion_i;
38
39 // general data check
40 logic [MaxWidth-1:0] creator_seed_padded, owner_seed_padded,
devid_padded, health_state_padded;
41
42 prim_msb_extend #(
43     .InWidth(KeyWidth),
44     .OutWidth(MaxWidth)
45 ) u_creator_seed (
46     .in_i(creator_seed_i),
47     .out_o(creator_seed_padded)
48 );
49
50 prim_msb_extend #(
51     .InWidth(KeyWidth),
52     .OutWidth(MaxWidth)
53 ) u_owner_seed (
54     .in_i(owner_seed_i),
55     .out_o(owner_seed_padded)
56 );
57
58 prim_msb_extend #(
59     .InWidth(DevIdWidth),
60     .OutWidth(MaxWidth)
61 ) u_devid (
62     .in_i(devid_i),
63     .out_o(devid_padded)
64 );
65
66 prim_msb_extend #(
67     .InWidth(HealthStateWidth),
68     .OutWidth(MaxWidth)
69 ) u_health_state (
70     .in_i(health_state_i),
71     .out_o(health_state_padded)
72 );
73
74 assign creator_seed_vld_o = valid_chk(creator_seed_padded);
75 assign owner_seed_vld_o = valid_chk(owner_seed_padded);
76 assign devid_vld_o = valid_chk(devid_padded);
77 assign health_state_vld_o = valid_chk(health_state_padded);
78
79 // key check
80 logic unused_key_vld;
81 assign unused_key_vld = key_i.valid;
82
83 localparam int KeyShares = KmacEnMasking ? Shares : 1;
84 logic [KeyShares-1:0][MaxWidth-1:0] key_padded;
85 logic [KeyShares-1:0] key_chk;
86
87 for (genvar i = 0; i < KeyShares; i++) begin : gen_key_chk
88     prim_msb_extend #(
89         .InWidth(KeyWidth),
90         .OutWidth(MaxWidth)
91     ) u_key_pad (
92         .in_i(key_i.key[i]),
93         .out_o(key_padded[i])
94     );
95
96     assign key_chk[i] = valid_chk(key_padded[i]);
97 end
98
99 assign key_vld_o = &key_chk;
100
101 logic [NumRomDigestInputs-1:0][MaxWidth-1:0] rom_di-
gest_padded;
102
103 for (genvar k = 0; k < NumRomDigestInputs; k++) begin :
gen_digest_pad
104     prim_msb_extend #(
105         .InWidth($bits(rom_digest_i[k].data)),
106         .OutWidth(MaxWidth)
107     ) u_rom_digest_pad (
108         .in_i(rom_digest_i[k].data),
109         .out_o(rom_digest_padded[k])
110     );
111 end
112
113 always_comb begin
114     rom_digest_vld_o = 'b1;
115     for (int k = 0; k < NumRomDigestInputs; k++) begin
116         rom_digest_vld_o &= rom_digest_i[k].valid &&
valid_chk(rom_digest_padded[k]);
117     end
118 end
119
120
121 endmodule // keymgr_input_checks

```

7. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_kmac_if.sv (408 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Key manager interface to kmac
6 //
7
8 `include "prim_assert.sv"
9
10 module keymgr_kmac_if
11 import keymgr_pkg::*;
12 #(
13     parameter rand_perm_t RndCnstRandPerm = RndCnstRandPermDe-
fault,
14     parameter int MaxAdvDataWidth = AdvDataWidth
15 ) (
16     input clk_i,
17     input rst_ni,
18
19     // data input interfaces
20     input [MaxAdvDataWidth-1:0] adv_data_i,
21     input [IdDataWidth-1:0] id_data_i,
22     input [GenDataWidth-1:0] gen_data_i,
23     input [3:0] inputs_invalid_i,
24     output logic inputs_invalid_o,
25
26     // keymgr control to select appropriate inputs
27     input adv_en_i,
28     input id_en_i,
29     input gen_en_i,
30     output logic done_o,
31     output logic [Shares-1:0][kmac_pkg::AppDigestW-1:0] data_o,
32
33     // actual connection to kmac
34     output kmac_pkg::app_req_t kmac_data_o,
35     input kmac_pkg::app_rsp_t kmac_data_i,
36
37     // entropy input
38     output logic prng_en_o,
39     input [Shares-1:0][Randwidth-1:0] entropy_i,
40
41     // error outputs
42     output logic fsm_error_o,
43     output logic kmac_error_o,
44     output logic kmac_done_error_o,
45     output logic cmd_error_o
46 );
47
48
49 // Encoding generated with:
50 // $ ./util/design/sparse-fsm-encode.py -d 5 -m 6 -n 10 \
51 // -s 2292624416 --language=sv
52 //
53 // Hamming distance histogram:
54 //
55 // 0: --
56 // 1: --
57 // 2: --
58 // 3: --
59 // 4: --
60 // 5: ||| (46.67%)
61 // 6: ||| (40.00%)
62 // 7: ||| (13.33%)
63 // 8: --
64 // 9: --
65 // 10: --
66 //
67 // Minimum Hamming distance: 5
68 // Maximum Hamming distance: 7
69 // Minimum Hamming weight: 2
70 // Maximum Hamming weight: 9
71 //
72 localparam int StateWidth = 10;
73 typedef enum logic [StateWidth-1:0] {
74     StIdle = 10'b110100010,
75     StTx = 10'b0010011011,
76     StTxLast = 10'b0101000000,
77     StOpWait = 10'b1000101001,
78     StClean = 10'b111111101,
79     StError = 10'b001101110
80 } data_state_e;
81
82 localparam int AdvRem = MaxAdvDataWidth % KmacDataIfWidth;
83 localparam int IdRem = IdDataWidth % KmacDataIfWidth;
84 localparam int GenRem = GenDataWidth % KmacDataIfWidth;
85
86 // the remainder must be in number of bytes
87 `ASSERT_INIT(AdvRemBytes_A, AdvRem % 8 == 0)
88 `ASSERT_INIT(IdRemBytes_A, IdRem % 8 == 0)
89 `ASSERT_INIT(GenRemBytes_A, GenRem % 8 == 0)
90
91 // Number of kmac transactions required
92 localparam int AdvRounds = (MaxAdvDataWidth +
KmacDataIfWidth - 1) / KmacDataIfWidth;

```

```

93  localparam int IdRounds = (IdDataWidth + KmacDataIfWidth -
1) / KmacDataIfWidth;
94  localparam int GenRounds = (GenDataWidth + KmacDataIfWidth -
1) / KmacDataIfWidth;
95  localparam int MaxRounds = KDFMaxWidth / KmacDataIfWidth;
96
97  // calculated parameters for number of rounds and interface
width
98  localparam int CntWidth = $clog2(MaxRounds);
99  localparam int IfBytes = KmacDataIfWidth / 8;
100 localparam int DecoyCopies = KmacDataIfWidth / RandWidth;
101 localparam int DecoyOutputCopies = (kmac_pkg::AppDigestW /
RandWidth);
102
103 localparam int unsigned LastAdvRoundInt = AdvRounds - 1;
104 localparam int unsigned LastIdRoundInt = IdRounds - 1;
105 localparam int unsigned LastGenRoundInt = GenRounds - 1;
106 localparam bit [CntWidth-1:0] LastAdvRound = LastAdvRoundInt[CntWidth-1:0];
107 localparam bit [CntWidth-1:0] LastIdRound = LastIdRoundInt[CntWidth-1:0];
108 localparam bit [CntWidth-1:0] LastGenRound = LastGenRoundInt[CntWidth-1:0];
109
110 // byte mask for the last transfer
111 localparam logic [IfBytes-1:0] AdvByteMask = (AdvRem > 0) ?
(2**(AdvRem/8)-1) : {IfBytes{1'b1}};
112 localparam logic [IfBytes-1:0] IdByteMask = (IdRem > 0) ?
(2**(IdRem/8)-1) : {IfBytes{1'b1}};
113 localparam logic [IfBytes-1:0] GenByteMask = (GenRem > 0) ?
(2**(GenRem/8)-1) : {IfBytes{1'b1}};
114
115 logic [MaxRounds-1:0][KmacDataIfWidth-1:0] adv_data;
116 logic [MaxRounds-1:0][KmacDataIfWidth-1:0] id_data;
117 logic [MaxRounds-1:0][KmacDataIfWidth-1:0] gen_data;
118 logic [CntWidth-1:0] cnt;
119 logic [CntWidth-1:0] rounds;
120 logic [KmacDataIfWidth-1:0] decoy_data;
121 logic valid;
122 logic last;
123 logic [IfBytes-1:0] strb;
124 logic cnt_clr, cnt_set, cnt_en;
125 logic start;
126 logic [3:0] inputs_invalid_d, inputs_invalid_q;
127 logic clr_err;
128 logic kmac_done_vld;
129 logic cmd_chk;
130
131 data_state_e state_q, state_d;
132
133 // 0 pad to the appropriate width
134 // this is basically for scenarios where *DataWidth %
KmacDataIfWidth != 0
135 assign adv_data = KDFMaxWidth'(adv_data_i);
136 assign id_data = KDFMaxWidth'(id_data_i);
137 assign gen_data = KDFMaxWidth'(gen_data_i);
138
139 assign start = adv_en_i | id_en_i | gen_en_i;
140
141 logic cnt_err;
142 // SEC_CM: KMAC_IF.CTR.REDUN
143 prim_count #(
144   .Width(CntWidth),
145   .ResetValue({CntWidth{1'b1}})
146 ) u_cnt (
147   .clk_i,
148   .rst_ni,
149   .clr_i(cnt_clr),
150   .set_i(cnt_set),
151   .set_cnt_i(rounds),
152   .incr_en_i(1'b0),
153   .decr_en_i(cnt_en),
154   .step_i(CntWidth'(1'b1)),
155   .commit_i(1'b1),
156   .cnt_o(cnt),
157   .cnt_after_commit_o(),
158   .err_o(cnt_err)
159 );
160
161 always_ff @(posedge clk_i or negedge rst_ni) begin
162   if (!rst_ni) begin
163     inputs_invalid_q <= '0;
164   end else begin
165     inputs_invalid_q <= inputs_invalid_d;
166   end
167 end
168
169 // SEC_CM: KMAC_IF.FSM.SPARSE
170 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
data_state_e, StIdle)
171
172 always_comb begin
173   cnt_clr = 1'b0;
174   cnt_set = 1'b0;
175   cnt_en = 1'b0;
176   valid = 1'b0;
177   last = 1'b0;
178   strb = '0;
179   done_o = 1'b0;
180   state_d = state_q;

```

```

181 rounds = '0;
182
183 clr_err = '0;
184 fsm_error_o = '0;
185 kmac_error_o = '0;
186
187 kmac_done_vld = '0;
188
189 cmd_chk = 1'b1;
190
191 unique case (state_q)
192
193   StIdle: begin
194     // if for some reason multiple bits are set, adv_en
has priority
195     // as the current key state will be destroyed
196
197     // cross check for commands once transaction begins
198     cmd_chk = '0;
199     if (start) begin
200       cnt_set = 1'b1;
201       if (adv_en_i) begin
202         rounds = LastAdvRound;
203       end else if (id_en_i) begin
204         rounds = LastIdRound;
205       end else if (gen_en_i) begin
206         rounds = LastGenRound;
207       end
208       // in case we are sending only 1 entry
209       state_d = (rounds == 0) ? StTxLast : StTx;
210     end
211   end
212
213   StTx: begin
214     valid = 1'b1;
215     strb = {IfBytes{1'b1}};
216
217     // transaction accepted
218     if (kmac_data_i.req_ready) begin
219       cnt_en = 1'b1;
220
221       // second to last beat
222       if (cnt == CntWidth'(1'b1)) begin
223         state_d = StTxLast;
224       end
225     end
226   end
227
228   StTxLast: begin
229     valid = 1'b1;
230     last = 1'b1;
231
232     if (adv_en_i) begin
233       strb = AdvByteMask;
234     end else if (id_en_i) begin
235       strb = IdByteMask;
236     end else if (gen_en_i) begin
237       strb = GenByteMask;
238     end
239
240     // transaction accepted
241     cnt_clr = kmac_data_i.req_ready;
242     state_d = kmac_data_i.req_ready ? StOpWait : StTxLast;
243   end
244
245   end
246
247   StOpWait: begin
248     kmac_done_vld = 1'b1;
249     if (kmac_data_i.rsp_valid) begin
250       kmac_error_o = kmac_data_i.error;
251       done_o = 1'b1;
252       state_d = StClean;
253     end
254   end
255
256   StClean: begin
257     cmd_chk = '0;
258     done_o = 1'b1;
259
260     // wait for control side to ack done by waiting start
de-assertion
261     if (!start) begin
262       done_o = 1'b0;
263       clr_err = 1'b1;
264       state_d = StIdle;
265     end
266   end
267
268   // trigger error
269   default: begin
270     // This state is terminal
271     done_o = 1'b1;
272     fsm_error_o = 1'b1;
273   end
274
275 endcase // unique case (state_q)
276
277 // unconditional error transitions
278 // counter errors may disturb the fsm flow and are
279 // treated like fsm errors

```

```

280   if (cnt_err) begin
281     state_d = StError;
282     fsm_error_o = 1;
283     done_o = 1'b1;
284   end
285 end
286
287 // when transaction is not complete, populate the data with
random
288 assign data_o = start && done_o ?
289   {kmac_data_i.digest_s1,
290    kmac_data_i.digest_s0} :
291   {DecoyOutputCopies{entropy_i[1]}},
292   {DecoyOutputCopies{entropy_i[0]}};
293
294 // The input invalid check is done whenever transactions are
ongoing with kmac
295 // once set, it cannot be unset until transactions are fully
complete
296 always_comb begin
297   inputs_invalid_d = inputs_invalid_q;
298
299   if (clr_err) begin
300     inputs_invalid_d = '0;
301   end else if (valid) begin
302     inputs_invalid_d[OpAdvance] = adv_en_i & (in-
puts_invalid_i[OpAdvance] |
303                                           in-
puts_invalid_q[OpAdvance]);
304     inputs_invalid_d[OpGenId] = id_en_i & (in-
puts_invalid_i[OpGenId] |
305                                           in-
puts_invalid_q[OpGenId]);
306     inputs_invalid_d[OpGenSwOut] = gen_en_i & (in-
puts_invalid_i[OpGenSwOut] |
307                                           in-
puts_invalid_q[OpGenSwOut]);
308     inputs_invalid_d[OpGenHwOut] = gen_en_i & (in-
puts_invalid_i[OpGenHwOut] |
309                                           in-
puts_invalid_q[OpGenHwOut]);
310   end
311 end
312
313 // immediately assert errors
314 assign inputs_invalid_o = |inputs_invalid_d;
315
316 // Permute Share 1 of the entropy input once more to get the
decoy data.
317 // Share 0 and 1 are used as is for data_o (connected to the
sideload ports).
318 logic [RandWidth-1:0] decoy_entropy;
319 assign decoy_entropy = perm_data(entropy_i[1], RndCnstRand-
Perm);
320 assign decoy_data = {DecoyCopies{decoy_entropy}};
321
322 logic [CntWidth-1:0] adv_sel, id_sel, gen_sel;
323 assign adv_sel = LastAdvRound - cnt;
324 assign id_sel = LastIdRound - cnt;
325 assign gen_sel = LastGenRound - cnt;
326
327 // The count is maintained as a downcount
328 // so a subtract is necessary to send the right byte
329 // alternatively we can also reverse the order of the input
330 always_comb begin
331   kmac_data_o.data_s0 = decoy_data;
332   if (|cmd_error_o || inputs_invalid_o || fsm_error_o) begin
333     kmac_data_o.data_s0 = decoy_data;
334   end else if (valid && adv_en_i) begin
335     kmac_data_o.data_s0 = adv_data[adv_sel];
336   end else if (valid && id_en_i) begin
337     kmac_data_o.data_s0 = id_data[id_sel];
338   end else if (valid && gen_en_i) begin
339     kmac_data_o.data_s0 = gen_data[gen_sel];
340   end
341 end
342
343 // Key manager operates on unshared data and accepts digest
responses immediately.
344 assign kmac_data_o.data_s1 = '0;
345 assign kmac_data_o.req_valid = valid;
346 assign kmac_data_o.req_last = last;
347 assign kmac_data_o.strb = strb;
348 assign kmac_data_o.rsp_ready = 1'b1;
349
350 // A static interface has no finish response.
351 logic unused_kmac_finish_rsp;
352 assign unused_kmac_finish_rsp = kmac_data_i.rsp_finish;
353
354 // kmac done is asserted outside of expected window
355 // SEC_CM: KMAC_IF_DONE.CTRL.CONSISTENCY
356 logic kmac_done_err_q, kmac_done_err_d;
357 assign kmac_done_err_d = ~kmac_done_vld &
kmac_data_i.rsp_valid |
358                               kmac_done_err_q;
359 assign kmac_done_error_o = kmac_done_err_q;
360

```

```

361
362 // the enables must be 1 hot
363 logic [2:0] enables_d, enables_q, enables_sub;
364 assign enables_d = {adv_en_i, id_en_i, gen_en_i};
365 assign enables_sub = enables_d - 1'b1;
366
367 // cross check to ensure the one-hot command that kicked off
368 // the transaction remains consistent throughout.
369 logic cmd_consty_err_q, cmd_consty_err_d;
370 always_ff @(posedge clk_i or negedge rst_ni) begin
371   if (!rst_ni) begin
372     enables_q <= '0;
373   end else if (cnt_set) begin
374     enables_q <= enables_d;
375   end
376 end
377 assign cmd_consty_err_d = (cmd_chk & (enables_q != en-
ables_d)) |
378                               cmd_consty_err_q;
379
380 // if a one hot error occurs, latch onto it permanently
381 // SEC_CM: KMAC_IF_CMD.CTRL.CONSISTENCY
382 logic one_hot_err_q, one_hot_err_d;
383 assign one_hot_err_d = |(enables_d & enables_sub) |
one_hot_err_q;
384
385 always_ff @(posedge clk_i or negedge rst_ni) begin
386   if (!rst_ni) begin
387     one_hot_err_q <= '0;
388     kmac_done_err_q <= '0;
389     cmd_consty_err_q <= '0;
390   end else begin
391     one_hot_err_q <= one_hot_err_d;
392     kmac_done_err_q <= kmac_done_err_d;
393     cmd_consty_err_q <= cmd_consty_err_d;
394   end
395 end
396
397 // command error occurs if kmac errors or if the command it-
self is invalid
398 assign cmd_error_o = one_hot_err_q | cmd_consty_err_q;
399
400 // request entropy to churn whenever a transaction is accep-
ted
401 assign prng_en_o = kmac_data_o.req_valid &
kmac_data_i.req_ready;
402
403 // as long as we are transmitting, the strobe should never
be 0.
404 `ASSERT(LastStrb_A, valid |-> strb != '0)
405
406
407
408 endmodule // keymgr_kmac_if

```

8. lowRISC_opentitan/hw/ip/keymgr/rtl/ keymgr_op_state_ctrl.sv (110 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Key manager operation state control
6 //
7
8 `include "prim_assert.sv"
9
10 module keymgr_op_state_ctrl
11   import keymgr_pkg::*;
12   import keymgr_reg_pkg::*;
13 (
14   input clk_i,
15   input rst_ni,
16
17   input adv_req_i,
18   input dis_req_i,
19   input id_req_i,
20   input gen_req_i,
21   input [CdiWidth-1:0] cnt_i,
22   output logic op_ack_o,
23   output logic op_busy_o,
24   output logic op_update_o,
25
26   input kmac_done_i,
27   output logic adv_en_o,
28   output logic id_en_o,
29   output logic gen_en_o,
30
31   output logic op_fsm_err_o
32 );
33
34 localparam int OpStateWidth = 8;
35 typedef enum logic [OpStateWidth-1:0] {
36   StIdle = 8'b10010101,
37   StAdv = 8'b00101000,
38   StAdvAck = 8'b01000011,
39   StWait = 8'b11111110
40 } state_e;

```

```

42
43 state_e state_q, state_d;
44 PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
state_e, StIdle)
45
46 logic gen_en;
47 assign id_en_o = gen_en & id_req_i;
48 assign gen_en_o = gen_en & gen_req_i;
49
50 always_comb begin
51 state_d = state_q;
52 op_update_o = 1'b0;
53 op_ack_o = 1'b0;
54 op_busy_o = 1'b1;
55
56 // output to kmac interface
57 adv_en_o = 1'b0;
58
59 gen_en = 1'b0;
60 op_fsm_err_o = 1'b0;
61
62 unique case (state_q)
63 StIdle: begin
64 op_busy_o = 1'b1;
65 if (adv_req_i || dis_req_i) begin
66 state_d = StAdv;
67 end else if (id_req_i || gen_req_i) begin
68 state_d = StWait;
69 end
70 end
71
72 StAdv: begin
73 adv_en_o = 1'b1;
74
75 if (kmac_done_i && (int'(cnt_i) == CDIs-1)) begin
76 op_ack_o = 1'b1;
77 state_d = StIdle;
78 end else if (kmac_done_i && (int'(cnt_i) < CDIs-1))
begin
79 op_update_o = 1'b1;
80 state_d = StAdvAck;
81 end
82 end
83
84 // drop adv_en_o to allow kmac interface handshake
85 StAdvAck: begin
86 state_d = StAdv;
87 end
88
89 // Not an advanced operation
90 StWait: begin
91 gen_en = 1'b1;
92
93 if (kmac_done_i) begin
94 op_ack_o = 1'b1;
95 state_d = StIdle;
96 end
97 end
98
99 // error state
100 default: begin
101 // allow completion of transaction
102 op_ack_o = 1'b1;
103 op_fsm_err_o = 1'b1;
104 end
105
106 endcase // unique case (adv_state_q)
107 end
108
109
110 endmodule // keymgr_op_state_ctrl

```

9. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_pkg.sv (279 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // key manager package
6 //
7
8 package keymgr_pkg;
9
10 parameter int KeyWidth = 256;
11 parameter int CDIs = 2; // 2 different CDIs, sealing / at-
testation
12 parameter int CdiWidth = prim_util_pkg::vbits(CDIs);
13 parameter int OtbnKeyWidth = 384;
14 parameter int DigestWidth = 128; // uses truncated hash
15 parameter int KmacDataIfWidth = 64; // KMAC interface data
width
16 parameter int KeyMgrStages = 3; // Number of key manager
stages (creator, ownerInt, owner)
17 parameter int SwBindingWidth = 32 * keym-
gr_reg_pkg::NumSwBindingReg;
18 parameter int SaltWidth = 32 * keymgr_reg_pkg::NumSaltReg;
19 parameter int Shares = 2; // number of key shares

```

```

20 parameter int KeyVersionWidth = 32; // Key version length
for individual DICE stage
21
22 // These should be defined in another module's package
23 parameter int HealthStateWidth = 128;
24 parameter int DevIdWidth = 256;
25 parameter int MaxWidth = 256;
26
27 // Default seeds
28 // These have been generated with the following command by
incrementing the --seed argument
29 // for every seed.
30 // util/design/gen-lfsr-seed.py --width 256 --seed 7535190
--prefix ""
31 typedef logic [KeyWidth-1:0] seed_t;
32 parameter seed_t RndCnstRevisionSeedDefault =
33
34 256'h69802e51_bacf8874_e650d692_e3d8a646_2d3f158f_0bf7961d_d346f880_
b4d52170;
35 parameter seed_t RndCnstCreatorIdentitySeedDefault =
36
37 256'h344bfff84_d3edf83e_aa565c5e_6c2d22f2_75283734_15cf6d83_32cd3752_
731817c;
38 parameter seed_t RndCnstOwnerIntIdentitySeedDefault =
39
40 256'h32a37b-
f9_2627382b_47180548_1fc4b847_a7eba1ce_c13e80ab_030f847e_9bd902e;
41 parameter seed_t RndCnstOwnerIdentitySeedDefault =
42
43 256'hc1ba416c_822ad676_e4f54f36_33d8dc7e_00ccfb8b_92209f1b_d3229348_
275ebda;
44 parameter seed_t RndCnstSoftOutputSeedDefault =
45
46 256'h23cbe85e_62e39992_4ab8d6a6_acbf12f0_0ca6f488_63eaa428_ffb11b26_
c5282b14;
47 parameter seed_t RndCnstHardOutputSeedDefault =
48
49 256'h877e-
f0b9_aceaeefc5_693b1aa7_e43a7f3d_ee7d63b6_4e73e182_b49cbf87_2872f2c;
50 // Target based derivation seeds
51 // These are used during the generation stages for sideload
52 parameter seed_t RndCnstNoneSeedDefault =
53
54 256'h6013ba4d_8d04fe4f_80cb0472_536c9679_08b82f1d_98a4e405_cb4680ec_
3a4a8d7a;
55 parameter seed_t RndCnstAesSeedDefault =
56
57 256'he16a8-
fa9_5b613cd5_fb9ad23f_bd8347e1_64e45dac_5d08a41b_e83caa37_e03d9482;
58 parameter seed_t RndCnstKmacSeedDefault =
59
60 256'hc57f4c0b_b308e83f_3fc4bc63_d87dd67d_9071dc1c_e19484c8_3c94fb97_
dd634369;
61 parameter seed_t RndCnstOtbnSeedDefault =
62
63 256'hcb-
cb4d2d_0abeb81b_ca7451ae_d1e2479d_ba13530a_d046b945_646aa127_bd4f6a3
8;
64 parameter seed_t RndCnstCdiDefault =
65
66 256'h54180905_d14c1d2f_2dda1522_f332bc0e_fcd6b92f_f0f9db75_3a9a9544_
26a42eab;
67 // Default Lfsr configurations
68 // These LFSR parameters have been generated with
69 // $ util/design/gen-lfsr-seed.py --width 64 --seed
691876113 --prefix ""
70 parameter int LfsrWidth = 64;
71 typedef logic [LfsrWidth-1:0] lfsr_seed_t;
72 typedef logic [LfsrWidth-1:0][$clog2(LfsrWidth)-1:0]
lfsr_perm_t;
73 parameter lfsr_seed_t RndCnstLfsrSeedDefault =
74
75 64'h22d326255bd24320;
76 parameter lfsr_perm_t RndCnstLfsrPermDefault = {
77
78 128'h16108c9f9008aa37e5118d1dec1df64a7,
79 256'h24f3f1b73537f42d38383ee8f897286df81d49-
ab54b6bbb666cbd1a16c41252
80 };
81 // Random permutation
82 parameter int RandWidth = LfsrWidth / 2;
83 typedef logic [RandWidth-1:0][$clog2(RandWidth)-1:0]
rand_perm_t;
84 parameter rand_perm_t RndCnstRandPermDefault = {
85
86 160'h62089181d2a6be2ce145e2e27099ededbd7dceb0
87 };
88 // Width calculations
89 // These are the largest calculations in use across all
stages
90 parameter int AdvDataWidth = SwBindingWidth + 2*KeyWidth +
DevIdWidth + HealthStateWidth;
91 parameter int IdDataWidth = KeyWidth;
92 // key version + salt + key ID + constant
93 parameter int GenDataWidth = 32 + SaltWidth + KeyWidth*2;
94 parameter int StageWidth = $clog2(KeyMgrStages);
95 // Max Payload Width to derivation function
96 // see security strength description https://keccak.team/
keccak.html
97 // The max width here is chosen arbitrarily to ensure we do
not get out of hand.
98
99 // Since KMAC is a MAC operation, the data can be as long as we
need.

```

```

88 parameter int KDFMaxWidth = 1984;
89
90 // Enumeration for operations
91 typedef enum logic [1:0] {
92     Creator,
93     OwnerInt,
94     Owner,
95     Disable
96 } keymgr_stage_e;
97
98 // Enumeration for sideload sel
99 typedef enum logic [1:0] {
100     None,
101     Aes,
102     Kmac,
103     Otbn
104 } keymgr_key_dest_e;
105
106 // Enumeration for actual key slot idx
107 typedef enum logic [1:0] {
108     AesIdx,
109     KmacIdx,
110     OtbnIdx,
111     LastIdx
112 } keymgr_sideload_slot_idx_e;
113
114 // Enumeration for key select
115 typedef enum logic {
116     HwKey = 0,
117     SwKey = 1
118 } keymgr_gen_out_e;
119
120 // Enumeration for operation
121 typedef enum logic [2:0] {
122     OpAdvance = 0,
123     OpGenId = 1,
124     OpGenSwOut = 2,
125     OpGenHwOut = 3,
126     OpDisable = 4
127 } keymgr_ops_e;
128
129 // Enumeration for working state exposed to software
130 typedef enum logic [2:0] {
131     StReset,
132     StInit,
133     StCreatorRootKey,
134     StOwnerIntKey,
135     StOwnerKey,
136     StDisabled,
137     StInvalid
138 } keymgr_working_state_e;
139
140 // Enumeration for operation status
141 typedef enum logic [1:0] {
142     OpIdle = 0,
143     OpWip = 1,
144     OpDoneSuccess = 2,
145     OpDoneFail = 3
146 } keymgr_op_status_e;
147
148 // keymgr has 4 categories of errors
149 // sync errors - recoverable errors that happen during
keymgr operation
150 // async errors - recoverable errors that happen asynchron-
ously
151 // sync faults - fatal errors that happen during keymgr op-
eration
152 // async faults - fatal errors that happen asynchronously
153
154 typedef enum logic [1:0] {
155     SyncErrInvalidOp,
156     SyncErrInvalidIn,
157     SyncErrLastIdx
158 } keymgr_sync_error_e;
159
160 typedef enum logic [1:0] {
161     AsyncErrShadowUpdate,
162     AsyncErrLastIdx
163 } keymgr_async_error_e;
164
165 typedef enum logic [1:0] {
166     SyncFaultKmacOp,
167     SyncFaultKmacOut,
168     SyncFaultSideSel,
169     SyncFaultLastIdx
170 } keymgr_sync_fault_e;
171
172 typedef enum logic [3:0] {
173     AsyncFaultKmacCmd,
174     AsyncFaultKmacFsm,
175     AsyncFaultKmacDone,
176     AsyncFaultRegIntg,
177     AsyncFaultShadow,
178     AsyncFaultFsmIntg,
179     AsyncFaultFsmChk,
180     AsyncFaultCntErr,
181     AsyncFaultRCntErr,
182     AsyncFaultSideErr,
183     AsyncFaultKeyEcc,
184     AsyncFaultLastIdx
185 } keymgr_async_fault_e;
186
187
188 // Bit position of error code
189 // Error is encoded as 1 error per bit
190 typedef enum logic [2:0] {
191     ErrInvalidOp,
192     ErrInvalidIn,
193     ErrShadowUpdate,
194     ErrLastPos
195 } keymgr_err_pos_e;
196
197 // Bit position of fault status
198 typedef enum logic [3:0] {
199     FaultKmacCmd,
200     FaultKmacFsm,
201     FaultKmacDone,
202     FaultKmacOp,
203     FaultKmacOut,
204     FaultRegIntg,
205     FaultShadow,
206     FaultCtrlFsm,
207     FaultCtrlFsmChk,
208     FaultCtrlCnt,
209     FaultReseedCnt,
210     FaultSideFsm,
211     FaultSideSel,
212     FaultKeyEcc,
213     FaultLastPos
214 } keymgr_fault_pos_e;
215
216 typedef enum logic [2:0] {
217     KeyUpdateIdle,
218     KeyUpdateRandom,
219     KeyUpdateRoot,
220     KeyUpdateKmac,
221     KeyUpdateWipe
222 } keymgr_key_update_e;
223
224 typedef enum logic [2:0] {
225     SideloadClrIdle,
226     SideloadClrAes,
227     SideloadClrKmac,
228     SideloadClrOtbn
229 } keymgr_sideload_clr_e;
230
231 // Key connection to various symmetric modules
232 typedef struct packed {
233     logic valid;
234     logic [Shares-1:0][KeyWidth-1:0] key;
235 } hw_key_req_t;
236
237 // Key connection to otbn
238 typedef struct packed {
239     logic valid;
240     logic [Shares-1:0][OtbnKeyWidth-1:0] key;
241 } otbn_key_req_t;
242
243 parameter hw_key_req_t HW_KEY_REQ_DEFAULT = '{
244     valid: 1'b0,
245     key: {Shares{KeyWidth'(32'hDEADBEEF)}}
246 };
247
248 parameter otbn_key_req_t OTBN_KEY_REQ_DEFAULT = '{
249     valid: 1'b0,
250     key: {Shares{OtbnKeyWidth'(32'hDEADBEEF)}}
251 };
252
253 // The following structs should be sourced from other mod-
ules
254 // defined here temporarily
255
256 // lc keymgr enable usage
257 typedef enum logic [1:0] {
258     KeyMgrEnCtrl,
259     KeyMgrEnCfgEn,
260     KeyMgrEnSwBindingEn,
261     KeyMgrEnLast
262 } keymgr_lc_en_usage_e;
263
264 // perm_data
265 function automatic logic[RandWidth-1:0] perm_data (logic
[RandWidth-1:0] data,
266     rand_perm_t perm_sel);
267
268     for (int k = 0; k < 32; k++) begin : gen_data_loop
269         perm_data[k] = data[perm_sel[k]];
270     end
271
272 endfunction
273
274 // checks for all 0's or all 1's of value
275 function automatic logic valid_data_chk (logic [Key-
width-1:0] value);
276     return |value & ~&value;
277 endfunction
278
279 endpackage : keymgr_pkg

```

10. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_reg_pkg.sv

(547 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Package auto-generated by `reggen` containing data
  structure
6
7 package keymgr_reg_pkg;
8
9 // Param list
10 parameter int NumSaltReg = 8;
11 parameter int NumSwBindingReg = 8;
12 parameter int NumOutReg = 8;
13 parameter int NumKeyVersion = 1;
14 parameter int NumAlerts = 2;
15
16 // Address widths within the block
17 parameter int BlockAw = 8;
18
19 // Number of registers for every interface
20 parameter int NumRegs = 63;
21
22 // Alert indices
23 typedef enum int {
24     AlertRecovOperationErrIdx = 0,
25     AlertFatalFaultErrIdx = 1
26 } keymgr_alert_idx_t;
27
28 ///////////////////////////////////////////////////
29 // Typedefs for registers //
30 ///////////////////////////////////////////////////
31
32 typedef struct packed {
33     logic q;
34 } keymgr_reg2hw_intr_state_reg_t;
35
36 typedef struct packed {
37     logic q;
38 } keymgr_reg2hw_intr_enable_reg_t;
39
40 typedef struct packed {
41     logic q;
42     logic qe;
43 } keymgr_reg2hw_intr_test_reg_t;
44
45 typedef struct packed {
46     struct packed {
47         logic q;
48         logic qe;
49     } fatal_fault_err;
50     struct packed {
51         logic q;
52         logic qe;
53     } recov_operation_err;
54 } keymgr_reg2hw_alert_test_reg_t;
55
56 typedef struct packed {
57     logic q;
58 } keymgr_reg2hw_start_reg_t;
59
60 typedef struct packed {
61     struct packed {
62         logic [1:0] q;
63     } dest_sel;
64     struct packed {
65         logic q;
66     } cdi_sel;
67     struct packed {
68         logic [2:0] q;
69     } operation;
70 } keymgr_reg2hw_control_shadowed_reg_t;
71
72 typedef struct packed {
73     logic [2:0] q;
74 } keymgr_reg2hw_sideload_clear_reg_t;
75
76 typedef struct packed {
77     logic [15:0] q;
78 } keymgr_reg2hw_reseed_interval_shadowed_reg_t;
79
80 typedef struct packed {
81     logic q;
82     logic qe;
83 } keymgr_reg2hw_sw_binding_regwen_reg_t;
84
85 typedef struct packed {
86     logic [31:0] q;
87 } keymgr_reg2hw_sealing_sw_binding_mreg_t;
88
89 typedef struct packed {
90     logic [31:0] q;
91 } keymgr_reg2hw_attest_sw_binding_mreg_t;
92
93 typedef struct packed {
94     logic [31:0] q;
95 } keymgr_reg2hw_salt_mreg_t;
96
97 typedef struct packed {
98     logic [31:0] q;
99 } keymgr_reg2hw_key_version_mreg_t;
100
101 typedef struct packed {
102     logic [31:0] q;
103 } keymgr_reg2hw_max_creator_key_ver_shadowed_reg_t;
104
105 typedef struct packed {
106     logic [31:0] q;
107 } keymgr_reg2hw_max_owner_int_key_ver_shadowed_reg_t;
108
109 typedef struct packed {
110     logic [31:0] q;
111 } keymgr_reg2hw_max_owner_key_ver_shadowed_reg_t;
112
113 typedef struct packed {
114     struct packed {
115         logic q;
116     } key_ecc;
117     struct packed {
118         logic q;
119     } side_ctrl_sel;
120     struct packed {
121         logic q;
122     } side_ctrl_fsm;
123     struct packed {
124         logic q;
125     } reseed_cnt;
126     struct packed {
127         logic q;
128     } ctrl_fsm_cnt;
129     struct packed {
130         logic q;
131     } ctrl_fsm_chk;
132     struct packed {
133         logic q;
134     } ctrl_fsm_intg;
135     struct packed {
136         logic q;
137     } shadow;
138     struct packed {
139         logic q;
140     } regfile_intg;
141     struct packed {
142         logic q;
143     } kmac_out;
144     struct packed {
145         logic q;
146     } kmac_op;
147     struct packed {
148         logic q;
149     } kmac_done;
150     struct packed {
151         logic q;
152     } kmac_fsm;
153     struct packed {
154         logic q;
155     } cmd;
156 } keymgr_reg2hw_fault_status_reg_t;
157
158 typedef struct packed {
159     logic d;
160     logic de;
161 } keymgr_hw2reg_intr_state_reg_t;
162
163 typedef struct packed {
164     logic d;
165 } keymgr_hw2reg_cfg_regwen_reg_t;
166
167 typedef struct packed {
168     logic d;
169     logic de;
170 } keymgr_hw2reg_start_reg_t;
171
172 typedef struct packed {
173     logic d;
174 } keymgr_hw2reg_sw_binding_regwen_reg_t;
175
176 typedef struct packed {
177     logic [31:0] d;
178     logic de;
179 } keymgr_hw2reg_sw_share0_output_mreg_t;
180
181 typedef struct packed {
182     logic [31:0] d;
183     logic de;
184 } keymgr_hw2reg_sw_share1_output_mreg_t;
185
186 typedef struct packed {
187     logic [2:0] d;
188     logic de;
189 } keymgr_hw2reg_working_state_reg_t;
190
191 typedef struct packed {
192     logic [1:0] d;
193     logic de;

```



```

194 } keymgr_hw2reg_op_status_reg_t;
195
196 typedef struct packed {
197     struct packed {
198         logic d;
199         logic de;
200     } invalid_shadow_update;
201     struct packed {
202         logic d;
203         logic de;
204     } invalid_kmac_input;
205     struct packed {
206         logic d;
207         logic de;
208     } invalid_op;
209 } keymgr_hw2reg_err_code_reg_t;
210
211 typedef struct packed {
212     struct packed {
213         logic d;
214         logic de;
215     } key_ecc;
216     struct packed {
217         logic d;
218         logic de;
219     } side_ctrl_sel;
220     struct packed {
221         logic d;
222         logic de;
223     } side_ctrl_fsm;
224     struct packed {
225         logic d;
226         logic de;
227     } reseed_cnt;
228     struct packed {
229         logic d;
230         logic de;
231     } ctrl_fsm_cnt;
232     struct packed {
233         logic d;
234         logic de;
235     } ctrl_fsm_chk;
236     struct packed {
237         logic d;
238         logic de;
239     } ctrl_fsm_intg;
240     struct packed {
241         logic d;
242         logic de;
243     } shadow;
244     struct packed {
245         logic d;
246         logic de;
247     } regfile_intg;
248     struct packed {
249         logic d;
250         logic de;
251     } kmac_out;
252     struct packed {
253         logic d;
254         logic de;
255     } kmac_op;
256     struct packed {
257         logic d;
258         logic de;
259     } kmac_done;
260     struct packed {
261         logic d;
262         logic de;
263     } kmac_fsm;
264     struct packed {
265         logic d;
266         logic de;
267     } cmd;
268 } keymgr_hw2reg_fault_status_reg_t;
269
270 typedef struct packed {
271     struct packed {
272         logic d;
273         logic de;
274     } invalid_digest;
275     struct packed {
276         logic d;
277         logic de;
278     } invalid_key;
279     struct packed {
280         logic d;
281         logic de;
282     } invalid_key_version;
283     struct packed {
284         logic d;
285         logic de;
286     } invalid_health_state;
287     struct packed {
288         logic d;
289         logic de;
290     } invalid_dev_id;
291     struct packed {
292         logic d;
293         logic de;
294     } invalid_owner_seed;

```

```

295     struct packed {
296         logic d;
297         logic de;
298     } invalid_creator_seed;
299 } keymgr_hw2reg_debug_reg_t;
300
301 // Register -> HW type
302 typedef struct packed {
303     keymgr_reg2hw_intr_state_reg_t intr_state; // [945:945]
304     keymgr_reg2hw_intr_enable_reg_t intr_enable; // [944:944]
305     keymgr_reg2hw_intr_test_reg_t intr_test; // [943:942]
306     keymgr_reg2hw_alert_test_reg_t alert_test; // [941:938]
307     keymgr_reg2hw_start_reg_t start; // [937:937]
308     keymgr_reg2hw_control_shadowed_reg_t control_shadowed; //
[936:931]
309     keymgr_reg2hw_sideload_clear_reg_t sideload_clear; //
[930:928]
310     keymgr_reg2hw_reseed_interval_shadowed_reg_t re-
seed_interval_shadowed; // [927:912]
311     keymgr_reg2hw_sw_binding_regwen_reg_t sw_bind-
ing_regwen; // [911:910]
312     keymgr_reg2hw_sealing_sw_binding_mreg_t [7:0] seal-
ing_sw_binding; // [909:654]
313     keymgr_reg2hw_attest_sw_binding_mreg_t [7:0] at-
test_sw_binding; // [653:398]
314     keymgr_reg2hw_salt_mreg_t [7:0] salt; // [397:142]
315     keymgr_reg2hw_key_version_mreg_t [0:0] key_version; //
[141:110]
316     keymgr_reg2hw_max_creator_key_ver_shadowed_reg_t max_cre-
ator_key_ver_shadowed; // [109:78]
317     keymgr_reg2hw_max_owner_int_key_ver_shadowed_reg_t
max_owner_int_key_ver_shadowed; // [77:46]
318     keymgr_reg2hw_max_owner_key_ver_shadowed_reg_t max_ow-
ner_key_ver_shadowed; // [45:14]
319     keymgr_reg2hw_fault_status_reg_t fault_status; // [13:0]
320 } keymgr_reg2hw_t;
321
322 // HW -> register type
323 typedef struct packed {
324     keymgr_hw2reg_intr_state_reg_t intr_state; // [588:587]
325     keymgr_hw2reg_cfg_regwen_reg_t cfg_regwen; // [586:586]
326     keymgr_hw2reg_start_reg_t start; // [585:584]
327     keymgr_hw2reg_sw_binding_regwen_reg_t sw_bind-
ing_regwen; // [583:583]
328     keymgr_hw2reg_sw_share0_output_mreg_t [7:0]
sw_share0_output; // [582:319]
329     keymgr_hw2reg_sw_share1_output_mreg_t [7:0]
sw_share1_output; // [318:55]
330     keymgr_hw2reg_working_state_reg_t working_state; //
[54:51]
331     keymgr_hw2reg_op_status_reg_t op_status; // [50:48]
332     keymgr_hw2reg_err_code_reg_t err_code; // [47:42]
333     keymgr_hw2reg_fault_status_reg_t fault_status; // [41:14]
334     keymgr_hw2reg_debug_reg_t debug; // [13:0]
335 } keymgr_hw2reg_t;
336
337 // Register offsets
338 parameter logic [BlockAw-1:0] KEYMGR_INTR_STATE_OFFSET = 8'h
0;
339 parameter logic [BlockAw-1:0] KEYMGR_INTR_ENABLE_OFFSET =
8'h 4;
340 parameter logic [BlockAw-1:0] KEYMGR_INTR_TEST_OFFSET = 8'h
8;
341 parameter logic [BlockAw-1:0] KEYMGR_ALERT_TEST_OFFSET = 8'h
c;
342 parameter logic [BlockAw-1:0] KEYMGR_CFG_REGWEN_OFFSET = 8'h
10;
343 parameter logic [BlockAw-1:0] KEYMGR_START_OFFSET = 8'h 14;
344 parameter logic [BlockAw-1:0] KEYMGR_CONTROL_SHADOWED_OFFSET =
8'h 18;
345 parameter logic [BlockAw-1:0] KEYMGR_SIDELOAD_CLEAR_OFFSET =
8'h 1c;
346 parameter logic [BlockAw-1:0] KEYM-
GR_RESEED_INTERVAL_REGWEN_OFFSET = 8'h 20;
347 parameter logic [BlockAw-1:0] KEYM-
GR_RESEED_INTERVAL_SHADOWED_OFFSET = 8'h 24;
348 parameter logic [BlockAw-1:0] KEYM-
GR_SW_BINDING_REGWEN_OFFSET = 8'h 28;
349 parameter logic [BlockAw-1:0] KEYM-
GR_SEALING_SW_BINDING_0_OFFSET = 8'h 2c;
350 parameter logic [BlockAw-1:0] KEYM-
GR_SEALING_SW_BINDING_1_OFFSET = 8'h 30;
351 parameter logic [BlockAw-1:0] KEYM-
GR_SEALING_SW_BINDING_2_OFFSET = 8'h 34;
352 parameter logic [BlockAw-1:0] KEYM-
GR_SEALING_SW_BINDING_3_OFFSET = 8'h 38;
353 parameter logic [BlockAw-1:0] KEYM-
GR_SEALING_SW_BINDING_4_OFFSET = 8'h 3c;
354 parameter logic [BlockAw-1:0] KEYM-
GR_SEALING_SW_BINDING_5_OFFSET = 8'h 40;
355 parameter logic [BlockAw-1:0] KEYM-
GR_SEALING_SW_BINDING_6_OFFSET = 8'h 44;
356 parameter logic [BlockAw-1:0] KEYM-
GR_SEALING_SW_BINDING_7_OFFSET = 8'h 48;
357 parameter logic [BlockAw-1:0] KEYM-
GR_ATTEST_SW_BINDING_0_OFFSET = 8'h 4c;

```

```

358 parameter logic [BlockAw-1:0] KEYM-
GR_ATTEST_SW_BINDING_1_OFFSET = 8'h 50;
359 parameter logic [BlockAw-1:0] KEYM-
GR_ATTEST_SW_BINDING_2_OFFSET = 8'h 54;
360 parameter logic [BlockAw-1:0] KEYM-
GR_ATTEST_SW_BINDING_3_OFFSET = 8'h 58;
361 parameter logic [BlockAw-1:0] KEYM-
GR_ATTEST_SW_BINDING_4_OFFSET = 8'h 5c;
362 parameter logic [BlockAw-1:0] KEYM-
GR_ATTEST_SW_BINDING_5_OFFSET = 8'h 60;
363 parameter logic [BlockAw-1:0] KEYM-
GR_ATTEST_SW_BINDING_6_OFFSET = 8'h 64;
364 parameter logic [BlockAw-1:0] KEYM-
GR_ATTEST_SW_BINDING_7_OFFSET = 8'h 68;
365 parameter logic [BlockAw-1:0] KEYMGR_SALT_0_OFFSET = 8'h 6c;
366 parameter logic [BlockAw-1:0] KEYMGR_SALT_1_OFFSET = 8'h 70;
367 parameter logic [BlockAw-1:0] KEYMGR_SALT_2_OFFSET = 8'h 74;
368 parameter logic [BlockAw-1:0] KEYMGR_SALT_3_OFFSET = 8'h 78;
369 parameter logic [BlockAw-1:0] KEYMGR_SALT_4_OFFSET = 8'h 7c;
370 parameter logic [BlockAw-1:0] KEYMGR_SALT_5_OFFSET = 8'h 80;
371 parameter logic [BlockAw-1:0] KEYMGR_SALT_6_OFFSET = 8'h 84;
372 parameter logic [BlockAw-1:0] KEYMGR_SALT_7_OFFSET = 8'h 88;
373 parameter logic [BlockAw-1:0] KEYMGR_KEY_VERSION_OFFSET =
8'h 8c;
374 parameter logic [BlockAw-1:0] KEYM-
GR_MAX_CREATOR_KEY_VER_REGWEN_OFFSET = 8'h 90;
375 parameter logic [BlockAw-1:0] KEYM-
GR_MAX_CREATOR_KEY_VER_SHADOWED_OFFSET = 8'h 94;
376 parameter logic [BlockAw-1:0] KEYM-
GR_MAX_OWNER_INT_KEY_VER_REGWEN_OFFSET = 8'h 98;
377 parameter logic [BlockAw-1:0] KEYM-
GR_MAX_OWNER_INT_KEY_VER_SHADOWED_OFFSET = 8'h 9c;
378 parameter logic [BlockAw-1:0] KEYM-
GR_MAX_OWNER_KEY_VER_REGWEN_OFFSET = 8'h a0;
379 parameter logic [BlockAw-1:0] KEYM-
GR_MAX_OWNER_KEY_VER_SHADOWED_OFFSET = 8'h a4;
380 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE0_OUTPUT_0_OFFSET = 8'h a8;
381 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE0_OUTPUT_1_OFFSET = 8'h ac;
382 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE0_OUTPUT_2_OFFSET = 8'h b0;
383 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE0_OUTPUT_3_OFFSET = 8'h b4;
384 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE0_OUTPUT_4_OFFSET = 8'h b8;
385 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE0_OUTPUT_5_OFFSET = 8'h bc;
386 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE0_OUTPUT_6_OFFSET = 8'h c0;
387 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE0_OUTPUT_7_OFFSET = 8'h c4;
388 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE1_OUTPUT_0_OFFSET = 8'h c8;
389 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE1_OUTPUT_1_OFFSET = 8'h cc;
390 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE1_OUTPUT_2_OFFSET = 8'h d0;
391 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE1_OUTPUT_3_OFFSET = 8'h d4;
392 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE1_OUTPUT_4_OFFSET = 8'h d8;
393 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE1_OUTPUT_5_OFFSET = 8'h dc;
394 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE1_OUTPUT_6_OFFSET = 8'h e0;
395 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE1_OUTPUT_7_OFFSET = 8'h e4;
396 parameter logic [BlockAw-1:0] KEYMGR_WORKING_STATE_OFFSET =
8'h e8;
397 parameter logic [BlockAw-1:0] KEYMGR_OP_STATUS_OFFSET = 8'h
ec;
398 parameter logic [BlockAw-1:0] KEYMGR_ERR_CODE_OFFSET = 8'h
f0;
399 parameter logic [BlockAw-1:0] KEYMGR_FAULT_STATUS_OFFSET =
8'h f4;
400 parameter logic [BlockAw-1:0] KEYMGR_DEBUG_OFFSET = 8'h f8;
401
402 // Reset values for hwext registers and their fields
403 parameter logic [0:0] KEYMGR_INTR_TEST_RESVAL = 1'h 0;
404 parameter logic [0:0] KEYMGR_INTR_TEST_OP_DONE_RESVAL = 1'h
0;
405 parameter logic [1:0] KEYMGR_ALERT_TEST_RESVAL = 2'h 0;
406 parameter logic [0:0] KEYM-
GR_ALERT_TEST_RECOV_OPERATION_ERR_RESVAL = 1'h 0;
407 parameter logic [0:0] KEYM-
GR_ALERT_TEST_FATAL_FAULT_ERR_RESVAL = 1'h 0;
408 parameter logic [0:0] KEYMGR_CFG_REGWEN_RESVAL = 1'h 1;
409 parameter logic [0:0] KEYMGR_CFG_REGWEN_EN_RESVAL = 1'h 1;
410 parameter logic [0:0] KEYMGR_SW_BINDING_REGWEN_RESVAL = 1'h
1;
411 parameter logic [0:0] KEYMGR_SW_BINDING_REGWEN_EN_RESVAL =
1'h 1;
412
413 // Register index
414 typedef enum int {
415     KEYMGR_INTR_STATE,
416     KEYMGR_INTR_ENABLE,
417     KEYMGR_INTR_TEST,
418     KEYMGR_ALERT_TEST,
419     KEYMGR_CFG_REGWEN,

```

```

420     KEYMGR_START,
421     KEYMGR_CONTROL_SHADOWED,
422     KEYMGR_SIDELOAD_CLEAR,
423     KEYMGR_RESEED_INTERVAL_REGWEN,
424     KEYMGR_RESEED_INTERVAL_SHADOWED,
425     KEYMGR_SW_BINDING_REGWEN,
426     KEYMGR_SEALING_SW_BINDING_0,
427     KEYMGR_SEALING_SW_BINDING_1,
428     KEYMGR_SEALING_SW_BINDING_2,
429     KEYMGR_SEALING_SW_BINDING_3,
430     KEYMGR_SEALING_SW_BINDING_4,
431     KEYMGR_SEALING_SW_BINDING_5,
432     KEYMGR_SEALING_SW_BINDING_6,
433     KEYMGR_SEALING_SW_BINDING_7,
434     KEYMGR_ATTEST_SW_BINDING_0,
435     KEYMGR_ATTEST_SW_BINDING_1,
436     KEYMGR_ATTEST_SW_BINDING_2,
437     KEYMGR_ATTEST_SW_BINDING_3,
438     KEYMGR_ATTEST_SW_BINDING_4,
439     KEYMGR_ATTEST_SW_BINDING_5,
440     KEYMGR_ATTEST_SW_BINDING_6,
441     KEYMGR_ATTEST_SW_BINDING_7,
442     KEYMGR_SALT_0,
443     KEYMGR_SALT_1,
444     KEYMGR_SALT_2,
445     KEYMGR_SALT_3,
446     KEYMGR_SALT_4,
447     KEYMGR_SALT_5,
448     KEYMGR_SALT_6,
449     KEYMGR_SALT_7,
450     KEYMGR_KEY_VERSION,
451     KEYMGR_MAX_CREATOR_KEY_VER_REGWEN,
452     KEYMGR_MAX_CREATOR_KEY_VER_SHADOWED,
453     KEYMGR_MAX_OWNER_INT_KEY_VER_REGWEN,
454     KEYMGR_MAX_OWNER_INT_KEY_VER_SHADOWED,
455     KEYMGR_MAX_OWNER_KEY_VER_REGWEN,
456     KEYMGR_MAX_OWNER_KEY_VER_SHADOWED,
457     KEYMGR_SW_SHARE0_OUTPUT_0,
458     KEYMGR_SW_SHARE0_OUTPUT_1,
459     KEYMGR_SW_SHARE0_OUTPUT_2,
460     KEYMGR_SW_SHARE0_OUTPUT_3,
461     KEYMGR_SW_SHARE0_OUTPUT_4,
462     KEYMGR_SW_SHARE0_OUTPUT_5,
463     KEYMGR_SW_SHARE0_OUTPUT_6,
464     KEYMGR_SW_SHARE0_OUTPUT_7,
465     KEYMGR_SW_SHARE1_OUTPUT_0,
466     KEYMGR_SW_SHARE1_OUTPUT_1,
467     KEYMGR_SW_SHARE1_OUTPUT_2,
468     KEYMGR_SW_SHARE1_OUTPUT_3,
469     KEYMGR_SW_SHARE1_OUTPUT_4,
470     KEYMGR_SW_SHARE1_OUTPUT_5,
471     KEYMGR_SW_SHARE1_OUTPUT_6,
472     KEYMGR_SW_SHARE1_OUTPUT_7,
473     KEYMGR_WORKING_STATE,
474     KEYMGR_OP_STATUS,
475     KEYMGR_ERR_CODE,
476     KEYMGR_FAULT_STATUS,
477     KEYMGR_DEBUG
478 } keymgr_id_e;
479
480 // Register width information to check illegal writes
481 parameter logic [3:0] KEYMGR_PERMIT [63] = '{
482     4'b 0001, // index[ 0] KEYMGR_INTR_STATE
483     4'b 0001, // index[ 1] KEYMGR_INTR_ENABLE
484     4'b 0001, // index[ 2] KEYMGR_INTR_TEST
485     4'b 0001, // index[ 3] KEYMGR_ALERT_TEST
486     4'b 0001, // index[ 4] KEYMGR_CFG_REGWEN
487     4'b 0001, // index[ 5] KEYMGR_START
488     4'b 0011, // index[ 6] KEYMGR_CONTROL_SHADOWED
489     4'b 0001, // index[ 7] KEYMGR_SIDELOAD_CLEAR
490     4'b 0001, // index[ 8] KEYMGR_RESEED_INTERVAL_REGWEN
491     4'b 0011, // index[ 9] KEYMGR_RESEED_INTERVAL_SHADOWED
492     4'b 0001, // index[10] KEYMGR_SW_BINDING_REGWEN
493     4'b 1111, // index[11] KEYMGR_SEALING_SW_BINDING_0
494     4'b 1111, // index[12] KEYMGR_SEALING_SW_BINDING_1
495     4'b 1111, // index[13] KEYMGR_SEALING_SW_BINDING_2
496     4'b 1111, // index[14] KEYMGR_SEALING_SW_BINDING_3
497     4'b 1111, // index[15] KEYMGR_SEALING_SW_BINDING_4
498     4'b 1111, // index[16] KEYMGR_SEALING_SW_BINDING_5
499     4'b 1111, // index[17] KEYMGR_SEALING_SW_BINDING_6
500     4'b 1111, // index[18] KEYMGR_SEALING_SW_BINDING_7
501     4'b 1111, // index[19] KEYMGR_ATTEST_SW_BINDING_0
502     4'b 1111, // index[20] KEYMGR_ATTEST_SW_BINDING_1
503     4'b 1111, // index[21] KEYMGR_ATTEST_SW_BINDING_2
504     4'b 1111, // index[22] KEYMGR_ATTEST_SW_BINDING_3
505     4'b 1111, // index[23] KEYMGR_ATTEST_SW_BINDING_4
506     4'b 1111, // index[24] KEYMGR_ATTEST_SW_BINDING_5
507     4'b 1111, // index[25] KEYMGR_ATTEST_SW_BINDING_6
508     4'b 1111, // index[26] KEYMGR_ATTEST_SW_BINDING_7
509     4'b 1111, // index[27] KEYMGR_SALT_0
510     4'b 1111, // index[28] KEYMGR_SALT_1
511     4'b 1111, // index[29] KEYMGR_SALT_2
512     4'b 1111, // index[30] KEYMGR_SALT_3
513     4'b 1111, // index[31] KEYMGR_SALT_4
514     4'b 1111, // index[32] KEYMGR_SALT_5
515     4'b 1111, // index[33] KEYMGR_SALT_6
516     4'b 1111, // index[34] KEYMGR_SALT_7
517     4'b 1111, // index[35] KEYMGR_KEY_VERSION
518     4'b 0001, // index[36] KEYMGR_MAX_CREATOR_KEY_VER_REGWEN
519     4'b 1111, // index[37] KEYMGR_MAX_CREATOR_KEY_VER_SHADOWED
520     4'b 0001, // index[38] KEYMGR_MAX_OWNER_INT_KEY_VER_REGWEN

```



```

521      4'b 1111, // index[39] KEYM-
GR_MAX_OWNER_INT_KEY_VER_SHADOWED
522      4'b 0001, // index[40] KEYMGR_MAX_OWNER_KEY_VER_REGWEN
523      4'b 1111, // index[41] KEYMGR_MAX_OWNER_KEY_VER_SHADOWED
524      4'b 1111, // index[42] KEYMGR_SW_SHARE0_OUTPUT_0
525      4'b 1111, // index[43] KEYMGR_SW_SHARE0_OUTPUT_1
526      4'b 1111, // index[44] KEYMGR_SW_SHARE0_OUTPUT_2
527      4'b 1111, // index[45] KEYMGR_SW_SHARE0_OUTPUT_3
528      4'b 1111, // index[46] KEYMGR_SW_SHARE0_OUTPUT_4
529      4'b 1111, // index[47] KEYMGR_SW_SHARE0_OUTPUT_5
530      4'b 1111, // index[48] KEYMGR_SW_SHARE0_OUTPUT_6
531      4'b 1111, // index[49] KEYMGR_SW_SHARE0_OUTPUT_7
532      4'b 1111, // index[50] KEYMGR_SW_SHARE1_OUTPUT_0
533      4'b 1111, // index[51] KEYMGR_SW_SHARE1_OUTPUT_1

534      4'b 1111, // index[52] KEYMGR_SW_SHARE1_OUTPUT_2
535      4'b 1111, // index[53] KEYMGR_SW_SHARE1_OUTPUT_3
536      4'b 1111, // index[54] KEYMGR_SW_SHARE1_OUTPUT_4
537      4'b 1111, // index[55] KEYMGR_SW_SHARE1_OUTPUT_5
538      4'b 1111, // index[56] KEYMGR_SW_SHARE1_OUTPUT_6
539      4'b 1111, // index[57] KEYMGR_SW_SHARE1_OUTPUT_7
540      4'b 0001, // index[58] KEYMGR_WORKING_STATE
541      4'b 0001, // index[59] KEYMGR_OP_STATUS
542      4'b 0001, // index[60] KEYMGR_ERR_CODE
543      4'b 0011, // index[61] KEYMGR_FAULT_STATUS
544      4'b 0001 // index[62] KEYMGR_DEBUG
545  };
546
547 endpackage

```

수명주기 컨트롤러

What this is. Manages life-cycle state (RAW → TEST → DEV → PROD → RMA) and gates features accordingly.

Why it exists. Because some capabilities must exist in the factory and not in the field — test modes, debug, key injection. Transitions are irreversible and authenticated by tokens.

Undergraduate review · EECS concepts. State machines; access control; irreversible transitions; fail-closed defaults.

Graduate · Practice note. This block encodes product strategy in silicon. Which features are locked and when is a business decision, which makes it the block whose requirements change most often.

9 files • 5,016 lines

API — lc_ctrl.sv

Table 1202. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl.sv — module / interface / package (1개)

이름	종류
lc_ctrl	module

Table 1203. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl.sv — parameter (17개)

파라미터	형/폭	기본값
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
SiliconCreatorId	logic [SiliconCreatorIdWidth-1:0]	'0
ProductId	logic [ProductIdWidth-1:0]	'0
RevisionId	logic [RevisionIdWidth-1:0]	'0
IdcodeValue	logic [31:0]	32'h00000001
UseDmiInterface	bit	1'b0
NumRmaAckSigs	int unsigned	2
RndCnstLcKeymgrDivInvalid	lc_keymgr_div_t	LcKeymgrDivWidth'(0)
RndCnstLcKeymgrDivTestUnlocked	lc_keymgr_div_t	LcKeymgrDivWidth'(1)
RndCnstLcKeymgrDivDev	lc_keymgr_div_t	LcKeymgrDivWidth'(2)
RndCnstLcKeymgrDivProduction	lc_keymgr_div_t	LcKeymgrDivWidth'(3)
RndCnstLcKeymgrDivRma	lc_keymgr_div_t	LcKeymgrDivWidth'(4)
RndCnstInvalidTokens	lc_token_mux_t	{TokenMuxBits{1'b1}}
SecVolatileRawUnlockEn	bit	0
EscNumSeverities	int	4
EscPingCountWidth	int	16

Table 1204. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl.sv — localparam (2개)

상수	형/폭	값
HwRevWidth	int	\$bits(lc_hw_rev_t)
HwRev	lc_hw_rev_t	{silicon_creator_id: SiliconCreatorId

Table 1205. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl.sv — 포트 (53개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
clk_kmac_i	input	
rst_kmac_ni	input	
regs_tl_i	input	tlul_pkg::tl_h2d_t
regs_tl_o	output	tlul_pkg::tl_d2h_t
dmi_tl_i	input	tlul_pkg::tl_h2d_t
dmi_tl_o	output	tlul_pkg::tl_d2h_t
jtag_i	input	jtag_pkg::jtag_req_t
jtag_o	output	jtag_pkg::jtag_rsp_t
scan_rst_ni	input	
scanmode_i	input	prim_mubi_pkg::mubi4_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
esc_scrap_state0_tx_i	input	prim_esc_pkg::esc_rx_t
esc_scrap_state0_rx_o	output	prim_esc_pkg::esc_tx_t
esc_scrap_state1_tx_i	input	prim_esc_pkg::esc_rx_t
esc_scrap_state1_rx_o	output	prim_esc_pkg::esc_tx_t
pwr_lc_i	input	pwr_lc_req_t
pwr_lc_o	output	pwr_lc_rsp_t
strap_en_override_o	output	logic
lc_otp_vendor_test_o	output	otp_macro_pkg::otp_test_req_t
lc_otp_vendor_test_i	input	otp_macro_pkg::otp_test_rsp_t
lc_otp_program_o	output	otp_ctrl_pkg::lc_otp_program_req_t
lc_otp_program_i	input	otp_ctrl_pkg::lc_otp_program_rsp_t
kmac_data_i	input	kmac_pkg::app_rsp_t
kmac_data_o	output	kmac_pkg::app_req_t
otp_lc_data_i	input	otp_ctrl_pkg::otp_lc_data_t
lc_init_done_o	output	lc_tx_t

포트	방향	형/폭
lc_dft_en_o	output	lc_tx_t
lc_raw_test_rma_o	output	lc_tx_t
lc_nvm_debug_en_o	output	lc_tx_t
lc_hw_debug_clr_o	output	lc_tx_t
lc_hw_debug_en_o	output	lc_tx_t
lc_cpu_en_o	output	lc_tx_t
lc_creator_seed_sw_rw_en_o	output	lc_tx_t
lc_owner_seed_sw_rw_en_o	output	lc_tx_t
lc_iso_part_sw_rd_en_o	output	lc_tx_t
lc_iso_part_sw_wr_en_o	output	lc_tx_t
lc_seed_hw_rd_en_o	output	lc_tx_t
lc_rma_state_o	output	lc_tx_t
lc_keymgr_en_o	output	lc_tx_t
lc_escalate_en_o	output	lc_tx_t
lc_check_byp_en_o	output	lc_tx_t
lc_clk_byp_req_o	output	lc_tx_t
lc_clk_byp_ack_i	input	lc_tx_t
lc_nvm_rma_seed_o	output	lc_nvm_rma_seed_t
lc_nvm_rma_req_o	output	lc_tx_t
lc_nvm_rma_ack_i	input	lc_tx_t [NumRmaAckSigs-1:0]
lc_keymgr_div_o	output	lc_keymgr_div_t
otp_device_id_i	input	otp_ctrl_pkg::otp_device_id_t
otp_manuf_state_i	input	otp_ctrl_pkg::otp_device_id_t
hw_rev_o	output	lc_hw_rev_t

API — lc_ctrl_dmi_reg_top.sv

Table 1206. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_dmi_reg_top.sv — module / interface / package (1개)

이름	종류
lc_ctrl_dmi_reg_top	module

Table 1207. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_dmi_reg_top.sv — 포트 (7개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t

포트	방향	형/폭
tl_o	output	tlul_pkg::tl_d2h_t
tl_win_o	output	tlul_pkg::tl_h2d_t
tl_win_i	input	tlul_pkg::tl_d2h_t
intg_err_o	output	logic

API — lc_ctrl_fsm.sv

Table 1208. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_fsm.sv — module / interface / package (1개)

이름	종류
lc_ctrl_fsm	module

Table 1209. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_fsm.sv — parameter (8개)

파라미터	형/폭	기본값
NumRmaAckSigs	int unsigned	2
RndCnstLcKeymgrDivInvalid	lc_keymgr_div_t	LcKeymgrDivWidth'(0)
RndCnstLcKeymgrDivTestUnlocked	lc_keymgr_div_t	LcKeymgrDivWidth'(1)
RndCnstLcKeymgrDivDev	lc_keymgr_div_t	LcKeymgrDivWidth'(2)
RndCnstLcKeymgrDivProduction	lc_keymgr_div_t	LcKeymgrDivWidth'(3)
RndCnstLcKeymgrDivRma	lc_keymgr_div_t	LcKeymgrDivWidth'(4)
RndCnstInvalidTokens	lc_token_mux_t	{TokenMuxBits{1'b1}}
SecVolatileRawUnlockEn	bit	0

Table 1210. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_fsm.sv — localparam (1개)

상수	형/폭	값
NumStrapDelayRegs	int	10

Table 1211. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_fsm.sv — 포트 (65개) — 앞 60개만 표시(전체 65개, 본문 소스 참조)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
init_req_i	input	
init_done_o	output	logic
idle_o	output	logic
esc_scrap_state0_i	input	
esc_scrap_state1_i	input	
lc_state_valid_i	input	
lc_state_i	input	lc_state_e

포트	방향	형/폭
lc_cnt_i	input	lc_cnt_e
secrets_valid_i	input	lc_tx_t
use_ext_clock_i	input	
ext_clock_switched_o	output	logic
volatile_raw_unlock_i	input	logic
strap_en_override_o	output	logic
test_unlock_token_i	input	lc_token_t
test_exit_token_i	input	lc_token_t
test_tokens_valid_i	input	lc_tx_t
rma_token_i	input	lc_token_t
rma_token_valid_i	input	lc_tx_t
trans_cmd_i	input	
trans_target_i	input	ext_dec_lc_state_t
dec_lc_state_o	output	ext_dec_lc_state_t
dec_lc_cnt_o	output	dec_lc_cnt_t
dec_lc_id_state_o	output	dec_lc_id_state_e
token_hash_req_o	output	logic
token_hash_req_chk_o	output	logic
token_hash_ack_i	input	
token_hash_err_i	input	
token_if_fsm_err_i	input	
hashed_token_i	input	lc_token_t
unhashed_token_i	input	lc_token_t
otp_prog_req_o	output	logic
otp_prog_lc_state_o	output	lc_state_e
otp_prog_lc_cnt_o	output	lc_cnt_e
otp_prog_ack_i	input	
otp_prog_err_i	input	
trans_success_o	output	logic
trans_cnt_oflw_error_o	output	logic
trans_invalid_error_o	output	logic
token_invalid_error_o	output	logic
nvm_rma_error_o	output	logic
otp_prog_error_o	output	logic
state_invalid_error_o	output	logic

포트	방향	형/폭
lc_raw_test_rma_o	output	lc_tx_t
lc_init_done_o	output	lc_tx_t
lc_dft_en_o	output	lc_tx_t
lc_nvm_debug_en_o	output	lc_tx_t
lc_hw_debug_clr_o	output	lc_tx_t
lc_hw_debug_en_o	output	lc_tx_t
lc_cpu_en_o	output	lc_tx_t
lc_creator_seed_sw_rw_en_o	output	lc_tx_t
lc_owner_seed_sw_rw_en_o	output	lc_tx_t
lc_iso_part_sw_rd_en_o	output	lc_tx_t
lc_iso_part_sw_wr_en_o	output	lc_tx_t
lc_seed_hw_rd_en_o	output	lc_tx_t
lc_rma_state_o	output	lc_tx_t
lc_keymgr_en_o	output	lc_tx_t
lc_escalate_en_o	output	lc_tx_t
lc_check_byp_en_o	output	lc_tx_t

API — lc_ctrl_kmac_if.sv

Table 1212. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_kmac_if.sv — module / interface / package (1개)

이름	종류
lc_ctrl_kmac_if	module

Table 1213. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_kmac_if.sv — localparam (1개)

상수	형/폭	값
StateWidth	int	8

Table 1214. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_kmac_if.sv — 포트 (13개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
clk_kmac_i	input	
rst_kmac_ni	input	
kmac_data_i	input	kmac_pkg::app_rsp_t
kmac_data_o	output	kmac_pkg::app_req_t
transition_token_i	input	lc_token_t
token_hash_req_i	input	

포트	방향	형/폭
token_hash_req_chk_i	input	
token_hash_ack_o	output	logic
token_hash_err_o	output	logic
token_if_fsm_err_o	output	logic
hashed_token_o	output	lc_token_t

API — lc_ctrl_pkg.sv

Table 1215. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_pkg.sv — module / interface / package (1개)

이름	종류
lc_ctrl_pkg	package

Table 1216. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_pkg.sv — parameter (10개)

파라미터	형/폭	기본값
PWR_LC_RSP_DEFAULT	pwr_lc_rsp_t	'{
NumTokens	int	6
TokenIdxWidth	int	vbits(NumTokens)
TokenMuxBits	int	2**TokenIdxWidth*LcTokenWidth
TxWidth	int	4
LC_TX_DEFAULT	lc_tx_t	lc_tx_t'(Off)
RmaSeedWidth	int	64
LC_NVM_RMA_SEED_DEFAULT	lc_nvm_rma_seed_t	'0
LcKeymgrDivWidth	int	128
TransTokenIdxMatrix	token_idx_e [NumLcStates-1:0][NumLcState...	{

Table 1217. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_pkg.sv — localparam (1개)

상수	형/폭	값
FsmStateWidth	int	16

Table 1218. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_pkg.sv — typedef / using (3개)

새 이름	원래 형
lc_token_mux_t	logic [TokenMuxBits-1:0]
lc_nvm_rma_seed_t	logic [RmaSeedWidth-1:0]
lc_keymgr_div_t	logic [LcKeymgrDivWidth-1:0]

Table 1219. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_pkg.sv — #define / `define 매크로 (2개)

매크로	인자	정의
TEST_UNLOCKED	(idx)	\
TEST_LOCKED	(idx)	\

API — lc_ctrl_reg_pkg.sv**Table 1220. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_reg_pkg.sv — module / interface / package (1개)**

이름	종류
lc_ctrl_reg_pkg	package

Table 1221. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_reg_pkg.sv — parameter (94개) — 앞 60개만 표시 (전체 94개, 본문 소스 참조)

파라미터	형/폭	기본값
SiliconCreatorIdWidth	int	16
ProductIdWidth	int	16
RevisionIdWidth	int	8
NumTokenWords	int	4
CsrLcStateWidth	int	30
CsrLcCountWidth	int	5
CsrLcIdStateWidth	int	32
CsrOtpTestCtrlWidth	int	32
CsrOtpTestStatusWidth	int	32
NumDeviceIdWords	int	8
NumManufStateWords	int	8
NumAlerts	int	3
RegsAw	int	8
DmiAw	int	12
NumRegsRegs	int	35
NumRegsDmi	int	0
LC_CTRL_ALERT_TEST_OFFSET	logic [RegsAw-1:0]	8'h 0
LC_CTRL_STATUS_OFFSET	logic [RegsAw-1:0]	8'h 4
LC_CTRL_CLAIM_TRANSITION_IF_REGWEN_OFFSET	logic [RegsAw-1:0]	8'h 8
LC_CTRL_CLAIM_TRANSITION_IF_OFFSET	logic [RegsAw-1:0]	8'h c
LC_CTRL_TRANSITION_REGWEN_OFFSET	logic [RegsAw-1:0]	8'h 10
LC_CTRL_TRANSITION_CMD_OFFSET	logic [RegsAw-1:0]	8'h 14
LC_CTRL_TRANSITION_CTRL_OFFSET	logic [RegsAw-1:0]	8'h 18
LC_CTRL_TRANSITION_TOKEN_0_OFFSET	logic [RegsAw-1:0]	8'h 1c
LC_CTRL_TRANSITION_TOKEN_1_OFFSET	logic [RegsAw-1:0]	8'h 20
LC_CTRL_TRANSITION_TOKEN_2_OFFSET	logic [RegsAw-1:0]	8'h 24
LC_CTRL_TRANSITION_TOKEN_3_OFFSET	logic [RegsAw-1:0]	8'h 28
LC_CTRL_TRANSITION_TARGET_OFFSET	logic [RegsAw-1:0]	8'h 2c
LC_CTRL_OTP_VENDOR_TEST_CTRL_OFFSET	logic [RegsAw-1:0]	8'h 30

파라미터	형/폭	기본값
LC_CTRL_OTP_VENDOR_TEST_STATUS_OFFSET	logic [RegsAw-1:0]	8'h 34
LC_CTRL_LC_STATE_OFFSET	logic [RegsAw-1:0]	8'h 38
LC_CTRL_LC_TRANSITION_CNT_OFFSET	logic [RegsAw-1:0]	8'h 3c
LC_CTRL_LC_ID_STATE_OFFSET	logic [RegsAw-1:0]	8'h 40
LC_CTRL_HW_REVISION0_OFFSET	logic [RegsAw-1:0]	8'h 44
LC_CTRL_HW_REVISION1_OFFSET	logic [RegsAw-1:0]	8'h 48
LC_CTRL_DEVICE_ID_0_OFFSET	logic [RegsAw-1:0]	8'h 4c
LC_CTRL_DEVICE_ID_1_OFFSET	logic [RegsAw-1:0]	8'h 50
LC_CTRL_DEVICE_ID_2_OFFSET	logic [RegsAw-1:0]	8'h 54
LC_CTRL_DEVICE_ID_3_OFFSET	logic [RegsAw-1:0]	8'h 58
LC_CTRL_DEVICE_ID_4_OFFSET	logic [RegsAw-1:0]	8'h 5c
LC_CTRL_DEVICE_ID_5_OFFSET	logic [RegsAw-1:0]	8'h 60
LC_CTRL_DEVICE_ID_6_OFFSET	logic [RegsAw-1:0]	8'h 64
LC_CTRL_DEVICE_ID_7_OFFSET	logic [RegsAw-1:0]	8'h 68
LC_CTRL_MANUF_STATE_0_OFFSET	logic [RegsAw-1:0]	8'h 6c
LC_CTRL_MANUF_STATE_1_OFFSET	logic [RegsAw-1:0]	8'h 70
LC_CTRL_MANUF_STATE_2_OFFSET	logic [RegsAw-1:0]	8'h 74
LC_CTRL_MANUF_STATE_3_OFFSET	logic [RegsAw-1:0]	8'h 78
LC_CTRL_MANUF_STATE_4_OFFSET	logic [RegsAw-1:0]	8'h 7c
LC_CTRL_MANUF_STATE_5_OFFSET	logic [RegsAw-1:0]	8'h 80
LC_CTRL_MANUF_STATE_6_OFFSET	logic [RegsAw-1:0]	8'h 84
LC_CTRL_MANUF_STATE_7_OFFSET	logic [RegsAw-1:0]	8'h 88
LC_CTRL_ALERT_TEST_RESVAL	logic [2:0]	3'h 0
LC_CTRL_ALERT_TEST_FATAL_PROG_ERROR_RESVAL	logic [0:0]	1'h 0
LC_CTRL_ALERT_TEST_FATAL_STATE_ERROR_RESVAL	logic [0:0]	1'h 0
LC_CTRL_ALERT_TEST_FATAL_BUS_INTEG_ERROR_RESVAL	logic [0:0]	1'h 0
LC_CTRL_STATUS_RESVAL	logic [11:0]	12'h 0
LC_CTRL_CLAIM_TRANSITION_IF_RESVAL	logic [7:0]	8'h 69
LC_CTRL_CLAIM_TRANSITION_IF_MUTEX_RESVAL	logic [7:0]	8'h 69
LC_CTRL_TRANSITION_REGWEN_RESVAL	logic [0:0]	1'h 0
LC_CTRL_TRANSITION_REGWEN_TRANSITION_REGWEN_RESVAL	logic [0:0]	1'h 0

API — lc_ctrl_regs_reg_top.sv

Table 1222. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_regs_reg_top.sv — module / interface / package (1개)

이름	종류
lc_ctrl_regs_reg_top	module

Table 1223. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_regs_reg_top.sv — localparam (3개)

상수	형/폭	값
AW	int	8
DW	int	32
DBW	int	DW/8

Table 1224. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_regs_reg_top.sv — 포트 (7개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
reg2hw	output	lc_ctrl_reg_pkg::lc_ctrl_regs_reg2hw_t
hw2reg	input	lc_ctrl_reg_pkg::lc_ctrl_regs_hw2reg_t
intg_err_o	output	logic

API — lc_ctrl_signal_decode.sv

Table 1225. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_signal_decode.sv — module / interface / package (1개)

이름	종류
lc_ctrl_signal_decode	module

Table 1226. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_signal_decode.sv — parameter (5개)

파라미터	형/폭	기본값
RndCnstLcKeymgrDivInvalid	lc_keymgr_div_t	LcKeymgrDivWidth'(0)
RndCnstLcKeymgrDivTestUnlocked	lc_keymgr_div_t	LcKeymgrDivWidth'(1)
RndCnstLcKeymgrDivDev	lc_keymgr_div_t	LcKeymgrDivWidth'(2)
RndCnstLcKeymgrDivProduction	lc_keymgr_div_t	LcKeymgrDivWidth'(3)
RndCnstLcKeymgrDivRma	lc_keymgr_div_t	LcKeymgrDivWidth'(4)

Table 1227. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_signal_decode.sv — 포트 (22개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
lc_state_valid_i	input	logic
lc_state_i	input	lc_state_e

포트	방향	형/폭
fsm_state_i	input	fsm_state_e
secrets_valid_i	input	lc_tx_t
lc_raw_test_rma_o	output	lc_tx_t
lc_init_done_o	output	lc_tx_t
lc_dft_en_o	output	lc_tx_t
lc_nvm_debug_en_o	output	lc_tx_t
lc_hw_debug_clr_o	output	lc_tx_t
lc_hw_debug_en_o	output	lc_tx_t
lc_cpu_en_o	output	lc_tx_t
lc_creator_seed_sw_rw_en_o	output	lc_tx_t
lc_owner_seed_sw_rw_en_o	output	lc_tx_t
lc_iso_part_sw_rd_en_o	output	lc_tx_t
lc_iso_part_sw_wr_en_o	output	lc_tx_t
lc_seed_hw_rd_en_o	output	lc_tx_t
lc_rma_state_o	output	lc_tx_t
lc_keymgr_en_o	output	lc_tx_t
lc_escalate_en_o	output	lc_tx_t
lc_keymgr_div_o	output	lc_keymgr_div_t

API — lc_ctrl_state_decode.sv

Table 1228. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_state_decode.sv — module / interface / package (1개)

이름	종류
lc_ctrl_state_decode	module

Table 1229. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_state_decode.sv — 포트 (9개)

포트	방향	형/폭
lc_state_valid_i	input	logic
lc_state_i	input	lc_state_e
lc_cnt_i	input	lc_cnt_e
secrets_valid_i	input	lc_tx_t
fsm_state_i	input	fsm_state_e
dec_lc_state_o	output	ext_dec_lc_state_t
dec_lc_id_state_o	output	dec_lc_id_state_e
dec_lc_cnt_o	output	dec_lc_cnt_t
state_invalid_error_o	output	logic [5:0]

1. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl.sv (912 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Life cycle controller top.
6 //
7
8 `include "prim_assert.sv"
9
10 module lc_ctrl
11   import lc_ctrl_pkg::*;
12   import lc_ctrl_reg_pkg::*;
13   import lc_ctrl_state_pkg::*;
14   # (
15     // Enable asynchronous transitions on alerts.
16     parameter logic [NumAlerts-1:0] AlertAsyncOn = {Num-
  Alerts[1'b1]},
17     // Number of cycles a differential skew is tolerated on the
  alert and escalation signal
18     parameter int unsigned AlertSkewCycles = 1,
19     // Hardware revision numbers exposed in the CSRs.

20     parameter logic [SiliconCreatorIdWidth-1:0] SiliconCreatorId =
  '0,

21     parameter logic [ProductIdWidth-1:0] ProductId =
  '0,

22     parameter logic [RevisionIdWidth-1:0] RevisionId =
  '0,

23     // Idcode value for the JTAG.
24     parameter logic [31:0] IdcodeValue = 32'h00000001,
25     parameter bit UseDmiInterface = 1'b0,
26     parameter int unsigned NumRmaAckSigs = 2,
27     // Random netlist constants
28     parameter lc_keymgr_div_t RndCnstLcKeymgrDivInvalid =
  LcKeymgrDivWidth'(0),
29     parameter lc_keymgr_div_t RndCnstLcKeymgrDivTestUnlocked =
  LcKeymgrDivWidth'(1),
30     parameter lc_keymgr_div_t RndCnstLcKeymgrDivDev =
  LcKeymgrDivWidth'(2),
31     parameter lc_keymgr_div_t RndCnstLcKeymgrDivProduction =
  LcKeymgrDivWidth'(3),
32     parameter lc_keymgr_div_t RndCnstLcKeymgrDivRma =
  LcKeymgrDivWidth'(4),
33     parameter lc_token_mux_t RndCnstInvalidTokens =
  {TokenMuxBits[1'b1]},
34     parameter bit SecVolatileRawUnlockEn =
  0,
35     parameter int EscNumSeverities =
  4,
36     parameter int EscPingCountWidth =
  16
37   ) (
38     // Life cycle controller clock
39     input clk_i,
40     input rst_ni,
41     // Clock for KMAC interface
42     input clk_k-,
43     input rst_k-,
44     // Bus Interface (device)
45     input tlul_pkg::tl_h2d_t
  mac_i,
46     output tlul_pkg::tl_d2h_t
  mac_ni,
47     // TL-UL-based DMI
48     input tlul_pkg::tl_h2d_t dmi_tl_i,
49     output tlul_pkg::tl_d2h_t dmi_tl_o,
50     // JTAG TAP.
51     input jtag_pkg::jtag_req_t jtag_i,
52     output jtag_pkg::jtag_rsp_t jtag_o,
53     // This bypasses the clock inverter inside the JTAG TAP for
  scanmode.
54     input scan_rst_ni,
55     input prim_mubi_pkg::mubi4_t scan-
  mode_i,
56     // Alert outputs.
57     input prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
  alert_rx_i,
58     output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
  alert_tx_o,
59     // Escalation inputs (severity 1 and 2).
60     // These need not be synchronized since the alert handler is
  in
61     // in the same clock domain as the LC controller.
62     input prim_esc_pkg::esc_rx_t
  esc_scrap_state0_tx_i,
63     output prim_esc_pkg::esc_tx_t
  esc_scrap_state0_rx_o,
64     input prim_esc_pkg::esc_rx_t
  esc_scrap_state1_tx_i,
65     output prim_esc_pkg::esc_tx_t
  esc_scrap_state1_rx_o,

66     // Power manager interface (inputs are synced to lifecycle
  clock domain).
67     input pwr_lc_req_t pwr_lc_i,
68     output pwr_lc_rsp_t pwr_lc_o,
69     // Strap sampling override that is only used when SecVolat-
  ileRawUnlockEn = 1,
70     // Otherwise this output is tied off to 0.
71     output logic
  strap_en_override_o,
72     // Strap override - this is only used when
73     // Macro-specific test registers going to lifecycle TAP/DMI
74     output otp_macro_pkg::otp_test_req_t
  lc_otp_vendor_test_o,
75     input otp_macro_pkg::otp_test_rsp_t
  lc_otp_vendor_test_i,
76     // Life cycle transition command interface.
77     // No sync required since LC and OTP are in the same clock
  domain.
78     output otp_ctrl_pkg::lc_otp_program_req_t
  lc_otp_program_o,
79     input otp_ctrl_pkg::lc_otp_program_rsp_t
  lc_otp_program_i,
80     // Life cycle hashing interface for raw unlock
81     // Synchronized in the life cycle controller.
82     // SEC_CM: TOKEN.DIGEST
83     input kmac_pkg::app_rsp_t
  kmac_data_i,
84     output kmac_pkg::app_req_t
  kmac_data_o,
85     // OTP broadcast outputs
86     // No sync required since LC and OTP are in the same clock
  domain.
87     // SEC_CM: TOKEN_VALID.CTRL.MUBI
88     input otp_ctrl_pkg::otp_lc_data_t
  otp_lc_data_i,
89     // Life cycle broadcast outputs (all of them are re-
  gistered).
90     // SEC_CM: INTERSIG.MUBI
91     output lc_tx_t lc_in-
  it_done_o,
92     output lc_tx_t
  lc_dft_en_o,
93     output lc_tx_t
  lc_raw_test_rma_o,
94     output lc_tx_t
  lc_nvm_debug_en_o,
95     output lc_tx_t lc_hw_de-
  bug_clr_o,
96     output lc_tx_t lc_hw_de-
  bug_en_o,
97     output lc_tx_t
  lc_cpu_en_o,
98     output lc_tx_t lc_creat-
  or_seed_sw_rw_en_o,
99     output lc_tx_t lc_own-
  er_seed_sw_rw_en_o,
100    output lc_tx_t
  lc_iso_part_sw_rd_en_o,
101    output lc_tx_t
  lc_iso_part_sw_wr_en_o,
102    output lc_tx_t
  lc_seed_hw_rd_en_o,
103    output lc_tx_t
  lc_rma_state_o,
104    output lc_tx_t lc_keym-
  gr_en_o,
105    output lc_tx_t lc_escal-
  ate_en_o,
106    output lc_tx_t
  lc_check_byp_en_o,
107    // Request and feedback to/from clock manager and AST.
108    // The ack is synced to the lc clock domain using
  prim_lc_sync.
109    // SEC_CM: INTERSIG.MUBI
110    output lc_tx_t
  lc_clk_byp_req_o,
111    input lc_tx_t
  lc_clk_byp_ack_i,
112    // Request and feedback to/from nvm controller.
113    // The ack is synced to the lc clock domain using
  prim_lc_sync.
114    output lc_nvm_rma_seed_t
  lc_nvm_rma_seed_o,
115    // SEC_CM: INTERSIG.MUBI
116    output lc_tx_t
  lc_nvm_rma_req_o,
117    input lc_tx_t [NumRmaAckSigs-1:0]
  lc_nvm_rma_ack_i,
118    // State group diversification value for keymgr.
119    output lc_keymgr_div_t lc_keym-
  gr_div_o,
120    // Hardware config input, needed for the DEVICE_ID field.
121    input otp_ctrl_pkg::otp_device_id_t
  otp_device_id_i,
122    // Hardware config input, needed for the MANUF_STATE field.
123    input otp_ctrl_pkg::otp_device_id_t
  otp_manuf_state_i,
124    // Hardware revision output (static)
125    output lc_hw_rev_t hw_rev_o
126  );
127

```

```

128 import prim_mubi_pkg::mubi8_t;
129 import prim_mubi_pkg::MuBi8False;
130 import prim_mubi_pkg::mubi8_test_true_strict;
131 import prim_mubi_pkg::mubi8_test_false_loose;
132
133 ///////////////////////////////////////////////////
134 // Integration Checks //
135 ///////////////////////////////////////////////////
136
137 // Check that the CSR parameters correspond with the ones
used in the design.
138 `ASSERT_INIT(DecLcStateWidthCheck_A, CsrLcStateWidth == Ext-
DecLcStateWidth)
139 `ASSERT_INIT(DecLcCountWidthCheck_A, CsrLcCountWidth == De-
cLcCountWidth)
140 `ASSERT_INIT(DecLcIdStateWidthCheck_A, CsrLcIdStateWidth ==
ExtDecLcIdStateWidth)
141 `ASSERT_INIT(NumTokenWordsCheck_A, NumTokenWords == LcToken-
Width/32)
142 `ASSERT_INIT(0tpTestCtrlWidth_A,
otp_macro_pkg::0tpTestCtrlWidth == Csr0tpTestCtrlWidth)
143
144 ///////////////////////////////////////////////////
145 // Regfile //
146 ///////////////////////////////////////////////////
147
148 lc_ctrl_reg_pkg::lc_ctrl_regs_reg2hw_t reg2hw;
149 lc_ctrl_reg_pkg::lc_ctrl_regs_hw2reg_t hw2reg;
150
151 // SEC_CM: TRANSITION.CONFIG.REGWEN, STATE.CONFIG.SPARSE
152 logic fatal_bus_integ_error_q, fatal_bus_integ_error_csr_d,
fatal_bus_integ_error_tap_dmi_d;
153 lc_ctrl_regs_reg_top u_reg_regs (
154   .clk_i,
155   .rst_ni,
156   .tl_i      ( regs_tl_i      ),
157   .tl_o      ( regs_tl_o      ),
158   .reg2hw    ( reg2hw         ),
159   .hw2reg    ( hw2reg         ),
160   // SEC_CM: BUS.INTEGRITY
161   .integ_err_o( fatal_bus_integ_error_csr_d )
162 );
163
164 ///////////////////////////////////////////////////
165 // Life Cycle TAP/DMI Regs //
166 ///////////////////////////////////////////////////
167
168 lc_ctrl_reg_pkg::lc_ctrl_regs_reg2hw_t tap_dmi_reg2hw;
169 lc_ctrl_reg_pkg::lc_ctrl_regs_hw2reg_t tap_dmi_hw2reg;
170
171 tlul_pkg::tl_h2d_t tap_dmi_tl_h2d;
172 tlul_pkg::tl_d2h_t tap_dmi_tl_d2h;
173 tlul_pkg::tl_h2d_t tap_tl_h2d;
174 tlul_pkg::tl_d2h_t tap_tl_d2h;
175
176 // Statically mux DMI TLUL port and the one coming from the
JTAG TAP
177 if (UseDmiInterface) begin : gen_dmi_tlul_ports
178   assign tap_dmi_tl_h2d = dmi_tl_i;
179   assign dmi_tl_o      = tap_dmi_tl_d2h;
180
181   // Tie-off other port
182   assign tap_tl_h2d = '0;
183   assign tap_tl_d2h = '0;
184   assign jtag_o      = '0;
185   logic unused_signal;
186   assign unused_signal = ^{jtag_i, tap_tl_h2d, tap_tl_d2h};
187 end else begin : gen_tap_tlul_ports
188   assign tap_dmi_tl_h2d = tap_tl_h2d;
189   assign tap_tl_d2h     = tap_dmi_tl_d2h;
190   // Tie-off other port
191   assign dmi_tl_o = tlul_pkg::TL_D2H_DEFAULT;
192   logic unused_signal;
193   assign unused_signal = ^{dmi_tl_i};
194 end
195
196 lc_ctrl_regs_reg_top u_reg_tap_dmi (
197   .clk_i,
198   .rst_ni,
199   .tl_i      ( tap_dmi_tl_h2d      ),
200   .tl_o      ( tap_dmi_tl_d2h      ),
201   .reg2hw    ( tap_dmi_reg2hw      ),
202   .hw2reg    ( tap_dmi_hw2reg      ),
203   // SEC_CM: BUS.INTEGRITY
204
205   // While the TAP does not have bus integrity, it does have a
WE checker
206   // that feeds into integ_err_o - hence this is wired up to
the fatal_bus_integ_error.
207   .integ_err_o( fatal_bus_integ_error_tap_dmi_d )
208 );
209
210 if (!UseDmiInterface) begin : gen_tap_tlul
211   // This reuses the JTAG DTM and DMI from the RISC-V ex-
ternal
212
213   // debug v0.13 specification to read and write the lc_ctrl
CSRs:
214   // https://github.com/riscv/riscv-debug-spec/blob/release/
riscv-debug-release.pdf
215   // The register addresses correspond to the byte offsets

```

```

of the lc_ctrl CSRs, divided by 4.
214 // Note that the DMI reset does not affect the LC control-
ler in any way.
215 dm::dmi_req_t dmi_req;
216 logic dmi_req_valid;
217 logic dmi_req_ready;
218 dm::dmi_resp_t dmi_resp;
219 logic dmi_resp_ready;
220 logic dmi_resp_valid;
221
222 logic scanmode;
223 prim_mubi4_dec u_prim_mubi4_dec (
224   .mubi_i(scanmode_i),
225   .mubi_dec_o(scanmode)
226 );
227
228 logic tck_muxed;
229 logic trst_n_muxed;
230 prim_clock_mux2 #(
231   .NoFpgaBuf6(1'b1)
232 ) u_prim_clock_mux2 (
233   .clk0_i(jtag_i.tck),
234   .clk1_i(clk_i),
235   .sel_i (scanmode),
236   .clk_o (tck_muxed)
237 );
238
239 prim_clock_mux2 #(
240   .NoFpgaBuf6(1'b1)
241 ) u_prim_rst_n_mux2 (
242   .clk0_i(jtag_i.trst_n),
243   .clk1_i(scan_rst_ni),
244   .sel_i (scanmode),
245   .clk_o (trst_n_muxed)
246 );
247
248 logic req_ready;
249 assign req_ready = dmi_req_ready & dmi_resp_ready;
250 dmi_jtag #(
251   .IdcodeValue(IdcodeValue),
252   .NumDmiWordAbits(7)
253 ) u_dmi_jtag (
254   .clk_i,
255   .rst_ni,
256   .testmode_i      ( scanmode      ),
257   .test_rst_ni     ( scan_rst_ni    ),
258   .dmi_rst_no      (                ), // unused
259   .dmi_req_o        ( dmi_req       ),
260   .dmi_req_valid_o  ( dmi_req_valid ),
261   // unless there is room for response, stall
262   .dmi_req_ready_i  ( req_ready     ),
263   .dmi_resp_i       ( dmi_resp      ),
264   .dmi_resp_ready_o ( dmi_resp_ready ),
265   .dmi_resp_valid_i ( dmi_resp_valid ),
266   .tck_i            ( tck_muxed     ),
267   .tms_i            ( jtag_i.tms    ),
268   .trst_ni          ( trst_n_muxed  ),
269   .td_i             ( jtag_i.tdi    ),
270   .td_o             ( jtag_o.tdo    ),
271   .tdo_oe_o         ( jtag_o.tdo_oe )
272 );
273
274 // DMI to TL-UL transducing
275 tlul_adapter_host #(
276   .EnableDataIntgGen(1)
277 ) u_tap_tlul_host (
278   .clk_i,
279   .rst_ni,
280   // do not make a request unless there is room for the
response
281   .req_i      ( dmi_req_valid &
dmi_resp_ready ),
282   .gnt_o      (                ),
283   .addr_i     ( top_pkg::TL_AW'{{dmi_req.addr,
2'b00}} ),
284   .we_i       ( dmi_req.op ==
dm::DTM_WRITE ),
285   .wdata_i    (                ),
286   .wdata_intg_i (                ),
287   .be_i       (                ),
288   .user_rsvd_i ( {top_pkg::TL_DBW{1'b1}} ),
289   ('0         ),
290   .instr_type_i ( prim_mubi_pkg::MuBi4False ),
291   .valid_o     ( dmi_resp_valid ),
292   .rdata_o     ( dmi_resp.data ),
293   .rdata_intg_o (                ),
294   .err_o       (                ),
295   .intg_err_o  (                ),
296   .tl_o       ( tap_tl_h2d ),
297   .tl_i       ( tap_tl_d2h )

```

```

297 );
298 // TL-UL to DMI transducing
299 assign dmi_resp.resp = '0; // unused inside dmi_jtag
300
301 // These signals are unused
302 logic unused_tap_tl_d2h;
303 assign unused_tap_tl_d2h = ^{
304   dmi_req.addr[31:30],
305   tap_tl_d2h.d_opcode,
306   tap_tl_d2h.d_param,
307   tap_tl_d2h.d_size,
308   tap_tl_d2h.d_source,
309   tap_tl_d2h.d_sink,
310   tap_tl_d2h.d_user,
311   tap_tl_d2h.d_error
312 };
313
314 end else begin : gen_dmi_tlul
315   // No scan chain needed in the DMI configuration
316   logic unused_signals;
317   assign unused_signals = ^{scan_rst_ni, scanmode_i};
318 end
319
320 //////////////////////////////////////////////////
321 // Transition Interface and HW Mutex //
322 //////////////////////////////////////////////////
323
324 // All registers are Hwext
325 logic trans_success_d, trans_success_q;
326 logic trans_cnt_oflw_error_d,
trans_cnt_oflw_error_q;
327 logic trans_invalid_error_d, trans_invalid_error_q;
328 logic token_invalid_error_d, token_invalid_error_q;
329 logic nvm_rma_error_d, nvm_rma_error_q;
330 logic otp_prog_error_d, fatal_prog_error_q;
331 logic state_invalid_error_d, fatal_state_error_q;
332 logic otp_part_error_q;
333 mubi8_t sw_claim_transition_if_d,
sw_claim_transition_if_q;
334 mubi8_t tap_dmi_claim_transition_if_d,
tap_dmi_claim_transition_if_q;
335 logic transition_cmd;
336 lc_token_t transition_token_d, transition_token_q;
337 ext_dec_lc_state_t transition_target_d, transition_target_q;
338 // No need to register these.
339 ext_dec_lc_state_t dec_lc_state;
340 dec_lc_cnt_t dec_lc_cnt;
341 dec_lc_id_state_e dec_lc_id_state;
342
343 logic lc_idle_d, lc_done_d;
344
345 // Create the hardware revision with the anchor const that
instantiates specific standard cells.
346 // This ensures the individual bits are not combined through
logic optimization and remain visible
347 // in the netlist, thereby enabling metal fixes to be re-
flected in the hardware revision.
348 localparam int HwRevWidth = $bits(lc_hw_rev_t);
349 localparam lc_hw_rev_t HwRev = '{silicon_creator_id: Silic-
onCreatorId,
350                                   product_id: Pro-
ductId,
351                                   revision_id: Revi-
sionId,
352                                   reserved: '0};
353
354 // We need to first cast HwRev to a logic array and then
back to lc_hw_rev_t
355 // This explicit cast is required because some tools misin-
terpret packed struct parameters as
356 // integers, leading to MSB truncation
357 logic [HwRevWidth-1:0] hw_rev_raw;
358 prim_const #(
359   .Width(HwRevWidth),
360   .ConstVal(HwRevWidth'(HwRev))
361 ) u_hw_rev_const (
362   .out_o(hw_rev_raw)
363 );
364 assign hw_rev_o = lc_hw_rev_t'(hw_rev_raw);
365
366 // OTP Vendor control bits
367 logic ext_clock_switched;
368 logic use_ext_clock_d, use_ext_clock_q;
369 logic volatile_raw_unlock_d, volatile_raw_unlock_q;
370 logic [CsrOtpTestCtrlWidth-1:0] otp_vendor_test_ctrl_d,
otp_vendor_test_ctrl_q;
371 logic [CsrOtpTestStatusWidth-1:0] otp_vendor_test_status;
372
373 always_comb begin : p_csr_assign_outputs
374   hw2reg = '0;
375   hw2reg.status.initialized = lc_done_d;
376   hw2reg.status.ready = lc_idle_d;
377   hw2reg.status.ext_clock_switched = ext_clock_switched;
378   hw2reg.status.transition_successful = trans_success_q;
379   hw2reg.status.transition_count_error =
trans_cnt_oflw_error_q;
380   hw2reg.status.transition_error =
trans_invalid_error_q;
381   hw2reg.status.token_error =
token_invalid_error_q;

```

```

382   hw2reg.status.nvm_rma_error = nvm_rma_error_q;
383   hw2reg.status.otp_error = fatal_prog_error_q;
384   hw2reg.status.state_error =
fatal_state_error_q;
385   hw2reg.status.otp_partition_error = otp_part_error_q;
386   hw2reg.status.bus_integ_error =
fatal_bus_integ_error_q;
387   hw2reg.lc_state = dec_lc_state;
388   hw2reg.lc_transition_cnt = dec_lc_cnt;
389   hw2reg.lc_id_state = {DecLcIdStateNum-
Rep{dec_lc_id_state}};
390   hw2reg.device_id = otp_device_id_i;
391   hw2reg.manuf_state = otp_manuf_state_i;
392   hw2reg.hw_revision0.silicon_creator_id = hw_rev_o.silic-
on_creator_id;
393   hw2reg.hw_revision0.product_id =
hw_rev_o.product_id;
394   hw2reg.hw_revision1.revision_id = hw_rev_o.revi-
sion_id;
395   hw2reg.hw_revision1.reserved = '0;
396
397   // The assignments above are identical for the TAP/DMI.
398   tap_dmi_hw2reg = hw2reg;
399
400   // Assignments gated by mutex. Again, the DMI has prior-
ity.
401   tap_dmi_hw2reg.claim_transition_if =
tap_dmi_claim_transition_if_q;
402   hw2reg.claim_transition_if = sw_claim_transition_if_q;
403
404   if (mubi8_test_true_strict(tap_dmi_claim_transition_if_q))
begin
405     tap_dmi_hw2reg.transition_ctrl.ext_clock_en = use_ext_c-
lock_q;
406     tap_dmi_hw2reg.transition_ctrl.volatile_raw_unlock =
volatile_raw_unlock_q;
407     tap_dmi_hw2reg.transition_token = transition_token_q;
408     tap_dmi_hw2reg.transition_target = transition_target_q;
409     // SEC_CM: TRANSITION.CONFIG.REGWEN
410     tap_dmi_hw2reg.transition_regwen = lc_idle_d;
411     tap_dmi_hw2reg.otp_vendor_test_ctrl =
otp_vendor_test_ctrl_q;
412     tap_dmi_hw2reg.otp_vendor_test_status =
otp_vendor_test_status;
413     end else if (mu-
bi8_test_true_strict(sw_claim_transition_if_q)) begin
414       hw2reg.transition_ctrl.ext_clock_en = use_ext_clock_q;
415       hw2reg.transition_ctrl.volatile_raw_unlock = volat-
ile_raw_unlock_q;
416       hw2reg.transition_token = transition_token_q;
417       hw2reg.transition_target = transition_target_q;
418       // SEC_CM: TRANSITION.CONFIG.REGWEN
419       hw2reg.transition_regwen = lc_idle_d;
420       hw2reg.otp_vendor_test_ctrl =
otp_vendor_test_ctrl_q;
421       hw2reg.otp_vendor_test_status =
otp_vendor_test_status;
422     end
423
424     always_comb begin : p_csr_assign_inputs
425       sw_claim_transition_if_d = sw_claim_transition_if_q;
426       tap_dmi_claim_transition_if_d =
tap_dmi_claim_transition_if_q;
427       transition_token_d = transition_token_q;
428       transition_target_d = transition_target_q;
429       transition_cmd = '1'b0;
430       otp_vendor_test_ctrl_d = otp_vendor_test_ctrl_q;
431       use_ext_clock_d = use_ext_clock_q;
432       volatile_raw_unlock_d = volatile_raw_unlock_q;
433
434       // Note that the mutex claims from the TAP/DMI and SW side
could arrive within the same cycle.
435       // In that case we give priority to the TAP/DMI mutex
claim in order to avoid a race condition.
436       // TAP/DMI mutex claim.
437       if (mubi8_test_false_loose(sw_claim_transition_if_q) &&
tap_dmi_reg2hw.claim_transition_if.qe) begin
438         tap_dmi_claim_transition_if_d = mu-
bi8_t'(tap_dmi_reg2hw.claim_transition_if.q);
439         // SW mutex claim.
440         end else if (mu-
bi8_test_false_loose(tap_dmi_claim_transition_if_q) &&
tap_dmi_reg2hw.claim_transition_if.qe) begin
441           sw_claim_transition_if_d = mu-
bi8_t'(reg2hw.claim_transition_if.q);
442         end
443
444         // The idle signal serves as the REGWEN in this case.
445         if (lc_idle_d) begin
446           // The DMI has priority.
447           if (mu-
bi8_test_true_strict(tap_dmi_claim_transition_if_q)) begin
448             transition_cmd = tap_dmi_reg2hw.transition_cmd.q &
tap_dmi_reg2hw.transition_cmd.qe;
449           if (tap_dmi_reg2hw.transition_ctrl.ext_clock_en.qe)
begin
450             use_ext_clock_d |=

```



```

tap_dmi_reg2hw.transition_ctrl.ext_clock_en.q;
456     end
457
458     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION
START -----
459     // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO
MITIGATE
460     // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DIS-
ABLED VIA
461     // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUC-
TION DEVICES.
462     //
-----
463     if
(tap_dmi_reg2hw.transition_ctrl.volatile_raw_unlock.q) begin
464         volatile_raw_unlock_d =
tap_dmi_reg2hw.transition_ctrl.volatile_raw_unlock.q;
465     end

466     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
-----
467
468     for (int k = 0; k < LcTokenWidth/32; k++) begin
469         if (tap_dmi_reg2hw.transition_token[k].qe) begin
470             transition_token_d[k*32+: 32] =
tap_dmi_reg2hw.transition_token[k].q;
471         end
472     end
473
474     if (tap_dmi_reg2hw.transition_target.qe) begin
475         for (int k = 0; k < DeclcStateNumRep; k++) begin
476             transition_target_d[k] = dec_lc_state_e'(
477
tap_dmi_reg2hw.transition_target.q[k*DeclcStateWidth+: Declc-
StateWidth]);
478         end
479     end
480
481     if (tap_dmi_reg2hw.otp_vendor_test_ctrl.qe) begin
482         otp_vendor_test_ctrl_d =
tap_dmi_reg2hw.otp_vendor_test_ctrl.q;
483     end
484     end else if (mu-
bi8_test_true_strict(sw_claim_transition_if_q)) begin
485         transition_cmd = reg2hw.transition_cmd.q &
reg2hw.transition_cmd.qe;
486     end
487
488     if (reg2hw.transition_ctrl.ext_clock_en.qe) begin
489         use_ext_clock_d |=
reg2hw.transition_ctrl.ext_clock_en.q;
490     end
491
492     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION
START -----
493     // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO
MITIGATE
494     // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DIS-
ABLED VIA
495     // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUC-
TION DEVICES.
496     //
-----
497     if (reg2hw.transition_ctrl.volatile_raw_unlock.qe) be-
gin
498         volatile_raw_unlock_d =
reg2hw.transition_ctrl.volatile_raw_unlock.q;
499     end

500     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
-----
501
502     for (int k = 0; k < LcTokenWidth/32; k++) begin
503         if (reg2hw.transition_token[k].qe) begin
504             transition_token_d[k*32+: 32] =
reg2hw.transition_token[k].q;
505         end
506     end
507
508     if (reg2hw.transition_target.qe) begin
509         for (int k = 0; k < DeclcStateNumRep; k++) begin
510             transition_target_d[k] = dec_lc_state_e'(
511                 reg2hw.transition_target.q[k*DeclcStateWidth
+: DeclcStateWidth]);
512         end
513     end
514
515     if (reg2hw.otp_vendor_test_ctrl.qe) begin
516         otp_vendor_test_ctrl_d =
reg2hw.otp_vendor_test_ctrl.q;
517     end
518     end
519     end
520 end
521
522 always_ff @(posedge clk_i or negedge rst_ni) begin : p_csrs
523     if (!rst_ni) begin
524         trans_success_q          <= 1'b0;
525         trans_cnt_oflw_error_q   <= 1'b0;
526         trans_invalid_error_q    <= 1'b0;
527         token_invalid_error_q    <= 1'b0;
528         nvm_rma_error_q          <= 1'b0;

```

```

529         fatal_prog_error_q      <= 1'b0;
530         fatal_state_error_q      <= 1'b0;
531         sw_claim_transition_if_q  <= MuBi8False;
532         tap_dmi_claim_transition_if_q <= MuBi8False;
533         transition_token_q       <= 1'b0;
534         transition_target_q      <= {DeclcStateNum-
Rep{DeclcStRaw}};
535         otp_part_error_q         <= 1'b0;
536         fatal_bus_integ_error_q  <= 1'b0;
537         otp_vendor_test_ctrl_q   <= 1'b0;
538         use_ext_clock_q          <= 1'b0;
539     end else begin
540         // ----- VOLATILE_TEST_UNLOCKED CODE SECTION START
-----
541         // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO
MITIGATE
542         // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DISABLED
VIA
543         // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUCTION
DEVICES.
544         //
-----
545         // In case of a volatile RAW unlock, this bit has to be
cleared when the volatile
546         // unlock is followed by a real transition.
547         // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
-----
548         if (SecVolatileRawUnlockEn && transition_cmd && !volat-
ile_raw_unlock_q) begin
549             trans_success_q <= 1'b0;
550         end else begin
551             trans_success_q <= trans_success_d | trans_success_q;
552         end
553         // All other status and error bits are terminal and re-
quire a reset cycle.
554         trans_cnt_oflw_error_q <= trans_cnt_oflw_error_d |
trans_cnt_oflw_error_q;
555         trans_invalid_error_q <= trans_invalid_error_d |
trans_invalid_error_q;
556         token_invalid_error_q <= token_invalid_error_d |
token_invalid_error_q;
557         nvm_rma_error_q <= nvm_rma_error_d |
nvm_rma_error_q;
558         fatal_prog_error_q <= otp_prog_error_d |
fatal_prog_error_q;
559         fatal_state_error_q <= state_invalid_error_d |
fatal_state_error_q;
560         otp_part_error_q <= otp_lc_data_i.error |
otp_part_error_q;
561         fatal_bus_integ_error_q <= fatal_bus_integ_error_csr_d |
fatal_bus_integ_error_tap_dmi_d |
562         fatal_bus_integ_error_q;
563         // Other regs, gated by mutex further below.
564         sw_claim_transition_if_d <=
565         tap_dmi_claim_transition_if_q <=
566         tap_dmi_claim_transition_if_d;
567         transition_token_q <= transition_token_d;
568         transition_target_q <= transition_target_d;
569         otp_vendor_test_ctrl_q <= otp_vendor_test_ctrl_d;
570         use_ext_clock_q <= use_ext_clock_d;
571     end
572 end
573
574 // ----- VOLATILE_TEST_UNLOCKED CODE SECTION START
-----
575 // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO MITIG-
ATE
576 // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DISABLED VIA
577 // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUCTION
DEVICES.
578 //
-----
579 // If not enabled, this register will become a constant.
580 if (SecVolatileRawUnlockEn) begin : gen_volat-
ile_raw_unlock_reg
581     always_ff @(posedge clk_i or negedge rst_ni) begin :
p_volatile_raw_unlock_reg
582         if (!rst_ni) begin
583             volatile_raw_unlock_q <= 1'b0;
584         end else begin
585             volatile_raw_unlock_q <= volatile_raw_unlock_d;
586         end
587     end
588 end else begin : gen_volatile_raw_unlock_const
589     logic unused_volatile_raw_unlock;
590     assign unused_volatile_raw_unlock = ^volat-
ile_raw_unlock_d;
591     assign volatile_raw_unlock_q = 1'b0;
592 end
593 // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
-----
594
595 assign lc_nvm_rma_seed_o = trans-
ition_token_q[RmaSeedWidth-1:0];
596
597 // Gate the vendor specific test ctrl/status bits to zero in

```



```

production states.
598 // Buffer the enable signal to prevent optimization of the
multibit signal.
599 lc_tx_t lc_raw_test_rma;
600 lc_tx_t [1:0] lc_raw_test_rma_buf;
601 prim_lc_sync #(
602     .NumCopies(2),
603     .AsyncOn(0)
604 ) u_prim_lc_sync (
605     .clk_i,
606     .rst_ni,
607     .lc_en_i(lc_raw_test_rma),
608     .lc_en_o(lc_raw_test_rma_buf)
609 );
610
611 assign lc_raw_test_rma_o = lc_raw_test_rma;
612
613 assign lc_otp_vendor_test_o.ctrl =
(lc_tx_test_true_strict(lc_raw_test_rma_buf[0])) ?
614
615 otp_vendor_test_ctrl_q : '0';
616 assign otp_vendor_test_status =
(lc_tx_test_true_strict(lc_raw_test_rma_buf[1])) ?
617
618 lc_otp_vendor_test_i.status : '0';
619
620 // Alert Sender //
621
622 logic [NumAlerts-1:0] alerts;
623 logic [NumAlerts-1:0] alert_test;
624 logic [NumAlerts-1:0] tap_dmi_alert_test;
625
626 assign alerts = {
627     fatal_bus_integ_error_q,
628     fatal_state_error_q,
629     fatal_prog_error_q
630 };
631
632 assign alert_test = {
633     reg2hw.alert_test.fatal_bus_integ_error.q &
634     reg2hw.alert_test.fatal_bus_integ_error.qe,
635     reg2hw.alert_test.fatal_state_error.q &
636     reg2hw.alert_test.fatal_state_error.qe,
637     reg2hw.alert_test.fatal_prog_error.q &
638     reg2hw.alert_test.fatal_prog_error.qe
639 };
640
641 assign tap_dmi_alert_test = {
642     tap_dmi_reg2hw.alert_test.fatal_bus_integ_error.q &
643     tap_dmi_reg2hw.alert_test.fatal_bus_integ_error.qe,
644     tap_dmi_reg2hw.alert_test.fatal_state_error.q &
645     tap_dmi_reg2hw.alert_test.fatal_state_error.qe,
646     tap_dmi_reg2hw.alert_test.fatal_prog_error.q &
647     tap_dmi_reg2hw.alert_test.fatal_prog_error.qe
648 };
649
650 for (genvar k = 0; k < NumAlerts; k++) begin : gen_alert_tx
651     prim_alert_sender #(
652         .AsyncOn(AlertAsyncOn[k]),
653         .SkewCycles(AlertSkewCycles),
654         .IsFatal(1)
655     ) u_prim_alert_sender (
656         .clk_i,
657         .rst_ni,
658         .alert_test_i ( alert_test[k] |
659             tap_dmi_alert_test[k] ),
660         .alert_req_i ( alerts[k] ),
661         .alert_ack_o ( ),
662         .alert_state_o ( ),
663         .alert_rx_i ( alert_rx_i[k] ),
664         .alert_tx_o ( alert_tx_o[k] )
665     );
666 end
667
668 // Escalation Receivers //
669
670 // SEC_CM: MAIN.FSM.GLOBAL_ESC
671 // We still have two escalation receivers here for historic-
672 al reasons.
673 // The two actions "wipe secrets" and "scrap lifecycle
674 state" have been
675 // combined in order to simplify both DV and the design, as
676 otherwise
677 // this separation of very intertwined actions would have
678 caused too many
679 // unnecessary corner cases. The escalation receivers are
680 now redundant and
681 // trigger both actions at once.
682 // This escalation action moves the life cycle
683 // state into a temporary "SCRAP" state named "ESCALATE",
684 // and asserts the lc_escalate_en life cycle control signal.
685 logic esc_scrap_state0;
686 prim_esc_receiver #(
687     .N_ESC_SEV (EscNumSeverities),
688     .PING_CNT_DW (EscPingCountWidth),
689     .SkewCycles (AlertSkewCycles)
690 ) u_prim_esc_receiver0 (
691     .clk_i,
692     .rst_ni,
693     .esc_req_o (esc_scrap_state0),
694     .esc_rx_o (esc_scrap_state0_rx_o),
695     .esc_tx_i (esc_scrap_state0_tx_i)
696 );
697
698 // This escalation action moves the life cycle
699 // state into a temporary "SCRAP" state named "ESCALATE".
700 logic esc_scrap_state1;
701 prim_esc_receiver #(
702     .N_ESC_SEV (EscNumSeverities),
703     .PING_CNT_DW (EscPingCountWidth),
704     .SkewCycles (AlertSkewCycles)
705 ) u_prim_esc_receiver1 (
706     .clk_i,
707     .rst_ni,
708     .esc_req_o (esc_scrap_state1),
709     .esc_rx_o (esc_scrap_state1_rx_o),
710     .esc_tx_i (esc_scrap_state1_tx_i)
711 );
712
713 // Synchronization of IOs //
714
715 // Signals going to and coming from power manager.
716 logic lc_init;
717 prim_flop_2sync #(
718     .Width(1)
719 ) u_prim_flop_2sync_init (
720     .clk_i,
721     .rst_ni,
722     .d_i(pwr_lc_i.lc_init),
723     .q_o(lc_init)
724 );
725
726 logic lc_done_q;
727 logic lc_idle_q;
728
729 always_ff @(posedge clk_i or negedge rst_ni) begin :
p_sync_regs
730     if (!rst_ni) begin
731         lc_done_q <= 1'b0;
732         lc_idle_q <= 1'b0;
733     end else begin
734         lc_done_q <= lc_done_d;
735         lc_idle_q <= lc_idle_d;
736     end
737 end
738
739 assign pwr_lc_o.lc_done = lc_done_q;
740 assign pwr_lc_o.lc_idle = lc_idle_q;
741
742 // KMAC Interface //
743
744 logic token_hash_req, token_hash_req_chk, token_hash_ack,
token_hash_err, token_if_fsm_err;
745 lc_token_t hashed_token;
746 lc_ctrl_kmac_if u_lc_ctrl_kmac_if (
747     .clk_i,
748     .rst_ni,
749     .clk_kmac_i,
750     .rst_kmac_ni,
751     .kmac_data_i,
752     .kmac_data_o,
753     .transition_token_i ( transition_token_q ),
754     .token_hash_req_i ( token_hash_req ),
755     .token_hash_req_chk_i ( token_hash_req_chk ),
756     .token_hash_ack_o ( token_hash_ack ),
757     .token_hash_err_o ( token_hash_err ),
758     .token_if_fsm_err_o ( token_if_fsm_err ),
759     .hashed_token_o ( hashed_token )
760 );
761
762 // LC FSM //
763
764 // SEC_CM: MANUF.STATE.SPARSE
765 // SEC_CM: TRANSITION.CTR.SPARSE
766 lc_ctrl_fsm #(
767     .NumRmaAckSigs ( NumRmaAck-
768         ),
769     .RndCnstLcKeymgrDivInvalid ( RndCnstLcKeymgrDivInval-
770         id ),
771     .RndCnstLcKeymgrDivTestUn-
772         locked( RndCnstLcKeymgrDivTestUnlocked ),
773     .RndCnstLcKeymgrDivDev ( RndCnstLcKeymgrDi-
774         vDev ),
775     .RndCnstLcKeymgrDivProduction ( RndCnstLcKeymgrDivProduc-
776         tion ),
777     .RndCnstLcKeymgrDivRma ( RndCnstLcKeymgrDi-
778         vRma ),
779     .RndCnstInvalidTokens ( RndCnstInvalid-
780         Tokens ),
781     .SecVolatileRawUnlockEn ( SecVolatileRawUnlock-
782         En )
783 )

```

```

779 ) u_lc_ctrl_fsm (
780   .clk_i,
781   .rst_ni,
782   .init_req_i      ( lc_in-
it
783   .init_done_o
( lc_done_d         ),
784   .idle_o
( lc_idle_d         ),
785   .esc_scrap_state0_i
( esc_scrap_state0  ),
786   .esc_scrap_state1_i
( esc_scrap_state1  ),
787   .lc_state_valid_i
( otp_lc_data_i.valid ),
788   .lc_state_i
( lc_state_e'(otp_lc_data_i.state) ),
789   .secrets_valid_i
( otp_lc_data_i.secrets_valid ),
790   .lc_cnt_i
( lc_cnt_e'(otp_lc_data_i.count) ),
791   .use_ext_clock_i  ( use_ext_c-
lock_q
792   .ext_clock_switched_o ( ext_c-
lock_switched
793   .volatile_raw_unlock_i ( volat-
ile_raw_unlock_q
794   .strap_en_override_o,
795   .test_unlock_token_i
( otp_lc_data_i.test_unlock_token ),
796   .test_exit_token_i
( otp_lc_data_i.test_exit_token ),
797   .test_tokens_valid_i
( otp_lc_data_i.test_tokens_valid ),
798   .rma_token_i
( otp_lc_data_i.rma_token ),
799   .rma_token_valid_i
( otp_lc_data_i.rma_token_valid ),
800   .trans_cmd_i      ( trans-
ition_cmd
801   .trans_target_i   ( trans-
ition_target_q
802   .dec_lc_state_o
( dec_lc_state
803   .dec_lc_cnt_o
( dec_lc_cnt
804   .dec_lc_id_state_o
( dec_lc_id_state
805   .token_hash_req_o
( token_hash_req
806   .token_hash_req_chk_o
( token_hash_req_chk
807   .token_hash_ack_i
( token_hash_ack
808   .token_hash_err_i
( token_hash_err
809   .token_if_fsm_err_i
( token_if_fsm_err
810   .hashed_token_i
( hashed_token
811   .unhashed_token_i ( trans-
ition_token_q
812   .otp_prog_req_o   ( lc_otp_pro-
gram.o.req
813   .otp_prog_lc_state_o ( lc_otp_pro-
gram.o.state
814   .otp_prog_lc_cnt_o ( lc_otp_pro-
gram.o.count
815   .otp_prog_ack_i   ( lc_otp_pro-
gram.i.ack
816   .otp_prog_err_i   ( lc_otp_pro-
gram.i.err
817   .trans_success_o
( trans_success_d
818   .trans_cnt_oflw_error_o
( trans_cnt_oflw_error_d
819   .trans_invalid_error_o
( trans_invalid_error_d
820   .token_invalid_error_o
( token_invalid_error_d
821   .nvm_rma_error_o  ( nvm_rma_er-
ror_d
822   .otp_prog_error_o
( otp_prog_error_d
823   .state_invalid_error_o
( state_invalid_error_d
824   .lc_raw_test_rma_o
( lc_raw_test_rma
825   .lc_init_done_o,
826   .lc_dft_en_o,
827   .lc_nvm_debug_en_o,
828   .lc_hw_debug_clr_o,
829   .lc_hw_debug_en_o,
830   .lc_cpu_en_o,
831   .lc_creator_seed_sw_rw_en_o,
832   .lc_owner_seed_sw_rw_en_o,
833   .lc_iso_part_sw_rd_en_o,
834   .lc_iso_part_sw_wr_en_o,
835   .lc_seed_hw_rd_en_o,
836   .lc_rma_state_o,
837   .lc_keymgr_en_o,

```

```

838   .lc_escalate_en_o,
839   .lc_check_byp_en_o,
840   .lc_clk_byp_req_o,
841   .lc_clk_byp_ack_i,
842   .lc_nvm_rma_req_o,
843   .lc_nvm_rma_ack_i,
844   .lc_keymgr_div_o
845 );
846
847 //////////////////////////////////////////////////////////////////
848 // Assertions //
849 //////////////////////////////////////////////////////////////////
850
851 `ASSERT_KNOWN(RegsTl0Known,
regs_tl_o
852 `ASSERT_KNOWN(DmiTl0Known,
dmi_tl_o
853 `ASSERT_KNOWN(JtagTD0Known_A,
jtag_o
854 `ASSERT_KNOWN(AlertTxKnown_A,
alert_tx_o
855 `ASSERT_KNOWN(EscScrapState0Known_A,
esc_scrap_state0_rx_o
856 `ASSERT_KNOWN(EscScrapState1Known_A,
esc_scrap_state1_rx_o
857 `ASSERT_KNOWN(PwrLcKnown_A,
pwr_lc_o
858 `ASSERT_KNOWN(StrapEnOverrideKnown_A,
strap_en_override_o
859 `ASSERT_KNOWN(VendorTestReqKnown_A,
lc_otp_vendor_test_o
860 `ASSERT_KNOWN(LcOtpProgramKnown_A,   lc_otp_pro-
gram.o
861 `ASSERT_KNOWN(LcOtpTokenKnown_A,
kmac_data_o
862 `ASSERT_KNOWN(LcInitDoneKnown_A,   lc_in-
it_done_o
863 `ASSERT_KNOWN(LcDftEnKnown_A,
lc_dft_en_o
864 `ASSERT_KNOWN(LcRawTestRmaKnown_A,
lc_raw_test_rma_o
865 `ASSERT_KNOWN(LcNvmDebugEnKnown_A,   lc_nvm_de-
bug_en_o
866 `ASSERT_KNOWN(LcHwDebugClrKnown_A,   lc_hw_de-
bug_clr_o
867 `ASSERT_KNOWN(LcHwDebugEnKnown_A,   lc_hw_de-
bug_en_o
868 `ASSERT_KNOWN(LcCpuEnKnown_A,
lc_cpu_en_o
869 `ASSERT_KNOWN(LcCreatorSwRwEn_A,   lc_creat-
or_seed_sw_rw_en_o
870 `ASSERT_KNOWN(LcOwnerSwRwEn_A,   lc_own-
er_seed_sw_rw_en_o
871 `ASSERT_KNOWN(LcIsoSwRwEn_A,
lc_iso_part_sw_rd_en_o
872 `ASSERT_KNOWN(LcIsoSwWrEn_A,
lc_iso_part_sw_wr_en_o
873 `ASSERT_KNOWN(LcSeedHwRdEn_A,
lc_seed_hw_rd_en_o
874 `ASSERT_KNOWN(LcRmaState_A,
lc_rma_state_o
875 `ASSERT_KNOWN(LcKeymgrEnKnown_A,   lc_keym-
gr_en_o
876 `ASSERT_KNOWN(LcEscalateEnKnown_A,   lc_escal-
ate_en_o
877 `ASSERT_KNOWN(LcCheckBypassEnKnown_A,
lc_check_byp_en_o
878 `ASSERT_KNOWN(LcClkBypReqKnown_A,
lc_clk_byp_req_o
879 `ASSERT_KNOWN(LcNvmRmaSeedKnown_A,
lc_nvm_rma_seed_o
880 `ASSERT_KNOWN(LcNvmRmaReqKnown_A,
lc_nvm_rma_req_o
881 `ASSERT_KNOWN(LcKeymgrDiv_A,   lc_keym-
gr_div_o
882 `ASSERT_KNOWN(HwRevKnown_A,
hw_rev_o
883
884 `ASSERT(LcInitDoneSticky_A,
885   ##1 !$fell(lc_tx_test_true_strict(lc_init_done_o)))
886
887 // Alert assertions for sparse FSMs.
888 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlLcFsmCheck_A,
889   u_lc_ctrl_fsm.u_fsm_state_regs, alert_tx_o[1])
890 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlLcStateCheck_A,
891   u_lc_ctrl_fsm.u_state_regs, alert_tx_o[1],
892   !$past(otp_lc_data_i.valid) ||
893   u_lc_ctrl_fsm.fsm_state_q inside {ResetSt, EscalateSt,
PostTransSt, InvalidSt, ScrapSt} ||
894   u_lc_ctrl_fsm.esc_scrap_state0_i ||
895   u_lc_ctrl_fsm.esc_scrap_state1_i)
896 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlLcCntCheck_A,
897   u_lc_ctrl_fsm.u_cnt_regs, alert_tx_o[1],
898   !$past(otp_lc_data_i.valid) ||
899   u_lc_ctrl_fsm.fsm_state_q inside {ResetSt, EscalateSt,
PostTransSt, InvalidSt, ScrapSt} ||
900   u_lc_ctrl_fsm.esc_scrap_state0_i ||
901   u_lc_ctrl_fsm.esc_scrap_state1_i)
902 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlKmacIfFsmCheck_A,
903   u_lc_ctrl_fsm.kmac_if.u_state_regs, alert_tx_o[1],
904   u_lc_ctrl_fsm.fsm_state_q inside {EscalateSt} ||

```

```

905     u_lc_ctrl_fsm.esc_scrap_state0_i ||
906     u_lc_ctrl_fsm.esc_scrap_state1_i)
907
908 // Alert assertions for reg_we onehot check
909 `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegsWeOnehotCheck_A,
u_reg_regs, alert_tx_o[2])
910 `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(TapDmiWeOnehotCheck_A,
911 u_reg_tap_dmi, alert_tx_o[2], 0)
912 endmodule : lc_ctrl

```

2. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_dmi_reg_top.sv (60 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"
8
9 module lc_ctrl_dmi_reg_top (
10     input clk_i,
11     input rst_ni,
12     input tlul_pkg::tl_h2d_t tl_i,
13     output tlul_pkg::tl_d2h_t tl_o,
14
15     // Output port for window
16     output tlul_pkg::tl_h2d_t tl_win_o,
17     input tlul_pkg::tl_d2h_t tl_win_i,
18
19     // To HW
20
21     // Integrity check errors
22     output logic intg_err_o
23 );
24
25 import lc_ctrl_reg_pkg::* ;
26
27
28 // Add an unloaded flop to make use of clock / reset
29 // This is done to specifically address lint complaints of
  unused clocks/resets
30 // Since the flop is unloaded it will be removed during syn-
  thesis
31 logic unused_reg;
32 always_ff @(posedge clk_i or negedge rst_ni) begin
33     if (!rst_ni) begin
34         unused_reg <= '0;
35     end else begin
36         unused_reg <= tl_i.a_valid;
37     end
38 end
39
40
41 // Since there are no registers in this block, commands are
  routed through to windows which
42 // can report their own integrity errors.
43 assign intg_err_o = 1'b0;
44
45 // outgoing integrity generation
46 tlul_pkg::tl_d2h_t tl_o_pre;
47 tlul_rsp_intg_gen #(
48     .EnableRspIntgGen(1),
49     .EnableDataIntgGen(0)
50 ) u_rsp_intg_gen (
51     .tl_i(tl_i),
52     .tl_o(tl_o_pre),
53     .tl_o(tl_o)
54 );
55
56 assign tl_win_o = tl_i;
57 assign tl_o_pre = tl_win_i;
58
59 // Unused signal tieoff
60 endmodule

```

3. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_fsm.sv (893 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Main Life Cycle Controller FSM.
6
7 `include "prim_assert.sv"
8
9 module lc_ctrl_fsm
10     import lc_ctrl_pkg::*;
11     import lc_ctrl_reg_pkg::*;
12     import lc_ctrl_state_pkg::*;
13     #(

```

```

14     parameter int unsigned NumRmaAckSigs = 2,
15     // Random netlist constants
16     parameter lc_keymgr_div_t RndCnstLcKeymgrDivInvalid =
  LcKeymgrDivWidth'(0),
17     parameter lc_keymgr_div_t RndCnstLcKeymgrDivTestUnlocked =
  LcKeymgrDivWidth'(1),
18     parameter lc_keymgr_div_t RndCnstLcKeymgrDivDev =
  LcKeymgrDivWidth'(2),
19     parameter lc_keymgr_div_t RndCnstLcKeymgrDivProduction =
  LcKeymgrDivWidth'(3),
20     parameter lc_keymgr_div_t RndCnstLcKeymgrDivRma =
  LcKeymgrDivWidth'(4),
21     parameter lc_token_mux_t RndCnstInvalidTokens =
  {TokenMuxBits{1'b1}},
22     parameter bit SecVolatileRawUnlockEn = 0
23 ) (
24     // This module is combinational, but we
25     // need the clock and reset for the assertions.
26     input clk_i,
27     input rst_ni,
28     // Initialization request from power manager.
29     input init_req_i,
30     output logic init_done_o,
31     output logic idle_o,
32     // Escalation input
33     input esc_scrap_state0_i,
34     input esc_scrap_state1_i,
35     // Life cycle state vector from OTP.
36     input lc_state_valid_i,
37     input lc_state_e lc_state_i,
38     input lc_cnt_e lc_cnt_i,
39     input lc_tx_t secrets_valid_i,
40     // Defines whether we switch to an external clock when ini-
  tiating a transition.
41     input use_ext_clock_i,
42     output logic ext_clock_switched_o,
43     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION START
44     // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO MITIG-
  ATE
45     // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DISABLED VIA
46     // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUCTION
  DEVICES.
47     //
48     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
49     input logic volatile_raw_unlock_i,
50     output logic strap_en_override_o,
51     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
52     // Token input from OTP (these are all hash post-images).
53     input lc_token_t test_unlock_token_i,
54     input lc_token_t test_exit_token_i,
55     input lc_tx_t test_tokens_valid_i,
56     input lc_token_t rma_token_i,
57     input lc_tx_t rma_token_valid_i,
58     // Transition trigger interface.
59     input trans_cmd_i,
60     input ext_dec_lc_state_t trans_target_i,
61     // Decoded life cycle state for CSRs.
62     output ext_dec_lc_state_t dec_lc_state_o,
63     output dec_lc_cnt_t dec_lc_cnt_o,
64     output dec_lc_id_state_e dec_lc_id_state_o,
65     // Token hashing interface
66     output logic token_hash_req_o,
67     output logic token_hash_req_chk_o,
68     input token_hash_ack_i,
69     input token_hash_err_i,
70     input lc_token_t token_if_fsm_err_i,
71     input lc_token_t hashed_token_i,
72     input lc_token_t unhashed_token_i,
73     // OTP programming interface
74     output logic otp_prog_req_o,
75     output lc_state_e otp_prog_lc_state_o,
76     output lc_cnt_e otp_prog_lc_cnt_o,
77     input otp_prog_ack_i,
78     input otp_prog_err_i,
79     // Error outputs going to CSRs
80     output logic trans_success_o,
81     output logic trans_cnt_oflw_error_o,
82     output logic trans_invalid_error_o,
83     output logic token_invalid_error_o,
84     output logic nvm_rma_error_o,
85     output logic otp_prog_error_o,
86     output logic state_invalid_error_o,
87     // Local life cycle signal
88     output lc_tx_t lc_raw_test_rma_o,
89     // Life cycle broadcast outputs.
90     output lc_tx_t lc_init_done_o,
91     output lc_tx_t lc_dft_en_o,
92     output lc_tx_t lc_nvm_debug_en_o,
93     output lc_tx_t lc_hw_debug_clr_o,
94     output lc_tx_t lc_hw_debug_en_o,
95     output lc_tx_t lc_cpu_en_o,
96     output lc_tx_t lc_creator_seed_sw_rw_en_o,
97     output lc_tx_t lc_owner_seed_sw_rw_en_o,
98     output lc_tx_t lc_iso_part_sw_rd_en_o,
99     output lc_tx_t lc_iso_part_sw_wr_en_o,
100    output lc_tx_t lc_seed_hw_rd_en_o,
101    output lc_tx_t lc_rma_state_o,
102    output lc_tx_t lc_keymgr_en_o,
103    output lc_tx_t lc_escalate_en_o,

```

```

103 output lc_tx_t lc_check_byp_en_o,
104 // Request and feedback to/from clock manager and AST.
105 output lc_tx_t lc_clk_byp_req_o,
106 input lc_tx_t lc_clk_byp_ack_i,
107 // Request and feedback to/from nvmm controller
108 output lc_tx_t lc_nvmm_rma_req_o,
109 input lc_tx_t [NumRmaAckSigs-1:0] lc_nvmm_rma_ack_i,
110 // State group diversification value for keymgr
111 output lc_keymgr_div_t lc_keymgr_div_o
112 );
113 import lc_ctrl_token_pkg::*;
114
115 ///////////////////////////////////////////////////
116 // Synchronizers / Buffers //
117 ///////////////////////////////////////////////////
118
119 // We use multiple copies of these signals in the
120 // FSM checks below.
121 lc_tx_t [3:0] lc_clk_byp_ack;
122 prim_lc_sync #(
123     .NumCopies(4)
124 ) u_prim_lc_sync_clk_byp_ack (
125     .clk_i,
126     .rst_ni,
127     .lc_en_i(lc_clk_byp_ack_i),
128     .lc_en_o(lc_clk_byp_ack)
129 );
130
131 // Indication for CSRs
132 assign ext_clock_switched_o =
lc_tx_test_true_strict(lc_clk_byp_ack[3]);
133
134 // We have multiple response channels for this signal since
the RMA wiping requests can go to
135 // multiple modules that perform wiping in parallel. For security
reasons, this signal is not
136 // daisy-chained - see #19136 for context. Synchronize ACK
signals
separately, combine with
137 // bitwise LC AND function and feed into FSM.
138 lc_tx_t [NumRmaAckSigs-1:0] lc_nvmm_rma_ack;
139 for (genvar k = 0; k < NumRmaAckSigs; k++) begin : gen_syncs
140     prim_lc_sync #(
141         .NumCopies(1)
142     ) u_prim_lc_sync_nvmm_rma_ack(
143         .clk_i,
144         .rst_ni,
145         .lc_en_i(lc_nvmm_rma_ack_i[k]),
146         .lc_en_o({lc_nvmm_rma_ack[k]})
147     );
148 end
149
150 lc_tx_t lc_nvmm_rma_ack_combined;
151 always_comb begin
152     lc_nvmm_rma_ack_combined = 0n;
153     for (int k = 0; k < NumRmaAckSigs; k++) begin
154         lc_nvmm_rma_ack_combined =
lc_tx_and_hi(lc_nvmm_rma_ack_combined, lc_nvmm_rma_ack[k]);
155     end
156 end
157
158 // Make buffered copies for consumption in the FSM below.
159 lc_tx_t [2:0] lc_nvmm_rma_ack_buf;
160 prim_lc_sync #(
161     .NumCopies(3),
162     .AsyncOn(0)
163 ) u_prim_lc_sync_nvmm_rma_ack_buf (
164     .clk_i,
165     .rst_ni,
166     .lc_en_i(lc_nvmm_rma_ack_combined),
167     .lc_en_o(lc_nvmm_rma_ack_buf)
168 );
169
170 ///////////////////////////////////////////////////
171 // FSM Logic //
172 ///////////////////////////////////////////////////
173 fsm_state_e fsm_state_d, fsm_state_q;
174
175 // Continuously feed in valid signal for LC state.
176 logic lc_state_valid_d, lc_state_valid_q;
177 assign lc_state_valid_d = lc_state_valid_i;
178
179 // Encoded state vector.
180 lc_state_e lc_state_d, lc_state_q, next_lc_state;
181 lc_cnt_e lc_cnt_d, lc_cnt_q, next_lc_cnt;
182
183 // Feed the next lc state reg back to the programming inter-
face of OTP.
184 assign otp_prog_lc_state_o = next_lc_state;
185 assign otp_prog_lc_cnt_o = next_lc_cnt;
186
187 // Conditional LC signal outputs
188 lc_tx_t lc_clk_byp_req, lc_nvmm_rma_req, lc_check_byp_en;
189
190 `ASSERT_KNOWN(LcStateKnown_A, lc_state_q )
191 `ASSERT_KNOWN(LcCntKnown_A, lc_cnt_q )
192 `ASSERT_KNOWN(FsmStateKnown_A, fsm_state_q )
193
194 // Hashed token to compare against.
195 logic [1:0] hashed_token_valid_mux;
196 lc_token_t hashed_token_mux;
197
198 // Multibit state error from state decoder
199 logic [5:0] state_invalid_error;
200
201 // Strap sample override signal.
202 logic set_strap_en_override;
203
204 // Registers whether volatile unlock has been successful
205 prim_mubi_pkg::mubi8_t volatile_raw_unlock_success_d, volatile_raw_unlock_success_q;
206
207 // SEC_CM: MAIN.CTRL_FLOW.CONSTISTENCY
208 always_comb begin : p_fsm
209     // FSM default state assignments.
210     fsm_state_d = fsm_state_q;
211     lc_state_d = lc_state_q;
212     lc_cnt_d = lc_cnt_q;
213
214     // Token hashing.
215     token_hash_req_o = 1'b0;
216     token_hash_req_chk_o = 1'b1;
217
218     // OTP Interface
219     otp_prog_req_o = 1'b0;
220
221     // Defaults for status/error signals.
222     token_invalid_error_o = 1'b0;
223     otp_prog_error_o = 1'b0;
224     nvmm_rma_error_o = 1'b0;
225     trans_success_o = 1'b0;
226     state_invalid_error_o = 1'b0;
227
228     // Status indication going to power manager.
229     init_done_o = 1'b1;
230     idle_o = 1'b0;
231
232     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION START
233     // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO MIT-
IGATE
234     // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DISABLED
VIA
235     // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUCTION
DEVICES.
236     //
237     set_strap_en_override = 1'b0;
238     volatile_raw_unlock_success_d = volatile_raw_unlock_success_q;
239     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
240
241     // These signals remain asserted once set to On.
242     // Note that the remaining life cycle signals are decoded
in
243     // the lc_ctrl_signal_decode submodule.
244     lc_clk_byp_req = lc_clk_byp_req_o;
245     lc_nvmm_rma_req = lc_nvmm_rma_req_o;
246     lc_check_byp_en = lc_check_byp_en_o;
247
248     unique case (fsm_state_q)
249     ///////////////////////////////////////////////////
250     // Wait here until OTP has initialized and the
251     // power manager sends an initialization request.
252     ResetSt: begin
253         init_done_o = 1'b0;
254         lc_clk_byp_req = Off;
255         lc_nvmm_rma_req = Off;
256         lc_check_byp_en = Off;
257         if (init_req_i && lc_state_valid_q) begin
258             fsm_state_d = IdleSt;
259             // Fetch LC state vector from OTP.
260             lc_state_d = lc_state_i;
261             lc_cnt_d = lc_cnt_i;
262         end
263     end
264     ///////////////////////////////////////////////////
250     // Idle state where life cycle control signals are
broadcast.
266     // Note that the life cycle signals are decoded and
broadcast
267     // in the lc_ctrl_signal_decode submodule.
268     IdleSt: begin
269         idle_o = 1'b1;
270
271     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION
START -----
272     // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO
MITIGATE
273     // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DIS-
ABLED VIA
274     // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUC-
TION DEVICES.
275     //
276     // Note that if the volatile unlock mechanism is
available,
277     // we have to stop fetching the OTP value after a
volatile unlock has succeeded.

```

```

278      // Otherwise we unconditionally fetch from OTP in this
state.
279      if (!(SecVolatileRawUnlockEn && lc_state_q == LcStTest-
Unlocked0 && lc_cnt_q != LcCnt0) ||
280      prim_mubi_pkg::mubi8_test_false_loose(volatile_raw_unlock_success_q)
) begin
281          // Continuously fetch LC state vector from OTP.
282          // The state is locked in once a transition is star-
ted.
283          lc_state_d = lc_state_i;
284          lc_cnt_d = lc_cnt_i;
285      end

286      // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
-----
287
288      // If the life cycle state is SCRAP, we move the FSM
into a terminal
289      // SCRAP state that does not allow any transitions to
be initiated anymore.
290      if (lc_state_q == LcStScrap) begin
291          fsm_state_d = ScrapSt;
292
293      // ----- VOLATILE_TEST_UNLOCKED CODE SECTION
START -----
294      // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO
MITIGATE
295      // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DIS-
ABLED VIA
296      // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUC-
TION DEVICES.
297      //
-----

298      // Only enter here if volatile RAW unlock is available
and enabled.
299      end else if (SecVolatileRawUnlockEn && volat-
ile_raw_unlock_i && trans_cmd_i) begin
300          // We only allow transitions from RAW -> TEST_UN-
LOCKED0
301          if (lc_state_q == LcStRaw &&
trans_target_i == {DecLcStateNum-
Rep{DecLcStTestUnlocked0}} &&
302              !trans_invalid_error_o) begin
303              // 128bit token check (without passing it through
the KMAC)
304              if (unhashed_token_i == RndCnstRawUnlockToken-
Hashed) begin
305                  // We stay in Idle, but update the life cycle
state register (volatile).
306                  lc_state_d = LcStTestUnlocked0;
307
308                  // If the count is 0, we set it to 1 - otherwise
we just leave it as is so that the
309                  // register value is in sync with what has been
programmed to OTP already (there may
310                  // have been unsuccessful raw unlock attempts be-
fore that already incremented it).
311                  lc_cnt_d = (lc_cnt_q == LcCnt0) ? LcCnt1 :
lc_cnt_q;
312                  // Re-sample the DFT straps in the pinmux.

313      // This signal will be delayed by several cycles
so that the LC_CTRL signals
314      // have time to propagate.
315      set_strap_en_override = 1'b1;
316      // We have to remember that the transition was
successful in order
317      // to correctly
318      // disable the continuous sampling of the life
cycle state vector coming from OTP.
319      volatile_raw_unlock_success_d =
prim_mubi_pkg::MuB18True;
320      // Indicate that the transition was successful.
321      trans_success_o = 1'b1;
322      end else begin
323          token_invalid_error_o = 1'b1;
324          fsm_state_d = PostTransSt;
325      end
326      end else begin
327          // Transition invalid error is set by lc_c-
trl_state_transition module.
328          fsm_state_d = PostTransSt;
329      end

329      // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
-----
330      // Initiate a transition. This will first increment
the
331      // life cycle counter before hashing and checking the
token.
332      end else if (trans_cmd_i) begin
333          fsm_state_d = ClkMuxSt;
334      end

335      // If we are in a non-PROD life cycle state, steer the
clock mux if requested. This
336      // action is available in IdleSt so that the mux can
be steered without having to initiate
337      // a life cycle transition. If a transition is initi-
ated however, the life cycle controller

```

```

338      // will wait for the clock mux acknowledgement in the
ClkMuxSt state before proceeding.
339      if (lc_state_q inside {LcStRaw,
340          LcStTestLocked0,
341          LcStTestLocked1,
342          LcStTestLocked2,
343          LcStTestLocked3,
344          LcStTestLocked4,
345          LcStTestLocked5,
346          LcStTestLocked6,
347          LcStTestUnlocked0,
348          LcStTestUnlocked1,
349          LcStTestUnlocked2,
350          LcStTestUnlocked3,
351          LcStTestUnlocked4,
352          LcStTestUnlocked5,
353          LcStTestUnlocked6,
354          LcStTestUnlocked7,
355          LcStRma}) begin
356          if (use_ext_clock_i) begin
357              lc_clk_byp_req = 0n;
358          end
359      end
360  end
361  //////////////////////////////////////////////////
362  // Clock mux state. If we are in RAW, TEST* or RMA, it
is permissible
363  // to switch to an external clock source. If the bypass
request is
364  // asserted, we have to wait until the clock mux and
clock manager
365  // have switched the mux and the clock divider. Also, we
disable the
366  // life cycle partition checks at this point since we
are going to
367  // alter the contents in the OTP memory array, which
could lead to
368  // spurious escalations.
369  ClkMuxSt: begin
370      lc_check_byp_en = 0n;
371      if (lc_state_q inside {LcStRaw,
372          LcStTestLocked0,
373          LcStTestLocked1,
374          LcStTestLocked2,
375          LcStTestLocked3,
376          LcStTestLocked4,
377          LcStTestLocked5,
378          LcStTestLocked6,
379          LcStTestUnlocked0,
380          LcStTestUnlocked1,
381          LcStTestUnlocked2,
382          LcStTestUnlocked3,
383          LcStTestUnlocked4,
384          LcStTestUnlocked5,
385          LcStTestUnlocked6,
386          LcStTestUnlocked7,
387          LcStRma}) begin
388          if (use_ext_clock_i) begin
389              lc_clk_byp_req = 0n;
390              if (lc_tx_test_true_strict(lc_clk_byp_ack[0])) be-
gin
391                  fsm_state_d = CntIncrSt;
392              end
393          end else begin
394              fsm_state_d = CntIncrSt;
395          end
396          end else begin
397              fsm_state_d = CntIncrSt;
398          end
399      end
400  //////////////////////////////////////////////////
401  // This increments the life cycle counter state.
402  CntIncrSt: begin
403      // If the counter has reached the maximum, bail out.
404      if (trans_cnt_oflw_error_o) begin
405          fsm_state_d = PostTransSt;
406      end else begin
407          fsm_state_d = CntProgSt;
408      end
409  end
410  //////////////////////////////////////////////////
411  // This programs the life cycle counter state.
412  CntProgSt: begin
413      otp_prog_req_o = 1'b1;
414
415      // If the clock mux has been steered, double check
that this is still the case.
416      // Otherwise abort the transition operation.
417      if (lc_clk_byp_req_o != lc_clk_byp_ack[1]) begin
418          fsm_state_d = PostTransSt;
419          otp_prog_error_o = 1'b1;
420      end
421
422      // Check return value and abort if there
423      // was an error.
424      if (otp_prog_ack_i) begin
425          if (otp_prog_err_i) begin

```



```

426         fsm_state_d = PostTransSt;
427         otp_prog_error_o = 1'b1;
428     end else begin
429         fsm_state_d = TransCheckSt;
430     end
431 end
432 end
433 //////////////////////////////////////////////////
434 // First transition valid check. This will be repeated
several
435 // times below.
436 TransCheckSt: begin
437     if (trans_invalid_error_o) begin
438         fsm_state_d = PostTransSt;
439     end else begin
440         fsm_state_d = TokenHashSt;
441     end
442 end
443 //////////////////////////////////////////////////
444 // Hash and compare the token, no matter whether this
transition
445 // is conditional or not. Unconditional transitions just
use a known
446 // all-zero token value. That way, we always compare a
hashed token
447 // and guarantee that no other control flow path exists
that could
448 // bypass the token check.
449 // SEC_CM: TOKEN.DIGEST
450 TokenHashSt: begin
451     token_hash_req_o = 1'b1;
452     if (token_hash_ack_i) begin
453         // This is the first comparison.
454         // The token is compared two more times further be-
low.
455         // Also note that conditional transitions won't be
possible if the
456         // corresponding token is not valid. This only ap-
plies to tokens
457         // stored in
458         // OTP. I.e., these tokens first have to be provi-
sioned, before
459         // they can be used.
460         if (hashed_token_i == hashed_token_mux &&
461             !token_hash_err_i &&
462             &hashed_token_valid_mux) begin
463             fsm_state_d = NvmRmaSt;
464         end else begin
465             fsm_state_d = PostTransSt;
466             token_invalid_error_o = 1'b1;
467         end
468     end
469 //////////////////////////////////////////////////
470 // NVM RMA state. Note that we check the nvm response
again
471 // two times later below.
472 NvmRmaSt: begin
473     if (trans_target_i == {DecLcStateNumRep{DecLcStRma}})
begin
474         lc_nvm_rma_req = 0n;
475         if (lc_tx_test_true_strict(lc_nvm_rma_ack_buf[0]))
begin
476             fsm_state_d = TokenCheck0St;
477         end
478     end else begin
479         fsm_state_d = TokenCheck0St;
480     end
481 //////////////////////////////////////////////////
482 // Check again two times whether this transition and the
hashed
483 // token are valid. Also check again whether the nvm RMA
484 // response is valid.
485 // SEC_CM: TOKEN.DIGEST
486 TokenCheck0St:
487 TokenCheck1St: begin
488     if (trans_invalid_error_o) begin
489         fsm_state_d = PostTransSt;
490     end else begin
491         // If any of these RMA are conditions are true,
492         // all of them must be true at the same time.
493         if ((trans_target_i != {DecLcStateNum-
Rep{DecLcStRma}} &&
494             lc_tx_test_false_strict(lc_nvm_rma_req_o) &&
495             lc_tx_test_false_strict(lc_nvm_rma_ack_buf[1]))
||
496             (trans_target_i == {DecLcStateNum-
Rep{DecLcStRma}} &&
497             lc_tx_test_true_strict(lc_nvm_rma_req_o) &&
498             lc_tx_test_true_strict(lc_nvm_rma_ack_buf[1])))
begin
499             if (hashed_token_i == hashed_token_mux &&
500                 !token_hash_err_i &&
501                 &hashed_token_valid_mux) begin

```

```

502         if (fsm_state_q == TokenCheck1St) begin
503             // This is the only way we can get into the
504             // programming state.
505             fsm_state_d = TransProgSt;
506         end else begin
507             fsm_state_d = TokenCheck1St;
508         end
509     end else begin
510         fsm_state_d = PostTransSt;
511         token_invalid_error_o = 1'b1;
512     end
513 // The NVM RMA process failed.
514 end else begin
515     fsm_state_d = PostTransSt;
516     nvm_rma_error_o = 1'b1;
517 end
518 end
519 end
520 //////////////////////////////////////////////////
521 // Initiate OTP transaction. Note that the concurrent
522 // LC state check is continuously checking whether the
523 // new LC state remains valid. Once the ack returns we
are
524 // done with the transition and can go into the terminal
PostTransSt.
525 TransProgSt: begin
526     otp_prog_req_o = 1'b1;
527
528     // If the clock mux has been steered, double check
that this is
529     // still the case.
530     // Otherwise abort the transition operation.
531     if (lc_clk_byp_req_o != lc_clk_byp_ack[2]) begin
532         fsm_state_d = PostTransSt;
533         otp_prog_error_o = 1'b1;
534     // Also double check that the RMA signals remain
stable.
535     // Otherwise abort the transition operation.
536     end else if ((trans_target_i != {DecLcStateNum-
Rep{DecLcStRma}} &&
537         (lc_nvm_rma_req_o != Off ||
538         lc_nvm_rma_ack_buf[2] != Off)) ||
539         (trans_target_i == {DecLcStateNum-
Rep{DecLcStRma}} &&
540         (lc_nvm_rma_req_o != On ||
541         lc_nvm_rma_ack_buf[2] != On))) begin
542         fsm_state_d = PostTransSt;
543         nvm_rma_error_o = 1'b1;
544     end else if (otp_prog_ack_i) begin
545         fsm_state_d = PostTransSt;
546         otp_prog_error_o = otp_prog_err_i;
547         trans_success_o = ~otp_prog_err_i;
548     end
549 //////////////////////////////////////////////////
550 // Terminal states.
551 ScrapSt,
552 PostTransSt: ;
553
554 EscalateSt: begin
555     // During an escalation it is okay to de-assert
token_hash_req
556     // without receiving ACK.
557     token_hash_req_chk_o = 1'b0;
558 end
559 InvalidSt: begin
560     // During an escalation it is okay to de-assert
token_hash_req
561     // without receiving ACK.
562     token_hash_req_chk_o = 1'b0;
563     state_invalid_error_o = 1'b1;
564 end
565 //////////////////////////////////////////////////
566 // Go to terminal error state if we get here.
567 default: begin
568     fsm_state_d = InvalidSt;
569     state_invalid_error_o = 1'b1;
570 end
571 //////////////////////////////////////////////////
572 endcase
573
574 // SEC_CM: MAIN.FSM.GLOBAL_ESC
575 if (esc_scrap_state0_i || esc_scrap_state1_i) begin
576     fsm_state_d = EscalateSt;
577 // SEC_CM: MAIN.FSM.LOCAL_ESC
578 // If at any time the life cycle state encoding or any
other FSM
579 // state within this module
580 // is not valid, we jump into the terminal error state
right away.
581 // Note that state_invalid_error is a multibit error sig-
nal
582 // with different error sources - need to reduce this to
one bit here.
583 end else if (([state_invalid_error | token_if_fsm_err_i]
&& (fsm_state_q != EscalateSt))) begin
584     fsm_state_d = InvalidSt;
585     state_invalid_error_o = 1'b1;

```

```

583     end
584 end
585
586 ////////////////
587 // State Flops //
588 ////////////////
589
590 `PRIM_FLOP_SPARSE_FSM(u_fsm_state_regs, fsm_state_d,
fsm_state_q, fsm_state_e, ResetSt)
591 `PRIM_FLOP_SPARSE_FSM(u_state_regs, lc_state_d, lc_state_q,
lc_state_e, LcStScrap)
592 `PRIM_FLOP_SPARSE_FSM(u_cnt_regs, lc_cnt_d, lc_cnt_q,
lc_cnt_e, LcCnt24)
593
594 always_ff @(posedge clk_i or negedge rst_ni) begin : p_regs
595   if (!rst_ni) begin
596     lc_state_valid_q <= 1'b0;
597   end else begin
598     lc_state_valid_q <= lc_state_valid_d;
599   end
600 end
601
602 // ----- VOLATILE_TEST_UNLOCKED CODE SECTION START
603
604 // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO MITIG-
ATE
605 // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DISABLED VIA
606 // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUCTION
DEVICES.
607 //
608 if (SecVolatileRawUnlockEn) begin : gen_strap_delay_regs
609   // The delay on the life cycle signals is 1 sender + 2 re-
ceiver domain
610   // cycles. We are delaying this cycle several cycles more
than that so
611   // that the life cycle signals have time to propagate (for
good measure).
612   localparam int NumStrapDelayRegs = 10;
613   logic [NumStrapDelayRegs-1:0] strap_en_override_q;
614   always_ff @(posedge clk_i or negedge rst_ni) begin :
p_volatile_raw_unlock_reg
615     if (!rst_ni) begin
616       strap_en_override_q <= '0;
617       volatile_raw_unlock_success_q <=
prim_mubi_pkg::MuB18False;
618     end else begin
619       strap_en_override_q <=
{strap_en_override_q[NumStrapDelayRegs-2:0],
620        // This is a set-reg that will
stay high until the next reset.
621        set_strap_en_override ||
strap_en_override_q[0]};
622       volatile_raw_unlock_success_q <= volat-
ile_raw_unlock_success_d;
623     end
624   end
625   assign strap_en_override_o =
strap_en_override_q[NumStrapDelayRegs-1];
626   end else begin : gen_no_strap_delay_regs
627     // In this case we tie the strap sampling off.
628     logic unused_sigs;
629     assign unused_sigs = ^{set_strap_en_override,
630        volatile_raw_unlock_success_d};
631     assign strap_en_override_o = 1'b0;
632     assign volatile_raw_unlock_success_q =
prim_mubi_pkg::MuB18False;
633   end
634   // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
635
636 ////////////////
637 // Token mux //
638 ////////////////
639
640 lc_ctrl_pkg::lc_tx_t [3:0] rma_token_valid;
641 prim_lc_sync #(
642   .NumCopies(4),
643   .AsyncOn(0),
644   .ResetValueIsOn(0)
645 ) u_prim_lc_sync_rma_token_valid (
646   .clk_i,
647   .rst_ni,
648   .lc_en_i(rma_token_valid_i),
649   .lc_en_o(rma_token_valid)
650 );
651
652 lc_ctrl_pkg::lc_tx_t [7:0] test_tokens_valid;
653 prim_lc_sync #(
654   .NumCopies(8),
655   .AsyncOn(0),
656   .ResetValueIsOn(0)
657 ) u_prim_lc_sync_test_token_valid (
658   .clk_i,
659   .rst_ni,
660   .lc_en_i(test_tokens_valid_i),
661   .lc_en_o(test_tokens_valid)
662 );

```

```

663
664 // SEC_CM: TOKEN_MUX.CTRL.REDUN
665 // The token mux is split into two halves for which we use
separate mux select signals
666 // that have both been generated from separately buffered
multibit lifecycle signals.
667 logic [2**TokenIdxWidth-1:0][LcTokenWidth/2-1:0]
hashed_tokens_lower, hashed_tokens_upper;
668 // These helper signals are only there to increase readabil-
ity of the mux code below.
669 logic [LcTokenWidth/2-1:0] test_unlock_token_lower, test_un-
lock_token_upper;
670 logic [LcTokenWidth/2-1:0] test_exit_token_lower,
test_exit_token_upper;
671 logic [LcTokenWidth/2-1:0] rma_token_lower, rma_token_upper;
672 assign {test_unlock_token_lower, test_unlock_token_upper} =
test_unlock_token_i;
673 assign {test_exit_token_lower, test_exit_token_upper} =
test_exit_token_i;
674 assign {rma_token_lower, rma_token_upper} =
rma_token_i;
675
676 // SEC_CM: TOKEN.DIGEST
677 // This indexes the correct token, based on the transition
arc.
678 // Note that we always perform a token comparison, even in
case of
679 // unconditional transitions. In the case of unconditional
tokens
680 // we just pass an all-zero constant through the hashing
function.
681 always_comb begin : p_token_assign
682   // Set the invalid token indices to a random netlist con-
stant, rather than all-zero.
683   {hashed_tokens_lower, hashed_tokens_upper} = RndCnstInval-
idTokens;
684   // All-zero token for unconditional transitions.
685   {hashed_tokens_lower[ZeroTokenIdx],
686    hashed_tokens_upper[ZeroTokenIdx]} = AllZeroTokenHashed;
687   {hashed_tokens_lower[RawUnlockTokenIdx],
688    hashed_tokens_upper[RawUnlockTokenIdx]} = RndCnstRawUn-
lockTokenHashed;
689   // This mux has two separate halves, steered with separ-
ately buffered life cycle signals.
690   if (lc_tx_test_true_strict(test_tokens_valid[0])) begin
691     hashed_tokens_lower[TestUnlockTokenIdx] = test_un-
lock_token_lower;
692   end
693   if (lc_tx_test_true_strict(test_tokens_valid[1])) begin
694     hashed_tokens_upper[TestUnlockTokenIdx] = test_un-
lock_token_upper;
695   end
696   // This mux has two separate halves, steered with separ-
ately buffered life cycle signals.
697   if (lc_tx_test_true_strict(test_tokens_valid[2])) begin
698     hashed_tokens_lower[TestExitTokenIdx] =
test_exit_token_lower;
699   end
700   if (lc_tx_test_true_strict(test_tokens_valid[3])) begin
701     hashed_tokens_upper[TestExitTokenIdx] =
test_exit_token_upper;
702   end
703   // This mux has two separate halves, steered with separ-
ately buffered life cycle signals.
704   if (lc_tx_test_true_strict(rma_token_valid[0])) begin
705     hashed_tokens_lower[RmaTokenIdx] = rma_token_lower;
706   end
707   if (lc_tx_test_true_strict(rma_token_valid[1])) begin
708     hashed_tokens_upper[RmaTokenIdx] = rma_token_upper;
709   end
710 end
711
712 // SEC_CM: TOKEN_VALID.MUX.REDUN
713 // The token valid mux is duplicated.
714 logic [TokenIdxWidth-1:0] token_idx0, token_idx1;
715 logic [2**TokenIdxWidth-1:0] hashed_tokens_valid0,
hashed_tokens_valid1;
716 always_comb begin : p_token_valid_assign
717   // First mux
718   hashed_tokens_valid0
719   hashed_tokens_valid0[ZeroTokenIdx] = 1'b1; // always
valid
720   hashed_tokens_valid0[RawUnlockTokenIdx] = 1'b1; // always
valid
721   hashed_tokens_valid0[TestUnlockTokenIdx] =
lc_tx_test_true_strict(test_tokens_valid[4]);
722   hashed_tokens_valid0[TestExitTokenIdx] =
lc_tx_test_true_strict(test_tokens_valid[5]);
723   hashed_tokens_valid0[RmaTokenIdx] =
lc_tx_test_true_strict(rma_token_valid[2]);
724   hashed_tokens_valid0[InvalidTokenIdx] = 1'b0; // always
invalid
725   // Second mux
726   hashed_tokens_valid1
727   hashed_tokens_valid1[ZeroTokenIdx] = 1'b1; // always
valid

```

```

728 hashed_tokens_valid1[RawUnlockTokenIdx] = 1'b1; // always
valid
729 hashed_tokens_valid1[TestUnlockTokenIdx] =
lc_tx_test_true_strict(test_tokens_valid[6]);
730 hashed_tokens_valid1[TestExitTokenIdx] =
lc_tx_test_true_strict(test_tokens_valid[7]);
731 hashed_tokens_valid1[RmaTokenIdx] =
lc_tx_test_true_strict(rma_token_valid[3]);
732 hashed_tokens_valid1[InvalidTokenIdx] = 1'b0; // always
invalid
733 end
734
735 // SEC_CM: STATE.CONFIG.SPARSE
736 // The trans_target_i signal comes from the CSR and uses a
replication encoding,
737 // hence we can use different indices of the array.
738 assign token_idx0 = (int'(dec_lc_state_o[0]) < NumLcStates
&&
739 int'(trans_target_i[0]) < NumLc-
States) ?
740 TransTokenIdxMatrix[dec_lc_state_o[0]]
[trans_target_i[0]] :
741 InvalidTokenIdx;
742 assign token_idx1 = (int'(dec_lc_state_o[1]) < NumLcStates
&&
743 int'(trans_target_i[1]) < NumLc-
States) ?
744 TransTokenIdxMatrix[dec_lc_state_o[1]]
[trans_target_i[1]] :
745 InvalidTokenIdx;
746 assign hashed_token_mux = {hashed_tokens_lower[token_idx0],
747 hashed_tokens_upper[token_idx1]};
748 assign hashed_token_valid_mux =
{hashed_tokens_valid0[token_idx0],
749 hashed_tokens_valid1[token_idx1]};
750
751 // If the indices are inconsistent, we also trigger a trans-
ition error.
752 // We do not trigger an alert right away if this happens,
since it could
753 // be due to an invalid value programmed to the CSRs.
754 logic trans_invalid_error;
755 assign trans_invalid_error_o = trans_invalid_error ||
(token_idx0 != token_idx1);
756
757 ///////////////////////////////////////////////////
758 ///////////////////////////////////////////////////
759 // Decoding and transition logic for redundantly encoded LC
state //
760 ///////////////////////////////////////////////////
761 // This decodes the state into a format that can be exposed
in the CSRs,
762 // and flags any errors in the state encoding. Errors will
move the
763 // main FSM into INVALID right away.
764 lc_ctrl_state_decode u_lc_ctrl_state_decode (
765 .lc_state_valid_i ( lc_state_valid_q ),
766 .lc_state_i ( lc_state_q ),
767 .lc_cnt_i ( lc_cnt_q ),
768 .secrets_valid_i,
769 .fsm_state_i ( fsm_state_q ),
770 .dec_lc_state_o,
771 .dec_lc_id_state_o,
772 .dec_lc_cnt_o,
773 .state_invalid_error_o ( state_invalid_error )
774 );
775
776 // LC transition checker logic and next state generation.
777 lc_ctrl_state_transition #(
778 .SecVolatileRawUnlockEn(SecVolatileRawUnlockEn)
779 ) u_lc_ctrl_state_transition (
780 .lc_state_i ( lc_state_q ),
781 .lc_cnt_i ( lc_cnt_q ),
782 .dec_lc_state_i ( dec_lc_state_o ),
783 .fsm_state_i ( fsm_state_q ),
784 .trans_target_i,
785 .volatile_raw_unlock_i,
786 .trans_cmd_i,
787 .next_lc_state_o ( next_lc_state ),
788 .next_lc_cnt_o ( next_lc_cnt ),
789 .trans_cnt_oflw_error_o,
790 .trans_invalid_error_o ( trans_invalid_error )
791 );
792
793 // LC signal decoder and broadcasting logic.
794 lc_ctrl_signal_decode #(
795 .RndCnstLcKeymgrDivInvalid ( RndCnstLcKeymgrDivInval-
id
796 ),
797 .RndCnstLcKeymgrDivTestUn-
locked( RndCnstLcKeymgrDivTestUnlocked ),
798 .RndCnstLcKeymgrDivDev ( RndCnstLcKeymgrDi-
vDev
799 ),
800 .RndCnstLcKeymgrDivProduction ( RndCnstLcKeymgrDivProduc-
tion
801 ),
802 .RndCnstLcKeymgrDivRma ( RndCnstLcKeymgrDi-
vRma
803 )
804 ) u_lc_ctrl_signal_decode (

```

```

801 .clk_i,
802 .rst_ni,
803 .lc_state_valid_i ( lc_state_valid_q ),
804 .lc_state_i ( lc_state_q ),
805 .secrets_valid_i,
806 .fsm_state_i ( fsm_state_q ),
807 .lc_init_done_o,
808 .lc_raw_test_rma_o,
809 .lc_dft_en_o,
810 .lc_nvm_debug_en_o,
811 .lc_hw_debug_clr_o,
812 .lc_hw_debug_en_o,
813 .lc_cpu_en_o,
814 .lc_creator_seed_sw_rw_en_o,
815 .lc_owner_seed_sw_rw_en_o,
816 .lc_iso_part_sw_rd_en_o,
817 .lc_iso_part_sw_wr_en_o,
818 .lc_seed_hw_rd_en_o,
819 .lc_rma_state_o,
820 .lc_keymgr_en_o,
821 .lc_escalate_en_o,
822 .lc_keymgr_div_o
823 );
824
825 // Conditional signals set by main FSM.
826 prim_lc_sender u_prim_lc_sender_clk_byp_req (
827 .clk_i,
828 .rst_ni,
829 .lc_en_i(lc_clk_byp_req),
830 .lc_en_o(lc_clk_byp_req_o)
831 );
832
833 prim_lc_sender u_prim_lc_sender_nvm_rma_req (
834 .clk_i,
835 .rst_ni,
836 .lc_en_i(lc_nvm_rma_req),
837 .lc_en_o(lc_nvm_rma_req_o)
838 );
839 prim_lc_sender u_prim_lc_sender_check_byp_en (
840 .clk_i,
841 .rst_ni,
842 .lc_en_i(lc_check_byp_en),
843 .lc_en_o(lc_check_byp_en_o)
844 );
845
846 ///////////////////////////////////////////////////
847 // Assertions //
848 ///////////////////////////////////////////////////
849
850 `ASSERT(EscStaysOnOnceAsserted_A,
851 lc_tx_test_true_strict(lc_escalate_en_o)
852 |>
853 lc_tx_test_true_strict(lc_escalate_en_o))
854
855 `ASSERT(ClkBypStaysOnOnceAsserted_A,
856 lc_tx_test_true_strict(lc_clk_byp_req_o)
857 |>
858 lc_tx_test_true_strict(lc_clk_byp_req_o))
859
860 `ASSERT(NvmRmaStaysOnOnceAsserted_A,
861 lc_tx_test_true_strict(lc_nvm_rma_req_o)
862 |>
863 lc_tx_test_true_strict(lc_nvm_rma_req_o))
864
865 `ASSERT(NoClkBypInProdStates_A,
866 lc_state_q inside {LcStProd, LcStProdEnd, LcStDev}
867 |>
868 lc_tx_test_false_strict(lc_clk_byp_req_o))
869
870 `ASSERT(SecCmCFITerminal0_A,
871 fsm_state_q == PostTransSt
872 |>
873 fsm_state_q inside {PostTransSt, InvalidSt, EscalateSt})
874
875 `ASSERT(SecCmCFITerminal1_A,
876 fsm_state_q == ScrapSt
877 |>
878 fsm_state_q inside {ScrapSt, InvalidSt, EscalateSt})
879
880 `ASSERT(SecCmCFITerminal2_A,
881 fsm_state_q == EscalateSt
882 |>
883 fsm_state_q == EscalateSt)
884
885 `ASSERT(SecCmCFITerminal3_A,
886 fsm_state_q == InvalidSt
887 |>
888 fsm_state_q inside {InvalidSt, EscalateSt})
889
890 // Check that the FSM is linear and does not contain any
loops
891 `ASSERT(FPV_LINEAR_FSM(SecCmCFILinear_A, fsm_state_q,
fsm_state_e)
892
893 endmodule : lc_ctrl_fsm

```


4. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_kmac_if.sv (219 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Synchronization interface between LC FSM and KMAC.
6 //
7
8 `include "prim_assert.sv"
9
10 module lc_ctrl_kmac_if
11 import lc_ctrl_pkg::*;
12 import lc_ctrl_state_pkg::*;
13 (
14 // Life cycle controller clock
15 input clk_i,
16 input rst_ni,
17 // Clock for KMAC interface
18 input clk_kmac_i,
19 input rst_kmac_ni,
20 // Life cycle hashing interface for raw unlock
21 // Synchronized in the life cycle controller.
22 input kmac_pkg::app_rsp_t kmac_data_i,
23 output kmac_pkg::app_req_t kmac_data_o,
24 // Token hashing interface to LC FSM
25 input lc_token_t transition_token_i,
26 input token_hash_req_i,
27 // Used for gating assertions inside CDC primitives.
28 input token_hash_req_chk_i,
29 output logic token_hash_ack_o,
30 output logic token_hash_err_o,
31 output logic token_if_fsm_err_o,
32 output lc_token_t hashed_token_o
33 );
34
35 ///////////////////////////////////////////////////
36 // Data and Handshake Synchronizers //
37 ///////////////////////////////////////////////////
38
39 // The transition_token_i register is guaranteed to remain
40 stable once a life cycle
41 // transition has been initiated.
42 // Hence no further synchronization registers are required
43 on the outgoing data.
44 lc_token_t kmac_transition_token;
45 assign kmac_transition_token = transition_token_i;
46
47 // SRC domain cannot change data while waiting for ACK.
48 `ASSERT(DataStable_A, token_hash_req_i && !token_hash_ack_o
49 |> $stable(transition_token_i))
50
51 // Second synchronizer instance for handshake and return
52 data synchronization.
53 logic kmac_req, kmac_ack;
54 logic token_hash_req;
55 logic token_hash_ack_d, token_hash_ack_q;
56 logic token_hash_err_q, token_hash_err_d;
57 lc_token_t hashed_token_q, hashed_token_d;
58 prim_sync_reqack_data #(
59 // Token + Error bit
60 .Width (LcTokenWidth + 1),
61 .DataSrc2Dst(1'b0),
62 // This instantiates a data register
63 // on the destination side.
64 .DataReg (1'b1)
65 ) u_prim_sync_reqack_data_in (
66 .clk_src_i ( clk_i ),
67 .rst_src_ni ( rst_ni ),
68 .clk_dst_i ( clk_kmac_i ),
69 .rst_dst_ni ( rst_kmac_ni ),
70 .req_chk_i ( token_hash_req_chk_i ),
71 .src_req_i ( token_hash_req ),
72 .src_ack_o ( token_hash_ack_d ),
73 .dst_req_o ( kmac_req ),
74 .dst_ack_i ( kmac_ack ),
75 // Truncate hash to 128bit and remove masking (not re-
76 quired here).
77 .data_i ( {kmac_data_i.error,
78 kmac_data_i.digest_s0[LcTokenWidth-1:0] ^
79 kmac_data_i.digest_s1[LcTokenWidth-1:0]} ),
80 .data_o ( {token_hash_err_d,
81 hashed_token_d} )
82 );
83
84 logic unused_sigs;
85 assign unused_sigs = ^{
86 kmac_data_i.digest_s0[LcTokenWidth+: (kmac_pkg::AppDi-
87 gestW - LcTokenWidth)],
88 kmac_data_i.digest_s1[LcTokenWidth+: (kmac_pkg::AppDi-
89 gestW - LcTokenWidth)],
90 // A static interface has no finish response.
91 kmac_data_i.rsp_finish
92 };
93

```

```

88 // Hashed Token Register Running on LC Clock
89 always_ff @(posedge clk_i or negedge rst_ni) begin :
  p_lc_regs
90 if (!rst_ni) begin
91 token_hash_ack_q <= 1'b0;
92 token_hash_err_q <= 1'b0;
93 hashed_token_q <= {LcTokenWidth{1'b1}};
94 end else begin
95 token_hash_ack_q <= token_hash_ack_d;
96 // Latch synchronized token and error bit
97 if (token_hash_req_i && token_hash_ack_d) begin
98 token_hash_err_q <= token_hash_err_d;
99 hashed_token_q <= hashed_token_d;
100 end
101 end
102 end
103
104 assign token_hash_ack_o = token_hash_ack_q;
105 assign token_hash_err_o = token_hash_err_q;
106 assign hashed_token_o = hashed_token_q;
107
108 // Stop requesting tokens upon latching on LC side.
109 assign token_hash_req = token_hash_req_i &
~token_hash_ack_q;
110
111 // Need to synchronize this error signal separately.
112 logic kmac_fsm_err_d, kmac_fsm_err_q;
113 prim_flop_2sync #(
114 .Width(1),
115 .ResetValue(0)
116 ) u_prim_flop_2sync (
117 .clk_i,
118 .rst_ni,
119 .d_i(kmac_fsm_err_q),
120 .q_o(token_if_fsm_err_o)
121 );
122
123 ///////////////////////////////////////////////////
124 // Serialization FSM Running on KMAC Clock //
125 ///////////////////////////////////////////////////
126
127 // SEC_CM: KMAC.FSM.SPARSE
128 // Encoding generated with:
129 // $ ./util/design/sparse-fsm-encode.py -d 5 -m 4 -n 8 \
130 // -s 3343913945 --language=sv
131 //
132 // Hamming distance histogram:
133 //
134 // 0: --
135 // 1: --
136 // 2: --
137 // 3: --
138 // 4: --
139 // 5: ||||| (66.67%)
140 // 6: ||||| (33.33%)
141 // 7: --
142 // 8: --
143 //
144 // Minimum Hamming distance: 5
145 // Maximum Hamming distance: 6
146 // Minimum Hamming weight: 2
147 // Maximum Hamming weight: 6
148 //
149 localparam int StateWidth = 8;
150 typedef enum logic [StateWidth-1:0] {
151 FirstSt = 8'b01011011,
152 SecondSt = 8'b10010100,
153 WaitSt = 8'b11100111,
154 DoneSt = 8'b00101000
155 } state_e;
156
157 state_e state_d, state_q;
158
159 // Serialize the 128bit token into two 64bit beats.
160 always_comb begin : p_kmac
161 state_d = state_q;
162 kmac_ack = 1'b0;
163 kmac_fsm_err_d = 1'b0;
164
165 // LC ctrl operates on unshared data and accepts digest
166 responses immediately.
167 kmac_data_o = '0;
168 kmac_data_o.data_s1 = '0;
169 kmac_data_o.rsp_ready = 1'b1;
170
171 unique case (state_q)
172 // Wait for request and transfer first half of
173 // LC token.
174 FirstSt: begin
175 if (kmac_req) begin
176 kmac_data_o.req_valid = 1'b1;
177 kmac_data_o.strb = 8'hFF;
178 kmac_data_o.data_s0 = kmac_transition_token[0+:
64];
179 if (kmac_data_i.req_ready) begin
180 state_d = SecondSt;
181 end
182 end
183 // Transfer second half of LC token.
184 SecondSt: begin

```

```

185     kmac_data_o.req_valid = 1'b1;
186     kmac_data_o.strb     = 8'hFF;
187     kmac_data_o.req_last = 1'b1;
188     kmac_data_o.data_s0  = kmac_transition_token[64 +
64];
189     if (kmac_data_i.req_ready) begin
190         state_d = WaitSt;
191     end
192 end
193 // Wait for hashed token response and go to terminal
state_e.
194 WaitSt: begin
195     if (kmac_data_i.rsp_valid) begin
196         kmac_ack = 1'b1;
197         state_d = DoneSt;
198     end
199 end
200 // Terminal state (by design we can only perform
201 // one token hashing operation per reset cycle).
202 DoneSt: ;
203 default: begin
204     kmac_fsm_err_d = 1'b1;
205 end
206 endcase // state_q
207 end
208
209 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
state_e, FirstSt, clk_kmac_i, rst_kmac_ni)
210
211 always_ff @(posedge clk_kmac_i or negedge rst_kmac_ni) be-
gin : p_kmac_fsm_err
212     if (!rst_kmac_ni) begin
213         kmac_fsm_err_q <= 1'b0;
214     end else begin
215         kmac_fsm_err_q <= kmac_fsm_err_d;
216     end
217 end
218
219 endmodule : lc_ctrl_kmac_if

```

5. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_pkg.sv (395 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5
6 `include "prim_assert.sv"
7
8 package lc_ctrl_pkg;
9
10 import prim_util_pkg::vbits;
11 import lc_ctrl_state_pkg::*;
12
13 // pwrmgr to lifecycle
14 typedef struct packed {
15     logic lc_init;
16 } pwr_lc_req_t;
17
18 // lifecycle to pwrmgr
19 typedef struct packed {
20     logic lc_done;
21     logic lc_idle;
22 } pwr_lc_rsp_t;
23
24 // default value (for dangling ports)
25 parameter pwr_lc_rsp_t PWR_LC_RSP_DEFAULT = '{
26     lc_done: 1'b1,
27     lc_idle: 1'b1
28 };
29
30 // Netlist Constants (Hashed Tokens) //
31 ///////////////////////////////////////////////////////////////////
32 ///////////////////////////////////////////////////////////////////
33
34 parameter int NumTokens = 6;
35 parameter int TokenIdxWidth = vbits(NumTokens);
36 typedef enum logic [TokenIdxWidth-1:0] {
37     // This is the index for the hashed all-zero constant.
38     // All unconditional transitions use this token.
39     ZeroTokenIdx = 3'h0,
40     RawUnlockTokenIdx = 3'h1,
41     TestUnlockTokenIdx = 3'h2,
42     TestExitTokenIdx = 3'h3,
43     RmaTokenIdx = 3'h4,
44     // This is the index for an all-zero value (i.e., hashed
value = '0).
45     // This is used as an additional blocker for some invalid
state transition edges.
46     InvalidTokenIdx = 3'h5
47 } token_idx_e;
48
49 parameter int TokenMuxBits = 2**TokenIdxWidth*LcTokenWidth;
50 typedef logic [TokenMuxBits-1:0] lc_token_mux_t;
51
52 ///////////////////////////////////////////////////////////////////
53 // Typedefs for LC Interfaces //
54 ///////////////////////////////////////////////////////////////////

```

```

55
56 parameter int TxWidth = 4;
57
58 // Note that changing this encoding has implications on
isolation cell
59 // values in RTL. Do not change this unless absolutely
needed.
60 typedef enum logic [TxWidth-1:0] {
61     On = 4'b0101,
62     Off = 4'b1010
63 } lc_tx_t;
64 parameter lc_tx_t LC_TX_DEFAULT = lc_tx_t'(Off);
65
66 parameter int RmaSeedWidth = 64;
67 typedef logic [RmaSeedWidth-1:0] lc_nvm_rma_seed_t;
68 parameter lc_nvm_rma_seed_t LC_NVM_RMA_SEED_DEFAULT = '0;
69
70 parameter int LcKeymgrDivWidth = 128;
71 typedef logic [LcKeymgrDivWidth-1:0] lc_keymgr_div_t;
72
73 typedef struct packed {
74     logic [lc_ctrl_reg_pkg::SiliconCreatorIdWidth-1:0] silic-
on_creator_id;
75     logic [lc_ctrl_reg_pkg::ProductIdWidth-1:0]
product_id;
76     logic [lc_ctrl_reg_pkg::RevisionIdWidth-1:0]
revision_id;
77     logic [32-lc_ctrl_reg_pkg::RevisionIdWidth-1:0]
re-
served;
78 } lc_hw_rev_t;
79
80 ///////////////////////////////////////////////////////////////////
81 // Helper Functions for Life Cycle Signals //
82 ///////////////////////////////////////////////////////////////////
83
84 // This is a prerequisite for the multibit functions below
to work.
85 `ASSERT_STATIC_IN_PACKAGE(CheckLcTxValsComplementary_A, On
== ~Off)
86 // Check for bit-width matching between lc_tx_t and mubi4_t
87
88 `ASSERT_STATIC_IN_PACKAGE(LcMuBi4WidthCheck_A, $bits(TxWidth) ==
$bits(prim_mubi_pkg::MuBi4Width))
89
90 // Convert a life cycle signal to mubi4
91 // If in the future other versions are desired, this should
really be
92 // moved to prim_mubi_pkg
93 // The On ^ MuBi4True determines the bit differences between
94 // an lc_ctrl_pkg::On and prim_mubi_pkg::MuBi4True.
95 // Once the required inversions are determined, it is then
applied
96 // to the incoming value. If the incoming value is true, it
will
97 // appropriately flip to the correct MuBi4Value.
98 // Since the false value is always complement of the true
value,
99 // this mechanism will also work for the other polarity.
100 function automatic prim_mubi_pkg::mubi4_t lc_to_mu-
bi4(lc_tx_t val);
101     return prim_mubi_pkg::mubi4_t'(val ^ (On ^
prim_mubi_pkg::MuBi4True));
102 endfunction : lc_to_mubi4
103
104 // Same as lc_to_mubi4 but, but for an input that is Off,
return MuBi4True
105 // for an input that is On, return MuBi4False
106 function automatic prim_mubi_pkg::mubi4_t lc_to_mu-
bi4_inv(lc_tx_t val);
107     return prim_mubi_pkg::mubi4_t'(val ^ (Off ^
prim_mubi_pkg::MuBi4True));
108 endfunction : lc_to_mubi4_inv
109
110 // Convert a life cycle signal to mubi8. See also lc_to_mu-
bi4.
111 function automatic prim_mubi_pkg::mubi8_t lc_to_mu-
bi8(lc_tx_t val);
112     return prim_mubi_pkg::mubi8_t'({
113         lc_to_mubi4_inv(val),
114         lc_to_mubi4(val)
115     });
116 endfunction : lc_to_mubi8
117
118 function automatic lc_tx_t mu-
bi4_to_lc(prim_mubi_pkg::mubi4_t val);
119     return lc_tx_t'(val ^ (prim_mubi_pkg::MuBi4True ^ On));
120 endfunction : mubi4_to_lc
121
122 // same function as above, but for an input that is Mu-
Bi4True, return Off
123 // for an input that is MuBi4False, return On
124 function automatic lc_tx_t mu-
bi4_to_lc_inv(prim_mubi_pkg::mubi4_t val);
125     return lc_tx_t'(val ^ (prim_mubi_pkg::MuBi4True ^ Off));
126 endfunction : mubi4_to_lc_inv
127
128 // Test whether the value is supplied is one of the valid
enumerations
129 function automatic logic lc_tx_test_invalid(lc_tx_t val);

```

```

130     return ~(val inside {On, Off});
131 endfunction : lc_tx_test_invalid
132
133 // Convert a 1 input value to a lc_tx output
134 function automatic lc_tx_t lc_tx_bool_to_lc_tx(logic val);
135     return (val ? On : Off);
136 endfunction : lc_tx_bool_to_lc_tx
137
138 // Test whether the multibit value signals an "enabled" con-
dition.
139 // The strict version of this function requires
140 // the multibit value to equal True.
141 function automatic logic lc_tx_test_true_strict(lc_tx_t
val);
142     return On == val;
143 endfunction : lc_tx_test_true_strict
144
145 // Test whether the multibit value signals a "disabled" con-
dition.
146 // The strict version of this function requires
147 // the multibit value to equal False.
148 function automatic logic lc_tx_test_false_strict(lc_tx_t
val);
149     return Off == val;
150 endfunction : lc_tx_test_false_strict
151
152 // Test whether the multibit value signals an "enabled" con-
dition.
153 // The loose version of this function interprets all
154 // values other than False as "enabled".
155 function automatic logic lc_tx_test_true_loose(lc_tx_t val);
156     return Off != val;
157 endfunction : lc_tx_test_true_loose
158
159 // Test whether the multibit value signals a "disabled" con-
dition.
160 // The loose version of this function interprets all
161 // values other than True as "disabled".
162 function automatic logic lc_tx_test_false_loose(lc_tx_t
val);
163     return On != val;
164 endfunction : lc_tx_test_false_loose
165
166 // Performs a logical OR operation between two multibit val-
ues.
167 // This treats "act" as logical 1, and all other values are
168 // treated as 0. Truth table:
169 //
170 // A   | B   | OUT
171 //-----+-----
172 // !act | !act | !act
173 // act  | !act | act
174 // !act | act  | act
175 // act  | act  | act
176 //
177
178 // Note: due to the nature of the lc_tx_or() function, it is
possible that two
179 // non-strictly "act" values may produce a strictly "act"
value. If this is
180 // of concern, e.g. if the output is consumed with a strict
check on "act",
181 // consider using the prim_lc_or_hardenened primitive instead.
182 function automatic lc_tx_t lc_tx_or(lc_tx_t a, lc_tx_t b,
lc_tx_t act);
183     logic [TxWidth-1:0] a_in, b_in, act_in, out;
184     a_in = a;
185     b_in = b;
186     act_in = act;
187     for (int k = 0; k < TxWidth; k++) begin
188         if (act_in[k]) begin
189             out[k] = a_in[k] || b_in[k];
190         end else begin
191             out[k] = a_in[k] && b_in[k];
192         end
193     end
194     return lc_tx_t'(out);
195 endfunction : lc_tx_or
196
197 // Performs a logical AND operation between two multibit
values.
198 // This treats "act" as logical 1, and all other values are
199 // treated as 0. Truth table:
200 //
201 // A   | B   | OUT
202 //-----+-----
203 // !act | !act | !act
204 // act  | !act | !act
205 // !act | act  | !act
206 // act  | act  | act
207 //
208 // Note: The lc_tx_and() function does not suffer from the
strictness problem
209 // that the lc_tx_or function above does, since only one
output value in the
210 // truth table is strictly "act". It can hence be used in
most scenarios without issues.
211 // If however the lc_tx_and() function should be strictly
rectifying (i.e., only
212 // output "act" or ~"act"), the prim_lc_and_hardenened can be

```

```

used.
213 function automatic lc_tx_t lc_tx_and(lc_tx_t a, lc_tx_t b,
lc_tx_t act);
214     logic [TxWidth-1:0] a_in, b_in, act_in, out;
215     a_in = a;
216     b_in = b;
217     act_in = act;
218     for (int k = 0; k < TxWidth; k++) begin
219         if (act_in[k]) begin
220             out[k] = a_in[k] && b_in[k];
221         end else begin
222             out[k] = a_in[k] || b_in[k];
223         end
224     end
225     return lc_tx_t'(out);
226 endfunction : lc_tx_and
227
228 // Performs a logical OR operation between two multibit val-
ues.
229 // This treats "True" as logical 1, and all other values are
230 // treated as 0.
231 function automatic lc_tx_t lc_tx_or_hi(lc_tx_t a, lc_tx_t
b);
232     return lc_tx_or(a, b, On);
233 endfunction : lc_tx_or_hi
234
235 // Performs a logical AND operation between two multibit
values.
236 // This treats "True" as logical 1, and all other values are
237 // treated as 0.
238 function automatic lc_tx_t lc_tx_and_hi(lc_tx_t a, lc_tx_t
b);
239     return lc_tx_and(a, b, On);
240 endfunction : lc_tx_and_hi
241
242 // Performs a logical OR operation between two multibit val-
ues.
243 // This treats "False" as logical 1, and all other values
are
244 // treated as 0.
245 function automatic lc_tx_t lc_tx_or_lo(lc_tx_t a, lc_tx_t
b);
246     return lc_tx_or(a, b, Off);
247 endfunction : lc_tx_or_lo
248
249 // Performs a logical AND operation between two multibit
values.
250 // Tlo treats "False" as logical 1, and all other values
are
251 // treated as 0.
252 function automatic lc_tx_t lc_tx_and_lo(lc_tx_t a, lc_tx_t
b);
253     return lc_tx_and(a, b, Off);
254 endfunction : lc_tx_and_lo
255
256 // Inverts the logical meaning of the multibit value.
257 function automatic lc_tx_t lc_tx_inv(lc_tx_t a);
258     return lc_tx_t'(~TxWidth'a);
259 endfunction : lc_tx_inv
260
261 ///////////////////////////////////////////////////////////////////
262 // Main FSM State //
263 ///////////////////////////////////////////////////////////////////
264
265 // SEC_CM: MAIN.FSM.SPARSE
266 // Encoding generated with:
267 // $ ./util/design/sparse-fsm-encode.py -d 5 -m 15 -n 16 \
268 // -s 2934212379 --language=sv
269 //
270 // Hamming distance histogram:
271 //
272 // 0: --
273 // 1: --
274 // 2: --
275 // 3: --
276 // 4: --
277 // 5: ||||| (7.62%)
278 // 6: ||||| (9.52%)
279 // 7: ||||| (17.14%)
280 // 8: ||||| (20.95%)
281 // 9: ||||| (18.10%)
282 // 10: ||||| (14.29%)
283 // 11: ||| (6.67%)
284 // 12: || (3.81%)
285 // 13: | (1.90%)
286 // 14: --
287 // 15: --
288 // 16: --
289 //
290 // Minimum Hamming distance: 5
291 // Maximum Hamming distance: 13
292 // Minimum Hamming weight: 3
293 // Maximum Hamming weight: 11
294 //
295 localparam int FsmStateWidth = 16;
296 typedef enum logic [FsmStateWidth-1:0] {
297     ResetSt      = 16'b111010101011100,
298     IdleSt       = 16'b0000011101010101,
299     ClkMuxSt     = 16'b1100111011001001,
300     CntIncrSt    = 16'b0011001111000111,
301     CntProgSt    = 16'b0000110001010100,

```

```

302 TransCheckSt = 16'b0110111010110000,
303 TokenHashSt = 16'b1101001000111111,
304 NvmRmaSt = 16'b1110100010001111,
305 TokenCheck0St = 16'b0010000011000000,
306 TokenCheck1St = 16'b1101010101011111,
307 TransProgSt = 16'b1000000110101011,
308 PostTransSt = 16'b0110110100101100,
309 ScrapSt = 16'b1010100001010001,
310 EscalateSt = 16'b1011110110011011,
311 InvalidSt = 16'b0011000101001100
312 } fsm_state_e;
313
314 //////////////////////////////////////////////////
315 // Manufacturing State Transition Matrix //
316 //////////////////////////////////////////////////
317
318 // Helper macro to assemble the token index matrix below.
319 // From TEST_UNLOCKED(N)
320 // -> SCRAP, RMA
321 // -> PROD, PROD_END, DEV
322 // -> TEST_UNLOCKED(N+1)-7
323 // -> TEST_LOCKED(N)-6
324 // -> TEST_UNLOCKED0-(N), RAW
325 `define TEST_UNLOCKED(idx) \
326     {2{ZeroTokenIdx}, \
327     {3{TestExitTokenIdx}, \
328     {(7-idx){InvalidTokenIdx}, \
329     ZeroTokenIdx}, \
330     {(2*idx+2){InvalidTokenIdx}}}
331
332 // Helper macro to assemble the token index matrix below.
333 // From TEST_LOCKED(N)
334 // -> SCRAP
335 // -> RMA
336 // -> PROD, PROD_END, DEV
337 // -> TEST_UNLOCKED(N+1)-7
338 // -> TEST_LOCKED(N)-6
339 // -> TEST_UNLOCKED0-(N), RAW
340 `define TEST_LOCKED(idx) \
341     ZeroTokenIdx, \
342     InvalidTokenIdx, \
343     {3{TestExitTokenIdx}, \
344     {(7-idx){TestUnlockTokenIdx}, \
345     InvalidTokenIdx}, \
346     {(2*idx+2){InvalidTokenIdx}}}
347
348 // The token index matrix below encodes 1) which transition
349 // edges are valid and 2) which token
350 // to use for a given transition edge. Note that uncondi-
351 // tional but otherwise valid transitions
352 // are assigned the ZeroTokenIdx, whereas invalid trans-
353 // itions are assigned an InvalidTokenIdx.
354 parameter token_idx_e [NumLcStates-1:0][NumLcStates-1:0]
355 TransTokenIdxMatrix = {
356     // SCRAP
357     {21{InvalidTokenIdx}}, // -> TEST_LOCKED0-6, TEST_UN-
358     LOCKED0-7, DEV, PROD, PROD_END, RMA, SCRAP
359     // RMA
360     ZeroTokenIdx, // -> SCRAP
361     {20{InvalidTokenIdx}}, // -> TEST_LOCKED0-6, TEST_UN-
362     LOCKED0-7, DEV, PROD, PROD_END, RMA
363     // PROD_END
364     ZeroTokenIdx, // -> SCRAP
365     {20{InvalidTokenIdx}}, // -> TEST_LOCKED0-6, TEST_UN-
366     LOCKED0-7, DEV, PROD, PROD_END, RMA
367     // PROD
368     ZeroTokenIdx, // -> SCRAP
369     RmaTokenIdx, // -> RMA
370     {19{InvalidTokenIdx}}, // -> TEST_LOCKED0-6, TEST_UN-
371     LOCKED0-7, DEV, PROD, PROD_END
372     // DEV
373     ZeroTokenIdx, // -> SCRAP
374     RmaTokenIdx, // -> RMA
375     {19{InvalidTokenIdx}}, // -> TEST_LOCKED0-6, TEST_UN-
376     LOCKED0-7, DEV, PROD, PROD_END
377     // TEST_UNLOCKED0-7, TEST_LOCKED0-6
378     `TEST_UNLOCKED(7),
379     `TEST_LOCKED(6),
380     `TEST_UNLOCKED(6),
381     `TEST_LOCKED(5),
382     `TEST_UNLOCKED(5),
383     `TEST_LOCKED(4),
384     `TEST_UNLOCKED(4),
385     `TEST_LOCKED(3),
386     `TEST_UNLOCKED(3),
387     `TEST_LOCKED(2),
388     `TEST_UNLOCKED(2),
389     `TEST_LOCKED(1),
390     `TEST_UNLOCKED(1),
391     `TEST_LOCKED(0),
392     `TEST_UNLOCKED(0),
393     // RAW
394     ZeroTokenIdx, // -> SCRAP
395     {4{InvalidTokenIdx}}, // -> RMA, PROD, PROD_END, DEV
396     {8{RawUnlockTokenIdx}, // -> TEST_UNLOCKED0-7
397     InvalidTokenIdx} // -> RAW, TEST_LOCKED0-6
398 };
399
400 // These macros are only used locally.
401 `undef TEST_LOCKED
402 `undef TEST_UNLOCKED

```

```

394
395 endpackage : lc_ctrl_pkg

```

6. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_reg_pkg.sv (395 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 // Register Package auto-generated by `reggen` containing data
6 // structure
7 package lc_ctrl_reg_pkg;
8
9 // Param list
10 parameter int SiliconCreatorIdWidth = 16;
11 parameter int ProductIdWidth = 16;
12 parameter int RevisionIdWidth = 8;
13 parameter int NumTokenWords = 4;
14 parameter int CsrLcStateWidth = 30;
15 parameter int CsrLcCountWidth = 5;
16 parameter int CsrLcIdStateWidth = 32;
17 parameter int CsrOtpTestCtrlWidth = 32;
18 parameter int CsrOtpTestStatusWidth = 32;
19 parameter int NumDeviceIdWords = 8;
20 parameter int NumManufStateWords = 8;
21 parameter int NumAlerts = 3;
22
23 // Address widths within the block
24 parameter int RegsAw = 8;
25 parameter int DmiAw = 12;
26
27 // Number of registers for every interface
28 parameter int NumRegsRegs = 35;
29 parameter int NumRegsDmi = 0;
30
31 // Alert indices
32 typedef enum int {
33     AlertFatalProgErrorIdx = 0,
34     AlertFatalStateErrorIdx = 1,
35     AlertFatalBusIntegErrorIdx = 2
36 } lc_ctrl_alert_idx_t;
37
38 //////////////////////////////////////////////////
39 // Typedefs for registers for regs interface //
40 //////////////////////////////////////////////////
41
42 typedef struct packed {
43     struct packed {
44         logic q;
45         logic qe;
46     } fatal_bus_integ_error;
47     struct packed {
48         logic q;
49         logic qe;
50     } fatal_state_error;
51     struct packed {
52         logic q;
53         logic qe;
54     } fatal_prog_error;
55 } lc_ctrl_reg2hw_alert_test_reg_t;
56
57 typedef struct packed {
58     logic [7:0] q;
59     logic qe;
60 } lc_ctrl_reg2hw_claim_transition_if_reg_t;
61
62 typedef struct packed {
63     logic q;
64     logic qe;
65 } lc_ctrl_reg2hw_transition_cmd_reg_t;
66
67 typedef struct packed {
68     struct packed {
69         logic q;
70         logic qe;
71     } volatile_raw_unlock;
72     struct packed {
73         logic q;
74         logic qe;
75     } ext_clock_en;
76 } lc_ctrl_reg2hw_transition_ctrl_reg_t;
77
78 typedef struct packed {
79     logic [31:0] q;
80     logic qe;
81 } lc_ctrl_reg2hw_transition_token_mreg_t;
82
83 typedef struct packed {
84     logic [29:0] q;
85     logic qe;
86 } lc_ctrl_reg2hw_transition_target_reg_t;
87
88 typedef struct packed {
89     logic [31:0] q;
90     logic qe;

```

```

91 } lc_ctrl_reg2hw_otp_vendor_test_ctrl_reg_t;
92
93 typedef struct packed {
94     struct packed {
95         logic d;
96     } otp_partition_error;
97     struct packed {
98         logic d;
99     } bus_integ_error;
100     struct packed {
101         logic d;
102     } state_error;
103     struct packed {
104         logic d;
105     } otp_error;
106     struct packed {
107         logic d;
108     } nvm_rma_error;
109     struct packed {
110         logic d;
111     } token_error;
112     struct packed {
113         logic d;
114     } transition_error;
115     struct packed {
116         logic d;
117     } transition_count_error;
118     struct packed {
119         logic d;
120     } transition_successful;
121     struct packed {
122         logic d;
123     } ext_clock_switched;
124     struct packed {
125         logic d;
126     } ready;
127     struct packed {
128         logic d;
129     } initialized;
130 } lc_ctrl_hw2reg_status_reg_t;
131
132 typedef struct packed {
133     logic [7:0] d;
134 } lc_ctrl_hw2reg_claim_transition_if_reg_t;
135
136 typedef struct packed {
137     logic d;
138 } lc_ctrl_hw2reg_transition_regwen_reg_t;
139
140 typedef struct packed {
141     struct packed {
142         logic d;
143     } volatile_raw_unlock;
144     struct packed {
145         logic d;
146     } ext_clock_en;
147 } lc_ctrl_hw2reg_transition_ctrl_reg_t;
148
149 typedef struct packed {
150     logic [31:0] d;
151 } lc_ctrl_hw2reg_transition_token_mreg_t;
152
153 typedef struct packed {
154     logic [29:0] d;
155 } lc_ctrl_hw2reg_transition_target_reg_t;
156
157 typedef struct packed {
158     logic [31:0] d;
159 } lc_ctrl_hw2reg_otp_vendor_test_ctrl_reg_t;
160
161 typedef struct packed {
162     logic [31:0] d;
163 } lc_ctrl_hw2reg_otp_vendor_test_status_reg_t;
164
165 typedef struct packed {
166     logic [29:0] d;
167 } lc_ctrl_hw2reg_lc_state_reg_t;
168
169 typedef struct packed {
170     logic [4:0] d;
171 } lc_ctrl_hw2reg_lc_transition_cnt_reg_t;
172
173 typedef struct packed {
174     logic [31:0] d;
175 } lc_ctrl_hw2reg_lc_id_state_reg_t;
176
177 typedef struct packed {
178     struct packed {
179         logic [15:0] d;
180     } silicon_creator_id;
181     struct packed {
182         logic [15:0] d;
183     } product_id;
184 } lc_ctrl_hw2reg_hw_revision0_reg_t;
185
186 typedef struct packed {
187     struct packed {
188         logic [23:0] d;
189     } reserved;
190     struct packed {
191         logic [7:0] d;
192     } revision_id;
193 } lc_ctrl_hw2reg_hw_revision1_reg_t;
194
195 typedef struct packed {
196     logic [31:0] d;
197 } lc_ctrl_hw2reg_device_id_mreg_t;
198
199 typedef struct packed {
200     logic [31:0] d;
201 } lc_ctrl_hw2reg_manuf_state_mreg_t;
202
203 // Register -> HW type for regs interface
204 typedef struct packed {
205     lc_ctrl_reg2hw_alert_test_reg_t alert_test; // [216:211]
206     lc_ctrl_reg2hw_claim_transition_if_reg_t
207 claim_transition_if; // [210:202]
208     lc_ctrl_reg2hw_transition_cmd_reg_t transition_cmd; //
209 [201:200]
210     lc_ctrl_reg2hw_transition_ctrl_reg_t transition_ctrl; //
211 [199:196]
212     lc_ctrl_reg2hw_transition_token_mreg_t [3:0] trans-
213 ition_token; // [195:64]
214     lc_ctrl_reg2hw_transition_target_reg_t trans-
215 ition_target; // [63:33]
216     lc_ctrl_reg2hw_otp_vendor_test_ctrl_reg_t
217 otp_vendor_test_ctrl; // [32:0]
218 } lc_ctrl_regs_reg2hw_t;
219
220 // HW -> register type for regs interface
221 typedef struct packed {
222     lc_ctrl_hw2reg_status_reg_t status; // [887:876]
223     lc_ctrl_hw2reg_claim_transition_if_reg_t
224 claim_transition_if; // [875:868]
225     lc_ctrl_hw2reg_transition_regwen_reg_t trans-
226 ition_regwen; // [867:867]
227     lc_ctrl_hw2reg_transition_ctrl_reg_t transition_ctrl; //
228 [866:865]
229     lc_ctrl_hw2reg_transition_token_mreg_t [3:0] trans-
230 ition_token; // [864:737]
231     lc_ctrl_hw2reg_transition_target_reg_t trans-
232 ition_target; // [736:707]
233     lc_ctrl_hw2reg_otp_vendor_test_ctrl_reg_t
234 otp_vendor_test_ctrl; // [706:675]
235     lc_ctrl_hw2reg_otp_vendor_test_status_reg_t
236 otp_vendor_test_status; // [674:643]
237     lc_ctrl_hw2reg_lc_state_reg_t lc_state; // [642:613]
238     lc_ctrl_hw2reg_lc_transition_cnt_reg_t lc.trans-
239 ition_cnt; // [612:608]
240     lc_ctrl_hw2reg_lc_id_state_reg_t lc_id_state; // [607:576]
241     lc_ctrl_hw2reg_hw_revision0_reg_t hw_revision0; //
242 [575:544]
243     lc_ctrl_hw2reg_hw_revision1_reg_t hw_revision1; //
244 [543:512]
245     lc_ctrl_hw2reg_device_id_mreg_t [7:0] device_id; //
246 [511:256]
247     lc_ctrl_hw2reg_manuf_state_mreg_t [7:0] manuf_state; //
248 [255:0]
249 } lc_ctrl_regs_hw2reg_t;
250
251 // Register offsets for regs interface
252
253 parameter logic [RegsAw-1:0] LC_CTRL_ALERT_TEST_OFFSET = 8'h
0;
254
255 parameter logic [RegsAw-1:0] LC_CTRL_STATUS_OFFSET = 8'h 4;
256
257 parameter logic [RegsAw-1:0] LC_C-
258 TRL_CLAIM_TRANSITION_IF_REGWEN_OFFSET = 8'h 8;
259
260 parameter logic [RegsAw-1:0] LC_C-
261 TRL_CLAIM_TRANSITION_IF_OFFSET = 8'h c;
262
263 parameter logic [RegsAw-1:0] LC_C-
264 TRL_TRANSITION_REGWEN_OFFSET = 8'h 10;
265
266 parameter logic [RegsAw-1:0] LC_CTRL_TRANSITION_CMD_OFFSET =
267 8'h 14;
268
269 parameter logic [RegsAw-1:0] LC_CTRL_TRANSITION_CTRL_OFFSET
270 = 8'h 18;
271
272 parameter logic [RegsAw-1:0] LC_C-
273 TRL_TRANSITION_TOKEN_0_OFFSET = 8'h 1c;
274
275 parameter logic [RegsAw-1:0] LC_C-
276 TRL_TRANSITION_TOKEN_1_OFFSET = 8'h 20;
277
278 parameter logic [RegsAw-1:0] LC_C-
279 TRL_TRANSITION_TOKEN_2_OFFSET = 8'h 24;
280
281 parameter logic [RegsAw-1:0] LC_C-
282 TRL_TRANSITION_TOKEN_3_OFFSET = 8'h 28;
283
284 parameter logic [RegsAw-1:0] LC_C-
285 TRL_TRANSITION_TARGET_OFFSET = 8'h 2c;
286
287 parameter logic [RegsAw-1:0] LC_C-
288 TRL_OTP_VENDOR_TEST_CTRL_OFFSET = 8'h 30;
289
290 parameter logic [RegsAw-1:0] LC_C-
291 TRL_OTP_VENDOR_TEST_STATUS_OFFSET = 8'h 34;
292
293 parameter logic [RegsAw-1:0] LC_CTRL_LC_STATE_OFFSET = 8'h
38;
294
295 parameter logic [RegsAw-1:0] LC_C-
296 TRL_LC_TRANSITION_CNT_OFFSET = 8'h 3c;
297
298 parameter logic [RegsAw-1:0] LC_CTRL_LC_ID_STATE_OFFSET =
299 8'h 40;
300
301 parameter logic [RegsAw-1:0] LC_CTRL_HW_REVISION0_OFFSET =
302 8'h 44;
303
304 parameter logic [RegsAw-1:0] LC_CTRL_HW_REVISION1_OFFSET =
305 8'h 48;
306
307 parameter logic [RegsAw-1:0] LC_CTRL_DEVICE_ID_0_OFFSET =
308 8'h 4c;

```



```

254 parameter logic [RegsAw-1:0] LC_CTRL_DEVICE_ID_1_OFFSET =
8'h 50;
255 parameter logic [RegsAw-1:0] LC_CTRL_DEVICE_ID_2_OFFSET =
8'h 54;
256 parameter logic [RegsAw-1:0] LC_CTRL_DEVICE_ID_3_OFFSET =
8'h 58;
257 parameter logic [RegsAw-1:0] LC_CTRL_DEVICE_ID_4_OFFSET =
8'h 5c;
258 parameter logic [RegsAw-1:0] LC_CTRL_DEVICE_ID_5_OFFSET =
8'h 60;
259 parameter logic [RegsAw-1:0] LC_CTRL_DEVICE_ID_6_OFFSET =
8'h 64;
260 parameter logic [RegsAw-1:0] LC_CTRL_DEVICE_ID_7_OFFSET =
8'h 68;
261 parameter logic [RegsAw-1:0] LC_CTRL_MANUF_STATE_0_OFFSET =
8'h 6c;
262 parameter logic [RegsAw-1:0] LC_CTRL_MANUF_STATE_1_OFFSET =
8'h 70;
263 parameter logic [RegsAw-1:0] LC_CTRL_MANUF_STATE_2_OFFSET =
8'h 74;
264 parameter logic [RegsAw-1:0] LC_CTRL_MANUF_STATE_3_OFFSET =
8'h 78;
265 parameter logic [RegsAw-1:0] LC_CTRL_MANUF_STATE_4_OFFSET =
8'h 7c;
266 parameter logic [RegsAw-1:0] LC_CTRL_MANUF_STATE_5_OFFSET =
8'h 80;
267 parameter logic [RegsAw-1:0] LC_CTRL_MANUF_STATE_6_OFFSET =
8'h 84;
268 parameter logic [RegsAw-1:0] LC_CTRL_MANUF_STATE_7_OFFSET =
8'h 88;
269
270 // Reset values for hwext registers and their fields for
regs interface
271 parameter logic [2:0] LC_CTRL_ALERT_TEST_RESVAL = 3'h 0;
272 parameter logic [0:0] LC_C-
TRL_ALERT_TEST_FATAL_PROG_ERROR_RESVAL = 1'h 0;
273 parameter logic [0:0] LC_C-
TRL_ALERT_TEST_FATAL_STATE_ERROR_RESVAL = 1'h 0;
274 parameter logic [0:0] LC_C-
TRL_ALERT_TEST_FATAL_BUS_INTEG_ERROR_RESVAL = 1'h 0;
275 parameter logic [11:0] LC_CTRL_STATUS_RESVAL = 12'h 0;
276 parameter logic [7:0] LC_CTRL_CLAIM_TRANSITION_IF_RESVAL =
8'h 69;
277 parameter logic [7:0] LC_C-
TRL_CLAIM_TRANSITION_IF_MUTEX_RESVAL = 8'h 69;

278 parameter logic [0:0] LC_CTRL_TRANSITION_REGWEN_RESVAL = 1'h
0;
279 parameter logic [0:0] LC_C-
TRL_TRANSITION_REGWEN_TRANSITION_REGWEN_RESVAL = 1'h 0;
280 parameter logic [0:0] LC_CTRL_TRANSITION_CMD_RESVAL = 1'h 0;
281 parameter logic [1:0] LC_CTRL_TRANSITION_CTRL_RESVAL = 2'h
0;
282 parameter logic [31:0] LC_CTRL_TRANSITION_TOKEN_0_RESVAL =
32'h 0;
283 parameter logic [31:0] LC_CTRL_TRANSITION_TOKEN_1_RESVAL =
32'h 0;
284 parameter logic [31:0] LC_CTRL_TRANSITION_TOKEN_2_RESVAL =
32'h 0;
285 parameter logic [31:0] LC_CTRL_TRANSITION_TOKEN_3_RESVAL =
32'h 0;
286 parameter logic [29:0] LC_CTRL_TRANSITION_TARGET_RESVAL =
30'h 0;

287 parameter logic [31:0] LC_CTRL_OTP_VENDOR_TEST_CTRL_RESVAL =
32'h 0;
288 parameter logic [31:0] LC_CTRL_OTP_VENDOR_TEST_STATUS_RESVAL =
32'h 0;
289 parameter logic [29:0] LC_CTRL_LC_STATE_RESVAL = 30'h 0;

290 parameter logic [4:0] LC_CTRL_LC_TRANSITION_CNT_RESVAL = 5'h
0;
291 parameter logic [31:0] LC_CTRL_LC_ID_STATE_RESVAL = 32'h 0;
292 parameter logic [31:0] LC_CTRL_HW_REVISION0_RESVAL = 32'h 0;
293 parameter logic [31:0] LC_CTRL_HW_REVISION1_RESVAL = 32'h 0;
294 parameter logic [23:0] LC_CTRL_HW_REVISION1_RESERVED_RESVAL
= 24'h 0;
295 parameter logic [31:0] LC_CTRL_DEVICE_ID_0_RESVAL = 32'h 0;
296 parameter logic [31:0] LC_CTRL_DEVICE_ID_1_RESVAL = 32'h 0;
297 parameter logic [31:0] LC_CTRL_DEVICE_ID_2_RESVAL = 32'h 0;
298 parameter logic [31:0] LC_CTRL_DEVICE_ID_3_RESVAL = 32'h 0;
299 parameter logic [31:0] LC_CTRL_DEVICE_ID_4_RESVAL = 32'h 0;
300 parameter logic [31:0] LC_CTRL_DEVICE_ID_5_RESVAL = 32'h 0;
301 parameter logic [31:0] LC_CTRL_DEVICE_ID_6_RESVAL = 32'h 0;
302 parameter logic [31:0] LC_CTRL_DEVICE_ID_7_RESVAL = 32'h 0;
303 parameter logic [31:0] LC_CTRL_MANUF_STATE_0_RESVAL = 32'h
0;
304 parameter logic [31:0] LC_CTRL_MANUF_STATE_1_RESVAL = 32'h
0;
305 parameter logic [31:0] LC_CTRL_MANUF_STATE_2_RESVAL = 32'h
0;
306 parameter logic [31:0] LC_CTRL_MANUF_STATE_3_RESVAL = 32'h
0;
307 parameter logic [31:0] LC_CTRL_MANUF_STATE_4_RESVAL = 32'h
0;
308 parameter logic [31:0] LC_CTRL_MANUF_STATE_5_RESVAL = 32'h
0;
309 parameter logic [31:0] LC_CTRL_MANUF_STATE_6_RESVAL = 32'h
0;
310 parameter logic [31:0] LC_CTRL_MANUF_STATE_7_RESVAL = 32'h

```

```

0;
311
312 // Register index for regs interface
313 typedef enum int {
314     LC_CTRL_ALERT_TEST,
315     LC_CTRL_STATUS,
316     LC_CTRL_CLAIM_TRANSITION_IF_REGWEN,
317     LC_CTRL_CLAIM_TRANSITION_IF,
318     LC_CTRL_TRANSITION_REGWEN,
319     LC_CTRL_TRANSITION_CMD,
320     LC_CTRL_TRANSITION_CTRL,
321     LC_CTRL_TRANSITION_TOKEN_0,
322     LC_CTRL_TRANSITION_TOKEN_1,
323     LC_CTRL_TRANSITION_TOKEN_2,
324     LC_CTRL_TRANSITION_TOKEN_3,
325     LC_CTRL_TRANSITION_TARGET,
326     LC_CTRL_OTP_VENDOR_TEST_CTRL,
327     LC_CTRL_OTP_VENDOR_TEST_STATUS,
328     LC_CTRL_LC_STATE,
329     LC_CTRL_LC_TRANSITION_CNT,
330     LC_CTRL_LC_ID_STATE,
331     LC_CTRL_HW_REVISION0,
332     LC_CTRL_HW_REVISION1,
333     LC_CTRL_DEVICE_ID_0,
334     LC_CTRL_DEVICE_ID_1,
335     LC_CTRL_DEVICE_ID_2,
336     LC_CTRL_DEVICE_ID_3,
337     LC_CTRL_DEVICE_ID_4,
338     LC_CTRL_DEVICE_ID_5,
339     LC_CTRL_DEVICE_ID_6,
340     LC_CTRL_DEVICE_ID_7,
341     LC_CTRL_MANUF_STATE_0,
342     LC_CTRL_MANUF_STATE_1,
343     LC_CTRL_MANUF_STATE_2,
344     LC_CTRL_MANUF_STATE_3,
345     LC_CTRL_MANUF_STATE_4,
346     LC_CTRL_MANUF_STATE_5,
347     LC_CTRL_MANUF_STATE_6,
348     LC_CTRL_MANUF_STATE_7
349 } lc_ctrl_regs_id_e;
350
351 // Register width information to check illegal writes for
regs interface
352 parameter logic [3:0] LC_CTRL_REGS_PERMIT [35] = '{
353     4'b 0001, // index[ 0] LC_CTRL_ALERT_TEST
354     4'b 0011, // index[ 1] LC_CTRL_STATUS
355     4'b 0001, // index[ 2] LC_CTRL_CLAIM_TRANSITION_IF_REGWEN
356     4'b 0001, // index[ 3] LC_CTRL_CLAIM_TRANSITION_IF
357     4'b 0001, // index[ 4] LC_CTRL_TRANSITION_REGWEN
358     4'b 0001, // index[ 5] LC_CTRL_TRANSITION_CMD
359     4'b 0001, // index[ 6] LC_CTRL_TRANSITION_CTRL
360     4'b 1111, // index[ 7] LC_CTRL_TRANSITION_TOKEN_0
361     4'b 1111, // index[ 8] LC_CTRL_TRANSITION_TOKEN_1
362     4'b 1111, // index[ 9] LC_CTRL_TRANSITION_TOKEN_2
363     4'b 1111, // index[10] LC_CTRL_TRANSITION_TOKEN_3
364     4'b 1111, // index[11] LC_CTRL_TRANSITION_TARGET
365     4'b 1111, // index[12] LC_CTRL_OTP_VENDOR_TEST_CTRL
366     4'b 1111, // index[13] LC_CTRL_OTP_VENDOR_TEST_STATUS
367     4'b 1111, // index[14] LC_CTRL_LC_STATE
368     4'b 0001, // index[15] LC_CTRL_LC_TRANSITION_CNT
369     4'b 1111, // index[16] LC_CTRL_LC_ID_STATE
370     4'b 1111, // index[17] LC_CTRL_HW_REVISION0
371     4'b 1111, // index[18] LC_CTRL_HW_REVISION1
372     4'b 1111, // index[19] LC_CTRL_DEVICE_ID_0
373     4'b 1111, // index[20] LC_CTRL_DEVICE_ID_1
374     4'b 1111, // index[21] LC_CTRL_DEVICE_ID_2
375     4'b 1111, // index[22] LC_CTRL_DEVICE_ID_3
376     4'b 1111, // index[23] LC_CTRL_DEVICE_ID_4
377     4'b 1111, // index[24] LC_CTRL_DEVICE_ID_5
378     4'b 1111, // index[25] LC_CTRL_DEVICE_ID_6
379     4'b 1111, // index[26] LC_CTRL_DEVICE_ID_7
380     4'b 1111, // index[27] LC_CTRL_MANUF_STATE_0
381     4'b 1111, // index[28] LC_CTRL_MANUF_STATE_1
382     4'b 1111, // index[29] LC_CTRL_MANUF_STATE_2
383     4'b 1111, // index[30] LC_CTRL_MANUF_STATE_3
384     4'b 1111, // index[31] LC_CTRL_MANUF_STATE_4
385     4'b 1111, // index[32] LC_CTRL_MANUF_STATE_5
386     4'b 1111, // index[33] LC_CTRL_MANUF_STATE_6
387     4'b 1111, // index[34] LC_CTRL_MANUF_STATE_7
388 };
389
390 // Window parameters for dmi interface
391 parameter logic [DmiAw-1:0] LC_CTRL_DMI_OFFSET = 12'h 0;
392 parameter int unsigned LC_CTRL_DMI_SIZE = 'h 1000;
393 parameter int unsigned LC_CTRL_DMI_IDX = 0;
394
395 endpackage

```

7. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_regs_reg_top.sv (1,525 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"

```

```

8
9 module lc_ctrl_regs_reg_top (
10     input clk_i,
11     input rst_ni,
12     input tlul_pkg::tl_h2d_t tl_i,
13     output tlul_pkg::tl_d2h_t tl_o,
14     // To HW
15     output lc_ctrl_reg_pkg::lc_ctrl_regs_reg2hw_t reg2hw, //
Write
16     input lc_ctrl_reg_pkg::lc_ctrl_regs_hw2reg_t hw2reg, //
Read
17
18     // Integrity check errors
19     output logic intg_err_o
20 );
21
22 import lc_ctrl_reg_pkg::* ;
23
24 localparam int AW = 8;
25 localparam int DW = 32;
26 localparam int DBW = DW/8; // Byte Width
27
28 // register signals
29 logic reg_we;
30 logic reg_re;
31 logic [AW-1:0] reg_addr;
32 logic [DW-1:0] reg_wdata;
33 logic [DBW-1:0] reg_be;
34 logic [DW-1:0] reg_rdata;
35 logic reg_error;
36
37 logic addrmiss, wr_err;
38
39 logic [DW-1:0] reg_rdata_next;
40 logic reg_busy;
41
42 tlul_pkg::tl_h2d_t tl_reg_h2d;
43 tlul_pkg::tl_d2h_t tl_reg_d2h;
44
45 // incoming payload check
46 logic intg_err;
47 tlul_cmd_intg_chk u_chk (
48     .tl_i(tl_i),
49     .err_o(intg_err)
50 );
51
52 // also check for spurious write enables
53 logic reg_we_err;
54 logic [34:0] reg_we_check;
55 prim_reg_we_check #(
56     .OneHotWidth(35)
57 ) u_prim_reg_we_check (
58     .clk_i(clk_i),
59     .rst_ni(rst_ni),
60     .oh_i (reg_we_check),
61     .en_i (reg_we && !addrmiss),
62     .err_o (reg_we_err)
63 );
64
65 logic err_q;
66 always_ff @(posedge clk_i or negedge rst_ni) begin
67     if (!rst_ni) begin
68         err_q <= '0;
69     end else if (intg_err || reg_we_err) begin
70         err_q <= '1'b1;
71     end
72 end
73
74 // integrity error output is permanent and should be used
for alert generation
75 // register errors are transactional
76 assign intg_err_o = err_q | intg_err | reg_we_err;
77
78 // outgoing integrity generation
79 tlul_pkg::tl_d2h_t tl_o_pre;
80 tlul_rsp_intg_gen #(
81     .EnableRspIntgGen(1),
82     .EnableDataIntgGen(1)
83 ) u_rsp_intg_gen (
84     .tl_i(tl_o_pre),
85     .tl_o(tl_o)
86 );
87
88 assign tl_reg_h2d = tl_i;
89 assign tl_o_pre = tl_reg_d2h;
90
91 tlul_adapter_reg #(
92     .RegAw(AW),
93     .RegDw(DW),
94     .EnableDataIntgGen(0)
95 ) u_reg_if (
96     .clk_i (clk_i),
97     .rst_ni (rst_ni),
98
99     .tl_i (tl_reg_h2d),
100    .tl_o (tl_reg_d2h),
101
102    .en_ifetch_i(prim_mubi_pkg::MuBi4False),
103    .intg_error_o(),
104
105    .we_o (reg_we),
106    .re_o (reg_re),
107    .addr_o (reg_addr),
108    .wdata_o (reg_wdata),
109    .be_o (reg_be),
110    .busy_i (reg_busy),
111    .rdata_i (reg_rdata),
112    .error_i (reg_error)
113 );
114
115 // cdc oversampling signals
116
117 assign reg_rdata = reg_rdata_next ;
118 assign reg_error = addrmiss | wr_err | intg_err;
119
120 // Define SW related signals
121 // Format: <reg>_<field>_<wd|we|qs>
122 // or <reg>_<wd|we|qs> if field == 1 or 0
123 logic alert_test_we;
124 logic alert_test_fatal_prog_error_wd;
125 logic alert_test_fatal_state_error_wd;
126 logic alert_test_fatal_bus_integ_error_wd;
127 logic status_re;
128 logic status_initialized_qs;
129 logic status_ready_qs;
130 logic status_ext_clock_switched_qs;
131 logic status_transition_successful_qs;
132 logic status_transition_count_error_qs;
133 logic status_transition_error_qs;
134 logic status_token_error_qs;
135 logic status_nvm_rma_error_qs;
136 logic status_otp_error_qs;
137 logic status_state_error_qs;
138 logic status_bus_integ_error_qs;
139 logic status_otp_partition_error_qs;
140 logic claim_transition_if_regwen_we;
141 logic claim_transition_if_regwen_qs;
142 logic claim_transition_if_regwen_wd;
143 logic claim_transition_if_re;
144 logic claim_transition_if_we;
145 logic [7:0] claim_transition_if_qs;
146 logic [7:0] claim_transition_if_wd;
147 logic transition_regwen_re;
148 logic transition_regwen_qs;
149 logic transition_cmd_we;
150 logic transition_cmd_wd;
151 logic transition_ctrl_re;
152 logic transition_ctrl_we;
153 logic transition_ctrl_ext_clock_en_qs;
154 logic transition_ctrl_ext_clock_en_wd;
155 logic transition_ctrl_volatile_raw_unlock_qs;
156 logic transition_ctrl_volatile_raw_unlock_wd;
157 logic transition_token_0_re;
158 logic transition_token_0_we;
159 logic [31:0] transition_token_0_qs;
160 logic [31:0] transition_token_0_wd;
161 logic transition_token_1_re;
162 logic transition_token_1_we;
163 logic [31:0] transition_token_1_qs;
164 logic [31:0] transition_token_1_wd;
165 logic transition_token_2_re;
166 logic transition_token_2_we;
167 logic [31:0] transition_token_2_qs;
168 logic [31:0] transition_token_2_wd;
169 logic transition_token_3_re;
170 logic transition_token_3_we;
171 logic [31:0] transition_token_3_qs;
172 logic [31:0] transition_token_3_wd;
173 logic transition_target_re;
174 logic transition_target_we;
175 logic [29:0] transition_target_qs;
176 logic [29:0] transition_target_wd;
177 logic otp_vendor_test_ctrl_re;
178 logic otp_vendor_test_ctrl_we;
179 logic [31:0] otp_vendor_test_ctrl_qs;
180 logic [31:0] otp_vendor_test_ctrl_wd;
181 logic otp_vendor_test_status_re;
182 logic [31:0] otp_vendor_test_status_qs;
183 logic lc_state_re;
184 logic [29:0] lc_state_qs;
185 logic lc_transition_cnt_re;
186 logic [4:0] lc_transition_cnt_qs;
187 logic lc_id_state_re;
188 logic [31:0] lc_id_state_qs;
189 logic hw_revision0_re;
190 logic [15:0] hw_revision0_product_id_qs;
191 logic [15:0] hw_revision0_silicon_creator_id_qs;
192 logic hw_revision1_re;
193 logic [7:0] hw_revision1_revision_id_qs;
194 logic [23:0] hw_revision1_reserved_qs;
195 logic device_id_0_re;
196 logic [31:0] device_id_0_qs;
197 logic device_id_1_re;
198 logic [31:0] device_id_1_qs;
199 logic device_id_2_re;
200 logic [31:0] device_id_2_qs;
201 logic device_id_3_re;
202 logic [31:0] device_id_3_qs;
203 logic device_id_4_re;
204 logic [31:0] device_id_4_qs;
205 logic device_id_5_re;
206

```

```

207 logic [31:0] device_id_5_qs;
208 logic device_id_6_re;
209 logic [31:0] device_id_6_qs;
210 logic device_id_7_re;
211 logic [31:0] device_id_7_qs;
212 logic manuf_state_0_re;
213 logic [31:0] manuf_state_0_qs;
214 logic manuf_state_1_re;
215 logic [31:0] manuf_state_1_qs;
216 logic manuf_state_2_re;
217 logic [31:0] manuf_state_2_qs;
218 logic manuf_state_3_re;
219 logic [31:0] manuf_state_3_qs;
220 logic manuf_state_4_re;
221 logic [31:0] manuf_state_4_qs;
222 logic manuf_state_5_re;
223 logic [31:0] manuf_state_5_qs;
224 logic manuf_state_6_re;
225 logic [31:0] manuf_state_6_qs;
226 logic manuf_state_7_re;
227 logic [31:0] manuf_state_7_qs;
228
229 // Register instances
230 // R[alert_test]: V(True)
231 logic alert_test_qe;
232 logic [2:0] alert_test_flds_we;
233 assign alert_test_qe = &alert_test_flds_we;
234 // F[fatal_prog_error]: 0:0
235 prim_subreg_ext #(
236   .DW (1)
237 ) u_alert_test_fatal_prog_error (
238   .re (1'b0),
239   .we (alert_test_we),
240   .wd (alert_test_fatal_prog_error_wd),
241   .d ('0'),
242   .qre (),
243   .qe (alert_test_flds_we[0]),
244   .q (reg2hw.alert_test_fatal_prog_error.q),
245   .ds (),
246   .qs ()
247 );
248 assign reg2hw.alert_test_fatal_prog_error.qe =
alert_test_qe;
249
250 // F[fatal_state_error]: 1:1
251 prim_subreg_ext #(
252   .DW (1)
253 ) u_alert_test_fatal_state_error (
254   .re (1'b0),
255   .we (alert_test_we),
256   .wd (alert_test_fatal_state_error_wd),
257   .d ('0'),
258   .qre (),
259   .qe (alert_test_flds_we[1]),
260   .q (reg2hw.alert_test_fatal_state_error.q),
261   .ds (),
262   .qs ()
263 );
264 assign reg2hw.alert_test_fatal_state_error.qe =
alert_test_qe;
265
266 // F[fatal_bus_integ_error]: 2:2
267 prim_subreg_ext #(
268   .DW (1)
269 ) u_alert_test_fatal_bus_integ_error (
270   .re (1'b0),
271   .we (alert_test_we),
272   .wd (alert_test_fatal_bus_integ_error_wd),
273   .d ('0'),
274   .qre (),
275   .qe (alert_test_flds_we[2]),
276   .q (reg2hw.alert_test_fatal_bus_integ_error.q),
277   .ds (),
278   .qs ()
279 );
280 assign reg2hw.alert_test_fatal_bus_integ_error.qe =
alert_test_qe;
281
282
283 // R[status]: V(True)
284 // F[initialized]: 0:0
285 prim_subreg_ext #(
286   .DW (1)
287 ) u_status_initialized (
288   .re (status_re),
289   .we (1'b0),
290   .wd ('0'),
291   .d (hw2reg.status.initialized.d),
292   .qre (),
293   .qe (),
294   .q (),
295   .ds (),
296   .qs (status_initialized_qs)
297 );
298
299 // F[ready]: 1:1
300 prim_subreg_ext #(
301   .DW (1)
302 ) u_status_ready (
303   .re (status_re),
304   .we (1'b0),

```

```

305   .wd ('0'),
306   .d (hw2reg.status.ready.d),
307   .qre (),
308   .qe (),
309   .q (),
310   .ds (),
311   .qs (status_ready_qs)
312 );
313
314 // F[ext_clock_switched]: 2:2
315 prim_subreg_ext #(
316   .DW (1)
317 ) u_status_ext_clock_switched (
318   .re (status_re),
319   .we (1'b0),
320   .wd ('0'),
321   .d (hw2reg.status.ext_clock_switched.d),
322   .qre (),
323   .qe (),
324   .q (),
325   .ds (),
326   .qs (status_ext_clock_switched_qs)
327 );
328
329 // F[transition_successful]: 3:3
330 prim_subreg_ext #(
331   .DW (1)
332 ) u_status_transition_successful (
333   .re (status_re),
334   .we (1'b0),
335   .wd ('0'),
336   .d (hw2reg.status.transition_successful.d),
337   .qre (),
338   .qe (),
339   .q (),
340   .ds (),
341   .qs (status_transition_successful_qs)
342 );
343
344 // F[transition_count_error]: 4:4
345 prim_subreg_ext #(
346   .DW (1)
347 ) u_status_transition_count_error (
348   .re (status_re),
349   .we (1'b0),
350   .wd ('0'),
351   .d (hw2reg.status.transition_count_error.d),
352   .qre (),
353   .qe (),
354   .q (),
355   .ds (),
356   .qs (status_transition_count_error_qs)
357 );
358
359 // F[transition_error]: 5:5
360 prim_subreg_ext #(
361   .DW (1)
362 ) u_status_transition_error (
363   .re (status_re),
364   .we (1'b0),
365   .wd ('0'),
366   .d (hw2reg.status.transition_error.d),
367   .qre (),
368   .qe (),
369   .q (),
370   .ds (),
371   .qs (status_transition_error_qs)
372 );
373
374 // F[token_error]: 6:6
375 prim_subreg_ext #(
376   .DW (1)
377 ) u_status_token_error (
378   .re (status_re),
379   .we (1'b0),
380   .wd ('0'),
381   .d (hw2reg.status.token_error.d),
382   .qre (),
383   .qe (),
384   .q (),
385   .ds (),
386   .qs (status_token_error_qs)
387 );
388
389 // F[nvm_rma_error]: 7:7
390 prim_subreg_ext #(
391   .DW (1)
392 ) u_status_nvm_rma_error (
393   .re (status_re),
394   .we (1'b0),
395   .wd ('0'),
396   .d (hw2reg.status.nvm_rma_error.d),
397   .qre (),
398   .qe (),
399   .q (),
400   .ds (),
401   .qs (status_nvm_rma_error_qs)
402 );
403
404 // F[otp_error]: 8:8
405 prim_subreg_ext #(

```



```

406 .DW (1)
407 ) u_status_otp_error (
408 .re (status_re),
409 .we (1'b0),
410 .wd ('0'),
411 .d (hw2reg.status.otp_error.d),
412 .qre (),
413 .qe (),
414 .q (),
415 .ds (),
416 .qs (status_otp_error_qs)
417 );
418
419 // F[state_error]: 9:9
420 prim_subreg_ext #(
421 .DW (1)
422 ) u_status_state_error (
423 .re (status_re),
424 .we (1'b0),
425 .wd ('0'),
426 .d (hw2reg.status.state_error.d),
427 .qre (),
428 .qe (),
429 .q (),
430 .ds (),
431 .qs (status_state_error_qs)
432 );
433
434 // F[bus_integ_error]: 10:10
435 prim_subreg_ext #(
436 .DW (1)
437 ) u_status_bus_integ_error (
438 .re (status_re),
439 .we (1'b0),
440 .wd ('0'),
441 .d (hw2reg.status.bus_integ_error.d),
442 .qre (),
443 .qe (),
444 .q (),
445 .ds (),
446 .qs (status_bus_integ_error_qs)
447 );
448
449 // F[otp_partition_error]: 11:11
450 prim_subreg_ext #(
451 .DW (1)
452 ) u_status_otp_partition_error (
453 .re (status_re),
454 .we (1'b0),
455 .wd ('0'),
456 .d (hw2reg.status.otp_partition_error.d),
457 .qre (),
458 .qe (),
459 .q (),
460 .ds (),
461 .qs (status_otp_partition_error_qs)
462 );
463
464 // R[claim_transition_if_regwen]: V(False)
465 prim_subreg_ext #(
466 .DW (1),
467 .SwAccess(prim_subreg_pkg::SwAccessW0C),
468 .RESVAL (1'h1),
469 .Mubi (1'b0)
470 ) u_claim_transition_if_regwen (
471 .clk_i (clk_i),
472 .rst_ni (rst_ni),
473
474 // from register interface
475 .we (claim_transition_if_regwen_we),
476 .wd (claim_transition_if_regwen_wd),
477
478 // from internal hardware
479 .de (1'b0),
480 .d ('0'),
481
482 // to internal hardware
483 .qe (),
484 .q (),
485 .ds (),
486
487 // to register interface (read)
488 .qs (claim_transition_if_regwen_qs)
489 );
490
491 // R[claim_transition_if]: V(True)
492 logic claim_transition_if_qe;
493 logic [0:0] claim_transition_if_flds_we;
494 assign claim_transition_if_qe =
&claim_transition_if_flds_we;
495 // Create REGWEN-gated WE signal
496 logic claim_transition_if_gated_we;
497
498 assign claim_transition_if_gated_we = claim_transition_if_we &
claim_transition_if_regwen_qs;
499 prim_subreg_ext #(
500 .DW (8)
501 ) u_claim_transition_if (
502 .re (claim_transition_if_re),

```

```

504 .we (claim_transition_if_gated_we),
505 .wd (claim_transition_if_wd),
506 .d (hw2reg.claim_transition_if.d),
507 .qre (),
508 .qe (claim_transition_if_flds_we[0]),
509 .q (reg2hw.claim_transition_if.q),
510 .ds (),
511 .qs (claim_transition_if_qs)
512 );
513 assign reg2hw.claim_transition_if.qe =
claim_transition_if_qe;
514
515 // R[transition_regwen]: V(True)
516 prim_subreg_ext #(
517 .DW (1)
518 ) u_transition_regwen (
519 .re (transition_regwen_re),
520 .we (1'b0),
521 .wd ('0'),
522 .d (hw2reg.transition_regwen.d),
523 .qre (),
524 .qe (),
525 .q (),
526 .ds (),
527 .qs (transition_regwen_qs)
528 );
529
530 // R[transition_cmd]: V(True)
531 logic transition_cmd_qe;
532 logic [0:0] transition_cmd_flds_we;
533 assign transition_cmd_qe = &transition_cmd_flds_we;
534 // Create REGWEN-gated WE signal
535 logic transition_cmd_gated_we;
536 assign transition_cmd_gated_we = transition_cmd_we & trans-
ition_regwen_qs;
537 prim_subreg_ext #(
538 .DW (1)
539 ) u_transition_cmd (
540 .re (1'b0),
541 .we (transition_cmd_gated_we),
542 .wd (transition_cmd_wd),
543 .d ('0'),
544 .qre (),
545 .qe (transition_cmd_flds_we[0]),
546 .q (reg2hw.transition_cmd.q),
547 .ds (),
548 .qs ()
549 );
550 assign reg2hw.transition_cmd.qe = transition_cmd_qe;
551
552 // R[transition_ctrl]: V(True)
553 logic transition_ctrl_qe;
554 logic [1:0] transition_ctrl_flds_we;
555 assign transition_ctrl_qe = &transition_ctrl_flds_we;
556 // Create REGWEN-gated WE signal
557 logic transition_ctrl_gated_we;
558 assign transition_ctrl_gated_we = transition_ctrl_we &
transition_regwen_qs;
559 // F[ext_clock_en]: 0:0
560 prim_subreg_ext #(
561 .DW (1)
562 ) u_transition_ctrl_ext_clock_en (
563 .re (transition_ctrl_re),
564 .we (transition_ctrl_gated_we),
565 .wd (transition_ctrl_ext_clock_en_wd),
566 .d (hw2reg.transition_ctrl_ext_clock_en.d),
567 .qre (),
568 .qe (transition_ctrl_flds_we[1]),
569 .q (reg2hw.transition_ctrl_ext_clock_en.q),
570 .ds (),
571 .qs (transition_ctrl_ext_clock_en_qs)
572 );
573 assign reg2hw.transition_ctrl_ext_clock_en.qe = trans-
ition_ctrl_qe;
574
575 // F[volatile_raw_unlock]: 1:1
576 prim_subreg_ext #(
577 .DW (1)
578 ) u_transition_ctrl_volatile_raw_unlock (
579 .re (transition_ctrl_re),
580 .we (transition_ctrl_gated_we),
581 .wd (transition_ctrl_volatile_raw_unlock_wd),
582 .d (hw2reg.transition_ctrl_volatile_raw_unlock.d),
583 .qre (),
584 .qe (transition_ctrl_flds_we[1]),
585 .q (reg2hw.transition_ctrl.volatile_raw_unlock.q),
586 .ds (),
587 .qs (transition_ctrl_volatile_raw_unlock_qs)
588 );
589 assign reg2hw.transition_ctrl.volatile_raw_unlock.qe =
transition_ctrl_qe;
590
591 // Subregister 0 of Multireg transition_token
592 // R[transition_token_0]: V(True)
593 logic transition_token_0_qe;
594 logic [0:0] transition_token_0_flds_we;
595 assign transition_token_0_qe = &transition_token_0_flds_we;

```

```

600 // Create REGWEN-gated WE signal
601 logic transition_token_0_gated_we;

602 assign transition_token_0_gated_we = transition_token_0_we &
transition_regwen_qs;
603 prim_subreg_ext #(
604     .DW (32)
605 ) u_transition_token_0 (
606     .re (transition_token_0_re),
607     .we (transition_token_0_gated_we),
608     .wd (transition_token_0_wd),
609     .d (hw2reg.transition_token[0].d),
610     .qre (),
611     .qe (transition_token_0_flds_we[0]),
612     .q (reg2hw.transition_token[0].q),
613     .ds (),
614     .qs (transition_token_0_qs)
615 );
616 assign reg2hw.transition_token[0].qe = trans-
ition_token_0_qe;
617
618 // Subregister 1 of Multireg transition_token
619 // R[transition_token_1]: V(True)
620 logic transition_token_1_qe;
621 logic [0:0] transition_token_1_flds_we;
622 assign transition_token_1_qe = &transition_token_1_flds_we;
623 // Create REGWEN-gated WE signal
624 logic transition_token_1_gated_we;

626 assign transition_token_1_gated_we = transition_token_1_we &
transition_regwen_qs;
627 prim_subreg_ext #(
628     .DW (32)
629 ) u_transition_token_1 (
630     .re (transition_token_1_re),
631     .we (transition_token_1_gated_we),
632     .wd (transition_token_1_wd),
633     .d (hw2reg.transition_token[1].d),
634     .qre (),
635     .qe (transition_token_1_flds_we[0]),
636     .q (reg2hw.transition_token[1].q),
637     .ds (),
638     .qs (transition_token_1_qs)
639 );
640 assign reg2hw.transition_token[1].qe = trans-
ition_token_1_qe;
641
642 // Subregister 2 of Multireg transition_token
643 // R[transition_token_2]: V(True)
644 logic transition_token_2_qe;
645 logic [0:0] transition_token_2_flds_we;
646 assign transition_token_2_qe = &transition_token_2_flds_we;
647 // Create REGWEN-gated WE signal
648 logic transition_token_2_gated_we;

650 assign transition_token_2_gated_we = transition_token_2_we &
transition_regwen_qs;
651 prim_subreg_ext #(
652     .DW (32)
653 ) u_transition_token_2 (
654     .re (transition_token_2_re),
655     .we (transition_token_2_gated_we),
656     .wd (transition_token_2_wd),
657     .d (hw2reg.transition_token[2].d),
658     .qre (),
659     .qe (transition_token_2_flds_we[0]),
660     .q (reg2hw.transition_token[2].q),
661     .ds (),
662     .qs (transition_token_2_qs)
663 );
664 assign reg2hw.transition_token[2].qe = trans-
ition_token_2_qe;
665
666 // Subregister 3 of Multireg transition_token
667 // R[transition_token_3]: V(True)
668 logic transition_token_3_qe;
669 logic [0:0] transition_token_3_flds_we;
670 assign transition_token_3_qe = &transition_token_3_flds_we;
671 // Create REGWEN-gated WE signal
672 logic transition_token_3_gated_we;

674 assign transition_token_3_gated_we = transition_token_3_we &
transition_regwen_qs;
675 prim_subreg_ext #(
676     .DW (32)
677 ) u_transition_token_3 (
678     .re (transition_token_3_re),
679     .we (transition_token_3_gated_we),
680     .wd (transition_token_3_wd),
681     .d (hw2reg.transition_token[3].d),
682     .qre (),
683     .qe (transition_token_3_flds_we[0]),
684     .q (reg2hw.transition_token[3].q),
685     .ds (),
686     .qs (transition_token_3_qs)
687 );
688 assign reg2hw.transition_token[3].qe = trans-
ition_token_3_qe;

```

```

689
690 // R[transition_target]: V(True)
691 logic transition_target_qe;
692 logic [0:0] transition_target_flds_we;
693 assign transition_target_qe = &transition_target_flds_we;
694 // Create REGWEN-gated WE signal
695 logic transition_target_gated_we;
696 assign transition_target_gated_we = transition_target_we &
transition_regwen_qs;
697 prim_subreg_ext #(
698     .DW (30)
699 ) u_transition_target (
700     .re (transition_target_re),
701     .we (transition_target_gated_we),
702     .wd (transition_target_wd),
703     .d (hw2reg.transition_target.d),
704     .qre (),
705     .qe (transition_target_flds_we[0]),
706     .q (reg2hw.transition_target.q),
707     .ds (),
708     .qs (transition_target_qs)
709 );
710 assign reg2hw.transition_target.qe = transition_target_qe;
711
712 // R[otp_vendor_test_ctrl]: V(True)
713 logic otp_vendor_test_ctrl_qe;
714 logic [0:0] otp_vendor_test_ctrl_flds_we;
715 assign otp_vendor_test_ctrl_qe =
&otp_vendor_test_ctrl_flds_we;
716 // Create REGWEN-gated WE signal
717 logic otp_vendor_test_ctrl_gated_we;
718 assign otp_vendor_test_ctrl_gated_we =
otp_vendor_test_ctrl_we & transition_regwen_qs;
719 prim_subreg_ext #(
720     .DW (32)
721 ) u_otp_vendor_test_ctrl (
722     .re (otp_vendor_test_ctrl_re),
723     .we (otp_vendor_test_ctrl_gated_we),
724     .wd (otp_vendor_test_ctrl_wd),
725     .d (hw2reg.otp_vendor_test_ctrl.d),
726     .qre (),
727     .qe (otp_vendor_test_ctrl_flds_we[0]),
728     .q (reg2hw.otp_vendor_test_ctrl.q),
729     .ds (),
730     .qs (otp_vendor_test_ctrl_qs)
731 );
732 assign reg2hw.otp_vendor_test_ctrl.qe =
otp_vendor_test_ctrl_qe;
733
734 // R[otp_vendor_test_status]: V(True)
735 prim_subreg_ext #(
736     .DW (32)
737 ) u_otp_vendor_test_status (
738     .re (otp_vendor_test_status_re),
739     .we (1'b0),
740     .wd ('0'),
741     .d (hw2reg.otp_vendor_test_status.d),
742     .qre (),
743     .qe (),
744     .q (),
745     .ds (),
746     .qs (otp_vendor_test_status_qs)
747 );
748
749 // R[lc_state]: V(True)
750 prim_subreg_ext #(
751     .DW (30)
752 ) u_lc_state (
753     .re (lc_state_re),
754     .we (1'b0),
755     .wd ('0'),
756     .d (hw2reg.lc_state.d),
757     .qre (),
758     .qe (),
759     .q (),
760     .ds (),
761     .qs (lc_state_qs)
762 );
763
764 // R[lc_transition_cnt]: V(True)
765 prim_subreg_ext #(
766     .DW (5)
767 ) u_lc_transition_cnt (
768     .re (lc_transition_cnt_re),
769     .we (1'b0),
770     .wd ('0'),
771     .d (hw2reg.lc_transition_cnt.d),
772     .qre (),
773     .qe (),
774     .q (),
775     .ds (),
776     .qs (lc_transition_cnt_qs)
777 );
778
779 // R[lc_id_state]: V(True)

```

```

786 prim_subreg_ext #(
787     .DW (32)
788 ) u_lc_id_state (
789     .re (lc_id_state_re),
790     .we (1'b0),
791     .wd ('0'),
792     .d (hw2reg.lc_id_state.d),
793     .qre (),
794     .qe (),
795     .q (),
796     .ds (),
797     .qs (lc_id_state_qs)
798 );
799
800 // R[hw_revision0]: V(True)
801 // F[product_id]: 15:0
802 prim_subreg_ext #(
803     .DW (16)
804 ) u_hw_revision0_product_id (
805     .re (hw_revision0_re),
806     .we (1'b0),
807     .wd ('0'),
808     .d (hw2reg.hw_revision0.product_id.d),
809     .qre (),
810     .qe (),
811     .q (),
812     .ds (),
813     .qs (hw_revision0_product_id_qs)
814 );
815
816 // F[silicon_creator_id]: 31:16
817 prim_subreg_ext #(
818     .DW (16)
819 ) u_hw_revision0_silicon_creator_id (
820     .re (hw_revision0_re),
821     .we (1'b0),
822     .wd ('0'),
823     .d (hw2reg.hw_revision0.silicon_creator_id.d),
824     .qre (),
825     .qe (),
826     .q (),
827     .ds (),
828     .qs (hw_revision0_silicon_creator_id_qs)
829 );
830
831 // R[hw_revision1]: V(True)
832 // F[revision_id]: 7:0
833 prim_subreg_ext #(
834     .DW (8)
835 ) u_hw_revision1_revision_id (
836     .re (hw_revision1_re),
837     .we (1'b0),
838     .wd ('0'),
839     .d (hw2reg.hw_revision1.revision_id.d),
840     .qre (),
841     .qe (),
842     .q (),
843     .ds (),
844     .qs (hw_revision1_revision_id_qs)
845 );
846
847 // F[reserved]: 31:8
848 prim_subreg_ext #(
849     .DW (24)
850 ) u_hw_revision1_reserved (
851     .re (hw_revision1_re),
852     .we (1'b0),
853     .wd ('0'),
854     .d (hw2reg.hw_revision1.reserved.d),
855     .qre (),
856     .qe (),
857     .q (),
858     .ds (),
859     .qs (hw_revision1_reserved_qs)
860 );
861
862 // Subregister 0 of Multireg device_id
863 // R[device_id_0]: V(True)
864 prim_subreg_ext #(
865     .DW (32)
866 ) u_device_id_0 (
867     .re (device_id_0_re),
868     .we (1'b0),
869     .wd ('0'),
870     .d (hw2reg.device_id[0].d),
871     .qre (),
872     .qe (),
873     .q (),
874     .ds (),
875     .qs (device_id_0_qs)
876 );
877
878 // Subregister 1 of Multireg device_id
879 // R[device_id_1]: V(True)
880 prim_subreg_ext #(
881     .DW (32)
882 ) u_device_id_1 (

```

```

887     .re (device_id_1_re),
888     .we (1'b0),
889     .wd ('0'),
890     .d (hw2reg.device_id[1].d),
891     .qre (),
892     .qe (),
893     .q (),
894     .ds (),
895     .qs (device_id_1_qs)
896 );
897
898 // Subregister 2 of Multireg device_id
899 // R[device_id_2]: V(True)
900 prim_subreg_ext #(
901     .DW (32)
902 ) u_device_id_2 (
903     .re (device_id_2_re),
904     .we (1'b0),
905     .wd ('0'),
906     .d (hw2reg.device_id[2].d),
907     .qre (),
908     .qe (),
909     .q (),
910     .ds (),
911     .qs (device_id_2_qs)
912 );
913
914 // Subregister 3 of Multireg device_id
915 // R[device_id_3]: V(True)
916 prim_subreg_ext #(
917     .DW (32)
918 ) u_device_id_3 (
919     .re (device_id_3_re),
920     .we (1'b0),
921     .wd ('0'),
922     .d (hw2reg.device_id[3].d),
923     .qre (),
924     .qe (),
925     .q (),
926     .ds (),
927     .qs (device_id_3_qs)
928 );
929
930 // Subregister 4 of Multireg device_id
931 // R[device_id_4]: V(True)
932 prim_subreg_ext #(
933     .DW (32)
934 ) u_device_id_4 (
935     .re (device_id_4_re),
936     .we (1'b0),
937     .wd ('0'),
938     .d (hw2reg.device_id[4].d),
939     .qre (),
940     .qe (),
941     .q (),
942     .ds (),
943     .qs (device_id_4_qs)
944 );
945
946 // Subregister 5 of Multireg device_id
947 // R[device_id_5]: V(True)
948 prim_subreg_ext #(
949     .DW (32)
950 ) u_device_id_5 (
951     .re (device_id_5_re),
952     .we (1'b0),
953     .wd ('0'),
954     .d (hw2reg.device_id[5].d),
955     .qre (),
956     .qe (),
957     .q (),
958     .ds (),
959     .qs (device_id_5_qs)
960 );
961
962 // Subregister 6 of Multireg device_id
963 // R[device_id_6]: V(True)
964 prim_subreg_ext #(
965     .DW (32)
966 ) u_device_id_6 (
967     .re (device_id_6_re),
968     .we (1'b0),
969     .wd ('0'),
970     .d (hw2reg.device_id[6].d),
971     .qre (),
972     .qe (),
973     .q (),
974     .ds (),
975     .qs (device_id_6_qs)
976 );
977
978 // Subregister 7 of Multireg device_id
979 // R[device_id_7]: V(True)
980 prim_subreg_ext #(
981     .DW (32)
982 ) u_device_id_7 (

```

```

988 ) u_device_id_7 (
989     .re (device_id_7_re),
990     .we (1'b0),
991     .wd ('0'),
992     .d (hw2reg.device_id[7].d),
993     .qre (),
994     .qe (),
995     .q (),
996     .ds (),
997     .qs (device_id_7_qs)
998 );
999
1000 // Subregister 0 of Multireg manuf_state
1001 // R[manuf_state_0]: V(True)
1002 prim_subreg_ext #(
1003     .DW (32)
1004 ) u_manuf_state_0 (
1005     .re (manuf_state_0_re),
1006     .we (1'b0),
1007     .wd ('0'),
1008     .d (hw2reg.manuf_state[0].d),
1009     .qre (),
1010     .qe (),
1011     .q (),
1012     .ds (),
1013     .qs (manuf_state_0_qs)
1014 );
1015
1016 // Subregister 1 of Multireg manuf_state
1017 // R[manuf_state_1]: V(True)
1018 prim_subreg_ext #(
1019     .DW (32)
1020 ) u_manuf_state_1 (
1021     .re (manuf_state_1_re),
1022     .we (1'b0),
1023     .wd ('0'),
1024     .d (hw2reg.manuf_state[1].d),
1025     .qre (),
1026     .qe (),
1027     .q (),
1028     .ds (),
1029     .qs (manuf_state_1_qs)
1030 );
1031
1032 // Subregister 2 of Multireg manuf_state
1033 // R[manuf_state_2]: V(True)
1034 prim_subreg_ext #(
1035     .DW (32)
1036 ) u_manuf_state_2 (
1037     .re (manuf_state_2_re),
1038     .we (1'b0),
1039     .wd ('0'),
1040     .d (hw2reg.manuf_state[2].d),
1041     .qre (),
1042     .qe (),
1043     .q (),
1044     .ds (),
1045     .qs (manuf_state_2_qs)
1046 );
1047
1048 // Subregister 3 of Multireg manuf_state
1049 // R[manuf_state_3]: V(True)
1050 prim_subreg_ext #(
1051     .DW (32)
1052 ) u_manuf_state_3 (
1053     .re (manuf_state_3_re),
1054     .we (1'b0),
1055     .wd ('0'),
1056     .d (hw2reg.manuf_state[3].d),
1057     .qre (),
1058     .qe (),
1059     .q (),
1060     .ds (),
1061     .qs (manuf_state_3_qs)
1062 );
1063
1064 // Subregister 4 of Multireg manuf_state
1065 // R[manuf_state_4]: V(True)
1066 prim_subreg_ext #(
1067     .DW (32)
1068 ) u_manuf_state_4 (
1069     .re (manuf_state_4_re),
1070     .we (1'b0),
1071     .wd ('0'),
1072     .d (hw2reg.manuf_state[4].d),
1073     .qre (),
1074     .qe (),
1075     .q (),
1076     .ds (),
1077     .qs (manuf_state_4_qs)
1078 );
1079
1080 // Subregister 5 of Multireg manuf_state
1081 // R[manuf_state_5]: V(True)
1082 prim_subreg_ext #(
1083     .DW (32)
1084 ) u_manuf_state_5 (
1085     .re (manuf_state_5_re),
1086     .we (1'b0),
1087     .wd ('0'),
1088     .d (hw2reg.manuf_state[5].d),
1089     .qre (),
1090     .qe (),
1091     .q (),
1092     .ds (),
1093     .qs (manuf_state_5_qs)
1094 );
1095
1096 // Subregister 6 of Multireg manuf_state
1097 // R[manuf_state_6]: V(True)
1098 prim_subreg_ext #(
1099     .DW (32)
1100 ) u_manuf_state_6 (
1101     .re (manuf_state_6_re),
1102     .we (1'b0),
1103     .wd ('0'),
1104     .d (hw2reg.manuf_state[6].d),
1105     .qre (),
1106     .qe (),
1107     .q (),
1108     .ds (),
1109     .qs (manuf_state_6_qs)
1110 );
1111
1112 // Subregister 7 of Multireg manuf_state
1113 // R[manuf_state_7]: V(True)
1114 prim_subreg_ext #(
1115     .DW (32)
1116 ) u_manuf_state_7 (
1117     .re (manuf_state_7_re),
1118     .we (1'b0),
1119     .wd ('0'),
1120     .d (hw2reg.manuf_state[7].d),
1121     .qre (),
1122     .qe (),
1123     .q (),
1124     .ds (),
1125     .qs (manuf_state_7_qs)
1126 );
1127
1128 logic [34:0] addr_hit;
1129 always_comb begin
1130     addr_hit[ 0] = (reg_addr == LC_CTRL_ALERT_TEST_OFFSET);
1131     addr_hit[ 1] = (reg_addr == LC_CTRL_STATUS_OFFSET);
1132     addr_hit[ 2] = (reg_addr == LC_C-
1133     TR_LCLAIM_TRANSITION_IF_REGWEN_OFFSET);
1134     addr_hit[ 3] = (reg_addr == LC_C-
1135     TR_LCLAIM_TRANSITION_IF_OFFSET);
1136     addr_hit[ 4] = (reg_addr == LC_C-
1137     TR_LTRANSITION_REGWEN_OFFSET);
1138     addr_hit[ 5] = (reg_addr == LC_C-
1139     TR_LTRANSITION_CMD_OFFSET);
1140     addr_hit[ 6] = (reg_addr == LC_C-
1141     TR_LTRANSITION_CTRL_OFFSET);
1142     addr_hit[ 7] = (reg_addr == LC_C-
1143     TR_LTRANSITION_TOKEN_0_OFFSET);
1144     addr_hit[ 8] = (reg_addr == LC_C-
1145     TR_LTRANSITION_TOKEN_1_OFFSET);
1146     addr_hit[ 9] = (reg_addr == LC_C-
1147     TR_LTRANSITION_TOKEN_2_OFFSET);
1148     addr_hit[10] = (reg_addr == LC_C-
1149     TR_LTRANSITION_TOKEN_3_OFFSET);
1150     addr_hit[11] = (reg_addr == LC_C-
1151     TR_LTRANSITION_TARGET_OFFSET);
1152     addr_hit[12] = (reg_addr == LC_C-
1153     TR_OTP_VENDOR_TEST_CTRL_OFFSET);
1154     addr_hit[13] = (reg_addr == LC_C-
1155     TR_OTP_VENDOR_TEST_STATUS_OFFSET);
1156     addr_hit[14] = (reg_addr == LC_CTRL_LC_STATE_OFFSET);
1157     addr_hit[15] = (reg_addr == LC_C-
1158     TR_LC_TRANSITION_CNT_OFFSET);
1159     addr_hit[16] = (reg_addr == LC_CTRL_LC_ID_STATE_OFFSET);
1160     addr_hit[17] = (reg_addr == LC_CTRL_HW_REVISION0_OFFSET);
1161     addr_hit[18] = (reg_addr == LC_CTRL_HW_REVISION1_OFFSET);
1162     addr_hit[19] = (reg_addr == LC_CTRL_DEVICE_ID_0_OFFSET);
1163     addr_hit[20] = (reg_addr == LC_CTRL_DEVICE_ID_1_OFFSET);
1164     addr_hit[21] = (reg_addr == LC_CTRL_DEVICE_ID_2_OFFSET);
1165     addr_hit[22] = (reg_addr == LC_CTRL_DEVICE_ID_3_OFFSET);
1166     addr_hit[23] = (reg_addr == LC_CTRL_DEVICE_ID_4_OFFSET);
1167     addr_hit[24] = (reg_addr == LC_CTRL_DEVICE_ID_5_OFFSET);
1168     addr_hit[25] = (reg_addr == LC_CTRL_DEVICE_ID_6_OFFSET);
1169     addr_hit[26] = (reg_addr == LC_CTRL_DEVICE_ID_7_OFFSET);
1170     addr_hit[27] = (reg_addr == LC_CTRL_MANUF_STATE_0_OFFSET);
1171     addr_hit[28] = (reg_addr == LC_CTRL_MANUF_STATE_1_OFFSET);
1172     addr_hit[29] = (reg_addr == LC_CTRL_MANUF_STATE_2_OFFSET);
1173     addr_hit[30] = (reg_addr == LC_CTRL_MANUF_STATE_3_OFFSET);
1174     addr_hit[31] = (reg_addr == LC_CTRL_MANUF_STATE_4_OFFSET);
1175     addr_hit[32] = (reg_addr == LC_CTRL_MANUF_STATE_5_OFFSET);
1176     addr_hit[33] = (reg_addr == LC_CTRL_MANUF_STATE_6_OFFSET);
1177     addr_hit[34] = (reg_addr == LC_CTRL_MANUF_STATE_7_OFFSET);
1178 end
1179

```

```

1177 assign addrmiss = (reg_re || reg_we) ? ~|addr_hit : 1'b0 ;
1178
1179 // Check sub-word write is permitted
1180 always_comb begin
1181   wr_err = (reg_we &
1182     ((addr_hit[ 0] & (!(LC_CTRL_REGS_PERMIT[ 0] &
~reg_be))) |
1183     (addr_hit[ 1] & (!(LC_CTRL_REGS_PERMIT[ 1] &
~reg_be))) |
1184     (addr_hit[ 2] & (!(LC_CTRL_REGS_PERMIT[ 2] &
~reg_be))) |
1185     (addr_hit[ 3] & (!(LC_CTRL_REGS_PERMIT[ 3] &
~reg_be))) |
1186     (addr_hit[ 4] & (!(LC_CTRL_REGS_PERMIT[ 4] &
~reg_be))) |
1187     (addr_hit[ 5] & (!(LC_CTRL_REGS_PERMIT[ 5] &
~reg_be))) |
1188     (addr_hit[ 6] & (!(LC_CTRL_REGS_PERMIT[ 6] &
~reg_be))) |
1189     (addr_hit[ 7] & (!(LC_CTRL_REGS_PERMIT[ 7] &
~reg_be))) |
1190     (addr_hit[ 8] & (!(LC_CTRL_REGS_PERMIT[ 8] &
~reg_be))) |
1191     (addr_hit[ 9] & (!(LC_CTRL_REGS_PERMIT[ 9] &
~reg_be))) |
1192     (addr_hit[10] & (!(LC_CTRL_REGS_PERMIT[10] &
~reg_be))) |
1193     (addr_hit[11] & (!(LC_CTRL_REGS_PERMIT[11] &
~reg_be))) |
1194     (addr_hit[12] & (!(LC_CTRL_REGS_PERMIT[12] &
~reg_be))) |
1195     (addr_hit[13] & (!(LC_CTRL_REGS_PERMIT[13] &
~reg_be))) |
1196     (addr_hit[14] & (!(LC_CTRL_REGS_PERMIT[14] &
~reg_be))) |
1197     (addr_hit[15] & (!(LC_CTRL_REGS_PERMIT[15] &
~reg_be))) |
1198     (addr_hit[16] & (!(LC_CTRL_REGS_PERMIT[16] &
~reg_be))) |
1199     (addr_hit[17] & (!(LC_CTRL_REGS_PERMIT[17] &
~reg_be))) |
1200     (addr_hit[18] & (!(LC_CTRL_REGS_PERMIT[18] &
~reg_be))) |
1201     (addr_hit[19] & (!(LC_CTRL_REGS_PERMIT[19] &
~reg_be))) |
1202     (addr_hit[20] & (!(LC_CTRL_REGS_PERMIT[20] &
~reg_be))) |
1203     (addr_hit[21] & (!(LC_CTRL_REGS_PERMIT[21] &
~reg_be))) |
1204     (addr_hit[22] & (!(LC_CTRL_REGS_PERMIT[22] &
~reg_be))) |
1205     (addr_hit[23] & (!(LC_CTRL_REGS_PERMIT[23] &
~reg_be))) |
1206     (addr_hit[24] & (!(LC_CTRL_REGS_PERMIT[24] &
~reg_be))) |
1207     (addr_hit[25] & (!(LC_CTRL_REGS_PERMIT[25] &
~reg_be))) |
1208     (addr_hit[26] & (!(LC_CTRL_REGS_PERMIT[26] &
~reg_be))) |
1209     (addr_hit[27] & (!(LC_CTRL_REGS_PERMIT[27] &
~reg_be))) |
1210     (addr_hit[28] & (!(LC_CTRL_REGS_PERMIT[28] &
~reg_be))) |
1211     (addr_hit[29] & (!(LC_CTRL_REGS_PERMIT[29] &
~reg_be))) |
1212     (addr_hit[30] & (!(LC_CTRL_REGS_PERMIT[30] &
~reg_be))) |
1213     (addr_hit[31] & (!(LC_CTRL_REGS_PERMIT[31] &
~reg_be))) |
1214     (addr_hit[32] & (!(LC_CTRL_REGS_PERMIT[32] &
~reg_be))) |
1215     (addr_hit[33] & (!(LC_CTRL_REGS_PERMIT[33] &
~reg_be))) |
1216     (addr_hit[34] & (!(LC_CTRL_REGS_PERMIT[34] &
~reg_be)))));
1217 end
1218
1219 // Generate write-enables
1220 assign alert_test_we = addr_hit[0] & reg_we & !reg_error;
1221
1222 assign alert_test_fatal_prog_error_wd = reg_wdata[0];
1223
1224 assign alert_test_fatal_state_error_wd = reg_wdata[1];
1225
1226 assign alert_test_fatal_bus_integ_error_wd = reg_wdata[2];
1227 assign status_re = addr_hit[1] & reg_re & !reg_error;
1228 assign claim_transition_if_regwen_we = addr_hit[2] & reg_we
& !reg_error;
1229
1230 assign claim_transition_if_regwen_wd = reg_wdata[0];
1231 assign claim_transition_if_re = addr_hit[3] & reg_re & !
reg_error;
1232 assign claim_transition_if_we = addr_hit[3] & reg_we & !
reg_error;
1233
1234 assign claim_transition_if_wd = reg_wdata[7:0];
1235 assign transition_regwen_re = addr_hit[4] & reg_re & !
reg_error;
1236 assign transition_cmd_we = addr_hit[5] & reg_we & !reg_er-
ror;
1237

```

```

1238 assign transition_cmd_wd = reg_wdata[0];
1239 assign transition_ctrl_re = addr_hit[6] & reg_re & !reg_er-
ror;
1240 assign transition_ctrl_we = addr_hit[6] & reg_we & !reg_er-
ror;
1241
1242 assign transition_ctrl_ext_clock_en_wd = reg_wdata[0];
1243
1244 assign transition_ctrl_volatile_raw_unlock_wd =
reg_wdata[1];
1245 assign transition_token_0_re = addr_hit[7] & reg_re & !
reg_error;
1246 assign transition_token_0_we = addr_hit[7] & reg_we & !
reg_error;
1247
1248 assign transition_token_0_wd = reg_wdata[31:0];
1249 assign transition_token_1_re = addr_hit[8] & reg_re & !
reg_error;
1250 assign transition_token_1_we = addr_hit[8] & reg_we & !
reg_error;
1251
1252 assign transition_token_1_wd = reg_wdata[31:0];
1253 assign transition_token_2_re = addr_hit[9] & reg_re & !
reg_error;
1254 assign transition_token_2_we = addr_hit[9] & reg_we & !
reg_error;
1255
1256 assign transition_token_2_wd = reg_wdata[31:0];
1257 assign transition_token_3_re = addr_hit[10] & reg_re & !
reg_error;
1258 assign transition_token_3_we = addr_hit[10] & reg_we & !
reg_error;
1259
1260 assign transition_token_3_wd = reg_wdata[31:0];
1261 assign transition_target_re = addr_hit[11] & reg_re & !
reg_error;
1262 assign transition_target_we = addr_hit[11] & reg_we & !
reg_error;
1263
1264 assign transition_target_wd = reg_wdata[29:0];
1265 assign otp_vendor_test_ctrl_re = addr_hit[12] & reg_re & !
reg_error;
1266 assign otp_vendor_test_ctrl_we = addr_hit[12] & reg_we & !
reg_error;
1267
1268 assign otp_vendor_test_ctrl_wd = reg_wdata[31:0];
1269 assign otp_vendor_test_status_re = addr_hit[13] & reg_re & !
reg_error;
1270 assign lc_state_re = addr_hit[14] & reg_re & !reg_error;
1271 assign lc_transition_cnt_re = addr_hit[15] & reg_re & !
reg_error;
1272 assign lc_id_state_re = addr_hit[16] & reg_re & !reg_error;
1273 assign hw_revision0_re = addr_hit[17] & reg_re & !reg_error;
1274 assign hw_revision1_re = addr_hit[18] & reg_re & !reg_error;
1275 assign device_id_0_re = addr_hit[19] & reg_re & !reg_error;
1276 assign device_id_1_re = addr_hit[20] & reg_re & !reg_error;
1277 assign device_id_2_re = addr_hit[21] & reg_re & !reg_error;
1278 assign device_id_3_re = addr_hit[22] & reg_re & !reg_error;
1279 assign device_id_4_re = addr_hit[23] & reg_re & !reg_error;
1280 assign device_id_5_re = addr_hit[24] & reg_re & !reg_error;
1281 assign device_id_6_re = addr_hit[25] & reg_re & !reg_error;
1282 assign device_id_7_re = addr_hit[26] & reg_re & !reg_error;
1283 assign manuf_state_0_re = addr_hit[27] & reg_re & !reg_er-
ror;
1284 assign manuf_state_1_re = addr_hit[28] & reg_re & !reg_er-
ror;
1285 assign manuf_state_2_re = addr_hit[29] & reg_re & !reg_er-
ror;
1286 assign manuf_state_3_re = addr_hit[30] & reg_re & !reg_er-
ror;
1287 assign manuf_state_4_re = addr_hit[31] & reg_re & !reg_er-
ror;
1288 assign manuf_state_5_re = addr_hit[32] & reg_re & !reg_er-
ror;
1289 assign manuf_state_6_re = addr_hit[33] & reg_re & !reg_er-
ror;
1290 assign manuf_state_7_re = addr_hit[34] & reg_re & !reg_er-
ror;
1291
1292 // Assign write-enables to checker logic vector.
1293 always_comb begin
1294   reg_we_check[0] = alert_test_we;
1295   reg_we_check[1] = 1'b0;
1296   reg_we_check[2] = claim_transition_if_regwen_we;
1297   reg_we_check[3] = claim_transition_if_gated_we;
1298   reg_we_check[4] = 1'b0;
1299   reg_we_check[5] = transition_cmd_gated_we;
1300   reg_we_check[6] = transition_ctrl_gated_we;
1301   reg_we_check[7] = transition_token_0_gated_we;
1302   reg_we_check[8] = transition_token_1_gated_we;
1303   reg_we_check[9] = transition_token_2_gated_we;
1304   reg_we_check[10] = transition_token_3_gated_we;
1305   reg_we_check[11] = transition_target_gated_we;
1306   reg_we_check[12] = otp_vendor_test_ctrl_gated_we;
1307   reg_we_check[13] = 1'b0;
1308   reg_we_check[14] = 1'b0;
1309   reg_we_check[15] = 1'b0;
1310   reg_we_check[16] = 1'b0;
1311   reg_we_check[17] = 1'b0;
1312   reg_we_check[18] = 1'b0;
1313   reg_we_check[19] = 1'b0;

```



```

1314 reg_we_check[20] = 1'b0;
1315 reg_we_check[21] = 1'b0;
1316 reg_we_check[22] = 1'b0;
1317 reg_we_check[23] = 1'b0;
1318 reg_we_check[24] = 1'b0;
1319 reg_we_check[25] = 1'b0;
1320 reg_we_check[26] = 1'b0;
1321 reg_we_check[27] = 1'b0;
1322 reg_we_check[28] = 1'b0;
1323 reg_we_check[29] = 1'b0;
1324 reg_we_check[30] = 1'b0;
1325 reg_we_check[31] = 1'b0;
1326 reg_we_check[32] = 1'b0;
1327 reg_we_check[33] = 1'b0;
1328 reg_we_check[34] = 1'b0;
1329 end
1330
1331 // Read data return
1332 always_comb begin
1333   reg_rdata_next = '0;
1334   unique case (1'b1)
1335     addr_hit[0]: begin
1336       reg_rdata_next[0] = '0;
1337       reg_rdata_next[1] = '0;
1338       reg_rdata_next[2] = '0;
1339     end
1340
1341     addr_hit[1]: begin
1342       reg_rdata_next[0] = status_initialized_qs;
1343       reg_rdata_next[1] = status_ready_qs;
1344       reg_rdata_next[2] = status_ext_clock_switched_qs;
1345       reg_rdata_next[3] = status_transition_successful_qs;
1346       reg_rdata_next[4] = status_transition_count_error_qs;
1347       reg_rdata_next[5] = status_transition_error_qs;
1348       reg_rdata_next[6] = status_token_error_qs;
1349       reg_rdata_next[7] = status_nvm_rma_error_qs;
1350       reg_rdata_next[8] = status_otp_error_qs;
1351       reg_rdata_next[9] = status_state_error_qs;
1352       reg_rdata_next[10] = status_bus_integ_error_qs;
1353       reg_rdata_next[11] = status_otp_partition_error_qs;
1354     end
1355
1356     addr_hit[2]: begin
1357       reg_rdata_next[0] = claim_transition_if_regwen_qs;
1358     end
1359
1360     addr_hit[3]: begin
1361       reg_rdata_next[7:0] = claim_transition_if_qs;
1362     end
1363
1364     addr_hit[4]: begin
1365       reg_rdata_next[0] = transition_regwen_qs;
1366     end
1367
1368     addr_hit[5]: begin
1369       reg_rdata_next[0] = '0;
1370     end
1371
1372     addr_hit[6]: begin
1373       reg_rdata_next[0] = transition_ctrl_ext_clock_en_qs;
1374       reg_rdata_next[1] = transition_ctrl_volatile_raw_unlock_qs;
1375     end
1376
1377     addr_hit[7]: begin
1378       reg_rdata_next[31:0] = transition_token_0_qs;
1379     end
1380
1381     addr_hit[8]: begin
1382       reg_rdata_next[31:0] = transition_token_1_qs;
1383     end
1384
1385     addr_hit[9]: begin
1386       reg_rdata_next[31:0] = transition_token_2_qs;
1387     end
1388
1389     addr_hit[10]: begin
1390       reg_rdata_next[31:0] = transition_token_3_qs;
1391     end
1392
1393     addr_hit[11]: begin
1394       reg_rdata_next[29:0] = transition_target_qs;
1395     end
1396
1397     addr_hit[12]: begin
1398       reg_rdata_next[31:0] = otp_vendor_test_ctrl_qs;
1399     end
1400
1401     addr_hit[13]: begin
1402       reg_rdata_next[31:0] = otp_vendor_test_status_qs;
1403     end
1404
1405     addr_hit[14]: begin
1406       reg_rdata_next[29:0] = lc_state_qs;
1407     end
1408
1409     addr_hit[15]: begin
1410       reg_rdata_next[4:0] = lc_transition_cnt_qs;
1411     end
1412
1413     addr_hit[16]: begin
1414       reg_rdata_next[31:0] = lc_id_state_qs;
1415     end
1416
1417     addr_hit[17]: begin
1418       reg_rdata_next[15:0] = hw_revision0_product_id_qs;
1419       reg_rdata_next[31:16] = hw_revision0_silicon_creator_id_qs;
1420     end
1421
1422     addr_hit[18]: begin
1423       reg_rdata_next[7:0] = hw_revision1_revision_id_qs;
1424       reg_rdata_next[31:8] = hw_revision1_reserved_qs;
1425     end
1426
1427     addr_hit[19]: begin
1428       reg_rdata_next[31:0] = device_id_0_qs;
1429     end
1430
1431     addr_hit[20]: begin
1432       reg_rdata_next[31:0] = device_id_1_qs;
1433     end
1434
1435     addr_hit[21]: begin
1436       reg_rdata_next[31:0] = device_id_2_qs;
1437     end
1438
1439     addr_hit[22]: begin
1440       reg_rdata_next[31:0] = device_id_3_qs;
1441     end
1442
1443     addr_hit[23]: begin
1444       reg_rdata_next[31:0] = device_id_4_qs;
1445     end
1446
1447     addr_hit[24]: begin
1448       reg_rdata_next[31:0] = device_id_5_qs;
1449     end
1450
1451     addr_hit[25]: begin
1452       reg_rdata_next[31:0] = device_id_6_qs;
1453     end
1454
1455     addr_hit[26]: begin
1456       reg_rdata_next[31:0] = device_id_7_qs;
1457     end
1458
1459     addr_hit[27]: begin
1460       reg_rdata_next[31:0] = manuf_state_0_qs;
1461     end
1462
1463     addr_hit[28]: begin
1464       reg_rdata_next[31:0] = manuf_state_1_qs;
1465     end
1466
1467     addr_hit[29]: begin
1468       reg_rdata_next[31:0] = manuf_state_2_qs;
1469     end
1470
1471     addr_hit[30]: begin
1472       reg_rdata_next[31:0] = manuf_state_3_qs;
1473     end
1474
1475     addr_hit[31]: begin
1476       reg_rdata_next[31:0] = manuf_state_4_qs;
1477     end
1478
1479     addr_hit[32]: begin
1480       reg_rdata_next[31:0] = manuf_state_5_qs;
1481     end
1482
1483     addr_hit[33]: begin
1484       reg_rdata_next[31:0] = manuf_state_6_qs;
1485     end
1486
1487     addr_hit[34]: begin
1488       reg_rdata_next[31:0] = manuf_state_7_qs;
1489     end
1490
1491     default: begin
1492       reg_rdata_next = '1;
1493     end
1494   endcase
1495 end
1496
1497 // shadow busy
1498 logic shadow_busy;
1499 assign shadow_busy = 1'b0;
1500
1501 // register busy
1502 assign reg_busy = shadow_busy;
1503
1504 // Unused signal tieoff
1505
1506 // wdata / byte enable are not always fully used
1507 // add a blanket unused statement to handle lint waivers
1508 logic unused_wdata;
1509 logic unused_be;
1510 assign unused_wdata = ^reg_wdata;
1511 assign unused_be = ^reg_be;
1512
1513 // Assertions for Register Interface

```

```

1514 `ASSERT_PULSE(wePulse, reg_we, clk_i, !rst_ni)
1515 `ASSERT_PULSE(rePulse, reg_re, clk_i, !rst_ni)
1516
1517 `ASSERT(reAfterRv, $rose(reg_re || reg_we) | =>
t1_o_pre_d_valid, clk_i, !rst_ni)
1518
1519 `ASSERT(en2addrHit, (reg_we || reg_re) | => $one-
hot0(addr_hit), clk_i, !rst_ni)
1520
1521 // this is formulated as an assumption such that the FPV
testbenches do disprove this
1522 // property by mistake
1523 // ASSUME(reqParity, t1_reg_h2d.a_valid | =>
t1_reg_h2d.a_user.chk_en == t1u_pkg::CheckDis)
1524
1525 endmodule

```

8. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/ lc_ctrl_signal_decode.sv (450 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Life cycle signal decoder and sender module.
6
7 `include "prim_assert.sv"
8
9 module lc_ctrl_signal_decode
10 import lc_ctrl_pkg::*;
11 import lc_ctrl_state_pkg::*;
12 #(
13 // Random netlist constants
14 // SCRAP, RAW, TEST_LOCKED*, INVALID
15 parameter lc_keymgr_div_t RndCnstLcKeymgrDivInvalid =
LcKeymgrDivWidth'(0),
16 // TEST_UNLOCKED*
17 parameter lc_keymgr_div_t RndCnstLcKeymgrDivTestUnlocked =
LcKeymgrDivWidth'(1),
18 // DEV
19 parameter lc_keymgr_div_t RndCnstLcKeymgrDivDev =
LcKeymgrDivWidth'(2),
20 // PROD, PROD_END
21 parameter lc_keymgr_div_t RndCnstLcKeymgrDivProduction =
LcKeymgrDivWidth'(3),
22 // RMA
23 parameter lc_keymgr_div_t RndCnstLcKeymgrDivRma =
LcKeymgrDivWidth'(4)
24 ) (
25 input clk_i,
26 input rst_ni,
27 // Life cycle state vector.
28 input logic lc_state_valid_i,
29 input lc_state_e lc_state_i,
30 input fsm_state_e fsm_state_i,
31 input lc_tx_t secrets_valid_i,
32 // Local life cycle signal
33 output lc_tx_t lc_raw_test_rma_o,
34 // Life cycle broadcast outputs.
35 output lc_tx_t lc_init_done_o,
36 output lc_tx_t lc_dft_en_o,
37 output lc_tx_t lc_nvm_debug_en_o,
38 output lc_tx_t lc_hw_debug_clr_o,
39 output lc_tx_t lc_hw_debug_en_o,
40 output lc_tx_t lc_cpu_en_o,
41 output lc_tx_t lc_creator_seed_sw_rw_en_o,
42 output lc_tx_t lc_owner_seed_sw_rw_en_o,
43 output lc_tx_t lc_iso_part_sw_rd_en_o,
44 output lc_tx_t lc_iso_part_sw_wr_en_o,
45 output lc_tx_t lc_seed_hw_rd_en_o,
46 output lc_tx_t lc_rma_state_o,
47 output lc_tx_t lc_keymgr_en_o,
48 output lc_tx_t lc_escalate_en_o,
49 // State group diversification value for keymgr
50 output lc_keymgr_div_t lc_keymgr_div_o
51 );
52
53 //////////////////////////////////////////////////
54 // Signal Decoder Logic //
55 //////////////////////////////////////////////////
56
57 lc_tx_t lc_init_done, lc_raw_test_rma;
58 lc_tx_t lc_dft_en, lc_nvm_debug_en, lc_hw_debug_clr,
lc_hw_debug_en, lc_cpu_en, lc_keymgr_en,
59 lc_escalate_en;
60 lc_tx_t lc_creator_seed_sw_rw_en, lc_owner_seed_sw_rw_en,
lc_iso_part_sw_rd_en;
61 lc_tx_t lc_iso_part_sw_wr_en, lc_seed_hw_rd_en;
62 lc_tx_t lc_rma_state;
63 lc_keymgr_div_t lc_keymgr_div_d, lc_keymgr_div_q;
64
65 always_comb begin : p_lc_signal_decode
66 // Life cycle control signal defaults
67 lc_init_done = Off;
68 lc_raw_test_rma = Off;
69 lc_dft_en = Off;
70 lc_nvm_debug_en = Off;
71 lc_hw_debug_clr = Off;

```

```

72 lc_hw_debug_en = Off;
73 lc_cpu_en = Off;
74 lc_creator_seed_sw_rw_en = Off;
75 lc_owner_seed_sw_rw_en = Off;
76 lc_iso_part_sw_rd_en = Off;
77 lc_iso_part_sw_wr_en = Off;
78 lc_seed_hw_rd_en = Off;
79 lc_rma_state = Off;
80 lc_keymgr_en = Off;
81 // This ensures that once escalation has been triggered,
it cannot go back to Off.
82 lc_escalate_en = lc_tx_or_hi(Off, lc_escal-
ate_en_o);
83 // Set to invalid diversification value by default.
84 lc_keymgr_div_d = RndCnstLcKeymgrDivInvalid;
85
86 unique case (fsm_state_i)
87 //////////////////////////////////////////////////
88 // Don't broadcast anything in this state.
89 ResetSt: ;
90 //////////////////////////////////////////////////
91 // Broadcasting of most signals is only enabled during
the following life cycle states.
92 IdleSt,
93 ClkMuxSt,
94 CntIncrSt,
95 CntProgSt,
96 TransCheckSt,
97 NvmRmaSt,
98 TokenHashSt,
99 TokenCheck0St,
100 TokenCheck1St,
101 TransProgSt: begin
102 if (lc_state_valid_i) begin
103 // After this, all life cycle controller broadcast
signals are valid and based on the
104 // OTP LC state.
105 lc_init_done = On;
106 unique case (lc_state_i)
107 //////////////////////////////////////////////////
108 // Only enable life cycle TAP register for OTP
test mechanisms.
109 LcStRaw,
110 LcStTestLocked0,
111 LcStTestLocked1,
112 LcStTestLocked2,
113 LcStTestLocked3,
114 LcStTestLocked4,
115 LcStTestLocked5,
116 LcStTestLocked6: begin
117 lc_raw_test_rma = On;
118 end
119 //////////////////////////////////////////////////
120 // Enable DFT and debug functionality, including
the CPU in the
121 // test unlocked states.
122 LcStTestUnlocked0,
123 LcStTestUnlocked1,
124 LcStTestUnlocked2,
125 LcStTestUnlocked3,
126 LcStTestUnlocked4,
127 LcStTestUnlocked5,
128 LcStTestUnlocked6: begin
129 lc_raw_test_rma = On;
130 lc_dft_en = On;
131 lc_nvm_debug_en = On;
132 lc_hw_debug_en = On;
133 lc_cpu_en = On;
134 lc_iso_part_sw_wr_en = On;
135 lc_keymgr_div_d = RndCnstLcKeymgrDivTestUn-
locked;
136 end
137 //////////////////////////////////////////////////
138 // This is the last TEST_UNLOCKED state. The same
feature set is enabled
139 // as in the other TEST_UNLOCKED states above, ex-
cept for NVM debug en,
140 // which is disabled in this state.
141 LcStTestUnlocked7: begin
142 lc_raw_test_rma = On;
143 lc_dft_en = On;
144 lc_hw_debug_en = On;
145 lc_cpu_en = On;
146 lc_iso_part_sw_wr_en = On;
147 lc_keymgr_div_d = RndCnstLcKeymgrDivTestUn-
locked;
148 end
149 //////////////////////////////////////////////////
150 // Enable production functions
151 LcStProd,
152 LcStProdEnd: begin
153 lc_cpu_en = On;
154 lc_keymgr_en = On;
155 lc_owner_seed_sw_rw_en = On;
156 lc_iso_part_sw_wr_en = On;

```

```

157         lc_iso_part_sw_rd_en = On;
158         lc_keymgr_div_d      = RndCnstLcKeymgrDivPro-
duction;

159         // Only allow provisioning if the device has not
yet been personalized.
160         // If secrets_valid_i is set to ON, we output
OFF.
161         // Note that we can convert ON to OFF with a
bitwise inversion due to the encoding.
162         lc_creator_seed_sw_rw_en =
lc_tx_t'(~secrets_valid_i);

163         // Only allow hardware to consume the seeds once
personalized.
164         // If secrets_valid_i is set to ON, we output
ON.
165         lc_seed_hw_rd_en = secrets_valid_i;
166         end
167         //////////////////////////////////
168         // Similar functions as PROD, with the following
differences:
169         // - hardware debug functionality (CPU TAP) is en-
abled,
170         // - access to the isolated NVM partition is dis-
abled.
171         LcStDev: begin
172             lc_hw_debug_en      = On;
173             lc_cpu_en           = On;
174             lc_keymgr_en        = On;
175             lc_owner_seed_sw_rw_en = On;
176             lc_iso_part_sw_wr_en = On;
177             lc_keymgr_div_d      = RndCnstLcKeymgrDivDev;

178         // Only allow provisioning if the device has not
yet been personalized.
179         // If secrets_valid_i is set to ON, we output
OFF.
180         // Note that we can convert ON to OFF with a
bitwise inversion due to the encoding.
181         lc_creator_seed_sw_rw_en =
lc_tx_t'(~secrets_valid_i);

182         // Only allow hardware to consume the seeds once
personalized.
183         // If secrets_valid_i is set to ON, we output
ON.
184         lc_seed_hw_rd_en = secrets_valid_i;
185         end
186         //////////////////////////////////
187         // Enable all test and production functions.
188         LcStRma: begin
189             lc_raw_test_rma      = On;
190             lc_dft_en            = On;
191             lc_nvm_debug_en      = On;
192             lc_hw_debug_en      = On;
193             lc_cpu_en           = On;
194             lc_keymgr_en        = On;
195             lc_creator_seed_sw_rw_en = On;
196             lc_owner_seed_sw_rw_en = On;
197             lc_iso_part_sw_wr_en = On;
198             lc_iso_part_sw_rd_en = On;
199             lc_seed_hw_rd_en     = On;
200             lc_rma_state         = On;
201             lc_keymgr_div_d      = RndCnstLcKeymgrDi-
vRma;
202         end
203         //////////////////////////////////
204         // Invalid or scrapped life cycle state, make sure
the escalation
205         // signal is also asserted in this case.
206         default: begin
207             lc_escalate_en = On;
208             lc_hw_debug_clr = On;
209         end
210         endcase // lc_state_i
211     end else begin
212         lc_escalate_en = On;
213         lc_hw_debug_clr = On;
214     end
215 end
216 //////////////////////////////////
217 // Post-transition state. Behaves similarly to the vir-
tual scrap
218 // states below, with the exception that escalate_en is
NOT asserted,
219 // since that could trigger unwanted alerts / escala-
tions and system resets.
220 PostTransSt: begin
221     // At this point, all life cycle controller broadcast
signals are valid and based on the
222     // OTP LC state
223     lc_init_done = On;
224     // Explicitly clear any flops holding the lc_hw_de-
bug_en signal. Those flops would retain
225     // their state if just the lc_hw_debug_en signal is

```

```

switched off, in order to let debug
226     // connections survive a reset of lc_ctrl.
227     lc_hw_debug_clr = On;
228     end
229     //////////////////////////////////
230     // Virtual scrap states, make sure the escalation signal
is
231     // also asserted in this case.
232     ScrapSt,
233     EscalateSt,
234     InvalidSt: begin
235         // At this point, all life cycle controller broadcast
signals are valid and based on the
236         // OTP LC state.
237         lc_init_done = On;
238         lc_escalate_en = On;
239         lc_hw_debug_clr = On;
240     end
241     default: begin
242         lc_escalate_en = On;
243         lc_hw_debug_clr = On;
244     end
245 endcase // fsm_state_i
246 end
247
248 //////////////////////////////////
249 // Control signal output flops //
250 //////////////////////////////////
251
252 prim_lc_sender u_prim_lc_sender_init_done (
253     .clk_i,
254     .rst_ni,
255     .lc_en_i(lc_init_done),
256     .lc_en_o(lc_init_done_o)
257 );
258 prim_lc_sender u_prim_lc_sender_raw_test_rma (
259     .clk_i,
260     .rst_ni,
261     .lc_en_i(lc_raw_test_rma),
262     .lc_en_o(lc_raw_test_rma_o)
263 );
264 prim_lc_sender u_prim_lc_sender_dft_en (
265     .clk_i,
266     .rst_ni,
267     .lc_en_i(lc_dft_en),
268     .lc_en_o(lc_dft_en_o)
269 );
270 prim_lc_sender u_prim_lc_sender_nvm_debug_en (
271     .clk_i,
272     .rst_ni,
273     .lc_en_i(lc_nvm_debug_en),
274     .lc_en_o(lc_nvm_debug_en_o)
275 );
276 prim_lc_sender u_prim_lc_sender_hw_debug_clr (
277     .clk_i,
278     .rst_ni,
279     .lc_en_i(lc_hw_debug_clr),
280     .lc_en_o(lc_hw_debug_clr_o)
281 );
282 prim_lc_sender u_prim_lc_sender_hw_debug_en (
283     .clk_i,
284     .rst_ni,
285     .lc_en_i(lc_hw_debug_en),
286     .lc_en_o(lc_hw_debug_en_o)
287 );
288 prim_lc_sender u_prim_lc_sender_cpu_en (
289     .clk_i,
290     .rst_ni,
291     .lc_en_i(lc_cpu_en),
292     .lc_en_o(lc_cpu_en_o)
293 );
294 prim_lc_sender u_prim_lc_sender_creator_seed_sw_rw_en (
295     .clk_i,
296     .rst_ni,
297     .lc_en_i(lc_creator_seed_sw_rw_en),
298     .lc_en_o(lc_creator_seed_sw_rw_en_o)
299 );
300 prim_lc_sender u_prim_lc_sender_owner_seed_sw_rw_en (
301     .clk_i,
302     .rst_ni,
303     .lc_en_i(lc_owner_seed_sw_rw_en),
304     .lc_en_o(lc_owner_seed_sw_rw_en_o)
305 );
306 prim_lc_sender u_prim_lc_sender_iso_part_sw_rd_en (
307     .clk_i,
308     .rst_ni,
309     .lc_en_i(lc_iso_part_sw_rd_en),
310     .lc_en_o(lc_iso_part_sw_rd_en_o)
311 );
312 prim_lc_sender u_prim_lc_sender_iso_part_sw_wr_en (
313     .clk_i,
314     .rst_ni,
315     .lc_en_i(lc_iso_part_sw_wr_en),
316     .lc_en_o(lc_iso_part_sw_wr_en_o)
317 );
318 prim_lc_sender u_prim_lc_sender_seed_hw_rd_en (
319     .clk_i,
320     .rst_ni,
321     .lc_en_i(lc_seed_hw_rd_en),

```



```

322     .lc_en_o(lc_seed_hw_rd_en_o)
323 );
324 prim_lc_sender u_prim_lc_sender_rma_state (
325     .clk_i,
326     .rst_ni,
327     .lc_en_i(lc_rma_state),
328     .lc_en_o(lc_rma_state_o)
329 );
330 prim_lc_sender u_prim_lc_sender_keymgr_en (
331     .clk_i,
332     .rst_ni,
333     .lc_en_i(lc_keymgr_en),
334     .lc_en_o(lc_keymgr_en_o)
335 );
336 prim_lc_sender u_prim_lc_sender_escalate_en (
337     .clk_i,
338     .rst_ni,
339     .lc_en_i(lc_escalate_en),
340     .lc_en_o(lc_escalate_en_o)
341 );
342
343 assign lc_keymgr_div_o = lc_keymgr_div_q;
344
345 prim_flop #(
346     .Width(LcKeymgrDivWidth),
347     .ResetValue(RndCnstLcKeymgrDivInvalid)
348 ) u_prim_flop_keymgr_div (
349     .clk_i ( clk_i ),
350     .rst_ni ( rst_ni ),
351     .d_i ( lc_keymgr_div_d ),
352     .q_o ( lc_keymgr_div_q )
353 );
354
355 //////////////////////////////////////////////////
356 // Assertions //
357 //////////////////////////////////////////////////
358
359 // Need to make sure that the random netlist constants are
360 // all unique.
361 `ASSERT_INIT(LcKeymgrDivUnique0_A,
362     !(RndCnstLcKeymgrDivInvalid inside {RndCnstLcKeymgrDivTestUnlocked,
363     RndCnstLcKeymgrDivDev,
364     RndCnstLcKeymgrDivRma,
365     RndCnstLcKeymgrDivProduction}))
366 `ASSERT_INIT(LcKeymgrDivUnique1_A,
367     !(RndCnstLcKeymgrDivTestUnlocked inside {RndCnstLcKeymgrDivInvalid,
368     RndCnstLcKeymgrDivDev,
369     RndCnstLcKeymgrDivRma,
370     RndCnstLcKeymgrDivProduction}))
371 `ASSERT_INIT(LcKeymgrDivUnique2_A,
372     !(RndCnstLcKeymgrDivDev inside {RndCnstLcKeymgrDivInvalid,
373     RndCnstLcKeymgrDivRma,
374     RndCnstLcKeymgrDivProduction}))
375 `ASSERT_INIT(LcKeymgrDivUnique3_A,
376     !(RndCnstLcKeymgrDivRma inside {RndCnstLcKeymgrDivInvalid,
377     RndCnstLcKeymgrDivDev,
378     RndCnstLcKeymgrDivProduction}))
379
380 `ASSERT(SignalsAreOffWhenNotEnabled_A,
381     !lc_state_valid_i
382     |>
383     lc_tx_test_false_strict(lc_raw_test_rma_o) &&
384     lc_tx_test_false_strict(lc_dft_en_o) &&
385     lc_tx_test_false_strict(lc_nvm_debug_en_o) &&
386     lc_tx_test_false_strict(lc_hw_debug_en_o) &&
387     lc_tx_test_false_strict(lc_cpu_en_o) &&
388     lc_tx_test_false_strict(lc_creator_seed_sw_rw_en_o) &&
389     lc_tx_test_false_strict(lc_owner_seed_sw_rw_en_o) &&
390     lc_tx_test_false_strict(lc_iso_part_sw_rd_en_o) &&
391     lc_tx_test_false_strict(lc_iso_part_sw_wr_en_o) &&
392     lc_tx_test_false_strict(lc_seed_hw_rd_en_o) &&
393     lc_tx_test_false_strict(lc_rma_state_o) &&
394     lc_tx_test_false_strict(lc_keymgr_en_o) &&
395     lc_tx_test_false_strict(lc_dft_en_o) &&
396     lc_keymgr_div_o == RndCnstLcKeymgrDivInvalid)
397
398
399
400 `ASSERT(FsmInScrap_A,
401     !(fsm_state_i inside {ResetSt,
402     TransProgSt,
403     IdleSt,
404     ClkMuxSt,
405     CntIncrSt,
406     CntProgSt,
407     TransCheckSt,

```

```

408     NvmRmaSt,
409     TokenHashSt,
410     TokenCheck0St,
411     TokenCheck1St,
412     PostTransSt}))
413 |>
414     lc_tx_test_true_strict(lc_escalate_en_o))
415
416 `ASSERT(StateInScrap_A,
417     lc_state_valid_i &&
418     fsm_state_i inside {IdleSt,
419     ClkMuxSt,
420     CntIncrSt,
421     CntProgSt,
422     TransCheckSt,
423     NvmRmaSt,
424     TokenHashSt,
425     TokenCheck0St,
426     TokenCheck1St} &&
427     !(lc_state_i inside {LcStRaw,
428     LcStTestUnlocked0,
429     LcStTestUnlocked1,
430     LcStTestUnlocked2,
431     LcStTestUnlocked3,
432     LcStTestUnlocked4,
433     LcStTestUnlocked5,
434     LcStTestUnlocked6,
435     LcStTestUnlocked7,
436     LcStTestLocked0,
437     LcStTestLocked1,
438     LcStTestLocked2,
439     LcStTestLocked3,
440     LcStTestLocked4,
441     LcStTestLocked5,
442     LcStTestLocked6,
443     LcStDev,
444     LcStProd,
445     LcStProdEnd,
446     LcStRma}))
447 |>
448     lc_tx_test_true_strict(lc_escalate_en_o))
449
450 endmodule : lc_ctrl_signal_decode

```

9. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_state_decode.sv (167 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 // Life cycle state decoder. This is a purely combinational
6 // module.
7 module lc_ctrl_state_decode
8     import lc_ctrl_pkg::*;
9     import lc_ctrl_state_pkg::*;
10 (
11     // Life cycle state vector.
12     input logic lc_state_valid_i,
13     input lc_state_e lc_state_i,
14     input lc_cnt_e lc_cnt_i,
15     input lc_tx_t secrets_valid_i,
16     // Main FSM state.
17     input fsm_state_e fsm_state_i,
18     // Decoded state vector.
19     output ext_dec_lc_state_t dec_lc_state_o,
20     output dec_lc_id_state_e dec_lc_id_state_o,
21     output dec_lc_cnt_t dec_lc_cnt_o,
22     output logic [5:0] state_invalid_error_o
23 );
24
25 //////////////////////////////////////////////////
26 // Signal Decoder Logic //
27 //////////////////////////////////////////////////
28
29 // SEC_CM: STATE.CONFIG.SPARSE
30 // The decoded life cycle state uses a redundant representation that is used internally
31 // and in the CSR node.
32 ext_dec_lc_state_t dec_lc_state;
33 logic [$bits(ext_dec_lc_state_t)-1:0] dec_lc_state_buf;
34 prim_sec_anchor_buf #(
35     .Width($bits(ext_dec_lc_state_t))
36 ) u_prim_sec_anchor_buf (
37     .in_i(dec_lc_state),
38     .out_o(dec_lc_state_buf)
39 );
40
41 // This cast is needed so that VCS does not throw warnings.
42 for (genvar k = 0; k < DecLcStateNumRep; k++) begin :
43     gen_enum_casts
44         assign dec_lc_state_o[k] =
45         dec_lc_state_e'(dec_lc_state_buf[k*DecLcStateWidth +:
46         DecLcStateWidth]);
47     end

```

```

46 // The decoder logic below decodes the life cycle state vec-
tor and counter
47 // into a format that can be exposed in the CSRs. If the
state is invalid,
48 // this will be flagged as well.
49
50 always_comb begin : p_lc_state_decode
51 // Decoded state defaults
52 dec_lc_state = {DecLcStateNumRep{DecLcStInvalid}};
53 dec_lc_cnt_o = {DecLcCountWidth{1'b1}};
54 dec_lc_id_state_o = DecLcIdInvalid;
55 state_invalid_error_o = '0;
56
57 unique case (fsm_state_i)
58 // Don't decode anything in ResetSt
59 ResetSt :
60 // These are temporary, terminal states that are not en-
coded
61 // in the persistent LC state vector from OTP, hence we
decode them first.
62 EscalateSt: dec_lc_state = {DecLcStateNum-
Rep{DecLcStEscalate}};
63 PostTransSt: dec_lc_state = {DecLcStateNum-
Rep{DecLcStPostTrans}};
64 InvalidSt: dec_lc_state = {DecLcStateNum-
Rep{DecLcStInvalid}};
65 ScrapSt: dec_lc_state = {DecLcStateNum-
Rep{DecLcStScrap}};
66 // Otherwise check and decode the life cycle state con-
tinuously.
67 default: begin
68
69 // Note that we require that the valid signal from OTP is
in ResetSt.
70 // This will trigger an invalid_state_error when the
OTP partition
71 // is corrupt and moved into an error state, where the
valid bit is
72 // deasserted.
73 state_invalid_error_o[0] = ~lc_state_valid_i;
74
75 unique case (lc_state_i)
76 LcStRaw: dec_lc_state = {DecLcStateNum-
Rep{DecLcStRaw}};
77 LcStTestUnlocked0: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestUnlocked0}};
78 LcStTestLocked0: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestLocked0}};
79 LcStTestUnlocked1: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestUnlocked1}};
80 LcStTestLocked1: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestLocked1}};
81 LcStTestUnlocked2: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestUnlocked2}};
82 LcStTestLocked2: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestLocked2}};
83 LcStTestUnlocked3: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestUnlocked3}};
84 LcStTestLocked3: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestLocked3}};
85 LcStTestUnlocked4: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestUnlocked4}};
86 LcStTestLocked4: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestLocked4}};
87 LcStTestUnlocked5: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestUnlocked5}};
88 LcStTestLocked5: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestLocked5}};
89 LcStTestUnlocked6: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestUnlocked6}};
90 LcStTestLocked6: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestLocked6}};
91 LcStTestUnlocked7: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestUnlocked7}};
92 LcStDev: dec_lc_state = {DecLcStateNum-
Rep{DecLcStDev}};
93 LcStProd: dec_lc_state = {DecLcStateNum-

```

```

Rep{DecLcStProd}};
94 LcStProdEnd: dec_lc_state = {DecLcStateNum-
Rep{DecLcStProdEnd}};
95 LcStRma: dec_lc_state = {DecLcStateNum-
Rep{DecLcStRma}};
96 LcStScrap: dec_lc_state = {DecLcStateNum-
Rep{DecLcStScrap}};
97 // SEC_CM: MANUF.STATE.BKGN_CHK
98 default: state_invalid_error_o[1] = 1'b1;
99 endcase // lc_state_i
100
101 unique case (lc_cnt_i)
102 LcCnt0: dec_lc_cnt_o = 5'd0;
103 LcCnt1: dec_lc_cnt_o = 5'd1;
104 LcCnt2: dec_lc_cnt_o = 5'd2;
105 LcCnt3: dec_lc_cnt_o = 5'd3;
106 LcCnt4: dec_lc_cnt_o = 5'd4;
107 LcCnt5: dec_lc_cnt_o = 5'd5;
108 LcCnt6: dec_lc_cnt_o = 5'd6;
109 LcCnt7: dec_lc_cnt_o = 5'd7;
110 LcCnt8: dec_lc_cnt_o = 5'd8;
111 LcCnt9: dec_lc_cnt_o = 5'd9;
112 LcCnt10: dec_lc_cnt_o = 5'd10;
113 LcCnt11: dec_lc_cnt_o = 5'd11;
114 LcCnt12: dec_lc_cnt_o = 5'd12;
115 LcCnt13: dec_lc_cnt_o = 5'd13;
116 LcCnt14: dec_lc_cnt_o = 5'd14;
117 LcCnt15: dec_lc_cnt_o = 5'd15;
118 LcCnt16: dec_lc_cnt_o = 5'd16;
119 LcCnt17: dec_lc_cnt_o = 5'd17;
120 LcCnt18: dec_lc_cnt_o = 5'd18;
121 LcCnt19: dec_lc_cnt_o = 5'd19;
122 LcCnt20: dec_lc_cnt_o = 5'd20;
123 LcCnt21: dec_lc_cnt_o = 5'd21;
124 LcCnt22: dec_lc_cnt_o = 5'd22;
125 LcCnt23: dec_lc_cnt_o = 5'd23;
126 LcCnt24: dec_lc_cnt_o = 5'd24;
127 // SEC_CM: TRANSITION.CTR.BKGN_CHK
128 default: state_invalid_error_o[2] = 1'b1;
129 endcase // lc_cnt_i
130
131 // SEC_CM: MANUF.STATE.BKGN_CHK
132 unique case (secrets_valid_i)
133 // If the secrets have not been provisioned, the ID
state is "blank".
134 Off: dec_lc_id_state_o = DecLcIdBlank;
135 // If the secrets have been provisioned, the ID
state is "personalized".
136 On: dec_lc_id_state_o = DecLcIdPersonalized;
137 default: state_invalid_error_o[3] = 1'b1;
138 endcase // secrets_valid_i
139
140 // Require that any non-raw state has a valid, nonzero
transition count.
141 // SEC_CM: TRANSITION.CTR.BKGN_CHK
142 if (lc_state_i != LcStRaw && lc_cnt_i == LcCnt0) begin
143 state_invalid_error_o[4] = 1'b1;
144 end
145
146 // We can't have a personalized device that is
// still in RAW or any of the test states.
147 // SEC_CM: MANUF.STATE.BKGN_CHK
148 if (lc_tx_test_true_strict(secrets_valid_i) &&
!(lc_state_i inside {LcStDev,
149 LcStProd,
150 LcStProdEnd,
151 LcStRma,
152 LcStScrap})) begin
153 state_invalid_error_o[5] = 1'b1;
154 end
155 end
156 endcase // lc_id_state_i
157 end
158
159 // Assertions //
160 //
161 endmodule : lc_ctrl_state_decode

```

플래시 컨트롤러

What this is. The flash controller's interface logic.

Why it exists. Because embedded flash holds the code that runs before anything else. Its controller therefore participates in secure boot: it must enforce read/write/erase permissions per region and cooperate with the life-cycle state.

Undergraduate review · EECS concepts. Non-volatile memory programming; wear levelling; region-based access control; integrity checking of stored code.

Graduate · Practice note. Flash operations are slow and blocking relative to the CPU, so the controller's interface must allow the core to make progress. The resulting concurrency — erase in progress while reads continue from another bank — is where the verification effort concentrates.

1 files • 29 lines

API — flash_ctrl_pkg.sv

Table 1230. lowRISC_opentitan/hw/ip/flash_ctrl/rtl/flash_ctrl_pkg.sv — module / interface / package (1개)

이름	종류
flash_ctrl_pkg	package

Table 1231. lowRISC_opentitan/hw/ip/flash_ctrl/rtl/flash_ctrl_pkg.sv — parameter (6개)

파라미터	형/폭	기본값
NumSeeds	int	2
CreatorSeedIdx	bit [0:0]	0
OwnerSeedIdx	bit [0:0]	1
SeedWidth	int	256
KeyWidth	int	128
KEYMGR_FLASH_DEFAULT	keymgr_flash_t	'{

Table 1232. lowRISC_opentitan/hw/ip/flash_ctrl/rtl/flash_ctrl_pkg.sv — typedef / using (1개)

새 이름	원래 형
flash_key_t	logic [KeyWidth-1:0]

1. lowRISC_opentitan/hw/ip/flash_ctrl/rtl/flash_ctrl_pkg.sv (29 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Flash Controller generic package.
6 // This has no top-specific items so it is okay to use by gen-
  eric IPs.
7
8 package flash_ctrl_pkg;
9 // Parameters for keymgr.
```

```

10 parameter int NumSeeds = 2;
11 parameter bit [0:0] CreatorSeedIdx = 0;
12 parameter bit [0:0] OwnerSeedIdx = 1;
13 parameter int SeedWidth = 256;
14 parameter int KeyWidth = 128;
15 typedef logic [KeyWidth-1:0] flash_key_t;
16
17 // flash_ctrl to keymgr
18 typedef struct packed {
19     logic [NumSeeds-1:0][SeedWidth-1:0] seeds;
20 } keymgr_flash_t;
21
22 parameter keymgr_flash_t KEYMGR_FLASH_DEFAULT = '{
23     seeds: '{
24         256'h9152e32c9380a4bcc3e0-
25         ab263581e6b0e8825186e1e445631646e8bef8c45d47,
26         256'hfa365df52da48cd752fb3a026a8e608f0098cfe5-
```

fa9810494829d0cd9479eb78

26

}

27

};

28

29 endpackage : flash_ctrl_pkg

OpenTitan 기본 셀 라이브러리

What this is. Synchronisers, FIFOs, counters, LFSRs, arbiters, parity and ECC helpers, and security primitives such as duplicated registers and complementary signal pairs.

Why it exists. To get the subtle things right once. A synchroniser written independently in every block will be wrong in at least one of them; written once and verified once, it is correct everywhere.

Undergraduate review · EECS concepts. Metastability and MTBF; Gray coding; LFSRs and primitive polynomials; arbitration fairness.

Graduate · Practice note. This library is the first thing a one-person IP house should build. A block cannot be sold unless the primitives inside it are already trusted. OpenTitan's `prim_` modules are Apache-2.0 licensed and are a sound starting point.

28 files • 5,123 lines

API — `prim_alert_pkg.sv`

Table 1233. `lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_pkg.sv` — module / interface / package (1개)

이름	종류
<code>prim_alert_pkg</code>	package

Table 1234. `lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_pkg.sv` — parameter (2개)

파라미터	형/폭	기본값
<code>ALERT_TX_DEFAULT</code>	<code>alert_tx_t</code>	<code>'{alert_p: 1'b0</code>
<code>ALERT_RX_DEFAULT</code>	<code>alert_rx_t</code>	<code>'{ping_p: 1'b0</code>

API — `prim_alert_receiver.sv`

Table 1235. `lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_receiver.sv` — module / interface / package (1개)

이름	종류
<code>prim_alert_receiver</code>	module

Table 1236. `lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_receiver.sv` — parameter (2개)

파라미터	형/폭	기본값
<code>AsyncOn</code>	bit	<code>1'b0</code>
<code>SkewCycles</code>	int unsigned	1

Table 1237. `lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_receiver.sv` — 포트 (9개)

포트	방향	형/폭
<code>clk_i</code>	input	
<code>rst_ni</code>	input	

포트	방향	형/폭
init_trig_i	input	mubi4_t
ping_req_i	input	
ping_ok_o	output	logic
integ_fail_o	output	logic
alert_o	output	logic
alert_rx_o	output	alert_rx_t
alert_tx_i	input	alert_tx_t

Table 1238. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_receiver.sv — typedef / using (1개)

새 이름	원래 형
state_e	enum logic [2:0] {Idle, HsAckWait, Pause0, Pause1, InitReq, InitAckWait}

API — prim_alert_sender.sv

Table 1239. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_sender.sv — module / interface / package (1개)

이름	종류
prim_alert_sender	module

Table 1240. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_sender.sv — parameter (3개)

파라미터	형/폭	기본값
Async0n	bit	1'b1
SkewCycles	int unsigned	1
IsFatal	bit	1'b0

Table 1241. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_sender.sv — 포트 (8개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
alert_test_i	input	
alert_req_i	input	
alert_ack_o	output	logic
alert_state_o	output	logic
alert_rx_i	input	alert_rx_t
alert_tx_o	output	alert_tx_t

API — prim_alert_to_diff.sv

Table 1242. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_to_diff.sv — module / interface / package (1개)

이름	종류
prim_alert_to_diff	module

Table 1243. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_to_diff.sv — parameter (2개)

파라미터	형/폭	기본값
AsyncOn	bit	1'b0
SkewCycles	int unsigned	1

Table 1244. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_to_diff.sv — 포트 (6개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
alert_rx_o	output	prim_alert_pkg::alert_rx_t
alert_tx_i	input	prim_alert_pkg::alert_tx_t
diff_po	output	logic
diff_no	output	logic

API — prim_arbiter_fixed.sv**Table 1245. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_fixed.sv — module / interface / package (1개)**

이름	종류
prim_arbiter_fixed	module

Table 1246. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_fixed.sv — parameter (3개)

파라미터	형/폭	기본값
N	int	8
DW	int	32
EnDataPort	bit	1

Table 1247. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_fixed.sv — localparam (6개)

상수	형/폭	값
IdxW	int	$\$clog2(N)$
Base0	int	$(2^{**level})-1$
Base1	int	$(2^{**level+1})-1$
Pa	int	Base0 + offset
C0	int	Base1 + 2*offset
C1	int	Base1 + 2*offset + 1

Table 1248. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_fixed.sv — 포트 (8개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
req_i	input	[N-1:0]
gnt_o	output	logic [N-1:0]
idx_o	output	logic [IdxW-1:0]
valid_o	output	logic
data_o	output	logic [DW-1:0]
ready_i	input	

API — prim_arbiter_ppc.sv

Table 1249. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_ppc.sv — module / interface / package (1개)

이름	종류
prim_arbiter_ppc	module

Table 1250. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_ppc.sv — parameter (3개)

파라미터	형/폭	기본값
N	int unsigned	8
DW	int unsigned	32
EnDataPort	bit	1

Table 1251. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_ppc.sv — localparam (1개)

상수	형/폭	값
IdxW	int	prim_util_pkg::vbits(N)

Table 1252. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_ppc.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
req_chk_i	input	
req_i	input	[N-1:0]
gnt_o	output	logic [N-1:0]
idx_o	output	logic [IdxW-1:0]
valid_o	output	logic
data_o	output	logic [DW-1:0]
ready_i	input	

API — prim_arbiter_tree.sv**Table 1253. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree.sv — module / interface / package (1개)**

이름	종류
prim_arbiter_tree	module

Table 1254. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree.sv — parameter (3개)

파라미터	형/폭	기본값
N	int	8
DW	int	32
EnDataPort	bit	1

Table 1255. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree.sv — localparam (6개)

상수	형/폭	값
IdxW	int	$\$clog2(N)$
Base0	int	$(2^{**level})-1$
Base1	int	$(2^{**(\text{level}+1)})-1$
Pa	int	Base0 + offset
C0	int	Base1 + 2*offset
C1	int	Base1 + 2*offset + 1

Table 1256. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
req_chk_i	input	
req_i	input	[N-1:0]
gnt_o	output	logic [N-1:0]
idx_o	output	logic [IdxW-1:0]
valid_o	output	logic
data_o	output	logic [DW-1:0]
ready_i	input	

API — prim_arbiter_tree_dup.sv**Table 1257. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree_dup.sv — module / interface / package (1개)**

이름	종류
prim_arbiter_tree_dup	module

Table 1258. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree_dup.sv — parameter (4개)

파라미터	형/폭	기본값
N	int	8
DW	int	32
EnDataPort	bit	1
FixedArb	bit	0

Table 1259. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree_dup.sv — localparam (2개)

상수	형/폭	값
IdxW	int	\$clog2(N)
ArbInstances	int	2

Table 1260. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree_dup.sv — 포트 (10개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
req_chk_i	input	
req_i	input	[N-1:0]
gnt_o	output	logic [N-1:0]
idx_o	output	logic [IdxW-1:0]
valid_o	output	logic
data_o	output	logic [DW-1:0]
ready_i	input	
err_o	output	logic

API — prim_ascon_duplex.sv

Table 1261. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_duplex.sv — module / interface / package (1개)

이름	종류
prim_ascon_duplex	module

Table 1262. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_duplex.sv — 포트 (24개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
ascon_variant	input	duplex_variant_e
ascon_operation	input	duplex_op_e
start_i	input	logic
done_o	output	logic
idle_o	output	prim_mubi_pkg::mubi4_t

포트	방향	형/폭
no_ad_i	input	prim_mubi_pkg::mubi4_t
no_msg_i	input	prim_mubi_pkg::mubi4_t
key_i	input	logic [127:0]
nonce_i	input	logic [127:0]
data_in_i	input	logic [127:0]
data_in_valid_bytes_i	input	logic [4:0]
last_block_ad_i	input	prim_mubi_pkg::mubi4_t
last_block_msg_i	input	prim_mubi_pkg::mubi4_t
data_in_valid_i	input	logic
data_in_ready_o	output	logic
data_out_o	output	logic [127:0]
data_out_ready_i	input	logic
data_out_valid_o	output	logic
tag_out_o	output	logic [127:0]
tag_out_valid_o	output	logic
fsm_state_o	output	duplex_fsm_state_e
err_o	output	logic

API — prim_ascon_pkg.sv

Table 1263. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_pkg.sv — module / interface / package (1개)

이름	종류
prim_ascon_pkg	package

Table 1264. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_pkg.sv — parameter (8개)

파라미터	형/폭	기본값
AsconRoundCountW	int	4
DUPLEX_OP_WIDTH	int	3
DUPLEX_VARIANT_WIDTH	int	2
KEY_HI_LOW_MUX_WIDTH	int	1
WORD_LOW_KEY_HI_MUX_WIDTH	int	1
ASCON_WORD_MUX_WIDTH	int	2
ROUND_INPUT_MUX_WIDTH	int	1
PADDING_MUX_WIDTH	int	2

Table 1265. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_pkg.sv — localparam (4개)

상수	형/폭	값
IV_128	logic [63:0]	64'h80400c0600000000
IV_128A	logic [63:0]	64'h80800c0800000000
ROUND_MAX	logic [3:0]	4'b1011
AsconDuplexFSMStateWidth	int	10

API — prim_ascon_round.sv

Table 1266. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_round.sv — module / interface / package (1개)

이름	종류
prim_ascon_round	module

Table 1267. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_round.sv — 포트 (3개)

포트	방향	형/폭
state_o	output	logic [319:0]
state_i	input	logic [319:0]
rcon_i	input	logic [7:0]

API — prim_ascon_sbox.sv

Table 1268. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_sbox.sv — module / interface / package (1개)

이름	종류
prim_ascon_sbox	module

Table 1269. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_sbox.sv — 포트 (2개)

포트	방향	형/폭
state_o	output	logic [4:0]
state_i	input	logic [4:0]

API — prim_assert.sv

Table 1270. lowRISC_opentitan/hw/ip/prim/rtl/prim_assert.sv — localparam (1개)

상수	형/폭	값
__name	int	(__prop) ? 1 : 2

Table 1271. lowRISC_opentitan/hw/ip/prim/rtl/prim_assert.sv — #define / `define 매크로 (16개)

매크로	인자	정의
PRIM_ASSERT_SV		////////////////////////////////
ASSERT_DEFAULT_CLK		clk_i
ASSERT_DEFAULT_RST		!rst_ni
PRIM_STRINGIFY	(__x)	`"__x"
ASSERT_ERROR	(__name)	\

매크로	인자	정의
ASSERT_STATIC_LINT_ERROR	(__name, __prop)	\
ASSERT_STATIC_IN_PACKAGE	(__name, __prop)	\
INC_ASSERT		`else
INC_ASSERT		`endif
ASSERT_PULSE	(__name, __sig, __clk = `ASSERT_DEFAULT_CLK, __rst = `ASSERT...	\
ASSERT_IF	(__name, __prop, __enable, __clk = `ASSERT_DEFAULT_CLK, __rs...	\
ASSERT_KNOWN_IF	(__name, __sig, __enable, __clk = `ASSERT_DEFAULT_CLK, __rst...	\
ASSUME_FPV	(__name, __prop, __clk = `ASSERT_DEFAULT_CLK, __rst = `ASSER...	\
ASSUME_I_FPV	(__name, __prop)	\
COVER_FPV	(__name, __prop, __clk = `ASSERT_DEFAULT_CLK, __rst = `ASSER...	\
ASSERT_FPV_LINEAR_FSM	(__name, __state, __type, __clk = `ASSERT_DEFAULT_CLK, __rst...	\

API — prim_blanker.sv

Table 1272. lowRISC_opentitan/hw/ip/prim/rtl/prim_blanker.sv — module / interface / package (1개)

이름	종류
prim_blanker	module

Table 1273. lowRISC_opentitan/hw/ip/prim/rtl/prim_blanker.sv — parameter (1개)

파라미터	형/폭	기본값
Width	int	1

Table 1274. lowRISC_opentitan/hw/ip/prim/rtl/prim_blanker.sv — 포트 (3개)

포트	방향	형/폭
in_i	input	logic [Width-1:0]
en_i	input	logic
out_o	output	logic [Width-1:0]

API — prim_cdc_rand_delay.sv

Table 1275. lowRISC_opentitan/hw/ip/prim/rtl/prim_cdc_rand_delay.sv — module / interface / package (1개)

이름	종류
prim_cdc_rand_delay	module

Table 1276. lowRISC_opentitan/hw/ip/prim/rtl/prim_cdc_rand_delay.sv — parameter (2개)

파라미터	형/폭	기본값
DataWidth	int	1
Enable	bit	1

Table 1277. lowRISC_opentitan/hw/ip/prim/rtl/prim_cdc_rand_delay.sv — 포트 (5개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
prev_data_i	input	logic [DataWidth-1:0]
src_data_i	input	logic [DataWidth-1:0]
dst_data_o	output	logic [DataWidth-1:0]

API — prim_cipher_pkg.sv

Table 1278. lowRISC_opentitan/hw/ip/prim/rtl/prim_cipher_pkg.sv — module / interface / package (1개)

이름	종류
prim_cipher_pkg	package

Table 1279. lowRISC_opentitan/hw/ip/prim/rtl/prim_cipher_pkg.sv — parameter (16개)

파라미터	형/폭	기본값
PRINCE_SBOX4	logic [15:0][3:0]	{4'h4
PRINCE_SBOX4_INV	logic [15:0][3:0]	{4'h1
PRINCE_SHIFT_ROWS64	logic [15:0][3:0]	{4'hF
PRINCE_SHIFT_ROWS64_INV	logic [15:0][3:0]	{4'hF
PRINCE_ROUND_CONST	logic [11:0][63:0]	{64'hC0AC29B7C97C50DD
PRINCE_ALPHA_CONST	logic [63:0]	64'hC0AC29B7C97C50DD
PRINCE_SHIFT_ROWS_CONST0	logic [15:0]	16'h7BDE
PRINCE_SHIFT_ROWS_CONST1	logic [15:0]	16'hBDE7
PRINCE_SHIFT_ROWS_CONST2	logic [15:0]	16'hDE7B
PRINCE_SHIFT_ROWS_CONST3	logic [15:0]	16'hE7BD
PRESENT_SBOX4	logic [15:0][3:0]	{4'h2
PRESENT_SBOX4_INV	logic [15:0][3:0]	{4'hA
PRESENT_PERM32	logic [31:0][4:0]	{5'd31
PRESENT_PERM32_INV	logic [31:0][4:0]	{5'd31
PRESENT_PERM64	logic [63:0][5:0]	{6'd63
PRESENT_PERM64_INV	logic [63:0][5:0]	{6'd63

API — prim_clock_gating_sync.sv

Table 1280. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_gating_sync.sv — module / interface / package (1개)

이름	종류
prim_clock_gating_sync	module

Table 1281. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_gating_sync.sv — 포트 (6개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
test_en_i	input	
async_en_i	input	
en_o	output	logic
clk_o	output	logic

API — prim_clock_meas.sv

Table 1282. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_meas.sv — module / interface / package (1개)

이름	종류
prim_clock_meas	module

Table 1283. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_meas.sv — parameter (4개)

파라미터	형/폭	기본값
Cnt	int	16
RefCnt	int	1
ClkTimeOutChkEn	bit	1
RefTimeOutChkEn	bit	1

Table 1284. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_meas.sv — localparam (6개)

상수	형/폭	값
CntWidth	int	prim_util_pkg::vbits(Cnt)
RefCntWidth	int	prim_util_pkg::vbits(RefCnt)
TimeOutChkEn	bit	ClkTimeOutChkEn RefTimeOutChkEn
ClkRatio	int	Cnt / RefCnt
MaxClkCdcLatency	int	$(1 + 2 * \text{ClkRatio} + 1 * \text{ClkRatio} + 2) * 2$
MaxRefCdcLatency	int	$1 + 1 + 2 + 2$

Table 1285. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_meas.sv — 포트 (12개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
clk_ref_i	input	
rst_ref_ni	input	
en_i	input	
max_cnt	input	[CntWidth-1:0]
min_cnt	input	[CntWidth-1:0]

포트	방향	형/폭
valid_o	output	logic
fast_o	output	logic
slow_o	output	logic
timeout_clk_ref_o	output	logic
ref_timeout_clk_o	output	logic

API — prim_clock_timeout.sv

Table 1286. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_timeout.sv — module / interface / package (1개)

이름	종류
prim_clock_timeout	module

Table 1287. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_timeout.sv — parameter (1개)

파라미터	형/폭	기본값
TimeOutCnt	int	16

Table 1288. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_timeout.sv — localparam (1개)

상수	형/폭	값
CntWidth	int	prim_util_pkg::vbits(TimeOutCnt+1)

Table 1289. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_timeout.sv — 포트 (6개)

포트	방향	형/폭
clk_chk_i	input	
rst_chk_ni	input	
clk_i	input	
rst_ni	input	
en_i	input	
timeout_o	output	logic

API — prim_count.sv

Table 1290. lowRISC_opentitan/hw/ip/prim/rtl/prim_count.sv — module / interface / package (1개)

이름	종류
prim_count	module

Table 1291. lowRISC_opentitan/hw/ip/prim/rtl/prim_count.sv — parameter (4개)

파라미터	형/폭	기본값
Width	int	2
ResetValue	logic [Width-1:0]	'0
EnableAlertTriggerSVA	bit	1

파라미터	형/폭	기본값
PossibleActions	action_mask_t	{ \$bits(action_mask_t){1'b1} }

Table 1292. lowRISC_opentitan/hw/ip/prim/rtl/prim_count.sv — localparam (2개)

상수	형/폭	값
NumCnt	int	2
ResetValues	logic [NumCnt-1:0][Width-1:0]	{{Width{1'b1}} - ResetValue

Table 1293. lowRISC_opentitan/hw/ip/prim/rtl/prim_count.sv — 포트 (12개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
clr_i	input	
set_i	input	
set_cnt_i	input	[Width-1:0]
incr_en_i	input	
decr_en_i	input	
step_i	input	[Width-1:0]
commit_i	input	
cnt_o	output	logic [Width-1:0]
cnt_after_commit_o	output	logic [Width-1:0]
err_o	output	logic

API — prim_count_pkg.sv**Table 1294. lowRISC_opentitan/hw/ip/prim/rtl/prim_count_pkg.sv — module / interface / package (1개)**

이름	종류
prim_count_pkg	package

Table 1295. lowRISC_opentitan/hw/ip/prim/rtl/prim_count_pkg.sv — typedef / using (1개)

새 이름	원래 형
action_mask_t	logic [3:0]

API — prim_crc32.sv**Table 1296. lowRISC_opentitan/hw/ip/prim/rtl/prim_crc32.sv — module / interface / package (1개)**

이름	종류
prim_crc32	module

Table 1297. lowRISC_opentitan/hw/ip/prim/rtl/prim_crc32.sv — parameter (1개)

파라미터	형/폭	기본값
BytesPerWord	int unsigned	4

Table 1298. lowRISC_opentitan/hw/ip/prim/rtl/prim_crc32.sv — 포트 (7개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
set_crc_i	input	logic
crc_in_i	input	logic [31:0]
data_valid_i	input	logic
data_i	input	logic [BytesPerWord*8-1:0]
crc_out_o	output	logic [31:0]

API — prim_diff_decode.sv**Table 1299. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_decode.sv — module / interface / package (1개)**

이름	종류
prim_diff_decode	module

Table 1300. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_decode.sv — parameter (2개)

파라미터	형/폭	기본값
AsyncOn	bit	1'b0
SkewCycles	int unsigned	1

Table 1301. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_decode.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
diff_pi	input	
diff_ni	input	
level_o	output	logic
rise_o	output	logic
fall_o	output	logic
event_o	output	logic
sigint_o	output	logic

Table 1302. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_decode.sv — typedef / using (1개)

새 이름	원래 형
state_e	enum logic [1:0] {IsStd, IsSkewing, SigInt}

API — prim_diff_encode.sv**Table 1303. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_encode.sv — module / interface / package (1개)**

이름	종류
prim_diff_encode	module

Table 1304. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_encode.sv — 포트 (5개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
req_i	input	logic
diff_po	output	logic
diff_no	output	logic

API — prim_diff_to_alert.sv**Table 1305. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_to_alert.sv — module / interface / package (1개)**

이름	종류
prim_diff_to_alert	module

Table 1306. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_to_alert.sv — parameter (3개)

파라미터	형/폭	기본값
Async0n	bit	1'b1
SkewCycles	int unsigned	1
IsFatal	bit	1'b1

Table 1307. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_to_alert.sv — 포트 (6개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
diff_pi	input	logic
diff_ni	input	logic
alert_rx_i	input	prim_alert_pkg::alert_rx_t
alert_tx_o	output	prim_alert_pkg::alert_tx_t

API — prim_dom_and_2share.sv**Table 1308. lowRISC_opentitan/hw/ip/prim/rtl/prim_dom_and_2share.sv — module / interface / package (1개)**

이름	종류
prim_dom_and_2share	module

Table 1309. lowRISC_opentitan/hw/ip/prim/rtl/prim_dom_and_2share.sv — parameter (2개)

파라미터	형/폭	기본값
DW	int	64
Pipeline	bit	1'b0 // Enable full pipelining

Table 1310. lowRISC_opentitan/hw/ip/prim/rtl/prim_dom_and_2share.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
a0_i	input	[DW-1:0]
a1_i	input	[DW-1:0]
b0_i	input	[DW-1:0]
b1_i	input	[DW-1:0]
z_valid_i	input	
z_i	input	[DW-1:0]
q0_o	output	logic [DW-1:0]
q1_o	output	logic [DW-1:0]
instances	output	logic [DW-1:0] prd_o // pseudo-random data for oth...

API — prim_double_lfsr.sv**Table 1311. lowRISC_opentitan/hw/ip/prim/rtl/prim_double_lfsr.sv — module / interface / package (1개)**

이름	종류
prim_double_lfsr	module

Table 1312. lowRISC_opentitan/hw/ip/prim/rtl/prim_double_lfsr.sv — parameter (13개)

파라미터	형/폭	기본값
LfsrType		"GAL_XOR"
LfsrDw	int unsigned	32
EntropyDw	int unsigned	8
StateOutDw	int unsigned	8
DefaultSeed	logic [LfsrDw-1:0]	LfsrDw'(1)
CustomCoeffs	logic [LfsrDw-1:0]	'0
StatePermEn	bit	1'b0
StatePerm	logic [LfsrDw-1:0][LfsrIdxDw-1:0]	'0
MaxLenSVA	bit	1'b1
LockupSVA	bit	1'b1
ExtSeedSVA	bit	1'b1
NonLinearOut	bit	1'b0

파라미터	형/폭	기본값
EnableAlertTriggerSVA	bit	1

Table 1313. lowRISC_opentitan/hw/ip/prim/rtl/prim_double_lfsr.sv — localparam (1개)

상수	형/폭	값
LfsrIdxDw	int unsigned	\$clog2(LfsrDw)

Table 1314. lowRISC_opentitan/hw/ip/prim/rtl/prim_double_lfsr.sv — 포트 (8개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
seed_en_i	input	
seed_i	input	[LfsrDw-1:0]
lfsr_en_i	input	
entropy_i	input	[EntropyDw-1:0]
state_o	output	logic [StateOutDw-1:0]
err_o	output	logic

API — prim_edge_detector.sv**Table 1315. lowRISC_opentitan/hw/ip/prim/rtl/prim_edge_detector.sv — module / interface / package (1개)**

이름	종류
prim_edge_detector	module

Table 1316. lowRISC_opentitan/hw/ip/prim/rtl/prim_edge_detector.sv — parameter (3개)

파라미터	형/폭	기본값
Width	int unsigned	1
ResetValue	logic [Width-1:0]	'0
EnSync	bit	1'b 1

Table 1317. lowRISC_opentitan/hw/ip/prim/rtl/prim_edge_detector.sv — 포트 (6개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
d_i	input	[Width-1:0]
q_sync_o	output	logic [Width-1:0]
q_posedge_pulse_o	output	logic [Width-1:0]
q_negedge_pulse_o	output	logic [Width-1:0]

1. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_pkg.sv (27 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 package prim_alert_pkg;
6
7   typedef struct packed {
8     logic alert_p;
9     logic alert_n;
10  } alert_tx_t;
11
12   typedef struct packed {
13     logic ping_p;
14     logic ping_n;
15     logic ack_p;
16     logic ack_n;
17  } alert_rx_t;
18
19   parameter alert_tx_t ALERT_TX_DEFAULT = '{alert_p: 1'b0,
20                                           alert_n: 1'b1};
21
22   parameter alert_rx_t ALERT_RX_DEFAULT = '{ping_p: 1'b0,
23                                           ping_n: 1'b1,
24                                           ack_p: 1'b0,
25                                           ack_n: 1'b1};
26
27 endpackage : prim_alert_pkg

```

2. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_receiver.sv (391 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // The alert receiver primitive decodes alerts that have been
  differentially
6 // encoded and transmitted via a handshake protocol on
  alert_p/n and
7 // ack_p/n. In case an alert handshake is initiated, the out-
  put alert_o will
8 // immediately be asserted (even before completion of the
  handshake).
9 //
10 // In case the differential input is not correctly encoded,
  this module will
11 // raise an error by asserting integ_fail_o.
12 //
13 // Further, the module supports ping testing of the alert diff
  pair. In order to
14 // initiate a ping test, ping_req_i shall be set to 1'b1 until
  ping_ok_o is
15 // asserted for one cycle. The signal may be de-asserted (e.g.
  after a long)
16 // timeout period. However note that all ping responses that
  come in after
17 // deasserting ping_req_i will be treated as native alerts.
18 //
19 // The protocol works in both asynchronous and synchronous
  cases. In the
20 // asynchronous case, the parameter AsyncOn must be set to
  1'b1 in order to
21 // instantiate additional synchronization logic. Further, it
  must be ensured
22 // that the timing skew between all diff pairs is smaller than
  the shortest
23 // clock period of the involved clocks.
24 //
25 // Note that in case of synchronous operation, alerts on the
  diffpair are
26 // decoded combinatorially and forwarded on alert_o within the
  same cycle.
27 //
28 // See also: prim_alert_sender, prim_diff_decode,
  alert_handler
29
30 `include "prim_assert.sv"
31
32 module prim_alert_receiver
33   import prim_alert_pkg::*;
34   import prim_mubi_pkg::mubi4_t;
35 (
36   // enables additional synchronization logic
37   parameter bit AsyncOn = 1'b0,
38   // Number of cycles a differential skew is tolerated on the
  differential alert signal.

```

```

39   parameter int unsigned SkewCycles = 1
40 ) (
41   input          clk_i,
42   input          rst_ni,
43   // if set to lc_ctrl_pkg::0n, this triggers the in-band
  alert channel
44   // reset, which resets both the sender and receiver FSMs
  into IDLE.
45   input mubi4_t   init_trig_i,
46   // this triggers a ping test. keep asserted
47   // until ping_ok_o is asserted.
48   input          ping_req_i,
49   output logic    ping_ok_o,
50   // asserted if signal integrity issue detected
51   output logic    integ_fail_o,
52   // alert output (pulsed high) if a handshake is initiated
53   // on alert_p/n and no ping request is outstanding
54   output logic    alert_o,
55   // ping input diff pair and ack diff pair
56   output alert_rx_t alert_rx_o,
57   // alert output diff pair
58   input alert_tx_t alert_tx_i
59 );
60
61 import prim_mubi_pkg::mubi4_test_true_strict;
62
63 ///////////////////////////////////////////////////////////////////
64 // decode differential signals //
65 ///////////////////////////////////////////////////////////////////
66 logic alert_level, alert_sigint, alert_p, alert_n;
67
68 // This prevents further tool optimizations of the differen-
  tial signal.
69 prim_sec_anchor_buf #(
70   .Width(2)
71 ) u_prim_buf_in (
72   .in_i({alert_tx_i.alert_n,
73         alert_tx_i.alert_p}),
74   .out_o({alert_n,
75         alert_p})
76 );
77
78 prim_diff_decode #(
79   .AsyncOn(AsyncOn),
80   .SkewCycles(SkewCycles)
81 ) u_decode_alert (
82   .clk_i,
83   .rst_ni,
84   .diff_pi ( alert_p ),
85   .diff_ni ( alert_n ),
86   .level_o ( alert_level ),
87   .rise_o ( ),
88   .fall_o ( ),
89   .event_o ( ),
90   .sigint_o ( alert_sigint )
91 );
92
93 ///////////////////////////////////////////////////////////////////
94 // main protocol FSM that drives the diff outputs //
95 ///////////////////////////////////////////////////////////////////
96 typedef enum logic [2:0] {Idle, HsAckWait, Pause0, Pause1,
  InitReq, InitAckWait} state_e;
97 state_e state_d, state_q;
98 logic ping_rise;
99 logic ping_tog_pd, ping_tog_pq, ping_tog_dn, ping_tog_nq;
100 logic ack_pd, ack_pq, ack_dn, ack_nq;
101 logic ping_req_d, ping_req_q;
102 logic ping_pending_d, ping_pending_q;
103 logic send_init;
104 logic send_ping;
105
106 // signal ping request upon positive transition on
  ping_req_i
107 // signalling is performed by a level change event on the
  diff output
108 assign ping_req_d = ping_req_i;
109 assign ping_rise = ping_req_d && !ping_req_q;
110 assign ping_tog_pd = (send_init) ? 1'b0 :
111                    (send_ping) ? ~ping_tog_pq :
112
113 // in-band reset is performed by sending out an integrity
  error on purpose.
114 assign ack_dn = (send_init) ? ack_pd : ~ack_pd;
115 assign ping_tog_dn = ~ping_tog_pd;
116
117 // This prevents further tool optimizations of the differen-
  tial signal.
118 prim_sec_anchor_flop #(
119   .Width (2),
120   .ResetValue(2'b10)
121 ) u_prim_generic_flop_ack (
122   .clk_i,
123   .rst_ni,
124   .d_i({ack_dn,
125         ack_pd}),
126   .q_o({ack_nq,
127         ack_pq})
128 );
129
130 prim_sec_anchor_flop #(

```

```

131     .Width      (2),
132     .ResetValue(2'b10)
133 ) u_prim_generic_flop_ping (
134     .clk_i,
135     .rst_ni,
136     .d_i({ping_tog_dn,
137           ping_tog_pd}),
138     .q_o({ping_tog_nq,
139           ping_tog_pq});
140 );
141
142 // the ping pending signal is used in the FSM to distinguish
whether the
143 // incoming handshake shall be treated as an alert or a ping
response.
144 // it is important that this is only set on a rising ping_en
level change, since
145 // otherwise the ping enable signal could be abused to
"mask" all native alerts
146 // as ping responses by constantly tying it to 1.
147 assign ping_pending_d = ping_rise | ((~ping_ok_o) &
ping_req_i & ping_pending_q);
148
149 // diff pair outputs
150 assign alert_rx_o.ack_p = ack_pq;
151 assign alert_rx_o.ack_n = ack_nq;
152
153 assign alert_rx_o.ping_p = ping_tog_pq;
154 assign alert_rx_o.ping_n = ping_tog_nq;
155
156 // this FSM receives the four phase handshakes from the
alert receiver
157 // note that the latency of the alert_p/n input diff pair is
at least one
158 // cycle until it enters the receiver FSM. the same holds
for the ack_* diff
159 // pair outputs.
160 always_comb begin : p_fsm
161     // default
162     state_d = state_q;
163     ack_pd = 1'b0;
164     ping_ok_o = 1'b0;
165     integ_fail_o = 1'b0;
166     alert_o = 1'b0;
167     send_init = 1'b0;
168     // by default, a ping request leads to a toggle on the
differential ping pair
169     send_ping = ping_rise;
170
171     unique case (state_q)
172     Idle: begin
173         // wait for handshake to be initiated
174         if (alert_level) begin
175             state_d = HSAckWait;
176             ack_pd = 1'b1;
177             // signal either an alert or ping received on the
output
178             if (ping_pending_q) begin
179                 ping_ok_o = 1'b1;
180             end else begin
181                 alert_o = 1'b1;
182             end
183         end
184     end
185     // waiting for deassertion of alert to complete HS
186     HSAckWait: begin
187         if (!alert_level) begin
188             state_d = Pause0;
189         end else begin
190             ack_pd = 1'b1;
191         end
192     end
193     // pause cycles between back-to-back handshakes
194     Pause0: state_d = Pause1;
195     Pause1: state_d = Idle;
196     // this state is only reached if an in-band reset is
197     // requested via the low-power logic.
198     InitReq: begin
199         // we deliberately place a sigint error on the ack and
ping lines in this case.
200         send_init = 1'b1;
201         // suppress any toggles on the ping line while we are
in the init phase.
202         send_ping = 1'b0;
203         // As long as init req is asserted, we remain in this
state and acknowledge all incoming
204         // ping requests. As soon as the init request is
dropped however, ping requests are not
205         // acked anymore such that the ping mechanism can also
flag alert channels that got stuck
206         // in the initialization sequence.
207         if (mubi4_test_true_strict(init_trig_i)) begin
208             ping_ok_o = ping_pending_q;
209             // the sender will respond to the sigint error above
with a sigint error on the alert lines.

```

```

210         // hence we treat the alert_sigint like an acknow-
ledge in this case.
211     end else if (alert_sigint) begin
212         state_d = InitAckWait;
213     end
214 end
215 // We get here if the sender has responded with
alert_sigint, and init_trig_i==lc_ctrl_pkg::On
216 // has been deasserted. At this point, we need to wait
for the alert_sigint to drop again
217 // before resuming normal operation.
218 InitAckWait: begin
219     // suppress any toggles on the ping line while we are
in the init phase.
220     send_ping = 1'b0;
221     if (!alert_sigint) begin
222         state_d = Pause0;
223         // If we get a ping request in this cycle, or if we
realize that there is an unhandled
224         // ping request that came in during initialization
(but after init_trig_i has been
225         // deasserted), we signal this to the alert sender
by toggling the request line.
226         send_ping = ping_rise || ping_pending_q;
227     end
228 end
229 default: state_d = Idle;
230 endcase
231
232 // once the initialization sequence has been triggered,
233 // overrides are not allowed anymore until the initializa-
tion has been completed.
234 if (!(state_q inside {InitReq, InitAckWait})) begin
235     // in this case, abort and jump into the initialization
sequence
236     if (mubi4_test_true_strict(init_trig_i)) begin
237         state_d = InitReq;
238         ack_pd = 1'b0;
239         ping_ok_o = 1'b0;
240         integ_fail_o = 1'b0;
241         alert_o = 1'b0;
242         send_init = 1'b1;
243     end
244 // if we're not busy with an init request, we clamp down
all outputs
245 // and indicate an integrity failure.
246 end else if (alert_sigint) begin
247     state_d = Idle;
248     ack_pd = 1'b0;
249     ping_ok_o = 1'b0;
250     integ_fail_o = 1'b1;
251     alert_o = 1'b0;
252 end
253 end
254
255 always_ff @(posedge clk_i or negedge rst_ni) begin : p_reg
256     if (!rst_ni) begin
257         // Reset into the init request so that an alert handler
reset implicitly
258         // triggers an in-band reset of all alert channels.
259         state_q <= InitReq;
260         ping_req_q <= 1'b0;
261         ping_pending_q <= 1'b0;
262     end else begin
263         state_q <= state_d;
264         ping_req_q <= ping_req_d;
265         ping_pending_q <= ping_pending_d;
266     end
267 end
268
269 // assertions //
270 ///////////////////////////////////////////////////////////////////
271 // assertions //
272 ///////////////////////////////////////////////////////////////////
273
274 `ifdef INC_ASSERT
275 import prim_mubi_pkg:mubi4_test_false_loose;
276 `endif
277
278 // check whether all outputs have a good known state after
reset
279 `ASSERT_KNOWN(PingOkKnown0_A, ping_ok_o)
280 `ASSERT_KNOWN(IntegFailKnown0_A, integ_fail_o)
281 `ASSERT_KNOWN(AlertKnown0_A, alert_o)
282 `ASSERT_KNOWN(PingPKnown0_A, alert_rx_o)
283
284 // check encoding of outgoing diffpairs. note that during
init, the outgoing diffpairs are
285 // supposed to be incorrectly encoded on purpose.
286 // shift sequence two cycles to the right to avoid reset ef-
fects.
287 `ASSERT(PingDiffOk_A, alert_rx_o.ping_p ^ alert_rx_o.ping_n)
288
289 `ASSERT(AckDiffOk_A, ##2 $past(send_init) ^ alert_rx_o.ack_p ^
alert_rx_o.ack_n)
290 `ASSERT(InitReq_A, mubi4_test_true_strict(init_trig_i) &&
!(state_q inside {InitReq, InitAckWait})) | =>
send_init)
291
292 // If there is a ping request on the input then we should

```

```

see an encoded ping request. This is
293 // squashed if we are in state InitReq or InitAckWait (be-
cause we are still initialising), or if
294 // we see the init_trig_i signal go high (because it will
start an initialisation).
295 `ASSERT(PingRequest0_A,
296     $rose(ping_req_i) |> $changed(alert_rx_o.ping_p),
297     clk_i, !rst_ni || init_trig_i || state_q inside
{InitReq, InitAckWait})
298
299 // ping response implies it has been requested
300 `ASSERT(PingResponse0_A, ping_ok_o |> ping_pending_q)
301 // correctly latch ping request
302 `ASSERT(PingPending_A, ##1 $rose(ping_req_i) |> ping_pend-
ing_q)
303
304 if (AsyncOn) begin : gen_async_assert
305 // signal integrity check propagation
306 `ASSERT(SigInt_A,
307     alert_tx_i.alert_p == alert_tx_i.alert_n [*2] ##2
308     !(state_q inside {InitReq, InitAckWait}) &&
309     mubi4_test_false_loose(init_trig_i)
310     |>
311     ##[0:SkewCycles] integ_fail_o)
312 `ASSERT(PingResponse1_A,
313     ##1 $rose(alert_tx_i.alert_p) &&
314     (alert_tx_i.alert_p ^ alert_tx_i.alert_n) ##2
315     state_q == Idle && ping_pending_q
316     |>
317     ##[0:SkewCycles] ping_ok_o,
318     clk_i, !rst_ni || integ_fail_o || mu-
bi4_test_true_strict(init_trig_i))
319 // alert
320 `ASSERT(Alert_A,
321     ##1 $rose(alert_tx_i.alert_p) &&
322     (alert_tx_i.alert_p ^ alert_tx_i.alert_n) ##2
323     state_q == Idle &&
324     !ping_pending_q
325     |>
326     ##[0:SkewCycles] alert_o,
327     clk_i, !rst_ni || integ_fail_o || mu-
bi4_test_true_strict(init_trig_i))
328 end else begin : gen_sync_assert
329 // signal integrity check propagation
330 `ASSERT(SigInt_A,
331     alert_tx_i.alert_p == alert_tx_i.alert_n &&
332     !(state_q inside {InitReq, InitAckWait}) &&
333     mubi4_test_false_loose(init_trig_i)
334     |>
335     integ_fail_o)
336 // ping response
337 `ASSERT(PingResponse1_A,
338     ##1 $rose(alert_tx_i.alert_p) &&
339     state_q == Idle &&
340     ping_pending_q
341     |>
342     ping_ok_o,
343     clk_i, !rst_ni || integ_fail_o || mu-
bi4_test_true_strict(init_trig_i))
344 // alert
345 `ASSERT(Alert_A,
346     ##1 $rose(alert_tx_i.alert_p) &&
347     state_q == Idle &&
348     !ping_pending_q
349     |>
350     alert_o,
351     clk_i, !rst_ni || integ_fail_o || mu-
bi4_test_true_strict(init_trig_i))
352 end
353
354 // check in-band init request is always accepted
355 `ASSERT(InBandInitRequest_A,
356     mubi4_test_true_strict(init_trig_i) &&
357     state_q != InitAckWait
358     |>
359     state_q == InitReq)
360 // check in-band init sequence moves FSM into IDLE state
361 `ASSERT(InBandInitSequence_A,
362     (state_q == InitReq &&
363     mubi4_test_true_strict(init_trig_i)) ##1
364     (alert_sigint &&
365     mubi4_test_false_loose(init_trig_i)) [*1:$] ##1
366     (!alert_sigint &&
367     mubi4_test_false_loose(init_trig_i)) [*3]
368     |>
369     state_q == Idle)
370 // check there are no spurious alerts during init
371 `ASSERT(NoSpuriousAlertsDuringInit_A,
372     mubi4_test_true_strict(init_trig_i) ||
373     (state_q inside {InitReq, InitAckWait})
374     |>
375     !alert_o)
376 // check that there are no spurious ping OKs
377 `ASSERT(NoSpuriousPingOKsDuringInit_A,
378     (mubi4_test_true_strict(init_trig_i) ||
379     (state_q inside {InitReq, InitAckWait})) &&
380     !ping_pending_q
381     |>
382     !ping_ok_o)
383 // check ping request is bypassed when in init state
384 `ASSERT(PingOkBypassDuringInit_A,

```

```

385     $rose(ping_req_i) ##1
386     state_q == InitReq &&
387     mubi4_test_true_strict(init_trig_i)
388     |>
389     ping_ok_o)
390
391 endmodule : prim_alert_receiver

```

3. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_sender.sv (398 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // The alert sender primitive module differentially encodes
and transmits an
6 // alert signal to the prim_alert_receiver module. An alert
will be signalled
7 // by a full handshake on alert_p/n and ack_p/n. The
alert_req_i signal may
8 // be continuously asserted, in which case the alert sig-
nalling handshake
9 // will be repeatedly initiated.
10 //
11 // The alert_req_i signal may also be used as part of req/ack.
The parent module
12 // can keep alert_req_i asserted until it has been ack'd
(transferred to the alert
13 // receiver). The parent module is not required to use this.
14 //
15 // In case the alert sender parameter IsFatal is set to 1, an
incoming alert
16 // alert_req_i is latched in a local register until the next
reset, causing the
17 // alert sender to behave as if alert_req_i were continuously
asserted.
18 // The alert_state_o output reflects the state of this intern-
al latching register.
19 //
20 // The alert sender also exposes an alert test input, which
can be used to trigger
21 // single alert handshakes. This input behaves exactly the
same way as the
22 // alert_req_i input with IsFatal set to 0. Test alerts do not
cause alert_ack_o
23 // to be asserted, nor are they latched until reset (regard-
less of the value of the
24 // IsFatal parameter).
25 //
26 // Further, this module supports in-band ping testing, which
means that a level
27 // change on the ping_p/n diff pair will result in a full-
handshake response
28 // on alert_p/n and ack_p/n.
29 //
30 // The protocol works in both asynchronous and synchronous
cases. In the
31 // asynchronous case, the parameter AsyncOn must be set to
1'b1 in order to
32 // instantiate additional synchronization logic. Further, it
must be ensured
33 // that the timing skew between all diff pairs is smaller than
the shortest
34 // clock period of the involved clocks.
35 //
36 // Incorrectly encoded diff inputs can be detected and will be
signalled
37 // to the receiver by placing an inconsistent diff value on
the differential
38 // output (and continuously toggling it).
39 //
40 // See also: prim_alert_receiver, prim_diff_decode,
alert_handler
41
42 `include "prim_assert.sv"
43
44 module prim_alert_sender
45     import prim_alert_pkg::*;
46 (
47     // enables additional synchronization logic
48     parameter bit AsyncOn = 1'b1,
49     // Number of cycles a differential skew is tolerated on the
differential ack/ping signal.
50     parameter int unsigned SkewCycles = 1,
51     // alert sender will latch the incoming alert event perman-
ently and
52     // keep on sending alert events until the next reset.
53     parameter bit IsFatal = 1'b0
54 ) (
55     input          clk_i,
56     input          rst_ni,
57     // alert test trigger (this will never be latched, even if
IsFatal == 1)

```



```

58 input          alert_test_i,
59 // native alert from the peripheral
60 input          alert_req_i,
61 output logic    alert_ack_o,
62 // state of the alert latching register
63 output logic    alert_state_o,
64 // ping input diff pair and ack diff pair
65 input alert_rx_t alert_rx_i,
66 // alert output diff pair
67 output alert_tx_t alert_tx_o
68 );
69
70
71 ////////////////////////////////////////////////////
72 // decode differential signals //
73 ////////////////////////////////////////////////////
74 logic ping_sigint, ping_event, ping_n, ping_p;
75
76 // This prevents further tool optimizations of the differen-
77 tial signal.
78 prim_sec_anchor_buf #(
79   .Width(2)
80 ) u_prim_buf_ping (
81   .in_i({alert_rx_i.ping_n,
82         alert_rx_i.ping_p}),
83   .out_o({ping_n,
84         ping_p})
85 );
86
87 prim_diff_decode #(
88   .AsyncOn(AsyncOn),
89   .SkewCycles(SkewCycles)
90 ) u_decode_ping (
91   .clk_i,
92   .rst_ni,
93   .diff_pi ( ping_p ),
94   .diff_ni ( ping_n ),
95   .level_o (      ),
96   .rise_o  (      ),
97   .fall_o  (      ),
98   .event_o ( ping_event ),
99   .sigint_o ( ping_sigint )
100 );
101
102 logic ack_sigint, ack_level, ack_n, ack_p;
103
104 // This prevents further tool optimizations of the differen-
105 tial signal.
106 prim_sec_anchor_buf #(
107   .Width(2)
108 ) u_prim_buf_ack (
109   .in_i({alert_rx_i.ack_n,
110         alert_rx_i.ack_p}),
111   .out_o({ack_n,
112         ack_p})
113 );
114
115 prim_diff_decode #(
116   .AsyncOn(AsyncOn),
117   .SkewCycles(SkewCycles)
118 ) u_decode_ack (
119   .clk_i,
120   .rst_ni,
121   .diff_pi ( ack_p ),
122   .diff_ni ( ack_n ),
123   .level_o ( ack_level ),
124   .rise_o  (      ),
125   .fall_o  (      ),
126   .event_o (      ),
127   .sigint_o ( ack_sigint )
128 );
129
130 ////////////////////////////////////////////////////
131 // main protocol FSM that drives the diff output //
132 ////////////////////////////////////////////////////
133 typedef enum logic [2:0] {
134   Idle,
135   AlertHsPhase1,
136   AlertHsPhase2,
137   PingHsPhase1,
138   PingHsPhase2,
139   Pause0,
140   Pause1
141 } state_e;
142 state_e state_d, state_q;
143 logic alert_pq, alert_nq, alert_pd, alert_nd;
144 logic sigint_detected;
145
146 assign sigint_detected = ack_sigint | ping_sigint;
147
148 // diff pair output
149 assign alert_tx_o.alert_p = alert_pq;
150 assign alert_tx_o.alert_n = alert_nq;
151
152 // alert and ping set regs
153 logic alert_set_d, alert_set_q, alert_clr;
154 logic alert_test_set_d, alert_test_set_q;
155 logic ping_set_d, ping_set_q, ping_clr;
156 logic alert_req_trigger, alert_test_trigger, ping_trigger;
157
158 // if handshake is ongoing, capture additional alert re-
159 quests.
160 logic alert_req;
161 prim_sec_anchor_buf #(
162   .Width(1)
163 ) u_prim_buf_in_req (
164   .in_i(alert_req_i),
165   .out_o(alert_req)
166 );
167
168 assign alert_req_trigger = alert_req | alert_set_q;
169 if (IsFatal) begin : gen_fatal
170   assign alert_set_d = alert_req_trigger;
171 end else begin : gen_recov
172   assign alert_set_d = (alert_clr) ? 1'b0 :
173   alert_req_trigger;
174 end
175
176 // the alert test request is always cleared.
177 assign alert_test_trigger = alert_test_i | alert_test_set_q;
178 assign alert_test_set_d = (alert_clr) ? 1'b0 :
179 alert_test_trigger;
180
181 logic alert_trigger;
182 assign alert_trigger = alert_req_trigger |
183 alert_test_trigger;
184
185 assign ping_trigger = ping_set_q | ping_event;
186 assign ping_set_d = (ping_clr) ? 1'b0 : ping_trigger;
187
188 // alert event acknowledge and state (not affected by
189 alert_test_i)
190 assign alert_ack_o = alert_clr & alert_set_q;
191 assign alert_state_o = alert_set_q;
192
193 // this FSM performs a full four phase handshake upon a ping
194 or alert trigger.
195 // note that the latency of the alert_p/n diff pair is at
196 least one cycle
197 // until it enters the receiver FSM. the same holds for the
198 ack_* diff pair
199 // input. in case a signal integrity issue is detected, the
200 FSM bails out,
201 // sets the alert_p/n diff pair to the same value and
202 toggles it in order to
203 // signal that condition over to the receiver.
204 always_comb begin : p_fsm
205   // default
206   state_d = state_q;
207   alert_pd = 1'b0;
208   alert_nd = 1'b1;
209   ping_clr = 1'b0;
210   alert_clr = 1'b0;
211
212   unique case (state_q)
213     Idle: begin
214       // alert always takes precedence
215       if (alert_trigger || ping_trigger) begin
216         state_d = (alert_trigger) ? AlertHsPhase1 : PingHs-
217 Phase1;
218       end
219       alert_pd = 1'b1;
220       alert_nd = 1'b0;
221     end
222     // waiting for ack from receiver
223     AlertHsPhase1: begin
224       if (ack_level) begin
225         state_d = AlertHsPhase2;
226       end else begin
227         alert_pd = 1'b1;
228         alert_nd = 1'b0;
229       end
230     end
231     // wait for deassertion of ack
232     AlertHsPhase2: begin
233       if (!ack_level) begin
234         state_d = Pause0;
235         alert_clr = 1'b1;
236       end
237     end
238     // waiting for ack from receiver
239     PingHsPhase1: begin
240       if (ack_level) begin
241         state_d = PingHsPhase2;
242       end else begin
243         alert_pd = 1'b1;
244         alert_nd = 1'b0;
245       end
246     end
247     // wait for deassertion of ack
248     PingHsPhase2: begin
249       if (!ack_level) begin
250         ping_clr = 1'b1;
251         state_d = Pause0;
252       end
253     end
254     Pause0: begin
255       // pause cycles between back-to-back handshakes
256       Pause0: begin

```

```

246     state_d = Pause1;
247 end
248 // clear and ack alert request if it was set
249 Pause1: begin
250     state_d = Idle;
251 end
252 // catch parasitic states
253 default : state_d = Idle;
254 endcase
255

256 // we have a signal integrity issue at one of the incoming
257 // diff pairs. this condition is
258 // signalled by setting the output diffpair to zero. If
259 // the sigint has disappeared, we clear
260 // the ping request state of this sender and go back to
261 // idle.
262 if (sigint_detected) begin
263     state_d = Idle;
264     alert_pd = 1'b0;
265     alert_nd = 1'b0;
266     ping_clr = 1'b1;
267     alert_clr = 1'b0;
268 end
269 end
270 // This prevents further tool optimizations of the differen-
271 // tial signal.
272 prim_sec_anchor_flop #(
273     .Width (2),
274     .ResetValue(2'b10)
275 ) u_prim_flop_alert (
276     .clk_i,
277     .rst_ni,
278     .d_i({alert_nd, alert_pd}),
279     .q_o({alert_nq, alert_pq})
280 );
281
282 always_ff @(posedge clk_i or negedge rst_ni) begin : p_reg
283     if (!rst_ni) begin
284         state_q <= Idle;
285         alert_set_q <= 1'b0;
286         alert_test_set_q <= 1'b0;
287         ping_set_q <= 1'b0;
288     end else begin
289         state_q <= state_d;
290         alert_set_q <= alert_set_d;
291         alert_test_set_q <= alert_test_set_d;
292         ping_set_q <= ping_set_d;
293     end
294 end
295
296 // assertions
297
298 // however, since we use sequence constructs below, we need to
299 // wrap the entire block again.
300 // typically, the ASSERT macros already contain this
301 // INC_ASSERT macro.
302 `ifdef INC_ASSERT
303 // check whether all outputs have a good known state after
304 // reset
305 `ASSERT_KNOWN(AlertPKnown0_A, alert_tx_o)
306
307 if (Async0n) begin : gen_async_assert
308     sequence PingSigInt_S;
309     alert_rx_i.ping_p == alert_rx_i.ping_n [*2];
310 endsequence
311 sequence AckSigInt_S;
312 alert_rx_i.ping_p == alert_rx_i.ping_n [*2];
313 endsequence
314
315 `ifndef FPV_ALERT_NO_SIGINT_ERR
316 // check propagation of sigint issues to output within
317 // three cycles, or four due to CDC
318 // shift sequence to the right to avoid reset effects.
319 `ASSERT(SigIntPing_A, ##1 PingSigInt_S |>
320     ##[SkewCycles+2:SkewCycles+3] alert_tx_o.alert_p ==
321     alert_tx_o.alert_n)
322 `ASSERT(SigIntAck_A, ##1 AckSigInt_S |>
323     ##[SkewCycles+2:SkewCycles+3] alert_tx_o.alert_p ==
324     alert_tx_o.alert_n)
325 `endif
326
327 // Test in-band FSM reset request (via signal integrity
328 // error)
329 `ASSERT(InBandInitFsm_A, PingSigInt_S or AckSigInt_S |>
330     state_q == Idle)
331 `ASSERT(InBandInitPing_A, PingSigInt_S or AckSigInt_S |
332     => !ping_set_q)
333 // output must be driven diff unless sigint issue detected
334 `ASSERT(DiffEncoding_A, (alert_rx_i.ack_p ^
335     alert_rx_i.ack_n) &&
336     (alert_rx_i.ping_p ^ alert_rx_i.ping_n) |>
337     alert_tx_o.alert_p ^ alert_tx_o.alert_n)
338 // handshakes can take indefinite time if blocked due to
339 // sigint on outgoing
340 // lines (which is not visible here). thus, we only check
341 // whether the handshake
342 // is correctly initiated and defer the full handshake
343 // checking to the testbench.
344 `ASSERT(PingHs_A, ##1 $changed(alert_rx_i.ping_p) &&
345     state_q == Idle |>
346     $rose(alert_tx_o.alert_p), clk_i, !rst_ni ||
347     (alert_tx_o.alert_p == alert_tx_o.alert_n))
348 end
349 // Test the alert state output.
350 `ASSERT(AlertState0_A, alert_set_q == alert_state_o)
351
352 if (IsFatal) begin : gen_fatal_assert
353     `ASSERT(AlertState1_A, alert_req_i |> alert_state_o)
354     `ASSERT(AlertState2_A, alert_state_o |>
355         $stable(alert_state_o))
356     `ASSERT(AlertState3_A, alert_ack_o |> alert_state_o)
357 end else begin : gen_recov_assert
358     `ASSERT(AlertState1_A, alert_req_i && !alert_clr |>
359         alert_state_o)
360     `ASSERT(AlertState2_A, alert_req_i && alert_ack_o |> !
361         alert_state_o)
362 end
363
364 // The alert test input should not set the alert state re-
365 // gister.
366 `ASSERT(AlertTest1_A, alert_test_i && !alert_req_i && !
367     alert_state_o |> $stable(alert_state_o))
368
369 // if alert_req_i is true, handshakes should be continuously
370 // repeated
371 `ASSERT(AlertHs_A, alert_req_i && state_q == Idle |>
372     $rose(alert_tx_o.alert_p),
373     clk_i, !rst_ni || (alert_tx_o.alert_p ==
374     alert_tx_o.alert_n))
375
376 // if alert_test_i is true, handshakes should be continu-
377 // ously repeated
378 `ASSERT(AlertTestHs_A, alert_test_i && state_q == Idle |>
379     $rose(alert_tx_o.alert_p),
380     clk_i, !rst_ni || (alert_tx_o.alert_p ==
381     alert_tx_o.alert_n))
382 `endif
383
384 // Test in-band FSM reset request (via signal integrity
385 // error)
386 `ASSERT(InBandInitFsm_A, PingSigInt_S or AckSigInt_S |>
387     ##[SkewCycles+2:SkewCycles+3] state_q == Idle)
388 `ASSERT(InBandInitPing_A, PingSigInt_S or AckSigInt_S |>
389     ##[SkewCycles+2:SkewCycles+3] !ping_set_q)
390 // output must be driven diff unless sigint issue detected
391 `ASSERT(DiffEncoding_A, (alert_rx_i.ack_p ^
392     alert_rx_i.ack_n) &&
393     (alert_rx_i.ping_p ^ alert_rx_i.ping_n) |>
394     alert_tx_o.alert_p ^ alert_tx_o.alert_n)
395 // handshakes can take indefinite time if blocked due to
396 // sigint on outgoing
397 // lines (which is not visible here). thus, we only check
398 // whether the handshake
399 // is correctly initiated and defer the full handshake
400 // checking to the testbench.
401 `ASSERT(PingHs_A, ##1 $changed(alert_rx_i.ping_p) &&
402     state_q == Idle |>
403     $rose(alert_tx_o.alert_p), clk_i, !rst_ni ||
404     (alert_tx_o.alert_p == alert_tx_o.alert_n))
405 end
406 // Test the alert state output.
407 `ASSERT(AlertState0_A, alert_set_q == alert_state_o)
408
409 if (IsFatal) begin : gen_fatal_assert
410     `ASSERT(AlertState1_A, alert_req_i |> alert_state_o)
411     `ASSERT(AlertState2_A, alert_state_o |>
412         $stable(alert_state_o))
413     `ASSERT(AlertState3_A, alert_ack_o |> alert_state_o)
414 end else begin : gen_recov_assert
415     `ASSERT(AlertState1_A, alert_req_i && !alert_clr |>
416         alert_state_o)
417     `ASSERT(AlertState2_A, alert_req_i && alert_ack_o |> !
418         alert_state_o)
419 end
420
421 // The alert test input should not set the alert state re-
422 // gister.
423 `ASSERT(AlertTest1_A, alert_test_i && !alert_req_i && !
424     alert_state_o |> $stable(alert_state_o))
425
426 // if alert_req_i is true, handshakes should be continuously
427 // repeated
428 `ASSERT(AlertHs_A, alert_req_i && state_q == Idle |>
429     $rose(alert_tx_o.alert_p),
430     clk_i, !rst_ni || (alert_tx_o.alert_p ==
431     alert_tx_o.alert_n))
432
433 // if alert_test_i is true, handshakes should be continu-
434 // ously repeated
435 `ASSERT(AlertTestHs_A, alert_test_i && state_q == Idle |>
436     $rose(alert_tx_o.alert_p),
437     clk_i, !rst_ni || (alert_tx_o.alert_p ==
438     alert_tx_o.alert_n))
439 `endif
440
441 // Assumptions for FPV security countermeasures to ensure
442 // the alert protocol functions correctly.
443 `ASSUME_FPV(AckPFollowsAlertP_S, alert_rx_i.ack_p ==
444     $past(alert_tx_o.alert_p))
445 `ASSUME_FPV(AckNFollowsAlertN_S, alert_rx_i.ack_n ==
446     $past(alert_tx_o.alert_n))
447 `ASSUME_FPV(TriggerAlertInit_S, $stable(rst_ni) == 0 |>
448     alert_rx_i.ping_p == alert_rx_i.ping_n)
449 `ASSUME_FPV(PingDiffPair_S, ##2 alert_rx_i.ping_p !=
450     alert_rx_i.ping_n)
451 `endif
452 endmodule : prim_alert_sender

```

4. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_to_diff.sv (63 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // prim_alert_to_diff: This module receives an alert signal
  (using the alert protocol) and
6 // translates it into a differentially encoded signal (without the
  alert protocol).
7 // It also handles integrity errors from the alert receiver,
  treating them as alerts.
8 //
9 // IMPORTANT: This primitive is NOT intended to be used in
  OpenTitan top-level designs.
10 // In top-level designs, prim_alert_{sender,receiver} pairs
  should be used instead.
11 // The intended use case of this primitive is to signal an
  alert through a differential
12 // encoding (for minimal integrity protection) to modules that do
  not support
13 // OpenTitan's alert protocol.
14
15 module prim_alert_to_diff #(
16     // AsyncOn: Enables additional synchronization logic within
  the alert receiver.
17     parameter bit AsyncOn = 1'b0,
18     // Number of cycles a differential skew is tolerated on the
  alert signal
19     parameter int unsigned SkewCycles = 1
20 ) (
21     input logic                clk_i,
22     input logic                rst_ni,
23     // Alert pair (interface signals for the alert protocol)
24     output prim_alert_pkg::alert_rx_t alert_rx_o,
25     input  prim_alert_pkg::alert_tx_t alert_tx_i,
26     // Output diff pair (differentially encoded alert signal)
27     output logic               diff_po,
28     output logic               diff_no
29 );
30
31 logic integ_error;
32 logic alert;
33
34 // u_prim_alert_receiver: Instantiates the alert receiver
35 module
36     prim_alert_receiver #(
37         .AsyncOn(AsyncOn),
38         .SkewCycles(SkewCycles)
39     ) u_prim_alert_receiver (
40         .clk_i,
41         .rst_ni,
42         .init_trig_i (prim_mubi_pkg::MuBi4False),
43         .ping_req_i  (1'b0),
44         .ping_ok_o   (),
45         .integ_fail_o (integ_error),
46         .alert_o     (alert),
47         .alert_rx_o,
48         .alert_tx_i
49     );
50 // Combines the decoded alert and the integrity error signal.
51
52 // An alert is triggered if either a valid alert is received or
  an integrity error occurs.
53 logic combined_alert;
54 assign combined_alert = integ_error | alert;
55
56 // Instantiates the differential encoder module.
57 prim_diff_encode u_prim_diff_encode (
58     .clk_i,
59     .rst_ni,
60     .req_i(combined_alert), // Input request signal (combined
  alert).
61     .diff_po,
62     .diff_no
63 );
64 endmodule

```

5. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_fixed.sv (170 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // N:1 fixed priority arbiter module (index 0 has highest
  prio)
6 //
7 // Verilog parameter

```

```

8 // N:          Number of request ports
9 // DW:         Data width
10 // DataPort:   Set to 1 to enable the data port. Otherwise
  that port will be ignored.
11 //
12 // See also: prim_arbiter_ppc, prim_arbiter_tree
13
14 `include "prim_assert.sv"
15
16 module prim_arbiter_fixed #(
17     parameter int N = 8,
18     parameter int DW = 32,
19
20     // Configurations
21     // EnDataPort: {0, 1}, if 0, input data will be ignored
22     parameter bit EnDataPort = 1,
23
24     // Derived parameters
25     localparam int IdxW = $clog2(N)
26 ) (
27     // used for assertions only
28     input clk_i,
29     input rst_ni,
30
31     input [N-1:0] req_i,
32     input [DW-1:0] data_i [N],
33     output logic [N-1:0] gnt_o,
34     output logic [IdxW-1:0] idx_o,
35
36     output logic valid_o,
37     output logic [DW-1:0] data_o,
38     input ready_i
39 );
40
41 `ASSERT_INIT(CheckNGreaterZero_A, N > 0)
42
43 // this case is basically just a bypass
44 if (N == 1) begin : gen_degenerate_case
45
46     assign valid_o = req_i[0];
47     assign data_o = data_i[0];
48     assign gnt_o[0] = valid_o & ready_i;
49     assign idx_o = '0;
50
51 end else begin : gen_normal_case
52
53     // align to powers of 2 for simplicity
54     // a full binary tree with N levels has 2**N + 2**N-1
55 nodes
56     logic [2**(IdxW+1)-2:0] req_tree;
57     logic [2**(IdxW+1)-2:0] gnt_tree;
58     logic [2**(IdxW+1)-2:0][IdxW-1:0] idx_tree;
59     logic [2**(IdxW+1)-2:0][DW-1:0] data_tree;
60
61     for (genvar level = 0; level < IdxW+1; level++) begin :
  gen_tree
62         //
63         // level+1 C0 C1 <- "Base1" points to the first
  node on "level+1",
64         // \ / these nodes are the children
  of the nodes one level below
65         // level Pa <- "Base0", points to the first
  node on "level",
66         // these nodes are the parents
  of the nodes one level above
67         //
68         // hence we have the following indices for the Pa, C0,
  C1 nodes:
69         // Pa = 2**level - 1 + offset = Base0 + offset
70         // C0 = 2**(level+1) - 1 + 2*offset = Base1 + 2*off-
  set
71         // C1 = 2**(level+1) - 1 + 2*offset + 1 = Base1 + 2*off-
  set + 1
72         //
73         localparam int Base0 = (2**level)-1;
74         localparam int Base1 = (2**(level+1))-1;
75
76         for (genvar offset = 0; offset < 2**level; offset++) be-
  gin : gen_level
77             localparam int Pa = Base0 + offset;
78             localparam int C0 = Base1 + 2*offset;
79             localparam int C1 = Base1 + 2*offset + 1;
80
81             // this assigns the gated interrupt source signals,
  their
82             // corresponding IDs and priorities to the tree leafs
83             if (level == IdxW) begin : gen_leaves
84                 if (offset < N) begin : gen_assign
85                     // forward path
86                     assign req_tree[Pa] = req_i[offset];
87                     assign idx_tree[Pa] = offset;
88                     assign data_tree[Pa] = data_i[offset];
89                     // backward (grant) path
90                     assign gnt_o[offset] = gnt_tree[Pa];
91
92             end else begin : gen_tie_off
93                 // forward path
94                 assign req_tree[Pa] = '0;
95                 assign idx_tree[Pa] = '0;
96                 assign data_tree[Pa] = '0;
97                 logic unused_sigs;

```

```

97         assign unused_sigs = gnt_tree[Pa];
98     end
99     // this creates the node assignments
100 end else begin : gen_nodes
101     // forward path
102     logic sel; // local helper variable
103     always_comb begin : p_node
104         // this always gives priority to the left child
105         sel = ~req_tree[C0];
106         // propagate requests
107         req_tree[Pa] = req_tree[C0] | req_tree[C1];
108         // data and index muxes
109         idx_tree[Pa] = (sel) ? idx_tree[C1] :
idx_tree[C0];
110     data_tree[Pa] = (sel) ? data_tree[C1] :
data_tree[C0];
111     // propagate the grants back to the input
112     gnt_tree[C0] = gnt_tree[Pa] & ~sel;
113     gnt_tree[C1] = gnt_tree[Pa] & sel;
114     end
115 end
116 end : gen_level
117 end : gen_tree
118
119 // the results can be found at the tree root
120 if (EnDataPort) begin : gen_data_port
121     assign data_o = data_tree[0];
122 end else begin : gen_no_dataport
123     logic [DW-1:0] unused_data;
124     assign unused_data = data_tree[0];
125     assign data_o = '1;
126 end
127
128 assign idx_o = idx_tree[0];
129 assign valid_o = req_tree[0];
130
131 // this propagates a grant back to the input
132 assign gnt_tree[0] = valid_o & ready_i;
133 end
134
135 // assertions
136 // assertions
137 // assertions
138
139 // KNOWN assertions on outputs, except for data as that may
be partially X in simulation
140 // e.g. when used on a BUS
141 `ASSERT_KNOWN(ValidKnown_A, valid_o)
142 `ASSERT_KNOWN(GrantKnown_A, gnt_o)
143 `ASSERT_KNOWN(IdxKnown_A, idx_o)
144
145 // Make sure no higher prio req is asserted
146 `ASSERT(Priority_A, |req_i |-> req_i[idx_o] && (((N'(1'b1)
<< idx_o) - 1'b1) & req_i) == '0)
147
148 // we can only grant one requester at a time
149 `ASSERT(CheckHotOne_A, $onehot0(gnt_o))
150 // A grant implies that the sink is ready
151 `ASSERT(GntImpliesReady_A, |gnt_o |-> ready_i)
152 // A grant implies that the arbiter asserts valid as well
153 `ASSERT(GntImpliesValid_A, |gnt_o |-> valid_o)
154 // A request and a sink that is ready imply a grant
155 `ASSERT(ReqAndReadyImpliesGrant_A, |req_i && ready_i |-> |
gnt_o)
156 // A request and a sink that is ready imply a grant
157 `ASSERT(ReqImpliesValid_A, |req_i |-> valid_o)
158 // Both conditions above combined and reversed
159 `ASSERT(ReadyAndValidImpliesGrant_A, ready_i && valid_o |-> |
gnt_o)
160 // Both conditions above combined and reversed
161 `ASSERT(NoReadyValidNoGrant_A, !(ready_i || valid_o) |->
gnt_o == 0)
162 // check index / grant correspond
163 `ASSERT(IndexIsCorrect_A, ready_i && valid_o |->
gnt_o[idx_o] && req_i[idx_o])
164
165 if (EnDataPort) begin: gen_data_port_assertion
166     // data flow
167     `ASSERT(DataFlow_A, ready_i && valid_o |-> data_o ==
data_i[idx_o])
168 end
169
170 endmodule : prim_arbiter_fixed

```

6. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_ppc.sv (210 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // N:1 arbiter module
6 //
7 // Verilog parameter
8 // N: Number of request ports
9 // DW: Data width
10 // DataPort: Set to 1 to enable the data port. Otherwise
that port will be ignored.

```

```

11 //
12 // This is the original implementation of the arbiter which
relies on parallel prefix computing
13 // optimization to optimize the request / arbiter tree. Not
all synthesis tools may support this.
14 //
15 // Note that the currently winning request is held if the data
sink is not ready. This behavior is
16 // required by some interconnect protocols (AXI, TL). The mod-
ule contains an assertion that checks
17 // this behavior.
18 //
19 // Also, this module contains a request stability assertion
that checks that requests stay asserted
20 // until they have been served. This assertion can be gated by
driving the req_chk_i low. This is
21 // a non-functional input and does not affect the designs be-
havior.
22 //
23 // See also: prim_arbiter_tree
24
25 `include "prim_assert.sv"
26
27 module prim_arbiter_ppc #(
28     parameter int unsigned N = 8,
29     parameter int unsigned DW = 32,
30
31     // Configurations
32     // EnDataPort: {0, 1}, if 0, input data will be ignored
33     parameter bit EnDataPort = 1,
34
35     // Derived parameters
36     localparam int IdxW = prim_util_pkg::vbits(N)
37 ) (
38     input clk_i,
39     input rst_ni,
40
41     input req_chk_i, // Used for gating as-
sertions. Drive to 1 during normal
42 // operation.
43     input [N-1:0] req_i,
44     input [DW-1:0] data_i [N],
45     output logic [N-1:0] gnt_o,
46     output logic [IdxW-1:0] idx_o,
47
48     output logic valid_o,
49     output logic [DW-1:0] data_o,
50     input ready_i
51 );
52
53 // req_chk_i is used for gating assertions only.
54 logic unused_req_chk;
55 assign unused_req_chk = req_chk_i;
56
57 `ASSERT_INIT(CheckNGreaterZero_A, N > 0)
58
59 // this case is basically just a bypass
60 if (N == 1) begin : gen_degenerate_case
61
62     assign valid_o = req_i[0];
63     assign data_o = data_i[0];
64     assign gnt_o[0] = valid_o & ready_i;
65     assign idx_o = '0;
66
67 end else begin : gen_normal_case
68     logic [N-1:0] masked_req;
69     logic [N-1:0] ppc_out;
70     logic [N-1:0] arb_req;
71     logic [N-1:0] mask, mask_next;
72     logic [N-1:0] winner;
73
74     assign masked_req = mask & req_i;
75     assign arb_req = (~masked_req) ? masked_req : req_i;
76
77     prim_leading_one_ppc #(
78         .N ( N )
79     ) u_leading_one (
80         .in_i ( arb_req ),
81         .leading_one_o ( winner ),
82         .ppc_out_o ( ppc_out ),
83         .idx_o ( idx_o )
84     );
85     assign gnt_o = (ready_i) ? winner : '0;
86     assign valid_o = |req_i;
87     // Mask Generation
88     assign mask_next = {ppc_out[N-2:0], 1'b0};
89     always_ff @(posedge clk_i or negedge rst_ni) begin
90         if (!rst_ni) begin
91             mask <= '0;
92         end else if (valid_o && ready_i) begin
93             // Latch only when requests accepted
94             mask <= mask_next;
95         end else if (valid_o && !ready_i) begin
96
97             // Downstream isn't yet ready so, keep current request
alive. (First come first serve)
98             mask <= ppc_out;
99         end

```

```

100
101   if (EnDataPort == 1) begin: gen_datapath
102     always_comb begin
103       data_o = '0;
104       for (int i = 0; i < N; i++) begin
105         if (winner[i]) begin
106           data_o = data_i[i];
107         end
108       end
109     end
110   end else begin: gen_nodatapath
111     assign data_o = '1;
112     // The following signal is used to avoid possible lint
113 errors.
114     logic [DW-1:0] unused_data [N];
115     assign unused_data = data_i;
116   end
117
118   // assertions //
119   // assertions //
120   // assertions //
121
122   // KNOWN assertions on outputs, except for data as that may
123 be partially X in simulation
124   // e.g. when used on a BUS
125   `ASSERT_KNOWN(ValidKnown_A, valid_o)
126   `ASSERT_KNOWN(GrantKnown_A, gnt_o)
127   `ASSERT_KNOWN(IdxKnown_A, idx_o)
128   // grant index shall be higher index than previous index,
129 unless no higher requests exist.
130   `ASSERT(RoundRobin_A,
131     ##1 valid_o && ready_i && $past(ready_i) && $past(val-
132 id_o) &&
133     ((req_i & ~(N'(1) << $past(idx_o)+1) - 1)) |->
134     idx_o > $past(idx_o))
135   // we can only grant one requester at a time
136   `ASSERT(CheckHotOne_A, $onehot0(gnt_o))
137   // A grant implies that the sink is ready
138   `ASSERT(GntImpliesReady_A, |gnt_o |-> ready_i)
139   // A grant implies that the arbiter asserts valid as well
140   `ASSERT(GntImpliesValid_A, |gnt_o |-> valid_o)
141   // A request and a sink that is ready imply a grant
142   `ASSERT(ReqAndReadyImpliesGrant_A, |req_i && ready_i |-> |
143 gnt_o)
144   // A request and a sink that is ready imply a grant
145   `ASSERT(ReqImpliesValid_A, |req_i |-> valid_o)
146   // Both conditions above combined and reversed
147   `ASSERT(ReadyAndValidImpliesGrant_A, ready_i && valid_o |-> |
148 gnt_o)
149   // Both conditions above combined and reversed
150   `ASSERT(NoReadyValidNoGrant_A, !(ready_i || valid_o) |->
151 gnt_o == 0)
152   // check index / grant correspond
153   `ASSERT(IndexIsCorrect_A, ready_i && valid_o |->
154 gnt_o[idx_o] && req_i[idx_o])
155
156   if (EnDataPort) begin: gen_data_port_assertion
157     // data flow
158     `ASSERT(DataFlow_A, ready_i && valid_o |-> data_o ==
159 data_i[idx_o])
160   end
161
162   // requests must stay asserted until they have been granted
163   `ASSUME(ReqStaysHighUntilGranted0_M, |req_i && !ready_i |>
164 (req_i & $past(req_i)) == $past(req_i), clk_i, !rst_ni
165 || !req_chk_i)
166   // check that the arbitration decision is held if the sink
167 is not ready
168   `ASSERT(LockArbDecision_A, |req_i && !ready_i |> idx_o ==
169 $past(idx_o),
170   clk_i, !rst_ni || !req_chk_i)
171
172   // FPV-only assertions with symbolic variables
173   `ifdef FPV_ON
174     // symbolic variables
175     bit [IdxW-1:0] k;
176     bit ReadyIsStable;
177     bit ReqsAreStable;
178
179     // constraints for symbolic variables
180     `ASSUME(KStable_M, ##1 $stable(k))
181     `ASSUME(KRange_M, k < N)
182     // this is used enable checking for stable and unstable
183 ready_i and req_i signals in the same run.
184     // the symbolic variables act like a switch that the solver
185 can turn on and off.
186     `ASSUME(ReadyIsStable_M, ##1 $stable(ReadyIsStable))
187     `ASSUME(ReqsAreStable_M, ##1 $stable(ReqsAreStable))
188     `ASSUME(ReadyStable_M, ##1 !ReadyIsStable ||
189 $stable(ready_i))
190     `ASSUME(ReqsStable_M, ##1 !ReqsAreStable || $stable(req_i))
191
192     // A grant implies a request
193     `ASSERT(GntImpliesReq_A, gnt_o[k] |-> req_i[k])
194
195     // if request and ready are constantly held at 1, we should
196 eventually get a grant
197     `ASSERT(NoStarvation_A,
198       ReqsAreStable && ReadyIsStable && ready_i && req_i[k] |->

```

```

>
185     strong(##[0:$] gnt_o[k]))
186
187   // if N requests are constantly asserted and ready is con-
188 stant 1, each request must
189   // be granted exactly once over a time window of N cycles
190 for the arbiter to be fair.
191   for (genvar n = 1; n <= N; n++) begin : gen_fairness
192     integer gnt_cnt;
193     `ASSERT(Fairness_A,
194       ReqsAreStable && ReadyIsStable && ready_i && req_i[k]
195 &&
196     $countones(req_i) == n |->
197     ##n gnt_cnt == $past(gnt_cnt, n) + 1)
198
199     always_ff @(posedge clk_i or negedge rst_ni) begin : p_cnt
200       if (!rst_ni) begin
201         gnt_cnt <= 0;
202       end else begin
203         gnt_cnt <= gnt_cnt + gnt_o[k];
204       end
205     end
206   end
207
208   // requests must stay asserted until they have been granted
209   `ASSUME(ReqStaysHighUntilGranted1_M, req_i[k] && !gnt_o[k] |
210 =>
211   req_i[k], clk_i, !rst_ni || !req_chk_i)
212 `endif
213 endmodule : prim_arbiter_ppc

```

7. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree.sv (291 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 details.
4 // SPDX-License-Identifier: Apache-2.0
5 //
6 // N:1 arbiter module
7 //
8 // Verilog parameter
9 // N: Number of request ports
10 // DW: Data width
11 // DataPort: Set to 1 to enable the data port. Otherwise
12 that port will be ignored.
13 //
14 // This is a tree implementation of a round robin arbiter. It
15 has the same behavior as the PPC
16 // implementation in prim_arbiter_ppc, and also uses a prefix
17 summing approach to determine the next
18 // request to be granted. The main difference with respect to
19 the PPC arbiter is that the leading 1
20 // detection and the prefix summation are performed with a
21 binary tree instead of a sequential loop.
22 // Also, if the data port is enabled, the data is muxed based
23 on the local arbitration decisions at
24
25 // each node of the arbiter tree. This means that the data can
26 propagate through the tree
27 // simultaneously with the requests, instead of waiting for
28 the arbitration to determine the winner
29 // index first. As a result, this design has a shorter critic-
30 al path than other implementations,
31 // leading to better overall timing.
32
33 // Note that the currently winning request is held if the data
34 sink is not ready. This behavior is
35 // required by some interconnect protocols (AXI, TL). The mod-
36 ule contains an assertion that checks
37 // this behavior.
38 //
39 // Also, this module contains a request stability assertion
40 that checks that requests stay asserted
41
42 // until they have been served. This assertion can be gated by
43 driving the req_chk_i low. This is
44 // a non-functional input and does not affect the designs be-
45 havior.
46 //
47 // See also: prim_arbiter_ppc
48
49 `include "prim_assert.sv"
50
51 module prim_arbiter_tree #(
52   parameter int N = 8,
53   parameter int DW = 32,
54
55   // Configurations
56   // EnDataPort: {0, 1}, if 0, input data will be ignored
57   parameter bit EnDataPort = 1,
58
59   // Derived parameters
60   localparam int IdxW = $clog2(N)
61 ) (
62   input clk_i,
63   input rst_ni,

```



```

47
48 input req_chk_i, // Used for gating as-
sections. Drive to 1 during normal
49 // operation.
50 input [N-1:0] req_i,
51 input [DW-1:0] data_i [N],
52 output logic [N-1:0] gnt_o,
53 output logic [IdxW-1:0] idx_o,
54
55 output logic valid_o,
56 output logic [DW-1:0] data_o,
57 input ready_i
58 );
59
60 // req_chk_i is used for gating assertions only.
61 logic unused_req_chk;
62 assign unused_req_chk = req_chk_i;
63
64 `ASSERT_INIT(CheckNGreaterZero_A, N > 0)
65
66 // this case is basically just a bypass
67 if (N == 1) begin : gen_degenerate_case
68
69 assign valid_o = req_i[0];
70 assign data_o = data_i[0];
71 assign gnt_o[0] = valid_o & ready_i;
72 assign idx_o = '0;
73
74 end else begin : gen_normal_case
75
76 // align to powers of 2 for simplicity
77 // a full binary tree with N levels has 2**N + 2**N-1
nodes
78 logic [2**(IdxW+1)-2:0] req_tree;
79 logic [2**(IdxW+1)-2:0] prio_tree;
80 logic [2**(IdxW+1)-2:0] sel_tree;
81 logic [2**(IdxW+1)-2:0] mask_tree;
82 logic [2**(IdxW+1)-2:0][IdxW-1:0] idx_tree;
83 logic [2**(IdxW+1)-2:0][DW-1:0] data_tree;
84 logic [N-1:0] prio_mask_d,
prio_mask_q;
85
86 for (genvar level = 0; level < IdxW+1; level++) begin :
gen_tree
87 //
88 // level+1 C0 C1 <- "Base1" points to the first
node on "level+1",
89 // \ / these nodes are the children
of the nodes one level below
90 // level Pa <- "Base0", points to the first
node on "level",
91 // these nodes are the parents
of the nodes one level above
92 //
93 // hence we have the following indices for the Pa, C0,
C1 nodes:
94 // Pa = 2**level - 1 + offset = Base0 + offset
95 // C0 = 2**(level+1) - 1 + 2*offset = Base1 + 2*off-
set
96 // C1 = 2**(level+1) - 1 + 2*offset + 1 = Base1 + 2*off-
set + 1
97 //
98 localparam int Base0 = (2**level)-1;
99 localparam int Base1 = (2**(level+1))-1;
100
101 for (genvar offset = 0; offset < 2**level; offset++) be-
gin : gen_level
102 localparam int Pa = Base0 + offset;
103 localparam int C0 = Base1 + 2*offset;
104 localparam int C1 = Base1 + 2*offset + 1;
105
106 // this assigns the gated interrupt source signals,
their
107 // corresponding IDs and priorities to the tree leafs
108 if (level == IdxW) begin : gen_leafs
109 if (offset < N) begin : gen_assign
110 // forward path (requests and data)
111
112 // all requests inputs are assigned to the request
tree
112 assign req_tree[Pa] = req_i[offset];
113 // we basically split the incoming request vector
into two halves with the following
114 // priority assignment. the prio_mask_q register
contains a prefix sum that has been
115 // computed using the last winning index, and
hence masks out all requests at offsets
116 // lower or equal the previously granted index.
hence, all higher indices are considered
117 // first in the arbitration tree nodes below, be-
fore considering the lower indices.
118 assign prio_tree[Pa] = req_i[offset] &
prio_mask_q[offset];
119 // input for the index muxes (used to compute the
winner index)
120 assign idx_tree[Pa] = offset;
121 // input for the data muxes
122 assign data_tree[Pa] = data_i[offset];
123
124 // backward path (grants and prefix sum)
125 // grant if selected, ready and request asserted

```

```

126 assign gnt_o[offset] = req_i[offset] &
sel_tree[Pa] & ready_i;
127 // only update mask if there is a valid request
128 assign prio_mask_d[offset] = (req_i) ?
mask_tree[Pa] |
129 sel_tree[Pa] & ~ready_i :
prio_mask_q[offset];
130
131 end else begin : gen_tie_off
132 // forward path
133 assign req_tree[Pa] = '0;
134 assign prio_tree[Pa] = '0;
135 assign idx_tree[Pa] = '0;
136 assign data_tree[Pa] = '0;
137 logic unused_sigs;
138 assign unused_sigs = ^{mask_tree[Pa],
sel_tree[Pa]};
139
140 end
141 // this creates the node assignments
142 end else begin : gen_nodes
143 // local helper variable
144 logic sel;
145
146 // forward path (requests and data)
147 // each node looks at its two children, and selects
the one with higher priority
148 assign sel = ~req_tree[C0] | ~prio_tree[C0] &
prio_tree[C1];
149 // propagate requests
150 assign req_tree[Pa] = req_tree[C0] | req_tree[C1];
151 assign prio_tree[Pa] = prio_tree[C1] |
prio_tree[C0];
152 // data and index muxes
153 // Note: these ternaries have triggered a synthesis
bug in Vivado versions older
154 // than 2020.2. If the problem resurfaces again,
have a look at issue #1408.
155 assign idx_tree[Pa] = (sel) ? idx_tree[C1] :
idx_tree[C0];
156 assign data_tree[Pa] = (sel) ? data_tree[C1] :
data_tree[C0];
157
158 // backward path (grants and prefix sum)
159 // this propagates the selection index back and com-
putes a hot one mask
160 assign sel_tree[C0] = sel_tree[Pa] & ~sel;
161 assign sel_tree[C1] = sel_tree[Pa] & sel;
162 // this performs a prefix sum for masking the input
requests in the next cycle
163 assign mask_tree[C0] = mask_tree[Pa];
164 assign mask_tree[C1] = mask_tree[Pa] | sel_tree[C0];
165 end
166 end : gen_level
167 end : gen_tree
168
169 // the results can be found at the tree root
170 if (EnDataPort) begin : gen_data_port
171 assign data_o = data_tree[0];
172 end else begin : gen_no_dataport
173 logic [DW-1:0] unused_data;
174 assign unused_data = data_tree[0];
175 assign data_o = '1;
176 end
177
178 // This index is unused.
179 logic unused_prio_tree;
180 assign unused_prio_tree = prio_tree[0];
181
182 assign idx_o = idx_tree[0];
183 assign valid_o = req_tree[0];
184
185 // the select tree computes a hot one signal that indic-
ates which request is currently selected
186 assign sel_tree[0] = 1'b1;
187 // the mask tree is basically a prefix sum of the hot one
select signal computed above
188 assign mask_tree[0] = 1'b0;
189
190 always_ff @(posedge clk_i or negedge rst_ni) begin :
p_mask_reg
191 if (!rst_ni) begin
192 prio_mask_q <= '0;
193 end else begin
194 prio_mask_q <= prio_mask_d;
195 end
196 end
197 end
198
199 // assertions //
200 // assertions //
201 // assertions //
202
203 // KNOWN assertions on outputs, except for data as that may
be partially X in simulation
204 // e.g. when used on a BUS
205 `ASSERT_KNOWN(ValidKnown_A, valid_o)
206 `ASSERT_KNOWN(GrantKnown_A, gnt_o)
207 `ASSERT_KNOWN(IdxKnown_A, idx_o)
208
209 // grant index shall be higher index than previous index,
unless no higher requests exist.
210 `ASSERT(RoundRobin_A,

```

```

211      ##1 valid_o && ready_i && $past(ready_i) && $past(val-
id_o) &&
212      |(req_i & ~(N'(1) << $past(idx_o)+1) - 1)) |->
213      idx_o > $past(idx_o))
214      // we can only grant one requester at a time
215      `ASSERT(CheckHotOne_A, $onehot0(gnt_o))
216      // A grant implies that the sink is ready
217      `ASSERT(GntImpliesReady_A, |gnt_o |-> ready_i)
218      // A grant implies that the arbiter asserts valid as well
219      `ASSERT(GntImpliesValid_A, |gnt_o |-> valid_o)
220      // A request and a sink that is ready imply a grant
221      `ASSERT(ReqAndReadyImpliesGrant_A, |req_i && ready_i |-> |
gnt_o)
222      // A request and a sink that is ready imply a grant
223      `ASSERT(ReqImpliesValid_A, |req_i |-> valid_o)
224      // Both conditions above combined and reversed
225      `ASSERT(ReadyAndValidImpliesGrant_A, ready_i && valid_o |-> |
gnt_o)
226      // Both conditions above combined and reversed
227      `ASSERT(NoReadyValidNoGrant_A, !(ready_i || valid_o) |->
gnt_o == 0)
228      // check index / grant correspond
229      `ASSERT(IndexIsCorrect_A, ready_i && valid_o |->
gnt_o[idx_o] && req_i[idx_o])
230
231      if (EnDataPort) begin: gen_data_port_assertion
232      // data flow
233      `ASSERT(DataFlow_A, ready_i && valid_o |-> data_o ==
data_i[idx_o])
234      end
235
236      // requests must stay asserted until they have been granted
237      `ASSUME(ReqStaysHighUntilGranted0_M, |req_i && !ready_i |>=
238      (req_i & $past(req_i)) == $past(req_i), clk_i, !rst_ni
|| req_chk_i)
239      // check that the arbitration decision is held if the sink
is not ready
240      `ASSERT(LockArbDecision_A, |req_i && !ready_i |>= idx_o ==
$past(idx_o),
241      clk_i, !rst_ni || !req_chk_i)
242
243      // FPV-only assertions with symbolic variables
244      `ifdef FPV_ON
245      // symbolic variables
246      int unsigned k;
247      bit ReadyIsStable;
248      bit ReqsAreStable;
249
250      // constraints for symbolic variables
251      `ASSUME(KStable_M, ##1 $stable(k))
252      `ASSUME(KRange_M, k < N)
253      // this is used enable checking for stable and unstable
ready_i and req_i signals in the same run.
254      // the symbolic variables act like a switch that the solver
can turn on and off.
255      `ASSUME(ReadyIsStable_M, ##1 $stable(ReadyIsStable))
256      `ASSUME(ReqsAreStable_M, ##1 $stable(ReqsAreStable))
257      `ASSUME(ReadyStable_M, ##1 !ReadyIsStable ||
$stable(ready_i))
258      `ASSUME(ReqsStable_M, ##1 !ReqsAreStable || $stable(req_i))
259
260      // A grant implies a request
261      `ASSERT(GntImpliesReq_A, gnt_o[k] |-> req_i[k])
262
263      // if request and ready are constantly held at 1, we should
eventually get a grant
264      `ASSERT(NoStarvation_A,
265      ReqsAreStable && ReadyIsStable && ready_i && req_i[k] |->
>
266      strong(##[0:$] gnt_o[k]))
267
268      // if N requests are constantly asserted and ready is con-
stant 1, each request must
269      // be granted exactly once over a time window of N cycles
for the arbiter to be fair.
270      for (genvar n = 1; n <= N; n++) begin : gen_fairness
271      integer gnt_cnt;
272      `ASSERT(Fairness_A,
273      ReqsAreStable && ReadyIsStable && ready_i && req_i[k]
&&
274      $countones(req_i) == n |->
275      ##n gnt_cnt == $past(gnt_cnt, n) + 1)
276
277      always_ff @(posedge clk_i or negedge rst_ni) begin : p_cnt
278      if (!rst_ni) begin
279      gnt_cnt <= 0;
280      end else begin
281      gnt_cnt <= gnt_cnt + gnt_o[k];
282      end
283      end
284
285      // requests must stay asserted until they have been granted
286      `ASSUME(ReqStaysHighUntilGranted1_M, req_i[k] && !gnt_o[k] |
=>
287      req_i[k], clk_i, !rst_ni || !req_chk_i)
288      `endif
289
290
291      endmodule : prim_arbiter_tree

```

8. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree_dup.sv (148 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // This is a wrapper module that instantiates two prim_arbit-
er_tree modules.
6 // The reason for two is similar to modules such as
prim_count/prim_lfsr where
7 // we use spatial redundancy to ensure the arbitration results are
not altered.
8 //
9 // Note there are limits to this check, as the inputs to the
duplicated arbiter
10 // is still a single lane. So an upstream attack can defeat
this module. The
11 // duplication merely protects against attacks directly on the
arbiter.
12
13 `include "prim_assert.sv"
14
15 module prim_arbiter_tree_dup #(
16     parameter int N = 8,
17     parameter int DW = 32,
18
19     // Configurations
20     `EnDataPort: {0, 1}, if 0, input data will be ignored
parameter bit EnDataPort = 1,
21
22     // if arbiter has fixed priority
parameter bit FixedArb = 0,
23
24     // Derived parameters
25     localparam int IdxW = $clog2(N)
26 ) (
27     input clk_i,
28     input rst_ni,
29
30     input req_chk_i, // Used for gating as-
sions. Drive to 1 during normal
31
32     // operation.
33     input [N-1:0] req_i,
34     input [DW-1:0] data_i [N],
35     output logic [N-1:0] gnt_o,
36     output logic [IdxW-1:0] idx_o,
37
38     output logic valid_o,
39     output logic [DW-1:0] data_o,
40     input ready_i,
41     output logic err_o
42 );
43
44 localparam int ArbInstances = 2;
45
46 //typedef struct packed {
47 //    logic [N-1:0] req;
48 //    logic [N-1:0][DW-1:0] data;
49 //} arb_inputs_t;
50
51 typedef struct packed {
52     logic valid;
53     logic [N-1:0] gnt;
54     logic [IdxW-1:0] idx;
55     logic [DW-1:0] data;
56 } arb_outputs_t;
57
58 // buffer up the inputs separately for each instance
59 //arb_inputs_t arb_in;
60 //arb_inputs_t [ArbInstances-1:0] arb_input_buf;
61 arb_outputs_t [ArbInstances-1:0] arb_output_buf;
62
63 for (genvar i = 0; i < ArbInstances; i++) begin : gen_in-
put_bufs
64     logic [N-1:0] req_buf;
65     prim_buf #(
66         .Width(N)
67     ) u_req_buf (
68         .in_i(req_i),
69         .out_o(req_buf)
70     );
71
72     logic [DW-1:0] data_buf [N];
73     for (genvar j = 0; j < N; j++) begin : gen_data_bufs
74         prim_buf #(
75             .Width(DW)
76         ) u_dat_buf (
77             .in_i(data_i[j]),
78             .out_o(data_buf[j])
79         );
80     end
81 end
82
83 if (FixedArb) begin : gen_fixed_arbiter
84     prim_arbiter_fixed #(

```

```

85     .N(N),
86     .Dw(Dw),
87     .EnDataPort(EnDataPort)
88   ) u_arb (
89     .clk_i,
90     .rst_ni,
91     .req_i(req_buf),
92     .data_i(data_buf),
93     .gnt_o(arb_output_buf[i].gnt),
94     .idx_o(arb_output_buf[i].idx),
95     .valid_o(arb_output_buf[i].valid),
96     .data_o(arb_output_buf[i].data),
97     .ready_i
98   );
99   logic unused_req_chk;
100   assign unused_req_chk = req_chk_i;
101
102   end else begin : gen_rr_arbiter
103     prim_arbiter_tree #(
104       .N(N),
105       .Dw(Dw),
106       .EnDataPort(EnDataPort)
107     ) u_arb (
108       .clk_i,
109       .rst_ni,
110       .req_chk_i,
111       .req_i(req_buf),
112       .data_i(data_buf),
113       .gnt_o(arb_output_buf[i].gnt),
114       .idx_o(arb_output_buf[i].idx),
115       .valid_o(arb_output_buf[i].valid),
116       .data_o(arb_output_buf[i].data),
117       .ready_i
118     );
119   end
120 end
121
122 // the last buffered position is sent out
123 assign gnt_o = arb_output_buf[ArbInstances-1].gnt;
124 assign idx_o = arb_output_buf[ArbInstances-1].idx;
125 assign valid_o = arb_output_buf[ArbInstances-1].valid;
126 assign data_o = arb_output_buf[ArbInstances-1].data;
127
128 // Check the last buffer index against all other instances
129 logic [ArbInstances-2:0] output_delta;
130
131 for (genvar i = 0; i < ArbInstances-1; i++) begin :
gen_checks
132   assign output_delta[i] = arb_output_buf[ArbInstances-1] !=
arb_output_buf[i];
133 end
134
135 logic err_d, err_q;
136 // There is an error if anything ever disagrees
137 assign err_d = |output_delta;
138 always_ff @(posedge clk_i or negedge rst_ni) begin
139   if (!rst_ni) begin
140     err_q <= '0;
141   end else begin
142     err_q <= err_d | err_q;
143   end
144 end
145
146 assign err_o = err_q;
147
148 endmodule // prim_arbiter_tree

```

9. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_duplex.sv

(621 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Ascon duplex function implementation
6
7 // Ascon uses big endian encoding!
8
9 // Thus prim_ascon_duplex expects partial input blocks to fill up
from left to right.
10 // This means data_in_i[127:120] must always be set for non
empty inputs!
11
12 // TODO: Add countermeasures: mubi for control signals,
random values for state_to_round input, blinding for
output signals
13 // TODO: Add backpressure logic, if output is not ready
14 // TODO: Add check what to do, if data_in_valid_bytes_i is
greater than blocksize
15
16 module prim_ascon_duplex
17   import prim_ascon_pkg::*;
18 (
19   input logic clk_i,
20   input logic rst_ni,
21
22   input duplex_variant_e ascon_variant,

```

```

23   input duplex_op_e ascon_operation,
24
25   input logic start_i,
26   output logic done_o,
27
28   output prim_mubi_pkg::mubi4_t idle_o,
29
30   // It is assumed that no_ad, no_msg, key, and nonce are al-
ways
31   // valid and constant, when the cipher is triggered by the
start command
32   input prim_mubi_pkg::mubi4_t no_ad_i,
33   input prim_mubi_pkg::mubi4_t no_msg_i,
34
35   input logic [127:0] key_i,
36   input logic [127:0] nonce_i,
37
38   // Cipher Input Port
39   input logic [127:0] data_in_i,
40   input logic [4:0] data_in_valid_bytes_i,
41   input prim_mubi_pkg::mubi4_t last_block_ad_i,
42   input prim_mubi_pkg::mubi4_t last_block_msg_i,
43   input logic data_in_valid_i,
44   output logic data_in_ready_o,
45
46   // Cipher Output Port
47   output logic [127:0] data_out_o,
48   input logic data_out_ready_i,
49   output logic data_out_valid_o,
50
51   output logic [127:0] tag_out_o,
52   output logic tag_out_valid_o,
53
54   output duplex_fsm_state_e fsm_state_o,
55
56   output logic err_o
57 );
58
59 // TODO: Add backpressure check
60 logic unused_data_out_ready_i;
61 assign unused_data_out_ready_i = data_out_ready_i;
62
63 logic round_count_error;
64 logic sparse_fsm_error;
65
66 logic set_round_counter;
67 logic inc_round_counter;
68
69 logic [4:0][63:0] ascon_state_q, ascon_state_d;
70
71 // Ascon's 320 bit state
72 always_ff @(posedge clk_i or negedge rst_ni) begin : as-
con_state_reg
73   if (!rst_ni) begin
74     ascon_state_q <= '0;
75   end else begin
76     ascon_state_q <= ascon_state_d;
77   end
78 end
79
80 logic [319:0] state_to_round;
81 logic [319:0] state_from_round;
82
83 logic [4:0][63:0] round_to_mux;
84 assign round_to_mux = state_from_round;
85
86 duplex_fsm_state_e fsm_state_d, fsm_state_q;
87 perm_offset_e perm_offset;
88
89 assign fsm_state_o = fsm_state_q;
90
91 logic [63:0] iv;
92 assign iv = (ascon_variant == ASCON_128) ? IV_128 : IV_128A;
93
94 // internal combinatorial signals
95 prim_mubi_pkg::mubi4_t complete_block;
96
97 // TODO add a check what to do, if data_in_valid_bytes_i is
greater than blocksize
98 assign complete_block = (ascon_variant == ASCON_128 &&
data_in_valid_bytes_i == 8)
99   || (ascon_variant == ASCON_128A &&
data_in_valid_bytes_i == 16) ?
100     prim_mubi_pkg::MuBi4True :
prim_mubi_pkg::MuBi4False;
101
102 // Padding:
103 // 1) Associated Data
104 // a) empty AD: No padding, AD-processing phase is skipped!
105 // b) incomplete last block: A 10* padding is added to the
input data.
106 // The padded associated data is XORed to the state.
107 // c) complete last block: No padding to the input, but an
additional state in the FSM
108 // is used to perform the padding. A 10+ block is XORed
to the state.
109 // 2) Encryption:
110 // a) empty or incomplete last block: A 10* padding is ad-
ded to the input data.
111 // The padded associated data is XORed blockwise.
112 // b) complete last block: No padding to the input, but an

```



```

additional state in the FSM
113 // is used to perform the padding. A 10+ block is XORed
to the state.
114 // 3) Decryption:
115 // a) empty or incomplete last block: The padded (output)
PLAINTEXT is XORed to the state.
116 // This is equivalent to:
117 // The unpadded part of the Ciphertext replaces the cor-
responding part of S_r
118 // The remaining part of S_r is XORed with 10*.
119 // b) complete last block: No padding to the input, but an
additional state in the FSM
120 // is used to perform the padding. A 10+ block is XORed
to the state,
121
122 // Padding logic
123 logic [127:0] empty_padding;
124 assign empty_padding = get_padding_mask(5'b000000);
125
126 logic [127:0] valid_bytes_bit_mask;
127 assign valid_bytes_bit_mask =
bin2thermo(data_in_valid_bytes_i);
128
129 logic [127:0] padding_byte_bit_mask;
130 assign padding_byte_bit_mask = get_pad-
ding_mask(data_in_valid_bytes_i);
131
132 logic [127:0] data_in_valid_bytes;
133 assign data_in_valid_bytes = data_in_i & valid_bytes_bit_mask;
134
135 // data output
136 logic [127:0] data_out;
137 assign data_out = (data_in_i ^ {ascon_state_q[0], as-
con_state_q[1]}) & valid_bytes_bit_mask;
138
139 logic [127:0] data_in_padded;
140 logic [127:0] data_out_padded; // is only used internally for
decryption.
141
142 // For BOTH encryption AND decryption the PLAINTEXT is XORed
to the state!
143 // For encryption this is straight forward: S_r = S_r XOR P
144 // For decryption this means: S_r = S_r XOR P = S_r XOR (S_r
XOR C) = C
145 // Thus the ciphertext replaces the rate. However, we cannot
simply implement
146 // S_r = C, because this does not work for incomplete blocks,
where only a part
147 // of the rate is replaced by C and the other part remains:
148 // S_r = S_l || S_r(r-l) = C_l || S_r(r-l)
149 // Therefore, it is easier to always XOR the padded plaintext
150
151 always_comb begin
152 if (prim_mubi_pkg::mubi4_test_true_strict(complete_block))
begin
153 data_in_padded = data_in_valid_bytes;
154 data_out_padded = data_out;
155 end else begin
156 data_in_padded = data_in_valid_bytes | pad-
ding_byte_bit_mask;
157 data_out_padded = data_out | padding_byte_bit_mask;
158 end
159 end
160
161 // TODO add blinding
162 assign data_out_o = data_out;
163 assign tag_out_o = ({ascon_state_q[3], ascon_state_q[4]} ^
key_i);
164
165 // Due to Ascon's round constants the current_round
166 // contains an offset:
167 // for P12 we count: from 0 to 11 = 12 rounds,
168 // for P8 we count: from 4 to 11 = 8 rounds,
169 // for P6 we count: from 5 to 11 = 6 rounds
170 logic [AsconRoundCountW-1:0] current_round;
171
172 prim_count #(
173 .Width(AsconRoundCountW)
174 ) u_round_counter (
175 .clk_i,
176 .rst_ni,
177 .clr_i(1'b0),
178 .set_i(set_round_counter),
179 .set_cnt_i(perm_offset),
180 .incr_en_i(inc_round_counter),
181 .decr_en_i(1'b0),
182 .step_i(AsconRoundCountW'(1)),
183 .commit_i(1'b1),
184 .cnt_o(current_round),
185 .cnt_after_commit_o(),
186 .err_o(round_count_error)
187 );
188
189 // Duplexswitch muxes
190 // Selects data input source
191 padding_mux_e sel_padding;
192
193 // Main multiplexer per word
194 ascon_word_mux_e sel_mux_word0;
195 ascon_word_mux_e sel_mux_word1;
196 ascon_word_mux_e sel_mux_word2;

```

```

197 ascon_word_mux_e sel_mux_word3;
198 ascon_word_mux_e sel_mux_word4;
199
200 // Multiplexer before XOR
201 word_low_key_hi_mux_e sel_mux_key_word1;
202 key_hi_low_mux_e sel_mux_key_word2;
203 key_hi_low_mux_e sel_mux_key_word3;
204
205 // Selects round input
206 ascon_round_input_mux_e sel_round_input;
207
208 // Set domain separation
209 logic set_dom_sep;
210
211 // Intermediate signals
212 logic [4:0][63:0] xor_with_state;
213 logic [63:0] word4_dom_sep;
214 logic [127:0] data_to_duplex;
215
216 // Enc Dec Padding Mux
217 always_comb begin : Padding
218 unique case (sel_padding)
219 DATA_IN_PAD: data_to_duplex = data_in_padded;
220 DATA_OUT_PAD: data_to_duplex = data_out_padded;
221 EMPTY_PAD: data_to_duplex = empty_padding;
222 default: data_to_duplex = empty_padding;
223 endcase
224 end
225
226 // State input mux
227 always_comb begin : state_input
228 unique case (sel_round_input)
229 STATE: state_to_round = ascon_state_q;
230 BLINDING: state_to_round = '0;
231 default: state_to_round = '0;
232 endcase
233 end
234
235 // Main mux word0
236 assign xor_with_state[0] = data_to_duplex[127:64];
237 always_comb begin : state_word0
238 unique case (sel_mux_word0)
239 INIT: ascon_state_d[0] = iv;
240 ABSORB: ascon_state_d[0] = ascon_state_q[0] ^
xor_with_state[0];
241 KEEP: ascon_state_d[0] = ascon_state_q[0];
242 ROUND: ascon_state_d[0] = round_to_mux[0];
243 default: ascon_state_d[0] = ascon_state_q[0];
244 endcase
245 end
246
247 // Select key high or data low for XOR
248 always_comb begin : key_word1
249 unique case (sel_mux_key_word1)
250 WORD: xor_with_state[1] = data_to_duplex[63:0];
251 KEY: xor_with_state[1] = key_i[127:64];
252 default: xor_with_state[1] = key_i[127:64];
253 endcase
254 end
255
256 // Main mux word1
257 always_comb begin : state_word1
258 unique case (sel_mux_word1)
259 INIT: ascon_state_d[1] = key_i[127:64];
260 ABSORB: ascon_state_d[1] = ascon_state_q[1] ^
xor_with_state[1];
261 KEEP: ascon_state_d[1] = ascon_state_q[1];
262 ROUND: ascon_state_d[1] = round_to_mux[1];
263 default: ascon_state_d[1] = ascon_state_q[1];
264 endcase
265 end
266
267 // Select key high or low for XOR
268 always_comb begin : key_word2
269 unique case (sel_mux_key_word2)
270 KEY_LOW: xor_with_state[2] = key_i[63:0];
271 KEY_HI: xor_with_state[2] = key_i[127:64];
272 default: xor_with_state[2] = key_i[127:64];
273 endcase
274 end
275
276 // Main mux word2
277 always_comb begin : state_word2
278 unique case (sel_mux_word2)
279 INIT: ascon_state_d[2] = key_i[63:0];
280 ABSORB: ascon_state_d[2] = ascon_state_q[2] ^
xor_with_state[2];
281 KEEP: ascon_state_d[2] = ascon_state_q[2];
282 ROUND: ascon_state_d[2] = round_to_mux[2];
283 default: ascon_state_d[2] = ascon_state_q[2];
284 endcase
285 end
286
287 // Select key high or low for XOR
288 always_comb begin : key_word3
289 unique case (sel_mux_key_word3)
290 KEY_LOW: xor_with_state[3] = key_i[63:0];
291 KEY_HI: xor_with_state[3] = key_i[127:64];
292 default: xor_with_state[3] = key_i[127:64];
293 endcase
294 end

```

```

295
296 // Main mux word3
297 always_comb begin : state_word3
298   unique case (sel_mux_word3)
299     INIT:   ascon_state_d[3] = nonce_i[127:64];
300     ABSORB: ascon_state_d[3] = ascon_state_q[3] ^
xor_with_state[3];
301     KEEP:   ascon_state_d[3] = ascon_state_q[3];
302     ROUND:  ascon_state_d[3] = round_to_mux[3];
303     default: ascon_state_d[3] = ascon_state_q[3];
304   endcase
305 end
306
307 // Set domain separation bit
308 assign word4_dom_sep = {ascon_state_q[4][63:1], as-
con_state_q[4][0] ^ set_dom_sep};
309 // For word 4 only the lower part of the key can be xored to
the state
310 assign xor_with_state[4] = key_i[63:0];
311
312 // Main mux word4
313 always_comb begin : state_word4
314   unique case (sel_mux_word4)
315     INIT:   ascon_state_d[4] = nonce_i[63:0];
316     ABSORB: ascon_state_d[4] = word4_dom_sep ^
xor_with_state[4];
317     KEEP:   ascon_state_d[4] = word4_dom_sep;
318     ROUND:  ascon_state_d[4] = round_to_mux[4];
319     default: ascon_state_d[4] = word4_dom_sep;
320   endcase
321 end
322
323 always_comb begin : p_fsm
324   // Default assignments
325   fsm_state_d = fsm_state_q;
326   data_in_ready_o = 1'b0;
327   data_out_valid_o = 1'b0;
328   tag_out_valid_o = 1'b0;
329   sparse_fsm_error = 1'b0;
330   set_round_counter = 1'b0;
331   inc_round_counter = 1'b0;
332   perm_offset = P12;
333   done_o = 1'b0;
334   idle_o = prim_mubi_pkg::MuBi4False;
335
336   // Default: Don't update state
337   set_dom_sep = 1'b0;
338   sel_mux_word0 = KEEP;
339   sel_mux_word1 = KEEP;
340   sel_mux_word2 = KEEP;
341   sel_mux_word3 = KEEP;
342   sel_mux_word4 = KEEP;
343
344   sel_mux_key_word3 = KEY_HI;
345   sel_mux_key_word2 = KEY_LOW;
346   sel_mux_key_word1 = KEY;
347
348   sel_padding = EMPTY_PAD;
349
350   sel_round_input = BLINDING;
351
352   unique case (fsm_state_q)
353     Idle: begin
354       idle_o = prim_mubi_pkg::MuBi4True;
355       if (start_i) begin
356         fsm_state_d = Init;
357       end
358     end
359     Init: begin
360       sel_mux_word0 = INIT;
361       sel_mux_word1 = INIT;
362       sel_mux_word2 = INIT;
363       sel_mux_word3 = INIT;
364       sel_mux_word4 = INIT;
365       fsm_state_d = PermInit;
366       perm_offset = P12;
367       set_round_counter = 1'b1;
368     end
369     PermInit: begin
370       sel_round_input = STATE;
371       sel_mux_word0 = ROUND;
372       sel_mux_word1 = ROUND;
373       sel_mux_word2 = ROUND;
374       sel_mux_word3 = ROUND;
375       sel_mux_word4 = ROUND;
376       if (current_round == ROUND_MAX) begin
377         fsm_state_d = Xor0Key;
378       end else begin
379         inc_round_counter = 1'b1;
380       end
381     end
382     Xor0Key: begin
383       sel_mux_word3 = ABSORB;
384       sel_mux_key_word3 = KEY_HI;
385       sel_mux_word4 = ABSORB;
386       if (prim_mubi_pkg::mubi4_test_true_strict(no_ad_i)) be-
gin
387         fsm_state_d = XorDomSep;
388       end else begin
389         fsm_state_d = AbsorbAD;
390       end

```

```

391   end
392   AbsorbAD: begin
393     // There will be AD, otherwise we wouldn't be in this
state/path
394     data_in_ready_o = 1'b1;
395     if (data_in_valid_i) begin
396       sel_mux_word0 = ABSORB;
397       sel_padding = DATA_IN_PAD;
398       if (ascon_variant == ASCON_128A) begin
399         sel_mux_word1 = ABSORB;
400         sel_mux_key_word1 = WORD;
401       end
402       if
(prim_mubi_pkg::mubi4_test_true_strict(last_block_ad_i)) begin
403         if
(prim_mubi_pkg::mubi4_test_true_strict(complete_block)) begin
404           fsm_state_d = PermADEmpty;
405         end else begin
406           fsm_state_d = PermADLast;
407         end
408       end else begin // there are more blocks to come
409         fsm_state_d = PermAD;
410       end
411     end
412     if (ascon_variant == ASCON_128) begin
413       perm_offset = P6;
414     end else begin // ASCON_128A
415       perm_offset = P8;
416     end
417     set_round_counter = 1'b1;
418   end
419   PermAD: begin
420     sel_round_input = STATE;
421     sel_mux_word0 = ROUND;
422     sel_mux_word1 = ROUND;
423     sel_mux_word2 = ROUND;
424     sel_mux_word3 = ROUND;
425     sel_mux_word4 = ROUND;
426     if (current_round == ROUND_MAX) begin
427       fsm_state_d = AbsorbAD;
428     end else begin
429       inc_round_counter = 1'b1;
430     end
431   end
432   PermADLast: begin
433     sel_round_input = STATE;
434     sel_mux_word0 = ROUND;
435     sel_mux_word1 = ROUND;
436     sel_mux_word2 = ROUND;
437     sel_mux_word3 = ROUND;
438     sel_mux_word4 = ROUND;
439     if (current_round == ROUND_MAX) begin
440       fsm_state_d = XorDomSep;
441     end else begin
442       inc_round_counter = 1'b1;
443     end
444   end
445   PermADEmpty: begin
446     sel_round_input = STATE;
447     sel_mux_word0 = ROUND;
448     sel_mux_word1 = ROUND;
449     sel_mux_word2 = ROUND;
450     sel_mux_word3 = ROUND;
451     sel_mux_word4 = ROUND;
452     if (current_round == ROUND_MAX) begin
453       fsm_state_d = AbsorbADEmpty;
454     end else begin
455       inc_round_counter = 1'b1;
456     end
457   end
458   AbsorbADEmpty: begin
459     sel_mux_word0 = ABSORB;
460     sel_padding = EMPTY_PAD;
461     fsm_state_d = PermADLast;
462     if (ascon_variant == ASCON_128) begin
463       perm_offset = P6;
464     end else begin // ASCON_128A
465       perm_offset = P8;
466       // This should be optimized by the tool.
467       // It is left here, so that the structure of the case
468       // is the same as AbsorbAD.
469       sel_mux_word1 = ABSORB;
470       sel_padding = EMPTY_PAD;
471     end
472     set_round_counter = 1'b1;
473   end
474   XorDomSep: begin
475     sel_dom_sep = 1'b1;
476     sel_mux_word4 = KEEP;
477     if (prim_mubi_pkg::mubi4_test_true_strict(no_msg_i)) be-
gin
478       fsm_state_d = AbsorbMSGEmpty;
479     end else begin
480       fsm_state_d = AbsorbMSG;
481     end
482   end
483   AbsorbMSG: begin
484     data_in_ready_o = 1'b1;
485     if (data_in_valid_i) begin
486       data_out_valid_o = 1'b1;
487       if (ascon_operation == ASCON_ENC) begin

```

```

488     sel_mux_word0 = ABSORB;
489     sel_padding = DATA_IN_PAD;
490     if (ascon_variant == ASCON_128A) begin
491         sel_mux_word1 = ABSORB;
492     end
493 end else begin // ASCON_DEC
494     sel_mux_word0 = ABSORB;
495     sel_padding = DATA_OUT_PAD;
496     if (ascon_variant == ASCON_128A) begin
497         sel_mux_word1 = ABSORB;
498         sel_mux_key_word1 = WORD;
499     end
500 end
501 if
502 (prim_mubi_pkg::mubi4_test_true_strict(last_block_msg_i)) begin
503     if
504 (prim_mubi_pkg::mubi4_test_true_strict(complete_block)) begin // we
505 need extra padding
506         fsm_state_d = PermMSGEmpty;
507     end else begin // padding is done on the fly
508         fsm_state_d = XORKey0;
509     end
510 end else begin
511     fsm_state_d = PermMSG;
512 end
513 end
514 if (ascon_variant == ASCON_128) begin
515     perm_offset = P6;
516 end else begin //ASCON_128A
517     perm_offset = P8;
518 end
519 set_round_counter = 1'b1;
520 end
521 PermMSG: begin
522     sel_round_input = STATE;
523     sel_mux_word0 = ROUND;
524     sel_mux_word1 = ROUND;
525     sel_mux_word2 = ROUND;
526     sel_mux_word3 = ROUND;
527     sel_mux_word4 = ROUND;
528     if (current_round == ROUND_MAX) begin
529         fsm_state_d = AbsorbMSG;
530     end else begin
531         inc_round_counter = 1'b1;
532     end
533 end
534 end
535 PermMSGEmpty: begin
536     sel_round_input = STATE;
537     sel_mux_word0 = ROUND;
538     sel_mux_word1 = ROUND;
539     sel_mux_word2 = ROUND;
540     sel_mux_word3 = ROUND;
541     sel_mux_word4 = ROUND;
542     if (current_round == ROUND_MAX) begin
543         fsm_state_d = AbsorbMSGEmpty;
544     end else begin
545         inc_round_counter = 1'b1;
546     end
547 end
548 end
549 AbsorbMSGEmpty: begin
550     // The padding for an empty block is the same for en-
551 crypton and decryption.
552     sel_padding = EMPTY_PAD;
553     sel_mux_word0 = ABSORB;
554     if (ascon_variant == ASCON_128A) begin
555         // This should be optimized by the tool.
556         // It is left here, so that the structure of the case
557 // is the same as AbsorbAD.
558         sel_mux_word1 = ABSORB;
559         sel_mux_key_word1 = WORD;
560     end
561     fsm_state_d = XORKey0;
562 end
563 end
564 XORKey0: begin
565     if (ascon_variant == ASCON_128) begin
566         sel_mux_word1 = ABSORB;
567         sel_mux_key_word1 = KEY;
568         sel_mux_word2 = ABSORB;
569         sel_mux_key_word2 = KEY_LOW;
570     end else begin //ASCON_128a
571         sel_mux_word2 = ABSORB;
572         sel_mux_key_word2 = KEY_HI;
573         sel_mux_word3 = ABSORB;
574         sel_mux_key_word3 = KEY_LOW;
575     end
576     fsm_state_d = PermFinal;
577     set_round_counter = 1'b1;
578     perm_offset = P12;
579 end
580 end
581 PermFinal: begin
582     sel_round_input = STATE;
583     sel_mux_word0 = ROUND;
584     sel_mux_word1 = ROUND;
585     sel_mux_word2 = ROUND;
586     sel_mux_word3 = ROUND;
587     sel_mux_word4 = ROUND;
588     if (current_round == ROUND_MAX) begin
589         fsm_state_d = SqueezeTagXorKey;
590     end else begin
591         inc_round_counter = 1'b1;

```

```

585     end
586 end
587 SqueezeTagXorKey: begin
588     tag_out_valid_o = 1'b1;
589     fsm_state_d = Idle;
590     done_o = 1'b1;
591 end
592 Error: begin
593     fsm_state_d = Error;
594     sparse_fsm_error = 1'b1;
595 end
596 default: begin
597     fsm_state_d = Error;
598     sparse_fsm_error = 1'b1;
599 end
600 endcase
601 end
602
603 `PRIM_FLOP_SPARSE_FSM(u_state_regs, fsm_state_d, fsm_state_q,
604 duplex_fsm_state_e, Idle)
605
606 logic mubi_error;
607 assign mubi_error = prim_mubi_pkg::mubi4_test_invalid(no_ad_i)
608 |
609 prim_mubi_pkg::mubi4_test_invalid(no_msg_i)
610 |
611 prim_mubi_pkg::mubi4_test_invalid(complete_block)
612 |
613 prim_mubi_pkg::mubi4_test_invalid(last_block_ad_i)
614 |
615 prim_mubi_pkg::mubi4_test_invalid(last_block_msg_i);
616
617 assign err_o = round_count_error | sparse_fsm_error | mubi_er-
618 ror;
619
620 prim_ascon_round u_prim_ascon_round (
621     .state_o(state_from_round),
622     .state_i(state_to_round),
623     .rcon_i(get_ascon_rcon(current_round))
624 );
625
626 endmodule

```

10. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_pkg.sv (192 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5
6 package prim_ascon_pkg;
7
8 parameter int AsconRoundCountW = 4;
9
10 // Due to Ascon's round constants the current_round
11 // contains an offset:
12 // for P12 we count: from 0 to 11 = 12 rounds,
13 // for P8 we count: from 4 to 11 = 8 rounds,
14 // for P6 we count: from 5 to 11 = 6 rounds
15
16 typedef enum logic [3:0] {
17     P12 = 4'b0000,
18     P8 = 4'b0100,
19     P6 = 4'b0110
20 } perm_offset_e;
21
22 parameter int DUPLEX_OP_WIDTH = 3;
23
24 typedef enum logic [DUPLEX_OP_WIDTH-1:0] {
25     ASCON_ENC = 3'b001,
26     ASCON_DEC = 3'b010,
27     ASCON_HASH = 3'b100
28 } duplex_op_e;
29
30 parameter int DUPLEX_VARIANT_WIDTH = 2;
31
32 typedef enum logic [DUPLEX_VARIANT_WIDTH-1:0] {
33     ASCON_128 = 2'b01,
34     ASCON_128A = 2'b10
35 } duplex_variant_e;
36
37 localparam logic [63:0] IV_128 = 64'h80400c0600000000;
38 localparam logic [63:0] IV_128A = 64'h80800c0800000000;
39
40 localparam logic [3:0] ROUND_MAX = 4'b1011;
41
42 // Round constants
43 function automatic logic [7:0] get_ascon_rcon(logic [3:0]
44 round);
45     logic [7:0] result;
46     unique case (round)
47         4'b0000: result = 8'hf0;
48         4'b0001: result = 8'he1;
49         4'b0010: result = 8'hd2;
50         4'b0011: result = 8'hc3;
51         4'b0100: result = 8'hb4;
52         4'b0101: result = 8'ha5;

```

```

51 4'b0110: result = 8'h96;
52 4'b0111: result = 8'h87;
53 4'b1000: result = 8'h78;
54 4'b1001: result = 8'h69;
55 4'b1010: result = 8'h5a;
56 4'b1011: result = 8'h4b;
57 default: result = 8'h00;
58 endcase
59
60 return result;
61 endfunction
62
63 parameter int KEY_HI_LOW_MUX_WIDTH = 1;
64 typedef enum logic [KEY_HI_LOW_MUX_WIDTH-1:0] {
65   KEY_LOW = 1'b0,
66   KEY_HI  = 1'b1
67 } key_hi_low_mux_e;
68
69 parameter int WORD_LOW_KEY_HI_MUX_WIDTH = 1;
70 typedef enum logic [WORD_LOW_KEY_HI_MUX_WIDTH-1:0] {
71   WORD = 1'b0,
72   KEY  = 1'b1
73 } word_low_key_hi_mux_e;
74
75 parameter int ASCON_WORD_MUX_WIDTH = 2;
76 typedef enum logic [ASCON_WORD_MUX_WIDTH-1:0] {
77   INIT   = 2'b00,
78   ABSORB = 2'b01,
79   KEEP   = 2'b10,
80   ROUND  = 2'b11
81 } ascon_word_mux_e;
82
83 parameter int ROUND_INPUT_MUX_WIDTH = 1;
84 typedef enum logic [ROUND_INPUT_MUX_WIDTH-1:0] {
85   STATE = 1'b0,
86   BLINDING = 1'b1
87 } ascon_round_input_mux_e;
88
89 parameter int PADDING_MUX_WIDTH = 2;
90 typedef enum logic [PADDING_MUX_WIDTH-1:0] {
91   DATA_IN_PAD = 2'b00,
92   DATA_OUT_PAD = 2'b01,
93   EMPTY_PAD    = 2'b10
94 } padding_mux_e;
95
96 // Encoding generated with:
97 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 18 -n 10 \
98 //   -s 3538518573 --language=sv
99 //
100 // Hamming distance histogram:
101 //
102 // 0: --
103 // 1: --
104 // 2: --
105 // 3: ||| (13.73%)
106 // 4: ||| (20.92%)
107 // 5: ||| (24.84%)
108 // 6: ||| (20.92%)
109 // 7: ||| (14.38%)
110 // 8: ||| (5.23%)
111 // 9: --
112 // 10: --
113 //
114 // Minimum Hamming distance: 3
115 // Maximum Hamming distance: 8
116 // Minimum Hamming weight: 3
117 // Maximum Hamming weight: 8
118 //
119 localparam int AsconDuplexFSMStateWidth = 10;
120 typedef enum logic [AsconDuplexFSMStateWidth-1:0] {
121   Idle      = 10'b101010110,
122   Init      = 10'b0110000110,
123   PermInit  = 10'b1011110110,
124   Xor0Key   = 10'b0001010011,
125   AbsorbAD  = 10'b1110010000,
126   PermAD    = 10'b1101101101,
127   XorDomSep = 10'b0100110010,
128   AbsorbMSG = 10'b0011100011,
129   PermMSG   = 10'b0001001100,
130   AbsorbMSGEmpty = 10'b0100101100,
131   XorKey0   = 10'b1000011011,
132   PermFinal = 10'b0011010000,
133   SqueezeTagXorKey = 10'b1111010111,
134   PermADLast = 10'b1010000010,
135   AbsorbADEmpty = 10'b1101111011,
136   PermMSGEmpty = 10'b1011111000,
137   PermADEmpty = 10'b0010111100,
138   Error      = 10'b0100011110
139 } duplex_fsm_state_e;
140
141 function automatic logic [127:0] bin2thermo(logic [4:0] val_id_bytes);
142   logic [127:0] valid_bytes_mask;
143   unique case (valid_bytes)
144     5'b00000 : valid_bytes_mask = {128{1'b0}};
145     5'b00001 : valid_bytes_mask = { 8{1'b1}}, {120{1'b0}};
146     5'b00010 : valid_bytes_mask = {16{1'b1}}, {112{1'b0}};
147     5'b00011 : valid_bytes_mask = {24{1'b1}}, {104{1'b0}};
148     5'b00100 : valid_bytes_mask = {32{1'b1}}, {96{1'b0}};
149     5'b00101 : valid_bytes_mask = {40{1'b1}}, {88{1'b0}};
150     5'b00110 : valid_bytes_mask = {48{1'b1}}, {80{1'b0}};

```

```

151 5'b00111 : valid_bytes_mask = {56{1'b1}}, {72{1'b0}};
152 5'b01000 : valid_bytes_mask = {64{1'b1}}, {64{1'b0}};
153 5'b01001 : valid_bytes_mask = {72{1'b1}}, {56{1'b0}};
154 5'b01010 : valid_bytes_mask = {80{1'b1}}, {48{1'b0}};
155 5'b01011 : valid_bytes_mask = {88{1'b1}}, {40{1'b0}};
156 5'b01100 : valid_bytes_mask = {96{1'b1}}, {32{1'b0}};
157 5'b01101 : valid_bytes_mask = {104{1'b1}}, {24{1'b0}};
158 5'b01110 : valid_bytes_mask = {112{1'b1}}, {16{1'b0}};
159 5'b01111 : valid_bytes_mask = {120{1'b1}}, {8{1'b0}};
160 default : valid_bytes_mask = {128{1'b1}};
161 endcase
162
163 return valid_bytes_mask;
164 endfunction
165
166 function automatic logic [127:0] get_padding_mask(logic [4:0]
valid_bytes);
167   logic [127:0] padding_byte_mask;
168   unique case (valid_bytes)
169     5'b00000 : padding_byte_mask = {8'h80,
170 {120{1'b0}}};
171     5'b00001 : padding_byte_mask = { 8{1'b0}}, 8'h80,
172 {112{1'b0}};
173     5'b00010 : padding_byte_mask = {16{1'b0}}, 8'h80,
174 {104{1'b0}};
175     5'b00011 : padding_byte_mask = {24{1'b0}}, 8'h80,
176 {96{1'b0}};
177     5'b00100 : padding_byte_mask = {32{1'b0}}, 8'h80,
178 {88{1'b0}};
179     5'b00101 : padding_byte_mask = {40{1'b0}}, 8'h80,
180 {80{1'b0}};
181     5'b00110 : padding_byte_mask = {48{1'b0}}, 8'h80,
182 {72{1'b0}};
183     5'b00111 : padding_byte_mask = {56{1'b0}}, 8'h80,
184 {64{1'b0}};
185     5'b01000 : padding_byte_mask = {64{1'b0}}, 8'h80,
186 {56{1'b0}};
187     5'b01001 : padding_byte_mask = {72{1'b0}}, 8'h80,
188 {48{1'b0}};
189     5'b01010 : padding_byte_mask = {80{1'b0}}, 8'h80,
190 {40{1'b0}};
191     5'b01011 : padding_byte_mask = {88{1'b0}}, 8'h80,
192 {32{1'b0}};
193     5'b01100 : padding_byte_mask = {96{1'b0}}, 8'h80,
194 {24{1'b0}};
195     5'b01101 : padding_byte_mask = {104{1'b0}}, 8'h80,
196 {16{1'b0}};
197     5'b01110 : padding_byte_mask = {112{1'b0}}, 8'h80,
198 {8{1'b0}};
199     5'b01111 : padding_byte_mask = {120{1'b0}},
8'h80
200 };
201 default : padding_byte_mask = {128{1'b0}};
202 endcase
203
204 return padding_byte_mask;
205 endfunction
206
207 endpackage : prim_ascon_pkg

```

11. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_round.sv (68 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Ascon round function implementation
6
7 module prim_ascon_round (
8   output logic [319:0] state_o,
9   input logic [319:0] state_i,
10  input logic [7:0] rcon_i
11 );
12
13 logic [319:0] ark_w, sbow_w;
14
15 // Add round constant
16 assign ark_w = state_i ^ {64'h0, 64'h0, 56'h0, rcon_i,
64'h0, 64'h0};
17
18 // Substitution layer
19 logic [63:0] x_w [5];
20 logic [4:0] xtranspose_w [64];
21 logic [4:0] ytranspose_w [64];
22 logic [63:0] y_w [5];
23
24 assign x_w[0] = ark_w[63:0];
25 assign x_w[1] = ark_w[127:64];
26 assign x_w[2] = ark_w[191:128];
27 assign x_w[3] = ark_w[255:192];
28 assign x_w[4] = ark_w[319:256];
29
30 for (genvar i = 0; i < 64; i = i + 1) begin : gen_s-
box_transpose
31   assign xtranspose_w[i] = {x_w[4][i],
32                               x_w[3][i],

```

```

34         x_w[2][i],
35         x_w[1][i],
36         x_w[0][i]);
37     prim_ascon_sbox u_sbox (
38         .state_i(xtranspose_w[i]),
39         .state_o(ytranspose_w[i])
40     );
41     assign {y_w[4][i],
42            y_w[3][i],
43            y_w[2][i],
44            y_w[1][i],
45            y_w[0][i]} = ytranspose_w[i];
46 end
47
48 assign sbox_w = {y_w[4], y_w[3], y_w[2], y_w[1], y_w[0]};
49
50 // Linear layer
51 logic [63:0] xl_w [5];
52 logic [63:0] yl_w [5];
53
54 assign xl_w[0] = sbox_w[ 63: 0];
55 assign xl_w[1] = sbox_w[127: 64];
56 assign xl_w[2] = sbox_w[191:128];
57 assign xl_w[3] = sbox_w[255:192];
58 assign xl_w[4] = sbox_w[319:256];
59
60 assign yl_w[0] = xl_w[0] ^ {xl_w[0][18:0], xl_w[0][63:19]} ^
{xl_w[0][27:0], xl_w[0][63:28]};
61 assign yl_w[1] = xl_w[1] ^ {xl_w[1][60:0], xl_w[1][63:61]} ^
{xl_w[1][38:0], xl_w[1][63:39]};
62 assign yl_w[2] = xl_w[2] ^ {xl_w[2][ 0], xl_w[2][63: 1]} ^
{xl_w[2][ 5:0], xl_w[2][63: 6]};
63 assign yl_w[3] = xl_w[3] ^ {xl_w[3][ 9:0], xl_w[3][63:10]} ^
{xl_w[3][16:0], xl_w[3][63:17]};
64 assign yl_w[4] = xl_w[4] ^ {xl_w[4][ 6:0], xl_w[4][63: 7]} ^
{xl_w[4][40:0], xl_w[4][63:41]};
65
66 assign state_o = {yl_w[4], yl_w[3], yl_w[2], yl_w[1],
yl_w[0]};
67
68 endmodule

```

12. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_sbox.sv (35 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Ascon SBox unprotected implementation
6
7 module prim_ascon_sbox (
8     output logic [4:0] state_o,
9     input  logic [4:0] state_i
10 );
11
12     logic [4:0] temp_w [4];
13
14     assign temp_w[0] = state_i;
15     assign temp_w[1] = {temp_w[0][3] ^ temp_w[0][4],
16                        temp_w[0][3],
17                        temp_w[0][1] ^ temp_w[0][2],
18                        temp_w[0][1],
19                        temp_w[0][0] ^ temp_w[0][4]
20                        };
21     assign temp_w[2] = {temp_w[1][4] ^ ((~temp_w[1][0]) &
temp_w[1][1]),
22                        temp_w[1][3] ^ ((~temp_w[1][4]) &
temp_w[1][0]),
23                        temp_w[1][2] ^ ((~temp_w[1][3]) &
temp_w[1][4]),
24                        temp_w[1][1] ^ ((~temp_w[1][2]) &
temp_w[1][3]),
25                        temp_w[1][0] ^ ((~temp_w[1][1]) &
temp_w[1][2])
26                        };
27     assign temp_w[3] = {temp_w[2][4],
28                        temp_w[2][3] ^ temp_w[2][2],
29                        ~temp_w[2][2],
30                        temp_w[2][1] ^ temp_w[2][0],
31                        temp_w[2][0] ^ temp_w[2][4]
32                        };
33     assign state_o = temp_w[3];
34
35 endmodule

```

13. lowRISC_opentitan/hw/ip/prim/rtl/prim_assert.sv (190 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.

```

```

3 // SPDX-License-Identifier: Apache-2.0
4
5 // Macros and helper code for using assertions.
6 // - Provides default clk and rst options to simplify code
7 // - Provides boiler plate template for common assertions
8
9 `ifndef PRIM_ASSERT_SV
10 `define PRIM_ASSERT_SV
11
12 // Helper macros //
13 // Helper macros //
14 // Helper macros //
15
16 // Default clk and reset signals used by assertion macros be-
low.
17 `define ASSERT_DEFAULT_CLK clk_i
18 `define ASSERT_DEFAULT_RST !rst_ni
19
20 // Converts an arbitrary block of code into a Verilog string
21 `define PRIM_STRINGIFY(x) "`x"
22
23 // ASSERT_ERROR logs an error message with either `uvm_error
or with $error.
24 //
25 // This somewhat duplicates `DV_ERROR macro defined in hw/dv/
sv/dv_utils/dv_macros.svh. The reason
26 // for redefining it here is to avoid creating a dependency.
27 `define AS-
SERT_ER-
ROR(
name)
28 `ifdef
UVM
29     `uvm_pkg::uvm_report_error("ASSERT FAILED", `PRIM_STRINGI-
FY(name), uvm_pkg::UVM_NONE, \
30         `FILE_, `LINE_, "",
31         1);
32 `else
33     $error("%0t: (%0s:%0d) [%m] [ASSERT FAILED] %0s", $time,
`FILE_, `LINE_, \
34         `PRIM_STRINGI-
FY(name));
35 `endif
36
37 // This macro is suitable for conditionally triggering lint
errors, e.g., if a Sec parameter takes
38 // on a non-default value. This may be required for pre-silic-
on/FPGA evaluation but we don't want
39 // to allow this for tapeout.
40 `define ASSERT_STATIC_LINT_ERROR(name, __prop) \
41     localparam int __name = (__prop) ? 1 : 2; \
42     always_comb begin \
43         logic unused_assert_static_lint_error; \
44         unused_assert_static_lint_error = __name'(1'b1); \
45     end
46
47 // Static assertions for checks inside SV packages. If the
conditions is not true, this will
48 // trigger an error during elaboration.
49 `define ASSERT_STATIC_IN_PACKAGE(name, __prop) \
50     function automatic bit assert_static_in_package_`__name(); \
51     bit unused_bit [((__prop) ? 1 : -1)]; \
52     unused_bit = '{default: 1'b0}; \
53     return unused_bit[0]; \
54 endfunction
55
56 // The basic helper macros are actually defined in "implement-
ation headers". The macros should do
57 // the same thing in each case (except for the dummy flavour),
but in a way that the respective
58 // tools support.
59 //
60 // If the tool supports assertions in some form, we also
define INC_ASSERT (which can be used to
61 // hide signal definitions that are only used for assertions).
62 //
63 // The list of basic macros supported is:
64 // ASSERT_I: Immediate assertion. Note that immediate as-
sertions are sensitive to simulation
65 // glitches.
66 //
67 // ASSERT_INIT: Assertion in initial block. Can be used for
things like parameter checking.
68 //
69 // ASSERT_INIT_NET: Assertion in initial block. Can be used
for initial value of a net.
70 //
71 // ASSERT_FINAL: Assertion in final block. Can be used for
things like queues being empty at end of
72 // sim, all credits returned at end of sim,
state machines in idle at end of sim.

```



```

73 //
74 // ASSERT_AT_RESET: Assertion just before reset. Can be used
to check sum-like properties that get
75 // cleared at reset.
76 // Note that unless your simulation ends
with a reset, the property does not get
77 // checked at end of simulation; use AS-
SERT_AT_RESET_AND_FINAL if the property
78 // should also get checked at end of simula-
tion.
79 //

80 // ASSERT_AT_RESET_AND_FINAL: Assertion just before reset and in
final block. Can be used to check
81 // sum-like properties before
every reset and at the end of simulation.
82 //
83 // ASSERT: Assert a concurrent property directly. It
can be called as a module (or
84 // interface) body item.
85 //
86 // Note: We use (__rst != '0') in the disable
iff statements instead of (__rst ==

87 // '1). This properly disables the assertion in
cases when reset is X at the
88 // beginning of a simulation. For that case,
(reset == '1) does not disable the
89 // assertion.
90 //
91 // ASSERT_NEVER: Assert a concurrent property NEVER happens
92 //
93 // ASSERT_KNOWN: Assert that signal has a known value (each
bit is either '0' or '1') after reset.
94 // It can be called as a module (or interface)
body item.
95 //
96 // COVER: Cover a concurrent property
97 //
98 // ASSUME: Assume a concurrent property
99 //
100 // ASSUME_I: Assume an immediate property
101 //
102 `ifdef VERILATOR
103 `include "prim_assert_dummy_macros.svh"
104 `elsif SYNTHESIS
105 `include "prim_assert_dummy_macros.svh"
106 `elsif YOSYS
107 `include "prim_assert_yosys_macros.svh"
108 `define INC_ASSERT
109 `else
110 `include "prim_assert_standard_macros.svh"
111 `define INC_ASSERT
112 `endif
113
114 //////////////////////////////////////////////////
115 // Complex assertion macros //
116 //////////////////////////////////////////////////
117
118 // Assert that signal is an active-high pulse with pulse
length of 1 clock cycle
119 `define ASSERT_PULSE(__name, __sig, __clk = `AS-
SERT_DEFAULT_CLK, __rst = `ASSERT_DEFAULT_RST) \
120 `ASSERT(__name, $rose(__sig) | => !(__sig), __clk, __rst)
121
122 // Assert that a property is true only when an enable signal
is set. It can be called as a module
123 // (or interface) body item.
124 `define ASSERT_IF(__name, __prop, __enable, __clk = `AS-
SERT_DEFAULT_CLK, __rst = `ASSERT_DEFAULT_RST) \
125 `ASSERT(__name, (__enable) | -> (__prop), __clk, __rst)
126
127 // Assert that signal has a known value (each bit is either
'0' or '1') after reset is known
128 // set. It can be called as a module (or interface) body
item.
129 `define ASSERT_KNOWN_IF(__name, __sig, __enable, __clk = `AS-
SERT_DEFAULT_CLK, __rst = `ASSERT_DEFAULT_RST) \
130 `ifdef
FPV_ON
131 `ASSERT_KNOWN(__name`KnownEnable, __enable, __clk,
__rst)
132 `ASSERT_IF(__name, !$isunknown(__sig), __enable, __clk,
__rst)
133 `endif
134
135 //////////////////////////////////////////////////
136 // For formal verification only //
137 //////////////////////////////////////////////////
138
139 // Note that the existing set of ASSERT macros specified above
shall be used for FPV,
140 // thereby ensuring that the assertions are evaluated during
DV simulations as well.
141
142 // ASSUME_FPV
143 // Assume a concurrent property during formal verification
only.
144 `define ASSUME_FPV(__name, __prop, __clk = `AS-

```

```

SERT_DEFAULT_CLK, __rst = `ASSERT_DEFAULT_RST) \
145 `ifdef
FPV_ON
146 `ASSUME(__name, __prop, __clk,
__rst)
147 `endif
148
149 // ASSUME_I_FPV
150 // Assume a concurrent property during formal verification
only.
151 `define ASSUME_I_FPV(__name, __prop) \
152 `ifdef FPV_ON
153 `ASSUME_I(__name, __prop)
154 `endif
155
156 // COVER_FPV
157 // Cover a concurrent property during formal verification

158 `define COVER_FPV(__name, __prop, __clk = `ASSERT_DEFAULT_CLK,
__rst = `ASSERT_DEFAULT_RST) \
159 `ifdef
FPV_ON
160 `COVER(__name, __prop, __clk,
__rst)
161 `endif
162
163 // FPV assertion that proves that the FSM control flow is lin-
ear (no loops)

164 // The sequence triggers whenever the state changes and stores
the current state as "initial_state".
165 // Then thereafter we must never see that state again until
reset.
166 // It is possible for the reset to release ahead of the clock.
167 // Create a small "gray" window beyond the usual rst time to
avoid
168 // checking.

169 `define ASSERT_FPV_LINEAR_FSM(__name, __state, __type, __clk =
`ASSERT_DEFAULT_CLK, __rst = `ASSERT_DEFAULT_RST) \
170 `ifdef
INC_ASSERT
171 bit __-
name`__cond;
172
173 always_ff @(posedge __clk or posedge __rst) be-
gin
174 if (__rst) be-
gin
175 __name`__cond <=
0;
176
177 end else be-
gin
178 __name`__cond <=
1;
179
180 end
181
182 property __-
name`__p;
183
184 __type ini-
tial_state;
185
186 ($stable(__state) & __name`__cond, initial_state =
$past(__state)) | ->
187 (__state != initial_state) until !(__-
name`__cond);
188
189 endprop-
erty
190
191 `ASSERT(__name, __name`__p, __clk,
0)
192
193 `endif
194
195 `include "prim_assert_sec_cm.svh"
196 `include "prim_flop_macros.sv"
197
198 `endif // PRIM_ASSERT_SV

```

14. lowRISC_opentitan/hw/ip/prim/rtl/prim_blanker.sv (20 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 // Convenience module for wrapping prim_and2 for use in blank-

```

```

ing.
6 // When en_i == 1 the input is fed through to the output.
7 // When en_i == 0 the output is 0.
8 module prim_blanker #(
9     parameter int Width = 1
10 ) (
11     input logic [Width-1:0] in_i,
12     input logic en_i,
13     output logic [Width-1:0] out_o
14 );
15     prim_and2 #(.Width(Width)) u_blank_and (
16         .in0_i(in_i),
17         .in1_i({Width{en_i}}),
18         .out_o
19     );
20 endmodule

```

15. lowRISC_opentitan/hw/ip/prim/rtl/prim_cdc_rand_delay.sv (67 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // This module should be instantiated within flops of CDC syn-
  chronization primitives,
6 // and allows DV to model real CDC delays within simulations,
  especially useful at the chip level
7 // or in IPs that communicate across clock domains.
8 //
9 // The instrumentation is very simple: when this is enabled
  the input into the first
10 // synchronizer flop has a mux where the select is randomly
  set. One of the mux inputs is the input
11 // of this module, and the other is the output of the first
  flop: selecting the latter models the
12 // effect of the first flop missing the input transition.
13 //
14 // Notice the delay should cause the input to be skipped by at
  most a single cycle. As a perhaps
15 // unnecessary precaution, the select will only be allowed to
  be random when the input changes.
16
17 module prim_cdc_rand_delay #(
18     parameter int DataWidth = 1,
19     parameter bit Enable = 1
20 ) (
21     input logic clk_i,
22     input logic rst_ni,
23     input logic [DataWidth-1:0] prev_data_i,
24     input logic [DataWidth-1:0] src_data_i,
25     output logic [DataWidth-1:0] dst_data_o
26 );
27 `ifdef SIMULATION
28     if (Enable) begin : gen_enable
29
30         // This controls dst_data_o: any bit with its data_sel set
        uses prev_data_i, others use
31         // src_data_i.
32         bit [DataWidth-1:0] data_sel;
33         bit cdc_instrumentation_enabled;
34
35         function automatic bit [DataWidth-1:0] fast_randomize();
36         bit [DataWidth-1:0] data;
37         if (DataWidth <= 32) begin
38             data = $urandom();
39         end else begin
40             if (!std::randomize(data)) $fatal(1, "%t: [%m] Failed
        to randomize data", $time);
41         end
42         return data;
43         endfunction
44
45         initial begin
46             void'($value$plusargs("cdc_instrumentation_enabled=%d",
        cdc_instrumentation_enabled));
47             data_sel = '0;
48         end
49
50         // Set data_sel at random combinationally when the input
        changes.
51         always @(src_data_i) begin
52             data_sel = cdc_instrumentation_enabled ? fast_random-
        ize() : 0;
53         end
54
55         // Clear data_del on any cycle start.
56         always @(posedge clk_i or negedge rst_ni) begin
57             data_sel <= 0;
58         end
59
60         always_comb dst_data_o = (prev_data_i & data_sel) |
        (src_data_i & ~data_sel);
61     end else begin : gen_no_enable
62         assign dst_data_o = src_data_i;
63     end

```

```

64 `else // SIMULATION
65     assign dst_data_o = src_data_i;
66 `endif // SIMULATION
67 endmodule

```

16. lowRISC_opentitan/hw/ip/prim/rtl/prim_cipher_pkg.sv (397 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // This package holds common constants and functions for
  PRESENT- and
6 // PRINCE-based scrambling devices.
7 //
8 // See also: prim_present, prim_prince
9 //
10 // References: - https://en.wikipedia.org/wiki/PRESENT
11 //              - https://en.wikipedia.org/wiki/Prince_(cipher)
12 //              - http://www.lightweightcrypto.org/present/
  present_ches2007.pdf
13 //              - https://eprint.iacr.org/2012/529.pdf
14 //              - https://eprint.iacr.org/2015/372.pdf
15 //              - https://eprint.iacr.org/2014/656.pdf
16
17 package prim_cipher_pkg;
18
19 ///////////////////////////////////////////////////
20 // PRINCE Cipher //
21 ///////////////////////////////////////////////////
22
23     parameter logic [15:0][3:0] PRINCE_SBOX4 = {4'h4, 4'hD,
4'h5, 4'hE,
24                                     4'h0, 4'h8,
4'h7, 4'h6,
25                                     4'h1, 4'h9,
4'hC, 4'hA,
26                                     4'h2, 4'h3,
4'hF, 4'hB};
27
28     parameter logic [15:0][3:0] PRINCE_SBOX4_INV = {4'h1, 4'hC,
4'hE, 4'h5,
29                                     4'h0, 4'h4,
4'h6, 4'hA,
30                                     4'h9, 4'h8,
4'hD, 4'hF,
31                                     4'h2, 4'h3,
4'h7, 4'hB};
32 // nibble permutations
33     parameter logic [15:0][3:0] PRINCE_SHIFT_ROWS64 = '{4'hF,
4'hA, 4'h5, 4'h0,
34                                     4'hB,
4'h6, 4'h1, 4'hC,
35                                     4'h7,
4'h2, 4'hD, 4'h8,
36                                     4'h3,
4'hE, 4'h9, 4'h4};
37
38     parameter logic [15:0][3:0] PRINCE_SHIFT_ROWS64_INV =
        '{4'hF, 4'h2, 4'h5, 4'h8,
39
40     4'hB, 4'hE, 4'h1, 4'h4,
41
42     4'h7, 4'hA, 4'hD, 4'h0,
43
44     4'h3, 4'h6, 4'h9, 4'hC};
45
46 // these are the round constants
47     parameter logic [11:0][63:0] PRINCE_ROUND_CONST =
        {64'hC0AC29B7C97C50DD,
48
49     64'hD3B5A399CA0C2399,
50
51     64'h64A51195E0E3610D,
52
53     64'hC882D32F25323C54,
54
55     64'h85840851F1AC43AA,
56
57     64'h7E-
        F84F78FD955CB1,
58
59     64'hBE5466CF34E90C6C,
60
61     64'h452821E638D01377,
62
63     64'h082E-
        FA98EC4E6C89,
64
65     64'hA4093822299F31D0,
66
67     64'h13198A2E03707344,
68
69     64'h0000000000000000};
70
71 // tweak constant for key modification between enc/dec modes
72     parameter logic [63:0] PRINCE_ALPHA_CONST =
        64'hC0AC29B7C97C50DD;
73

```

```

60 // masking constants for shift rows function below
61 parameter logic [15:0] PRINCE_SHIFT_ROWS_CONST0 = 16'h7BDE;
62 parameter logic [15:0] PRINCE_SHIFT_ROWS_CONST1 = 16'hBDE7;
63 parameter logic [15:0] PRINCE_SHIFT_ROWS_CONST2 = 16'hDE7B;
64 parameter logic [15:0] PRINCE_SHIFT_ROWS_CONST3 = 16'hE7BD;
65
66 // nibble shifts
67
68 function automatic logic [31:0] prince_shiftrows_32bit(logic
[31:0] state_in,
69                                     logic
[15:0][3:0] shifts );
70     logic [31:0] state_out;
71     // note that if simulation performance becomes an issue,
this loop can be unrolled
72     for (int k = 0; k < 32/2; k++) begin
73         // operate on pairs of 2bit instead of nibbles
74         state_out[k*2 +: 2] = state_in[shifts[k]*2 +: 2];
75     end
76     return state_out;
77 endfunction : prince_shiftrows_32bit
78
79 function automatic logic [63:0] prince_shiftrows_64bit(logic
[63:0] state_in,
80                                     logic
[15:0][3:0] shifts );
81     logic [63:0] state_out;
82     // note that if simulation performance becomes an issue,
this loop can be unrolled
83     for (int k = 0; k < 64/4; k++) begin
84         state_out[k*4 +: 4] = state_in[shifts[k]*4 +: 4];
85     end
86     return state_out;
87 endfunction : prince_shiftrows_64bit
88
89 // XOR reduction of four nibbles in a 16bit subvector
90 function automatic logic [3:0] prince_nibble_red16(logic
[15:0] vect);
91     return vect[0 +: 4] ^ vect[4 +: 4] ^ vect[8 +: 4] ^
vect[12 +: 4];
92 endfunction : prince_nibble_red16
93
94 // M prime multiplication
95 function automatic logic [31:0]
prince_mult_prime_32bit(logic [31:0] state_in);
96     logic [31:0] state_out;
97     // M0
98     state_out[0 +: 4] = prince_nibble_red16(state_in[ 0 +:
16] & PRINCE_SHIFT_ROWS_CONST3);
99     state_out[4 +: 4] = prince_nibble_red16(state_in[ 0 +:
16] & PRINCE_SHIFT_ROWS_CONST2);
100    state_out[8 +: 4] = prince_nibble_red16(state_in[ 0 +:
16] & PRINCE_SHIFT_ROWS_CONST1);
101    state_out[12 +: 4] = prince_nibble_red16(state_in[ 0 +:
16] & PRINCE_SHIFT_ROWS_CONST0);
102    // M1
103    state_out[16 +: 4] = prince_nibble_red16(state_in[16 +:
16] & PRINCE_SHIFT_ROWS_CONST0);
104    state_out[20 +: 4] = prince_nibble_red16(state_in[16 +:
16] & PRINCE_SHIFT_ROWS_CONST3);
105    state_out[24 +: 4] = prince_nibble_red16(state_in[16 +:
16] & PRINCE_SHIFT_ROWS_CONST2);
106    state_out[28 +: 4] = prince_nibble_red16(state_in[16 +:
16] & PRINCE_SHIFT_ROWS_CONST1);
107    return state_out;
108 endfunction : prince_mult_prime_32bit
109
110 // M prime multiplication
111 function automatic logic [63:0]
prince_mult_prime_64bit(logic [63:0] state_in);
112     logic [63:0] state_out;
113     // M0
114     state_out[0 +: 4] = prince_nibble_red16(state_in[ 0 +:
16] & PRINCE_SHIFT_ROWS_CONST3);
115     state_out[4 +: 4] = prince_nibble_red16(state_in[ 0 +:
16] & PRINCE_SHIFT_ROWS_CONST2);
116     state_out[8 +: 4] = prince_nibble_red16(state_in[ 0 +:
16] & PRINCE_SHIFT_ROWS_CONST1);
117     state_out[12 +: 4] = prince_nibble_red16(state_in[ 0 +:
16] & PRINCE_SHIFT_ROWS_CONST0);
118    // M1
119    state_out[16 +: 4] = prince_nibble_red16(state_in[16 +:
16] & PRINCE_SHIFT_ROWS_CONST0);
120    state_out[20 +: 4] = prince_nibble_red16(state_in[16 +:
16] & PRINCE_SHIFT_ROWS_CONST3);
121    state_out[24 +: 4] = prince_nibble_red16(state_in[16 +:
16] & PRINCE_SHIFT_ROWS_CONST2);
122    state_out[28 +: 4] = prince_nibble_red16(state_in[16 +:
16] & PRINCE_SHIFT_ROWS_CONST1);
123    // M1
124    state_out[32 +: 4] = prince_nibble_red16(state_in[32 +:
16] & PRINCE_SHIFT_ROWS_CONST0);
125    state_out[36 +: 4] = prince_nibble_red16(state_in[32 +:
16] & PRINCE_SHIFT_ROWS_CONST3);
126    state_out[40 +: 4] = prince_nibble_red16(state_in[32 +:
16] & PRINCE_SHIFT_ROWS_CONST2);
127    state_out[44 +: 4] = prince_nibble_red16(state_in[32 +:
16] & PRINCE_SHIFT_ROWS_CONST1);

```

```

127 // M0
128 state_out[48 +: 4] = prince_nibble_red16(state_in[48 +:
16] & PRINCE_SHIFT_ROWS_CONST3);
129 state_out[52 +: 4] = prince_nibble_red16(state_in[48 +:
16] & PRINCE_SHIFT_ROWS_CONST2);
130 state_out[56 +: 4] = prince_nibble_red16(state_in[48 +:
16] & PRINCE_SHIFT_ROWS_CONST1);
131 state_out[60 +: 4] = prince_nibble_red16(state_in[48 +:
16] & PRINCE_SHIFT_ROWS_CONST0);
132 return state_out;
133 endfunction : prince_mult_prime_64bit
134
135
136 // PRESENT Cipher
137
138 // this is the sbox from the present cipher
139 parameter logic [15:0][3:0] PRESENT_SBOX4 = {4'h2, 4'h1,
4'h7, 4'h4,
140                                     4'h8, 4'hF,
4'hE, 4'h3,
141                                     4'hD, 4'hA,
4'h0, 4'h9,
142                                     4'hB, 4'h6,
4'h5, 4'hC};
143
144 parameter logic [15:0][3:0] PRESENT_SBOX4_INV = {4'hA, 4'h9,
4'h7, 4'h0,
145                                     4'h3, 4'h6,
4'h4, 4'hB,
146                                     4'hD, 4'h2,
4'h1, 4'hC,
147                                     4'h8, 4'hF,
4'hE, 4'h5};
148
149 // these are modified permutation indices for a 32bit ver-
sion that
150 // follow the same pattern as for the 64bit version
151 parameter logic [31:0][4:0] PRESENT_PERM32 = {5'd31, 5'd23,
5'd15, 5'd07,
152                                     5'd30, 5'd22,
5'd14, 5'd06,
153                                     5'd29, 5'd21,
5'd13, 5'd05,
154                                     5'd28, 5'd20,
5'd12, 5'd04,
155                                     5'd27, 5'd19,
5'd11, 5'd03,
156                                     5'd26, 5'd18,
5'd10, 5'd02,
157                                     5'd25, 5'd17,
5'd09, 5'd01,
158                                     5'd24, 5'd16,
5'd08, 5'd00};
159
160 parameter logic [31:0][4:0] PRESENT_PERM32_INV = {5'd31,
5'd27, 5'd23, 5'd19,
161                                     5'd15,
5'd11, 5'd07, 5'd03,
162                                     5'd30,
5'd26, 5'd22, 5'd18,
163                                     5'd14,
5'd10, 5'd06, 5'd02,
164                                     5'd29,
5'd25, 5'd21, 5'd17,
165                                     5'd13,
5'd09, 5'd05, 5'd01,
166                                     5'd28,
5'd24, 5'd20, 5'd16,
167                                     5'd12,
5'd08, 5'd04, 5'd00};
168
169 // these are the permutation indices of the present cipher
170 parameter logic [63:0][5:0] PRESENT_PERM64 = {6'd63, 6'd47,
6'd31, 6'd15,
171                                     6'd62, 6'd46,
6'd30, 6'd14,
172                                     6'd61, 6'd45,
6'd29, 6'd13,
173                                     6'd60, 6'd44,
6'd28, 6'd12,
174                                     6'd59, 6'd43,
6'd27, 6'd11,
175                                     6'd58, 6'd42,
6'd26, 6'd10,
176                                     6'd57, 6'd41,
6'd25, 6'd09,
177                                     6'd56, 6'd40,
6'd24, 6'd08,
178                                     6'd55, 6'd39,
6'd23, 6'd07,
179                                     6'd54, 6'd38,
6'd22, 6'd06,
180                                     6'd53, 6'd37,
6'd21, 6'd05,
181                                     6'd52, 6'd36,
6'd20, 6'd04,
182                                     6'd51, 6'd35,
6'd19, 6'd11,
183                                     6'd50, 6'd34,
6'd18, 6'd10,
184                                     6'd49, 6'd33,
6'd17, 6'd09,
185                                     6'd48, 6'd32,
6'd16, 6'd08,
186                                     6'd47, 6'd31,
6'd15, 6'd07,
187                                     6'd46, 6'd30,
6'd14, 6'd06,
188                                     6'd45, 6'd29,
6'd13, 6'd05,
189                                     6'd44, 6'd28,
6'd12, 6'd04,
190                                     6'd43, 6'd27,
6'd11, 6'd03,
191                                     6'd42, 6'd26,
6'd10, 6'd02,
192                                     6'd41, 6'd25,
6'd09, 6'd01,
193                                     6'd40, 6'd24,
6'd08, 6'd00};

```



```

6'd20, 6'd04,
184
6'd19, 6'd03,
185
6'd18, 6'd02,
186
6'd17, 6'd01,
187
6'd16, 6'd00;
188
189 parameter logic [63:0][5:0] PRESENT_PERM64_INV = {6'd63,
6'd59, 6'd55, 6'd51,
190
6'd43, 6'd39, 6'd35,
191
6'd27, 6'd23, 6'd19,
192
6'd11, 6'd07, 6'd03,
193
6'd58, 6'd54, 6'd50,
194
6'd42, 6'd38, 6'd34,
195
6'd26, 6'd22, 6'd18,
196
6'd10, 6'd06, 6'd02,
197
6'd57, 6'd53, 6'd49,
198
6'd41, 6'd37, 6'd33,
199
6'd25, 6'd21, 6'd17,
200
6'd09, 6'd05, 6'd01,
201
6'd56, 6'd52, 6'd48,
202
6'd40, 6'd36, 6'd32,
203
6'd24, 6'd20, 6'd16,
204
6'd08, 6'd04, 6'd00;
205
206 // forward key schedule
207 function automatic logic [63:0] present_update_key64(logic
[63:0] key_in,
208
logic
[4:0] round_idx);
209 logic [63:0] key_out;
210 // rotate by 61 to the left
211 key_out = {key_in[63-61:0], key_in[63:64-61]};
212 // sbox on uppermost 4 bits
213 key_out[63 -: 4] = PRESENT_SBOX4[key_out[63 -: 4]];
214 // xor in round counter on bits 19 to 15
215 key_out[19:15] ^= round_idx;
216 return key_out;
217 endfunction : present_update_key64
218
219 function automatic logic [79:0] present_update_key80(logic
[79:0] key_in,
220
logic
[4:0] round_idx);
221 logic [79:0] key_out;
222 // rotate by 61 to the left
223 key_out = {key_in[79-61:0], key_in[79:80-61]};
224 // sbox on uppermost 4 bits
225 key_out[79 -: 4] = PRESENT_SBOX4[key_out[79 -: 4]];
226 // xor in round counter on bits 19 to 15
227 key_out[19:15] ^= round_idx;
228 return key_out;
229 endfunction : present_update_key80
230
231 function automatic logic [127:0] present_update_key128(logic
[127:0] key_in,
232
logic
[4:0] round_idx);
233 logic [127:0] key_out;
234 // rotate by 61 to the left
235 key_out = {key_in[127-61:0], key_in[127:128-61]};
236 // sbox on uppermost 4 bits
237 key_out[127 -: 4] = PRESENT_SBOX4[key_out[127 -: 4]];
238 // sbox on second nibble from top
239 key_out[123 -: 4] = PRESENT_SBOX4[key_out[123 -: 4]];
240 // xor in round counter on bits 66 to 62
241 key_out[66:62] ^= round_idx;
242 return key_out;
243 endfunction : present_update_key128
244
245
246 // inverse key schedule
247 function automatic logic [63:0]
present_inv_update_key64(logic [63:0] key_in,
248
lo-
gic [4:0] round_idx);
249 logic [63:0] key_out = key_in;
250 // xor in round counter on bits 19 to 15
251 key_out[19:15] ^= round_idx;
252 // sbox on uppermost 4 bits
253 key_out[63 -: 4] = PRESENT_SBOX4_INV[key_out[63 -: 4]];

```

```

254 // rotate by 61 to the right
255 key_out = {key_out[60:0], key_out[63:61]};
256 return key_out;
257 endfunction : present_inv_update_key64
258
259 function automatic logic [79:0]
present_inv_update_key80(logic [79:0] key_in,
260
lo-
gic [4:0] round_idx);
261 logic [79:0] key_out = key_in;
262 // xor in round counter on bits 19 to 15
263 key_out[19:15] ^= round_idx;
264 // sbox on uppermost 4 bits
265 key_out[79 -: 4] = PRESENT_SBOX4_INV[key_out[79 -: 4]];
266 // rotate by 61 to the right
267 key_out = {key_out[60:0], key_out[79:61]};
268 return key_out;
269 endfunction : present_inv_update_key80
270
271 function automatic logic [127:0]
present_inv_update_key128(logic [127:0] key_in,
272
logic [4:0] round_idx);
273 logic [127:0] key_out = key_in;
274 // xor in round counter on bits 66 to 62
275 key_out[66:62] ^= round_idx;
276 // sbox on second highest nibble
277 key_out[123 -: 4] = PRESENT_SBOX4_INV[key_out[123 -: 4]];
278 // sbox on uppermost 4 bits
279 key_out[127 -: 4] = PRESENT_SBOX4_INV[key_out[127 -: 4]];
280 // rotate by 61 to the right
281 key_out = {key_out[60:0], key_out[127:61]};
282 return key_out;
283 endfunction : present_inv_update_key128
284
285
286 // these functions can be used to derive the DEC key from
the ENC key by
287 // stepping the key by the correct number of rounds using
the keyschedule functions above.
288 function automatic logic [63:0] present_get_dec_key64(logic
[63:0] key_in,
289
//
total number of rounds employed
290
logic
[4:0] round_cnt);
291 logic [63:0] key_out;
292 key_out = key_in;
293 for (int unsigned k = 0; k < round_cnt; k++) begin
294 key_out = present_update_key64(key_out, 5'(k + 1));
295 end
296 return key_out;
297 endfunction : present_get_dec_key64
298
299 function automatic logic [79:0] present_get_dec_key80(logic
[79:0] key_in,
300
//
total number of rounds employed
301
logic
[4:0] round_cnt);
302 logic [79:0] key_out;
303 key_out = key_in;
304 for (int unsigned k = 0; k < round_cnt; k++) begin
305 key_out = present_update_key80(key_out, 5'(k + 1));
306 end
307 return key_out;
308 endfunction : present_get_dec_key80
309
310 function automatic logic [127:0]
present_get_dec_key128(logic [127:0] key_in,
311
//
total number of rounds employed
312
lo-
gic [4:0] round_cnt);
313 logic [127:0] key_out;
314 key_out = key_in;
315 for (int unsigned k = 0; k < round_cnt; k++) begin
316 key_out = present_update_key128(key_out, 5'(k + 1));
317 end
318 return key_out;
319 endfunction : present_get_dec_key128
320
321 //////////////////////////////////////////////////
322 // Common Subfunctions //
323 //////////////////////////////////////////////////
324
325 function automatic logic [7:0] sbox4_8bit(logic [7:0]
state_in, logic [15:0][3:0] sbox4);
326 logic [7:0] state_out;
327 // note that if simulation performance becomes an issue,
this loop can be unrolled
328 for (int k = 0; k < 8/4; k++) begin
329 state_out[k*4 +: 4] = sbox4[state_in[k*4 +: 4]];
330 end
331 return state_out;
332 endfunction : sbox4_8bit
333
334 function automatic logic [15:0] sbox4_16bit(logic [15:0]
state_in, logic [15:0][3:0] sbox4);
335 logic [15:0] state_out;
336 // note that if simulation performance becomes an issue,

```

```

this loop can be unrolled
337   for (int k = 0; k < 2; k++) begin
338       state_out[k*8 +: 8] = sbox4_8bit(state_in[k*8 +: 8],
sbox4);
339   end
340   return state_out;
341   endfunction : sbox4_16bit
342
343   function automatic logic [31:0] sbox4_32bit(logic [31:0]
state_in, logic [15:0][3:0] sbox4);
344       logic [31:0] state_out;
345       // note that if simulation performance becomes an issue,
this loop can be unrolled
346       for (int k = 0; k < 4; k++) begin
347           state_out[k*8 +: 8] = sbox4_8bit(state_in[k*8 +: 8],
sbox4);
348       end
349       return state_out;
350   endfunction : sbox4_32bit
351
352   function automatic logic [63:0] sbox4_64bit(logic [63:0]
state_in, logic [15:0][3:0] sbox4);
353       logic [63:0] state_out;
354       // note that if simulation performance becomes an issue,
this loop can be unrolled
355       for (int k = 0; k < 8; k++) begin
356           state_out[k*8 +: 8] = sbox4_8bit(state_in[k*8 +: 8],
sbox4);
357       end
358       return state_out;
359   endfunction : sbox4_64bit
360
361   function automatic logic [7:0] perm_8bit(logic [7:0]
state_in, logic [7:0][2:0] perm);
362       logic [7:0] state_out;
363       // note that if simulation performance becomes an issue,
this loop can be unrolled
364       for (int k = 0; k < 8; k++) begin
365           state_out[perm[k]] = state_in[k];
366       end
367       return state_out;
368   endfunction : perm_8bit
369
370   function automatic logic [15:0] perm_16bit(logic [15:0]
state_in, logic [15:0][3:0] perm);
371       logic [15:0] state_out;
372       // note that if simulation performance becomes an issue,
this loop can be unrolled
373       for (int k = 0; k < 16; k++) begin
374           state_out[perm[k]] = state_in[k];
375       end
376       return state_out;
377   endfunction : perm_16bit
378
379   function automatic logic [31:0] perm_32bit(logic [31:0]
state_in, logic [31:0][4:0] perm);
380       logic [31:0] state_out;
381       // note that if simulation performance becomes an issue,
this loop can be unrolled
382       for (int k = 0; k < 32; k++) begin
383           state_out[perm[k]] = state_in[k];
384       end
385       return state_out;
386   endfunction : perm_32bit
387
388   function automatic logic [63:0] perm_64bit(logic [63:0]
state_in, logic [63:0][5:0] perm);
389       logic [63:0] state_out;
390       // note that if simulation performance becomes an issue,
this loop can be unrolled
391       for (int k = 0; k < 64; k++) begin
392           state_out[perm[k]] = state_in[k];
393       end
394       return state_out;
395   endfunction : perm_64bit
396
397   endpackage : prim_cipher_pkg

```

17. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_gating_sync.sv (34 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Common Library: Clock Gating cell with synchronizer
6
7 module prim_clock_gating_sync (
8     input      clk_i,
9     input      rst_ni,
10    input      test_en_i,
11    input      async_en_i,
12    output logic en_o,
13    output logic clk_o
14);
15
16
17    prim_flop_2sync #(

```

```

18        .Width(1)
19    ) i_sync (
20        .clk_i,
21        .rst_ni,
22        .d_i(async_en_i),
23        .q_o(en_o)
24    );
25
26    prim_clock_gating i_cg (
27        .clk_i,
28        .en_i(en_o),
29        .test_en_i,
30        .clk_o
31    );
32
33
34 endmodule

```

18. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_meas.sv (279 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // prim_clk_meas is a generic module that measures two clocks
against each other.
6 // One clock is the reference, and another is the input.
7 // If the input clocks becomes too fast or too slow, an error
condition is created.
8
9 // The default parameters assume the reference clock is signi-
fically slower than
10 // the input clock
11
12 `include "prim_assert.sv"
13
14 module prim_clock_meas #(
15     // Maximum value of input clock counts over measurement
period
16     parameter int Cnt = 16,
17     // Maximum value of reference clock counts over measurement
period
18     parameter int RefCnt = 1,
19     parameter bit ClkTimeOutChkEn = 1,
20     parameter bit RefTimeOutChkEn = 1,
21     localparam int CntWidth = prim_util_pkg::vbits(Cnt),
22     localparam int RefCntWidth = prim_util_pkg::vbits(RefCnt)
23 ) (
24     input clk_i,
25     input rst_ni,
26     input clk_ref_i,
27     input rst_ref_ni,
28     input en_i,
29     input [CntWidth-1:0] max_cnt,
30     input [CntWidth-1:0] min_cnt,
31     // the output valid and fast/slow indications are all on the
// input clock domain
32     output logic valid_o,
33     output logic fast_o,
34     output logic slow_o,
35
36     // signal on clk_i domain that indicates clk_ref has timed
out
37     output logic timeout_clk_ref_o,
38
39     // signal on clk_ref_i domain that indicates clk has timed
out
40     output logic ref_timeout_clk_o
41 );
42
43
44
45 ///////////////////////////////////////////////////////////////////
46 // Reference clock logic
47 ///////////////////////////////////////////////////////////////////
48
49 logic ref_en;
50 prim_flop_2sync #(
51     .Width(1)
52 ) u_ref_meas_en_sync (
53     .d_i(en_i),
54     .clk_i(clk_ref_i),
55     .rst_ni(rst_ref_ni),
56     .q_o(ref_en)
57 );
58
59
60 ///////////////////////////////////////////////////////////////////
61 // Domain sequencing
62 ///////////////////////////////////////////////////////////////////
63
64 // the enable comes in on the clk_i domain, however, to en-
sure the
65 // clk and clk_ref domains remain in sync, the following se-
quencing is
66 // done to enable/disable the measurement feature
67
68 typedef enum logic [1:0] {

```

```

69   StDisable,
70   StEnabling,
71   StEnable,
72   StDisabling
73 } meas_chk_fsm_e;
74
75 // sync reference clock view of enable back into the source
domain
76   logic en_ref_sync;
77   prim_flop_2sync #(
78     .Width(1)
79   ) ack_sync (
80     .clk_i,
81     .rst_ni,
82     .d_i(ref_en),
83     .q_o(en_ref_sync)
84   );
85
86   meas_chk_fsm_e state_d, state_q;
87   always_ff @(posedge clk_i or negedge rst_ni) begin
88     if (!rst_ni) begin
89       state_q <= StDisable;
90     end else begin
91       state_q <= state_d;
92     end
93   end
94
95   // The following fsm sequence ensures that even if the
source
96   // side changes the enable too quickly, the measurement con-
trols
97   // remain consistent.
98   logic cnt_en;
99   always_comb begin
100     state_d = state_q;
101     cnt_en = '0;
102
103     unique case (state_q)
104
105       StDisable: begin
106         if (en_i) begin
107           state_d = StEnabling;
108         end
109       end
110
111       StEnabling: begin
112         if (en_ref_sync) begin
113           state_d = StEnable;
114         end
115       end
116
117       StEnable: begin
118         cnt_en = 1'b1;
119         if (!en_i) begin
120           state_d = StDisabling;
121         end
122       end
123
124       StDisabling: begin
125         if (!en_ref_sync) begin
126           state_d = StDisable;
127         end
128       end
129
130       //VCS coverage off
131       // pragma coverage off
132       default:;
133       //VCS coverage on
134       // pragma coverage on
135
136     endcase // unique case (state_q)
137   end
138
139   // Input Clock Logic
140   // Input Clock Logic
141   // Input Clock Logic
142
143   logic valid_ref;
144   logic valid;
145   // The valid pulse causes the count to reset and start
counting again
146   // for each reference cycle.
147   // The count obtained during the last reference cycle is
used
148   // to measure how fast/slow the input clock is.
149   prim_pulse_sync u_sync_ref (
150     .clk_src_i(clk_ref_i),
151     .rst_src_ni(rst_ref_ni),
152     .src_pulse_i(ref_en),
153     .clk_dst_i(clk_i),
154     .rst_dst_ni(rst_ni),
155     .dst_pulse_o(valid_ref)
156   );
157
158   if (RefCnt == 1) begin : gen_degenerate_case
159     // if reference count is one, cnt_ref is always 0.
160     // So there is no need to maintain a counter, and
161     // valid just becomes valid_ref
162     assign valid = valid_ref;
163   end else begin : gen_normal_case

```

```

165     logic [RefCntWidth-1:0] cnt_ref;
166     assign valid = valid_ref & (int'(cnt_ref) == RefCnt - 1);
167     always_ff @(posedge clk_i or negedge rst_ni) begin
168       if (!rst_ni) begin
169         cnt_ref <= '0;
170       end else if (!cnt_en && |cnt_ref) begin
171         cnt_ref <= '0;
172       end else if (cnt_en && valid) begin
173         cnt_ref <= '0;
174       end else if (cnt_en && valid_ref) begin
175         cnt_ref <= cnt_ref + 1'b1;
176       end
177     end
178   end
179
180   logic cnt_ovfl;
181   logic [CntWidth-1:0] cnt;
182   always_ff @(posedge clk_i or negedge rst_ni) begin
183     if (!rst_ni) begin
184       cnt <= '0;
185       cnt_ovfl <= '0;
186     end else if (!cnt_en && |cnt) begin
187       cnt <= '0;
188       cnt_ovfl <= '0;
189     end else if (valid_o) begin
190       cnt <= '0;
191       cnt_ovfl <= '0;
192     end else if (cnt_ovfl) begin
193       cnt <= '{default: '1};
194     end else if (cnt_en) begin
195       {cnt_ovfl, cnt} <= cnt + 1'b1;
196     end
197   end
198
199   assign valid_o = valid & |cnt;
200   assign fast_o = valid_o & ((cnt > max_cnt) | cnt_ovfl);
201   assign slow_o = valid_o & (cnt < min_cnt);
202
203   // Timeout Handling
204   // Timeout Handling
205   // Timeout Handling
206
207   localparam bit TimeOutChkEn = ClkTimeOutChkEn | Ref-
TimeOutChkEn;
208
209   // determine ratio between
210   localparam int ClkRatio = Cnt / RefCnt;
211
212   // maximum cdc latency from the perspective of the measured
clock
213   // 1 cycle to output request
214   // 2 ref cycles for synchronization
215   // 1 ref cycle to send ack
216   // 2 cycles to sync ack
217   // Double for margin
218   localparam int MaxClkCdcLatency = (1 + 2*ClkRatio + 1*ClkRa-
tio + 2)*2;
219
220   // maximum cdc latency from the perspective of the reference
clock
221   // 1 ref cycle to output request
222   // 2 cycles to sync + 1 cycle to ack are less than 1 cycle
of ref based on assertion requirement
223   // 2 ref cycles to sync ack
224   // 2 extra ref cycles for margin
225   localparam int MaxRefCdcLatency = 1 + 1 + 2 + 2;
226
227   if (RefTimeOutChkEn) begin : gen_ref_timeout_chk
228     // check whether reference clock has timed out
229     prim_clock_timeout #(
230       .TimeOutCnt(MaxClkCdcLatency)
231     ) u_timeout_clk_to_ref (
232       .clk_chk_i(clk_ref_i),
233       .rst_chk_ni(rst_ref_ni),
234       .clk_i,
235       .rst_ni,
236       .en_i,
237       .timeout_o(timeout_clk_ref_o)
238     );
239   end else begin : gen_unused_ref_timeout
240     assign timeout_clk_ref_o = 1'b0;
241   end
242
243   if (ClkTimeOutChkEn) begin : gen_clk_timeout_chk
244     // check whether clock has timed out
245     prim_clock_timeout #(
246       .TimeOutCnt(MaxRefCdcLatency)
247     ) u_timeout_ref_to_clk (
248       .clk_chk_i(clk_i),
249       .rst_chk_ni(rst_ni),
250       .clk_i(clk_ref_i),
251       .rst_ni(rst_ref_ni),
252       .en_i(ref_en),
253       .timeout_o(ref_timeout_clk_o)
254     );
255   end else begin : gen_unused_clk_timeout
256     assign ref_timeout_clk_o = 1'b0;
257   end
258
259

```

```

260 //////////////////////////////////////////////////
261 // Assertions
262 //////////////////////////////////////////////////
263
264 if (TimeoutChkEn) begin : gen_timeout_assert
265     // the measured clock must be faster than the reference
clock
266     `ASSERT_INIT(ClkRatios_A, ClkRatio > 2)
267 end
268
269 // reference count has to be at least 1
270 `ASSERT_INIT(RefCntVal_A, RefCnt >= 1)
271
272 // if we've reached the max count, enable must be 0 next.
273 // Otherwise the width of the counter is too small to accom-
modate the use case
274 `ASSERT(MaxWidth_A, (cnt == Cnt-1) |>= !cnt_en )
275
276
277
278
279 endmodule // prim_clk_meas

```

19. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_timeout.sv (65 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // prim_clk_timeout is a simple module that assesses whether
the input clock
6 // has stopped ticking as measured by the reference clock.
7 //
8 // If both clocks are stopped for whatever reason, this module is
effectively dead.
9
10 `include "prim_assert.sv"
11
12 module prim_clock_timeout #(
13     parameter int TimeoutCnt = 16,
14     localparam int CntWidth = prim_util_pkg::vbits(TimeoutCnt+1)
15 ) (
16     // clock to be checked
17     input clk_chk_i,
18     input rst_chk_ni,
19
20     // clock used to measure whether clk_chk has stopped ticking
21     input clk_i,
22     input rst_ni,
23     input en_i,
24     output logic timeout_o
25 );
26
27 logic [CntWidth-1:0] cnt;
28 logic ack;
29 logic timeout;
30 assign timeout = int'(cnt) >= TimeoutCnt;
31 always_ff @(posedge clk_i or negedge rst_ni) begin
32     if (!rst_ni) begin
33         cnt <= '0;
34     end else if (ack || !en_i) begin
35         cnt <= '0;
36     end else if (timeout) begin
37         cnt <= '{default: '1};
38     end else if (en_i) begin
39         cnt <= cnt + 1'b1;
40     end
41 end
42
43 logic chk_req;
44 prim_sync_reqack u_ref_timeout (
45     .clk_src_i(clk_i),
46     .rst_src_ni(rst_ni),
47     .clk_dst_i(clk_chk_i),
48     .rst_dst_ni(rst_chk_ni),
49     .req_chk_i('0),
50     .src_req_i(1'b1),
51     .src_ack_o(ack),
52     .dst_req_o(chk_req),
53     .dst_ack_i(chk_req)
54 );
55
56 prim_flop #(
57     .ResetValue('0)
58 ) u_out (
59     .clk_i,
60     .rst_ni,
61     .d_i(timeout),
62     .q_o(timeout_o)
63 );
64
65 endmodule // prim_clk_timeout

```

20. lowRISC_opentitan/hw/ip/prim/rtl/prim_count.sv (313 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Hardened counter primitive:
6 //
7 // This internally uses a cross counter scheme with primary
and a secondary counter, where the
8 // secondary counter counts in reverse direction. The sum of
both counters must remain constant and
9 // equal to 2**Width-1 or otherwise an err_o will be asserted.
10 //
11 // This counter supports a generic clear / set / increment /
decrement interface:
12 //
13 // clr_i: This clears the primary counter to ResetValue and
adjusts the secondary counter to match
14 // (i.e. 2**Width-1-ResetValue). Clear has priority
over set, increment and decrement.
15 // set_i: This sets the primary counter to set_cnt_i and ad-
justs the secondary counter to match
16 // (i.e. 2**Width-1-set_cnt_i). Set has priority over
increment and decrement.
17 // incr_en_i: Increments the primary counter by step_i, and
decrements the secondary by step_i.
18 // decr_en_i: Decrements the primary counter by step_i, and
increments the secondary by step_i.
19 // commit_i: Counter changes only take effect when `commit_i`
is set. This does not effect the
20 // `cnt_after_commit_o` output which gives the next
counter state if the change is
21 // committed.
22 //
23 // Note that if both incr_en_i and decr_en_i are asserted at
the same time, the counter remains
24 // unchanged. The counter is also protected against under- and
overflows.
25
26 `include "prim_assert.sv"
27
28 module prim_count
29     import prim_count_pkg::*;
30 (
31     parameter int Width = 2,
32     // Can be used to reset the counter to a different value
than 0, for example when
33     // the counter is used as a down-counter.
34     parameter logic [Width-1:0] ResetValue = '0,
35     // This should only be disabled in special circumstances,
for example
36     // in non-comportable IPs where an error does not trigger an
alert.
37     parameter bit EnableAlertTriggersVA = 1,
38
39     // We have some assertions below with preconditions that de-
pend on particular input actions
40     // (clear, set, incr, decr). If the design has instantiated
prim_count with one of these actions
41     // tied to zero, the preconditions for the associated asser-
tions will not be satisfiable. The
42     // result is an unreachable item, which we treat as a failed
assertion in the report.
43     //
44     // To avoid this, we the instantiation to specify the ac-
tions which might happen. If this is not
45     // '1, we will have an assertion which assert the corres-
ponding action is never triggered. We can
46     // then use this to avoid the unreachable assertions.
47     parameter action_mask_t PossibleActions = {$bits(ac-
tion_mask_t){1'b1}}
48 ) (
49     input clk_i,
50     input rst_ni,
51     input clr_i,
52     input set_i,
53     input [Width-1:0] set_cnt_i, // Set value
for the counter.
54     input incr_en_i,
55     input decr_en_i,
56     input [Width-1:0] step_i, // Increment/
decrement step when enabled.
57     input commit_i,
58     output logic [Width-1:0] cnt_o, // Current
counter state
59     output logic [Width-1:0] cnt_after_commit_o, // Next counter
state if committed
60     output logic err_o
61 );
62
63 //////////////////////////////////////////////////
64 // Counter logic //

```

```

65 //////////////////////////////////////////////////
66 // Reset Values for primary and secondary counters.
67 localparam int NumCnt = 2;
68 localparam logic [NumCnt-1:0][Width-1:0] ResetValues =
{{Width{1'b1}} - ResetValue, // secondary
69 setVal}; // primary
70 // primary
71 // The fpv_force signal can be used in FPV runs to make the
72 internal counters (cnt_q) jump
73 // unexpectedly. We only want to use this mechanism when
74 we're doing FPV on prim_count itself. In
75 // that situation, we will have the PrimCountFpv define and
76 wish to leave fpv_force undriven so
77 // that it becomes a free variable in FPV. In any other
78 situation, we drive the signal with zero.
79 logic [NumCnt-1:0][Width-1:0] fpv_force;
80 `ifndef PrimCountFpv
81 assign fpv_force = '0;
82 `endif
83 for (genvar k = 0; k < NumCnt; k++) begin : gen_cnts
84 // Note that increments / decrements are reversed for the
85 secondary counter.
86 logic incr_en, decr_en;
87 logic [Width-1:0] set_val;
88 if (k == 0) begin : gen_up_cnt
89 assign incr_en = incr_en_i;
90 assign decr_en = decr_en_i;
91 assign set_val = set_cnt_i;
92 end else begin : gen_dn_cnt
93 assign incr_en = decr_en_i;
94 assign decr_en = incr_en_i;
95 // The secondary value needs to be adjusted accordingly.
96 assign set_val = {Width{1'b1}} - set_cnt_i;
97 end
98 // Main counter logic
99 logic [Width:0] ext_cnt;
100 assign ext_cnt = (decr_en) ? {1'b0, cnt_q[k]} - {1'b0,
step_i} :
101 (incr_en) ? {1'b0, cnt_q[k]} + {1'b0,
step_i} : {1'b0, cnt_q[k]};
102 // Saturation logic
103 logic oflow, oflow;
104 assign oflow = incr_en && ext_cnt[Width];
105 assign oflow = decr_en && ext_cnt[Width];
106 logic [Width-1:0] cnt_sat;
107 assign cnt_sat = (oflow) ? '0 :
108 (oflow) ? {Width{1'b1}} :
ext_cnt[Width-1:0];
109 // Clock gate flops when in saturation, and do not
110 // count if both incr_en and decr_en are asserted.
111 logic cnt_en;
112 assign cnt_en = (incr_en ^ decr_en) &&
113 ((incr_en && !(cnt_q[k])) ||
114 (decr_en && !(cnt_q[k] == '0)));
115 // Counter muxes
116 assign cnt_d[k] = (clr_i) ? ResetValues[k] :
117 (set_i) ? set_val :
118 (cnt_en) ? cnt_sat : cnt_q[k];
119 assign cnt_d_committed[k] = commit_i ? cnt_d[k] :
cnt_q[k];
120 logic [Width-1:0] cnt_unforced_q;
121 prim_flop #(
122 .Width(Width),
123 .ResetValue(ResetValues[k])
124 ) u_cnt_flop (
125 .clk_i,
126 .rst_ni,
127 .d_i(cnt_d_committed[k]),
128 .q_o(cnt_unforced_q)
129 );
130 // fpv_force is only used during FPV.
131 assign cnt_q[k] = fpv_force[k] + cnt_unforced_q;
132 end
133 // The sum of both counters must always equal the counter
134 maximum.
135 logic [Width:0] sum;
136 assign sum = (cnt_q[0] + cnt_q[1]);
137 // Register the error signal to avoid potential CDC issues
138 downstream.
139 logic err_d, err_q;
140 assign err_d = (sum != {1'b0, {Width{1'b1}}});
141 always_ff @(posedge clk_i or negedge rst_ni) begin
142 if (!rst_ni) begin
143 err_q <= 'b0;
144 end else begin
145 err_q <= err_d;
146 end
147 end

```

```

153 end
154 assign err_o = err_q;
155 // Output count values
156 assign cnt_o = cnt_q[0];
157 assign cnt_after_commit_o = cnt_d[0];
158 //////////////////////////////////////////////////
159 // Assertions //
160 //////////////////////////////////////////////////
161 `ifdef INC_ASSERT
162 // VCS coverage off
163 // pragma coverage off
164 // We need to disable most assertions in that case using a
165 helper signal.
166 // We can't rely on err_o since some error patterns cannot
167 be detected (e.g. all error
168 // patterns that still fulfil the sum constraint).
169 logic fpv_err_present;
170 assign fpv_err_present = |fpv_force;
171 // Helper functions for assertions.
172
173 function automatic logic signed [Width+1:0] max(logic signed
[Width+1:0] a,
174 logic signed
[Width+1:0] b);
175 return (a > b) ? a : b;
176 endfunction
177
178 function automatic logic signed [Width+1:0] min(logic signed
[Width+1:0] a,
179 logic signed
[Width+1:0] b);
180 return (a < b) ? a : b;
181 endfunction
182 // VCS coverage on
183 // pragma coverage on
184 if (!(PossibleActions & Clr)) begin : g_check_no_clr
185 `ASSERT(ClrNeverTrue_A, clr_i !== 1'b1)
186 end
187 if (!(PossibleActions & Set)) begin : g_check_no_set
188 `ASSERT(SetNeverTrue_A, set_i !== 1'b1)
189 end
190 if (!(PossibleActions & Incr)) begin : g_check_no_incr
191 `ASSERT(IncrNeverTrue_A, incr_en_i !== 1'b1)
192 end
193 if (!(PossibleActions & Decr)) begin : g_check_no_decr
194 `ASSERT(DecrNeverTrue_A, decr_en_i !== 1'b1)
195 end
196 // Cnt next
197 `ASSERT(CntNext_A,
200 rst_ni
201 |>
202 $past(!commit_i) || (cnt_o ==
203 $past(cnt_after_commit_o)),
204 clk_i, err_d || fpv_err_present || !rst_ni)
205 // Clear
206 if (PossibleActions & Clr) begin : g_check_clr_fwd_a
207 `ASSERT(ClrFwd_A,
208 rst_ni && commit_i && clr_i
209 |>
210 (cnt_o == ResetValue) &&
211 (cnt_q[1] == ({Width{1'b1}} - ResetValue)),
212 clk_i, err_d || fpv_err_present || !rst_ni)
213 end
214 // Set
215 if (PossibleActions & Set) begin : g_check_set_fwd_a
216 `ASSERT(SetFwd_A,
217 rst_ni && commit_i && set_i && !clr_i
218 |>
219 (cnt_o == $past(set_cnt_i)) &&
220 (cnt_q[1] == ({Width{1'b1}} - $past(set_cnt_i))),
221 clk_i, err_d || fpv_err_present || !rst_ni)
222 end
223 // Do not count if both increment and decrement are asser-
224 ted.
225 if ((PossibleActions & Incr) && (PossibleActions & Decr))
226 begin : g_check_inc_and_dec
227 `ASSERT(IncrDecrUpDnCnt_A,
228 rst_ni && incr_en_i && decr_en_i && !(clr_i || set_i)
229 |>
230 $stable(cnt_o) && $stable(cnt_q[1]),
231 clk_i, err_d || fpv_err_present || !rst_ni)
232 end
233 // Increment
234 if ((PossibleActions & Incr)) begin : g_check_incr
235 `ASSERT(IncrUpCnt_A,
236 rst_ni && incr_en_i && !(clr_i || set_i || decr_en_i)
237 && commit_i
238 |>
239

```



```

240 cnt_o == min($past(cnt_o) + $past({2'b0, step_i}),
{2'b0, {Width{1'b1}}}),
241 clk_i, err_d || fpv_err_present || !rst_ni)
242 `ASSERT(IncrDnCnt_A,
243 rst_ni && incr_en_i && !(clr_i || set_i || decr_en_i)
&& commit_i
244 |>
245 cnt_q[1] == max($past(signed'({2'b0, cnt_q[1]})) -
$past({2'b0, step_i}), '0),
246 clk_i, err_d || fpv_err_present || !rst_ni)
247 `ASSERT(UpCntIncrStable_A,
248 incr_en_i && !(clr_i || set_i || decr_en_i) &&
249 cnt_o == {Width{1'b1}}
250 |>
251 $stable(cnt_o),
252 clk_i, err_d || fpv_err_present || !rst_ni)
253 `ASSERT(DnCntIncrStable_A,
254 rst_ni && incr_en_i && !(clr_i || set_i || decr_en_i)
&&
255 cnt_q[1] == '0
256 |>
257 $stable(cnt_q[1]),
258 clk_i, err_d || fpv_err_present || !rst_ni)
259 end
260
261 // Decrement
262 if ((PossibleActions & Decr)) begin : g_check_decr
263 `ASSERT(DecrDnCnt_A,
264 rst_ni && decr_en_i && !(clr_i || set_i || incr_en_i)
&& commit_i
265 |>
266 cnt_o == max($past(signed'({2'b0, cnt_o})) -
$past({2'b0, step_i}), '0),
267 clk_i, err_d || fpv_err_present || !rst_ni)
268 `ASSERT(DecrDnCnt_A,
269 rst_ni && decr_en_i && !(clr_i || set_i || incr_en_i)
&& commit_i
270 |>
271 cnt_q[1] == min($past(cnt_q[1]) + $past({2'b0,
step_i}), {2'b0, {Width{1'b1}}}),
272 clk_i, err_d || fpv_err_present || !rst_ni)
273 `ASSERT(UpCntDecrStable_A,
274 decr_en_i && !(clr_i || set_i || incr_en_i) &&
275 cnt_o == '0
276 |>
277 $stable(cnt_o),
278 clk_i, err_d || fpv_err_present || !rst_ni)
279 `ASSERT(DnCntDecrStable_A,
280 rst_ni && decr_en_i && !(clr_i || set_i || incr_en_i)
&&
281 cnt_q[1] == {Width{1'b1}}
282 |>
283 $stable(cnt_q[1]),
284 clk_i, err_d || fpv_err_present || !rst_ni)
285 end
286
287 // A backwards check for count changes. This asserts that
the count only changes if one of the
288 // inputs that should tell it to change (clear, set, incre-
ment, decrement) does so.
289 `ASSERT(ChangeBackward_A,
290 rst_ni ##1 $changed(cnt_o) && $changed(cnt_q[1])
291 |>
292 $past(clr_i || set_i || (commit_i && (incr_en_i ||
decr_en_i))),
293 clk_i, err_d || fpv_err_present || !rst_ni)
294
295 // Check that count errors are reported properly in err_o
296 //
297 // This is essentially a "|>" implication, but is struc-
tured in a way to avoid generating a cover
298 // property for the left hand side if PrimCountFpv is not
defined (because we won't have a way to
299 // inject an error if not)
300 `ASSERT(CntErrReported_A, ##1 $past((cnt_q[1] + cnt_q[0]) !=
{Width{1'b1}}) == err_o)
301 `ifdef PrimCountFpv
302 `COVER(CntErr_C, err_o)
303 `endif
304
305 // This logic that will be assign to one, when user adds
macro
306 // ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT to check the error
with alert, in case that prim_count
307 // is used in design without adding this assertion check.
308 logic unused_assert_connected;
309
310 `ASSERT_INIT_NET(AssertConnected_A, unused_assert_connected
=== '1'b1 || !EnableAlertTriggerSVA)
311 `endif
312
313 endmodule // prim_count

```

21. lowRISC_opentitan/hw/ip/prim/rtl/prim_count_pkg.sv (15 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for

```

details.

```

3 // SPDX-License-Identifier: Apache-2.0
4
5 package prim_count_pkg;
6
7 // An enum that names the possible actions that the inputs
might ask for. See the PossibleActions
8 // parameter in prim_count for how this is used.
9 typedef logic [3:0] action_mask_t;
10 typedef enum action_mask_t {Clr = 4'h1,
11 Set = 4'h2,
12 Incr = 4'h4,
13 Decr = 4'h8} action_e;
14
15 endpackage : prim_count_pkg

```

22. lowRISC_opentitan/hw/ip/prim/rtl/prim_crc32.sv (324 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // CRC32 calculator
6 //
7 // This module takes in n-bits data words (n defined by Byte-
PerWord parameter) and updates an
8 // internally stored CRC with each valid data word. The poly-
nomial used is the standard CRC32 IEEE
9 // one. An interface is provided to set the internal CRC to an ar-
bitrary value. The output CRC is an
10 // inverted version of the internally stored CRC and the input
CRC is inverted before being stored.
11 // This is done so results match existing widely used software
libraries (e.g. the crc32
12 // functionality available in Python). Note that a initial CRC of
0x0 (corresponding to an internal
13 // CRC of 0xffffffff) must be used to match results generated
elsewhere.
14
15 module prim_crc32 #(
16 parameter int unsigned BytesPerWord = 4
17 ) (
18 input logic clk_i,
19 input logic rst_ni,
20
21 input logic set_crc_i,
22 input logic [31:0] crc_in_i,
23
24 input logic data_valid_i,
25 input logic [BytesPerWord*8-1:0] data_i,
26
27 output logic [31:0] crc_out_o
28 );
29 // Generated using hw/ip/prim/util/prim_crc32_table_gen.py
30 function automatic logic [31:0] crc32_byte_calc(logic [7:0]
b);
31 unique case (b)
32 8'hff: crc32_byte_calc = 32'h2d02ef8d;
33 8'hfe: crc32_byte_calc = 32'h5a05df1b;
34 8'hfd: crc32_byte_calc = 32'h30c8ea1;
35 8'hfc: crc32_byte_calc = 32'h40bbe37;
36 8'hfb: crc32_byte_calc = 32'h2a6f2b94;
37 8'hfa: crc32_byte_calc = 32'h5d681b02;
38 8'hf9: crc32_byte_calc = 32'hc4614ab8;
39 8'hf8: crc32_byte_calc = 32'h3667a2e;
40 8'hf7: crc32_byte_calc = 32'h23d967bf;
41 8'hf6: crc32_byte_calc = 32'h54de5729;
42 8'hf5: crc32_byte_calc = 32'hcd70693;
43 8'hf4: crc32_byte_calc = 32'hbad03605;
44 8'hf3: crc32_byte_calc = 32'h24b4a3a6;
45 8'hf2: crc32_byte_calc = 32'h53b39330;
46 8'hf1: crc32_byte_calc = 32'hcabac28a;
47 8'hf0: crc32_byte_calc = 32'hbdbdf21c;
48 8'hef: crc32_byte_calc = 32'h30b5ffe9;
49 8'hee: crc32_byte_calc = 32'h47b2cf7f;
50 8'hed: crc32_byte_calc = 32'hdebb9ec5;
51 8'hec: crc32_byte_calc = 32'ha9bcae53;
52 8'heb: crc32_byte_calc = 32'h37d83bf0;
53 8'hea: crc32_byte_calc = 32'h40df0b66;
54 8'he9: crc32_byte_calc = 32'hd9d65adc;
55 8'he8: crc32_byte_calc = 32'haed16a4a;
56 8'he7: crc32_byte_calc = 32'h3e6e77db;
57 8'he6: crc32_byte_calc = 32'h4969474d;
58 8'he5: crc32_byte_calc = 32'hd06016f7;
59 8'he4: crc32_byte_calc = 32'ha7672661;
60 8'he3: crc32_byte_calc = 32'h3903b3c2;
61 8'he2: crc32_byte_calc = 32'h4e048354;
62 8'he1: crc32_byte_calc = 32'hd70dd2ee;
63 8'he0: crc32_byte_calc = 32'ha00ae278;
64 8'hdf: crc32_byte_calc = 32'h166ccf45;
65 8'hde: crc32_byte_calc = 32'h616bffd3;
66 8'hdd: crc32_byte_calc = 32'hf862ae69;
67 8'hdc: crc32_byte_calc = 32'h8f659eff;

```

69	8'hdb:	crc32_byte_calc = 32'h11010b5c;	170	8'h76:	crc32_byte_calc = 32'hb966d409;
70	8'hda:	crc32_byte_calc = 32'h66063bca;	171	8'h75:	crc32_byte_calc = 32'h206f85b3;
71	8'hd9:	crc32_byte_calc = 32'hff0f6a70;	172	8'h74:	crc32_byte_calc = 32'h5768b525;
72	8'hd8:	crc32_byte_calc = 32'h80805ae6;	173	8'h73:	crc32_byte_calc = 32'hc902086;
73	8'hd7:	crc32_byte_calc = 32'h18b74777;	174	8'h72:	crc32_byte_calc = 32'hbe0b1010;
74	8'hd6:	crc32_byte_calc = 32'h6fb077e1;	175	8'h71:	crc32_byte_calc = 32'h270241aa;
75	8'hd5:	crc32_byte_calc = 32'hf6b9265b;	176	8'h70:	crc32_byte_calc = 32'h5005713c;
76	8'hd4:	crc32_byte_calc = 32'h81be16cd;	177	8'h6f:	crc32_byte_calc = 32'hdd0d7cc9;
77	8'hd3:	crc32_byte_calc = 32'h1fda836e;	178	8'h6e:	crc32_byte_calc = 32'haa0a4c5f;
78	8'hd2:	crc32_byte_calc = 32'h68ddb3f8;	179	8'h6d:	crc32_byte_calc = 32'h33031de5;
79	8'hd1:	crc32_byte_calc = 32'hf1d4e242;	180	8'h6c:	crc32_byte_calc = 32'h44042d73;
80	8'hd0:	crc32_byte_calc = 32'h86d3d2d4;	181	8'h6b:	crc32_byte_calc = 32'hda60bd0;
81	8'hcf:	crc32_byte_calc = 32'h0bdbdf21;	182	8'h6a:	crc32_byte_calc = 32'had678846;
82	8'hce:	crc32_byte_calc = 32'h7cdcefb7;	183	8'h69:	crc32_byte_calc = 32'h346ed9fc;
83	8'hcd:	crc32_byte_calc = 32'he5d5be0d;	184	8'h68:	crc32_byte_calc = 32'h4369e96a;
84	8'hcc:	crc32_byte_calc = 32'h92d28e9b;	185	8'h67:	crc32_byte_calc = 32'hd3d6f4fb;
85	8'hcb:	crc32_byte_calc = 32'h0cb61b38;	186	8'h66:	crc32_byte_calc = 32'ha4d1c46d;
86	8'hca:	crc32_byte_calc = 32'h7bb12bae;	187	8'h65:	crc32_byte_calc = 32'h3dd895d7;
87	8'hc9:	crc32_byte_calc = 32'he2b87a14;	188	8'h64:	crc32_byte_calc = 32'h4adf5a51;
88	8'hc8:	crc32_byte_calc = 32'h95bf4a82;	189	8'h63:	crc32_byte_calc = 32'hd4bb30e2;
89	8'hc7:	crc32_byte_calc = 32'h05005713;	190	8'h62:	crc32_byte_calc = 32'ha3bc0074;
90	8'hc6:	crc32_byte_calc = 32'h72076785;	191	8'h61:	crc32_byte_calc = 32'h3ab551ce;
91	8'hc5:	crc32_byte_calc = 32'hebb0e363f;	192	8'h60:	crc32_byte_calc = 32'h4db26158;
92	8'hc4:	crc32_byte_calc = 32'h9c0906a9;	193	8'h5f:	crc32_byte_calc = 32'hfbd44c65;
93	8'hc3:	crc32_byte_calc = 32'h026d930a;	194	8'h5e:	crc32_byte_calc = 32'h8cd37cf3;
94	8'hc2:	crc32_byte_calc = 32'h756aa39c;	195	8'h5d:	crc32_byte_calc = 32'h15da2d49;
95	8'hc1:	crc32_byte_calc = 32'h6c3f226;	196	8'h5c:	crc32_byte_calc = 32'h62dd1ddf;
96	8'hc0:	crc32_byte_calc = 32'h9b64c2b0;	197	8'h5b:	crc32_byte_calc = 32'hfc9887c;
97	8'hbf:	crc32_byte_calc = 32'h5bdeae1d;	198	8'h5a:	crc32_byte_calc = 32'h8bbeb8ea;
98	8'hbe:	crc32_byte_calc = 32'h2cd99e8b;	199	8'h59:	crc32_byte_calc = 32'h12b7e950;
99	8'hbd:	crc32_byte_calc = 32'hb5d0cf31;	200	8'h58:	crc32_byte_calc = 32'h65b0d9c6;
100	8'hbc:	crc32_byte_calc = 32'hc2d7ffa7;	201	8'h57:	crc32_byte_calc = 32'hf50fc457;
101	8'hbb:	crc32_byte_calc = 32'h5cb36a04;	202	8'h56:	crc32_byte_calc = 32'h8208f4c1;
102	8'hba:	crc32_byte_calc = 32'h2bb45a92;	203	8'h55:	crc32_byte_calc = 32'h1b01a57b;
103	8'hb9:	crc32_byte_calc = 32'hb2bd0b28;	204	8'h54:	crc32_byte_calc = 32'h6c0695ed;
104	8'hb8:	crc32_byte_calc = 32'hc5ba3bbe;	205	8'h53:	crc32_byte_calc = 32'hf22004e;
105	8'hb7:	crc32_byte_calc = 32'h5505262f;	206	8'h52:	crc32_byte_calc = 32'h856530d8;
106	8'hb6:	crc32_byte_calc = 32'h202016b9;	207	8'h51:	crc32_byte_calc = 32'h1c6c6162;
107	8'hb5:	crc32_byte_calc = 32'hbb0b4703;	208	8'h50:	crc32_byte_calc = 32'h6b6b51f4;
108	8'hb4:	crc32_byte_calc = 32'hcc0c7795;	209	8'h4f:	crc32_byte_calc = 32'he6635c01;
109	8'hb3:	crc32_byte_calc = 32'h5268e236;	210	8'h4e:	crc32_byte_calc = 32'h91646c97;
110	8'hb2:	crc32_byte_calc = 32'h256fd2a0;	211	8'h4d:	crc32_byte_calc = 32'h086d3d2d;
111	8'hb1:	crc32_byte_calc = 32'hbc66831a;	212	8'h4c:	crc32_byte_calc = 32'h7f6a0dbb;
112	8'hb0:	crc32_byte_calc = 32'hcb61b38c;	213	8'h4b:	crc32_byte_calc = 32'he10e9818;
113	8'haf:	crc32_byte_calc = 32'h4669be79;	214	8'h4a:	crc32_byte_calc = 32'h9609a88e;
114	8'hae:	crc32_byte_calc = 32'h316e8eef;	215	8'h49:	crc32_byte_calc = 32'h0f00f934;
115	8'had:	crc32_byte_calc = 32'ha867df55;	216	8'h48:	crc32_byte_calc = 32'h7807c9a2;
116	8'hac:	crc32_byte_calc = 32'hdf60efc3;	217	8'h47:	crc32_byte_calc = 32'he8b8d433;
117	8'hab:	crc32_byte_calc = 32'h41047a60;	218	8'h46:	crc32_byte_calc = 32'h9fbf4e4a;
118	8'haa:	crc32_byte_calc = 32'h36034af6;	219	8'h45:	crc32_byte_calc = 32'h066b51f;
119	8'ha9:	crc32_byte_calc = 32'haf0a1b4c;	220	8'h44:	crc32_byte_calc = 32'h71b18589;
120	8'ha8:	crc32_byte_calc = 32'hd80d2bda;	221	8'h43:	crc32_byte_calc = 32'h6fd5102a;
121	8'ha7:	crc32_byte_calc = 32'h48b2364b;	222	8'h42:	crc32_byte_calc = 32'h98d220bc;
122	8'ha6:	crc32_byte_calc = 32'h3fb506dd;	223	8'h41:	crc32_byte_calc = 32'h01db7106;
123	8'ha5:	crc32_byte_calc = 32'ha6bc5767;	224	8'h40:	crc32_byte_calc = 32'h76dc4190;
124	8'ha4:	crc32_byte_calc = 32'hd1bb67f1;	225	8'h3f:	crc32_byte_calc = 32'hb6662d3d;
125	8'ha3:	crc32_byte_calc = 32'h4fdff252;	226	8'h3e:	crc32_byte_calc = 32'hc1611dab;
126	8'ha2:	crc32_byte_calc = 32'h38d8c2c4;	227	8'h3d:	crc32_byte_calc = 32'h58684c11;
127	8'ha1:	crc32_byte_calc = 32'ha1d1937e;	228	8'h3c:	crc32_byte_calc = 32'h2f6f7c87;
128	8'ha0:	crc32_byte_calc = 32'hd6d6a3e8;	229	8'h3b:	crc32_byte_calc = 32'h10be924;
129	8'h9f:	crc32_byte_calc = 32'h0b080ed5;	230	8'h3a:	crc32_byte_calc = 32'hc60cd9b2;
130	8'h9e:	crc32_byte_calc = 32'h17b7be43;	231	8'h39:	crc32_byte_calc = 32'h5f058808;
131	8'h9d:	crc32_byte_calc = 32'h8ebeeef9;	232	8'h38:	crc32_byte_calc = 32'h2802b89e;
132	8'h9c:	crc32_byte_calc = 32'hf9b9d6f6;	233	8'h37:	crc32_byte_calc = 32'hb8bda50f;
133	8'h9b:	crc32_byte_calc = 32'h67dd4acc;	234	8'h36:	crc32_byte_calc = 32'hcfba9599;
134	8'h9a:	crc32_byte_calc = 32'h10da7a5a;	235	8'h35:	crc32_byte_calc = 32'h56b3c423;
135	8'h99:	crc32_byte_calc = 32'h89d32be0;	236	8'h34:	crc32_byte_calc = 32'h21b4f4b5;
136	8'h98:	crc32_byte_calc = 32'hfed41b76;	237	8'h33:	crc32_byte_calc = 32'hbfd06116;
137	8'h97:	crc32_byte_calc = 32'h66eb06e7;	238	8'h32:	crc32_byte_calc = 32'hc8d75180;
138	8'h96:	crc32_byte_calc = 32'h196c3671;	239	8'h31:	crc32_byte_calc = 32'h51de003a;
139	8'h95:	crc32_byte_calc = 32'h806567cb;	240	8'h30:	crc32_byte_calc = 32'h26d930ac;
140	8'h94:	crc32_byte_calc = 32'hf762575d;	241	8'h2f:	crc32_byte_calc = 32'habd13d59;
141	8'h93:	crc32_byte_calc = 32'h6906c2fe;	242	8'h2e:	crc32_byte_calc = 32'hdc60dcf;
142	8'h92:	crc32_byte_calc = 32'h1e01f268;	243	8'h2d:	crc32_byte_calc = 32'h45df5c75;
143	8'h91:	crc32_byte_calc = 32'h8708a3d2;	244	8'h2c:	crc32_byte_calc = 32'h32d86ce3;
144	8'h90:	crc32_byte_calc = 32'hf00f9344;	245	8'h2b:	crc32_byte_calc = 32'hacbcf940;
145	8'h8f:	crc32_byte_calc = 32'h7d079eb1;	246	8'h2a:	crc32_byte_calc = 32'hdbbbc9d6;
146	8'h8e:	crc32_byte_calc = 32'h0a00ae27;	247	8'h29:	crc32_byte_calc = 32'h42b2986c;
147	8'h8d:	crc32_byte_calc = 32'h9309ff9d;	248	8'h28:	crc32_byte_calc = 32'h35b5a8fa;
148	8'h8c:	crc32_byte_calc = 32'he40ecf0b;	249	8'h27:	crc32_byte_calc = 32'ha50ab56b;
149	8'h8b:	crc32_byte_calc = 32'h7a6a5aa8;	250	8'h26:	crc32_byte_calc = 32'hd20d85fd;
150	8'h8a:	crc32_byte_calc = 32'h0d6d6a3e;	251	8'h25:	crc32_byte_calc = 32'h4b04d447;
151	8'h89:	crc32_byte_calc = 32'h94643b84;	252	8'h24:	crc32_byte_calc = 32'h3c03e4d1;
152	8'h88:	crc32_byte_calc = 32'he3630b12;	253	8'h23:	crc32_byte_calc = 32'ha2677172;
153	8'h87:	crc32_byte_calc = 32'h73dc1683;	254	8'h22:	crc32_byte_calc = 32'hd56041e4;
154	8'h86:	crc32_byte_calc = 32'h04db2615;	255	8'h21:	crc32_byte_calc = 32'h4c69105e;
155	8'h85:	crc32_byte_calc = 32'h9dd277af;	256	8'h20:	crc32_byte_calc = 32'h3b6e20c8;
156	8'h84:	crc32_byte_calc = 32'head54739;	257	8'h1f:	crc32_byte_calc = 32'h8d080df5;
157	8'h83:	crc32_byte_calc = 32'h74b1d29a;	258	8'h1e:	crc32_byte_calc = 32'hfa0f3d63;
158	8'h82:	crc32_byte_calc = 32'h03b6e20c;	259	8'h1d:	crc32_byte_calc = 32'h63066cd9;
159	8'h81:	crc32_byte_calc = 32'h9abfb3b6;	260	8'h1c:	crc32_byte_calc = 32'h14015c4f;
160	8'h80:	crc32_byte_calc = 32'hedb88320;	261	8'h1b:	crc32_byte_calc = 32'h8a65c9ec;
161	8'h7f:	crc32_byte_calc = 32'h0ba6cad;	262	8'h1a:	crc32_byte_calc = 32'hfd62f97a;
162	8'h7e:	crc32_byte_calc = 32'hb7bd5c3b;	263	8'h19:	crc32_byte_calc = 32'h646ba8c0;
163	8'h7d:	crc32_byte_calc = 32'h2eb40d81;	264	8'h18:	crc32_byte_calc = 32'h136c9856;
164	8'h7c:	crc32_byte_calc = 32'h59b33d17;	265	8'h17:	crc32_byte_calc = 32'h83d38c57;
165	8'h7b:	crc32_byte_calc = 32'hc7d7a8b4;	266	8'h16:	crc32_byte_calc = 32'hf4d4b551;
166	8'h7a:	crc32_byte_calc = 32'hb0d09822;	267	8'h15:	crc32_byte_calc = 32'h6ddde4eb;
167	8'h79:	crc32_byte_calc = 32'h29d9c998;	268	8'h14:	crc32_byte_calc = 32'h1ada47d;
168	8'h78:	crc32_byte_calc = 32'h5edef90e;	269	8'h13:	crc32_byte_calc = 32'h84be41de;
169	8'h77:	crc32_byte_calc = 32'hce61e49f;	270	8'h12:	crc32_byte_calc = 32'hf3b97148;

```

271      8'h11:  crc32_byte_calc = 32'h6ab020f2;
272      8'h10:  crc32_byte_calc = 32'h1db71064;
273      8'h0f:  crc32_byte_calc = 32'h90bf1d91;
274      8'h0e:  crc32_byte_calc = 32'he7b82d07;
275      8'h0d:  crc32_byte_calc = 32'h7eb17cbd;
276      8'h0c:  crc32_byte_calc = 32'h09b64c2b;
277      8'h0b:  crc32_byte_calc = 32'h97d2d988;
278      8'h0a:  crc32_byte_calc = 32'he0d5e91e;
279      8'h09:  crc32_byte_calc = 32'h79dcb8a4;
280      8'h08:  crc32_byte_calc = 32'h0edb8832;
281      8'h07:  crc32_byte_calc = 32'h9e6495a3;
282      8'h06:  crc32_byte_calc = 32'he963a535;
283      8'h05:  crc32_byte_calc = 32'h706af48f;
284      8'h04:  crc32_byte_calc = 32'h076dc419;
285      8'h03:  crc32_byte_calc = 32'h990951ba;
286      8'h02:  crc32_byte_calc = 32'hee0e612c;
287      8'h01:  crc32_byte_calc = 32'h77073096;
288      8'h00:  crc32_byte_calc = 32'h00000000;
289      default: crc32_byte_calc = '0;
290    endcase
291  endfunction
292
293  logic [31:0] crc_d, crc_q;
294  logic        crc_en;
295  logic [31:0] crc_stages[BytesPerWord + 1];
296
297  assign crc_en = set_crc_i | data_valid_i;
298
299  assign crc_stages[0] = crc_q;
300
301  for (genvar i = 0; i < BytesPerWord; ++i) begin :
g_crc_stages
302    assign crc_stages[i + 1] =
303      {8'h00, crc_stages[i][31:8]} ^
304      crc32_byte_calc(crc_stages[i][7:0] ^ data_i[i * 8 +:
8]);
305  end
306
307  always_comb begin
308    if (set_crc_i) begin
309      crc_d = ~crc_in_i;
310    end else begin
311      crc_d = crc_stages[BytesPerWord];
312    end
313  end
314
315  always_ff @(posedge clk_i or negedge rst_ni) begin
316    if (!rst_ni) begin
317      crc_q <= 32'hFFFFFFF;
318    end else if (crc_en) begin
319      crc_q <= crc_d;
320    end
321  end
322
323  assign crc_out_o = ~crc_q;
324 endmodule

```

23. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_decode.sv (333 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // This module decodes a differentially encoded signal and de-
fects
6 // incorrectly encoded differential states.
7 //
8 // In case the differential pair crosses an asynchronous
boundary, it has
9 // to be re-synchronized to the local clock. This can be
achieved by
10 // setting the AsyncOn parameter to 1'b1. In that case, two
additional
11 // input registers are added (to counteract metastability),
and
12 // a pattern detector is instantiated that detects skewed
level changes on
13 // the differential pair (i.e., when level changes on the diff
pair are
14 // sampled one cycle apart due to a timing skew between the
two wires).
15 //
16 // See also: prim_alert_sender, prim_alert_receiver,
alert_handler
17
18 `include "prim_assert.sv"
19
20 module prim_diff_decode #(
21   // enables additional synchronization logic
22   parameter bit AsyncOn = 1'b0,
23   // Number of cycles a differential skew is tolerated before
a signal integrity issue is flagged.
24   // Only has an effect if AsyncOn = 1
25   // 0 means no skew is tolerated (any mismatch is an immedi-
ate signal integrity error).
26   // 1 means a one-cycle skew is tolerated.

```

```

27 // Values larger than 1 are also supported.
28 parameter int unsigned SkewCycles = 1
29 ) (
30   input        clk_i,
31   input        rst_ni,
32   // input diff pair
33   input        diff_pi,
34   input        diff_ni,
35   // logical level and
36   // detected edges
37   output logic level_o,
38   output logic rise_o,
39   output logic fall_o,
40   // either rise or fall
41   output logic event_o,
42   //signal integrity issue detected
43   output logic sigint_o
44 );
45
46 logic level_d, level_q;
47
48 ///////////////////////////////////////////////////
49 // synchronization regs for incoming diff pair (if re-
quired) //
50 ///////////////////////////////////////////////////
51
52 if (AsyncOn) begin : gen_async
53   typedef enum logic [1:0] {IsStd, IsSkewing, SigInt}
state_e;
54   state_e state_d, state_q;
55   logic diff_p_edge, diff_n_edge, diff_check_ok, level;
56
57   // 2 sync regs, one reg for edge detection
58   logic diff_pq, diff_nq, diff_pd, diff_nd;
59
60   // Counter for skew cycles tolerated before flagging an
issue
61   // The width needs to accommodate SkewCycles + 1 to count
up to SkewCycles.
62   logic [prim_util_pkg::vbits(SkewCycles + 1)-1:0]
skew_cnt_d, skew_cnt_q;
63
64   prim_flop_2sync #(
65     .Width(1),
66     .ResetValue('0)
67   ) i_sync_p (
68     .clk_i,
69     .rst_ni,
70     .d_i(diff_pi),
71     .q_o(diff_pd)
72   );
73
74   prim_flop_2sync #(
75     .Width(1),
76     .ResetValue(1'b1)
77   ) i_sync_n (
78     .clk_i,
79     .rst_ni,
80     .d_i(diff_ni),
81     .q_o(diff_nd)
82   );
83
84   // detect level transitions
85   assign diff_p_edge = diff_pq ^ diff_pd;
86   assign diff_n_edge = diff_nq ^ diff_nd;
87
88   // detect sigint issue
89   assign diff_check_ok = diff_pd ^ diff_nd;
90
91   // this is the current logical level
92   assign level = diff_pd;
93
94   // outputs
95   assign level_o = level_d;
96   assign event_o = rise_o | fall_o;
97
98   // sigint detection is a bit more involved in async case
since
99   // we might have skew on the diff pair, which can result
in a
100   // N cycle sampling delay between the two wires
101   // so we need a simple pattern matcher
102   // the following waves are legal
103   // clk | | | | | | | |
104   //
105   // p _____ ... _____
106   //
107   // n _____ ... _____
108   //
109   // p _____ ... _____
110   //
111   // n _____ ... _____
112   //
113   // i.e., level changes may be off by N cycle - which is
permissible
114   // as long as this condition is only N cycle long.
115
116   always_comb begin : p_diff_fsm

```



```

118 // default
119 state_d = state_q;
120 level_d = level_q;
121 skew_cnt_d = skew_cnt_q;
122 rise_o = '1'b0;
123 fall_o = '1'b0;
124 sigint_o = '1'b0;
125
126 unique case (state_q)
127 // we remain here as long as
128 // the diff pair is correctly encoded
129 IsStd: begin
130   if (diff_check_ok) begin
131     level_d = level;
132     if (diff_p_edge && diff_n_edge) begin
133       if (level) begin
134         rise_o = '1'b1;
135       end else begin
136         fall_o = '1'b1;
137       end
138     end
139   end else begin
140     if (SkewCycles == 0) begin
141       // If no skew is tolerated, immediate signal in-
142       // tegrity error
143       state_d = SigInt;
144       sigint_o = '1'b1;
145     end else begin
146       // Mismatch with an edge: likely start of a tol-
147       // erated skew
148       state_d = IsSkewing;
149       skew_cnt_d = 1;
150     end
151   end
152 // diff pair must be correctly encoded, otherwise we
153 // got a sigint
154 IsSkewing: begin
155   if (diff_check_ok) begin
156     state_d = IsStd;
157     level_d = level;
158     // Reset the skew counter
159     skew_cnt_d = '0;
160     // Assert event that was delayed due to skew res-
161     // olution
162     if (level) rise_o = '1'b1;
163     else fall_o = '1'b1;
164   end else begin
165     if (skew_cnt_q < SkewCycles) begin
166       // Still within tolerated skew cycles
167       skew_cnt_d = skew_cnt_q + 1;
168     end else begin
169       // Maximum skew cycles exceeded, raise an integ-
170       // rity issue
171       state_d = SigInt;
172       sigint_o = '1'b1;
173       skew_cnt_d = '0;
174     end
175   end
176 // Signal integrity issue detected, remain here until
177 // resolved
178 SigInt: begin
179   sigint_o = '1'b1;
180   if (diff_check_ok) begin
181     state_d = IsStd;
182     sigint_o = '1'b0;
183     level_d = level;
184     // Assert any event that was pending while in
185     // SigInt
186     if (level) rise_o = '1'b1;
187     else fall_o = '1'b1;
188   end
189   default : ;
190   endcase
191 end
192 always_ff @(posedge clk_i or negedge rst_ni) begin :
193 p_sync_reg
194   if (!rst_ni) begin
195     state_q <= IsStd;
196     diff_pq <= '1'b0;
197     diff_nq <= '1'b1;
198     level_q <= '1'b0;
199     skew_cnt_q <= '0;
200   end else begin
201     state_q <= state_d;
202     diff_pq <= diff_pd;
203     diff_nq <= diff_nd;
204     level_q <= level_d;
205     skew_cnt_q <= skew_cnt_d;
206   end
207 end
208 ///////////////////////////////////////////////////////////////////
209 // fully synchronous case, no skew present in this case //
210 ///////////////////////////////////////////////////////////////////
211 end else begin : gen_no_async
212   logic diff_pq, diff_pd;
213 end

```

```

211 // one reg for edge detection
212 assign diff_pd = diff_pi;
213
214 // Raise a signal integrity error when the differential
215 // signals have equal values. This is
216 // implemented with a `prim_xnor2` instead of behavioral
217 // code to prevent the synthesis tool from
218 // optimizing away combinational logic on the complement-
219 // ary differential signals.
220 prim_xnor2 #(
221   .Width (1)
222 ) u_xnor2_sigint (
223   .in0_i (diff_pi),
224   .in1_i (diff_ni),
225   .out_o (sigint_o)
226 );
227
228 assign level_o = (sigint_o) ? level_q : diff_pi;
229 assign level_d = level_o;
230
231 // detect level transitions
232 assign rise_o = (~diff_pq & diff_pi) & ~sigint_o;
233 assign fall_o = (diff_pq & ~diff_pi) & ~sigint_o;
234 assign event_o = rise_o | fall_o;
235
236 always_ff @(posedge clk_i or negedge rst_ni) begin :
237 p_edge_reg
238   if (!rst_ni) begin
239     diff_pq <= '1'b0;
240     level_q <= '1'b0;
241   end else begin
242     diff_pq <= diff_pd;
243     level_q <= level_d;
244   end
245 end
246
247 // assertions //
248 ///////////////////////////////////////////////////////////////////
249 // shared assertions
250 // sigint -> level stays the same during sigint
251 // $isunknown is needed to avoid false assertion in first
252 // clock cycle
253 `ASSERT(SigintLevelCheck_A, ##1 sigint_o |>-
254 $stable(level_o))
255 // sigint -> no additional events asserted at output
256 `ASSERT(SigintEventCheck_A, sigint_o |>- !event_o)
257 `ASSERT(SigintRiseCheck_A, sigint_o |>- !rise_o)
258 `ASSERT(SigintFallCheck_A, sigint_o |>- !fall_o)
259
260 if (AsyncOn) begin : gen_async_assert
261 // assertions for asynchronous case
262 `ifdef INC_ASSERT
263 `ifndef FPV_ALERT_NO_SIGINT_ERR
264 // Correctly detect signal integrity issue:
265 // If diff_pd and diff_nd are equal for (SkewCycles + 1)
266 // consecutive cycles, sigint_o must be
267 // asserted.
268 `ASSERT(SigintCheck0_A,
269   gen_async.diff_pd == gen_async.diff_nd [* (Skew-
270 Cycles + 1)] |>- sigint_o)
271
272 // The following assertions (SigintCheck1_A to
273 SigintCheck4_A) describe specific
274 // 1-cycle skew patterns that should lead to an edge.
275 // These are highly
276 // specific to SkewCycles = 1. Therefore, they are only
277 // included when SkewCycles is 1.
278 // The synchronizer adds 2 cycles of latency with respect
279 // to input signals.
280 if (SkewCycles == 1) begin : gen_specific_skew_asserts
281   `ASSERT(SigintCheck1_A,
282     ##1 (gen_async.diff_pd ^ gen_async.diff_nd) &&
283     $stable(gen_async.diff_pd) &&
284     $stable(gen_async.diff_nd) ##1
285     $rose(gen_async.diff_pd) &&
286     $stable(gen_async.diff_nd) ##1
287     $stable(gen_async.diff_pd) && $fell(gen_a-
288 sync.diff_nd)
289     |>- rise_o)
290   `ASSERT(SigintCheck2_A,
291     ##1 (gen_async.diff_pd ^ gen_async.diff_nd) &&
292     $stable(gen_async.diff_pd) &&
293     $stable(gen_async.diff_nd) ##1
294     $fell(gen_async.diff_pd) &&
295     $stable(gen_async.diff_nd) ##1
296     $stable(gen_async.diff_pd) && $rose(gen_a-
297 sync.diff_nd)
298     |>- fall_o)
299   `ASSERT(SigintCheck3_A,
300     ##1 (gen_async.diff_pd ^ gen_async.diff_nd) &&
301     $stable(gen_async.diff_pd) &&
302     $stable(gen_async.diff_nd) ##1
303     $rose(gen_async.diff_nd) &&
304     $stable(gen_async.diff_pd) ##1
305     $stable(gen_async.diff_nd) && $fell(gen_a-
306 sync.diff_pd)
307     |>- fall_o)
308   `ASSERT(SigintCheck4_A,

```

```

291      ##1 (gen_async.diff_pd ^ gen_async.diff_nd) &&
292      $stable(gen_async.diff_pd) &&
$stable(gen_async.diff_nd) ##1
293      $fell(gen_async.diff_nd) &&
$stable(gen_async.diff_pd) ##1
294      $stable(gen_async.diff_nd) && $rose(gen_a-
sync.diff_pd)
295      |-> rise_o)
296  end
297  `endif
298  // Correctly detect edges: an event should be asserted
within SkewCycles cycles after a valid
299  // transition
300  `ASSERT(RiseCheck_A,
301  !sigint_o ##1 $rose(gen_async.diff_pd) && (gen_a-
sync.diff_pd ^ gen_async.diff_nd) |->
302  ##[0:SkewCycles] rise_o, clk_i, !rst_ni || sigint_o)
303  `ASSERT(FallCheck_A,
304  !sigint_o ##1 $fell(gen_async.diff_pd) && (gen_a-
sync.diff_pd ^ gen_async.diff_nd) |->
305  ##[0:SkewCycles] fall_o, clk_i, !rst_ni || sigint_o)
306  `ASSERT(EventCheck_A,
307  !sigint_o ##1 $changed(gen_async.diff_pd) && (gen_a-
sync.diff_pd ^ gen_async.diff_nd) |->
308  ##[0:SkewCycles] event_o, clk_i, !rst_ni || sigint_o)
309  // Correctly detect level: the output level should match
diff_pd once the differential pair is
310  // stable
311  `ASSERT(LevelCheck0_A,
312  // Stable for SkewCycles + 1 cycles
313  !sigint_o && (gen_async.diff_pd ^ gen_async.diff_nd)
[* (SkewCycles + 1)] |->
314  gen_async.diff_pd == level_o,
315  clk_i, !rst_ni || sigint_o)
316  `endif
317  end else begin : gen_sync_assert
318  // assertions for synchronous case
319
320  `ifndef FPV_ALERT_NO_SIGINT_ERR
321  // correctly detect sigint issue
322  `ASSERT(SigintCheck_A, diff_pi == diff_ni |-> sigint_o)
323  `endif
324
325  // correctly detect edges
326  `ASSERT(RiseCheck_A, ##1 $rose(diff_pi) && (diff_pi ^
diff_ni) |-> rise_o)
327  `ASSERT(FallCheck_A, ##1 $fell(diff_pi) && (diff_pi ^
diff_ni) |-> fall_o)
328  `ASSERT(EventCheck_A, ##1 $changed(diff_pi) && (diff_pi ^
diff_ni) |-> event_o)
329  // correctly detect level
330  `ASSERT(LevelCheck_A, (diff_pi ^ diff_ni) |-> diff_pi ==
level_o)
331  end
332
333 endmodule : prim_diff_decode

```

24. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_encode.sv (61 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // This module encodes a single bit signal to a differentially en-
coded signal.
6 //
7 // In case the differential pair crosses an asynchronous
boundary, it has
8 // to be re-synchronized to the local clock.
9
10 module prim_diff_encode (
11   input logic clk_i,
12   input logic rst_ni,
13   // Single line input signal
14   input logic req_i,
15   // Output diff pair
16   output logic diff_po,
17   output logic diff_no
18 );
19 // Buff the input signal to avoid any optimization
logic req;
20 prim_sec_anchor_buf #(
21   .Width(1)
22 ) u_prim_buf_in_req (
23   .in_i(req_i),
24   .out_o(req)
25 );
26
27 // Differentially encode input
logic diff_p, diff_n;
28 assign diff_p = req;
31 assign diff_n = ~req;
32
33 // This prevents further tool optimizations of the differen-
tial signal.
34 prim_sec_anchor_buf #(

```

```

35   .Width(2)
36 ) u_prim_buf_ack (
37   .in_i({diff_n, diff_p}),
38   .out_o({diff_n_buf, diff_p_buf})
39 );
40
41 // Flop differentially encoded signal at the output
prim_flop #(
42   .Width(1),
43   .ResetValue(1'b0)
44 ) u_diff_p (
45   .clk_i,
46   .rst_ni,
47   .d_i ( diff_p_buf ),
48   .q_o ( diff_po )
49 );
50
51 prim_flop #(
52   .Width(1),
53   .ResetValue(1'b1)
54 ) u_diff_n (
55   .clk_i,
56   .rst_ni,
57   .d_i ( diff_n_buf ),
58   .q_o ( diff_no )
59 );
60
61 endmodule : prim_diff_encode

```

25. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_to_alert.sv (70 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // prim_diff_to_alert: This module receives a differentially
encoded alert request and translates
6 // that to an alert signal with the alert protocol.
7
8 module prim_diff_to_alert #(
9   // AsyncOn: Enables additional synchronization logic within
the alert receiver.
10   parameter bit AsyncOn = 1'b1,
11   // Number of cycles a differential skew is tolerated on the
alert signal
12   parameter int unsigned SkewCycles = 1,
13   // Alert sender will latch the incoming alert event perman-
ently and
14   // keep on sending alert events until the next reset.
15   parameter bit IsFatal = 1'b1
16 ) (
17   input logic clk_i,
18   input logic rst_ni,
19   // Input diff pair (differentially encoded alert signal)
20   input logic diff_pi,
21   input logic diff_ni,
22   // Alert pair (interface signals for the alert protocol)
23   input prim_alert_pkg::alert_rx_t alert_rx_i,
24   output prim_alert_pkg::alert_tx_t alert_tx_o
25 );
26 logic diff_p_sync, diff_n_sync;
27
28 if (AsyncOn) begin : gen_async
29   prim_flop_2sync #(
30     .Width(2),
31     .ResetValue(2'b10)
32   ) u_sync (
33     .clk_i,
34     .rst_ni,
35     .d_i ( {diff_ni, diff_pi} ),
36     .q_o ( {diff_n_sync, diff_p_sync} )
37   );
38 end else begin : gen_sync
39   assign diff_p_sync = diff_pi;
40   assign diff_n_sync = diff_ni;
41 end
42
43 // This prevents further tool optimizations of the differen-
tial signal.
44 logic diff_p_buf, diff_n_buf;
45 prim_sec_anchor_buf #(
46   .Width(2)
47 ) u_prim_buf_ack (
48   .in_i ( {diff_n_sync, diff_p_sync} ),
49   .out_o ( {diff_n_buf, diff_p_buf} )
50 );
51
52 // Treat any positive value on `diff_p_buf` and any negative
value on `diff_n_buf` as alert.
53 logic alert_req;
54 assign alert_req = diff_p_buf | ~diff_n_buf;
55
56 prim_alert_sender #(
57   .AsyncOn(AsyncOn),
58   .SkewCycles(SkewCycles),
59   .IsFatal(IsFatal)
60 ) u_prim_alert_sender (

```

```

61 .clk_i,
62 .rst_ni,
63 .alert_test_i ( 1'b0 ),
64 .alert_req_i ( alert_req ),
65 .alert_ack_o ( ),
66 .alert_state_o ( ),
67 .alert_rx_i,
68 .alert_tx_o
69 );
70 endmodule

```

26. lowRISC_opentitan/hw/ip/prim/rtl/prim_dom_and_2share.sv (173 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Domain-Oriented Masking GF(2) Multiplier with 2-shares
6 // ref: Higher-Order Side-Channel Protected Implementations of
  Keccak
7 // https://eprint.iacr.org/2017/395.pdf
8 //
9 // q0 = a0 & b0 + (a0 & b1 + z)
10 // q1 = a1 & b1 + (a1 & b0 + z)
11 // () ==> registered
12 //
13 // all input should be stable for two clocks
14 // as the output is valid after a clock
15 // For z, it can use other slice from the state
16 // as it is fairly random w.r.t the current inputs.
17 //
18 // General formula of Q in the paper
19 // Qi = t{i,i} + Sig(j>i,d)(t{i,j}+Z{i+j*(j-1)/2}) +
  Sig(j<i,d)(t{i,j}+Z{j+i*(i-1)/2})
20 // for d=1 (NumShare 2 for first order protection)
21 // Q0 = t{0,0} + Sig(j>0,1)(t{0,j}+Z{j*(j-1)/2}) + Sig(j<0,d)
  (...)
22 // = a0&b0 + (a0&b1 + z0 + 0)
23 // Q1 = t{1,1} + sig(j>1,1)(...) + sig(j<1,1)(t{1,j} + Z{j})
24 // = a1&b1 + (0 + a1&b0 + z0)
25
26 `include "prim_assert.sv"
27
28 module prim_dom_and_2share #(
29     parameter int DW = 64, // Input width
30     parameter bit Pipeline = 1'b0 // Enable full pipelining
31 ) (
32     input clk_i,
33     input rst_ni,
34
35     input [DW-1:0] a0_i, // share0 of a
36     input [DW-1:0] a1_i, // share1 of a
37     input [DW-1:0] b0_i, // share0 of b
38     input [DW-1:0] b1_i, // share1 of b
39     input z_valid_i, // random number input validity
40     input [DW-1:0] z_i, // random number
41
42     output logic [DW-1:0] q0_o, // share0 of q
43     output logic [DW-1:0] q1_o, // share1 of q
44     output logic [DW-1:0] prd_o // pseudo-random data for other
  instances
45 );
46
47 logic [DW-1:0] t0_d, t0_q, t1_d, t1_q;
48 logic [DW-1:0] t_a0b0, t_a1b1;
49 logic [DW-1:0] t_a0b0_d, t_a1b1_d;
50 logic [DW-1:0] t_a0b1, t_a1b0;
51
52 // Calculation //
53 // Inner-domain terms
54 // Inner-domain terms
55 assign t_a0b0_d = a0_i & b0_i;
56 assign t_a1b1_d = a1_i & b1_i;
57
58 // Cross-domain terms
59 assign t_a0b1 = a0_i & b1_i;
60 assign t_a1b0 = a1_i & b0_i;
61
62 // Resharing //
63 // Resharing of cross-domain terms
64 // Preserve the logic sequence for XOR not to proceed cross-
  domain AND.
65 prim_xor2 #(
66     .Width ( DW*2 )
67 ) u_prim_xor_t01 (
68     .in0_i ( {t_a0b1, t_a1b0} ),
69     .in1_i ( {z_i, z_i} ),
70     .out_o ( {t0_d, t1_d} )
71 );
72
73 // Register stage
74 prim_flop_en #(

```

```

75     .Width ( DW*2 ),
76     .ResetValue ( '0 )
77 ) u_prim_flop_t01 (
78     .clk_i ( clk_i ),
79     .rst_ni ( rst_ni ),
80     .en_i ( z_valid_i ),
81     .d_i ( {t0_d, t1_d} ),
82     .q_o ( {t0_q, t1_q} )
83 );
84
85 // Optional Pipelining //
86
87 if (Pipeline == 1'b1) begin : gen_inner_domain_regs
88     // Add pipeline registers on inner-domain terms prior to
  integration. This allows accepting new
89     // input data every clock cycle and prevents SCA leakage
  occurring due to the integration of
90     // reshared cross-domain terms with inner-domain terms de-
  rived from different input data.
91
92     logic [DW-1:0] t_a0b0_q, t_a1b1_q;
93     prim_flop_en #(
94         .Width ( DW*2 ),
95         .ResetValue ( '0 )
96     ) u_prim_flop_tab01 (
97         .clk_i ( clk_i ),
98         .rst_ni ( rst_ni ),
99         .en_i ( z_valid_i ),
100        .d_i ( {t_a0b0_d, t_a1b1_d} ),
101        .q_o ( {t_a0b0_q, t_a1b1_q} )
102    );
103    assign t_a0b0 = t_a0b0_q;
104    assign t_a1b1 = t_a1b1_q;
105
106    end else begin : gen_no_inner_domain_regs
107        // Do not add the optional pipeline registers on the in-
  ner-domain terms. This allows to save
108        // some area in case the multiplier does not need to ac-
  cept new data in every cycle. However,
109
110        // this can cause SCA leakage as during the clock cycle in
  which new data arrives, the new
111        // inner-domain terms are integrated with the previous,
  reshared cross-domain terms.
112
113        assign t_a0b0 = t_a0b0_d;
114        assign t_a1b1 = t_a1b1_d;
115    end
116
117 // Integration //
118
119 // Preserve the logic sequence for XOR not to proceed the
  inner-domain AND.
120 prim_xor2 #(
121     .Width ( DW*2 )
122 ) u_prim_xor_q01 (
123     .in0_i ( {t_a0b0, t_a1b1} ),
124     .in1_i ( {t0_q, t1_q} ),
125     .out_o ( {q0_o, q1_o} )
126 );
127
128 // Use intermediate results for remasking computations in
  another instance in the following
129 // clock cycle. Use one share only. Directly use output of
  flops updating with z_valid_i.
130 // t1_q is obtained by remasking t_a1b0 with z_i. Since z_i
  is uniformly distributed and
131 // independent of a1/b0_i, t1_q is also uniformly distrib-
  uted and independent of a1/b0_i.
132 // For details, see Lemma 1 in Canright, "A very compact
  'perfectly masked' S-box for AES
133 // (corrected)" available at https://eprint.iacr.org/
  2009/011.pdf
134 assign prd_o = t1_q;
135
136 // DOM AND should be same as unmasked computation
137 // The correct test sequence will be:
138 // 1. inputs are changed
139 // 2. check if z_valid_i,
140 // 3. at the next cycle, inputs are still stable (assump-
  tion) - only in case Pipeline = 0
141 // 4. and results Q == A & B (assertion)
142
143 // To speed up the FPV process, random value is ready in
  less than or
144 // equal to two cycles.
145 `ASSUME_FPV(RandomReadyInShortTime_A,
146     $changed(a0_i) || $changed(a1_i) || $changed(b0_i) ||
  $changed(b1_i)
147     |-> ##[0:2] z_valid_i,
148     clk_i, !rst_ni)
149
150 if (Pipeline == 0) begin: g_assert_stable
151     // If Pipeline is not set, the computation takes two
  cycles without flop

```

```

160 // crossing the domain. In this case, the signal should be
stable for at
161 // least two cycles.
162 `ASSUME(StableTwoCycles_M,
163 ($changed(a0_i) || $changed(a1_i) || $changed(b0_i) ||
$changed(b1_i)))
164 ##[0:$] z_valid_i |>=
165 $stable(a0_i) && $stable(a1_i) && $stable(b0_i) &&
$stable(b1_i))
166 end
167
168 `ASSERT(UnmaskedAndMatched_A,
169 z_valid_i |>= (q0_o ^ q1_o) ==
170 (($past(a0_i) ^ $past(a1_i)) & ($past(b0_i) ^
$past(b1_i))),
171 clk_i, !rst_ni)
172
173 endmodule

```

27. lowRISC_opentitan/hw/ip/prim/rtl/prim_double_lfsr.sv (113 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Hardened LFSR module that instantiates two LFSRs of the
same type.

6 // The state vector of both LFSRs is constantly checked and an er-
ror is asserted if the
7 // two states are inconsistent.
8
9 module prim_double_lfsr #(
10 // prim_lfsr parameters - refer to prim_lfsr for their mean-
ing/
11 parameter                LfsrType      = "GAL_XOR",
12 parameter int unsigned   LfsrDw       = 32,
13 localparam int unsigned  LfsrIdxDw    = $clog2(LfsrDw),
14 parameter int unsigned   EntropyDw     = 8,
15 parameter int unsigned   StateOutDw    = 8,
16 parameter logic [LfsrDw-1:0] DefaultSeed = LfsrDw'(1),
17 parameter logic [LfsrDw-1:0] CustomCoeffs = '0,
18 parameter bit            StatePermEn   = 1'b0,
19 parameter logic [LfsrDw-1:0] [LfsrIdxDw-1:0] StatePerm = '0,
20 parameter bit            MaxLenSVA     = 1'b1,
21 parameter bit            LockupSVA     = 1'b1,
22 parameter bit            ExtSeedSVA    = 1'b1,
23 parameter bit            NonLinearOut  = 1'b0,
24 // This should only be disabled in special circumstances,
for example

25 // in non-comportable IPs where an error does not trigger an
alert.
26 parameter bit            EnableAlertTriggerSVA = 1
27 ) (
28 input                clk_i,
29 input                rst_ni,
30 input                seed_en_i,
31 input                [LfsrDw-1:0] seed_i,
32 input                lfsr_en_i,
33 input                [EntropyDw-1:0] entropy_i,
34 output logic [StateOutDw-1:0] state_o,
35 // Asserted if the parallel LFSR states are inconsistent.
36 output logic          err_o
37 );
38
39
40 logic [1:0][LfsrDw-1:0] lfsr_state;
41 // We employ redundant LFSRs to guard against FI attacks.
42 for (genvar k = 0; k < 2; k++) begin : gen_double_lfsr
43 // Instantiate size_only buffers to prevent
44 // optimization / merging of redundant logic.
45 logic lfsr_en_buf, seed_en_buf;
46 logic [EntropyDw-1:0] entropy_buf;
47 logic [LfsrDw-1:0] seed_buf, lfsr_state_unbuf;
48 prim_buf #(
49 .Width(EntropyDw + LfsrDw + 2)
50 ) u_prim_buf_input (
51 .in_i({seed_en_i, seed_i, lfsr_en_i, entropy_i}),
52 .out_o({seed_en_buf, seed_buf, lfsr_en_buf, en-
tropy_buf})
53 );
54
55 prim_lfsr #(
56 .LfsrType(LfsrType),
57 .LfsrDw(LfsrDw),
58 .EntropyDw(EntropyDw),
59 // output the full width so that the states can be cross
checked.
60 .StateOutDw(LfsrDw),
61 .DefaultSeed(DefaultSeed),
62 .CustomCoeffs(CustomCoeffs),
63 .StatePermEn(StatePermEn),
64 .StatePerm(StatePerm),
65 .MaxLenSVA(MaxLenSVA),
66 .LockupSVA(LockupSVA),

```

```

67 .ExtSeedSVA(ExtSeedSVA),
68 .NonLinearOut(NonLinearOut)
69 ) u_prim_lfsr (
70 .clk_i,
71 .rst_ni,
72 .seed_en_i ( seed_en_buf ),
73 .seed_i ( seed_buf ),
74 .lfsr_en_i ( lfsr_en_buf ),
75 .entropy_i ( entropy_buf ),
76 .state_o ( lfsr_state_unbuf )
77 );
78
79 prim_buf #(
80 .Width(LfsrDw)
81 ) u_prim_buf_output (
82 .in_i(lfsr_state_unbuf),
83 .out_o(lfsr_state[k])
84 );
85 end
86
87 `ifdef SIMULATION
88 `ifndef VERILATOR
89 // Ensure both LFSRs start off with the same default seed.
if randomized in simulations.
90 initial begin : p_sync_lfsr_default_seed
91 wait (!$isun-
known(gen_double_lfsr[0].u_prim_lfsr.DefaultSeedLocal));
92 wait (!$isun-
known(gen_double_lfsr[1].u_prim_lfsr.DefaultSeedLocal));
93 gen_double_lfsr[1].u_prim_lfsr.DefaultSeedLocal =
94 gen_double_lfsr[0].u_prim_lfsr.DefaultSeedLocal;
95 $display("%m: Updated
gen_double_lfsr[1].u_prim_lfsr.DefaultSeedLocal = 0x%0h",
96 gen_double_lfsr[1].u_prim_lfsr.DefaultSeedLocal);
97 end
98 `endif
99 `endif
100
101 // Output the state from the first LFSR
102 assign state_o = lfsr_state[0][StateOutDw-1:0];
103 assign err_o = lfsr_state[0] != lfsr_state[1];
104
105 // This logic that will be assign to one, when user adds
macro
106 // ASSERT_PRIM_DOUBLE_LFSR_ERROR_TRIGGER_ALERT to check the
error with alert, in case that
107 // prim_double_lfsr is used in design without adding this
assertion check.
108 `ifdef INC_ASSERT
109 logic unused_assert_connected;
110
111 `ASSERT_INIT_NET(AssertConnected_A, unused_assert_connected
=== 1'b1 || !EnableAlertTriggerSVA)
112 `endif
113 endmodule : prim_double_lfsr

```

28. lowRISC_opentitan/hw/ip/prim/rtl/prim_edge_detector.sv (55 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Edge Detector
6
7 module prim_edge_detector #(
8 parameter int unsigned Width = 1,
9
10 parameter logic [Width-1:0] ResetValue = '0,
11
12 // EnSync
13 //
14 // Enable Synchronizer to the input signal.
15 // It is assumed that the input signal is glitch free (re-
gistered input).
16 parameter bit EnSync = 1'b1
17 ) (
18 input clk_i,
19 input rst_ni,
20
21 input                [Width-1:0] d_i,
22 output logic [Width-1:0] q_sync_o,
23
24 output logic [Width-1:0] q_posedge_pulse_o,
25 output logic [Width-1:0] q_negedge_pulse_o
26 );
27
28 logic [Width-1:0] q_sync_d, q_sync_q;
29
30 if (EnSync) begin : g_sync
31 prim_flop_2sync #(
32 .Width(Width),
33 .ResetValue(ResetValue)
34 ) u_sync (
35 .clk_i,
36 .rst_ni,
37 .d_i,
38 .q_o(q_sync_d)

```

```
39     );
40 end : g_sync
41 else begin : g_nosync
42     assign q_sync_d = d_i;
43 end : g_nosync
44
45 assign q_sync_o = q_sync_d;
46
47 always_ff @(posedge clk_i or negedge rst_ni) begin
```

```
48     if (!rst_ni) q_sync_q <= ResetValue;
49     else
50         q_sync_q <= q_sync_d;
51     end
52     assign q_posedge_pulse_o = q_sync_d & ~q_sync_q;
53     assign q_negedge_pulse_o = ~q_sync_d & q_sync_q;
54
55 endmodule : prim_edge_detector
```

Caliptra SHA-512

What this is. SHA-512 from a datacentre root-of-trust design.

Why it exists. Because measurement of firmware needs a collision-resistant digest, and SHA-512 is the width used for attestation in that ecosystem.

Undergraduate review · EECS concepts. Hash construction; 64-bit datapath arithmetic; message scheduling.

Graduate · Practice note. Compare the round structure with the SHA-256 core: the same algorithmic shape at double width shows how word width propagates through a design.

8 files • 4,374 lines

API — sha512.sv

Table 1318. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512.sv — module / interface / package (1개)

이름	종류
sha512	module

Table 1319. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512.sv — parameter (2개)

파라미터	형/폭	기본값
ADDR_WIDTH		32
DATA_WIDTH		64

Table 1320. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512.sv — localparam (5개)

상수	형/폭	값
BLOCK_SIZE		1024
DIG_SIZE		512
BLOCK_NUM_DWORDS		BLOCK_SIZE / DATA_WIDTH
DIG_NUM_DWORDS		DIG_SIZE / DATA_WIDTH
NONCE_NUM_DWORDS		PV_SIZE_OF_NONCE / DATA_WIDTH

Table 1321. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512.sv — 포트 (17개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
cptra_pwrgood	input	wire
cs	input	wire
we	input	wire
address	input	wire [ADDR_WIDTH-1 : 0]

포트	방향	형/폭
write_data	input	wire [DATA_WIDTH-1 : 0]
read_data	output	wire [DATA_WIDTH-1 : 0]
err	output	wire
pv_read	output	pv_read_t
pv_write	output	pv_write_t
pv_rd_resp	input	pv_rd_resp_t
pv_wr_resp	input	pv_wr_resp_t
pcr_signing_hash	output	logic [PCR_HASH_NUM_DWORDS-1:0][DATA_WIDTH-1:0]
error_intr	output	wire
notif_intr	output	wire
debugUnlock_or_scan_mode_switch	input	logic

API — sha512_core.v

Table 1322. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_core.v — module / interface / package (1개)

이름	종류
sha512_core	module

Table 1323. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_core.v — localparam (4개)

상수	형/폭	값
SHA512_ROUNDS		79
CTRL_IDLE		2'h0
CTRL_ROUNDS		2'h1
CTRL_DONE		2'h2

Table 1324. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_core.v — 포트 (12개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
init_cmd	input	wire
next_cmd	input	wire
restore_cmd	input	wire
mode	input	wire [1 : 0]
block_msg	input	wire [1023 : 0]
restore_digest	input	wire [511 : 0]
ready	output	wire

포트	방향	형/폭
digest	output	wire [511 : 0]
digest_valid	output	wire

API — sha512_ctrl.sv

Table 1325. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_ctrl.sv — module / interface / package (1개)

이름	종류
sha512_ctrl	module

Table 1326. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_ctrl.sv — parameter (2개)

파라미터	형/폭	기본값
AHB_DATA_WIDTH		32
AHB_ADDR_WIDTH		32

Table 1327. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_ctrl.sv — 포트 (21개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
cptra_pwrgood	input	wire
haddr_i	input	logic [AHB_ADDR_WIDTH-1:0]
hdata_i	input	logic [AHB_DATA_WIDTH-1:0]
hsel_i	input	logic
hwrite_i	input	logic
hready_i	input	logic
htrans_i	input	logic [1:0]
hsize_i	input	logic [2:0]
hresp_o	output	logic
hreadyout_o	output	logic
hrdata_o	output	logic [AHB_DATA_WIDTH-1:0]
pv_read	output	pv_read_t
pv_write	output	pv_write_t
pv_rd_resp	input	pv_rd_resp_t
pv_wr_resp	input	pv_wr_resp_t
pcr_signing_hash	output	logic [PCR_HASH_NUM_DWORDS-1:0][31:0]
error_intr	output	logic
notif_intr	output	logic
debugUnlock_or_scan_mode_switch	input	logic

API — sha512_h_constants.v**Table 1328. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_h_constants.v — module / interface / package (1개)**

이름	종류
sha512_h_constants	module

Table 1329. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_h_constants.v — 포트 (9개)

포트	방향	형/폭
mode	input	wire [1 : 0]
H0	output	wire [63 : 0]
H1	output	wire [63 : 0]
H2	output	wire [63 : 0]
H3	output	wire [63 : 0]
H4	output	wire [63 : 0]
H5	output	wire [63 : 0]
H6	output	wire [63 : 0]
H7	output	wire [63 : 0]

API — sha512_k_constants.v**Table 1330. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_k_constants.v — module / interface / package (1개)**

이름	종류
sha512_k_constants	module

Table 1331. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_k_constants.v — 포트 (2개)

포트	방향	형/폭
addr	input	wire [6 : 0]
K_val	output	wire [63 : 0]

API — sha512_params_pkg.sv**Table 1332. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_params_pkg.sv — module / interface / package (1개)**

이름	종류
sha512_params_pkg	package

Table 1333. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_params_pkg.sv — localparam (8개)

상수	형/폭	값
SHA512_CORE_NAME0		32'h61327368
SHA512_CORE_NAME1		32'h31322d35

상수	형/폭	값
SHA512_CORE_VERSION0		32'h3830302e
SHA512_CORE_VERSION1		32'h00000000
MODE_SHA_512_224		2'h0
MODE_SHA_512_256		2'h1
MODE_SHA_384		2'h2
MODE_SHA_512		2'h3

API — sha512_reg.sv

Table 1334. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_reg.sv — module / interface / package (1개)

이름	종류
sha512_reg	module

Table 1335. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_reg.sv — 포트 (16개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_cpuif_req	input	wire
s_cpuif_req_is_wr	input	wire
s_cpuif_addr	input	wire [11:0]
s_cpuif_wr_data	input	wire [31:0]
s_cpuif_wr_bitlen	input	wire [31:0]
s_cpuif_req_stall_wr	output	wire
s_cpuif_req_stall_rd	output	wire
s_cpuif_rd_ack	output	wire
s_cpuif_rd_err	output	wire
s_cpuif_rd_data	output	wire [31:0]
s_cpuif_wr_ack	output	wire
s_cpuif_wr_err	output	wire
hwif_in	input	sha512_reg_pkg::sha512_reg__in_t
hwif_out	output	sha512_reg_pkg::sha512_reg__out_t

API — sha512_reg_pkg.sv

Table 1336. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_reg_pkg.sv — module / interface / package (1개)

이름	종류
sha512_reg_pkg	package

Table 1337. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_reg_pkg.sv — localparam (3개)

상수	형/폭	값
SHA512_REG_DATA_WIDTH		32
SHA512_REG_MIN_ADDR_WIDTH		12
SHA512_REG_ADDR_WIDTH		32'd12

1. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512.sv (546 lines)

```

1 //
=====
2 // Updated by Caliptra team to modify data access width
3 // and removing the work factor
4 //
5 // sha512.sv
6 // -----
7 // Top level wrapper for the SHA-512 hash function providing
8 // a simple memory like interface with 32 bit data access.
9 //
10 //
11 // Author: Joachim Strombergson
12 // Copyright (c) 2014, Secworks Sweden AB
13 // All rights reserved.
14 //
15 // Redistribution and use in source and binary forms, with or
16 // without modification, are permitted provided that the fol-
17 // lowing
18 // conditions are met:
19 //
20 // 1. Redistributions of source code must retain the above
21 // copyright
22 // notice, this list of conditions and the following dis-
23 // claimer.
24 //
25 // 2. Redistributions in binary form must reproduce the above
26 // copyright
27 // notice, this list of conditions and the following dis-
28 // claimer in
29 // the documentation and/or other materials provided with
30 // the
31 // distribution.
32 //
33 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
34 // TRIBUTORS
35 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
36 // BUT NOT
37 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
38 // FITNESS
39 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
40 // THE
41 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
42 // INDIRECT,
43 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
44 // (INCLUDING,
45 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
46 // VICES;
47 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
48 // HOWEVER
49 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
50 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
51 // WISE)
52 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
53 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
54 //
55 //
=====
56 module sha512
57 import sha512_reg_pkg::*;
58 import sha512_params_pkg::*;
59 import kv_defines_pkg::*;
60 import pv_defines_pkg::*;
61 #(
62     parameter ADDR_WIDTH = 32,
63     parameter DATA_WIDTH = 64
64 )
65 (
66     // Clock and reset.
67     input wire clk,
68     input wire reset_n,
69     input wire cptra_pwrgood,
70
71     // Control.
72     input wire cs,
73     input wire we,
74
75     // Data ports.
76     input wire [ADDR_WIDTH-1 : 0] address,
77     input wire [DATA_WIDTH-1 : 0] write_data,
78     output wire [DATA_WIDTH-1 : 0] read_data,

```

```

64     output wire err,
65
66     // PV interface
67     output pv_read_t pv_read,
68     output pv_write_t pv_write,
69     input pv_rd_resp_t pv_rd_resp,
70     input pv_wr_resp_t pv_wr_resp,
71
72     output logic [PCR_HASH_NUM_DWORDS-1:0][DATA_WIDTH-1:0]
73     pcr_signing_hash,
74
75     // Interrupts
76     output wire error_intr,
77     output wire notif_intr,
78     input logic debugUnlock_or_scan_mode_switch
79 );
80
81 `include "caliptra_prim_assert.sv"
82 `include "kv_macros.svh"
83
84 //-----
85 // Registers including update variables and write enable.
86 //-----
87
88 reg init_reg;
89 reg next_reg;
90 reg restore_reg;
91 reg ready_reg;
92 reg [1 : 0] mode_reg;
93 logic zeroize_reg;
94 logic last_reg;
95
96 localparam BLOCK_SIZE = 1024;
97 localparam DIG_SIZE = 512;
98
99 localparam BLOCK_NUM_DWORDS = BLOCK_SIZE / DATA_WIDTH;
100 localparam DIG_NUM_DWORDS = DIG_SIZE / DATA_WIDTH;
101 localparam NONCE_NUM_DWORDS = PV_SIZE_OF_NONCE / DATA_WIDTH;
102
103 reg [DATA_WIDTH-1 : 0][BLOCK_NUM_DWORDS-1 : 0]
104 block_reg;
105 reg [DIG_NUM_DWORDS-1 : 0][DATA_WIDTH-1 : 0] di-
106 gest_reg;
107 reg [PV_NUM_DWORDS - 1 : 0][DATA_WIDTH-1 : 0] kv_reg;
108
109 reg [PCR_HASH_NUM_DWORDS - 1 : 0][DATA_WIDTH-1 : 0]
110 pcr_sign_reg;
111 logic [PCR_HASH_NUM_DWORDS-1 : 0][DATA_WIDTH-1 : 0]
112 pcr_sign;
113 logic
114 pcr_sign_we;
115 reg di-
116 gest_valid_reg;
117 logic di-
118 gest_we;
119
120 sha512_reg_in_t hwif_in;
121 sha512_reg_out_t hwif_out;
122 logic read_error, write_error;
123
124 //KV/PV Gasket
125 kv_read_t vault_read;
126 kv_write_t vault_write;
127 kv_rd_resp_t vault_rd_resp;
128 kv_wr_resp_t vault_wr_resp;
129
130 pv_read_t gen_hash_pv_read;
131
132 //interface with client
133 logic kv_src_write_en;
134 logic [4:0] kv_src_write_offset;
135 logic [31:0] kv_src_write_data;
136
137 kv_error_code_e kv_src_error, kv_dest_error;
138 logic kv_src_ready, kv_src_done;
139 logic kv_dest_ready, kv_dest_done;
140
141 logic dest_keyvault;
142 kv_write_ctrl_reg_t kv_write_ctrl_reg;
143 kv_write_ctrl_reg_t kv_write_ctrl_reg_q;
144 kv_read_ctrl_reg_t kv_read_ctrl_reg;
145 kv_read_filter_metrics_t kv_read_metrics;
146 kv_write_filter_metrics_t kv_write_metrics;
147
148

```

```

139 //KV Read Data Present
140 logic kv_read_data_present;
141 logic kv_read_data_present_set, kv_read_data_present_reset;
142 //PCR Hash Extend Function
143 logic pcr_hash_extend_ip;
144 logic pcr_hash_extend_set, pcr_hash_extend_reset;
145 logic [KV_ENTRY_ADDR_W-1:0] hash_extend_entry;
146 logic dest_data_avail;
147 logic [31:0] block_reg_lock, block_reg_lock_nxt;
148 //PCR Gen Hash Function
149 logic gen_hash_start;
150 logic gen_hash_ip;
151 logic gen_hash_init_reg;
152 logic gen_hash_next_reg;
153 logic gen_hash_last_reg;
154 logic gen_hash_block_write_en;
155 logic [4:0] gen_hash_block_write_offset;
156 logic [31:0] gen_hash_block_write_data;
157
158 logic [NONCE_NUM_DWORDS-1 : 0][DATA_WIDTH-1 : 0] pv_nonce;
159 //-----
160 // Wires.
161 //-----
162 wire core_ready;
163 wire [BLOCK_SIZE-1 : 0] core_block;
164 wire [DIG_NUM_DWORDS-1 : 0][DATA_WIDTH-1 : 0] core_digest;
165 wire core_digest_valid;
166 logic [DIG_NUM_DWORDS-1 : 0][DATA_WIDTH-1 : 0] re-
store_digest;
167
168 //-----
169 // Concurrent connectivity for ports etc.
170 //-----
171 assign core_block = { block_reg[00], block_reg[01],
block_reg[02], block_reg[03],
172 block_reg[04], block_reg[05],
block_reg[06], block_reg[07],
173 block_reg[08], block_reg[09],
block_reg[10], block_reg[11],
174 block_reg[12], block_reg[13],
block_reg[14], block_reg[15],
175 block_reg[16], block_reg[17],
block_reg[18], block_reg[19],
176 block_reg[20], block_reg[21],
block_reg[22], block_reg[23],
177 block_reg[24], block_reg[25],
block_reg[26], block_reg[27],
178 block_reg[28], block_reg[29],
block_reg[30], block_reg[31]};
179
180 assign err = read_error | write_error;
181
182 always_comb begin // ecc_reg_writing
183   for (int dword=0; dword < PCR_HASH_NUM_DWORDS; dword+
+begin
184     pcr_signing_hash[dword] =
pcr_sign_reg[(PCR_HASH_NUM_DWORDS-1)-dword];
185   end
186 end
187
188 //-----
189 // core instantiation.
190 //-----
191 sha512_core core(
192   .clk(clk),
193   .reset_n(reset_n),
194   .zeroize(zeroize_reg),
195
196   .init_cmd(init_reg),
197   .next_cmd(next_reg),
198   .restore_cmd(restore_reg),
199   .mode(mode_reg),
200
201   .block_msg(core_block),
202   .restore_digest(restore_digest),
203
204   .ready(core_ready),
205
206   .digest(core_digest),
207   .digest_valid(core_digest_valid)
208 );
209
210 //-----
211 // reg_update
212 //
213 // Update functionality for all registers in the core.
214 // All registers are positive edge triggered with asynchron-
ous
215 // active low reset. All registers have write enable.
216 //-----
217
218 always @ (posedge clk or negedge reset_n) begin : reg_update

```

```

219 if (!reset_n) begin
220   ready_reg <= '0;
221   digest_reg <= '0;
222   digest_valid_reg <= '0;
223   kv_reg <= '0;
224   pcr_sign_reg <= '0;
225   block_reg_lock <= '0;
226   pcr_hash_extend_ip <= '0;
227   hash_extend_entry <= '0;
228   kv_read_data_present <= '0;
229   digest_we <= '0;
230 end
231 else if (zeroize_reg) begin
232   ready_reg <= '0;
233   digest_reg <= '0;
234   digest_valid_reg <= '0;
235   kv_reg <= '0;
236   pcr_sign_reg <= '0;
237   kv_read_data_present <= '0;
238   digest_we <= '0;
239   block_reg_lock <= '0;
240 end
241 else begin
242   ready_reg <= core_ready;
243   digest_valid_reg <= core_digest_valid;
244
245   if (core_digest_valid & ~digest_valid_reg &
~(dest_keyvault | kv_read_data_present)) begin
246     digest_reg <= core_digest;
247     digest_we <= '1'b1;
248   end
249   else
250     digest_we <= '0;
251   if (core_digest_valid & ~digest_valid_reg &
(dest_keyvault | kv_read_data_present))
252     kv_reg <= core_digest[DIG_NUM_DWORDS-1:DIG_NUM_DWORDS-
PV_NUM_DWORDS];
253   if (pcr_sign_we)
254     pcr_sign_reg <= pcr_sign;
255
256   block_reg_lock <= block_reg_lock_nxt;
257   pcr_hash_extend_ip <= pcr_hash_extend_set ? '1 :
pcr_hash_extend_reset ? '0 :
pcr_hash_extend_ip;
258   hash_extend_entry <= pcr_hash_extend_set ?
kv_read_ctrl_reg.read_entry : hash_extend_entry;
259   kv_read_data_present <= kv_read_data_present_set ? '1 :
kv_read_data_present_reset ?
'0 : kv_read_data_present;
260   end
261   end // reg_update
262
263 always_comb begin
264   pcr_sign_we = (dest_data_avail & gen_hash_ip);
265   pcr_sign = core_digest[DIG_NUM_DWORDS-1:DIG_NUM_DWORDS-
PCR_HASH_NUM_DWORDS];
266 end
267
268 //register hw interface
269 always_comb begin
270
271   hwif_in.SHA512_NAME[0].NAME.next = SHA512_CORE_NAME0;
272   hwif_in.SHA512_NAME[1].NAME.next = SHA512_CORE_NAME1;
273
274   hwif_in.SHA512_VERSION[0].VERSION.next =
SHA512_CORE_VERSION0;
275   hwif_in.SHA512_VERSION[1].VERSION.next =
SHA512_CORE_VERSION1;
276
277   //Mask commands when keyvault is busy to prevent runs with
partial keys.
278   init_reg = gen_hash_ip ? gen_hash_init_reg :
(hwif_out.SHA512_CTRL.INIT.value & kv_src_ready);
279   next_reg = gen_hash_ip ? gen_hash_next_reg :
(hwif_out.SHA512_CTRL.NEXT.value & kv_src_ready);
280   restore_reg = gen_hash_ip ? '0 :
(hwif_out.SHA512_CTRL.RESTORE.value & kv_src_ready);
281   mode_reg = gen_hash_ip ? MODE_SHA_512 :
hwif_out.SHA512_CTRL.MODE.value;
282   zeroize_reg = hwif_out.SHA512_CTRL.ZEROIZE.value || debu-
gUnlock_or_scan_mode_switch;
283   last_reg = gen_hash_ip ? gen_hash_last_reg :
hwif_out.SHA512_CTRL.LAST.value;
284   hwif_in.SHA512_CTRL.LAST.hwclr = core_digest_valid & ~di-
gest_valid_reg;
285
286   hwif_in.SHA512_STATUS.READY.next = ready_reg;
287   hwif_in.SHA512_STATUS.VALID.next = digest_valid_reg;
288
289   //output comes in big endian
290   for (int dword =0; dword < DIG_NUM_DWORDS; dword++) begin
291     restore_digest[dword] =
hwif_out.SHA512_DIGEST[(DIG_NUM_DWORDS-1)-dword].DIGEST.value;
292     hwif_in.SHA512_DIGEST[dword].DIGEST.we = zeroize_reg?
0 : digest_we;
293     hwif_in.SHA512_DIGEST[dword].DIGEST.next = di-
gest_reg[(DIG_NUM_DWORDS-1)-dword];
294     hwif_in.SHA512_DIGEST[dword].DIGEST.hwclr = zeroize_reg;
295   end
296   //output comes in big endian

```

```

299   for (int dword=0; dword < PCR_HASH_NUM_DWORDS; dword++)
begin
300     hwif_in.SHA512_GEN_PCR_HASH_DIGEST[dword].DIGEST.next =
pcr_sign_reg[(PCR_HASH_NUM_DWORDS-1)-dword];

301     hwif_in.SHA512_GEN_PCR_HASH_DIGEST[dword].DIGEST.hwclr =
zeroize_reg;
302   end
303
304   for (int dword=0; dword< BLOCK_NUM_DWORDS; dword++) begin
305     block_reg[dword] =
hwif_out.SHA512_BLOCK[dword].BLOCK.value;
306     hwif_in.SHA512_BLOCK[dword].BLOCK.we = zeroize_reg ?
0 :
307       gen_hash_ip ?
gen_hash_block_write_en & (gen_hash_block_write_offset == dword) :
308       (kv_src_write_en
& (kv_src_write_offset == dword));
309     hwif_in.SHA512_BLOCK[dword].BLOCK.next = gen_hash_ip ?
gen_hash_block_write_data : kv_src_write_data;
310     hwif_in.SHA512_BLOCK[dword].BLOCK.hwclr = zeroize_reg |
kv_read_data_present_reset | (kv_src_error != KV_SUCCESS);
311     //lock the block during gen hash operation
312     hwif_in.SHA512_BLOCK[dword].BLOCK.swwel = gen_hash_ip ?
'1' : block_reg_lock[dword];
313   end
314   //Set valid when fsm is done
315   hwif_in.SHA512_VAULT_RD_STATUS.ERROR.next = kv_src_error;
316   hwif_in.SHA512_KV_WR_STATUS.ERROR.next = kv_dest_error;
317   //Set valid when fsm is done
318   hwif_in.SHA512_VAULT_RD_STATUS.READY.next = kv_src_ready;
319   hwif_in.SHA512_KV_WR_STATUS.READY.next = kv_dest_ready;
320   //Set valid when fsm is done
321   hwif_in.SHA512_VAULT_RD_STATUS.VALID.hwset = kv_src_done;
322   hwif_in.SHA512_KV_WR_STATUS.VALID.hwset = kv_dest_done;
323   //reset valid when fsm is started
324   hwif_in.SHA512_VAULT_RD_STATUS.VALID.hwclr =
kv_read_ctrl_reg.read_en;
325   hwif_in.SHA512_KV_WR_STATUS.VALID.hwclr =
kv_write_ctrl_reg.write_en;
326   //clear en when busy
327   hwif_in.SHA512_VAULT_RD_CTRL.read_en.hwclr =
~kv_src_ready;
328   hwif_in.SHA512_KV_WR_CTRL.write_en.hwclr = ~kv_dest_ready;
329
330   gen_hash_start =
hwif_out.SHA512_GEN_PCR_HASH_CTRL.START.value;
331   hwif_in.SHA512_GEN_PCR_HASH_STATUS.READY.next =
~gen_hash_ip & ~pcr_hash_extend_ip & ready_reg;
332   hwif_in.SHA512_GEN_PCR_HASH_STATUS.VALID.hwset =
gen_hash_ip & dest_data_avail;
333   hwif_in.SHA512_GEN_PCR_HASH_STATUS.VALID.hwclr =
gen_hash_start;
334
335   end
336
337   // Software write-enables to prevent KV reg manipulation
mid-operation
338   always_comb
hwif_in.SHA512_VAULT_RD_CTRL.read_en.swwe = !
kv_read_data_present && ready_reg && !gen_hash_ip;
339   always_comb
hwif_in.SHA512_VAULT_RD_CTRL.read_entry.swwe = !
kv_read_data_present && ready_reg && !gen_hash_ip;
340   always_comb
hwif_in.SHA512_VAULT_RD_CTRL.pcr_hash_extend.swwe = !
kv_read_data_present && ready_reg && !gen_hash_ip;
341   always_comb
hwif_in.SHA512_VAULT_RD_CTRL.rsvd.swwe = 0;
342
343   // KV write control must be written before SHA core opera-
tion begins, even though
344   // output isn't written to KV until the end of the opera-
tion.
345   // Prevent partial-key attacks by blocking register modi-
fications during core execution.
346   always_comb
hwif_in.SHA512_KV_WR_CTRL.write_en.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
347   always_comb
hwif_in.SHA512_KV_WR_CTRL.write_entry.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
348   always_comb
hwif_in.SHA512_KV_WR_CTRL.hmac_key_dest_valid.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
349   always_comb
hwif_in.SHA512_KV_WR_CTRL.hmac_block_dest_valid.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
350   always_comb
hwif_in.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
351   always_comb
hwif_in.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
352   always_comb
hwif_in.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
353   always_comb
hwif_in.SHA512_KV_WR_CTRL.aes_key_dest_valid.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
354   always_comb

```

```

hwif_in.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
355   always_comb
hwif_in.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
356   always_comb
hwif_in.SHA512_KV_WR_CTRL.dma_data_dest_valid.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
357   always_comb
hwif_in.SHA512_KV_WR_CTRL.rsvd.swwe = 0;
358
359   `CALIPTRA_KV_WRITE_CTRL_REG2STRUCT(kv_write_ctrl_reg,
SHA512_KV_WR_CTRL)
360   `CALIPTRA_KV_READ_CTRL_REG2STRUCT(kv_read_ctrl_reg,
SHA512_VAULT_RD_CTRL)
361
362   //Force result into KV reg whenever source came from KV
363   always_comb kv_read_data_present_set =
kv_read_ctrl_reg.read_en & ~kv_read_ctrl_reg.pcr_hash_extend;
364   always_comb kv_read_data_present_reset =
kv_read_data_present & dest_data_avail;
365
366   //Hash extend logic
367   always_comb pcr_hash_extend_set = kv_read_ctrl_reg.read_en &
kv_read_ctrl_reg.pcr_hash_extend;
368   always_comb pcr_hash_extend_reset = pcr_hash_extend_ip &
kv_dest_done;
369
370   //set the lock for the part of the block being written by KV
logic during PCR hash extend
371   //release the lock once init has been seen
372   always_comb begin
373     for (int dword=0; dword< BLOCK_NUM_DWORDS; dword++) begin
374       if (init_reg | next_reg) begin
375         block_reg_lock_nxt[dword] = '0;
376       end
377     else begin
378       //Lock the block for any keyvault data
379       block_reg_lock_nxt[dword] = (kv_src_write_en &
(kv_src_write_offset == dword)) ? '1' : block_reg_lock[dword];
380     end
381   end
382 end
383
384 // Register Block
385 sha512_reg i_sha512_reg (
386   .clk(clk),
387   .rst(1'b0),
388
389   .s_cpuif_req          (cs),
390   .s_cpuif_req_is_wr    (we),
391   .s_cpuif_addr         (address[SHA512_REG_ADDR_WIDTH-1:0]),
392   .s_cpuif_wr_data      (write_data),
393   .s_cpuif_wr_bitlen    ('1),
394   .s_cpuif_req_stall_wr( ),
395   .s_cpuif_req_stall_rd( ),
396   .s_cpuif_rd_ack       ( ),
397   .s_cpuif_rd_err       (read_error),
398   .s_cpuif_rd_data      (read_data),
399   .s_cpuif_wr_ack       ( ),
400   .s_cpuif_wr_err       (write_error),
401
402   .hwif_in (hwif_in ),
403   .hwif_out(hwif_out)
404 );
405
406 //interrupt register hw interface
407 assign hwif_in.reset_b = reset_n;
408 assign hwif_in.error_reset_b = cptra_pwrgood;
409 assign hwif_in.sha512_ready = ready_reg;
410 assign hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset = core_di-
gest_valid & ~digest_valid_reg;
411 assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.error0_sts.hwset = 1'b0; // TODO
412 assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.error1_sts.hwset = 1'b0; // TODO
413 assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset = 1'b0; // TODO
414 assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.error3_sts.hwset = 1'b0; // TODO
415
416 assign error_intr = hwif_out.intr_b-
lock_rf.error_global_intr_r.intr;
417 assign notif_intr = hwif_out.intr_b-
lock_rf.notif_global_intr_r.intr;
418
419 //Read Block
420 always_comb pv_read = gen_hash_ip ? gen_hash_pv_read :
pcr_hash_extend_ip ? vault_read : '0;
421
422 always_comb vault_rd_resp = pv_rd_resp;
423
424
425
426 kv_read_client #(
427   .DATA_WIDTH(BLOCK_SIZE),
428   .PAD(1),
429   .LEN_CHECK(0)
430 )
431 sha512_block_kv_read

```

```

432 (
433     .clk(clk),
434     .rst_b(reset_n),
435     .zeroize(zeroize_reg),
436
437     //client control register
438     .read_ctrl_reg(kv_read_ctrl_reg),
439     .read_metrics(kv_read_metrics),
440
441     //interface with kv
442     .kv_read(vault_read),
443     .kv_resp(vault_rd_resp),
444
445     //interface with client
446     .write_en(kv_src_write_en),
447     .write_offset(kv_src_write_offset),
448     .write_data(kv_src_write_data),
449
450     .error_code(kv_src_error),
451     .kv_ready(kv_src_ready),
452     .read_done(kv_src_done),
453     .check_key_size(1'b0),
454     .expected_key_size('0)
455 );
456
457 always_comb begin
458     pv_write.write_data = pcr_hash_extend_ip ?
vault_write.write_data : '0;
459     pv_write.write_en = pcr_hash_extend_ip ?
vault_write.write_en : '0;
460     pv_write.write_entry = pcr_hash_extend_ip ?
vault_write.write_entry : '0;
461     pv_write.write_offset = pcr_hash_extend_ip ?
vault_write.write_offset : '0;
462 end
463 always_comb vault_wr_resp = pv_wr_resp;
464
465 //during PCR hash extend overload write control
466 //force write enable and always write to the source address
that we read from
467 always_comb begin
468     kv_write_ctrl_reg.q = kv_write_ctrl_reg;
469     kv_write_ctrl_reg.q.write_en = ~pcr_hash_extend_ip ?
kv_write_ctrl_reg.write_en : '1;
470     kv_write_ctrl_reg.q.write_entry = ~pcr_hash_extend_ip ?
kv_write_ctrl_reg.write_entry : hash_extend_entry;
471 end
472
473 //write out the dest data to KV or PCR on last iteration of
SHA
474 always_comb dest_data_avail = core_digest_valid & ~di-
gest_valid_reg & last_reg;
475
476 always_comb begin
477     for (int dword=0; dword< NONCE_NUM_DWORDS; dword++) begin
478         pv_nonce[dword] =
hwif_out.SHA512_GEN_PCR_HASH_NONCE[dword].NONCE.value;
479     end
480 end
481
482 //No filtering rules for SHA512, goes to PCR vault
483 always_comb begin
484     kv_read_metrics = '{default: '0};
485     kv_write_metrics = '{default: '0};
486 end
487
488 kv_write_client #(
489     .DATA_WIDTH(PV_NUM_DWORDS*PV_DATA_W)
490 )
sha512_result_kv_write
491 (
492     .clk(clk),
493     .rst_b(reset_n),
494     .zeroize(zeroize_reg),
495
496     //client control register
497     .write_ctrl_reg(kv_write_ctrl_reg.q),
498     .num_dwords(PV_NUM_DWORDS[4:0]),
499     .write_metrics(kv_write_metrics),
500
501     //interface with kv
502     .kv_write(vault_write),
503     .kv_resp(vault_wr_resp),
504
505     //interface with client
506     .dest_keyvault(dest_keyvault),
507     .dest_data_avail(dest_data_avail),
508     .dest_data(kv_reg),
509
510     .error_code(kv_dest_error),
511     .kv_ready(kv_dest_ready),
512     .dest_done(kv_dest_done)
513 );
514
515
516 pv_gen_hash
517 pv_gen_hash1
518 (
519     .clk(clk),
520     .rst_b(reset_n),
521     .zeroize(zeroize_reg),
522

```

```

523     .core_ready(ready_reg),
524     .core_digest_valid(gen_hash_ip & dest_data_avail),
525
526     .start(gen_hash_start),
527     .nonce(pv_nonce),
528
529     .gen_hash_ip(gen_hash_ip),
530     .gen_hash_init_reg(gen_hash_init_reg),
531     .gen_hash_next_reg(gen_hash_next_reg),
532     .gen_hash_last_reg(gen_hash_last_reg),
533
534     .block_we(gen_hash_block_write_en),
535     .block_offset(gen_hash_block_write_offset),
536     .block_wr_data(gen_hash_block_write_data),
537
538     .pv_read(gen_hash_pv_read),
539     .pv_rd_resp(pv_rd_resp)
540 );
541
542 endmodule // sha512
543
544 //
=====
==
545 // EOF sha512.sv
546 //
=====
==

```

2. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_core.v (609 lines)

```

1 //
=====
==
2 //
3 // sha512_core.v
4 // -----
5 // Verilog 2001 implementation of the SHA-512 hash function.
6 // This is the internal core with wide interfaces.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2014 Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the fol-
lowing
15 // conditions are met:
16 //
17 // 1. Redistributions of source code must retain the above
copyright
18 // notice, this list of conditions and the following dis-
claimer.
19 //
20 // 2. Redistributions in binary form must reproduce the above
copyright
21 // notice, this list of conditions and the following dis-
claimer in
22 // the documentation and/or other materials provided with
the
23 // distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
TRIBUTORS
26 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
27 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
28 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
29 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
30 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
31 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
32 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
33 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
35 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====
==
39
40 module sha512_core(
41     // Clock and reset.
42     input wire clk,
43     input wire reset_n,
44     input wire zeroize,
45     // Control.
46     input wire init_cmd,
47     input wire next_cmd,
48     input wire restore_cmd,
49     input wire [1 : 0] mode,

```

```

50
51          // Data port.
52          input wire [1023 : 0] block_msg,
53          input wire [511 : 0] restore_digest,
54
55          output wire      ready,
56          output wire [511 : 0] digest,
57          output wire      digest_valid
58      );
59
60
61  //-----
62  // Internal constant and parameter definitions.
63  //-----
64  localparam SHA512_ROUNDS = 79;
65
66  localparam CTRL_IDLE  = 2'h0;
67  localparam CTRL_ROUNDS = 2'h1;
68  localparam CTRL_DONE  = 2'h2;
69
70
71  //-----
72  // Registers including update variables and write enable.
73  //-----
74  reg [63 : 0] a_reg;
75  reg [63 : 0] a_new;
76  reg [63 : 0] b_reg;
77  reg [63 : 0] b_new;
78  reg [63 : 0] c_reg;
79  reg [63 : 0] c_new;
80  reg [63 : 0] d_reg;
81  reg [63 : 0] d_new;
82  reg [63 : 0] e_reg;
83  reg [63 : 0] e_new;
84  reg [63 : 0] f_reg;
85  reg [63 : 0] f_new;
86  reg [63 : 0] g_reg;
87  reg [63 : 0] g_new;
88  reg [63 : 0] h_reg;
89  reg [63 : 0] h_new;
90  reg          a_h_we;
91
92  reg [63 : 0] H0_reg;
93  reg [63 : 0] H0_new;
94  reg [63 : 0] H1_reg;
95  reg [63 : 0] H1_new;
96  reg [63 : 0] H2_reg;
97  reg [63 : 0] H2_new;
98  reg [63 : 0] H3_reg;
99  reg [63 : 0] H3_new;
100 reg [63 : 0] H4_reg;
101 reg [63 : 0] H4_new;
102 reg [63 : 0] H5_reg;
103 reg [63 : 0] H5_new;
104 reg [63 : 0] H6_reg;
105 reg [63 : 0] H6_new;
106 reg [63 : 0] H7_reg;
107 reg [63 : 0] H7_new;
108 reg          H_we;
109
110 reg [6 : 0] round_ctr_reg;
111 reg [6 : 0] round_ctr_new;
112 reg          round_ctr_we;
113 reg          round_ctr_inc;
114 reg          round_ctr_rst;
115
116 reg          ready_reg;
117 reg          ready_new;
118 reg          ready_we;
119
120 reg          digest_valid_reg;
121 reg          digest_valid_new;
122 reg          digest_valid_we;
123
124 reg [1 : 0] sha512_ctrl_reg;
125 reg [1 : 0] sha512_ctrl_new;
126 reg          sha512_ctrl_we;
127
128
129 //-----
130 // Wires.
131 //-----
132
133 reg digest_init;
134 reg digest_update;
135
136 reg state_init;
137 reg state_update;
138
139 reg first_block;
140 reg restore;
141
142 reg [63 : 0] t1;
143 reg [63 : 0] t2;
144
145 wire [63 : 0] k_data;

```

```

145
146 reg          w_init;
147 reg          w_next;
148 wire [63 : 0] w_data;
149
150 wire [63 : 0] H0_0;
151 wire [63 : 0] H0_1;
152 wire [63 : 0] H0_2;
153 wire [63 : 0] H0_3;
154 wire [63 : 0] H0_4;
155 wire [63 : 0] H0_5;
156 wire [63 : 0] H0_6;
157 wire [63 : 0] H0_7;
158
159
160 //-----
161 // Module instantiations.
162 //-----
163 sha512_k_constants k_constants_inst(
164     .addr(round_ctr_reg),
165     .K_val(k_data)
166 );
167
168 sha512_h_constants h_constants_inst(
169     .mode(mode),
170
171     .H0(H0_0),
172     .H1(H0_1),
173     .H2(H0_2),
174     .H3(H0_3),
175     .H4(H0_4),
176     .H5(H0_5),
177     .H6(H0_6),
178     .H7(H0_7)
179 );
180
181 sha512_w_mem w_mem_inst(
182     .clk(clk),
183     .reset_n(reset_n),
184     .zeroize(zeroize),
185
186     .block_msg(block_msg),
187
188     .init_cmd(w_init),
189     .next_cmd(w_next),
190     .w_val(w_data)
191 );
192
193
194 //-----
195 // Concurrent connectivity for ports etc.
196 //-----
197
198 assign ready = ready_reg;
199
200 assign digest = {H0_reg, H1_reg, H2_reg, H3_reg,
201                 H4_reg, H5_reg, H6_reg, H7_reg};
202
203 assign digest_valid = digest_valid_reg;
204
205 //-----
206 // reg_update
207 // Update functionality for all registers in the core.
208 // All registers are positive edge triggered with asynchron-
209 // ous
210 // active low reset. All registers have write enable.
211 //-----
212
213 always @ (posedge clk or negedge reset_n)
214     begin : reg_update
215         if (!reset_n)
216             begin
217                 a_reg          <= 64'h0;
218                 b_reg          <= 64'h0;
219                 c_reg          <= 64'h0;
220                 d_reg          <= 64'h0;
221                 e_reg          <= 64'h0;
222                 f_reg          <= 64'h0;
223                 g_reg          <= 64'h0;
224                 h_reg          <= 64'h0;
225                 H0_reg         <= 64'h0;
226                 H1_reg         <= 64'h0;
227                 H2_reg         <= 64'h0;
228                 H3_reg         <= 64'h0;
229                 H4_reg         <= 64'h0;
230                 H5_reg         <= 64'h0;
231                 H6_reg         <= 64'h0;
232                 H7_reg         <= 64'h0;
233                 ready_reg      <= 1'b1;
234                 digest_valid_reg <= 1'b0;
235                 round_ctr_reg  <= 7'h0;
236                 sha512_ctrl_reg <= CTRL_IDLE;
237             end
238         end

```



```

239     else if (zeroize)
240     begin
241         a_reg <= 64'h0;
242         b_reg <= 64'h0;
243         c_reg <= 64'h0;
244         d_reg <= 64'h0;
245         e_reg <= 64'h0;
246         f_reg <= 64'h0;
247         g_reg <= 64'h0;
248         h_reg <= 64'h0;
249         H0_reg <= 64'h0;
250         H1_reg <= 64'h0;
251         H2_reg <= 64'h0;
252         H3_reg <= 64'h0;
253         H4_reg <= 64'h0;
254         H5_reg <= 64'h0;
255         H6_reg <= 64'h0;
256         H7_reg <= 64'h0;
257         ready_reg <= 1'b1;
258         digest_valid_reg <= 1'b0;
259         round_ctr_reg <= 7'h0;
260         sha512_ctrl_reg <= CTRL_IDLE;
261     end
262
263     else
264     begin
265         if (a_h_we)
266         begin
267             a_reg <= a_new;
268             b_reg <= b_new;
269             c_reg <= c_new;
270             d_reg <= d_new;
271             e_reg <= e_new;
272             f_reg <= f_new;
273             g_reg <= g_new;
274             h_reg <= h_new;
275         end
276
277         if (H_we)
278         begin
279             H0_reg <= H0_new;
280             H1_reg <= H1_new;
281             H2_reg <= H2_new;
282             H3_reg <= H3_new;
283             H4_reg <= H4_new;
284             H5_reg <= H5_new;
285             H6_reg <= H6_new;
286             H7_reg <= H7_new;
287         end
288
289         if (round_ctr_we)
290             round_ctr_reg <= round_ctr_new;
291
292         if (ready_we)
293             ready_reg <= ready_new;
294
295         if (digest_valid_we)
296             digest_valid_reg <= digest_valid_new;
297
298         if (sha512_ctrl_we)
299             sha512_ctrl_reg <= sha512_ctrl_new;
300     end
301 end // reg_update
302
303 //-----
304
305 // digest_logic
306 //
307 // The logic needed to init as well as update the digest.
308 //-----
309
310 always @*
311 begin : digest_logic
312     H0_new = 64'h0;
313     H1_new = 64'h0;
314     H2_new = 64'h0;
315     H3_new = 64'h0;
316     H4_new = 64'h0;
317     H5_new = 64'h0;
318     H6_new = 64'h0;
319     H7_new = 64'h0;
320     H_we = 0;
321
322     if (digest_init)
323     begin
324         H0_new = H0_0;
325         H1_new = H0_1;
326         H2_new = H0_2;
327         H3_new = H0_3;
328         H4_new = H0_4;
329         H5_new = H0_5;
330         H6_new = H0_6;
331         H7_new = H0_7;
332         H_we = 1;
333     end
334     else if (restore)
335     begin
336         H0_new = restore_digest[511 : 448];
337         H1_new = restore_digest[447 : 384];
338         H2_new = restore_digest[383 : 320];
339
340         H3_new = restore_digest[319 : 256];
341         H4_new = restore_digest[255 : 192];
342         H5_new = restore_digest[191 : 128];
343         H6_new = restore_digest[127 : 64];
344         H7_new = restore_digest[63 : 0];
345         H_we = 1;
346     end
347
348     if (digest_update)
349     begin
350         H0_new = H0_reg + a_reg;
351         H1_new = H1_reg + b_reg;
352         H2_new = H2_reg + c_reg;
353         H3_new = H3_reg + d_reg;
354         H4_new = H4_reg + e_reg;
355         H5_new = H5_reg + f_reg;
356         H6_new = H6_reg + g_reg;
357         H7_new = H7_reg + h_reg;
358         H_we = 1;
359     end
360 end // digest_logic
361
362 //-----
363 // t1_logic
364 //
365 // The logic for the T1 function.
366 //-----
367
368 always @*
369 begin : t1_logic
370     reg [63 : 0] sum1;
371     reg [63 : 0] ch;
372
373     sum1 = {e_reg[13 : 0], e_reg[63 : 14]} ^
374           {e_reg[17 : 0], e_reg[63 : 18]} ^
375           {e_reg[40 : 0], e_reg[63 : 41]};
376
377     ch = (e_reg & f_reg) ^ ((~e_reg) & g_reg);
378
379     t1 = h_reg + sum1 + ch + k_data + w_data;
380 end // t1_logic
381
382 //-----
383 // t2_logic
384 //
385 // The logic for the T2 function
386 //-----
387
388 always @*
389 begin : t2_logic
390     reg [63 : 0] sum0;
391     reg [63 : 0] maj;
392
393     sum0 = {a_reg[27 : 0], a_reg[63 : 28]} ^
394           {a_reg[33 : 0], a_reg[63 : 34]} ^
395           {a_reg[38 : 0], a_reg[63 : 39]};
396
397     maj = (a_reg & b_reg) ^ (a_reg & c_reg) ^ (b_reg &
398           c_reg);
399
400     t2 = sum0 + maj;
401 end // t2_logic
402
403 //-----
404 // state_logic
405 //
406 // The logic needed to init as well as update the state during
407 // round processing.
408 //-----
409
410 always @*
411 begin : state_logic
412     a_new = 64'h0;
413     b_new = 64'h0;
414     c_new = 64'h0;
415     d_new = 64'h0;
416     e_new = 64'h0;
417     f_new = 64'h0;
418     g_new = 64'h0;
419     h_new = 64'h0;
420     a_h_we = 0;
421
422     if (state_init)
423     begin
424         if (first_block)
425         begin
426             a_new = H0_0;
427             b_new = H0_1;
428             c_new = H0_2;
429             d_new = H0_3;
430             e_new = H0_4;
431             f_new = H0_5;
432             g_new = H0_6;
433             h_new = H0_7;
434         end
435     end
436 end

```



```

431     a_h_we = 1;
432 end
433 else if (restore)
434 begin
435     a_new = restore_digest[511 : 448];
436     b_new = restore_digest[447 : 384];
437     c_new = restore_digest[383 : 320];
438     d_new = restore_digest[319 : 256];
439     e_new = restore_digest[255 : 192];
440     f_new = restore_digest[191 : 128];
441     g_new = restore_digest[127 : 64];
442     h_new = restore_digest[63 : 0];
443     a_h_we = 1;
444 end
445 else
446 begin
447     a_new = H0_reg;
448     b_new = H1_reg;
449     c_new = H2_reg;
450     d_new = H3_reg;
451     e_new = H4_reg;
452     f_new = H5_reg;
453     g_new = H6_reg;
454     h_new = H7_reg;
455     a_h_we = 1;
456 end
457 end
458
459 if (state_update)
460 begin
461     a_new = t1 + t2;
462     b_new = a_reg;
463     c_new = b_reg;
464     d_new = c_reg;
465     e_new = d_reg + t1;
466     f_new = e_reg;
467     g_new = f_reg;
468     h_new = g_reg;
469     a_h_we = 1;
470 end
471 end // state_logic
472
473 //-----
474
475 // round_ctr
476 //
477 // Update logic for the round counter, a monotonically
478 // increasing counter with reset.
479 //-----
480
481 always @*
482 begin : round_ctr
483     round_ctr_new = 7'h00;
484     round_ctr_we = 0;
485
486     if (round_ctr_rst)
487     begin
488         round_ctr_new = 7'h00;
489         round_ctr_we = 1;
490     end
491
492     if (round_ctr_inc)
493     begin
494         round_ctr_new = round_ctr_reg + 1'b1;
495         round_ctr_we = 1;
496     end
497 end // round_ctr
498
499 //-----
500 // sha512_ctrl_fsm
501 //
502 // Logic for the state machine controlling the core beha-
503 // viour.
504 //-----
505
506 always @*
507 begin : sha512_ctrl_fsm
508     digest_init      = 1'b0;
509     digest_update     = 1'b0;
510     state_init       = 1'b0;
511     state_update     = 1'b0;
512     first_block      = 1'b0;
513     w_init           = 1'b0;
514     w_next           = 1'b0;
515     round_ctr_inc    = 1'b0;
516     round_ctr_rst    = 1'b0;
517     digest_valid_new = 1'b0;
518     digest_valid_we  = 1'b0;
519     ready_new        = 1'b0;
520     ready_we         = 1'b0;
521     sha512_ctrl_new  = CTRL_IDLE;
522     sha512_ctrl_we   = 1'b0;
523     restore          = 1'b0;
524
525     unique case (sha512_ctrl_reg)
526         CTRL_IDLE:
527         begin
528             if (init_cmd)

```

```

527         begin
528             ready_new      = 1'b0;
529             ready_we       = 1'b1;
530             digest_init    = 1;
531             w_init         = 1;
532             state_init     = 1;
533             first_block    = 1;
534             round_ctr_rst  = 1;
535             digest_valid_new = 0;
536             digest_valid_we = 1;
537             sha512_ctrl_new = CTRL_ROUNDS;
538             sha512_ctrl_we = 1;
539         end
540
541         if (next_cmd)
542         begin
543             ready_new      = 1'b0;
544             ready_we       = 1'b1;
545             w_init         = 1;
546             state_init     = 1;
547             round_ctr_rst  = 1;
548             digest_valid_new = 0;
549             digest_valid_we = 1;
550             sha512_ctrl_new = CTRL_ROUNDS;
551             sha512_ctrl_we = 1;
552             restore       = restore_cmd;
553         end
554     end
555
556     CTRL_ROUNDS:
557     begin
558         w_next      = 1;
559         state_update = 1;
560         round_ctr_inc = 1;
561
562         if (round_ctr_reg == SHA512_ROUNDS)
563         begin
564             sha512_ctrl_new = CTRL_DONE;
565             sha512_ctrl_we = 1;
566         end
567     end
568
569     CTRL_DONE:
570     begin
571         ready_new      = 1'b1;
572         ready_we       = 1'b1;
573         digest_update   = 1'b1;
574         digest_valid_new = 1'b1;
575         digest_valid_we = 1'b1;
576         sha512_ctrl_new = CTRL_IDLE;
577         sha512_ctrl_we = 1'b1;
578     end
579
580     default:
581     begin
582         digest_init      = 1'b0;
583         digest_update     = 1'b0;
584         state_init       = 1'b0;
585         state_update     = 1'b0;
586         first_block      = 1'b0;
587         w_init           = 1'b0;
588         w_next           = 1'b0;
589         round_ctr_inc    = 1'b0;
590         round_ctr_rst    = 1'b0;
591         digest_valid_new = 1'b0;
592         digest_valid_we  = 1'b0;
593         ready_new        = 1'b0;
594         ready_we         = 1'b0;
595         sha512_ctrl_new  = CTRL_IDLE;
596         sha512_ctrl_we   = 1'b0;
597     end
598
599     endcase // case (sha512_ctrl_reg)
600 end // sha512_ctrl_fsm
601
602 endmodule // sha512_core
603
604
605
606
607 //
=====
608 // EOF sha512_core.v
609 //
=====

```

3. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_ctrl.sv (138 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
4 // cense");
5 // you may not use this file except in compliance with the Li-
6 // cense.
7 // You may obtain a copy of the License at
8 //
9 // http://www.apache.org/licenses/LICENSE-2.0

```

```

8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
==
16 //
17 // sha512_ctrl.sv
18 // -----
19 // SHA512 controller for the AHB_lite interface.
20 //
21 //
22 //
=====
==
23
24 `include "kv_macros.svh"
25
26 module sha512_ctrl
27 import kv_defines_pkg::*;
28 import pv_defines_pkg::*;
29 #(
30 parameter AHB_DATA_WIDTH = 32,
31 parameter AHB_ADDR_WIDTH = 32
32 )
33 (
34 // Clock and reset.
35 input wire          clk,
36 input wire          reset_n,
37 input wire          cptra_pwrgood,
38
39 // from SLAVES PORT
40 input logic [AHB_ADDR_WIDTH-1:0] haddr_i,
41 input logic [AHB_DATA_WIDTH-1:0] hwdata_i,
42 input logic hsel_i,
43 input logic hwrite_i,
44 input logic hready_i,
45 input logic [1:0] htrans_i,
46 input logic [2:0] hsize_i,
47
48 output logic hresp_o,
49 output logic hreadyout_o,
50 output logic [AHB_DATA_WIDTH-1:0] hrdata_o,
51
52 // pcr vault interface
53 output pv_read_t pv_read,
54 output pv_write_t pv_write,
55 input pv_rd_resp_t pv_rd_resp,
56 input pv_wr_resp_t pv_wr_resp,
57 output logic [PCR_HASH_NUM_DWORDS-1:0][31:0] pcr_sign-
ing_hash,
58
59 // Interrupt
60 output logic error_intr,
61 output logic notif_intr,
62 input logic debugUnlock_or_scan_mode_switch
63 );
64
65 //-----
66 // sha512
67 //-----
68
69 logic          sha512_cs;
70 logic          sha512_we;
71 logic [AHB_ADDR_WIDTH - 1 : 0] sha512_address;
72 logic [31 : 0] sha512_write_data;
73 logic [31 : 0] sha512_read_data;
74 logic          sha512_err;
75
76 sha512 #(
77 .ADDR_WIDTH(AHB_ADDR_WIDTH),
78 .DATA_WIDTH(32)
79 )
80 sha512_inst(
81 .clk(clk),
82 .reset_n(reset_n),
83 .cptra_pwrgood(cptra_pwrgood),
84 .cs(sha512_cs),
85 .we(sha512_we),
86 .address(sha512_address),
87 .write_data(sha512_write_data),
88 .read_data(sha512_read_data),
89 .err(sha512_err),
90 .pv_read(pv_read),
91 .pv_write(pv_write),
92 .pv_rd_resp(pv_rd_resp),
93 .pv_wr_resp(pv_wr_resp),
94 .pcr_signing_hash(pcr_signing_hash),
95 .error_intr(error_intr),
96 .notif_intr(notif_intr),
97 .debugUn-
lock_or_scan_mode_switch(debugUnlock_or_scan_mode_switch)

```

```

97 );
98
99 //instantiate ahb lite module
100 ahb_slv_sif #(
101 .AHB_ADDR_WIDTH(AHB_ADDR_WIDTH),
102 .AHB_DATA_WIDTH(AHB_DATA_WIDTH),
103 .CLIENT_DATA_WIDTH(32)
104 ) ahb_slv_sif_uut
105 (
106 //AMBA AHB Lite INF
107 .hclk(clk),
108 .hreset_n(reset_n),
109 .haddr_i(haddr_i),
110 .hwdata_i(hwdata_i),
111 .hsel_i(hsel_i),
112 .hwrite_i(hwrite_i),
113 .hready_i(hready_i),
114 .htrans_i(htrans_i),
115 .hsize_i(hsize_i),
116
117 .hresp_o(hresp_o),
118 .hreadyout_o(hreadyout_o),
119 .hrdata_o(hrdata_o),
120
121
122 //COMPONENT INF
123 .dv(sha512_cs),
124 .hld(1'b0), //no holds from sha512
125 .err(1'b0),
126 .write(sha512_we),
127 .wdata(sha512_write_data),
128 .addr(sha512_address),
129
130 .rdata(sha512_read_data)
131 );
132
133 endmodule // sha512_ctrl
134
135
136 //
=====
==
137 // EOF sha512_ctrl.sv
138 //
=====
==

```

4. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_h_constants.v (156 lines)

```

1 //
=====
==
2 //
3 // sha512_h_constants.v
4 // -----
5 // The H initial constants for the different modes in SHA-512.
6 //
7 //
8 // Author: Joachim Strombergson
9 // Copyright (c) 2014 Secworks Sweden AB
10 // All rights reserved.
11 //
12 // Redistribution and use in source and binary forms, with or
13 // without modification, are permitted provided that the fol-
lowing
14 // conditions are met:
15 //
16 // 1. Redistributions of source code must retain the above
copyright
17 // notice, this list of conditions and the following dis-
claimer.
18 //
19 // 2. Redistributions in binary form must reproduce the above
copyright
20 // notice, this list of conditions and the following dis-
claimer in
21 // the documentation and/or other materials provided with
the
22 // distribution.
23 //
24 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
TRIBUTORS
25 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
26 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
27 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
28 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
29 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
30 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
31 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
32 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
33 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)

```

```

34 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
35 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
36 //
37 //
=====
38
39 module sha512_h_constants(
40     input wire [1 : 0] mode,
41
42     output wire [63 : 0] H0,
43     output wire [63 : 0] H1,
44     output wire [63 : 0] H2,
45     output wire [63 : 0] H3,
46     output wire [63 : 0] H4,
47     output wire [63 : 0] H5,
48     output wire [63 : 0] H6,
49     output wire [63 : 0] H7
50 );
51
52 //-----
53 // Wires.
54 //-----
55
56 reg [63 : 0] tmp_H0;
57 reg [63 : 0] tmp_H1;
58 reg [63 : 0] tmp_H2;
59 reg [63 : 0] tmp_H3;
60 reg [63 : 0] tmp_H4;
61 reg [63 : 0] tmp_H5;
62 reg [63 : 0] tmp_H6;
63 reg [63 : 0] tmp_H7;
64
65 //-----
66 // Concurrent connectivity for ports etc.
67 //-----
68
69 assign H0 = tmp_H0;
70 assign H1 = tmp_H1;
71 assign H2 = tmp_H2;
72 assign H3 = tmp_H3;
73 assign H4 = tmp_H4;
74 assign H5 = tmp_H5;
75 assign H6 = tmp_H6;
76 assign H7 = tmp_H7;
77
78 //-----
79 // mode_mux
80 //
81 // Based on the given mode, the correct H constants are se-
lected.
82 //-----
83
84 always @*
85 begin : mode_mux
86     unique case (mode)
87     0:
88         begin
89             // SHA-512/224
90             tmp_H0 = 64'h8c3d37c819544da2;
91             tmp_H1 = 64'h73e1996689dcd4d6;
92             tmp_H2 = 64'h1dfab7ae32ff9c82;
93             tmp_H3 = 64'h679dd514582f9fcf;
94             tmp_H4 = 64'h0f6d2b697bd44da8;
95             tmp_H5 = 64'h77e36f7304c48942;
96             tmp_H6 = 64'h3f9d85a86a1d36c8;
97             tmp_H7 = 64'h1112e6ad91d692a1;
98         end
99     1:
100         begin
101             // SHA-512/256
102             tmp_H0 = 64'h22312194fc2bf72c;
103             tmp_H1 = 64'h9f555fa3c84c64c2;
104             tmp_H2 = 64'h2393b86b6f53b151;
105             tmp_H3 = 64'h963877195940eabd;
106             tmp_H4 = 64'h96283ee2a88effe3;
107             tmp_H5 = 64'hbe5e1e2553863992;
108             tmp_H6 = 64'h2b0199fc2c85b8aa;
109             tmp_H7 = 64'h0eb72ddc81c52ca2;
110         end
111     2:
112         begin
113             // SHA-384
114             tmp_H0 = 64'hcbbb9d5dc1059ed8;
115             tmp_H1 = 64'h629a292a367cd507;
116             tmp_H2 = 64'h9159015a3070dd17;
117             tmp_H3 = 64'h152fec8f70e5939;
118             tmp_H4 = 64'h6732667ffc00b31;
119             tmp_H5 = 64'h8eb44a8768581511;
120             tmp_H6 = 64'hdb0c2e0d64f98fa7;
121             tmp_H7 = 64'h47b5481dbefa4fa4;
122         end
123     end
124
125 3:

```

```

126     begin
127         // SHA-512
128         tmp_H0 = 64'h6a09e667f3bcc908;
129         tmp_H1 = 64'hbb67ae8584caa73b;
130         tmp_H2 = 64'h3c6ef372fe94f82b;
131         tmp_H3 = 64'ha54ff53a5f1d36f1;
132         tmp_H4 = 64'h510e527fde682d1;
133         tmp_H5 = 64'h9b0e5688c2b3e6c1f;
134         tmp_H6 = 64'h1f83d9abfb41bd6b;
135         tmp_H7 = 64'h5be0cd19137e2179;
136     end
137
138     default:
139         begin
140             // SHA-384
141             tmp_H0 = 64'hcbbb9d5dc1059ed8;
142             tmp_H1 = 64'h629a292a367cd507;
143             tmp_H2 = 64'h9159015a3070dd17;
144             tmp_H3 = 64'h152fec8f70e5939;
145             tmp_H4 = 64'h6732667ffc00b31;
146             tmp_H5 = 64'h8eb44a8768581511;
147             tmp_H6 = 64'hdb0c2e0d64f98fa7;
148             tmp_H7 = 64'h47b5481dbefa4fa4;
149         end
150     endcase // case (addr)
151 end // block: mode_mux
152 endmodule // sha512_h_constants
153
154 //
=====
155 // sha512_h_constants.v
156 //
=====

```

5. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_k_constants.v (151 lines)

```

1 //
=====
2 //
3 // sha512_k_constants.v
4 // -----
5 // The table K with constants in the SHA-512 hash function.
6 //
7 //
8 // Author: Joachim Strombergson
9 // Copyright (c) 2014 Secworks Sweden AB
10 // All rights reserved.
11 //
12 // Redistribution and use in source and binary forms, with or
13 // without modification, are permitted provided that the fol-
lowing
14 // conditions are met:
15 //
16 // 1. Redistributions of source code must retain the above
copyright
17 // notice, this list of conditions and the following dis-
claimer.
18 //
19 // 2. Redistributions in binary form must reproduce the above
copyright
20 // notice, this list of conditions and the following dis-
claimer in
21 // the documentation and/or other materials provided with
the
22 // distribution.
23 //
24 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
TRIBUTORS
25 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
26 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
27 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
28 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
29 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
30 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
31 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
32 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
33 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
34 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
35 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
36 //
37 //
=====
38
39 module sha512_k_constants(
40     input wire [6 : 0] addr,
41     output wire [63 : 0] K_val
42 );

```

```

43
44 //-----
45 // Wires.
46 //-----
47 reg [63 : 0] tmp_K;
48
49
50 //-----
51 // Concurrent connectivity for ports etc.
52 //-----
53 assign K_val = tmp_K;
54
55
56 //-----
57 // addr_mux
58 //-----
59 always @*
60 begin : addr_mux
61   unique case(addr)
62     0: tmp_K = 64'h428a2f98d728ae22;
63     1: tmp_K = 64'h1737449123ef65cd;
64     2: tmp_K = 64'hb5c0fbcfec4d3b2f;
65     3: tmp_K = 64'h9b5dba58189dbbc;
66     4: tmp_K = 64'h3956c25bf348b538;
67     5: tmp_K = 64'h59f111f1b605d019;
68     6: tmp_K = 64'h923f82a4af194f9b;
69     7: tmp_K = 64'hab1c5ed5da6d8118;
70     8: tmp_K = 64'hd807aa98a3030242;
71     9: tmp_K = 64'h12835b0145706fbc;
72    10: tmp_K = 64'h243185be4ee4b28c;
73    11: tmp_K = 64'h550c7dc3d5ffb4e2;
74    12: tmp_K = 64'h72be5d74f27b896f;
75    13: tmp_K = 64'h80deb1fe3b1696b1;
76    14: tmp_K = 64'h9bdc06a725c71235;
77    15: tmp_K = 64'hc19bf174cf692694;
78    16: tmp_K = 64'he49b69c19ef14ad2;
79    17: tmp_K = 64'hef8b4786384f25e3;
80    18: tmp_K = 64'h0fc19dc68b8cd5b5;
81    19: tmp_K = 64'h240ca1cc77ac9c65;
82    20: tmp_K = 64'h2de92c6f592b0275;
83    21: tmp_K = 64'h4a748aa6eae483;
84    22: tmp_K = 64'h5cb0a9dcdb41fbd4;
85    23: tmp_K = 64'h76f988da831153b5;
86    24: tmp_K = 64'h983e5152ee66dfab;
87    25: tmp_K = 64'ha831c66d2db43210;
88    26: tmp_K = 64'hb00327c998fb213f;
89    27: tmp_K = 64'hbf597fc7beef0ee4;
90    28: tmp_K = 64'hc6e00bf33da88fc2;
91    29: tmp_K = 64'hd5a79147930aa725;
92    30: tmp_K = 64'h06ca6351e003826f;
93    31: tmp_K = 64'h142929670a0e6e70;
94    32: tmp_K = 64'h27b70a8546d22ffc;
95    33: tmp_K = 64'h2e1b21385c26c926;
96    34: tmp_K = 64'h4d2c6dfc5ac42aed;
97    35: tmp_K = 64'h53380d139d95b3df;
98    36: tmp_K = 64'h650a73548baf63de;
99    37: tmp_K = 64'h766a0abb3c77b2a8;
100   38: tmp_K = 64'h81c2c92e47edae6;
101   39: tmp_K = 64'h92722c851482353b;
102   40: tmp_K = 64'ha2bfe8a14cf10364;
103   41: tmp_K = 64'ha81a664bb4c23001;
104   42: tmp_K = 64'hc24b8b70d0f89791;
105   43: tmp_K = 64'hc76c51a30654be30;
106   44: tmp_K = 64'hd192e819d6ef5218;
107   45: tmp_K = 64'hd69906245565a910;
108   46: tmp_K = 64'hf40e35855771202a;
109   47: tmp_K = 64'h106aa07032bbd1b8;
110   48: tmp_K = 64'h19a4c116b8d2d0c8;
111   49: tmp_K = 64'h1e376c085141ab53;
112   50: tmp_K = 64'h2748774cdf8eeb99;
113   51: tmp_K = 64'h34b0bcb5e19b48a8;
114   52: tmp_K = 64'h391c0cb3c5c95a63;
115   53: tmp_K = 64'h4ed8aa4ae3418acb;
116   54: tmp_K = 64'h5b9cca4f7763e373;
117   55: tmp_K = 64'h682e6ff3d6b2b8a3;
118   56: tmp_K = 64'h748f82ee5defb2fc;
119   57: tmp_K = 64'h78a5636f43172f60;
120   58: tmp_K = 64'h84c87814a1f0ab72;
121   59: tmp_K = 64'h8cc702081a6439ec;
122   60: tmp_K = 64'h90bafffa23631e28;
123   61: tmp_K = 64'ha4506cebd82bde9;
124   62: tmp_K = 64'hbf9a3f7b2c67915;
125   63: tmp_K = 64'hc67178f2e372532b;
126   64: tmp_K = 64'hca273ceea26619c;
127   65: tmp_K = 64'hd186b8c721c0c207;
128   66: tmp_K = 64'heada7dd6cde0eb1e;
129   67: tmp_K = 64'hf57d4f7fee6ed178;
130   68: tmp_K = 64'h06f067aa72176fba;
131   69: tmp_K = 64'h0a637dc5a2c898a6;
132   70: tmp_K = 64'h113f9804bef90dae;
133   71: tmp_K = 64'h1b710b35131c471b;
134   72: tmp_K = 64'h28db77f523047d84;
135   73: tmp_K = 64'h32caab7b40c72493;
136   74: tmp_K = 64'h3c9ebe0a15c9bebc;
137   75: tmp_K = 64'h431d67c49c100d4c;

```

```

138     76: tmp_K = 64'h4cc5d4becb3e42b6;
139     77: tmp_K = 64'h597f299cfc657e2a;
140     78: tmp_K = 64'h5fcb6fab3ad6faec;
141     79: tmp_K = 64'h6c44198c4a475817;
142
143     default:
144       tmp_K = 64'h0;
145     endcase // case (addr)
146   end // block: addr_mux
147 endmodule // sha512_k_constants
148
149 //
=====
150 // sha512_k_constants.v
151 //
=====

```

6. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_params_pkg.sv (31 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
4 // you may not use this file except in compliance with the Li-
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
10 // distributed under the License is distributed on an "AS IS"
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
12 // See the License for the specific language governing permis-
13 // limitations under the License.
14 //
15 //
16 package sha512_params_pkg;
17
18   localparam SHA512_CORE_NAME0      = 32'h61327368; //
19   localparam SHA512_CORE_NAME1      = 32'h31322d35; //
20   localparam SHA512_CORE_VERSION0    = 32'h3830302e; //
21   localparam SHA512_CORE_VERSION1    = 32'h00000000; //
22
23   localparam MODE_SHA_512_224       = 2'h0;
24   localparam MODE_SHA_512_256       = 2'h1;
25   localparam MODE_SHA_384           = 2'h2;
26   localparam MODE_SHA_512           = 2'h3;
27
28 endpackage
29 //
=====
30 // EOF sha512_params_pkg.sv
31 //
=====

```

7. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_reg.sv (2,297 lines)

```

1 // Generated by PeakRDL-regblock - A free and open-source Sys-
2 // https://github.com/SystemRDL/PeakRDL-regblock
3
4 module sha512_reg (
5   input wire clk,
6   input wire rst,
7
8   input wire s_cpuif_req,
9   input wire s_cpuif_req_is_wr,
10  input wire [11:0] s_cpuif_addr,
11  input wire [31:0] s_cpuif_wr_data,
12  input wire [31:0] s_cpuif_wr_biten,
13  output wire s_cpuif_req_stall_wr,
14  output wire s_cpuif_req_stall_rd,
15  output wire s_cpuif_rd_ack,
16  output wire s_cpuif_rd_err,
17  output wire [31:0] s_cpuif_rd_data,
18  output wire s_cpuif_wr_ack,
19  output wire s_cpuif_wr_err,
20
21  input sha512_reg_pkg::sha512_reg_in_t hwif_in,
22  output sha512_reg_pkg::sha512_reg_out_t hwif_out
23 );
24
25 //-----
26 // CPU Bus interface logic

```

```

27 //-----
28 logic cpuif_req;
29 logic cpuif_req_is_wr;
30 logic [11:0] cpuif_addr;
31 logic [31:0] cpuif_wr_data;
32 logic [31:0] cpuif_wr_biten;
33 logic cpuif_req_stall_wr;
34 logic cpuif_req_stall_rd;
35
36 logic cpuif_rd_ack;
37 logic cpuif_rd_err;
38 logic [31:0] cpuif_rd_data;
39
40 logic cpuif_wr_ack;
41 logic cpuif_wr_err;
42
43 assign cpuif_req = s_cpuif_req;
44 assign cpuif_req_is_wr = s_cpuif_req_is_wr;
45 assign cpuif_addr = s_cpuif_addr;
46 assign cpuif_wr_data = s_cpuif_wr_data;
47 assign cpuif_wr_biten = s_cpuif_wr_biten;
48 assign s_cpuif_req_stall_wr = cpuif_req_stall_wr;
49 assign s_cpuif_req_stall_rd = cpuif_req_stall_rd;
50 assign s_cpuif_rd_ack = cpuif_rd_ack;
51 assign s_cpuif_rd_err = cpuif_rd_err;
52 assign s_cpuif_rd_data = cpuif_rd_data;
53 assign s_cpuif_wr_ack = cpuif_wr_ack;
54 assign s_cpuif_wr_err = cpuif_wr_err;
55
56 logic cpuif_req_masked;
57
58 // Read & write latencies are balanced. Stalls not re-
quired
59 assign cpuif_req_stall_rd = '0;
60 assign cpuif_req_stall_wr = '0;
61 assign cpuif_req_masked = cpuif_req
62 & !(cpuif_req_is_wr &
cpuif_req_stall_rd)
63 & !(cpuif_req_is_wr &
cpuif_req_stall_wr);
64
65 //-----
66 // Address Decode
67 //-----
68 typedef struct packed{
69     logic [2-1:0]SHA512_NAME;
70     logic [2-1:0]SHA512_VERSION;
71     logic SHA512_CTRL;
72     logic SHA512_STATUS;
73     logic [32-1:0]SHA512_BLOCK;
74     logic [16-1:0]SHA512_DIGEST;
75     logic SHA512_VAULT_RD_CTRL;
76     logic SHA512_VAULT_RD_STATUS;
77     logic SHA512_KV_WR_CTRL;
78     logic SHA512_KV_WR_STATUS;
79     logic [8-1:0]SHA512_GEN_PCR_HASH_NONCE;
80     logic SHA512_GEN_PCR_HASH_CTRL;
81     logic SHA512_GEN_PCR_HASH_STATUS;
82     logic [16-1:0]SHA512_GEN_PCR_HASH_DIGEST;
83     struct packed{
84         logic global_intr_en_r;
85         logic error_intr_en_r;
86         logic notif_intr_en_r;
87         logic error_global_intr_r;
88         logic notif_global_intr_r;
89         logic error_internal_intr_r;
90         logic notif_internal_intr_r;
91         logic error_intr_trig_r;
92         logic notif_intr_trig_r;
93         logic error0_intr_count_r;
94         logic error1_intr_count_r;
95         logic error2_intr_count_r;
96         logic error3_intr_count_r;
97         logic notif_cmd_done_intr_count_r;
98         logic error0_intr_count_incr_r;
99         logic error1_intr_count_incr_r;
100        logic error2_intr_count_incr_r;
101        logic error3_intr_count_incr_r;
102        logic notif_cmd_done_intr_count_incr_r;
103    } intr_block_rf;
104 } decoded_reg_strb_t;
105 decoded_reg_strb_t decoded_reg_strb;
106 logic decoded_req;
107 logic decoded_req_is_wr;
108 logic [31:0] decoded_wr_data;
109 logic [31:0] decoded_wr_biten;
110
111 always_comb begin
112     for(int i0=0; i0<2; i0++) begin
113         decoded_reg_strb.SHA512_NAME[i0] =
cpuif_req_masked & (cpuif_addr == 12'h0 + i0*12'h4);
114     end
115     for(int i0=0; i0<2; i0++) begin
116         decoded_reg_strb.SHA512_VERSION[i0] =
cpuif_req_masked & (cpuif_addr == 12'h8 + i0*12'h4);
117     end
118     decoded_reg_strb.SHA512_CTRL = cpuif_req_masked &
(cpuif_addr == 12'h10);

```

```

119     decoded_reg_strb.SHA512_STATUS = cpuif_req_masked &
(cpuif_addr == 12'h18);
120     for(int i0=0; i0<32; i0++) begin
121         decoded_reg_strb.SHA512_BLOCK[i0] =
cpuif_req_masked & (cpuif_addr == 12'h80 + i0*12'h4);
122     end
123     for(int i0=0; i0<16; i0++) begin
124         decoded_reg_strb.SHA512_DIGEST[i0] =
cpuif_req_masked & (cpuif_addr == 12'h100 + i0*12'h4);
125     end
126     decoded_reg_strb.SHA512_VAULT_RD_CTRL =
cpuif_req_masked & (cpuif_addr == 12'h600);
127     decoded_reg_strb.SHA512_VAULT_RD_STATUS =
cpuif_req_masked & (cpuif_addr == 12'h604);
128     decoded_reg_strb.SHA512_KV_WR_CTRL = cpuif_req_masked
& (cpuif_addr == 12'h608);
129     decoded_reg_strb.SHA512_KV_WR_STATUS =
cpuif_req_masked & (cpuif_addr == 12'h60c);
130     for(int i0=0; i0<8; i0++) begin
131         decoded_reg_strb.SHA512_GEN_PCR_HASH_NONCE[i0] =
cpuif_req_masked & (cpuif_addr == 12'h610 + i0*12'h4);
132     end
133     decoded_reg_strb.SHA512_GEN_PCR_HASH_CTRL =
cpuif_req_masked & (cpuif_addr == 12'h630);
134     decoded_reg_strb.SHA512_GEN_PCR_HASH_STATUS =
cpuif_req_masked & (cpuif_addr == 12'h634);
135     for(int i0=0; i0<16; i0++) begin
136         decoded_reg_strb.SHA512_GEN_PCR_HASH_DIGEST[i0] =
cpuif_req_masked & (cpuif_addr == 12'h638 + i0*12'h4);
137     end
138     decoded_reg_strb.intr_block_rf.global_intr_en_r =
cpuif_req_masked & (cpuif_addr == 12'h800);
139     decoded_reg_strb.intr_block_rf.error_intr_en_r =
cpuif_req_masked & (cpuif_addr == 12'h804);
140     decoded_reg_strb.intr_block_rf.notif_intr_en_r =
cpuif_req_masked & (cpuif_addr == 12'h808);
141     decoded_reg_strb.intr_block_rf.error_global_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h80c);
142     decoded_reg_strb.intr_block_rf.notif_global_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h810);
143     decoded_reg_strb.intr_block_rf.error_internal_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h814);
144     decoded_reg_strb.intr_block_rf.notif_internal_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h818);
145     decoded_reg_strb.intr_block_rf.error_intr_trig_r =
cpuif_req_masked & (cpuif_addr == 12'h81c);
146     decoded_reg_strb.intr_block_rf.notif_intr_trig_r =
cpuif_req_masked & (cpuif_addr == 12'h820);
147     decoded_reg_strb.intr_block_rf.error0_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h900);
148     decoded_reg_strb.intr_block_rf.error1_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h904);
149     decoded_reg_strb.intr_block_rf.error2_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h908);
150     decoded_reg_strb.intr_block_rf.error3_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h90c);
151     de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h980);
152     de-
coded_reg_strb.intr_block_rf.error0_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha00);
153     de-
coded_reg_strb.intr_block_rf.error1_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha04);
154     de-
coded_reg_strb.intr_block_rf.error2_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha08);
155     de-
coded_reg_strb.intr_block_rf.error3_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha0c);
156     de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha10);
157     end
158
159 // Pass down signals to next stage
160 assign decoded_req = cpuif_req_masked;
161 assign decoded_req_is_wr = cpuif_req_is_wr;
162 assign decoded_wr_data = cpuif_wr_data;
163 assign decoded_wr_biten = cpuif_wr_biten;
164
165 //-----
166 // Field logic
167 //-----
168 typedef struct packed{
169     struct packed{
170         struct packed{
171             logic next;
172             logic load_next;
173         } INIT;
174         struct packed{
175             logic next;
176             logic load_next;
177         } NEXT;
178         struct packed{
179             logic [1:0] next;

```



```

180         logic load_next;
181     } MODE;
182     struct packed{
183         logic next;
184         logic load_next;
185     } ZEROIZE;
186     struct packed{
187         logic next;
188         logic load_next;
189     } LAST;
190     struct packed{
191         logic next;
192         logic load_next;
193     } RESTORE;
194 } SHA512_CTRL;
195 struct packed{
196     struct packed{
197         logic [31:0] next;
198         logic load_next;
199     } BLOCK;
200 } [32-1:0]SHA512_BLOCK;
201 struct packed{
202     struct packed{
203         logic [31:0] next;
204         logic load_next;
205     } DIGEST;
206 } [16-1:0]SHA512_DIGEST;
207 struct packed{
208     struct packed{
209         logic next;
210         logic load_next;
211     } read_en;
212     struct packed{
213         logic [4:0] next;
214         logic load_next;
215     } read_entry;
216     struct packed{
217         logic next;
218         logic load_next;
219     } pcr_hash_extend;
220     struct packed{
221         logic [24:0] next;
222         logic load_next;
223     } rsvd;
224 } SHA512_VAULT_RD_CTRL;
225 struct packed{
226     struct packed{
227         logic next;
228         logic load_next;
229     } VALID;
230 } SHA512_VAULT_RD_STATUS;
231 struct packed{
232     struct packed{
233         logic next;
234         logic load_next;
235     } write_en;
236     struct packed{
237         logic [4:0] next;
238         logic load_next;
239     } write_entry;
240     struct packed{
241         logic next;
242         logic load_next;
243     } hmac_key_dest_valid;
244     struct packed{
245         logic next;
246         logic load_next;
247     } hmac_block_dest_valid;
248     struct packed{
249         logic next;
250         logic load_next;
251     } mldsa_seed_dest_valid;
252     struct packed{
253         logic next;
254         logic load_next;
255     } ecc_pkey_dest_valid;
256     struct packed{
257         logic next;
258         logic load_next;
259     } ecc_seed_dest_valid;
260     struct packed{
261         logic next;
262         logic load_next;
263     } aes_key_dest_valid;
264     struct packed{
265         logic next;
266         logic load_next;
267     } mlkem_seed_dest_valid;
268     struct packed{
269         logic next;
270         logic load_next;
271     } mlkem_msg_dest_valid;
272     struct packed{
273         logic next;
274         logic load_next;
275     } dma_data_dest_valid;
276     struct packed{
277         logic [16:0] next;
278         logic load_next;
279     } rsvd;
280 } SHA512_KV_WR_CTRL;

```

```

281 struct packed{
282     struct packed{
283         logic next;
284         logic load_next;
285     } VALID;
286 } SHA512_KV_WR_STATUS;
287 struct packed{
288     struct packed{
289         logic [31:0] next;
290         logic load_next;
291     } NONCE;
292 } [8-1:0]SHA512_GEN_PCR_HASH_NONCE;
293 struct packed{
294     struct packed{
295         logic next;
296         logic load_next;
297     } START;
298 } SHA512_GEN_PCR_HASH_CTRL;
299 struct packed{
300     struct packed{
301         logic next;
302         logic load_next;
303     } VALID;
304 } SHA512_GEN_PCR_HASH_STATUS;
305 struct packed{
306     struct packed{
307         logic [31:0] next;
308         logic load_next;
309     } DIGEST;
310 } [16-1:0]SHA512_GEN_PCR_HASH_DIGEST;
311 struct packed{
312     struct packed{
313         struct packed{
314             logic next;
315             logic load_next;
316         } error_en;
317         struct packed{
318             logic next;
319             logic load_next;
320         } notif_en;
321     } global_intr_en_r;
322     struct packed{
323         struct packed{
324             logic next;
325             logic load_next;
326         } error0_en;
327         struct packed{
328             logic next;
329             logic load_next;
330         } error1_en;
331     } struct packed{
332         logic next;
333         logic load_next;
334     } error2_en;
335     struct packed{
336         logic next;
337         logic load_next;
338     } error3_en;
339 } error_intr_en_r;
340 struct packed{
341     struct packed{
342         logic next;
343         logic load_next;
344     } notif_cmd_done_en;
345     } notif_intr_en_r;
346     struct packed{
347         struct packed{
348             logic next;
349             logic load_next;
350         } agg_sts;
351     } error_global_intr_r;
352     struct packed{
353         struct packed{
354             logic next;
355             logic load_next;
356         } agg_sts;
357     } notif_global_intr_r;
358     struct packed{
359         struct packed{
360             logic next;
361             logic load_next;
362         } error0_sts;
363         struct packed{
364             logic next;
365             logic load_next;
366         } error1_sts;
367         struct packed{
368             logic next;
369             logic load_next;
370         } error2_sts;
371     } struct packed{
372         logic next;
373         logic load_next;
374     } error3_sts;
375 } error_internal_intr_r;
376     struct packed{
377         struct packed{
378             logic next;
379             logic load_next;
380         } notif_cmd_done_sts;
381     } notif_internal_intr_r;

```

```

382 struct packed{
383     struct packed{
384         logic next;
385         logic load_next;
386     } error0_trig;
387     struct packed{
388         logic next;
389         logic load_next;
390     } error1_trig;
391     struct packed{
392         logic next;
393         logic load_next;
394     } error2_trig;
395     struct packed{
396         logic next;
397         logic load_next;
398     } error3_trig;
399 } error_intr_trig_r;
400 struct packed{
401     struct packed{
402         logic next;
403         logic load_next;
404     } notif_cmd_done_trig;
405 } notif_intr_trig_r;
406 struct packed{
407     struct packed{
408         logic [31:0] next;
409         logic load_next;
410         logic incrthreshold;
411         logic incrsaturate;
412     } cnt;
413 } error0_intr_count_r;
414 struct packed{
415     struct packed{
416         logic [31:0] next;
417         logic load_next;
418         logic incrthreshold;
419         logic incrsaturate;
420     } cnt;
421 } error1_intr_count_r;
422 struct packed{
423     struct packed{
424         logic [31:0] next;
425         logic load_next;
426         logic incrthreshold;
427         logic incrsaturate;
428     } cnt;
429 } error2_intr_count_r;
430 struct packed{
431     struct packed{
432         logic [31:0] next;
433         logic load_next;
434         logic incrthreshold;
435         logic incrsaturate;
436     } cnt;
437 } error3_intr_count_r;
438 struct packed{
439     struct packed{
440         logic [31:0] next;
441         logic load_next;
442         logic incrthreshold;
443         logic incrsaturate;
444     } cnt;
445 } notif_cmd_done_intr_count_r;
446 struct packed{
447     struct packed{
448         logic next;
449         logic load_next;
450         logic decrthreshold;
451         logic underflow;
452     } pulse;
453 } error0_intr_count_incr_r;
454 struct packed{
455     struct packed{
456         logic next;
457         logic load_next;
458         logic decrthreshold;
459         logic underflow;
460     } pulse;
461 } error1_intr_count_incr_r;
462 struct packed{
463     struct packed{
464         logic next;
465         logic load_next;
466         logic decrthreshold;
467         logic underflow;
468     } pulse;
469 } error2_intr_count_incr_r;
470 struct packed{
471     struct packed{
472         logic next;
473         logic load_next;
474         logic decrthreshold;
475         logic underflow;
476     } pulse;
477 } error3_intr_count_incr_r;
478 struct packed{
479     struct packed{
480         logic next;
481         logic load_next;
482         logic decrthreshold;
483         logic underflow;
484     } pulse;
485     } notif_cmd_done_intr_count_incr_r;
486     } intr_block_rf;
487 } field_combo_t;
488 field_combo_t field_combo;
489
490 typedef struct packed{
491     struct packed{
492         struct packed{
493             logic value;
494         } INIT;
495         struct packed{
496             logic value;
497         } NEXT;
498         struct packed{
499             logic [1:0] value;
500         } MODE;
501         struct packed{
502             logic value;
503         } ZEROIZE;
504         struct packed{
505             logic value;
506         } LAST;
507         struct packed{
508             logic value;
509         } RESTORE;
510     } SHA512_CTRL;
511     struct packed{
512         struct packed{
513             logic [31:0] value;
514         } BLOCK;
515     } [32-1:0]SHA512_BLOCK;
516     struct packed{
517         struct packed{
518             logic [31:0] value;
519         } DIGEST;
520     } [16-1:0]SHA512_DIGEST;
521     struct packed{
522         struct packed{
523             logic value;
524         } read_en;
525         struct packed{
526             logic [4:0] value;
527         } read_entry;
528         struct packed{
529             logic value;
530         } pcr_hash_extend;
531         struct packed{
532             logic [24:0] value;
533         } rsvd;
534     } SHA512_VAULT_RD_CTRL;
535     struct packed{
536         struct packed{
537             logic value;
538         } VALID;
539     } SHA512_VAULT_RD_STATUS;
540     struct packed{
541         struct packed{
542             logic value;
543         } write_en;
544         struct packed{
545             logic [4:0] value;
546         } write_entry;
547         struct packed{
548             logic value;
549         } hmac_key_dest_valid;
550         struct packed{
551             logic value;
552         } hmac_block_dest_valid;
553         struct packed{
554             logic value;
555         } mldsa_seed_dest_valid;
556         struct packed{
557             logic value;
558         } ecc_pkey_dest_valid;
559         struct packed{
560             logic value;
561         } ecc_seed_dest_valid;
562         struct packed{
563             logic value;
564         } aes_key_dest_valid;
565         struct packed{
566             logic value;
567         } mlkem_seed_dest_valid;
568         struct packed{
569             logic value;
570         } mlkem_msg_dest_valid;
571         struct packed{
572             logic value;
573         } dma_data_dest_valid;
574         struct packed{
575             logic [16:0] value;
576         } rsvd;
577     } SHA512_KV_WR_CTRL;
578     struct packed{
579         struct packed{
580             logic value;
581         } VALID;
582     } SHA512_KV_WR_STATUS;
583     struct packed{

```

```

584     struct packed{
585         logic [31:0] value;
586     } NONCE;
587 } [8-1:0]SHA512_GEN_PCR_HASH_NONCE;
588 struct packed{
589     struct packed{
590         logic value;
591     } START;
592 } SHA512_GEN_PCR_HASH_CTRL;
593 struct packed{
594     struct packed{
595         logic value;
596     } VALID;
597 } SHA512_GEN_PCR_HASH_STATUS;
598 struct packed{
599     struct packed{
600         logic [31:0] value;
601     } DIGEST;
602 } [16-1:0]SHA512_GEN_PCR_HASH_DIGEST;
603 struct packed{
604     struct packed{
605         struct packed{
606             logic value;
607         } error_en;
608     } struct packed{
609         logic value;
610     } notif_en;
611 } global_intr_en_r;
612 struct packed{
613     struct packed{
614         logic value;
615     } error0_en;
616     struct packed{
617         logic value;
618     } error1_en;
619     struct packed{
620         logic value;
621     } error2_en;
622     struct packed{
623         logic value;
624     } error3_en;
625 } error_intr_en_r;
626 struct packed{
627     struct packed{
628         logic value;
629     } notif_cmd_done_en;
630 } notif_intr_en_r;
631 struct packed{
632     struct packed{
633         logic value;
634     } agg_sts;
635 } error_global_intr_r;
636 struct packed{
637     struct packed{
638         logic value;
639     } agg_sts;
640 } notif_global_intr_r;
641 struct packed{
642     struct packed{
643         logic value;
644     } error0_sts;
645     struct packed{
646         logic value;
647     } error1_sts;
648     struct packed{
649         logic value;
650     } error2_sts;
651     struct packed{
652         logic value;
653     } error3_sts;
654 } error_internal_intr_r;
655 struct packed{
656     struct packed{
657         logic value;
658     } notif_cmd_done_sts;
659 } notif_internal_intr_r;
660 struct packed{
661     struct packed{
662         logic value;
663     } error0_trig;
664     struct packed{
665         logic value;
666     } error1_trig;
667     struct packed{
668         logic value;
669     } error2_trig;
670     struct packed{
671         logic value;
672     } error3_trig;
673 } error_intr_trig_r;
674 struct packed{
675     struct packed{
676         logic value;
677     } notif_cmd_done_trig;
678 } notif_intr_trig_r;
679 struct packed{
680     struct packed{
681         logic [31:0] value;
682     } cnt;
683 } error0_intr_count_r;
684 struct packed{

```

```

685     struct packed{
686         logic [31:0] value;
687     } cnt;
688 } error1_intr_count_r;
689 struct packed{
690     struct packed{
691         logic [31:0] value;
692     } cnt;
693 } error2_intr_count_r;
694 struct packed{
695     struct packed{
696         logic [31:0] value;
697     } cnt;
698 } error3_intr_count_r;
699 struct packed{
700     struct packed{
701         logic [31:0] value;
702     } cnt;
703 } notif_cmd_done_intr_count_r;
704 struct packed{
705     struct packed{
706         logic value;
707     } pulse;
708 } error0_intr_count_incr_r;
709 struct packed{
710     struct packed{
711         logic value;
712     } pulse;
713 } error1_intr_count_incr_r;
714 struct packed{
715     struct packed{
716         logic value;
717     } pulse;
718 } error2_intr_count_incr_r;
719 struct packed{
720     struct packed{
721         logic value;
722     } pulse;
723 } error3_intr_count_incr_r;
724 struct packed{
725     struct packed{
726         logic value;
727     } pulse;
728 } notif_cmd_done_intr_count_incr_r;
729 } intr_block_rf;
730 } field_storage_t;
731 field_storage_t field_storage;
732
733 // Field: sha512_reg.SHA512_CTRL.INIT
734 always_comb begin
735     automatic logic [0:0] next_c;
736     automatic logic load_next_c;
737     next_c = field_storage.SHA512_CTRL.INIT.value;
738     load_next_c = '0;
739     if(decoded_reg_strb.SHA512_CTRL && decoded_req_is_wr
&& hwif_in.sha512_ready) begin // SW write
740         next_c = (field_storage.SHA512_CTRL.INIT.value &
~decoded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
741         load_next_c = '1;
742     end else begin // singlepulse clears back to 0
743         next_c = '0;
744         load_next_c = '1;
745     end
746     field_combo.SHA512_CTRL.INIT.next = next_c;
747     field_combo.SHA512_CTRL.INIT.load_next = load_next_c;
748 end
749 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
750     if(~hwif_in.reset_b) begin
751         field_storage.SHA512_CTRL.INIT.value <= 1'h0;
752     end else if(field_combo.SHA512_CTRL.INIT.load_next)
begin
753         field_storage.SHA512_CTRL.INIT.value <=
field_combo.SHA512_CTRL.INIT.next;
754     end
755 end
756 assign hwif_out.SHA512_CTRL.INIT.value =
field_storage.SHA512_CTRL.INIT.value;
757 // Field: sha512_reg.SHA512_CTRL.NEXT
758 always_comb begin
759     automatic logic [0:0] next_c;
760     automatic logic load_next_c;
761     next_c = field_storage.SHA512_CTRL.NEXT.value;
762     load_next_c = '0;
763     if(decoded_reg_strb.SHA512_CTRL && decoded_req_is_wr
&& hwif_in.sha512_ready) begin // SW write
764         next_c = (field_storage.SHA512_CTRL.NEXT.value &
~decoded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
coded_wr_biten[1:1]);
765         load_next_c = '1;
766     end else begin // singlepulse clears back to 0
767         next_c = '0;
768         load_next_c = '1;
769     end
770     field_combo.SHA512_CTRL.NEXT.next = next_c;
771     field_combo.SHA512_CTRL.NEXT.load_next = load_next_c;
772 end
773 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
774     if(~hwif_in.reset_b) begin
775         field_storage.SHA512_CTRL.NEXT.value <= 1'h0;
776     end else if(field_combo.SHA512_CTRL.NEXT.load_next)

```



```

begin
    777         field_storage.SHA512_CTRL.NEXT.value <=
field_combo.SHA512_CTRL.NEXT.next;
    778     end
    779 end
    780 assign hwif_out.SHA512_CTRL.NEXT.value =
field_storage.SHA512_CTRL.NEXT.value;
    781 // Field: sha512_reg.SHA512_CTRL.MODE
    782 always_comb begin
    783     automatic logic [1:0] next_c;
    784     automatic logic load_next_c;
    785     next_c = field_storage.SHA512_CTRL.MODE.value;
    786     load_next_c = '0;
    787     if(decoded_reg_strb.SHA512_CTRL && decoded_req_is_wr
&& hwif_in.sha512_ready) begin // SW write
    788         next_c = (field_storage.SHA512_CTRL.MODE.value &
~decoded_wr_biten[3:2]) | (decoded_wr_data[3:2] & de-
coded_wr_biten[3:2]);
    789         load_next_c = '1;
    790     end
    791     field_combo.SHA512_CTRL.MODE.next = next_c;
    792     field_combo.SHA512_CTRL.MODE.load_next = load_next_c;
    793 end
    794 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
    795     if(~hwif_in.reset_b) begin
    796         field_storage.SHA512_CTRL.MODE.value <= 2'h2;
    797     end else if(field_combo.SHA512_CTRL.MODE.load_next)
begin
    798         field_storage.SHA512_CTRL.MODE.value <=
field_combo.SHA512_CTRL.MODE.next;
    799     end
    800 end
    801 assign hwif_out.SHA512_CTRL.MODE.value =
field_storage.SHA512_CTRL.MODE.value;
    802 // Field: sha512_reg.SHA512_CTRL.ZEROIZE
    803 always_comb begin
    804     automatic logic [0:0] next_c;
    805     automatic logic load_next_c;
    806     next_c = field_storage.SHA512_CTRL.ZEROIZE.value;
    807     load_next_c = '0;
    808     if(decoded_reg_strb.SHA512_CTRL && decoded_req_is_wr)
begin // SW write
    809         next_c = (field_storage.SHA512_CTRL.ZEROIZE.value
& ~decoded_wr_biten[4:4]) | (decoded_wr_data[4:4] & de-
coded_wr_biten[4:4]);
    810         load_next_c = '1;
    811     end else begin // singlepulse clears back to 0
    812         next_c = '0;
    813         load_next_c = '1;
    814     end
    815     field_combo.SHA512_CTRL.ZEROIZE.next = next_c;
    816     field_combo.SHA512_CTRL.ZEROIZE.load_next =
load_next_c;
    817 end
    818 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
    819     if(~hwif_in.reset_b) begin
    820         field_storage.SHA512_CTRL.ZEROIZE.value <= 1'h0;
    821     end else if(field_combo.SHA512_CTRL.ZEROIZE.load_next)
begin
    822         field_storage.SHA512_CTRL.ZEROIZE.value <=
field_combo.SHA512_CTRL.ZEROIZE.next;
    823     end
    824 end
    825 assign hwif_out.SHA512_CTRL.ZEROIZE.value =
field_storage.SHA512_CTRL.ZEROIZE.value;
    826 // Field: sha512_reg.SHA512_CTRL.LAST
    827 always_comb begin
    828     automatic logic [0:0] next_c;
    829     automatic logic load_next_c;
    830     next_c = field_storage.SHA512_CTRL.LAST.value;
    831     load_next_c = '0;
    832     if(decoded_reg_strb.SHA512_CTRL && decoded_req_is_wr
&& hwif_in.sha512_ready) begin // SW write
    833         next_c = (field_storage.SHA512_CTRL.LAST.value &
~decoded_wr_biten[5:5]) | (decoded_wr_data[5:5] & de-
coded_wr_biten[5:5]);
    834         load_next_c = '1;
    835     end else if(hwif_in.SHA512_CTRL.LAST.hwclr) begin //
HW Clear
    836         next_c = '0;
    837         load_next_c = '1;
    838     end
    839     field_combo.SHA512_CTRL.LAST.next = next_c;
    840     field_combo.SHA512_CTRL.LAST.load_next = load_next_c;
    841 end
    842 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
    843     if(~hwif_in.reset_b) begin
    844         field_storage.SHA512_CTRL.LAST.value <= 1'h0;
    845     end else if(field_combo.SHA512_CTRL.LAST.load_next)
begin
    846         field_storage.SHA512_CTRL.LAST.value <=
field_combo.SHA512_CTRL.LAST.next;
    847     end
    848 end
    849 assign hwif_out.SHA512_CTRL.LAST.value =
field_storage.SHA512_CTRL.LAST.value;
    850 // Field: sha512_reg.SHA512_CTRL.RESTORE
    851 always_comb begin
    852     automatic logic [0:0] next_c;
    853     automatic logic load_next_c;
    854     next_c = field_storage.SHA512_CTRL.RESTORE.value;
    855     load_next_c = '0;
    856     if(decoded_reg_strb.SHA512_CTRL && decoded_req_is_wr
&& hwif_in.sha512_ready) begin // SW write
    857         next_c = (field_storage.SHA512_CTRL.RESTORE.value
& ~decoded_wr_biten[6:6]) | (decoded_wr_data[6:6] & de-
coded_wr_biten[6:6]);
    858         load_next_c = '1;
    859     end else begin // singlepulse clears back to 0
    860         next_c = '0;
    861         load_next_c = '1;
    862     end
    863     field_combo.SHA512_CTRL.RESTORE.next = next_c;
    864     field_combo.SHA512_CTRL.RESTORE.load_next =
load_next_c;
    865 end
    866 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
    867     if(~hwif_in.reset_b) begin
    868         field_storage.SHA512_CTRL.RESTORE.value <= 1'h0;
    869     end else if(field_combo.SHA512_CTRL.RESTORE.load_next)
begin
    870         field_storage.SHA512_CTRL.RESTORE.value <=
field_combo.SHA512_CTRL.RESTORE.next;
    871     end
    872 end
    873 assign hwif_out.SHA512_CTRL.RESTORE.value =
field_storage.SHA512_CTRL.RESTORE.value;
    874 for(genvar i0=0; i0<32; i0++) begin
    875     // Field: sha512_reg.SHA512_BLOCK[i].BLOCK
    876     always_comb begin
    877         automatic logic [31:0] next_c;
    878         automatic logic load_next_c;
    879         next_c =
field_storage.SHA512_BLOCK[i0].BLOCK.value;
    880         load_next_c = '0;
    881         if(decoded_reg_strb.SHA512_BLOCK[i0] && de-
coded_req_is_wr && !(hwif_in.SHA512_BLOCK[i0].BLOCK.swwel)) begin //
SW write
    882             next_c =
(field_storage.SHA512_BLOCK[i0].BLOCK.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
    883             load_next_c = '1;
    884         end else if(hwif_in.SHA512_BLOCK[i0].BLOCK.we) be-
gin // HW Write - we
    885             next_c = hwif_in.SHA512_BLOCK[i0].BLOCK.next;
    886             load_next_c = '1;
    887         end else if(hwif_in.SHA512_BLOCK[i0].BLOCK.hwclr)
begin // HW Clear
    888             next_c = '0;
    889             load_next_c = '1;
    890         end
    891         field_combo.SHA512_BLOCK[i0].BLOCK.next = next_c;
    892         field_combo.SHA512_BLOCK[i0].BLOCK.load_next =
load_next_c;
    893     end
    894     always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
    895         if(~hwif_in.reset_b) begin
    896             field_storage.SHA512_BLOCK[i0].BLOCK.value <=
32'h0;
    897         end else
    898         if(field_combo.SHA512_BLOCK[i0].BLOCK.load_next) begin
    899             field_storage.SHA512_BLOCK[i0].BLOCK.value <=
field_combo.SHA512_BLOCK[i0].BLOCK.next;
    900         end
    901     end
    902     assign hwif_out.SHA512_BLOCK[i0].BLOCK.value =
field_storage.SHA512_BLOCK[i0].BLOCK.value;
    903 end
    904 for(genvar i0=0; i0<16; i0++) begin
    905     // Field: sha512_reg.SHA512_DIGEST[i].DIGEST
    906     always_comb begin
    907         automatic logic [31:0] next_c;
    908         automatic logic load_next_c;
    909         next_c =
field_storage.SHA512_DIGEST[i0].DIGEST.value;
    910         load_next_c = '0;
    911         if(decoded_reg_strb.SHA512_DIGEST[i0] && de-
coded_req_is_wr) begin // SW write
    912             next_c =
(field_storage.SHA512_DIGEST[i0].DIGEST.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
    913             load_next_c = '1;
    914         end else if(hwif_in.SHA512_DIGEST[i0].DIGEST.we)
begin // HW Write - we
    915             next_c =
hwif_in.SHA512_DIGEST[i0].DIGEST.next;
    916             load_next_c = '1;
    917         end else
    918         if(hwif_in.SHA512_DIGEST[i0].DIGEST.hwclr) begin // HW Clear
    919             next_c = '0;
    920             load_next_c = '1;
    921         end
    922         field_combo.SHA512_DIGEST[i0].DIGEST.next =
next_c;
    923         field_combo.SHA512_DIGEST[i0].DIGEST.load_next =
load_next_c;
    924     end

```

```

923         always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
924             if(~hwif_in.reset_b) begin
925                 field_storage.SHA512_DIGEST[i0].DIGEST.value
<= 32'h0;
926             end else
if(field_combo.SHA512_DIGEST[i0].DIGEST.load_next) begin
927                 field_storage.SHA512_DIGEST[i0].DIGEST.value
<= field_combo.SHA512_DIGEST[i0].DIGEST.next;
928             end
929         end
930         assign hwif_out.SHA512_DIGEST[i0].DIGEST.value =
field_storage.SHA512_DIGEST[i0].DIGEST.value;
931     end
932     // Field: sha512_reg.SHA512_VAULT_RD_CTRL.read_en
933     always_comb begin
934         automatic logic [0:0] next_c;
935         automatic logic load_next_c;
936         next_c =
field_storage.SHA512_VAULT_RD_CTRL.read_en.value;
937         load_next_c = '0;
938         if(decoded_reg_strb.SHA512_VAULT_RD_CTRL && de-
coded_req_is_wr && hwif_in.SHA512_VAULT_RD_CTRL.read_en.swwe) be-
gin // SW write
939             next_c =
(field_storage.SHA512_VAULT_RD_CTRL.read_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
940             load_next_c = '1;
941         end else
if(hwif_in.SHA512_VAULT_RD_CTRL.read_en.hwclr) begin // HW Clear
942             next_c = '0;
943             load_next_c = '1;
944         end
945         field_combo.SHA512_VAULT_RD_CTRL.read_en.next =
next_c;
946         field_combo.SHA512_VAULT_RD_CTRL.read_en.load_next =
load_next_c;
947     end
948     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
949         if(~hwif_in.reset_b) begin
950             field_storage.SHA512_VAULT_RD_CTRL.read_en.value
<= 1'h0;
951         end else
if(field_combo.SHA512_VAULT_RD_CTRL.read_en.load_next) begin
952             field_storage.SHA512_VAULT_RD_CTRL.read_en.value
<= field_combo.SHA512_VAULT_RD_CTRL.read_en.next;
953         end
954     end
955     assign hwif_out.SHA512_VAULT_RD_CTRL.read_en.value =
field_storage.SHA512_VAULT_RD_CTRL.read_en.value;
956     // Field: sha512_reg.SHA512_VAULT_RD_CTRL.read_entry
957     always_comb begin
958         automatic logic [4:0] next_c;
959         automatic logic load_next_c;
960         next_c =
field_storage.SHA512_VAULT_RD_CTRL.read_entry.value;
961         load_next_c = '0;
962         if(decoded_reg_strb.SHA512_VAULT_RD_CTRL && de-
coded_req_is_wr && hwif_in.SHA512_VAULT_RD_CTRL.read_entry.swwe) be-
gin // SW write
963             next_c =
(field_storage.SHA512_VAULT_RD_CTRL.read_entry.value & ~de-
coded_wr_biten[5:1]) | (decoded_wr_data[5:1] & de-
coded_wr_biten[5:1]);
964             load_next_c = '1;
965         end
966         field_combo.SHA512_VAULT_RD_CTRL.read_entry.next =
next_c;
967         field_combo.SHA512_VAULT_RD_CTRL.read_entry.load_next
= load_next_c;
968     end
969     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
970         if(~hwif_in.reset_b) begin
971             field_storage.SHA512_VAULT_RD_CTRL.read_entry.value <= 5'h0;
972         end else
if(field_combo.SHA512_VAULT_RD_CTRL.read_entry.load_next) begin
973             field_storage.SHA512_VAULT_RD_CTRL.read_entry.value <=
field_combo.SHA512_VAULT_RD_CTRL.read_entry.next;
974         end
975     end
976     assign hwif_out.SHA512_VAULT_RD_CTRL.read_entry.value =
field_storage.SHA512_VAULT_RD_CTRL.read_entry.value;
977     // Field: sha512_reg.SHA512_VAULT_RD_CTRL.pcr_hash_extend
978     always_comb begin
979         automatic logic [0:0] next_c;
980         automatic logic load_next_c;
981         next_c =
field_storage.SHA512_VAULT_RD_CTRL.pcr_hash_extend.value;
982         load_next_c = '0;
983         if(decoded_reg_strb.SHA512_VAULT_RD_CTRL && de-
coded_req_is_wr && hwif_in.SHA512_VAULT_RD_CTRL.pcr_hash_extend.swwe) begin // SW write
984             next_c =
(field_storage.SHA512_VAULT_RD_CTRL.pcr_hash_extend.value & ~de-
coded_wr_biten[6:6]) | (decoded_wr_data[6:6] & de-
coded_wr_biten[6:6]);
985             load_next_c = '1;
986         end

```

```

987         field_combo.SHA512_VAULT_RD_CTRL.pcr_hash_extend.next
= next_c;
988     end
field_combo.SHA512_VAULT_RD_CTRL.pcr_hash_extend.load_next =
load_next_c;
989     end
990     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
991         if(~hwif_in.reset_b) begin
992             field_storage.SHA512_VAULT_RD_CTRL.pcr_hash_extend.value <= 1'h0;
993         end else
if(field_combo.SHA512_VAULT_RD_CTRL.pcr_hash_extend.load_next) begin
994             field_storage.SHA512_VAULT_RD_CTRL.pcr_hash_extend.value <=
field_combo.SHA512_VAULT_RD_CTRL.pcr_hash_extend.next;
995         end
996     end

997     assign hwif_out.SHA512_VAULT_RD_CTRL.pcr_hash_extend.value =
field_storage.SHA512_VAULT_RD_CTRL.pcr_hash_extend.value;
998     // Field: sha512_reg.SHA512_VAULT_RD_CTRL.rsvd
999     always_comb begin
1000         automatic logic [24:0] next_c;
1001         automatic logic load_next_c;
1002         next_c =
field_storage.SHA512_VAULT_RD_CTRL.rsvd.value;
1003         load_next_c = '0;
1004         if(decoded_reg_strb.SHA512_VAULT_RD_CTRL && de-
coded_req_is_wr && hwif_in.SHA512_VAULT_RD_CTRL.rsvd.swwe) begin //
SW write
1005             next_c =
(field_storage.SHA512_VAULT_RD_CTRL.rsvd.value & ~de-
coded_wr_biten[31:7]) | (decoded_wr_data[31:7] & de-
coded_wr_biten[31:7]);
1006             load_next_c = '1;
1007         end
1008         field_combo.SHA512_VAULT_RD_CTRL.rsvd.next = next_c;
1009         field_combo.SHA512_VAULT_RD_CTRL.rsvd.load_next =
load_next_c;
1010     end
1011     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1012         if(~hwif_in.reset_b) begin
1013             field_storage.SHA512_VAULT_RD_CTRL.rsvd.value <=
25'h0;
1014         end else
if(field_combo.SHA512_VAULT_RD_CTRL.rsvd.load_next) begin
1015             field_storage.SHA512_VAULT_RD_CTRL.rsvd.value <=
field_combo.SHA512_VAULT_RD_CTRL.rsvd.next;
1016         end
1017     end
1018     assign hwif_out.SHA512_VAULT_RD_CTRL.rsvd.value =
field_storage.SHA512_VAULT_RD_CTRL.rsvd.value;
1019     // Field: sha512_reg.SHA512_VAULT_RD_STATUS.VALID
1020     always_comb begin
1021         automatic logic [0:0] next_c;
1022         automatic logic load_next_c;
1023         next_c =
field_storage.SHA512_VAULT_RD_STATUS.VALID.value;
1024         load_next_c = '0;
1025         if(hwif_in.SHA512_VAULT_RD_STATUS.VALID.hwset) be-
gin // HW Set
1026             next_c = '1;
1027             load_next_c = '1;
1028         end else
if(hwif_in.SHA512_VAULT_RD_STATUS.VALID.hwclr) begin // HW Clear
1029             next_c = '0;
1030             load_next_c = '1;
1031         end
1032         field_combo.SHA512_VAULT_RD_STATUS.VALID.next =
next_c;
1033         field_combo.SHA512_VAULT_RD_STATUS.VALID.load_next =
load_next_c;
1034     end
1035     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1036         if(~hwif_in.reset_b) begin
1037             field_storage.SHA512_VAULT_RD_STATUS.VALID.value
<= 1'h0;
1038         end else
if(field_combo.SHA512_VAULT_RD_STATUS.VALID.load_next) begin
1039             field_storage.SHA512_VAULT_RD_STATUS.VALID.value
<= field_combo.SHA512_VAULT_RD_STATUS.VALID.next;
1040         end
1041     end
1042     // Field: sha512_reg.SHA512_KV_WR_CTRL.write_en
1043     always_comb begin
1044         automatic logic [0:0] next_c;
1045         automatic logic load_next_c;
1046         next_c =
field_storage.SHA512_KV_WR_CTRL.write_en.value;
1047         load_next_c = '0;
1048         if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.SHA512_KV_WR_CTRL.write_en.swwe) begin //
SW write
1049             next_c =
(field_storage.SHA512_KV_WR_CTRL.write_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1050             load_next_c = '1;
1051         end else if(hwif_in.SHA512_KV_WR_CTRL.write_en.hwclr)
begin // HW Clear
1052             next_c = '0;

```

```

1053         load_next_c = '1';
1054     end
1055     field_combo.SHA512_KV_WR_CTRL.write_en.next = next_c;
1056     field_combo.SHA512_KV_WR_CTRL.write_en.load_next =
load_next_c;
1057 end
1058 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1059     if(~hwif_in.reset_b) begin
1060         field_storage.SHA512_KV_WR_CTRL.write_en.value <=
1'h0;
1061     end else
1062     if(field_combo.SHA512_KV_WR_CTRL.write_en.load_next) begin
1063         field_storage.SHA512_KV_WR_CTRL.write_en.value <=
field_combo.SHA512_KV_WR_CTRL.write_en.next;
1064     end
1065     assign hwif_out.SHA512_KV_WR_CTRL.write_en.value =
field_storage.SHA512_KV_WR_CTRL.write_en.value;
1066     // Field: sha512_reg.SHA512_KV_WR_CTRL.write_entry
1067     always_comb begin
1068         automatic logic [4:0] next_c;
1069         automatic logic load_next_c;
1070         next_c =
field_storage.SHA512_KV_WR_CTRL.write_entry.value;
1071         load_next_c = '0;
1072         if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.SHA512_KV_WR_CTRL.write_entry.swwe) be-
gin // SW write
1073             next_c =
(field_storage.SHA512_KV_WR_CTRL.write_entry.value & ~de-
coded_wr_biten[5:1]) | (decoded_wr_data[5:1] & de-
coded_wr_biten[5:1]);
1074             load_next_c = '1;
1075         end
1076         field_combo.SHA512_KV_WR_CTRL.write_entry.next =
next_c;
1077         field_combo.SHA512_KV_WR_CTRL.write_entry.load_next =
load_next_c;
1078     end
1079     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1080         if(~hwif_in.reset_b) begin
1081             field_storage.SHA512_KV_WR_CTRL.write_entry.value
<= 5'h0;
1082         end else
1083         if(field_combo.SHA512_KV_WR_CTRL.write_entry.load_next) begin
1084             field_storage.SHA512_KV_WR_CTRL.write_entry.value
<= field_combo.SHA512_KV_WR_CTRL.write_entry.next;
1085         end
1086         assign hwif_out.SHA512_KV_WR_CTRL.write_entry.value =
field_storage.SHA512_KV_WR_CTRL.write_entry.value;
1087         // Field: sha512_reg.SHA512_KV_WR_CTRL.hmac_key_dest_valid
1088         always_comb begin
1089             automatic logic [0:0] next_c;
1090             automatic logic load_next_c;
1091             next_c =
field_storage.SHA512_KV_WR_CTRL.hmac_key_dest_valid.value;
1092             load_next_c = '0;
1093             if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.SHA512_KV_WR_CTRL.hmac_key_dest_valid.swwe) begin // SW
write
1094                 next_c =
(field_storage.SHA512_KV_WR_CTRL.hmac_key_dest_valid.value & ~de-
coded_wr_biten[6:6]) | (decoded_wr_data[6:6] & de-
coded_wr_biten[6:6]);
1095                 load_next_c = '1;
1096             end
1097             field_combo.SHA512_KV_WR_CTRL.hmac_key_dest_valid.next
= next_c;
1098         end
1099         field_combo.SHA512_KV_WR_CTRL.hmac_key_dest_valid.load_next =
load_next_c;
1100     end
1101     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1102         if(~hwif_in.reset_b) begin
1103             field_storage.SHA512_KV_WR_CTRL.hmac_key_dest_valid.value <= 1'h0;
1104         end else
1105         if(field_combo.SHA512_KV_WR_CTRL.hmac_key_dest_valid.load_next) be-
gin
1106             field_storage.SHA512_KV_WR_CTRL.hmac_key_dest_valid.value <=
field_combo.SHA512_KV_WR_CTRL.hmac_key_dest_valid.next;
1107         end
1108     end
1109     assign
hwif_out.SHA512_KV_WR_CTRL.hmac_key_dest_valid.value =
field_storage.SHA512_KV_WR_CTRL.hmac_key_dest_valid.value;
1110     // Field:
sha512_reg.SHA512_KV_WR_CTRL.hmac_block_dest_valid
1111     always_comb begin
1112         automatic logic [0:0] next_c;
1113         automatic logic load_next_c;
1114         next_c =
field_storage.SHA512_KV_WR_CTRL.hmac_block_dest_valid.value;
1115         load_next_c = '0;
1116         if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.SHA512_KV_WR_CTRL.hmac_block_dest_valid.swwe) begin // SW

```

```

write
1115         next_c =
(field_storage.SHA512_KV_WR_CTRL.hmac_block_dest_valid.value & ~de-
coded_wr_biten[7:7]) | (decoded_wr_data[7:7] & de-
coded_wr_biten[7:7]);
1116         load_next_c = '1;
1117     end
1118     field_combo.SHA512_KV_WR_CTRL.hmac_block_dest_valid.next = next_c;
1119     field_combo.SHA512_KV_WR_CTRL.hmac_block_dest_valid.load_next =
load_next_c;
1120     end
1121     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1122         if(~hwif_in.reset_b) begin
1123             field_storage.SHA512_KV_WR_CTRL.hmac_block_dest_valid.value <= 1'h0;
1124         end else
1125         if(field_combo.SHA512_KV_WR_CTRL.hmac_block_dest_valid.load_next)
begin
1126             field_storage.SHA512_KV_WR_CTRL.hmac_block_dest_valid.value <=
field_combo.SHA512_KV_WR_CTRL.hmac_block_dest_valid.next;
1127         end
1128     end
1129     assign
hwif_out.SHA512_KV_WR_CTRL.hmac_block_dest_valid.value =
field_storage.SHA512_KV_WR_CTRL.hmac_block_dest_valid.value;
1130     // Field:
sha512_reg.SHA512_KV_WR_CTRL.mlds_seed_dest_valid
1131     always_comb begin
1132         automatic logic [0:0] next_c;
1133         automatic logic load_next_c;
1134         next_c =
field_storage.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.value;
1135         load_next_c = '0;
1136         if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.swwe) begin // SW
write
1137             next_c =
(field_storage.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.value & ~de-
coded_wr_biten[8:8]) | (decoded_wr_data[8:8] & de-
coded_wr_biten[8:8]);
1138             load_next_c = '1;
1139         end
1140         field_combo.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.next = next_c;
1141         field_combo.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.load_next =
load_next_c;
1142     end
1143     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1144         if(~hwif_in.reset_b) begin
1145             field_storage.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.value <= 1'h0;
1146         end else
1147         if(field_combo.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.load_next)
begin
1148             field_storage.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.value <=
field_combo.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.next;
1149         end
1150     end
1151     assign
hwif_out.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.value =
field_storage.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.value;
1152     // Field: sha512_reg.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid
1153     always_comb begin
1154         automatic logic [0:0] next_c;
1155         automatic logic load_next_c;
1156         next_c =
field_storage.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.value;
1157         load_next_c = '0;
1158         if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.swwe) begin // SW
write
1159             next_c =
(field_storage.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.value & ~de-
coded_wr_biten[9:9]) | (decoded_wr_data[9:9] & de-
coded_wr_biten[9:9]);
1160             load_next_c = '1;
1161         end
1162         field_combo.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.next
= next_c;
1163         field_combo.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.load_next =
load_next_c;
1164     end
1165     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1166         if(~hwif_in.reset_b) begin
1167             field_storage.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.value <= 1'h0;
1168         end else
1169         if(field_combo.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.load_next) be-
gin
1170             field_storage.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.value <=
field_combo.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.next;

```

```

1168     end
1169   end
1170   assign
hwif_out.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.value =
field_storage.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.value;
1171   // Field: sha512_reg.SHA512_KV_WR_CTRL.ecc_seed_dest_valid
1172   always_comb begin
1173     automatic logic [0:0] next_c;
1174     automatic logic load_next_c;
1175     next_c =
field_storage.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.value;
1176     load_next_c = '0;
1177     if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.swve) begin // SW
write
1178       next_c =
(field_storage.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.value & ~de-
coded_wr_biten[10:10]) | (decoded_wr_data[10:10] & de-
coded_wr_biten[10:10]);
1179       load_next_c = '1;
1180     end

1181     field_combo.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.next
= next_c;
1182   field_combo.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.load_next =
load_next_c;
1183   end
1184   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1185     if(~hwif_in.reset_b) begin
1186       field_storage.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.value <= 1'h0;
1187     end else
if(field_combo.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.load_next) be-
gin
1188       field_storage.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.value <=
field_combo.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.next;
1189     end
1190   end
1191   assign
hwif_out.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.value =
field_storage.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.value;
1192   // Field: sha512_reg.SHA512_KV_WR_CTRL.aes_key_dest_valid
1193   always_comb begin
1194     automatic logic [0:0] next_c;
1195     automatic logic load_next_c;
1196     next_c =
field_storage.SHA512_KV_WR_CTRL.aes_key_dest_valid.value;
1197     load_next_c = '0;
1198     if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.SHA512_KV_WR_CTRL.aes_key_dest_valid.swve) begin // SW write
1199       next_c =
(field_storage.SHA512_KV_WR_CTRL.aes_key_dest_valid.value & ~de-
coded_wr_biten[11:11]) | (decoded_wr_data[11:11] & de-
coded_wr_biten[11:11]);
1200       load_next_c = '1;
1201     end
1202     field_combo.SHA512_KV_WR_CTRL.aes_key_dest_valid.next
= next_c;
1203   field_combo.SHA512_KV_WR_CTRL.aes_key_dest_valid.load_next =
load_next_c;
1204   end
1205   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1206     if(~hwif_in.reset_b) begin
1207       field_storage.SHA512_KV_WR_CTRL.aes_key_dest_valid.value <= 1'h0;
1208     end else
if(field_combo.SHA512_KV_WR_CTRL.aes_key_dest_valid.load_next) begin
1209       field_storage.SHA512_KV_WR_CTRL.aes_key_dest_valid.value <=
field_combo.SHA512_KV_WR_CTRL.aes_key_dest_valid.next;
1210     end
1211   end

1212   assign hwif_out.SHA512_KV_WR_CTRL.aes_key_dest_valid.value
= field_storage.SHA512_KV_WR_CTRL.aes_key_dest_valid.value;
1213   // Field:
sha512_reg.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid
1214   always_comb begin
1215     automatic logic [0:0] next_c;
1216     automatic logic load_next_c;
1217     next_c =
field_storage.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.value;
1218     load_next_c = '0;
1219     if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.swve) begin // SW
write
1220       next_c =
(field_storage.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.value & ~de-
coded_wr_biten[12:12]) | (decoded_wr_data[12:12] & de-
coded_wr_biten[12:12]);
1221       load_next_c = '1;
1222     end
1223   field_combo.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.next = next_c;
1224

```

```

field_combo.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.load_next =
load_next_c;
1225   end
1226   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1227     if(~hwif_in.reset_b) begin
1228       field_storage.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.value <= 1'h0;
1229     end else
if(field_combo.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.load_next)
begin
1230       field_storage.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.value <=
field_combo.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.next;
1231     end
1232   end
1233   assign
hwif_out.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.value =
field_storage.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.value;
1234   // Field:
sha512_reg.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid
1235   always_comb begin
1236     automatic logic [0:0] next_c;
1237     automatic logic load_next_c;
1238     next_c =
field_storage.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.value;
1239     load_next_c = '0;
1240     if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.swve) begin // SW
write
1241       next_c =
(field_storage.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.value & ~de-
coded_wr_biten[13:13]) | (decoded_wr_data[13:13] & de-
coded_wr_biten[13:13]);
1242       load_next_c = '1;
1243     end
1244   field_combo.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.next = next_c;
1245   field_combo.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.load_next =
load_next_c;
1246   end
1247   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1248     if(~hwif_in.reset_b) begin
1249       field_storage.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.value <= 1'h0;
1250     end else
if(field_combo.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.load_next) be-
gin
1251       field_storage.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.value <=
field_combo.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.next;
1252     end
1253   end
1254   assign
hwif_out.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.value =
field_storage.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.value;
1255   // Field: sha512_reg.SHA512_KV_WR_CTRL.dma_data_dest_valid
1256   always_comb begin
1257     automatic logic [0:0] next_c;
1258     automatic logic load_next_c;
1259     next_c =
field_storage.SHA512_KV_WR_CTRL.dma_data_dest_valid.value;
1260     load_next_c = '0;
1261     if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.SHA512_KV_WR_CTRL.dma_data_dest_valid.swve) begin // SW
write
1262       next_c =
(field_storage.SHA512_KV_WR_CTRL.dma_data_dest_valid.value & ~de-
coded_wr_biten[14:14]) | (decoded_wr_data[14:14] & de-
coded_wr_biten[14:14]);
1263       load_next_c = '1;
1264     end

1265     field_combo.SHA512_KV_WR_CTRL.dma_data_dest_valid.next
= next_c;
1266   field_combo.SHA512_KV_WR_CTRL.dma_data_dest_valid.load_next =
load_next_c;
1267   end
1268   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1269     if(~hwif_in.reset_b) begin
1270       field_storage.SHA512_KV_WR_CTRL.dma_data_dest_valid.value <= 1'h0;
1271     end else
if(field_combo.SHA512_KV_WR_CTRL.dma_data_dest_valid.load_next) be-
gin
1272       field_storage.SHA512_KV_WR_CTRL.dma_data_dest_valid.value <=
field_combo.SHA512_KV_WR_CTRL.dma_data_dest_valid.next;
1273     end
1274   end
1275   assign
hwif_out.SHA512_KV_WR_CTRL.dma_data_dest_valid.value =
field_storage.SHA512_KV_WR_CTRL.dma_data_dest_valid.value;
1276   // Field: sha512_reg.SHA512_KV_WR_CTRL.rsvd
1277   always_comb begin
1278     automatic logic [16:0] next_c;
1279     automatic logic load_next_c;
1280     next_c = field_storage.SHA512_KV_WR_CTRL.rsvd.value;

```



```

1281     load_next_c = '0;
1282     if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.SHA512_KV_WR_CTRL.rsvd.swve) begin // SW
write
1283         next_c =
(field_storage.SHA512_KV_WR_CTRL.rsvd.value & ~de-
coded_wr_biten[31:15]) | (decoded_wr_data[31:15] & de-
coded_wr_biten[31:15]);
1284         load_next_c = '1;
1285     end
1286     field_combo.SHA512_KV_WR_CTRL.rsvd.next = next_c;
1287     field_combo.SHA512_KV_WR_CTRL.rsvd.load_next =
load_next_c;
1288 end
1289 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1290     if(~hwif_in.reset_b) begin
1291         field_storage.SHA512_KV_WR_CTRL.rsvd.value <=
17'h0;
1292     end else
if(field_combo.SHA512_KV_WR_CTRL.rsvd.load_next) begin
1293         field_storage.SHA512_KV_WR_CTRL.rsvd.value <=
field_combo.SHA512_KV_WR_CTRL.rsvd.next;
1294     end
1295 end
1296 assign hwif_out.SHA512_KV_WR_CTRL.rsvd.value =
field_storage.SHA512_KV_WR_CTRL.rsvd.value;
1297 // Field: sha512_reg.SHA512_KV_WR_STATUS.VALID
1298 always_comb begin
1299     automatic logic [0:0] next_c;
1300     automatic logic load_next_c;
1301     next_c =
field_storage.SHA512_KV_WR_STATUS.VALID.value;
1302     load_next_c = '0;
1303     if(hwif_in.SHA512_KV_WR_STATUS.VALID.hwset) begin //
HW Set
1304         next_c = '1;
1305         load_next_c = '1;
1306     end else if(hwif_in.SHA512_KV_WR_STATUS.VALID.hwclr)
begin // HW Clear
1307         next_c = '0;
1308         load_next_c = '1;
1309     end
1310     field_combo.SHA512_KV_WR_STATUS.VALID.next = next_c;
1311     field_combo.SHA512_KV_WR_STATUS.VALID.load_next =
load_next_c;
1312 end
1313 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1314     if(~hwif_in.reset_b) begin
1315         field_storage.SHA512_KV_WR_STATUS.VALID.value <=
1'h0;
1316     end else
if(field_combo.SHA512_KV_WR_STATUS.VALID.load_next) begin
1317         field_storage.SHA512_KV_WR_STATUS.VALID.value <=
field_combo.SHA512_KV_WR_STATUS.VALID.next;
1318     end
1319 end
1320 for(genvar i0=0; i0<8; i0++) begin
1321     // Field: sha512_reg.SHA512_GEN_PCR_HASH_NONCE[].NONCE
1322     always_comb begin
1323         automatic logic [31:0] next_c;
1324         automatic logic load_next_c;
1325         next_c =
field_storage.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.value;
1326         load_next_c = '0;
1327         if(decoded_reg_strb.SHA512_GEN_PCR_HASH_NONCE[i0]
&& decoded_req_is_wr) begin // SW write
1328             next_c =
(field_storage.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1329             load_next_c = '1;
1330         end
1331         field_combo.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.next = next_c;
1332     end
1333     field_combo.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.load_next =
load_next_c;
1334     end
1335     always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
1336         if(~hwif_in.reset_b) begin
1337             field_storage.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.value <= 32'h0;
1338         end else
if(field_combo.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.load_next) begin
1339             field_storage.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.value <=
field_combo.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.next;
1340         end
1341     end
1342     assign
hwif_out.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.value =
field_storage.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.value;
1343 end
1344 // Field: sha512_reg.SHA512_GEN_PCR_HASH_CTRL.START
1345 always_comb begin
1346     automatic logic [0:0] next_c;
1347     automatic logic load_next_c;
1348     next_c =
field_storage.SHA512_GEN_PCR_HASH_CTRL.START.value;
1349     load_next_c = '0;

```

```

1349     if(decoded_reg_strb.SHA512_GEN_PCR_HASH_CTRL && de-
coded_req_is_wr && hwif_in.sha512_ready) begin // SW write
1350         next_c =
(field_storage.SHA512_GEN_PCR_HASH_CTRL.START.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1351         load_next_c = '1;
1352     end else begin // singlepulse clears back to 0
1353         next_c = '0;
1354         load_next_c = '1;
1355     end
1356     field_combo.SHA512_GEN_PCR_HASH_CTRL.START.next =
next_c;
1357     field_combo.SHA512_GEN_PCR_HASH_CTRL.START.load_next =
load_next_c;
1358 end
1359 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1360     if(~hwif_in.reset_b) begin
1361         field_storage.SHA512_GEN_PCR_HASH_CTRL.START.value
<= 1'h0;
1362     end else
if(field_combo.SHA512_GEN_PCR_HASH_CTRL.START.load_next) begin
1363         field_storage.SHA512_GEN_PCR_HASH_CTRL.START.value
<= field_combo.SHA512_GEN_PCR_HASH_CTRL.START.next;
1364     end
1365 end
1366 assign hwif_out.SHA512_GEN_PCR_HASH_CTRL.START.value =
field_storage.SHA512_GEN_PCR_HASH_CTRL.START.value;
1367 // Field: sha512_reg.SHA512_GEN_PCR_HASH_STATUS.VALID
1368 always_comb begin
1369     automatic logic [0:0] next_c;
1370     automatic logic load_next_c;
1371     next_c =
field_storage.SHA512_GEN_PCR_HASH_STATUS.VALID.value;
1372     load_next_c = '0;
1373     if(hwif_in.SHA512_GEN_PCR_HASH_STATUS.VALID.hwset) be-
gin // HW Set
1374         next_c = '1;
1375         load_next_c = '1;
1376     end else
if(hwif_in.SHA512_GEN_PCR_HASH_STATUS.VALID.hwclr) begin // HW Clear
1377         next_c = '0;
1378         load_next_c = '1;
1379     end
1380     field_combo.SHA512_GEN_PCR_HASH_STATUS.VALID.next =
next_c;
1381     field_combo.SHA512_GEN_PCR_HASH_STATUS.VALID.load_next
= load_next_c;
1382 end
1383 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1384     if(~hwif_in.reset_b) begin
1385         field_storage.SHA512_GEN_PCR_HASH_STATUS.VALID.value <= 1'h0;
1386     end else
if(field_combo.SHA512_GEN_PCR_HASH_STATUS.VALID.load_next) begin
1387         field_storage.SHA512_GEN_PCR_HASH_STATUS.VALID.value <=
field_combo.SHA512_GEN_PCR_HASH_STATUS.VALID.next;
1388     end
1389 end
1390 for(genvar i0=0; i0<16; i0++) begin
1391     // Field:
sha512_reg.SHA512_GEN_PCR_HASH_DIGEST[].DIGEST
1392     always_comb begin
1393         automatic logic [31:0] next_c;
1394         automatic logic load_next_c;
1395         next_c =
field_storage.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.value;
1396         load_next_c = '0;
1397     end
1398     if(hwif_in.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.hwclr) begin // HW
Clear
1399         next_c = '0;
1400         load_next_c = '1;
1401     end else begin // HW Write
1402         next_c =
hwif_in.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.next;
1403         load_next_c = '1;
1404     end
1405     field_combo.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.next = next_c;
1406     field_combo.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.load_next =
load_next_c;
1407     end
1408     always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
1409         if(~hwif_in.reset_b) begin
1410             field_storage.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.value <= 32'h0;
1411         end else
if(field_combo.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.load_next) be-
gin
1412             field_storage.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.value <=
field_combo.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.next;
1413         end

```

```

1413     end
1414   end
1415   // Field:
sha512_reg.intr_block_rf.global_intr_en_r.error_en
1416   always_comb begin
1417     automatic logic [0:0] next_c;
1418     automatic logic load_next_c;
1419     next_c =
field_storage.intr_block_rf.global_intr_en_r.error_en.value;
1420     load_next_c = '0;
1421     if(decoded_reg_strb.intr_block_rf.global_intr_en_r &&
decoded_req_is_wr) begin // SW write
1422       next_c =
(field_storage.intr_block_rf.global_intr_en_r.error_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1423       load_next_c = '1;
1424     end
1425   end
field_combo.intr_block_rf.global_intr_en_r.error_en.next = next_c;
1426   field_combo.intr_block_rf.global_intr_en_r.error_en.load_next =
load_next_c;
1427   end
1428   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1429     if(~hwif_in.reset_b) begin
1430       field_storage.intr_block_rf.global_intr_en_r.error_en.value <= '1h0;
1431     end else
1432       if(field_combo.intr_block_rf.global_intr_en_r.error_en.load_next)
begin
1433         field_storage.intr_block_rf.global_intr_en_r.error_en.value <=
field_combo.intr_block_rf.global_intr_en_r.error_en.next;
1434       end
1435     end
// Field:
sha512_reg.intr_block_rf.global_intr_en_r.notif_en
1436   always_comb begin
1437     automatic logic [0:0] next_c;
1438     automatic logic load_next_c;
1439     next_c =
field_storage.intr_block_rf.global_intr_en_r.notif_en.value;
1440     load_next_c = '0;
1441     if(decoded_reg_strb.intr_block_rf.global_intr_en_r &&
decoded_req_is_wr) begin // SW write
1442       next_c =
(field_storage.intr_block_rf.global_intr_en_r.notif_en.value & ~de-
coded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
coded_wr_biten[1:1]);
1443       load_next_c = '1;
1444     end
1445   end
field_combo.intr_block_rf.global_intr_en_r.notif_en.next = next_c;
1446   field_combo.intr_block_rf.global_intr_en_r.notif_en.load_next =
load_next_c;
1447   end
1448   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1449     if(~hwif_in.reset_b) begin
1450       field_storage.intr_block_rf.global_intr_en_r.notif_en.value <= '1h0;
1451     end else
1452       if(field_combo.intr_block_rf.global_intr_en_r.notif_en.load_next)
begin
1453         field_storage.intr_block_rf.global_intr_en_r.notif_en.value <=
field_combo.intr_block_rf.global_intr_en_r.notif_en.next;
1454       end
1455     end
// Field:
sha512_reg.intr_block_rf.error_intr_en_r.error0_en
1456   always_comb begin
1457     automatic logic [0:0] next_c;
1458     automatic logic load_next_c;
1459     next_c =
field_storage.intr_block_rf.error_intr_en_r.error0_en.value;
1460     load_next_c = '0;
1461     if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
1462       next_c =
(field_storage.intr_block_rf.error_intr_en_r.error0_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1463       load_next_c = '1;
1464     end
1465   end
field_combo.intr_block_rf.error_intr_en_r.error0_en.next = next_c;
1466   field_combo.intr_block_rf.error_intr_en_r.error0_en.load_next =
load_next_c;
1467   end
1468   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1469     if(~hwif_in.reset_b) begin
1470       field_storage.intr_block_rf.error_intr_en_r.error0_en.value <= '1h0;
1471     end else
1472       if(field_combo.intr_block_rf.error_intr_en_r.error0_en.load_next)
begin
1473         field_storage.intr_block_rf.error_intr_en_r.error0_en.value <=

```

```

field_combo.intr_block_rf.error_intr_en_r.error0_en.next;
1473   end
1474   end
// Field:
sha512_reg.intr_block_rf.error_intr_en_r.error1_en
1475   always_comb begin
1476     automatic logic [0:0] next_c;
1477     automatic logic load_next_c;
1478     next_c =
field_storage.intr_block_rf.error_intr_en_r.error1_en.value;
1479     load_next_c = '0;
1480     if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
1481       next_c =
(field_storage.intr_block_rf.error_intr_en_r.error1_en.value & ~de-
coded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
coded_wr_biten[1:1]);
1482       load_next_c = '1;
1483     end
1484   end
field_combo.intr_block_rf.error_intr_en_r.error1_en.next = next_c;
1485   field_combo.intr_block_rf.error_intr_en_r.error1_en.load_next =
load_next_c;
1486   end
1487   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1488     if(~hwif_in.reset_b) begin
1489       field_storage.intr_block_rf.error_intr_en_r.error1_en.value <= '1h0;
1490     end else
1491       if(field_combo.intr_block_rf.error_intr_en_r.error1_en.load_next)
begin
1492         field_storage.intr_block_rf.error_intr_en_r.error1_en.value <=
field_combo.intr_block_rf.error_intr_en_r.error1_en.next;
1493       end
1494     end
// Field:
sha512_reg.intr_block_rf.error_intr_en_r.error2_en
1495   always_comb begin
1496     automatic logic [0:0] next_c;
1497     automatic logic load_next_c;
1498     next_c =
field_storage.intr_block_rf.error_intr_en_r.error2_en.value;
1499     load_next_c = '0;
1500     if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
1501       next_c =
(field_storage.intr_block_rf.error_intr_en_r.error2_en.value & ~de-
coded_wr_biten[2:2]) | (decoded_wr_data[2:2] & de-
coded_wr_biten[2:2]);
1502       load_next_c = '1;
1503     end
1504   end
field_combo.intr_block_rf.error_intr_en_r.error2_en.next = next_c;
1505   field_combo.intr_block_rf.error_intr_en_r.error2_en.load_next =
load_next_c;
1506   end
1507   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1508     if(~hwif_in.reset_b) begin
1509       field_storage.intr_block_rf.error_intr_en_r.error2_en.value <= '1h0;
1510     end else
1511       if(field_combo.intr_block_rf.error_intr_en_r.error2_en.load_next)
begin
1512         field_storage.intr_block_rf.error_intr_en_r.error2_en.value <=
field_combo.intr_block_rf.error_intr_en_r.error2_en.next;
1513       end
1514     end
// Field:
sha512_reg.intr_block_rf.error_intr_en_r.error3_en
1515   always_comb begin
1516     automatic logic [0:0] next_c;
1517     automatic logic load_next_c;
1518     next_c =
field_storage.intr_block_rf.error_intr_en_r.error3_en.value;
1519     load_next_c = '0;
1520     if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
1521       next_c =
(field_storage.intr_block_rf.error_intr_en_r.error3_en.value & ~de-
coded_wr_biten[3:3]) | (decoded_wr_data[3:3] & de-
coded_wr_biten[3:3]);
1522       load_next_c = '1;
1523     end
1524   end
field_combo.intr_block_rf.error_intr_en_r.error3_en.next = next_c;
1525   field_combo.intr_block_rf.error_intr_en_r.error3_en.load_next =
load_next_c;
1526   end
1527   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1528     if(~hwif_in.reset_b) begin
1529       field_storage.intr_block_rf.error_intr_en_r.error3_en.value <= '1h0;
1530     end else
1531       if(field_combo.intr_block_rf.error_intr_en_r.error3_en.load_next)
begin
1532         field_storage.intr_block_rf.error_intr_en_r.error3_en.value <=

```

```

field_storage.intr_block_rf.error_intr_en_r.error3_en.value <=
field_combo.intr_block_rf.error_intr_en_r.error3_en.next;
1533     end
1534 end
1535 // Field:
sha512_reg.intr_block_rf.notif_intr_en_r.notif_cmd_done_en
1536 always_comb begin
1537     automatic logic [0:0] next_c;
1538     automatic logic load_next_c;
1539     next_c =
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value;
1540     load_next_c = '0;
1541     if(decoded_reg_strb.intr_block_rf.notif_intr_en_r &&
decoded_req_is_wr) begin // SW write
1542         next_c =
(field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
& ~decoded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1543         load_next_c = '1;
1544     end
1545 end
field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.next =
next_c;
1546
field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.load_ne
xt = load_next_c;
1547 end
1548 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1549     if(~hwif_in.reset_b) begin
1550
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
<= 1'h0;
1551     end else
if(field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.load_
next) begin
1552
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
<= field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.next;
1553     end
1554 end
1555 // Field:
sha512_reg.intr_block_rf.error_global_intr_r.agg_sts
1556 always_comb begin
1557     automatic logic [0:0] next_c;
1558     automatic logic load_next_c;
1559     next_c =
field_storage.intr_block_rf.error_global_intr_r.agg_sts.value;
1560     load_next_c = '0;
1561
1562     // HW Write
1563     next_c = hwif_out.intr_b-
lock_rf.error_internal_intr_r.intr;
1564     load_next_c = '1;
1565
field_combo.intr_block_rf.error_global_intr_r.agg_sts.next = next_c;
1566
field_combo.intr_block_rf.error_global_intr_r.agg_sts.load_next =
load_next_c;
1567 end
1568 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1569     if(~hwif_in.reset_b) begin
1570
field_storage.intr_block_rf.error_global_intr_r.agg_sts.value <=
1'h0;
1571     end else
if(field_combo.intr_block_rf.error_global_intr_r.agg_sts.load_next)
begin
1572
field_storage.intr_block_rf.error_global_intr_r.agg_sts.value <=
field_combo.intr_block_rf.error_global_intr_r.agg_sts.next;
1573     end
1574 end
1575 assign hwif_out.intr_block_rf.error_global_intr_r.intr =
1576 |
(field_storage.intr_block_rf.error_global_intr_r.agg_sts.value &
field_storage.intr_block_rf.global_intr_en_r.error_en.value);
1577 // Field:
sha512_reg.intr_block_rf.notif_global_intr_r.agg_sts
1578 always_comb begin
1579     automatic logic [0:0] next_c;
1580     automatic logic load_next_c;
1581     next_c =
field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value;
1582     load_next_c = '0;
1583
1584     // HW Write
1585     next_c = hwif_out.intr_b-
lock_rf.notif_internal_intr_r.intr;
1586     load_next_c = '1;
1587
field_combo.intr_block_rf.notif_global_intr_r.agg_sts.next = next_c;
1588
field_combo.intr_block_rf.notif_global_intr_r.agg_sts.load_next =
load_next_c;
1589 end
1590 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1591     if(~hwif_in.reset_b) begin
1592
field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value <=
1'h0;
1593     end else

```

```

if(field_combo.intr_block_rf.notif_global_intr_r.agg_sts.load_next)
begin
1594
field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value <=
field_combo.intr_block_rf.notif_global_intr_r.agg_sts.next;
1595     end
1596 end
1597 assign hwif_out.intr_block_rf.notif_global_intr_r.intr =
1598 |
(field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value &
field_storage.intr_block_rf.global_intr_en_r.notif_en.value);
1599 // Field:
sha512_reg.intr_block_rf.error_internal_intr_r.error0_sts
1600 always_comb begin
1601     automatic logic [0:0] next_c;
1602     automatic logic load_next_c;
1603     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value;
1604     load_next_c = '0;
1605
if(field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value !=
'0) begin // stickybit
1606     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1607     load_next_c = '1;
1608     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error0_sts.hwset) begin // HW Set
1609         next_c = '1;
1610         load_next_c = '1;
1611     end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1612         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value &
~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1613         load_next_c = '1;
1614     end
1615
field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next =
next_c;
1616
field_combo.intr_block_rf.error_internal_intr_r.error0_sts.load_next
= load_next_c;
1617 end
1618 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1619     if(~hwif_in.error_reset_b) begin
1620
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
<= 1'h0;
1621     end else
if(field_combo.intr_block_rf.error_internal_intr_r.error0_sts.load_
next) begin
1622
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next;
1623     end
1624 end
1625 // Field:
sha512_reg.intr_block_rf.error_internal_intr_r.error1_sts
1626 always_comb begin
1627     automatic logic [0:0] next_c;
1628     automatic logic load_next_c;
1629     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value;
1630     load_next_c = '0;
1631
if(field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value !=
'0) begin // stickybit
1632     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value;
1633     load_next_c = '1;
1634     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error1_sts.hwset) begin // HW Set
1635         next_c = '1;
1636         load_next_c = '1;
1637     end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1638         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value &
~(decoded_wr_data[1:1] & decoded_wr_biten[1:1]);
1639         load_next_c = '1;
1640     end
1641
field_combo.intr_block_rf.error_internal_intr_r.error1_sts.next =
next_c;
1642
field_combo.intr_block_rf.error_internal_intr_r.error1_sts.load_next
= load_next_c;
1643 end
1644 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1645     if(~hwif_in.error_reset_b) begin
1646
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
<= 1'h0;
1647     end else
if(field_combo.intr_block_rf.error_internal_intr_r.error1_sts.load_

```

```

next) begin
1648
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error1_sts.next;
1649    end
1650  end
1651  // Field:
sha512_reg.intr_block_rf.error_internal_intr_r.error2_sts
1652    always_comb begin
1653      automatic logic [0:0] next_c;
1654      automatic logic load_next_c;
1655      next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value;
1656      load_next_c = '0;
1657
if(field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value !=
'0) begin // stickybit
1658    next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1659    load_next_c = '1;
1660  end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset) begin // HW Set
1661    next_c = '1;
1662    load_next_c = '1;
1663  end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1664    next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value &
~(decoded_wr_data[2:2] & decoded_wr_biten[2:2]);
1665    load_next_c = '1;
1666  end
1667
field_combo.intr_block_rf.error_internal_intr_r.error2_sts.next =
next_c;
1668
field_combo.intr_block_rf.error_internal_intr_r.error2_sts.load_next
= load_next_c;
1669  end
1670  always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1671    if(~hwif_in.error_reset_b) begin
1672      field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
<= 1'h0;
1673    end else
if(field_combo.intr_block_rf.error_internal_intr_r.error2_sts.load_
next) begin
1674      field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error2_sts.next;
1675    end
1676  end
1677  // Field:
sha512_reg.intr_block_rf.error_internal_intr_r.error3_sts
1678    always_comb begin
1679      automatic logic [0:0] next_c;
1680      automatic logic load_next_c;
1681      next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value;
1682      load_next_c = '0;
1683
if(field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value !=
'0) begin // stickybit
1684    next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1685    load_next_c = '1;
1686  end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error3_sts.hwset) begin // HW Set
1687    next_c = '1;
1688    load_next_c = '1;
1689  end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1690    next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value &
~(decoded_wr_data[3:3] & decoded_wr_biten[3:3]);
1691    load_next_c = '1;
1692  end
1693
field_combo.intr_block_rf.error_internal_intr_r.error3_sts.next =
next_c;
1694
field_combo.intr_block_rf.error_internal_intr_r.error3_sts.load_next
= load_next_c;
1695  end
1696  always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1697    if(~hwif_in.error_reset_b) begin
1698      field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
<= 1'h0;
1699    end else
if(field_combo.intr_block_rf.error_internal_intr_r.error3_sts.load_
next) begin
1700      field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error3_sts.next;
1701    end
1702  end
1703    assign hwif_out.intr_block_rf.error_internal_intr_r.intr =
1704    |
(field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error0_en.value)
1705    ||
(field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error1_en.value)
1706    ||
(field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error2_en.value)
1707    ||
(field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error3_en.value);
1708  // Field:
sha512_reg.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
1709    always_comb begin
1710      automatic logic [0:0] next_c;
1711      automatic logic load_next_c;
1712      next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value;
1713      load_next_c = '0;
1714
if(field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
.value != '0) begin // stickybit
1715    next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value |
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
1716    load_next_c = '1;
1717  end else if(hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset) begin // HW
Set
1718    next_c = '1;
1719    load_next_c = '1;
1720  end else if(de-
coded_reg_strb.intr_block_rf.notif_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1721    next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value & ~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1722    load_next_c = '1;
1723  end
1724
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.
next = next_c;
1725
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.
load_next = load_next_c;
1726  end
1727  always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1728    if(~hwif_in.reset_b) begin
1729      field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value <= 1'h0;
1730    end else
if(field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_s
ts.load_next) begin
1731      field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value <=
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.
next;
1732    end
1733  end
1734    assign hwif_out.intr_block_rf.notif_internal_intr_r.intr =
1735    |
(field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_s
ts.value &
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value)
;
1736  // Field:
sha512_reg.intr_block_rf.error_intr_trig_r.error0_trig
1737    always_comb begin
1738      automatic logic [0:0] next_c;
1739      automatic logic load_next_c;
1740      next_c =
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1741      load_next_c = '0;
1742
if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1743    next_c =
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value |
(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1744    load_next_c = '1;
1745  end else begin // singlepulse clears back to 0
1746    next_c = '0;
1747    load_next_c = '1;
1748  end
1749
field_combo.intr_block_rf.error_intr_trig_r.error0_trig.next =
next_c;
1750
field_combo.intr_block_rf.error_intr_trig_r.error0_trig.load_next =
load_next_c;
1751  end
1752  always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1753    if(~hwif_in.reset_b) begin

```



```

1754
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value <=
1'h0;
1755     end else
if(field_combo.intr_block_rf.error_intr_trig_r.error0_trig.load_next
) begin
1756
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error0_trig.next;
1757     end
1758     end
1759     // Field:
sha512_reg.intr_block_rf.error_intr_trig_r.error1_trig
1760     always_comb begin
1761         automatic logic [0:0] next_c;
1762         automatic logic load_next_c;
1763         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value;
1764         load_next_c = '0;

1765         if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1766             next_c =
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value |
(decoded_wr_data[1:1] & decoded_wr_biten[1:1]);
1767             load_next_c = '1;
1768         end else begin // singlepulse clears back to 0
1769             next_c = '0;
1770             load_next_c = '1;
1771         end
1772
field_combo.intr_block_rf.error_intr_trig_r.error1_trig.next =
next_c;
1773
field_combo.intr_block_rf.error_intr_trig_r.error1_trig.load_next =
load_next_c;
1774     end
1775     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1776         if(~hwif_in.reset_b) begin
1777
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value <=
1'h0;
1778         end else
if(field_combo.intr_block_rf.error_intr_trig_r.error1_trig.load_next
) begin
1779
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error1_trig.next;
1780         end
1781     end
1782     // Field:
sha512_reg.intr_block_rf.error_intr_trig_r.error2_trig
1783     always_comb begin
1784         automatic logic [0:0] next_c;
1785         automatic logic load_next_c;
1786         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1787         load_next_c = '0;

1788         if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1789             next_c =
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value |
(decoded_wr_data[2:2] & decoded_wr_biten[2:2]);
1790             load_next_c = '1;
1791         end else begin // singlepulse clears back to 0
1792             next_c = '0;
1793             load_next_c = '1;
1794         end
1795
field_combo.intr_block_rf.error_intr_trig_r.error2_trig.next =
next_c;
1796
field_combo.intr_block_rf.error_intr_trig_r.error2_trig.load_next =
load_next_c;
1797     end
1798     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1799         if(~hwif_in.reset_b) begin
1800
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value <=
1'h0;
1801         end else
if(field_combo.intr_block_rf.error_intr_trig_r.error2_trig.load_next
) begin
1802
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error2_trig.next;
1803         end
1804     end
1805     // Field:
sha512_reg.intr_block_rf.error_intr_trig_r.error3_trig
1806     always_comb begin
1807         automatic logic [0:0] next_c;
1808         automatic logic load_next_c;
1809         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1810         load_next_c = '0;

1811         if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1812             next_c =

```

```

field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value |
(decoded_wr_data[3:3] & decoded_wr_biten[3:3]);
1813         load_next_c = '1;
1814     end else begin // singlepulse clears back to 0
1815         next_c = '0;
1816         load_next_c = '1;
1817     end
1818
field_combo.intr_block_rf.error_intr_trig_r.error3_trig.next =
next_c;
1819
field_combo.intr_block_rf.error_intr_trig_r.error3_trig.load_next =
load_next_c;
1820     end
1821     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1822         if(~hwif_in.reset_b) begin
1823
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value <=
1'h0;
1824         end else
if(field_combo.intr_block_rf.error_intr_trig_r.error3_trig.load_next
) begin
1825
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error3_trig.next;
1826         end
1827     end
1828     // Field:
sha512_reg.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
1829     always_comb begin
1830         automatic logic [0:0] next_c;
1831         automatic logic load_next_c;
1832         next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
1833         load_next_c = '0;

1834         if(decoded_reg_strb.intr_block_rf.notif_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1835             next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue | (decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1836             load_next_c = '1;
1837         end else begin // singlepulse clears back to 0
1838             next_c = '0;
1839             load_next_c = '1;
1840         end
1841
field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.next
= next_c;
1842
field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.loa
d_next = load_next_c;
1843     end
1844     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1845         if(~hwif_in.reset_b) begin
1846
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue <= 1'h0;
1847         end else
if(field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.
load_next) begin
1848
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue <=
field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.next
;
1849     end
1850     end
1851     // Field: sha512_reg.intr_block_rf.error0_intr_count_r.cnt
1852     always_comb begin
1853         automatic logic [31:0] next_c;
1854         automatic logic load_next_c;
1855         next_c =
field_storage.intr_block_rf.error0_intr_count_r.cnt.value;
1856         load_next_c = '0;
1857         if(decoded_reg_strb.intr_block_rf.error0_intr_count_r
&& decoded_req_is_wr) begin // SW write
1858             next_c =
(field_storage.intr_block_rf.error0_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1859             load_next_c = '1;
1860         end
1861
if(field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value)
begin // increment
1862         if(((33)'(next_c) + 32'h1) > 32'hfffffff) be-
gin // up-counter saturated
1863             next_c = 32'hfffffff;
1864         end else begin
1865             next_c = next_c + 32'h1;
1866         end
1867         load_next_c = '1;
1868     end
1869
field_combo.intr_block_rf.error0_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error0_intr_count_r.cnt.value >=
32'hfffffff);
1870
field_combo.intr_block_rf.error0_intr_count_r.cnt.incrsaturate =

```

```

(field_storage.intr_block_rf.error0_intr_count_r.cnt.value >=
32'hffffffff);
1871     if(next_c > 32'hffffffff) begin
1872         next_c = 32'hffffffff;
1873         load_next_c = '1;
1874     end

1875     field_combo.intr_block_rf.error0_intr_count_r.cnt.next
= next_c;
1876
field_combo.intr_block_rf.error0_intr_count_r.cnt.load_next =
load_next_c;
1877     end
1878     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1879         if(~hwif_in.error_reset_b) begin
1880
field_storage.intr_block_rf.error0_intr_count_r.cnt.value <= 32'h0;
1881         end else
if(field_combo.intr_block_rf.error0_intr_count_r.cnt.load_next) be-
gin
1882
field_storage.intr_block_rf.error0_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error0_intr_count_r.cnt.next;
1883         end
1884     end
1885     // Field: sha512_reg.intr_block_rf.error1_intr_count_r.cnt
1886     always_comb begin
1887         automatic logic [31:0] next_c;
1888         automatic logic load_next_c;
1889         next_c =
field_storage.intr_block_rf.error1_intr_count_r.cnt.value;
1890         load_next_c = '0;
1891         if(decoded_reg_strb.intr_block_rf.error1_intr_count_r
&& decoded_req_is_wr) begin // SW write
1892             next_c =
(field_storage.intr_block_rf.error1_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1893             load_next_c = '1;
1894         end
1895
if(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value)
begin // increment
1896         if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
gin // up-counter saturated
1897             next_c = 32'hffffffff;
1898         end else begin
1899             next_c = next_c + 32'h1;
1900         end
1901         load_next_c = '1;
1902     end
1903
field_combo.intr_block_rf.error1_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=
32'hffffffff);
1904
field_combo.intr_block_rf.error1_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=
32'hffffffff);
1905     if(next_c > 32'hffffffff) begin
1906         next_c = 32'hffffffff;
1907         load_next_c = '1;
1908     end

1909     field_combo.intr_block_rf.error1_intr_count_r.cnt.next
= next_c;
1910
field_combo.intr_block_rf.error1_intr_count_r.cnt.load_next =
load_next_c;
1911     end
1912     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1913         if(~hwif_in.error_reset_b) begin
1914
field_storage.intr_block_rf.error1_intr_count_r.cnt.value <= 32'h0;
1915         end else
if(field_combo.intr_block_rf.error1_intr_count_r.cnt.load_next) be-
gin
1916
field_storage.intr_block_rf.error1_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error1_intr_count_r.cnt.next;
1917         end
1918     end
1919     // Field: sha512_reg.intr_block_rf.error2_intr_count_r.cnt
1920     always_comb begin
1921         automatic logic [31:0] next_c;
1922         automatic logic load_next_c;
1923         next_c =
field_storage.intr_block_rf.error2_intr_count_r.cnt.value;
1924         load_next_c = '0;
1925         if(decoded_reg_strb.intr_block_rf.error2_intr_count_r
&& decoded_req_is_wr) begin // SW write
1926             next_c =
(field_storage.intr_block_rf.error2_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1927             load_next_c = '1;
1928         end
1929
if(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value)

```

```

begin // increment
1930     if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
gin // up-counter saturated
1931         next_c = 32'hffffffff;
1932     end else begin
1933         next_c = next_c + 32'h1;
1934     end
1935     load_next_c = '1;
1936 end
1937
field_combo.intr_block_rf.error2_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error2_intr_count_r.cnt.value >=
32'hffffffff);
1938
field_combo.intr_block_rf.error2_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error2_intr_count_r.cnt.value >=
32'hffffffff);
1939     if(next_c > 32'hffffffff) begin
1940         next_c = 32'hffffffff;
1941         load_next_c = '1;
1942     end

1943     field_combo.intr_block_rf.error2_intr_count_r.cnt.next
= next_c;
1944
field_combo.intr_block_rf.error2_intr_count_r.cnt.load_next =
load_next_c;
1945     end
1946     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1947         if(~hwif_in.error_reset_b) begin
1948
field_storage.intr_block_rf.error2_intr_count_r.cnt.value <= 32'h0;
1949         end else
if(field_combo.intr_block_rf.error2_intr_count_r.cnt.load_next) be-
gin
1950
field_storage.intr_block_rf.error2_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error2_intr_count_r.cnt.next;
1951         end
1952     end
1953     // Field: sha512_reg.intr_block_rf.error3_intr_count_r.cnt
1954     always_comb begin
1955         automatic logic [31:0] next_c;
1956         automatic logic load_next_c;
1957         next_c =
field_storage.intr_block_rf.error3_intr_count_r.cnt.value;
1958         load_next_c = '0;
1959         if(decoded_reg_strb.intr_block_rf.error3_intr_count_r
&& decoded_req_is_wr) begin // SW write
1960             next_c =
(field_storage.intr_block_rf.error3_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1961             load_next_c = '1;
1962         end
1963
if(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value)
begin // increment
1964         if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
gin // up-counter saturated
1965             next_c = 32'hffffffff;
1966         end else begin
1967             next_c = next_c + 32'h1;
1968         end
1969         load_next_c = '1;
1970     end
1971
field_combo.intr_block_rf.error3_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error3_intr_count_r.cnt.value >=
32'hffffffff);
1972
field_combo.intr_block_rf.error3_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error3_intr_count_r.cnt.value >=
32'hffffffff);
1973     if(next_c > 32'hffffffff) begin
1974         next_c = 32'hffffffff;
1975         load_next_c = '1;
1976     end

1977     field_combo.intr_block_rf.error3_intr_count_r.cnt.next
= next_c;
1978
field_combo.intr_block_rf.error3_intr_count_r.cnt.load_next =
load_next_c;
1979     end
1980     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1981         if(~hwif_in.error_reset_b) begin
1982
field_storage.intr_block_rf.error3_intr_count_r.cnt.value <= 32'h0;
1983         end else
if(field_combo.intr_block_rf.error3_intr_count_r.cnt.load_next) be-
gin
1984
field_storage.intr_block_rf.error3_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error3_intr_count_r.cnt.next;
1985         end
1986     end
1987     // Field:
sha512_reg.intr_block_rf.notif_cmd_done_intr_count_r.cnt

```

```

1988     always_comb begin
1989         automatic logic [31:0] next_c;
1990         automatic logic load_next_c;
1991         next_c =
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value;
1992         load_next_c = '0;
1993         if(de-
coded_req_strb.intr_block_rf.notif_cmd_done_intr_count_r && de-
coded_req_is_wr) begin // SW write
1994             next_c =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value &
-decoded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1995             load_next_c = '1;
1996         end
1997     if(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pul
se.value) begin // increment
1998         if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
gin // up-counter saturated
1999             next_c = 32'hffffffff;
2000         end else begin
2001             next_c = next_c + 32'h1;
2002         end
2003         load_next_c = '1;
2004     end
2005     field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.incrthres
hold =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value
>= 32'hffffffff);
2006     field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.incrsatur
ate =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value
>= 32'hffffffff);
2007     if(next_c > 32'hffffffff) begin
2008         next_c = 32'hffffffff;
2009         load_next_c = '1;
2010     end
2011     field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.next =
next_c;
2012     field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.load_next
= load_next_c;
2013     end
2014     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2015         if(~hwif_in.reset_b) begin
2016             field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value <=
32'h0;
2017         end else
if(field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.load_n
ext) begin
2018             field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value <=
field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.next;
2019         end
2020     end
2021     // Field:
sha512_reg.intr_block_rf.error0_intr_count_incr_r.pulse
2022     always_comb begin
2023         automatic logic [0:0] next_c;
2024         automatic logic load_next_c;
2025         next_c =
field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value;
2026         load_next_c = '0;
2027     if(field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value)
begin // HW Write - we
2028         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
2029         load_next_c = '1;
2030     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error0_sts.hwset) begin // HW Set
2031         next_c = '1;
2032         load_next_c = '1;
2033     end
2034     if(field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value)
begin // decrement
2035         field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
2036         next_c = next_c - 1'h1;
2037         load_next_c = '1;
2038     end else begin
2039         field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.underflow =
'0;
2040     end
2041     field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value <=
1'd0);
2042     field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.next =
next_c;
2043

```

```

field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.load_next =
load_next_c;
2044     end
2045     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2046         if(~hwif_in.reset_b) begin
2047             field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value <=
1'h0;
2048         end else
if(field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.load_ne
xt) begin
2049             field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.next;
2050         end
2051     end
2052     // Field:
sha512_reg.intr_block_rf.error1_intr_count_incr_r.pulse
2053     always_comb begin
2054         automatic logic [0:0] next_c;
2055         automatic logic load_next_c;
2056         next_c =
field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value;
2057         load_next_c = '0;
2058     if(field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value)
begin // HW Write - we
2059         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value;
2060         load_next_c = '1;
2061     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error1_sts.hwset) begin // HW Set
2062         next_c = '1;
2063         load_next_c = '1;
2064     end
2065     if(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value)
begin // decrement
2066         field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
2067         next_c = next_c - 1'h1;
2068         load_next_c = '1;
2069     end else begin
2070         field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.underflow =
'0;
2071     end
2072     field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
1'd0);
2073     field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.next =
next_c;
2074     field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.load_next =
load_next_c;
2075     end
2076     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2077         if(~hwif_in.reset_b) begin
2078             field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
1'h0;
2079         end else
if(field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.load_ne
xt) begin
2080             field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.next;
2081         end
2082     end
2083     // Field:
sha512_reg.intr_block_rf.error2_intr_count_incr_r.pulse
2084     always_comb begin
2085         automatic logic [0:0] next_c;
2086         automatic logic load_next_c;
2087         next_c =
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value;
2088         load_next_c = '0;
2089     if(field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value)
begin // HW Write - we
2090         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
2091         load_next_c = '1;
2092     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset) begin // HW Set
2093         next_c = '1;
2094         load_next_c = '1;
2095     end
2096     if(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value)
begin // decrement
2097         field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
2098         next_c = next_c - 1'h1;
2099         load_next_c = '1;
2100     end else begin

```

```

2101 field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.underflow =
2102 '0;
2103     end
2104 field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.decrthresh
2105 old =
2106 (field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
2107 1'd0);
2108 field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next =
2109 next_c;
2110 field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_next =
2111 load_next_c;
2112     end
2113     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2114         if(~hwif_in.reset_b) begin
2115             field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
2116             1'h0;
2117         end else
2118             if(field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_ne
2119 xt) begin
2120                 field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
2121                 field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next;
2122             end
2123         end
2124 // Field:
2125 sha512_reg.intr_block_rf.error3_intr_count_incr_r.pulse
2126 always_comb begin
2127     automatic logic [0:0] next_c;
2128     automatic logic load_next_c;
2129     next_c =
2130 field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value;
2131     load_next_c = '0;
2132 end
2133 if(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value)
2134 begin // HW Write - we
2135     next_c =
2136 field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value;
2137     load_next_c = '1;
2138 end else if(hwif_in.intr_b-
2139 lock_rf.error_internal_intr_r.error3_sts.hwset) begin // HW Set
2140     next_c = '1;
2141     load_next_c = '1;
2142 end
2143 end
2144 if(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value)
2145 begin // decrement
2146     field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.underflow =
2147     (next_c < (1'h1));
2148     next_c = next_c - 1'h1;
2149     load_next_c = '1;
2150 end else begin
2151     field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.underflow =
2152     '0;
2153     end
2154 field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.decrthresh
2155 old =
2156 (field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
2157 1'd0);
2158 field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.next =
2159 next_c;
2160 field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.load_next =
2161 load_next_c;
2162     end
2163     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2164         if(~hwif_in.reset_b) begin
2165             field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
2166             1'h0;
2167         end else
2168             if(field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.load_ne
2169 xt) begin
2170                 field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
2171                 field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.next;
2172             end
2173         end
2174 // Field:
2175 sha512_reg.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse
2176 always_comb begin
2177     automatic logic [0:0] next_c;
2178     automatic logic load_next_c;
2179     next_c =
2180 field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.value;
2181     load_next_c = '0;
2182 end
2183 if(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.value)
2184 begin // HW Write - we
2185     next_c =
2186 field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.value;
2187     load_next_c = '1;
2188 end else if(hwif_in.intr_b-
2189 lock_rf.error_internal_intr_r.error3_sts.hwset) begin // HW Set
2190     next_c = '1;
2191     load_next_c = '1;
2192 end
2193 end
2194 if(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.value)
2195 begin // decrement
2196     field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.underflow =
2197     (next_c < (1'h1));
2198     next_c = next_c - 1'h1;
2199     load_next_c = '1;
2200 end else begin
2201     field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.underflow =
2202     '0;
2203     end
2204 field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.decrthresh
2205 old =
2206 (field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.value <=
2207 1'd0);
2208 field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.next =
2209 next_c;
2210 field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.load_next =
2211 load_next_c;
2212     end
2213     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2214         if(~hwif_in.reset_b) begin
2215             field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.value <=
2216             1'h0;
2217         end else
2218             if(field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.load_ne
2219 xt) begin
2220                 field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.value <=
2221                 field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.next;
2222             end
2223         end
2224 // Field:
2225 sha512_reg.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
2226 value) begin // HW Write - we
2227     next_c =
2228 field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
2229 alue;

```

```

2153     load_next_c = '1;
2154     end else if(hwif_in.intr_b-
2155 lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset) begin // HW
2156 Set
2157     next_c = '1;
2158     load_next_c = '1;
2159     end
2160 if(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pul
2161 se.value) begin // decrement
2162     field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.un
2163 derflow = (next_c < (1'h1));
2164     next_c = next_c - 1'h1;
2165     load_next_c = '1;
2166     end else begin
2167         field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.un
2168 derflow = '0;
2169     end
2170 field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.de
2171 crthreshold =
2172 (field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
2173 value <= 1'd0);
2174 field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.ne
2175 xt = next_c;
2176 field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.lo
2177 ad_next = load_next_c;
2178     end
2179     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2180         if(~hwif_in.reset_b) begin
2181             field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
2182             value <= 1'h0;
2183         end else
2184             if(field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
2185             load_next) begin
2186                 field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
2187                 value <=
2188                 field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.ne
2189 xt;
2190             end
2191         end
2192 //-----
2193 // Write response
2194 //-----
2195 assign cpuif_wr_ack = decoded_req & decoded_req_is_wr;
2196 // Writes are always granted with no error response
2197 assign cpuif_wr_err = '0;
2198 //-----
2199 // Readback
2200 //-----
2201 logic readback_err;
2202 logic readback_done;
2203 logic [31:0] readback_data;
2204 // Assign readback values to a flattened array
2205 logic [61:1:0][31:0] readback_array;
2206 for(genvar i0=0; i0<2; i0++) begin
2207     assign readback_array[i0*1 + 0][31:0] = (de-
2208 coded_reg_strb.SHA512_NAME[i0] && !decoded_req_is_wr) ?
2209 hwif_in.SHA512_NAME[i0].NAME.next : '0;
2210 end
2211 for(genvar i0=0; i0<2; i0++) begin
2212     assign readback_array[i0*1 + 2][31:0] = (de-
2213 coded_reg_strb.SHA512_VERSION[i0] && !decoded_req_is_wr) ?
2214 hwif_in.SHA512_VERSION[i0].VERSION.next : '0;
2215 end
2216 assign readback_array[4][0:0] = (de-
2217 coded_reg_strb.SHA512_STATUS && !decoded_req_is_wr) ?
2218 hwif_in.SHA512_STATUS.READY.next : '0;
2219 assign readback_array[4][1:1] = (de-
2220 coded_reg_strb.SHA512_STATUS && !decoded_req_is_wr) ?
2221 hwif_in.SHA512_STATUS.VALID.next : '0;
2222 assign readback_array[4][31:2] = '0;
2223 for(genvar i0=0; i0<16; i0++) begin
2224     assign readback_array[i0*1 + 5][31:0] = (de-
2225 coded_reg_strb.SHA512_DIGEST[i0] && !decoded_req_is_wr) ?
2226 field_storage.SHA512_DIGEST[i0].DIGEST.value : '0;
2227 end
2228 assign readback_array[21][0:0] = (de-
2229 coded_reg_strb.SHA512_VAULT_RD_CTRL && !decoded_req_is_wr) ?
2230 field_storage.SHA512_VAULT_RD_CTRL.read_en.value : '0;
2231 assign readback_array[21][5:1] = (de-
2232 coded_reg_strb.SHA512_VAULT_RD_CTRL && !decoded_req_is_wr) ?
2233 field_storage.SHA512_VAULT_RD_CTRL.read_entry.value : '0;
2234 assign readback_array[21][6:6] = (de-
2235 coded_reg_strb.SHA512_VAULT_RD_CTRL && !decoded_req_is_wr) ?
2236 field_storage.SHA512_VAULT_RD_CTRL.pcr_hash_extend.value : '0;
2237 assign readback_array[21][31:7] = (de-

```

```

coded_reg_strb.SHA512_VAULT_RD_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_VAULT_RD_CTRL.rsvd.value : '0';
2210 assign readback_array[22][0:0] = (de-
coded_reg_strb.SHA512_VAULT_RD_STATUS && !decoded_req_is_wr) ?
hwif_in.SHA512_VAULT_RD_STATUS.READY.next : '0';
2211 assign readback_array[22][1:1] = (de-
coded_reg_strb.SHA512_VAULT_RD_STATUS && !decoded_req_is_wr) ?
field_storage.SHA512_VAULT_RD_STATUS.VALID.value : '0';
2212 assign readback_array[22][9:2] = (de-
coded_reg_strb.SHA512_VAULT_RD_STATUS && !decoded_req_is_wr) ?
hwif_in.SHA512_VAULT_RD_STATUS.ERROR.next : '0';
2213 assign readback_array[22][31:10] = '0';
2214 assign readback_array[23][0:0] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.write_en.value : '0';
2215 assign readback_array[23][5:1] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.write_entry.value : '0';
2216 assign readback_array[23][6:6] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.hmac_key_dest_valid.value : '0';
2217 assign readback_array[23][7:7] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.hmac_block_dest_valid.value : '0';
2218 assign readback_array[23][8:8] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.value : '0';
2219 assign readback_array[23][9:9] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.value : '0';
2220 assign readback_array[23][10:10] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.value : '0';
2221 assign readback_array[23][11:11] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.aes_key_dest_valid.value : '0';
2222 assign readback_array[23][12:12] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.value : '0';
2223 assign readback_array[23][13:13] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.value : '0';
2224 assign readback_array[23][14:14] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.dma_data_dest_valid.value : '0';
2225 assign readback_array[23][31:15] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.rsrvd.value : '0';
2226 assign readback_array[24][0:0] = (de-
coded_reg_strb.SHA512_KV_WR_STATUS && !decoded_req_is_wr) ?
hwif_in.SHA512_KV_WR_STATUS.READY.next : '0';
2227 assign readback_array[24][1:1] = (de-
coded_reg_strb.SHA512_KV_WR_STATUS && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_STATUS.VALID.value : '0';
2228 assign readback_array[24][9:2] = (de-
coded_reg_strb.SHA512_KV_WR_STATUS && !decoded_req_is_wr) ?
hwif_in.SHA512_KV_WR_STATUS.ERROR.next : '0';
2229 assign readback_array[24][31:10] = '0';
2230 assign readback_array[25][0:0] = (de-
coded_reg_strb.SHA512_GEN_PCR_HASH_STATUS && !decoded_req_is_wr) ?
hwif_in.SHA512_GEN_PCR_HASH_STATUS.READY.next : '0';
2231 assign readback_array[25][1:1] = (de-
coded_reg_strb.SHA512_GEN_PCR_HASH_STATUS && !decoded_req_is_wr) ?
field_storage.SHA512_GEN_PCR_HASH_STATUS.VALID.value : '0';
2232 assign readback_array[25][31:2] = '0';
2233 for(genvar i0=0; i0<16; i0++) begin
2234 assign readback_array[i0*1 + 26][31:0] = (de-
coded_reg_strb.SHA512_GEN_PCR_HASH_DIGEST[i0] && !de-
coded_req_is_wr) ?
field_storage.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.value : '0';
2235 end
2236 assign readback_array[42][0:0] = (de-
coded_reg_strb.intr_block_rf.global_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.global_intr_en_r.error_en.value : '0';
2237 assign readback_array[42][1:1] = (de-
coded_reg_strb.intr_block_rf.global_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.global_intr_en_r.notif_en.value : '0';
2238 assign readback_array[42][31:2] = '0';
2239 assign readback_array[43][0:0] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error0_en.value : '0';
2240 assign readback_array[43][1:1] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error1_en.value : '0';
2241 assign readback_array[43][2:2] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error2_en.value : '0';
2242 assign readback_array[43][3:3] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error3_en.value : '0';
2243 assign readback_array[43][31:4] = '0';
2244 assign readback_array[44][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value :
: '0';
2245 assign readback_array[44][31:1] = '0';
2246 assign readback_array[45][0:0] = (de-
coded_reg_strb.intr_block_rf.error_global_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_global_intr_r.agg_sts.value : '0';
2247 assign readback_array[45][31:1] = '0';
2248 assign readback_array[46][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_global_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value : '0';
2249 assign readback_array[46][31:1] = '0';
2250 assign readback_array[47][0:0] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value :
'0';
2251 assign readback_array[47][1:1] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value :
'0';
2252 assign readback_array[47][2:2] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value :
'0';
2253 assign readback_array[47][3:3] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value :
'0';
2254 assign readback_array[47][31:4] = '0';
2255 assign readback_array[48][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value : '0';
2256 assign readback_array[48][31:1] = '0';
2257 assign readback_array[49][0:0] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value :
'0';
2258 assign readback_array[49][1:1] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value :
'0';
2259 assign readback_array[49][2:2] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value :
'0';
2260 assign readback_array[49][3:3] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value :
'0';
2261 assign readback_array[49][31:4] = '0';
2262 assign readback_array[50][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue : '0';
2263 assign readback_array[50][31:1] = '0';
2264 assign readback_array[51][31:0] = (de-
coded_reg_strb.intr_block_rf.error0_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error0_intr_count_r.cnt.value : '0';
2265 assign readback_array[52][31:0] = (de-
coded_reg_strb.intr_block_rf.error1_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error1_intr_count_r.cnt.value : '0';
2266 assign readback_array[53][31:0] = (de-
coded_reg_strb.intr_block_rf.error2_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error2_intr_count_r.cnt.value : '0';
2267 assign readback_array[54][31:0] = (de-
coded_reg_strb.intr_block_rf.error3_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error3_intr_count_r.cnt.value : '0';
2268 assign readback_array[55][31:0] = (de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value :
'0';
2269 assign readback_array[56][0:0] = (de-
coded_reg_strb.intr_block_rf.error0_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value :
'0';
2270 assign readback_array[56][31:1] = '0';
2271 assign readback_array[57][0:0] = (de-
coded_reg_strb.intr_block_rf.error1_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value :
'0';
2272 assign readback_array[57][31:1] = '0';
2273 assign readback_array[58][0:0] = (de-
coded_reg_strb.intr_block_rf.error2_intr_count_incr_r && !de-

```



```

coded_req_is_wr) ?
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value :
'0;
2274 assign readback_array[58][31:1] = '0;
2275 assign readback_array[59][0:0] = (de-
coded_req_strb.intr_block_rf.error3_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value :
'0;
2276 assign readback_array[59][31:1] = '0;
2277 assign readback_array[60][0:0] = (de-
coded_req_strb.intr_block_rf.notif_cmd_done_intr_count_incr_r && !
decoded_req_is_wr) ?
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value : '0;
2278 assign readback_array[60][31:1] = '0;
2279
2280 // Reduce the array
2281 always_comb begin
2282     automatic logic [31:0] readback_data_var;
2283     readback_done = decoded_req & ~decoded_req_is_wr;
2284     readback_err = '0;
2285     readback_data_var = '0;
2286     for(int i=0; i<61; i++) readback_data_var |= read-
back_array[i];
2287     readback_data = readback_data_var;
2288 end
2289
2290 assign cpuif_rd_ack = readback_done;
2291 assign cpuif_rd_data = readback_data;
2292 assign cpuif_rd_err = readback_err;
2293
2294 `include "caliptra_prim_assert.sv"
2295 `CALIPTRA_ASSERT_KNOWN(ERR_HWIF_IN, hwif_in, clk, !hwif_in.er-
ror_reset_b)
2296
2297 endmodule

```

8. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_reg_pkg.sv

(446 lines)

```

1 // Generated by PeakRDL-regblock - A free and open-source Sys-
temVerilog generator
2 // https://github.com/SystemRDL/PeakRDL-regblock
3
4 package sha512_reg_pkg;
5
6     localparam SHA512_REG_DATA_WIDTH = 32;
7     localparam SHA512_REG_MIN_ADDR_WIDTH = 12;
8
9     typedef struct packed{
10         logic [31:0] next;
11     } sha512_reg_SHA512_NAME__NAME__in_t;
12
13     typedef struct packed{
14         sha512_reg_SHA512_NAME__NAME__in_t NAME;
15     } sha512_reg_SHA512_NAME__in_t;
16
17     typedef struct packed{
18         logic [31:0] next;
19     } sha512_reg_SHA512_VERSION__VERSION__in_t;
20
21     typedef struct packed{
22         sha512_reg_SHA512_VERSION__VERSION__in_t VERSION;
23     } sha512_reg_SHA512_VERSION__in_t;
24
25     typedef struct packed{
26         logic hwcldr;
27     } sha512_reg_SHA512_CTRL__LAST__in_t;
28
29     typedef struct packed{
30         sha512_reg_SHA512_CTRL__LAST__in_t LAST;
31     } sha512_reg_SHA512_CTRL__in_t;
32
33     typedef struct packed{
34         logic next;
35     } sha512_reg_SHA512_STATUS__READY__in_t;
36
37     typedef struct packed{
38         logic next;
39     } sha512_reg_SHA512_STATUS__VALID__in_t;
40
41     typedef struct packed{
42         sha512_reg_SHA512_STATUS__READY__in_t READY;
43         sha512_reg_SHA512_STATUS__VALID__in_t VALID;
44     } sha512_reg_SHA512_STATUS__in_t;
45
46     typedef struct packed{
47         logic [31:0] next;
48         logic we;
49         logic swwe1;
50         logic hwcldr;
51     } sha512_reg_SHA512_BLOCK__BLOCK__in_t;
52
53     typedef struct packed{
54         sha512_reg_SHA512_BLOCK__BLOCK__in_t BLOCK;
55     } sha512_reg_SHA512_BLOCK__in_t;
56
57     typedef struct packed{
58         logic [31:0] next;

```

```

59         logic we;
60         logic hwcldr;
61     } sha512_reg_SHA512_DIGEST__DIGEST__in_t;
62
63     typedef struct packed{
64         sha512_reg_SHA512_DIGEST__DIGEST__in_t DIGEST;
65     } sha512_reg_SHA512_DIGEST__in_t;
66
67     typedef struct packed{
68         logic swwe;
69         logic hwcldr;
70     } kv_read_ctrl_reg__read_en__in_t;
71
72     typedef struct packed{
73         logic swwe;
74     } kv_read_ctrl_reg__read_entry__in_t;
75
76     typedef struct packed{
77         logic swwe;
78     } kv_read_ctrl_reg__pcr_hash_extend__in_t;
79
80     typedef struct packed{
81         logic swwe;
82     } kv_read_ctrl_reg__rsvd__in_t;
83
84     typedef struct packed{
85         kv_read_ctrl_reg__read_en__in_t read_en;
86         kv_read_ctrl_reg__read_entry__in_t read_entry;
87         kv_read_ctrl_reg__pcr_hash_extend__in_t
pcr_hash_extend;
88         kv_read_ctrl_reg__rsvd__in_t rsvd;
89     } kv_read_ctrl_reg__in_t;
90
91     typedef struct packed{
92         logic next;
93     } kv_status_reg__READY__in_t;
94
95     typedef struct packed{
96         logic hwcldr;
97         logic hwset;
98     } kv_status_reg__VALID__in_t;
99
100     typedef struct packed{
101         logic [7:0] next;
102     } kv_status_reg__ERROR__in_t;
103
104     typedef struct packed{
105         kv_status_reg__READY__in_t READY;
106         kv_status_reg__VALID__in_t VALID;
107         kv_status_reg__ERROR__in_t ERROR;
108     } kv_status_reg__in_t;
109
110     typedef struct packed{
111         logic swwe;
112         logic hwcldr;
113     } kv_write_ctrl_reg__write_en__in_t;
114
115     typedef struct packed{
116         logic swwe;
117     } kv_write_ctrl_reg__write_entry__in_t;
118
119     typedef struct packed{
120         logic swwe;
121     } kv_write_ctrl_reg__hmac_key_dest_valid__in_t;
122
123     typedef struct packed{
124         logic swwe;
125     } kv_write_ctrl_reg__hmac_block_dest_valid__in_t;
126
127     typedef struct packed{
128         logic swwe;
129     } kv_write_ctrl_reg__mldsa_seed_dest_valid__in_t;
130
131     typedef struct packed{
132         logic swwe;
133     } kv_write_ctrl_reg__ecc_pkey_dest_valid__in_t;
134
135     typedef struct packed{
136         logic swwe;
137     } kv_write_ctrl_reg__ecc_seed_dest_valid__in_t;
138
139     typedef struct packed{
140         logic swwe;
141     } kv_write_ctrl_reg__aes_key_dest_valid__in_t;
142
143     typedef struct packed{
144         logic swwe;
145     } kv_write_ctrl_reg__mlkem_seed_dest_valid__in_t;
146
147     typedef struct packed{
148         logic swwe;
149     } kv_write_ctrl_reg__mlkem_msg_dest_valid__in_t;
150
151     typedef struct packed{
152         logic swwe;
153     } kv_write_ctrl_reg__dma_data_dest_valid__in_t;
154
155     typedef struct packed{
156         logic swwe;
157     } kv_write_ctrl_reg__rsvd__in_t;
158

```

```

159     typedef struct packed{
160         kv_write_ctrl_reg_write_en_in_t write_en;
161         kv_write_ctrl_reg_write_entry_in_t write_entry;
162         kv_write_ctrl_reg_hmac_key_dest_valid_in_t
hmac_key_dest_valid;
163         kv_write_ctrl_reg_hmac_block_dest_valid_in_t hmac_b-
lock_dest_valid;
164         kv_write_ctrl_reg_mldsa_seed_dest_valid_in_t
mldsa_seed_dest_valid;
165         kv_write_ctrl_reg_ecc_pkey_dest_valid_in_t ecc_p-
key_dest_valid;
166         kv_write_ctrl_reg_ecc_seed_dest_valid_in_t
ecc_seed_dest_valid;
167         kv_write_ctrl_reg_aes_key_dest_valid_in_t aes_key_d-
est_valid;
168         kv_write_ctrl_reg_mlkem_seed_dest_valid_in_t
mlkem_seed_dest_valid;
169         kv_write_ctrl_reg_mlkem_msg_dest_valid_in_t
mlkem_msg_dest_valid;
170         kv_write_ctrl_reg_dma_data_dest_valid_in_t
dma_data_dest_valid;
171         kv_write_ctrl_reg_rsvd_in_t rsvd;
172     } kv_write_ctrl_reg_in_t;
173
174     typedef struct packed{
175         logic next;
176     } sha512_reg_SHA512_GEN_PCR_HASH_STATUS_READY_in_t;
177
178     typedef struct packed{
179         logic hwclr;
180         logic hwset;
181     } sha512_reg_SHA512_GEN_PCR_HASH_STATUS_VALID_in_t;
182
183     typedef struct packed{
184         sha512_reg_SHA512_GEN_PCR_HASH_STATUS_READY_in_t
READY;
185         sha512_reg_SHA512_GEN_PCR_HASH_STATUS_VALID_in_t
VALID;
186     } sha512_reg_SHA512_GEN_PCR_HASH_STATUS_in_t;
187
188     typedef struct packed{
189         logic [31:0] next;
190         logic hwclr;
191     } sha512_reg_SHA512_GEN_PCR_HASH_DIGEST_DIGEST_in_t;
192
193     typedef struct packed{
194         sha512_reg_SHA512_GEN_PCR_HASH_DIGEST_DIGEST_in_t
DIGEST;
195     } sha512_reg_SHA512_GEN_PCR_HASH_DIGEST_in_t;
196
197     typedef struct packed{
198         logic hwset;
199     }
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_error0_sts_enable_528ccada_n
ext_b1018582_resetsignal_939e99d4_in_t;
200
201     typedef struct packed{
202         logic hwset;
203     }
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_error1_sts_enable_938cafef_n
ext_f460eb81_resetsignal_939e99d4_in_t;
204
205     typedef struct packed{
206         logic hwset;
207     }
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_error2_sts_enable_0dacf7a6_n
ext_4b5b9e74_resetsignal_939e99d4_in_t;
208
209     typedef struct packed{
210         logic hwset;
211     }
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_error3_sts_enable_fc3af94b_n
ext_c3125d40_resetsignal_939e99d4_in_t;
212
213     typedef struct packed{
214
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_error0_sts_enable_528ccada_n
ext_b1018582_resetsignal_939e99d4_in_t error0_sts;
215
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_error1_sts_enable_938cafef_n
ext_f460eb81_resetsignal_939e99d4_in_t error1_sts;
216
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_error2_sts_enable_0dacf7a6_n
ext_4b5b9e74_resetsignal_939e99d4_in_t error2_sts;
217
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_error3_sts_enable_fc3af94b_n
ext_c3125d40_resetsignal_939e99d4_in_t error3_sts;
218
    }
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_in_t;
219
220     typedef struct packed{
221         logic hwset;

```

```

222     }
sha512_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_notif_cmd_don
e_sts_enable_dabe0b8b_next_540fa3b7_in_t;
223
224     typedef struct packed{
225
sha512_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_notif_cmd_don
e_sts_enable_dabe0b8b_next_540fa3b7_in_t notif_cmd_done_sts;
226
    }
sha512_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_in_t;
227
228     typedef struct packed{
229
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_in_t error_internal_intr_r;
230
sha512_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_in_t no-
tif_internal_intr_r;
231
    } sha512_reg_intr_block_t_in_t;
232
233     typedef struct packed{
234         logic reset_b;
235         logic error_reset_b;
236         logic sha512_ready;
237         sha512_reg_SHA512_NAME_in_t [2-1:0]SHA512_NAME;
238         sha512_reg_SHA512_VERSION_in_t
[2-1:0]SHA512_VERSION;
239         sha512_reg_SHA512_CTRL_in_t SHA512_CTRL;
240         sha512_reg_SHA512_STATUS_in_t SHA512_STATUS;
241         sha512_reg_SHA512_BLOCK_in_t [32-1:0]SHA512_BLOCK;
242         sha512_reg_SHA512_DIGEST_in_t [16-1:0]SHA512_DIGEST;
243         kv_read_ctrl_reg_in_t SHA512_VAULT_RD_CTRL;
244         kv_status_reg_in_t SHA512_VAULT_RD_STATUS;
245         kv_write_ctrl_reg_in_t SHA512_KV_WR_CTRL;
246         kv_status_reg_in_t SHA512_KV_WR_STATUS;
247         sha512_reg_SHA512_GEN_PCR_HASH_STATUS_in_t
SHA512_GEN_PCR_HASH_STATUS;
248         sha512_reg_SHA512_GEN_PCR_HASH_DIGEST_in_t
[16-1:0]SHA512_GEN_PCR_HASH_DIGEST;
249         sha512_reg_intr_block_t_in_t intr_block_rf;
250     } sha512_reg_in_t;
251
252     typedef struct packed{
253         logic value;
254     } sha512_reg_SHA512_CTRL_INIT_out_t;
255
256     typedef struct packed{
257         logic value;
258     } sha512_reg_SHA512_CTRL_NEXT_out_t;
259
260     typedef struct packed{
261         logic [1:0] value;
262     } sha512_reg_SHA512_CTRL_MODE_out_t;
263
264     typedef struct packed{
265         logic value;
266     } sha512_reg_SHA512_CTRL_ZEROIZE_out_t;
267
268     typedef struct packed{
269         logic value;
270     } sha512_reg_SHA512_CTRL_LAST_out_t;
271
272     typedef struct packed{
273         logic value;
274     } sha512_reg_SHA512_CTRL_RESTORE_out_t;
275
276     typedef struct packed{
277         sha512_reg_SHA512_CTRL_INIT_out_t INIT;
278         sha512_reg_SHA512_CTRL_NEXT_out_t NEXT;
279         sha512_reg_SHA512_CTRL_MODE_out_t MODE;
280         sha512_reg_SHA512_CTRL_ZEROIZE_out_t ZEROIZE;
281         sha512_reg_SHA512_CTRL_LAST_out_t LAST;
282         sha512_reg_SHA512_CTRL_RESTORE_out_t RESTORE;
283     } sha512_reg_SHA512_CTRL_out_t;
284
285     typedef struct packed{
286         logic [31:0] value;
287     } sha512_reg_SHA512_BLOCK_BLOCK_out_t;
288
289     typedef struct packed{
290         sha512_reg_SHA512_BLOCK_BLOCK_out_t BLOCK;
291     } sha512_reg_SHA512_BLOCK_out_t;
292
293     typedef struct packed{
294         logic [31:0] value;
295     } sha512_reg_SHA512_DIGEST_DIGEST_out_t;
296
297     typedef struct packed{
298         sha512_reg_SHA512_DIGEST_DIGEST_out_t DIGEST;
299     } sha512_reg_SHA512_DIGEST_out_t;
300
301     typedef struct packed{
302         logic value;
303     } kv_read_ctrl_reg_read_en_out_t;
304
305     typedef struct packed{
306         logic [4:0] value;
307     } kv_read_ctrl_reg_read_entry_out_t;
308
309     typedef struct packed{
310         logic value;

```



```

311 } kv_read_ctrl_reg_pcr_hash_extend_out_t;
312
313 typedef struct packed{
314     logic [24:0] value;
315 } kv_read_ctrl_reg_rsvd_out_t;
316
317 typedef struct packed{
318     kv_read_ctrl_reg_read_en_out_t read_en;
319     kv_read_ctrl_reg_read_entry_out_t read_entry;
320     kv_read_ctrl_reg_pcr_hash_extend_out_t
pcr_hash_extend;
321     kv_read_ctrl_reg_rsvd_out_t rsvd;
322 } kv_read_ctrl_reg_out_t;
323
324 typedef struct packed{
325     logic value;
326 } kv_write_ctrl_reg_write_en_out_t;
327
328 typedef struct packed{
329     logic [4:0] value;
330 } kv_write_ctrl_reg_write_entry_out_t;
331
332 typedef struct packed{
333     logic value;
334 } kv_write_ctrl_reg_hmac_key_dest_valid_out_t;
335
336 typedef struct packed{
337     logic value;
338 } kv_write_ctrl_reg_hmac_block_dest_valid_out_t;
339
340 typedef struct packed{
341     logic value;
342 } kv_write_ctrl_reg_mldsa_seed_dest_valid_out_t;
343
344 typedef struct packed{
345     logic value;
346 } kv_write_ctrl_reg_ecc_pkey_dest_valid_out_t;
347
348 typedef struct packed{
349     logic value;
350 } kv_write_ctrl_reg_ecc_seed_dest_valid_out_t;
351
352 typedef struct packed{
353     logic value;
354 } kv_write_ctrl_reg_aes_key_dest_valid_out_t;
355
356 typedef struct packed{
357     logic value;
358 } kv_write_ctrl_reg_mlkem_seed_dest_valid_out_t;
359
360 typedef struct packed{
361     logic value;
362 } kv_write_ctrl_reg_mlkem_msg_dest_valid_out_t;
363
364 typedef struct packed{
365     logic value;
366 } kv_write_ctrl_reg_dma_data_dest_valid_out_t;
367
368 typedef struct packed{
369     logic [16:0] value;
370 } kv_write_ctrl_reg_rsvd_out_t;
371
372 typedef struct packed{
373     kv_write_ctrl_reg_write_en_out_t write_en;
374     kv_write_ctrl_reg_write_entry_out_t write_entry;
375     kv_write_ctrl_reg_hmac_key_dest_valid_out_t
hmac_key_dest_valid;
376     kv_write_ctrl_reg_hmac_block_dest_valid_out_t
hmac_block_dest_valid;
377     kv_write_ctrl_reg_mldsa_seed_dest_valid_out_t
mldsa_seed_dest_valid;
378     kv_write_ctrl_reg_ecc_pkey_dest_valid_out_t ecc_p-
key_dest_valid;
379     kv_write_ctrl_reg_ecc_seed_dest_valid_out_t
ecc_seed_dest_valid;
380     kv_write_ctrl_reg_aes_key_dest_valid_out_t
aes_key_dest_valid;
381     kv_write_ctrl_reg_mlkem_seed_dest_valid_out_t
mlkem_seed_dest_valid;
382     kv_write_ctrl_reg_mlkem_msg_dest_valid_out_t

```

```

mlkem_msg_dest_valid;
383     kv_write_ctrl_reg_dma_data_dest_valid_out_t
dma_data_dest_valid;
384     kv_write_ctrl_reg_rsvd_out_t rsvd;
385 } kv_write_ctrl_reg_out_t;
386
387 typedef struct packed{
388     logic [31:0] value;
389 } sha512_reg_SHA512_GEN_PCR_HASH_NONCE_NONCE_out_t;
390
391 typedef struct packed{
392     sha512_reg_SHA512_GEN_PCR_HASH_NONCE_NONCE_out_t
NONCE;
393     sha512_reg_SHA512_GEN_PCR_HASH_NONCE_NONCE_out_t;
394
395 typedef struct packed{
396     logic value;
397 } sha512_reg_SHA512_GEN_PCR_HASH_CTRL_START_out_t;
398
399 typedef struct packed{
400     sha512_reg_SHA512_GEN_PCR_HASH_CTRL_START_out_t
START;
401     sha512_reg_SHA512_GEN_PCR_HASH_CTRL_out_t;
402
403 typedef struct packed{
404     logic intr;
405 } sha512_reg_global_intr_t_agg_sts_dd3dcf0a_out_t;
406
407 typedef struct packed{
408     logic intr;
409 } sha512_reg_global_intr_t_agg_sts_e6399b4a_out_t;
410
411 typedef struct packed{
412     logic intr;
413 }
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_out_t;
414
415 typedef struct packed{
416     logic intr;
417 }
sha512_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_out_t;
418
419 typedef struct packed{
420     sha512_reg_global_intr_t_agg_sts_dd3dcf0a_out_t er-
ror_global_intr_r;
421     sha512_reg_global_intr_t_agg_sts_e6399b4a_out_t no-
tif_global_intr_r;
422
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_out_t error_internal_intr_r;
423
sha512_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_out_t no-
tif_internal_intr_r;
424     sha512_reg_intr_block_t_out_t;
425
426 typedef struct packed{
427     sha512_reg_SHA512_CTRL_out_t SHA512_CTRL;
428     sha512_reg_SHA512_BLOCK_out_t [32-1:0]SHA512_BLOCK;
429     sha512_reg_SHA512_DIGEST_out_t
[16-1:0]SHA512_DIGEST;
430     kv_read_ctrl_reg_out_t SHA512_VAULT_RD_CTRL;
431     kv_write_ctrl_reg_out_t SHA512_KV_WR_CTRL;
432     sha512_reg_SHA512_GEN_PCR_HASH_NONCE_out_t
[8-1:0]SHA512_GEN_PCR_HASH_NONCE;
433     sha512_reg_SHA512_GEN_PCR_HASH_CTRL_out_t
SHA512_GEN_PCR_HASH_CTRL;
434     sha512_reg_intr_block_t_out_t intr_block_rf;
435 } sha512_reg_out_t;
436
437 typedef enum logic [31:0] {
438     kv_status_reg_ERROR_kv_error_e_SUCCESS = 'h0,
439     kv_status_reg_ERROR_kv_error_e_KV_READ_FAIL = 'h1,
440     kv_status_reg_ERROR_kv_error_e_KV_WRITE_FAIL = 'h2,
441     kv_status_reg_ERROR_kv_error_e_KV_FSM_ERROR = 'h4
442 } kv_status_reg_ERROR_kv_error_e;
443
444 localparam SHA512_REG_ADDR_WIDTH = 32'd12;
445
446 endpackage

```

Caliptra SHA-256

What this is. SHA-256 in a root-of-trust context.

Why it exists. Because the boot measurement chain requires it and a root of trust cannot rely on software for its own integrity check.

Undergraduate review · EECS concepts. Hash functions; boot-time measurement; chain of trust.

Graduate · Practice note. A root of trust is interesting less for its algorithms than for its *ordering* guarantees: what is measured before what, and what cannot be changed afterwards.

9 files • 4,409 lines

API — sha256.sv

Table 1338. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256.sv — module / interface / package (1개)

이름	종류
sha256	module

Table 1339. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256.sv — parameter (2개)

파라미터	형/폭	기본값
ADDR_WIDTH		32
DATA_WIDTH		32

Table 1340. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256.sv — localparam (1개)

상수	형/폭	값
BLOCK_NO		512 / DATA_WIDTH

Table 1341. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256.sv — 포트 (12개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
cptra_pwrgood	input	wire
cs	input	wire
we	input	wire
address	input	wire [ADDR_WIDTH-1 : 0]
write_data	input	wire [DATA_WIDTH-1 : 0]
read_data	output	wire [DATA_WIDTH-1 : 0]
err	output	wire
error_intr	output	wire

포트	방향	형/폭
notif_intr	output	wire
debugUnlock_or_scan_mode_switch	input	logic

Table 1342. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256.sv — typedef / using (1개)

새 이름	원래 형
wntz_fsm_t	enum logic [1:0] {WNTZ_IDLE, WNTZ_1ST, WNTZ_OTHERS}

API — sha256_core.v**Table 1343. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_core.v — module / interface / package (1개)**

이름	종류
sha256_core	module

Table 1344. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_core.v — localparam (20개)

상수	형/폭	값
SHA224_H0_0		32'hc1059ed8
SHA224_H0_1		32'h367cd507
SHA224_H0_2		32'h3070dd17
SHA224_H0_3		32'hf70e5939
SHA224_H0_4		32'hffc00b31
SHA224_H0_5		32'h68581511
SHA224_H0_6		32'h64f98fa7
SHA224_H0_7		32'hbfa4fa4
SHA256_H0_0		32'h6a09e667
SHA256_H0_1		32'hbb67ae85
SHA256_H0_2		32'h3c6ef372
SHA256_H0_3		32'ha54ff53a
SHA256_H0_4		32'h510e527f
SHA256_H0_5		32'h9b05688c
SHA256_H0_6		32'h1f83d9ab
SHA256_H0_7		32'h5be0cd19
SHA256_ROUNDS		63
CTRL_IDLE	[1:0]	0
CTRL_ROUNDS	[1:0]	1
CTRL_DONE	[1:0]	2

Table 1345. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_core.v — 포트 (10개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
init_cmd	input	wire
next_cmd	input	wire
mode	input	wire
block_msg	input	wire [511 : 0]
ready	output	wire
digest	output	wire [255 : 0]
digest_valid	output	wire

API — sha256_ctrl.sv

Table 1346. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_ctrl.sv — module / interface / package (1개)

이름	종류
sha256_ctrl	module

Table 1347. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_ctrl.sv — parameter (2개)

파라미터	형/폭	기본값
AHB_DATA_WIDTH		32
AHB_ADDR_WIDTH		32

Table 1348. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_ctrl.sv — 포트 (16개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
cptra_pwrgood	input	wire
haddr_i	input	logic [AHB_ADDR_WIDTH-1:0]
hdata_i	input	logic [AHB_DATA_WIDTH-1:0]
hsel_i	input	logic
hwrite_i	input	logic
hready_i	input	logic
htrans_i	input	logic [1:0]
hsize_i	input	logic [2:0]
hresp_o	output	logic
hreadyout_o	output	logic
hrdata_o	output	logic [AHB_DATA_WIDTH-1:0]

포트	방향	형/폭
error_intr	output	logic
notif_intr	output	logic
debugUnlock_or_scan_mode_switch	input	logic

API — sha256_k_constants.v

Table 1349. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_k_constants.v — module / interface / package (1개)

이름	종류
sha256_k_constants	module

Table 1350. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_k_constants.v — 포트 (2개)

포트	방향	형/폭
round	input	wire [5 : 0]
K_val	output	wire [31 : 0]

API — sha256_params_pkg.sv

Table 1351. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_params_pkg.sv — module / interface / package (1개)

이름	종류
sha256_params_pkg	package

Table 1352. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_params_pkg.sv — localparam (6개)

상수	형/폭	값
SHA256_CORE_NAME0		32'h61327368
SHA256_CORE_NAME1		32'h35362d32
SHA256_CORE_VERSION0		32'h3830312e
SHA256_CORE_VERSION1		32'h00000000
SHA256_MODE_SHA_224		1'h0
SHA256_MODE_SHA_256		1'h1

API — sha256_reg.sv

Table 1353. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_reg.sv — module / interface / package (1개)

이름	종류
sha256_reg	module

Table 1354. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_reg.sv — 포트 (16개)

포트	방향	형/폭
clk	input	wire
rst	input	wire

포트	방향	형/폭
s_cpuif_req	input	wire
s_cpuif_req_is_wr	input	wire
s_cpuif_addr	input	wire [11:0]
s_cpuif_wr_data	input	wire [31:0]
s_cpuif_wr_biten	input	wire [31:0]
s_cpuif_req_stall_wr	output	wire
s_cpuif_req_stall_rd	output	wire
s_cpuif_rd_ack	output	wire
s_cpuif_rd_err	output	wire
s_cpuif_rd_data	output	wire [31:0]
s_cpuif_wr_ack	output	wire
s_cpuif_wr_err	output	wire
hwif_in	input	sha256_reg_pkg::sha256_reg__in_t
hwif_out	output	sha256_reg_pkg::sha256_reg__out_t

API — sha256_reg_pkg.sv

Table 1355. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_reg_pkg.sv — module / interface / package (1개)

이름	종류
sha256_reg_pkg	package

Table 1356. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_reg_pkg.sv — localparam (3개)

상수	형/폭	값
SHA256_REG_DATA_WIDTH		32
SHA256_REG_MIN_ADDR_WIDTH		12
SHA256_REG_ADDR_WIDTH		32'd12

API — sha256_reg_uvm.sv

Table 1357. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_reg_uvm.sv — module / interface / package (1개)

이름	종류
sha256_reg_uvm	package

API — sha256_w_mem.v

Table 1358. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_w_mem.v — module / interface / package (1개)

이름	종류
sha256_w_mem	module

Table 1359. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_w_mem.v — 포트 (7개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
block_msg	input	wire [511 : 0]
init_cmd	input	wire
next_cmd	input	wire
w_val	output	wire [31 : 0]

1. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256.v (446 lines)

```

1 //
=====
2 //
3 // Updated by Caliptra team to modify data access width
4 //
5 // sha256.v
6 // -----
7 // Top level wrapper for the SHA-256 hash function providing
8 // a simple memory like interface with 32 bit data access.
9 //
10 //
11 // Author: Joachim Strombergson
12 // Copyright (c) 2013, 201, Secworks Sweden AB
13 // All rights reserved.
14 //
15 // Redistribution and use in source and binary forms, with or
16 // without modification, are permitted provided that the fol-
lowing
17 // conditions are met:
18 //
19 // 1. Redistributions of source code must retain the above
copyright
20 // notice, this list of conditions and the following dis-
claimer.
21 //
22 // 2. Redistributions in binary form must reproduce the above
copyright
23 // notice, this list of conditions and the following dis-
claimer in
24 // the documentation and/or other materials provided with
the
25 // distribution.
26 //
27 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
TRIBUTORS
28 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
29 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
30 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
31 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
32 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
33 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
34 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
35 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
36 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
37 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
38 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
39 //
40 //
=====
41 //
42 module sha256
43     import sha256_reg_pkg::*;
44     import sha256_params_pkg::*;
45     #(
46         parameter ADDR_WIDTH = 32,
47         parameter DATA_WIDTH = 32
48     )(
49         // Clock and reset.
50         input wire clk,
51         input wire reset_n,
52         input wire cptra_pwrgood,

```

```

53
54         // Control.
55         input wire cs,
56         input wire we,
57
58         // Data ports.
59         input wire [ADDR_WIDTH-1 : 0] address,
60         input wire [DATA_WIDTH-1 : 0] write_data,
61         output wire [DATA_WIDTH-1 : 0] read_data,
62         output wire err,
63
64
65         // Interrupts
66         output wire error_intr,
67         output wire notif_intr,
68         input logic debugUnlock_or_scan_mode_switch
69     );
70
71 //-----
72 // Internal constant and parameter definitions.
73 //-----
74 // `include "sha256_param.v"
75
76 //-----
77 // Registers including update variables and write enable.
78 //-----
79 reg init_reg;
80 reg next_reg;
81 reg mode_reg;
82 reg ready_reg;
83 reg zeroize_reg;
84 logic mode;
85
86 localparam BLOCK_NO = 512 / DATA_WIDTH;
87 reg [DATA_WIDTH-1 : 0] block_reg [BLOCK_NO-1 : 0];
88
89 reg [0 : 7][31 : 0] digest_reg;
90 reg [0 : 7][31 : 0] get_mask;
91 reg digest_valid_reg;
92
93 // Interrupts
94 sha256_reg_in_t hwif_in;
95 sha256_reg_out_t hwif_out;
96 logic read_error, write_error;
97
98 //-----
99 // Wires.
100 //-----
101 wire core_ready;
102 logic [511 : 0] core_block;
103 wire [0:7][31:0] core_digest; //Intentionally reverse
ordered to prepare block input in wntz mode
104 wire core_digest_valid;
105 logic core_digest_valid_reg;
106
107 logic ready_flag;
108 logic ready_flag_reg;
109 logic valid_flag;
110 logic valid_flag_reg;
111
112
113 logic wntz_enable;
114 logic wntz_j_inc;
115 logic [7:0] wntz_j_init, wntz_j_reg;
116 logic wntz_busy; // to regiser
117 logic wntz_mode; // from registers
118 logic wntz_n_mode, wntz_n_mode_reg;
119 logic [3:0] wntz_w, wntz_w_reg;

```



```

120 logic                wntz_w_invalid, wntz_mode_invalid,
wntz_j_invalid;
121 logic                invalid_sha_op;
122 logic                core_init, core_next, core_mode;
123 logic                wntz_init;
124 logic                wntz_init_reg;
125 logic                wntz_blk_done;
126 logic [7:0]          wntz_iter, wntz_iter_reg;
127 logic [175: 0]       wntz_prefix;
128
129 typedef enum logic [1:0] {WNTZ_IDLE, WNTZ_1ST, WNTZ_OTHERS}
wntz_fsm_t;
130 wntz_fsm_t            wntz_fsm, wntz_fsm_next;
131
132 //-----
133 // Concurrent connectivity for ports etc.
134 //-----
135
136 always_comb begin
137   if (wntz_busy) begin
138     core_block = wntz_n_mode_reg ?
139       //SHA256
140       {wntz_prefix, wntz_j_reg,
141        digest_reg[0], digest_reg[1],
142        digest_reg[2], digest_reg[3],
143        digest_reg[4], digest_reg[5],
144        digest_reg[6], digest_reg[7],
145        8'h80, // Padding
146        64'h01b8} // L = 440bits per SHA256
147       : //SHA192
148       {wntz_prefix, wntz_j_reg,
149        digest_reg[0], digest_reg[1],
150        digest_reg[2], digest_reg[3],
151        digest_reg[4], digest_reg[5],
152        8'h80, // Padding
153        64'h0, // L = 376bits per SHA256
154        64'h0178};
155   else
156     core_init = wntz_init_reg;
157     core_next = 1'b0;
158     core_mode = 1'b1; //always SHA256 for Winternitz.
159   end else begin
160     core_block = {block_reg[00], block_reg[01],
161                  block_reg[02], block_reg[03],
162                  block_reg[04], block_reg[05],
163                  block_reg[06], block_reg[07],
164                  block_reg[08], block_reg[09],
165                  block_reg[10], block_reg[11],
166                  block_reg[12], block_reg[13],
167                  block_reg[14], block_reg[15]};
168     core_init = init_reg;
169     core_next = next_reg;
170     core_mode = mode;
171   end
172 end // always_comb
173
174 assign err = read_error | write_error;
175
176 //-----
177 // core instantiation.
178 //-----
179 sha256_core core(
180   .clk(clk),
181   .reset_n(reset_n),
182   .zeroize(zeroize_reg),
183   .init_cmd(core_init),
184   .next_cmd(core_next),
185   .mode(core_mode),
186   .block_msg(core_block),
187   .ready(core_ready),
188   .digest(core_digest),
189   .digest_valid(core_digest_valid)
190 );
191
192 //-----
193 assign wntz_busy          = (wntz_fsm != WNTZ_IDLE);
194 assign wntz_blk_done      = core_digest_valid & ~core_di-
195   gest_valid_reg;
196 assign wntz_w_invalid     = wntz_busy & ~(wntz_w_reg inside
197   {'h1, 'h2, 'h4, 'h8});
198 assign wntz_mode_invalid = wntz_busy & ~mode_reg;
199 assign wntz_j_invalid     = wntz_mode & (wntz_j_init >
200   wntz_iter);
201 assign invalid_sha_op     = init_reg & next_reg; //Trigger

```

an error when init and next are high in the same cycle

```

199
200 always_comb begin
201   unique case(wntz_w)
202     4'h1: wntz_iter = 'd0; //2**w - 1 (-1) (1st itera-
203   tion is considered separately)
204     4'h2: wntz_iter = 'd2;
205     4'h4: wntz_iter = 'd14;
206     4'h8: wntz_iter = 'd254;
207     default: wntz_iter = 'd0;
208   endcase
209 end
210
211 assign wntz_j_init = block_reg[5][15:8];
212
213 always_comb begin
214   wntz_init = 1'b0;
215   wntz_enable = 1'b0;
216   wntz_j_inc = 1'b0;
217   case (wntz_fsm)
218     WNTZ_IDLE:
219       begin
220         if (wntz_mode && init_reg && ready_reg &&
221           (wntz_j_init <= wntz_iter)) begin
222           wntz_fsm_next = WNTZ_1ST;
223           wntz_enable = 1'b1;
224         end else
225           wntz_fsm_next = WNTZ_IDLE;
226       end
227     WNTZ_1ST:
228       begin
229         if (wntz_blk_done && (wntz_j_reg < wntz_iter_reg))
230           wntz_fsm_next = WNTZ_OTHERS;
231         wntz_init = 1'b1;
232         wntz_j_inc = 1'b1;
233         end else if (wntz_blk_done)
234           wntz_fsm_next = WNTZ_IDLE;
235         else
236           wntz_fsm_next = WNTZ_1ST;
237       end
238     WNTZ_OTHERS:
239       begin
240         if (wntz_blk_done && (wntz_j_reg < wntz_iter_reg))
241           wntz_fsm_next = WNTZ_OTHERS;
242         wntz_init = 1'b1;
243         wntz_j_inc = 1'b1;
244         end else if (wntz_blk_done)
245           wntz_fsm_next = WNTZ_IDLE;
246         else
247           wntz_fsm_next = WNTZ_OTHERS;
248       end
249     default:
250       wntz_fsm_next = WNTZ_IDLE;
251   endcase
252 end
253
254 always @ (posedge clk or negedge reset_n) begin
255   if (!reset_n)
256     wntz_fsm <= WNTZ_IDLE;
257   else if (zeroize_reg)
258     wntz_fsm <= WNTZ_IDLE;
259   else
260     wntz_fsm <= wntz_fsm_next;
261 end
262
263 always @ (posedge clk or negedge reset_n) begin
264   if (!reset_n) begin
265     wntz_j_reg <= '0;
266     wntz_prefix <= '0;
267     wntz_n_mode_reg <= '0;
268     wntz_w_reg <= '0;
269     wntz_iter_reg <= '0;
270   end else if (zeroize_reg) begin
271     wntz_j_reg <= '0;
272     wntz_prefix <= '0;
273     wntz_n_mode_reg <= '0;
274     wntz_w_reg <= '0;
275     wntz_iter_reg <= '0;
276   end else begin
277     if (wntz_enable) begin
278       wntz_j_reg <= wntz_j_init;
279       wntz_prefix <= {block_reg[00], block_reg[01],
280         block_reg[02], block_reg[03], // I: 16byte, q: 4bytes, i: 2bytes
281         block_reg[04], block_reg[05]
282       [31:16]};
283       wntz_n_mode_reg <= wntz_n_mode;
284       wntz_w_reg <= wntz_w;
285       wntz_iter_reg <= wntz_iter;
286     end else if (wntz_j_inc)
287       wntz_j_reg <= wntz_j_reg + 1;
288   end
289 end

```

```

291 end
292
293 always @ (posedge clk or negedge reset_n) begin
294     if (!reset_n)
295         wntz_init_reg <= 1'b0;
296     else if (zeroize_reg)
297         wntz_init_reg <= 1'b0;
298     else begin
299         if (wntz_init)
300             wntz_init_reg <= 1'b1;
301         else
302             wntz_init_reg <= 1'b0;
303     end
304 end
305
306
307 always @ (posedge clk or negedge reset_n) begin
308     if (!reset_n)
309         mode_reg <= '0;
310     else if (zeroize_reg)
311         mode_reg <= '0;
312     else begin
313         if (ready_reg)
314             mode_reg <= mode;
315     end
316 end
317
318 //-----
319 // reg_update
320 //
321 // Update functionality for all registers in the core.
322 // All registers are positive edge triggered with asynchron-
ous
323 // active low reset. All registers have write enable.
324 //-----
325
326 always @ (posedge clk or negedge reset_n) begin
327     if (!reset_n) begin
328         get_mask <= {8{32'hffff_ffff}};
329     end
330     else if (wntz_busy) begin
331         unique case (wntz_n_mode_reg)
332             0: get_mask <= {{6{32'hffff_ffff}},
333 {2{32'h0000_0000}}};
334             default: get_mask <= {8{32'hffff_ffff}};
335         endcase
336     end
337     else begin
338         unique case (mode_reg)
339             0: get_mask <= {{7{32'hffff_ffff}},
340 {1{32'h0000_0000}}};
341             default: get_mask <= {8{32'hffff_ffff}};
342         endcase
343     end
344 end
345
346 assign ready_flag = core_ready & ~wntz_busy;
347 assign ready_reg = ready_flag & ready_flag_reg;
348 assign valid_flag = core_digest_valid & ~wntz_busy;
349 assign digest_valid_reg = valid_flag & valid_flag_reg;
350
351 always @ (posedge clk or negedge reset_n)
352 begin : reg_update
353     if (!reset_n) begin
354         ready_flag_reg <= '0;
355         digest_reg <= '0;
356         valid_flag_reg <= '0;
357         core_digest_valid_reg <= '0;
358     end
359     else if (zeroize_reg) begin
360         ready_flag_reg <= '0;
361         digest_reg <= '0;
362         valid_flag_reg <= '0;
363         core_digest_valid_reg <= '0;
364     end
365     else begin
366         ready_flag_reg <= ready_flag;
367         valid_flag_reg <= valid_flag;
368         core_digest_valid_reg <= core_digest_valid;
369     end
370 end
371 end // reg_update
372
373
374 //register hw interface
375 always_comb begin
376
377     hwif_in.SHA256_NAME[0].NAME.next = SHA256_CORE_NAME0;
378     hwif_in.SHA256_NAME[1].NAME.next = SHA256_CORE_NAME1;
379
380     hwif_in.SHA256_VERSION[0].VERSION.next =
SHA256_CORE_VERSION0;
381     hwif_in.SHA256_VERSION[1].VERSION.next =
SHA256_CORE_VERSION1;
382
383     init_reg = hwif_out.SHA256_CTRL.INIT.value;
384     next_reg = hwif_out.SHA256_CTRL.NEXT.value;
385
386     mode = hwif_out.SHA256_CTRL.MODE.value;
387     zeroize_reg = hwif_out.SHA256_CTRL.ZEROIZE.value || debu-
gUnlock_or_scan_mode_switch;
388     wntz_mode = hwif_out.SHA256_CTRL.WNTZ_MODE.value;
389     wntz_n_mode = hwif_out.SHA256_CTRL.WNTZ_N_MODE.value;
390     wntz_w = hwif_out.SHA256_CTRL.WNTZ_W.value;
391
392     hwif_in.SHA256_STATUS.READY.next = ready_reg;
393     hwif_in.SHA256_STATUS.VALID.next = digest_valid_reg;
394     hwif_in.SHA256_STATUS.WNTZ_BUSY.next = wntz_busy;
395
396     for (int dword=0; dword < 8; dword++) begin
397         hwif_in.SHA256_DIGEST[dword].DIGEST.next = di-
gest_reg[7-dword];
398         hwif_in.SHA256_DIGEST[dword].DIGEST.next = di-
gest_reg[dword];
399         hwif_in.SHA256_DIGEST[dword].DIGEST.hwclr = zeroize_reg;
400     end
401
402     for (int dword=0; dword < BLOCK_N0; dword++) begin
403         block_reg[dword] =
hwif_out.SHA256_BLOCK[dword].BLOCK.value;
404         hwif_in.SHA256_BLOCK[dword].BLOCK.hwclr = zeroize_reg;
405     end
406 end
407
408 // Register Block
409 sha256_reg i_sha256_reg (
410     .clk(clk),
411     .rst(1'b0),
412     .s_cpuif_req      (cs),
413     .s_cpuif_req_is_wr (we),
414     .s_cpuif_addr     (ad-
dress[SHA256_REG_ADDR_WIDTH-1:0]),
415     .s_cpuif_wr_data   (write_data),
416     .s_cpuif_wr_biten  ('1),
417     .s_cpuif_req_stall_wr( ),
418     .s_cpuif_req_stall_rd( ),
419     .s_cpuif_rd_ack    ( ),
420     .s_cpuif_rd_err    (read_error),
421     .s_cpuif_rd_data    (read_data),
422     .s_cpuif_wr_ack    ( ),
423     .s_cpuif_wr_err    (write_error),
424     .hwif_in (hwif_in ),
425     .hwif_out (hwif_out)
426 );
427
428 //interrupt register hw interface
429 assign hwif_in.sha256_ready = ready_reg;
430 assign hwif_in.reset_b = reset_n;
431 assign hwif_in.error_reset_b = cptra_pwrgood;
432 assign hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset = (~wntz_busy
& core_digest_valid & ~digest_valid_reg);
433 assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.error0_sts.hwset = wntz_w_invalid |
wntz_mode_invalid | wntz_j_invalid;
434 assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.error1_sts.hwset = invalid_sha_op;
435 assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset = 1'b0;
436 assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.error3_sts.hwset = 1'b0;
437
438 assign error_intr = hwif_out.intr_b-
lock_rf.error_global_intr_r.intr;
439 assign notif_intr = hwif_out.intr_b-
lock_rf.notif_global_intr_r.intr;
440 endmodule // sha256
441
=====
442 // E0F sha256.v
443 //
=====

```

2. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_core.v (582 lines)

```

1 //
=====
2 //
3 // sha256_core.v
4 // -----
5 // Verilog 2001 implementation of the SHA-256 hash function.
6 // This is the internal core with wide interfaces.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2013, Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the fol-

```

```

lowing
15 // conditions are met:
16 //
17 // 1. Redistributions of source code must retain the above
copyright
18 // notice, this list of conditions and the following dis-
claimer.
19 //
20 // 2. Redistributions in binary form must reproduce the above
copyright
21 // notice, this list of conditions and the following dis-
claimer in
22 // the documentation and/or other materials provided with
the
23 // distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
TRIBUTORS
26 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
27 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
28 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
29 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
30 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
31 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
32 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
33 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
35 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====
==
39
40 module sha256_core(
41     // Clock and reset.
42     input wire          clk,
43     input wire          reset_n,
44     input wire          zeroize,
45
46     // Control.
47     input wire          init_cmd,
48     input wire          next_cmd,
49     input wire          mode,
50
51     // Data ports.
52     input wire [511 : 0] block_msg,
53
54     output wire          ready,
55     output wire [255 : 0] digest,
56     output wire          digest_valid
57 );
58
59
60 //-----
61 // Internal constant and parameter definitions.
62 //-----
63
64 localparam SHA224_H0_0 = 32'hc1059ed8;
65 localparam SHA224_H0_1 = 32'h367cd507;
66 localparam SHA224_H0_2 = 32'h3070dd17;
67 localparam SHA224_H0_3 = 32'hf70e5939;
68 localparam SHA224_H0_4 = 32'hffc00b31;
69 localparam SHA224_H0_5 = 32'h68581511;
70 localparam SHA224_H0_6 = 32'h64f98fa7;
71 localparam SHA224_H0_7 = 32'hbfa4fa4;
72
73 localparam SHA256_H0_0 = 32'h6a09e667;
74 localparam SHA256_H0_1 = 32'hbb67ae85;
75 localparam SHA256_H0_2 = 32'h3c6ef372;
76 localparam SHA256_H0_3 = 32'ha54ff53a;
77 localparam SHA256_H0_4 = 32'h510e527f;
78 localparam SHA256_H0_5 = 32'h9b05688c;
79 localparam SHA256_H0_6 = 32'h1f83d9ab;
80 localparam SHA256_H0_7 = 32'h5be0cd19;
81
82 localparam SHA256_ROUNDS = 63;
83
84 localparam [1:0] CTRL_IDLE   = 0;
85 localparam [1:0] CTRL_ROUNDS = 1;
86 localparam [1:0] CTRL_DONE   = 2;
87
88 //-----
89 // Registers including update variables and write enable.
90 //-----
91
92 reg [31 : 0] a_reg;
93 reg [31 : 0] a_new;
94 reg [31 : 0] b_reg;
95 reg [31 : 0] b_new;

```

```

95 reg [31 : 0] c_reg;
96 reg [31 : 0] c_new;
97 reg [31 : 0] d_reg;
98 reg [31 : 0] d_new;
99 reg [31 : 0] e_reg;
100 reg [31 : 0] e_new;
101 reg [31 : 0] f_reg;
102 reg [31 : 0] f_new;
103 reg [31 : 0] g_reg;
104 reg [31 : 0] g_new;
105 reg [31 : 0] h_reg;
106 reg [31 : 0] h_new;
107 reg          a_h_we;
108
109 reg [31 : 0] H0_reg;
110 reg [31 : 0] H0_new;
111 reg [31 : 0] H1_reg;
112 reg [31 : 0] H1_new;
113 reg [31 : 0] H2_reg;
114 reg [31 : 0] H2_new;
115 reg [31 : 0] H3_reg;
116 reg [31 : 0] H3_new;
117 reg [31 : 0] H4_reg;
118 reg [31 : 0] H4_new;
119 reg [31 : 0] H5_reg;
120 reg [31 : 0] H5_new;
121 reg [31 : 0] H6_reg;
122 reg [31 : 0] H6_new;
123 reg [31 : 0] H7_reg;
124 reg [31 : 0] H7_new;
125 reg          H_we;
126
127 reg [5 : 0] t_ctr_reg;
128 reg [5 : 0] t_ctr_new;
129 reg          t_ctr_we;
130 reg          t_ctr_inc;
131 reg          t_ctr_rst;
132
133 reg digest_valid_reg;
134 reg digest_valid_new;
135 reg digest_valid_we;
136
137 reg [1 : 0] sha256_ctrl_reg;
138 reg [1 : 0] sha256_ctrl_new;
139 reg          sha256_ctrl_we;
140
141 //-----
142 // Wires.
143 //-----
144
145 reg digest_init;
146 reg digest_update;
147
148 reg state_init;
149 reg state_update;
150
151 reg first_block;
152
153 reg ready_flag;
154
155 reg [31 : 0] t1;
156 reg [31 : 0] t2;
157
158 wire [31 : 0] k_data;
159
160 reg          w_init;
161 reg          w_next;
162 wire [31 : 0] w_data;
163
164 //-----
165 // Module instantiations.
166 //-----
167
168 sha256_k_constants k_constants_inst(
169     .round(t_ctr_reg),
170     .K_val(k_data)
171 );
172
173 sha256_w_mem w_mem_inst(
174     .clk(clk),
175     .reset_n(reset_n),
176     .zeroize(zeroize),
177
178     .block_msg(block_msg),
179
180     .init_cmd(w_init),
181     .next_cmd(w_next),
182     .w_val(w_data)
183 );
184
185 //-----
186 // Concurrent connectivity for ports etc.
187 //-----
188
189 //-----

```

```

190 assign ready = ready_flag;
191
192 assign digest = {H0_reg, H1_reg, H2_reg, H3_reg,
193                 H4_reg, H5_reg, H6_reg, H7_reg};
194
195 assign digest_valid = digest_valid_reg;
196
197
198 //-----
199 // reg_update
200 // Update functionality for all registers in the core.
201 // All registers are positive edge triggered with asynchron-
ous
202 // active low reset. All registers have write enable.
203 //-----
204 always @ (posedge clk or negedge reset_n)
205 begin : reg_update
206   if (!reset_n)
207     begin
208       a_reg <= 32'h0;
209       b_reg <= 32'h0;
210       c_reg <= 32'h0;
211       d_reg <= 32'h0;
212       e_reg <= 32'h0;
213       f_reg <= 32'h0;
214       g_reg <= 32'h0;
215       h_reg <= 32'h0;
216       H0_reg <= 32'h0;
217       H1_reg <= 32'h0;
218       H2_reg <= 32'h0;
219       H3_reg <= 32'h0;
220       H4_reg <= 32'h0;
221       H5_reg <= 32'h0;
222       H6_reg <= 32'h0;
223       H7_reg <= 32'h0;
224       digest_valid_reg <= 0;
225       t_ctr_reg <= 6'h0;
226       sha256_ctrl_reg <= CTRL_IDLE;
227     end
228   else if (zeroize)
229     begin
230       a_reg <= 32'h0;
231       b_reg <= 32'h0;
232       c_reg <= 32'h0;
233       d_reg <= 32'h0;
234       e_reg <= 32'h0;
235       f_reg <= 32'h0;
236       g_reg <= 32'h0;
237       h_reg <= 32'h0;
238       H0_reg <= 32'h0;
239       H1_reg <= 32'h0;
240       H2_reg <= 32'h0;
241       H3_reg <= 32'h0;
242       H4_reg <= 32'h0;
243       H5_reg <= 32'h0;
244       H6_reg <= 32'h0;
245       H7_reg <= 32'h0;
246       digest_valid_reg <= 0;
247       t_ctr_reg <= 6'h0;
248       sha256_ctrl_reg <= CTRL_IDLE;
249     end
250   else
251     begin
252       if (a_h_we)
253         begin
254           a_reg <= a_new;
255           b_reg <= b_new;
256           c_reg <= c_new;
257           d_reg <= d_new;
258           e_reg <= e_new;
259           f_reg <= f_new;
260           g_reg <= g_new;
261           h_reg <= h_new;
262         end
263       if (H_we)
264         begin
265           H0_reg <= H0_new;
266           H1_reg <= H1_new;
267           H2_reg <= H2_new;
268           H3_reg <= H3_new;
269           H4_reg <= H4_new;
270           H5_reg <= H5_new;
271           H6_reg <= H6_new;
272           H7_reg <= H7_new;
273         end
274       if (t_ctr_we)
275         t_ctr_reg <= t_ctr_new;
276       if (digest_valid_we)
277         digest_valid_reg <= digest_valid_new;
278       if (sha256_ctrl_we)
279         sha256_ctrl_reg <= sha256_ctrl_new;
280     end
281 end // reg_update
282
283
284
285
286
287

```

```

288
289 //-----
290 // digest_logic
291 //
292 // The logic needed to init as well as update the digest.
293 //-----
294 always @*
295 begin : digest_logic
296   H0_new = 32'h0;
297   H1_new = 32'h0;
298   H2_new = 32'h0;
299   H3_new = 32'h0;
300   H4_new = 32'h0;
301   H5_new = 32'h0;
302   H6_new = 32'h0;
303   H7_new = 32'h0;
304   H_we = 0;
305
306   if (digest_init)
307     begin
308       H_we = 1;
309       if (mode)
310         begin
311           H0_new = SHA256_H0_0;
312           H1_new = SHA256_H0_1;
313           H2_new = SHA256_H0_2;
314           H3_new = SHA256_H0_3;
315           H4_new = SHA256_H0_4;
316           H5_new = SHA256_H0_5;
317           H6_new = SHA256_H0_6;
318           H7_new = SHA256_H0_7;
319         end
320       else
321         begin
322           H0_new = SHA224_H0_0;
323           H1_new = SHA224_H0_1;
324           H2_new = SHA224_H0_2;
325           H3_new = SHA224_H0_3;
326           H4_new = SHA224_H0_4;
327           H5_new = SHA224_H0_5;
328           H6_new = SHA224_H0_6;
329           H7_new = SHA224_H0_7;
330         end
331       end
332     end
333   if (digest_update)
334     begin
335       H0_new = H0_reg + a_reg;
336       H1_new = H1_reg + b_reg;
337       H2_new = H2_reg + c_reg;
338       H3_new = H3_reg + d_reg;
339       H4_new = H4_reg + e_reg;
340       H5_new = H5_reg + f_reg;
341       H6_new = H6_reg + g_reg;
342       H7_new = H7_reg + h_reg;
343       H_we = 1;
344     end
345   end // digest_logic
346
347 //-----
348
349 // t1_logic
350 //
351 // The logic for the T1 function.
352 //-----
353 always @*
354 begin : t1_logic
355   reg [31 : 0] sum1;
356   reg [31 : 0] ch;
357
358   sum1 = {e_reg[5 : 0], e_reg[31 : 6]} ^
359         {e_reg[10 : 0], e_reg[31 : 11]} ^
360         {e_reg[24 : 0], e_reg[31 : 25]};
361
362   ch = (e_reg & f_reg) ^ ((~e_reg) & g_reg);
363
364   t1 = h_reg + sum1 + ch + w_data + k_data;
365 end // t1_logic
366
367 //-----
368
369 // t2_logic
370 //
371 // The logic for the T2 function
372 //-----
373 always @*
374 begin : t2_logic
375   reg [31 : 0] sum0;
376   reg [31 : 0] maj;
377
378   sum0 = {a_reg[1 : 0], a_reg[31 : 2]} ^
379         {a_reg[12 : 0], a_reg[31 : 13]} ^
380         {a_reg[21 : 0], a_reg[31 : 22]};
381
382   maj = (a_reg & b_reg) ^ (a_reg & c_reg) ^ (b_reg &

```

```

c_reg);
383
384     t2 = sum0 + maj;
385     end // t2_logic
386
387
388 //-----
389 // state_logicerr
390 //
391 // The logic needed to init as well as update the state dur-
ing
392 // round processing.
393 //-----
394 always @*
395 begin : state_logic
396     a_new = 32'h0;
397     b_new = 32'h0;
398     c_new = 32'h0;
399     d_new = 32'h0;
400     e_new = 32'h0;
401     f_new = 32'h0;
402     g_new = 32'h0;
403     h_new = 32'h0;
404     a_h_we = 0;
405
406     if (state_init)
407     begin
408         a_h_we = 1;
409         if (first_block)
410         begin
411             if (mode)
412             begin
413                 a_new = SHA256_H0_0;
414                 b_new = SHA256_H0_1;
415                 c_new = SHA256_H0_2;
416                 d_new = SHA256_H0_3;
417                 e_new = SHA256_H0_4;
418                 f_new = SHA256_H0_5;
419                 g_new = SHA256_H0_6;
420                 h_new = SHA256_H0_7;
421             end
422             else
423             begin
424                 a_new = SHA224_H0_0;
425                 b_new = SHA224_H0_1;
426                 c_new = SHA224_H0_2;
427                 d_new = SHA224_H0_3;
428                 e_new = SHA224_H0_4;
429                 f_new = SHA224_H0_5;
430                 g_new = SHA224_H0_6;
431                 h_new = SHA224_H0_7;
432             end
433         end
434         else
435         begin
436             a_new = H0_reg;
437             b_new = H1_reg;
438             c_new = H2_reg;
439             d_new = H3_reg;
440             e_new = H4_reg;
441             f_new = H5_reg;
442             g_new = H6_reg;
443             h_new = H7_reg;
444         end
445     end
446
447     if (state_update)
448     begin
449         a_new = t1 + t2;
450         b_new = a_reg;
451         c_new = b_reg;
452         d_new = c_reg;
453         e_new = d_reg + t1;
454         f_new = e_reg;
455         g_new = f_reg;
456         h_new = g_reg;
457         a_h_we = 1;
458     end
459 end // state_logic
460
461
462 //-----
463 // t_ctr
464 //
465 // Update logic for the round counter, a monotonically
466 // increasing counter with reset.
467 //-----
468 always @*
469 begin : t_ctr
470     t_ctr_new = 0;
471     t_ctr_we = 0;
472
473     if (t_ctr_rst)
474     begin
475         t_ctr_new = 0;
476         t_ctr_we = 1;
477     end

```

```

478
479     if (t_ctr_inc)
480     begin
481         t_ctr_new = t_ctr_reg + 1'b1;
482         t_ctr_we = 1;
483     end
484 end // t_ctr
485
486 //-----
487
488 // sha256_ctrl_fsm
489 //
490 // Logic for the state machine controlling the core beha-
viour.
491 //-----
492 always @*
493 begin : sha256_ctrl_fsm
494     digest_init = 0;
495     digest_update = 0;
496
497     state_init = 0;
498     state_update = 0;
499
500     first_block = 0;
501     ready_flag = 0;
502
503     w_init = 0;
504     w_next = 0;
505
506     t_ctr_inc = 0;
507     t_ctr_rst = 0;
508
509     digest_valid_new = 0;
510     digest_valid_we = 0;
511
512     sha256_ctrl_new = CTRL_IDLE;
513     sha256_ctrl_we = 0;
514
515
516     case (sha256_ctrl_reg)
517     CTRL_IDLE:
518     begin
519         ready_flag = 1;
520
521         if (init_cmd)
522         begin
523             digest_init = 1;
524             w_init = 1;
525             state_init = 1;
526             first_block = 1;
527             t_ctr_rst = 1;
528             digest_valid_new = 0;
529             digest_valid_we = 1;
530             sha256_ctrl_new = CTRL_ROUNDS;
531             sha256_ctrl_we = 1;
532         end
533
534         if (next_cmd)
535         begin
536             t_ctr_rst = 1;
537             w_init = 1;
538             state_init = 1;
539             digest_valid_new = 0;
540             digest_valid_we = 1;
541             sha256_ctrl_new = CTRL_ROUNDS;
542             sha256_ctrl_we = 1;
543         end
544     end
545
546     CTRL_ROUNDS:
547     begin
548         w_next = 1;
549         state_update = 1;
550         t_ctr_inc = 1;
551
552         if (t_ctr_reg == SHA256_ROUNDS)
553         begin
554             sha256_ctrl_new = CTRL_DONE;
555             sha256_ctrl_we = 1;
556         end
557     end
558
559
560     CTRL_DONE:
561     begin
562         digest_update = 1;
563         digest_valid_new = 1;
564         digest_valid_we = 1;
565
566         sha256_ctrl_new = CTRL_IDLE;
567         sha256_ctrl_we = 1;
568     end
569
570     default:
571     begin
572         sha256_ctrl_new = sha256_ctrl_reg;
573         sha256_ctrl_we = 0;
574     end
575 endcase // case (sha256_ctrl_reg)

```

```

576     end // sha256_ctrl_fsm
577
578 endmodule // sha256_core
579
580 //
=====
==
581 // EOF sha256_core.v
582 //
=====
==

3. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_ctrl.v (123 lines)

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
==
16 //
17 // sha256_ctrl.v
18 // -----
19 // SHA256 controller for the AHB_lite interface.
20 //
21 //
22 //
=====
==
23
24 module sha256_ctrl #(
25     parameter AHB_DATA_WIDTH = 32,
26     parameter AHB_ADDR_WIDTH = 32
27 )
28 (
29     // Clock and reset.
30     input wire      clk,
31     input wire      reset_n,
32     input wire      cptra_pwrgood,
33
34     // from SLAVES PORT
35     input logic [AHB_ADDR_WIDTH-1:0] haddr_i,
36     input logic [AHB_DATA_WIDTH-1:0] hwdata_i,
37     input logic hsel_i,
38     input logic hwrite_i,
39     input logic hready_i,
40     input logic [1:0] htrans_i,
41     input logic [2:0] hsize_i,
42
43     output logic hresp_o,
44     output logic hreadyout_o,
45     output logic [AHB_DATA_WIDTH-1:0] hrdata_o,
46
47     // Interrupt
48     output logic error_intr,
49     output logic notif_intr,
50     input  logic debugUnlock_or_scan_mode_switch
51 );
52
53 //-----
54 // sha256
55 //-----
56
57 logic      sha256_cs;
58 logic      sha256_we;
59 logic [AHB_ADDR_WIDTH-1 : 0] sha256_address;
60 logic [31 : 0] sha256_write_data;
61 logic [31 : 0] sha256_read_data;
62 logic      sha256_err;
63
64 sha256 #(
65     .ADDR_WIDTH(AHB_ADDR_WIDTH),
66     .DATA_WIDTH(32)
67 ) sha256_inst(
68     .clk(clk),
69     .reset_n(reset_n),
70     .cptra_pwrgood(cptra_pwrgood),
71     .cs(sha256_cs),
72     .we(sha256_we),
73     .address(sha256_address),
74     .write_data(sha256_write_data),

```

```

74     .read_data(sha256_read_data),
75     .err(sha256_err),
76     .error_intr(error_intr),
77     .notif_intr(notif_intr),
78     .debugUn-
lock_or_scan_mode_switch(debugUnlock_or_scan_mode_switch)
79 );
80
81
82 //-----
83 // AHB Slave node
84 //-----
85
86 //instantiate ahb lite module
87 ahb_slv_sif #(
88     .AHB_ADDR_WIDTH(AHB_ADDR_WIDTH),
89     .AHB_DATA_WIDTH(AHB_DATA_WIDTH),
90     .CLIENT_DATA_WIDTH(32)
91 ) ahb_slv_sif_uut
92 (
93     //AMBA AHB Lite INF
94     .hclk(clk),
95     .hreset_n(reset_n),
96     .haddr_i(haddr_i),
97     .hwdata_i(hwdata_i),
98     .hsel_i(hsel_i),
99     .hwrite_i(hwrite_i),
100    .hready_i(hready_i),
101    .htrans_i(htrans_i),
102    .hsize_i(hsize_i),
103
104    .hresp_o(hresp_o),
105    .hreadyout_o(hreadyout_o),
106    .hrdata_o(hrdata_o),
107
108    //COMPONENT INF
109    .dv(sha256_cs),
110    .hld(1'b0), //no holds from sha256
111    .err(1'b0),
112    .write(sha256_we),
113    .wdata(sha256_write_data),
114    .addr(sha256_address),
115
116    .rdata(sha256_read_data)
117 );
118
119 endmodule //sha256_ctrl
120
121 //
=====
==
122 // EOF sha256_ctrl.v
123 //
=====
==

4. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_k_constants.v (132 lines)

1 //
=====
==
2 //
3 // sha256_k_constants.v
4 // -----
5 // The table K with constants in the SHA-256 hash function.
6 //
7 //
8 // Author: Joachim Strombergson
9 // Copyright (c) 2013, Secworks Sweden AB
10 // All rights reserved.
11 //
12 // Redistribution and use in source and binary forms, with or
13 // without modification, are permitted provided that the fol-
lowing
14 // conditions are met:
15 //
16 // 1. Redistributions of source code must retain the above
copyright
17 // notice, this list of conditions and the following dis-
claimer.
18 //
19 // 2. Redistributions in binary form must reproduce the above
copyright
20 // notice, this list of conditions and the following dis-
claimer in
21 // the documentation and/or other materials provided with
the
22 // distribution.
23 //
24 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
TRIBUTORS
25 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
26 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
27 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE

```



```

28 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
29 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
30 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
31 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
32 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
33 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
34 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
35 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
36 //
37 //
=====
==
38
39 module sha256_k_constants(
40     input wire [5 : 0] round,
41     output wire [31 : 0] K_val
42 );
43
44 //-----
45 // Wires.
46 //-----
47 reg [31 : 0] tmp_K;
48
49 //-----
50 // Concurrent connectivity for ports etc.
51 //-----
52
53 assign K_val = tmp_K;
54
55 //-----
56
57 // round_mux
58 //-----
59 always @*
60 begin : round_mux
61     case(round)
62         00: tmp_K = 32'h428a2f98;
63         01: tmp_K = 32'h1374491;
64         02: tmp_K = 32'hb5c0fbcf;
65         03: tmp_K = 32'he9b5dba5;
66         04: tmp_K = 32'h3956c25b;
67         05: tmp_K = 32'h59f111f1;
68         06: tmp_K = 32'h923f82a4;
69         07: tmp_K = 32'hab1c5ed5;
70         08: tmp_K = 32'hd807aa98;
71         09: tmp_K = 32'h12835b01;
72         10: tmp_K = 32'h243185be;
73         11: tmp_K = 32'h550c7dc3;
74         12: tmp_K = 32'h72be5d74;
75         13: tmp_K = 32'h80deb1fe;
76         14: tmp_K = 32'h9bdc06a7;
77         15: tmp_K = 32'hc19bf174;
78         16: tmp_K = 32'he49b69c1;
79         17: tmp_K = 32'hef8b4786;
80         18: tmp_K = 32'h0fc19dc6;
81         19: tmp_K = 32'h240ca1cc;
82         20: tmp_K = 32'h2de92c6f;
83         21: tmp_K = 32'h4a7484aa;
84         22: tmp_K = 32'h5cb0a9dc;
85         23: tmp_K = 32'h76f988da;
86         24: tmp_K = 32'h983e5152;
87         25: tmp_K = 32'ha831c66d;
88         26: tmp_K = 32'hb00327c8;
89         27: tmp_K = 32'hbf597fc7;
90         28: tmp_K = 32'hc6e00bf3;
91         29: tmp_K = 32'hd5a79147;
92         30: tmp_K = 32'h06ca6351;
93         31: tmp_K = 32'h14292967;
94         32: tmp_K = 32'h27b70a85;
95         33: tmp_K = 32'h2e1b2138;
96         34: tmp_K = 32'h4d2c6dfc;
97         35: tmp_K = 32'h53380d13;
98         36: tmp_K = 32'h650a7354;
99         37: tmp_K = 32'h766a0abb;
100        38: tmp_K = 32'h81c2c92e;
101        39: tmp_K = 32'h92722c85;
102        40: tmp_K = 32'ha2bfe8a1;
103        41: tmp_K = 32'ha81a664b;
104        42: tmp_K = 32'hc24b8b70;
105        43: tmp_K = 32'hc76c51a3;
106        44: tmp_K = 32'hd192e819;
107        45: tmp_K = 32'hd6990624;
108        46: tmp_K = 32'hf40e3585;
109        47: tmp_K = 32'h106aa070;
110        48: tmp_K = 32'h19a4c116;
111        49: tmp_K = 32'h1e376c08;
112        50: tmp_K = 32'h2748774c;
113        51: tmp_K = 32'h34b0bcb5;
114        52: tmp_K = 32'h391c0cb3;
115        53: tmp_K = 32'h4ed8aa4a;

```

```

116        54: tmp_K = 32'h5b9cca4f;
117        55: tmp_K = 32'h682e6ff3;
118        56: tmp_K = 32'h748f82ee;
119        57: tmp_K = 32'h78a5636f;
120        58: tmp_K = 32'h84c87814;
121        59: tmp_K = 32'h8cc70208;
122        60: tmp_K = 32'h90befffa;
123        61: tmp_K = 32'ha4506ceb;
124        62: tmp_K = 32'hbe99a3f7;
125        63: tmp_K = 32'hc67178f2;
126        endcase // case (round)
127        end // block: round_mux
128    endmodule // sha256_k_constants
129
130 //
=====
==

```

```

131 // sha256_k_constants.v
132 //
=====
==

```

5. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_params_pkg.sv (30 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
16
17 package sha256_params_pkg;
18
19 localparam SHA256_CORE_NAME0    = 32'h61327368; //
"sha2"
20 localparam SHA256_CORE_NAME1    = 32'h35362d32; //
"-256"
21 localparam SHA256_CORE_VERSION0  = 32'h3830312e; //
"1.80"
22 localparam SHA256_CORE_VERSION1  = 32'h00000000; // "0"
23
24 localparam SHA256_MODE_SHA_224   = 1'h0;
25 localparam SHA256_MODE_SHA_256  = 1'h1;
26
27 endpackage
28 //
=====
==
29 // EOF sha256_params_pkg.sv
30 //
=====
==

```

6. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_reg.sv (1,597 lines)

```

1 // Generated by PeakRDL-regblock - A free and open-source Sys-
temVerilog generator
2 // https://github.com/SystemRDL/PeakRDL-regblock
3
4 module sha256_reg (
5     input wire clk,
6     input wire rst,
7
8     input wire s_cpuif_req,
9     input wire s_cpuif_req_is_wr,
10    input wire [11:0] s_cpuif_addr,
11    input wire [31:0] s_cpuif_wr_data,
12    input wire [31:0] s_cpuif_wr_biten,
13    output wire s_cpuif_req_stall_wr,
14    output wire s_cpuif_req_stall_rd,
15    output wire s_cpuif_rd_ack,
16    output wire s_cpuif_rd_err,
17    output wire [31:0] s_cpuif_rd_data,
18    output wire s_cpuif_wr_ack,
19    output wire s_cpuif_wr_err,
20
21    input sha256_reg_pkg::sha256_reg_in_t hwif_in,
22    output sha256_reg_pkg::sha256_reg_out_t hwif_out
23 );
24
25 //-----

```



```

26 // CPU Bus interface logic
27 //-----
28 logic cpuif_req;
29 logic cpuif_req_is_wr;
30 logic [11:0] cpuif_addr;
31 logic [31:0] cpuif_wr_data;
32 logic [31:0] cpuif_wr_biten;
33 logic cpuif_req_stall_wr;
34 logic cpuif_req_stall_rd;
35
36 logic cpuif_rd_ack;
37 logic cpuif_rd_err;
38 logic [31:0] cpuif_rd_data;
39
40 logic cpuif_wr_ack;
41 logic cpuif_wr_err;
42
43 assign cpuif_req = s_cpuif_req;
44 assign cpuif_req_is_wr = s_cpuif_req_is_wr;
45 assign cpuif_addr = s_cpuif_addr;
46 assign cpuif_wr_data = s_cpuif_wr_data;
47 assign cpuif_wr_biten = s_cpuif_wr_biten;
48 assign s_cpuif_req_stall_wr = cpuif_req_stall_wr;
49 assign s_cpuif_req_stall_rd = cpuif_req_stall_rd;
50 assign s_cpuif_rd_ack = cpuif_rd_ack;
51 assign s_cpuif_rd_err = cpuif_rd_err;
52 assign s_cpuif_rd_data = cpuif_rd_data;
53 assign s_cpuif_wr_ack = cpuif_wr_ack;
54 assign s_cpuif_wr_err = cpuif_wr_err;
55
56 logic cpuif_req_masked;
57
58 // Read & write latencies are balanced. Stalls not re-
quired
59 assign cpuif_req_stall_rd = '0;
60 assign cpuif_req_stall_wr = '0;
61 assign cpuif_req_masked = cpuif_req
62 & !(cpuif_req_is_wr &
cpuif_req_stall_rd)
63 & !(cpuif_req_is_wr &
cpuif_req_stall_wr);
64
65 //-----
66 // Address Decode
67 //-----
68 typedef struct packed{
69     logic [2-1:0]SHA256_NAME;
70     logic [2-1:0]SHA256_VERSION;
71     logic SHA256_CTRL;
72     logic SHA256_STATUS;
73     logic [16-1:0]SHA256_BLOCK;
74     logic [8-1:0]SHA256_DIGEST;
75     struct packed{
76         logic global_intr_en_r;
77         logic error_intr_en_r;
78         logic notif_intr_en_r;
79         logic error_global_intr_r;
80         logic notif_global_intr_r;
81         logic error_internal_intr_r;
82         logic notif_internal_intr_r;
83         logic error_intr_trig_r;
84         logic notif_intr_trig_r;
85         logic error0_intr_count_r;
86         logic error1_intr_count_r;
87         logic error2_intr_count_r;
88         logic error3_intr_count_r;
89         logic notif_cmd_done_intr_count_r;
90         logic error0_intr_count_incr_r;
91         logic error1_intr_count_incr_r;
92         logic error2_intr_count_incr_r;
93         logic error3_intr_count_incr_r;
94         logic notif_cmd_done_intr_count_incr_r;
95     } intr_block_rf;
96 } decoded_reg_strb_t;
97 decoded_reg_strb_t decoded_reg_strb;
98 logic decoded_req;
99 logic decoded_req_is_wr;
100 logic [31:0] decoded_wr_data;
101 logic [31:0] decoded_wr_biten;
102
103 always_comb begin
104     for(int i0=0; i0<2; i0++) begin
105         decoded_reg_strb.SHA256_NAME[i0] =
cpuif_req_masked & (cpuif_addr == 12'h0 + i0*12'h4);
106     end
107     for(int i0=0; i0<2; i0++) begin
108         decoded_reg_strb.SHA256_VERSION[i0] =
cpuif_req_masked & (cpuif_addr == 12'h8 + i0*12'h4);
109     end
110     decoded_reg_strb.SHA256_CTRL = cpuif_req_masked &
(cpuif_addr == 12'h10);
111     decoded_reg_strb.SHA256_STATUS = cpuif_req_masked &
(cpuif_addr == 12'h18);
112     for(int i0=0; i0<16; i0++) begin
113         decoded_reg_strb.SHA256_BLOCK[i0] =
cpuif_req_masked & (cpuif_addr == 12'h80 + i0*12'h4);
114     end
115     for(int i0=0; i0<8; i0++) begin

```

```

116         decoded_reg_strb.SHA256_DIGEST[i0] =
cpuif_req_masked & (cpuif_addr == 12'h100 + i0*12'h4);
117     end
118     decoded_reg_strb.intr_block_rf.global_intr_en_r =
cpuif_req_masked & (cpuif_addr == 12'h800);
119     decoded_reg_strb.intr_block_rf.error_intr_en_r =
cpuif_req_masked & (cpuif_addr == 12'h804);
120     decoded_reg_strb.intr_block_rf.notif_intr_en_r =
cpuif_req_masked & (cpuif_addr == 12'h808);
121     decoded_reg_strb.intr_block_rf.error_global_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h80c);
122     decoded_reg_strb.intr_block_rf.notif_global_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h810);
123     decoded_reg_strb.intr_block_rf.error_internal_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h814);
124     decoded_reg_strb.intr_block_rf.notif_internal_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h818);
125     decoded_reg_strb.intr_block_rf.error_intr_trig_r =
cpuif_req_masked & (cpuif_addr == 12'h81c);
126     decoded_reg_strb.intr_block_rf.notif_intr_trig_r =
cpuif_req_masked & (cpuif_addr == 12'h820);
127     decoded_reg_strb.intr_block_rf.error0_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h900);
128     decoded_reg_strb.intr_block_rf.error1_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h904);
129     decoded_reg_strb.intr_block_rf.error2_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h908);
130     decoded_reg_strb.intr_block_rf.error3_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h90c);
131     de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h980);
132     de-
coded_reg_strb.intr_block_rf.error0_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha00);
133     de-
coded_reg_strb.intr_block_rf.error1_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha04);
134     de-
coded_reg_strb.intr_block_rf.error2_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha08);
135     de-
coded_reg_strb.intr_block_rf.error3_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha0c);
136     de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha10);
137     end
138
139 // Pass down signals to next stage
140 assign decoded_req = cpuif_req_masked;
141 assign decoded_req_is_wr = cpuif_req_is_wr;
142 assign decoded_wr_data = cpuif_wr_data;
143 assign decoded_wr_biten = cpuif_wr_biten;
144
145 //-----
146 // Field logic
147 //-----
148 typedef struct packed{
149     struct packed{
150         struct packed{
151             logic next;
152             logic load_next;
153         } INIT;
154         struct packed{
155             logic next;
156             logic load_next;
157         } NEXT;
158         struct packed{
159             logic next;
160             logic load_next;
161         } MODE;
162         struct packed{
163             logic next;
164             logic load_next;
165         } ZEROIZE;
166         struct packed{
167             logic next;
168             logic load_next;
169         } WNTZ_MODE;
170         struct packed{
171             logic [3:0] next;
172             logic load_next;
173         } WNTZ_W;
174         struct packed{
175             logic next;
176             logic load_next;
177         } WNTZ_N_MODE;
178     } SHA256_CTRL;
179     struct packed{
180         struct packed{
181             logic [31:0] next;
182             logic load_next;
183         } BLOCK;
184     } [16-1:0]SHA256_BLOCK;
185     struct packed{
186         struct packed{

```

```

187         logic [31:0] next;
188         logic load_next;
189     } DIGEST;
190 } [8-1:0]SHA256_DIGEST;
191 struct packed{
192     struct packed{
193         struct packed{
194             logic next;
195             logic load_next;
196         } error_en;
197         struct packed{
198             logic next;
199             logic load_next;
200         } notif_en;
201     } global_intr_en_r;
202     struct packed{
203         struct packed{
204             logic next;
205             logic load_next;
206         } error0_en;
207         struct packed{
208             logic next;
209             logic load_next;
210         } error1_en;
211         struct packed{
212             logic next;
213             logic load_next;
214         } error2_en;
215         struct packed{
216             logic next;
217             logic load_next;
218         } error3_en;
219     } error_intr_en_r;
220     struct packed{
221         struct packed{
222             logic next;
223             logic load_next;
224         } notif_cmd_done_en;
225     } notif_intr_en_r;
226     struct packed{
227         struct packed{
228             logic next;
229             logic load_next;
230         } agg_sts;
231     } error_global_intr_r;
232     struct packed{
233         struct packed{
234             logic next;
235             logic load_next;
236         } agg_sts;
237     } notif_global_intr_r;
238     struct packed{
239         struct packed{
240             logic next;
241             logic load_next;
242         } error0_sts;
243         struct packed{
244             logic next;
245             logic load_next;
246         } error1_sts;
247         struct packed{
248             logic next;
249             logic load_next;
250         } error2_sts;
251         struct packed{
252             logic next;
253             logic load_next;
254         } error3_sts;
255     } error_internal_intr_r;
256     struct packed{
257         struct packed{
258             logic next;
259             logic load_next;
260         } notif_cmd_done_sts;
261     } notif_internal_intr_r;
262     struct packed{
263         struct packed{
264             logic next;
265             logic load_next;
266         } error0_trig;
267         struct packed{
268             logic next;
269             logic load_next;
270         } error1_trig;
271         struct packed{
272             logic next;
273             logic load_next;
274         } error2_trig;
275         struct packed{
276             logic next;
277             logic load_next;
278         } error3_trig;
279     } error_intr_trig_r;
280     struct packed{
281         struct packed{
282             logic next;
283             logic load_next;
284         } notif_cmd_done_trig;
285     } notif_intr_trig_r;
286     struct packed{
287         struct packed{

```

```

288         logic [31:0] next;
289         logic load_next;
290         logic incrthreshold;
291         logic incrsaturate;
292     } cnt;
293     } error0_intr_count_r;
294     struct packed{
295         struct packed{
296             logic [31:0] next;
297             logic load_next;
298             logic incrthreshold;
299             logic incrsaturate;
300         } cnt;
301     } error1_intr_count_r;
302     struct packed{
303         struct packed{
304             logic [31:0] next;
305             logic load_next;
306             logic incrthreshold;
307             logic incrsaturate;
308         } cnt;
309     } error2_intr_count_r;
310     struct packed{
311         struct packed{
312             logic [31:0] next;
313             logic load_next;
314             logic incrthreshold;
315             logic incrsaturate;
316         } cnt;
317     } error3_intr_count_r;
318     struct packed{
319         struct packed{
320             logic [31:0] next;
321             logic load_next;
322             logic incrthreshold;
323             logic incrsaturate;
324         } cnt;
325     } notif_cmd_done_intr_count_r;
326     struct packed{
327         struct packed{
328             logic next;
329             logic load_next;
330             logic decrthreshold;
331             logic underflow;
332         } pulse;
333     } error0_intr_count_incr_r;
334     struct packed{
335         struct packed{
336             logic next;
337             logic load_next;
338             logic decrthreshold;
339             logic underflow;
340         } pulse;
341     } error1_intr_count_incr_r;
342     struct packed{
343         struct packed{
344             logic next;
345             logic load_next;
346             logic decrthreshold;
347             logic underflow;
348         } pulse;
349     } error2_intr_count_incr_r;
350     struct packed{
351         struct packed{
352             logic next;
353             logic load_next;
354             logic decrthreshold;
355             logic underflow;
356         } pulse;
357     } error3_intr_count_incr_r;
358     struct packed{
359         struct packed{
360             logic next;
361             logic load_next;
362             logic decrthreshold;
363             logic underflow;
364         } pulse;
365     } notif_cmd_done_intr_count_incr_r;
366     } intr_block_rf;
367     } field_combo_t;
368     field_combo_t field_combo;
369
370 typedef struct packed{
371     struct packed{
372         struct packed{
373             logic value;
374         } INIT;
375         struct packed{
376             logic value;
377         } NEXT;
378         struct packed{
379             logic value;
380         } MODE;
381         struct packed{
382             logic value;
383         } ZEROIZE;
384         struct packed{
385             logic value;
386         } WNTZ_MODE;
387         struct packed{
388             logic [3:0] value;

```

```

389     } WNTZ_W;
390     struct packed{
391         logic value;
392     } WNTZ_N_MODE;
393 } SHA256_CTRL;
394 struct packed{
395     struct packed{
396         logic [31:0] value;
397     } BLOCK;
398 } [16-1:0]SHA256_BLOCK;
399 struct packed{
400     struct packed{
401         logic [31:0] value;
402     } DIGEST;
403 } [8-1:0]SHA256_DIGEST;
404 struct packed{
405     struct packed{
406         struct packed{
407             logic value;
408         } error_en;
409         struct packed{
410             logic value;
411         } notif_en;
412     } global_intr_en_r;
413     struct packed{
414         struct packed{
415             logic value;
416         } error0_en;
417         struct packed{
418             logic value;
419         } error1_en;
420         struct packed{
421             logic value;
422         } error2_en;
423         struct packed{
424             logic value;
425         } error3_en;
426     } error_intr_en_r;
427     struct packed{
428         struct packed{
429             logic value;
430         } notif_cmd_done_en;
431     } notif_intr_en_r;
432     struct packed{
433         struct packed{
434             logic value;
435         } agg_sts;
436     } error_global_intr_r;
437     struct packed{
438         struct packed{
439             logic value;
440         } agg_sts;
441     } notif_global_intr_r;
442     struct packed{
443         struct packed{
444             logic value;
445         } error0_sts;
446         struct packed{
447             logic value;
448         } error1_sts;
449         struct packed{
450             logic value;
451         } error2_sts;
452         struct packed{
453             logic value;
454         } error3_sts;
455     } error_internal_intr_r;
456     struct packed{
457         struct packed{
458             logic value;
459         } notif_cmd_done_sts;
460     } notif_internal_intr_r;
461     struct packed{
462         struct packed{
463             logic value;
464         } error0_trig;
465         struct packed{
466             logic value;
467         } error1_trig;
468         struct packed{
469             logic value;
470         } error2_trig;
471         struct packed{
472             logic value;
473         } error3_trig;
474     } error_intr_trig_r;
475     struct packed{
476         struct packed{
477             logic value;
478         } notif_cmd_done_trig;
479     } notif_intr_trig_r;
480     struct packed{
481         struct packed{
482             logic [31:0] value;
483         } cnt;
484     } error0_intr_count_r;
485     struct packed{
486         struct packed{
487             logic [31:0] value;
488         } cnt;
489     } error1_intr_count_r;

```

```

490     struct packed{
491         struct packed{
492             logic [31:0] value;
493         } cnt;
494     } error2_intr_count_r;
495     struct packed{
496         struct packed{
497             logic [31:0] value;
498         } cnt;
499     } error3_intr_count_r;
500     struct packed{
501         struct packed{
502             logic [31:0] value;
503         } cnt;
504     } notif_cmd_done_intr_count_r;
505     struct packed{
506         struct packed{
507             logic value;
508         } pulse;
509     } error0_intr_count_incr_r;
510     struct packed{
511         struct packed{
512             logic value;
513         } pulse;
514     } error1_intr_count_incr_r;
515     struct packed{
516         struct packed{
517             logic value;
518         } pulse;
519     } error2_intr_count_incr_r;
520     struct packed{
521         struct packed{
522             logic value;
523         } pulse;
524     } error3_intr_count_incr_r;
525     struct packed{
526         struct packed{
527             logic value;
528         } pulse;
529     } notif_cmd_done_intr_count_incr_r;
530     } intr_block_rf;
531 } field_storage_t;
532 field_storage_t field_storage;
533
534 // Field: sha256_reg.SHA256_CTRL.INIT
535 always_comb begin
536     automatic logic [0:0] next_c;
537     automatic logic load_next_c;
538     next_c = field_storage.SHA256_CTRL.INIT.value;
539     load_next_c = '0;
540     if(decoded_reg_strb.SHA256_CTRL && decoded_req_is_wr
&& hwif_in.sha256_ready) begin // SW write
541         next_c = (field_storage.SHA256_CTRL.INIT.value &
~decoded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
542         load_next_c = '1;
543     end else begin // singlepulse clears back to 0
544         next_c = '0;
545         load_next_c = '1;
546     end
547     field_combo.SHA256_CTRL.INIT.next = next_c;
548     field_combo.SHA256_CTRL.INIT.load_next = load_next_c;
549 end
550 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
551     if(~hwif_in.reset_b) begin
552         field_storage.SHA256_CTRL.INIT.value <= 1'h0;
553     end else if(field_combo.SHA256_CTRL.INIT.load_next)
begin
554         field_storage.SHA256_CTRL.INIT.value <=
field_combo.SHA256_CTRL.INIT.next;
555     end
556 end
557 assign hwif_out.SHA256_CTRL.INIT.value =
field_storage.SHA256_CTRL.INIT.value;
558 // Field: sha256_reg.SHA256_CTRL.NEXT
559 always_comb begin
560     automatic logic [0:0] next_c;
561     automatic logic load_next_c;
562     next_c = field_storage.SHA256_CTRL.NEXT.value;
563     load_next_c = '0;
564     if(decoded_reg_strb.SHA256_CTRL && decoded_req_is_wr
&& hwif_in.sha256_ready) begin // SW write
565         next_c = (field_storage.SHA256_CTRL.NEXT.value &
~decoded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
coded_wr_biten[1:1]);
566         load_next_c = '1;
567     end else begin // singlepulse clears back to 0
568         next_c = '0;
569         load_next_c = '1;
570     end
571     field_combo.SHA256_CTRL.NEXT.next = next_c;
572     field_combo.SHA256_CTRL.NEXT.load_next = load_next_c;
573 end
574 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
575     if(~hwif_in.reset_b) begin
576         field_storage.SHA256_CTRL.NEXT.value <= 1'h0;
577     end else if(field_combo.SHA256_CTRL.NEXT.load_next)
begin
578         field_storage.SHA256_CTRL.NEXT.value <=
field_combo.SHA256_CTRL.NEXT.next;
579     end
end

```

```

580     end
581     assign hwif_out.SHA256_CTRL.NEXT.value =
field_storage.SHA256_CTRL.NEXT.value;
582     // Field: sha256_reg.SHA256_CTRL.MODE
583     always_comb begin
584         automatic logic [0:0] next_c;
585         automatic logic load_next_c;
586         next_c = field_storage.SHA256_CTRL.MODE.value;
587         load_next_c = '0;
588         if(decoded_reg_strb.SHA256_CTRL && decoded_req_is_wr
&& hwif_in.sha256_ready) begin // SW write
589             next_c = (field_storage.SHA256_CTRL.MODE.value &
~decoded_wr_biten[2:2]) | (decoded_wr_data[2:2] & de-
coded_wr_biten[2:2]);
590             load_next_c = '1;
591         end
592         field_combo.SHA256_CTRL.MODE.next = next_c;
593         field_combo.SHA256_CTRL.MODE.load_next = load_next_c;
594     end
595     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
596         if(~hwif_in.reset_b) begin
597             field_storage.SHA256_CTRL.MODE.value <= 1'h1;
598         end else if(field_combo.SHA256_CTRL.MODE.load_next)
begin
599             field_storage.SHA256_CTRL.MODE.value <=
field_combo.SHA256_CTRL.MODE.next;
600         end
601     end
602     assign hwif_out.SHA256_CTRL.MODE.value =
field_storage.SHA256_CTRL.MODE.value;
603     // Field: sha256_reg.SHA256_CTRL.ZEROIZE
604     always_comb begin
605         automatic logic [0:0] next_c;
606         automatic logic load_next_c;
607         next_c = field_storage.SHA256_CTRL.ZEROIZE.value;
608         load_next_c = '0;
609         if(decoded_reg_strb.SHA256_CTRL && decoded_req_is_wr)
begin // SW write
610             next_c = (field_storage.SHA256_CTRL.ZEROIZE.value
& ~decoded_wr_biten[3:3]) | (decoded_wr_data[3:3] & de-
coded_wr_biten[3:3]);
611             load_next_c = '1;
612         end else begin // singlepulse clears back to 0
613             next_c = '0;
614             load_next_c = '1;
615         end
616         field_combo.SHA256_CTRL.ZEROIZE.next = next_c;
617         field_combo.SHA256_CTRL.ZEROIZE.load_next =
load_next_c;
618     end
619     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
620         if(~hwif_in.reset_b) begin
621             field_storage.SHA256_CTRL.ZEROIZE.value <= 1'h0;
622         end else if(field_combo.SHA256_CTRL.ZEROIZE.load_next)
begin
623             field_storage.SHA256_CTRL.ZEROIZE.value <=
field_combo.SHA256_CTRL.ZEROIZE.next;
624         end
625     end
626     assign hwif_out.SHA256_CTRL.ZEROIZE.value =
field_storage.SHA256_CTRL.ZEROIZE.value;
627     // Field: sha256_reg.SHA256_CTRL.WNTZ_MODE
628     always_comb begin
629         automatic logic [0:0] next_c;
630         automatic logic load_next_c;
631         next_c = field_storage.SHA256_CTRL.WNTZ_MODE.value;
632         load_next_c = '0;
633         if(decoded_reg_strb.SHA256_CTRL && decoded_req_is_wr
&& hwif_in.sha256_ready) begin // SW write
634             next_c =
(field_storage.SHA256_CTRL.WNTZ_MODE.value & ~decoded_wr_biten[4:4])
| (decoded_wr_data[4:4] & decoded_wr_biten[4:4]);
635             load_next_c = '1;
636         end else begin // singlepulse clears back to 0
637             next_c = '0;
638             load_next_c = '1;
639         end
640         field_combo.SHA256_CTRL.WNTZ_MODE.next = next_c;
641         field_combo.SHA256_CTRL.WNTZ_MODE.load_next =
load_next_c;
642     end
643     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
644         if(~hwif_in.reset_b) begin
645             field_storage.SHA256_CTRL.WNTZ_MODE.value <= 1'h0;
646         end else
if(field_combo.SHA256_CTRL.WNTZ_MODE.load_next) begin
647             field_storage.SHA256_CTRL.WNTZ_MODE.value <=
field_combo.SHA256_CTRL.WNTZ_MODE.next;
648         end
649     end
650     assign hwif_out.SHA256_CTRL.WNTZ_MODE.value =
field_storage.SHA256_CTRL.WNTZ_MODE.value;
651     // Field: sha256_reg.SHA256_CTRL.WNTZ_W
652     always_comb begin
653         automatic logic [3:0] next_c;
654         automatic logic load_next_c;
655         next_c = field_storage.SHA256_CTRL.WNTZ_W.value;
656         load_next_c = '0;
657         if(decoded_reg_strb.SHA256_CTRL && decoded_req_is_wr
&& hwif_in.sha256_ready) begin // SW write

```

```

658             next_c = (field_storage.SHA256_CTRL.WNTZ_W.value &
~decoded_wr_biten[8:5]) | (decoded_wr_data[8:5] & de-
coded_wr_biten[8:5]);
659             load_next_c = '1;
660         end
661         field_combo.SHA256_CTRL.WNTZ_W.next = next_c;
662         field_combo.SHA256_CTRL.WNTZ_W.load_next =
load_next_c;
663     end
664     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
665         if(~hwif_in.reset_b) begin
666             field_storage.SHA256_CTRL.WNTZ_W.value <= 4'h4;
667         end else if(field_combo.SHA256_CTRL.WNTZ_W.load_next)
begin
668             field_storage.SHA256_CTRL.WNTZ_W.value <=
field_combo.SHA256_CTRL.WNTZ_W.next;
669         end
670     end
671     assign hwif_out.SHA256_CTRL.WNTZ_W.value =
field_storage.SHA256_CTRL.WNTZ_W.value;
672     // Field: sha256_reg.SHA256_CTRL.WNTZ_N_MODE
673     always_comb begin
674         automatic logic [0:0] next_c;
675         automatic logic load_next_c;
676         next_c = field_storage.SHA256_CTRL.WNTZ_N_MODE.value;
677         load_next_c = '0;
678         if(decoded_reg_strb.SHA256_CTRL && decoded_req_is_wr
&& hwif_in.sha256_ready) begin // SW write
679             next_c =
(field_storage.SHA256_CTRL.WNTZ_N_MODE.value & ~de-
coded_wr_biten[9:9]) | (decoded_wr_data[9:9] & de-
coded_wr_biten[9:9]);
680             load_next_c = '1;
681         end
682         field_combo.SHA256_CTRL.WNTZ_N_MODE.next = next_c;
683         field_combo.SHA256_CTRL.WNTZ_N_MODE.load_next =
load_next_c;
684     end
685     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
686         if(~hwif_in.reset_b) begin
687             field_storage.SHA256_CTRL.WNTZ_N_MODE.value <=
1'h0;
688         end else
if(field_combo.SHA256_CTRL.WNTZ_N_MODE.load_next) begin
689             field_storage.SHA256_CTRL.WNTZ_N_MODE.value <=
field_combo.SHA256_CTRL.WNTZ_N_MODE.next;
690         end
691     end
692     assign hwif_out.SHA256_CTRL.WNTZ_N_MODE.value =
field_storage.SHA256_CTRL.WNTZ_N_MODE.value;
693     for(genvar i0=0; i0<16; i0++) begin
694         // Field: sha256_reg.SHA256_BLOCK[i].BLOCK
695         always_comb begin
696             automatic logic [31:0] next_c;
697             automatic logic load_next_c;
698             next_c =
field_storage.SHA256_BLOCK[i0].BLOCK.value;
699             load_next_c = '0;
700             if(decoded_reg_strb.SHA256_BLOCK[i0] && de-
coded_req_is_wr) begin // SW write
701                 next_c =
(field_storage.SHA256_BLOCK[i0].BLOCK.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
702                 load_next_c = '1;
703             end else if(hwif_in.SHA256_BLOCK[i0].BLOCK.hwclr)
begin // HW Clear
704                 next_c = '0;
705                 load_next_c = '1;
706             end
707             field_combo.SHA256_BLOCK[i0].BLOCK.next = next_c;
708             field_combo.SHA256_BLOCK[i0].BLOCK.load_next =
load_next_c;
709         end
710         always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
711             if(~hwif_in.reset_b) begin
712                 field_storage.SHA256_BLOCK[i0].BLOCK.value <=
32'h0;
713             end else
if(field_combo.SHA256_BLOCK[i0].BLOCK.load_next) begin
714                 field_storage.SHA256_BLOCK[i0].BLOCK.value <=
field_combo.SHA256_BLOCK[i0].BLOCK.next;
715             end
716         end
717         assign hwif_out.SHA256_BLOCK[i0].BLOCK.value =
field_storage.SHA256_BLOCK[i0].BLOCK.value;
718     end
719     for(genvar i0=0; i0<8; i0++) begin
720         // Field: sha256_reg.SHA256_DIGEST[i].DIGEST
721         always_comb begin
722             automatic logic [31:0] next_c;
723             automatic logic load_next_c;
724             next_c =
field_storage.SHA256_DIGEST[i0].DIGEST.value;
725             load_next_c = '0;
726             if(hwif_in.SHA256_DIGEST[i0].DIGEST.hwclr) be-
gin // HW Clear
727                 next_c = '0;
728                 load_next_c = '1;

```

```

729         end else begin // HW Write
730             next_c =
hwif_in.SHA256_DIGEST[i0].DIGEST.next;
731             load_next_c = '1;
732         end
733         field_combo.SHA256_DIGEST[i0].DIGEST.next =
next_c;
734         field_combo.SHA256_DIGEST[i0].DIGEST.load_next =
load_next_c;
735     end
736     always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
737         if(~hwif_in.reset_b) begin
738             field_storage.SHA256_DIGEST[i0].DIGEST.value
<= 32'h0;
739         end else
if(field_combo.SHA256_DIGEST[i0].DIGEST.load_next) begin
740             field_storage.SHA256_DIGEST[i0].DIGEST.value
<= field_combo.SHA256_DIGEST[i0].DIGEST.next;
741         end
742     end
743 end
744 // Field:
sha256_reg.intr_block_rf.global_intr_en_r.error_en
745     always_comb begin
746         automatic logic [0:0] next_c;
747         automatic logic load_next_c;
748         next_c =
field_storage.intr_block_rf.global_intr_en_r.error_en.value;
749         load_next_c = '0;
750         if(decoded_reg_strb.intr_block_rf.global_intr_en_r &&
decoded_req_is_wr) begin // SW write
751             next_c =
(field_storage.intr_block_rf.global_intr_en_r.error_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
752             load_next_c = '1;
753         end
754         field_combo.intr_block_rf.global_intr_en_r.error_en.next = next_c;
755         field_combo.intr_block_rf.global_intr_en_r.error_en.load_next =
load_next_c;
756     end
757     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
758         if(~hwif_in.reset_b) begin
759             field_storage.intr_block_rf.global_intr_en_r.error_en.value <= 1'h0;
760         end else
if(field_combo.intr_block_rf.global_intr_en_r.error_en.load_next)
begin
761             field_storage.intr_block_rf.global_intr_en_r.error_en.value <=
field_combo.intr_block_rf.global_intr_en_r.error_en.next;
762         end
763     end
764 // Field:
sha256_reg.intr_block_rf.global_intr_en_r.notif_en
765     always_comb begin
766         automatic logic [0:0] next_c;
767         automatic logic load_next_c;
768         next_c =
field_storage.intr_block_rf.global_intr_en_r.notif_en.value;
769         load_next_c = '0;
770         if(decoded_reg_strb.intr_block_rf.global_intr_en_r &&
decoded_req_is_wr) begin // SW write
771             next_c =
(field_storage.intr_block_rf.global_intr_en_r.notif_en.value & ~de-
coded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
coded_wr_biten[1:1]);
772             load_next_c = '1;
773         end
774         field_combo.intr_block_rf.global_intr_en_r.notif_en.next = next_c;
775         field_combo.intr_block_rf.global_intr_en_r.notif_en.load_next =
load_next_c;
776     end
777     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
778         if(~hwif_in.reset_b) begin
779             field_storage.intr_block_rf.global_intr_en_r.notif_en.value <= 1'h0;
780         end else
if(field_combo.intr_block_rf.global_intr_en_r.notif_en.load_next)
begin
781             field_storage.intr_block_rf.global_intr_en_r.notif_en.value <=
field_combo.intr_block_rf.global_intr_en_r.notif_en.next;
782         end
783     end
784 // Field:
sha256_reg.intr_block_rf.error_intr_en_r.error0_en
785     always_comb begin
786         automatic logic [0:0] next_c;
787         automatic logic load_next_c;
788         next_c =
field_storage.intr_block_rf.error_intr_en_r.error0_en.value;
789         load_next_c = '0;
790         if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
791             next_c =

```

```

(field_storage.intr_block_rf.error_intr_en_r.error0_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
792             load_next_c = '1;
793         end
794         field_combo.intr_block_rf.error_intr_en_r.error0_en.next = next_c;
795         field_combo.intr_block_rf.error_intr_en_r.error0_en.load_next =
load_next_c;
796     end
797     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
798         if(~hwif_in.reset_b) begin
799             field_storage.intr_block_rf.error_intr_en_r.error0_en.value <= 1'h0;
800         end else
if(field_combo.intr_block_rf.error_intr_en_r.error0_en.load_next)
begin
801             field_storage.intr_block_rf.error_intr_en_r.error0_en.value <=
field_combo.intr_block_rf.error_intr_en_r.error0_en.next;
802         end
803     end
804 // Field:
sha256_reg.intr_block_rf.error_intr_en_r.error1_en
805     always_comb begin
806         automatic logic [0:0] next_c;
807         automatic logic load_next_c;
808         next_c =
field_storage.intr_block_rf.error_intr_en_r.error1_en.value;
809         load_next_c = '0;
810         if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
811             next_c =
(field_storage.intr_block_rf.error_intr_en_r.error1_en.value & ~de-
coded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
coded_wr_biten[1:1]);
812             load_next_c = '1;
813         end
814         field_combo.intr_block_rf.error_intr_en_r.error1_en.next = next_c;
815         field_combo.intr_block_rf.error_intr_en_r.error1_en.load_next =
load_next_c;
816     end
817     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
818         if(~hwif_in.reset_b) begin
819             field_storage.intr_block_rf.error_intr_en_r.error1_en.value <= 1'h0;
820         end else
if(field_combo.intr_block_rf.error_intr_en_r.error1_en.load_next)
begin
821             field_storage.intr_block_rf.error_intr_en_r.error1_en.value <=
field_combo.intr_block_rf.error_intr_en_r.error1_en.next;
822         end
823     end
824 // Field:
sha256_reg.intr_block_rf.error_intr_en_r.error2_en
825     always_comb begin
826         automatic logic [0:0] next_c;
827         automatic logic load_next_c;
828         next_c =
field_storage.intr_block_rf.error_intr_en_r.error2_en.value;
829         load_next_c = '0;
830         if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
831             next_c =
(field_storage.intr_block_rf.error_intr_en_r.error2_en.value & ~de-
coded_wr_biten[2:2]) | (decoded_wr_data[2:2] & de-
coded_wr_biten[2:2]);
832             load_next_c = '1;
833         end
834         field_combo.intr_block_rf.error_intr_en_r.error2_en.next = next_c;
835         field_combo.intr_block_rf.error_intr_en_r.error2_en.load_next =
load_next_c;
836     end
837     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
838         if(~hwif_in.reset_b) begin
839             field_storage.intr_block_rf.error_intr_en_r.error2_en.value <= 1'h0;
840         end else
if(field_combo.intr_block_rf.error_intr_en_r.error2_en.load_next)
begin
841             field_storage.intr_block_rf.error_intr_en_r.error2_en.value <=
field_combo.intr_block_rf.error_intr_en_r.error2_en.next;
842         end
843     end
844 // Field:
sha256_reg.intr_block_rf.error_intr_en_r.error3_en
845     always_comb begin
846         automatic logic [0:0] next_c;
847         automatic logic load_next_c;
848         next_c =
field_storage.intr_block_rf.error_intr_en_r.error3_en.value;
849         load_next_c = '0;
850         if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write

```



```

967         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value &
~(decoded_wr_data[1:1] & decoded_wr_biten[1:1]);
968         load_next_c = '1';
969     end
970
field_combo.intr_block_rf.error_internal_intr_r.error1_sts.next =
next_c;
971
field_combo.intr_block_rf.error_internal_intr_r.error1_sts.load_next
= load_next_c;
972     end
973     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
974         if(~hwif_in.error_reset_b) begin
975
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
<= '1'h0;
976         end else
if(field_combo.intr_block_rf.error_internal_intr_r.error1_sts.load_
next) begin
977
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error1_sts.next;
978         end
979     end
980     // Field:
sha256_reg.intr_block_rf.error_internal_intr_r.error2_sts
981     always_comb begin
982         automatic logic [0:0] next_c;
983         automatic logic load_next_c;
984         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value;
985         load_next_c = '0';
986
if(field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value !
= '0') begin // stickybit
987             next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
988             load_next_c = '1';
989         end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset) begin // HW Set
990             next_c = '1';
991             load_next_c = '1';
992         end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
993             next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value &
~(decoded_wr_data[2:2] & decoded_wr_biten[2:2]);
994             load_next_c = '1';
995         end
996
field_combo.intr_block_rf.error_internal_intr_r.error2_sts.next =
next_c;
997
field_combo.intr_block_rf.error_internal_intr_r.error2_sts.load_next
= load_next_c;
998     end
999     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1000         if(~hwif_in.error_reset_b) begin
1001
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
<= '1'h0;
1002         end else
if(field_combo.intr_block_rf.error_internal_intr_r.error2_sts.load_
next) begin
1003
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error2_sts.next;
1004         end
1005     end
1006     // Field:
sha256_reg.intr_block_rf.error_internal_intr_r.error3_sts
1007     always_comb begin
1008         automatic logic [0:0] next_c;
1009         automatic logic load_next_c;
1010         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value;
1011         load_next_c = '0';
1012
if(field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value !
= '0') begin // stickybit
1013             next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1014             load_next_c = '1';
1015         end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error3_sts.hwset) begin // HW Set
1016             next_c = '1';
1017             load_next_c = '1';
1018         end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1019             next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value &
~(decoded_wr_data[3:3] & decoded_wr_biten[3:3]);
1020             load_next_c = '1';
1021         end

```

```

1022
field_combo.intr_block_rf.error_internal_intr_r.error3_sts.next =
next_c;
1023
field_combo.intr_block_rf.error_internal_intr_r.error3_sts.load_next
= load_next_c;
1024     end
1025     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1026         if(~hwif_in.error_reset_b) begin
1027
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
<= '1'h0;
1028         end else
if(field_combo.intr_block_rf.error_internal_intr_r.error3_sts.load_
next) begin
1029
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error3_sts.next;
1030         end
1031     end
1032     assign hwif_out.intr_block_rf.error_internal_intr_r.intr =
1033     |
(field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error0_en.value)
1034     ||
(field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error1_en.value)
1035     ||
(field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error2_en.value)
1036     ||
(field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error3_en.value);
1037     // Field:
sha256_reg.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
1038     always_comb begin
1039         automatic logic [0:0] next_c;
1040         automatic logic load_next_c;
1041         next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value;
1042         load_next_c = '0';
1043
if(field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
.value != '0') begin // stickybit
1044             next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value |
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
1045             load_next_c = '1';
1046         end else if(hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset) begin // HW
Set
1047             next_c = '1';
1048             load_next_c = '1';
1049         end else if(de-
coded_reg_strb.intr_block_rf.notif_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1050             next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value & ~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1051             load_next_c = '1';
1052         end
1053
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.
next = next_c;
1054
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.
load_next = load_next_c;
1055     end
1056     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1057         if(~hwif_in.reset_b) begin
1058
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value <= '1'h0;
1059         end else
if(field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_s
ts.load_next) begin
1060
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value <=
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.
next;
1061         end
1062     end
1063     assign hwif_out.intr_block_rf.notif_internal_intr_r.intr =
1064     |
(field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_s
ts.value &
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value)
;
1065     // Field:
sha256_reg.intr_block_rf.error_intr_trig_r.error0_trig
1066     always_comb begin
1067         automatic logic [0:0] next_c;
1068         automatic logic load_next_c;
1069         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1070         load_next_c = '0';

```



```

1071         if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1072             next_c =
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value |
(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1073             load_next_c = '1;
1074         end else begin // singlepulse clears back to 0
1075             next_c = '0;
1076             load_next_c = '1;
1077         end
1078     field_combo.intr_block_rf.error_intr_trig_r.error0_trig.next =
next_c;
1079     field_combo.intr_block_rf.error_intr_trig_r.error0_trig.load_next =
load_next_c;
1080     end
1081     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1082         if(~hwif_in.reset_b) begin
1083             field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value <=
1'h0;
1084         end else
1085             if(field_combo.intr_block_rf.error_intr_trig_r.error0_trig.load_next
) begin
1086                 field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error0_trig.next;
1087             end
1088             // Field:
sha256_reg.intr_block_rf.error_intr_trig_r.error1_trig
1089             always_comb begin
1090                 automatic logic [0:0] next_c;
1091                 automatic logic load_next_c;
1092                 next_c =
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value;
1093                 load_next_c = '0;
1094             if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1095                 next_c =
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value |
(decoded_wr_data[1:1] & decoded_wr_biten[1:1]);
1096                 load_next_c = '1;
1097             end else begin // singlepulse clears back to 0
1098                 next_c = '0;
1099                 load_next_c = '1;
1100             end
1101         end
1102     field_combo.intr_block_rf.error_intr_trig_r.error1_trig.next =
next_c;
1103     field_combo.intr_block_rf.error_intr_trig_r.error1_trig.load_next =
load_next_c;
1104     end
1105     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1106         if(~hwif_in.reset_b) begin
1107             field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value <=
1'h0;
1108         end else
1109             if(field_combo.intr_block_rf.error_intr_trig_r.error1_trig.load_next
) begin
1110                 field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error1_trig.next;
1111             end
1112             // Field:
sha256_reg.intr_block_rf.error_intr_trig_r.error2_trig
1113             always_comb begin
1114                 automatic logic [0:0] next_c;
1115                 automatic logic load_next_c;
1116                 next_c =
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1117                 load_next_c = '0;
1118             if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1119                 next_c =
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value |
(decoded_wr_data[2:2] & decoded_wr_biten[2:2]);
1120                 load_next_c = '1;
1121             end else begin // singlepulse clears back to 0
1122                 next_c = '0;
1123                 load_next_c = '1;
1124             end
1125         end
1126     field_combo.intr_block_rf.error_intr_trig_r.error2_trig.next =
next_c;
1127     field_combo.intr_block_rf.error_intr_trig_r.error2_trig.load_next =
load_next_c;
1128     end
1129     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1130         if(~hwif_in.reset_b) begin
1131             field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value <=
1'h0;
1132         end else
1133             if(field_combo.intr_block_rf.error_intr_trig_r.error2_trig.load_next
) begin
1134                 field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error2_trig.next;
1135             end
1136             // Field:
sha256_reg.intr_block_rf.error_intr_trig_r.error3_trig
1137             always_comb begin
1138                 automatic logic [0:0] next_c;
1139                 automatic logic load_next_c;
1140                 next_c =
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1141                 load_next_c = '0;
1142             if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1143                 next_c =
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value |
(decoded_wr_data[3:3] & decoded_wr_biten[3:3]);
1144                 load_next_c = '1;
1145             end else begin // singlepulse clears back to 0
1146                 next_c = '0;
1147                 load_next_c = '1;
1148             end
1149         end
1150     field_combo.intr_block_rf.error_intr_trig_r.error3_trig.next =
next_c;
1151     field_combo.intr_block_rf.error_intr_trig_r.error3_trig.load_next =
load_next_c;
1152     end
1153     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1154         if(~hwif_in.reset_b) begin
1155             field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value <=
1'h0;
1156         end else
1157             if(field_combo.intr_block_rf.error_intr_trig_r.error3_trig.load_next
) begin
1158                 field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error3_trig.next;
1159             end
1160             // Field:
sha256_reg.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
1161             always_comb begin
1162                 automatic logic [0:0] next_c;
1163                 automatic logic load_next_c;
1164                 next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
1165                 load_next_c = '0;
1166             if(decoded_reg_strb.intr_block_rf.notif_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1167                 next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue | (decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1168                 load_next_c = '1;
1169             end else begin // singlepulse clears back to 0
1170                 next_c = '0;
1171                 load_next_c = '1;
1172             end
1173         end
1174     field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.next =
next_c;
1175     field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.loa
d_next = load_next_c;
1176     end
1177     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1178         if(~hwif_in.reset_b) begin
1179             field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue <= 1'h0;
1180         end else
1181             if(field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.
load_next) begin
1182                 field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue <=
field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.next
;
1183             end
1184             // Field: sha256_reg.intr_block_rf.error0_intr_count_r.cnt
1185             always_comb begin
1186                 automatic logic [31:0] next_c;
1187                 automatic logic load_next_c;
1188                 next_c =
field_storage.intr_block_rf.error0_intr_count_r.cnt.value;
1189                 load_next_c = '0;
1190             if(decoded_reg_strb.intr_block_rf.error0_intr_count_r
&& decoded_req_is_wr) begin // SW write
1191                 next_c =
(field_storage.intr_block_rf.error0_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-

```

```

coded_wr_biten[31:0]);
1188         load_next_c = '1';
1189     end
1190
if(field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value)
begin // increment
1191     if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
begin // up-counter saturated
1192         next_c = 32'hffffffff;
1193     end else begin
1194         next_c = next_c + 32'h1;
1195     end
1196     load_next_c = '1';
1197 end
1198
field_combo.intr_block_rf.error0_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error0_intr_count_r.cnt.value >=
32'hffffffff);
1199
field_combo.intr_block_rf.error0_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error0_intr_count_r.cnt.value >=
32'hffffffff);
1200     if(next_c > 32'hffffffff) begin
1201         next_c = 32'hffffffff;
1202         load_next_c = '1';
1203     end
1204
1205     field_combo.intr_block_rf.error0_intr_count_r.cnt.next
= next_c;
1206
field_combo.intr_block_rf.error0_intr_count_r.cnt.load_next =
load_next_c;
1207     end
1208     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1209         if(~hwif_in.error_reset_b) begin
1210             field_storage.intr_block_rf.error0_intr_count_r.cnt.value <= 32'h0;
1211         end else
1212             if(field_combo.intr_block_rf.error0_intr_count_r.cnt.load_next) be-
gin
1213                 field_storage.intr_block_rf.error0_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error0_intr_count_r.cnt.next;
1214             end
1215             // Field: sha256_reg.intr_block_rf.error1_intr_count_r.cnt
always_comb begin
1216                 automatic logic [31:0] next_c;
1217                 automatic logic load_next_c;
1218                 next_c =
field_storage.intr_block_rf.error1_intr_count_r.cnt.value;
1219                 load_next_c = '0';
1220                 if(decoded_reg_strb.intr_block_rf.error1_intr_count_r
&& decoded_req_is_wr) begin // SW write
1221                     next_c =
(field_storage.intr_block_rf.error1_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1222                     load_next_c = '1';
1223                 end
1224
if(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value)
begin // increment
1225     if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
begin // up-counter saturated
1226         next_c = 32'hffffffff;
1227     end else begin
1228         next_c = next_c + 32'h1;
1229     end
1230     load_next_c = '1';
1231 end
1232
field_combo.intr_block_rf.error1_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=
32'hffffffff);
1233
field_combo.intr_block_rf.error1_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=
32'hffffffff);
1234     if(next_c > 32'hffffffff) begin
1235         next_c = 32'hffffffff;
1236         load_next_c = '1';
1237     end
1238
1239     field_combo.intr_block_rf.error1_intr_count_r.cnt.next
= next_c;
1240
field_combo.intr_block_rf.error1_intr_count_r.cnt.load_next =
load_next_c;
1241     end
1242     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1243         if(~hwif_in.error_reset_b) begin
1244             field_storage.intr_block_rf.error1_intr_count_r.cnt.value <= 32'h0;
1245         end else
1246             if(field_combo.intr_block_rf.error1_intr_count_r.cnt.load_next) be-
gin
1247                 field_storage.intr_block_rf.error1_intr_count_r.cnt.value <=

```

```

1242 field_combo.intr_block_rf.error1_intr_count_r.cnt.next;
1243     end
1244 end
1245 // Field: sha256_reg.intr_block_rf.error2_intr_count_r.cnt
1246 always_comb begin
1247     automatic logic [31:0] next_c;
1248     automatic logic load_next_c;
1249     next_c =
1250 field_storage.intr_block_rf.error2_intr_count_r.cnt.value;
1251     load_next_c = '0;
1252     if(decoded_reg_strb.intr_block_rf.error2_intr_count_r
1253 && decoded_req_is_wr) begin // SW write
1254         next_c =
1255 (field_storage.intr_block_rf.error2_intr_count_r.cnt.value & ~de-
1256 coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
1257 coded_wr_biten[31:0]);
1258         load_next_c = '1;
1259     end
1260 end
1261 if(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value)
1262 begin // increment
1263     if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
1264 gin // up-counter saturated
1265         next_c = 32'hffffffff;
1266     end else begin
1267         next_c = next_c + 32'h1;
1268     end
1269     load_next_c = '1;
1270 end
1271 end
1272 field_combo.intr_block_rf.error2_intr_count_r.cnt.incrthreshold =
1273 (field_storage.intr_block_rf.error2_intr_count_r.cnt.value >=
1274 32'hffffffff);
1275
1276 field_combo.intr_block_rf.error2_intr_count_r.cnt.incrsaturate =
1277 (field_storage.intr_block_rf.error2_intr_count_r.cnt.value >=
1278 32'hffffffff);
1279
1280 if(next_c > 32'hffffffff) begin
1281     next_c = 32'hffffffff;
1282     load_next_c = '1;
1283 end
1284 end
1285
1286 field_combo.intr_block_rf.error2_intr_count_r.cnt.next
1287 = next_c;
1288
1289 field_combo.intr_block_rf.error2_intr_count_r.cnt.load_next =
1290 load_next_c;
1291 end
1292 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
1293 begin
1294     if(~hwif_in.error_reset_b) begin
1295         field_storage.intr_block_rf.error2_intr_count_r.cnt.value <= 32'h0;
1296         end else
1297 if(field_combo.intr_block_rf.error2_intr_count_r.cnt.load_next) be-
1298 gin
1299     field_storage.intr_block_rf.error2_intr_count_r.cnt.value <=
1300 field_combo.intr_block_rf.error2_intr_count_r.cnt.next;
1301     end
1302 end
1303 // Field: sha256_reg.intr_block_rf.error3_intr_count_r.cnt
1304 always_comb begin
1305     automatic logic [31:0] next_c;
1306     automatic logic load_next_c;
1307     next_c =
1308 field_storage.intr_block_rf.error3_intr_count_r.cnt.value;
1309     load_next_c = '0;
1310     if(decoded_reg_strb.intr_block_rf.error3_intr_count_r
1311 && decoded_req_is_wr) begin // SW write
1312         next_c =
1313 (field_storage.intr_block_rf.error3_intr_count_r.cnt.value & ~de-
1314 coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
1315 coded_wr_biten[31:0]);
1316         load_next_c = '1;
1317     end
1318 end
1319 if(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value)
1320 begin // increment
1321     if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
1322 gin // up-counter saturated
1323         next_c = 32'hffffffff;
1324     end else begin
1325         next_c = next_c + 32'h1;
1326     end
1327     load_next_c = '1;
1328 end
1329 end
1330 field_combo.intr_block_rf.error3_intr_count_r.cnt.incrthreshold =
1331 (field_storage.intr_block_rf.error3_intr_count_r.cnt.value >=
1332 32'hffffffff);
1333
1334 field_combo.intr_block_rf.error3_intr_count_r.cnt.incrsaturate =
1335 (field_storage.intr_block_rf.error3_intr_count_r.cnt.value >=
1336 32'hffffffff);
1337
1338 if(next_c > 32'hffffffff) begin
1339     next_c = 32'hffffffff;
1340     load_next_c = '1;
1341 end
1342 end
1343
1344 field_combo.intr_block_rf.error3_intr_count_r.cnt.next

```

```

= next_c;
1307
field_combo.intr_block_rf.error3_intr_count_r.cnt.load_next =
load_next_c;
1308 end
1309 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1310     if(~hwif_in.error_reset_b) begin
1311
field_storage.intr_block_rf.error3_intr_count_r.cnt.value <= 32'h0;
1312     end else
if(field_combo.intr_block_rf.error3_intr_count_r.cnt.load_next) be-
gin
1313
field_storage.intr_block_rf.error3_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error3_intr_count_r.cnt.next;
1314     end
1315 end
1316 // Field:
sha256_reg.intr_block_rf.notif_cmd_done_intr_count_r.cnt
1317 always_comb begin
1318     automatic logic [31:0] next_c;
1319     automatic logic load_next_c;
1320     next_c =
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value;
1321     load_next_c = '0;
1322     if(de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r && de-
coded_reg_is_wr) begin // SW write
1323         next_c =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value &
~decoded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1324         load_next_c = '1;
1325     end
1326
if(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pul-
se.value) begin // increment
1327         if(((33)'(next_c) + 32'h1) > 32'hfffffff) be-
gin // up-counter saturated
1328             next_c = 32'hfffffff;
1329         end else begin
1330             next_c = next_c + 32'h1;
1331         end
1332         load_next_c = '1;
1333     end
1334
field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.incrthres
hold =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value
>= 32'hfffffff);
1335
field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.incrsatur
ate =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value
>= 32'hfffffff);
1336     if(next_c > 32'hfffffff) begin
1337         next_c = 32'hfffffff;
1338         load_next_c = '1;
1339     end
1340
field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.next =
next_c;
1341
field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.load_next
= load_next_c;
1342 end
1343 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1344     if(~hwif_in.reset_b) begin
1345
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value <=
32'h0;
1346     end else
if(field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.load_n
ext) begin
1347
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value <=
field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.next;
1348     end
1349 end
1350 // Field:
sha256_reg.intr_block_rf.error0_intr_count_incr_r.pulse
1351 always_comb begin
1352     automatic logic [0:0] next_c;
1353     automatic logic load_next_c;
1354     next_c =
field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value;
1355     load_next_c = '0;
1356
if(field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value)
begin // HW Write - we
1357     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1358     load_next_c = '1;
1359     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error0_sts.hwset) begin // HW Set
1360         next_c = '1;
1361         load_next_c = '1;
1362     end
1363
if(field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value)

```

```

begin // decrement
1364
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
1365     next_c = next_c - 1'h1;
1366     load_next_c = '1;
1367     end else begin
1368
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.underflow =
'0;
1369     end
1370
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value <=
1'd0);
1371
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.next =
next_c;
1372
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.load_next =
load_next_c;
1373 end
1374 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1375     if(~hwif_in.reset_b) begin
1376
field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value <=
1'h0;
1377     end else
if(field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.load_ne
xt) begin
1378
field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.next;
1379     end
1380 end
1381 // Field:
sha256_reg.intr_block_rf.error1_intr_count_incr_r.pulse
1382 always_comb begin
1383     automatic logic [0:0] next_c;
1384     automatic logic load_next_c;
1385     next_c =
field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value;
1386     load_next_c = '0;
1387
if(field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value)
begin // HW Write - we
1388     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value;
1389     load_next_c = '1;
1390     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error1_sts.hwset) begin // HW Set
1391         next_c = '1;
1392         load_next_c = '1;
1393     end
1394
if(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value)
begin // decrement
1395
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
1396     next_c = next_c - 1'h1;
1397     load_next_c = '1;
1398     end else begin
1399
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.underflow =
'0;
1400     end
1401
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
1'd0);
1402
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.next =
next_c;
1403
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.load_next =
load_next_c;
1404 end
1405 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1406     if(~hwif_in.reset_b) begin
1407
field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
1'h0;
1408     end else
if(field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.load_ne
xt) begin
1409
field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.next;
1410     end
1411 end
1412 // Field:
sha256_reg.intr_block_rf.error2_intr_count_incr_r.pulse
1413 always_comb begin
1414     automatic logic [0:0] next_c;
1415     automatic logic load_next_c;
1416     next_c =
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value;
1417     load_next_c = '0;

```

```

1418
if(field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value)
begin // HW Write - we
1419     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1420     load_next_c = '1';
1421     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset) begin // HW Set
1422         next_c = '1';
1423         load_next_c = '1';
1424     end
1425
if(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value)
begin // decrement
1426
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
1427     next_c = next_c - 1'h1;
1428     load_next_c = '1';
1429     end else begin
1430
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.underflow =
'0';
1431     end
1432
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
1'd0);
1433
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next =
next_c;
1434
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_next =
load_next_c;
1435     end
1436     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1437         if(~hwif_in.reset_b) begin
1438
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
1'h0;
1439         end else
if(field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_ne
xt) begin
1440
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next;
1441         end
1442     end
1443     // Field:
sha256_reg.intr_block_rf.error3_intr_count_incr_r.pulse
1444     always_comb begin
1445         automatic logic [0:0] next_c;
1446         automatic logic load_next_c;
1447         next_c =
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value;
1448         load_next_c = '0';
1449
if(field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value)
begin // HW Write - we
1450     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1451     load_next_c = '1';
1452     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error3_sts.hwset) begin // HW Set
1453         next_c = '1';
1454         load_next_c = '1';
1455     end
1456
if(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value)
begin // decrement
1457
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
1458     next_c = next_c - 1'h1;
1459     load_next_c = '1';
1460     end else begin
1461
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.underflow =
'0';
1462     end
1463
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
1'd0);
1464
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.next =
next_c;
1465
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.load_next =
load_next_c;
1466     end
1467     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1468         if(~hwif_in.reset_b) begin
1469
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
1'h0;
1470         end else
if(field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.load_ne
xt) begin

```

```

1471
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.next;
1472     end
1473     // Field:
sha256_reg.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse
1475     always_comb begin
1476         automatic logic [0:0] next_c;
1477         automatic logic load_next_c;
1478         next_c =
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value;
1479         load_next_c = '0';
1480
if(field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
.value) begin // HW Write - we
1481     next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
1482     load_next_c = '1';
1483     end else if(hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset) begin // HW
Set
1484         next_c = '1';
1485         load_next_c = '1';
1486     end
1487
if(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pul
se.value) begin // decrement
1488
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.un
derflow = (next_c < (1'h1));
1489     next_c = next_c - 1'h1;
1490     load_next_c = '1';
1491     end else begin
1492
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.un
derflow = '0';
1493     end
1494
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.de
crthreshold =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <= 1'd0);
1495
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.ne
xt = next_c;
1496
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.lo
ad_next = load_next_c;
1497     end
1498     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1499         if(~hwif_in.reset_b) begin
1500
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <= 1'h0;
1501         end else
if(field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
load_next) begin
1502
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <=
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.ne
xt;
1503     end
1504     end
1505     //-----
1506     // Write response
1507     //-----
1508     //-----
1509     assign cpuif_wr_ack = decoded_req & decoded_req_is_wr;
1510     // Writes are always granted with no error response
1511     assign cpuif_wr_err = '0';
1512
1513     //-----
1514     // Readback
1515     //-----
1516
1517     logic readback_err;
1518     logic readback_done;
1519     logic [31:0] readback_data;
1520
1521     // Assign readback values to a flattened array
1522     logic [32:1:0][31:0] readback_array;
1523     for(genvar i0=0; i0<2; i0++) begin
1524         assign readback_array[i0*1 + 0][31:0] = (de-
coded_reg_strb.SHA256_NAME[i0] && !decoded_req_is_wr) ?
hwif_in.SHA256_NAME[i0].NAME.next : '0;
1525     end
1526     for(genvar i0=0; i0<2; i0++) begin
1527         assign readback_array[i0*1 + 2][31:0] = (de-
coded_reg_strb.SHA256_VERSION[i0] && !decoded_req_is_wr) ?
hwif_in.SHA256_VERSION[i0].VERSION.next : '0;
1528     end
1529     assign readback_array[4][0:0] = (de-
coded_reg_strb.SHA256_STATUS && !decoded_req_is_wr) ?

```

```

hwif_in.SHA256_STATUS_READY.next : '0;
1530 assign readback_array[4][1:1] = (de-
coded_reg_strb.SHA256_STATUS && !decoded_req_is_wr) ?
hwif_in.SHA256_STATUS_VALID.next : '0;
1531 assign readback_array[4][2:2] = (de-
coded_reg_strb.SHA256_STATUS && !decoded_req_is_wr) ?
hwif_in.SHA256_STATUS_WNTZ_BUSY.next : '0;
1532 assign readback_array[4][3:3] = '0;
1533 for(genvar i0=0; i0<8; i0++) begin
1534 assign readback_array[i0*1 + 5][31:0] = (de-
coded_reg_strb.SHA256_DIGEST[i0] && !decoded_req_is_wr) ?
field_storage.SHA256_DIGEST[i0].DIGEST.value : '0;
1535 end
1536 assign readback_array[13][0:0] = (de-
coded_reg_strb.intr_block_rf.global_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.global_intr_en_r.error_en.value : '0;
1537 assign readback_array[13][1:1] = (de-
coded_reg_strb.intr_block_rf.global_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.global_intr_en_r.notif_en.value : '0;
1538 assign readback_array[13][31:2] = '0;
1539 assign readback_array[14][0:0] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error0_en.value : '0;
1540 assign readback_array[14][1:1] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error1_en.value : '0;
1541 assign readback_array[14][2:2] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error2_en.value : '0;
1542 assign readback_array[14][3:3] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error3_en.value : '0;
1543 assign readback_array[14][31:4] = '0;
1544 assign readback_array[15][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value : '0;
1545 assign readback_array[15][31:1] = '0;
1546 assign readback_array[16][0:0] = (de-
coded_reg_strb.intr_block_rf.error_global_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_global_intr_r.agg_sts.value : '0;
1547 assign readback_array[16][31:1] = '0;
1548 assign readback_array[17][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_global_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value : '0;
1549 assign readback_array[17][31:1] = '0;
1550 assign readback_array[18][0:0] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value : '0;
1551 assign readback_array[18][1:1] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value : '0;
1552 assign readback_array[18][2:2] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value : '0;
1553 assign readback_array[18][3:3] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value : '0;
1554 assign readback_array[18][31:4] = '0;
1555 assign readback_array[19][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value : '0;
1556 assign readback_array[19][31:1] = '0;
1557 assign readback_array[20][0:0] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value : '0;
1558 assign readback_array[20][1:1] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value : '0;
1559 assign readback_array[20][2:2] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value : '0;
1560 assign readback_array[20][3:3] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value :

```

```

'0;
1561 assign readback_array[20][31:4] = '0;
1562 assign readback_array[21][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue : '0;
1563 assign readback_array[21][31:1] = '0;
1564 assign readback_array[22][31:0] = (de-
coded_reg_strb.intr_block_rf.error0_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error0_intr_count_r.cnt.value : '0;
1565 assign readback_array[23][31:0] = (de-
coded_reg_strb.intr_block_rf.error1_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error1_intr_count_r.cnt.value : '0;
1566 assign readback_array[24][31:0] = (de-
coded_reg_strb.intr_block_rf.error2_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error2_intr_count_r.cnt.value : '0;
1567 assign readback_array[25][31:0] = (de-
coded_reg_strb.intr_block_rf.error3_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error3_intr_count_r.cnt.value : '0;
1568 assign readback_array[26][31:0] = (de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value : '0;
1569 assign readback_array[27][0:0] = (de-
coded_reg_strb.intr_block_rf.error0_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value : '0;
1570 assign readback_array[27][31:1] = '0;
1571 assign readback_array[28][0:0] = (de-
coded_reg_strb.intr_block_rf.error1_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value : '0;
1572 assign readback_array[28][31:1] = '0;
1573 assign readback_array[29][0:0] = (de-
coded_reg_strb.intr_block_rf.error2_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value : '0;
1574 assign readback_array[29][31:1] = '0;
1575 assign readback_array[30][0:0] = (de-
coded_reg_strb.intr_block_rf.error3_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value : '0;
1576 assign readback_array[30][31:1] = '0;
1577 assign readback_array[31][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_incr_r && !
decoded_req_is_wr) ?
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse
.value : '0;
1578 assign readback_array[31][31:1] = '0;
1579
1580 // Reduce the array
1581 always_comb begin
1582 automatic logic [31:0] readback_data_var;
1583 readback_done = decoded_req & ~decoded_req_is_wr;
1584 readback_err = '0;
1585 readback_data_var = '0;
1586 for(int i=0; i<32; i++) readback_data_var |= read-
back_array[i];
1587 readback_data = readback_data_var;
1588 end
1589
1590 assign cpuif_rd_ack = readback_done;
1591 assign cpuif_rd_data = readback_data;
1592 assign cpuif_rd_err = readback_err;
1593
1594 `include "caliptra_prim_assert.sv"
1595 `CALIPTRA_ASSERT_KNOWN(ERR_HWIF_IN, hwif_in, clk, !hwif_in.er-
ror_reset_b)
1596
1597 endmodule

```

7. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_reg_pkg.sv (185 lines)

```

1 // Generated by PeakRDL-regblock - A free and open-source Sys-
temVerilog generator
2 // https://github.com/SystemRDL/PeakRDL-regblock
3
4 package sha256_reg_pkg;
5
6 localparam SHA256_REG_DATA_WIDTH = 32;
7 localparam SHA256_REG_MIN_ADDR_WIDTH = 12;
8
9 typedef struct packed{
10     logic [31:0] next;
11 } sha256_reg_SHA256_NAME_NAME_in_t;
12
13 typedef struct packed{
14     sha256_reg_SHA256_NAME_NAME_in_t NAME;
15 } sha256_reg_SHA256_NAME_in_t;

```



```

sha256_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_notif_cmd_done_sts_enable_dabe0b8b_next_540fa3b7_in_t_notif_cmd_done_sts;
89     }
sha256_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_in_t;
90
91     typedef struct packed{
92
sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_err
ror2_sts_b1cf2205_error3_sts_74a35378_in_t_error_internal_intr_r;
93
sha256_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_in_t no-
tif_internal_intr_r;
94     } sha256_reg_intr_block_t_in_t;
95
96     typedef struct packed{
97         logic reset_b;
98         logic error_reset_b;
99         logic sha256_ready;
100        sha256_reg_SHA256_NAME_in_t [2-1:0]SHA256_NAME;
101        sha256_reg_SHA256_VERSION_in_t
[2-1:0]SHA256_VERSION;
102        sha256_reg_SHA256_STATUS_in_t SHA256_STATUS;
103        sha256_reg_SHA256_BLOCK_in_t [16-1:0]SHA256_BLOCK;
104        sha256_reg_SHA256_DIGEST_in_t [8-1:0]SHA256_DIGEST;
105        sha256_reg_intr_block_t_in_t intr_block_rf;
106    } sha256_reg_in_t;
107
108     typedef struct packed{
109         logic value;
110     } sha256_reg_SHA256_CTRL_INIT_out_t;
111
112     typedef struct packed{
113         logic value;
114     } sha256_reg_SHA256_CTRL_NEXT_out_t;
115
116     typedef struct packed{
117         logic value;
118     } sha256_reg_SHA256_CTRL_MODE_out_t;
119
120     typedef struct packed{
121         logic value;
122     } sha256_reg_SHA256_CTRL_ZEROIZE_out_t;
123
124     typedef struct packed{
125         logic value;
126     } sha256_reg_SHA256_CTRL_WNTZ_MODE_out_t;
127
128     typedef struct packed{
129         logic [3:0] value;
130     } sha256_reg_SHA256_CTRL_WNTZ_W_out_t;
131
132     typedef struct packed{
133         logic value;
134     } sha256_reg_SHA256_CTRL_WNTZ_N_MODE_out_t;
135
136     typedef struct packed{
137         sha256_reg_SHA256_CTRL_INIT_out_t INIT;
138         sha256_reg_SHA256_CTRL_NEXT_out_t NEXT;
139         sha256_reg_SHA256_CTRL_MODE_out_t MODE;
140         sha256_reg_SHA256_CTRL_ZEROIZE_out_t ZEROIZE;
141         sha256_reg_SHA256_CTRL_WNTZ_MODE_out_t WNTZ_MODE;
142         sha256_reg_SHA256_CTRL_WNTZ_W_out_t WNTZ_W;
143         sha256_reg_SHA256_CTRL_WNTZ_N_MODE_out_t
WNTZ_N_MODE;
144     } sha256_reg_SHA256_CTRL_out_t;
145
146     typedef struct packed{
147         logic [31:0] value;
148     } sha256_reg_SHA256_BLOCK_BLOCK_out_t;
149
150     typedef struct packed{
151         sha256_reg_SHA256_BLOCK_BLOCK_out_t BLOCK;
152     } sha256_reg_SHA256_BLOCK_out_t;
153
154     typedef struct packed{
155         logic intr;
156     } sha256_reg_global_intr_t_agg_sts_dd3dcf0a_out_t;
157
158     typedef struct packed{
159         logic intr;
160     } sha256_reg_global_intr_t_agg_sts_e6399b4a_out_t;
161
162     typedef struct packed{
163         logic intr;
164     }
sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_err
ror2_sts_b1cf2205_error3_sts_74a35378_out_t;
165
166     typedef struct packed{
167         logic intr;
168     }
sha256_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_out_t;
169
170     typedef struct packed{
171         sha256_reg_global_intr_t_agg_sts_dd3dcf0a_out_t er-
ror_global_intr_r;
172         sha256_reg_global_intr_t_agg_sts_e6399b4a_out_t no-
tif_global_intr_r;
173
sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_err
ror2_sts_b1cf2205_error3_sts_74a35378_out_t_error_internal_intr_r;

```

```

174 sha256_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_out_t no-
tif_internal_intr_r;
175 } sha256_reg_intr_block_t_out_t;
176
177 typedef struct packed{
178     sha256_reg_SHA256_CTRL_out_t SHA256_CTRL;
179     sha256_reg_SHA256_BLOCK_out_t [16-1:0]SHA256_BLOCK;
180     sha256_reg_intr_block_t_out_t intr_block_rf;
181 } sha256_reg_out_t;
182
183 localparam SHA256_REG_ADDR_WIDTH = 32'd12;
184
185 endpackage

```

8. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_reg_uvm.sv (1,035 lines)

```

1 // This file was autogenerated by PeakRDL-uvm
2 package sha256_reg_uvm;
3 `include "uvm_macros.svh"
4 import uvm_pkg::*;
5 `include "sha256_reg_covergroups.svh"
6 // Reg - sha256_reg::SHA256_NAME
7 class sha256_reg_SHA256_NAME extends uvm_reg;
8     protected uvm_reg_data_t m_current;
9     protected uvm_reg_data_t m_data;
10    protected bit m_is_read;
11
12    sha256_reg_SHA256_NAME_bit_cg NAME_bit_cg[32];
13    sha256_reg_SHA256_NAME_fld_cg fld_cg;
14    rand uvm_reg_field NAME;
15
16    function new(string name = "sha256_reg_SHA256_NAME");
17        super.new(name, 32, build_coverage(UVM_CVR_ALL));
18    endfunction : new
19    extern virtual function void sample_values();
20    extern protected virtual function void
sample(uvm_reg_data_t data,
21
22    uvm_reg_data_t byte_en,
23    bit is_read,
24    uvm_reg_map map);
25
26    virtual function void build();
27        this.NAME = new("NAME");
28        this.NAME.configure(this, 32, 0, "R0", 1, 'h0, 0,
1, 0);
29        if (has_coverage(UVM_CVR_REG_BITS)) begin
30            foreach(NAME_bit_cg[bt]) NAME_bit_cg[bt] =
new();
31        end
32        if (has_coverage(UVM_CVR_FIELD_VALS))
33            fld_cg = new();
34        endfunction : build
35    endclass : sha256_reg_SHA256_NAME
36
37    // Reg - sha256_reg::SHA256_VERSION
38    class sha256_reg_SHA256_VERSION extends uvm_reg;
39        protected uvm_reg_data_t m_current;
40        protected uvm_reg_data_t m_data;
41        protected bit m_is_read;
42
43        sha256_reg_SHA256_VERSION_bit_cg VERSION_bit_cg[32];
44        sha256_reg_SHA256_VERSION_fld_cg fld_cg;
45        rand uvm_reg_field VERSION;
46
47        function new(string name =
"sha256_reg_SHA256_VERSION");
48            super.new(name, 32, build_coverage(UVM_CVR_ALL));
49        endfunction : new
50        extern virtual function void sample_values();
51        extern protected virtual function void
sample(uvm_reg_data_t data,
52
53    uvm_reg_data_t byte_en,
54    bit is_read,
55    uvm_reg_map map);
56
57    virtual function void build();
58        this.VERSION = new("VERSION");
59        this.VERSION.configure(this, 32, 0, "R0", 1, 'h0,
0, 1, 0);
60        if (has_coverage(UVM_CVR_REG_BITS)) begin
61            foreach(VERSION_bit_cg[bt]) VERSION_bit_cg[bt] =
new();
62        end
63        if (has_coverage(UVM_CVR_FIELD_VALS))
64            fld_cg = new();
65        endfunction : build
66    endclass : sha256_reg_SHA256_VERSION
67
68    // Reg - sha256_reg::SHA256_CTRL
69    class sha256_reg_SHA256_CTRL extends uvm_reg;

```

```

69    protected uvm_reg_data_t m_current;
70    protected uvm_reg_data_t m_data;
71    protected bit m_is_read;
72
73    sha256_reg_SHA256_CTRL_bit_cg INIT_bit_cg[1];
74    sha256_reg_SHA256_CTRL_bit_cg NEXT_bit_cg[1];
75    sha256_reg_SHA256_CTRL_bit_cg MODE_bit_cg[1];
76    sha256_reg_SHA256_CTRL_bit_cg ZEROIZE_bit_cg[1];
77    sha256_reg_SHA256_CTRL_bit_cg WNTZ_MODE_bit_cg[1];
78    sha256_reg_SHA256_CTRL_bit_cg WNTZ_W_bit_cg[4];
79    sha256_reg_SHA256_CTRL_bit_cg WNTZ_N_MODE_bit_cg[1];
80    sha256_reg_SHA256_CTRL_fld_cg fld_cg;
81    rand uvm_reg_field INIT;
82    rand uvm_reg_field NEXT;
83    rand uvm_reg_field MODE;
84    rand uvm_reg_field ZEROIZE;
85    rand uvm_reg_field WNTZ_MODE;
86    rand uvm_reg_field WNTZ_W;
87    rand uvm_reg_field WNTZ_N_MODE;
88
89    function new(string name = "sha256_reg_SHA256_CTRL");
90        super.new(name, 32, build_coverage(UVM_CVR_ALL));
91    endfunction : new
92    extern virtual function void sample_values();
93    extern protected virtual function void
sample(uvm_reg_data_t data,
94
95    uvm_reg_data_t byte_en,
96    bit is_read,
97    uvm_reg_map map);
98
99    virtual function void build();
100        this.INIT = new("INIT");
101        this.INIT.configure(this, 1, 0, "W0", 0, 'h0, 1,
1, 0);
102        this.NEXT = new("NEXT");
103        this.NEXT.configure(this, 1, 1, "W0", 0, 'h0, 1,
1, 0);
104        this.MODE = new("MODE");
105        this.MODE.configure(this, 1, 2, "W0", 0, 'h1, 1,
1, 0);
106        this.ZEROIZE = new("ZEROIZE");
107        this.ZEROIZE.configure(this, 1, 3, "W0", 0, 'h0,
1, 1, 0);
108        this.WNTZ_MODE = new("WNTZ_MODE");
109        this.WNTZ_MODE.configure(this, 1, 4, "W0", 0, 'h0,
1, 1, 0);
110        this.WNTZ_W = new("WNTZ_W");
111        this.WNTZ_W.configure(this, 4, 5, "W0", 0, 'h4, 1,
1, 0);
112        this.WNTZ_N_MODE = new("WNTZ_N_MODE");
113        this.WNTZ_N_MODE.configure(this, 1, 9, "W0", 0,
'h0, 1, 1, 0);
114        if (has_coverage(UVM_CVR_REG_BITS)) begin
115            foreach(INIT_bit_cg[bt]) INIT_bit_cg[bt] =
new();
116            foreach(NEXT_bit_cg[bt]) NEXT_bit_cg[bt] =
new();
117            foreach(MODE_bit_cg[bt]) MODE_bit_cg[bt] =
new();
118            foreach(ZEROIZE_bit_cg[bt]) ZEROIZE_bit_cg[bt] =
new();
119            foreach(WNTZ_MODE_bit_cg[bt]) WNTZ_MODE_bit_cg[bt] =
new();
120            foreach(WNTZ_W_bit_cg[bt]) WNTZ_W_bit_cg[bt] =
new();
121            foreach(WNTZ_N_MODE_bit_cg[bt]) WNTZ_N_MODE_bit_cg[bt] =
new();
122            if (has_coverage(UVM_CVR_FIELD_VALS))
123                fld_cg = new();
124            endfunction : build
125        endclass : sha256_reg_SHA256_CTRL
126
127    // Reg - sha256_reg::SHA256_STATUS
128    class sha256_reg_SHA256_STATUS extends uvm_reg;
129        protected uvm_reg_data_t m_current;
130        protected uvm_reg_data_t m_data;
131        protected bit m_is_read;
132
133        sha256_reg_SHA256_STATUS_bit_cg READY_bit_cg[1];
134        sha256_reg_SHA256_STATUS_bit_cg VALID_bit_cg[1];
135        sha256_reg_SHA256_STATUS_bit_cg WNTZ_BUSY_bit_cg[1];
136        sha256_reg_SHA256_STATUS_fld_cg fld_cg;
137        rand uvm_reg_field READY;
138        rand uvm_reg_field VALID;
139        rand uvm_reg_field WNTZ_BUSY;
140
141        function new(string name =
"sha256_reg_SHA256_STATUS");
142            super.new(name, 32, build_coverage(UVM_CVR_ALL));
143        endfunction : new
144        extern virtual function void sample_values();
145        extern protected virtual function void
sample(uvm_reg_data_t data,

```



```

146 uvm_reg_data_t byte_en,
147 bit is_read,
148 uvm_reg_map map);
149
150 virtual function void build();
151 this.READY = new("READY");
152 this.READY.configure(this, 1, 0, "R0", 1, 'h0, 1,
1, 0);
153 this.VALID = new("VALID");
154 this.VALID.configure(this, 1, 1, "R0", 1, 'h0, 1,
1, 0);
155 this.WNTZ_BUSY = new("WNTZ_BUSY");
156 this.WNTZ_BUSY.configure(this, 1, 2, "R0", 1, 'h0,
1, 1, 0);
157 if (has_coverage(UVM_CVR_REG_BITS)) begin
158   foreach(READY_bit_cg[bt]) READY_bit_cg[bt] =
new();
159   foreach(VALID_bit_cg[bt]) VALID_bit_cg[bt] =
new();
160   foreach(WNTZ_BUSY_bit_cg[bt])
WNTZ_BUSY_bit_cg[bt] = new();
161 end
162 if (has_coverage(UVM_CVR_FIELD_VALS))
163   fld_cg = new();
164 endfunction : build
165 endclass : sha256_reg_SHA256_STATUS
166
167 // Reg - sha256_reg::SHA256_BLOCK
168 class sha256_reg_SHA256_BLOCK extends uvm_reg;
169 protected uvm_reg_data_t m_current;
170 protected uvm_reg_data_t m_data;
171 protected bit m_is_read;
172
173 sha256_reg_SHA256_BLOCK_bit_cg BLOCK_bit_cg[32];
174 sha256_reg_SHA256_BLOCK_fld_cg fld_cg;
175 rand uvm_reg_field BLOCK;
176
177 function new(string name =
"sha256_reg_SHA256_BLOCK");
178   super.new(name, 32, build_coverage(UVM_CVR_ALL));
179 endfunction : new
180 extern virtual function void sample_values();
181 extern protected virtual function void
sample(uvm_reg_data_t data,
182 uvm_reg_data_t byte_en,
183 bit is_read,
184 uvm_reg_map map);
185
186 virtual function void build();
187 this.BLOCK = new("BLOCK");
188
189 this.BLOCK.configure(this, 32, 0, "W0", 1, 'h0, 1,
1, 0);
190 if (has_coverage(UVM_CVR_REG_BITS)) begin
191   foreach(BLOCK_bit_cg[bt]) BLOCK_bit_cg[bt] =
new();
192 end
193 if (has_coverage(UVM_CVR_FIELD_VALS))
194   fld_cg = new();
195 endfunction : build
196 endclass : sha256_reg_SHA256_BLOCK
197
198 // Reg - sha256_reg::SHA256_DIGEST
199 class sha256_reg_SHA256_DIGEST extends uvm_reg;
200 protected uvm_reg_data_t m_current;
201 protected uvm_reg_data_t m_data;
202 protected bit m_is_read;
203
204 sha256_reg_SHA256_DIGEST_bit_cg DIGEST_bit_cg[32];
205 sha256_reg_SHA256_DIGEST_fld_cg fld_cg;
206 rand uvm_reg_field DIGEST;
207
208 function new(string name =
"sha256_reg_SHA256_DIGEST");
209   super.new(name, 32, build_coverage(UVM_CVR_ALL));
210 endfunction : new
211 extern virtual function void sample_values();
212 extern protected virtual function void
sample(uvm_reg_data_t data,
213 uvm_reg_data_t byte_en,
214 bit is_read,
215 uvm_reg_map map);
216
217 virtual function void build();
218 this.DIGEST = new("DIGEST");
219 this.DIGEST.configure(this, 32, 0, "R0", 1, 'h0,
1, 1, 0);
220 if (has_coverage(UVM_CVR_REG_BITS)) begin
221   foreach(DIGEST_bit_cg[bt]) DIGEST_bit_cg[bt] =
new();

```

```

221 end
222 if (has_coverage(UVM_CVR_FIELD_VALS))
223   fld_cg = new();
224 endfunction : build
225 endclass : sha256_reg_SHA256_DIGEST
226
227 // Reg - sha256_reg::global_intr_en_t
228 class sha256_reg_global_intr_en_t extends uvm_reg;
229 protected uvm_reg_data_t m_current;
230 protected uvm_reg_data_t m_data;
231 protected bit m_is_read;
232
233 sha256_reg_global_intr_en_t_bit_cg error_en_bit_cg[1];
234 sha256_reg_global_intr_en_t_bit_cg no_tif_en_bit_cg[1];
235 sha256_reg_global_intr_en_t_fld_cg fld_cg;
236 rand uvm_reg_field error_en;
237 rand uvm_reg_field notif_en;
238
239 function new(string name =
"sha256_reg_global_intr_en_t");
240   super.new(name, 32, build_coverage(UVM_CVR_ALL));
241 endfunction : new
242 extern virtual function void sample_values();
243 extern protected virtual function void
sample(uvm_reg_data_t data,
244 uvm_reg_data_t byte_en,
245 bit is_read,
246 uvm_reg_map map);
247
248 virtual function void build();
249 this.error_en = new("error_en");
250 this.error_en.configure(this, 1, 0, "RW", 0, 'h0,
1, 1, 0);
251 this.notif_en = new("notif_en");
252 this.notif_en.configure(this, 1, 1, "RW", 0, 'h0,
1, 1, 0);
253 if (has_coverage(UVM_CVR_REG_BITS)) begin
254   foreach(error_en_bit_cg[bt]) error_en_bit_cg[bt] =
new();
255   foreach(notif_en_bit_cg[bt]) no_tif_en_bit_cg[bt] =
new();
256 end
257 if (has_coverage(UVM_CVR_FIELD_VALS))
258   fld_cg = new();
259 endfunction : build
260 endclass : sha256_reg_global_intr_en_t
261
262 // Reg - sha256_reg::error_intr_en_t
263 class sha256_reg_error_intr_en_t extends uvm_reg;
264 protected uvm_reg_data_t m_current;
265 protected uvm_reg_data_t m_data;
266 protected bit m_is_read;
267
268 sha256_reg_error_intr_en_t_bit_cg error0_en_bit_cg[1];
269 sha256_reg_error_intr_en_t_bit_cg error1_en_bit_cg[1];
270 sha256_reg_error_intr_en_t_bit_cg error2_en_bit_cg[1];
271 sha256_reg_error_intr_en_t_bit_cg error3_en_bit_cg[1];
272 sha256_reg_error_intr_en_t_fld_cg fld_cg;
273 rand uvm_reg_field error0_en;
274 rand uvm_reg_field error1_en;
275 rand uvm_reg_field error2_en;
276 rand uvm_reg_field error3_en;
277
278 function new(string name =
"sha256_reg_error_intr_en_t");
279   super.new(name, 32, build_coverage(UVM_CVR_ALL));
280 endfunction : new
281 extern virtual function void sample_values();
282 extern protected virtual function void
sample(uvm_reg_data_t data,
283 uvm_reg_data_t byte_en,
284 bit is_read,
285 uvm_reg_map map);
286
287 virtual function void build();
288 this.error0_en = new("error0_en");
289 this.error0_en.configure(this, 1, 0, "RW", 0, 'h0,
1, 1, 0);
290 this.error1_en = new("error1_en");
291 this.error1_en.configure(this, 1, 1, "RW", 0, 'h0,
1, 1, 0);
292 this.error2_en = new("error2_en");
293 this.error2_en.configure(this, 1, 2, "RW", 0, 'h0,
1, 1, 0);
294 this.error3_en = new("error3_en");

```

```

295         this.error3_en.configure(this, 1, 3, "RW", 0, 'h0',
1, 1, 0);
296         if (has_coverage(UVM_CVR_REG_BITS)) begin
297             foreach(error0_en_bit_cg[bt]) er-
ror0_en_bit_cg[bt] = new();
298             foreach(error1_en_bit_cg[bt]) er-
ror1_en_bit_cg[bt] = new();
299             foreach(error2_en_bit_cg[bt]) er-
ror2_en_bit_cg[bt] = new();
300             foreach(error3_en_bit_cg[bt]) er-
ror3_en_bit_cg[bt] = new();
301         end
302         if (has_coverage(UVM_CVR_FIELD_VALS))
303             fld_cg = new();
304         endfunction : build
305     endclass : sha256_reg_error_intr_en_t
306
307     // Reg - sha256_reg::notif_intr_en_t
308     class sha256_reg_notif_intr_en_t extends uvm_reg;
309         protected uvm_reg_data_t m_current;
310         protected uvm_reg_data_t m_data;
311         protected bit m_is_read;
312
313         sha256_reg_notif_intr_en_t_bit_cg no-
tif_cmd_done_en_bit_cg[1];
314         sha256_reg_notif_intr_en_t_fld_cg fld_cg;
315         rand uvm_reg_field notif_cmd_done_en;
316
317         function new(string name =
"sha256_reg_notif_intr_en_t");
318             super.new(name, 32, build_coverage(UVM_CVR_ALL));
319         endfunction : new
320         extern virtual function void sample_values();
321         extern protected virtual function void
sample(uvm_reg_data_t data,
322
uvm_reg_data_t byte_en,
323
bit is_read,
324
uvm_reg_map map);
325
326         virtual function void build();
327             this.notif_cmd_done_en = new("notif_cmd_done_en");
328
329         this.notif_cmd_done_en.configure(this, 1, 0, "RW",
0, 'h0, 1, 1, 0);
330         if (has_coverage(UVM_CVR_REG_BITS)) begin
331             foreach(notif_cmd_done_en_bit_cg[bt]) no-
tif_cmd_done_en_bit_cg[bt] = new();
332         end
333         if (has_coverage(UVM_CVR_FIELD_VALS))
334             fld_cg = new();
335         endfunction : build
336     endclass : sha256_reg_notif_intr_en_t
337
338     // Reg - sha256_reg::global_intr_t_agg_sts_dd3dcf0a
339     class sha256_reg_global_intr_t_agg_sts_dd3dcf0a extends
uvm_reg;
340         protected uvm_reg_data_t m_current;
341         protected uvm_reg_data_t m_data;
342         protected bit m_is_read;
343
344         sha256_reg_global_intr_t_agg_sts_dd3dcf0a_bit_cg
agg_sts_bit_cg[1];
345         sha256_reg_global_intr_t_agg_sts_dd3dcf0a_fld_cg
fld_cg;
346         rand uvm_reg_field agg_sts;
347
348         function new(string name =
"sha256_reg_global_intr_t_agg_sts_dd3dcf0a");
349             super.new(name, 32, build_coverage(UVM_CVR_ALL));
350         endfunction : new
351         extern virtual function void sample_values();
352         extern protected virtual function void
sample(uvm_reg_data_t data,
353
uvm_reg_data_t byte_en,
354
bit is_read,
355
uvm_reg_map map);
356
357         virtual function void build();
358             this.agg_sts = new("agg_sts");
359             this.agg_sts.configure(this, 1, 0, "R0", 1, 'h0,
1, 1, 0);
360         if (has_coverage(UVM_CVR_REG_BITS)) begin
361             foreach(agg_sts_bit_cg[bt]) agg_sts_bit_cg[bt] =
new();
362         end
363         if (has_coverage(UVM_CVR_FIELD_VALS))
364             fld_cg = new();
365         endfunction : build
366     endclass : sha256_reg_global_intr_t_agg_sts_dd3dcf0a
367
368     // Reg - sha256_reg::global_intr_t_agg_sts_e6399b4a
369     class sha256_reg_global_intr_t_agg_sts_e6399b4a extends
uvm_reg;
370         protected uvm_reg_data_t m_current;

```

```

371         protected bit m_is_read;
372
373         sha256_reg_global_intr_t_agg_sts_e6399b4a_bit_cg
agg_sts_bit_cg[1];
374         sha256_reg_global_intr_t_agg_sts_e6399b4a_fld_cg
fld_cg;
375         rand uvm_reg_field agg_sts;
376
377         function new(string name =
"sha256_reg_global_intr_t_agg_sts_e6399b4a");
378             super.new(name, 32, build_coverage(UVM_CVR_ALL));
379         endfunction : new
380         extern virtual function void sample_values();
381         extern protected virtual function void
sample(uvm_reg_data_t data,
382
uvm_reg_data_t byte_en,
383
bit is_read,
384
uvm_reg_map map);
385
386         virtual function void build();
387             this.agg_sts = new("agg_sts");
388             this.agg_sts.configure(this, 1, 0, "R0", 1, 'h0,
1, 1, 0);
389         if (has_coverage(UVM_CVR_REG_BITS)) begin
390             foreach(agg_sts_bit_cg[bt]) agg_sts_bit_cg[bt] =
new();
391         end
392         if (has_coverage(UVM_CVR_FIELD_VALS))
393             fld_cg = new();
394         endfunction : build
395     endclass : sha256_reg_global_intr_t_agg_sts_e6399b4a
396
397     // Reg -
398     sha256_reg::error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378
399     class
400     sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378 extends uvm_reg;
401         protected uvm_reg_data_t m_current;
402         protected uvm_reg_data_t m_data;
403         protected bit m_is_read;
404
405         sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_bit_cg error0_sts_bit_cg[1];
406         sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_bit_cg error1_sts_bit_cg[1];
407         sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_bit_cg error2_sts_bit_cg[1];
408         sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_fld_cg fld_cg;
409         rand uvm_reg_field error0_sts;
410         rand uvm_reg_field error1_sts;
411         rand uvm_reg_field error2_sts;
412         rand uvm_reg_field error3_sts;
413
414         function new(string name =
"sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_e
rror2_sts_b1cf2205_error3_sts_74a35378");
415             super.new(name, 32, build_coverage(UVM_CVR_ALL));
416         endfunction : new
417         extern virtual function void sample_values();
418         extern protected virtual function void
sample(uvm_reg_data_t data,
419
uvm_reg_data_t byte_en,
420
bit is_read,
421
uvm_reg_map map);
422
423         virtual function void build();
424             this.error0_sts = new("error0_sts");
425             this.error0_sts.configure(this, 1, 0, "W1C", 1,
'h0, 1, 1, 0);
426             this.error1_sts = new("error1_sts");
427             this.error1_sts.configure(this, 1, 1, "W1C", 1,
'h0, 1, 1, 0);
428             this.error2_sts = new("error2_sts");
429             this.error2_sts.configure(this, 1, 2, "W1C", 1,
'h0, 1, 1, 0);
430             this.error3_sts = new("error3_sts");
431             this.error3_sts.configure(this, 1, 3, "W1C", 1,
'h0, 1, 1, 0);
432         if (has_coverage(UVM_CVR_REG_BITS)) begin
433             foreach(error0_sts_bit_cg[bt]) er-
ror0_sts_bit_cg[bt] = new();
434             foreach(error1_sts_bit_cg[bt]) er-
ror1_sts_bit_cg[bt] = new();
435             foreach(error2_sts_bit_cg[bt]) er-

```

```

ror2_sts_bit_cg[bt] = new();
435     foreach(error3_sts_bit_cg[bt]) er-
ror3_sts_bit_cg[bt] = new();
436     end
437     if (has_coverage(UVM_CVR_FIELD_VALS))
438         fld_cg = new();
439     endfunction : build
440 endclass :
sha256_reg__error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_bicf2205_error3_sts_74a35378
441
442 // Reg -
sha256_reg::notif_intr_t_notif_cmd_done_sts_1c68637e
443 class sha256_reg__notif_intr_t_notif_cmd_done_sts_1c68637e
extends uvm_reg;
444     protected uvm_reg_data_t m_current;
445     protected uvm_reg_data_t m_data;
446     protected bit m_is_read;
447
448 sha256_reg__notif_intr_t_notif_cmd_done_sts_1c68637e_bit_cg no-
tif_cmd_done_sts_bit_cg[1];
449
sha256_reg__notif_intr_t_notif_cmd_done_sts_1c68637e_fld_cg fld_cg;
450     rand uvm_reg_field notif_cmd_done_sts;
451
452     function new(string name =
"sha256_reg__notif_intr_t_notif_cmd_done_sts_1c68637e");
453         super.new(name, 32, build_coverage(UVM_CVR_ALL));
454     endfunction : new
455     extern virtual function void sample_values();
456     extern protected virtual function void
sample(uvm_reg_data_t data,
457 uvm_reg_data_t byte_en,
458 bit is_read,
459 uvm_reg_map map);
460
461     virtual function void build();
462         this.notif_cmd_done_sts = new("no-
tif_cmd_done_sts");
463         this.notif_cmd_done_sts.configure(this, 1, 0,
"W1C", 1, 'h0, 1, 1, 0);
464         if (has_coverage(UVM_CVR_REG_BITS)) begin
465             foreach(notif_cmd_done_sts_bit_cg[bt]) no-
tif_cmd_done_sts_bit_cg[bt] = new();
466         end
467         if (has_coverage(UVM_CVR_FIELD_VALS))
468             fld_cg = new();
469         endfunction : build
470     endclass :
sha256_reg__notif_intr_t_notif_cmd_done_sts_1c68637e
471
472 // Reg - sha256_reg::error_intr_trig_t
473 class sha256_reg__error_intr_trig_t extends uvm_reg;
474     protected uvm_reg_data_t m_current;
475     protected uvm_reg_data_t m_data;
476     protected bit m_is_read;
477
478     sha256_reg__error_intr_trig_t_bit_cg er-
ror0_trig_bit_cg[1];
479     sha256_reg__error_intr_trig_t_bit_cg er-
ror1_trig_bit_cg[1];
480     sha256_reg__error_intr_trig_t_bit_cg er-
ror2_trig_bit_cg[1];
481     sha256_reg__error_intr_trig_t_bit_cg er-
ror3_trig_bit_cg[1];
482     sha256_reg__error_intr_trig_t_fld_cg fld_cg;
483     rand uvm_reg_field error0_trig;
484     rand uvm_reg_field error1_trig;
485     rand uvm_reg_field error2_trig;
486     rand uvm_reg_field error3_trig;
487
488     function new(string name =
"sha256_reg__error_intr_trig_t");
489         super.new(name, 32, build_coverage(UVM_CVR_ALL));
490     endfunction : new
491     extern virtual function void sample_values();
492     extern protected virtual function void
sample(uvm_reg_data_t data,
493 uvm_reg_data_t byte_en,
494 bit is_read,
495 uvm_reg_map map);
496
497     virtual function void build();
498         this.error0_trig = new("error0_trig");
499         this.error0_trig.configure(this, 1, 0, "W1S", 0,
'h0, 1, 1, 0);
500         this.error1_trig = new("error1_trig");
501         this.error1_trig.configure(this, 1, 1, "W1S", 0,
'h0, 1, 1, 0);
502         this.error2_trig = new("error2_trig");
503         this.error2_trig.configure(this, 1, 2, "W1S", 0,
'h0, 1, 1, 0);
504         this.error3_trig = new("error3_trig");

```

```

505         this.error3_trig.configure(this, 1, 3, "W1S", 0,
'h0, 1, 1, 0);
506         if (has_coverage(UVM_CVR_REG_BITS)) begin
507             foreach(error0_trig_bit_cg[bt]) er-
ror0_trig_bit_cg[bt] = new();
508             foreach(error1_trig_bit_cg[bt]) er-
ror1_trig_bit_cg[bt] = new();
509             foreach(error2_trig_bit_cg[bt]) er-
ror2_trig_bit_cg[bt] = new();
510             foreach(error3_trig_bit_cg[bt]) er-
ror3_trig_bit_cg[bt] = new();
511         end
512         if (has_coverage(UVM_CVR_FIELD_VALS))
513             fld_cg = new();
514         endfunction : build
515     endclass : sha256_reg__error_intr_trig_t
516
517 // Reg - sha256_reg::notif_intr_trig_t
518 class sha256_reg__notif_intr_trig_t extends uvm_reg;
519     protected uvm_reg_data_t m_current;
520     protected uvm_reg_data_t m_data;
521     protected bit m_is_read;
522
523     sha256_reg__notif_intr_trig_t_bit_cg no-
tif_cmd_done_trig_bit_cg[1];
524     sha256_reg__notif_intr_trig_t_fld_cg fld_cg;
525     rand uvm_reg_field notif_cmd_done_trig;
526
527     function new(string name =
"sha256_reg__notif_intr_trig_t");
528         super.new(name, 32, build_coverage(UVM_CVR_ALL));
529     endfunction : new
530     extern virtual function void sample_values();
531     extern protected virtual function void
sample(uvm_reg_data_t data,
532 uvm_reg_data_t byte_en,
533 bit is_read,
534 uvm_reg_map map);
535
536     virtual function void build();
537         this.notif_cmd_done_trig = new("no-
tif_cmd_done_trig");
538         this.notif_cmd_done_trig.configure(this, 1, 0,
"W1S", 0, 'h0, 1, 1, 0);
539         if (has_coverage(UVM_CVR_REG_BITS)) begin
540             foreach(notif_cmd_done_trig_bit_cg[bt]) no-
tif_cmd_done_trig_bit_cg[bt] = new();
541         end
542         if (has_coverage(UVM_CVR_FIELD_VALS))
543             fld_cg = new();
544         endfunction : build
545     endclass : sha256_reg__notif_intr_trig_t
546
547 // Reg - sha256_reg::intr_count_t_cnt_35ace267
548 class sha256_reg__intr_count_t_cnt_35ace267 extends
uvm_reg;
549     protected uvm_reg_data_t m_current;
550     protected uvm_reg_data_t m_data;
551     protected bit m_is_read;
552
553     sha256_reg__intr_count_t_cnt_35ace267_bit_cg
cnt_bit_cg[32];
554     sha256_reg__intr_count_t_cnt_35ace267_fld_cg fld_cg;
555     rand uvm_reg_field cnt;
556
557     function new(string name =
"sha256_reg__intr_count_t_cnt_35ace267");
558         super.new(name, 32, build_coverage(UVM_CVR_ALL));
559     endfunction : new
560     extern virtual function void sample_values();
561     extern protected virtual function void
sample(uvm_reg_data_t data,
562 uvm_reg_data_t byte_en,
563 bit is_read,
564 uvm_reg_map map);
565
566     virtual function void build();
567         this.cnt = new("cnt");
568         this.cnt.configure(this, 32, 0, "RW", 1, 'h0, 1,
1, 0);
569         if (has_coverage(UVM_CVR_REG_BITS)) begin
570             foreach(cnt_bit_cg[bt]) cnt_bit_cg[bt] =
new();
571         end
572         if (has_coverage(UVM_CVR_FIELD_VALS))
573             fld_cg = new();
574         endfunction : build
575     endclass : sha256_reg__intr_count_t_cnt_35ace267
576
577 // Reg - sha256_reg::intr_count_t_cnt_73c42c28
578 class sha256_reg__intr_count_t_cnt_73c42c28 extends
uvm_reg;
579     protected uvm_reg_data_t m_current;
580     protected uvm_reg_data_t m_data;
581     protected bit m_is_read;

```

```

582
583     sha256_reg_intr_count_t_cnt_73c42c28_bit_cg
cnt_bit_cg[32];
584     sha256_reg_intr_count_t_cnt_73c42c28_fld_cg fld_cg;
585     rand uvm_reg_field cnt;
586
587     function new(string name =
"sha256_reg_intr_count_t_cnt_73c42c28");
588         super.new(name, 32, build_coverage(UVM_CVR_ALL));
589     endfunction : new
590     extern virtual function void sample_values();
591     extern protected virtual function void
sample(uvm_reg_data_t data,
592
593     uvm_reg_data_t byte_en,
594
595     bit is_read,
596
597     uvm_reg_map map);
598
599     virtual function void build();
600     this.cnt = new("cnt");
601     this.cnt.configure(this, 32, 0, "RW", 1, 'h0, 1,
1, 0);
602     if (has_coverage(UVM_CVR_REG_BITS)) begin
603         foreach(cnt_bit_cg[bt]) cnt_bit_cg[bt] =
new();
604     end
605     if (has_coverage(UVM_CVR_FIELD_VALS))
606     fld_cg = new();
607     endfunction : build
608     endclass : sha256_reg_intr_count_t_cnt_73c42c28
609
610     // Reg - sha256_reg::intr_count_t_cnt_d8af96ff
611     class sha256_reg_intr_count_t_cnt_d8af96ff extends
uvm_reg;
612         protected uvm_reg_data_t m_current;
613         protected uvm_reg_data_t m_data;
614         protected bit m_is_read;
615
616         sha256_reg_intr_count_t_cnt_d8af96ff_bit_cg
cnt_bit_cg[32];
617         sha256_reg_intr_count_t_cnt_d8af96ff_fld_cg fld_cg;
618         rand uvm_reg_field cnt;
619
620         function new(string name =
"sha256_reg_intr_count_t_cnt_d8af96ff");
621             super.new(name, 32, build_coverage(UVM_CVR_ALL));
622         endfunction : new
623         extern virtual function void sample_values();
624         extern protected virtual function void
sample(uvm_reg_data_t data,
625
626         uvm_reg_data_t byte_en,
627
628         bit is_read,
629
630         uvm_reg_map map);
631
632         virtual function void build();
633         this.cnt = new("cnt");
634         this.cnt.configure(this, 32, 0, "RW", 1, 'h0, 1,
1, 0);
635         if (has_coverage(UVM_CVR_REG_BITS)) begin
636             foreach(cnt_bit_cg[bt]) cnt_bit_cg[bt] =
new();
637         end
638         if (has_coverage(UVM_CVR_FIELD_VALS))
639         fld_cg = new();
640         endfunction : build
641         endclass : sha256_reg_intr_count_t_cnt_d8af96ff
642
643     // Reg - sha256_reg::intr_count_t_cnt_9bd7f809
644     class sha256_reg_intr_count_t_cnt_9bd7f809 extends
uvm_reg;
645         protected uvm_reg_data_t m_current;
646         protected uvm_reg_data_t m_data;
647         protected bit m_is_read;
648
649         sha256_reg_intr_count_t_cnt_9bd7f809_bit_cg
cnt_bit_cg[32];
650         sha256_reg_intr_count_t_cnt_9bd7f809_fld_cg fld_cg;
651         rand uvm_reg_field cnt;
652
653         function new(string name =
"sha256_reg_intr_count_t_cnt_9bd7f809");
654             super.new(name, 32, build_coverage(UVM_CVR_ALL));
655         endfunction : new
656         extern virtual function void sample_values();
657         extern protected virtual function void
sample(uvm_reg_data_t data,
658
659         uvm_reg_data_t byte_en,
660
661         bit is_read,
662
663         uvm_reg_map map);
664
665         virtual function void build();
666         this.cnt = new("cnt");
667         this.cnt.configure(this, 32, 0, "RW", 1, 'h0, 1,

```

```

1, 0);
668     if (has_coverage(UVM_CVR_REG_BITS)) begin
669         foreach(cnt_bit_cg[bt]) cnt_bit_cg[bt] =
new();
670     end
671     if (has_coverage(UVM_CVR_FIELD_VALS))
672     fld_cg = new();
673     endfunction : build
674     endclass : sha256_reg_intr_count_t_cnt_9bd7f809
675
676     // Reg - sha256_reg::intr_count_t_cnt_be67d6d5
677     class sha256_reg_intr_count_t_cnt_be67d6d5 extends
uvm_reg;
678         protected uvm_reg_data_t m_current;
679         protected uvm_reg_data_t m_data;
680         protected bit m_is_read;
681
682         sha256_reg_intr_count_t_cnt_be67d6d5_bit_cg
cnt_bit_cg[32];
683         sha256_reg_intr_count_t_cnt_be67d6d5_fld_cg fld_cg;
684         rand uvm_reg_field cnt;
685
686         function new(string name =
"sha256_reg_intr_count_t_cnt_be67d6d5");
687             super.new(name, 32, build_coverage(UVM_CVR_ALL));
688         endfunction : new
689         extern virtual function void sample_values();
690         extern protected virtual function void
sample(uvm_reg_data_t data,
691
692         uvm_reg_data_t byte_en,
693
694         bit is_read,
695
696         uvm_reg_map map);
697
698         virtual function void build();
699         this.cnt = new("cnt");
700         this.cnt.configure(this, 32, 0, "RW", 1, 'h0, 1,
1, 0);
701         if (has_coverage(UVM_CVR_REG_BITS)) begin
702             foreach(cnt_bit_cg[bt]) cnt_bit_cg[bt] =
new();
703         end
704         if (has_coverage(UVM_CVR_FIELD_VALS))
705         fld_cg = new();
706         endfunction : build
707         endclass : sha256_reg_intr_count_t_cnt_be67d6d5
708
709     // Reg - sha256_reg::intr_count_incr_t_pulse_37026c97
710     class sha256_reg_intr_count_incr_t_pulse_37026c97 extends
uvm_reg;
711         protected uvm_reg_data_t m_current;
712         protected uvm_reg_data_t m_data;
713         protected bit m_is_read;
714
715         sha256_reg_intr_count_incr_t_pulse_37026c97_bit_cg
pulse_bit_cg[1];
716         sha256_reg_intr_count_incr_t_pulse_37026c97_fld_cg
fld_cg;
717         rand uvm_reg_field pulse;
718
719         function new(string name =
"sha256_reg_intr_count_incr_t_pulse_37026c97");
720             super.new(name, 32, build_coverage(UVM_CVR_ALL));
721         endfunction : new
722         extern virtual function void sample_values();
723         extern protected virtual function void
sample(uvm_reg_data_t data,
724
725         uvm_reg_data_t byte_en,
726
727         bit is_read,
728
729         uvm_reg_map map);
730
731         virtual function void build();
732         this.pulse = new("pulse");
733         this.pulse.configure(this, 1, 0, "R0", 1, 'h0, 1,
1, 0);
734         if (has_coverage(UVM_CVR_REG_BITS)) begin
735             foreach(pulse_bit_cg[bt]) pulse_bit_cg[bt] =
new();
736         end
737         if (has_coverage(UVM_CVR_FIELD_VALS))
738         fld_cg = new();
739         endfunction : build
740         endclass : sha256_reg_intr_count_incr_t_pulse_37026c97
741
742     // Reg - sha256_reg::intr_count_incr_t_pulse_d860d977
743     class sha256_reg_intr_count_incr_t_pulse_d860d977 extends
uvm_reg;
744         protected uvm_reg_data_t m_current;
745         protected uvm_reg_data_t m_data;
746         protected bit m_is_read;
747
748         sha256_reg_intr_count_incr_t_pulse_d860d977_bit_cg
pulse_bit_cg[1];
749         sha256_reg_intr_count_incr_t_pulse_d860d977_fld_cg

```

```

fld_cg;
735     rand uvm_reg_field pulse;
736
737     function new(string name =
"sha256_reg__intr_count_incr_t_pulse_d860d977");
738         super.new(name, 32, build_coverage(UVM_CVR_ALL));
739     endfunction : new
740     extern virtual function void sample_values();
741     extern protected virtual function void
sample(uvm_reg_data_t data,
742
uvm_reg_data_t byte_en,
743
bit         is_read,
744
uvm_reg_map map);
745
746     virtual function void build();
747         this.pulse = new("pulse");
748         this.pulse.configure(this, 1, 0, "R0", 1, 'h0, 1,
1, 0);
749         if (has_coverage(UVM_CVR_REG_BITS)) begin
750             foreach(pulse_bit_cg[bt]) pulse_bit_cg[bt] =
new();
751         end
752         if (has_coverage(UVM_CVR_FIELD_VALS))
753             fld_cg = new();
754         endfunction : build
755     endclass : sha256_reg__intr_count_incr_t_pulse_d860d977
756
757     // Reg - sha256_reg::intr_count_incr_t_pulse_87b45fe7

758     class sha256_reg__intr_count_incr_t_pulse_87b45fe7 extends
uvm_reg;
759         protected uvm_reg_data_t m_current;
760         protected uvm_reg_data_t m_data;
761         protected bit         m_is_read;
762
763         sha256_reg__intr_count_incr_t_pulse_87b45fe7_bit_cg
pulse_bit_cg[1];
764         sha256_reg__intr_count_incr_t_pulse_87b45fe7_fld_cg
fld_cg;
765         rand uvm_reg_field pulse;
766
767         function new(string name =
"sha256_reg__intr_count_incr_t_pulse_87b45fe7");
768             super.new(name, 32, build_coverage(UVM_CVR_ALL));
769         endfunction : new
770         extern virtual function void sample_values();
771         extern protected virtual function void
sample(uvm_reg_data_t data,
772
uvm_reg_data_t byte_en,
773
bit         is_read,
774
uvm_reg_map map);
775
776         virtual function void build();
777             this.pulse = new("pulse");
778             this.pulse.configure(this, 1, 0, "R0", 1, 'h0, 1,
1, 0);
779             if (has_coverage(UVM_CVR_REG_BITS)) begin
780                 foreach(pulse_bit_cg[bt]) pulse_bit_cg[bt] =
new();
781             end
782             if (has_coverage(UVM_CVR_FIELD_VALS))
783                 fld_cg = new();
784             endfunction : build
785         endclass : sha256_reg__intr_count_incr_t_pulse_87b45fe7
786
787         // Reg - sha256_reg::intr_count_incr_t_pulse_c1689ee6

788     class sha256_reg__intr_count_incr_t_pulse_c1689ee6 extends
uvm_reg;
789         protected uvm_reg_data_t m_current;
790         protected uvm_reg_data_t m_data;
791         protected bit         m_is_read;
792
793         sha256_reg__intr_count_incr_t_pulse_c1689ee6_bit_cg
pulse_bit_cg[1];
794         sha256_reg__intr_count_incr_t_pulse_c1689ee6_fld_cg
fld_cg;
795         rand uvm_reg_field pulse;
796
797         function new(string name =
"sha256_reg__intr_count_incr_t_pulse_c1689ee6");
798             super.new(name, 32, build_coverage(UVM_CVR_ALL));
799         endfunction : new
800         extern virtual function void sample_values();
801         extern protected virtual function void
sample(uvm_reg_data_t data,
802
uvm_reg_data_t byte_en,
803
bit         is_read,
804
uvm_reg_map map);
805
806         virtual function void build();
807             this.pulse = new("pulse");

```

```

808         this.pulse.configure(this, 1, 0, "R0", 1, 'h0, 1,
1, 0);
809         if (has_coverage(UVM_CVR_REG_BITS)) begin
810             foreach(pulse_bit_cg[bt]) pulse_bit_cg[bt] =
new();
811         end
812         if (has_coverage(UVM_CVR_FIELD_VALS))
813             fld_cg = new();
814         endfunction : build
815     endclass : sha256_reg__intr_count_incr_t_pulse_c1689ee6
816
817     // Reg - sha256_reg::intr_count_incr_t_pulse_6173128e

818     class sha256_reg__intr_count_incr_t_pulse_6173128e extends
uvm_reg;
819         protected uvm_reg_data_t m_current;
820         protected uvm_reg_data_t m_data;
821         protected bit         m_is_read;
822
823         sha256_reg__intr_count_incr_t_pulse_6173128e_bit_cg
pulse_bit_cg[1];
824         sha256_reg__intr_count_incr_t_pulse_6173128e_fld_cg
fld_cg;
825         rand uvm_reg_field pulse;
826
827         function new(string name =
"sha256_reg__intr_count_incr_t_pulse_6173128e");
828             super.new(name, 32, build_coverage(UVM_CVR_ALL));
829         endfunction : new
830         extern virtual function void sample_values();
831         extern protected virtual function void
sample(uvm_reg_data_t data,
832
uvm_reg_data_t byte_en,
833
bit         is_read,
834
uvm_reg_map map);
835
836         virtual function void build();
837             this.pulse = new("pulse");
838             this.pulse.configure(this, 1, 0, "R0", 1, 'h0, 1,
1, 0);
839             if (has_coverage(UVM_CVR_REG_BITS)) begin
840                 foreach(pulse_bit_cg[bt]) pulse_bit_cg[bt] =
new();
841             end
842             if (has_coverage(UVM_CVR_FIELD_VALS))
843                 fld_cg = new();
844             endfunction : build
845         endclass : sha256_reg__intr_count_incr_t_pulse_6173128e
846
847         // Regfile - sha256_reg::intr_block_t
848         class sha256_reg__intr_block_t extends uvm_reg_block;
849             rand sha256_reg__global_intr_en_t global_intr_en_r;
850             rand sha256_reg__error_intr_en_t error_intr_en_r;
851             rand sha256_reg__notif_intr_en_t notif_intr_en_r;
852             rand sha256_reg__global_intr_t_agg_sts_dd3dcf0a er-
ror_global_intr_r;
853             rand sha256_reg__global_intr_t_agg_sts_e6399b4a no-
tif_global_intr_r;
854             rand
sha256_reg__error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1 er-
ror2_sts_b1cf2205_error3_sts_74a35378 error_internal_intr_r;
855             rand
sha256_reg__notif_intr_t_notif_cmd_done_sts_1c68637e no-
tif_internal_intr_r;
856             rand sha256_reg__error_intr_trig_t error_intr_trig_r;
857             rand sha256_reg__notif_intr_trig_t notif_intr_trig_r;
858             rand sha256_reg__intr_count_t_cnt_35ace267 er-
ror0_intr_count_r;
859             rand sha256_reg__intr_count_t_cnt_73c42c28 er-
ror1_intr_count_r;
860             rand sha256_reg__intr_count_t_cnt_d8af96ff er-
ror2_intr_count_r;
861             rand sha256_reg__intr_count_t_cnt_9bd7f809 er-
ror3_intr_count_r;
862             rand sha256_reg__intr_count_t_cnt_be67d6d5 no-
tif_cmd_done_intr_count_r;
863             rand sha256_reg__intr_count_incr_t_pulse_37026c97 er-
ror0_intr_count_incr_r;
864             rand sha256_reg__intr_count_incr_t_pulse_d860d977 er-
ror1_intr_count_incr_r;
865             rand sha256_reg__intr_count_incr_t_pulse_87b45fe7 er-
ror2_intr_count_incr_r;
866             rand sha256_reg__intr_count_incr_t_pulse_c1689ee6 er-
ror3_intr_count_incr_r;
867             rand sha256_reg__intr_count_incr_t_pulse_6173128e no-
tif_cmd_done_intr_count_incr_r;
868
869             function new(string name =
"sha256_reg__intr_block_t");
870                 super.new(name);
871             endfunction : new
872
873             virtual function void build();
874                 this.default_map = create_map("reg_map", 0, 4,
UVM_NO_ENDIAN);
875                 this.global_intr_en_r = new("global_intr_en_r");
876                 this.global_intr_en_r.configure(this);
877

```



```

878         this.global_intr_en_r.build();
879         this.default_map.add_reg(this.global_intr_en_r,
'h0);
880         this.error_intr_en_r = new("error_intr_en_r");
881         this.error_intr_en_r.configure(this);
882
883         this.error_intr_en_r.build();
884         this.default_map.add_reg(this.error_intr_en_r,
'h4);
885         this.notif_intr_en_r = new("notif_intr_en_r");
886         this.notif_intr_en_r.configure(this);
887
888         this.notif_intr_en_r.build();
889         this.default_map.add_reg(this.notif_intr_en_r,
'h8);
890         this.error_global_intr_r = new("er-
ror_global_intr_r");
891         this.error_global_intr_r.configure(this);
892
893         this.error_global_intr_r.build();
894
895         this.default_map.add_reg(this.error_global_intr_r,
'hc);
896         this.notif_global_intr_r = new("no-
tif_global_intr_r");
897         this.notif_global_intr_r.configure(this);
898
899         this.notif_global_intr_r.build();
900
901         this.default_map.add_reg(this.notif_global_intr_r,
'h10);
902         this.error_internal_intr_r = new("er-
ror_internal_intr_r");
903         this.error_internal_intr_r.configure(this);
904
905         this.error_internal_intr_r.build();
906         this.de-
fault_map.add_reg(this.error_internal_intr_r, 'h14);
907         this.notif_internal_intr_r = new("no-
tif_internal_intr_r");
908         this.notif_internal_intr_r.configure(this);
909
910         this.notif_internal_intr_r.build();
911         this.de-
fault_map.add_reg(this.notif_internal_intr_r, 'h18);
912         this.error_intr_trig_r = new("error_intr_trig_r");
913         this.error_intr_trig_r.configure(this);
914
915         this.error_intr_trig_r.build();
916         this.default_map.add_reg(this.error_intr_trig_r,
'h1c);
917         this.notif_intr_trig_r = new("notif_intr_trig_r");
918         this.notif_intr_trig_r.configure(this);
919
920         this.notif_intr_trig_r.build();
921         this.default_map.add_reg(this.notif_intr_trig_r,
'h20);
922         this.error0_intr_count_r = new("er-
ror0_intr_count_r");
923         this.error0_intr_count_r.configure(this);
924
925         this.error0_intr_count_r.build();
926
927         this.default_map.add_reg(this.error0_intr_count_r,
'h100);
928         this.error1_intr_count_r = new("er-
ror1_intr_count_r");
929         this.error1_intr_count_r.configure(this);
930
931         this.error1_intr_count_r.build();
932
933         this.default_map.add_reg(this.error1_intr_count_r,
'h104);
934         this.error2_intr_count_r = new("er-
ror2_intr_count_r");
935         this.error2_intr_count_r.configure(this);
936
937         this.error2_intr_count_r.build();
938
939         this.default_map.add_reg(this.error2_intr_count_r,
'h108);
940         this.error3_intr_count_r = new("er-
ror3_intr_count_r");
941         this.error3_intr_count_r.configure(this);
942
943         this.error3_intr_count_r.build();
944
945         this.default_map.add_reg(this.error3_intr_count_r,
'h10c);
946         this.notif_cmd_done_intr_count_r = new("no-
tif_cmd_done_intr-
count_r");
947         this.notif_cmd_done_intr_count_r.configure(this);
948
949         this.notif_cmd_done_intr_count_r.build();
950         this.de-
fault_map.add_reg(this.notif_cmd_done_intr_count_r, 'h180);
951         this.error0_intr_count_incr_r = new("er-
ror0_intr_count_incr_r");
952         this.error0_intr_count_incr_r.configure(this);
953
954         this.error0_intr_count_incr_r.build();

```

```

949         this.de-
fault_map.add_reg(this.error0_intr_count_incr_r, 'h200);
950         this.error1_intr_count_incr_r = new("er-
ror1_intr_count_incr_r");
951         this.error1_intr_count_incr_r.configure(this);
952
953         this.error1_intr_count_incr_r.build();
954         this.de-
fault_map.add_reg(this.error1_intr_count_incr_r, 'h204);
955         this.error2_intr_count_incr_r = new("er-
ror2_intr_count_incr_r");
956         this.error2_intr_count_incr_r.configure(this);
957
958         this.error2_intr_count_incr_r.build();
959         this.de-
fault_map.add_reg(this.error2_intr_count_incr_r, 'h208);
960         this.error3_intr_count_incr_r = new("er-
ror3_intr_count_incr_r");
961         this.error3_intr_count_incr_r.configure(this);
962
963         this.error3_intr_count_incr_r.build();
964         this.de-
fault_map.add_reg(this.error3_intr_count_incr_r, 'h20c);
965         this.notif_cmd_done_intr_count_incr_r = new("no-
tif_cmd_done_intr_count_incr_r");
966         this.no-
tif_cmd_done_intr_count_incr_r.configure(this);
967
968         this.notif_cmd_done_intr_count_incr_r.build();
969         this.de-
fault_map.add_reg(this.notif_cmd_done_intr_count_incr_r, 'h210);
970         endfunction : build
971         endclass : sha256_reg_intr_block_t
972
973         // Addrmap - sha256_reg
974         class sha256_reg extends uvm_reg_block;
975             rand sha256_reg_SHA256_NAME SHA256_NAME[2];
976             rand sha256_reg_SHA256_VERSION SHA256_VERSION[2];
977             rand sha256_reg_SHA256_CTRL SHA256_CTRL;
978             rand sha256_reg_SHA256_STATUS SHA256_STATUS;
979             rand sha256_reg_SHA256_BLOCK SHA256_BLOCK[16];
980             rand sha256_reg_SHA256_DIGEST SHA256_DIGEST[8];
981             rand sha256_reg_intr_block_t intr_block_rf;
982
983             function new(string name = "sha256_reg");
984                 super.new(name);
985             endfunction : new
986
987             virtual function void build();
988                 this.default_map = create_map("reg_map", 0, 4,
UVM_NO_ENDIAN);
989                 foreach(this.SHA256_NAME[i0]) begin
990                     this.SHA256_NAME[i0] = new($sform-
atf("SHA256_NAME[%0d]", i0));
991                     this.SHA256_NAME[i0].configure(this);
992
993                     this.SHA256_NAME[i0].build();
994
995                     this.default_map.add_reg(this.SHA256_NAME[i0],
'h0 + i0*'h4);
996                 end
997                 foreach(this.SHA256_VERSION[i0]) begin
998                     this.SHA256_VERSION[i0] = new($sform-
atf("SHA256_VERSION[%0d]", i0));
999                     this.SHA256_VERSION[i0].configure(this);
1000
1001                     this.SHA256_VERSION[i0].build();
1002                     this.de-
fault_map.add_reg(this.SHA256_VERSION[i0], 'h8 + i0*'h4);
1003                 end
1004                 this.SHA256_CTRL = new("SHA256_CTRL");
1005                 this.SHA256_CTRL.configure(this);
1006
1007                 this.SHA256_CTRL.build();
1008                 this.default_map.add_reg(this.SHA256_CTRL, 'h10);
1009                 this.SHA256_STATUS = new("SHA256_STATUS");
1010                 this.SHA256_STATUS.configure(this);
1011
1012                 this.SHA256_STATUS.build();
1013                 this.default_map.add_reg(this.SHA256_STATUS,
'h18);
1014                 foreach(this.SHA256_BLOCK[i0]) begin
1015                     this.SHA256_BLOCK[i0] = new($sform-
atf("SHA256_BLOCK[%0d]", i0));
1016                     this.SHA256_BLOCK[i0].configure(this);
1017
1018                     this.SHA256_BLOCK[i0].build();
1019                     this.de-
fault_map.add_reg(this.SHA256_BLOCK[i0], 'h80 + i0*'h4);
1020                 end
1021                 foreach(this.SHA256_DIGEST[i0]) begin
1022                     this.SHA256_DIGEST[i0] = new($sform-
atf("SHA256_DIGEST[%0d]", i0));
1023                     this.SHA256_DIGEST[i0].configure(this);
1024
1025                     this.SHA256_DIGEST[i0].build();
1026                     this.de-
fault_map.add_reg(this.SHA256_DIGEST[i0], 'h100 + i0*'h4);
1027                 end
1028                 this.intr_block_rf = new("intr_block_rf");
1029                 this.intr_block_rf.configure(this);

```

```

1029         this.intr_block_rf.build();
1030         this.de-
fault_map.add_submap(this.intr_block_rf.default_map, 'h800');
1031         endfunction : build
1032     endclass : sha256_reg
1033
1034     `include "sha256_reg_sample.svh"
1035 endpackage: sha256_reg_uvm

```

9. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_w_mem.v

(279 lines)

```

1 //
=====
2 //
3 // sha256_w_mem_regs.v
4 // -----
5 // The W memory. This version uses 16 32-bit registers as a
sliding
6 // window to generate the 64 words.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2013, Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the fol-
lowing
15 // conditions are met:
16 //
17 // 1. Redistributions of source code must retain the above
copyright
18 // notice, this list of conditions and the following dis-
claimer.
19 //
20 // 2. Redistributions in binary form must reproduce the above
copyright
21 // notice, this list of conditions and the following dis-
claimer in
22 // the documentation and/or other materials provided with
the
23 // distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
TRIBUTORS
26 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
27 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
28 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
29 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
30 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
31 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
32 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
33 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
35 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====
39
40 module sha256_w_mem(
41     // Clock and reset.
42     input wire      clk,
43     input wire      reset_n,
44     input wire      zeroize,
45     // Data port.
46     input wire [511 : 0] block_msg,
47     // Control.
48     input wire      init_cmd,
49     input wire      next_cmd,
50     output wire [31 : 0] w_val
51 );
52
53
54 //-----
55 // Registers including update variables and write enable.
56 //-----
57 reg [31 : 0] w_mem [0 : 15];
58 reg [31 : 0] w_mem00_new;
59 reg [31 : 0] w_mem01_new;
60 reg [31 : 0] w_mem02_new;
61 reg [31 : 0] w_mem03_new;
62 reg [31 : 0] w_mem04_new;
63 reg [31 : 0] w_mem05_new;
64 reg [31 : 0] w_mem06_new;
65 reg [31 : 0] w_mem07_new;
66 reg [31 : 0] w_mem08_new;

```

```

67 reg [31 : 0] w_mem09_new;
68 reg [31 : 0] w_mem10_new;
69 reg [31 : 0] w_mem11_new;
70 reg [31 : 0] w_mem12_new;
71 reg [31 : 0] w_mem13_new;
72 reg [31 : 0] w_mem14_new;
73 reg [31 : 0] w_mem15_new;
74 reg          w_mem_we;
75
76 reg [5 : 0] w_ctr_reg;
77 reg [5 : 0] w_ctr_new;
78 reg          w_ctr_we;
79
80
81 //-----
82 // Wires.
83 //-----
84 reg [31 : 0] w_tmp;
85 reg [31 : 0] w_new;
86
87
88 //-----
89 // Concurrent connectivity for ports etc.
90 //-----
91 assign w_val = w_tmp;
92
93
94 //-----
95 // reg_update
96 // Update functionality for all registers in the core.
97 // All registers are positive edge triggered with synchron-
ous
98 // active low reset. All registers have write enable.
99 //-----
100 always @ (posedge clk or negedge reset_n)
101     begin : reg_update
102         integer ii;
103
104         if (!reset_n)
105             begin
106                 for (ii = 0 ; ii < 16 ; ii = ii + 1)
107                     w_mem[ii] <= 32'h0;
108
109                 w_ctr_reg <= 6'h0;
110             end
111         else if (zeroize)
112             begin
113                 for (ii = 0 ; ii < 16 ; ii = ii + 1)
114                     w_mem[ii] <= 32'h0;
115
116                 w_ctr_reg <= 6'h0;
117             end
118         else
119             begin
120                 if (w_mem_we)
121                     begin
122                         w_mem[00] <= w_mem00_new;
123                         w_mem[01] <= w_mem01_new;
124                         w_mem[02] <= w_mem02_new;
125                         w_mem[03] <= w_mem03_new;
126                         w_mem[04] <= w_mem04_new;
127                         w_mem[05] <= w_mem05_new;
128                         w_mem[06] <= w_mem06_new;
129                         w_mem[07] <= w_mem07_new;
130                         w_mem[08] <= w_mem08_new;
131                         w_mem[09] <= w_mem09_new;
132                         w_mem[10] <= w_mem10_new;
133                         w_mem[11] <= w_mem11_new;
134                         w_mem[12] <= w_mem12_new;
135                         w_mem[13] <= w_mem13_new;
136                         w_mem[14] <= w_mem14_new;
137                         w_mem[15] <= w_mem15_new;
138                     end
139
140                 if (w_ctr_we)
141                     w_ctr_reg <= w_ctr_new;
142             end
143         end // reg_update
144
145
146 //-----
147 // select_w
148 //
149 // Mux for the external read operation. This is where we ex-
ract
150 // the w variable.
151 //-----
152 always @*
153     begin : select_w
154         if (w_ctr_reg < 16)
155             w_tmp = w_mem[w_ctr_reg[3 : 0]];
156         else
157             w_tmp = w_new;

```



```

158     end // select_w
159
160
161 //-----
162 // w_new_logic
163 //
164 // Logic that calculates the next value to be inserted into
165 // the sliding window of the memory.
166 //-----
167 always @*
168   begin : w_mem_update_logic
169     reg [31 : 0] w_0;
170     reg [31 : 0] w_1;
171     reg [31 : 0] w_9;
172     reg [31 : 0] w_14;
173     reg [31 : 0] d0;
174     reg [31 : 0] d1;
175
176     w_mem00_new = 32'h0;
177     w_mem01_new = 32'h0;
178     w_mem02_new = 32'h0;
179     w_mem03_new = 32'h0;
180     w_mem04_new = 32'h0;
181     w_mem05_new = 32'h0;
182     w_mem06_new = 32'h0;
183     w_mem07_new = 32'h0;
184     w_mem08_new = 32'h0;
185     w_mem09_new = 32'h0;
186     w_mem10_new = 32'h0;
187     w_mem11_new = 32'h0;
188     w_mem12_new = 32'h0;
189     w_mem13_new = 32'h0;
190     w_mem14_new = 32'h0;
191     w_mem15_new = 32'h0;
192     w_mem_we     = 0;
193
194     w_0 = w_mem[0];
195     w_1 = w_mem[1];
196     w_9 = w_mem[9];
197     w_14 = w_mem[14];
198
199     d0 = {w_1[6 : 0], w_1[31 : 7]} ^
200         {w_1[17 : 0], w_1[31 : 18]} ^
201         {3'b000, w_1[31 : 3]};
202
203     d1 = {w_14[16 : 0], w_14[31 : 17]} ^
204         {w_14[18 : 0], w_14[31 : 19]} ^
205         {10'b0000000000, w_14[31 : 10]};
206
207     w_new = d1 + w_9 + d0 + w_0;
208
209     if (init_cmd)
210       begin
211         w_mem00_new = block_msg[511 : 480];
212         w_mem01_new = block_msg[479 : 448];
213         w_mem02_new = block_msg[447 : 416];
214         w_mem03_new = block_msg[415 : 384];
215         w_mem04_new = block_msg[383 : 352];
216         w_mem05_new = block_msg[351 : 320];
217         w_mem06_new = block_msg[319 : 288];
218         w_mem07_new = block_msg[287 : 256];
219         w_mem08_new = block_msg[255 : 224];
220         w_mem09_new = block_msg[223 : 192];

```

```

221     w_mem10_new = block_msg[191 : 160];
222     w_mem11_new = block_msg[159 : 128];
223     w_mem12_new = block_msg[127 : 96];
224     w_mem13_new = block_msg[95 : 64];
225     w_mem14_new = block_msg[63 : 32];
226     w_mem15_new = block_msg[31 : 0];
227     w_mem_we     = 1;
228   end
229
230   if (next_cmd && (w_ctr_reg > 15))
231     begin
232       w_mem00_new = w_mem[01];
233       w_mem01_new = w_mem[02];
234       w_mem02_new = w_mem[03];
235       w_mem03_new = w_mem[04];
236       w_mem04_new = w_mem[05];
237       w_mem05_new = w_mem[06];
238       w_mem06_new = w_mem[07];
239       w_mem07_new = w_mem[08];
240       w_mem08_new = w_mem[09];
241       w_mem09_new = w_mem[10];
242       w_mem10_new = w_mem[11];
243       w_mem11_new = w_mem[12];
244       w_mem12_new = w_mem[13];
245       w_mem13_new = w_mem[14];
246       w_mem14_new = w_mem[15];
247       w_mem15_new = w_new;
248       w_mem_we     = 1;
249     end
250   end // w_mem_update_logic
251
252
253 //-----
254 // w_ctr
255 // W schedule adress counter. Counts from 0x10 to 0x3f and
256 // is used to expand the block into words.
257 //-----
258 always @*
259   begin : w_ctr
260     w_ctr_new = 6'h0;
261     w_ctr_we  = 1'h0;
262
263     if (init_cmd)
264       begin
265         w_ctr_new = 6'h0;
266         w_ctr_we  = 1'h1;
267       end
268
269     if (next_cmd)
270       begin
271         w_ctr_new = w_ctr_reg + 6'h01;
272         w_ctr_we  = 1'h1;
273       end
274     end // w_ctr
275 endmodule // sha256_w_mem
276
277 //
=====
278 // sha256_w_mem.v
279 //
=====

```

Caliptra HMAC

What this is. Keyed-hash message authentication in a root of trust.

Why it exists. Because integrity without authentication is insufficient: a plain digest can be recomputed by an attacker who can modify the data.

Undergraduate review · EECS concepts. HMAC construction; key handling; constant-time comparison.

Graduate · Practice note. Compare digests in constant time. A byte-by-byte comparison that returns early leaks the position of the first difference through timing — a classic and easily fixed vulnerability.

6 files • 4,099 lines

API — hmac.sv

Table 1360. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac.sv — module / interface / package (1개)

이름	종류
hmac	module

Table 1361. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac.sv — localparam (4개)

상수	형/폭	값
BLOCK_NUM_DWORDS		BLOCK_SIZE / DATA_WIDTH
KEY_NUM_DWORDS		KEY_SIZE / DATA_WIDTH
TAG_NUM_DWORDS		TAG_SIZE / DATA_WIDTH
SEED_NUM_DWORDS		((LFSR_SEED_SIZE - 1) / DATA_WIDTH) + 1

Table 1362. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac.sv — 포트 (18개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
cptra_pwrgood	input	wire
cptra_csr_hmac_key	input	logic [CLP_CSR_HMAC_KEY_DWORDS-1:0][31:0]
cs	input	wire
we	input	wire
address	input	wire [ADDR_WIDTH - 1 : 0]
write_data	input	wire [DATA_WIDTH - 1 : 0]
read_data	output	wire [DATA_WIDTH - 1 : 0]
kv_read	output	kv_read_t [1:0]
kv_write	output	kv_write_t

포트	방향	형/폭
kv_rd_resp	input	kv_rd_resp_t [1:0]
kv_wr_resp	input	kv_wr_resp_t
busy_o	output	logic
error_intr	output	wire
notif_intr	output	wire
ocp_lock_in_progress	input	logic
debugUnlock_or_scan_mode_switch	input	logic

API — hmac_core.sv

Table 1363. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_core.sv — module / interface / package (1개)

이름	종류
hmac_core	module

Table 1364. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_core.sv — localparam (7개)

상수	형/폭	값
CTRL_IDLE	[2 : 0]	3'd0
CTRL_ENTROPY	[2 : 0]	3'd1
CTRL_IPAD	[2 : 0]	3'd2
CTRL_MSG	[2 : 0]	3'd3
CTRL_OPAD	[2 : 0]	3'd4
CTRL_HMAC	[2 : 0]	3'd5
CTRL_DONE	[2 : 0]	3'd6

Table 1365. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_core.sv — 포트 (13개)

포트	방향	형/폭
clk	input	logic
reset_n	input	logic
zeroize	input	logic
init_cmd	input	logic
next_cmd	input	logic
last_cmd	input	logic
mode_cmd	input	logic
ready	output	logic
tag_valid	output	logic
lfsr_seed	input	logic [LFSR_SEED_SIZE-1 : 0]
key	input	logic [KEY_SIZE-1 : 0]

포트	방향	형/폭
block_msg	input	logic [BLOCK_SIZE-1 : 0]
tag	output	logic [TAG_SIZE-1 : 0]

API — hmac_ctrl.sv

Table 1366. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_ctrl.sv — module / interface / package (1개)

이름	종류
hmac_ctrl	module

Table 1367. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_ctrl.sv — parameter (2개)

파라미터	형/폭	기본값
AHB_DATA_WIDTH		32
AHB_ADDR_WIDTH		32

Table 1368. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_ctrl.sv — 포트 (23개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
cptra_pwrgood	input	wire
cptra_csr_hmac_key	input	logic [CLP_CSR_HMAC_KEY_DWORDS-1:0][31:0]
haddr_i	input	logic [AHB_ADDR_WIDTH-1:0]
hdata_i	input	logic [AHB_DATA_WIDTH-1:0]
hsel_i	input	logic
hwrite_i	input	logic
hready_i	input	logic
htrans_i	input	logic [1:0]
hsize_i	input	logic [2:0]
hresp_o	output	logic
hreadyout_o	output	logic
hrdata_o	output	logic [AHB_DATA_WIDTH-1:0]
kv_read	output	kv_read_t [1:0]
kv_write	output	kv_write_t
kv_rd_resp	input	kv_rd_resp_t [1:0]
kv_wr_resp	input	kv_wr_resp_t
busy_o	output	logic
error_intr	output	logic
notif_intr	output	logic

포트	방향	형/폭
ocp_lock_in_progress	input	logic
debugUnlock_or_scan_mode_switch	input	logic

API — hmac_param_pkg.sv

Table 1369. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_param_pkg.sv — module / interface / package (1개)

이름	종류
hmac_param_pkg	package

Table 1370. `chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_param_pkg.sv` — `localparam` (207H)[illegible]

Table 1371. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_param_pkg.sv — #define / `define 매크로 (1개)

매크로	인자	정의
HMAC_PARAM_PKG		package hmac_param_pkg;

API — hmac_reg.sv**Table 1372. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_reg.sv — module / interface / package (1개)**

이름	종류
hmac_reg	module

Table 1373. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_reg.sv — 포트 (16개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_cpuif_req	input	wire
s_cpuif_req_is_wr	input	wire
s_cpuif_addr	input	wire [11:0]
s_cpuif_wr_data	input	wire [31:0]
s_cpuif_wr_biten	input	wire [31:0]
s_cpuif_req_stall_wr	output	wire
s_cpuif_req_stall_rd	output	wire
s_cpuif_rd_ack	output	wire
s_cpuif_rd_err	output	wire
s_cpuif_rd_data	output	wire [31:0]
s_cpuif_wr_ack	output	wire
s_cpuif_wr_err	output	wire
hwif_in	input	hmac_reg_pkg::hmac_reg__in_t
hwif_out	output	hmac_reg_pkg::hmac_reg__out_t

API — hmac_reg_pkg.sv**Table 1374. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_reg_pkg.sv — module / interface / package (1개)**

이름	종류
hmac_reg_pkg	package

Table 1375. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_reg_pkg.sv — localparam (3개)

상수	형/폭	값
HMAC_REG_DATA_WIDTH		32
HMAC_REG_MIN_ADDR_WIDTH		12
HMAC_REG_ADDR_WIDTH		32'd12

1. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac.sv (583 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-

```

```

cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,

```

```

11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 //
17 // hmac.v
18 // -----
19 // HMAC-512/384 top-level wrapper with 32 bit data access.
20 //
21 //
=====
22 //`include "kv_defines.svh"
23 `include "caliptra_macros.svh"
24 `include "kv_macros.svh"
25 `include "caliptra_prim_assert.sv"
26
27 module hmac
28     import hmac_param_pkg::*;
29     import hmac_reg_pkg::*;
30     import kv_defines_pkg::*;
31     #(
32         ADDR_WIDTH = 32,
33         DATA_WIDTH = 32
34     )(
35         // Clock and reset.
36         input wire          clk,
37         input wire          reset_n,
38         input wire          cptra_pwrgood,
39
40         //csr key
41         input logic [ `CLP_CSR_HMAC_KEY_DWORDS-1:0][31:0]
cptra_csr_hmac_key,
42
43         // Control.
44         input wire          cs,
45         input wire          we,
46
47         // Data ports.
48         input wire [ADDR_WIDTH - 1 : 0] address,
49         input wire [DATA_WIDTH - 1 : 0] write_data,
50         output wire [DATA_WIDTH - 1 : 0] read_data,
51
52         // KV interface
53         output kv_read_t [1:0] kv_read,
54         output kv_write_t kv_write,
55         input kv_rd_resp_t [1:0] kv_rd_resp,
56         input kv_wr_resp_t kv_wr_resp,
57
58         output logic busy_o,
59
60         output wire error_intr,
61         output wire notif_intr,
62         input logic ocp_lock_in_progress,
63         input logic debugunlock_or_scan_mode_switch
64     );
65
66 //-----
67 // Registers including update variables and write enable.
68 //-----
69 reg init_reg;
70 reg next_reg;
71 reg last_reg;
72
73 reg mode_reg;
74
75 reg ready_reg;
76 reg tag_valid_reg;
77
78 localparam BLOCK_NUM_DWORDS = BLOCK_SIZE / DATA_WIDTH;
79 localparam KEY_NUM_DWORDS   = KEY_SIZE / DATA_WIDTH;
80 localparam TAG_NUM_DWORDS   = TAG_SIZE / DATA_WIDTH;
81 localparam SEED_NUM_DWORDS  = ((LFSR_SEED_SIZE - 1) /
DATA_WIDTH) + 1;
82
83 reg [KEY_NUM_DWORDS - 1 : 0][DATA_WIDTH - 1 : 0]
key_reg;
84 reg [BLOCK_NUM_DWORDS - 1 : 0][DATA_WIDTH - 1 : 0]
block_reg;
85 reg [SEED_NUM_DWORDS - 1 : 0][DATA_WIDTH - 1 : 0]
lfsr_seed_reg;
86
87 logic zeroize_reg;
88
89 //-----
90 // Wires.
91 //-----
92 wire [KEY_SIZE-1 : 0]          core_key;
93 wire [BLOCK_SIZE-1 : 0]        core_block;
94 wire                          core_ready;
95 wire [TAG_SIZE-1 : 0]          core_tag;
96 wire                          core_tag_valid;

```

```

97 wire [LFSR_SEED_SIZE-1 : 0]   core_lfsr_seed;
98 reg [TAG_NUM_DWORDS - 1 : 0][DATA_WIDTH - 1 : 0] tag_reg;
99 reg [TAG_NUM_DWORDS - 1 : 0][DATA_WIDTH - 1 : 0] kv_reg;
100
101 reg [TAG_NUM_DWORDS - 1 : 0][DATA_WIDTH - 1 : 0] get_mask;
102
103 hmac_reg_in_t hwif_in;
104 hmac_reg_out_t hwif_out;
105
106 //interface with kv client
107 logic kv_key_write_en;
108 logic [3:0] kv_key_write_offset;
109 logic [31:0] kv_key_write_data;
110 logic kv_block_write_en;
111 logic [4:0] kv_block_write_offset;
112 logic [31:0] kv_block_write_data;
113 //KV Read Data Present
114 logic kv_key_data_present, kv_key_data_present_set;
115 logic [BLOCK_NUM_DWORDS-1:0] block_reg_lock,
block_reg_lock_nxt;
116 logic kv_block_data_present, kv_block_data_present_set;
117 logic kv_data_present, kv_data_present_reset;
118
119 logic dest_keyvault;
120 kv_error_code_e kv_key_error, kv_block_error,
kv_write_error;
121 logic kv_key_ready, kv_key_done;
122 logic kv_block_ready, kv_block_done;
123 logic kv_write_ready, kv_write_done;
124 logic [KV_ENTRY_SIZE_W-1:0] hmac_expected_key_size;
125
126 kv_read_ctrl_reg_t kv_key_read_ctrl_reg;
127 kv_read_ctrl_reg_t kv_block_read_ctrl_reg;
128 kv_write_ctrl_reg_t kv_write_ctrl_reg;
129 kv_read_filter_metrics_t kv_key_read_metrics;
130 kv_read_filter_metrics_t kv_block_read_metrics;
131 kv_write_filter_metrics_t kv_write_metrics;
132 logic core_tag_we;
133
134 logic key_zero_error, key_zero_error_reg,
key_zero_error_edge;
135 logic key_mode_error, key_mode_error_reg,
key_mode_error_edge;
136 logic last_alone_error, last_alone_error_reg,
last_alone_error_edge;
137
138 logic error_flag;
139 logic error_flag_reg;
140 logic error_flag_edge;
141 //-----
142 // Concurrent connectivity for ports etc.
143 //-----
144 assign core_block = {block_reg[00], block_reg[01],
block_reg[02], block_reg[03],
145                     block_reg[04], block_reg[05],
block_reg[06], block_reg[07],
146                     block_reg[08], block_reg[09],
block_reg[10], block_reg[11],
147                     block_reg[12], block_reg[13],
block_reg[14], block_reg[15],
148                     block_reg[16], block_reg[17],
block_reg[18], block_reg[19],
149                     block_reg[20], block_reg[21],
block_reg[22], block_reg[23],
150                     block_reg[24], block_reg[25],
block_reg[26], block_reg[27],
151                     block_reg[28], block_reg[29],
block_reg[30], block_reg[31]};
152
153 assign core_key = {key_reg[00], key_reg[01], key_reg[02],
key_reg[03], key_reg[04], key_reg[05],
154                  key_reg[06], key_reg[07], key_reg[08],
key_reg[09], key_reg[10], key_reg[11],
155                  key_reg[12], key_reg[13], key_reg[14],
key_reg[15]} & get_mask;
156
157 assign core_lfsr_seed = {lfsr_seed_reg[00],
lfsr_seed_reg[01], lfsr_seed_reg[02],
158                        lfsr_seed_reg[03],
lfsr_seed_reg[04], lfsr_seed_reg[05]};
159
160 //rising edge detect on core tag valid
161 assign core_tag_we = (core_tag_valid & ~tag_valid_get) &
~error_flag_reg;
162
163 //-----
164 // core instantiation.
165 //-----
166 hmac_core core(
167     .clk(clk),
168     .reset_n(reset_n),
169     .zeroize(zeroize_reg),
170
171     .init_cmd(init_reg),
172     .next_cmd(next_reg),
173     .last_cmd(last_reg),
174     .mode_cmd(mode_reg),

```



```

175
176     .lfsr_seed(core_lfsr_seed),
177
178     .key(core_key),
179
180     .block_msg(core_block),
181
182     .ready(core_ready),
183     .tag(core_tag),
184     .tag_valid(core_tag_valid)
185 );
186
187 //-----
188 // reg_update
189 //
190 // Update functionality for all registers in the core.
191 // All registers are positive edge triggered with asynchron-
192 ous
193 // active low reset. All registers have write enable.
194 //-----
195 always @ (posedge clk or negedge reset_n)
196 begin : reg_update
197     integer ii;
198
199     if (!reset_n)
200     begin
201         kv_reg        <= '0;
202         tag_reg        <= '0;
203         tag_valid_reg  <= '0;
204         ready_reg      <= '0;
205         block_reg_lock <= '0;
206         kv_key_data_present <= '0;
207         kv_block_data_present <= '0;
208     end
209     else if (zeroize_reg)
210     begin
211         kv_reg        <= '0;
212         tag_reg        <= '0;
213         tag_valid_reg  <= '0;
214         ready_reg      <= '0;
215         block_reg_lock <= '0;
216         kv_key_data_present <= '0;
217         kv_block_data_present <= '0;
218     end
219     else
220     begin
221         tag_valid_reg <= core_tag_valid & ~error_flag_reg;
222         ready_reg     <= core_ready;
223
224         //write to sw register
225         if (core_tag_we & ~(dest_keyvault |
kv_data_present))
226             tag_reg <= core_tag & get_mask;
227         if (core_tag_we & (dest_keyvault | kv_data_present))
228             kv_reg <= core_tag & get_mask;
229
230         block_reg_lock <= block_reg_lock_nxt;
231         kv_key_data_present <= kv_key_data_present_set ?
'1 :
232             kv_data_present_reset ? '0 :
kv_key_data_present;
233         kv_block_data_present <= kv_block_data_present_set ?
'1 :
234             kv_data_present_reset ?
'0 : kv_block_data_present;
235     end
236     end // reg_update
237
238 //HMAC register hardware interfaces
239 always_comb begin
240     //drive resets to register block
241     hwif_in.error_reset_b = cptra_pwrngood;
242     hwif_in.reset_b = reset_n;
243     //drive hardware writeable registers from hmac core
244     hwif_in.HMAC512_NAME[0].NAME.next = HMAC_CORE_NAME[31:0];
245     hwif_in.HMAC512_NAME[1].NAME.next = HMAC_CORE_NAME[63:32];
246     hwif_in.HMAC512_VERSION[0].VERSION.next =
HMAC_CORE_VERSION[31:0];
247     hwif_in.HMAC512_VERSION[1].VERSION.next =
HMAC_CORE_VERSION[63:32];
248     //Only allow setting CSR MODE when ready is set
249     hwif_in.HMAC512_CTRL.CSR_MODE.swwe = ready_reg;
250     hwif_in.HMAC512_CTRL.INIT.swwe = ready_reg;
251     hwif_in.HMAC512_CTRL.NEXT.swwe = ready_reg;
252     hwif_in.HMAC512_CTRL.LAST.swwe = ready_reg;
253     hwif_in.HMAC512_CTRL.MODE.swwe = ready_reg;
254
255     //assign hardware readable registers to drive hmac core
256     //mask the command until kv clients are idle
257
258     init_reg = hwif_out.HMAC512_CTRL.INIT.value & kv_key_ready &
kv_block_ready;
259
260     next_reg = hwif_out.HMAC512_CTRL.NEXT.value & kv_key_ready &
kv_block_ready;
261
262     last_reg = hwif_out.HMAC512_CTRL.LAST.value & kv_key_ready &

```

```

kv_block_ready;
260     zeroize_reg = hwif_out.HMAC512_CTRL.ZEROIZE.value || debu-
gUnlock_or_scan_mode_switch;
261     mode_reg = hwif_out.HMAC512_CTRL.MODE.value;
262
263     //drive hardware writeable registers from hmac core
264     hwif_in.HMAC512_STATUS.READY.next = ready_reg;
265     hwif_in.HMAC512_STATUS.VALID.next = tag_valid_reg;
266     for (int dword=0; dword < TAG_NUM_DWORDS; dword++) begin
267         hwif_in.HMAC512_TAG[dword].TAG.next = tag_reg[(TAG_NUM_D-
WORDS - 1)-dword];
268         hwif_in.HMAC512_TAG[dword].TAG.hwclr = zeroize_reg;
269     end
270     //drive hardware writable registers from key vault
271     for (int dword=0; dword < BLOCK_NUM_DWORDS; dword++)begin
272         hwif_in.HMAC512_BLOCK[dword].BLOCK.we = (kv_block_write_en &
(kv_block_write_offset == dword)) & !(zeroize_reg |
kv_data_present_reset);
273         hwif_in.HMAC512_BLOCK[dword].BLOCK.next = kv_b-
lock_write_data;
274         hwif_in.HMAC512_BLOCK[dword].BLOCK.hwclr = zeroize_reg |
kv_data_present_reset | (kv_block_error != KV_SUCCESS);
275         hwif_in.HMAC512_BLOCK[dword].BLOCK.swwe1 =
block_reg_lock[dword];
276     end
277     for (int dword=0; dword < KEY_NUM_DWORDS; dword++)begin
278         hwif_in.HMAC512_KEY[dword].KEY.we = (kv_key_write_en &
(kv_key_write_offset == dword)) & !(zeroize_reg |
kv_data_present_reset);
279         hwif_in.HMAC512_KEY[dword].KEY.next = kv_key_write_data;
280         hwif_in.HMAC512_KEY[dword].KEY.hwclr = zeroize_reg |
kv_data_present_reset | (kv_key_error != KV_SUCCESS);
281         hwif_in.HMAC512_KEY[dword].KEY.swwe1 =
kv_key_data_present;
282     end
283     //set ready when keyvault isn't busy
284     hwif_in.HMAC512_KV_RD_KEY_STATUS.READY.next = kv_key_ready;
285     hwif_in.HMAC512_KV_RD_BLOCK_STATUS.READY.next = kv_b-
lock_ready;
286     hwif_in.HMAC512_KV_WR_STATUS.READY.next = kv_write_ready;
287     //set error code
288     hwif_in.HMAC512_KV_RD_KEY_STATUS.ERROR.next = kv_key_error;
289     hwif_in.HMAC512_KV_RD_BLOCK_STATUS.ERROR.next = kv_b-
lock_error;
290     hwif_in.HMAC512_KV_WR_STATUS.ERROR.next = kv_write_error;
291     //set valid when fsm is done
292     hwif_in.HMAC512_KV_RD_KEY_STATUS.VALID.hwset = kv_key_done;
293     hwif_in.HMAC512_KV_RD_BLOCK_STATUS.VALID.hwset = kv_b-
lock_done;
294     hwif_in.HMAC512_KV_WR_STATUS.VALID.hwset = kv_write_done;
295     //clear valid when new request is made
296     hwif_in.HMAC512_KV_RD_KEY_STATUS.VALID.hwclr =
kv_key_read_ctrl_reg.read_en;
297     hwif_in.HMAC512_KV_RD_BLOCK_STATUS.VALID.hwclr = kv_b-
lock_read_ctrl_reg.read_en;
298     hwif_in.HMAC512_KV_WR_STATUS.VALID.hwclr =
kv_write_ctrl_reg.write_en;
299     //clear enable when busy
300     hwif_in.HMAC512_KV_RD_KEY_CTRL.read_en.hwclr =
~kv_key_ready;
301     hwif_in.HMAC512_KV_RD_BLOCK_CTRL.read_en.hwclr = ~kv_b-
lock_ready;
302     hwif_in.HMAC512_KV_WR_CTRL.write_en.hwclr = ~kv_write_ready;
303     //assign hardware readable registers to drive hmac core
304     for (int dword=0; dword < KEY_NUM_DWORDS; dword++) begin
305         key_reg[dword] = hwif_out.HMAC512_CTRL.CSR_MODE.value ?
cptra_csr_hmac_key[dword] : hwif_out.HMAC512_KEY[dword].KEY.value;
306     end
307     for (int dword=0; dword < BLOCK_NUM_DWORDS; dword++)begin
308         block_reg[dword] =
hwif_out.HMAC512_BLOCK[dword].BLOCK.value;
309     end
310
311     for (int dword=0; dword < SEED_NUM_DWORDS; dword++)begin
312         lfsr_seed_reg[dword] =
hwif_out.HMAC512_LFSR_SEED[dword].LFSR_SEED.value;
313     end
314 end
315
316 always_comb begin
317     unique case (mode_reg)
318         1'b0 : get_mask = {(HMAC384_TAG_SIZE/DATA_WIDTH)
{ DATA_WIDTH{1'b1} } },
{(HMAC384_TAG_PAD /DATA_WIDTH)
{ DATA_WIDTH{1'b0} } } }; //HMAC384
319         default : get_mask = {TAG_NUM_D-
WORDS{ DATA_WIDTH{1'b1} } }; //HMAC512
320     endcase
321 end
322
323 //set the lock for the part of the block being written by KV
logic
324 //release the lock once init has been seen
325 always_comb begin
326     for (int dword=0; dword< BLOCK_NUM_DWORDS; dword++) begin
327         if (init_reg | next_reg) begin
328             block_reg_lock_nxt[dword] = '0;
329         end
330     end
331     else begin
332         block_reg_lock_nxt[dword] = (kv_block_write_en & (kv_b-

```

```

lock_write_offset == dword)) ? '1' : block_reg_lock[dword];
333     end
334 end
335 end
336
337 // Software write-enables to prevent KV reg manipulation mid-
operation
338 always_comb
hwif_in.HMAC512_KV_RD_KEY_CTRL.read_en.swwe = !
kv_key_data_present && !busy_o;
339 always_comb
hwif_in.HMAC512_KV_RD_KEY_CTRL.read_entry.swwe = !
kv_key_data_present && !busy_o;
340 always_comb
hwif_in.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.swwe = 0; //NA for
keyvault
341 always_comb
hwif_in.HMAC512_KV_RD_KEY_CTRL.rsvd.swwe = 0;
342
343 always_comb
hwif_in.HMAC512_KV_RD_BLOCK_CTRL.read_en.swwe = !kv_b-
lock_data_present && !busy_o;
344 always_comb
hwif_in.HMAC512_KV_RD_BLOCK_CTRL.read_entry.swwe = !kv_b-
lock_data_present && !busy_o;
345 always_comb
hwif_in.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.swwe = 0; //NA for
keyvault
346 always_comb
hwif_in.HMAC512_KV_RD_BLOCK_CTRL.rsvd.swwe = 0;
347
348 // KV write control must be written before HMAC core operation
begins, even though
349 // output isn't written to KV until the end of the operation.
350 // Prevent partial-key attacks by blocking register modifica-
tions during core execution.
351 always_comb
hwif_in.HMAC512_KV_WR_CTRL.write_en.swwe = !busy_o;
352 always_comb
hwif_in.HMAC512_KV_WR_CTRL.write_entry.swwe = !busy_o;
353 always_comb
hwif_in.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.swwe = !busy_o;
354 always_comb
hwif_in.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.swwe = !busy_o;
355 always_comb
hwif_in.HMAC512_KV_WR_CTRL.mldsa_seed_dest_valid.swwe = !busy_o;
356 always_comb
hwif_in.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.swwe = !busy_o;
357 always_comb
hwif_in.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.swwe = !busy_o;
358 always_comb
hwif_in.HMAC512_KV_WR_CTRL.aes_key_dest_valid.swwe = !busy_o;
359 always_comb
hwif_in.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.swwe = !busy_o;
360 always_comb
hwif_in.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.swwe = !busy_o;
361 always_comb
hwif_in.HMAC512_KV_WR_CTRL.dma_data_dest_valid.swwe = !busy_o;
362 always_comb
hwif_in.HMAC512_KV_WR_CTRL.rsvd.swwe = 0;
363
364 //keyvault control reg macros for assigning to struct
365 `CALIPTRA_KV_READ_CTRL_REG2STRUCT(kv_key_read_ctrl_reg,
HMAC512_KV_RD_KEY_CTRL)
366 `CALIPTRA_KV_READ_CTRL_REG2STRUCT(kv_block_read_ctrl_reg,
HMAC512_KV_RD_BLOCK_CTRL)
367 `CALIPTRA_KV_WRITE_CTRL_REG2STRUCT(kv_write_ctrl_reg,
HMAC512_KV_WR_CTRL)
368
369 //Force result into KV reg whenever source came from KV
370 always_comb kv_key_data_present_set =
kv_key_read_ctrl_reg.read_en;
371 always_comb kv_block_data_present_set = kv_b-
lock_read_ctrl_reg.read_en;
372 always_comb kv_data_present = kv_key_data_present | kv_b-
lock_data_present;
373 always_comb kv_data_present_reset = kv_data_present &
core_tag_we;
374
375 // Register block
376 hmac_reg_i hmac_reg (
377     .clk(clk),
378     .rst(1'b0),
379
380     .s_cpuif_req      (cs),
381     .s_cpuif_req_is_wr (we),
382     .s_cpuif_addr      (address[HMAC_REG_ADDR_WIDTH-1:0]),
383     .s_cpuif_wr_data    (write_data),
384     .s_cpuif_wr_biten   ('1'),
385     .s_cpuif_req_stall_wr(),
386     .s_cpuif_req_stall_rd(),
387     .s_cpuif_rd_ack     (),
388     .s_cpuif_rd_err     (),
389     .s_cpuif_rd_data    (read_data),
390     .s_cpuif_wr_ack     (),
391     .s_cpuif_wr_err     (),
392
393     .hwif_in (hwif_in),
394     .hwif_out(hwif_out)
395 );

```

```

396
397 always_comb key_mode_error = kv_key_data_present & (init_reg |
next_reg) & (mode_reg == HMAC512_MODE) & (key_reg[KEY_NUM_DWORDS-1 :
HMAC384_KEY_SIZE/DATA_WIDTH] == '0');
398 always_comb key_zero_error = kv_key_data_present & (init_reg |
next_reg) & (key_reg == '0');
399 always_comb last_alone_error =
hwif_out.HMAC512_CTRL.LAST.value & ~hwif_out.HMAC512_CTRL.INIT.value
& ~hwif_out.HMAC512_CTRL.NEXT.value;
400
401 // last_alone_error is a soft notification: the engine never
started,
402 // so it must NOT join error_flag (which gates core_tag_we and
is
403 // sticky until zeroize). Keep it for the interrupt status bit
only.
404 always_comb error_flag = key_zero_error | key_mode_error |
(kv_key_error == KV_RD_LEN_MISMATCH);
405
406 always_ff @(posedge clk or negedge reset_n)
407 begin : error_detection
408     if(!reset_n) begin
409         error_flag_reg <= 1'b0;
410         key_mode_error_reg <= 1'b0;
411         key_zero_error_reg <= 1'b0;
412         last_alone_error_reg <= 1'b0;
413     end
414     else if(zeroize_reg) begin
415         error_flag_reg <= 1'b0;
416         key_mode_error_reg <= 1'b0;
417         key_zero_error_reg <= 1'b0;
418         last_alone_error_reg <= 1'b0;
419     end
420     else begin
421         if (error_flag)
422             error_flag_reg <= 1'b1;
423         key_mode_error_reg <= key_mode_error;
424         key_zero_error_reg <= key_zero_error;
425         last_alone_error_reg <= last_alone_error;
426     end
427 end // error_detection
428
429 always_comb error_flag_edge = error_flag & (!error_flag_reg);
430 always_comb key_mode_error_edge = key_mode_error & (!
key_mode_error_reg);
431 always_comb key_zero_error_edge = key_zero_error & (!
key_zero_error_reg);
432 always_comb last_alone_error_edge = last_alone_error & (!
last_alone_error_reg);
433
434 //Interrupts hardware interface
435 assign hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset =
core_tag_we;
436 assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.key_mode_error_sts.hwset =
key_mode_error_edge;
437 assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.key_zero_error_sts.hwset =
key_zero_error_edge;
438 assign hwif_in.intr_b-
block_rf.error_internal_intr_r.error2_sts.hwset =
last_alone_error_edge;
439 assign hwif_in.intr_b-
block_rf.error_internal_intr_r.error3_sts.hwset = 1'b0; // TODO
440
441 assign error_intr = hwif_out.intr_b-
block_rf.error_global_intr_r.intr;
442 assign notif_intr = hwif_out.intr_b-
block_rf.notif_global_intr_r.intr;
443
444 //Key Vault Control Modules
445 always_comb begin
446     kv_key_read_metrics.ocp_lock_in_progress =
ocp_lock_in_progress;
447     kv_key_read_metrics.kv_read_dest =
KV_NUM_READ'(1<<KV_DEST_IDX_HMAC_KEY);
448     kv_key_read_metrics.kv_key_entry =
kv_key_read_ctrl_reg.read_entry;
449 end
450
451 //Read Key
452 kv_read_client #(
453     .DATA_WIDTH(KEY_SIZE),
454     .PAD(0),
455     .LEN_CHECK_AT_KEY_USE(1) //HMAC mode_reg (- expected
length) may be
456                                     //programmed after the KV read com-
pletes.
457 )
458 hmac_key_kv_read
459 (
460     .clk(clk),
461     .rst_b(reset_n),
462     .zeroize(zeroize_reg),
463
464     //client control register

```

```

465     .read_ctrl_reg(kv_key_read_ctrl_reg),
466     .read_metrics (kv_key_read_metrics ),
467
468     //interface with kv
469     .kv_read(kv_read[0]),
470     .kv_resp(kv_rd_resp[0]),
471
472     //interface with client
473     .write_en(kv_key_write_en),
474     .write_offset(kv_key_write_offset),
475     .write_data(kv_key_write_data),
476
477     .error_code(kv_key_error),
478     .kv_ready(kv_key_ready),
479     .read_done(kv_key_done),
480     .check_key_size((init_reg | next_reg) &
kv_key_data_present),
481     .expected_key_size(hmac_expected_key_size)
482 );
483
484 //KV key-length-mismatch expected value (HMAC mode is runtime-
selectable).
485 always_comb hmac_expected_key_size = (mode_reg ==
HMAC512_MODE) ? KV_ENTRY_SIZE_W' (HMAC512_KV_LAST_DWORD)
486
487 : KV_ENTRY_SIZE_W' (HMAC384_KV_LAST_DWORD);
488 //Key Vault Control Modules
489 always_comb begin
490     kv_block_read_metrics.ocp_lock_in_progress =
ocp_lock_in_progress;
491     kv_block_read_metrics.kv_read_dest      =
KV_NUM_READ'(1<<KV_DEST_IDX_HMAC_BLOCK);
492     kv_block_read_metrics.kv_key_entry      = kv_b-
lock_read_ctrl_reg.read_entry;
493 end
494
495 //Read Block
496 // Length check disabled on the BLOCK path: the HMAC block is
a message chunk,
497 // not a security-sized key. Legitimate consumers (e.g., OCP
LOCK HEK seed,
498 // OCP_LOCK_HEK_NUM_DWORDS=8) may write KV entries shorter
than the mode's
499 // key size and rely on PAD=1 zero-extension.
500 kv_read_client #(
501     .DATA_WIDTH(BLOCK_SIZE),
502     .HMAC(1),
503     .PAD(1),
504     .LEN_CHECK(0)
505 )
506 hmac_block_kv_read
507 (
508     .clk(clk),
509     .rst_b(reset_n),
510     .zeroize(zeroize_reg),
511
512     //client control register
513     .read_ctrl_reg(kv_block_read_ctrl_reg),
514     .read_metrics(kv_block_read_metrics),
515
516     //interface with kv
517     .kv_read(kv_read[1]),
518     .kv_resp(kv_rd_resp[1]),
519
520     //interface with client
521     .write_en(kv_block_write_en),
522     .write_offset(kv_block_write_offset),
523     .write_data(kv_block_write_data),
524
525     .error_code(kv_block_error),
526     .kv_ready(kv_block_ready),
527     .read_done(kv_block_done),
528     .check_key_size(1'b0),
529     .expected_key_size('0)
530 );
531
532 //write 512 or 384 result based on mode bit
533 logic [$clog2(TAG_SIZE/32):0] num_dwords;
534 always_comb num_dwords = mode_reg ? 'd16 : 'd12;
535
536 //Write to keyvault
537 always_comb begin
538     kv_write_metrics.ocp_lock_in_progress =
ocp_lock_in_progress;
539     kv_write_metrics.kv_data0_present =
kv_key_data_present;
540     kv_write_metrics.kv_data0_entry =
kv_key_read_ctrl_reg.read_entry; // Stable after start-time due to
SWWE
541     kv_write_metrics.kv_data1_present = kv_b-
lock_data_present;
542     kv_write_metrics.kv_data1_entry = kv_b-
lock_read_ctrl_reg.read_entry; // Stable after start-time due to
SWWE
543
544     kv_write_metrics.kv_write_src      = KV_NUM_WRITE'(1 <<
KV_WRITE_IDX_HMAC);
545     kv_write_metrics.kv_write_entry    =
kv_write_ctrl_reg.write_entry;

```

```

545     kv_write_metrics.aes_decrypt_ecb_op    = 1'b0;
546 end
547
548 kv_write_client #(
549     .DATA_WIDTH(TAG_SIZE)
550 )
551 hmac_result_kv_write
552 (
553     .clk(clk),
554     .rst_b(reset_n),
555     .zeroize(zeroize_reg),
556
557     //client control register
558     .write_ctrl_reg(kv_write_ctrl_reg),
559     .write_metrics(kv_write_metrics),
560     .num_dwords(num_dwords),
561
562     //interface with kv
563     .kv_write(kv_write),
564     .kv_resp(kv_wr_resp),
565
566     //interface with client
567     .dest_keyvault(dest_keyvault),
568     .dest_data_avail(core_tag_we),
569     .dest_data(kv_reg),
570
571     .error_code(kv_write_error),
572     .kv_ready(kv_write_ready),
573     .dest_done(kv_write_done)
574 );
575
576 always_comb busy_o = ~kv_write_ready | ~kv_block_ready |
~kv_key_ready | ~core_ready;
577
578 endmodule // hmac
579
580
581 //
=====
582 // EOF hmac.sv
583 //
=====

```

2. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_core.sv (447 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 //
17 // hmac_core.sv
18 // -----
19 // HMAC-SHA-512/384 core with SCA countermeasures.
20 // Single SHA-512 masked core architecture for area optimiza-
tion.
21 //
22 // Commands:
23 //   init_cmd - SHA-init with (K xor ipad), then SHA-next
with first
24 //               message block. If issued together with
last_cmd,
25 //               continues directly to inner-digest capture,
OPAD, and
26 //               HMAC finalization.
27 //   next_cmd - SHA-next with one intermediate message block.
28 //   last_cmd - SHA-next with the final message block, cap-
tures the
29 //               inner digest, then SHA-init with (K xor opad)
and
30 //               SHA-next with {inner_digest, FINAL_PAD} to
produce
31 //               the HMAC tag.
32 //
33 //
=====
34

```

```

35 module hmac_core
36   import hmac_param_pkg::*;
37 (
38   // Clock and reset.
39   input logic      clk,
40   input logic      reset_n,
41   input logic      zeroize,
42
43   // Control.
44   input logic      init_cmd,
45   input logic      next_cmd,
46   input logic      last_cmd,
47   input logic      mode_cmd,
48   output logic      ready,
49   output logic      tag_valid,
50
51   // Data ports.
52   input logic [LFSR_SEED_SIZE-1 : 0] lfsr_seed,
53
54   input logic [KEY_SIZE-1 : 0] key,
55   input logic [BLOCK_SIZE-1 : 0] block_msg,
56   output logic [TAG_SIZE-1 : 0] tag
57 );
58
59 //-----
60 // Internal constant and parameter definitions.
61 //-----
62
63 // IPAD/OPAD and SHA-512 family final-padding constants come
64 // from hmac_param_pkg so the same widths/values are used every-
65 // where.
66 localparam [2 : 0] CTRL_IDLE   = 3'd0;
67 localparam [2 : 0] CTRL_ENTROPY = 3'd1;
68 localparam [2 : 0] CTRL_IPAD   = 3'd2;
69 localparam [2 : 0] CTRL_MSG    = 3'd3;
70 localparam [2 : 0] CTRL_OPAD   = 3'd4;
71 localparam [2 : 0] CTRL_HMAC   = 3'd5;
72 localparam [2 : 0] CTRL_DONE   = 3'd6;
73
74 //-----
75 // Registers including update variables and write enable.
76 //-----
77
78 logic [2 : 0] hmac_ctrl_reg;
79 logic [2 : 0] hmac_ctrl_new;
80 logic      hmac_ctrl_we;
81 logic [2 : 0] hmac_ctrl_ff;
82
83 logic      ready_flag;
84 logic      digest_valid_reg;
85 logic      digest_valid_new;
86 logic      digest_valid_we;
87
88 logic [BLOCK_SIZE-1:0] key_ipadded;
89 logic [BLOCK_SIZE-1:0] key_opadded;
90 logic [BLOCK_SIZE-1:0] HMAC_padded;
91
92 logic      first_round;
93 logic      IPAD_ready;
94 logic      MSG_ready;
95 logic      OPAD_ready;
96 logic      HMAC_ready;
97 logic      ENTROPY_ready;
98 logic [1:0] mode_reg;
99
100 // Latched on entry to MSG flow: 1 if the current message
101 // block is the
102 // final one (so MSG must capture inner_digest and continue
103 // to OPAD).
104 logic      is_last_block;
105
106 // Captured inner-hash digest at the final MSG -> OPAD
107 // transition; used
108 // as data into the HMAC finalization block.
109 logic [TAG_SIZE-1:0] inner_digest_reg;
110 logic      inner_digest_we;
111
112 logic      sha_init;
113 logic      sha_next;
114 logic [BLOCK_SIZE-1 : 0] sha_block;
115 logic      sha_ready;
116 logic [TAG_SIZE-1 : 0] sha_digest;
117 logic      sha_digest_valid;
118
119 // Entropy.
120 logic [LFSR_SEED_SIZE-1 : 0] entropy;
121 logic [LFSR_SEED_SIZE-1 : 0] lfsr_entropy;
122 logic [LFSR_SEED_SIZE-1 : 0] entropy_digest;
123 logic [63 : 0] counter_reg;
124 logic [BLOCK_SIZE-1 : 0] entropy_block;
125 logic      set_entropy;
126
127 //-----

```

```

125 // Concurrent connectivity for ports etc.
126 //-----
127 assign ready      = ready_flag;
128
129 assign tag        = digest_valid_reg ? sha_digest : {TAG_S-
130 IZE{1'b0}};
131 assign tag_valid = digest_valid_reg;
132 //-----
133 // core instantiation.
134 //-----
135 sha512_masked_core u_sha512_core_h1
136 (
137   .clk(clk),
138   .reset_n(reset_n),
139   .zeroize(zeroize),
140
141   .init_cmd(sha_init),
142   .next_cmd(sha_next),
143   .mode(mode_reg),
144
145   .entropy(entropy),
146
147   .block_msg(sha_block),
148
149   .ready(sha_ready),
150   .digest(sha_digest),
151   .digest_valid(sha_digest_valid)
152 );
153
154 // One 32-bit LFSR per dword of entropy (LFSR_SEED_SIZE bits
155 // total).
156 genvar i;
157 generate
158   for (i = 0; i < LFSR_SEED_SIZE/32; i++) begin : gen_lfsr
159     caliptra_prim_lfsr
160     #(
161       .LfsrType("FIB_XNOR"),
162       .LfsrDw(32),
163       .StateOutDw(32)
164     ) caliptra_prim_lfsr_inst_i
165     (
166       .clk_i(clk),
167       .rst_ni(reset_n),
168       .seed_en_i(init_cmd),
169       .seed_i(lfsr_entropy[i*32 +: 32]),
170       .lfsr_en_i(1'b1),
171       .entropy_i('0),
172       .state_o(entropy[i*32 +: 32])
173     );
174   end
175 endgenerate
176 //-----
177 // reg_update
178 //
179 // Update functionality for all registers in the core.
180 // All registers are positive edge triggered with asynchron-
181 // ous
182 // active low reset. All registers have write enable.
183 //-----
184 always_ff @(posedge clk or negedge reset_n)
185   begin : reg_update
186     if (!reset_n)
187       begin
188         digest_valid_reg <= 1'b0;
189         hmac_ctrl_reg <= CTRL_IDLE;
190         hmac_ctrl_ff <= CTRL_IDLE;
191         inner_digest_reg <= '0;
192         is_last_block <= 1'b0;
193       end
194     else if (zeroize)
195       begin
196         digest_valid_reg <= 1'b0;
197         hmac_ctrl_reg <= CTRL_IDLE;
198         hmac_ctrl_ff <= CTRL_IDLE;
199         inner_digest_reg <= '0;
200         is_last_block <= 1'b0;
201       end
202     else
203       begin
204         hmac_ctrl_ff <= hmac_ctrl_reg;
205
206         if (digest_valid_we)
207           digest_valid_reg <= digest_valid_new;
208
209         if (hmac_ctrl_we)
210           hmac_ctrl_reg <= hmac_ctrl_new;
211
212         if (inner_digest_we)
213           inner_digest_reg <= sha_digest;
214
215         // Sample at IDLE: LAST is a modifier on INIT or
216         // command kicked off at IDLE latches is_last_block

```

```

from last_cmd.
216 // This handles all firmware patterns: INIT, INIT|
LAST, NEXT and NEXT|LAST.
217 if (hmac_ctrl_reg == CTRL_IDLE) begin
218     if (init_cmd | next_cmd)
219         is_last_block <= last_cmd;
220     end
221 end
222 end // reg_update
223
224
225 always_ff @(posedge clk or negedge reset_n)
226 begin
227     if (!reset_n)
228         mode_reg <= '0;
229     else if (zeroize)
230         mode_reg <= '0;
231     else begin
232         if (hmac_ctrl_reg == CTRL_IDLE)
233             mode_reg <= {1'b1, mode_cmd}; //hashing algorithm
mode: 00 for SHA512/224, 01 for SHA512/256, 10 for SHA384, 11 for
SHA512
234     end
235 end
236
237 // without zeroize to make it more complex
238 always_ff @(posedge clk or negedge reset_n) begin
239     if (!reset_n)
240         entropy_digest <= '0;
241     else if (set_entropy)
242         entropy_digest <= sha_digest[LFSR_SEED_SIZE-1 : 0];
243     end
244
245 //without zeroize to make it more complex
246 always_ff @(posedge clk or negedge reset_n) begin
247     if (!reset_n)
248         counter_reg <= '0;
249     else
250         counter_reg <= counter_reg + 1;
251     end
252 end
253
254 always_comb lfsr_entropy = entropy_digest ^ lfsr_seed;
255 always_comb entropy_block = {entropy_digest, lfsr_seed,
counter_reg, ENTROPY_PAD};
256
257 //-----
258 // sha_block_fsm
259 //
260 // Logic to select the appropriate SHA input block based on
the current state of the HMAC control FSM.
261 //-----
262 always_comb
263 begin : sha_block_fsm
264     unique case (hmac_ctrl_reg)
265         CTRL_ENTROPY: sha_block = entropy_block;
266         CTRL_IPAD: sha_block = key_ipadded;
267         CTRL_MSG: sha_block = block_msg;
268         CTRL_OPAD: sha_block = key_opadded;
269         CTRL_HMAC: sha_block = HMAC_padded;
270         default: sha_block = '0;
271     endcase
272 end
273
274 //-----
275 // state_logic
276 //
277 // Mux and control for the single SHA core based on FSM
state.
278 //-----
279 always_comb
280 begin : state_logic
281     key_ipadded = {key, {(BLOCK_SIZE-KEY_SIZE){1'b0}}} ^
IPAD;
282     key_opadded = {key, {(BLOCK_SIZE-KEY_SIZE){1'b0}}} ^
OPAD;
283     HMAC_padded = mode_reg[0] ? {inner_digest_reg,
HMAC512_FINAL_PAD}
284         : {in-
ner_digest_reg[TAG_SIZE-1 : HMAC384_TAG_PAD], HMAC384_FINAL_PAD};
285
286     // Defaults
287     sha_init = 1'b0;
288     sha_next = 1'b0;
289
290     first_round = (hmac_ctrl_reg == hmac_ctrl_ff) ? 1'b0 :
1'b1;
291     // IPAD/OPAD finish with sha_init only – digest is not
yet meaningful,
292     IPAD_ready = sha_ready;
293     MSG_ready = sha_ready & sha_digest_valid;
294     OPAD_ready = sha_ready;
295     HMAC_ready = sha_ready & sha_digest_valid;
296     ENTROPY_ready = sha_ready & sha_digest_valid;
297
298     unique case (hmac_ctrl_reg)
299         CTRL_ENTROPY:

```

```

300         if (first_round)
301             begin
302                 sha_init = 1'b1;
303                 ENTROPY_ready = 1'b0;
304             end
305
306         CTRL_IPAD:
307             if (first_round)
308                 begin
309                     sha_init = 1'b1;
310                     IPAD_ready = 1'b0;
311                 end
312
313         CTRL_MSG:
314             if (first_round)
315                 begin
316                     sha_next = 1'b1;
317                     MSG_ready = 1'b0;
318                 end
319
320         CTRL_OPAD:
321             if (first_round)
322                 begin
323                     sha_init = 1'b1;
324                     OPAD_ready = 1'b0;
325                 end
326
327         CTRL_HMAC:
328             if (first_round)
329                 begin
330                     sha_next = 1'b1;
331                     HMAC_ready = 1'b0;
332                 end
333
334         default: begin
335             sha_init = 1'b0;
336             sha_next = 1'b0;
337         end
338     endcase
339 end
340
341 //-----
342 // hmac_ctrl_fsm
343 //
344 // Logic for the state machine controlling the core beha-
viour.
345 //-----
346 always_comb
347 begin : hmac_ctrl_fsm
348     ready_flag = 1'b0;
349     digest_valid_new = 1'b0;
350     digest_valid_we = 1'b0;
351     hmac_ctrl_new = CTRL_IDLE;
352     hmac_ctrl_we = 1'b0;
353     inner_digest_we = 1'b0;
354     set_entropy = 1'b0;
355
356     unique case (hmac_ctrl_reg)
357         CTRL_IDLE:
358             begin
359                 ready_flag = 1'b1;
360
361                 if (init_cmd)
362                     begin
363                         digest_valid_new = 1'b0;
364                         digest_valid_we = 1'b1;
365                         hmac_ctrl_new = CTRL_ENTROPY;
366                         hmac_ctrl_we = 1'b1;
367                     end
368                 else if (next_cmd)
369                     begin
370                         digest_valid_new = 1'b0;
371                         digest_valid_we = 1'b1;
372                         hmac_ctrl_new = CTRL_MSG;
373                         hmac_ctrl_we = 1'b1;
374                     end
375                 end
376
377         CTRL_ENTROPY:
378             if (ENTROPY_ready)
379                 begin
380                     set_entropy = 1'b1;
381                     hmac_ctrl_new = CTRL_IPAD;
382                     hmac_ctrl_we = 1'b1;
383                 end
384
385         CTRL_IPAD:
386             if (IPAD_ready)
387                 begin
388                     hmac_ctrl_new = CTRL_MSG;
389                     hmac_ctrl_we = 1'b1;
390                 end
391
392         CTRL_MSG:
393             if (MSG_ready)
394                 begin
395                     if (is_last_block)
396                         begin
397                             inner_digest_we = 1'b1;

```



```

398         hmac_ctrl_new = CTRL_OPAD;
399     end
400     else
401         begin
402             digest_valid_new = 1'b1;
403             digest_valid_we = 1'b1;
404             hmac_ctrl_new = CTRL_IDLE;
405         end
406         hmac_ctrl_we = 1'b1;
407     end
408
409     CTRL_OPAD:
410     if (OPAD_ready)
411     begin
412         hmac_ctrl_new = CTRL_HMAC;
413         hmac_ctrl_we = 1'b1;
414     end
415
416     CTRL_HMAC:
417     if (HMAC_ready)
418     begin
419         hmac_ctrl_new = CTRL_DONE;
420         hmac_ctrl_we = 1'b1;
421     end
422
423     CTRL_DONE:
424     begin
425         digest_valid_new = 1'b1;
426         digest_valid_we = 1'b1;
427         hmac_ctrl_new = CTRL_IDLE;
428         hmac_ctrl_we = 1'b1;
429     end
430
431     default:
432     begin
433         ready_flag = 1'b0;
434         digest_valid_new = 1'b0;
435         digest_valid_we = 1'b0;
436         hmac_ctrl_new = CTRL_IDLE;
437         hmac_ctrl_we = 1'b0;
438     end
439
440     endcase // case (hmac_ctrl_reg)
441     end // hmac_ctrl_fsm
442
443 endmodule // hmac_core
444
445 //
=====
446 // EOF hmac_core.sv
447 //
=====

```

3. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_ctrl.sv (143 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
4 // you may not use this file except in compliance with the Li-
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
10 // distributed under the License is distributed on an "AS IS"
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
12 // See the License for the specific language governing permis-
13 // limitations under the License.
14 //
15 //
=====
16 //
17 // hmac_ctrl.sv
18 // -----
19 // HMAC controller for the AHB_lite interface.
20 //
21 //
22 //
23 //
=====
24
25
26 `include "caliptra_macros.svh"
27 `include "kv_macros.svh"
28
29 module hmac_ctrl
30     import kv_defines_pkg::*;
31     #(
32         parameter AHB_DATA_WIDTH = 32,

```

```

33     parameter AHB_ADDR_WIDTH = 32
34 )
35 (
36     // Clock and reset.
37     input wire      clk,
38     input wire      reset_n,
39     input wire      cptra_pwrgood,
40
41     //csr key
42     input logic [ `CLP_CSR_HMAC_KEY_DWORDS-1:0][31:0]
cptra_csr_hmac_key,
43
44     // from SLAVES PORT
45     input logic [AHB_ADDR_WIDTH-1:0] haddr_i,
46     input logic [AHB_DATA_WIDTH-1:0] hdata_i,
47     input logic hsel_i,
48     input logic hwrite_i,
49     input logic hready_i,
50     input logic [1:0] htrans_i,
51     input logic [2:0] hsize_i,
52
53     output logic hresp_o,
54     output logic hreadyout_o,
55     output logic [AHB_DATA_WIDTH-1:0] hrdata_o,
56
57     // kv interface
58     output kv_read_t [1:0] kv_read,
59     output kv_write_t kv_write,
60     input kv_rd_resp_t [1:0] kv_rd_resp,
61     input kv_wr_resp_t kv_wr_resp,
62
63     output logic busy_o,
64
65     // Interrupt
66     output logic error_intr,
67     output logic notif_intr,
68     input logic ocp_lock_in_progress,
69     input logic debugUnlock_or_scan_mode_switch
70 );
71
72 //-----
73 // hmac
74 //-----
75
76 logic      hmac_cs;
77 logic      hmac_we;
78 logic [AHB_ADDR_WIDTH-1 : 0] hmac_address;
79 logic [31 : 0] hmac_write_data;
80 logic [31 : 0] hmac_read_data;
81
82 hmac #(
83     .ADDR_WIDTH (AHB_ADDR_WIDTH),
84     .DATA_WIDTH (32)
85 ) hmac_inst
86 (
87     .clk(clk),
88     .reset_n(reset_n),
89     .cptra_pwrgood(cptra_pwrgood),
90     .cptra_csr_hmac_key(cptra_csr_hmac_key),
91     .cs(hmac_cs),
92     .we(hmac_we),
93     .address(hmac_address),
94     .write_data(hmac_write_data),
95     .read_data(hmac_read_data),
96     .kv_read(kv_read),
97     .kv_write(kv_write),
98     .kv_rd_resp(kv_rd_resp),
99     .kv_wr_resp(kv_wr_resp),
100     .error_intr(error_intr),
101     .notif_intr(notif_intr),
102     .busy_o(busy_o),
103     .ocp_lock_in_progress(ocp_lock_in_progress),
104     .debugUn-
lock_or_scan_mode_switch(debugUnlock_or_scan_mode_switch)
105 );
106
107 //instantiate ahb lite module
108 ahb_slv_sif #(
109     .AHB_ADDR_WIDTH(AHB_ADDR_WIDTH),
110     .AHB_DATA_WIDTH(AHB_DATA_WIDTH),
111     .CLIENT_DATA_WIDTH(32)
112 ) ahb_slv_sif_uut
113 (
114     //AMBA AHB Lite INF
115     .hclk(clk),
116     .hreset_n(reset_n),
117     .haddr_i(haddr_i),
118     .hdata_i(hdata_i),
119     .hsel_i(hsel_i),
120     .hwrite_i(hwrite_i),
121     .hready_i(hready_i),
122     .htrans_i(htrans_i),
123     .hsize_i(hsize_i),
124
125     .hresp_o(hresp_o),
126     .hreadyout_o(hreadyout_o),
127     .hrdata_o(hrdata_o),
128
129     //COMPONENT INF

```

```

130 .dv(hmac_cs),
131 .hld(1'b0), //no holes from hmac
132 .err(1'b0), //no errors from hmac
133 .write(hmac_we),
134 .wdata(hmac_write_data),
135 .addr(hmac_address),
136
137 .rdata(hmac_read_data)
138 );
139
140
141
142
143 endmodule

```

4. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_param_pkg.sv

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
4 // cense");
5 // you may not use this file except in compliance with the Li-
6 // cense.
7 // You may obtain a copy of the License at
8 //
9 // http://www.apache.org/licenses/LICENSE-2.0
10 //
11 // Unless required by applicable law or agreed to in writing,
12 // distributed under the License is distributed on an "AS IS"
13 // BASIS,
14 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
15 // press or implied.
16 // See the License for the specific language governing permis-
17 // sions and
18 // limitations under the License.
19 //
20 //
21 //
22 //
23 //
24 //
25 //
26 //
27 //
28 //
29 //
30 //
31 //
32 //
33 //
34 //
35 //
36 //
37 //
38 //
39 //
40 //
41 //
42 //
43 //
44 //
45 //
46 //
47 //
48 //
49 //
50 //
51 //
52 //
53 //
54 //
55 //
56 //
57 //
58 //
59 //
60 //
61 //
62 //
63 //
64 //
65 //
66 //
67 //
68 //
69 //
70 //
71 //
72 //
73 //
74 //
75 //
76 //
77 //
78 //
79 //
80 //
81 //
82 //
83 //
84 //
85 //
86 //
87 //
88 //
89 //
90 //
91 //
92 //
93 //
94 //
95 //
96 //
97 //
98 //
99 //
100 //
101 //
102 //
103 //
104 //
105 //
106 //
107 //
108 //
109 //
110 //
111 //
112 //
113 //
114 //
115 //
116 //
117 //
118 //
119 //
120 //
121 //
122 //
123 //
124 //
125 //
126 //
127 //
128 //
129 //
130 //
131 //
132 //
133 //
134 //
135 //
136 //
137 //
138 //
139 //
140 //
141 //
142 //
143 //
144 //
145 //
146 //
147 //
148 //
149 //
150 //
151 //
152 //
153 //
154 //
155 //
156 //
157 //
158 //
159 //
160 //
161 //
162 //
163 //
164 //
165 //
166 //
167 //
168 //
169 //
170 //
171 //
172 //
173 //
174 //
175 //
176 //
177 //
178 //
179 //
180 //
181 //
182 //
183 //
184 //
185 //
186 //
187 //
188 //
189 //
190 //
191 //
192 //
193 //
194 //
195 //
196 //
197 //
198 //
199 //
200 //
201 //
202 //
203 //
204 //
205 //
206 //
207 //
208 //
209 //
210 //
211 //
212 //
213 //
214 //
215 //
216 //
217 //
218 //
219 //
220 //
221 //
222 //
223 //
224 //
225 //
226 //
227 //
228 //
229 //
230 //
231 //
232 //
233 //
234 //
235 //
236 //
237 //
238 //
239 //
240 //
241 //
242 //
243 //
244 //
245 //
246 //
247 //
248 //
249 //
250 //
251 //
252 //
253 //
254 //
255 //
256 //
257 //
258 //
259 //
260 //
261 //
262 //
263 //
264 //
265 //
266 //
267 //
268 //
269 //
270 //
271 //
272 //
273 //
274 //
275 //
276 //
277 //
278 //
279 //
280 //
281 //
282 //
283 //
284 //
285 //
286 //
287 //
288 //
289 //
290 //
291 //
292 //
293 //
294 //
295 //
296 //
297 //
298 //
299 //
300 //
301 //
302 //
303 //
304 //
305 //
306 //
307 //
308 //
309 //
310 //
311 //
312 //
313 //
314 //
315 //
316 //
317 //
318 //
319 //
320 //
321 //
322 //
323 //
324 //
325 //
326 //
327 //
328 //
329 //
330 //
331 //
332 //
333 //
334 //
335 //
336 //
337 //
338 //
339 //
340 //
341 //
342 //
343 //
344 //
345 //
346 //
347 //
348 //
349 //
350 //
351 //
352 //
353 //
354 //
355 //
356 //
357 //
358 //
359 //
360 //
361 //
362 //
363 //
364 //
365 //
366 //
367 //
368 //
369 //
370 //
371 //
372 //
373 //
374 //
375 //
376 //
377 //
378 //
379 //
380 //
381 //
382 //
383 //
384 //
385 //
386 //
387 //
388 //
389 //
390 //
391 //
392 //
393 //
394 //
395 //
396 //
397 //
398 //
399 //
400 //
401 //
402 //
403 //
404 //
405 //
406 //
407 //
408 //
409 //
410 //
411 //
412 //
413 //
414 //
415 //
416 //
417 //
418 //
419 //
420 //
421 //
422 //
423 //
424 //
425 //
426 //
427 //
428 //
429 //
430 //
431 //
432 //
433 //
434 //
435 //
436 //
437 //
438 //
439 //
440 //
441 //
442 //
443 //
444 //
445 //
446 //
447 //
448 //
449 //
450 //
451 //
452 //
453 //
454 //
455 //
456 //
457 //
458 //
459 //
460 //
461 //
462 //
463 //
464 //
465 //
466 //
467 //
468 //
469 //
470 //
471 //
472 //
473 //
474 //
475 //
476 //
477 //
478 //
479 //
480 //
481 //
482 //
483 //
484 //
485 //
486 //
487 //
488 //
489 //
490 //
491 //
492 //
493 //
494 //
495 //
496 //
497 //
498 //
499 //
500 //
501 //
502 //
503 //
504 //
505 //
506 //
507 //
508 //
509 //
510 //
511 //
512 //
513 //
514 //
515 //
516 //
517 //
518 //
519 //
520 //
521 //
522 //
523 //
524 //
525 //
526 //
527 //
528 //
529 //
530 //
531 //
532 //
533 //
534 //
535 //
536 //
537 //
538 //
539 //
540 //
541 //
542 //
543 //
544 //
545 //
546 //
547 //
548 //
549 //
550 //
551 //
552 //
553 //
554 //
555 //
556 //
557 //
558 //
559 //
560 //
561 //
562 //
563 //
564 //
565 //
566 //
567 //
568 //
569 //
570 //
571 //
572 //
573 //
574 //
575 //
576 //
577 //
578 //
579 //
580 //
581 //
582 //
583 //
584 //
585 //
586 //
587 //
588 //
589 //
590 //
591 //
592 //
593 //
594 //
595 //
596 //
597 //
598 //
599 //
600 //
601 //
602 //
603 //
604 //
605 //
606 //
607 //
608 //
609 //
610 //
611 //
612 //
613 //
614 //
615 //
616 //
617 //
618 //
619 //
620 //
621 //
622 //
623 //
624 //
625 //
626 //
627 //
628 //
629 //
630 //
631 //
632 //
633 //
634 //
635 //
636 //
637 //
638 //
639 //
640 //
641 //
642 //
643 //
644 //
645 //
646 //
647 //
648 //
649 //
650 //
651 //
652 //
653 //
654 //
655 //
656 //
657 //
658 //
659 //
660 //
661 //
662 //
663 //
664 //
665 //
666 //
667 //
668 //
669 //
670 //
671 //
672 //
673 //
674 //
675 //
676 //
677 //
678 //
679 //
680 //
681 //
682 //
683 //
684 //
685 //
686 //
687 //
688 //
689 //
690 //
691 //
692 //
693 //
694 //
695 //
696 //
697 //
698 //
699 //
700 //
701 //
702 //
703 //
704 //
705 //
706 //
707 //
708 //
709 //
710 //
711 //
712 //
713 //
714 //
715 //
716 //
717 //
718 //
719 //
720 //
721 //
722 //
723 //
724 //
725 //
726 //
727 //
728 //
729 //
730 //
731 //
732 //
733 //
734 //
735 //
736 //
737 //
738 //
739 //
740 //
741 //
742 //
743 //
744 //
745 //
746 //
747 //
748 //
749 //
750 //
751 //
752 //
753 //
754 //
755 //
756 //
757 //
758 //
759 //
760 //
761 //
762 //
763 //
764 //
765 //
766 //
767 //
768 //
769 //
770 //
771 //
772 //
773 //
774 //
775 //
776 //
777 //
778 //
779 //
780 //
781 //
782 //
783 //
784 //
785 //
786 //
787 //
788 //
789 //
790 //
791 //
792 //
793 //
794 //
795 //
796 //
797 //
798 //
799 //
800 //
801 //
802 //
803 //
804 //
805 //
806 //
807 //
808 //
809 //
810 //
811 //
812 //
813 //
814 //
815 //
816 //
```

[illegible]

5. **chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_reg.sv** (2,408 lines)

```

1 // Generated by PeakRDL-regblock - A free and open-source Sys-
temVerilog generator
2 // https://github.com/SystemRDL/PeakRDL-regblock
3
4 module hmac_reg (
5     input wire clk,
6     input wire rst,
7
8     input wire s_cpuif_req,
9     input wire s_cpuif_req_is_wr,
10    input wire [11:0] s_cpuif_addr,
11    input wire [31:0] s_cpuif_wr_data,
12    input wire [31:0] s_cpuif_wr_biten,
13    output wire s_cpuif_req_stall_rd,
14    output wire s_cpuif_req_stall_wr,
15    output wire s_cpuif_rd_ack,
16    output wire s_cpuif_rd_err,
17    output wire [31:0] s_cpuif_rd_data,
18    output wire s_cpuif_wr_ack,
19    output wire s_cpuif_wr_err,
20
21    input hmac_reg_pkg::hmac_reg_in_t hwif_in,
22    output hmac_reg_pkg::hmac_reg_out_t hwif_out
23 );
24
25 //-----
26 // CPU Bus interface logic
27 //-----
28
29 logic cpuif_req;
30 logic cpuif_req_is_wr;
31 logic [11:0] cpuif_addr;
32 logic [31:0] cpuif_wr_data;
33 logic [31:0] cpuif_wr_biten;
34 logic cpuif_req_stall_wr;
35 logic cpuif_req_stall_rd;
36
37 logic cpuif_rd_ack;
38 logic cpuif_rd_err;
39 logic [31:0] cpuif_rd_data;
40
41 logic cpuif_wr_ack;
42 logic cpuif_wr_err;
43
44 assign cpuif_req = s_cpuif_req;
45 assign cpuif_req_is_wr = s_cpuif_req_is_wr;
46 assign cpuif_addr = s_cpuif_addr;
47 assign cpuif_wr_data = s_cpuif_wr_data;
48 assign cpuif_wr_biten = s_cpuif_wr_biten;
49 assign s_cpuif_req_stall_wr = cpuif_req_stall_wr;
50 assign s_cpuif_req_stall_rd = cpuif_req_stall_rd;
51 assign s_cpuif_rd_ack = cpuif_rd_ack;
52 assign s_cpuif_rd_err = cpuif_rd_err;
53 assign s_cpuif_rd_data = cpuif_rd_data;
54 assign s_cpuif_wr_ack = cpuif_wr_ack;
55 assign s_cpuif_wr_err = cpuif_wr_err;
56
57 logic cpuif_req_masked;
58
59 // Read & write latencies are balanced. Stalls not re-
60 quired
61 assign cpuif_req_stall_rd = '0;
62 assign cpuif_req_stall_wr = '0;
63 assign cpuif_req_masked = cpuif_req
64                        & !(cpuif_req_is_wr &
65                           cpuif_req_stall_rd)
66                        & !(cpuif_req_is_wr &
67                           cpuif_req_stall_wr);
68
69 //-----
70 // Address Decode
71 //-----
72
73 typedef struct packed{
74     logic [2-1:0]HMAC512_NAME;
75 }

```



```

70 logic [2-1:0]HMAC512_VERSION;
71 logic HMAC512_CTRL;
72 logic HMAC512_STATUS;
73 logic [16-1:0]HMAC512_KEY;
74 logic [32-1:0]HMAC512_BLOCK;
75 logic [16-1:0]HMAC512_TAG;
76 logic [6-1:0]HMAC512_LFSR_SEED;
77 logic HMAC512_KV_RD_KEY_CTRL;
78 logic HMAC512_KV_RD_KEY_STATUS;
79 logic HMAC512_KV_RD_BLOCK_CTRL;
80 logic HMAC512_KV_RD_BLOCK_STATUS;
81 logic HMAC512_KV_WR_CTRL;
82 logic HMAC512_KV_WR_STATUS;
83 struct packed{
84     logic global_intr_en_r;
85     logic error_intr_en_r;
86     logic notif_intr_en_r;
87     logic error_global_intr_r;
88     logic notif_global_intr_r;
89     logic error_internal_intr_r;
90     logic notif_internal_intr_r;
91     logic error_intr_trig_r;
92     logic notif_intr_trig_r;
93     logic key_mode_error_intr_count_r;
94     logic key_zero_error_intr_count_r;
95     logic error2_intr_count_r;
96     logic error3_intr_count_r;
97     logic notif_cmd_done_intr_count_r;
98     logic key_mode_error_intr_count_incr_r;
99     logic key_zero_error_intr_count_incr_r;
100    logic error2_intr_count_incr_r;
101    logic error3_intr_count_incr_r;
102    logic notif_cmd_done_intr_count_incr_r;
103 } intr_block_rf;
104 } decoded_reg_strb_t;
105 decoded_reg_strb_t decoded_reg_strb;
106 logic decoded_req;
107 logic decoded_req_is_wr;
108 logic [31:0] decoded_wr_data;
109 logic [31:0] decoded_wr_biten;
110
111 always_comb begin
112     for(int i0=0; i0<2; i0++) begin
113         decoded_reg_strb.HMAC512_NAME[i0] =
114         cpuif_req_masked & (cpuif_addr == 12'h0 + i0*12'h4);
115     end
116     for(int i0=0; i0<2; i0++) begin
117         decoded_reg_strb.HMAC512_VERSION[i0] =
118         cpuif_req_masked & (cpuif_addr == 12'h8 + i0*12'h4);
119     end
120     decoded_reg_strb.HMAC512_CTRL = cpuif_req_masked &
121     (cpuif_addr == 12'h10);
122     decoded_reg_strb.HMAC512_STATUS = cpuif_req_masked &
123     (cpuif_addr == 12'h18);
124     for(int i0=0; i0<16; i0++) begin
125         decoded_reg_strb.HMAC512_KEY[i0] =
126         cpuif_req_masked & (cpuif_addr == 12'h40 + i0*12'h4);
127     end
128     for(int i0=0; i0<32; i0++) begin
129         decoded_reg_strb.HMAC512_BLOCK[i0] =
130         cpuif_req_masked & (cpuif_addr == 12'h80 + i0*12'h4);
131     end
132     for(int i0=0; i0<16; i0++) begin
133         decoded_reg_strb.HMAC512_TAG[i0] =
134         cpuif_req_masked & (cpuif_addr == 12'h100 + i0*12'h4);
135     end
136     for(int i0=0; i0<6; i0++) begin
137         decoded_reg_strb.HMAC512_LFSR_SEED[i0] =
138         cpuif_req_masked & (cpuif_addr == 12'h140 + i0*12'h4);
139     end
140     decoded_reg_strb.HMAC512_KV_RD_KEY_CTRL =
141     cpuif_req_masked & (cpuif_addr == 12'h600);
142     decoded_reg_strb.HMAC512_KV_RD_KEY_STATUS =
143     cpuif_req_masked & (cpuif_addr == 12'h604);
144     decoded_reg_strb.HMAC512_KV_RD_BLOCK_CTRL =
145     cpuif_req_masked & (cpuif_addr == 12'h608);
146     decoded_reg_strb.HMAC512_KV_RD_BLOCK_STATUS =
147     cpuif_req_masked & (cpuif_addr == 12'h60c);
148     decoded_reg_strb.HMAC512_KV_WR_CTRL = cpuif_req_masked &
149     (cpuif_addr == 12'h610);
150     decoded_reg_strb.HMAC512_KV_WR_STATUS =
151     cpuif_req_masked & (cpuif_addr == 12'h614);
152     decoded_reg_strb.intr_block_rf.global_intr_en_r =
153     cpuif_req_masked & (cpuif_addr == 12'h800);
154     decoded_reg_strb.intr_block_rf.error_intr_en_r =
155     cpuif_req_masked & (cpuif_addr == 12'h804);
156     decoded_reg_strb.intr_block_rf.notif_intr_en_r =
157     cpuif_req_masked & (cpuif_addr == 12'h808);
158     decoded_reg_strb.intr_block_rf.error_global_intr_r =
159     cpuif_req_masked & (cpuif_addr == 12'h80c);
160     decoded_reg_strb.intr_block_rf.notif_global_intr_r =
161     cpuif_req_masked & (cpuif_addr == 12'h810);
162     decoded_reg_strb.intr_block_rf.error_internal_intr_r =
163     cpuif_req_masked & (cpuif_addr == 12'h814);
164     decoded_reg_strb.intr_block_rf.notif_internal_intr_r =
165     cpuif_req_masked & (cpuif_addr == 12'h818);
166     decoded_reg_strb.intr_block_rf.error_intr_trig_r =
167     cpuif_req_masked & (cpuif_addr == 12'h81c);

```

```

146     decoded_reg_strb.intr_block_rf.notif_intr_trig_r =
147     cpuif_req_masked & (cpuif_addr == 12'h820);
148     de-
149     coded_reg_strb.intr_block_rf.key_mode_error_intr_count_r =
150     cpuif_req_masked & (cpuif_addr == 12'h900);
151     de-
152     coded_reg_strb.intr_block_rf.key_zero_error_intr_count_r =
153     cpuif_req_masked & (cpuif_addr == 12'h904);
154     de-
155     coded_reg_strb.intr_block_rf.error2_intr_count_r =
156     cpuif_req_masked & (cpuif_addr == 12'h908);
157     de-
158     coded_reg_strb.intr_block_rf.error3_intr_count_r =
159     cpuif_req_masked & (cpuif_addr == 12'h90c);
160     de-
161     coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r =
162     cpuif_req_masked & (cpuif_addr == 12'h980);
163     de-
164     coded_reg_strb.intr_block_rf.key_mode_error_intr_count_incr_r =
165     cpuif_req_masked & (cpuif_addr == 12'ha00);
166     de-
167     coded_reg_strb.intr_block_rf.key_zero_error_intr_count_incr_r =
168     cpuif_req_masked & (cpuif_addr == 12'ha04);
169     de-
170     coded_reg_strb.intr_block_rf.error2_intr_count_incr_r =
171     cpuif_req_masked & (cpuif_addr == 12'ha08);
172     de-
173     coded_reg_strb.intr_block_rf.error3_intr_count_incr_r =
174     cpuif_req_masked & (cpuif_addr == 12'ha0c);
175     de-
176     coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_incr_r =
177     cpuif_req_masked & (cpuif_addr == 12'ha10);
178     end
179
180 // Pass down signals to next stage
181 assign decoded_req = cpuif_req_masked;
182 assign decoded_req_is_wr = cpuif_req_is_wr;
183 assign decoded_wr_data = cpuif_wr_data;
184 assign decoded_wr_biten = cpuif_wr_biten;
185
186 //-----
187 // Field logic
188 //-----
189
190 typedef struct packed{
191     struct packed{
192         logic next;
193         logic load_next;
194     } INIT;
195     struct packed{
196         logic next;
197         logic load_next;
198     } NEXT;
199     struct packed{
200         logic next;
201         logic load_next;
202     } ZEROIZE;
203     struct packed{
204         logic next;
205         logic load_next;
206     } MODE;
207     struct packed{
208         logic next;
209         logic load_next;
210     } CSR_MODE;
211     struct packed{
212         logic next;
213         logic load_next;
214     } LAST;
215     struct packed{
216         logic next;
217         logic load_next;
218     } Reserved;
219     } HMAC512_CTRL;
220     struct packed{
221         struct packed{
222             logic [31:0] next;
223             logic load_next;
224         } KEY;
225     } [16-1:0]HMAC512_KEY;
226     struct packed{
227         struct packed{
228             logic [31:0] next;
229             logic load_next;
230         } BLOCK;
231     } [32-1:0]HMAC512_BLOCK;
232     struct packed{
233         struct packed{
234             logic [31:0] next;
235             logic load_next;
236         } TAG;
237     } [16-1:0]HMAC512_TAG;
238     struct packed{
239         struct packed{
240             logic [31:0] next;
241             logic load_next;
242         } LFSR_SEED;
243     } [6-1:0]HMAC512_LFSR_SEED;
244     struct packed{
245         struct packed{
246             logic next;

```

```

226         logic load_next;
227     } read_en;
228     struct packed{
229         logic [4:0] next;
230         logic load_next;
231     } read_entry;
232     struct packed{
233         logic next;
234         logic load_next;
235     } pcr_hash_extend;
236     struct packed{
237         logic [24:0] next;
238         logic load_next;
239     } rsvd;
240 } HMAC512_KV_RD_KEY_CTRL;
241 struct packed{
242     struct packed{
243         logic next;
244         logic load_next;
245     } VALID;
246 } HMAC512_KV_RD_KEY_STATUS;
247 struct packed{
248     struct packed{
249         logic next;
250         logic load_next;
251     } read_en;
252     struct packed{
253         logic [4:0] next;
254         logic load_next;
255     } read_entry;
256     struct packed{
257         logic next;
258         logic load_next;
259     } pcr_hash_extend;
260     struct packed{
261         logic [24:0] next;
262         logic load_next;
263     } rsvd;
264 } HMAC512_KV_RD_BLOCK_CTRL;
265 struct packed{
266     struct packed{
267         logic next;
268         logic load_next;
269     } VALID;
270 } HMAC512_KV_RD_BLOCK_STATUS;
271 struct packed{
272     struct packed{
273         logic next;
274         logic load_next;
275     } write_en;
276     struct packed{
277         logic [4:0] next;
278         logic load_next;
279     } write_entry;
280     struct packed{
281         logic next;
282         logic load_next;
283     } hmac_key_dest_valid;
284     struct packed{
285         logic next;
286         logic load_next;
287     } hmac_block_dest_valid;
288     struct packed{
289         logic next;
290         logic load_next;
291     } mldsa_seed_dest_valid;
292     struct packed{
293         logic next;
294         logic load_next;
295     } ecc_pkey_dest_valid;
296     struct packed{
297         logic next;
298         logic load_next;
299     } ecc_seed_dest_valid;
300     struct packed{
301         logic next;
302         logic load_next;
303     } aes_key_dest_valid;
304     struct packed{
305         logic next;
306         logic load_next;
307     } mlkem_seed_dest_valid;
308     struct packed{
309         logic next;
310         logic load_next;
311     } mlkem_msg_dest_valid;
312     struct packed{
313         logic next;
314         logic load_next;
315     } dma_data_dest_valid;
316     struct packed{
317         logic [16:0] next;
318         logic load_next;
319     } rsvd;
320 } HMAC512_KV_WR_CTRL;
321 struct packed{
322     struct packed{
323         logic next;
324         logic load_next;
325     } VALID;
326 } HMAC512_KV_WR_STATUS;

```

```

327 struct packed{
328     struct packed{
329         struct packed{
330             logic next;
331             logic load_next;
332         } error_en;
333         struct packed{
334             logic next;
335             logic load_next;
336         } notif_en;
337     } global_intr_en_r;
338     struct packed{
339         struct packed{
340             logic next;
341             logic load_next;
342         } key_mode_error_en;
343         struct packed{
344             logic next;
345             logic load_next;
346         } key_zero_error_en;
347         struct packed{
348             logic next;
349             logic load_next;
350         } error2_en;
351         struct packed{
352             logic next;
353             logic load_next;
354         } error3_en;
355     } error_intr_en_r;
356     struct packed{
357         struct packed{
358             logic next;
359             logic load_next;
360         } notif_cmd_done_en;
361     } notif_intr_en_r;
362     struct packed{
363         struct packed{
364             logic next;
365             logic load_next;
366         } agg_sts;
367     } error_global_intr_r;
368     struct packed{
369         struct packed{
370             logic next;
371             logic load_next;
372         } agg_sts;
373     } notif_global_intr_r;
374     struct packed{
375         struct packed{
376             logic next;
377             logic load_next;
378         } key_mode_error_sts;
379         struct packed{
380             logic next;
381             logic load_next;
382         } key_zero_error_sts;
383         struct packed{
384             logic next;
385             logic load_next;
386         } error2_sts;
387         struct packed{
388             logic next;
389             logic load_next;
390         } error3_sts;
391     } error_internal_intr_r;
392     struct packed{
393         struct packed{
394             logic next;
395             logic load_next;
396         } notif_cmd_done_sts;
397     } notif_internal_intr_r;
398     struct packed{
399         struct packed{
400             logic next;
401             logic load_next;
402         } key_mode_error_trig;
403         struct packed{
404             logic next;
405             logic load_next;
406         } key_zero_error_trig;
407         struct packed{
408             logic next;
409             logic load_next;
410         } error2_trig;
411         struct packed{
412             logic next;
413             logic load_next;
414         } error3_trig;
415     } error_intr_trig_r;
416     struct packed{
417         struct packed{
418             logic next;
419             logic load_next;
420         } notif_cmd_done_trig;
421     } notif_intr_trig_r;
422     struct packed{
423         struct packed{
424             logic [31:0] next;
425             logic load_next;
426             logic incrthreshold;
427             logic incrsaturate;

```

```

428     } cnt;
429 } key_mode_error_intr_count_r;
430 struct packed{
431     struct packed{
432         logic [31:0] next;
433         logic load_next;
434         logic incrtthreshold;
435         logic incrsaturate;
436     } cnt;
437 } key_zero_error_intr_count_r;
438 struct packed{
439     struct packed{
440         logic [31:0] next;
441         logic load_next;
442         logic incrtthreshold;
443         logic incrsaturate;
444     } cnt;
445 } error2_intr_count_r;
446 struct packed{
447     struct packed{
448         logic [31:0] next;
449         logic load_next;
450         logic incrtthreshold;
451         logic incrsaturate;
452     } cnt;
453 } error3_intr_count_r;
454 struct packed{
455     struct packed{
456         logic [31:0] next;
457         logic load_next;
458         logic incrtthreshold;
459         logic incrsaturate;
460     } cnt;
461 } notif_cmd_done_intr_count_r;
462 struct packed{
463     struct packed{
464         logic next;
465         logic load_next;
466         logic decrthreshold;
467         logic underflow;
468     } pulse;
469 } key_mode_error_intr_count_incr_r;
470 struct packed{
471     struct packed{
472         logic next;
473         logic load_next;
474         logic decrthreshold;
475         logic underflow;
476     } pulse;
477 } key_zero_error_intr_count_incr_r;
478 struct packed{
479     struct packed{
480         logic next;
481         logic load_next;
482         logic decrthreshold;
483         logic underflow;
484     } pulse;
485 } error2_intr_count_incr_r;
486 struct packed{
487     struct packed{
488         logic next;
489         logic load_next;
490         logic decrthreshold;
491         logic underflow;
492     } pulse;
493 } error3_intr_count_incr_r;
494 struct packed{
495     struct packed{
496         logic next;
497         logic load_next;
498         logic decrthreshold;
499         logic underflow;
500     } pulse;
501 } notif_cmd_done_intr_count_incr_r;
502 } intr_block_rf;
503 } field_combo_t;
504 field_combo_t field_combo;
505
506 typedef struct packed{
507     struct packed{
508         struct packed{
509             logic value;
510         } INIT;
511         struct packed{
512             logic value;
513         } NEXT;
514         struct packed{
515             logic value;
516         } ZEROIZE;
517         struct packed{
518             logic value;
519         } MODE;
520         struct packed{
521             logic value;
522         } CSR_MODE;
523         struct packed{
524             logic value;
525         } LAST;
526         struct packed{
527             logic value;
528         } Reserved;
529     } HMAC512_CTRL;
530     struct packed{
531         struct packed{
532             logic [31:0] value;
533         } KEY;
534     } [16-1:0]HMAC512_KEY;
535     struct packed{
536         struct packed{
537             logic [31:0] value;
538         } BLOCK;
539     } [32-1:0]HMAC512_BLOCK;
540     struct packed{
541         struct packed{
542             logic [31:0] value;
543         } TAG;
544     } [16-1:0]HMAC512_TAG;
545     struct packed{
546         struct packed{
547             logic [31:0] value;
548         } LFSR_SEED;
549     } [6-1:0]HMAC512_LFSR_SEED;
550     struct packed{
551         struct packed{
552             logic value;
553         } read_en;
554         struct packed{
555             logic [4:0] value;
556         } read_entry;
557         struct packed{
558             logic value;
559         } pcr_hash_extend;
560         struct packed{
561             logic [24:0] value;
562         } rsvd;
563     } HMAC512_KV_RD_KEY_CTRL;
564     struct packed{
565         struct packed{
566             logic value;
567         } VALID;
568     } HMAC512_KV_RD_KEY_STATUS;
569     struct packed{
570         struct packed{
571             logic value;
572         } read_en;
573         struct packed{
574             logic [4:0] value;
575         } read_entry;
576         struct packed{
577             logic value;
578         } pcr_hash_extend;
579         struct packed{
580             logic [24:0] value;
581         } rsvd;
582     } HMAC512_KV_RD_BLOCK_CTRL;
583     struct packed{
584         struct packed{
585             logic value;
586         } VALID;
587     } HMAC512_KV_RD_BLOCK_STATUS;
588     struct packed{
589         struct packed{
590             logic value;
591         } write_en;
592         struct packed{
593             logic [4:0] value;
594         } write_entry;
595         struct packed{
596             logic value;
597         } hmac_key_dest_valid;
598         struct packed{
599             logic value;
600         } hmac_block_dest_valid;
601         struct packed{
602             logic value;
603         } mlds_seed_dest_valid;
604         struct packed{
605             logic value;
606         } ecc_pkey_dest_valid;
607         struct packed{
608             logic value;
609         } ecc_seed_dest_valid;
610         struct packed{
611             logic value;
612         } aes_key_dest_valid;
613         struct packed{
614             logic value;
615         } mlkem_seed_dest_valid;
616         struct packed{
617             logic value;
618         } mlkem_msg_dest_valid;
619         struct packed{
620             logic value;
621         } dma_data_dest_valid;
622         struct packed{
623             logic [16:0] value;
624         } rsvd;
625     } HMAC512_KV_WR_CTRL;
626     struct packed{
627         struct packed{
628             logic value;
629         } VALID;

```

```

630 } HMAC512_KV_WR_STATUS;
631 struct packed{
632     struct packed{
633         struct packed{
634             logic value;
635         } error_en;
636         struct packed{
637             logic value;
638         } notif_en;
639     } global_intr_en_r;
640     struct packed{
641         struct packed{
642             logic value;
643         } key_mode_error_en;
644         struct packed{
645             logic value;
646         } key_zero_error_en;
647         struct packed{
648             logic value;
649         } error2_en;
650         struct packed{
651             logic value;
652         } error3_en;
653     } error_intr_en_r;
654     struct packed{
655         struct packed{
656             logic value;
657         } notif_cmd_done_en;
658     } notif_intr_en_r;
659     struct packed{
660         struct packed{
661             logic value;
662         } agg_sts;
663     } error_global_intr_r;
664     struct packed{
665         struct packed{
666             logic value;
667         } agg_sts;
668     } notif_global_intr_r;
669     struct packed{
670         struct packed{
671             logic value;
672         } key_mode_error_sts;
673         struct packed{
674             logic value;
675         } key_zero_error_sts;
676         struct packed{
677             logic value;
678         } error2_sts;
679         struct packed{
680             logic value;
681         } error3_sts;
682     } error_internal_intr_r;
683     struct packed{
684         struct packed{
685             logic value;
686         } notif_cmd_done_sts;
687     } notif_internal_intr_r;
688     struct packed{
689         struct packed{
690             logic value;
691         } key_mode_error_trig;
692         struct packed{
693             logic value;
694         } key_zero_error_trig;
695         struct packed{
696             logic value;
697         } error2_trig;
698         struct packed{
699             logic value;
700         } error3_trig;
701     } error_intr_trig_r;
702     struct packed{
703         struct packed{
704             logic value;
705         } notif_cmd_done_trig;
706     } notif_intr_trig_r;
707     struct packed{
708         struct packed{
709             logic [31:0] value;
710         } cnt;
711     } key_mode_error_intr_count_r;
712     struct packed{
713         struct packed{
714             logic [31:0] value;
715         } cnt;
716     } key_zero_error_intr_count_r;
717     struct packed{
718         struct packed{
719             logic [31:0] value;
720         } cnt;
721     } error2_intr_count_r;
722     struct packed{
723         struct packed{
724             logic [31:0] value;
725         } cnt;
726     } error3_intr_count_r;
727     struct packed{
728         struct packed{
729             logic [31:0] value;
730         } cnt;
731     } notif_cmd_done_intr_count_r;
732     struct packed{
733         struct packed{
734             logic value;
735         } pulse;
736     } key_mode_error_intr_count_incr_r;
737     struct packed{
738         struct packed{
739             logic value;
740         } pulse;
741     } key_zero_error_intr_count_incr_r;
742     struct packed{
743         struct packed{
744             logic value;
745         } pulse;
746     } error2_intr_count_incr_r;
747     struct packed{
748         struct packed{
749             logic value;
750         } pulse;
751     } error3_intr_count_incr_r;
752     struct packed{
753         struct packed{
754             logic value;
755         } pulse;
756     } notif_cmd_done_intr_count_incr_r;
757     } intr_block_rf;
758 } field_storage_t;
759 field_storage_t field_storage;
760
761 // Field: hmac_reg.HMAC512_CTRL.INIT
762 always_comb begin
763     automatic logic [0:0] next_c;
764     automatic logic load_next_c;
765     next_c = field_storage.HMAC512_CTRL.INIT.value;
766     load_next_c = '0;
767     if(decoded_reg_strb.HMAC512_CTRL && decoded_req_is_wr
768     && hwif_in.HMAC512_CTRL.INIT.swwe) begin // SW write
769         next_c = (field_storage.HMAC512_CTRL.INIT.value &
770         ~decoded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
771         coded_wr_biten[0:0]);
772         load_next_c = '1;
773     end else begin // singlepulse clears back to 0
774         next_c = '0;
775         load_next_c = '1;
776     end
777     field_combo.HMAC512_CTRL.INIT.next = next_c;
778     field_combo.HMAC512_CTRL.INIT.load_next = load_next_c;
779 end
780 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
781     if(~hwif_in.reset_b) begin
782         field_storage.HMAC512_CTRL.INIT.value <= 1'h0;
783     end else if(field_combo.HMAC512_CTRL.INIT.load_next)
784         field_storage.HMAC512_CTRL.INIT.value <=
785         field_combo.HMAC512_CTRL.INIT.next;
786 end
787 assign hwif_out.HMAC512_CTRL.INIT.value =
788 field_storage.HMAC512_CTRL.INIT.value;
789 // Field: hmac_reg.HMAC512_CTRL.NEXT
790 always_comb begin
791     automatic logic [0:0] next_c;
792     automatic logic load_next_c;
793     next_c = field_storage.HMAC512_CTRL.NEXT.value;
794     load_next_c = '0;
795     if(decoded_reg_strb.HMAC512_CTRL && decoded_req_is_wr
796     && hwif_in.HMAC512_CTRL.NEXT.swwe) begin // SW write
797         next_c = (field_storage.HMAC512_CTRL.NEXT.value &
798         ~decoded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
799         coded_wr_biten[1:1]);
800         load_next_c = '1;
801     end else begin // singlepulse clears back to 0
802         next_c = '0;
803         load_next_c = '1;
804     end
805     field_combo.HMAC512_CTRL.NEXT.next = next_c;
806     field_combo.HMAC512_CTRL.NEXT.load_next = load_next_c;
807 end
808 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
809     if(~hwif_in.reset_b) begin
810         field_storage.HMAC512_CTRL.NEXT.value <= 1'h0;
811     end else if(field_combo.HMAC512_CTRL.NEXT.load_next)
812         field_storage.HMAC512_CTRL.NEXT.value <=
813         field_combo.HMAC512_CTRL.NEXT.next;
814 end
815 assign hwif_out.HMAC512_CTRL.NEXT.value =
816 field_storage.HMAC512_CTRL.NEXT.value;
817 // Field: hmac_reg.HMAC512_CTRL.ZEROIZE
818 always_comb begin
819     automatic logic [0:0] next_c;
820     automatic logic load_next_c;
821     next_c = field_storage.HMAC512_CTRL.ZEROIZE.value;
822     load_next_c = '0;
823     if(decoded_reg_strb.HMAC512_CTRL && decoded_req_is_wr)
824         begin // SW write
825             next_c = (field_storage.HMAC512_CTRL.ZEROIZE.value &
826             ~decoded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
827             coded_wr_biten[0:0]);
828             load_next_c = '1;
829         end else begin // singlepulse clears back to 0
830             next_c = '0;
831             load_next_c = '1;
832         end
833     field_combo.HMAC512_CTRL.ZEROIZE.next = next_c;
834     field_combo.HMAC512_CTRL.ZEROIZE.load_next = load_next_c;
835 end
836 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
837     if(~hwif_in.reset_b) begin
838         field_storage.HMAC512_CTRL.ZEROIZE.value <= 1'h0;
839     end else if(field_combo.HMAC512_CTRL.ZEROIZE.load_next)
840         field_storage.HMAC512_CTRL.ZEROIZE.value <=
841         field_combo.HMAC512_CTRL.ZEROIZE.next;
842 end
843 assign hwif_out.HMAC512_CTRL.ZEROIZE.value =
844 field_storage.HMAC512_CTRL.ZEROIZE.value;

```

```

~decoded_wr_biten[2:2]) | (decoded_wr_data[2:2] & de-
coded_wr_biten[2:2]));
817     load_next_c = '1;
818     end else begin // singlepulse clears back to 0
819     next_c = '0;
820     load_next_c = '1;
821     end
822     field_combo.HMAC512_CTRL.ZEROIZE.next = next_c;
823     field_combo.HMAC512_CTRL.ZEROIZE.load_next =
load_next_c;
824     end
825     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
826     if(~hwif_in.reset_b) begin
827     field_storage.HMAC512_CTRL.ZEROIZE.value <= '1'h0;
828     end else
if(field_combo.HMAC512_CTRL.ZEROIZE.load_next) begin
829     field_storage.HMAC512_CTRL.ZEROIZE.value <=
field_combo.HMAC512_CTRL.ZEROIZE.next;
830     end
831     end
832     assign hwif_out.HMAC512_CTRL.ZEROIZE.value =
field_storage.HMAC512_CTRL.ZEROIZE.value;
833     // Field: hmac_reg.HMAC512_CTRL.MODE
834     always_comb begin
835     automatic logic [0:0] next_c;
836     automatic logic load_next_c;
837     next_c = field_storage.HMAC512_CTRL.MODE.value;
838     load_next_c = '0;
839     if(decoded_reg_strb.HMAC512_CTRL && decoded_req_is_wr
&& hwif_in.HMAC512_CTRL.MODE.swwe) begin // SW write
840     next_c = (field_storage.HMAC512_CTRL.MODE.value &
~decoded_wr_biten[3:3]) | (decoded_wr_data[3:3] & de-
coded_wr_biten[3:3]);
841     load_next_c = '1;
842     end
843     field_combo.HMAC512_CTRL.MODE.next = next_c;
844     field_combo.HMAC512_CTRL.MODE.load_next = load_next_c;
845     end
846     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
847     if(~hwif_in.reset_b) begin
848     field_storage.HMAC512_CTRL.MODE.value <= '1'h1;
849     end else if(field_combo.HMAC512_CTRL.MODE.load_next)
begin
850     field_storage.HMAC512_CTRL.MODE.value <=
field_combo.HMAC512_CTRL.MODE.next;
851     end
852     end
853     assign hwif_out.HMAC512_CTRL.MODE.value =
field_storage.HMAC512_CTRL.MODE.value;
854     // Field: hmac_reg.HMAC512_CTRL.CSR_MODE
855     always_comb begin
856     automatic logic [0:0] next_c;
857     automatic logic load_next_c;
858     next_c = field_storage.HMAC512_CTRL.CSR_MODE.value;
859     load_next_c = '0;
860     if(decoded_reg_strb.HMAC512_CTRL && decoded_req_is_wr
&& hwif_in.HMAC512_CTRL.CSR_MODE.swwe) begin // SW write
861     next_c =
(field_storage.HMAC512_CTRL.CSR_MODE.value & ~decoded_wr_biten[4:4])
| (decoded_wr_data[4:4] & decoded_wr_biten[4:4]);
862     load_next_c = '1;
863     end
864     field_combo.HMAC512_CTRL.CSR_MODE.next = next_c;
865     field_combo.HMAC512_CTRL.CSR_MODE.load_next =
load_next_c;
866     end
867     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
868     if(~hwif_in.reset_b) begin
869     field_storage.HMAC512_CTRL.CSR_MODE.value <= '1'h0;
870     end else
if(field_combo.HMAC512_CTRL.CSR_MODE.load_next) begin
871     field_storage.HMAC512_CTRL.CSR_MODE.value <=
field_combo.HMAC512_CTRL.CSR_MODE.next;
872     end
873     end
874     assign hwif_out.HMAC512_CTRL.CSR_MODE.value =
field_storage.HMAC512_CTRL.CSR_MODE.value;
875     // Field: hmac_reg.HMAC512_CTRL.LAST
876     always_comb begin
877     automatic logic [0:0] next_c;
878     automatic logic load_next_c;
879     next_c = field_storage.HMAC512_CTRL.LAST.value;
880     load_next_c = '0;
881     if(decoded_reg_strb.HMAC512_CTRL && decoded_req_is_wr
&& hwif_in.HMAC512_CTRL.LAST.swwe) begin // SW write
882     next_c = (field_storage.HMAC512_CTRL.LAST.value &
~decoded_wr_biten[5:5]) | (decoded_wr_data[5:5] & de-
coded_wr_biten[5:5]);
883     load_next_c = '1;
884     end else begin // singlepulse clears back to 0
885     next_c = '0;
886     load_next_c = '1;
887     end
888     field_combo.HMAC512_CTRL.LAST.next = next_c;
889     field_combo.HMAC512_CTRL.LAST.load_next = load_next_c;
890     end
891     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
892     if(~hwif_in.reset_b) begin
893     field_storage.HMAC512_CTRL.LAST.value <= '1'h0;
894     end else if(field_combo.HMAC512_CTRL.LAST.load_next)
begin

```

```

895     field_storage.HMAC512_CTRL.LAST.value <=
field_combo.HMAC512_CTRL.LAST.next;
896     end
897     end
898     assign hwif_out.HMAC512_CTRL.LAST.value =
field_storage.HMAC512_CTRL.LAST.value;
899     // Field: hmac_reg.HMAC512_CTRL.Reserved
900     always_comb begin
901     automatic logic [0:0] next_c;
902     automatic logic load_next_c;
903     next_c = field_storage.HMAC512_CTRL.Reserved.value;
904     load_next_c = '0;
905     if(decoded_reg_strb.HMAC512_CTRL && decoded_req_is_wr)
begin // SW write
906     next_c =
(field_storage.HMAC512_CTRL.Reserved.value & ~decoded_wr_biten[6:6])
| (decoded_wr_data[6:6] & decoded_wr_biten[6:6]);
907     load_next_c = '1;
908     end
909     field_combo.HMAC512_CTRL.Reserved.next = next_c;
910     field_combo.HMAC512_CTRL.Reserved.load_next =
load_next_c;
911     end
912     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
913     if(~hwif_in.reset_b) begin
914     field_storage.HMAC512_CTRL.Reserved.value <= '1'h0;
915     end else
if(field_combo.HMAC512_CTRL.Reserved.load_next) begin
916     field_storage.HMAC512_CTRL.Reserved.value <=
field_combo.HMAC512_CTRL.Reserved.next;
917     end
918     end
919     assign hwif_out.HMAC512_CTRL.Reserved.value =
field_storage.HMAC512_CTRL.Reserved.value;
920     for(genvar i0=0; i0<16; i0++) begin
921     // Field: hmac_reg.HMAC512_KEY[i].KEY
922     always_comb begin
923     automatic logic [31:0] next_c;
924     automatic logic load_next_c;
925     next_c = field_storage.HMAC512_KEY[i0].KEY.value;
926     load_next_c = '0;
927     if(decoded_reg_strb.HMAC512_KEY[i0] && de-
coded_req_is_wr && !(hwif_in.HMAC512_KEY[i0].KEY.swwe)) begin // SW
write
928     next_c =
(field_storage.HMAC512_KEY[i0].KEY.value & ~decoded_wr_biten[31:0])
| (decoded_wr_data[31:0] & decoded_wr_biten[31:0]);
929     load_next_c = '1;
930     end else if(hwif_in.HMAC512_KEY[i0].KEY.we) be-
gin // HW Write - we
931     next_c = hwif_in.HMAC512_KEY[i0].KEY.next;
932     load_next_c = '1;
933     end else if(hwif_in.HMAC512_KEY[i0].KEY.hwclr) be-
gin // HW Clear
934     next_c = '0;
935     load_next_c = '1;
936     end
937     field_combo.HMAC512_KEY[i0].KEY.next = next_c;
938     field_combo.HMAC512_KEY[i0].KEY.load_next =
load_next_c;
939     end
940     always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
941     if(~hwif_in.reset_b) begin
942     field_storage.HMAC512_KEY[i0].KEY.value <=
32'h0;
943     end else
if(field_combo.HMAC512_KEY[i0].KEY.load_next) begin
944     field_storage.HMAC512_KEY[i0].KEY.value <=
field_combo.HMAC512_KEY[i0].KEY.next;
945     end
946     end
947     assign hwif_out.HMAC512_KEY[i0].KEY.value =
field_storage.HMAC512_KEY[i0].KEY.value;
948     end
949     for(genvar i0=0; i0<32; i0++) begin
950     // Field: hmac_reg.HMAC512_BLOCK[i].BLOCK
951     always_comb begin
952     automatic logic [31:0] next_c;
953     automatic logic load_next_c;
954     next_c =
field_storage.HMAC512_BLOCK[i0].BLOCK.value;
955     load_next_c = '0;
956     if(decoded_reg_strb.HMAC512_BLOCK[i0] && de-
coded_req_is_wr && !(hwif_in.HMAC512_BLOCK[i0].BLOCK.swwe)) be-
gin // SW write
957     next_c =
(field_storage.HMAC512_BLOCK[i0].BLOCK.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
958     load_next_c = '1;
959     end else if(hwif_in.HMAC512_BLOCK[i0].BLOCK.we)
begin // HW Write - we
960     next_c = hwif_in.HMAC512_BLOCK[i0].BLOCK.next;
961     load_next_c = '1;
962     end else if(hwif_in.HMAC512_BLOCK[i0].BLOCK.hwclr)
begin // HW Clear
963     next_c = '0;
964     load_next_c = '1;

```



```

965         end
966         field_combo.HMAC512_BLOCK[i0].BLOCK.next = next_c;
967         field_combo.HMAC512_BLOCK[i0].BLOCK.load_next =
load_next_c;
968     end
969     always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
970         if(~hwif_in.reset_b) begin
971             field_storage.HMAC512_BLOCK[i0].BLOCK.value <=
32'h0;
972         end else
if(field_combo.HMAC512_BLOCK[i0].BLOCK.load_next) begin
973             field_storage.HMAC512_BLOCK[i0].BLOCK.value <=
field_combo.HMAC512_BLOCK[i0].BLOCK.next;
974         end
975     end
976     assign hwif_out.HMAC512_BLOCK[i0].BLOCK.value =
field_storage.HMAC512_BLOCK[i0].BLOCK.value;
977 end
978 for(genvar i0=0; i0<16; i0++) begin
979     // Field: hmac_reg.HMAC512_TAG[i].TAG
980     always_comb begin
981         automatic logic [31:0] next_c;
982         automatic logic load_next_c;
983         next_c = field_storage.HMAC512_TAG[i0].TAG.value;
984         load_next_c = '0;
985         if(hwif_in.HMAC512_TAG[i0].TAG.hwclr) begin // HW
Clear
986             next_c = '0;
987             load_next_c = '1;
988         end else begin // HW Write
989             next_c = hwif_in.HMAC512_TAG[i0].TAG.next;
990             load_next_c = '1;
991         end
992         field_combo.HMAC512_TAG[i0].TAG.next = next_c;
993         field_combo.HMAC512_TAG[i0].TAG.load_next =
load_next_c;
994     end
995     always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
996         if(~hwif_in.reset_b) begin
997             field_storage.HMAC512_TAG[i0].TAG.value <=
32'h0;
998         end else
if(field_combo.HMAC512_TAG[i0].TAG.load_next) begin
999             field_storage.HMAC512_TAG[i0].TAG.value <=
field_combo.HMAC512_TAG[i0].TAG.next;
1000         end
1001     end
1002 end
1003 for(genvar i0=0; i0<6; i0++) begin
1004     // Field: hmac_reg.HMAC512_LFSR_SEED[i].LFSR_SEED
1005     always_comb begin
1006         automatic logic [31:0] next_c;
1007         automatic logic load_next_c;
1008         next_c =
field_storage.HMAC512_LFSR_SEED[i0].LFSR_SEED.value;
1009         load_next_c = '0;
1010         if(decoded_reg_strb.HMAC512_LFSR_SEED[i0] && de-
coded_req_is_wr) begin // SW write
1011             next_c =
(field_storage.HMAC512_LFSR_SEED[i0].LFSR_SEED.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1012             load_next_c = '1;
1013         end
1014         field_combo.HMAC512_LFSR_SEED[i0].LFSR_SEED.next =
next_c;
1015     end
1016     field_combo.HMAC512_LFSR_SEED[i0].LFSR_SEED.load_next = load_next_c;
1017     always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
1018         if(~hwif_in.reset_b) begin
1019             field_storage.HMAC512_LFSR_SEED[i0].LFSR_SEED.value <= 32'h0;
1020         end else
if(field_combo.HMAC512_LFSR_SEED[i0].LFSR_SEED.load_next) begin
1021             field_storage.HMAC512_LFSR_SEED[i0].LFSR_SEED.value <=
field_combo.HMAC512_LFSR_SEED[i0].LFSR_SEED.next;
1022         end
1023     end
1024     assign hwif_out.HMAC512_LFSR_SEED[i0].LFSR_SEED.value =
field_storage.HMAC512_LFSR_SEED[i0].LFSR_SEED.value;
1025 end
1026 // Field: hmac_reg.HMAC512_KV_RD_KEY_CTRL.read_en
1027 always_comb begin
1028     automatic logic [0:0] next_c;
1029     automatic logic load_next_c;
1030     next_c =
field_storage.HMAC512_KV_RD_KEY_CTRL.read_en.value;
1031     load_next_c = '0;
1032     if(decoded_reg_strb.HMAC512_KV_RD_KEY_CTRL && de-
coded_req_is_wr && hwif_in.HMAC512_KV_RD_KEY_CTRL.read_en.swwe) be-
gin // SW write
1033         next_c =
(field_storage.HMAC512_KV_RD_KEY_CTRL.read_en.value & ~de-

```

```

coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1034         load_next_c = '1;
1035     end else
if(hwif_in.HMAC512_KV_RD_KEY_CTRL.read_en.hwclr) begin // HW Clear
1036         next_c = '0;
1037         load_next_c = '1;
1038     end
1039     field_combo.HMAC512_KV_RD_KEY_CTRL.read_en.next =
next_c;
1040     field_combo.HMAC512_KV_RD_KEY_CTRL.read_en.load_next =
load_next_c;
1041     end
1042     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1043         if(~hwif_in.reset_b) begin
1044             field_storage.HMAC512_KV_RD_KEY_CTRL.read_en.value
<= 1'h0;
1045         end else
if(field_combo.HMAC512_KV_RD_KEY_CTRL.read_en.load_next) begin
1046             field_storage.HMAC512_KV_RD_KEY_CTRL.read_en.value
<= field_combo.HMAC512_KV_RD_KEY_CTRL.read_en.next;
1047         end
1048     end
1049     assign hwif_out.HMAC512_KV_RD_KEY_CTRL.read_en.value =
field_storage.HMAC512_KV_RD_KEY_CTRL.read_en.value;
1050     // Field: hmac_reg.HMAC512_KV_RD_KEY_CTRL.read_entry
1051     always_comb begin
1052         automatic logic [4:0] next_c;
1053         automatic logic load_next_c;
1054         next_c =
field_storage.HMAC512_KV_RD_KEY_CTRL.read_entry.value;
1055         load_next_c = '0;
1056         if(decoded_reg_strb.HMAC512_KV_RD_KEY_CTRL && de-
coded_req_is_wr && hwif_in.HMAC512_KV_RD_KEY_CTRL.read_entry.swwe)
begin // SW write
1057             next_c =
(field_storage.HMAC512_KV_RD_KEY_CTRL.read_entry.value & ~de-
coded_wr_biten[5:1]) | (decoded_wr_data[5:1] & de-
coded_wr_biten[5:1]);
1058             load_next_c = '1;
1059         end
1060         field_combo.HMAC512_KV_RD_KEY_CTRL.read_entry.next =
next_c;
1061     end
1062     field_combo.HMAC512_KV_RD_KEY_CTRL.read_entry.load_next =
load_next_c;
1063     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1064         if(~hwif_in.reset_b) begin
1065             field_storage.HMAC512_KV_RD_KEY_CTRL.read_entry.value <= 5'h0;
1066         end else
if(field_combo.HMAC512_KV_RD_KEY_CTRL.read_entry.load_next) begin
1067             field_storage.HMAC512_KV_RD_KEY_CTRL.read_entry.value <=
field_combo.HMAC512_KV_RD_KEY_CTRL.read_entry.next;
1068         end
1069     end
1070     assign hwif_out.HMAC512_KV_RD_KEY_CTRL.read_entry.value =
field_storage.HMAC512_KV_RD_KEY_CTRL.read_entry.value;
1071     // Field: hmac_reg.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend
1072     always_comb begin
1073         automatic logic [0:0] next_c;
1074         automatic logic load_next_c;
1075         next_c =
field_storage.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.value;
1076         load_next_c = '0;
1077         if(decoded_reg_strb.HMAC512_KV_RD_KEY_CTRL && de-
coded_req_is_wr && hwif_in.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.swwe)
begin // SW write
1078             next_c =
(field_storage.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.value & ~de-
coded_wr_biten[6:6]) | (decoded_wr_data[6:6] & de-
coded_wr_biten[6:6]);
1079             load_next_c = '1;
1080         end
1081     end
1082     field_combo.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.next = next_c;
1083     field_combo.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.load_next =
load_next_c;
1084     end
1085     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1086         if(~hwif_in.reset_b) begin
1087             field_storage.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.value <= 1'h0;
1088         end else
if(field_combo.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.load_next) be-
gin
1089             field_storage.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.value <=
field_combo.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.next;
1090         end
1091     end
1092     assign
hwif_out.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.value =
field_storage.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.value;

```

```

1092 // Field: hmac_reg.HMAC512_KV_RD_KEY_CTRL.rsvd
1093 always_comb begin
1094     automatic logic [24:0] next_c;
1095     automatic logic load_next_c;
1096     next_c =
field_storage.HMAC512_KV_RD_KEY_CTRL.rsvd.value;
1097     load_next_c = '0;
1098     if(decoded_reg_strb.HMAC512_KV_RD_KEY_CTRL && de-
coded_req_is_wr && hwif_in.HMAC512_KV_RD_KEY_CTRL.rsvd.swwe) be-
gin // SW write
1099         next_c =
(field_storage.HMAC512_KV_RD_KEY_CTRL.rsvd.value & ~de-
coded_wr_biten[31:7]) | (decoded_wr_data[31:7] & de-
coded_wr_biten[31:7]);
1100         load_next_c = '1;
1101     end
1102     field_combo.HMAC512_KV_RD_KEY_CTRL.rsvd.next = next_c;
1103     field_combo.HMAC512_KV_RD_KEY_CTRL.rsvd.load_next =
load_next_c;
1104 end
1105 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1106     if(~hwif_in.reset_b) begin

1107         field_storage.HMAC512_KV_RD_KEY_CTRL.rsvd.value <=
25'h0;
1108     end else
if(field_combo.HMAC512_KV_RD_KEY_CTRL.rsvd.load_next) begin

1109         field_storage.HMAC512_KV_RD_KEY_CTRL.rsvd.value <=
field_combo.HMAC512_KV_RD_KEY_CTRL.rsvd.next;
1110     end
1111 end
1112 assign hwif_out.HMAC512_KV_RD_KEY_CTRL.rsvd.value =
field_storage.HMAC512_KV_RD_KEY_CTRL.rsvd.value;
1113 // Field: hmac_reg.HMAC512_KV_RD_KEY_STATUS.VALID
1114 always_comb begin
1115     automatic logic [0:0] next_c;
1116     automatic logic load_next_c;
1117     next_c =
field_storage.HMAC512_KV_RD_KEY_STATUS.VALID.value;
1118     load_next_c = '0;
1119     if(hwif_in.HMAC512_KV_RD_KEY_STATUS.VALID.hwset) be-
gin // HW Set
1120         next_c = '1;
1121         load_next_c = '1;
1122     end else
if(hwif_in.HMAC512_KV_RD_KEY_STATUS.VALID.hwclr) begin // HW Clear
1123         next_c = '0;
1124         load_next_c = '1;
1125     end
1126     field_combo.HMAC512_KV_RD_KEY_STATUS.VALID.next =
next_c;

1127     field_combo.HMAC512_KV_RD_KEY_STATUS.VALID.load_next =
load_next_c;
1128 end
1129 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1130     if(~hwif_in.reset_b) begin

1131         field_storage.HMAC512_KV_RD_KEY_STATUS.VALID.value
<= 1'h0;
1132     end else
if(field_combo.HMAC512_KV_RD_KEY_STATUS.VALID.load_next) begin

1133         field_storage.HMAC512_KV_RD_KEY_STATUS.VALID.value
<= field_combo.HMAC512_KV_RD_KEY_STATUS.VALID.next;
1134     end
1135 end
1136 // Field: hmac_reg.HMAC512_KV_RD_BLOCK_CTRL.read_en
1137 always_comb begin
1138     automatic logic [0:0] next_c;
1139     automatic logic load_next_c;
1140     next_c =
field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_en.value;
1141     load_next_c = '0;
1142     if(decoded_reg_strb.HMAC512_KV_RD_BLOCK_CTRL && de-
coded_req_is_wr && hwif_in.HMAC512_KV_RD_BLOCK_CTRL.read_en.swwe)
begin // SW write
1143         next_c =
(field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1144         load_next_c = '1;
1145     end else
if(hwif_in.HMAC512_KV_RD_BLOCK_CTRL.read_en.hwclr) begin // HW Clear
1146         next_c = '0;
1147         load_next_c = '1;
1148     end
1149     field_combo.HMAC512_KV_RD_BLOCK_CTRL.read_en.next =
next_c;

1150     field_combo.HMAC512_KV_RD_BLOCK_CTRL.read_en.load_next
= load_next_c;
1151 end
1152 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1153     if(~hwif_in.reset_b) begin

1154         field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_en.value <= 1'h0;
1155     end else
if(field_combo.HMAC512_KV_RD_BLOCK_CTRL.read_en.load_next) begin
1156

```

```

field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_en.value <=
field_combo.HMAC512_KV_RD_BLOCK_CTRL.read_en.next;
1157 end
1158 end
1159 assign hwif_out.HMAC512_KV_RD_BLOCK_CTRL.read_en.value =
field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_en.value;
1160 // Field: hmac_reg.HMAC512_KV_RD_BLOCK_CTRL.read_entry
1161 always_comb begin
1162     automatic logic [4:0] next_c;
1163     automatic logic load_next_c;
1164     next_c =
field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_entry.value;
1165     load_next_c = '0;
1166     if(decoded_reg_strb.HMAC512_KV_RD_BLOCK_CTRL && de-
coded_req_is_wr && hwif_in.HMAC512_KV_RD_BLOCK_CTRL.read_entry.swwe)
begin // SW write
1167         next_c =
(field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_entry.value & ~de-
coded_wr_biten[5:1]) | (decoded_wr_data[5:1] & de-
coded_wr_biten[5:1]);
1168         load_next_c = '1;
1169     end

1170     field_combo.HMAC512_KV_RD_BLOCK_CTRL.read_entry.next =
next_c;
1171 end
1172 field_combo.HMAC512_KV_RD_BLOCK_CTRL.read_entry.load_next =
load_next_c;
1173 end
1174 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1175     if(~hwif_in.reset_b) begin

1176         field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_entry.value <= 5'h0;
1177     end else
if(field_combo.HMAC512_KV_RD_BLOCK_CTRL.read_entry.load_next) begin

1178         field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_entry.value <=
field_combo.HMAC512_KV_RD_BLOCK_CTRL.read_entry.next;
1179     end
1180 assign hwif_out.HMAC512_KV_RD_BLOCK_CTRL.read_entry.value =
field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_entry.value;
1181 // Field:
hmac_reg.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend
1182 always_comb begin
1183     automatic logic [0:0] next_c;
1184     automatic logic load_next_c;
1185     next_c =
field_storage.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.value;
1186     load_next_c = '0;
1187     if(decoded_reg_strb.HMAC512_KV_RD_BLOCK_CTRL && de-
coded_req_is_wr && hwif_in.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.swwe)
begin // SW
write
1188         next_c =
(field_storage.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.value & ~de-
coded_wr_biten[6:6]) | (decoded_wr_data[6:6] & de-
coded_wr_biten[6:6]);
1189         load_next_c = '1;
1190     end
1191 end
1192 field_combo.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.next = next_c;
1193 field_combo.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.load_next =
load_next_c;
1194 end
1195 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1196     if(~hwif_in.reset_b) begin

1197         field_storage.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.value <=
1'h0;
1198     end else
if(field_combo.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.load_next)
begin

1199         field_storage.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.value <=
field_combo.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.next;
1200     end
1201 end
1202 assign
hwif_out.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.value =
field_storage.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.value;
1203 // Field: hmac_reg.HMAC512_KV_RD_BLOCK_CTRL.rsvd
1204 always_comb begin
1205     automatic logic [24:0] next_c;
1206     automatic logic load_next_c;
1207     next_c =
field_storage.HMAC512_KV_RD_BLOCK_CTRL.rsvd.value;
1208     load_next_c = '0;
1209     if(decoded_reg_strb.HMAC512_KV_RD_BLOCK_CTRL && de-
coded_req_is_wr && hwif_in.HMAC512_KV_RD_BLOCK_CTRL.rsvd.swwe) be-
gin // SW write
1210         next_c =
(field_storage.HMAC512_KV_RD_BLOCK_CTRL.rsvd.value & ~de-
coded_wr_biten[31:7]) | (decoded_wr_data[31:7] & de-
coded_wr_biten[31:7]);
1211         load_next_c = '1;
1212     end
1213     field_combo.HMAC512_KV_RD_BLOCK_CTRL.rsvd.next =
next_c;
1214 end
1215 field_combo.HMAC512_KV_RD_BLOCK_CTRL.rsvd.load_next =

```



```

load_next_c;
1214     end
1215     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1216         if(~hwif_in.reset_b) begin
1217             field_storage.HMAC512_KV_RD_BLOCK_CTRL.rsvd.value
<= 25'h0;
1218         end else
1219             if(field_combo.HMAC512_KV_RD_BLOCK_CTRL.rsvd.load_next) begin
1220                 field_storage.HMAC512_KV_RD_BLOCK_CTRL.rsvd.value
<= field_combo.HMAC512_KV_RD_BLOCK_CTRL.rsvd.next;
1221             end
1222             assign hwif_out.HMAC512_KV_RD_BLOCK_CTRL.rsvd.value =
field_storage.HMAC512_KV_RD_BLOCK_CTRL.rsvd.value;
1223             // Field: hmac_reg.HMAC512_KV_RD_BLOCK_STATUS.VALID
1224             always_comb begin
1225                 automatic logic [0:0] next_c;
1226                 automatic logic load_next_c;
1227                 next_c =
field_storage.HMAC512_KV_RD_BLOCK_STATUS.VALID.value;
1228                 load_next_c = '0;
1229                 if(hwif_in.HMAC512_KV_RD_BLOCK_STATUS.VALID.hwset) be-
gin // HW Set
1230                     next_c = '1;
1231                     load_next_c = '1;
1232                 end else
1233                 if(hwif_in.HMAC512_KV_RD_BLOCK_STATUS.VALID.hwclr) begin // HW Clear
1234                     next_c = '0;
1235                     load_next_c = '1;
1236                 end
1237                 field_combo.HMAC512_KV_RD_BLOCK_STATUS.VALID.next =
next_c;

1238             end
1239             always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1240                 if(~hwif_in.reset_b) begin
1241                     field_storage.HMAC512_KV_RD_BLOCK_STATUS.VALID.value <= 1'h0;
1242                 end else
1243                 if(field_combo.HMAC512_KV_RD_BLOCK_STATUS.VALID.load_next) begin
1244                     field_storage.HMAC512_KV_RD_BLOCK_STATUS.VALID.value <=
field_combo.HMAC512_KV_RD_BLOCK_STATUS.VALID.next;
1245                 end
1246                 // Field: hmac_reg.HMAC512_KV_WR_CTRL.write_en
1247                 always_comb begin
1248                     automatic logic [0:0] next_c;
1249                     automatic logic load_next_c;
1250                     next_c =
field_storage.HMAC512_KV_WR_CTRL.write_en.value;
1251                     load_next_c = '0;
1252                     if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.HMAC512_KV_WR_CTRL.write_en.swwe) be-
gin // SW write
1253                         next_c =
(field_storage.HMAC512_KV_WR_CTRL.write_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1254                         load_next_c = '1;
1255                     end else if(hwif_in.HMAC512_KV_WR_CTRL.write_en.hwclr)
begin // HW Clear
1256                         next_c = '0;
1257                         load_next_c = '1;
1258                     end
1259                     field_combo.HMAC512_KV_WR_CTRL.write_en.next = next_c;
1260                     field_combo.HMAC512_KV_WR_CTRL.write_en.load_next =
load_next_c;
1261                 end
1262                 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1263                     if(~hwif_in.reset_b) begin
1264                         field_storage.HMAC512_KV_WR_CTRL.write_en.value <=
1'h0;
1265                     end else
1266                     if(field_combo.HMAC512_KV_WR_CTRL.write_en.load_next) begin
1267                         field_storage.HMAC512_KV_WR_CTRL.write_en.value <=
field_combo.HMAC512_KV_WR_CTRL.write_en.next;
1268                     end
1269                     assign hwif_out.HMAC512_KV_WR_CTRL.write_en.value =
field_storage.HMAC512_KV_WR_CTRL.write_en.value;
1270                     // Field: hmac_reg.HMAC512_KV_WR_CTRL.write_entry
1271                     always_comb begin
1272                         automatic logic [4:0] next_c;
1273                         automatic logic load_next_c;
1274                         next_c =
field_storage.HMAC512_KV_WR_CTRL.write_entry.value;
1275                         load_next_c = '0;
1276                         if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.HMAC512_KV_WR_CTRL.write_entry.swwe) be-
gin // SW write
1277                             next_c =
(field_storage.HMAC512_KV_WR_CTRL.write_entry.value & ~de-
coded_wr_biten[5:1]) | (decoded_wr_data[5:1] & de-
coded_wr_biten[5:1]);
1278                             load_next_c = '1;

```

```

1279     end
1280     field_combo.HMAC512_KV_WR_CTRL.write_entry.next =
next_c;

1281     field_combo.HMAC512_KV_WR_CTRL.write_entry.load_next =
load_next_c;
1282     end
1283     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1284         if(~hwif_in.reset_b) begin
1285             field_storage.HMAC512_KV_WR_CTRL.write_entry.value
<= 5'h0;
1286         end else
1287         if(field_combo.HMAC512_KV_WR_CTRL.write_entry.load_next) begin
1288             field_storage.HMAC512_KV_WR_CTRL.write_entry.value
<= field_combo.HMAC512_KV_WR_CTRL.write_entry.next;
1289         end
1290         assign hwif_out.HMAC512_KV_WR_CTRL.write_entry.value =
field_storage.HMAC512_KV_WR_CTRL.write_entry.value;
1291         // Field: hmac_reg.HMAC512_KV_WR_CTRL.hmac_key_dest_valid
1292         always_comb begin
1293             automatic logic [0:0] next_c;
1294             automatic logic load_next_c;
1295             next_c =
field_storage.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.value;
1296             load_next_c = '0;
1297             if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.swwe) begin // SW
write
1298                 next_c =
(field_storage.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.value & ~de-
coded_wr_biten[6:6]) | (decoded_wr_data[6:6] & de-
coded_wr_biten[6:6]);
1299                 load_next_c = '1;
1300             end
1301             field_combo.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.next = next_c;
1302             field_combo.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.load_next =
load_next_c;
1303         end
1304         always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1305             if(~hwif_in.reset_b) begin
1306                 field_storage.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.value <= 1'h0;
1307             end else
1308             if(field_combo.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.load_next) be-
gin
1309                 field_storage.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.value <=
field_combo.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.next;
1310             end
1311             assign
hwif_out.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.value =
field_storage.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.value;
1312             // Field:
hmac_reg.HMAC512_KV_WR_CTRL.hmac_block_dest_valid
1313             always_comb begin
1314                 automatic logic [0:0] next_c;
1315                 automatic logic load_next_c;
1316                 next_c =
field_storage.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.value;
1317                 load_next_c = '0;
1318                 if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.swwe) begin // SW
write
1319                     next_c =
(field_storage.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.value & ~de-
coded_wr_biten[7:7]) | (decoded_wr_data[7:7] & de-
coded_wr_biten[7:7]);
1320                     load_next_c = '1;
1321                 end
1322                 field_combo.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.next = next_c;
1323                 field_combo.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.load_next =
load_next_c;
1324             end
1325             always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1326                 if(~hwif_in.reset_b) begin
1327                     field_storage.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.value <=
1'h0;
1328                 end else
1329                 if(field_combo.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.load_next)
begin
1330                     field_storage.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.value <=
field_combo.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.next;
1331                 end
1332                 assign
hwif_out.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.value =
field_storage.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.value;
1333                 // Field:
hmac_reg.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid

```

```

1334     always_comb begin
1335         automatic logic [0:0] next_c;
1336         automatic logic load_next_c;
1337         next_c =
field_storage.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.value;
1338         load_next_c = '0;
1339         if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.swwe) begin // SW
write
1340             next_c =
(field_storage.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.value & ~de-
coded_wr_biten[8:8]) | (decoded_wr_data[8:8] & de-
coded_wr_biten[8:8]);
1341             load_next_c = '1;
1342         end
1343     end
field_combo.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.next = next_c;
1344     field_combo.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.load_next =
load_next_c;
1345     end
1346     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1347         if(~hwif_in.reset_b) begin
1348             field_storage.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.value <=
1'h0;
1349         end else
if(field_combo.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.load_next)
begin
1350             field_storage.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.value <=
field_combo.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.next;
1351         end
1352     end
1353     assign
hwif_out.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.value =
field_storage.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.value;
1354     // Field: hmac_reg.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid
1355     always_comb begin
1356         automatic logic [0:0] next_c;
1357         automatic logic load_next_c;
1358         next_c =
field_storage.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.value;
1359         load_next_c = '0;
1360         if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.swwe) begin // SW
write
1361             next_c =
(field_storage.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.value & ~de-
coded_wr_biten[9:9]) | (decoded_wr_data[9:9] & de-
coded_wr_biten[9:9]);
1362             load_next_c = '1;
1363         end
1364     end
field_combo.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.next = next_c;
1365     field_combo.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.load_next =
load_next_c;
1366     end
1367     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1368         if(~hwif_in.reset_b) begin
1369             field_storage.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.value <=
1'h0;
1370         end else
if(field_combo.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.load_next) be-
gin
1371             field_storage.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.value <=
field_combo.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.next;
1372         end
1373     end
1374     assign
hwif_out.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.value =
field_storage.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.value;
1375     // Field: hmac_reg.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid
1376     always_comb begin
1377         automatic logic [0:0] next_c;
1378         automatic logic load_next_c;
1379         next_c =
field_storage.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.value;
1380         load_next_c = '0;
1381         if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.swwe) begin // SW
write
1382             next_c =
(field_storage.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.value & ~de-
coded_wr_biten[10:10]) | (decoded_wr_data[10:10] & de-
coded_wr_biten[10:10]);
1383             load_next_c = '1;
1384         end
1385     end
field_combo.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.next = next_c;
1386     field_combo.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.load_next =
load_next_c;
1387     end
1388     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1389         if(~hwif_in.reset_b) begin

```

```

1390     field_storage.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.value <=
1'h0;
1391     end else
if(field_combo.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.load_next) be-
gin
1392         field_storage.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.value <=
field_combo.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.next;
1393     end
1394     end
1395     assign
hwif_out.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.value =
field_storage.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.value;
1396     // Field: hmac_reg.HMAC512_KV_WR_CTRL.aes_key_dest_valid
1397     always_comb begin
1398         automatic logic [0:0] next_c;
1399         automatic logic load_next_c;
1400         next_c =
field_storage.HMAC512_KV_WR_CTRL.aes_key_dest_valid.value;
1401         load_next_c = '0;
1402         if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.HMAC512_KV_WR_CTRL.aes_key_dest_valid.swwe) begin // SW
write
1403             next_c =
(field_storage.HMAC512_KV_WR_CTRL.aes_key_dest_valid.value & ~de-
coded_wr_biten[11:11]) | (decoded_wr_data[11:11] & de-
coded_wr_biten[11:11]);
1404             load_next_c = '1;
1405         end
1406     end
field_combo.HMAC512_KV_WR_CTRL.aes_key_dest_valid.next = next_c;
1407     field_combo.HMAC512_KV_WR_CTRL.aes_key_dest_valid.load_next =
load_next_c;
1408     end
1409     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1410         if(~hwif_in.reset_b) begin
1411             field_storage.HMAC512_KV_WR_CTRL.aes_key_dest_valid.value <=
1'h0;
1412         end else
if(field_combo.HMAC512_KV_WR_CTRL.aes_key_dest_valid.load_next) be-
gin
1413             field_storage.HMAC512_KV_WR_CTRL.aes_key_dest_valid.value <=
field_combo.HMAC512_KV_WR_CTRL.aes_key_dest_valid.next;
1414         end
1415     end
1416     assign
hwif_out.HMAC512_KV_WR_CTRL.aes_key_dest_valid.value =
field_storage.HMAC512_KV_WR_CTRL.aes_key_dest_valid.value;
1417     // Field:
hmac_reg.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid
1418     always_comb begin
1419         automatic logic [0:0] next_c;
1420         automatic logic load_next_c;
1421         next_c =
field_storage.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.value;
1422         load_next_c = '0;
1423         if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.swwe) begin // SW
write
1424             next_c =
(field_storage.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.value & ~de-
coded_wr_biten[12:12]) | (decoded_wr_data[12:12] & de-
coded_wr_biten[12:12]);
1425             load_next_c = '1;
1426         end
1427     end
field_combo.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.next = next_c;
1428     field_combo.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.load_next =
load_next_c;
1429     end
1430     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1431         if(~hwif_in.reset_b) begin
1432             field_storage.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.value <=
1'h0;
1433         end else
if(field_combo.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.load_next)
begin
1434             field_storage.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.value <=
field_combo.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.next;
1435         end
1436     end
1437     assign
hwif_out.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.value =
field_storage.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.value;
1438     // Field: hmac_reg.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid
1439     always_comb begin
1440         automatic logic [0:0] next_c;
1441         automatic logic load_next_c;
1442         next_c =
field_storage.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.value;
1443         load_next_c = '0;
1444         if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr &&

```

```

hwif_in.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.swve) begin // SW
write
1445     next_c =
1446     (field_storage.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.value & ~de-
1447     coded_wr_biten[13:13]) | (decoded_wr_data[13:13] & de-
1448     coded_wr_biten[13:13]);
1449     load_next_c = '1';
1450 end
1451 field_combo.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.next = next_c;
1452 field_combo.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.load_next =
1453 load_next_c;
1454 end
1455 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1456 if(~hwif_in.reset_b) begin
1457     field_storage.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.value <= 1'h0;
1458 end else
1459 if(field_combo.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.load_next)
1460 begin
1461     field_storage.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.value <=
1462     field_combo.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.next;
1463 end
1464 end
1465 assign
1466 hwif_out.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.value =
1467 field_storage.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.value;
1468 // Field: hmac_reg.HMAC512_KV_WR_CTRL.dma_data_dest_valid
1469 always_comb begin
1470     automatic logic [0:0] next_c;
1471     automatic logic load_next_c;
1472     next_c =
1473     field_storage.HMAC512_KV_WR_CTRL.dma_data_dest_valid.value;
1474     load_next_c = '0';
1475     if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
1476     coded_req_is_wr &&
1477     hwif_in.HMAC512_KV_WR_CTRL.dma_data_dest_valid.swve) begin // SW
1478 write
1479     next_c =
1480     (field_storage.HMAC512_KV_WR_CTRL.dma_data_dest_valid.value & ~de-
1481     coded_wr_biten[14:14]) | (decoded_wr_data[14:14] & de-
1482     coded_wr_biten[14:14]);
1483     load_next_c = '1';
1484 end
1485 end
1486 field_combo.HMAC512_KV_WR_CTRL.dma_data_dest_valid.next = next_c;
1487 field_combo.HMAC512_KV_WR_CTRL.dma_data_dest_valid.load_next =
1488 load_next_c;
1489 end
1490 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1491 if(~hwif_in.reset_b) begin
1492     field_storage.HMAC512_KV_WR_CTRL.dma_data_dest_valid.value <= 1'h0;
1493 end else
1494 if(field_combo.HMAC512_KV_WR_CTRL.dma_data_dest_valid.load_next) be-
1495 gin
1496     field_storage.HMAC512_KV_WR_CTRL.dma_data_dest_valid.value <=
1497     field_combo.HMAC512_KV_WR_CTRL.dma_data_dest_valid.next;
1498 end
1499 end
1500 assign
1501 hwif_out.HMAC512_KV_WR_CTRL.dma_data_dest_valid.value =
1502 field_storage.HMAC512_KV_WR_CTRL.dma_data_dest_valid.value;
1503 // Field: hmac_reg.HMAC512_KV_WR_STATUS.VALID
1504 always_comb begin
1505     automatic logic [0:0] next_c;

```

```

1506     automatic logic load_next_c;
1507     next_c =
1508     field_storage.HMAC512_KV_WR_STATUS.VALID.value;
1509     load_next_c = '0';
1510     if(hwif_in.HMAC512_KV_WR_STATUS.VALID.hwset) begin //
1511 HW Set
1512     next_c = '1';
1513     load_next_c = '1';
1514 end else if(hwif_in.HMAC512_KV_WR_STATUS.VALID.hwc1r)
1515 begin // HW Clear
1516     next_c = '0';
1517     load_next_c = '1';
1518 end
1519 end
1520 field_combo.HMAC512_KV_WR_STATUS.VALID.next = next_c;
1521 field_combo.HMAC512_KV_WR_STATUS.VALID.load_next =
1522 load_next_c;
1523 end
1524 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1525 if(~hwif_in.reset_b) begin
1526     field_storage.HMAC512_KV_WR_STATUS.VALID.value <=
1527 1'h0;
1528 end else
1529 if(field_combo.HMAC512_KV_WR_STATUS.VALID.load_next) begin
1530     field_storage.HMAC512_KV_WR_STATUS.VALID.value <=
1531     field_combo.HMAC512_KV_WR_STATUS.VALID.next;
1532 end
1533 end
1534 // Field: hmac_reg.intr_block_rf.global_intr_en_r.error_en
1535 always_comb begin
1536     automatic logic [0:0] next_c;
1537     automatic logic load_next_c;
1538     next_c =
1539     field_storage.intr_block_rf.global_intr_en_r.error_en.value;
1540     load_next_c = '0';
1541     if(decoded_reg_strb.intr_block_rf.global_intr_en_r &&
1542     decoded_req_is_wr) begin // SW write
1543     next_c =
1544     (field_storage.intr_block_rf.global_intr_en_r.error_en.value & ~de-
1545     coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
1546     coded_wr_biten[0:0]);
1547     load_next_c = '1';
1548 end
1549 end
1550 field_combo.intr_block_rf.global_intr_en_r.error_en.next = next_c;
1551 field_combo.intr_block_rf.global_intr_en_r.error_en.load_next =
1552 load_next_c;
1553 end
1554 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1555 if(~hwif_in.reset_b) begin
1556     field_storage.intr_block_rf.global_intr_en_r.error_en.value <= 1'h0;
1557 end else
1558 if(field_combo.intr_block_rf.global_intr_en_r.error_en.load_next)
1559 begin
1560     field_storage.intr_block_rf.global_intr_en_r.error_en.value <=
1561     field_combo.intr_block_rf.global_intr_en_r.error_en.next;
1562 end
1563 end
1564 // Field: hmac_reg.intr_block_rf.global_intr_en_r.notif_en
1565 always_comb begin
1566     automatic logic [0:0] next_c;
1567     automatic logic load_next_c;
1568     next_c =
1569     field_storage.intr_block_rf.global_intr_en_r.notif_en.value;
1570     load_next_c = '0';
1571     if(decoded_reg_strb.intr_block_rf.global_intr_en_r &&
1572     decoded_req_is_wr) begin // SW write
1573     next_c =
1574     (field_storage.intr_block_rf.global_intr_en_r.notif_en.value & ~de-
1575     coded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
1576     coded_wr_biten[1:1]);
1577     load_next_c = '1';
1578 end
1579 end
1580 field_combo.intr_block_rf.global_intr_en_r.notif_en.next = next_c;
1581 field_combo.intr_block_rf.global_intr_en_r.notif_en.load_next =
1582 load_next_c;
1583 end
1584 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1585 if(~hwif_in.reset_b) begin
1586     field_storage.intr_block_rf.global_intr_en_r.notif_en.value <= 1'h0;
1587 end else
1588 if(field_combo.intr_block_rf.global_intr_en_r.notif_en.load_next)
1589 begin
1590     field_storage.intr_block_rf.global_intr_en_r.notif_en.value <=
1591     field_combo.intr_block_rf.global_intr_en_r.notif_en.next;
1592 end
1593 end
1594 // Field: hmac_reg.intr_b-
1595 lock_rf.error_intr_en_r.key_mode_error_en
1596 always_comb begin
1597     automatic logic [0:0] next_c;
1598     automatic logic load_next_c;
1599     next_c =
1600     field_storage.intr_block_rf.error_intr_en_r.key_mode_error_en.value;
1601     load_next_c = '0';

```

```

1570     if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
1571         next_c =
(field_storage.intr_block_rf.error_intr_en_r.key_mode_error_en.value
& ~decoded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1572         load_next_c = '1';
1573     end
1574
field_combo.intr_block_rf.error_intr_en_r.key_mode_error_en.next =
next_c;
1575
field_combo.intr_block_rf.error_intr_en_r.key_mode_error_en.load_ne
xt = load_next_c;
1576     end
1577     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1578         if(~hwif_in.reset_b) begin
1579
field_storage.intr_block_rf.error_intr_en_r.key_mode_error_en.value
<= 1'h0;
1580         end else
if(field_combo.intr_block_rf.error_intr_en_r.key_mode_error_en.load_
next) begin
1581
field_storage.intr_block_rf.error_intr_en_r.key_mode_error_en.value
<= field_combo.intr_block_rf.error_intr_en_r.key_mode_error_en.next;
1582         end
1583     end
1584     // Field: hmac_reg.intr_b-
lock_rf.error_intr_en_r.key_zero_error_en
1585     always_comb begin
1586         automatic logic [0:0] next_c;
1587         automatic logic load_next_c;
1588         next_c =
field_storage.intr_block_rf.error_intr_en_r.key_zero_error_en.value;
1589         load_next_c = '0';
1590         if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
1591             next_c =
(field_storage.intr_block_rf.error_intr_en_r.key_zero_error_en.value
& ~decoded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
coded_wr_biten[1:1]);
1592             load_next_c = '1';
1593         end
1594
field_combo.intr_block_rf.error_intr_en_r.key_zero_error_en.next =
next_c;
1595
field_combo.intr_block_rf.error_intr_en_r.key_zero_error_en.load_ne
xt = load_next_c;
1596     end
1597     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1598         if(~hwif_in.reset_b) begin
1599
field_storage.intr_block_rf.error_intr_en_r.key_zero_error_en.value
<= 1'h0;
1600         end else
if(field_combo.intr_block_rf.error_intr_en_r.key_zero_error_en.load_
next) begin
1601
field_storage.intr_block_rf.error_intr_en_r.key_zero_error_en.value
<= field_combo.intr_block_rf.error_intr_en_r.key_zero_error_en.next;
1602         end
1603     end
1604     // Field: hmac_reg.intr_block_rf.error_intr_en_r.error2_en
1605     always_comb begin
1606         automatic logic [0:0] next_c;
1607         automatic logic load_next_c;
1608         next_c =
field_storage.intr_block_rf.error_intr_en_r.error2_en.value;
1609         load_next_c = '0';
1610         if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
1611             next_c =
(field_storage.intr_block_rf.error_intr_en_r.error2_en.value & ~de-
coded_wr_biten[2:2]) | (decoded_wr_data[2:2] & de-
coded_wr_biten[2:2]);
1612             load_next_c = '1';
1613         end
1614
field_combo.intr_block_rf.error_intr_en_r.error2_en.next = next_c;
1615
field_combo.intr_block_rf.error_intr_en_r.error2_en.load_next =
load_next_c;
1616     end
1617     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1618         if(~hwif_in.reset_b) begin
1619
field_storage.intr_block_rf.error_intr_en_r.error2_en.value <= 1'h0;
1620         end else
if(field_combo.intr_block_rf.error_intr_en_r.error2_en.load_next)
begin
1621
field_storage.intr_block_rf.error_intr_en_r.error2_en.value <=
field_combo.intr_block_rf.error_intr_en_r.error2_en.next;
1622         end
1623     end
1624     // Field: hmac_reg.intr_block_rf.error_intr_en_r.error3_en
1625     always_comb begin
1626         automatic logic [0:0] next_c;
1627         automatic logic load_next_c;

```

```

1628         next_c =
field_storage.intr_block_rf.error_intr_en_r.error3_en.value;
1629         load_next_c = '0';
1630         if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
1631             next_c =
(field_storage.intr_block_rf.error_intr_en_r.error3_en.value & ~de-
coded_wr_biten[3:3]) | (decoded_wr_data[3:3] & de-
coded_wr_biten[3:3]);
1632             load_next_c = '1';
1633         end
1634
field_combo.intr_block_rf.error_intr_en_r.error3_en.next = next_c;
1635
field_combo.intr_block_rf.error_intr_en_r.error3_en.load_next =
load_next_c;
1636     end
1637     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1638         if(~hwif_in.reset_b) begin
1639
field_storage.intr_block_rf.error_intr_en_r.error3_en.value <= 1'h0;
1640         end else
if(field_combo.intr_block_rf.error_intr_en_r.error3_en.load_next)
begin
1641
field_storage.intr_block_rf.error_intr_en_r.error3_en.value <=
field_combo.intr_block_rf.error_intr_en_r.error3_en.next;
1642         end
1643     end
1644     // Field: hmac_reg.intr_b-
lock_rf.notif_intr_en_r.notif_cmd_done_en
1645     always_comb begin
1646         automatic logic [0:0] next_c;
1647         automatic logic load_next_c;
1648         next_c =
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value;
1649         load_next_c = '0';
1650         if(decoded_reg_strb.intr_block_rf.notif_intr_en_r &&
decoded_req_is_wr) begin // SW write
1651             next_c =
(field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
& ~decoded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1652             load_next_c = '1';
1653         end
1654
field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.next =
next_c;
1655
field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.load_ne
xt = load_next_c;
1656     end
1657     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1658         if(~hwif_in.reset_b) begin
1659
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
<= 1'h0;
1660         end else
if(field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.load_
next) begin
1661
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
<= field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.next;
1662         end
1663     end
1664     // Field: hmac_reg.intr_b-
lock_rf.error_global_intr_r.agg_sts
1665     always_comb begin
1666         automatic logic [0:0] next_c;
1667         automatic logic load_next_c;
1668         next_c =
field_storage.intr_block_rf.error_global_intr_r.agg_sts.value;
1669         load_next_c = '0';
1670
// HW Write
1671         next_c = hwif_out.intr_b-
lock_rf.error_internal_intr_r.intr;
1672         load_next_c = '1';
1673
field_combo.intr_block_rf.error_global_intr_r.agg_sts.next = next_c;
1674
field_combo.intr_block_rf.error_global_intr_r.agg_sts.load_next =
load_next_c;
1675     end
1676     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1677         if(~hwif_in.reset_b) begin
1678
field_storage.intr_block_rf.error_global_intr_r.agg_sts.value <=
1'h0;
1679         end else
if(field_combo.intr_block_rf.error_global_intr_r.agg_sts.load_next)
begin
1680
field_storage.intr_block_rf.error_global_intr_r.agg_sts.value <=
field_combo.intr_block_rf.error_global_intr_r.agg_sts.next;
1681         end
1682     end
1683     end
1684     assign hwif_out.intr_block_rf.error_global_intr_r.intr =
1685     |
(field_storage.intr_block_rf.error_global_intr_r.agg_sts.value &
field_storage.intr_block_rf.error_global_intr_r.error_en.value);

```

```

1686 // Field: hmac_reg.intr_b-
lock_rf.notif_global_intr_r.agg_sts
1687 always_comb begin
1688     automatic logic [0:0] next_c;
1689     automatic logic load_next_c;
1690     next_c =
field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value;
1691     load_next_c = '0;
1692
1693     // HW Write
1694     next_c = hwif_out.intr_b-
lock_rf.notif_internal_intr_r.intr;
1695     load_next_c = '1;
1696
1697 field_combo.intr_block_rf.notif_global_intr_r.agg_sts.next = next_c;
1698
1699 field_combo.intr_block_rf.notif_global_intr_r.agg_sts.load_next =
load_next_c;
1700 end
1701 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1702     if(~hwif_in.reset_b) begin
1703         field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value <=
1'h0;
1704     end else
1705         if(field_combo.intr_block_rf.notif_global_intr_r.agg_sts.load_next)
1706             begin
1707                 field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value <=
1708                 field_combo.intr_block_rf.notif_global_intr_r.agg_sts.next;
1709             end
1710         end
1711         assign hwif_out.intr_block_rf.notif_global_intr_r.intr =
1712         |
1713         (field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value &
1714         field_storage.intr_block_rf.global_intr_en_r.notif_en.value);
1715         // Field: hmac_reg.intr_b-
lock_rf.error_internal_intr_r.key_mode_error_sts
1716         always_comb begin
1717             automatic logic [0:0] next_c;
1718             automatic logic load_next_c;
1719             next_c =
field_storage.intr_block_rf.error_internal_intr_r.key_mode_error_sts
.value;
1720             load_next_c = '0;
1721
1722         if(field_storage.intr_block_rf.error_intr_trig_r.key_mode_error_trig
.value != '0) begin // stickybit
1723             next_c =
field_storage.intr_block_rf.error_internal_intr_r.key_mode_error_sts
.value |
field_storage.intr_block_rf.error_intr_trig_r.key_mode_error_trig.v
alue;
1724             load_next_c = '1;
1725         end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.key_mode_error_sts.hwset) begin // HW
Set
1726             next_c = '1;
1727             load_next_c = '1;
1728         end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1729             next_c =
field_storage.intr_block_rf.error_internal_intr_r.key_mode_error_sts
.value & ~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1730             load_next_c = '1;
1731         end
1732     end
1733     field_combo.intr_block_rf.error_internal_intr_r.key_mode_error_sts.
next = next_c;
1734
1735     field_combo.intr_block_rf.error_internal_intr_r.key_mode_error_sts.
load_next = load_next_c;
1736 end
1737 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1738     if(~hwif_in.error_reset_b) begin
1739         field_storage.intr_block_rf.error_internal_intr_r.key_mode_error_sts
.value <= 1'h0;
1740     end else
1741         if(field_combo.intr_block_rf.error_internal_intr_r.key_mode_error_s
ts.load_next) begin
1742             field_storage.intr_block_rf.error_internal_intr_r.key_mode_error_sts
.value <=
1743             field_combo.intr_block_rf.error_internal_intr_r.key_mode_error_sts.
next;
1744         end
1745     end
1746     // Field: hmac_reg.intr_b-
lock_rf.error_internal_intr_r.key_zero_error_sts
1747     always_comb begin
1748         automatic logic [0:0] next_c;
1749         automatic logic load_next_c;
1750         next_c =
field_storage.intr_block_rf.error_internal_intr_r.key_zero_error_sts
.value;
1751         load_next_c = '0;
1752
1753         if(field_storage.intr_block_rf.error_intr_trig_r.key_zero_error_trig
.value != '0) begin // stickybit
1754             next_c =
field_storage.intr_block_rf.error_internal_intr_r.key_zero_error_sts
.value |
field_storage.intr_block_rf.error_intr_trig_r.key_zero_error_trig.v
alue;
1755             load_next_c = '1;
1756         end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.key_zero_error_sts.hwset) begin // HW
Set
1757             next_c = '1;
1758             load_next_c = '1;
1759         end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1760             next_c =
field_storage.intr_block_rf.error_internal_intr_r.key_zero_error_sts
.value & ~(decoded_wr_data[1:1] & decoded_wr_biten[1:1]);
1761             load_next_c = '1;
1762         end
1763     end
1764     field_combo.intr_block_rf.error_internal_intr_r.key_zero_error_sts.
next = next_c;
1765
1766     field_combo.intr_block_rf.error_internal_intr_r.key_zero_error_sts.
load_next = load_next_c;
1767 end
1768 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1769     if(~hwif_in.error_reset_b) begin
1770         field_storage.intr_block_rf.error_internal_intr_r.key_zero_error_sts
.value <= 1'h0;
1771     end else
1772         if(field_combo.intr_block_rf.error_internal_intr_r.key_zero_error_s
ts.load_next) begin
1773             field_storage.intr_block_rf.error_internal_intr_r.key_zero_error_sts
.value <=
1774             field_combo.intr_block_rf.error_internal_intr_r.key_zero_error_sts.
next;
1775         end
1776     end
1777     // Field: hmac_reg.intr_b-
lock_rf.error_internal_intr_r.error2_sts
1778     always_comb begin
1779         automatic logic [0:0] next_c;
1780         automatic logic load_next_c;
1781         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value;
1782         load_next_c = '0;
1783
1784         if(field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value !=
'0) begin // stickybit
1785             next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1786             load_next_c = '1;
1787         end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset) begin // HW Set
1788             next_c = '1;
1789             load_next_c = '1;
1790         end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1791             next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value &
~(decoded_wr_data[2:2] & decoded_wr_biten[2:2]);
1792             load_next_c = '1;
1793         end
1794     end
1795     field_combo.intr_block_rf.error_internal_intr_r.error2_sts.next =
next_c;
1796
1797     field_combo.intr_block_rf.error_internal_intr_r.error2_sts.load_next =
load_next_c;
1798 end
1799 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1800     if(~hwif_in.error_reset_b) begin
1801         field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value <=
1'h0;
1802     end else
1803         if(field_combo.intr_block_rf.error_internal_intr_r.error2_sts.load_
next) begin
1804             field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value <=
1805             field_combo.intr_block_rf.error_internal_intr_r.error2_sts.next;
1806         end
1807     end
1808     // Field: hmac_reg.intr_b-
lock_rf.error_internal_intr_r.error3_sts
1809     always_comb begin
1810         automatic logic [0:0] next_c;
1811         automatic logic load_next_c;
1812         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value;
1813         load_next_c = '0;
1814
1815         if(field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value !=

```

```

.value != '0) begin // stickybit
1741     next_c =
field_storage.intr_block_rf.error_internal_intr_r.key_zero_error_sts
.value |
field_storage.intr_block_rf.error_intr_trig_r.key_zero_error_trig.v
alue;
1742     load_next_c = '1;
1743     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.key_zero_error_sts.hwset) begin // HW
Set
1744         next_c = '1;
1745         load_next_c = '1;
1746     end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1747         next_c =
field_storage.intr_block_rf.error_internal_intr_r.key_zero_error_sts
.value & ~(decoded_wr_data[1:1] & decoded_wr_biten[1:1]);
1748         load_next_c = '1;
1749     end
1750
1751     field_combo.intr_block_rf.error_internal_intr_r.key_zero_error_sts.
next = next_c;
1752
1753     field_combo.intr_block_rf.error_internal_intr_r.key_zero_error_sts.
load_next = load_next_c;
1754 end
1755 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1756     if(~hwif_in.error_reset_b) begin
1757         field_storage.intr_block_rf.error_internal_intr_r.key_zero_error_sts
.value <= 1'h0;
1758     end else
1759         if(field_combo.intr_block_rf.error_internal_intr_r.key_zero_error_s
ts.load_next) begin
1760             field_storage.intr_block_rf.error_internal_intr_r.key_zero_error_sts
.value <=
1761             field_combo.intr_block_rf.error_internal_intr_r.key_zero_error_sts.
next;
1762         end
1763     end
1764     // Field: hmac_reg.intr_b-
lock_rf.error_internal_intr_r.error2_sts
1765     always_comb begin
1766         automatic logic [0:0] next_c;
1767         automatic logic load_next_c;
1768         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value;
1769         load_next_c = '0;
1770
1771         if(field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value !=
'0) begin // stickybit
1772             next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1773             load_next_c = '1;
1774         end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset) begin // HW Set
1775             next_c = '1;
1776             load_next_c = '1;
1777         end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1778             next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value &
~(decoded_wr_data[2:2] & decoded_wr_biten[2:2]);
1779             load_next_c = '1;
1780         end
1781     end
1782     field_combo.intr_block_rf.error_internal_intr_r.error2_sts.next =
next_c;
1783
1784     field_combo.intr_block_rf.error_internal_intr_r.error2_sts.load_next =
load_next_c;
1785 end
1786 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1787     if(~hwif_in.error_reset_b) begin
1788         field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value <=
1'h0;
1789     end else
1790         if(field_combo.intr_block_rf.error_internal_intr_r.error2_sts.load_
next) begin
1791             field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value <=
1792             field_combo.intr_block_rf.error_internal_intr_r.error2_sts.next;
1793         end
1794     end
1795     // Field: hmac_reg.intr_b-
lock_rf.error_internal_intr_r.error3_sts
1796     always_comb begin
1797         automatic logic [0:0] next_c;
1798         automatic logic load_next_c;
1799         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value;
1800         load_next_c = '0;
1801
1802         if(field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value !=

```



```

= '0) begin // stickybit
1793     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1794     load_next_c = '1;
1795     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error3_sts.hwset) begin // HW Set
1796         next_c = '1;
1797         load_next_c = '1;
1798     end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1799         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value &
~(decoded_wr_data[3:3] & decoded_wr_biten[3:3]);
1800         load_next_c = '1;
1801     end
1802     field_combo.intr_block_rf.error_internal_intr_r.error3_sts.next =
next_c;
1803     field_combo.intr_block_rf.error_internal_intr_r.error3_sts.load_next
= load_next_c;
1804     end
1805     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1806         if(~hwif_in.error_reset_b) begin
1807             field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
<= 1'h0;
1808         end else
if(field_combo.intr_block_rf.error_internal_intr_r.error3_sts.load_
next) begin
1809             field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error3_sts.next;
1810         end
1811     end
1812     assign hwif_out.intr_block_rf.error_internal_intr_r.intr =
1813     |
(field_storage.intr_block_rf.error_internal_intr_r.key_mode_error_s
ts.value &
field_storage.intr_block_rf.error_intr_en_r.key_mode_error_en.value)
1814     || |
(field_storage.intr_block_rf.error_internal_intr_r.key_zero_error_s
ts.value &
field_storage.intr_block_rf.error_intr_en_r.key_zero_error_en.value)
1815     || |
(field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error2_en.value)
1816     || |
(field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error3_en.value);
1817     // Field: hmac_reg.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts
1818     always_comb begin
1819         automatic logic [0:0] next_c;
1820         automatic logic load_next_c;
1821         next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value;
1822         load_next_c = '0;
1823     end
if(field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
.value != '0) begin // stickybit
1824         next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value |
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
1825         load_next_c = '1;
1826     end else if(hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset) begin // HW
Set
1827         next_c = '1;
1828         load_next_c = '1;
1829     end else if(de-
coded_reg_strb.intr_block_rf.notif_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1830         next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value & ~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1831         load_next_c = '1;
1832     end
1833     field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.
next = next_c;
1834     field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.
load_next = load_next_c;
1835     end
1836     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1837         if(~hwif_in.reset_b) begin
1838             field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value <= 1'h0;
1839         end else
if(field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_s
ts.load_next) begin
1840             field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts

```

```

.value <=
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.
next;
1841     end
1842     end
1843     assign hwif_out.intr_block_rf.notif_internal_intr_r.intr =
1844     |
(field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_s
ts.value &
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value)
;
1845     // Field: hmac_reg.intr_b-
lock_rf.error_intr_trig_r.key_mode_error_trig
1846     always_comb begin
1847         automatic logic [0:0] next_c;
1848         automatic logic load_next_c;
1849         next_c =
field_storage.intr_block_rf.error_intr_trig_r.key_mode_error_trig.v
alue;
1850         load_next_c = '0;
1851     end
if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1852         next_c =
field_storage.intr_block_rf.error_intr_trig_r.key_mode_error_trig.v
alue | (decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1853         load_next_c = '1;
1854     end else begin // singlepulse clears back to 0
1855         next_c = '0;
1856         load_next_c = '1;
1857     end
1858     field_combo.intr_block_rf.error_intr_trig_r.key_mode_error_trig.next
= next_c;
1859     field_combo.intr_block_rf.error_intr_trig_r.key_mode_error_trig.loa
d_next = load_next_c;
1860     end
1861     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1862         if(~hwif_in.reset_b) begin
1863             field_storage.intr_block_rf.error_intr_trig_r.key_mode_error_trig.v
alue <= 1'h0;
1864         end else
if(field_combo.intr_block_rf.error_intr_trig_r.key_mode_error_trig.
load_next) begin
1865             field_storage.intr_block_rf.error_intr_trig_r.key_mode_error_trig.v
alue <=
field_combo.intr_block_rf.error_intr_trig_r.key_mode_error_trig.next
;
1866     end
1867     end
1868     // Field: hmac_reg.intr_b-
lock_rf.error_intr_trig_r.key_zero_error_trig
1869     always_comb begin
1870         automatic logic [0:0] next_c;
1871         automatic logic load_next_c;
1872         next_c =
field_storage.intr_block_rf.error_intr_trig_r.key_zero_error_trig.v
alue;
1873         load_next_c = '0;
1874     end
if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1875         next_c =
field_storage.intr_block_rf.error_intr_trig_r.key_zero_error_trig.v
alue | (decoded_wr_data[1:1] & decoded_wr_biten[1:1]);
1876         load_next_c = '1;
1877     end else begin // singlepulse clears back to 0
1878         next_c = '0;
1879         load_next_c = '1;
1880     end
1881     field_combo.intr_block_rf.error_intr_trig_r.key_zero_error_trig.next
= next_c;
1882     field_combo.intr_block_rf.error_intr_trig_r.key_zero_error_trig.loa
d_next = load_next_c;
1883     end
1884     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1885         if(~hwif_in.reset_b) begin
1886             field_storage.intr_block_rf.error_intr_trig_r.key_zero_error_trig.v
alue <= 1'h0;
1887         end else
if(field_combo.intr_block_rf.error_intr_trig_r.key_zero_error_trig.
load_next) begin
1888             field_storage.intr_block_rf.error_intr_trig_r.key_zero_error_trig.v
alue <=
field_combo.intr_block_rf.error_intr_trig_r.key_zero_error_trig.next
;
1889     end
1890     end
1891     // Field: hmac_reg.intr_b-
lock_rf.error_intr_trig_r.error2_trig
1892     always_comb begin
1893         automatic logic [0:0] next_c;
1894         automatic logic load_next_c;
1895         next_c =

```

```

1896         field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1897         load_next_c = '0';
1898     end
1899
1900     if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
1901        decoded_req_is_wr) begin // SW write 1 set
1902         next_c =
1903         field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value |
1904         (decoded_wr_data[2:2] & decoded_wr_biten[2:2]);
1905         load_next_c = '1';
1906     end else begin // singlepulse clears back to 0
1907         next_c = '0';
1908         load_next_c = '1';
1909     end
1910
1911     field_combo.intr_block_rf.error_intr_trig_r.error2_trig.next =
1912     next_c;
1913
1914     field_combo.intr_block_rf.error_intr_trig_r.error2_trig.load_next =
1915     load_next_c;
1916     end
1917     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1918         if(~hwif_in.reset_b) begin
1919             field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value <=
1920             1'h0;
1921             end else
1922             if(field_combo.intr_block_rf.error_intr_trig_r.error2_trig.load_next
1923             ) begin
1924                 field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value <=
1925                 field_combo.intr_block_rf.error_intr_trig_r.error2_trig.next;
1926             end
1927             end
1928         // Field: hmac_reg.intr-b-
1929         lock_rf.error_intr_trig_r.error3_trig
1930         always_comb begin
1931             automatic logic [0:0] next_c;
1932             automatic logic load_next_c;
1933             next_c =
1934             field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1935             load_next_c = '0';
1936         end
1937
1938         if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
1939            decoded_req_is_wr) begin // SW write 1 set
1940             next_c =
1941             field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value |
1942             (decoded_wr_data[3:3] & decoded_wr_biten[3:3]);
1943             load_next_c = '1';
1944         end else begin // singlepulse clears back to 0
1945             next_c = '0';
1946             load_next_c = '1';
1947         end
1948
1949         field_combo.intr_block_rf.error_intr_trig_r.error3_trig.next =
1950         next_c;
1951
1952         field_combo.intr_block_rf.error_intr_trig_r.error3_trig.load_next =
1953         load_next_c;
1954         end
1955         always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1956             if(~hwif_in.reset_b) begin
1957                 field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value <=
1958                 1'h0;
1959                 end else
1960                 if(field_combo.intr_block_rf.error_intr_trig_r.error3_trig.load_next
1961                 ) begin
1962                     field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value <=
1963                     field_combo.intr_block_rf.error_intr_trig_r.error3_trig.next;
1964                 end
1965                     end
1966                 // Field: hmac_reg.intr-b-
1967                 lock_rf.notif_intr_trig_r.notif_cmd_done_trig
1968                 always_comb begin
1969                     automatic logic [0:0] next_c;
1970                     automatic logic load_next_c;
1971                     next_c =
1972                     field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
1973                     alue;
1974                     load_next_c = '0';
1975                 end
1976
1977                 if(decoded_reg_strb.intr_block_rf.notif_intr_trig_r &&
1978                    decoded_req_is_wr) begin // SW write 1 set
1979                     next_c =
1980                     field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
1981                     alue | (decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1982                     load_next_c = '1';
1983                 end else begin // singlepulse clears back to 0
1984                     next_c = '0';
1985                     load_next_c = '1';
1986                 end
1987
1988                 field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.next =
1989                 next_c;
1990
1991                 field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig loa
1992                 d_next = load_next_c;
1993                 end
1994                 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1995                     if(~hwif_in.reset_b) begin

```

```

1955 field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
1956     alue <= 1'h0;
1957     end else
1958 if(field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.
1959 load_next) begin
1960     field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
1961     alue <=
1962 field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.next
1963 ;
1964     end
1965     end
1966 // Field: hmac_reg.intr_b-
1967 lock_rf.key_mode_error_intr_count_r.cnt
1968 always_comb begin
1969     automatic logic [31:0] next_c;
1970     automatic logic load_next_c;
1971     next_c =
1972 field_storage.intr_block_rf.key_mode_error_intr_count_r.cnt.value;
1973     load_next_c = '0;
1974     if(de-
1975 coded_reg_strb.intr_block_rf.key_mode_error_intr_count_r && de-
1976 coded_req_is_wr) begin // SW write
1977         next_c =
1978 (field_storage.intr_block_rf.key_mode_error_intr_count_r.cnt.value &
1979 ~decoded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
1980 coded_wr_biten[31:0]);
1981         load_next_c = '1;
1982     end
1983     end
1984 if(field_storage.intr_block_rf.key_mode_error_intr_count_incr_r.pul
1985 se.value) begin // increment
1986     1971     if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
1987 gin // up-counter saturated
1988     1972         next_c = 32'hffffffff;
1989     1973     end else begin
1990     1974         next_c = next_c + 32'h1;
1991     1975     end
1992     1976     load_next_c = '1;
1993     1977     end
1994     1978
1995 field_combo.intr_block_rf.key_mode_error_intr_count_r.cnt.incrthres
1996 hold =
1997 (field_storage.intr_block_rf.key_mode_error_intr_count_r.cnt.value
1998 >= 32'hffffffff);
1999     1979
2000 field_combo.intr_block_rf.key_mode_error_intr_count_r.cnt.incrsatur
2001 ate =
2002 (field_storage.intr_block_rf.key_mode_error_intr_count_r.cnt.value
2003 >= 32'hffffffff);
2004     1980     if(next_c > 32'hffffffff) begin
2005     1981         next_c = 32'hffffffff;
2006     1982         load_next_c = '1;
2007     1983     end
2008     1984
2009 field_combo.intr_block_rf.key_mode_error_intr_count_r.cnt.next =
2010 next_c;
2011     1985
2012 field_combo.intr_block_rf.key_mode_error_intr_count_r.cnt.load_next
2013 = load_next_c;
2014     1986     end
2015     1987     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
2016 begin
2017     1988         if(~hwif_in.error_reset_b) begin
2018     1989
2019 field_storage.intr_block_rf.key_mode_error_intr_count_r.cnt.value <=
2020 32'h0;
2021     1990     end else
2022 if(field_combo.intr_block_rf.key_mode_error_intr_count_r.cnt.load_n
2023 ext) begin
2024     1991
2025 field_storage.intr_block_rf.key_mode_error_intr_count_r.cnt.value <=
2026 field_combo.intr_block_rf.key_mode_error_intr_count_r.cnt.next;
2027     1992     end
2028     1993     end
2029 // Field: hmac_reg.intr_b-
2030 lock_rf.key_zero_error_intr_count_r.cnt
2031 always_comb begin
2032     1996     automatic logic [31:0] next_c;
2033     1997     automatic logic load_next_c;
2034     1998     next_c =
2035 field_storage.intr_block_rf.key_zero_error_intr_count_r.cnt.value;
2036     1999     load_next_c = '0;
2037     2000     if(de-
2038 coded_reg_strb.intr_block_rf.key_zero_error_intr_count_r && de-
2039 coded_req_is_wr) begin // SW write
2040     2001         next_c =
2041 (field_storage.intr_block_rf.key_zero_error_intr_count_r.cnt.value &
2042 ~decoded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
2043 coded_wr_biten[31:0]);
2044     2002         load_next_c = '1;
2045     2003     end
2046     2004
2047 if(field_storage.intr_block_rf.key_zero_error_intr_count_incr_r.pul
2048 se.value) begin // increment
2049     2005     if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
2050 gin // up-counter saturated
2051     2006         next_c = 32'hffffffff;
2052     2007     end else begin
2053     2008         next_c = next_c + 32'h1;

```



```

2009         end
2010         load_next_c = '1';
2011     end
2012     field_combo.intr_block_rf.key_zero_error_intr_count_r.cnt.incrthresh
hold =
(field_storage.intr_block_rf.key_zero_error_intr_count_r.cnt.value
>= 32'hffffffff);
2013     field_combo.intr_block_rf.key_zero_error_intr_count_r.cnt.incrsat
ate =
(field_storage.intr_block_rf.key_zero_error_intr_count_r.cnt.value
>= 32'hffffffff);
2014     if(next_c > 32'hffffffff) begin
2015         next_c = 32'hffffffff;
2016         load_next_c = '1';
2017     end
2018     field_combo.intr_block_rf.key_zero_error_intr_count_r.cnt.next =
next_c;
2019     field_combo.intr_block_rf.key_zero_error_intr_count_r.cnt.load_next
= load_next_c;
2020     end
2021     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
2022         if(~hwif_in.error_reset_b) begin
2023             field_storage.intr_block_rf.key_zero_error_intr_count_r.cnt.value <=
32'h0;
2024             end else
if(field_combo.intr_block_rf.key_zero_error_intr_count_r.cnt.load_n
ext) begin
2025                 field_storage.intr_block_rf.key_zero_error_intr_count_r.cnt.value <=
field_combo.intr_block_rf.key_zero_error_intr_count_r.cnt.next;
2026                 end
2027             end
2028             // Field: hmac_reg.intr_block_rf.error2_intr_count_r.cnt
always_comb begin
2029                 automatic logic [31:0] next_c;
2030                 automatic logic load_next_c;
2031                 next_c =
field_storage.intr_block_rf.error2_intr_count_r.cnt.value;
2032                 load_next_c = '0';
2033                 if(decoded_reg_strb.intr_block_rf.error2_intr_count_r
&& decoded_req_is_wr) begin // SW write
2034                     next_c =
(field_storage.intr_block_rf.error2_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
2035                     load_next_c = '1';
2036                 end
2037             end
2038             if(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value)
begin // increment
2039                 if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
gin // up-counter saturated
2040                     next_c = 32'hffffffff;
2041                 end else begin
2042                     next_c = next_c + 32'h1;
2043                 end
2044                 load_next_c = '1';
2045             end
2046             field_combo.intr_block_rf.error2_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error2_intr_count_r.cnt.value >=
32'hffffffff);
2047             field_combo.intr_block_rf.error2_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error2_intr_count_r.cnt.value >=
32'hffffffff);
2048             if(next_c > 32'hffffffff) begin
2049                 next_c = 32'hffffffff;
2050                 load_next_c = '1';
2051             end
2052             field_combo.intr_block_rf.error2_intr_count_r.cnt.next =
next_c;
2053             field_combo.intr_block_rf.error2_intr_count_r.cnt.load_next =
load_next_c;
2054             end
2055             always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
2056                 if(~hwif_in.error_reset_b) begin
2057                     field_storage.intr_block_rf.error2_intr_count_r.cnt.value <= 32'h0;
2058                     end else
if(field_combo.intr_block_rf.error2_intr_count_r.cnt.load_next) be-
gin
2059                         field_storage.intr_block_rf.error2_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error2_intr_count_r.cnt.next;
2060                         end
2061                     end
2062                     // Field: hmac_reg.intr_block_rf.error3_intr_count_r.cnt
always_comb begin
2063                         automatic logic [31:0] next_c;
2064                         automatic logic load_next_c;
2065                         next_c =

```

```

field_storage.intr_block_rf.error3_intr_count_r.cnt.value;
2067         load_next_c = '0';
2068         if(decoded_reg_strb.intr_block_rf.error3_intr_count_r
&& decoded_req_is_wr) begin // SW write
2069             next_c =
(field_storage.intr_block_rf.error3_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
2070             load_next_c = '1';
2071         end
2072         if(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value)
begin // increment
2073             if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
gin // up-counter saturated
2074                 next_c = 32'hffffffff;
2075             end else begin
2076                 next_c = next_c + 32'h1;
2077             end
2078             load_next_c = '1';
2079         end
2080         field_combo.intr_block_rf.error3_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error3_intr_count_r.cnt.value >=
32'hffffffff);
2081         field_combo.intr_block_rf.error3_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error3_intr_count_r.cnt.value >=
32'hffffffff);
2082         if(next_c > 32'hffffffff) begin
2083             next_c = 32'hffffffff;
2084             load_next_c = '1';
2085         end
2086         field_combo.intr_block_rf.error3_intr_count_r.cnt.next =
next_c;
2087         field_combo.intr_block_rf.error3_intr_count_r.cnt.load_next =
load_next_c;
2088         end
2089         always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
2090             if(~hwif_in.error_reset_b) begin
2091                 field_storage.intr_block_rf.error3_intr_count_r.cnt.value <= 32'h0;
2092                 end else
if(field_combo.intr_block_rf.error3_intr_count_r.cnt.load_next) be-
gin
2093                     field_storage.intr_block_rf.error3_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error3_intr_count_r.cnt.next;
2094                     end
2095                 end
2096                 // Field: hmac_reg.intr_b-
lock_rf.notif_cmd_done_intr_count_r.cnt
always_comb begin
2097                     automatic logic [31:0] next_c;
2098                     automatic logic load_next_c;
2099                     next_c =
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value;
2100                     load_next_c = '0';
2101                     if(de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r && de-
coded_req_is_wr) begin // SW write
2102                         next_c =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value &
~decoded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
2103                         load_next_c = '1';
2104                     end
2105                     if(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pul
se.value) begin // increment
2106                         if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
gin // up-counter saturated
2107                             next_c = 32'hffffffff;
2108                         end else begin
2109                             next_c = next_c + 32'h1;
2110                         end
2111                         load_next_c = '1';
2112                     end
2113                     field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.incrthres
hold =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value
>= 32'hffffffff);
2114                     field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.incrsatur
ate =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value
>= 32'hffffffff);
2115                     if(next_c > 32'hffffffff) begin
2116                         next_c = 32'hffffffff;
2117                         load_next_c = '1';
2118                     end
2119                     field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.next =
next_c;
2120                     field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.load_next
= load_next_c;

```

```

2122     end
2123     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2124         if(~hwif_in.reset_b) begin
2125             field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value <=
2126                 32'h0;
2127         end else
2128             if(field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.load_n
2129                 xt) begin
2130                 field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value <=
2131                     field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.next;
2132             end
2133         end
2134         // Field: hmac_reg.intr_b-
2135         lock_rf.key_mode_error_intr_count_incr_r.pulse
2136         always_comb begin
2137             automatic logic [0:0] next_c;
2138             automatic logic load_next_c;
2139             next_c =
2140                 field_storage.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.
2141                 value;
2142             load_next_c = '0;
2143         end
2144         if(field_storage.intr_block_rf.error_intr_trig_r.key_mode_error_trig
2145             .value) begin // HW Write - we
2146             next_c =
2147                 field_storage.intr_block_rf.error_intr_trig_r.key_mode_error_trig.v
2148                 alue;
2149             load_next_c = '1;
2150         end else if(hwif_in.intr_b-
2151             lock_rf.error_internal_intr_r.key_mode_error_sts.hwset) begin // HW
2152             Set
2153             next_c = '1;
2154             load_next_c = '1;
2155         end
2156         if(field_storage.intr_block_rf.key_mode_error_intr_count_incr_r.pul
2157             se.value) begin // decrement
2158             field_combo.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.un
2159             derflow = (next_c < (1'h1));
2160             next_c = next_c - 1'h1;
2161             load_next_c = '1;
2162         end else begin
2163             field_combo.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.un
2164             derflow = '0;
2165         end
2166         field_combo.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.de
2167         crthreshold =
2168             (field_storage.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.
2169             value <= 1'd0);
2170         field_combo.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.ne
2171         xt = next_c;
2172         field_combo.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.lo
2173         ad_next = load_next_c;
2174         end
2175         always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2176             if(~hwif_in.reset_b) begin
2177                 field_storage.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.
2178                 value <= 1'h0;
2179             end else
2180                 if(field_combo.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.
2181                     load_next) begin
2182                     field_combo.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.ne
2183                     xt = next_c;
2184                     field_combo.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.lo
2185                     ad_next = load_next_c;
2186                 end
2187             end
2188             always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2189                 if(~hwif_in.reset_b) begin
2190                     field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value)
2191                     begin // HW Write - we
2192                         next_c =
2193                             field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
2194                         load_next_c = '1;
2195                     end else if(hwif_in.intr_b-
2196                         lock_rf.error_internal_intr_r.error2_sts.hwset) begin // HW Set
2197                         next_c = '1;
2198                         load_next_c = '1;
2199                     end
2200                     if(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value)
2201                     begin // decrement
2202                         field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.underflow =
2203                         (next_c < (1'h1));
2204                         next_c = next_c - 1'h1;
2205                         load_next_c = '1;
2206                     end else begin
2207                         field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.underflow =
2208                         '0;
2209                     end
2210                     end
2211                     field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.decrthresh
2212                     old =
2213                         (field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
2214                         1'd0);
2215                     field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next =
2216                     next_c;
2217                     field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_next =
2218                     load_next_c;
2219                     end
2220                     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2221                         if(~hwif_in.reset_b) begin
2222                             field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
2223                             1'h0;
2224                         end else
2225                             if(field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_ne
2226                                 xt) begin
2227                                 field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
2228                                 field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next;
2229                             end
2230                             end
2231                             // Field: hmac_reg.intr_b-
2232                             lock_rf.error3_intr_count_incr_r.pulse
2233                             always_comb begin
2234                                 automatic logic [0:0] next_c;
2235                                 automatic logic load_next_c;
2236                                 next_c =
2237                                     field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value;

```

```

2175     field_combo.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.un
2176     derflow = (next_c < (1'h1));
2177     next_c = next_c - 1'h1;
2178     load_next_c = '1;
2179     end else begin
2180         field_combo.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.un
2181         derflow = '0;
2182     end
2183     field_combo.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.de
2184     crthreshold =
2185         (field_storage.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.
2186         value <= 1'd0);
2187     field_combo.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.ne
2188     xt = next_c;
2189     field_combo.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.lo
2190     ad_next = load_next_c;
2191     end
2192     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2193         if(~hwif_in.reset_b) begin
2194             field_storage.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.
2195             value <= 1'h0;
2196         end else
2197             if(field_combo.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.
2198                 load_next) begin
2199                 field_storage.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.
2200                 value <=
2201                 field_combo.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.ne
2202                 xt;
2203                 end
2204                 // Field: hmac_reg.intr_b-
2205                 lock_rf.error2_intr_count_incr_r.pulse
2206                 always_comb begin
2207                     automatic logic [0:0] next_c;
2208                     automatic logic load_next_c;
2209                     next_c =
2210                         field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value;
2211                     load_next_c = '0;
2212                 end
2213                 if(field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value)
2214                 begin // HW Write - we
2215                     next_c =
2216                         field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
2217                     load_next_c = '1;
2218                 end else if(hwif_in.intr_b-
2219                     lock_rf.error_internal_intr_r.error2_sts.hwset) begin // HW Set
2220                     next_c = '1;
2221                     load_next_c = '1;
2222                 end
2223                 if(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value)
2224                 begin // decrement
2225                     field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.underflow =
2226                     (next_c < (1'h1));
2227                     next_c = next_c - 1'h1;
2228                     load_next_c = '1;
2229                 end else begin
2230                     field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.underflow =
2231                     '0;
2232                 end
2233                 end
2234                 field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.decrthresh
2235                 old =
2236                     (field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
2237                     1'd0);
2238                 field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next =
2239                 next_c;
2240                 field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_next =
2241                 load_next_c;
2242                 end
2243                 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2244                     if(~hwif_in.reset_b) begin
2245                         field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
2246                         1'h0;
2247                     end else
2248                         if(field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_ne
2249                             xt) begin
2250                             field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
2251                             field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next;
2252                         end
2253                         end
2254                         // Field: hmac_reg.intr_b-
2255                         lock_rf.error3_intr_count_incr_r.pulse
2256                         always_comb begin
2257                             automatic logic [0:0] next_c;
2258                             automatic logic load_next_c;
2259                             next_c =
2260                                 field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value;

```

```

2228     load_next_c = '0';
2229
2230 if(field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value)
begin // HW Write - we
2231     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
2232     load_next_c = '1';
2233 end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error3_sts.hwset) begin // HW Set
2234     next_c = '1';
2235     load_next_c = '1';
2236 end
2237
2238 if(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value)
begin // decrement
2239     field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
2240     next_c = next_c - 1'h1;
2241     load_next_c = '1';
2242 end else begin
2243     field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.underflow =
'0';
2244     end
2245
2246 field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
1'd0);
2247
2248 field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.next =
next_c;
2249
2250 field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.load_next =
load_next_c;
2251 end
2252 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2253     if(~hwif_in.reset_b) begin
2254         field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
1'h0;
2255     end else
2256     if(field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.load_ne
xt) begin
2257         field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.next;
2258     end
2259 end
2260 // Field: hmac_reg.intr_b-
lock_rf.notif_cmd_done_intr_count_incr_r.pulse
2261 always_comb begin
2262     automatic logic [0:0] next_c;
2263     automatic logic load_next_c;
2264     next_c =
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value;
2265     load_next_c = '0';
2266
2267 if(field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
.value) begin // HW Write - we
2268     next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
2269     load_next_c = '1';
2270 end else if(hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset) begin // HW
Set
2271     next_c = '1';
2272     load_next_c = '1';
2273 end
2274
2275 if(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pul
se.value) begin // decrement
2276     field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.un
derflow = (next_c < (1'h1));
2277     next_c = next_c - 1'h1;
2278     load_next_c = '1';
2279 end else begin
2280     field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.un
derflow = '0';
2281     end
2282
2283 field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.de
crthreshol
d =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <= 1'd0);
2284
2285 field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.ne
xt = next_c;
2286
2287 field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.lo
ad_next = load_next_c;
2288 end
2289 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2290     if(~hwif_in.reset_b) begin
2291         field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.

```

```

value <= 1'h0;
2292     end else
2293     if(field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
load_next) begin
2294         field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <=
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.ne
xt;
2295     end
2296 end
2297
2298 //-----
2299 // Write response
2300 //-----
2301
2302 assign cpuif_wr_ack = decoded_req & decoded_req_is_wr;
2303 // Writes are always granted with no error response
2304 assign cpuif_wr_err = '0';
2305
2306 //-----
2307 // Readback
2308 //-----
2309
2310 logic readback_err;
2311 logic readback_done;
2312 logic [31:0] readback_data;
2313
2314 // Assign readback values to a flattened array
2315 logic [46:1:0][31:0] readback_array;
2316 for(genvar i0=0; i0<2; i0++) begin
2317     assign readback_array[i0*1 + 0][31:0] = (de-
coded_reg_strb.HMAC512_NAME[i0] && !decoded_req_is_wr) ?
hwif_in.HMAC512_NAME[i0].NAME.next : '0';
2318 end
2319 for(genvar i0=0; i0<2; i0++) begin
2320     assign readback_array[i0*1 + 2][31:0] = (de-
coded_reg_strb.HMAC512_VERSION[i0] && !decoded_req_is_wr) ?
hwif_in.HMAC512_VERSION[i0].VERSION.next : '0';
2321 end
2322 assign readback_array[4][0:0] = (de-
coded_reg_strb.HMAC512_STATUS && !decoded_req_is_wr) ?
hwif_in.HMAC512_STATUS.READY.next : '0';
2323 assign readback_array[4][1:1] = (de-
coded_reg_strb.HMAC512_STATUS && !decoded_req_is_wr) ?
hwif_in.HMAC512_STATUS.VALID.next : '0';
2324 assign readback_array[4][31:2] = '0';
2325 for(genvar i0=0; i0<16; i0++) begin
2326     assign readback_array[i0*1 + 5][31:0] = (de-
coded_reg_strb.HMAC512_TAG[i0] && !decoded_req_is_wr) ?
field_storage.HMAC512_TAG[i0].TAG.value : '0';
2327 end
2328 assign readback_array[21][0:0] = (de-
coded_reg_strb.HMAC512_KV_RD_KEY_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_RD_KEY_CTRL.read_en.value : '0';
2329 assign readback_array[21][5:1] = (de-
coded_reg_strb.HMAC512_KV_RD_KEY_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.value : '0';
2330 assign readback_array[21][31:7] = (de-
coded_reg_strb.HMAC512_KV_RD_KEY_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_RD_KEY_CTRL.rsvd.value : '0';
2331 assign readback_array[22][0:0] = (de-
coded_reg_strb.HMAC512_KV_RD_KEY_STATUS && !decoded_req_is_wr) ?
hwif_in.HMAC512_KV_RD_KEY_STATUS.READY.next : '0';
2332 assign readback_array[22][31:10] = '0';
2333 assign readback_array[23][0:0] = (de-
coded_reg_strb.HMAC512_KV_RD_BLOCK_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_en.value : '0';
2334 assign readback_array[23][5:1] = (de-
coded_reg_strb.HMAC512_KV_RD_BLOCK_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.value : '0';
2335 assign readback_array[23][31:7] = (de-
coded_reg_strb.HMAC512_KV_RD_BLOCK_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_RD_BLOCK_CTRL.rsvd.value : '0';
2336 assign readback_array[24][0:0] = (de-
coded_reg_strb.HMAC512_KV_RD_BLOCK_STATUS && !decoded_req_is_wr) ?
hwif_in.HMAC512_KV_RD_BLOCK_STATUS.READY.next : '0';
2337 assign readback_array[24][1:1] = (de-
coded_reg_strb.HMAC512_KV_RD_BLOCK_STATUS && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_RD_BLOCK_STATUS.VALID.next : '0';
2338 assign readback_array[24][9:2] = (de-
coded_reg_strb.HMAC512_KV_RD_BLOCK_STATUS && !decoded_req_is_wr) ?
hwif_in.HMAC512_KV_RD_BLOCK_STATUS.ERROR.next : '0';
2339 assign readback_array[24][31:10] = '0';
2340 assign readback_array[25][0:0] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?

```

```

field_storage.HMAC512_KV_WR_CTRL.write_en.value : '0';
2332 assign readback_array[25][5:1] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.write_entry.value : '0';
2333 assign readback_array[25][6:6] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.value : '0';
2334 assign readback_array[25][7:7] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.value : '0';
2335 assign readback_array[25][8:8] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.mldsa_seed_dest_valid.value : '0';
2336 assign readback_array[25][9:9] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.value : '0';
2337 assign readback_array[25][10:10] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.value : '0';
2338 assign readback_array[25][11:11] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.aes_key_dest_valid.value : '0';
2339 assign readback_array[25][12:12] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.value : '0';
2340 assign readback_array[25][13:13] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.value : '0';
2341 assign readback_array[25][14:14] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.dma_data_dest_valid.value : '0';
2342 assign readback_array[25][31:15] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.rsvd.value : '0';
2343 assign readback_array[26][0:0] = (de-
coded_reg_strb.HMAC512_KV_WR_STATUS && !decoded_req_is_wr) ?
hwif_in.HMAC512_KV_WR_STATUS.READY.next : '0';
2344 assign readback_array[26][1:1] = (de-
coded_reg_strb.HMAC512_KV_WR_STATUS && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_STATUS.VALID.value : '0';
2345 assign readback_array[26][9:2] = (de-
coded_reg_strb.HMAC512_KV_WR_STATUS && !decoded_req_is_wr) ?
hwif_in.HMAC512_KV_WR_STATUS.ERROR.next : '0';
2346 assign readback_array[26][31:10] = '0';
2347 assign readback_array[27][0:0] = (de-
coded_reg_strb.intr_block_rf.global_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.global_intr_en_r.error_en.value : '0';
2348 assign readback_array[27][1:1] = (de-
coded_reg_strb.intr_block_rf.global_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.global_intr_en_r.notif_en.value : '0';
2349 assign readback_array[27][31:2] = '0';
2350 assign readback_array[28][0:0] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.key_mode_error_en.value : '0';
2351 assign readback_array[28][1:1] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.key_zero_error_en.value : '0';
2352 assign readback_array[28][2:2] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error2_en.value : '0';
2353 assign readback_array[28][3:3] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error3_en.value : '0';
2354 assign readback_array[28][31:4] = '0';
2355 assign readback_array[29][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value : '0';
2356 assign readback_array[29][31:1] = '0';
2357 assign readback_array[30][0:0] = (de-
coded_reg_strb.intr_block_rf.error_global_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_global_intr_r.agg_sts.value : '0';
2358 assign readback_array[30][31:1] = '0';
2359 assign readback_array[31][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_global_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value : '0';
2360 assign readback_array[31][31:1] = '0';
2361 assign readback_array[32][0:0] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.key_mode_error_sts.value : '0';
2362 assign readback_array[32][1:1] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.key_zero_error_sts.value : '0';
2363 assign readback_array[32][2:2] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?

```

```

field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value : '0';
2364 assign readback_array[32][3:3] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value : '0';
2365 assign readback_array[32][31:4] = '0';
2366 assign readback_array[33][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.value : '0';
2367 assign readback_array[33][31:1] = '0';
2368 assign readback_array[34][0:0] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.key_mode_error_trig.v alue : '0';
2369 assign readback_array[34][1:1] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.key_zero_error_trig.v alue : '0';
2370 assign readback_array[34][2:2] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value : '0';
2371 assign readback_array[34][3:3] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value : '0';
2372 assign readback_array[34][31:4] = '0';
2373 assign readback_array[35][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v alue : '0';
2374 assign readback_array[35][31:1] = '0';
2375 assign readback_array[36][31:0] = (de-
coded_reg_strb.intr_block_rf.key_mode_error_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.key_mode_error_intr_count_r.cnt.value : '0';
2376 assign readback_array[37][31:0] = (de-
coded_reg_strb.intr_block_rf.key_zero_error_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.key_zero_error_intr_count_r.cnt.value : '0';
2377 assign readback_array[38][31:0] = (de-
coded_reg_strb.intr_block_rf.error2_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error2_intr_count_r.cnt.value : '0';
2378 assign readback_array[39][31:0] = (de-
coded_reg_strb.intr_block_rf.error3_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error3_intr_count_r.cnt.value : '0';
2379 assign readback_array[40][31:0] = (de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value : '0';
2380 assign readback_array[41][0:0] = (de-
coded_reg_strb.intr_block_rf.key_mode_error_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.value : '0';
2381 assign readback_array[41][31:1] = '0';
2382 assign readback_array[42][0:0] = (de-
coded_reg_strb.intr_block_rf.key_zero_error_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.value : '0';
2383 assign readback_array[42][31:1] = '0';
2384 assign readback_array[43][0:0] = (de-
coded_reg_strb.intr_block_rf.error2_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value : '0';
2385 assign readback_array[43][31:1] = '0';
2386 assign readback_array[44][0:0] = (de-
coded_reg_strb.intr_block_rf.error3_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value : '0';
2387 assign readback_array[44][31:1] = '0';
2388 assign readback_array[45][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.value : '0';
2389 assign readback_array[45][31:1] = '0';
2390
2391 // Reduce the array
2392 always_comb begin
2393     automatic logic [31:0] readback_data_var;
2394     readback_done = decoded_req & ~decoded_req_is_wr;
2395     readback_err = '0';
2396     readback_data_var = '0';

```

```

2397     for(int i=0; i<46; i++) readback_data_var |= read-
back_array[i];
2398     readback_data = readback_data_var;
2399     end
2400
2401     assign cpuif_rd_ack = readback_done;
2402     assign cpuif_rd_data = readback_data;
2403     assign cpuif_rd_err = readback_err;
2404
2405     `include "caliptra_prim_assert.sv"
2406     `CALIPTRA_ASSERT_KNOWN(ERR_HWIF_IN, hwif_in, clk, !hwif_in.er-
ror_reset_b)
2407
2408 endmodule

```

6. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_reg_pkg.sv (450 lines)

```

1 // Generated by PeakRDL-regblock - A free and open-source Sys-
temVerilog generator
2 // https://github.com/SystemRDL/PeakRDL-regblock
3
4 package hmac_reg_pkg;
5
6     localparam HMAC_REG_DATA_WIDTH = 32;
7     localparam HMAC_REG_MIN_ADDR_WIDTH = 12;
8
9     typedef struct packed{
10         logic [31:0] next;
11     } hmac_reg_HMAC512_NAME_NAME__in_t;
12
13     typedef struct packed{
14         hmac_reg_HMAC512_NAME_NAME__in_t NAME;
15     } hmac_reg_HMAC512_NAME__in_t;
16
17     typedef struct packed{
18         logic [31:0] next;
19     } hmac_reg_HMAC512_VERSION_VERSION__in_t;
20
21     typedef struct packed{
22         hmac_reg_HMAC512_VERSION_VERSION__in_t VERSION;
23     } hmac_reg_HMAC512_VERSION__in_t;
24
25     typedef struct packed{
26         logic swwe;
27     } hmac_reg_HMAC512_CTRL_INIT__in_t;
28
29     typedef struct packed{
30         logic swwe;
31     } hmac_reg_HMAC512_CTRL_NEXT__in_t;
32
33     typedef struct packed{
34         logic swwe;
35     } hmac_reg_HMAC512_CTRL_MODE__in_t;
36
37     typedef struct packed{
38         logic swwe;
39     } hmac_reg_HMAC512_CTRL_CSR_MODE__in_t;
40
41     typedef struct packed{
42         logic swwe;
43     } hmac_reg_HMAC512_CTRL_LAST__in_t;
44
45     typedef struct packed{
46         hmac_reg_HMAC512_CTRL_INIT__in_t INIT;
47         hmac_reg_HMAC512_CTRL_NEXT__in_t NEXT;
48         hmac_reg_HMAC512_CTRL_MODE__in_t MODE;
49         hmac_reg_HMAC512_CTRL_CSR_MODE__in_t CSR_MODE;
50         hmac_reg_HMAC512_CTRL_LAST__in_t LAST;
51     } hmac_reg_HMAC512_CTRL__in_t;
52
53     typedef struct packed{
54         logic next;
55     } hmac_reg_HMAC512_STATUS_READY__in_t;
56
57     typedef struct packed{
58         logic next;
59     } hmac_reg_HMAC512_STATUS_VALID__in_t;
60
61     typedef struct packed{
62         hmac_reg_HMAC512_STATUS_READY__in_t READY;
63         hmac_reg_HMAC512_STATUS_VALID__in_t VALID;
64     } hmac_reg_HMAC512_STATUS__in_t;
65
66     typedef struct packed{
67         logic [31:0] next;
68         logic we;
69         logic swwe1;
70         logic hwclr;
71     } hmac_reg_HMAC512_KEY_KEY__in_t;
72
73     typedef struct packed{
74         hmac_reg_HMAC512_KEY_KEY__in_t KEY;
75     } hmac_reg_HMAC512_KEY__in_t;
76
77     typedef struct packed{
78         logic [31:0] next;
79         logic we;
80         logic swwe1;
81         logic hwclr;

```

```

82     } hmac_reg_HMAC512_BLOCK_BLOCK__in_t;
83
84     typedef struct packed{
85         hmac_reg_HMAC512_BLOCK_BLOCK__in_t BLOCK;
86     } hmac_reg_HMAC512_BLOCK__in_t;
87
88     typedef struct packed{
89         logic [31:0] next;
90         logic hwclr;
91     } hmac_reg_HMAC512_TAG_TAG__in_t;
92
93     typedef struct packed{
94         hmac_reg_HMAC512_TAG_TAG__in_t TAG;
95     } hmac_reg_HMAC512_TAG__in_t;
96
97     typedef struct packed{
98         logic swwe;
99         logic hwclr;
100    } kv_read_ctrl_reg_read_en__in_t;
101
102    typedef struct packed{
103        logic swwe;
104    } kv_read_ctrl_reg_read_entry__in_t;
105
106    typedef struct packed{
107        logic swwe;
108    } kv_read_ctrl_reg_pcr_hash_extend__in_t;
109
110    typedef struct packed{
111        logic swwe;
112    } kv_read_ctrl_reg_rsvd__in_t;
113
114    typedef struct packed{
115        kv_read_ctrl_reg_read_en__in_t read_en;
116        kv_read_ctrl_reg_read_entry__in_t read_entry;
117        kv_read_ctrl_reg_pcr_hash_extend__in_t
pcr_hash_extend;
118        kv_read_ctrl_reg_rsvd__in_t rsvd;
119    } kv_read_ctrl_reg__in_t;
120
121    typedef struct packed{
122        logic next;
123    } kv_status_reg_READY__in_t;
124
125    typedef struct packed{
126        logic hwclr;
127        logic hwset;
128    } kv_status_reg_VALID__in_t;
129
130    typedef struct packed{
131        logic [7:0] next;
132    } kv_status_reg_ERROR__in_t;
133
134    typedef struct packed{
135        kv_status_reg_READY__in_t READY;
136        kv_status_reg_VALID__in_t VALID;
137        kv_status_reg_ERROR__in_t ERROR;
138    } kv_status_reg__in_t;
139
140    typedef struct packed{
141        logic swwe;
142        logic hwclr;
143    } kv_write_ctrl_reg_write_en__in_t;
144
145    typedef struct packed{
146        logic swwe;
147    } kv_write_ctrl_reg_write_entry__in_t;
148
149    typedef struct packed{
150        logic swwe;
151    } kv_write_ctrl_reg_hmac_key_dest_valid__in_t;
152
153    typedef struct packed{
154        logic swwe;
155    } kv_write_ctrl_reg_hmac_block_dest_valid__in_t;
156
157    typedef struct packed{
158        logic swwe;
159    } kv_write_ctrl_reg_mlds_seed_dest_valid__in_t;
160
161    typedef struct packed{
162        logic swwe;
163    } kv_write_ctrl_reg_ecc_pkey_dest_valid__in_t;
164
165    typedef struct packed{
166        logic swwe;
167    } kv_write_ctrl_reg_ecc_seed_dest_valid__in_t;
168
169    typedef struct packed{
170        logic swwe;
171    } kv_write_ctrl_reg_aes_key_dest_valid__in_t;
172
173    typedef struct packed{
174        logic swwe;
175    } kv_write_ctrl_reg_mlkem_seed_dest_valid__in_t;
176
177    typedef struct packed{
178        logic swwe;
179    } kv_write_ctrl_reg_mlkem_msg_dest_valid__in_t;
180
181    typedef struct packed{

```



```

182     logic swwe;
183 } kv_write_ctrl_reg_dma_data_dest_valid_in_t;
184
185 typedef struct packed{
186     logic swwe;
187 } kv_write_ctrl_reg_rsvd_in_t;
188
189 typedef struct packed{
190     kv_write_ctrl_reg_write_en_in_t write_en;
191     kv_write_ctrl_reg_write_entry_in_t write_entry;
192     kv_write_ctrl_reg_hmac_key_dest_valid_in_t
hmac_key_dest_valid;
193     kv_write_ctrl_reg_hmac_block_dest_valid_in_t hmac_b-
lock_dest_valid;
194     kv_write_ctrl_reg_mldsa_seed_dest_valid_in_t
mldsa_seed_dest_valid;
195     kv_write_ctrl_reg_ecc_pkey_dest_valid_in_t ecc_p-
key_dest_valid;
196     kv_write_ctrl_reg_ecc_seed_dest_valid_in_t
ecc_seed_dest_valid;
197     kv_write_ctrl_reg_aes_key_dest_valid_in_t aes_key_d-
est_valid;
198     kv_write_ctrl_reg_mlkem_seed_dest_valid_in_t
mlkem_seed_dest_valid;
199     kv_write_ctrl_reg_mlkem_msg_dest_valid_in_t
mlkem_msg_dest_valid;
200     kv_write_ctrl_reg_dma_data_dest_valid_in_t
dma_data_dest_valid;
201     kv_write_ctrl_reg_rsvd_in_t rsvd;
202 } kv_write_ctrl_reg_in_t;
203
204 typedef struct packed{
205     logic hwset;
206 } hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_key_zero_error_sts_enable_28f0af4b_next_
7027a86d_resetsignal_939e99d4_in_t;
207
208     typedef struct packed{
209         logic hwset;
210     } hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_key_zero_error_sts_enable_8d7a01ba_next_
938a4efa_resetsignal_939e99d4_in_t;
211
212     typedef struct packed{
213         logic hwset;
214     } hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_error2_sts_enable_0dacf7a6_next_4b5b9e74
_resetsignal_939e99d4_in_t;
215
216     typedef struct packed{
217         logic hwset;
218     } hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_error3_sts_enable_fc3af94b_next_c3125d40
_resetsignal_939e99d4_in_t;
219
220     typedef struct packed{
221         hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_key_mode_error_sts_enable_28f0af4b_next_
7027a86d_resetsignal_939e99d4_in_t key_mode_error_sts;
222         hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_key_zero_error_sts_enable_8d7a01ba_next_
938a4efa_resetsignal_939e99d4_in_t key_zero_error_sts;
223         hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_error2_sts_enable_0dacf7a6_next_4b5b9e74
_resetsignal_939e99d4_in_t error2_sts;
224         hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_error3_sts_enable_fc3af94b_next_c3125d40
_resetsignal_939e99d4_in_t error3_sts;
225     } hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_in_t;
226
227     typedef struct packed{
228         logic hwset;
229     } hmac_reg_no-
tif_in-
tr_t_no-
tif_cmd_done_sts_1c68637e__notif_cmd_done_sts_enable_dabe0b8b_next_5
40fa3b7_in_t;
230
231     typedef struct packed{
232         hmac_reg_no-
tif_in-
tr_t_no-
tif_cmd_done_sts_1c68637e__notif_cmd_done_sts_enable_dabe0b8b_next_5
40fa3b7_in_t notif_cmd_done_sts;
233     } hmac_reg_no-
tif_intr_t_notif_cmd_done_sts_1c68637e_in_t;
234
235     typedef struct packed{
236         hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_in_t error_internal_intr_r;
237         hmac_reg_no-
tif_intr_t_notif_cmd_done_sts_1c68637e_in_t notif_internal_intr_r;
238     } hmac_reg_intr_block_t_in_t;
239
240     typedef struct packed{
241         logic reset_b;
242         logic error_reset_b;
243         hmac_reg_HMAC512_NAME_in_t [2-1:0]HMAC512_NAME;
244         hmac_reg_HMAC512_VERSION_in_t
[2-1:0]HMAC512_VERSION;
245         hmac_reg_HMAC512_CTRL_in_t HMAC512_CTRL;
246         hmac_reg_HMAC512_STATUS_in_t HMAC512_STATUS;
247         hmac_reg_HMAC512_KEY_in_t [16-1:0]HMAC512_KEY;
248         hmac_reg_HMAC512_BLOCK_in_t [32-1:0]HMAC512_BLOCK;
249         hmac_reg_HMAC512_TAG_in_t [16-1:0]HMAC512_TAG;
250         kv_read_ctrl_reg_in_t HMAC512_KV_RD_KEY_CTRL;
251         kv_status_reg_in_t HMAC512_KV_RD_KEY_STATUS;
252         kv_read_ctrl_reg_in_t HMAC512_KV_RD_BLOCK_CTRL;
253         kv_status_reg_in_t HMAC512_KV_RD_BLOCK_STATUS;
254         kv_write_ctrl_reg_in_t HMAC512_KV_WR_CTRL;
255         kv_status_reg_in_t HMAC512_KV_WR_STATUS;
256         hmac_reg_intr_block_t_in_t intr_block_rf;
257     } hmac_reg_in_t;
258
259     typedef struct packed{
260         logic value;
261     } hmac_reg_HMAC512_CTRL_INIT_out_t;
262
263     typedef struct packed{
264         logic value;
265     } hmac_reg_HMAC512_CTRL_NEXT_out_t;
266
267     typedef struct packed{
268         logic value;
269     } hmac_reg_HMAC512_CTRL_ZEROIZE_out_t;
270
271     typedef struct packed{
272         logic value;
273     } hmac_reg_HMAC512_CTRL_MODE_out_t;
274
275     typedef struct packed{
276         logic value;
277     } hmac_reg_HMAC512_CTRL_CSR_MODE_out_t;
278
279     typedef struct packed{
280         logic value;
281     } hmac_reg_HMAC512_CTRL_LAST_out_t;
282
283     typedef struct packed{
284         logic value;
285     } hmac_reg_HMAC512_CTRL_Reserved_out_t;
286
287     typedef struct packed{
288         hmac_reg_HMAC512_CTRL_INIT_out_t INIT;
289         hmac_reg_HMAC512_CTRL_NEXT_out_t NEXT;
290         hmac_reg_HMAC512_CTRL_ZEROIZE_out_t ZEROIZE;
291         hmac_reg_HMAC512_CTRL_MODE_out_t MODE;
292         hmac_reg_HMAC512_CTRL_CSR_MODE_out_t CSR_MODE;
293         hmac_reg_HMAC512_CTRL_LAST_out_t LAST;
294         hmac_reg_HMAC512_CTRL_Reserved_out_t Reserved;
295     } hmac_reg_HMAC512_CTRL_out_t;
296
297     typedef struct packed{
298         logic [31:0] value;
299     } hmac_reg_HMAC512_KEY_KEY_out_t;
300
301     typedef struct packed{
302         hmac_reg_HMAC512_KEY_KEY_out_t KEY;
303     } hmac_reg_HMAC512_KEY_out_t;
304
305     typedef struct packed{
306         logic [31:0] value;
307     } hmac_reg_HMAC512_BLOCK_BLOCK_out_t;
308
309     typedef struct packed{
310         hmac_reg_HMAC512_BLOCK_BLOCK_out_t BLOCK;
311     } hmac_reg_HMAC512_BLOCK_out_t;
312
313     typedef struct packed{
314         logic [31:0] value;
315     } hmac_reg_HMAC512_LFSR_SEED_LFSR_SEED_out_t;

```

```

316
317 typedef struct packed{
318     hmac_reg_HMAC512_LFSR_SEED__LFSR_SEED__out_t
LFSR_SEED;
319 } hmac_reg_HMAC512_LFSR_SEED__out_t;
320
321 typedef struct packed{
322     logic value;
323 } kv_read_ctrl_reg_read_en__out_t;
324
325 typedef struct packed{
326     logic [4:0] value;
327 } kv_read_ctrl_reg_read_entry__out_t;
328
329 typedef struct packed{
330     logic value;
331 } kv_read_ctrl_reg_pcr_hash_extend__out_t;
332
333 typedef struct packed{
334     logic [24:0] value;
335 } kv_read_ctrl_reg_rsvd__out_t;
336
337 typedef struct packed{
338     kv_read_ctrl_reg_read_en__out_t read_en;
339     kv_read_ctrl_reg_read_entry__out_t read_entry;
340     kv_read_ctrl_reg_pcr_hash_extend__out_t
pcr_hash_extend;
341     kv_read_ctrl_reg_rsvd__out_t rsvd;
342 } kv_read_ctrl_reg__out_t;
343
344 typedef struct packed{
345     logic value;
346 } kv_write_ctrl_reg_write_en__out_t;
347
348 typedef struct packed{
349     logic [4:0] value;
350 } kv_write_ctrl_reg_write_entry__out_t;
351
352 typedef struct packed{
353     logic value;
354 } kv_write_ctrl_reg_hmac_key_dest_valid__out_t;
355
356 typedef struct packed{
357     logic value;
358 } kv_write_ctrl_reg_hmac_block_dest_valid__out_t;
359
360 typedef struct packed{
361     logic value;
362 } kv_write_ctrl_reg_mldsa_seed_dest_valid__out_t;
363
364 typedef struct packed{
365     logic value;
366 } kv_write_ctrl_reg_ecc_pkey_dest_valid__out_t;
367
368 typedef struct packed{
369     logic value;
370 } kv_write_ctrl_reg_ecc_seed_dest_valid__out_t;
371
372 typedef struct packed{
373     logic value;
374 } kv_write_ctrl_reg_aes_key_dest_valid__out_t;
375
376 typedef struct packed{
377     logic value;
378 } kv_write_ctrl_reg_mlkem_seed_dest_valid__out_t;
379
380 typedef struct packed{
381     logic value;
382 } kv_write_ctrl_reg_mlkem_msg_dest_valid__out_t;
383
384 typedef struct packed{
385     logic value;
386 } kv_write_ctrl_reg_dma_data_dest_valid__out_t;
387
388 typedef struct packed{
389     logic [16:0] value;
390 } kv_write_ctrl_reg_rsvd__out_t;
391
392 typedef struct packed{
393     kv_write_ctrl_reg_write_en__out_t write_en;

```

```

394     kv_write_ctrl_reg_write_entry__out_t write_entry;
395     kv_write_ctrl_reg_hmac_key_dest_valid__out_t
hmac_key_dest_valid;
396     kv_write_ctrl_reg_hmac_block_dest_valid__out_t
hmac_block_dest_valid;
397     kv_write_ctrl_reg_mldsa_seed_dest_valid__out_t
mldsa_seed_dest_valid;
398     kv_write_ctrl_reg_ecc_pkey_dest_valid__out_t ecc_p-
key_dest_valid;
399     kv_write_ctrl_reg_ecc_seed_dest_valid__out_t
ecc_seed_dest_valid;
400     kv_write_ctrl_reg_aes_key_dest_valid__out_t
aes_key_dest_valid;
401     kv_write_ctrl_reg_mlkem_seed_dest_valid__out_t
mlkem_seed_dest_valid;
402     kv_write_ctrl_reg_mlkem_msg_dest_valid__out_t
mlkem_msg_dest_valid;
403     kv_write_ctrl_reg_dma_data_dest_valid__out_t
dma_data_dest_valid;
404     kv_write_ctrl_reg_rsvd__out_t rsvd;
405 } kv_write_ctrl_reg__out_t;
406
407 typedef struct packed{
408     logic intr;
409 } hmac_reg_global_intr_t_agg_sts_dd3dcf0a__out_t;
410
411 typedef struct packed{
412     logic intr;
413 } hmac_reg_global_intr_t_agg_sts_e6399b4a__out_t;
414
415 typedef struct packed{
416     logic intr;
417 } hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183__out_t;
418
419 typedef struct packed{
420     logic intr;
421 } hmac_reg_no-
tif_intr_t_notif_cmd_done_sts_1c68637e__out_t;
422
423 typedef struct packed{
424     hmac_reg_global_intr_t_agg_sts_dd3dcf0a__out_t er-
ror_global_intr_r;
425     hmac_reg_global_intr_t_agg_sts_e6399b4a__out_t no-
tif_global_intr_r;
426     hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183__out_t error_internal_intr_r;
427     hmac_reg_no-
tif_intr_t_notif_cmd_done_sts_1c68637e__out_t notif_internal_intr_r;
428 } hmac_reg_intr_block_t__out_t;
429
430 typedef struct packed{
431     hmac_reg_HMAC512_CTRL__out_t HMAC512_CTRL;
432     hmac_reg_HMAC512_KEY__out_t [16:1:0]HMAC512_KEY;
433     hmac_reg_HMAC512_BLOCK__out_t [32:1:0]HMAC512_BLOCK;
434     hmac_reg_HMAC512_LFSR_SEED__out_t
[6:1:0]HMAC512_LFSR_SEED;
435     kv_read_ctrl_reg__out_t HMAC512_KV_RD_KEY_CTRL;
436     kv_read_ctrl_reg__out_t HMAC512_KV_RD_BLOCK_CTRL;
437     kv_write_ctrl_reg__out_t HMAC512_KV_WR_CTRL;
438     hmac_reg_intr_block_t__out_t intr_block_rf;
439 } hmac_reg_out_t;
440
441 typedef enum logic [31:0] {
442     kv_status_reg_ERROR_kv_error_e_SUCCESS = 'h0,
443     kv_status_reg_ERROR_kv_error_e_KV_READ_FAIL = 'h1,
444     kv_status_reg_ERROR_kv_error_e_KV_WRITE_FAIL = 'h2,
445     kv_status_reg_ERROR_kv_error_e_KV_FSM_ERROR = 'h4
446 } kv_status_reg_ERROR_kv_error_e_e;
447
448 localparam HMAC_REG_ADDR_WIDTH = 32'd12;
449
450 endpackage

```


Caliptra ECC (P-384)

What this is. Elliptic-curve arithmetic over the NIST P-384 curve.

Why it exists. Because attestation requires digital signatures, and ECC provides them at far smaller key sizes than RSA for equivalent classical security.

Undergraduate review · EECS concepts. Elliptic curve group law; modular arithmetic; Montgomery ladder for constant-time scalar multiplication; projective coordinates to avoid inversions.

Graduate · Practice note. Scalar multiplication must be constant time and regular: any dependence of the operation sequence on secret scalar bits leaks the key. The Montgomery ladder exists for this reason, not for speed.

16 files • 3,663 lines

API — ecc_add_sub_mod_alter.sv

Table 1376. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_add_sub_mod_alter.sv — module / interface / package (1개)

이름	종류
ecc_add_sub_mod_alter	module

Table 1377. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_add_sub_mod_alter.sv — parameter (1개)

파라미터	형/폭	기본값
REG_SIZE		384

Table 1378. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_add_sub_mod_alter.sv — 포트 (10개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
add_en_i	input	wire
sub_i	input	wire
opa_i	input	wire [REG_SIZE-1:0]
opb_i	input	wire [REG_SIZE-1:0]
prime_i	input	wire [REG_SIZE-1:0]
res_o	output	logic [REG_SIZE-1:0]
ready_o	output	logic

API — ecc_adder.sv

Table 1379. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_adder.sv — module / interface / package (1개)

이름	종류
ecc_adder	module

Table 1380. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_adder.sv — parameter (1개)

파라미터	형/폭	기본값
RADIX		8

Table 1381. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_adder.sv — 포트 (5개)

포트	방향	형/폭
a_i	input	wire [RADIX-1:0]
b_i	input	wire [RADIX-1:0]
cin_i	input	wire
s_o	output	wire [RADIX-1:0]
cout_o	output	wire

API — ecc_arith_unit.sv**Table 1382. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_arith_unit.sv — module / interface / package (1개)**

이름	종류
ecc_arith_unit	module

Table 1383. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_arith_unit.sv — parameter (8개)

파라미터	형/폭	기본값
REG_SIZE		384
RND_SIZE		192
RADIX		32
ADDR_WIDTH		6
p_prime	[REG_SIZE-1 : 0]	384'hff...
p_mu	[RADIX-1 : 0]	32'h00000001
q_grouporder	[REG_SIZE-1 : 0]	384'hfffffffffffffffffffffffffffffffffffffc7634d8...
q_mu	[RADIX-1 : 0]	32'he88fdc45

Table 1384. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_arith_unit.sv — 포트 (11개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
ecc_cmd_i	input	wire [3 : 0]
addr_i	input	wire [ADDR_WIDTH-1 : 0]

포트	방향	형/폭
wr_op_sel_i	input	wire
wr_en_i	input	wire
rd_reg_i	input	wire
RND_SIZE	input	wire [(REG_SIZE+
data_o	output	wire [REG_SIZE-1: 0]
busy_o	output	wire

API — ecc_defines_pkg.sv

Table 1385. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_defines_pkg.sv — module / interface / package (1개)

이름	종류
ecc_defines_pkg	package

Table 1386. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_defines_pkg.sv — localparam (2개)

상수	형/폭	값
ECC_ADDR_W		32
ECC_DATA_W		32

Table 1387. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_defines_pkg.sv — #define / `define 매크로 (1개)

매크로	인자	정의
CALIPTRA_ECC_DEFINES		package ecc_defines_pkg;

API — ecc_dsa_ctrl.sv

Table 1388. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_ctrl.sv — module / interface / package (1개)

이름	종류
ecc_dsa_ctrl	module

Table 1389. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_ctrl.sv — localparam (2개)

상수	형/폭	값
zero_pad	[RND_SIZE-1 : 0]	'0
REG_NUM_DWORDS		REG_SIZE / DATA_WIDTH

Table 1390. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_ctrl.sv — 포트 (15개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
cptra_pwrgood	input	wire
hwif_out	input	ecc_reg__out_t

포트	방향	형/폭
hwif_in	output	ecc_reg__in_t
kv_read	output	kv_read_t [1:0]
kv_write	output	kv_write_t
kv_rd_resp	input	kv_rd_resp_t [1:0]
kv_wr_resp	input	kv_wr_resp_t
pcr_signing_data	input	pcr_signing_t
ocp_lock_in_progress	input	logic
busy_o	output	logic
error_intr	output	logic
notif_intr	output	logic
debugUnlock_or_scan_mode_switch	input	logic

API — ecc_dsa_sequencer.sv

Table 1391. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_sequencer.sv — module / interface / package (1개)

이름	종류
ecc_dsa_sequencer	module

Table 1392. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_sequencer.sv — parameter (2개)

파라미터	형/폭	기본값
ADDR_WIDTH		8
DATA_WIDTH		32

Table 1393. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_sequencer.sv — 포트 (6개)

포트	방향	형/폭
clka	input	wire
reset_n	input	wire
zeroize	input	wire
ena	input	wire
addra	input	wire [ADDR_WIDTH-1 : 0]
douta	output	logic [DATA_WIDTH-1 : 0]

API — ecc_dsa_uop_pkg.sv

Table 1394. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_uop_pkg.sv — module / interface / package (1개)

이름	종류
ecc_dsa_uop_pkg	package

Table 1395. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_uop_pkg.sv — localparam (54개)

상수	형/폭	값
DSA_UOP_ADDR_WIDTH	integer	9
DSA_OPR_ADDR_WIDTH	integer	6
DSA_PROG_ADDR_W		7
DSA_INSTRUCTION_LENGTH		DSA_UOP_ADDR_WIDTH + (2*DSA_OPR_ADDR_WIDTH)
DSA_UOP_NOP	opcode_t	{op_sel:1'b0
DSA_UOP_WR_CORE	opcode_t	{op_sel:1'b0
DSA_UOP_WR_SCALAR	opcode_t	{op_sel:1'b1
DSA_UOP_RD_CORE	opcode_t	{op_sel:1'b0
DSA_UOP_KEYGEN	opcode_t	{op_sel:1'b0
DSA_UOP_SIGN	opcode_t	{op_sel:1'b0
DSA_UOP_VERIFY0	opcode_t	{op_sel:1'b0
DSA_UOP_VERIFY1	opcode_t	{op_sel:1'b0
DSA_UOP_VERIFY2	opcode_t	{op_sel:1'b0
DSA_UOP_PK_CHK	opcode_t	{op_sel:1'b0
DSA_UOP_HMAC_DRBG	opcode_t	{op_sel:1'b0
DSA_UOP_SCALAR_SCA	opcode_t	{op_sel:1'b0
DH_UOP_SHAREDKEY	opcode_t	{op_sel:1'b0
NOP_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd0
CONST_ZERO_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd1
CONST_ONE_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd2
CONST_E_a_MONT_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd3
CONST_E_3b_MONT_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd4
CONST_ONE_p_MONT_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd5
CONST_R2_p_MONT_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd6
CONST_G_X_MONT_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd7
CONST_G_Y_MONT_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd8
CONST_R2_q_MONT_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd9
CONST_ONE_q_MONT_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd10
CONST_E_b_MONT_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd11
SEED_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd16
MSG_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd17
PRIVKEY_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd18
PUBKEYX_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd19
PUBKEYY_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd20

상수	형/폭	값
R_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd21
S_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd22
SCALAR_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd23
SCALAR_G_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd24
SCALAR_PK_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd25
VERIFY_R_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd26
LAMBDA_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd27
MASKING_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd28
PK_VALID_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd29
DH_SHAREDKEY_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd30
ECC_RESET	[DSA_PROG_ADDR_W-1 : 0]	7'd0
ECC_NOP	[DSA_PROG_ADDR_W-1 : 0]	7'd12
DSA_KG_S	[DSA_PROG_ADDR_W-1 : 0]	ECC_NOP + 2
DSA_KG_E	[DSA_PROG_ADDR_W-1 : 0]	DSA_KG_S + 12
DSA_SGN_S	[DSA_PROG_ADDR_W-1 : 0]	DSA_KG_E + 2
DSA_SGN_E	[DSA_PROG_ADDR_W-1 : 0]	DSA_SGN_S + 14
DSA_VER_S	[DSA_PROG_ADDR_W-1 : 0]	DSA_SGN_E + 2
DSA_VER_E	[DSA_PROG_ADDR_W-1 : 0]	DSA_VER_S + 23
DH_SHARED_S	[DSA_PROG_ADDR_W-1 : 0]	DSA_VER_E + 2
DH_SHARED_E	[DSA_PROG_ADDR_W-1 : 0]	DH_SHARED_S + 17

Table 1396. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_uop_pkg.sv — #define / `define 매크로 (1개)

매크로	인자	정의
ECC_DSA_UOP_PKG		package ecc_dsa_uop_pkg;

API — ecc_fau.sv

Table 1397. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_fau.sv — module / interface / package (1개)

이름	종류
ecc_fau	module

Table 1398. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_fau.sv — parameter (2개)

파라미터	형/폭	기본값
REG_SIZE		384
RADIX		32

Table 1399. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_fau.sv — 포트 (12개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
add_en_i	input	wire
sub_i	input	wire
mult_en_i	input	wire
prime_i	input	wire [REG_SIZE-1:0]
mult_mu_i	input	wire [RADIX-1 : 0]
opa_i	input	wire [REG_SIZE-1:0]
opb_i	input	wire [REG_SIZE-1:0]
add_res_o	output	wire [REG_SIZE-1:0]
mult_res_o	output	wire [REG_SIZE-1:0]

API — ecc_hmac_drbg_interface.sv

Table 1400. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_hmac_drbg_interface.sv — module / interface / package (1개)

이름	종류
ecc_hmac_drbg_interface	module

Table 1401. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_hmac_drbg_interface.sv — parameter (2개)

파라미터	형/폭	기본값
REG_SIZE		384
GROUP_ORDER	[REG_SIZE-1 : 0]	384'hffc7634d8...

Table 1402. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_hmac_drbg_interface.sv — localparam (12개)

상수	형/폭	값
KEYGEN_CMD	[1 : 0]	2'b00
SIGN_CMD	[1 : 0]	2'b01
DH_SHARED_CMD	[1 : 0]	2'b10
IDLE_ST	[3 : 0]	4'd0
LFSR_ST	[3 : 0]	4'd1
LAMBDA_ST	[3 : 0]	4'd2
SCALAR_RND_ST	[3 : 0]	4'd3
RND_DONE_ST	[3 : 0]	4'd4
MASKING_RND_ST	[3 : 0]	4'd5
KEYGEN_ST	[3 : 0]	4'd6
SIGN_ST	[3 : 0]	4'd7

상수	형/폭	값
DONE_ST	[3 : 0]	4'd8

Table 1403. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_hmac_drbg_interface.sv — 포트 (15개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
hmac_mode	input	wire [1 : 0]
en	input	wire
ready	output	wire
keygen_seed	input	wire [REG_SIZE-1 : 0]
keygen_nonce	input	wire [REG_SIZE-1 : 0]
privKey	input	wire [REG_SIZE-1 : 0]
hashed_msg	input	wire [REG_SIZE-1 : 0]
IV	input	wire [REG_SIZE-1 : 0]
lambda	output	wire [REG_SIZE-1 : 0]
scalar_rnd	output	wire [REG_SIZE-1 : 0]
masking_rnd	output	wire [REG_SIZE-1 : 0]
drbg	output	wire [REG_SIZE-1 : 0]

API — ecc_montgomerymultiplier.sv**Table 1404. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_montgomerymultiplier.sv — module / interface / package (1개)**

이름	종류
ecc_montgomerymultiplier	module

Table 1405. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_montgomerymultiplier.sv — parameter (2개)

파라미터	형/폭	기본값
REG_SIZE		384
RADIX		32

Table 1406. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_montgomerymultiplier.sv — localparam (7개)

상수	형/폭	값
S_NUM	int unsigned	((REG_SIZE + RADIX - 1) / RADIX) + 1
FULL_REG_SIZE	int unsigned	S_NUM * RADIX
PE_UNITS		((S_NUM - 1) / 2) - 1
t0_WIDTH	int	\$clog2((2*(PE_UNITS+1))+1)

상수	형/폭	값
t0_unsigned	bit [t0_WIDTH-1:0]	t0[t0_WIDTH-1:0]
t2_WIDTH	int	$\$clog2((2*(PE_UNITS+1))+1)$
t2_unsigned	bit [t2_WIDTH-1:0]	t2[t2_WIDTH-1:0]

Table 1407. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_montgomerymultiplier.sv — 포트 (10개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
start_i	input	wire
opa_i	input	wire [REG_SIZE-1:0]
opb_i	input	wire [REG_SIZE-1:0]
n_i	input	wire [REG_SIZE-1:0]
n_prime_i	input	wire [RADIX-1:0]
p_o	output	logic [REG_SIZE-1:0]
ready_o	output	logic

API — ecc_mult_dsp.sv**Table 1408. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_mult_dsp.sv — module / interface / package (1개)**

이름	종류
ecc_mult_dsp	module

Table 1409. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_mult_dsp.sv — parameter (1개)

파라미터	형/폭	기본값
RADIX		32

Table 1410. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_mult_dsp.sv — 포트 (3개)

포트	방향	형/폭
A_i	input	wire [RADIX-1 : 0]
B_i	input	wire [RADIX-1 : 0]
RADIX	output	logic [(2*

API — ecc_params_pkg.sv**Table 1411. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_params_pkg.sv — module / interface / package (1개)**

이름	종류
ecc_params_pkg	package

Table 1412. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_params_pkg.sv — parameter (30개)

파라미터	형/폭	기본값
KEYGEN		3'b001
SIGN		3'b010
VERIFY		3'b011
SHARED_KEY		3'b100
ECC_CORE_NAME	[63 : 0]	64'h38342D33_63707365
ECC_CORE_VERSION	[63 : 0]	64'h00000000_3030312e
REG_SIZE	[9 : 0]	10'd384
RND_SIZE	[9 : 0]	10'd192
DATA_WIDTH		32
REG_NUM_DWORDS		REG_SIZE/DATA_WIDTH
REG_OFFSET_W		\$clog2(REG_NUM_DWORDS)
MULT_RADIX		48
SCALAR_BLIND_RADIX		32
ADD_NUM_ADDS		1
ADD_BASE_SZ		384
PRIME	[REG_SIZE-1 : 0]	384'hff...
GROUP_ORDER	[REG_SIZE-1 : 0]	384'hffc7634d8...
ZERO_CONST	[REG_SIZE-1 : 0]	384'h0
ONE_CONST	[REG_SIZE-1 : 0]	384'h1
E_a_MONT	[REG_SIZE-1 : 0]	384'hffcfff...
E_b_MONT	[REG_SIZE-1 : 0]	384'hbfff9b62b21f41f022094e3374bee94938ae277f2209b1920022fc43...
E_3b_MONT	[REG_SIZE-1 : 0]	384'h3fed228165dc5d0661bea9a5e3cbbdbaa0a767d661d14b60068f4c9...
ONE_p_MONT	[REG_SIZE-1 : 0]	384'h100000000ffffffff00000001000000000000
R2_p_MONT	[REG_SIZE-1 : 0]	384'h10000000200000000ffffffe0000000000000200000000ffffff...
G_X_MONT	[REG_SIZE-1 : 0]	384'h1513812ff723614ede2b6454868459a30eff879c3afc541b4d6e6e1...
G_Y_MONT	[REG_SIZE-1 : 0]	384'hc5e9dd8002263969a840c6c3521968f4ffd98bade7562e83b050cd3...
PRIME_mu	[MULT_RADIX-1 : 0]	64'h100000001
R2_q_MONT	[REG_SIZE-1 : 0]	384'h28266895d40d49174aab1cc5bf030606de609f43cc9601f9ebbfed4...
ONE_q_MONT	[REG_SIZE-1 : 0]	384'h389cb27e0bc8d220a7e5f24db74f58851313e695333ad68d000000...
GROUP_ORDER_mu	[MULT_RADIX-1 : 0]	64'h6089e88fdc45

Table 1413. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_params_pkg.sv — #define / `define 매크로 (1개)

매크로	인자	정의
CALIPTRA_ECC_PARAMS_PKG		package ecc_params_pkg;

API — ecc_pe.sv**Table 1414. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe.sv — module / interface / package (1개)**

이름	종류
ecc_pe	module

Table 1415. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe.sv — parameter (1개)

파라미터	형/폭	기본값
RADIX		32

Table 1416. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe.sv — 포트 (15개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
start_in	input	wire
a_in	input	wire [RADIX-1:0]
b_in	input	wire [RADIX-1:0]
p_in	input	wire [RADIX-1:0]
m_in	input	wire [RADIX-1:0]
s_in	input	wire [RADIX-1:0]
c_in	input	wire [RADIX :0]
odd	input	wire
a_out	output	logic [RADIX-1:0]
m_out	output	logic [RADIX-1:0]
s_out	output	logic [RADIX-1:0]
c_out	output	logic [RADIX :0]

API — ecc_pe_final.sv**Table 1417. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe_final.sv — module / interface / package (1개)**

이름	종류
ecc_pe_final	module

Table 1418. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe_final.sv — parameter (1개)

파라미터	형/폭	기본값
RADIX		32

Table 1419. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe_final.sv — 포트 (13개)

포트	방향	형/폭
clk	input	wire

포트	방향	형/폭
reset_n	input	wire
zeroize	input	wire
start_in	input	wire
a_in	input	wire [RADIX-1:0]
b_in	input	wire [RADIX-1:0]
p_in	input	wire [RADIX-1:0]
m_in	input	wire [RADIX-1:0]
s_in	input	wire [RADIX-1:0]
c_in	input	wire [RADIX :0]
odd	input	wire
s_out	output	logic [RADIX-1:0]
c_out	output	logic [RADIX :0]

API — ecc_pe_first.sv

Table 1420. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe_first.sv — module / interface / package (1개)

이름	종류
ecc_pe_first	module

Table 1421. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe_first.sv — parameter (1개)

파라미터	형/폭	기본값
RADIX		32

Table 1422. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe_first.sv — 포트 (13개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
start_in	input	wire
a_in	input	wire [RADIX-1:0]
b_in	input	wire [RADIX-1:0]
p_in	input	wire [RADIX-1:0]
s_in	input	wire [RADIX-1:0]
n_prime_in	input	wire [RADIX-1:0]
odd	input	wire
a_out	output	logic [RADIX-1:0]
m_out	output	logic [RADIX-1:0]

포트	방향	형/폭
c_out	output	logic [RADIX :0]

API — ecc_pm_ctrl.sv

Table 1423. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pm_ctrl.sv — module / interface / package (1개)

이름	종류
ecc_pm_ctrl	module

Table 1424. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pm_ctrl.sv — parameter (3개)

파라미터	형/폭	기본값
REG_SIZE		384
RND_SIZE		192
INSTR_SIZE		24

Table 1425. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pm_ctrl.sv — localparam (4개)

상수	형/폭	값
MULT_DELAY	[7 : 0]	8'd27
ADD_DELAY	[7 : 0]	8'd1
Secp384_SCA_MONT_COUNT	[9 : 0]	REG_SIZE[9 : 0] + RND_SIZE[9 : 0]
Secp384_MONT_COUNT	[9 : 0]	REG_SIZE[9 : 0]

Table 1426. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pm_ctrl.sv — 포트 (8개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
ecc_cmd_i	input	wire [3 : 0]
digit_i	input	wire
instr_o	output	pm_instr_struct_t
req_digit_o	output	logic
busy_o	output	wire

```

1. chipsalliance_caliptra-rtl/src/ecc/rtl/
ecc_add_sub_mod_alter.sv (122 lines)
1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
```

```

11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 //
17 // ecc_add_sub_mod_alter.sv
18 // -----
19 // modular addition/subtraction module to compute opa+opb %
prime
20 //
21 //
22 //
=====
```

```

23
24 module ecc_add_sub_mod_alter #(
25     parameter REG_SIZE      = 384
26 )
27 (
28     // Clock and reset.
29     input wire      clk,
30     input wire      reset_n,
31     input wire      zeroize,
32
33     // DATA PORT
34     input wire      add_en_i,
35     input wire      sub_i,
36     input wire [REG_SIZE-1:0] opa_i,
37     input wire [REG_SIZE-1:0] opb_i,
38     input wire [REG_SIZE-1:0] prime_i,
39     output logic [REG_SIZE-1:0] res_o,
40     output logic      ready_o
41 );
42
43     logic [REG_SIZE-1 : 0] opb0;
44     logic [REG_SIZE-1 : 0] opb1;
45     logic [REG_SIZE-1 : 0] r0;
46     logic [REG_SIZE-1 : 0] r1;
47     logic      carry0;
48
49     logic      sub_n;
50     logic [REG_SIZE-1 : 0] r0_reg;
51     logic      carry0_reg;
52
53     logic      carry1;
54     logic [1 : 0]      push_result_reg;
55
56     ecc_adder #(
57         .RADIX(REG_SIZE)
58     )
59     adder_inst_0(
60         .a_i(opa_i),
61         .b_i(opb0),
62         .cin_i(sub_i),
63         .s_o(r0),
64         .cout_o(carry0)
65     );
66
67     ecc_adder #(
68         .RADIX(REG_SIZE)
69     )
70     adder_inst_1(
71         .a_i(r0_reg),
72         .b_i(opb1),
73         .cin_i(sub_n),
74         .s_o(r1),
75         .cout_o(carry1)
76     );
77
78     assign opb0 = sub_i ? -opb_i : opb_i;
79
80     always_ff @(posedge clk or negedge reset_n)
81     begin
82         if(!reset_n) begin
83             r0_reg <= '0;
84             carry0_reg <= '0;
85             sub_n <= '0;
86             opb1 <= '0;
87         end
88         else if (zeroize) begin
89             r0_reg <= '0;
90             carry0_reg <= '0;
91             sub_n <= '0;
92             opb1 <= '0;
93         end
94         else if (add_en_i) begin
95             r0_reg <= r0;
96             carry0_reg <= carry0;
97             sub_n <= !sub_i;
98             if (sub_i)
99                 opb1 <= prime_i;
100             else
101                 opb1 <= -prime_i;
102         end
103     end
104
105     // Determines when results are ready
106     always_ff @(posedge clk or negedge reset_n) begin
107         if (!reset_n)
108             push_result_reg <= 2'b0;
109         else if (zeroize)
110             push_result_reg <= 2'b0;
111         else if (add_en_i)
112             push_result_reg <= 2'b10;
113         else // one shift to right
114             push_result_reg <= 2'(push_result_reg >> 1);
115     end
116
117     assign ready_o = push_result_reg[0];
118
119     assign res_o = sub_n ? (carry0_reg ^ carry1)? r1 :
120     r0_reg : (carry0_reg) ? r0_reg : r1;
121
122 endmodule

```

2. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_adder.sv (46 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
4 // cense");
5 // you may not use this file except in compliance with the Li-
6 // cense.
7 // You may obtain a copy of the License at
8 // http://www.apache.org/licenses/LICENSE-2.0
9 // Unless required by applicable law or agreed to in writing,
10 // distributed under the License is distributed on an "AS IS"
11 // BASIS,
12 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
13 // press or implied.
14 // See the License for the specific language governing permis-
15 // sions and
16 // limitations under the License.
17 //
18 // =====
19 // Full adder module to compute a + b in RADIX bits and output
20 // add result
21 // with carry
22 //
23 // =====
24
25 module ecc_adder #(
26     parameter RADIX      = 8
27 )
28 (
29     // DATA PORT
30     input wire [RADIX-1:0] a_i,
31     input wire [RADIX-1:0] b_i,
32     input wire      cin_i,
33     output wire [RADIX-1:0] s_o,
34     output wire      cout_o
35 );
36
37     logic [RADIX : 0]      s_full;
38
39     always_comb begin
40         s_full = a_i + b_i + cin_i;
41     end
42
43     assign s_o = s_full[RADIX-1 : 0];
44     assign cout_o = s_full[RADIX];
45
46 endmodule

```

3. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_arith_unit.sv (224 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
4 // cense");
5 // you may not use this file except in compliance with the Li-
6 // cense.
7 // You may obtain a copy of the License at
8 // http://www.apache.org/licenses/LICENSE-2.0
9 // Unless required by applicable law or agreed to in writing,
10 // distributed under the License is distributed on an "AS IS"
11 // BASIS,
12 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
13 // press or implied.
14 // See the License for the specific language governing permis-
15 // sions and
16 // limitations under the License.
17 //
18 // =====
19 // ECC arithmetic unit to perform point multiplication includ-
20 // ing
21 // data memory, point mult (pm) controller, and field arith-
22 // metic
23 // - The module controlol memory acces from outside and pm
24 // controller
25 // - Secret key is stored in this madule's register instead of

```


[illegible]

```

202         if (wr_op_sel_i == 1'b0)
203             reg_web_r <= wr_en_i;
204         else
205             reg_web_r <= '0;
206
207         // Read multiplexer
208         if (rd_reg_i)
209             d_o <= opb_s;
210         else
211             d_o <= '0;
212     end
213 end // reg_update
214
215 //Push key bit to ecc pm control
216 assign digit_in = secret_key[0];
217
218 assign addrb_mux_s = ecc_busy_s ? ecc_instr_s.opb_addr :
reg_addr_r;
219 assign web_mux_s = ecc_busy_s ? ecc_in-
str_s.opcode.mult_we : reg_web_r;
220 assign dinb_mux_s = ecc_busy_s ?
mult_res_s : reg_dinb_r;
221 assign busy_o = ecc_busy_s;
222 assign data_o = d_o;
223
224 endmodule

```

4. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_defines_pkg.sv (40 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
=====
15 //
16 // ecc_defines_pkg.sv
17 // -----
18 // ECC interface parameters for the digital signature al-
gorithm (DSA).
19 //
20 //
21 //
=====
22
23 `ifndef CALIPTRA_ECC_DEFINES
24 `define CALIPTRA_ECC_DEFINES
25
26 package ecc_defines_pkg;
27
28 localparam ECC_ADDR_W = 32;
29 localparam ECC_DATA_W = 32;
30
31 //ECC REQ
32 typedef struct packed {
33     logic [ECC_ADDR_W-1:0] addr;
34     logic [ECC_DATA_W-1:0] wdata;
35     logic write;
36 } ecc_req_t;
37
38 endpackage
39
40 `endif

```

5. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_ctrl.sv (1,044 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,

```

```

11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 //
17 // ecc_dsa_ctrl.sv
18 // -----
19 // Elliptic Curve Cryptography (ECC) digital signature al-
gorithm (DSA)
20 // controller to support deterministic ECDSA based on RFC
6979.
21 // The dsa architecture includes several countermeasures to
be protected
22 // against a subset of side-channel analysis (SCA) attacks.
23 //
24 // The architecture includes:
25 // 1) ecc_dsa_sequencer: including the required sequence to
perform
26 //    keygen, signing, and verifying.
27 // 2) ecc_arith_unit: the arithmetic unit to perform point
multiplication
28 //    and other required operations
29 // 3) hmac_drbg: the hmac384 drbg component to support RFC
6979
30 // 4) ecc_scalar_blinding: the SCA countermeasure to random-
ized scalar
31 //    to avoid information leakage.
32 //
33 // The embedded countermeasures are:
34 // - point randomization:
35 //
36 // Randomized Projective Coordinates countermeasure based on
section 5.3
37 //
38 // "Resistance against Differential Power Analysis for
Elliptic Curve
39 // Cryptosystems" in CHES 1999 by Coron, J.
40 // This countermeasure uses a random value "lambda" in
point multiplication
41 // (Q = k * P) in keygen/signing to randomize base point
P = (X, Y, 1)
42 // as follows:
43 // P = (X * Lambda, Y * lambda, Lambda)
44 // - scalar blinding:
45 // Scalar blinding countermeasure based on section 5.1
46 // "Resistance against Differential Power Analysis for
Elliptic Curve
47 // Cryptosystems" in CHES 1999 by Coron, J.
48 // This countermeasure uses a random value "r" in the
point multiplication
49 // (Q = k * P) in keygen/signing to randomize scalar k as
follows:
50 // k = k + r*group_order
51 // - masking signature:
52 //
53 // Masking sign countermeasure uses a random value "d" in
the signing
54 //
55 // operation to generate signature proof s = (privkey * r +
h)*k_inv
56 // as follows:
57 // s = [((privkey - d) * r + (h - d)) * k_inv] + [(d * r
+ d) * k_inv]
58 // -----
59
60 `include "kv_macros.svh"
61
62 module ecc_dsa_ctrl
63     import ecc_params_pkg::*;
64     import ecc_dsa_uop_pkg::*;
65     import ecc_reg_pkg::*;
66     import kv_defines_pkg::*;
67
68     // Clock and reset.
69     input wire clk,
70     input wire reset_n,
71     input wire cptra_pwrgood,
72
73     // Reg ports.
74     input ecc_reg_out_t hwif_out,
75     output ecc_reg_in_t hwif_in,
76
77     // KV interface
78     output kv_read_t [1:0] kv_read,
79     output kv_write_t kv_write,
80     input kv_rd_resp_t [1:0] kv_rd_resp,
81     input kv_wr_resp_t kv_wr_resp,
82
83     //PCR Signing
84     input pcr_signing_t pcr_signing_data,
85     input logic ocp_lock_in_progress,
86     output logic busy_o,
87
88 endmodule

```

```

85 // Interrupts (from ecc_reg)
86 output logic error_intr,
87 output logic notif_intr,
88 input logic debugUnlock_or_scan_mode_switch
89 );
90
91 `include "caliptra_prim_assert.sv"
92
93 //-----
94 // Internal constant and parameter definitions.
95 //-----
96
97 localparam [RND_SIZE-1 : 0] zero_pad = '0;
98 localparam REG_NUM_DWORDS = REG_SIZE / DATA_WIDTH;
99 //-----
100 // Registers including update variables and write enable.
101 //-----
102 logic [DSA_PROG_ADDR_W-1 : 0] prog_cntr;
103
104 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] read_reg;
105 logic [(REG_SIZE+RND_SIZE)-1 : 0] write_reg;
106 logic [1 : 0] cycle_cnt;
107
108 logic zeroize_reg;
109
110 logic dsa_busy;
111 logic subcomponent_busy;
112 logic pm_busy_o;
113
114 logic hw_privkey_we;
115 logic privkey_we_reg;
116 logic sharedkey_we_reg;
117 logic secretkey_we;
118 logic hw_pubkeyx_we;
119 logic hw_pubkeyy_we;
120 logic hw_r_we;
121 logic hw_s_we;
122 logic hw_scalar_G_we;
123 logic hw_scalar_PK_we;
124 logic hw_verify_r_we;
125 logic hw_pk_chk_we;
126 logic hw_sharedkey_we;
127 logic scalar_G_sel;
128
129 //Explicit hardware signature verification result
130 logic ecc_cmd_start;
131 logic ecc_verify_start;
132 logic ecc_verify_res_match;
133 logic ecc_verify_pass_reg;
134
135 logic ecc_valid_reg;
136 logic ecc_ready_reg;
137
138 logic ecc_status_done_d;
139 logic ecc_status_done_p;
140
141 logic [2 : 0] cmd_reg;
142 logic [3 : 0] pm_cmd_reg;
143 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] msg_reg;
144 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] msg_re-
duced_reg;
145 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] privkey_reg;
146 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] kv_reg;
147 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] pubkeyx_reg;
148 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] pubkeyy_reg;
149 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] seed_reg;
150 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] nonce_reg;
151 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] r_reg;
152 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] s_reg;
153 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] IV_reg;
154 logic [REG_SIZE-1 : 0] lambda_reg;
155 logic [REG_SIZE-1 : 0] masking_rnd_reg;
156 logic [REG_SIZE-1 : 0] pk_chk_reg;
157
158 logic [REG_SIZE-1 : 0] scalar_G_reg;
159 logic [REG_SIZE-1 : 0] scalar_PK_reg;
160
161 logic [REG_SIZE-1 : 0] scalar_in_reg;
162 logic [REG_SIZE-1 : 0] scalar_rnd_reg;
163 logic [(REG_SIZE+RND_SIZE)-1 : 0] scalar_out_reg;
164 logic scalar_sca_en;
165 logic scalar_sca_busy_o;
166
167 logic [1 : 0] hmac_mode;
168 logic hmac_init;
169 logic hmac_ready;
170 logic [REG_SIZE-1 : 0] hmac_drbg_result;
171 logic hmac_busy;
172
173 //interface with kv client
174 logic kv_privkey_write_en;
175 logic [REG_OFFSET_W-1:0] kv_privkey_write_offset;
176 logic [31:0] kv_privkey_write_data;
177 logic kv_seed_write_en;
178 logic [REG_OFFSET_W-1:0] kv_seed_write_offset;
179 logic [31:0] kv_seed_write_data;
180
181 logic dest_keyvault;
182 kv_error_code_e kv_privkey_error, kv_seed_error,
kv_nonce_error, kv_write_error;
183 logic kv_privkey_ready, kv_privkey_done;
184 logic kv_seed_ready, kv_seed_done;
185 logic kv_write_ready, kv_write_done;
186 //KV Seed Data Present
187 logic kv_seed_data_present;
188 logic kv_seed_data_present_set,
kv_seed_data_present_reset;
189 //KV Key Data Present
190 logic kv_key_data_present;
191 logic kv_key_data_present_set, kv_key_data_present_reset;
192
193 kv_read_ctrl_reg_t kv_privkey_read_ctrl_reg;
194 kv_read_ctrl_reg_t kv_seed_read_ctrl_reg;
195 kv_write_ctrl_reg_t kv_write_ctrl_reg;
196 kv_read_filter_metrics_t kv_privkey_read_metrics;
197 kv_read_filter_metrics_t kv_seed_read_metrics;
198 kv_write_filter_metrics_t kv_write_metrics;
199
200 logic pcr_sign_mode;
201
202 instr_struct_t prog_instr;
203
204 // ERROR
205 logic keygen_process;
206 logic signing_process;
207 logic verifying_process;
208 logic sharedkey_process;
209
210 logic privkey_input_outofrange;
211 logic r_output_outofrange;
212 logic s_output_outofrange;
213 logic r_input_outofrange;
214 logic s_input_outofrange;
215 logic pubkeyx_input_outofrange;
216 logic pubkeyy_input_outofrange;
217 logic pubkey_input_invalid;
218 logic pcr_sign_input_invalid;
219 logic privkey_output_outofrange, pub-
key_output_outofrange, pubkeyy_output_outofrange;
220 logic sharedkey_outofrange;
221
222 logic error_flag;
223 logic error_flag_reg;
224 logic error_flag_edge;
225
226 //-----
227 // Module instantiations.
228 //-----
229
230 ecc_dsa_sequencer #(
231 .ADDR_WIDTH(DSA_PROG_ADDR_W),
232 .DATA_WIDTH(DSA_INSTRUCTION_LENGTH)
233 )
234 ecc_dsa_sequencer_i(
235 .clk(clk),
236 .reset_n(reset_n),
237 .zeroize(zeroize_reg),
238 .ena(1'b1),
239 .addr(prog_cntr),
240 .douta(prog_instr)
241 );
242
243 ecc_arith_unit #(
244 .REG_SIZE(REG_SIZE),
245 .RND_SIZE(RND_SIZE),
246 .RADIX(MULT_RADIX),
247 .ADDR_WIDTH(DSA_OPR_ADDR_WIDTH),
248 .p_prime(PRIME),
249 .p_mu(PRIME_mu),
250 .q_grouporder(GROUP_ORDER),
251 .q_mu(GROUP_ORDER_mu)
252 )
253 ecc_arith_unit_i (
254 .clk(clk),
255 .reset_n(reset_n),
256 .zeroize(zeroize_reg),
257 .ecc_cmd_i(pm_cmd_reg),
258 .addr_i(prog_instr.mem_addr),
259 .wr_op_sel_i(prog_instr.opcode.op_sel),
260 .wr_en_i(prog_instr.opcode.wr_en),
261 .rd_reg_i(prog_instr.opcode.rd_en),
262 .data_i(write_reg),
263 .data_o(read_reg),
264 .busy_o(pm_busy_o)
265 );
266
267 ecc_hmac_drbg_interface #(
268 .REG_SIZE(REG_SIZE),
269 .GROUP_ORDER(GROUP_ORDER)
270 )
271 ecc_hmac_drbg_interface_i (
272 .clk(clk),
273 .reset_n(reset_n),
274 .zeroize(zeroize_reg),
275 .hmac_mode(hmac_mode),
276 .en(hmac_init),

```

```

277     .ready(hmac_ready),
278     .keygen_seed(seed_reg),
279     .keygen_nonce(nonce_reg),
280     .privKey(privkey_reg),
281     .hashed_msg(msg_reduced_reg),
282     .IV(IV_reg),
283     .lambda(lambda_reg),
284     .scalar_rnd(scalar_rnd_reg),
285     .masking_rnd(masking_rnd_reg),
286     .drbg(hmac_drbg_result)
287 );
288
289 ecc_scalar_blinding #(
290     .REG_SIZE(REG_SIZE),
291     .RND_SIZE(RND_SIZE),
292     .RADIX(SCALAR_BLIND_RADIX),
293     .GROUP_ORDER(GROUP_ORDER)
294 )
295 ecc_scalar_blinding_i(
296     .clk(clk),
297     .reset_n(reset_n),
298     .zeroize(zeroize_reg),
299     .en_i(scalar_sca_en),
300     .data_i(scalar_in_reg),
301     .rnd_i(scalar_rnd_reg[RND_SIZE-1 : 0]),
302     .data_o(scalar_out_reg),
303     .busy_o(scalar_sca_busy_o)
304 );
305
306 //-----
307 // ecc_reg_update
308
309 // Update functionality for all interface registers in the
310 core.
311 //-----
312 // read the registers written by sw
313 always_comb begin
314     //Mask the command if KV clients are not idle
315     cmd_reg = {hwif_out.ECC_CTRL.DH_SHAREDKEY.value,
316     hwif_out.ECC_CTRL.CTRL.value} & {3{kv_seed_ready}} &
317     {3{kv_privkey_ready}};
318     zeroize_reg = hwif_out.ECC_CTRL.ZEROIZE.value || debu-
319     gUnlock_or_scan_mode_switch;
320 end
321
322 //there is a clk cycle memory read delay between
323 hw_privkey_we and read_reg
324 always_ff @(posedge clk or negedge reset_n)
325 begin : ecc_kv_reg
326     if (!reset_n) begin
327         privkey_we_reg <= '0;
328         sharedkey_we_reg <= '0;
329         kv_reg <= '0;
330         kv_seed_data_present <= '0;
331         kv_key_data_present <= '0;
332     end
333     else if (zeroize_reg) begin
334         privkey_we_reg <= '0;
335         sharedkey_we_reg <= '0;
336         kv_reg <= '0;
337         kv_seed_data_present <= '0;
338         kv_key_data_present <= '0;
339     end
340     //Store private key here before pushing to keyvault
341     else begin
342         privkey_we_reg <= hw_privkey_we;
343         sharedkey_we_reg <= hw_sharedkey_we;
344         if (secretkey_we & (dest_keyvault |
345         kv_seed_data_present))
346             kv_reg <= read_reg;
347     end
348 end
349
350 kv_seed_data_present <= kv_seed_data_present_set ?
351 '1 :
352 kv_seed_data_present_reset ? '0 : kv_seed_data_present;
353 kv_key_data_present <= kv_key_data_present_set ?
354 '1 :
355 kv_key_data_present_reset ? '0 : kv_key_data_present;
356 end
357
358 always_comb secretkey_we = (privkey_we_reg |
359 sharedkey_we_reg);
360
361 assign error_intr = hwif_out.intr_b-
362 lock_rf.error_global_intr_r.intr;
363 assign notif_intr = hwif_out.intr_b-
364 lock_rf.notif_global_intr_r.intr;
365
366 // write the registers by hw
367 always_comb hwif_in.reset_b = reset_n;
368 always_comb hwif_in.hard_reset_b = cptra_pwrgood;
369 always_comb hwif_in.ecc_ready = ecc_ready_reg;
370 always_comb hwif_in.ECC_NAME[0].NAME.next =
371 ECC_CORE_NAME[31 : 0];

```

```

360     always_comb hwif_in.ECC_NAME[1].NAME.next =
361 ECC_CORE_NAME[63 : 32];
362     always_comb hwif_in.ECC_VERSION[0].VERSION.next =
363 ECC_CORE_VERSION[31 : 0];
364     always_comb hwif_in.ECC_VERSION[1].VERSION.next =
365 ECC_CORE_VERSION[63 : 32];
366
367 always_comb hwif_in.ECC_STATUS.READY.next = ecc_ready_reg;
368 always_comb hwif_in.ECC_STATUS.VALID.next = ecc_valid_reg;
369 // Only expose the verification result once the engine is
370 idle again.
371 always_comb hwif_in.ECC_STATUS.VERIFY_PASS.next =
372 ecc_verify_pass_reg & ~error_flag_reg & ecc_ready_reg;
373
374 always_comb begin // ecc_reg_writing
375     for (int dword=0; dword < 12; dword++)begin
376         //Key Vault has priority if enabled to drive these
377         registers
378         //don't store the private key generated in sw ac-
379         cessible register if it's going to keyvault
380         privkey_reg[dword] = hwif_out.ECC_PRIVKEY_IN[11-
381         dword].PRIVKEY_IN.value;
382         hwif_in.ECC_PRIVKEY_IN[dword].PRIVKEY_IN.we =
383         (pcr_sign_mode | (kv_privkey_write_en & (kv_privkey_write_offset ==
384         dword))) & !zeroize_reg;
385         hwif_in.ECC_PRIVKEY_IN[dword].PRIVKEY_IN.next =
386         pcr_sign_mode
387         ? pcr_sign-
388         ing_data.pcr_ecc_signing_privkey[dword] :
389         kv_privkey_write_en ? kv_privkey_write_data :
390
391         read_reg[11-dword];
392         hwif_in.ECC_PRIVKEY_IN[dword].PRIVKEY_IN.hwclr =
393         zeroize_reg | kv_key_data_present_reset | (kv_privkey_error !=
394         KV_SUCCESS);
395         hwif_in.ECC_PRIVKEY_IN[dword].PRIVKEY_IN.swwe =
396         ecc_ready_reg & ~kv_key_data_present;
397     end
398     for (int dword=0; dword < 12; dword++)begin
399         //If keyvault is not enabled, grab the sw value as
400         usual
401         hwif_in.ECC_PRIVKEY_OUT[dword].PRIVKEY_OUT.we =
402         (privkey_we_reg & ~(dest_keyvault | kv_seed_data_present)) & !zero-
403         ize_reg;
404         hwif_in.ECC_PRIVKEY_OUT[dword].PRIVKEY_OUT.next =
405         read_reg[11-dword];
406         hwif_in.ECC_PRIVKEY_OUT[dword].PRIVKEY_OUT.hwclr =
407         zeroize_reg;
408     end
409     for (int dword=0; dword < 12; dword++)begin
410         seed_reg[dword] = hwif_out.ECC_SEED[11-
411         dword].SEED.value;
412         hwif_in.ECC_SEED[dword].SEED.we =
413         (kv_seed_write_en & (kv_seed_write_offset == dword)) & !zeroize_reg;
414         hwif_in.ECC_SEED[dword].SEED.next =
415         kv_seed_write_data;
416         hwif_in.ECC_SEED[dword].SEED.hwclr = zeroize_reg |
417         kv_seed_data_present_reset | (kv_seed_error != KV_SUCCESS);
418         hwif_in.ECC_SEED[dword].SEED.swwe = ecc_ready_reg &
419         ~kv_seed_data_present;
420     end
421     for (int dword=0; dword < 12; dword++)begin
422         nonce_reg[dword] = hwif_out.ECC_NONCE[11-
423         dword].NONCE.value;
424         hwif_in.ECC_NONCE[dword].NONCE.hwclr = zero-
425         ize_reg;
426     end
427     for (int dword=0; dword < 12; dword++)begin
428         msg_reg[dword] = hwif_out.ECC_MSG[11-
429         dword].MSG.value;
430         hwif_in.ECC_MSG[dword].MSG.we = pcr_sign_mode & !
431         zeroize_reg;
432         hwif_in.ECC_MSG[dword].MSG.next = pcr_sign-
433         ing_data.pcr_hash[dword];
434         hwif_in.ECC_MSG[dword].MSG.hwclr = zeroize_reg;
435     end
436     for (int dword=0; dword < 12; dword++)begin
437         pubkeyx_reg[dword] = hwif_out.ECC_PUBKEY_X[11-
438         dword].PUBKEY_X.value;
439         hwif_in.ECC_PUBKEY_X[dword].PUBKEY_X.we = hw-pub-
440         keyx_we & !zeroize_reg;
441         hwif_in.ECC_PUBKEY_X[dword].PUBKEY_X.next =
442         read_reg[11-dword];
443         hwif_in.ECC_PUBKEY_X[dword].PUBKEY_X.hwclr = zero-
444         ize_reg;
445     end
446     for (int dword=0; dword < 12; dword++)begin
447         pubkeyy_reg[dword] = hwif_out.ECC_PUBKEY_Y[11-

```

```

dword].PUBKEY_Y.value;
419 hwif_in.ECC_PUBKEY_Y[dword].PUBKEY_Y.we = hw_pub-
key_we & !zeroize_reg;
420 hwif_in.ECC_PUBKEY_Y[dword].PUBKEY_Y.next =
read_reg[11-dword];
421 hwif_in.ECC_PUBKEY_Y[dword].PUBKEY_Y.hwclr = zero-
ize_reg;
422 end
423
424 for (int dword=0; dword < 12; dword++)begin
425 r_reg[dword] = hwif_out.ECC_SIGN_R[11-
dword].SIGN_R.value;
426 hwif_in.ECC_SIGN_R[dword].SIGN_R.we = hw_r_we & !
zeroize_reg;
427 hwif_in.ECC_SIGN_R[dword].SIGN_R.next =
read_reg[11-dword];
428 hwif_in.ECC_SIGN_R[dword].SIGN_R.hwclr = zero-
ize_reg;
429 end
430
431 for (int dword=0; dword < 12; dword++)begin
432 s_reg[dword] = hwif_out.ECC_SIGN_S[11-
dword].SIGN_S.value;
433 hwif_in.ECC_SIGN_S[dword].SIGN_S.we = hw_s_we & !
zeroize_reg;
434 hwif_in.ECC_SIGN_S[dword].SIGN_S.next =
read_reg[11-dword];
435 hwif_in.ECC_SIGN_S[dword].SIGN_S.hwclr = zero-
ize_reg;
436 end
437
438 for (int dword=0; dword < 12; dword++)begin
439 //Preload with the complement of the supplied sig-
nature R at the start of a
440 //VERIFY operation so an aborted verification can-
not leave a stale
match
441 hwif_in.ECC_VERIFY_R[dword].VERIFY_R.we =
(hw_verify_r_we |
ecc_verify_start) & !zeroize_reg;
442 hwif_in.ECC_VERIFY_R[dword].VERIFY_R.next =
ecc_verify_start ? ~r_reg[11-dword] : read_reg[11-dword];
443 hwif_in.ECC_VERIFY_R[dword].VERIFY_R.hwclr = zero-
ize_reg;
444 end
445
446 for (int dword=0; dword < 12; dword++)begin
447 IV_reg[dword] = hwif_out.ECC_IV[11-
dword].IV.value;
448 hwif_in.ECC_IV[dword].IV.hwclr = zeroize_reg;
449 end
450
451 for (int dword=0; dword < 12; dword++)begin
452 hwif_in.ECC_DH_SHARED_KEY[dword].DH_SHARED_KEY.we
= (sharedkey_we_reg & ~(dest_keyvault | kv_key_data_present)) & !
zeroize_reg;
453
hwif_in.ECC_DH_SHARED_KEY[dword].DH_SHARED_KEY.next = read_reg[11-
dword];
454
hwif_in.ECC_DH_SHARED_KEY[dword].DH_SHARED_KEY.hwclr = zeroize_reg;
455 end
456
457 //transformed msg into modulo q
458 always_ff @(posedge clk or negedge reset_n)
459 begin : reduced_msg
460 if (!reset_n)
461 msg_reduced_reg <= '0;
462 else if (zeroize_reg)
463 msg_reduced_reg <= '0;
464 else begin
465 if (msg_reg >= GROUP_ORDER)
466 msg_reduced_reg <= msg_reg - GROUP_ORDER;
467 else
468 msg_reduced_reg <= msg_reg;
469 end
470 end
471
472
473
474 always_comb hwif_in.ECC_CTRL.CTRL.hwclr = |cmd_reg;
475 always_comb hwif_in.ECC_CTRL.DH_SHAREDKEY.hwclr = |
cmd_reg;
476 always_comb hwif_in.ECC_CTRL.PCR_SIGN.hwclr =
hwif_out.ECC_CTRL.PCR_SIGN.value;
477
478 // TODO add other interrupt hwset signals (errors)
479 always_comb hwif_in.intr_b-
lock_rf.error_internal_intr_r.error_internal_sts.hwset = er-
ror_flag_edge;
480 always_comb hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset =
ecc_status_done_p;
481
482
483 always_comb begin: ecc_kv_ctrl_reg
484 //ready when fsm is not busy
485 hwif_in.ecc_kv_rd_pkey_status.ERROR.next =
kv_privkey_error;
486 hwif_in.ecc_kv_rd_seed_status.ERROR.next =
kv_seed_error;
487 hwif_in.ecc_kv_wr_pkey_status.ERROR.next =
kv_write_error;

```

```

488 //ready when fsm is not busy
489 hwif_in.ecc_kv_rd_pkey_status.READY.next =
kv_privkey_ready;
490 hwif_in.ecc_kv_rd_seed_status.READY.next =
kv_seed_ready;
491 hwif_in.ecc_kv_wr_pkey_status.READY.next =
kv_write_ready;
492 //set valid when fsm is done
493 hwif_in.ecc_kv_rd_pkey_status.VALID.hwset =
kv_privkey_done;
494 hwif_in.ecc_kv_rd_seed_status.VALID.hwset =
kv_seed_done;
495 hwif_in.ecc_kv_wr_pkey_status.VALID.hwset =
kv_write_done;
496 //clear valid when new request is made
497 hwif_in.ecc_kv_rd_pkey_status.VALID.hwclr =
kv_privkey_read_ctrl_reg.read_en;
498 hwif_in.ecc_kv_rd_seed_status.VALID.hwclr =
kv_seed_read_ctrl_reg.read_en;
499 hwif_in.ecc_kv_wr_pkey_status.VALID.hwclr =
kv_write_ctrl_reg.write_en;
500 //clear enable when busy
501 hwif_in.ecc_kv_rd_pkey_ctrl.read_en.hwclr =
~kv_privkey_ready;
502 hwif_in.ecc_kv_rd_seed_ctrl.read_en.hwclr =
~kv_seed_ready;
503 hwif_in.ecc_kv_wr_pkey_ctrl.write_en.hwclr =
~kv_write_ready;
504 end
505
506 // Software write-enables to prevent KV reg manipulation
mid-operation
507 always_comb hwif_in.ecc_kv_rd_p-
key_ctrl.read_en.swwe = !kv_key_data_present && !busy_o;
508 always_comb hwif_in.ecc_kv_rd_p-
key_ctrl.read_entry.swwe = !kv_key_data_present && !busy_o;
509 always_comb hwif_in.ecc_kv_rd_p-
key_ctrl.pcr_hash_extend.swwe = 0; //NA for keyvault
510 always_comb hwif_in.ecc_kv_rd_p-
key_ctrl.rsvd.swwe = 0;
511
512 always_comb
hwif_in.ecc_kv_rd_seed_ctrl.read_en.swwe = !
kv_seed_data_present && !busy_o;
513 always_comb
hwif_in.ecc_kv_rd_seed_ctrl.read_entry.swwe = !
kv_seed_data_present && !busy_o;
514 always_comb
hwif_in.ecc_kv_rd_seed_ctrl.pcr_hash_extend.swwe = 0; //NA for
keyvault
515 always_comb
hwif_in.ecc_kv_rd_seed_ctrl.rsvd.swwe = 0;
516
517 // KV write control must be written before ECC core opera-
tion begins, even though
518 // output isn't written to KV until the end of the opera-
tion.
519 // Prevent partial-key attacks by blocking register modi-
fications during core execution.
520 always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.write_en.swwe = !busy_o;
521 always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.write_entry.swwe = !busy_o;
522 always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.hmac_key_dest_valid.swwe = !busy_o;
523 always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.hmac_block_dest_valid.swwe = !busy_o;
524 always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.mlds_seed_dest_valid.swwe = !busy_o;
525 always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.ecc_pkey_dest_valid.swwe = !busy_o;
526 always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.ecc_seed_dest_valid.swwe = !busy_o;
527 always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.aes_key_dest_valid.swwe = !busy_o;
528 always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.mlkem_seed_dest_valid.swwe = !busy_o;
529 always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.mlkem_msg_dest_valid.swwe = !busy_o;
530 always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.dma_data_dest_valid.swwe = !busy_o;
531 always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.rsvd.swwe = 0;
532
533 //keyvault control reg macros for assigning to struct
`CALIPT-
RA_KV_READ_CTRL_REG2STRUCT(kv_privkey_read_ctrl_reg, ecc_kv_rd_p-
key_ctrl)
534
535 `CALIPTRA_KV_READ_CTRL_REG2STRUCT(kv_seed_read_ctrl_reg,
ecc_kv_rd_seed_ctrl)
536
537 `CALIPTRA_KV_WRITE_CTRL_REG2STRUCT(kv_write_ctrl_reg,
ecc_kv_wr_pkey_ctrl)
538
539 //Detect keyvault data coming in to lock api registers and
protect outputs
540 always_comb kv_seed_data_present_set =
kv_seed_read_ctrl_reg.read_en;
541 always_comb kv_seed_data_present_reset =
kv_seed_data_present & ecc_status_done_p;
542
543

```



```

542     always_comb kv_key_data_present_set =
kv_privkey_read_ctrl_reg.read_en;
543     always_comb kv_key_data_present_reset =
kv_key_data_present & ecc_status_done_p;
544
545     always_comb pcr_sign_mode = hwif_out.ECC_C-
TRL.PCR_SIGN.value;
546
547     //-----
548     // register updates
549     //
550     // update the internal registers and their wr_en
551     //-----
552     always_ff @(posedge clk or negedge reset_n)
553     begin : SCALAR_REG
554         if(!reset_n) begin
555             scalar_g_reg <= '0;
556             scalar_pk_reg <= '0;
557         end
558         else if(zeroize_reg) begin
559             scalar_g_reg <= '0;
560             scalar_pk_reg <= '0;
561         end
562         else begin
563             if (!scalar_g_sel)
564                 scalar_g_reg <= hmac_drbg_result;
565             else if (hw_scalar_g_we)
566                 scalar_g_reg <= read_reg;
567
568             if (hw_scalar_pk_we)
569                 scalar_pk_reg <= read_reg;
570         end
571     end
572
573     // Set the write enable flag for different register based
on sequencer output of "DSA_UOP_RD_CORE"
574     // to read them from data memory inside ecc_arith_unit
575     always_comb
576     begin : wr_en_signals
577         hw_privkey_we = 0;
578         hw_pubkeyx_we = 0;
579         hw_pubkeyy_we = 0;
580         hw_r_we = 0;
581         hw_s_we = 0;
582         hw_scalar_g_we = 0;
583         hw_scalar_pk_we = 0;
584         hw_verify_r_we = 0;
585         hw_pk_chk_we = 0;
586         hw_sharedkey_we = 0;
587         if ((prog_instr.opcode == DSA_UOP_RD_CORE) &
(cycle_cnt == 0)) begin
588             unique case (prog_instr.reg_id)
589                 PRIVKEY_ID      : hw_privkey_we = 1;
590                 PUBKEYX_ID      : hw_pubkeyx_we = 1;
591                 PUBKEYY_ID      : hw_pubkeyy_we = 1;
592                 R_ID            : hw_r_we = 1;
593                 S_ID            : hw_s_we = 1;
594                 SCALAR_G_ID     : hw_scalar_g_we = 1;
595                 SCALAR_PK_ID    : hw_scalar_pk_we = 1;
596                 VERIFY_R_ID     : hw_verify_r_we = 1;
597                 PK_VALID_ID     : hw_pk_chk_we = 1;
598                 DH_SHAREDKEY_ID : hw_sharedkey_we = 1;
599                 default         :
600                     begin
601                         hw_privkey_we = 0;
602                         hw_pubkeyx_we = 0;
603                         hw_pubkeyy_we = 0;
604                         hw_r_we = 0;
605                         hw_s_we = 0;
606                         hw_scalar_g_we = 0;
607                         hw_scalar_pk_we = 0;
608                         hw_verify_r_we = 0;
609                         hw_pk_chk_we = 0;
610                         hw_sharedkey_we = 0;
611                     end
612             endcase
613         end
614     end // wr_en_signals
615
616     // set the write register value based on sequencer output
of "DSA_UOP_WR_CORE"
617     // to write into data memory inside ecc_arith_unit
618     always_comb
619     begin : write_to_pm_core
620         write_reg = '0;
621         if (prog_instr.opcode == DSA_UOP_WR_CORE) begin
622             unique case (prog_instr.reg_id)
623
624                 CONST_ZERO_ID      : write_reg = {zero_pad,
ZERO_CONST};
625
626                 CONST_ONE_ID       : write_reg = {zero_pad,
ONE_CONST};
627
628                 CONST_E_a_MONT_ID  : write_reg = {zero_pad,
E_a_MONT};
629
630                 CONST_E_b_MONT_ID  : write_reg = {zero_pad,
E_b_MONT};
631
632                 CONST_E_3b_MONT_ID : write_reg = {zero_pad,
E_3b_MONT};
633
634                 CONST_ONE_p_MONT_ID : write_reg = {zero_pad,
ONE_p_MONT};
635
636                 CONST_R2_p_MONT_ID : write_reg = {zero_pad,
R2_p_MONT};
637
638                 CONST_G_X_MONT_ID  : write_reg = {zero_pad,
G_X_MONT};
639
640                 CONST_G_Y_MONT_ID  : write_reg = {zero_pad,
G_Y_MONT};
641
642                 CONST_R2_q_MONT_ID : write_reg = {zero_pad,
R2_q_MONT};
643
644                 CONST_ONE_q_MONT_ID : write_reg = {zero_pad,
ONE_q_MONT};
645
646                 MSG_ID              : write_reg = {zero_pad,
msg_reduced_reg};
647
648                 PRIVKEY_ID          : write_reg = {zero_pad,
privkey_reg};
649
650                 PUBKEYX_ID          : write_reg = {zero_pad,
pubkeyx_reg};
651
652                 PUBKEYY_ID          : write_reg = {zero_pad,
pubkeyy_reg};
653
654                 R_ID                : write_reg = {zero_pad,
r_reg};
655
656                 S_ID                : write_reg = {zero_pad,
s_reg};
657
658                 SCALAR_G_ID         : write_reg = {zero_pad,
scalar_g_reg};
659
660                 LAMBDA_ID           : write_reg = {zero_pad,
lambda_reg};
661
662                 MASKING_ID          : write_reg = {zero_pad,
masking_rnd_reg};
663
664                 default             : write_reg = '0;
665             endcase
666         end
667         else if (prog_instr.opcode == DSA_UOP_WR_SCALAR) begin
668             unique case (prog_instr.reg_id)
669                 SCALAR_PK_ID      : write_reg = (REG_S-
IZE+RND_SIZE)'(scalar_pk_reg << RND_SIZE);
670                 SCALAR_G_ID       : write_reg = (REG_S-
IZE+RND_SIZE)'(scalar_g_reg << RND_SIZE);
671                 SCALAR_ID         : write_reg = scal-
ar_out_reg; // SCA
672                 default           : write_reg = '0;
673             endcase
674         end // write_to_pm_core
675
676     // Set the input scalar for scalar_blinding module
always_ff @(posedge clk or negedge reset_n)
677     begin : scalar_sca_ctrl
678         if(!reset_n) begin
679             scalar_in_reg <= '0;
680         end
681         else if(zeroize_reg) begin
682             scalar_in_reg <= '0;
683         end
684         else begin
685             if (prog_instr.opcode == DSA_UOP_SCALAR_SCA) begin
686                 scalar_in_reg <= scalar_g_reg;
687             end
688         end
689     end // scalar_sca_ctrl
690
691     assign hmac_busy = ~hmac_ready;
692     assign subcomponent_busy = pm_busy_o | hmac_busy | scal-
ar_sca_busy_o;
693
694     //-----
695     // ERROR detection
696     //
697     // bound check for different input/output registers.
698     //-----
699     always_ff @(posedge clk or negedge reset_n)
700     begin : pk_chk
701         if(!reset_n) begin
702             pk_chk_reg <= '0;
703         end
704         else if(zeroize_reg) begin
705             pk_chk_reg <= '0;
706         end
707     end

```

```

690     else begin
691         if (hw_pk_chk_we) begin
692             pk_chk_reg <= read_reg;
693         end
694     end
695 end // pk_chk
696
697 always_ff @(posedge clk or negedge reset_n)
698 begin : error_detection
699     if(!reset_n)
700         error_flag_reg <= 1'b0;
701     else if(zeroize_reg)
702         error_flag_reg <= 1'b0;
703     else if (error_flag)
704         error_flag_reg <= 1'b1;
705 end // error_detection
706
707 assign error_flag_edge = error_flag & (!error_flag_reg);
708
709 assign privkey_input_outofrange = signing_process &
((privkey_reg == 0) | (privkey_reg >= GROUP_ORDER));
710 assign r_output_outofrange = signing_process &
(hw_r_we & ((read_reg == 0) | (read_reg >= GROUP_ORDER)));
711 assign s_output_outofrange = signing_process &
(hw_s_we & ((read_reg == 0) | (read_reg >= GROUP_ORDER)));
712
713 assign r_input_outofrange = verifying_process &
((r_reg == 0) | (r_reg >= GROUP_ORDER));
714 assign s_input_outofrange = verifying_process &
((s_reg == 0) | (s_reg >= GROUP_ORDER));
715 assign pubkeyx_input_outofrange = (verifying_process |
sharedkey_process) & (pubkeyx_reg >= PRIME);
716 assign pubkeyy_input_outofrange = (verifying_process |
sharedkey_process) & (pubkeyy_reg >= PRIME);
717 assign pubkey_input_invalid = (verifying_process |
sharedkey_process) & (pk_chk_reg != 0);
718
719 assign pcr_sign_input_invalid = ((cmd_reg == KEYGEN) |
(cmd_reg == VERIFY) | (cmd_reg == SHARED_KEY)) & pcr_sign_mode;
720
721 assign privkey_output_outofrange = keygen_process &
(hw_privkey_we & ((read_reg == 0) | (read_reg >= GROUP_ORDER)));
722 assign pubkeyx_output_outofrange = keygen_process &
(hw_pubkeyx_we & (read_reg >= PRIME));
723 assign pubkeyy_output_outofrange = keygen_process &
(hw_pubkeyy_we & (read_reg >= PRIME));
724
725 assign sharedkey_outofrange = sharedkey_process &
(hw_sharedkey_we & (read_reg >= PRIME));
726
727 assign error_flag = privkey_input_outofrange | r_out-
put_outofrange | s_output_outofrange |
728 r_input_outofrange | s_in-
put_outofrange | pubkeyx_input_outofrange | pubkeyy_input_outofrange |
729 pubkey_input_invalid |
pcr_sign_input_invalid |
730 privkey_output_outofrange | pub-
keyx_output_outofrange | pubkeyy_output_outofrange | pub-
731 sharedkey_outofrange;
732
733 //-----
734 // Explicit hardware signature verification result
735 //
736 // The recomputed R is compared against the supplied sig-
nature R by
737 // hardware, in the exact same cycle (and with the exact
same data)
738 // that populates ECC_VERIFY_R. This guarantees the repor-
ted pass/fail
739 // bit and the data returned to firmware can never dis-
agree. Firmware
740 // may still compare ECC_VERIFY_R against its own copy of
R as a
741 // fault-attack check on this bit.
742 //-----
743 always_comb ecc_cmd_start = (prog_cntr == ECC_NOP) & (!
cmd_reg);
744 always_comb ecc_verify_start = (prog_cntr == ECC_NOP) &
(cmd_reg == VERIFY);
745 always_comb ecc_verify_res_match = (read_reg == r_reg) &
(!r_reg);
746
747 always_ff @(posedge clk or negedge reset_n)
748 begin : ecc_verify_result
749     if (!reset_n)
750         ecc_verify_pass_reg <= 1'b0;
751     else if (zeroize_reg | error_flag | error_flag_reg |
ecc_cmd_start)
752         ecc_verify_pass_reg <= 1'b0;
753     else if (verifying_process & hw_verify_r_we)
754         ecc_verify_pass_reg <= ecc_verify_res_match;
755 end // ecc_verify_result
756
757 //-----
758 // ECDSA_FSM_flow
759 //
760 // This FSM starts with the keygen/signing/verifying com-

```

```

mand to
761 // perform different operations.
762 // Active low and async reset.
763 //-----
764
765 always_ff @(posedge clk or negedge reset_n)
766 begin : ECDSA_FSM
767     if(!reset_n) begin
768         prog_cntr <= ECC_RESET;
769         cycle_cnt <= '0;
770         pm_cmd_reg <= '0;
771         ecc_valid_reg <= '0;
772         scalar_G_sel <= '0;
773         hmac_mode <= '0;
774         hmac_init <= '0;
775         scalar_sca_en <= '0;
776         keygen_process <= '0;
777         signing_process <= '0;
778         verifying_process <= '0;
779         sharedkey_process <= '0;
780     end
781     else if(zeroize_reg) begin
782         prog_cntr <= ECC_RESET;
783         cycle_cnt <= '0;
784         pm_cmd_reg <= '0;
785         ecc_valid_reg <= '0;
786         scalar_G_sel <= '0;
787         hmac_mode <= '0;
788         hmac_init <= '0;
789         scalar_sca_en <= '0;
790         keygen_process <= '0;
791         signing_process <= '0;
792         verifying_process <= '0;
793         sharedkey_process <= '0;
794     end
795     else if (error_flag | error_flag_reg) begin
796         prog_cntr <= ECC_NOP;
797         cycle_cnt <= '0;
798         pm_cmd_reg <= '0;
799         ecc_valid_reg <= '0;
800         scalar_G_sel <= '0;
801         hmac_mode <= '0;
802         hmac_init <= '0;
803         scalar_sca_en <= '0;
804         keygen_process <= '0;
805         signing_process <= '0;
806         verifying_process <= '0;
807         sharedkey_process <= '0;
808     end
809     else begin
810         if (subcomponent_busy) begin //Stalled until sub-
component is done
811             prog_cntr <= prog_cntr;
812             cycle_cnt <= 2'd3;
813             pm_cmd_reg <= '0;
814             scalar_sca_en <= '0;
815             hmac_init <= '0;
816         end
817         else if (dsa_busy & (cycle_cnt != 2'd3)) begin
818             cycle_cnt <= cycle_cnt + 1;
819         end
820         else begin
821             cycle_cnt <= '0;
822             unique case (prog_cntr)
823                 ECC_NOP : begin
824                     keygen_process <= '0;
825                     signing_process <= '0;
826                     verifying_process <= '0;
827                     sharedkey_process <= '0;
828                     // Waiting for new valid command
829                     unique case (cmd_reg)
830                         KEYGEN : begin // keygen
831                             prog_cntr <= DSA_KG_S;
832                             ecc_valid_reg <= '0;
833                             scalar_G_sel <= '0;
834                             hmac_mode <= 2'b00;
835                             keygen_process <= 1;
836                         end
837                     SIGN : begin // signing
838                         prog_cntr <= DSA_SGN_S;
839                         ecc_valid_reg <= '0;
840                         scalar_G_sel <= '0;
841                         hmac_mode <= 2'b01;
842                         signing_process <= 1;
843                     end
844                 end
845             end
846         end
847     end
848
849     VERIFY : begin // verifying
850         prog_cntr <= DSA_VER_S;
851         ecc_valid_reg <= '0;
852         scalar_G_sel <= 1;
853         verifying_process <= 1;
854     end
855
856     SHARED_KEY : begin // DH
857         prog_cntr <= DH_SHARED_S;
858         ecc_valid_reg <= '0;
859         scalar_G_sel <= 1;
860         hmac_mode <= 2'b10;

```



```

857         sharedkey_process <= 1;
858     end
859
860     default : begin
861         prog_cntr <= ECC_NOP;
862         scalar_g_sel <= 0;
863     end
864 endcase
865 pm_cmd_reg <= '0';
866 hmac_init <= 0;
867 end
868
869 DSA_KG_E : begin // end of keygen
870     prog_cntr <= ECC_NOP;
871     ecc_valid_reg <= 1;
872 end
873
874 DSA_SGN_E : begin // end of signing
875     prog_cntr <= ECC_NOP;
876     ecc_valid_reg <= 1;
877 end
878
879 DSA_VER_E : begin // end of verifying
880     prog_cntr <= ECC_NOP;
881     ecc_valid_reg <= 1;
882 end
883
884 DH_SHARED_E : begin // end of DH shared key
885     prog_cntr <= ECC_NOP;
886     ecc_valid_reg <= 1;
887 end
888
889 ECC_RESET,
890 DSA_KG_S,
891 DSA_SGN_S,
892 DSA_VER_S,
893 DH_SHARED_S : begin
894     prog_cntr <= prog_cntr + 1;
895     pm_cmd_reg <= prog_in-
896
897     hmac_init <= prog_in-
898
899     scalar_sca_en <= prog_in-
900 end
901
902 default : begin
903     prog_cntr <= prog_cntr + 1;
904     pm_cmd_reg <= prog_in-
905
906     hmac_init <= prog_in-
907
908     scalar_sca_en <= prog_in-
909 end
910 endcase
911 end
912 end // ECDSA_FSM
913
914 // Generate a pulse to trig the interrupt after finishing
915 the operation
916 always_ff @(posedge clk or negedge reset_n)
917 if (!reset_n)
918     ecc_status_done_d <= 1'b0;
919 else if (zeroize_reg)
920     ecc_status_done_d <= 1'b0;
921 else
922     ecc_status_done_d <=
923 hwif_in.ECC_STATUS.VALID.next;
924 always_comb ecc_status_done_p =
925 hwif_in.ECC_STATUS.VALID.next && !ecc_status_done_d;
926
927 // Set the ready/busy flag of ECC
928 assign dsa_busy = (prog_cntr == ECC_NOP)? 1'b0 : 1'b1;
929 always_comb ecc_ready_reg = !(dsa_busy | pm_busy_o);
930
931 //Key Vault Control Modules
932 always_comb begin
933     kv_privkey_read_metrics.ocp_lock_in_progress =
934 ocp_lock_in_progress;
935     kv_privkey_read_metrics.kv_read_dest =
936 KV_NUM_READ'(1<<KV_DEST_IDX_ECC_PKEY);
937     kv_privkey_read_metrics.kv_key_entry =
938 kv_privkey_read_ctrl_reg.read_entry;
939 end
940
941 //Read PRIVKEY
942 kv_read_client #(
943     .DATA_WIDTH(REG_SIZE),
944     .PAD(0)
945 )
946 ecc_privkey_kv_read
947 (
948     .clk(clk),
949     .rst_b(reset_n),
950     .zeroize(zeroize_reg),
951
952     //client control register
953     .read_ctrl_reg(kv_privkey_read_ctrl_reg),
954     .read_metrics(kv_privkey_read_metrics),
955
956     .error_code(kv_privkey_error),
957     .kv_ready(kv_privkey_ready),
958     .read_done(kv_privkey_done),
959     .check_key_size(1'b0),
960     .expected_key_size(KV_ENTRY_SIZE_W'((REG_SIZE/32)-1))
961 );

```

```

946
947 //interface with kv
948 .kv_read(kv_read[0]),
949 .kv_resp(kv_rd_resp[0]),
950
951 //interface with client
952 .write_en(kv_privkey_write_en),
953 .write_offset(kv_privkey_write_offset),
954 .write_data(kv_privkey_write_data),
955
956 .error_code(kv_privkey_error),
957 .kv_ready(kv_privkey_ready),
958 .read_done(kv_privkey_done),
959 .check_key_size(1'b0),
960 .expected_key_size(KV_ENTRY_SIZE_W'((REG_SIZE/32)-1))
961 );
962
963 //Read SEED
964 always_comb begin
965     kv_seed_read_metrics.ocp_lock_in_progress =
966 ocp_lock_in_progress;
967     kv_seed_read_metrics.kv_read_dest =
968 KV_NUM_READ'(1<<KV_DEST_IDX_ECC_SEED);
969     kv_seed_read_metrics.kv_key_entry =
970 kv_seed_read_ctrl_reg.read_entry;
971 end
972
973 kv_read_client #(
974     .DATA_WIDTH(REG_SIZE),
975     .PAD(0)
976 )
977 ecc_seed_kv_read
978 (
979     .clk(clk),
980     .rst_b(reset_n),
981     .zeroize(zeroize_reg),
982
983     //client control register
984     .read_ctrl_reg(kv_seed_read_ctrl_reg),
985     .read_metrics(kv_seed_read_metrics),
986
987     //interface with kv
988     .kv_read(kv_read[1]),
989     .kv_resp(kv_rd_resp[1]),
990
991     //interface with client
992     .write_en(kv_seed_write_en),
993     .write_offset(kv_seed_write_offset),
994     .write_data(kv_seed_write_data),
995
996     .error_code(kv_seed_error),
997     .kv_ready(kv_seed_ready),
998     .read_done(kv_seed_done),
999     .check_key_size(1'b0),
1000     .expected_key_size(KV_ENTRY_SIZE_W'((REG_SIZE/32)-1))
1001 );
1002
1003 //Write to keyvault
1004 always_comb begin
1005     kv_write_metrics.ocp_lock_in_progress =
1006 ocp_lock_in_progress;
1007     kv_write_metrics.kv_data0_present =
1008 kv_seed_data_present;
1009     kv_write_metrics.kv_data0_entry =
1010 kv_seed_read_ctrl_reg.read_entry; // FIXME latch this at start-time
1011     kv_write_metrics.kv_data1_present =
1012 kv_key_data_present;
1013     kv_write_metrics.kv_data1_entry =
1014 kv_privkey_read_ctrl_reg.read_entry; // FIXME latch this at start-
1015 time
1016     kv_write_metrics.kv_write_src =
1017 KV_NUM_WRITE'(1 << KV_WRITE_IDX_ECC);
1018     kv_write_metrics.kv_write_entry =
1019 kv_write_ctrl_reg.write_entry;
1020     kv_write_metrics.aes_decrypt_ecb_op = 1'b0;
1021 end
1022
1023 kv_write_client #(
1024     .DATA_WIDTH(REG_SIZE)
1025 )
1026 ecc_privkey_kv_write
1027 (
1028     .clk(clk),
1029     .rst_b(reset_n),
1030     .zeroize(zeroize_reg),
1031
1032     //client control register
1033     .write_ctrl_reg(kv_write_ctrl_reg),
1034     .num_dwords(REG_NUM_DWORDS[4:0]),
1035     .write_metrics(kv_write_metrics),
1036
1037     //interface with kv
1038     .kv_write(kv_write),
1039     .kv_resp(kv_wr_resp),
1040
1041     //interface with client
1042     .dest_keyvault(dest_keyvault),
1043     .dest_data_avail(secretkey_we),
1044     .dest_data(kv_reg),
1045
1046     .error_code(kv_write_error),

```

```

1036         .kv_ready(kv_write_ready),
1037         .dest_done(kv_write_done)
1038     );
1039
1040     always_comb busy_o = ~ecc_ready_reg | ~kv_write_ready |
~kv_seed_ready | ~kv_privkey_ready;
1041     `CALIPTRA_ASSERT_MUTEX(ERR_ECC_PRIVKEY_WE_MUTEX,
{hw_privkey_we, privkey_we_reg}, clk, !reset_n)
1042     `CALIPTRA_ASSERT_MUTEX(ERR_ECC_SHAREDKEY_WE_MUTEX,
{hw_sharedkey_we, sharedkey_we_reg}, clk, !reset_n)
1043
1044 endmodule

```

6. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_sequencer.sv (167 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 //
17 // ecc_dsa_sequencer.sv
18 // -----
19 // This module contains the sequence of ECC operation. it in-
cludes 4 sequences
20 // of operation:
21 // 1) reset sequence: After reset, the core will initilze the
data memory based
22 // on the first lines of sequencer. Then,
it will stay on NOP
23 // line until getting the command.
24 // 2) keygen sequence: After getting the keygen command, it
will generate and
25 // randomize privkey and feed it into the
pm core and return
26 // key pairs of (privkey, pubkey)
27 // 3) sign sequence: After getting the sign command, it will
generate and
28 // randomize nonce and feed it into the pm
core and return
29 // the signature (R,
S).
30 // 4) verify sequence: After getting the verify command, it
will feed the signature
31 // into the pm core and return the com-
puted verify_R.
32 //
33 //
34 //
=====
35 module ecc_dsa_sequencer
36 import ecc_pm_uop_pkg::*;
37 import ecc_dsa_uop_pkg::*;
38 #(
39     parameter ADDR_WIDTH = 8,
40     parameter DATA_WIDTH = 32
41 )
42 (
43     input wire clk,
44     input wire reset_n,
45     input wire zeroize,
46     input wire ena,
47     input wire [ADDR_WIDTH-1 : 0] addra,
48     output logic [DATA_WIDTH-1 : 0] douta
49 );
50
51
52 //-----
53
54 // ROM content
55 //-----
56
57 always_ff @(posedge clk or negedge reset_n)
58 begin
59     if (!reset_n) begin

```

```

60     douta <= '0;
61 end
62 else if (zeroize) begin
63     douta <= '0;
64 end
65 else begin
66     if (ena) begin
67         unique case(addra)
68             //PM CORE INIT
69             0 : douta <= {DSA_UOP_NOP,
UOP_OPR_DONTCARE};
70             1 : douta <= {DSA_UOP_WR_CORE,
UOP_OPR_CONST_ZERO};
71             2 : douta <= {DSA_UOP_WR_CORE,
UOP_OPR_CONST_ONE};
72             3 : douta <= {DSA_UOP_WR_CORE,
UOP_OPR_CONST_E_a};
73             4 : douta <= {DSA_UOP_WR_CORE,
UOP_OPR_CONST_E_3b};
74             5 : douta <= {DSA_UOP_WR_CORE,
UOP_OPR_CONST_ONE_MONT};
75             6 : douta <= {DSA_UOP_WR_CORE,
UOP_OPR_CONST_R2_p};
76             7 : douta <= {DSA_UOP_WR_CORE,
UOP_OPR_CONST_GX_MONT};
77             8 : douta <= {DSA_UOP_WR_CORE,
UOP_OPR_CONST_GY_MONT};
78             9 : douta <= {DSA_UOP_WR_CORE,
UOP_OPR_CONST_R2_q};
79             10 : douta <= {DSA_UOP_WR_CORE,
UOP_OPR_CONST_ONE_q_MONT};
80             11 : douta <= {DSA_UOP_NOP,
UOP_OPR_DONTCARE};
81
82             //KEYGEN
83             DSA_KG_S : douta <= {DSA_UOP_H-
MAC_DRBG, NOP_ID,
UOP_OPR_DONTCARE};
84             DSA_KG_S+ 1 : douta <=
{DSA_UOP_NOP, NOP_ID,
UOP_OPR_DONTCARE};
85             DSA_KG_S+ 2 : douta <=
{DSA_UOP_WR_CORE, SCALAR_G_ID,
UOP_OPR_SCALAR_G};
86             DSA_KG_S+ 3 : douta <=
{DSA_UOP_RD_CORE, PRIVKEY_ID,
UOP_OPR_SCALAR_G};
87             DSA_KG_S+ 4 : douta <= {DSA_UOP_SCAL-
AR_SCA, SCALAR_G_ID,
UOP_OPR_DONTCARE};
88             DSA_KG_S+ 5 : douta <=
{DSA_UOP_NOP, NOP_ID,
UOP_OPR_DONTCARE};
89             DSA_KG_S+ 6 : douta <= {DSA_UOP_WR_S-
CALAR, SCALAR_ID,
UOP_OPR_DONTCARE};
90             DSA_KG_S+ 7 : douta <=
{DSA_UOP_WR_CORE, LAMBDA_ID,
UOP_OPR_LAMBDA};
91             DSA_KG_S+ 8 : douta <= {DSA_UOP_KEY-
GEN, NOP_ID,
UOP_OPR_DONTCARE};
92             DSA_KG_S+ 9 : douta <=
{DSA_UOP_NOP, NOP_ID,
UOP_OPR_DONTCARE};
93             DSA_KG_S+ 10 : douta <=
{DSA_UOP_RD_CORE, PUBKEYX_ID,
UOP_OPR_Qx_AFFN};
94             DSA_KG_S+ 11 : douta <=
{DSA_UOP_RD_CORE, PUBKEYY_ID,
UOP_OPR_Qy_AFFN};
95             DSA_KG_S+ 12 : douta <=
{DSA_UOP_NOP, NOP_ID,
UOP_OPR_DONTCARE};
96
97             //SIGN
98             DSA_SGN_S : douta <=
{DSA_UOP_WR_CORE, MSG_ID,
UOP_OPR_HASH_MSG};
99             DSA_SGN_S+ 1 : douta <=
{DSA_UOP_WR_CORE, PRIVKEY_ID,
UOP_OPR_PRIVKEY};
100             DSA_SGN_S+ 2 : douta <= {DSA_UOP_H-
MAC_DRBG, NOP_ID,
UOP_OPR_DONTCARE};
101             DSA_SGN_S+ 3 : douta <=
{DSA_UOP_NOP, NOP_ID,
UOP_OPR_DONTCARE};
102             DSA_SGN_S+ 4 : douta <=
{DSA_UOP_WR_CORE, SCALAR_G_ID,
UOP_OPR_SCALAR_G};
103             DSA_SGN_S+ 5 : douta <= {DSA_UOP_SCAL-
AR_SCA, SCALAR_G_ID,
UOP_OPR_DONTCARE};
104             DSA_SGN_S+ 6 : douta <=
{DSA_UOP_NOP, NOP_ID,
UOP_OPR_DONTCARE};
105             DSA_SGN_S+ 7 : douta <= {DSA_UOP_WR_S-
CALAR, SCALAR_ID,
UOP_OPR_DONTCARE};
106             DSA_SGN_S+ 8 : douta <=
{DSA_UOP_WR_CORE, LAMBDA_ID,
UOP_OPR_LAMBDA};
107             DSA_SGN_S+ 9 : douta <=
{DSA_UOP_WR_CORE, MASKING_ID,
UOP_OPR_MASKING};
108             DSA_SGN_S+ 10 : douta <=
{DSA_UOP_SIGN, NOP_ID,
UOP_OPR_DONTCARE};
109             DSA_SGN_S+ 11 : douta <=
{DSA_UOP_NOP, NOP_ID,
UOP_OPR_DONTCARE};
110             DSA_SGN_S+ 12 : douta <=
{DSA_UOP_RD_CORE, R_ID,
UOP_OPR_SIGN_R};
111             DSA_SGN_S+ 13 : douta <=
{DSA_UOP_RD_CORE, S_ID,
UOP_OPR_SIGN_S};
112             DSA_SGN_S+ 14 : douta <=
{DSA_UOP_NOP, NOP_ID,
UOP_OPR_DONTCARE};
113
114             //VERIFY
115             DSA_VER_S : douta <=
{DSA_UOP_WR_CORE, CONST_E_b_MONT_ID,
UOP_OPR_CONST_E_b};
116             DSA_VER_S+ 1 : douta <=
{DSA_UOP_WR_CORE, PUBKEYX_ID,
UOP_OPR_Qx_AFFN};
117             DSA_VER_S+ 2 : douta <=
{DSA_UOP_WR_CORE, PUBKEYY_ID,
UOP_OPR_Qy_AFFN};

```

```

118     DSA_VER_S+ 3 : douta <=
{DSA_UOP_PK_CHK,    NOP_ID,    UOP_OPR_DONTCARE};
119     DSA_VER_S+ 4 : douta <=
{DSA_UOP_NOP,       NOP_ID,    UOP_OPR_DONTCARE};
120     DSA_UOP_RD_CORE,
121     PK_VALID_ID,    UOP_OPR_PK_VALID};
122     DSA_VER_S+ 6 : douta <=
{DSA_UOP_WR_CORE,   MSG_ID,    UOP_OPR_HASH_MSG};
123     DSA_VER_S+ 7 : douta <=
{DSA_UOP_WR_CORE,   R_ID,      UOP_OPR_SIGN_R};
124     DSA_VER_S+ 8 : douta <=
{DSA_UOP_WR_CORE,   S_ID,      UOP_OPR_SIGN_S};
125     DSA_UOP_WR_CORE,
CONST_ONE_ID,    UOP_OPR_LAMBDA};
126     DSA_VER_S+ 10 : douta <= {DSA_UOP_VERIFY-
FY0,             NOP_ID,    UOP_OPR_DONTCARE};
127     DSA_UOP_NOP,
NOP_ID,    UOP_OPR_DONTCARE};
128     DSA_VER_S+ 11 : douta <=
{DSA_UOP_NOP,    NOP_ID,    UOP_OPR_DONTCARE};
129     DSA_UOP_RD_CORE,
SCALAR_G_ID,    UOP_OPR_SCALAR_G};
130     DSA_VER_S+ 13 : douta <=
{DSA_UOP_RD_CORE, SCALAR_PK_ID, UOP_OPR_SCALAR_PK};
131     DSA_UOP_WR_SCAL-
AR,             SCALAR_G_ID,    UOP_OPR_DONTCARE};
132     DSA_VER_S+ 15 : douta <= {DSA_UOP_VERIFY-
FY1,             NOP_ID,    UOP_OPR_DONTCARE};
133     DSA_UOP_NOP,
NOP_ID,    UOP_OPR_DONTCARE};
134     DSA_VER_S+ 17 : douta <=
{DSA_UOP_WR_CORE, PUBKEYX_ID, UOP_OPR_Qx_AFFN};
135     DSA_UOP_WR_CORE,
PUBKEYY_ID,    UOP_OPR_Qy_AFFN};
136     DSA_VER_S+ 19 : douta <= {DSA_UOP_WR_SCAL-
AR,             SCALAR_PK_ID,    UOP_OPR_DONTCARE};
137     DSA_UOP_VERIFY-
FY2,             NOP_ID,    UOP_OPR_DONTCARE};
138     DSA_UOP_NOP,
NOP_ID,    UOP_OPR_DONTCARE};
139     DSA_UOP_RD_CORE,
VERIFY_R_ID,    UOP_OPR_Qx_AFFN};
140     DSA_UOP_RD_CORE,
DSA_VER_S+ 23 : douta <=
NOP_ID,    UOP_OPR_DONTCARE};
141     //DH_SHARED_KEY
142     DH_SHARED_S : douta <=
CONST_E_b_MONT_ID, UOP_OPR_CONST_E_b};
143     DH_SHARED_S+ 1 : douta <=
PUBKEYX_ID,    UOP_OPR_Qx_AFFN};
144     DH_SHARED_S+ 2 : douta <=
PUBKEYY_ID,    UOP_OPR_Qy_AFFN};
145     DH_SHARED_S+ 3 : douta <=
NOP_ID,    UOP_OPR_DONTCARE};
146     DH_SHARED_S+ 4 : douta <=
NOP_ID,    UOP_OPR_DONTCARE};
147     DH_SHARED_S+ 5 : douta <=
PK_VALID_ID,    UOP_OPR_PK_VALID};
148     DH_SHARED_S+ 6 : douta <= {DSA_UOP_H-
MAC_DRBG,        NOP_ID,    UOP_OPR_DONTCARE};
149     DH_SHARED_S+ 7 : douta <=
NOP_ID,    UOP_OPR_DONTCARE};
150     DH_SHARED_S+ 8 : douta <=
PRIVKEY_ID,    UOP_OPR_SCALAR_PK};
151     DH_SHARED_S+ 9 : douta <=
SCALAR_G_ID,    UOP_OPR_SCALAR_PK};
152     DH_SHARED_S+ 10 : douta <= {DSA_UOP_SCAL-
AR_SCA,          SCALAR_G_ID,    UOP_OPR_DONTCARE};
153     DH_SHARED_S+ 11 : douta <=
NOP_ID,    UOP_OPR_DONTCARE};
154     DH_SHARED_S+ 12 : douta <= {DSA_UOP_WR_S-
CALAR,          SCALAR_ID,    UOP_OPR_DONTCARE};
155     DH_SHARED_S+ 13 : douta <=
LAMBDA_ID,    UOP_OPR_LAMBDA};
156     DH_SHARED_S+ 14 : douta <=
NOP_ID,    UOP_OPR_DONTCARE};
157     DH_SHARED_S+ 15 : douta <=
NOP_ID,    UOP_OPR_DONTCARE};
158     DH_SHARED_S+ 16 : douta <=
DH_SHAREDKEY_ID, UOP_OPR_Qx_AFFN};
159     DH_SHARED_S+ 17 : douta <=
NOP_ID,    UOP_OPR_DONTCARE};
160     default :
161     NOP_ID,    UOP_OPR_DONTCARE};
162     endcase
163     end
164     end
165     end
166     end
167 endmodule

```

7. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_uop_pkg.sv (126 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.

```

```

5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 //
17 // ecc_dsa_uop_pkg.sv
18 // -----
19 // ECC instructin for the digital signature algorithm (DSA).
20 //
21 //
=====
22 //
23
24 `ifndef ECC_DSA_UOP_PKG
25 `define ECC_DSA_UOP_PKG
26
27 package ecc_dsa_uop_pkg;
28
29
30 localparam integer DSA_UOP_ADDR_WIDTH = 9;
31 localparam integer DSA_OPR_ADDR_WIDTH = 6;
32
33 localparam DSA_PROG_ADDR_W = 7; // $clo-
g2(DSA_VER_E+2);
34 localparam DSA_INSTRUCTION_LENGTH = DSA_UOP_AD-
DR_WIDTH + (2*DSA_OPR_ADDR_WIDTH); // opcode + 2 * operand
35
36 typedef enum logic[3 : 0]
37 {
38     no_cmd = 4'b0000,
39     keygen_cmd = 4'b0001,
40     sign_cmd = 4'b0010,
41     verify0_cmd = 4'b0100,
42     verify1_cmd = 4'b0101,
43     verify2_cmd = 4'b0110,
44     pk_chk_cmd = 4'b0111,
45     shared_key_cmd = 4'b1000
46 } cmd_t;
47
48 typedef struct packed
49 {
50     logic op_sel;
51     logic wr_en;
52     logic rd_en;
53     cmd_t pm_cmd;
54     logic hmac_drbg_en;
55     logic sca_en;
56 } opcode_t;
57
58 typedef struct packed
59 {
60     opcode_t opcode;
61     logic [DSA_OPR_ADDR_WIDTH-1 : 0] reg_id;
62     logic [DSA_OPR_ADDR_WIDTH-1 : 0] mem_addr;
63 } instr_struct_t;
64
65 // DSA INSTRUCTIONS LIST
66 localparam opcode_t DSA_UOP_NOP = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:no_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0000_0000;
67 localparam opcode_t DSA_UOP_WR_CORE = '{op_sel:1'b0,
wr_en:1'b1, rd_en:1'b0, pm_cmd:no_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0000_0010;
68 localparam opcode_t DSA_UOP_WR_SCALAR = '{op_sel:1'b1,
wr_en:1'b1, rd_en:1'b0, pm_cmd:no_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0000_0011;
69 localparam opcode_t DSA_UOP_RD_CORE = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b1, pm_cmd:no_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0000_0100;
70 localparam opcode_t DSA_UOP_KEYGEN = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:keygen_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0000_1000;
71 localparam opcode_t DSA_UOP_SIGN = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:sign_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0001_0000;
72 localparam opcode_t DSA_UOP_VERIFY0 = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:verify0_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0010_0000;
73 localparam opcode_t DSA_UOP_VERIFY1 = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:verify1_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0010_1000;
74 localparam opcode_t DSA_UOP_VERIFY2 = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:verify2_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0011_0000;
75 localparam opcode_t DSA_UOP_PK_CHK = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:pk_chk_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0011_1000;

```

```

76 localparam opcode_t DSA_UOP_HMAC_DRBG      = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:no_cmd,      hmac_drbg_en:1'b1,
sca_en:1'b0}; // = 8'b0100_0000;
77 localparam opcode_t DSA_UOP_SCALAR_SCA     = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:no_cmd,      hmac_drbg_en:1'b0,
sca_en:1'b1}; // = 8'b1000_0000;
78 localparam opcode_t DH_UOP_SHAREDKEY       = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:shared_key_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0000_1000;
79
80 // DSA REGISTERS ID listing

81 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] NOP_ID      =
6'd0;

82 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_ZERO_ID =
6'd1;

83 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_ONE_ID  =
6'd2;

84 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_E_a_MONT_ID =
6'd3;

85 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_E_3b_MONT_ID =
6'd4;

86 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_ONE_p_MONT_ID =
6'd5;

87 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_R2_p_MONT_ID =
6'd6;

88 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_G_X_MONT_ID =
6'd7;

89 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_G_Y_MONT_ID =
6'd8;

90 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_R2_q_MONT_ID =
6'd9;

91 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_ONE_q_MONT_ID =
6'd10;

92 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_E_b_MONT_ID =
6'd11;
93

94 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] SEED_ID      =
6'd16;

95 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] MSG_ID       =
6'd17;

96 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] PRIVKEY_ID   =
6'd18;

97 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] PUBKEYX_ID   =
6'd19;

98 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] PUBKEYY_ID   =
6'd20;

99 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] R_ID         =
6'd21;

100 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] S_ID        =
6'd22;

101 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] SCALAR_ID    =
6'd23;

102 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] SCALAR_G_ID  =
6'd24;

103 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] SCALAR_PK_ID =
6'd25;

104 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] VERIFY_R_ID =
6'd26;

105 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] LAMBDA_ID    =
6'd27;

106 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] MASKING_ID   =
6'd28;

107 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] PK_VALID_ID  =
6'd29;

108 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] DH_SHAREDKEY_ID =
6'd30;
109
110
111 // DSA Subroutine listing
112 localparam [DSA_PROG_ADDR_W-1 : 0] ECC_RE-
SET      = 7'd0;
113 localparam [DSA_PROG_ADDR_W-1 : 0]
ECC_NOP  = 7'd12;
114 localparam [DSA_PROG_ADDR_W-1 : 0]

```

```

DSA_KG_S      = ECC_NOP + 2;
115 localparam [DSA_PROG_ADDR_W-1 : 0]
DSA_KG_E      = DSA_KG_S + 12;
116 localparam [DSA_PROG_ADDR_W-1 : 0]
DSA_SGN_S     = DSA_KG_E + 2;
117 localparam [DSA_PROG_ADDR_W-1 : 0]
DSA_SGN_E     = DSA_SGN_S + 14;
118 localparam [DSA_PROG_ADDR_W-1 : 0]
DSA_VER_S     = DSA_SGN_E + 2;
119 localparam [DSA_PROG_ADDR_W-1 : 0]
DSA_VER_E     = DSA_VER_S + 23;
120 localparam [DSA_PROG_ADDR_W-1 : 0]
DH_SHARED_S   = DSA_VER_E + 2;
121 localparam [DSA_PROG_ADDR_W-1 : 0]
DH_SHARED_E   = DH_SHARED_S + 17;
122
123
124 endpackage
125
126 `endif

```

8. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_fau.sv (161 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 //
17 // ecc_fau.sv
18 // -----
19 // ECC Finite field Arithmetic Unit including Montgomery Mul-
tiplier and Adder
20 //
21 //
22 //
=====
23
24 module ecc_fau #(
25     parameter REG_SIZE      = 384,
26     parameter RADIX         = 32
27 )
28 (
29     // Clock and reset.
30     input wire               clk,
31     input wire               reset_n,
32     input wire               zeroize,
33
34     // DATA PORT
35     input wire               add_en_i,
36     input wire               sub_i,
37     input wire               mult_en_i,
38     input wire [REG_SIZE-1:0] prime_i,
39     input wire [RADIX-1 : 0] mult_mu_i,
40     input wire [REG_SIZE-1:0] opa_i,
41     input wire [REG_SIZE-1:0] opb_i,
42     output wire [REG_SIZE-1:0] add_res_o,
43     output wire [REG_SIZE-1:0] mult_res_o
44 );
45
46     logic [REG_SIZE-1 : 0] mult_res_s;
47     logic [REG_SIZE-1 : 0] mult_opa;
48     logic [REG_SIZE-1 : 0] mult_opb;
49     logic [REG_SIZE-1 : 0] add_res_s;
50
51     reg                mult_start;
52     reg                mult_start_dly;
53     wire               mult_start_edge;
54     logic               add_start;
55     logic               add_start_dly;
56     logic               add_start_edge;
57     reg                sub;
58
59     logic               add_ready_o;
60     logic               mult_ready_o;
61
62     logic               ready_garbage_bit;
63
64     assign mult_opa = opa_i;
65     assign mult_opb = opb_i;
66

```

```

67 //-----
68 // MULTIPLIER
69 //-----
70 ecc_montgomerymultiplier #(
71     .REG_SIZE(REG_SIZE),
72     .RADIX(RADIX)
73 )
74 i_MULTIPLIER (
75     // Clock and reset.
76     .clk(clk),
77     .reset_n(reset_n),
78     .zeroize(zeroize),
79
80     // DATA PORT
81     .start_i(mult_start_edge),
82     .opa_i(mult_opa),
83     .opb_i(mult_opb),
84     .n_i(prime_i),
85     .n_prime_i(mult_mu_i), // only need the last few bits
86     .p_o(mult_res_s),
87     .ready_o(mult_ready_o)
88 );
89
90 //-----
91
92 // ADDER/SUBTRACTOR
93 //-----
94 ecc_add_sub_mod_alter #(
95     .REG_SIZE(REG_SIZE)
96 )
97 i_ADDER_SUBTRACTOR (
98     .clk(clk),
99     .reset_n(reset_n),
100     .zeroize(zeroize),
101
102     .add_en_i(add_start_edge),
103     .sub_i(sub),
104     .opa_i(opa_i),
105     .opb_i(opb_i),
106     .prime_i(prime_i),
107     .res_o(add_res_s),
108     .ready_o(add_ready_o)
109 );
110
111 //-----
112
113 // Registers including update enables.
114 //-----
115 always_ff @(posedge clk or negedge reset_n)
116 begin
117     if (!reset_n) begin
118         mult_start <= '0;
119         add_start <= '0;
120         sub <= '0;
121     end
122     else if (zeroize) begin
123         mult_start <= '0;
124         add_start <= '0;
125         sub <= '0;
126     end
127     else begin
128         mult_start <= mult_en_i;
129         add_start <= add_en_i;
130         sub <= sub_i;
131     end
132 end
133
134 always_ff @(posedge clk or negedge reset_n) begin
135     if (!reset_n) begin
136         mult_start_dly <= '0;
137         add_start_dly <= '0;
138     end
139     else if (zeroize) begin
140         mult_start_dly <= '0;
141         add_start_dly <= '0;
142     end
143     else begin
144         mult_start_dly <= mult_start;
145         add_start_dly <= add_start;
146     end
147 end
148
149 assign ready_garbage_bit = add_ready_o & mult_ready_o;
150
151 // generate a pulse enable for mult
152 assign mult_start_edge = mult_start & ~mult_start_dly;
153 assign add_start_edge = add_start & ~add_start_dly;
154
155 //-----
156
157 // Concurrent connectivity for ports etc.
158 //-----
159
160 assign mult_res_o = mult_res_s;
161 assign add_res_o = add_res_s;

```

```

160
161 endmodule

```

9. chipsalliance_caliptra-rtl/src/ecc/rtl/ ecc_hmac_drbg_interface.sv (344 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
4 // you may not use this file except in compliance with the Li-
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
10 // distributed under the License is distributed on an "AS IS"
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
12 // press or implied.
13 // See the License for the specific language governing permis-
14 // sions and
15 // limitations under the License.
16 //
17 // ecc_hmac_drbg_interface.sv
18 // -----
19 // interface with hmac_drbg component to generate the required
20 // values:
21 // 1) in keygen mode:
22 // 1.1. generate lambda from IV for point randomization
23 // SCA countermeasure
24 // 1.2. generate scalar_rnd from IV for scalar blinding
25 // SCA countermeasure
26 // 1.3. generate privkey from seed and nonce for key gen-
27 // eration
28 // 2) in sign mode:
29 // 2.1. generate lambda from IV for point randomization
30 // SCA countermeasure
31 // 2.2. generate scalar_rnd from IV for scalar blinding
32 // SCA countermeasure
33 // 2.3. generate masking_rnd from IV for masking signa-
34 // ture SCA countermeasure
35 // 2.4. generate k from privkey and hashed_msg for sign-
36 // ing
37 //
38 // To generate random values using IV, the hmac_drbg is con-
39 // tinued by triggering
40 // next command (instead of init).
41 //
42 // - INIT: Instantiates HMAC-DRBG with entropy,
43 // nonce, (personalization string is empty).
44 // Then, it calls
45 // HMAC_DRBG_Generate_algorithm (additional input string is empty)
46 // to generate a 384-bit random value while
47 // it's in (0, prime) range.
48 // if the random value is zero or >= prime,
49 // it is rejected and a new random value
50 // is generated by calling
51 // HMAC_DRBG_Generate_algorithm.
52 // There is NO RESEED process.
53 // - NEXT: it calls HMAC_DRBG_Generate_algorithm (ad-
54 // ditional input string is empty)
55 // to generate a 384-bit random value while
56 // it's in (0, prime) range.
57 // if the random value is zero or >= prime,
58 // it is rejected and a new random value
59 // is generated by calling
60 // HMAC_DRBG_Generate_algorithm.
61 // There is NO RESEED process.
62 //
63 // -----
64
65 import hmac_param_pkg::*;
66
67 module ecc_hmac_drbg_interface#(
68     parameter REG_SIZE = 384,
69     parameter [REG_SIZE-1 : 0] GROUP_ORDER =
70 384'hfffffffffffffffffffffffffffffffffffffffffffffffffffffc7634d81f4372d
71 df581a0db248b0a77aecec196accc52973
72 )
73 (
74     // Clock and reset.
75     input wire clk,
76     input wire reset_n,
77     input wire zeroize,
78     input wire [1 : 0] hmac_mode,
79     input wire en,
80     output wire ready,
81
82     //Data
83     input wire [REG_SIZE-1 : 0] keygen_seed,
84     input wire [REG_SIZE-1 : 0] keygen_nonce,

```



```

64   input wire [REG_SIZE-1 : 0] privKey,
65   input wire [REG_SIZE-1 : 0] hashed_msg,
66   input wire [REG_SIZE-1 : 0] IV,
67
68   output wire [REG_SIZE-1 : 0] lambda,
69   output wire [REG_SIZE-1 : 0] scalar_rnd,
70   output wire [REG_SIZE-1 : 0] masking_rnd,
71   output wire [REG_SIZE-1 : 0] drbg
72 );
73
74 //-----
75 // Registers including update variables and write enable.
76 //-----
77 logic [REG_SIZE-1 : 0] lfsr_seed_reg;
78 logic [LFSR_SEED_SIZE-1 : 0] hmac_lfsr_seed;
79
80 logic hmac_drbg_init;
81 logic hmac_drbg_next;
82 logic hmac_drbg_ready;
83 logic hmac_drbg_valid;
84 logic [REG_SIZE-1 : 0] hmac_drbg_entropy;
85 logic [REG_SIZE-1 : 0] hmac_drbg_nonce;
86 logic [REG_SIZE-1 : 0] hmac_drbg_result;
87
88 logic first_round;
89 logic [REG_SIZE-1 : 0] lambda_reg;
90 logic [REG_SIZE-1 : 0] scalar_rnd_reg;
91 logic [REG_SIZE-1 : 0] masking_rnd_reg;
92 logic [REG_SIZE-1 : 0] drbg_reg;
93 logic hmac_drbg_valid_last;
94 logic hmac_done_edge;
95
96 logic [63 : 0] counter_reg;
97 logic [REG_SIZE-1 : 0] counter_nonce;
98 logic [REG_SIZE-1 : 0] counter_nonce_reg;
99 logic [REG_SIZE-1 : 0] sca_entropy, sca_entropy_reg;
100
101 localparam [1 : 0] KEYGEN_CMD = 2'b00;
102 localparam [1 : 0] SIGN_CMD = 2'b01;
103 localparam [1 : 0] DH_SHARED_CMD = 2'b10;
104
105 /*State register*/
106 reg [3 : 0] state_reg;
107 reg [3 : 0] state_next;
108 reg [3 : 0] state_reg_last;
109
110 /*STATES*/
111 localparam [3 : 0] IDLE_ST = 4'd0;
112 localparam [3 : 0] LFSR_ST = 4'd1;
113 localparam [3 : 0] LAMBDA_ST = 4'd2;
114 localparam [3 : 0] SCALAR_RND_ST = 4'd3;
115 localparam [3 : 0] RND_DONE_ST = 4'd4;
116 localparam [3 : 0] MASKING_RND_ST = 4'd5;
117 localparam [3 : 0] KEYGEN_ST = 4'd6;
118 localparam [3 : 0] SIGN_ST = 4'd7;
119 localparam [3 : 0] DONE_ST = 4'd8;
120
121 //-----
122 // Module instantiations.
123 //-----
124
125 hmac_drbg #(
126     .REG_SIZE(REG_SIZE),
127     .HMAC_DRBG_PRIME(GROUP_ORDER)
128 )
129 hmac_drbg_i (
130     .clk(clk),
131     .reset_n(reset_n),
132     .zeroize(zeroize),
133     .init_cmd(hmac_drbg_init),
134     .next_cmd(hmac_drbg_next),
135     .ready(hmac_drbg_ready),
136     .valid(hmac_drbg_valid),
137     .lfsr_seed(hmac_lfsr_seed),
138     .entropy(hmac_drbg_entropy),
139     .nonce(hmac_drbg_nonce),
140     .drbg(hmac_drbg_result)
141 );
142
143 genvar i;
144 generate
145     for (i=0; i < LFSR_SEED_SIZE/32; i++) begin : gen_lfsr
146         caliptra_prim_lfsr
147         #(
148             .LfsrType("FIB_XNOR"),
149             .LfsrDw(32),
150             .StateOutDw(32)
151         ) caliptra_prim_lfsr_inst_i
152         (
153             .clk_i(clk),
154             .rst_ni(reset_n),
155             .seed_en_i(hmac_drbg_init),
156             .seed_i(sca_entropy[i*32 +: 32]),
157             .lfsr_en_i(1'b1),
158             .entropy_i(1'b0),
159             .state_o(hmac_lfsr_seed[i*32 +: 32])
160         );
161     end
162 endgenerate
163
164 end
165 endgenerate
166
167 //-----
168 // hmac_drbg_interface_logic
169 //
170 // The logic needed to init as well as update the
171 hmac_drbg commands.
172 //-----
173
174 always_comb first_round = (state_reg == state_reg_last)?
175 1'b0 : 1'b1;
176 always_comb hmac_done_edge = hmac_drbg_valid & (!
177 hmac_drbg_valid_last);
178
179 always_comb
180 begin : hmac_drbg_entropy_input
181     unique case (state_reg)
182         LFSR_ST: hmac_drbg_entropy = sca_en-
183         trophy_reg;
184         LAMBDA_ST: hmac_drbg_entropy = sca_en-
185         trophy_reg;
186         SCALAR_RND_ST: hmac_drbg_entropy = sca_en-
187         trophy_reg;
188         MASKING_RND_ST: hmac_drbg_entropy = sca_en-
189         trophy_reg;
190         KEYGEN_ST: hmac_drbg_entropy = keygen_seed;
191         SIGN_ST: hmac_drbg_entropy = privKey;
192         default: hmac_drbg_entropy = sca_en-
193         trophy_reg;
194     endcase
195 end // hmac_drbg_entropy_input
196
197 always_comb
198 begin : hmac_drbg_nonce_input
199     unique case (state_reg)
200         LFSR_ST: hmac_drbg_nonce =
201         counter_nonce_reg;
202         LAMBDA_ST: hmac_drbg_nonce =
203         counter_nonce_reg;
204         SCALAR_RND_ST: hmac_drbg_nonce =
205         counter_nonce_reg;
206         MASKING_RND_ST: hmac_drbg_nonce =
207         counter_nonce_reg;
208         KEYGEN_ST: hmac_drbg_nonce = keygen_nonce;
209         SIGN_ST: hmac_drbg_nonce = hashed_msg;
210         default: hmac_drbg_nonce =
211         counter_nonce_reg;
212     endcase
213 end // hmac_drbg_nonce_input
214
215 always_comb
216 begin : hmac_trigger
217     hmac_drbg_init = 0;
218     hmac_drbg_next = 0;
219     if (first_round) begin
220         unique case (state_reg)
221             LFSR_ST: hmac_drbg_init = 1;
222             LAMBDA_ST: hmac_drbg_next = 1;
223             SCALAR_RND_ST: hmac_drbg_next = 1;
224             MASKING_RND_ST: hmac_drbg_next = 1;
225             KEYGEN_ST: hmac_drbg_init = 1;
226             SIGN_ST: hmac_drbg_init = 1;
227             default: begin
228                 hmac_drbg_init = 0;
229                 hmac_drbg_next = 0;
230             end
231         endcase
232     end
233 end //hmac_trigger
234
235 //-----
236 // register updates
237 //
238 // update the internal registers
239 //-----
240
241 always_ff @(posedge clk or negedge reset_n)
242 begin //reg_update
243     if (!reset_n) begin
244         lambda_reg <= '0;
245         scalar_rnd_reg <= '0;
246         masking_rnd_reg <= '0;
247         drbg_reg <= '0;
248         lfsr_seed_reg <= '0;
249     end
250     else if (zeroize) begin
251         lambda_reg <= '0;
252         scalar_rnd_reg <= '0;
253         masking_rnd_reg <= '0;
254         drbg_reg <= '0;
255         //lfsr_seed_reg <= '0; // without zeroize to make
256     end
257     else
258         if (hmac_done_edge) begin
259             unique case (state_reg) inside
260                 LFSR_ST: lfsr_seed_reg <=

```

```

hmac_drbg_result;
244         LAMBDA_ST:    lambda_reg    <=
hmac_drbg_result;
245         SCALAR_RND_ST: scalar_rnd_reg <=
hmac_drbg_result;
246         MASKING_RND_ST: masking_rnd_reg <=
hmac_drbg_result;
247         KEYGEN_ST:    drbg_reg      <=
hmac_drbg_result;
248         SIGN_ST:      drbg_reg      <=
hmac_drbg_result;
249         default: begin
250             lambda_reg <= '0;
251             scalar_rnd_reg <= '0;
252             masking_rnd_reg <= '0;
253             drbg_reg <= '0;
254         end
255     endcase
256 end
257 end // reg_update
258
259 always_ff @(posedge clk or negedge reset_n)
260 begin : state_reg_update
261     if (!reset_n)
262         state_reg <= IDLE_ST;
263     else if (zeroize)
264         state_reg <= IDLE_ST;
265     else
266         state_reg <= state_next;
267 end // state_reg_update
268
269 always_ff @(posedge clk or negedge reset_n)
270 begin : ff_state_reg
271     if (!reset_n)
272         state_reg_last <= IDLE_ST;
273     else if (zeroize)
274         state_reg_last <= IDLE_ST;
275     else
276         state_reg_last <= state_reg;
277 end // ff_state_reg
278
279 always_ff @(posedge clk or negedge reset_n)
280 begin : ff_hmac_valid
281     if (!reset_n)
282         hmac_drbg_valid_last <= '0;
283     else if (zeroize)
284         hmac_drbg_valid_last <= '0;
285     else
286         hmac_drbg_valid_last <= hmac_drbg_valid;
287 end // ff_hmac_valid
288
289 // without zeroize to make it more complex
290 always_ff @(posedge clk or negedge reset_n)
291 begin : counter_reg_update
292     if (!reset_n)
293         counter_reg <= '0;
294     else
295         counter_reg <= counter_reg + 1;
296 end // counter_reg_update
297
298 // without zeroize to make it more complex
299 always_ff @(posedge clk or negedge reset_n)
300 begin
301     if (!reset_n) begin
302         counter_nonce_reg <= '0;
303         sca_entropy_reg <= '0;
304     end else if (en && (state_reg == IDLE_ST)) begin
305         counter_nonce_reg <= counter_nonce;
306         sca_entropy_reg <= sca_entropy;
307     end
308 end
309
310 always_comb counter_nonce = {counter_reg, counter_reg,
counter_reg, counter_reg, counter_reg, counter_reg};
311 always_comb sca_entropy = IV ^ lfsr_seed_reg ^
counter_nonce;
312 //-----
313 // FSM_flow
314 //
315 // This FSM starts with the en command to perform HMAC-
DRBG.
316 // Active low and async reset.
317 //-----
318
319 always_comb
320 begin : interface_fsm
321     state_next = IDLE_ST;
322     unique case(state_reg)
323         IDLE_ST: state_next = (en &
hmac_drbg_ready)? LFSR_ST : IDLE_ST;
323         LFSR_ST: state_next = (hmac_done_edge)?
LAMBDA_ST : LFSR_ST;
324         LAMBDA_ST: state_next = (hmac_done_edge)?
SCALAR_RND_ST : LAMBDA_ST;
325         SCALAR_RND_ST: state_next = (hmac_done_edge)?
RND_DONE_ST : SCALAR_RND_ST;
326         RND_DONE_ST: state_next = (hmac_mode ==
SIGN_CMD)? MASKING_RND_ST : (hmac_mode == KEYGEN_CMD)? KEYGEN_ST :
DONE_ST;
327         MASKING_RND_ST: state_next = (hmac_done_edge)?

```

```

SIGN_ST : MASKING_RND_ST;
328         KEYGEN_ST: state_next = (hmac_done_edge)?
DONE_ST : KEYGEN_ST;
329         SIGN_ST: state_next = (hmac_done_edge)?
DONE_ST : SIGN_ST;
330         DONE_ST: state_next = IDLE_ST;
331         default: state_next = IDLE_ST;
332     endcase
333 end // interface_fsm
334
335 //-----
336 // Concurrent connectivity for ports etc.
337 //-----
338 assign lambda = lambda_reg;
339 assign scalar_rnd = scalar_rnd_reg;
340 assign masking_rnd = masking_rnd_reg;
341 assign drbg = drbg_reg;
342 assign ready = (state_reg == IDLE_ST);
343
344 endmodule

```

10. chipsalliance_caliptra-rtl/src/ecc/rtl/ ecc_montgomerymultiplier.sv (375 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 // Module:
17 // ecc_montgomerymultiplier.sv
18 //
19 // Montgomery Multiplier based on
20 // Optimized Algorithms and Architectures for Montgomery Mul-
tiplication
21 // for Post-quantum Cryptography by Rami Elkhatab et. al.
22 //
23 // Description:
24 // The module produces a Montgomery product of input op-
erands with the following constraints:
25 //
26 // Input operands must be less than n_i, i.e. opa_i <
n_i and opb_i < n_i.
27 //
28 // RADIX > 2
29 //
30 // R = 2^((ceil($bits(n_i) / RADIX) + 1) * RADIX)
31 //
32 // R > n_i.
33 //
34 // (1) T = opa_i * opb_i < n_i * n_i = n_i^2
35 //
36 // (2) p_internal = (T + (T*n_prime mod R) * n_i) / R
37 //
38 // (3) p_subtracted_internal = p_internal - n_i
39 //
40 // (4) p_o = (p_internal >= ni)? p_subtracted_internal :
p_internal
41 //
42 // From (1) and (2) given R > n_i, and opa_i, opb_i <
n_i:
43 //
44 // p_internal < (n_i^2 + R * n_i) / R = n_i + n_i^2 / R
45 //
46 // < n_i + n_i^2 / n_i = n_i + n_i = 2*n_i
47 //
48 // From (3) and (4):
49 //
50 // p_subtracted_internal < 2*n_i - n_i = n_i
51 //
52 // p_o < n_i
53 //
=====
54
55 module ecc_montgomerymultiplier #(
56     parameter REG_SIZE = 384,
57     parameter RADIX = 32
58 )
59 (
60     // Clock and reset.
61     input wire clk,
62     input wire reset_n,
63     input wire zeroize,
64
65     // DATA PORT
66     input wire start_i,
67     input wire [REG_SIZE-1:0] opa_i,
68     input wire [REG_SIZE-1:0] opb_i,
69     input wire [REG_SIZE-1:0] n_i,
70     input wire [RADIX-1:0] n_prime_i, // only need the
last few bits

```



```

56 output logic [REG_SIZE-1:0] p_o,
57 output logic ready_o
58 );
59 //-----
60 // Local Parameters
61 //-----
62
63 // Equivalent to $ceil(real'(REG_SIZE) / RADIX) + 1;
64 localparam int unsigned S_NUM = ((REG_SIZE + RADIX -
1) / RADIX) + 1;
65 localparam int unsigned FULL_REG_SIZE = S_NUM * RADIX;
66 // Each PE performs two iterations out of S_NUM - 1
67 // See section 4.2 of Elkhatab_2019 paper.
68 // PE_UNITS does not include the last PE.
69 // The last PE is instantiated separately.
70 localparam PE_UNITS = ((S_NUM - 1) / 2) - 1;
71 localparam [(FULL_REG_SIZE-REG_SIZE)-1 : 0]
zero_pad = '0;
72
73 //-----
74 // Registers
75 //-----
76 logic [RADIX-1 : 0] a_array[0:PE_UNITS+1];
77 logic [RADIX-1 : 0] b_array[0:PE_UNITS+1];
78 logic [RADIX-1 : 0] p_array[0:PE_UNITS+1];
79 logic [RADIX-1 : 0] m_array[0:PE_UNITS];
80 logic [RADIX : 0] c_array[0:PE_UNITS+1];
81 logic [RADIX-1 : 0] s_array[0:PE_UNITS+1];
82
83 logic [RADIX-1 : 0] p_neg_array[0:2*(PE_UNITS+1)];
84 logic [RADIX-1 : 0] t_reg[0:2*(PE_UNITS+1)];
85 logic [RADIX-1 : 0] t_subtracted_reg[0:2*(PE_UNITS+1)];
86 logic [RADIX : 0] sub_res[0:2*(PE_UNITS+1)];
87
88 logic sub_b_i[0:2*(PE_UNITS+1)];
89 logic sub_b_o[0:2*(PE_UNITS+1)];
90
91 logic [FULL_REG_SIZE-1 : 0] a_reg; //extended with
zero
92 logic [FULL_REG_SIZE-1 : 0] b_reg; //extended with
zero
93 logic [FULL_REG_SIZE-1 : 0] p_reg; //extended with
zero
94 logic [FULL_REG_SIZE-1 : 0] p_neg_reg; //extended
with one
95 logic [RADIX-1:0] n_prime_reg;
96 logic [(3*S_NUM)-1 : 0] push_reg;
97 logic push_reg_eq_zero;
98 logic push_reg_eq_zero_ff;
99 logic odd;
100 logic [RADIX-1 : 0] last_s_reg;
101 logic [FULL_REG_SIZE-1:0] p_internal;
102 logic [FULL_REG_SIZE-1:0] p_subtracted_internal;
103 logic last_reduction;
104
105 //-----
106 // Processing elements (PEs)
107 //-----
108 ecc_pe_first #(
109 .RADIX(RADIX)
110 )
111 first_box
112 (
113 .clk(clk),
114 .reset_n(reset_n),
115 .zeroize(zeroize),
116 .start_in(start_i),
117 .a_in(a_array[0]),
118 .b_in(b_array[0]),
119 .p_in(p_array[0]),
120 .s_in(s_array[0]),
121 .n_prime_in(n_prime_reg),
122 .odd(odd),
123
124 .a_out(a_array[1]),
125 .m_out(m_array[0]),
126 .c_out(c_array[0])
127 );
128
129 genvar i0;
130 generate
131 for (i0=0; i0 < PE_UNITS; i0++) begin : gen_PE //PE
general boxes
132 ecc_pe #(
133 .RADIX(RADIX)
134 )
135 box_i
136 (
137 .clk(clk),
138 .reset_n(reset_n),
139 .zeroize(zeroize),
140 .start_in(start_i),
141 .a_in(a_array[i0+1]),
142 .b_in(b_array[i0+1]),
143 .p_in(p_array[i0+1]),

```

```

144 .m_in(m_array[i0]),
145 .s_in(s_array[i0+1]),
146 .c_in(c_array[i0]),
147 .odd(odd),
148
149 .a_out(a_array[i0+2]),
150 .m_out(m_array[i0+1]),
151 .s_out(s_array[i0]),
152 .c_out(c_array[i0+1])
153 );
154 end
155 endgenerate
156
157 ecc_pe_final #(
158 .RADIX(RADIX)
159 )
160 final_box
161 (
162 .clk(clk),
163 .reset_n(reset_n),
164 .zeroize(zeroize),
165 .start_in(start_i),
166 .a_in(a_array[PE_UNITS+1]),
167 .b_in(b_array[PE_UNITS+1]),
168 .p_in(p_array[PE_UNITS+1]),
169 .m_in(m_array[PE_UNITS]),
170 .s_in(s_array[PE_UNITS+1]),
171 .c_in(c_array[PE_UNITS]),
172 .odd(odd),
173
174 .s_out(s_array[PE_UNITS]),
175 .c_out(c_array[PE_UNITS+1])
176 );
177
178 // Determines the input s for the final PE
179 assign s_array[PE_UNITS+1] = last_s_reg;
180 always_ff @(posedge clk or negedge reset_n)
181 begin
182 if (~reset_n)
183 last_s_reg <= 'b0;
184 else if (zeroize)
185 last_s_reg <= 'b0;
186 else if (start_i)
187 last_s_reg <= 'b0;
188 else
189 last_s_reg <= c_array[PE_UNITS+1][RADIX-1:0];
190 end
191
192 // Set odd flag
193 always_ff @(posedge clk or negedge reset_n)
194 begin
195 if (~reset_n)
196 odd <= 0;
197 else if (zeroize)
198 odd <= 0;
199 else if (start_i)
200 odd <= 1'b1;
201 else
202 odd <= ~odd;
203 end
204
205 always_ff @(posedge clk or negedge reset_n)
206 begin : input_reg
207 if (~reset_n) begin
208 a_reg <= 'b0;
209 b_reg <= 'b0;
210 p_reg <= 'b0;
211 n_prime_reg <= 'b0;
212 end
213 else if (zeroize) begin
214 a_reg <= 'b0;
215 b_reg <= 'b0;
216 p_reg <= 'b0;
217 n_prime_reg <= 'b0;
218 end
219 else if (start_i) begin
220 a_reg <= {zero_pad, opa_i};
221 b_reg <= {zero_pad, opb_i};
222 p_reg <= {zero_pad, n_i};
223 n_prime_reg <= n_prime_i;
224 end else begin
225 if (odd) begin
226 a_reg[(FULL_REG_SIZE-RADIX)-1 : 0] <=
a_reg[FULL_REG_SIZE-1 : RADIX];
227 a_reg[FULL_REG_SIZE-1 : FULL_REG_SIZE-RADIX]
<= 'b0; //Shift
228 end
229 end
230 end // input_reg
231
232 assign a_array[0] = a_reg[RADIX-1 : 0];
233
234 // Determines which b and p is pushed through the system
235 assign b_array[0] = b_reg[RADIX-1 : 0];
236 assign p_array[0] = p_reg[RADIX-1 : 0];
237 genvar j0;
238 generate
239 for (j0=1; j0 < (PE_UNITS+2); j0++) begin :
gen_b_p_array
240 assign b_array[j0] = odd ?
b_reg[(((2*j0)+1)*RADIX)-1 : (2*j0)*RADIX] :

```

```

b_reg[((2*j0)*RADIX)-1 : ((2*j0)-1)*RADIX];
241     assign p_array[j0] = odd ?
p_reg[(((2*j0)+1)*RADIX)-1 : (2*j0)*RADIX] :
p_reg[(((2*j0)*RADIX)-1 : ((2*j0)-1)*RADIX];
242     end
243     endgenerate
244
245     assign p_neg_reg = ~p_reg;
246
247     genvar i1;
248     generate
249     for (i1=0; i1 < ((2*(PE_UNITS+1))+1); i1++) begin :
gen_n_neg_array
250         assign p_neg_array[i1] = p_neg_reg[i1*RADIX +:
RADIX];
251     end
252     endgenerate
253
254     // Storing the partial results from the system
255     genvar t0;
256     generate
257     for (t0=0; t0 < ((2*(PE_UNITS+1))+1); t0++) begin :
gen_t_reg
258         if ((t0 == 0) | (t0 == 1)) begin
259             always_ff @(posedge clk or negedge reset_n)
260             begin
261                 if (~reset_n)
262                     t_reg[t0] <= '0;
263                 else if (zeroize)
264                     t_reg[t0] <= '0;
265                 else if (push_reg[(2*(PE_UNITS+1)) - t0])
266                     t_reg[t0] <= s_array[0];
267                 else
268                     t_reg[t0] <= t_reg[t0];
269             end
270         end
271         else begin
272             localparam int t0_WIDTH = $clog2((2*(PE_UNITS+1))+1);
273             localparam bit [t0_WIDTH-1:0] t0_unsigned =
t0[t0_WIDTH-1:0];
274             always_ff @(posedge clk or negedge reset_n)
275             begin
276                 if (~reset_n)
277                     t_reg[t0] <= '0;
278                 else if (zeroize)
279                     t_reg[t0] <= '0;
280                 else if (push_reg[(2*(PE_UNITS+1)) - t0])
281                     t_reg[t0] <= s_array[t0_unsigned >>
1];
282                 else
283                     t_reg[t0] <= t_reg[t0];
284             end
285         end
286     end // gen_t_reg
287     endgenerate
288
289     genvar t1;
290     generate
291     for (t1=0; t1 < ((2*(PE_UNITS+1))+1); t1++) begin :
gen_sub_t
292         if (t1 == 0)
293             always_comb sub_b_i[t1] = 1;
294         else
295             always_comb sub_b_i[t1] = sub_b_o[t1 - 1];
296
297         always_ff @(posedge clk or negedge reset_n) begin
298             if (~reset_n)
299                 sub_b_o[t1] <= '0;
300             else if (zeroize)
301                 sub_b_o[t1] <= '0;
302             else
303                 sub_b_o[t1] <= sub_res[t1][RADIX];
304         end
305     end
306     endgenerate
307
308     genvar t2;
309     generate
310     for (t2=0; t2 < ((2*(PE_UNITS+1))+1); t2++)
311     begin : gen_sub_res_t
312         //if (~reset_n)
313         //    sub_res[t2] = 0;
314         //else if (push_reg[2*(PE_UNITS+1) - t2])
315         //    localparam int t2_WIDTH = $clog2((2*(PE_UNITS+1))
+1);
316         //    localparam bit [t2_WIDTH-1:0] t2_unsigned =
t2[t2_WIDTH-1:0];
317         //    if ((t2 == 0) | (t2 == 1))
318         //        always_comb sub_res[t2] = s_array[0] +
p_neg_array[t2] + sub_b_i[t2];
319         //    else
320         //        always_comb sub_res[t2] = s_array[t2_unsigned
>> 1] + p_neg_array[t2] + sub_b_i[t2];
321     end // gen_sub_res_t
322     endgenerate
323
324     genvar t3;
325     generate
326     for (t3=0; t3 < ((2*(PE_UNITS+1))+1); t3++) begin :

```

```

gen_t_sub_reg
326     always_ff @(posedge clk or negedge reset_n) begin
327         if (~reset_n)
328             t_subtracted_reg[t3] <= '0;
329         else if (zeroize)
330             t_subtracted_reg[t3] <= '0;
331         else if (push_reg[(2*(PE_UNITS+1)) - t3])
332             t_subtracted_reg[t3] <= sub_res[t3]
[RADIX-1 : 0];
333         else
334             t_subtracted_reg[t3] <= t_subtrac-
ted_reg[t3];
335     end
336     end // gen_t_sub_reg
337     endgenerate
338
339     // Storing the final results from the system
340     genvar k0;
341     generate
342     for (k0=0; k0 < ((2*(PE_UNITS+1))+1); k0++) begin :
gen_p_o
343         assign p_internal[k0*RADIX +: RADIX] = t_reg[k0];
344         assign p_subtracted_internal[k0*RADIX +: RADIX] =
t_subtracted_reg[k0];
345     end
346     endgenerate
347
348     always_comb last_reduction = (sub_b_o[2*(PE_UNITS+1)]);
349     assign p_o = (last_reduction)? p_subtrac-
ted_internal[REG_SIZE-1:0] : p_internal[REG_SIZE-1:0];
350
351     // Determines when results are ready based on S_NUM
352     always_ff @(posedge clk or negedge reset_n) begin
353         if (~reset_n)
354             push_reg <= 'b0;
355         else if (zeroize)
356             push_reg <= 'b0;
357         else if (start_i)
358             push_reg[(3*S_NUM)-1] <= 'b1;
359         else // one shift to right
360             push_reg <= {1'b0, push_reg[(3*S_NUM)-1 : 1]};
361     end
362
363     always_ff @(posedge clk or negedge reset_n) begin
364         if (~reset_n)
365             push_reg_eq_zero_ff <= '0;
366         else if (zeroize)
367             push_reg_eq_zero_ff <= '0;
368         else
369             push_reg_eq_zero_ff <= push_reg_eq_zero;
370     end
371
372     assign push_reg_eq_zero = push_reg == 'b0;
373     assign ready_o = push_reg_eq_zero & ~push_reg_eq_zero_ff;
374
375 endmodule

```

11. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_mult_dsp.sv (39 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 //
17 // ecc_mult_dsp.sv
18 // -----
19 // General multiplier to perform P = A*B.
20 //
21 //
22 //
=====
23
24 module ecc_mult_dsp #(
25     parameter RADIX = 32
26 )
27 (
28     // DATA PORT
29     input wire [RADIX-1 : 0] A_i,
30     input wire [RADIX-1 : 0] B_i,

```

```

31
32     output logic [(2*RADIX)-1 : 0]      P_o
33 );
34
35     always_comb begin
36         P_o = A_i * B_i;
37     end
38
39 endmodule

```

12. `chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_params_pkg.sv`
(77 lines)

[illegible]

```

ab5584462933e220ef;
61 parameter [REG_SIZE-1 : 0] ONE_p_MONT =
384'h100000000ffffffffff00000001000000000000;
62 parameter [REG_SIZE-1 : 0] R2_p_MONT =
384'h10000000200000000ffffffffffe00000000000000200000000ffffff-
fe0000000100000000000000000000000000;
63 parameter [REG_SIZE-1 : 0] G_X_MONT =
384'h1513812ff723614ede2b6454868459a30eff879c3afc541b4d6e6e1e26a517
af7bfa676e7565fc860766239cadc2299e;
64 parameter [REG_SIZE-1 : 0] G_Y_MONT = 384'hc5e9d-
d800263969a840c63521968f4ff-
d98bade7562e83b050cd38548201425567d193dad1f8af93bd163abc25a15;
65 parameter [MULT_RADIX-1 : 0] PRIME_mu = 64'h100000001;
66
67 // group order parameters in Montgomery domain
68 parameter [REG_SIZE-1 : 0] R2_q_MONT =
384'h28266895d40d49174aab1cc5bf030606de609f43cc9601f9ebbfbfd4b3f-
fe90bfead8c2590449c1c55daf7abdb883e5e32;
69 parameter [REG_SIZE-1 : 0] ONE_q_MONT =
384'h389bc27e0bc8d220a7e5f24db74f58851313e695333ad68d000000000000;
70 parameter [MULT_RADIX-1 : 0] GROUP_ORDER_mu = 64'h6089e88fd-
c45;
71
72 endpackage
73
74 `endif
75 //

```

```
76 // EOF ecc_params.sv
77 //
```

13. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe.sv (162 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
==
16 //
17 // ecc_pe.sv
18 // -----
19 // processing element for Montgomery Multiplier based on
20 // Optimized Algorithms and Architectures for Montgomery Mul-
tiplication
21 // for Post-quantum Cryptography by Rami Elkhatib et. al.
22 //
23 //
=====
==
24
25 module ecc_pe #(
26     parameter RADIX = 32
27 )
28 (
29     // Clock and reset.
30     input wire          clk,
31     input wire          reset_n,
32     input wire          zeroize,
33
34     input wire          start_in,
35     // DATA PORT
36     input wire [RADIX-1:0] a_in,
37     input wire [RADIX-1:0] b_in,
38     input wire [RADIX-1:0] p_in,
39     input wire [RADIX-1:0] m_in,
40     input wire [RADIX-1:0] s_in,
41     input wire [RADIX :0] c_in,
42     input wire          odd,
43
44     output logic [RADIX-1:0] a_out,
45     output logic [RADIX-1:0] m_out,
46     output logic [RADIX-1:0] s_out,
47     output logic [RADIX :0] c_out
48 );
49
50
51 //-----
-----
52 // ecc_pe

```

```

53 //-----
54 logic [(2*RADIX)-1 : 0] mult0_out;
55 logic [(2*RADIX)-1 : 0] mult1_out;
56
57 logic [RADIX : 0] c_mux;
58 logic [RADIX-1 : 0] s_mux;
59
60 logic [2*RADIX : 0] res;
61 logic [RADIX-1 : 0] res_LSW;
62 logic [RADIX : 0] res_MSW;
63
64 logic carry_garbage_bit;
65
66 //-----
67 // Module instantiations.
68 //-----
69 ecc_mult_dsp #(
70     .RADIX(RADIX)
71 ) MULT1 (
72     .A_i(a_in),
73     .B_i(b_in),
74     .P_o(mult0_out)
75 );
76
77 ecc_mult_dsp #(
78     .RADIX(RADIX)
79 ) MULT2 (
80     .A_i(p_in),
81     .B_i(m_in),
82     .P_o(mult1_out)
83 );
84
85 //-----
86 // pe_logic
87 //
88 // The logic needed to update the multiplier inputs.
89 //-----
90 assign c_mux = odd ? c_out : c_in;
91 assign s_mux = odd ? s_in : s_out;
92
93 assign res_MSW = res[2*RADIX : RADIX];
94 assign res_LSW = res[RADIX-1 : 0];
95
96 always_comb {carry_garbage_bit, res} = mult0_out + mul-
t1_out + c_mux + s_mux;
97
98 //-----
99 // register updates
100 //
101 //-----
102
103 always_ff @(posedge clk or negedge reset_n) begin
104     if (~reset_n) begin
105         c_out <= 'b0;
106     end
107     else if (zeroize) begin
108         c_out <= 'b0;
109     end
110     else if (start_in) begin
111         c_out <= 'b0;
112     end
113     else begin
114         c_out <= res_MSW;
115     end
116 end
117
118 always_ff @(posedge clk or negedge reset_n) begin
119     if (~reset_n) begin
120         s_out <= 'b0;
121     end
122     else if (zeroize) begin
123         s_out <= 'b0;
124     end
125     else if (start_in) begin
126         s_out <= 'b0;
127     end
128     else begin
129         s_out <= res_LSW;
130     end
131 end
132
133 always_ff @(posedge clk or negedge reset_n) begin
134     if (~reset_n) begin
135         a_out <= 'b0;
136     end
137     else if (zeroize) begin
138         a_out <= 'b0;
139     end
140     else if (start_in) begin
141         a_out <= 'b0;
142     end
143     else begin
144         if (odd) begin
145             a_out <= a_in;
146         end
147     end
148 end
149
150 always_ff @(posedge clk or negedge reset_n) begin
151     if (~reset_n) begin
152         m_out <= 'b0;
153     end
154     else if (zeroize) begin
155         m_out <= 'b0;
156     end
157     else if (start_in) begin
158         m_out <= 'b0;
159     end
160     else begin
161         if (odd) begin
162             m_out <= m_in;
163         end
164     end
165 end
166
167 endmodule

```

```

146 always_ff @(posedge clk or negedge reset_n) begin
147     if (~reset_n) begin
148         m_out <= 'b0;
149     end
150     else if (zeroize) begin
151         m_out <= 'b0;
152     end
153     else if (start_in) begin
154         m_out <= 'b0;
155     end
156     else begin
157         if (odd) begin
158             m_out <= m_in;
159         end
160     end
161 end
162 endmodule

```

14. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe_final.sv (128 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
4 // cense");
5 // you may not use this file except in compliance with the Li-
6 // cense.
7 // You may obtain a copy of the License at
8 // http://www.apache.org/licenses/LICENSE-2.0
9 //
10 // Unless required by applicable law or agreed to in writing,
11 // software
12 // distributed under the License is distributed on an "AS IS"
13 // BASIS,
14 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
15 // press or implied.
16 // See the License for the specific language governing permis-
17 // sions and
18 // limitations under the License.
19 //
20 //
21 //
22 //
23 //
24 //
25 //
26 //
27 //
28 //
29 //
30 //
31 //
32 //
33 //
34 //
35 //
36 //
37 //
38 //
39 //
40 //
41 //
42 //
43 //
44 //
45 //
46 //
47 //
48 //
49 //
50 //
51 //
52 //
53 //
54 //
55 //
56 //
57 //
58 //
59 //
60 //
61 //
62 //
63 //
64 //

```

```

65 // Module instantiations.
66 //-----
67 ecc_mult_dsp #(
68     .RADIX(RADIX)
69 ) MULT1 (
70     .A_i(a_in),
71     .B_i(b_in),
72     .P_o(mult0_out)
73 );
74
75 ecc_mult_dsp #(
76     .RADIX(RADIX)
77 ) MULT2 (
78     .A_i(p_in),
79     .B_i(m_in),
80     .P_o(mult1_out)
81 );
82
83 //-----
84 // pe_logic
85 //
86 // The logic needed to update the multiplier inputs.
87 //-----
88 assign c_mux = odd ? c_out : c_in;
89 assign s_mux = odd ? s_in : s_out;
90
91 assign res_MSW = res[2*RADIX : RADIX];
92 assign res_LSW = res[RADIX-1 : 0];
93
94 always_comb {carry_garbage_bit, res} = mult0_out + mul-
t1_out + c_mux + s_mux;
95
96 //-----
97 // register updates
98 //
99 //-----
100 always_ff @(posedge clk or negedge reset_n) begin
101     if (~reset_n) begin
102         c_out <= 'b0;
103     end
104     else if (zeroize) begin
105         c_out <= 'b0;
106     end
107     else if (start_in) begin
108         c_out <= 'b0;
109     end
110     else begin
111         c_out <= res_MSW;
112     end
113 end
114
115 always_ff @(posedge clk or negedge reset_n) begin
116     if (~reset_n) begin
117         s_out <= 'b0;
118     end
119     else if (zeroize) begin
120         s_out <= 'b0;
121     end
122     else if (start_in) begin
123         s_out <= 'b0;
124     end
125     else begin
126         s_out <= res_LSW;
127     end
128 end
129 endmodule

```

15. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe_first.sv (204 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
4 // cense");
5 // you may not use this file except in compliance with the Li-
6 // cense.
7 // You may obtain a copy of the License at
8 //
9 // http://www.apache.org/licenses/LICENSE-2.0
10 //
11 // Unless required by applicable law or agreed to in writing,
12 // software
13 // distributed under the License is distributed on an "AS IS"
14 // BASIS,
15 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
16 // press or implied.
17 // See the License for the specific language governing permis-
18 // sions and
19 // limitations under the License.
20 //
21 //
22 //-----
23 //
24 // ecc_pe_first.sv
25 // -----

```

```

19 // first processing element for Montgomery Multiplier based
20 // on
21 // Optimized Algorithms and Architectures for Montgomery Mul-
22 // tiplication
23 // for Post-quantum Cryptography by Rami Elkhathib et. al.
24 //
25 //-----
26 module ecc_pe_first #(
27     parameter RADIX = 32
28 )
29 (
30     // Clock and reset.
31     input wire clk,
32     input wire reset_n,
33     input wire zeroize,
34     input wire start_in,
35     // DATA PORT
36     input wire [RADIX-1:0] a_in,
37     input wire [RADIX-1:0] b_in,
38     input wire [RADIX-1:0] p_in,
39     input wire [RADIX-1:0] s_in,
40     input wire [RADIX-1:0] n_prime_in,
41     input wire odd,
42     output logic [RADIX-1:0] a_out,
43     output logic [RADIX-1:0] m_out,
44     output logic [RADIX :0] c_out
45 );
46
47 //-----
48 // ecc_pe_first
49 //-----
50
51 logic [RADIX-1:0] s_reg;
52 logic [RADIX-1:0] s_val;
53 logic [RADIX-1:0] a_reg;
54 logic [RADIX-1:0] a_val;
55
56 logic [(2*RADIX)-1 : 0] mult_out_0;
57 logic [RADIX-1 : 0] mult_out_0_MSW;
58 logic [RADIX-1 : 0] mult_out_0_LSW;
59
60 logic [RADIX : 0] res_0;
61 logic [RADIX-1 : 0] sum_0;
62 logic [RADIX-1 : 0] carry_0;
63 logic [RADIX-1 : 0] sum_0_reg;
64
65 logic [RADIX : 0] res_1;
66 logic [RADIX-1 : 0] carry_1;
67
68 logic [RADIX : 0] c_0;
69 logic [RADIX : 0] c_0_reg;
70 logic [RADIX : 0] c_1;
71
72 logic [(2*RADIX)-1 : 0] mult_out_1;
73 logic [RADIX-1 : 0] m_temp;
74 logic [RADIX-1 : 0] m_temp_reg;
75
76 logic [(2*RADIX)-1 : 0] mult_out_2;
77 logic [RADIX-1 : 0] mult_out_2_MSW;
78 logic [RADIX-1 : 0] mult_out_2_LSW;
79
80 logic carry_garbage_bit;
81
82 //-----
83 // Module instantiations.
84 //-----
85
86 ecc_mult_dsp #(
87     .RADIX(RADIX)
88 ) MULT0 (
89     .A_i(a_val),
90     .B_i(b_in),
91     .P_o(mult_out_0)
92 );
93
94 ecc_mult_dsp #(
95     .RADIX(RADIX)
96 ) MULT1 (
97     .A_i(n_prime_in),
98     .B_i(sum_0),
99     .P_o(mult_out_1)
100 );
101
102 ecc_mult_dsp #(
103     .RADIX(RADIX)
104 ) MULT2 (
105     .A_i(m_temp_reg),
106     .B_i(p_in),
107     .P_o(mult_out_2)
108 );
109
110 //-----

```

```

111 // pe_logic
112 //
113 // The logic needed to update the multiplier inputs.
114 //-----
115 assign m_temp = mult_out_1[RADIX-1:0];
116
117 assign mult_out_0_MSW = mult_out_0[(2*RADIX)-1 : RADIX];
118 assign mult_out_0_LSW = mult_out_0[RADIX-1 : 0];
119 assign mult_out_2_MSW = mult_out_2[(2*RADIX)-1 : RADIX];
120 assign mult_out_2_LSW = mult_out_2[RADIX-1 : 0];
121
122 assign s_val = odd ? s_in : s_reg;
123 assign a_val = odd ? a_in : a_reg;
124
125 always_comb begin
126     res_0 = s_val + mult_out_0_LSW;
127     sum_0 = res_0[RADIX-1 : 0];
128     carry_0 = res_0[RADIX];
129 end
130
131 always_comb begin
132     res_1 = sum_0_reg + mult_out_2_LSW;
133     carry_1 = res_1[RADIX];
134 end
135
136 always_comb c_0 = mult_out_0_MSW + carry_0;
137
138 always_comb c_1 = mult_out_2_MSW + carry_1;
139
140 assign {carry_garbage_bit, c_out} = c_0_reg + c_1;
141
142 //-----
143 // register updates
144 //
145 //-----
146 always_ff @(posedge clk or negedge reset_n) begin
147     if (~reset_n) begin
148         c_0_reg <= 'b0;
149         sum_0_reg <= 'b0;
150         m_temp_reg <= 'b0;
151         s_reg <= 'b0;
152         a_reg <= 'b0;
153     end
154     else if (zeroize) begin
155         c_0_reg <= 'b0;
156         sum_0_reg <= 'b0;
157         m_temp_reg <= 'b0;
158         s_reg <= 'b0;
159         a_reg <= 'b0;
160     end
161     else if (start_in) begin
162         c_0_reg <= 'b0;
163         sum_0_reg <= 'b0;
164         m_temp_reg <= 'b0;
165         s_reg <= 'b0;
166         a_reg <= 'b0;
167     end
168     else begin
169         s_reg <= s_in;
170         a_reg <= a_in;
171         m_temp_reg <= m_temp;
172         c_0_reg <= c_0;
173         sum_0_reg <= sum_0;
174     end
175 end
176
177 always_ff @(posedge clk or negedge reset_n) begin
178     if (~reset_n) begin
179         a_out <= 'b0;
180     end
181     else if (zeroize) begin
182         a_out <= 'b0;
183     end
184     else if (start_in) begin
185         a_out <= 'b0;
186     end
187     else begin
188         a_out <= a_val;
189     end
190 end
191
192 always_ff @(posedge clk or negedge reset_n) begin
193     if (~reset_n) begin
194         m_out <= 'b0;
195     end
196     else if (zeroize) begin
197         m_out <= 'b0;
198     end
199     else if (start_in) begin
200         m_out <= 'b0;
201     end
202     else begin
203         m_out <= m_temp;
204     end
205 end
206 endmodule

```

16. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pm_ctrl.sv (404 lines)

```

1 // SPD-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
4 // cense");
5 // you may not use this file except in compliance with the Li-
6 // cense.
7 // You may obtain a copy of the License at
8 //
9 // http://www.apache.org/licenses/LICENSE-2.0
10 // Unless required by applicable law or agreed to in writing,
11 // software
12 // distributed under the License is distributed on an "AS IS"
13 // BASIS,
14 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
15 // press or implied.
16 // See the License for the specific language governing permis-
17 // sions and
18 // limitations under the License.
19 //
20 //
21 //=====
22 //
23 // ecc_pm_ctrl.sv
24 // -----
25 // ECC point multiplication controller for the Secp384.
26 //
27 // The architecture includes ecc_pm_sequencer including the
28 // required
29 // sequence to perform:
30 // - point multiplication (pm) based on Montgomery ladder
31 // and Fermat's little
32 // theorem (FLT) inversion
33 // - Montgomery ladder initialization with base point G or
34 // public key PK in
35 // keygen, signing, or verifying operations.
36 // - Montgomery ladder steps: including the iterative point
37 // addition (PA)
38 //
39 // and point doubling (PD) operations in Montgomery domain
40 // using projective
41 // coordinates. The number of iteration is equal to REG_S-
42 // IZE of scalar in
43 //
44 // unprotected point multiplication, and REG_SIZE+RND_SIZE of
45 // scalar in the
46 //
47 // SCA protected operation.
48 // - Modular Inversion (mod p, mod q) is based on Fermat's
49 // little theorem (FLT)
50 // inversion.
51 // - Conversion from projective coordinates in Montgomery
52 // domain (X,Y,Z) to
53 // affine coordinates in the normal domain (x,y).
54 // - Modular arithmetic operations such as (h + r*privKey)
55 // in signing
56 // or (h*s_inv) and (r*s_inv) in verifying
57 //
58 //
59 //=====
60 //
61 module ecc_pm_ctrl
62 import ecc_pm_uop_pkg::*;
63 #(
64     parameter REG_SIZE      = 384,
65     parameter RND_SIZE      = 192,
66     parameter INSTR_SIZE    = 24
67 )
68 (
69     // Clock and reset.
70     input wire      clk,
71     input wire      reset_n,
72     input wire      zeroize,
73
74     // from arith_unit
75     input wire [3 : 0] ecc_cmd_i,
76     input wire      digit_i,
77     output pm_instr_struct_t instr_o,
78     output logic     req_digit_o,
79     output wire      busy_o
80 );
81 //-----
82 // Internal constant and parameter definitions.
83 //-----
84 localparam [7 : 0] MULT_DELAY      = 8'd27; //28 -1;
85 localparam [7 : 0] ADD_DELAY       = 8'd1;  // 2 -1;
86
87 localparam [9 : 0] Secp384_SCA_MONT_COUNT = REG_SIZE[9 : 0]
88 + RND_SIZE[9 : 0];

```



```

71 localparam [9 : 0] Secp384_MONT_COUNT = REG_SIZE[9 :
0];
72
73 //-----
74 // Registers
75 //-----
76
77 logic [PROG_ADDR_W-1 : 0] prog_cntr;
78 logic [9 : 0] mont_cntr;
79 logic [7 : 0] stall_cntr;
80
81 // Program pipeline signals
82 logic [PROG_ADDR_W-1 : 0] prog_addr;
83 pm_instr_struct_t prog_instr;
84 pm_instr_struct_t prog_instr_pipe1;
85 pm_instr_struct_t prog_instr_pipe2;
86
87 logic [3 : 0] ecc_cmd_reg;
88
89 logic stalled, stalled_pipe1;
90 logic stall_flag;
91 logic mont_ladder;
92 //-----
93 // pm sequencer fsm
94 //-----
95 assign prog_addr = prog_cntr;
96
97 ecc_pm_sequencer #(
98     .ADDR_WIDTH(PROG_ADDR_W),
99     .DATA_WIDTH(INSTRUCTION_LENGTH)
100 )
101 i_ecc_pm_sequencer(
102     .clk(clk),
103     .reset_n(reset_n),
104     .zeroize(zeroize),
105     .ena(1'b1),
106     .addr(prog_addr),
107     .dout(prog_instr)
108 );
109
110 always_ff @(posedge clk or negedge reset_n)
111 begin : ff_stalled_reg
112     if(!reset_n)
113         stalled_pipe1 <= '0;
114     else if (zeroize)
115         stalled_pipe1 <= '0;
116     else
117         stalled_pipe1 <= stalled;
118 end // ff_stalled_reg
119
120 //-----
121 // Point_multiplication_FSM_flow
122 //
123 // This FSM starts with the keygen/signing/verifying com-
mand to
124 // perform different operations.
125 // Active low and async reset.
126 //-----
127 always_ff @(posedge clk or negedge reset_n)
128 begin : pm_fsm
129     if(!reset_n) begin
130         prog_cntr <= '0;
131         mont_cntr <= '0;
132         stall_cntr <= '0;
133         stalled <= '0;
134         req_digit_o <= '0;
135         mont_ladder <= '0;
136         ecc_cmd_reg <= '0;
137     end
138     else if (zeroize) begin
139         prog_cntr <= '0;
140         mont_cntr <= '0;
141         stall_cntr <= '0;
142         stalled <= '0;
143         req_digit_o <= '0;
144         mont_ladder <= '0;
145         ecc_cmd_reg <= '0;
146     end
147     else begin
148         if (stalled & (stall_cntr > 0)) begin
149             stall_cntr <= stall_cntr - 1;
150         end
151         else if (stall_flag & (!stalled) & (!
stalled_pipe1)) begin
152             unique case (prog_instr.opcode)
153                 UOP_DO_ADD_p : begin stalled <= 1'b1;
stall_cntr <= ADD_DELAY; end // ADD
154                 UOP_DO_SUB_p : begin stalled <= 1'b1;
stall_cntr <= ADD_DELAY; end // SUB
155                 UOP_DO_MUL_p : begin stalled <= 1'b1;
stall_cntr <= MULT_DELAY; end // MULT
156                 UOP_DO_ADD_q : begin stalled <= 1'b1;
stall_cntr <= ADD_DELAY; end // ADD
157                 UOP_DO_SUB_q : begin stalled <= 1'b1;

```

```

stall_cntr <= ADD_DELAY; end // SUB
158                 UOP_DO_MUL_q : begin stalled <= 1'b1;
stall_cntr <= MULT_DELAY; end // MULT
159                 default : begin stalled <= 1'b0;
stall_cntr <= '0; end
160             endcase
161         end
162     else begin
163         stalled <= 0;
164         unique case (prog_cntr)
165             NOP : begin // Waiting for new valid
command
166                 ecc_cmd_reg <= ecc_cmd_i;
167                 unique case (ecc_cmd_i)
168                     KEYGEN_CMD : begin // keygen
169                         mont_cntr <=
Secp384_SCA_MONT_COUNT;
170                         prog_cntr <= PM_INIT_G_S;
171                     end
172                     SIGN_CMD : begin // signing
173                         mont_cntr <=
Secp384_SCA_MONT_COUNT;
174                         prog_cntr <= PM_INIT_G_S;
175                     end
176                 end
177             end
178             VER_PART0_CMD : begin // verify-
ing part0
179                 prog_cntr <= VER0_P0_S;
180             end
181             VER_PART1_CMD : begin // verify-
ing part1
182                 mont_cntr <=
Secp384_SCA_MONT_COUNT;
183                 prog_cntr <= PM_INIT_G_S;
184             end
185             VER_PART2_CMD : begin // verify-
ing part2
186                 prog_cntr <= VER1_ST_S;
187             end
188             CHK_PK_CMD : begin
189                 prog_cntr <= CHK_PK_S;
190             end
191             DH_SHARED_CMD : begin // DH
192                 mont_cntr <=
Secp384_SCA_MONT_COUNT;
193                 prog_cntr <= PM_INIT_DH_S;
194             end
195             default :
196                 prog_cntr <= NOP;
197             endcase
198             req_digit_o <= 0;
199         end
200     end
201
202 VER1_ST_E : begin // Storing the VER1 MONT
results before starting VER2
203     mont_cntr <= Secp384_MONT_COUNT;
204     prog_cntr <= PM_INIT_PK_S;
205 end
206
207 // Montgomery Ladder
208 PM_INIT_G_E : begin //End of initilize R0
= Randomized G
209     prog_cntr <= PM_INIT_S;
210 end
211
212 PM_INIT_PK_E : begin //End of initilize R0 =
PK
213     prog_cntr <= PM_INIT_S;
214 end
215
216 PM_INIT_DH_E : begin //End of initilize R0 =
Randomized PK
217     prog_cntr <= PM_INIT_S;
218 end
219
220 PM_INIT_E : begin // End of initilaze R1 = 0
221     mont_cntr <= mont_cntr - 1;
222     req_digit_o <= 1;
223     prog_cntr <= PA_S;
224     mont_ladder <= 1;
225 end
226
227 PA_S : begin //Start of point addition
228     req_digit_o <= 0;
229     prog_cntr <= prog_cntr + 1;
230 end
231
232 PA_E : begin //End of point addition
233     prog_cntr <= PD_S;
234 end
235
236
237

```



```

238     end
239
240     PD_E : begin //End of point doubling
241         if (mont_cntr == 0) begin // Mont-
gomery ladder is done
242             unique case (ecc_cmd_reg)
243                 VER_PART1_CMD : prog_cntr <=
NOP;
244                 VER_PART2_CMD : prog_cntr <=
VER2_PA_S;
245                 default : prog_cntr <=
INV_S;
246             endcase
247             mont_ladder <= 0;
248             req_digit_o <= 0;
249         end
250     else begin
251         mont_cntr <= mont_cntr - 1;
252         req_digit_o <= 1;
253         prog_cntr <= PA_S;
254     end
255 end
256
257 INV_E : begin // End of inversion mod p
258     prog_cntr <= CONV_S;
259 end
260
261 CONV_E : begin // End of conversion from
projective Mont (X,Y,Z) to affine normanl (x,y)
262     unique case (ecc_cmd_reg)
263         SIGN_CMD : prog_cntr <= SIGN0_S;
264         default : prog_cntr <= NOP;
265     endcase
266 end
267
268 SIGN0_E : begin // End of signing part 0
to convert inputs to Mont domain & compute (h + r*privKey)
269     prog_cntr <= INVq_S;
270 end
271
272 INVq_E : begin // End of inversion mod q
273     unique case (ecc_cmd_reg)
274         SIGN_CMD : prog_cntr <=
SIGN1_S;
275         VER_PART0_CMD : prog_cntr <=
VER0_P1_S;
276         default : prog_cntr <= NOP;
277     endcase
278 end
279
280 SIGN1_E : begin // End of signing part 1
to compute s
281     prog_cntr <= NOP;
282 end
283
284 CHK_PK_E : begin // End of public key val-
idation before verifying process
285     prog_cntr <= NOP;
286 end
287
288 VER0_P0_E : begin // End of verfyng part
0 to convert inputs to Mont domain
289     prog_cntr <= INVq_S;
290 end
291
292 VER0_P1_E : begin // End of verfyng part
1 to compute (h*s_inv) and (r*s_inv)
293     prog_cntr <= NOP;
294 end
295
296 VER2_PA_E : begin // End of point addition
of PA((h*s_inv)*G, (r*s_inv)*PK)
297     prog_cntr <= INV_S;
298 end
299
300 PM_INIT_G_S,
301 PM_INIT_PK_S,
302 PM_INIT_S,
303 PD_S,
304 INV_S,
305 INVq_S,
306 CONV_S,
307 SIGN0_S,
308 SIGN1_S,
309 VER0_P0_S,
310 VER0_P1_S,
311 VER1_ST_S,
312 VER2_PA_S,
313 PM_INIT_DH_S: begin
314     prog_cntr <= prog_cntr + 1;
315 end
316
317 default : begin
318     prog_cntr <= prog_cntr + 1;
319 end
320 endcase

```

```

321     end
322 end
323 end // pm_fsm
324
325 always_ff @(posedge clk or negedge reset_n)
326 begin : sequencer_pipeline1
327     if (!reset_n)
328         prog_instr_pipe1 <= '{UOP_NOP, '0, '0};
329     else if (zeroize)
330         prog_instr_pipe1 <= '{UOP_NOP, '0, '0};
331     else begin
332         if (stalled_pipe1)
333             prog_instr_pipe1 <= prog_instr_pipe1;
334         else
335             prog_instr_pipe1 <= prog_instr;
336     end
337 end // sequencer_pipeline1
338
339 always_ff @(posedge clk or negedge reset_n)
340 begin : sequencer_pipeline2
341     if (!reset_n)
342         prog_instr_pipe2 <= '{UOP_NOP, '0, '0};
343     else if (zeroize)
344         prog_instr_pipe2 <= '{UOP_NOP, '0, '0};
345     else begin
346         if (stalled_pipe1)
347             prog_instr_pipe2 <= prog_instr_pipe2;
348         else
349             prog_instr_pipe2 <= prog_instr_pipe1;
350     end
351 end // sequencer_pipeline2
352
353 /*
354 always_comb
355 begin : delay_flag
356     unique case (prog_instr.opcode)
357         UOP_DO_ADD_p : stall_flag = 1;
358         UOP_DO_SUB_p : stall_flag = 1;
359         UOP_DO_MUL_p : stall_flag = 1;
360         UOP_DO_ADD_q : stall_flag = 1;
361         UOP_DO_SUB_q : stall_flag = 1;
362         UOP_DO_MUL_q : stall_flag = 1;
363         default : stall_flag = 0;
364     endcase
365 end // delay_flag
366 */
367
368 always_comb stall_flag = (prog_instr.opcode.add_en ||
prog_instr.opcode.mult_en);
369
370 //Instruction out to arithmetic units and memory
371 always_ff @(posedge clk or negedge reset_n)
372 begin : instruction_out
373     if (!reset_n)
374         instr_o <= '{UOP_NOP, '0, '0};
375     else if (zeroize)
376         instr_o <= '{UOP_NOP, '0, '0};
377     else begin
378         instr_o.opcode <= prog_instr_pipe2.opcode;
379
380         if (mont_ladder) begin
381             // constant-time conditional swaps depending
on the value of the currently processed bit of secret key
382
383             //Addr A for ADD/SUB result
384             if (prog_instr_pipe2.opa_addr[3 +: OPR_AD-
DR_WIDTH-3] == 3'b001)
385                 instr_o.opa_addr <= {prog_in-
str_pipe2.opa_addr[3 +: OPR_ADDR_WIDTH-3], digit_i ^ prog_in-
str_pipe2.opa_addr[2], prog_instr_pipe2.opa_addr[1 : 0]};
386             else
387                 instr_o.opa_addr <= prog_in-
str_pipe2.opa_addr;
388
389             //Addr B for MULT result
390             if (prog_instr_pipe2.opb_addr[3 +: OPR_AD-
DR_WIDTH-3] == 3'b001)
391                 instr_o.opb_addr <= {prog_in-
str_pipe2.opb_addr[3 +: OPR_ADDR_WIDTH-3], digit_i ^ prog_in-
str_pipe2.opb_addr[2], prog_instr_pipe2.opb_addr[1 : 0]};
392             else
393                 instr_o.opb_addr <= prog_in-
str_pipe2.opb_addr;
394         end
395     else begin
396         instr_o.opa_addr <= prog_in-
str_pipe2.opa_addr; //Addr A for ADD/SUB result
397         instr_o.opb_addr <= prog_in-
str_pipe2.opb_addr; //Addr B for MULT result
398     end
399 end
400 end // instruction_out
401
402 assign busy_o = ~(prog_cntr == NOP);
403
404 endmodule

```

Caliptra AES

What this is. AES within a datacentre root of trust.

Why it exists. Because confidentiality is required alongside integrity, and a root of trust must provide both without exporting keys.

Undergraduate review · EECS concepts. Block cipher modes; key hierarchies; hardware key paths.

Graduate · Practice note. Compare this implementation's countermeasure set with OpenTitan's. Different threat models — a datacentre module versus a device exposed to physical attack — produce different designs from the same standard.

7 files • 5,137 lines

API — aes.sv

Table 1427. chipsalliance_caliptra-rtl/src/aes/rtl/aes.sv — module / interface / package (1개)

이름	종류
aes	module

Table 1428. chipsalliance_caliptra-rtl/src/aes/rtl/aes.sv — parameter (13개)

파라미터	형/폭	기본값
AES192Enable	bit	1
AESGCMEnable	bit	1
SecMasking	bit	1
SecSBoxImpl	sbox_impl_e	SBoxImplDom
SecStartTriggerDelay	int unsigned	0
SecAllowForcingMasks	bit	0
SecSkipPRNGReseeding	bit	0
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
RndCnstClearingLfsrSeed	clearing_lfsr_seed_t	RndCnstClearingLfsrSeedDefault
RndCnstClearingLfsrPerm	clearing_lfsr_perm_t	RndCnstClearingLfsrPermDefault
RndCnstClearingSharePerm	clearing_lfsr_perm_t	RndCnstClearingSharePermDefault
RndCnstMaskingLfsrSeed	masking_lfsr_seed_t	RndCnstMaskingLfsrSeedDefault
RndCnstMaskingLfsrPerm	masking_lfsr_perm_t	RndCnstMaskingLfsrPermDefault

Table 1429. chipsalliance_caliptra-rtl/src/aes/rtl/aes.sv — localparam (2개)

상수	형/폭	값
EntropyWidth	int unsigned	edn_pkg::ENDPOINT_BUS_WIDTH
CLP_AES_KV_CHUNK_SIZE		NumRegsData*32

Table 1430. chipsalliance_caliptra-rtl/src/aes/rtl/aes.sv — 포트 (18개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
rst_shadowed_ni	input	logic
idle_o	output	caliptra_prim_mubi_pkg::mubi4_t
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
clk_edn_i	input	logic
rst_edn_ni	input	logic
edn_o	output	edn_pkg::edn_req_t
edn_i	input	edn_pkg::edn_rsp_t
input_ready_o	output	logic
output_valid_o	output	logic
caliptra2aes	input	caliptra2aes_t
aes2caliptra	output	aes2caliptra_t
keymgr_key_i	input	keymgr_pkg::hw_key_req_t
tl_i	input	caliptra_tlul_pkg::tl_h2d_t
tl_o	output	caliptra_tlul_pkg::tl_d2h_t
alert_rx_i	input	caliptra_prim_alert_pkg::alert_rx_t [NumAlerts-1:0...
alert_tx_o	output	caliptra_prim_alert_pkg::alert_tx_t [NumAlerts-1:0...

API — aes_cipher_control.sv**Table 1431. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control.sv — module / interface / package (1개)**

이름	종류
aes_cipher_control	module

Table 1432. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control.sv — parameter (3개)

파라미터	형/폭	기본값
CiphOpFwdOnly	bit	0
SecMasking	bit	0
SecSBoxImpl	sbox_impl_e	SBoxImplDom

Table 1433. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control.sv — localparam (2개)

상수	형/폭	값
NumSp2VSIg	int unsigned	8
Sp2VEnSecBuf	bit [NumSp2VSIg-1:0]	8'b1100_0000

Table 1434. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control.sv — 포트 (45개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_i	input	sp2v_e
in_ready_o	output	sp2v_e
out_valid_o	output	sp2v_e
out_ready_i	input	sp2v_e
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_i	input	sp2v_e
crypt_o	output	sp2v_e
dec_key_gen_i	input	sp2v_e
dec_key_gen_o	output	sp2v_e
prng_reseed_i	input	logic
prng_reseed_o	output	logic
key_clear_i	input	logic
key_clear_o	output	logic
data_out_clear_i	input	logic
data_out_clear_o	output	logic
mux_sel_err_i	input	logic
sp_enc_err_i	input	logic
op_err_i	input	logic
alert_fatal_i	input	logic
alert_o	output	logic
prng_update_o	output	logic
prng_reseed_req_o	output	logic
prng_reseed_ack_i	input	logic
state_sel_o	output	state_sel_e
state_we_o	output	sp2v_e
sub_bytes_en_o	output	sp2v_e
sub_bytes_out_req_i	input	sp2v_e
sub_bytes_out_ack_o	output	sp2v_e
add_rk_sel_o	output	add_rk_sel_e
key_expand_op_o	output	ciph_op_e

포트	방향	형/폭
key_full_sel_o	output	key_full_sel_e
key_full_we_o	output	sp2v_e
key_dec_sel_o	output	key_dec_sel_e
key_dec_we_o	output	sp2v_e
key_expand_en_o	output	sp2v_e
key_expand_out_req_i	input	sp2v_e
key_expand_out_ack_o	output	sp2v_e
key_expand_clear_o	output	logic
key_expand_round_o	output	logic [3:0]
key_words_sel_o	output	key_words_sel_e
round_key_sel_o	output	round_key_sel_e

API — aes_cipher_control_fsm.sv

Table 1435. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm.sv — module / interface / package (1개)

이름	종류
aes_cipher_control_fsm	module

Table 1436. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm.sv — parameter (2개)

파라미터	형/폭	기본값
SecMasking	bit	0
SecSBoxImpl	sbox_impl_e	SBoxImplDom

Table 1437. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm.sv — 포트 (50개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_i	input	logic
in_ready_o	output	logic
out_valid_o	output	logic
out_ready_i	input	logic
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_i	input	logic
dec_key_gen_i	input	logic
prng_reseed_i	input	logic

포트	방향	형/폭
key_clear_i	input	logic
data_out_clear_i	input	logic
mux_sel_err_i	input	logic
sp_enc_err_i	input	logic
rnd_ctr_err_i	input	logic
op_err_i	input	logic
alert_fatal_i	input	logic
alert_o	output	logic
prng_update_o	output	logic
prng_reseed_req_o	output	logic
prng_reseed_ack_i	input	logic
state_sel_o	output	state_sel_e
state_we_o	output	logic
sub_bytes_en_o	output	logic
sub_bytes_out_req_i	input	logic
sub_bytes_out_ack_o	output	logic
add_rk_sel_o	output	add_rk_sel_e
key_full_sel_o	output	key_full_sel_e
key_full_we_o	output	logic
key_dec_sel_o	output	key_dec_sel_e
key_dec_we_o	output	logic
key_expand_en_o	output	logic
key_expand_out_req_i	input	logic
key_expand_out_ack_o	output	logic
key_expand_clear_o	output	logic
rnd_ctr_o	output	logic [3:0]
key_words_sel_o	output	key_words_sel_e
round_key_sel_o	output	round_key_sel_e
crypt_q_i	input	logic
crypt_d_o	output	logic
dec_key_gen_q_i	input	logic
dec_key_gen_d_o	output	logic
prng_reseed_q_i	input	logic
prng_reseed_d_o	output	logic
key_clear_q_i	input	logic

포트	방향	형/폭
key_clear_d_o	output	logic
data_out_clear_q_i	input	logic
data_out_clear_d_o	output	logic

API — aes_cipher_control_fsm_n.sv

Table 1438. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm_n.sv — module / interface / package (1개)

이름	종류
aes_cipher_control_fsm_n	module

Table 1439. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm_n.sv — parameter (2개)

파라미터	형/폭	기본값
SecMasking	bit	0
SecSBoxImpl	sbox_impl_e	SBoxImplDom

Table 1440. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm_n.sv — localparam (2개)

상수	형/폭	값
NumInBufBits	int	\$bits({
NumOutBufBits	int	\$bits({

Table 1441. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm_n.sv — 포트 (50개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_ni	input	logic
in_ready_no	output	logic
out_valid_no	output	logic
out_ready_ni	input	logic
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_ni	input	logic
dec_key_gen_ni	input	logic
prng_reseed_i	input	logic
key_clear_i	input	logic
data_out_clear_i	input	logic
mux_sel_err_i	input	logic

포트	방향	형/폭
sp_enc_err_i	input	logic
rnd_ctr_err_i	input	logic
op_err_i	input	logic
alert_fatal_i	input	logic
alert_o	output	logic
prng_update_o	output	logic
prng_reseed_req_o	output	logic
prng_reseed_ack_i	input	logic
state_sel_o	output	state_sel_e
state_we_no	output	logic
sub_bytes_en_no	output	logic
sub_bytes_out_req_ni	input	logic
sub_bytes_out_ack_no	output	logic
add_rk_sel_o	output	add_rk_sel_e
key_full_sel_o	output	key_full_sel_e
key_full_we_no	output	logic
key_dec_sel_o	output	key_dec_sel_e
key_dec_we_no	output	logic
key_expand_en_no	output	logic
key_expand_out_req_ni	input	logic
key_expand_out_ack_no	output	logic
key_expand_clear_o	output	logic
rnd_ctr_o	output	logic [3:0]
key_words_sel_o	output	key_words_sel_e
round_key_sel_o	output	round_key_sel_e
crypt_q_ni	input	logic
crypt_d_no	output	logic
dec_key_gen_q_ni	input	logic
dec_key_gen_d_no	output	logic
prng_reseed_q_i	input	logic
prng_reseed_d_o	output	logic
key_clear_q_i	input	logic
key_clear_d_o	output	logic
data_out_clear_q_i	input	logic
data_out_clear_d_o	output	logic

API — aes_cipher_control_fsm_p.sv**Table 1442. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm_p.sv — module / interface / package (1개)**

이름	종류
aes_cipher_control_fsm_p	module

Table 1443. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm_p.sv — parameter (2개)

파라미터	형/폭	기본값
SecMasking	bit	0
SecSBoxImpl	sbox_impl_e	SBoxImplDom

Table 1444. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm_p.sv — localparam (2개)

상수	형/폭	값
NumInBufBits	int	\$bits({
NumOutBufBits	int	\$bits({

Table 1445. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm_p.sv — 포트 (50개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_i	input	logic
in_ready_o	output	logic
out_valid_o	output	logic
out_ready_i	input	logic
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_i	input	logic
dec_key_gen_i	input	logic
prng_reseed_i	input	logic
key_clear_i	input	logic
data_out_clear_i	input	logic
mux_sel_err_i	input	logic
sp_enc_err_i	input	logic
rnd_ctr_err_i	input	logic
op_err_i	input	logic
alert_fatal_i	input	logic
alert_o	output	logic

포트	방향	형/폭
prng_update_o	output	logic
prng_reseed_req_o	output	logic
prng_reseed_ack_i	input	logic
state_sel_o	output	state_sel_e
state_we_o	output	logic
sub_bytes_en_o	output	logic
sub_bytes_out_req_i	input	logic
sub_bytes_out_ack_o	output	logic
add_rk_sel_o	output	add_rk_sel_e
key_full_sel_o	output	key_full_sel_e
key_full_we_o	output	logic
key_dec_sel_o	output	key_dec_sel_e
key_dec_we_o	output	logic
key_expand_en_o	output	logic
key_expand_out_req_i	input	logic
key_expand_out_ack_o	output	logic
key_expand_clear_o	output	logic
rnd_ctr_o	output	logic [3:0]
key_words_sel_o	output	key_words_sel_e
round_key_sel_o	output	round_key_sel_e
crypt_q_i	input	logic
crypt_d_o	output	logic
dec_key_gen_q_i	input	logic
dec_key_gen_d_o	output	logic
prng_reseed_q_i	input	logic
prng_reseed_d_o	output	logic
key_clear_q_i	input	logic
key_clear_d_o	output	logic
data_out_clear_q_i	input	logic
data_out_clear_d_o	output	logic

API — aes_cipher_core.sv

Table 1446. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_core.sv — module / interface / package (1개)

이름	종류
aes_cipher_core	module

Table 1447. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_core.sv — parameter (9개)

파라미터	형/폭	기본값
AES192Enable	bit	1
CiphOpFwdOnly	bit	0
SecMasking	bit	1
SecSBoxImpl	sbox_impl_e	SBoxImplDom
SecAllowForcingMasks	bit	0
SecSkipPRNGReseeding	bit	0
EntropyWidth	int unsigned	edn_pkg::ENDPOINT_BUS_WIDTH
RndCnstMaskingLfsrSeed	masking_lfsr_seed_t	RndCnstMaskingLfsrSeedDefault
RndCnstMaskingLfsrPerm	masking_lfsr_perm_t	RndCnstMaskingLfsrPermDefault

Table 1448. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_core.sv — localparam (5개)

상수	형/폭	값
NumShares	int	SecMasking ? 2 : 1
WidthPRDRow	int unsigned	4*WidthPRDSBox
NumSp2VSig	int unsigned	3
Sp2VEnSecBuf	bit [NumSp2VSig-1:0]	{NumSp2VSig{1'b1}}
NumCyclesPerRound	int unsigned	(SecSBoxImpl == SBoxImplDom) ? 5 : 1

Table 1449. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_core.sv — 포트 (26개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_i	input	sp2v_e
in_ready_o	output	sp2v_e
out_valid_o	output	sp2v_e
out_ready_i	input	sp2v_e
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_i	input	sp2v_e
crypt_o	output	sp2v_e
dec_key_gen_i	input	sp2v_e

포트	방향	형/폭
dec_key_gen_o	output	sp2v_e
prng_reseed_i	input	logic
prng_reseed_o	output	logic
key_clear_i	input	logic
key_clear_o	output	logic
data_out_clear_i	input	logic
data_out_clear_o	output	logic
alert_fatal_i	input	logic
alert_o	output	logic
force_masks_i	input	logic
data_in_mask_o	output	logic [3:0][3:0][7:0]
entropy_req_o	output	logic
entropy_ack_i	input	logic
entropy_i	input	logic [EntropyWidth-1:0]

API — aes_clp_reg.sv

Table 1450. chipsalliance_caliptra-rtl/src/aes/rtl/aes_clp_reg.sv — module / interface / package (1개)

이름	종류
aes_clp_reg	module

Table 1451. chipsalliance_caliptra-rtl/src/aes/rtl/aes_clp_reg.sv — 포트 (16개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_cpuif_req	input	wire
s_cpuif_req_is_wr	input	wire
s_cpuif_addr	input	wire [10:0]
s_cpuif_wr_data	input	wire [31:0]
s_cpuif_wr_biten	input	wire [31:0]
s_cpuif_req_stall_wr	output	wire
s_cpuif_req_stall_rd	output	wire
s_cpuif_rd_ack	output	wire
s_cpuif_rd_err	output	wire
s_cpuif_rd_data	output	wire [31:0]
s_cpuif_wr_ack	output	wire
s_cpuif_wr_err	output	wire

포트	방향	형/폭
hwif_in	input	aes_clp_reg_pkg::aes_clp_reg__in_t
hwif_out	output	aes_clp_reg_pkg::aes_clp_reg__out_t

1. chipsalliance_caliptra-rtl/src/aes/rtl/aes.sv (486 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES top-level wrapper
6
7 `include "caliptra_prim_assert.sv"
8
9 module aes
10     import aes_pkg::*;
11     import aes_reg_pkg::*;
12     #(
13         parameter bit AES192Enable = 1, // Can be
0 (disable), or 1 (enable).
14         parameter bit AESGCMEnable = 1, // Can be
0 (disable), or 1 (enable).
15         parameter bit SecMasking = 1, // Can be
0 (no masking), or
16         // 1
17         (first-order masking) of the cipher // core.
18         Masking requires the use of a // masked
19         S-Box, see SecSBoxImpl parameter.
20         parameter sbox_impl_e SecSBoxImpl = SBoxImpl-
21         Dom, // See aes_pkg.sv
22         parameter int unsigned SecStartTriggerDelay = 0, // Manual
23         start trigger delay, useful for
24         measurements. A value of e.g. 40
25         // allows
26         the processor to go into sleep // before
27         AES starts operation.
28         parameter bit SecAllowForcingMasks = 0, // Allow
29         forcing masks to constant values using
30         //
31         FORCE_MASKS bit in Auxiliary Control
32         // Re-
33         gister. Useful for SCA only.
34         parameter bit SecSkipPRNGReseeding = 0, // The
35         current SCA setup doesn't provide enough
36         // re-
37         sources to implement the infrastucture
38         // re-
39         quired for PRNG reseeding (CSRNG, EDN).
40         // To en-
41         able SCA resistance evaluations, we
42
43         // need to
44         skip reseeding requests.
45         // Useful
46         for SCA only.
47         parameter logic [NumAlerts-1:0] AlertAsyncOn = {Nu-
48         mAlerts{1'b1}},
49         parameter clearing_lfsr_seed_t RndCnstClearingLfsrSeed =
50         RndCnstClearingLfsrSeedDefault,
51         parameter clearing_lfsr_perm_t RndCnstClearingLfsrPerm =
52         RndCnstClearingLfsrPermDefault,
53         parameter clearing_lfsr_perm_t RndCnstClearingSharePerm =
54         RndCnstClearingSharePermDefault,
55         parameter masking_lfsr_seed_t RndCnstMaskingLfsrSeed =
56         RndCnstMaskingLfsrSeedDefault,
57         parameter masking_lfsr_perm_t RndCnstMaskingLfsrPerm =
58         RndCnstMaskingLfsrPermDefault
59     ) (
60         input logic clk_i,
61         input logic rst_ni,
62         input logic rst_shad-
63         owed_ni,
64         // Idle indicator for clock manager
65         output caliptra_prim_mubi_pkg::mubi4_t
66         idle_o,
67         // Life cycle
68         input lc_ctrl_pkg::lc_tx_t lc_escal-
69         ate_en_i,
70         // Entropy distribution network (EDN) interface
71         input logic clk_edn_i,
72         input logic rst_edn_ni,
73         output edn_pkg::edn_req_t edn_o,
74         input edn_pkg::edn_rsp_t edn_i,
75

```

```

56 // status signals
57 output logic input_ready_o,
58 output logic output_valid_o,
59
60 // Caliptra interface
61 input caliptra2aes_t caliptra2aes,
62 output aes2caliptra_t aes2caliptra,
63
64 // Key manager (keymgr) key sideload interface
65 input keymgr_pkg::hw_key_req_t keymgr_key_i,
66 // Bus interface
67 input caliptra_tlul_pkg::tl_h2d_t tl_i,
68 output caliptra_tlul_pkg::tl_d2h_t tl_o,
69
70 // Alerts
71 input caliptra_prim_alert_pkg::alert_rx_t [NumAlerts-1:0] alert_rx_i,
72 output caliptra_prim_alert_pkg::alert_tx_t [NumAlerts-1:0] alert_tx_o,
73
74 // Localparam int unsigned EntropyWidth = edn_pkg::END-
75 POINT_BUS_WIDTH;
76
77 // Signals
78 aes_reg2hw_t reg2hw;
79 aes_hw2reg_t hw2reg;
80
81 logic [NumAlerts-1:0] alert;
82 lc_ctrl_pkg::lc_tx_t lc_escalate_en;
83
84 logic edn_req_int;
85 logic edn_req_hold_d, edn_req_hold_q;
86 logic edn_req;
87 logic edn_ack;
88 logic [EntropyWidth-1:0] edn_data;
89 logic unused_edn_fips;
90 logic entropy_clearing_req, en-
91 trophy_masking_req;
92 trophy_masking_ack;
93
94 // Inputs //
95 //////////////////////////////////////////////////
96 // SEC_CM: BUS INTEGRITY
97 // SEC_CM: AUX.CONFIG.SHADOW
98 // SEC_CM: AUX.CONFIG.REGWEN
99 // SEC_CM: KEY.SW.UNREADABLE
100 // SEC_CM: DATA.REG.SW.UNREADABLE
101 // Register interface
102 logic intg_err_alert;
103 logic shadowed_storage_err, shadowed_update_err;
104 aes_reg_top u_reg (
105     .clk_i,
106     .rst_ni,
107     .rst_shadowed_ni,
108     .tl_i,
109     .tl_o,
110     .input_ready_o,
111     .output_valid_o,
112     .reg2hw,
113     .hw2reg,
114     .shadowed_storage_err_o(shadowed_storage_err),
115     .shadowed_update_err_o(shadowed_update_err),
116     .intg_err_o(intg_err_alert)
117 );
118
119 // SEC_CM: LC_ESCALATE.EN.INTERSIG.MUBI
120 // Synchronize life cycle input
121 caliptra_prim_lc_sync #(
122     .NumCopies(1)
123 ) u_caliptra_prim_lc_sync (
124     .clk_i,
125     .rst_ni,
126     .lc_en_i ( lc_escalate_en_i ),
127     .lc_en_o ( {lc_escalate_en} )
128 );
129
130 //////////////////////////////////////////////////
131 // Caliptra Control Intercept //
132 //////////////////////////////////////////////////
133 localparam CLP_AES_KV_CHUNK_SIZE = NumRegsData*32; // Size
134 of data that can be stored at once, atomically, at end of AES op
135 genvar kv_ii;

```

```

138
139 aes_hw2reg_t hw2reg_caliptra;
140 aes_reg2hw_t reg2hw_caliptra;
141 logic aes2caliptra_kv_data_out_valid;
142 logic [CLP_AES_KV_WR_DW-1:0] aes2caliptra_kv_data_out;
143 logic kv_data_intercept;
144 logic kv_data_intercept_end;
145 logic [$clog2(CLP_AES_KV_WR_DW/CLP_AES_KV_CHUNK_SIZE)-1:0]
kv_data_counter; // This will peg at the key_size value if decrypted
plaintext is larger than CLP_AES_KV_WR_DW
146 logic [$clog2(CLP_AES_KV_WR_DW/CLP_AES_KV_CHUNK_SIZE)-1:0]
kv_data_thresh; // Comparison value for kv_data_counter to saturate,
i.e. the key_size normalized to data chunk size
147 logic incr_kv_data_counter;
148 logic hw2reg_data_out_mask_en;
149 logic [$bits(aes_hw2reg_data_out_mreg_t)-1:0]
hw2reg_data_out_mask;
150 logic output_blocked_de;
151 logic output_valid_r;
152
153 // Mask to conceal data_out from reg API (when dest is KV)
154 assign hw2reg_data_out_mask = {$bits(aes_h-
w2reg_data_out_mreg_t){hw2reg_data_out_mask_en}};
155
156 // Enable signal to drive an update to the
status.output_lost register field
157 always_comb output_blocked_de =
hw2reg_caliptra.status.output_valid.de &&
158 hw2reg_caliptra.status.output_valid.d &&
159 calipt-
raaes.block_reg_output;
160
161 always_comb begin
162 // Passthrough
163 hw2reg
=
164 // Augmented
165 // For OCP LOCK use-cases, add a condition to set out-
put_lost if output data is required to go to KV.
166 // Output Lost is usually only used for Manual mode.
TODO if we support manual mode with OCP LOCK flows (and KV WR) this
may
167 // need to be updated.
168 hw2reg.status.output_lost.d
= out-
put_blocked_de ? 1'b1 :
169
170 hw2reg_caliptra.status.output_lost.d;
171 hw2reg.status.output_lost.de
=
172 hw2reg_caliptra.status.output_lost.de ||
173 out-
put_blocked_de;
174 // Concealed
175 foreach (hw2reg.data_out[idx]) begin
176 hw2reg.data_out[idx].d
=
177 hw2reg_caliptra.data_out[idx].d & hw2reg_data_out_mask;
178 end
179 always_comb begin
180 // Passthrough
181 reg2hw_caliptra
= reg2hw;
182 // RE intercept
183 foreach (reg2hw.data_out[idx]) begin
184 reg2hw_caliptra.data_out[idx].q
=
185 reg2hw.data_out[idx].q ;
186 reg2hw_caliptra.data_out[idx].re
= kv_data_intercept ?
output_valid_r :
187 reg2hw.data_out[idx].re;
188 end
189 // Flag to detect when data out shall be routed to Caliptra
KeyVault
190 always_ff @(posedge clk_i or negedge rst_ni) begin:
kv_data_intercept_reg
191 if (!rst_ni) begin
192 kv_data_intercept
<= 1'b0;
193 hw2reg_data_out_mask_en
<= 1'b1;
194 end
195 // FW must arm the KV write prior to starting AES opera-
tion
196 else if ((kv_data_counter == 0) &&
reg2hw_caliptra.data_in[0].qe) begin
197 kv_data_intercept
<= caliptra2aes.kv_en;
198 hw2reg_data_out_mask_en
<= ~caliptra2aes.kv_en &&
~caliptra2aes.block_reg_output; // This signal winds up being ef-
fectively (~kv_data_intercept && ~caliptra2aes.block_reg_output)
199 end
200 // TODO support for Manual operation mode with trig-
ger.start.q?
201 else if (kv_data_intercept_end) begin
202 kv_data_intercept
<= 1'b0;
203 hw2reg_data_out_mask_en
<= 1'b1;
204 end
205
206 // NOTE if key size is not an integer multiple of
CLP_AES_KV_CHUNK_SIZE, this is a rounded down value
207 always_comb kv_data_thresh = ($clog2(CLP_AES_KV_WR_DW/

```

```

CLP_AES_KV_CHUNK_SIZE))'(caliptra2aes.key_release_key_size/
(CLP_AES_KV_CHUNK_SIZE/8)-1);
208 // NOTE: This assumes that output_valid will always assert
prior to entering idle state, which should be true.
209 // If this doesn't hold, then kv_data_intercept will
deassert before the final data beat is captured, and
210 // the KV write won't be issued
211 always_comb kv_data_intercept_end = hw2reg.status.idle.de &&
hw2reg.status.idle.d && !reg2hw_caliptra.status.idle.q &&
(kv_data_counter == kv_data_thresh);
212
213 // Latch when data_out is valid, used to generate read-en-
able and signal data capture
214 always_ff @(posedge clk_i or negedge rst_ni) begin: out-
put_valid_dd_reg
215 if (!rst_ni) begin
216 output_valid_r
<= 1'b0;
217 end
218 else if (hw2reg_caliptra.status.output_valid.de) begin
219 output_valid_r
<=
hw2reg_caliptra.status.output_valid.d;
220 end
221 end
222
223 // Index into the KV output data based on number of AES
rounds observed
224 always_comb incr_kv_data_counter = kv_data_intercept && out-
put_valid_r;
225 always_ff @(posedge clk_i or negedge rst_ni) begin: aes2c-
aliptra_kv_data_counter_reg
226 if (!rst_ni) begin
227 kv_data_counter
<= '0;
228 end
229 else if (!kv_data_intercept) begin
230 kv_data_counter
<= '0;
231 end
232 else if (incr_kv_data_counter && (kv_data_counter ==
kv_data_thresh)) begin
233 kv_data_counter
<= kv_data_counter;
234 end
235 else if (incr_kv_data_counter) begin
236 kv_data_counter
<= kv_data_counter + 1;
237 end
238 end
239
240 // Signal data is valid for KV write client once full AES
operation is done
241 always_ff @(posedge clk_i or negedge rst_ni) begin: aes2c-
aliptra_kv_data_valid_reg
242 if (!rst_ni) begin
243 aes2caliptra_kv_data_out_valid
<= 1'b0;
244 end
245 else if (caliptra2aes.clear_secrets ||
reg2hw_caliptra.trigger.data_out_clear.q) begin
246 aes2caliptra_kv_data_out_valid
<= 1'b0;
247 end
248 else if (kv_data_intercept_end) begin
249 aes2caliptra_kv_data_out_valid
<= 1'b1;
250 end
251 else if (caliptra2aes.kv_write_done) begin
252 aes2caliptra_kv_data_out_valid
<= 1'b0;
253 end
254 end
255 assign aes2caliptra.kv_data_out_valid = aes2calipt-
ra_kv_data_out_valid;
256 assign aes2caliptra.kv_key_in_use =
hw2reg.ctrl_shadowed.sideload.d;
257 assign aes2caliptra.key_len =
key_len_e'(hw2reg.ctrl_shadowed.operation.d);
258 // TODO: Qualify this with anything?
259 // Probably not needed. Timing of the kv write request
is tightly controlled, and that's the only
260 // place this signal is used.
261 always_comb aes2caliptra.aes_operation_is_ecb_decrypt =
( aes_op_e'(hw2reg.ctrl_shadowed.operation.d) == AES_DEC) &&
262 (aes_mode_e'(hw2reg.ctrl_shadowed.mode.d) == AES_ECB);
263
264 // Capture data_out until operation is done
265 generate
266 for (kv_ii=0; kv_ii < CLP_AES_KV_WR_DW/
CLP_AES_KV_CHUNK_SIZE; kv_ii++) begin
267 always_ff @(posedge clk_i or negedge rst_ni) begin:
aes2caliptra_kv_data_reg
268 if (!rst_ni) begin
269 aes2calipt-
ra_kv_data_out[(kv_ii)*CLP_AES_KV_CHUNK_SIZE+:CLP_AES_KV_CHUNK_SIZE]
<= CLP_AES_KV_CHUNK_SIZE'(0);
270 end
271 else if (caliptra2aes.clear_secrets ||
reg2hw_caliptra.trigger.data_out_clear.q) begin
272 aes2calipt-
ra_kv_data_out[(kv_ii)*CLP_AES_KV_CHUNK_SIZE+:CLP_AES_KV_CHUNK_SIZE]
<= CLP_AES_KV_CHUNK_SIZE'(0);
273 end
274 else if (incr_kv_data_counter &&
(kv_data_counter == kv_ii)) begin
275 aes2calipt-
ra_kv_data_out[(kv_ii)*CLP_AES_KV_CHUNK_SIZE+:CLP_AES_KV_CHUNK_SIZE]

```



```

<= {hw2reg_caliptra.data_out[3].d,
276                                     hw2reg_caliptra.data_out[2].d,
277                                     hw2reg_caliptra.data_out[1].d,
278                                     hw2reg_caliptra.data_out[0].d};
279     end
280     else if (caliptra2aes.kv_write_done) begin
281         aes2calipt-
ra_kv_data_out[(kv_ii)*CLP_AES_KV_CHUNK_SIZE+:CLP_AES_KV_CHUNK_SIZE]
<= CLP_AES_KV_CHUNK_SIZE'(0);
282     end
283 end
284 end
285 endgenerate
286 assign aes2caliptra.kv_data_out = aes2caliptra_kv_data_out;
287
288 //////////////////////////////////////////////////
289 // EDN Interface //
290 //////////////////////////////////////////////////
291
292 // Internally, we have up to two PRNGs that share the EDN
interface for reseeding. Here, we just
293 // arbitrate the requests. Upsizing of the entropy to the
correct width is performed inside the
294 // PRNGs.
295 // Reseed operations for the clearing PRNG are initiated by
software. Reseed operations for the
296 // masking PRNG can also be automatically initiated.
297 assign edn_req_int = entropy_clearing_req | en-
tropy_masking_req;
298 // Only forward ACK to PRNG currently requesting entropy.
Give higher priority to clearing PRNG.
299 assign entropy_clearing_ack = entropy_clearing_req &
edn_ack;
300 assign entropy_masking_ack = ~entropy_clearing_req & en-
tropy_masking_req & edn_ack;
301
302 // Upon escalation or detection of a fatal alert, an EDN re-
quest signal can be dropped before
303 // getting acknowledged. This is okay with respect to AES as
the module will need to be reset
304 // anyway. However, to not leave EDN in a strange state, we
hold the request until it's actually
305 // acknowledged.
306 assign edn_req = edn_req_int | edn_req_hold_q;
307 assign edn_req_hold_d = (edn_req_hold_q | edn_req) &
~edn_ack;
308 always_ff @(posedge clk_i or negedge rst_ni) begin :
edn_req_reg
309     if (!rst_ni) begin
310         edn_req_hold_q <= '0;
311     end else begin
312         edn_req_hold_q <= edn_req_hold_d;
313     end
314 end
315
316 // Synchronize EDN interface
caliptra_prim_sync_reqack_data #(
317     .Width(EntropyWidth),
318     .DataSrcDst(1'b0),
319     .DataReg(1'b0)
320 ) u_caliptra_prim_sync_reqack_data (
321     .clk_src_i ( clk_i ),
322     .rst_src_ni ( rst_ni ),
323     .clk_dst_i ( clk_edn_i ),
324     .rst_dst_ni ( rst_edn_ni ),
325     .req_chk_i ( 1'b1 ),
326     .src_req_i ( edn_req ),
327     .src_ack_o ( edn_ack ),
328     .dst_req_o ( edn_o.edn_req ),
329     .dst_ack_i ( edn_i.edn_ack ),
330     .data_i ( edn_i.edn_bus ),
331     .data_o ( edn_data )
332 );
333
334 // We don't track whether the entropy is pre-FIPS or not in-
side AES.
335 assign unused_edn_fips = edn_i.edn_fips;
336
337 //////////////////////////////////////////////////
338 // Core //
339 //////////////////////////////////////////////////
340
341 // AES core
342 aes_core #(
343     .AES192Enable ( AES192Enable ),
344     .AESGCMEnable ( AESGCMEnable ),
345     .SecMasking ( SecMasking ),
346     .SecSBoxImpl ( SecSBoxImpl ),
347     .SecStartTriggerDelay ( SecStartTriggerDelay ),
348     .SecAllowForcingMasks ( SecAllowForcingMasks ),
349     .SecSkipPRNGReseeding ( SecSkipPRNGReseeding ),
350     .EntropyWidth ( EntropyWidth ),
351     .RndCnstClearingLfsrSeed ( RndCnstClearingLfsrSeed ),
352     .RndCnstClearingLfsrPerm ( RndCnstClearingLfsrPerm ),
353     .RndCnstClearingSharePerm ( RndCnstClearingSharePerm ),

```

```

354     .RndCnstMaskingLfsrSeed ( RndCnstMaskingLfsrSeed ),
355     .RndCnstMaskingLfsrPerm ( RndCnstMaskingLfsrPerm )
356 ) u_aes_core (
357     .clk_i ( clk_i ),
358     .rst_ni ( rst_ni ),
359     .rst_shadowed_ni ( rst_shadowed_ni ),
360     .entropy_clearing_req_o ( entropy_clearing_req ),
361     .entropy_clearing_ack_i ( entropy_clearing_ack ),
362     .entropy_clearing_i ( edn_data ),
363     .entropy_masking_req_o ( entropy_masking_req ),
364     .entropy_masking_ack_i ( entropy_masking_ack ),
365     .entropy_masking_i ( edn_data ),
366
367     .keymgr_key_i ( keymgr_key_i ),
368
369     .lc_escalate_en_i ( lc_escalate_en ),
370
371     .shadowed_storage_err_i ( shadowed_storage_err ),
372     .shadowed_update_err_i ( shadowed_update_err ),
373     .intg_err_alert_i ( intg_err_alert ),
374     .alert_recov_o ( alert[0] ),
375     .alert_fatal_o ( alert[1] ),
376
377     .reg2hw ( reg2hw_caliptra ),
378     .hw2reg ( hw2reg_caliptra )
379 );
380
381 assign idle_o = calipt-
ra_prim_mubi_pkg::mubi4_bool_to_mubi(reg2hw.status.idle.q);
382
383 //////////////////////////////////////////////////
384 // Alerts //
385 //////////////////////////////////////////////////
386
387 logic [NumAlerts-1:0] alert_test;
388 assign alert_test = {
389     reg2hw.alert_test.fatal_fault.q &
390     reg2hw.alert_test.fatal_fault.qe,
391     reg2hw.alert_test.recov_ctrl_update_err.q &
392     reg2hw.alert_test.recov_ctrl_update_err.qe
393 };
394
395 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
396     caliptra_prim_alert_sender #(
397         .AsyncOn(AlertAsyncOn[i]),
398         .IsFatal(i)
399     ) u_caliptra_prim_alert_sender (
400         .clk_i,
401         .rst_ni,
402         .alert_test_i ( alert_test[i] ),
403         .alert_req_i ( alert[i] ),
404         .alert_ack_o ( ),
405         .alert_state_o ( ),
406         .alert_rx_i ( alert_rx_i[i] ),
407         .alert_tx_o ( alert_tx_o[i] )
408     );
409 end
410
411 //////////////////////////////////////////////////
412 // Assertions //
413 //////////////////////////////////////////////////
414
415 // All outputs should have a known value after reset
416 `CALIPTRA_ASSERT_KNOWN(Tl0DValidKnown, tl_o.d_valid)
417 `CALIPTRA_ASSERT_KNOWN(Tl0AReadyKnown, tl_o.a_ready)
418 `CALIPTRA_ASSERT_KNOWN(IdleKnown, idle_o)
419 `CALIPTRA_ASSERT_KNOWN(EdnReqKnown, edn_o)
420 `CALIPTRA_ASSERT_KNOWN(AlertTxKnown, alert_tx_o)
421
422 // Alert assertions for sparse FSMs.
423 for (genvar i = 0; i < Sp2VWidth; i++) begin : gen_con-
trol_fsm_svas
424     if (SP2V_LOGIC_HIGH[i] == 1'b1) begin : gen_con-
trol_fsm_svas_p
425         `CALIPT-
RA_ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesControlFsmCheck_A,
426             u_aes_core.u_aes_control.gen_fsm[i].gen_fsm.p,
427             u_aes_con-
trol_fsm_i.u_aes_control_fsm.u_state_regs,
428             alert_tx_o[1])
429     end else begin : gen_control_fsm_svas_n
430         `CALIPT-
RA_ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCtrFsmCheck_A,
431             u_aes_core.u_aes_ctr.gen_fsm[i].gen_fsm.p,
432             u_aes_ctr_fsm_i.u_aes_ctr_fsm.u_state_regs,
433             alert_tx_o[1])
434     end
435 end
436
437 for (genvar i = 0; i < Sp2VWidth; i++) begin :
gen_ctr_fsm_svas
438     if (SP2V_LOGIC_HIGH[i] == 1'b1) begin : gen_ctr_fsm_svas_p
439         `CALIPT-
RA_ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCtrFsmCheck_A,
440             u_aes_core.u_aes_ctr.gen_fsm[i].gen_fsm.p,
441             u_aes_ctr_fsm_i.u_aes_ctr_fsm.u_state_regs,
442             alert_tx_o[1])
443     end else begin : gen_ctr_fsm_svas_n
444         `CALIPT-

```

```

RA_ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCtrFsmCheck_A,
446     u_aes_core.u_aes_ctr.gen_fsm[i].gen_fsm_n.
447     u_aes_ctr_fsm_i.u_aes_ctr_fsm.u_state_regs,
448     alert_tx_o[1])
449 end
450 end
451
452 for (genvar i = 0; i < Sp2VWidth; i++) begin :
gen_cipher_control_fsm_svas
453     if (SP2V_LOGIC_HIGH[i] == 1'b1) begin :
gen_cipher_control_fsm_svas_p
454         `CALIPT-
RA_ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCipherControlFsmCheck_A,
455
456     u_aes_core.u_aes_cipher_core.u_aes_cipher_control.gen_fsm[i].gen_fsm_p.
457     u_aes_cipher_control_fsm_i.u_aes_cipher_control_fsm.u_state_regs,
458     alert_tx_o[1])
459     end else begin : gen_cipher_control_fsm_svas_n
460         `CALIPT-
RA_ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCipherControlFsmCheck_A,
461
462     u_aes_core.u_aes_cipher_core.u_aes_cipher_control.gen_fsm[i].gen_fsm_n.
463     u_aes_cipher_control_fsm_i.u_aes_cipher_control_fsm.u_state_regs,
464     alert_tx_o[1])
465     end
466     if (AESGCMEnable) begin : gen_ghash_fsm_sva
467         `CALIPT-
RA_ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesGhashFsmCheck_A,
468     u_aes_core.gen_ghash.u_aes_ghash.u_state_regs,
469     alert_tx_o[1])
470     end
471
472     if (AESGCMEnable && SecMasking) begin : gen_ghash_onehot_sva
473         for (genvar s = 0; s < 2; s++) begin :
gen_ghash_onehot_add_in_sva
474             `CALIPT-
RA_ASSERT_PRIM_ONEHOT_ERROR_TRIGGER_ALERT(GhashAadOnehotCheck_A,
475
476     u_aes_core.gen_ghash.u_aes_ghash.gen_masked_add.gen_add_in_muxes[s].
477     u_caliptra_prim_onehot_check_add_in_sel,
478     alert_tx_o[1])
479     end
480     `CALIPT-
RA_ASSERT_PRIM_ONEHOT_ERROR_TRIGGER_ALERT(GhashMultOnehotCheck_A,
481
482     u_aes_core.gen_ghash.u_aes_ghash.gen_gf_mult1_mux.u_caliptra_prim_o
483     nehot_check_gf_mult1_in_sel,
484     alert_tx_o[1])
485     end
486     // Alert assertions for reg_we onehot check
487     `CALIPT-
RA_ASSERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
488     u_reg, alert_tx_o[1])
489 endmodule

```

2. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control.sv (485 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core control
6 //
7 // This module controls the AES cipher core including the key
  expand module.
8
9 `include "caliptra_prim_assert.sv"
10
11 module aes_cipher_control import aes_pkg::*;
12 #(
13     parameter bit CiphOpFwdOnly = 0,
14     parameter bit SecMasking = 0,
15     parameter sbox_impl_e SecSBoxImpl = SBoxImplDom
16 ) (
17     input logic clk_i,
18     input logic rst_ni,
19
20     // Input handshake signals
21     input sp2v_e in_valid_i,
22     output sp2v_e in_ready_o,
23
24     // Output handshake signals
25     output sp2v_e out_valid_o,
26     input sp2v_e out_ready_i,
27
28     // Control and sync signals
29     input logic cfg_valid_i,
30     input ciph_op_e op_i,
31     input key_len_e key_len_i,
32     input sp2v_e crypt_i,

```

```

33     output sp2v_e crypt_o,
34     input sp2v_e dec_key_gen_i,
35     output sp2v_e dec_key_gen_o,
36     input logic prng_reseed_i,
37     output logic prng_reseed_o,
38     input logic key_clear_i,
39     output logic key_clear_o,
40     input logic data_out_clear_i,
41     output logic data_out_clear_o,
42     input logic mux_sel_err_i,
43     input logic sp_enc_err_i,
44     input logic op_err_i,
45     input logic alert_fatal_i,
46     output logic alert_o,
47
48     // Control signals for masking PRNG
49     output logic prng_update_o,
50     output logic prng_reseed_req_o,
51     input logic prng_reseed_ack_i,
52
53     // Control and sync signals for cipher data path
54     output state_sel_e state_sel_o,
55     output sp2v_e state_we_o,
56     output sp2v_e sub_bytes_en_o,
57     input sp2v_e sub_bytes_out_req_i,
58     output sp2v_e sub_bytes_out_ack_o,
59     output add_rk_sel_e add_rk_sel_o,
60
61     // Control and sync signals for key expand data path
62     output ciph_op_e key_expand_op_o,
63     output key_full_sel_e key_full_sel_o,
64     output sp2v_e key_full_we_o,
65     output key_dec_sel_e key_dec_sel_o,
66     output sp2v_e key_dec_we_o,
67     output sp2v_e key_expand_en_o,
68     input sp2v_e key_expand_out_req_i,
69     output sp2v_e key_expand_out_ack_o,
70     output logic key_expand_clear_o,
71     output logic [3:0] key_expand_round_o,
72     output key_words_sel_e key_words_sel_o,
73     output round_key_sel_e round_key_sel_o,
74 );
75
76 // Signals
77 logic [3:0] rnd_ctr;
78 sp2v_e crypt_d, crypt_q;
79 sp2v_e dec_key_gen_d,
80 dec_key_gen_q;
81 logic prng_reseed_d, prng_reseed_q;
82 logic key_clear_d,
83 key_clear_q;
84 logic data_out_clear_d,
85 data_out_clear_q;
86 sp2v_e sub_bytes_out_req;
87 sp2v_e key_expand_out_req;
88 sp2v_e in_valid;
89 sp2v_e out_ready;
90 sp2v_e crypt;
91 sp2v_e dec_key_gen;
92 sp2v_e mux_sel_err;
93 sp2v_e mr_err;
94 sp2v_e sp_enc_err;
95 sp2v_e rnd_ctr_err;
96
97 // Sparsified FSM signals. These are needed for connecting
  the individual bits of the Sp2V
  // signals to the single-rail FSMs.
98 logic [Sp2VWidth-1:0] sp_in_valid;
99 logic [Sp2VWidth-1:0] sp_in_ready;
100 logic [Sp2VWidth-1:0] sp_out_valid;
101 logic [Sp2VWidth-1:0] sp_out_ready;
102 logic [Sp2VWidth-1:0] sp_crypt;
103 logic [Sp2VWidth-1:0] sp_dec_key_gen;
104 logic [Sp2VWidth-1:0] sp_state_we;
105 logic [Sp2VWidth-1:0] sp_sub_bytes_en;
106 logic [Sp2VWidth-1:0] sp_sub_bytes_out_req;
107 logic [Sp2VWidth-1:0] sp_sub_bytes_out_ack;
108 logic [Sp2VWidth-1:0] sp_key_full_we;
109 logic [Sp2VWidth-1:0] sp_key_dec_we;
110 logic [Sp2VWidth-1:0] sp_key_expand_en;
111 logic [Sp2VWidth-1:0] sp_key_expand_out_req;
112 logic [Sp2VWidth-1:0] sp_key_expand_out_ack;
113 logic [Sp2VWidth-1:0] sp_crypt_d;
114 logic [Sp2VWidth-1:0] sp_crypt_q;
115 logic [Sp2VWidth-1:0] sp_dec_key_gen_d;
116 logic [Sp2VWidth-1:0] sp_dec_key_gen_q;
117
118 // Multi-rail signals. These are outputs of the single-rail
  FSMs and need combining.
119 logic [Sp2VWidth-1:0] mr_alert;
120 logic [Sp2VWidth-1:0] mr_prng_update;
121 logic [Sp2VWidth-1:0] mr_prng_reseed_req;
122 logic [Sp2VWidth-1:0] mr_key_expand_clear;
123 logic [Sp2VWidth-1:0] mr_prng_reseed_d;
124 logic [Sp2VWidth-1:0] mr_key_clear_d;
125 logic [Sp2VWidth-1:0] mr_data_out_clear_d;
126
127 state_sel_e [Sp2VWidth-1:0] mr_state_sel;
128 add_rk_sel_e [Sp2VWidth-1:0] mr_add_rk_sel;
129 key_full_sel_e [Sp2VWidth-1:0] mr_key_full_sel;

```

```

128 key_dec_sel_e [Sp2VWidth-1:0] mr_key_dec_sel;
129 key_words_sel_e [Sp2VWidth-1:0] mr_key_words_sel;
130 round_key_sel_e [Sp2VWidth-1:0] mr_round_key_sel;
131
132 logic [Sp2VWidth-1:0][3:0] mr_rnd_ctr;
133
134 // FSM //
135 // FSM //
136 // FSM //
137
138 // Convert sp2v_e signals to sparsified inputs.
139 assign sp_in_valid = {in_valid};
140 assign sp_out_ready = {out_ready};
141 assign sp_crypt = {crypt};
142 assign sp_dec_key_gen = {dec_key_gen};
143 assign sp_sub_bytes_out_req = {sub_bytes_out_req};
144 assign sp_key_expand_out_req = {key_expand_out_req};
145 assign sp_crypt_q = {crypt_q};
146 assign sp_dec_key_gen_q = {dec_key_gen_q};
147
148 // SEC_CM: CIPHER.FSM.REDUN
149 // SEC_CM: CIPHER.CTR.REDUN
150 // For every bit in the Sp2V signals, one separate rail is
151 // of every rail are buffered to prevent aggressive synthesis
152 // optimizations.
153 for (genvar i = 0; i < Sp2VWidth; i++) begin : gen_fsm
154   if (SP2V_LOGIC_HIGH[i] == 1'b1) begin : gen_fsm_p
155     aes_cipher_control_fsm_p #(
156       .SecMasking ( SecMasking ),
157       .SecSBoxImpl ( SecSBoxImpl ),
158       .u_aes_cipher_control_fsm_i (
159         .clk_i ( clk_i ),
160         .rst_ni ( rst_ni ),
161         .in_valid_i ( sp_in_val-
162           id[i] ), // Sparsified
163         .in_ready_o ( sp_in_ready[i] ), // Sparsified
164         .out_valid_o ( sp_out_val-
165           id[i] ), // Sparsified
166         .out_ready_i ( sp_out_ready[i] ), // Sparsified
167         .cfg_valid_i ( cfg_valid_i ),
168         .op_i ( op_i ),
169         .key_len_i ( key_len_i ),
170         .crypt_i ( sp_crypt[i] ), // Sparsified
171         .dec_key_gen_i ( sp_dec_key_gen[i] ), // Sparsified
172         .prng_reseed_i ( prng_reseed_i ),
173         .key_clear_i ( key_clear_i ),
174         .data_out_clear_i ( data_out_clear_i ),
175         .mux_sel_err_i ( mux_sel_err ),
176         .sp_enc_err_i ( sp_enc_err ),
177         .rnd_ctr_err_i ( rnd_ctr_err ),
178         .op_err_i ( op_err_i ),
179         .alert_fatal_i ( alert_fatal_i ),
180         .alert_o ( mr_alert[i] ), // OR-combine
181         .prng_update_o ( mr_prng_update[i] ), // OR-combine
182         .prng_reseed_req_o ( mr_prng_reseed_req[i] ), // OR-combine
183         .prng_reseed_ack_i ( prng_reseed_ack_i ),
184         .state_sel_o ( mr_state_sel[i] ), // OR-combine
185         .state_we_o ( sp_state_we[i] ), // Sparsified
186         .sub_bytes_en_o ( sp_sub_bytes_en[i] ), // Sparsified
187         .sub_bytes_out_req_i ( sp_sub_bytes_out_req[i] ), // Sparsified
188         .sub_bytes_out_ack_o ( sp_sub_bytes_out_ack[i] ), // Sparsified
189         .add_rk_sel_o ( mr_add_rk_sel[i] ), // OR-combine
190         .key_full_sel_o ( mr_key_full_sel[i] ), // OR-combine
191         .key_full_we_o ( sp_key_full_we[i] ), // Sparsified
192         .key_dec_sel_o ( mr_key_dec_sel[i] ), // OR-combine
193         .key_dec_we_o ( sp_key_dec_we[i] ), // Sparsified
194         .key_expand_en_o ( sp_key_expand_en[i] ), // Sparsified
195         .key_expand_out_req_i ( sp_key_expand_out_req[i] ), // Sparsified
196         .key_expand_out_ack_o ( sp_key_expand_out_ack[i] ), // Sparsified
197         .key_expand_clear_o ( mr_key_expand_clear[i] ), // OR-combine
198         .rnd_ctr_o ( mr_rnd_ctr[i] ), // OR-combine
199         .key_words_sel_o ( mr_key_words_sel[i] ), // OR-combine
200         .round_key_sel_o ( mr_round_key_sel[i] ), // OR-combine
201         .crypt_q_ni ( crypt_q_ni )

```

```

202         .crypt_q_ni ( crypt_q_ni ), // OR-combine
203         .round_key_sel_o ( mr_round_key_sel[i] ), // OR-combine
204         .crypt_q_i ( sp_crypt_q[i] ), // Sparsified
205         .crypt_d_o ( sp_crypt_d[i] ), // Sparsified
206         .dec_key_gen_q_i ( sp_dec_key_gen_q[i] ), // Sparsified
207         .dec_key_gen_d_o ( sp_dec_key_gen_d[i] ), // Sparsified
208         .prng_reseed_q_i ( prng_reseed_q ),
209         .prng_reseed_d_o ( mr_prng_reseed_d[i] ), // AND-combine
210         .key_clear_q_i ( key_clear_q ),
211         .key_clear_d_o ( mr_key_clear_d[i] ), // AND-combine
212         .data_out_clear_q_i ( data_out_clear_q ),
213         .data_out_clear_d_o ( mr_data_out_clear_d[i] ), // AND-combine
214         .clk_i ( clk_i ),
215         .rst_ni ( rst_ni ),
216         end else begin : gen_fsm_n
217           aes_cipher_control_fsm_n #(
218             .SecMasking ( SecMasking ),
219             .SecSBoxImpl ( SecSBoxImpl ),
220             .u_aes_cipher_control_fsm_i (
221               .clk_i ( clk_i ),
222               .rst_ni ( rst_ni ),
223               .in_valid_ni ( sp_in_val-
224                 id[i] ), // Sparsified
225               .in_ready_no ( sp_in_ready[i] ), // Sparsified
226               .out_valid_no ( sp_out_val-
227                 id[i] ), // Sparsified
228               .out_ready_ni ( sp_out_ready[i] ), // Sparsified
229               .cfg_valid_i ( cfg_valid_i ),
230               .op_i ( op_i ),
231               .key_len_i ( key_len_i ),
232               .crypt_ni ( sp_crypt[i] ), // Sparsified
233               .dec_key_gen_ni ( sp_dec_key_gen[i] ), // Sparsified
234               .prng_reseed_i ( prng_reseed_i ),
235               .key_clear_i ( key_clear_i ),
236               .data_out_clear_i ( data_out_clear_i ),
237               .mux_sel_err_i ( mux_sel_err ),
238               .sp_enc_err_i ( sp_enc_err ),
239               .rnd_ctr_err_i ( rnd_ctr_err ),
240               .op_err_i ( op_err_i ),
241               .alert_fatal_i ( alert_fatal_i ),
242               .alert_o ( mr_alert[i] ), // OR-combine
243               .prng_update_o ( mr_prng_update[i] ), // OR-combine
244               .prng_reseed_req_o ( mr_prng_reseed_req[i] ), // OR-combine
245               .prng_reseed_ack_i ( prng_reseed_ack_i ),
246               .state_sel_o ( mr_state_sel[i] ), // OR-combine
247               .state_we_no ( sp_state_we[i] ), // Sparsified
248               .sub_bytes_en_no ( sp_sub_bytes_en[i] ), // Sparsified
249               .sub_bytes_out_req_ni ( sp_sub_bytes_out_req[i] ), // Sparsified
250               .sub_bytes_out_ack_no ( sp_sub_bytes_out_ack[i] ), // Sparsified
251               .add_rk_sel_o ( mr_add_rk_sel[i] ), // OR-combine
252               .key_full_sel_o ( mr_key_full_sel[i] ), // OR-combine
253               .key_full_we_no ( sp_key_full_we[i] ), // Sparsified
254               .key_dec_sel_o ( mr_key_dec_sel[i] ), // OR-combine
255               .key_dec_we_no ( sp_key_dec_we[i] ), // Sparsified
256               .key_expand_en_no ( sp_key_expand_en[i] ), // Sparsified
257               .key_expand_out_req_ni ( sp_key_expand_out_req[i] ), // Sparsified
258               .key_expand_out_ack_no ( sp_key_expand_out_ack[i] ), // Sparsified
259               .key_expand_clear_o ( mr_key_expand_clear[i] ), // OR-combine
260               .rnd_ctr_o ( mr_rnd_ctr[i] ), // OR-combine
261               .key_words_sel_o ( mr_key_words_sel[i] ), // OR-combine
262               .round_key_sel_o ( mr_round_key_sel[i] ), // OR-combine
263               .crypt_q_ni ( crypt_q_ni )

```

```

( sp_crypt_q[i] ), // Sparsified
269 .crypt_d_no ), // Sparsified
( sp_crypt_d[i] ), // Sparsified
270 .dec_key_gen_q_ni ), // Sparsified
( sp_dec_key_gen_q[i] ), // Sparsified
271 .dec_key_gen_d_no ), // Sparsified
( sp_dec_key_gen_d[i] ), // Sparsified
272 .prng_reseed_q_i ( prng_reseed_q ),
273 .prng_reseed_d_o ), // AND-combine
( mr_prng_reseed_d[i] ), // AND-combine
274 .key_clear_q_i ( key_clear_q ),
275 .key_clear_d_o ), // AND-combine
( mr_key_clear_d[i] ), // AND-combine
276 .data_out_clear_q_i ( data_out_clear_q ),
277 .data_out_clear_d_o ), // AND-combine
( mr_data_out_clear_d[i] ) // AND-combine
278 );
279 end
280 end
281
282 // Convert sparsified outputs to sp2v_e type.
283 assign in_ready_o = sp2v_e'(sp_in_ready);
284 assign out_valid_o = sp2v_e'(sp_out_valid);
285 assign state_we_o = sp2v_e'(sp_state_we);
286 assign sub_bytes_en_o = sp2v_e'(sp_sub_bytes_en);
287 assign sub_bytes_out_ack_o = sp2v_e'(sp_sub_bytes_out_ack);
288 assign key_full_we_o = sp2v_e'(sp_key_full_we);
289 assign key_dec_we_o = sp2v_e'(sp_key_dec_we);
290 assign key_expand_en_o = sp2v_e'(sp_key_expand_en);
291 assign key_expand_out_ack_o = sp2v_e'(sp_key_ex-
pand_out_ack);
292 assign crypt_d = sp2v_e'(sp_crypt_d);
293 assign dec_key_gen_d = sp2v_e'(sp_dec_key_gen_d);
294
295 // Combine single-bit FSM outputs.
296 // OR: One bit is sufficient to drive the corresponding out-
put bit high.
297 assign alert_o = |mr_alert;
298 assign prng_update_o = |mr_prng_update;
299 assign prng_reseed_req_o = |mr_prng_reseed_req;
300 assign key_expand_clear_o = |mr_key_expand_clear;
301 // AND: Only if all bits are high, the corresponding status
is signaled which will lead to
302 // the clearing of these trigger bits.
303 assign prng_reseed_d = &mr_prng_reseed_d;
304 assign key_clear_d = &mr_key_clear_d;
305 assign data_out_clear_d = &mr_data_out_clear_d;
306
307 // Combine multi-bit, sparse FSM outputs. We simply OR them
together. If the FSMs don't provide
308 // the same outputs, two cases are possible:
309 // - An invalid encoding results: A downstream checker will
fire, see mux_sel_err_i.
310 // - A valid encoding results: The outputs are compared be-
low to cover this case, see mr_err;
311 always_comb begin : combine_sparse_signals
312 state_sel_o = state_sel_e'({StateSelWidth{1'b0}});
313 add_rk_sel_o = add_rk_sel_e'({AddRkSelWidth{1'b0}});
314 key_full_sel_o =
key_full_sel_e'({KeyFullSelWidth{1'b0}});
315 key_dec_sel_o = key_dec_sel_e'({KeyDecSelWidth{1'b0}});
316 key_words_sel_o =
key_words_sel_e'({KeyWordsSelWidth{1'b0}});
317 round_key_sel_o =
round_key_sel_e'({RoundKeySelWidth{1'b0}});
318 mr_err = 1'b0;
319
320 for (int i = 0; i < Sp2VWidth; i++) begin
321 state_sel_o = state_sel_e'({state_sel_o} |
{mr_state_sel[i]});
322 add_rk_sel_o = add_rk_sel_e'({add_rk_sel_o} |
{mr_add_rk_sel[i]});
323 key_full_sel_o = key_full_sel_e'({key_full_sel_o} |
{mr_key_full_sel[i]});
324 key_dec_sel_o = key_dec_sel_e'({key_dec_sel_o} |
{mr_key_dec_sel[i]});
325 key_words_sel_o = key_words_sel_e'({key_words_sel_o} |
{mr_key_words_sel[i]});
326 round_key_sel_o = round_key_sel_e'({round_key_sel_o} |
{mr_round_key_sel[i]});
327 end
328
329 for (int i = 0; i < Sp2VWidth; i++) begin
330 if (state_sel_o != mr_state_sel[i] ||
331 add_rk_sel_o != mr_add_rk_sel[i] ||
332 key_full_sel_o != mr_key_full_sel[i] ||
333 key_dec_sel_o != mr_key_dec_sel[i] ||
334 key_words_sel_o != mr_key_words_sel[i] ||
335 round_key_sel_o != mr_round_key_sel[i]) begin
336 mr_err = 1'b1;
337 end
338 end
339 end
340
341 // Collect errors in mux selector signals.
342 assign mux_sel_err = mux_sel_err_i | mr_err;
343
344 // Combine counter signals. We simply OR them together. If
the FSMs don't provide the same
345 // outputs, rnd_ctr_err will be set.
346 always_comb begin : combine_counter_signals
347 rnd_ctr = '0;
348 rnd_ctr_err = 1'b0;
349 for (int i = 0; i < Sp2VWidth; i++) begin
350 rnd_ctr |= mr_rnd_ctr[i];
351 end
352
353 for (int i = 0; i < Sp2VWidth; i++) begin
354 if (rnd_ctr != mr_rnd_ctr[i]) begin
355 rnd_ctr_err = 1'b1;
356 end
357 end
358 end
359
360 always_ff @(posedge clk_i or negedge rst_ni) begin : reg_fsm
361 if (!rst_ni) begin
362 prng_reseed_q <= 1'b0;
363 key_clear_q <= 1'b0;
364 data_out_clear_q <= 1'b0;
365 end else begin
366 prng_reseed_q <= prng_reseed_d;
367 key_clear_q <= key_clear_d;
368 data_out_clear_q <= data_out_clear_d;
369 end
370 end
371
372 // Use separate signal for key expand operation, forward
round.
373 assign key_expand_op_o = (dec_key_gen_d == SP2V_HIGH ||
374 dec_key_gen_q == SP2V_HIGH) ||
CiphOpFwdOnly ? CIPH_FWD : op_i;
375 assign key_expand_round_o = rnd_ctr;
376
377 // Let the main controller know what we are doing.
378 assign crypt_o = crypt_q;
379 assign dec_key_gen_o = dec_key_gen_q;
380 assign prng_reseed_o = prng_reseed_q;
381 assign key_clear_o = key_clear_q;
382 assign data_out_clear_o = data_out_clear_q;
383
384 ///////////////////////////////////////////////////
385 // Sparsely Encoded Signals //
386 ///////////////////////////////////////////////////
387
388 // SEC_CM: CTRL.SPARSE
389 // We use sparse encodings for various critical signals and
must ensure that:
390 // 1. The synthesis tool doesn't optimize away the sparse
encoding.
391 // 2. The sparsely encoded signal is always valid. More pre-
cisely, an alert or SVA is triggered
392 // 3. The alert signal remains asserted until reset even if
the sparse signal becomes valid again
393 // This is achieved by driving the control FSM into the
terminal error state whenever any
394 // sparsely encoded signal becomes invalid.
395 // If any sparsely encoded signal becomes invalid, the
cipher core further immediately de-asserts
396 // the out_valid_o signal to prevent any data from being re-
leased.
397
398 // The following primitives are used to place a size-only
constraint on the
399 // flops in order to prevent optimizations on these status
signals.
400 logic [Sp2VWidth-1:0] crypt_q_raw;
401 caliptra_prim_flop #(
402 .Width ( Sp2VWidth ),
403 .ResetValue ( Sp2VWidth'(SP2V_LOW) )
404 ) u_crypt_regs (
405 .clk_i ( clk_i ),
406 .rst_ni ( rst_ni ),
407 .d_i ( crypt_d ),
408 .q_o ( crypt_q_raw )
409 );
410
411 logic [Sp2VWidth-1:0] dec_key_gen_q_raw;
412 caliptra_prim_flop #(
413 .Width ( Sp2VWidth ),
414 .ResetValue ( Sp2VWidth'(SP2V_LOW) )
415 ) u_dec_key_gen_regs (
416 .clk_i ( clk_i ),
417 .rst_ni ( rst_ni ),
418 .d_i ( dec_key_gen_d ),
419 .q_o ( dec_key_gen_q_raw )
420 );
421
422 // We use vectors of sparsely encoded signals to reduce code
duplication.
423 localparam int unsigned NumSp2VSig = 8;
424 sp2v_e [NumSp2VSig-1:0] sp2v_sig;
425 sp2v_e [NumSp2VSig-1:0] sp2v_sig_chk;
426 logic [NumSp2VSig-1:0][Sp2VWidth-1:0] sp2v_sig_chk_raw;
427 logic [NumSp2VSig-1:0] sp2v_sig_err;
428
429 assign sp2v_sig[0] = in_valid_i;
430 assign sp2v_sig[1] = out_ready_i;
431 assign sp2v_sig[2] = crypt_i;

```

```

435 assign sp2v_sig[3] = dec_key_gen_i;
436 assign sp2v_sig[4] = sp2v_e'(crypt_q_raw);
437 assign sp2v_sig[5] = sp2v_e'(dec_key_gen_q_raw);
438 assign sp2v_sig[6] = sub_bytes_out_req_i;
439 assign sp2v_sig[7] = key_expand_out_req_i;
440
441 // All signals inside sp2v_sig except for sub_bytes/key_ex-
pand_out_req_i are driven and consumed
442 // by multi-rail FSMs.
443 localparam bit [NumSp2Vsig-1:0] Sp2VEnSecBuf = 8'b1100_0000;
444
445 // Individually check sparsely encoded signals.
446 for (genvar i = 0; i < NumSp2Vsig; i++) begin :
gen_sel_buf_chk
447     aes_sel_buf_chk #(
448         .Num      ( Sp2VNum      ),
449         .Width    ( Sp2VWidth    ),
450         .EnSecBuf ( Sp2VEnSecBuf[i] ),
451     ) u_aes_sp2v_sig_buf_chk_i (
452         .clk_i   ( clk_i   ),
453         .rst_ni  ( rst_ni  ),
454         .sel_i   ( sp2v_sig[i] ),
455         .sel_o   ( sp2v_sig_chk_raw[i] ),
456         .err_o   ( sp2v_sig_err[i] )
457     );
458     assign sp2v_sig_chk[i] = sp2v_e'(sp2v_sig_chk_raw[i]);
459 end
460
461 assign in_valid      = sp2v_sig_chk[0];
462 assign out_ready     = sp2v_sig_chk[1];
463 assign crypt         = sp2v_sig_chk[2];
464 assign dec_key_gen   = sp2v_sig_chk[3];
465 assign crypt_q       = sp2v_sig_chk[4];
466 assign dec_key_gen_q = sp2v_sig_chk[5];
467 assign sub_bytes_out_req = sp2v_sig_chk[6];
468 assign key_expand_out_req = sp2v_sig_chk[7];
469
470 // Collect encoding errors.
471 // We instantiate the checker modules as close as possible
to where the sparsely encoded signals
472 // are used. Here, we collect also encoding errors detected
in other places of the cipher core.
473 assign sp_enc_err = |sp2v_sig_err | sp_enc_err_i;
474
475 // Assertions //
476 // Assertions //
477 // Assertions //
478
479 // Selectors must be known/valid
480 `CALIPTRA_ASSERT(AesCiphOpValid, cfg_valid_i |-> op_i inside {
481     CIPH_FWD,
482     CIPH_INV
483 })
484
485 endmodule

```

3. chipsalliance_caliptra-rtl/src/aes/rtl/ aes_cipher_control_fsm.sv (515 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core control FSM
6 //
7 // This module contains the AES cipher core control FSM.
8
9 `include "caliptra_prim_assert.sv"
10
11 module aes_cipher_control_fsm import aes_pkg::*;
12 #(
13     parameter bit SecMasking = 0,
14     parameter sbox_impl_e SecSBoxImpl = SBoxImplDom
15 ) (
16     input logic clk_i,
17     input logic rst_ni,
18
19     // Input handshake signals
20     input logic in_valid_i, // Sparsify
using multi-rail.
21     output logic in_ready_o, // Sparsify
using multi-rail.
22
23     // Output handshake signals
24     output logic out_valid_o, // Sparsify
using multi-rail.
25     input logic out_ready_i, // Sparsify
using multi-rail.
26
27     // Control and sync signals
28     input logic cfg_valid_i, // Used for
SVAs only.
29     input ciph_op_e op_i,
30     input key_len_e key_len_i,
31     input logic crypt_i, // Sparsify
using multi-rail.
32     input logic dec_key_gen_i, // Sparsify

```

```

using multi-rail.
33     input logic prng_reseed_i,
34     input logic key_clear_i,
35     input logic data_out_clear_i,
36     input logic mux_sel_err_i,
37     input logic sp_enc_err_i,
38     input logic rnd_ctr_err_i,
39     input logic op_err_i,
40     input logic alert_fatal_i,
41     output logic alert_o,
42
43     // Control signals for masking PRNG
44     output logic prng_update_o,
45     output logic prng_reseed_req_o,
46     input logic prng_reseed_ack_i,
47
48     // Control and sync signals for cipher data path
49     output state_sel_e state_sel_o,
50     output logic state_we_o, // Sparsify
using multi-rail.
51     output logic sub_bytes_en_o, // Sparsify
using multi-rail.
52     input logic sub_bytes_out_req_i, // Sparsify
using multi-rail.
53     output logic sub_bytes_out_ack_o, // Sparsify
using multi-rail.
54     output add_rk_sel_e add_rk_sel_o,
55
56     // Control and sync signals for key expand data path
57     output key_full_sel_e key_full_sel_o,
58     output logic key_full_we_o, // Sparsify
using multi-rail.
59     output key_dec_sel_e key_dec_sel_o,
60     output logic key_dec_we_o, // Sparsify
using multi-rail.
61     output logic key_expand_en_o, // Sparsify
using multi-rail.
62     input logic key_expand_out_req_i, // Sparsify
using multi-rail.
63     output logic key_expand_out_ack_o, // Sparsify
using multi-rail.
64     output logic key_expand_clear_o,
65     output logic [3:0] rnd_ctr_o,
66     output key_words_sel_e key_words_sel_o,
67     output round_key_sel_e round_key_sel_o,
68
69     // Register signals
70     input logic crypt_q_i, // Sparsify
using multi-rail.
71     output logic crypt_d_o, // Sparsify
using multi-rail.
72     input logic dec_key_gen_q_i, // Sparsify
using multi-rail.
73     output logic dec_key_gen_d_o, // Sparsify
using multi-rail.
74     input logic prng_reseed_q_i,
75     output logic prng_reseed_d_o,
76     input logic key_clear_q_i,
77     output logic key_clear_d_o,
78     input logic data_out_clear_q_i,
79     output logic data_out_clear_d_o
80 );
81
82 // cfg_valid_i is used for SVAs only.
83 logic unused_cfg_valid;
84 assign unused_cfg_valid = cfg_valid_i;
85
86 // Tie off unused inputs.
87 if (!SecMasking) begin : gen_unused_prng_reseed
88     logic unused_prng_reseed;
89     assign unused_prng_reseed = prng_reseed_i;
90 end
91
92 // Signals
93 aes_cipher_ctrl_e aes_cipher_ctrl_ns, aes_cipher_ctrl_cs;
94 logic advance;
95 logic [2:0] cyc_ctr_d, cyc_ctr_q;
96 logic cyc_ctr_expr;
97 logic prng_reseed_done_d, prng_reseed_done_q;
98 logic [3:0] rnd_ctr_d, rnd_ctr_q;
99 logic [3:0] num_rounds_d, num_rounds_q;
100 logic [3:0] num_rounds_regular;
101
102 // Use separate signal for number of regular rounds.
103 assign num_rounds_regular = num_rounds_q - 4'd1;
104
105 // FSM
106 always_comb begin : aes_cipher_ctrl_fsm
107
108     // Handshake signals
109     in_ready_o = 1'b0;
110     out_valid_o = 1'b0;
111
112     // Masking PRNG signals
113     prng_update_o = 1'b0;
114     prng_reseed_req_o = 1'b0;
115
116     // Cipher data path
117     state_sel_o = STATE_ROUND;
118     state_we_o = 1'b0;
119     add_rk_sel_o = ADD_RK_ROUND;

```



```

120 sub_bytes_en_o = 1'b0;
121 sub_bytes_out_ack_o = 1'b0;
122
123 // Key expand data path
124 key_full_sel_o = KEY_FULL_ROUND;
125 key_full_we_o = 1'b0;
126 key_dec_sel_o = KEY_DEC_EXPAND;
127 key_dec_we_o = 1'b0;
128 key_expand_en_o = 1'b0;
129 key_expand_out_ack_o = 1'b0;
130 key_expand_clear_o = 1'b0;
131 key_words_sel_o = KEY_WORDS_ZERO;
132 round_key_sel_o = ROUND_KEY_DIRECT;
133
134 // FSM
135 aes_cipher_ctrl_ns = aes_cipher_ctrl_cs;
136 num_rounds_d = num_rounds_q;
137 rnd_ctr_d = rnd_ctr_q;
138 crypt_d_o = crypt_q_i;
139 dec_key_gen_d_o = dec_key_gen_q_i;
140 prng_reseed_d_o = prng_reseed_q_i;
141 key_clear_d_o = key_clear_q_i;
142 data_out_clear_d_o = data_out_clear_q_i;
143 prng_reseed_done_d = prng_reseed_done_q | prng_reseed_ack_i;
144 advance = 1'b0;
145 cyc_ctr_d = (SecSBoxImpl == SBoxImplDom) ?
cyc_ctr_q + 3'd1 : 3'd0;
146
147 // Alert
148 alert_o = 1'b0;
149
150 unique case (aes_cipher_ctrl_cs)
151
152 CIPHER_CTRL_IDLE: begin
153   cyc_ctr_d = 3'd0;
154
155   // Signal that we are ready, wait for handshake.
156   in_ready_o = 1'b1;
157   if (in_valid_i) begin
158     if (SecMasking && prng_reseed_i && !dec_key_gen_i
&& !crypt_i) begin
159       // Reseed the masking PRNG without starting encryption/decryption or generation of the
160       // start key for decryption.
161       prng_reseed_d_o = 1'b1;
162       prng_reseed_done_d = 1'b0;
163       aes_cipher_ctrl_ns = CIPHER_CTRL_PRNG_RESEED;
164
165     end else if (key_clear_i || data_out_clear_i) begin
166       // Clear internal key registers. The cipher core
muxes are used to clear the data
167       // output registers.
168       key_clear_d_o = key_clear_i;
169       data_out_clear_d_o = data_out_clear_i;
170
171       // To clear the data output registers, we must
first clear the state.
172       aes_cipher_ctrl_ns = data_out_clear_i ?
CIPHER_CTRL_CLEAR_S : CIPHER_CTRL_CLEAR_KD;
173
174     end else if (dec_key_gen_i || crypt_i) begin
175       // Start encryption/decryption or generation of
start key for decryption.
176       crypt_d_o = ~dec_key_gen_i & crypt_i;
177       dec_key_gen_d_o = dec_key_gen_i;
178
179       // Latch whether we shall reseed the masking PRNG.
180       prng_reseed_d_o = SecMasking & prng_reseed_i;
181
182       // Load input data to state
183       state_sel_o = dec_key_gen_i ? STATE_CLEAR :
STATE_INIT;
184       state_we_o = 1'b1;
185
186       // Make the masking PRNG advance. The current
pseudo-random data is used to mask the
187       // input data.
188       prng_update_o = SecMasking;
189
190       // Init key expand
191       key_expand_clear_o = 1'b1;
192
193       // Load full key
194       key_full_sel_o = dec_key_gen_i ?
KEY_FULL_ENC_INIT :
195       (op_i == CIPH_FWD) ?
KEY_FULL_ENC_INIT :
196       (op_i == CIPH_INV) ?
KEY_FULL_DEC_INIT :
197       KEY_FULL_ENC_INIT;
198       key_full_we_o = 1'b1;
199
200       // Load num_rounds, initialize round counter.
201       num_rounds_d = (key_len_i == AES_128) ? 4'd10 :
202       (key_len_i == AES_192) ? 4'd12 :
203       4'd14;
204       rnd_ctr_d = 0;
205       aes_cipher_ctrl_ns = CIPHER_CTRL_INIT;
206

```

```

207     end else begin
208       // Handshake without a valid command. We should
never get here. If we do (e.g. via a
209       // malicious glitch), error out immediately.
210       aes_cipher_ctrl_ns = CIPHER_CTRL_ERROR;
211     end
212   end
213 end
214
215 CIPHER_CTRL_INIT: begin
216   // Initial round: just add key to state
217   add_rk_sel_o = ADD_RK_INIT;
218
219   // Select key words for initial add_round_key
220   key_words_sel_o = (dec_key_gen_q_i) ?
KEY_WORDS_ZERO :
221   (key_len_i == AES_128) ?
KEY_WORDS_0123 :
222   (key_len_i == AES_192 && op_i == CIPH_FWD) ?
KEY_WORDS_0123 :
223   (key_len_i == AES_192 && op_i == CIPH_INV) ?
KEY_WORDS_2345 :
224   (key_len_i == AES_256 && op_i == CIPH_FWD) ?
KEY_WORDS_0123 :
225   (key_len_i == AES_256 && op_i == CIPH_INV) ?
KEY_WORDS_4567 : KEY_WORDS_ZERO;
226
227   // Clear masking PRNG reseed status.
228   prng_reseed_done_d = 1'b0;
229
230   // AES-256 has two round keys available right from beginning. Pseudo-random data is
231   // required by KeyExpand only.
232   if (key_len_i != AES_256) begin
233     // Advance in sync with KeyExpand. Based on the S-
Box implementation, it can take
234     // multiple cycles to finish. Wait for handshake.
The DOM S-Boxes consume fresh PRD
235     // only in the first clock cycle and that PRD is
taken from the buffer stage updated
236     // based on key_full_we_o. The PRNG itself is updated
in every clock cycle to increase
237     // the noise.
238     advance = key_expand_out_req_i & cyc_ctr_expr;
239
240     prng_update_o = SecMasking;
241     key_expand_en_o = 1'b1;
242     if (advance) begin
243       key_expand_out_ack_o = 1'b1;
244       state_we_o = ~dec_key_gen_q_i;
245       key_full_we_o = 1'b1;
246       rnd_ctr_d = rnd_ctr_q + 4'b0001;
247       cyc_ctr_d = 3'd0;
248       aes_cipher_ctrl_ns = CIPHER_CTRL_ROUND;
249     end
250   end else begin
251     prng_update_o = SecMasking;
252     state_we_o = ~dec_key_gen_q_i;
253     rnd_ctr_d = rnd_ctr_q + 4'b0001;
254     cyc_ctr_d = 3'd0;
255     aes_cipher_ctrl_ns = CIPHER_CTRL_ROUND;
256   end
257 end
258
259 CIPHER_CTRL_ROUND: begin
260   // Normal rounds
261
262   // Select key words for add_round_key
263   key_words_sel_o = (dec_key_gen_q_i) ?
KEY_WORDS_ZERO :
264   (key_len_i == AES_128) ?
KEY_WORDS_0123 :
265   (key_len_i == AES_192 && op_i == CIPH_FWD) ?
KEY_WORDS_2345 :
266   (key_len_i == AES_192 && op_i == CIPH_INV) ?
KEY_WORDS_0123 :
267   (key_len_i == AES_256 && op_i == CIPH_FWD) ?
KEY_WORDS_4567 :
268   (key_len_i == AES_256 && op_i == CIPH_INV) ?
KEY_WORDS_0123 : KEY_WORDS_ZERO;
269
270   // Keep requesting PRNG reseeding until it is acknowledged.
271   prng_reseed_req_o = SecMasking & prng_reseed_q_i &
~prng_reseed_done_q;
272
273   // Select round key: direct or mixed (equivalent inverse cipher)
274   round_key_sel_o = (op_i == CIPH_FWD) ?
ROUND_KEY_DIRECT :
275   (op_i == CIPH_INV) ?
ROUND_KEY_MIXED : ROUND_KEY_DIRECT;
276
277   // Advance in sync with SubBytes and KeyExpand. Based on the S-Box implementation, both can
278   // take multiple cycles to finish. Wait for handshake.
The DOM S-Boxes consume fresh PRD
279
280   // only in the first clock cycle and that PRD is taken from the buffer stages updated

```

```

279 // with state_we_o / based on key_full_we_o. The PRNG
itself is updated in every clock
280 // cycle to increase the noise.
281 advance = key_expand_out_req_i & cys_ctr_expr &
(dec_key_gen_q_i | sub_bytes_out_req_i);
282 prng_update_o = SecMasking;
283 sub_bytes_en_o = ~dec_key_gen_q_i;
284 key_expand_en_o = 1'b1;
285
286 if (advance) begin
287   sub_bytes_out_ack_o = ~dec_key_gen_q_i;
288   key_expand_out_ack_o = 1'b1;
289
290   state_we_o = ~dec_key_gen_q_i;
291   key_full_we_o = 1'b1;
292
293   // Update round
294   rnd_ctr_d = rnd_ctr_q + 4'b0001;
295   cys_ctr_d = 3'd0;
296
297   // Are we doing the last regular round?
298   if (rnd_ctr_q >= num_rounds_regular) begin
299     aes_cipher_ctrl_ns = CIPHER_CTRL_FINISH;
300
301     if (dec_key_gen_q_i) begin
302       // Write decryption key.
303       key_dec_we_o = 1'b1;
304
305       // Indicate that we are done, try to perform the
306       handshake. But we don't wait here.
307       // If we don't get the handshake now, we will
308       wait in the finish state. When using
309       // masking, we only finish if the masking PRNG
310       has been reseeded.
311       out_valid_o = SecMasking ? (prng_reseed_q_i ?
prng_reseed_done_q : 1'b1) : 1'b1;
312       if (out_valid_o && out_ready_i) begin
313         // Go to idle state directly.
314         dec_key_gen_d_o = 1'b0;
315         prng_reseed_d_o = 1'b0;
316         aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
317       end
318     end
319   end
320   end // rnd_ctr_q
321   end // SubBytes/KeyExpand REQ/ACK
322   end
323   CIPHER_CTRL_FINISH: begin
324     // Final round
325     // Select key words for add_round_key
326     key_words_sel_o = (dec_key_gen_q_i) ?
KEY_WORDS_ZERO :
327     (key_len_i == AES_128) ?
KEY_WORDS_0123 :
328     (key_len_i == AES_192 && op_i == CIPH_FWD) ?
KEY_WORDS_2345 :
329     (key_len_i == AES_192 && op_i == CIPH_INV) ?
KEY_WORDS_0123 :
330     (key_len_i == AES_256 && op_i == CIPH_FWD) ?
KEY_WORDS_4567 :
331     (key_len_i == AES_256 && op_i == CIPH_INV) ?
KEY_WORDS_0123 : KEY_WORDS_ZERO;
332
333     // Skip mix_columns
334     add_rk_sel_o = ADD_RK_FINAL;
335
336     // Keep requesting PRNG reseeding until it is acknow-
337     ledged.
338     prng_reseed_req_o = SecMasking & prng_reseed_q_i &
~prng_reseed_done_q;
339
340     // SEC_CM: DATA_REG.KEY.SCA
341     // Once we're done, we won't need the state anymore.
342     // We actually clear it when progressing
343     // to the next state.
344     state_sel_o = STATE_CLEAR;
345
346     // SEC_CM: DATA_REG.LOCAL_ESC
347     // Advance in sync with SubBytes. Based on the S-Box
348     implementation, it can take multiple
349     // cycles to finish. Only indicate that we are done
350     if:
351     // - we have valid output (SubBytes finished),
352     // - the masking PRNG has been reseeded (if masking is
353     used),
354     // - all mux selector signals are valid (don't release
355     data in case of errors), and
356     // - all sparsely encoded signals are valid (don't re-
357     lease data in case of errors).
358     // Perform both handshakes simultaneously.
359     advance = (sub_bytes_out_req_i & cys_ctr_expr)
| dec_key_gen_q_i;
360     sub_bytes_en_o = ~dec_key_gen_q_i;
361     out_valid_o = (mux_sel_err_i || sp_enc_err_i ||
op_err_i) ? 1'b0 :
362     SecMasking ? (prng_reseed_q_i ? prng_reseed_done_q &

```

```

advance : advance) : advance;
354
355 // Stop updating the cycle counter once we have valid
output.
356 cys_ctr_d =
357 (SecSBoxImpl == SBoxImplDom) ? (!advance ?
cys_ctr_q + 3'd1 : cys_ctr_q) : 3'd0;
358
359 // The DOM S-Boxes consume fresh PRD only in the first
360 clock cycle and that PRD is taken
361 // from the buffer stages updated with state_we_o /
362 based on key_full_we_o. The PRNG itself
363 // is updated in every but the last processing clock
364 cycle to increase the noise. Once the
365 // processing is all done (e.g. if we're just waiting
366 for the PRNG reseeding to finish or
367 // if we're waiting for out_ready_i), the PRNG is no
368 longer updated to save power. In the
369 // very last clock cycle, we update the PRNG again to
370 get ready for the next block.
371 prng_update_o =
372 ((SecSBoxImpl == SBoxImplDom) ? !advance : 1'b0) |
(out_valid_o & out_ready_i);
373
374 if (out_valid_o && out_ready_i) begin
375   sub_bytes_out_ack_o = ~dec_key_gen_q_i;
376
377   // Clear the state.
378   state_we_o = 1'b1;
379   crypt_d_o = 1'b0;
380   cys_ctr_d = 3'd0;
381   // If we were generating the decryption key and
382   didn't get the handshake in the last
383   // regular round, we should clear dec_key_gen now.
384   dec_key_gen_d_o = 1'b0;
385   prng_reseed_d_o = 1'b0;
386   aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
387 end
388 end
389 CIPHER_CTRL_PRNG_RESEED: begin
390 // Keep requesting PRNG reseeding until it is acknow-
391 ledged.
392 prng_reseed_req_o = prng_reseed_q_i & ~prng_re-
393 seed_done_q;
394
395 // Don't update the cycle counter as we don't need it.
396 cys_ctr_d = 3'd0;
397
398 // Once we're done, wait for handshake.
399 out_valid_o = prng_reseed_done_q;
400 if (out_valid_o && out_ready_i) begin
401   prng_reseed_d_o = 1'b0;
402   aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
403 end
404 end
405 CIPHER_CTRL_CLEAR_S: begin
406 // Clear the state with pseudo-random data.
407 state_we_o = 1'b1;
408 state_sel_o = STATE_CLEAR;
409 aes_cipher_ctrl_ns = CIPHER_CTRL_CLEAR_KD;
410 end
411 CIPHER_CTRL_CLEAR_KD: begin
412 // Clear internal key registers and/or external data
413 output registers.
414 if (key_clear_q_i) begin
415   key_full_sel_o = KEY_FULL_CLEAR;
416   key_full_we_o = 1'b1;
417   key_dec_sel_o = KEY_DEC_CLEAR;
418   key_dec_we_o = 1'b1;
419 end
420 if (data_out_clear_q_i) begin
421   // Forward the state (previously cleared with
422   pseudo-random data).
423   // SEC_CM: DATA_REG.SEC_WIPE
424   add_rk_sel_o = ADD_RK_INIT;
425   key_words_sel_o = KEY_WORDS_ZERO;
426   round_key_sel_o = ROUND_KEY_DIRECT;
427 end
428 // Indicate that we are done, wait for handshake.
429 out_valid_o = 1'b1;
430 if (out_ready_i) begin
431   key_clear_d_o = 1'b0;
432   data_out_clear_d_o = 1'b0;
433   aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
434 end
435 end
436 CIPHER_CTRL_ERROR: begin
437 // SEC_CM: CIPHER.FSM.LOCAL_ESC
438 // Terminal error state
439 alert_o = 1'b1;
440 end
441 // We should never get here. If we do (e.g. via a mali-
442 cious glitch), error out immediately.
443 default: begin

```



```

437     aes_cipher_ctrl_ns = CIPHER_CTRL_ERROR;
438     alert_o = 1'b1;
439 end
440 endcase
441
442 // Unconditionally jump into the terminal error state in
case a mux selector or a sparsely
443 // encoded signal becomes invalid, in case we have detec-
ted a fault in the round counter,
444 // or if a fatal alert has been triggered.
445 if (mux_sel_err_i || sp_enc_err_i || rnd_ctr_err_i ||
op_err_i || alert_fatal_i) begin
446     aes_cipher_ctrl_ns = CIPHER_CTRL_ERROR;
447 end
448 end
449
450 // SEC_CM: CIPHER.FSM.SPARSE
451 `CALIPTRA_PRIM_FLOP_SPARSE_FSM(u_state_regs,
aes_cipher_ctrl_ns,
452     aes_cipher_ctrl_cs, aes_cipher_ctrl_e, CIPHER_CTRL_IDLE)
453
454 always_ff @(posedge clk_i or negedge rst_ni) begin : reg_fsm
455     if (!rst_ni) begin
456         prng_reseed_done_q <= 1'b0;
457         rnd_ctr_q <= '0;
458         num_rounds_q <= '0;
459     end else begin
460         prng_reseed_done_q <= prng_reseed_done_d;
461         rnd_ctr_q <= rnd_ctr_d;
462         num_rounds_q <= num_rounds_d;
463     end
464 end
465
466 assign rnd_ctr_o = rnd_ctr_q;
467
468 if (SecSBoxImpl == SBoxImplDom) begin : gen_cyc_ctr
469     always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_cyc_ctr
470         if (!rst_ni) begin
471             cyc_ctr_q <= 3'd0;
472         end else begin
473             cyc_ctr_q <= cyc_ctr_d;
474         end
475     end
476     assign cyc_ctr_expr = cyc_ctr_q >= 3'd4;
477 end else begin : gen_no_cyc_ctr
478     logic [2:0] unused_cyc_ctr;
479     assign cyc_ctr_q = cyc_ctr_d;
480     assign unused_cyc_ctr = cyc_ctr_q;
481     assign cyc_ctr_expr = 1'b1;
482 end
483
484 //////////////////////////////////////////////////
485 // Assertions //
486 //////////////////////////////////////////////////
487
488 // Create a lint error to reduce the risk of accidentally
disabling the masking.
489 CALIPT-
RA_ASSERT_STATIC_LINT_ERROR(AesCipherControlFsmSecMaskingNonDefault,
SecMasking == 1)
490
491 // Create a lint error to reduce the risk of accidentally
using a less secure SBox
492 // implementation.
493 `CALIPT-
RA_ASSERT_STATIC_LINT_ERROR(AesCipherControlFsmSecSBoxImplNonDefault
, SecSBoxImpl == SBoxImplDom)
494
495 // Selectors must be known/valid
496
497 `CALIPTRA_ASSERT(AesCiphOpValid, cfg_valid_i |> op_i inside {
498     CIPH_FWD,
499     CIPH_INV
500 })
501 `CALIPTRA_ASSERT(AesKeyLenValid, cfg_valid_i |> key_len_i
inside {
502     AES_128,
503     AES_192,
504     AES_256
505 })
506 `CALIPTRA_ASSERT(AesCipherControlStateValid, !alert_o |>
aes_cipher_ctrl_cs inside {
507     CIPHER_CTRL_IDLE,
508     CIPHER_CTRL_INIT,
509     CIPHER_CTRL_ROUND,
510     CIPHER_CTRL_FINISH,
511     CIPHER_CTRL_PRNG_RESEED,
512     CIPHER_CTRL_CLEAR_S,
513     CIPHER_CTRL_CLEAR_KD
514 })
515 endmodule

```

4. chipsalliance_caliptra-rtl/src/aes/rtl/

aes_cipher_control_fsm_n.sv (400 lines)

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for

details.

```

3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core control FSM
6 //
7 // This module contains the AES cipher core control FSM oper-
ating on and producing the negated
8 // values of important control signals. This is achieved by:
9 // - instantiating the regular AES cipher core control FSM op-
erating on and producing the positive
10 // values of these signals, and
11 // - inverting these signals between the regular FSM and the
caliptra_prim_buf synthesis barriers.
12 // Synthesis tools will then push the inverters into the actu-
al FSM.
13
14 module aes_cipher_control_fsm_n import aes_pkg::*;
15 #()
16     parameter bit SecMasking = 0,
17     parameter sbox_impl_e SecSBoxImpl = SBoxImplDom
18 ) (
19     input logic clk_i,
20     input logic rst_ni,
21
22     // Input handshake signals
23     input logic in_valid_ni, // Sparsify
using multi-rail.
24     output logic in_ready_no, // Sparsify
using multi-rail.
25
26     // Output handshake signals
27     output logic out_valid_no, // Sparsify
using multi-rail.
28     input logic out_ready_ni, // Sparsify
using multi-rail.
29
30     // Control and sync signals
31     input logic cfg_valid_i, // Used for
SVAs only.
32     input ciph_op_e op_i,
33     input key_len_e key_len_i,
34     input logic crypt_ni, // Sparsify
using multi-rail.
35     input logic dec_key_gen_ni, // Sparsify
using multi-rail.
36     input logic prng_reseed_i,
37     input logic key_clear_i,
38     input logic data_out_clear_i,
39     input logic mux_sel_err_i,
40     input logic sp_enc_err_i,
41     input logic rnd_ctr_err_i,
42     input logic op_err_i,
43     input logic alert_fatal_i,
44     output logic alert_o,
45
46     // Control signals for masking PRNG
47     output logic prng_update_o,
48     output logic prng_reseed_req_o,
49     input logic prng_reseed_ack_i,
50
51     // Control and sync signals for cipher data path
52     output state_sel_e state_sel_o,
53     output logic state_we_no, // Sparsify
using multi-rail.
54     output logic sub_bytes_en_no, // Sparsify
using multi-rail.
55     input logic sub_bytes_out_req_ni, // Sparsify
using multi-rail.
56     output logic sub_bytes_out_ack_no, // Sparsify
using multi-rail.
57     output add_rk_sel_e add_rk_sel_o,
58
59     // Control and sync signals for key expand data path
60     output key_full_sel_e key_full_sel_o,
61     output logic key_full_we_no, // Sparsify
using multi-rail.
62     output key_dec_sel_e key_dec_sel_o,
63     output logic key_dec_we_no, // Sparsify
using multi-rail.
64     output logic key_expand_en_no, // Sparsify
using multi-rail.
65     input logic key_expand_out_req_ni, // Sparsify
using multi-rail.
66     output logic key_expand_out_ack_no, // Sparsify
using multi-rail.
67     output logic key_expand_clear_o,
68     output logic [3:0] rnd_ctr_o,
69     output key_words_sel_e key_words_sel_o,
70     output round_key_sel_e round_key_sel_o,
71
72     // Register signals
73     input logic crypt_q_ni, // Sparsify
using multi-rail.
74     output logic crypt_d_no, // Sparsify
using multi-rail.
75     input logic dec_key_gen_q_ni, // Sparsify
using multi-rail.
76     output logic dec_key_gen_d_no, // Sparsify
using multi-rail.
77     input logic prng_reseed_q_i,
78     output logic prng_reseed_d_o,

```

```

79  input  logic      key_clear_q_i,
80  output logic      key_clear_d_o,
81  input  logic      data_out_clear_q_i,
82  output logic      data_out_clear_d_o
83  );
84
85  //////////////////////////////////////////////////
86  // Input Buffering //
87  //////////////////////////////////////////////////
88
89  localparam int NumInBufBits = $bits({
90      in_valid_ni,
91      out_ready_ni,
92      cfg_valid_i,
93      op_i,
94      key_len_i,
95      crypt_ni,
96      dec_key_gen_ni,
97      prng_reseed_i,
98      key_clear_i,
99      data_out_clear_i,
100     mux_sel_err_i,
101     sp_enc_err_i,
102     rnd_ctr_err_i,
103     op_err_i,
104     alert_fatal_i,
105     prng_reseed_ack_i,
106     sub_bytes_out_req_ni,
107     key_expand_out_req_ni,
108     crypt_q_ni,
109     dec_key_gen_q_ni,
110     prng_reseed_q_i,
111     key_clear_q_i,
112     data_out_clear_q_i
113 });
114
115 logic [NumInBufBits-1:0] in, in_buf;
116
117 assign in = {
118     in_valid_ni,
119     out_ready_ni,
120     cfg_valid_i,
121     op_i,
122     key_len_i,
123     crypt_ni,
124     dec_key_gen_ni,
125     prng_reseed_i,
126     key_clear_i,
127     data_out_clear_i,
128     mux_sel_err_i,
129     sp_enc_err_i,
130     rnd_ctr_err_i,
131     op_err_i,
132     alert_fatal_i,
133     prng_reseed_ack_i,
134     sub_bytes_out_req_ni,
135     key_expand_out_req_ni,
136     crypt_q_ni,
137     dec_key_gen_q_ni,
138     prng_reseed_q_i,
139     key_clear_q_i,
140     data_out_clear_q_i
141 };
142
143 // This primitive is used to place a size-only constraint on
the
144 // buffers to act as a synthesis optimization barrier.
145 caliptra_prim_buf #(
146     .Width(NumInBufBits)
147 ) u_caliptra_prim_buf_in (
148     .in_i(in),
149     .out_o(in_buf)
150 );
151
152 logic      in_valid_n;
153 logic      out_ready_n;
154 logic      cfg_valid;
155 logic      op;
156 logic [$bits(op)-1:0] op_raw;
157 logic      key_len_e;
158 logic      crypt_n;
159 logic      dec_key_gen_n;
160 logic      prng_reseed;
161 logic      key_clear;
162 logic      data_out_clear;
163 logic      mux_sel_err;
164 logic      sp_enc_err;
165 logic      rnd_ctr_err;
166 logic      op_err;
167 logic      alert_fatal;
168 logic      prng_reseed_ack;
169 logic      sub_bytes_out_req_n;
170 logic      key_expand_out_req_n;
171 logic      crypt_q_n;
172 logic      dec_key_gen_q_n;
173 logic      prng_reseed_q;
174 logic      key_clear_q;
175 logic      data_out_clear_q;
176
177 assign {in_valid_n,
178
179     out_ready_n,
180     cfg_valid,
181     op_raw,
182     key_len,
183     crypt_n,
184     dec_key_gen_n,
185     prng_reseed,
186     key_clear,
187     data_out_clear,
188     mux_sel_err,
189     sp_enc_err,
190     rnd_ctr_err,
191     op_err,
192     alert_fatal,
193     prng_reseed_ack,
194     sub_bytes_out_req_n,
195     key_expand_out_req_n,
196     crypt_q_n,
197     dec_key_gen_q_n,
198     prng_reseed_q,
199     key_clear_q,
200     data_out_clear_q} = in_buf;
201
202 assign op = ciph_op_e'(op_raw);
203
204 // Intermediate output signals
205 logic      in_ready;
206 logic      out_valid;
207 logic      alert;
208 logic      prng_update;
209 logic      prng_reseed_req;
210 logic      state_sel_e;
211 logic      state_we;
212 logic      sub_bytes_en;
213 logic      sub_bytes_out_ack;
214 logic      add_rk_sel_e;
215 logic      key_full_sel_e;
216 logic      key_full_we;
217 logic      key_dec_sel_e;
218 logic      key_dec_we;
219 logic      key_expand_en;
220 logic      key_expand_out_ack;
221 logic      key_expand_clear;
222 logic      key_words_sel_e;
223 logic      round_key_sel_e;
224 logic [3:0] rnd_ctr;
225 logic      crypt_d;
226 logic      dec_key_gen_d;
227 logic      prng_reseed_d;
228 logic      key_clear_d;
229 logic      data_out_clear_d;
230
231 //////////////////////////////////////////////////
232 // Regular FSM //
233 //////////////////////////////////////////////////
234 // The regular FSM operates on and produces the positive
values of important control signals.
235 // Invert *_n input signals here to get the positive values
for the regular FSM. To obtain the
236 // negated outputs, important output signals are inverted
further below. Thanks to the caliptra_prim_buf
237 // synthesis optimization barriers, tools will push the in-
verters into the regular FSM.
238 aes_cipher_control_fsm #(
239     .SecMasking ( SecMasking ),
240     .SecSBoxImpl ( SecSBoxImpl )
241 ) u_aes_cipher_control_fsm (
242     .clk_i      ( clk_i ),
243     .rst_ni     ( rst_ni ),
244
245     .in_valid_i ( ~in_valid_n ), // In-
vert for regular FSM.
246     .in_ready_o ( in_ready ), // In-
vert below for negated output.
247
248     .out_valid_o ( out_valid ), // In-
vert below for negated output.
249     .out_ready_i ( ~out_ready_n ), // In-
vert for regular FSM.
250
251     .cfg_valid_i ( cfg_valid ),
252     .op_i         ( op ),
253     .key_len_i    ( key_len ),
254     .crypt_i      ( ~crypt_n ), // In-
vert for regular FSM.
255     .dec_key_gen_i ( ~dec_key_gen_n ), // In-
vert for regular FSM.
256     .prng_reseed_i ( prng_reseed ),
257     .key_clear_i   ( key_clear ),
258     .data_out_clear_i ( data_out_clear ),
259     .mux_sel_err_i ( mux_sel_err ),
260     .sp_enc_err_i  ( sp_enc_err ),
261     .rnd_ctr_err_i ( rnd_ctr_err ),
262     .op_err_i      ( op_err ),
263     .alert_fatal_i ( alert_fatal ),
264     .alert_o       ( alert ),
265
266     .prng_update_o ( prng_update ),
267     .prng_reseed_req_o ( prng_reseed_req ),
268     .prng_reseed_ack_i ( prng_reseed_ack ),

```

```

269
270 .state_sel_o      ( state_sel      ), // In-
271 .state_we_o      ( state_we       ), // In-
vert below for negated output.
272 .sub_bytes_en_o  ( sub_bytes_en   ), // In-
vert below for negated output.
273 .sub_bytes_out_req_i  ( ~sub_bytes_out_req_n ), // In-
vert for regular FSM.
274 .sub_bytes_out_ack_o  ( sub_bytes_out_ack ), // In-
vert below for negated output.
275 .add_rk_sel_o    ( add_rk_sel     ),
276
277 .key_full_sel_o   ( key_full_sel   ), // In-
278 .key_full_we_o   ( key_full_we    ), // In-
vert below for negated output.
279 .key_dec_sel_o    ( key_dec_sel    ), // In-
280 .key_dec_we_o    ( key_dec_we     ), // In-
vert below for negated output.
281 .key_expand_en_o  ( key_expand_en  ), // In-
vert below for negated output.
282 .key_expand_out_req_i  ( ~key_expand_out_req_n ), // In-
vert for regular FSM.
283 .key_expand_out_ack_o  ( key_expand_out_ack ), // In-
vert below for negated output.
284 .key_expand_clear_o  ( key_expand_clear ),
285 .rnd_ctr_o         ( rnd_ctr       ),
286 .key_words_sel_o   ( key_words_sel ),
287 .round_key_sel_o   ( round_key_sel ),
288
289 .crypt_q_i        ( ~crypt_q_n    ), // In-
vert for regular FSM.
290 .crypt_d_o        ( crypt_d       ), // In-
vert below for negated output.
291 .dec_key_gen_q_i  ( ~dec_key_gen_q_n ), // In-
vert for regular FSM.
292 .dec_key_gen_d_o  ( dec_key_gen_d ), // In-
vert below for negated output.
293 .key_clear_q_i    ( key_clear_q   ),
294 .key_clear_d_o    ( key_clear_d   ),
295 .prng_reseed_q_i  ( prng_reseed_q ),
296 .prng_reseed_d_o  ( prng_reseed_d ),
297 .data_out_clear_q_i  ( data_out_clear_q ),
298 .data_out_clear_d_o  ( data_out_clear_d )
299 );
300
301 //////////////////////////////////////////////////
302 // Output Buffering //
303 //////////////////////////////////////////////////
304
305 localparam int NumOutBufBits = $bits({
306   in_ready_no,
307   out_valid_no,
308   alert_o,
309   prng_update_o,
310   prng_reseed_req_o,
311   state_sel_o,
312   state_we_no,
313   sub_bytes_en_no,
314   sub_bytes_out_ack_no,
315   add_rk_sel_o,
316   key_full_sel_o,
317   key_full_we_no,
318   key_dec_sel_o,
319   key_dec_we_no,
320   key_expand_en_no,
321   key_expand_out_ack_no,
322   key_expand_clear_o,
323   rnd_ctr_o,
324   key_words_sel_o,
325   round_key_sel_o,
326   crypt_d_no,
327   dec_key_gen_d_no,
328   key_clear_d_o,
329   prng_reseed_d_o,
330   data_out_clear_d_o
331 });
332
333 logic [NumOutBufBits-1:0] out, out_buf;
334
335 // Important output control signals need to be inverted
336 here. Synthesis tools will push the
337 // inverters back into the regular FSM.
338 assign out = {
339   ~in_ready,
340   ~out_valid,
341   alert,
342   prng_update,
343   prng_reseed_req,
344   state_sel,
345   ~state_we,
346   ~sub_bytes_en,
347   ~sub_bytes_out_ack,
348   add_rk_sel,
349   key_full_sel,
350   ~key_full_we,
351   key_dec_sel,
352   ~key_dec_we,
353   ~key_expand_en,
354   ~key_expand_out_ack,
355   key_expand_clear,
356   rnd_ctr,

```

```

356   key_words_sel,
357   round_key_sel,
358   ~crypt_d,
359   ~dec_key_gen_d,
360   key_clear_d,
361   prng_reseed_d,
362   data_out_clear_d
363   };
364
365 // This primitive is used to place a size-only constraint on
the
366 // buffers to act as a synthesis optimization barrier.
367 caliptra_prim_buf #(
368   .Width(NumOutBufBits)
369 ) u_caliptra_prim_buf_out (
370   .in_i(out),
371   .out_o(out_buf)
372 );
373
374 assign {in_ready_no,
375   out_valid_no,
376   alert_o,
377   prng_update_o,
378   prng_reseed_req_o,
379   state_sel_o,
380   state_we_no,
381   sub_bytes_en_no,
382   sub_bytes_out_ack_no,
383   add_rk_sel_o,
384   key_full_sel_o,
385   key_full_we_no,
386   key_dec_sel_o,
387   key_dec_we_no,
388   key_expand_en_no,
389   key_expand_out_ack_no,
390   key_expand_clear_o,
391   rnd_ctr_o,
392   key_words_sel_o,
393   round_key_sel_o,
394   crypt_d_no,
395   dec_key_gen_d_no,
396   key_clear_d_o,
397   prng_reseed_d_o,
398   data_out_clear_d_o} = out_buf;
399
400 endmodule

```

5. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm_p.sv (390 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core control FSM
6 //
7 // This module contains the AES cipher core control FSM oper-
ating on
8 // and producing the positive values of important control sig-
nals.
9
10 module aes_cipher_control_fsm_p import aes_pkg::*;
11 #(
12   parameter bit          SecMasking = 0,
13   parameter sbox_impl_e SecSBoxImpl = SBoxImplDom
14 ) (
15   input logic             clk_i,
16   input logic             rst_ni,
17
18   // Input handshake signals
19   input logic             in_valid_i,           // Sparsify
using multi-rail.
20   output logic            in_ready_o,           // Sparsify
using multi-rail.
21
22   // Output handshake signals
23   output logic            out_valid_o,           // Sparsify
using multi-rail.
24   input logic            out_ready_i,           // Sparsify
using multi-rail.
25
26   // Control and sync signals
27   input logic            cfg_valid_i,           // Used for
SVAs only.
28   input ciph_op_e        op_i,
29   input key_len_e        key_len_i,
30   input logic            crypt_i,           // Sparsify
using multi-rail.
31   input logic            dec_key_gen_i,       // Sparsify
using multi-rail.
32   input logic            prng_reseed_i,
33   input logic            key_clear_i,
34   input logic            data_out_clear_i,
35   input logic            mux_sel_err_i,
36   input logic            sp_enc_err_i,
37   input logic            rnd_ctr_err_i,
38   input logic            op_err_i,

```

```

39  input logic      alert_fatal_i,
40  output logic     alert_o,
41
42  // Control signals for masking PRNG
43  output logic     prng_update_o,
44  output logic     prng_reseed_req_o,
45  input logic      prng_reseed_ack_i,
46
47  // Control and sync signals for cipher data path
48  output state_sel_e state_sel_o,
49  output logic      state_we_o,          // Sparsify
using multi-rail.
50  output logic     sub_bytes_en_o,      // Sparsify
using multi-rail.
51  input logic      sub_bytes_out_req_i, // Sparsify
using multi-rail.
52  output logic     sub_bytes_out_ack_o, // Sparsify
using multi-rail.
53  output add_rk_sel_e add_rk_sel_o,
54
55  // Control and sync signals for key expand data path
56  output key_full_sel_e key_full_sel_o,
57  output logic      key_full_we_o,      // Sparsify
using multi-rail.
58  output key_dec_sel_e key_dec_sel_o,
59  output logic      key_dec_we_o,      // Sparsify
using multi-rail.
60  output logic     key_expand_en_o,     // Sparsify
using multi-rail.
61  input logic      key_expand_out_req_i, // Sparsify
using multi-rail.
62  output logic     key_expand_out_ack_o, // Sparsify
using multi-rail.
63  output logic     key_expand_clear_o,
64  output logic [3:0] rnd_ctr_o,
65  output key_words_sel_e key_words_sel_o,
66  output round_key_sel_e round_key_sel_o,
67
68  // Register signals
69  input logic      crypt_q_i,          // Sparsify
using multi-rail.
70  output logic     crypt_d_o,          // Sparsify
using multi-rail.
71  input logic      dec_key_gen_q_i,    // Sparsify
using multi-rail.
72  output logic     dec_key_gen_d_o,    // Sparsify
using multi-rail.
73  input logic      prng_reseed_q_i,
74  output logic     prng_reseed_d_o,
75  input logic      key_clear_q_i,
76  output logic     key_clear_d_o,
77  input logic      data_out_clear_q_i,
78  output logic     data_out_clear_d_o
79 );
80
81 ///////////////////////////////////////////////////
82 // Input Buffering //
83 ///////////////////////////////////////////////////
84
85 localparam int NumInBufBits = $bits({
86  in_valid_i,
87  out_ready_i,
88  cfg_valid_i,
89  op_i,
90  key_len_i,
91  crypt_i,
92  dec_key_gen_i,
93  prng_reseed_i,
94  key_clear_i,
95  data_out_clear_i,
96  mux_sel_err_i,
97  sp_enc_err_i,
98  rnd_ctr_err_i,
99  op_err_i,
100 alert_fatal_i,
101 prng_reseed_ack_i,
102 sub_bytes_out_req_i,
103 key_expand_out_req_i,
104 crypt_q_i,
105 dec_key_gen_q_i,
106 prng_reseed_q_i,
107 key_clear_q_i,
108 data_out_clear_q_i
109 });
110
111 logic [NumInBufBits-1:0] in, in_buf;
112
113 assign in = {
114  in_valid_i,
115  out_ready_i,
116  cfg_valid_i,
117  op_i,
118  key_len_i,
119  crypt_i,
120  dec_key_gen_i,
121  prng_reseed_i,
122  key_clear_i,
123  data_out_clear_i,
124  mux_sel_err_i,
125  sp_enc_err_i,
126  rnd_ctr_err_i,

```

```

127  op_err_i,
128  alert_fatal_i,
129  prng_reseed_ack_i,
130  sub_bytes_out_req_i,
131  key_expand_out_req_i,
132  crypt_q_i,
133  dec_key_gen_q_i,
134  prng_reseed_q_i,
135  key_clear_q_i,
136  data_out_clear_q_i
137 };
138
139 // This primitive is used to place a size-only constraint on
the
140 // buffers to act as a synthesis optimization barrier.
141 caliptra_prim_buf #(
142  .Width(NumInBufBits)
143 ) u_caliptra_prim_buf_in (
144  .in_i(in),
145  .out_o(in_buf)
146 );
147
148 logic      in_valid;
149 logic      out_ready;
150 logic      cfg_valid;
151 ciph_op_e  op;
152 logic [$bits(op)-1:0] op_raw;
153 key_len_e  key_len;
154 logic      crypt;
155 logic      dec_key_gen;
156 logic      prng_reseed;
157 logic      key_clear;
158 logic      data_out_clear;
159 logic      mux_sel_err;
160 logic      sp_enc_err;
161 logic      rnd_ctr_err;
162 logic      op_err;
163 logic      alert_fatal;
164 logic      prng_reseed_ack;
165 logic      sub_bytes_out_req;
166 logic      key_expand_out_req;
167 logic      crypt_q;
168 logic      dec_key_gen_q;
169 logic      prng_reseed_q;
170 logic      key_clear_q;
171 logic      data_out_clear_q;
172
173 assign {in_valid,
174  out_ready,
175  cfg_valid,
176  op_raw,
177  key_len,
178  crypt,
179  dec_key_gen,
180  prng_reseed,
181  key_clear,
182  data_out_clear,
183  mux_sel_err,
184  sp_enc_err,
185  rnd_ctr_err,
186  op_err,
187  alert_fatal,
188  prng_reseed_ack,
189  sub_bytes_out_req,
190  key_expand_out_req,
191  crypt_q,
192  dec_key_gen_q,
193  prng_reseed_q,
194  key_clear_q,
195  data_out_clear_q} = in_buf;
196
197 assign op = ciph_op_e'(op_raw);
198
199 // Intermediate output signals
200 logic      in_ready;
201 logic      out_valid;
202 logic      alert;
203 logic      prng_update;
204 logic      prng_reseed_req;
205 state_sel_e state_sel;
206 logic      state_we;
207 logic      sub_bytes_en;
208 logic      sub_bytes_out_ack;
209 add_rk_sel_e add_rk_sel;
210 key_full_sel_e key_full_sel;
211 logic      key_full_we;
212 key_dec_sel_e key_dec_sel;
213 logic      key_dec_we;
214 logic      key_expand_en;
215 logic      key_expand_out_ack;
216 logic      key_expand_clear;
217 logic [3:0] rnd_ctr;
218 key_words_sel_e key_words_sel;
219 round_key_sel_e round_key_sel;
220 logic      crypt_d;
221 logic      dec_key_gen_d;
222 logic      prng_reseed_d;
223 logic      key_clear_d;
224 logic      data_out_clear_d;
225

```

```

226 //////////////////////////////////////////////////
227 // Regular FSM //
228 //////////////////////////////////////////////////
229
230 aes_cipher_control_fsm #(
231     .SecMasking ( SecMasking ),
232     .SecSBoxImpl ( SecSBoxImpl )
233 ) u_aes_cipher_control_fsm (
234     .clk_i      ( clk_i          ),
235     .rst_ni     ( rst_ni         ),
236
237     .in_valid_i ( in_valid       ),
238     .in_ready_o ( in_ready      ),
239
240     .out_valid_o ( out_valid     ),
241     .out_ready_i ( out_ready    ),
242
243     .cfg_valid_i ( cfg_valid     ),
244     .op_i        ( op           ),
245     .key_len_i   ( key_len      ),
246     .crypt_i     ( crypt        ),
247     .dec_key_gen_i ( dec_key_gen ),
248     .prng_reseed_i ( prng_reseed ),
249     .key_clear_i  ( key_clear    ),
250     .data_out_clear_i ( data_out_clear ),
251     .mux_sel_err_i ( mux_sel_err ),
252     .sp_enc_err_i ( sp_enc_err  ),
253     .rnd_ctr_err_i ( rnd_ctr_err ),
254     .op_err_i     ( op_err      ),
255     .alert_fatal_i ( alert_fatal ),
256     .alert_o      ( alert       ),
257
258     .prng_update_o ( prng_update ),
259     .prng_reseed_req_o ( prng_reseed_req ),
260     .prng_reseed_ack_i ( prng_reseed_ack ),
261
262     .state_sel_o ( state_sel     ),
263     .state_we_o  ( state_we     ),
264     .sub_bytes_en_o ( sub_bytes_en ),
265     .sub_bytes_out_req_i ( sub_bytes_out_req ),
266     .sub_bytes_out_ack_o ( sub_bytes_out_ack ),
267     .add_rk_sel_o ( add_rk_sel   ),
268
269     .key_full_sel_o ( key_full_sel ),
270     .key_full_we_o  ( key_full_we  ),
271     .key_dec_sel_o  ( key_dec_sel  ),
272     .key_dec_we_o   ( key_dec_we   ),
273     .key_expand_en_o ( key_expand_en ),
274     .key_expand_out_req_i ( key_expand_out_req ),
275     .key_expand_out_ack_o ( key_expand_out_ack ),
276     .key_expand_clear_o ( key_expand_clear ),
277     .rnd_ctr_o      ( rnd_ctr      ),
278     .key_words_sel_o ( key_words_sel ),
279     .round_key_sel_o ( round_key_sel ),
280
281     .crypt_q_i      ( crypt_q      ),
282     .crypt_d_o      ( crypt_d      ),
283     .dec_key_gen_q_i ( dec_key_gen_q ),
284     .dec_key_gen_d_o ( dec_key_gen_d ),
285     .key_clear_q_i   ( key_clear_q  ),
286     .key_clear_d_o   ( key_clear_d  ),
287     .prng_reseed_q_i ( prng_reseed_q ),
288     .prng_reseed_d_o ( prng_reseed_d ),
289     .data_out_clear_q_i ( data_out_clear_q ),
290     .data_out_clear_d_o ( data_out_clear_d ),
291 );
292
293 //////////////////////////////////////////////////
294 // Output Buffering //
295 //////////////////////////////////////////////////
296
297 localparam int NumOutBufBits = $bits({
298     in_ready_o,
299     out_valid_o,
300     alert_o,
301     prng_update_o,
302     prng_reseed_req_o,
303     state_sel_o,
304     state_we_o,
305     sub_bytes_en_o,
306     sub_bytes_out_ack_o,
307     add_rk_sel_o,
308     key_full_sel_o,
309     key_full_we_o,
310     key_dec_sel_o,
311     key_dec_we_o,
312     key_expand_en_o,
313     key_expand_out_ack_o,
314     key_expand_clear_o,
315     rnd_ctr_o,
316     key_words_sel_o,
317     round_key_sel_o,
318     crypt_d_o,
319     dec_key_gen_d_o,
320     key_clear_d_o,
321     prng_reseed_d_o,
322     data_out_clear_d_o
323 });
324
325 logic [NumOutBufBits-1:0] out, out_buf;
326

```

```

327 assign out = {
328     in_ready,
329     out_valid,
330     alert,
331     prng_update,
332     prng_reseed_req,
333     state_sel,
334     state_we,
335     sub_bytes_en,
336     sub_bytes_out_ack,
337     add_rk_sel,
338     key_full_sel,
339     key_full_we,
340     key_dec_sel,
341     key_dec_we,
342     key_expand_en,
343     key_expand_out_ack,
344     key_expand_clear,
345     rnd_ctr,
346     key_words_sel,
347     round_key_sel,
348     crypt_d,
349     dec_key_gen_d,
350     key_clear_d,
351     prng_reseed_d,
352     data_out_clear_d
353 };
354
355 // This primitive is used to place a size-only constraint on
the
356 // buffers to act as a synthesis optimization barrier.
357 caliptra_prim_buf #(
358     .Width(NumOutBufBits)
359 ) u_caliptra_prim_buf_out (
360     .in_i(out),
361     .out_o(out_buf)
362 );
363
364 assign {in_ready_o,
365     out_valid_o,
366     alert_o,
367     prng_update_o,
368     prng_reseed_req_o,
369     state_sel_o,
370     state_we_o,
371     sub_bytes_en_o,
372     sub_bytes_out_ack_o,
373     add_rk_sel_o,
374     key_full_sel_o,
375     key_full_we_o,
376     key_dec_sel_o,
377     key_dec_we_o,
378     key_expand_en_o,
379     key_expand_out_ack_o,
380     key_expand_clear_o,
381     rnd_ctr_o,
382     key_words_sel_o,
383     round_key_sel_o,
384     crypt_d_o,
385     dec_key_gen_d_o,
386     key_clear_d_o,
387     prng_reseed_d_o,
388     data_out_clear_d_o} = out_buf;
389
390 endmodule

```

6. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_core.sv (932 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core implementation
6 //
7 // This module contains the AES cipher core including, state
register, full key and decryption key
8 // registers as well as key expand module and control unit.
9 //
10 //
11 // Masking
12 // -----
13 //
14 // If the parameter "Masking" is set to one, first-order mask-
ing is applied to the entire
15 // cipher core including key expand module. For details, see
Rivain et al., "Provably secure
16 // higher-order masking of AES" available at https://
eprint.iacr.org/2010/441.pdf .
17 //
18 //
19 // Details on the data formats
20 // -----
21 //
22 // This implementation uses 4-dimensional SystemVerilog arrays to
represent the AES state:

```

```

23 //
24 // logic [3:0][3:0][7:0] state_q [NumShares];
25 //
26 // The fourth dimension (unpacked) corresponds to the differ-
ent shares. The first element holds the
27 // (masked) data share whereas the other elements hold the
masks (masked implementation only).

28 // The three packed dimensions correspond to the 128-bit state
matrix per share. This
29 // implementation uses the same encoding as the Advanced En-
cryption Standard (AES) FIPS Publication
30 // 197 available at https://www.nist.gov/publications/ad-
vanced-encryption-standard-aes (see Section
31 // 3.4). An input sequence of 16 bytes (128-bit, left most
byte is the first one)
32 //
33 // b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 b10 b11 b12 b13 b14 b15
34 //
35 // is mapped to the state matrix as
36 //
37 // [ b0 b4 b8 b12 ]
38 // [ b1 b5 b9 b13 ]
39 // [ b2 b6 b10 b14 ]
40 // [ b3 b7 b11 b15 ] .
41 //
42 // This is mapped to three packed dimensions of SystemVerilog
array as follows:
43 // - The first dimension corresponds to the rows. Thus,
state_q[0] gives
44 // - The first row of the state matrix [ b0 b4 b8
b12 ], or
45 // - A 32-bit packed SystemVerilog array 32h'{ b12, b8, b4,
b0 }.
46 //
47 // - The second dimension corresponds to the columns. To ac-
cess complete columns, the state matrix
48 // must be transposed first. Thus state_transposed =
aes_pkg::aes_transpose(state_q) and then
49 // state_transposed[1] gives
50 // - The second column of the state matrix [ b4 b5 b6
b7 ], or
51 // - A 32-bit packed SystemVerilog array 32h'{ b7, b6, b5,
b4 }.
52 //
53 // - The third dimension corresponds to the bytes.
54 //
55 // Note that the CSRs are little-endian. The input sequence
above is provided to 32-bit DATA_IN_0 -
56 // DATA_IN_3 registers as
57 //
58 // - DATA_IN_0 32h'{ b3 , b2 , b1 , b0 }
59 // - DATA_IN_1 32h'{ b7 , b6 , b4 , b4 }
60 // - DATA_IN_2 32h'{ b11, b10, b9 , b8 }
61 // - DATA_IN_3 32h'{ b15, b14, b13, b12 } .
62 //
63 // The input state can thus be obtained by transposing the
content of the DATA_IN_0 - DATA_IN_3
64 // registers.
65 //

66 // Similarly, the implementation uses a 3-dimensional array to
represent the AES keys:
67 //
68 // logic [7:0][31:0] key_full_q [NumShares]
69 //

70 // The third dimension (unpacked) corresponds to the different
shares. The first element holds the
71 // (masked) key share whereas the other elements hold the
masks (masked implementation only).

72 // The two packed dimensions correspond to the 256-bit key per
share. This implementation uses
73 // the same encoding as the Advanced Encryption Standard (AES)
FIPS Publication
74 // 197 available at https://www.nist.gov/publications/ad-
vanced-encryption-standard-aes .
75 //
76 // The first packed dimension corresponds to the 8 key words.
The second packed dimension
77 // corresponds to the 32 bits per key word. A key sequence of
32 bytes (256-bit, left most byte is
78 // the first one)
79 //
80 // b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 b10 b11 b12 b13 b14
b15 ... b28 b29 b30 b31
81 //
82 // is mapped to the key words and registers (little-endian) as
83 //
84 // - KEY_SHARE0_0 32h'{ b3 , b2 , b1 , b0 }
85 // - KEY_SHARE0_1 32h'{ b7 , b6 , b4 , b4 }
86 // - KEY_SHARE0_2 32h'{ b11, b10, b9 , b8 }
87 // - KEY_SHARE0_3 32h'{ b15, b14, b13, b12 }
88 // - KEY_SHARE0_4 32h'{ . . . . }
89 // - KEY_SHARE0_5 32h'{ . . . . }
90 // - KEY_SHARE0_6 32h'{ . . . . }
91 // - KEY_SHARE0_7 32h'{ b31, b30, b29, b28 } .
92 //
93 `include "caliptra_prim_assert.sv"

```

```

94
95 module aes_cipher_core import aes_pkg::*;
96 #(
97     parameter bit AES192Enable = 1,
98     parameter bit CiphOpFwdOnly = 0,
99     parameter bit SecMasking = 1,
100    parameter sbbox_impl_e SecSBoxImpl = SBoxImplDom,
101    parameter bit SecAllowForcingMasks = 0,
102    parameter bit SecSkipPRNGReseeding = 0,
103    parameter int unsigned EntropyWidth = edn_pkg::END-
POINT_BUS_WIDTH,
104
105    localparam int NumShares = SecMasking ?
2 : 1, // derived parameter
106
107    parameter masking_lfsr_seed_t RndCnstMaskingLfsrSeed = Rnd-
CnstMaskingLfsrSeedDefault,
108    parameter masking_lfsr_perm_t RndCnstMaskingLfsrPerm = Rnd-
CnstMaskingLfsrPermDefault
109 ) (
110     input logic clk_i,
111     input logic rst_ni,
112
113     // Input handshake signals
114     input sp2v_e in_valid_i,
115     output sp2v_e in_ready_o,
116
117     // Output handshake signals
118     output sp2v_e out_valid_o,
119     input sp2v_e out_ready_i,
120
121     // Control and sync signals
122     input logic cfg_valid_i, // Used for
gating assertions only.
123     input ciph_op_e op_i,
124     input key_len_e key_len_i,
125     input sp2v_e crypt_i,
126     output sp2v_e crypt_o,
127     input sp2v_e dec_key_gen_i,
128     output sp2v_e dec_key_gen_o,
129     input logic prng_reseed_i,
130     output logic prng_reseed_o,
131     input logic key_clear_i,
132     output logic key_clear_o,
133     input logic data_out_clear_i, // Re-
use the cipher core muxes.
134     output logic data_out_clear_o,
135     input logic alert_fatal_i,
136     output logic alert_o,
137
138     // Pseudo-random data for register clearing
139     input logic [3:0][3:0][7:0] prd_clearing_state_i
[NumShares],
140     input logic [7:0][31:0] prd_clearing_key_i [Num-
Shares],
141
142     // Masking PRNG
143     input logic force_masks_i, // Useful
for SCA only.
144     output logic [3:0][3:0][7:0] data_in_mask_o,
145     output logic entropy_req_o,
146     input logic entropy_ack_i,
147     input logic [EntropyWidth-1:0] entropy_i,
148
149     // I/O data & initial key
150     input logic [3:0][3:0][7:0] state_init_i [Num-
Shares],
151     input logic [7:0][31:0] key_init_i [NumShares],
152     output logic [3:0][3:0][7:0] state_o [NumShares]
153 );
154
155 // Signals
156 logic [3:0][3:0][7:0] state_d [NumShares];
157 logic [3:0][3:0][7:0] state_q [NumShares];
158 sp2v_e state_we_ctrl;
159 sp2v_e state_we;
160 logic [StateSelWidth-1:0] state_sel_raw;
161 state_sel_e state_sel_ctrl;
162 state_sel_e state_sel;
163 logic state_sel_err;
164
165 sp2v_e sub_bytes_en;
166 sp2v_e sub_bytes_out_req;
167 sp2v_e sub_bytes_out_ack;
168 logic sub_bytes_err;
169 logic [3:0][3:0][7:0] sub_bytes_out;
170 logic [3:0][3:0][7:0] sb_in_mask;
171 logic [3:0][3:0][7:0] sb_out_mask;
172 logic [3:0][3:0][7:0] shift_rows_in [Num-
Shares];
173 logic [3:0][3:0][7:0] shift_rows_out [Num-
Shares];
174 logic [3:0][3:0][7:0] mix_columns_out [Num-
Shares];
175 logic [3:0][3:0][7:0] add_round_key_in [Num-
Shares];
176 logic [3:0][3:0][7:0] add_round_key_out [Num-
Shares];
177 logic [AddrRKSelWidth-1:0] add_rk_sel_raw;

```



```

178 add_rk_sel_e      add_rk_sel_ctrl;
179 add_rk_sel_e      add_rk_sel;
180 logic             add_rk_sel_err;
181
182 logic             [7:0][31:0] key_full_d [NumShares];
183 logic             [7:0][31:0] key_full_q [NumShares];
184 sp2v_e            key_full_we_ctrl;
185 sp2v_e            key_full_we;
186 logic             [KeyFullSelWidth-1:0] key_full_sel_raw;
187 key_full_sel_e    key_full_sel_ctrl;
188 key_full_sel_e    key_full_sel;
189 logic             key_full_sel_err;
190 logic             [7:0][31:0] key_dec_d [NumShares];
191 logic             [7:0][31:0] key_dec_q [NumShares];
192 sp2v_e            key_dec_we_ctrl;
193 sp2v_e            key_dec_we;
194 logic             [KeyDecSelWidth-1:0] key_dec_sel_raw;
195 key_dec_sel_e     key_dec_sel_ctrl;
196 key_dec_sel_e     key_dec_sel;
197 logic             key_dec_sel_err;
198 logic             [7:0][31:0] key_expand_out [Num-
Shares];
199 ciph_op_e         key_expand_op;
200 sp2v_e            key_expand_en;
201 logic             key_expand_prd_we;
202 sp2v_e            key_expand_out_req;
203 sp2v_e            key_expand_out_ack;
204 logic             key_expand_err;
205 logic             key_expand_clear;
206 logic             [3:0] key_expand_round;
207 logic             [KeywordsSelWidth-1:0] key_words_sel_raw;
208 key_words_sel_e   key_words_sel_ctrl;
209 key_words_sel_e   key_words_sel;
210 logic             key_words_sel_err;
211 logic             [3:0][31:0] key_words [NumShares];
212 logic             [3:0][3:0][7:0] key_bytes [NumShares];
213 logic             [3:0][3:0][7:0] key_mix_columns_out
[NumShares];
214 logic             [3:0][3:0][7:0] round_key [NumShares];
215 logic             [RoundKeySelWidth-1:0] round_key_sel_raw;
216 round_key_sel_e   round_key_sel_ctrl;
217 round_key_sel_e   round_key_sel;
218 logic             round_key_sel_err;
219
220 logic             cfg_valid;
221 logic             mux_sel_err;
222 logic             sp_enc_err_d,
sp_enc_err_q;
223 logic             op_err;
224
225 // Pseudo-random data for masking purposes
226 logic             [WidthPRDMasking-1:0] prd_masking;
227 logic             [3:0][3:0][WidthPRDSBox-1:0] prd_sub_bytes_d;
228 logic             [3:0][3:0][WidthPRDSBox-1:0] prd_sub_bytes_q;
229 logic             [WidthPRDKey-1:0] prd_key_expand;
230 logic             prd_masking_upd;
231 logic             prd_masking_rsd_req;
232 logic             prd_masking_rsd_ack;
233
234 logic             [3:0][3:0][7:0] data_in_mask;
235
236 // op_i is one-hot encoded. Check the provided value and
trigger an alert upon detecting invalid
237 // encodings.
238 assign op_err      = ~(op_i == CIPH_FWD || op_i == CIPH_INV);
239 assign cfg_valid   = cfg_valid_i & ~op_err;
240
241 //////////////////////////////////////////////////
242 // Data //
243 //////////////////////////////////////////////////
244
245 // SEC_CM: DATA_REG.SEC_WIPE
246 // State registers
247 always_comb begin : state_mux
248     unique case (state_sel)
249         STATE_INIT: state_d = state_init_i;
250         STATE_ROUND: state_d = add_round_key_out;
251         STATE_CLEAR: state_d = prd_clearing_state_i;
252         default: state_d = prd_clearing_state_i;
253     endcase
254 end
255
256 always_ff @(posedge clk_i or negedge rst_ni) begin :
state_reg
257     if (!rst_ni) begin
258         state_q <= '{default: '0};
259     end else if (state_we == SP2V_HIGH) begin
260         state_q <= state_d;
261     end
262 end
263
264 // Masking
265 if (!SecMasking) begin : gen_no_masks
266     // The masks are ignored anyway, they can be 0.
267     assign sb_in_mask = '0;
268     assign prd_masking = '0;
269
270     // Tie-off unused signals.
271     logic unused_entropy_ack;
272     logic [EntropyWidth-1:0] unused_entropy;
273     assign unused_entropy_ack = entropy_ack_i;

```

```

274 assign unused_entropy      = entropy_i;
275 assign entropy_req_o       = '1'b0;
276
277 logic unused_force_masks;
278 logic unused_prd_masking_upd;
279 logic unused_prd_masking_rsd_req;
280 assign unused_force_masks  = force_masks_i;
281 assign unused_prd_masking_upd = prd_masking_upd;
282 assign unused_prd_masking_rsd_req = prd_masking_rsd_req;
283 assign prd_masking_rsd_ack = '1'b0;
284
285 logic [3:0][3:0][7:0] unused_sb_out_mask;
286 assign unused_sb_out_mask = sb_out_mask;
287
288 end else begin : gen_masks
289     // The input mask is the mask share of the state.
290     assign sb_in_mask = state_q[1];
291
292     // The masking PRNG generates:
293     // - the pseudo-random data (PRD) required by SubBytes,
294     // - the PRD required by the key expand module (has 4 S-
Boxes internally).
295     aes_prng_masking #(
296         .Width           ( WidthPRDMasking           ),
297         .EntropyWidth     ( EntropyWidth             ),
298         .SecAllowForcingMasks ( SecAllowForcingMasks   ),
299         .SecSkipPRNGReseeding ( SecSkipPRNGReseeding   ),
300         .RndCnstLfSrSeed   ( RndCnstMaskingLfSrSeed   ),
301         .RndCnstLfSrPerm   ( RndCnstMaskingLfSrPerm   )
302     ) u_aes_prng_masking (
303         .clk_i             ( clk_i                   ),
304         .rst_ni            ( rst_ni                   ),
305         .force_masks_i     ( force_masks_i            ),
306         .data_update_i     ( prd_masking_upd          ),
307         .data_o             ( prd_masking              ),
308         .reseed_req_i      ( prd_masking_rsd_req      ),
309         .reseed_ack_o      ( prd_masking_rsd_ack      ),
310         .entropy_req_o     ( entropy_req_o            ),
311         .entropy_ack_i     ( entropy_ack_i            ),
312         .entropy_i         ( entropy_i                )
313     );
314 end
315
316 // Extract randomness for key expand module and SubBytes.
317 //
318 // The masking PRNG output has the following shape:
319 // prd_masking = { prd_key_expand, prd_sub_bytes_d }
320 assign prd_key_expand = prd_masking[WidthPRDMasking-1 -:
WidthPRDKey];
321 assign prd_sub_bytes_d = prd_masking[WidthPRDData-1 -:
WidthPRDData];
322
323 // PRD buffering
324 if (!SecMasking) begin : gen_no_prd_buffer
325     // The masks are ignored anyway.
326     assign prd_sub_bytes_q = prd_sub_bytes_d;
327
328 end else begin : gen_prd_buffer
329     // PRD buffer stage to:
330     // 1. Make sure the S-Boxes get always presented new data/
mask inputs together with fresh PRD
331     // for remasking.
332
333     // 2. Prevent glitches originating from inside the masking
PRNG from propagating into the
334     // masked S-Boxes.
335     always_ff @(posedge clk_i or negedge rst_ni) begin :
prd_sub_bytes_reg
336         if (!rst_ni) begin
337             prd_sub_bytes_q <= '0;
338         end else if (state_we == SP2V_HIGH) begin
339             prd_sub_bytes_q <= prd_sub_bytes_d;
340         end
341     end
342
343     // Convert the 3-dimensional prd_sub_bytes_q array to a 1-
dimensional packed array for the
344     // aes_prd_get_lsbs() function used below.
345     logic [WidthPRDData-1:0] prd_sub_bytes;
346     assign prd_sub_bytes = prd_sub_bytes_q;
347
348     // Extract randomness for masking the input data.
349     //
350     // The masking PRNG is used for generating both the PRD for
the S-Boxes/SubBytes operation as
351     // well as for the input data masks. When using any of the
masked Canright S-Box implementations,
352     // it is important that the SubBytes input masks (generated
by the PRNG in Round X-1) and the
353     // SubBytes output masks (generated by the PRNG in Round X)
are independent. This can be achieved
354     // by using e.g. an unrolled Bivium stream cipher primitive
inside the PRNG. Since the input data
355     // masks become the SubBytes input masks in the first round,
we select the same 8 bit lanes for
356     // the input data masks which are also used to form the Sub-
Bytes output mask for the masked
357     // Canright S-Box implementations, i.e., the 8 LSBs of the
per S-Box PRD. In particular, we have:

```



```

358 //
359 // prd_masking = { prd_key_expand, ... , sb_prd[4],
sb_out_mask[4], sb_prd[0], sb_out_mask[0] }
360 //
361 // Where sb_out_mask[x] contains the SubBytes output mask
for byte x (when using a masked
362 // Canright S-Box implementation) and sb_prd[x] contains ad-
ditional PRD consumed by SubBytes for
363 // byte x.
364 //
365 // When using a masked S-Box implementation other than Can-
right, we still select the 8 LSBs of
366 // the per-S-Box PRD to form the input data mask of the cor-
responding byte. We do this to
367 // distribute the input data masks over all output bits the
masking PRNG. We do the extraction on
368 // a row basis.
369 localparam int unsigned WidthPRDRow = 4*WidthPRDSBox;
370 for (genvar i = 0; i < 4; i++) begin : gen_in_mask
371     assign data_in_mask[i] = aes_prd_get_lsbs(prd_sub_bytes[i
* WidthPRDRow +: WidthPRDRow]);
372 end
373
374 // Rotate the data input masks by 64 bits to ensure the data
input masks are independent
375 // from the PRD fed to the S-Boxes/SubBytes operation.
376 assign data_in_mask_o = {data_in_mask[1], data_in_mask[0],
data_in_mask[3], data_in_mask[2]};
377
378 // Cipher data path
379 aes_sub_bytes #(
380     .SecSBoxImpl ( SecSBoxImpl )
381 ) u_aes_sub_bytes (
382     .clk_i      ( clk_i      ),
383     .rst_ni     ( rst_ni     ),
384     .en_i       ( sub_bytes_en ),
385     .out_req_o  ( sub_bytes_out_req ),
386     .out_ack_i  ( sub_bytes_out_ack ),
387     .op_i       ( op_i       ),
388     .data_i     ( state_q[0] ),
389     .mask_i     ( sb_in_mask ),
390     .prd_i      ( prd_sub_bytes_q ),
391     .data_o     ( sub_bytes_out ),
392     .mask_o     ( sb_out_mask ),
393     .err_o     ( sub_bytes_err )
394 );
395
396 for (genvar s = 0; s < NumShares; s++) begin :
gen_shares_shift_mix
397     if (s == 0) begin : gen_shift_in_data
398         // The (masked) data share
399         assign shift_rows_in[s] = sub_bytes_out;
400     end else begin : gen_shift_in_mask
401         // The mask share
402         assign shift_rows_in[s] = sb_out_mask;
403     end
404
405     aes_shift_rows u_aes_shift_rows (
406         .op_i ( op_i ),
407         .data_i ( shift_rows_in[s] ),
408         .data_o ( shift_rows_out[s] )
409     );
410
411     aes_mix_columns u_aes_mix_columns (
412         .op_i ( op_i ),
413         .data_i ( shift_rows_out[s] ),
414         .data_o ( mix_columns_out[s] )
415     );
416 end
417
418 always_comb begin : add_round_key_in_mux
419     unique case (add_rk_sel)
420         ADD_RK_INIT: add_round_key_in = state_q;
421         ADD_RK_ROUND: add_round_key_in = mix_columns_out;
422         ADD_RK_FINAL: add_round_key_in = shift_rows_out;
423         default: add_round_key_in = state_q;
424     endcase
425 end
426
427 for (genvar s = 0; s < NumShares; s++) begin :
gen_shares_add_round_key
428     assign add_round_key_out[s] = add_round_key_in[s] ^
round_key[s];
429 end
430
431 // Key
432 // Key
433 // Key
434
435 // SEC_CM: KEY.SEC_WIPE
436 // Full Key registers
437 always_comb begin : key_full_mux
438     unique case (key_full_sel)
439         KEY_FULL_ENC_INIT: key_full_d = key_init_i;
440         KEY_FULL_DEC_INIT: key_full_d = !CiphOpFwdOnly ?
key_dec_q : prd_clearing_key_i;
441         KEY_FULL_ROUND: key_full_d = key_expand_out;
442         KEY_FULL_CLEAR: key_full_d = prd_clearing_key_i;
443         default: key_full_d = prd_clearing_key_i;
444     endcase
445 end
446
447 always_ff @(posedge clk_i or negedge rst_ni) begin :
key_full_reg
448     if (!rst_ni) begin
449         key_full_q <= '{default: '0};
450     end else if (key_full_we == SP2V_HIGH) begin
451         key_full_q <= key_full_d;
452     end
453 end
454
455 if (!CiphOpFwdOnly) begin : gen_key_dec
456     // SEC_CM: KEY.SEC_WIPE
457     // Decryption Key registers
458     always_comb begin : key_dec_mux
459         unique case (key_dec_sel)
460             KEY_DEC_EXPAND: key_dec_d = key_expand_out;
461             KEY_DEC_CLEAR: key_dec_d = prd_clearing_key_i;
462             default: key_dec_d = prd_clearing_key_i;
463         endcase
464     end
465
466     always_ff @(posedge clk_i or negedge rst_ni) begin :
key_dec_reg
467         if (!rst_ni) begin
468             key_dec_q <= '{default: '0};
469         end else if (key_dec_we == SP2V_HIGH) begin
470             key_dec_q <= key_dec_d;
471         end
472     end
473 end else begin : gen_no_key_dec
474     // No Decryption Key registers
475     assign key_dec_q = '{default: '0};
476     assign key_dec_d = key_dec_q;
477
478     // Tie-off unused signals.
479     logic unused_key_dec;
480     always_comb begin
481         unused_key_dec = ^{key_dec_sel, key_dec_we};
482         for (int s = 0; s < NumShares; s++) begin
483             unused_key_dec ^= ^{key_dec_d[s]};
484         end
485     end
486 end
487
488 // Make sure that whenever the data/mask inputs of the S-
Boxes update, the internally buffered
489 // PRD is updated in sync.
490 assign key_expand_prd_we = (key_full_we == SP2V_HIGH) ?
1'b1 : 1'b0;
491
492 // Key expand data path
493 aes_key_expand #(
494     .AES192Enable ( AES192Enable ),
495     .SecMasking ( SecMasking ),
496     .SecSBoxImpl ( SecSBoxImpl )
497 ) u_aes_key_expand (
498     .clk_i      ( clk_i      ),
499     .rst_ni     ( rst_ni     ),
500     .cfg_valid_i ( cfg_valid ),
501     .op_i       ( key_expand_op ),
502     .en_i       ( key_expand_en ),
503     .prd_we_i   ( key_expand_prd_we ),
504     .out_req_o  ( key_expand_out_req ),
505     .out_ack_i  ( key_expand_out_ack ),
506     .clear_i    ( key_expand_clear ),
507     .round_i    ( key_expand_round ),
508     .key_len_i  ( key_len_i ),
509     .key_i      ( key_full_q ),
510     .key_o      ( key_expand_out ),
511     .prd_i      ( prd_key_expand ),
512     .err_o      ( key_expand_err )
513 );
514
515 for (genvar s = 0; s < NumShares; s++) begin :
gen_shares_round_key
516     always_comb begin : key_words_mux
517         unique case (key_words_sel)
518             KEY_WORDS_0123: key_words[s] = key_full_q[s][3:0];
519             KEY_WORDS_2345: key_words[s] = AES192Enable ?
key_full_q[s][5:2] : '0;
520             KEY_WORDS_4567: key_words[s] = key_full_q[s][7:4];
521             KEY_WORDS_ZERO: key_words[s] = '0;
522             default: key_words[s] = '0;
523         endcase
524     end
525
526     // Convert words to bytes (every key word contains one
column).
527     assign key_bytes[s] = aes_transpose(key_words[s]);
528
529     aes_mix_columns u_aes_key_mix_columns (
530         .op_i ( CIPH_INV ),
531         .data_i ( key_bytes[s] ),
532         .data_o ( key_mix_columns_out[s] )
533     );
534 end
535
536 always_comb begin : round_key_mux
537     unique case (round_key_sel)
538         ROUND_KEY_DIRECT: round_key = key_bytes;
539     endcase
540 end

```

```

539     ROUND_KEY_MIXED: round_key = !CiphOpFwdOnly ?
key_mix_columns_out : key_bytes;
540     default: round_key = key_bytes;
541     endcase
542 end
543
544 if (CiphOpFwdOnly) begin : gen_unused_key_mix_columns_out
545     // Tie-off unused signals.
546     logic unused_key_mix_columns_out;
547     always_comb begin
548         unused_key_mix_columns_out = 1'b0;
549         for (int s = 0; s < NumShares; s++) begin
550             unused_key_mix_columns_out ^=
551             ^{key_mix_columns_out[s]};
552         end
553     end
554 end
555
556 // Control
557 // Control
558
559 // Control
560 aes_cipher_control #(
561     .CiphOpFwdOnly ( CiphOpFwdOnly ),
562     .SecMasking ( SecMasking ),
563     .SecSBoxImpl ( SecSBoxImpl )
564 ) u_aes_cipher_control (
565     .clk_i ( clk_i ),
566     .rst_ni ( rst_ni ),
567
568     .in_valid_i ( in_valid_i ),
569     .in_ready_o ( in_ready_o ),
570
571     .out_valid_o ( out_valid_o ),
572     .out_ready_i ( out_ready_i ),
573
574     .cfg_valid_i ( cfg_valid ),
575     .op_i ( op_i ),
576     .key_len_i ( key_len_i ),
577     .crypt_i ( crypt_i ),
578     .crypt_o ( crypt_o ),
579     .dec_key_gen_i ( dec_key_gen_i ),
580     .dec_key_gen_o ( dec_key_gen_o ),
581     .prng_reseed_i ( prng_reseed_i ),
582     .prng_reseed_o ( prng_reseed_o ),
583     .key_clear_i ( key_clear_i ),
584     .key_clear_o ( key_clear_o ),
585     .data_out_clear_i ( data_out_clear_i ),
586     .data_out_clear_o ( data_out_clear_o ),
587     .mux_sel_err_i ( mux_sel_err ),
588     .sp_enc_err_i ( sp_enc_err_q ),
589     .op_err_i ( op_err ),
590     .alert_fatal_i ( alert_fatal_i ),
591     .alert_o ( alert_o ),
592
593     .prng_update_o ( prd_masking_upd ),
594     .prng_reseed_req_o ( prd_masking_rsd_req ),
595     .prng_reseed_ack_i ( prd_masking_rsd_ack ),
596
597     .state_sel_o ( state_sel_ctrl ),
598     .state_we_o ( state_we_ctrl ),
599     .sub_bytes_en_o ( sub_bytes_en ),
600     .sub_bytes_out_req_i ( sub_bytes_out_req ),
601     .sub_bytes_out_ack_o ( sub_bytes_out_ack ),
602     .add_rk_sel_o ( add_rk_sel_ctrl ),
603
604     .key_expand_op_o ( key_expand_op ),
605     .key_full_sel_o ( key_full_sel_ctrl ),
606     .key_full_we_o ( key_full_we_ctrl ),
607     .key_dec_sel_o ( key_dec_sel_ctrl ),
608     .key_dec_we_o ( key_dec_we_ctrl ),
609     .key_expand_en_o ( key_expand_en ),
610     .key_expand_out_req_i ( key_expand_out_req ),
611     .key_expand_out_ack_o ( key_expand_out_ack ),
612     .key_expand_clear_o ( key_expand_clear ),
613     .key_expand_round_o ( key_expand_round ),
614     .key_words_sel_o ( key_words_sel_ctrl ),
615     .round_key_sel_o ( round_key_sel_ctrl )
616 );
617
618 // Control
619 // Selectors
620 // Control
621
622 // We use sparse encodings for these mux selector signals
623 // and must ensure that:
624 // 1. The synthesis tool doesn't optimize away the sparse
625 // encoding.
626 // 2. The selector signal is always valid. More precisely,
627 // an alert or SVA is triggered if a
628 // selector signal takes on an invalid value.
629 // 3. The alert signal remains asserted until reset even if
630 // the selector signal becomes valid
631 // again. This is achieved by driving the control FSM
632 // into the terminal error state whenever
633 // any mux selector signal becomes invalid.
634 // If any mux selector signal becomes invalid, the cipher
635 // core further immediately de-asserts
636 // the out_valid_o signal to prevent any data from being re-

```

```

leased.
632
633 aes_sel_buf_chk #(
634     .Num ( StateSelNum ),
635     .Width ( StateSelWidth ),
636     .EnSecBuf ( 1'b1 )
637 ) u_aes_state_sel_buf_chk (
638     .clk_i ( clk_i ),
639     .rst_ni ( rst_ni ),
640     .sel_i ( state_sel_ctrl ),
641     .sel_o ( state_sel_raw ),
642     .err_o ( state_sel_err )
643 );
644 assign state_sel = state_sel_e'(state_sel_raw);
645
646 aes_sel_buf_chk #(
647     .Num ( AddRKSelNum ),
648     .Width ( AddRKSelWidth ),
649     .EnSecBuf ( 1'b1 )
650 ) u_aes_add_rk_sel_buf_chk (
651     .clk_i ( clk_i ),
652     .rst_ni ( rst_ni ),
653     .sel_i ( add_rk_sel_ctrl ),
654     .sel_o ( add_rk_sel_raw ),
655     .err_o ( add_rk_sel_err )
656 );
657 assign add_rk_sel = add_rk_sel_e'(add_rk_sel_raw);
658
659 aes_sel_buf_chk #(
660     .Num ( KeyFullSelNum ),
661     .Width ( KeyFullSelWidth ),
662     .EnSecBuf ( 1'b1 )
663 ) u_aes_key_full_sel_buf_chk (
664     .clk_i ( clk_i ),
665     .rst_ni ( rst_ni ),
666     .sel_i ( key_full_sel_ctrl ),
667     .sel_o ( key_full_sel_raw ),
668     .err_o ( key_full_sel_err )
669 );
670 assign key_full_sel = key_full_sel_e'(key_full_sel_raw);
671
672 aes_sel_buf_chk #(
673     .Num ( KeyDecSelNum ),
674     .Width ( KeyDecSelWidth ),
675     .EnSecBuf ( 1'b1 )
676 ) u_aes_key_dec_sel_buf_chk (
677     .clk_i ( clk_i ),
678     .rst_ni ( rst_ni ),
679     .sel_i ( key_dec_sel_ctrl ),
680     .sel_o ( key_dec_sel_raw ),
681     .err_o ( key_dec_sel_err )
682 );
683 assign key_dec_sel = key_dec_sel_e'(key_dec_sel_raw);
684
685 aes_sel_buf_chk #(
686     .Num ( KeyWordsSelNum ),
687     .Width ( KeyWordsSelWidth ),
688     .EnSecBuf ( 1'b1 )
689 ) u_aes_key_words_sel_buf_chk (
690     .clk_i ( clk_i ),
691     .rst_ni ( rst_ni ),
692     .sel_i ( key_words_sel_ctrl ),
693     .sel_o ( key_words_sel_raw ),
694     .err_o ( key_words_sel_err )
695 );
696 assign key_words_sel = key_words_sel_e'(key_words_sel_raw);
697
698 aes_sel_buf_chk #(
699     .Num ( RoundKeySelNum ),
700     .Width ( RoundKeySelWidth ),
701     .EnSecBuf ( 1'b1 )
702 ) u_aes_round_key_sel_buf_chk (
703     .clk_i ( clk_i ),
704     .rst_ni ( rst_ni ),
705     .sel_i ( round_key_sel_ctrl ),
706     .sel_o ( round_key_sel_raw ),
707     .err_o ( round_key_sel_err )
708 );
709 assign round_key_sel = round_key_sel_e'(round_key_sel_raw);
710
711 // Signal invalid mux selector signals to control FSM which
712 // will lock up and trigger an alert.
713 assign mux_sel_err = state_sel_err | add_rk_sel_err |
714 key_full_sel_err |
715 key_dec_sel_err | key_words_sel_err | round_key_sel_err;
716
717 // Sparse Encoded Signals
718
719 // We use sparse encodings for various critical signals and
720 // must ensure that:
721 // 1. The synthesis tool doesn't optimize away the sparse
722 // encoding.
723 // 2. The sparsely encoded signal is always valid. More pre-
724 // cisely, an alert or SVA is triggered
725 // if a sparse signal takes on an invalid value.
726 // 3. The alert signal remains asserted until reset even if
727 // the sparse signal becomes valid again
728 // This is achieved by driving the control FSM into the
729 // terminal error state whenever any

```

```

725 // sparsely encoded signal becomes invalid.
726 //
727 // If any sparsely encoded signal becomes invalid, the
cipher core further immediately de-asserts
728 // the out_valid_o signal to prevent any data from being re-
leased.
729

730 // We use vectors of sparsely encoded signals to reduce code
duplication.
731 localparam int unsigned NumSp2Vsig = 3;
732 sp2v_e [NumSp2Vsig-1:0] sp2v_sig;
733 sp2v_e [NumSp2Vsig-1:0] sp2v_sig_chk;
734 logic [NumSp2Vsig-1:0][Sp2Vwidth-1:0] sp2v_sig_chk_raw;
735 logic [NumSp2Vsig-1:0] sp2v_sig_err;
736
737 assign sp2v_sig[0] = state_we_ctrl;
738 assign sp2v_sig[1] = key_full_we_ctrl;
739 assign sp2v_sig[2] = key_dec_we_ctrl;
740
741 // All signals inside sp2v_sig are eventually converted to
single-rail signals.
742 localparam bit [NumSp2Vsig-1:0] Sp2VnSecBuf = {Num-
Sp2Vsig{1'b1}};
743
744 // Individually check sparsely encoded signals.
745 for (genvar i = 0; i < NumSp2Vsig; i++) begin :
gen_sel_buf_chk
746     aes_sel_buf_chk #(
747         .Num ( Sp2Vnum ),
748         .Width ( Sp2Vwidth ),
749         .EnSecBuf ( Sp2VnSecBuf[i] )
750     ) u_aes_sp2v_sig_buf_chk_i (
751         .clk_i ( clk_i ),
752         .rst_ni ( rst_ni ),
753         .sel_i ( sp2v_sig[i] ),
754         .sel_o ( sp2v_sig_chk_raw[i] ),
755         .err_o ( sp2v_sig_err[i] )
756     );
757     assign sp2v_sig_chk[i] = sp2v_e'(sp2v_sig_chk_raw[i]);
758 end
759
760 assign state_we = sp2v_sig_chk[0];
761 assign key_full_we = sp2v_sig_chk[1];
762 assign key_dec_we = sp2v_sig_chk[2];
763
764 // Collect encoding errors.
765 // We instantiate the checker modules as close as possible
to where the sparsely encoded signals
766 // are used. Here, we collect also encoding errors detected
in other places of the cipher core.
767 assign sp_enc_err_d = |sp2v_sig_err | sub_bytes_err |
key_expand_err;
768
769 // We need to register the collected error signal to avoid
circular loops in the cipher core
770
771 // controller related to out_valid_o and detecting errors in
state_we_o and sub_bytes_out_ack.
772 always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_sp_enc_err
773     if (!rst_ni) begin
774         sp_enc_err_q <= 1'b0;
775     end else if (sp_enc_err_d) begin
776         sp_enc_err_q <= 1'b1;
777     end
778 end
779
780 // Outputs //
781 //
782
783 // The output of the last round is not stored into the state
register but forwarded directly.
784 assign state_o = add_round_key_out;
785
786 //
787 // Assertions //
788 //
789
790 // Typically assertions already contain this macro, which en-
sures that assertions are only compiled
791 // in simulation and FPV. However, we wrap the entire asser-
tion section with CALIPTRA_INC_ASSERT so that the
792 // helper logic below is not synthesized either, since that
could cause issues in DC.
793 `ifdef CALIPTRA_INC_ASSERT
794 //VCS coverage off
795 // pragma coverage off
796
797 // Create a lint error to reduce the risk of accidentally
disabling the masking.
798 CALIPTRA_ASSERT_STATIC_LINT_ERROR(AesSecMaskingNonDefault,
SecMasking == 1)
799
800 // Cipher core masking requires a masked SBox and vice
versa.
801 `CALIPTRA_ASSERT_INIT(AesMaskedCoreAndSBox,
802 (SecMasking &&
803 (SecSBoxImpl == SBoxImplCanrightMasked ||

```

```

804 SecSBoxImpl == SBoxImplCanrightMaskedNoreuse ||
805 SecSBoxImpl == SBoxImplDom)) ||
806 (!SecMasking &&
807 (SecSBoxImpl == SBoxImplLut ||
808 SecSBoxImpl == SBoxImplCanright)))
809
810 // Signals used for assertions only.
811 logic prd_clearing_equals_output, un-
used_prd_clearing_equals_output;
812
813 assign prd_clearing_equals_output = (prd_clearing_state_i ==
add_round_key_out);
814 assign unused_prd_clearing_equals_output = prd_clear-
ing_equals_output;
815
816 // Ensure that the state register gets cleared with pseudo-
random data at the end of the last
817 // round. The following two scenarios are unlikely but not
illegal:
818 // 1. The newly loaded initial state matches the previous
output (the round counter is only
819 // cleared upon loading the new initial state).
820 // 2. The previous pseudo-random data is equal to the previ-
ous output.
821 // Otherwise, we must see an alert e.g. because the state
multiplexer got glitched.
822 `CALIPTRA_ASSERT(AesSecCmDataRegKeySca, (state_we ==
SP2V_HIGH) &&
823 ((key_len_i == AES_128 && u_aes_cipher_control.rnd_ctr
== 4'd10) ||
824 (key_len_i == AES_192 && u_aes_cipher_control.rnd_ctr
== 4'd12) ||
825 (key_len_i == AES_256 && u_aes_cipher_control.rnd_ctr
== 4'd14)) |>
826 (state_q != $past(add_round_key_out)) ||
827 (state_q == $past(state_init_i)) ||
828 $past(prd_clearing_equals_output) || alert_o)
829
830 if (SecMasking) begin : gen_sec_cm_key_masking_svas
831 // The number of clock cycles a regular AES round takes
- only used for assertions.
832
833 localparam int unsigned NumCyclesPerRound = (SecSBoxImpl
== SBoxImplDom) ? 5 : 1;
834 logic unused_param;
835 assign unused_param = (NumCyclesPerRound == 1);
836 // Ensure that SubBytes gets fresh PRD input for every
evaluation unless mask forcing is
837 // enabled. We effectively check that the PRNG has been
updated at least once within the
838 // last NumCyclesPerRound cycles. This also holds for
the very first round, as the PRNG
839 // is always updated in the last cycle of the IDLE state
and/or the first cycle of the
840 // INIT state.
841 `CALIPTRA_ASSERT(AesSecCmKeyMaskingPrdSubBytes,
842 sub_bytes_en == SP2V_HIGH && ($past(sub_bytes_en) ==
SP2V_LOW ||
843 ($past(sub_bytes_out_req) == SP2V_HIGH &&
844 $past(sub_bytes_out_ack) == SP2V_HIGH)) |>
845 $past(prd_sub_bytes_q) != $past(prd_sub_bytes_q,
NumCyclesPerRound + 1) ||
846 SecAllowForcingMasks && force_masks_i)
847
848 // Ensure that the PRNG has been updated between masking
the input and starting the first
849 // SubBytes evaluation/KeyExpand operation unless mask
forcing is enabled. For AES-256,
850
851 // we just spend 1 cycle in the INIT state and KeyExpand
isn't evaluating its S-Boxes,
852 // i.e., no fresh randomness is required. For the other
key lengths, KeyExpand evaluates
853 // its S-Boxes which takes NumCyclesPerRound cycles.
When computing the start key for
854 // decryption, the input isn't loaded and the PRNG is
thus not advanced.
855 `CALIPTRA-
RA_ASSERT(AesSecCmKeyMaskingInitialPrngUpdateSubBytes,
856 sub_bytes_en == SP2V_HIGH && $past(sub_bytes_en) ==
SP2V_LOW |>
857 (key_len_i == AES_256 &&
858 $past(prd_masking) != $past(prd_masking, 3)) ||
859 ((key_len_i == AES_128 || key_len_i == AES_192) &&
860 $past(prd_masking) != $past(prd_masking, Num-
CyclesPerRound + 2)) ||
861 (SecAllowForcingMasks && force_masks_i))
862 `CALIPTRA-
RA_ASSERT(AesSecCmKeyMaskingInitialPrngUpdateKeyExpand,
863 key_expand_en == SP2V_HIGH && $past(key_expand_en)
== SP2V_LOW |>
864 (key_len_i == AES_256 &&
865 $past(prd_masking) != $past(prd_masking, 3)) ||
866 ((key_len_i == AES_128 || key_len_i == AES_192) &&
867 $past(prd_masking) != $past(prd_masking, 2)) ||
868 (SecAllowForcingMasks && force_masks_i) ||
dec_key_gen_o == SP2V_HIGH)
869
870

```

```

867 // Ensure none of the state shares keeps being constant
during encryption/decryption
868 // unless mask forcing is enabled. Even though unlikely
it's not impossible that one

869 // share remains constant throughout one round. The SVAs
thus only fire if a share
870 // remains constant across two rounds.
871 for (genvar s = 0; s < NumShares; s++) begin :
gen_sec_cm_key_masking_share_svas
872     CALIPTRA_ASSERT(AesSecCmKeyMaskingStateShare,
state_we == SP2V_HIGH &&
873         (crypt_i == SP2V_HIGH || crypt_o == SP2V_HIGH) | =>
874             state_q[s] != $past(state_q[s], NumCyclesPerRound)
||
875             $past(state_q[s], NumCyclesPerRound) !=
$past(state_q[s], 2*NumCyclesPerRound) ||
876             (SecAllowForcingMasks && force_masks_i) ||
dec_key_gen_o == SP2V_HIGH)
877     `CALIPTRA_ASSERT(AesSecCmKeyMaskingOutputShare,
878         (out_valid_o == SP2V_HIGH && $past(out_valid_o) ==
SP2V_LOW) &&
879         (crypt_o == SP2V_HIGH) | =>
880             $past(state_o[s]) != $past(state_q[s], NumCycles-
PerRound) ||
881             $past(state_q[s], NumCyclesPerRound) !=
$past(state_q[s], 2*NumCyclesPerRound) ||
882             (SecAllowForcingMasks && force_masks_i) ||
dec_key_gen_o == SP2V_HIGH)
883     end
884 end
885
886 // Make sure the output of the masking PRNG is properly ex-
tracted without creating overlaps
887 // in the data input masks, or between the PRD fed to the
key expand module and SubBytes.
888 if (WidthPRDSBox > 8) begin : gen_prd_extract_assert
889     // For one row of the state matrix, extract the WidthPRDS-
Box-8 MSBs of the per-S-Box PRD from
890     // the PRNG output.
891     function automatic logic [3:0][(WidthPRDSBox-8)-1:0]
aes_prd_get_msbs(
892         logic [(4*WidthPRDSBox)-1:0] in
893     );
894     logic [3:0][(WidthPRDSBox-8)-1:0] prd_msbs;
895     for (int i = 0; i < 4; i++) begin
896         prd_msbs[i] = in[(i*WidthPRDSBox) + 8 +: (WidthPRDS-
Box-8)];
897     end
898     return prd_msbs;
899 endfunction
900
901 // For one row of the state matrix, undo the extraction of
LSBs and MSBs of the per-S-Box PRD
902 // from the PRNG output. This can be used to verify proper
extraction (no overlap of output
903 // masks and PRD for masked Canright S-Box implementa-
tions, no unused PRNG output).
904 function automatic logic [4*WidthPRDSBox-1:0] aes_prd_con-
cat_bits(
905     logic [3:0] [7:0] prd_lsb,
906     logic [3:0][(WidthPRDSBox-8)-1:0] prd_msbs
907 );
908     logic [(4*WidthPRDSBox)-1:0] prd;
909     for (int i = 0; i < 4; i++) begin
910         prd[(i*WidthPRDSBox) +: WidthPRDSBox] = {prd_msbs[i],
prd_lsb[i]};
911     end
912     return prd;
913 endfunction
914
915 // Check for correct extraction of masking PRNG output
without overlaps.
916 logic [WidthPRDMasking-1:0] unused_prd_masking;
917 logic [3:0][3:0][(WidthPRDSBox-8)-1:0] unused_prd_msbs;
918 for (genvar i = 0; i < 4; i++) begin : gen_unused_prd_msbs
919     assign unused_prd_msbs[i] =
aes_prd_get_msbs(prd_masking[i * WidthPRDRow +: WidthPRDRow]);
920 end
921 for (genvar i = 0; i < 4; i++) begin : gen_un-
used_prd_masking
922     assign unused_prd_masking[i * WidthPRDRow +: Width-
PRDRow] =
923         aes_prd_concat_bits(data_in_mask[i], un-
used_prd_msbs[i]);
924 end
925 assign unused_prd_masking[WidthPRDMasking-1 -: Width-
PRDKey] = prd_key_expand;
926 CALIPTRA_ASSERT(AesMskgPrdExtraction, prd_masking == un-
used_prd_masking)
927 end
928 //VCS coverage on
929 // pragma coverage on
930 `endif
931
932 endmodule

```

7. chipsalliance_caliptra-rtl/src/aes/rtl/aes_clp_reg.sv (1,929 lines)

```

1 // Generated by PeakRDL-regblock - A free and open-source Sys-
temVerilog generator
2 // https://github.com/SystemRDL/PeakRDL-regblock
3
4 module aes_clp_reg (
5     input wire clk,
6     input wire rst,
7
8     input wire s_cpuif_req,
9     input wire s_cpuif_req_is_wr,
10    input wire [10:0] s_cpuif_addr,
11    input wire [31:0] s_cpuif_wr_data,
12    input wire [31:0] s_cpuif_wr_biten,
13    output wire s_cpuif_req_stall_wr,
14    output wire s_cpuif_req_stall_rd,
15    output wire s_cpuif_rd_ack,
16    output wire s_cpuif_rd_err,
17    output wire [31:0] s_cpuif_rd_data,
18    output wire s_cpuif_wr_ack,
19    output wire s_cpuif_wr_err,
20
21    input aes_clp_reg_pkg::aes_clp_reg__in_t hwif_in,
22    output aes_clp_reg_pkg::aes_clp_reg__out_t hwif_out
23);
24
25 //-----
26 // CPU Bus interface logic
27 //-----
28
29 logic cpuif_req;
30 logic cpuif_req_is_wr;
31 logic [10:0] cpuif_addr;
32 logic [31:0] cpuif_wr_data;
33 logic [31:0] cpuif_wr_biten;
34 logic cpuif_req_stall_wr;
35 logic cpuif_req_stall_rd;
36
37 logic cpuif_rd_ack;
38 logic cpuif_rd_err;
39 logic [31:0] cpuif_rd_data;
40
41 logic cpuif_wr_ack;
42 logic cpuif_wr_err;
43
44 assign cpuif_req = s_cpuif_req;
45 assign cpuif_req_is_wr = s_cpuif_req_is_wr;
46 assign cpuif_addr = s_cpuif_addr;
47 assign cpuif_wr_data = s_cpuif_wr_data;
48 assign cpuif_wr_biten = s_cpuif_wr_biten;
49 assign s_cpuif_req_stall_wr = cpuif_req_stall_wr;
50 assign s_cpuif_req_stall_rd = cpuif_req_stall_rd;
51 assign s_cpuif_rd_ack = cpuif_rd_ack;
52 assign s_cpuif_rd_err = cpuif_rd_err;
53 assign s_cpuif_rd_data = cpuif_rd_data;
54 assign s_cpuif_wr_ack = cpuif_wr_ack;
55 assign s_cpuif_wr_err = cpuif_wr_err;
56
57 logic cpuif_req_masked;
58
59 // Read & write latencies are balanced. Stalls not re-
quired
60 assign cpuif_req_stall_rd = '0;
61 assign cpuif_req_stall_wr = '0;
62 assign cpuif_req_masked = cpuif_req
63     & !(cpuif_req_is_wr &
64         & !(cpuif_req_is_wr &
65             & !(cpuif_req_is_wr &
66                 & !(cpuif_req_is_wr &
67                     & !(cpuif_req_is_wr &
68                         & !(cpuif_req_is_wr &
69                             & !(cpuif_req_is_wr &
70                                 & !(cpuif_req_is_wr &
71                                     & !(cpuif_req_is_wr &
72                                         & !(cpuif_req_is_wr &
73                                             & !(cpuif_req_is_wr &
74                                                 & !(cpuif_req_is_wr &
75                                                     & !(cpuif_req_is_wr &
76                                                         & !(cpuif_req_is_wr &
77                                                             & !(cpuif_req_is_wr &
78                                                                 & !(cpuif_req_is_wr &
79                                                                     & !(cpuif_req_is_wr &
80                                                                         & !(cpuif_req_is_wr &
81                                                                             & !(cpuif_req_is_wr &
82                                                                                 & !(cpuif_req_is_wr &
83                                                                                     & !(cpuif_req_is_wr &
84                                                                                         & !(cpuif_req_is_wr &
85                                                                                             & !(cpuif_req_is_wr &
86                                                                                                 & !(cpuif_req_is_wr &
87                                                                                                     & !(cpuif_req_is_wr &
88                                                                                                         & !(cpuif_req_is_wr &

```

```

89         logic error2_intr_count_r;
90         logic error3_intr_count_r;
91         logic notif_cmd_done_intr_count_r;
92         logic error0_intr_count_incr_r;
93         logic error1_intr_count_incr_r;
94         logic error2_intr_count_incr_r;
95         logic error3_intr_count_incr_r;
96         logic notif_cmd_done_intr_count_incr_r;
97     } intr_block_rf;
98 } decoded_reg_strb_t;
99 decoded_reg_strb_t decoded_reg_strb;
100 logic decoded_req;
101 logic decoded_req_is_wr;
102 logic [31:0] decoded_wr_data;
103 logic [31:0] decoded_wr_biten;
104
105 always_comb begin
106     for(int i0=0; i0<2; i0++) begin
107         decoded_reg_strb.AES_NAME[i0] = cpuif_req_masked &
(cpuif_addr == 11'h0 + i0*11'h4);
108     end
109     for(int i0=0; i0<2; i0++) begin
110         decoded_reg_strb.AES_VERSION[i0] =
cpuif_req_masked & (cpuif_addr == 11'h8 + i0*11'h4);
111     end
112     for(int i0=0; i0<9; i0++) begin
113         decoded_reg_strb.ENTROPY_IF_SEED[i0] =
cpuif_req_masked & (cpuif_addr == 11'h10 + i0*11'h4);
114     end
115     decoded_reg_strb.CTRL0 = cpuif_req_masked &
(cpuif_addr == 11'h134);
116     decoded_reg_strb.AES_KV_RD_KEY_CTRL = cpuif_req_masked &
(cpuif_addr == 11'h200);
117     decoded_reg_strb.AES_KV_RD_KEY_STATUS =
cpuif_req_masked & (cpuif_addr == 11'h204);
118     decoded_reg_strb.AES_KV_WR_CTRL = cpuif_req_masked &
(cpuif_addr == 11'h208);
119     decoded_reg_strb.AES_KV_WR_STATUS = cpuif_req_masked &
(cpuif_addr == 11'h20c);
120     decoded_reg_strb.intr_block_rf.global_intr_en_r =
cpuif_req_masked & (cpuif_addr == 11'h400);
121     decoded_reg_strb.intr_block_rf.error_intr_en_r =
cpuif_req_masked & (cpuif_addr == 11'h404);
122     decoded_reg_strb.intr_block_rf.notif_intr_en_r =
cpuif_req_masked & (cpuif_addr == 11'h408);
123     decoded_reg_strb.intr_block_rf.error_global_intr_r =
cpuif_req_masked & (cpuif_addr == 11'h40c);
124     decoded_reg_strb.intr_block_rf.notif_global_intr_r =
cpuif_req_masked & (cpuif_addr == 11'h410);
125     decoded_reg_strb.intr_block_rf.error_internal_intr_r =
cpuif_req_masked & (cpuif_addr == 11'h414);
126     decoded_reg_strb.intr_block_rf.notif_internal_intr_r =
cpuif_req_masked & (cpuif_addr == 11'h418);
127     decoded_reg_strb.intr_block_rf.error_intr_trig_r =
cpuif_req_masked & (cpuif_addr == 11'h41c);
128     decoded_reg_strb.intr_block_rf.notif_intr_trig_r =
cpuif_req_masked & (cpuif_addr == 11'h420);
129     decoded_reg_strb.intr_block_rf.error0_intr_count_r =
cpuif_req_masked & (cpuif_addr == 11'h500);
130     decoded_reg_strb.intr_block_rf.error1_intr_count_r =
cpuif_req_masked & (cpuif_addr == 11'h504);
131     decoded_reg_strb.intr_block_rf.error2_intr_count_r =
cpuif_req_masked & (cpuif_addr == 11'h508);
132     decoded_reg_strb.intr_block_rf.error3_intr_count_r =
cpuif_req_masked & (cpuif_addr == 11'h50c);
133     de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r =
cpuif_req_masked & (cpuif_addr == 11'h580);
134     de-
coded_reg_strb.intr_block_rf.error0_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 11'h600);
135     de-
coded_reg_strb.intr_block_rf.error1_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 11'h604);
136     de-
coded_reg_strb.intr_block_rf.error2_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 11'h608);
137     de-
coded_reg_strb.intr_block_rf.error3_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 11'h60c);
138     de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 11'h610);
139     end
140
141     // Pass down signals to next stage
142     assign decoded_req = cpuif_req_masked;
143     assign decoded_req_is_wr = cpuif_req_is_wr;
144     assign decoded_wr_data = cpuif_wr_data;
145     assign decoded_wr_biten = cpuif_wr_biten;
146
147     //-----
148     // Field logic
149     //-----

```

```

150 typedef struct packed{
151     struct packed{
152         struct packed{
153             logic [31:0] next;
154             logic load_next;
155         } ENTROPY_IF_SEED;
156     } [9:1:0]ENTROPY_IF_SEED;
157     struct packed{
158         struct packed{
159             logic next;
160             logic load_next;
161         } ENDIAN_SWAP;
162     } CTRL0;
163     struct packed{
164         struct packed{
165             logic next;
166             logic load_next;
167         } read_en;
168         struct packed{
169             logic [4:0] next;
170             logic load_next;
171         } read_entry;
172         struct packed{
173             logic next;
174             logic load_next;
175         } pcr_hash_extend;
176         struct packed{
177             logic [24:0] next;
178             logic load_next;
179         } rsvd;
180     } AES_KV_RD_KEY_CTRL;
181     struct packed{
182         struct packed{
183             logic next;
184             logic load_next;
185         } VALID;
186     } AES_KV_RD_KEY_STATUS;
187     struct packed{
188         struct packed{
189             logic next;
190             logic load_next;
191         } write_en;
192         struct packed{
193             logic [4:0] next;
194             logic load_next;
195         } write_entry;
196         struct packed{
197             logic next;
198             logic load_next;
199         } hmac_key_dest_valid;
200         struct packed{
201             logic next;
202             logic load_next;
203         } hmac_block_dest_valid;
204         struct packed{
205             logic next;
206             logic load_next;
207         } mldsa_seed_dest_valid;
208         struct packed{
209             logic next;
210             logic load_next;
211         } ecc_pkey_dest_valid;
212         struct packed{
213             logic next;
214             logic load_next;
215         } ecc_seed_dest_valid;
216         struct packed{
217             logic next;
218             logic load_next;
219         } aes_key_dest_valid;
220         struct packed{
221             logic next;
222             logic load_next;
223         } mlkem_seed_dest_valid;
224         struct packed{
225             logic next;
226             logic load_next;
227         } mlkem_msg_dest_valid;
228         struct packed{
229             logic next;
230             logic load_next;
231         } dma_data_dest_valid;
232         struct packed{
233             logic [16:0] next;
234             logic load_next;
235         } rsvd;
236     } AES_KV_WR_CTRL;
237     struct packed{
238         struct packed{
239             logic next;
240             logic load_next;
241         } VALID;
242     } AES_KV_WR_STATUS;
243     struct packed{
244         struct packed{
245             struct packed{
246                 logic next;
247                 logic load_next;
248             } error_en;
249         } struct packed{
250             logic next;

```



```

251         logic load_next;
252     } notif_en;
253 } global_intr_en_r;
254 struct packed{
255     struct packed{
256         logic next;
257         logic load_next;
258     } error0_en;
259     struct packed{
260         logic next;
261         logic load_next;
262     } error1_en;
263     struct packed{
264         logic next;
265         logic load_next;
266     } error2_en;
267     struct packed{
268         logic next;
269         logic load_next;
270     } error3_en;
271 } error_intr_en_r;
272 struct packed{
273     struct packed{
274         logic next;
275         logic load_next;
276     } notif_cmd_done_en;
277 } notif_intr_en_r;
278 struct packed{
279     struct packed{
280         logic next;
281         logic load_next;
282     } agg_sts;
283 } error_global_intr_r;
284 struct packed{
285     struct packed{
286         logic next;
287         logic load_next;
288     } agg_sts;
289 } notif_global_intr_r;
290 struct packed{
291     struct packed{
292         logic next;
293         logic load_next;
294     } error0_sts;
295     struct packed{
296         logic next;
297         logic load_next;
298     } error1_sts;
299     struct packed{
300         logic next;
301         logic load_next;
302     } error2_sts;
303     struct packed{
304         logic next;
305         logic load_next;
306     } error3_sts;
307 } error_internal_intr_r;
308 struct packed{
309     struct packed{
310         logic next;
311         logic load_next;
312     } notif_cmd_done_sts;
313 } notif_internal_intr_r;
314 struct packed{
315     struct packed{
316         logic next;
317         logic load_next;
318     } error0_trig;
319     struct packed{
320         logic next;
321         logic load_next;
322     } error1_trig;
323     struct packed{
324         logic next;
325         logic load_next;
326     } error2_trig;
327     struct packed{
328         logic next;
329         logic load_next;
330     } error3_trig;
331 } error_intr_trig_r;
332 struct packed{
333     struct packed{
334         logic next;
335         logic load_next;
336     } notif_cmd_done_trig;
337 } notif_intr_trig_r;
338 struct packed{
339     struct packed{
340         logic [31:0] next;
341         logic load_next;
342         logic incrtthreshold;
343         logic incrsaturate;
344     } cnt;
345 } error0_intr_count_r;
346 struct packed{
347     struct packed{
348         logic [31:0] next;
349         logic load_next;
350         logic incrtthreshold;
351         logic incrsaturate;
352     } cnt;
353 } error1_intr_count_r;
354 struct packed{
355     struct packed{
356         logic [31:0] next;
357         logic load_next;
358         logic incrtthreshold;
359         logic incrsaturate;
360     } cnt;
361 } error2_intr_count_r;
362 struct packed{
363     struct packed{
364         logic [31:0] next;
365         logic load_next;
366         logic incrtthreshold;
367         logic incrsaturate;
368     } cnt;
369 } error3_intr_count_r;
370 struct packed{
371     struct packed{
372         logic [31:0] next;
373         logic load_next;
374         logic incrtthreshold;
375         logic incrsaturate;
376     } cnt;
377 } notif_cmd_done_intr_count_r;
378 struct packed{
379     struct packed{
380         logic next;
381         logic load_next;
382         logic decrtthreshold;
383         logic underflow;
384     } pulse;
385 } error0_intr_count_incr_r;
386 struct packed{
387     struct packed{
388         logic next;
389         logic load_next;
390         logic decrtthreshold;
391         logic underflow;
392     } pulse;
393 } error1_intr_count_incr_r;
394 struct packed{
395     struct packed{
396         logic next;
397         logic load_next;
398         logic decrtthreshold;
399         logic underflow;
400     } pulse;
401 } error2_intr_count_incr_r;
402 struct packed{
403     struct packed{
404         logic next;
405         logic load_next;
406         logic decrtthreshold;
407         logic underflow;
408     } pulse;
409 } error3_intr_count_incr_r;
410 struct packed{
411     struct packed{
412         logic next;
413         logic load_next;
414         logic decrtthreshold;
415         logic underflow;
416     } pulse;
417 } notif_cmd_done_intr_count_incr_r;
418 } intr_block_rf;
419 } field_combo_t;
420 field_combo_t field_combo;
421
422 typedef struct packed{
423     struct packed{
424         struct packed{
425             logic [31:0] value;
426         } ENTROPY_IF_SEED;
427     } [9:1:0]ENTROPY_IF_SEED;
428     struct packed{
429         struct packed{
430             logic value;
431         } ENDIAN_SWAP;
432     } CTRL0;
433     struct packed{
434         struct packed{
435             logic value;
436         } read_en;
437     } struct packed{
438         logic [4:0] value;
439     } read_entry;
440     struct packed{
441         logic value;
442     } pcr_hash_extend;
443     struct packed{
444         logic [24:0] value;
445     } rsvd;
446 } AES_KV_RD_KEY_CTRL;
447 struct packed{
448     struct packed{
449         logic value;
450     } VALID;
451 } AES_KV_RD_KEY_STATUS;
452 struct packed{

```

```

453     struct packed{
454         logic value;
455     } write_en;
456     struct packed{
457         logic [4:0] value;
458     } write_entry;
459     struct packed{
460         logic value;
461     } hmac_key_dest_valid;
462     struct packed{
463         logic value;
464     } hmac_block_dest_valid;
465     struct packed{
466         logic value;
467     } mldsa_seed_dest_valid;
468     struct packed{
469         logic value;
470     } ecc_pkey_dest_valid;
471     struct packed{
472         logic value;
473     } ecc_seed_dest_valid;
474     struct packed{
475         logic value;
476     } aes_key_dest_valid;
477     struct packed{
478         logic value;
479     } mlkem_seed_dest_valid;
480     struct packed{
481         logic value;
482     } mlkem_msg_dest_valid;
483     struct packed{
484         logic value;
485     } dma_data_dest_valid;
486     struct packed{
487         logic [16:0] value;
488     } rsvd;
489 } AES_KV_WR_CTRL;
490 struct packed{
491     struct packed{
492         logic value;
493     } VALID;
494 } AES_KV_WR_STATUS;
495 struct packed{
496     struct packed{
497         struct packed{
498             logic value;
499         } error_en;
500         struct packed{
501             logic value;
502         } notif_en;
503     } global_intr_en_r;
504     struct packed{
505         struct packed{
506             logic value;
507         } error0_en;
508         struct packed{
509             logic value;
510         } error1_en;
511         struct packed{
512             logic value;
513         } error2_en;
514         struct packed{
515             logic value;
516         } error3_en;
517     } error_intr_en_r;
518     struct packed{
519         struct packed{
520             logic value;
521         } notif_cmd_done_en;
522     } notif_intr_en_r;
523     struct packed{
524         struct packed{
525             logic value;
526         } agg_sts;
527     } error_global_intr_r;
528     struct packed{
529         struct packed{
530             logic value;
531         } agg_sts;
532     } notif_global_intr_r;
533     struct packed{
534         struct packed{
535             logic value;
536         } error0_sts;
537         struct packed{
538             logic value;
539         } error1_sts;
540         struct packed{
541             logic value;
542         } error2_sts;
543         struct packed{
544             logic value;
545         } error3_sts;
546     } error_internal_intr_r;
547     struct packed{
548         struct packed{
549             logic value;
550         } notif_cmd_done_sts;
551     } notif_internal_intr_r;
552     struct packed{
553         struct packed{
554             logic value;
555         } error0_trig;
556         struct packed{
557             logic value;
558         } error1_trig;
559         struct packed{
560             logic value;
561         } error2_trig;
562         struct packed{
563             logic value;
564         } error3_trig;
565     } error_intr_trig_r;
566     struct packed{
567         struct packed{
568             logic value;
569         } notif_cmd_done_trig;
570     } notif_intr_trig_r;
571     struct packed{
572         struct packed{
573             logic [31:0] value;
574         } cnt;
575     } error0_intr_count_r;
576     struct packed{
577         struct packed{
578             logic [31:0] value;
579         } cnt;
580     } error1_intr_count_r;
581     struct packed{
582         struct packed{
583             logic [31:0] value;
584         } cnt;
585     } error2_intr_count_r;
586     struct packed{
587         struct packed{
588             logic [31:0] value;
589         } cnt;
590     } error3_intr_count_r;
591     struct packed{
592         struct packed{
593             logic [31:0] value;
594         } cnt;
595     } notif_cmd_done_intr_count_r;
596     struct packed{
597         struct packed{
598             logic value;
599         } pulse;
600     } error0_intr_count_incr_r;
601     struct packed{
602         struct packed{
603             logic value;
604         } pulse;
605     } error1_intr_count_incr_r;
606     struct packed{
607         struct packed{
608             logic value;
609         } pulse;
610     } error2_intr_count_incr_r;
611     struct packed{
612         struct packed{
613             logic value;
614         } pulse;
615     } error3_intr_count_incr_r;
616     struct packed{
617         struct packed{
618             logic value;
619         } pulse;
620     } notif_cmd_done_intr_count_incr_r;
621     } intr_block_rf;
622     } field_storage_t;
623     field_storage_t field_storage;
624
625     for(genvar i0=0; i0<9; i0++) begin
626         // Field: aes_clp_reg.EN-
627         TROPY_IF_SEED[i].ENTROPY_IF_SEED
628         always_comb begin
629             automatic logic [31:0] next_c;
630             automatic logic load_next_c;
631             next_c =
632             field_storage.ENTROPY_IF_SEED[i].ENTROPY_IF_SEED.value;
633             load_next_c = '0;
634             if(decoded_reg_strb.ENTROPY_IF_SEED[i] && de-
635             coded_req_is_wr) begin // SW write
636                 next_c =
637                 (field_storage.ENTROPY_IF_SEED[i].ENTROPY_IF_SEED.value & ~de-
638                 coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
639                 coded_wr_biten[31:0]);
640                 load_next_c = '1;
641             end
642             field_combo.ENTROPY_IF_SEED[i].ENTROPY_IF_SEED.next = next_c;
643             field_combo.ENTROPY_IF_SEED[i].ENTROPY_IF_SEED.load_next =
644             load_next_c;
645         end
646     end
647     always_ff @(posedge clk) begin
648         if(field_combo.ENTROPY_IF_SEED[i0].ENTROPY_IF_SEED.load_next) begin
649             field_storage.ENTROPY_IF_SEED[i0].ENTROPY_IF_SEED.value <=
650             field_combo.ENTROPY_IF_SEED[i0].ENTROPY_IF_SEED.next;

```



```

643     end
644   end
645   assign hwif_out.EN-
TROPY_IF_SEED[i0].ENTROPY_IF_SEED.value =
field_storage.ENTROPY_IF_SEED[i0].ENTROPY_IF_SEED.value;
646   assign hwif_out.EN-
TROPY_IF_SEED[i0].ENTROPY_IF_SEED.swmod = de-
coded_reg_strb.ENTROPY_IF_SEED[i0] && decoded_req_is_wr;
647   end
648   // Field: aes_clp_reg.CTRL0.ENDIAN_SWAP
649   always_comb begin
650     automatic logic [0:0] next_c;
651     automatic logic load_next_c;
652     next_c = field_storage.CTRL0.ENDIAN_SWAP.value;
653     load_next_c = '0;
654     if(decoded_reg_strb.CTRL0 && decoded_req_is_wr) be-
gin // SW write
655       next_c = (field_storage.CTRL0.ENDIAN_SWAP.value &
~decoded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
656       load_next_c = '1;
657     end
658     field_combo.CTRL0.ENDIAN_SWAP.next = next_c;
659     field_combo.CTRL0.ENDIAN_SWAP.load_next = load_next_c;
660   end
661   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
662     if(~hwif_in.reset_b) begin
663       field_storage.CTRL0.ENDIAN_SWAP.value <= 1'h0;
664     end else if(field_combo.CTRL0.ENDIAN_SWAP.load_next)
begin
665       field_storage.CTRL0.ENDIAN_SWAP.value <=
field_combo.CTRL0.ENDIAN_SWAP.next;
666     end
667   end
668   assign hwif_out.CTRL0.ENDIAN_SWAP.value =
field_storage.CTRL0.ENDIAN_SWAP.value;
669   // Field: aes_clp_reg.AES_KV_RD_KEY_CTRL.read_en
670   always_comb begin
671     automatic logic [0:0] next_c;
672     automatic logic load_next_c;
673     next_c =
field_storage.AES_KV_RD_KEY_CTRL.read_en.value;
674     load_next_c = '0;
675     if(decoded_reg_strb.AES_KV_RD_KEY_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_RD_KEY_CTRL.read_en.swwe) begin //
SW write
676       next_c =
(field_storage.AES_KV_RD_KEY_CTRL.read_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
677       load_next_c = '1;
678     end else if(hwif_in.AES_KV_RD_KEY_CTRL.read_en.hwclr)
begin // HW Clear
679       next_c = '0;
680       load_next_c = '1;
681     end
682     field_combo.AES_KV_RD_KEY_CTRL.read_en.next = next_c;
683     field_combo.AES_KV_RD_KEY_CTRL.read_en.load_next =
load_next_c;
684   end
685   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
686     if(~hwif_in.reset_b) begin
687       field_storage.AES_KV_RD_KEY_CTRL.read_en.value <=
1'h0;
688     end else
if(field_combo.AES_KV_RD_KEY_CTRL.read_en.load_next) begin
689       field_storage.AES_KV_RD_KEY_CTRL.read_en.value <=
field_combo.AES_KV_RD_KEY_CTRL.read_en.next;
690     end
691   end
692   assign hwif_out.AES_KV_RD_KEY_CTRL.read_en.value =
field_storage.AES_KV_RD_KEY_CTRL.read_en.value;
693   // Field: aes_clp_reg.AES_KV_RD_KEY_CTRL.read_entry
694   always_comb begin
695     automatic logic [4:0] next_c;
696     automatic logic load_next_c;
697     next_c =
field_storage.AES_KV_RD_KEY_CTRL.read_entry.value;
698     load_next_c = '0;
699     if(decoded_reg_strb.AES_KV_RD_KEY_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_RD_KEY_CTRL.read_entry.swwe) be-
gin // SW write
700       next_c =
(field_storage.AES_KV_RD_KEY_CTRL.read_entry.value & ~de-
coded_wr_biten[5:1]) | (decoded_wr_data[5:1] & de-
coded_wr_biten[5:1]);
701       load_next_c = '1;
702     end
703     field_combo.AES_KV_RD_KEY_CTRL.read_entry.next =
next_c;
704     field_combo.AES_KV_RD_KEY_CTRL.read_entry.load_next =
load_next_c;
705   end
706   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
707     if(~hwif_in.reset_b) begin
708       field_storage.AES_KV_RD_KEY_CTRL.read_entry.value
<= 5'h0;
709     end else
if(field_combo.AES_KV_RD_KEY_CTRL.read_entry.load_next) begin
710       field_storage.AES_KV_RD_KEY_CTRL.read_entry.value
<= field_combo.AES_KV_RD_KEY_CTRL.read_entry.next;

```

```

711     end
712   end
713   assign hwif_out.AES_KV_RD_KEY_CTRL.read_entry.value =
field_storage.AES_KV_RD_KEY_CTRL.read_entry.value;
714   // Field: aes_clp_reg.AES_KV_RD_KEY_CTRL.pcr_hash_extend
715   always_comb begin
716     automatic logic [0:0] next_c;
717     automatic logic load_next_c;
718     next_c =
field_storage.AES_KV_RD_KEY_CTRL.pcr_hash_extend.value;
719     load_next_c = '0;
720     if(decoded_reg_strb.AES_KV_RD_KEY_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_RD_KEY_CTRL.pcr_hash_extend.swwe)
begin // SW write
721       next_c =
(field_storage.AES_KV_RD_KEY_CTRL.pcr_hash_extend.value & ~de-
coded_wr_biten[6:6]) | (decoded_wr_data[6:6] & de-
coded_wr_biten[6:6]);
722       load_next_c = '1;
723     end
724     field_combo.AES_KV_RD_KEY_CTRL.pcr_hash_extend.next =
next_c;
725     field_combo.AES_KV_RD_KEY_CTRL.pcr_hash_extend.load_next =
load_next_c;
726   end
727   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
728     if(~hwif_in.reset_b) begin
729       field_storage.AES_KV_RD_KEY_CTRL.pcr_hash_extend.value <= 1'h0;
730     end else
if(field_combo.AES_KV_RD_KEY_CTRL.pcr_hash_extend.load_next) begin
731       field_storage.AES_KV_RD_KEY_CTRL.pcr_hash_extend.value <=
field_combo.AES_KV_RD_KEY_CTRL.pcr_hash_extend.next;
732     end
733   end
734   assign hwif_out.AES_KV_RD_KEY_CTRL.pcr_hash_extend.value =
field_storage.AES_KV_RD_KEY_CTRL.pcr_hash_extend.value;
735   // Field: aes_clp_reg.AES_KV_RD_KEY_CTRL.rsvd
736   always_comb begin
737     automatic logic [24:0] next_c;
738     automatic logic load_next_c;
739     next_c = field_storage.AES_KV_RD_KEY_CTRL.rsvd.value;
740     load_next_c = '0;
741     if(decoded_reg_strb.AES_KV_RD_KEY_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_RD_KEY_CTRL.rsvd.swwe) begin // SW
write
742       next_c =
(field_storage.AES_KV_RD_KEY_CTRL.rsvd.value & ~de-
coded_wr_biten[31:7]) | (decoded_wr_data[31:7] & de-
coded_wr_biten[31:7]);
743       load_next_c = '1;
744     end
745     field_combo.AES_KV_RD_KEY_CTRL.rsvd.next = next_c;
746     field_combo.AES_KV_RD_KEY_CTRL.rsvd.load_next =
load_next_c;
747   end
748   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
749     if(~hwif_in.reset_b) begin
750       field_storage.AES_KV_RD_KEY_CTRL.rsvd.value <=
25'h0;
751     end else
if(field_combo.AES_KV_RD_KEY_CTRL.rsvd.load_next) begin
752       field_storage.AES_KV_RD_KEY_CTRL.rsvd.value <=
field_combo.AES_KV_RD_KEY_CTRL.rsvd.next;
753     end
754   end
755   assign hwif_out.AES_KV_RD_KEY_CTRL.rsvd.value =
field_storage.AES_KV_RD_KEY_CTRL.rsvd.value;
756   // Field: aes_clp_reg.AES_KV_RD_KEY_STATUS.VALID
757   always_comb begin
758     automatic logic [0:0] next_c;
759     automatic logic load_next_c;
760     next_c =
field_storage.AES_KV_RD_KEY_STATUS.VALID.value;
761     load_next_c = '0;
762     if(hwif_in.AES_KV_RD_KEY_STATUS.VALID.hwset) begin //
HW Set
763       next_c = '1;
764       load_next_c = '1;
765     end else if(hwif_in.AES_KV_RD_KEY_STATUS.VALID.hwclr)
begin // HW Clear
766       next_c = '0;
767       load_next_c = '1;
768     end
769     field_combo.AES_KV_RD_KEY_STATUS.VALID.next = next_c;
770     field_combo.AES_KV_RD_KEY_STATUS.VALID.load_next =
load_next_c;
771   end
772   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
773     if(~hwif_in.reset_b) begin
774       field_storage.AES_KV_RD_KEY_STATUS.VALID.value <=
1'h0;
775     end else
if(field_combo.AES_KV_RD_KEY_STATUS.VALID.load_next) begin
776       field_storage.AES_KV_RD_KEY_STATUS.VALID.value <=
field_combo.AES_KV_RD_KEY_STATUS.VALID.next;
777     end
778   end

```

```

779 // Field: aes_clp_reg.AES_KV_WR_CTRL.write_en
780 always_comb begin
781     automatic logic [0:0] next_c;
782     automatic logic load_next_c;
783     next_c = field_storage.AES_KV_WR_CTRL.write_en.value;
784     load_next_c = '0;
785     if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.write_en.swwe) begin // SW
write
786         next_c =
(field_storage.AES_KV_WR_CTRL.write_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
787         load_next_c = '1;
788     end else if(hwif_in.AES_KV_WR_CTRL.write_en.hwclr) be-
gin // HW Clear
789         next_c = '0;
790         load_next_c = '1;
791     end
792     field_combo.AES_KV_WR_CTRL.write_en.next = next_c;
793     field_combo.AES_KV_WR_CTRL.write_en.load_next =
load_next_c;
794 end
795 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
796     if(~hwif_in.reset_b) begin
797         field_storage.AES_KV_WR_CTRL.write_en.value <=
1'h0;
798     end else
799     if(field_combo.AES_KV_WR_CTRL.write_en.load_next) begin
800         field_storage.AES_KV_WR_CTRL.write_en.value <=
field_combo.AES_KV_WR_CTRL.write_en.next;
801     end
802     assign hwif_out.AES_KV_WR_CTRL.write_en.value =
field_storage.AES_KV_WR_CTRL.write_en.value;
803 // Field: aes_clp_reg.AES_KV_WR_CTRL.write_entry
804 always_comb begin
805     automatic logic [4:0] next_c;
806     automatic logic load_next_c;
807     next_c =
field_storage.AES_KV_WR_CTRL.write_entry.value;
808     load_next_c = '0;
809     if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.write_entry.swwe) begin //
SW write
810         next_c =
(field_storage.AES_KV_WR_CTRL.write_entry.value & ~de-
coded_wr_biten[5:1]) | (decoded_wr_data[5:1] & de-
coded_wr_biten[5:1]);
811         load_next_c = '1;
812     end
813     field_combo.AES_KV_WR_CTRL.write_entry.next = next_c;
814     field_combo.AES_KV_WR_CTRL.write_entry.load_next =
load_next_c;
815 end
816 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
817     if(~hwif_in.reset_b) begin
818         field_storage.AES_KV_WR_CTRL.write_entry.value <=
5'h0;
819     end else
820     if(field_combo.AES_KV_WR_CTRL.write_entry.load_next) begin
821         field_storage.AES_KV_WR_CTRL.write_entry.value <=
field_combo.AES_KV_WR_CTRL.write_entry.next;
822     end
823     assign hwif_out.AES_KV_WR_CTRL.write_entry.value =
field_storage.AES_KV_WR_CTRL.write_entry.value;
824 // Field: aes_clp_reg.AES_KV_WR_CTRL.hmac_key_dest_valid
825 always_comb begin
826     automatic logic [0:0] next_c;
827     automatic logic load_next_c;
828     next_c =
field_storage.AES_KV_WR_CTRL.hmac_key_dest_valid.value;
829     load_next_c = '0;
830     if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.hmac_key_dest_valid.swwe)
begin // SW write
831         next_c =
(field_storage.AES_KV_WR_CTRL.hmac_key_dest_valid.value & ~de-
coded_wr_biten[6:6]) | (decoded_wr_data[6:6] & de-
coded_wr_biten[6:6]);
832         load_next_c = '1;
833     end
834     field_combo.AES_KV_WR_CTRL.hmac_key_dest_valid.next =
next_c;
835     field_combo.AES_KV_WR_CTRL.hmac_key_dest_valid.load_next =
load_next_c;
836 end
837 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
838     if(~hwif_in.reset_b) begin
839         field_storage.AES_KV_WR_CTRL.hmac_key_dest_valid.value <=
1'h0;
840     end else
841     if(field_combo.AES_KV_WR_CTRL.hmac_key_dest_valid.load_next) begin
842         field_storage.AES_KV_WR_CTRL.hmac_key_dest_valid.value <=
field_combo.AES_KV_WR_CTRL.hmac_key_dest_valid.next;
843     end
844 end

```

```

844     assign hwif_out.AES_KV_WR_CTRL.hmac_key_dest_valid.value =
field_storage.AES_KV_WR_CTRL.hmac_key_dest_valid.value;
845 // Field: aes_clp_reg.AES_KV_WR_CTRL.hmac_block_dest_valid
846 always_comb begin
847     automatic logic [0:0] next_c;
848     automatic logic load_next_c;
849     next_c =
field_storage.AES_KV_WR_CTRL.hmac_block_dest_valid.value;
850     load_next_c = '0;
851     if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_C-
TRL.hmac_block_dest_valid.swwe) begin // SW write
852         next_c =
(field_storage.AES_KV_WR_CTRL.hmac_block_dest_valid.value & ~de-
coded_wr_biten[7:7]) | (decoded_wr_data[7:7] & de-
coded_wr_biten[7:7]);
853         load_next_c = '1;
854     end
855     field_combo.AES_KV_WR_CTRL.hmac_block_dest_valid.next =
next_c;
856     field_combo.AES_KV_WR_CTRL.hmac_block_dest_valid.load_next =
load_next_c;
857 end
858 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
859     if(~hwif_in.reset_b) begin
860         field_storage.AES_KV_WR_CTRL.hmac_block_dest_valid.value <=
1'h0;
861     end else
862     if(field_combo.AES_KV_WR_CTRL.hmac_block_dest_valid.load_next) begin
863         field_storage.AES_KV_WR_CTRL.hmac_block_dest_valid.value <=
field_combo.AES_KV_WR_CTRL.hmac_block_dest_valid.next;
864     end
865     assign hwif_out.AES_KV_WR_CTRL.hmac_block_dest_valid.value =
field_storage.AES_KV_WR_CTRL.hmac_block_dest_valid.value;
866 // Field: aes_clp_reg.AES_KV_WR_CTRL.mlds_seed_dest_valid
867 always_comb begin
868     automatic logic [0:0] next_c;
869     automatic logic load_next_c;
870     next_c =
field_storage.AES_KV_WR_CTRL.mlds_seed_dest_valid.value;
871     load_next_c = '0;
872     if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_C-
TRL.mlds_seed_dest_valid.swwe) begin // SW write
873         next_c =
(field_storage.AES_KV_WR_CTRL.mlds_seed_dest_valid.value & ~de-
coded_wr_biten[8:8]) | (decoded_wr_data[8:8] & de-
coded_wr_biten[8:8]);
874         load_next_c = '1;
875     end
876     field_combo.AES_KV_WR_CTRL.mlds_seed_dest_valid.next =
next_c;
877     field_combo.AES_KV_WR_CTRL.mlds_seed_dest_valid.load_next =
load_next_c;
878 end
879 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
880     if(~hwif_in.reset_b) begin
881         field_storage.AES_KV_WR_CTRL.mlds_seed_dest_valid.value <=
1'h0;
882     end else
883     if(field_combo.AES_KV_WR_CTRL.mlds_seed_dest_valid.load_next) begin
884         field_storage.AES_KV_WR_CTRL.mlds_seed_dest_valid.value <=
field_combo.AES_KV_WR_CTRL.mlds_seed_dest_valid.next;
885     end
886     assign hwif_out.AES_KV_WR_CTRL.mlds_seed_dest_valid.value =
field_storage.AES_KV_WR_CTRL.mlds_seed_dest_valid.value;
887 // Field: aes_clp_reg.AES_KV_WR_CTRL.ecc_pkey_dest_valid
888 always_comb begin
889     automatic logic [0:0] next_c;
890     automatic logic load_next_c;
891     next_c =
field_storage.AES_KV_WR_CTRL.ecc_pkey_dest_valid.value;
892     load_next_c = '0;
893     if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.ecc_pkey_dest_valid.swwe)
begin // SW write
894         next_c =
(field_storage.AES_KV_WR_CTRL.ecc_pkey_dest_valid.value & ~de-
coded_wr_biten[9:9]) | (decoded_wr_data[9:9] & de-
coded_wr_biten[9:9]);
895         load_next_c = '1;
896     end
897     field_combo.AES_KV_WR_CTRL.ecc_pkey_dest_valid.next =
next_c;
898     field_combo.AES_KV_WR_CTRL.ecc_pkey_dest_valid.load_next =
load_next_c;
899 end
900 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
901     if(~hwif_in.reset_b) begin
902         field_storage.AES_KV_WR_CTRL.ecc_pkey_dest_valid.value <=
1'h0;
903     end else

```

```

if(field_combo.AES_KV_WR_CTRL.ecc_pkey_dest_valid.load_next) begin
904
field_storage.AES_KV_WR_CTRL.ecc_pkey_dest_valid.value <=
field_combo.AES_KV_WR_CTRL.ecc_pkey_dest_valid.next;
905
end
906
end

907
assign hwif_out.AES_KV_WR_CTRL.ecc_pkey_dest_valid.value =
field_storage.AES_KV_WR_CTRL.ecc_pkey_dest_valid.value;
908
// Field: aes_clp_reg.AES_KV_WR_CTRL.ecc_seed_dest_valid
909
always_comb begin
910
automatic logic [0:0] next_c;
911
automatic logic load_next_c;
912
next_c =
field_storage.AES_KV_WR_CTRL.ecc_seed_dest_valid.value;
913
load_next_c = '0;
914
if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.ecc_seed_dest_valid.swwe)
begin // SW write
915
next_c =
(field_storage.AES_KV_WR_CTRL.ecc_seed_dest_valid.value & ~de-
coded_wr_biten[10:10]) | (decoded_wr_data[10:10] & de-
coded_wr_biten[10:10]);
916
load_next_c = '1;
917
end
918
field_combo.AES_KV_WR_CTRL.ecc_seed_dest_valid.next =
next_c;
919
field_combo.AES_KV_WR_CTRL.ecc_seed_dest_valid.load_next =
load_next_c;
920
end
921
always_ff @(posedge clk or negedge hwif_in.reset_b) begin
922
if(~hwif_in.reset_b) begin
923
field_storage.AES_KV_WR_CTRL.ecc_seed_dest_valid.value <= 1'h0;
924
end else
if(field_combo.AES_KV_WR_CTRL.ecc_seed_dest_valid.load_next) begin
925
field_storage.AES_KV_WR_CTRL.ecc_seed_dest_valid.value <=
field_combo.AES_KV_WR_CTRL.ecc_seed_dest_valid.next;
926
end
927
end

928
assign hwif_out.AES_KV_WR_CTRL.ecc_seed_dest_valid.value =
field_storage.AES_KV_WR_CTRL.ecc_seed_dest_valid.value;
929
// Field: aes_clp_reg.AES_KV_WR_CTRL.aes_key_dest_valid
930
always_comb begin
931
automatic logic [0:0] next_c;
932
automatic logic load_next_c;
933
next_c =
field_storage.AES_KV_WR_CTRL.aes_key_dest_valid.value;
934
load_next_c = '0;
935
if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.aes_key_dest_valid.swwe)
begin // SW write
936
next_c =
(field_storage.AES_KV_WR_CTRL.aes_key_dest_valid.value & ~de-
coded_wr_biten[11:11]) | (decoded_wr_data[11:11] & de-
coded_wr_biten[11:11]);
937
load_next_c = '1;
938
end
939
field_combo.AES_KV_WR_CTRL.aes_key_dest_valid.next =
next_c;
940
field_combo.AES_KV_WR_CTRL.aes_key_dest_valid.load_next =
load_next_c;
941
end
942
always_ff @(posedge clk or negedge hwif_in.reset_b) begin
943
if(~hwif_in.reset_b) begin
944
field_storage.AES_KV_WR_CTRL.aes_key_dest_valid.value <= 1'h0;
945
end else
if(field_combo.AES_KV_WR_CTRL.aes_key_dest_valid.load_next) begin
946
field_storage.AES_KV_WR_CTRL.aes_key_dest_valid.value <=
field_combo.AES_KV_WR_CTRL.aes_key_dest_valid.next;
947
end
948
end
949
assign hwif_out.AES_KV_WR_CTRL.aes_key_dest_valid.value =
field_storage.AES_KV_WR_CTRL.aes_key_dest_valid.value;
950
// Field: aes_clp_reg.AES_KV_WR_CTRL.mlkem_seed_dest_valid
951
always_comb begin
952
automatic logic [0:0] next_c;
953
automatic logic load_next_c;
954
next_c =
field_storage.AES_KV_WR_CTRL.mlkem_seed_dest_valid.value;
955
load_next_c = '0;
956
if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_C-
TRL.mlkem_seed_dest_valid.swwe) begin // SW write
957
next_c =
(field_storage.AES_KV_WR_CTRL.mlkem_seed_dest_valid.value & ~de-
coded_wr_biten[12:12]) | (decoded_wr_data[12:12] & de-
coded_wr_biten[12:12]);
958
load_next_c = '1;
959
end
960
field_combo.AES_KV_WR_CTRL.mlkem_seed_dest_valid.next =
next_c;
961
field_combo.AES_KV_WR_CTRL.mlkem_seed_dest_valid.load_next =
load_next_c;

```

```

962
end
963
always_ff @(posedge clk or negedge hwif_in.reset_b) begin
964
if(~hwif_in.reset_b) begin
965
field_storage.AES_KV_WR_CTRL.mlkem_seed_dest_valid.value <= 1'h0;
966
end else
if(field_combo.AES_KV_WR_CTRL.mlkem_seed_dest_valid.load_next) begin
967
field_storage.AES_KV_WR_CTRL.mlkem_seed_dest_valid.value <=
field_combo.AES_KV_WR_CTRL.mlkem_seed_dest_valid.next;
968
end
969
end

970
assign hwif_out.AES_KV_WR_CTRL.mlkem_seed_dest_valid.value =
field_storage.AES_KV_WR_CTRL.mlkem_seed_dest_valid.value;
971
// Field: aes_clp_reg.AES_KV_WR_CTRL.mlkem_msg_dest_valid
972
always_comb begin
973
automatic logic [0:0] next_c;
974
automatic logic load_next_c;
975
next_c =
field_storage.AES_KV_WR_CTRL.mlkem_msg_dest_valid.value;
976
load_next_c = '0;
977
if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.mlkem_msg_dest_valid.swwe)
begin // SW write
978
next_c =
(field_storage.AES_KV_WR_CTRL.mlkem_msg_dest_valid.value & ~de-
coded_wr_biten[13:13]) | (decoded_wr_data[13:13] & de-
coded_wr_biten[13:13]);
979
load_next_c = '1;
980
end

981
field_combo.AES_KV_WR_CTRL.mlkem_msg_dest_valid.next =
next_c;
982
field_combo.AES_KV_WR_CTRL.mlkem_msg_dest_valid.load_next =
load_next_c;
983
end
984
always_ff @(posedge clk or negedge hwif_in.reset_b) begin
985
if(~hwif_in.reset_b) begin
986
field_storage.AES_KV_WR_CTRL.mlkem_msg_dest_valid.value <= 1'h0;
987
end else
if(field_combo.AES_KV_WR_CTRL.mlkem_msg_dest_valid.load_next) begin
988
field_storage.AES_KV_WR_CTRL.mlkem_msg_dest_valid.value <=
field_combo.AES_KV_WR_CTRL.mlkem_msg_dest_valid.next;
989
end
990
end
991
assign hwif_out.AES_KV_WR_CTRL.mlkem_msg_dest_valid.value =
field_storage.AES_KV_WR_CTRL.mlkem_msg_dest_valid.value;
992
// Field: aes_clp_reg.AES_KV_WR_CTRL.dma_data_dest_valid
993
always_comb begin
994
automatic logic [0:0] next_c;
995
automatic logic load_next_c;
996
next_c =
field_storage.AES_KV_WR_CTRL.dma_data_dest_valid.value;
997
load_next_c = '0;
998
if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.dma_data_dest_valid.swwe)
begin // SW write
999
next_c =
(field_storage.AES_KV_WR_CTRL.dma_data_dest_valid.value & ~de-
coded_wr_biten[14:14]) | (decoded_wr_data[14:14] & de-
coded_wr_biten[14:14]);
1000
load_next_c = '1;
1001
end
1002
field_combo.AES_KV_WR_CTRL.dma_data_dest_valid.next =
next_c;
1003
field_combo.AES_KV_WR_CTRL.dma_data_dest_valid.load_next =
load_next_c;
1004
end
1005
always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1006
if(~hwif_in.reset_b) begin
1007
field_storage.AES_KV_WR_CTRL.dma_data_dest_valid.value <= 1'h0;
1008
end else
if(field_combo.AES_KV_WR_CTRL.dma_data_dest_valid.load_next) begin
1009
field_storage.AES_KV_WR_CTRL.dma_data_dest_valid.value <=
field_combo.AES_KV_WR_CTRL.dma_data_dest_valid.next;
1010
end
1011
end

1012
assign hwif_out.AES_KV_WR_CTRL.dma_data_dest_valid.value =
field_storage.AES_KV_WR_CTRL.dma_data_dest_valid.value;
1013
// Field: aes_clp_reg.AES_KV_WR_CTRL.rsvd
1014
always_comb begin
1015
automatic logic [16:0] next_c;
1016
automatic logic load_next_c;
1017
next_c = field_storage.AES_KV_WR_CTRL.rsvd.value;
1018
load_next_c = '0;
1019
if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.rsrd.swwe) begin // SW
write
1020
next_c = (field_storage.AES_KV_WR_CTRL.rsrd.value
& ~decoded_wr_biten[31:15]) | (decoded_wr_data[31:15] & de-
coded_wr_biten[31:15]);
1021
load_next_c = '1;
1022
end

```

```

1023     field_combo.AES_KV_WR_CTRL.rsvd.next = next_c;
1024     field_combo.AES_KV_WR_CTRL.rsvd.load_next =
load_next_c;
1025     end
1026     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1027         if(~hwif_in.reset_b) begin
1028             field_storage.AES_KV_WR_CTRL.rsvd.value <= 17'h0;
1029         end else if(field_combo.AES_KV_WR_CTRL.rsvd.load_next)
begin
1030             field_storage.AES_KV_WR_CTRL.rsvd.value <=
field_combo.AES_KV_WR_CTRL.rsvd.next;
1031         end
1032     end
1033     assign hwif_out.AES_KV_WR_CTRL.rsvd.value =
field_storage.AES_KV_WR_CTRL.rsvd.value;
1034     // Field: aes_clp_reg.AES_KV_WR_STATUS.VALID
1035     always_comb begin
1036         automatic logic [0:0] next_c;
1037         automatic logic load_next_c;
1038         next_c = field_storage.AES_KV_WR_STATUS.VALID.value;
1039         load_next_c = '0;
1040         if(hwif_in.AES_KV_WR_STATUS.VALID.hwset) begin // HW
Set
1041             next_c = '1;
1042             load_next_c = '1;
1043         end else if(hwif_in.AES_KV_WR_STATUS.VALID.hwclr) be-
gin // HW Clear
1044             next_c = '0;
1045             load_next_c = '1;
1046         end
1047         field_combo.AES_KV_WR_STATUS.VALID.next = next_c;
1048         field_combo.AES_KV_WR_STATUS.VALID.load_next =
load_next_c;
1049     end
1050     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1051         if(~hwif_in.reset_b) begin
1052             field_storage.AES_KV_WR_STATUS.VALID.value <=
1'h0;
1053         end else
if(field_combo.AES_KV_WR_STATUS.VALID.load_next) begin
1054             field_storage.AES_KV_WR_STATUS.VALID.value <=
field_combo.AES_KV_WR_STATUS.VALID.next;
1055         end
1056     end
1057     // Field: aes_clp_reg.intr_b-
lock_rf.global_intr_en_r.error_en
1058     always_comb begin
1059         automatic logic [0:0] next_c;
1060         automatic logic load_next_c;
1061         next_c =
field_storage.intr_block_rf.global_intr_en_r.error_en.value;
1062         load_next_c = '0;
1063         if(decoded_reg_strb.intr_block_rf.global_intr_en_r &&
decoded_req_is_wr) begin // SW write
1064             next_c =
(field_storage.intr_block_rf.global_intr_en_r.error_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1065             load_next_c = '1;
1066         end
1067         field_combo.intr_block_rf.global_intr_en_r.error_en.next = next_c;
1068         field_combo.intr_block_rf.global_intr_en_r.error_en.load_next =
load_next_c;
1069     end
1070     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1071         if(~hwif_in.reset_b) begin
1072             field_storage.intr_block_rf.global_intr_en_r.error_en.value <= 1'h0;
1073         end else
if(field_combo.intr_block_rf.global_intr_en_r.error_en.load_next)
begin
1074             field_storage.intr_block_rf.global_intr_en_r.error_en.value <=
field_combo.intr_block_rf.global_intr_en_r.error_en.next;
1075         end
1076     end
1077     // Field: aes_clp_reg.intr_b-
lock_rf.global_intr_en_r.notif_en
1078     always_comb begin
1079         automatic logic [0:0] next_c;
1080         automatic logic load_next_c;
1081         next_c =
field_storage.intr_block_rf.global_intr_en_r.notif_en.value;
1082         load_next_c = '0;
1083         if(decoded_reg_strb.intr_block_rf.global_intr_en_r &&
decoded_req_is_wr) begin // SW write
1084             next_c =
(field_storage.intr_block_rf.global_intr_en_r.notif_en.value & ~de-
coded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
coded_wr_biten[1:1]);
1085             load_next_c = '1;
1086         end
1087         field_combo.intr_block_rf.global_intr_en_r.notif_en.next = next_c;
1088         field_combo.intr_block_rf.global_intr_en_r.notif_en.load_next =
load_next_c;
1089     end

```

```

1090     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1091         if(~hwif_in.reset_b) begin
1092             field_storage.intr_block_rf.global_intr_en_r.notif_en.value <= 1'h0;
1093         end else
if(field_combo.intr_block_rf.global_intr_en_r.notif_en.load_next)
begin
1094             field_storage.intr_block_rf.global_intr_en_r.notif_en.value <=
field_combo.intr_block_rf.global_intr_en_r.notif_en.next;
1095         end
1096     end
1097     // Field: aes_clp_reg.intr_b-
lock_rf.error_intr_en_r.error0_en
1098     always_comb begin
1099         automatic logic [0:0] next_c;
1100         automatic logic load_next_c;
1101         next_c =
field_storage.intr_block_rf.error_intr_en_r.error0_en.value;
1102         load_next_c = '0;
1103         if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
1104             next_c =
(field_storage.intr_block_rf.error_intr_en_r.error0_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1105             load_next_c = '1;
1106         end
1107         field_combo.intr_block_rf.error_intr_en_r.error0_en.next = next_c;
1108         field_combo.intr_block_rf.error_intr_en_r.error0_en.load_next =
load_next_c;
1109     end
1110     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1111         if(~hwif_in.reset_b) begin
1112             field_storage.intr_block_rf.error_intr_en_r.error0_en.value <= 1'h0;
1113         end else
if(field_combo.intr_block_rf.error_intr_en_r.error0_en.load_next)
begin
1114             field_storage.intr_block_rf.error_intr_en_r.error0_en.value <=
field_combo.intr_block_rf.error_intr_en_r.error0_en.next;
1115         end
1116     end
1117     // Field: aes_clp_reg.intr_b-
lock_rf.error_intr_en_r.error1_en
1118     always_comb begin
1119         automatic logic [0:0] next_c;
1120         automatic logic load_next_c;
1121         next_c =
field_storage.intr_block_rf.error_intr_en_r.error1_en.value;
1122         load_next_c = '0;
1123         if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
1124             next_c =
(field_storage.intr_block_rf.error_intr_en_r.error1_en.value & ~de-
coded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
coded_wr_biten[1:1]);
1125             load_next_c = '1;
1126         end
1127         field_combo.intr_block_rf.error_intr_en_r.error1_en.next = next_c;
1128         field_combo.intr_block_rf.error_intr_en_r.error1_en.load_next =
load_next_c;
1129     end
1130     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1131         if(~hwif_in.reset_b) begin
1132             field_storage.intr_block_rf.error_intr_en_r.error1_en.value <= 1'h0;
1133         end else
if(field_combo.intr_block_rf.error_intr_en_r.error1_en.load_next)
begin
1134             field_storage.intr_block_rf.error_intr_en_r.error1_en.value <=
field_combo.intr_block_rf.error_intr_en_r.error1_en.next;
1135         end
1136     end
1137     // Field: aes_clp_reg.intr_b-
lock_rf.error_intr_en_r.error2_en
1138     always_comb begin
1139         automatic logic [0:0] next_c;
1140         automatic logic load_next_c;
1141         next_c =
field_storage.intr_block_rf.error_intr_en_r.error2_en.value;
1142         load_next_c = '0;
1143         if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
1144             next_c =
(field_storage.intr_block_rf.error_intr_en_r.error2_en.value & ~de-
coded_wr_biten[2:2]) | (decoded_wr_data[2:2] & de-
coded_wr_biten[2:2]);
1145             load_next_c = '1;
1146         end
1147         field_combo.intr_block_rf.error_intr_en_r.error2_en.next = next_c;
1148         field_combo.intr_block_rf.error_intr_en_r.error2_en.load_next =
load_next_c;

```

```

1149     end
1150     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1151         if(~hwif_in.reset_b) begin
1152             field_storage.intr_block_rf.error_intr_en_r.error2_en.value <= 1'h0;
1153         end else
1154             if(field_combo.intr_block_rf.error_intr_en_r.error2_en.load_next)
1155                 begin
1156                     field_storage.intr_block_rf.error_intr_en_r.error2_en.value <=
1157                         field_combo.intr_block_rf.error_intr_en_r.error2_en.next;
1158                     end
1159                 end
1160             // Field: aes_clp_reg.intr_b-
1161             lock_rf.error_intr_en_r.error3_en
1162             always_comb begin
1163                 automatic logic [0:0] next_c;
1164                 automatic logic load_next_c;
1165                 next_c =
1166                     field_storage.intr_block_rf.error_intr_en_r.error3_en.value;
1167                 load_next_c = '0;
1168                 if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
1169                     decoded_req_is_wr) begin // SW write
1170                     next_c =
1171                         (field_storage.intr_block_rf.error_intr_en_r.error3_en.value & ~de-
1172                             coded_wr_biten[3:3]) | (decoded_wr_data[3:3] & de-
1173                             coded_wr_biten[3:3]);
1174                     load_next_c = '1;
1175                 end
1176             end
1177             field_combo.intr_block_rf.error_intr_en_r.error3_en.next = next_c;
1178             field_combo.intr_block_rf.error_intr_en_r.error3_en.load_next =
1179                 load_next_c;
1180             end
1181             always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1182                 if(~hwif_in.reset_b) begin
1183                     field_storage.intr_block_rf.error_intr_en_r.error3_en.value <= 1'h0;
1184                 end else
1185                     if(field_combo.intr_block_rf.error_intr_en_r.error3_en.load_next)
1186                         begin
1187                             field_storage.intr_block_rf.error_intr_en_r.error3_en.value <=
1188                                 field_combo.intr_block_rf.error_intr_en_r.error3_en.next;
1189                             end
1190                         end
1191                     // Field: aes_clp_reg.intr_b-
1192                     lock_rf.notif_intr_en_r.notif_cmd_done_en
1193                     always_comb begin
1194                         automatic logic [0:0] next_c;
1195                         automatic logic load_next_c;
1196                         next_c =
1197                             field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value;
1198                         load_next_c = '0;
1199                         if(decoded_reg_strb.intr_block_rf.notif_intr_en_r &&
1200                             decoded_req_is_wr) begin // SW write
1201                             next_c =
1202                                 (field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
1203                                     & ~decoded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
1204                                     coded_wr_biten[0:0]);
1205                             load_next_c = '1;
1206                         end
1207                     end
1208                     field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.next =
1209                         next_c;
1210                     field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.load_ne
1211                         xt = load_next_c;
1212                     end
1213                     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1214                         if(~hwif_in.reset_b) begin
1215                             field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
1216                                 <= 1'h0;
1217                         end else
1218                             if(field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.load_
1219                                 next) begin
1220                                 field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
1221                                    <= field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.next;
1222                                 end
1223                             end
1224                             // Field: aes_clp_reg.intr_b-
1225                             lock_rf.error_global_intr_r.agg_sts
1226                             always_comb begin
1227                                 automatic logic [0:0] next_c;
1228                                 automatic logic load_next_c;
1229                                 next_c =
1230                                     field_storage.intr_block_rf.error_global_intr_r.agg_sts.value;
1231                                 load_next_c = '0;
1232                             end
1233                             // HW Write
1234                             next_c = hwif_out.intr_b-
1235                             lock_rf.error_internal_intr_r.intr;
1236                             load_next_c = '1;
1237                         end
1238                     end
1239                     field_combo.intr_block_rf.error_global_intr_r.agg_sts.next = next_c;
1240                     field_combo.intr_block_rf.error_global_intr_r.agg_sts.load_next =
1241                         load_next_c;
1242                     end
1243                     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1244                         if(~hwif_in.reset_b) begin
1245                             field_storage.intr_block_rf.error_global_intr_r.agg_sts.value <=
1246                                 1'h0;
1247                         end else
1248                             if(field_combo.intr_block_rf.error_global_intr_r.agg_sts.load_next)
1249                                 begin
1250                                     field_storage.intr_block_rf.error_global_intr_r.agg_sts.value <=
1251                                         field_combo.intr_block_rf.error_global_intr_r.agg_sts.next;
1252                                     end
1253                                 end
1254                             assign hwif_out.intr_block_rf.error_global_intr_r.intr =
1255                                 |
1256                                 (field_storage.intr_block_rf.error_global_intr_r.agg_sts.value &
1257                                     field_storage.intr_block_rf.error_global_intr_r.error_en.value);
1258                             // Field: aes_clp_reg.intr_b-
1259                             lock_rf.notif_global_intr_r.agg_sts
1260                             always_comb begin
1261                                 automatic logic [0:0] next_c;
1262                                 automatic logic load_next_c;
1263                                 next_c =
1264                                     field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value;
1265                                 load_next_c = '0;
1266                             end
1267                             // HW Write
1268                             next_c = hwif_out.intr_b-
1269                             lock_rf.notif_internal_intr_r.intr;
1270                             load_next_c = '1;
1271                         end
1272                     end
1273                     field_combo.intr_block_rf.notif_global_intr_r.agg_sts.next = next_c;
1274                     field_combo.intr_block_rf.notif_global_intr_r.agg_sts.load_next =
1275                         load_next_c;
1276                     end
1277                     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1278                         if(~hwif_in.reset_b) begin
1279                             field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value <=
1280                                 1'h0;
1281                         end else
1282                             if(field_combo.intr_block_rf.notif_global_intr_r.agg_sts.load_next)
1283                                 begin
1284                                     field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value <=
1285                                         field_combo.intr_block_rf.notif_global_intr_r.agg_sts.next;
1286                                     end
1287                                 end
1288                             assign hwif_out.intr_block_rf.notif_global_intr_r.intr =
1289                                 |
1290                                 (field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value &
1291                                     field_storage.intr_block_rf.error_global_intr_r.error_en.value);
1292                             // Field: aes_clp_reg.intr_b-
1293                             lock_rf.error_internal_intr_r.error0_sts
1294                             always_comb begin
1295                                 automatic logic [0:0] next_c;
1296                                 automatic logic load_next_c;
1297                                 next_c =
1298                                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value;
1299                                 load_next_c = '0;
1300                             end
1301                             if(field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value !=
1302                                 '0) begin // stickybit
1303                                 next_c =
1304                                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value |
1305                                     field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1306                                 load_next_c = '1;
1307                             end else if(hwif_in.intr_b-
1308                                 lock_rf.error_internal_intr_r.error0_sts.hwset) begin // HW Set
1309                                 next_c = '1;
1310                                 load_next_c = '1;
1311                             end else if(de-
1312                                 coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
1313                                 coded_req_is_wr) begin // SW write 1 clear
1314                                 next_c =
1315                                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value &
1316                                     ~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1317                                 load_next_c = '1;
1318                             end
1319                         end
1320                     end
1321                     field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next =
1322                         next_c;
1323                     field_combo.intr_block_rf.error_internal_intr_r.error0_sts.load_next
1324                         = load_next_c;
1325                     end
1326                     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
1327                     begin
1328                         if(~hwif_in.error_reset_b) begin
1329                             field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
1330                                 <= 1'h0;
1331                         end else
1332                             if(field_combo.intr_block_rf.error_internal_intr_r.error0_sts.load_
1333                                 next) begin
1334                                 field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
1335                                    <= field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next;
1336                                 end
1337                             end
1338                             field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
1339                                 <= field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next;
1340                             end
1341                         end
1342                     end

```

```

1209     end
1210     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1211         if(~hwif_in.reset_b) begin
1212             field_storage.intr_block_rf.error_global_intr_r.agg_sts.value <=
1213                 1'h0;
1214         end else
1215             if(field_combo.intr_block_rf.error_global_intr_r.agg_sts.load_next)
1216                 begin
1217                     field_storage.intr_block_rf.error_global_intr_r.agg_sts.value <=
1218                         field_combo.intr_block_rf.error_global_intr_r.agg_sts.next;
1219                     end
1220                 end
1221             assign hwif_out.intr_block_rf.error_global_intr_r.intr =
1222                 |
1223                 (field_storage.intr_block_rf.error_global_intr_r.agg_sts.value &
1224                     field_storage.intr_block_rf.error_global_intr_r.error_en.value);
1225             // Field: aes_clp_reg.intr_b-
1226             lock_rf.notif_global_intr_r.agg_sts
1227             always_comb begin
1228                 automatic logic [0:0] next_c;
1229                 automatic logic load_next_c;
1230                 next_c =
1231                     field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value;
1232                 load_next_c = '0;
1233             end
1234             // HW Write
1235             next_c = hwif_out.intr_b-
1236             lock_rf.notif_internal_intr_r.intr;
1237             load_next_c = '1;
1238         end
1239         field_combo.intr_block_rf.notif_global_intr_r.agg_sts.next = next_c;
1240         field_combo.intr_block_rf.notif_global_intr_r.agg_sts.load_next =
1241             load_next_c;
1242         end
1243         always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1244             if(~hwif_in.reset_b) begin
1245                 field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value <=
1246                     1'h0;
1247             end else
1248                 if(field_combo.intr_block_rf.notif_global_intr_r.agg_sts.load_next)
1249                     begin
1250                         field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value <=
1251                             field_combo.intr_block_rf.notif_global_intr_r.agg_sts.next;
1252                         end
1253                     end
1254                 assign hwif_out.intr_block_rf.notif_global_intr_r.intr =
1255                     |
1256                     (field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value &
1257                         field_storage.intr_block_rf.error_global_intr_r.error_en.value);
1258                 // Field: aes_clp_reg.intr_b-
1259                 lock_rf.error_internal_intr_r.error0_sts
1260                 always_comb begin
1261                     automatic logic [0:0] next_c;
1262                     automatic logic load_next_c;
1263                     next_c =
1264                         field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value;
1265                     load_next_c = '0;
1266                 end
1267                 if(field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value !=
1268                     '0) begin // stickybit
1269                     next_c =
1270                         field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value |
1271                         field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1272                     load_next_c = '1;
1273                 end else if(hwif_in.intr_b-
1274                     lock_rf.error_internal_intr_r.error0_sts.hwset) begin // HW Set
1275                     next_c = '1;
1276                     load_next_c = '1;
1277                 end else if(de-
1278                     coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
1279                     coded_req_is_wr) begin // SW write 1 clear
1280                     next_c =
1281                         field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value &
1282                         ~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1283                     load_next_c = '1;
1284                 end
1285             end
1286         end
1287         field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next =
1288             next_c;
1289         field_combo.intr_block_rf.error_internal_intr_r.error0_sts.load_next
1290             = load_next_c;
1291         end
1292         always_ff @(posedge clk or negedge hwif_in.error_reset_b)
1293         begin
1294             if(~hwif_in.error_reset_b) begin
1295                 field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
1296                     <= 1'h0;
1297             end else
1298                 if(field_combo.intr_block_rf.error_internal_intr_r.error0_sts.load_
1299                     next) begin
1300                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
1301                         <= field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next;
1302                     end
1303                 end
1304                 field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
1305                     <= field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next;
1306                 end
1307             end
1308         end

```



```

1266     end
1267     // Field: aes_clp_reg.intr_b-
lock_rf.error_internal_intr_r.error1_sts
1268     always_comb begin
1269         automatic logic [0:0] next_c;
1270         automatic logic load_next_c;
1271         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value;
1272         load_next_c = '0;
1273     end
if(field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value !
= '0) begin // stickybit
1274         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value;
1275         load_next_c = '1;
1276     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error1_sts.hwset) begin // HW Set
1277         next_c = '1;
1278         load_next_c = '1;
1279     end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1280         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value &
~(decoded_wr_data[1:1] & decoded_wr_biten[1:1]);
1281         load_next_c = '1;
1282     end
1283     field_combo.intr_block_rf.error_internal_intr_r.error1_sts.next =
next_c;
1284     field_combo.intr_block_rf.error_internal_intr_r.error1_sts.load_next
= load_next_c;
1285     end
1286     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1287         if(~hwif_in.error_reset_b) begin
1288             field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
<= 1'h0;
1289         end else
if(field_combo.intr_block_rf.error_internal_intr_r.error1_sts.load_
next) begin
1290             field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error1_sts.next;
1291         end
1292     end
1293     // Field: aes_clp_reg.intr_b-
lock_rf.error_internal_intr_r.error2_sts
1294     always_comb begin
1295         automatic logic [0:0] next_c;
1296         automatic logic load_next_c;
1297         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value;
1298         load_next_c = '0;
1299     end
if(field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value !
= '0) begin // stickybit
1300         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1301         load_next_c = '1;
1302     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset) begin // HW Set
1303         next_c = '1;
1304         load_next_c = '1;
1305     end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1306         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value &
~(decoded_wr_data[2:2] & decoded_wr_biten[2:2]);
1307         load_next_c = '1;
1308     end
1309     field_combo.intr_block_rf.error_internal_intr_r.error2_sts.next =
next_c;
1310     field_combo.intr_block_rf.error_internal_intr_r.error2_sts.load_next
= load_next_c;
1311     end
1312     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1313         if(~hwif_in.error_reset_b) begin
1314             field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
<= 1'h0;
1315         end else
if(field_combo.intr_block_rf.error_internal_intr_r.error2_sts.load_
next) begin
1316             field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error2_sts.next;
1317         end
1318     end
1319     // Field: aes_clp_reg.intr_b-
lock_rf.error_internal_intr_r.error3_sts
1320     always_comb begin
1321         automatic logic [0:0] next_c;

```

```

1322         automatic logic load_next_c;
1323         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value;
1324         load_next_c = '0;
1325     end
if(field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value !
= '0) begin // stickybit
1326         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1327         load_next_c = '1;
1328     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error3_sts.hwset) begin // HW Set
1329         next_c = '1;
1330         load_next_c = '1;
1331     end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1332         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value &
~(decoded_wr_data[3:3] & decoded_wr_biten[3:3]);
1333         load_next_c = '1;
1334     end
1335     field_combo.intr_block_rf.error_internal_intr_r.error3_sts.next =
next_c;
1336     field_combo.intr_block_rf.error_internal_intr_r.error3_sts.load_next
= load_next_c;
1337     end
1338     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1339         if(~hwif_in.error_reset_b) begin
1340             field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
<= 1'h0;
1341         end else
if(field_combo.intr_block_rf.error_internal_intr_r.error3_sts.load_
next) begin
1342             field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error3_sts.next;
1343         end
1344     end
1345     assign hwif_out.intr_block_rf.error_internal_intr_r.intr =
1346     |
(field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error0_en.value)
1347     ||
(field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error1_en.value)
1348     ||
(field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error2_en.value)
1349     ||
(field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error3_en.value);
1350     // Field: aes_clp_reg.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts
1351     always_comb begin
1352         automatic logic [0:0] next_c;
1353         automatic logic load_next_c;
1354         next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value;
1355         load_next_c = '0;
1356     end
if(field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
.value != '0) begin // stickybit
1357         next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value |
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
1358         load_next_c = '1;
1359     end else if(hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset) begin // HW
Set
1360         next_c = '1;
1361         load_next_c = '1;
1362     end else if(de-
coded_reg_strb.intr_block_rf.notif_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1363         next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value & ~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1364         load_next_c = '1;
1365     end
1366     field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.
next = next_c;
1367     field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.
load_next = load_next_c;
1368     end
1369     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1370         if(~hwif_in.reset_b) begin
1371             field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value <= 1'h0;
1372         end else

```

```

if(field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_s
ts.load_next) begin
1373
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value <=
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.
next;
1374     end
1375 end
1376 assign hwif_out.intr_block_rf.notif_internal_intr_r.intr =
1377 |
(field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_s
ts.value &
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value)
;
1378 // Field: aes_clp_reg.intr_b-
lock_rf.error_intr_trig_r.error0_trig
1379 always_comb begin
1380     automatic logic [0:0] next_c;
1381     automatic logic load_next_c;
1382     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1383     load_next_c = '0;

1384     if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1385         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value |
(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1386         load_next_c = '1;
1387     end else begin // singlepulse clears back to 0
1388         next_c = '0;
1389         load_next_c = '1;
1390     end
1391 end
field_combo.intr_block_rf.error_intr_trig_r.error0_trig.next =
next_c;
1392
field_combo.intr_block_rf.error_intr_trig_r.error0_trig.load_next =
load_next_c;
1393 end
1394 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1395     if(~hwif_in.reset_b) begin
1396
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value <=
1'h0;
1397     end else
if(field_combo.intr_block_rf.error_intr_trig_r.error0_trig.load_next
) begin
1398
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error0_trig.next;
1399     end
1400 end
1401 // Field: aes_clp_reg.intr_b-
lock_rf.error_intr_trig_r.error1_trig
1402 always_comb begin
1403     automatic logic [0:0] next_c;
1404     automatic logic load_next_c;
1405     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value;
1406     load_next_c = '0;

1407     if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1408         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value |
(decoded_wr_data[1:1] & decoded_wr_biten[1:1]);
1409         load_next_c = '1;
1410     end else begin // singlepulse clears back to 0
1411         next_c = '0;
1412         load_next_c = '1;
1413     end
1414 end
field_combo.intr_block_rf.error_intr_trig_r.error1_trig.next =
next_c;
1415
field_combo.intr_block_rf.error_intr_trig_r.error1_trig.load_next =
load_next_c;
1416 end
1417 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1418     if(~hwif_in.reset_b) begin
1419
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value <=
1'h0;
1420     end else
if(field_combo.intr_block_rf.error_intr_trig_r.error1_trig.load_next
) begin
1421
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error1_trig.next;
1422     end
1423 end
1424 // Field: aes_clp_reg.intr_b-
lock_rf.error_intr_trig_r.error2_trig
1425 always_comb begin
1426     automatic logic [0:0] next_c;
1427     automatic logic load_next_c;
1428     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1429     load_next_c = '0;

```

```

1430     if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1431         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value |
(decoded_wr_data[2:2] & decoded_wr_biten[2:2]);
1432         load_next_c = '1;
1433     end else begin // singlepulse clears back to 0
1434         next_c = '0;
1435         load_next_c = '1;
1436     end
1437 end
field_combo.intr_block_rf.error_intr_trig_r.error2_trig.next =
next_c;
1438
field_combo.intr_block_rf.error_intr_trig_r.error2_trig.load_next =
load_next_c;
1439 end
1440 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1441     if(~hwif_in.reset_b) begin
1442
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value <=
1'h0;
1443     end else
if(field_combo.intr_block_rf.error_intr_trig_r.error2_trig.load_next
) begin
1444
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error2_trig.next;
1445     end
1446 end
1447 // Field: aes_clp_reg.intr_b-
lock_rf.error_intr_trig_r.error3_trig
1448 always_comb begin
1449     automatic logic [0:0] next_c;
1450     automatic logic load_next_c;
1451     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1452     load_next_c = '0;

1453     if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1454         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value |
(decoded_wr_data[3:3] & decoded_wr_biten[3:3]);
1455         load_next_c = '1;
1456     end else begin // singlepulse clears back to 0
1457         next_c = '0;
1458         load_next_c = '1;
1459     end
1460 end
field_combo.intr_block_rf.error_intr_trig_r.error3_trig.next =
next_c;
1461
field_combo.intr_block_rf.error_intr_trig_r.error3_trig.load_next =
load_next_c;
1462 end
1463 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1464     if(~hwif_in.reset_b) begin
1465
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value <=
1'h0;
1466     end else
if(field_combo.intr_block_rf.error_intr_trig_r.error3_trig.load_next
) begin
1467
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error3_trig.next;
1468     end
1469 end
1470 // Field: aes_clp_reg.intr_b-
lock_rf.notif_intr_trig_r.notif_cmd_done_trig
1471 always_comb begin
1472     automatic logic [0:0] next_c;
1473     automatic logic load_next_c;
1474     next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
1475     load_next_c = '0;

1476     if(decoded_reg_strb.intr_block_rf.notif_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1477         next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue | (decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1478         load_next_c = '1;
1479     end else begin // singlepulse clears back to 0
1480         next_c = '0;
1481         load_next_c = '1;
1482     end
1483 end
field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.next
= next_c;
1484
field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig loa
d_next = load_next_c;
1485 end
1486 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1487     if(~hwif_in.reset_b) begin
1488
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v

```



```

alue <= 1'h0;
1489     end else
if(field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.
load_next) begin
1490     field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue <=
field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.next
;
1491     end
1492     end
1493     // Field: aes_clp_reg.intr_b-
lock_rf.error0_intr_count_r.cnt
1494     always_comb begin
1495         automatic logic [31:0] next_c;
1496         automatic logic load_next_c;
1497         next_c =
field_storage.intr_block_rf.error0_intr_count_r.cnt.value;
1498         load_next_c = '0;
1499         if(decoded_reg_strb.intr_block_rf.error0_intr_count_r
&& decoded_req_is_wr) begin // SW write
1500             next_c =
(field_storage.intr_block_rf.error0_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1501             load_next_c = '1;
1502         end
1503     end
if(field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value)
begin // increment
1504         if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
gin // up-counter saturated
1505             next_c = 32'hffffffff;
1506         end else begin
1507             next_c = next_c + 32'h1;
1508         end
1509         load_next_c = '1;
1510     end
1511 end
field_combo.intr_block_rf.error0_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error0_intr_count_r.cnt.value >=
32'hffffffff);
1512
field_combo.intr_block_rf.error0_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error0_intr_count_r.cnt.value >=
32'hffffffff);
1513     if(next_c > 32'hffffffff) begin
1514         next_c = 32'hffffffff;
1515         load_next_c = '1;
1516     end

1517     field_combo.intr_block_rf.error0_intr_count_r.cnt.next
= next_c;
1518
field_combo.intr_block_rf.error0_intr_count_r.cnt.load_next =
load_next_c;
1519     end
1520     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1521         if(~hwif_in.error_reset_b) begin
1522
field_storage.intr_block_rf.error0_intr_count_r.cnt.value <= 32'h0;
1523         end else
if(field_combo.intr_block_rf.error0_intr_count_r.cnt.load_next) be-
gin
1524
field_storage.intr_block_rf.error0_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error0_intr_count_r.cnt.next;
1525         end
1526     end
1527     // Field: aes_clp_reg.intr_b-
lock_rf.error1_intr_count_r.cnt
1528     always_comb begin
1529         automatic logic [31:0] next_c;
1530         automatic logic load_next_c;
1531         next_c =
field_storage.intr_block_rf.error1_intr_count_r.cnt.value;
1532         load_next_c = '0;
1533         if(decoded_reg_strb.intr_block_rf.error1_intr_count_r
&& decoded_req_is_wr) begin // SW write
1534             next_c =
(field_storage.intr_block_rf.error1_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1535             load_next_c = '1;
1536         end
1537     end
if(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value)
begin // increment
1538         if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
gin // up-counter saturated
1539             next_c = 32'hffffffff;
1540         end else begin
1541             next_c = next_c + 32'h1;
1542         end
1543         load_next_c = '1;
1544     end
1545 end
field_combo.intr_block_rf.error1_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=
32'hffffffff);

```

```

1546
field_combo.intr_block_rf.error1_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=
32'hffffffff);
1547     if(next_c > 32'hffffffff) begin
1548         next_c = 32'hffffffff;
1549         load_next_c = '1;
1550     end

1551     field_combo.intr_block_rf.error1_intr_count_r.cnt.next
= next_c;
1552
field_combo.intr_block_rf.error1_intr_count_r.cnt.load_next =
load_next_c;
1553     end
1554     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1555         if(~hwif_in.error_reset_b) begin
1556
field_storage.intr_block_rf.error1_intr_count_r.cnt.value <= 32'h0;
1557         end else
if(field_combo.intr_block_rf.error1_intr_count_r.cnt.load_next) be-
gin
1558
field_storage.intr_block_rf.error1_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error1_intr_count_r.cnt.next;
1559         end
1560     end
1561     // Field: aes_clp_reg.intr_b-
lock_rf.error2_intr_count_r.cnt
1562     always_comb begin
1563         automatic logic [31:0] next_c;
1564         automatic logic load_next_c;
1565         next_c =
field_storage.intr_block_rf.error2_intr_count_r.cnt.value;
1566         load_next_c = '0;
1567         if(decoded_reg_strb.intr_block_rf.error2_intr_count_r
&& decoded_req_is_wr) begin // SW write
1568             next_c =
(field_storage.intr_block_rf.error2_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1569             load_next_c = '1;
1570         end
1571     end
if(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value)
begin // increment
1572         if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
gin // up-counter saturated
1573             next_c = 32'hffffffff;
1574         end else begin
1575             next_c = next_c + 32'h1;
1576         end
1577         load_next_c = '1;
1578     end
1579 end
field_combo.intr_block_rf.error2_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error2_intr_count_r.cnt.value >=
32'hffffffff);
1580
field_combo.intr_block_rf.error2_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error2_intr_count_r.cnt.value >=
32'hffffffff);
1581     if(next_c > 32'hffffffff) begin
1582         next_c = 32'hffffffff;
1583         load_next_c = '1;
1584     end

1585     field_combo.intr_block_rf.error2_intr_count_r.cnt.next
= next_c;
1586
field_combo.intr_block_rf.error2_intr_count_r.cnt.load_next =
load_next_c;
1587     end
1588     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1589         if(~hwif_in.error_reset_b) begin
1590
field_storage.intr_block_rf.error2_intr_count_r.cnt.value <= 32'h0;
1591         end else
if(field_combo.intr_block_rf.error2_intr_count_r.cnt.load_next) be-
gin
1592
field_storage.intr_block_rf.error2_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error2_intr_count_r.cnt.next;
1593         end
1594     end
1595     // Field: aes_clp_reg.intr_b-
lock_rf.error3_intr_count_r.cnt
1596     always_comb begin
1597         automatic logic [31:0] next_c;
1598         automatic logic load_next_c;
1599         next_c =
field_storage.intr_block_rf.error3_intr_count_r.cnt.value;
1600         load_next_c = '0;
1601         if(decoded_reg_strb.intr_block_rf.error3_intr_count_r
&& decoded_req_is_wr) begin // SW write
1602             next_c =
(field_storage.intr_block_rf.error3_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);

```

```

1603         load_next_c = '1';
1604     end
1605 if(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value)
begin // increment
1606     if(((33)'(next_c) + 32'h1) > 32'hfffffff) be-
gin // up-counter saturated
1607         next_c = 32'hfffffff;
1608     end else begin
1609         next_c = next_c + 32'h1;
1610     end
1611     load_next_c = '1';
1612 end
1613 field_combo.intr_block_rf.error3_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error3_intr_count_r.cnt.value >=
32'hfffffff);
1614 field_combo.intr_block_rf.error3_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error3_intr_count_r.cnt.value >=
32'hfffffff);
1615     if(next_c > 32'hfffffff) begin
1616         next_c = 32'hfffffff;
1617         load_next_c = '1';
1618     end
1619     field_combo.intr_block_rf.error3_intr_count_r.cnt.next
= next_c;
1620     field_combo.intr_block_rf.error3_intr_count_r.cnt.load_next =
load_next_c;
1621     end
1622     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1623         if(~hwif_in.error_reset_b) begin
1624             field_storage.intr_block_rf.error3_intr_count_r.cnt.value <= 32'h0;
1625         end else
1626             if(field_combo.intr_block_rf.error3_intr_count_r.cnt.load_next) be-
gin
1627                 field_storage.intr_block_rf.error3_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error3_intr_count_r.cnt.next;
1628             end
1629             // Field: aes_clp_reg.intr_b-
lock_rf.notif_cmd_done_intr_count_r.cnt
1630             always_comb begin
1631                 automatic logic [31:0] next_c;
1632                 automatic logic load_next_c;
1633                 next_c =
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value;
1634                 load_next_c = '0';
1635                 if(de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r && de-
coded_req_is_wr) begin // SW write
1636                     next_c =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value &
~decoded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1637                     load_next_c = '1';
1638                 end
1639             end
1640             if(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pul-
se.value) begin // increment
1641                 if(((33)'(next_c) + 32'h1) > 32'hfffffff) be-
gin // up-counter saturated
1642                     next_c = 32'hfffffff;
1643                 end else begin
1644                     next_c = next_c + 32'h1;
1645                 end
1646                 load_next_c = '1';
1647             end
1648             field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.incrthres-
hold =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value
>= 32'hfffffff);
1649             field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.incrsatur-
ate =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value
>= 32'hfffffff);
1650             if(next_c > 32'hfffffff) begin
1651                 next_c = 32'hfffffff;
1652                 load_next_c = '1';
1653             end
1654             field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.next =
next_c;
1655             field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.load_next
= load_next_c;
1656             end
1657             always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1658                 if(~hwif_in.reset_b) begin
1659                     field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value <=
32'h0;
1660                 end else
1661                     if(field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.load_n

```

```

ext) begin
1660     field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value <=
field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.next;
1661     end
1662 end
1663 // Field: aes_clp_reg.intr_b-
lock_rf.error0_intr_count_incr_r.pulse
1664     always_comb begin
1665         automatic logic [0:0] next_c;
1666         automatic logic load_next_c;
1667         next_c =
field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value;
1668         load_next_c = '0';
1669     end
1670     if(field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value)
begin // HW Write - we
1671         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1672         load_next_c = '1';
1673     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error0_sts.hwset) begin // HW Set
1674         next_c = '1';
1675         load_next_c = '1';
1676     end
1677     if(field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value)
begin // decrement
1678         field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
1679         next_c = next_c - 1'h1;
1680         load_next_c = '1';
1681     end else begin
1682         field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.underflow =
'0';
1683     end
1684     field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value <=
1'd0);
1685     field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.next =
next_c;
1686     field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.load_next =
load_next_c;
1687     end
1688     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1689         if(~hwif_in.reset_b) begin
1690             field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value <=
1'h0;
1691         end else
1692             if(field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.load_ne-
xt) begin
1693                 field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.next;
1694             end
1695             end
1696             // Field: aes_clp_reg.intr_b-
lock_rf.error1_intr_count_incr_r.pulse
1697             always_comb begin
1698                 automatic logic [0:0] next_c;
1699                 automatic logic load_next_c;
1700                 next_c =
field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value;
1701                 load_next_c = '0';
1702             end
1703             if(field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value)
begin // HW Write - we
1704                 next_c =
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value;
1705                 load_next_c = '1';
1706             end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error1_sts.hwset) begin // HW Set
1707                 next_c = '1';
1708                 load_next_c = '1';
1709             end
1710             if(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value)
begin // decrement
1711                 field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
1712                 next_c = next_c - 1'h1;
1713                 load_next_c = '1';
1714             end else begin
1715                 field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.underflow =
'0';
1716             end
1717             field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
1'd0);
1718             field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.next =
next_c;
1719             field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.load_next =
load_next_c;
1720             end
1721             always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1722                 if(~hwif_in.reset_b) begin
1723                     field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
1'h0;
1724                 end else
1725                     if(field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.load_n

```

```

field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.next =
next_c;
1716
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.load_next =
load_next_c;
1717     end
1718     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1719         if(~hwif_in.reset_b) begin
1720
field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
1'h0;
1721         end else
if(field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.load_ne
xt) begin
1722
field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.next;
1723         end
1724     end
1725     // Field: aes_clp_reg.intr_b-
lock_rf.error2_intr_count_incr_r.pulse
1726     always_comb begin
1727         automatic logic [0:0] next_c;
1728         automatic logic load_next_c;
1729         next_c =
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value;
1730         load_next_c = '0;
1731
if(field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value)
begin // HW Write - we
1732         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1733         load_next_c = '1;
1734     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset) begin // HW Set
1735         next_c = '1;
1736         load_next_c = '1;
1737     end
1738
if(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value)
begin // decrement
1739
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
1740         next_c = next_c - 1'h1;
1741         load_next_c = '1;
1742     end else begin
1743
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.underflow =
'0;
1744     end
1745
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
1'd0);
1746
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next =
next_c;
1747
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_next =
load_next_c;
1748     end
1749     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1750         if(~hwif_in.reset_b) begin
1751
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
1'h0;
1752         end else
if(field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_ne
xt) begin
1753
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next;
1754         end
1755     end
1756     // Field: aes_clp_reg.intr_b-
lock_rf.error3_intr_count_incr_r.pulse
1757     always_comb begin
1758         automatic logic [0:0] next_c;
1759         automatic logic load_next_c;
1760         next_c =
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value;
1761         load_next_c = '0;
1762
if(field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value)
begin // HW Write - we
1763         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1764         load_next_c = '1;
1765     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error3_sts.hwset) begin // HW Set
1766         next_c = '1;
1767         load_next_c = '1;
1768     end
1769
if(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value)
begin // decrement
1770
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));

```

```

1771         next_c = next_c - 1'h1;
1772         load_next_c = '1;
1773     end else begin
1774
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.underflow =
'0;
1775     end
1776
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
1'd0);
1777
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.next =
next_c;
1778
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.load_next =
load_next_c;
1779     end
1780     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1781         if(~hwif_in.reset_b) begin
1782
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
1'h0;
1783         end else
if(field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.load_ne
xt) begin
1784
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.next;
1785     end
1786     end
1787     // Field: aes_clp_reg.intr_b-
lock_rf.notif_cmd_done_intr_count_incr_r.pulse
1788     always_comb begin
1789         automatic logic [0:0] next_c;
1790         automatic logic load_next_c;
1791         next_c =
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value;
1792         load_next_c = '0;
1793
if(field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
.value) begin // HW Write - we
1794         next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
1795         load_next_c = '1;
1796     end else if(hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset) begin // HW
Set
1797         next_c = '1;
1798         load_next_c = '1;
1799     end
1800
if(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pul
se.value) begin // decrement
1801
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.un
derflow = (next_c < (1'h1));
1802         next_c = next_c - 1'h1;
1803         load_next_c = '1;
1804     end else begin
1805
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.un
derflow = '0;
1806     end
1807
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.de
crthreshold =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <= 1'd0);
1808
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.ne
xt = next_c;
1809
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.lo
ad_next = load_next_c;
1810     end
1811     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1812         if(~hwif_in.reset_b) begin
1813
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <= 1'h0;
1814         end else
if(field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
load_next) begin
1815
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <=
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.ne
xt;
1816         end
1817     end
1818
-----
1820 // Write response
1821 //-----
1822     assign cpuif_wr_ack = decoded_req & decoded_req_is_wr;

```

```

1823 // Writes are always granted with no error response
1824 assign cpuif_wr_err = '0';
1825
1826 //-----
1827 // Readback
1828 //-----
1829
1830 logic readback_err;
1831 logic readback_done;
1832 logic [31:0] readback_data;
1833
1834 // Assign readback values to a flattened array
1835 logic [28-1:0][31:0] readback_array;
1836 for(genvar i0=0; i0<2; i0++) begin
1837     assign readback_array[i0*1 + 0][31:0] = (de-
coded_reg_strb.AES_NAME[i0] && !decoded_req_is_wr) ?
hwif_in.AES_NAME[i0].NAME.next : '0';
1838 end
1839 for(genvar i0=0; i0<2; i0++) begin
1840     assign readback_array[i0*1 + 2][31:0] = (de-
coded_reg_strb.AES_VERSION[i0] && !decoded_req_is_wr) ?
hwif_in.AES_VERSION[i0].VERSION.next : '0';
1841 end
1842 assign readback_array[4][0:0] = (decoded_reg_strb.CTRL0
&& !decoded_req_is_wr) ? field_storage.CTRL0.ENDIAN_SWAP.value : '0';
1843 assign readback_array[4][31:1] = '0';
1844 assign readback_array[5][0:0] = (de-
coded_reg_strb.AES_KV_RD_KEY_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_RD_KEY_CTRL.read_en.value : '0';
1845 assign readback_array[5][5:1] = (de-
coded_reg_strb.AES_KV_RD_KEY_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_RD_KEY_CTRL.read_entry.value : '0';
1846 assign readback_array[5][6:6] = (de-
coded_reg_strb.AES_KV_RD_KEY_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_RD_KEY_CTRL.pcr_hash_extend.value : '0';
1847 assign readback_array[5][31:7] = (de-
coded_reg_strb.AES_KV_RD_KEY_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_RD_KEY_CTRL.rsvd.value : '0';
1848 assign readback_array[6][0:0] = (de-
coded_reg_strb.AES_KV_RD_KEY_STATUS && !decoded_req_is_wr) ?
hwif_in.AES_KV_RD_KEY_STATUS.READY.next : '0';
1849 assign readback_array[6][1:1] = (de-
coded_reg_strb.AES_KV_RD_KEY_STATUS && !decoded_req_is_wr) ?
field_storage.AES_KV_RD_KEY_STATUS.VALID.value : '0';
1850 assign readback_array[6][9:2] = (de-
coded_reg_strb.AES_KV_RD_KEY_STATUS && !decoded_req_is_wr) ?
hwif_in.AES_KV_RD_KEY_STATUS.ERROR.next : '0';
1851 assign readback_array[6][31:10] = '0';
1852 assign readback_array[7][0:0] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.write_en.value : '0';
1853 assign readback_array[7][5:1] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.write_entry.value : '0';
1854 assign readback_array[7][6:6] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.hmac_key_dest_valid.value : '0';
1855 assign readback_array[7][7:7] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.hmac_block_dest_valid.value : '0';
1856 assign readback_array[7][8:8] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.mldsa_seed_dest_valid.value : '0';
1857 assign readback_array[7][9:9] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.ecc_pkey_dest_valid.value : '0';
1858 assign readback_array[7][10:10] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.ecc_seed_dest_valid.value : '0';
1859 assign readback_array[7][11:11] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.aes_key_dest_valid.value : '0';
1860 assign readback_array[7][12:12] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.mlkem_seed_dest_valid.value : '0';
1861 assign readback_array[7][13:13] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.mlkem_msg_dest_valid.value : '0';
1862 assign readback_array[7][14:14] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.dma_data_dest_valid.value : '0';
1863 assign readback_array[7][31:15] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.rsvd.value : '0';
1864 assign readback_array[8][0:0] = (de-
coded_reg_strb.AES_KV_WR_STATUS && !decoded_req_is_wr) ?
hwif_in.AES_KV_WR_STATUS.READY.next : '0';
1865 assign readback_array[8][1:1] = (de-
coded_reg_strb.AES_KV_WR_STATUS && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_STATUS.VALID.value : '0';
1866 assign readback_array[8][9:2] = (de-
coded_reg_strb.AES_KV_WR_STATUS && !decoded_req_is_wr) ?
hwif_in.AES_KV_WR_STATUS.ERROR.next : '0';
1867 assign readback_array[8][31:10] = '0';
1868 assign readback_array[9][0:0] = (de-
coded_reg_strb.intr_block_rf.global_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.global_intr_en_r.error_en.value : '0';
1869 assign readback_array[9][1:1] = (de-

```

```

coded_reg_strb.intr_block_rf.global_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.global_intr_en_r.notif_en.value : '0';
1870 assign readback_array[9][31:2] = '0';
1871 assign readback_array[10][0:0] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error0_en.value : '0';
1872 assign readback_array[10][1:1] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error1_en.value : '0';
1873 assign readback_array[10][2:2] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error2_en.value : '0';
1874 assign readback_array[10][3:3] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error3_en.value : '0';
1875 assign readback_array[10][31:4] = '0';
1876 assign readback_array[11][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
: '0';
1877 assign readback_array[11][31:1] = '0';
1878 assign readback_array[12][0:0] = (de-
coded_reg_strb.intr_block_rf.error_global_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_global_intr_r.agg_sts.value : '0';
1879 assign readback_array[12][31:1] = '0';
1880 assign readback_array[13][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_global_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value : '0';
1881 assign readback_array[13][31:1] = '0';
1882 assign readback_array[14][0:0] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value :
'0';
1883 assign readback_array[14][1:1] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value :
'0';
1884 assign readback_array[14][2:2] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value :
'0';
1885 assign readback_array[14][3:3] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value :
'0';
1886 assign readback_array[14][31:4] = '0';
1887 assign readback_array[15][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value : '0';
1888 assign readback_array[15][31:1] = '0';
1889 assign readback_array[16][0:0] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value :
'0';
1890 assign readback_array[16][1:1] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value :
'0';
1891 assign readback_array[16][2:2] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value :
'0';
1892 assign readback_array[16][3:3] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value :
'0';
1893 assign readback_array[16][31:4] = '0';
1894 assign readback_array[17][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue : '0';
1895 assign readback_array[17][31:1] = '0';
1896 assign readback_array[18][31:0] = (de-
coded_reg_strb.intr_block_rf.error0_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error0_intr_count_r.cnt.value : '0';
1897 assign readback_array[19][31:0] = (de-
coded_reg_strb.intr_block_rf.error1_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error1_intr_count_r.cnt.value : '0';
1898 assign readback_array[20][31:0] = (de-
coded_reg_strb.intr_block_rf.error2_intr_count_r && !de-

```

```

coded_req_is_wr) ?
field_storage.intr_block_rf.error2_intr_count_r.cnt.value : '0;
1899 assign readback_array[21][31:0] = (de-
coded_req_strb.intr_block_rf.error3_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error3_intr_count_r.cnt.value : '0;
1900 assign readback_array[22][31:0] = (de-
coded_req_strb.intr_block_rf.notif_cmd_done_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value :
'0;
1901 assign readback_array[23][0:0] = (de-
coded_req_strb.intr_block_rf.error0_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value :
'0;
1902 assign readback_array[23][31:1] = '0;
1903 assign readback_array[24][0:0] = (de-
coded_req_strb.intr_block_rf.error1_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value :
'0;
1904 assign readback_array[24][31:1] = '0;
1905 assign readback_array[25][0:0] = (de-
coded_req_strb.intr_block_rf.error2_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value :
'0;
1906 assign readback_array[25][31:1] = '0;
1907 assign readback_array[26][0:0] = (de-
coded_req_strb.intr_block_rf.error3_intr_count_incr_r && !de-

```

```

coded_req_is_wr) ?
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value :
'0;
1908 assign readback_array[26][31:1] = '0;
1909 assign readback_array[27][0:0] = (de-
coded_req_strb.intr_block_rf.notif_cmd_done_intr_count_incr_r && !
decoded_req_is_wr) ?
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value : '0;
1910 assign readback_array[27][31:1] = '0;
1911
1912 // Reduce the array
1913 always_comb begin
1914     automatic logic [31:0] readback_data_var;
1915     readback_done = decoded_req & ~decoded_req_is_wr;
1916     readback_err = '0;
1917     readback_data_var = '0;
1918     for(int i=0; i<28; i++) readback_data_var |= read-
back_array[i];
1919     readback_data = readback_data_var;
1920 end
1921
1922 assign cpuif_rd_ack = readback_done;
1923 assign cpuif_rd_data = readback_data;
1924 assign cpuif_rd_err = readback_err;
1925
1926 `include "caliptra_prim_assert.sv"
1927 `CALIPTRA_ASSERT_KNOWN(ERR_HWIF_IN, hwif_in, clk, !hwif_in.er-
ror_reset_b)
1928
1929 endmodule

```

PULP 기본 셀 (FIFO·CDC·아비터)

What this is. The PULP platform's common cells: FIFOs, two-phase and Gray-coded CDC, round-robin arbiter trees, stream registers and spill registers.

Why it exists. Because every SoC needs them and getting them wrong is expensive. The spill register in particular exists to break the combinational valid/ready loop that appears whenever two modules are connected directly.

Undergraduate review · EECS concepts. Metastability; Gray coding; handshake protocols; arbitration; pipeline decoupling.

Graduate · Practice note. Read `cc_spill_register.sv` closely. It solves a problem — the zero-delay handshake cycle — that is invisible when each module is tested alone and appears immediately on integration.

28 files • 5,148 lines

API — `assert_rpt_pkg.sv`

Table 1452. `pulp-platform_common_cells/src/assert_rpt_pkg.sv` — module / interface / package (1개)

이름	종류
<code>assert_rpt_pkg</code>	package

API — `cc_addr_decode.sv`

Table 1453. `pulp-platform_common_cells/src/cc_addr_decode.sv` — module / interface / package (1개)

이름	종류
<code>cc_addr_decode</code>	module

Table 1454. `pulp-platform_common_cells/src/cc_addr_decode.sv` — parameter (7개)

파라미터	형/폭	기본값
<code>NoIndices</code>	int unsigned	32'd0
<code>NoRules</code>	int unsigned	32'd1
<code>addr_t</code>		logic
<code>Napot</code>	bit	0
<code>IdxWidth</code>	int unsigned	<code>cc_pkg::idx_width(NoIndices)</code>
<code>idx_t</code>		logic [<code>IdxWidth</code> -1:0]
<code>rule_t</code>		struct packed {

Table 1455. `pulp-platform_common_cells/src/cc_addr_decode.sv` — 포트 (7개)

포트	방향	형/폭
<code>addr_i</code>	input	<code>addr_t</code>
<code>addr_map_i</code>	input	<code>rule_t</code> [<code>NoRules</code> -1:0]
<code>idx_o</code>	output	<code>idx_t</code>

포트	방향	형/폭
dec_valid_o	output	logic
dec_error_o	output	logic
en_default_idx_i	input	logic
default_idx_i	input	idx_t

API — cc_addr_decode_dync.sv

Table 1456. pulp-platform_common_cells/src/cc_addr_decode_dync.sv — module / interface / package (1개)

이름	종류
cc_addr_decode_dync	module

Table 1457. pulp-platform_common_cells/src/cc_addr_decode_dync.sv — parameter (7개)

파라미터	형/폭	기본값
NoIndices	int unsigned	32'd0
NoRules	int unsigned	32'd1
addr_t		logic
Napot	bit	0
IdxWidth	int unsigned	cc_pkg::idx_width(NoIndices)
idx_t		logic [IdxWidth-1:0]
rule_t		struct packed {

Table 1458. pulp-platform_common_cells/src/cc_addr_decode_dync.sv — 포트 (8개)

포트	방향	형/폭
addr_i	input	addr_t
addr_map_i	input	rule_t [NoRules-1:0]
idx_o	output	idx_t
dec_valid_o	output	logic
dec_error_o	output	logic
en_default_idx_i	input	logic
default_idx_i	input	idx_t
config_ongoing_i	input	logic

API — cc_addr_decode_napot.sv

Table 1459. pulp-platform_common_cells/src/cc_addr_decode_napot.sv — module / interface / package (1개)

이름	종류
cc_addr_decode_napot	module

Table 1460. pulp-platform_common_cells/src/cc_addr_decode_napot.sv — parameter (6개)

파라미터	형/폭	기본값
NoIndices	int unsigned	32'd0
NoRules	int unsigned	32'd1
addr_t		logic
IdxWidth	int unsigned	cc_pkg::idx_width(NoIndices)
idx_t		logic [IdxWidth-1:0]
rule_t		struct packed {

Table 1461. pulp-platform_common_cells/src/cc_addr_decode_napot.sv — 포트 (7개)

포트	방향	형/폭
addr_i	input	addr_t
addr_map_i	input	rule_t [NoRules-1:0]
idx_o	output	idx_t
dec_valid_o	output	logic
dec_error_o	output	logic
en_default_idx_i	input	logic
default_idx_i	input	idx_t

API — cc_binary_to_gray.sv

Table 1462. pulp-platform_common_cells/src/cc_binary_to_gray.sv — module / interface / package (1개)

이름	종류
cc_binary_to_gray	module

Table 1463. pulp-platform_common_cells/src/cc_binary_to_gray.sv — parameter (1개)

파라미터	형/폭	기본값
Width	int unsigned	1

Table 1464. pulp-platform_common_cells/src/cc_binary_to_gray.sv — 포트 (2개)

포트	방향	형/폭
a_i	input	logic [Width-1:0]
z_o	output	logic [Width-1:0]

API — cc_boxcar.sv

Table 1465. pulp-platform_common_cells/src/cc_boxcar.sv — module / interface / package (1개)

이름	종류
cc_boxcar	module

Table 1466. pulp-platform_common_cells/src/cc_boxcar.sv — parameter (1개)

파라미터	형/폭	기본값
Width	int unsigned	32

Table 1467. pulp-platform_common_cells/src/cc_boxcar.sv — localparam (3개)

상수	형/폭	값
IdxWidth	int unsigned	cc_pkg::idx_width(Width)
idx_t		logic [IdxWidth-1:0]
mask_t		logic [Width-1:0]

Table 1468. pulp-platform_common_cells/src/cc_boxcar.sv — 포트 (3개)

포트	방향	형/폭
lsb_i	input	idx_t
msb_i	input	idx_t
mask_o	output	mask_t

API — cc_cb_filter.sv**Table 1469. pulp-platform_common_cells/src/cc_cb_filter.sv — module / interface / package (2개)**

이름	종류
cc_cb_filter	module
cc_hash_block	module

Table 1470. pulp-platform_common_cells/src/cc_cb_filter.sv — parameter (11개)

파라미터	형/폭	기본값
KHashes	int unsigned	32'd3
HashWidth	int unsigned	32'd4
HashRounds	int unsigned	32'd1
InpWidth	int unsigned	32'd32
BucketWidth	int unsigned	32'd4
Seeds	cc_pkg::cb_seed_t [KHashes-1:0]	cc_pkg::CbEgSeeds
NoHashes	int unsigned	32'd3
InpWidth	int unsigned	32'd11
HashWidth	int unsigned	32'd5
NoRounds	int unsigned	32'd1
Seeds	cc_pkg::cb_seed_t [NoHashes-1:0]	cc_pkg::CbEgSeeds

Table 1471. pulp-platform_common_cells/src/cc_cb_filter.sv — localparam (1개)

상수	형/폭	값
NoCounters	int unsigned	2**HashWidth

Table 1472. pulp-platform_common_cells/src/cc_cb_filter.sv — 포트 (15개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
clr_i	input	logic
look_data_i	input	logic [InpWidth-1:0]
look_valid_o	output	logic
incr_data_i	input	logic [InpWidth-1:0]
incr_valid_i	input	logic
decr_data_i	input	logic [InpWidth-1:0]
decr_valid_i	input	logic
filter_usage_o	output	logic [HashWidth-1:0]
filter_full_o	output	logic
filter_empty_o	output	logic
filter_error_o	output	logic
data_i	input	logic [InpWidth-1:0]
indicator_o	output	logic [2**HashWidth-1:0]

API — cc_cdc_2phase.sv**Table 1473. pulp-platform_common_cells/src/cc_cdc_2phase.sv — module / interface / package (3개)**

이름	종류
cc_cdc_2phase	module
cc_cdc_2phase_src	module
cc_cdc_2phase_dst	module

Table 1474. pulp-platform_common_cells/src/cc_cdc_2phase.sv — parameter (6개)

파라미터	형/폭	기본값
data_t		logic
SyncStages	int unsigned	2
data_t		logic
SyncStages	int unsigned	2
data_t		logic
SyncStages	int unsigned	2

Table 1475. pulp-platform_common_cells/src/cc_cdc_2phase.sv — 포트 (26개)

포트	방향	형/폭
src_rst_ni	input	logic

포트	방향	형/폭
src_clk_i	input	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
rst_ni	input	logic
clk_i	input	logic
data_i	input	data_t
valid_i	input	logic
ready_o	output	logic
async_req_o	output	logic
async_ack_i	input	logic
async_data_o	output	data_t
rst_ni	input	logic
clk_i	input	logic
data_o	output	data_t
valid_o	output	logic
ready_i	input	logic
async_req_i	input	logic
async_ack_o	output	logic
async_data_i	input	data_t

API — cc_cdc_2phase_clearable.sv

Table 1476. pulp-platform_common_cells/src/cc_cdc_2phase_clearable.sv — module / interface / package (5개)

이름	종류
cc_cdc_2phase_clearable	module
cc_cdc_2phase_clearable_dst	module
cc_cdc_2phase_clearable_src	module
cc_cdc_2phase_src_clearable	module
cc_cdc_2phase_dst_clearable	module

Table 1477. pulp-platform_common_cells/src/cc_cdc_2phase_clearable.sv — parameter (13개)

파라미터	형/폭	기본값
data_t		logic
SyncStages	int unsigned	3
ClearOnAsyncReset	bit	1
data_t		logic
SyncStages	int unsigned	2
ClearOnAsyncReset	bit	1
data_t		logic
SyncStages	int unsigned	2
ClearOnAsyncReset	bit	1
data_t		logic
SyncStages	int unsigned	2
data_t		logic
SyncStages	int unsigned	2

Table 1478. pulp-platform_common_cells/src/cc_cdc_2phase_clearable.sv — localparam (2개)

상수	형/폭	값
ResetSyncStages	int unsigned	(SyncStages > 2) ? SyncStages - 1 : 2
ResetSyncStages	int unsigned	(SyncStages > 2) ? SyncStages - 1 : 2

Table 1479. pulp-platform_common_cells/src/cc_cdc_2phase_clearable.sv — 포트 (64개) — 앞 60개만 표시(전체 64개, 본문 소스 참조)

포트	방향	형/폭
src_rst_ni	input	logic
src_clk_i	input	logic
src_clear_i	input	logic
src_clear_pending_o	output	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_clear_i	input	logic
dst_clear_pending_o	output	logic
dst_data_o	output	data_t
dst_valid_o	output	logic

포트	방향	형/폭
dst_ready_i	input	logic
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_clear_i	input	logic
dst_clear_pending_o	output	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
async_payload_req_i	input	logic
async_payload_ack_o	output	logic
async_payload_data_i	input	data_t
async_reset_next_phase_o	output	cc_pkg::cdc_clear_seq_phase_e
async_reset_req_o	output	logic
async_reset_ack_i	input	logic
async_reset_next_phase_i	input	cc_pkg::cdc_clear_seq_phase_e
async_reset_req_i	input	logic
async_reset_ack_o	output	logic
src_rst_ni	input	logic
src_clk_i	input	logic
src_clear_i	input	logic
src_clear_pending_o	output	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
async_payload_req_o	output	logic
async_payload_ack_i	input	logic
async_payload_data_o	output	data_t
async_reset_next_phase_o	output	cc_pkg::cdc_clear_seq_phase_e
async_reset_req_o	output	logic
async_reset_ack_i	input	logic
async_reset_next_phase_i	input	cc_pkg::cdc_clear_seq_phase_e
async_reset_req_i	input	logic
async_reset_ack_o	output	logic
rst_ni	input	logic
clk_i	input	logic

포트	방향	형/폭
clear_i	input	logic
data_i	input	data_t
valid_i	input	logic
ready_o	output	logic
async_req_o	output	logic
async_ack_i	input	logic
async_data_o	output	data_t
rst_ni	input	logic
clk_i	input	logic
clear_i	input	logic
data_o	output	data_t
valid_o	output	logic

API — cc_cdc_4phase.sv

Table 1480. pulp-platform_common_cells/src/cc_cdc_4phase.sv — module / interface / package (3개)

이름	종류
cc_cdc_4phase	module
cc_cdc_4phase_src	module
cc_cdc_4phase_dst	module

Table 1481. pulp-platform_common_cells/src/cc_cdc_4phase.sv — parameter (13개)

파라미터	형/폭	기본값
data_t		logic
Decoupled	bit	1'b1
SendResetMsg	bit	1'b0
ResetMsg	data_t	data_t('0)
SyncStages	int unsigned	2
data_t		logic
SyncStages	int unsigned	2
Decoupled	bit	1'b1
SendResetMsg	bit	1'b0
ResetMsg	data_t	data_t('0)
data_t		logic
SyncStages	int unsigned	2
Decoupled	bit	1'b1

Table 1482. pulp-platform_common_cells/src/cc_cdc_4phase.sv — localparam (2개)

상수	형/폭	값
ReqSrcResetValue	logic	SendResetMsg ? 1'b1 : 1'b0
DataSrcResetValue	data_t	SendResetMsg ? ResetMsg : data_t'(0)

Table 1483. pulp-platform_common_cells/src/cc_cdc_4phase.sv — 포트 (26개)

포트	방향	형/폭
src_rst_ni	input	logic
src_clk_i	input	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
rst_ni	input	logic
clk_i	input	logic
data_i	input	data_t
valid_i	input	logic
ready_o	output	logic
async_req_o	output	logic
async_ack_i	input	logic
async_data_o	output	data_t
rst_ni	input	logic
clk_i	input	logic
data_o	output	data_t
valid_o	output	logic
ready_i	input	logic
async_req_i	input	logic
async_ack_o	output	logic
async_data_i	input	data_t

Table 1484. pulp-platform_common_cells/src/cc_cdc_4phase.sv — typedef / using (2개)

새 이름	원래 형
state_e	enum logic[1:0] {IDLE, WAIT_ACK_ASSERT, WAIT_ACK_DEASSERT}

새 이름	원래 형
state_e	enum logic[1:0] {IDLE, WAIT_DOWNSTREAM_ACK, WAIT_REQ_DEASSERT}

API — cc_cdc_fifo_2phase.sv

Table 1485. pulp-platform_common_cells/src/cc_cdc_fifo_2phase.sv — module / interface / package (3개)

이름	종류
cc_cdc_fifo_2phase	module
cc_cdc_fifo_2phase_src	module
cc_cdc_fifo_2phase_dst	module

Table 1486. pulp-platform_common_cells/src/cc_cdc_fifo_2phase.sv — parameter (9개)

파라미터	형/폭	기본값
data_t		logic
LogDepth	int unsigned	3
SyncStages	int unsigned	2
data_t		logic
LogDepth	int unsigned	3
SyncStages	int unsigned	2
data_t		logic
LogDepth	int unsigned	3
SyncStages	int unsigned	2

Table 1487. pulp-platform_common_cells/src/cc_cdc_fifo_2phase.sv — localparam (5개)

상수	형/폭	값
PtrWidth	int unsigned	LogDepth+1
PtrWidth	int unsigned	LogDepth+1
PtrFull	pointer_t	(1 << LogDepth)
PtrWidth	int unsigned	LogDepth+1
PtrEmpty	pointer_t	'0

Table 1488. pulp-platform_common_cells/src/cc_cdc_fifo_2phase.sv — 포트 (34개)

포트	방향	형/폭
src_rst_ni	input	logic
src_clk_i	input	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
dst_rst_ni	input	logic

포트	방향	형/폭
dst_clk_i	input	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
src_rst_ni	input	logic
src_clk_i	input	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
async_data_o	output	data_t [2**LogDepth-1:0]
async_wptr_req_o	output	logic
async_wptr_ack_i	input	logic
async_wptr_data_o	output	logic [LogDepth:0]
async_rptr_req_i	input	logic
async_rptr_ack_o	output	logic
async_rptr_data_i	input	logic [LogDepth:0]
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
async_data_i	input	data_t [2**LogDepth-1:0]
async_wptr_req_i	input	logic
async_wptr_ack_o	output	logic
async_wptr_data_i	input	logic [LogDepth:0]
async_rptr_req_o	output	logic
async_rptr_ack_i	input	logic
async_rptr_data_o	output	logic [LogDepth:0]

Table 1489. pulp-platform_common_cells/src/cc_cdc_fifo_2phase.sv — typedef / using (5개)

새 이름	원래 형
pointer_t	logic [PtrWidth-1:0]
pointer_t	logic [PtrWidth-1:0]
index_t	logic [LogDepth-1:0]
pointer_t	logic [PtrWidth-1:0]

새 이름	원래 형
index_t	logic [LogDepth-1:0]

API — cc_cdc_fifo_gray.sv

Table 1490. pulp-platform_common_cells/src/cc_cdc_fifo_gray.sv — module / interface / package (7개)

이름	종류
cc_cdc_fifo_gray	module
cc_cdc_fifo_gray_src	module
cc_cdc_fifo_gray_dst	module
cc_cdc_fifo_gray_src_data	module
cc_cdc_fifo_gray_src_ptr	module
cc_cdc_fifo_gray_dst_ptr	module
cc_cdc_fifo_gray_dst_data	module

Table 1491. pulp-platform_common_cells/src/cc_cdc_fifo_gray.sv — parameter (18개)

파라미터	형/폭	기본값
Width	int unsigned	1
data_t		logic [Width-1:0]
LogDepth	int unsigned	3
SyncStages	int unsigned	2
data_t		logic
LogDepth	int unsigned	3
SyncStages	int unsigned	2
data_t		logic
LogDepth	int unsigned	3
SyncStages	int unsigned	2
data_t		logic
LogDepth	int unsigned	3
LogDepth	int unsigned	3
SyncStages	int unsigned	2
LogDepth	int unsigned	3
SyncStages	int unsigned	2
data_t		logic
LogDepth	int unsigned	3

Table 1492. pulp-platform_common_cells/src/cc_cdc_fifo_gray.sv — localparam (4개)

상수	형/폭	값
PtrWidth	int unsigned	LogDepth+1
PtrFull	logic [PtrWidth-1:0]	(1 << LogDepth)
PtrWidth	int unsigned	LogDepth+1
PtrEmpty	logic [PtrWidth-1:0]	'0

Table 1493. pulp-platform_common_cells/src/cc_cdc_fifo_gray.sv — 포트 (55개)

포트	방향	형/폭
src_rst_ni	input	logic
src_clk_i	input	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
src_rst_ni	input	logic
src_clk_i	input	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
async_data_o	output	data_t [2**LogDepth-1:0]
async_wptr_o	output	logic [LogDepth:0]
async_rptr_i	input	logic [LogDepth:0]
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
async_data_i	input	data_t [2**LogDepth-1:0]
async_wptr_i	input	logic [LogDepth:0]
async_rptr_o	output	logic [LogDepth:0]
src_rst_ni	input	logic
src_clk_i	input	logic

포트	방향	형/폭
src_data_i	input	data_t
src_widx_i	input	logic [LogDepth-1:0]
src_write_i	input	logic
async_data_o	output	data_t [2**LogDepth-1:0]
src_rst_ni	input	logic
src_clk_i	input	logic
src_valid_i	input	logic
src_ready_o	output	logic
src_widx_o	output	logic [LogDepth-1:0]
async_wptr_o	output	logic [LogDepth:0]
async_rptra_i	input	logic [LogDepth:0]
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_ready_i	input	logic
dst_valid_o	output	logic
dst_ridx_o	output	logic [LogDepth-1:0]
async_wptr_i	input	logic [LogDepth:0]
async_rptra_o	output	logic [LogDepth:0]
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
dst_valid_i	input	logic
dst_ready_o	output	logic
dst_ridx_i	input	logic [LogDepth-1:0]
async_data_i	input	data_t [2**LogDepth-1:0]

API — cc_cdc_fifo_gray_clearable.sv

Table 1494. pulp-platform_common_cells/src/cc_cdc_fifo_gray_clearable.sv — module / interface / package (9개)

이름	종류
cc_cdc_fifo_gray_clearable	module
cc_cdc_fifo_gray_clearable_src	module
cc_cdc_fifo_gray_clearable_dst	module

이름	종류
cc_cdc_fifo_gray_src_clearable	module
cc_cdc_fifo_gray_dst_clearable	module
cc_cdc_fifo_gray_src_data_clearable	module
cc_cdc_fifo_gray_src_ptr_clearable	module
cc_cdc_fifo_gray_dst_ptr_clearable	module
cc_cdc_fifo_gray_dst_data_clearable	module

Table 1495. pulp-platform_common_cells/src/cc_cdc_fifo_gray_clearable.sv — parameter (27개)

파라미터	형/폭	기본값
Width	int unsigned	1
data_t		logic [Width-1:0]
LogDepth	int unsigned	3
SyncStages	int unsigned	3
ClearOnAsyncReset	bit	1
data_t		logic
LogDepth	int unsigned	3
SyncStages	int unsigned	2
ClearOnAsyncReset	bit	1
data_t		logic
LogDepth	int unsigned	3
SyncStages	int unsigned	2
ClearOnAsyncReset	bit	1
data_t		logic
LogDepth	int unsigned	3
SyncStages	int unsigned	2
data_t		logic
LogDepth	int unsigned	3
SyncStages	int unsigned	2
data_t		logic
LogDepth	int unsigned	3
LogDepth	int unsigned	3
SyncStages	int unsigned	2
LogDepth	int unsigned	3
SyncStages	int unsigned	2
data_t		logic

파라미터	형/폭	기본값
LogDepth	int unsigned	3

Table 1496. pulp-platform_common_cells/src/cc_cdc_fifo_gray_clearable.sv — localparam (6개)

상수	형/폭	값
ResetSyncStages	int unsigned	(SyncStages > 2) ? SyncStages - 1 : 2
ResetSyncStages	int unsigned	(SyncStages > 2) ? SyncStages - 1 : 2
PtrWidth	int unsigned	LogDepth+1
PtrFull	logic [PtrWidth-1:0]	(1 << LogDepth)
PtrWidth	int unsigned	LogDepth+1
PtrEmpty	logic [PtrWidth-1:0]	'0

Table 1497. pulp-platform_common_cells/src/cc_cdc_fifo_gray_clearable.sv — 포트 (96개) — 앞 60개만 표시(전체 96개, 본문 소스 참조)

포트	방향	형/폭
src_rst_ni	input	logic
src_clk_i	input	logic
src_clear_i	input	logic
src_clear_pending_o	output	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_clear_i	input	logic
dst_clear_pending_o	output	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
src_rst_ni	input	logic
src_clk_i	input	logic
src_clear_i	input	logic
src_clear_pending_o	output	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
async_data_o	output	data_t [2**LogDepth-1:0]

포트	방향	형/폭
async_wptr_o	output	logic [LogDepth:0]
async_rptr_i	input	logic [LogDepth:0]
async_reset_next_phase_o	output	cc_pkg::cdc_clear_seq_phase_e
async_reset_req_o	output	logic
async_reset_ack_i	input	logic
async_reset_next_phase_i	input	cc_pkg::cdc_clear_seq_phase_e
async_reset_req_i	input	logic
async_reset_ack_o	output	logic
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_clear_i	input	logic
dst_clear_pending_o	output	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
async_data_i	input	data_t [2**LogDepth-1:0]
async_wptr_i	input	logic [LogDepth:0]
async_rptr_o	output	logic [LogDepth:0]
async_reset_next_phase_o	output	cc_pkg::cdc_clear_seq_phase_e
async_reset_req_o	output	logic
async_reset_ack_i	input	logic
async_reset_next_phase_i	input	cc_pkg::cdc_clear_seq_phase_e
async_reset_req_i	input	logic
async_reset_ack_o	output	logic
src_rst_ni	input	logic
src_clk_i	input	logic
src_clear_i	input	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
async_data_o	output	data_t [2**LogDepth-1:0]
async_wptr_o	output	logic [LogDepth:0]
async_rptr_i	input	logic [LogDepth:0]
dst_rst_ni	input	logic
dst_clk_i	input	logic

포트	방향	형/폭
dst_clear_i	input	logic
dst_data_o	output	data_t
dst_valid_o	output	logic

API — cc_cdc_reset_ctrlr.sv

Table 1498. pulp-platform_common_cells/src/cc_cdc_reset_ctrlr.sv — module / interface / package (2개)

이름	종류
cc_cdc_reset_ctrlr	module
cc_cdc_reset_ctrlr_half	module

Table 1499. pulp-platform_common_cells/src/cc_cdc_reset_ctrlr.sv — parameter (4개)

파라미터	형/폭	기본값
SyncStages	int unsigned	2
ClearOnAsyncReset	logic	1'b1
SyncStages	int unsigned	2
ClearOnAsyncReset	logic	1'b1

Table 1500. pulp-platform_common_cells/src/cc_cdc_reset_ctrlr.sv — localparam (2개)

상수	형/폭	값
InitiatorStateResetValue	initiator_state_e	ClearOnAsyncReset ? ISOLATE : IDLE
ResetSyncStages	int unsigned	(SyncStages > 2) ? SyncStages : 2

Table 1501. pulp-platform_common_cells/src/cc_cdc_reset_ctrlr.sv — 포트 (27개)

포트	방향	형/폭
a_clk_i	input	logic
a_rst_ni	input	logic
a_clear_i	input	logic
a_clear_o	output	logic
a_clear_ack_i	input	logic
a_isolate_o	output	logic
a_isolate_ack_i	input	logic
b_clk_i	input	logic
b_rst_ni	input	logic
b_clear_i	input	logic
b_clear_o	output	logic
b_clear_ack_i	input	logic
b_isolate_o	output	logic

포트	방향	형/폭
b_isolate_ack_i	input	logic
clk_i	input	logic
rst_ni	input	logic
clear_i	input	logic
isolate_o	output	logic
isolate_ack_i	input	logic
clear_o	output	logic
clear_ack_i	input	logic
async_next_phase_o	output	cdc_clear_seq_phase_e
async_req_o	output	logic
async_ack_i	input	logic
async_next_phase_i	input	cdc_clear_seq_phase_e
async_req_i	input	logic
async_ack_o	output	logic

API — cc_clk_int_div.sv

Table 1502. pulp-platform_common_cells/src/cc_clk_int_div.sv — module / interface / package (1개)

이름	종류
cc_clk_int_div	module

Table 1503. pulp-platform_common_cells/src/cc_clk_int_div.sv — parameter (3개)

파라미터	형/폭	기본값
DivValueWidth	int unsigned	4
DefaultDivValue	int unsigned	0
EnableClockInReset	bit	1'b0

Table 1504. pulp-platform_common_cells/src/cc_clk_int_div.sv — localparam (3개)

상수	형/폭	값
DivResetValue	int unsigned	(DefaultDivValue != 0)? DefaultDivValue: 1
UseOddDivisionResetValue	logic	DefaultDivValue[0]
ClkDivBypassEnResetValue	logic	(DefaultDivValue < 2)? 1'b1: 1'b0

Table 1505. pulp-platform_common_cells/src/cc_clk_int_div.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
en_i	input	logic

포트	방향	형/폭
test_mode_en_i	input	logic
div_i	input	logic [DivValueWidth-1:0]
div_valid_i	input	logic
div_ready_o	output	logic
clk_o	output	logic
cycl_count_o	output	logic [DivValueWidth-1:0]

Table 1506. pulp-platform_common_cells/src/cc_clk_int_div.sv — typedef / using (1개)

새 이름	원래 형
clk_gate_state_e	enum logic[1:0] {IDLE, LOAD_DIV, WAIT_END_PERIOD}

API — cc_clk_int_div_static.sv**Table 1507. pulp-platform_common_cells/src/cc_clk_int_div_static.sv — module / interface / package (1개)**

이름	종류
cc_clk_int_div_static	module

Table 1508. pulp-platform_common_cells/src/cc_clk_int_div_static.sv — parameter (2개)

파라미터	형/폭	기본값
DivValue	int unsigned	1
EnableClockInReset	bit	1'b1

Table 1509. pulp-platform_common_cells/src/cc_clk_int_div_static.sv — localparam (1개)

상수	형/폭	값
DivValueWidth	int unsigned	\$clog2(DivValue+1)

Table 1510. pulp-platform_common_cells/src/cc_clk_int_div_static.sv — 포트 (5개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
en_i	input	logic
test_mode_en_i	input	logic
clk_o	output	logic

API — cc_clk_mux_glitch_free.sv**Table 1511. pulp-platform_common_cells/src/cc_clk_mux_glitch_free.sv — module / interface / package (2개)**

이름	종류
cc_clk_mux_glitch_free	module

이름	종류
cc_clk_or_tree	module

Table 1512. pulp-platform_common_cells/src/cc_clk_mux_glitch_free.sv — parameter (3개)

파라미터	형/폭	기본값
NumInputs	int unsigned	2
NumSyncStages	int unsigned	2
ClockDuringReset	bit	1'b1

Table 1513. pulp-platform_common_cells/src/cc_clk_mux_glitch_free.sv — localparam (1개)

상수	형/폭	값
SelWidth	int unsigned	\$clog2(NumInputs)

Table 1514. pulp-platform_common_cells/src/cc_clk_mux_glitch_free.sv — 포트 (8개)

포트	방향	형/폭
clks_i	input	logic [NumInputs-1:0]
test_clk_i	input	logic
test_en_i	input	logic
async_rstn_i	input	logic
async_sel_i	input	logic [SelWidth-1:0]
clk_o	output	logic
clks_i	input	logic [NumInputs-1:0]
clk_o	output	logic

API — cc_counter.sv**Table 1515.** pulp-platform_common_cells/src/cc_counter.sv — module / interface / package (1개)

이름	종류
cc_counter	module

Table 1516. pulp-platform_common_cells/src/cc_counter.sv — parameter (2개)

파라미터	형/폭	기본값
Width	int unsigned	4
StickyOverflow	bit	1'b0

Table 1517. pulp-platform_common_cells/src/cc_counter.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
clr_i	input	logic

포트	방향	형/폭
en_i	input	logic
load_i	input	logic
down_i	input	logic
d_i	input	logic [Width-1:0]
q_o	output	logic [Width-1:0]
overflow_o	output	logic

API — cc_credit_counter.sv

Table 1518. pulp-platform_common_cells/src/cc_credit_counter.sv — module / interface / package (1개)

이름	종류
cc_credit_counter	module

Table 1519. pulp-platform_common_cells/src/cc_credit_counter.sv — parameter (2개)

파라미터	형/폭	기본값
NumCredits	int unsigned	0
InitCreditEmpty	bit	1'b0

Table 1520. pulp-platform_common_cells/src/cc_credit_counter.sv — localparam (2개)

상수	형/폭	값
InitNumCredits	int unsigned	InitCreditEmpty ? '0 : NumCredits
credit_cnt_t		logic [\$clog2(NumCredits):0]

Table 1521. pulp-platform_common_cells/src/cc_credit_counter.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
clr_i	input	logic
credit_o	output	credit_cnt_t
credit_give_i	input	logic
credit_take_i	input	logic
credit_left_o	output	logic
credit_crit_o	output	logic
credit_full_o	output	logic

API — cc_delta_counter.sv

Table 1522. pulp-platform_common_cells/src/cc_delta_counter.sv — module / interface / package (1개)

이름	종류
cc_delta_counter	module

Table 1523. pulp-platform_common_cells/src/cc_delta_counter.sv — parameter (2개)

파라미터	형/폭	기본값
Width	int unsigned	4
StickyOverflow	bit	1'b0

Table 1524. pulp-platform_common_cells/src/cc_delta_counter.sv — 포트 (10개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
clr_i	input	logic
en_i	input	logic
load_i	input	logic
down_i	input	logic
delta_i	input	logic [Width-1:0]
d_i	input	logic [Width-1:0]
q_o	output	logic [Width-1:0]
overflow_o	output	logic

API — cc_ecc_decode.sv**Table 1525. pulp-platform_common_cells/src/cc_ecc_decode.sv — module / interface / package (1개)**

이름	종류
cc_ecc_decode	module

Table 1526. pulp-platform_common_cells/src/cc_ecc_decode.sv — parameter (1개)

파라미터	형/폭	기본값
DataWidth	int unsigned	64

Table 1527. pulp-platform_common_cells/src/cc_ecc_decode.sv — localparam (4개)

상수	형/폭	값
data_t		logic [DataWidth-1:0]
parity_t		logic [ecc_get_parity_width(DataWidth)-1:0]
code_word_t		logic [ecc_get_cw_width(DataWidth)-1:0]
encoded_data_t		struct packed {

Table 1528. pulp-platform_common_cells/src/cc_ecc_decode.sv — 포트 (6개)

포트	방향	형/폭
data_i	input	encoded_data_t
data_o	output	data_t
syndrome_o	output	parity_t
single_error_o	output	logic
parity_error_o	output	logic
double_error_o	output	logic

API — cc_ecc_encode.sv

Table 1529. pulp-platform_common_cells/src/cc_ecc_encode.sv — module / interface / package (1개)

이름	종류
cc_ecc_encode	module

Table 1530. pulp-platform_common_cells/src/cc_ecc_encode.sv — parameter (1개)

파라미터	형/폭	기본값
DataWidth	int unsigned	64

Table 1531. pulp-platform_common_cells/src/cc_ecc_encode.sv — localparam (4개)

상수	형/폭	값
data_t		logic [DataWidth-1:0]
parity_t		logic [ecc_get_parity_width(DataWidth)-1:0]
code_word_t		logic [ecc_get_cw_width(DataWidth)-1:0]
encoded_data_t		struct packed {

Table 1532. pulp-platform_common_cells/src/cc_ecc_encode.sv — 포트 (2개)

포트	방향	형/폭
data_i	input	data_t
data_o	output	encoded_data_t

API — cc_edge_detect.sv

Table 1533. pulp-platform_common_cells/src/cc_edge_detect.sv — module / interface / package (1개)

이름	종류
cc_edge_detect	module

Table 1534. pulp-platform_common_cells/src/cc_edge_detect.sv — 포트 (5개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
d_i	input	logic

포트	방향	형/폭
re_o	output	logic
detected	output	logic fe_o // Falling edge

API — cc_edge_propagator.sv

Table 1535. pulp-platform_common_cells/src/cc_edge_propagator.sv — module / interface / package (1개)

이름	종류
cc_edge_propagator	module

Table 1536. pulp-platform_common_cells/src/cc_edge_propagator.sv — 포트 (6개)

포트	방향	형/폭
clk_tx_i	input	logic
rst_tx_ni	input	logic
edge_i	input	logic
clk_rx_i	input	logic
rst_rx_ni	input	logic
edge_o	output	logic

API — cc_edge_propagator_ack.sv

Table 1537. pulp-platform_common_cells/src/cc_edge_propagator_ack.sv — module / interface / package (1개)

이름	종류
cc_edge_propagator_ack	module

Table 1538. pulp-platform_common_cells/src/cc_edge_propagator_ack.sv — 포트 (7개)

포트	방향	형/폭
clk_tx_i	input	logic
rst_tx_ni	input	logic
edge_i	input	logic
ack_tx_o	output	logic
clk_rx_i	input	logic
rst_rx_ni	input	logic
edge_o	output	logic

API — cc_edge_propagator_rx.sv

Table 1539. pulp-platform_common_cells/src/cc_edge_propagator_rx.sv — module / interface / package (1개)

이름	종류
cc_edge_propagator_rx	module

Table 1540. pulp-platform_common_cells/src/cc_edge_propagator_rx.sv — 포트 (5개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
valid_i	input	logic
ack_o	output	logic
valid_o	output	logic

API — cc_edge_propagator_tx.sv**Table 1541. pulp-platform_common_cells/src/cc_edge_propagator_tx.sv — module / interface / package (1개)**

이름	종류
cc_edge_propagator_tx	module

Table 1542. pulp-platform_common_cells/src/cc_edge_propagator_tx.sv — 포트 (5개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
valid_i	input	logic
ack_i	input	logic
valid_o	output	logic

API — cc_exp_backoff.sv**Table 1543. pulp-platform_common_cells/src/cc_exp_backoff.sv — module / interface / package (1개)**

이름	종류
cc_exp_backoff	module

Table 1544. pulp-platform_common_cells/src/cc_exp_backoff.sv — parameter (2개)

파라미터	형/폭	기본값
Seed	int unsigned	'hffff
MaxExp	int unsigned	16

Table 1545. pulp-platform_common_cells/src/cc_exp_backoff.sv — localparam (1개)

상수	형/폭	값
WIDTH	int unsigned	16

Table 1546. pulp-platform_common_cells/src/cc_exp_backoff.sv — 포트 (6개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic

포트	방향	형/폭
clr_i	input	logic
set_cnt_i	input	logic
clr_cnt_i	input	logic
is_zero_o	output	logic

1. pulp-platform_common_cells/src/assert_rpt_pkg.sv (17 lines)

```

1 // Copyright lowRISC contributors.
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `ifndef UVM
6 package assert_rpt_pkg;
7
8 import uvm_pkg::*;
9
10 `include "uvm_macros.svh"
11
12 function automatic void assert_rpt(string msg);
13     `uvm_error("ASSERT FAILED", msg)
14 endfunction
15
16 endpackage
17 `endif

```

2. pulp-platform_common_cells/src/cc_addr_decode.sv (114 lines)

```

1 // Copyright 2019 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 // Authors:
12 // - Wolfgang Roenninger <wroennin@ethz.ch>
13 // - Thomas Benz <tbenz@iis.ee.ethz.ch>
14
15 /// Address Decoder: Maps the input address combinatorially to an
  index.
16 /// The address map `addr_map_i` is a packed array of rule_t
  structs.
17 /// The ranges of any two rules may overlap. If so, the rule
  at the higher (more significant)
18 /// position in `addr_map_i` prevails.
19 ///
20 /// There can be an arbitrary number of address rules. There
  can be multiple
21 /// ranges defined for the same index. The start address has
  to be
22 /// less than the end address.
23 ///
24 /// There is the possibility to add a default mapping:
25 /// `en_default_idx_i`: Driving this port to `1'b1` maps all
  input addresses
26 /// for which no rule in `addr_map_i` exists to the default
  index
27 /// specified by
28 /// `default_idx_i`. In this case, `dec_error_o` is always
  `1'b0`.
29 ///
30 ///
31 /// The `Napot` parameter allows using naturally-aligned power of
  two (NAPOT) regions,
32 /// using base addresses and masks instead of address ranges
  to specify rules.
33 ///
34 ///
35 /// Assertions: The module checks every time there is a change in
  the address mapping
36 /// if the resulting map is valid. It fatals if `start_addr`
  is higher than `end_addr` (non-NAPOT

```

```

33 /// only) or if a mapping targets an index that is outside the
  number of allowed indices.
34 /// It issues warnings if the address regions of any two map-
  pings overlap (non-NAPOT only).
35 module cc_addr_decode #(
36     /// Highest index which can happen in a rule.
37     parameter int unsigned NoIndices = 32'd0,
38     /// Total number of rules.
39     parameter int unsigned NoRules = 32'd1,
40     /// Address type inside the rules and to decode.
41     parameter type addr_t = logic,
42     /// Whether this is a NAPOT (base and mask) or regular range
  decoder
43     parameter bit Napot = 0,
44     /// The output index type `idx_t` can be specified either
  with the width `IdxWidth`
45     /// or directly with the type `idx_t`. By default, it will
  use the
46     /// maximum index
47     parameter int unsigned IdxWidth =
  cc_pkg::idx_width(NoIndices),
48     parameter type idx_t = logic [IdxWidth-1:0],
49     /// Rule packed struct type.
50     /// The address decoder expects three fields in `rule_t`:
51     ///
52     /// typedef struct packed {
53     ///     idx_t idx;
54     ///     addr_t start_addr;
55     ///     addr_t end_addr;
56     /// } rule_t;
57     ///
58     /// - `idx`: The index to be returned for a matching
  rule. Usually an integer but can
59     /// be any type of data.
60     /// - `start_addr`: start address of the range the rule de-
  scribes, value is included in range
61     /// - `end_addr`: end address of the range the rule de-
  scribes, value is NOT included in range
62     /// if `end_addr == '0` end of address
  space is assumed
63     ///
64     /// If `Napot` is 1, The field names remain the same, but
  the rule describes a naturally-aligned
65     /// power of two (NAPOT) region instead of an address range:
  `start_addr` becomes the base address
66     /// and `end_addr` the mask. See the wrapping module `cc_ad-
  dr_decode_napot` for details.
67     /// verilog_lint: waive typedef-structs-unions
68     parameter type rule_t = struct packed {
69         idx_t idx;
70         addr_t start_addr;
71         addr_t end_addr;
72     }
73 ) (
74     /// Address to decode.
75     input addr_t addr_i,
76     /// Address map: rule with the highest array position wins
  on collision
77     input rule_t [NoRules-1:0] addr_map_i,
78     /// Decoded index.
79     output idx_t idx_o,
80     /// Decode is valid.
81     output logic dec_valid_o,
82     /// Decode is not valid, no matching rule found.
83     output logic dec_error_o,
84     /// Enable default port mapping.
85     ///
86     /// When not used, tie to `0`.
87     input logic en_default_idx_i,
88     /// Default port index.
89     ///
90     /// When `en_default_idx_i` is `1`, this will be the index
  when no rule matches.
91     ///
92     /// When not used, tie to `0`.
93     input idx_t default_idx_i
94 );
95
96 /// wraps the dynamic configuration version of the address
  decoder
97 cc_addr_decode_dyn #(
98     .NoRules ( NoRules ),

```

```

99     .addr_t      ( addr_t      ),
100    .rule_t      ( rule_t      ),
101    .Napot       ( Napot       ),
102    .idx_t       ( idx_t       )
103 ) i_addr_decode_dync (
104     .addr_i,
105     .addr_map_i,
106     .idx_o,
107     .dec_valid_o,
108     .dec_error_o,
109     .en_default_idx_i,
110     .default_idx_i,
111     .config_ongoing_i ( 1'b0 )
112 );
113
114 endmodule

```

3. pulp-platform_common_cells/src/ cc_addr_decode_dync.sv (182 lines)

```

1 // Copyright 2019 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 // Authors:
12 // - Wolfgang Roenninger <wroennin@ethz.ch>
13 // - Paul Scheffler <paulsc@iis.ee.ethz.ch>
14 // - Michael Rogenmoser <michaelr@iis.ee.ethz.ch>
15 // - Thomas Benz <tbenz@iis.ee.ethz.ch>
16
17 `include "common_cells/assertions.svh"
18
19 /// Address Decoder: Maps the input address combinatorially to an
  index.
20 /// DYnamic Configuration (DYNc) version
21 /// The address map `addr_map_i` is a packed array of rule_t
  structs.
22 /// The ranges of any two rules may overlap. If so, the rule
  at the higher (more significant)
23 /// position in `addr_map_i` prevails.
24 ///
25 /// There can be an arbitrary number of address rules. There
  can be multiple
26 /// ranges defined for the same index. The start address has
  to be less
  than the end address.
27 ///
28 /// There is the possibility to add a default mapping:
29 /// `en_default_idx_i`: Driving this port to `1'b1` maps all
  input addresses
30 /// for which no rule in `addr_map_i` exists to the default
  index specified by
31 /// `default_idx_i`. In this case, `dec_error_o` is always
  `1'b0`.
32 ///
33 /// The `Napot` parameter allows using naturally-aligned power of
  two (NAPOT) regions,
34 /// using base addresses and masks instead of address ranges
  to specify rules.
35 ///
36 /// Assertions: The module checks every time there is a change in
  the address mapping
37 /// if the resulting map is valid. It fatals if `start_addr`
  is higher than `end_addr` (non-NAPOT
38 /// only) or if a mapping targets an index that is outside the
  number of allowed indices.
39 /// It issues warnings if the address regions of any two map-
  pings overlap (non-NAPOT only).
40 module cc_addr_decode_dync #(
41     /// Highest index which can happen in a rule.
42     parameter int unsigned NoIndices = 32'd0,
43     /// Total number of rules.
44     parameter int unsigned NoRules  = 32'd1,
45     /// Address type inside the rules and to decode.
46     parameter type      addr_t      = logic,
47     /// Whether this is a NAPOT (base and mask) or regular range
  decoder
48     parameter bit      Napot       = 0,
49     /// The output index type `idx_t` can be specified either
  with the width `IdxWidth`
50     /// or directly with the type `idx_t`. By default, it will

```

```

  use the maximum index
51     /// `NoIndices` to calculate the required width.
52     parameter int unsigned IdxWidth =
  cc_pkg::idx_width(NoIndices),
53     parameter type      idx_t      = logic [IdxWidth-1:0],
54     /// Rule packed struct type.
55     /// The address decoder expects three fields in `rule_t`:
56     ///
57     /// typedef struct packed {
58     ///     idx_t idx;
59     ///     addr_t start_addr;
60     ///     addr_t end_addr;
61     /// } rule_t;
62     ///
63     /// - `idx`: The index to be returned for a matching
  rule. Usually an integer but can
64     /// be any type of data.
65     /// - `start_addr`: start address of the range the rule de-
  scribes, value is included in range
66     /// - `end_addr`: end address of the range the rule de-
  scribes, value is NOT included in range
67     /// if `end_addr == '0` end of address
  space is assumed
68     ///
69     /// If `Napot` is 1, The field names remain the same, but
  the rule describes a naturally-aligned
70     /// power of two (NAPOT) region instead of an address range:
  `start_addr` becomes the base address
71     /// and `end_addr` the mask. See the wrapping module `cc_ad-
  dr_decode_napot` for details.
72     /// verilog_lint: waive typedef-structs-unions
73     parameter type      rule_t      = struct packed {
74         idx_t idx;
75         addr_t start_addr;
76         addr_t end_addr;
77     }
78 ) (
79     /// Address to decode.
80     input addr_t      addr_i,
81     /// Address map: rule with the highest array position wins
  on collision
82     input rule_t [NoRules-1:0] addr_map_i,
83     /// Decoded index.
84     output idx_t      idx_o,
85     /// Decode is valid.
86     output logic      dec_valid_o,
87     /// Decode is not valid, no matching rule found.
88     output logic      dec_error_o,
89     /// Enable default port mapping.
90     ///
91     /// When not used, tie to `0`.
92     input logic      en_default_idx_i,
93     /// Default port index.
94     ///
95     /// When `en_default_idx_i` is `1`, this will be the index
  when no rule matches.
96     ///
97     /// When not used, tie to `0`.
98     input idx_t      default_idx_i,
99     /// The module is dynamically configured, this deactivates
  its output and the integrated
100    /// assertions.
101    input logic      config_ongoing_i
102 );
103
104 logic [NoRules-1:0] matched_rules; // purely for address map
  debugging
105
106 always_comb begin
107     // default assignments
108     matched_rules = '0;
109     dec_valid_o   = 1'b0;
110     dec_error_o   = (en_default_idx_i) ? 1'b0 : 1'b1;
111     idx_o         = (en_default_idx_i) ? default_idx_i : '0;
112
113     // match the rules
114     for (int unsigned i = 0; i < NoRules; i++) begin
115         if (
116             !Napot && (addr_i >= addr_map_i[i].start_addr) &&
117             ((addr_i < addr_map_i[i].end_addr) ||
118             (addr_map_i[i].end_addr == '0)) ||
119             Napot && (addr_map_i[i].start_addr &
120             addr_map_i[i].end_addr) ==
121             (addr_i & addr_map_i[i].end_addr)
122         ) begin
123             matched_rules[i] = ~config_ongoing_i;
124             dec_valid_o      = ~config_ongoing_i;
125             dec_error_o      = 1'b0;
126             idx_o            = config_ongoing_i ? default_idx_i :
127             addr_map_i[i].idx;
128         end
129     end
130
131     // Assumptions and assertions
132     `ifndef COMMON_CELLS_ASSERTS_OFF
133     `ifndef SYNTHESIS
134     initial begin : proc_check_parameters

```

```

133 `ASSUME_I(addr_width_mismatch, $bits(addr_i) ==
$bits(addr_map_i[0].start_addr),
134 $formatf("Input address has %d bits and address
map has %d bits.",
135 $bits(addr_i),
$bits(addr_map_i[0].start_addr)))
136 `ASSUME_I(norules_0, NoRules > 0, $formatf("At least one
rule needed"))
137 end
138
139 `ASSERT_FINAL(more_than_1_bit_set, $onehot0(matched_rules)
|| config_ongoing_i,
140 "More than one bit set in the one-hot signal,
matched_rules")
141
142 // These following assumptions check the validity of the ad-
dress map.
143 // The assumptions gets generated for each distinct pair of
rules.
144 // Each assumption is present two times, as they rely on one
rules being
145 // effectively ordered. Only one of the rules with the same
function is
146 // active at a time for a given pair.
147 // check_start: Enforces a smaller start than end ad-
dress.
148 // check_idx: Enforces a valid index in the rule.
149 // check_overlap: Warns if there are overlapping ad-
dress regions.
150 always_comb begin : proc_check_addr_map
151 if (!$isunknown(addr_map_i) && ~config_ongoing_i) begin
152 for (int unsigned i = 0; i < NoRules; i++) begin
153 `ASSUME_I(check_start, Napot ||
addr_map_i[i].start_addr < addr_map_i[i].end_addr ||
154 addr_map_i[i].end_addr == '0,
155 $formatf("This rule has a higher start than end ad-
dress!!!\n\
156 Violating rule %d.\n\
157 Rule> IDX: %h START: %h END: %h\n\
158
#####",
159 i, addr_map_i[i].idx, addr_map_i[i].start_addr,
addr_map_i[i].end_addr))
160 for (int unsigned j = i + 1; j < NoRules; j++) begin
161 // overlap check
162 `ASSUME_I(check_overlap, Napot ||
163 !((addr_map_i[j].start_addr
< addr_map_i[i].end_addr) &&
164 (addr_map_i[j].end_addr >
addr_map_i[i].start_addr)) ||
165 !((addr_map_i[i].end_addr ==
'0) &&
166 (addr_map_i[j].end_addr >
addr_map_i[i].start_addr)) ||
167 !((addr_map_i[j].start_addr
< addr_map_i[i].end_addr) &&
168 (addr_map_i[j].end_addr ==
'0))),
169 $formatf("Overlapping address region found!!!
\n\
170 Rule %d: IDX: %h START: %h END: %h\n\
171 Rule %d: IDX: %h START: %h END: %h\n\
172
#####",
173 i, addr_map_i[i].idx, addr_map_i[i].start_addr,
addr_map_i[i].end_addr,
174 j, addr_map_i[j].idx, addr_map_i[j].start_addr,
addr_map_i[j].end_addr))
175 end
176 end
177 end
178 end
179 `endif
180 `endif
181
182 endmodule

```

4. pulp-platform_common_cells/src/ cc_addr_decode_napot.v (102 lines)

```

1 // Copyright 2019 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-

```

der the License.

```

10
11 // Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
12 // Author: Paul Scheffler <paulsc@iis.ee.ethz.ch>
13
14 /// This module wraps `cc_addr_decode_dync` in its naturally-
aligned power of two (NAPOT) variant,
15 /// alleviating the need to set the `Napot` parameter and us-
ing more descriptive `rule_t`
16 /// field names. See the `cc_addr_decode` documentation for de-
tails.
17 module cc_addr_decode_napot #(
18 /// Highest index which can happen in a rule.
19 parameter int unsigned NoIndices = 32'd0,
20 /// Total number of rules.
21 parameter int unsigned NoRules = 32'd1,
22 /// Address type inside the rules and to decode.
23 parameter type addr_t = logic,
24 /// The output index type `idx_t` can be specified either
with the width `IdxWidth`
25 /// or directly with the type `idx_t`. By default, it will
use the maximum index
26 /// `NoIndices` to calculate the required width.
27 parameter int unsigned IdxWidth =
cc_pkg::idx_width(NoIndices),
28 parameter type idx_t = logic [IdxWidth-1:0],
29 /// Rule packed struct type.
30 /// The NAPOT address decoder expects three fields in
`rule_t`:
31 ///
32 /// typedef struct packed {
33 ///     idx_t     idx;
34 ///     addr_t     base;
35 ///     addr_t     mask;
36 /// } rule_t;
37 ///
38 /// - `idx`: index of the rule, has to be < `NoIndices`.
39 /// - `base`: base address whose specified bits should
match `addr_i`.
40 /// - `mask`: set for bits which are to be checked to de-
termine a match.
41 // verilog lint: waive typedef-structs-unions
42 parameter type rule_t = struct packed {
43     idx_t     idx;
44     addr_t     base;
45     addr_t     mask;
46 }
47 ) (
48 /// Address to decode.
49 input addr_t addr_i,
50 /// Address map: rule with the highest array position wins
on collision
51 input rule_t [NoRules-1:0] addr_map_i,
52 /// Decoded index.
53 output idx_t idx_o,
54 /// Decode is valid.
55 output logic dec_valid_o,
56 /// Decode is not valid, no matching rule found.
57 output logic dec_error_o,
58 /// Enable default port mapping.
59 ///
60 /// When not used, tie to `0`.
61 input logic en_default_idx_i,
62 /// Default port index.
63 ///
64 /// When `en_default_idx_i` is `1`, this will be the index
when no rule matches.
65 ///
66 /// When not used, tie to `0`.
67 input idx_t default_idx_i
68 );
69
70 // Rename struct field names to those expected by `cc_ad-
dr_decode`
71 typedef struct packed {
72     idx_t idx;
73     addr_t start_addr;
74     addr_t end_addr;
75 } rule_range_t;
76
77 rule_range_t [NoRules-1:0] range_addr_map;
78
79 for (genvar i = 0; i < NoRules; i++) begin : gen_rule_assign
80 assign range_addr_map[i].idx = addr_map_i[i].idx;
81 assign range_addr_map[i].start_addr = addr_map_i[i].base;
82 assign range_addr_map[i].end_addr = addr_map_i[i].mask;
83 end
84
85 cc_addr_decode_dync #(
86     .NoIndices ( NoIndices ),
87     .NoRules ( NoRules ),
88     .addr_t ( addr_t ),
89     .rule_t ( rule_range_t ),
90     .Napot ( 1 )
91 ) i_addr_decode_dync (
92     .addr_i,
93     .addr_map_i ( range_addr_map ),
94     .idx_o,
95     .dec_valid_o,
96     .dec_error_o,
97     .en_default_idx_i,

```



```

98     .default_idx_i,
99     .config_ongoing_i ( 1'b0 )
100 );
101
102 endmodule

```

5. pulp-platform_common_cells/src/cc_binary_to_gray.sv (25 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 //
3 // Copyright and related rights are licensed under the Solder-
4 // pad Hardware
5 //
6 // License, Version 0.51 (the "License"); you may not use this
7 // file except in
8 // compliance with the License. You may obtain a copy of the
9 // License at
10 // http://solderpad.org/licenses/SHL-0.51. Unless required by
11 // applicable law
12 // or agreed to in writing, software, hardware and materials
13 // distributed under
14 // this License is distributed on an "AS IS" BASIS, WITHOUT
15 // WARRANTIES OR
16 // CONDITIONS OF ANY KIND, either express or implied. See the
17 // License for the
18 // specific language governing permissions and limitations un-
19 // der the License.
20 //
21 // Fabian Schuiki <fschuiki@iis.ee.ethz.ch>
22 //
23 /// A binary to gray code converter.
24 module cc_binary_to_gray #(
25     parameter int unsigned Width = 1
26 ) (
27     input  logic [Width-1:0] a_i,
28     output logic [Width-1:0] z_o
29 );
30     logic [Width:0] a_ext;
31     assign a_ext = {1'b0, a_i};
32     assign z_o = a_i ^ a_ext[Width:1];
33 endmodule

```

6. pulp-platform_common_cells/src/cc_boxcar.sv (31 lines)

```

1 // Copyright 2025 ETH Zurich and University of Bologna.
2 // Solderpad Hardware License, Version 0.51, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: SHL-0.51
5 //
6 // Author: Luca Colagrande <colluca@iis.ee.ethz.ch>
7 //
8 // This module can be used to generate any mask that can be
9 // obtained
10 // by a boxcar function (https://en.wikipedia.org/wiki/Box-
11 // car_function).
12 // Specifically, it generates a mask with all and only the
13 // bits in
14 // the {lsb_i, msb_i} interval asserted.
15 //
16 module cc_boxcar #(
17     parameter int unsigned Width = 32,
18     /// Derived parameter *Do not override*
19     localparam int unsigned IdxWidth =
20 cc_pkg::idx_width(Width),
21     localparam type idx_t = logic [IdxWidth-1:0],
22     localparam type mask_t = logic [Width-1:0]
23 ) (
24     input idx_t lsb_i,
25     input idx_t msb_i,
26     output mask_t mask_o
27 );
28
29     mask_t low_mask, high_mask_n;
30
31     cc_heaviside #(.Width(Width)) i_lo
32     (.x_i(lsb_i), .mask_o(low_mask));
33     cc_heaviside #(.Width(Width)) i_hi
34     (.x_i(msb_i), .mask_o(high_mask_n));
35
36     assign mask_o = ~low_mask & high_mask_n;
37 endmodule

```

7. pulp-platform_common_cells/src/cc_cb_filter.sv (248 lines)

```

1 // Copyright (c) 2019 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
3 // pad Hardware
4 //
5 // License, Version 0.51 (the "License"); you may not use this
6 // file except in
7 // compliance with the License. You may obtain a copy of the
8 // License at
9 // http://solderpad.org/licenses/SHL-0.51. Unless required by
10 // applicable law
11 // or agreed to in writing, software, hardware and materials
12 // distributed under

```

```

7 // this License is distributed on an "AS IS" BASIS, WITHOUT
8 // WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
10 // License for the
11 // specific language governing permissions and limitations un-
12 // der the License.
13 //
14 // Author: Wolfgang Roenninger <wroennin@ethz.ch>
15 //
16 // `cc_cb_filter`: This module implements a counting bloom
17 // filter with parameterizable hash functions.
18 //
19 // Functionality: A counting bloom filter is a data structure
20 // to efficiently implement
21 // set lookups. It does so by hashing its data
22 // inputs onto multiple pointers
23 // which serve as indicators for an array of
24 // buckets. For lookups can be
25 // false positives, but no false negatives.
26 // - Seeding: The pseudo random generators need seeds at
27 // elaboration time to generate
28 // different hashes. In principle any combina-
29 // tion of seeds can be used.
30 // But one should look that the hash outputs
31 // give sufficient different patterns,
32 // such that the resulting collision rate is
33 // low. The package `cc`
34 //
35 // contains the struct for seeding the PRG's in
36 // the hash functions.
37 // - Lookup:
38 // - Ports: `look_data_i`, `look_valid_o`
39 // - Description: Lookup combinational, `look_valid_o` is
40 // high, when `look_data_i` was
41 // previously put into the filter.
42 // - Increment:
43 // - Ports: `incr_data_i`, `incr_valid_i`
44 // - Description: Put data into the counting bloom filter,
45 // when valid is high.
46 // - Decrement:
47 // - Ports: `decr_data_i`, `decr_valid_i`
48 // - Description: Remove data from the counting bloom fil-
49 // ter. Only remove data that was
50 // previously put in, otherwise will go in a
51 // wrong state.
52 // - Status:
53 // - `clr_i`: Clears the filter and sets all coun-
54 // ters to 0.
55 //
56 // - `filter_usage_o`: How many data items are currently in
57 // the filter.
58 // - `filter_full_o`: Filter is full, can no longer hold
59 // more items.
60 // - `filter_empty_o`: Filter is empty.
61 // - `filter_error_o`: One of the internal counters or
62 // buckets overflowed.
63 //
64 // include "common_cells/assertions.svh"
65 //
66 // This is a counting bloom filter
67 module cc_cb_filter #(
68     parameter int unsigned KHashes = 32'd3, // Number of
69     hash functions
70     parameter int unsigned HashWidth = 32'd4, // Number of
71     counters is 2**HashWidth
72     parameter int unsigned HashRounds = 32'd1, // Number of
73     permutation substitution rounds
74     parameter int unsigned InpWidth = 32'd32, // Input data
75     width
76     parameter int unsigned BucketWidth = 32'd4, // Width of
77     Bucket counters
78     // the seeds used for seeding the PRG's inside each hash,
79     one `cb_seed_t` per hash function.
80     parameter cc_pkg::cb_seed_t [KHashes-1:0] Seeds =
81 cc_pkg::CbEgSeeds
82 ) (
83     input logic clk_i, // Clock
84     input logic rst_ni, // Asynchronous reset
85     active low
86     input logic clr_i, // Synchronous clear
87     active high
88     // data lookup
89     input logic [InpWidth-1:0] look_data_i,
90     output logic look_valid_o,
91     // data increment
92     input logic [InpWidth-1:0] incr_data_i,
93     input logic incr_valid_i,
94     // data decrement
95     input logic [InpWidth-1:0] decr_data_i,
96     input logic decr_valid_i,
97     // status signals
98     output logic [HashWidth-1:0] filter_usage_o,
99     output logic filter_full_o,
100    output logic filter_empty_o,
101    output logic filter_error_o
102 );
103
104     localparam int unsigned NoCounters = 2**HashWidth;
105
106     // signal declarations
107     logic [NoCounters-1:0] look_ind; // hash function pointers

```

```

77 logic [NoCounters-1:0] incr_ind; // hash function pointers
78 logic [NoCounters-1:0] decr_ind; // hash function pointers
79 // bucket (counter signals)
80 logic [NoCounters-1:0] bucket_en;
81 logic [NoCounters-1:0] bucket_down;
82 logic [NoCounters-1:0] bucket_occupied;
83 logic [NoCounters-1:0] bucket_overflow;
84 logic [NoCounters-1:0] bucket_full;
85 logic [NoCounters-1:0] bucket_empty;
86 // membership lookup signals
87 logic [NoCounters-1:0] data_in_bucket;
88 // tot count signals (filter usage)
89 logic cnt_en;
90 logic cnt_down;
91 logic cnt_overflow;
92
93 // -----
94 // Lookup Hash - Membership Detection
95 // -----
96 cc_hash_block #(
97     .NoHashes      ( KHashes      ),
98     .InpWidth       ( InpWidth      ),
99     .HashWidth      ( HashWidth     ),
100    .NoRounds       ( HashRounds    ),
101    .Seeds           ( Seeds         ),
102    ) i_look_hashes (
103        .data_i      ( look_data_i ),
104        .indicator_o  ( look_ind   )
105    );
106 assign data_in_bucket = look_ind & bucket_occupied;
107 assign look_valid_o   = (data_in_bucket == look_ind) ?
1'b1 : 1'b0;
108
109 // -----
110 // Increment Hash - Add Member to Set
111 // -----
112 cc_hash_block #(
113     .NoHashes      ( KHashes      ),
114     .InpWidth       ( InpWidth      ),
115     .HashWidth      ( HashWidth     ),
116     .NoRounds       ( HashRounds    ),
117     .Seeds          ( Seeds         ),
118    ) i_incr_hashes (
119        .data_i      ( incr_data_i ),
120        .indicator_o  ( incr_ind   )
121    );
122
123 // -----
124 // Decrement Hash - Remove Member from Set
125 // -----
126 cc_hash_block #(
127     .NoHashes      ( KHashes      ),
128     .InpWidth       ( InpWidth      ),
129     .HashWidth      ( HashWidth     ),
130     .NoRounds       ( HashRounds    ),
131     .Seeds          ( Seeds         ),
132    ) i_decr_hashes (
133        .data_i      ( decr_data_i ),
134        .indicator_o  ( decr_ind   )
135    );
136
137 // -----
138 // Control the incr/decr of buckets
139 // -----
140 assign bucket_down = decr_valid_i ? decr_ind : '0;
141
142 always_comb begin : proc_bucket_control
143     case ({incr_valid_i, decr_valid_i})
144         2'b00 : bucket_en = '0;
145         2'b10 : bucket_en = incr_ind;
146         2'b01 : bucket_en = decr_ind;
147         2'b11 : bucket_en = incr_ind ^ decr_ind;
148         default: bucket_en = '0; // unreachable
149     endcase
150 end
151
152 // -----
153 // Counters
154 // -----
155 for (genvar i = 0; i < NoCounters; i++) begin : gen_buckets
156     logic [BucketWidth-1:0] bucket_content;
157     cc_counter #(
158         .Width( BucketWidth )
159     ) i_bucket (
160         .clk_i      ( clk_i      ),
161         .rst_ni     ( rst_ni     ),
162         .clr_i      ( clr_i      ),
163         .en_i       ( bucket_en[i] ),
164         .load_i     ( '0         ),
165         .down_i     ( bucket_down[i] ),
166         .d_i        ( '0         ),
167         .q_o        ( bucket_content ),
168         .overflow_o  ( bucket_overflow[i] )
169     );
170     assign bucket_full[i]   = bucket_overflow[i] | (&bucket_content);
171     assign bucket_occupied[i] = |bucket_content;
172     assign bucket_empty[i]  = ~bucket_occupied[i];
173 end
174
175 // -----

```

```

176 // Filter tot item counter
177 // -----
178 assign cnt_en   = incr_valid_i ^ decr_valid_i;
179 assign cnt_down = decr_valid_i;
180 cc_counter #(
181     .Width( HashWidth )
182 ) i_tot_count (
183     .clk_i      ( clk_i      ),
184     .rst_ni     ( rst_ni     ),
185     .clr_i      ( clr_i      ),
186     .en_i       ( cnt_en     ),
187     .load_i     ( '0         ),
188     .down_i     ( cnt_down   ),
189     .d_i        ( '0         ),
190     .q_o        ( filter_usage_o ),
191     .overflow_o  ( cnt_overflow )
192 );
193
194 // -----
195 // Filter Output Flags
196 // -----
197 assign filter_full_o   = |bucket_full;
198 assign filter_empty_o  = &bucket_empty;
199 assign filter_error_o  = |bucket_overflow | cnt_overflow;
200 endmodule
201
202 // gives out the or 'onehots' of all hash functions
203 module cc_hash_block #(
204     parameter int unsigned NoHashes
205     = 32'd3,
206     parameter int unsigned InpWidth
207     = 32'd11,
208     parameter int unsigned HashWidth
209     = 32'd5,
210     parameter int unsigned NoRounds
211     = 32'd1,
212     parameter cc_pkg::cb_seed_t [NoHashes-1:0] Seeds =
213     cc_pkg::CbEgSeeds
214 ) (
215     input logic [InpWidth-1:0] data_i,
216     output logic [2*HashWidth-1:0] indicator_o
217 );
218
219 logic [NoHashes-1:0][2*HashWidth-1:0] hashes;
220
221 for (genvar i = 0; i < NoHashes; i++) begin : gen_hashes
222     cc_sub_per_hash #(
223         .InpWidth      ( InpWidth      ),
224         .HashWidth     ( HashWidth     ),
225         .NoRounds      ( NoRounds      ),
226         .PermuteKey    ( Seeds[i].PermuteSeed ),
227         .XorKey        ( Seeds[i].XorSeed )
228     ) i_hash (
229         .data_i      ( data_i      ),
230         .hash_o      (             ), // not used, because we
231         want the onehot
232         .hash_onehot_o ( hashes[i] )
233     );
234 end
235
236 // output assignment
237 always_comb begin : proc_hash_or
238     indicator_o = '0;
239     for (int unsigned j = 0; j < NoHashes; j++) begin
240         indicator_o = indicator_o | hashes[j];
241     end
242 end
243
244 `ifndef COMMON_CELLS_ASSERTS_OFF
245 `ifndef SYNTHESIS
246 // assertions
247 initial begin
248     `ASSUME_I(hash_conf, InpWidth > HashWidth,
249
250     $formatf("%m:\nA Hash Function reduces the width of the
251     input>\nInpWidth: %s\nOUT_WIDTH: %s",
252     InpWidth, HashWidth))
253 end
254 `endif
255 `endif
256 endmodule

```

8. pulp-platform_common_cells/src/cc_cdc_2phase.sv (206 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 //
3 // Copyright and related rights are licensed under the Solder-
4 pad Hardware
5
6 // License, Version 0.51 (the "License"); you may not use this
7 file except in
8 // compliance with the License. You may obtain a copy of the
9 License at
10 // http://solderpad.org/licenses/SHL-0.51. Unless required by
11 applicable law
12 // or agreed to in writing, software, hardware and materials
13 distributed under
14 // this License is distributed on an "AS IS" BASIS, WITHOUT
15 WARRANTIES OR

```

```

9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations under
the License.
11 //
12 // Fabian Schuiki <fschuiki@iis.ee.ethz.ch>
13
14 /// A two-phase clock domain crossing.
15 ///
16 /// # Reset Behavior!!
17 ///
18 /// This module must not be used if warm reset capability is a
requirement. The
19 /// only exception is if you consistently use a reset controller
that sequences
20 /// the resets while gating both clock domains (be very careful
if you follow
21 /// this strategy!). If you need warm reset/clear/flush capabilities,
use (AND
22 /// CAREFULLY READ THE DESCRIPTION) the
cc_cdc_2phase_clearable module.
23 ///
24 /// After this disclaimer, here is how you connect the
src_rst_ni and the
25 /// dst_rst_ni of this module for power-on-reset (POR). The
src_rst_ni and
26 /// dst_rst_ni signal must be asserted SIMULTANEOUSLY (i.e.
asynchronous
27 /// assertion). Otherwise, spurious transactions could occur
in the domain
28 /// where the reset arrives later than the other. The de-assertion
of both reset
29 /// must be synchronized to their respective clock domain
(i.e. src_rst_ni must
30 /// be deasserted synchronously to the src_clk_i and
dst_rst_ni must be
31 /// deasserted synchronously to dst_clk_i.) You can use the
cc_rstgen cell in the
32 /// common_cells library to achieve this (synchronization of
only the
33 /// de-assertion). However, be careful about reset domain
crossings; If you
34 /// reset both domain asynchronously in their entirety (i.e.
POR) you are fine.
35 /// However, if you use this strategy for warm resets (some
parts of the circuit
36 /// are not reset) you might introduce metastability in this
separate
37 /// reset-domain when you assert the reset (the deassertion
synchronizer doesn't
38 /// help here).
39 ///
40 /// CONSTRAINT: Requires max_delay of min_period(src_clk_i,
dst_clk_i) through
41 /// the paths async_req, async_ack, async_data.
42 `include "common_cells/registers.svh"
43
44 module cc_cdc_2phase #(
45     parameter type data_t = logic,
46     parameter int unsigned SyncStages = 2
47 ) (
48     input logic src_rst_ni,
49     input logic src_clk_i,
50     input data_t src_data_i,
51     input logic src_valid_i,
52     output logic src_ready_o,
53
54     input logic dst_rst_ni,
55     input logic dst_clk_i,
56     output data_t dst_data_o,
57     output logic dst_valid_o,
58     input logic dst_ready_i
59 );
60
61 // Asynchronous handshake signals.
62 (* dont_touch = "true" *) logic async_req;
63 (* dont_touch = "true" *) logic async_ack;
64 (* dont_touch = "true" *) data_t async_data;
65
66 if (SyncStages < 2) begin : gen_sync_stage_assertion
67     $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
68 end
69
70 // The sender in the source domain.
71 cc_cdc_2phase_src #(
72     .data_t ( data_t ),
73     .SyncStages ( SyncStages )
74 ) i_src (
75     .rst_ni ( src_rst_ni ),
76     .clk_i ( src_clk_i ),
77     .data_i ( src_data_i ),
78     .valid_i ( src_valid_i ),
79     .ready_o ( src_ready_o ),
80     .async_req_o ( async_req ),
81     .async_ack_i ( async_ack ),
82     .async_data_o ( async_data )
83 );
84
85 // The receiver in the destination domain.

```

```

86 cc_cdc_2phase_dst #(
87     .data_t ( data_t ),
88     .SyncStages ( SyncStages )
89 ) i_dst (
90     .rst_ni ( dst_rst_ni ),
91     .clk_i ( dst_clk_i ),
92     .data_o ( dst_data_o ),
93     .valid_o ( dst_valid_o ),
94     .ready_i ( dst_ready_i ),
95     .async_req_i ( async_req ),
96     .async_ack_o ( async_ack ),
97     .async_data_i ( async_data )
98 );
99
100 endmodule
101
102
103 /// Half of the two-phase clock domain crossing located in the
source domain.
104 module cc_cdc_2phase_src #(
105     parameter type data_t = logic,
106     parameter int unsigned SyncStages = 2
107 ) (
108     input logic rst_ni,
109     input logic clk_i,
110     input data_t data_i,
111     input logic valid_i,
112     output logic ready_o,
113     output logic async_req_o,
114     input logic async_ack_i,
115     output data_t async_data_o
116 );
117
118 (* dont_touch = "true" *)
119 logic req_src_q, ack_synced;
120 (* dont_touch = "true" *)
121 data_t data_src_q;
122 logic src_accept;
123
124 assign src_accept = valid_i && ready_o;
125
126 if (SyncStages < 2) begin : gen_sync_stage_assertion
127     $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
128 end
129
130 // Synchronize the async ACK.
131 tc_sync #(
132     .Stages ( SyncStages )
133 ) i_sync (
134     .clk_i,
135     .rst_ni,
136     .serial_i ( async_ack_i ),
137     .serial_o ( ack_synced )
138 );
139
140 // The req_src and data_src registers change when a new data
item is accepted.
141 `FFL(req_src_q, ~req_src_q, src_accept, 1'b0, clk_i,
rst_ni)
142
143 `FFL(data_src_q, data_i, src_accept, data_t('0), clk_i,
rst_ni)
144
145 // Output assignments.
146 assign ready_o = (req_src_q == ack_synced);
147 assign async_req_o = req_src_q;
148 assign async_data_o = data_src_q;
149 endmodule
150
151
152 /// Half of the two-phase clock domain crossing located in the
destination
153 /// domain.
154 module cc_cdc_2phase_dst #(
155     parameter type data_t = logic,
156     parameter int unsigned SyncStages = 2
157 ) (
158     input logic rst_ni,
159     input logic clk_i,
160     output data_t data_o,
161     output logic valid_o,
162     input logic ready_i,
163     input logic async_req_i,
164     output logic async_ack_o,
165     input data_t async_data_i
166 );
167
168 (* dont_touch = "true" *)
169 logic req_synced, req_synced_q1, ack_dst_q;
170 (* dont_touch = "true" *)
171 data_t data_dst_q;
172 logic dst_accept, dst_data_load;
173
174 assign dst_accept = valid_o && ready_i;
175 assign dst_data_load = req_synced != req_synced_q1 && !val-

```

```

id_o;
176
177 if (SyncStages < 2) begin : gen_sync_stage_assertion
178     $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
179 end
180
181 // Synchronize the async request.
182 tc_sync #(
183     .Stages ( SyncStages )
184 ) i_sync (
185     .clk_i,
186     .rst_ni,
187     .serial_i ( async_req_i ),
188     .serial_o ( req_synced )
189 );
190
191 // The ack_dst register changes when a new data item is ac-
cepted.
192 `FFL(ack_dst_q, ~ack_dst_q, dst_accept, 1'b0, clk_i, rst_ni)
193
194 // The data_dst register changes when a new data item is
presented. This is
195 // indicated by the async_req line changing levels.
196 `FFL(data_dst_q, async_data_i, dst_data_load, data_t('0),
clk_i, rst_ni)
197
198 // Delayed version of the synchronized request for trans-
199 // action detection.
`FF(req_synced_q1, req_synced, 1'b0, clk_i, rst_ni)
200
201 // Output assignments.
202 assign valid_o = (ack_dst_q != req_synced_q1);
203 assign data_o = data_dst_q;
204 assign async_ack_o = ack_dst_q;
205
206 endmodule

```

9. pulp-platform_common_cells/src/ cc_cdc_2phase_clearable.sv (485 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the License.
11 //
12 // Authors:
13 // - Fabian Schuiki <fschuiki@iis.ee.ethz.ch> (original CDC)
14 // - Manuel Eggimann <meggimann@iis.ee.ethz.ch> (clearability
feature)
15 // - Philippe Sauter <phsauter@iis.ee.ethz.ch> (source/destin-
ation split)
16 //
17 // Description: Clearable Two-Phase CDC
18 // Transfer payloads with a two-phase handshake while coordin-
ating synchronous
19 // and optional asynchronous one-sided clears across source
and destination.
20
21 /// A two-phase clock domain crossing.
22 ///
23 /// CONSTRAINT: Requires max_delay of min_period(src_clk_i,
dst_clk_i) through
24 /// all asynchronous paths between the domains. The payload
CDC uses
25 /// async_payload_req, async_payload_ack, and
async_payload_data for the actual
26 /// data handshake. The clear sequencer CDCs use
async_reset_src2dst_req,
27 /// async_reset_dst2src_ack, async_reset_src2dst_next_phase,
28 /// async_reset_dst2src_req, async_reset_src2dst_ack, and
29 /// async_reset_dst2src_next_phase to handshake the clearing
state machines.
30 ///
31 ///
32 /// Reset Behavior:
33 ///
34 /// In contrast to the cc_cdc_2phase version without clear
signal, this module
35 /// supports one-sided warm resets (asynchronously and syn-
chronously). The way
36 /// this is implemented is described in more detail in the
cc_cdc_reset_ctrlr
37 /// module. To summarize a synchronous clear request i.e. src/

```

```

dst_clear_i will
38 /// cause the respective other clock domain to reset as well
without introducing
39 /// any spurious transactions. This is accomplished by an in-
ternal module
40 /// (cc_cdc_reset_ctrlr) that starts a reset sequence on both
sides of the CDC in
41 /// lock-step that first isolates the CDC from the outside
world and then resets
42 /// it. The reset sequencer provides the following behavior:
43 /// 1. There are no spurious invalid or duplicated transac-
tions regardless how
44 /// the individual sides are reset (can also happen roughly
simultaneously)
45 /// 2. The CDC becomes unready at the src side in the next
cycle after
46 /// synchronous reset request until the reset sequence is
completed. A currently
47 /// pending transactions might still complete (if the dst
accepts at the
48 /// exact time the reset is request on the src die).
49 /// 3. During the reset sequence the dst might withdraw the
valid signal. This
50 /// might violate higher level protocols. If you need this
feature you would
51 /// have to patch the existing implementation to wait with
the isolate_ack
52 /// assertion until all open handshakes were acknowledged.
53 /// 4. If the parameter ClearOnAsyncReset is enabled, the same
behavior as
54 /// above is also valid for asynchronous resets on either
side. However, this
55 /// increases the minimum number of synchronization stages
(SyncStages
56 /// parameter) from 2 to 3 (read the cc_cdc_reset_ctrlr
header to figure out
57 /// why).
58 ///
59 ///
60 `include "common_cells/registers.svh"
61 `include "common_cells/assertions.svh"
62
63 module cc_cdc_2phase_clearable #(
64     parameter type data_t = logic,
65     parameter int unsigned SyncStages = 3,
66     parameter bit ClearOnAsyncReset = 1
67 ) (
68     input logic src_rst_ni,
69     input logic src_clk_i,
70     input logic src_clear_i,
71     output logic src_clear_pending_o,
72     input data_t src_data_i,
73     input logic src_valid_i,
74     output logic src_ready_o,
75
76     input logic dst_rst_ni,
77     input logic dst_clk_i,
78     input logic dst_clear_i,
79     output logic dst_clear_pending_o,
80     output data_t dst_data_o,
81     output logic dst_valid_o,
82     input logic dst_ready_i
83 );
84
85 // Asynchronous payload handshake signals between the source
and destination.
86 (* dont_touch = "true" *) logic async_payload_req;
87 (* dont_touch = "true" *) logic async_payload_ack;
88 (* dont_touch = "true" *) data_t async_payload_data;
89
90 // Asynchronous clear-controller phase handshakes between
the domains.
91 (* dont_touch = "true" *) logic async_reset_src2dst_req;
92 (* dont_touch = "true" *) logic async_reset_dst2src_ack;
93 (* dont_touch = "true" *) cc_pkg::cdc_clear_seq_phase_e
async_reset_src2dst_next_phase;
94 (* dont_touch = "true" *) logic async_reset_dst2src_req;
95 (* dont_touch = "true" *) logic async_reset_src2dst_ack;
96 (* dont_touch = "true" *) cc_pkg::cdc_clear_seq_phase_e
async_reset_dst2src_next_phase;
97
98 // Clear/isolate contract:
99 // - src/dst_clear_pending_o are the local isolate requests.
100 // - local isolate and clear acknowledgements are one-cycle
delayed copies of
101 // the corresponding local requests.
102 // - isolate gates the external valid/ready interface on
both sides.
103
104 // - transactions accepted before or during a clear sequence
may complete or
105 // be dropped, but fresh post-clear traffic must not be
duplicated,
106 // reordered, or corrupted.
107
108 if (ClearOnAsyncReset) begin : gen_elaboration_assertion
109     if (SyncStages < 3)
110         $error("The clearable 2-phase CDC with async reset",

```

```

109     "synchronization requires at least 3 synchronizer
stages for the FIFO.");
110 end else begin : gen_elaboration_assertion
111     if (SyncStages < 2) begin : gen_elaboration_assertion
112         $error("A minimum of 2 synchronizer stages is required
for proper functionality.");
113     end
114 end
115
116 // The source side owns the payload CDC half, local clear/
isolate
117 // acknowledgements, and the local reset-controller half.
118 cc_cdc_2phase_clearable_src #(
119     .data_t          ( data_t          ),
120     .SyncStages      ( SyncStages      ),
121     .ClearOnAsyncReset ( ClearOnAsyncReset )
122 ) i_src (
123     .src_rst_ni
124     .src_clk_i
125     .src_clear_i
126     .src_clear_pending_o
127     .src_clear_pending_o
128     .src_data_i
129     .src_valid_i
130     .src_ready_o
131     (* async *) .async_payload_req_o ( async_payload_req ),
132     (* async *) .async_payload_ack_i ( async_payload_ack ),
133     (* async *) .async_payload_data_o ( async_payload_data ),
134     (* async *) .async_reset_next_phase_o
135     (* async *) .async_reset_req_o
136     (* async *) .async_reset_ack_i
137     (* async *) .async_reset_next_phase_i
138     (* async *) .async_reset_req_i
139     (* async *) .async_reset_ack_o
140     (* async *) .async_reset_src2dst_ack
141 );
142
143 // The destination side owns the payload CDC half, local
144 // clear/isolate acknowledgements, and the local reset-con-
troller half.
145 cc_cdc_2phase_clearable_dst #(
146     .data_t          ( data_t          ),
147     .SyncStages      ( SyncStages      ),
148     .ClearOnAsyncReset ( ClearOnAsyncReset )
149 ) i_dst (
150     .dst_rst_ni
151     .dst_clk_i
152     .dst_clear_i
153     .dst_clear_pending_o
154     .dst_data_o
155     .dst_valid_o
156     .dst_ready_i
157     (* async *) .async_payload_req_i ( async_payload_req ),
158     (* async *) .async_payload_ack_o ( async_payload_ack ),
159     (* async *) .async_payload_data_i ( async_payload_data ),
160     (* async *) .async_reset_next_phase_o
161     (* async *) .async_reset_req_o
162     (* async *) .async_reset_ack_i
163     (* async *) .async_reset_next_phase_i
164     (* async *) .async_reset_req_i
165     (* async *) .async_reset_ack_o
166     (* async *) .async_reset_dst2src_ack
167 );
168 `ifndef COMMON_CELLS_ASSERTS_OFF
169
170 `ASSERT(no_valid_i_during_clear_i, src_clear_i |> !src_val-
id_i, src_clk_i, !src_rst_ni)
171
172 `endif
173 endmodule
174
175 // Destination side for the clearable two-phase CDC.
176 module cc_cdc_2phase_clearable_dst #(

```

```

179     parameter type data_t = logic,
180     parameter int unsigned SyncStages = 2,
181     parameter bit ClearOnAsyncReset = 1
182 ) (
183     input logic dst_rst_ni,
184     input logic dst_clk_i,
185     input logic dst_clear_i,
186     output logic dst_clear_pending_o,
187     output data_t dst_data_o,
188     output logic dst_valid_o,
189     input logic dst_ready_i,
190     input logic async_payload_req_i,
191     output logic async_payload_ack_o,
192     input data_t async_payload_data_i,
193     output cc_pkg::cdc_clear_seq_phase_e
async_reset_next_phase_o,
194     output logic async_reset_req_o,
195     input logic async_reset_ack_i,
196     input cc_pkg::cdc_clear_seq_phase_e
async_reset_next_phase_i,
197     input logic async_reset_req_i,
198     output logic async_reset_ack_o
199 );
200
201 logic dst_clear_req;
202 logic dst_clear_ack_q;
203 logic dst_isolate_req;
204 logic dst_isolate_ack_q;
205 logic dst_valid;
206
207 if (ClearOnAsyncReset) begin : gen_elaboration_assertion
208     if (SyncStages < 3)
209         $error("The clearable 2-phase CDC with async reset",
210
211 "synchronization requires at least 3 synchronizer
stages for the FIFO.");
212 end else begin : gen_elaboration_assertion
213     if (SyncStages < 2) begin : gen_sync_stage_assertion
214         $error("A minimum of 2 synchronizer stages is required
for proper functionality.");
215     end
216 end
217 // Keep the clear-control path one synchronizer stage ahead
of the payload CDC
218 // when async reset propagation is enabled, but never below
two stages.
219 localparam int unsigned ResetSyncStages = (SyncStages > 2) ?
SyncStages - 1 : 2;
220
221 cc_cdc_reset_ctrlr_half #(
222     .SyncStages      ( ResetSyncStages ),
223     .ClearOnAsyncReset ( ClearOnAsyncReset )
224 ) i_cdc_reset_ctrlr_half (
225     .clk_i          ( dst_clk_i          ),
226     .rst_ni         ( dst_rst_ni         ),
227     .clear_i        ( dst_clear_i        ),
228     .clear_o        ( dst_clear_req       ),
229     .clear_ack_i    ( dst_clear_ack_q    ),
230     .isolate_o      ( dst_isolate_req    ),
231     .isolate_ack_i  ( dst_isolate_ack_q  ),
232     (* async *) .async_next_phase_o
233     (* async *) .async_req_o
234     (* async *) .async_ack_i
235     (* async *) .async_next_phase_i
236     (* async *) .async_req_i
237     (* async *) .async_ack_o
238 );
239
240 cc_cdc_2phase_dst_clearable #(
241     .data_t          ( data_t          ),
242     .SyncStages      ( SyncStages      )
243 ) i_dst (
244     .rst_ni         ( dst_rst_ni         ),
245     .clk_i          ( dst_clk_i         ),
246     .clear_i        ( dst_clear_req      ),
247     .data_o         ( dst_data_o         ),
248     .valid_o        ( dst_valid         ),
249     .ready_i        ( dst_ready_i & !dst_isolate_req ),
250     (* async *) .async_req_i ( async_payload_req_i ),
251     (* async *) .async_ack_o ( async_payload_ack_o ),
252     (* async *) .async_data_i ( async_payload_data_i )
253 );
254
255 assign dst_valid_o = dst_valid & !dst_isolate_req;
256
257 // Isolation and clear are acknowledged one cycle after the
local request.
258 `FF(dst_isolate_ack_q, dst_isolate_req, 1'b0, dst_clk_i,
dst_rst_ni)
259 `FF(dst_clear_ack_q, dst_clear_req, 1'b0, dst_clk_i,
dst_rst_ni)
260
261 assign dst_clear_pending_o = dst_isolate_req;

```



```

262
263 endmodule
264
265
266 /// Source side for the clearable two-phase CDC.
267 module cc_cdc_2phase_clearable_src #(
268     parameter type data_t = logic,
269     parameter int unsigned SyncStages = 2,
270     parameter bit ClearOnAsyncReset = 1
271 ) (
272     input logic src_rst_ni,
273     input logic src_clk_i,
274     input logic src_clear_i,
275     output logic src_clear_pending_o,
276     input data_t src_data_i,
277     input logic src_valid_i,
278     output logic src_ready_o,
279     output logic async_payload_req_o,
280     input logic async_payload_ack_i,
281     output data_t async_payload_data_o,
282     output cc_pkg::cdc_clear_seq_phase_e
async_reset_next_phase_o,
283     output logic async_reset_req_o,
284     input logic async_reset_ack_i,
285     input cc_pkg::cdc_clear_seq_phase_e
async_reset_next_phase_i,
286     input logic async_reset_req_i,
287     output logic async_reset_ack_o
288 );
289
290     logic src_clear_req;
291     logic src_clear_ack_q;
292     logic src_ready;
293     logic src_isolate_req;
294     logic src_isolate_ack_q;
295
296     if (ClearOnAsyncReset) begin : gen_elaboration_assertion
297         if (SyncStages < 3)
298             $error("The clearable 2-phase CDC with async reset",
299                 "synchronization requires at least 3 synchronizer
stages for the FIFO.");
300     end else begin : gen_elaboration_assertion
301         if (SyncStages < 2) begin : gen_sync_stage_assertion
302             $error("A minimum of 2 synchronizer stages is required
for proper functionality.");
303         end
304     end
305
306     // Keep the clear-control path one synchronizer stage ahead
of the payload CDC
307     // when async reset propagation is enabled, but never below
two stages.
308     localparam int unsigned ResetSyncStages = (SyncStages > 2) ?
SyncStages - 1 : 2;
309
310     cc_cdc_reset_ctrlr_half #(
311         .SyncStages ( ResetSyncStages ),
312         .ClearOnAsyncReset ( ClearOnAsyncReset )
313     ) i_cdc_reset_ctrlr_half (
314         .clk_i ( src_clk_i ),
315         .rst_ni ( src_rst_ni ),
316         .clear_i ( src_clear_i ),
317         .clear_o ( src_clear_req ),
318         .clear_ack_i ( src_clear_ack_q ),
319         .isolate_o ( src_isolate_req ),
320         .isolate_ack_i ( src_isolate_ack_q ),
321         (* async *) .async_next_phase_o
( async_reset_next_phase_o ),
322         (* async *) .async_req_o
( async_reset_req_o ),
323         (* async *) .async_ack_i
( async_reset_ack_i ),
324         (* async *) .async_next_phase_i
( async_reset_next_phase_i ),
325         (* async *) .async_req_i
( async_reset_req_i ),
326         (* async *) .async_ack_o
( async_reset_ack_o )
327     );
328
329     cc_cdc_2phase_src_clearable #(
330         .data_t ( data_t ),
331         .SyncStages ( SyncStages )
332     ) i_src (
333         .rst_ni ( src_rst_ni ),
334         .clk_i ( src_clk_i ),
335         .clear_i ( src_clear_req ),
336         .data_i ( src_data_i ),
337         .valid_i ( src_valid_i & !src_isolate_req ),
338         .ready_o ( src_ready ),
339         (* async *) .async_req_o ( async_payload_req_o ),
340         (* async *) .async_ack_i ( async_payload_ack_i ),
341         (* async *) .async_data_o ( async_payload_data_o )
342     );
343
344     assign src_ready_o = src_ready & !src_isolate_req;
345
346     // Isolation and clear are acknowledged one cycle after the
local request.

```

```

347     `FF(src_isolate_ack_q, src_isolate_req, 1'b0, src_clk_i,
src_rst_ni)
348     `FF(src_clear_ack_q, src_clear_req, 1'b0, src_clk_i,
src_rst_ni)
349
350     assign src_clear_pending_o = src_isolate_req; // The isolate
signal stays
351     // asserted
352     // clear se-
quence.
353
354 endmodule
355
356
357 /// Half of the two-phase clock domain crossing located in the
source domain.
358 module cc_cdc_2phase_src_clearable #(
359     parameter type data_t = logic,
360     parameter int unsigned SyncStages = 2
361 ) (
362     input logic rst_ni,
363     input logic clk_i,
364     input logic clear_i,
365     input data_t data_i,
366     input logic valid_i,
367     output logic ready_o,
368     output logic async_req_o,
369     input logic async_ack_i,
370     output data_t async_data_o
371 );
372
373     (* dont_touch = "true" *)
374     logic req_src_d, req_src_q, ack_synced;
375     (* dont_touch = "true" *)
376     data_t data_src_d, data_src_q;
377
378     // Synchronize the async ACK
379     tc_sync #(
380         .Stages(SyncStages)
381     ) i_sync(
382         .clk_i,
383         .rst_ni,
384         .serial_i( async_ack_i ),
385         .serial_o( ack_synced )
386     );
387
388     // If we receive the clear signal clear the content of the
request flip-flop
389     // and the data register
390     always_comb begin
391         data_src_d = data_src_q;
392         req_src_d = req_src_q;
393         if (clear_i) begin
394             req_src_d = 1'b0;
395             // The req_src and data_src registers change when a new
data item is accepted.
396         end else if (valid_i && ready_o) begin
397             req_src_d = ~req_src_q;
398             data_src_d = data_i;
399         end
400     end
401
402     `FNR(data_src_q, data_src_d, clk_i)
403
404     `FF(req_src_q, req_src_d, 1'b0, clk_i, rst_ni)
405
406     // Output assignments.
407     assign ready_o = (req_src_q == ack_synced);
408     assign async_req_o = req_src_q;
409     assign async_data_o = data_src_q;
410
411     // Assertions
412     `ifndef COMMON_CELLS_ASSERTS_OFF
413     `ASSUME(no_clear_and_request, clear_i |> ~valid_i, clk_i, !
rst_ni,
414         "No request allowed while clear_i is asserted.")
415     `endif
416
417 endmodule
418
419
420 /// Half of the two-phase clock domain crossing located in the
destination
421 /// domain.
422 module cc_cdc_2phase_dst_clearable #(
423     parameter type data_t = logic,
424     parameter int unsigned SyncStages = 2
425 ) (
426     input logic rst_ni,
427     input logic clk_i,
428     input logic clear_i,
429     output data_t data_o,
430     output logic valid_o,
431     input logic ready_i,
432     input logic async_req_i,
433     output logic async_ack_o,
434     input data_t async_data_i

```

```

435 );
436
437 (* dont_touch = "true" *)
438 (* async_req = "true" *)
439 logic ack_dst_d, ack_dst_q, req_synced, req_synced_q1;
440 (* dont_touch = "true" *)
441 data_t data_dst_d, data_dst_q;
442
443
444 //Synchronize the request
445 tc_sync #(
446   .Stages(SyncStages)
447 ) i_sync(
448   .clk_i,
449   .rst_ni,
450   .serial_i( async_req_i ),
451   .serial_o( req_synced )
452 );
453
454 // The ack_dst register changes when a new data item is ac-
cepted.
455 always_comb begin
456   ack_dst_d = ack_dst_q;
457   if (clear_i) begin
458     ack_dst_d = 1'b0;
459   end else if (valid_o && ready_i) begin
460     ack_dst_d = ~ack_dst_q;
461   end
462 end
463
464 // The data_dst register samples when a new data item is
presented. This is
465 // indicated by a transition in the req_synced line.
466 always_comb begin
467   data_dst_d = data_dst_q;
468   if (req_synced != req_synced_q1 && !valid_o) begin
469     data_dst_d = async_data_i;
470   end
471 end
472
473 `FFNR(data_dst_q, data_dst_d, clk_i)
474
475 `FF(ack_dst_q, ack_dst_d, 1'b0, clk_i, rst_ni)
476 // The req_synced_q1 is the delayed version of the synchron-
ized req_synced
477 // used to detect transitions in the request.
478 `FF(req_synced_q1, req_synced, 1'b0, clk_i, rst_ni)
479
480 // Output assignments.
481 assign valid_o = (ack_dst_q != req_synced_q1);
482 assign data_o = data_dst_q;
483 assign async_ack_o = ack_dst_q;
484
485 endmodule

```

10. pulp-platform_common_cells/src/cc_cdc_4phase.sv (338 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the License.
11 //
12 // Manuel Eggimann <meggimann@iis.ee.ethz.ch>
13
14 /// A 4-phase clock domain crossing. While this is less effi-
cient than a 2-phase
15 /// CDC, it doesn't suffer from the same issues during one
sided resets since
16 /// the IDLE state doesn't alternate with every transaction.
17 ///
18 /// Parameters: data_t - The type of the data to transmit
through the CDC.
19 ///
20 /// Decoupled - If decoupled is disabled, the 4phase cdc will
not consume the
21 /// src item until the handshake with the other side is com-
pleted. This
22 /// increases the latency of the first transaction but has no
effect on
23 /// throughput. However, critical paths might be slightly
longer. Use this mode
24 /// if you want to ensure that there are no in-flight transac-
tions within the
25 /// CDC.

```

```

26 ///
27 /// SendResetMsg - If send reset msg is enabled, the 4phase
cdc starts sending
28 /// the ResetMsg within its' asynchronous reset state. This
can be usefull if
29 /// we need to transmit a message to the other side of the CDC
immediately
30 /// during an async reset even if there is no clock available.
This mode is
31 /// required for proper functionality of the cc_cdc_re-
set_ctrlr module.
32 ///
33 /// SyncStages - Number of synchronization stages for
async_req and async_ack.
34 ///
35 /// CONSTRAINT: Requires max_delay of min_period(src_clk_i,
dst_clk_i) through
36 /// the paths async_req, async_ack, async_data.
37 `include "common_cells/registers.svh"
38
39 module cc_cdc_4phase #(
40   parameter type data_t = logic,
41   parameter bit Decoupled = 1'b1,
42   parameter bit SendResetMsg = 1'b0,
43   parameter data_t ResetMsg = data_t('0),
44   parameter int unsigned SyncStages = 2
45 )(
46   input logic src_rst_ni,
47   input logic src_clk_i,
48   input data_t src_data_i,
49   input logic src_valid_i,
50   output logic src_ready_o,
51
52   input logic dst_rst_ni,
53   input logic dst_clk_i,
54   output data_t dst_data_o,
55   output logic dst_valid_o,
56   input logic dst_ready_i
57 );
58
59 // Asynchronous handshake signals.
60 (* dont_touch = "true" *) logic async_req;
61 (* dont_touch = "true" *) logic async_ack;
62 (* dont_touch = "true" *) data_t async_data;
63
64 if (SyncStages < 2) begin : gen_sync_stage_assertion
65   $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
66 end
67
68 // The sender in the source domain.
69 cc_cdc_4phase_src #(
70   .data_t(data_t),
71   .SyncStages(SyncStages),
72   .Decoupled(Decoupled),
73   .SendResetMsg(SendResetMsg),
74   .ResetMsg(ResetMsg)
75 ) i_src (
76   .rst_ni ( src_rst_ni ),
77   .clk_i ( src_clk_i ),
78   .data_i ( src_data_i ),
79   .valid_i ( src_valid_i ),
80   .ready_o ( src_ready_o ),
81   .async_req_o ( async_req ),
82   .async_ack_i ( async_ack ),
83   .async_data_o ( async_data )
84 );
85
86 // The receiver in the destination domain.
87 cc_cdc_4phase_dst #(
88   .data_t(data_t),
89   .SyncStages(SyncStages),
90   .Decoupled(Decoupled)
91 ) i_dst (
92   .rst_ni ( dst_rst_ni ),
93   .clk_i ( dst_clk_i ),
94   .data_o ( dst_data_o ),
95   .valid_o ( dst_valid_o ),
96   .ready_i ( dst_ready_i ),
97   .async_req_i ( async_req ),
98   .async_ack_o ( async_ack ),
99   .async_data_i ( async_data )
100 );
101 endmodule
102
103
104 /// Half of the 4-phase clock domain crossing located in the
source domain.
105 module cc_cdc_4phase_src #(
106   parameter type data_t = logic,
107   parameter int unsigned SyncStages = 2,
108   parameter bit Decoupled = 1'b1,
109   parameter bit SendResetMsg = 1'b0,
110   parameter data_t ResetMsg = data_t('0)
111 )(
112   input logic rst_ni,
113   input logic clk_i,
114   input data_t data_i,

```



```

115 input logic valid_i,
116 output logic ready_o,
117 output logic async_req_o,
118 input logic async_ack_i,
119 output data_t async_data_o
120 );
121
122 (* dont_touch = "true" *)
123 logic req_src_d, req_src_q;
124 (* dont_touch = "true" *)
125 data_t data_src_d, data_src_q;
126 (* dont_touch = "true" *)
127 logic ack_synced;
128
129 typedef enum logic[1:0] {IDLE, WAIT_ACK_ASSERT, WAIT_ACK_DE-
ASSERT} state_e;
130 state_e state_d, state_q;
131 localparam logic ReqSrcResetValue = SendResetMsg ? 1'b1 :
1'b0;
132 localparam data_t DataSrcResetValue = SendResetMsg ? Re-
setMsg : data_t('0);
133
134 if (SyncStages < 2) begin : gen_sync_stage_assertion
135     $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
136 end
137
138 // Synchronize the async ACK
139 tc_sync #(
140     .Stages(SyncStages)
141 ) i_sync(
142     .clk_i,
143     .rst_ni,
144     .serial_i( async_ack_i ),
145     .serial_o( ack_synced )
146 );
147
148 // Output logic derived from registered local state only.
149 always_comb begin
150     ready_o = 1'b0;
151     case (state_q)
152     IDLE: begin
153         ready_o = Decoupled;
154     end
155     WAIT_ACK_DEASSERT: begin
156         ready_o = !Decoupled && (ack_synced == 1'b0);
157     end
158     default: begin
159         ready_o = 1'b0;
160     end
161     endcase
162 end
163
164 // FSM for the 4-phase handshake; calculates next state from
inputs and local state.
165 always_comb begin
166     state_d = state_q;
167     req_src_d = 1'b0;
168     data_src_d = data_src_q;
169     case (state_q)
170     IDLE: begin
171         // Sample a new item when the valid signal is asser-
ted.
172         if (valid_i) begin
173             data_src_d = data_i;
174             req_src_d = 1'b1;
175             state_d = WAIT_ACK_ASSERT;
176         end
177     end
178     WAIT_ACK_ASSERT: begin
179         req_src_d = 1'b1;
180         if (ack_synced == 1'b1) begin
181             req_src_d = 1'b0;
182             state_d = WAIT_ACK_DEASSERT;
183         end
184     end
185     WAIT_ACK_DEASSERT: begin
186         if (ack_synced == 1'b0) begin
187             state_d = IDLE;
188         end
189     end
190     default: begin
191         state_d = IDLE;
192     end
193     endcase
194 end
195
196 `FF(state_q, state_d, IDLE, clk_i, rst_ni)
197
198 // Sample the data and the request signal to filter combina-
tional glitches
199 `FF(req_src_q, req_src_d, ReqSrcResetValue, clk_i,
rst_ni)
200 `FF(data_src_q, data_src_d, DataSrcResetValue, clk_i,
rst_ni)
201
202 // Async output assignments.
203 assign async_req_o = req_src_q;
204 assign async_data_o = data_src_q;

```

```

205
206 endmodule
207
208
209 /// Half of the 4-phase clock domain crossing located in the
destination
210 /// domain.
211 module cc_cdc_4phase_dst #(
212     parameter type data_t = logic,
213     parameter int unsigned SyncStages = 2,
214     parameter bit Decoupled = 1'b1
215 ) (
216     input logic rst_ni,
217     input logic clk_i,
218     output data_t data_o,
219     output logic valid_o,
220     input logic ready_i,
221     input logic async_req_i,
222     output logic async_ack_o,
223     input data_t async_data_i
224 );
225
226 (* dont_touch = "true" *)
227 logic ack_dst_d, ack_dst_q;
228 (* dont_touch = "true" *)
229 logic req_synced;
230
231 logic data_valid;
232
233 logic output_ready;
234
235
236 typedef enum logic[1:0] {IDLE, WAIT_DOWNSTREAM_ACK,
WAIT_REQ_DEASSERT} state_e;
237 state_e state_d, state_q;
238
239 if (SyncStages < 2) begin : gen_sync_stage_assertion
240     $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
241 end
242
243 //Synchronize the request
244 tc_sync #(
245     .Stages(SyncStages)
246 ) i_sync(
247     .clk_i,
248     .rst_ni,
249     .serial_i( async_req_i ),
250     .serial_o( req_synced )
251 );
252
253 // Output logic derived from registered local state only.
254 always_comb begin
255     data_valid = 1'b0;
256     case (state_q)
257     IDLE: begin
258         data_valid = req_synced;
259     end
260     WAIT_DOWNSTREAM_ACK: begin
261         data_valid = 1'b1;
262     end
263     default: begin
264         data_valid = 1'b0;
265     end
266     endcase
267 end
268
269 // FSM for the 4-phase handshake; calculates next state from
inputs and local state.
270 always_comb begin
271     state_d = state_q;
272     ack_dst_d = 1'b0;
273
274     case (state_q)
275     IDLE: begin
276         // Sample the data upon a new request and transition
to the next state
277         if (req_synced == 1'b1) begin
278             if (output_ready == 1'b1) begin
279                 state_d = WAIT_REQ_DEASSERT;
280             end else begin
281                 state_d = WAIT_DOWNSTREAM_ACK;
282             end
283         end
284     end
285
286     WAIT_DOWNSTREAM_ACK: begin
287         if (output_ready == 1'b1) begin
288             state_d = WAIT_REQ_DEASSERT;
289             ack_dst_d = 1'b1;
290         end
291     end
292
293     WAIT_REQ_DEASSERT: begin
294         ack_dst_d = 1'b1;
295         if (req_synced == 1'b0) begin
296             ack_dst_d = 1'b0;
297             state_d = IDLE;
298         end
299     end
300 end

```

```

299     end
300
301     default: begin
302         state_d = IDLE;
303     end
304 endcase
305 end
306
307 `FF(state_q, state_d, IDLE, clk_i, rst_ni)
308
309 // Filter glitches on ack signal before sending it through
the asynchronous channel
310 `FF(ack_dst_q, ack_dst_d, 1'b0, clk_i, rst_ni)
311
312 if (Decoupled) begin : gen_decoupled
313     // Decouple the output from the asynchronous data bus
without introducing
314     // additional latency by inserting a spill register
315     cc_spill_register #(
316         .data_t(data_t),
317         .Bypass(1'b0)
318     ) i_spill_register (
319         .clk_i,
320         .rst_ni,
321         .clr_i('0),
322         .valid_i(data_valid),
323         .ready_o(output_ready),
324         .data_i(async_data_i),
325         .valid_o,
326         .ready_i,
327         .data_o
328     );
329 end else begin : gen_not_decoupled
330     assign valid_o    = data_valid;
331     assign output_ready = ready_i;
332     assign data_o     = async_data_i;
333 end
334
335 // Output assignments.
336 assign async_ack_o = ack_dst_q;
337
338 endmodule

```

11. pulp-platform_common_cells/src/cc_cdc_fifo_2phase.sv (312 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the license.
11 //
12 // Fabian Schuiki <fschuiki@iis.ee.ethz.ch>
13
14 `include "common_cells/registers.svh"
15 `include "common_cells/assertions.svh"
16
17 /// A clock domain crossing FIFO, using 2-phase hand shakes.
18 ///
19 /// This FIFO has its push and pop ports in two separate clock
domains. Its size
20 /// can only be powers of two, which is why its depth is given as
2**LogDepth.
21 /// LogDepth must be at least 1.
22 ///
23 /// # Reset Behavior!!
24 ///
25 /// This module must not be used if warm reset capability is a
requirement. The
26 /// only exception is if you consistently use a reset control-
ler that sequences
27 /// the resets while gating both clock domains (be very care-
ful if you follow
28 /// this strategy!).
29 ///
30 /// After this disclaimer, here is how you connect the
src_rst_ni and the
31 /// dst_rst_ni of this module for power-on-reset (POR). The
src_rst_ni and
32 /// dst_rst_ni signal must be asserted SIMULTANEOUSLY (i.e.
asynchronous
33 /// assertion). Otherwise, spurious transactions could occur
in the domain
34 /// where the reset arrives later than the other. The de-as-

```

```

section of both reset
35 /// must be synchronized to their respective clock domain
(i.e. src_rst_ni must
36 /// be deasserted synchronously to the src_clk_i and
dst_rst_ni must be
37 /// deasserted synchronously to dst_clk_i.) You can use the
cc_rstgen cell in the
38 /// common_cells library to achieve this (synchronization of
only the
39 /// de-assertion). However, be careful about reset domain
crossings; If you
40 /// reset both domain asynchronously in their entirety (i.e.
POR) you are fine.
41 /// However, if you use this strategy for warm resets (some
parts of the circuit
42 /// are not reset) you might introduce metastability in this
separate
43 /// reset-domain when you assert the reset (the deassertion
synchronizer doesn't
44 /// help here).
45 ///
46 /// CONSTRAINT: Requires max_delay of min_period(src_clk_i,
dst_clk_i) through
47 /// async_wptra_req, async_wptra_ack, async_wptra_data,
async_rptra_req,
48 /// async_rptra_ack, async_rptra_data, and async_data. The
async_wptra_* and
49 /// async_rptra_* signals are the actual pointer handshakes,
while async_data is
50 /// the FIFO storage exposed from the source domain to the
destination domain.
51 module cc_cdc_fifo_2phase #(
52     /// The data type of the payload transported by the FIFO.
parameter type data_t = logic,
53     /// The FIFO's depth given as 2**LogDepth.
parameter int unsigned LogDepth = 3,
54     /// The number of synchronization registers to insert on the
async pointers.
parameter int unsigned SyncStages = 2
55 ){
56     input logic src_rst_ni,
57     input logic src_clk_i,
58     input data_t src_data_i,
59     input logic src_valid_i,
60     output logic src_ready_o,
61
62     input logic dst_rst_ni,
63     input logic dst_clk_i,
64     output data_t dst_data_o,
65     output logic dst_valid_o,
66     input logic dst_ready_i
67 );
68
69 localparam int unsigned PtrWidth = LogDepth+1;
70 typedef logic [PtrWidth-1:0] pointer_t;
71
72 data_t [2**LogDepth-1:0] async_data;
73 logic async_wptra_req;
74 logic async_wptra_ack;
75 pointer_t async_wptra_data;
76 logic async_rptra_req;
77 logic async_rptra_ack;
78 pointer_t async_rptra_data;
79
80 cc_cdc_fifo_2phase_src #(
81     .data_t ( data_t ),
82     .LogDepth ( LogDepth ),
83     .SyncStages ( SyncStages )
84 ) i_src (
85     .src_rst_ni ( src_rst_ni ),
86     .src_clk_i ( src_clk_i ),
87     .src_data_i ( src_data_i ),
88     .src_valid_i ( src_valid_i ),
89     .src_ready_o ( src_ready_o ),
90
91     (* async *) .async_data_o ( async_data ),
92     (* async *) .async_wptra_req_o ( async_wptra_req ),
93     (* async *) .async_wptra_ack_i ( async_wptra_ack ),
94     (* async *) .async_wptra_data_o ( async_wptra_data ),
95     (* async *) .async_rptra_req_i ( async_rptra_req ),
96     (* async *) .async_rptra_ack_o ( async_rptra_ack ),
97     (* async *) .async_rptra_data_i ( async_rptra_data )
98 );
99
100 cc_cdc_fifo_2phase_dst #(
101     .data_t ( data_t ),
102     .LogDepth ( LogDepth ),
103     .SyncStages ( SyncStages )
104 ) i_dst (
105     .dst_rst_ni ( dst_rst_ni ),
106     .dst_clk_i ( dst_clk_i ),
107     .dst_data_o ( dst_data_o ),
108     .dst_valid_o ( dst_valid_o ),
109     .dst_ready_i ( dst_ready_i ),
110
111     (* async *) .async_data_i ( async_data ),
112     (* async *) .async_wptra_req_i ( async_wptra_req ),
113     (* async *) .async_wptra_ack_o ( async_wptra_ack ),
114     (* async *) .async_wptra_data_i ( async_wptra_data ),
115     (* async *) .async_rptra_req_o ( async_rptra_req ),

```

```

119 (* async *) .async_rptr_ack_i ( async_rptr_ack ),
120 (* async *) .async_rptr_data_o ( async_rptr_data )
121 );
122
123 // Check the invariants.
124 `ifndef COMMON_CELLS_ASSERTS_OFF
125 `ASSERT_INIT(log_depth_0, LogDepth > 0)
126 `ASSERT_INIT(sync_stages_gt_2, SyncStages >= 2)
127 `endif
128 endmodule
129
130
131
132 /// Source side for the 2-phase CDC FIFO.
133 module cc_cdc_fifo_2phase_src #(
134     parameter type data_t = logic,
135     parameter int unsigned LogDepth = 3,
136     parameter int unsigned SyncStages = 2
137 ) (
138     input logic src_rst_ni,
139     input logic src_clk_i,
140     input data_t src_data_i,
141     input logic src_valid_i,
142     output logic src_ready_o,
143
144     output data_t [2**LogDepth-1:0] async_data_o,
145     output logic async_wptr_req_o,
146     input logic async_wptr_ack_i,
147     output logic [LogDepth:0] async_wptr_data_o,
148     input logic async_rptr_req_i,
149     output logic async_rptr_ack_o,
150     input logic [LogDepth:0] async_rptr_data_i
151 );
152
153 localparam int unsigned PtrWidth = LogDepth+1;
154 typedef logic [PtrWidth-1:0] pointer_t;
155 typedef logic [LogDepth-1:0] index_t;
156
157 localparam pointer_t PtrFull = (1 << LogDepth);
158
159 index_t fifo_widx;
160 logic fifo_write;
161 data_t [2**LogDepth-1:0] fifo_data_q;
162
163 pointer_t src_wptr_q;
164 pointer_t src_rptr;
165
166 if (SyncStages < 2) begin : gen_sync_stage_assertion
167     $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
168 end
169
170 assign fifo_widx = src_wptr_q[LogDepth-1:0];
171 assign fifo_write = src_valid_i && src_ready_o;
172 assign async_data_o = fifo_data_q;
173
174 for (genvar i = 0; i < 2**LogDepth; i++) begin : gen_word
175     always_ff @(posedge src_clk_i, negedge src_rst_ni) begin
176         if (!src_rst_ni) begin
177             fifo_data_q[i] <= data_t'('0);
178         end else if (fifo_write && (fifo_widx == i)) begin
179             fifo_data_q[i] <= src_data_i;
180         end
181     end
182 end
183
184 `FFL(src_wptr_q, src_wptr_q + 1, fifo_write, '0, src_clk_i,
src_rst_ni)
185
186 // The pointers into the FIFO are one bit wider than the ac-
tual address into
187 // the FIFO. This makes detecting critical states very
simple: if all but the
188 // topmost bit of rptr and wptr agree, the FIFO is in a
critical state. If the
189 // topmost bit is equal, the FIFO is empty, otherwise it is
full.
190 assign src_ready_o = ((src_wptr_q ^ src_rptr) != PtrFull);
191
192 cc_cdc_2phase_src #(
193     .data_t ( pointer_t ),
194     .SyncStages ( SyncStages )
195 ) i_cdc_wptr_src (
196     .rst_ni ( src_rst_ni ),
197     .clk_i ( src_clk_i ),
198     .data_i ( src_wptr_q ),
199     .valid_i ( 'b1 ),
200     .ready_o ( ),
201     .async_req_o ( async_wptr_req_o ),
202     .async_ack_i ( async_wptr_ack_i ),
203     .async_data_o ( async_wptr_data_o )
204 );
205
206 cc_cdc_2phase_dst #(
207     .data_t ( pointer_t ),
208     .SyncStages ( SyncStages )
209 ) i_cdc_rptr_dst (
210     .rst_ni ( src_rst_ni ),
211     .clk_i ( src_clk_i ),
212     .data_o ( src_rptr ),

```

```

213     .valid_o ( ),
214     .ready_i ( 'b1 ),
215     .async_req_i ( async_rptr_req_i ),
216     .async_ack_o ( async_rptr_ack_o ),
217     .async_data_i ( async_rptr_data_i )
218 );
219
220 // Check the invariants.
221 `ifndef COMMON_CELLS_ASSERTS_OFF
222 `ASSERT_INIT(log_depth_0, LogDepth > 0)
223 `ASSERT_INIT(sync_stages_gt_2, SyncStages >= 2)
224 `endif
225
226 endmodule
227
228
229 /// Destination side for the 2-phase CDC FIFO.
230 module cc_cdc_fifo_2phase_dst #(
231     parameter type data_t = logic,
232     parameter int unsigned LogDepth = 3,
233     parameter int unsigned SyncStages = 2
234 ) (
235     input logic dst_rst_ni,
236     input logic dst_clk_i,
237     output data_t dst_data_o,
238     output logic dst_valid_o,
239     input logic dst_ready_i,
240
241     input data_t [2**LogDepth-1:0] async_data_i,
242     input logic async_wptr_req_i,
243     output logic async_wptr_ack_o,
244     input logic [LogDepth:0] async_wptr_data_i,
245     output logic async_rptr_req_o,
246     input logic async_rptr_ack_i,
247     output logic [LogDepth:0] async_rptr_data_o
248 );
249
250 localparam int unsigned PtrWidth = LogDepth+1;
251 typedef logic [PtrWidth-1:0] pointer_t;
252 typedef logic [LogDepth-1:0] index_t;
253
254 localparam pointer_t PtrEmpty = '0;
255
256 index_t fifo_ridx;
257 data_t fifo_rdata;
258
259 pointer_t dst_wptr;
260 pointer_t dst_rptr_q;
261
262 if (SyncStages < 2) begin : gen_sync_stage_assertion
263     $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
264 end
265
266 assign fifo_ridx = dst_rptr_q[LogDepth-1:0];
267 assign fifo_rdata = async_data_i[fifo_ridx];
268 assign dst_data_o = fifo_rdata;
269
270 `FFL(dst_rptr_q, dst_rptr_q + 1, dst_valid_o && dst_ready_i,
'0, dst_clk_i, dst_rst_ni)
271
272 // The pointers into the FIFO are one bit wider than the ac-
tual address into
273 // the FIFO. This makes detecting critical states very
simple: if all but the
274 // topmost bit of rptr and wptr agree, the FIFO is in a
critical state. If the
275 // topmost bit is equal, the FIFO is empty, otherwise it is
full.
276 assign dst_valid_o = ((dst_rptr_q ^ dst_wptr) != PtrEmpty);
277
278 cc_cdc_2phase_dst #(
279     .data_t ( pointer_t ),
280     .SyncStages ( SyncStages )
281 ) i_cdc_wptr_dst (
282     .rst_ni ( dst_rst_ni ),
283     .clk_i ( dst_clk_i ),
284     .data_o ( dst_wptr ),
285     .valid_o ( ),
286     .ready_i ( 'b1 ),
287     .async_req_i ( async_wptr_req_i ),
288     .async_ack_o ( async_wptr_ack_o ),
289     .async_data_i ( async_wptr_data_i )
290 );
291
292 cc_cdc_2phase_src #(
293     .data_t ( pointer_t ),
294     .SyncStages ( SyncStages )
295 ) i_cdc_rptr_src (
296     .rst_ni ( dst_rst_ni ),
297     .clk_i ( dst_clk_i ),
298     .data_i ( dst_rptr_q ),
299     .valid_i ( 'b1 ),
300     .ready_o ( ),
301     .async_req_o ( async_rptr_req_o ),
302     .async_ack_i ( async_rptr_ack_i ),
303     .async_data_o ( async_rptr_data_o )
304 );
305

```

```

306 // Check the invariants.
307 `ifndef COMMON_CELLS_ASSERTS_OFF
308   `ASSERT_INIT(log_depth_0, LogDepth > 0)
309   `ASSERT_INIT(sync_stages_gt_2, SyncStages >= 2)
310 `endif
311
312 endmodule

```

12. pulp-platform_common_cells/src/cc_cdc_fifo_gray.sv (447 lines)

```

1 // Copyright 2018-2019 ETH Zurich and University of Bologna.
2 //
3 // Copyright and related rights are licensed under the Solder-
  pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
  file except in
5 // compliance with the License. You may obtain a copy of the
  License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
7 // or agreed to in writing, software, hardware and materials
  distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
10 // specific language governing permissions and limitations under
  the License.
11 //
12 // Fabian Schuiki <fschuiki@iis.ee.ethz.ch>
13 // Florian Zaruba <zarubaf@iis.ee.ethz.ch>
14 // Philippe Sauter <phsauter@iis.ee.ethz.ch> (source/destination split)
15
16 /// A clock domain crossing FIFO, using gray counters.
17 ///
18 /// # Architecture
19 ///
20 /// The design is split into two parts, each one being clocked
  and reset
21 /// separately.
22 /// 1. The data to be transferred over the clock domain
  boundary is
23 /// is stored in a FIFO. The corresponding write pointer is
  managed
24 /// (incremented) in the source clock domain.
25 /// 2. The entire FIFO content is exposed over the
  `async_data` port.
26 /// The destination clock domain increments its read pointer
  er
27 /// in its destination clock domain.
28 ///
29 /// Read and write pointers are then gray coded, communicated
30 /// and synchronized using a classic multi-stage FF synchronizer
31 /// in the other clock domain. The gray coding ensures that
  only
32 /// one bit changes at each pointer increment, preventing the
33 /// synchronizer to accidentally latch an inconsistent state
34 /// on a multi-bit bus.
35 ///
36 /// The not full signal e.g. `src_ready_o` (on the sending
  side)
37 /// is generated using the local write pointer and the pessimistic
38 /// read pointer from the destination clock domain (pessimistic
39 /// because it is delayed at least two cycles because of the
  synchronizer
40 /// stages). This prevents the FIFO from overflowing.
41 ///
42 /// The not empty signal e.g. `dst_valid_o` is generated using
43 /// the pessimistic write pointer and the local read pointer
  in
44 /// the destination clock domain. This means the FIFO content
45 /// does not need to be synchronized as we are sure we are
  reading
46 /// data which has been written at least two cycles earlier.
47 /// Furthermore, the read select logic into the FIFO is completely
48 /// clocked by the destination clock domain which avoids
49 /// inefficient data synchronization.
50 ///
51 /// The FIFO size must be powers of two, which is why its
  depth is
52 /// given as 2**LogDepth. LogDepth must be at least 1.
53 ///
54 /// # Reset Behavior!!
55 ///
56 /// This module must not be used if warm reset capability is a
  requirement. The
57 /// only exception is if you consistently use a reset controller
  that sequences
58 /// the resets while gating both clock domains (be very careful
  if you follow
59 /// this strategy!). If you need warm reset/clear/flush capability,

```

```

  ilities, use (AND
60 /// CAREFULLY READ THE DESCRIPTION) the
  cc_cdc_fifo_gray_clearable module.
61 ///
62 /// After this disclaimer, here is how you connect the
  src_rst_ni and the
63 /// dst_rst_ni of this module for power-on-reset (POR). The
  src_rst_ni and
64 /// dst_rst_ni signal must be asserted SIMULTANEOUSLY (i.e.
  asynchronous
65 /// assertion). Otherwise, spurious transactions could occur
  in the domain
66 /// where the reset arrives later than the other. The de-assertion
  of both reset
67 /// must be synchronized to their respective clock domain
  (i.e. src_rst_ni must
68 /// be deasserted synchronously to the src_clk_i and
  dst_rst_ni must be
69 /// deasserted synchronously to dst_clk_i.) You can use the
  cc_rstgen cell in the
70 /// common_cells library to achieve this (synchronization of
  only the
71 /// de-assertion). However be careful about reset domain
  crossings; If you
72 /// reset both domain asynchronously in their entirety (i.e.
  POR) you are fine.
73 /// However, if you use this strategy for warm resets (some
  parts of the circuit
74 /// are not reset) you might introduce metastability in this
  separate
75 /// reset-domain when you assert the reset (the deassertion
  synchronizer doesn't
76 /// help here).
77 ///
78 /// # Constraints
79 ///
80 /// We need to make sure that the propagation delay of the
81 /// data, read and write pointer is bound to the minimum of
82 /// either the sending or receiving clock period to prevent
83 /// an inconsistent state to be latched (if for example the
  one
84 /// bit of the read/write pointer have an excessive delay).
85 /// Furthermore, we should deactivate setup and hold checks on
  the asynchronous signals.
86 ///
87 /// ...
88 ///
89 /// set_ungroup [get_designs cc_cdc_fifo_gray] false
90 /// set_boundary_optimization [get_designs cc_cdc_fifo_gray]
  false
91 /// set_max_delay min(T_src, T_dst) \
92 /// -through [get_pins -hierarchical -filter async] \
93 /// -through [get_pins -hierarchical -filter async]
94 /// set_false_path -hold \
95 /// -through [get_pins -hierarchical -filter async] \
96 /// -through [get_pins -hierarchical -filter async]
97 /// ...
98
99 `include "common_cells/registers.svh"
100 `include "common_cells/assertions.svh"
101
102 (* no_ungroup *)
103 (* no_boundary_optimization *)
104 module cc_cdc_fifo_gray #(
105   /// The width of the default logic type.
106   parameter int unsigned Width = 1,
107   /// The data type of the payload transported by the FIFO.
108   parameter type data_t = logic [Width-1:0],
109   /// The FIFO's depth given as 2**LogDepth.
110   parameter int unsigned LogDepth = 3,
111
112   /// The number of synchronization registers to insert on the
  async pointers.
113   parameter int unsigned SyncStages = 2
114 ) (
115   input logic src_rst_ni,
116   input logic src_clk_i,
117   input data_t src_data_i,
118   input logic src_valid_i,
119   output logic src_ready_o,
120
121   input logic dst_rst_ni,
122   input logic dst_clk_i,
123   output data_t dst_data_o,
124   output logic dst_valid_o,
125   input logic dst_ready_i
126 );
127
128 data_t [2**LogDepth-1:0] async_data;
129 logic [LogDepth:0] async_wptr;
130 logic [LogDepth:0] async_rptr;
131
132 cc_cdc_fifo_gray_src #(
133   .data_t ( data_t ),
134   .LogDepth ( LogDepth ),
135   .SyncStages ( SyncStages )
136 ) i_src (
137   .src_rst_ni,
138   .src_clk_i,
139   .src_data_i,
140   .src_valid_i,
141   .src_ready_o,

```

```

141
142 (* async *) .async_data_o ( async_data ),
143 (* async *) .async_wptr_o ( async_wptr ),
144 (* async *) .async_rptr_i ( async_rptr )
145 );
146
147 cc_cdc_fifo_gray_dst #(
148   .data_t ( data_t ),
149   .LogDepth ( LogDepth ),
150   .SyncStages ( SyncStages )
151 ) i_dst (
152   .dst_rst_ni,
153   .dst_clk_i,
154   .dst_data_o,
155   .dst_valid_o,
156   .dst_ready_i,
157
158   (* async *) .async_data_i ( async_data ),
159   (* async *) .async_wptr_i ( async_wptr ),
160   (* async *) .async_rptr_o ( async_rptr )
161 );
162
163 // Check the invariants.
164 `ifndef COMMON_CELLS_ASSERTS_OFF
165 `ASSERT_INIT(log_depth_0, LogDepth > 0)
166 `ASSERT_INIT(sync_stages_gt_2, SyncStages >= 2)
167 `endif
168
169 endmodule
170
171
172 (* no_ungroup *)
173 (* no_boundary_optimization *)
174 module cc_cdc_fifo_gray_src #(
175   parameter type data_t = logic,
176   parameter int unsigned LogDepth = 3,
177   parameter int unsigned SyncStages = 2
178 )(
179   input logic src_rst_ni,
180   input logic src_clk_i,
181   input data_t src_data_i,
182   input logic src_valid_i,
183   output logic src_ready_o,
184
185   output data_t [2*LogDepth-1:0] async_data_o,
186   output logic [LogDepth:0] async_wptr_o,
187   input logic [LogDepth:0] async_rptr_i
188 );
189
190 logic [LogDepth-1:0] src_widx;
191 logic src_write;
192
193 assign src_write = src_valid_i & src_ready_o;
194
195 cc_cdc_fifo_gray_src_data #(
196   .data_t ( data_t ),
197   .LogDepth ( LogDepth )
198 ) i_src_data (
199   .src_rst_ni,
200   .src_clk_i,
201   .src_data_i,
202   .src_widx_i ( src_widx ),
203   .src_write_i ( src_write ),
204   .async_data_o
205 );
206
207 cc_cdc_fifo_gray_src_ptr #(
208   .LogDepth ( LogDepth ),
209   .SyncStages ( SyncStages )
210 ) i_src_ptr (
211   .src_rst_ni,
212   .src_clk_i,
213   .src_valid_i,
214   .src_ready_o,
215   .src_widx_o ( src_widx ),
216   .async_wptr_o,
217   .async_rptr_i
218 );
219
220 endmodule
221
222
223 (* no_ungroup *)
224 (* no_boundary_optimization *)
225 module cc_cdc_fifo_gray_dst #(
226   parameter type data_t = logic,
227   parameter int unsigned LogDepth = 3,
228   parameter int unsigned SyncStages = 2
229 )(
230   input logic dst_rst_ni,
231   input logic dst_clk_i,
232   output data_t dst_data_o,
233   output logic dst_valid_o,
234   input logic dst_ready_i,
235
236   input data_t [2*LogDepth-1:0] async_data_i,
237   input logic [LogDepth:0] async_wptr_i,
238   output logic [LogDepth:0] async_rptr_o
239 );
240
241 logic dst_valid, dst_ready;

```

```

242 logic [LogDepth-1:0] dst_ridx;
243
244 cc_cdc_fifo_gray_dst_ptr #(
245   .LogDepth ( LogDepth ),
246   .SyncStages ( SyncStages )
247 ) i_dst_ptr (
248   .dst_rst_ni,
249   .dst_clk_i,
250   .dst_ready_i ( dst_ready ),
251   .dst_valid_o ( dst_valid ),
252   .dst_ridx_o ( dst_ridx ),
253   .async_wptr_i,
254   .async_rptr_o
255 );
256
257 cc_cdc_fifo_gray_dst_data #(
258   .data_t ( data_t ),
259   .LogDepth ( LogDepth )
260 ) i_dst_data (
261   .dst_rst_ni,
262   .dst_clk_i,
263   .dst_data_o,
264   .dst_valid_o,
265   .dst_ready_i,
266   .dst_valid_i ( dst_valid ),
267   .dst_ready_o ( dst_ready ),
268   .dst_ridx_i ( dst_ridx ),
269   .async_data_i
270 );
271
272 endmodule
273
274
275 (* no_ungroup *)
276 (* no_boundary_optimization *)
277 module cc_cdc_fifo_gray_src_data #(
278   parameter type data_t = logic,
279   parameter int unsigned LogDepth = 3
280 )(
281   input logic src_rst_ni,
282   input logic src_clk_i,
283   input data_t src_data_i,
284   input logic [LogDepth-1:0] src_widx_i,
285   input logic src_write_i,
286   output data_t [2*LogDepth-1:0] async_data_o
287 );
288
289 data_t [2*LogDepth-1:0] data_d, data_q;
290
291 always_comb begin
292   data_d = data_q;
293   data_d[src_widx_i] = src_data_i;
294 end
295
296 `FLL(data_q, data_d, src_write_i, '0, src_clk_i, src_rst_ni)
297
298 assign async_data_o = data_q;
299
300 endmodule
301
302
303 (* no_ungroup *)
304 (* no_boundary_optimization *)
305 module cc_cdc_fifo_gray_src_ptr #(
306   parameter int unsigned LogDepth = 3,
307   parameter int unsigned SyncStages = 2
308 )(
309   input logic src_rst_ni,
310   input logic src_clk_i,
311   input logic src_valid_i,
312   output logic src_ready_o,
313   output logic [LogDepth-1:0] src_widx_o,
314   output logic [LogDepth:0] async_wptr_o,
315   input logic [LogDepth:0] async_rptr_i
316 );
317
318 localparam int unsigned PtrWidth = LogDepth+1;
319 localparam logic [PtrWidth-1:0] PtrFull = (1 << LogDepth);
320
321 logic [PtrWidth-1:0] wptr_q, wptr_d, wptr_bin, wptr_next,
322 rptr, rptr_bin;
323
324 if (SyncStages < 2) begin : gen_sync_stage_assertion
325   $error("A minimum of 2 synchronizer stages is required for
326 proper functionality.");
327 end
328
329 // Read pointer from the destination domain, synchronized
330 into the source
331 // domain for the full check.
332 for (genvar i = 0; i < PtrWidth; i++) begin : gen_sync
333   tc_sync #(SyncStages) i_sync (
334     .clk_i ( src_clk_i ),
335     .rst_ni ( src_rst_ni ),
336     .serial_i ( async_rptr_i[i] ),
337     .serial_o ( rptr[i] )
338   );
339 end
340
341 cc_gray_to_binary #(PtrWidth) i_rptr_g2b
342 (.a_i(rptr), .z_o(rptr_bin));

```



```

338
339 // Local write pointer.
340 assign wptr_next = wptr_bin+1;
341 cc_gray_to_binary #(PtrWidth) i_wptr_g2b
(.a_i(wptr_q), .z_o(wptr_bin));
342 cc_binary_to_gray #(PtrWidth) i_wptr_b2g
(.a_i(wptr_next), .z_o(wptr_d));
343 `FFL(wptr_q, wptr_d, src_valid_i & src_ready_o, '0,
src_clk_i, src_rst_ni)
344
345 assign src_widx_o = wptr_bin[LogDepth-1:0];
346 assign async_wptr_o = wptr_q;
347
348 // The pointers into the FIFO are one bit wider than the ac-
tual address into
349 // the FIFO. This makes detecting critical states very
simple: if all but the
350 // topmost bit of rptr and wptr agree, the FIFO is in a
critical state. If the
351 // topmost bit is equal, the FIFO is empty, otherwise it is
full.
352 assign src_ready_o = ((wptr_bin ^ rptr_bin) != PtrFull);
353
354 endmodule
355
356 (* no_ungroup *)
357 (* no_boundary_optimization *)
358 module cc_cdc_fifo_gray_dst_ptr #(
359     parameter int unsigned LogDepth = 3,
360     parameter int unsigned SyncStages = 2
361 ) (
362     input logic dst_rst_ni,
363     input logic dst_clk_i,
364     input logic dst_ready_i,
365     output logic dst_valid_o,
366     output logic [LogDepth-1:0] dst_ridx_o,
367     input logic [LogDepth:0] async_wptr_i,
368     output logic [LogDepth:0] async_rptr_o
369 );
370
371 localparam int unsigned PtrWidth = LogDepth+1;
372 localparam logic [PtrWidth-1:0] PtrEmpty = '0;
373
374 logic [PtrWidth-1:0] rptr_q, rptr_d, rptr_bin, rptr_next,
wptr, wptr_bin;
376
377 if (SyncStages < 2) begin : gen_sync_stage_assertion
378     $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
379 end
380
381 // Local read pointer.
382 assign rptr_next = rptr_bin+1;
383 cc_gray_to_binary #(PtrWidth) i_rptr_g2b
(.a_i(rptr_q), .z_o(rptr_bin));
384 cc_binary_to_gray #(PtrWidth) i_rptr_b2g
(.a_i(rptr_next), .z_o(rptr_d));
385 `FFL(rptr_q, rptr_d, dst_valid_o & dst_ready_i, '0,
dst_clk_i, dst_rst_ni)
386
387 assign dst_ridx_o = rptr_bin[LogDepth-1:0];
388 assign async_rptr_o = rptr_q;
389
390 // Write pointer from the source domain, synchronized into
the destination
391 // domain for the empty check.
392 for (genvar i = 0; i < PtrWidth; i++) begin : gen_sync
393     tc_sync #(Stages(SyncStages)) i_sync (
394         .clk_i ( dst_clk_i ),
395         .rst_ni ( dst_rst_ni ),
396         .serial_i ( async_wptr_i[i] ),
397         .serial_o ( wptr[i] )
398     );
399 end
400 cc_gray_to_binary #(PtrWidth) i_wptr_g2b
(.a_i(wptr), .z_o(wptr_bin));
401
402 // The pointers into the FIFO are one bit wider than the ac-
tual address into
403 // the FIFO. This makes detecting critical states very
simple: if all but the
404 // topmost bit of rptr and wptr agree, the FIFO is in a
critical state. If the
405 // topmost bit is equal, the FIFO is empty, otherwise it is
full.
406 assign dst_valid_o = ((wptr_bin ^ rptr_bin) != PtrEmpty);
407
408 endmodule
409
410 (* no_ungroup *)
411 (* no_boundary_optimization *)
412 module cc_cdc_fifo_gray_dst_data #(
413     parameter type data_t = logic,
414     parameter int unsigned LogDepth = 3
415 ) (
416     input logic dst_rst_ni,
417     input logic dst_clk_i,
418     output data_t dst_data_o,

```

```

420 output logic dst_valid_o,
421 input logic dst_ready_i,
422 input logic dst_valid_i,
423 output logic dst_ready_o,
424 input logic [LogDepth-1:0] dst_ridx_i,
425 input data_t [2*LogDepth-1:0] async_data_i
426 );
427
428 data_t dst_data;
429
430 assign dst_data = async_data_i[dst_ridx_i];
431
432 // Cut the combinatorial path with a spill register.
433 cc_spill_register #(
434     .data_t ( data_t )
435 ) i_spill_register (
436     .clk_i ( dst_clk_i ),
437     .rst_ni ( dst_rst_ni ),
438     .clr_i ( '0 ),
439     .valid_i ( dst_valid_i ),
440     .ready_o ( dst_ready_o ),
441     .data_i ( dst_data ),
442     .valid_o ( dst_valid_o ),
443     .ready_i ( dst_ready_i ),
444     .data_o ( dst_data_o )
445 );
446
447 endmodule

```

13. pulp-platform_common_cells/src/ cc_cdc_fifo_gray_clearable.sv (698 lines)

```

1 // Copyright 2018-2019 ETH Zurich and University of Bologna.
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this license is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the License.
11 //
12 // Fabian Schuiki <fschuiki@iis.ee.ethz.ch>
13 // Florian Zaruba <zarubaf@iis.ee.ethz.ch>
14 // Manuel Eggmann <meggmann@iis.ee.ethz.ch> (clearability
feature)
15 // Philippe Sauter <phsauter@iis.ee.ethz.ch> (source/destina-
tion split)
16
17 /// A clock domain crossing FIFO, using gray counters.
18 ///
19 /// # Architecture
20 ///
21 /// The design is split into two parts, each one being clocked
and reset
22 /// separately.
23 /// 1. The data to be transferred over the clock domain
boundary is
24 /// is stored in a FIFO. The corresponding write pointer is
managed
25 /// (incremented) in the source clock domain.
26 /// 2. The entire FIFO content is exposed over the
`async_data` port.
27 /// The destination clock domain increments its read point-
er
28 /// in its destination clock domain.
29 ///
30 /// Read and write pointers are then gray coded, communicated
31 /// and synchronized using a classic multi-stage FF synchron-
izer
32 /// in the other clock domain. The gray coding ensures that
only
33 /// one bit changes at each pointer increment, preventing the
34 /// synchronizer to accidentally latch an inconsistent state
35 /// on a multi-bit bus.
36 ///
37 /// The not full signal e.g. `src_ready_o` (on the sending
side)
38 /// is generated using the local write pointer and the pessim-
istic
39 /// read pointer from the destination clock domain (pessimist-
ic
40 /// because it is delayed at least two cycles because of the
synchronizer
41 /// stages). This prevents the FIFO from overflowing.
42 ///
43 /// The not empty signal e.g. `dst_valid_o` is generated using
44 /// the pessimistic write pointer and the local read pointer

```

```

in
45 /// the destination clock domain. This means the FIFO content
46 /// does not need to be synchronized as we are sure we are
reading
47 /// data which has been written at least two cycles earlier.
48 /// Furthermore, the read select logic into the FIFO is com-
pletely
49 /// clocked by the destination clock domain which avoids
50 /// inefficient data synchronization.
51 ///
52 /// The FIFO size must be powers of two, which is why its
depth is
53 /// given as 2**LogDepth. LogDepth must be at least 1.
54
55 /// Reset Behavior:
56 ///
57 /// In contrast to the cc_cdc_fifo_gray version without clear
signal, this module
58 /// supports one-sided warm resets (asynchronously and syn-
chronously). The way
59 /// this is implemented is described in more detail in the
cc_cdc_reset_ctrlr
60 /// module. To summarize a synchronous clear request i.e. src/
dst_clear_i will
61 /// cause the respective other clock domain to reset as well
without introducing
62 /// any spurious transactions. This is accomplished by an in-
ternal module
63 /// (cc_cdc_reset_ctrlr) the starts a reset sequence on both
sides of the CDC in
64 /// lock-step that first isolates the CDC from the outside
world and then resets
65 /// it. The reset sequencer provides the following behavior:
66 /// 1. There are no spurious invalid or duplicated transac-
tions regardless how
67 /// the individual sides are reset (can also happen roughly
simultaneously)
68 /// 2. The FIFO becomes unready at the src side in the next
cycle after
69 /// synchronous reset request until the reset sequence is
completed. Some of
70 /// the pending transactions might still complete (if the
dst accepts at the
71 /// exact time the reset is request on the src die), some
of them will be
72 /// dropped (of course still guaranteeing FIFO order).
73 /// 3. During the reset sequence the dst might withdraw the
valid signal. This
74 /// might violate higher level protocols. If you need this
feature you would
75 /// have to patch the existing implementation to wait with
the isolate_ack
76 /// assertion until all open handshakes were acknowledged.
77 /// 4. If the parameter ClearOnAsyncReset is enabled, the same
behavior as
78 /// above is also valid for asynchronous resets on either
side. However, this
79 /// increases the minimum number of synchronization stages
(SyncStages
80 /// parameter) from 2 to 3 (read the cc_cdc_reset_ctrlr
header to figure out
81 /// why).
82 ///
83 ///
84 /// # Constraints
85 ///
86 /// We need to make sure that the propagation delay of the
data, read and write
87 /// pointer is bound to the minimum of either the sending or
receiving clock
88 /// period to prevent an inconsistent state to be latched (if
for example the
89 /// one bit of the read/write pointer have an excessive
delay). Furthermore, we
90 /// should deactivate setup and hold checks on the asynchron-
ous signals.
91 ///
92 /// async_data, async_wptr, and async_rptr are the FIFO stor-
age and pointer CDCs
93 /// for the actual data path. async_reset_src2dst_req,
94 /// async_reset_dst2src_ack, async_reset_src2dst_next_phase,
95 /// async_reset_dst2src_req, async_reset_src2dst_ack, and
96 /// async_reset_dst2src_next_phase are the CDCs for the clear-
ing state machines.
97 ///
98 /// `` set_ungroup [get_designs cc_cdc_fifo_gray*] false
set_boundary_optimization
99 /// [get_designs cc_cdc_fifo_gray*] false set_max_delay
min(T_src, T_dst) \
100 /// -through [get_pins -hierarchical -filter async] \ -through
[get_pins
101 /// -hierarchical -filter async] set_false_path -hold \ -
through [get_pins
102 /// -hierarchical -filter async] \ -through [get_pins -hier-
archical -filter
103 /// async] ``
104
105 `include "common_cells/registers.svh"

```

```

106 `include "common_cells/assertions.svh"
107
108 (* no_ungroup *)
109 (* no_boundary_optimization *)
110 module cc_cdc_fifo_gray_clearable #(
111     /// The width of the default logic type.
112     parameter int unsigned Width = 1,
113     /// The data type of the payload transported by the FIFO.
114     parameter type data_t = logic [Width-1:0],
115     /// The FIFO's depth given as 2**LogDepth.
116     parameter int unsigned LogDepth = 3,
117     /// The number of synchronization registers to insert on the
async pointers
118     /// between the FIFOs. If ClearOnAsyncReset is enabled, we
need at least 3
119     /// synchronizer stages to provide the clear synchronizer
lower latency than
120     /// the async reset. I.e. if ClearOnAsyncReset==1 -> Sync-
Stages >= 3 else
121     /// SyncStages >= 2.
122     parameter int unsigned SyncStages = 3,
123     parameter bit ClearOnAsyncReset = 1
124 ) (
125     input logic src_rst_ni,
126     input logic src_clk_i,
127     input logic src_clear_i,
128     output logic src_clear_pending_o,
129     input data_t src_data_i,
130     input logic src_valid_i,
131     output logic src_ready_o,
132
133     input logic dst_rst_ni,
134     input logic dst_clk_i,
135     input logic dst_clear_i,
136     output logic dst_clear_pending_o,
137     output data_t dst_data_o,
138     output logic dst_valid_o,
139     input logic dst_ready_i
140 );
141
142 data_t [2**LogDepth-1:0] async_data;
143 logic [LogDepth:0] async_wptr;
144 logic [LogDepth:0] async_rptr;
145 cc_pkg::cdc_clear_seq_phase_e
async_reset_src2dst_next_phase;
146 logic async_reset_src2dst_req;
147 logic async_reset_dst2src_ack;
148 cc_pkg::cdc_clear_seq_phase_e
async_reset_dst2src_next_phase;
149 logic async_reset_dst2src_req;
150 logic async_reset_src2dst_ack;
151
152 if (ClearOnAsyncReset) begin : gen_elaboration_assertion
153     if (SyncStages < 3)
154         $error("The clearable CDC FIFO with async reset syn-
chronization requires at least",
155             "3 synchronizer stages for the FIFO.");
156     end else begin : gen_elaboration_assertion
157         if (SyncStages < 2) begin : gen_elaboration_assertion
158             $error("A minimum of 2 synchronizer stages is required
for proper functionality.");
159         end
160     end
161
162 if (2*SyncStages > 2**LogDepth) begin : gen_elabora-
tion_assertion2
163     $warning("The FIFOs depth of %0d is insufficient to com-
pletely hide the latency of",
164         " %0d SyncStages. The FIFO will stall in the case
where f_src != f_dst. ",
165         "It is recommended to increase the FIFO's log
depth to at least %0d.",
166         2**LogDepth, SyncStages, $clog2(2*SyncStages));
167     end
168
169
170
171 cc_cdc_fifo_gray_clearable_src #(
172     .data_t (data_t),
173     .LogDepth (LogDepth),
174     .SyncStages (SyncStages),
175     .ClearOnAsyncReset (ClearOnAsyncReset)
176 ) i_src (
177     .src_rst_ni,
178     .src_clk_i,
179     .src_clear_i,
180     .src_clear_pending_o,
181     .src_data_i,
182     .src_valid_i,
183     .src_ready_o,
184
185     (* async *) .async_data_o
186     (* async *) .async_wptr_o
187     (* async *) .async_rptr_i
188     (* async *) .async_reset_next_phase_o
189     (* async *) .async_reset_src2dst_next_phase ),

```



```

189 (* async *) .async_reset_req_o
( async_reset_src2dst_req ),
190 (* async *) .async_reset_ack_i
( async_reset_dst2src_ack ),
191 (* async *) .async_reset_next_phase_i
( async_reset_dst2src_next_phase ),
192 (* async *) .async_reset_req_i
( async_reset_dst2src_req ),
193 (* async *) .async_reset_ack_o
( async_reset_src2dst_ack )
194 );
195
196 cc_cdc_fifo_gray_clearable_dst #(
197   .data_t ( data_t ),
198   .LogDepth ( LogDepth ),
199   .SyncStages ( SyncStages ),
200   .ClearOnAsyncReset ( ClearOnAsyncReset )
201 ) i_dst (
202   .dst_rst_ni,
203   .dst_clk_i,
204   .dst_clear_i,
205   .dst_clear_pending_o,
206   .dst_data_o,
207   .dst_valid_o,
208   .dst_ready_i,
209
210   (* async *) .async_data_i
( async_data ),
211   (* async *) .async_wptr_i
( async_wptr ),
212   (* async *) .async_rptr_o
( async_rptr ),
213   (* async *) .async_reset_next_phase_o
( async_reset_dst2src_next_phase ),
214   (* async *) .async_reset_req_o
( async_reset_dst2src_req ),
215   (* async *) .async_reset_ack_i
( async_reset_src2dst_ack ),
216   (* async *) .async_reset_next_phase_i
( async_reset_src2dst_next_phase ),
217   (* async *) .async_reset_req_i
( async_reset_src2dst_req ),
218   (* async *) .async_reset_ack_o
( async_reset_dst2src_ack )
219 );
220
221 // Check the invariants.
222 `ifndef COMMON_CELLS_ASSERTS_OFF
223 `ASSERT_INIT(log_depth_0, LogDepth > 0)
224 `ASSERT_INIT(sync_stages_lt_2, SyncStages >= 2)
225 `endif
226
227 endmodule
228
229
230 (* no_ungroup *)
231 (* no_boundary_optimization *)
232 module cc_cdc_fifo_gray_clearable_src #(
233   parameter type data_t = logic,
234   parameter int unsigned LogDepth = 3,
235   parameter int unsigned SyncStages = 2,
236   parameter bit ClearOnAsyncReset = 1
237 )(
238   input logic src_rst_ni,
239   input logic src_clk_i,
240   input logic src_clear_i,
241   output logic src_clear_pending_o,
242   input data_t src_data_i,
243   input logic src_valid_i,
244   output logic src_ready_o,
245
246   output data_t [2**LogDepth-1:0] async_data_o,
247   output logic [LogDepth:0] async_wptr_o,
248   input logic [LogDepth:0] async_rptr_i,
249   output cc_pkg::cdc_clear_seq_phase_e
async_reset_next_phase_o,
250   output logic async_reset_req_o,
251   input logic async_reset_ack_i,
252   input cc_pkg::cdc_clear_seq_phase_e
async_reset_next_phase_i,
253   input logic async_reset_req_i,
254   output logic async_reset_ack_o
255 );
256
257   logic src_clear_req;
258   logic src_clear_ack_q;
259   logic src_ready;
260   logic src_isolate_req;
261   logic src_isolate_ack_q;
262
263   if (ClearOnAsyncReset) begin : gen_elaboration_assertion
264     if (SyncStages < 3)
265       $error("The clearable CDC FIFO with async reset syn-
chronization requires at least",
266         "3 synchronizer stages for the FIFO.");
267   end else begin : gen_elaboration_assertion
268     if (SyncStages < 2) begin : gen_sync_stage_assertion
269       $error("A minimum of 2 synchronizer stages is required
for proper functionality.");
270     end
271   end

```

```

272
273 localparam int unsigned ResetSyncStages = (SyncStages > 2) ?
SyncStages - 1 : 2;
274
275 cc_cdc_reset_ctrlr_half #(
276   .SyncStages ( ResetSyncStages ),
277   .ClearOnAsyncReset ( ClearOnAsyncReset )
278 ) i_cdc_reset_ctrlr_half (
279   .clk_i ( src_clk_i ),
280   .rst_ni ( src_rst_ni ),
281   .clear_i ( src_clear_i ),
282   .clear_o ( src_clear_req ),
283   .clear_ack_i ( src_clear_ack_q ),
284   .isolate_o ( src_isolate_req ),
285   .isolate_ack_i ( src_isolate_ack_q ),
286   (* async *) .async_next_phase_o
( async_reset_next_phase_o ),
287   (* async *) .async_req_o
( async_reset_req_o ),
288   (* async *) .async_ack_i
( async_reset_ack_i ),
289   (* async *) .async_next_phase_i
( async_reset_next_phase_i ),
290   (* async *) .async_req_i
( async_reset_req_i ),
291   (* async *) .async_ack_o
( async_reset_ack_o )
292 );
293
294 cc_cdc_fifo_gray_src_clearable #(
295   .data_t ( data_t ),
296   .LogDepth ( LogDepth ),
297   .SyncStages ( SyncStages )
298 ) i_src (
299   .src_rst_ni,
300   .src_clk_i,
301   .src_clear_i ( src_clear_req ),
302   .src_data_i,
303   .src_valid_i ( src_valid_i & !src_isolate_req ),
304   .src_ready_o ( src_ready ),
305
306   (* async *) .async_data_o ( async_data_o ),
307   (* async *) .async_wptr_o ( async_wptr_o ),
308   (* async *) .async_rptr_i ( async_rptr_i )
309 );
310
311   assign src_ready_o = src_ready & !src_isolate_req;
312
313   // Isolation and clear are acknowledged one cycle after the
local request.
314   `FF(src_isolate_ack_q, src_isolate_req, 1'b0, src_clk_i,
src_rst_ni)
315   `FF(src_clear_ack_q, src_clear_req, 1'b0, src_clk_i,
src_rst_ni)
316
317   assign src_clear_pending_o = src_isolate_req;
318
319 endmodule
320
321
322 (* no_ungroup *)
323 (* no_boundary_optimization *)
324 module cc_cdc_fifo_gray_clearable_dst #(
325   parameter type data_t = logic,
326   parameter int unsigned LogDepth = 3,
327   parameter int unsigned SyncStages = 2,
328   parameter bit ClearOnAsyncReset = 1
329 )(
330   input logic dst_rst_ni,
331   input logic dst_clk_i,
332   input logic dst_clear_i,
333   output logic dst_clear_pending_o,
334   output data_t dst_data_o,
335   output logic dst_valid_o,
336   input logic dst_ready_i,
337
338   input data_t [2**LogDepth-1:0] async_data_i,
339   input logic [LogDepth:0] async_wptr_i,
340   output logic [LogDepth:0] async_rptr_o,
341   output cc_pkg::cdc_clear_seq_phase_e
async_reset_next_phase_o,
342   output logic async_reset_req_o,
343   input logic async_reset_ack_i,
344   input cc_pkg::cdc_clear_seq_phase_e
async_reset_next_phase_i,
345   input logic async_reset_req_i,
346   output logic async_reset_ack_o
347 );
348
349   logic dst_clear_req;
350   logic dst_clear_ack_q;
351   logic dst_valid;
352   logic dst_isolate_req;
353   logic dst_isolate_ack_q;
354
355   if (ClearOnAsyncReset) begin : gen_elaboration_assertion
356     if (SyncStages < 3)
357       $error("The clearable CDC FIFO with async reset syn-
chronization requires at least",
358         "3 synchronizer stages for the FIFO.");

```

```

359 end else begin : gen_elaboration_assertion
360   if (SyncStages < 2) begin : gen_sync_stage_assertion
361     $error("A minimum of 2 synchronizer stages is required
for proper functionality.");
362   end
363 end
364

365 localparam int unsigned ResetSyncStages = (SyncStages > 2) ?
SyncStages - 1 : 2;
366
367 cc_cdc_reset_ctrlr_half #(
368   .SyncStages      ( ResetSyncStages ),
369   .ClearOnAsyncReset ( ClearOnAsyncReset )
370 ) i_cdc_reset_ctrlr_half (
371   .clk_i      ( dst_clk_i ),
372   .rst_ni     ( dst_rst_ni ),
373   .clear_i    ( dst_clear_i ),
374   .clear_o    ( dst_clear_req ),
375   .clear_ack_i ( dst_clear_ack_q ),
376   .isolate_o  ( dst_isolate_req ),
377   .isolate_ack_i ( dst_isolate_ack_q ),
378   (* async *) .async_next_phase_o
( async_reset_next_phase_o ),
379   (* async *) .async_req_o
( async_reset_req_o ),
380   (* async *) .async_ack_i
( async_reset_ack_i ),
381   (* async *) .async_next_phase_i
( async_reset_next_phase_i ),
382   (* async *) .async_req_i
( async_reset_req_i ),
383   (* async *) .async_ack_o
( async_reset_ack_o )
384 );
385
386 cc_cdc_fifo_gray_dst_clearable #(
387   .data_t      ( data_t ),
388   .LogDepth    ( LogDepth ),
389   .SyncStages  ( SyncStages )
390 ) i_dst (
391   .dst_rst_ni,
392   .dst_clk_i,
393   .dst_clear_i ( dst_clear_req ),
394   .dst_data_o,
395   .dst_valid_o ( dst_valid ),
396   .dst_ready_i ( dst_ready_i & !dst_isolate_req ),
397
398   (* async *) .async_data_i ( async_data_i ),
399   (* async *) .async_wptr_i ( async_wptr_i ),
400   (* async *) .async_rpтр_o ( async_rpтр_o )
401 );
402
403 assign dst_valid_o = dst_valid & !dst_isolate_req;
404
405 // Isolation and clear are acknowledged one cycle after the
local request.
406 `FF(dst_isolate_ack_q, dst_isolate_req, 1'b0, dst_clk_i,
dst_rst_ni)
407 `FF(dst_clear_ack_q, dst_clear_req, 1'b0, dst_clk_i,
dst_rst_ni)
408
409 assign dst_clear_pending_o = dst_isolate_req;
410
411 endmodule
412
413 (* no_ungroup *)
414 (* no_boundary_optimization *)
415 module cc_cdc_fifo_gray_src_clearable #(
416   parameter type data_t = logic,
417   parameter int unsigned LogDepth = 3,
418   parameter int unsigned SyncStages = 2
419 )(
420   input logic src_rst_ni,
421   input logic src_clk_i,
422   input logic src_clear_i,
423   input data_t src_data_i,
424   input logic src_valid_i,
425   output logic src_ready_o,
426
427   output data_t [2**LogDepth-1:0] async_data_o,
428   output logic [LogDepth:0] async_wptr_o,
429   input logic [LogDepth:0] async_rpтр_i
430 );
431
432 logic [LogDepth-1:0] src_widx;
433 logic src_write;
434
435 assign src_write = src_valid_i & src_ready_o;
436
437 cc_cdc_fifo_gray_src_data_clearable #(
438   .data_t      ( data_t ),
439   .LogDepth    ( LogDepth )
440 ) i_src_data (
441   .src_rst_ni,
442   .src_clk_i,
443   .src_data_i,
444   .src_widx_i ( src_widx ),
445   .src_write_i ( src_write ),
446   .async_data_o
447

```

```

448 );
449
450 cc_cdc_fifo_gray_src_ptr_clearable #(
451   .LogDepth    ( LogDepth ),
452   .SyncStages  ( SyncStages )
453 ) i_src_ptr (
454   .src_rst_ni,
455   .src_clk_i,
456   .src_clear_i,
457   .src_valid_i,
458   .src_ready_o,
459   .src_widx_o ( src_widx ),
460   .async_wptr_o,
461   .async_rpтр_i
462 );
463
464 endmodule
465
466 (* no_ungroup *)
467 (* no_boundary_optimization *)
468 module cc_cdc_fifo_gray_dst_clearable #(
469   parameter type data_t = logic,
470   parameter int unsigned LogDepth = 3,
471   parameter int unsigned SyncStages = 2
472 )(
473   input logic dst_rst_ni,
474   input logic dst_clk_i,
475   input logic dst_clear_i,
476   output data_t dst_data_o,
477   output logic dst_valid_o,
478   input logic dst_ready_i,
479
480   input data_t [2**LogDepth-1:0] async_data_i,
481   input logic [LogDepth:0] async_wptr_i,
482   output logic [LogDepth:0] async_rpтр_o
483 );
484
485 logic dst_valid, dst_ready;
486 logic [LogDepth-1:0] dst_ridx;
487
488 cc_cdc_fifo_gray_dst_ptr_clearable #(
489   .LogDepth    ( LogDepth ),
490   .SyncStages  ( SyncStages )
491 ) i_dst_ptr (
492   .dst_rst_ni,
493   .dst_clk_i,
494   .dst_clear_i,
495   .dst_ready_i ( dst_ready ),
496   .dst_valid_o ( dst_valid ),
497   .dst_ridx_o ( dst_ridx ),
498   .async_wptr_i,
499   .async_rpтр_o
500 );
501
502 cc_cdc_fifo_gray_dst_data_clearable #(
503   .data_t      ( data_t ),
504   .LogDepth    ( LogDepth )
505 ) i_dst_data (
506   .dst_rst_ni,
507   .dst_clk_i,
508   .dst_clear_i,
509   .dst_data_o,
510   .dst_valid_o,
511   .dst_ready_i,
512   .dst_valid_i ( dst_valid ),
513   .dst_ready_o ( dst_ready ),
514   .dst_ridx_i ( dst_ridx ),
515   .async_data_i
516 );
517
518 endmodule
519
520 (* no_ungroup *)
521 (* no_boundary_optimization *)
522 module cc_cdc_fifo_gray_src_data_clearable #(
523   parameter type data_t = logic,
524   parameter int unsigned LogDepth = 3
525 )(
526   input logic src_rst_ni,
527   input logic src_clk_i,
528   input data_t src_data_i,
529   input logic [LogDepth-1:0] src_widx_i,
530   input logic src_write_i,
531   output data_t [2**LogDepth-1:0] async_data_o
532 );
533
534 data_t [2**LogDepth-1:0] data_d, data_q;
535
536 always_comb begin
537   data_d = data_q;
538   data_d[src_widx_i] = src_data_i;
539 end
540
541 `FFL(data_q, data_d, src_write_i, '0, src_clk_i, src_rst_ni)
542
543 assign async_data_o = data_q;
544
545 endmodule
546
547 endmodule
548

```

```

549
550 (* no_ungroup *)
551 (* no_boundary_optimization *)
552 module cc_cdc_fifo_gray_src_ptr_clearable #(
553     parameter int unsigned LogDepth = 3,
554     parameter int unsigned SyncStages = 2
555 ) (
556     input logic src_rst_ni,
557     input logic src_clk_i,
558     input logic src_clear_i,
559     input logic src_valid_i,
560     output logic src_ready_o,
561     output logic [LogDepth-1:0] src_widx_o,
562     output logic [LogDepth:0] async_wptr_o,
563     input logic [LogDepth:0] async_rptr_i
564 );
565
566 localparam int unsigned PtrWidth = LogDepth+1;
567 localparam logic [PtrWidth-1:0] PtrFull = (1 << LogDepth);
568
569 logic [PtrWidth-1:0] wptr_q, wptr_d, wptr_bin, wptr_next,
rptr, rptr_bin;
570
571 if (SyncStages < 2) begin : gen_sync_stage_assertion
572     $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
573 end
574
575 // Read pointer from the destination domain, synchronized
into the source
576 // domain for the full check.
577 for (genvar i = 0; i < PtrWidth; i++) begin : gen_sync
578     tc_sync #(.Stages(SyncStages)) i_sync (
579         .clk_i ( src_clk_i ),
580         .rst_ni ( src_rst_ni ),
581         .serial_i ( async_rptr_i[i] ),
582         .serial_o ( rptr[i] )
583     );
584 end
585 cc_gray_to_binary #(PtrWidth) i_rptr_g2b
(.a_i(rptr), .z_o(rptr_bin));
586
587 // Local write pointer.
588 assign wptr_next = wptr_bin+1;
589 cc_gray_to_binary #(PtrWidth) i_wptr_g2b
(.a_i(wptr_q), .z_o(wptr_bin));
590 cc_binary_to_gray #(PtrWidth) i_wptr_b2g
(.a_i(wptr_next), .z_o(wptr_d));
591 `FLLRNC(wptr_q, wptr_d, src_valid_i & src_ready_o,
src_clear_i, '0, src_clk_i, src_rst_ni)
592
593 assign src_widx_o = wptr_bin[LogDepth-1:0];
594 assign async_wptr_o = wptr_q;
595
596 // The pointers into the FIFO are one bit wider than the ac-
tual address into
597 // the FIFO. This makes detecting critical states very
simple: if all but the
598 // topmost bit of rptr and wptr agree, the FIFO is in a
critical state. If the
599 // topmost bit is equal, the FIFO is empty, otherwise it is
full.
600 assign src_ready_o = ((wptr_bin ^ rptr_bin) != PtrFull);
601
602 endmodule
603
604 (* no_ungroup *)
605 (* no_boundary_optimization *)
606 module cc_cdc_fifo_gray_dst_ptr_clearable #(
607     parameter int unsigned LogDepth = 3,
608     parameter int unsigned SyncStages = 2
609 ) (
610     input logic dst_rst_ni,
611     input logic dst_clk_i,
612     input logic dst_clear_i,
613     input logic dst_ready_i,
614     output logic dst_valid_o,
615     output logic [LogDepth-1:0] dst_ridx_o,
616     input logic [LogDepth:0] async_wptr_i,
617     output logic [LogDepth:0] async_rptr_o
618 );
619
620 localparam int unsigned PtrWidth = LogDepth+1;
621 localparam logic [PtrWidth-1:0] PtrEmpty = '0;
622
623 logic [PtrWidth-1:0] rptr_q, rptr_d, rptr_bin, rptr_next,
wptr, wptr_bin;
624
625 if (SyncStages < 2) begin : gen_sync_stage_assertion
626     $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
627 end
628
629 // Local read pointer.
630 assign rptr_next = rptr_bin+1;
631 cc_gray_to_binary #(PtrWidth) i_rptr_g2b
(.a_i(rptr_q), .z_o(rptr_bin));
632 cc_binary_to_gray #(PtrWidth) i_rptr_b2g

```

```

(.a_i(rptr_next), .z_o(rptr_d));
634 `FLLRNC(rptr_q, rptr_d, dst_valid_o & dst_ready_i,
dst_clear_i, '0, dst_clk_i, dst_rst_ni)
635
636 assign dst_ridx_o = rptr_bin[LogDepth-1:0];
637 assign async_rptr_o = rptr_q;
638
639 // Write pointer from the source domain, synchronized into
the destination
640 // domain for the empty check.
641 for (genvar i = 0; i < PtrWidth; i++) begin : gen_sync
642     tc_sync #(.Stages(SyncStages)) i_sync (
643         .clk_i ( dst_clk_i ),
644         .rst_ni ( dst_rst_ni ),
645         .serial_i ( async_wptr_i[i] ),
646         .serial_o ( wptr[i] )
647     );
648 end
649 cc_gray_to_binary #(PtrWidth) i_wptr_g2b
(.a_i(wptr), .z_o(wptr_bin));
650
651 // The pointers into the FIFO are one bit wider than the ac-
tual address into
652 // the FIFO. This makes detecting critical states very
simple: if all but the
653 // topmost bit of rptr and wptr agree, the FIFO is in a
critical state. If the
654 // topmost bit is equal, the FIFO is empty, otherwise it is
full.
655 assign dst_valid_o = ((wptr_bin ^ rptr_bin) != PtrEmpty);
656
657 endmodule
658
659 (* no_ungroup *)
660 (* no_boundary_optimization *)
661 module cc_cdc_fifo_gray_dst_data_clearable #(
662     parameter type data_t = logic,
663     parameter int unsigned LogDepth = 3
664 ) (
665     input logic dst_rst_ni,
666     input logic dst_clk_i,
667     input logic dst_clear_i,
668     output data_t dst_data_o,
669     output logic dst_valid_o,
670     input logic dst_ready_i,
671     input logic dst_valid_i,
672     output logic dst_ready_o,
673     input logic [LogDepth-1:0] dst_ridx_i,
674     input data_t [2**LogDepth-1:0] async_data_i
675 );
676
677 data_t dst_data;
678
679 assign dst_data = async_data_i[dst_ridx_i];
680
681 // Cut the combinatorial path with a spill register.
682 cc_spill_register_flushable #(
683     .data_t ( data_t )
684 ) i_spill_register (
685     .clk_i ( dst_clk_i ),
686     .rst_ni ( dst_rst_ni ),
687     .clr_i ( '0 ),
688     .flush_i ( dst_clear_i ),
689     .valid_i ( dst_valid_i & !dst_clear_i ),
690     .ready_o ( dst_ready_o ),
691     .data_i ( dst_data ),
692     .valid_o ( dst_valid_o ),
693     .ready_i ( dst_ready_i ),
694     .data_o ( dst_data_o )
695 );
696
697
698 endmodule

```

14. pulp-platform_common_cells/src/cc_cdc_reset_ctrlr.sv (522 lines)

```

1 // Copyright 2021 ETH Zurich and University of Bologna.
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the License.
11 //
12 // Authors:
13 // - Manuel Eggimann <eggimann@iis.ee.ethz.ch>
14 // - Philippe Sauter <phsauter@iis.ee.ethz.ch> (formalization

```

```

and stage cleanup)
15 //
16 // Description: CDC Clear Signaling Synchronization
17 // Coordinate clear and isolate sequencing between two CDC
clock domains so
18 // one-sided reset or clear requests do not break the protec-
ted CDC protocol.
19 //
20 // Detailed Description:
21 // This module is mainly used internally to synchronize the
clear requests
22 // between both sides of a CDC module. It aims to solve the
problem of
23 // initiating a CDC clear, reset one-sidedly without running
into
24 // reset-domain-crossing issues and breaking CDC protocol as-
sumption.
25 //
26 // Problem Formulation:
27 //

28 // CDC implementations usually face the issue that one side of
the CDC must not
29 // be cleared without clearing the other side. E.g. clearing
the write-pointer
30 // without clearing the read-pointer in a gray-counting CDC
FIFO results in an
31 // invalid fill-state an may cause spurious transactions of
invalid data to be
32 // propagated accross the CDC. A similar effect is caused in
2-phase CDC
33 // implementations.
34 //

35 // A naive mitigation technique would be to reset both domains
asynchronously
36 // with the same reset signal. This will cause intra-clock do-
main RDC issues
37 // since the asynchronous clear event (assertion of the reset
signal) might
38 // happen close to the active edge of the CDC's periphery and
thus might induce

39 // metastability. A better, but still flawed approach would be to
naively
40 // synchronize assertion AND deassertion (the usual rst sync
only synchronize
41 // deassertion) of the resets into the respective other do-
main. However, this
42 // might cause the classic issue of fast-to-slow clock domain
crossing where the
43 // clear signal is not asserted long enough to be captured by
the receiving
44 // side. The common mitigation strategy is to use a feedback
acknowledge signal
45 // to handshake the reset request into the other domain. One
even more peculiar
46 // corner case this approach might suffer is the scenario
where the synchronized

47 // clear signal arrives at the other side of the CDC within or
even worse after

48 // the same clock cylice that the other domain crossing signals
(e.g. read/write
49 // pointers) are cleared. In this scenario, multiple signals
change within the
50 // same clock cycle and due to metastability we cannot be
sure, that the other
51 // side of the CDC sees the reset assertion before the first
bits of e.g. the
52 // write/read pointer start to switch to their reset state.
Care must also be

53 // taken to handle the corner cases where both sides are reset
simultaneously or
54 // the case where one side leaves reset earlier than the oth-
er.
55 //
56 // How this Module Works
57 //
58 // This module has two interfaces, the 'a' side and the 'b'
side. Each side can
59 // be triggered using the a/b_clear_i signal or (optionally)
by the asynchronous
60 // a/b_rst_ni. Once e.g. 'a' is triggered it will initiate a
clear sequence that
61 // first asserts an 'a_isolate_o' signal, waits until the ex-
ternal circuitry
62 // acknowledges isolation using the 'a_isolate_ack_i'. Then
the module asserts
63 // the 'a_clear_o' signals before some cycles later, the isol-
ate signal is
64 // deasserted. This sequence ensures that no transactions can
arrive to the CDC

65 // while the state is cleared. Now the important part is, that
those four phases
66 // (asser isolate, assert clear, deassert clear, deassert
isolate) are mirrored
67 // on the other side ('b') in lock-step. The cc_cdc_re-

```

```

set_ctrlr module uses a
68 // dedicated 4-phase handshaking CDC to transmit the current
phase of the clear
69 // sequence to the other domain. We use a 4-phase rather than
a 2-phase CDC to
70 // avoid the issues of one-sided async reset that might trig-
ger spurious
71 // transactions. Furthermore, the 4-phase CDC within this mod-
ule is operated in
72 // a special mode: Decoupled=0 ensures that there are no in-
flight transactions.
73 // The src side only consumes the item once the destination
side acknowledged
74 // the reception. This property is required to transition
through the phases in

75 // lock-step. Furthermore, (SendResetMsg=1) will cause the src
side of the
76 // 4-phase CDC to immediately initiate the isolation phase in
the dst domain

77 // upon asynchronous reset regardless how long the async reset
stays asserted or
78 // whether the source clock is gated. Both sides of this mod-
ule independently

79 // generate the sequence signals as an initiator (triggered by
the clear_i or
80 // rst_ni signal) or receiver (triggered for the other side).
The or-ed version
81 // of initiator and receiver are used to generate the actual
a/b_isolate_o and

82 // a/b_clear_o signal. That way, it doesn't matter wheter both
sides
83 // simultanously trigger a clear sequence, proper sequencing
is still
84 // guaranteed.
85 //
86 // The time it takes to complete an entire clear sequence can
be bounded as follows:
87 //
88 //  $t_{clear} \leq 20 \cdot T + 16 \cdot SyncStages \cdot T$ , with  $T = \max(T_a, T_b)$ 
(clock periods of src and dst)
89 //
90 // How to Use the Module
91 //
92 // Instantiate the module within your CDC and connect a/
b_clk_i, the
93 // asynchronous a/b_rst_ni and the synchronous a/b_clear_i
signals. The 'a' and

94 // 'b' port are entirely symetric so it doesn't matter whether
you connect src
95 // to 'a' or 'b'. If you enable support for async reset
96 // (ClearOnAsyncReset==1), parametrize the number of synchron-
ization stages

97 // (for metastability resolution) to be strictly less than the
latency of the
98 // CDC. E.g. if your CDC uses 3 (the minimum) sync stages,
parametrize this
99 // module with SyncStages < 2! Your CDC must implement a src/
dst_clear_i port
100 // that SYNCHRONOUSLY clears all FFs on the respective side.
Connect the CDC's

101 // src/dst_clear ports to this module's a/b_clear_o port. Once
the a/b_isolate_o
102 // signal is asserted, the respective CDC side (src/dst) must
be isolated from
103 // the outside world (i.e. must no longer accept any transac-
tion on the src side
104 // and cease presenting or even withdrawing data on the dst
side). Once your CDC
105 // side is isolated (depending on protocol this might take
several cycles),
106 // assert the a/b_isolate_ack_i signal.
107 //
108 // include "common_cells/registers.svh"
109 //
110 module cc_cdc_reset_ctrlr
111 import cc_pkg::*;
112 #(
113     /// The number of synchronization stages to use for the
114     /// clear signal request/acknowledge. Must be less than the
115     /// number of sync stages used in the CDC.
116     parameter int unsigned SyncStages = 2,
117     /// Whether an asynchronous reset shall cause a clear
118     /// request to be sent to the other side.
119     parameter logic ClearOnAsyncReset = 1'b1
120 ) (
121     // Side A (both sides are symmetric)
122     input logic a_clk_i,
123     input logic a_rst_ni,
124     input logic a_clear_i,
125     output logic a_clear_o,
126     input logic a_clear_ack_i,
127     output logic a_isolate_o,
128     input logic a_isolate_ack_i,
129     // Side B (both sides are symmetric)

```

```

130 input logic b_clk_i,
131 input logic b_rst_ni,
132 input logic b_clear_i,
133 output logic b_clear_o,
134 input logic b_clear_ack_i,
135 output logic b_isolate_o,
136 input logic b_isolate_ack_i
137 );
138
139 (* dont_touch = "true" *)
140 logic async_a2b_req, async_b2a_ack;
141 (* dont_touch = "true" *)
142 cdc_clear_seq_phase_e async_a2b_next_phase;
143 (* dont_touch = "true" *)
144 logic async_b2a_req, async_a2b_ack;
145 (* dont_touch = "true" *)
146 cdc_clear_seq_phase_e async_b2a_next_phase;
147
148 cc_cdc_reset_ctrlr_half #(
149     .SyncStages ( SyncStages ),
150     .ClearOnAsyncReset ( ClearOnAsyncReset )
151 ) i_cdc_reset_ctrlr_half_a (
152     .clk_i ( a_clk_i ),
153     .rst_ni ( a_rst_ni ),
154     .clear_i ( a_clear_i ),
155     .clear_o ( a_clear_o ),
156     .clear_ack_i ( a_clear_ack_i ),
157     .isolate_o ( a_isolate_o ),
158     .isolate_ack_i ( a_isolate_ack_i ),
159     (* async *) .async_next_phase_o ( async_a2b_next_phase ),
160     (* async *) .async_req_o ( async_a2b_req ),
161     (* async *) .async_ack_i ( async_b2a_ack ),
162     (* async *) .async_next_phase_i ( async_b2a_next_phase ),
163     (* async *) .async_req_i ( async_b2a_req ),
164     (* async *) .async_ack_o ( async_a2b_ack )
165 );
166
167 cc_cdc_reset_ctrlr_half #(
168     .SyncStages ( SyncStages ),
169     .ClearOnAsyncReset ( ClearOnAsyncReset )
170 ) i_cdc_reset_ctrlr_half_b (
171     .clk_i ( b_clk_i ),
172     .rst_ni ( b_rst_ni ),
173     .clear_i ( b_clear_i ),
174     .clear_o ( b_clear_o ),
175     .clear_ack_i ( b_clear_ack_i ),
176     .isolate_o ( b_isolate_o ),
177     .isolate_ack_i ( b_isolate_ack_i ),
178     (* async *) .async_next_phase_o ( async_b2a_next_phase ),
179     (* async *) .async_req_o ( async_b2a_req ),
180     (* async *) .async_ack_i ( async_a2b_ack ),
181     (* async *) .async_next_phase_i ( async_a2b_next_phase ),
182     (* async *) .async_req_i ( async_a2b_req ),
183     (* async *) .async_ack_o ( async_b2a_ack )
184 );
185 endmodule
186
187
188 module cc_cdc_reset_ctrlr_half
189     import cc_pkg::*;
190     #(
191         /// The number of synchronization stages to use for the
192         /// clear signal request/acknowledge. Must be less than
193         /// the number of sync stages used in the CDC
194         parameter int unsigned SyncStages = 2,
195         /// Whether an asynchronous reset shall cause a clear
196         /// request to be sent to the other side.
197         parameter logic ClearOnAsyncReset = 1'b1
198     ) (
199         /// Synchronous side
200         input logic clk_i,
201         input logic rst_ni,
202         input logic clear_i,
203         output logic isolate_o,
204         input logic isolate_ack_i,
205         output logic clear_o,
206         input logic clear_ack_i,
207         /// Asynchronous clear sequence handshaking
208         output cdc_clear_seq_phase_e async_next_phase_o,
209         output logic async_req_o,
210         input logic async_ack_i,
211         input cdc_clear_seq_phase_e async_next_phase_i,
212         input logic async_req_i,
213         output logic async_ack_o
214     );
215
216     /// How this module works:
217
218     /// The module is split into two parts. The initiator part
219     /// consists of an FSM
220     /// that is triggered by the clear_i signal and transitions
221     /// through reset
222     /// sequence. During those transitions, the `initiat-
223     /// or_isolate_out` and
224     /// `initiator_clear_out` signals are asserted appropriately.
225
226     /// The receiver part receives the state transitions from the
227     /// other clock
228     /// domain (initiator part of the `cc_cdc_reset_ctrlr_half`

```

```

instance in the other
226     /// clock domain) and asserts the `receiver_isolate_out` and
227     /// `receiver_clear_out` appropriately (considering the
228     /// `isolate_ack_i`
229     /// signal).
230     /// In both, the initiator and the receiver part, the re-
231     /// spective FSM
232     /// transitions through 4 phases. In the ISOLATE phase, the
233     /// isolate signal is
234     /// asserted and the connected CDCs are expected to block all
235     /// further
236     /// interactions with the outside world and acknowledge the
237     /// isolation with the
238     /// isolate_ack_i signal. In the CLEAR phase, the clear sig-
239     /// nal is asserted
240     /// which resets the internal state of the CDC while keeping
241     /// the isolate signal
242     /// asserted. In the POST_CLEAR phase, the clear signal is
243     /// deasserted. Finally,
244     /// when returning to the IDLE phase, the isolate signal is
245     /// deasserted to
246     /// continue normal operation. The FSM uses a dedicated 4-
247     /// phase handshaking CDC
248     /// to transition between the phases in lock-step and trans-
249     /// mits the current
250     /// state to the other domain to avoid issues if the other
251     /// domain is reset
252     /// asynchronously while a clear procedure is pending.
253     /// ----- Initiator Side
254     /// Sends clear sequence state transitions to the other side.
255     typedef enum logic[3:0] {
256         IDLE,
257         ISOLATE,
258         WAIT_ISOLATE_PHASE_ACK,
259         WAIT_ISOLATE_ACK,
260         CLEAR,
261         WAIT_CLEAR_PHASE_ACK,
262         WAIT_CLEAR_ACK,
263         POST_CLEAR,
264         FINISHED
265     } initiator_state_e;
266     initiator_state_e initiator_state_d, initiator_state_q;
267     localparam initiator_state_e InitiatorStateResetValue =
268     ClearOnAsyncReset ? ISOLATE : IDLE;
269     /// The current phase of the clear sequence, sent to the oth-
270     /// er side using a
271     /// 4-phase CDC
272     cdc_clear_seq_phase_e initiator_clear_seq_phase;
273     logic initiator_phase_transition_req;
274     logic initiator_phase_transition_ack;
275     logic initiator_isolate_out;
276     logic initiator_clear_out;
277     logic initiator_clear_seq_phase = CDC_CLEAR_PHASE_IDLE;
278     /// The phase CDC is a real CDC even when callers pass a re-
279     /// duced latency for
280     /// synchronous-clear-only configurations.
281     localparam int unsigned ResetSyncStages = (SyncStages > 2) ?
282     SyncStages : 2;
283     always_comb begin
284         initiator_state_d = initiator_state_q;
285         initiator_phase_transition_req = 1'b0;
286         initiator_isolate_out = 1'b0;
287         initiator_clear_out = 1'b0;
288         initiator_clear_seq_phase = CDC_CLEAR_PHASE_IDLE;
289         case (initiator_state_q)
290             IDLE: begin
291                 if (clear_i) begin
292                     initiator_state_d = ISOLATE;
293                 end
294             end
295             ISOLATE: begin
296                 initiator_phase_transition_req = 1'b1;
297                 initiator_clear_seq_phase =
298                 CDC_CLEAR_PHASE_ISOLATE;
299                 initiator_isolate_out = 1'b1;
300                 initiator_clear_out = 1'b0;
301                 if (initiator_phase_transition_ack && isolate_ack_i)
302                     begin
303                         initiator_state_d = CLEAR;
304                     end
305                 else if (initiator_phase_transition_ack) begin
306                     initiator_state_d = WAIT_ISOLATE_ACK;
307                 end
308                 else if (isolate_ack_i) begin
309                     initiator_state_d = WAIT_ISOLATE_PHASE_ACK;
310                 end
311             end
312             WAIT_ISOLATE_ACK: begin
313                 initiator_isolate_out = 1'b1;
314                 initiator_clear_out = 1'b0;
315                 initiator_clear_seq_phase = CDC_CLEAR_PHASE_ISOLATE;
316                 if (isolate_ack_i) begin
317                     initiator_state_d = CLEAR;
318                 end
319             end
320             WAIT_ISOLATE_PHASE_ACK: begin
313                 initiator_isolate_out = 1'b1;
314                 initiator_clear_out = 1'b0;
315                 initiator_clear_seq_phase = CDC_CLEAR_PHASE_ISOLATE;
316                 if (isolate_ack_i) begin
317                     initiator_state_d = CLEAR;
318                 end
319             end

```



```

305     end
306   end
307
308   WAIT_ISOLATE_PHASE_ACK: begin
309     initiator_phase_transition_req = 1'b1;
310     initiator_clear_seq_phase     =
CDC_CLEAR_PHASE_ISOLATE;
311     initiator_isolate_out         = 1'b1;
312     initiator_clear_out           = 1'b0;
313     if (initiator_phase_transition_ack) begin
314       initiator_state_d = CLEAR;
315     end
316   end
317
318   CLEAR: begin
319     initiator_isolate_out         = 1'b1;
320     initiator_clear_out           = 1'b1;
321     initiator_phase_transition_req = 1'b1;
322     initiator_clear_seq_phase     =
CDC_CLEAR_PHASE_CLEAR;
323     if (initiator_phase_transition_ack && clear_ack_i) be-
gin
324       initiator_state_d = POST_CLEAR;
325     end else if (initiator_phase_transition_ack) begin
326       initiator_state_d = WAIT_CLEAR_ACK;
327     end else if (clear_ack_i) begin
328       initiator_state_d = WAIT_CLEAR_PHASE_ACK;
329     end
330   end
331
332   WAIT_CLEAR_ACK: begin
333     initiator_isolate_out         = 1'b1;
334     initiator_clear_out           = 1'b1;
335     initiator_clear_seq_phase     = CDC_CLEAR_PHASE_CLEAR;
336     if (clear_ack_i) begin
337       initiator_state_d = POST_CLEAR;
338     end
339   end
340
341   WAIT_CLEAR_PHASE_ACK: begin
342     initiator_phase_transition_req = 1'b1;
343     initiator_clear_seq_phase     =
CDC_CLEAR_PHASE_CLEAR;
344     initiator_isolate_out         = 1'b1;
345     initiator_clear_out           = 1'b1;
346     if (initiator_phase_transition_ack) begin
347       initiator_state_d = POST_CLEAR;
348     end
349   end
350
351   POST_CLEAR: begin
352     initiator_isolate_out         = 1'b1;
353     initiator_clear_out           = 1'b0;
354     initiator_phase_transition_req = 1'b1;
355     initiator_clear_seq_phase     =
CDC_CLEAR_PHASE_POST_CLEAR;
356     if (initiator_phase_transition_ack) begin
357       initiator_state_d = FINISHED;
358     end
359   end
360
361   FINISHED: begin
362     initiator_isolate_out         = 1'b1;
363     initiator_clear_out           = 1'b0;
364     initiator_phase_transition_req = 1'b1;
365     initiator_clear_seq_phase     = CDC_CLEAR_PHASE_IDLE;
366     if (initiator_phase_transition_ack) begin
367       initiator_state_d = IDLE;
368     end
369   end
370
371   default: begin
372     initiator_state_d = ISOLATE;
373   end
374 endcase
375 end
376
377 // Start in the ISOLATE state if asynchronous reset should
launch a clear sequence.
378 `FF(initiator_state_q, initiator_state_d, InitiatorStateR-
esetValue, clk_i, rst_ni)
379
380 // Initiator CDC SRC
381
381 // We use 4 phase handshaking. That way it doesn't matter if
one side is
382 // suddenly reset asynchronously. With a 2phase CDC, one-
sided async resets might
383 // introduce spurious transactions.
384
385 cc_cdc_4phase_src #(
386   .data_t(cdc_clear_seq_phase_e),
387   .SyncStages(ResetSyncStages),
388   .Decoupled(0), // Important! The CDC must not be in de-
coupled mode.
389   // Otherwise we will proceed to the next
state without
390   // waiting for the new state to arrive on
the other side.
391   .SendResetMsg(ClearOnAsyncReset), // Send the ISOLATE
phase request immediately on async

```

```

392   // reset if async reset
synchronization is enabled.
393   .ResetMsg(CDC_CLEAR_PHASE_ISOLATE)
394 ) i_state_transition_cdc_src(
395   .clk_i,
396   .rst_ni,
397   .data_i(initiator_clear_seq_phase),
398   .valid_i(initiator_phase_transition_req),
399   .ready_o(initiator_phase_transition_ack),
400   .async_req_o,
401   .async_ack_i,
402   .async_data_o(async_next_phase_o)
403 );
404
405 //----- Receiver Side
406
407 // This part of the circuit receives clear sequence state
transitions from the
408 // other side.
409
410 cdc_clear_seq_phase_e receiver_phase_d, receiver_phase_q;
411 cdc_clear_seq_phase_e receiver_effective_phase;
412 cdc_clear_seq_phase_e receiver_next_phase;
413
414 logic receiver_phase_req;
415 logic receiver_phase_ack;
416 logic receiver_phase_pending_d, receiver_phase_pending_q;
417 logic receiver_phase_done_d, receiver_phase_done_q;
418
419 logic receiver_capture_phase;
420
421 logic receiver_isolate_out;
422 logic receiver_clear_out;
423
424 cc_cdc_4phase_dst #(
425   .data_t(cdc_clear_seq_phase_e),
426   .SyncStages(ResetSyncStages),
427   .Decoupled(0) // Important! The CDC must not be in de-
coupled mode. Otherwise
428   // we will proceed to the next state without
waiting for the
429   // new state to arrive on the other side.
430 ) i_state_transition_cdc_dst(
431   .clk_i,
432   .rst_ni,
433   .data_o(receiver_next_phase),
434   .valid_o(receiver_phase_req),
435   .ready_i(receiver_phase_ack),
436   .async_req_i,
437   .async_ack_o,
438   .async_data_i(async_next_phase_i)
439 );
440
441 // Capture async phase payload after synchronized request
442 // Do not decode receiver_next_phase into clear/isolate
assign receiver_capture_phase =
443   receiver_phase_req
444   && !receiver_phase_pending_q
445   && !receiver_phase_done_q;
446
447 always_comb begin
448   receiver_phase_d = receiver_phase_q;
449   receiver_phase_pending_d = receiver_phase_pending_q;
450   receiver_phase_done_d = receiver_phase_done_q;
451
452   if (!receiver_phase_req) begin
453     receiver_phase_done_d = 1'b0;
454   end
455
456   if (receiver_capture_phase) begin
457     receiver_phase_d = receiver_next_phase;
458     receiver_phase_pending_d = 1'b1;
459   end
460
461   if (receiver_phase_ack) begin
462     receiver_phase_pending_d = 1'b0;
463     receiver_phase_done_d = 1'b1;
464   end
465 end
466
467 `FF(receiver_phase_q, receiver_phase_d,
CDC_CLEAR_PHASE_IDLE, clk_i, rst_ni)
468 `FF(receiver_phase_pending_q, receiver_phase_pending_d,
1'b0, clk_i, rst_ni)
469 `FF(receiver_phase_done_q, receiver_phase_done_d,
1'b0, clk_i, rst_ni)
470
471 assign receiver_effective_phase = receiver_phase_q;
472
473 always_comb begin
474   receiver_isolate_out = 1'b0;
475   receiver_clear_out = 1'b0;
476   receiver_phase_ack = 1'b0;
477
478   unique case (receiver_effective_phase)
479     CDC_CLEAR_PHASE_IDLE: begin
480       receiver_clear_out = 1'b0;
481       receiver_isolate_out = 1'b0;
482       receiver_phase_ack = receiver_phase_req && receiv-

```

```

er_phase_pending_q;
484     end
485     CDC_CLEAR_PHASE_ISOLATE: begin
486         receiver_clear_out = 1'b0;
487         receiver_isolate_out = 1'b1;
488         receiver_phase_ack = receiver_phase_req && receive-
er_phase_pending_q && isolate_ack_i;
489     end
490     CDC_CLEAR_PHASE_CLEAR: begin
491         receiver_clear_out = 1'b1;
492         receiver_isolate_out = 1'b1;
493         receiver_phase_ack = receiver_phase_req && receive-
er_phase_pending_q && clear_ack_i;
494     end
495     CDC_CLEAR_PHASE_POST_CLEAR: begin
496         receiver_clear_out = 1'b0;
497         receiver_isolate_out = 1'b1;
498         receiver_phase_ack = receiver_phase_req && receive-
er_phase_pending_q;
499     end
500     default: begin
501         receiver_clear_out = 1'b0;
502         receiver_isolate_out = 1'b0;
503         receiver_phase_ack = 1'b0;
504     end
505 endcase
506 // Preserve async-reset ordering margin without
507 async_next_phase_i decode
508 if (receiver_capture_phase) begin
509     receiver_isolate_out = 1'b1;
510 end
511 end
512 // Output Assignment
513 // The clear and isolate signal are the OR combination of
514 // the receiver and
515 // initiator's clear/isolate signal. This ensures that the
516 // correct sequence is
517 // followed even if both sides are cleared independently at
518 // roughly the same
519 // time.
520 assign clear_o = initiator_clear_out || receiver_clear_out;
521 assign isolate_o = initiator_isolate_out || receive-
er_isolate_out;
522 endmodule : cc_cdc_reset_ctrlr_half

```

15. pulp-platform_common_cells/src/cc_clk_int_div.v (383 lines)

```

1 //-----
2 // Title : Configurable Integer Clock Divider
3 //
4 // File : cc_clk_int_div.v Author : Manuel Eggimann <meggim-
mann@iis.ee.ethz.ch>
5 // Created : 17.03.2022
6 //
7 // Description :
8 //
9 // This module implements an at runtime configurable integer
10 // clock divider that
11 // always generates clean 50% duty cycle output clock. Clock
12 // divider setting
13 // changes are handshaked and during the transitioning phase
14 // between cc_clk_div
15 // value changes, the output clock is gated to prevent clock
16 // glitches and no
17 // other cc_clk_div change request is accepted. clk_o remains
18 // gated for at most
19 // 3x<new clk period> clk_i cycles. If the new div_i value
20 // equals the currently
21 // configured value, the clock is not gated and the handshake
22 // is immediately
23 // granted. It is thus safe to statically tie the valid signal to
24 // logic high if
25 // we can guarantee, that the div_i value remains stable long
26 // enough (upper
27 // limit 2 output clock cycles).
28 //
29 // The `en_i` signal can be used to enable or disable the out-
30 // put clock in a safe
31 // manner.
32 //
33 // If a div value of 0 or 1 is requested, the input clock is
34 // feed through to the
35 // output clock. However, the same gating rules apply (again
36 // to prevent
37 // glitches).
38 //
39 // If test_mode_en_i is asserted, the output clock gate is by-
40 // passed entirely and
41 // clk_o will always be directly driven by clk_i. Use this

```

```

mode for DFT of the
29 // downstream logic.
30 //
31 // Parameters:
32 //
33 // DivValueWidth: The number of bits to use for the internal
34 // counter. Defines the maximum division factor.
35 //
36 // DefaultDivValue: The default division factor to use after
37 // reset. Use this
38 // parameter and tie div_valid_i to zero if you don't need at
39 // runtime
40 // configurability. An elaboration time error will be issued
41 // if the supplied
42 // default div value is not representable with DivValueWidth
43 // bits.
44 //
45 // EnableClockInReset: If 1'b1, the clock gate will be enabled
46 // during reset
47 // which allows the cc_clk_int_div instance to bypass the
48 // clock during reset, IFF
49 // the DefaultDivValue is 1. For all other DefaultDivValues,
50 // the output
51 // clock will not be available until rst_ni is deasserted!
52 //
53 // IMPORTANT!!!
54 //
55 // All clock gating/logic within this design is performed by
56 // dedicated clock logic
57 // tech cells. By default the common_cell library uses the be-
58 // havioral models in
59 // the `tech_cells_generic` repository. However, for synthesis
60 // these cells need to be
61 // mapped to dedicated cells from your standard cell library,
62 // preferably ones
63 // that are designed for clock logic (they have balanced rise
64 // and fall time).
65 // During synthesis you furthermore have to properly set
66 // `dont_touch` or
67 // `size_only` attributes to prevent the logic synthesizer
68 // from replacing those
69 // cells with regular logic gates which could end up being
70 // glitchty!
71 //
72 //-----
73 // Copyright (C) 2022 ETH Zurich, University of Bologna Copy-
74 // right and related
75 // rights are licensed under the Solderpad Hardware License,
76 // Version 0.51 (the
77 // "License"); you may not use this file except in compliance
78 // with the License.
79 // You may obtain a copy of the License at
80 // http://solderpad.org/licenses/SHL-0.51. Unless required by
81 // applicable law or
82 // agreed to in writing, software, hardware and materials dis-
83 // tributed under this
84 // license is distributed on an "AS IS" BASIS, WITHOUT WAR-
85 // RANTIES OR CONDITIONS
86 // OF ANY KIND, either express or implied. See the License for
87 // the specific
88 // language governing permissions and limitations under the
89 // License.
90 // SPDX-License-Identifier: SHL-0.51
91 //
92 //-----
93 // include "common_cells/registers.svh"
94 //
95 // module cc_clk_int_div #(
96 //     // The with
97 //     parameter int unsigned DivValueWidth = 4,
98 //     // The default divider value which is used right after re-
99 //     set
100 //     parameter int unsigned DefaultDivValue = 0,
101 //     // If 1'b1, the output clock is enabled during async reset
102 //     assertion
103 //     parameter bit EnableClockInReset = 1'b0
104 // ) (
105 //     // Clock.
106 //     input logic clk_i,
107 //     // Asynchronous reset active low.
108 //     input logic rst_ni,
109 //     // Active-high output clock enable. Controls a glitch-free
110 //     clock gate so the
111 //     // enable signal may be driven by combinational logic
112 //     without introducing
113 //     // glitches.
114 //     input logic en_i,
115 //     // If asserted (active-high) bypass the clock divider and
116 //     drive clk_o
117 //     // directly with clk_i.
118 //     input logic test_mode_en_i,
119 //     // Divider select value. The output clock has a frequency
120 //     of f_clk_i/div_i.
121 //     // For div_i == 0 or div_i == 1, the output clock has the
122 //     same frequency as

```



```

93  /// th input clock.
94  input logic [DivValueWidth-1:0] div_i,
95  /// Valid handshake signal. Must not combinationaly depend
on 'div_ready_o'.
96  /// Once asserted, the valid signal must not be deasserted
until it is
97  /// accepted with 'div_ready_o'.
98  input logic div_valid_i,
99  output logic div_ready_o,

100  /// Generated output clock. Given a glitch free input clock,
the output clock
101  /// is guaranteed to be glitch free with 50% duty cycle, re-
gardless the timing
102  /// of reconfiguration requests or en_i de/assetion. During
the

103  /// reconfiguration, the output clock is gated with its next
falling edge and
104  /// remains gated (idle-low) for at least one period of the
new target output
105  /// period to filter out any glitches during the config
transition.
106  output logic clk_o,
107  /// Current value of the internal cycle counter. Might be
usefull if you need
108  /// to do some phase shifting relative to the generated
clock.
109  output logic [DivValueWidth-1:0] cycl_count_o
110  );
111
112  if ($clog2(DefaultDivValue+1) > DivValueWidth) begin :
gen_elab_error
113    $error("Default divider value %0d is not representable
with the configured",
114           "div value width of %0d bits.",
115           DefaultDivValue, DivValueWidth);
116  end
117
118  /// We have to preset the div_q register with a value larger
than one to avoid
119  /// an infinite loop in the WAIT_END_PERIOD state. If the de-
fault state of the

120  /// clock divider should be bypass, we thus always preset the
div_q with 1
121  /// rather than 1 or zero.
122  localparam int unsigned DivResetValue = (DefaultDivValue !=
0)? DefaultDivValue: 1;
123
124  logic [DivValueWidth-1:0] div_i_normalized;
125  logic [DivValueWidth-1:0] div_d, div_q;
126  logic toggle_ffs_en;
127  logic t_ff1_d, t_ff1_q;
128  logic t_ff1_en;
129
130  logic t_ff2_d, t_ff2_q;
131  logic t_ff2_en;
132
133  logic [DivValueWidth-1:0] cycle_cntr_d, cycle_cntr_q;
134  logic cycle_counter_en;
135  logic clk_div_bypass_en_d, clk_div_bypass_en_q;
136  logic odd_clk;
137  logic even_clk;
138  logic generated_clock;
139  logic ungated_output_clock;
140
141  logic use_odd_division_d, use_odd_division_q;
142  logic gate_en_d, gate_en_q;
143  logic gate_is_open_q;
144  logic clear_cycle_counter;
145  logic clear_toggle_flops;
146
147  typedef enum logic[1:0] {IDLE, LOAD_DIV, WAIT_END_PERIOD}
clk_gate_state_e;
148  clk_gate_state_e clk_gate_state_d, clk_gate_state_q;
149
150  /// Normalize the div_i value. div_i == 0 and div_i == 1 have
the same meaning
151  /// but actually loading 0 causes issues in the FSM. We thus
always load 1 for
152  /// both cases
153  assign div_i_normalized = (div_i != 0)? div_i : 1;
154
155
156  //----- Divider Load FSM -----
157  always_comb begin
158    div_d = div_q;
159    div_ready_o = 1'b0;
160    clk_div_bypass_en_d = clk_div_bypass_en_q;
161    use_odd_division_d = use_odd_division_q;
162    clk_gate_state_d = clk_gate_state_q;
163    cycle_counter_en = 1'b1;
164    clear_cycle_counter = 1'b0;
165    clear_toggle_flops = 1'b0;
166    toggle_ffs_en = 1'b1;
167    gate_en_d = 1'b0;
168
169    case (clk_gate_state_q)
170      IDLE: begin

```

```

171    gate_en_d = 1'b1;
172    toggle_ffs_en = 1'b1;
173    if (div_valid_i) begin
174      if (div_i_normalized == div_q) begin
175        div_ready_o = 1'b1;
176      end else begin
177        clk_gate_state_d = LOAD_DIV;
178        gate_en_d = 1'b0;
179      end
180    // If we disabled the clock output, stop the cycle
counter and the
181    // toggle flip-flops at the start of the next period
to safe energy.
182    end else if (!en_i && gate_is_open_q == 1'b0) begin
183      cycle_counter_en = 1'b0;
184      toggle_ffs_en = 1'b0;
185    end
186  end
187
188  LOAD_DIV: begin
189    gate_en_d = 1'b0;
190    toggle_ffs_en = 1'b1;
191    // Wait until the ouptut clock is currently deasser-
ted. This ensures
192    // that the clock gate disable (gate_en) was latched
and changing the
193    // div_q and the cycle_counter_q value cannot affect
the output clock
194    // any longer.
195    if ((gate_is_open_q == 1'b0) || clk_div_bypass_en_q)
begin
196      // Now clear the cycle counter and the toggle flip
flops to have a
197      // deterministic output phase (rising edge of output
clock always
198      // coincides with first rising edge of input clock
when cycl_count_o
199      // == 0).
200      toggle_ffs_en = 1'b0;
201      div_d = div_i_normalized;
202      div_ready_o = 1'b1;
203      clear_cycle_counter = 1'b1;
204      clear_toggle_flops = 1'b1;
205      use_odd_division_d = div_i_normalized[0];
206      clk_div_bypass_en_d = div_i_normalized == 1;
207      clk_gate_state_d = WAIT_END_PERIOD;
208    end
209  end
210
211  WAIT_END_PERIOD: begin
212    gate_en_d = 1'b0;
213    // Keep the toggle flip-flops disabled until we reach
idle state.
214    // Otherwise, the start state of the t-ffs depends on
the number of wait
215    // cycles which would yield different output clock
phase depending on
216    // the number of wait cycles.
217    toggle_ffs_en = 1'b0;
218    if (cycle_cntr_q == div_q - 1) begin
219      clk_gate_state_d = IDLE;
220    end
221  end
222
223  default: begin
224    clk_gate_state_d = IDLE;
225  end
226 endcase
227 end
228
229  localparam logic UseOddDivisionResetValue = DefaultDiv-
Value[0];
230  localparam logic ClkDivBypassEnResetValue = (DefaultDivValue <
2)? 1'b1: 1'b0;
231
232  `FF(use_odd_division_q, use_odd_division_d, UseOddDivi-
sionResetValue, clk_i, rst_ni)
233  `FF(clk_div_bypass_en_q, clk_div_bypass_en_d, ClkDivBy-
passEnResetValue, clk_i, rst_ni)
234  `FF(div_q, clk_i, rst_ni) div_d, DivReset-
Value,
235  `FF(clk_gate_state_q, clk_gate_state_d,
IDLE, clk_i, rst_ni)
236  `FF(gate_en_q, gate_en_d, EnableClockIn-
Reset, clk_i, rst_ni)
237
238  //----- Cycle Counter
239
240  // Cycle Counter
241  always_comb begin
242    cycle_cntr_d = cycle_cntr_q;
243    // Reset the counter if we load a new divider value.
244    if (clear_cycle_counter) begin
245      cycle_cntr_d = '0;
246    end else begin
247      if (cycle_counter_en) begin
248        // During normal operation, reset the counter whenever
it reaches

```

```

249 // <target_count>-1. In bypass mode (div == 0 or div
== 1) disable the
250 // counter to save power.
251 if (clk_div_bypass_en_q || (cycle_cntr_q == div_q-1))
begin
252     cycle_cntr_d = '0;
253 end else begin
254     cycle_cntr_d = cycle_cntr_q + 1;
255 end
256 end
257 end
258 end
259 `FF(cycle_cntr_q, cycle_cntr_d, '0, clk_i, rst_ni)
260 assign cycl_count_o = cycle_cntr_q;
261
262 //----- T-Flip-Flops
263
264 // These T-Flip-Flop intentionally use blocking assignments!
265 // If we were to use
266 // non-blocking assignment like we normally do for flip-
267 // flops, we would create
268 // a race condition when sampling data from the fast clock
269 // domain into
270 // flip-flops clocked by t_ff1_q and t_ff2_q. To avoid this,
271 // we use blocking assignments
272 // which is the recommended method according to:
273 // S. Sutherland and D. Mills,
274 // Verilog and System Verilog gotchas: 101 common coding er-
275 // rors and how to
276 // avoid them. New York: Springer, 2007. page 64.
277 always_ff @(posedge clk_i, negedge rst_ni) begin
278     if (!rst_ni) begin
279         // verilog_lint: waive always-ff-non-blocking
280         t_ff1_q = '0; // Intentional blocking assignment! Do not
281         // replace!
282         end else begin
283             if (t_ff1_en) begin
284                 // verilog_lint: waive always-ff-non-blocking
285                 t_ff1_q = t_ff1_d; // Intentional blocking assignment!
286                 // Do not replace!
287             end
288         end
289     end
290
291 // The second flip-flop is required for odd integer division
292 // and needs to
293 // negative edge triggered.
294 always_ff @(negedge clk_i, negedge rst_ni) begin
295     if (!rst_ni) begin
296         // verilog_lint: waive always-ff-non-blocking
297         t_ff2_q = '0; // Intentional blocking assignment! Do not
298         // replace!
299     end else begin
300         if (t_ff2_en) begin
301             // verilog_lint: waive always-ff-non-blocking
302             t_ff2_q = t_ff2_d; // Intentional blocking assignment!
303             // Do not replace!
304         end
305     end
306
307 always_comb begin
308     if (clear_toggle_flops) begin
309         t_ff1_d = '0;
310         t_ff2_d = '0;
311     end else begin
312         t_ff1_d = t_ff1_en? !t_ff1_q: t_ff1_q;
313         t_ff2_d = t_ff2_en? !t_ff2_q: t_ff2_q;
314     end
315 end
316
317 //----- FSM to control T-FF enable and clk_div_bypass_en
318
319 always_comb begin
320     t_ff1_en = '1'b0;
321     t_ff2_en = '1'b0;
322     if (!clk_div_bypass_en_q && toggle_ffs_en) begin
323         if (use_odd_division_q) begin
324             t_ff1_en = (cycle_cntr_q == 0)? '1'b1: '1'b0;
325             t_ff2_en = (cycle_cntr_q == (div_q+1)/2)? '1'b1: '1'b0;
326         end else begin
327             t_ff1_en = (cycle_cntr_q == 0 || cycle_cntr_q ==
div_q/2)? '1'b1: '1'b0;
328         end
329     end
330 end
331
332 assign even_clk = t_ff1_q;

```

```

328
329 //----- Clock XOR for the odd division logic
330 tc_clk_xor2 i_odd_clk_xor (
331     .clk0_i ( t_ff1_q ),
332     .clk1_i ( t_ff2_q ),
333     .clk_o ( odd_clk )
334 );
335
336 //---- Clock MUX to select between odd and even div logic
337 tc_clk_mux2 i_clk_mux (
338     .clk0_i ( even_clk ),
339     .clk1_i ( odd_clk ),
340     .clk_sel_i ( use_odd_division_q ),
341     .clk_o ( generated_clock )
342 );
343
344 //----- clock mux to bypass clock if divide-
345 // by-1
346 tc_clk_mux2 i_clk_bypass_mux (
347     .clk0_i ( generated_clock ),
348     .clk1_i ( clk_i ),
349     .clk_sel_i ( clk_div_bypass_en_q || test_mode_en_i ),
350     .clk_o ( ungated_output_clock )
351 );
352
353 //----- Clock gate logic
354 // During the transitioning phase, we gate the clock to pre-
355 // vent clock glitches
356
357 // The gate_is_open_q signal is used by the FSM to determine
358 // whether the
359 // gate_en signal has been latched by the clock gate cell,
360 // i.e. if this signal
361 // is '1'b0, it means not only that the clock gate should be
362 // disabled but also that
363 // clk_o is currently LOW and thus the enable signal has
364 // been latched and the
365 // output clock will remain LOW until we enable the clock
366 // again.
367 // The FSM needs to know this to not disable the T-FFs and
368 // the cycle counter
369
370 // to early. Otherwise the clock might get stuck active high
371 // or we deassert a
372 // clock to early
373 always_ff @(posedge ungated_output_clock, negedge rst_ni)
374 begin
375     if (!rst_ni) begin
376         gate_is_open_q <= '1'b0;
377     end else begin
378         gate_is_open_q <= gate_en_q & en_i;
379     end
380 end
381
382 tc_clk_gating #(
383     .IS_FUNCTIONAL(1) // The gate is required to prevent
384     // glitches during
385     // transitioning. Target specific imple-
386     // mentations must not
387     // remove it to save ICGs (e.g. in FP-
388     // GAs).
389 ) i_clk_gate (
390     .clk_i ( ungated_output_clock ),
391     .en_i ( gate_en_q & en_i ),
392     .test_en_i ( test_mode_en_i ),
393     .clk_o
394 );
395
396 endmodule

```

16. pulp-platform_common_cells/src/cc_clk_int_div_static.sv (95 lines)

```

1 //-----
2 // Title : Static Integer Clock Divider
3 //-----
4 // File : cc_clk_int_div_static.sv
5 // Author : Manuel Eggimann <meggimann@iis.ee.ethz.ch>
6 // Created : 08.05.2023
7 //-----
8 // Description :
9 //
10 // This module implements a static arbitrary integer divider.
11 // Static in this
12 // case means, the divider value is constant at elaboration
13 // time (SV parameter).
14 // It supports arbitrary integer division with a guaranteed
15 // 50% duty cycle for
16 // odd and even division.
17 //
18 // Internally, this module is wrapper around the "at-runtime"
19 // configurable

```

```

16 // `cc_clk_int_div` module. If you need to change the division
factor at-runtime you

17 // should directly use `cc_clk_int_div` instead. However if all
you need is a single
18 // division factor this module provides a convenience wrapper
for you with a
19 // simplified interface.
20 //
21 // The `en_i` signal can be used to enable or disable the out-
put clock in a safe
22 // manner (there is an internal, glitch-free clock gate).
23 //
24 // Parameters:
25 //
26 // DivValue: The integer value by which the clock shall be di-
vided. Must be
27 // non-zero integer smaller than 2^32-1.
28 //

29 // EnableClockInReset: If 1'b1, the clock gate will be enabled
during reset
30 // which allows the cc_clk_int_div instance to bypass the
clock during reset, IFF
31 // the DefaultDivValue is 1. For all other DefaultDivValues,
the output
32 // clock will not be available until rst_ni is deasserted!
33 //
34 // IMPORTANT!!!
35 //
36 // All clock gating/logic within this design is performed by
dedicated clock
37 // logic tech cells. By default the common_cell library uses
the behavioral

38 // models in the `tech_cells_generic` repository. However, for
synthesis these
39 // cells need to be mapped to dedicated cells from your stand-
ard cell library,
40 // preferably ones that are designed for clock logic (they
have balanced rise
41 // and fall time). During synthesis you furthermore have to
properly set

42 // `dont_touch` or `size_only` attributes to prevent the logic
synthesizer from
43 // replacing those cells with regular logic gates which could
end up being
44 // glitchy!
45 //
46 //-----
47 // Copyright (C) 2023 ETH Zurich, University of Bologna Copy-
right and related
48 // rights are licensed under the Solderpad Hardware License,
Version 0.51 (the
49 // "License"); you may not use this file except in compliance
with the License.
50 // You may obtain a copy of the License at
51 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law or
52 // agreed to in writing, software, hardware and materials dis-
tributed under this
53 // License is distributed on an "AS IS" BASIS, WITHOUT WAR-
RANTIES OR CONDITIONS

54 // OF ANY KIND, either express or implied. See the License for
the specific
55 // language governing permissions and limitations under the
License.
56 // SPDX-License-Identifier: SHL-0.51
57 //
58 //-----
59
60 module cc_clk_int_div_static #(
61     parameter int unsigned DivValue = 1,
62     parameter bit EnableClockInReset = 1'b1
63 ) (
64     input logic clk_i,           // Clock
65     input logic rst_ni,         // Asynchronous reset active
66     input logic en_i,           //
67     input logic test_mode_en_i, //
68     output logic clk_o
69 );
70 if (DivValue == 0) begin : gen_elab_error
71     $error("DivValue must be strictly larger than 0.");
72 end
73
74 localparam int unsigned DivValueWidth = $clog2(DivValue+1);
75
76 logic [DivValueWidth-1:0] div_value;
77 assign div_value = DivValue;
78
79 cc_clk_int_div #(
80     .DivValueWidth ( DivValueWidth ),
81     .DefaultDivValue ( DivValue ),
82     .EnableClockInReset ( EnableClockInReset )
83 ) i_clk_int_div (

```

```

84     .clk_i,
85     .rst_ni,
86     .en_i,
87     .test_mode_en_i,
88     .div_i ( div_value ),
89     .div_valid_i ( 1'b0 ),
90     .div_ready_o ( ),
91     .clk_o,
92     .cycl_count_o ( )
93 );
94
95 endmodule

```

17. pulp-platform_common_cells/src/ cc_clk_mux_glitch_free.sv (291 lines)

```

1 //-----
2 // Title           : Glitch-free Clock Multiplexer
3 //-----
4 // File            : cc_clk_mux_glitch_free.sv
5 // Author           : Manuel Eggimann <meggimann@iis.ee.ethz.ch>
6 // Created          : 10.12.2022
7 //-----
8 // Description :
9 //
10 // This module allows glitch free clock multiplexing between N
arbitrary input
11 // clock with completely unknown phase relation ships. The
module will make
12 // sure to first synchronize the clock multiplexer signal to
the relevant clock
13 // domains and ensures glitch free switching between the
source clock and the
14 // new target clock by silencing the output at appropriate
times. The clock
15 // signals themselves only pass through: 1 clock-gate, 1 N-in-
put clock-OR Gate,
16 // 1 2-input clock mux. All these cells are referenced from
the tech_cells
17 // repository and thus no conventional logic gate is directly
in the clock path.
18
19 // The correctness of this module is based on the following
assumptions:
20 // 1. The select signal stays stable for a duration of at
least min(clks_i
21 // period)
22 // 2. Glitches on the select signal are shorter than
min(clks_i) - t_setup
23
24 // 3. During a transition from clock input a to clock input b,
both clocks have
25 // a stable period.
26
27 // A clock switching procedure from clock a to clock b has the
following timing behavior:
28 // 1. After at most NumSyncStages clock cycle of clock a, the
output clock is
29 // disabled with its next falling edge.
30 // 2. After clock cycle of clock a and another NumSyncStages
clock cycles of clock b, the output is
31 // enabled with the next rising edge of clock B.
32 // So in total, an upper bound for the worst case clock
switching delay is 2x
33 // NumSyncStages x max(clock_periods)
34 //
35 // The design has a parameter (ClockDuringReset) that allows
the clock
36 // multiplexer to propagate the selected clock even during re-
set assertion.
37 // However, during reset assertion the glitch filtering and
the synchronization
38 // registers are bypassed (since they are frozen in reset
state). Thus no glitch
39 // filtering is performed during reset. This is ok if the
async_sel_i signal
40 // stays constant during reset assertion. Once you deassert
the reset, regular
41 // glitch filtering and synchronization will kick in. However,
you must wait for
42 // at least 1x max(input clock periods) before changing the
async_sel_i after a
43 // reset to be sure the switch to regular operation has com-
pleted. During the
44 // transition from async_reset operation to regular operation
there will be a
45 // short phase where the clock is gated (similar to what hap-
pens when you switch
46 // from one clock to the other).
47 //
48 // IMPORTANT!!!
49 //
50 // All clock gating/logic within this design is performed by

```

```

dedicated clock
51 // logic tech cells. By default the common_cell library uses
the behavioral

52 // models in the `tech_cells_generic` repository. However, for
synthesis these
53 // cells need to be mapped to dedicated cells from your stand-
ard cell library,
54 // preferably ones that are designed for clock logic (they
have balanced rise
55 // and fall time). During synthesis you furthermore have to
properly set

56 // `dont_touch` or `size_only` attributes to prevent the logic
synthesizer from
57 // replacing those cells with regular logic gates which could
end up being
58 // glitchy!
59 //
60 //-----
61 // Copyright (C) 2013-2022 ETH Zurich, University of Bologna
62 // Copyright and related rights are licensed under the Solder-
pad Hardware

63 // License, Version 0.51 (the "License"); you may not use this
file except in
64 // compliance with the License. You may obtain a copy of the
License at
65 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
66 // or agreed to in writing, software, hardware and materials
distributed under
67 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
68 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
69 // specific language governing permissions and limitations un-
der the License.
70 //-----
71
72 `include "common_cells/registers.svh"
73
74 module cc_clk_mux_glitch_free #(
75     parameter int unsigned NumInputs = 2,
76     parameter int unsigned NumSyncStages = 2,
77     parameter bit ClockDuringReset = 1'b1, //< If 1, allow the
selected clock to
78                                     //propagate even dur-
ing reset
79                                     //assertion.
80     localparam int unsigned SelWidth = $clog2(NumInputs)
81 ) (
82     input logic [NumInputs-1:0] clks_i,
83     input logic test_clk_i,
84     input logic test_en_i,
85     input logic async_rstn_i,
86     input logic [SelWidth-1:0] async_sel_i,
87     output logic clk_o
88 );
89
90
91
92 if (NumInputs < 2)
93     $error("Num inputs must be parametrized to a value >=
2.");
94
95 // For each input, we generate an enable signal that enables
the clock

96 // propagation through an N-input clock OR gate. The crucial
and most critical
97 // part is to make sure that these clock enable signal
transitions are
98 // non-overlapping and have enough timing separation to pre-
vent any glitches

99 // on the clock output during the transition. We ensure this as
follows:
100 //
101 // 1. Decode the sel_i input to a onehot signal.
102 // 2. For each clock input, take the correspondi onehot sig-
nal. For each clock
103 // input we also have a correspondding output clock enable
signal that
104 // controls the corresponding clock's clk gate. We thus and-
gate the one-hot
105 // signal of the current clock with the inverse of every
other clocks enable
106 // signal. In other words, we only allow the propagation of
the onehot enable
107 // signal if the clock is currently disabled.
108 // 3. Filter any glitches on this and-gated signal by
passing it through a
109 // flip-flop clocked by the current clock and and-gating
both the output and
110 // the input. I.e. the output is only becomes active if the
signal stays
111 // stable HIGH for at least one clock period.

```

```

112 // 4. Synchronize this glitch-filtered signal into the cur-
rent clock domain
113 // with an M-stage synchronizer.
114 // 5. Use this synchronized signal as the enable signal for
a glitch-free
115 // clock gate.
116 // 7. Feed the output of the clock gate to an N-input clock-
AND gate.
117 // 8. Latch the gate enable signal with an active low latch
before using the
118 // signal as a gating signal for the other clock input's
onehot signal.
119
120 // Internal signals
121 logic [NumInputs-1:0] s_sel_onehot;
122 (*dont_touch*)
123 (*async_reg*)
124 logic [NumInputs-1:0][1:0] glitch_filter_arr_d,
glitch_filter_arr_q;
125 logic [NumInputs-1:0] s_gate_enable_unfiltered_async;
126 logic [NumInputs-1:0] s_glitch_filter_output_async;
127 logic [NumInputs-1:0] s_gate_enable_sync;
128 logic [NumInputs-1:0] s_gate_enable;
129 logic [NumInputs-1:0] clock_has_been_disabled_arr_q;
130 logic [NumInputs-1:0] s_gated_clock;
131 logic s_output_clock;
132
133 logic [NumInputs-1:0] s_reset_synced;
134 logic [NumInputs-1:0] async_reset_bypass_active_q;
135
136
137 // Onehot decoder
138 always_comb begin
139     s_sel_onehot = '0;
140     s_sel_onehot[async_sel_i] = 1'b1;
141 end
142
143 // Input stages
144 for (genvar i = 0; i < NumInputs; i++) begin : gen_in-
put_stages
145
146     // Slice the glitch_filter and clock_has_been_disabled ar-
rays
147     // to avoid Verilator MULTIDRIVEN warnings.
148     (*dont_touch*)
149     (*async_reg*)
150     logic [1:0] glitch_filter_d, glitch_filter_q;
151     logic clock_has_been_disabled_q;
152     assign glitch_filter_arr_d[i] = glitch_filter_d;
153     assign glitch_filter_arr_q[i] = glitch_filter_q;
154     assign clock_has_been_disabled_arr_q[i] =
clock_has_been_disabled_q;
155
156     // Synchronize the reset into each clock domain
157     cc_rstgen i_rstgen(
158         .clk_i ( clks_i[i] ),
159         .rst_ni ( async_rstn_i ),
160         .test_mode_i ( test_en_i ),
161         .rst_no ( s_reset_synced[i] ),
162         .init_no ( )
163     );
164
165     // Gate onehot signal with other clocks' output gate en-
able
166     always_comb begin
167         s_gate_enable_unfiltered_async[i] = 1'b1;
168         for (int j = 0; j < NumInputs; j++) begin
169             if (i==j) begin
170                 s_gate_enable_unfiltered_async[i] &= s_sel_one-
hot[j];
171             end else begin
172                 s_gate_enable_unfiltered_async[i] &=
clock_has_been_disabled_arr_q[j];
173             end
174         end
175     end
176     assign glitch_filter_d[0] = s_g-
ate_enable_unfiltered_async[i];
177     assign glitch_filter_d[1] = glitch_filter_q[0];
178
179     // Filter HIGH-pulse glitches
180     `FF(glitch_filter_q, glitch_filter_d, '0, clks_i[i], s_re-
set_synced[i])
181     assign s_glitch_filter_output_async[i] =
glitch_filter_q[1] &
182                                     glitch_filter_q[0] &
183                                     s_g-
ate_enable_unfiltered_async[i];
184
185     // Synchronize to current clock
186     tc_sync #(.Stages(NumSyncStages)) i_sync_en(
187         .clk_i ( clks_i[i] ),
188         .rst_ni ( s_reset_synced[i] ),
189         .serial_i ( s_glitch_filter_output_async[i] ),
190         .serial_o ( s_gate_enable_sync[i] )
191     );
192
193     // If the design is parametrized to propagate a clock dur-
ing asserted reset,
194     // we have to provide a bypass path that directly connects

```

```

the unfiltered

195 // gate enable signal to the clock gate for as long as the
reset is active.
196
197 if (ClockDuringReset) begin : gen_a-
sync_reset_clock_bypass_logic
198     always_ff @(posedge clks_i[i], negedge s_re-
set_synced[i]) begin
199         if (!s_reset_synced[i]) begin
200             async_reset_bypass_active_q[i] <= 1'b1;
201         end else begin
202             async_reset_bypass_active_q[i] <= 1'b0;
203         end
204     end
205
206     assign s_gate_enable[i] =
async_reset_bypass_active_q[i]?
207         s_g-
ate_enable_unfiltered_async[i]
208         : s_gate_enable_sync[i];
209 end else begin : gen_no_async_reset_bypass_logic
210     assign s_gate_enable[i] = s_gate_enable_sync[i];
211 end
212
213 // Gate the input clock with the synced enable signal
214 tc_clk_gating #(
215     .IS_FUNCTIONAL(1'b1)
216 ) i_clk_gate (
217     .clk_i      ( clks_i[i] ),
218     .en_i       ( s_gate_enable[i] ),
219     .test_en_i  ( 1'b0 ),
220     .clk_o      ( s_gated_clock[i] )
221 );
222
223 // Latch the enable signal with the next rising edge of
the input clock and
224 // feed the output back to the other stage's input. If we
were to directly

225 // use the clock gate enable signal to determine wether it
is save to enable
226 // another clock (i.e. the signal becomes low) we would
risk enabling the
227 // other clock to early. This is because the glitch free
clock gate will
228 // only really disable the clock with the next falling
edge. By delaying the
229 // enable signal one more cycle, we ensure that the clock
stays low for at
230 // least one clock period of the original clock input be-
fore any other clock
231 // even has the chance to become active.
232 `FF(clock_has_been_disabled_q, ~s_gate_enable[i], 1'b1,
clks_i[i], s_reset_synced[i])
233 end
234
235 // Output OR-gate. At this stage, we should be already sure
that the clocks
236 // are enabled/disabled at the proper time to prevent any
glitches from
237 // escaping.
238
239 cc_clk_or_tree #(NumInputs) i_clk_or_tree (
240     .clks_i(s_gated_clock),
241     .clk_o(s_output_clock)
242 );
243
244 // Mux between the regular muxed clock and the test_clk_i
used for DFT.
245 tc_clk_mux2 i_test_clk_mux(
246     .clk0_i(s_output_clock),
247     .clk1_i(test_clk_i),
248     .clk_sel_i(test_en_i),
249     .clk_o
250 );
251
252 endmodule
253
254 // Helper Module to generate an N-input clock OR-gate from a
tree of tc_clk_or2 cells.
255 module cc_clk_or_tree #(
256     parameter int unsigned NumInputs
257 ) (
258     input logic [NumInputs-1:0] clks_i,
259     output logic clk_o
260 );
261
262 if (NumInputs < 1) begin : gen_error
263     $error("Cannot parametrize clk_or with less than 1 input
but was %0d", NumInputs);
264 end else if (NumInputs == 1) begin : gen_leaf
265     assign clk_o = clks_i[0];
266 end else if (NumInputs == 2) begin : gen_leaf
267     tc_clk_or2 i_clk_or2 (
268         .clk0_i(clks_i[0]),
269         .clk1_i(clks_i[1]),
270         .clk_o
271 );
272 end else begin : gen_recursive
273     logic branch_a, branch_b;

```

```

274     cc_clk_or_tree #(NumInputs/2) i_or_branch_a (
275         .clks_i(clks_i[0+:NumInputs/2]),
276         .clk_o(branch_a)
277 );
278
279 cc_clk_or_tree #(NumInputs/2 + NumInputs%2) i_or_branch_b
(
280     .clks_i(clks_i[NumInputs-1:NumInputs/2]),
281     .clk_o(branch_b)
282 );
283
284 tc_clk_or2 i_clk_or2 (
285     .clk0_i(branch_a),
286     .clk1_i(branch_b),
287     .clk_o
288 );
289 end
290
291 endmodule

```

18. pulp-platform_common_cells/src/cc_counter.sv (43 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the License.
10
11 // Author: Florian Zaruba
12 // Description: Generic up/down counter
13
14 module cc_counter #(
15     parameter int unsigned Width = 4,
16     parameter bit StickyOverflow = 1'b0
17 ) (
18     input logic      clk_i, // Clock
19     input logic      rst_ni, // Asynchronous reset
20     active low
21     input logic      clr_i, // Synchronous clear
22     active high
23     input logic      en_i, // Enable the counter
24     input logic      load_i, // Load a new value
25     input logic      down_i, // Downcount, default is up
26     input logic [Width-1:0] d_i,
27     output logic [Width-1:0] q_o,
28     output logic      overflow_o
29 );
30
31 cc_delta_counter #(
32     .Width (Width),
33     .StickyOverflow (StickyOverflow)
34 ) i_counter (
35     .clk_i,
36     .rst_ni,
37     .clr_i,
38     .en_i,
39     .load_i,
40     .down_i,
41     .delta_i({{Width-1{1'b0}}, 1'b1}),
42     .d_i,
43     .q_o,
44     .overflow_o
45 );
46
47 endmodule

```

19. pulp-platform_common_cells/src/cc_credit_counter.sv (57 lines)

```

1 // Copyright 2020 ETH Zurich and University of Bologna.
2 // Solderpad Hardware License, Version 0.51, see LICENSE for
details.
3 // SPDX-License-Identifier: SHL-0.51
4
5 // Author: Fabian Schuiki <fschuiki@iis.ee.ethz.ch>
6 // Author: Paul Scheffler <paulsc@iis.ee.ethz.ch>
7
8 `include "common_cells/registers.svh"
9 `include "common_cells/assertions.svh"
10
11 module cc_credit_counter #(
12     parameter int unsigned NumCredits = 0,
13     /// Whether credit is full or empty on reset
14     parameter bit InitCreditEmpty = 1'b0,
15     /// Derived parameters *Do not override*
16     localparam int unsigned InitNumCredits = InitCreditEmpty ?
'0 : NumCredits,

```



```

17 localparam type credit_cnt_t = logic [$clog2(NumCredits):0]
18 ) (
19 input logic clk_i, // Clock
20 input logic rst_ni, // Asynchronous reset active low
21 input logic clr_i, // Synchronous clear active high; reinitializes credit, takes priority
22
23 output credit_cnt_t credit_o,
24
25 input logic credit_give_i,
26 input logic credit_take_i,
27
28 output logic credit_left_o,
29 output logic credit_crit_o, // Giving one more credit will fill the credits
30 output logic credit_full_o
31 );
32
33 credit_cnt_t credit_d, credit_q;
34 logic increment, decrement;
35
36 assign decrement = credit_take_i & ~credit_give_i;
37 assign increment = ~credit_take_i & credit_give_i;
38
39 always_comb begin
40 credit_d = credit_q;
41 if (decrement) credit_d = credit_q - 1;
42 else if (increment) credit_d = credit_q + 1;
43 end
44
45 `FFARNC(credit_q, credit_d, clr_i, InitNumCredits, clk_i, rst_ni)
46
47 assign credit_o = credit_q;
48 assign credit_left_o = (credit_q != '0);
49 assign credit_crit_o = (credit_q == NumCredits-1);
50 assign credit_full_o = (credit_q == NumCredits);
51
52 `ifndef COMMON_CELLS_ASSERTS_OFF
53 `ASSERT_NEVER(CreditUnderflow, credit_o == '0 && decrement)
54 `ASSERT_NEVER(CreditOverflow, credit_o == NumCredits && increment)
55 `endif
56
57 endmodule

```

20. pulp-platform_common_cells/src/cc_delta_counter.sv (70 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solderpad Hardware
3 // License, Version 0.51 (the "License"); you may not use this file except in
4 // compliance with the License. You may obtain a copy of the License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by applicable law
6 // or agreed to in writing, software, hardware and materials distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the License for the
9 // specific language governing permissions and limitations under the License.
10
11 // Up/down counter with variable delta
12
13 `include "common_cells/registers.svh"
14
15 module cc_delta_counter #(
16     parameter int unsigned Width = 4,
17     parameter bit StickyOverflow = 1'b0
18 ) (
19     input logic clk_i, // Clock
20     input logic rst_ni, // Asynchronous reset
21     input logic clr_i, // Synchronous clear
22     input logic en_i, // Enable the counter
23     input logic load_i, // Load a new value
24     input logic down_i, // Downcount, default is up
25     input logic [Width-1:0] delta_i,
26     input logic [Width-1:0] d_i,
27     output logic [Width-1:0] q_o,
28     output logic overflow_o
29 );
30 logic [Width:0] counter_q, counter_d;
31 if (StickyOverflow) begin : gen_sticky_overflow
32 logic overflow_d, overflow_q;
33 logic overflow_clr;
34
35 assign overflow_clr = clr_i || load_i;
36 `FFARNC(overflow_q, overflow_d, overflow_clr, 1'b0, clk_i, rst_ni)
37

```

```

38 always_comb begin
39 overflow_d = overflow_q;
40 if (!overflow_q && en_i) begin
41 if (down_i) begin
42 overflow_d = delta_i >
43 counter_q[Width-1:0];
44 end else begin
45 overflow_d = counter_q[Width-1:0] >
46 ({Width[1'b1]} - delta_i);
47 end
48 end
49 assign overflow_o = overflow_q;
50 end else begin : gen_transient_overflow
51 // counter overflowed if the MSB is set
52 assign overflow_o = counter_q[Width];
53 end
54 assign q_o = counter_q[Width-1:0];
55
56 always_comb begin
57 counter_d = counter_q;
58
59 if (load_i) begin
60 counter_d = {1'b0, d_i};
61 end else if (en_i) begin
62 if (down_i) begin
63 counter_d = counter_q - delta_i;
64 end else begin
65 counter_d = counter_q + delta_i;
66 end
67 end
68
69 `FFARNC(counter_q, counter_d, clr_i, '0, clk_i, rst_ni)
70 endmodule

```

21. pulp-platform_common_cells/src/cc_ecc_decode.sv (129 lines)

```

1 // Copyright 2020 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solderpad Hardware
3 // License, Version 0.51 (the "License"); you may not use this file except in
4 // compliance with the License. You may obtain a copy of the License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by applicable law
6 // or agreed to in writing, software, hardware and materials distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the License for the
9 // specific language governing permissions and limitations under the License.
10
11 // Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
12
13 /// # ECC Decoder
14
15 /// Implements SECDED (Single Error Correction, Double Error Detection) Hamming Code
16 /// with extended parity bit [1].
17 /// The module receives a data word including parity bit and decodes it according to the
18 /// number of data and parity bit.
19
20 /// 1. If no error has been detected, the syndrome will be zero and all flags will be zero.
21 /// 2. If a single error has been detected, the syndrome is non-zero, and `single_error_o` will be asserted. The output word contains the corrected data.
22 /// 3. If the parity bit contained an error, the module will assert `parity_error_o`.
23 /// 4. In case of a double fault the syndrome is non-zero, `double_error_o` will be asserted.
24 /// All other status flags will be de-asserted.
25
26 /// [1] https://en.wikipedia.org/wiki/Hamming_code
27
28 module cc_ecc_decode import cc_pkg::*; #(
29     parameter int unsigned DataWidth = 64,
30     parameter int unsigned DataWidthParity = 64,
31     parameter int unsigned DataWidthCode = 64,
32     parameter int unsigned DataWidthParityCode = 64,
33     localparam type data_t = logic [DataWidth-1:0],
34     localparam type parity_t = logic [ecc_get_parity_width(DataWidth)-1:0],
35     localparam type code_word_t = logic [ecc_get_cw_width(DataWidth)-1:0],
36     localparam type encoded_data_t = struct packed {
37         logic parity;
38         code_word_t code_word;
39     }
40 ) (
41     /// Encoded data in
42     input encoded_data_t data_i,
43     /// Corrected data out
44     output data_t data_o,

```

```

45 /// Error syndrome indicates the erroneous bit position
46 output parity_t syndrome_o,
47 /// A single error occurred
48 output logic single_error_o,
49 /// Error received in parity bit (MSB)
50 output logic parity_error_o,
51 /// A double error occurred
52 output logic double_error_o
53 );
54
55 logic parity;
56 data_t data_wo_parity;
57 parity_t syndrome;
58 logic syndrome_not_zero;
59 code_word_t correct_data;
60
61 // Check parity bit. 0 = parity equal, 1 = different parity
62 assign parity = data_i.parity ^ (^data_i.code_word);
63
64 ///! | 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
65 ///! | p1 p2 d1 p4 d2 d3 d4 p8 d5 d6 d7 d8 d9 d10 d11
66 ///! |-----|
67 ///! p1 | x x x x x x x x x x
68 ///! p2 | x x x x x x x x x x
69 ///! p4 | x x x x x x x x x x
70 ///! p8 | x x x x x x x x x x
71
72 ///! 1. Parity bit 1 covers all bit positions which have the
least significant bit
73 ///! set: bit 1 (the parity bit itself), 3, 5, 7, 9, etc.
74
75 ///! 2. Parity bit 2 covers all bit positions which have the
second least
76 ///! significant bit set: bit 2 (the parity bit itself),
3, 6, 7, 10, 11, etc.
77
78 ///! 3. Parity bit 4 covers all bit positions which have the
third least
79 ///! significant bit set: bits 4-7, 12-15, 20-23, etc.
80
81 ///! 4. Parity bit 8 covers all bit positions which have the
fourth least
82 ///! significant bit set: bits 8-15, 24-31, 40-47, etc.
83
84 ///! 5. In general each parity bit covers all bits where the
bitwise AND of the
85 ///! parity position and the bit position is non-zero.
86 always_comb begin : calculate_syndrome
87 syndrome = 0;
88 for (int unsigned i = 0; i < unsigned'($bits(parity_t));
i++) begin
89 for (int unsigned j = 0; j < un-
signed'($bits(code_word_t)); j++) begin
90 if ((!(unsigned'(2**i) & (j + 1))) syndrome[i] = syn-
drome[i] ^ data_i.code_word[j];
91 end
92 end
93
94 assign syndrome_not_zero = |syndrome;
95 assign syndrome_o = syndrome;
96
97 // correct the data word if the syndrome is non-zero
98 always_comb begin
99 correct_data = data_i.code_word;
100 if (syndrome_not_zero) begin
101 correct_data[syndrome - 1] = ~data_i.code_word[syndrome
- 1];
102 end
103 end
104
105 ///! Syndrome | Overall Parity (MSB) | Error Type | Notes
106 ///!
107 -----
108 ///! 0 | 0 | No Error |
109 ///! !=0 | 1 | Single Error | Cor-
rectable. Syndrome holds incorrect bit position.
110
111 ///! 0 | 1 | Parity Error | Over-
all parity, MSB is in error and can be corrected.
112
113 ///! !=0 | 0 | Double Error | Not
correctable.
114
115 assign single_error_o = parity & syndrome_not_zero;
116 assign parity_error_o = parity & ~syndrome_not_zero;
117 assign double_error_o = ~parity & syndrome_not_zero;
118
119 // Extract data vector
120 always_comb begin
121 automatic int unsigned idx; // bit index
122 data_wo_parity = '0;
123 idx = 0;
124
125 for (int unsigned i = 1; i < unsigned'($bits(code_word_t)) +
1; i++) begin
126 // if i is a power of two we are indexing a parity bit
127 if (unsigned'(2**$clog2(i)) != i) begin
128 data_wo_parity[idx] = correct_data[i - 1];
129 idx++;
130 end
131 end
132 end

```

```

125 end
126
127 assign data_o = data_wo_parity;
128
129 endmodule

```

22. pulp-platform_common_cells/src/cc_ecc_encode.sv (78 lines)

```

1 // Copyright 2020 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this license is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations under
the License.
10 //
11 // Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
12 // # ECC Encoder
13 //
14 // Implements SECDED (Single Error Correction, Double Error
Detection) Hamming Code
15 // with extended parity bit [1].
16
17 /// The module receives a data word and encodes it using above
mentioned error
18 /// detection and correction code. The corresponding decode
module
19 /// can be found in `cc_ecc_decode.sv`
20 /// [1] https://en.wikipedia.org/wiki/Hamming\_code
21
22 module cc_ecc_encode import cc_pkg::*; #(
23 /// Data width of unencoded word.
24 parameter int unsigned DataWidth = 64,
25 // Do not change
26 localparam type data_t = logic [DataWidth-1:0],
27 localparam type parity_t = logic [ecc_get_par-
ity_width(DataWidth)-1:0],
28 localparam type code_word_t = logic
[ecc_get_cw_width(DataWidth)-1:0],
29 localparam type encoded_data_t = struct packed {
30 logic parity;
31 code_word_t code_word;
32 }
33 ) (
34 /// Unencoded data in
35 input data_t data_i,
36 /// Encoded data out
37 output encoded_data_t data_o
38 );
39
40 parity_t parity_code_word;
41 code_word_t data, codeword;
42
43 // Expand incoming data to codeword width
44 always_comb begin : expand_data
45 automatic int unsigned idx;
46 data = '0;
47 idx = 0;
48
49 for (int unsigned i = 1; i < unsigned'($bits(code_word_t)) +
1; i++) begin
50 // if it is not a power of two word it is a normal data
51 if (unsigned'(2**$clog2(i)) != i) begin
52 data[i - 1] = data_i[idx];
53 idx++;
54 end
55 end
56
57 // calculate code word
58 always_comb begin : calculate_syndrome
59 parity_code_word = 0;
60 for (int unsigned i = 0; i < unsigned'($bits(parity_t));
i++) begin
61 for (int unsigned j = 1; j < un-
signed'($bits(code_word_t)) + 1; j++) begin
62 if ((!(unsigned'(2**i) & j)) parity_code_word[i] = par-
ity_code_word[i] ^ data[j - 1];
63 end
64 end
65 end
66
67 // fuse the final codeword
68 always_comb begin : generate_codeword
69 codeword = data;

```



```

70     for (int unsigned i = 0; i < unsigned'($bits(parity_t)); i+
+) begin
71         codeword[2**i-1] = parity_code_word[i];
72     end
73 end
74
75 assign data_o.code_word = codeword;
76 assign data_o.parity = ^codeword;
77
78 endmodule

```

23. pulp-platform_common_cells/src/cc_edge_detect.sv (31 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 // Author: Florian Zaruba
12 // Description: Edge detector, clock needs to oversample for
  proper edge detection
13
14 module cc_edge_detect (
15     input logic clk_i, // Clock
16     input logic rst_ni, // Asynchronous reset active low
17     input logic d_i, // Data stream in
18     output logic re_o, // Rising edge detected
19     output logic fe_o // Falling edge detected
20 );
21
22     cc_sync_wedge i_sync_wedge (
23         .clk_i ( clk_i ),
24         .rst_ni ( rst_ni ),
25         .serial_i ( d_i ),
26         .r_edge_o ( re_o ),
27         .f_edge_o ( fe_o ),
28         .serial_o ( )
29     );
30
31 endmodule

```

24. pulp-platform_common_cells/src/cc_edge_propagator.sv (32 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 // Antonio Pullini <pullinia@iis.ee.ethz.ch>
12
13 module cc_edge_propagator (
14     input logic clk_tx_i,
15     input logic rst_tx_ni,
16     input logic edge_i,
17     input logic clk_rx_i,
18     input logic rst_rx_ni,
19     output logic edge_o
20 );
21
22     cc_edge_propagator_ack i_edge_propagator_ack (
23         .clk_tx_i,
24         .rst_tx_ni,
25         .edge_i,
26         .ack_tx_o (* unused *),
27         .clk_rx_i,
28         .rst_rx_ni,
29         .edge_o
30     );

```

```

31
32 endmodule

```

25. pulp-platform_common_cells/src/cc_edge_propagator_ack.sv (52 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 // Antonio Pullini <pullinia@iis.ee.ethz.ch>
12
13 module cc_edge_propagator_ack (
14     input logic clk_tx_i,
15     input logic rst_tx_ni,
16     input logic edge_i,
17     output logic ack_tx_o,
18     input logic clk_rx_i,
19     input logic rst_rx_ni,
20     output logic edge_o
21 );
22
23     logic [1:0] sync_a;
24     logic sync_b;
25
26     logic r_input_reg;
27     logic s_input_reg_next;
28
29     assign ack_tx_o = sync_a[0];
30
31     assign s_input_reg_next = edge_i | (r_input_reg &
  ~sync_a[0]);
32
33     always @(negedge rst_tx_ni or posedge clk_tx_i) begin
34         if (~rst_tx_ni) begin
35             r_input_reg <= 1'b0;
36             sync_a <= 2'b00;
37         end else begin
38             r_input_reg <= s_input_reg_next;
39             sync_a <= {sync_b, sync_a[1]};
40         end
41     end
42
43     cc_sync_wedge u_sync_clkb (
44         .clk_i ( clk_rx_i ),
45         .rst_ni ( rst_rx_ni ),
46         .serial_i ( r_input_reg ),
47         .r_edge_o ( edge_o ),
48         .f_edge_o ( ),
49         .serial_o ( sync_b )
50     );
51
52 endmodule

```

26. pulp-platform_common_cells/src/cc_edge_propagator_rx.sv (30 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 // Antonio Pullini <pullinia@iis.ee.ethz.ch>
12
13 module cc_edge_propagator_rx (
14     input logic clk_i,
15     input logic rst_ni,
16     input logic valid_i,
17     output logic ack_o,
18     output logic valid_o

```

```

19 );
20
21 cc_sync_wedge i_sync_clkb (
22     .clk_i   ( clk_i   ),
23     .rst_ni  ( rst_ni  ),
24     .serial_i ( valid_i ),
25     .r_edge_o ( valid_o ),
26     .f_edge_o (         ),
27     .serial_o ( ack_o   )
28 );
29
30 endmodule

```

27. pulp-platform_common_cells/src/cc_edge_propagator_tx.sv (40 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 // Antonio Pullini <pullinia@iis.ee.ethz.ch>
12
13 module cc_edge_propagator_tx (
14     input logic clk_i,
15     input logic rst_ni,
16     input logic valid_i,
17     input logic ack_i,
18     output logic valid_o
19 );
20
21     logic [1:0] sync_a;
22
23     logic r_input_reg;
24     logic s_input_reg_next;
25
26     assign s_input_reg_next = valid_i | (r_input_reg &
  ~sync_a[0]);
27
28     always @(negedge rst_ni or posedge clk_i) begin
29         if (~rst_ni) begin
30             r_input_reg <= 1'b0;
31             sync_a <= 2'b00;
32         end else begin
33             r_input_reg <= s_input_reg_next;
34             sync_a <= {ack_i, sync_a[1]};
35         end
36     end
37
38     assign valid_o = r_input_reg;
39
40 endmodule

```

28. pulp-platform_common_cells/src/cc_exp_backoff.sv (90 lines)

```

1 // Copyright 2019 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10 //
11 // Author: Michael Schaffner <schaffner@iis.ee.ethz.ch>, ETH
  Zurich
12 // Date: 10.04.2019

```

```

13 // Description: exponential backoff counter with randomiza-
  tion.
14 //
15 // For each failed trial (set_cnt_i pulsed), this unit expo-
  nentially increases the
16 // (average) backoff time by masking an LFSR with a shifted
  mask in order to
17 // create the backoff counter initial value.
18 //
19 // The shift register mask and the counter value are both re-
  set to '0 in case of
20 // a successful trial (clr_cnt_i).
21 //
22
23 `include "common_cells/assertions.svh"
24 `include "common_cells/registers.svh"
25
26 module cc_exp_backoff #(
27     /// Seed for 16bit LFSR
28     parameter int unsigned Seed = 'hffff,
29     /// 2*MaxExp-1 determines the maximum range from which ran-
  dom wait counts are drawn
30     parameter int unsigned MaxExp = 16
31 ) (
32     /// Clock
33     input logic clk_i,
34     /// Asynchronous reset active low
35     input logic rst_ni,
36     /// Synchronous clear active high
37     input logic clr_i,
38     /// Sets the backoff counter (pulse) -> use when trial did
  not succeed
39     input logic set_cnt_i,
40     /// Clears the backoff counter (pulse) -> use when trial
  succeeded
41     input logic clr_cnt_i,
42     /// Indicates whether the backoff counter is equal to zero
  and a new trial can be launched
43     output logic is_zero_o
44 );
45
46 // leave this constant
47 localparam int unsigned WIDTH = 16;
48
49 logic [WIDTH-1:0] lfsr_d, lfsr_q, cnt_d, cnt_q, mask_d,
  mask_q;
50
51 logic lfsr;
52
53 // generate random wait counts
54 // note: we use a flipped lfsr here to
55 // avoid strange correlation effects between
56 // the (left-shifted) mask and the lfsr
57 assign lfsr = lfsr_q[15-15] ^
58             lfsr_q[15-13] ^
59             lfsr_q[15-12] ^
60             lfsr_q[15-10];
61
62 assign lfsr_d = (set_cnt_i) ? {lfsr, lfsr_q[$high(lfsr_q):
  1]} :
63             lfsr_q;
64
65 // mask the wait counts with exponentially increasing mask
  (shift reg)
66 assign mask_d = (clr_cnt_i) ?
67             :
68             (set_cnt_i) ? {(WIDTH-MaxExp)
  {1'b0}}, mask_q[MaxExp-2:0], 1'b1} :
69             mask_q;
70
71 assign cnt_d = (clr_cnt_i) ? '0 :
72             (set_cnt_i) ? (mask_q & lfsr_q) :
73             (!is_zero_o) ? cnt_q - 1'b1 : '0;
74
75 assign is_zero_o = (cnt_q=='0);
76
77 `FFARNC(lfsr_q, lfsr_d, clr_i, WIDTH'(Seed), clk_i, rst_ni)
78 `FFARNC(mask_q, mask_d, clr_i, '0, clk_i, rst_ni)
79 `FFARNC(cnt_q, cnt_d, clr_i, '0, clk_i, rst_ni)
80
81 // assertions
82
83 `ifndef COMMON_CELLS_ASSERTS_OFF
84 // assert wrong parameterizations
85 `ASSERT_INIT(max_exp_0, MaxExp>0, "MaxExp must be greater
  than 0")
86 `ASSERT_INIT(max_exp_gt_16, MaxExp<=16, "MaxExp cannot be
  greater than 16")
87 `ASSERT_INIT(seed_0, Seed>0, "Zero seed is not allowed for
  LFSR")
88 `endif
89
90 endmodule // cc_exp_backoff

```

AXI4 상호연결

What this is. AXI4 crossbars, multiplexers, demultiplexers, cuts and error slaves.

Why it exists. Because **AXI is the de facto SoC interconnect**, and because its ordering rules — ordering guaranteed only within an ID — shape every verification component that touches it.

Undergraduate review · EECS concepts. Channel-based protocols; out-of-order completion; transaction identifiers; deadlock avoidance in interconnect.

Graduate · Practice note. A scoreboard for AXI needs **one queue per ID**. A single queue both reports false failures on legal reordering and accepts genuine intra-ID violations.

13 files • 4,754 lines

API — axi_atop_filter.sv

Table 1547. pulp-platform_axi/src/axi_atop_filter.sv — module / interface / package (2개)

이름	종류
axi_atop_filter	module
axi_atop_filter_intf	module

Table 1548. pulp-platform_axi/src/axi_atop_filter.sv — parameter (9개)

파라미터	형/폭	기본값
AxiIdWidth	int unsigned	0
AxiMaxWriteTxns	int unsigned	0
axi_req_t		logic
axi_resp_t		logic
AXI_ID_WIDTH	int unsigned	0
AXI_ADDR_WIDTH	int unsigned	0
AXI_DATA_WIDTH	int unsigned	0
AXI_USER_WIDTH	int unsigned	0
AXI_MAX_WRITE_TXNS	int unsigned	0

Table 1549. pulp-platform_axi/src/axi_atop_filter.sv — localparam (1개)

상수	형/폭	값
COUNTER_WIDTH	int unsigned	(AxiMaxWriteTxns == 1) ? 2 : \$clog2(AxiMaxWriteTxns+1)

Table 1550. pulp-platform_axi/src/axi_atop_filter.sv — 포트 (8개)

포트	방향	형/폭
clk_i	input	logic

포트	방향	형/폭
rst_ni	input	logic
slv_req_i	input	axi_req_t
slv_resp_o	output	axi_resp_t
mst_req_o	output	axi_req_t
mst_resp_i	input	axi_resp_t
clk_i	input	logic
rst_ni	input	logic

Table 1551. pulp-platform_axi/src/axi_atop_filter.sv — typedef / using (8개)

새 이름	원래 형
r_state_e	enum logic [1:0] { R_RESET, R_FEEDTHROUGH, INJECT_R, R_HOLD }
id_t	logic [AxIdWidth-1:0]
len_t	logic [7:0]
id_t	logic [AXI_ID_WIDTH-1:0]
addr_t	logic [AXI_ADDR_WIDTH-1:0]
data_t	logic [AXI_DATA_WIDTH-1:0]
strb_t	logic [AXI_DATA_WIDTH/8-1:0]
user_t	logic [AXI_USER_WIDTH-1:0]

API — axi_burst_splitter.sv**Table 1552. pulp-platform_axi/src/axi_burst_splitter.sv — module / interface / package (17개)**

이름	종류
axi_burst_splitter	module

Table 1553. pulp-platform_axi/src/axi_burst_splitter.sv — parameter (10개)

파라미터	형/폭	기본값
MaxReadTxns	int unsigned	32'd0
MaxWriteTxns	int unsigned	32'd0
FullBW	bit	0
AddrWidth	int unsigned	32'd0
DataWidth	int unsigned	32'd0
IdWidth	int unsigned	32'd0
AxiLookBits	int unsigned	IdWidth
UserWidth	int unsigned	32'd0
axi_req_t		logic
axi_resp_t		logic

Table 1554. pulp-platform_axi/src/axi_burst_splitter.sv — 포트 (6개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
slv_req_i	input	axi_req_t
slv_resp_o	output	axi_resp_t
mst_req_o	output	axi_req_t
mst_resp_i	input	axi_resp_t

Table 1555. pulp-platform_axi/src/axi_burst_splitter.sv — typedef / using (5개)

새 이름	원래 형
addr_t	logic [AddrWidth-1:0]
data_t	logic [DataWidth-1:0]
strb_t	logic [DataWidth/8-1:0]
id_t	logic [IdWidth-1:0]
user_t	logic [UserWidth-1:0]

API — axi_burst_splitter_gran.sv**Table 1556. pulp-platform_axi/src/axi_burst_splitter_gran.sv — module / interface / package (3개)**

이름	종류
axi_burst_splitter_gran	module
axi_burst_splitter_gran_ax_chan	module
axi_burst_splitter_gran_counters	module

Table 1557. pulp-platform_axi/src/axi_burst_splitter_gran.sv — parameter (29개)

파라미터	형/폭	기본값
MaxReadTxns	int unsigned	32'd0
MaxWriteTxns	int unsigned	32'd0
FullBW	bit	1'b0
CutPath	bit	1'b0
DisableChecks	bit	1'b0
AddrWidth	int unsigned	32'd0
DataWidth	int unsigned	32'd0
IdWidth	int unsigned	32'd0
AxiLookBits	int unsigned	IdWidth
UserWidth	int unsigned	32'd0
axi_req_t		logic

파라미터	형/폭	기본값
axi_resp_t		logic
axi_aw_chan_t		logic
axi_w_chan_t		logic
axi_b_chan_t		logic
axi_ar_chan_t		logic
axi_r_chan_t		logic
chan_t		logic
IdWidth	int unsigned	32'd0
MaxTxns	int unsigned	32'd0
CutPath	bit	1'b0
FullBW	bit	1'b0
id_t		logic[IdWidth-1:0]
MaxTxns	int unsigned	32'd0
IdWidth	int unsigned	32'd0
CutPath	bit	1'b0
FullBW	bit	1'b0
id_t		logic [IdWidth-1:0]
cnt_t		logic [axi_pkg::LenWidth:0]

Table 1558. pulp-platform_axi/src/axi_burst_splitter_gran.sv — localparam (2개)

상수	형/폭	값
MaxTxns	int unsigned	(MaxReadTxns > MaxWriteTxns) ? MaxReadTxns : MaxWriteTxns
CntIdxWidth	int unsigned	(MaxTxns > 1) ? \$clog2(MaxTxns) : 32'd1

Table 1559. pulp-platform_axi/src/axi_burst_splitter_gran.sv — 포트 (37개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
len_limit_i	input	axi_pkg::len_t
slv_req_i	input	axi_req_t
slv_resp_o	output	axi_resp_t
mst_req_o	output	axi_req_t
mst_resp_i	input	axi_resp_t
clk_i	input	logic
rst_ni	input	logic
len_limit_i	input	axi_pkg::len_t

포트	방향	형/폭
ax_i	input	chan_t
ax_valid_i	input	logic
ax_ready_o	output	logic
ax_o	output	chan_t
ax_valid_o	output	logic
ax_ready_i	input	logic
cnt_id_i	input	id_t
cnt_len_o	output	axi_pkg::len_t
cnt_set_err_i	input	logic
cnt_err_o	output	logic
cnt_dec_i	input	logic
cnt_req_i	input	logic
cnt_gnt_o	output	logic
clk_i	input	logic
rst_ni	input	logic
alloc_id_i	input	id_t
alloc_len_i	input	axi_pkg::len_t
alloc_req_i	input	logic
alloc_gnt_o	output	logic
cnt_id_i	input	id_t
cnt_len_o	output	axi_pkg::len_t
cnt_set_err_i	input	logic
cnt_err_o	output	logic
cnt_dec_i	input	logic
cnt_delta_i	input	cnt_t
cnt_req_i	input	logic
cnt_gnt_o	output	logic

Table 1560. `pulp-platform_axi/src/axi_burst_splitter_gran.sv` — typedef / using (37개)

새 이름	원래 형
cnt_id_t	logic[IdWidth-1:0]
num_beats_t	logic[axi_pkg::LenWidth:0]
cnt_idx_t	logic [CntIdxWidth-1:0]

API — `axi_burst_unwrap.sv`Table 1561. `pulp-platform_axi/src/axi_burst_unwrap.sv` — module / interface / package (37개)

이름	종류
axi_burst_unwrap	module
axi_burst_unwrap_ax_chan	module
axi_burst_counters	module

Table 1562. pulp-platform_axi/src/axi_burst_unwrap.sv — parameter (18개)

파라미터	형/폭	기본값
MaxReadTxns	int unsigned	32'd0
MaxWriteTxns	int unsigned	32'd0
AddrWidth	int unsigned	32'd0
DataWidth	int unsigned	32'd0
IdWidth	int unsigned	32'd0
AxiLookBits	int unsigned	IdWidth
UserWidth	int unsigned	32'd0
axi_req_t		logic
axi_resp_t		logic
AwChan	bit	0
chan_t		logic
AddrWidth	int unsigned	0
IdWidth	int unsigned	0
MaxTxns	int unsigned	0
id_t		logic[IdWidth-1:0]
MaxTxns	int unsigned	0
IdWidth	int unsigned	0
id_t		logic [IdWidth-1:0]

Table 1563. pulp-platform_axi/src/axi_burst_unwrap.sv — localparam (2개)

상수	형/폭	값
MaxTxns	int unsigned	(MaxReadTxns > MaxWriteTxns) ? MaxReadTxns : MaxWriteTxns
CntIdxWidth	int unsigned	(MaxTxns > 1) ? \$clog2(MaxTxns) : 32'd1

Table 1564. pulp-platform_axi/src/axi_burst_unwrap.sv — 포트 (34개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
slv_req_i	input	axi_req_t
slv_resp_o	output	axi_resp_t

포트	방향	형/폭
mst_req_o	output	axi_req_t
mst_resp_i	input	axi_resp_t
clk_i	input	logic
rst_ni	input	logic
ax_i	input	chan_t
ax_valid_i	input	logic
ax_ready_o	output	logic
ax_o	output	chan_t
ax_valid_o	output	logic
ax_ready_i	input	logic
cnt_id_i	input	id_t
cnt_len_o	output	axi_pkg::len_t [1:0]
cnt_set_err_i	input	logic
cnt_err_o	output	logic
cnt_dec_i	input	logic [1:0]
cnt_req_i	input	logic [1:0]
cnt_gnt_o	output	logic [1:0]
clk_i	input	logic
rst_ni	input	logic
alloc_id_i	input	id_t
alloc_len_i	input	axi_pkg::len_t
alloc_req_i	input	logic
alloc_gnt_o	output	logic
cnt_id_i	input	id_t
cnt_len_o	output	axi_pkg::len_t
cnt_set_err_i	input	logic
cnt_err_o	output	logic
cnt_dec_i	input	logic
cnt_req_i	input	logic
cnt_gnt_o	output	logic

Table 1565. pulp-platform_axi/src/axi_burst_unwrap.sv — typedef / using (8개)

새 이름	원래 형
addr_t	logic [AddrWidth-1:0]
data_t	logic [DataWidth-1:0]

새 이름	원래 형
id_t	logic [IdWidth-1:0]
strb_t	logic [DataWidth/8-1:0]
user_t	logic [UserWidth-1:0]
cnt_id_t	logic[IdWidth-1:0]
cnt_idx_t	logic [CntIdxWidth-1:0]
cnt_t	logic [\$bits(axi_pkg::len_t):0]

API — axi_bus_compare.sv

Table 1566. pulp-platform_axi/src/axi_bus_compare.sv — module / interface / package (1개)

이름	종류
axi_bus_compare	module

Table 1567. pulp-platform_axi/src/axi_bus_compare.sv — parameter (12개)

파라미터	형/폭	기본값
AxiIdWidth	int unsigned	32'd0
FifoDepth	int unsigned	32'd0
UseSize	bit	1'b0
DataWidth	int unsigned	32'd8
axi_aw_chan_t		logic
axi_w_chan_t		logic
axi_b_chan_t		logic
axi_ar_chan_t		logic
axi_r_chan_t		logic
axi_req_t		logic
axi_rsp_t		logic
id_t		logic [2**AxiIdWidth-1:0]

Table 1568. pulp-platform_axi/src/axi_bus_compare.sv — 포트 (17개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
axi_a_req_i	input	axi_req_t
axi_a_rsp_o	output	axi_rsp_t
axi_a_req_o	output	axi_req_t
axi_a_rsp_i	input	axi_rsp_t
axi_b_req_i	input	axi_req_t

포트	방향	형/폭
axi_b_rsp_o	output	axi_rsp_t
axi_b_req_o	output	axi_req_t
axi_b_rsp_i	input	axi_rsp_t
aw_mismatch_o	output	id_t
w_mismatch_o	output	logic
b_mismatch_o	output	id_t
ar_mismatch_o	output	id_t
r_mismatch_o	output	id_t
mismatch_o	output	logic
busy_o	output	logic

API — axi_cdc.sv

Table 1569. pulp-platform_axi/src/axi_cdc.sv — module / interface / package (3개)

이름	종류
axi_cdc	module
axi_cdc_intf	module
axi_lite_cdc_intf	module

Table 1570. pulp-platform_axi/src/axi_cdc.sv — parameter (19개)

파라미터	형/폭	기본값
aw_chan_t		logic
w_chan_t		logic
b_chan_t		logic
ar_chan_t		logic
r_chan_t		logic
axi_req_t		logic
axi_resp_t		logic
LogDepth	int unsigned	1
SyncStages	int unsigned	2
AXI_ID_WIDTH	int unsigned	0
AXI_ADDR_WIDTH	int unsigned	0
AXI_DATA_WIDTH	int unsigned	0
AXI_USER_WIDTH	int unsigned	0
LOG_DEPTH	int unsigned	1
SYNC_STAGES	int unsigned	2
AXI_ADDR_WIDTH	int unsigned	0

파라미터	형/폭	기본값
AXI_DATA_WIDTH	int unsigned	0
LOG_DEPTH	int unsigned	1
SYNC_STAGES	int unsigned	2

Table 1571. pulp-platform_axi/src/axi_cdc.sv — 포트 (16개)

포트	방향	형/폭
src_clk_i	input	logic
src_rst_ni	input	logic
src_req_i	input	axi_req_t
src_resp_o	output	axi_resp_t
dst_clk_i	input	logic
dst_rst_ni	input	logic
dst_req_o	output	axi_req_t
dst_resp_i	input	axi_resp_t
src_clk_i	input	logic
src_rst_ni	input	logic
dst_clk_i	input	logic
dst_rst_ni	input	logic
src_clk_i	input	logic
src_rst_ni	input	logic
dst_clk_i	input	logic
dst_rst_ni	input	logic

Table 1572. pulp-platform_axi/src/axi_cdc.sv — typedef / using (8개)

새 이름	원래 형
id_t	logic [AXI_ID_WIDTH-1:0]
addr_t	logic [AXI_ADDR_WIDTH-1:0]
data_t	logic [AXI_DATA_WIDTH-1:0]
strb_t	logic [AXI_DATA_WIDTH/8-1:0]
user_t	logic [AXI_USER_WIDTH-1:0]
addr_t	logic [AXI_ADDR_WIDTH-1:0]
data_t	logic [AXI_DATA_WIDTH-1:0]
strb_t	logic [AXI_DATA_WIDTH/8-1:0]

API — axi_cdc_dst.sv**Table 1573. pulp-platform_axi/src/axi_cdc_dst.sv — module / interface / package (3개)**

이름	종류
axi_cdc_dst	module
axi_cdc_dst_intf	module
axi_lite_cdc_dst_intf	module

Table 1574. pulp-platform_axi/src/axi_cdc_dst.sv — parameter (19개)

파라미터	형/폭	기본값
LogDepth	int unsigned	1
SyncStages	int unsigned	2
aw_chan_t		logic
w_chan_t		logic
b_chan_t		logic
ar_chan_t		logic
r_chan_t		logic
axi_req_t		logic
axi_resp_t		logic
AXI_ID_WIDTH	int unsigned	0
AXI_ADDR_WIDTH	int unsigned	0
AXI_DATA_WIDTH	int unsigned	0
AXI_USER_WIDTH	int unsigned	0
LOG_DEPTH	int unsigned	1
SYNC_STAGES	int unsigned	2
AXI_ADDR_WIDTH	int unsigned	0
AXI_DATA_WIDTH	int unsigned	0
LOG_DEPTH	int unsigned	1
SYNC_STAGES	int unsigned	2

Table 1575. pulp-platform_axi/src/axi_cdc_dst.sv — 포트 (23개)

포트	방향	형/폭
async_data_slave_aw_data_i	input	aw_chan_t [2**LogDepth-1:0]
async_data_slave_aw_wptr_i	input	logic [LogDepth:0]
async_data_slave_aw_rptr_o	output	logic [LogDepth:0]
async_data_slave_w_data_i	input	w_chan_t [2**LogDepth-1:0]
async_data_slave_w_wptr_i	input	logic [LogDepth:0]
async_data_slave_w_rptr_o	output	logic [LogDepth:0]
async_data_slave_b_data_o	output	b_chan_t [2**LogDepth-1:0]
async_data_slave_b_wptr_o	output	logic [LogDepth:0]

포트	방향	형/폭
async_data_slave_b_rptr_i	input	logic [LogDepth:0]
async_data_slave_ar_data_i	input	ar_chan_t [2**LogDepth-1:0]
async_data_slave_ar_wptr_i	input	logic [LogDepth:0]
async_data_slave_ar_rptr_o	output	logic [LogDepth:0]
async_data_slave_r_data_o	output	r_chan_t [2**LogDepth-1:0]
async_data_slave_r_wptr_o	output	logic [LogDepth:0]
async_data_slave_r_rptr_i	input	logic [LogDepth:0]
dst_clk_i	input	logic
dst_rst_ni	input	logic
dst_req_o	output	axi_req_t
dst_resp_i	input	axi_resp_t
dst_clk_i	input	logic
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_rst_ni	input	logic

Table 1576. pulp-platform_axi/src/axi_cdc_dst.sv — typedef / using (8개)

새 이름	원래 형
id_t	logic [AXI_ID_WIDTH-1:0]
addr_t	logic [AXI_ADDR_WIDTH-1:0]
data_t	logic [AXI_DATA_WIDTH-1:0]
strb_t	logic [AXI_DATA_WIDTH/8-1:0]
user_t	logic [AXI_USER_WIDTH-1:0]
addr_t	logic [AXI_ADDR_WIDTH-1:0]
data_t	logic [AXI_DATA_WIDTH-1:0]
strb_t	logic [AXI_DATA_WIDTH/8-1:0]

API — axi_cdc_src.sv

Table 1577. pulp-platform_axi/src/axi_cdc_src.sv — module / interface / package (3개)

이름	종류
axi_cdc_src	module
axi_cdc_src_intf	module
axi_lite_cdc_src_intf	module

Table 1578. pulp-platform_axi/src/axi_cdc_src.sv — parameter (19개)

파라미터	형/폭	기본값
LogDepth	int unsigned	1
SyncStages	int unsigned	2
aw_chan_t		logic
w_chan_t		logic
b_chan_t		logic
ar_chan_t		logic
r_chan_t		logic
axi_req_t		logic
axi_resp_t		logic
AXI_ID_WIDTH	int unsigned	0
AXI_ADDR_WIDTH	int unsigned	0
AXI_DATA_WIDTH	int unsigned	0
AXI_USER_WIDTH	int unsigned	0
LOG_DEPTH	int unsigned	1
SYNC_STAGES	int unsigned	2
AXI_ADDR_WIDTH	int unsigned	0
AXI_DATA_WIDTH	int unsigned	0
LOG_DEPTH	int unsigned	1
SYNC_STAGES	int unsigned	2

Table 1579. pulp-platform_axi/src/axi_cdc_src.sv — 포트 (23개)

포트	방향	형/폭
src_clk_i	input	logic
src_rst_ni	input	logic
src_req_i	input	axi_req_t
src_resp_o	output	axi_resp_t
async_data_master_aw_data_o	output	aw_chan_t [2**LogDepth-1:0]
async_data_master_aw_wptr_o	output	logic [LogDepth:0]
async_data_master_aw_rptr_i	input	logic [LogDepth:0]
async_data_master_w_data_o	output	w_chan_t [2**LogDepth-1:0]
async_data_master_w_wptr_o	output	logic [LogDepth:0]
async_data_master_w_rptr_i	input	logic [LogDepth:0]
async_data_master_b_data_i	input	b_chan_t [2**LogDepth-1:0]
async_data_master_b_wptr_i	input	logic [LogDepth:0]
async_data_master_b_rptr_o	output	logic [LogDepth:0]

포트	방향	형/폭
async_data_master_ar_data_o	output	ar_chan_t [2**LogDepth-1:0]
async_data_master_ar_wptr_o	output	logic [LogDepth:0]
async_data_master_ar_rptr_i	input	logic [LogDepth:0]
async_data_master_r_data_i	input	r_chan_t [2**LogDepth-1:0]
async_data_master_r_wptr_i	input	logic [LogDepth:0]
async_data_master_r_rptr_o	output	logic [LogDepth:0]
src_clk_i	input	logic
src_rst_ni	input	logic
src_clk_i	input	logic
src_rst_ni	input	logic

Table 1580. pulp-platform_axi/src/axi_cdc_src.sv — typedef / using (8개)

새 이름	원래 형
id_t	logic [AXI_ID_WIDTH-1:0]
addr_t	logic [AXI_ADDR_WIDTH-1:0]
data_t	logic [AXI_DATA_WIDTH-1:0]
strb_t	logic [AXI_DATA_WIDTH/8-1:0]
user_t	logic [AXI_USER_WIDTH-1:0]
addr_t	logic [AXI_ADDR_WIDTH-1:0]
data_t	logic [AXI_DATA_WIDTH-1:0]
strb_t	logic [AXI_DATA_WIDTH/8-1:0]

API — axi_chan_compare.sv

Table 1581. pulp-platform_axi/src/axi_chan_compare.sv — module / interface / package (1개)

이름	종류
axi_chan_compare	module

Table 1582. pulp-platform_axi/src/axi_chan_compare.sv — parameter (10개)

파라미터	형/폭	기본값
IgnoreId	bit	1'b0
AllowReordering	bit	1'b0
IdWidth	int unsigned	1
aw_chan_t		logic
w_chan_t		logic
b_chan_t		logic
ar_chan_t		logic

파라미터	형/폭	기본값
r_chan_t		logic
req_t		logic
resp_t		logic

Table 1583. pulp-platform_axi/src/axi_chan_compare.sv — localparam (1개)

상수	형/폭	값
NumIds		(AllowReordering)? 2**IdWidth : 1

Table 1584. pulp-platform_axi/src/axi_chan_compare.sv — 포트 (16개)

포트	방향	형/폭
clk_a_i	input	logic
clk_b_i	input	logic
axi_a_req	input	req_t
axi_a_res	input	resp_t
axi_b_req	input	req_t
axi_b_res	input	resp_t
aw_expected	input	aw_chan_t
aw_received	input	aw_chan_t
ar_expected	input	ar_chan_t
ar_received	input	ar_chan_t
w_expected	input	w_chan_t
w_received	input	w_chan_t
b_expected	input	b_chan_t
b_received	input	b_chan_t
r_expected	input	r_chan_t
r_received	input	r_chan_t

API — axi_cut.sv**Table 1585. pulp-platform_axi/src/axi_cut.sv — module / interface / package (3개)**

이름	종류
axi_cut	module
axi_cut_intf	module
axi_lite_cut_intf	module

Table 1586. pulp-platform_axi/src/axi_cut.sv — parameter (31개)

파라미터	형/폭	기본값
Bypass	bit	1'b0

파라미터	형/폭	기본값
BypassAw	bit	Bypass
BypassW	bit	Bypass
BypassB	bit	Bypass
BypassAr	bit	Bypass
BypassR	bit	Bypass
aw_chan_t		logic
w_chan_t		logic
b_chan_t		logic
ar_chan_t		logic
r_chan_t		logic
axi_req_t		logic
axi_resp_t		logic
BYPASS	bit	1'b0
BYPASS_AW	bit	BYPASS
BYPASS_W	bit	BYPASS
BYPASS_B	bit	BYPASS
BYPASS_AR	bit	BYPASS
BYPASS_R	bit	BYPASS
ADDR_WIDTH	int unsigned	0
DATA_WIDTH	int unsigned	0
ID_WIDTH	int unsigned	0
USER_WIDTH	int unsigned	0
BYPASS	bit	1'b0
BYPASS_AW	bit	BYPASS
BYPASS_W	bit	BYPASS
BYPASS_B	bit	BYPASS
BYPASS_AR	bit	BYPASS
BYPASS_R	bit	BYPASS
ADDR_WIDTH	int unsigned	0
DATA_WIDTH	int unsigned	0

Table 1587. `pulp-platform_axi/src/axi_cut.sv` — 포트 (10개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic

포트	방향	형/폭
slv_req_i	input	axi_req_t
slv_resp_o	output	axi_resp_t
mst_req_o	output	axi_req_t
mst_resp_i	input	axi_resp_t
clk_i	input	logic
rst_ni	input	logic
clk_i	input	logic
rst_ni	input	logic

Table 1588. pulp-platform_axi/src/axi_cut.sv — typedef / using (8개)

새 이름	원래 형
id_t	logic [ID_WIDTH-1:0]
addr_t	logic [ADDR_WIDTH-1:0]
data_t	logic [DATA_WIDTH-1:0]
strb_t	logic [DATA_WIDTH/8-1:0]
user_t	logic [USER_WIDTH-1:0]
addr_t	logic [ADDR_WIDTH-1:0]
data_t	logic [DATA_WIDTH-1:0]
strb_t	logic [DATA_WIDTH/8-1:0]

API — axi_delayer.sv

Table 1589. pulp-platform_axi/src/axi_delayer.sv — module / interface / package (2개)

이름	종류
axi_delayer	module
axi_delayer_intf	module

Table 1590. pulp-platform_axi/src/axi_delayer.sv — parameter (19개)

파라미터	형/폭	기본값
aw_chan_t		logic
w_chan_t		logic
b_chan_t		logic
ar_chan_t		logic
r_chan_t		logic
axi_req_t		logic
axi_resp_t		logic
StallRandomInput	bit	0

파라미터	형/폭	기본값
StallRandomOutput	bit	0
FixedDelayInput	int unsigned	1
FixedDelayOutput	int unsigned	1
AXI_ID_WIDTH	int unsigned	0
AXI_ADDR_WIDTH	int unsigned	0
AXI_DATA_WIDTH	int unsigned	0
AXI_USER_WIDTH	int unsigned	0
STALL_RANDOM_INPUT	bit	0
STALL_RANDOM_OUTPUT	bit	0
FIXED_DELAY_INPUT	int unsigned	1
FIXED_DELAY_OUTPUT	int unsigned	1

Table 1591. pulp-platform_axi/src/axi_delayer.sv — 포트 (8개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
slv_req_i	input	axi_req_t
slv_resp_o	output	axi_resp_t
mst_req_o	output	axi_req_t
mst_resp_i	input	axi_resp_t
clk_i	input	logic
rst_ni	input	logic

Table 1592. pulp-platform_axi/src/axi_delayer.sv — typedef / using (5개)

새 이름	원래 형
id_t	logic [AXI_ID_WIDTH-1:0]
addr_t	logic [AXI_ADDR_WIDTH-1:0]
data_t	logic [AXI_DATA_WIDTH-1:0]
strb_t	logic [AXI_DATA_WIDTH/8-1:0]
user_t	logic [AXI_USER_WIDTH-1:0]

API — axi_demux.sv

Table 1593. pulp-platform_axi/src/axi_demux.sv — module / interface / package (2개)

이름	종류
axi_demux	module
axi_demux_intf	module

Table 1594. pulp-platform_axi/src/axi_demux.sv — parameter (36개)

파라미터	형/폭	기본값
AxiIdWidth	int unsigned	32'd0
AtopSupport	bit	1'b1
aw_chan_t		logic
w_chan_t		logic
b_chan_t		logic
ar_chan_t		logic
r_chan_t		logic
axi_req_t		logic
axi_resp_t		logic
NoMstPorts	int unsigned	32'd0
MaxTrans	int unsigned	32'd8
AxiLookBits	int unsigned	32'd3
UniqueIds	bit	1'b0
SpillAw	bit	1'b1
SpillW	bit	1'b0
SpillB	bit	1'b0
SpillAr	bit	1'b1
SpillR	bit	1'b0
SelectWidth	int unsigned	(NoMstPorts > 32'd1) ? \$clog2(NoMstPorts) : 32'd1
select_t		logic [SelectWidth-1:0]
AXI_ID_WIDTH	int unsigned	32'd0
ATOP_SUPPORT	bit	1'b1
AXI_ADDR_WIDTH	int unsigned	32'd0
AXI_DATA_WIDTH	int unsigned	32'd0
AXI_USER_WIDTH	int unsigned	32'd0
NO_MST_PORTS	int unsigned	32'd3
MAX_TRANS	int unsigned	32'd8
AXI_LOOK_BITS	int unsigned	32'd3
UNIQUE_IDS	bit	1'b0
SPILL_AW	bit	1'b1
SPILL_W	bit	1'b0
SPILL_B	bit	1'b0
SPILL_AR	bit	1'b1
SPILL_R	bit	1'b0

파라미터	형/폭	기본값
SELECT_WIDTH	int unsigned	(NO_MST_PORTS > 32'd1) ? \$clog2(NO_MST_PORTS) : 32'd1
select_t		logic [SELECT_WIDTH-1:0] // MST port select type

Table 1595. pulp-platform_axi/src/axi_demux.sv — 포트 (12개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
slv_req_i	input	axi_req_t
slv_aw_select_i	input	select_t
slv_ar_select_i	input	select_t
slv_resp_o	output	axi_resp_t
mst_reqs_o	output	axi_req_t [NoMstPorts-1:0]
mst_resps_i	input	axi_resp_t [NoMstPorts-1:0]
clk_i	input	logic
rst_ni	input	logic
slv_aw_select_i	input	select_t
slv_ar_select_i	input	select_t

Table 1596. pulp-platform_axi/src/axi_demux.sv — typedef / using (5개)

새 이름	원래 형
id_t	logic [AXI_ID_WIDTH-1:0]
addr_t	logic [AXI_ADDR_WIDTH-1:0]
data_t	logic [AXI_DATA_WIDTH-1:0]
strb_t	logic [AXI_DATA_WIDTH/8-1:0]
user_t	logic [AXI_USER_WIDTH-1:0]

Table 1597. pulp-platform_axi/src/axi_demux.sv — #define / `define 매크로 (1개)

매크로	인자	정의
TARGET_VSIM		`endif

API — axi_demux_id_counters.sv**Table 1598. pulp-platform_axi/src/axi_demux_id_counters.sv — module / interface / package (1개)**

이름	종류
axi_demux_id_counters	module

Table 1599. pulp-platform_axi/src/axi_demux_id_counters.sv — parameter (4개)

파라미터	형/폭	기본값
AxiIdBits	int unsigned	2

파라미터	형/폭	기본값
CounterWidth	int unsigned	4
MaxTrans	int unsigned	2**CounterWidth - 1
mst_port_select_t		logic

Table 1600. pulp-platform_axi/src/axi_demux_id_counters.sv — localparam (1개)

상수	형/폭	값
NoCounters	int unsigned	2**AxildBits

Table 1601. pulp-platform_axi/src/axi_demux_id_counters.sv — 포트 (14개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
lookup_axi_id_i	input	logic [AxildBits-1:0]
lookup_mst_select_o	output	mst_port_select_t
lookup_mst_select_occupied_o	output	logic
full_o	output	logic
push_axi_id_i	input	logic [AxildBits-1:0]
push_mst_select_i	input	mst_port_select_t
push_i	input	logic
inject_axi_id_i	input	logic [AxildBits-1:0]
inject_i	input	logic
pop_axi_id_i	input	logic [AxildBits-1:0]
pop_i	input	logic
any_outstanding_trx_o	output	logic

Table 1602. pulp-platform_axi/src/axi_demux_id_counters.sv — typedef / using (17개)

새 이름	원래 형
cnt_t	logic [CounterWidth-1:0]

1. pulp-platform_axi/src/axi_atop_filter.sv (447 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10 //
11 // Authors:

```

```

12 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
13 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
14
15 /// Filter atomic operations (ATOPs) in a protocol-compliant
  manner.
16 ///
17 /// This module filters atomic operations (ATOPs), i.e., write
  transactions that have a non-zero
18 /// `aw_atop` value, from its `slv` to its `mst` port. This
  module guarantees that:
19 ///
20 /// 1) `aw_atop` is always zero on the `mst` port;
21 ///
22 /// 2) write transactions with non-zero `aw_atop` on the `slv`
  port are handled in conformance with
23 /// the AXI standard by replying to such write transactions
  with the proper B and R responses.
24 /// The response code on atomic operations that reach this
  module is always SLVERR
25 /// (implementation-specific, not defined in the AXI stand-

```

```

ard).
26 ///
27 /// ## Intended usage
28 /// This module is intended to be placed between masters that
may issue ATOPs and slaves that do not
29 /// support ATOPs. That way, this module ensures that the AXI
protocol remains in a defined state on
30 /// systems with mixed ATOP capabilities.
31 ///
32 /// ## Specification reminder
33 /// The AXI standard specifies that there may be no ordering
requirements between different atomic
34 /// bursts (i.e., a burst started by an AW with ATOP other
than 0) and none between atomic bursts

35 /// and non-atomic bursts [E2.1.4]. That is, **an atomic burst
may never have the same ID as any
36 /// other write or read burst that is in-flight at the same
time**.
37 module axi_atop_filter #(
38     /// AXI ID width
39     parameter int unsigned AxiIdWidth = 0,
40     /// Maximum number of in-flight AXI write transactions
41     parameter int unsigned AxiMaxWriteTxns = 0,
42     /// AXI request type
43     parameter type axi_req_t = logic,
44     /// AXI response type
45     parameter type axi_resp_t = logic
46 ) (
47     /// Rising-edge clock of both ports
48     input logic clk_i,
49     /// Asynchronous reset, active low
50     input logic rst_ni,
51     /// Slave port request
52     input axi_req_t slv_req_i,
53     /// Slave port response
54     output axi_resp_t slv_resp_o,
55     /// Master port request
56     output axi_req_t mst_req_o,
57     /// Master port response
58     input axi_resp_t mst_resp_i
59 );
60
61     // Minimum counter width is 2 to detect underflows.
62     localparam int unsigned COUNTER_WIDTH = (AxiMaxWriteTxns ==
1) ? 2 : $clog2(AxiMaxWriteTxns+1);
63     typedef struct packed {
64         logic underflow;
65         logic [COUNTER_WIDTH-1:0] cnt;
66     } cnt_t;
67     cnt_t w_cnt_d, w_cnt_q;
68
69     typedef enum logic [2:0] {
70         W_RESET, W_FEEDTHROUGH, BLOCK_AW, ABSORB_W, HOLD_B, IN-
JECT_B, WAIT_R
71     } w_state_e;
72     w_state_e w_state_d, w_state_q;
73
74     typedef enum logic [1:0] { R_RESET, R_FEEDTHROUGH, INJECT_R,
R_HOLD } r_state_e;
75     r_state_e r_state_d, r_state_q;
76
77     typedef logic [AxiIdWidth-1:0] id_t;
78     id_t id_d, id_q;
79
80     typedef logic [7:0] len_t;
81     len_t r_beats_d, r_beats_q;
82
83     typedef struct packed {
84         len_t len;
85     } r_resp_cmd_t;
86     r_resp_cmd_t r_resp_cmd_push, r_resp_cmd_pop;
87
88     logic aw_without_complete_w_downstream,
89         complete_w_without_aw_downstream,
90         r_resp_cmd_push_valid, r_resp_cmd_push_ready,
91         r_resp_cmd_pop_valid, r_resp_cmd_pop_ready;
92
93     // An AW without a complete W burst is in-flight downstream
if the W counter is > 0 and not
94     // underflowed.

95     assign aw_without_complete_w_downstream = !w_cnt_q.underflow &&
(w_cnt_q.cnt > 0);
96     // A complete W burst without AW is in-flight downstream if
the W counter is -1.
97     assign complete_w_without_aw_downstream = w_cnt_q.underflow
&& (w_cnt_q.cnt);
98
99     // Manage AW, W, and B channels.
100     always_comb begin
101         // Defaults:
102         // Disable AW and W handshakes.
103         mst_req_o.aw_valid = 1'b0;
104         slv_resp_o.aw_ready = 1'b0;
105         mst_req_o.w_valid = 1'b0;
106         slv_resp_o.w_ready = 1'b0;
107         // Feed write responses through.
108         mst_req_o.b_ready = slv_req_i.b_ready;
109         slv_resp_o.b_valid = mst_resp_i.b_valid;

```

```

110     slv_resp_o.b = mst_resp_i.b;
111     // Keep ID stored for B and R response.
112     id_d = id_q;
113     // Do not push R response commands.
114     r_resp_cmd_push_valid = 1'b0;
115     // Keep the current state.
116     w_state_d = w_state_q;
117
118     unique case (w_state_q)
119         W_RESET: w_state_d = W_FEEDTHROUGH;
120
121         W_FEEDTHROUGH: begin
122             // Feed AW channel through if the maximum number of
outstanding bursts is not reached.

123             if (complete_w_without_aw_downstream || (w_cnt_q.cnt <
AxiMaxWriteTxns)) begin
124                 mst_req_o.aw_valid = slv_req_i.aw_valid;
125                 slv_resp_o.aw_ready = mst_resp_i.aw_ready;
126             end
127             // Feed W channel through if ..
128             if (aw_without_complete_w_downstream // .. downstream
is missing W bursts ..
129                 // .. or a new non-ATOP AW is being applied and
there is not already a complete W burst
130                 // downstream (to prevent underflows of w_cnt).
131                 || ((slv_req_i.aw_valid && slv_req_i.aw.atop[5:4]
== axi_pkg::ATOP_NONE)
132                     && !complete_w_without_aw_downstream)
133             ) begin
134                 mst_req_o.w_valid = slv_req_i.w_valid;
135                 slv_resp_o.w_ready = mst_resp_i.w_ready;
136             end
137             // Filter out AWs that are atomic operations.
138             if (slv_req_i.aw_valid && slv_req_i.aw.atop[5:4] !=
axi_pkg::ATOP_NONE) begin

139                 mst_req_o.aw_valid = 1'b0; // Do not let AW pass to
master port.
140                 slv_resp_o.aw_ready = 1'b1; // Absorb AW on slave
port.
141                 id_d = slv_req_i.aw.id; // Store ID for B response.
142                 // Some atomic operations require a response on the
R channel.
143                 if (slv_req_i.aw.atop[axi_pkg::ATOP_R_RESP]) begin
144                     // Push R response command. We do not have to
wait for the ready of the register
145                     // because we know it is ready: we are its only
master and will wait for the register to
146                     // be emptied before going back to the `W_FEED-
THROUGH` state.
147                     r_resp_cmd_push_valid = 1'b1;
148                 end
149                 // If downstream is missing W beats, block the AW
channel and let the W bursts complete.
150                 if (aw_without_complete_w_downstream) begin
151                     w_state_d = BLOCK_AW;
152                     // If downstream is not missing W beats, absorb the
W beats for this atomic AW.
153                 end else begin
154                     mst_req_o.w_valid = 1'b0; // Do not let W beats
pass to master port.
155                     slv_resp_o.w_ready = 1'b1; // Absorb W beats on
slave port.
156                     if (slv_req_i.w_valid && slv_req_i.w.last) begin
157                         // If the W beat is valid and the last, proceed
by injecting the B response.
158                         // However, if there is a non-handshaked B on
our response port, we must let that
159                         // complete first.
160                         if (slv_resp_o.b_valid && !slv_req_i.b_ready)
begin
161                             w_state_d = HOLD_B;
162                         end else begin
163                             w_state_d = INJECT_B;
164                         end
165                     end else begin
166                         // Otherwise continue with absorbing W beats.
167                         w_state_d = ABSORB_W;
168                     end
169                 end
170             end
171         end

172         BLOCK_AW: begin
173             // Feed W channel through to let outstanding bursts
complete.
174             if (aw_without_complete_w_downstream) begin
175                 mst_req_o.w_valid = slv_req_i.w_valid;
176                 slv_resp_o.w_ready = mst_resp_i.w_ready;
177             end else begin
178                 // If there are no more outstanding W bursts, start
absorbing the next W burst.
179                 slv_resp_o.w_ready = 1'b1;
180                 if (slv_req_i.w_valid && slv_req_i.w.last) begin

181                     // If the W beat is valid and the last, proceed by
injecting the B response.
182                     if (slv_resp_o.b_valid && !slv_req_i.b_ready) be-
gin
183                         w_state_d = HOLD_B;

```

```

185         end else begin
186             w_state_d = INJECT_B;
187         end
188     end else begin
189         // Otherwise continue with absorbing W beats.
190         w_state_d = ABSORB_W;
191     end
192 end
193 end
194
195 ABSORB_W: begin
196     // Absorb all W beats of the current burst.
197     slv_resp_o.w_ready = 1'b1;
198     if (slv_req_i.w_valid && slv_req_i.w.last) begin
199         if (slv_resp_o.b_valid && !slv_req_i.b_ready) begin
200             w_state_d = HOLD_B;
201         end else begin
202             w_state_d = INJECT_B;
203         end
204     end
205 end
206
207 HOLD_B: begin
208     // Proceed with injection of B response upon hand-
shake.
209     if (slv_resp_o.b_valid && slv_req_i.b_ready) begin
210         w_state_d = INJECT_B;
211     end
212 end
213
214 INJECT_B: begin
215     // Pause forwarding of B response.
216     mst_req_o.b_ready = 1'b0;
217
218     // Inject error response instead. Since the B channel
has an ID and the atomic burst we are
219     // replying to is guaranteed to be the only burst with
this ID in flight, we do not have to
220     // observe any ordering and can immediately inject on
the B channel.
221     slv_resp_o.b = '0;
222     slv_resp_o.b.id = id_q;
223     slv_resp_o.b.resp = axi_pkg::RESP_SLVERR;
224     slv_resp_o.b_valid = 1'b1;
225     if (slv_req_i.b_ready) begin
226         // If not all beats of the R response have been in-
jected, wait for them. Otherwise, return
227         // to 'W_FEEDTHROUGH'.
228         if (r_resp_cmd_pop_valid && !r_resp_cmd_pop_ready)
begin
229             w_state_d = WAIT_R;
230         end else begin
231             w_state_d = W_FEEDTHROUGH;
232         end
233     end
234 end
235
236 WAIT_R: begin
237     // Wait with returning to 'W_FEEDTHROUGH' until all
beats of the R response have been
238     // injected.
239     if (!r_resp_cmd_pop_valid) begin
240         w_state_d = W_FEEDTHROUGH;
241     end
242 end
243
244 default: w_state_d = W_RESET;
245 endcase
246 end
247
248 // Connect signals on AW and W channel that are not managed
by the control FSM from slave port to
249 // master port.
250 // Feed-through of the AW and W vectors, make sure that
downstream aw.atop is always zero
251 always_comb begin
252     // overwrite the atop signal
253     mst_req_o.aw = slv_req_i.aw;
254     mst_req_o.aw.atop = '0;
255 end
256
257 assign mst_req_o.w = slv_req_i.w;
258
259 // Manage R channel.
260 always_comb begin
261     // Defaults:
262     // Feed read responses through.
263     slv_resp_o.r = mst_resp_i.r;
264     slv_resp_o.r_valid = mst_resp_i.r_valid;
265     mst_req_o.r_ready = slv_req_i.r_ready;
266     // Do not pop R response command.
267     r_resp_cmd_pop_ready = 1'b0;
268     // Keep the current value of the beats counter.
269     r_beats_d = r_beats_q;
270     // Keep the current state.
271     r_state_d = r_state_q;
272
273     unique case (r_state_q)
274         R_RESET: r_state_d = R_FEEDTHROUGH;
275
276         R_FEEDTHROUGH: begin
277             if (mst_resp_i.r_valid && !slv_req_i.r_ready) begin
278                 r_state_d = R_HOLD;
279             end else if (r_resp_cmd_pop_valid) begin
280                 // Upon a command to inject an R response, immedi-
ately proceed with doing so because there
281                 // are no ordering requirements with other bursts
that may be ongoing on the R channel at
282                 // this moment.
283                 r_beats_d = r_resp_cmd_pop.len;
284                 r_state_d = INJECT_R;
285             end
286         end
287
288         INJECT_R: begin
289             mst_req_o.r_ready = 1'b0;
290             slv_resp_o.r = '0;
291             slv_resp_o.r.id = id_q;
292             slv_resp_o.r.resp = axi_pkg::RESP_SLVERR;
293             slv_resp_o.r.last = (r_beats_q == '0);
294             slv_resp_o.r_valid = 1'b1;
295             if (slv_req_i.r_ready) begin
296                 if (slv_resp_o.r.last) begin
297                     r_resp_cmd_pop_ready = 1'b1;
298                     r_state_d = R_FEEDTHROUGH;
299                 end else begin
300                     r_beats_d -= 1;
301                 end
302             end
303         end
304
305         R_HOLD: begin
306             if (mst_resp_i.r_valid && slv_req_i.r_ready) begin
307                 r_state_d = R_FEEDTHROUGH;
308             end
309         end
310
311         default: r_state_d = R_RESET;
312     endcase
313 end
314
315 // Feed all signals on AR through.
316 assign mst_req_o.ar = slv_req_i.ar;
317 assign mst_req_o.ar_valid = slv_req_i.ar_valid;
318 assign slv_resp_o.ar_ready = mst_resp_i.ar_ready;
319
320 // Keep track of outstanding downstream write bursts and re-
sponses.
321 always_comb begin
322     w_cnt_d = w_cnt_q;
323     if (mst_req_o.aw_valid && mst_resp_i.aw_ready) begin
324         w_cnt_d.cnt += 1;
325     end
326     if (mst_req_o.w_valid && mst_resp_i.w_ready &&
mst_req_o.w.last) begin
327         w_cnt_d.cnt -= 1;
328     end
329     if (w_cnt_q.underflow && (w_cnt_d.cnt == '0)) begin
330         w_cnt_d.underflow = 1'b0;
331     end else if (w_cnt_q.cnt == '0 && (w_cnt_d.cnt)) begin
332         w_cnt_d.underflow = 1'b1;
333     end
334 end
335
336 always_ff @(posedge clk_i, negedge rst_ni) begin
337     if (!rst_ni) begin
338         id_q <= '0;
339         r_beats_q <= '0;
340         r_state_q <= R_RESET;
341         w_cnt_q <= '{default: '0};
342         w_state_q <= W_RESET;
343     end else begin
344         id_q <= id_d;
345         r_beats_q <= r_beats_d;
346         r_state_q <= r_state_d;
347         w_cnt_q <= w_cnt_d;
348         w_state_q <= w_state_d;
349     end
350 end
351
352 cc_stream_register #(
353     .data_t (r_resp_cmd_t)
354 ) r_resp_cmd (
355     .clk_i (clk_i),
356     .rst_ni (rst_ni),
357     .clr_i (1'b0),
358     .valid_i (r_resp_cmd_push_valid),
359     .ready_o (r_resp_cmd_push_ready),
360     .data_i (r_resp_cmd_push),
361     .valid_o (r_resp_cmd_pop_valid),
362     .ready_i (r_resp_cmd_pop_ready),
363     .data_o (r_resp_cmd_pop)
364 );
365 assign r_resp_cmd_push.len = slv_req_i.aw.len;
366
367 // pragma translate_off
368 `ifndef VERILATOR
369     initial begin: p_assertions
370         assert (AxiIdWidth >= 1) else $fatal(1, "AXI ID width must
be at least 1!");
371         assert (AxiMaxWriteTxns >= 1)
372             else $fatal(1, "Maximum number of outstanding write
transactions must be at least 1!");
373     end
374 `endif

```

```

369 end
370 `endif
371 // pragma translate_on
372 endmodule
373
374 `include "axi/assign.svh"
375 `include "axi/typedef.svh"
376
377 /// Interface variant of [`axi_atop_filter`](mod-
ule.axi_atop_filter).
378 module axi_atop_filter_intf #(
379     /// AXI ID width
380     parameter int unsigned AXI_ID_WIDTH = 0,
381     /// AXI address width
382     parameter int unsigned AXI_ADDR_WIDTH = 0,
383     /// AXI data width
384     parameter int unsigned AXI_DATA_WIDTH = 0,
385     /// AXI user signal width
386     parameter int unsigned AXI_USER_WIDTH = 0,
387     /// Maximum number of in-flight AXI write transactions
388     parameter int unsigned AXI_MAX_WRITE_TXNS = 0
389 ) (
390     /// Rising-edge clock of both ports
391     input logic clk_i,
392     /// Asynchronous reset, active low
393     input logic rst_ni,
394     /// Slave interface port
395     AXI_BUS.Slave slv,
396     /// Master interface port
397     AXI_BUS.Master mst
398 );
399
400 typedef logic [AXI_ID_WIDTH-1:0] id_t;
401 typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
402 typedef logic [AXI_DATA_WIDTH-1:0] data_t;
403 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
404 typedef logic [AXI_USER_WIDTH-1:0] user_t;
405
406 `AXI_TYPEDEF_AW_CHAN_T(aw_chan_t, addr_t, id_t, user_t)
407 `AXI_TYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)
408 `AXI_TYPEDEF_B_CHAN_T(b_chan_t, id_t, user_t)
409 `AXI_TYPEDEF_AR_CHAN_T(ar_chan_t, addr_t, id_t, user_t)
410 `AXI_TYPEDEF_R_CHAN_T(r_chan_t, data_t, id_t, user_t)
411 `AXI_TYPEDEF_REQ_T(axi_req_t, aw_chan_t, w_chan_t,
ar_chan_t)
412 `AXI_TYPEDEF_RESP_T(axi_resp_t, b_chan_t, r_chan_t)
413
414 axi_req_t slv_req, mst_req;
415 axi_resp_t slv_resp, mst_resp;
416
417 `AXI_ASSIGN_TO_REQ(slv_req, slv)
418 `AXI_ASSIGN_FROM_RESP(slv, slv_resp)
419
420 `AXI_ASSIGN_FROM_REQ(mst, mst_req)
421 `AXI_ASSIGN_TO_RESP(mst_resp, mst)
422
423 axi_atop_filter #(
424     .AxIdWidth ( AXI_ID_WIDTH ),
425     // Maximum number of AXI write bursts outstanding at the
same time
426     .AxMaxWriteTxns ( AXI_MAX_WRITE_TXNS ),
427     // AXI request & response type
428     .axi_req_t ( axi_req_t ),
429     .axi_resp_t ( axi_resp_t )
430 ) i_axi_atop_filter (
431     .clk_i,
432     .rst_ni,
433     .slv_req_i ( slv_req ),
434     .slv_resp_o ( slv_resp ),
435     .mst_req_o ( mst_req ),
436     .mst_resp_i ( mst_resp )
437 );
438 // pragma translate_off
439 `ifndef VERILATOR
440     initial begin: p_assertions
441         assert (AXI_ADDR_WIDTH >= 1) else $fatal(1, "AXI ADDR
width must be at least 1!");
442         assert (AXI_DATA_WIDTH >= 1) else $fatal(1, "AXI DATA
width must be at least 1!");
443         assert (AXI_USER_WIDTH >= 1) else $fatal(1, "AXI USER
width must be at least 1!");
444     end
445 `endif
446 // pragma translate_on
447 endmodule

```

2. pulp-platform_axi/src/axi_burst_splitter.sv (97 lines)

```

1 // Copyright (c) 2020 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials

```

```

distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the License.
11 //
12 // Authors:
13 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
14 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
15
16 `include "axi/typedef.svh"
17
18 /// Split AXI4 bursts into single-beat transactions.
19 ///
20 /// ## Limitations
21 ///
22 /// - This module does not support wrapping
23 ([`axi_pkg::BURST_WRAP`](package.axi_pkg)) bursts and
24 responds to such bursts with slave error(s).
25 /// - This module does not support atomic operations (ATOPs)
and responds to ATOPs with a slave
26 error. Place an [`axi_atop_filter`](mod-
ule.axi_atop_filter) before this module if upstream
27 modules can generate ATOPs.
28 module axi_burst_splitter #(
29     // Maximum number of AXI read bursts outstanding at the same
time
30     parameter int unsigned MaxReadTxns = 32'd0,
31     // Maximum number of AXI write bursts outstanding at the
same time
32     parameter int unsigned MaxWriteTxns = 32'd0,
33     // Internal ID queue can work in two bandwidth modes: see
id_queue.sv for details
34     parameter bit FullBW = 0,
35     // AXI Bus Types
36     parameter int unsigned AddrWidth = 32'd0,
37     parameter int unsigned DataWidth = 32'd0,
38     parameter int unsigned IdWidth = 32'd0,
39     // Number of least-significant ID bits used by the internal
demux to track in-flight
40     parameter int unsigned AxilookBits = IdWidth, see doc/
axi_demux.md for the trade-off.
41     parameter int unsigned UserWidth = 32'd0,
42     parameter type axi_req_t = logic,
43     parameter type axi_resp_t = logic
44 ) (
45     input logic clk_i,
46     input logic rst_ni,
47
48     // Input / Slave Port
49     input axi_req_t slv_req_i,
50     output axi_resp_t slv_resp_o,
51
52     // Output / Master Port
53     output axi_req_t mst_req_o,
54     input axi_resp_t mst_resp_i
55 );
56
57 typedef logic [AddrWidth-1:0] addr_t;
58 typedef logic [DataWidth-1:0] data_t;
59 typedef logic [DataWidth/8-1:0] strb_t;
60 typedef logic [IdWidth-1:0] id_t;
61 typedef logic [UserWidth-1:0] user_t;
62
63 `AXI_TYPEDEF_AW_CHAN_T(axi_aw_chan_t, addr_t, id_t, user_t)
64 `AXI_TYPEDEF_W_CHAN_T(axi_w_chan_t, data_t, strb_t, user_t)
65 `AXI_TYPEDEF_B_CHAN_T(axi_b_chan_t, id_t, user_t)
66 `AXI_TYPEDEF_AR_CHAN_T(axi_ar_chan_t, addr_t, id_t, user_t)
67 `AXI_TYPEDEF_R_CHAN_T(axi_r_chan_t, data_t, id_t, user_t)
68
69 axi_burst_splitter_gran #(
70     .MaxReadTxns ( MaxReadTxns ),
71     .MaxWriteTxns ( MaxWriteTxns ),
72     .CutPath ( 1'b0 ),
73     .DisableChecks ( 1'b0 ),
74     .FullBW ( FullBW ),
75     .AddrWidth ( AddrWidth ),
76     .DataWidth ( DataWidth ),
77     .IdWidth ( IdWidth ),
78     .AxilookBits ( AxilookBits ),
79     .UserWidth ( UserWidth ),
80     .axi_req_t ( axi_req_t ),
81     .axi_resp_t ( axi_resp_t ),
82     .axi_aw_chan_t ( axi_aw_chan_t ),
83     .axi_w_chan_t ( axi_w_chan_t ),
84     .axi_b_chan_t ( axi_b_chan_t ),
85     .axi_ar_chan_t ( axi_ar_chan_t ),
86     .axi_r_chan_t ( axi_r_chan_t )
87 ) i_axi_burst_splitter_gran (
88     .clk_i,
89     .rst_ni,
90     .len_limit_i ( 8'h00 ),
91     .slv_req_i,
92     .slv_resp_o,
93     .mst_req_o,
94     .mst_resp_i
95 );

```

```

96
97 endmodule

```

3. pulp-platform_axi/src/axi_burst_splitter_gran.sv (751 lines)

```

1 // Copyright 2023 ETH Zurich and University of Bologna.
2 // Solderpad Hardware License, Version 0.51, see LICENSE for
details.
3 // SPDX-License-Identifier: SHL-0.51
4 //
5 // Authors:
6 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
7 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
8 // - Thomas Benz <tbenz@iis.ee.ethz.ch>
9
10 `include "axi/typedef.svh"
11 `include "common_cells/registers.svh"
12
13 /// Split AXI4 bursts into single-beat transactions.
14 ///
15 /// ## Limitations
16 ///
17 /// - This module does not support wrapping
18 ([`axi_pkg::BURST_WRAP](package.axi_pkg)) bursts and
19 /// responds to such bursts with slave error(s).
20 /// - This module does not support atomic operations (ATOPs)
and responds to ATOPs with a slave
21 /// error. Place an [`axi_atop_filter`](mod-
ule.axi_atop_filter) before this module if upstream
22 /// modules can generate ATOPs.
23 module axi_burst_splitter_gran #(
24 /// Maximum number of AXI read bursts outstanding at the
same time
25 parameter int unsigned MaxReadTxns = 32'd0,
26 /// Maximum number of AXI write bursts outstanding at the
same time
27 parameter int unsigned MaxWriteTxns = 32'd0,
28 /// Internal ID queue can work in two bandwidth modes: see
id_queue.sv for details
29 parameter bit FullBW = 1'b0,
30 /// Cut paths through the IP
31 parameter bit CutPath = 1'b0,
32 /// Disable checks, handle unsupported transfers as bypass
instead of reporting an error
33 parameter bit DisableChecks = 1'b0,
34 /// AXI Bus Types
35 parameter int unsigned AddrWidth = 32'd0,
36 parameter int unsigned DataWidth = 32'd0,
37 parameter int unsigned IdWidth = 32'd0,
38
39 /// Number of least-significant ID bits used by the internal
demux to track in-flight
40 /// transactions (0 <= AxilookBits <= IdWidth), see doc/
axi_demux.md for the trade-off.
41 parameter int unsigned AxilookBits = IdWidth,
42 parameter int unsigned UserWidth = 32'd0,
43 parameter type axi_req_t = logic,
44 parameter type axi_resp_t = logic,
45 parameter type axi_aw_chan_t = logic,
46 parameter type axi_w_chan_t = logic,
47 parameter type axi_b_chan_t = logic,
48 parameter type axi_ar_chan_t = logic,
49 parameter type axi_r_chan_t = logic
50 ) (
51 input logic clk_i,
52 input logic rst_ni,
53
54 // length
55 input axi_pkg::len_t len_limit_i,
56
57 // Input / Slave Port
58 input axi_req_t slv_req_i,
59 output axi_resp_t slv_resp_o,
60
61 // Output / Master Port
62 output axi_req_t mst_req_o,
63 input axi_resp_t mst_resp_i
64 );
65
66 // Demultiplex between supported and unsupported transac-
tions.
67 axi_req_t slv_req, act_req, unsupported_req;
68 axi_resp_t slv_resp, act_resp, unsupported_resp;
69
70 axi_multicut #(
71 .NoCuts ( CutPath ),
72 .aw_chan_t ( axi_aw_chan_t ),
73 .w_chan_t ( axi_w_chan_t ),
74 .b_chan_t ( axi_b_chan_t ),
75 .ar_chan_t ( axi_ar_chan_t ),
76 .r_chan_t ( axi_r_chan_t ),
77 .axi_req_t ( axi_req_t ),
78 .axi_resp_t ( axi_resp_t )
79 ) i_axi_multicut (
80 .clk_i,
81 .rst_ni,
82 .slv_req_i,
83 .slv_resp_o,
84 .mst_req_o ( slv_req ),
85 .mst_resp_i ( slv_resp )

```

```

84 );
85
86 logic sel_aw_unsupported, sel_ar_unsupported;
87 localparam int unsigned MaxTxns = (MaxReadTxns > Max-
WriteTxns) ? MaxReadTxns : MaxWriteTxns;
88 axi_demux_simple #(
89 .AxilookBits ( IdWidth ),
90 .axi_req_t ( axi_req_t ),
91 .axi_resp_t ( axi_resp_t ),
92 .NoMstPorts ( 2 ),
93 .MaxTxns ( MaxTxns ),
94 .AxilookBits ( AxilookBits )
95 ) i_demux_supported_vs_unsupported (
96 .clk_i,
97 .rst_ni,
98 .slv_req_i ( slv_req ),
99 .slv_aw_select_i ( sel_aw_unsupported ),
100 .slv_ar_select_i ( sel_ar_unsupported ),
101 .slv_resp_o ( slv_resp ),
102 .mst_reqs_o ( {unsupported_req, act_req} ),
103 .mst_resps_i ( {unsupported_resp, act_resp} )
104 );
105
106 // Define supported transactions.
107 function bit txn_supported(axi_pkg::atop_t atop,
axi_pkg::burst_t burst, axi_pkg::cache_t cache,
108 axi_pkg::len_t len, axi_pkg::len_t len_limit);
109
110 // if the splitter does not touch the transaction: allow
it
111 if (len <= len_limit) begin
112 return 1'b1;
113 end else begin
114
115 // Wrapping bursts are currently not supported to be
split
116 if (burst == axi_pkg::BURST_WRAP) return 1'b0;
117
118 // ATOP bursts are not supported.
119 if (atop != '0 & len > 0) return 1'b0;
120
121 // The AXI Spec (A3.4.1) only allows splitting non-modi-
fiable transactions ..
122 if (!axi_pkg::modifiable(cache)) begin
123 // .. if they are INCR bursts and longer than 16
beats.
124 return (burst == axi_pkg::BURST_INCR) & (len > 16);
125 end
126
127 // All other transactions are supported for splitting.
128 return 1'b1;
129 end
130 endfunction
131
132 assign sel_aw_unsupported = DisableChecks ? 1'b0 : ~txn_sup-
ported(slv_req.aw.atop,
133 slv_req.aw.burst, slv_req.aw.cache, slv_req.aw.len,
134 len_limit_i);
135
136 assign sel_ar_unsupported = DisableChecks ? 1'b0 : ~txn_sup-
ported('0, slv_req.ar.burst,
137 slv_req.ar.cache, slv_req.ar.len, len_limit_i);
138
139 // Respond to unsupported transactions with slave errors.
140 axi_err_slv #(
141 .AxilookBits ( IdWidth ),
142 .axi_req_t ( axi_req_t ),
143 .axi_resp_t ( axi_resp_t ),
144 .Resp ( axi_pkg::RESP_SLVERR ),
145 .ATOPs ( 1'b0 ), // The burst
splitter does not support ATOPs.
146 .MaxTxns ( 1 ) // Splitting
bursts implies a low-performance bus.
147 ) i_err_slv (
148 .clk_i,
149 .rst_ni,
150 .slv_req_i ( unsupported_req ),
151 .slv_resp_o ( unsupported_resp )
152 );
153
154 // -----
155 // AW Channel
156 // -----
157 logic w_cnt_dec, w_cnt_req, w_cnt_gnt, w_cnt_err;
158 axi_pkg::len_t w_cnt_len;
159 axi_burst_splitter_gran_ax_chan #(
160 .chan_t ( axi_aw_chan_t ),
161 .IdWidth ( IdWidth ),
162 .MaxTxns ( MaxWriteTxns ),
163 .CutPath ( CutPath ),
164 .FullBW ( FullBW )
165 ) i_axi_burst_splitter_gran_ax_chan (
166 .clk_i,
167 .rst_ni,
168 .len_limit_i,

```



```

173 .ax_i      ( act_req.aw      ),
174 .ax_valid_i ( act_req.aw_valid ),
175 .ax_ready_o ( act_resp.aw_ready ),
176 .ax_o      ( mst_req_o.aw   ),
177 .ax_valid_o ( mst_req_o.aw_valid ),
178 .ax_ready_i ( mst_resp_i.aw_ready ),
179 .cnt_id_i   ( mst_resp_i.b.id ),
180 .cnt_len_o  ( w_cnt_len     ),
181 .cnt_set_err_i ( mst_resp_i.b.resp[1] ),
182 .cnt_err_o  ( w_cnt_err     ),
183 .cnt_dec_i  ( w_cnt_dec     ),
184 .cnt_req_i  ( w_cnt_req     ),
185 .cnt_gnt_o  ( w_cnt_gnt     ),
186 );
187
188 // -----
189 // W Channel
190 // -----
191 // keep a state where we are in the fragmentation of the w
192 axi_pkg::len_t w_len_d, w_len_q;
193 logic          w_len_vld_q, w_len_vld_d;
194
195
196 // Feed through, except 'last', which needs to be modified
197 always_comb begin : proc_w_frag
198     mst_req_o.w      = act_req.w;
199     w_len_d          = w_len_q;
200     w_len_vld_d      = w_len_vld_q;
201     // the entire detection machine is only required if
202     len_limit > 0
203     if (len_limit_i != 8'h00) begin
204
205         // In the case we are in the granular mode: last is from
206         req or when w_len valid and '0.
207         mst_req_o.w.last = (w_len_vld_q & (w_len_q == 8'h00)) |
208         act_req.w.last;
209         // only advance the machine if w ready and valid
210         if (act_resp.w_ready & act_req.w_valid) begin
211             // the counter is not yet valid, set it to valid and
212             initialize
213             if (!w_len_vld_q) begin
214                 w_len_vld_d = 1'b1;
215                 w_len_d     = len_limit_i - 8'h01;
216             end else begin
217                 w_len_d = w_len_q - 8'h01;
218                 // in the last case, reinitialize the counter
219                 if (w_len_q == 8'h00) begin
220                     w_len_d = len_limit_i;
221                 end
222             end
223             // final overwrite. if a downstream last comes, the
224             counter is invalid and set to 0
225             if (act_req.w.last) begin
226                 w_len_vld_d = 1'b0;
227                 w_len_d     = 8'h00;
228             end
229         end else begin
230             // we operate in the legacy mode -> every w is last
231             mst_req_o.w.last = 1'b1;
232         end
233     end
234
235     assign mst_req_o.w_valid = act_req.w_valid;
236     assign act_resp.w_ready = mst_resp_i.w_ready;
237
238 // -----
239 // B Channel
240 // -----
241 // Filter B response, except for the last one
242 enum logic {BReady, BWait} b_state_d, b_state_q;
243 logic b_err_d, b_err_q;
244 always_comb begin
245     mst_req_o.b_ready = 1'b0;
246     act_resp.b        = '0;
247     act_resp.b_valid  = 1'b0;
248     w_cnt_dec         = 1'b0;
249     w_cnt_req         = 1'b0;
250     b_err_d           = b_err_q;
251     b_state_d         = b_state_q;
252
253     unique case (b_state_q)
254         BReady: begin
255             if (mst_resp_i.b_valid) begin
256                 w_cnt_req = 1'b1;
257                 if (w_cnt_gnt) begin
258                     if (w_cnt_len < ({1'b0, len_limit_i} + 9'h001))
259                         begin
260                             act_resp.b = mst_resp_i.b;
261                             if (w_cnt_err) begin
262                                 act_resp.b.resp = axi_pkg::RESP_SLVERR;
263                             end
264                             act_resp.b_valid = 1'b1;
265                             w_cnt_dec       = 1'b1;
266                             if (act_req.b_ready) begin
267                                 mst_req_o.b_ready = 1'b1;
268                             end else begin
269                                 b_state_d = BWait;
270                                 b_err_d  = w_cnt_err;
271                             end
272                         end else begin
273                             b_state_d = BWait;
274                             b_err_d  = w_cnt_err;
275                         end
276                     end
277                 end
278             end
279         end
280         BWait: begin
281             act_resp.b = mst_resp_i.b;
282             if (b_err_q) begin
283                 act_resp.b.resp = axi_pkg::RESP_SLVERR;
284             end
285             act_resp.b_valid = 1'b1;
286             if (mst_resp_i.b_valid && act_req.b_ready) begin
287                 mst_req_o.b_ready = 1'b1;
288                 b_state_d         = BReady;
289             end
290         end
291         default: /*do nothing*/;
292     endcase
293 end
294
295 // -----
296 // AR Channel
297 // -----
298 // See description of 'ax_chan' module.
299 logic          r_cnt_dec, r_cnt_req, r_cnt_gnt;
300 axi_pkg::len_t r_cnt_len;
301 axi_burst_splitter_gran_ax_chan #(
302     .chan_t      ( axi_ar_chan_t ),
303     .IdWidth     ( IdWidth       ),
304     .MaxTxns     ( MaxReadTxns   ),
305     .CutPath     ( CutPath       ),
306     .FullBW      ( FullBW       ),
307 ) i_axi_burst_splitter_gran_ar_chan (
308     .clk_i,
309     .rst_ni,
310     .len_limit_i,
311     .ax_i      ( act_req.ar      ),
312     .ax_valid_i ( act_req.ar_valid ),
313     .ax_ready_o ( act_resp.ar_ready ),
314     .ax_o      ( mst_req_o.ar   ),
315     .ax_valid_o ( mst_req_o.ar_valid ),
316     .ax_ready_i ( mst_resp_i.ar_ready ),
317     .cnt_id_i   ( mst_resp_i.r.id ),
318     .cnt_len_o  ( r_cnt_len     ),
319     .cnt_set_err_i ( 1'b0      ),
320     .cnt_err_o  (               ),
321     .cnt_dec_i  ( r_cnt_dec     ),
322     .cnt_req_i  ( r_cnt_req     ),
323     .cnt_gnt_o  ( r_cnt_gnt     ),
324 );
325
326 // -----
327 // R Channel
328 // -----
329 // Reconstruct 'last', feed rest through.
330 logic r_last_d, r_last_q;
331 enum logic {RFeedthrough, RWait} r_state_d, r_state_q;
332 always_comb begin
333     r_cnt_dec         = 1'b0;
334     r_cnt_req         = 1'b0;
335     r_last_d          = r_last_q;
336     r_state_d         = r_state_q;
337     mst_req_o.r_ready = 1'b0;
338     act_resp.r        = mst_resp_i.r;
339     act_resp.r.last   = 1'b0;
340     act_resp.r_valid  = 1'b0;
341
342     unique case (r_state_q)
343         RFeedthrough: begin
344             // If downstream has an R beat and the R counters can
345             give us the remaining length of
346             // that burst, ...
347             if (mst_resp_i.r_valid) begin
348                 // if downstream is last
349                 if (mst_resp_i.r.last) begin
350                     r_cnt_req = 1'b1;
351                     if (r_cnt_gnt) begin
352                         r_last_d = (r_cnt_len < ({1'b0, len_limit_i} +
353                         9'h001));
354                         act_resp.r.last = r_last_d;
355                         // Decrement the counter.
356                         r_cnt_dec       = 1'b1;
357                         // Try to forward the beat upstream.
358                         act_resp.r_valid = 1'b1;
359                         if (act_req.r_ready) begin
360                             // Acknowledge downstream.
361                             mst_req_o.r_ready = 1'b1;
362                         end else begin
363                             // Wait for upstream to become ready.
364                             r_state_d = RWait;
365                         end
366                     end
367                 end else begin
368                     // downstream was not last, just a normal read to
369                     pass through
370                     r_last_d = 1'b0;
371                     act_resp.r.last = r_last_d;
372                     // Try to forward the beat upstream.
373                     act_resp.r_valid = 1'b1;
374                     if (act_req.r_ready) begin
375

```



```

365         // Acknowledge downstream.
366         mst_req_o.r_ready = 1'b1;
367     end else begin
368         // Wait for upstream to become ready.
369         r_state_d = RWait;
370     end
371 end
372 end
373 end
374 RWait: begin
375     act_resp.r.last = r_last_q;
376     act_resp.r_valid = mst_resp_i.r_valid;
377     if (mst_resp_i.r_valid && act_req.r_ready) begin
378         mst_req_o.r_ready = 1'b1;
379         r_state_d = RFeedthrough;
380     end
381 end
382 default: /*do nothing*/;
383 endcase
384 end
385
386 // -----
387 // Flip-Flops
388 // -----
389 `FF(b_err_q, b_err_d, 1'b0, clk_i, rst_ni)
390 `FF(b_state_q, b_state_d, BReady, clk_i, rst_ni)
391 `FF(r_last_q, r_last_d, 1'b0, clk_i, rst_ni)
392 `FF(r_state_q, r_state_d, RFeedthrough, clk_i, rst_ni)
393 `FF(w_len_q, w_len_d, 8'h00, clk_i, rst_ni)
394 `FF(w_len_vld_q, w_len_vld_d, 1'b0, clk_i, rst_ni)
395
396 // -----
397 // Assumptions and assertions
398 // -----
399 `ifndef VERILATOR
400 `ifndef XILINX_SIMULATOR
401 // pragma translate_off
402 default disable iff (!rst_ni);
403 // Inputs
404 assume property (@(posedge clk_i) slv_req_i.aw_valid |->
405     txn_supported(slv_req_i.aw.atop, slv_req_i.aw.burst,
406         slv_req_i.aw.cache, slv_req_i.aw.len,
407         len_limit_i)
408     ) else $warning("Unsupported AW transaction received, re-
409         turning slave error!");
410     assume property (@(posedge clk_i) slv_req_i.ar_valid |->
411         txn_supported('0, slv_req_i.ar.burst,
412         slv_req_i.ar.cache, slv_req_i.ar.len, len_limit_i)
413     ) else $warning("Unsupported AR transaction received, re-
414         turning slave error!");
415 // Outputs
416 assert property (@(posedge clk_i) mst_req_o.aw_valid |->
417     mst_req_o.aw.len <= len_limit_i)
418     else $fatal(1, "AW burst longer than a single beat emit-
419         ted!");
420     assert property (@(posedge clk_i) mst_req_o.ar_valid |->
421     mst_req_o.ar.len <= len_limit_i)
422     else $fatal(1, "AR burst longer than a single beat emit-
423         ted!");
424     initial begin
425         assume (AxilookBits > 0 && AxilookBits <= IdWidth) else
426             $fatal(1, "AxilookBits (%0d) must be in [0, IdWidth
427             (%0d)]", AxilookBits, IdWidth);
428     end
429 // pragma translate_on
430 `endif
431 `endif
432 endmodule
433
434 // Internal module of [axi_burst_splitter_gran](mod-
435 ule.axi_burst_splitter_gran) to control
436 // Ax channels.
437 //
438 // Store burst lengths in counters, which are associated to
439 AXI IDs through ID queues (to allow
440 // reordering of responses w.r.t. requests).
441 module axi_burst_splitter_gran_ax_chan #(
442     parameter type chan_t = logic,
443     parameter int unsigned IdWidth = 32'd0,
444     parameter int unsigned MaxTxns = 32'd0,
445     parameter bit CutPath = 1'b0,
446     parameter bit FullBW = 1'b0,
447     parameter type id_t = logic[IdWidth-1:0]
448 ) (
449     input logic clk_i,
450     input logic rst_ni,
451
452     // length
453     input axi_pkg::len_t len_limit_i,
454
455     input chan_t ax_i,
456     input logic ax_valid_i,
457     output logic ax_ready_o,
458     output chan_t ax_o,
459     output logic ax_valid_o,
460     input logic ax_ready_i,
461
462     input id_t cnt_id_i,

```

```

455     output axi_pkg::len_t cnt_len_o,
456     input logic cnt_set_err_i,
457     output logic cnt_err_o,
458     input logic cnt_dec_i,
459     input logic cnt_req_i,
460     output logic cnt_gnt_o
461 );
462
463 typedef logic[IdWidth-1:0] cnt_id_t;
464 typedef logic[axi_pkg::LenWidth:0] num_beats_t;
465
466 chan_t ax_d, ax_q;
467 // keep the number of remaining beats. != len
468 num_beats_t num_beats_d, num_beats_q;
469 // maximum number of beats to subtract in one go
470 num_beats_t max_beats;
471
472 logic cnt_alloc_req, cnt_alloc_gnt;
473 axi_burst_splitter_gran_counters #(
474     .MaxTxns (MaxTxns),
475     .IdWidth (IdWidth),
476     .CutPath (CutPath),
477     .FullBW (FullBW)
478 ) i_axi_burst_splitter_gran_counters (
479     .clk_i,
480     .rst_ni,
481     .alloc_id_i (ax_i.id),
482     .alloc_len_i (ax_i.len),
483     .alloc_req_i (cnt_alloc_req),
484     .alloc_gnt_o (cnt_alloc_gnt),
485     .cnt_id_i (cnt_id_i),
486     .cnt_len_o (cnt_len_o),
487     .cnt_set_err_i (cnt_set_err_i),
488     .cnt_err_o (cnt_err_o),
489     .cnt_dec_i (cnt_dec_i),
490     .cnt_delta_i (max_beats),
491     .cnt_req_i (cnt_req_i),
492     .cnt_gnt_o (cnt_gnt_o)
493 );
494
495 // assign the max_beats depending on the limit value. Limit
496 value is given as an AXI len.
497 // limit = 0 means one beat each AX
498 assign max_beats = {1'b0, len_limit_i} + 9'h001;
499
500 enum logic {Idle, Busy} state_d, state_q;
501 always_comb begin
502     cnt_alloc_req = 1'b0;
503     ax_d = ax_q;
504     state_d = state_q;
505     num_beats_d = num_beats_q;
506     ax_o = '0;
507     ax_valid_o = 1'b0;
508     ax_ready_o = 1'b0;
509     unique case (state_q)
510         Idle: begin
511             if (ax_valid_i && cnt_alloc_gnt) begin
512                 // No splitting required -> feed through.
513                 if (ax_i.len <= len_limit_i) begin
514                     ax_o = ax_i;
515                     ax_valid_o = 1'b1;
516                     // As soon as downstream is ready, allocate a
517                     counter and acknowledge upstream.
518                     if (ax_ready_i) begin
519                         cnt_alloc_req = 1'b1;
520                         ax_ready_o = 1'b1;
521                     end
522                 end
523             end
524             // Splitting required.
525             end else begin
526                 // Store Ax, allocate a counter, and acknowledge
527                 upstream.
528                 ax_d = ax_i;
529                 cnt_alloc_req = 1'b1;
530                 ax_ready_o = 1'b1;
531                 // As burst is too long, we will need to send mul-
532                 tiple
533                 state_d = Busy;
534                 num_beats_d = ({1'b0, ax_i.len} + 9'h001);
535                 // Try to feed first burst through.
536                 ax_o = ax_d;
537                 // if we are here we can send the length limit
538                 once for sure
539                 ax_o.len = len_limit_i;
540                 ax_valid_o = 1'b1;
541                 if (ax_ready_i) begin
542                     // Reduce number of bursts still to be sent by
543                     one and increment address.
544                     num_beats_d = ({1'b0, ax_i.len} + 9'h001) -
545                     max_beats;
546                     if (ax_d.burst == axi_pkg::BURST_INCR) begin
547                         // Align
548                         ax_d.addr =
549                         axi_pkg::aligned_addr(axi_pkg::largest_addr_t'(ax_d.addr),
550                         ax_d.size);
551                         // modify the address
552                         ax_d.addr += (1 << ax_d.size) * max_beats;
553                     end
554                 end
555             end
556         end

```

```

547     end
548   end
549   end
550   Busy: begin
551     // Sent next burst from split.
552     ax_o      = ax_q;
553     ax_valid_o = 1'b1;
554     // emit the proper length
555     if (num_beats_q <= max_beats) begin
556       // this is the remainder
557       ax_o.len = axi_pkg::len_t'(num_beats_q - 9'h001);
558     end else begin
559       ax_o.len = len_limit_i;
560     end
561     // next state
562     if (ax_ready_i) begin
563       if (num_beats_q <= max_beats) begin
564         // If this was the last burst, go back to idle.
565         state_d = Idle;
566       end else begin
567         // Otherwise, continue with the next burst.
568         num_beats_d = num_beats_q - max_beats;
569         if (ax_q.burst == axi_pkg::BURST_INCR) begin
570           ax_d.addr =
axi_pkg::aligned_addr(axi_pkg::largest_addr_t'(ax_q.addr),
ax_q.size);
571           ax_d.addr += (1 << ax_q.size) * max_beats;
572         end
573       end
574     end
575   end
576   default: /*do nothing*/;
577 endcase
578 end
579 // registers
580 `FF(ax_q, ax_d, '0, clk_i, rst_ni)
581 `FF(state_q, state_d, Idle, clk_i, rst_ni)
582 `FF(num_beats_q, num_beats_d, 9'h000, clk_i, rst_ni)
583 endmodule
584
585
586
587
588 /// Internal module of [axi_burst_splitter_gran](mod-
ule_axi_burst_splitter_gran) to order
589 /// transactions.
590 module axi_burst_splitter_gran_counters #(
591   parameter int unsigned MaxTxns = 32'd0,
592   parameter int unsigned IdWidth = 32'd0,
593   parameter bit      CutPath = 1'b0,
594   parameter bit      FullBW = 1'b0,
595   parameter type     id_t = logic [IdWidth-1:0],
596   parameter type     cnt_t = logic [axi_pkg::LenWidth:0]
597 ) (
598   input  logic      clk_i,
599   input  logic      rst_ni,
600
601   input  id_t        id_t,
602   input  axi_pkg::len_t alloc_len_i,
603   input  logic        alloc_req_i,
604   output logic        alloc_gnt_o,
605
606   input  id_t        cnt_id_i,
607   output axi_pkg::len_t cnt_len_o,
608   input  logic        cnt_set_err_i,
609   output logic        cnt_err_o,
610   input  logic        cnt_dec_i,
611   input  cnt_t        cnt_delta_i,
612   input  logic        cnt_req_i,
613   output logic        cnt_gnt_o
614 );
615
616 // the allocation interface can be cut
617 typedef struct packed {
618   id_t      id;
619   axi_pkg::len_t len;
620 } alloc_pld_t;
621
622 alloc_pld_t alloc_pld_in, alloc_pld_out;
623 logic      alloc_req;
624 logic      alloc_gnt;
625
626 assign alloc_pld_in.id = alloc_id_i;
627 assign alloc_pld_in.len = alloc_len_i;
628
629 if (CutPath) begin : gen_spill
630   cc_spill_register #(
631     .data_t ( alloc_pld_t ),
632     .Bypass ( 1'b0 )
633   ) i_spill_register_alloc (
634     .clk_i,
635     .rst_ni,
636     .clr_i ( 1'b0 ),
637     .valid_i ( alloc_req_i ),
638     .ready_o ( alloc_gnt_o ),
639     .data_i ( alloc_pld_in ),
640     .valid_o ( alloc_req ),
641     .ready_i ( alloc_gnt ),
642     .data_o ( alloc_pld_out )
643   );
644 end else begin : gen_no_spill

```

```

645   assign alloc_req = alloc_req_i;
646   assign alloc_gnt_o = alloc_gnt;
647   assign alloc_pld_out = alloc_pld_in;
648 end
649
650 localparam int unsigned CntIdxWidth = (MaxTxns > 1) ? $clog2(MaxTxns) : 32'd1;
651 typedef logic [CntIdxWidth-1:0] cnt_idx_t;
652 logic [MaxTxns-1:0] cnt_dec, cnt_free, cnt_set, err_d,
err_q, cnt_clr;
653 cnt_t cnt_inp;
654 cnt_t [MaxTxns-1:0] cnt_outp;
655 cnt_idx_t cnt_free_idx, cnt_r_idx;
656 for (genvar i = 0; i < MaxTxns; i++) begin : gen_cnt
657   cc_delta_counter #(
658     .Width ( $bits(cnt_t) )
659   ) i_cnt (
660     .clk_i,
661     .rst_ni,
662     .clr_i ( cnt_clr[i] ),
663     .en_i ( cnt_dec[i] ),
664     .load_i ( cnt_set[i] ),
665     .down_i ( 1'b1 ),
666     .delta_i ( cnt_delta_i ),
667     .d_i ( cnt_inp ),
668     .q_o ( cnt_outp[i] ),
669     .overflow_o ( cnt_clr[i] )
670   );
671   assign cnt_free[i] = (cnt_outp[i] == '0);
672 end
673 assign cnt_inp = {1'b0, alloc_pld_out.len} + 1;
674
675 cc_lzc #(
676   .Width ( MaxTxns ),
677   .Mode ( cc_pkg::LZC_TRAILING_ZERO_CNT )
678 ) i_lzc (
679   .in_i ( cnt_free ),
680   .cnt_o ( cnt_free_idx ),
681   .empty_o ( )
682 );
683
684 logic idq_inp_req, idq_inp_gnt,
idq_outp_gnt, idq_outp_valid, idq_outp_pop;
685
686 cc_id_queue #(
687   .IdWidth ( $bits(id_t) ),
688   .Capacity ( MaxTxns ),
689   .FullBW ( FullBW ),
690   .data_t ( cnt_idx_t )
691 ) i_idq (
692   .clk_i,
693   .rst_ni,
694   .clr_i ( 1'b0 ),
695   .inp_id_i ( alloc_pld_out.id ),
696   .inp_data_i ( cnt_free_idx ),
697   .inp_req_i ( idq_inp_req ),
698   .inp_gnt_o ( idq_inp_gnt ),
699   .exists_data_i ( '0 ),
700   .exists_mask_i ( '0 ),
701   .exists_req_i ( 1'b0 ),
702   .exists_o ( /* keep open */),
703   .exists_gnt_o ( /* keep open */),
704   .oup_id_i ( cnt_id_i ),
705   .oup_pop_i ( idq_outp_pop ),
706   .oup_req_i ( cnt_req_i ),
707   .oup_data_o ( cnt_r_idx ),
708   .oup_data_valid_o ( idq_outp_valid ),
709   .oup_gnt_o ( idq_outp_gnt ),
710   .full_o ( /* keep open */),
711   .empty_o ( /* keep open */)
712 );
713 assign idq_inp_req = alloc_req & alloc_gnt;
714 assign alloc_gnt = idq_inp_gnt & |cnt_free;
715 assign cnt_gnt_o = idq_outp_gnt & idq_outp_valid;
716 logic [8:0] read_len;
717 assign read_len = cnt_outp[cnt_r_idx] - 1;
718 assign cnt_len_o = read_len[7:0];
719
720 assign idq_outp_pop = cnt_req_i & cnt_gnt_o & cnt_dec_i &
(cnt_len_o < cnt_delta_i);
721 always_comb begin
722   cnt_dec = '0;
723   cnt_dec[cnt_r_idx] = cnt_req_i & cnt_gnt_o & cnt_dec_i;
724 end
725 always_comb begin
726   cnt_set = '0;
727   cnt_set[cnt_free_idx] = alloc_req & alloc_gnt;
728 end
729 always_comb begin
730   err_d = err_q;
731   cnt_err_o = err_q[cnt_r_idx];
732   if (cnt_req_i && cnt_gnt_o && cnt_set_err_i) begin
733     err_d[cnt_r_idx] = 1'b1;
734     cnt_err_o = 1'b1;
735   end
736   if (alloc_req && alloc_gnt) begin
737     err_d[cnt_free_idx] = 1'b0;
738   end
739 end
740
741 // registers
742 `FF(err_q, err_d, '0, clk_i, rst_ni)

```

```

743
744 `ifndef VERILATOR
745 // pragma translate_off
746 assume property (@(posedge clk_i) idq_oup_gnt |->
747 idq_oup_valid)
748   else $warning("Invalid output at ID queue, read not gran-
749 ted!");
750 // pragma translate_on
751 `endif
752 endmodule

```

4. pulp-platform_axi/src/axi_burst_unwrap.sv (656 lines)

```

1 // Copyright (c) 2023 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the Solder-
4 pad Hardware
5
6 // License, Version 0.51 (the "License"); you may not use this
7 file except in
8 // compliance with the License. You may obtain a copy of the
9 License at
10 // http://solderpad.org/licenses/SHL-0.51. Unless required by
11 applicable law
12 // or agreed to in writing, software, hardware and materials
13 distributed under
14 // this License is distributed on an "AS IS" BASIS, WITHOUT
15 WARRANTIES OR
16 // CONDITIONS OF ANY KIND, either express or implied. See the
17 License for the
18 // specific language governing permissions and limitations un-
19 der the License.
20 //
21 // Authors:
22 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
23 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
24 // - Nils Wistoff <nwistoff@iis.ee.ethz.ch>
25
26 `include "axi/typedef.svh"
27 `include "common_cells/registers.svh"
28
29 /// Splits wrapping AXI4 bursts into incremental bursts.
30 ///
31 /// ## Limitations
32 ///
33 /// - This module does not support atomic operations (ATOPs)
34 and responds to ATOPs with a slave
35 error. Place an [axi_atop_filter](mod-
36 ule.axi_atop_filter) before this module if upstream
37 modules can generate ATOPs.
38 module axi_burst_unwrap #(
39
40 // Maximum number of AXI read bursts outstanding at the same
41 time
42 parameter int unsigned MaxReadTxns = 32'd0,
43 // Maximum number of AXI write bursts outstanding at the
44 same time
45 parameter int unsigned MaxWriteTxns = 32'd0,
46 // AXI Bus Types
47 parameter int unsigned AddrWidth = 32'd0,
48 parameter int unsigned DataWidth = 32'd0,
49 parameter int unsigned IdWidth = 32'd0,
50 // Number of least-significant ID bits used by the internal
51 demux to track in-flight
52 // transactions (0 < AxiLookBits <= IdWidth), see doc/
53 axi_demux.md for the trade-off.
54 parameter int unsigned AxiLookBits = IdWidth,
55 parameter int unsigned UserWidth = 32'd0,
56 parameter type axi_req_t = logic,
57 parameter type axi_resp_t = logic
58 ) (
59 input logic clk_i,
60 input logic rst_ni,
61
62 // Input / Slave Port
63 input axi_req_t slv_req_i,
64 output axi_resp_t slv_resp_o,
65
66 // Output / Master Port
67 output axi_req_t mst_req_o,
68 input axi_resp_t mst_resp_i
69 );
70
71 typedef logic [AddrWidth-1:0] addr_t;
72 typedef logic [DataWidth-1:0] data_t;
73 typedef logic [IdWidth-1:0] id_t;
74 typedef logic [DataWidth/8-1:0] strb_t;
75 typedef logic [UserWidth-1:0] user_t;
76
77 `AXI_TYPEDEF_AW_CHAN_T(axi_req_t, addr_t, id_t, user_t)
78 `AXI_TYPEDEF_W_CHAN_T(axi_req_t, data_t, strb_t, user_t)
79 `AXI_TYPEDEF_B_CHAN_T(axi_resp_t, id_t, user_t)
80 `AXI_TYPEDEF_AR_CHAN_T(axi_resp_t, addr_t, id_t, user_t)
81 `AXI_TYPEDEF_R_CHAN_T(axi_resp_t, data_t, id_t, user_t)
82
83 // Demultiplex between supported and unsupported transac-
84 tions.
85 axi_req_t act_req, unsupported_req;
86 axi_resp_t act_resp, unsupported_resp;
87 logic sel_aw_unsupported, sel_ar_unsupported;

```

```

70 localparam int unsigned MaxTxns = (MaxReadTxns > Max-
71 WriteTxns) ? MaxReadTxns : MaxWriteTxns;
72
73 axi_demux #(
74   .AxiIdWidth ( IdWidth ),
75   .aw_chan_t ( axi_req_t ),
76   .w_chan_t ( w_chan_t ),
77   .b_chan_t ( b_chan_t ),
78   .ar_chan_t ( ar_chan_t ),
79   .r_chan_t ( r_chan_t ),
80   .axi_req_t ( axi_req_t ),
81   .axi_resp_t ( axi_resp_t ),
82   .NoMstPorts ( 2 ),
83   .MaxTrans ( MaxTxns ),
84   .AxiLookBits ( AxiLookBits ),
85   .SpillAw ( 1'b0 ),
86   .SpillW ( 1'b0 ),
87   .SpillB ( 1'b0 ),
88   .SpillAr ( 1'b0 ),
89   .SpillR ( 1'b0 )
90 ) i_demux_supported_vs_unsupported (
91   .clk_i,
92   .rst_ni,
93   .slv_req_i,
94   .slv_aw_select_i ( sel_aw_unsupported ),
95   .slv_ar_select_i ( sel_ar_unsupported ),
96   .slv_resp_o,
97   .mst_reqs_o ( {unsupported_req, act_req} ),
98   .mst_resps_i ( {unsupported_resp, act_resp} )
99 );
100 // Define supported transactions.
101 function bit txn_supported(axi_pkg::atop_t atop,
102 axi_pkg::burst_t burst, axi_pkg::cache_t cache,
103 axi_pkg::len_t len);
104 // Single-beat transactions do not need splitting, so all
105 are supported.
106 if (len == '0) return 1'b1;
107 // ATOPs are not supported.
108 if (atop != '0) return 1'b0;
109 // The AXI Spec (A3.4.1) only allows splitting non-modifi-
110 cable transactions ..
111 // ... but this module only splits WRAP bursts, so ignore
112 all others
113 if (!axi_pkg::modifiable(cache) && (burst ==
114 axi_pkg::BURST_WRAP)) begin
115   return 1'b0;
116 end
117 // All other transactions are supported.
118 return 1'b1;
119 endfunction
120 assign sel_aw_unsupported = ~txn_supper-
121 ted(slv_req_i.aw.atop, slv_req_i.aw.burst,
122 slv_req_i.ar.cache, slv_req_i.ar.len);
123 assign sel_ar_unsupported = ~txn_supported('0,
124 slv_req_i.ar.burst,
125 slv_req_i.ar.cache, slv_req_i.ar.len);
126 // Respond to unsupported transactions with slave errors.
127 axi_err_slv #(
128   .AxiIdWidth ( IdWidth ),
129   .axi_req_t ( axi_req_t ),
130   .axi_resp_t ( axi_resp_t ),
131   .Resp ( axi_pkg::RESP_SLVERR ),
132   .ATOPs ( 1'b0 ), // The burst
133   .splitter_does_not_support_ATOPs (
134     .MaxTrans ( 1 ) // Splitting
135     bursts implies a low-performance bus.
136   )
137 ) i_err_slv (
138   .clk_i,
139   .rst_ni,
140   .slv_req_i ( unsupported_req ),
141   .slv_resp_o ( unsupported_resp )
142 );
143
144 // -----
145 // AW Channel
146 // -----
147 logic b_cnt_dec, b_cnt_req, b_cnt_gnt, b_cnt_err;
148 logic w_cnt_dec, w_cnt_req, w_cnt_gnt;
149 axi_pkg::len_t b_cnt_len, w_cnt_len;
150 axi_burst_unwrap_ax_chan #(
151   .AwChan ( 1'b1 ),
152   .chan_t ( axi_req_t ),
153   .AddrWidth ( AddrWidth ),
154   .IdWidth ( IdWidth ),
155   .MaxTxns ( MaxWriteTxns )
156 ) i_axi_burst_unwrap_ax_chan (
157   .clk_i,
158   .rst_ni,
159   .ax_i ( act_req.aw ),
160   .ax_valid_i ( act_req.aw_valid ),
161   .ax_ready_o ( act_resp.aw_ready ),
162   .ax_o ( mst_req_o.aw ),
163   .ax_valid_o ( mst_req_o.aw_valid ),
164   .ax_ready_i ( mst_resp_i.aw_ready ),
165   .cnt_id_i ( mst_resp_i.b.id ),
166   .cnt_len_o ( {w_cnt_len, b_cnt_len} ),
167   .cnt_set_err_i ( mst_resp_i.b.resp[1] ),
168   .cnt_err_o ( b_cnt_err ),
169   .cnt_dec_i ( {w_cnt_dec, b_cnt_dec} ),
170   .cnt_req_i ( {w_cnt_req, b_cnt_req} ),

```

```

159 .cnt_gnt_o      ({ w_cnt_gnt      , b_cnt_gnt      })
160 );
161 // -----
162 // W Channel
163 // -----
164 // Set `last` flag in last beat of downstream burst
165 logic w_last_d, w_last_q;
166 enum logic [1:0] {WReady, WWait, WFeedthrough} w_state_d,
167 w_state_q;
168 always_comb begin
169     w_cnt_dec      = 1'b0;
170     w_cnt_req      = 1'b0;
171     w_last_d       = w_last_q;
172     w_state_d      = w_state_q;
173     mst_req_o.w_valid = 1'b0;
174     mst_req_o.w      = act_req.w;
175     act_resp.w_ready = 1'b0;
176
177     unique case (w_state_q)
178     WReady: begin
179         if (act_req.w_valid) begin
180             w_cnt_req = 1'b1;
181             if (w_cnt_gnt) begin
182                 w_last_d = act_req.w.last | (w_cnt_len == 8'd0);
183                 mst_req_o.w.last = w_last_d;
184                 w_cnt_dec = 1'b1;
185                 // Try to forward the beat downstream.
186                 mst_req_o.w_valid = 1'b1;
187                 if (mst_resp.i.w_ready) begin
188                     act_resp.w_ready = 1'b1;
189                     if (w_last_d && !act_req.w.last) begin
190                         w_state_d = WFeedthrough;
191                     end
192                 end else begin
193                     w_state_d = WWait;
194                 end
195             end // if (w_cnt_gnt)
196         end // if (act_req.w_valid)
197     end // case: WReady
198     WWait: begin
199         mst_req_o.w.last = w_last_q;
200         mst_req_o.w_valid = 1'b1;
201         if (mst_resp.i.w_ready) begin
202             act_resp.w_ready = 1'b1;
203             w_state_d = (!w_last_q || act_req.w.last) ?
204             WReady : WFeedthrough;
205         end
206     end
207     WFeedthrough: begin
208         // Feed through second incremental burst.
209         mst_req_o.w_valid = act_req.w_valid;
210         act_resp.w_ready = mst_resp.i.w_ready;
211         if (act_req.w_valid && mst_resp.i.w_ready &&
212         act_req.w.last) begin
213             w_state_d = WReady;
214         end
215     end
216     default: /*do nothing*/;
217 endcase // unique case (w_state_q)
218 end
219 // -----
220 // B Channel
221 // -----
222 // Filter B response, except for the last one
223 enum logic {BReady, BWait} b_state_d, b_state_q;
224 logic b_err_d, b_err_q;
225 always_comb begin
226     mst_req_o.b_ready = 1'b0;
227     act_resp.b        = 1'b0;
228     act_resp.b_valid  = 1'b0;
229     b_cnt_dec         = 1'b0;
230     b_cnt_req         = 1'b0;
231     b_err_d           = b_err_q;
232     b_state_d         = b_state_q;
233
234     unique case (b_state_q)
235     BReady: begin
236         if (mst_resp.i.b_valid) begin
237             b_cnt_req = 1'b1;
238             if (b_cnt_gnt) begin
239                 if (b_cnt_len == 8'd0) begin
240                     act_resp.b = mst_resp.i.b;
241                     if (b_cnt_err) begin
242                         act_resp.b_resp = axi_pkg::RESP_SLVERR;
243                     end
244                     act_resp.b_valid = 1'b1;
245                     b_cnt_dec = 1'b1;
246                     if (act_req.b_ready) begin
247                         mst_req_o.b_ready = 1'b1;
248                     end else begin
249                         b_state_d = BWait;
250                         b_err_d = b_cnt_err;
251                     end
252                 end else begin
253                     mst_req_o.b_ready = 1'b1;
254                     b_cnt_dec = 1'b1;
255                 end
256             end
257         end
258     end
259     BWait: begin
260         act_resp.b = mst_resp.i.b;
261         if (b_err_q) begin
262             act_resp.b_resp = axi_pkg::RESP_SLVERR;
263         end
264         act_resp.b_valid = 1'b1;
265         if (mst_resp.i.b_valid && act_req.b_ready) begin
266             mst_req_o.b_ready = 1'b1;
267             b_state_d = BReady;
268         end
269         default: /*do nothing*/;
270     endcase
271 end
272 // -----
273 // AR Channel
274 // -----
275 // See description of `ax_chan` module.
276 logic r_cnt_dec, r_cnt_req, r_cnt_gnt, unc0;
277 axi_pkg::len_t r_cnt_len, unc1;
278 axi_burst_unwrap_ax_chan #(
279     .AwChan ( 0 ),
280     .chan_t ( ar_chan_t ),
281     .Addrwidth ( Addrwidth ),
282     .Idwidth ( Idwidth ),
283     .MaxTxns ( MaxReadTxns )
284 ) i_axi_burst_unwrap_ar_chan (
285     .clk_i,
286     .rst_ni,
287     .ax_i ( act_req.ar ),
288     .ax_valid_i ( act_req.ar_valid ),
289     .ax_ready_o ( act_resp.ar_ready ),
290     .ax_o ( mst_req_o.ar ),
291     .ax_valid_o ( mst_req_o.ar_valid ),
292     .ax_ready_i ( mst_resp.i.ar_ready ),
293     .cnt_id_i ( mst_resp.i.r.id ),
294     .cnt_len_o ( { unc1 , r_cnt_len } ),
295     .cnt_set_err_i ( 1'b0 ),
296     .cnt_err_o ( ),
297     .cnt_dec_i ( { 1'b0 , r_cnt_dec } ),
298     .cnt_req_i ( { 1'b0 , r_cnt_req } ),
299     .cnt_gnt_o ( { unc0 , r_cnt_gnt } )
300 );
301 // -----
302 // R Channel
303 // -----
304 // Reconstruct `last`, feed rest through.
305 logic r_last_d, r_last_q;
306 enum logic {RFeedthrough, RWait} r_state_d, r_state_q;
307 always_comb begin
308     r_cnt_dec      = 1'b0;
309     r_cnt_req      = 1'b0;
310     r_last_d       = r_last_q;
311     r_state_d      = r_state_q;
312     mst_req_o.r_ready = 1'b0;
313     act_resp.r       = mst_resp.i.r;
314     act_resp.r.last  = 1'b0;
315     act_resp.r_valid = 1'b0;
316
317     unique case (r_state_q)
318     RFeedthrough: begin
319         // If downstream has an R beat and the R counters can
320         // give us the remaining length of
321         // that burst, ...
322         if (mst_resp.i.r_valid) begin
323             r_cnt_req = 1'b1;
324             if (r_cnt_gnt) begin
325                 r_last_d = (r_cnt_len == 8'd0);
326                 act_resp.r.last = r_last_d;
327                 // Decrement the counter.
328                 r_cnt_dec = 1'b1;
329                 // Try to forward the beat upstream.
330                 act_resp.r_valid = 1'b1;
331                 if (act_req.r_ready) begin
332                     // Acknowledge downstream.
333                     mst_req_o.r_ready = 1'b1;
334                 end else begin
335                     // Wait for upstream to become ready.
336                     r_state_d = RWait;
337                 end
338             end
339         end
340     end
341     RWait: begin
342         act_resp.r.last = r_last_q;
343         act_resp.r_valid = mst_resp.i.r_valid;
344         if (mst_resp.i.r_valid && act_req.r_ready) begin
345             mst_req_o.r_ready = 1'b1;
346             r_state_d = RFeedthrough;
347         end
348         default: /*do nothing*/;
349     endcase
350 end
351 // -----
352 // Flip-Flops
353 // -----

```

```

257 end
258 BWait: begin
259     act_resp.b = mst_resp.i.b;
260     if (b_err_q) begin
261         act_resp.b_resp = axi_pkg::RESP_SLVERR;
262     end
263     act_resp.b_valid = 1'b1;
264     if (mst_resp.i.b_valid && act_req.b_ready) begin
265         mst_req_o.b_ready = 1'b1;
266         b_state_d = BReady;
267     end
268     default: /*do nothing*/;
269 endcase
270 end
271 // -----
272 // AR Channel
273 // -----
274 // See description of `ax_chan` module.
275 logic r_cnt_dec, r_cnt_req, r_cnt_gnt, unc0;
276 axi_pkg::len_t r_cnt_len, unc1;
277 axi_burst_unwrap_ax_chan #(
278     .AwChan ( 0 ),
279     .chan_t ( ar_chan_t ),
280     .Addrwidth ( Addrwidth ),
281     .Idwidth ( Idwidth ),
282     .MaxTxns ( MaxReadTxns )
283 ) i_axi_burst_unwrap_ar_chan (
284     .clk_i,
285     .rst_ni,
286     .ax_i ( act_req.ar ),
287     .ax_valid_i ( act_req.ar_valid ),
288     .ax_ready_o ( act_resp.ar_ready ),
289     .ax_o ( mst_req_o.ar ),
290     .ax_valid_o ( mst_req_o.ar_valid ),
291     .ax_ready_i ( mst_resp.i.ar_ready ),
292     .cnt_id_i ( mst_resp.i.r.id ),
293     .cnt_len_o ( { unc1 , r_cnt_len } ),
294     .cnt_set_err_i ( 1'b0 ),
295     .cnt_err_o ( ),
296     .cnt_dec_i ( { 1'b0 , r_cnt_dec } ),
297     .cnt_req_i ( { 1'b0 , r_cnt_req } ),
298     .cnt_gnt_o ( { unc0 , r_cnt_gnt } )
299 );
300 // -----
301 // R Channel
302 // -----
303 // Reconstruct `last`, feed rest through.
304 logic r_last_d, r_last_q;
305 enum logic {RFeedthrough, RWait} r_state_d, r_state_q;
306 always_comb begin
307     r_cnt_dec      = 1'b0;
308     r_cnt_req      = 1'b0;
309     r_last_d       = r_last_q;
310     r_state_d      = r_state_q;
311     mst_req_o.r_ready = 1'b0;
312     act_resp.r       = mst_resp.i.r;
313     act_resp.r.last  = 1'b0;
314     act_resp.r_valid = 1'b0;
315
316     unique case (r_state_q)
317     RFeedthrough: begin
318         // If downstream has an R beat and the R counters can
319         // give us the remaining length of
320         // that burst, ...
321         if (mst_resp.i.r_valid) begin
322             r_cnt_req = 1'b1;
323             if (r_cnt_gnt) begin
324                 r_last_d = (r_cnt_len == 8'd0);
325                 act_resp.r.last = r_last_d;
326                 // Decrement the counter.
327                 r_cnt_dec = 1'b1;
328                 // Try to forward the beat upstream.
329                 act_resp.r_valid = 1'b1;
330                 if (act_req.r_ready) begin
331                     // Acknowledge downstream.
332                     mst_req_o.r_ready = 1'b1;
333                 end else begin
334                     // Wait for upstream to become ready.
335                     r_state_d = RWait;
336                 end
337             end
338         end
339     end
340     RWait: begin
341         act_resp.r.last = r_last_q;
342         act_resp.r_valid = mst_resp.i.r_valid;
343         if (mst_resp.i.r_valid && act_req.r_ready) begin
344             mst_req_o.r_ready = 1'b1;
345             r_state_d = RFeedthrough;
346         end
347         default: /*do nothing*/;
348     endcase
349 end
350 // -----
351 // Flip-Flops
352 // -----

```

```

357 `FF(b_err_q, b_err_d, 1'b0, clk_i, rst_ni)
358 `FF(b_state_q, b_state_d, BReady, clk_i, rst_ni)
359 `FF(r_last_q, r_last_d, 1'b0, clk_i, rst_ni)
360 `FF(r_state_q, r_state_d, RFeedthrough, clk_i, rst_ni)
361 `FF(w_last_q, w_last_d, 1'b0, clk_i, rst_ni)
362 `FF(w_state_q, w_state_d, WReady, clk_i, rst_ni)
363
364 // -----
365 // Assumptions and assertions
366 // -----
367 `ifndef VERILATOR
368 `ifndef XILINX_SIMULATOR
369 // pragma translate_off
370 default disable iff (!rst_ni);
371 // Inputs
372 assume property (@(posedge clk_i) slv_req_i.aw_valid |->
373   txn_supported(slv_req_i.aw.atop, slv_req_i.aw.burst,
slv_req_i.aw.cache, slv_req_i.aw.len)
374   ) else $warning("Unsupported AW transaction received, re-
turning slave error!");
375   assume property (@(posedge clk_i) slv_req_i.ar_valid |->
376     txn_supported('0, slv_req_i.ar.burst,
slv_req_i.ar.cache, slv_req_i.ar.len)
377     ) else $warning("Unsupported AR transaction received, re-
turning slave error!");
378   assume property (@(posedge clk_i) slv_req_i.aw_valid |->
379     slv_req_i.aw.atop == '0 || slv_req_i.aw.atop[5:4] ==
axi_pkg::ATOP_ATOMICSTORE
380     ) else $fatal(1, "Unsupported ATOP that gives rise to a R
response received,\
381       cannot respond in protocol-compliant
manner!");
382 // pragma translate_on
383 `endif
384 `endif
385
386 endmodule
387
388 /// Internal module of [`axi_burst_splitter`](mod-
ule.axi_burst_splitter) to control Ax channels.
389 ///
390 /// Store burst lengths in counters, which are associated to
AXI IDs through ID queues (to allow
391 /// reordering of responses w.r.t. requests).
392 module axi_burst_unwrap_ax_chan #(
393   parameter bit      AwChan = 0,
394   parameter type     chan_t = logic,
395   parameter int unsigned AddrWidth = 0,
396   parameter int unsigned IdWidth = 0,
397   parameter int unsigned MaxTxns = 0,
398   parameter type     id_t = logic[IdWidth-1:0]
399 ) (
400   input  logic      clk_i,
401   input  logic      rst_ni,
402
403   input  chan_t      ax_i,
404   input  logic      ax_valid_i,
405   output logic      ax_ready_o,
406   output chan_t      ax_o,
407   output logic      ax_valid_o,
408   input  logic      ax_ready_i,
409
410   input  id_t        cnt_id_i,
411   output axi_pkg::len_t [1:0] cnt_len_o,
412   input  logic      cnt_set_err_i,
413   output logic      cnt_err_o,
414   input  logic      [1:0] cnt_dec_i,
415   input  logic      [1:0] cnt_req_i,
416   output logic      [1:0] cnt_gnt_o
417 );
418 typedef logic[IdWidth-1:0] cnt_id_t;
419
420 logic [1:0] cnt_alloc_req, cnt_alloc_gnt;
421 axi_pkg::len_t split_len, alloc_len_0;
422
423 // Count down from upstream burst len to generate upstream
last flag.
424 axi_burst_counters #(
425   .MaxTxns ( MaxTxns ),
426   .IdWidth ( IdWidth )
427 ) i_axi_burst_counters0 (
428   .clk_i,
429   .rst_ni,
430   .alloc_id_i ( ax_i.id ),
431   .alloc_len_i ( alloc_len_0 ),
432   .alloc_req_i ( cnt_alloc_req[0] ),
433   .alloc_gnt_o ( cnt_alloc_gnt[0] ),
434   .cnt_id_i ( cnt_id_i ),
435   .cnt_len_o ( cnt_len_o[0] ),
436   .cnt_set_err_i ( cnt_set_err_i ),
437   .cnt_err_o ( cnt_err_o ),
438   .cnt_dec_i ( cnt_dec_i[0] ),
439   .cnt_req_i ( cnt_req_i[0] ),
440   .cnt_gnt_o ( cnt_gnt_o[0] )
441 );
442
443 // For AR we need to count the r responses, for AW we need
to count downstream b responses
444 assign alloc_len_0 = AwChan ? split_len : ax_i.len;
445
446 // For AwChan, count down W beats until end of first incr

```

```

burst to set last flag.
447 // Since the W channel is ordered and has no IDs, just use
the same ID for all transactions to
448 // implement a FIFO. TODO: replace internal ID queue with
FIFO
449 axi_burst_counters #(
450   .MaxTxns ( MaxTxns ),
451   .IdWidth ( 1 )
452 ) i_axi_burst_counters1 (
453   .clk_i,
454   .rst_ni,
455   .alloc_id_i ( 1'b0 ),
456   .alloc_len_i ( ax_o.len ),
457   .alloc_req_i ( cnt_alloc_req[1] ),
458   .alloc_gnt_o ( cnt_alloc_gnt[1] ),
459   .cnt_id_i ( 1'b0 ),
460   .cnt_len_o ( cnt_len_o[1] ),
461   .cnt_set_err_i ( 1'b0 ),
462   .cnt_err_o ( ),
463   .cnt_dec_i ( cnt_dec_i[1] ),
464   .cnt_req_i ( cnt_req_i[1] ),
465   .cnt_gnt_o ( cnt_gnt_o[1] )
466 );
467
468 chan_t ax_d, ax_q;
469
470 logic [10:0] container_size;
471 logic [AddrWidth-1:0] wrap_boundary;
472
473 // The total size of this burst (beat_size * burst_length)
474 assign container_size = (ax_i.len + 1) << ax_i.size;
475 // For wrapping bursts, this returns the wrap boundary (con-
tainer
size is power of two according to A.3.4.1)
476 assign wrap_boundary = ax_i.addr & ~(Ad-
drWidth'(container_size) - 1);
477
478 enum logic {Idle, Busy} state_d, state_q;
479 always_comb begin
480   cnt_alloc_req = '0;
481   ax_d = ax_q;
482   state_d = state_q;
483   ax_o = '0;
484   ax_valid_o = 1'b0;
485   ax_ready_o = 1'b0;
486   split_len = 8'd0;
487   unique case (state_q)
488     Idle: begin
489       if (ax_valid_i && cnt_alloc_gnt) begin
490         if (ax_i.burst == axi_pkg::BURST_WRAP && ax_i.addr !
= wrap_boundary) begin // Splitting required.
491           // Store Ax, allocate a counter, and acknowledge
upstream.
492             ax_d = ax_i;
493             ax_d.burst = axi_pkg::BURST_INCR;
494             split_len = 8'd1;
495             // Try to feed first burst through.
496             ax_o = ax_d;
497             // First (this) incr burst from addr to wrap
boundary + container size
498             ax_o.len = ((wrap_boundary + container_size -
ax_i.addr) >> ax_i.size) - 1;
499             // Next incr burst from wrap boundary to addr
500             ax_d.len = ((ax_i.addr - wrap_boundary) >>
ax_i.size) - 1;
501             ax_d.addr = wrap_boundary;
502             ax_valid_o = 1'b1;
503             if (ax_ready_i) begin
504               ax_ready_o = 1'b1;
505               state_d = Busy;
506               // Allocate second counter only for AwChan
cnt_alloc_req = { AwChan, 1'b1 };
507             end
508             end else begin // No splitting required -> feed
through.
509               ax_o = ax_i;
510               // A wrapping burst starting on the wrap boundary
maps directly to an incrementing burst.
511               if (ax_i.burst == axi_pkg::BURST_WRAP) begin
512                 ax_o.burst = axi_pkg::BURST_INCR;
513               end
514               ax_valid_o = 1'b1;
515               // As soon as downstream is ready, allocate a
counter and acknowledge upstream.
516               if (ax_ready_i) begin
517                 cnt_alloc_req = { AwChan, 1'b1 };
518                 ax_ready_o = 1'b1;
519               end
520             end
521             end
522             end
523             end
524             Busy: begin
525               // Sent next burst from split.
526               ax_o = ax_q;
527               ax_valid_o = 1'b1;
528               if (ax_ready_i) begin
529                 state_d = Idle;
530               end
531             end
532             default: /*do nothing*/;
533           endcase
534         end

```



```

535
536 // registers
537 `FF(ax_q, ax_d, '0, clk_i, rst_ni)
538 `FF(state_q, state_d, Idle, clk_i, rst_ni)
539 endmodule
540
541 /// Internal module of ['axi_burst_splitter'](module-
ule.axi_burst_splitter) to order transactions.
542 module axi_burst_counters #(
543     parameter int unsigned MaxTxns = 0,
544     parameter int unsigned IdWidth = 0,
545     parameter type id_t = logic [IdWidth-1:0]
546 ) (
547     input logic clk_i,
548     input logic rst_ni,
549
550     input id_t alloc_id_i,
551     input axi_pkg::len_t alloc_len_i,
552     input logic alloc_req_i,
553     output logic alloc_gnt_o,
554
555     input id_t cnt_id_i,
556     output axi_pkg::len_t cnt_len_o,
557     input logic cnt_set_err_i,
558     output logic cnt_err_o,
559     input logic cnt_dec_i,
560     input logic cnt_req_i,
561     output logic cnt_gnt_o
562 );
563 localparam int unsigned CntIdxWidth = (MaxTxns > 1) ? $clog2(MaxTxns) : 32'd1;
564 typedef logic [CntIdxWidth-1:0] cnt_idx_t;
565 typedef logic [$bits(axi_pkg::len_t):0] cnt_t;
566 logic [MaxTxns-1:0] cnt_dec, cnt_free, cnt_set, err_d,
err_q;
567 cnt_t cnt_inp;
568 cnt_t [MaxTxns-1:0] cnt_outp;
569 cnt_idx_t cnt_free_idx, cnt_r_idx;
570 for (genvar i = 0; i < MaxTxns; i++) begin : gen_cnt
571     cc_counter #(
572         .Width ( $bits(cnt_t) )
573     ) i_cnt (
574         .clk_i,
575         .rst_ni,
576         .clr_i ( 1'b0 ),
577         .en_i ( cnt_dec[i] ),
578         .load_i ( cnt_set[i] ),
579         .down_i ( 1'b1 ),
580         .d_i ( cnt_inp ),
581         .q_o ( cnt_outp[i] ),
582         .overflow_o ( ) // not used
583     );
584     assign cnt_free[i] = (cnt_outp[i] == '0);
585 end
586 assign cnt_inp = {1'b0, alloc_len_i} + 1;
587
588 cc_lzc #(
589     .Width ( MaxTxns ),
590     .Mode ( cc_pkg::LZC_TRAILING_ZERO_CNT ) // start count-
ing at index 0
591 ) i_lzc (
592     .in_i ( cnt_free ),
593     .cnt_o ( cnt_free_idx ),
594     .empty_o ( )
595 );
596
597 logic idq_inp_req, idq_inp_gnt,
598 idq_outp_gnt, idq_outp_valid, idq_outp_pop;
599 cc_id_queue #(
600     .IdWidth ( $bits(id_t) ),
601     .Capacity ( MaxTxns ),
602     .data_t ( cnt_idx_t )
603 ) i_idq (
604     .clk_i,
605     .rst_ni,
606     .clr_i ( 1'b0 ),
607     .inp_id_i ( alloc_id_i ),
608     .inp_data_i ( cnt_free_idx ),
609     .inp_req_i ( idq_inp_req ),
610     .inp_gnt_o ( idq_inp_gnt ),
611     .exists_data_i ( '0 ),
612     .exists_mask_i ( '0 ),
613     .exists_req_i ( 1'b0 ),
614     .exists_o ( /* keep open */ ),
615     .exists_gnt_o ( /* keep open */ ),
616     .oup_id_i ( cnt_id_i ),
617     .oup_pop_i ( idq_outp_pop ),
618     .oup_req_i ( cnt_req_i ),
619     .oup_data_o ( cnt_r_idx ),
620     .oup_data_valid_o ( idq_outp_valid ),
621     .oup_gnt_o ( idq_outp_gnt ),
622     .full_o ( /* keep open */ ),
623     .empty_o ( /* keep open */ )
624 );
625 assign idq_inp_req = alloc_req_i & alloc_gnt_o;
626 assign alloc_gnt_o = idq_inp_gnt & !(cnt_free);
627 assign cnt_gnt_o = idq_outp_gnt & idq_outp_valid;
628 logic [8:0] read_len;
629 assign read_len = cnt_outp[cnt_r_idx] - 1;
630 assign cnt_len_o = read_len[7:0];
631

```

```

632 assign idq_outp_pop = cnt_req_i & cnt_gnt_o & cnt_dec_i &
(cnt_len_o == 8'd0);
633 always_comb begin
634     cnt_dec = '0;
635     cnt_dec[cnt_r_idx] = cnt_req_i & cnt_gnt_o & cnt_dec_i;
636 end
637 always_comb begin
638     cnt_set = '0;
639     cnt_set[cnt_free_idx] = alloc_req_i & alloc_gnt_o;
640 end
641 always_comb begin
642     err_d = err_q;
643     cnt_err_o = err_q[cnt_r_idx];
644     if (cnt_req_i && cnt_gnt_o && cnt_set_err_i) begin
645         err_d[cnt_r_idx] = 1'b1;
646         cnt_err_o = 1'b1;
647     end
648     if (alloc_req_i && alloc_gnt_o) begin
649         err_d[cnt_free_idx] = 1'b0;
650     end
651 end
652
653 // registers
654 `FF(err_q, err_d, '0, clk_i, rst_ni)
655
656 endmodule

```

5. pulp-platform_axi/src/axi_bus_compare.sv (730 lines)

```

1 // Copyright (c) 2019-2020 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this license is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations under
the License.
11 //
12 // Authors:
13 // - Thomas Benz <tbenz@iis.ee.ethz.ch>
14
15 `include "axi/assign.svh"
16 /// Synthesizable test module comparing two AXI channels of
the same type
17 /// This module is meant to be used in FPGA-based verifica-
tion.
18 module axi_bus_compare #(
19     /// ID width of the AXI4+ATOP interface
20     parameter int unsigned AxiIdWidth = 32'd0,
21     /// FIFO depth
22     parameter int unsigned FifoDepth = 32'd0,
23     /// Consider size field in comparison
24     parameter bit UseSize = 1'b0,
25     /// Data width of the AXI4+ATOP interface
26     parameter int unsigned DataWidth = 32'd8,
27     /// AW channel type of the AXI4+ATOP interface
28     parameter type axi_aw_chan_t = logic,
29     /// W channel type of the AXI4+ATOP interface
30     parameter type axi_w_chan_t = logic,
31     /// B channel type of the AXI4+ATOP interface
32     parameter type axi_b_chan_t = logic,
33     /// AR channel type of the AXI4+ATOP interface
34     parameter type axi_ar_chan_t = logic,
35     /// R channel type of the AXI4+ATOP interface
36     parameter type axi_r_chan_t = logic,
37     /// Request struct type of the AXI4+ATOP slave port
38     parameter type axi_req_t = logic,
39     /// Response struct type of the AXI4+ATOP slave port
40     parameter type axi_rsp_t = logic,
41     /// ID type (*do not overwrite*)
42     parameter type id_t = logic [2**AxiIdWidth-1:0]
43 ) (
44     /// Clock
45     input logic clk_i,
46     /// Asynchronous reset, active low
47     input logic rst_ni,
48     /// AXI4+ATOP A channel request in
49     input axi_req_t axi_a_req_i,
50     /// AXI4+ATOP A channel response out
51     output axi_rsp_t axi_a_rsp_o,
52     /// AXI4+ATOP A channel request out
53     output axi_req_t axi_a_req_o,
54     /// AXI4+ATOP A channel response in
55     input axi_rsp_t axi_a_rsp_i,
56     /// AXI4+ATOP B channel request in
57     input axi_req_t axi_b_req_i,
58     /// AXI4+ATOP B channel response out
59     output axi_rsp_t axi_b_rsp_o,
60     /// AXI4+ATOP B channel request out

```

```

61 output axi_req_t axi_b_req_o,
62 /// AXI4+ATOP B channel response in
63 input axi_rsp_t axi_b_rsp_i,
64 /// AW mismatch
65 output id_t aw_mismatch_o,
66 /// W mismatch
67 output logic w_mismatch_o,
68 /// B mismatch
69 output id_t b_mismatch_o,
70 /// AR mismatch
71 output id_t ar_mismatch_o,
72 /// R mismatch
73 output id_t r_mismatch_o,
74 /// General mismatch
75 output logic mismatch_o,
76 /// Unit is busy
77 output logic busy_o
78 );
79
80
81 //-----
82 // Channel Signals
83 //-----
84 // assign request payload A
85
86 `AXI_ASSIGN_AW_STRUCT(axi_a_req_o.aw, axi_a_req_i.aw)
87 `AXI_ASSIGN_W_STRUCT(axi_a_req_o.w, axi_a_req_i.w)
88 `AXI_ASSIGN_AR_STRUCT(axi_a_req_o.ar, axi_a_req_i.ar)
89
90 // assign response payload A
91 `AXI_ASSIGN_R_STRUCT(axi_a_rsp_o.r, axi_a_rsp_i.r)
92 `AXI_ASSIGN_B_STRUCT(axi_a_rsp_o.b, axi_a_rsp_i.b)
93
94 // assign request payload B
95 `AXI_ASSIGN_AW_STRUCT(axi_b_req_o.aw, axi_b_req_i.aw)
96 `AXI_ASSIGN_W_STRUCT(axi_b_req_o.w, axi_b_req_i.w)
97 `AXI_ASSIGN_AR_STRUCT(axi_b_req_o.ar, axi_b_req_i.ar)
98
99 // assign response payload B
100 `AXI_ASSIGN_R_STRUCT(axi_b_rsp_o.r, axi_b_rsp_i.r)
101 `AXI_ASSIGN_B_STRUCT(axi_b_rsp_o.b, axi_b_rsp_i.b)
102
103 // fifo handshaking signals A
104 id_t fifo_valid_aw_a, fifo_ready_aw_a;
105 id_t fifo_valid_b_a, fifo_ready_b_a;
106 id_t fifo_valid_ar_a, fifo_ready_ar_a;
107 id_t fifo_valid_r_a, fifo_ready_r_a;
108
109 logic fifo_sel_valid_aw_a, fifo_sel_ready_aw_a;
110 logic fifo_sel_valid_w_a, fifo_sel_ready_w_a;
111 logic fifo_sel_valid_b_a, fifo_sel_ready_b_a;
112 logic fifo_sel_valid_ar_a, fifo_sel_ready_ar_a;
113 logic fifo_sel_valid_r_a, fifo_sel_ready_r_a;
114
115 // fifo handshaking signals B
116 id_t fifo_valid_aw_b, fifo_ready_aw_b;
117 id_t fifo_valid_b_b, fifo_ready_b_b;
118 id_t fifo_valid_ar_b, fifo_ready_ar_b;
119 id_t fifo_valid_r_b, fifo_ready_r_b;
120
121 logic fifo_sel_valid_aw_b, fifo_sel_ready_aw_b;
122 logic fifo_sel_valid_w_b, fifo_sel_ready_w_b;
123 logic fifo_sel_valid_b_b, fifo_sel_ready_b_b;
124 logic fifo_sel_valid_ar_b, fifo_sel_ready_ar_b;
125 logic fifo_sel_valid_r_b, fifo_sel_ready_r_b;
126
127
128 //-----
129 // FIFO Output Signals
130 //-----
131 id_t fifo_cmp_valid_aw_a;
132 logic fifo_cmp_valid_w_a;
133 id_t fifo_cmp_valid_b_a;
134 id_t fifo_cmp_valid_ar_a;
135 id_t fifo_cmp_valid_r_a;
136
137 id_t fifo_cmp_valid_aw_b;
138 logic fifo_cmp_valid_w_b;
139 id_t fifo_cmp_valid_b_b;
140 id_t fifo_cmp_valid_ar_b;
141 id_t fifo_cmp_valid_r_b;
142
143 axi_aw_chan_t [2**AxiIdWidth-1:0] fifo_cmp_data_aw_a;
144 axi_w_chan_t fifo_cmp_data_w_a;
145 axi_b_chan_t [2**AxiIdWidth-1:0] fifo_cmp_data_b_a;
146 axi_ar_chan_t [2**AxiIdWidth-1:0] fifo_cmp_data_ar_a;
147 axi_r_chan_t [2**AxiIdWidth-1:0] fifo_cmp_data_r_a;
148
149 axi_aw_chan_t [2**AxiIdWidth-1:0] fifo_cmp_data_aw_b;
150 axi_w_chan_t fifo_cmp_data_w_b;
151 axi_b_chan_t [2**AxiIdWidth-1:0] fifo_cmp_data_b_b;
152 axi_ar_chan_t [2**AxiIdWidth-1:0] fifo_cmp_data_ar_b;
153 axi_r_chan_t [2**AxiIdWidth-1:0] fifo_cmp_data_r_b;
154
155
156 // Size alignment signals
157 logic [2:0] w_size;
158 logic [$clog2(DataWidth/8)-1:0] w_lower, w_offset, w_in-
crement;
159 logic [2**AxiIdWidth-1:0][2:0] r_size;
160 logic [2**AxiIdWidth-1:0][$clog2(DataWidth/8)-1:0]

```

```

r_lower, r_offset, r_increment;
161
162 //-----
163 // Channel A stream forks
164 //-----
165 cc_stream_fork #(
166     .NumOutp ( 32'd2 )
167 ) i_stream_fork_aw_a (
168     .clk_i,
169     .rst_ni,
170     .clr_i
171     ),
172     .valid_i ( axi_a_req_i.aw_val-
id
173     ),
174     .ready_o
175     ),
176     .valid_o ( {fifo_sel_valid_aw_a, axi_a_req_o.aw_val-
id} ),
177     .ready_i ( {fifo_sel_ready_aw_a,
axi_a_rsp_i.aw_ready} )
178 );
179
180 cc_stream_fork #(
181     .NumOutp ( 32'd2 )
182 ) i_stream_fork_w_a (
183     .clk_i,
184     .rst_ni,
185     .clr_i
186     ),
187     .valid_i ( axi_a_req_i.w_val-
id
188     ),
189     .ready_o
190     ),
191     .valid_o ( {fifo_sel_valid_w_a, axi_a_req_o.w_val-
id} ),
192     .ready_i ( {fifo_sel_ready_w_a, axi_a_rsp_i.w_ready} )
193 );
194
195 cc_stream_fork #(
196     .NumOutp ( 32'd2 )
197 ) i_stream_fork_b_a (
198     .clk_i,
199     .rst_ni,
200     .clr_i
201     ),
202     .valid_i ( axi_a_rsp_i.b_val-
id
203     ),
204     .ready_o
205     ),
206     .valid_o ( {fifo_sel_valid_b_a, axi_a_rsp_o.b_val-
id} ),
207     .ready_i ( {fifo_sel_ready_b_a, axi_a_req_i.b_ready} )
208 );
209
210 cc_stream_fork #(
211     .NumOutp ( 32'd2 )
212 ) i_stream_fork_ar_a (
213     .clk_i,
214     .rst_ni,
215     .clr_i
216     ),
217     .valid_i ( axi_a_req_i.ar_val-
id
218     ),
219     .ready_o
220     ),
221     .valid_o ( {fifo_sel_valid_ar_a, axi_a_req_o.ar_val-
id} ),
222     .ready_i ( {fifo_sel_ready_ar_a,
axi_a_rsp_i.ar_ready} )
223 );
224
225 cc_stream_fork #(
226     .NumOutp ( 32'd2 )
227 ) i_stream_fork_r_a (
228     .clk_i,
229     .rst_ni,
230     .clr_i
231     ),
232     .valid_i ( axi_a_rsp_i.r_val-
id
233     ),
234     .ready_o
235     ),
236     .valid_o ( {fifo_sel_valid_r_a, axi_a_rsp_o.r_val-
id} ),
237     .ready_i ( {fifo_sel_ready_r_a, axi_a_req_i.r_ready} )
238 );
239
240 //-----
241 // Channel A FIFOs
242 //-----
243 for (genvar id = 0; id < 2**AxiIdWidth; id++) begin :
gen_fifos_a
244
245     cc_stream_fifo #(
246         .FallThrough ( 1'b0 ),
247         .DataWidth ( 1'b0 ),
248         .Depth ( FifoDepth ),
249         .data_t ( axi_aw_chan_t )
250     ) i_stream_fifo_aw_a (
251         .clk_i,

```



```

238     .rst_ni,
239     .clr_i
240   ( 1'b0
241     .flush_i
242   ( 1'b0
243     .usage_o ( /* NOT CONNECTED
244   */
245     .data_i
246   ( axi_a_req_i.aw
247   [id]
248     .valid_i ( fifo_valid_aw_a
249   [id]
250     .ready_o ( fifo_ready_aw_a
251   [id]
252     .data_o ( fifo_cmp_data_aw_a
253   [id]
254     .valid_o ( fifo_cmp_valid_aw_a
255   [id]
256     .ready_i ( fifo_cmp_valid_aw_a [id] &
257   fifo_cmp_valid_b [id] )
258   );
259   cc_stream_fifo #(
260     .FallThrough ( 1'b0 ),
261     .DataWidth ( 1'b0 ),
262     .Depth ( FifoDepth ),
263     .data_t ( axi_b_chan_t )
264   ) i_stream_fifo_b_a (
265     .clk_i,
266     .rst_ni,
267     .clr_i
268   ( 1'b0
269     .flush_i
270   ( 1'b0
271     .usage_o ( /* NOT CONNECTED
272   */
273     .data_i
274   ( axi_a_rsp_i.b
275   [id]
276     .valid_i ( fifo_valid_b_a
277   [id]
278     .ready_o ( fifo_ready_b_a
279   [id]
280     .data_o ( fifo_cmp_data_b_a
281   [id]
282     .valid_o ( fifo_cmp_valid_b_a
283   [id]
284     .ready_i ( fifo_cmp_valid_b_a [id] &
285   fifo_cmp_valid_b_b [id] )
286   );
287   cc_stream_fifo #(
288     .FallThrough ( 1'b0 ),
289     .DataWidth ( 1'b0 ),
290     .Depth ( FifoDepth ),
291     .data_t ( axi_ar_chan_t )
292   ) i_stream_fifo_ar_a (
293     .clk_i,
294     .rst_ni,
295     .clr_i
296   ( 1'b0
297     .flush_i
298   ( 1'b0
299     .usage_o ( /* NOT CONNECTED
300   */
301     .data_i
302   ( axi_a_req_i.ar
303   [id]
304     .valid_i ( fifo_valid_ar_a
305   [id]
306     .ready_o ( fifo_ready_ar_a
307   [id]
308     .data_o ( fifo_cmp_data_ar_a
309   [id]
310     .valid_o ( fifo_cmp_valid_ar_a
311   [id]
312     .ready_i ( fifo_cmp_valid_ar_a [id] &
313   fifo_cmp_valid_ar_b [id] )
314   );
315   if (UseSize) begin : gen_r_size
316     cc_stream_fifo #(
317       .FallThrough ( 1'b0 ),
318       .DataWidth ( $clog2(DataWidth/8)+3 ),
319       .Depth ( 2*FifoDepth )
320     ) i_stream_fifo_w_size (
321       .clk_i,
322       .rst_ni,
323       .clr_i
324     ( 1'b0
325       .flush_i ( 1'b0 ),
326       .usage_o ( ),
327       .data_i ( {axi_a_req_i.aw.addr[$clog2(DataWidth/8)-1:0], axi_a_req_i.aw.size} ),
328       .valid_i ( fifo_valid_ar_a [id] &
329       axi_a_rsp_o.aw_ready ),
330       .ready_o ( ),
331       .data_o ( {w_offset, w_size} ),
332       .valid_o ( ),
333       .ready_i ( fifo_cmp_valid_w_a & fifo_cmp_val-
334       id_w_b & fifo_cmp_data_w_a.last )
335     );
336     always_ff @(posedge clk_i or negedge rst_ni) begin :
337     proc_w_increment
338       if(!rst_ni) begin
339         w_increment <= '0;
340       end else begin
341         if (fifo_cmp_valid_w_a && fifo_cmp_valid_w_b)
342         begin
343           if (fifo_cmp_data_w_a.last) begin
344             w_increment <= '0;
345           end else begin
346             w_increment <= (w_increment + 2**w_s-
347             ize) - ((w_offset+w_increment)%(2**w_size));
348           end
349         end
350       end
351       end
352       assign w_lower = w_offset + w_increment;
353     end else begin : gen_no_size
354       assign w_offset = '0;
355       assign w_size = '1;
356       assign w_lower = '0;
357     end
358   cc_stream_fifo #(

```

```

gin : proc_r_increment
307   if(!rst_ni) begin
308     r_increment[id] <= '0;
309   end else begin
310     if (fifo_cmp_valid_r_a[id] &&
311     fifo_cmp_valid_r_b[id]) begin
312       if (fifo_cmp_data_r_a[id].last) begin
313         r_increment[id] <= '0;
314       end else begin
315         r_increment[id] <= r_increment[id] +
316         2**r_size[id] - ((r_offset[id]+r_increment[id])%(2**r_size[id]));
317       end
318     end
319     end
320     assign r_lower[id] = r_offset[id] + r_incre-
321     ment[id];
322   end else begin : gen_no_size
323     assign r_offset[id] = '0;
324     assign r_size[id] = '1;
325     assign r_lower[id] = '0;
326   end
327   cc_stream_fifo #(
328     .FallThrough ( 1'b0 ),
329     .DataWidth ( 1'b0 ),
330     .Depth ( FifoDepth ),
331     .data_t ( axi_r_chan_t )
332   ) i_stream_fifo_r_a (
333     .clk_i,
334     .rst_ni,
335     .clr_i
336   ( 1'b0
337     .flush_i
338   ( 1'b0
339     .usage_o ( /* NOT CONNECTED
340   */
341     .data_i
342   ( axi_a_rsp_i.r
343   [id]
344     .valid_i ( fifo_valid_r_a
345   [id]
346     .ready_o ( fifo_ready_r_a
347   [id]
348     .data_o ( fifo_cmp_data_r_a
349   [id]
350     .valid_o ( fifo_cmp_valid_r_a
351   [id]
352     .ready_i ( fifo_cmp_valid_r_a [id] &
353   fifo_cmp_valid_r_b [id] )
354   );
355   end
356   if (UseSize) begin : gen_w_size
357     cc_stream_fifo #(
358       .FallThrough ( 1'b0 ),
359       .DataWidth ( $clog2(DataWidth/8)+3 ),
360       .Depth ( FifoDepth )
361     ) i_stream_fifo_w_size (
362       .clk_i,
363       .rst_ni,
364       .clr_i
365     ( 1'b0
366       .flush_i ( 1'b0 ),
367       .usage_o ( ),
368       .data_i ( {axi_a_req_i.aw.addr[$clog2(DataWidth/8)-1:0], axi_a_req_i.aw.size} ),
369       .valid_i ( axi_a_req_i.aw_valid &
370       axi_a_rsp_o.aw_ready ),
371       .ready_o ( ),
372       .data_o ( {w_offset, w_size} ),
373       .valid_o ( ),
374       .ready_i ( fifo_cmp_valid_w_a & fifo_cmp_val-
375       id_w_b & fifo_cmp_data_w_a.last )
376     );
377     always_ff @(posedge clk_i or negedge rst_ni) begin :
378     proc_w_increment
379       if(!rst_ni) begin
380         w_increment <= '0;
381       end else begin
382         if (fifo_cmp_valid_w_a && fifo_cmp_valid_w_b)
383         begin
384           if (fifo_cmp_data_w_a.last) begin
385             w_increment <= '0;
386           end else begin
387             w_increment <= (w_increment + 2**w_s-
388             ize) - ((w_offset+w_increment)%(2**w_size));
389           end
390         end
391       end
392       end
393       assign w_lower = w_offset + w_increment;
394     end else begin : gen_no_size
395       assign w_offset = '0;
396       assign w_size = '1;
397       assign w_lower = '0;
398     end
399   cc_stream_fifo #(

```

```

386         .FallThrough ( 1'b0          ),
387         .DataWidth   ( 1'b0          ),
388         .Depth       ( FifoDepth     ),
389         .data_t      ( axi_w_chan_t   )
390     ) i_stream_fifo_w_a (
391         .clk_i,
392         .rst_ni,
393         .clr_i
394     ),
395     ( 1'b0          ),
396     .flush_i
397     ),
398     .usage_o ( /* NOT CONNECTED
399     */
400     ),
401     .data_i
402     ),
403     ( axi_a_req_i.w
404     .valid_i ( fifo_sel_val-
405     id_w_a
406     ),
407     .ready_o
408     ( fifo_sel_ready_w_a
409     ),
410     .data_o
411     ( fifo_cmp_data_w_a
412     ),
413     .valid_o ( fifo_cmp_val-
414     id_w_a
415     ),
416     .ready_i ( fifo_cmp_valid_w_a & fifo_cmp_val-
417     id_w_b )
418     );
419
420 //-----
421 // Input Handshaking A
422 //-----
423 always_comb begin : gen_handshaking_a
424     // aw
425     // defaults
426     fifo_valid_aw_a = '0;
427     fifo_sel_ready_aw_a = '0;
428     // assign according id
429     fifo_valid_aw_a [axi_a_req_i.aw.id] = fifo_sel_val-
430     id_aw_a;
431     fifo_sel_ready_aw_a =
432     fifo_ready_aw_a[axi_a_req_i.aw.id];
433
434     // b
435     // defaults
436     fifo_valid_b_a = '0;
437     fifo_sel_ready_b_a = '0;
438     // assign according id
439     fifo_valid_b_a [axi_a_rsp_i.b.id] = fifo_sel_val-
440     id_b_a;
441     fifo_sel_ready_b_a =
442     fifo_ready_b_a[axi_a_rsp_i.b.id];
443
444     // ar
445     // defaults
446     fifo_valid_ar_a = '0;
447     fifo_sel_ready_ar_a = '0;
448     // assign according id
449     fifo_valid_ar_a [axi_a_req_i.ar.id] = fifo_sel_val-
450     id_ar_a;
451     fifo_sel_ready_ar_a =
452     fifo_ready_ar_a[axi_a_req_i.ar.id];
453
454     // b
455     // defaults
456     fifo_valid_r_a = '0;
457     fifo_sel_ready_r_a = '0;
458     // assign according id
459     fifo_valid_r_a [axi_a_rsp_i.r.id] = fifo_sel_val-
460     id_r_a;
461     fifo_sel_ready_r_a =
462     fifo_ready_r_a[axi_a_rsp_i.r.id];
463 end
464
465 //-----
466 // Channel B stream forks
467 //-----
468 cc_stream_fork #(
469     .NumOut ( 32'd2 )
470 ) i_stream_fork_aw_b (
471     .clk_i,
472     .rst_ni,
473     .clr_i
474     ),
475     ( 1'b0
476     .valid_i ( axi_b_req_i.aw_val-
477     id
478     ),
479     .ready_o
480     ( axi_b_rsp_o.w_ready
481     ),
482     .valid_o ( {fifo_sel_valid_w_b, axi_b_req_o.w_val-
483     id} ),
484     .ready_i ( {fifo_sel_ready_w_b, axi_b_rsp_i.w_ready} )
485     );
486
487 cc_stream_fork #(
488     .NumOut ( 32'd2 )
489 ) i_stream_fork_b_b (
490     .clk_i,
491     .rst_ni,
492     .clr_i
493     ),
494     ( 1'b0
495     .valid_i ( axi_b_rsp_i.b_val-
496     id
497     ),
498     .ready_o
499     ( axi_b_req_o.b_ready
500     ),
501     .valid_o ( {fifo_sel_valid_b_b, axi_b_rsp_o.b_val-
502     id} ),
503     .ready_i ( {fifo_sel_ready_b_b, axi_b_req_i.b_ready} )
504     );
505
506 cc_stream_fork #(
507     .NumOut ( 32'd2 )
508 ) i_stream_fork_ar_b (
509     .clk_i,
510     .rst_ni,
511     .clr_i
512     ),
513     ( 1'b0
514     .valid_i ( axi_b_req_i.ar_val-
515     id
516     ),
517     .ready_o
518     ( axi_b_rsp_o.ar_ready
519     ),
520     .valid_o ( {fifo_sel_valid_ar_b, axi_b_req_o.ar_val-
521     id} ),
522     .ready_i ( {fifo_sel_ready_ar_b,
523     axi_b_rsp_i.ar_ready} )
524     );
525
526 cc_stream_fork #(
527     .NumOut ( 32'd2 )
528 ) i_stream_fork_r_b (
529     .clk_i,
530     .rst_ni,
531     .clr_i
532     ),
533     ( 1'b0
534     .valid_i ( axi_b_rsp_i.r_val-
535     id
536     ),
537     .ready_o
538     ( axi_b_req_o.r_ready
539     ),
540     .valid_o ( {fifo_sel_valid_r_b, axi_b_rsp_o.r_val-
541     id} ),
542     .ready_i ( {fifo_sel_ready_r_b, axi_b_req_i.r_ready} )
543     );
544
545 //-----
546 // Channel B FIFOs
547 //-----
548 for (genvar id = 0; id < 2*AxidWidth; id++) begin :
549     gen_fifos_b
550
551         cc_stream_fifo #(
552             .FallThrough ( 1'b0          ),
553             .DataWidth   ( 1'b0          ),
554             .Depth       ( FifoDepth     ),
555             .data_t      ( axi_w_chan_t   )
556         ) i_stream_fifo_aw_b (
557             .clk_i,
558             .rst_ni,
559             .clr_i
560             ),
561             ( 1'b0
562             .flush_i
563             ),
564             ( 1'b0
565             .usage_o ( /* NOT CONNECTED
566             */
567             ),
568             .data_i
569             ),
570             ( axi_b_req_i.aw
571             .valid_i ( fifo_valid_aw_b
572             [id] ),
573             .ready_o ( fifo_ready_aw_b
574             [id] ),
575             .data_o ( fifo_cmp_data_aw_b
576             [id] ),
577             .valid_o ( fifo_cmp_valid_aw_b
578             [id] ),
579             .ready_i ( fifo_cmp_valid_aw_a [id] &
580             fifo_cmp_valid_aw_b [id] )
581             );
582
583         cc_stream_fifo #(
584             .FallThrough ( 1'b0          ),
585             .DataWidth   ( 1'b0          ),
586             .Depth       ( FifoDepth     ),
587             .data_t      ( axi_b_chan_t   )
588         ) i_stream_fifo_b_b (
589             .clk_i,

```

```

( 1'b0
465     .valid_i ( axi_b_req_i.w_val-
466     id
467     ),
468     .ready_o
469     ( axi_b_rsp_o.w_ready
470     ),
471     .valid_o ( {fifo_sel_valid_w_b, axi_b_req_o.w_val-
472     id} ),
473     .ready_i ( {fifo_sel_ready_w_b, axi_b_rsp_i.w_ready} )
474     );
475
476 cc_stream_fork #(
477     .NumOut ( 32'd2 )
478 ) i_stream_fork_b_b (
479     .clk_i,
480     .rst_ni,
481     .clr_i
482     ),
483     ( 1'b0
484     .valid_i ( axi_b_rsp_i.b_val-
485     id
486     ),
487     .ready_o
488     ( axi_b_req_o.b_ready
489     ),
490     .valid_o ( {fifo_sel_valid_b_b, axi_b_rsp_o.b_val-
491     id} ),
492     .ready_i ( {fifo_sel_ready_b_b, axi_b_req_i.b_ready} )
493     );
494
495 cc_stream_fork #(
496     .NumOut ( 32'd2 )
497 ) i_stream_fork_ar_b (
498     .clk_i,
499     .rst_ni,
500     .clr_i
501     ),
502     ( 1'b0
503     .valid_i ( axi_b_req_i.ar_val-
504     id
505     ),
506     .ready_o
507     ( axi_b_rsp_o.ar_ready
508     ),
509     .valid_o ( {fifo_sel_valid_ar_b, axi_b_req_o.ar_val-
510     id} ),
511     .ready_i ( {fifo_sel_ready_ar_b,
512     axi_b_rsp_i.ar_ready} )
513     );
514
515 cc_stream_fork #(
516     .NumOut ( 32'd2 )
517 ) i_stream_fork_r_b (
518     .clk_i,
519     .rst_ni,
520     .clr_i
521     ),
522     ( 1'b0
523     .valid_i ( axi_b_rsp_i.r_val-
524     id
525     ),
526     .ready_o
527     ( axi_b_req_o.r_ready
528     ),
529     .valid_o ( {fifo_sel_valid_r_b, axi_b_rsp_o.r_val-
530     id} ),
531     .ready_i ( {fifo_sel_ready_r_b, axi_b_req_i.r_ready} )
532     );
533
534 //-----
535 // Channel B FIFOs
536 //-----
537 for (genvar id = 0; id < 2*AxidWidth; id++) begin :
538     gen_fifos_b
539
540         cc_stream_fifo #(
541             .FallThrough ( 1'b0          ),
542             .DataWidth   ( 1'b0          ),
543             .Depth       ( FifoDepth     ),
544             .data_t      ( axi_w_chan_t   )
545         ) i_stream_fifo_aw_b (
546             .clk_i,
547             .rst_ni,
548             .clr_i
549             ),
550             ( 1'b0
551             .flush_i
552             ),
553             ( 1'b0
554             .usage_o ( /* NOT CONNECTED
555             */
556             ),
557             .data_i
558             ),
559             ( axi_b_req_i.aw
560             .valid_i ( fifo_valid_aw_b
561             [id] ),
562             .ready_o ( fifo_ready_aw_b
563             [id] ),
564             .data_o ( fifo_cmp_data_aw_b
565             [id] ),
566             .valid_o ( fifo_cmp_valid_aw_b
567             [id] ),
568             .ready_i ( fifo_cmp_valid_aw_a [id] &
569             fifo_cmp_valid_aw_b [id] )
570             );
571
572         cc_stream_fifo #(
573             .FallThrough ( 1'b0          ),
574             .DataWidth   ( 1'b0          ),
575             .Depth       ( FifoDepth     ),
576             .data_t      ( axi_b_chan_t   )
577         ) i_stream_fifo_b_b (
578             .clk_i,

```

```

539      .rst_ni,
540      .clr_i
541    ),
542    .flush_i
543    ( 'b0
544    ),
545    .usage_o ( /* NOT CONNECTED
546    ),
547    .data_i
548    ( axi_b_rsp_i.b
549    ),
550    .valid_i ( fifo_valid_b_b
551    [id]
552    ),
553    .ready_o ( fifo_ready_b_b
554    [id]
555    ),
556    .data_o ( fifo_cmp_data_b_b
557    [id]
558    ),
559    .valid_o ( fifo_cmp_valid_b_b
560    [id]
561    ),
562    .ready_i ( fifo_cmp_valid_b_a [id] &
563    fifo_cmp_valid_b_b [id] )
564  );
565
566  cc_stream_fifo #(
567    .FallThrough ( 'b0
568    ),
569    .DataWidth ( 'b0
570    ),
571    .Depth ( FifoDepth
572    ),
573    .data_t ( axi_ar_chan_t
574    )
575  ) i_stream_fifo_ar_b (
576    .clk_i,
577    .rst_ni,
578    .clr_i
579    ( 'b0
580    ),
581    .flush_i
582    ( 'b0
583    ),
584    .usage_o ( /* NOT CONNECTED
585    ),
586    .data_i
587    ( axi_b_req_i.ar
588    ),
589    .valid_i ( fifo_valid_ar_b
590    [id]
591    ),
592    .ready_o ( fifo_ready_ar_b
593    [id]
594    ),
595    .data_o ( fifo_cmp_data_ar_b
596    [id]
597    ),
598    .valid_o ( fifo_cmp_valid_ar_b
599    [id]
600    ),
601    .ready_i ( fifo_cmp_valid_ar_a [id] &
602    fifo_cmp_valid_ar_b [id] )
603  );
604
605  cc_stream_fifo #(
606    .FallThrough ( 'b0
607    ),
608    .DataWidth ( 'b0
609    ),
610    .Depth ( FifoDepth
611    ),
612    .data_t ( axi_r_chan_t
613    )
614  ) i_stream_fifo_r_b (
615    .clk_i,
616    .rst_ni,
617    .clr_i
618    ( 'b0
619    ),
620    .flush_i
621    ( 'b0
622    ),
623    .usage_o ( /* NOT CONNECTED
624    ),
625    .data_i
626    ( axi_b_rsp_i.r
627    ),
628    .valid_i ( fifo_valid_r_b
629    [id]
630    ),
631    .ready_o ( fifo_ready_r_b
632    [id]
633    ),
634    .data_o ( fifo_cmp_data_r_b
635    [id]
636    ),
637    .valid_o ( fifo_cmp_valid_r_b
638    [id]
639    ),
640    .ready_i ( fifo_cmp_valid_r_a [id] &
641    fifo_cmp_valid_r_b [id] )
642  );
643
644  end
645
646  cc_stream_fifo #(
647    .FallThrough ( 'b0
648    ),
649    .DataWidth ( 'b0
650    ),
651    .Depth ( FifoDepth
652    ),
653    .data_t ( axi_w_chan_t
654    )
655  ) i_stream_fifo_w_b (
656    .clk_i,
657    .rst_ni,
658    .clr_i
659    ( 'b0
660    ),
661    .flush_i
662    ( 'b0
663    ),
664    .usage_o ( /* NOT CONNECTED
665    ),
666    .data_i
667    ( axi_b_req_i.w
668    ),
669    .valid_i ( fifo_sel_val-
670    id_w_b
671    ),
672    .ready_o ( fifo_sel_ready_w_b
673    [id]
674    ),
675    .data_o ( fifo_cmp_data_w_b
676    [id]
677    ),
678    .valid_o ( fifo_cmp_val-

```

```

679    id_w_b
680    ),
681    .ready_i ( fifo_cmp_valid_w_a & fifo_cmp_val-
682    id_w_b
683    );
684
685    //-----
686    // Input Handshaking B
687    //-----
688    always_comb begin : gen_handshaking_b
689      // aw
690      // defaults
691      fifo_valid_aw_b = '0;
692      fifo_sel_ready_aw_b = '0;
693      // assign according id
694      fifo_valid_aw_b [axi_b_req_i.aw.id] = fifo_sel_val-
695      id_aw_b;
696      fifo_sel_ready_aw_b
697      =
698      fifo_ready_aw_b[axi_b_req_i.aw.id];
699
700      // b
701      // defaults
702      fifo_valid_b_b = '0;
703      fifo_sel_ready_b_b = '0;
704      // assign according id
705      fifo_valid_b_b [axi_b_rsp_i.b.id] = fifo_sel_val-
706      id_b_b;
707      fifo_sel_ready_b_b
708      =
709      fifo_ready_b_b[axi_b_rsp_i.b.id];
710
711      // ar
712      // defaults
713      fifo_valid_ar_b = '0;
714      fifo_sel_ready_ar_b = '0;
715      // assign according id
716      fifo_valid_ar_b [axi_b_req_i.ar.id] = fifo_sel_val-
717      id_ar_b;
718      fifo_sel_ready_ar_b
719      =
720      fifo_ready_ar_b[axi_b_req_i.ar.id];
721
722      // r
723      // defaults
724      fifo_valid_r_b = '0;
725      fifo_sel_ready_r_b = '0;
726      // assign according id
727      fifo_valid_r_b [axi_b_rsp_i.r.id] = fifo_sel_val-
728      id_r_b;
729      fifo_sel_ready_r_b
730      =
731      fifo_ready_r_b[axi_b_rsp_i.r.id];
732    end
733
734    //-----
735    // Comparison
736    //-----
737    for (genvar id = 0; id < 2**AxIdWidth; id++) begin :
738      gen_cmp
739        logic [DataWidth/8-1:0] r_data_partial_mismatch;
740        logic r_data_mismatch;
741
742        if (UseSize) begin : gen_r_mismatch_sized
743          for (genvar j = 0; j < DataWidth/8; j++) begin :
744            gen_r_partial_mismatch
745              assign r_data_partial_mismatch[j] =
746              fifo_cmp_data_r_a[id].data[8*j+:8] !=
747              fifo_cmp_data_r_b[id].data[8*j+:8];
748            end
749          end
750          always_comb begin : proc_r_data_mismatch
751            r_data_mismatch = '0;
752
753            for (int unsigned j = 0; j < DataWidth/8; j++)
754              begin
755                if (j >= r_lower[id] && j < (r_lower[id] +
756                2**r_size[id]) - ((r_lower[id] + 2**r_size[id])%(2**r_size[id]))) be-
757                gin
758                  r_data_mismatch |=
759                  r_data_partial_mismatch[j];
760                end
761              end
762            end else begin : gen_r_mismatch_unsized
763              assign r_data_mismatch =
764              fifo_cmp_data_r_a[id].data != fifo_cmp_data_r_b[id].data;
765            end
766
767            assign aw_mismatch_o [id] = (fifo_cmp_valid_aw_a [id]
768            & fifo_cmp_valid_aw_b [id]) ?
769            fifo_cmp_data_aw_a
770            [id] != fifo_cmp_data_aw_b [id] : '0;
771            assign b_mismatch_o [id] = (fifo_cmp_valid_b_a [id]
772            & fifo_cmp_valid_b_b [id]) ?
773            fifo_cmp_data_b_a
774            [id] != fifo_cmp_data_b_b [id] : '0;
775            assign ar_mismatch_o [id] = (fifo_cmp_valid_ar_a [id]
776            & fifo_cmp_valid_ar_b [id]) ?
777            fifo_cmp_data_ar_a
778            [id] != fifo_cmp_data_ar_b [id] : '0;
779            assign r_mismatch_o [id] = (fifo_cmp_valid_r_a [id]

```

6. pulp-platform_axi/src/axi_cdc.sv (254 lines)

```

92     axi_cdc_dst #(
93         .aw_chan_t  ( aw_chan_t  ),
94         .w_chan_t   ( w_chan_t   ),
95         .b_chan_t   ( b_chan_t   ),
96         .ar_chan_t  ( ar_chan_t  ),
97         .r_chan_t   ( r_chan_t   ),
98         .axi_req_t   ( axi_req_t  ),
99         .axi_resp_t  ( axi_resp_t ),
100        .LogDepth    ( LogDepth   ),
101        .SyncStages   ( SyncStages ),
102    ) i_axi_cdc_dst (
103        .dst_clk_i,
104        .dst_rst_ni,
105        .dst_req_o,
106        .dst_resp_i,
107        (* async *) .async_data_slave_aw_wptr_i
108    ) ( async_data_slave_aw_wptr_i ),
109    (* async *) .async_data_slave_aw_rptr_o
110    ) ( async_data_slave_aw_rptr_o ),
111    (* async *) .async_data_slave_aw_data_i
112    ) ( async_data_slave_aw_data_i ),
113    (* async *) .async_data_slave_w_wptr_i
114    ) ( async_data_slave_w_wptr_i ),
115    (* async *) .async_data_slave_w_rptr_o
116    ) ( async_data_slave_w_rptr_o ),
117    (* async *) .async_data_slave_w_data_i
118    ) ( async_data_slave_w_data_i ),
119    (* async *) .async_data_slave_b_wptr_o
120    ) ( async_data_slave_b_wptr_o ),
121    (* async *) .async_data_slave_b_rptr_i
122    ) ( async_data_slave_b_rptr_i ),
123    (* async *) .async_data_slave_b_data_o
124    ) ( async_data_slave_b_data_o ),
125    (* async *) .async_data_slave_ar_wptr_i
126    ) ( async_data_slave_ar_wptr_i ),
127    (* async *) .async_data_slave_ar_rptr_o
128    ) ( async_data_slave_ar_rptr_o ),
129    (* async *) .async_data_slave_ar_data_i
130    ) ( async_data_slave_ar_data_i ),
131    (* async *) .async_data_slave_r_wptr_o
132    ) ( async_data_slave_r_wptr_o ),
133    (* async *) .async_data_slave_r_rptr_i
134    ) ( async_data_slave_r_rptr_i ),
135    (* async *) .async_data_slave_r_data_o
136    ) ( async_data_slave_r_data_o ),
137    );
138
139 endmodule
140
141 // interface wrapper
142 module axi_cdc_intf #(
143     parameter int unsigned AXI_ID_WIDTH    = 0,
144     parameter int unsigned AXI_ADDR_WIDTH  = 0,
145     parameter int unsigned AXI_DATA_WIDTH  = 0,
146     parameter int unsigned AXI_USER_WIDTH  = 0,
147     /// Depth of the FIFO crossing the clock domain, given as
148     2**LOG_DEPTH,
149     parameter int unsigned LOG_DEPTH = 1,
150     /// Number of synchronization registers to insert on the
151     async pointers
152     parameter int unsigned SYNC_STAGES = 2
153 ) (
154     // slave side - clocked by `src_clk_i`
155     input logic src_clk_i,
156     input logic src_rst_ni,
157     AXI_BUS.Slave src,
158     // master side - clocked by `dst_clk_i`
159     input logic dst_clk_i,
160     input logic dst_rst_ni,
161     AXI_BUS.Master dst
162 );
163
164 typedef logic [AXI_ID_WIDTH-1:0] id_t;
165 typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
166 typedef logic [AXI_DATA_WIDTH-1:0] data_t;
167 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
168 typedef logic [AXI_USER_WIDTH-1:0] user_t;
169 `AXI_TYPERDEF_AW_CHAN_T(aw_chan_t, addr_t, id_t, user_t)
170 `AXI_TYPERDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)
171 `AXI_TYPERDEF_B_CHAN_T(b_chan_t, id_t, user_t)
172 `AXI_TYPERDEF_AR_CHAN_T(ar_chan_t, addr_t, id_t, user_t)
173 `AXI_TYPERDEF_R_CHAN_T(r_chan_t, data_t, id_t, user_t)
174 `AXI_TYPERDEF_REQ_T(req_t, aw_chan_t, w_chan_t, ar_chan_t)
175 `AXI_TYPERDEF_RESP_T(resp_t, b_chan_t, r_chan_t)
176
177 req_t src_req, dst_req;
178 resp_t src_resp, dst_resp;
179
180 `AXI_ASSIGN_TO_REQ(src_req, src)
181 `AXI_ASSIGN_FROM_RESP(src, src_resp)
182
183 `AXI_ASSIGN_FROM_REQ(dst, dst_req)
184 `AXI_ASSIGN_TO_RESP(dst_resp, dst)
185
186 axi_cdc #(
187     .aw_chan_t  ( aw_chan_t  ),
188     .w_chan_t   ( w_chan_t   ),
189     .b_chan_t   ( b_chan_t   ),
190     .ar_chan_t  ( ar_chan_t  ),
191     .r_chan_t   ( r_chan_t   ),
192     .axi_req_t   ( axi_req_t  ),
193     .axi_resp_t  ( axi_resp_t ),
194     .LogDepth    ( LogDepth   ),
195     .SyncStages   ( SyncStages ),
196 ) i_axi_cdc_dst (
197     .dst_clk_i,
198     .dst_rst_ni,
199     .dst_req_o,
200     .dst_resp_i,
201     (* async *) .async_data_slave_aw_wptr_i
202 ) ( async_data_slave_aw_wptr_i ),
203 (* async *) .async_data_slave_aw_rptr_o
204 ) ( async_data_slave_aw_rptr_o ),
205 (* async *) .async_data_slave_aw_data_i
206 ) ( async_data_slave_aw_data_i ),
207 (* async *) .async_data_slave_w_wptr_i
208 ) ( async_data_slave_w_wptr_i ),
209 (* async *) .async_data_slave_w_rptr_o
210 ) ( async_data_slave_w_rptr_o ),
211 (* async *) .async_data_slave_w_data_i
212 ) ( async_data_slave_w_data_i ),
213 (* async *) .async_data_slave_b_wptr_o
214 ) ( async_data_slave_b_wptr_o ),
215 (* async *) .async_data_slave_b_rptr_i
216 ) ( async_data_slave_b_rptr_i ),
217 (* async *) .async_data_slave_b_data_o
218 ) ( async_data_slave_b_data_o ),
219 (* async *) .async_data_slave_ar_wptr_i
220 ) ( async_data_slave_ar_wptr_i ),
221 (* async *) .async_data_slave_ar_rptr_o
222 ) ( async_data_slave_ar_rptr_o ),
223 (* async *) .async_data_slave_ar_data_i
224 ) ( async_data_slave_ar_data_i ),
225 (* async *) .async_data_slave_r_wptr_o
226 ) ( async_data_slave_r_wptr_o ),
227 (* async *) .async_data_slave_r_rptr_i
228 ) ( async_data_slave_r_rptr_i ),
229 (* async *) .async_data_slave_r_data_o
230 ) ( async_data_slave_r_data_o ),
231 );
232
233 endmodule

```

```

176     .ar_chan_t    ( ar_chan_t ),
177     .r_chan_t    ( r_chan_t ),
178     .axi_req_t   ( req_t ),
179     .axi_resp_t  ( resp_t ),
180     .LogDepth    ( LOG_DEPTH ),
181     .SyncStages  ( SYNC_STAGES )
182 ) i_axi_cdc (
183     .src_clk_i,
184     .src_rst_ni,
185     .src_req_i  ( src_req ),
186     .src_resp_o ( src_resp ),
187     .dst_clk_i,
188     .dst_rst_ni,
189     .dst_req_o  ( dst_req ),
190     .dst_resp_i ( dst_resp )
191 );
192
193 endmodule
194
195 module axi_lite_cdc_intf #(
196     parameter int unsigned AXI_ADDR_WIDTH = 0,
197     parameter int unsigned AXI_DATA_WIDTH = 0,
198     // Depth of the FIFO crossing the clock domain, given as
2*LOG_DEPTH.
199     parameter int unsigned LOG_DEPTH = 1,
200     /// Number of synchronization registers to insert on the
async pointers
201     parameter int unsigned SYNC_STAGES = 2
202 ) (
203     // slave side - clocked by `src_clk_i`
204     input logic    src_clk_i,
205     input logic    src_rst_ni,
206     AXI_LITE.Slave  src,
207     // master side - clocked by `dst_clk_i`
208     input logic    dst_clk_i,
209     input logic    dst_rst_ni,
210     AXI_LITE.Master dst
211 );
212
213 typedef logic [AXI_ADDR_WIDTH-1:0]  addr_t;
214 typedef logic [AXI_DATA_WIDTH-1:0]  data_t;
215 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
216 `AXI_LITE_TTYPEDEF_AW_CHAN_T(aw_chan_t, addr_t)
217 `AXI_LITE_TTYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t)
218 `AXI_LITE_TTYPEDEF_B_CHAN_T(b_chan_t)
219 `AXI_LITE_TTYPEDEF_AR_CHAN_T(ar_chan_t, addr_t)
220 `AXI_LITE_TTYPEDEF_R_CHAN_T(r_chan_t, data_t)
221 `AXI_LITE_TTYPEDEF_REQ_T(req_t, aw_chan_t, w_chan_t,
ar_chan_t)
222 `AXI_LITE_TTYPEDEF_RESP_T(resp_t, b_chan_t, r_chan_t)
223
224 req_t src_req, dst_req;
225 resp_t src_resp, dst_resp;
226
227 `AXI_LITE_ASSIGN_TO_REQ(src_req, src)
228 `AXI_LITE_ASSIGN_FROM_RESP(src, src_resp)
229
230 `AXI_LITE_ASSIGN_FROM_REQ(dst, dst_req)
231 `AXI_LITE_ASSIGN_TO_RESP(dst_resp, dst)
232
233 axi_cdc #(
234     .aw_chan_t    ( aw_chan_t ),
235     .w_chan_t     ( w_chan_t ),
236     .b_chan_t     ( b_chan_t ),
237     .ar_chan_t    ( ar_chan_t ),
238     .r_chan_t     ( r_chan_t ),
239     .axi_req_t    ( req_t ),
240     .axi_resp_t   ( resp_t ),
241     .LogDepth     ( LOG_DEPTH ),
242     .SyncStages   ( SYNC_STAGES )
243 ) i_axi_cdc (
244     .src_clk_i,
245     .src_rst_ni,
246     .src_req_i  ( src_req ),
247     .src_resp_o ( src_resp ),
248     .dst_clk_i,
249     .dst_rst_ni,
250     .dst_req_o  ( dst_req ),
251     .dst_resp_i ( dst_resp )
252 );
253
254 endmodule

```

7. [pulp-platform_axi/src/axi_cdc_dst.sv](#) (297 lines)

```

1 // Copyright (c) 2019-2020 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the Solder-
  pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
  file except in
5 // compliance with the License. You may obtain a copy of the
  License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
7 // or agreed to in writing, software, hardware and materials
  distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR

```

```

9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the license.
11 //
12 // Authors:
13 // - Luca Valente <luca.valente@unibo.it>
14 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
15
16 `include "axi/assign.svh"
17 `include "axi/typedef.svh"
18
19 /// Destination-clock-domain half of the AXI CDC crossing.
20 ///
21 /// For each of the five AXI channels, this module instanti-
ates the source or destination half of
22 /// a CDC FIFO. IMPORTANT: For each AXI channel, you MUST
properly constrain three paths through
23 /// the FIFO; see the header of `cdc_fifo_gray` for instruc-
tions.
24 module axi_cdc_dst #(
25     /// Depth of the FIFO crossing the clock domain, given as
2**LOG_DEPTH.
26     parameter int unsigned LogDepth = 1,
27     /// Number of synchronization registers to insert on the
async pointers
28     parameter int unsigned SyncStages = 2,
29     parameter type aw_chan_t = logic,
30     parameter type w_chan_t = logic,
31     parameter type b_chan_t = logic,
32     parameter type ar_chan_t = logic,
33     parameter type r_chan_t = logic,
34     parameter type axi_req_t = logic,
35     parameter type axi_resp_t = logic
36 ) (
37     // asynchronous slave port
38     input aw_chan_t [2**LogDepth-1:0]
async_data_slave_aw_data_i,
39     input logic [LogDepth:0]
async_data_slave_aw_wptr_i,
40     output logic [LogDepth:0]
async_data_slave_aw_rptr_o,
41     input w_chan_t [2**LogDepth-1:0]
async_data_slave_w_data_i,
42     input logic [LogDepth:0]
async_data_slave_w_wptr_i,
43     output logic [LogDepth:0]
async_data_slave_w_rptr_o,
44     output b_chan_t [2**LogDepth-1:0]
async_data_slave_b_data_o,
45     output logic [LogDepth:0]
async_data_slave_b_wptr_o,
46     input logic [LogDepth:0]
async_data_slave_b_rptr_i,
47     input ar_chan_t [2**LogDepth-1:0]
async_data_slave_ar_data_i,
48     input logic [LogDepth:0]
async_data_slave_ar_wptr_i,
49     output logic [LogDepth:0]
async_data_slave_ar_rptr_o,
50     output r_chan_t [2**LogDepth-1:0]
async_data_slave_r_data_o,
51     output logic [LogDepth:0]
async_data_slave_r_wptr_o,
52     input logic [LogDepth:0]
async_data_slave_r_rptr_i,
53     // synchronous master port - clocked by `dst_clk_i`
54     input logic dst_clk_i,
55     input logic dst_rst_ni,
56     output axi_req_t dst_req_o,
57     input axi_resp_t dst_resp_i
58 );
59
60 cc_cdc_fifo_gray_dst #(
61     `ifdef QUESTA
62         // Workaround for a bug in Questa: Pass flat logic vector
instead of struct to type parameter.
63         .data_t ( logic [$bits(aw_chan_t)-1:0] ),
64     `else
65         // Other tools, such as VCS, have problems with type para-
meters constructed through `$bits()`.
66         .data_t ( aw_chan_t ),
67     `endif
68     .LogDepth ( LogDepth ),
69     .SyncStages ( SyncStages )
70 ) i_cdc_fifo_gray_dst_aw (
71     .async_data_i ( async_data_slave_aw_data_i ),
72     .async_wptr_i ( async_data_slave_aw_wptr_i ),
73     .async_rptr_o ( async_data_slave_aw_rptr_o ),
74     .dst_clk_i,
75     .dst_rst_ni,
76     .dst_data_o ( dst_req_o.aw ),
77     .dst_valid_o ( dst_req_o.aw_valid ),
78     .dst_ready_i ( dst_resp_i.aw_ready )
79 );
80
81 cc_cdc_fifo_gray_dst #(
82     `ifdef QUESTA
83         .data_t ( logic [$bits(w_chan_t)-1:0] ),
84     `else
85         .data_t ( w_chan_t ),

```

```

86     `endif
87     .LogDepth ( LogDepth ),
88     .SyncStages ( SyncStages )
89 ) i_cdc_fifo_gray_dst_w (
90     .async_data_i ( async_data_slave_w_data_i ),
91     .async_wptr_i ( async_data_slave_w_wptr_i ),
92     .async_rptr_o ( async_data_slave_w_rptr_o ),
93     .dst_clk_i,
94     .dst_rst_ni,
95     .dst_data_o ( dst_req_o.w ),
96     .dst_valid_o ( dst_req_o.w_valid ),
97     .dst_ready_i ( dst_resp_i.w_ready )
98 );
99
100 cc_cdc_fifo_gray_src #(
101     `ifdef QUESTA
102         .data_t ( logic [$bits(b_chan_t)-1:0] ),
103     `else
104         .data_t ( b_chan_t ),
105     `endif
106     .LogDepth ( LogDepth ),
107     .SyncStages ( SyncStages )
108 ) i_cdc_fifo_gray_src_b (
109     .src_clk_i ( dst_clk_i ),
110     .src_rst_ni ( dst_rst_ni ),
111     .src_data_i ( dst_resp_i.b ),
112     .src_valid_i ( dst_resp_i.b_valid ),
113     .src_ready_o ( dst_req_o.b_ready ),
114     .async_data_o ( async_data_slave_b_data_o ),
115     .async_wptr_o ( async_data_slave_b_wptr_o ),
116     .async_rptr_i ( async_data_slave_b_rptr_i )
117 );
118
119 cc_cdc_fifo_gray_dst #(
120     `ifdef QUESTA
121         .data_t ( logic [$bits(ar_chan_t)-1:0] ),
122     `else
123         .data_t ( ar_chan_t ),
124     `endif
125     .LogDepth ( LogDepth ),
126     .SyncStages ( SyncStages )
127 ) i_cdc_fifo_gray_dst_ar (
128     .dst_clk_i,
129     .dst_rst_ni,
130     .dst_data_o ( dst_req_o.ar ),
131     .dst_valid_o ( dst_req_o.ar_valid ),
132     .dst_ready_i ( dst_resp_i.ar_ready ),
133     .async_data_i ( async_data_slave_ar_data_i ),
134     .async_wptr_i ( async_data_slave_ar_wptr_i ),
135     .async_rptr_o ( async_data_slave_ar_rptr_o )
136 );
137
138 cc_cdc_fifo_gray_src #(
139     `ifdef QUESTA
140         .data_t ( logic [$bits(r_chan_t)-1:0] ),
141     `else
142         .data_t ( r_chan_t ),
143     `endif
144     .LogDepth ( LogDepth ),
145     .SyncStages ( SyncStages )
146 ) i_cdc_fifo_gray_src_r (
147     .src_clk_i ( dst_clk_i ),
148     .src_rst_ni ( dst_rst_ni ),
149     .src_data_i ( dst_resp_i.r ),
150     .src_valid_i ( dst_resp_i.r_valid ),
151     .src_ready_o ( dst_req_o.r_ready ),
152     .async_data_o ( async_data_slave_r_data_o ),
153     .async_wptr_o ( async_data_slave_r_wptr_o ),
154     .async_rptr_i ( async_data_slave_r_rptr_i )
155 );
156
157 endmodule
158
159
160 module axi_cdc_dst_intf #(
161     parameter int unsigned AXI_ID_WIDTH = 0,
162     parameter int unsigned AXI_ADDR_WIDTH = 0,
163     parameter int unsigned AXI_DATA_WIDTH = 0,
164     parameter int unsigned AXI_USER_WIDTH = 0,
165     /// Depth of the FIFO crossing the clock domain, given as
2**LOG_DEPTH.
166     parameter int unsigned LOG_DEPTH = 1,
167     /// Number of synchronization registers to insert on the
async pointers
168     parameter int unsigned SYNC_STAGES = 2
169 ) (
170     // asynchronous slave port
171     AXI_BUS_ASYNC_GRAY.Slave src,
172     // synchronous master port - clocked by `dst_clk_i`
173     input logic dst_clk_i,
174     input logic dst_rst_ni,
175     AXI_BUS.Master dst
176 );
177
178 typedef logic [AXI_ID_WIDTH-1:0] id_t;
179 typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
180 typedef logic [AXI_DATA_WIDTH-1:0] data_t;
181 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
182 typedef logic [AXI_USER_WIDTH-1:0] user_t;
183 `AXI_TYPEDEF_AW_CHAN_T(aw_chan_t, addr_t, id_t, user_t)
184 `AXI_TYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)

```



```

185 `AXI_TYPEDEF_B_CHAN_T(b_chan_t, id_t, user_t)
186 `AXI_TYPEDEF_AR_CHAN_T(ar_chan_t, addr_t, id_t, user_t)
187 `AXI_TYPEDEF_R_CHAN_T(r_chan_t, data_t, id_t, user_t)
188 `AXI_TYPEDEF_REQ_T(req_t, aw_chan_t, w_chan_t, ar_chan_t)
189 `AXI_TYPEDEF_RESP_T(resp_t, b_chan_t, r_chan_t)
190
191 req_t dst_req;
192 resp_t dst_resp;
193
194 axi_cdc_dst #(
195     .aw_chan_t ( aw_chan_t ),
196     .w_chan_t ( w_chan_t ),
197     .b_chan_t ( b_chan_t ),
198     .ar_chan_t ( ar_chan_t ),
199     .r_chan_t ( r_chan_t ),
200     .axi_req_t ( req_t ),
201     .axi_resp_t ( resp_t ),
202     .LogDepth ( LOG_DEPTH ),
203     .SyncStages ( SYNC_STAGES )
204 ) i_axi_cdc_dst (
205     .async_data_slave_aw_data_i ( src.aw_data ),
206     .async_data_slave_aw_wptr_i ( src.aw_wptr ),
207     .async_data_slave_aw_rptr_o ( src.aw_rptr ),
208     .async_data_slave_w_data_i ( src.w_data ),
209     .async_data_slave_w_wptr_i ( src.w_wptr ),
210     .async_data_slave_w_rptr_o ( src.w_rptr ),
211     .async_data_slave_b_data_o ( src.b_data ),
212     .async_data_slave_b_wptr_o ( src.b_wptr ),
213     .async_data_slave_b_rptr_i ( src.b_rptr ),
214     .async_data_slave_ar_data_i ( src.ar_data ),
215     .async_data_slave_ar_wptr_i ( src.ar_wptr ),
216     .async_data_slave_ar_rptr_o ( src.ar_rptr ),
217     .async_data_slave_r_data_o ( src.r_data ),
218     .async_data_slave_r_wptr_o ( src.r_wptr ),
219     .async_data_slave_r_rptr_i ( src.r_rptr ),
220     .dst_clk_i,
221     .dst_rst_ni,
222     .dst_req_o ( dst_req ),
223     .dst_resp_i ( dst_resp )
224 );
225
226 `AXI_ASSIGN_FROM_REQ(dst, dst_req)
227 `AXI_ASSIGN_TO_RESP(dst_resp, dst)
228
229 endmodule
230
231
232 module axi_lite_cdc_dst_intf #(
233     parameter int unsigned AXI_ADDR_WIDTH = 0,
234     parameter int unsigned AXI_DATA_WIDTH = 0,
235     /// Depth of the FIFO crossing the clock domain, given as
236     2**LOG_DEPTH.
237     parameter int unsigned LOG_DEPTH = 1,
238     /// Number of synchronization registers to insert on the
239     async pointers
240     parameter int unsigned SYNC_STAGES = 2
241 ) (
242     // asynchronous slave port
243     AXI_LITE_ASYNC_GRAY.Slave src,
244     // synchronous master port - clocked by `dst_clk_i`
245     input logic dst_clk_i,
246     input logic dst_rst_ni,
247     AXI_LITE.Master dst
248 );
249
250 typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
251 typedef logic [AXI_DATA_WIDTH-1:0] data_t;
252 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
253 `AXI_LITE_TYPEDEF_AW_CHAN_T(aw_chan_t, addr_t)
254 `AXI_LITE_TYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t)
255 `AXI_LITE_TYPEDEF_B_CHAN_T(b_chan_t)
256 `AXI_LITE_TYPEDEF_AR_CHAN_T(ar_chan_t, addr_t)
257 `AXI_LITE_TYPEDEF_R_CHAN_T(r_chan_t, data_t)
258 `AXI_LITE_TYPEDEF_REQ_T(req_t, aw_chan_t, w_chan_t,
259 ar_chan_t)
260 `AXI_LITE_TYPEDEF_RESP_T(resp_t, b_chan_t, r_chan_t)
261
262 req_t dst_req;
263 resp_t dst_resp;
264
265 axi_cdc_dst #(
266     .aw_chan_t ( aw_chan_t ),
267     .w_chan_t ( w_chan_t ),
268     .b_chan_t ( b_chan_t ),
269     .ar_chan_t ( ar_chan_t ),
270     .r_chan_t ( r_chan_t ),
271     .axi_req_t ( req_t ),
272     .axi_resp_t ( resp_t ),
273     .LogDepth ( LOG_DEPTH ),
274     .SyncStages ( SYNC_STAGES )
275 ) i_axi_cdc_dst (
276     .async_data_slave_aw_data_i ( src.aw_data ),
277     .async_data_slave_aw_wptr_i ( src.aw_wptr ),
278     .async_data_slave_aw_rptr_o ( src.aw_rptr ),
279     .async_data_slave_w_data_i ( src.w_data ),
280     .async_data_slave_w_wptr_i ( src.w_wptr ),
281     .async_data_slave_w_rptr_o ( src.w_rptr ),
282     .async_data_slave_b_data_o ( src.b_data ),
283     .async_data_slave_b_wptr_o ( src.b_wptr ),
284     .async_data_slave_b_rptr_i ( src.b_rptr ),
285     .async_data_slave_ar_data_i ( src.ar_data ),

```

```

283     .async_data_slave_ar_wptr_i ( src.ar_wptr ),
284     .async_data_slave_ar_rptr_o ( src.ar_rptr ),
285     .async_data_slave_r_data_o ( src.r_data ),
286     .async_data_slave_r_wptr_o ( src.r_wptr ),
287     .async_data_slave_r_rptr_i ( src.r_rptr ),
288     .dst_clk_i,
289     .dst_rst_ni,
290     .dst_req_o ( dst_req ),
291     .dst_resp_i ( dst_resp )
292 );
293
294 `AXI_LITE_ASSIGN_FROM_REQ(dst, dst_req)
295 `AXI_LITE_ASSIGN_TO_RESP(dst_resp, dst)
296
297 endmodule

```

8. pulp-platform_axi/src/axi_cdc_src.sv (296 lines)

```

1 // Copyright (c) 2019-2020 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the Solder-
4 pad Hardware
5
6 // License, Version 0.51 (the "License"); you may not use this
7 file except in
8 // compliance with the License. You may obtain a copy of the
9 License at
10 // http://solderpad.org/licenses/SHL-0.51. Unless required by
11 applicable law
12 // or agreed to in writing, software, hardware and materials
13 distributed under
14 // this license is distributed on an "AS IS" BASIS, WITHOUT
15 WARRANTIES OR
16 // CONDITIONS OF ANY KIND, either express or implied. See the
17 License for the
18 // specific language governing permissions and limitations un-
19 der the License.
20 //
21 // Authors:
22 // - Luca Valente <luca.valente@unibo.it>
23 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
24
25 `include "axi/assign.svh"
26 `include "axi/typedef.svh"
27
28 /// Source-clock-domain half of the AXI CDC crossing.
29
30 /// For each of the five AXI channels, this module instanti-
31 ates the source or destination half of
32 /// a CDC FIFO. IMPORTANT: For each AXI channel, you MUST
33 properly constrain three paths through
34 /// the FIFO; see the header of `cdc_fifo_gray` for instruc-
35 tions.
36
37 module axi_cdc_src #(
38     /// Depth of the FIFO crossing the clock domain, given as
39     2**LOG_DEPTH.
40     parameter int unsigned LogDepth = 1,
41     /// Number of synchronization registers to insert on the
42     async pointers
43     parameter int unsigned SyncStages = 2,
44     parameter type aw_chan_t = logic,
45     parameter type w_chan_t = logic,
46     parameter type b_chan_t = logic,
47     parameter type ar_chan_t = logic,
48     parameter type r_chan_t = logic,
49     parameter type axi_req_t = logic,
50     parameter type axi_resp_t = logic
51 ) (
52     // synchronous slave port - clocked by `src_clk_i`
53     input logic src_clk_i,
54     input logic src_rst_ni,
55     input axi_req_t src_req_i,
56     output axi_resp_t src_resp_o,
57     // asynchronous master port
58     output aw_chan_t [2**LogDepth-1:0]
59     async_data_master_aw_data_o,
60     output logic [LogDepth:0]
61     async_data_master_aw_wptr_o,
62     input logic [LogDepth:0]
63     async_data_master_aw_rptr_i,
64     output w_chan_t [2**LogDepth-1:0]
65     async_data_master_w_data_o,
66     output logic [LogDepth:0]
67     async_data_master_w_wptr_o,
68     input logic [LogDepth:0]
69     async_data_master_w_rptr_i,
70     input b_chan_t [2**LogDepth-1:0]
71     async_data_master_b_data_i,
72     input logic [LogDepth:0]
73     async_data_master_b_wptr_i,
74     output logic [LogDepth:0]
75     async_data_master_b_rptr_o,
76     output ar_chan_t [2**LogDepth-1:0]
77     async_data_master_ar_data_o,
78     output logic [LogDepth:0]
79     async_data_master_ar_wptr_o,
80     input logic [LogDepth:0]
81     async_data_master_ar_rptr_i,
82     input r_chan_t [2**LogDepth-1:0]
83     async_data_master_r_data_i,

```



```

56  input logic [LogDepth:0]
async_data_master_r_wptr_i,
57  output logic [LogDepth:0]
async_data_master_r_rptr_o
58 );
59
60 cc_cdc_fifo_gray_src #(
61 // Workaround for a bug in Questa (see comment in
`axi_cdc_dst` for details).
62 `ifdef QUESTA
63 .data_t ( logic [$bits(aw_chan_t)-1:0] ),
64 `else
65 .data_t ( aw_chan_t ),
66 `endif
67 .LogDepth ( LogDepth ),
68 .SyncStages ( SyncStages )
69 ) i_cdc_fifo_gray_src_aw (
70 .src_clk_i,
71 .src_rst_ni,
72 .src_data_i ( src_req_i.aw ),
73 .src_valid_i ( src_req_i.aw_valid ),
74 .src_ready_o ( src_resp_o.aw_ready ),
75 .async_data_o ( async_data_master_aw_data_o ),
76 .async_wptr_o ( async_data_master_aw_wptr_o ),
77 .async_rptr_i ( async_data_master_aw_rptr_i )
78 );
79
80 cc_cdc_fifo_gray_dst #(
81 `ifdef QUESTA
82 .data_t ( logic [$bits(w_chan_t)-1:0] ),
83 `else
84 .data_t ( w_chan_t ),
85 `endif
86 .LogDepth ( LogDepth ),
87 .SyncStages ( SyncStages )
88 ) i_cdc_fifo_gray_src_w (
89 .src_clk_i,
90 .src_rst_ni,
91 .src_data_i ( src_req_i.w ),
92 .src_valid_i ( src_req_i.w_valid ),
93 .src_ready_o ( src_resp_o.w_ready ),
94 .async_data_o ( async_data_master_w_data_o ),
95 .async_wptr_o ( async_data_master_w_wptr_o ),
96 .async_rptr_i ( async_data_master_w_rptr_i )
97 );
98
99 cc_cdc_fifo_gray_dst #(
100 `ifdef QUESTA
101 .data_t ( logic [$bits(b_chan_t)-1:0] ),
102 `else
103 .data_t ( b_chan_t ),
104 `endif
105 .LogDepth ( LogDepth ),
106 .SyncStages ( SyncStages )
107 ) i_cdc_fifo_gray_dst_b (
108 .dst_clk_i ( src_clk_i ),
109 .dst_rst_ni ( src_rst_ni ),
110 .dst_data_o ( src_resp_o.b ),
111 .dst_valid_o ( src_resp_o.b_valid ),
112 .dst_ready_i ( src_req_i.b_ready ),
113 .async_data_i ( async_data_master_b_data_i ),
114 .async_wptr_i ( async_data_master_b_wptr_i ),
115 .async_rptr_o ( async_data_master_b_rptr_o )
116 );
117
118 cc_cdc_fifo_gray_src #(
119 `ifdef QUESTA
120 .data_t ( logic [$bits(ar_chan_t)-1:0] ),
121 `else
122 .data_t ( ar_chan_t ),
123 `endif
124 .LogDepth ( LogDepth ),
125 .SyncStages ( SyncStages )
126 ) i_cdc_fifo_gray_src_ar (
127 .src_clk_i,
128 .src_rst_ni,
129 .src_data_i ( src_req_i.ar ),
130 .src_valid_i ( src_req_i.ar_valid ),
131 .src_ready_o ( src_resp_o.ar_ready ),
132 .async_data_o ( async_data_master_ar_data_o ),
133 .async_wptr_o ( async_data_master_ar_wptr_o ),
134 .async_rptr_i ( async_data_master_ar_rptr_i )
135 );
136
137 cc_cdc_fifo_gray_dst #(
138 `ifdef QUESTA
139 .data_t ( logic [$bits(r_chan_t)-1:0] ),
140 `else
141 .data_t ( r_chan_t ),
142 `endif
143 .LogDepth ( LogDepth ),
144 .SyncStages ( SyncStages )
145 ) i_cdc_fifo_gray_dst_r (
146 .dst_clk_i ( src_clk_i ),
147 .dst_rst_ni ( src_rst_ni ),
148 .dst_data_o ( src_resp_o.r ),
149 .dst_valid_o ( src_resp_o.r_valid ),
150 .dst_ready_i ( src_req_i.r_ready ),
151 .async_data_i ( async_data_master_r_data_i ),
152 .async_wptr_i ( async_data_master_r_wptr_i ),
153 .async_rptr_o ( async_data_master_r_rptr_o )

```

```

154 );
155
156 endmodule
157
158
159 module axi_cdc_src_intf #(
160 parameter int unsigned AXI_ID_WIDTH = 0,
161 parameter int unsigned AXI_ADDR_WIDTH = 0,
162 parameter int unsigned AXI_DATA_WIDTH = 0,
163 parameter int unsigned AXI_USER_WIDTH = 0,
164 // Depth of the FIFO crossing the clock domain, given as
2**LOG_DEPTH.
165 parameter int unsigned LOG_DEPTH = 1,
166 // Number of synchronization registers to insert on the
async pointers
167 parameter int unsigned SYNC_STAGES = 2
168 ) (
169 // synchronous slave port - clocked by `src_clk_i`
170 input logic src_clk_i,
171 input logic src_rst_ni,
172 AXI_BUS.Slave src,
173 // asynchronous master port
174 AXI_BUS.ASYNC_GRAY.Master dst
175 );
176
177 typedef logic [AXI_ID_WIDTH-1:0] id_t;
178 typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
179 typedef logic [AXI_DATA_WIDTH-1:0] data_t;
180 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
181 typedef logic [AXI_USER_WIDTH-1:0] user_t;
182 `AXI_TYPERDEF_AW_CHAN_T(aw_chan_t, addr_t, id_t, user_t)
183 `AXI_TYPERDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)
184 `AXI_TYPERDEF_B_CHAN_T(b_chan_t, id_t, user_t)
185 `AXI_TYPERDEF_AR_CHAN_T(ar_chan_t, addr_t, id_t, user_t)
186 `AXI_TYPERDEF_R_CHAN_T(r_chan_t, data_t, id_t, user_t)
187 `AXI_TYPERDEF_REQ_T(req_t, aw_chan_t, w_chan_t, ar_chan_t)
188 `AXI_TYPERDEF_RESP_T(resp_t, b_chan_t, r_chan_t)
189
190 req_t src_req;
191 resp_t src_resp;
192
193 `AXI_ASSIGN_TO_REQ(src_req, src)
194 `AXI_ASSIGN_FROM_RESP(src, src_resp)
195
196 axi_cdc_src #(
197 .aw_chan_t ( aw_chan_t ),
198 .w_chan_t ( w_chan_t ),
199 .b_chan_t ( b_chan_t ),
200 .ar_chan_t ( ar_chan_t ),
201 .r_chan_t ( r_chan_t ),
202 .axi_req_t ( req_t ),
203 .axi_resp_t ( resp_t ),
204 .LogDepth ( LOG_DEPTH ),
205 .SyncStages ( SYNC_STAGES )
206 ) i_axi_cdc_src (
207 .src_clk_i,
208 .src_rst_ni,
209 .src_req_i ( src_req ),
210 .src_resp_o ( src_resp ),
211 .async_data_master_aw_data_o ( dst.aw_data ),
212 .async_data_master_aw_wptr_o ( dst.aw_wptr ),
213 .async_data_master_aw_rptr_i ( dst.aw_rptr ),
214 .async_data_master_w_data_o ( dst.w_data ),
215 .async_data_master_w_wptr_o ( dst.w_wptr ),
216 .async_data_master_w_rptr_i ( dst.w_rptr ),
217 .async_data_master_b_data_i ( dst.b_data ),
218 .async_data_master_b_wptr_i ( dst.b_wptr ),
219 .async_data_master_b_rptr_o ( dst.b_rptr ),
220 .async_data_master_ar_data_o ( dst.ar_data ),
221 .async_data_master_ar_wptr_o ( dst.ar_wptr ),
222 .async_data_master_ar_rptr_i ( dst.ar_rptr ),
223 .async_data_master_r_data_i ( dst.r_data ),
224 .async_data_master_r_wptr_i ( dst.r_wptr ),
225 .async_data_master_r_rptr_o ( dst.r_rptr )
226 );
227
228 endmodule
229
230
231 module axi_lite_cdc_src_intf #(
232 parameter int unsigned AXI_ADDR_WIDTH = 0,
233 parameter int unsigned AXI_DATA_WIDTH = 0,
234 // Depth of the FIFO crossing the clock domain, given as
2**LOG_DEPTH.
235 parameter int unsigned LOG_DEPTH = 1,
236 // Number of synchronization registers to insert on the
async pointers
237 parameter int unsigned SYNC_STAGES = 2
238 ) (
239 // synchronous slave port - clocked by `src_clk_i`
240 input logic src_clk_i,
241 input logic src_rst_ni,
242 AXI_BUS.Slave src,
243 // asynchronous master port
244 AXI_LITE_ASYNCH_GRAY.Master dst
245 );
246
247 typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
248 typedef logic [AXI_DATA_WIDTH-1:0] data_t;
249 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
250 `AXI_LITE_TYPERDEF_AW_CHAN_T(aw_chan_t, addr_t)

```

```

251 `AXI_LITE_TYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t)
252 `AXI_LITE_TYPEDEF_B_CHAN_T(b_chan_t)
253 `AXI_LITE_TYPEDEF_AR_CHAN_T(ar_chan_t, addr_t)
254 `AXI_LITE_TYPEDEF_R_CHAN_T(r_chan_t, data_t)
255 `AXI_LITE_TYPEDEF_REQ_T(req_t, aw_chan_t, w_chan_t,
ar_chan_t)
256 `AXI_LITE_TYPEDEF_RESP_T(resp_t, b_chan_t, r_chan_t)
257
258 req_t src_req;
259 resp_t src_resp;
260
261 `AXI_LITE_ASSIGN_TO_REQ(src_req, src)
262 `AXI_LITE_ASSIGN_FROM_RESP(src, src_resp)
263
264 axi_cdc_src #(
265     .aw_chan_t ( aw_chan_t ),
266     .w_chan_t ( w_chan_t ),
267     .b_chan_t ( b_chan_t ),
268     .ar_chan_t ( ar_chan_t ),
269     .r_chan_t ( r_chan_t ),
270     .axi_req_t ( req_t ),
271     .axi_resp_t ( resp_t ),
272     .LogDepth ( LOG_DEPTH ),
273     .SyncStages ( SYNC_STAGES )
274 ) i_axi_cdc_src (
275     .src_clk_i,
276     .src_rst_ni,
277     .src_req_i ( src_req ),
278     .src_resp_o ( src_resp ),
279     .async_data_master_aw_data_o ( dst.aw_data ),
280     .async_data_master_aw_wptr_o ( dst.aw_wptr ),
281     .async_data_master_aw_rptr_i ( dst.aw_rptr ),
282     .async_data_master_w_data_o ( dst.w_data ),
283     .async_data_master_w_wptr_o ( dst.w_wptr ),
284     .async_data_master_w_rptr_i ( dst.w_rptr ),
285     .async_data_master_b_data_i ( dst.b_data ),
286     .async_data_master_b_wptr_i ( dst.b_wptr ),
287     .async_data_master_b_rptr_o ( dst.b_rptr ),
288     .async_data_master_ar_data_o ( dst.ar_data ),
289     .async_data_master_ar_wptr_o ( dst.ar_wptr ),
290     .async_data_master_ar_rptr_i ( dst.ar_rptr ),
291     .async_data_master_r_data_i ( dst.r_data ),
292     .async_data_master_r_wptr_i ( dst.r_wptr ),
293     .async_data_master_r_rptr_o ( dst.r_rptr )
294 );
295
296 endmodule

```

9. pulp-platform_axi/src/axi_chan_compare.sv (266 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10 //
11 // Authors:
12 // - Thomas Benz <tbenz@iis.ee.ethz.ch>
13 // - Paul Scheffler <paulsc@iis.ee.ethz.ch>
14 // - Tim Fischer <fischetti@iis.ee.ethz.ch>
15
16 /// Non-synthesizable module comparing two AXI channels of the
  same type
17 module axi_chan_compare #(
18     /// Ignore ID field if it was remapped
19     parameter bit IgnoreId = 1'b0,
20     /// Allow reordered responses of different IDs,
21     /// not compatible with 'IgnoreId'
22     parameter bit AllowReordering = 1'b0,
23     /// AXI ID Width
24     parameter int unsigned IdWidth = 1,
25     parameter type aw_chan_t = logic,
26     parameter type w_chan_t = logic,
27     parameter type b_chan_t = logic,
28     parameter type ar_chan_t = logic,
29     parameter type r_chan_t = logic,
30     parameter type req_t = logic,
31     parameter type resp_t = logic
32 )(
33     input logic clk_a_i,
34     input logic clk_b_i,
35     input req_t axi_a_req,
36     input resp_t axi_a_res,
37     input req_t axi_b_req,
38     input resp_t axi_b_res
39 );

```

```

40
41 function automatic void print_aw (
42     input aw_chan_t aw_expected,
43     input aw_chan_t aw_received
44 );
45     // verilog_lint: waive-start line-length
46     $display("AW      | expec-
ted
ceived
| re-
");
47     $dis-
play("-----|-----
-----|-----
-----");
48     $display("id:      | %64d | %64d", aw_expected.id,
aw_received.id);
49     $display("addr:    | %64x | %64x", aw_expected.addr,
aw_received.addr);
50     $display("len:     | %64d | %64d", aw_expected.len,
aw_received.len);
51     $display("size:    | %64d | %64d", aw_expected.size,
aw_received.size);
52     $display("burst:   | %64d | %64d", aw_expected.burst,
aw_received.burst);
53     $display("lock:    | %64d | %64d", aw_expected.lock,
aw_received.lock);
54     $display("cache:   | %64d | %64d", aw_expected.cache,
aw_received.cache);
55     $display("prot:    | %64d | %64d", aw_expected.prot,
aw_received.prot);
56     $display("qos:     | %64d | %64d", aw_expected.qos,
aw_received.qos);
57     $display("region:  | %64d | %64d", aw_expected.region,
aw_received.region);
58     $display("user:    | %64d | %64d", aw_expected.user,
aw_received.user);
59     $display("atop:    | %64d | %64d", aw_expected.atop,
aw_received.atop);
60     $dis-
play("-----|-----
-----|-----
-----");
61     // verilog_lint: waive-stop line-length
62     endfunction
63
64 function automatic void print_ar (
65     input ar_chan_t ar_expected,
66     input ar_chan_t ar_received
67 );
68     // verilog_lint: waive-start line-length
69     $display("AR      | expec-
ted
ceived
| re-
");
70     $dis-
play("-----|-----
-----|-----
-----");
71     $display("id:      | %64d | %64d", ar_expected.id,
ar_received.id);
72     $display("addr:    | %64x | %64x", ar_expected.addr,
ar_received.addr);
73     $display("len:     | %64d | %64d", ar_expected.len,
ar_received.len);
74     $display("size:    | %64d | %64d", ar_expected.size,
ar_received.size);
75     $display("burst:   | %64d | %64d", ar_expected.burst,
ar_received.burst);
76     $display("lock:    | %64d | %64d", ar_expected.lock,
ar_received.lock);
77     $display("cache:   | %64d | %64d", ar_expected.cache,
ar_received.cache);
78     $display("prot:    | %64d | %64d", ar_expected.prot,
ar_received.prot);
79     $display("qos:     | %64d | %64d", ar_expected.qos,
ar_received.qos);
80     $display("region:  | %64d | %64d", ar_expected.region,
ar_received.region);
81     $display("user:    | %64d | %64d", ar_expected.user,
ar_received.user);
82     $dis-
play("-----|-----
-----|-----
-----");
83     // verilog_lint: waive-stop line-length
84     endfunction
85
86 function automatic void print_w (
87     input w_chan_t w_expected,
88     input w_chan_t w_received
89 );
90     // verilog_lint: waive-start line-length
91     $display("W      | expec-
ted
ceived
| re-
");
92     $dis-
play("-----|-----
-----|-----
-----");
93     $display("data:    | %64x | %64x", w_expected.data,
w_received.data);
94     $display("strb:    | %64d | %64d", w_expected.strb,
w_received.strb);

```

```

95      $display("last:   | %64d | %64d", w_expected.last,
w_received.last);
96      $display("user:   | %64d | %64d", w_expected.user,
w_received.user);
97      $dis-
play("-----|-----
-----");
98      // verilog_lint: waive-stop line-length
99      endfunction
100
101      function automatic void print_b (
102          input b_chan_t b_expected,
103          input b_chan_t b_received
104      );
105          // verilog_lint: waive-start line-length
106          $display("B      | expec-
ted      | re-
ceived");
107          $dis-
play("-----|-----
-----");
108          $display("id:    | %64d | %64d", b_expected.id, b_re-
ceived.id);
109          $display("resp:   | %64d | %64d", b_expected.resp,
b_received.resp);
110          $display("user:   | %64d | %64d", b_expected.user,
b_received.user);
111          $dis-
play("-----|-----
-----");
112          // verilog_lint: waive-stop line-length
113          endfunction
114
115      function automatic void print_r (
116          input r_chan_t r_expected,
117          input r_chan_t r_received
118      );
119          // verilog_lint: waive-start line-length
120          $display("R      | expec-
ted      | re-
ceived");
121          $dis-
play("-----|-----
-----");
122          $display("id:    | %64d | %64d", r_expected.id, r_re-
ceived.id);
123          $display("data:   | %64x | %64x", r_expected.data,
r_received.data);
124          $display("resp:   | %64d | %64d", r_expected.resp,
r_received.resp);
125          $display("last:   | %64d | %64d", r_expected.last,
r_received.last);
126          $display("user:   | %64d | %64d", r_expected.user,
r_received.user);
127          $dis-
play("-----|-----
-----");
128          // verilog_lint: waive-stop line-length
129          endfunction
130
131      localparam NumIds = (AllowReordering)? 2**IdWidth : 1;
132
133      // queues
134      aw_chan_t aw_queue [NumIds-1:0][$];
135      w_chan_t w_queue   [$];
136      b_chan_t b_queue   [NumIds-1:0][$];
137      ar_chan_t ar_queue [NumIds-1:0][$];
138      r_chan_t r_queue   [NumIds-1:0][$];
139
140      // requests generated at axi A: enqueue elements
141      always_ff @(posedge clk_a_i) begin : proc_enqueue_a
142          // aw
143          if (axi_a_req.aw_valid & axi_a_res.aw_ready)
144              if (AllowReordering)
145                  aw_queue[axi_a_req.aw.id].push_back(axi_a_req.aw);
146              else aw_queue[0].push_back(axi_a_req.aw);
147          // w
148          if (axi_a_req.w_valid & axi_a_res.w_ready)
149              w_queue.push_back(axi_a_req.w);
150          // ar
151          if (axi_a_req.ar_valid & axi_a_res.ar_ready)
152              if (AllowReordering)
153                  ar_queue[axi_a_req.ar.id].push_back(axi_a_req.ar);
154              else ar_queue[0].push_back(axi_a_req.ar);
155          end
156          // responses generated at axi B: enqueue elements
157          always_ff @(posedge clk_b_i) begin : proc_enqueue_b
158              // b
159              if (axi_b_res.b_valid & axi_b_req.b_ready)
160                  if (AllowReordering)
161                      b_queue[axi_b_res.b.id].push_back(axi_b_res.b);
162                  else b_queue[0].push_back(axi_b_res.b);
163              // r
164              if (axi_b_res.r_valid & axi_b_req.r_ready)
165                  if (AllowReordering)

```

```

r_queue[axi_b_res.r.id].push_back(axi_b_res.r);
164          else r_queue[0].push_back(axi_b_res.r);
165          end
166
167      // requests arriving at axi B from A: dequeue elements and
check
168      always_ff @(posedge clk_b_i) begin : proc_d-
enqueue_and_check_b
169          // aw
170          if (axi_b_req.aw_valid & axi_b_res.aw_ready) begin
171              automatic aw_chan_t aw_exp, aw_rcv;
172              if (AllowReordering) begin
173                  if (aw_queue[axi_b_req.aw.id].size() == 0)
174                      $error("AW queue is empty!");
175                  aw_exp =
aw_queue[axi_b_req.aw.id].pop_front(); // verilog_lint: waive al-
ways-ff-non-blocking
176              end else begin
177                  if (aw_queue[0].size() == 0) $error("AW queue
is empty!");
178                  aw_exp = aw_queue[0].pop_front(); // veri-
log_lint: waive always-ff-non-blocking
179              end
180              aw_rcv = axi_b_req.aw;
181              if (IgnoreId) begin
182                  aw_exp.id = 'X;
183                  aw_rcv.id = 'X;
184              end
185              if (aw_exp != aw_rcv) begin
186                  $error("AW mismatch!");
187                  print_aw(aw_exp, aw_rcv);
188              end
189              // w
190              if (axi_b_req.w_valid & axi_b_res.w_ready) begin
191                  automatic w_chan_t w_exp, w_rcv;
192                  if (w_queue.size() == 0) $error("W queue is
empty!");
193                  w_exp = w_queue.pop_front(); // verilog_lint:
waive always-ff-non-blocking
194                  w_rcv = axi_b_req.w;
195                  if (w_exp != w_rcv) begin
196                      $error("W mismatch!");
197                      print_w(w_exp, w_rcv);
198                  end
199              end
200              // ar
201              if (axi_b_req.ar_valid & axi_b_res.ar_ready) begin
202                  automatic ar_chan_t ar_exp, ar_rcv;
203                  if (AllowReordering) begin
204                      if (ar_queue[axi_b_req.ar.id].size() == 0)
205                          $error("AR queue is empty!");
206                      ar_exp =
ar_queue[axi_b_req.ar.id].pop_front(); // verilog_lint: waive al-
ways-ff-non-blocking
207                  end else begin
208                      if (ar_queue[0].size() == 0) $error("AR queue
is empty!");
209                      ar_exp = ar_queue[0].pop_front(); // veri-
log_lint: waive always-ff-non-blocking
210                  end
211                  ar_rcv = axi_b_req.ar;
212                  if (IgnoreId) begin
213                      ar_exp.id = 'X;
214                      ar_rcv.id = 'X;
215                  end
216                  if (ar_exp != ar_rcv) begin
217                      $error("AR mismatch!");
218                      print_ar(ar_exp, ar_rcv);
219                  end
220              end
221          end
222      // responses arriving at axi A from B: dequeue elements
and check
223      always_ff @(posedge clk_a_i) begin : proc_d-
enqueue_and_check_a
224          // b
225          if (axi_a_res.b_valid & axi_a_req.b_ready) begin
226              automatic b_chan_t b_exp, b_rcv;
227              if (AllowReordering) begin
228                  if (b_queue[axi_a_res.b.id].size() == 0) $er-
ror("B queue is empty!");
229                  b_exp =
b_queue[axi_a_res.b.id].pop_front(); // verilog_lint: waive always-
ff-non-blocking
230              end else begin
231                  if (b_queue[0].size() == 0) $error("B queue is
empty!");
232                  b_exp = b_queue[0].pop_front(); // veri-
log_lint: waive always-ff-non-blocking
233              end
234              b_rcv = axi_a_res.b;
235              if (IgnoreId) begin
236                  b_exp.id = 'X;
237                  b_rcv.id = 'X;
238              end
239              if (b_exp != b_rcv) begin
240                  $error("B mismatch!");

```

```

241         print_b(b_exp, b_recv);
242     end
243 end
244 // r
245 if (axi_a_res.r_valid & axi_a_req.r_ready) begin
246     automatic r_chan_t r_exp, r_recv;
247     if (AllowReordering) begin
248         if (r_queue[axi_a_res.r.id].size() == 0) $error("R queue is empty!");
249         r_exp =
r_queue[axi_a_res.r.id].pop_front(); // verilog_lint: waive always-
ff-non-blocking
250     end else begin
251         if (r_queue[0].size() == 0) $error("R queue is
empty!");
252         r_exp = r_queue[0].pop_front(); // veri-
log_lint: waive always-ff-non-blocking
253     end
254     r_recv = axi_a_res.r;
255     if (IgnoreId) begin
256         r_exp.id = 'X;
257         r_recv.id = 'X;
258     end
259     if (r_exp !== r_recv) begin
260         $error("R mismatch!");
261         print_r(r_exp, r_recv);
262     end
263 end
264 end
265
266 endmodule : axi_chan_compare

```

10. pulp-platform_axi/src/axi_cut.sv (295 lines)

```

1 // Copyright (c) 2014-2018 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the License.
11 //
12 // Authors:
13 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
14 // - Fabian Schuiki <fschuiki@iis.ee.ethz.ch>
15 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
16
17 /// An AXI4 cut.
18 ///
19 /// Breaks all combinatorial paths between its input and out-
put.
20 module axi_cut #(
21     // bypass enable
22     parameter bit Bypass = 1'b0,
23     parameter bit BypassAw = Bypass,
24     parameter bit BypassW = Bypass,
25     parameter bit BypassB = Bypass,
26     parameter bit BypassAr = Bypass,
27     parameter bit BypassR = Bypass,
28     // AXI channel structs
29     parameter type aw_chan_t = logic,
30     parameter type w_chan_t = logic,
31     parameter type b_chan_t = logic,
32     parameter type ar_chan_t = logic,
33     parameter type r_chan_t = logic,
34     // AXI request & response structs
35     parameter type axi_req_t = logic,
36     parameter type axi_resp_t = logic
37 ) (
38     input logic clk_i,
39     input logic rst_ni,
40     // slave port
41     input axi_req_t slv_req_i,
42     output axi_resp_t slv_resp_o,
43     // master port
44     output axi_req_t mst_req_o,
45     input axi_resp_t mst_resp_i
46 );
47
48 // a spill register for each channel
49 cc_spill_register #(
50     .data_t ( aw_chan_t ),
51     .Bypass ( BypassAw )
52 ) i_reg_aw (
53     .clk_i ( clk_i ),
54     .rst_ni ( rst_ni ),
55     .clr_i ( 1'b0 ),

```

```

56     .valid_i ( slv_req_i.aw_valid ),
57     .ready_o ( slv_resp_o.aw_ready ),
58     .data_i ( slv_req_i.aw ),
59     .valid_o ( mst_req_o.aw_valid ),
60     .ready_i ( mst_resp_i.aw_ready ),
61     .data_o ( mst_req_o.aw )
62 );
63
64 cc_spill_register #(
65     .data_t ( w_chan_t ),
66     .Bypass ( BypassW )
67 ) i_reg_w (
68     .clk_i ( clk_i ),
69     .rst_ni ( rst_ni ),
70     .clr_i ( 1'b0 ),
71     .valid_i ( slv_req_i.w_valid ),
72     .ready_o ( slv_resp_o.w_ready ),
73     .data_i ( slv_req_i.w ),
74     .valid_o ( mst_req_o.w_valid ),
75     .ready_i ( mst_resp_i.w_ready ),
76     .data_o ( mst_req_o.w )
77 );
78
79 cc_spill_register #(
80     .data_t ( b_chan_t ),
81     .Bypass ( BypassB )
82 ) i_reg_b (
83     .clk_i ( clk_i ),
84     .rst_ni ( rst_ni ),
85     .clr_i ( 1'b0 ),
86     .valid_i ( mst_resp_i.b_valid ),
87     .ready_o ( mst_req_o.b_ready ),
88     .data_i ( mst_resp_i.b ),
89     .valid_o ( slv_resp_o.b_valid ),
90     .ready_i ( slv_req_i.b_ready ),
91     .data_o ( slv_resp_o.b )
92 );
93
94 cc_spill_register #(
95     .data_t ( ar_chan_t ),
96     .Bypass ( BypassAr )
97 ) i_reg_ar (
98     .clk_i ( clk_i ),
99     .rst_ni ( rst_ni ),
100     .clr_i ( 1'b0 ),
101     .valid_i ( slv_req_i.ar_valid ),
102     .ready_o ( slv_resp_o.ar_ready ),
103     .data_i ( slv_req_i.ar ),
104     .valid_o ( mst_req_o.ar_valid ),
105     .ready_i ( mst_resp_i.ar_ready ),
106     .data_o ( mst_req_o.ar )
107 );
108
109 cc_spill_register #(
110     .data_t ( r_chan_t ),
111     .Bypass ( BypassR )
112 ) i_reg_r (
113     .clk_i ( clk_i ),
114     .rst_ni ( rst_ni ),
115     .clr_i ( 1'b0 ),
116     .valid_i ( mst_resp_i.r_valid ),
117     .ready_o ( mst_req_o.r_ready ),
118     .data_i ( mst_resp_i.r ),
119     .valid_o ( slv_resp_o.r_valid ),
120     .ready_i ( slv_req_i.r_ready ),
121     .data_o ( slv_resp_o.r )
122 );
123 endmodule
124
125 `include "axi/assign.svh"
126 `include "axi/typedef.svh"
127
128 // interface wrapper
129 module axi_cut_intf #(
130     // Bypass enable
131     parameter bit BYPASS = 1'b0,
132     parameter bit BYPASS_AW = BYPASS,
133     parameter bit BYPASS_W = BYPASS,
134     parameter bit BYPASS_B = BYPASS,
135     parameter bit BYPASS_AR = BYPASS,
136     parameter bit BYPASS_R = BYPASS,
137     // The address width.
138     parameter int unsigned ADDR_WIDTH = 0,
139     // The data width.
140     parameter int unsigned DATA_WIDTH = 0,
141     // The ID width.
142     parameter int unsigned ID_WIDTH = 0,
143     // The user data width.
144     parameter int unsigned USER_WIDTH = 0
145 ) (
146     input logic clk_i,
147     input logic rst_ni,
148     AXI_BUS.Slave in,
149     AXI_BUS.Master out
150 );
151
152 typedef logic [ID_WIDTH-1:0] id_t;
153 typedef logic [ADDR_WIDTH-1:0] addr_t;
154 typedef logic [DATA_WIDTH-1:0] data_t;
155 typedef logic [DATA_WIDTH/8-1:0] strb_t;
156 typedef logic [USER_WIDTH-1:0] user_t;

```

```

157
158 `AXI_TPEDEF_AW_CHAN_T(aw_chan_t, addr_t, id_t, user_t)
159 `AXI_TPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)
160 `AXI_TPEDEF_B_CHAN_T(b_chan_t, id_t, user_t)
161 `AXI_TPEDEF_AR_CHAN_T(ar_chan_t, addr_t, id_t, user_t)
162 `AXI_TPEDEF_R_CHAN_T(r_chan_t, data_t, id_t, user_t)
163 `AXI_TPEDEF_REQ_T(axi_req_t, aw_chan_t, w_chan_t,
ar_chan_t)
164 `AXI_TPEDEF_RESP_T(axi_resp_t, b_chan_t, r_chan_t)
165
166 axi_req_t slv_req, mst_req;
167 axi_resp_t slv_resp, mst_resp;
168
169 `AXI_ASSIGN_TO_REQ(slv_req, in)
170 `AXI_ASSIGN_FROM_RESP(in, slv_resp)
171
172 `AXI_ASSIGN_FROM_REQ(out, mst_req)
173 `AXI_ASSIGN_TO_RESP(mst_resp, out)
174
175 axi_cut #(
176     .Bypass      ( BYPASS ),
177     .BypassAw    ( BYPASS_AW ),
178     .BypassW     ( BYPASS_W ),
179     .BypassB     ( BYPASS_B ),
180     .BypassAr    ( BYPASS_AR ),
181     .BypassR     ( BYPASS_R ),
182     .aw_chan_t   ( aw_chan_t ),
183     .w_chan_t    ( w_chan_t ),
184     .b_chan_t    ( b_chan_t ),
185     .ar_chan_t   ( ar_chan_t ),
186     .r_chan_t    ( r_chan_t ),
187     .axi_req_t   ( axi_req_t ),
188     .axi_resp_t  ( axi_resp_t )
189 ) i_axi_cut (
190     .clk_i,
191     .rst_ni,
192     .slv_req_i ( slv_req ),
193     .slv_resp_o ( slv_resp ),
194     .mst_req_o  ( mst_req ),
195     .mst_resp_i ( mst_resp )
196 );
197
198 // Check the invariants.
199 // pragma translate_off
200 `ifndef VERILATOR
201 initial begin
202     assert (ADDR_WIDTH > 0) else $fatal(1, "Wrong addr width
parameter");
203     assert (DATA_WIDTH > 0) else $fatal(1, "Wrong data width
parameter");
204     assert (ID_WIDTH > 0) else $fatal(1, "Wrong id width
parameter");
205     assert (USER_WIDTH > 0) else $fatal(1, "Wrong user width
parameter");
206     assert (in.AXI_ADDR_WIDTH == ADDR_WIDTH) else $fatal(1,
"Wrong interface definition");
207     assert (in.AXI_DATA_WIDTH == DATA_WIDTH) else $fatal(1,
"Wrong interface definition");
208     assert (in.AXI_ID_WIDTH == ID_WIDTH) else $fatal(1,
"Wrong interface definition");
209     assert (in.AXI_USER_WIDTH == USER_WIDTH) else $fatal(1,
"Wrong interface definition");
210     assert (out.AXI_ADDR_WIDTH == ADDR_WIDTH) else $fatal(1,
"Wrong interface definition");
211     assert (out.AXI_DATA_WIDTH == DATA_WIDTH) else $fatal(1,
"Wrong interface definition");
212     assert (out.AXI_ID_WIDTH == ID_WIDTH) else $fatal(1,
"Wrong interface definition");
213     assert (out.AXI_USER_WIDTH == USER_WIDTH) else $fatal(1,
"Wrong interface definition");
214 end
215 `endif
216 // pragma translate_on
217 endmodule
218
219 module axi_lite_cut_intf #(
220     // bypass enable
221     parameter bit BYPASS = 1'b0,
222     parameter bit BYPASS_AW = BYPASS,
223     parameter bit BYPASS_W = BYPASS,
224     parameter bit BYPASS_B = BYPASS,
225     parameter bit BYPASS_AR = BYPASS,
226     parameter bit BYPASS_R = BYPASS,
227     /// The address width.
228     parameter int unsigned ADDR_WIDTH = 0,
229     /// The data width.
230     parameter int unsigned DATA_WIDTH = 0
231 ) (
232     input logic clk_i ,
233     input logic rst_ni ,
234     AXI_LITE.Slave in ,
235     AXI_LITE.Master out
236 );
237
238 typedef logic [ADDR_WIDTH-1:0] addr_t;
239 typedef logic [DATA_WIDTH-1:0] data_t;
240 typedef logic [DATA_WIDTH/8-1:0] strb_t;
241
242 `AXI_LITE_TPEDEF_AW_CHAN_T(aw_chan_t, addr_t)
243 `AXI_LITE_TPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t)
244 `AXI_LITE_TPEDEF_B_CHAN_T(b_chan_t)

```

```

245 `AXI_LITE_TPEDEF_AR_CHAN_T(ar_chan_t, addr_t)
246 `AXI_LITE_TPEDEF_R_CHAN_T(r_chan_t, data_t)
247 `AXI_LITE_TPEDEF_REQ_T(axi_req_t, aw_chan_t, w_chan_t,
ar_chan_t)
248 `AXI_LITE_TPEDEF_RESP_T(axi_resp_t, b_chan_t, r_chan_t)
249
250 axi_req_t slv_req, mst_req;
251 axi_resp_t slv_resp, mst_resp;
252
253 `AXI_LITE_ASSIGN_TO_REQ(slv_req, in)
254 `AXI_LITE_ASSIGN_FROM_RESP(in, slv_resp)
255
256 `AXI_LITE_ASSIGN_FROM_REQ(out, mst_req)
257 `AXI_LITE_ASSIGN_TO_RESP(mst_resp, out)
258
259 axi_cut #(
260     .Bypass      ( BYPASS ),
261     .BypassAw    ( BYPASS_AW ),
262     .BypassW     ( BYPASS_W ),
263     .BypassB     ( BYPASS_B ),
264     .BypassAr    ( BYPASS_AR ),
265     .BypassR     ( BYPASS_R ),
266     .aw_chan_t   ( aw_chan_t ),
267     .w_chan_t    ( w_chan_t ),
268     .b_chan_t    ( b_chan_t ),
269     .ar_chan_t   ( ar_chan_t ),
270     .r_chan_t    ( r_chan_t ),
271     .axi_req_t   ( axi_req_t ),
272     .axi_resp_t  ( axi_resp_t )
273 ) i_axi_cut (
274     .clk_i,
275     .rst_ni,
276     .slv_req_i ( slv_req ),
277     .slv_resp_o ( slv_resp ),
278     .mst_req_o  ( mst_req ),
279     .mst_resp_i ( mst_resp )
280 );
281
282 // Check the invariants.
283 // pragma translate_off
284 `ifndef VERILATOR
285 initial begin
286     assert (ADDR_WIDTH > 0) else $fatal(1, "Wrong addr width
parameter");
287     assert (DATA_WIDTH > 0) else $fatal(1, "Wrong data width
parameter");
288     assert (in.AXI_ADDR_WIDTH == ADDR_WIDTH) else $fatal(1,
"Wrong interface definition");
289     assert (in.AXI_DATA_WIDTH == DATA_WIDTH) else $fatal(1,
"Wrong interface definition");
290     assert (out.AXI_ADDR_WIDTH == ADDR_WIDTH) else $fatal(1,
"Wrong interface definition");
291     assert (out.AXI_DATA_WIDTH == DATA_WIDTH) else $fatal(1,
"Wrong interface definition");
292 end
293 `endif
294 // pragma translate_on
295 endmodule

```

11. pulp-platform_axi/src/axi_delayer.sv (203 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this license is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10 //
11 // Authors:
12 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
13 // - Florian Zaruba <zarubaf@iis.ee.ethz.ch>
14 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
15
16 /// Synthesizable module that (randomly) delays AXI channels.
17 module axi_delayer #(
18     // AXI channel types
19     parameter type aw_chan_t = logic,
20     parameter type w_chan_t = logic,
21     parameter type b_chan_t = logic,
22     parameter type ar_chan_t = logic,
23     parameter type r_chan_t = logic,
24     // AXI request & response types
25     parameter type axi_req_t = logic,
26     parameter type axi_resp_t = logic,
27     // delay parameters
28     parameter bit StallRandomInput = 0,
29     parameter bit StallRandomOutput = 0,
30     parameter int unsigned FixedDelayInput = 1,

```



```

31 parameter int unsigned FixedDelayOutput = 1
32 ) (
33   input logic      clk_i,      // Clock
34   input logic      rst_ni,     // Asynchronous reset active
35   // slave port
36   input axi_req_t  slv_req_i,
37   output axi_resp_t slv_resp_o,
38   // master port
39   output axi_req_t  mst_req_o,
40   input  axi_resp_t mst_resp_i
41 );
42 // AW
43 cc_stream_delay #(
44   .StallRandom ( StallRandomInput ),
45   .FixedDelay  ( FixedDelayInput ),
46   .payload_t   ( aw_chan_t )
47 ) i_stream_delay_aw (
48   .clk_i,
49   .rst_ni,
50   .clr_i   ( 1'b0 ),
51   .payload_i ( slv_req_i.aw ),
52   .ready_o  ( slv_resp_o.aw_ready ),
53   .valid_i  ( slv_req_i.aw_valid ),
54   .payload_o ( mst_req_o.aw ),
55   .ready_i  ( mst_resp_i.aw_ready ),
56   .valid_o  ( mst_req_o.aw_valid )
57 );
58 // AR
59 cc_stream_delay #(
60   .StallRandom ( StallRandomInput ),
61   .FixedDelay  ( FixedDelayInput ),
62   .payload_t   ( ar_chan_t )
63 ) i_stream_delay_ar (
64   .clk_i,
65   .rst_ni,
66   .clr_i   ( 1'b0 ),
67   .payload_i ( slv_req_i.ar ),
68   .ready_o  ( slv_resp_o.ar_ready ),
69   .valid_i  ( slv_req_i.ar_valid ),
70   .payload_o ( mst_req_o.ar ),
71   .ready_i  ( mst_resp_i.ar_ready ),
72   .valid_o  ( mst_req_o.ar_valid )
73 );
74 // W
75 cc_stream_delay #(
76   .StallRandom ( StallRandomInput ),
77   .FixedDelay  ( FixedDelayInput ),
78   .payload_t   ( w_chan_t )
79 ) i_stream_delay_w (
80   .clk_i,
81   .rst_ni,
82   .clr_i   ( 1'b0 ),
83   .payload_i ( slv_req_i.w ),
84   .ready_o  ( slv_resp_o.w_ready ),
85   .valid_i  ( slv_req_i.w_valid ),
86   .payload_o ( mst_req_o.w ),
87   .ready_i  ( mst_resp_i.w_ready ),
88   .valid_o  ( mst_req_o.w_valid )
89 );
90 // B
91 cc_stream_delay #(
92   .StallRandom ( StallRandomOutput ),
93   .FixedDelay  ( FixedDelayOutput ),
94   .payload_t   ( b_chan_t )
95 ) i_stream_delay_b (
96   .clk_i,
97   .rst_ni,
98   .clr_i   ( 1'b0 ),
99   .payload_i ( mst_resp_i.b ),
100  .ready_o  ( mst_req_o.b_ready ),
101  .valid_i  ( mst_resp_i.b_valid ),
102  .payload_o ( slv_resp_o.b ),
103  .ready_i  ( slv_req_i.b_ready ),
104  .valid_o  ( slv_resp_o.b_valid )
105 );
106 // R
107 cc_stream_delay #(
108   .StallRandom ( StallRandomOutput ),
109   .FixedDelay  ( FixedDelayOutput ),
110   .payload_t   ( r_chan_t )
111 ) i_stream_delay_r (
112   .clk_i,
113   .rst_ni,
114   .clr_i   ( 1'b0 ),
115   .payload_i ( mst_resp_i.r ),
116   .ready_o  ( mst_req_o.r_ready ),
117   .valid_i  ( mst_resp_i.r_valid ),
118   .payload_o ( slv_resp_o.r ),
119   .ready_i  ( slv_req_i.r_ready ),
120   .valid_o  ( slv_resp_o.r_valid )
121 );
122 endmodule
123
124 `include "axi/typedef.svh"
125 `include "axi/assign.svh"
126
127
128
129
130

```

```

131 // interface wrapper
132 module axi_delayer_intf #(
133   // Synopsys DC requires a default value for parameters.
134   parameter int unsigned AXI_ID_WIDTH      = 0,
135   parameter int unsigned AXI_ADDR_WIDTH    = 0,
136   parameter int unsigned AXI_DATA_WIDTH    = 0,
137   parameter int unsigned AXI_USER_WIDTH    = 0,
138   parameter bit          STALL_RANDOM_INPUT = 0,
139   parameter bit          STALL_RANDOM_OUTPUT = 0,
140   parameter int unsigned FIXED_DELAY_INPUT  = 1,
141   parameter int unsigned FIXED_DELAY_OUTPUT = 1
142 ) (
143   input logic      clk_i,
144   input logic      rst_ni,
145   AXI_BUS.Slave    slv,
146   AXI_BUS.Master    mst
147 );
148
149 typedef logic [AXI_ID_WIDTH-1:0] id_t;
150 typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
151 typedef logic [AXI_DATA_WIDTH-1:0] data_t;
152 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
153 typedef logic [AXI_USER_WIDTH-1:0] user_t;
154
155 `AXI_TYPEDEF_AW_CHAN_T(aw_chan_t, addr_t, id_t, user_t)
156 `AXI_TYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)
157 `AXI_TYPEDEF_B_CHAN_T(b_chan_t, id_t, user_t)
158 `AXI_TYPEDEF_AR_CHAN_T(ar_chan_t, addr_t, id_t, user_t)
159 `AXI_TYPEDEF_R_CHAN_T(r_chan_t, data_t, id_t, user_t)
160 `AXI_TYPEDEF_REQ_T(axi_req_t, aw_chan_t, w_chan_t,
161   ar_chan_t)
162 `AXI_TYPEDEF_RESP_T(axi_resp_t, b_chan_t, r_chan_t)
163
164 axi_req_t slv_req, mst_req;
165 axi_resp_t slv_resp, mst_resp;
166
167 `AXI_ASSIGN_TO_REQ(slv_req, slv)
168 `AXI_ASSIGN_FROM_RESP(slv, slv_resp)
169
170 `AXI_ASSIGN_FROM_REQ(mst, mst_req)
171 `AXI_ASSIGN_TO_RESP(mst_resp, mst)
172
173 axi_delayer #(
174   .aw_chan_t ( aw_chan_t ),
175   .w_chan_t  ( w_chan_t ),
176   .b_chan_t  ( b_chan_t ),
177   .ar_chan_t ( ar_chan_t ),
178   .r_chan_t  ( r_chan_t ),
179   .axi_req_t ( axi_req_t ),
180   .axi_resp_t ( axi_resp_t ),
181   .StallRandomInput ( STALL_RANDOM_INPUT ),
182   .StallRandomOutput ( STALL_RANDOM_OUTPUT ),
183   .FixedDelayInput ( FIXED_DELAY_INPUT ),
184   .FixedDelayOutput ( FIXED_DELAY_OUTPUT )
185 ) i_axi_delayer (
186   .clk_i, // Clock
187   .rst_ni, // Asynchronous reset active low
188   .slv_req_i ( slv_req ),
189   .slv_resp_o ( slv_resp ),
190   .mst_req_o ( mst_req ),
191   .mst_resp_i ( mst_resp )
192 );
193 // pragma translate_off
194 `ifndef VERILATOR
195   initial begin: p_assertions
196     assert (AXI_ID_WIDTH >= 1) else $fatal(1, "AXI ID width
197 must be at least 1!");
198     assert (AXI_ADDR_WIDTH >= 1) else $fatal(1, "AXI ADDR
199 width must be at least 1!");
200     assert (AXI_DATA_WIDTH >= 1) else $fatal(1, "AXI DATA
201 width must be at least 1!");
202     assert (AXI_USER_WIDTH >= 1) else $fatal(1, "AXI USER
203 width must be at least 1!");
204   end
205 `endif
206 // pragma translate_on
207 endmodule

```

12. pulp-platform_axi/src/axi_demux.sv (305 lines)

```

1 // Copyright (c) 2019 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
3 // pad Hardware
4
5 // License, Version 0.51 (the "License"); you may not use this
6 // file except in
7 // 4 // compliance with the License. You may obtain a copy of the
8 // License at
9 // 5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
10 // applicable law
11 // 6 // or agreed to in writing, software, hardware and materials
12 // distributed under
13 // 7 // this License is distributed on an "AS IS" BASIS, WITHOUT
14 // WARRANTIES OR
15 // 8 // CONDITIONS OF ANY KIND, either express or implied. See the
16 // License for the
17 // 9 // specific language governing permissions and limitations under
18 // the License.
19 // 10 //

```

```

11 // Authors:
12 // - Michael Rogenmoser <michaero@iis.ee.ethz.ch>
13 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
14 // - Thomas Benz <tbenz@iis.ee.ethz.ch>
15 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
16
17 `include "common_cells/assertions.svh"
18 `include "common_cells/registers.svh"
19
20 `ifdef QUESTA
21 // Derive `TARGET_VSIM`, which is used for tool-specific work-
22 // arounds in this file, from `QUESTA`,
23 // which is automatically set in Questa.
24 `define TARGET_VSIM
25 `endif
26
27 /// Demultiplex one AXI4+ATOP slave port to multiple AXI4+ATOP
28 /// master ports.
29 ///
30 /// The AW and AR slave channels each have a `select` input to
31 /// determine to which master port the
32 /// current request is sent. The `select` can, for example,
33 /// be driven by an address decoding module
34 /// to map address ranges to different AXI slaves.
35 ///
36 /// ## Design overview
37 ///
38 /// [Block diagram](module.axi_demux.png "Block diagram")
39 ///
40 /// Beats on the W channel are routed by demultiplexer accord-
41 /// ing to the selection for the
42 /// corresponding AW beat. This relies on the AXI property
43 /// that W bursts must be sent in the same
44 /// order as AW beats and beats from different W bursts may
45 /// not be interleaved.
46 ///
47 /// Beats on the B and R channel are multiplexed from the mas-
48 /// ter ports to the slave port with
49 /// a round-robin arbitration tree.
50 module axi_demux #(
51     parameter int unsigned AxiIdWidth      = 32'd0,
52     parameter bit   AtopSupport            = 1'b1,
53     parameter type  aw_chan_t              = logic,
54     parameter type  w_chan_t              = logic,
55     parameter type  b_chan_t              = logic,
56     parameter type  ar_chan_t             = logic,
57     parameter type  r_chan_t              = logic,
58     parameter type  axi_req_t              = logic,
59     parameter type  axi_resp_t             = logic,
60     parameter int unsigned NoMstPorts      = 32'd0,
61     parameter int unsigned MaxTrans        = 32'd8,
62     parameter int unsigned AxiLookBits    = 32'd3,
63     parameter bit   UniqueIds             = 1'b0,
64     parameter bit   SpillAw               = 1'b1,
65     parameter bit   SpillW                = 1'b0,
66     parameter bit   SpillB                = 1'b0,
67     parameter bit   SpillAr               = 1'b1,
68     parameter bit   SpillR                = 1'b0,
69     // Dependent parameters, DO NOT OVERRIDE!
70     parameter int unsigned SelectWidth     = (NoMstPorts >
71     32'd1) ? $clog2(NoMstPorts) : 32'd1,
72     parameter type  select_t              = logic [Se-
73     lectWidth-1:0]
74 ) (
75     input logic      clk_i,
76     input logic      rst_ni,
77     // Slave Port
78     input axi_req_t  slv_req_i,
79     input select_t   slv_aw_select_i,
80     input select_t   slv_ar_select_i,
81     output axi_resp_t slv_resp_o,
82     // Master Ports
83     output axi_req_t  [NoMstPorts-1:0] mst_reqs_o,
84     input axi_resp_t  [NoMstPorts-1:0] mst_resps_i
85 );
86
87 axi_req_t slv_req_cut;
88 axi_resp_t slv_resp_cut;
89
90 logic slv_aw_ready_chan, slv_aw_ready_sel;
91 logic slv_ar_ready_chan, slv_ar_ready_sel;
92 logic slv_aw_valid_chan, slv_aw_valid_sel;
93
94 select_t slv_aw_select, slv_ar_select;
95
96 cc_spill_register #(
97     .data_t ( aw_chan_t ),
98     .Bypass ( ~SpillAw )
99 ) i_aw_spill_reg (
100     .clk_i,
101     .rst_ni,
102     .clr_i ( 1'b0 ),
103     .valid_i ( slv_req_i.aw_valid ),
104     .ready_o ( slv_aw_ready_chan ),
105     .data_i ( slv_req_i.aw ),
106     .valid_o ( slv_aw_valid_chan ),
107     .ready_i ( slv_resp_cut.aw_ready ),
108     .data_o ( slv_resp_cut.aw ),
109 );
110
111 assign slv_resp_o.aw_ready = slv_aw_ready_chan &
112 slv_aw_ready_sel;
113
114 assign slv_req_cut.aw_valid = slv_aw_valid_chan &
115 slv_aw_valid_sel;

```

```

100     .data_o ( slv_req_cut.aw )
101 );
102 cc_spill_register #(
103     .data_t ( select_t ),
104     .Bypass ( ~SpillAw )
105 ) i_aw_select_spill_reg (
106     .clk_i,
107     .rst_ni,
108     .clr_i ( 1'b0 ),
109     .valid_i ( slv_req_i.aw_valid ),
110     .ready_o ( slv_aw_ready_sel ),
111     .data_i ( slv_aw_select_i ),
112     .valid_o ( slv_aw_valid_sel ),
113     .ready_i ( slv_resp_cut.aw_ready ),
114     .data_o ( slv_aw_select )
115 );
116
117 assign slv_resp_o.aw_ready = slv_aw_ready_chan &
118 slv_aw_ready_sel;
119
120 assign slv_req_cut.aw_valid = slv_aw_valid_chan &
121 slv_aw_valid_sel;
122
123 cc_spill_register #(
124     .data_t ( w_chan_t ),
125     .Bypass ( ~SpillW )
126 ) i_w_spill_reg (
127     .clk_i,
128     .rst_ni,
129     .clr_i ( 1'b0 ),
130     .valid_i ( slv_req_i.w_valid ),
131     .ready_o ( slv_resp_o.w_ready ),
132     .data_i ( slv_req_i.w ),
133     .valid_o ( slv_req_cut.w_valid ),
134     .ready_i ( slv_resp_cut.w_ready ),
135     .data_o ( slv_req_cut.w )
136 );
137
138 cc_spill_register #(
139     .data_t ( ar_chan_t ),
140     .Bypass ( ~SpillAr )
141 ) i_ar_spill_reg (
142     .clk_i,
143     .rst_ni,
144     .clr_i ( 1'b0 ),
145     .valid_i ( slv_req_i.ar_valid ),
146     .ready_o ( slv_ar_ready_chan ),
147     .data_i ( slv_req_i.ar ),
148     .valid_o ( slv_ar_valid_chan ),
149     .ready_i ( slv_resp_cut.ar_ready ),
150     .data_o ( slv_req_cut.ar )
151 );
152
153 cc_spill_register #(
154     .data_t ( select_t ),
155     .Bypass ( ~SpillAr )
156 ) i_ar_sel_spill_reg (
157     .clk_i,
158     .rst_ni,
159     .clr_i ( 1'b0 ),
160     .valid_i ( slv_req_i.ar_valid ),
161     .ready_o ( slv_ar_ready_sel ),
162     .data_i ( slv_ar_select_i ),
163     .valid_o ( slv_ar_valid_sel ),
164     .ready_i ( slv_resp_cut.ar_ready ),
165     .data_o ( slv_ar_select )
166 );
167
168 assign slv_resp_o.ar_ready = slv_ar_ready_chan &
169 slv_ar_ready_sel;
170
171 assign slv_req_cut.ar_valid = slv_ar_valid_chan &
172 slv_ar_valid_sel;
173
174 cc_spill_register #(
175     .data_t ( b_chan_t ),
176     .Bypass ( ~SpillB )
177 ) i_b_spill_reg (
178     .clk_i,
179     .rst_ni,
180     .clr_i ( 1'b0 ),
181     .valid_i ( slv_resp_cut.b_valid ),
182     .ready_o ( slv_req_cut.b_ready ),
183     .data_i ( slv_resp_cut.b ),
184     .valid_o ( slv_resp_o.b_valid ),
185     .ready_i ( slv_req_i.b_ready ),
186     .data_o ( slv_resp_o.b )
187 );
188
189 cc_spill_register #(
190     .data_t ( r_chan_t ),
191     .Bypass ( ~SpillR )
192 ) i_r_spill_reg (
193     .clk_i,
194     .rst_ni,
195     .clr_i ( 1'b0 ),
196     .valid_i ( slv_resp_cut.r_valid ),
197     .ready_o ( slv_req_cut.r_ready ),
198     .data_i ( slv_resp_cut.r ),
199     .valid_o ( slv_resp_o.r_valid ),
200     .ready_i ( slv_req_i.r_ready ),
201     .data_o ( slv_resp_o.r )
202 );
203
204 axi_demux_simple #(
205     .AxiIdWidth ( AxiIdWidth ),

```



```

197 .AtopSupport( AtopSupport ),
198 .axi_req_t ( axi_req_t ),
199 .axi_resp_t ( axi_resp_t ),
200 .NoMstPorts ( NoMstPorts ),
201 .MaxTrans ( MaxTrans ),
202 .AxiLookBits( AxiLookBits ),
203 .UniqueIds ( UniqueIds )
204 ) i_demux_simple (
205 .clk_i,
206 .rst_ni,
207
208 .slv_req_i ( slv_req_cut ),
209 .slv_aw_select_i ( slv_aw_select ),
210 .slv_ar_select_i ( slv_ar_select ),
211 .slv_resp_o ( slv_resp_cut ),
212 .mst_reqs_o ( mst_reqs_o ),
213 .mst_resps_i ( mst_resps_i )
214 );
215
216 endmodule
217
218 // interface wrapper
219 `include "axi/assign.svh"
220 `include "axi/typedef.svh"
221 module axi_demux_intf #(
222     parameter int unsigned AXI_ID_WIDTH = 32'd0, // Synopsys
    DC requires default value for params
223     parameter bit ATOP_SUPPORT = 1'b1,
224     parameter int unsigned AXI_ADDR_WIDTH = 32'd0,
225     parameter int unsigned AXI_DATA_WIDTH = 32'd0,
226     parameter int unsigned AXI_USER_WIDTH = 32'd0,
227     parameter int unsigned NO_MST_PORTS = 32'd3,
228     parameter int unsigned MAX_TRANS = 32'd8,
229     parameter int unsigned AXI_LOOK_BITS = 32'd3,
230     parameter bit UNIQUE_IDS = 1'b0,
231     parameter bit SPILL_AW = 1'b1,
232     parameter bit SPILL_W = 1'b0,
233     parameter bit SPILL_B = 1'b0,
234     parameter bit SPILL_AR = 1'b1,
235     parameter bit SPILL_R = 1'b0,
236     // Dependent parameters, DO NOT OVERRIDE!
237     parameter int unsigned SELECT_WIDTH = (NO_MST_PORTS >
32'd1) ? $clog2(NO_MST_PORTS) : 32'd1,
238     parameter type select_t = logic [SE-
LECT_WIDTH-1:0] // MST port select type
239 ) (
240     input logic clk_i, // Clock
241     input logic rst_ni, // Asynchronous reset
    active low
242     input select_t slv_aw_select_i, // has to be stable,
    when aw_valid
243     input select_t slv_ar_select_i, // has to be stable,
    when ar_valid
244     AXI_BUS.Slave slv, // slave port
245     AXI_BUS.Master mst [NO_MST_PORTS-1:0] // master ports
246 );
247
248 typedef logic [AXI_ID_WIDTH-1:0] id_t;
249 typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
250 typedef logic [AXI_DATA_WIDTH-1:0] data_t;
251 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
252 typedef logic [AXI_USER_WIDTH-1:0] user_t;
253 `AXI_TYPERDEF_AW_CHAN_T(aw_chan_t, addr_t, id_t, user_t)
254 `AXI_TYPERDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)
255 `AXI_TYPERDEF_B_CHAN_T(b_chan_t, id_t, user_t)
256 `AXI_TYPERDEF_AR_CHAN_T(ar_chan_t, addr_t, id_t, user_t)
257 `AXI_TYPERDEF_R_CHAN_T(r_chan_t, data_t, id_t, user_t)
258 `AXI_TYPERDEF_REQ_T(axi_req_t, aw_chan_t, w_chan_t,
ar_chan_t)
259 `AXI_TYPERDEF_RESP_T(axi_resp_t, b_chan_t, r_chan_t)
260
261 axi_req_t slv_req;
262 axi_resp_t slv_resp;
263 axi_req_t [NO_MST_PORTS-1:0] mst_req;
264 axi_resp_t [NO_MST_PORTS-1:0] mst_resp;
265
266 `AXI_ASSIGN_TO_REQ(slv_req, slv)
267 `AXI_ASSIGN_FROM_RESP(slv, slv_resp)
268
269 for (genvar i = 0; i < NO_MST_PORTS; i++) begin :
gen_assign_mst_ports
270     `AXI_ASSIGN_FROM_REQ(mst[i], mst_req[i])
271     `AXI_ASSIGN_TO_RESP(mst_resp[i], mst[i])
272 end
273
274 axi_demux #(
275     .AxiIdWidth ( AXI_ID_WIDTH ), // ID Width
276     .AtopSupport ( ATOP_SUPPORT ),
277     .aw_chan_t ( aw_chan_t ), // AW Channel Type
278     .w_chan_t ( w_chan_t ), // W Channel Type
279     .b_chan_t ( b_chan_t ), // B Channel Type
280     .ar_chan_t ( ar_chan_t ), // AR Channel Type
281     .r_chan_t ( r_chan_t ), // R Channel Type
282     .axi_req_t ( axi_req_t ),
283     .axi_resp_t ( axi_resp_t ),
284     .NoMstPorts ( NO_MST_PORTS ),
285     .MaxTrans ( MAX_TRANS ),
286     .AxiLookBits ( AXI_LOOK_BITS ),
287     .UniqueIds ( UNIQUE_IDS ),

```

```

288     .SpillAw ( SPILL_AW ),
289     .SpillW ( SPILL_W ),
290     .SpillB ( SPILL_B ),
291     .SpillAr ( SPILL_AR ),
292     .SpillR ( SPILL_R )
293 ) i_axi_demux (
294     .clk_i, // Clock
295     .rst_ni, // Asynchronous reset active low
296     // slave port
297     .slv_req_i ( slv_req ),
298     .slv_aw_select_i ( slv_aw_select_i ),
299     .slv_ar_select_i ( slv_ar_select_i ),
300     .slv_resp_o ( slv_resp ),
301     // master port
302     .mst_reqs_o ( mst_req ),
303     .mst_resps_i ( mst_resp )
304 );
305 endmodule

```

13. pulp-platform_axi/src/axi_demux_id_counters.sv (157 lines)

```

1 // Copyright (c) 2019 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
    pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
    file except in
4 // compliance with the License. You may obtain a copy of the
    License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
    applicable law
6 // or agreed to in writing, software, hardware and materials
    distributed under
7 // this license is distributed on an "AS IS" BASIS, WITHOUT
    WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
    License for the
9 // specific language governing permissions and limitations un-
    der the License.
10 //
11 // Authors:
12 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
13 // - Michael Rogenmoser <michaelro@iis.ee.ethz.ch>
14 // - Thomas Benz <tbenz@iis.ee.ethz.ch>
15 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
16
17 `include "common_cells/registers.svh"
18
19 module axi_demux_id_counters #(
20     // the lower bits of the AXI ID that should be considered,
    results in 2**AXI_ID_BITS counters
21     parameter int unsigned AxiIdBits = 2,
22     parameter int unsigned CounterWidth = 4,
23     // Maximum number of in-flight transactions per counter; a
    counter reports full once it
24     // reaches this value. Defaults to the counter's saturation
    value, which preserves the
25     // legacy behavior for instantiations that do not set this
    parameter.
26     parameter int unsigned MaxTrans = 2**CounterWidth - 1,
27     parameter type mst_port_select_t = logic
28 ) (
29     input logic clk_i, // Clock
30     input logic rst_ni, // Asynchronous reset
    active low
31     // lookup
32     input logic [AxiIdBits-1:0] lookup_axi_id_i,
33     output mst_port_select_t lookup_mst_select_o,
34     output logic lookup_mst_select_occupied_o,
35     // push
36     output logic full_o,
37     input logic [AxiIdBits-1:0] push_axi_id_i,
38     input mst_port_select_t push_mst_select_i,
39     input logic push_i,
40     // inject ATOPs in AR channel
41     input logic [AxiIdBits-1:0] inject_axi_id_i,
42     input logic inject_i,
43     // pop
44     input logic [AxiIdBits-1:0] pop_axi_id_i,
45     input logic pop_i,
46     // outstanding transactions
47     output logic any_outstanding_trx_o
48 );
49 localparam int unsigned NoCounters = 2**AxiIdBits;
50 typedef logic [CounterWidth-1:0] cnt_t;
51
52 // registers, each gets loaded when push_en[i]
53 mst_port_select_t [NoCounters-1:0] mst_select_q;
54
55 // counter signals
56 logic [NoCounters-1:0] push_en, inject_en, pop_en, occupied,
    cnt_full;
57
58 //-----
59 // Lookup
60 //-----

```

```

61 assign lookup_mst_select_o          = mst_se-
lect_q[lookup_axi_id_i];
62 assign lookup_mst_select_occupied_o = occu-
pied[lookup_axi_id_i];
63 //-----
64 // Push and Pop
65 //-----
66 assign push_en   = (push_i) ? (1 << push_axi_id_i) : '0;
67 assign inject_en = (inject_i) ? (1 << inject_axi_id_i) : '0;
68 assign pop_en    = (pop_i) ? (1 << pop_axi_id_i) : '0;
69 assign full_o    = |cnt_full;
70 //-----
71 // Status
72 //-----
73 assign any_outstanding_trx_o = |occupied;
74
75 // counters
76 for (genvar i = 0; i < NoCounters; i++) begin : gen_counters
77     logic cnt_en, cnt_down, overflow;
78     cnt_t cnt_delta, in_flight;
79     always_comb begin
80         unique case ({push_en[i], inject_en[i], pop_en[i]})
81             3'b001 : begin // pop_i = -1
82                 cnt_en   = 1'b1;
83                 cnt_down = 1'b1;
84                 cnt_delta = cnt_t'(1);
85             end
86             3'b010 : begin // inject_i = +1
87                 cnt_en   = 1'b1;
88                 cnt_down = 1'b0;
89                 cnt_delta = cnt_t'(1);
90             end
91             // 3'b011, inject_i & pop_i = 0 --> use default
92             3'b100 : begin // push_i = +1
93                 cnt_en   = 1'b1;
94                 cnt_down = 1'b0;
95                 cnt_delta = cnt_t'(1);
96             end
97             // 3'b101, push_i & pop_i = 0 --> use default
98             3'b110 : begin // push_i & inject_i = +2
99                 cnt_en   = 1'b1;
100                cnt_down = 1'b0;
101                cnt_delta = cnt_t'(2);
102            end
103            3'b111 : begin // push_i & inject_i & pop_i = +1
104                cnt_en   = 1'b1;
105                cnt_down = 1'b0;
106                cnt_delta = cnt_t'(1);
107            end
108            default : begin // do nothing to the counters
109                cnt_en   = 1'b0;
110                cnt_down = 1'b0;
111                cnt_delta = cnt_t'(0);
112            end
113        endcase

```

```

114 end
115
116 cc_delta_counter #(
117     .Width      ( CounterWidth ),
118     .StickyOverflow ( 1'b0 )
119 ) i_in_flight_cnt (
120     .clk_i      ( clk_i ),
121     .rst_ni     ( rst_ni ),
122     .clr_i      ( 1'b0 ),
123     .en_i       ( cnt_en ),
124     .load_i     ( 1'b0 ),
125     .down_i     ( cnt_down ),
126     .delta_i    ( cnt_delta ),
127     .d_i        ( '0 ),
128     .q_o        ( in_flight ),
129     .overflow_o ( overflow )
130 );
131 assign occupied[i] = |{overflow, in_flight};
132 // '>=' rather than '==': a simultaneous push and ATOP in-
jection advances the counter
133 // by 2, which could step over an exact-equality threshold
from `MaxTrans - 1`.
134 assign cnt_full[i] = overflow | (in_flight >= cnt_t'(Max-
Trans));
135
136 // holds the selection signal for this id
137 `FFL(mst_select_q[i], push_mst_select_i, push_en[i], '0,
clk_i, rst_ni)
138
139 // pragma translate_off
140 `ifndef VERILATOR
141 `ifndef XILINX_SIMULATOR
142 // Validate parameters.
143 cnt_underflow: assert property(
144     @(posedge clk_i) disable iff (~rst_ni) (pop_en[i] |>= !
overflow) ) else
145     $fatal(1, "axi_demux_id_counters > Counter: %0d under-
flowed.\n
146             The reason is probably a faulty AXI re-
sponse.", i);
147 cnt_full_means_limit: assert property(
148     @(posedge clk_i) disable iff (~rst_ni)
149     (cnt_full[i] |>= ({overflow, in_flight} >= MaxTrans)))
else
150     $fatal(1, "axi_demux_id_counters > Counter %0d reports
full at %0d < MaxTrans = %0d.",
151             i, {overflow, in_flight}, MaxTrans);
152 `endif
153 `endif
154 // pragma translate_on
155 end
156 endmodule
157

```

APB

What this is. The APB peripheral bus.

Why it exists. Because not everything needs AXI. Configuration registers and slow peripherals are better served by a simple non-pipelined bus, and bridging AXI to APB is standard practice.

Undergraduate review · EECS concepts. Simple bus protocols; address decoding; bridging between protocols.

Graduate · Practice note. APB's simplicity is its value: a peripheral with an APB interface is trivial to verify and to integrate. Consider it for any block whose bandwidth is not the point.

7 files • 1,178 lines

API — apb_cdc.sv

Table 1603. pulp-platform_apb/src/apb_cdc.sv — module / interface / package (2개)

이름	종류
apb_cdc	module
apb_cdc_intf	module

Table 1604. pulp-platform_apb/src/apb_cdc.sv — parameter (9개)

파라미터	형/폭	기본값
LogDepth		1
req_t		logic
resp_t		logic
addr_t		logic
data_t		logic
strb_t		logic
APB_ADDR_WIDTH	int unsigned	0
APB_DATA_WIDTH	int unsigned	0
LOG_DEPTH	int unsigned	1

Table 1605. pulp-platform_apb/src/apb_cdc.sv — 포트 (12개)

포트	방향	형/폭
src_pclk_i	input	logic
src_preset_ni	input	logic
src_req_i	input	req_t
src_resp_o	output	resp_t
dst_pclk_i	input	logic

포트	방향	형/폭
dst_preset_ni	input	logic
dst_req_o	output	req_t
dst_resp_i	input	resp_t
src_pclk_i	input	logic
src_preset_ni	input	logic
dst_pclk_i	input	logic
dst_preset_ni	input	logic

Table 1606. pulp-platform_apb/src/apb_cdc.sv — typedef / using (5개)

새 이름	원래 형
src_fsm_state_e	enum logic {Src_Idle, Src_Busy}
dst_fsm_state_e	enum logic[1:0] {Dst_Idle, Dst_Access, Dst_Busy}
addr_t	logic [APB_ADDR_WIDTH-1:0]
data_t	logic [APB_DATA_WIDTH-1:0]
strb_t	logic [APB_DATA_WIDTH/8-1:0]

API — apb_demux.sv

Table 1607. pulp-platform_apb/src/apb_demux.sv — module / interface / package (2개)

이름	종류
apb_demux	module
apb_demux_intf	module

Table 1608. pulp-platform_apb/src/apb_demux.sv — parameter (10개)

파라미터	형/폭	기본값
NoMstPorts	int unsigned	32'd2
req_t		logic
resp_t		logic
SelectWidth	int unsigned	(NoMstPorts > 32'd1)? \$clog2(NoMstPorts) : 32'd1
select_t		logic [SelectWidth-1:0]
APB_ADDR_WIDTH	int unsigned	0
APB_DATA_WIDTH	int unsigned	0
NoMstPorts	int unsigned	32'd2
SelectWidth	int unsigned	(NoMstPorts > 32'd1)? \$clog2(NoMstPorts) : 32'd1
select_t		logic [SelectWidth-1:0]

Table 1609. pulp-platform_apb/src/apb_demux.sv — 포트 (6개)

포트	방향	형/폭
slv_req_i	input	req_t
slv_resp_o	output	resp_t
mst_req_o	output	req_t [NoMstPorts-1:0]
mst_resp_i	input	resp_t [NoMstPorts-1:0]
select_i	input	select_t
select_i	input	select_t

Table 1610. pulp-platform_apb/src/apb_demux.sv — typedef / using (3개)

새 이름	원래 형
addr_t	logic [APB_ADDR_WIDTH-1:0]
data_t	logic [APB_DATA_WIDTH-1:0]
strb_t	logic [APB_DATA_WIDTH/8-1:0]

API — apb_err_slv.sv

Table 1611. pulp-platform_apb/src/apb_err_slv.sv — module / interface / package (2개)

이름	종류
apb_err_slv	module
apb_err_slv_intf	module

Table 1612. pulp-platform_apb/src/apb_err_slv.sv — parameter (7개)

파라미터	형/폭	기본값
req_t		logic
resp_t		logic
RespWidth	int unsigned	32'd32
RespData	logic [RespWidth-1:0]	32'hBADDCAB1E // Hexvalue for the data to return on error.
APB_ADDR_WIDTH	int unsigned	0
APB_DATA_WIDTH	int unsigned	0
RespData	logic [APB_DATA_WIDTH-1:0]	32'hBADDCAB1E

Table 1613. pulp-platform_apb/src/apb_err_slv.sv — 포트 (2개)

포트	방향	형/폭
slv_req_i	input	req_t
slv_resp_o	output	resp_t

Table 1614. pulp-platform_apb/src/apb_err_slv.sv — typedef / using (3개)

새 이름	원래 형
addr_t	logic [APB_ADDR_WIDTH-1:0]
data_t	logic [APB_DATA_WIDTH-1:0]

새 이름	원래 형
strb_t	logic [APB_DATA_WIDTH/8-1:0]

API — apb_intf.sv

Table 1615. pulp-platform_apb/src/apb_intf.sv — module / interface / package (2개)

이름	종류
APB	interface
APB_DV	interface

Table 1616. pulp-platform_apb/src/apb_intf.sv — parameter (4개)

파라미터	형/폭	기본값
ADDR_WIDTH	int unsigned	32'd32
DATA_WIDTH	int unsigned	32'd32
ADDR_WIDTH	int unsigned	32'd32
DATA_WIDTH	int unsigned	32'd32

Table 1617. pulp-platform_apb/src/apb_intf.sv — localparam (2개)

상수	형/폭	값
STRB_WIDTH	int unsigned	cf_math_pkg::ceil_div(DATA_WIDTH
STRB_WIDTH	int unsigned	cf_math_pkg::ceil_div(DATA_WIDTH

Table 1618. pulp-platform_apb/src/apb_intf.sv — 포트 (9개)

포트	방향	형/폭
paddr	output	
pready	input	
paddr	input	
pready	output	
clk_i	input	logic
paddr	output	
pready	input	
paddr	input	
pready	output	

Table 1619. pulp-platform_apb/src/apb_intf.sv — typedef / using (6개)

새 이름	원래 형
addr_t	logic [ADDR_WIDTH-1:0]
data_t	logic [DATA_WIDTH-1:0]
strb_t	logic [STRB_WIDTH-1:0]
addr_t	logic [ADDR_WIDTH-1:0]

새 이름	원래 형
data_t	logic [DATA_WIDTH-1:0]
strb_t	logic [STRB_WIDTH-1:0]

API — apb_pkg.sv

Table 1620. pulp-platform_apb/src/apb_pkg.sv — module / interface / package (1개)

이름	종류
apb_pkg	package

Table 1621. pulp-platform_apb/src/apb_pkg.sv — localparam (2개)

상수	형/폭	값
RESP_OKAY		1'b0
RESP_SLVERR		1'b1

Table 1622. pulp-platform_apb/src/apb_pkg.sv — typedef / using (1개)

새 이름	원래 형
prot_t	logic [2:0]

API — apb_regs.sv

Table 1623. pulp-platform_apb/src/apb_regs.sv — module / interface / package (2개)

이름	종류
apb_regs	module
apb_regs_intf	module

Table 1624. pulp-platform_apb/src/apb_regs.sv — parameter (18개)

파라미터	형/폭	기본값
NoApbRegs	int unsigned	32'd0
ApbAddrWidth	int unsigned	32'd0
AddrOffset	int unsigned	32'd4
ApbDataWidth	int unsigned	32'd0
RegDataWidth	int unsigned	32'd0
ReadOnly	bit [NoApbRegs-1:0]	32'h0
req_t		logic
resp_t		logic
apb_addr_t		logic[ApbAddrWidth-1:0]
reg_data_t		logic[RegDataWidth-1:0]
NO_APB_REGS	int unsigned	32'd0
APB_ADDR_WIDTH	int unsigned	32'd0

파라미터	형/폭	기본값
ADDR_OFFSET	int unsigned	32'd4
APB_DATA_WIDTH	int unsigned	32'd0
REG_DATA_WIDTH	int unsigned	32'd0
READ_ONLY	bit [NO_APB_REGS-1:0]	32'h0
apb_addr_t		logic[APB_ADDR_WIDTH-1:0]
reg_data_t		logic[REG_DATA_WIDTH-1:0]

Table 1625. pulp-platform_apb/src/apb_regs.sv — localparam (2개)

상수	형/폭	값
IdxWidth	int unsigned	(NoApbRegs > 32'd1) ? \$clog2(NoApbRegs) : 32'd1
APB_STRB_WIDTH	int unsigned	cf_math_pkg::ceil_div(APB_DATA_WIDTH

Table 1626. pulp-platform_apb/src/apb_regs.sv — 포트 (12개)

포트	방향	형/폭
pclk_i	input	logic
preset_ni	input	logic
req_i	input	req_t
resp_o	output	resp_t
base_addr_i	input	apb_addr_t
reg_init_i	input	reg_data_t [NoApbRegs-1:0]
reg_q_o	output	reg_data_t [NoApbRegs-1:0]
pclk_i	input	logic
preset_ni	input	logic
base_addr_i	input	apb_addr_t
reg_init_i	input	reg_data_t [NO_APB_REGS-1:0]
reg_q_o	output	reg_data_t [NO_APB_REGS-1:0]

Table 1627. pulp-platform_apb/src/apb_regs.sv — typedef / using (4개)

새 이름	원래 형
idx_t	logic [IdxWidth-1:0]
apb_data_t	logic [ApbAddrWidth-1:0]
apb_data_t	logic [APB_DATA_WIDTH-1:0]
apb_strb_t	logic [APB_STRB_WIDTH-1:0]

API — apb_test.sv

Table 1628. pulp-platform_apb/src/apb_test.sv — module / interface / package (17개)

이름	종류
apb_test	package

Table 1629. pulp-platform_apb/src/apb_test.sv — parameter (19개)

파라미터	형/폭	기본값
ADDR_WIDTH		32'd32
DATA_WIDTH		32'd32
DATA_WIDTH		32'd32
ADDR_WIDTH	int unsigned	32'd32
DATA_WIDTH	int unsigned	32'd32
TA	time	0ns
TT	time	0ns // test time
ADDR_WIDTH	int unsigned	32'd32
DATA_WIDTH	int unsigned	32'd32
TA	time	0ps
TT	time	0ps
RESP_MIN_WAIT_CYCLES	int	0
RESP_MAX_WAIT_CYCLES	int	20
ADDR_WIDTH	int unsigned	32'd32
DATA_WIDTH	int unsigned	32'd32
TA	time	0ps
TT	time	0ps
REQ_MIN_WAIT_CYCLES	int	0
REQ_MAX_WAIT_CYCLES	int	20

Table 1630. pulp-platform_apb/src/apb_test.sv — localparam (2개)

상수	형/폭	값
STRB_WIDTH		cf_math_pkg::ceil_div(DATA_WIDTH
STRB_WIDTH	int unsigned	cf_math_pkg::ceil_div(DATA_WIDTH

Table 1631. pulp-platform_apb/src/apb_test.sv — 포트 (9개)

포트	방향	형/폭
addr	input	addr_t
data	output	data_t
err	output	logic
addr	input	addr_t
data	input	data_t

포트	방향	형/폭
strb	input	strb_t
err	output	logic
request	output	apb_request_t
response	input	apb_response_t

Table 1632. pulp-platform_apb/src/apb_test.sv — typedef / using (15개)

새 이름	원래 형
addr_t	logic [ADDR_WIDTH-1:0]
data_t	logic [DATA_WIDTH-1:0]
strb_t	logic [STRB_WIDTH-1:0]
apb_request_t	apb_request #(ADDR_WIDTH(ADDR_WIDTH), DATA_WIDTH(DATA_WIDTH))
apb_response_t	apb_response #(DATA_WIDTH(DATA_WIDTH))
addr_t	apb_driver_t::addr_t
data_t	apb_driver_t::data_t
strb_t	apb_driver_t::strb_t
apb_request_t	apb_driver_t::apb_request_t
apb_response_t	apb_driver_t::apb_response_t
addr_t	apb_driver_t::addr_t
data_t	apb_driver_t::data_t
strb_t	apb_driver_t::strb_t
apb_request_t	apb_driver_t::apb_request_t
apb_response_t	apb_driver_t::apb_response_t

1. pulp-platform_apb/src/apb_cdc.sv (285 lines)

```

1 // Copyright 2021 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 // APB Clock Domain Crossing
12 // Author: Manuel Eggimann <eggimann@iis.ee.ethz.ch>
13 // Description: A clock domain crossing module on an APB in-
  terface. The module uses gray-counting
14 // CDC FIFOs in both directions to synchronize source side
  with destination side.
15 // Parameters:
16 // - `LogDepth`: The FIFO crossing the clock domain has
  `2*LogDepth` entries
17 // - `req_t`: APB4 request struct. See macro definition in
  `include/typedef.svh`
18 // - `resp_t`: APB4 response struct. See macro defini-
  tion in `include/typedef.svh`

```

```

19 //
20 // Ports:
21 //
22 // - `src_pclk_i`: Source side clock input signal (1-bit).
23 // - `src_preset_ni`: Source side asynchronous active low reset
  signal (1-bit).
24 // - `src_req_i`: Source side APB4 request struct, bundles
  all APB4 signals from the master (req_t).
25 // - `src_resp_o`: Source side APB4 response struct, bundles
  all APB4 signals to the master (resp_t).
26 // - `dst_pclk_i`: Destination side clock input signal (1-
  bit).
27 // - `dst_preset_ni`: Destination side asynchronous active low
  reset signal (1-bit).
28 // - `dst_req_o`: Destination side APB4 request struct,
  bundles all APB4 signals to the slave (req_t).
29 // - `dst_resp_i`: Destination side APB4 response struct,
  bundles all APB4 signals from the slave (resp_t).
30 //
31 // This file also features the module `apb_cdc_intf`. The dif-
  ference is that instead of the
32 // request and response structs it uses a `APB` interfaces.
  The parameters have the same
33 // function, however are defined in `ALL_CAPS`.
34
35 (* no_ungroup *)
36 (* no_boundary_optimization *)
37 module apb_cdc #(
38     parameter LogDepth = 1,
39     parameter type req_t = logic,
40     parameter type resp_t = logic,
41     parameter type addr_t = logic,
42     parameter type data_t = logic,
43     parameter type strb_t = logic
44 ) (

```

```

45 // synchronous slave port - clocked by `src_pclk_i`
46 input logic src_pclk_i,
47 input logic src_preset_ni,
48 input req_t src_req_i,
49 output resp_t src_resp_o,
50 // synchronous master port - clocked by `dst_pclk_i`
51 input logic dst_pclk_i,
52 input logic dst_preset_ni,
53 output req_t dst_req_o,
54 input resp_t dst_resp_i
55 );
56
57 typedef struct packed {
58     addr_t paddr;
59     apb_pkg::prot_t pprot;
60     logic pwrite;
61     data_t pwrite;
62     strb_t pstrb;
63 } apb_async_req_data_t;
64
65 typedef struct packed {
66     data_t prdata;
67     logic pslverr;
68 } apb_async_resp_data_t;
69
70 typedef enum logic {Src_Idle, Src_Busy} src_fsm_state_e;
71 typedef enum logic[1:0] {Dst_Idle, Dst_Access, Dst_Busy}
dst_fsm_state_e;
72 src_fsm_state_e src_state_d, src_state_q;
73 dst_fsm_state_e dst_state_d, dst_state_q;
74 logic src_req_valid, src_req_ready, src_resp_valid,
src_resp_ready;
75 logic dst_req_valid, dst_req_ready, dst_resp_valid,
dst_resp_ready;
76
77 apb_async_req_data_t src_req_data, dst_req_data;
78
79 apb_async_resp_data_t dst_resp_data_d, dst_resp_data_q,
src_resp_data;
80
81 assign src_req_data.paddr = src_req_i.paddr;
82 assign src_req_data.pprot = src_req_i.pprot;
83 assign src_req_data.pwrite = src_req_i.pwrite;
84 assign src_req_data.pwrite = src_req_i.pwrite;
85 assign src_req_data.pwrite = src_req_i.pwrite;
86 assign src_req_data.pstrb = src_req_i.pstrb;
87
88 assign dst_req_o.paddr = dst_req_data.paddr;
89 assign dst_req_o.pprot = dst_req_data.pprot;
90 assign dst_req_o.pwrite = dst_req_data.pwrite;
91 assign dst_req_o.pwrite = dst_req_data.pwrite;
92 assign dst_req_o.pwrite = dst_req_data.pwrite;
93 assign dst_req_o.pstrb = dst_req_data.pstrb;
94
95 assign src_resp_o.prdata = src_resp_data.prdata;
96 assign src_resp_o.pslverr = src_resp_data.pslverr;
97
98 // SRC DOMAIN HANDSHAKE //
99
100 // In the source domain we translate simultaneous assertion
of psel and penable into a transaction
101 // on the CDC. The FSM then transitions into a busy state
where it waits for a response coming
102 // back on the other CDC FIFO. Once this response appears
the pready signal is asserted to finish
103 // the APB transaction.
104
105 always_comb begin
106     src_state_d = src_state_q;
107     src_req_valid = 1'b0;
108     src_resp_ready = 1'b0;
109     src_resp_o.pready = 1'b0;
110     case (src_state_q)
111     Src_Idle: begin
112         if (src_req_i.psel & src_req_i.penable) begin
113             src_req_valid = 1'b1;
114             if (src_req_ready) src_state_d = Src_Busy;
115         end
116     end
117     Src_Busy: begin
118         src_resp_ready = 1'b1;
119         if (src_resp_valid) begin
120             src_resp_o.pready = 1'b1;
121             src_state_d = Src_Idle;
122         end
123     end
124     default:;
125     endcase
126 end
127
128 always_ff @(posedge src_pclk_i, negedge src_preset_ni) begin
129     if (!src_preset_ni)
130         src_state_q <= Src_Idle;
131     else
132         src_state_q <= src_state_d;
133     end
134
135
136 // DST DOMAIN HANDSHAKE //
137
138

```

```

139
140 // In the destination domain we need to perform a proper APB
handshake with setup and access
141 // phase. Once the destination slave asserts the pready sig-
nal we store the response data and
142 // transition into a busy state. In the busy state we send
the response data back to the src
143 // domain and wait for the transaction to complete.
144
145 always_comb begin
146     dst_state_d = dst_state_q;
147     dst_req_ready = 1'b0;
148     dst_resp_valid = 1'b0;
149     dst_req_o.psel = 1'b0;
150     dst_req_o.penable = 1'b0;
151     dst_resp_data_d = dst_resp_data_q;
152     case (dst_state_q)
153     Dst_Idle: begin
154         if (dst_req_valid) begin
155             dst_req_o.psel = 1'b1;
156             dst_state_d = Dst_Access;
157         end
158     end
159
160     Dst_Access: begin
161         dst_req_o.psel = 1'b1;
162         dst_req_o.penable = 1'b1;
163         if (dst_resp_i.pready) begin
164             dst_req_ready = 1'b1;
165             dst_resp_data_d.prdata = dst_resp_i.prdata;
166             dst_resp_data_d.pslverr = dst_resp_i.pslverr;
167             dst_state_d = Dst_Busy;
168         end
169     end
170
171     Dst_Busy: begin
172         dst_resp_valid = 1'b1;
173         if (dst_resp_ready) begin
174             dst_state_d = Dst_Idle;
175         end
176     end
177
178     default: begin
179         dst_state_d = Dst_Idle;
180     end
181     endcase
182 end
183
184 always_ff @(posedge dst_pclk_i, negedge dst_preset_ni) begin
185     if (!dst_preset_ni) begin
186         dst_state_q <= Dst_Idle;
187         dst_resp_data_q <= '0;
188     end else begin
189         dst_state_q <= dst_state_d;
190         dst_resp_data_q <= dst_resp_data_d;
191     end
192 end
193
194
195 // CDC FIFOs //
196
197
198
199 cdc_fifo_gray #(
200     .T ( apb_async_req_data_t ),
201     .LOG_DEPTH ( LogDepth )
202 ) i_cdc_fifo_gray_req (
203     .src_clk_i ( src_pclk_i ),
204     .src_rst_ni ( src_preset_ni ),
205     .src_data_i ( src_req_data ),
206     .src_valid_i ( src_req_valid ),
207     .src_ready_o ( src_req_ready ),
208
209     .dst_clk_i ( dst_pclk_i ),
210     .dst_rst_ni ( dst_preset_ni ),
211     .dst_data_o ( dst_req_data ),
212     .dst_valid_o ( dst_req_valid ),
213     .dst_ready_i ( dst_req_ready )
214 );
215
216 cdc_fifo_gray #(
217     .T ( apb_async_resp_data_t ),
218     .LOG_DEPTH ( LogDepth )
219 ) i_cdc_fifo_gray_resp (
220     .src_clk_i ( dst_pclk_i ),
221     .src_rst_ni ( dst_preset_ni ),
222     .src_data_i ( dst_resp_data_q ),
223     .src_valid_i ( dst_resp_valid ),
224     .src_ready_o ( dst_resp_ready ),
225
226     .dst_clk_i ( src_pclk_i ),
227     .dst_rst_ni ( src_preset_ni ),
228     .dst_data_o ( src_resp_data ),
229     .dst_valid_o ( src_resp_valid ),
230     .dst_ready_i ( src_resp_ready )
231 );
232
233 endmodule // apb_cdc
234
235

```

```

236 `include "apb/typedef.svh"
237 `include "apb/assign.svh"
238
239 module apb_cdc_intf #(
240     parameter int unsigned APB_ADDR_WIDTH = 0,
241     parameter int unsigned APB_DATA_WIDTH = 0,
242     // Depth of the FIFO crossing the clock domain, given as
243     2*LOG_DEPTH.
244     parameter int unsigned LOG_DEPTH = 1
245 ) (
246     input logic src_pclk_i,
247     input logic src_preset_ni,
248     APB.Slave src,
249     input logic dst_pclk_i,
250     input logic dst_preset_ni,
251     APB.Master dst
252 );
253
254 typedef logic [APB_ADDR_WIDTH-1:0] addr_t;
255 typedef logic [APB_DATA_WIDTH-1:0] data_t;
256 typedef logic [APB_DATA_WIDTH/8-1:0] strb_t;
257
258 `APB_TYPEDEF_REQ_T(apb_req_t, addr_t, data_t, strb_t)
259 `APB_TYPEDEF_RESP_T(apb_resp_t, data_t)
260
261 apb_req_t src_req, dst_req;
262 apb_resp_t dst_resp, src_resp;
263
264 `APB_ASSIGN_TO_REQ(src_req, src)
265 `APB_ASSIGN_FROM_REQ(dst, dst_req)
266 `APB_ASSIGN_FROM_RESP(src, src_resp)
267 `APB_ASSIGN_TO_RESP(dst_resp, dst)
268
269 apb_cdc #(
270     .LogDepth ( LOG_DEPTH ),
271     .req_t ( apb_req_t ),
272     .resp_t ( apb_resp_t ),
273     .addr_t ( addr_t ),
274     .data_t ( data_t ),
275     .strb_t ( strb_t )
276 ) i_apb_cdc (
277     .src_pclk_i,
278     .src_preset_ni,
279     .src_req_i ( src_req ),
280     .src_resp_o ( src_resp ),
281     .dst_pclk_i,
282     .dst_preset_ni,
283     .dst_req_o ( dst_req ),
284     .dst_resp_i ( dst_resp )
285 );
286 endmodule

```

2. pulp-platform_apb/src/apb_demux.sv (108 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
3 // pad Hardware
4
5 // License, Version 0.51 (the "License"); you may not use this
6 // file except in
7 // compliance with the License. You may obtain a copy of the
8 // License at
9 // http://solderpad.org/licenses/SHL-0.51. Unless required by
10 // applicable law
11 // or agreed to in writing, software, hardware and materials
12 // distributed under
13 // this License is distributed on an "AS IS" BASIS, WITHOUT
14 // WARRANTIES OR
15 // CONDITIONS OF ANY KIND, either express or implied. See the
16 // License for the
17 // specific language governing permissions and limitations under
18 // the License.
19
20 // APB Demultiplexer Description:
21 // This module demultiplexes one APB slave port to several master
22 // ports using
23 // the select_i signal.
24
25 module apb_demux #(
26     parameter int unsigned NoMstPorts = 32'd2, // The number of
27     demux ports. If
28     // 1, degener-
29     ates gracefully to a
30     // feedthrough.
31     parameter type req_t = logic, // APB request
32     parameter type resp_t = logic, // APB response
33     struct
34     {
35         // DEPENDENT PARAMETERS DO NOT OVERWRITE!
36         parameter int unsigned SelectWidth = (NoMstPorts > 32'd1)?
37         $clog2(NoMstPorts) : 32'd1,
38         parameter type select_t = logic [SelectWidth-1:0]
39     }
40 ) (
41     input req_t slv_req_i,
42     output resp_t slv_resp_o,
43     output req_t [NoMstPorts-1:0] mst_req_o,
44     input resp_t [NoMstPorts-1:0] mst_resp_i,
45     input select_t select_i
46 );

```

```

31
32 if (NoMstPorts == 32'd1) begin
33     assign mst_req_o[0] = slv_req_i;
34     assign slv_resp_o = mst_resp_i[0];
35 end else begin
36     for (genvar idx = 0; idx < NoMstPorts; idx++) begin
37         assign mst_req_o[idx].paddr = slv_req_i.paddr;
38         assign mst_req_o[idx].pprot = slv_req_i.pprot;
39         assign mst_req_o[idx].psel = (select_i == idx)?
40         slv_req_i.psel: 1'b0;
41         assign mst_req_o[idx].penable = (select_i == idx)?
42         slv_req_i.penable: 1'b0;
43         assign mst_req_o[idx].pwrite = slv_req_i.pwrite;
44         assign mst_req_o[idx].pwrite = slv_req_i.pwrite;
45         assign mst_req_o[idx].pstrb = slv_req_i.pstrb;
46     end
47
48 always_comb begin
49     if (select_i < NoMstPorts) begin
50         slv_resp_o.pready = mst_resp_i[select_i].pready;
51         slv_resp_o.prdata = mst_resp_i[select_i].prdata;
52         slv_resp_o.pslverr = mst_resp_i[select_i].pslverr;
53     end else begin
54         slv_resp_o.pready = 1'b1;
55         slv_resp_o.prdata = '0;
56         slv_resp_o.pslverr = 1'b1;
57     end
58 end
59 endmodule
60
61 `include "apb/typedef.svh"
62 `include "apb/assign.svh"
63
64 module apb_demux_intf #(
65     parameter int unsigned APB_ADDR_WIDTH = 0,
66     parameter int unsigned APB_DATA_WIDTH = 0,
67     parameter int unsigned NoMstPorts = 32'd2,
68     // DEPENDENT PARAMETERS DO NOT OVERWRITE!
69     parameter int unsigned SelectWidth = (NoMstPorts > 32'd1)?
70     $clog2(NoMstPorts) : 32'd1,
71     parameter type select_t = logic [SelectWidth-1:0]
72 ) (
73     APB.Slave slv,
74     APB.Master mst [NoMstPorts-1:0],
75     input select_t select_i
76 );
77
78 typedef logic [APB_ADDR_WIDTH-1:0] addr_t;
79 typedef logic [APB_DATA_WIDTH-1:0] data_t;
80 typedef logic [APB_DATA_WIDTH/8-1:0] strb_t;
81
82 `APB_TYPEDEF_REQ_T(apb_req_t, addr_t, data_t, strb_t)
83 `APB_TYPEDEF_RESP_T(apb_resp_t, data_t)
84
85 apb_req_t slv_req;
86 apb_resp_t slv_resp;
87
88 apb_req_t [NoMstPorts-1:0] mst_req;
89 apb_resp_t [NoMstPorts-1:0] mst_resp;
90
91 for (genvar idx = 0; idx < NoMstPorts; idx++) begin
92     `APB_ASSIGN_FROM_REQ(mst[idx], mst_req[idx])
93     `APB_ASSIGN_TO_RESP(mst_resp[idx], mst[idx])
94 end
95
96 `APB_ASSIGN_TO_REQ(slv_req, slv)
97 `APB_ASSIGN_FROM_RESP(slv_resp, slv)
98
99 apb_demux #(
100     .NoMstPorts ( NoMstPorts ),
101     .req_t ( apb_req_t ),
102     .resp_t ( apb_resp_t )
103 ) i_apb_cdc (
104     .slv_req_i ( slv_req ),
105     .slv_resp_o ( slv_resp ),
106     .mst_req_o ( mst_req ),
107     .mst_resp_i ( mst_resp ),
108     .select_i
109 );
110 endmodule

```

3. pulp-platform_apb/src/apb_err_slv.sv (79 lines)

```

1 //-----
2 // Title : APB Error Slave
3 //-----
4 // File : apb_err_slv.sv
5 // Author : Manuel Eggimann <meggimann@iis.ee.ethz.ch>
6 // Created : 05.12.2022
7 //-----
8 // Description :
9 // This module always responds with an error response on any
10 // request of its APB
11 // slave port. Use this module e.g. as the default port in a
12 // APB demultiplexer
13 // to handle illegal access. Following the recommendation of

```

```

the APB 2.0
12 // specification, the error response will only be asserted on
a valid
13 // transaction.
14 //-----
15 // Copyright (C) 2013-2022 ETH Zurich, University of Bologna
16 // Copyright and related rights are licensed under the Solder-
pad Hardware

17 // License, Version 0.51 (the "License"); you may not use this
file except in
18 // compliance with the License. You may obtain a copy of the
License at
19 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
20 // or agreed to in writing, software, hardware and materials
distributed under
21 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
22 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
23 // specific language governing permissions and limitations un-
der the license.
24 //-----
25 module apb_err_slv #(
26     parameter                                type req_t = logic,    //
APB request struct
27     parameter                                type resp_t = logic,    //
APB response struct
28     parameter int unsigned                   RespWidth = 32'd32,    //
Data width of the response. Gets zero extended or truncated to
r.data.
29     parameter logic [RespWidth-1:0] RespData = 32'hBADCAB1E //
Hexvalue for the data to return on error.
30 ) (
31     input req_t slv_req_i,
32     output resp_t slv_resp_o
33 );
34
35     assign slv_resp_o.prdata = RespData;
36     assign slv_resp_o.pready = 1'b1;
37     // Following the APB recommendations, we only assert the er-
ror signal if there is actually a
38     // request.
39     assign slv_resp_o.pslverr = (slv_req_i.psel & slv_req_i.pen-
able)?
40
41     apb_pkg::RESP_OKAY;
42 endmodule
43
44
45 `include "apb/typedef.svh"
46 `include "apb/assign.svh"
47
48 module apb_err_slv_intf #(
49     parameter int unsigned                   APB_ADDR_WIDTH = 0,
50     parameter int unsigned                   APB_DATA_WIDTH = 0,
51     parameter logic [APB_DATA_WIDTH-1:0] RespData = 32'hBAD-
CAB1E
52 ) (
53     APB.Slave slv
54 );
55
56     typedef logic [APB_ADDR_WIDTH-1:0] addr_t;
57     typedef logic [APB_DATA_WIDTH-1:0] data_t;
58     typedef logic [APB_DATA_WIDTH/8-1:0] strb_t;
59
60     `APB_TYPEDEF_REQ_T(apb_req_t, addr_t, data_t, strb_t)
61     `APB_TYPEDEF_RESP_T(apb_resp_t, data_t)
62
63     apb_req_t slv_req;
64     apb_resp_t slv_resp;
65
66     `APB_ASSIGN_TO_REQ(slv_req, slv)
67     `APB_ASSIGN_FROM_RESP(slv, slv_resp)
68
69     apb_err_slv #(
70         .req_t      ( apb_req_t      ),
71         .resp_t     ( apb_resp_t     ),
72         .RespWidth  ( APB_DATA_WIDTH ),
73         .RespData   ( RespData       )
74     ) i_apb_err_slv (
75         .slv_req_i  ( slv_req       ),
76         .slv_resp_o ( slv_resp      )
77     );
78
79 endmodule

```

4. pulp-platform_apb/src/apb_intf.sv (78 lines)

```

1 // Copyright (c) 2014-2020 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware

4 // License, Version 0.51 (the "License"); you may not use this
file except in

```

```

5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the License.
11
12 // An APB4 (v2.0) interface
13 interface APB #(
14     parameter int unsigned ADDR_WIDTH = 32'd32,
15     parameter int unsigned DATA_WIDTH = 32'd32
16 );
17     localparam int unsigned STRB_WIDTH =
cf_math_pkg::ceil_div(DATA_WIDTH, 8);
18     typedef logic [ADDR_WIDTH-1:0] addr_t;
19     typedef logic [DATA_WIDTH-1:0] data_t;
20     typedef logic [STRB_WIDTH-1:0] strb_t;
21
22     addr_t      paddr;
23     apb_pkg::prot_t pprot;
24     logic       psel;
25     logic       penable;
26     logic       pwrite;
27     data_t      pwrdata;
28     strb_t      pstrb;
29     logic        pready;
30     data_t      prdata;
31     logic        pslverr;
32
33     modport Master (
34         output paddr, pprot, psel, penable, pwrite, pwrdata, pstrb,
35         input  pready, prdata, pslverr
36     );
37
38     modport Slave (
39         input  paddr, pprot, psel, penable, pwrite, pwrdata, pstrb,
40         output pready, prdata, pslverr
41     );
42
43 endinterface
44
45 // A clocked APB4 (v2.0) interface for use in design verifica-
tion
46 interface APB_DV #(
47     parameter int unsigned ADDR_WIDTH = 32'd32,
48     parameter int unsigned DATA_WIDTH = 32'd32
49 ) (
50     input logic clk_i
51 );
52     localparam int unsigned STRB_WIDTH =
cf_math_pkg::ceil_div(DATA_WIDTH, 8);
53     typedef logic [ADDR_WIDTH-1:0] addr_t;
54     typedef logic [DATA_WIDTH-1:0] data_t;
55     typedef logic [STRB_WIDTH-1:0] strb_t;
56
57     addr_t      paddr;
58     apb_pkg::prot_t pprot;
59     logic       psel;
60     logic       penable;
61     logic       pwrite;
62     data_t      pwrdata;
63     strb_t      pstrb;
64     logic        pready;
65     data_t      prdata;
66     logic        pslverr;
67
68     modport Master (
69         output paddr, pprot, psel, penable, pwrite, pwrdata, pstrb,
70         input  pready, prdata, pslverr
71     );
72
73     modport Slave (
74         input  paddr, pprot, psel, penable, pwrite, pwrdata, pstrb,
75         output pready, prdata, pslverr
76     );
77
78 endinterface

```

5. pulp-platform_apb/src/apb_pkg.sv (22 lines)

```

1 // Copyright (c) 2019 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware

3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR

```

```

8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the License.
10
11 // Author: Wolfgang Roenninger <wroennin@ethz.ch>
12
13 // Description: Package with constant APB v2.0 constants
14
15 package apb_pkg;
16
17   typedef logic [2:0]  prot_t;
18
19   localparam RESP_OKAY  = 1'b0;
20   localparam RESP_SLVERR = 1'b1;
21
22 endpackage

```

6. pulp-platform_apb/src/apb_regs.sv (264 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the License.
10
11 // APB Read-Write Registers
12 // Description: This module exposes a number of registers on
an APB interface.
13 // It responds to not mapped accesses with a
slave error.
14 // Some of the registers can be configured to be
read only.
15 // Parameters:
16 // - `NoApbRegs`: Number of registers.
17 // - `ApbAddrWidth`: Address width of `req_i.paddr`, is used
to generate internal address map.
18 // - `AddrOffset`: The address offset in bytes between the
registers. Is asserted to be a least
19 // `32'd4`, the reason is to prevent overlap in
the address map.
20 // Each register is mapped to a 4 byte wide
address range. When this parameter
21 // is bigger than `32'd4` there will be
holes in the address map and the module
22 // answers with `apb_pkg::RESP_SLVERR`. It
is recommended that this value is a
23 // power of 2, to prevent data alignment is-
sues on upstream buses.
24 // - `ApbDataWidth`: The data width of the APB4 bus, this
value can be up to `32'd32` (bit).
25 // - `RegDataWidth`: The data width of the registers, this
value has to be less or equal to
26 // `ApbDataWidth`. If it is less the re-
gister gets zero extended for reads and
27 // higher bits on writes get ignored.
28 // - `ReadOnly`: This flag can specify a read only re-
gister at the given register index of the
29 // array. When in the array the correspond-
ing bit is set, the `reg_init_i` signal
30 // at given index can be read out. Writes
are ignored if the flag is set.
31 // - `req_t`: APB4 request struct. See macro definition in
`include/typedef.svh`
32 // - `resp_t`: APB4 response struct. See macro defini-
tion in `include/typedef.svh`
33 //
34 // Ports:
35 //
36 // - `pclk_i`: Clock input signal (1-bit).
37 // - `preset_ni`: Asynchronous active low reset signal (1-
bit).
38 // - `req_i`: APB4 request struct, bundles all APB4
signals from the master (req_t).
39 // - `resp_o`: APB4 response struct, bundles all APB4
signals to the master (resp_t).
40 // - `base_addr_i`: Base address of this module, from here
the registers `0` are mapped, starting
41 // with index `0`. All subsequent register
with higher indices have their bases
42 // mapped with reg_index * `AddrOffset` from
this value (ApbAddrWidth-bit).
43 // - `reg_init_i`: Initialization value for each register,
when the register index is configured

```

```

44 // as `ReadOnly[reg_index] == 1'b1` this
value is passed through directly to
45 // the APB4 bus (Array size `NoApbRegs` *
RegDataWidth-bit).
46 // - `reg_q_o`: The current value of the register. If the
register at the array index is
47 // read only, the `reg_init_i` value is
passed through to the respective index
48 // (Array size `NoApbRegs` * RegDataWidth-
bit).
49 //
50 // This file also features the module `apb_regs_intf`. The
difference is that instead of the
51 // request and response structs it uses an `APB.Salve` inter-
face. The parameters have the same
52 // Function, however are defined in `ALL_CAPS`.
53
54
55 `include "common_cells/registers.svh"
56
57 module apb_regs #(
58   parameter int unsigned NoApbRegs = 32'd0,
59   parameter int unsigned ApbAddrWidth = 32'd0,
60   parameter int unsigned AddrOffset = 32'd4,
61   parameter int unsigned ApbDataWidth = 32'd0,
62   parameter int unsigned RegDataWidth = 32'd0,
63   parameter bit [NoApbRegs-1:0] ReadOnly = 32'h0,
64   parameter type req_t = logic,
65   parameter type resp_t = logic,
66   // DEPENDENT PARAMETERS DO NOT OVERWRITE!
67   parameter type apb_addr_t = lo-
gic[ApbAddrWidth-1:0],
68   parameter type reg_data_t = lo-
gic[RegDataWidth-1:0]
69 ) (
70   // APB Interface
71   input logic pclk_i,
72   input logic preset_ni,
73   input req_t req_i,
74   output resp_t resp_o,
75   // Register Interface
76   input apb_addr_t base_addr_i, // base ad-
dress of the read/write registers
77   input reg_data_t [NoApbRegs-1:0] reg_init_i,
78   output reg_data_t [NoApbRegs-1:0] reg_q_o
79 );
80 localparam int unsigned IdxWidth = (NoApbRegs > 32'd1) ?
$ceil($log2(NoApbRegs)) : 32'd1;
81 typedef logic [IdxWidth-1:0] idx_t;
82 typedef logic [ApbAddrWidth-1:0] apb_data_t;
83 typedef struct packed {
84   int unsigned idx;
85   apb_addr_t start_addr;
86   apb_addr_t end_addr;
87 } rule_t;
88
89 logic has_reset_d, has_reset_q;
90 `FFARN(has_reset_q, has_reset_d, 1'b0, pclk_i, preset_ni)
91 assign has_reset_d = 1'b1;
92
93 // signal declarations
94 rule_t [NoApbRegs-1:0] addr_map;
95 idx_t reg_idx;
96 logic decode_valid;
97 // register signals
98 reg_data_t [NoApbRegs-1:0] reg_d, reg_q;
99 logic [NoApbRegs-1:0] reg_update;
100
101 // generate address map for the registers
102 for (genvar i = 0; i < NoApbRegs; i++) begin: gen_reg_ad-
dr_map
103   assign addr_map[i] = rule_t'{
104     idx: unsigned'(i),
105     start_addr: base_addr_i + apb_addr_t'( i * Ad-
drOffset),
106     end_addr: base_addr_i + apb_addr_t'((i+32'd1) * Ad-
drOffset)
107   };
108 end
109
110 always_comb begin
111   // default assignments
112   reg_d = has_reset_q ? reg_q : reg_init_i;
113   reg_update = '0;
114   resp_o = '{
115     pready: req_i.psel & req_i.penable,
116     prdata: apb_data_t'(32'h0BAD_B10C),
117     pslverr: apb_pkg::RESP_OKAY
118   };
119   if (req_i.psel) begin
120     if (!decode_valid) begin
121       // Error response on decode errors
122       resp_o.pslverr = apb_pkg::RESP_SLVERR;
123     end else begin
124       if (req_i.pwrite) begin
125         if (!ReadOnly[reg_idx]) begin
126           for (int unsigned i = 0; i < RegDataWidth; i++)
127             if (req_i.pstrb[i/8]) begin
128               reg_d[reg_idx][i] = req_i.pwdata[i];

```



```

129         end
130     end
131     reg_update[reg_idx] = |req_i.pstrb;
132     end else begin
133         // this register is read only
134         resp_o.pslverr = apb_pkg::RESP_SLVERR;
135     end
136     end else begin
137         if (!ReadOnly[reg_idx]) begin
138             resp_o.prdata = apb_data_t'(reg_q[reg_idx]);
139         end else begin
140             // for read only register the directly connect the
141             resp_o.prdata = apb_data_t'(reg_init_i[reg_idx]);
142         end
143     end
144     end
145     end
146     end
147
148     // output assignment and registers
149     for (genvar i = 0; i < NoApbRegs; i++) begin : gen_rw_regs
150         assign reg_q_o[i] = ReadOnly[i] ? reg_init_i[i] :
151         reg_q[i];
152     end
153     preset_ni
154     end
155
156     addr_decode #(
157         .NoIndices ( NoApbRegs ),
158         .NoRules ( NoApbRegs ),
159         .addr_t ( apb_addr_t ),
160         .rule_t ( rule_t )
161     ) i_addr_decode (
162         .addr_i ( req_i.paddr ),
163         .addr_map_i ( addr_map ),
164         .idx_o ( reg_idx ),
165         .dec_valid_o ( decode_valid ),
166         .dec_error_o ( /*not used*/ ),
167         .en_default_idx_i ( '0 ),
168         .default_idx_i ( '0 )
169     );
170
171     // Validate parameters.
172     // pragma translate_off
173     `ifndef VERILATOR
174     initial begin: p_assertions
175         assert (NoApbRegs > 32'd0)
176         else $fatal(1, "The number of registers must be at
177         least 1!");
178         assert (ApbAddrWidth > 32'd2)
179         else $fatal(1, "ApbAddrWidth is not wide enough, has
180         to be at least 3 bit wide!");
181         assert (AddrOffset > 32'd3)
182         else $fatal(1, "AddrOffset has to be at least 4 and
183         is recommended to be a power of 2!");
184         assert ($bits(req_i.paddr) == ApbAddrWidth)
185         else $fatal(1, "AddrWidth does not match req_i.padr-
186         dr!");
187         assert (ApbDataWidth == $bits(resp_o.prdata))
188         else $fatal(1, "ApbDataWidth has to be: ApbDataWidth
189         == $bits(req_i.prdata)!");
190         assert (ApbDataWidth > 32'd0 && ApbDataWidth <= 32'd32)
191         else $fatal(1, "ApbDataWidth has to be: 32'd32 >=
192         RegDataWidth > 0!");
193         assert ($bits(resp_o.prdata) == $bits(req_i.pwdata))
194         else $fatal(1, "req_i.pwdata has to match
195         resp_o.prdata in width!");
196         assert (RegDataWidth > 32'd0 && RegDataWidth <= 32'd32)
197         else $fatal(1, "RegDataWidth has to be: 32'd32 >=
198         RegDataWidth > 0!");
199         assert (RegDataWidth <= $bits(resp_o.prdata))
200         else $fatal(1, "RegDataWidth has to be: RegDataWidth
201         <= $bits(req_i.prdata)!");
202         assert (NoApbRegs == $bits(ReadOnly))
203         else $fatal(1, "Each register need a `ReadOnly`
204         flag!");
205     end
206     `endif
207     // pragma translate_on
208 endmodule
209
210 `include "apb/assign.svh"
211 `include "apb/typedef.svh"
212
213 module apb_regs_intf #(
214     parameter int unsigned NO_APB_REGS = 32'd0, //
215     number of read only registers
216     parameter int unsigned APB_ADDR_WIDTH = 32'd0, //
217     address width of 'paddr'
218     parameter int unsigned ADDR_OFFSET = 32'd4, //
219     address offset in bytes
220     parameter int unsigned APB_DATA_WIDTH = 32'd0, //
221     data width of the registers
222     parameter int unsigned REG_DATA_WIDTH = 32'd0,
223     parameter bit [NO_APB_REGS-1:0] READ_ONLY = 32'h0,
224     // DEPENDENT PARAMETERS DO NOT OVERWRITE!

```

```

209     parameter type apb_addr_t = lo-
210     gic[APB_ADDR_WIDTH-1:0],
211     parameter type reg_data_t = lo-
212     gic[REG_DATA_WIDTH-1:0]
213 ) (
214     // APB Interface
215     input logic pclk_i,
216     input logic preset_ni,
217     APB_Slave slv,
218     // Register Interface
219     input apb_addr_t base_addr_i, // base ad-
220     dress of the read only registers
221     input reg_data_t [NO_APB_REGS-1:0] reg_init_i, // initial-
222     isation value for the registers
223     output reg_data_t [NO_APB_REGS-1:0] reg_q_o
224 );
225 localparam int unsigned APB_STRB_WIDTH =
226 cf_math_pkg::ceil_div(APB_DATA_WIDTH, 8);
227 typedef logic [APB_DATA_WIDTH-1:0] apb_data_t;
228 typedef logic [APB_STRB_WIDTH-1:0] apb_strb_t;
229
230 `APB_TYPEDEF_REQ_T(apb_req_t, apb_addr_t, apb_data_t,
231 apb_strb_t)
232 `APB_TYPEDEF_RESP_T(apb_resp_t, apb_data_t)
233
234 apb_req_t apb_req;
235 apb_resp_t apb_resp;
236
237 `APB_ASSIGN_TO_REQ(apb_req, slv)
238 `APB_ASSIGN_FROM_RESP(slv, apb_resp )
239
240 apb_regs #(
241     .NoApbRegs ( NO_APB_REGS ),
242     .ApbAddrWidth ( APB_ADDR_WIDTH ),
243     .AddrOffset ( ADDR_OFFSET ),
244     .ApbDataWidth ( APB_DATA_WIDTH ),
245     .RegDataWidth ( REG_DATA_WIDTH ),
246     .ReadOnly ( READ_ONLY ),
247     .req_t ( apb_req_t ),
248     .resp_t ( apb_resp_t )
249 ) i_apb_regs (
250     .pclk_i,
251     .preset_ni,
252     .req_i ( apb_req ),
253     .resp_o ( apb_resp ),
254     .base_addr_i,
255     .reg_init_i,
256     .reg_q_o
257 );
258
259 // Validate parameters.
260 // pragma translate_off
261 `ifndef VERILATOR
262 initial begin: p_assertions
263     assert (APB_ADDR_WIDTH == $bits(slv.paddr))
264     else $fatal(1, "APB_ADDR_WIDTH does not match slv
265     interface!");
266     assert (APB_DATA_WIDTH == $bits(slv.pwdata))
267     else $fatal(1, "APB_DATA_WIDTH does not match slv
268     interface!");
269 end
270 `endif
271 // pragma translate_on
272 endmodule

```

7. pulp-platform_apb/src/apb_test.sv (342 lines)

```

1 // Copyright (c) 2018 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the Solder-
4 pad Hardware
5
6 // License, Version 0.51 (the "License"); you may not use this
7 file except in
8 // compliance with the License. You may obtain a copy of the
9 License at
10 // http://solderpad.org/licenses/SHL-0.51. Unless required by
11 applicable law
12 // or agreed to in writing, software, hardware and materials
13 distributed under
14 // this license is distributed on an "AS IS" BASIS, WITHOUT
15 WARRANTIES OR
16 // CONDITIONS OF ANY KIND, either express or implied. See the
17 License for the
18 // specific language governing permissions and limitations un-
19 der the License.
20
21 // Test infrastructure for APB interfaces
22 package apb_test;
23
24 class apb_request #(
25     parameter ADDR_WIDTH = 32'd32,
26     parameter DATA_WIDTH = 32'd32
27 );
28     localparam STRB_WIDTH = cf_math_pkg::ceil_div(DATA_WIDTH,
29 8);
30     rand logic [ADDR_WIDTH-1:0] paddr = '0;
31     rand logic [DATA_WIDTH-1:0] pwdata = '0;
32     rand logic [STRB_WIDTH-1:0] pstrb = '0;

```

```

24     rand logic                pwrite = 1'b1;
25   endclass;
26
27   class apb_response #(
28     parameter DATA_WIDTH = 32'd32
29   );
30     rand logic [DATA_WIDTH-1:0] prdata = '0;
31     rand logic                pslverr = 1'b0;
32   endclass;
33
34   // Simple APB driver with thread-safe read and write func-
35   tions
36   class apb_driver #(
37     parameter int unsigned ADDR_WIDTH = 32'd32, // APB4 ad-
38     dress width
39     parameter int unsigned DATA_WIDTH = 32'd32, // APB4 data
40     width
41     parameter time        TA        = 0ns,    // application
42     time
43     parameter time        TT        = 0ns     // test time
44   );
45     localparam int unsigned STRB_WIDTH =
46     cf_math_pkg::ceil_div(DATA_WIDTH, 8);
47     typedef logic [ADDR_WIDTH-1:0] addr_t;
48     typedef logic [DATA_WIDTH-1:0] data_t;
49     typedef logic [STRB_WIDTH-1:0] strb_t;
50     typedef apb_request
51     #(ADDR_WIDTH(ADDR_WIDTH), .DATA_WIDTH(DATA_WIDTH)) apb_request_t;
52     typedef apb_response #(DATA_WIDTH(DATA_WIDTH)) apb_re-
53     sponse_t;
54     virtual APB_DV #(
55       .ADDR_WIDTH(ADDR_WIDTH),
56       .DATA_WIDTH(DATA_WIDTH)
57     ) apb;
58     std::semaphore lock;
59
60     function new(virtual APB_DV
61     #(ADDR_WIDTH(ADDR_WIDTH), .DATA_WIDTH(DATA_WIDTH)) apb);
62       this.apb = apb;
63       this.lock = new(1);
64     endfunction
65
66     function void reset_master();
67       apb.paddr <= '0;
68       apb.pprot <= '0;
69       apb.psel <= 1'b0;
70       apb.penable <= 1'b0;
71       apb.pwrite <= 1'b0;
72       apb.pwdata <= '0;
73       apb.pstrb <= '0;
74     endfunction
75
76     function void reset_slave();
77       apb.pready <= 1'b0;
78       apb.prdata <= '0;
79       apb.pslverr <= 1'b0;
80     endfunction
81
82     task cycle_start;
83       #TT;
84     endtask
85
86     task cycle_end;
87       @(posedge apb.clk_i);
88     endtask
89
90     // this task reads from an APB4 slave, acts as master
91     task read(
92       input addr_t addr,
93       output data_t data,
94       output logic err
95     );
96       while (!lock.try_get()) begin
97         cycle_end();
98       end
99       apb.paddr <= #TA addr;
100       apb.pwrite <= #TA 1'b0;
101       apb.psel <= #TA 1'b1;
102       cycle_end();
103       apb.penable <= #TA 1'b1;
104       cycle_start();
105       while (!apb.pready) begin
106         cycle_end();
107         cycle_start();
108       end
109       data = apb.prdata;
110       err = apb.pslverr;
111       cycle_end();
112       apb.paddr <= #TA '0;
113       apb.psel <= #TA 1'b0;
114       apb.penable <= #TA 1'b0;
115       lock.put();
116     endtask
117
118     // this task writes to an APB4 slave, acts as master
119     task write(
120       input addr_t addr,
121       input data_t data,
122       input strb_t strb,
123       output logic err

```

```

116   );
117   while (!lock.try_get()) begin
118     cycle_end();
119   end
120   apb.paddr <= #TA addr;
121   apb.pwdata <= #TA data;
122   apb.pstrb <= #TA strb;
123   apb.pwrite <= #TA 1'b1;
124   apb.psel <= #TA 1'b1;
125   cycle_end();
126   apb.penable <= #TA 1'b1;
127   cycle_start();
128   while (!apb.pready) begin
129     cycle_end();
130     cycle_start();
131   end
132   err = apb.pslverr;
133   cycle_end();
134   apb.paddr <= #TA '0;
135   apb.pwdata <= #TA '0;
136   apb.pstrb <= #TA '0;
137   apb.pwrite <= #TA 1'b0;
138   apb.psel <= #TA 1'b0;
139   apb.penable <= #TA 1'b0;
140   lock.put();
141   endtask
142
143   // Wait for incoming apb request
144   task recv_request (
145     output apb_request_t request
146   );
147     cycle_start();
148     while (!apb.psel) begin cycle_end(); cycle_start(); end
149     while (!apb.penable) begin cycle_end(); cycle_start(); end
150   end
151   request = new;
152   request.paddr = apb.paddr;
153   request.pwdata = apb.pwdata;
154   request.pwrite = apb.pwrite;
155   request.pstrb = apb.pstrb;
156   cycle_end();
157   endtask
158
159   // Acknowledge incoming APB write request (assert pready).
160   task send_write_ack();
161     apb.pready <= #TA 1;
162     cycle_end();
163     apb.pready <= #TA 0;
164   endtask
165
166   // Send response for incoming APB read request acknow-
167   ledging the request (assert pready).
168   task send_read_response (
169     input apb_response_t response
170   );
171     apb.pready <= #TA 1;
172     apb.prdata <= #TA response.prdata;
173     apb.pslverr <= #TA response.pslverr;
174     cycle_end();
175     apb.pready <= #TA 0;
176     apb.prdata <= #TA '0;
177     apb.pslverr <= #TA 0;
178   endtask
179
180   class apb_rand_slave #(
181     // APB interface parameters
182     parameter int unsigned ADDR_WIDTH = 32'd32,
183     parameter int unsigned DATA_WIDTH = 32'd32,
184     // Stimuli application and test time
185     parameter time TA = 0ps,
186     parameter time TT = 0ps,
187     // Upper and lower bounds on wait states on Response
188     (pready)
189     parameter int RESP_MIN_WAIT_CYCLES = 0,
190     parameter int RESP_MAX_WAIT_CYCLES = 20
191   );
192     typedef apb_test::apb_driver #(
193       .ADDR_WIDTH(ADDR_WIDTH), .DATA_WIDTH(DATA_WIDTH), .TA(TA
194     ), .TT(TT)
195   ) apb_driver_t;
196
197   typedef apb_driver_t::addr_t addr_t;
198   typedef apb_driver_t::data_t data_t;
199   typedef apb_driver_t::strb_t strb_t;
200
201   typedef apb_driver_t::apb_request_t apb_request_t;
202   typedef apb_driver_t::apb_response_t apb_response_t;
203
204   apb_driver_t drv;
205   std::mailbox #(apb_request_t) request_queue;
206   std::mailbox #(apb_response_t) response_queue;
207
208   function new(
209     virtual APB_DV #(
210       .ADDR_WIDTH(ADDR_WIDTH),
211       .DATA_WIDTH(DATA_WIDTH)
212     ) apb

```

```

213     this.drv          = new(apb);
214     this.request_queue = new;
215     this.response_queue = new;
216     this.reset();
217 endfunction
218
219 function reset();
220     drv.reset_slave();
221 endfunction
222
223 // TODO: The `rand_wait` task exists in `rand_verif_pkg`,
but that task cannot be called with
224 // `this.drv.apb.clk_i` as `clk` argument. What is the
syntax getting an assignable reference?
225 task automatic rand_wait(input int unsigned min, max);
226     int unsigned rand_success, cycles;
227     rand_success = std::randomize(cycles) with {
228         cycles >= min;
229         cycles <= max;
230     };
231     assert (rand_success) else $error("Failed to randomize
wait cycles!");
232     repeat (cycles) @(posedge this.drv.apb.clk_i);
233     endtask : rand_wait
234
235 task handle_requests();
236     forever begin
237         automatic apb_request_t request;
238         automatic apb_response_t read_response = new;
239         automatic logic rand_success;
240         drv.recv_request(request);
241         request_queue.put(request);
242         if (request.pwrite) begin
243             rand_wait(REQ_MIN_WAIT_CYCLES,
RESP_MAX_WAIT_CYCLES);
244             drv.send_write_ack();
245         end else begin
246             rand_success = read_response.randomize(); as-
sert(rand_success);
247             response_queue.put(read_response);
248             rand_wait(REQ_MIN_WAIT_CYCLES,
RESP_MAX_WAIT_CYCLES);
249             drv.send_read_response(read_response);
250         end
251     end
252 endtask : handle_requests // recv_requests
253
254 task run();
255     handle_requests();
256 endtask
257
258 endclass
259
260 class apb_rand_master #(
261     // APB interface parameters
262     parameter int unsigned ADDR_WIDTH = 32'd32,
263     parameter int unsigned DATA_WIDTH = 32'd32,
264     // Stimuli application and test time
265     parameter time TA = 0ps,
266     parameter time TT = 0ps,
267     // Upper and lower bounds on wait cycles on request chan-
nel
268     parameter int REQ_MIN_WAIT_CYCLES = 0,
269     parameter int REQ_MAX_WAIT_CYCLES = 20
270 );
271
272 typedef apb_test::apb_driver #(
273     .ADDR_WIDTH(ADDR_WIDTH), .DATA_WIDTH(DATA_WIDTH), .TA(TA
), .TT(TT)
274 ) apb_driver_t;
275
276 typedef apb_driver_t::addr_t addr_t;
277 typedef apb_driver_t::data_t data_t;
278 typedef apb_driver_t::strb_t strb_t;

```

```

279
280 typedef apb_driver_t::apb_request_t apb_request_t;
281 typedef apb_driver_t::apb_response_t apb_response_t;
282
283 apb_driver_t      drv;
284 std::mailbox #(apb_request_t) request_queue;
285 std::mailbox #(apb_response_t) response_queue;
286
287
288 function new(
289     virtual APB_DV #(
290         .ADDR_WIDTH(ADDR_WIDTH),
291         .DATA_WIDTH(DATA_WIDTH)
292     ) apb
293 );
294     this.drv          = new(apb);
295     this.request_queue = new;
296     this.response_queue = new;
297     this.reset();
298 endfunction
299
300 function reset();
301     drv.reset_master();
302 endfunction
303
304 // TODO: The `rand_wait` task exists in `rand_verif_pkg`,
but that task cannot be called with
305 // `this.drv.apb.clk_i` as `clk` argument. What is the
syntax getting an assignable reference?
306 task automatic rand_wait(input int unsigned min, max);
307     int unsigned rand_success, cycles;
308     rand_success = std::randomize(cycles) with {
309         cycles >= min;
310         cycles <= max;
311     };
312     assert (rand_success) else $error("Failed to randomize
wait cycles!");
313     repeat (cycles) @(posedge this.drv.apb.clk_i);
314     endtask
315
316 task automatic send_requests(input int unsigned n_re-
quests);
317     automatic apb_request_t request = new;
318     automatic apb_response_t response = new;
319     automatic logic write_err;
320     automatic logic rand_success;
321     repeat (n_requests) begin
322         rand_wait(REQ_MIN_WAIT_CYCLES, REQ_MAX_WAIT_CYCLES);
323         rand_success = request.randomize(); as-
sert(rand_success);
324         request_queue.put(request);
325         if (request.pwrite) begin
326             $info("Sending write request with addr: %h, pstrb:
%b and pwwdata: %h", request.paddr, request.pstrb, request.pwwdata);
327             drv.write(request.paddr, request.pwwdata, re-
quest.pstrb, write_err);
328         end else begin
329             $info("Sending read request with addr: %h...", re-
quest.paddr);
330             drv.read(request.paddr, response.prrdata, re-
sponse.pslverr);
331             response_queue.put(response);
332         end
333     end
334 endtask : send_requests
335
336 task automatic run(input int unsigned n_request);
337     $info("Run apb_rand_master for %0d transactions", n_re-
quest);
338     send_requests(n_request);
339 endtask
340
341 endclass
342 endpackage

```

RISC-V 디버그

What this is. The PULP RISC-V debug module.

Why it exists. Because **debug must work when nothing else does**, which is why it is a separate module with its own transport and clocking.

Undergraduate review · EECS concepts. JTAG; debug transport modules; abstract commands; program buffers.

Graduate · Practice note. Compare with the OpenTitan debug module: both implement the same specification with different structural choices, which is a useful exercise in reading two implementations of one document.

12 files • 3,665 lines

API — dm_csrs.sv

Table 1633. `pulp-platform_riscv-dbg/src/dm_csrs.sv` — module / interface / package (1개)

이름	종류
dm_csrs	module

Table 1634. `pulp-platform_riscv-dbg/src/dm_csrs.sv` — parameter (3개)

파라미터	형/폭	기본값
NrHarts	int unsigned	1
BusWidth	int unsigned	32
SelectableHarts	logic [NrHarts-1:0]	{NrHarts{1'b1}}

Table 1635. `pulp-platform_riscv-dbg/src/dm_csrs.sv` — localparam (6개)

상수	형/폭	값
DataIndexWidth	int unsigned	$\$clog2(dm::DataCount)$
HartSelLen	int unsigned	$(NrHarts == 1) ? 1 : \$clog2(NrHarts)$
NrHartsAligned	int unsigned	$2^{HartSelLen}$
HartSelMask	logic [19:0]	$20'((2^{HartSelLen} - 1))$
DataEnd	dm::dm_csr_e	$dm::dm_csr_e'(dm::Data0 + \{4'h0\})$
ProgBufEnd	dm::dm_csr_e	$dm::dm_csr_e'(dm::ProgBuf0 + \{4'h0\})$

Table 1636. `pulp-platform_riscv-dbg/src/dm_csrs.sv` — 포트 (4개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
next_dm_addr_i	input	logic [31:0]
testmode_i	input	logic

포트	방향	형/폭
dmi_rst_ni	input	logic
dmi_req_valid_i	input	logic
dmi_req_ready_o	output	logic
dmi_req_i	input	dm::dmi_req_t
dmi_resp_valid_o	output	logic
dmi_resp_ready_i	input	logic
dmi_resp_o	output	dm::dmi_resp_t
ndmreset_o	output	logic
ndmreset_ack_i	input	logic
dmactive_o	output	logic
hartinfo_i	input	dm::hartinfo_t [NrHarts-1:0]
halted_i	input	logic [NrHarts-1:0]
unavailable_i	input	logic [NrHarts-1:0]
resumeack_i	input	logic [NrHarts-1:0]
hartsel_o	output	logic [19:0]
haltreq_o	output	logic [NrHarts-1:0]
resumereq_o	output	logic [NrHarts-1:0]
clear_resumeack_o	output	logic [NrHarts-1:0]
cmd_valid_o	output	logic
cmd_o	output	dm::command_t
cmderror_valid_i	input	logic
cmderror_i	input	dm::cmderr_e
cmdbusy_i	input	logic
progbuf_o	output	logic [dm::ProgBufSize-1:0][31:0]
data_o	output	logic [dm::DataCount-1:0][31:0]
data_i	input	logic [dm::DataCount-1:0][31:0]
data_valid_i	input	logic
sbaddress_o	output	logic [BusWidth-1:0]
sbaddress_i	input	logic [BusWidth-1:0]
sbaddress_write_valid_o	output	logic
sbreadonaddr_o	output	logic
sbautoincrement_o	output	logic
sbaccess_o	output	logic [2:0]
sbreadondata_o	output	logic
sbdata_o	output	logic [BusWidth-1:0]

포트	방향	형/폭
sbdata_read_valid_o	output	logic
sbdata_write_valid_o	output	logic
sbdata_i	input	logic [BusWidth-1:0]
sbdata_valid_i	input	logic
sbbusy_i	input	logic
sberror_valid_i	input	logic
occurred	input	logic [2:0] sberror_i // bus error

API — dm_mem.sv

Table 1637. pulp-platform_riscv-dbg/src/dm_mem.sv — module / interface / package (1개)

이름	종류
dm_mem	module

Table 1638. pulp-platform_riscv-dbg/src/dm_mem.sv — parameter (5개)

파라미터	형/폭	기본값
NrHarts	int unsigned	1
BusWidth	int unsigned	32
SelectableHarts	logic [NrHarts-1:0]	{NrHarts{1'b1}}
DmBaseAddress	int unsigned	'0
MaxRegisterAccessWidth	int unsigned	BusWidth

Table 1639. pulp-platform_riscv-dbg/src/dm_mem.sv — localparam (20개)

상수	형/폭	값
DbgAddressBits	int unsigned	12
DataIndexWidth	int unsigned	$\$clog2(dm::DataCount)$
HartSelLen	int unsigned	$(NrHarts == 1) ? 1 : \$clog2(NrHarts)$
NrHartsAligned	int unsigned	$2^{HartSelLen}$
MaxAar	int unsigned	$(MaxRegisterAccessWidth == 64) ? 4 : 3$
HasSndScratch	bit	$(DmBaseAddress != 0)$
LoadBaseAddr	logic [4:0]	$(DmBaseAddress == 0) ? 5'd0 : 5'd10$
DataBaseAddr	logic [DbgAddressBits-1:0]	$(dm::DataAddr)$
DataEndAddr	logic [DbgAddressBits-1:0]	$(dm::DataAddr + 4*dm::DataCount - 1)$
ProgBufBaseAddr	logic [DbgAddressBits-1:0]	$(dm::DataAddr - 4*dm::ProgBufSize)$
ProgBufEndAddr	logic [DbgAddressBits-1:0]	$(dm::DataAddr - 1)$
AbstractCmdBaseAddr	logic [DbgAddressBits-1:0]	$(ProgBufBaseAddr - 4*10)$
AbstractCmdEndAddr	logic [DbgAddressBits-1:0]	$(ProgBufBaseAddr - 1)$

상수	형/폭	값
WhereToAddr	logic [DbgAddressBits-1:0]	'h300
FlagsBaseAddr	logic [DbgAddressBits-1:0]	'h400
FlagsEndAddr	logic [DbgAddressBits-1:0]	'h7FF
HaltedAddr	logic [DbgAddressBits-1:0]	'h100
GoingAddr	logic [DbgAddressBits-1:0]	'h108
ResumingAddr	logic [DbgAddressBits-1:0]	'h110
ExceptionAddr	logic [DbgAddressBits-1:0]	'h118

Table 1640. pulp-platform_riscv-dbg/src/dm_mem.sv — 포트 (26개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
dmactive_i	input	logic
debug_req_o	output	logic [NrHarts-1:0]
ndmreset_i	input	logic
hartsel_i	input	logic [19:0]
haltreq_i	input	logic [NrHarts-1:0]
resumereq_i	input	logic [NrHarts-1:0]
clear_resumeack_i	input	logic [NrHarts-1:0]
halted_o	output	logic [NrHarts-1:0]
resuming_o	output	logic [NrHarts-1:0]
progbuf_i	input	logic [dm::ProgBufSize-1:0][31:0]
data_i	input	logic [dm::DataCount-1:0][31:0]
data_o	output	logic [dm::DataCount-1:0][31:0]
data_valid_o	output	logic
cmd_valid_i	input	logic
cmd_i	input	dm::command_t
cmderror_valid_o	output	logic
cmderror_o	output	dm::cmderr_e
cmdbusy_o	output	logic
req_i	input	logic
we_i	input	logic
addr_i	input	logic [BusWidth-1:0]
wdata_i	input	logic [BusWidth-1:0]
be_i	input	logic [BusWidth/8-1:0]

포트	방향	형/폭
rdata_o	output	logic [BusWidth-1:0]

Table 1641. pulp-platform_riscv-dbg/src/dm_mem.sv — typedef / using (1개)

새 이름	원래 형
state_e	enum logic [1:0] { Idle, Go, CmdExecuting }

API — dm_obi_top.sv**Table 1642. pulp-platform_riscv-dbg/src/dm_obi_top.sv — module / interface / package (1개)**

이름	종류
dm_obi_top	module

Table 1643. pulp-platform_riscv-dbg/src/dm_obi_top.sv — parameter (6개)

파라미터	형/폭	기본값
IdWidth	int unsigned	1
NrHarts	int unsigned	1
BusWidth	int unsigned	32
DmBaseAddress	int unsigned	'h1000
SelectableHarts	logic [NrHarts-1:0]	{NrHarts{1'b1}}
MaxRegisterAccessWidth	int unsigned	BusWidth

Table 1644. pulp-platform_riscv-dbg/src/dm_obi_top.sv — 포트 (35개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
testmode_i	input	logic
ndmreset_o	output	logic
dmactive_o	output	logic
debug_req_o	output	logic [NrHarts-1:0]
unavailable_i	input	logic [NrHarts-1:0]
hartinfo_i	input	dm::hartinfo_t [NrHarts-1:0]
slave_req_i	input	logic
slave_gnt_o	output	logic
slave_we_i	input	logic
slave_addr_i	input	logic [BusWidth-1:0]
slave_be_i	input	logic [BusWidth/8-1:0]
slave_wdata_i	input	logic [BusWidth-1:0]
slave_aid_i	input	logic [IdWidth-1:0]

포트	방향	형/폭
slave_rvalid_o	output	logic
slave_rdata_o	output	logic [BusWidth-1:0]
slave_rid_o	output	logic [IdWidth-1:0]
master_req_o	output	logic
master_addr_o	output	logic [BusWidth-1:0]
master_we_o	output	logic
master_wdata_o	output	logic [BusWidth-1:0]
master_be_o	output	logic [BusWidth/8-1:0]
master_gnt_i	input	logic
master_rvalid_i	input	logic
master_err_i	input	logic
master_other_err_i	input	logic
master_rdata_i	input	logic [BusWidth-1:0]
dmi_rst_ni	input	logic
dmi_req_valid_i	input	logic
dmi_req_ready_o	output	logic
dmi_req_i	input	dm::dmi_req_t
dmi_resp_valid_o	output	logic
dmi_resp_ready_i	input	logic
dmi_resp_o	output	dm::dmi_resp_t

API — dm_pkg.sv

Table 1645. pulp-platform_riscv-dbg/src/dm_pkg.sv — module / interface / package (1개)

이름	종류
dm	package

Table 1646. pulp-platform_riscv-dbg/src/dm_pkg.sv — localparam (11개)

상수	형/폭	값
DbgVersion013	logic [3:0]	4'h2
ProgBufSize	logic [4:0]	5'h8
DataCount	logic [3:0]	4'h2
HaltAddress	logic [63:0]	64'h800
ResumeAddress	logic [63:0]	HaltAddress + 8
ExceptionAddress	logic [63:0]	HaltAddress + 16
DataAddr	logic [11:0]	12'h380
CauseBreakpoint	logic [2:0]	3'h1

상수	형/폭	값
CauseTrigger	logic [2:0]	3'h2
CauseRequest	logic [2:0]	3'h3
CauseSingleStep	logic [2:0]	3'h4

API — dm_sba.sv

Table 1647. pulp-platform_riscv-dbg/src/dm_sba.sv — module / interface / package (1개)

이름	종류
dm_sba	module

Table 1648. pulp-platform_riscv-dbg/src/dm_sba.sv — parameter (2개)

파라미터	형/폭	기본값
BusWidth	int unsigned	32
ReadByteEnable	bit	1

Table 1649. pulp-platform_riscv-dbg/src/dm_sba.sv — localparam (1개)

상수	형/폭	값
BeIdxWidth	int	\$clog2(BusWidth/8)

Table 1650. pulp-platform_riscv-dbg/src/dm_sba.sv — 포트 (28개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
dmactive_i	input	logic
master_req_o	output	logic
master_add_o	output	logic [BusWidth-1:0]
master_we_o	output	logic
master_wdata_o	output	logic [BusWidth-1:0]
master_be_o	output	logic [BusWidth/8-1:0]
master_gnt_i	input	logic
master_r_valid_i	input	logic
master_r_err_i	input	logic
master_r_other_err_i	input	logic
master_r_rdata_i	input	logic [BusWidth-1:0]
sbaddress_i	input	logic [BusWidth-1:0]
sbaddress_write_valid_i	input	logic
sbreadonaddr_i	input	logic
sbaddress_o	output	logic [BusWidth-1:0]

포트	방향	형/폭
sbautoincrement_i	input	logic
sbaccess_i	input	logic [2:0]
sbreadondata_i	input	logic
sbdata_i	input	logic [BusWidth-1:0]
sbdata_read_valid_i	input	logic
sbdata_write_valid_i	input	logic
sbdata_o	output	logic [BusWidth-1:0]
sbdata_valid_o	output	logic
sbbusy_o	output	logic
sberror_valid_o	output	logic
occurred	output	logic [2:0] sberror_o // bus error

API — dm_top.sv

Table 1651. pulp-platform_riscv-dbg/src/dm_top.sv — module / interface / package (1개)

이름	종류
dm_top	module

Table 1652. pulp-platform_riscv-dbg/src/dm_top.sv — parameter (6개)

파라미터	형/폭	기본값
NrHarts	int unsigned	1
BusWidth	int unsigned	32
DmBaseAddress	int unsigned	'h1000
SelectableHarts	logic [NrHarts-1:0]	{NrHarts{1'b1}}
ReadByteEnable	bit	1
MaxRegisterAccessWidth	int unsigned	BusWidth

Table 1653. pulp-platform_riscv-dbg/src/dm_top.sv — 포트 (33개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
next_dm_addr_i	input	logic [31:0]
testmode_i	input	logic
ndmreset_o	output	logic
ndmreset_ack_i	input	logic
dmactive_o	output	logic
debug_req_o	output	logic [NrHarts-1:0]

포트	방향	형/폭
unavailable_i	input	logic [NrHarts-1:0]
hartinfo_i	input	dm::hartinfo_t [NrHarts-1:0]
slave_req_i	input	logic
slave_we_i	input	logic
slave_addr_i	input	logic [BusWidth-1:0]
slave_be_i	input	logic [BusWidth/8-1:0]
slave_wdata_i	input	logic [BusWidth-1:0]
slave_rdata_o	output	logic [BusWidth-1:0]
master_req_o	output	logic
master_add_o	output	logic [BusWidth-1:0]
master_we_o	output	logic
master_wdata_o	output	logic [BusWidth-1:0]
master_be_o	output	logic [BusWidth/8-1:0]
master_gnt_i	input	logic
master_r_valid_i	input	logic
master_r_err_i	input	logic
master_r_other_err_i	input	logic
master_r_rdata_i	input	logic [BusWidth-1:0]
dmi_rst_ni	input	logic
dmi_req_valid_i	input	logic
dmi_req_ready_o	output	logic
dmi_req_i	input	dm::dmi_req_t
dmi_resp_valid_o	output	logic
dmi_resp_ready_i	input	logic
dmi_resp_o	output	dm::dmi_resp_t

API — dmi_bscane_tap.sv

Table 1654. pulp-platform_riscv-dbg/src/dmi_bscane_tap.sv — module / interface / package (1개)

이름	종류
dmi_jtag_tap	module

Table 1655. pulp-platform_riscv-dbg/src/dmi_bscane_tap.sv — parameter (2개)

파라미터	형/폭	기본값
IrLength	int unsigned	5
IdcodeValue	logic [31:0]	32'h00000001

Table 1656. pulp-platform_riscv-dbg/src/dmi_bscane_tap.sv — 포트 (17개)

포트	방향	형/폭
tck_i	input	logic
tms_i	input	logic
trst_ni	input	logic
td_i	input	logic
td_o	output	logic
tdo_oe_o	output	logic
testmode_i	input	logic
tck_o	output	logic
dmi_clear_o	output	logic
update_o	output	logic
capture_o	output	logic
shift_o	output	logic
tdi_o	output	logic
dtmcs_select_o	output	logic
dtmcs_tdo_i	input	logic
dmi_select_o	output	logic
dmi_tdo_i	input	logic

API — dmi_cdc.sv**Table 1657. pulp-platform_riscv-dbg/src/dmi_cdc.sv — module / interface / package (1개)**

이름	종류
dmi_cdc	module

Table 1658. pulp-platform_riscv-dbg/src/dmi_cdc.sv — 포트 (18개)

포트	방향	형/폭
tck_i	input	logic
trst_ni	input	logic
jtag_dmi_req_i	input	dm::dmi_req_t
jtag_dmi_ready_o	output	logic
jtag_dmi_valid_i	input	logic
jtag_dmi_cdc_clear_i	input	logic
jtag_dmi_resp_o	output	dm::dmi_resp_t
jtag_dmi_valid_o	output	logic
jtag_dmi_ready_i	input	logic
clk_i	input	logic

포트	방향	형/폭
rst_ni	input	logic
core_dmi_rst_no	output	logic
core_dmi_req_o	output	dm::dmi_req_t
core_dmi_valid_o	output	logic
core_dmi_ready_i	input	logic
core_dmi_resp_i	input	dm::dmi_resp_t
core_dmi_ready_o	output	logic
core_dmi_valid_i	input	logic

API — dmi_intf.sv

Table 1659. pulp-platform_riscv-dbg/src/dmi_intf.sv — module / interface / package (1개)

이름	종류
DMI_BUS_DV	interface

Table 1660. pulp-platform_riscv-dbg/src/dmi_intf.sv — parameter (1개)

파라미터	형/폭	기본값
ADDR_WIDTH	int	-1

Table 1661. pulp-platform_riscv-dbg/src/dmi_intf.sv — 포트 (6개)

포트	방향	형/폭
clk_i	input	logic
q_addr	input	
q_ready	output	
q_addr	output	
q_ready	input	
q_addr	input	

Table 1662. pulp-platform_riscv-dbg/src/dmi_intf.sv — typedef / using (2개)

새 이름	원래 형
addr_t	logic [ADDR_WIDTH-1:0]
data_t	logic [31:0]

API — dmi_jtag.sv

Table 1663. pulp-platform_riscv-dbg/src/dmi_jtag.sv — module / interface / package (1개)

이름	종류
dmi_jtag	module

Table 1664. pulp-platform_riscv-dbg/src/dmi_jtag.sv — parameter (1개)

파라미터	형/폭	기본값
IdcodeValue	logic [31:0]	32'h00000DB3

Table 1665. pulp-platform_riscv-dbg/src/dmi_jtag.sv — 포트 (16개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
testmode_i	input	logic
dmi_rst_no	output	logic
dmi_req_o	output	dm::dmi_req_t
dmi_req_valid_o	output	logic
dmi_req_ready_i	input	logic
dmi_resp_i	input	dm::dmi_resp_t
dmi_resp_ready_o	output	logic
dmi_resp_valid_i	input	logic
tck_i	input	logic
tms_i	input	logic
trst_ni	input	logic
td_i	input	logic
td_o	output	logic
enable	output	logic tdo_oe_o // Data out output

Table 1666. pulp-platform_riscv-dbg/src/dmi_jtag.sv — typedef / using (1개)

새 이름	원래 형
state_e	enum logic [2:0] { Idle, Read, WaitReadValid, Write, WaitWriteValid }

API — dmi_jtag_tap.sv**Table 1667. pulp-platform_riscv-dbg/src/dmi_jtag_tap.sv — module / interface / package (1개)**

이름	종류
dmi_jtag_tap	module

Table 1668. pulp-platform_riscv-dbg/src/dmi_jtag_tap.sv — parameter (2개)

파라미터	형/폭	기본값
IrLength	int unsigned	5
IdcodeValue	logic [31:0]	32'h00000001

Table 1669. pulp-platform_riscv-dbg/src/dmi_jtag_tap.sv — 포트 (17개)

포트	방향	형/폭
tck_i	input	logic

포트	방향	형/폭
tms_i	input	logic
trst_ni	input	logic
td_i	input	logic
td_o	output	logic
tdo_oe_o	output	logic
testmode_i	input	logic
tck_o	output	logic
dmi_clear_o	output	logic
update_o	output	logic
capture_o	output	logic
shift_o	output	logic
tdi_o	output	logic
dtmcs_select_o	output	logic
dtmcs_tdo_i	input	logic
dmi_select_o	output	logic
dmi_tdo_i	input	logic

API — dmi_test.sv

Table 1670. pulp-platform_riscv-dbg/src/dmi_test.sv — module / interface / package (1개)

이름	종류
dmi_test	package

Table 1671. pulp-platform_riscv-dbg/src/dmi_test.sv — parameter (24개)

파라미터	형/폭	기본값
AW	int	7
AW	int	-1
TA	realtime	0
TT	realtime	0 // stimuli test time
AW	int	32
TA	realtime	0ps
TT	realtime	0ps
AW	int	32
TA	realtime	0ps
TT	realtime	0ps
REQ_MIN_WAIT_CYCLES	int unsigned	1
REQ_MAX_WAIT_CYCLES	int unsigned	20

파라미터	형/폭	기본값
RSP_MIN_WAIT_CYCLES	int unsigned	1
RSP_MAX_WAIT_CYCLES	int unsigned	20
AW	int	32
TA	realtime	Ops
TT	realtime	Ops
REQ_MIN_WAIT_CYCLES	int unsigned	0
REQ_MAX_WAIT_CYCLES	int unsigned	10
RSP_MIN_WAIT_CYCLES	int unsigned	0
RSP_MAX_WAIT_CYCLES	int unsigned	10
AW	int	32
TA	realtime	Ops
TT	realtime	Ops

1. pulp-platform_riscv-dbg/src/dm_csrs.sv (722 lines)

```

1 /* Copyright 2018 ETH Zurich and University of Bologna.
2  * Copyright and related rights are licensed under the Solder-
3  * pad Hardware
4  * License, Version 0.51 (the "License"); you may not use this
5  * file except in
6  * compliance with the License. You may obtain a copy of the
7  * License at
8  * http://solderpad.org/licenses/SHL-0.51. Unless required by
9  * applicable law
10  * or agreed to in writing, software, hardware and materials
11  * distributed under
12  * this License is distributed on an "AS IS" BASIS, WITHOUT
13  * WARRANTIES OR
14  * CONDITIONS OF ANY KIND, either express or implied. See the
15  * License for the
16  * specific language governing permissions and limitations un-
17  * der the License.
18  *
19  * File: dm_csrs.sv
20  * Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
21  * Date: 30.6.2018
22  *
23  * Description: Debug CSRs. Communication over Debug Transport
24  * Module (DTM)
25  */
26
27 module dm_csrs #(
28     parameter int unsigned      NrHarts      = 1,
29     parameter int unsigned      BusWidth     = 32,
30     parameter logic [NrHarts-1:0] SelectableHarts =
{NrHarts{1'b1}}
31 ) (
32     input logic                 clk_i,           // Clock
33     input logic                 rst_i,           // Asynchronous reset active low
34     input logic [31:0]          next_dm_ad-
dr_i, // Static next_dm word address.
35     input logic                 testmode_i,
36     input logic                 dmi_rst_i,       // sync. DTM reset,
37
38     // active-low
39     input logic                 dmi_req_valid_i,
40     output logic                dmi_req_ready_o,
41     input dm::dmi_req_t         dmi_req_i,
42     // every request needs a response one cycle later
43     output logic                dmi_resp_valid_o,
44     input logic                 dmi_resp_ready_i,
45     output dm::dmi_resp_t       dmi_resp_o,
46     // global ctrl
47     output logic                ndmre-
set_o, // non-debug module reset active-high
48     input logic                 ndmre-
set_ack_i, // non-debug module reset ack pulse
49     output logic                dmact-
ive_o, // 1 -> debug-module is active,

```

```

40
/ 0 -> synchronous re-set
41 // hart status
42 input dm::hartinfo_t [NrHarts-1:0]
hartinfo_i, // static hartinfo
43 input logic [NrHarts-1:0]          hal-
ted_i, // hart is halted
44 input logic [NrHarts-1:0]          unavail-
able_i, // e.g.: powered down
45 input logic [NrHarts-1:0]          re-
sumeack_i, // hart acknowledged resume request
46 // hart control
47 output logic [19:0]
hartsel_o, // hartselect to ctrl module
48 output logic [NrHarts-1:0]          hal-
req_o, // request to halt a hart
49 output logic [NrHarts-1:0]          resume-
req_o, // request hart to resume
50 // Clears stale acknowledgements before issuing new resume
requests
51 output logic [NrHarts-1:0]          clear_resumeack_o,
52
53 output logic                        cmd_val-
id_o, // debugger writing to cmd field
54 output dm::command_t
cmd_o, // abstract command
55 input logic                        cmd-
ror_valid_i, // an error occurred
56 input dm::cmderr_e                cmd-
ror_i, // this error occurred
57 input logic                        cmd-
busy_i, // cmd is currently busy executing
58
59 output logic [dm::ProgBufSize-1:0][31:0] progbuf_o, // to
system bus
60 output logic [dm::DataCount-1:0][31:0] data_o,
61
62 input logic [dm::DataCount-1:0][31:0] data_i,
63 input logic
data_valid_i,
64 // system bus access module (SBA)
65 output logic [BusWidth-1:0]          sbaddress_o,
66 input logic [BusWidth-1:0]          sbaddress_i,
67 output logic
sbaddress_o,
68 // control signals in
69 output logic
sbreadonaddr_o,
70 output logic
sbautoincrement_o,
71 output logic [2:0]
sbaccess_o,
72 // data out
73 output logic
sbreadondata_o,
74 output logic [BusWidth-1:0]          sbdata_o,
75 output logic
sb-
data_read_valid_o,
76 output logic
sb-
data_write_valid_o,
77 // read data in
78 input logic [BusWidth-1:0]          sbdata_i,
79 input logic
sbdata_valid_i,
80 // control signals
81 input logic
sbbusy_i,
82 input logic
sber-
ror_valid_i, // bus error occurred
83 input logic [2:0]
serror_i // bus

```

```

error occurred
84 );
85 // the amount of bits we need to represent all harts
86 localparam int unsigned DataIndexWidth = $clog2(NrHarts);
87 localparam int unsigned HartSelLen = (NrHarts == 1) ? 1 :
88 $clog2(NrHarts);
89 localparam int unsigned NrHartsAligned = 2**HartSelLen;
90 // Keep only the implemented WARL hart-select bits
91 localparam logic [19:0] HartSelMask = 20'((2**$clog2(NrHarts)) - 1);
92
93 dm::dtm_op_e dtm_op;
94 assign dtm_op = dm::dtm_op_e'(dmi_req_i.op);
95
96 logic resp_queue_full;
97 logic resp_queue_empty;
98 logic resp_queue_push;
99 logic resp_queue_pop;
100
101 localparam dm::dm_csr_e DataEnd = dm::dm_csr_e'(dm::Data0 +
102 {4'h0, dm::DataCount} - 8'h1);
103 localparam dm::dm_csr_e ProgBufEnd = dm::dm_csr_e'(dm::Prog-
104 Buf0 + {4'h0, dm::ProgBufSize} - 8'h1);
105
106 logic [31:0] haltsum0, haltsum1, haltsum2, haltsum3;
107 logic [((NrHarts-1)/2**5 + 1) * 32 - 1 : 0] halted;
108 logic [((NrHarts-1)/2**5:0)[31:0] halted_resaped0;
109 logic [((NrHarts-1)/2**10:0)[31:0] halted_resaped1;
110 logic [((NrHarts-1)/2**15:0)[31:0] halted_resaped2;
111 logic [((NrHarts-1)/2**10+1)*32-1:0] halted_flat1;
112 logic [((NrHarts-1)/2**15+1)*32-1:0] halted_flat2;
113 logic [31:0] halted_flat3;
114
115 // haltsum0
116 logic [14:0] hartsel_idx0;
117 always_comb begin : p_haltsum0
118   halted = '0;
119   haltsum0 = '0;
120   hartsel_idx0 = hartsel_o[19:5];
121   halted[NrHarts-1:0] = halted_i;
122   halted_resaped0 = halted;
123   if (hartsel_idx0 < 15'((NrHarts-1)/2**5+1)) begin
124     haltsum0 = halted_resaped0[hartsel_idx0];
125   end
126 end
127
128 // haltsum1
129 logic [9:0] hartsel_idx1;
130 always_comb begin : p_reduction1
131   halted_flat1 = '0;
132   haltsum1 = '0;
133   hartsel_idx1 = hartsel_o[19:10];
134
135   for (int unsigned k = 0; k < (NrHarts-1)/2**5+1; k++) be-
136   gin
137     halted_flat1[k] = |halted_resaped0[k];
138   end
139   halted_resaped1 = halted_flat1;
140
141   if (hartsel_idx1 < 10'(((NrHarts-1)/2**10+1))) begin
142     haltsum1 = halted_resaped1[hartsel_idx1];
143   end
144 end
145
146 // haltsum2
147 logic [4:0] hartsel_idx2;
148 always_comb begin : p_reduction2
149   halted_flat2 = '0;
150   haltsum2 = '0;
151   hartsel_idx2 = hartsel_o[19:15];
152
153   for (int unsigned k = 0; k < (NrHarts-1)/2**10+1; k++) be-
154   gin
155     halted_flat2[k] = |halted_resaped1[k];
156   end
157   halted_resaped2 = halted_flat2;
158
159   if (hartsel_idx2 < 5'(((NrHarts-1)/2**15+1))) begin
160     haltsum2 = halted_resaped2[hartsel_idx2];
161   end
162 end
163
164 // haltsum3
165 always_comb begin : p_reduction3
166   halted_flat3 = '0;
167   for (int unsigned k = 0; k < NrHarts/2**15+1; k++) begin
168     halted_flat3[k] = |halted_resaped2[k];
169   end
170   haltsum3 = halted_flat3;
171 end
172
173 dm::dmstatus_t dmstatus;
174 dm::dmcontrol_t dmcontrol_d, dmcontrol_q, dmcon-
175 trol_read;
176 dm::abstractcs_t abstractcs;
177 dm::cmderr_e cmderr_d, cmderr_q;
178 dm::command_t command_d, command_q;
179 logic cmd_valid_d, cmd_valid_q;
180 dm::abstractauto_t abstractauto_d, abstractauto_q;
181
182 dm::sbcs_t sbcs_d, sbcs_q;
183 logic [63:0] sbaddr_d, sbaddr_q;
184 logic [63:0] sbdata_d, sbdata_q;
185
186 logic [NrHarts-1:0] havereset_d, havereset_q;
187 logic [NrHarts-1:0] resume_pending_d, resume_pending_q;
188 // program buffer
189 logic [dm::ProgBufSize-1:0][31:0] progbuf_d, progbuf_q;
190 logic [dm::DataCount-1:0][31:0] data_d, data_d_dmi, data_q;
191
192 logic [HartSelLen-1:0] selected_hart;
193 logic selected_hart_valid;
194
195 dm::dmi_resp_t resp_queue_inp;
196
197 assign dmi_resp_valid_o = ~resp_queue_empty;
198 assign dmi_req_ready_o = ~resp_queue_full;
199 assign resp_queue_push = dmi_req_valid_i &
200 dmi_req_ready_o;
201 // SBA
202 assign sbautoincrement_o = sbcs_q.sbautoincrement;
203 assign sbreadonaddr_o = sbcs_q.sbreadonaddr;
204 assign sbreadondata_o = sbcs_q.sbreadondata;
205 assign sbaccess_o = sbcs_q.sbaccess;
206 assign sbdata_o = sbdata_q[BusWidth-1:0];
207 assign sbaddress_o = sbaddr_q[BusWidth-1:0];
208
209 assign hartsel_o = {dmcontrol_q.hartselhi, dmcon-
210 trol_q.hartsello};
211 assign selected_hart = hartsel_o[HartSelLen-1:0];
212 assign selected_hart_valid = selected_hart <= HartSel-
213 Len'(NrHarts - 1);
214
215 logic [NrHartsAligned-1:0] havereset_q_aligned, re-
216 sumeack_aligned, unavailable_aligned,
217 halted_aligned;
218 assign resumeack_aligned = NrHartsAligned'(resumeack_i);
219 assign unavailable_aligned = NrHartsAligned'(unavailable_i);
220 assign halted_aligned = NrHartsAligned'(halted_i);
221
222 assign havereset_q_aligned = NrHartsAligned'(havereset_q);
223
224 dm::hartinfo_t [NrHartsAligned-1:0] hartinfo_aligned;
225 always_comb begin : p_hartinfo_align
226   hartinfo_aligned = '0;
227   hartinfo_aligned[NrHarts-1:0] = hartinfo_i;
228 end
229
230 // helper variables
231 dm::dm_csr_e dm_csr_addr;
232 dm::sbcs_t sbcs;
233 logic [3:0] autoexecdata_idx; // 0 == Data0 ... 11 == Data11
234
235 // Decode immediate write effects independently of stored
236 CSR state
237 logic [19:0] dmcontrol_write_hartsel;
238 logic [HartSelLen-1:0] dmcontrol_write_hart;
239 logic dmcontrol_write_hart_valid;
240 logic dmcontrol_write_accepted;
241 logic dmcontrol_write_resumereq;
242 logic dmcontrol_write_haltreq;
243 logic dmcontrol_write_dmactive;
244
245 // Get the data index, i.e. 0 for dm::Data0 up to 11 for
246 dm::Data11
247 assign dm_csr_addr = dm::dm_csr_e'({1'b0, dmi_req_i.addr});
248 // Xilinx Vivado 2020.1 does not allow subtraction of two
249 enums; do the subtraction with logic
250 // types instead.
251 assign autoexecdata_idx = 4'({dm_csr_addr} - {dm::Data0});
252
253 assign dmcontrol_write_hartsel = {dmi_req_i.data[15:6],
254 dmi_req_i.data[25:16]}
255 & HartSelMask;
256 assign dmcontrol_write_hart = dmcon-
257 trol_write_hartsel[HartSelLen-1:0];
258 assign dmcontrol_write_hart_valid = dmcontrol_write_hartsel
259 <= 20'(NrHarts - 1);
260 assign dmcontrol_write_accepted = dmi_req_ready_o &
261 dmi_req_valid_i &
262 dtm_op == dm::DTM_WRITE &
263 dm_csr_addr == dm::DMCon-
264 trol;
265 assign dmcontrol_write_resumereq = dmcontrol_write_accepted
266 & !cmdbusy_i & dmi_req_i.data[30];
267 assign dmcontrol_write_haltreq = dmi_req_i.data[31];
268 assign dmcontrol_write_dmactive = dmi_req_i.data[0];
269
270 always_comb begin : p_clear_resumeack
271   clear_resumeack_o = '0;
272   if (dmcontrol_write_resumereq & !dmcontrol_write_haltreq
273 &
274 dmcontrol_write_hart_valid & Selectable-
275 Harts[dmcontrol_write_hart]) begin
276     clear_resumeack_o[dmcontrol_write_hart] = 1'b1;
277   end
278 end
279
280 always_comb (*xprop_off *) begin : csr_read_write
281 // -----
282 // Static Values (R/O)

```

```

262 // -----
263 // dmstatus
264 dmstatus = '0;
265 dmstatus.version = dm::DbgVersion013;
266 // no authentication implemented
267 dmstatus.authenticated = 1'b1;
268 // we do not support halt-on-reset sequence
269 dmstatus.hasrethaltreq = 1'b0;
270 // TODO(zarubaf) things need to change here if we imple-
ment the array mask
271 dmstatus.allhavereset = haver-
reset_q_aligned[selected_hart];
272 dmstatus.anyhavereset = haver-
reset_q_aligned[selected_hart];
273
274 dmstatus.allresumeack = resumeack_aligned[selected_hart];
275 dmstatus.anyresumeack = resumeack_aligned[selected_hart];
276
277 dmstatus.allunavail = unavail-
able_aligned[selected_hart];
278 dmstatus.anyunavail = unavail-
able_aligned[selected_hart];
279
280 // as soon as we are out of the legal Hart region tell the
debugger
281 // that there are only non-existent harts
282 dmstatus.allnonexistent = logic'(32'(hartsel_o) > (NrHarts -
1));
283 dmstatus.anynonexistent = logic'(32'(hartsel_o) > (NrHarts -
1));
284
285 // We are not allowed to be in multiple states at once.
This is a to
286 // make the running/halted and unavailable states exclus-
ive.
287 dmstatus.allhalted = halted_aligned[selected_hart] &
~unavailable_aligned[selected_hart];
288 dmstatus.anyhalted = halted_aligned[selected_hart] &
~unavailable_aligned[selected_hart];
289
290 dmstatus.allrunning = ~halted_aligned[selected_hart] &
~unavailable_aligned[selected_hart];
291 dmstatus.anyrunning = ~halted_aligned[selected_hart] &
~unavailable_aligned[selected_hart];
292
293 // abstractcs
294 abstractcs = '0;
295 abstractcs.datacount = dm::DataCount;
296 abstractcs.progbufsize = dm::ProgBufSize;
297 abstractcs.busy = cmdbusy_i;
298 abstractcs.cmderr = cmderr_q;
299
300 // abstractautoexec
301 abstractauto_d = abstractauto_q;
302 abstractauto_d.zero0 = '0;
303
304 // default assignments
305 havereset_d = havereset_q;
306 dmcontrol_d = dmcontrol_q;
307 dmcontrol_read = dmcontrol_q;
308 dmcontrol_read.haltreq = 1'b0;
309 dmcontrol_read.resumereq = 1'b0;
310 resume_pending_d = resume_pending_q;
311 // Acknowledgements retire only the corresponding pending
actions
312 resume_pending_d &= ~resumeack_i;
313 cmderr_d = cmderr_q;
314 command_d = command_q;
315 progbuf_d = progbuf_q;
316 data_d_dmi = data_q;
317 sbcs_d = sbcs_q;
318 sbaddr_d = 64'(sbaddress_i);
319 sbdata_d = sbdata_q;
320
321 resp_queue_inp.data = 32'h0;
322 resp_queue_inp.resp = dm::DTM_SUCCESS;
323 cmd_valid_d = 1'b0;
324 sbaddress_write_valid_o = 1'b0;
325 sbdata_read_valid_o = 1'b0;
326 sbdata_write_valid_o = 1'b0;
327
328 // helper variables
329 sbcs = '0;
330
331 // reads
332 if (dmi_req_ready_o && dmi_req_valid_i && dtm_op ==
dm::DTM_READ) begin
333   unique case (dm_csr_addr) inside
334     [(dm::Data0):DataEnd]: begin
335       resp_queue_inp.data = data_q[DataIndex-
Width'(autoexecdata_idx)];
336       if (!cmdbusy_i) begin
337         // check whether we need to re-execute the command
(just give a cmd_valid)
338         if (cmderr_q == dm::CmdErrNone)
339           cmd_valid_d = abstractau-
to_q.autoexecdata[autoexecdata_idx];
340
341         // An abstract command was executing while one of
the data registers was read
342         end else begin
343           resp_queue_inp.resp = dm::DTM_BUSY;
344           if (cmderr_q == dm::CmdErrNone) begin
345             cmderr_d = dm::CmdErrBusy;
346           end
347         end
348         dm::DMControl: resp_queue_inp.data = dmcon-
trol_read;
349         dm::DMStatus: resp_queue_inp.data = dmstatus;
350         dm::Hartinfo: resp_queue_inp.data =
hartinfo_aligned[selected_hart];
351         dm::AbstractCS: resp_queue_inp.data = abstractcs;
352         dm::AbstractAuto: resp_queue_inp.data = abstractau-
to_q;
353         dm::Command: resp_queue_inp.data = '0;
354         dm::NextDM: resp_queue_inp.data = next_dm_ad-
dr_i;
355         [(dm::ProgBuf0):ProgBufEnd]: begin
356           resp_queue_inp.data = prog-
buf_q[dmi_req_i.addr[$clog2(dm::ProgBufSize)-1:0]];
357           if (!cmdbusy_i) begin
358             // check whether we need to re-execute the command
(just give a cmd_valid)
359             // range of autoexecprogbuf is 31:16
360             if (cmderr_q == dm::CmdErrNone)
361               cmd_valid_d = abstractau-
to_q.autoexecprogbuf[{1'b1, dmi_req_i.addr[3:0]}];
362             // An abstract command was executing while one of
the progbuf registers was read
363             end else begin
364               resp_queue_inp.resp = dm::DTM_BUSY;
365               if (cmderr_q == dm::CmdErrNone) begin
366                 cmderr_d = dm::CmdErrBusy;
367               end
368             end
369             dm::HaltSum0: resp_queue_inp.data = haltsum0;
370             dm::HaltSum1: resp_queue_inp.data = haltsum1;
371             dm::HaltSum2: resp_queue_inp.data = haltsum2;
372             dm::HaltSum3: resp_queue_inp.data = haltsum3;
373             dm::SBCS: begin
374               resp_queue_inp.data = sbcs_q;
375             end
376             dm::SBAddress0: begin
377               resp_queue_inp.data = sbaddr_q[31:0];
378             end
379             dm::SBAddress1: begin
380               resp_queue_inp.data = sbaddr_q[63:32];
381             end
382             dm::SBData0: begin
383               // access while the SBA was busy
384               if (sbbusy_i || sbcs.q.sbbusyserror) begin
385                 sbcs_d.sbbusyserror = 1'b1;
386                 resp_queue_inp.resp = dm::DTM_BUSY;
387               end else begin
388                 sbdata_read_valid_o = (sbcs.q.serror == '0);
389                 resp_queue_inp.data = sbdata_q[31:0];
390               end
391             end
392             dm::SBData1: begin
393               // access while the SBA was busy
394               if (sbbusy_i || sbcs.q.sbbusyserror) begin
395                 sbcs_d.sbbusyserror = 1'b1;
396                 resp_queue_inp.resp = dm::DTM_BUSY;
397               end else begin
398                 resp_queue_inp.data = sbdata_q[63:32];
399               end
400             end
401             default:;
402             endcase
403           end
404           if (dmi_req_ready_o && dmi_req_valid_i && dtm_op ==
dm::DTM_WRITE) begin
405             unique case (dm_csr_addr) inside
406               [(dm::Data0):DataEnd]: begin
407                 if (dm::DataCount > 0) begin
408                   // attempts to write them while busy is set does
not change their value
409                   if (!cmdbusy_i) begin
410                     data_d_dmi[dmi_req_i.addr[DataIndexWidth-1:0]] =
dmi_req_i.data;
411                     // check whether we need to re-execute the com-
mand (just give a cmd_valid)
412                     if (cmderr_q == dm::CmdErrNone)
413                       cmd_valid_d = abstractau-
to_q.autoexecdata[autoexecdata_idx];
414                     //An abstract command was executing while one of
the data registers was written
415                     end else begin
416                       resp_queue_inp.resp = dm::DTM_BUSY;
417                       if (cmderr_q == dm::CmdErrNone) begin
418                         cmderr_d = dm::CmdErrBusy;
419                       end
420                     end
421                   end
422                 end
423             end

```

```

340 // An abstract command was executing while one of
the data registers was read
341 end else begin
342   resp_queue_inp.resp = dm::DTM_BUSY;
343   if (cmderr_q == dm::CmdErrNone) begin
344     cmderr_d = dm::CmdErrBusy;
345   end
346 end
347 end
348 dm::DMControl: resp_queue_inp.data = dmcon-
trol_read;
349 dm::DMStatus: resp_queue_inp.data = dmstatus;
350 dm::Hartinfo: resp_queue_inp.data =
hartinfo_aligned[selected_hart];
351 dm::AbstractCS: resp_queue_inp.data = abstractcs;
352 dm::AbstractAuto: resp_queue_inp.data = abstractau-
to_q;
353 dm::Command: resp_queue_inp.data = '0;
354 dm::NextDM: resp_queue_inp.data = next_dm_ad-
dr_i;
355 [(dm::ProgBuf0):ProgBufEnd]: begin
356   resp_queue_inp.data = prog-
buf_q[dmi_req_i.addr[$clog2(dm::ProgBufSize)-1:0]];
357   if (!cmdbusy_i) begin
358     // check whether we need to re-execute the command
(just give a cmd_valid)
359     // range of autoexecprogbuf is 31:16
360     if (cmderr_q == dm::CmdErrNone)
361       cmd_valid_d = abstractau-
to_q.autoexecprogbuf[{1'b1, dmi_req_i.addr[3:0]}];
362     // An abstract command was executing while one of
the progbuf registers was read
363     end else begin
364       resp_queue_inp.resp = dm::DTM_BUSY;
365       if (cmderr_q == dm::CmdErrNone) begin
366         cmderr_d = dm::CmdErrBusy;
367       end
368     end
369     dm::HaltSum0: resp_queue_inp.data = haltsum0;
370     dm::HaltSum1: resp_queue_inp.data = haltsum1;
371     dm::HaltSum2: resp_queue_inp.data = haltsum2;
372     dm::HaltSum3: resp_queue_inp.data = haltsum3;
373     dm::SBCS: begin
374       resp_queue_inp.data = sbcs_q;
375     end
376     dm::SBAddress0: begin
377       resp_queue_inp.data = sbaddr_q[31:0];
378     end
379     dm::SBAddress1: begin
380       resp_queue_inp.data = sbaddr_q[63:32];
381     end
382     dm::SBData0: begin
383       // access while the SBA was busy
384       if (sbbusy_i || sbcs.q.sbbusyserror) begin
385         sbcs_d.sbbusyserror = 1'b1;
386         resp_queue_inp.resp = dm::DTM_BUSY;
387       end else begin
388         sbdata_read_valid_o = (sbcs.q.serror == '0);
389         resp_queue_inp.data = sbdata_q[31:0];
390       end
391     end
392     dm::SBData1: begin
393       // access while the SBA was busy
394       if (sbbusy_i || sbcs.q.sbbusyserror) begin
395         sbcs_d.sbbusyserror = 1'b1;
396         resp_queue_inp.resp = dm::DTM_BUSY;
397       end else begin
398         resp_queue_inp.data = sbdata_q[63:32];
399       end
400     end
401     default:;
402     endcase
403   end
404   if (dmi_req_ready_o && dmi_req_valid_i && dtm_op ==
dm::DTM_WRITE) begin
405     unique case (dm_csr_addr) inside
406       [(dm::Data0):DataEnd]: begin
407         if (dm::DataCount > 0) begin
408           // attempts to write them while busy is set does
not change their value
409           if (!cmdbusy_i) begin
410             data_d_dmi[dmi_req_i.addr[DataIndexWidth-1:0]] =
dmi_req_i.data;
411             // check whether we need to re-execute the com-
mand (just give a cmd_valid)
412             if (cmderr_q == dm::CmdErrNone)
413               cmd_valid_d = abstractau-
to_q.autoexecdata[autoexecdata_idx];
414             //An abstract command was executing while one of
the data registers was written
415             end else begin
416               resp_queue_inp.resp = dm::DTM_BUSY;
417               if (cmderr_q == dm::CmdErrNone) begin
418                 cmderr_d = dm::CmdErrBusy;
419               end
420             end
421           end
422         end
423       end

```

```

424     end
425   end
426 end
427 dm::DMControl: begin
428   if (cmdbusy_i) begin
429     dmcontrol_d.ndmreset = dmi_req_i.data[1];
430     dmcontrol_d.dmactive = dmi_req_i.data[0];
431   end else begin
432     dmcontrol_d = dmi_req_i.data;
433   end
434   // clear the havreset of the selected hart
435   if (dmcontrol_d.ackhavereset && selected_hart_valid)
begin
436     havreset_d[selected_hart] = 1'b0;
437   end
438 end
439 dm::DMStatus:; // write are ignored to R/O register
440 dm::Hartinfo:; // hartinfo is R/O
441 // only command error is write-able
442 dm::AbstractCS: begin // W1C
443   // Gets set if an abstract command fails. The bits
in this
444   // field remain set until they are cleared by writ-
ing 1 to
445   // them. No abstract command is started until the
value is
446   // reset to 0.
447   // reads during abstract command execution are not
allowed
448   if (!cmdbusy_i) begin
449     cmderr_d = dm::cmderr_e'(~dmi_req_i.data[10:8] &
cmderr_q);
450   end else begin
451     resp_queue_inp.resp = dm::DTM_BUSY;
452     if (cmderr_q == dm::CmdErrNone) begin
453       cmderr_d = dm::CmdErrBusy;
454     end
455   end
456 end
457 dm::Command: begin
458   // writes are ignored if a command is already busy
or if cmderr is non-zero
459   if (!cmdbusy_i && cmderr_q == dm::CmdErrNone) begin
460     cmd_valid_d = 1'b1;
461     command_d = dm::command_t'(dmi_req_i.data);
462     // if there was an attempted to write during a busy
execution
463     // and the cmderror field is zero set the busy error
464   end else if (cmdbusy_i) begin
465     resp_queue_inp.resp = dm::DTM_BUSY;
466     if (cmderr_q == dm::CmdErrNone) begin
467       cmderr_d = dm::CmdErrBusy;
468     end
469   end
470 end
471 dm::NextDM:; // nextdm is R/O
472 dm::AbstractAuto: begin
473   // this field can only be written legally when there
is no command executing
474   if (!cmdbusy_i) begin
475     abstractauto_d = 32'h0;
476     abstractauto_d.autoexecdata =
12'(dmi_req_i.data[dm::DataCount-1:0]);
477     abstractauto_d.autoexecprogbuf =
16'(dmi_req_i.data[dm::ProgBufSize-1+16:16]);
478   end else begin
479     resp_queue_inp.resp = dm::DTM_BUSY;
480     if (cmderr_q == dm::CmdErrNone) begin
481       cmderr_d = dm::CmdErrBusy;
482     end
483   end
484 end
485 [(dm::ProgBuf0):ProgBufEnd]: begin
486   // attempts to write them while busy is set does not
change their value
487   if (!cmdbusy_i) begin
488     prog-
buf_d[dmi_req_i.addr[$clog2(dm::ProgBufSize)-1:0]] = dmi_req_i.data;
489   // check whether we need to re-execute the command
(just give a cmd_valid)
490   // this should probably throw an error if executed
during another command
491   // was busy
492   // range of autoexecprogbuf is 31:16
493   if (cmderr_q == dm::CmdErrNone)
494     cmd_valid_d = abstractau-
to_q.autoexecprogbuf[{1'b1, dmi_req_i.addr[3:0]}];
495   //An abstract command was executing while one of the
progbuf registers was written
496   end else begin
497     resp_queue_inp.resp = dm::DTM_BUSY;
498     if (cmderr_q == dm::CmdErrNone) begin
499       cmderr_d = dm::CmdErrBusy;
500     end
501   end

```

```

502   end
503   dm::SBCS: begin
504     // access while the SBA was busy
505     if (sbbusy_i) begin
506       sbcs_d.sbbusyyerror = 1'b1;
507       resp_queue_inp.resp = dm::DTM_BUSY;
508     end else begin
509       sbcs = dm::sbcs_t'(dmi_req_i.data);
510       sbcs_d = sbcs;
511       // R/W1C
512       sbcs_d.sbbusyyerror = sbcs_q.sbbusyyerror &
(~sbcs.sbbusyyerror);
513       sbcs_d.sbberror = (|sbcs.sbberror) ? 3'b0 :
sbcs_q.sbberror;
514     end
515   end
516   dm::SBAddress0: begin
517     // access while the SBA was busy
518     if (sbbusy_i || sbcs_q.sbbusyyerror) begin
519       sbcs_d.sbbusyyerror = 1'b1;
520       resp_queue_inp.resp = dm::DTM_BUSY;
521     end else begin
522       sbaddr_d[31:0] = dmi_req_i.data;
523       sbaddress_write_valid_o = (sbcs_q.sbberror == '0);
524     end
525   end
526   dm::SBAddress1: begin
527     // access while the SBA was busy
528     if (sbbusy_i || sbcs_q.sbbusyyerror) begin
529       sbcs_d.sbbusyyerror = 1'b1;
530       resp_queue_inp.resp = dm::DTM_BUSY;
531     end else begin
532       sbaddr_d[63:32] = dmi_req_i.data;
533     end
534   end
535   dm::SBData0: begin
536     // access while the SBA was busy
537     if (sbbusy_i || sbcs_q.sbbusyyerror) begin
538       sbcs_d.sbbusyyerror = 1'b1;
539       resp_queue_inp.resp = dm::DTM_BUSY;
540     end else begin
541       sbdata_d[31:0] = dmi_req_i.data;
542       sbdata_write_valid_o = (sbcs_q.sbberror == '0);
543     end
544   end
545   dm::SBData1: begin
546     // access while the SBA was busy
547     if (sbbusy_i || sbcs_q.sbbusyyerror) begin
548       sbcs_d.sbbusyyerror = 1'b1;
549       resp_queue_inp.resp = dm::DTM_BUSY;
550     end else begin
551       sbdata_d[63:32] = dmi_req_i.data;
552     end
553   end
554   default:;
555 endcase
556 end
557 // hart threw a command error and has precedence over bus
writes
558 if (cmderror_valid_i) begin
559   cmderr_d = cmderror_i;
560 end
561 // set the havreset flag when the ndmreset completed
562 if (ndmreset_ack_i) begin
563   havreset_d = 1;
564 end
565 // -----
566 // System Bus
567 // -----
568 // set bus error
569 if (sberror_valid_i) begin
570   sbcs_d.sbberror = sberror_i;
571 end
572 // update read data
573 if (sbdata_valid_i) begin
574   sbdata_d = 64'(sbdata_i);
575 end
576 // dmcontrol
577 // TODO(zarubaf) we currently do not implement the har-
tarry mask
578 dmcontrol_d.hasel = 1'b0;
579 // we do not support resetting an individual hart
580 dmcontrol_d.hartreset = 1'b0;
581 dmcontrol_d.setresetaltreq = 1'b0;
582 dmcontrol_d.clrresetaltreq = 1'b0;
583 dmcontrol_d.zero1 = '0;
584 dmcontrol_d.zero0 = '0;
585 // Non-writeable, clear only
586 dmcontrol_d.ackhavereset = 1'b0;
587 // resumereq is W1/read-zero and is represented by the
per-hart pending vector
588 dmcontrol_d.resumereq = 1'b0;
589 // Set a request only for an effective, selectable hart
that is halted when
590 // the write is accepted. A new request wins over a same-
cycle acknowledgement
591 if (dmcontrol_write_resumereq && dmcon-
trol_write_hart_valid &&

```



```

595     SelectableHarts[dmcontrol_write_hart] &&
596     halted_aligned[dmcontrol_write_hart] &&
597     !unavailable_aligned[dmcontrol_write_hart] &&
598     !dmcontrol_write_haltreq) begin
599         resume_pending_d[dmcontrol_write_hart] = 1'b1;
600     end
601
602     // Deactivating the debug module clears all pending ac-
tions
603     if (dmcontrol_write_accepted && !dmcontrol_write_dmativce)
begin
604         resume_pending_d = '0;
605     end
606     // WARL behavior of hartsel, depending on NrHarts.
607     // If NrHarts = 1 this is just masked to all-zeros.
608     {dmcontrol_d.hartselhi, dmcontrol_d.hartsello} &= HartSel-
Mask;
609     // static values for dcsr
610     sbcs_d.sbversion = 3'd1;
611     sbcs_d.sbbusy = sbbusy_i;
612     sbcs_d.sbasize = $bits(sbcs_d.sbas-
ize)'(BusWidth);
613     sbcs_d.sbaccess128 = logic'(BusWidth >= 32'd128);
614     sbcs_d.sbaccess64 = logic'(BusWidth >= 32'd64);
615     sbcs_d.sbaccess32 = logic'(BusWidth >= 32'd32);
616     sbcs_d.sbaccess16 = logic'(BusWidth >= 32'd16);
617     sbcs_d.sbaccess8 = logic'(BusWidth >= 32'd8);
618 end
619
620 // Hart writes take precedence over DMI writes to the ab-
struct data registers.
621 assign data_d = data_valid_i ? data_i : data_d_dmi;
622
623 // output multiplexer
624 always_comb begin : p_outmux
625     // default assignment
626     haltreq_o = '0;
627     resumereq_o = resume_pending_q;
628     if (selected_hart_valid) begin
629         haltreq_o[selected_hart] = dmcontrol_q.haltreq;
630     end
631 end
632
633 assign dmativce_o = dmcontrol_q.dmativce;
634 assign cmd_o = command_q;
635 assign cmd_valid_o = cmd_valid_q;
636 assign progbuf_o = progbuf_q;
637 assign data_o = data_q;
638
639 assign resp_queue_pop = dmi_resp_ready_i &
~resp_queue_empty;
640
641 assign ndmreset_o = dmcontrol_q.ndmreset;
642
643 // response FIFO
644 fifo_v3 #(
645     .dtype (logic [$bits(dmi_resp_o)-1:0] ),
646     .DEPTH (2)
647 ) i_fifo (
648     .clk_i,
649     .rst_ni,
650     .flush_i ( ~dmi_rst_ni ), // Flush the
queue if the DTM is
651     // reset
652     .testmode_i (testmode_i ),
653     .full_o (resp_queue_full ),
654     .empty_o (resp_queue_empty ),
655     .usage_o ( ),
656     .data_i (resp_queue_inp ),
657     .push_i (resp_queue_push ),
658     .data_o (dmi_resp_o ),
659     .pop_i (resp_queue_pop )
660 );
661
662 always_ff @(posedge clk_i or negedge rst_ni) begin : p_regs
663     // PoR
664     if (!rst_ni) begin
665         dmcontrol_q <= '0;
666         // this is the only write-able bit during reset
667         cmderr_q <= dm::CmdErrNone;
668         command_q <= '0;
669         cmd_valid_q <= '0;
670         abstractauto_q <= '0;
671         progbuf_q <= '0;
672         data_q <= '0;
673         sbcs_q <= '{default: '0, sbaccess: 3'd2};
674         sbaddr_q <= '0;
675         sbdata_q <= '0;
676         havereset_q <= '1;
677         resume_pending_q <= '0;
678     end else begin
679         havereset_q <= SelectableHarts & havereset_d;
680         // synchronous re-set of debug module, active-low, ex-
cept for dmativce
681         if (!dmcontrol_q.dmativce) begin
682             dmcontrol_q.haltreq <= '0;
683             dmcontrol_q.resumereq <= '0;
684             dmcontrol_q.hartreset <= '0;
685             dmcontrol_q.ackhavereset <= '0;
686             dmcontrol_q.zero1 <= '0;

```

```

687         dmcontrol_q.hasel <= '0;
688         dmcontrol_q.hartsello <= '0;
689         dmcontrol_q.hartselhi <= '0;
690         dmcontrol_q.zero0 <= '0;
691         dmcontrol_q.setresethaltreq <= '0;
692         dmcontrol_q.clrresethaltreq <= '0;
693         dmcontrol_q.ndmreset <= '0;
694         resume_pending_q <= '0;
695         // this is the only write-able bit during reset
696         dmcontrol_q.dmativce <= dmcontrol_d.dmativce;
697         cmderr_q <= dm::CmdErrNone;
698         command_q <= '0;
699         cmd_valid_q <= '0;
700         abstractauto_q <= '0;
701         progbuf_q <= '0;
702         data_q <= '0;
703         sbcs_q <= '{default: '0, sbac-
cess: 3'd2};
704         sbaddr_q <= '0;
705         sbdata_q <= '0;
706     end else begin
707         dmcontrol_q <= dmcontrol_d;
708         resume_pending_q <= SelectableHarts & re-
sume_pending_d;
709         cmderr_q <= cmderr_d;
710         command_q <= command_d;
711         cmd_valid_q <= cmd_valid_d;
712         abstractauto_q <= abstractauto_d;
713         progbuf_q <= progbuf_d;
714         data_q <= data_d;
715         sbcs_q <= sbcs_d;
716         sbaddr_q <= sbaddr_d;
717         sbdata_q <= sbdata_d;
718     end
719 end
720 end
721
722 endmodule : dm_csr

```

2. pulp-platform_riscv-dbg/src/dm_mem.sv (571 lines)

```

1 /* Copyright 2018 ETH Zurich and University of Bologna.
2 * Copyright and related rights are licensed under the Solder-
pad Hardware
3 * License, Version 0.51 (the "License"); you may not use this
file except in
4 * compliance with the License. You may obtain a copy of the
License at
5 * http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 * or agreed to in writing, software, hardware and materials
distributed under
7 * this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 * CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 * specific language governing permissions and limitations un-
der the License.
10 *
11 * File: dm_mem.sv
12 * Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
13 * Date: 11.7.2018
14 *
15 * Description: Memory module for execution-based debug clients
16 *
17 */
18
19 module dm_mem #(
20     parameter int unsigned NrHarts = 1,
21     parameter int unsigned BusWidth = 32,
22     parameter logic [NrHarts-1:0] SelectableHarts =
{NrHarts{1'b1}},
23     parameter int unsigned DmBaseAddress = '0,
24     // Maximum width supported by Access Register commands; must be
32 or 64.
25     parameter int unsigned MaxRegisterAccessWidth =
BusWidth
26 ) (
27     input logic clk_i, //
28     input logic rst_ni, // de-
bug module reset
29     input logic dmativce_i, // de-
bug module is active
30
31     output logic [NrHarts-1:0] debug_req_o,
32     input logic ndmreset_i,
33     input logic [19:0] hartsel_i,
34     // from Ctrl and Status register
35     input logic [NrHarts-1:0] haltreq_i,
36     input logic [NrHarts-1:0] resumereq_i,
37     // Clears stale acknowledgements before new resume requests
38     input logic [NrHarts-1:0] clear_resumeack_i,
39
40     // state bits
41     output logic [NrHarts-1:0] halted_o, //
hart acknowledge halt
42     output logic [NrHarts-1:0] resuming_o, //
hart is resuming

```



```

43
44 input logic [dm::ProgBufSize-1:0][31:0] progbuf_i, //
program buffer to expose
45
46 input logic [dm::DataCount-1:0][31:0] data_i, //
data in
47 output logic [dm::DataCount-1:0][31:0] data_o, //
data out
48 output logic
data out is valid
data_valid_o, //
49 // abstract command interface
50 input logic cmd_valid_i,
51 input dm::command_t cmd_i,
52 output logic cmderror_valid_o,
53 output dm::cmderr_e cmderror_o,
54 output logic cmdbusy_o,
55 // data interface
56
57 // SRAM interface
58 input logic req_i,
59 input logic we_i,
60 input logic [BusWidth-1:0] addr_i,
61 input logic [BusWidth-1:0] wdata_i,
62 input logic [BusWidth/8-1:0] be_i,
63 output logic [BusWidth-1:0] rdata_o
64 );
65 localparam int unsigned DbgAddressBits = 12;
66 localparam int unsigned DataIndexWidth = $clog2(dm::DataCount);
67 localparam int unsigned HartSellen = (NrHarts == 1) ?
1 : $clog2(NrHarts);
68 localparam int unsigned NrHartsAligned = 2**HartSellen;
69 localparam int unsigned MaxAar = (MaxRegisterAccess-
Width == 64) ? 4 : 3;
70 localparam bit HasSndScratch = (DmBaseAddress !=
0);
71 // Depending on whether we are at the zero page or not we
either use `x0` or `x10/a0`
72 localparam logic [4:0] LoadBaseAddr = (DmBaseAddress ==
0) ? 5'd0 : 5'd10;
73
74 localparam logic [DbgAddressBits-1:0] DataBaseAddr =
(dm::DataAddr);
75 localparam logic [DbgAddressBits-1:0] DataEndAddr =
(dm::DataAddr + 4*dm::DataCount - 1);
76 localparam logic [DbgAddressBits-1:0] ProgBufBaseAddr =
(dm::DataAddr - 4*dm::ProgBufSize);
77 localparam logic [DbgAddressBits-1:0] ProgBufEndAddr =
(dm::DataAddr - 1);
78 localparam logic [DbgAddressBits-1:0] AbstractCmdBaseAddr =
(ProgBufBaseAddr - 4*10);
79 localparam logic [DbgAddressBits-1:0] AbstractCmdEndAddr =
(ProgBufBaseAddr - 1);
80
81 localparam logic [DbgAddressBits-1:0] WhereToAddr = 'h300;
82 localparam logic [DbgAddressBits-1:0] FlagsBaseAddr = 'h400;
83 localparam logic [DbgAddressBits-1:0] FlagsEndAddr = 'h7FF;
84
85 localparam logic [DbgAddressBits-1:0] HaltedAddr = 'h100;
86 localparam logic [DbgAddressBits-1:0] GoingAddr = 'h108;
87 localparam logic [DbgAddressBits-1:0] ResumingAddr = 'h110;
88 localparam logic [DbgAddressBits-1:0] ExceptionAddr = 'h118;
89
90 initial begin : p_validate_max_register_access_width
91 assert (MaxRegisterAccessWidth == 32 || MaxRegisterAccess-
Width == 64) else
92 $fatal(1, "MaxRegisterAccessWidth must be 32 or 64.");
93 end
94
95 logic [dm::ProgBufSize/2-1:0][63:0] progbuf;
96 logic [7:0][63:0] abstract_cmd;
97 logic [NrHarts-1:0] halted_d, halted_q;
98 logic [NrHarts-1:0] resuming_d, resuming_q;
99 logic go, going;
100
101 logic exception;
102 logic unsupported_command;
103
104 logic [63:0] rom_rdata;
105 logic [63:0] rdata_d, rdata_q;
106 logic word_enable32_q;
107
108 logic [HartSellen-1:0] hartsel, wdata_hartsel;
109 logic [HartSellen-1:0] cmd_hartsel_d, cmd_hartsel_q;
110 logic wdata_hartsel_valid;
111 logic [DbgAddressBits-1:0] flags_hart_idx;
112 logic [HartSellen-1:0] flags_hart;
113
114 assign hartsel = hartsel_i[HartSellen-1:0];
115 assign wdata_hartsel = wdata_i[HartSellen-1:0];
116 assign wdata_hartsel_valid = wdata_i[31:0] <= 32'(NrHarts -
1);
117 assign flags_hart_idx = addr_i[DbgAddressBits-1:0] -
FlagsBaseAddr;
118 assign flags_hart = HartSellen'(flags_hart_idx);
119
120 logic [NrHartsAligned-1:0] resumereq_aligned, hal-
ted_q_aligned, halted_aligned;
121
122 assign resumereq_aligned = NrHarts-
Aligned'(resumereq_i);
123
124 assign halted_q_aligned = NrHartsAligned'(halted_q);
125
126 // distinguish whether we need to forward data from the ROM
or the FSM
127 // latch the address for this
128 logic fwd_rom_d, fwd_rom_q;
129 dm::ac_ar_cmd_t ac_ar;
130
131 // Abstract Command Access Register
132 assign ac_ar = dm::ac_ar_cmd_t'(cmd_i.control);
133 assign debug_req_o = haltreq_i;
134 assign halted_o = halted_q;
135 assign resuming_o = resuming_q;
136
137 // reshape progbuf
138 assign progbuf = progbuf_i;
139
140 typedef enum logic [1:0] { Idle, Go, CmdExecuting } state_e;
141 state_e state_d, state_q;
142
143 // hart ctrl queue
144 always_comb begin : p_hart_ctrl_queue
145 cmderror_valid_o = 1'b0;
146 cmderror_o = dm::CmdErrNone;
147 state_d = state_q;
148 cmd_hartsel_d = cmd_hartsel_q;
149 go = 1'b0;
150 cmdbusy_o = 1'b1;
151
152 unique case (state_q)
153 Idle: begin
154 cmdbusy_o = 1'b0;
155 if (cmd_valid_i && halted_q_aligned[hartsel] &&
!resumereq_aligned[hartsel] && !unsupport-
ted_command) begin
156 // give the go signal
157 state_d = Go;
158 cmd_hartsel_d = hartsel;
159 end else if (cmd_valid_i) begin
160 // hart must be halted for all requests
161 cmderror_valid_o = 1'b1;
162 cmderror_o = dm::CmdErrorHaltResume;
163 end
164 end
165 Go: begin
166 // we are already busy here since we scheduled the ex-
ecution of a program
167 cmdbusy_o = 1'b1;
168 go = 1'b1;
169 // the thread is now executing the command, track its
state
170 if (going) begin
171 state_d = CmdExecuting;
172 end
173 end
174 CmdExecuting: begin
175 cmdbusy_o = 1'b1;
176 go = 1'b0;
177 // wait until the hart has halted again
178 if (halted_aligned[cmd_hartsel_q]) begin
179 state_d = Idle;
180 end
181 end
182 default: ;
183 endcase
184
185 // only signal once that cmd is unsupported so that we can
clear cmderr
186 // in subsequent writes to abstractcs
187 if (unsupported_command && cmd_valid_i) begin
188 cmderror_valid_o = 1'b1;
189 cmderror_o = dm::CmdErrNotSupported;
190 end
191
192 if (exception) begin
193 cmderror_valid_o = 1'b1;
194 cmderror_o = dm::CmdErrorException;
195 end
196
197 if (ndmreset_i) begin
198 // Clear state of hart and its control signals when it
is being reset.
199 state_d = Idle;
200 go = 1'b0;
201 end
202 end
203
204 // word mux for 32bit and 64bit buses
205 logic [63:0] word_mux;
206 assign word_mux = (fwd_rom_q) ? rom_rdata : rdata_q;
207
208 if (BusWidth == 64) begin : gen_word_mux64
209 assign rdata_o = word_mux;
210 end else begin : gen_word_mux32
211 assign rdata_o = (word_enable32_q) ? word_mux[32 +: 32] :
word_mux[0 +: 32];
212 end

```

```

216 end
217
218 // read/write logic
219 logic [dm::DataCount-1:0][31:0] data_bits;
220 logic [DataIndexWidth-1:0] data_reg_idx;
221 logic [7:0] rdata;
222
223 // Forward writes from the hart to the abstract data registers.
224 // VCS Xprop cannot instrument case-inside range items (see PR #150).
225 always_comb (* xprop_off *) begin : p_data_write
226     data_bits = data_i;
227     data_reg_idx = DataIndexWidth'(
228         addr_i[DbgAddressBits-1:2] - DataBaseAddr
229     );
230     data_valid_o = 1'b0;
231
232     if (req_i && we_i) begin
233         unique case (addr_i[DbgAddressBits-1:0]) inside
234             [DataBaseAddr:DataEndAddr]: begin
235                 data_valid_o = 1'b1;
236                 for (int unsigned byte_idx = 0; byte_idx <
237                     $bits(be_i); byte_idx++) begin
238                     if (be_i[byte_idx] &&
239                         (data_reg_idx + byte_idx / 4) < dm::DataCount)
240                         begin
241                             data_bits[data_reg_idx + byte_idx / 4][(byte_idx %
242                                 4) * 8 +: 8] =
243                                 wdata_i[byte_idx * 8 +: 8];
244                         end
245                     end
246                 default: ;
247             endcase
248         end
249     end
250
251 // Track resume acknowledgements with hart writes taking
252 // priority over clear requests
253 always_comb begin : p_resuming
254     resuming_d = resuming_q;
255     resuming_d &= ~clear_resumeack_i;
256
257     if (req_i && we_i && addr_i[DbgAddressBits-1:0] == Re-
258         sumingAddr &&
259         wdata_hartsel_valid) begin
260         resuming_d[wdata_hartsel] = 1'b1;
261     end
262
263     if (!dmactive_i) begin
264         resuming_d = '0;
265     end
266 end
267 // VCS Xprop cannot instrument case-inside range items (see
268 // PR #150).
269 always_comb (* xprop_off *) begin : p_rw_logic
270
271     halted_d = halted_q;
272     rdata_d = rdata_q;
273     rdata = '0;
274
275     exception = 1'b0;
276     halted_aligned = '0;
277     going = 1'b0;
278
279     // we've got a new request
280     if (req_i) begin
281         // this is a write
282         if (we_i) begin
283             unique case (addr_i[DbgAddressBits-1:0]) inside
284                 HaltedAddr: begin
285                     if (wdata_hartsel_valid) begin
286                         halted_aligned[wdata_hartsel] = 1'b1;
287                         halted_d[wdata_hartsel] = 1'b1;
288                     end
289                 end
290                 GoingAddr: begin
291                     going = 1'b1;
292                 end
293                 ResumingAddr: begin
294                     // clear the halted flag as the hart resumed exe-
295                     // cution
296                     if (wdata_hartsel_valid) begin
297                         halted_d[wdata_hartsel] = 1'b0;
298                     end
299                 end
300                 // an exception occurred during execution
301                 ExceptionAddr: exception = 1'b1;
302                 default: ;
303             endcase
304         end else begin

```

```

305         unique case (addr_i[DbgAddressBits-1:0]) inside
306             // variable ROM content
307             WhereToAddr: begin
308                 // variable jump to abstract cmd, program_buffer
309                 if (wdata_hartsel_valid && resume-
310                     req_aligned[wdata_hartsel]) begin
311                     rdata_d = {32'b0, dm::jal('0, 21'(dm::ResumeAd-
312                         dress[11:0])-21'(WhereToAddr))};
313                     end
314                 // there is a command active so jump there
315                 if (cmdbusy_o) begin
316                     // transfer not set is shortcut to the program
317                     buffer if postexec is set
318                     // keep this statement narrow to not catch in-
319                     // valid commands
320                     if (cmd_i.cmdtype == dm::AccessRegister &&
321                         !ac_ar.transfer && ac_ar.postexec) begin
322                         rdata_d = {32'b0, dm::jal('0, 21'(ProgBufBase-
323                             Addr)-21'(WhereToAddr))};
324                     // this is a legit abstract cmd -> execute it
325                     end else begin
326                         rdata_d = {32'b0, dm::jal('0, 21'(AbstractCmd-
327                             BaseAddr)-21'(WhereToAddr))};
328                     end
329                 end
330             end
331
332             [DataBaseAddr:DataEndAddr]: begin
333                 rdata_d = {
334                     data_i[DataIndexWidth'(
335                         ((addr_i[DbgAddressBits-1:3] -
336                             DataBaseAddr[DbgAddressBits-1:3])
337                         << 1) + 1'b1)],
338                     data_i[DataIndexWidth'(
339                         ((addr_i[DbgAddressBits-1:3] -
340                             DataBaseAddr[DbgAddressBits-1:3])
341                         << 1))]
342                 };
343             end
344
345             [ProgBufBaseAddr:ProgBufEndAddr]: begin
346                 rdata_d = prog-
347                 buf[$clog2(dm::ProgBufSize)'(addr_i[DbgAddressBits-1:3] -
348                     ProgBufBaseAd-
349                     dr[DbgAddressBits-1:3])];
350             end
351
352             // two slots for abstract command
353             [AbstractCmdBaseAddr:AbstractCmdEndAddr]: begin
354                 // return the correct address index
355                 rdata_d = ab-
356                 stract_cmd[3'(addr_i[DbgAddressBits-1:3] -
357                     AbstractCmdBaseAd-
358                     dr[DbgAddressBits-1:3])];
359             end
360
361             // harts are polling for flags here
362             [FlagsBaseAddr:FlagsEndAddr]: begin
363                 // Route resume by flag address and go to the act-
364                 // ive command hart
365                 if (20'(flags_hart_idx) <= 20'(NrHarts - 1)) begin
366                     rdata[addr_i[2:0]] = {
367                         6'b0,
368                         resumereq_aligned[flags_hart] && dmactive_i
369                     }
370                 end
371             end
372
373             // if ndmreset_i,
374             go && dmactive_i && flags_hart ==
375             cmd_hartsel_q
376         end
377     end
378     rdata_d = rdata;
379 end
380 default: ;
381 endcase
382 end
383
384 if (ndmreset_i) begin
385     // When harts are reset, they are not halted.
386     halted_d = '0;
387 end
388
389 always_comb begin : p_abstract_cmd_rom
390     // this abstract command is currently unsupported
391     unsupported_command = 1'b0;
392     // default memory
393     if ac_ar.transfer is not set then we can take a short-
394     cut to the program buffer
395     abstract_cmd[0][31:0] = dm::illegal();
396     // load debug module base address into a0, this is shared
397     among all commands
398     abstract_cmd[0][63:32] = HasSndScratch ? dm::auipc(5'd10,
399         '0) : dm::nop();
400     // clr lowest 12b -> DM base offset
401     abstract_cmd[1][31:0] = HasSndScratch ? dm::srli(5'd10,
402         5'd10, 6'd12) : dm::nop();
403     abstract_cmd[1][63:32] = HasSndScratch ? dm::slli(5'd10,
404         5'd10, 6'd12) : dm::nop();
405     abstract_cmd[2][31:0] = dm::nop();
406     abstract_cmd[2][63:32] = dm::nop();

```

```

385 abstract_cmd[3][31:0] = dm::nop();
386 abstract_cmd[3][63:32] = dm::nop();
387 abstract_cmd[4][31:0] = HasSndScratch ?
dm::csrr(dm::CSR_DSCRATCH1, 5'd10) : dm::nop();
388 abstract_cmd[4][63:32] = dm::ebreak();
389 abstract_cmd[7:5] = '0';
390
391 // this depends on the command being executed
392 unique case (cmd_i.cmdtype)
393 // -----
394 // Access Register
395 // -----
396 dm::AccessRegister: begin
397 if (32'(ac_ar.aarsize) < MaxAar && ac_ar.transfer &&
ac_ar.write) begin
398 // store a0 in dscratch1
399 abstract_cmd[0][31:0] = HasSndScratch ?
dm::csrr(dm::CSR_DSCRATCH1, 5'd10) : dm::nop();
400 // this range is reserved
401 if (ac_ar.regno[15:14] != '0) begin
402 abstract_cmd[0][31:0] = dm::ebreak(); // we leave
asap
403 unsupported_command = 1'b1;
404 // A0 access needs to be handled separately, as we
use A0 to load
405 // the DM address offset need to access DSCRATCH1 in
this case
406 end else if (HasSndScratch && ac_ar.regno[12] && (!
ac_ar.regno[5]) &&
407 (ac_ar.regno[4:0] == 5'd10)) begin
408 // store s0 in dscratch
409 abstract_cmd[2][31:0] = dm::csrw(dm::CSR_D-
SCRATCH0, 5'd8);
410 // load from data register
411 abstract_cmd[2][63:32] = dm::load(ac_ar.aarsize,
5'd8, LoadBaseAddr, dm::DataAddr);
412 // and store it in the corresponding CSR
413 abstract_cmd[3][31:0] = dm::csrw(dm::CSR_D-
SCRATCH1, 5'd8);
414 // restore s0 again from dscratch
415 abstract_cmd[3][63:32] = dm::csrr(dm::CSR_D-
SCRATCH0, 5'd8);
416 // GPR/FPR access
417 end else if (ac_ar.regno[12]) begin
418 // determine whether we want to access the float-
ing point register or not
419 if (ac_ar.regno[5]) begin
420 abstract_cmd[2][31:0] =
dm::float_load(ac_ar.aarsize,
ac_ar.regno[4:0], LoadBaseAddr, dm::DataAddr);
421 end else begin
422 abstract_cmd[2][31:0] =
dm::load(ac_ar.aarsize, ac_ar.regno[4:0],
LoadBaseAddr, dm::DataAddr);
423 end
424 // CSR access
425 end else begin
426 // data register to CSR
427 // store s0 in dscratch
428 abstract_cmd[2][31:0] = dm::csrw(dm::CSR_D-
SCRATCH0, 5'd8);
429 // load from data register
430 abstract_cmd[2][63:32] = dm::load(ac_ar.aarsize,
5'd8, LoadBaseAddr, dm::DataAddr);
431 // and store it in the corresponding CSR
432 abstract_cmd[3][31:0] =
dm::csrw(dm::csr_reg_t'(ac_ar.regno[11:0]), 5'd8);
433 // restore s0 again from dscratch
434 abstract_cmd[3][63:32] = dm::csrr(dm::CSR_D-
SCRATCH0, 5'd8);
435 end
436 end else if (32'(ac_ar.aarsize) < MaxAar &&
ac_ar.transfer && !ac_ar.write) begin
437 // store a0 in dscratch1
438 abstract_cmd[0][31:0] = HasSndScratch ?
dm::csrw(dm::CSR_DSCRATCH1,
LoadBaseAddr) :
dm::nop();
439 // this range is reserved
440 if (ac_ar.regno[15:14] != '0) begin
441 abstract_cmd[0][31:0] = dm::ebreak(); // we
leave asap
442 unsupported_command = 1'b1;
443 // A0 access needs to be handled separately, as we
use A0 to load
444 // the DM address offset need to access DSCRATCH1 in
this case
445 end else if (HasSndScratch && ac_ar.regno[12] && (!
ac_ar.regno[5]) &&
446 (ac_ar.regno[4:0] == 5'd10)) begin
447 // store s0 in dscratch
448 abstract_cmd[2][31:0] = dm::csrw(dm::CSR_D-
SCRATCH0, 5'd8);
449 // read value from CSR into s0
450 abstract_cmd[2][63:32] = dm::csrr(dm::CSR_D-
SCRATCH1, 5'd8);
451 // and store s0 into data section
452 abstract_cmd[3][31:0] = dm::store(ac_ar.aarsize,

```

```

5'd8, LoadBaseAddr, dm::DataAddr);
453 // restore s0 again from dscratch
454 abstract_cmd[3][63:32] = dm::csrr(dm::CSR_D-
SCRATCH0, 5'd8);
455 // GPR/FPR access
456 end else if (ac_ar.regno[12]) begin
457 // determine whether we want to access the float-
ing point register or not
458 if (ac_ar.regno[5]) begin
459 abstract_cmd[2][31:0] =
dm::float_store(ac_ar.aarsize,
ac_ar.regno[4:0], LoadBaseAddr, dm::DataAddr);
460 end else begin
461 abstract_cmd[2][31:0] =
dm::store(ac_ar.aarsize, ac_ar.regno[4:0],
LoadBaseAddr, dm::DataAddr);
462 end
463 // CSR access
464 end else begin
465 // CSR register to data
466 // store s0 in dscratch
467 abstract_cmd[2][31:0] = dm::csrw(dm::CSR_D-
SCRATCH0, 5'd8);
468 // read value from CSR into s0
469 abstract_cmd[2][63:32] =
dm::csrr(dm::csr_reg_t'(ac_ar.regno[11:0]), 5'd8);
470 // and store s0 into data section
471 abstract_cmd[3][31:0] = dm::store(ac_ar.aarsize,
5'd8, LoadBaseAddr, dm::DataAddr);
472 // restore s0 again from dscratch
473 abstract_cmd[3][63:32] = dm::csrr(dm::CSR_D-
SCRATCH0, 5'd8);
474 end
475 end else if (32'(ac_ar.aarsize) >= MaxAar ||
ac_ar.aarpostincrement == 1'b1) begin
476 // this should happen when e.g. ac_ar.aarsize >=
MaxAar
477 // Openocd will try to do an access with aarsize=64
bits
478 // first before falling back to 32 bits.
479 abstract_cmd[0][31:0] = dm::ebreak(); // we leave
asap
480 unsupported_command = 1'b1;
481 end
482 // Check whether we need to execute the program buf-
fer. When we
483 // get an unsupported command we really should abort
instead of
484 // still trying to execute the program buffer, makes
it easier
485 // for the debugger to recover
486 if (ac_ar.postexec && !unsupported_command) begin
487 // issue a nop, we will automatically run into the
program buffer
488 abstract_cmd[4][63:32] = dm::nop();
489 end
490 // not supported at the moment
491 // dm::QuickAccess;
492 // dm::AccessMemory;
493 default: begin
494 abstract_cmd[0][31:0] = dm::ebreak();
495 unsupported_command = 1'b1;
496 end
497 endcase
498 end
499 logic [63:0] rom_addr;
500 assign rom_addr = 64'(addr_i);
501
502 // Depending on whether the debug module is located
503 // at the zero page we can instantiate a simplified version
504 // which only requires one scratch register per hart.
505 // For all other cases we need to set aside
506 // two registers per hart, hence we also need
507 // two scratch registers.
508 if (HasSndScratch) begin : gen_rom_snd_scratch
509 debug_rom i_debug_rom (
510 .clk_i,
511 .rst_ni,
512 .req_i,
513 .addr_i ( rom_addr ),
514 .rdata_o ( rom_rdata )
515 );
516 end else begin : gen_rom_one_scratch
517 // It uses the zero register ('x0') as the base
518 // for its loads. The zero register does not need to
519 // be saved.
520 debug_rom_one_scratch i_debug_rom (
521 .clk_i,
522 .rst_ni,
523 .req_i,
524 .addr_i ( rom_addr ),
525 .rdata_o ( rom_rdata )
526 );
527 end
528 // ROM starts at the HaltAddress of the core e.g.: it imme-
diately jumps to
529 // the ROM base address

```

```

540 assign fwd_rom_d = logic'(addr_i[DbgAddressBits-1:0] >=
dm::HaltAddress[DbgAddressBits-1:0]);
541
542 always_ff @(posedge clk_i or negedge rst_ni) begin : p_regs
543   if (!rst_ni) begin
544     cmd_hartset_q <= '0;
545   end else begin
546     cmd_hartset_q <= cmd_hartset_d;
547   end
548   if (!rst_ni) begin
549     fwd_rom_q <= 1'b0;
550     rdata_q <= '0;
551     state_q <= Idle;
552     word_enable32_q <= 1'b0;
553   end else begin
554     fwd_rom_q <= fwd_rom_d;
555     rdata_q <= rdata_d;
556     state_q <= state_d;
557     word_enable32_q <= addr_i[2];
558   end
559 end
560
561 always_ff @(posedge clk_i or negedge rst_ni) begin
562   if (!rst_ni) begin
563     halted_q <= '0;
564     resuming_q <= '0;
565   end else begin
566     halted_q <= SelectableHarts & halted_d;
567     resuming_q <= SelectableHarts & resuming_d;
568   end
569 end
570
571 endmodule : dm_mem

```

3. pulp-platform_riscv-dbg/src/dm_obi_top.sv (196 lines)

```

1 // Copyright 2020 Silicon Labs, Inc.
2 //
3 // This file, and derivatives thereof are licensed under the
4 // Solderpad License, Version 2.0 (the "License").
5 //
6 // Use of this file means you agree to the terms and condi-
7 // tions of the license and are in full compliance with the License.
8 //
9 // You may obtain a copy of the License at:
10 //
11 // https://solderpad.org/licenses/SHL-2.0/
12 //
13 // Unless required by applicable law or agreed to in writing,
14 // software and hardware implementations thereof distributed under the
15 // License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR
16 // CONDITIONS
17 // OF ANY KIND, EITHER EXPRESSED OR IMPLIED.
18 // See the License for the specific language governing permis-
19 // sions and limitations under the License.
20 //
21 ///////////////////////////////////////////////////////////////////
22 // Engineer: Arjan Bink - ar-
23 // jan.bink@silabs.com
24 //
25 // Design Name: OBI Wrapper for Debug Module
26 // (dm_top)
27 // Project Name:
28 // CV32E40P
29 // Language: SystemVeri-
30 // log
31 //
32 //
33 // Description: Wrapper for the Debug Module (dm_top) which
34 // gives it an // OBI (Open Bus Interface) compatible inter-
35 // faces so that it // can be integrated without further glue lo-
36 // gic (other than // tie offs in an OBI compliant sys-
37 // tem.
38 //
39 //
40 // This wrapper is only intended for OBI com-
41 // pliant systems; //
42 //
43 // in other systems the existing dm_top can be
44 // used as before // and this wrapper can be ig-
45 // nored.
46 //
47 //
48 // The OBI spec is available

```

```

at: //
38 //
39 // // - https://github.com/openhwgroup/core-v-
docs/blob/master/ //
40 // // cores/
cv32e40p/ //
41 //
42 // // Compared to 'logint' interfaces of dm_top
the following //
43 // // signals are ad-
ded: //
44 //
45 // // - slave_* OBI inter-
face: //
46 //
47 // //
48 // // slave_gnt_o //
49 // // slave_rvalid_o //
50 // // slave_aid_i //
51 // // slave_rid_o //
52 // // Compared to 'logint' interfaces of dm_top
the following //
53 // // signals have been re-
named: //
54 //
55 // // - master_* OBI inter-
face: //
56 //
57 // // - Renamed master_add_o to mas-
ter_addr_o //
58 // // - Renamed master_r_valid_i to mas-
ter_rvalid_i //
59 // // - Renamed master_r_rdata_i to mas-
ter_rdata_i //
60 //
61 ///////////////////////////////////////////////////////////////////
62 //
63 module dm_obi_top #(
64   parameter int unsigned IdWidth = 1, //
width of aid/rid
65   parameter int unsigned NrHarts = 1,
66   parameter int unsigned BusWidth = 32,
67   parameter int unsigned DmBaseAddress = 'h1000, //
default to non-zero page
68   // Bitmask to select physically available harts for systems
69   // that don't use hart numbers in a contiguous fashion.
70   parameter logic [NrHarts-1:0] SelectableHarts =
{NrHarts{1'b1}},
71   // Maximum width supported by Access Register commands; must be
32 or 64.
72   parameter int unsigned MaxRegisterAccessWidth =
BusWidth
73 ) (
74   input logic clk_i, // clock
75   // asynchronous reset active low, connect PoR here, not the
system reset
76   input logic rst_ni,
77   input logic testmode_i,
78   output logic ndmreset_o, // non-debug
module reset
79   output logic dmactive_o, // debug mod-
ule is active
80   output logic [NrHarts-1:0] debug_req_o, // async de-
bug request
81   // communicate whether the hart is unavailable (e.g.: power
down)
82   input logic [NrHarts-1:0] unavailable_i,
83   input dm::hartinfo_t [NrHarts-1:0] hartinfo_i,
84
85   input logic slave_req_i,
86   // OBI grant for slave_req_i (not present on dm_top)
87   output logic slave_gnt_o,
88   input logic slave_we_i,
89   input logic [BusWidth-1:0] slave_addr_i,
90   input logic [BusWidth/8-1:0] slave_be_i,
91   input logic [BusWidth-1:0] slave_wdata_i,
92   // Address phase transaction identifier (not present on
dm_top)
93   input logic [IdWidth-1:0] slave_aid_i,

```

```

94 // OBI rvalid signal (end of response phase for reads/
writes) (not present on dm_top)
95 output logic slave_rvalid_o,
96 output logic [BusWidth-1:0] slave_rdata_o,
97 // Response phase transaction identifier (not present on
dm_top)
98 output logic [IdWidth-1:0] slave_rid_o,
99
100 output logic master_req_o,
101 output logic [BusWidth-1:0] master_addr_o, // Renamed
according to OBI spec
102 output logic master_we_o,
103 output logic [BusWidth-1:0] master_wdata_o,
104 output logic [BusWidth/8-1:0] master_be_o,
105 input logic master_gnt_i,

106 input logic master_rvalid_i, // Renamed
according to OBI spec
107 input logic master_err_i,
108 input logic master_other_err_i, // *oth-
er_err_i has priority over *err_i

109 input logic [BusWidth-1:0] master_rdata_i, // Renamed
according to OBI spec
110
111 // Connection to DTM - compatible to RocketChip Debug Module
112 input logic dmi_rst_ni,
113 input logic dmi_req_valid_i,
114 output logic dmi_req_ready_o,
115 input dm::dmi_req_t dmi_req_i,
116
117 output logic dmi_resp_valid_o,
118 input logic dmi_resp_ready_i,
119 output dm::dmi_resp_t dmi_resp_o
120 );
121
122 // Slave response phase (rvalid and identifier)
123 logic slave_rvalid_q;
124 logic [IdWidth-1:0] slave_rid_q;
125
126 // dm_top instance
127 dm_top #(
128     .NrHarts (NrHarts),
129     .BusWidth (BusWidth),
130     .DmBaseAddress (DmBaseAddress),
131     .SelectableHarts (SelectableHarts),
132     .MaxRegisterAccessWidth (MaxRegisterAccessWidth)
133 ) i_dm_top (
134     .clk_i (clk_i),
135     .rst_ni (rst_ni),
136     .next_dm_addr_i ('0'), //
Note exposed at the top yet.
137     .testmode_i (testmode_i),
138     .ndmreset_o (ndmreset_o),
139     .ndmreset_ack_i (ndmreset_o), //
This is currently not exposed yet.
140     .dmactive_o (dmactive_o),
141     .debug_req_o (debug_req_o),
142     .unavailable_i (unavailable_i),
143     .hartinfo_i (hartinfo_i),
144
145     .slave_req_i (slave_req_i),
146     .slave_we_i (slave_we_i),
147     .slave_addr_i (slave_addr_i),
148     .slave_be_i (slave_be_i),
149     .slave_wdata_i (slave_wdata_i),
150     .slave_rdata_o (slave_rdata_o),
151
152     .master_req_o (master_req_o),
153     .master_add_o (master_addr_o), // Re-
named according to OBI spec
154     .master_we_o (master_we_o),
155     .master_wdata_o (master_wdata_o),
156     .master_be_o (master_be_o),
157     .master_gnt_i (master_gnt_i),
158     .master_rvalid_i (master_rvalid_i), // Re-
named according to OBI spec
159     .master_r_err_i (master_err_i),
160     .master_r_other_err_i (master_other_err_i), //
*other_err_i has priority over *err_i
161     .master_rdata_i (master_rdata_i), // Re-
named according to OBI spec
162
163     .dmi_rst_ni (dmi_rst_ni),
164     .dmi_req_valid_i (dmi_req_valid_i),
165     .dmi_req_ready_o (dmi_req_ready_o),
166     .dmi_req_i (dmi_req_i),
167
168     .dmi_resp_valid_o (dmi_resp_valid_o),
169     .dmi_resp_ready_i (dmi_resp_ready_i),
170     .dmi_resp_o (dmi_resp_o)
171 );
172
173 // Extension to wrap dm_top as an OBI-compliant module
174 //
175 // dm_top has an implied rvalid pulse the cycle after its
granted request.
176
177 // Registers
178 always_ff @(posedge clk_i or negedge rst_ni) begin :
obi_regs

```

```

179 if (!rst_ni) begin
180     slave_rvalid_q <= 1'b0;
181     slave_rid_q <= 'b0;
182 end else begin
183     if (slave_req_i && slave_gnt_o) begin // 1 cycle pulse
on rvalid for every granted request
184         slave_rvalid_q <= 1'b1;
185         slave_rid_q <= slave_aid_i; // Mirror aid to
rid
186     end else begin
187         slave_rvalid_q <= 1'b0; // rid is don't
care if rvalid = 0
188     end
189 end
190 end
191
192 assign slave_gnt_o = 1'b1; // Always recept-
ive to request (slave_req_i)
193 assign slave_rvalid_o = slave_rvalid_q;
194 assign slave_rid_o = slave_rid_q;
195
196 endmodule : dm_obi_top

```

4. pulp-platform_riscv-dbg/src/dm_pkg.sv (451 lines)

```

1 /* Copyright 2018 ETH Zurich and University of Bologna.
2 * Copyright and related rights are licensed under the Solder-
pad Hardware
3
4 * License, Version 0.51 (the "License"); you may not use this
file except in
5 * compliance with the License. You may obtain a copy of the
License at
6 * http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 * or agreed to in writing, software, hardware and materials
distributed under
8 * this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 * CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 * specific language governing permissions and limitations un-
der the License.
11
12 * File: dm_pkg.sv
13 * Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
14 * Date: 30.6.2018
15
16 * Description: Debug-module package, contains common system
definitions.
17 */
18
19 package dm;
20 localparam logic [3:0] DbgVersion013 = 4'h2;
21 // size of program buffer in junks of 32-bit words
22 localparam logic [4:0] ProgBufSize = 5'h8;
23
24 // amount of data count registers implemented
25 localparam logic [3:0] DataCount = 4'h2;
26
27 // address to which a hart should jump when it was requested to
halt
28 localparam logic [63:0] HaltAddress = 64'h800;
29 localparam logic [63:0] ResumeAddress = HaltAddress + 8;
30 localparam logic [63:0] ExceptionAddress = HaltAddress + 16;
31
32 // address where data0-15 is shadowed or if shadowed in a
CSR
33 // address of the first CSR used for shadowing the data
34 localparam logic [11:0] DataAddr = 12'h380; // we are
aligned with Rocket here
35
36 // debug registers
37 typedef enum logic [7:0] {
38     Data0 = 8'h04,
39     Data1 = 8'h05,
40     Data2 = 8'h06,
41     Data3 = 8'h07,
42     Data4 = 8'h08,
43     Data5 = 8'h09,
44     Data6 = 8'h0A,
45     Data7 = 8'h0B,
46     Data8 = 8'h0C,
47     Data9 = 8'h0D,
48     Data10 = 8'h0E,
49     Data11 = 8'h0F,
50     DMControl = 8'h10,
51     DMStatus = 8'h11, // r/o
52     Hartinfo = 8'h12,
53     HaltSum1 = 8'h13,
54     HAWindowSel = 8'h14,
55     HAWindow = 8'h15,
56     AbstractCS = 8'h16,
57     Command = 8'h17,
58     AbstractAuto = 8'h18,
59     DevTreeAddr0 = 8'h19,
60     DevTreeAddr1 = 8'h1A,
61     DevTreeAddr2 = 8'h1B,

```



```

62  DevTreeAddr3 = 8'h1C,
63  NextDM       = 8'h1D,
64  ProgBuf0     = 8'h20,
65  ProgBuf1     = 8'h21,
66  ProgBuf2     = 8'h22,
67  ProgBuf3     = 8'h23,
68  ProgBuf4     = 8'h24,
69  ProgBuf5     = 8'h25,
70  ProgBuf6     = 8'h26,
71  ProgBuf7     = 8'h27,
72  ProgBuf8     = 8'h28,
73  ProgBuf9     = 8'h29,
74  ProgBuf10    = 8'h2A,
75  ProgBuf11    = 8'h2B,
76  ProgBuf12    = 8'h2C,
77  ProgBuf13    = 8'h2D,
78  ProgBuf14    = 8'h2E,
79  ProgBuf15    = 8'h2F,
80  AuthData     = 8'h30,
81  HaltSum2     = 8'h34,
82  HaltSum3     = 8'h35,
83  SBAddress3   = 8'h37,
84  SBCS         = 8'h38,
85  SBAddress0   = 8'h39,
86  SBAddress1   = 8'h3A,
87  SBAddress2   = 8'h3B,
88  SBData0      = 8'h3C,
89  SBData1      = 8'h3D,
90  SBData2      = 8'h3E,
91  SBData3      = 8'h3F,
92  HaltSum0     = 8'h40
93  } dm_csr_e;
94
95  // debug causes
96  localparam logic [2:0] CauseBreakpoint = 3'h1;
97  localparam logic [2:0] CauseTrigger    = 3'h2;
98  localparam logic [2:0] CauseRequest    = 3'h3;
99  localparam logic [2:0] CauseSingleStep = 3'h4;
100
101  typedef struct packed {
102      logic [31:23] zero1;
103      logic         impebreak;
104      logic [21:20] zero0;
105      logic         allhavereset;
106      logic         anyhavereset;
107      logic         allresumeack;
108      logic         anyresumeack;
109      logic         allnonexistent;
110      logic         anynonexistent;
111      logic         allunavail;
112      logic         anyunavail;
113      logic         allrunning;
114      logic         anyrunning;
115      logic         allhalted;
116      logic         anyhalted;
117      logic         authenticated;
118      logic         authbusy;
119      logic         hasresethaltreq;
120      logic         devtreevalid;
121      logic [3:0]    version;
122  } dmstatus_t;
123
124  typedef struct packed {
125      logic         haltreq;
126      logic         resumereq;
127      logic         hartreset;
128      logic         ackhavereset;
129      logic         zero1;
130      logic         hsel;
131      logic [25:16] hartsello;
132      logic [15:6]  hartselhi;
133      logic [5:4]   zero0;
134      logic         setresethaltreq;
135      logic         clrresethaltreq;
136      logic         ndmreset;
137      logic         dmactive;
138  } dmcontrol_t;
139
140  typedef struct packed {
141      logic [31:24] zero1;
142      logic [23:20] nscratch;
143      logic [19:17] zero0;
144      logic         dataaccess;
145      logic [15:12] datasize;
146      logic [11:0]  dataaddr;
147  } hartinfo_t;
148
149  typedef enum logic [2:0] {
150      CmdErrNone, CmdErrBusy, CmdErrNotSupported,
151      CmdErrorException, CmdErrorHaltResume,
152      CmdErrorBus, CmdErrorOther = 7
153  } cmderr_e;
154
155  typedef struct packed {
156      logic [31:29] zero3;
157      logic [28:24] progbufsize;
158      logic [23:13] zero2;
159      logic         busy;
160      logic         zero1;
161      cmderr_e      cmderr;
162      logic [7:4]   zero0;
163
164      logic [3:0]    datacount;
165  } abstractcs_t;
166
167  typedef enum logic [7:0] {
168      AccessRegister = 8'h0,
169      QuickAccess    = 8'h1,
170      AccessMemory   = 8'h2
171  } cmd_e;
172
173  typedef struct packed {
174      cmd_e      cmdtype;
175      logic [23:0] control;
176  } command_t;
177
178  typedef struct packed {
179      logic [31:16] autoexecprogbuf;
180      logic [15:12] zero0;
181      logic [11:0]  autoexecdata;
182  } abstractauto_t;
183
184  typedef struct packed {
185      logic         zero1;
186      logic [22:20] aarsize;
187      logic         aarpostincrement;
188      logic         postexec;
189      logic         transfer;
190      logic         write;
191      logic [15:0]   regno;
192  } ac_ar_cmd_t;
193
194  // DTM
195  typedef enum logic [1:0] {
196      DTM_NOP = 2'h0,
197      DTM_READ = 2'h1,
198      DTM_WRITE = 2'h2
199  } dtm_op_e;
200
201  typedef enum logic [1:0] {
202      DTM_SUCCESS = 2'h0,
203      DTM_ERR = 2'h2,
204      DTM_BUSY = 2'h3
205  } dtm_op_status_e;
206
207  typedef struct packed {
208      logic [31:29] sbversion;
209      logic [28:23] zero0;
210      logic         sbbusyerror;
211      logic         sbbusy;
212      logic         sbreadonaddr;
213      logic [19:17] sbaccess;
214      logic         sbautoincrement;
215      logic         sbreadondata;
216      logic [14:12] sberror;
217      logic [11:5]  sbasize;
218      logic         sbaccess128;
219      logic         sbaccess64;
220      logic         sbaccess32;
221      logic         sbaccess16;
222      logic         sbaccess8;
223  } sbcs_t;
224
225  typedef struct packed {
226      logic [6:0] addr;
227      dtm_op_e    op;
228      logic [31:0] data;
229  } dmi_req_t;
230
231  typedef struct packed {
232      logic [31:0] data;
233      logic [1:0] resp;
234  } dmi_resp_t;
235
236  typedef struct packed {
237      logic [31:18] zero1;
238      logic         dmihardreset;
239      logic         dmireset;
240      logic         zero0;
241      logic [14:12] idle;
242      logic [11:10] dmistat;
243      logic [9:4]   abits;
244      logic [3:0]   version;
245  } dtmcs_t;
246
247  // privilege levels
248  typedef enum logic [1:0] {
249      PRIV_LVL_M = 2'b11,
250      PRIV_LVL_S = 2'b01,
251      PRIV_LVL_U = 2'b00
252  } priv_lvl_t;
253
254  // debugregs in core
255  typedef struct packed {
256      logic [31:28] xdebugver;
257      logic [27:16] zero2;
258      logic         ebreakm;
259      logic         zero1;
260      logic         ebreaks;
261      logic         ebreaku;
262      logic         stepie;
263      logic         stopcount;
264      logic         stoptime;

```

```

264 logic [8:6] cause;
265 logic zero0;
266 logic mprven;
267 logic nmlp;
268 logic step;
269 priv_lvl_t prv;
270 } dcsr_t;
271
272 // CSRs
273 typedef enum logic [11:0] {
274 // Floating-Point CSRs
275 CSR_FFLAGS = 12'h001,
276 CSR_FRM = 12'h002,
277 CSR_FCSR = 12'h003,
278 CSR_FTRAN = 12'h000,
279 // Supervisor Mode CSRs
280 CSR_SSTATUS = 12'h100,
281 CSR_SIE = 12'h104,
282 CSR_STVEC = 12'h105,
283 CSR_SCOUNTEREN = 12'h106,
284 CSR_SSCRATCH = 12'h140,
285 CSR_SEPC = 12'h141,
286 CSR_SCAUSE = 12'h142,
287 CSR_STVAL = 12'h143,
288 CSR_SIP = 12'h144,
289 CSR_SATP = 12'h180,
290 // Machine Mode CSRs
291 CSR_MSTATUS = 12'h300,
292 CSR_MISA = 12'h301,
293 CSR_MEDELEG = 12'h302,
294 CSR_MIDELEG = 12'h303,
295 CSR_MIE = 12'h304,
296 CSR_MTVEC = 12'h305,
297 CSR_MCOUNTEREN = 12'h306,
298 CSR_MSCRATCH = 12'h340,
299 CSR_MEPC = 12'h341,
300 CSR_MCAUSE = 12'h342,
301 CSR_MTVAL = 12'h343,
302 CSR_MIP = 12'h344,
303 CSR_PMPCFG0 = 12'h3A0,
304 CSR_PMPADDR0 = 12'h3B0,
305 CSR_MVENDORID = 12'hF11,
306 CSR_MARCHID = 12'hF12,
307 CSR_MIMPID = 12'hF13,
308 CSR_MHARTID = 12'hF14,
309 CSR_MCYCLE = 12'hB00,
310 CSR_MINSTRET = 12'hB02,
311 CSR_DCACHE = 12'h701,
312 CSR_ICACHE = 12'h700,
313
314 CSR_TSELECT = 12'h7A0,
315 CSR_TDATA1 = 12'h7A1,
316 CSR_TDATA2 = 12'h7A2,
317 CSR_TDATA3 = 12'h7A3,
318 CSR_TINFO = 12'h7A4,
319
320 // Debug CSR
321 CSR_DCSR = 12'h7B0,
322 CSR_DPC = 12'h7B1,
323 CSR_DSCRATCH0 = 12'h7B2, // optional
324 CSR_DSCRATCH1 = 12'h7B3, // optional
325
326 // Counters and Timers
327 CSR_CYCLE = 12'hC00,
328 CSR_TIME = 12'hC01,
329 CSR_INSTRET = 12'hC02
330 } csr_reg_t;
331
332 // SBA state
333 typedef enum logic [2:0] {
334 Idle,
335 Read,
336 Write,
337 WaitRead,
338 WaitWrite
339 } sba_state_e;
340
341 // Instruction Generation Helpers
342 function automatic logic [31:0] jal (logic [4:0] rd,
343                                     logic [20:0] imm);
344 // OpCode Jal
345 return {imm[20], imm[10:1], imm[11], imm[19:12], rd,
7'h6f};
346 endfunction
347
348 function automatic logic [31:0] jalr (logic [4:0] rd,
349                                     logic [4:0] rs1,
350                                     logic [11:0] offset);
351 // OpCode Jalr
352 return {offset[11:0], rs1, 3'b0, rd, 7'h67};
353 endfunction
354
355 function automatic logic [31:0] andi (logic [4:0] rd,
356                                     logic [4:0] rs1,
357                                     logic [11:0] imm);
358 // OpCode andi
359 return {imm[11:0], rs1, 3'h7, rd, 7'h13};
360 endfunction
361
362 function automatic logic [31:0] slli (logic [4:0] rd,
363                                     logic [4:0] rs1,

```

```

364                                     logic [5:0] shamt);
365 // OpCode slli
366 return {6'b0, shamt[5:0], rs1, 3'h1, rd, 7'h13};
367 endfunction
368
369 function automatic logic [31:0] srli (logic [4:0] rd,
370                                     logic [4:0] rs1,
371                                     logic [5:0] shamt);
372 // OpCode srli
373 return {6'b0, shamt[5:0], rs1, 3'h5, rd, 7'h13};
374 endfunction
375
376 function automatic logic [31:0] load (logic [2:0] size,
377                                     logic [4:0] dest,
378                                     logic [4:0] base,
379                                     logic [11:0] offset);
380 // OpCode Load
381 return {offset[11:0], base, size, dest, 7'h03};
382 endfunction
383
384 function automatic logic [31:0] auipc (logic [4:0] rd,
385                                     logic [20:0] imm);
386 // OpCode Auipc
387 return {imm[20], imm[10:1], imm[11], imm[19:12], rd,
7'h17};
388 endfunction
389
390 function automatic logic [31:0] store (logic [2:0] size,
391                                     logic [4:0] src,
392                                     logic [4:0] base,
393                                     logic [11:0] offset);
394 // OpCode Store
395 return {offset[11:5], src, base, size, offset[4:0],
7'h23};
396 endfunction
397
398 function automatic logic [31:0] float_load (logic [2:0]
size,
399                                     logic [4:0]
dest,
400                                     logic [4:0]
base,
401                                     logic [11:0]
offset);
402 // OpCode Load
403 return {offset[11:0], base, size, dest, 7'b00_001_11};
404 endfunction
405
406 function automatic logic [31:0] float_store (logic [2:0]
size,
407                                     logic [4:0]
src,
408                                     logic [4:0]
base,
409                                     logic [11:0]
offset);
410 // OpCode Store
411 return {offset[11:5], src, base, size, offset[4:0],
7'b01_001_11};
412 endfunction
413
414 function automatic logic [31:0] csrw (csr_reg_t csr,
415                                     logic [4:0] rs1);
416 // CSRRW, rd, OpCode System
417 return {csr, rs1, 3'h1, 5'h0, 7'h73};
418 endfunction
419
420 function automatic logic [31:0] csrr (csr_reg_t csr,
421                                     logic [4:0] dest);
422 // rs1, CSRRS, rd, OpCode System
423 return {csr, 5'h0, 3'h2, dest, 7'h73};
424 endfunction
425
426 function automatic logic [31:0] branch (logic [4:0] src2,
427                                     logic [4:0] src1,
428                                     logic [2:0] funct3,
429                                     logic [11:0] offset);
430 // OpCode Branch
431 return {offset[11], offset[9:4], src2, src1, funct3,
offset[3:0], offset[10], 7'b11_000_11};
432 endfunction
433
434 function automatic logic [31:0] ebreak ();
435 return 32'h00100073;
436 endfunction
437
438 function automatic logic [31:0] wfi ();
439 return 32'h10500073;
440 endfunction
441
442 function automatic logic [31:0] nop ();
443 return 32'h00000013;
444 endfunction
445
446 function automatic logic [31:0] illegal ();
447 return 32'h00000000;
448 endfunction
449
450 endpackage : dm

```


5. pulp-platform_riscv-dbg/src/dm_sba.sv (211 lines)

```

1 /* Copyright 2018 ETH Zurich and University of Bologna.
2 * Copyright and related rights are licensed under the Solder-
3 * pad Hardware
4 * License, Version 0.51 (the "License"); you may not use this
5 * file except in
6 * compliance with the License. You may obtain a copy of the
7 * License at
8 * http://solderpad.org/licenses/SHL-0.51. Unless required by
9 * applicable law
10 * or agreed to in writing, software, hardware and materials
11 * distributed under
12 * this License is distributed on an "AS IS" BASIS, WITHOUT
13 * WARRANTIES OR
14 * CONDITIONS OF ANY KIND, either express or implied. See the
15 * License for the
16 * specific language governing permissions and limitations un-
17 * der the
18 * License.
19 * File: dm_sba.sv
20 * Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
21 * Date: 1.8.2018
22 *
23 * Description: System Bus Access Module
24 *
25 */
26 module dm_sba #(
27     parameter int unsigned BusWidth = 32,
28     parameter bit ReadByteEnable = 1
29 ) (
30     input logic clk_i, // Clock
31     input logic rst_ni,
32     input logic dmactive_i, // synchronous
33     reset active low
34
35     output logic master_req_o,
36     output logic [BusWidth-1:0] master_add_o,
37     output logic master_we_o,
38     output logic [BusWidth-1:0] master_wdata_o,
39     output logic [BusWidth/8-1:0] master_be_o,
40     input logic master_gnt_i,
41     input logic master_r_valid_i,
42     input logic master_r_err_i,
43     input logic master_r_other_err_i, //
44     *other_err_i has priority over *err_i
45     input logic [BusWidth-1:0] master_r_rdata_i,
46
47     input logic [BusWidth-1:0] sbaddress_i,
48     input logic sbaddress_write_valid_i,
49     // control signals in
50     input logic sbreadonaddr_i,
51     output logic [BusWidth-1:0] sbaddress_o,
52     input logic sbautoincrement_i,
53     input logic [2:0] sbaccess_i,
54     // data in
55     input logic sbreadondata_i,
56     input logic [BusWidth-1:0] sbdata_i,
57     input logic sbdata_read_valid_i,
58     input logic sbdata_write_valid_i,
59     // read data out
60     output logic [BusWidth-1:0] sbdata_o,
61     output logic sbdata_valid_o,
62     // control signals
63     output logic sbbusy_o,
64
65     output logic sberror_valid_o, // bus error
66     occurred
67     output logic [2:0] sberror_o // bus error oc-
68     curred
69 );
70
71 localparam int BeIdxWidth = $clog2(BusWidth/8);
72 dm::sba_state_e state_d, state_q;
73
74 logic [BusWidth-1:0] address;
75 logic req;
76 logic gnt;
77 logic we;
78 logic [BusWidth/8-1:0] be;
79 logic [BusWidth/8-1:0] be_mask;
80 logic [BeIdxWidth-1:0] be_idx;
81
82 assign sbbusy_o = logic'(state_q != dm::Idle);
83
84 always_comb begin : p_be_mask
85     be_mask = '0;
86     // generate byte enable mask
87     unique case (sbaccess_i)
88         3'b000: begin
89             be_mask[be_idx] = '1;
90         end
91         3'b001: begin
92             be_mask[int'({be_idx[$high(be_idx):1], 1'b0}) +: 2] =
93             '1;
94         end
95         3'b010: begin
96             if (BusWidth == 32'd64)

```

```

be_mask[int'({be_idx[$high(be_idx)], 2'h0}) +: 4] = '1;
97         else
98             be_mask = '1;
99         end
100     endcase
101 end
102
103 logic [BusWidth-1:0] sbaccess_mask;
104 assign sbaccess_mask = {BusWidth{1'b1}} << sbaccess_i;
105
106 logic addr_incr_en;
107 logic [BusWidth-1:0] addr_incr;
108 assign addr_incr = (addr_incr_en) ? (BusWidth'(1'b1) <<
109 sbaccess_i) : '0;
110 assign sbaddress_o = sbaddress_i + addr_incr;
111
112 always_comb begin : p_fsm
113     req = 'b0;
114     address = sbaddress_i;
115     we = 'b0;
116     be = '0;
117     be_idx = sbaddress_i[BeIdxWidth-1:0];
118
119     sberror_o = '0;
120     sberror_valid_o = 'b0;
121
122     addr_incr_en = 'b0;
123
124     state_d = state_q;
125
126     unique case (state_q)
127         dm::Idle: begin
128             // debugger requested a read
129             if (sbaddress_write_valid_i && sbreadonaddr_i)
130                 state_d = dm::Read;
131             // debugger requested a write
132             if (sbdata_write_valid_i) state_d = dm::Write;
133             // perform another read
134             if (sbdata_read_valid_i && sbreadondata_i) state_d =
135             dm::Read;
136         end
137         dm::Read: begin
138             req = 'b1;
139             if (ReadByteEnable) be = be_mask;
140             if (gnt) state_d = dm::WaitRead;
141         end
142         dm::Write: begin
143             req = 'b1;
144             we = 'b1;
145             be = be_mask;
146             if (gnt) state_d = dm::WaitWrite;
147         end
148         dm::WaitRead: begin
149             if (sbdata_valid_o) begin
150                 state_d = dm::Idle;
151                 // auto-increment address
152                 addr_incr_en = sbautoincrement_i;
153                 // check whether an "other" error has been en-
154                 countered.
155                 if (master_r_other_err_i) begin
156                     sberror_valid_o = 'b1;
157                     sberror_o = 3'd7;
158                     // check whether there was a bus error (== bad ad-
159                     dress).
160                     end else if (master_r_err_i) begin
161                         sberror_valid_o = 'b1;
162                         sberror_o = 3'd2;
163                     end
164                 end
165             end
166             state_d = dm::WaitWrite;
167             if (sbdata_valid_o) begin
168                 state_d = dm::Idle;
169                 // auto-increment address
170                 addr_incr_en = sbautoincrement_i;
171                 // check whether an "other" error has been en-
172                 countered.
173                 if (master_r_other_err_i) begin
174                     sberror_valid_o = 'b1;
175                     sberror_o = 3'd7;
176                     // check whether there was a bus error (== bad ad-
177                     dress).
178                     end else if (master_r_err_i) begin
179                         sberror_valid_o = 'b1;
180                         sberror_o = 3'd2;
181                     end
182                 end
183             end
184         end
185         default: state_d = dm::Idle; // catch parasitic state
186     endcase
187
188     // handle error case
189     if (32'(sbaccess_i) > BeIdxWidth && state_q != dm::Idle)
190         begin

```

```

176     req          = 1'b0;
177     state_d      = dm::Idle;
178     sberror_valid_o = 1'b1;
179     sberror_o    = 3'd4; // unsupported size was requested
180 end
181
182 //if sbaccess_i lsbs of address are not 0 - report mis-
alignment error
183 if ((sbaddress_i & ~sbaccess_mask) && state_q !=
dm::Idle) begin
184     req          = 1'b0;
185     state_d      = dm::Idle;
186     sberror_valid_o = 1'b1;
187     sberror_o    = 3'd3; // alignment error
188 end
189 // further error handling should go here ...
190 end
191
192 always_ff @(posedge clk_i or negedge rst_ni) begin : p_regs
193     if (!rst_ni) begin
194         state_q <= dm::Idle;
195     end else begin
196         state_q <= state_d;
197     end
198 end
199
200 logic [BeIdxWidth-1:0] be_idx_masked;
201 assign be_idx_masked = be_idx & BeIdx-
Width'(sbaccess_mask);
202 assign master_req_o   = req;
203 assign master_add_o   = address[BusWidth-1:0];
204 assign master_we_o    = we;
205 assign master_wdata_o = sbdata_i[BusWidth-1:0] << (8 *
be_idx_masked);
206 assign master_be_o    = be[BusWidth/8-1:0];
207 assign gnt            = master_gnt_i;
208 assign sbdata_valid_o = master_r_valid_i;
209 assign sbdata_o       = master_r_rdata_i[BusWidth-1:0] >>
(8 * be_idx_masked);
210
211 endmodule : dm_sba

```

6. pulp-platform_riscv-dbg/src/dm_top.sv (243 lines)

```

1 /* Copyright 2018 ETH Zurich and University of Bologna.
2 * Copyright and related rights are licensed under the Solder-
pad Hardware
3 * License, Version 0.51 (the "License"); you may not use this
file except in
4 * compliance with the License. You may obtain a copy of the
License at
5 * http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 * or agreed to in writing, software, hardware and materials
distributed under
7 * this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 * CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 * specific language governing permissions and limitations un-
der the License.
10 *
11 * File: dm_top.sv
12 * Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
13 * Date: 30.6.2018
14 *
15 * Description: Top-level of debug module (DM). This is an AXI-
Slave.
16 *
17 * DTM protocol is equal to SiFives debug protocol to
leverage
18 * SW infrastructure re-use. As of version 0.13
19 */
20 module dm_top #(
21     parameter int unsigned NrHarts = 1,
22     parameter int unsigned BusWidth = 32,
23     parameter int unsigned DmBaseAddress = 'h1000, //
default to non-zero page
24     // Bitmask to select physically available harts for systems
25     // that don't use hart numbers in a contiguous fashion.
26     parameter logic [NrHarts-1:0] SelectableHarts =
{NrHarts{1'b1}},
27     // toggle new behavior to drive master_be_o during a read
28     parameter bit ReadByteEnable = 1,
29     // Maximum width supported by Access Register commands; must be
32 or 64.
30     parameter int unsigned MaxRegisterAccessWidth =
BusWidth
31 ) (
32     input logic clk_i, // clock
33     // asynchronous reset active low, connect PoR here, not the
system reset
34     input logic rst_ni,
35     // Subsequent debug modules can be chained by setting the
nextdm
36     // register value to the offset of
37     // the next debug module. The RISC-V debug spec mandates
that the first debug module located at

```

```

37 // 0x0, and that the last debug module in the chain sets the
nextdm
38 // register is a word address and not a byte address. This
value is passed in as a static signal
39 // so that it becomes possible to assign this value with
chiplet tie-offs or straps, if needed.
40 input logic [31:0] next_dm_addr_i,
41 input logic testmode_i,
42 output logic ndmreset_o, // non-debug mod-
ule reset
43 input logic ndmreset_ack_i, // non-debug
module reset acknowledgement pulse
44 output logic dmactive_o, // debug module
is active
45 output logic [NrHarts-1:0] debug_req_o, // async debug
request
46 // communicate whether the hart is unavailable (e.g.: power
down)
47 input logic [NrHarts-1:0] unavailable_i,
48 input dm::hartinfo_t [NrHarts-1:0] hartinfo_i,
49
50 input logic slave_req_i,
51 input logic slave_we_i,
52 input logic [BusWidth-1:0] slave_addr_i,
53 input logic [BusWidth/8-1:0] slave_be_i,
54 input logic [BusWidth-1:0] slave_wdata_i,
55 output logic [BusWidth-1:0] slave_rdata_o,
56
57 output logic master_req_o,
58 output logic [BusWidth-1:0] master_add_o,
59 output logic master_we_o,
60 output logic [BusWidth-1:0] master_wdata_o,
61 output logic [BusWidth/8-1:0] master_be_o,
62 input logic master_gnt_i,
63 input logic master_r_valid_i,
64 input logic master_r_err_i,
65 input logic master_r_other_err_i, // *oth-
er_err_i has priority over *err_i
66 input logic [BusWidth-1:0] master_r_rdata_i,
67
68 // Connection to DTM - compatible to RocketChip Debug Module
69 input logic dmi_rst_ni, // Synchronous
clear request from
70 // the DTM to
clear the DMI response
71 // FIFO.
72 input logic dmi_req_valid_i,
73 output logic dmi_req_ready_o,
74 input dm::dmi_req_t dmi_req_i,
75
76 output logic dmi_resp_valid_o,
77 input logic dmi_resp_ready_i,
78 output dm::dmi_resp_t dmi_resp_o
79 );
80
81 // Debug CSRs
82 logic [NrHarts-1:0] halted;
83 // logic [NrHarts-1:0] running;
84 logic [NrHarts-1:0] resumeack;
85 logic [NrHarts-1:0] haltreq;
86 logic [NrHarts-1:0] resumereq;
87 logic [NrHarts-1:0] clear_resumeack;
88 logic cmd_valid;
89 dm::command_t cmd;
90
91 logic cmderror_valid;
92 dm::cmderr_e cmderror;
93 logic cmdbusy;
94 logic [dm::ProgBufSize-1:0][31:0] progbuf;
95 logic [dm::DataCount-1:0][31:0] data_csrs_mem;
96 logic [dm::DataCount-1:0][31:0] data_mem_csrs;
97 logic data_valid;
98 logic ndmreset;
99 logic [19:0] hartsel;
100 // System Bus Access Module
101 logic [BusWidth-1:0] sbaddress_csrs_sba;
102 logic [BusWidth-1:0] sbaddress_sba_csrs;
103 logic sbaddress_write_valid;
104 logic sbreadonaddr;
105 logic sbautoincrement;
106 logic [2:0] sbaccess;
107 logic sbreadondata;
108 logic [BusWidth-1:0] sbdata_write;
109 logic sbdata_read_valid;
110 logic sbdata_write_valid;
111 logic [BusWidth-1:0] sbdata_read;
112 logic sbdata_valid;
113 logic sbbusy;
114 logic sberror_valid;
115 logic [2:0] sberror;
116
117 assign ndmreset_o = ndmreset;
118
119 dm_csrs #(
120     .NrHarts(NrHarts),
121     .BusWidth(BusWidth),
122     .SelectableHarts(SelectableHarts)
123 ) i_dm_csrs (
124     .clk_i,
125     .rst_ni,

```

```

126 .next_dm_addr_i,
127 .testmode_i,
128 .dmi_rst_ni,
129 .dmi_req_valid_i,
130 .dmi_req_ready_o,
131 .dmi_req_i,
132 .dmi_resp_valid_o,
133 .dmi_resp_ready_i,
134 .dmi_resp_o,
135 .ndmreset_o      ( ndmreset      ),
136 .ndmreset_ack_i  ( ndmreset_ack_i ),
137 .dmactive_o,
138 .hartsel_o       ( hartsel       ),
139 .hartinfo_i,
140 .halted_i        ( halted        ),
141 .unavailable_i,
142 .resumeack_i     ( resumeack     ),
143 .haltreq_o       ( haltreq       ),
144 .resumereq_o     ( resumereq     ),
145 .clear_resumeack_o ( clear_resumeack ),
146 .cmd_valid_o     ( cmd_valid     ),
147 .cmd_o           ( cmd           ),
148 .cmderror_valid_i ( cmderror_valid ),
149 .cmderror_i      ( cmderror      ),
150 .cmdbusy_i       ( cmdbusy       ),
151 .progbuf_o       ( progbuf       ),
152 .data_i          ( data_i        ),
153 .data_valid_i    ( data_valid    ),
154 .data_o          ( data_o        ),
155 .sbaddress_o     ( sbaddress_csrs_sba ),
156 .sbaddress_i     ( sbaddress_sba_csrs ),
157 .sbaddress_write_valid_o ( sbaddress_write_valid ),
158 .sbreadonaddr_o ( sbreadonaddr ),
159 .sbautoincrement_o ( sbautoincrement ),
160 .sbaccess_o      ( sbaccess      ),
161 .sbreadondata_o  ( sbreadondata ),
162 .sbdata_o        ( sbdata_write   ),
163 .sbdata_read_valid_o ( sbdata_read_valid ),
164 .sbdata_write_valid_o ( sbdata_write_valid ),
165 .sbdata_i        ( sbdata_read   ),
166 .sbdata_valid_i  ( sbdata_valid   ),
167 .sbbusy_i        ( sbbusy        ),
168 .sberror_valid_i ( sberror_valid  ),
169 .sberror_i       ( sberror       ),
170 );
171
172 dm_sba #(
173     .BusWidth(BusWidth),
174     .ReadByteEnable(ReadByteEnable)
175 ) i_dm_sba (
176     .clk_i,
177     .rst_ni,
178     .dmactive_i      ( dmactive_o      ),
179
180     .master_req_o,
181     .master_add_o,
182     .master_we_o,
183     .master_wdata_o,
184     .master_be_o,
185     .master_gnt_i,
186     .master_r_valid_i,
187     .master_r_err_i,
188     .master_r_other_err_i,
189     .master_r_rdata_i,
190
191     .sbaddress_i      ( sbaddress_csrs_sba ),
192     .sbaddress_o      ( sbaddress_sba_csrs ),
193     .sbaddress_write_valid_i ( sbaddress_write_valid ),
194     .sbreadonaddr_i   ( sbreadonaddr ),
195     .sbautoincrement_i ( sbautoincrement ),
196     .sbaccess_i       ( sbaccess ),
197     .sbreadondata_i   ( sbreadondata ),
198     .sbdata_i         ( sbdata_write ),
199     .sbdata_read_valid_i ( sbdata_read_valid ),
200     .sbdata_write_valid_i ( sbdata_write_valid ),
201     .sbdata_o         ( sbdata_read ),
202     .sbdata_valid_o   ( sbdata_valid ),
203     .sbbusy_o         ( sbbusy ),
204     .sberror_valid_o  ( sberror_valid ),
205     .sberror_o       ( sberror ),
206 );
207
208 dm_mem #(
209     .NrHarts      ( NrHarts      ),
210     .BusWidth     ( BusWidth     ),
211     .SelectableHarts ( SelectableHarts ),
212     .DmBaseAddress ( DmBaseAddress ),
213     .MaxRegisterAccessWidth ( MaxRegisterAccessWidth )
214 ) i_dm_mem (
215     .clk_i,
216     .rst_ni,
217     .dmactive_i      ( dmactive_o      ),
218     .debug_req_o,
219     .ndmreset_i      ( ndmreset      ),
220     .hartsel_i       ( hartsel       ),
221     .haltreq_i       ( haltreq       ),
222     .resumereq_i     ( resumereq     ),
223     .clear_resumeack_i ( clear_resumeack ),
224     .halted_o        ( halted        ),
225     .resuming_o      ( resumeack     ),
226     .cmd_valid_i     ( cmd_valid     ),

```

```

227 .cmd_i      ( cmd      ),
228 .cmderror_valid_o ( cmderror_valid ),
229 .cmderror_o  ( cmderror ),
230 .cmdbusy_o   ( cmdbusy  ),
231 .progbuf_i   ( progbuf  ),
232 .data_i      ( data_csrs_mem ),
233 .data_o      ( data_mem_csrs ),
234 .data_valid_o ( data_valid ),
235 .req_i       ( slave_req_i ),
236 .we_i       ( slave_we_i ),
237 .addr_i     ( slave_addr_i ),
238 .wdata_i    ( slave_wdata_i ),
239 .be_i       ( slave_be_i ),
240 .rdata_o    ( slave_rdata_o ),
241 );
242
243 endmodule : dm_top

```

7. pulp-platform_riscv-dbg/src/dmi_bscane_tap.sv (80 lines)

```

1 // Copyright 2020 ETH Zurich and University of Bologna.
2 // Solderpad Hardware License, Version 0.51, see LICENSE for
details.
3 // SPDX-License-Identifier: SHL-0.51
4
5 // Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
6
7 /// Replacement for the full JTAG tap with `BSCANE2` Xilinx
elements which hook
8 /// into the FPGA native scan chain. Meant for FPGA boards
which do not expose a
9 /// usable pin-header or a separate, programmable FTDI chip.
10
11 /// They replace the functionality of `dmi_jtag_tap.sv`. The
file is
12 /// pin-compatible so that by selecting the appropriate file
for the target it
13 /// can be transparently managed without relying on tick
defines.
14 module dmi_jtag_tap #(
15     // Ignored, defined by the FPGA model.
16     parameter int unsigned IrLength = 5,
17     // JTAG IDCODE Value
18     parameter logic [31:0] IdcodeValue = 32'h00000001
19     // xxxx version
20     // xxxxxxxxxxxxxxxx part number
21     // xxxxxxxxxxxx manufacturer id
22     // 1 required by standard
23 ) (
24     /// Unused. Here to maintain pin compatibility with
`dmi_jtag_tap` so that it
25     /// can be used as a drop-in replacement.
26     input logic tck_i,
27     input logic tms_i,
28     input logic trst_ni,
29     input logic td_i,
30     output logic td_o,
31     output logic tdo_oe_o,
32     input logic testmode_i,
33
34     output logic tck_o,
35     output logic dmi_clear_o,
36     output logic update_o,
37     output logic capture_o,
38     output logic shift_o,
39     output logic tdi_o,
40     output logic dtmcs_select_o,
41     input logic dtmcs_tdo_i,
42     // we want to access DMI register
43     output logic dmi_select_o,
44     input logic dmi_tdo_i,
45 );
46
47 BSCANE2 #(
48     .JTAG_CHAIN (3)
49 ) i_tap_dtmcs (
50     .CAPTURE (capture_o),
51     .DRCK (),
52     .RESET (dmi_clear_o),
53     .RUNTEST (),
54     .SEL (dtmcs_select_o),
55     .SHIFT (shift_o),
56     .TCK (tck_o),
57     .TDI (tdi_o),
58     .TMS (),
59     .TDO (dtmcs_tdo_i),
60     .UPDATE (update_o)
61 );
62
63 /// DMI Register
64 BSCANE2 #(
65     .JTAG_CHAIN (4)
66 ) i_tap_dmi (
67     .CAPTURE (),
68     .DRCK (),
69     .RESET (),
70     .RUNTEST (),
71     .SEL (dmi_select_o),
72     .SHIFT (),
73     .TCK (),

```

```

74 .TDI (),
75 .TMS (),
76 .TDO (dmi_tdo_i),
77 .UPDATE (),
78 );
79
80 endmodule

```

8. pulp-platform_riscv-dbg/src/dmi_cdc.sv (114 lines)

```

1 /* Copyright 2018 ETH Zurich and University of Bologna.
2 * Copyright and related rights are licensed under the Solder-
3 * pad Hardware
4 * License, Version 0.51 (the "License"); you may not use this
5 * file except in
6 * compliance with the License. You may obtain a copy of the
7 * License at
8 * http://solderpad.org/licenses/SHL-0.51. Unless required by
9 * applicable law
10 * or agreed to in writing, software, hardware and materials
11 * distributed under
12 * this License is distributed on an "AS IS" BASIS, WITHOUT
13 * WARRANTIES OR
14 * CONDITIONS OF ANY KIND, either express or implied. See the
15 * License for the
16 * specific language governing permissions and limitations un-
17 * der the License.
18 *
19 * File: axi_riscv_debug_module.sv
20 * Author: Andreas Traber <atraber@iis.ee.ethz.ch>
21 * Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
22 * Description: Clock domain crossings for JTAG to DMI very
23 * heavily based
24
25 on previous work by Andreas Traber for the PULP
26 project.
27 This is mainly a wrapper around the existing
28 CDCs.
29 */
30 module dmi_cdc (
31 // JTAG side (master side)
32 input logic tck_i,
33 input logic trst_ni,
34 input dmi_req_t dmi_req_i,
35 output logic jtag_dmi_ready_o,
36 input logic jtag_dmi_valid_i,
37 input logic jtag_dmi_cdc_clear_i, // Synchron-
38 ous clear signal.
39 // Triggers
40 reset sequencing
41 // accross
42 CDC
43
44 output dmi::dmi_resp_t jtag_dmi_resp_o,
45 output logic jtag_dmi_valid_o,
46 input logic jtag_dmi_ready_i,
47
48 // core side (slave side)
49 input logic clk_i,
50 input logic rst_ni,
51
52 output logic core_dmi_rst_no,
53 output dmi::dmi_req_t core_dmi_req_o,
54 output logic core_dmi_valid_o,
55 input logic core_dmi_ready_i,
56
57 input dmi::dmi_resp_t core_dmi_resp_i,
58 output logic core_dmi_ready_o,
59 input logic core_dmi_valid_i,
60 );
61
62 logic core_clear_pending;
63
64 cdc_2phase_clearable #(T(dmi_req_t)) i_cdc_req (
65 .src_rst_ni ( trst_ni ),
66 .src_clear_i ( jtag_dmi_cdc_clear_i ),
67 .src_clk_i ( tck_i ), // Not used
68 .src_data_i ( jtag_dmi_req_i ),
69 .src_valid_i ( jtag_dmi_valid_i ),
70 .src_ready_o ( jtag_dmi_ready_o ),
71
72 .dst_rst_ni ( rst_ni ),
73 .dst_clear_i ( 1'b0 ), // No functional
74 reset from core side
75 // used (only
76 async).
77 .dst_clear_pending_o( core_clear_pending ), // use the
78 clear pending signal
79 // to syn-
80 chronously clear the
81 // response
82 FIFO in the dm_top
83 // csrs
84 .dst_clk_i ( clk_i ),
85 .dst_data_o ( core_dmi_req_o ),
86 .dst_valid_o ( core_dmi_valid_o ),
87 .dst_ready_i ( core_dmi_ready_i )
88 );

```

```

71
72 cdc_2phase_clearable #(T(dmi_resp_t)) i_cdc_resp (
73 .src_rst_ni ( rst_ni ),
74 .src_clear_i ( 1'b0 ), // No functional
75 reset from core side
76 // used (only
77 async).
78 .src_clear_pending_o(), // Not used
79 .src_clk_i ( clk_i ),
80 .src_data_i ( core_dmi_resp_i ),
81 .src_valid_i ( core_dmi_valid_i ),
82 .src_ready_o ( core_dmi_ready_o ),
83
84 .dst_rst_ni ( trst_ni ),
85 .dst_clear_i ( jtag_dmi_cdc_clear_i ),
86 .dst_clear_pending_o(), //Not used
87 .dst_clk_i ( tck_i ),
88 .dst_data_o ( jtag_dmi_resp_o ),
89 .dst_valid_o ( jtag_dmi_valid_o ),
90 .dst_ready_i ( jtag_dmi_ready_i )
91 );
92
93 // We need to flush the DMI response FIFO in DM top using
94 the core clock
95 // synchronous clear signal core_dmi_rst_no. We repurpose
96 the clear
97
98 // pending signal in the core clock domain by generating a 1
99 cycle pulse from
100 // it.
101 logic core_clear_pending_q;
102 logic core_dmi_rst_nq;
103 logic clear_pending_rise_edge_detect;
104
105 assign clear_pending_rise_edge_detect = !
106 core_clear_pending_q && core_clear_pending;
107
108 always_ff @(posedge clk_i, negedge rst_ni) begin
109 if (!rst_ni) begin
110 core_dmi_rst_nq <= 1'b1;
111 core_clear_pending_q <= 1'b0;
112 end else begin
113 core_dmi_rst_nq <=
114 ~clear_pending_rise_edge_detect; // active-low!
115 core_clear_pending_q <= core_clear_pending;
116 end
117
118 assign core_dmi_rst_no = core_dmi_rst_nq;
119
120 endmodule : dmi_cdc

```

9. pulp-platform_riscv-dbg/src/dmi_intf.sv (59 lines)

```

1 // Copyright 2021 ETH Zurich and University of Bologna.
2 // Solderpad Hardware License, Version 0.51, see LICENSE for
3 details.
4 // SPDX-License-Identifier: SHL-0.51
5
6 // Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
7
8 // The DV interface additionally carries a clock signal.
9 interface DMI_BUS_DV #(
10 /// The width of the address.
11 parameter int ADDR_WIDTH = -1
12 ) (
13 input logic clk_i
14 );
15
16 import dm::*;
17
18 typedef logic [ADDR_WIDTH-1:0] addr_t;
19 typedef logic [31:0] data_t;
20 /// The request channel (Q).
21 addr_t q_addr;
22 dtm_op_e q_op;
23 data_t q_data;
24 logic q_valid;
25 logic q_ready;
26
27 /// The response channel (P).
28 data_t p_data;
29 logic [1:0] p_resp;
30 logic p_valid;
31 logic p_ready;
32
33 modport in (
34 input q_addr, q_op, q_data, q_valid, p_ready,
35 output q_ready, p_data, p_resp, p_valid
36 );
37 modport out (
38 output q_addr, q_op, q_data, q_valid, p_ready,
39 input q_ready, p_data, p_resp, p_valid
40 );
41 modport monitor (
42 input q_addr, q_op, q_data, q_valid, p_ready,
43 q_ready, p_data, p_resp, p_valid
44 );

```

```

45
46 // pragma translate_off
47 `ifndef VERILATOR
48 assert property (@(posedge clk_i) (q_valid && !q_ready =>
$stable(q_addr)));
49 assert property (@(posedge clk_i) (q_valid && !q_ready =>
$stable(q_op)));
50 assert property (@(posedge clk_i) (q_valid && !q_ready =>
$stable(q_data)));
51 assert property (@(posedge clk_i) (q_valid && !q_ready =>
q_valid));
52
53 assert property (@(posedge clk_i) (p_valid && !p_ready =>
$stable(p_data)));
54 assert property (@(posedge clk_i) (p_valid && !p_ready =>
$stable(p_resp)));
55 assert property (@(posedge clk_i) (p_valid && !p_ready =>
p_valid));
56 endif
57 // pragma translate_on
58
59 endinterface

```

10. pulp-platform_riscv-dbg/src/dmi_jtag.sv (353 lines)

```

1 /* Copyright 2018 ETH Zurich and University of Bologna.
2 * Copyright and related rights are licensed under the Solder-
pad Hardware
3 * License, Version 0.51 (the "License"); you may not use this
file except in
4 * compliance with the License. You may obtain a copy of the
License at
5 * http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 * or agreed to in writing, software, hardware and materials
distributed under
7 * this license is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 * CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 * specific language governing permissions and limitations un-
der the License.
10 *
11 * File: axi_riscv_debug_module.sv
12 * Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
13 * Date: 19.7.2018
14 *
15 * Description: JTAG DMI (debug module interface)
16 */
17
18 module dmi_jtag #(
19     parameter logic [31:0] IdcodeValue = 32'h000000B3
20 ) (
21     input logic clk_i, // DMI Clock
22     input logic rst_ni, // Asynchronous reset act-
ive low
23     input logic testmode_i,
24     // active-low glitch free reset signal. Is asserted for one
dmi clock cycle
25
26     // (clk_i) whenever the dmi_jtag is reset (POR or functional
reset).
27     output logic dmi_rst_no,
28     output dm::dmi_req_t dmi_req_o,
29     output logic dmi_req_valid_o,
30     input logic dmi_req_ready_i,
31
32     input dm::dmi_resp_t dmi_resp_i,
33     output logic dmi_resp_ready_o,
34     input logic dmi_resp_valid_i,
35
36     input logic tck_i, // JTAG test clock pad
37     input logic tms_i, // JTAG test mode select pad
38     input logic trst_ni, // JTAG test reset pad
39     input logic td_i, // JTAG test data input pad
40     output logic td_o, // JTAG test data output pad
41     output logic tdo_oe_o // Data out output enable
42 );
43
44 typedef enum logic [1:0] {
45     DMI_NoError = 2'h0, DMI_ReservedError = 2'h1,
46     DMI_OPFailed = 2'h2, DMI_Busy = 2'h3
47 } dmi_error_e;
48 dmi_error_e error_d, error_q;
49
50 logic tck;
51 logic jtag_dmi_clear; // Synchronous reset of DMI triggered
by TestLogicReset in
52 // jtag TAP
53 logic dmi_clear; // Functional (warm) reset of the entire
DMI
54
55 logic update;
56 logic capture;
57 logic shift;
58 logic tdi;
59
60 logic dtmcs_select;
61

```

```

62 dm::dtmcs_t dtmcs_d, dtmcs_q;
63
64 assign dmi_clear = jtag_dmi_clear || (dtmcs_select && update &&
dtmcs_q.dmihardreset);
65
66 // -----
67 // Debug Module Control and Status
68 // -----
69
70 always_comb begin
71     dtmcs_d = dtmcs_q;
72     if (capture) begin
73         if (dtmcs_select) begin
74             dtmcs_d = '{
75                 zero1 : '0,
76                 dmihardreset : 1'b0,
77                 dmireset : 1'b0,
78                 zero0 : '0,
79                 idle : 3'd1, // 1: Enter Run-
Test/Idle and leave it immediately
80                 dmistat : error_q, // 0: No error,
2: Op failed, 3: too fast
81                 abits : 6'd7, // The size of ad-
dress in dmi
82                 version : 4'd1 // Version de-
scribed in spec version 0.13 (and later?)
83             };
84         end
85     end
86
87     if (shift) begin
88         if (dtmcs_select) dtmcs_d = {tdi, 31'(dtmcs_q >> 1)};
89     end
90 end
91
92 always_ff @(posedge tck or negedge trst_ni) begin
93     if (!trst_ni) begin
94         dtmcs_q <= '0;
95     end else begin
96         dtmcs_q <= dtmcs_d;
97     end
98 end
99
100 // -----
101 // DMI (Debug Module Interface)
102 // -----
103
104 logic dmi_select;
105 logic dmi_tdo;
106
107 dm::dmi_req_t dmi_req;
108 logic dmi_req_ready;
109 logic dmi_req_valid;
110
111 dm::dmi_resp_t dmi_resp;
112 logic dmi_resp_valid;
113 logic dmi_resp_ready;
114
115 typedef struct packed {
116     logic [6:0] address;
117     logic [31:0] data;
118     logic [1:0] op;
119 } dmi_t;
120
121 typedef enum logic [2:0] { Idle, Read, WaitReadValid, Write,
WaitWriteValid } state_e;
122 state_e state_d, state_q;
123
124 logic [$bits(dmi_t)-1:0] dr_d, dr_q;
125 logic [6:0] address_d, address_q;
126 logic [31:0] data_d, data_q;
127
128 dmi_t dmi;
129 assign dmi = dmi_t'(dr_q);
130 assign dmi_req.addr = address_q;
131 assign dmi_req.data = data_q;
132 assign dmi_req.op = (state_q == Write) ? dm::DTM_WRITE :
dm::DTM_READ;
133 // We will always be ready to accept the data we requested.
134 assign dmi_resp_ready = 1'b1;
135
136 logic error_dmi_busy;
137 logic error_dmi_op_failed;
138
139 always_comb begin : p_fsm
140     error_dmi_busy = 1'b0;
141     error_dmi_op_failed = 1'b0;
142     // default assignments
143     state_d = state_q;
144     address_d = address_q;
145     data_d = data_q;
146     error_d = error_q;
147
148     dmi_req_valid = 1'b0;
149
150     if (dmi_clear) begin
151         state_d = Idle;
152         data_d = '0;
153         error_d = DMI_NoError;

```



```

154     address_d = '0';
155 end else begin
156     unique case (state_q)
157     Idle: begin
158         // make sure that no error is sticky
159     if (dmi_select && update && (error_q == DMInoError))
begin
160         // save address and value
161         address_d = dmi.address;
162         data_d = dmi.data;
163         if (dm::dtm_op_e'(dmi.op) == dm::DTM_READ) begin
164             state_d = Read;
165         end else if (dm::dtm_op_e'(dmi.op) ==
dm::DTM_WRITE) begin
166             state_d = Write;
167         end
168         // else this is a nop and we can stay here
169     end
170 end
171 Read: begin
172     dmi_req_valid = 1'b1;
173     if (dmi_req_ready) begin
174         state_d = WaitReadValid;
175     end
176 end
177 WaitReadValid: begin
178     // load data into register and shift out
179     if (dmi_resp_valid) begin
180         unique case (dmi_resp.resp)
181         dm::DTM_SUCCESS: begin
182             data_d = dmi_resp.data;
183         end
184         dm::DTM_ERR: begin
185             data_d = 32'hDEAD_BEEF;
186             error_dmi_op_failed = 1'b1;
187         end
188         dm::DTM_BUSY: begin
189             data_d = 32'hB051_B051;
190             error_dmi_busy = 1'b1;
191         end
192         default: begin
193             data_d = 32'hBAAD_C0DE;
194         end
195     endcase
196     state_d = Idle;
197 end
198 end
199 Write: begin
200     dmi_req_valid = 1'b1;
201 end
202 // request sent, wait for response before going back
203 to idle
204 if (dmi_req_ready) begin
205     state_d = WaitWriteValid;
206 end
207 end
208 WaitWriteValid: begin
209     // got a valid answer go back to idle
210     if (dmi_resp_valid) begin
211         unique case (dmi_resp.resp)
212         dm::DTM_ERR: error_dmi_op_failed = 1'b1;
213         dm::DTM_BUSY: error_dmi_busy = 1'b1;
214         default: ;
215     endcase
216     state_d = Idle;
217 end
218 end
219 default: begin
220     // just wait for idle here
221     if (dmi_resp_valid) begin
222         state_d = Idle;
223     end
224 end
225 endcase
226 // update means we got another request but we didn't
227 finish
228 // the one in progress, this state is sticky
229 if (update && state_q != Idle) begin
230     error_dmi_busy = 1'b1;
231 end
232 // if capture goes high while we are in the read state
233 // or in the corresponding wait state we are not giving
234 back a valid word
235 // -> throw an error
236 if (capture && state_q inside {Read, WaitReadValid}) be-
gin
237     error_dmi_busy = 1'b1;
238 end
239 if (error_dmi_busy && error_q == DMInoError) begin
240     error_d = DMIBusy;
241 end
242 end

```

```

247 if (error_dmi_op_failed && error_q == DMInoError) begin
248     error_d = DMIOpFailed;
249 end
250 // clear sticky error flag
251 if (update && dtmcs_q.dmireset && dtmcs_select) begin
252     error_d = DMInoError;
253 end
254 end
255 // shift register
256 assign dmi_tdo = dr_q[0];
257 always_comb begin : p_shift
258     dr_d = dr_q;
259     if (dmi_clear) begin
260         dr_d = '0;
261     end else begin
262         if (capture) begin
263             if (dmi_select) begin
264                 if (error_q == DMInoError && !error_dmi_busy) begin
265                     dr_d = {address_q, data_q, DMInoError};
266                     // DMI was busy, report an error
267                 end else if (error_q == DMIBusy || error_dmi_busy)
begin
268                     dr_d = {address_q, data_q, DMIBusy};
269                 end
270             end
271         end
272         if (shift) begin
273             if (dmi_select) begin
274                 dr_d = {tdi, dr_q[$bits(dr_q)-1:1]};
275             end
276         end
277     end
278 end
279 always_ff @(posedge tck or negedge trst_ni) begin
280     if (!trst_ni) begin
281         dr_q <= '0;
282         state_q <= Idle;
283         address_q <= '0;
284         data_q <= '0;
285         error_q <= DMInoError;
286     end else begin
287         dr_q <= dr_d;
288         state_q <= state_d;
289         address_q <= address_d;
290         data_q <= data_d;
291         error_q <= error_d;
292     end
293 end
294 // -----
295 // TAP
296 // -----
297 dmi_jtag_tap #(
298     .IrLength(5),
299     .IdcodeValue(IdcodeValue)
300 ) i_dmi_jtag_tap (
301     .tck_i,
302     .tms_i,
303     .trst_ni,
304     .tdi_i,
305     .tdo_i,
306     .tdo_oe_o,
307     .testmode_i,
308     .tck_o ( tck ),
309     .dmi_clear_o ( jtag_dmi_clear ),
310     .update_o ( update ),
311     .capture_o ( capture ),
312     .shift_o ( shift ),
313     .tdi_o ( tdi ),
314     .dtmcs_select_o ( dtmcs_select ),
315     .dtmcs_tdo_i ( dtmcs_q[0] ),
316     .dmi_select_o ( dmi_select ),
317     .dmi_tdo_i ( dmi_tdo )
318 );
319 // -----
320 // CDC
321 // -----
322 dmi_cdc i_dmi_cdc (
323     // JTAG side (master side)
324     .tck_i ( tck ),
325     .trst_ni ( trst_ni ),
326     .jtag_dmi_cdc_clear_i ( dmi_clear ),
327     .jtag_dmi_req_i ( dmi_req ),
328     .jtag_dmi_ready_o ( dmi_req_ready ),
329     .jtag_dmi_valid_i ( dmi_req_valid ),
330     .jtag_dmi_resp_o ( dmi_resp ),
331     .jtag_dmi_valid_o ( dmi_resp_valid ),
332     .jtag_dmi_ready_i ( dmi_resp_ready ),
333     // core side
334     .clk_i,
335     .rst_ni,
336     .core_dmi_rst_no ( dmi_rst_no ),
337     .core_dmi_req_o ( dmi_req_o ),
338     .core_dmi_valid_o ( dmi_req_valid_o ),

```

```

347 .core_dmi_ready_i    ( dmi_req_ready_i ),
348 .core_dmi_resp_i     ( dmi_resp_i ),
349 .core_dmi_ready_o    ( dmi_resp_ready_o ),
350 .core_dmi_valid_i    ( dmi_resp_valid_i )
351 );
352
353 endmodule : dmi_jtag

```

11. pulp-platform_riscv-dbg/src/dmi_jtag_tap.sv (326 lines)

```

1 /* Copyright 2018 ETH Zurich and University of Bologna.
2 * Copyright and related rights are licensed under the Solder-
  pad Hardware
3 * License, Version 0.51 (the "License"); you may not use this
  file except in
4 * compliance with the License. You may obtain a copy of the
  License at
5 * http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 * or agreed to in writing, software, hardware and materials
  distributed under
7 * this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 * CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 * specific language governing permissions and limitations un-
  der the License.
10 *
11 * File: dmi_jtag_tap.sv
12 * Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
13 * Date: 19.7.2018
14 *
15 * Description: JTAG TAP for DMI (according to debug spec
0.13)
16 *
17 */
18
19 module dmi_jtag_tap #(
20     parameter int unsigned IrLength = 5,
21     // JTAG IDCODE Value
22     parameter logic [31:0] IdcodeValue = 32'h00000001
23     // xxxx version
24     // xxxxxxxxxxxxxxxx part number
25     // xxxxxxxxxxxxxxxx manufacturer id
26     // 1 required by standard
27 ) (
28     input logic tck_i, // JTAG test clock pad
29     input logic tms_i, // JTAG test mode select pad
30     input logic trst_ni, // JTAG test reset pad
31     input logic td_i, // JTAG test data input pad
32     output logic td_o, // JTAG test data output pad
33     output logic tdo_oe_o, // Data out output enable
34     input logic testmode_i,
35     // JTAG is interested in writing the DTM CSR register
36     output logic tck_o,
37     // Synchronous reset of the dmi module triggered by JTAG TAP
38     output logic dmi_clear_o,
39     output logic update_o,
40     output logic capture_o,
41     output logic shift_o,
42     output logic tdi_o,
43     output logic dtmcs_select_o,
44     input logic dtmcs_tdo_i,
45     // we want to access DMI register
46     output logic dmi_select_o,
47     input logic dmi_tdo_i
48 );
49
50 typedef enum logic [3:0] {
51     TestLogicReset, RunTestIdle, SelectDrScan,
52     CaptureDr, ShiftDr, Exit1Dr, PauseDr, Exit2Dr,
53     UpdateDr, SelectIrScan, CaptureIr, ShiftIr,
54     Exit1Ir, PauseIr, Exit2Ir, UpdateIr
55 } tap_state_e;
56
57 tap_state_e tap_state_q, tap_state_d;
58 logic update_dr, shift_dr, capture_dr;
59
60 typedef enum logic [IrLength-1:0] {
61     BYPASS0 = 'h0,
62     IDCODE = 'h1,
63     DTMCSR = 'h10,
64     DMIACCESS = 'h11,
65     BYPASS1 = 'h1f
66 } ir_reg_e;
67
68 // -----
69 // IR logic
70 // -----
71
72 // shift register
73 logic [IrLength-1:0] jtag_ir_shift_d, jtag_ir_shift_q;
74 // IR register -> this gets captured from shift register
  upon update_ir
75     ir_reg_e jtag_ir_d, jtag_ir_q;
76     logic capture_ir, shift_ir, update_ir, test_logic_reset; //
  pause_ir
77
78     always_comb begin : p_jtag

```

```

79     jtag_ir_shift_d = jtag_ir_shift_q;
80     jtag_ir_d = jtag_ir_q;
81
82     // IR shift register
83     if (shift_ir) begin
84         jtag_ir_shift_d = {td_i, jtag_ir_shift_q[IrLength-1:1]};
85     end
86
87     // capture IR register
88     if (capture_ir) begin
89         jtag_ir_shift_d = IrLength'(4'b0101);
90     end
91
92     // update IR register
93     if (update_ir) begin
94         jtag_ir_d = ir_reg_e'(jtag_ir_shift_q);
95     end
96
97     if (test_logic_reset) begin
98         // Bring all TAP state to the initial value.
99         jtag_ir_shift_d = '0;
100        jtag_ir_d = IDCODE;
101    end
102 end
103
104 always_ff @(posedge tck_i, negedge trst_ni) begin :
  p_jtag_ir_reg
105     if (!trst_ni) begin
106         jtag_ir_shift_q <= '0;
107         jtag_ir_q <= IDCODE;
108     end else begin
109         jtag_ir_shift_q <= jtag_ir_shift_d;
110         jtag_ir_q <= jtag_ir_d;
111     end
112 end
113
114 // -----
115 // TAP DR Regs
116 // -----
117 // - Bypass
118 // - IDCODE
119 // - DTM CS
120 logic [31:0] idcode_d, idcode_q;
121 logic idcode_select;
122 logic bypass_select;
123
124 logic bypass_d, bypass_q; // this is a 1-bit re-
  gister
125
126 always_comb begin
127     idcode_d = idcode_q;
128     bypass_d = bypass_q;
129
130     if (capture_dr) begin
131         if (idcode_select) idcode_d = IdcodeValue;
132         if (bypass_select) bypass_d = 1'b0;
133     end
134
135     if (shift_dr) begin
136         if (idcode_select) idcode_d = {td_i, 31'(idcode_q >>
137         1)};
138         if (bypass_select) bypass_d = td_i;
139     end
140
141     if (test_logic_reset) begin
142         // Bring all TAP state to the initial value.
143         idcode_d = IdcodeValue;
144         bypass_d = 1'b0;
145     end
146 end
147
148 // -----
149 // Data reg select
150 // -----
151 always_comb begin : p_data_reg_sel
152     dmi_select_o = 1'b0;
153     dtmcs_select_o = 1'b0;
154     idcode_select = 1'b0;
155     bypass_select = 1'b0;
156     unique case (jtag_ir_q)
157         BYPASS0: bypass_select = 1'b1;
158         IDCODE: idcode_select = 1'b1;
159         DTMCSR: dtmcs_select_o = 1'b1;
160         DMIACCESS: dmi_select_o = 1'b1;
161         BYPASS1: bypass_select = 1'b1;
162         default: bypass_select = 1'b1;
163     endcase
164 end
165
166 // -----
167 // Output select
168 // -----
169 logic tdo_mux;
170
171 always_comb begin : p_out_sel
172     // we are shifting out the IR register
173     if (shift_ir) begin
174         tdo_mux = jtag_ir_shift_q[0];
175     // here we are shifting the DR register
176     end else begin
177         unique case (jtag_ir_q)

```



```

177         IDCODE:      tdo_mux = idcode_q[0]; // Reading ID
code
178         DTMCsr:      tdo_mux = dtmcs_tdo_i; // Read from
DTMCS TDO
179         DMIACCESS:   tdo_mux = dmi_tdo_i; // Read from
DMI TDO
180         default:     tdo_mux = bypass_q; // BYPASS in-
struction
181         endcase
182     end
183 end
184
185 // -----
186 // DFT
187 // -----
188 logic tck_n, tck_ni;
189
190 tc_clk_inverter i_tck_inv (
191     .clk_i ( tck_i ),
192     .clk_o ( tck_ni )
193 );
194
195 tc_clk_mux2 i_dft_tck_mux (
196     .clk0_i ( tck_ni ),
197     .clk1_i ( tck_i ), // bypass the inverted clock
for testing
198     .clk_sel_i ( testmode_i ),
199     .clk_o ( tck_n )
200 );
201
202 // TDO changes state at negative edge of TCK
203 always_ff @(posedge tck_n, negedge trst_ni) begin :
p_tdo_regs
204     if (!trst_ni) begin
205         td_o <= 1'b0;
206         tdo_oe_o <= 1'b0;
207     end else begin
208         td_o <= tdo_mux;
209         tdo_oe_o <= (shift_ir | shift_dr);
210     end
211 end
212 // -----
213 // TAP FSM
214 // -----
215 // Determination of next state; purely combinatorial
216 always_comb begin : p_tap_fsm
217
218     test_logic_reset = 1'b0;
219
220     capture_dr = 1'b0;
221     shift_dr = 1'b0;
222     update_dr = 1'b0;
223
224     capture_ir = 1'b0;
225     shift_ir = 1'b0;
226     // pause_ir = 1'b0; unused
227     update_ir = 1'b0;
228
229     unique case (tap_state_q)
230     TestLogicReset: begin
231         tap_state_d = (tms_i) ? TestLogicReset : RunTestIdle;
232         test_logic_reset = 1'b1;
233     end
234     RunTestIdle: begin
235         tap_state_d = (tms_i) ? SelectDrScan : RunTestIdle;
236     end
237     // DR Path
238     SelectDrScan: begin
239         tap_state_d = (tms_i) ? SelectIrScan : CaptureDr;
240     end
241     CaptureDr: begin
242         capture_dr = 1'b1;
243         tap_state_d = (tms_i) ? Exit1Dr : ShiftDr;
244     end
245     ShiftDr: begin
246         shift_dr = 1'b1;
247         tap_state_d = (tms_i) ? Exit1Dr : ShiftDr;
248     end
249     Exit1Dr: begin
250         tap_state_d = (tms_i) ? UpdateDr : PauseDr;
251     end
252     PauseDr: begin
253         tap_state_d = (tms_i) ? Exit2Dr : PauseDr;
254     end
255     Exit2Dr: begin
256         tap_state_d = (tms_i) ? UpdateDr : ShiftDr;
257     end
258     UpdateDr: begin
259         update_dr = 1'b1;
260         tap_state_d = (tms_i) ? SelectDrScan : RunTestIdle;
261     end
262     // IR Path
263     SelectIrScan: begin
264         tap_state_d = (tms_i) ? TestLogicReset : CaptureIr;
265     end
266     // In this controller state, the shift register bank in
the
267     // Instruction Register parallel loads a pattern of
fixed values on

```

```

268     // the rising edge of TCK. The last two significant bits
must always
269     // be "01".
270     CaptureIr: begin
271         capture_ir = 1'b1;
272         tap_state_d = (tms_i) ? Exit1Ir : ShiftIr;
273     end
274     // In this controller state, the instruction register
gets connected
275     // between TDI and TDO, and the captured pattern gets
shifted on
276
277     // each rising edge of TCK. The instruction available on
the TDI
278     // pin is also shifted in to the instruction register.
279     ShiftIr: begin
280         shift_ir = 1'b1;
281         tap_state_d = (tms_i) ? Exit1Ir : ShiftIr;
282     end
283     Exit1Ir: begin
284         tap_state_d = (tms_i) ? UpdateIr : PauseIr;
285     end
286     PauseIr: begin
287         // pause_ir = 1'b1; // unused
288         tap_state_d = (tms_i) ? Exit2Ir : PauseIr;
289     end
290     Exit2Ir: begin
291         tap_state_d = (tms_i) ? UpdateIr : ShiftIr;
292     end
293     // In this controller state, the instruction in the in-
struction
294     // shift register is latched to the latch bank of the
Instruction
295     // Register on every falling edge of TCK. This instruc-
tion becomes
296     // the current instruction once it is latched.
297     UpdateIr: begin
298         update_ir = 1'b1;
299         tap_state_d = (tms_i) ? SelectDrScan : RunTestIdle;
300     end
301     default: ; // can't actually happen since case is full
endcase
302 end
303
304 always_ff @(posedge tck_i or negedge trst_ni) begin : p_regs
305     if (!trst_ni) begin
306         tap_state_q <= TestLogicReset;
307         idcode_q <= IdcodeValue;
308         bypass_q <= 1'b0;
309     end else begin
310         tap_state_q <= tap_state_d;
311         idcode_q <= idcode_d;
312         bypass_q <= bypass_d;
313     end
314 end
315
316 // Pass through JTAG signals to debug custom DR logic.
317 // In case of a single TAP those are just feed-through.
318 assign tck_o = tck_i;
319 assign tdi_o = td_i;
320 assign update_o = update_dr;
321 assign shift_o = shift_dr;
322 assign capture_o = capture_dr;
323 assign dmi_clear_o = test_logic_reset;
324
325 endmodule : dmi_jtag_tap

```

12. pulp-platform_riscv-dbg/src/dmi_test.sv (339 lines)

```

1 // Copyright 2021 ETH Zurich and University of Bologna.
2 // Solderpad Hardware License, Version 0.51, see LICENSE for
details.
3 // SPDX-License-Identifier: SHL-0.51
4
5 // Florian Zaruba <zarubaf@iis.ee.ethz.ch>
6
7 /// A set of testbench utilities for the DMI interfaces.
8 package dmi_test;
9
10 import dm::*;
11
12 class req_t #(
13     parameter int AW = 7
14 );
15     rand logic [AW-1:0] addr;
16     rand dtm_op_e op;
17     rand logic [31:0] data;
18
19     /// Compare objects of same type.
20     function do_compare(req_t rhs);
21         return addr == rhs.addr &
22             op == rhs.op &
23             data == rhs.data;
24     endfunction
25
26 endclass
27
28 class rsp_t;
29     rand logic [31:0] data;

```

```

30  rand logic [1:0]  resp;
31
32  /// Compare objects of same type.
33  function do_compare(rsp_t rhs);
34    return data == rhs.data &
35           resp == rhs.resp;
36  endfunction
37
38  endclass
39
40  /// A driver for the DMI interface.
41  class dmi_driver #(
42    parameter int  AW = -1,
43    parameter realtime TA = 0 , // stimuli application time
44    parameter realtime TT = 0 // stimuli test time
45  );
46    virtual DMI_BUS_DV #(
47      .ADDR_WIDTH(AW)
48    ) bus;
49
50    function new(
51      virtual DMI_BUS_DV #(
52        .ADDR_WIDTH(AW)
53      ) bus
54    );
55      this.bus = bus;
56    endfunction
57
58    task reset_master;
59      bus.q_addr <= '0;
60      bus.q_op <= DTM_NOP;
61      bus.q_data <= '0;
62      bus.q_valid <= '0;
63      bus.p_ready <= '0;
64    endtask
65
66    task reset_slave;
67      bus.q_ready <= '0;
68      bus.p_data <= '0;
69      bus.p_resp <= '0;
70      bus.p_valid <= '0;
71    endtask
72
73    task cycle_start;
74      #TT;
75    endtask
76
77    task cycle_end;
78      @(posedge bus.clk_i);
79    endtask
80
81    /// Send a request.
82    task send_req (input req_t req);
83      bus.q_addr <= #TA req.addr;
84      bus.q_op <= #TA req.op;
85      bus.q_data <= #TA req.data;
86      bus.q_valid <= #TA 1;
87      cycle_start();
88      while (bus.q_ready != 1) begin cycle_end();
89    cycle_start(); end
89      cycle_end();
90      bus.q_addr <= #TA '0;
91      bus.q_op <= #TA DTM_NOP;
92      bus.q_data <= #TA '0;
93      bus.q_valid <= #TA 0;
94    endtask
95
96    /// Send a response.
97    task send_rsp (input rsp_t rsp);
98      bus.p_data <= #TA rsp.data;
99      bus.p_resp <= #TA rsp.resp;
100     bus.p_valid <= #TA 1;
101     cycle_start();
102     while (bus.p_ready != 1) begin cycle_end();
103   cycle_start(); end
103     cycle_end();
104     bus.p_data <= #TA '0;
105     bus.p_resp <= #TA '0;
106     bus.p_valid <= #TA 0;
107   endtask
108
109   /// Receive a request.
110   task recv_req (output req_t req);
111     bus.q_ready <= #TA 1;
112     cycle_start();
113     while (bus.q_valid != 1) begin cycle_end();
114   cycle_start(); end
114     req = new;
115     req.addr = bus.q_addr;
116     req.op = bus.q_op;
117     req.data = bus.q_data;
118     cycle_end();
119     bus.q_ready <= #TA 0;
120   endtask
121
122   /// Receive a response.
123   task recv_rsp (output rsp_t rsp);
124     bus.p_ready <= #TA 1;
125     cycle_start();
126     while (bus.p_valid != 1) begin cycle_end();
127   cycle_start(); end

```

```

127     rsp = new;
128     rsp.data = bus.p_data;
129     rsp.resp = bus.p_resp;
130     cycle_end();
131     bus.p_ready <= #TA 0;
132   endtask
133
134   /// Monitor request.
135   task mon_req (output req_t req);
136     cycle_start();
137   while (!(bus.q_valid && bus.q_ready)) begin cycle_end();
138   cycle_start(); end
138     req = new;
139     req.addr = bus.q_addr;
140     req.op = bus.q_op;
141     req.data = bus.q_data;
142     cycle_end();
143   endtask
144
145   /// Monitor response.
146   task mon_rsp (output rsp_t rsp);
147     cycle_start();
148   while (!(bus.p_valid && bus.p_ready)) begin cycle_end();
149   cycle_start(); end
149     rsp = new;
150     rsp.data = bus.p_data;
151     rsp.resp = bus.p_resp;
152     cycle_end();
153   endtask
154
155   endclass
156
157   // Super class for random dmi drivers.
158   virtual class rand_dmi #(
159     // dmi interface parameters
160     parameter int  AW = 32,
161     // Stimuli application and test time
162     parameter realtime TA = 0ps,
163     parameter realtime TT = 0ps
164   );
165
166     typedef dmi_test::dmi_driver #(
167       // dmi bus interface parameters;
168       .AW ( AW ),
169       // Stimuli application and test time
170       .TA ( TA ),
171       .TT ( TT )
172     ) dmi_driver_t;
173
174     dmi_driver_t drv;
175
176     function new(virtual DMI_BUS_DV #( .ADDR_WIDTH (AW)) bus);
177       this.drv = new (bus);
178     endfunction
179
180     task automatic rand_wait(input int unsigned min, input int
181   unsigned max);
182       int unsigned cycles, rand_success, selection;
183       assert (min <= max) else $fatal(1, "Invalid random wait
184   range.");
185       // Give the minimum ten times the weight of every other
186   value. Expressing
187       // this directly avoids simulator-dependent support for
188   `dist` constraints.
189       rand_success = std::randomize(selection) with {
190     selection <= max - min + 9;
191   };
192       assert (rand_success == 1) else $fatal(1, "Failed to
193   randomize wait cycles.");
194       cycles = (selection < 10) ? min : min + selection - 9;
195       repeat (cycles) @(posedge this.drv.bus.clk_i);
196     endtask
197
198   endclass
199
200   /// Generate random requests as a master device.
201   class rand_dmi_master #(
202     // dmi interface parameters
203     parameter int  AW = 32,
204     // Stimuli application and test time
205     parameter realtime TA = 0ps,
206     parameter realtime TT = 0ps,
207     parameter int unsigned REQ_MIN_WAIT_CYCLES = 1,
208     parameter int unsigned REQ_MAX_WAIT_CYCLES = 20,
209     parameter int unsigned RSP_MIN_WAIT_CYCLES = 1,
210     parameter int unsigned RSP_MAX_WAIT_CYCLES = 20
211   ) extends rand_dmi #( .AW(AW), .TA(TA), .TT(TT));
212
213     int unsigned cnt = 0;
214     bit req_done = 0;
215
216     /// Reset the driver.
217     task reset();
218       drv.reset_master();
219     endtask
220
221     /// Constructor.
222     function new(virtual DMI_BUS_DV #( .ADDR_WIDTH (AW)) bus);

```

```

218     super.new(bus);
219 endfunction
220
221 task run(input int n);
222     fork
223         send_requests(n);
224         recv_response();
225     join
226 endtask
227
228 /// Send random requests.
229 task send_requests(input int n);
230     automatic req_t r = new;
231
232     repeat (n) begin
233         this.cnt++;
234         assert(r.randomize());
235         rand_wait(REQ_MIN_WAIT_CYCLES, REQ_MAX_WAIT_CYCLES);
236         this.drv.send_req(r);
237     end
238     this.req_done = 1;
239 endtask
240
241 /// Receive random responses.
242 task recv_response;
243     while (!this.req_done || this.cnt > 0) begin
244         automatic rsp_t rsp;
245         this.cnt--;
246         rand_wait(RSP_MIN_WAIT_CYCLES, RSP_MAX_WAIT_CYCLES);
247         this.drv.recv_rsp(rsp);
248     end
249 endtask
250 endclass
251
252 class rand_dmi_slave #(
253     // dmi interface parameters
254     parameter int    AW = 32,
255     // Stimuli application and test time
256     parameter realtime TA = 0ps,
257     parameter realtime TT = 0ps,
258     parameter int unsigned REQ_MIN_WAIT_CYCLES = 0,
259     parameter int unsigned REQ_MAX_WAIT_CYCLES = 10,
260     parameter int unsigned RSP_MIN_WAIT_CYCLES = 0,
261     parameter int unsigned RSP_MAX_WAIT_CYCLES = 10
262 ) extends rand_dmi #(.AW(AW), .TA(TA), .TT(TT));
263
264     mailbox #(req_t) req_mbx = new();
265
266     /// Reset the driver.
267     task reset();
268         drv.reset_slave();
269     endtask
270
271     task run();
272         fork
273             recv_requests();
274             send_responses();
275         join
276     endtask
277
278     /// Constructor.

```

```

279 function new(virtual DMI_BUS_DV #( .ADDR_WIDTH (AW)) bus);
280     super.new(bus);
281 endfunction
282
283 task recv_requests();
284     forever begin
285         automatic req_t req;
286         rand_wait(REQ_MIN_WAIT_CYCLES, REQ_MAX_WAIT_CYCLES);
287         this.drv.recv_req(req);
288         req_mbx.put(req);
289     end
290 endtask
291
292 task send_responses();
293     automatic rsp_t rsp = new;
294     automatic req_t req;
295     forever begin
296         req_mbx.get(req);
297         rsp.data = $urandom;
298         rsp.resp = DTM_SUCCESS;
299         @(posedge this.drv.bus.clk_i);
300         rand_wait(RSP_MIN_WAIT_CYCLES, RSP_MAX_WAIT_CYCLES);
301         this.drv.send_rsp(rsp);
302     end
303 endtask
304 endclass
305
306 class dmi_monitor #(
307     // dmi interface parameters
308     parameter int    AW = 32,
309     // Stimuli application and test time
310     parameter realtime TA = 0ps,
311     parameter realtime TT = 0ps
312 ) extends rand_dmi #(.AW(AW), .TA(TA), .TT(TT));
313
314     mailbox #(req_t) req_mbx = new;
315     mailbox #(rsp_t) rsp_mbx = new;
316
317     /// Constructor.
318     function new(virtual DMI_BUS_DV #( .ADDR_WIDTH (AW)) bus);
319         super.new(bus);
320     endfunction
321
322     // dmi Monitor.
323     task monitor;
324         fork
325             forever begin
326                 automatic dmi_test::req_t req;
327                 this.drv.mon_req(req);
328                 req_mbx.put(req);
329             end
330             forever begin
331                 automatic dmi_test::rsp_t rsp;
332                 this.drv.mon_rsp(rsp);
333                 rsp_mbx.put(rsp);
334             end
335         join
336     endtask
337 endclass
338
339 endpackage

```

CV32E40P RISC-V 코어

What this is. A 32-bit RISC-V core with PULP extensions, taped out in several projects.

Why it exists. Because it is the design under test in the **step-and-compare methodology** described in the verification chapters, and small enough to read completely.

Undergraduate review · EECS concepts. Pipelining; hazard detection and forwarding; exception handling; CSR implementation.

Graduate · Practice note. Read the decode stage first, then the controller. Nearly all of a small core's complexity is in hazard and exception control, not in the datapath.

6 files • 3,930 lines

API — cv32e40p_aligner.sv

Table 1672. openhwgroup_cv32e40p/rtl/cv32e40p_aligner.sv — module / interface / package (1개)

이름	종류
cv32e40p_aligner	module

Table 1673. openhwgroup_cv32e40p/rtl/cv32e40p_aligner.sv — 포트 (13개)

포트	방향	형/폭
clk	input	logic
rst_n	input	logic
fetch_valid_i	input	logic
aligner_ready_o	output	logic
if_valid_i	input	logic
fetch_rdata_i	input	logic [31:0]
instr_aligned_o	output	logic [31:0]
instr_valid_o	output	logic
branch_addr_i	input	logic [31:0]
branch_i	input	logic
hwlp_addr_i	input	logic [31:0]
hwlp_update_pc_i	input	logic
pc_o	output	logic [31:0]

API — cv32e40p_alu.sv

Table 1674. openhwgroup_cv32e40p/rtl/cv32e40p_alu.sv — module / interface / package (1개)

이름	종류
cv32e40p_alu	module

Table 1675. openhwgroup_cv32e40p/rtl/cv32e40p_alu.sv — 포트 (18개)

포트	방향	형/폭
clk	input	logic
rst_n	input	logic
enable_i	input	logic
operator_i	input	alu_opcode_e
operand_a_i	input	logic [31:0]
operand_b_i	input	logic [31:0]
operand_c_i	input	logic [31:0]
vector_mode_i	input	logic [1:0]
bmask_a_i	input	logic [4:0]
bmask_b_i	input	logic [4:0]
imm_vec_ext_i	input	logic [1:0]
is_clpx_i	input	logic
is_subrot_i	input	logic
clpx_shift_i	input	logic [1:0]
result_o	output	logic [31:0]
comparison_result_o	output	logic
ready_o	output	logic
ex_ready_i	input	logic

API — cv32e40p_alu_div.sv**Table 1676. openhwgroup_cv32e40p/rtl/cv32e40p_alu_div.sv — module / interface / package (1개)**

이름	종류
cv32e40p_alu_div	module

Table 1677. openhwgroup_cv32e40p/rtl/cv32e40p_alu_div.sv — parameter (2개)

파라미터	형/폭	기본값
C_WIDTH		32
C_LOG_WIDTH		6

Table 1678. openhwgroup_cv32e40p/rtl/cv32e40p_alu_div.sv — 포트 (12개)

포트	방향	형/폭
Clk_CI	input	logic
Rst_RBI	input	logic

포트	방향	형/폭
OpA_DI	input	logic [C_WIDTH-1:0]
OpB_DI	input	logic [C_WIDTH-1:0]
OpBShift_DI	input	logic [C_LOG_WIDTH-1:0]
OpBIsZero_SI	input	logic
OpBSign_SI	input	logic
OpCode_SI	input	logic [1:0]
InVld_SI	input	logic
OutRdy_SI	input	logic
OutVld_S0	output	logic
Res_D0	output	logic [C_WIDTH-1:0]

API — cv32e40p_apu_disp.sv

Table 1679. openhwgroup_cv32e40p/rtl/cv32e40p_apu_disp.sv — module / interface / package (1개)

이름	종류
cv32e40p_apu_disp	module

Table 1680. openhwgroup_cv32e40p/rtl/cv32e40p_apu_disp.sv — 포트 (23개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
enable_i	input	logic
apu_lat_i	input	logic [1:0]
apu_waddr_i	input	logic [5:0]
apu_waddr_o	output	logic [5:0]
apu_multicycle_o	output	logic
apu_singlecycle_o	output	logic
active_o	output	logic
stall_o	output	logic
is_decoding_i	input	logic
read_regs_i	input	logic [2:0][5:0]
read_regs_valid_i	input	logic [2:0]
read_dep_o	output	logic
read_dep_for_jalr_o	output	logic
write_regs_i	input	logic [1:0][5:0]
write_regs_valid_i	input	logic [1:0]
write_dep_o	output	logic

포트	방향	형/폭
perf_type_o	output	logic
perf_cont_o	output	logic
apu_req_o	output	logic
apu_gnt_i	input	logic
apu_rvalid_i	input	logic

API — cv32e40p_compressed_decoder.sv

Table 1681. openhwgroup_cv32e40p/rtl/cv32e40p_compressed_decoder.sv — module / interface / package (1개)

이름	종류
cv32e40p_compressed_decoder	module

Table 1682. openhwgroup_cv32e40p/rtl/cv32e40p_compressed_decoder.sv — parameter (2개)

파라미터	형/폭	기본값
FPU		0
ZFINX		0

Table 1683. openhwgroup_cv32e40p/rtl/cv32e40p_compressed_decoder.sv — 포트 (4개)

포트	방향	형/폭
instr_i	input	logic [31:0]
instr_o	output	logic [31:0]
is_compressed_o	output	logic
illegal_instr_o	output	logic

API — cv32e40p_controller.sv

Table 1684. openhwgroup_cv32e40p/rtl/cv32e40p_controller.sv — module / interface / package (1개)

이름	종류
cv32e40p_controller	module

Table 1685. openhwgroup_cv32e40p/rtl/cv32e40p_controller.sv — parameter (3개)

파라미터	형/폭	기본값
COREV_CLUSTER		0
COREV_PULP		0
FPU		0

Table 1686. openhwgroup_cv32e40p/rtl/cv32e40p_controller.sv — 포트 (108개) — 앞 60개만 표시(전체 108개, 본문 소스 참조)

포트	방향	형/폭
clk	input	logic

포트	방향	형/폭
clk_ungated_i	input	logic
rst_n	input	logic
fetch_enable_i	input	logic
ctrl_busy_o	output	logic
is_decoding_o	output	logic
is_fetch_failed_i	input	logic
deassert_we_o	output	logic
illegal_insn_i	input	logic
ecall_insn_i	input	logic
mret_insn_i	input	logic
uret_insn_i	input	logic
dret_insn_i	input	logic
mret_dec_i	input	logic
uret_dec_i	input	logic
dret_dec_i	input	logic
wfi_i	input	logic
ebrk_insn_i	input	logic
fencei_insn_i	input	logic
csr_status_i	input	logic
hwlp_mask_o	output	logic
instr_valid_i	input	logic
instr_req_o	output	logic
pc_set_o	output	logic
pc_mux_o	output	logic [3:0]
exc_pc_mux_o	output	logic [2:0]
trap_addr_mux_o	output	logic [1:0]
pc_id_i	input	logic [31:0]
hwlp_start_addr_i	input	logic [1:0] [31:0]
hwlp_end_addr_i	input	logic [1:0] [31:0]
hwlp_counter_i	input	logic [1:0] [31:0]
hwlp_dec_cnt_o	output	logic [1:0]
hwlp_jump_o	output	logic
hwlp_targ_addr_o	output	logic [31:0]
data_req_ex_i	input	logic
data_we_ex_i	input	logic

포트	방향	형/폭
data_misaligned_i	input	logic
data_load_event_i	input	logic
data_err_i	input	logic
data_err_ack_o	output	logic
mult_multicycle_i	input	logic
apu_en_i	input	logic
apu_read_dep_i	input	logic
apu_read_dep_for_jalr_i	input	logic
apu_write_dep_i	input	logic
apu_stall_o	output	logic
branch_taken_ex_i	input	logic
ctrl_transfer_insn_in_id_i	input	logic [1:0]
ctrl_transfer_insn_in_dec_i	input	logic [1:0]
irq_req_ctrl_i	input	logic
irq_sec_ctrl_i	input	logic
irq_id_ctrl_i	input	logic [4:0]
irq_wu_ctrl_i	input	logic
current_priv_lvl_i	input	PrivLvl_t
irq_ack_o	output	logic
irq_id_o	output	logic [4:0]
exc_cause_o	output	logic [4:0]
debug_mode_o	output	logic
debug_cause_o	output	logic [2:0]
debug_csr_save_o	output	logic

1. openhwgroup_cv32e40p/rtl/cv32e40p_aligner.sv (252 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 ///////////////////////////////////////////////////////////////////
12 // Engineer:      Pasquale Davide Schiavone - ps-
  chiamo@iis.ee.ethz.ch
13 //
  //

```

```

14 // Additional contributions
  by:
15 //                                //
  gies.com                        // Igor Loi - igor.loi@greenwaves-technolo-
16 //
  //
17 // Design Name:   Instrctuan Align-
  er                               //
18 // Project Name:
  RI5CY                            //
19 // Language:      SystemVeri-
  log                               //
20 //
  //
21 ///////////////////////////////////////////////////////////////////
22
23 module cv32e40p_aligner (
24     input logic clk,
25     input logic rst_n,
26
27     input logic fetch_valid_i,
28     output logic aligner_ready_o, //prevents overwriting the
  fethced instruction
29
30     input logic if_valid_i,
31

```

```

32  input logic [31:0] fetch_rdata_i,
33  output logic [31:0] instr_aligned_o,
34  output logic      instr_valid_o,
35
36  input logic [31:0] branch_addr_i,
37  input logic      branch_i, // Asserted if we are
branching/jumping now
38
39  input logic [31:0] hwlp_addr_i,
40  input logic      hwlp_update_pc_i,
41
42  output logic [31:0] pc_o
43  );
44
45  enum logic [2:0] {
46    ALIGNED32,
47    MISALIGNED32,
48    MISALIGNED16,
49    BRANCH_MISALIGNED,
50    WAIT_VALID_BRANCH
51  }
52  state, next_state;
53
54  logic [15:0] r_instr_h;
55  logic [31:0] hwlp_addr_q;
56  logic [31:0] pc_q, pc_n;
57  logic update_state;
58  logic [31:0] pc_plus4, pc_plus2;
59  logic aligner_ready_q, hwlp_update_pc_q;
60
61  assign pc_o      = pc_q;
62
63  assign pc_plus2 = pc_q + 2;
64  assign pc_plus4 = pc_q + 4;
65
66  always_ff @(posedge clk or negedge rst_n) begin :
proc_SEQ_FSM
67    if (~rst_n) begin
68      state      <= ALIGNED32;
69      r_instr_h  <= '0;
70      hwlp_addr_q <= '0;
71      pc_q       <= '0;
72      aligner_ready_q <= 1'b0;
73      hwlp_update_pc_q <= 1'b0;
74    end else begin
75      if (update_state) begin
76        pc_q      <= pc_n;
77        state     <= next_state;
78        r_instr_h <= fetch_rdata_i[31:16];
79        aligner_ready_q <= aligner_ready_o;
80        hwlp_update_pc_q <= 1'b0;
81      end else begin
82        if (hwlp_update_pc_i) begin
83          hwlp_addr_q <= hwlp_addr_i; // Save the JUMP
target address to keep pc_n up to date during the stall
84          hwlp_update_pc_q <= 1'b1;
85        end
86      end
87    end
88  end
89
90  always_comb begin
91    //default outputs
92    pc_n      = pc_q;
93    instr_valid_o = fetch_valid_i;
94    instr_aligned_o = fetch_rdata_i;
95    aligner_ready_o = 1'b1;
96    update_state = 1'b0;
97    next_state  = state;
100
101    case (state)
102      ALIGNED32: begin
103        if (fetch_rdata_i[1:0] == 2'b11) begin
104          /*
105            Before we fetched a 32bit aligned instruction
106            Therefore, now the address is aligned too
107            and it is 32bits
108            */
109            next_state = ALIGNED32;
110            pc_n      = pc_plus4;
111            instr_aligned_o = fetch_rdata_i;
112            //gate id_valid with fetch_valid as the next state
should be evaluated only if mem content is valid
113            update_state = fetch_valid_i & if_valid_i;
114            if (hwlp_update_pc_i || hwlp_update_pc_q)
115              pc_n = hwlp_update_pc_i ? hwlp_addr_i : hwlp_ad-
dr_q;
116          end else begin
117            /*
118              Before we fetched a 32bit aligned instruction
119              Therefore, now the address is aligned too
120              and it is 16bits
121              */
122            next_state = MISALIGNED32;
123            pc_n      = pc_plus2;
124            instr_aligned_o = fetch_rdata_i; //only the first

```

```

16b are used
124          //gate id_valid with fetch_valid as the next state
should be evaluated only if mem content is valid
125          update_state = fetch_valid_i & if_valid_i;
126        end
127      end
128
129      MISALIGNED32: begin
130        if (r_instr_h[1:0] == 2'b11) begin
131          /*
132            Before we fetched a 32bit misaligned in-
struction
133
134            So now the beginning of the next instruction
is the stored one
135            The instruction is 32bits so it is misaligned
again
136            */
137            next_state = MISALIGNED32;
138            pc_n      = pc_plus4;
139            instr_aligned_o = {fetch_rdata_i[15:0], r_in-
str_h[15:0]};
140            //gate id_valid with fetch_valid as the next state
should be evaluated only if mem content is valid
141            update_state = fetch_valid_i & if_valid_i;
142          end else begin
143            /*
144              Before we fetched a 32bit misaligned in-
struction
145              So now the beginning of the next instruction
is the stored one
146              The instruction is 16bits misaligned
147              */
148            instr_aligned_o = {fetch_rdata_i[31:16], r_in-
str_h[15:0]}; //only the first 16b are used
149            next_state = MISALIGNED16;
150            instr_valid_o = 1'b1;
151            pc_n      = pc_plus2;
152            //we cannot overwrite the 32bit instruction just
fetched
153            //so tell the IF stage to stall, the coming instruc-
tion goes to the FIFO
154            aligner_ready_o = !fetch_valid_i;
155            //not need to gate id_valid with fetch_valid as the
next state depends only on r_instr_h
156            update_state = if_valid_i;
157          end
158        end
159
160        MISALIGNED16: begin
161          /*this is 1 as we holded the value before with raw_in-
str_hold_o
162          instr_valid_o = !aligner_ready_q || fetch_valid_i;
163          if (fetch_rdata_i[1:0] == 2'b11) begin
164            /*
165              Before we fetched a 16bit misaligned in-
struction
166
167              So now the beginning of the next instruction
is the new one
168              The instruction is 32bits so it is aligned
169              */
170            next_state = ALIGNED32;
171            pc_n      = pc_plus4;
172            instr_aligned_o = fetch_rdata_i;
173            //no gate id_valid with fetch_valid as the next
state sdepends only on mem content that has be held the previous
cycle with raw_instr_hold_o
174            update_state = (!aligner_ready_q | fetch_valid_i) &
if_valid_i;
175          end else begin
176            /*
177              Before we fetched a 16bit misaligned in-
struction
178              So now the beginning of the next instruction
is the new one
179              The instruction is 16bit aligned
180              */
181            next_state = MISALIGNED32;
182            pc_n      = pc_plus2;
183            instr_aligned_o = fetch_rdata_i; //only the first
16b are used
184            //no gate id_valid with fetch_valid as the next
state sdepends only on mem content that has be held the previous
cycle with raw_instr_hold_o
185            update_state = (!aligner_ready_q | fetch_valid_i) &
if_valid_i;
186          end
187        end
188
189        BRANCH_MISALIGNED: begin
190          //we jumped to a misaligned location, so now we re-
ceived {TARGET, XXXX}
191          if (fetch_rdata_i[17:16] == 2'b11) begin

```

```

193      /*
194      We jumped to a misaligned location that con-
195      tains 32bits instruction
196      */
197      next_state = MISALIGNED32;
198      instr_valid_o = 1'b0;
199      pc_n = pc_q;
200      instr_aligned_o = fetch_rdata_i;
201      //gate id_valid with fetch_valid as the next state
202      //should be evaluated only if mem content is valid
203      update_state = fetch_valid_i & if_valid_i;
204      end else begin
205      /*
206      We jumped to a misaligned location that con-
207      tains 16bits instruction, as we consumed the whole word, we can
208      pretend to start again from ALIGNED32
209      */
210      next_state = ALIGNED32;
211      pc_n = pc_plus2;
212      instr_aligned_o = {
213          fetch_rdata_i[31:16], fetch_rdata_i[31:16]
214      }; //only the first 16b are used
215      //gate id_valid with fetch_valid as the next state
216      //should be evaluated only if mem content is valid
217      update_state = fetch_valid_i & if_valid_i;
218      end
219      end
220      endcase // state
221
222      // JUMP, BRANCH, SPECIAL JUMP control
223      if (branch_i) begin
224          update_state = 1'b1;
225          pc_n = branch_addr_i;
226          next_state = branch_addr_i[1] ? BRANCH_MISALIGNED :
227          ALIGNED32;
228      end
229
230      /*
231      When a branch is taken in EX, if_valid_i is asserted because
232      the BRANCH is resolved also in
233      case of stalls. This is because the branch information is
234      stored in the IF stage (in the prefetcher)
235      when branch_i is asserted. We introduced here an apparently
236      useless special case for
237      the JUMPS for a cleaner and more robust HW: theoretically,
238      we don't need to save the instruction
239      after a taken branch in EX, thus we will not do it.
240      */
241      ///////////////////////////////////////////////////////////////////
242      // Assertions
243      ///////////////////////////////////////////////////////////////////
244
245      `ifdef CV32E40P_ASSERT_ON
246      // Hardware Loop check
247      property p_hwlp_update_pc;
248      @(posedge clk) disable iff (!rst_n) (1'b1) |> (! (hwlp_up-
249      date_pc_i && hwlp_update_pc_q));
250      endproperty
251      a_hwlp_update_pc :
252      assert property (p_hwlp_update_pc);
253      `endif
254
255      endmodule

```

2. openhwgroup_cv32e40p/rtl/cv32e40p_alu.sv (986 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
3 // pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
5 // file except in
6 // compliance with the License. You may obtain a copy of the
7 // License at
8 // http://solderpad.org/licenses/SHL-0.51. Unless required by
9 // applicable law
10 // or agreed to in writing, software, hardware and materials
11 // distributed under
12 // this License is distributed on an "AS IS" BASIS, WITHOUT
13 // WARRANTIES OR
14 // CONDITIONS OF ANY KIND, either express or implied. See the
15 // License for the
16 // specific language governing permissions and limitations un-
17 // der the License.
18
19 ///////////////////////////////////////////////////////////////////
20 // Engineer: Matthias Baer - baer-
21 // matt@student.ethz.ch

```

```

13 //
14 // Additional contributions
15 // Igor Loi -
16 // igor.loi@unibo.it
17 // Andreas Traber - at-
18 // raber@student.ethz.ch
19 // Michael Gautschi - gauts-
20 // chi@iis.ee.ethz.ch
21 // Davide Schiavone - ps-
22 // chiavo@iis.ee.ethz.ch
23 //
24 // Design Name:
25 // ALU
26 // Project Name:
27 // RI5CY
28 // Language: SystemVeri-
29 // log
30 // Description: Arithmetic logic unit of the pipelined pro-
31 // cessor
32 //
33 //
34 //
35 //
36 //
37 //
38 //
39 //
40 //
41 //
42 //
43 //
44 //
45 //
46 //
47 //
48 //
49 //
50 //
51 //
52 //
53 //
54 //
55 //
56 //
57 //
58 //
59 //
60 //
61 //
62 //
63 //
64 //
65 //
66 //
67 //
68 //
69 //
70 //
71 //
72 //
73 //
74 //
75 //
76 //
77 //
78 //
79 //
80 //
81 //
82 //
83 //
84 //
85 //
86 //
87 //
88 //
89 //

```

[illegible][illegible]

```

shift_amt);
258
259 assign shift_amt_norm = is_clpx_i ? {clpx_shift_ex,
clpx_shift_ex} : {4{3'b000, bmask_b_i}};
260
261 assign clpx_shift_ex = $unsigned(clpx_shift_i);
262
263 // right shifts, we let the synthesizer optimize this
264 logic [63:0] shift_op_a_32;
265
266 assign shift_op_a_32 = (operator_i == ALU_ROR) ? {
267     shift_op_a, shift_op_a
268 } : $signed(
269     {{32{shift_arithmetic & shift_op_a[31]}},
shift_op_a});
270 );
271
272 always_comb begin
273     case (vector_mode_i)
274         VEC_MODE16: begin
275             shift_right_result[31:16] = $signed(
shift_op_a[31:16])
276                 ) >>> shift_amt_int[19:16];
277             shift_right_result[15:0] = $signed(
shift_op_a[15:0])
278                 ) >>> shift_amt_int[3:0];
279             end
280         VEC_MODE8: begin
281             shift_right_result[31:24] = $signed(
shift_op_a[31:24])
282                 ) >>> shift_amt_int[26:24];
283             shift_right_result[23:16] = $signed(
shift_op_a[23:16])
284                 ) >>> shift_amt_int[18:16];
285             shift_right_result[15:8] = $signed(
shift_op_a[15:8])
286                 ) >>> shift_amt_int[10:8];
287             shift_right_result[7:0] = $signed(
shift_op_a[7:0])
288                 ) >>> shift_amt_int[2:0];
289             end
290         default: // VEC_MODE32
291             begin
292                 shift_right_result = shift_op_a_32 >>
shift_amt_int[4:0];
293             end
294         ; // case (vec_mode_i)
295     end
296
297     // bit reverse the shift_right_result for left shifts
298     genvar j;
299     generate
300         for (j = 0; j < 32; j++) begin : gen_shift_left_result
301             assign shift_left_result[j] = shift_right_result[31-j];
302         end
303     endgenerate
304
305     assign shift_result = shift_left ? shift_left_result :
shift_right_result;
306
307 ////////////////////////////////////////
308 /////
309 // _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
310 // // / _/ _ \/ | \/ | _ \/\ | _ \| _/_ / _/_ \/_ \
| //
311 // | | | | | | | | | | | | | | | | | | | | | | | | | | | |
312 // // \_ \_/ | | | | // \_\ \_\_| _/ \_\_| |
\_ \_/ | |
| //
313 //
314 ////////////////////////////////////////
315 /////
316 logic [ 3:0] is_equal;
317 logic [ 3:0] is_greater; // handles both signed and un-
signed forms
318
319 // 8-bit vector comparisons, basic building blocks
320 logic [ 3:0] cmp_signed;
321 logic [ 3:0] is_equal_vec;
322 logic [ 3:0] is_greater_vec;
323 logic [31:0] operand_b_eq;
324 logic
is_equal_clin;

```

```

335
336
337 //second == comparator for CLIP instructions
338 always_comb begin
339     operand_b_eq = operand_b_neg;
340     if (operator_i == ALU_CLIPU) operand_b_eq = '0;
341     else operand_b_eq = operand_b_neg;
342 end
343 assign is_equal_clip = operand_a_i == operand_b_eq;
344
345 always_comb begin
346     cmp_signed = 4'b0;
347
348     unique case (operator_i)
349         ALU_GTS,
350         ALU_GES,
351         ALU_LTS,
352         ALU_LES,
353         ALU_SLTS,
354         ALU_SLETS,
355         ALU_MIN,
356         ALU_MAX,
357         ALU_ABS,
358         ALU_CLIP,
359         ALU_CLIPU: begin
360             case (vector_mode_i)
361                 VEC_MODE8: cmp_signed[3:0] = 4'b1111;
362                 VEC_MODE16: cmp_signed[3:0] = 4'b1010;
363                 default: cmp_signed[3:0] = 4'b1000;
364             endcase
365         end
366
367         default: ;
368     endcase
369 end
370
371 // generate vector equal and greater than signals,
cmp_signed decides if the
372 // comparison is done signed or unsigned
373 genvar i;
374 generate
375     for (i = 0; i < 4; i++) begin : gen_is_vec
376         assign is_equal_vec[i] = (operand_a_i[8*i+7:8*i] == op-
operand_b_i[8*i+7:8*i]);
377         assign is_greater_vec[i] = $signed(
378             {operand_a_i[8*i+7] & cmp_signed[i], oper-
and_a_i[8*i+7:8*i]}
379             ) > $signed(
380             {operand_b_i[8*i+7] & cmp_signed[i], oper-
and_b_i[8*i+7:8*i]}
381             );
382     end
383 endgenerate
384
385 // generate the real equal and greater than signals that
take the vector
386 // mode into account
387 always_comb begin
388     // 32-bit mode
389     is_equal[3:0] = {4{is_equal_vec[3] & is_equal_vec[2] &
is_equal_vec[1] & is_equal_vec[0]}};
390     is_greater[3:0] = {4{is_greater_vec[3] | (is_equal_vec[3]
& is_greater_vec[2]
391                                     | (is_equal_vec[2] &
(is_greater_vec[1]
392                                     | (is_equal_vec[1] & (is_greater_vec[0]))))}};
393
394     case (vector_mode_i)
395         VEC_MODE16: begin
396             is_equal[1:0] = {2{is_equal_vec[0] &
is_equal_vec[1]}};
397             is_equal[3:2] = {2{is_equal_vec[2] &
is_equal_vec[3]}};
398             is_greater[1:0] = {2{is_greater_vec[1] |
(is_equal_vec[1] & is_greater_vec[0])}};
399             is_greater[3:2] = {2{is_greater_vec[3] |
(is_equal_vec[3] & is_greater_vec[2])}};
400         end
401
402         VEC_MODE8: begin
403             is_equal[3:0] = is_equal_vec[3:0];
404             is_greater[3:0] = is_greater_vec[3:0];
405         end
406
407         default: ; // see default assignment
408     endcase
409 end
410
411 // generate comparison result
412 logic [3:0] cmp_result;
413
414 always_comb begin
415     cmp_result = is_equal;
416     unique case (operator_i)
417         ALU_EQ: cmp_result =
is_equal;
418         ALU_NE: cmp_result =
~is_equal;
419         ALU_GTS, ALU_GTU: cmp_result =

```

```

is_greater;
420     ALU_GES, ALU_GEU:                cmp_result =
is_greater | is_equal;
421     ALU_LTS, ALU_SLTS, ALU_LTU, ALU_SLTU: cmp_result =
~(is_greater | is_equal);
422     ALU_SLETS, ALU_SLETU, ALU_LES, ALU_LEU: cmp_result =
~is_greater;
423     default:                        ;
424     endcase
425     end
426
427     assign comparison_result_o = cmp_result[3];
428
429     // min/max/abs handling
430     logic [31:0] result_minmax;
431     logic [ 3:0] sel_minmax;
432     logic        do_min;
433     logic [31:0] minmax_b;
434
435     assign minmax_b = (operator_i == ALU_ABS) ? adder_result :
operand_b_i;
436
437
438     assign do_min    = (operator_i == ALU_MIN) || (operator_i ==
ALU_MINU) ||
439
440
441     (operator_i == ALU_CLIP) || (operator_i ==
ALU_CLIPU);
442
443     assign sel_minmax[3:0] = is_greater ^ {4{do_min}};
444
445     assign result_minmax[31:24] = (sel_minmax[3] == '1'b1) ? op-
erand_a_i[31:24] : minmax_b[31:24];
446     assign result_minmax[23:16] = (sel_minmax[2] == '1'b1) ? op-
erand_a_i[23:16] : minmax_b[23:16];
447     assign result_minmax[15:8]  = (sel_minmax[1] == '1'b1) ? oper-
and_a_i[15:8] : minmax_b[15:8];
448     assign result_minmax[7:0]  = (sel_minmax[0] == '1'b1) ? oper-
and_a_i[7:0] : minmax_b[7:0];
449
450     //////////////////////////////////////////
451     // Clip
452     //////////////////////////////////////////
453     logic [31:0] clip_result; // result of clip and clip
454
455     always_comb begin
456         clip_result = result_minmax;
457         if (operator_i == ALU_CLIPU) begin
458             if (operand_a_i[31] || is_equal_clip) begin
459                 clip_result = '0;
460             end else begin
461                 clip_result = result_minmax;
462             end
463         end else begin
464             //CLIP
465             if (adder_result_expanded[36] || is_equal_clip) begin
466                 clip_result = operand_b_neg;
467             end else begin
468                 clip_result = result_minmax;
469             end
470         end
471     end
472
473     //////////////////////////////////////////
474     //
475     //////////////////////////////////////////
476     //
477     //////////////////////////////////////////
478     //
479     //////////////////////////////////////////
480
481     logic [3:0][1:0] shuffle_byte_sel; // select byte in re-
gister: 31:24, 23:16, 15:8, 7:0
482     logic [3:0]      shuffle_reg_sel; // select register: rD/
rS2 or rS1
483
484     logic [1:0]      shuffle_reg1_sel; // select register rD or
rS2 for next stage
485     logic [1:0]      shuffle_reg0_sel;
486     logic [3:0]      shuffle_through;
487
488     logic [31:0] shuffle_r1, shuffle_r0;
489     logic [31:0] shuffle_r1_in, shuffle_r0_in;
490     logic [31:0] shuffle_result;
491     logic [31:0] pack_result;
492
493     always_comb begin
494         shuffle_reg_sel = '0;
495         shuffle_reg1_sel = 2'b01;
496         shuffle_reg0_sel = 2'b10;
497         shuffle_through = '1;
498
499         unique case (operator_i)
500             ALU_EXT, ALU_EXTs: begin
501                 if (operator_i == ALU_EXTs) shuffle_reg1_sel = 2'b11;
502             end
503             if (vector_mode_i == VEC_MODE8) begin

```

```

504         shuffle_reg_sel[3:1] = 3'b111;
505         shuffle_reg_sel[0]   = 1'b0;
506     end else begin
507         shuffle_reg_sel[3:2] = 2'b11;
508         shuffle_reg_sel[1:0] = 2'b00;
509     end
510 end
511
512 ALU_PCKLO: begin
513     shuffle_reg1_sel = 2'b00;
514
515     if (vector_mode_i == VEC_MODE8) begin
516         shuffle_through = 4'b0011;
517         shuffle_reg_sel = 4'b0001;
518     end else begin
519         shuffle_reg_sel = 4'b0011;
520     end
521 end
522
523 ALU_PCKHI: begin
524     shuffle_reg1_sel = 2'b00;
525
526     if (vector_mode_i == VEC_MODE8) begin
527         shuffle_through = 4'b1100;
528         shuffle_reg_sel = 4'b0100;
529     end else begin
530         shuffle_reg_sel = 4'b0011;
531     end
532 end
533
534 ALU_SHUF2: begin
535     unique case (vector_mode_i)
536         VEC_MODE8: begin
537             shuffle_reg_sel[3] = ~operand_b_i[26];
538             shuffle_reg_sel[2] = ~operand_b_i[18];
539             shuffle_reg_sel[1] = ~operand_b_i[10];
540             shuffle_reg_sel[0] = ~operand_b_i[2];
541         end
542         VEC_MODE16: begin
543             shuffle_reg_sel[3] = ~operand_b_i[17];
544             shuffle_reg_sel[2] = ~operand_b_i[17];
545             shuffle_reg_sel[1] = ~operand_b_i[1];
546             shuffle_reg_sel[0] = ~operand_b_i[1];
547         end
548         default: ;
549     endcase
550 end
551
552 ALU_INS: begin
553     unique case (vector_mode_i)
554         VEC_MODE8: begin
555             shuffle_reg0_sel = 2'b00;
556             unique case (imm_vec_ext_i)
557                 2'b00: begin
558                     shuffle_reg_sel[3:0] = 4'b1110;
559                 end
560                 2'b01: begin
561                     shuffle_reg_sel[3:0] = 4'b1101;
562                 end
563                 2'b10: begin
564                     shuffle_reg_sel[3:0] = 4'b1011;
565                 end
566                 2'b11: begin
567                     shuffle_reg_sel[3:0] = 4'b0111;
568                 end
569             endcase
570         end
571         VEC_MODE16: begin
572             shuffle_reg0_sel = 2'b01;
573             shuffle_reg_sel[3] = ~imm_vec_ext_i[0];
574             shuffle_reg_sel[2] = ~imm_vec_ext_i[0];
575             shuffle_reg_sel[1] = imm_vec_ext_i[0];
576             shuffle_reg_sel[0] = imm_vec_ext_i[0];
577         end
578         default: ;
579     endcase
580 end
581
582     default: ;
583     endcase
584 end
585
586 always_comb begin
587     shuffle_byte_sel = '0;
588
589     // byte selector
590     unique case (operator_i)
591         ALU_EXTs, ALU_EXT: begin
592             unique case (vector_mode_i)
593                 VEC_MODE8: begin
594                     shuffle_byte_sel[3] = imm_vec_ext_i[1:0];
595                     shuffle_byte_sel[2] = imm_vec_ext_i[1:0];
596                     shuffle_byte_sel[1] = imm_vec_ext_i[1:0];
597                     shuffle_byte_sel[0] = imm_vec_ext_i[1:0];
598                 end
599                 VEC_MODE16: begin
600                     shuffle_byte_sel[3] = {imm_vec_ext_i[0], 1'b1};
601                     shuffle_byte_sel[2] = {imm_vec_ext_i[0], 1'b1};
602                     shuffle_byte_sel[1] = {imm_vec_ext_i[0], 1'b1};
603                     shuffle_byte_sel[0] = {imm_vec_ext_i[0], 1'b1};
604                 end

```



```

605     shuffle_byte_sel[0] = {imm_vec_ext_i[0], 1'b0};
606 end
607
608 default: ;
609 endcase
610 end
611
612 ALU_PCKLO: begin
613     unique case (vector_mode_i)
614         VEC_MODE8: begin
615             shuffle_byte_sel[3] = 2'b00;
616             shuffle_byte_sel[2] = 2'b00;
617             shuffle_byte_sel[1] = 2'b00;
618             shuffle_byte_sel[0] = 2'b00;
619         end
620
621         VEC_MODE16: begin
622             shuffle_byte_sel[3] = 2'b01;
623             shuffle_byte_sel[2] = 2'b00;
624             shuffle_byte_sel[1] = 2'b01;
625             shuffle_byte_sel[0] = 2'b00;
626         end
627
628         default: ;
629     endcase
630 end
631
632 ALU_PCKHI: begin
633     unique case (vector_mode_i)
634         VEC_MODE8: begin
635             shuffle_byte_sel[3] = 2'b00;
636             shuffle_byte_sel[2] = 2'b00;
637             shuffle_byte_sel[1] = 2'b00;
638             shuffle_byte_sel[0] = 2'b00;
639         end
640
641         VEC_MODE16: begin
642             shuffle_byte_sel[3] = 2'b11;
643             shuffle_byte_sel[2] = 2'b10;
644             shuffle_byte_sel[1] = 2'b11;
645             shuffle_byte_sel[0] = 2'b10;
646         end
647
648         default: ;
649     endcase
650 end
651
652 ALU_SHUF2, ALU_SHUF: begin
653     unique case (vector_mode_i)
654         VEC_MODE8: begin
655             shuffle_byte_sel[3] = operand_b_i[25:24];
656             shuffle_byte_sel[2] = operand_b_i[17:16];
657             shuffle_byte_sel[1] = operand_b_i[9:8];
658             shuffle_byte_sel[0] = operand_b_i[1:0];
659         end
660
661         VEC_MODE16: begin
662             shuffle_byte_sel[3] = {operand_b_i[16], 1'b1};
663             shuffle_byte_sel[2] = {operand_b_i[16], 1'b0};
664             shuffle_byte_sel[1] = {operand_b_i[0], 1'b1};
665             shuffle_byte_sel[0] = {operand_b_i[0], 1'b0};
666         end
667
668         default: ;
669     endcase
670 end
671
672 ALU_INS: begin
673     shuffle_byte_sel[3] = 2'b11;
674     shuffle_byte_sel[2] = 2'b10;
675     shuffle_byte_sel[1] = 2'b01;
676     shuffle_byte_sel[0] = 2'b00;
677 end
678
679 default: ;
680 endcase
681 end
682
683 assign shuffle_r0_in = shuffle_reg0_sel[1] ?
684     operand_a_i :
685     (shuffle_reg0_sel[0] ? {2(oper-
686 and_a_i[15:0])} : {4(operand_a_i[7:0])});
687
688 assign shuffle_r1_in = shuffle_reg1_sel[1] ? {
689     {8(operand_a_i[31])}, {8(operand_a_i[23])}, {8(oper-
690 and_a_i[15])}, {8(operand_a_i[7])}
691 } : (shuffle_reg1_sel[0] ? operand_c_i : operand_b_i);
692
693 assign shuffle_r0[31:24] = shuffle_byte_sel[3][1] ?
694     (shuffle_byte_sel[3][0] ?
695 shuffle_r0_in[31:24] : shuffle_r0_in[23:16]) :
696     (shuffle_byte_sel[3][0] ?
697 shuffle_r0_in[15: 8] : shuffle_r0_in[ 7: 0]);
698 assign shuffle_r0[23:16] = shuffle_byte_sel[2][1] ?
699     (shuffle_byte_sel[2][0] ?
700 shuffle_r0_in[31:24] : shuffle_r0_in[23:16]) :
701     (shuffle_byte_sel[2][0] ?
702 shuffle_r0_in[15: 8] : shuffle_r0_in[ 7: 0]);
703 assign shuffle_r0[15: 8] = shuffle_byte_sel[1][1] ?
704     (shuffle_byte_sel[1][0] ?
705 shuffle_r0_in[31:24] : shuffle_r0_in[23:16]) :
706     (shuffle_byte_sel[1][0] ?
707 shuffle_r0_in[15: 8] : shuffle_r0_in[ 7: 0]);
708 assign shuffle_r0[7: 0] = shuffle_byte_sel[0][1] ?
709     (shuffle_byte_sel[0][0] ?
710 shuffle_r0_in[31:24] : shuffle_r0_in[23:16]) :
711     (shuffle_byte_sel[0][0] ?
712 shuffle_r0_in[15: 8] : shuffle_r0_in[ 7: 0]);
713
714 assign shuffle_r1[31:24] = shuffle_reg_sel[3] ?
715     shuffle_r1[31:24] : shuffle_r0[31:24];
716 assign shuffle_result[23:16] = shuffle_reg_sel[2] ?
717     shuffle_r1[23:16] : shuffle_r0[23:16];
718 assign shuffle_result[15:8] = shuffle_reg_sel[1] ?
719     shuffle_r1[15:8] : shuffle_r0[15:8];
720 assign shuffle_result[7:0] = shuffle_reg_sel[0] ?
721     shuffle_r1[7:0] : shuffle_r0[7:0];
722
723 assign pack_result[31:24] = shuffle_through[3] ?
724     shuffle_result[31:24] : operand_c_i[31:24];
725 assign pack_result[23:16] = shuffle_through[2] ?
726     shuffle_result[23:16] : operand_c_i[23:16];
727 assign pack_result[15:8] = shuffle_through[1] ?
728     shuffle_result[15:8] : operand_c_i[15:8];
729 assign pack_result[7:0] = shuffle_through[0] ?
730     shuffle_result[7:0] : operand_c_i[7:0];
731
732 //////////////////////////////////////////
733 //////////////////////////////////////////
734 //////////////////////////////////////////
735 //////////////////////////////////////////
736
737 logic [31:0] ff_input; // either op_a_i or its bit reversed
738 version
739 logic [ 5:0] cnt_result; // population count
740 logic [ 5:0] clb_result; // count leading bits
741 logic [ 4:0] ff1_result; // holds the index of the first
742 '1'
743 logic ff_no_one; // if no ones are found
744 logic [ 4:0] fl1_result; // holds the index of the last '1'
745 logic [ 5:0] bitop_result; // result of all bitop opera-
746 tions muxed together
747
748 cv32e40p_popcnt popcnt_i (
749     .in_i (operand_a_i),
750     .result_o(cnt_result)
751 );
752
753 always_comb begin
754     ff_input = '0;
755
756     case (operator_i)
757         ALU_FF1: ff_input = operand_a_i;
758         ALU_DIVU, ALU_REMU, ALU_FL1: ff_input = operand_a_rev;
759         ALU_DIV, ALU_REM, ALU_CLB: begin
760             if (operand_a_i[31]) ff_input = operand_a_neg_rev;
761             else ff_input = operand_a_rev;
762         end
763     endcase
764 end
765
766 cv32e40p_ff_one ff_one_i (
767     .in_i (ff_input),
768     .first_one_o(ff1_result),
769     .no_ones_o (ff_no_one)
770 )

```

```

771 shuffle_r0_in[15: 8] : shuffle_r0_in[ 7: 0]);
772 assign shuffle_r0[ 7: 0] = shuffle_byte_sel[0][1] ?
773     (shuffle_byte_sel[0][0] ?
774 shuffle_r0_in[31:24] : shuffle_r0_in[23:16]) :
775     (shuffle_byte_sel[0][0] ?
776 shuffle_r0_in[15: 8] : shuffle_r0_in[ 7: 0]);
777
778 assign shuffle_r1[31:24] = shuffle_byte_sel[3][1] ?
779     (shuffle_byte_sel[3][0] ?
780 shuffle_r1_in[31:24] : shuffle_r1_in[23:16]) :
781     (shuffle_byte_sel[3][0] ?
782 shuffle_r1_in[15: 8] : shuffle_r1_in[ 7: 0]);
783 assign shuffle_r1[23:16] = shuffle_byte_sel[2][1] ?
784     (shuffle_byte_sel[2][0] ?
785 shuffle_r1_in[31:24] : shuffle_r1_in[23:16]) :
786     (shuffle_byte_sel[2][0] ?
787 shuffle_r1_in[15: 8] : shuffle_r1_in[ 7: 0]);
788 assign shuffle_r1[15: 8] = shuffle_byte_sel[1][1] ?
789     (shuffle_byte_sel[1][0] ?
790 shuffle_r1_in[31:24] : shuffle_r1_in[23:16]) :
791     (shuffle_byte_sel[1][0] ?
792 shuffle_r1_in[15: 8] : shuffle_r1_in[ 7: 0]);
793 assign shuffle_r1[7: 0] = shuffle_byte_sel[0][1] ?
794     (shuffle_byte_sel[0][0] ?
795 shuffle_r1_in[31:24] : shuffle_r1_in[23:16]) :
796     (shuffle_byte_sel[0][0] ?
797 shuffle_r1_in[15: 8] : shuffle_r1_in[ 7: 0]);
798
799 assign shuffle_result[31:24] = shuffle_reg_sel[3] ?
800     shuffle_r1[31:24] : shuffle_r0[31:24];
801 assign shuffle_result[23:16] = shuffle_reg_sel[2] ?
802     shuffle_r1[23:16] : shuffle_r0[23:16];
803 assign shuffle_result[15:8] = shuffle_reg_sel[1] ?
804     shuffle_r1[15:8] : shuffle_r0[15:8];
805 assign shuffle_result[7:0] = shuffle_reg_sel[0] ?
806     shuffle_r1[7:0] : shuffle_r0[7:0];
807
808 assign pack_result[31:24] = shuffle_through[3] ?
809     shuffle_result[31:24] : operand_c_i[31:24];
810 assign pack_result[23:16] = shuffle_through[2] ?
811     shuffle_result[23:16] : operand_c_i[23:16];
812 assign pack_result[15:8] = shuffle_through[1] ?
813     shuffle_result[15:8] : operand_c_i[15:8];
814 assign pack_result[7:0] = shuffle_through[0] ?
815     shuffle_result[7:0] : operand_c_i[7:0];
816
817 //////////////////////////////////////////
818 //////////////////////////////////////////
819 //////////////////////////////////////////
820 //////////////////////////////////////////
821
822 logic [31:0] ff_input; // either op_a_i or its bit reversed
823 version
824 logic [ 5:0] cnt_result; // population count
825 logic [ 5:0] clb_result; // count leading bits
826 logic [ 4:0] ff1_result; // holds the index of the first
827 '1'
828 logic ff_no_one; // if no ones are found
829 logic [ 4:0] fl1_result; // holds the index of the last '1'
830 logic [ 5:0] bitop_result; // result of all bitop opera-
831 tions muxed together
832
833 cv32e40p_popcnt popcnt_i (
834     .in_i (operand_a_i),
835     .result_o(cnt_result)
836 );
837
838 always_comb begin
839     ff_input = '0;
840
841     case (operator_i)
842         ALU_FF1: ff_input = operand_a_i;
843         ALU_DIVU, ALU_REMU, ALU_FL1: ff_input = operand_a_rev;
844         ALU_DIV, ALU_REM, ALU_CLB: begin
845             if (operand_a_i[31]) ff_input = operand_a_neg_rev;
846             else ff_input = operand_a_rev;
847         end
848     endcase
849 end
850
851 cv32e40p_ff_one ff_one_i (
852     .in_i (ff_input),
853     .first_one_o(ff1_result),
854     .no_ones_o (ff_no_one)
855 )

```

```

768 );
769 // special case if ff1_res is 0 (no 1 found), then we keep
the 0
771 // this is done in the result mux
772 assign fl1_result = 5'd31 - ff1_result;
773 assign clb_result = ff1_result - 5'd1;
774
775 always_comb begin
776     bitop_result = '0;
777     case (operator_i)
778         ALU_FF1: bitop_result = ff_no_one ? 6'd32 : {1'b0,
ff1_result};
779         ALU_FL1: bitop_result = ff_no_one ? 6'd32 : {1'b0,
fl1_result};
780         ALU_CNT: bitop_result = cnt_result;
781         ALU_CLB: begin
782             if (ff_no_one) begin
783                 if (operand_a_i[31]) bitop_result = 6'd31;
784                 else bitop_result = '0;
785             end else begin
786                 bitop_result = clb_result;
787             end
788         end
789         default: ;
790     endcase
791 end
792
793 ///////////////////////////////////////////////////
794 //
795 //
796 //
797 //
798 //
799 //
800 //
801 //
802
803 logic extract_is_signed;
804 logic extract_sign;
805 logic [31:0] bmask_first, bmask_inv;
806 logic [31:0] bextins_and;
807 logic [31:0] bextins_result, bclr_result, bset_result;
808
809 // construct bit mask for insert/extract/bclr/bset
810 // bmask looks like this 00..0011..1100..00
811 assign bmask_first = {32'hFFFFFFFE} << bmask_a_i;
812 assign bmask = (~bmask_first) << bmask_b_i;
813 assign bmask_inv = ~bmask;
814
815 assign bextins_and = (operator_i == ALU_BINS) ? oper-
and_c_i : {32{extract_sign}};
816
817 assign extract_is_signed = (operator_i == ALU_BEXT);
818 assign extract_sign = extract_is_signed &
shift_result[bmask_a_i];
819
820 assign bextins_result = (bmask & shift_result) |
(bextins_and & bmask_inv);
821
822 assign bclr_result = operand_a_i & bmask_inv;
823 assign bset_result = operand_a_i | bmask;
824
825 ///////////////////////////////////////////////////
826 //
827 //
828 //
829 //
830 //
831 //
832 //
833 //
834 //
835
836 logic [31:0] radix_2_rev;
837 logic [31:0] radix_4_rev;
838 logic [31:0] radix_8_rev;
839 logic [31:0] reverse_result;
840 logic [1:0] radix_mux_sel;
841
842 assign radix_mux_sel = bmask_a_i[1:0];
843
844 generate
845     // radix-2 bit reverse
846     for (j = 0; j < 32; j++) begin : gen_radix_2_rev
847         assign radix_2_rev[j] = shift_result[31-j];
848     end
849     // radix-4 bit reverse
850     for (j = 0; j < 16; j++) begin : gen_radix_4_rev
851         assign radix_4_rev[2*j+1:2*j] = shift_result[31-j*2:31-

```

```

j*2-1];
852 end
853 // radix-8 bit reverse
854 for (j = 0; j < 10; j++) begin : gen_radix_8_rev
855     assign radix_8_rev[3*j+2:3*j] = shift_result[31-j*3:31-
j*3-2];
856 end
857 assign radix_8_rev[31:30] = 2'b0;
858 endgenerate
859
860 always_comb begin
861     reverse_result = '0;
862
863     unique case (radix_mux_sel)
864         2'b00: reverse_result = radix_2_rev;
865         2'b01: reverse_result = radix_4_rev;
866         2'b10: reverse_result = radix_8_rev;
867
868         default: reverse_result = radix_2_rev;
869     endcase
870 end
871
872 ///////////////////////////////////////////////////
873 //
874 //
875 //
876 //
877 //
878 //
879 //
880
881 logic [31:0] result_div;
882 logic div_ready;
883 logic div_signed;
884 logic div_op_a_signed;
885 logic [5:0] div_shift_int;
886
887 assign div_signed = operator_i[0];
888
889 assign div_op_a_signed = operand_a_i[31] & div_signed;
890
891 assign div_shift_int = ff_no_one ? 6'd31 : clb_result;
892
893 assign div_shift = div_shift_int + (div_op_a_signed ? 6'd0 :
6'd1);
894
895 assign div_valid = enable_i & ((operator_i == ALU_DIV) ||
(operator_i == ALU_DIVU) ||
(operator_i == ALU_REM) || (operator_i ==
ALU_REMU));
896
897 // inputs A and B are swapped
898 cv32e40p_alu_div alu_div_i (
899     .Clk_CI (clk),
900     .Rst_RBI(rst_n),
901
902     // input IF
903     .OpA_DI (operand_b_i),
904     .OpB_DI (shift_left_result),
905     .OpBShift_DI (div_shift),
906     .OpBIsZero_SI((cnt_result == 0)),
907
908     .OpBSign_SI(div_op_a_signed),
909     .OpCode_SI (operator_i[1:0]),
910
911     .Res_D0(result_div),
912
913     // Hand-Shake
914     .Invld_SI (div_valid),
915     .OutRdy_SI(ex_ready_i),
916     .OutVld_S0(div_ready)
917 );
918
919 ///////////////////////////////////////////////////
920 //
921 //
922 //
923 //
924 //
925 //
926 //
927
928 always_comb begin
929     result_o = '0;
930
931     unique case (operator_i)
932         // Standard Operations
933         ALU_AND: result_o = operand_a_i & operand_b_i;
934         ALU_OR: result_o = operand_a_i | operand_b_i;
935         ALU_XOR: result_o = operand_a_i ^ operand_b_i;
936
937         // Shift Operations
938         ALU_ADD, ALU_ADDR, ALU_ADDU, ALU_ADDUR,
939         ALU_SUB, ALU_SUBR, ALU_SUBU, ALU_SUBUR,
940         ALU_SLL,
941         ALU_SRL, ALU_SRA,
942         ALU_ROR:
943             result_o = shift_result;
944
945         // bit manipulation instructions

```

```

946     ALU_BINS, ALU_BEXT, ALU_BEXTU: result_o =
bextins_result;
947
948     ALU_BCLR: result_o = bclr_result;
949     ALU_BSET: result_o = bset_result;
950
951     // Bit reverse instruction
952     ALU_BREV: result_o = reverse_result;
953
954     // pack and shuffle operations
955     ALU_SHUF, ALU_SHUF2, ALU_PCKL0, ALU_PCKHI, ALU_EXT,
ALU_EXTS, ALU_INS: result_o = pack_result;
956
957     // Min/Max/Ins
958     ALU_MIN, ALU_MINU, ALU_MAX, ALU_MAXU: result_o = res-
ult_minmax;
959
960     //Abs/Cplxconj, ABS is used to do 0 - A for Cplxconj
961     ALU_ABS: result_o = is_cplx_i ? {adder_result[31:16],
operand_a_i[15:0]} : result_minmax;
962
963     ALU_CLIP, ALU_CLIPU: result_o = clip_result;
964
965     // Comparison Operations
966     ALU_EQ, ALU_NE, ALU_GTU, ALU_GEU, ALU_LTU, ALU_LEU,
ALU_GTS, ALU_GES, ALU_LTS, ALU_LES: begin
967         result_o[31:24] = {8{cmp_result[3]}};
968         result_o[23:16] = {8{cmp_result[2]}};
969         result_o[15:8] = {8{cmp_result[1]}};
970         result_o[7:0] = {8{cmp_result[0]}};
971     end
972     // Non-vector comparisons
973     ALU_SLTS, ALU_SLTU, ALU_SLETS, ALU_SLETU: result_o =
{31'b0, comparison_result_o};
974
975     ALU_FF1, ALU_FL1, ALU_CLB, ALU_CNT: result_o = {26'h0,
bitop_result[5:0]};
976
977     // Division Unit Commands
978     ALU_DIV, ALU_DIVU, ALU_REM, ALU_REMU: result_o = res-
ult_div;
979
980     default: ; // default case to suppress unique warning
981     endcase
982 end
983
984 assign ready_o = div_ready;
985
986 endmodule

```

3. openhwgroup_cv32e40p/rtl/cv32e40p_alu_div.sv (226 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the License.
10
11 ///////////////////////////////////////////////////////////////////
12 // File           : Simple Serial Divider
13 // Ver            : 1.0
14 // Date           : 15.03.2016
15 ///////////////////////////////////////////////////////////////////
16 //
17 // Description: this is a simple serial divider for signed in-
tegers (int32).
18 //
19 ///////////////////////////////////////////////////////////////////
20 //
21 // Authors       : Michael Schaffner (schaffner@iis.ee.ethz.ch)
22 //                 Andreas Traber   (atraber@iis.ee.ethz.ch)
23 //
24 ///////////////////////////////////////////////////////////////////
25
26 module cv32e40p_alu_div #(
27     parameter C_WIDTH = 32,
28     parameter C_LOG_WIDTH = 6
29 ) (
30     input logic          Clk_CI,
31     input logic          Rst_RBI,
32     // input IF
33     input logic [C_WIDTH-1:0] OpA_DI,
34     input logic [C_WIDTH-1:0] OpB_DI,

```

```

35     input logic [C_LOG_WIDTH-1:0] OpBShift_DI,
36     input logic          OpBIsZero_SI,
37     //
38     input logic          OpBSign_SI, // gate this
to 0 in case of unsigned ops
39     input logic [C_WIDTH-1:0] OpCode_SI, // 0: udiv, 2:
urem, 1: div, 3: rem
40     // handshake
41     input logic          Invld_SI,
42     // output IF
43     input logic          OutRdy_SI,
44     output logic         OutVld_S0,
45     output logic [C_WIDTH-1:0] Res_D0
46 );
47
48 ///////////////////////////////////////////////////////////////////
49 // signal declarations
50 ///////////////////////////////////////////////////////////////////
51
52 logic [C_WIDTH-1:0] ResReg_DP, ResReg_DN;
53 logic [C_WIDTH-1:0] ResReg_DP_rev;
54 logic [C_WIDTH-1:0] AReg_DP, AReg_DN;
55 logic [C_WIDTH-1:0] BReg_DP, BReg_DN;
56
57 logic RemSel_SN, RemSel_SP;
58 logic CompInv_SN, CompInv_SP;
59 logic ResInv_SN, ResInv_SP;
60
61 logic [C_WIDTH-1:0] AddMux_D;
62 logic [C_WIDTH-1:0] AddOut_D;
63 logic [C_WIDTH-1:0] AddTmp_D;
64 logic [C_WIDTH-1:0] BMux_D;
65 logic [C_WIDTH-1:0] OutMux_D;
66
67 logic [C_LOG_WIDTH-1:0] Cnt_DP, Cnt_DN;
68 logic CntZero_S;
69
70 logic ARegEn_S, BRegEn_S, ResRegEn_S, ABComp_S, PmSel_S,
LoadEn_S;
71
72 enum logic [1:0] {
73     IDLE,
74     DIVIDE,
75     FINISH
76 }
77
78 State_SN, State_SP;
79
80 ///////////////////////////////////////////////////////////////////
81 // datapath
82 ///////////////////////////////////////////////////////////////////
83
84 assign PmSel_S = LoadEn_S & ~(OpCode_SI[0] &
(OpA_DI[high(OpA_DI)] ^ OpBSign_SI));
85
86 // muxes
87 assign AddMux_D = (LoadEn_S) ? OpA_DI : BReg_DP;
88
89 // attention: logical shift in case of negative operand B!
90 assign BMux_D = (LoadEn_S) ? OpB_DI : {CompInv_SP,
(BReg_DP[high(BReg_DP):1])};
91
92 genvar index;
93 generate
94     for (index = 0; index < C_WIDTH; index++) begin :
gen_bit_swapping
95         assign ResReg_DP_rev[index] = ResReg_DP[C_WIDTH-1-in-
dex];
96     end
97 endgenerate
98
99 assign OutMux_D = (RemSel_SP) ? AReg_DP : ResReg_DP_rev;
100
101 // invert if necessary
102 assign Res_D0 = (ResInv_SP) ? ~$signed(OutMux_D) : OutMux_D;
103
104 // main comparator
105 assign ABComp_S = ((AReg_DP == BReg_DP) | ((AReg_DP >
BReg_DP) ^ CompInv_SP)) & ((AReg_DP | OpBIsZero_SI);
106
107 // main adder
108 assign AddTmp_D = (LoadEn_S) ? 0 : AReg_DP;
109
110 assign AddOut_D = (PmSel_S) ? AddTmp_D + AddMux_D : AddTmp_D -
$signed(AddMux_D);
111
112 // counter
113 ///////////////////////////////////////////////////////////////////
114
115 assign Cnt_DN = (LoadEn_S) ? OpBShift_DI : (~CntZero_S) ?
Cnt_DP - 1 : Cnt_DP;
116
117 assign CntZero_S = ~(Cnt_DP);
118

```

```

119 //////////////////////////////////////////////////
120 // FSM
121 //////////////////////////////////////////////////
122
123 always_comb begin : p_fsm
124     // default
125     State_SN = State_SP;
126
127     OutVld_S0 = 1'b0;
128
129     LoadEn_S = 1'b0;
130
131     ARegEn_S = 1'b0;
132     BRegEn_S = 1'b0;
133     ResRegEn_S = 1'b0;
134
135     case (State_SP)
136     //////////////////////////////////////////////////
137     IDLE: begin
138         OutVld_S0 = 1'b1;
139
140         if (InVld_SI) begin
141             OutVld_S0 = 1'b0;
142             ARegEn_S = 1'b1;
143             BRegEn_S = 1'b1;
144             LoadEn_S = 1'b1;
145             State_SN = DIVIDE;
146         end
147     end
148     //////////////////////////////////////////////////
149     DIVIDE: begin
150
151         ARegEn_S = ABComp_S;
152         BRegEn_S = 1'b1;
153         ResRegEn_S = 1'b1;
154
155         // calculation finished
156         // one more divide cycle (32nd divide cycle)
157         if (CntZero_S) begin
158             State_SN = FINISH;
159         end
160     end
161     //////////////////////////////////////////////////
162     FINISH: begin
163         OutVld_S0 = 1'b1;
164
165         if (OutRdy_SI) begin
166             State_SN = IDLE;
167         end
168     end
169     //////////////////////////////////////////////////
170     default: /* default */;
171     //////////////////////////////////////////////////
172 endcase
173 end
174
175 //////////////////////////////////////////////////
176 //////////////////////////////////////////////////
177 // regs
178 //////////////////////////////////////////////////
179
180 // get flags
181 assign RemSel_SN = (LoadEn_S) ? OpCode_SI[1] : RemSel_SP;
182 assign CompInv_SN = (LoadEn_S) ? OpBSign_SI : CompInv_SP;
183 assign ResInv_SN = (LoadEn_S) ? (~OpBIsZero_SI | Op-
Code_SI[1]) & OpCode_SI[0] & (OpA_DI[Shigh(
184     OpA_DI
185     ]) ^ OpBSign_SI) : ResInv_SP;
186
187 assign AReg_DN = (ARegEn_S) ? AddOut_D : AReg_DP;
188 assign BReg_DN = (BRegEn_S) ? BMux_D : BReg_DP;
189 assign ResReg_DN = (LoadEn_S) ? '0 : (ResRegEn_S) ? {
190     ABComp_S, ResReg_DP[Shigh(ResReg_DP):1]
191 } : ResReg_DP;
192
193 always_ff @(posedge Clk_CI or negedge Rst_RBI) begin :
p_regs
194     if (~Rst_RBI) begin
195         State_SP <= IDLE;
196         AReg_DP <= '0;
197         BReg_DP <= '0;
198         ResReg_DP <= '0;
199         Cnt_DP <= '0;
200         RemSel_SP <= 1'b0;
201         CompInv_SP <= 1'b0;
202         ResInv_SP <= 1'b0;
203     end else begin
204         State_SP <= State_SN;
205         AReg_DP <= AReg_DN;
206         BReg_DP <= BReg_DN;
207         ResReg_DP <= ResReg_DN;
208         Cnt_DP <= Cnt_DN;
209         RemSel_SP <= RemSel_SN;
210         CompInv_SP <= CompInv_SN;
211         ResInv_SP <= ResInv_SN;
212     end
213 end

```

```

214
215 //////////////////////////////////////////////////
216 // assertions
217 //////////////////////////////////////////////////
218
219 `ifndef CV32E40P_ASSERT_ON
220     initial begin : p_assertions
221         assert (C_LOG_WIDTH == $clog2(C_WIDTH + 1))
222         else $error("C_LOG_WIDTH must be $clog2(C_WIDTH+1)");
223     end
224 `endif
225
226 endmodule // serDiv

```

4. openhwgroup_cv32e40p/rtl/cv32e40p_apu_disp.sv (242 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the License.
10
11 //////////////////////////////////////////////////
12 // Engineer:          Lukas Mueller - lukas-
mue@student.ethz.ch //
13 //
14 // Additional contributions
15 //
16 //
17 // Design Name:      Simple APU dispatch-
er //
18 // Project Name:
PULP //
19 // Language:         SystemVeri-
log //
20 //
21 // Description:      Dispatcher for sending instructions to the
Marx //
22 //                  intercon-
nect. //
23 //////////////////////////////////////////////////
24
25 module cv32e40p_apu_disp (
26     input logic clk_i,
27     input logic rst_ni,
28
29     // request input
30     input logic enable_i,
31     input logic [1:0] apu_lat_i,
32     input logic [5:0] apu_waddr_i,
33
34     // response output
35     output logic [5:0] apu_waddr_o,
36     output logic apu_multicycle_o,
37     output logic apu_singlecycle_o,
38
39     // active signal
40     output logic active_o,
41
42     // stall signals
43     output logic stall_o,
44
45     // dependency checks
46     input logic is_decoding_i,
47     input logic [2:0][5:0] read_regs_i,
48     input logic [2:0] read_regs_valid_i,
49     output logic read_dep_o,
50     output logic read_dep_for_jalr_o,
51
52     input logic [1:0][5:0] write_regs_i,
53     input logic [1:0] write_regs_valid_i,
54     output logic write_dep_o,
55
56     // perf counter stuff

```

```

57 output logic perf_type_o,
58 output logic perf_cont_o,
59
60 // apu-interconnect
61 // handshake signals
62 output logic apu_req_o,
63 input logic apu_gnt_i,
64 // response channel
65 input logic apu_rvalid_i
66 );
67
68 logic [5:0] addr_req, addr_inflight, addr_waiting;
69 logic [5:0] addr_inflight_dn, addr_waiting_dn;
70 logic valid_req, valid_inflight, valid_waiting;
71 logic valid_inflight_dn, valid_waiting_dn;
72 logic returned_req, returned_inflight, returned_waiting;
73
74 logic req_accepted;
75 logic active;
76 logic [1:0] apu_lat;
77
78
79 logic [2:0] read_deps_req, read_deps_inflight,
read_deps_waiting;
81 logic [1:0] write_deps_req, write_deps_inflight,
write_deps_waiting;
82 logic read_dep_req, read_dep_inflight, read_dep_waiting;
83 logic write_dep_req, write_dep_inflight, write_dep_waiting;
84
85 logic stall_full, stall_type, stall_nack;
86
87 // Generate request signal; do not generate request if
stalled unless it's a nack stall
88 assign valid_req = enable_i & !(stall_full |
stall_type);
89 assign addr_req = apu_waddr_i;
90
91 assign req_accepted = valid_req & apu_gnt_i;
92
93 //
94 // In-flight instructions
95 //
96 // Check whether the instructions have returned
97 assign returned_req = valid_req & apu_rvalid_i & !val-
id_inflight & !valid_waiting;
98 assign returned_inflight = valid_inflight & (apu_rvalid_i
& !valid_waiting;
99 assign returned_waiting = valid_waiting & (apu_rvalid_i);
100
101 // Inflight and waiting registers
102 always_ff @(posedge clk_i or negedge rst_ni) begin
103 if (~rst_ni) begin
104 valid_inflight <= 1'b0;
105 valid_waiting <= 1'b0;
106 addr_inflight <= '0;
107 addr_waiting <= '0;
108 end else begin
109 valid_inflight <= valid_inflight_dn;
110 valid_waiting <= valid_waiting_dn;
111 addr_inflight <= addr_inflight_dn;
112 addr_waiting <= addr_waiting_dn;
113 end
114 end
115
116 always_comb begin
117 valid_inflight_dn = valid_inflight;
118 valid_waiting_dn = valid_waiting;
119 addr_inflight_dn = addr_inflight;
120 addr_waiting_dn = addr_waiting;
121
122 if (req_accepted & !returned_req) begin // this is a mul-
ticycle request
123 valid_inflight_dn = 1'b1;
124 addr_inflight_dn = addr_req;
125 if (valid_inflight & !(returned_inflight)) begin // we
already have an inflight instruction!
126 valid_waiting_dn = 1'b1;
127 addr_waiting_dn = addr_inflight;
128 end
129 if (returned_waiting) begin // we have received a new
request and waiting goes out of the pipe but will be refilled
130 valid_waiting_dn = 1'b1;
131 addr_waiting_dn = addr_inflight;
132 end
133 end // no new request
134 else if (returned_inflight) begin // multicycle request
has returned
135 valid_inflight_dn = '0;
136 valid_waiting_dn = '0;
137 addr_inflight_dn = '0;
138 addr_waiting_dn = '0;
139 end else if (returned_waiting) begin // multicycle re-
quest has returned
140 valid_waiting_dn = '0;
141 addr_waiting_dn = '0;
142 end
143 end
144
145 //
146 // Active type

```

```

147 //
148 // Dispatcher is active when there is an unreturned instruc-
tion
149 assign active = valid_inflight | valid_waiting;
150
151 // Store the latency type whenever there is a request
152 always_ff @(posedge clk_i or negedge rst_ni) begin
153 if (~rst_ni) begin
154 apu_lat <= '0;
155 end else begin
156 if (valid_req) begin
157 apu_lat <= apu_lat_i;
158 end
159 end
160 end
161
162 //
163 // Dependency checks
164 //
165 // There is a dependency if the register is equal to one of
the instructions
166 generate
167 for (genvar i = 0; i < 3; i++) begin : gen_read_deps
168 assign read_deps_req[i] = (read_regs_i[i] ==
addr_req) & read_regs_valid_i[i];
169 assign read_deps_inflight[i] = (read_regs_i[i] ==
addr_inflight) & read_regs_valid_i[i];
170 assign read_deps_waiting[i] = (read_regs_i[i] ==
addr_waiting) & read_regs_valid_i[i];
171 end
172 endgenerate
173
174 generate
175 for (genvar i = 0; i < 2; i++) begin : gen_write_deps
176 assign write_deps_req[i] = (write_regs_i[i] ==
addr_req) & write_regs_valid_i[i];
177 assign write_deps_inflight[i] = (write_regs_i[i] ==
addr_inflight) & write_regs_valid_i[i];
178 assign write_deps_waiting[i] = (write_regs_i[i] ==
addr_waiting) & write_regs_valid_i[i];
179 end
180 endgenerate
181
182 // Reduce the individual dependency signals into one read
and one write dependency
183 assign read_dep_req = |read_deps_req & valid_req & !re-
turned_req;
184 assign read_dep_inflight = |read_deps_inflight & val-
id_inflight & !returned_inflight;
185 assign read_dep_waiting = |read_deps_waiting & valid_waiting
& !returned_waiting;
186 assign write_dep_req = |write_deps_req & valid_req & !re-
turned_req;
187 assign write_dep_inflight = |write_deps_inflight & val-
id_inflight & !returned_inflight;
188 assign write_dep_waiting = |write_deps_waiting & val-
id_waiting & !returned_waiting;
189
190 assign read_dep_o = (read_dep_req | read_dep_inflight |
read_dep_waiting) & is_decoding_i;
191 assign write_dep_o = (write_dep_req | write_dep_inflight |
write_dep_waiting) & is_decoding_i;
192
193 assign read_dep_for_jalr_o = is_decoding_i & ((|
read_deps_req & enable_i) |
194
195 read_deps_inflight & valid_inflight) |
196
197 read_deps_waiting & valid_waiting));
197 //
198 // Stall signals
199 //
200 // Stall if we cannot store any more outstanding requests
201 assign stall_full = valid_inflight & valid_waiting;
202 // Stall if there is a type conflict. if apu is active we
can only issue requests with a larger or equal latency
203 // than the latency of the inflight operation
(apu_lat_i >= apu_lat). otherwise operations would overtake each oth-
er!
204 // so we stall if: (apu_lat_i = 1 & apu_lat = 2/3) |
(apu_lat_i = 2 & apu_lat = 3) | (apu_lat_i = 3 (multicycle))
205 assign stall_type = enable_i & active &
((apu_lat_i == 2'h1) | ((apu_lat_i == 2'h2) & (apu_lat == 2'h3)) |
(apu_lat_i == 2'h3));
206 assign stall_nack = valid_req & !apu_gnt_i;
207 assign stall_o = stall_full | stall_type | stall_nack;
208
209 //
210 // Generate Apu_master request
211 //
212 assign apu_req_o = valid_req;
213
214
215 // Determine write register based on where the instruction
returned.
216 always_comb begin
217 apu_waddr_o = '0;
218 if (returned_req) apu_waddr_o = addr_req;
219 if (returned_inflight) apu_waddr_o = addr_inflight;

```

```

220     if (returned_waiting) apu_waddr_o = addr_waiting;
221 end
222
223 // Output active signal
224 assign active_o = active;
225
226 // Performance counter signals
227 assign perf_type_o = stall_type;
228 assign perf_cont_o = stall_nack;
229
230 assign apu_multicycle_o = (apu_lat == 2'h3);
231 assign apu_singlecycle_o = ~(valid_inflight | val-
id_waiting);
232
233 //
234 // Assertions
235 //
236
237 `ifdef CV32E40P_ASSERT_ON
238     assert property (@(posedge clk_i) (apu_rvalid_i) |-> (val-
id_req | valid_inflight | valid_waiting))
239     else $warning("[APU Dispatcher] instruction returned while
no instruction is in-flight");
240 `endif
241
242 endmodule

```

5. openhwgroup_cv32e40p/rtl/ cv32e40p_compressed_decoder.sv (601 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the License.
10
11 ///////////////////////////////////////////////////////////////////
12 // Engineer:      Sven Stucki - svs-
tucki@student.ethz.ch
13 //
14 // Additional contributions
15 //
16 //
17 // Design Name:    Compressed instruction de-
coder
18 // Project Name:
19 // Language:      SystemVeri-
log
20 //
21 // Description:    Decodes RISC-V compressed instructions into
their RV32
22 //
23 //
24 //
25 //
26 //
27 module cv32e40p_compressed_decoder #(
28     parameter FPU = 0,
29     parameter ZFINX = 0
30 ) (
31     input logic [31:0] instr_i,
32     output logic [31:0] instr_o,
33     output logic is_compressed_o,
34     output logic illegal_instr_o
35 );
36
37 import cv32e40p_pkg::*;
38
39 ///////////////////////////////////////////////////////////////////
40 //

```

```

41 //
42 //
43 //
44 //
45 //
46 ///////////////////////////////////////////////////////////////////
47
48 always_comb begin
49     illegal_instr_o = 1'b0;
50     instr_o = '0;
51
52     unique case (instr_i[1:0])
53         // C0
54         2'b00: begin
55             unique case (instr_i[15:13])
56                 3'b000: begin
57                     // c.addi4spn -> addi rd', x2, imm
58                     instr_o = {
59                         2'b0,
60                         instr_i[10:7],
61                         instr_i[12:11],
62                         instr_i[5],
63                         instr_i[6],
64                         2'b00,
65                         5'h02,
66                         3'b000,
67                         2'b01,
68                         instr_i[4:2],
69                         OPCODE_OPIMM
70                     };
71                     if (instr_i[12:5] == 8'b0) illegal_instr_o = 1'b1;
72                 end
73                 3'b001: begin
74                     // c.fld -> fld rd', imm(rs1')
75                     if (FPU == 1 && ZFINX == 0)
76                         // instr_i[12:10]-> offset[5:3], instr_i[6:5]->
offset[7:6]
77                     instr_o = {
78                         4'b0,
79                         instr_i[6:5],
80                         instr_i[12:10],
81                         3'b000,
82                         2'b01,
83                         instr_i[9:7],
84                         3'b011,
85                         2'b01,
86                         instr_i[4:2],
87                         OPCODE_LOAD_FP
88                     };
89                     else illegal_instr_o = 1'b1;
90                 end
91                 3'b010: begin
92                     // c.lw -> lw rd', imm(rs1')
93                     instr_o = {
94                         5'b0,
95                         instr_i[5],
96                         instr_i[12:10],
97                         instr_i[6],
98                         2'b00,
99                         2'b01,
100                        instr_i[9:7],
101                        3'b010,
102                        2'b01,
103                        instr_i[4:2],
104                        OPCODE_LOAD
105                    };
106                 end
107                 3'b011: begin
108                     // c.flw -> flw rd', imm(rs1')
109                     if (FPU == 1 && ZFINX == 0)
110                     instr_o = {
111                         5'b0,
112                         instr_i[5],
113                         instr_i[12:10],
114                         instr_i[6],
115                         2'b00,
116                         2'b01,
117                         instr_i[9:7],
118                         3'b010,
119                         2'b01,
120                         instr_i[4:2],
121                         OPCODE_LOAD_FP
122                     };
123                     else illegal_instr_o = 1'b1;
124                 end
125             end
126         end
127     end

```



```

128
129 3'b101: begin
130 // c.fsd -> fsd rs2', imm(rs1')
131 if (FPU == 1 && ZFINX == 0)
132 // instr_i[12:10] -> offset[5:3], instr_i[6:5] -
> offset[7:6]
133 instr_o = {
134 4'b0,
135 instr_i[6:5],
136 instr_i[12],
137 2'b01,
138 instr_i[4:2],
139 2'b01,
140 instr_i[9:7],
141 3'b011,
142 instr_i[11:10],
143 3'b000,
144 OPCODE_STORE_FP
145 };
146 else illegal_instr_o = 1'b1;
147 end
148
149 3'b110: begin
150 // c.sw -> sw rs2', imm(rs1')
151 instr_o = {
152 5'b0,
153 instr_i[5],
154 instr_i[12],
155 2'b01,
156 instr_i[4:2],
157 2'b01,
158 instr_i[9:7],
159 3'b010,
160 instr_i[11:10],
161 instr_i[6],
162 2'b00,
163 OPCODE_STORE
164 };
165 end
166
167 3'b111: begin
168 // c.fsw -> fsw rs2', imm(rs1')
169 if (FPU == 1 && ZFINX == 0)
170 instr_o = {
171 5'b0,
172 instr_i[5],
173 instr_i[12],
174 2'b01,
175 instr_i[4:2],
176 2'b01,
177 instr_i[9:7],
178 3'b010,
179 instr_i[11:10],
180 instr_i[6],
181 2'b00,
182 OPCODE_STORE_FP
183 };
184 else illegal_instr_o = 1'b1;
185 end
186 default: begin
187 illegal_instr_o = 1'b1;
188 end
189 endcase
190 end
191
192 // C1
193 2'b01: begin
194 unique case (instr_i[15:13])
195 3'b000: begin
196 // c.addi -> addi rd, rd, nzimm
197 // c.nop
198 instr_o = {
199 {6{instr_i[12]}},
200 instr_i[12],
201 instr_i[6:2],
202 instr_i[11:7],
203 3'b0,
204 instr_i[11:7],
205 OPCODE_OPIMM
206 };
207 end
208
209 3'b001, 3'b101: begin
210 // 001: c.jal -> jal x1, imm
211 // 101: c.j -> jal x0, imm
212 instr_o = {
213 instr_i[12],
214 instr_i[8],
215 instr_i[10:9],
216 instr_i[6],
217 instr_i[7],
218 instr_i[2],
219 instr_i[11],
220 instr_i[5:3],
221 {9{instr_i[12]}},
222 4'b0,
223 ~instr_i[15],
224 OPCODE_JAL
225 };
226 end
227

```

```

228
229 3'b010: begin
230 if (instr_i[11:7] == 5'b0) begin
231 // Hint -> addi x0, x0, nzimm
232 instr_o = {
233 {6{instr_i[12]}}, instr_i[12], instr_i[6:2],
234 5'b0, 3'b0, instr_i[11:7], OPCODE_OPIMM
235 };
236 end else begin
237 // c.li -> addi rd, x0, nzimm
238 instr_o = {
239 {6{instr_i[12]}}, instr_i[12], instr_i[6:2],
240 5'b0, 3'b0, instr_i[11:7], OPCODE_OPIMM
241 };
242 end
243
244 3'b011: begin
245 if ({instr_i[12], instr_i[6:2]} == 6'b0) begin
246 illegal_instr_o = 1'b1;
247 end else begin
248 if (instr_i[11:7] == 5'h02) begin
249 // c.addi16sp -> addi x2, x2, nzimm
250 instr_o = {
251 {3{instr_i[12]}},
252 instr_i[4:3],
253 instr_i[5],
254 instr_i[2],
255 instr_i[6],
256 4'b0,
257 5'h02,
258 3'b000,
259 5'h02,
260 OPCODE_OPIMM
261 };
262 end else if (instr_i[11:7] == 5'b0) begin
263 // Hint -> lui x0, imm
264 instr_o = {{15{instr_i[12]}}, instr_i[6:2],
265 instr_i[11:7], OPCODE_LUI};
266 end else begin
267 // c.lui -> lui rd, imm
268 instr_o = {{15{instr_i[12]}}, instr_i[6:2],
269 instr_i[11:7], OPCODE_LUI};
270 end
271 end
272
273 3'b100: begin
274 unique case (instr_i[11:10])
275 2'b00, 2'b01: begin
276 // 00: c.srli -> srli rd, rd, shamt
277 // 01: c.srai -> srai rd, rd, shamt
278 if (instr_i[12] == 1'b1) begin
279 // Reserved for future custom extensions
280 (instr_o don't care)
281 instr_o = {
282 1'b0,
283 instr_i[10],
284 5'b0,
285 instr_i[6:2],
286 2'b01,
287 instr_i[9:7],
288 3'b101,
289 2'b01,
290 instr_i[9:7],
291 OPCODE_OPIMM
292 };
293 illegal_instr_o = 1'b1;
294 end else begin
295 if (instr_i[6:2] == 5'b0) begin
296 // Hint
297 instr_o = {
298 1'b0,
299 instr_i[10],
300 5'b0,
301 instr_i[6:2],
302 2'b01,
303 instr_i[9:7],
304 3'b101,
305 2'b01,
306 instr_i[9:7],
307 OPCODE_OPIMM
308 };
309 end else begin
310 instr_o = {
311 1'b0,
312 instr_i[10],
313 5'b0,
314 instr_i[6:2],
315 2'b01,
316 instr_i[9:7],
317 3'b101,
318 2'b01,
319 instr_i[9:7],
320 OPCODE_OPIMM
321 };
322 end
323 end
324
325 2'b10: begin

```



```

324 // c.andi -> andi rd, rd, imm
325 instr_o = {
326     {6{instr_i[12]}},
327     instr_i[12],
328     instr_i[6:2],
329     2'b01,
330     instr_i[9:7],
331     3'b111,
332     2'b01,
333     instr_i[9:7],
334     OP CODE_OPIMM
335 };
336 end
337
338 2'b11: begin
339     unique case ({
340         instr_i[12], instr_i[6:5]
341     })
342     3'b000: begin
343         // c.sub -> sub rd', rd', rs2'
344         instr_o = {
345             2'b01,
346             5'b0,
347             2'b01,
348             instr_i[4:2],
349             2'b01,
350             instr_i[9:7],
351             3'b000,
352             2'b01,
353             instr_i[9:7],
354             OP CODE_OP
355         };
356     end
357
358     3'b001: begin
359         // c.xor -> xor rd', rd', rs2'
360         instr_o = {
361             7'b0,
362             2'b01,
363             instr_i[4:2],
364             2'b01,
365             instr_i[9:7],
366             3'b100,
367             2'b01,
368             instr_i[9:7],
369             OP CODE_OP
370         };
371     end
372
373     3'b010: begin
374         // c.or -> or rd', rd', rs2'
375         instr_o = {
376             7'b0,
377             2'b01,
378             instr_i[4:2],
379             2'b01,
380             instr_i[9:7],
381             3'b110,
382             2'b01,
383             instr_i[9:7],
384             OP CODE_OP
385         };
386     end
387
388     3'b011: begin
389         // c.and -> and rd', rd', rs2'
390         instr_o = {
391             7'b0,
392             2'b01,
393             instr_i[4:2],
394             2'b01,
395             instr_i[9:7],
396             3'b111,
397             2'b01,
398             instr_i[9:7],
399             OP CODE_OP
400         };
401     end
402
403     3'b100, 3'b101, 3'b110, 3'b111: begin
404         // 100: c.subw
405         // 101: c.addw
406         illegal_instr_o = 1'b1;
407     end
408 endcase
409 end
410 endcase
411 end
412
413 3'b110, 3'b111: begin
414     // 0: c.beqz -> beq rs1', x0, imm
415     // 1: c.bnez -> bne rs1', x0, imm
416     instr_o = {
417         {4{instr_i[12]}},
418         instr_i[6:5],
419         instr_i[2],
420         5'b0,
421         2'b01,
422         instr_i[9:7],
423         2'b00,
424         instr_i[13],

```

```

425         instr_i[11:10],
426         instr_i[4:3],
427         instr_i[12],
428         OP CODE_BRANCH
429     };
430 end
431 endcase
432 end
433
434 // C2
435 2'b10: begin
436     unique case (instr_i[15:13])
437     3'b000: begin
438         if (instr_i[12] == 1'b1) begin
439             // Reserved for future extensions (instr_o don't
440             care)
441             instr_o = {7'b0, instr_i[6:2], instr_i[11:7],
442             3'b001, instr_i[11:7], OP CODE_OPIMM};
443             illegal_instr_o = 1'b1;
444         end else begin
445             if ((instr_i[6:2] == 5'b0) || (instr_i[11:7] ==
446             5'b0)) begin
447                 // Hint -> slli rd, rd, shamt
448                 instr_o = {7'b0, instr_i[6:2], instr_i[11:7],
449                 3'b001, instr_i[11:7], OP CODE_OPIMM};
450             end else begin
451                 // c.slli -> slli rd, rd, shamt
452                 instr_o = {7'b0, instr_i[6:2], instr_i[11:7],
453                 3'b001, instr_i[11:7], OP CODE_OPIMM};
454             end
455         end
456     end
457
458     3'b001: begin
459         // c.fldsp -> fld rd, imm(x2)
460         if (FPU == 1 && ZFINX == 0)
461             // instr_i[6:5] -> offset[4:3], instr_i[4:2] ->
462             offset[8:6], instr_i[12] -> offset[5]
463             instr_o = {
464                 3'b0,
465                 instr_i[4:2],
466                 instr_i[12],
467                 instr_i[6:5],
468                 3'b000,
469                 5'h02,
470                 3'b011,
471                 instr_i[11:7],
472                 OP CODE_LOAD_FP
473             };
474         else illegal_instr_o = 1'b1;
475     end
476
477     3'b010: begin
478         // c.lwsp -> lw rd, imm(x2)
479         instr_o = {
480             4'b0,
481             instr_i[3:2],
482             instr_i[12],
483             instr_i[6:4],
484             2'b00,
485             5'h02,
486             3'b010,
487             instr_i[11:7],
488             OP CODE_LOAD
489         };
490         if (instr_i[11:7] == 5'b0) illegal_instr_o = 1'b1;
491     end
492
493     3'b011: begin
494         // c.flwsp -> flw rd, imm(x2)
495         if (FPU == 1 && ZFINX == 0)
496             instr_o = {
497                 4'b0,
498                 instr_i[3:2],
499                 instr_i[12],
500                 instr_i[6:4],
501                 2'b00,
502                 5'h02,
503                 3'b010,
504                 instr_i[11:7],
505                 OP CODE_LOAD_FP
506             };
507         else illegal_instr_o = 1'b1;
508     end
509
510     3'b100: begin
511         if (instr_i[12] == 1'b0) begin
512             if (instr_i[6:2] == 5'b0) begin
513                 // c.jr -> jalr x0, rd/rs1, 0
514                 instr_o = {12'b0, instr_i[11:7], 3'b0, 5'b0,
515                 OP CODE_JALR};
516                 // c.jr with rs1 = 0 is reserved
517                 if (instr_i[11:7] == 5'b0) illegal_instr_o =
518                 1'b1;
519             end else begin
520                 if (instr_i[11:7] == 5'b0) begin
521                     // Hint -> add x0, x0, rs2
522                     instr_o = {7'b0, instr_i[6:2], 5'b0, 3'b0,
523                     instr_i[11:7], OP CODE_OP};
524                 end else begin
525

```

```

516          // c.mv -> add rd, x0, rs2
517          instr_o = {7'b0, instr_i[6:2], 5'b0, 3'b0,
instr_i[11:7], OPCODE_OP};
518          end
519          end
520          end else begin
521          if (instr_i[6:2] == 5'b0) begin
522          if (instr_i[11:7] == 5'b0) begin
523          // c.ebreak -> ebreak
524          instr_o = {32'h00_10_00_73};
525          end else begin
526          // c.jalr -> jalr x1, rs1, 0
527          instr_o = {12'b0, instr_i[11:7], 3'b000,
5'b00001, OPCODE_JALR};
528          end
529          end else begin
530          if (instr_i[11:7] == 5'b0) begin
531          // Hint -> add x0, x0, rs2
532          instr_o = {7'b0, instr_i[6:2], in-
str_i[11:7], 3'b0, instr_i[11:7], OPCODE_OP};
533          end else begin
534          // c.add -> add rd, rd, rs2
535          instr_o = {7'b0, instr_i[6:2], in-
str_i[11:7], 3'b0, instr_i[11:7], OPCODE_OP};
536          end
537          end
538          end
539          end
540          end
541          3'b101: begin
542          // c.fsdsp -> fsd rs2, imm(x2)
543          if (FPU == 1 && ZFINX == 0)
544          // instr_i[12:10] -> offset[5:3], instr_i[9:7] -
> offset[8:6]
545          instr_o = {
546          3'b0,
547          instr_i[9:7],
548          instr_i[12],
549          instr_i[6:2],
550          5'h02,
551          3'b011,
552          instr_i[11:10],
553          3'b000,
554          OPCODE_STORE_FP
555          };
556          else illegal_instr_o = 1'b1;
557          end
558          3'b110: begin
559          // c.swsp -> sw rs2, imm(x2)
560          instr_o = {
561          4'b0,
562          instr_i[8:7],
563          instr_i[12],
564          instr_i[6:2],
565          5'h02,
566          3'b010,
567          instr_i[11:9],
568          2'b00,
569          OPCODE_STORE
570          };
571          end
572          end
573          3'b111: begin
574          // c.fwsp -> fsw rs2, imm(x2)
575          if (FPU == 1 && ZFINX == 0)
576          instr_o = {
577          4'b0,
578          instr_i[8:7],
579          instr_i[12],
580          instr_i[6:2],
581          5'h02,
582          3'b010,
583          instr_i[11:9],
584          2'b00,
585          OPCODE_STORE_FP
586          };
587          else illegal_instr_o = 1'b1;
588          end
589          endcase
590          end
591          default: begin
592          // 32 bit (or more) instruction
593          instr_o = instr_i;
594          end
595          endcase
596          end
597          end
598          assign is_compressed_o = (instr_i[1:0] != 2'b11);
599
600
601 endmodule

```

6. openhwgroup_cv32e40p/rtl/cv32e40p_controller.sv (1,623 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this

```

```

file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the License.
10
11 ///////////////////////////////////////////////////////////////////
12 // Engineer:      Matthias Baer - baer-
matt@student.ethz.ch
13 //
14 // Additional contributions
by:
15 //      Igor Loi -
igor.loi@unibo.it
16 //      Andreas Traber - at-
raber@student.ethz.ch
17 //      Sven Stucki - svs-
tucki@student.ethz.ch
18 //      Michael Gautschi - gauts-
chi@iis.ee.ethz.ch
19 //      Davide Schiavone - ps-
chiavo@iis.ee.ethz.ch
20 //      Robert Balas -
balasr@iis.ee.ethz.ch
21 //      Andrea Bettati - an-
drea.bettati@studenti.unipr.it
22 //
23 // Design Name:    Main control-
ler
24 // Project Name:
RISCY
25 // Language:      SystemVeri-
log
26 //
27 // Description:    Main CPU controller of the pro-
cessor
28 //
29 ///////////////////////////////////////////////////////////////////
30
31 module cv32e40p_controller import cv32e40p_pkg::*;
32 #(
33     parameter COREV_CLUSTER = 0,
34     parameter COREV_PULP    = 0,
35     parameter FPU            = 0
36 )
37 (
38     input logic      clk,                // Gated
clock
39     input logic      clk_ungated_i,      // Ungated
clock
40     input logic      rst_n,
41
42     input logic      fetch_enable_i,      // Start the
decoding
43     output logic      ctrl_busy_o,        // Core is
busy processing instructions
44     output logic      is_decoding_o,      // Core is
in decoding state
45     input logic      is_fetch_failed_i,
46
47     // decoder related signals
48     output logic      deassert_we_o,      // deassert
write enable for next instruction
49
50     input logic      illegal_insn_i,      // decoder
encountered an invalid instruction
51     input logic      ecall_insn_i,       // decoder
encountered an ecall instruction
52     input logic      mret_insn_i,        // decoder
encountered an mret instruction
53     input logic      uret_insn_i,        // decoder
encountered an uret instruction
54
55     input logic      dret_insn_i,        // decoder
encountered an dret instruction
56
57     input logic      mret_dec_i,
58     input logic      uret_dec_i,
59     input logic      dret_dec_i,
60
61     input logic      wfi_i,                // decoder
wants to execute a WFI

```

```

62  input logic      ebrk_insn_i,          // decoder
encountered an ebreak instruction
63  input logic      fencei_insn_i,        // decoder
encountered an fencei instruction
64  input logic      csr_status_i,         // decoder
encountered an csr status instruction
65
66  output logic      hwlp_mask_o,          // prevent
writes on the hwloop instructions in case interrupt are taken
67
68  // from IF/ID pipeline
69  input logic      instr_valid_i,         // instruc-
tion coming from IF/ID pipeline is valid
70
71  // from prefetcher
72  output logic      instr_req_o,          // Start
fetching instructions
73
74  // to prefetcher
75  output logic      pc_set_o,             // jump to
address set by pc_mux
76  output logic [3:0] pc_mux_o,           // Selector
in the Fetch stage to select the right PC (normal, jump ...)
77  output logic [2:0] exc_pc_mux_o,       // Selects
target PC for exception
78  output logic [1:0] trap_addr_mux_o,     // Selects
trap address base
79
80  // HWLoop signals
81  input logic [31:0] pc_id_i,
82
83  // from hwloop_regs
84  input logic [1:0] [31:0] hwlp_start_addr_i,
85  input logic [1:0] [31:0] hwlp_end_addr_i,
86  input logic [1:0] [31:0] hwlp_counter_i,
87
88  // to hwloop_regs
89  output logic [1:0] hwlp_dec_cnt_o,
90
91  output logic      hwlp_jump_o,
92  output logic [31:0] hwlp_targ_addr_o,
93
94  // LSU
95  input logic      data_req_ex_i,         // data
memory access is currently performed in EX stage
96  input logic      data_we_ex_i,
97  input logic      data_misaligned_i,
98  input logic      data_load_event_i,
99  input logic      data_err_i,
100 output logic      data_err_ack_o,
101
102 // from ALU
103 input logic      mult_multicycle_i,     // multipli-
er is taken multiple cycles and uses op c as storage
104
105 // APU dependency checks
106 input logic      apu_en_i,
107 input logic      apu_read_dep_i,
108 input logic      apu_read_dep_for_jalr_i,
109 input logic      apu_write_dep_i,
110
111 output logic      apu_stall_o,
112
113 // jump/branch signals
114 input logic      branch_taken_ex_i,     // branch
taken signal from EX ALU
115 input logic [1:0] ctrl_trans-
fer_insn_in_id_i, // jump is being calculated in ALU
116 input logic [1:0] ctrl_trans-
fer_insn_in_dec_i, // jump is being calculated in ALU
117
118 // Interrupt Controller Signals
119 input logic      irq_req_ctrl_i,
120 input logic      irq_sec_ctrl_i,
121 input logic [4:0] irq_id_ctrl_i,
122 input logic      irq_wu_ctrl_i,
123 input PrivLvl_t current_priv_lvl_i,
124
125 output logic      irq_ack_o,
126 output logic [4:0] irq_id_o,
127
128 output logic [4:0] exc_cause_o,
129
130 // Debug Signal
131 output logic      debug_mode_o,
132 output logic [2:0] debug_cause_o,
133 output logic      debug_csr_save_o,
134 input logic      debug_req_i,
135 input logic      debug_single_step_i,
136 input logic      debug_ebreakm_i,
137 input logic      debug_ebreaku_i,
138 input logic      trigger_match_i,
139 output logic      debug_p_elw_no_sleep_o,
140 output logic      debug_wfi_no_sleep_o,
141 output logic      debug_havereset_o,
142 output logic      debug_running_o,
143 output logic      debug_halted_o,
144
145 // Wakeup Signal
146 output logic      wake_from_sleep_o,
147

```

```

148 output logic      csr_save_if_o,
149 output logic      csr_save_id_o,
150 output logic      csr_save_ex_o,
151 output logic [5:0] csr_cause_o,
152 output logic      csr_irq_sec_o,
153 output logic      csr_restore_mret_id_o,
154 output logic      csr_restore_uret_id_o,
155
156 output logic      csr_restore_dret_id_o,
157
158 output logic      csr_save_cause_o,
159
160 // Regfile target
161
162 input logic      regfile_we_id_i,       // currently
decoded we enable
163 input logic [5:0] regfile_alu_waddr_id_i, // currently
decoded target address
164
165 // Forwarding signals from regfile
166 input logic      regfile_we_ex_i,       // FW: write
enable from EX stage
167 input logic [5:0] regfile_waddr_ex_i,   // FW: write
address from EX stage
168 input logic      regfile_we_wb_i,       // FW: write
enable from WB stage
169 input logic      regfile_alu_we_fw_i,   // FW: ALU/
MUL write enable from EX stage
170
171 // forwarding signals
172 output logic [1:0] operand_a_fw_mux_sel_o, // regfile
ra data selector form ID stage
173 output logic [1:0] operand_b_fw_mux_sel_o, // regfile
rb data selector form ID stage
174 output logic [1:0] operand_c_fw_mux_sel_o, // regfile
rc data selector form ID stage
175
176 // forwarding detection signals
177 input logic      reg_d_ex_is_reg_a_i,
178 input logic      reg_d_ex_is_reg_b_i,
179 input logic      reg_d_ex_is_reg_c_i,
180 input logic      reg_d_wb_is_reg_a_i,
181 input logic      reg_d_wb_is_reg_b_i,
182 input logic      reg_d_wb_is_reg_c_i,
183 input logic      reg_d_alu_is_reg_a_i,
184 input logic      reg_d_alu_is_reg_b_i,
185 input logic      reg_d_alu_is_reg_c_i,
186
187 // stall signals
188 output logic      halt_if_o,
189 output logic      halt_id_o,
190
191 output logic      misaligned_stall_o,
192 output logic      jr_stall_o,
193 output logic      load_stall_o,
194
195 input logic      id_ready_i,            // ID stage
is ready
196 input logic      id_valid_i,           // ID stage
is valid
197
198 input logic      ex_valid_i,           // EX stage
is done
199
200 input logic      wb_ready_i,           // WB stage
is ready
201
202 // Performance Counters
203 output logic      perf_pipeline_stall_o // stall due
to cv.elw extra cycles
204 );
205
206 // FSM state encoding
207 ctrl_state_e ctrl_fsm_cs, ctrl_fsm_ns;
208
209 // Debug state
210 debug_state_e debug_fsm_cs, debug_fsm_ns;
211
212 logic jump_done, jump_done_q, jump_in_dec, branch_in_id;
213
214 logic data_err_q;
215
216 logic debug_mode_q, debug_mode_n;
217 logic ebrk_force_debug_mode;
218 logic is_hwlp_body;
219 logic illegal_insn_q, illegal_insn_n;
220 logic debug_req_entry_q, debug_req_entry_n;
221 logic debug_force_wakeup_q, debug_force_wakeup_n;
222
223 logic hwlp_end0_eq_pc;
224 logic hwlp_end1_eq_pc;
225 logic hwlp_counter0_gt_1;
226 logic hwlp_counter1_gt_1;
227 logic hwlp_counter0_eq_1;
228 logic hwlp_counter1_eq_1;

```

```

229 logic hwlp_counter0_eq_0;
230 logic hwlp_counter1_eq_0;
231 logic hwlp_end0_eq_pc_plus4;
232 logic hwlp_end1_eq_pc_plus4;
233 logic hwlp_start0_leq_pc;
234 logic hwlp_start1_leq_pc;
235 logic hwlp_end0_geq_pc;
236 logic hwlp_end1_geq_pc;

237 // Auxiliary signals to make hwlp_jump_o last only one cycle
(converting it into a pulse)
238 logic hwlp_end4_id_d, hwlp_end4_id_q;
239
240 logic debug_req_q;
241 logic debug_req_pending;
242
243 // qualify wfi vs nosleep locally
244 logic wfi_active;
245
246
247 ////////////////////////////////////////
248 // _____ - - - - -
249 // / _/_ \_ \| _\| / _/_ \_ \| \ | _ _| _ \| _ \|
_ \| | | | | _ _| _ \| _ \|
250 // | | | | | | ) | _| | | | | | \ | | | | | ) | | |
| | | | | | | | | |
251 // | | | | | | _ <| _ _ | | | | | | \ | | | | | _ <|
- | | | | | | | | | | _ < //
252 // \_ \| \_ \| \_ \| \_ \| \_ \| \_ \| \_ \| \_ \| \_ \|
\_ \| | | | | | | | | \_ \| _ \|
253 //
254 ////////////////////////////////////////
255
256 always_comb
257 begin
258     // Default values
259
260     instr_req_o          = 1'b1;
261
262     data_err_ack_o       = 1'b0;
263
264     csr_save_if_o        = 1'b0;
265     csr_save_id_o        = 1'b0;
266     csr_save_ex_o        = 1'b0;
267     csr_restore_mret_id_o = 1'b0;
268     csr_restore_uret_id_o = 1'b0;
269
270     csr_restore_dret_id_o = 1'b0;
271
272     csr_save_cause_o      = 1'b0;
273
274     exc_cause_o           = '0;
275     exc_pc_mux_o          = EXC_PC_IRQ;
276     trap_addr_mux_o       = TRAP_MACHINE;
277
278     csr_cause_o           = '0;
279     csr_irq_sec_o         = 1'b0;
280
281     pc_mux_o              = PC_BOOT;
282     pc_set_o              = 1'b0;
283     jump_done             = jump_done_q;
284
285     ctrl_fsm_ns           = ctrl_fsm_cs;
286
287     ctrl_busy_o           = 1'b1;
288
289     halt_if_o             = 1'b0;
290     halt_id_o             = 1'b0;
291     is_decoding_o         = 1'b0;
292     irq_ack_o             = 1'b0;
293     irq_id_o              = 5'b0;
294
295     jump_in_dec           = ctrl_transfer_insn_in_dec_i ==
BRANCH_JALR || ctrl_transfer_insn_in_dec_i == BRANCH_JAL;
296
297     branch_in_id          = ctrl_transfer_insn_in_id_i ==
BRANCH_COND;
298
299     ebrk_force_debug_mode = (debug_ebreakm_i && cur-
rent_priv_lvl_i == PRIV_LVL_M) ||
300                             (debug_ebreakui_i && cur-
rent_priv_lvl_i == PRIV_LVL_U);
301     debug_csr_save_o      = 1'b0;
302     debug_cause_o         = DBG_CAUSE_EBREAK;
303     debug_mode_n          = debug_mode_q;
304
305     illegal_insn_n        = illegal_insn_q;
306     // a trap towards the debug unit is generated when one of
the
307     // following conditions are true:
308     // - ebreak instruction encountered
309     // - single-stepping mode enabled
310     // - illegal instruction exception and IIE bit is set
311     // - IRQ0 and INTE bit is set and no exception is currently

```

```

running
312 // - Debugger requests halt
313
314 debug_req_entry_n      = debug_req_entry_q;
315
316 debug_force_wakeup_n    = debug_force_wakeup_q;
317
318 perf_pipeline_stall_o   = 1'b0;
319
320 hwlp_mask_o             = 1'b0;
321
322 hwlp_dec_cnt_o          = '0;
323 hwlp_end_4_id_d         = 1'b0;
324
325 // When the controller tells to hwlp-jump, the prefetcher
does not always jump immediately,
326 // but the aligner immediately modifies pc_id to HWLP_BE-
GIN. This condition on hwlp_targ_addr_o
327 // ensures that the target is kept constant even if pc_id
is no more HWLP_END
328 hwlp_targ_addr_o        = ((hwlp_start1_leq_pc &&
hwlp_end1_geq_pc) && !(hwlp_start0_leq_pc && hwlp_end0_geq_pc)) ?
hwlp_start_addr_i[1] : hwlp_start_addr_i[0];
329
330 unique case (ctrl_fsm_cs)
331 // We were just reset, wait for fetch_enable
332 RESET:
333 begin
334     is_decoding_o = 1'b0;
335     instr_req_o   = 1'b0;
336     if (fetch_enable_i == 1'b1)
337     begin
338         ctrl_fsm_ns = BOOT_SET;
339     end
340 end
341
342 // copy boot address to instr fetch address
343 BOOT_SET:
344 begin
345     is_decoding_o = 1'b0;
346     instr_req_o   = 1'b1;
347     pc_mux_o      = PC_BOOT;
348     pc_set_o      = 1'b1;
349     if (debug_req_pending) begin
350         ctrl_fsm_ns = DBG_TAKEN_IF;
351         debug_force_wakeup_n = 1'b1;
352     end else begin
353         ctrl_fsm_ns = FIRST_FETCH;
354     end
355 end
356
357 WAIT_SLEEP:
358 begin
359     is_decoding_o = 1'b0;
360     ctrl_busy_o   = 1'b0;
361     instr_req_o   = 1'b0;
362     halt_if_o     = 1'b1;
363     halt_id_o     = 1'b1;
364     ctrl_fsm_ns   = SLEEP;
365 end
366
367 // instruction in if_stage is already valid
368 SLEEP:
369 begin
370     // we begin execution when an
371     // interrupt has arrived
372     is_decoding_o = 1'b0;
373     instr_req_o   = 1'b0;
374     halt_if_o     = 1'b1;
375     halt_id_o     = 1'b1;
376
377     // normal execution flow
378     // in debug mode or single step mode we leave immedi-
ately (wfi=nop)
379     if (wake_from_sleep_o) begin
380         if (debug_req_pending) begin
381             ctrl_fsm_ns = DBG_TAKEN_IF;
382             debug_force_wakeup_n = 1'b1;
383         end else begin
384             ctrl_fsm_ns = FIRST_FETCH;
385         end
386     end else begin
387         ctrl_busy_o = 1'b0;
388     end
389 end
390
391 FIRST_FETCH:
392 begin
393     is_decoding_o = 1'b0;
394
395     // ID stage is always ready
396     ctrl_fsm_ns = DECODE;
397
398     // handle interrupts
399     if (irq_req_ctrl_i && ~(debug_req_pending || de-
bug_mode_q)) begin
400         // This assumes that the pipeline is always flushed
before
401         // going to sleep.
402         // Debug mode takes precedence over irq (see DE-
CODE:)
```

```

403
404 // Taken IRQ
405 halt_if_o = 1'b1;
406 halt_id_o = 1'b1;
407
408 pc_set_o = 1'b1;
409 pc_mux_o = PC_EXCEPTION;
410 exc_pc_mux_o = EXC_PC_IRQ;
411 exc_cause_o = irq_id_ctrl_i;
412 csr_irq_sec_o = irq_sec_ctrl_i;
413
414 // IRQ interface
415 irq_ack_o = 1'b1;
416 irq_id_o = irq_id_ctrl_i;
417
418 if (irq_sec_ctrl_i)
419   trap_addr_mux_o = TRAP_MACHINE;
420 else
421   trap_addr_mux_o = current_priv_lvl_i ==
PRIV_LVL_U ? TRAP_USER : TRAP_MACHINE;
422
423 csr_save_cause_o = 1'b1;
424 csr_cause_o = {1'b1, irq_id_ctrl_i};
425 csr_save_if_o = 1'b1;
426 end
427 end
428
429 DECODE:
430 begin
431
432   if (branch_taken_ex_i)
433   begin //taken branch
434     // there is a branch in the EX stage that is taken
435
436     is_decoding_o = 1'b0;
437
438     pc_mux_o = PC_BRANCH;
439     pc_set_o = 1'b1;
440
441     // if we want to debug, flush the pipeline
442     // the current_pc_if will take the value of the
next instruction to
443     // be executed (NPC)
444
445   end //taken branch
446
447   else if (data_err_i)
448   begin //data error
449     // the current LW or SW have been blocked by the
PMP
450
451     is_decoding_o = 1'b0;
452     halt_if_o = 1'b1;
453     halt_id_o = 1'b1;
454     csr_save_ex_o = 1'b1;
455     csr_save_cause_o = 1'b1;
456     data_err_ack_o = 1'b1;
457     //no jump in this stage as we have to wait one
cycle to go to Machine Mode
458
459     csr_cause_o = {1'b0, data_we_ex_i ?
EXC_CAUSE_STORE_FAULT : EXC_CAUSE_LOAD_FAULT};
460     ctrl_fsm_ns = FLUSH_WB;
461
462   end //data error
463
464   else if (is_fetch_failed_i)
465   begin
466
467     // the current instruction has been blocked by the
PMP
468
469     is_decoding_o = 1'b0;
470     halt_id_o = 1'b1;
471     halt_if_o = 1'b1;
472     csr_save_if_o = 1'b1;
473     csr_save_cause_o = !debug_mode_q;
474
475     //no jump in this stage as we have to wait one
cycle to go to Machine Mode
476
477     csr_cause_o = {1'b0, EXC_CAUSE_INSTR_FAULT};
478     ctrl_fsm_ns = FLUSH_WB;
479
480   end
481   // decode and execute instructions only if the cur-
rent conditional
482   // branch in the EX stage is either not taken, or
there is no
483   // conditional branch in the EX stage
484   else if (instr_valid_i) //valid block
485   begin: blk_decode_level1 // now analyze the current
instruction in the ID stage
486
487     is_decoding_o = 1'b1;
488     illegal_insn_n = 1'b0;
489
490   if ( (debug_req_pending || trigger_match_i) & ~de-
bug_mode_q )

```

```

492   begin
493     //Serving the debug
494     is_decoding_o = COREV_PULP ? 1'b0 : 1'b1;
495     halt_if_o = 1'b1;
496     halt_id_o = 1'b1;
497     ctrl_fsm_ns = DBG_FLUSH;
498     debug_req_entry_n = 1'b1;
499   end
500   else if (irq_req_ctrl_i && ~debug_mode_q)
501   begin
502     // Taken IRQ
503     hwlp_mask_o = COREV_PULP ? 1'b1 : 1'b0;
504
505     is_decoding_o = 1'b0;
506     halt_if_o = 1'b1;
507     halt_id_o = 1'b1;
508
509     pc_set_o = 1'b1;
510     pc_mux_o = PC_EXCEPTION;
511     exc_pc_mux_o = EXC_PC_IRQ;
512     exc_cause_o = irq_id_ctrl_i;
513     csr_irq_sec_o = irq_sec_ctrl_i;
514
515     // IRQ interface
516     irq_ack_o = 1'b1;
517     irq_id_o = irq_id_ctrl_i;
518
519     if (irq_sec_ctrl_i)
520     trap_addr_mux_o = TRAP_MACHINE;
521   else
522     trap_addr_mux_o = current_priv_lvl_i ==
PRIV_LVL_U ? TRAP_USER : TRAP_MACHINE;
523
524     csr_save_cause_o = 1'b1;
525     csr_cause_o = {1'b1, irq_id_ctrl_i};
526     csr_save_id_o = 1'b1;
527   end
528   else
529   begin
530
531     if (illegal_insn_i) begin
532
533       halt_if_o = 1'b1;
534       halt_id_o = 1'b0;
535       ctrl_fsm_ns = id_ready_i ? FLUSH_EX :
DECODE;
536
537       illegal_insn_n = 1'b1;
538
539     end else begin
540
541       //decoding block
542       unique case (1'b1)
543
544         jump_in_dec: begin
545           // handle unconditional jumps
546           // we can jump directly since we know the
address already
547           // we don't need to worry about condition-
al branches here as they
548           // will be evaluated in the EX stage
549           pc_mux_o = PC_JUMP;
550           // if there is a jr stall, wait for it
551           if ((~jr_stall_o) && (~jump_done_q)) be-
gin
552             pc_set_o = 1'b1;
553             jump_done = 1'b1;
554           end
555
556         ebrk_insn_i: begin
557           halt_if_o = 1'b1;
558           halt_id_o = 1'b0;
559
560           if (debug_mode_q)
561
562             // we got back to the park loop in the
debug rom
563             ctrl_fsm_ns = DBG_FLUSH;
564
565           else if (ebrk_force_debug_mode) begin
566             // debug module commands us to enter
debug mode anyway
567             ctrl_fsm_ns = DBG_FLUSH;
568           end else begin
569             // otherwise just a normal ebreak ex-
ception
570             ctrl_fsm_ns = id_ready_i ? FLUSH_EX :
DECODE;
571           end
572
573         wfi_active: begin
574           halt_if_o = 1'b1;
575           halt_id_o = 1'b0;
576           ctrl_fsm_ns = id_ready_i ?
FLUSH_EX : DECODE;
577
578         end
579
580         ecall_insn_i: begin

```

```

581      halt_if_o      = 1'b1;
582      halt_id_o      = 1'b0;
583      ctrl_fsm_ns     = id_ready_i ?
FLUSH_EX : DECODE;
584      end
585
586      fencei_insn_i: begin
587      halt_if_o      = 1'b1;
588      halt_id_o      = 1'b0;
589      ctrl_fsm_ns     = id_ready_i ?
FLUSH_EX : DECODE;
590      end
591
592      mret_insn_i | uret_insn_i | dret_insn_i:
begin
593      halt_if_o      = 1'b1;
594      halt_id_o      = 1'b0;
595      ctrl_fsm_ns     = id_ready_i ?
FLUSH_EX : DECODE;
596      end
597
598      csr_status_i: begin
599      halt_if_o      = 1'b1;
600      if (~id_ready_i) begin
601      ctrl_fsm_ns = DECODE;
602      end else begin
603      ctrl_fsm_ns = FLUSH_EX;
604      if (hwlp_end0_eq_pc) begin
605      hwlp_dec_cnt_o[0] = 1'b1;
606      end
607      if (hwlp_end1_eq_pc) begin
608      hwlp_dec_cnt_o[1] = 1'b1;
609      end
610      end
611      end
612
613      data_load_event_i: begin
614      ctrl_fsm_ns     = id_ready_i ? ELW_EXE :
DECODE;
615      halt_if_o      = 1'b1;
616      end
617
618      default: begin
619
620      if(is_hwlp_body) begin
621      //we are at the inside of an HWloop,
thus change state
622
623      //We stay here in case we returned
from the second last instruction, otherwise the next cycle
624
625      //in DECODE_HWLOOP we miss to jump, we
jump at PC_END.
626      //This way looses a cycle but it's a
corner case of returning from exceptions or interrupts
627      ctrl_fsm_ns     = hwlp_end0_eq_pc_plus4
|| hwlp_end1_eq_pc_plus4 ? DECODE : DECODE_HWLOOP;
628
629      // we can be at the end of HWloop due
to a return from interrupt or exceptions
630      if (hwlp_end0_eq_pc && hwlp_coun-
ter0_gt_1) begin
631      pc_mux_o        = PC_HWLOOP;
632      if (~jump_done_q) begin
633      pc_set_o        = 1'b1;
634      // Keep the instruction and the
related address in the Aligner if
635      // ID is stalled during a jump
636      jump_done       = 1'b1;
637      hwlp_dec_cnt_o[0] = 1'b1;
638      end
639      end
640      if (hwlp_end1_eq_pc && hwlp_coun-
ter1_gt_1) begin
641      pc_mux_o        = PC_HWLOOP;
642      if (~jump_done_q) begin
643      pc_set_o        = 1'b1;
644      // Keep the instruction and the
related address in the Aligner if
645      // ID is stalled during a jump
646      jump_done       = 1'b1;
647      hwlp_dec_cnt_o[1] = 1'b1;
648      end
649      end
650      end
651
652      if (hwlp_end0_eq_pc && hwlp_coun-
ter0_eq_1) begin
653      hwlp_dec_cnt_o[0] = 1'b1;
654      end
655      if (hwlp_end1_eq_pc && hwlp_coun-
ter1_eq_1) begin
656      hwlp_dec_cnt_o[1] = 1'b1;
657      end
658      end
659
660      endcase // unique case (1'b1)
661      end
662
663

```

```

664      if (debug_single_step_i & ~debug_mode_q) begin
665      // prevent any more instructions from ex-
cuting
666      halt_if_o = 1'b1;
667
668      // we don't handle dret here because its
should be illegal
669      // anyway in this context
670
671      // illegal, ecall, ebrk and xrettransition
to later to a DBG
672      // state since we need the return address
which is
673      // determined later
674
675      if (id_ready_i) begin
676      // make sure the current instruction has
been executed
677      unique case(1'b1)
678
679      illegal_insn_i | ecall_insn_i:
begin
680      ctrl_fsm_ns = FLUSH_EX;
681      end
682
683      (~ebrk_force_debug_mode & ebrk_in-
sn_i):
begin
684      ctrl_fsm_ns = FLUSH_EX;
685      end
686
687      mret_insn_i | uret_insn_i:
begin
688      ctrl_fsm_ns = FLUSH_EX;
689      end
690
691      branch_in_id:
begin
692      ctrl_fsm_ns     = DBG_WAIT_BRANCH;
693      end
694
695      default:
696      // regular instruction or ebrk
697      ctrl_fsm_ns = DBG_FLUSH;
698      endcase // unique case (1'b1)
699      end
700
701      end // else: !if (irq_req_ctrl_i && ~de-
bug_mode_q)
702
703      end //valid block
704      else begin
705      is_decoding_o      = 1'b0;
706      perf_pipeline_stall_o = data_load_event_i;
707      end
708      end
709
710      DECODE_HWLOOP:
begin
711      if (COREV_PULP) begin
712      if (instr_valid_i) // valid block
713      begin // now analyze the current instruction in the
ID stage
714
715      is_decoding_o = 1'b1;
716
717      if ( (debug_req_pending || trigger_match_i) & ~de-
bug_mode_q )
718      begin
719      //Serving the debug
720      is_decoding_o = COREV_PULP ? 1'b0 : 1'b1;
721      halt_if_o      = 1'b1;
722      halt_id_o      = 1'b1;
723      ctrl_fsm_ns     = DBG_FLUSH;
724      debug_req_entry_n = 1'b1;
725      end
726      else if (irq_req_ctrl_i && ~debug_mode_q)
727      begin
728      // Taken IRQ
729      hwlp_mask_o      = COREV_PULP ? 1'b1 : 1'b0;
730
731      is_decoding_o = 1'b0;
732      halt_if_o      = 1'b1;
733      halt_id_o      = 1'b1;
734
735      pc_set_o        = 1'b1;
736      pc_mux_o        = PC_EXCEPTION;
737      exc_pc_mux_o     = EXC_PC_IRQ;
738      exc_cause_o      = irq_id_ctrl_i;
739      csr_irq_sec_o     = irq_sec_ctrl_i;
740
741      // IRQ interface
742      irq_ack_o        = 1'b1;
743      irq_id_o         = irq_id_ctrl_i;
744
745      if (irq_sec_ctrl_i)
746      trap_addr_mux_o = TRAP_MACHINE;
747      else

```



```

754      trap_addr_mux_o = current_priv_lvl_i ==
PRIV_LVL_U ? TRAP_USER : TRAP_MACHINE;
755
756      csr_save_cause_o = 1'b1;
757      csr_cause_o      = {1'b1, irq_id_ctrl_i};
758      csr_save_id_o    = 1'b1;
759
760      ctrl_fsm_ns      = DECODE;
761    end
762  else
763    begin
764      if (illegal_insn_i) begin
765        halt_if_o      = 1'b1;
766        halt_id_o      = 1'b1;
767        ctrl_fsm_ns    = FLUSH_EX;
768        illegal_insn_n = 1'b1;
769      end else begin
770        //decoding block
771        unique case (1'b1)
772          ebrk_insn_i: begin
773            halt_if_o = 1'b1;
774            halt_id_o = 1'b0;
775          if (debug_mode_q)
776            // we got back to the park loop in the
777            debug rom
778              ctrl_fsm_ns = DBG_FLUSH;
779          else if (ebrk_force_debug_mode)
780            // debug module commands us to enter
781            debug mode anyway
782              ctrl_fsm_ns = DBG_FLUSH;
783          else begin
784            // otherwise just a normal ebreak ex-
785            ception
786              ctrl_fsm_ns = id_ready_i ? FLUSH_EX :
787            DECODE_HWLOOP;
788          end
789          ecall_insn_i: begin
790            halt_if_o = 1'b1;
791            halt_id_o = 1'b0;
792            ctrl_fsm_ns = id_ready_i ? FLUSH_EX :
793            DECODE_HWLOOP;
794          end
795          csr_status_i: begin
796            halt_if_o = 1'b1;
797            if (~id_ready_i) begin
798              ctrl_fsm_ns = DECODE_HWLOOP;
799            end else begin
800              ctrl_fsm_ns = FLUSH_EX;
801              if (hwlp_end0_eq_pc) begin
802                hwlp_dec_cnt_o[0] = 1'b1;
803              end
804              if (hwlp_end1_eq_pc) begin
805                hwlp_dec_cnt_o[1] = 1'b1;
806              end
807            end
808            data_load_event_i: begin
809              ctrl_fsm_ns = id_ready_i ? ELW_EXE :
810            DECODE_HWLOOP;
811            halt_if_o = 1'b1;
812          end
813          default: begin
814            // we can be at the end of Hwloop due
815            to a return from interrupt or ecall or ebreak or exceptions
816            if (hwlp_end1_eq_pc_plus4) begin
817              if (hwlp_counter1_gt_1) begin
818                hwlp_end_4_id_d = 1'b1;
819                hwlp_targ_addr_o =
820                hwlp_start_addr_i[1];
821                ctrl_fsm_ns = DECODE_HWLOOP;
822              end else
823                ctrl_fsm_ns = is_hwlp_body ?
824            DECODE_HWLOOP : DECODE;
825          end
826          if (hwlp_end0_eq_pc_plus4) begin
827            if (hwlp_counter0_gt_1) begin
828              hwlp_end_4_id_d = 1'b1;
829              hwlp_targ_addr_o =
830              hwlp_start_addr_i[0];
831              ctrl_fsm_ns = DECODE_HWLOOP;
832            end else
833              ctrl_fsm_ns = is_hwlp_body ?
834            DECODE_HWLOOP : DECODE;
835          end
836        end
837      end
838    end
839  end
840  ctrl_fsm_ns = is_hwlp_body ?
841  DECODE_HWLOOP : DECODE;
842

```

```

842      hwlp_dec_cnt_o[0] = hwlp_end0_eq_pc && !
843      hwlp_counter0_eq_0;
844      hwlp_dec_cnt_o[1] = hwlp_end1_eq_pc && !
845      hwlp_counter1_eq_0;
846    end
847  endcase // unique case (1'b1)
848  end // else: !if(illegal_insn_i)
849
850  if (debug_single_step_i & ~debug_mode_q) begin
851    // prevent any more instructions from ex-
852    ecuting
853    halt_if_o = 1'b1;
854    // we don't handle dret here because its
855    should be illegal
856    // anyway in this context
857
858  // illegal, ecall, ebrk and xrettransition
859  to later to a DBG
860  // state since we need the return address
861  which is
862  // determined later
863  if (id_ready_i) begin
864    // make sure the current instruction has
865    been executed
866    unique case(1'b1)
867      illegal_insn_i | ecall_insn_i:
868        begin
869          ctrl_fsm_ns = FLUSH_EX;
870        end
871      (-ebrk_force_debug_mode & ebrk_in-
872      sn_i):
873        begin
874          ctrl_fsm_ns = FLUSH_EX;
875        end
876      mret_insn_i | uret_insn_i:
877        begin
878          ctrl_fsm_ns = FLUSH_EX;
879        end
880      branch_in_id:
881        begin
882          ctrl_fsm_ns = DBG_WAIT_BRANCH;
883        end
884      default:
885        // regular instruction or ebrk
886        ctrl_fsm_ns = DBG_FLUSH;
887      endcase // unique case (1'b1)
888    end
889    end // if (debug_single_step_i & ~de-
890    bug_mode_q)
891    end // else: !if (irq_req_ctrl_i && ~de-
892    bug_mode_q)
893    end // block: blk_decode_level1 : valid block
894    else begin
895      is_decoding_o = 1'b0;
896      perf_pipeline_stall_o = data_load_event_i;
897    end
898  end
899  end
900  end
901  // flush the pipeline, insert NOP into EX stage
902  FLUSH_EX:
903  begin
904    is_decoding_o = 1'b0;
905    halt_if_o = 1'b1;
906    halt_id_o = 1'b1;
907  end
908  if (data_err_i)
909    begin //data error
910      // the current LW or SW have been blocked by the
911      PMP
912      csr_save_ex_o = 1'b1;
913      csr_save_cause_o = 1'b1;
914      data_err_ack_o = 1'b1;
915      //no jump in this stage as we have to wait one
916      cycle to go to Machine Mode
917      csr_cause_o = {1'b0, data_we_ex_i ?
918      EXC_CAUSE_STORE_FAULT : EXC_CAUSE_LOAD_FAULT};
919      ctrl_fsm_ns = FLUSH_WB;
920
921  //putting illegal to 0 as if it was 1, the core is
922  going to jump to the exception of the EX stage,
923  //so the illegal was never executed
924  illegal_insn_n = 1'b0;
925  end //data erro
926  else if (ex_valid_i) begin
927    //check done to prevent data harzard in the CSR re-
928    gisters
929  end
930

```



```

925     ctrl_fsm_ns = FLUSH_WB;
926
927     if(illegal_insn_q) begin
928         csr_save_id_o = 1'b1;
929         csr_save_cause_o = !debug_mode_q;
930         csr_cause_o = {1'b0,
EXC_CAUSE_ILLEGAL_INSN};
931     end else begin
932         unique case (1'b1)
933             ebrk_insn_i: begin
934                 csr_save_id_o = 1'b1;
935                 csr_save_cause_o = 1'b1;
936                 csr_cause_o = {1'b0,
EXC_CAUSE_BREAKPOINT};
937             end
938             ecall_insn_i: begin
939                 csr_save_id_o = 1'b1;
940                 csr_save_cause_o = !debug_mode_q;
941                 csr_cause_o = {1'b0, current_priv_lvl_i
== PRIV_LVL_U ? EXC_CAUSE_ECALL_UMODE : EXC_CAUSE_ECALL_MMODE};
942             end
943             default;;
944             endcase // unique case (1'b1)
945         end
946     end
947 end
948 end
949
950 IRQ_FLUSH_ELW:
951 begin
952     if (COREV_CLUSTER == 1'b1) begin
953         is_decoding_o = 1'b0;
954
955         halt_if_o = 1'b1;
956         halt_id_o = 1'b1;
957
958         ctrl_fsm_ns = DECODE;
959
960         perf_pipeline_stall_o = data_load_event_i;
961
962         if (irq_req_ctrl_i && ~(debug_req_pending || de-
bug_mode_q)) begin
963             // Taken IRQ
964             is_decoding_o = 1'b0;
965             halt_if_o = 1'b1;
966             halt_id_o = 1'b1;
967
968             pc_set_o = 1'b1;
969             pc_mux_o = PC_EXCEPTION;
970             exc_pc_mux_o = EXC_PC_IRQ;
971             exc_cause_o = irq_id_ctrl_i;
972             csr_irq_sec_o = irq_sec_ctrl_i;
973
974             // IRQ interface
975             irq_ack_o = 1'b1;
976             irq_id_o = irq_id_ctrl_i;
977
978             if (irq_sec_ctrl_i)
979                 trap_addr_mux_o = TRAP_MACHINE;
980             else
981                 trap_addr_mux_o = current_priv_lvl_i ==
PRIV_LVL_U ? TRAP_USER : TRAP_MACHINE;
982
983             csr_save_cause_o = 1'b1;
984             csr_cause_o = {1'b1, irq_id_ctrl_i};
985             csr_save_id_o = 1'b1;
986         end
987     end
988 end
989
990 ELW_EXE:
991 begin
992     if (COREV_CLUSTER == 1'b1) begin
993         is_decoding_o = 1'b0;
994
995         halt_if_o = 1'b1;
996         halt_id_o = 1'b1;
997
998         //if we are here, a cv.elw is executing now in the
EX stage
999         //or if an interrupt has been received
1000         //the ID stage contains the PC_ID of the cv.elw,
therefore halt_id is set to invalid the instruction
1001         //If an interrupt occurs, we replay the ELW
1002         //No needs to check irq_int_req_i since in the EX
stage there is only the cv.elw, no CSR pendings
1003         if(id_ready_i)
1004             ctrl_fsm_ns = ((debug_req_pending || trig-
ger_match_i) & ~debug_mode_q) ? DBG_FLUSH : IRQ_FLUSH_ELW;
1005         // if from the ELW EXE we go to IRQ_FLUSH_ELW, it
is assumed that if there was an IRQ req together with the grant and
IE was valid, then
1006         // there must be no hazard due to xIE
1007         else
1008             ctrl_fsm_ns = ELW_EXE;
1009
1010         perf_pipeline_stall_o = data_load_event_i;
1011     end
1012 end
1013
1014 // flush the pipeline, insert NOP into EX and WB stage

```

```

1015 FLUSH_WB:
1016 begin
1017     is_decoding_o = 1'b0;
1018
1019     halt_if_o = 1'b1;
1020     halt_id_o = 1'b1;
1021
1022     ctrl_fsm_ns = DECODE;
1023
1024     if(data_err_q) begin
1025         //PMP data_error
1026         pc_mux_o = PC_EXCEPTION;
1027         pc_set_o = 1'b1;
1028         trap_addr_mux_o = TRAP_MACHINE;
1029         //little hack during testing
1030         exc_pc_mux_o = EXC_PC_EXCEPTION;
1031         exc_cause_o = data_we_ex_i ?
EXC_CAUSE_LOAD_FAULT : EXC_CAUSE_STORE_FAULT;
1032
1033     end
1034     else if (is_fetch_failed_i) begin
1035         //instruction fetch error
1036         pc_mux_o = PC_EXCEPTION;
1037         pc_set_o = 1'b1;
1038         trap_addr_mux_o = TRAP_MACHINE;
1039         exc_pc_mux_o = debug_mode_q ?
EXC_PC_DBE : EXC_PC_EXCEPTION;
1040         exc_cause_o = EXC_CAUSE_INSTR_FAULT;
1041
1042     end
1043     else begin
1044         if(illegal_insn_q) begin
1045             //exceptions
1046             pc_mux_o = PC_EXCEPTION;
1047             pc_set_o = 1'b1;
1048             trap_addr_mux_o = TRAP_MACHINE;
1049             exc_pc_mux_o = debug_mode_q ?
EXC_PC_DBE : EXC_PC_EXCEPTION;
1050             illegal_insn_n = 1'b0;
1051             if (debug_single_step_i && ~debug_mode_q)
1052                 ctrl_fsm_ns = DBG_TAKEN_IF;
1053         end else begin
1054             unique case(1'b1)
1055                 ebrk_insn_i: begin
1056                     //ebreak
1057                     pc_mux_o = PC_EXCEPTION;
1058                     pc_set_o = 1'b1;
1059                     trap_addr_mux_o = TRAP_MACHINE;
1060                     exc_pc_mux_o = EXC_PC_EXCEPTION;
1061
1062                     if (debug_single_step_i && ~debug_mode_q)
1063                         ctrl_fsm_ns = DBG_TAKEN_IF;
1064                 end
1065                 ecall_insn_i: begin
1066                     //ecall
1067                     pc_mux_o = PC_EXCEPTION;
1068                     pc_set_o = 1'b1;
1069                     trap_addr_mux_o = TRAP_MACHINE;
1070                     exc_pc_mux_o = debug_mode_q ?
EXC_PC_DBE : EXC_PC_EXCEPTION;
1071
1072                     if (debug_single_step_i && ~debug_mode_q)
1073                         ctrl_fsm_ns = DBG_TAKEN_IF;
1074                 end
1075             end
1076
1077             mret_insn_i: begin
1078                 csr_restore_mret_id_o = !debug_mode_q;
1079                 ctrl_fsm_ns = XRET_JUMP;
1080             end
1081             urret_insn_i: begin
1082                 csr_restore_uret_id_o = !debug_mode_q;
1083                 ctrl_fsm_ns = XRET_JUMP;
1084             end
1085             dret_insn_i: begin
1086                 csr_restore_dret_id_o = 1'b1;
1087                 ctrl_fsm_ns = XRET_JUMP;
1088             end
1089
1090             csr_status_i: begin
1091                 if ((hwlp_end0_eq_pc && !hwlp_counter0_eq_0)
||
1092                     (hwlp_end1_eq_pc && !hwlp_counter1_eq_0))
1093                     begin
1094                         pc_mux_o = PC_HWLOOP;
1095                         pc_set_o = 1'b1;
1096                     end
1097                 end
1098             wfi_i: begin
1099                 if ( debug_req_pending) begin
1100                     ctrl_fsm_ns = DBG_TAKEN_IF;
1101                     debug_force_wakeup_n = 1'b1;
1102                 end else begin
1103                     ctrl_fsm_ns = WAIT_SLEEP;
1104                 end
1105             fencei_insn_i: begin
1106                 // we just jump to instruction after the
fence.i since that
1107                 // forces the instruction cache to refetch
1108                 pc_mux_o = PC_FENCEI;

```

```

1109         pc_set_o = 1'b1;
1110     end
1111     default;;
1112 endcase
1113 end
1114 end
1115 end
1116 end
1117
1118 XRET_JUMP:
1119 begin
1120     is_decoding_o = 1'b0;
1121     ctrl_fsm_ns = DECODE;
1122     unique case(1'b1)
1123     mret_dec_i: begin
1124         //mret
1125         pc_mux_o = debug_mode_q ? PC_EXCEP-
TION : PC_MRET;
1126         pc_set_o = 1'b1;
1127         exc_pc_mux_o = EXC_PC_DBE; // only used
1128     end
1129     uret_dec_i: begin
1130         //uret
1131         pc_mux_o = debug_mode_q ? PC_EXCEP-
TION : PC_URET;
1132         pc_set_o = 1'b1;
1133         exc_pc_mux_o = EXC_PC_DBE; // only used
1134     end
1135     dret_dec_i: begin
1136         //dret
1137         // this case is only reachable while in de-
bug_mode
1138         pc_mux_o = PC_DRET;
1139         pc_set_o = 1'b1;
1140         debug_mode_n = 1'b0;
1141     end
1142     default;;
1143 endcase
1144
1145     if (debug_single_step_i && ~debug_mode_q) begin
1146         ctrl_fsm_ns = DBG_TAKEN_IF;
1147     end
1148 end
1149
1150 // a branch was in ID when trying to go to debug rom.
Wait until we can
1151 // determine branch target address (for saving into dpc)
before proceeding
1152 DBG_WAIT_BRANCH:
1153 begin
1154     is_decoding_o = 1'b0;
1155     halt_if_o = 1'b1;
1156
1157     if (branch_taken_ex_i) begin
1158         // there is a branch in the EX stage that is taken
1159         pc_mux_o = PC_BRANCH;
1160         pc_set_o = 1'b1;
1161     end
1162
1163     ctrl_fsm_ns = DBG_FLUSH;
1164 end
1165
1166 // We enter this state when we encounter
1167 // 1. ebreak during debug mode
1168 // 2. trigger match
1169 // 3. ebreak with forced entry into debug mode (ebreakm
or ebreaku set).
1170 // 4. halt request during decode
1171 // Regular ebreak's go through FLUSH_EX and FLUSH_WB.
1172 // For 1. we don't update dcsr and dpc while for 2., 3.,
& 4. we do
1173 // dpc is set to the address of ebreak and trigger match
1174 // not to the next instruction's (which is why we save
the pc in id).
1175 DBG_TAKEN_ID:
1176 begin
1177     is_decoding_o = 1'b0;
1178     pc_set_o = 1'b1;
1179     pc_mux_o = PC_EXCEPTION;
1180     exc_pc_mux_o = EXC_PC_DBD;
1181     // If not in debug mode then save cause and dpc csrs
1182     // else it was an ebreak in debug mode, so don't up-
date csrs
1183     if (~debug_mode_q) begin
1184         csr_save_cause_o = 1'b1;
1185         csr_save_id_o = 1'b1;
1186         debug_csr_save_o = 1'b1;
1187         if (trigger_match_i)
1188             debug_cause_o = DBG_CAUSE_TRIGGER; // pri 4
(highest)
1189         else if (ebrk_force_debug_mode & ebrk_insn_i)
1190             debug_cause_o = DBG_CAUSE_EBREAK; // pri 3
1191         else if (debug_req_entry_q)
1192             debug_cause_o = DBG_CAUSE_HALTREQ; // pri 2 and

```

```

1
1193
1194     end
1195     debug_req_entry_n = 1'b0;
1196     ctrl_fsm_ns = DECODE;
1197     debug_mode_n = 1'b1;
1198 end
1199
1200 // We enter this state for single stepping
1201 // DPC is set the next instruction to be executed/
1202
1203 DBG_TAKEN_IF:
1204 begin
1205     is_decoding_o = 1'b0;
1206     pc_set_o = 1'b1;
1207     pc_mux_o = PC_EXCEPTION;
1208     exc_pc_mux_o = EXC_PC_DBD;
1209     csr_save_cause_o = 1'b1;
1210     debug_csr_save_o = 1'b1;
1211     if (debug_force_wakeup_q)
1212         debug_cause_o = DBG_CAUSE_HALTREQ;
1213     else if (debug_single_step_i)
1214         debug_cause_o = DBG_CAUSE_STEP; // pri 0
1215     csr_save_if_o = 1'b1;
1216     ctrl_fsm_ns = DECODE;
1217     debug_mode_n = 1'b1;
1218     debug_force_wakeup_n = 1'b0;
1219 end
1220
1221 DBG_FLUSH:
1222 begin
1223     is_decoding_o = 1'b0;
1224
1225     halt_if_o = 1'b1;
1226     halt_id_o = 1'b1;
1227
1228     perf_pipeline_stall_o = data_load_event_i;
1229
1230     if (data_err_i)
1231     begin //data error
1232         // the current LW or SW have been blocked by the
PMP
1233         csr_save_ex_o = 1'b1;
1234         csr_save_cause_o = 1'b1;
1235         data_err_ack_o = 1'b1;
1236         //no jump in this stage as we have to wait one
cycle to go to Machine Mode
1237         csr_cause_o = {1'b0, data_we_ex_i ?
EXC_CAUSE_STORE_FAULT : EXC_CAUSE_LOAD_FAULT};
1238         ctrl_fsm_ns = FLUSH_WB;
1239     end //data error
1240     else begin
1241         if(debug_mode_q
1242             trigger_match_i
1243             (ebrk_force_debug_mode & ebrk_insn_i)
1244             data_load_event_i
1245             debug_req_entry_q
1246         begin
1247             ctrl_fsm_ns = DBG_TAKEN_ID;
1248         end else
1249         begin
1250             // else must be debug_single_step_i
1251             ctrl_fsm_ns = DBG_TAKEN_IF;
1252         end
1253     end
1254 end
1255 // Debug end
1256
1257 default: begin
1258     is_decoding_o = 1'b0;
1259     instr_req_o = 1'b0;
1260     ctrl_fsm_ns = RESET;
1261 end
1262 endcase
1263 end
1264
1265
1266 generate
1267 if(COREV_PULP) begin : gen_hwlp
1268     // Convert hwlp_jump_o to a pulse
1269     // hwlp_jump_o should last one cycle only, as the
prefetcher
1270     // reacts immediately. If it last more cycles, the
prefetcher
1271     // goes on requesting HWLP_BEGIN more than one time
(wrong!).
1272     // This signal is not controlled by id_ready because oth-
erwise,
1273     // in case of stall, the jump would happen at the end of
the stall.
1274     // Make hwlp_jump_o last only one cycle
1275     assign hwlp_jump_o = (hwlp_end_4_id_d && !
hwlp_end_4_id_q) ? 1'b1 : 1'b0;

```

```

1281
1282 always_ff @(posedge clk or negedge rst_n) begin
1283     if(!rst_n) begin
1284         hwlp_end_4_id_q <= 1'b0;
1285     end else begin
1286         hwlp_end_4_id_q <= hwlp_end_4_id_d;
1287     end
1288 end
1289
1290 assign hwlp_end0_eq_pc = hwlp_end_addr_i[0] ==
pc_id_i + 4; // Equivalent to hwlp_end_addr_i[0] - 4 == pc_id_i
1291 assign hwlp_end1_eq_pc = hwlp_end_addr_i[1] ==
pc_id_i + 4; // Equivalent to hwlp_end_addr_i[1] - 4 == pc_id_i
1292 assign hwlp_counter0_gt_1 = hwlp_counter_i[0] > 1;
1293 assign hwlp_counter1_gt_1 = hwlp_counter_i[1] > 1;
1294 assign hwlp_counter0_eq_1 = hwlp_counter_i[0] == 1;
1295 assign hwlp_counter1_eq_1 = hwlp_counter_i[1] == 1;
1296 assign hwlp_counter0_eq_0 = hwlp_counter_i[0] == 0;
1297 assign hwlp_counter1_eq_0 = hwlp_counter_i[1] == 0;
1298 assign hwlp_end0_eq_pc_plus4 = hwlp_end_addr_i[0] ==
pc_id_i + 8; // Equivalent to hwlp_end_addr_i[0] - 4 == pc_id_i +
4
1299 assign hwlp_end1_eq_pc_plus4 = hwlp_end_addr_i[1] ==
pc_id_i + 8; // Equivalent to hwlp_end_addr_i[1] - 4 == pc_id_i +
4
1300 assign hwlp_start0_leq_pc = hwlp_start_addr_i[0] <=
pc_id_i;
1301 assign hwlp_start1_leq_pc = hwlp_start_addr_i[1] <=
pc_id_i;
1302 assign hwlp_end0_geq_pc = hwlp_end_addr_i[0] >=
pc_id_i + 4; // Equivalent to hwlp_end_addr_i[0] - 4 >= pc_id_i
1303 assign hwlp_end1_geq_pc = hwlp_end_addr_i[1] >=
pc_id_i + 4; // Equivalent to hwlp_end_addr_i[1] - 4 >= pc_id_i;
1304 assign is_hwlp_body = ((hwlp_start0_leq_pc &&
hwlp_end0_geq_pc) && hwlp_counter0_gt_1) || ((hwlp_start1_leq_pc &&
hwlp_end1_geq_pc) && hwlp_counter1_gt_1);
1305
1306 end else begin : gen_no_hwlp
1307
1308     assign hwlp_jump_o = 1'b0;
1309     assign hwlp_end_4_id_q = 1'b0;
1310     assign hwlp_end0_eq_pc = 1'b0;
1311     assign hwlp_end1_eq_pc = 1'b0;
1312     assign hwlp_counter0_gt_1 = 1'b0;
1313     assign hwlp_counter1_gt_1 = 1'b0;
1314     assign hwlp_counter0_eq_1 = 1'b0;
1315     assign hwlp_counter1_eq_1 = 1'b0;
1316     assign hwlp_counter0_eq_0 = 1'b0;
1317     assign hwlp_counter1_eq_0 = 1'b0;
1318     assign hwlp_end0_eq_pc_plus4 = 1'b0;
1319     assign hwlp_end1_eq_pc_plus4 = 1'b0;
1320     assign hwlp_start0_leq_pc = 1'b0;
1321     assign hwlp_start1_leq_pc = 1'b0;
1322     assign hwlp_end0_geq_pc = 1'b0;
1323     assign hwlp_end1_geq_pc = 1'b0;
1324     assign is_hwlp_body = 1'b0;
1325
1326 end
1327
1328 endgenerate
1329
1330 ///////////////////////////////////////////////////
1331 //      -      -      -      -      -
1332 // /  _||  | _ _ | || /  _||  _ _ | | _ _ _ |
1333 // \  _\  | / _ ' | || | / _\  ' _\  | ' _\  _\
1334 //  _ ) | || ( | | | | | ( ) | | | | | | ( ) |
1335 // | _ / \ _\ _\ | | | \ _\ _\ | | | \ _\ | \ _\ |
1336 //
1337 ///////////////////////////////////////////////////
1338 always_comb
1339 begin
1340     load_stall_o = 1'b0;
1341     deassert_we_o = 1'b0;
1342
1343     // deassert WE when the core is not decoding instructions
1344     if (~is_decoding_o)
1345         deassert_we_o = 1'b1;
1346
1347     // deassert WE in case of illegal instruction
1348     if (illegal_insn_i)
1349         deassert_we_o = 1'b1;
1350
1351     // Stall because of load operation
1352     if (
1353         ( (data_req_ex_i == 1'b1) && (regfile_we_ex_i ==
1'b1) ) ||
1354         (wb_ready_i == 1'b0) && (regfile_we_wb_i == 1'b1)
1355     ) &&
1356         ( (reg_d_ex_is_reg_a_i == 1'b1) ||
1357         (reg_d_ex_is_reg_b_i == 1'b1) || (reg_d_ex_is_reg_c_i == 1'b1) ||
1358         (is_decoding_o && (regfile_we_id_i && !data_mis-
aligned_i) && (regfile_waddr_ex_i == regfile_alu_waddr_id_i)) )
1359     )

```

```

1359 begin
1360     deassert_we_o = 1'b1;
1361     load_stall_o = 1'b1;
1362 end
1363
1364 // Stall because of jr path
1365 // - always stall if a result is to be forwarded to the PC
1366 // we don't care about in which state the ctrl_fsm is as
we deassert_we
1367 // anyway when we are not in DECODE
1368 if ((ctrl_transfer_insn_in_dec_i == BRANCH_JALR) &&
1369     (((regfile_we_wb_i == 1'b1) && (reg_d_wb_is_reg_a_i ==
1'b1)) ||
1370     ((regfile_we_ex_i == 1'b1) && (reg_d_ex_is_reg_a_i ==
1'b1)) ||
1371     ((regfile_alu_we_fw_i == 1'b1) &&
1372     (reg_d_alu_is_reg_a_i == 1'b1)) ||
1373     (FPU && (apu_read_dep_for_jalr_i == 1'b1)))
1374 )
1375 begin
1376     jr_stall_o = 1'b1;
1377     deassert_we_o = 1'b1;
1378 end
1379 else
1380 begin
1381     jr_stall_o = 1'b0;
1382 end
1383 end
1384
1385 // stall because of misaligned data access
1386 assign misaligned_stall_o = data_misaligned_i;
1387
1388 // APU dependency stalls (data hazards)
1389 assign apu_stall_o = apu_read_dep_i | (apu_write_dep_i &
~apu_en_i);
1390
1391 // Forwarding control unit
1392 always_comb
1393 begin
1394     // default assignments
1395     operand_a_fw_mux_sel_o = SEL_REGFILE;
1396     operand_b_fw_mux_sel_o = SEL_REGFILE;
1397     operand_c_fw_mux_sel_o = SEL_REGFILE;
1398
1399     // Forwarding WB -> ID
1400     if (regfile_we_wb_i == 1'b1)
1401     begin
1402         if (reg_d_wb_is_reg_a_i == 1'b1)
1403             operand_a_fw_mux_sel_o = SEL_FW_WB;
1404         if (reg_d_wb_is_reg_b_i == 1'b1)
1405             operand_b_fw_mux_sel_o = SEL_FW_WB;
1406         if (reg_d_wb_is_reg_c_i == 1'b1)
1407             operand_c_fw_mux_sel_o = SEL_FW_WB;
1408     end
1409
1410     // Forwarding EX -> ID
1411     if (regfile_alu_we_fw_i == 1'b1)
1412     begin
1413         if (reg_d_alu_is_reg_a_i == 1'b1)
1414             operand_a_fw_mux_sel_o = SEL_FW_EX;
1415         if (reg_d_alu_is_reg_b_i == 1'b1)
1416             operand_b_fw_mux_sel_o = SEL_FW_EX;
1417         if (reg_d_alu_is_reg_c_i == 1'b1)
1418             operand_c_fw_mux_sel_o = SEL_FW_EX;
1419     end
1420
1421     // for misaligned memory accesses
1422     if (data_misaligned_i)
1423     begin
1424         operand_a_fw_mux_sel_o = SEL_FW_EX;
1425         operand_b_fw_mux_sel_o = SEL_REGFILE;
1426     end else if (mult_multicycle_i) begin
1427         operand_c_fw_mux_sel_o = SEL_FW_EX;
1428     end
1429 end
1430 end
1431
1432 // update registers
1433 always_ff @(posedge clk , negedge rst_n)
1434 begin : UPDATE_REGS
1435     if ( rst_n == 1'b0 )
1436     begin
1437         ctrl_fsm_cs <= RESET;
1438         jump_done_q <= 1'b0;
1439         data_err_q <= 1'b0;
1440
1441         debug_mode_q <= 1'b0;
1442         illegal_insn_q <= 1'b0;
1443
1444         debug_req_entry_q <= 1'b0;
1445         debug_force_wakeup_q <= 1'b0;
1446     end
1447     else
1448     begin
1449         ctrl_fsm_cs <= ctrl_fsm_ns;
1450
1451         // clear when id is valid (no instruction incoming)
1452         jump_done_q <= jump_done & (~id_ready_i);

```

```

1453
1454     data_err_q      <= data_err_i;
1455
1456     debug_mode_q    <= debug_mode_n;
1457
1458     illegal_insn_q  <= illegal_insn_n;
1459
1460     debug_req_entry_q <= debug_req_entry_n;
1461     debug_force_wakeup_q <= debug_force_wakeup_n;
1462 end
1463 end
1464
1465 // wakeup from sleep conditions
1466 assign wake_from_sleep_o = irq_wu_ctrl_i || de-
bug_req_pending || debug_mode_q;
1467
1468 // debug mode
1469 assign debug_mode_o = debug_mode_q;
1470 assign debug_req_pending = debug_req_i || debug_req_q;
1471
1472 // Do not let cv.elw cause core_sleep_o during debug
1473 assign debug_p_elw_no_sleep_o = debug_mode_q || debug_req_q
|| debug_single_step_i || trigger_match_i;
1474
1475 // Do not let WFI cause core_sleep_o (but treat as NOP):
1476 //
1477 // - During debug
1478 // - For PULP Cluster (only cv.elw can trigger sleep)
1479
1480 assign debug_wfi_no_sleep_o = debug_mode_q || de-
bug_req_pending || debug_single_step_i || trigger_match_i ||
COREV_CLUSTER;
1481
1482 // Gate off wfi
1483 assign wfi_active = wfi_i & ~debug_wfi_no_sleep_o;
1484
1485 // sticky version of debug_req (must be on clk_ungated_i
such that incoming pulse before core is enabled is not missed)
1486 always_ff @(posedge clk_ungated_i, negedge rst_n)
1487   if ( !rst_n )
1488     debug_req_q <= 1'b0;
1489   else
1490     if( debug_req_i )
1491       debug_req_q <= 1'b1;
1492     else if( debug_mode_q )
1493       debug_req_q <= 1'b0;
1494
1495 // Debug state FSM
1496 always_ff @(posedge clk , negedge rst_n)
1497 begin
1498   if ( rst_n == 1'b0 )
1499     begin
1500       debug_fsm_cs <= HAVERESET;
1501     end
1502   else
1503     begin
1504       debug_fsm_cs <= debug_fsm_ns;
1505     end
1506 end
1507
1508 always_comb
1509 begin
1510   debug_fsm_ns = debug_fsm_cs;
1511
1512   case (debug_fsm_cs)
1513     HAVERESET:
1514       begin
1515         if (debug_mode_n || (ctrl_fsm_ns == FIRST_FETCH)) be-
gin
1516           if (debug_mode_n) begin
1517             debug_fsm_ns = HALTED;
1518           end else begin
1519             debug_fsm_ns = RUNNING;
1520           end
1521         end
1522       end
1523     RUNNING:
1524       begin
1525         if (debug_mode_n) begin
1526           debug_fsm_ns = HALTED;
1527         end
1528       end
1529     end
1530     HALTED:
1531       begin
1532         if (!debug_mode_n) begin
1533           debug_fsm_ns = RUNNING;
1534         end
1535       end
1536     default: begin
1537       debug_fsm_ns = HAVERESET;
1538     end
1539   endcase
1540 end
1541
1542 assign debug_havereset_o = debug_fsm_cs[HAVERESET_INDEX];
1543 assign debug_running_o = debug_fsm_cs[RUNNING_INDEX];
1544 assign debug_halted_o = debug_fsm_cs[HALTED_INDEX];
1545
1546
1547

```

```

1548 //-----
1549 // Assertions
1550 //-----
1551
1552 `ifdef CV32E40P_ASSERT_ON
1553
1554 // make sure that taken branches do not happen back-to-back,
as this is not
1555 // possible without branch prediction in the IF stage
1556 assert property (
1557   @(posedge clk) (branch_taken_ex_i) |>
(~branch_taken_ex_i) ) else $warning("Two branches back-to-back are
taken");
1558
1559 // ELW_EXE and IRQ_FLUSH_ELW states are only used for
COREV_CLUSTER = 1
1560 property p_pulp_cluster_only_states;
1561   @(posedge clk) (1'b1) |> ( !((COREV_CLUSTER == 1'b0) &&
((ctrl_fsm_cs == ELW_EXE) || (ctrl_fsm_cs == IRQ_FLUSH_ELW))) );
1562 endproperty
1563
1564 a_pulp_cluster_only_states : assert prop-
erty(p_pulp_cluster_only_states);
1565
1566 // WAIT_SLEEP and SLEEP states are never used for
COREV_CLUSTER = 1
1567 property p_pulp_cluster_excluded_states;
1568   @(posedge clk) (1'b1) |> ( !((COREV_CLUSTER == 1'b1) &&
((ctrl_fsm_cs == SLEEP) || (ctrl_fsm_cs == WAIT_SLEEP))) );
1569 endproperty
1570
1571 a_pulp_cluster_excluded_states : assert prop-
erty(p_pulp_cluster_excluded_states);
1572
1573 generate
1574   if (COREV_PULP) begin : gen_pulp_xpulp_assertions
1575
1576     // HWLoop 0 and 1 having target address constraints
1577     property p_hwlp_same_target_address;
1578       @(posedge clk) (hwlp_counter_i[1] > 1 &&
hwlp_counter_i[0] > 1 && pc_id_i >= hwlp_start_addr_i[0] && pc_id_i
<= hwlp_end_addr_i[0] - 4) |> ( hwlp_end_addr_i[1] - 4 >=
hwlp_end_addr_i[0] - 4 + 8 );
1579     endproperty
1580
1581     a_hwlp_same_target_address : assert prop-
erty(p_hwlp_same_target_address) else $warning("%t, HWLoops target
address do not respect constraints", $time);
1582
1583     end else begin : gen_no_pulp_xpulp_assertions
1584
1585       property p_no_hwlp;
1586         @(posedge clk) (1'b1) |> ((pc_mux_o != PC_HWLOOP) &&
(ctrl_fsm_cs != DECODE_HWLOOP) &&
(hwlp_mask_o == 1'b0) &&
(is_hwlp_body == 'b0) &&
(hwlp_start_addr_i == 'b0)
&& (hwlp_end_addr_i == 'b0) && (hwlp_counter_i[1] == 32'b0) &&
(hwlp_counter_i[0] == 32'b0) &&
(hwlp_dec_cnt_o == 2'b0) &&
(hwlp_jump_o == 1'b0) && (hwlp_targ_addr_o == 32'b0) &&
(hwlp_end0_eq_pc == 1'b0) && (hwlp_end1_eq_pc == 1'b0) &&
(hwlp_counter1_gt_1 == 1'b0) &&
(hwlp_counter0_eq_1 == 1'b0) &&
(hwlp_counter1_eq_1 == 1'b0) &&
(hwlp_end0_eq_pc_plus4 == 1'b0) && (hwlp_start0_leq_pc == 0) &&
(hwlp_start1_leq_pc == 0) &&
(hwlp_end0_geq_pc == 1'b0) && (hwlp_end4_id_d == 1'b0) &&
(hwlp_end4_id_q == 1'b0));
1587     endproperty
1588
1589     a_no_hwlp : assert property(p_no_hwlp);
1590
1591   end
1592 endgenerate
1593
1594 // Ensure DBG_TAKEN_IF can only be entered if in single
step mode or woken
1595 // up from sleep by debug_req_i
1596
1597 a_single_step_dbg_taken_if : assert property (@(posedge
clk) disable iff (!rst_n) (ctrl_fsm_ns==DBG_TAKEN_IF) |> ((~de-
bug_mode_q && debug_single_step_i) || debug_force_wakeup_n));
1598
1599 // Ensure DBG_FLUSH state is only one cycle. This implies
that cause is either trigger, debug_req_entry, or ebreak
1600 a_dbg_flush : assert property (@(posedge clk) disable iff
(!rst_n) (ctrl_fsm_cs==DBG_FLUSH) |> (ctrl_fsm_ns!=DBG_FLUSH) ));
1601
1602 // Ensure that debug state outputs are one-hot
1603 a_debug_state_onehot : assert property (@(posedge clk) $one-
hot({debug_havereset_o, debug_running_o, debug_halted_o}));
1604
1605

```

```
1612 // Ensure that debug_halted_o equals debug_mode_q
1613 a_debug_halted_equals_debug_mode : assert property
  (@(posedge clk) disable iff (!rst_n) (1'b1) |-> (debug_mode_q == de-
bug_halted_o));
1614
1615 // Ensure ID always ready in FIRST_FETCH state
1616 a_first_fetch_id_ready : assert property (@(posedge clk)
disable iff (!rst_n) (ctrl_fsm_cs == FIRST_FETCH) |-> (id_ready_i ==
1'b1));
1617
```

```
1618 // Ensure that the only way to get to DBG_TAKEN_IF from
DBG_FLUSH is if debug_single_step_i is asserted
1619 a_dbg_flush_to_taken_if : assert property (@(posedge clk)
disable iff (!rst_n) (ctrl_fsm_cs == DBG_FLUSH) && (ctrl_fsm_ns ==
DBG_TAKEN_IF) |-> debug_single_step_i);
1620
1621 `endif
1622
1623 endmodule // cv32e40p_controller
```

CV32E40P RVFI 트레이스

What this is. The RISC-V Formal Interface instrumentation for the core: retirement trace output.

Why it exists. Because model and RTL cannot be compared unless the RTL reports what it did. This module exposes, per retired instruction, the PC, the instruction word, register reads and writes, memory accesses and traps.

Undergraduate review · EECS concepts. Observability; trace interfaces; instrumentation that does not alter behaviour.

Graduate · Practice note. If your project lacks an equivalent interface, **building one is the highest-value first task** a new modelling engineer can undertake: nothing else in the comparison flow can exist without it.

8 files • 4,745 lines

API — cv32e40p_apu_tracer.sv

Table 1687. openhwgroup_cv32e40p/bhv/cv32e40p_apu_tracer.sv — module / interface / package (1개)

이름	종류
cv32e40p_apu_tracer	module

Table 1688. openhwgroup_cv32e40p/bhv/cv32e40p_apu_tracer.sv — 포트 (6개)

포트	방향	형/폭
clk_i	input	logic
rst_n	input	logic
hart_id_i	input	logic [31:0]
apu_valid_i	input	logic
apu_waddr_i	input	logic [5:0]
apu_result_i	input	logic [31:0]

API — cv32e40p_core_log.sv

Table 1689. openhwgroup_cv32e40p/bhv/cv32e40p_core_log.sv — module / interface / package (1개)

이름	종류
cv32e40p_core_log	module

Table 1690. openhwgroup_cv32e40p/bhv/cv32e40p_core_log.sv — parameter (5개)

파라미터	형/폭	기본값
COREV_PULP		1
COREV_CLUSTER		0
FPU		0
ZFINX		0
NUM_MHPMCOUNTERS		1

Table 1691. openhwgroup_cv32e40p/bhv/cv32e40p_core_log.sv — 포트 (5개)

포트	방향	형/폭
clk_i	input	logic
is_decoding_i	input	logic
illegal_insn_dec_i	input	logic
hart_id_i	input	logic [31:0]
pc_id_i	input	logic [31:0]

API — cv32e40p_rvfi.sv**Table 1692. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi.sv — module / interface / package (1개)**

이름	종류
cv32e40p_rvfi	module

Table 1693. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi.sv — parameter (3개)

파라미터	형/폭	기본값
FPU		0
ZFINX		0
NUM_MHPMCOUNTERS		1

Table 1694. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi.sv — localparam (4개)

상수	형/폭	값
MSTATUS_WRITE_MASK	logic [31:0]	32'h0000_6088
MCOUNTINHIBIT_WRITE_MASK	logic [31:0]	{{(29-NUM_MHPMCOUNTERS){1'b0}}}
NUM_HPM_EVENTS		16
MHPMEVENT_WRITE_MASK	logic [31:0]	{{(31-NUM_HPM_EVENTS){1'b0}}}

Table 1695. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi.sv — 포트 (505개) — 앞 60개만 표시(전체 505개, 본문 소스 참조)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
is_decoding_i	input	logic
is_illegal_i	input	logic
trigger_match_i	input	logic
data_misaligned_i	input	logic
lsu_data_we_ex_i	input	logic
debug_mode_i	input	logic
debug_cause_i	input	logic [2:0]

포트	방향	형/폭
instr_req_i	input	logic
instr_grant_i	input	logic
instr_rvalid_i	input	logic
prefetch_req_i	input	logic
pc_set_i	input	logic
if_valid_i	input	logic
if_ready_i	input	logic
instr_valid_if_i	input	logic
instr_if_i	input	logic [31:0]
pc_if_i	input	logic [31:0]
instr_pmp_err_if_i	input	logic
instr_valid_id_i	input	logic
instr_rdata_id_i	input	logic [31:0]
is_fetch_failed_id_i	input	logic
instr_req_int_i	input	logic
clear_instr_valid_i	input	logic
pc_id_i	input	logic [31:0]
id_valid_i	input	logic
id_ready_i	input	logic
rf_re_id_i	input	logic [1:0]
sys_en_id_i	input	logic
sys_mret_insn_id_i	input	logic
jump_in_id_i	input	logic
jump_target_id_i	input	logic [31:0]
is_compressed_id_i	input	logic
ebrk_insn_dec_i	input	logic
ecall_insn_dec_i	input	logic
mret_insn_dec_i	input	logic
mret_dec_i	input	logic
csr_cause_i	input	logic [5:0]
debug_csr_save_i	input	logic
hwlp_start_q_i	input	logic [1:0][31:0]
hwlp_end_q_i	input	logic [1:0][31:0]
hwlp_counter_q_i	input	logic [1:0][31:0]
hwlp_counter_n_i	input	logic [1:0][31:0]

포트	방향	형/폭
minstret_i	input	logic
lsu_en_id_i	input	logic
lsu_we_id_i	input	logic
lsu_size_id_i	input	logic [1:0]
rs1_addr_id_i	input	logic [5:0]
rs2_addr_id_i	input	logic [5:0]
rs3_addr_id_i	input	logic [5:0]
operand_a_fw_id_i	input	logic [31:0]
operand_b_fw_id_i	input	logic [31:0]
operand_c_fw_id_i	input	logic [31:0]
ex_ready_i	input	logic
ex_valid_i	input	logic
ex_reg_we_i	input	logic
ex_reg_addr_i	input	logic [5:0]
ex_reg_wdata_i	input	logic [31:0]
apu_en_ex_i	input	logic

Table 1696. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi.sv — #define / `define 매크로 (2개)

매크로	인자	정의
SET_RVFI_CSR_FROM_INSN	(CSR_NAME)	\
CSR_FROM_PIPE	(TRACE_NAME, CSR_NAME)	\

API — cv32e40p_rvfi_trace.sv

Table 1697. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi_trace.sv — module / interface / package (1개)

이름	종류
cv32e40p_rvfi_trace	module

Table 1698. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi_trace.sv — parameter (2개)

파라미터	형/폭	기본값
FPU		0
ZFINX		0

Table 1699. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi_trace.sv — 포트 (33개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
hart_id_i	input	logic [31:0]

포트	방향	형/폭
imm_s3_type	input	logic [31:0]
rvfi_valid	input	logic
rvfi_insn	input	logic [31:0]
rvfi_start_cycle	input	integer
rvfi_start_time	input	time
rvfi_stop_cycle	input	integer
rvfi_stop_time	input	time
rvfi_pc_rdata	input	logic [31:0]
rvfi_trap	input	rvfi_trap_t
rvfi_2_rd	input	logic
rvfi_rs1_addr	input	logic [4:0]
rvfi_rs2_addr	input	logic [4:0]
rvfi_rs3_addr	input	logic [4:0]
rvfi_rs1_rdata	input	logic [31:0]
rvfi_rs2_rdata	input	logic [31:0]
rvfi_rs3_rdata	input	logic [31:0]
rvfi_frs1_addr	input	logic [4:0]
rvfi_frs2_addr	input	logic [4:0]
rvfi_frs3_addr	input	logic [4:0]
rvfi_frs1_rvalid	input	logic
rvfi_frs2_rvalid	input	logic
rvfi_frs3_rvalid	input	logic
rvfi_frs1_rdata	input	logic [31:0]
rvfi_frs2_rdata	input	logic [31:0]
rvfi_frs3_rdata	input	logic [31:0]
rvfi_mem_addr	input	logic [31:0]
rvfi_mem_rmask	input	logic [31:0]
rvfi_mem_wmask	input	logic [31:0]
rvfi_mem_rdata	input	logic [31:0]
rvfi_mem_wdata	input	logic [31:0]

API — cv32e40p_sim_clock_gate.sv

Table 1700. openhwgroup_cv32e40p/bhv/cv32e40p_sim_clock_gate.sv — module / interface / package (17개)

이름	종류
cv32e40p_clock_gate	module

Table 1701. openhwgroup_cv32e40p/bhv/cv32e40p_sim_clock_gate.sv — 포트 (4개)

포트	방향	형/폭
clk_i	input	logic
en_i	input	logic
scan_cg_en_i	input	logic
clk_o	output	logic

API — cv32e40p_tb_wrapper.sv**Table 1702. openhwgroup_cv32e40p/bhv/cv32e40p_tb_wrapper.sv — module / interface / package (1개)**

이름	종류
cv32e40p_tb_wrapper	module

Table 1703. openhwgroup_cv32e40p/bhv/cv32e40p_tb_wrapper.sv — parameter (7개)

파라미터	형/폭	기본값
COREV_PULP		0
COREV_CLUSTER		0
FPU		0
FPU_ADDMUL_LAT		0
FPU_OTHERS_LAT		0
ZFINX		0
NUM_MHPMCOUNTERS		1

Table 1704. openhwgroup_cv32e40p/bhv/cv32e40p_tb_wrapper.sv — 포트 (31개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
pulp_clock_en_i	input	logic
scan_cg_en_i	input	logic
boot_addr_i	input	logic [31:0]
mtvec_addr_i	input	logic [31:0]
dm_halt_addr_i	input	logic [31:0]
hart_id_i	input	logic [31:0]
dm_exception_addr_i	input	logic [31:0]
instr_req_o	output	logic
instr_gnt_i	input	logic
instr_rvalid_i	input	logic
instr_addr_o	output	logic [31:0]

포트	방향	형/폭
instr_rdata_i	input	logic [31:0]
data_req_o	output	logic
data_gnt_i	input	logic
data_rvalid_i	input	logic
data_we_o	output	logic
data_be_o	output	logic [3:0]
data_addr_o	output	logic [31:0]
data_wdata_o	output	logic [31:0]
data_rdata_i	input	logic [31:0]
irq_i	input	logic [31:0]
irq_ack_o	output	logic
irq_id_o	output	logic [4:0]
debug_req_i	input	logic
debug_havereset_o	output	logic
debug_running_o	output	logic
debug_halted_o	output	logic
fetch_enable_i	input	logic
core_sleep_o	output	logic

API — cv32e40p_tracer.sv

Table 1705. openhwgroup_cv32e40p/bhv/cv32e40p_tracer.sv — module / interface / package (1개)

이름	종류
cv32e40p_tracer	module

Table 1706. openhwgroup_cv32e40p/bhv/cv32e40p_tracer.sv — parameter (2개)

파라미터	형/폭	기본값
FPU		0
ZFINX		0

Table 1707. openhwgroup_cv32e40p/bhv/cv32e40p_tracer.sv — 포트 (54개)

포트	방향	형/폭
clk_i	input	logic
rst_n	input	logic
hart_id_i	input	logic [31:0]
pc	input	logic [31:0]
instr	input	logic [31:0]

포트	방향	형/폭
controller_state_i	input	ctrl_state_e
compressed	input	logic
id_valid	input	logic
is_decoding	input	logic
is_illegal	input	logic
trigger_match	input	logic
rs1_value	input	logic [31:0]
rs2_value	input	logic [31:0]
rs3_value	input	logic [31:0]
rs2_value_vec	input	logic [31:0]
rd_is_fp	input	logic
rs1_is_fp	input	logic
rs2_is_fp	input	logic
rs3_is_fp	input	logic
ex_valid	input	logic
ex_reg_addr	input	logic [5:0]
ex_reg_we	input	logic
ex_reg_wdata	input	logic [31:0]
ex_data_req	input	logic
ex_data_gnt	input	logic
ex_data_we	input	logic
ex_data_addr	input	logic [31:0]
ex_data_wdata	input	logic [31:0]
data_misaligned	input	logic
ebrk_insn	input	logic
debug_mode	input	logic
ebrk_force_debug_mode	input	logic
wb_bypass	input	logic
wb_valid	input	logic
wb_reg_addr	input	logic [5:0]
wb_reg_we	input	logic
wb_reg_wdata	input	logic [31:0]
imm_u_type	input	logic [31:0]
imm_uj_type	input	logic [31:0]
imm_i_type	input	logic [31:0]

포트	방향	형/폭
imm_iz_type	input	logic [11:0]
imm_z_type	input	logic [31:0]
imm_s_type	input	logic [31:0]
imm_sb_type	input	logic [31:0]
imm_s2_type	input	logic [31:0]
imm_s3_type	input	logic [31:0]
imm_vs_type	input	logic [31:0]
imm_vu_type	input	logic [31:0]
imm_shuffle_type	input	logic [31:0]
imm_clip_type	input	logic [4:0]
apu_en_i	input	logic
apu_singlecycle_i	input	logic
apu_multicycle_i	input	logic
apu_rvalid_i	input	logic

API — insn_trace.sv

Table 1708. openhwgroup_cv32e40p/bhv/insn_trace.sv — #define / `define 매크로 (3개)

매크로	인자	정의
DEFINE_CSR	(CSR_NAME)	\
ASSIGN_CSR	(CSR_NAME)	\
INIT_CSR	(CSR_NAME)	\

1. openhwgroup_cv32e40p/bhv/cv32e40p_apu_tracer.sv (79 lines) <pre> 1 // Copyright 2020 Silicon Labs, Inc. 2 // 3 // This file, and derivatives thereof are licensed under the 4 // Solderpad License, Version 2.0 (the "License"). 5 // 6 // Use of this file means you agree to the terms and condi- 7 // tions of the license and are in full compliance with the License. 8 // 9 // You may obtain a copy of the License at: 10 // 11 // https://solderpad.org/licenses/SHL-2.0/ 12 // 13 // Unless required by applicable law or agreed to in writing, 14 // software and hardware implementations thereof distributed under the 15 // License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR 16 // CONDITIONS OF ANY KIND, EITHER EXPRESSED OR IMPLIED. 17 // 18 // See the License for the specific language governing permis- 19 // sions and limitations under the License. 20 // 21 /// 22 // Engineer: Arjan Bink - ar- 23 // jan.bink@silabs.com 24 // 25 // Design Name: cv32e40p_apu_tracer.sv (APU 26 // trace) </pre>	<pre> 25 // Project Name: CV32E40P 26 // Language: SystemVeri- log 27 // 28 // Description: Logs the follow- ing: 29 // 30 // - APU register file write ad- 31 // dress 32 // - APU register file write 33 // data 34 // 35 // Note: This code was here from cv32e40p_core.sv in 36 // order to 37 // remove the use of global defines in the RTL 38 // code. 39 // 40 /// 41 // `ifdef CV32E40P_APU_TRACE 42 // 43 // module cv32e40p_apu_tracer (44 // input logic clk_i, 45 // input logic rst_n, 46 // input logic [31:0] hart_id_i, 47 // input logic apu_valid_i, 48 // input logic [5:0] apu_waddr_i, </pre>
---	---


```

46     input logic [31:0] apu_result_i
47 );
48
49     int    apu_trace;
50     string fn;
51     string apu_waddr_trace;
52
53     // open/close output file for writing
54     initial begin
55         wait(rst_n == 1'b1);
56         $format(fn, "apu_trace_core%h.log", hart_id_i);
57         $display("[APU_TRACER %2d] Output filename is: %s",
58             hart_id_i, fn);
59         apu_trace = $fopen(fn, "w");
60         $fwrite(apu_trace, "time      register \tresult\n");
61
62         while (1) begin
63             @(negedge clk_i);
64             if (apu_valid_i == 1'b1) begin
65
66                 if (apu_waddr_i > 31) $format(apu_waddr_trace, "%d",
67                     apu_waddr_i[4:0]);
68                 else $format(apu_waddr_trace, "%d", apu_wad-
69                     dr_i[4:0]);
70                 $fwrite(apu_trace, "%t %s \t\t%h\n", $time, apu_wad-
71                     dr_trace, apu_result_i);
72             end
73         end
74     end
75
76     final begin
77         $fclose(apu_trace);
78     end
79 endmodule // cv32e40p_apu_tracer
80 `endif // CV32E40P_APU_TRACE

```

2. openhwgroup_cv32e40p/bhv/cv32e40p_core_log.sv (68 lines)

```

1 // Copyright 2020 Silicon Labs, Inc.
2 //
3 // This file, and derivatives thereof are licensed under the
4 // Solderpad License, Version 2.0 (the "License").
5 //
6 // Use of this file means you agree to the terms and condi-
7 // tions of the license and are in full compliance with the License.
8 //
9 // You may obtain a copy of the License at:
10 //
11 //     https://solderpad.org/licenses/SHL-2.0/
12 //
13 // Unless required by applicable law or agreed to in writing,
14 // software and hardware implementations thereof distributed under the
15 // License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR
16 // CONDITIONS
17 // OF ANY KIND, EITHER EXPRESSED OR IMPLIED.
18 // See the License for the specific language governing permis-
19 // sions and
20 // limitations under the License.
21 //
22 // Engineer:      Arjan Bink - ar-
23 // jan.bink@silabs.com
24 //
25 // Design Name:   cv32e40p_core_log.sv (cv32e40p_core simula-
26 // tion log)
27 // Project Name:   CV32E40P
28 // Language:      SystemVeri-
29 // log
30 //
31 // Description:   Logs the follow-
32 // ing:
33 //
34 // - top level parameter set-
35 // tings
36 // - illegal instruc-
37 // tions
38 //
39 // Note:          This code was here from cv32e40p_core.sv
40 // and
41 // cv32e40p_controller.sv in order to remove
42 // the use of

```

```

35 //          global defines in the RTL
36 //          code.
37 //
38 //          //
39 //          //////////////////////////////////////
40 //          module cv32e40p_core_log #(
41 //              parameter COREV_PULP = 1, // PULP ISA Extension (incl.
42 //              custom CSRs and hardware loop, excl. cv.elw)
43 //              parameter COREV_CLUSTER = 0, // PULP Cluster interface
44 //              (incl. cv.elw)
45 //              parameter FPU = 0, // Floating Point Unit (interfaced via
46 //              APU interface)
47 //              parameter ZFINX = 0, // Float-in-General Purpose re-
48 //              gisters
49 //              parameter NUM_MHPMCOUNTERS = 1
50 //          ) (
51 //              input logic      clk_i,
52 //              input logic      is_decoding_i,
53 //              input logic      illegal_insn_dec_i,
54 //              input logic [31:0] hart_id_i,
55 //              input logic [31:0] pc_id_i
56 //          );
57 // Log top level parameter values
58 initial begin
59     $display(
60
61         "[cv32e40p_core]: COREV_PULP = %d, COREV_CLUSTER = %d,
62         FPU %d, ZFINX %d, NUM_MHPMCOUNTERS %d",
63         COREV_PULP, COREV_CLUSTER, FPU, ZFINX, NUM_MHPMCOUN-
64         TERS);
65     end
66 // Log illegal instructions
67 always_ff @(negedge clk_i) begin
68     // print warning in case of decoding errors
69     if (is_decoding_i && illegal_insn_dec_i) begin
70         $display("%t: Illegal instruction (core %0d) at PC
71         0x%h:", $time, hart_id_i[3:0], pc_id_i);
72     end
73 end
74 endmodule // cv32e40p_core_log

```

3. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi.sv (2,062 lines)

```

1 // Copyright 2024 Dolphin Design
2 // SPDX-License-Identifier: Apache-2.0 WITH SHL-2.1
3 //
4 // Licensed under the Solderpad Hardware License v 2.1 (the
5 // "License");
6 // you may not use this file except in compliance with the Li-
7 // cense, or,
8 // at your option, the Apache License version 2.0.
9 // You may obtain a copy of the License at
10 //
11 //     https://solderpad.org/licenses/SHL-2.1/
12 //
13 // Unless required by applicable law or agreed to in writing,
14 // any work
15 // distributed under the License is distributed on an "AS IS"
16 // BASIS,
17 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
18 // press or implied.
19 // See the License for the specific language governing permis-
20 // sions and
21 // limitations under the License.
22 //
23 //
24 // Description:   CV32E40P RVFI inter-
25 // face
26 //
27 //
28 //
29 // Contributors: Davide Schiavone, OpenHW Group <dav-
30 // ide@openhwgroup.org>
31 // Halfdan Bechmann, Silicon Labs <half-
32 // dan.bechmann@silabs.com>
33 // Yoann Pruvost, Dolphin Design
34 // <yoann.pruvost@dolphin.fr>
35 //
36 //
37 // Description:   CV32E40P RVFI inter-
38 // face
39 //
40 //
41 //
42 //
43 //
44 //
45 //
46 //
47 //
48 //
49 //
50 //
51 //
52 //
53 //
54 //
55 //
56 //
57 //
58 //
59 //
60 //
61 //
62 //
63 //
64 //
65 //
66 //
67 //
68 //
69 //
70 //
71 //
72 //
73 //
74 //
75 //
76 //
77 //
78 //
79 //
80 //
81 //
82 //
83 //
84 //
85 //
86 //
87 //
88 //
89 //
90 //
91 //
92 //
93 //
94 //
95 //
96 //
97 //
98 //
99 //
100 //
101 //
102 //
103 //
104 //
105 //
106 //
107 //
108 //
109 //
110 //
111 //
112 //
113 //
114 //
115 //
116 //
117 //
118 //
119 //
120 //
121 //
122 //
123 //
124 //
125 //
126 //
127 //
128 //
129 //
130 //
131 //
132 //
133 //
134 //
135 //
136 //
137 //
138 //
139 //
140 //
141 //
142 //
143 //
144 //
145 //
146 //
147 //
148 //
149 //
150 //
151 //
152 //
153 //
154 //
155 //
156 //
157 //
158 //
159 //
160 //
161 //
162 //
163 //
164 //
165 //
166 //
167 //
168 //
169 //
170 //
171 //
172 //
173 //
174 //
175 //
176 //
177 //
178 //
179 //
180 //
181 //
182 //
183 //
184 //
185 //
186 //
187 //
188 //
189 //
190 //
191 //
192 //
193 //
194 //
195 //
196 //
197 //
198 //
199 //
200 //
201 //
202 //
203 //
204 //
205 //
206 //
207 //
208 //
209 //
210 //
211 //
212 //
213 //
214 //
215 //
216 //
217 //
218 //
219 //
220 //
221 //
222 //
223 //
224 //
225 //
226 //
227 //
228 //
229 //
230 //
231 //
232 //
233 //
234 //
235 //
236 //
237 //
238 //
239 //
240 //
241 //
242 //
243 //
244 //
245 //
246 //
247 //
248 //
249 //
250 //
251 //
252 //
253 //
254 //
255 //
256 //
257 //
258 //
259 //
260 //
261 //
262 //
263 //
264 //
265 //
266 //
267 //
268 //
269 //
270 //
271 //
272 //
273 //
274 //
275 //
276 //
277 //
278 //
279 //
280 //
281 //
282 //
283 //
284 //
285 //
286 //
287 //
288 //
289 //
290 //
291 //
292 //
293 //
294 //
295 //
296 //
297 //
298 //
299 //
300 //
301 //
302 //
303 //
304 //
305 //
306 //
307 //
308 //
309 //
310 //
311 //
312 //
313 //
314 //
315 //
316 //
317 //
318 //
319 //
320 //
321 //
322 //
323 //
324 //
325 //
326 //
327 //
328 //
329 //
330 //
331 //
332 //
333 //
334 //
335 //
336 //
337 //
338 //
339 //
340 //
341 //
342 //
343 //
344 //
345 //
346 //
347 //
348 //
349 //
350 //
351 //
352 //
353 //
354 //
355 //
356 //
357 //
358 //
359 //
360 //
361 //
362 //
363 //
364 //
365 //
366 //
367 //
368 //
369 //
370 //
371 //
372 //
373 //
374 //
375 //
376 //
377 //
378 //
379 //
380 //
381 //
382 //
383 //
384 //
385 //
386 //
387 //
388 //
389 //
390 //
391 //
392 //
393 //
394 //
395 //
396 //
397 //
398 //
399 //
400 //
401 //
402 //
403 //
404 //
405 //
406 //
407 //
408 //
409 //
410 //
411 //
412 //
413 //
414 //
415 //
416 //
417 //
418 //
419 //
420 //
421 //
422 //
423 //
424 //
425 //
426 //
427 //
428 //
429 //
430 //
431 //
432 //
433 //
434 //
435 //
436 //
437 //
438 //
439 //
440 //
441 //
442 //
443 //
444 //
445 //
446 //
447 //
448 //
449 //
450 //
451 //
452 //
453 //
454 //
455 //
456 //
457 //
458 //
459 //
460 //
461 //
462 //
463 //
464 //
465 //
466 //
467 //
468 //
469 //
470 //
471 //
472 //
473 //
474 //
475 //
476 //
477 //
478 //
479 //
480 //
481 //
482 //
483 //
484 //
485 //
486 //
487 //
488 //
489 //
490 //
491 //
492 //
493 //
494 //
495 //
496 //
497 //
498 //
499 //
500 //
501 //
502 //
503 //
504 //
505 //
506 //
507 //
508 //
509 //
510 //
511 //
512 //
513 //
514 //
515 //
516 //
517 //
518 //
519 //
520 //
521 //
522 //
523 //
524 //
525 //
526 //
527 //
528 //
529 //
530 //
531 //
532 //
533 //
534 //
535 //
536 //
537 //
538 //
539 //
540 //
541 //
542 //
543 //
544 //
545 //
546 //
547 //
548 //
549 //
550 //
551 //
552 //
553 //
554 //
555 //
556 //
557 //
558 //
559 //
560 //
561 //
562 //
563 //
564 //
565 //
566 //
567 //
568 //
569 //
570 //
571 //
572 //
573 //
574 //
575 //
576 //
577 //
578 //
579 //
580 //
581 //
582 //
583 //
584 //
585 //
586 //
587 //
588 //
589 //
590 //
591 //
592 //
593 //
594 //
595 //
596 //
597 //
598 //
599 //
600 //
601 //
602 //
603 //
604 //
605 //
606 //
607 //
608 //
609 //
610 //
611 //
612 //
613 //
614 //
615 //
616 //
617 //
618 //
619 //
620 //
621 //
622 //
623 //
624 //
625 //
626 //
627 //
628 //
629 //
630 //
631 //
632 //
633 //
634 //
635 //
636 //
637 //
638 //
639 //
640 //
641 //
642 //
643 //
644 //
645 //
646 //
647 //
648 //
649 //
650 //
651 //
652 //
653 //
654 //
655 //
656 //
657 //
658 //
659 //
660 //
661 //
662 //
663 //
664 //
665 //
666 //
667 //
668 //
669 //
670 //
671 //
672 //
673 //
674 //
675 //
676 //
677 //
678 //
679 //
680 //
681 //
682 //
683 //
684 //
685 //
686 //
687 //
688 //
689 //
690 //
691 //
692 //
693 //
694 //
695 //
696 //
697 //
698 //
699 //
700 //
701 //
702 //
703 //
704 //
705 //
706 //
707 //
708 //
709 //
710 //
711 //
712 //
713 //
714 //
715 //
716 //
717 //
718 //
719 //
720 //
721 //
722 //
723 //
724 //
725 //
726 //
727 //
728 //
729 //
730 //
731 //
732 //
733 //
734 //
735 //
736 //
737 //
738 //
739 //
740 //
741 //
742 //
743 //
744 //
745 //
746 //
747 //
748 //
749 //
750 //
751 //
752 //
753 //
754 //
755 //
756 //
757 //
758 //
759 //
760 //
761 //
762 //
763 //
764 //
765 //
766 //
767 //
768 //
769 //
770 //
771 //
772 //
773 //
774 //
775 //
776 //
777 //
778 //
779 //
780 //
781 //
782 //
783 //
784 //
785 //
786 //
787 //
788 //
789 //
790 //
791 //
792 //
793 //
794 //
795 //
796 //
797 //
798 //
799 //
800 //
801 //
802 //
803 //
804 //
805 //
806 //
807 //
808 //
809 //
810 //
811 //
812 //
813 //
814 //
815 //
816 //
817 //
818 //
819 //
820 //
821 //
822 //
823 //
824 //
825 //
826 //
827 //
828 //
829 //
830 //
831 //
832 //
833 //
834 //
835 //
836 //
837 //
838 //
839 //
840 //
841 //
842 //
843 //
844 //
845 //
846 //
847 //
848 //
849 //
850 //
851 //
852 //
853 //
854 //
855 //
856 //
857 //
858 //
859 //
860 //
861 //
862 //
863 //
864 //
865 //
866 //
867 //
868 //
869 //
870 //
871 //
872 //
873 //
874 //
875 //
876 //
877 //
878 //
879 //
880 //
881 //
882 //
883 //
884 //
885 //
886 //
887 //
888 //
889 //
890 //
891 //
892 //
893 //
894 //
895 //
896 //
897 //
898 //
899 //
900 //
901 //
902 //
903 //
904 //
905 //
906 //
907 //
908 //
909 //
910 //
911 //
912 //
913 //
914 //
915 //
916 //
917 //
918 //
919 //
920 //
921 //
922 //
923 //
924 //
925 //
926 //
927 //
928 //
929 //
930 //
931 //
932 //
933 //
934 //
935 //
936 //
937 //
938 //
939 //
940 //
941 //
942 //
943 //
944 //
945 //
946 //
947 //
948 //
949 //
950 //
951 //
952 //
953 //
954 //
955 //
956 //
957 //
958 //
959 //
960 //
961 //
962 //
963 //
964 //
965 //
966 //
967 //
968 //
969 //
970 //
971 //
972 //
973 //
974 //
975 //
976 //
977 //
978 //
979 //
980 //
981 //
982 //
983 //
984 //
985 //
986 //
987 //
988 //
989 //
990 //
991 //
992 //
993 //
994 //
995 //
996 //
997 //
998 //
999 //
1000 //

```

```

33 parameter FPU = 0,
34 parameter ZFINX = 0,
35 parameter NUM_MHPMCOUNTERS = 1
36 ) (
37   input logic clk_i,
38   input logic rst_ni,
39
40   input logic is_decoding_i,
41   input logic is_illegal_i,
42   input logic trigger_match_i,
43   input logic data_misaligned_i,
44   input logic lsu_data_we_ex_i,
45
46   input logic debug_mode_i,
47   input logic [2:0] debug_cause_i,
48   /// INSTR IF Probes ///
49   input logic instr_req_i,
50   input logic instr_grant_i,
51   input logic instr_rvalid_i,
52
53   input logic prefetch_req_i,
54   input logic pc_set_i,
55   /// IF Probes ///
56   input logic if_valid_i,
57   input logic if_ready_i,
58   input logic instr_valid_if_i,
59   input logic [31:0] instr_if_i,
60   input logic [31:0] pc_if_i,
61   input logic instr_pmp_err_if_i,
62
63   input logic instr_valid_id_i,
64   input logic [31:0] instr_rdata_id_i,
65   input logic is_fetch_failed_id_i,
66   input logic instr_req_int_i,
67   input logic clear_instr_valid_i,
68   /// ID probes ///
69   input logic [31:0] pc_id_i,
70   input logic id_valid_i,
71
72   input logic id_ready_i,
73   input logic [1:0] rf_re_id_i,
74   input logic sys_en_id_i,
75   input logic sys_mret_insn_id_i,
76   input logic jump_in_id_i,
77   input logic [31:0] jump_target_id_i,
78   input logic is_compressed_id_i,
79
80   input logic ebrk_insn_dec_i,
81   input logic ecall_insn_dec_i,
82
83   input logic mret_insn_dec_i,
84   input logic mret_dec_i,
85
86   input logic [5:0] csr_cause_i,
87
88   input logic debug_csr_save_i,
89   // HWLOOP regs
90   input logic [1:0][31:0] hwl_start_q_i,
91   input logic [1:0][31:0] hwl_end_q_i,
92   input logic [1:0][31:0] hwl_counter_q_i,
93   input logic [1:0][31:0] hwl_counter_n_i,
94   input logic minstret_i,
95   // LSU
96   input logic lsu_en_id_i,
97   input logic lsu_we_id_i,
98   input logic [1:0] lsu_size_id_i,
99   // Register reads
100  input logic [5:0] rs1_addr_id_i,
101  input logic [5:0] rs2_addr_id_i,
102  input logic [5:0] rs3_addr_id_i,
103  input logic [31:0] operand_a_fw_id_i,
104  input logic [31:0] operand_b_fw_id_i,
105  input logic [31:0] operand_c_fw_id_i,
106
107  /// EX probes ///
108
109  // Register writes in EX
110  input logic ex_ready_i,
111  input logic ex_valid_i,
112
113  input logic ex_reg_we_i,
114  input logic [5:0] ex_reg_addr_i,
115  input logic [31:0] ex_reg_wdata_i,
116
117  input logic apu_en_ex_i,
118  input logic apu_singlecycle_i,
119  input logic apu_multicycle_i,
120  input logic wb_contention_lsu_i,
121  input logic wb_contention_i,
122  input logic regfile_we_lsu_i,
123
124  input logic branch_in_ex_i,
125  input logic branch_decision_ex_i,
126  input logic dret_in_ex_i,
127  // LSU
128  input logic lsu_en_ex_i,
129  input logic lsu_pmp_err_ex_i,
130  input logic lsu_pma_err_atomic_ex_i,
131
132  input logic [31:0] branch_target_ex_i,
133

```

```

134 input logic [31:0] data_addr_ex_i,
135 input logic [31:0] data_wdata_ex_i,
136 input logic lsu_split_q_ex_i,
137
138 input logic mult_ready_i,
139 input logic alu_ready_i,
140
141 /// WB probes ///
142 input logic [31:0] pc_wb_i,
143 input logic wb_ready_i,
144 input logic wb_valid_i,
145 input logic ebreak_in_wb_i,
146 input logic [31:0] instr_rdata_wb_i,
147 input logic csr_en_wb_i,
148 input logic sys_wfi_insn_wb_i,
149 // Register writes
150 input logic rf_we_wb_i,
151 input logic [5:0] rf_addr_wb_i,
152 input logic [31:0] rf_wdata_wb_i,
153 input logic regfile_alu_we_ex_i,
154 // LSU
155 input logic [31:0] lsu_rdata_wb_i,
156
157 input logic data_we_ex_i,
158 input logic [5:0] data_atop_ex_i,
159 input logic [1:0] data_type_ex_i,
160 input logic [31:0] alu_operand_c_ex_i,
161 input logic [1:0] data_reg_offset_ex_i,
162 input logic data_load_event_ex_i,
163 input logic [1:0] data_sign_ext_ex_i,
164 input logic [31:0] lsu_rdata_i,
165 input logic data_req_ex_i,
166 input logic [31:0] alu_operand_a_ex_i,
167 input logic [31:0] alu_operand_b_ex_i,
168 input logic useincr_addr_ex_i,
169 input logic data_misaligned_ex_i,
170 input logic p_elw_start_i,
171 input logic p_elw_finish_i,
172 input logic lsu_ready_ex_i,
173 input logic lsu_ready_wb_i,
174
175 input logic [3:0] lsu_data_be_i,
176
177 input logic data_req_pmp_i,
178 input logic data_gnt_pmp_i,
179 input logic data_rvalid_i,
180 input logic data_err_pmp_i,
181 input logic [31:0] data_addr_pmp_i,
182 input logic data_we_i,
183 input logic [5:0] data_atop_i,
184 input logic [3:0] data_be_i,
185 input logic [31:0] data_wdata_i,
186 input logic [31:0] data_rdata_i,
187
188 /// APU ///
189 input logic apu_req_i,
190 input logic apu_gnt_i,
191 input logic apu_rvalid_i,
192
193
194 // PC //
195 input logic [31:0] branch_addr_n_i,
196
197 // Controller FSM probes
198 input ctrl_state_e ctrl_fsm_cs_i,
199 input ctrl_state_e ctrl_fsm_ns_i,
200 input logic pending_single_step_i,
201 input logic single_step_allowed_i,
202 input logic pending_debug_i,
203 input logic debug_mode_q_i,
204 input logic [3:0] pc_mux_i,
205 input logic [2:0] exc_pc_mux_i,
206
207 // Interrupt Controller probes
208 input logic [31:0] irq_i,
209 input logic irq_wu_ctrl_i,
210 input logic [4:0] irq_id_ctrl_i,
211
212 /// CSR Probes ///
213 input csr_num_e csr_addr_i,
214 input logic csr_we_i,
215 input logic [31:0] csr_wdata_int_i,
216
217 input logic csr_fregs_we_i,
218 input logic csr_jvt_we_i,
219 input Status_t csr_mstatus_n_i,
220 input Status_t csr_mstatus_q_i,
221 input FS_t csr_mstatus_fs_n_i,
222 input FS_t csr_mstatus_fs_q_i,
223
224 input logic csr_mstatusth_we_i,
225 input logic [31:0] csr_misa_n_i,
226 input logic [31:0] csr_misa_q_i,
227
228 input logic [31:0] csr_mie_n_i,
229 input logic [31:0] csr_mie_q_i,
230 input logic csr_mie_we_i,
231 input logic [23:0] csr_mtvec_n_i,
232 input logic [23:0] csr_mtvec_q_i,
233 input logic [1:0] csr_mtvec_mode_n_i,
234 input logic [1:0] csr_mtvec_mode_q_i,

```

```

235
236 input logic csr_mvtvt_we_i,
237 input logic [31:0] csr_mcountinhibit_n_i,
238 input logic [31:0] csr_mcountinhibit_q_i,
239 input logic csr_mcountinhibit_we_i,
240 input logic [31:0][31:0] csr_mhpmevent_n_i,
241 input logic [31:0][31:0] csr_mhpmevent_q_i,
242 input logic [31:0] csr_mhpmevent_we_i,
243 input logic [31:0] csr_mscratch_n_i,
244 input logic [31:0] csr_mscratch_q_i,
245 input logic [31:0] csr_mepc_n_i,
246 input logic [31:0] csr_mepc_q_i,
247
248 input logic [ 5:0] csr_mcause_n_i,
249 input logic [ 5:0] csr_mcause_q_i,
250 input logic [31:0] csr_mip_n_i,
251 input logic [31:0] csr_mip_q_i,
252 input logic csr_mip_we_i,
253 input logic [31:0] csr_mnxti_n_i,
254 input logic [31:0] csr_mnxti_q_i,
255 input logic [31:0] csr_mnxti_we_i,
256
257 input logic csr_mintstatus_we_i,
258 input logic [31:0] csr_minthresh_n_i,
259 input logic [31:0] csr_minthresh_q_i,
260 input logic csr_minthresh_we_i,
261 input logic [31:0] csr_mscratchcsw_n_i,
262 input logic [31:0] csr_mscratchcsw_q_i,
263 input logic csr_mscratchcsw_we_i,
264 input logic [31:0] csr_mscratchcswl_n_i,
265 input logic [31:0] csr_mscratchcswl_q_i,
266 input logic csr_mscratchcswl_we_i,
267 input logic [31:0] csr_mclibase_n_i,
268 input logic [31:0] csr_mclibase_q_i,
269 input logic csr_mclibase_we_i,
270 input logic [31:0] csr_tdata1_n_i,
271 input logic [31:0] csr_tdata1_q_i,
272 input logic csr_tdata1_we_i,
273 input logic [31:0] csr_tdata2_n_i,
274 input logic [31:0] csr_tdata2_q_i,
275 input logic csr_tdata2_we_i,
276 input logic [31:0] csr_tinfo_n_i,
277 input logic [31:0] csr_tinfo_q_i,
278
279 input Dcsr_t csr_dcsr_n_i,
280 input Dcsr_t csr_dcsr_q_i,
281
282 input logic [31:0] csr_dpc_n_i,
283 input logic [31:0] csr_dpc_q_i,
284 input logic csr_dpc_we_i,
285 input logic [31:0] csr_dscratch0_n_i,
286 input logic [31:0] csr_dscratch0_q_i,
287 input logic csr_dscratch0_we_i,
288 input logic [31:0] csr_dscratch1_n_i,
289 input logic [31:0] csr_dscratch1_q_i,
290 input logic csr_dscratch1_we_i,
291 input logic [31:0] csr_mconfigptr_n_i,
292 input logic [31:0] csr_mconfigptr_q_i,
293 input logic csr_mconfigptr_we_i,
294
295 // performance counters
296 // cycle, instret, hpcounter, cycleh, instreth, hp-
counterh
297 // mcycle, minstret, mhpcounter, mcycleh, minstreth, mhp-
counterh
298 input logic [63:0][MHPMCOUNTER_WIDTH-1:0] csr_mhpm-
counter_q_i,
299 input logic [31:0] csr_mhpm-
counter_write_lower_i,
300 input logic [31:0] csr_mhpm-
counter_write_upper_i,
301
302 input logic [31:0] csr_mvendid_i,
303 input logic [31:0] csr_marchid_i,
304 input logic [31:0] csr_mhartid_i,
305 input logic [31:0] csr_mimpid_i,
306
307 input logic [31:0] csr_mcounteren_n_i,
308 input logic [31:0] csr_mcounteren_q_i,
309 input logic csr_mcounteren_we_i,
310
311 input logic [ 7:0] csr_pmpcfg_n_i [16],
312 input logic [ 7:0] csr_pmpcfg_q_i [16],
313 input logic [15:0] csr_pmpcfg_we_i,
314 input logic [31:0] csr_pmpaddr_n_i, // PMP address input
shared for all pmpaddr
registers
315 input logic [31:0] csr_pmpaddr_q_i [16],
316 input logic [15:0] csr_pmpaddr_we_i,
317 input logic [31:0] csr_mseccfg_n_i,
318 input logic [31:0] csr_mseccfg_q_i,
319 input logic csr_mseccfg_we_i,
320 input logic [31:0] csr_mseccfg_n_i,
321 input logic [31:0] csr_mseccfg_h_q_i,
322 input logic [31:0] csr_mseccfg_h_we_i,
323
324 input logic [4:0] csr_fcsr_fflags_n_i,
325 input logic [4:0] csr_fcsr_fflags_q_i,
326 input logic csr_fcsr_fflags_we_i,
327 input logic [2:0] csr_fcsr_frm_n_i,
328 input logic [2:0] csr_fcsr_frm_q_i,
329
330 // RISC-V Formal Interface
331 // Does not comply with the coding standards of _i/_o suf-
fixes, but follow,
332 // the convention of RISC-V Formal Interface Specifica-
tion.
333 output logic [ 0:0] rvfi_valid,
334 output logic [63:0] rvfi_order,
335 output integer rvfi_start_cycle,
336 output time rvfi_start_time,
337 output integer rvfi_stop_cycle,
338 output time rvfi_stop_time,
339 output logic [31:0] rvfi_insn,
340 output rvfi_trap_t rvfi_trap,
341 output logic [ 0:0] rvfi_halt,
342 output rvfi_intr_t rvfi_intr,
343 output logic [ 1:0] rvfi_mode,
344 output logic [ 1:0] rvfi_ixl,
345 output logic [ 1:0] rvfi_nmip,
346
347 output logic [2:0] rvfi_dbg,
348 output logic [0:0] rvfi_dbg_mode,
349
350 output rvfi_wu_t rvfi_wu,
351 output logic [0:0] rvfi_sleep,
352
353 output logic [ 4:0] rvfi_rd_addr [1:0],
354 output logic [31:0] rvfi_rd_wdata [1:0],
355 output logic rvfi_frd_wvalid [1:0],
356 output logic [ 4:0] rvfi_frd_addr [1:0],
357 output logic [31:0] rvfi_frd_wdata [1:0],
358 output logic rvfi_2_rd,
359 output logic [ 4:0] rvfi_rs1_addr,
360 output logic [ 4:0] rvfi_rs2_addr,
361 output logic [ 4:0] rvfi_rs3_addr,
362 output logic [31:0] rvfi_rs1_rdata,
363 output logic [31:0] rvfi_rs2_rdata,
364 output logic [31:0] rvfi_rs3_rdata,
365 output logic [ 4:0] rvfi_frs1_addr,
366 output logic [ 4:0] rvfi_frs2_addr,
367 output logic [ 4:0] rvfi_frs3_addr,
368 output logic rvfi_frs1_rvalid,
369 output logic rvfi_frs2_rvalid,
370 output logic rvfi_frs3_rvalid,
371 output logic [31:0] rvfi_frs1_rdata,
372 output logic [31:0] rvfi_frs2_rdata,
373 output logic [31:0] rvfi_frs3_rdata,
374
375 output logic [31:0] rvfi_pc_rdata,
376 output logic [31:0] rvfi_pc_wdata,
377
378 output logic [31:0] rvfi_mem_addr,
379 output logic [31:0] rvfi_mem_rmask,
380 output logic [31:0] rvfi_mem_wmask,
381 output logic [31:0] rvfi_mem_rdata,
382 output logic [31:0] rvfi_mem_wdata,
383
384 // CSRs
385 output logic [31:0] rvfi_csr_fflags_rmask,
386 output logic [31:0] rvfi_csr_fflags_wmask,
387 output logic [31:0] rvfi_csr_fflags_rdata,
388 output logic [31:0] rvfi_csr_fflags_wdata,
389 output logic [31:0] rvfi_csr_frm_rmask,
390 output logic [31:0] rvfi_csr_frm_wmask,
391 output logic [31:0] rvfi_csr_frm_rdata,
392 output logic [31:0] rvfi_csr_frm_wdata,
393 output logic [31:0] rvfi_csr_fcsr_rmask,
394 output logic [31:0] rvfi_csr_fcsr_wmask,
395 output logic [31:0] rvfi_csr_fcsr_rdata,
396 output logic [31:0] rvfi_csr_fcsr_wdata,
397 output logic [31:0] rvfi_csr_jvt_rmask,
398 output logic [31:0] rvfi_csr_jvt_wmask,
399 output logic [31:0] rvfi_csr_jvt_rdata,
400 output logic [31:0] rvfi_csr_jvt_wdata,
401 output logic [31:0] rvfi_csr_mstatus_rmask,
402 output logic [31:0] rvfi_csr_mstatus_wmask,
403 output logic [31:0] rvfi_csr_mstatus_rdata,
404 output logic [31:0] rvfi_csr_mstatus_wdata,
405 output logic [31:0] rvfi_csr_mstatush_rmask,
406 output logic [31:0] rvfi_csr_mstatush_wmask,
407 output logic [31:0] rvfi_csr_mstatush_rdata,
408 output logic [31:0] rvfi_csr_mstatush_wdata,
409 output logic [31:0] rvfi_csr_misa_rmask,
410 output logic [31:0] rvfi_csr_misa_wmask,
411 output logic [31:0] rvfi_csr_misa_rdata,
412 output logic [31:0] rvfi_csr_misa_wdata,
413 output logic [31:0] rvfi_csr_mie_rmask,
414 output logic [31:0] rvfi_csr_mie_wmask,
415 output logic [31:0] rvfi_csr_mie_rdata,
416 output logic [31:0] rvfi_csr_mie_wdata,
417 output logic [31:0] rvfi_csr_mtvec_rmask,
418 output logic [31:0] rvfi_csr_mtvec_wmask,
419 output logic [31:0] rvfi_csr_mtvec_rdata,
420 output logic [31:0] rvfi_csr_mtvec_wdata,
421 output logic [31:0] rvfi_csr_mvtvt_rmask,
422 output logic [31:0] rvfi_csr_mvtvt_wmask,
423 output logic [31:0] rvfi_csr_mvtvt_rdata,
424 output logic [31:0] rvfi_csr_mvtvt_wdata,
425 output logic [31:0] rvfi_csr_mcountinhibit_rmask,
426 output logic [31:0] rvfi_csr_mcountinhibit_wmask,
427 output logic [31:0] rvfi_csr_mcountinhibit_rdata,
428 output logic [31:0] rvfi_csr_mcountinhibit_wdata,

```

```

429 output logic [31:0][31:0] rvfi_csr_mhpmevent_rmask, //
3-31 implemented
430 output logic [31:0][31:0] rvfi_csr_mhpmevent_wmask,
431 output logic [31:0][31:0] rvfi_csr_mhpmevent_rdata,
432 output logic [31:0][31:0] rvfi_csr_mhpmevent_wdata,
433 output logic [31:0] rvfi_csr_mscratch_rmask,
434 output logic [31:0] rvfi_csr_mscratch_wmask,
435 output logic [31:0] rvfi_csr_mscratch_rdata,
436 output logic [31:0] rvfi_csr_mscratch_wdata,
437 output logic [31:0] rvfi_csr_mepc_rmask,
438 output logic [31:0] rvfi_csr_mepc_wmask,
439 output logic [31:0] rvfi_csr_mepc_rdata,
440 output logic [31:0] rvfi_csr_mepc_wdata,
441 output logic [31:0] rvfi_csr_mcause_rmask,
442 output logic [31:0] rvfi_csr_mcause_wmask,
443 output logic [31:0] rvfi_csr_mcause_rdata,
444 output logic [31:0] rvfi_csr_mcause_wdata,
445 output logic [31:0] rvfi_csr_mtval_rmask,
446 output logic [31:0] rvfi_csr_mtval_wmask,
447 output logic [31:0] rvfi_csr_mtval_rdata,
448 output logic [31:0] rvfi_csr_mtval_wdata,
449 output logic [31:0] rvfi_csr_mip_rmask,
450 output logic [31:0] rvfi_csr_mip_wmask,
451 output logic [31:0] rvfi_csr_mip_rdata,
452 output logic [31:0] rvfi_csr_mip_wdata,
453 output logic [31:0] rvfi_csr_mnxti_rmask,
454 output logic [31:0] rvfi_csr_mnxti_wmask,
455 output logic [31:0] rvfi_csr_mnxti_rdata,
456 output logic [31:0] rvfi_csr_mnxti_wdata,
457 output logic [31:0] rvfi_csr_mintstatus_rmask,
458 output logic [31:0] rvfi_csr_mintstatus_wmask,
459 output logic [31:0] rvfi_csr_mintstatus_rdata,
460 output logic [31:0] rvfi_csr_mintstatus_wdata,
461 output logic [31:0] rvfi_csr_mintthresh_rmask,
462 output logic [31:0] rvfi_csr_mintthresh_wmask,
463 output logic [31:0] rvfi_csr_mintthresh_rdata,
464 output logic [31:0] rvfi_csr_mintthresh_wdata,
465 output logic [31:0] rvfi_csr_mscratchcsw_rmask,
466 output logic [31:0] rvfi_csr_mscratchcsw_wmask,
467 output logic [31:0] rvfi_csr_mscratchcsw_rdata,
468 output logic [31:0] rvfi_csr_mscratchcsw_wdata,
469 output logic [31:0] rvfi_csr_mscratchcswl_rmask,
470 output logic [31:0] rvfi_csr_mscratchcswl_wmask,
471 output logic [31:0] rvfi_csr_mscratchcswl_rdata,
472 output logic [31:0] rvfi_csr_mscratchcswl_wdata,
473 output logic [31:0] rvfi_csr_mclibase_rmask,
474 output logic [31:0] rvfi_csr_mclibase_wmask,
475 output logic [31:0] rvfi_csr_mclibase_rdata,
476 output logic [31:0] rvfi_csr_mclibase_wdata,
477 output logic [31:0] rvfi_csr_tselect_rmask,
478 output logic [31:0] rvfi_csr_tselect_wmask,
479 output logic [31:0] rvfi_csr_tselect_rdata,
480 output logic [31:0] rvfi_csr_tselect_wdata,
481 output logic [3:0][31:0] rvfi_csr_tdata_rmask, // 1-3
implemented
482 output logic [3:0][31:0] rvfi_csr_tdata_wmask,
483 output logic [3:0][31:0] rvfi_csr_tdata_rdata,
484 output logic [3:0][31:0] rvfi_csr_tdata_wdata,
485 output logic [31:0] rvfi_csr_tinfo_rmask,
486 output logic [31:0] rvfi_csr_tinfo_wmask,
487 output logic [31:0] rvfi_csr_tinfo_rdata,
488 output logic [31:0] rvfi_csr_tinfo_wdata,
489 output logic [31:0] rvfi_csr_mcontext_rmask,
490 output logic [31:0] rvfi_csr_mcontext_wmask,
491 output logic [31:0] rvfi_csr_mcontext_rdata,
492 output logic [31:0] rvfi_csr_mcontext_wdata,
493 output logic [31:0] rvfi_csr_scontext_rmask,
494 output logic [31:0] rvfi_csr_scontext_wmask,
495 output logic [31:0] rvfi_csr_scontext_rdata,
496 output logic [31:0] rvfi_csr_scontext_wdata,
497 output logic [31:0] rvfi_csr_dcsr_rmask,
498 output logic [31:0] rvfi_csr_dcsr_wmask,
499 output logic [31:0] rvfi_csr_dcsr_rdata,
500 output logic [31:0] rvfi_csr_dcsr_wdata,
501 output logic [31:0] rvfi_csr_dpc_rmask,
502 output logic [31:0] rvfi_csr_dpc_wmask,
503 output logic [31:0] rvfi_csr_dpc_rdata,
504 output logic [31:0] rvfi_csr_dpc_wdata,
505 output logic [1:0][31:0] rvfi_csr_dscratch_rmask, // 0-1
implemented
506 output logic [1:0][31:0] rvfi_csr_dscratch_wmask,
507 output logic [1:0][31:0] rvfi_csr_dscratch_rdata,
508 output logic [1:0][31:0] rvfi_csr_dscratch_wdata,
509 output logic [31:0] rvfi_csr_mcycle_rmask,
510 output logic [31:0] rvfi_csr_mcycle_wmask,
511 output logic [31:0] rvfi_csr_mcycle_rdata,
512 output logic [31:0] rvfi_csr_mcycle_wdata,
513 output logic [31:0] rvfi_csr_minstret_rmask,
514 output logic [31:0] rvfi_csr_minstret_wmask,
515 output logic [31:0] rvfi_csr_minstret_rdata,
516 output logic [31:0] rvfi_csr_minstret_wdata,
517 output logic [31:0][31:0] rvfi_csr_hpmcounter_rmask, //
3-31 implemented
518 output logic [31:0][31:0] rvfi_csr_hpmcounter_wmask,
519 output logic [31:0][31:0] rvfi_csr_hpmcounter_rdata,
520 output logic [31:0][31:0] rvfi_csr_hpmcounter_wdata,
521 output logic [31:0] rvfi_csr_mcycleh_rmask,
522 output logic [31:0] rvfi_csr_mcycleh_wmask,
523 output logic [31:0] rvfi_csr_mcycleh_rdata,
524 output logic [31:0] rvfi_csr_mcycleh_wdata,
525 output logic [31:0] rvfi_csr_minstret_rmask,
526 output logic [31:0] rvfi_csr_minstret_wmask,
527 output logic [31:0] rvfi_csr_minstret_rdata,
528 output logic [31:0] rvfi_csr_minstret_wdata,
529 output logic [31:0][31:0] rvfi_csr_hpmcounterh_rmask, //
3-31 implemented
530 output logic [31:0][31:0] rvfi_csr_hpmcounterh_wmask,
531 output logic [31:0][31:0] rvfi_csr_hpmcounterh_rdata,
532 output logic [31:0][31:0] rvfi_csr_hpmcounterh_wdata,
533 output logic [31:0] rvfi_csr_cycle_rmask,
534 output logic [31:0] rvfi_csr_cycle_wmask,
535 output logic [31:0] rvfi_csr_cycle_rdata,
536 output logic [31:0] rvfi_csr_cycle_wdata,
537 output logic [31:0] rvfi_csr_instret_rmask,
538 output logic [31:0] rvfi_csr_instret_wmask,
539 output logic [31:0] rvfi_csr_instret_rdata,
540 output logic [31:0] rvfi_csr_instret_wdata,
541 output logic [31:0][31:0] rvfi_csr_hpmcounter_rmask, //
3-31 implemented
542 output logic [31:0][31:0] rvfi_csr_hpmcounter_wmask,
543 output logic [31:0][31:0] rvfi_csr_hpmcounter_rdata,
544 output logic [31:0][31:0] rvfi_csr_hpmcounter_wdata,
545 output logic [31:0] rvfi_csr_cycleh_rmask,
546 output logic [31:0] rvfi_csr_cycleh_wmask,
547 output logic [31:0] rvfi_csr_cycleh_rdata,
548 output logic [31:0] rvfi_csr_cycleh_wdata,
549 output logic [31:0] rvfi_csr_instreth_rmask,
550 output logic [31:0] rvfi_csr_instreth_wmask,
551 output logic [31:0] rvfi_csr_instreth_rdata,
552 output logic [31:0] rvfi_csr_instreth_wdata,
553 output logic [31:0][31:0] rvfi_csr_hpmcounterh_rmask, //
3-31 implemented
554 output logic [31:0][31:0] rvfi_csr_hpmcounterh_wmask,
555 output logic [31:0][31:0] rvfi_csr_hpmcounterh_rdata,
556 output logic [31:0][31:0] rvfi_csr_hpmcounterh_wdata,
557 output logic [31:0] rvfi_csr_mvendid_rmask,
558 output logic [31:0] rvfi_csr_mvendid_wmask,
559 output logic [31:0] rvfi_csr_mvendid_rdata,
560 output logic [31:0] rvfi_csr_mvendid_wdata,
561 output logic [31:0] rvfi_csr_marchid_rmask,
562 output logic [31:0] rvfi_csr_marchid_wmask,
563 output logic [31:0] rvfi_csr_marchid_rdata,
564 output logic [31:0] rvfi_csr_marchid_wdata,
565 output logic [31:0] rvfi_csr_mimpid_rmask,
566 output logic [31:0] rvfi_csr_mimpid_wmask,
567 output logic [31:0] rvfi_csr_mimpid_rdata,
568 output logic [31:0] rvfi_csr_mimpid_wdata,
569 output logic [31:0] rvfi_csr_mhartid_rmask,
570 output logic [31:0] rvfi_csr_mhartid_wmask,
571 output logic [31:0] rvfi_csr_mhartid_rdata,
572 output logic [31:0] rvfi_csr_mhartid_wdata,
573
574 output logic [31:0] rvfi_csr_mcounteren_rmask,
575 output logic [31:0] rvfi_csr_mcounteren_wmask,
576 output logic [31:0] rvfi_csr_mcounteren_rdata,
577 output logic [31:0] rvfi_csr_mcounteren_wdata,
578
579 output logic [3:0][31:0] rvfi_csr_pmpcfg_rmask,
580 output logic [3:0][31:0] rvfi_csr_pmpcfg_wmask,
581 output logic [3:0][31:0] rvfi_csr_pmpcfg_rdata,
582 output logic [3:0][31:0] rvfi_csr_pmpcfg_wdata,
583 output logic [15:0][31:0] rvfi_csr_pmpaddr_rmask,
584 output logic [15:0][31:0] rvfi_csr_pmpaddr_wmask,
585 output logic [15:0][31:0] rvfi_csr_pmpaddr_rdata,
586 output logic [15:0][31:0] rvfi_csr_pmpaddr_wdata,
587 output logic [31:0] rvfi_csr_mseccfg_rmask,
588 output logic [31:0] rvfi_csr_mseccfg_wmask,
589 output logic [31:0] rvfi_csr_mseccfg_rdata,
590 output logic [31:0] rvfi_csr_mseccfg_wdata,
591 output logic [31:0] rvfi_csr_mseccfgh_rmask,
592 output logic [31:0] rvfi_csr_mseccfgh_wmask,
593 output logic [31:0] rvfi_csr_mseccfgh_rdata,
594 output logic [31:0] rvfi_csr_mseccfgh_wdata,
595
596 output logic [31:0] rvfi_csr_mconfigptr_rmask,
597 output logic [31:0] rvfi_csr_mconfigptr_wmask,
598 output logic [31:0] rvfi_csr_mconfigptr_rdata,
599 output logic [31:0] rvfi_csr_mconfigptr_wdata,
600
601 output logic [31:0] rvfi_csr_lpstart0_rmask,
602 output logic [31:0] rvfi_csr_lpstart0_wmask,
603 output logic [31:0] rvfi_csr_lpstart0_rdata,
604 output logic [31:0] rvfi_csr_lpstart0_wdata,
605 output logic [31:0] rvfi_csr_lpend0_rmask,
606 output logic [31:0] rvfi_csr_lpend0_wmask,
607 output logic [31:0] rvfi_csr_lpend0_rdata,
608 output logic [31:0] rvfi_csr_lpend0_wdata,
609 output logic [31:0] rvfi_csr_lpcount0_rmask,
610 output logic [31:0] rvfi_csr_lpcount0_wmask,
611 output logic [31:0] rvfi_csr_lpcount0_rdata,
612 output logic [31:0] rvfi_csr_lpcount0_wdata,
613 output logic [31:0] rvfi_csr_lpstart1_rmask,
614 output logic [31:0] rvfi_csr_lpstart1_wmask,
615 output logic [31:0] rvfi_csr_lpstart1_rdata,
616 output logic [31:0] rvfi_csr_lpstart1_wdata,
617 output logic [31:0] rvfi_csr_lpend1_rmask,
618 output logic [31:0] rvfi_csr_lpend1_wmask,
619 output logic [31:0] rvfi_csr_lpend1_rdata,
620 output logic [31:0] rvfi_csr_lpend1_wdata,
621 output logic [31:0] rvfi_csr_lpcount1_rmask,

```

```

622 output logic [31:0] rvfi_csr_lpcount1_wmask,
623 output logic [31:0] rvfi_csr_lpcount1_rdata,
624 output logic [31:0] rvfi_csr_lpcount1_wdata
625 );
626
627 bit clk_i_d;
628 assign #0.01 clk_i_d = clk_i;
629
630 integer cycles;
631 // cycle counter
632 always_ff @(posedge clk_i_d, negedge rst_ni) begin
633     if (rst_ni == 1'b0) cycles <= 0;
634     else cycles <= cycles + 1;
635 end
636
637 logic pc_mux_debug;
638 logic pc_mux_dret;
639 logic pc_mux_exception;
640 logic pc_mux_debug_exception;
641 logic pc_mux_interrupt;
642 logic pc_mux_nmi;
643
644 localparam logic [31:0] MSTATUS_WRITE_MASK = 32'h0000_6088;
645 localparam logic [31:0] MCAUSEINHIBIT_WRITE_MASK = {{(29-
NUM_MHPMCOUNTERS){1'b0}}, {(NUM_MHPMCOUNTERS){1'b1}}, 3'b101};
646 localparam NUM_HPM_EVENTS = 16;
647 localparam logic [31:0] HPMMEVENT_WRITE_MASK = {{(31-
NUM_HPM_EVENTS){1'b0}}, {(NUM_HPM_EVENTS){1'b1}}};
648
649 `include "pipe_freeze_trace.sv"
650
651 `include "insn_trace.sv"
652
653 insn_trace_t trace_if, trace_id, trace_ex, trace_ex_next,
trace_wb;
654 insn_trace_t tmp_trace_wb;
655 insn_trace_t rvfi_trace_q[$], wb_bypass_trace_q[$];
656
657 bit clear_trace_ex; // = 1'b1;
658 bit clear_trace_wb;
659
660 logic csr_is_irq;
661 logic is_dbg_taken;
662 logic [2:0] saved_debug_cause;
663 integer next_send;
664
665 event e_empty_queue;
666 function void empty_fifo();
667     integer i, trace_q_size;
668     trace_q_size = wb_bypass_trace_q.size();
669     if (trace_q_size > 50) begin
670         $fatal("Reorder queue too long\n");
671     end
672     if (trace_q_size != 0) begin
673         for (i = 0; i < trace_q_size; i++) begin
674             insn_trace_t new_rvfi_trace;
675             new_rvfi_trace = new();
676             new_r-
677             vfi_trace.copy_full(wb_bypass_trace_q.pop_front());
678             if (next_send == new_rvfi_trace.m_order) begin
679                 new_rvfi_trace.m_csr.mstatus_fs_rdata =
r_pipe_freeze_trace.csr.mstatus_fs_n;
680                 rvfi_trace_q.push_back(new_rvfi_trace);
681                 next_send = next_send + 1;
682                 ->e_empty_queue;
683             end else begin
684                 wb_bypass_trace_q.push_back(new_rvfi_trace);
685             end
686         end
687     end
688     trace_q_size = wb_bypass_trace_q.size(); //Re-calculate
here for accurate status
689 endfunction
690 /*
691 * Function used to allocate a new insn and send it to the
rvfi driver
692 */
693 event e_add_to_bypass;
694 function void send_rvfi(insn_trace_t m_wb_insn);
695     insn_trace_t new_rvfi_trace;
696     new_rvfi_trace = new();
697     new_rvfi_trace.copy_full(m_wb_insn);
698     if (next_send == new_rvfi_trace.m_order) begin
699         rvfi_trace_q.push_back(new_rvfi_trace);
700         next_send = next_send + 1;
701     end else begin
702         wb_bypass_trace_q.push_back(new_rvfi_trace);
703         ->e_add_to_bypass;
704     end
705     empty_fifo();
706 endfunction
707
708 /*
709 * Assing rvfi signals once the instruction is completed
710 */
711 `define SET_RVFI_CSR_FROM_INSN(CSR_NAME) \
712     rvfi_csr_`CSR_NAME`_rdata = new_r-
vfi_trace.m_csr_`CSR_NAME`_rdata; \
713     rvfi_csr_`CSR_NAME`_rmask = new_r-
vfi_trace.m_csr_`CSR_NAME`_rmask; \

```

```

714     rvfi_csr_`CSR_NAME`_wdata = new_r-
vfi_trace.m_csr_`CSR_NAME`_wdata; \
715     rvfi_csr_`CSR_NAME`_wmask = new_r-
vfi_trace.m_csr_`CSR_NAME`_wmask;
716
717 logic [31:0] s_fflags_mirror;
718 logic [31:0] s_frm_mirror;
719 logic [31:0] s_fcsr_mirror;
720
721 function void set_rvfi();
722     insn_trace_t new_rvfi_trace;
723     new_rvfi_trace = rvfi_trace_q.pop_front();
724
725     if (new_rvfi_trace.m_is_apu) begin
726         if (new_rvfi_trace.m_csr.fflags_we) begin
727             s_fflags_mirror = new_rvfi_trace.m_csr.fflags_wdata;
728         end else begin
729             s_fflags_mirror = new_rvfi_trace.m_csr.fflags_rdata;
730         end
731         if (new_rvfi_trace.m_csr.frm_we) begin
732             s_frm_mirror = new_rvfi_trace.m_csr.frm_wdata;
733         end else begin
734             s_frm_mirror = new_rvfi_trace.m_csr.frm_rdata;
735         end
736         if (new_rvfi_trace.m_csr.fcsr_we) begin
737             s_fcsr_mirror = new_rvfi_trace.m_csr.fcsr_wdata;
738         end else begin
739             s_fcsr_mirror = new_rvfi_trace.m_csr.fcsr_rdata;
740         end
741     end else begin
742         new_rvfi_trace.m_csr.fflags_rdata = s_fflags_mirror;
743         new_rvfi_trace.m_csr.frm_rdata = s_frm_mirror;
744         new_rvfi_trace.m_csr.fcsr_rdata = s_fcsr_mirror;
745
746         if (new_rvfi_trace.m_fflags_we_non_apu) begin
747             s_fflags_mirror = new_rvfi_trace.m_csr.fflags_wdata;
748             s_fcsr_mirror = new_rvfi_trace.m_csr.fcsr_wdata;
749             new_rvfi_trace.m_csr.fflags_wmask = 32'hFFFF_FFFF;
750             new_rvfi_trace.m_csr.fcsr_wmask = 32'hFFFF_FFFF;
751         end
752         if (new_rvfi_trace.m_frm_we_non_apu) begin
753             s_frm_mirror = new_rvfi_trace.m_csr.frm_wdata;
754             s_fcsr_mirror = new_rvfi_trace.m_csr.fcsr_wdata;
755             new_rvfi_trace.m_csr.frm_wmask = 32'hFFFF_FFFF;
756             new_rvfi_trace.m_csr.fcsr_wmask = 32'hFFFF_FFFF;
757         end
758         if (new_rvfi_trace.m_fcsr_we_non_apu) begin
759             s_fcsr_mirror = new_rvfi_trace.m_csr.fcsr_wdata;
760             s_fflags_mirror = new_rvfi_trace.m_csr.fflags_wdata;
761             s_frm_mirror = new_rvfi_trace.m_csr.frm_wdata;
762             new_rvfi_trace.m_csr.fcsr_wmask = 32'hFFFF_FFFF;
763             new_rvfi_trace.m_csr.fflags_wmask = 32'hFFFF_FFFF;
764             new_rvfi_trace.m_csr.frm_wmask = 32'hFFFF_FFFF;
765         end
766     end
767
768 rvfi_order = new_rvfi_trace.m_order;
769 rvfi_start_cycle = new_rvfi_trace.m_start_cycle;
770 rvfi_start_time = new_rvfi_trace.m_start_time;
771 rvfi_stop_cycle = new_rvfi_trace.m_stop_cycle;
772 rvfi_stop_time = new_rvfi_trace.m_stop_time;
773 rvfi_pc_rdata = new_rvfi_trace.m_pc_rdata;
774 rvfi_insn = new_rvfi_trace.m_insn;
775
776 rvfi_rs1_addr = '0;
777 rvfi_rs1_rdata = '0;
778 rvfi_rs2_addr = '0;
779 rvfi_rs2_rdata = '0;
780 rvfi_rs3_addr = '0;
781 rvfi_rs3_rdata = '0;
782 rvfi_frs1_rvalid = '0;
783 rvfi_frs1_addr = '0;
784 rvfi_frs1_rdata = '0;
785 rvfi_frs2_rvalid = '0;
786 rvfi_frs2_addr = '0;
787 rvfi_frs2_rdata = '0;
788 rvfi_frs3_rvalid = '0;
789 rvfi_frs3_addr = '0;
790 rvfi_frs3_rdata = '0;
791
792 if (new_rvfi_trace.m_rs1_addr[5]) begin
793     rvfi_frs1_rvalid = 1'b1;
794     rvfi_frs1_addr = new_rvfi_trace.m_rs1_addr[4:0];
795     rvfi_frs1_rdata = new_rvfi_trace.m_rs1_rdata;
796 end else begin
797     rvfi_rs1_addr = new_rvfi_trace.m_rs1_addr[4:0];
798     rvfi_rs1_rdata = new_rvfi_trace.m_rs1_rdata;
799 end
800
801 if (new_rvfi_trace.m_rs2_addr[5]) begin
802     rvfi_frs2_rvalid = 1'b1;
803     rvfi_frs2_addr = new_rvfi_trace.m_rs2_addr[4:0];
804     rvfi_frs2_rdata = new_rvfi_trace.m_rs2_rdata;
805 end else begin
806     rvfi_rs2_addr = new_rvfi_trace.m_rs2_addr[4:0];
807     rvfi_rs2_rdata = new_rvfi_trace.m_rs2_rdata;
808 end
809
810 if (new_rvfi_trace.m_rs3_addr[5]) begin
811     rvfi_frs3_rvalid = 1'b1;

```



```

813     rvfi_frs3_addr = new_rvfi_trace.m_rs3_addr[4:0];
814     rvfi_frs3_rdata = new_rvfi_trace.m_rs3_rdata;
815   end else begin
816     rvfi_rs3_addr = new_rvfi_trace.m_rs3_addr[4:0];
817     rvfi_rs3_rdata = new_rvfi_trace.m_rs3_rdata;
818   end
819
820   rvfi_frd_wvalid[0] = '0';
821   rvfi_frd_addr[0] = '0';
822   rvfi_frd_wdata[0] = '0';
823
824   rvfi_frd_wvalid[1] = '0';
825   rvfi_frd_addr[1] = '0';
826   rvfi_frd_wdata[1] = '0';
827
828   rvfi_2_rd = new_rvfi_trace.m_2_rd_insn;
829   if (new_rvfi_trace.m_rd_addr[0][5] == 1'b0) begin
830     rvfi_rd_addr[0] = new_rvfi_trace.m_rd_addr[0][4:0];
831     rvfi_rd_wdata[0] = new_rvfi_trace.m_rd_wdata[0];
832   end else begin
833     rvfi_rd_addr[0] = '0';
834     rvfi_rd_wdata[0] = '0';
835
836     rvfi_frd_wvalid[0] = 1'b1;
837     rvfi_frd_addr[0] = new_rvfi_trace.m_rd_addr[0][4:0];
838     rvfi_frd_wdata[0] = new_rvfi_trace.m_rd_wdata[0];
839   end
840
841   if (new_rvfi_trace.m_2_rd_insn) begin
842     if (new_rvfi_trace.m_rd_addr[1][5] == 1'b0) begin
843       rvfi_rd_addr[1] = new_rvfi_trace.m_rd_addr[1][4:0];
844       rvfi_rd_wdata[1] = new_rvfi_trace.m_rd_wdata[1];
845     end else begin
846       rvfi_frd_wvalid[1] = 1'b1;
847       rvfi_frd_addr[1] = new_rvfi_trace.m_rd_addr[1][4:0];
848       rvfi_frd_wdata[1] = new_rvfi_trace.m_rd_wdata[1];
849     end
850
851     rvfi_rd_addr[1] = '0';
852     rvfi_rd_wdata[1] = '0';
853   end
854   end else begin
855     rvfi_rd_addr[1] = '0';
856     rvfi_rd_wdata[1] = '0';
857   end
858
859   rvfi_mem_addr = new_rvfi_trace.m_mem_addr;
860   rvfi_mem_rmask = new_rvfi_trace.m_mem_rmask;
861   rvfi_mem_wmask = new_rvfi_trace.m_mem_wmask;
862   rvfi_mem_rdata = new_rvfi_trace.m_mem_rdata;
863   rvfi_mem_wdata = new_rvfi_trace.m_mem_wdata;
864
865   rvfi_dbg = new_rvfi_trace.m_dbg_cause;
866   rvfi_dbg_mode = new_rvfi_trace.m_dbg_taken;
867
868   rvfi_trap.trap = 0;
869   if (new_rvfi_trace.m_is_illegal) begin
870     rvfi_trap.trap = 1'b1;
871   end
872
873   if (new_rvfi_trace.m_trap) begin
874     rvfi_trap.trap = 1'b1;
875   end
876
877   //CSR
878   rvfi_csr_mstatus_rmask = new_r-
vfi_trace.m_csr.mstatus_rmask | new_r-
vfi_trace.m_csr.mstatus_fs_rmask;
879   rvfi_csr_mstatus_wmask = new_r-
vfi_trace.m_csr.mstatus_wmask & MSTATUS_WRITE_MASK;
880   rvfi_csr_mstatus_wmask[31] = new_r-
vfi_trace.m_csr.mstatus_fs_wmask[31];
881   rvfi_csr_mstatus_wmask[14:13] = new_r-
vfi_trace.m_csr.mstatus_fs_wmask[14:13];
882
883   if (FPU == 1 && ZFINX == 0) begin
884     rvfi_csr_mstatus_rdata[31] = (new_r-
vfi_trace.m_csr.mstatus_fs_rdata == FS_DIRTY) ? 1'b1 : 1'b0;
885   end else begin
886     rvfi_csr_mstatus_rdata[31] = '0';
887   end
888   rvfi_csr_mstatus_rdata[30:18] = '0';
889   rvfi_csr_mstatus_rdata[17] = new_r-
vfi_trace.m_csr.mstatus_rdata.mprv;
890   rvfi_csr_mstatus_rdata[16:15] = '0';
891   if (FPU == 1 && ZFINX == 0) begin
892     rvfi_csr_mstatus_rdata[14:13] = new_r-
vfi_trace.m_csr.mstatus_fs_rdata;
893   end else begin
894     rvfi_csr_mstatus_rdata[14:13] = '0';
895   end
896   rvfi_csr_mstatus_rdata[12:11] = new_r-
vfi_trace.m_csr.mstatus_rdata.mpp;
897   rvfi_csr_mstatus_rdata[10:8] = '0';
898   rvfi_csr_mstatus_rdata[7] = new_r-
vfi_trace.m_csr.mstatus_rdata.mpie;
899   rvfi_csr_mstatus_rdata[6:5] = '0';
900   rvfi_csr_mstatus_rdata[4] = new_r-
vfi_trace.m_csr.mstatus_rdata.upie;
901   rvfi_csr_mstatus_rdata[3] = new_r-
vfi_trace.m_csr.mstatus_rdata.mie;

```

```

902   rvfi_csr_mstatus_rdata[2:1] = '0';
903   rvfi_csr_mstatus_rdata[0] = new_r-
vfi_trace.m_csr.mstatus_rdata.uiie;
904
905   if (FPU == 1 && ZFINX == 0) begin
906     rvfi_csr_mstatus_wdata[31] = (new_r-
vfi_trace.m_csr.mstatus_fs_wdata == FS_DIRTY) ? 1'b1 : 1'b0;
907   end else begin
908     rvfi_csr_mstatus_wdata[31] = '0';
909   end
910   rvfi_csr_mstatus_wdata[30:18] = '0';
911   // MPRV is not implemented in the target configuration,
writes to it are ignored
912   rvfi_csr_mstatus_wdata[17] = 1'b0; //new_r-
vfi_trace.m_csr.mstatus_wdata.mprv;
913   rvfi_csr_mstatus_wdata[16:15] = '0';
914   if (FPU == 1 && ZFINX == 0) begin
915     rvfi_csr_mstatus_wdata[14:13] = new_r-
vfi_trace.m_csr.mstatus_fs_wdata;
916   end else begin
917     rvfi_csr_mstatus_wdata[14:13] = '0';
918   end
919   rvfi_csr_mstatus_wdata[12:11] = new_r-
vfi_trace.m_csr.mstatus_wdata.mpp;
920   rvfi_csr_mstatus_wdata[10:8] = '0';
921   rvfi_csr_mstatus_wdata[7] = new_r-
vfi_trace.m_csr.mstatus_wdata.mpie;
922   rvfi_csr_mstatus_wdata[6:5] = '0';
923   // UPIE is not implemented in the target configuration,
writes to it are ignored
924   rvfi_csr_mstatus_wdata[4] = 1'b0; //new_r-
vfi_trace.m_csr.mstatus_wdata.upie;
925   rvfi_csr_mstatus_wdata[3] = new_r-
vfi_trace.m_csr.mstatus_wdata.mie;
926   rvfi_csr_mstatus_wdata[2:1] = '0';
927   // UIE is not implemented in the target configuration,
writes to it are ignored
928   rvfi_csr_mstatus_wdata[0] = 1'b0; //new_r-
vfi_trace.m_csr.mstatus_wdata.uiie;
929
930   `SET_RVFI_CSR_FROM_INSN(misa)
931   `SET_RVFI_CSR_FROM_INSN(mie)
932   `SET_RVFI_CSR_FROM_INSN(mtvec)
933
934   rvfi_csr_mcountinhibit_rdata = new_r-
vfi_trace.m_csr.mcountinhibit_rdata;
935   rvfi_csr_mcountinhibit_rmask = new_r-
vfi_trace.m_csr.mcountinhibit_rmask;
936   rvfi_csr_mcountinhibit_wdata = new_r-
vfi_trace.m_csr.mcountinhibit_wdata;
937   rvfi_csr_mcountinhibit_wmask = new_r-
vfi_trace.m_csr.mcountinhibit_wmask & MCOUNTINHIBIT_WRITE_MASK;
938
939   `SET_RVFI_CSR_FROM_INSN(mscratch)
940   `SET_RVFI_CSR_FROM_INSN(mepc)
941   `SET_RVFI_CSR_FROM_INSN(mcause)
942   `SET_RVFI_CSR_FROM_INSN(mcycle)
943   `SET_RVFI_CSR_FROM_INSN(minstret)
944   `SET_RVFI_CSR_FROM_INSN(minstreth)
945
946   // `SET_RVFI_CSR_FROM_INSN(cycle)
947   // `SET_RVFI_CSR_FROM_INSN(instret)
948   rvfi_csr_instret_rdata = new_r-
vfi_trace.m_csr.minstret_rdata;
949   rvfi_csr_instret_rmask = new_r-
vfi_trace.m_csr.minstret_rmask;
950   rvfi_csr_instret_wdata = new_r-
vfi_trace.m_csr.minstret_wdata;
951   rvfi_csr_instret_wmask = new_r-
vfi_trace.m_csr.minstret_wmask;
952
953   for(int idx=3; idx<32; idx++) begin
954     rvfi_csr_mhpmcounter_rmask[idx] = new_r-
vfi_trace.m_csr.mhpmcounter_rmask[idx][31:0];
955     rvfi_csr_mhpmcounter_wmask[idx] = new_r-
vfi_trace.m_csr.mhpmcounter_wmask[idx][31:0];
956     rvfi_csr_mhpmcounter_rdata[idx] = new_r-
vfi_trace.m_csr.mhpmcounter_rdata[idx][31:0];
957     rvfi_csr_mhpmcounter_wdata[idx] = new_r-
vfi_trace.m_csr.mhpmcounter_wdata[idx][31:0];
958
959     rvfi_csr_mhpmcounterh_rmask[idx] = new_r-
vfi_trace.m_csr.mhpmcounterh_rmask[idx][63:32];
960     rvfi_csr_mhpmcounterh_wmask[idx] = new_r-
vfi_trace.m_csr.mhpmcounterh_wmask[idx][63:32];
961     rvfi_csr_mhpmcounterh_rdata[idx] = new_r-
vfi_trace.m_csr.mhpmcounterh_rdata[idx][63:32];
962     rvfi_csr_mhpmcounterh_wdata[idx] = new_r-
vfi_trace.m_csr.mhpmcounterh_wdata[idx][63:32];
963
964     rvfi_csr_mhpmevent_rmask[idx] = new_r-
vfi_trace.m_csr.mhpmevent_rmask[idx];
965     rvfi_csr_mhpmevent_wmask[idx] = new_r-
vfi_trace.m_csr.mhpmevent_wmask[idx] & MHPMEVENT_WRITE_MASK;
966     rvfi_csr_mhpmevent_rdata[idx] = new_r-
vfi_trace.m_csr.mhpmevent_rdata[idx];
967     rvfi_csr_mhpmevent_wdata[idx] = new_r-
vfi_trace.m_csr.mhpmevent_wdata[idx];
968   end
969   // `SET_RVFI_CSR_FROM_INSN(instreth)
970   rvfi_csr_instreth_rdata = new_r-

```

```

vfi_trace.m_csr.minstreth_rdata;
971   rvfi_csr_instreth_rmask = new_r-
vfi_trace.m_csr.minstreth_rmask;
972   rvfi_csr_instreth_wdata = new_r-
vfi_trace.m_csr.minstreth_wdata;
973   rvfi_csr_instreth_wmask = new_r-
vfi_trace.m_csr.minstreth_wmask;
974
975   `SET_RVFI_CSR_FROM_INSN(mip)
976
977   rvfi_csr_tdata_rdata[0] = 'Z';
978   rvfi_csr_tdata_rmask[0] = '0'; // Does not exist
979   rvfi_csr_tdata_wdata[0] = 'Z'; // Does not exist
980   rvfi_csr_tdata_wmask[0] = '0';
981
982   rvfi_csr_tdata_rdata[1] = new_r-
vfi_trace.m_csr.tdata1_rdata;
983   rvfi_csr_tdata_rmask[1] = new_r-
vfi_trace.m_csr.tdata1_rmask; // '1
984   rvfi_csr_tdata_wdata[1] = new_r-
vfi_trace.m_csr.tdata1_wdata;
985   rvfi_csr_tdata_wmask[1] = new_r-
vfi_trace.m_csr.tdata1_wmask;
986
987   rvfi_csr_tdata_rdata[2] = new_r-
vfi_trace.m_csr.tdata2_rdata;
988   rvfi_csr_tdata_rmask[2] = new_r-
vfi_trace.m_csr.tdata2_rmask; // '1
989   rvfi_csr_tdata_wdata[2] = new_r-
vfi_trace.m_csr.tdata2_wdata;
990   rvfi_csr_tdata_wmask[2] = new_r-
vfi_trace.m_csr.tdata2_wmask;
991
992   rvfi_csr_tdata_rmask[3] = '1';
993
994   `SET_RVFI_CSR_FROM_INSN(tinfo)
995   `SET_RVFI_CSR_FROM_INSN(dcsr)
996   `SET_RVFI_CSR_FROM_INSN(dpc)
997   rvfi_csr_dscratch_rmask[0] = new_r-
vfi_trace.m_csr.dscratch0_rmask;
998   rvfi_csr_dscratch_wmask[0] = new_r-
vfi_trace.m_csr.dscratch0_wmask;
999   rvfi_csr_dscratch_rdata[0] = new_r-
vfi_trace.m_csr.dscratch0_rdata;
1000  rvfi_csr_dscratch_wdata[0] = new_r-
vfi_trace.m_csr.dscratch0_wdata;
1001  rvfi_csr_dscratch_rmask[1] = new_r-
vfi_trace.m_csr.dscratch1_rmask;
1002  rvfi_csr_dscratch_wmask[1] = new_r-
vfi_trace.m_csr.dscratch1_wmask;
1003  rvfi_csr_dscratch_rdata[1] = new_r-
vfi_trace.m_csr.dscratch1_rdata;
1004  rvfi_csr_dscratch_wdata[1] = new_r-
vfi_trace.m_csr.dscratch1_wdata;
1005  // `SET_RVFI_CSR_FROM_INSN(dscratch0)
1006  `SET_RVFI_CSR_FROM_INSN(mvendordid)
1007  `SET_RVFI_CSR_FROM_INSN(marchid)
1008
1009  `SET_RVFI_CSR_FROM_INSN(fflags)
1010  `SET_RVFI_CSR_FROM_INSN(frm)
1011  `SET_RVFI_CSR_FROM_INSN(fcsr)
1012
1013  `SET_RVFI_CSR_FROM_INSN(lpstart0)
1014  `SET_RVFI_CSR_FROM_INSN(lpend0)
1015  `SET_RVFI_CSR_FROM_INSN(lpcount0)
1016  `SET_RVFI_CSR_FROM_INSN(lpstart1)
1017  `SET_RVFI_CSR_FROM_INSN(lpend1)
1018  `SET_RVFI_CSR_FROM_INSN(lpcount1)
1019
1020  endfunction // set_rvfi
1021
1022  function void sample_perf_counter_to_id(int idx);
1023    trace_id.m_csr.mhpmcounter_rdata[idx][31:0] =
r_pipe_freeze_trace.csr.mhpmcounter_q[idx][31:0];
1024    trace_id.m_csr.mhpmcounter_rmask[idx][31:0] = '1';
1025  endfunction
1026
1027  function void perf_counter_to_id(int idx);
1028    if(!trace_id.m_csr.mhpmcounter_we[idx][0]) begin
1029      trace_id.m_csr.mhpmcounter_wdata[idx][31:0] =
r_pipe_freeze_trace.csr.wdata_int;
1030    end
1031    if(r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx])
begin
1032      trace_id.m_csr.mhpmcounter_we[idx][0] =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx];
1033      trace_id.m_csr.mhpmcounter_wdata[idx][31:0] =
r_pipe_freeze_trace.csr.wdata_int;
1034      trace_id.m_csr.mhpmcounter_wmask[idx][31:0] =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx] ? '1 : '0;
1035    end
1036    sample_perf_counter_to_id(idx);
1037  endfunction
1038
1039  function void sample_perf_event_to_trace(int idx,
insn_trace_t m_trace);
1040    m_trace.m_csr.mhpmevent_rdata[idx] =
r_pipe_freeze_trace.csr.mhpmevent_q[idx];
1041    m_trace.m_csr.mhpmevent_rmask[idx] = '1';
1042  endfunction
1043

```

```

1044  function void perf_event_to_trace(int idx, insn_trace_t
m_trace);
1045    if(!m_trace.m_csr.mhpmevent_we[idx]) begin
1046      m_trace.m_csr.mhpmevent_wdata[idx] =
r_pipe_freeze_trace.csr.wdata_int;
1047    end
1048    if(r_pipe_freeze_trace.csr.mhpmevent_we[idx]) begin
1049      m_trace.m_csr.mhpmevent_we[idx] =
r_pipe_freeze_trace.csr.mhpmevent_we[idx];
1050      m_trace.m_csr.mhpmevent_wdata[idx] =
r_pipe_freeze_trace.csr.wdata_int;
1051      m_trace.m_csr.mhpmevent_wmask[idx] =
r_pipe_freeze_trace.csr.mhpmevent_we[idx] ? '1 : '0;
1052    end
1053    sample_perf_event_to_trace(idx, m_trace);
1054  endfunction
1055
1056  function void sample_minstret_to_trace(insn_trace_t
m_trace);
1057    m_trace.m_csr.minstret_rdata =
r_pipe_freeze_trace.csr.mhpmcounter_q[2][31:0];
1058    m_trace.m_csr.minstret_rmask = '1';
1059  endfunction
1060
1061  function void minstret_to_trace(insn_trace_t m_trace);
1062    if(!m_trace.m_csr.minstret_we) begin
1063      m_trace.m_csr.minstret_wdata =
r_pipe_freeze_trace.csr.wdata_int;
1064    end
1065    if(r_pipe_freeze_trace.csr.mhpmcounter_write_lower[2]) be-
gin
1066      m_trace.m_csr.minstret_we =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[2];
1067      m_trace.m_csr.minstret_wdata =
r_pipe_freeze_trace.csr.wdata_int;
1068      m_trace.m_csr.minstret_wmask =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[2] ? '1 : '0;
1069    end
1070    sample_minstret_to_trace(m_trace);
1071  endfunction
1072
1073  function void sample_perf_counter_h_to_id(int idx);
1074    trace_id.m_csr.mhpmcounter_rdata[idx][63:32] =
r_pipe_freeze_trace.csr.mhpmcounter_q[idx][63:0];
1075    trace_id.m_csr.mhpmcounter_rmask[idx][63:32] = '1';
1076  endfunction
1077
1078  function void perf_counter_h_to_id(int idx);
1079    if(!trace_id.m_csr.mhpmcounter_we[idx][1]) begin
1080      trace_id.m_csr.mhpmcounter_wdata[idx][63:32] =
r_pipe_freeze_trace.csr.wdata_int;
1081    end
1082    if(r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx])
begin
1083      trace_id.m_csr.mhpmcounter_we[idx][1] =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx];
1084      trace_id.m_csr.mhpmcounter_wdata[idx][63:32] =
r_pipe_freeze_trace.csr.wdata_int;
1085      trace_id.m_csr.mhpmcounter_wmask[idx][63:32] =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx] ? '1 : '0;
1086    end
1087    sample_perf_counter_h_to_id(idx);
1088  endfunction
1089
1090  function void sample_minstreth_to_trace(insn_trace_t
m_trace);
1091    m_trace.m_csr.minstreth_rdata =
r_pipe_freeze_trace.csr.mhpmcounter_q[2][63:32];
1092    m_trace.m_csr.minstreth_rmask = '1';
1093  endfunction
1094
1095  function void sample_mcycle_to_trace(insn_trace_t m_trace);
1096    m_trace.m_csr.mcycle_we =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[0];
1097    m_trace.m_csr.mcycle_rdata =
r_pipe_freeze_trace.csr.mhpmcounter_q[0][31:0];
1098    m_trace.m_csr.mcycle_rmask = '1';
1099    m_trace.m_csr.mcycle_wdata =
r_pipe_freeze_trace.csr.mhpmcounter_q[31:0];
1100    m_trace.m_csr.mcycle_wmask =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[0] ? '1 : '0;
1101  endfunction
1102
1103  function void minstreth_to_trace(insn_trace_t m_trace);
1104    if(!m_trace.m_csr.minstreth_we) begin
1105      m_trace.m_csr.minstreth_wdata =
r_pipe_freeze_trace.csr.wdata_int;
1106    end
1107    if(r_pipe_freeze_trace.csr.mhpmcounter_write_upper[2]) be-
gin
1108      m_trace.m_csr.minstreth_we =
r_pipe_freeze_trace.csr.mhpmcounter_write_upper[2];
1109      m_trace.m_csr.minstreth_wdata =
r_pipe_freeze_trace.csr.wdata_int;
1110      m_trace.m_csr.minstreth_wmask =
r_pipe_freeze_trace.csr.mhpmcounter_write_upper[2] ? '1 : '0;
1111    end
1112    sample_minstreth_to_trace(m_trace);
1113  endfunction
1114
1115  function void sample_perf_counter_to_trace(insn_trace_t

```



```

m_trace);
1116 sample_minstret_to_trace(m_trace);
1117 sample_minstreth_to_trace(m_trace);
1118 sample_mcycle_to_trace(m_trace);
1119 for(int idx=3; idx<32; idx++)begin
1120   sample_perf_event_to_trace(idx, m_trace); //TO CHANGE
1121 end
1122 endfunction
1123
1124 function void perf_counter_to_trace(insn_trace_t m_trace);
1125   if(r_pipe_freeze_trace.csr.mhpmcounter_write_lower[2])
begin
1126     minstret_to_trace(m_trace);
1127   end
1128   if(r_pipe_freeze_trace.csr.mhpmcounter_write_upper[2])
begin
1129     minstreth_to_trace(m_trace);
1130   end
1131   for(int idx=3; idx<32; idx++) begin
1132     if(r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx]) begin
1133       perf_counter_to_id(idx);
1134     end
1135   end
1136   if(r_pipe_freeze_trace.csr.mhpmcounter_write_upper[idx]) begin
1137     perf_counter_h_to_id(idx);
1138   end
1139   if(r_pipe_freeze_trace.csr.mhpmevent_we[idx]) begin
1140     perf_event_to_trace(idx, m_trace);
1141   end
1142 endfunction
1143
1144 function void tinfo_to_trace(insn_trace_t m_trace);
1145   m_trace.m_csr.tinfo_we = '0'; // READ ONLY csr_t-
info_we_i;
1146   m_trace.m_csr.tinfo_rdata =
r_pipe_freeze_trace.csr.tinfo_q;
1147   m_trace.m_csr.tinfo_rmask = '1;
1148   m_trace.m_csr.tinfo_wdata =
r_pipe_freeze_trace.csr.tinfo_n;
1149   m_trace.m_csr.tinfo_wmask = '0;
1150 endfunction
1151
1152   function void mtvec_to_id();
1153     trace_id.m_csr.mtvec_we =
r_pipe_freeze_trace.csr.mtvec_we;
1154     trace_id.m_csr.mtvec_rdata = {
1155       r_pipe_freeze_trace.csr.mtvec_q, 6'h0,
r_pipe_freeze_trace.csr.mtvec_mode_q
1156     };
1157     trace_id.m_csr.mtvec_rmask = '1;
1158     trace_id.m_csr.mtvec_wdata = {
1159       r_pipe_freeze_trace.csr.mtvec_n, 6'h0,
r_pipe_freeze_trace.csr.mtvec_mode_n
1160     };
1161   };
1162   trace_id.m_csr.mtvec_wmask =
r_pipe_freeze_trace.csr.mtvec_we ? '1 : '0;
1163 endfunction
1164
1165   function void dcsr_to_id();
1166     trace_id.m_csr.dcsr_wdata = trace_id.m_csr.dcsr_we ?
trace_id.m_csr.dcsr_wdata : r_pipe_freeze_trace.csr.dcsr_n;
1167     trace_id.m_csr.dcsr_we = r_pipe_freeze_trace.csr.dcsr_we |
trace_id.m_csr.dcsr_we;
1168     trace_id.m_csr.dcsr_rdata =
r_pipe_freeze_trace.csr.dcsr_q;
1169     trace_id.m_csr.dcsr_rmask = '1;
1170     trace_id.m_csr.dcsr_wdata =
r_pipe_freeze_trace.csr.dcsr_n;
1171     trace_id.m_csr.dcsr_wmask =
r_pipe_freeze_trace.csr.dcsr_we ? '1 : '0;
1172 endfunction
1173
1174   function void csrId_to_id();
1175     trace_id.m_csr.mvendorid_we = '0;
1176     trace_id.m_csr.mvendorid_rdata =
r_pipe_freeze_trace.csr.mvendorid;
1177     trace_id.m_csr.mvendorid_rmask = '1;
1178     trace_id.m_csr.mvendorid_wdata = '0;
1179     trace_id.m_csr.mvendorid_wmask = '0;
1180
1181     trace_id.m_csr.marchid_we = '0;
1182     trace_id.m_csr.marchid_rdata =
r_pipe_freeze_trace.csr.marchid;
1183     trace_id.m_csr.marchid_rmask = '0;
1184     trace_id.m_csr.marchid_wdata = '0;
1185     trace_id.m_csr.marchid_wmask = '0;
1186 endfunction
1187
1188   function void lpcount1_to_id();
1189     trace_id.m_csr.lpcount1_we = '0;
1190     trace_id.m_csr.lpcount1_rdata =
r_pipe_freeze_trace.hwloop.counter_q[1];
1191     trace_id.m_csr.lpcount1_rmask = '1;
1192     trace_id.m_csr.lpcount1_wdata = '0;
1193     trace_id.m_csr.lpcount1_wmask = '0;
1194 endfunction
1195

```

```

1196   function void lpcount0_to_id();
1197     trace_id.m_csr.lpcount0_we = '0;
1198     trace_id.m_csr.lpcount0_rdata =
r_pipe_freeze_trace.hwloop.counter_q[0];
1199     trace_id.m_csr.lpcount0_rmask = '1;
1200     trace_id.m_csr.lpcount0_wdata = '0;
1201     trace_id.m_csr.lpcount0_wmask = '0;
1202 endfunction
1203
1204   function void lpend0_to_id();
1205     trace_id.m_csr.lpend0_we = '0;
1206     trace_id.m_csr.lpend0_rdata =
r_pipe_freeze_trace.hwloop.end_q[0];
1207     trace_id.m_csr.lpend0_rmask = '1;
1208     trace_id.m_csr.lpend0_wdata = '0;
1209     trace_id.m_csr.lpend0_wmask = '0;
1210 endfunction
1211
1212   function void lpend1_to_id();
1213     trace_id.m_csr.lpend1_we = '0;
1214     trace_id.m_csr.lpend1_rdata =
r_pipe_freeze_trace.hwloop.end_q[1];
1215     trace_id.m_csr.lpend1_rmask = '1;
1216     trace_id.m_csr.lpend1_wdata = '0;
1217     trace_id.m_csr.lpend1_wmask = '0;
1218 endfunction
1219
1220   function void lpstart0_to_id();
1221     trace_id.m_csr.lpstart0_we = '0;
1222     trace_id.m_csr.lpstart0_rdata =
r_pipe_freeze_trace.hwloop.start_q[0];
1223     trace_id.m_csr.lpstart0_rmask = '1;
1224     trace_id.m_csr.lpstart0_wdata = '0;
1225     trace_id.m_csr.lpstart0_wmask = '0;
1226 endfunction
1227
1228   function void lpstart1_to_id();
1229     trace_id.m_csr.lpstart1_we = '0;
1230     trace_id.m_csr.lpstart1_rdata =
r_pipe_freeze_trace.hwloop.start_q[1];
1231     trace_id.m_csr.lpstart1_rmask = '1;
1232     trace_id.m_csr.lpstart1_wdata = '0;
1233     trace_id.m_csr.lpstart1_wmask = '0;
1234 endfunction
1235
1236   function void hwloop_to_id();
1237     lpcount0_to_id();
1238     lpend0_to_id();
1239     lpstart0_to_id();
1240
1241     lpcount1_to_id();
1242     lpend1_to_id();
1243     lpstart1_to_id();
1244 endfunction
1245
1246   bit s_was_flush; //debug exception is flagged as trap only
if preceed by a flush
1247   //work around until I find the coreect way to distinguish
trap
1248   function void check_trap();
1249     bit s_dbg_exception, s_exception, s_irq;
1250     s_dbg_exception = '1'b0;
1251     s_exception = '1'b0;
1252     s_irq = '1'b0;
1253
1254     if (r_pipe_freeze_trace.pc_mux == PC_EXCEPTION) begin
1255       if (r_pipe_freeze_trace.exc_pc_mux[1] == '1'b1) begin
1256         if (r_pipe_freeze_trace.ctrl_fsm_cs != XRET_JUMP) be-
gin
1257           s_dbg_exception = '1'b1;
1258         end
1259       end
1260       if (r_pipe_freeze_trace.exc_pc_mux == EXC_PC_EXCEPTION)
begin
1261         s_exception = '1'b1;
1262       end
1263       if (r_pipe_freeze_trace.exc_pc_mux == EXC_PC_IRQ) begin
1264         s_irq = '1'b1;
1265         trace_if.m_is_irq = '1'b1;
1266       end
1267     end
1268
1269     if (s_was_flush == '1'b0) begin
1270       s_dbg_exception = '1'b0;
1271     end
1272
1273   endfunction
1274   /*
1275    * This tracer works with three process,
1276
1277   * The first one simply stores information from the pipeline
every clock cycles and sends those information to the second process
1278   *
1279   * The second process uses those information to fill the
rvfi interface with the correct values
1280   *
1281   * The third updates the rvfi interface
1282   */
1283   `define CSR_FROM_PIPE(TRACE_NAME, CSR_NAME) \
1284     if(!trace.`TRACE_NAME`.m_csr.`CSR_NAME`.we) begin \

```

```

1285     trace_`TRACE_NAME`.m_csr.`CSR_NAME`.wdata =
r_pipe_freeze_trace.csr.`CSR_NAME`.n; \
1286     end \
1287     if (r_pipe_freeze_trace.csr.`CSR_NAME`.we) begin \
1288         trace_`TRACE_NAME`.m_csr.`CSR_NAME`.we =
r_pipe_freeze_trace.csr.`CSR_NAME`.we; \
1289         trace_`TRACE_NAME`.m_csr.`CSR_NAME`.wdata =
r_pipe_freeze_trace.csr.`CSR_NAME`.n; \
1290         trace_`TRACE_NAME`.m_csr.`CSR_NAME`.wmask = '1'; \
1291     end \
1292     trace_`TRACE_NAME`.m_csr.`CSR_NAME`.rdata =
r_pipe_freeze_trace.csr.`CSR_NAME`.q; \
1293     trace_`TRACE_NAME`.m_csr.`CSR_NAME`.rmask = '1';
1294
1295     event e_mstatus_to_id;
1296     event e_fregs_dirty_1, e_fregs_dirty_2, e_fregs_dirty_3;
1297     function void mstatus_to_id();
1298         `CSR_FROM_PIPE(id, mstatus)
1299         `CSR_FROM_PIPE(id, mstatus_fs)
1300         if(r_pipe_freeze_trace.csr.fregs_we && !
r_pipe_freeze_trace.csr.mstatus_fs_we && !
(r_pipe_freeze_trace.csr.we &&
r_pipe_freeze_trace.csr.mstatus_fs_we)) begin //writes happening in
ex that needs to be reported to id
1301             trace_id.m_csr.mstatus_fs_rdata =
r_pipe_freeze_trace.csr.mstatus_fs_n;
1302             ->e_fregs_dirty_2;
1303         end
1304         ->e_mstatus_to_id;
1305     endfunction
1306     //those event are for debug purpose
1307     event e_dev_send_wb_1, e_dev_send_wb_2;
1308     event
1309         e_dev_commit_rf_to_ex_1,
1310         e_dev_commit_rf_to_ex_2,
1311         e_dev_commit_rf_to_ex_3,
1312         e_dev_commit_rf_to_ex_4,
1313         e_dev_commit_rf_to_ex_5;
1314     event e_if_2_id_1, e_if_2_id_2, e_if_2_id_3, e_if_2_id_4;
1315     event e_ex_to_wb_1, e_ex_to_wb_2;
1316     event e_id_to_ex_1, e_id_to_ex_2;
1317     event e_commit_dpc;
1318     event e_csr_in_ex, e_csr_irq;
1319
1320     event e_send_rvfi_trace_apu_resp;
1321     event
1322         e_send_rvfi_trace_ex_1,
1323         e_send_rvfi_trace_ex_2,
1324         e_send_rvfi_trace_ex_3,
1325         e_send_rvfi_trace_ex_4,
1326         e_send_rvfi_trace_ex_5,
1327         e_send_rvfi_trace_ex_6;
1328     event e_send_rvfi_trace_wb_1, e_send_rvfi_trace_wb_2,
e_send_rvfi_trace_wb_3;
1329     event e_send_rvfi_trace_id_1;
1330
1331     //used to match memory response to memory request and cor-
responding instruction
1332     integer cnt_data_req, cnt_data_resp;
1333     integer cnt_apu_req, cnt_apu_resp;
1334
1335     insn_trace_t apu_trace_q[$];
1336     insn_trace_t trace_apu_req, trace_apu_resp;
1337
1338     function void csr_to_apu_resp();
1339         `CSR_FROM_PIPE(apu_resp, fcsr)
1340         `CSR_FROM_PIPE(apu_resp, fflags)
1341
1342         `CSR_FROM_PIPE(apu_resp, mstatus_fs)
1343
1344         if (r_pipe_freeze_trace.csr.mstatus_fs_we &&
(trace_id.m_order > trace_apu_resp.m_order)) begin
1345             trace_id.m_csr.mstatus_fs_rdata =
r_pipe_freeze_trace.csr.mstatus_fs_n;
1346         end
1347         if (r_pipe_freeze_trace.csr.mstatus_fs_we &&
(trace_ex.m_order > trace_apu_resp.m_order)) begin
1348             trace_ex.m_csr.mstatus_fs_rdata =
r_pipe_freeze_trace.csr.mstatus_fs_n;
1349         end
1350         if (r_pipe_freeze_trace.csr.mstatus_fs_we &&
(trace_wb.m_order > trace_apu_resp.m_order)) begin
1351             trace_wb.m_csr.mstatus_fs_rdata =
r_pipe_freeze_trace.csr.mstatus_fs_n;
1352         end
1353     endfunction
1354
1355     function void csr_to_apu_req();
1356         `CSR_FROM_PIPE(apu_req, misa)
1357         `CSR_FROM_PIPE(apu_req, tdata1)
1358         `CSR_FROM_PIPE(apu_req, tdata2)
1359         trace_apu_req.m_csr.tinfo_we = '0'; // READ ONLY csr-t-
info_we_i;
1360         trace_apu_req.m_csr.tinfo_rdata =
r_pipe_freeze_trace.csr.tinfo_q;
1361         trace_apu_req.m_csr.tinfo_rmask = '1';
1362         trace_apu_req.m_csr.tinfo_wdata =
r_pipe_freeze_trace.csr.tinfo_n;
1363         trace_apu_req.m_csr.tinfo_wmask = '0';
1364
1365         trace_apu_req.m_csr.minstret_we =

```

```

r_pipe_freeze_trace.csr.mhpmcounter_write_lower[2];
1366     trace_apu_req.m_csr.minstret_rdata =
r_pipe_freeze_trace.csr.mhpmcounter_q[2];
1367     trace_apu_req.m_csr.minstret_rmask = '1';
1368     trace_apu_req.m_csr.minstret_wdata =
r_pipe_freeze_trace.csr.mhpmcounter_q;
1369     trace_apu_req.m_csr.minstret_wmask =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[2] ? '1' : '0';
1370
1371     trace_apu_req.m_csr.lpcount0_we = '0';
1372     trace_apu_req.m_csr.lpcount0_rdata =
r_pipe_freeze_trace.hwloop.counter_q[0];
1373     trace_apu_req.m_csr.lpcount0_rmask = '1';
1374     trace_apu_req.m_csr.lpcount0_wdata = '0';
1375     trace_apu_req.m_csr.lpcount0_wmask = '0';
1376
1377     trace_apu_req.m_csr.lpcount1_we = '0';
1378     trace_apu_req.m_csr.lpcount1_rdata =
r_pipe_freeze_trace.hwloop.counter_q[1];
1379     trace_apu_req.m_csr.lpcount1_rmask = '1';
1380     trace_apu_req.m_csr.lpcount1_wdata = '0';
1381     trace_apu_req.m_csr.lpcount1_wmask = '0';
1382
1383     `CSR_FROM_PIPE(apu_req, frm)
1384
1385     endfunction
1386
1387     function void fcsr_to_wb();
1388         `CSR_FROM_PIPE(wb, fflags)
1389         `CSR_FROM_PIPE(wb, frm)
1390         `CSR_FROM_PIPE(wb, fcsr)
1391         trace_wb.m_csr.fflags_wmask = '0';
1392         trace_wb.m_csr.frm_wmask = '0';
1393         trace_wb.m_csr.fcsr_wmask = '0';
1394     endfunction
1395
1396     bit s_apu_to_alu_port;
1397     bit s_apu_to_lsu_port;
1398
1399     function void apu_resp();
1400         if (!r_pipe_freeze_trace.wb.contention_lsu) begin
1401             if (apu_trace_q.size() > 0) begin
1402                 trace_apu_resp = apu_trace_q.pop_front();
1403                 if (s_apu_to_alu_port) begin
1404                     if (r_pipe_freeze_trace.ex_reg_we) begin
1405                         trace_apu_resp.m_rd_addr[0] =
r_pipe_freeze_trace.ex_reg_addr;
1406                         trace_apu_resp.m_rd_wdata[0] =
r_pipe_freeze_trace.ex_reg_wdata;
1407                     end
1408                     end else if (s_apu_to_lsu_port) begin
1409                         if (r_pipe_freeze_trace.rf_we_wb) begin
1410                             trace_apu_resp.m_rd_addr[0] =
r_pipe_freeze_trace.rf_addr_wb;
1411                             trace_apu_resp.m_rd_wdata[0] =
r_pipe_freeze_trace.rf_wdata_wb;
1412                         end
1413                     end
1414                 end
1415                 csr_to_apu_resp();
1416
1417                 trace_apu_resp.m_stop_cycle = cycles;
1418                 trace_apu_resp.m_stop_time = $time;
1419                 send_rvfi(trace_apu_resp);
1420                 ->e_send_rvfi_trace_apu_resp;
1421             end
1422         end
1423     endfunction
1424
1425     /*
1426     * Decoding is complete and instruction enters execute stage
1427     * If at that time, minstret is not asserted it means we
have a trap
1428     */
1429     bit s_is_pc_set; //If pc_set, wait until next trace_id to
commit csr changes
1430     bit s_is_irq_start;
1431     bit s_id_done;
1432     function void if_to_id();
1433         if (trace_id.m_valid) begin
1434             `CSR_FROM_PIPE(id, misa)
1435             `CSR_FROM_PIPE(id, tdata1)
1436             `CSR_FROM_PIPE(id, tdata2)
1437             tinfo_to_trace(trace_id);
1438             `CSR_FROM_PIPE(id, mip)
1439             send_rvfi(trace_id);
1440         end
1441         trace_id.init(trace_if);
1442         trace_id.m_trap = ~r_pipe_freeze_trace.minstret;
1443         trace_id.m_is_illegal = trace_id.m_is_illegal |
r_pipe_freeze_trace.is_illegal;
1444         `CSR_FROM_PIPE(id, dpc)
1445         s_is_pc_set = '1'b0;
1446         s_is_irq_start = '1'b0;
1447         trace_if.m_valid = '1'b0;
1448         s_id_done = '1'b0;
1449     endfunction
1450
1451     function logic [31:0] be_to_mask(logic [3:0] be);
1452         logic [31:0] mask;
1453         mask[7:0] = (be[0] == '1'b1) ? 8'hff : 8'h00;

```

```

1454 mask[15:8] = (be[1] == 1'b1) ? 8'hFF : 8'h00;
1455 mask[23:16] = (be[2] == 1'b1) ? 8'hFF : 8'h00;
1456 mask[31:24] = (be[3] == 1'b1) ? 8'hFF : 8'h00;
1457
1458 be_to_mask = mask;
1459 return mask;
1460 endfunction
1461
1462 function void commit_rf_to_trace(insn_trace_t m_trace);
1463 if (m_trace.m_got_ex_reg) begin
1464   m_trace.m_rd_addr[1] = r_pipe_freeze_trace.rf_addr_wb;
1465   m_trace.m_rd_wdata[1] =
r_pipe_freeze_trace.rf_wdata_wb;
1466   m_trace.m_2_rd_insn = 1'b1;
1467   m_trace.m_got_first_data = 1'b1;
1468 end else begin
1469   m_trace.m_rd_addr[0] = r_pipe_freeze_trace.rf_addr_wb;
1470   m_trace.m_rd_wdata[0] =
r_pipe_freeze_trace.rf_wdata_wb;
1471   m_trace.m_got_first_data = 1'b1;
1472 end
1473 endfunction
1474
1475 task compute_pipeline();
1476 bit s_new_valid_insn;
1477 bit s_ex_valid_adjusted;
1478 bit s_wb_valid_adjusted;
1479
1480 bit s_apu_wb_ok;
1481 bit s_apu_0_cycle_reps;
1482
1483 bit s_fflags_we_non_apu;
1484 bit s_frm_we_non_apu;
1485 bit s_fcsr_we_non_apu;
1486
1487 bit s_skip_wb; // used to skip wb monitoring when apu
resp and not lsu
1488
1489 bit s_core_is_decoding; // For readability, ctrl_fsm is
DECODE or DECODE_HWLOOP
1490
1491 bit s_ex_reg_we_adjusted; //ex_reg_we
1492 bit s_rf_we_wb_adjusted; //
1493
1494 bit s_dont_override_mstatus_fs_id;
1495
1496 trace_if = new();
1497 trace_id = new();
1498 trace_ex = new();
1499 trace_wb = new();
1500 s_new_valid_insn = 1'b0;
1501 s_ex_valid_adjusted = 1'b0;
1502
1503 s_id_done = 1'b0;
1504 s_apu_wb_ok = 1'b0;
1505 s_apu_0_cycle_reps = 1'b0;
1506
1507 next_send = 1;
1508 cnt_data_req = 0;
1509 cnt_data_resp = 0;
1510 cnt_apu_req = 0;
1511 cnt_apu_resp = 0;
1512 csr_is_irq = 0;
1513 is_dbg_taken = 0;
1514 s_was_flush = 1'b0;
1515
1516 s_is_pc_set = 1'b0;
1517 s_is_irq_start = 1'b0;
1518
1519 s_is_pc_set = 1'b0;
1520 s_is_irq_start = 1'b0;
1521 s_skip_wb = 1'b0;
1522
1523 s_core_is_decoding = 1'b0;
1524
1525 s_ex_reg_we_adjusted = 1'b0;
1526 s_rf_we_wb_adjusted = 1'b0;
1527
1528 s_dont_override_mstatus_fs_id = 1'b0;
1529
1530 forever begin
1531   wait(e_pipe_monitor_ok.triggered); // event triggered
1532   #1;
1533
1534   s_core_is_decoding = (r_pipe_freeze_trace.ctrl_fsm_cs ==
DECODE) || (r_pipe_freeze_trace.ctrl_fsm_cs == DECODE_HWLOOP);
1535   check_trap();
1536
1537   pc_mux_interrupt = 1'b0;
1538   if (r_pipe_freeze_trace.pc_mux == 4'b0100) begin
1539     if (r_pipe_freeze_trace.exc_pc_mux == 3'b001) begin
1540       pc_mux_interrupt = 1'b1;
1541       s_is_irq_start = 1'b1;
1542     end
1543   end
1544
1545   if (r_pipe_freeze_trace.ctrl_fsm_cs == DBG_TAKEN_ID &&
r_pipe_freeze_trace.ebrk_insn_dec) begin
1546     if (trace_id.m_valid) begin
1547       `CSR_FROM_PIPE(id, misa)
1548       `CSR_FROM_PIPE(id, tdata1)
1549       `CSR_FROM_PIPE(id, tdata2)
1550       tinfo_to_trace(trace_id);
1551       `CSR_FROM_PIPE(id, mip)
1552     end
1553   end
1554
1555   if (r_pipe_freeze_trace.data_rvalid) begin
1556     cnt_data_resp = cnt_data_resp + 1;
1557   end
1558
1559   if (r_pipe_freeze_trace.apu_req &&
r_pipe_freeze_trace.apu_gnt) begin
1560     cnt_apu_req = cnt_apu_req + 1;
1561   end
1562   if (r_pipe_freeze_trace.apu_rvalid) begin
1563     cnt_apu_resp = cnt_apu_resp + 1;
1564     if (r_pipe_freeze_trace.apu_singlecycle |
r_pipe_freeze_trace.apu_multicycle) begin
1565       s_apu_to_alu_port = 1'b1;
1566       s_apu_to_lsu_port = 1'b0;
1567     end else begin
1568       s_apu_to_lsu_port = 1'b1;
1569       s_apu_to_alu_port = 1'b0;
1570     end
1571   end else begin
1572     s_apu_to_lsu_port = 1'b0;
1573     s_apu_to_alu_port = 1'b0;
1574   end
1575
1576   if(r_pipe_freeze_trace.apu_req &&
r_pipe_freeze_trace.apu_gnt && r_pipe_freeze_trace.apu_rvalid &&
(cnt_apu_resp == cnt_apu_req)) begin
1577     s_apu_0_cycle_reps = 1'b1;
1578   end else begin
1579     s_apu_0_cycle_reps = 1'b0;
1580   end
1581
1582   if (trace_ex.m_valid & s_wb_valid_adjusted) begin
1583     // Used flopped values in case write happened before
wb_valid
1584     sample_perf_counter_to_trace(trace_ex);
1585     trace_ex.m_csr.got_minstret = 1;
1586   end
1587
1588   if (trace_wb.m_valid & trace_wb.m_is_apu &
(trace_wb.m_apu_req_id == cnt_apu_resp)) begin
1589     s_apu_wb_ok = 1'b1;
1590   end else begin
1591     s_apu_wb_ok = 1'b0;
1592   end
1593
1594   s_new_valid_insn = r_pipe_freeze_trace.id_valid &&
r_pipe_freeze_trace.is_decoding; // && !
r_pipe_freeze_trace.apu_rvalid;
1595
1596   s_wb_valid_adjusted = r_pipe_freeze_trace.wb_valid &&
(s_core_is_decoding || (r_pipe_freeze_trace.ctrl_fsm_cs == FLUSH_EX)
|| (r_pipe_freeze_trace.ctrl_fsm_cs == FLUSH_WB) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == DBG_FLUSH) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == DBG_TAKEN_ID) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == DBG_TAKEN_IF)); // && !
r_pipe_freeze_trace.apu_rvalid;
1597   s_ex_reg_we_adjusted = r_pipe_freeze_trace.ex_reg_we &&
r_pipe_freeze_trace.mult_ready && r_pipe_freeze_trace.alu_ready &&
r_pipe_freeze_trace.lsu_ready_ex && !s_apu_to_alu_port;
1598   s_rf_we_wb_adjusted = r_pipe_freeze_trace.rf_we_wb &&
(~r_pipe_freeze_trace.data_misaligned_ex &&
r_pipe_freeze_trace.wb_ready) && (!s_apu_to_lsu_port ||
r_pipe_freeze_trace.wb_contention_lsu);
1599
1600   s_fflags_we_non_apu = 1'b0;
1601   if (r_pipe_freeze_trace.csr.fflags_we) begin
1602     if (cnt_apu_resp == cnt_apu_req) begin //No ongoing
apu instruction
1603       s_fflags_we_non_apu = 1'b1;
1604     end
1605   end
1606
1607   s_frm_we_non_apu = 1'b0;
1608   if (r_pipe_freeze_trace.csr.frm_we) begin
1609     if (cnt_apu_resp == cnt_apu_req) begin //No ongoing
apu instruction
1610       s_frm_we_non_apu = 1'b1;
1611     end
1612   end
1613
1614   s_fcsr_we_non_apu = 1'b0;
1615   if (r_pipe_freeze_trace.csr.fcsr_we) begin
1616     if (cnt_apu_resp == cnt_apu_req) begin //No ongoing
apu instruction
1617       s_fcsr_we_non_apu = 1'b1;
1618     end
1619   end
1620
1621   //WB_STAGE
1622   s_skip_wb = 1'b0;
1623   if (r_pipe_freeze_trace.apu_rvalid &&
(apu_trace.q.size() > 0)) begin
1624     apu_resp();
1625     if (!r_pipe_freeze_trace.data_rvalid) begin

```

```

1626     s_skip_wb = 1'b1;
1627 end
1628 end
1629
1630 begin if (trace_wb.m_valid && !s_skip_wb && s_rf_we_wb_adjus-
ted) begin
1631     if (trace_wb.m_2_rd_insn) begin
1632         trace_wb.m_rd_addr[1] =
r_pipe_freeze_trace.rf_addr_wb;
1633         trace_wb.m_rd_wdata[1] =
r_pipe_freeze_trace.rf_wdata_wb;
1634     end else if (trace_wb.m_ex_fw) begin
1635         trace_wb.m_rd_addr[1] =
r_pipe_freeze_trace.rf_addr_wb;
1636         trace_wb.m_rd_wdata[1] =
r_pipe_freeze_trace.rf_wdata_wb;
1637         trace_wb.m_2_rd_insn = 1'b1;
1638     end else begin
1639         trace_wb.m_rd_addr[0] =
r_pipe_freeze_trace.rf_addr_wb;
1640         trace_wb.m_rd_wdata[0] =
r_pipe_freeze_trace.rf_wdata_wb;
1641     end
1642
1643     if(r_pipe_freeze_trace.csr.fregs_we &&
r_pipe_freeze_trace.rf_we_wb && r_pipe_freeze_trace.rf_addr_wb[5])
begin //Catching mstatus updates caused by flw
1644         `CSR_FROM_PIPE(wb, mstatus_fs)
1645         trace_wb.m_csr.mstatus_fs_we = 1'b1;
1646         trace_wb.m_csr.mstatus_fs_wmask = '1;
1647         if(r_pipe_freeze_trace.csr.we &&
r_pipe_freeze_trace.csr.mstatus_fs_we) begin //In this specific
case, two writes to mstatus_fs happen at the same time. We need to
recreate the writes caused by fregs_we
1648             trace_wb.m_csr.mstatus_fs_wdata = FS_DIRTY;
1649         end
1650         ->e_fregs_dirty_1;
1651     end
1652
1653     send_rvfi(trace_wb);
1654     ->e_dev_send_wb_1; ->e_send_rvfi_trace_wb_2;
1655     trace_wb.m_valid = 1'b0;
1656 end
1657
1658 if (trace_ex.m_valid) begin
1659     if(trace_ex.m_instret_smample_trigger == 1) begin //
time to sample instret
1660         sample_perf_counter_to_trace(trace_ex);
1661     end
1662     trace_ex.m_instret_smample_trigger =
trace_ex.m_instret_smample_trigger + 1;
1663
1664     `CSR_FROM_PIPE(ex, misa)
1665     `CSR_FROM_PIPE(ex, tdata1)
1666     `CSR_FROM_PIPE(ex, tdata2)
1667     tinfo_to_trace(trace_ex);
1668
1669     if (s_rf_we_wb_adjusted) begin
1670         ->e_dev_commit_rf_to_ex_4;
1671         if (! (trace_ex.m_got_ex_reg) &&
trace_ex.m_mem_req_id_valid[0]) begin
1672             trace_ex.m_rd_addr[0] =
r_pipe_freeze_trace.rf_addr_wb;
1673             trace_ex.m_rd_wdata[0] =
r_pipe_freeze_trace.rf_wdata_wb;
1674             trace_ex.m_got_first_data = 1'b1;
1675             trace_ex.m_mem_req_id_valid[0] = 1'b0;
1676         end else if (trace_ex.m_mem_req_id_valid[1]) begin
1677             trace_ex.m_rd_addr[1] =
r_pipe_freeze_trace.rf_addr_wb;
1678             trace_ex.m_rd_wdata[1] =
r_pipe_freeze_trace.rf_wdata_wb;
1679             trace_ex.m_got_first_data = 1'b1;
1680             trace_ex.m_mem_req_id_valid[1] = 1'b0;
1681         end
1682     end
1683
1684     if (s_wb_valid_adjusted) begin
1685         if (trace_wb.m_valid) begin
1686             send_rvfi(trace_ex);
1687             trace_ex.m_valid = 1'b0;
1688             ->e_send_rvfi_trace_ex_2;
1689         end else begin
1690             if (s_rf_we_wb_adjusted) begin
1691                 ->e_dev_commit_rf_to_ex_1;
1692                 commit_rf_to_trace(trace_ex);
1693             end
1694             if (r_pipe_freeze_trace.csr.fregs_we &&
(r_pipe_freeze_trace.rf_we_wb && r_pipe_freeze_trace.rf_addr_wb[5]))
begin //Catching mstatus_fs updates caused by flw
1695                 `CSR_FROM_PIPE(ex, mstatus_fs)
1696                 trace_ex.m_csr.mstatus_fs_we = 1'b1;
1697                 trace_ex.m_csr.mstatus_fs_wmask = '1;
1698                 if(r_pipe_freeze_trace.csr.we &&
r_pipe_freeze_trace.csr.mstatus_fs_we) begin //In this specific
case, two writes to mstatus_fs happen at the same time. We need to
recreate the writes caused by fregs_we
1701                     trace_ex.m_csr.mstatus_fs_wdata = FS_DIRTY;
1702                     end else begin

```

```

1703         trace_id.m_csr.mstatus_fs_rdata =
trace_ex.m_csr.mstatus_fs_wdata;
1704         s_dont_override_mstatus_fs_id = 1'b1;
1705     end
1706     ->e_fregs_dirty_3;
1707 end
1708
1709 send_rvfi(trace_ex);
1710 trace_ex.m_valid = 1'b0;
1711
1712 end else begin
1713     if (trace_ex.m_is_load) begin // only move rel-
evant instr in wb stage
1714         ->e_ex_to_wb_1;
1715         trace_wb.move_down_pipe(trace_ex);
1716     end else begin
1717         send_rvfi(trace_ex);
1718         ->e_send_rvfi_trace_ex_6;
1719     end
1720     trace_ex.m_valid = 1'b0;
1721 end
1722 end
1723 end else if (s_rf_we_wb_adjusted && !s_was_flush) be-
gin
1724     ->e_dev_commit_rf_to_ex_2;
1725     commit_rf_to_trace(trace_ex);
1726 end
1727 end
1728
1729 // If mret, we need to keep the instruction in Id during
flush_ex because mstatus update happens at that time
1730 s_ex_valid_adjusted = (r_pipe_freeze_trace.ex_valid &&
r_pipe_freeze_trace.ex_ready) && (s_core_is_decoding ||
(r_pipe_freeze_trace.ctrl_fsm_cs == DBG_TAKEN_IF) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == DBG_TAKEN_ID) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == DBG_FLUSH) ||
((r_pipe_freeze_trace.ctrl_fsm_cs == FLUSH_EX) && !
r_pipe_freeze_trace.mret_insn_dec));
1731 //EX_STAGE
1732
1733 if (trace_id.m_valid) begin
1734     if(trace_id.m_instret_smample_trigger == 1) begin //
time to sample instret
1735         sample_perf_counter_to_trace(trace_id);
1736         for(int idx=3; idx<32; idx++) begin
1737             sample_perf_counter_to_id(idx);
1738             sample_perf_counter_h_to_id(idx);
1739             sample_perf_event_to_trace(idx, trace_id);
1740         end
1741     end
1742     trace_id.m_instret_smample_trigger =
trace_id.m_instret_smample_trigger + 1;
1743
1744     if(trace_id.m_sample_csr_write_in_ex && !csr_is_irq
&& !s_is_irq_start) begin //First cycle after id_ready, csr write is
asserted in this cycle
1745         `CSR_FROM_PIPE(id, mstatus)
1746         if(!s_dont_override_mstatus_fs_id) begin
1747             `CSR_FROM_PIPE(id, mstatus_fs)
1748         end
1749         `CSR_FROM_PIPE(id, mepc)
1750         `CSR_FROM_PIPE(id, mcause)
1751         `CSR_FROM_PIPE(id, dscratch0)
1752         `CSR_FROM_PIPE(id, dscratch1)
1753         if(r_pipe_freeze_trace.csr.we &&
(r_pipe_freeze_trace.csr_addr == CSR_DPC)) begin
1754             `CSR_FROM_PIPE(id, dpc)
1755         end
1756
1757         `CSR_FROM_PIPE(id, mcountinhibit)
1758
1759         perf_counter_to_trace(trace_id);
1760         ->e_csr_in_ex;
1761     end
1762
1763     if(r_pipe_freeze_trace.is_decoding) begin
1764         trace_id.m_sample_csr_write_in_ex = 1'b0;
1765     end
1766     mtvec_to_id();
1767
1768     `CSR_FROM_PIPE(id, mip)
1769     `CSR_FROM_PIPE(id, misa)
1770
1771     `CSR_FROM_PIPE(id, mcountinhibit)
1772     `CSR_FROM_PIPE(id, mscratch)
1773     `CSR_FROM_PIPE(id, mie)
1774
1775     `CSR_FROM_PIPE(id, fflags)
1776     `CSR_FROM_PIPE(id, frm)
1777     `CSR_FROM_PIPE(id, fcsr)
1778
1779     if (r_pipe_freeze_trace.csr.dcsr_we) begin
1780         dcsr_to_id();
1781     end
1782
1783     if (s_fflags_we_non_apu) begin
1784         trace_id.m_fflags_we_non_apu = 1'b1;
1785     end
1786
1787     if (s_frm_we_non_apu) begin

```

```

1788     trace_id.m_frm_we_non_apu = 1'b1;
1789 end
1790
1791 if (s_fcsr_we_non_apu) begin
1792     trace_id.m_fcsr_we_non_apu = 1'b1;
1793 end
1794
1795 trace_ex.m_csr.fflags_wmask = '0;
1796 trace_ex.m_csr.frm_wmask   = '0;
1797 trace_ex.m_csr.fcsr_wmask  = '0;
1798
1799 if (r_pipe_freeze_trace.ctrl_fsm_cs == XRET_JUMP) begin
1800 //xret exit pipeline
1801     tinfo_to_trace(trace_id);
1802     `CSR_FROM_PIPE(id, tdata1);
1803     `CSR_FROM_PIPE(id, tdata2);
1804     send_rvfi(trace_id);
1805     trace_id.m_valid = 1'b0;
1806     s_dont_override_mstatus_fs_id = 1'b0;
1807 end
1808
1809 if (r_pipe_freeze_trace.apu_req &&
1810 r_pipe_freeze_trace.apu_gnt) begin
1811     trace_id.m_is_apu = 1'b1;
1812     trace_id.m_apu_req_id = cnt_apu_req;
1813     trace_apu_req = new();
1814     trace_apu_req.copy_full(trace_id);
1815     csr_to_apu_req();
1816     trace_apu_req.set_to_apu();
1817     apu_trace.q.push_back(trace_apu_req);
1818     trace_id.m_valid = 1'b0;
1819     s_dont_override_mstatus_fs_id = 1'b0;
1820
1821 if (r_pipe_freeze_trace.apu_rvalid && (cnt_apu_req ==
1822 cnt_apu_resp)) begin //APU return in the same cycle
1823     apu_resp();
1824 end
1825 end
1826
1827 if (s_ex_valid_adjusted) begin //A valid instruction
1828 goes from ID to EX
1829     if (trace_ex.m_valid) begin //We need to bypass wb
1830         send_rvfi(trace_ex);
1831         trace_ex.m_valid = 1'b0;
1832         ->e_send_rvfi_trace_ex_3;
1833     end
1834     if (r_pipe_freeze_trace.ex_reg_we && !
1835 s_apu_to_alu_port) begin
1836         trace_id.m_ex_fw = 1'b1;
1837         trace_id.m_rd_addr[0] =
1838 r_pipe_freeze_trace.ex_reg_addr;
1839         trace_id.m_rd_wdata[0] =
1840 r_pipe_freeze_trace.ex_reg_wdata;
1841         trace_id.m_got_ex_reg = 1'b1;
1842     end else if (!trace_ex.m_valid & s_rf_we_wb_adjusted
1843 & !trace_id.m_ex_fw) begin
1844         trace_id.m_rd_addr[0] =
1845 r_pipe_freeze_trace.rf_addr_wb;
1846         trace_id.m_rd_wdata[0] =
1847 r_pipe_freeze_trace.rf_wdata_wb;
1848     end else if (s_rf_we_wb_adjusted) begin
1849         trace_id.m_rd_addr[1] =
1850 r_pipe_freeze_trace.rf_addr_wb;
1851         trace_id.m_rd_wdata[1] =
1852 r_pipe_freeze_trace.rf_wdata_wb;
1853         trace_id.m_2_rd_insn = 1'b1;
1854     end
1855
1856 if (r_pipe_freeze_trace.data_req_ex) begin
1857     cnt_data_req = cnt_data_req + 1;
1858     trace_id.m_is_memory = 1'b1;
1859     trace_id.m_mem_req_id[0] = cnt_data_req;
1860     trace_id.m_mem_req_id_valid[0] = 1'b1;
1861
1862     trace_id.m_mem_addr =
1863 r_pipe_freeze_trace.data_addr_pmp;
1864 if (r_pipe_freeze_trace.data_misaligned) begin
1865     cnt_data_req = cnt_data_req + 1;
1866     trace_id.m_mem_req_id[0] = cnt_data_req;
1867 end
1868
1869 if (!r_pipe_freeze_trace.data_we_ex) begin
1870     trace_id.m_is_load = 1'b1;
1871     trace_id.m_mem.wmask =
1872 be_to_mask(r_pipe_freeze_trace.lsu_data_be); //1;
1873 end else begin
1874     trace_id.m_mem.rmask =
1875 be_to_mask(r_pipe_freeze_trace.lsu_data_be); //1;
1876 end
1877
1878 if (trace_id.m_got_ex_reg) begin // Shift index 0
1879 to 1
1880     trace_id.m_mem_req_id[1] =
1881 trace_id.m_mem_req_id[0];
1882     trace_id.m_mem_req_id[0] = 0;
1883     trace_id.m_mem_req_id_valid[1] = 1'b1;
1884     trace_id.m_mem_req_id_valid[0] = 1'b0;
1885 end

```

```

1869     end
1870     hwlloop_to_id();
1871     trace_ex.move_down_pipe(trace_id); // The instruc-
1872 tion moves forward from ID to EX
1873     trace_id.m_valid = 1'b0;
1874     s_dont_override_mstatus_fs_id = 1'b0;
1875     ->e_id_to_ex_1;
1876 end else if (r_pipe_freeze_trace.ex_reg_we &&
1877 r_pipe_freeze_trace.rf_alu_we_ex) begin
1878     trace_id.m_ex_fw = 1'b1;
1879     trace_id.m_rd_addr[0] =
1880 r_pipe_freeze_trace.ex_reg_addr;
1881     trace_id.m_rd_wdata[0] =
1882 r_pipe_freeze_trace.ex_reg_wdata;
1883     trace_id.m_got_ex_reg = 1'b1;
1884     trace_id.m_got_regs_write = 1'b1;
1885     // mem_req_id[0] already set here indicates req_id
1886 was set before rf write from EX
1887 // Hence adjust the req_id again here for such cases
1888 if (trace_id.m_mem_req_id[0] != 0) begin
1889     trace_id.m_mem_req_id[1] =
1890 trace_id.m_mem_req_id[0];
1891     trace_id.m_mem_req_id[0] = 0;
1892     trace_id.m_mem_req_id_valid[0] = 1'b0;
1893     trace_id.m_mem_req_id_valid[1] = 1'b1;
1894 end
1895 end
1896 end //trace_if.m_valid
1897
1898 //ID_STAGE
1899 if (s_new_valid_insn) begin // There is a new valid in-
1900 struction
1901     if (trace_id.m_valid) begin
1902         if (trace_ex.m_valid) begin
1903             if (trace_wb.m_valid) begin
1904                 send_rvfi(trace_ex);
1905                 ->e_send_rvfi_trace_ex_4;
1906             end else begin
1907                 if (trace_ex.m_is_load) begin // only move rel-
1908 evant instr in wb stage
1909                     ->e_ex_to_wb_2;
1910                     trace_wb.move_down_pipe(trace_ex);
1911                 end else begin
1912                     send_rvfi(trace_ex);
1913                     ->e_send_rvfi_trace_ex_5;
1914                 end
1915             end
1916             trace_ex.m_valid = 1'b0;
1917         end
1918         if (r_pipe_freeze_trace.data_req_ex) begin
1919             cnt_data_req = cnt_data_req + 1;
1920             trace_id.m_is_memory = 1'b1;
1921             trace_id.m_mem_req_id[0] = cnt_data_req;
1922             trace_id.m_mem_req_id_valid[0] = 1'b1;
1923             trace_id.m_mem_addr =
1924 r_pipe_freeze_trace.data_addr_pmp;
1925 if (r_pipe_freeze_trace.data_misaligned) begin
1926             cnt_data_req = cnt_data_req + 1;
1927             trace_id.m_mem_req_id[0] = cnt_data_req;
1928         end
1929         if (!r_pipe_freeze_trace.data_we_ex) begin
1930             trace_id.m_is_load = 1'b1;
1931         end
1932     end
1933
1934 if (trace_id.m_got_ex_reg) begin // Shift index 0
1935 to 1
1936     trace_id.m_mem_req_id[1] =
1937 trace_id.m_mem_req_id[0];
1938     trace_id.m_mem_req_id[0] = 0;
1939     trace_id.m_mem_req_id_valid[0] = 1'b0;
1940     trace_id.m_mem_req_id_valid[1] = 1'b1;
1941 end
1942
1943 end else if (s_rf_we_wb_adjusted && !
1944 r_pipe_freeze_trace.ex_reg_we) begin
1945     trace_id.m_rd_addr[0] =
1946 r_pipe_freeze_trace.rf_addr_wb;
1947     trace_id.m_rd_wdata[0] =
1948 r_pipe_freeze_trace.rf_wdata_wb;
1949 end
1950
1951 hwlloop_to_id();
1952 trace_ex.move_down_pipe(trace_id);
1953 trace_id.m_valid = 1'b0;
1954 s_dont_override_mstatus_fs_id = 1'b0;
1955 ->e_id_to_ex_2;
1956 end
1957
1958 if_to_id();
1959 trace_id.m_is_ebreak = trace_if.m_is_ebreak;
1960 csrId_to_id();
1961 `CSR_FROM_PIPE(id, dscratch0);
1962 `CSR_FROM_PIPE(id, dscratch1);
1963 mstatus_to_id();
1964 ->e_if_2_id_1;
1965 end else begin
1966     if (trace_id.m_valid) begin
1967         `CSR_FROM_PIPE(id, dscratch0);
1968         `CSR_FROM_PIPE(id, dscratch1);
1969     end
1970 end
1971 end

```



```

1955 //IF_STAGE
1956 if(trace_if.m_valid) begin
1957   if(r_pipe_freeze_trace.is_illegal &&
r_pipe_freeze_trace.is_decoding) begin
1958     trace_if.m_is_illegal = 1'b1;
1959   end
1960 end
1961
1962 if (r_pipe_freeze_trace.if_valid &&
r_pipe_freeze_trace.if_ready && r_pipe_freeze_trace.instr_valid_if)
begin
1963   if (trace_if.m_valid) begin
1964     if (r_pipe_freeze_trace.id_valid &&
r_pipe_freeze_trace.id_ready && !trace_id.m_valid &&
r_pipe_freeze_trace.ebrk_insn_dec) begin
1965       if_to_id();
1966       trace_id.m_is_ebreak = '1; //
trace_if.m_is_ebreak;
1967       ->e_if_2_id_2;
1968     end else if (trace_if.m_is_illegal) begin
1969       if_to_id();
1970       ->e_if_2_id_3;
1971     end else if (r_pipe_freeze_trace.ecall_insn_dec) be-
gin
1972       if_to_id();
1973       ->e_if_2_id_4;
1974     end
1975   end
1976 end
1977
1978 trace_if.m_insn = r_pipe_freeze_trace.instr_if; //In-
str comes from if, buffer for one cycle
1979 trace_if.m_pc_rdata = r_pipe_freeze_trace.pc_if;
1980 trace_if.m_dbg_taken = is_dbg_taken;
1981 trace_if.m_dbg_cause = saved_debug_cause;
1982 trace_if.m_is_ebreak = '0;
1983 trace_if.m_trap = 1'b0;
1984
1985 trace_if.m_valid = 1'b1;
1986 end
1987
1988 if (csr_is_irq && !s_is_pc_set) begin
1989   mstatus_to_id();
1990   `CSR_FROM_PIPE(id, mepc)
1991   `CSR_FROM_PIPE(id, mcause)
1992   ->e_csr_irq;
1993 end
1994
1995 if (!s_id_done && r_pipe_freeze_trace.is_decoding) begin
1996   dcsr_to_id();
1997   ->e_commit_dpc;
1998 end
1999
2000 if (r_pipe_freeze_trace.pc_set) begin
2001   s_is_pc_set = 1'b1;
2002 end
2003
2004 csr_is_irq = r_pipe_freeze_trace.csr_cause[5];
2005
2006 is_dbg_taken = ((r_pipe_freeze_trace.ctrl_fsm_cs ==
DBG_TAKEN_ID) | (r_pipe_freeze_trace.ctrl_fsm_cs == DBG_TAKEN_IF)) ?
1'b1 : 1'b0;
2006 saved_debug_cause = r_pipe_freeze_trace.debug_cause;
2007 s_id_done = r_pipe_freeze_trace.id_valid;
2008 if((r_pipe_freeze_trace.ctrl_fsm_cs == FLUSH) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == FLUSH_EX) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == FLUSH_WB)) begin
2009   s_was_flush = 1'b1;
2010 end else begin
2011   s_was_flush = 1'b0;
2012 end
2013 #1;
2014 end
2015 endtask
2016
2017 task update_rvfi();
2018 rvfi_valid = 1'b0;
2019 rvfi_order = '0;
2020 rvfi_insn = '0;
2021 rvfi_pc_rdata = '0;
2022 rvfi_rd_addr[0] = '0;
2023 rvfi_rd_wdata[0] = '0;
2024 rvfi_rd_addr[1] = '0;
2025 rvfi_rd_wdata[1] = '0;
2026 rvfi_rs1_addr = '0;
2027 rvfi_rs2_addr = '0;
2028 rvfi_rs3_addr = '0;
2029 rvfi_rs1_rdata = '0;
2030 rvfi_rs2_rdata = '0;
2031 rvfi_rs3_rdata = '0;
2032 rvfi_mem_addr = '0;
2033 rvfi_mem_rmask = '0;
2034 rvfi_mem_wmask = '0;
2035 rvfi_mem_rdata = '0;
2036 rvfi_mem_wdata = '0;
2037
2038 rvfi_mode = 2'b11; //priv_lvl_i; //TODO: correct
this if needed
2039
2040 $display("*****Starting update rvfi task*****\n");
2041 wait(clk_i_d == 1'b1);

```

```

2042 forever begin
2043   wait(clk_i_d == 1'b1);
2044   if (rvfi_trace.q.size() != 0) begin
2045     set_rvfi();
2046     rvfi_valid = 1'b1;
2047   end else begin
2048     rvfi_valid = 1'b0;
2049   end
2050   wait(clk_i_d == 1'b0);
2051 end
2052 endtask
2053
2054 initial begin
2055   fork
2056     monitor_pipeline();
2057     compute_pipeline();
2058     update_rvfi();
2059   join
2060 end
2061
2062 endmodule // cv32e40p_rvfi

```

4. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi_trace.sv (262 lines)

```

1 // Copyright 2024 Dolphin Design
2 // SPDX-License-Identifier: Apache-2.0 WITH SHL-2.1
3 //
4 // Licensed under the Solderpad Hardware License v 2.1 (the
"License");
5 // you may not use this file except in compliance with the Li-
cense, or,
6 // at your option, the Apache License version 2.0.
7 // You may obtain a copy of the License at
8 //
9 // https://solderpad.org/licenses/SHL-2.1/
10 //
11 // Unless required by applicable law or agreed to in writing,
any work
12 // distributed under the License is distributed on an "AS IS"
BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14 // See the License for the specific language governing permis-
sions and
15 // limitations under the License.
16
17 ///////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////
18 //
19 // Contributors: Halfdan Bechmann, Silicon Labs <half-
dan.bechmann@silabs.com> //
20 // Yoann Pruvost, Dolphin Design
<yoann.pruvost@dolphin.fr> //
21 //
22 // Description: CV32E40P RVFI
tracer //
23 //
24 ///////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////
25
26 module cv32e40p_rvfi_trace
27 import cv32e40p_pkg::*;
28 import cv32e40p_rvfi_pkg::*;
29 #(
30   parameter FPU = 0,
31   parameter ZFINX = 0
32 ) (
33   input logic clk_i,
34   input logic rst_ni,
35
36   input logic [31:0] hart_id_i,
37
38   input logic [31:0] imm_s3_type,
39
40   input logic rvfi_valid,
41   input logic [31:0] rvfi_insn,
42   input integer rvfi_start_cycle,
43   input time rvfi_start_time,
44   input integer rvfi_stop_cycle,
45   input time rvfi_stop_time,
46   input logic [31:0] rvfi_pc_rdata,
47   input rvfi_trap_t rvfi_trap,
48
49   input logic [4:0] rvfi_rd_addr [1:0],
50   input logic [31:0] rvfi_rd_wdata[1:0],
51
52   input logic rvfi_frđ_wvalid[1:0],
53   input logic [4:0] rvfi_frđ_addr [1:0],
54   input logic [31:0] rvfi_frđ_wdata [1:0],
55   input logic rvfi_2_rd,
56
57   input logic [4:0] rvfi_rs1_addr,
58   input logic [4:0] rvfi_rs2_addr,

```

```

59  input logic [ 4:0] rvfi_rs3_addr,
60  input logic [31:0] rvfi_rs1_rdata,
61  input logic [31:0] rvfi_rs2_rdata,
62  input logic [31:0] rvfi_rs3_rdata,
63
64  input logic [ 4:0] rvfi_frs1_addr,
65  input logic [ 4:0] rvfi_frs2_addr,
66  input logic [ 4:0] rvfi_frs3_addr,
67  input logic      rvfi_frs1_rvalid,
68  input logic      rvfi_frs2_rvalid,
69  input logic      rvfi_frs3_rvalid,
70  input logic [31:0] rvfi_frs1_rdata,
71  input logic [31:0] rvfi_frs2_rdata,
72  input logic [31:0] rvfi_frs3_rdata,
73
74  input logic [31:0] rvfi_mem_addr,
75  input logic [31:0] rvfi_mem_rmask,
76  input logic [31:0] rvfi_mem_wmask,
77  input logic [31:0] rvfi_mem_rdata,
78  input logic [31:0] rvfi_mem_wdata
79 );
80
81 import cv32e40p_tracer_pkg::*;
82
83 logic rst_n;
84 assign rst_n = rst_ni;
85
86 integer f; //file pointer
87 string fn;
88 // integer cycles;
89 string info_tag;
90
91 logic is_compressed;
92 logic [31:0] decomp_insn;
93
94 logic [5:0] rd, rs1, rs2, rs3, rs4;
95 //TODO get from rvfi
96 logic [31:0] rs1_value;
97 logic [31:0] rs2_value;
98 logic [31:0] rs3_value;
99
100 logic [31:0] rs2_value_vec;
101
102 logic [31:0] imm_u_type;
103 logic [31:0] imm_uj_type;
104 logic [31:0] imm_i_type;
105 logic [11:0] imm_iz_type;
106 logic [31:0] imm_z_type;
107 logic [31:0] imm_s_type;
108 logic [31:0] imm_sb_type;
109 logic [31:0] imm_s2_type;
110 logic [31:0] imm_vs_type;
111 logic [31:0] imm_vu_type;
112 logic [31:0] imm_shuffle_type;
113 logic [ 4:0] imm_clip_type;
114
115 always_comb begin
116   if (rvfi_frs1_rvalid) begin
117     rs1 = {1'b1, rvfi_frs1_addr};
118     rs1_value = rvfi_frs1_rdata;
119   end else begin
120     rs1 = {1'b0, rvfi_rs1_addr};
121     rs1_value = rvfi_rs1_rdata;
122   end
123   if (rvfi_frs2_rvalid) begin
124     rs2 = {1'b1, rvfi_frs2_addr};
125     rs2_value = rvfi_frs2_rdata;
126   end else begin
127     rs2 = {1'b0, rvfi_rs2_addr};
128     rs2_value = rvfi_rs2_rdata;
129   end
130
131   if (rvfi_frs3_rvalid) begin
132     rs3 = {1'b1, rvfi_frs3_addr};
133     rs3_value = rvfi_frs3_rdata;
134   end else begin
135     rs3 = {1'b0, rvfi_rs3_addr};
136     rs3_value = rvfi_rs3_rdata;
137   end
138
139   if (rvfi_2_rd) begin
140     if (rvfi_frd_wvalid[1]) begin
141       rd = {1'b1, rvfi_frd_addr[1]};
142     end else begin
143       rd = {1'b0, rvfi_rd_addr[1]};
144     end
145   end else if (rvfi_frd_wvalid[0]) begin
146     rd = {1'b1, rvfi_frd_addr[0]};
147   end else begin
148     rd = {1'b0, rvfi_rd_addr[0]};
149   end
150 end
151
152 assign rs4 = rs3;
153
154 cv32e40p_compressed_decoder #(
155   .FPU(FPU)
156 ) rvfi_trace_decompress_i (
157   .instr_i(rvfi_insn),
158   .instr_o(decomp_insn),
159   .is_compressed_o(is_compressed)
160 );
161
162 assign imm_i_type = {{20{decomp_insn[31]}}, decomp_insn[31:20]};
163 assign imm_iz_type = {20'b0, decomp_insn[31:20]};
164 assign imm_s_type = {{20{decomp_insn[31]}}, decomp_insn[31:25], decomp_insn[11:7]};
165 assign imm_sb_type = {
166   {19{decomp_insn[31]}},
167   decomp_insn[31],
168   decomp_insn[7],
169   decomp_insn[30:25],
170   decomp_insn[11:8],
171   1'b0
172 };
173 assign imm_u_type = {decomp_insn[31:12], 12'b0};
174 assign imm_uj_type = {
175   {12{decomp_insn[31]}}, decomp_insn[19:12], decomp_insn[20], decomp_insn[30:21], 1'b0
176 };
177
178 assign imm_z_type = '0; //{{27'b0, decomp_insn[REG_S1_MSB:REG_S1_LSB]}};
179
180 assign imm_s2_type = {27'b0, decomp_insn[24:20]};
181 assign imm_vs_type = '0;
182 assign imm_vu_type = '0;
183 assign imm_shuffle_type = '0;
184 assign imm_clip_type = '0;
185
186 `include "cv32e40p_instr_trace.svh"
187 instr_trace_t trace_retire;
188
189 function instr_trace_t trace_new_instr();
190   instr_trace_t trace;
191   trace = new();
192   trace.external_time = 1;
193   trace.simtime = rvfi_start_time - 1ns;
194   trace.stoptime = rvfi_stop_time;
195   trace.stopcycles = rvfi_stop_cycle;
196   trace.ctx = (rvfi_trap.trap) ? "(C)" : "";
197
198   trace.init(.cycles(rvfi_start_cycle), .pc(rvfi_pc_rdata), .compressed(is_compressed),
199     .instr(decomp_insn));
200   return trace;
201 endfunction : trace_new_instr
202
203 function void apply_reg_write();
204   foreach (trace_retire.regs_write[i]) begin
205     if (rvfi_frd_wvalid[1] && (trace_retire.regs_write[i].addr == {1'b1, rvfi_frd_addr[1]})) begin
206       trace_retire.regs_write[i].value = rvfi_frd_wdata[1];
207     end else if (trace_retire.regs_write[i].addr == rvfi_rd_addr[1]) begin
208       trace_retire.regs_write[i].value = rvfi_rd_wdata[1];
209     end
210   end
211   foreach (trace_retire.regs_write[i]) begin
212     if (rvfi_frd_wvalid[0] && (trace_retire.regs_write[i].addr == {1'b1, rvfi_frd_addr[0]})) begin
213       trace_retire.regs_write[i].value = rvfi_frd_wdata[0];
214     end else if (trace_retire.regs_write[i].addr == rvfi_rd_addr[0]) begin
215       trace_retire.regs_write[i].value = rvfi_rd_wdata[0];
216     end
217   end
218 endfunction : apply_reg_write
219
220 function void apply_mem_access();
221   mem_acc_t mem_acc;
222   mem_acc.addr = rvfi_mem_addr;
223   if (rvfi_mem_wmask) begin
224     mem_acc.we = 1'b1;
225     mem_acc.wdata = rvfi_mem_wdata;
226     trace_retire.mem_access.push_back(mem_acc);
227   end else if (rvfi_mem_rmask) begin
228     mem_acc.we = 1'b0;
229     mem_acc.wdata = 'x;
230     trace_retire.mem_access.push_back(mem_acc);
231   end
232 endfunction : apply_mem_access
233
234 string      insn_disas;
235 logic [31:0] insn_pc;
236 logic [31:0] insn_val;
237
238 always @(posedge clk_i) begin
239   if (rvfi_valid) begin
240     trace_retire = trace_new_instr();
241     apply_reg_write();
242     apply_mem_access();
243     trace_retire.printInstrTrace();
244     insn_disas = trace_retire.str;
245     insn_pc = trace_retire.pc;
246     insn_val = trace_retire.instr;
247   end
248 end
249
250 initial begin

```



```

251 wait(rst_n == 1'b1);
252 $sformat(fn, "trace_core.log");
253 $sformat(info_tag, "CORE_TRACER %2d", hart_id_i);
254 $display("[%s] Output filename is: %s", info_tag, fn);
255 f = $fopen(fn, "w");
256 $fwrite(f,
257 "          Time          Cycle PC          In-
str Ctx Decoded instruction Register and memory con-
tents                               Stop cycle Stop time\n");
258 end
259
260
261
262 endmodule

```

5. openhwgroup_cv32e40p/bhv/cv32e40p_sim_clock_gate.sv (30 lines)

```

1 // Copyright 2017 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 // !!! cv32e40p_sim_clock_gate file is meant for simulation
  only !!!
12 // !!! It must not be used for ASIC synthe-
  is
13 // !!! It must not be used for FPGA synthe-
  is
14
15 module cv32e40p_clock_gate (
16   input logic clk_i,
17   input logic en_i,
18   input logic scan_cg_en_i,
19   output logic clk_o
20 );
21
22   logic clk_en;
23
24   always_latch begin
25     if (clk_i == 1'b0) clk_en <= en_i | scan_cg_en_i;
26   end
27
28   assign clk_o = clk_i & clk_en;
29
30 endmodule // cv32e40p_clock_gate

```

6. openhwgroup_cv32e40p/bhv/cv32e40p_tb_wrapper.sv (549 lines)

```

1 // Copyright 2024 Dolphin Design
2 // SPDX-License-Identifier: Apache-2.0 WITH SHL-2.1
3 //
4 // Licensed under the Solderpad Hardware License v 2.1 (the
  "License");
5 // you may not use this file except in compliance with the Li-
  cense, or,
6 // at your option, the Apache License version 2.0.
7 // You may obtain a copy of the License at
8 //
9 // https://solderpad.org/licenses/SHL-2.1/
10 //
11 // Unless required by applicable law or agreed to in writing,
  any work
12 // distributed under the License is distributed on an "AS IS"
  BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
  press or implied.
14 // See the License for the specific language governing permis-
  sions and
15 // limitations under the License.
16
17 ///////////////////////////////////////////////////////////////////
18 //
19 // Contributors: Davide Schiavone, OpenHW Group <dav-
  ide@openhwgroup.org>
20 // Yoann Pruvost, Dolphin Design
  <yoann.pruvost@dolphin.fr>
21 //
22 // Description: Test-bench wrapper for cv32e40p_top, tracer

```

```

and and rvfi_tracer //
23 //
24 ///////////////////////////////////////////////////////////////////
25
26 `ifdef CV32E40P_ASSERT_ON
27 `include "cv32e40p_prefetch_controller_sva.sv"
28 `endif
29
30 `ifdef CV32E40P_CORE_LOG
31 `include "cv32e40p_core_log.sv"
32 `endif
33
34 `ifdef CV32E40P_APU_TRACE
35 `include "cv32e40p_apu_tracer.sv"
36 `endif
37
38 `ifdef CV32E40P_TRACE_EXECUTION
39 `include "cv32e40p_tracer.sv"
40 `endif
41
42 `ifdef CV32E40P_RVFI
43 `include "cv32e40p_rvfi.sv"
44 `endif
45
46 `ifdef CV32E40P_RVFI_TRACE_EXECUTION
47 `include "cv32e40p_rvfi_trace.sv"
48 `endif
49
50 module cv32e40p_tb_wrapper
51   import cv32e40p_pkg::*;
52   #(
53     parameter COREV_PULP = 0, // PULP ISA Extension (incl.
  custom CSRs and hardware loop, excl. cv.elw)
54     parameter COREV_CLUSTER = 0, // PULP Cluster interface
  (incl. cv.elw)
55     parameter FPU = 0, // Floating Point Unit (interfaced via
  APU interface)
56     parameter FPU_ADDMUL_LAT = 0, // Floating-Point Addition/
  Multiplication computing lane pipeline registers number
57     parameter FPU_OTHERS_LAT = 0, // Floating-Point COMPari-
  son/CONVersion computing lanes pipeline registers number
58     parameter ZFINX = 0, // Float-in-General Purpose re-
  gisters
59     parameter NUM_MHPMCOUNTERS = 1
60   ) (
61     // Clock and Reset
62     input logic clk_i,
63     input logic rst_ni,
64
65     input logic pulp_clock_en_i, // PULP clock enable (only
  used if COREV_CLUSTER = 1)
66     input logic scan_cg_en_i, // Enable all clock gates for
  testing
67
68     // Core ID, Cluster ID, debug mode halt address and boot
  address are considered more or less static
69     input logic [31:0] boot_addr_i,
70     input logic [31:0] mtvec_addr_i,
71     input logic [31:0] dm_halt_addr_i,
72     input logic [31:0] hart_id_i,
73     input logic [31:0] dm_exception_addr_i,
74
75     // Instruction memory interface
76     output logic instr_req_o,
77     input logic instr_gnt_i,
78     input logic instr_rvalid_i,
79     output logic [31:0] instr_addr_o,
80     input logic [31:0] instr_rdata_i,
81
82     // Data memory interface
83     output logic data_req_o,
84     input logic data_gnt_i,
85     input logic data_rvalid_i,
86     output logic data_we_o,
87     output logic [3:0] data_be_o,
88     output logic [31:0] data_addr_o,
89     output logic [31:0] data_wdata_o,
90     input logic [31:0] data_rdata_i,
91
92     // Interrupt inputs
93     input logic [31:0] irq_i, // CLINT interrupts + CLINT
  extension interrupts
94     output logic irq_ack_o,
95     output logic [4:0] irq_id_o,
96
97     // Debug Interface
98     input logic debug_req_i,
99     output logic debug_havereset_o,
100    output logic debug_running_o,
101    output logic debug_halted_o,
102
103    // CPU Control Signals
104    input logic fetch_enable_i,
105    output logic core_sleep_o
106  );
107
108 `ifdef CV32E40P_ASSERT_ON

```

```

109
110 // RTL Assertions
111 bind cv32e40p_prefetch_controller:
112
cv32e40p_top_i.core_i.if_stage_i.prefetch_buffer_i.prefetch_control
ler_i
113     cv32e40p_prefetch_controller_sva
114     #(
115         .DEPTH                (DEPTH),
116         .COREV_PULP            (COREV_PULP),
117         .PULP_OBI              (PULP_OBI),
118         .FIFO_ADDR_DEPTH      (FIFO_ADDR_DEPTH)
119     ) prefetch_controller_sva (.*) ;
120
121 `endif // CV32E40P_ASSERT_ON
122
123 `ifdef CV32E40P_CORE_LOG
124     cv32e40p_core_log #(
125         .COREV_PULP            (COREV_PULP),
126         .COREV_CLUSTER        (COREV_CLUSTER),
127         .FPU                   (FPU),
128         .ZFINX                 (ZFINX),
129         .NUM_MHPMCOUNTERS      (NUM_MHPMCOUNTERS)
130     ) core_log_i (
131         .clk_i
132     ) (cv32e40p_top_i.core_i.id_stage_i.clk_i),
133     .is_decoding_i
134     (cv32e40p_top_i.core_i.id_stage_i.is_decoding_o),
135     .illeg-
136     al_insn_dec_i(cv32e40p_top_i.core_i.id_stage_i.illegal_insn_dec),
137     .hart_id_i      (cv32e40p_top_i.core_i.hart_id_i),
138     .pc_id_i        (cv32e40p_top_i.core_i.pc_id)
139 );
140 `endif // CV32E40P_CORE_LOG
141
142 `ifdef CV32E40P_APU_TRACE
143     cv32e40p_apu_tracer apu_tracer_i (
144         .clk_i      (cv32e40p_top_i.core_i.rst_ni),
145         .rst_n      (cv32e40p_top_i.core_i.clk_i),
146         .hart_id_i  (cv32e40p_top_i.core_i.hart_id_i),
147         .apu_valid_i
148     ) (cv32e40p_top_i.core_i.ex_stage_i.apu_valid),
149     .apu_waddr_i
150     (cv32e40p_top_i.core_i.ex_stage_i.apu_waddr),
151     .apu_res-
152     ult_i(cv32e40p_top_i.core_i.ex_stage_i.apu_result)
153 );
154 `endif
155
156 `ifdef CV32E40P_TRACE_EXECUTION
157     cv32e40p_tracer #(
158         .FPU (FPU),
159         .ZFINX(ZFINX)
160     ) tracer_i (
161         .clk_i(cv32e40p_top_i.core_i.clk_i), // always-running
162         clock for tracing
163         .rst_n(cv32e40p_top_i.core_i.rst_ni),
164         .hart_id_i(cv32e40p_top_i.core_i.hart_id_i),
165         .pc
166     ) (cv32e40p_top_i.core_i.id_stage_i.pc_id_i),
167     .instr
168     (cv32e40p_top_i.core_i.id_stage_i.instr),
169     .control-
170     ler_state_i(cv32e40p_top_i.core_i.id_stage_i.controller_i.ctrl_fsm-
171     cs),
172     .compressed
173     (cv32e40p_top_i.core_i.id_stage_i.is_compressed_i),
174     .id_valid
175     (cv32e40p_top_i.core_i.id_stage_i.id_valid_o),
176     .is_decoding
177     (cv32e40p_top_i.core_i.id_stage_i.is_decoding_o),
178     .is_illegal
179     (cv32e40p_top_i.core_i.id_stage_i.illegal_insn_dec),
180     .trigger_match
181     (cv32e40p_top_i.core_i.id_stage_i.trigger_match_i),
182     .rs1_value
183     (cv32e40p_top_i.core_i.id_stage_i.operand_a_fw_id),
184     .rs2_value
185     (cv32e40p_top_i.core_i.id_stage_i.operand_b_fw_id),
186     .rs3_value
187     (cv32e40p_top_i.core_i.id_stage_i.alu_operand_c),
188     .rs2_value_vec
189     (cv32e40p_top_i.core_i.id_stage_i.alu_operand_b),
190     .rs1_is_fp(cv32e40p_top_i.core_i.id_stage_i.regfile_fp_a
191 ),
192     .rs2_is_fp(cv32e40p_top_i.core_i.id_stage_i.regfile_fp_b
193 ),
194     .rs3_is_fp(cv32e40p_top_i.core_i.id_stage_i.regfile_fp_c
195 ),
196     .rd_is_fp
197     (cv32e40p_top_i.core_i.id_stage_i.regfile_fp_d),
198     .ex_valid      (cv32e40p_top_i.core_i.ex_valid),
199     .ex_reg_addr
200     (cv32e40p_top_i.core_i.regfile_alu_waddr_fw),
201     .ex_reg_we      (cv32e40p_top_i.core_i.regfile_alu_we_fw),
202     .ex_reg_wdata   (cv32e40p_top_i.core_i.regfile_alu_wdata_fw
203 ),

```

```

204
205     .ex_data_addr   (cv32e40p_top_i.core_i.data_addr_o),
206     .ex_data_req     (cv32e40p_top_i.core_i.data_req_o),
207     .ex_data_gnt     (cv32e40p_top_i.core_i.data_gnt_i),
208     .ex_data_we      (cv32e40p_top_i.core_i.data_we_o),
209     .ex_data_wdata   (cv32e40p_top_i.core_i.data_wdata_o),
210     .data_misaligned(cv32e40p_top_i.core_i.data_misaligned),
211
212     .ebrk_in-
213     sn(cv32e40p_top_i.core_i.id_stage_i.ebrk_insn_dec),
214     .debug_mode(cv32e40p_top_i.core_i.debug_mode),
215     .ebrk_force_debug_mode(cv32e40p_top_i.core_i.id_stage_i.
216     controller_i.ebrk_force_debug_mode),
217
218     .wb_by-
219     pass(cv32e40p_top_i.core_i.ex_stage_i.branch_in_ex_i),
220
221     .wb_valid      (cv32e40p_top_i.core_i.wb_valid),
222     .wb_reg_addr   (cv32e40p_top_i.core_i.regfile_waddr_fw_wb_o),
223     .wb_reg_we      (cv32e40p_top_i.core_i.regfile_we_wb),
224     .wb_reg_wdata   (cv32e40p_top_i.core_i.regfile_wdata),
225
226     .imm_u_type
227     (cv32e40p_top_i.core_i.id_stage_i.imm_u_type),
228     .imm_uj_type
229     (cv32e40p_top_i.core_i.id_stage_i.imm_uj_type),
230     .imm_i_type
231     (cv32e40p_top_i.core_i.id_stage_i.imm_i_type),
232     .imm_iz_type
233     (cv32e40p_top_i.core_i.id_stage_i.imm_iz_type[11:0]),
234     .imm_z_type
235     (cv32e40p_top_i.core_i.id_stage_i.imm_z_type),
236     .imm_s_type
237     (cv32e40p_top_i.core_i.id_stage_i.imm_s_type),
238     .imm_sb_type
239     (cv32e40p_top_i.core_i.id_stage_i.imm_sb_type),
240     .imm_s2_type
241     (cv32e40p_top_i.core_i.id_stage_i.imm_s2_type),
242     .imm_s3_type
243     (cv32e40p_top_i.core_i.id_stage_i.imm_s3_type),
244     .imm_vs_type
245     (cv32e40p_top_i.core_i.id_stage_i.imm_vs_type),
246     .imm_vu_type
247     (cv32e40p_top_i.core_i.id_stage_i.imm_vu_type),
248     .imm_shuffle_type
249     (cv32e40p_top_i.core_i.id_stage_i.imm_shuffle_type),
250     .imm_clip_type
251     (cv32e40p_top_i.core_i.id_stage_i.instr[11:7]),
252     .apu_en_i      (cv32e40p_top_i.apu_req),
253     .apu_single-
254     cycle_i(cv32e40p_top_i.core_i.ex_stage_i.apu_singlecycle),
255     .apu_multicycle_i
256     (cv32e40p_top_i.core_i.ex_stage_i.apu_multicycle),
257     .apu_rvalid_i
258     (cv32e40p_top_i.core_i.ex_stage_i.apu_valid)
259 );
260 `endif
261
262 `ifdef CV32E40P_RVFI
263     logic [1:0][31:0] hwlpl_start_q;
264     logic [1:0][31:0] hwlpl_end_q;
265     logic [1:0][31:0] hwlpl_counter_q;
266     logic [1:0][31:0] hwlpl_counter_n;
267     generate
268     if (COREV_PULP) begin
269         assign hwlpl_start_q =
270         cv32e40p_top_i.core_i.id_stage_i.gen_hwloop_regs.hwloop_regs_i.hwlpl-
271         start_q ;
272         assign hwlpl_end_q =
273         cv32e40p_top_i.core_i.id_stage_i.gen_hwloop_regs.hwloop_regs_i.hwlpl-
274         end_q;
275         assign hwlpl_counter_q =
276         cv32e40p_top_i.core_i.id_stage_i.gen_hwloop_regs.hwloop_regs_i.hwlpl-
277         counter_q;
278         assign hwlpl_counter_n =
279         cv32e40p_top_i.core_i.id_stage_i.gen_hwloop_regs.hwloop_regs_i.hwlpl-
280         counter_n;
281     end else begin
282         assign hwlpl_start_q = '0;
283         assign hwlpl_end_q = '0;
284         assign hwlpl_counter_q = '0;
285         assign hwlpl_counter_n = '0;
286     end
287 endgenerate
288
289 cv32e40p_rvfi #(
290     .FPU(FPU),
291     .ZFINX(ZFINX),
292     .NUM_MHPMCOUNTERS(NUM_MHPMCOUNTERS)
293 ) rvfi_i (
294     .clk_i (cv32e40p_top_i.core_i.clk_i),
295     .rst_ni(cv32e40p_top_i.core_i.rst_ni),
296
297     .is_decoding_i
298     (cv32e40p_top_i.core_i.id_stage_i.is_decoding_o),
299     .is_illegal_i
300     (cv32e40p_top_i.core_i.id_stage_i.illegal_insn_dec),
301     .trigger_match_i
302     (cv32e40p_top_i.core_i.id_stage_i.trigger_match_i),
303     .data_mis-

```

```

aligned_i(cv32e40p_top_i.core_i.data_misaligned),
252 .lsu_data_we_ex_i (cv32e40p_top_i.core_i.data_we_ex),
253 .debug_mode_i (cv32e40p_top_i.core_i.debug_mode),
254 .debug_cause_i (cv32e40p_top_i.core_i.debug_cause),
255 /// Instr IF probes ///
256 .instr_req_i (cv32e40p_top_i.core_i.instr_req_o),
257 .instr_grant_i (cv32e40p_top_i.core_i.instr_gnt_i),
258 .instr_rvalid_i
(cv32e40p_top_i.core_i.instr_rvalid_i),
259 .prefetch_req_i (cv32e40p_top_i.core_i.instr_req_int),
260 .pc_set_i (cv32e40p_top_i.core_i.pc_set),
261
262 .instr_valid_id_i
(cv32e40p_top_i.core_i.instr_valid_id),
263 .instr_rdata_id_i
(cv32e40p_top_i.core_i.instr_rdata_id),
264 .is_fetch_failed_id_i(cv32e40p_top_i.core_i.is_fetch_fa
iled_id),
265 .instr_req_int_i
(cv32e40p_top_i.core_i.instr_req_int),
266 .clear_instr_valid_i
(cv32e40p_top_i.core_i.clear_instr_valid),
267 /// IF probes ///
268 .instr_valid_if_i
(cv32e40p_top_i.core_i.if_stage_i.instr_valid),
269 .if_valid_i
(cv32e40p_top_i.core_i.if_stage_i.if_valid),
270 .if_ready_i
(cv32e40p_top_i.core_i.if_stage_i.if_ready),
271 .instr_if_i
(cv32e40p_top_i.core_i.if_stage_i.instr_aligned),
272 .pc_if_i (cv32e40p_top_i.core_i.pc_if),
273 /// ID probes ///
274 .pc_id_i
(cv32e40p_top_i.core_i.id_stage_i.pc_id_i),
275 .id_valid_i
(cv32e40p_top_i.core_i.id_stage_i.id_valid_o),
276 .id_ready_i
(cv32e40p_top_i.core_i.id_stage_i.id_ready_o),
277
278 .rs1_addr_id_i
(cv32e40p_top_i.core_i.id_stage_i.regfile_addr_ra_id),
279 .rs2_addr_id_i
(cv32e40p_top_i.core_i.id_stage_i.regfile_addr_rb_id),
280 .rs3_addr_id_i
(cv32e40p_top_i.core_i.id_stage_i.regfile_addr_rc_id),
281 .operand_a_fw_id_i
(cv32e40p_top_i.core_i.id_stage_i.operand_a_fw_id),
282 .operand_b_fw_id_i
(cv32e40p_top_i.core_i.id_stage_i.operand_b_fw_id),
283 .operand_c_fw_id_i
(cv32e40p_top_i.core_i.id_stage_i.operand_c_fw_id),
284 // instr
(cv32e40p_top_i.core_i.id_stage_i.instr ),
285 .is_com-
pressed_id_i(cv32e40p_top_i.core_i.id_stage_i.is_compressed_i),
286 .ebrk_insn_dec_i
(cv32e40p_top_i.core_i.id_stage_i.ebrk_insn_dec),
287 .ecall_insn_dec_i
(cv32e40p_top_i.core_i.id_stage_i.ecall_insn_dec),
288 .mret_insn_dec_i
(cv32e40p_top_i.core_i.id_stage_i.mret_insn_dec),
289 .mret_dec_i
(cv32e40p_top_i.core_i.id_stage_i.mret_dec),
290
291 .csr_cause_i (cv32e40p_top_i.core_i.csr_cause),
292 .debug_csr_save_i(cv32e40p_top_i.core_i.debug_csr_save),
293
294 // HWLOOP regs
295 .hwlp_start_q_i (hwlp_start_q),
296 .hwlp_end_q_i (hwlp_end_q),
297 .hwlp_counter_q_i(hwlp_counter_q),
298 .hwlp_counter_n_i(hwlp_counter_n),
299
300 .minstret_i
(cv32e40p_top_i.core_i.id_stage_i.minstret),
301 /// EX probes ///
302 .ex_valid_i (cv32e40p_top_i.core_i.ex_valid),
303 .ex_ready_i (cv32e40p_top_i.core_i.ex_ready),
304 .ex_reg_addr_i
(cv32e40p_top_i.core_i.regfile_alu_waddr_fw),
305 .ex_reg_we_i
(cv32e40p_top_i.core_i.regfile_alu_we_fw),
306 .ex_reg_wdata_i
(cv32e40p_top_i.core_i.regfile_alu_wdata_fw),
307 .apu_en_ex_i (cv32e40p_top_i.core_i.apu_en_ex),
308 .apu_singlecycle_i
(cv32e40p_top_i.core_i.ex_stage_i.apu_singlecycle),
309 .apu_multicycle_i
(cv32e40p_top_i.core_i.ex_stage_i.apu_multicycle),
310 .wb_conten-
tion_lsu_i(cv32e40p_top_i.core_i.ex_stage_i.wb_contention_lsu),
311 .wb_contention_i
(cv32e40p_top_i.core_i.ex_stage_i.wb_contention),
312 .regfile_we_lsu_i
(cv32e40p_top_i.core_i.ex_stage_i.regfile_we_lsu),
313 // .rf_we_alu_i
(cv32e40p_top_i.core_i.id_stage_i.regfile_alu_we_fw_i),
314 // .rf_addr_alu_i
(cv32e40p_top_i.core_i.id_stage_i.regfile_alu_waddr_fw_i),
315 // .rf_wdata_alu_i

```

```

(cv32e40p_top_i.core_i.id_stage_i.regfile_alu_wdata_fw_i),
316
317 .mult_ready_i
(cv32e40p_top_i.core_i.ex_stage_i.mult_ready),
318 .alu_ready_i
(cv32e40p_top_i.core_i.ex_stage_i.alu_ready),
319 /// WB probes ///
320 .wb_valid_i (cv32e40p_top_i.core_i.wb_valid),
321 .wb_ready_i
(cv32e40p_top_i.core_i.lsu_ready_wb),
322 /// LSU probes ///
323 .data_we_ex_i (cv32e40p_top_i.core_i.data_we_ex),
324 .data_atop_ex_i
(cv32e40p_top_i.core_i.data_atop_ex),
325 .data_type_ex_i
(cv32e40p_top_i.core_i.data_type_ex),
326 .alu_operand_c_ex_i
(cv32e40p_top_i.core_i.alu_operand_c_ex),
327 .data_reg_off-
set_ex_i(cv32e40p_top_i.core_i.data_reg_offset_ex),
328 .data_load_event_ex_i(cv32e40p_top_i.core_i.data_load_e
vent_ex),
329 .data_sign_ext_ex_i
(cv32e40p_top_i.core_i.data_sign_ext_ex),
330 .lsu_rdata_i (cv32e40p_top_i.core_i.lsu_rdata),
331 .data_req_ex_i
(cv32e40p_top_i.core_i.data_req_ex),
332 .alu_operand_a_ex_i
(cv32e40p_top_i.core_i.alu_operand_a_ex),
333 .alu_operand_b_ex_i
(cv32e40p_top_i.core_i.alu_operand_b_ex),
334 .useincr_addr_ex_i
(cv32e40p_top_i.core_i.useincr_addr_ex),
335 .data_mis-
aligned_ex_i(cv32e40p_top_i.core_i.data_misaligned_ex),
336 .p_elw_start_i
(cv32e40p_top_i.core_i.p_elw_start),
337 .p_elw_finish_i
(cv32e40p_top_i.core_i.p_elw_finish),
338 .lsu_ready_ex_i
(cv32e40p_top_i.core_i.lsu_ready_ex),
339 .lsu_ready_wb_i
(cv32e40p_top_i.core_i.lsu_ready_wb),
340
341 .lsu_data_be_i(cv32e40p_top_i.core_i.load_store_unit_i.
data_be),
342
343 .data_req_pmp_i(cv32e40p_top_i.core_i.data_req_pmp),
344 .data_gnt_pmp_i(cv32e40p_top_i.core_i.data_gnt_pmp),
345 .data_rvalid_i(cv32e40p_top_i.core_i.data_rvalid_i),
346 .data_err_pmp_i(cv32e40p_top_i.core_i.data_err_pmp),
347 .data_addr_pmp_i(cv32e40p_top_i.core_i.data_addr_pmp),
348 .data_we_i(cv32e40p_top_i.core_i.data_we_o),
349 .data_atop_i(cv32e40p_top_i.core_i.data_atop_o),
350 .data_be_i(cv32e40p_top_i.core_i.data_be_o),
351 .data_wdata_i(cv32e40p_top_i.core_i.data_wdata_o),
352 .data_rdata_i(cv32e40p_top_i.core_i.data_rdata_i),
353 // Register writes
354 .rf_we_wb_i(cv32e40p_top_i.core_i.id_stage_i.regfile_we
wb_i),
355 .rf_ad-
dr_wb_i(cv32e40p_top_i.core_i.id_stage_i.regfile_waddr_wb_i),
356 .rf_wdata_wb_i(cv32e40p_top_i.core_i.id_stage_i.regfile_
wdata_wb_i),
357 .reg-
file_alu_we_ex_i(cv32e40p_top_i.core_i.id_stage_i.regfile_alu_we_ex
o),
358
359 // APU
360 .apu_req_i (cv32e40p_top_i.core_i.apu_req_o),
361 .apu_gnt_i (cv32e40p_top_i.core_i.apu_gnt_i),
362 .apu_rval-
id_i(cv32e40p_top_i.core_i.ex_stage_i.apu_valid),
363
364 // Controller FSM probes
365 .ctrl_fsm_cs_i(cv32e40p_top_i.core_i.id_stage_i.control
ler_i.ctrl_fsm_cs),
366 .pc_mux_i
(cv32e40p_top_i.core_i.id_stage_i.controller_i.pc_mux_o),
367 .exc_pc_mux_i
(cv32e40p_top_i.core_i.id_stage_i.controller_i.exc_pc_mux_o),
368
369 //CSR
370 .csr_addr_i
(cv32e40p_top_i.core_i.cs_registers_i.csr_addr_i),
371 .csr_we_i
(cv32e40p_top_i.core_i.cs_registers_i.csr_we_int),
372 .csr_wdata_int_i(cv32e40p_top_i.core_i.cs_registers_i.c
sr_wdata_int),
373
374 .csr_fregs_we_i(cv32e40p_top_i.core_i.cs_registers_i.fr
egs_we_i),
375
376 .csr_mstatus_n_i
(cv32e40p_top_i.core_i.cs_registers_i.mstatus_n),
377 .csr_mstatus_q_i
(cv32e40p_top_i.core_i.cs_registers_i.mstatus_q),
378 .csr_m-
status_fs_n_i(cv32e40p_top_i.core_i.cs_registers_i.mstatus_fs_n),
379 .csr_m-
status_fs_q_i(cv32e40p_top_i.core_i.cs_registers_i.mstatus_fs_q),

```

```

380
381     .csr_misa_n_i(cv32e40p_top_i.core_i.cs_registers_i.MISA_
VALUE), // WARL
382     .csr_misa_q_i(cv32e40p_top_i.core_i.cs_registers_i.MISA_
VALUE),
383
384     .csr_tdata1_n_i
(cv32e40p_top_i.core_i.cs_registers_i.tmatch_control_rdata),//
csr_wdata_int
385     .csr_tdata1_q_i
(cv32e40p_top_i.core_i.cs_registers_i.tmatch_control_rdata),//
gen_trigger_regs.tmatch_control_exec_q
386     .csr_t-
data1_we_i(cv32e40p_top_i.core_i.cs_registers_i.gen_trigger_regs.tm
atch_control_we),
387
388     .csr_tdata2_n_i
(cv32e40p_top_i.core_i.cs_registers_i.tmatch_value_rdata),//
csr_wdata_int
389     .csr_tdata2_q_i
(cv32e40p_top_i.core_i.cs_registers_i.tmatch_value_rdata),//
gen_trigger_regs.tmatch_control_exec_q
390     .csr_t-
data2_we_i(cv32e40p_top_i.core_i.cs_registers_i.gen_trigger_regs.tm
atch_value_we),
391
392     .csr_tinfo_n_i({16'h0,
cv32e40p_top_i.core_i.cs_registers_i.tinfo_types}),
393     .csr_tinfo_q_i({16'h0,
cv32e40p_top_i.core_i.cs_registers_i.tinfo_types}),
394
395     .csr_mie_n_i
(cv32e40p_top_i.core_i.cs_registers_i.mie_n),
396     .csr_mie_q_i
(cv32e40p_top_i.core_i.cs_registers_i.mie_q),
397     .csr_mie_we_i
(cv32e40p_top_i.core_i.cs_registers_i.csr_mie_we),
398     .csr_mtvec_n_i
(cv32e40p_top_i.core_i.cs_registers_i.mtvec_n),
399     .csr_mtvec_q_i
(cv32e40p_top_i.core_i.cs_registers_i.mtvec_q),
400     .csr_mtvec_mode_n_i(cv32e40p_top_i.core_i.cs_registers_i
.mtvec_mode_n),
401     .csr_mtvec_mode_q_i(cv32e40p_top_i.core_i.cs_registers_i
.mtvec_mode_q),
402
403     .csr_mcountinhibit_q_i
(cv32e40p_top_i.core_i.cs_registers_i.mcountinhibit_q),
404     .csr_mcountinhibit_n_i
(cv32e40p_top_i.core_i.cs_registers_i.mcountinhibit_n),
405     .csr_mcountinhib-
it_we_i(cv32e40p_top_i.core_i.cs_registers_i.mcountinhibit_we),
406
407     .csr_mhp-
mevent_n_i(cv32e40p_top_i.core_i.cs_registers_i.mhpmevent_n),
408     .csr_mhp-
mevent_q_i(cv32e40p_top_i.core_i.cs_registers_i.mhpmevent_q),
409     .csr_mhp-
mevent_we_i(cv32e40p_top_i.core_i.cs_registers_i.mhpmevent_we),
410     .csr_m-
scratch_q_i(cv32e40p_top_i.core_i.cs_registers_i.mscratch_q),
411     .csr_m-
scratch_n_i(cv32e40p_top_i.core_i.cs_registers_i.mscratch_n),
412     .csr_mepc_q_i(cv32e40p_top_i.core_i.cs_registers_i.mepc_
q),
413     .csr_mepc_n_i(cv32e40p_top_i.core_i.cs_registers_i.mepc_
n),
414     .csr_m-
cause_q_i(cv32e40p_top_i.core_i.cs_registers_i.mcause_q),
415     .csr_m-
cause_n_i(cv32e40p_top_i.core_i.cs_registers_i.mcause_n),
416     .csr_mip_n_i(cv32e40p_top_i.core_i.cs_registers_i.mip),
417     .csr_mip_q_i(cv32e40p_top_i.core_i.cs_registers_i.mip),
418     .csr_mip_we_i('0), //
(cv32e40p_top_i.core_i.cs_registers_i.mip)
419
420
421     .csr_dc-
sr_q_i(cv32e40p_top_i.core_i.cs_registers_i.dcsr_q),
422     .csr_dc-
sr_n_i(cv32e40p_top_i.core_i.cs_registers_i.dcsr_n),
423
424     .csr_dpc_n_i(cv32e40p_top_i.core_i.cs_registers_i.depc_n
),
425     .csr_dpc_q_i(cv32e40p_top_i.core_i.cs_registers_i.depc_q
),
426     .csr_dpc_we_i('0), //
cv32e40p_top_i.core_i.cs_registers_i.),
427     .csr_d-
scratch0_n_i(cv32e40p_top_i.core_i.cs_registers_i.dscratch0_n),
428     .csr_d-
scratch0_q_i(cv32e40p_top_i.core_i.cs_registers_i.dscratch0_q),
429     .csr_dscratch0_we_i('0), //
cv32e40p_top_i.core_i.cs_registers_i.),
430
431     .csr_d-
scratch1_n_i(cv32e40p_top_i.core_i.cs_registers_i.dscratch1_n),
432     .csr_d-
scratch1_q_i(cv32e40p_top_i.core_i.cs_registers_i.dscratch1_q),
433     .csr_dscratch1_we_i('0), //
cv32e40p_top_i.core_i.cs_registers_i.),

```

```

434
435     .csr_mhpmcounter_q_i
(cv32e40p_top_i.core_i.cs_registers_i.mhpmcounter_q),
436     .csr_mhpm-
counter_write_lower_i(cv32e40p_top_i.core_i.cs_registers_i.mhpmcoun
ter_write_lower),
437     .csr_mhpm-
counter_write_upper_i(cv32e40p_top_i.core_i.cs_registers_i.mhpmcoun
ter_write_upper),
438
439     .csr_mvvendorid_i({
440         MVENDORID_BANK, MVENDORID_OFFSET
441     }), //TODO: get this from the design instead of the pkg
442     .csr_marchid_i(MARCHID), //TODO: get this from the
design instead of the pkg
443
444     .csr_fcsr_fflags_n_i
(cv32e40p_top_i.core_i.cs_registers_i.fflags_n),
445     .csr_fcsr_fflags_q_i
(cv32e40p_top_i.core_i.cs_registers_i.fflags_q),
446     .csr_fc-
sr_fflags_we_i(cv32e40p_top_i.core_i.cs_registers_i.fflags_we_i),
447     .csr_fcsr_frm_n_i
(cv32e40p_top_i.core_i.cs_registers_i.frm_n),
448     .csr_fcsr_frm_q_i
(cv32e40p_top_i.core_i.cs_registers_i.frm_q)
449 `);
450 `endif
451
452 `ifdef CV32E40P_RVFI_TRACE_EXECUTION
453     bind cv32e40p_rvfi: rvfi_i cv32e40p_rvfi_trace #(
454         .FPU (FPU),
455         .ZFIDX(ZFIDX)
456     ) cv32e40p_tracer_i (
457         .clk_i(clk_i),
458         .rst_ni(rst_ni),
459         .hart_id_i(cv32e40p_top_i.core_i.hart_id_i),
460
461         .imm_s3_type(cv32e40p_top_i.core_i.id_stage_i.imm_s3_ty
pe),
462
463         .rvfi_valid(rvfi_valid),
464         .rvfi_insn(rvfi_insn),
465         .rvfi_start_cycle(rvfi_start_cycle),
466         .rvfi_start_time(rvfi_start_time),
467         .rvfi_stop_cycle(rvfi_stop_cycle),
468         .rvfi_stop_time(rvfi_stop_time),
469         .rvfi_pc_rdata(rvfi_pc_rdata),
470         .rvfi_trap(rvfi_trap),
471         .rvfi_rd_addr(rvfi_rd_addr),
472         .rvfi_rd_wdata(rvfi_rd_wdata),
473         .rvfi_frd_wvalid(rvfi_frd_wvalid),
474         .rvfi_frd_addr(rvfi_frd_addr),
475         .rvfi_frd_wdata(rvfi_frd_wdata),
476         .rvfi_2_rd(rvfi_2_rd),
477         .rvfi_rs1_addr(rvfi_rs1_addr),
478         .rvfi_rs2_addr(rvfi_rs2_addr),
479         .rvfi_rs3_addr(rvfi_rs3_addr),
480         .rvfi_rs1_rdata(rvfi_rs1_rdata),
481         .rvfi_rs2_rdata(rvfi_rs2_rdata),
482         .rvfi_rs3_rdata(rvfi_rs3_rdata),
483         .rvfi_frs1_addr(rvfi_frs1_addr),
484         .rvfi_frs2_addr(rvfi_frs2_addr),
485         .rvfi_frs3_addr(rvfi_frs3_addr),
486         .rvfi_frs1_rvalid(rvfi_frs1_rvalid),
487         .rvfi_frs2_rvalid(rvfi_frs2_rvalid),
488         .rvfi_frs3_rvalid(rvfi_frs3_rvalid),
489         .rvfi_frs1_rdata(rvfi_frs1_rdata),
490         .rvfi_frs2_rdata(rvfi_frs2_rdata),
491         .rvfi_frs3_rdata(rvfi_frs3_rdata),
492         .rvfi_mem_addr(rvfi_mem_addr),
493         .rvfi_mem_rmask(rvfi_mem_rmask),
494         .rvfi_mem_wmask(rvfi_mem_wmask),
495         .rvfi_mem_rdata(rvfi_mem_rdata),
496         .rvfi_mem_wdata(rvfi_mem_wdata)
497     );
498 `endif
499 // Instantiate the Core and the optional FPU
500 cv32e40p_top #(
501     .COREV_PULP (COREV_PULP),
502     .COREV_CLUSTER (COREV_CLUSTER),
503     .FPU (FPU),
504     .FPU_ADDMUL_LAT (FPU_ADDMUL_LAT),
505     .FPU_OTHERS_LAT (FPU_OTHERS_LAT),
506     .ZFIDX (ZFIDX),
507     .NUM_MHPMCOUNTERS(NUM_MHPMCOUNTERS)
508 ) cv32e40p_top_i (
509     .clk_i (clk_i),
510     .rst_ni(rst_ni),
511
512     .pulp_clock_en_i(pulp_clock_en_i),
513     .scan_cg_en_i (scan_cg_en_i),
514
515     .boot_addr_i (boot_addr_i),
516     .mtvec_addr_i (mtvec_addr_i),
517     .dm_halt_addr_i (dm_halt_addr_i),
518     .hart_id_i (hart_id_i),
519     .dm_exception_addr_i(dm_exception_addr_i),
520
521     .instr_req_o (instr_req_o),
522     .instr_gnt_i (instr_gnt_i),

```

```

523     .instr_rvalid_i(instr_rvalid_i),
524     .instr_addr_o  (instr_addr_o),
525     .instr_rdata_i (instr_rdata_i),
526
527     .data_req_o    (data_req_o),
528     .data_gnt_i    (data_gnt_i),
529     .data_rvalid_i(data_rvalid_i),
530     .data_we_o     (data_we_o),
531     .data_be_o     (data_be_o),
532     .data_addr_o   (data_addr_o),
533     .data_wdata_o  (data_wdata_o),
534     .data_rdata_i  (data_rdata_i),
535
536     .irq_i         (irq_i),
537     .irq_ack_o     (irq_ack_o),
538     .irq_id_o      (irq_id_o),
539
540     .debug_req_i   (debug_req_i),
541     .debug_havereset_o(debug_havereset_o),
542     .debug_running_o (debug_running_o),
543     .debug_halted_o (debug_halted_o),
544
545     .fetch_enable_i(fetch_enable_i),
546     .core_sleep_o  (core_sleep_o)
547 );
548
549 endmodule

```

7. openhwgroup_cv32e40p/bhv/cv32e40p_tracer.sv (563 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations under
  the License.
10
11 ///////////////////////////////////////////////////////////////////
12 // Engineer:      Andreas Traber - at-
  raber@iis.ee.ethz.ch //
13 //
14 // Additional contributions
  by:
15 //              Davide Schiavone - ps-
  chiavo@iis.ee.ethz.ch //
16 //
17 // Design Name:   RISC-V
  Tracer
18 // Project Name:
  RISCY
19 // Language:     SystemVeri-
  log //
20 //
21 // Description:   Traces the executed instruc-
  tions //
22 //
23 ///////////////////////////////////////////////////////////////////
24
25 `ifdef CV32E40P_TRACE_EXECUTION
26
27 `include "uvm_macros.svh"
28
29 module cv32e40p_tracer
30 import cv32e40p_pkg::*;
31 import uvm_pkg::*;
32 #(
33     parameter FPU    = 0,
34     parameter ZFINX  = 0
35 ) (
36     // Clock and Reset
37     input logic clk_i,
38     input logic rst_n,
39
40     input logic [31:0] hart_id_i,
41
42
43     input logic [31:0] pc,
44     input logic [31:0] instr,

```

```

45     input ctrl_state_e controller_state_i,
46
47     input logic compressed,
48     input logic id_valid,
49     input logic is_decoding,
50     input logic is_illegal,
51     input logic trigger_match,
52
53     input logic [31:0] rs1_value,
54     input logic [31:0] rs2_value,
55     input logic [31:0] rs3_value,
56
57     input logic [31:0] rs2_value_vec,
58
59     input logic rd_is_fp,
60     input logic rs1_is_fp,
61     input logic rs2_is_fp,
62     input logic rs3_is_fp,
63
64     input logic      ex_valid,
65     input logic [ 5:0] ex_reg_addr,
66     input logic      ex_reg_we,
67     input logic [31:0] ex_reg_wdata,
68
69     input logic      ex_data_req,
70     input logic      ex_data_gnt,
71     input logic      ex_data_we,
72     input logic [31:0] ex_data_addr,
73     input logic [31:0] ex_data_wdata,
74     input logic      data_misaligned,
75
76     input logic ebrk_insn,
77     input logic debug_mode,
78     input logic ebrk_force_debug_mode,
79
80     input logic wb_bypass,
81
82     input logic      wb_valid,
83     input logic [ 5:0] wb_reg_addr,
84     input logic      wb_reg_we,
85     input logic [31:0] wb_reg_wdata,
86
87     input logic [31:0] imm_u_type,
88     input logic [31:0] imm_uj_type,
89     input logic [31:0] imm_i_type,
90     input logic [11:0] imm_iz_type,
91     input logic [31:0] imm_z_type,
92     input logic [31:0] imm_s_type,
93     input logic [31:0] imm_sb_type,
94     input logic [31:0] imm_s2_type,
95     input logic [31:0] imm_s3_type,
96     input logic [31:0] imm_vs_type,
97     input logic [31:0] imm_vu_type,
98     input logic [31:0] imm_shuffle_type,
99     input logic [ 4:0] imm_clip_type,
100
101     input logic apu_en_i,
102     input logic apu_singlecycle_i,
103     input logic apu_multicycle_i,
104     input logic apu_rvalid_i,
105
106 );
107
108 import cv32e40p_tracer_pkg::*;
109
110 // Make clock a bit to avoid x->0 transitions in tracer lo-
  gic
111 bit clk_i_d, eval_comb;
112 assign #0.01 clk_i_d = clk_i;
113 assign eval_comb = ~(clk_i & ~clk_i_d);
114
115 //event   ovp_retire;
116 //bit     use_iss;
117
118 integer f;
119 string fn;
120 integer cycles;
121 logic [5:0] rd, rs1, rs2, rs3, rs4;
122
123 logic [31:0] pc_ex_stage;
124 logic [31:0] pc_ex_delay_stage;
125 logic [31:0] pc_wb_stage;
126 logic [31:0] pc_wb_delay_stage;
127 logic [31:0] pc_retire_head_q;
128
129 `include "cv32e40p_instr_trace.svh"
130
131 string      info_tag;
132
133 event       retire;
134
135 instr_trace_t trace_ex;
136 instr_trace_t trace_wb;
137 instr_trace_t trace_wb_delay;
138 instr_trace_t trace_ex_delay;
139
140 bit          clear_trace_ex = 0;
141 bit          move_trace_ex_to_trace_wb = 0;
142 bit          clear_trace_wb = 0;
143 bit          move_trace_wb_to_trace_wb_delay = 0;
144 bit          clear_trace_wb_delay = 0;

```



```

145 bit          move_trace_ex_to_trace_ex_delay = 0;
146 bit          clear_trace_ex_delay = 0;
147 bit          move_trace_ex_delay_to_trace_wb = 0;
148
149 bit          trace_new = 0;
150 bit          trace_new_ebreak = 0;
151 bit          trace_ex_misaligned = 0;
152 bit          trace_ex_retire = 0;
153 bit          trace_ex_wb_bypass = 0;
154 bit          trace_wb_retire = 0;
155 bit          trace_wb_delay_retire = 0;
156
157 bit          trace_ex_is_null = 1;
158 bit          trace_ex_delay_is_null = 1;
159 bit          trace_wb_is_null = 1;
160 bit          trace_wb_delay_is_null = 1;
161
162 bit          trace_ex_is_delay_instr = 0;
163 bit          trace_wb_is_delay_instr = 0;
164
165 string        insn_disas;
166 logic         insn_compressed;
167 logic         insn_wb_bypass;
168 logic         insn_ebreak;
169 logic         [31:0] insn_pc;
170 logic         [31:0] insn_val;
171 reg_t         insn_regs_write
[$];
172
173 instr_trace_t  trace_q
[$];
174 instr_trace_t  trace_retire;
175
176 // cycle counter
177 always_ff @(posedge clk_i, negedge rst_n) begin
178     if (rst_n == 1'b0) cycles <= 0;
179     else cycles <= cycles + 1;
180 end
181
182 initial begin
183     wait(rst_n == 1'b1);
184     $format(fn, "trace_core %h.log", hart_id_i);
185     $format(info_tag, "CORE_TRACER %2d", hart_id_i);
186     $display("[%s] Output filename is: %s", info_tag, fn);
187     f = $fopen(fn, "w");
188     $fwrite(f,
189         "
190         Time          Cycle PC      In-
str Ctx Decoded instruction Register and memory contents\n");
191 end
192 //initial begin
193 // use_iss = 0;
194 // if ($test$plusargs("USE-ISS")) use_iss = 1;
195 //end
196
197 always @(trace_ex or trace_ex_delay or trace_wb or
trace_wb_delay or trace_retire) begin
198     pc_ex_stage = (trace_ex != null) ? trace_ex.pc : 'x;
199     pc_ex_delay_stage = (trace_ex_delay != null) ?
trace_ex_delay.pc : 'x;
200     pc_wb_stage = (trace_wb != null) ? trace_wb.pc : 'x;
201     pc_wb_delay_stage = (trace_wb_delay != null) ?
trace_wb_delay.pc : 'x;
202     pc_retire_head_q = (trace_retire != null) ?
trace_retire.pc : 'x;
203 end
204
205 always @(trace_wb)
206     trace_wb_is_delay_instr = (trace_wb != null &&
is_wb_delay_instr(
207         trace_wb
208     )) ? 1 : 0;
209
210 assign rd = {rd_is_fp, instr[11:07]};
211 assign rs1 = {rs1_is_fp, instr[19:15]};
212 assign rs2 = {rs2_is_fp, instr[24:20]};
213 assign rs3 = {rs3_is_fp, instr[29:25]};
214 assign rs4 = {rs3_is_fp, instr[31:27]};
215
216 function void apply_reg_write(instr_trace_t trace, int un-
signed reg_addr, int unsigned wdata);
217     foreach (trace.reg_write[i])
218         if (trace.reg_write[i].addr == reg_addr) begin
219             trace.reg_write[i].value = wdata;
220             `uvm_info(info_tag, $formatf(
221                 "Write mapped %0d, %0d:0x%08x pc:0x%08x", i,
reg_addr, wdata, trace.pc),
222                 UVM_DEBUG)
223         end else begin
224             `uvm_info(info_tag, $formatf(
225                 "Unmapped write to %0d:0x%08x, expected
write to %0d",
226                 reg_addr,
227                 wdata,
228                 trace.reg_write[i].addr
229                 ), UVM_DEBUG)
230         end
231     endfunction : apply_reg_write
232
233 function void apply_mem_access(instr_trace_t trace, bit we,

```

```

int unsigned addr,
234                                     int unsigned wdata);
235     mem_acc_t mem_acc;
236
237     mem_acc.addr = addr;
238     mem_acc.we = we;
239
240     if (we) mem_acc.wdata = wdata;
241     else mem_acc.wdata = 'x;
242
243     trace.mem_access.push_back(mem_acc);
244 endfunction : apply_mem_access
245
246 function instr_trace_t trace_new_instr();
247     instr_trace_t trace;
248
249     trace = new();
250
251     trace.init(.cycles(cycles), .pc(pc), .compressed(compressed), .instr
(instr));
252
253     return trace;
254 endfunction : trace_new_instr
255
256 // Funnel all handoffs to the ISS here, note that this must
be automatic
257 // as multiple retire events may occur at a time (wb_bypass)
258 always begin
259     wait(trace_q.size() != 0);
260     trace_retire = trace_q.pop_front();
261     wait(trace_retire.retire != 0);
262
263     if (trace_retire.ebreak) wait(debug_mode == 1);
264
265     // Write signals and data structures used by step-and-com-
pare
266     insn_regs_write = trace_retire.reg_write;
267     insn_disas = trace_retire.str;
268     insn_compressed = trace_retire.compressed;
269     insn_pc = trace_retire.pc;
270     insn_val = trace_retire.instr;
271     insn_wb_bypass = trace_retire.wb_bypass;
272
273     trace_retire.printInstrTrace();
274
275     ->retire;
276     // if (use_iss) @(ovp_retire);
277     #0.1ns;
278 end
279
280 // EX stage
281 always @(negedge clk_i_d or negedge rst_n) begin
282     if (!rst_n) begin
283         trace_ex <= null;
284         trace_ex_is_null <= 1;
285     end else begin
286         if (trace_ex_retire) trace_ex.retire = 1;
287         if (trace_ex_wb_bypass) trace_ex.wb_bypass = 1;
288         if (trace_ex_misaligned) trace_ex.misaligned = 1;
289
290         if (trace_new_ebreak) begin
291             instr_trace_t new_instr;
292             new_instr = trace_new_instr();
293
294             // The EBREAK bypasses the pipeline so it retirable
immediately upon debug_mode entry
295             new_instr.ebreak = 1;
296             new_instr.retire = 1;
297             trace_q.push_back(new_instr);
298         end
299
300         if (trace_new) begin
301             instr_trace_t new_instr;
302             new_instr = trace_new_instr();
303             trace_q.push_back(new_instr);
304             trace_ex <= new_instr;
305             trace_ex_is_null <= 0;
306         end else if (clear_trace_ex) begin
307             trace_ex <= null;
308             trace_ex_is_null <= 1;
309         end
310     end
311
312 // EX delay stage
313 always @(negedge clk_i_d or negedge rst_n) begin
314     if (!rst_n) begin
315         trace_ex_delay <= null;
316         trace_ex_delay_is_null <= 1;
317     end else begin
318         if (move_trace_ex_to_trace_ex_delay) begin
319             trace_ex_delay <= trace_ex;
320             trace_ex_delay_is_null <= (trace_ex == null) ? 1 : 0;
321         end else if (clear_trace_ex_delay) begin
322             trace_ex_delay <= null;
323             trace_ex_delay_is_null <= 1;
324         end
325     end
326 end
327
328 // WB stage

```

```

329 always @(negedge clk_i_d or negedge rst_n) begin
330     if (rst_n) begin
331         if (trace_wb_retire) begin
332             if (trace_wb != null) begin
333                 trace_wb.retire = 1;
334             end else begin
335                 `uvm_fatal(info_tag, "Received retire signal with no
valid trace_wb instruction");
336             end
337         end
338     end
339 end
340
341 always @(negedge clk_i_d or negedge rst_n) begin
342     if (!rst_n) begin
343         trace_wb <= null;
344         trace_wb_is_null <= 1;
345     end else begin
346         if (move_trace_ex_to_trace_wb) begin
347             trace_wb <= trace_ex;
348             trace_wb_is_null <= (trace_ex == null) ? 1 : 0;
349         end else if (move_trace_ex_delay_to_trace_wb) begin
350             trace_wb <= trace_ex_delay;
351             trace_wb_is_null <= (trace_ex_delay == null) ? 1 : 0;
352         end else if (clear_trace_wb) begin
353             trace_wb <= null;
354             trace_wb_is_null <= 1;
355         end
356     end
357 end
358
359 // WB delay stage
360 always @(negedge clk_i_d or negedge rst_n) begin
361     if (!rst_n) begin
362         trace_wb_delay <= null;
363         trace_wb_delay_is_null <= 1;
364     end else begin
365         if (trace_wb_delay_retire) trace_wb_delay.retire = 1;
366
367         if (move_trace_wb_to_trace_wb_delay) begin
368             trace_wb_delay <= trace_wb;
369             trace_wb_delay_is_null <= (trace_wb == null) ? 1 : 0;
370         end else if (clear_trace_wb_delay) begin
371             trace_wb_delay <= null;
372             trace_wb_delay_is_null <= 1;
373         end
374     end
375 end
376
377 function bit is_wb_delay_instr(instr_trace_t trace_wb);
378     if (trace_wb.str == "mret" || trace_wb.str == "uret" ||
trace_wb.str == "ebreak" || trace_wb.str == "c.ebreak")
379         return 1;
380     return 0;
381 endfunction : is_wb_delay_instr
382
383
384 always @* begin
385     trace_new = 0;
386     trace_new_ebreak = 0;
387     trace_ex_misaligned = 0;
388     trace_ex_retire = 0;
389     trace_ex_wb_bypass = 0;
390     trace_wb_retire = 0;
391     trace_wb_delay_retire = 0;
392
393     clear_trace_ex = 0;
394
395     move_trace_ex_to_trace_wb = 0;
396     clear_trace_wb = 0;
397
398     move_trace_wb_to_trace_wb_delay = 0;
399     clear_trace_wb_delay = 0;
400
401     move_trace_ex_to_trace_ex_delay = 0;
402     move_trace_ex_delay_to_trace_wb = 0;
403     clear_trace_ex_delay = 0;
404
405     // -----
406     // WB Delay logic
407     // -----
408     if (!trace_wb_delay_is_null && eval_comb) begin
409         // Always retire
410         trace_wb_delay_retire = 1;
411         clear_trace_wb_delay = 1;
412     end
413
414     // -----
415     // WB logic
416     // -----
417     if (!trace_wb_is_null && eval_comb) begin
418         if (wb_valid) begin
419             // Some instructions get an extra cycle to retire
420             if (trace_wb_is_delay_instr) begin
421                 move_trace_wb_to_trace_wb_delay = 1;
422             end else begin
423                 trace_wb_retire = 1;
424                 clear_trace_wb = 1;
425             end
426         end

```

```

427     end
428
429     // -----
430     // EX Delay logic
431     // -----
432     // apu_rvalid_i for variable latency apu offloading
433     // non apu instructions will always go to wb as fast as
possible (i.e. next cycle)
434     if (!trace_ex_delay_is_null && eval_comb) begin
435         if (apu_rvalid_i || !trace_ex_delay.is_apu) begin
436             move_trace_ex_delay_to_trace_wb = 1;
437             clear_trace_ex_delay = 1;
438         end
439     end
440
441     // -----
442     // EX logic
443     // -----
444
445     // New instruction created if is a legal decoded instruc-
tion
446     if (id_valid && is_decoding && !is_illegal) begin
447         trace_new = 1;
448         // Create a new EBREAK instruction (will bypass
pipeline execution)
449     end else if (is_decoding && !trigger_match && ebrk_insn &&
(ebrk_force_debug_mode || debug_mode)) begin
450         trace_new_ebreak = 1;
451     end
452
453     // Instruction remains in the pipeline for one more cycle
if misaligned
454     if (!trace_ex_is_null && eval_comb && ex_valid &&
data_misaligned) begin
455         trace_ex_misaligned = 1;
456         // Instruction bypasses WB - mark as retireable and do
not advance
457     end else if (wb_bypass) begin
458         trace_ex_retire = 1;
459         trace_ex_wb_bypass = 1;
460         clear_trace_ex = 1;
461
462         // Some instructions get an extra cycle to retire
463         // offload to (potentially) multicyle APU/FPU
464         end else if (!trace_ex_is_null && eval_comb && apu_en_i
&& !apu_rvalid_i) begin
465             move_trace_ex_to_trace_ex_delay = 1;
466             clear_trace_ex = 1;
467
468             // Instruction leaves EX
469         end else if (!trace_ex_is_null && eval_comb && ex_valid
&& !data_misaligned) begin
470             // if ex delay is already writing to wb then we also
delay this one
471             if (move_trace_ex_delay_to_trace_wb) begin
472                 move_trace_ex_to_trace_ex_delay = 1;
473                 clear_trace_ex = 1;
474             end else begin
475                 move_trace_ex_to_trace_wb = 1;
476                 clear_trace_ex = 1;
477             end
478         end
479
480         if (move_trace_ex_to_trace_wb &&
move_trace_ex_delay_to_trace_wb) begin
481             `uvm_info(info_tag, "ex delay stage and ex stage col-
lide", UVM_DEBUG);
482         end
483     end
484
485     // Monitors for memory access and register writeback
486     always @(negedge clk_i or negedge rst_n) begin
487         if (!rst_n) begin
488             end else begin
489                 // Register updates in EX
490                 // !wb_valid necessary for CSR to GPR write when OBI
Data stalled
491                 if (ex_reg_we && (ex_valid || !wb_valid || apu_rval-
id_i)) begin
492                     `uvm_info(info_tag, $sformatf("EX: Reg WR %02d =
0x%08x", ex_reg_addr, ex_reg_wdata),
493                         UVM_DEBUG);
494                     if (!trace_ex_delay_is_null && !
trace_ex_delay.got_regs_write && !trace_ex_delay.is_load &&
495                         (!trace_ex_delay.is_apu && (ex_valid || !wb_val-
id_i)) ||
496                         (trace_ex_delay.is_apu && apu_rvalid_i &&
(apu_singlecycle_i || apu_multicycle_i))
497                     )
498                         ) begin
499                             apply_reg_write(trace_ex_delay, ex_reg_addr,
ex_reg_wdata);
500                             trace_ex_delay.got_regs_write = 1;
501                         end else if (!trace_ex_is_null) begin
502                             apply_reg_write(trace_ex, ex_reg_addr,
ex_reg_wdata);
503                             if (trace_ex.got_regs_write) begin
504                                 `uvm_info(info_tag, $sformatf(
505                                     "EX: Multiple Reg WR %02d = 0x%08x",
ex_reg_addr, ex_reg_wdata), UVM_DEBUG);

```



```

506         end
507         trace_ex.got_regs_write = 1;
508     end else begin
509         `uvm_info(info_tag, $sformatf(
510             "EX: Reg WR %02d:0x%08x but no active EX
instruction", ex_reg_addr, ex_reg_wdata
511             ), UVM_DEBUG);
512     end
513 end
514
515 // Register updates in WB
516 if (wb_reg_we) begin
517     `uvm_info(info_tag, $sformatf("WB: Reg WR %02d =
0x%08x", wb_reg_addr, wb_reg_wdata),
518         UVM_DEBUG);
519     if (!trace_ex_delay.is_null &&
520         ((trace_ex_delay.is_load ||
521          (trace_ex_delay.is_apu && apu_rvalid_i && !
apu_singlecycle_i && !apu_multicycle_i)
522         )) begin
523         apply_reg_write(trace_ex_delay, wb_reg_addr,
wb_reg_wdata);
524     end else if (!trace_wb_is_null && !
trace_wb.got_regs_write) begin
525         if (!trace_wb.is_load || (trace_wb.is_load &&
wb_valid)) begin
526             apply_reg_write(trace_wb, wb_reg_addr,
wb_reg_wdata);
527             trace_wb.got_regs_write = 1;
528         end
529     end else if (!trace_ex_is_null && !
trace_ex.got_regs_write && trace_ex.misaligned) begin
530         // Do nothing as double load concatenation will be
managed by trace_wb
531     end else begin
532         `uvm_info(info_tag, $sformatf(
533             "WB: Reg WR %02d:0x%08x but no active WB
instruction", wb_reg_addr, wb_reg_wdata
534             ), UVM_DEBUG);
535     end
536 end
537
538 // Memory access in EX
539 if (ex_data_req && ex_data_gnt) begin
540     if (ex_data_we) begin
541         `uvm_info(info_tag, $sformatf("EX: Mem WR 0x%08x =
0x%08x", ex_data_addr, ex_data_wdata),
542             UVM_DEBUG);
543     end else begin
544         `uvm_info(info_tag, $sformatf("EX: Mem RD 0x%08x",
ex_data_addr), UVM_DEBUG);
545     end
546     if (trace_ex_is_null) begin
547         `uvm_info(info_tag, $sformatf(
548             "EX: Mem %s 0x%08x:0x%08x but no active EX
instruction",
549                 ex_data_we ? "WR" : "RD",
550                 ex_data_addr,
551                 ex_reg_wdata
552             ), UVM_DEBUG);
553     end else apply_mem_access(trace_ex, ex_data_we,
ex_data_addr, ex_data_wdata);
554 end
555 end
556 endmodule : cv32e40p_tracer
557
558 `endif // CV32E40P_TRACE_EXECUTION

```

8. openhwgroup_cv32e40p/bhv/insn_trace.sv (1,132 lines)

```

1 // Copyright 2024 OpenHW Group and Dolphin Design
2 // SPDX-License-Identifier: Apache-2.0 WITH SHL-2.1
3 //
4 // Licensed under the Solderpad Hardware License v 2.1 (the
"License");
5 // you may not use this file except in compliance with the Li-
cense, or,
6 // at your option, the Apache License version 2.0.
7 // You may obtain a copy of the License at
8 //
9 // https://solderpad.org/licenses/SHL-2.1/
10 //
11 // Unless required by applicable law or agreed to in writing,
any work
12 // distributed under the License is distributed on an "AS IS"
BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14 // See the License for the specific language governing permis-
sions and
15 // limitations under the License.
16
17 ///////////////////////////////////////////////////////////////////
18 ///////////////////////////////////////////////////////////////////

```

```

18 //
19 // Contributors: Yoann Pruvost, Dolphin Design
<yoann.pruvost@dolphin.fr>
20 //
21 // Description: Macros and Functions to print information on
RVFI interface
22 //
23 ///////////////////////////////////////////////////////////////////
24
25 `define DEFINE_CSR(CSR_NAME) \
26     logic ``CSR_NAME``_we; \
27     logic [31:0] ``CSR_NAME``_rdata; \
28     logic [31:0] ``CSR_NAME``_rmask; \
29     logic [31:0] ``CSR_NAME``_wdata; \
30     logic [31:0] ``CSR_NAME``_wmask;
31
32 `define ASSIGN_CSR(CSR_NAME) \
33     this.m_csr.``CSR_NAME``_we =
m_source.m_csr.``CSR_NAME``_we; \
34     this.m_csr.``CSR_NAME``_rdata =
m_source.m_csr.``CSR_NAME``_rdata; \
35     this.m_csr.``CSR_NAME``_rmask =
m_source.m_csr.``CSR_NAME``_rmask; \
36     this.m_csr.``CSR_NAME``_wdata =
m_source.m_csr.``CSR_NAME``_wdata; \
37     this.m_csr.``CSR_NAME``_wmask =
m_source.m_csr.``CSR_NAME``_wmask;
38
39 `define INIT_CSR(CSR_NAME) \
40     this.m_csr.``CSR_NAME``_we = '0; \
41     this.m_csr.``CSR_NAME``_wmask = '0;
42
43 import cv32e40p_tracer_pkg::*;
44 class insn_trace_t;
45     bit m_valid;
46     logic [63:0] m_order;
47     integer m_start_cycle;
48     integer m_stop_cycle;
49     time m_start_time;
50     time m_stop_time;
51     bit m_skip_order; //next order was used by trap;
52     logic [31:0] m_pc_rdata;
53     logic [31:0] m_insn;
54     string m_mnemonic;
55     logic m_is_ebreak;
56     logic m_is_illegal;
57     logic m_is_irq;
58     logic m_is_memory;
59     logic m_is_load;
60     logic m_is_apu;
61     logic m_is_apu_ok;
62     integer m_apu_req_id;
63     integer m_mem_req_id[1:0];
64     logic [ 1:0] m_mem_req_id_valid;
65     logic m_data_misaligned;
66     logic m_got_first_data;
67     logic m_got_ex_reg;
68     logic m_dbg_taken;
69     logic [2:0] m_dbg_cause;
70
71     logic m_fflags_we_non_apu;
72     logic m_frm_we_non_apu;
73     logic m_fcsr_we_non_apu;
74     logic [5:0] m_rs1_addr;
75     logic [5:0] m_rs2_addr;
76     logic [5:0] m_rs3_addr;
77     logic [31:0] m_rs1_rdata;
78     logic [31:0] m_rs2_rdata;
79     logic [31:0] m_rs3_rdata;
80
81     bit m_trap;
82
83     bit m_got_regs_write;
84     bit m_ex_fw;
85     logic [ 5:0] m_rd_addr [1:0];
86     logic [31:0] m_rd_wdata[1:0];
87     logic m_2_rd_insn; //this instruction uses 2 des-
tination registers
88     rvfi_intr_t m_intr;
89
90     bit m_move_down_pipe;
91
92     int m_instret_cnt;
93
94     int m_instret_smample_trigger; //We need to sample minstret
from csr 2 cycle after id is doen
95
96     bit m_sample_csr_write_in_ex;
97
98     struct {
99         logic [31:0] addr ;
100         logic [31:0] rmask;
101         logic [31:0] rdata;
102         logic [31:0] wmask;

```

```

102     logic [31:0] wdata;
103 } m_mem;
104
105 struct {
106     logic mstatus_we;
107     logic [31:0] mstatus_rmask;
108     Status_t mstatus_wdata;
109     logic [31:0] mstatus_wmask;
110     Status_t mstatus_rdata;
111
112     logic mstatus_fs_we;
113     FS_t mstatus_fs_rdata;
114     logic [31:0] mstatus_fs_rmask;
115     FS_t mstatus_fs_wdata;
116     logic [31:0] mstatus_fs_wmask;
117
118     // mstatush
119
120     `DEFINE_CSR(misa)
121     `DEFINE_CSR(mie)
122     `DEFINE_CSR(mtvec)
123     //mtvec_mode
124     //mtvt
125     `DEFINE_CSR(mcountinhibit)
126     //mhpmevent
127
128     `DEFINE_CSR(mscratch)
129     `DEFINE_CSR(mepc)
130     `DEFINE_CSR(mcause)
131     `DEFINE_CSR(mcycle)
132     `DEFINE_CSR(minstret)
133     bit got_minstret;
134     `DEFINE_CSR(mcycleh)
135     `DEFINE_CSR(minstreth)
136     `DEFINE_CSR(cycle)
137     `DEFINE_CSR(instret)
138     // bit got_minstret;
139     `DEFINE_CSR(cycleh)
140     `DEFINE_CSR(instreth)
141
142     logic [31:0][1:0] mhpmmcounter_we;
143     logic [31:0][63:0] mhpmmcounter_rdata;
144     logic [31:0][63:0] mhpmmcounter_rmask;
145     logic [31:0][63:0] mhpmmcounter_wdata;
146     logic [31:0][63:0] mhpmmcounter_wmask;
147
148     logic [31:0] mhpmevent_we;
149     logic [31:0][31:0] mhpmevent_rdata;
150     logic [31:0][31:0] mhpmevent_rmask;
151     logic [31:0][31:0] mhpmevent_wdata;
152     logic [31:0][31:0] mhpmevent_wmask;
153     `DEFINE_CSR(mip)
154     //mnxti
155     //mintstatus
156     //mintthresh
157     //mscratchcs
158     //mscratchcs
159     //mclicbase
160
161     `DEFINE_CSR(tdata1)
162     `DEFINE_CSR(tdata2)
163     `DEFINE_CSR(tinfo)
164     `DEFINE_CSR(dcsr)
165
166     `DEFINE_CSR(dpc)
167     `DEFINE_CSR(dscratch0)
168     `DEFINE_CSR(dscratch1)
169     //dscratch1
170     //mconfigptr
171     //mhpmmcounter
172
173     `DEFINE_CSR(mvendordid)
174     `DEFINE_CSR(marchid)
175
176     `DEFINE_CSR(fflags)
177     `DEFINE_CSR(frm)
178     `DEFINE_CSR(fcsr)
179
180     `DEFINE_CSR(lpstart0)
181     `DEFINE_CSR(lpend0)
182     `DEFINE_CSR(lpcount0)
183     `DEFINE_CSR(lpstart1)
184     `DEFINE_CSR(lpend1)
185     `DEFINE_CSR(lpcount1)
186
187 } m_csr;
188
189 enum logic[2:0] {
190     IF, ID, EX, WB, WB_2, APU
191 } m_stage;
192
193
194 function new();
195     this.m_order = 0;
196     this.m_start_cycle = 0;
197     this.m_stop_cycle = 0;
198     this.m_start_time = 0;
199     this.m_stop_time = 0;
200     this.m_skip_order = 1'b0;
201     this.m_valid = 1'b0;
202     this.m_move_down_pipe = 1'b0;
203
204     this.m_data_misaligned = 1'b0;
205     this.m_got_first_data = 1'b0;
206     this.m_got_ex_reg = 1'b0;
207     this.m_intr = 0;
208     this.m_dbg_taken = 1'b0;
209     this.m_dbg_cause = 0;
210     this.m_is_ebreak = 0;
211     this.m_is_illegal = 0;
212     this.m_is_irq = 0;
213     this.m_is_memory = 1'b0;
214     this.m_is_load = 1'b0;
215     this.m_is_apu = 1'b0;
216     this.m_is_apu_ok = 1'b0;
217     this.m_apu_req_id = 0;
218     this.m_mem_req_id[0] = 0;
219     this.m_mem_req_id[1] = 0;
220     this.m_mem_req_id_valid = 0;
221     this.m_trap = 1'b0;
222     this.m_fflags_we_non_apu = 1'b0;
223     this.m_frm_we_non_apu = 1'b0;
224     this.m_fcsr_we_non_apu = 1'b0;
225     this.m_instret_cnt = 0;
226     this.m_instret_smample_trigger = 0;
227     this.m_sample_csr_write_in_ex = 1'b1;
228
229 endfunction
230
231 function void get_mnemonic();
232
233     this.m_mnemonic = "INVALID";
234
235     if(!is_compressed_id_i) begin
236         // use casex instead of case inside due to ModelSim
237         casex (this.m_insn)
238             // Aliases
239             32'h00_00_00_13: this.m_mnemonic = "nop";
240             // Regular opcodes
241             INSTR_LUI: this.m_mnemonic = "lui";
242             INSTR_AUIPC: this.m_mnemonic = "auipc";
243             INSTR_JAL: this.m_mnemonic = "jal";
244             INSTR_JALR: this.m_mnemonic = "jalr";
245             // BRANCH
246             INSTR_BEQ: this.m_mnemonic = "beq";
247             INSTR_BNE: this.m_mnemonic = "bne";
248             INSTR_BLT: this.m_mnemonic = "blt";
249             INSTR_BGE: this.m_mnemonic = "bge";
250             INSTR_BLTU: this.m_mnemonic = "bltu";
251             INSTR_BGEU: this.m_mnemonic = "bgeu";
252             INSTR_BEQIMM: this.m_mnemonic = "cv.beqimm";
253             INSTR_BNEIMM: this.m_mnemonic = "cv.bneimm";
254             // OPIMM
255             INSTR_ADDI: this.m_mnemonic = "addi";
256             INSTR_SLTI: this.m_mnemonic = "slti";
257             INSTR_SLTIU: this.m_mnemonic = "sltiu";
258             INSTR_XORI: this.m_mnemonic = "xori";
259             INSTR_ORI: this.m_mnemonic = "ori";
260             INSTR_ANDI: this.m_mnemonic = "andi";
261             INSTR_SLLI: this.m_mnemonic = "slli";
262             INSTR_SRLI: this.m_mnemonic = "srli";
263             INSTR_SRAI: this.m_mnemonic = "srai";
264             // OP
265             INSTR_ADD: this.m_mnemonic = "add";
266             INSTR_SUB: this.m_mnemonic = "sub";
267             INSTR_SLL: this.m_mnemonic = "sll";
268             INSTR_SLT: this.m_mnemonic = "slt";
269             INSTR_SLTU: this.m_mnemonic = "sltu";
270             INSTR_XOR: this.m_mnemonic = "xor";
271             INSTR_SRL: this.m_mnemonic = "srl";
272             INSTR_SRA: this.m_mnemonic = "sra";
273             INSTR_OR: this.m_mnemonic = "or";
274             INSTR_AND: this.m_mnemonic = "and";
275             INSTR_EXTHS: this.m_mnemonic = "cv.exth";
276             INSTR_EXTBZ: this.m_mnemonic = "cv.exthz";
277             INSTR_EXTBS: this.m_mnemonic = "cv.exthb";
278             INSTR_EXTBZ: this.m_mnemonic = "cv.exthb";
279             INSTR_PAVG: this.m_mnemonic = "cv.avg";
280             INSTR_PAVGU: this.m_mnemonic = "cv.avgu";
281
282             INSTR_PADDN: this.m_mnemonic = "cv.addn";
283             INSTR_PADDUN: this.m_mnemonic = "cv.addun";
284             INSTR_PADDRN: this.m_mnemonic = "cv.addrn";
285             INSTR_PADDURN: this.m_mnemonic = "cv.addurn";
286             INSTR_PSUBN: this.m_mnemonic = "cv.subn";
287             INSTR_PSUBUN: this.m_mnemonic = "cv.subun";
288             INSTR_PSUBRN: this.m_mnemonic = "cv.subrn";
289             INSTR_PSUBURN: this.m_mnemonic = "cv.suburn";
290
291             INSTR_PADDNR: this.m_mnemonic = "cv.addnr";
292             INSTR_PADDUNR: this.m_mnemonic = "cv.addunr";
293             INSTR_PADDRNR: this.m_mnemonic = "cv.addrn";
294             INSTR_PADDURNR: this.m_mnemonic = "cv.addurnr";
295             INSTR_PSUBNR: this.m_mnemonic = "cv.subnr";
296             INSTR_PSUBUNR: this.m_mnemonic = "cv.subunr";
297             INSTR_PSUBNRN: this.m_mnemonic = "cv.subnrn";
298             INSTR_PSUBURNR: this.m_mnemonic = "cv.suburnr";
299
300             INSTR_PSLET: this.m_mnemonic = "cv.slet";
301             INSTR_PSLETU: this.m_mnemonic = "cv.sletu";
302             INSTR_PMIN: this.m_mnemonic = "cv.min";
303             INSTR_PMINU: this.m_mnemonic = "cv.minu";
304             INSTR_PMAX: this.m_mnemonic = "cv.max";

```

```

303 INSTR_PMAXU: this.m_mnemonic = "cv.maxu";
304 INSTR_PABS: this.m_mnemonic = "cv.abs";
305 INSTR_PCLIP: this.m_mnemonic = "cv.clip";
306 INSTR_PCLIPU: this.m_mnemonic = "cv.clipu";
307 INSTR_PBEXT: this.m_mnemonic = "cv.extract";
308 INSTR_PBXTU: this.m_mnemonic = "cv.extractu";
309 INSTR_PBINS: this.m_mnemonic = "cv.insert";
310 INSTR_PBCLR: this.m_mnemonic = "cv.bclr";
311 INSTR_PBSET: this.m_mnemonic = "cv.bset";
312 INSTR_PBREV: this.m_mnemonic = "cv.bitrev";
313
314 INSTR_PCLIPR: this.m_mnemonic = "cv.clipr";
315 INSTR_PCLIPUR: this.m_mnemonic = "cv.clipur";
316 INSTR_PBEXTR: this.m_mnemonic = "cv.extractr";
317 INSTR_PBEXTUR: this.m_mnemonic = "cv.extractur";
318 INSTR_PBINSR: this.m_mnemonic = "cv.inserttr";
319 INSTR_PBCLRR: this.m_mnemonic = "cv.bclrr";
320 INSTR_PBSETR: this.m_mnemonic = "cv.bsettr";
321
322 INSTR_FF1: this.m_mnemonic = "cv.ff1";
323 INSTR_FL1: this.m_mnemonic = "cv.fl1";
324 INSTR_CLB: this.m_mnemonic = "cv.clb";
325 INSTR_CNT: this.m_mnemonic = "cv.cnt";
326 INSTR_ROR: this.m_mnemonic = "cv.ror";
327
328 // FENCE
329 INSTR_FENCE: this.m_mnemonic = "fence";
330 INSTR_FENCEI: this.m_mnemonic = "fencei";
331 // SYSTEM (CSR manipulation)
332 INSTR_CSRRW: this.m_mnemonic = "csrrw";
333 INSTR_CSRRS: this.m_mnemonic = "csrrs";
334 INSTR_CSRRC: this.m_mnemonic = "csrrc";
335 INSTR_CSRRWI: this.m_mnemonic = "csrrwi";
336 INSTR_CSRRSI: this.m_mnemonic = "csrrsi";
337 INSTR_CSRRCI: this.m_mnemonic = "csrrci";
338 // SYSTEM (others)
339 INSTR_ECALL: this.m_mnemonic = "ecall";
340 INSTR_EBREAK: this.m_mnemonic = "ebreak";
341 INSTR_URET: this.m_mnemonic = "uret";
342 INSTR_MRET: this.m_mnemonic = "mret";
343 INSTR_WFI: this.m_mnemonic = "wfi";
344
345 INSTR_DRET: this.m_mnemonic = "dret";
346
347 // RV32M
348 INSTR_PMUL: this.m_mnemonic = "mul";
349 INSTR_PMUH: this.m_mnemonic = "mulh";
350 INSTR_PMULHSU: this.m_mnemonic = "mulhsu";
351 INSTR_PMULHU: this.m_mnemonic = "mulhu";
352 INSTR_DIV: this.m_mnemonic = "div";
353 INSTR_DIVU: this.m_mnemonic = "divu";
354 INSTR_REM: this.m_mnemonic = "rem";
355 INSTR_REMU: this.m_mnemonic = "remu";
356
357 // PULP MULTIPLIER
358 INSTR_PMAC: this.m_mnemonic = "cv.mac";
359 INSTR_PMSU: this.m_mnemonic = "cv.msu";
360 INSTR_PMULSN: this.m_mnemonic = "cv.mulsN";
361 INSTR_PMULHSN: this.m_mnemonic = "cv.mulhsN";
362 INSTR_PMULSRN: this.m_mnemonic = "cv.mulsRN";
363 INSTR_PMULHSRN: this.m_mnemonic = "cv.mulhsRN";
364 INSTR_PMULUN: this.m_mnemonic = "cv.muluN";
365 INSTR_PMULHUN: this.m_mnemonic = "cv.mulhuN";
366 INSTR_PMULURN: this.m_mnemonic = "cv.muluRN";
367 INSTR_PMULHURN: this.m_mnemonic = "cv.mulhuRN";
368 INSTR_PMACSN: this.m_mnemonic = "cv.macsN";
369 INSTR_PMACHSN: this.m_mnemonic = "cv.machsN";
370 INSTR_PMACSRN: this.m_mnemonic = "cv.macsRN";
371 INSTR_PMACHSRN: this.m_mnemonic = "cv.machsRN";
372 INSTR_PMACUN: this.m_mnemonic = "cv.macuN";
373 INSTR_PMACHUN: this.m_mnemonic = "cv.machuN";
374 INSTR_PMACURN: this.m_mnemonic = "cv.macuRN";
375 INSTR_PMACHURN: this.m_mnemonic = "cv.machuRN";
376
377 // FP-OP
378 INSTR_FMADD: this.m_mnemonic = "fmadd.s";
379 INSTR_FMSUB: this.m_mnemonic = "fmsub.s";
380 INSTR_FNMADD: this.m_mnemonic = "fnmadd.s";
381 INSTR_FNMSUB: this.m_mnemonic = "fnmsub.s";
382 INSTR_FADD: this.m_mnemonic = "fadd.s";
383 INSTR_FSUB: this.m_mnemonic = "fsub.s";
384 INSTR_FML: this.m_mnemonic = "fmul.s";
385 INSTR_FDIV: this.m_mnemonic = "fdiv.s";
386 INSTR_FSQRT: this.m_mnemonic = "fsqrt.s";
387 INSTR_FSGNJ: this.m_mnemonic = "fsgnj.s";
388 INSTR_FSGNJNS: this.m_mnemonic = "fsgnjn.s";
389 INSTR_FSGNJXS: this.m_mnemonic = "fsgnjx.s";
390 INSTR_FMIN: this.m_mnemonic = "fmin.s";
391 INSTR_FMAX: this.m_mnemonic = "fmax.s";
392 INSTR_FCVTWS: this.m_mnemonic = "fcvt.w.s";
393 INSTR_FCVTWUS: this.m_mnemonic = "fcvt.wu.s";
394 INSTR_FMVXS: this.m_mnemonic = "fmv.x.s";
395 INSTR_FEQS: this.m_mnemonic = "feq.s";
396 INSTR_FLTS: this.m_mnemonic = "flt.s";
397 INSTR_FLES: this.m_mnemonic = "fle.s";
398 INSTR_FCLASS: this.m_mnemonic = "fclass.s";
399 INSTR_FCVTSW: this.m_mnemonic = "fcvt.s.w";
400 INSTR_FCVTSWU: this.m_mnemonic = "fcvt.s.wu";
401 INSTR_FMVXSX: this.m_mnemonic = "fmv.s.x";
402
403 // RV32A
404 INSTR_LR: this.m_mnemonic = "lr.w";
405 INSTR_SC: this.m_mnemonic = "sc.w";
406 INSTR_AMOSWAP: this.m_mnemonic = "amoswap.w";
407 INSTR_AMOADD: this.m_mnemonic = "amoadd.w";
408 INSTR_AMOXOR: this.m_mnemonic = "amoxor.w";
409 INSTR_AMOAND: this.m_mnemonic = "amoand.w";
410 INSTR_AMOOR: this.m_mnemonic = "amoor.w";
411 INSTR_AMOMIN: this.m_mnemonic = "amomin.w";
412 INSTR_AMOMAX: this.m_mnemonic = "amomax.w";
413 INSTR_AMOMINU: this.m_mnemonic = "amominu.w";
414 INSTR_AMOMAXU: this.m_mnemonic = "amomaxu.w";
415
416 // LOAD STORE
417 INSTR_LB: this.m_mnemonic = "lb";
418 INSTR_LH: this.m_mnemonic = "lh";
419 INSTR_LW: this.m_mnemonic = "lw";
420 INSTR_LBU: this.m_mnemonic = "lbu";
421 INSTR_LHU: this.m_mnemonic = "lhu";
422 INSTR_SB: this.m_mnemonic = "sb";
423 INSTR_SH: this.m_mnemonic = "sh";
424 INSTR_SW: this.m_mnemonic = "sw";
425
426 // CUSTOM 0
427 // Post-Increment Register-Immediate Load
428 INSTR_CVLBI: this.m_mnemonic = "cv.lb";
429 INSTR_CVLBU: this.m_mnemonic = "cv.lbu";
430 INSTR_CVLHI: this.m_mnemonic = "cv.lh";
431 INSTR_CVLHIU: this.m_mnemonic = "cv.lhu";
432 INSTR_CVLWI: this.m_mnemonic = "cv.lw";
433
434 // Event Load
435 INSTR_CVELW: this.m_mnemonic = "cv.elw";
436
437 // CUSTOM_1
438 // Post-Increment Register-Register Load
439 INSTR_CVLBR: this.m_mnemonic = "cv.lb";
440 INSTR_CVLBUR: this.m_mnemonic = "cv.lbu";
441 INSTR_CVLHR: this.m_mnemonic = "cv.lh";
442 INSTR_CVLHUR: this.m_mnemonic = "cv.lhu";
443 INSTR_CVLWR: this.m_mnemonic = "cv.lw";
444
445 // Register-Register Load
446 INSTR_CVLBR: this.m_mnemonic = "cv.lb";
447 INSTR_CVLBUR: this.m_mnemonic = "cv.lbu";
448 INSTR_CVLHR: this.m_mnemonic = "cv.lh";
449 INSTR_CVLHUR: this.m_mnemonic = "cv.lhu";
450 INSTR_CVLWR: this.m_mnemonic = "cv.lw";
451
452 // Post-Increment Register-Immediate Store
453 INSTR_CVSB: this.m_mnemonic = "cv.sb";
454 INSTR_CVSH: this.m_mnemonic = "cv.sh";
455 INSTR_CVSW: this.m_mnemonic = "cv.sw";
456
457 // Post-Increment Register-Register Store operations
458 INSTR_CVSB: this.m_mnemonic = "cv.sb";
459 INSTR_CVSH: this.m_mnemonic = "cv.sh";
460 INSTR_CVSW: this.m_mnemonic = "cv.sw";
461
462 // Register-Register Store operations
463 INSTR_CVSBRR: this.m_mnemonic = "cv.sb";
464 INSTR_CVSHRR: this.m_mnemonic = "cv.sh";
465 INSTR_CVSWRR: this.m_mnemonic = "cv.sw";
466
467 // Hardware Loops
468 INSTR_CVSTARTI0: this.m_mnemonic = "cv.starti 0";
469 INSTR_CVSTART0: this.m_mnemonic = "cv.start 0";
470 INSTR_CVSENDI0: this.m_mnemonic = "cv.endi 0";
471 INSTR_CVEND0: this.m_mnemonic = "cv.end 0";
472 INSTR_CVCOUNTI0: this.m_mnemonic = "cv.counti 0";
473 INSTR_CVCOUNT0: this.m_mnemonic = "cv.count 0";
474 INSTR_CVSETUPI0: this.m_mnemonic = "cv.setupi 0";
475 INSTR_CVSETUP0: this.m_mnemonic = "cv.setup 0";
476 INSTR_CVSTARTI1: this.m_mnemonic = "cv.starti 1";
477 INSTR_CVSTART1: this.m_mnemonic = "cv.start 1";
478 INSTR_CVSENDI1: this.m_mnemonic = "cv.endi 1";
479 INSTR_CVEND1: this.m_mnemonic = "cv.end 1";
480 INSTR_CVCOUNTI1: this.m_mnemonic = "cv.counti 1";
481 INSTR_CVCOUNT1: this.m_mnemonic = "cv.count 1";
482 INSTR_CVSETUPI1: this.m_mnemonic = "cv.setupi 1";
483 INSTR_CVSETUP1: this.m_mnemonic = "cv.setup 1";
484
485 // CUSTOM 3
486 // SIMD ALU
487 INSTR_CVADDH: this.m_mnemonic = "cv.add.h";
488 INSTR_CVADDSC: this.m_mnemonic = "cv.add.sc.h";
489 INSTR_CVADDSCIH: this.m_mnemonic = "cv.add.sci.h";
490 INSTR_CVADDDB: this.m_mnemonic = "cv.add.b";
491 INSTR_CVADDSCB: this.m_mnemonic = "cv.add.sc.b";
492 INSTR_CVADDSCIB: this.m_mnemonic = "cv.add.sci.b";
493 INSTR_CVSUBH: this.m_mnemonic = "cv.sub.h";
494 INSTR_CVSUBSCH: this.m_mnemonic = "cv.sub.sc.h";

```

495	INSTR_CVSUBSCIH	: this.m_mnemonic =	"cv.sub.sci.h";	495	INSTR_CVSUBSCIH	: this.m_mnemonic =	"cv.sub.sci.h";
496	INSTR_CVSUBB	: this.m_mnemonic =	"cv.sub.b";	496	INSTR_CVSUBB	: this.m_mnemonic =	"cv.sub.b";
497	INSTR_CVSUBSCB	: this.m_mnemonic =	"cv.sub.sc.b";	497	INSTR_CVSUBSCB	: this.m_mnemonic =	"cv.sub.sc.b";
498	INSTR_CVSUBSCIB	: this.m_mnemonic =	"cv.sub.sci.b";	498	INSTR_CVSUBSCIB	: this.m_mnemonic =	"cv.sub.sci.b";
499	INSTR_CVAVGH	: this.m_mnemonic =	"cv.avg.h";	499	INSTR_CVAVGH	: this.m_mnemonic =	"cv.avg.h";
500	INSTR_CVAVGSCH	: this.m_mnemonic =	"cv.avg.sc.h";	500	INSTR_CVAVGSCH	: this.m_mnemonic =	"cv.avg.sc.h";
501	INSTR_CVAVGSCIH	: this.m_mnemonic =	"cv.avg.sci.h";	501	INSTR_CVAVGSCIH	: this.m_mnemonic =	"cv.avg.sci.h";
502	INSTR_CVAVGB	: this.m_mnemonic =	"cv.avg.b";	502	INSTR_CVAVGB	: this.m_mnemonic =	"cv.avg.b";
503	INSTR_CVAVGSCB	: this.m_mnemonic =	"cv.avg.sc.b";	503	INSTR_CVAVGSCB	: this.m_mnemonic =	"cv.avg.sc.b";
504	INSTR_CVAVGSCIB	: this.m_mnemonic =	"cv.avg.sci.b";	504	INSTR_CVAVGSCIB	: this.m_mnemonic =	"cv.avg.sci.b";
505	INSTR_CVAVGUH	: this.m_mnemonic =	"cv.avgu.h";	505	INSTR_CVAVGUH	: this.m_mnemonic =	"cv.avgu.h";
506	INSTR_CVAVGUSCH	: this.m_mnemonic =	"cv.avgu.sc.h";	506	INSTR_CVAVGUSCH	: this.m_mnemonic =	"cv.avgu.sc.h";
507	INSTR_CVAVGUSCIH	: this.m_mnemonic =	"cv.avgu.sci.h";	507	INSTR_CVAVGUSCIH	: this.m_mnemonic =	"cv.avgu.sci.h";
508	INSTR_CVAVGUB	: this.m_mnemonic =	"cv.avgu.b";	508	INSTR_CVAVGUB	: this.m_mnemonic =	"cv.avgu.b";
509	INSTR_CVAVGUSCB	: this.m_mnemonic =	"cv.avgu.sc.b";	509	INSTR_CVAVGUSCB	: this.m_mnemonic =	"cv.avgu.sc.b";
510	INSTR_CVAVGUSCIB	: this.m_mnemonic =	"cv.avgu.sci.b";	510	INSTR_CVAVGUSCIB	: this.m_mnemonic =	"cv.avgu.sci.b";
511	INSTR_CVMINH	: this.m_mnemonic =	"cv.min.h";	511	INSTR_CVMINH	: this.m_mnemonic =	"cv.min.h";
512	INSTR_CVMINSCH	: this.m_mnemonic =	"cv.min.sc.h";	512	INSTR_CVMINSCH	: this.m_mnemonic =	"cv.min.sc.h";
513	INSTR_CVMINSCH	: this.m_mnemonic =	"cv.min.sci.h";	513	INSTR_CVMINSCH	: this.m_mnemonic =	"cv.min.sci.h";
514	INSTR_CVMINB	: this.m_mnemonic =	"cv.min.b";	514	INSTR_CVMINB	: this.m_mnemonic =	"cv.min.b";
515	INSTR_CVMINSCB	: this.m_mnemonic =	"cv.min.sc.b";	515	INSTR_CVMINSCB	: this.m_mnemonic =	"cv.min.sc.b";
516	INSTR_CVMINSCIB	: this.m_mnemonic =	"cv.min.sci.b";	516	INSTR_CVMINSCIB	: this.m_mnemonic =	"cv.min.sci.b";
517	INSTR_CVMINUH	: this.m_mnemonic =	"cv.minu.h";	517	INSTR_CVMINUH	: this.m_mnemonic =	"cv.minu.h";
518	INSTR_CVMINUSCH	: this.m_mnemonic =	"cv.minu.sc.h";	518	INSTR_CVMINUSCH	: this.m_mnemonic =	"cv.minu.sc.h";
519	INSTR_CVMINUSCIH	: this.m_mnemonic =	"cv.minu.sci.h";	519	INSTR_CVMINUSCIH	: this.m_mnemonic =	"cv.minu.sci.h";
520	INSTR_CVMINUB	: this.m_mnemonic =	"cv.minu.b";	520	INSTR_CVMINUB	: this.m_mnemonic =	"cv.minu.b";
521	INSTR_CVMINUSCB	: this.m_mnemonic =	"cv.minu.sc.b";	521	INSTR_CVMINUSCB	: this.m_mnemonic =	"cv.minu.sc.b";
522	INSTR_CVMINUSCIB	: this.m_mnemonic =	"cv.minu.sci.b";	522	INSTR_CVMINUSCIB	: this.m_mnemonic =	"cv.minu.sci.b";
523	INSTR_CVMAXH	: this.m_mnemonic =	"cv.max.h";	523	INSTR_CVMAXH	: this.m_mnemonic =	"cv.max.h";
524	INSTR_CVMAXSCH	: this.m_mnemonic =	"cv.max.sc.h";	524	INSTR_CVMAXSCH	: this.m_mnemonic =	"cv.max.sc.h";
525	INSTR_CVMAXSCIH	: this.m_mnemonic =	"cv.max.sci.h";	525	INSTR_CVMAXSCIH	: this.m_mnemonic =	"cv.max.sci.h";
526	INSTR_CVMAXB	: this.m_mnemonic =	"cv.max.b";	526	INSTR_CVMAXB	: this.m_mnemonic =	"cv.max.b";
527	INSTR_CVMAXSCB	: this.m_mnemonic =	"cv.max.sc.b";	527	INSTR_CVMAXSCB	: this.m_mnemonic =	"cv.max.sc.b";
528	INSTR_CVMAXSCIB	: this.m_mnemonic =	"cv.max.sci.b";	528	INSTR_CVMAXSCIB	: this.m_mnemonic =	"cv.max.sci.b";
529	INSTR_CVMAXUH	: this.m_mnemonic =	"cv.maxu.h";	529	INSTR_CVMAXUH	: this.m_mnemonic =	"cv.maxu.h";
530	INSTR_CVMAXUSCH	: this.m_mnemonic =	"cv.maxu.sc.h";	530	INSTR_CVMAXUSCH	: this.m_mnemonic =	"cv.maxu.sc.h";
531	INSTR_CVMAXUSCIH	: this.m_mnemonic =	"cv.maxu.sci.h";	531	INSTR_CVMAXUSCIH	: this.m_mnemonic =	"cv.maxu.sci.h";
532	INSTR_CVMAXUB	: this.m_mnemonic =	"cv.maxu.b";	532	INSTR_CVMAXUB	: this.m_mnemonic =	"cv.maxu.b";
533	INSTR_CVMAXUSCB	: this.m_mnemonic =	"cv.maxu.sc.b";	533	INSTR_CVMAXUSCB	: this.m_mnemonic =	"cv.maxu.sc.b";
534	INSTR_CVMAXUSCIB	: this.m_mnemonic =	"cv.maxu.sci.b";	534	INSTR_CVMAXUSCIB	: this.m_mnemonic =	"cv.maxu.sci.b";
535	INSTR_CVSRLH	: this.m_mnemonic =	"cv.srl.h";	535	INSTR_CVSRLH	: this.m_mnemonic =	"cv.srl.h";
536	INSTR_CVSRLSCH	: this.m_mnemonic =	"cv.srl.sc.h";	536	INSTR_CVSRLSCH	: this.m_mnemonic =	"cv.srl.sc.h";
537	INSTR_CVSRLSCIH	: this.m_mnemonic =	"cv.srl.sci.h";	537	INSTR_CVSRLSCIH	: this.m_mnemonic =	"cv.srl.sci.h";
538	INSTR_CVSRLB	: this.m_mnemonic =	"cv.srl.b";	538	INSTR_CVSRLB	: this.m_mnemonic =	"cv.srl.b";
539	INSTR_CVSRLSCB	: this.m_mnemonic =	"cv.srl.sc.b";	539	INSTR_CVSRLSCB	: this.m_mnemonic =	"cv.srl.sc.b";
540	INSTR_CVSRLSCIB	: this.m_mnemonic =	"cv.srl.sci.b";	540	INSTR_CVSRLSCIB	: this.m_mnemonic =	"cv.srl.sci.b";
541	INSTR_CVSRAH	: this.m_mnemonic =	"cv.sra.h";	541	INSTR_CVSRAH	: this.m_mnemonic =	"cv.sra.h";
542	INSTR_CVSRASCH	: this.m_mnemonic =	"cv.sra.sc.h";	542	INSTR_CVSRASCH	: this.m_mnemonic =	"cv.sra.sc.h";
543	INSTR_CVSRASCIH	: this.m_mnemonic =	"cv.sra.sci.h";	543	INSTR_CVSRASCIH	: this.m_mnemonic =	"cv.sra.sci.h";
544	INSTR_CVSRAH	: this.m_mnemonic =	"cv.sra.b";	544	INSTR_CVSRAH	: this.m_mnemonic =	"cv.sra.b";
545	INSTR_CVSRASCB	: this.m_mnemonic =	"cv.sra.sc.b";	545	INSTR_CVSRASCB	: this.m_mnemonic =	"cv.sra.sc.b";
				546	INSTR_CVSRASCIH	: this.m_mnemonic =	"cv.sra.sci.b";
				547	INSTR_CVSLLH	: this.m_mnemonic =	"cv.sll.h";
				548	INSTR_CVSLLSCH	: this.m_mnemonic =	"cv.sll.sc.h";
				549	INSTR_CVSLLSCIH	: this.m_mnemonic =	"cv.sll.sci.h";
				550	INSTR_CVSLLB	: this.m_mnemonic =	"cv.sll.b";
				551	INSTR_CVSLLSCB	: this.m_mnemonic =	"cv.sll.sc.b";
				552	INSTR_CVSLLSCIB	: this.m_mnemonic =	"cv.sll.sci.b";
				553	INSTR_CVORH	: this.m_mnemonic =	"cv.or.h";
				554	INSTR_CVORSCH	: this.m_mnemonic =	"cv.or.sc.h";
				555	INSTR_CVORSCIH	: this.m_mnemonic =	"cv.or.sci.h";
				556	INSTR_CVORB	: this.m_mnemonic =	"cv.or.b";
				557	INSTR_CVORSCB	: this.m_mnemonic =	"cv.or.sc.b";
				558	INSTR_CVORSCIB	: this.m_mnemonic =	"cv.or.sci.b";
				559	INSTR_CVXORH	: this.m_mnemonic =	"cv.xor.h";
				560	INSTR_CVXORSCH	: this.m_mnemonic =	"cv.xor.sc.h";
				561	INSTR_CVXORSCIH	: this.m_mnemonic =	"cv.xor.sci.h";
				562	INSTR_CVXORB	: this.m_mnemonic =	"cv.xor.b";
				563	INSTR_CVXORSCB	: this.m_mnemonic =	"cv.xor.sc.b";
				564	INSTR_CVXORSCIB	: this.m_mnemonic =	"cv.xor.sci.b";
				565	INSTR_CVANDH	: this.m_mnemonic =	"cv.and.h";
				566	INSTR_CVANDSCH	: this.m_mnemonic =	"cv.and.sc.h";
				567	INSTR_CVANDSCIH	: this.m_mnemonic =	"cv.and.sci.h";
				568	INSTR_CVANDB	: this.m_mnemonic =	"cv.and.b";
				569	INSTR_CVANDSCB	: this.m_mnemonic =	"cv.and.sc.b";
				570	INSTR_CVANDSCIB	: this.m_mnemonic =	"cv.and.sci.b";
				571	INSTR_CVABSH	: this.m_mnemonic =	"cv.abs.h";
				572	INSTR_CVABSB	: this.m_mnemonic =	"cv.abs.b";
				573	INSTR_CVEXTRACTH	: this.m_mnemonic =	"cv.ex-tract.h";
				574	INSTR_CVEXTRACTB	: this.m_mnemonic =	"cv.ex-tract.b";
				575	INSTR_CVEXTRACTUH	: this.m_mnemonic =	"cv.ex-tractu.h";
				576	INSTR_CVEXTRACTUB	: this.m_mnemonic =	"cv.ex-tractu.b";
				577	INSTR_CVINERTH	: this.m_mnemonic =	"cv.in-sert.h";
				578	INSTR_CVINERTB	: this.m_mnemonic =	"cv.in-sert.b";
				579	INSTR_CVDOTUPH	: this.m_mnemonic =	"cv.dotup.h";
				580	INSTR_CVDOTUPSCH	: this.m_mnemonic =	"cv.dotup.sc.h";
				581	INSTR_CVDOTUPSCIH	: this.m_mnemonic =	"cv.dotup.sci.h";
				582	INSTR_CVDOTUPB	: this.m_mnemonic =	"cv.dotup.b";
				583	INSTR_CVDOTUPSCB	: this.m_mnemonic =	"cv.dotup.sc.b";
				584	INSTR_CVDOTUPSCIB	: this.m_mnemonic =	"cv.dotup.sci.b";
				585	INSTR_CVDOTUSPH	: this.m_mnemonic =	"cv.dotusp.h";
				586	INSTR_CVDOTUSPSCH	: this.m_mnemonic =	"cv.dotusp.sc.h";
				587	INSTR_CVDOTUSPSCIH	: this.m_mnemonic =	"cv.dotusp.sci.h";
				588	INSTR_CVDOTUSPB	: this.m_mnemonic =	"cv.dotusp.b";
				589	INSTR_CVDOTUSPSCB	: this.m_mnemonic =	"cv.dotusp.sc.b";
				590	INSTR_CVDOTUSPSCIB	: this.m_mnemonic =	"cv.dotusp.sci.b";
				591	INSTR_CVDOTUSPH	: this.m_mnemonic =	"cv.dot-sp.h";
				592	INSTR_CVDOTUSPSCH	: this.m_mnemonic =	"cv.dot-sp.sc.h";
				593	INSTR_CVDOTUSPSCIH	: this.m_mnemonic =	"cv.dot-sp.sci.h";
				594	INSTR_CVDOTUSPB	: this.m_mnemonic =	"cv.dot-sp.b";
				595	INSTR_CVDOTUSPCB	: this.m_mnemonic =	"cv.dot-sp.sc.b";
				596	INSTR_CVDOTUSPCIB	: this.m_mnemonic =	"cv.dot-sp.sci.b";

```

597 INSTR_CVSDOTUPH : this.m_mnemonic =
"cv.sdotup.h";
598 INSTR_CVSDOTUPSCH : this.m_mnemonic =
"cv.sdotup.sc.h";
599 INSTR_CVSDOTUPSCIH : this.m_mnemonic =
"cv.sdotup.sci.h";
600 INSTR_CVSDOTUPB : this.m_mnemonic =
"cv.sdotup.b";
601 INSTR_CVSDOTUPSCB : this.m_mnemonic =
"cv.sdotup.sc.b";
602 INSTR_CVSDOTUPSCIB : this.m_mnemonic =
"cv.sdotup.sci.b";
603 INSTR_CVSDOTUSPH : this.m_mnemonic =
"cv.sdotusp.h";
604 INSTR_CVSDOTUSPSCH : this.m_mnemonic =
"cv.sdotusp.sc.h";
605 INSTR_CVSDOTUSPSCIH : this.m_mnemonic =
"cv.sdotusp.sci.h";
606 INSTR_CVSDOTUSPB : this.m_mnemonic =
"cv.sdotusp.b";
607 INSTR_CVSDOTUSPCB : this.m_mnemonic =
"cv.sdotusp.sc.b";
608 INSTR_CVSDOTUSPCIB : this.m_mnemonic =
"cv.sdotusp.sci.b";
609 INSTR_CVSDOTSPH : this.m_mnemonic = "cv.sdot-
sp.h";
610 INSTR_CVSDOTSPSCH : this.m_mnemonic = "cv.sdot-
sp.sc.h";
611 INSTR_CVSDOTSPSCIH : this.m_mnemonic = "cv.sdot-
sp.sci.h";
612 INSTR_CVSDOTSPB : this.m_mnemonic = "cv.sdot-
sp.b";
613 INSTR_CVSDOTSPCB : this.m_mnemonic = "cv.sdot-
sp.sc.b";
614 INSTR_CVSDOTSPCIB : this.m_mnemonic = "cv.sdot-
sp.sci.b";
615 INSTR_CVSHUFFLEH : this.m_mnemonic =
"cv.shuffle.h";
616 INSTR_CVSHUFFLESCIH : this.m_mnemonic =
"cv.shuffle.sci.h";
617 INSTR_CVSHUFFLEB : this.m_mnemonic =
"cv.shuffle.b";
618 INSTR_CVSHUFFLEL0SCIB : this.m_mnemonic =
"cv.shufflel0.sci.b";
619 INSTR_CVSHUFFLEL1SCIB : this.m_mnemonic =
"cv.shufflel1.sci.b";
620 INSTR_CVSHUFFLEL2SCIB : this.m_mnemonic =
"cv.shufflel2.sci.b";
621 INSTR_CVSHUFFLEL3SCIB : this.m_mnemonic =
"cv.shufflel3.sci.b";
622 INSTR_CVSHUFFLE2H : this.m_mnemonic =
"cv.shuffle2.h";
623 INSTR_CVSHUFFLE2B : this.m_mnemonic =
"cv.shuffle2.b";
624 INSTR_CVPACK : this.m_mnemonic = "cv.pack";
625 INSTR_CVPACKH : this.m_mnemonic =
"cv.pack.h";
626 INSTR_CVPACKHIB : this.m_mnemonic =
"cv.packhi.b";
627 INSTR_CVPACKLOB : this.m_mnemonic = "cv.pack-
lo.b";
628
629 // SIMD COMPARISON
630 INSTR_CVCMPQEH : this.m_mnemonic = "cv.cmpq.h";
631 INSTR_CVCMPQSCH : this.m_mnemonic = "cv.cm-
peq.sc.h";
632 INSTR_CVCMPQSCIH : this.m_mnemonic = "cv.cm-
peq.sci.h";
633 INSTR_CVCMPQEB : this.m_mnemonic = "cv.cmpq.b";
634 INSTR_CVCMPQSCB : this.m_mnemonic = "cv.cm-
peq.sc.b";
635 INSTR_CVCMPQSCIB : this.m_mnemonic = "cv.cm-
peq.sci.b";
636 INSTR_CVCMPNEH : this.m_mnemonic = "cv.cmpne.h";
637 INSTR_CVCMPNESCH : this.m_mnemonic =
"cv.cmpne.sc.h";
638 INSTR_CVCMPNESCIH : this.m_mnemonic =
"cv.cmpne.sci.h";
639 INSTR_CVCMPNEB : this.m_mnemonic = "cv.cmpne.b";
640 INSTR_CVCMPNEB : this.m_mnemonic =
"cv.cmpne.sc.b";
641 INSTR_CVCMPNEB : this.m_mnemonic =
"cv.cmpne.sci.b";
642 INSTR_CVCMPGTH : this.m_mnemonic = "cv.cmpgt.h";
643 INSTR_CVCMPGTSC : this.m_mnemonic = "cv.cmp-
gt.sc.h";
644 INSTR_CVCMPGTSCIH : this.m_mnemonic = "cv.cmp-
gt.sci.h";
645 INSTR_CVCMPGTB : this.m_mnemonic = "cv.cmpgt.b";
646 INSTR_CVCMPGTSCB : this.m_mnemonic = "cv.cmp-
gt.sc.b";
647 INSTR_CVCMPGTSCIB : this.m_mnemonic = "cv.cmp-
gt.sci.b";
648 INSTR_CVCMPGEH : this.m_mnemonic = "cv.cmpge.h";
649 INSTR_CVCMPGESCH : this.m_mnemonic = "cv.cmp-
ge.sc.h";
650 INSTR_CVCMPGESCIH : this.m_mnemonic = "cv.cmp-
ge.sci.h";
651 INSTR_CVCMPGEB : this.m_mnemonic = "cv.cmpge.b";
652 INSTR_CVCMPGESCB : this.m_mnemonic = "cv.cmp-
ge.sc.b";

```

```

653 INSTR_CVCMPGESCB : this.m_mnemonic = "cv.cmp-
ge.sci.b";
654 INSTR_CVCMPPLTH : this.m_mnemonic = "cv.cmpplt.h";
655 INSTR_CVCMPPLTSC : this.m_mnemonic = "cv.cm-
plt.sc.h";
656 INSTR_CVCMPPLTSCIH : this.m_mnemonic = "cv.cm-
plt.sci.h";
657 INSTR_CVCMPPLTB : this.m_mnemonic = "cv.cmpplt.b";
658 INSTR_CVCMPPLTSCB : this.m_mnemonic = "cv.cm-
plt.sc.b";
659 INSTR_CVCMPPLTSCIB : this.m_mnemonic = "cv.cm-
plt.sci.b";
660 INSTR_CVCMPLEH : this.m_mnemonic = "cv.cmp.le.h";
661 INSTR_CVCMPLESCH : this.m_mnemonic =
"cv.cmp.le.sc.h";
662 INSTR_CVCMPLESCH : this.m_mnemonic =
"cv.cmp.le.sci.h";
663 INSTR_CVCMPLEB : this.m_mnemonic = "cv.cmp.le.b";
664 INSTR_CVCMPLEB : this.m_mnemonic =
"cv.cmp.le.sc.b";
665 INSTR_CVCMPLEB : this.m_mnemonic =
"cv.cmp.le.sci.b";
666 INSTR_CVCMPGTU : this.m_mnemonic = "cv.cmp-
gtu.h";
667 INSTR_CVCMPGTUSCH : this.m_mnemonic = "cv.cmp-
gtu.sc.h";
668 INSTR_CVCMPGTUSCIH : this.m_mnemonic = "cv.cmp-
gtu.sci.h";
669 INSTR_CVCMPGTUB : this.m_mnemonic = "cv.cmp-
gtu.b";
670 INSTR_CVCMPGTUSCB : this.m_mnemonic = "cv.cmp-
gtu.sc.b";
671 INSTR_CVCMPGTUSCIB : this.m_mnemonic = "cv.cmp-
gtu.sci.b";
672 INSTR_CVCMPGEU : this.m_mnemonic = "cv.cmp-
geu.h";
673 INSTR_CVCMPGEUSCH : this.m_mnemonic = "cv.cmp-
geu.sc.h";
674 INSTR_CVCMPGEUSCIH : this.m_mnemonic = "cv.cmp-
geu.sci.h";
675 INSTR_CVCMPGEUB : this.m_mnemonic = "cv.cmp-
geu.b";
676 INSTR_CVCMPGEUSCB : this.m_mnemonic = "cv.cmp-
geu.sc.b";
677 INSTR_CVCMPGEUSCIB : this.m_mnemonic = "cv.cmp-
geu.sci.b";
678 INSTR_CVCMPPLTU : this.m_mnemonic = "cv.cm-
pltu.h";
679 INSTR_CVCMPPLTUSCH : this.m_mnemonic = "cv.cm-
pltu.sc.h";
680 INSTR_CVCMPPLTUSCIH : this.m_mnemonic = "cv.cm-
pltu.sci.h";
681 INSTR_CVCMPPLTUB : this.m_mnemonic = "cv.cm-
pltu.b";
682 INSTR_CVCMPPLTUSCB : this.m_mnemonic = "cv.cm-
pltu.sc.b";
683 INSTR_CVCMPPLTUSCIB : this.m_mnemonic = "cv.cm-
pltu.sci.b";
684 INSTR_CVCMPLEU : this.m_mnemonic = "cv.cm-
pleu.h";
685 INSTR_CVCMPLEUSCH : this.m_mnemonic = "cv.cm-
pleu.sc.h";
686 INSTR_CVCMPLEUSCIH : this.m_mnemonic = "cv.cm-
pleu.sci.h";
687 INSTR_CVCMPLEUB : this.m_mnemonic = "cv.cm-
pleu.b";
688 INSTR_CVCMPLEUSCB : this.m_mnemonic = "cv.cm-
pleu.sc.b";
689 INSTR_CVCMPLEUSCIB : this.m_mnemonic = "cv.cm-
pleu.sci.b";
690
691 // SIMD CPLX
692 INSTR_CVCPLXMULR : this.m_mnemonic =
"cv.cplxmul.r";
693 INSTR_CVCPLXMULRDIV2 : this.m_mnemonic =
"cv.cplxmul.r.div2";
694 INSTR_CVCPLXMULRDIV4 : this.m_mnemonic =
"cv.cplxmul.r.div4";
695 INSTR_CVCPLXMULRDIV8 : this.m_mnemonic =
"cv.cplxmul.r.div8";
696 INSTR_CVCPLXMULI : this.m_mnemonic =
"cv.cplxmul.i";
697 INSTR_CVCPLXMULIDIV2 : this.m_mnemonic =
"cv.cplxmul.i.div2";
698 INSTR_CVCPLXMULIDIV4 : this.m_mnemonic =
"cv.cplxmul.i.div4";
699 INSTR_CVCPLXMULIDIV8 : this.m_mnemonic =
"cv.cplxmul.i.div8";
700 INSTR_CVCPLXCONJ : this.m_mnemonic =
"cv.cplxconj";
701 INSTR_CVSUBROT MJ : this.m_mnemonic = "cv.subrot-
mj";
702 INSTR_CVSUBROT MJDIV2 : this.m_mnemonic = "cv.subrot-
mj.div2";
703 INSTR_CVSUBROT MJDIV4 : this.m_mnemonic = "cv.subrot-
mj.div4";
704 INSTR_CVSUBROT MJDIV8 : this.m_mnemonic = "cv.subrot-
mj.div8";
705 INSTR_CVADDIV2 : this.m_mnemonic =
"cv.add.div2";
706 INSTR_CVADDIV4 : this.m_mnemonic =

```



```

"cv.add.div4";
707 INSTR_CVADDIV8 : this.m_mnemonic =
"cv.add.div8";
708 INSTR_CVSUBIV2 : this.m_mnemonic =
"cv.sub.div2";
709 INSTR_CVSUBIV4 : this.m_mnemonic =
"cv.sub.div4";
710 INSTR_CVSUBIV8 : this.m_mnemonic =
"cv.sub.div8";
711
712 default : this.m_mnemonic = "INVALID";
713 endcase // unique case (instr)
714 end else begin //Compressed instruction
715 unique case (this.m_insn[1:0])
716 // C0
717 2'b00: begin
718 unique case (this.m_insn[15:13])
719 3'b000: begin
720 // c.addi4spn -> addi rd', x2, imm
721 this.m_mnemonic = "c.addi4spn";
722 end
723
724 3'b001: begin this.m_mnemonic = "c.fld"; end
725 3'b010: begin this.m_mnemonic = "c.lw"; end
726 3'b011: begin this.m_mnemonic = "c.flw"; end
727 3'b101: begin this.m_mnemonic = "c.fsd"; end
728 3'b110: begin this.m_mnemonic = "c.sw"; end
729 3'b111: begin this.m_mnemonic = "c.fsw"; end
730 default: begin this.m_mnemonic = "INVALID"; end
731 endcase
732 end
733
734 // C1
735 2'b01: begin
736 unique case (this.m_insn[15:13])
737 3'b000: begin
738 // c.addi -> addi rd, rd, nzimm
739 // c.nop
740 if (this.m_insn[11:7] == '0) begin
741 this.m_mnemonic = "c.nop";
742 end else begin
743 this.m_mnemonic = "c.addi";
744 end
745 end
746 3'b001: this.m_mnemonic = "c.jal";
747 3'b101: this.m_mnemonic = "c.j";
748
749 3'b010: begin
750 if (this.m_insn[11:7] == 5'b0) begin
751 // Hint -> addi x0, x0, nzimm
752 this.m_mnemonic = "HINT";
753 end else begin
754 this.m_mnemonic = "c.li";
755 end
756 end
757
758 3'b011: begin
759 if ({this.m_insn[12], this.m_insn[6:2]} ==
6'b0) begin
760 this.m_mnemonic = "INVALID";
761 end else begin
762 if (this.m_insn[11:7] == 5'h02) begin
763 // c.addi16sp -> addi x2, x2, nzimm
764 this.m_mnemonic = "c.addi16sp";
765 end else if (this.m_insn[11:7] == 5'b0) be-
gin
766 // Hint -> lui x0, imm
767 this.m_mnemonic = "HINT";
768 end else begin
769 this.m_mnemonic = "c.lui";
770 end
771 end
772 end
773
774 3'b100: begin
775 unique case (this.m_insn[11:10])
776 2'b00 : begin
777 // 00: c.srli -> srli rd, rd, shamt
778 // 01: c.srai -> srai rd, rd, shamt
779 if (this.m_insn[12] == 1'b1) begin
780 // Reserved for future custom extensions
781 (instr_o don't care)
782 this.m_mnemonic = "INVALID";
783 end else begin
784 if (this.m_insn[6:2] == 5'b0) begin
785 // Hint
786 this.m_mnemonic = "HINT";
787 end else begin
788 this.m_mnemonic = "c.srli";
789 end
790 end
791 2'b01 : begin
792 if (this.m_insn[12] == 1'b1) begin
793 // Reserved for future custom extensions
794 (instr_o don't care)
795 this.m_mnemonic = "INVALID";
796 end else begin
797 if (this.m_insn[6:2] == 5'b0) begin

```

```

797 // Hint
798 this.m_mnemonic = "HINT";
799 end else begin
800 this.m_mnemonic = "c.srai";
801 end
802 end
803 end
804
805 2'b10: begin this.m_mnemonic = "c.andi"; end
806
807 2'b11: begin
808 unique case ({ this.m_insn[12], this.m_in-
sn[6:5] })
809 3'b000: begin this.m_mnemonic = "c.sub";
810 end
811 3'b001: begin this.m_mnemonic = "c.xor";
812 end
813 3'b010: begin this.m_mnemonic = "c.or";
814 end
815 3'b011: begin this.m_mnemonic = "c.and";
816 end
817 3'b100 : this.m_mnemonic = "c.subw";
818 3'b101 : this.m_mnemonic = "c.addw";
819
820 3'b110, 3'b111: begin
821 this.m_mnemonic = "INVALID";
822 end
823 endcase
824 end
825 endcase
826
827 3'b110: begin this.m_mnemonic = "c.beqz"; end
828 3'b111: begin this.m_mnemonic = "c.bnez"; end
829 endcase
830 end
831
832 // C2
833 2'b10: begin
834 unique case (this.m_insn[15:13])
835 3'b000: begin
836 if (this.m_insn[12] == 1'b1) begin
837 // Reserved for future extensions (instr_o
don't care)
838 this.m_mnemonic = "TODO";
839 end else begin
840 if ((this.m_insn[6:2] == 5'b0) ||
(this.m_insn[11:7] == 5'b0)) begin
841 // Hint -> slli rd, rd, shamt
842 this.m_mnemonic = "HINT";
843 end else begin
844 this.m_mnemonic = "c.slli";
845 end
846 end
847 end
848 3'b001: begin this.m_mnemonic = "c.fldsp"; end
849 3'b010: begin this.m_mnemonic = "c.lwsp"; end
850 3'b011: begin this.m_mnemonic = "c.flwsp"; end
851
852 3'b100: begin
853 if (this.m_insn[12] == 1'b0) begin
854 if (this.m_insn[6:2] == 5'b0) begin
855 this.m_mnemonic = "c.jr";
856 end else begin
857 if (this.m_insn[11:7] == 5'b0) begin
858 // Hint -> add x0, x0, rs2
859 this.m_mnemonic = "HINT";
860 end else begin
861 this.m_mnemonic = "c.mv";
862 end
863 end
864 end else begin
865 if (this.m_insn[6:2] == 5'b0) begin
866 if (this.m_insn[11:7] == 5'b0) begin
867 this.m_mnemonic = "c.ebreak";
868 end else begin
869 this.m_mnemonic = "c.jalr";
870 end
871 end else begin
872 if (this.m_insn[11:7] == 5'b0) begin
873 // Hint -> add x0, x0, rs2
874 this.m_mnemonic = "HINT";
875 end else begin
876 this.m_mnemonic = "c.add";
877 end
878 end
879 end
880 end
881 3'b101: begin this.m_mnemonic = "c.fsdsp"; end
882 3'b110: begin this.m_mnemonic = "c.swsp"; end
883 3'b111: begin this.m_mnemonic = "c.fswsp"; end
884 endcase
885 end
886 endcase

```

```

887     end
888 endfunction
889
890 function void init_csr();
891     `INIT_CSR(mstatus)
892     `INIT_CSR(mstatus_fs)
893     `INIT_CSR(misa)
894     `INIT_CSR(mie)
895     `INIT_CSR(mtvec)
896     `INIT_CSR(mcountinhibit)
897     `INIT_CSR(mscratch)
898     `INIT_CSR(mepc)
899     `INIT_CSR(mcause)
900     `INIT_CSR(mcycle)
901     `INIT_CSR(minstret)
902     `INIT_CSR(mcycleh)
903     `INIT_CSR(minstreth)
904     `INIT_CSR(cycle)
905     `INIT_CSR(instret)
906     `INIT_CSR(cycleh)
907     `INIT_CSR(instreth)
908     this.m_csr.mhpmcounter_we = '0;
909     this.m_csr.mhpmcounter_wmask = '0;
910     this.m_csr.mhpmevent_we = '0;
911     this.m_csr.mhpmevent_wmask = '0;
912     `INIT_CSR(mip)
913     `INIT_CSR(tdata1)
914     `INIT_CSR(tdata2)
915     `INIT_CSR(tinfo)
916     `INIT_CSR(dcsr)
917     `INIT_CSR(dpc)
918     `INIT_CSR(dscratch0)
919     `INIT_CSR(dscratch1)
920     `INIT_CSR(mvendorid)
921     `INIT_CSR(marchid)
922     `INIT_CSR(fflags)
923     `INIT_CSR(frm)
924     `INIT_CSR(fcsr)
925     `INIT_CSR(lpstart0)
926     `INIT_CSR(lpend0)
927     `INIT_CSR(lpcount0)
928     `INIT_CSR(lpstart1)
929     `INIT_CSR(lpend1)
930     `INIT_CSR(lpcount1)
931 endfunction
932 /*
933 *
934 */
935 function void init( insn_trace_t m_source); //logic[31:0]
instr_id );
936     this.m_valid = 1'b1;
937     this.m_stage = ID;
938     this.m_order = this.m_order + 64'h1;
939     this.m_start_cycle = cycles;
940     this.m_stop_cycle = 0;
941     this.m_start_time = $time;
942     this.m_stop_time = 0;
943     if(this.m_skip_order) begin
944         this.m_order = this.m_order + 64'h1;
945     end
946     this.m_skip_order = 1'b0;
947     this.m_pc_rdata =
r_pipe_freeze_trace.pc_id;
948     this.m_is_illegal = 1'b0;
949     this.m_is_irq = 1'b0;
950     this.m_is_memory = 1'b0;
951     this.m_is_load = 1'b0;
952     this.m_is_apu = 1'b0;
953     this.m_is_apu_ok = 1'b0;
954     this.m_apu_req_id = 0;
955     this.m_mem_req_id[0] = 0;
956     this.m_mem_req_id[1] = 0;
957     this.m_mem_req_id_valid = 0;
958     this.m_data_misaligned = 1'b0;
959     this.m_got_first_data = 1'b0;
960     this.m_got_ex_reg = 1'b0;
961     this.m_got_regs_write = 1'b0;
962     this.m_move_down_pipe = 1'b0;
963     this.m_instret_cnt = 0;
964     this.m_instret_smample_trigger = 0;
965     this.m_sample_csr_write_in_ex = 1'b1;
966     this.m_rd_addr[0] = 0;
967     this.m_rd_addr[1] = 0;
968     this.m_2_rd_insn = 1'b0;
969     this.m_rs1_addr = 0;
970     this.m_rs2_addr = 0;
971     this.m_rs3_addr = 0;
972     this.m_ex_fw = 0;
973     this.m_csr.got_minstret = 0;
974     this.m_dbg_taken = 0;
975     this.m_trap = 1'b0;
976     this.m_fflags_we_non_apu = 1'b0;
977     this.m_frm_we_non_apu = 1'b0;
978     this.m_fcsr_we_non_apu = 1'b0;
979     this.m_csr.mcause_we = 0;
980     if (is_compressed_id_i) begin
981         this.m_insn[31:16] = 0;
982         this.m_insn[15:0] = m_source.m_insn[15:0];
983     end else begin
984         this.m_insn = m_source.m_insn;
985     end

```

```

986     this.get_mnemonic();
987
988     this.m_intr = m_source.m_intr;
989     this.m_dbg_taken = m_source.m_dbg_taken;
990     this.m_dbg_cause = m_source.m_dbg_cause;
991     this.m_trap = m_source.m_trap;
992
993
994     this.m_rs1_addr = r_pipe_freeze_trace.rs1_addr_id;
995     this.m_rs2_addr = r_pipe_freeze_trace.rs2_addr_id;
996     this.m_rs3_addr = r_pipe_freeze_trace.rs3_addr_id;
997     this.m_rs1_rdata = r_pipe_freeze_trace.operand_a_fw_id;
998     this.m_rs2_rdata = r_pipe_freeze_trace.operand_b_fw_id;
999     this.m_rs3_rdata = r_pipe_freeze_trace.operand_c_fw_id;
1000
1001     this.m_mem.addr = 0;
1002     this.m_mem.rmask = 0;
1003     this.m_mem.wmask = 0;
1004     this.m_mem.rdata = 0;
1005     this.m_mem.wdata = 0;
1006
1007     init_csr();
1008 endfunction
1009
1010 function logic [63:0] get_order_for_trap();
1011     // this.m_order = this.m_order + 64'h1;
1012     this.m_skip_order = 1'b1;
1013     return (this.m_order + 64'h1);
1014 endfunction
1015
1016 function void copy_full(insn_trace_t m_source);
1017     this.m_valid = m_source.m_valid;
1018     this.m_stage = m_source.m_stage;
1019     this.m_order = m_source.m_order;
1020     this.m_start_cycle = m_source.m_start_cycle;
1021     this.m_stop_cycle = m_source.m_stop_cycle;
1022     this.m_start_time = m_source.m_start_time;
1023     this.m_stop_time = m_source.m_stop_time;
1024     this.m_pc_rdata = m_source.m_pc_rdata;
1025     this.m_insn = m_source.m_insn;
1026     this.m_mnemonic = m_source.m_mnemonic;
1027     this.m_is_memory = m_source.m_is_memory;
1028     this.m_is_load = m_source.m_is_load;
1029     this.m_is_apu = m_source.m_is_apu;
1030     this.m_is_apu_ok = m_source.m_is_apu_ok;
1031     this.m_apu_req_id = m_source.m_apu_req_id;
1032     this.m_mem_req_id = m_source.m_mem_req_id;
1033     this.m_mem_req_id_valid =
m_source.m_mem_req_id_valid;
1034     this.m_data_misaligned =
m_source.m_data_misaligned;
1035     this.m_got_first_data =
m_source.m_got_first_data;
1036     this.m_got_ex_reg = m_source.m_got_ex_reg;
1037     this.m_dbg_taken = m_source.m_dbg_taken;
1038     this.m_dbg_cause = m_source.m_dbg_cause;
1039     this.m_is_ebreak = m_source.m_is_ebreak;
1040     this.m_is_illegal = m_source.m_is_illegal;
1041     this.m_is_irq = m_source.m_is_irq;
1042     this.m_instret_cnt = m_source.m_instret_cnt;
1043     this.m_instret_smample_trigger =
m_source.m_instret_smample_trigger;
1044     this.m_sample_csr_write_in_ex =
m_source.m_sample_csr_write_in_ex;
1045     this.m_rs1_addr = m_source.m_rs1_addr;
1046     this.m_rs2_addr = m_source.m_rs2_addr;
1047     this.m_rs3_addr = m_source.m_rs3_addr;
1048     this.m_rs1_rdata = m_source.m_rs1_rdata;
1049     this.m_rs2_rdata = m_source.m_rs2_rdata;
1050     this.m_rs3_rdata = m_source.m_rs3_rdata;
1051
1052     this.m_ex_fw = m_source.m_ex_fw;
1053     this.m_rd_addr = m_source.m_rd_addr;
1054     this.m_2_rd_insn = m_source.m_2_rd_insn;
1055     this.m_rd_wdata = m_source.m_rd_wdata;
1056
1057     this.m_intr = m_source.m_intr;
1058     this.m_trap = m_source.m_trap;
1059     this.m_fflags_we_non_apu =
m_source.m_fflags_we_non_apu;
1060     this.m_frm_we_non_apu =
m_source.m_frm_we_non_apu;
1061     this.m_fcsr_we_non_apu =
m_source.m_fcsr_we_non_apu;
1062
1063     this.m_mem = m_source.m_mem;
1064     //CRS
1065     `ASSIGN_CSR(mstatus)
1066     `ASSIGN_CSR(mstatus_fs)
1067     `ASSIGN_CSR(misa)
1068     `ASSIGN_CSR(mie)
1069     `ASSIGN_CSR(mtvec)
1070     `ASSIGN_CSR(mcountinhibit)
1071     `ASSIGN_CSR(mscratch)
1072     `ASSIGN_CSR(mepc)
1073     `ASSIGN_CSR(mcause)
1074     `ASSIGN_CSR(mcycle)
1075     `ASSIGN_CSR(minstret)
1076     this.m_csr.got_minstret = m_source.m_csr.got_minstret;
1077     `ASSIGN_CSR(mcycleh)
1078     `ASSIGN_CSR(minstreth)

```



```

1079     `ASSIGN_CSR(cycle)
1080     `ASSIGN_CSR(instret)
1081     // this.m_csr.got_minstret =
m_source.m_csr.got_minstret;
1082     `ASSIGN_CSR(cycleh)
1083     `ASSIGN_CSR(instreth)
1084     this.m_csr.mhpmcounter_we =
m_source.m_csr.mhpmcounter_we;
1085     this.m_csr.mhpmcounter_rdata =
m_source.m_csr.mhpmcounter_rdata;
1086     this.m_csr.mhpmcounter_rmask =
m_source.m_csr.mhpmcounter_rmask;
1087     this.m_csr.mhpmcounter_wdata =
m_source.m_csr.mhpmcounter_wdata;
1088     this.m_csr.mhpmcounter_wmask =
m_source.m_csr.mhpmcounter_wmask;
1089     this.m_csr.mhpmevent_we = m_source.m_csr.mhpmevent_we;
1090     this.m_csr.mhpmevent_rdata =
m_source.m_csr.mhpmevent_rdata;
1091     this.m_csr.mhpmevent_rmask =
m_source.m_csr.mhpmevent_rmask;
1092     this.m_csr.mhpmevent_wdata =
m_source.m_csr.mhpmevent_wdata;
1093     this.m_csr.mhpmevent_wmask =
m_source.m_csr.mhpmevent_wmask;
1094     `ASSIGN_CSR(mip)
1095     `ASSIGN_CSR(tdata1)
1096     `ASSIGN_CSR(tdata2)
1097     `ASSIGN_CSR(tinfo)
1098     `ASSIGN_CSR(dcsr)
1099     `ASSIGN_CSR(dpc)
1100     `ASSIGN_CSR(dscratch0)

```

```

1101     `ASSIGN_CSR(dscratch1)
1102     `ASSIGN_CSR(mvendorid)
1103     `ASSIGN_CSR(marchid)
1104
1105     `ASSIGN_CSR(fflags)
1106     `ASSIGN_CSR(frm )
1107     `ASSIGN_CSR(fcsr )
1108
1109     `ASSIGN_CSR(lpstart0)
1110     `ASSIGN_CSR(lpend0 )
1111     `ASSIGN_CSR(lpcount0)
1112     `ASSIGN_CSR(lpstart1)
1113     `ASSIGN_CSR(lpend1 )
1114     `ASSIGN_CSR(lpcount1)
1115
1116 endfunction
1117
1118 function void move_down_pipe(insn_trace_t m_source);
1119     this.copy_full(m_source);
1120     case(this.m_stage)
1121         IF : this.m_stage = ID;
1122         ID : this.m_stage = EX;
1123         EX : this.m_stage = WB;
1124         WB : this.m_stage = WB_2;
1125         APU: this.m_stage = APU;
1126     endcase
1127 endfunction
1128
1129 function void set_to_apu();
1130     this.m_stage = APU;
1131 endfunction
1132 endclass

```

PicoRV32 -- 초소형 RISC-V

What this is. A very small RISC-V core, widely used in FPGA designs.

Why it exists. Because it shows how little is needed for a working processor. It trades cycles per instruction for area and is the clearest introduction to RISC-V RTL.

Undergraduate review · EECS concepts. Multi-cycle versus pipelined execution; area–performance trade-offs; simple bus interfaces.

Graduate · Practice note. A good first core to read end to end. Understanding one complete core makes reading a complex one far easier.

13 files • 5,076 lines

API — picorv32.v

Table 1709. YosysHQ_picorv32/picorv32.v — module / interface / package (8개)

이름	종류
picorv32	module
picorv32_regs	module
picorv32_pcpi_mul	module
picorv32_pcpi_fast_mul	module
picorv32_pcpi_div	module
picorv32_axi	module
picorv32_axi_adapter	module
picorv32_wb	module

Table 1710. YosysHQ_picorv32/picorv32.v — parameter (81개) — 앞 60개만 표시(전체 81개, 본문 소스 참조)

파라미터	형/폭	기본값
ENABLE_COUNTERS	[0:0]	1
ENABLE_COUNTERS64	[0:0]	1
ENABLE_REGS_16_31	[0:0]	1
ENABLE_REGS_DUALPORT	[0:0]	1
LATCHED_MEM_RDATA	[0:0]	0
TWO_STAGE_SHIFT	[0:0]	1
BARREL_SHIFTER	[0:0]	0
TWO_CYCLE_COMPARE	[0:0]	0
TWO_CYCLE_ALU	[0:0]	0
COMPRESSED_ISA	[0:0]	0

파라미터	형/폭	기본값
CATCH_MISALIGN	[0:0]	1
CATCH_ILLINSN	[0:0]	1
ENABLE_PCPI	[0:0]	0
ENABLE_MUL	[0:0]	0
ENABLE_FAST_MUL	[0:0]	0
ENABLE_DIV	[0:0]	0
ENABLE_IRQ	[0:0]	0
ENABLE_IRQ_QREGS	[0:0]	1
ENABLE_IRQ_TIMER	[0:0]	1
ENABLE_TRACE	[0:0]	0
REGS_INIT_ZERO	[0:0]	0
MASKED_IRQ	[31:0]	32'h 0000_0000
LATCHED_IRQ	[31:0]	32'h ffff_ffff
PROGADDR_RESET	[31:0]	32'h 0000_0000
PROGADDR_IRQ	[31:0]	32'h 0000_0010
STACKADDR	[31:0]	32'h ffff_ffff
STEPS_AT_ONCE		1
CARRY_CHAIN		4
EXTRA_MUL_FFS		0
EXTRA_INSN_FFS		0
MUL_CLKGATE		0
ENABLE_COUNTERS	[0:0]	1
ENABLE_COUNTERS64	[0:0]	1
ENABLE_REGS_16_31	[0:0]	1
ENABLE_REGS_DUALPORT	[0:0]	1
TWO_STAGE_SHIFT	[0:0]	1
BARREL_SHIFTER	[0:0]	0
TWO_CYCLE_COMPARE	[0:0]	0
TWO_CYCLE_ALU	[0:0]	0
COMPRESSED_ISA	[0:0]	0
CATCH_MISALIGN	[0:0]	1
CATCH_ILLINSN	[0:0]	1
ENABLE_PCPI	[0:0]	0
ENABLE_MUL	[0:0]	0
ENABLE_FAST_MUL	[0:0]	0

파라미터	형/폭	기본값
ENABLE_DIV	[0:0]	0
ENABLE_IRQ	[0:0]	0
ENABLE_IRQ_QREGS	[0:0]	1
ENABLE_IRQ_TIMER	[0:0]	1
ENABLE_TRACE	[0:0]	0
REGS_INIT_ZERO	[0:0]	0
MASKED_IRQ	[31:0]	32'h 0000_0000
LATCHED_IRQ	[31:0]	32'h ffff_ffff
PROGADDR_RESET	[31:0]	32'h 0000_0000
PROGADDR_IRQ	[31:0]	32'h 0000_0010
STACKADDR	[31:0]	32'h ffff_ffff
ENABLE_COUNTERS	[0:0]	1
ENABLE_COUNTERS64	[0:0]	1
ENABLE_REGS_16_31	[0:0]	1
ENABLE_REGS_DUALPORT	[0:0]	1

Table 1711. YosysHQ_picorv32/picorv32.v — localparam (21개)

상수	형/폭	값
irq_timer	integer	0
irq_ebreak	integer	1
irq_buserror	integer	2
irqregs_offset	integer	ENABLE_REGS_16_31 ? 32 : 16
regfile_size	integer	(ENABLE_REGS_16_31 ? 32 : 16) + 4*ENABLE_IRQ*ENABLE_IRQ_QREG...
regindex_bits	integer	(ENABLE_REGS_16_31 ? 5 : 4) + ENABLE_IRQ*ENABLE_IRQ_QREGS
WITH_PCPI		ENABLE_PCPI ENABLE_MUL ENABLE_FAST_MUL ENABLE_DIV
TRACE_BRANCH	[35:0]	{4'b 0001
TRACE_ADDR	[35:0]	{4'b 0010
TRACE_IRQ	[35:0]	{4'b 1000
cpu_state_trap		8'b10000000
cpu_state_fetch		8'b01000000
cpu_state_ld_rs1		8'b00100000
cpu_state_ld_rs2		8'b00010000
cpu_state_exec		8'b00001000
cpu_state_shift		8'b00000100
cpu_state_stmem		8'b00000010

상수	형/폭	값
cpu_state_ldmem		8'b00000001
IDLE		2'b00
WBSTART		2'b01
WBEND		2'b10

Table 1712. YosysHQ_picorv32/picorv32.v — 포트 (207개) — 앞 60개만 표시(전체 207개, 본문 소스 참조)

포트	방향	형/폭
clk	input	
trap	output	reg
mem_valid	output	reg
mem_instr	output	reg
mem_ready	input	
mem_addr	output	reg [31:0]
mem_wdata	output	reg [31:0]
mem_wstrb	output	reg [3:0]
mem_rdata	input	[31:0]
mem_la_read	output	
mem_la_write	output	
mem_la_addr	output	[31:0]
mem_la_wdata	output	reg [31:0]
mem_la_wstrb	output	reg [3:0]
pcpi_valid	output	reg
pcpi_insn	output	reg [31:0]
pcpi_rs1	output	[31:0]
pcpi_rs2	output	[31:0]
pcpi_wr	input	
pcpi_rd	input	[31:0]
pcpi_wait	input	
pcpi_ready	input	
irq	input	[31:0]
eoi	output	reg [31:0]
rvfi_valid	output	reg
rvfi_order	output	reg [63:0]
rvfi_insn	output	reg [31:0]
rvfi_trap	output	reg

포트	방향	형/폭
rvfi_halt	output	reg
rvfi_intr	output	reg
rvfi_mode	output	reg [1:0]
rvfi_ixl	output	reg [1:0]
rvfi_rs1_addr	output	reg [4:0]
rvfi_rs2_addr	output	reg [4:0]
rvfi_rs1_rdata	output	reg [31:0]
rvfi_rs2_rdata	output	reg [31:0]
rvfi_rd_addr	output	reg [4:0]
rvfi_rd_wdata	output	reg [31:0]
rvfi_pc_rdata	output	reg [31:0]
rvfi_pc_wdata	output	reg [31:0]
rvfi_mem_addr	output	reg [31:0]
rvfi_mem_rmask	output	reg [3:0]
rvfi_mem_wmask	output	reg [3:0]
rvfi_mem_rdata	output	reg [31:0]
rvfi_mem_wdata	output	reg [31:0]
rvfi_csr_mcycle_rmask	output	reg [63:0]
rvfi_csr_mcycle_wmask	output	reg [63:0]
rvfi_csr_mcycle_rdata	output	reg [63:0]
rvfi_csr_mcycle_wdata	output	reg [63:0]
rvfi_csr_minstret_rmask	output	reg [63:0]
rvfi_csr_minstret_wmask	output	reg [63:0]
rvfi_csr_minstret_rdata	output	reg [63:0]
rvfi_csr_minstret_wdata	output	reg [63:0]
trace_valid	output	reg
trace_data	output	reg [35:0]
clk	input	
waddr	input	[5:0]
raddr1	input	[5:0]
raddr2	input	[5:0]
wdata	input	[31:0]

Table 1713. YosysHQ_picorv32/picorv32.v — #define / `define 매크로 (8개)

매크로	인자	정의
debug	(debug_command)	debug_command
debug	(debug_command)	`endif
FORMAL_KEEP		(* keep *)
assert	(assert_expr)	assert(assert_expr)
FORMAL_KEEP		(* keep *)
FORMAL_KEEP		`endif
assert	(assert_expr)	empty_statement
PICORV32_V		/*****

API — testbench.v

Table 1714. YosysHQ_picorv32/testbench.v — module / interface / package (3개)

이름	종류
testbench	module
picorv32_wrapper	module
axi4_memory	module

Table 1715. YosysHQ_picorv32/testbench.v — parameter (6개)

파라미터	형/폭	기본값
AXI_TEST		0
VERBOSE		0
AXI_TEST		0
VERBOSE		0
AXI_TEST		0
VERBOSE		0

Table 1716. YosysHQ_picorv32/testbench.v — 포트 (24개)

포트	방향	형/폭
clk	input	
resetsn	input	
trap	output	
trace_valid	output	
trace_data	output	[35:0]
clk	input	
mem_axi_awvalid	input	
mem_axi_awready	output	reg
mem_axi_awaddr	input	[31:0]

포트	방향	형/폭
mem_axi_awprot	input	[2:0]
mem_axi_wvalid	input	
mem_axi_wready	output	reg
mem_axi_wdata	input	[31:0]
mem_axi_wstrb	input	[3:0]
mem_axi_bvalid	output	reg
mem_axi_bready	input	
mem_axi_arvalid	input	
mem_axi_arready	output	reg
mem_axi_araddr	input	[31:0]
mem_axi_arprot	input	[2:0]
mem_axi_rvalid	output	reg
mem_axi_rready	input	
mem_axi_rdata	output	reg [31:0]
tests_passed	output	reg

API — testbench_ez.v

Table 1717. YosysHQ_picorv32/testbench_ez.v — module / interface / package (1개)

이름	종류
testbench	module

API — testbench_wb.v

Table 1718. YosysHQ_picorv32/testbench_wb.v — module / interface / package (3개)

이름	종류
testbench	module
picorv32_wrapper	module
wb_ram	module

Table 1719. YosysHQ_picorv32/testbench_wb.v — parameter (5개)

파라미터	형/폭	기본값
VERBOSE		0
VERBOSE		0
depth		256
memfile		""
VERBOSE		0

Table 1720. YosysHQ_picorv32/testbench_wb.v — 포트 (17개)

포트	방향	형/폭
wb_clk	input	
wb_rst	input	
trap	output	
trace_valid	output	
trace_data	output	[35:0]
wb_clk_i	input	
wb_rst_i	input	
wb_adr_i	input	[31:0]
wb_dat_i	input	[31:0]
wb_sel_i	input	[3:0]
wb_we_i	input	
wb_cyc_i	input	
wb_stb_i	input	
wb_ack_o	output	reg
wb_dat_o	output	reg [31:0]
mem_instr	input	
tests_passed	output	reg

API — axicheck.v

Table 1721. YosysHQ_picorv32/scripts/smtbmc/axicheck.v — module / interface / package (1개)

이름	종류
testbench	module

Table 1722. YosysHQ_picorv32/scripts/smtbmc/axicheck.v — 포트 (19개)

포트	방향	형/폭
clk	input	
trap	output	
mem_axi_awvalid	output	
mem_axi_awready	input	
mem_axi_awaddr	output	[31:0]
mem_axi_awprot	output	[2:0]
mem_axi_wvalid	output	
mem_axi_wready	input	
mem_axi_wdata	output	[31:0]
mem_axi_wstrb	output	[3:0]
mem_axi_bvalid	input	

포트	방향	형/폭
mem_axi_bready	output	
mem_axi_arvalid	output	
mem_axi_arready	input	
mem_axi_araddr	output	[31:0]
mem_axi_arprot	output	[2:0]
mem_axi_rvalid	input	
mem_axi_rready	output	
mem_axi_rdata	input	[31:0]

API — axicheck2.v

Table 1723. YosysHQ_picorv32/scripts/smtbmc/axicheck2.v — module / interface / package (1개)

이름	종류
testbench	module

Table 1724. YosysHQ_picorv32/scripts/smtbmc/axicheck2.v — 포트 (35개)

포트	방향	형/폭
clk	input	
resetsn	input	
trap_0	output	
trap_1	output	
mem_axi_awvalid_0	output	
mem_axi_awready_0	input	
mem_axi_awaddr_0	output	[31:0]
mem_axi_awprot_0	output	[2:0]
mem_axi_awvalid_1	output	
mem_axi_awready_1	input	
mem_axi_awaddr_1	output	[31:0]
mem_axi_awprot_1	output	[2:0]
mem_axi_wvalid_0	output	
mem_axi_wready_0	input	
mem_axi_wdata_0	output	[31:0]
mem_axi_wstrb_0	output	[3:0]
mem_axi_wvalid_1	output	
mem_axi_wready_1	input	
mem_axi_wdata_1	output	[31:0]
mem_axi_wstrb_1	output	[3:0]

포트	방향	형/폭
mem_axi_bvalid	input	
mem_axi_bready_0	output	
mem_axi_bready_1	output	
mem_axi_arvalid_0	output	
mem_axi_arready_0	input	
mem_axi_araddr_0	output	[31:0]
mem_axi_arprot_0	output	[2:0]
mem_axi_arvalid_1	output	
mem_axi_arready_1	input	
mem_axi_araddr_1	output	[31:0]
mem_axi_arprot_1	output	[2:0]
mem_axi_rvalid	input	
mem_axi_rready_0	output	
mem_axi_rready_1	output	
mem_axi_rdata	input	[31:0]

API — mulcmp.v

Table 1725. YosysHQ_picorv32/scripts/smtbmc/mulcmp.v — module / interface / package (1개)

이름	종류
testbench	module

API — notrap_validop.v

Table 1726. YosysHQ_picorv32/scripts/smtbmc/notrap_validop.v — module / interface / package (1개)

이름	종류
testbench	module

API — opcode.v

Table 1727. YosysHQ_picorv32/scripts/smtbmc/opcode.v — 포트 (8개)

포트	방향	형/폭
opcode	input	[31:0]
opcode	input	[31:0]
opcode	input	[31:0]
opcode	input	[31:0]
opcode	input	[31:0]
opcode	input	[31:0]
opcode	input	[31:0]

포트	방향	형/폭
opcode	input	[31:0]

API — tracecmp.v

Table 1728. YosysHQ_picorv32/scripts/smtbmc/tracecmp.v — module / interface / package (1개)

이름	종류
testbench	module

Table 1729. YosysHQ_picorv32/scripts/smtbmc/tracecmp.v — localparam (1개)

상수	형/폭	값
ENABLE_COUNTERS		0

API — tracecmp2.v

Table 1730. YosysHQ_picorv32/scripts/smtbmc/tracecmp2.v — module / interface / package (1개)

이름	종류
testbench	module

Table 1731. YosysHQ_picorv32/scripts/smtbmc/tracecmp2.v — localparam (1개)

상수	형/폭	값
ENABLE_COUNTERS		0

Table 1732. YosysHQ_picorv32/scripts/smtbmc/tracecmp2.v — 포트 (2개)

포트	방향	형/폭
clk	input	
mem_rdata	input	[31:0]

API — tracecmp3.v

Table 1733. YosysHQ_picorv32/scripts/smtbmc/tracecmp3.v — module / interface / package (1개)

이름	종류
testbench	module

Table 1734. YosysHQ_picorv32/scripts/smtbmc/tracecmp3.v — 포트 (6개)

포트	방향	형/폭
clk	input	
mem_rdata_in	input	[31:0]
pcpi_wr	input	
pcpi_rd	input	[31:0]
pcpi_wait	input	
pcpi_ready	input	

API — testbench.v

Table 1735. YosysHQ_picorv32/scripts/cxxdemo/testbench.v — module / interface / package (17개)

이름	종류
testbench	module

Table 1736. YosysHQ_picorv32/scripts/cxxdemo/testbench.v — localparam (1개)

상수	형/폭	값
MEM_SIZE		4*1024*1024

1. YosysHQ_picorv32/picorv32.v (3,049 lines)

```

1 /*
2  * PicoRV32 -- A Small RISC-V (RV32I) Processor Core
3  *
4  * Copyright (C) 2015 Claire Xenia Wolf <claire@yosyshq.com>
5  *
6  * Permission to use, copy, modify, and/or distribute this
7  * software for any purpose with or without fee is hereby granted, provided
8  * that the above copyright notice and this permission notice appear in all
9  * copies.
10 *
11 * THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS
12 * ALL WARRANTIES WITH REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WAR-
13 * RANTIES OF MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL THE AUTHOR
14 * BE LIABLE FOR ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL DAMAGES OR
15 * ANY DAMAGES WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS,
16 * WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION,
17 * ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFT-
18 * WARE.
19 */
20 /* verilator lint_off WIDTH */
21 /* verilator lint_off PINMISSING */
22 /* verilator lint_off CASEOVERLAP */
23 /* verilator lint_off CASEINCOMPLETE */
24
25 `timescale 1 ns / 1 ps
26 // `default_nettype none
27 // `define DEBUGNETS
28 // `define DEBUGREGS
29 // `define DEBUGASM
30 // `define DEBUG
31
32 `ifdef DEBUG
33     `define debug(debug_command) debug_command
34 `else
35     `define debug(debug_command)
36 `endif
37
38 `ifdef FORMAL
39     `define FORMAL_KEEP (* keep *)
40     `define assert(assert_expr) assert(assert_expr)
41 `else
42     `ifdef DEBUGNETS
43         `define FORMAL_KEEP (* keep *)
44     `else
45         `define FORMAL_KEEP
46     `endif
47     `define assert(assert_expr) empty_statement
48 `endif
49
50 // uncomment this for register file in extra module
51 // `define PICORV32_REGS picorv32_regs
52
53 // this macro can be used to check if the verilog files in
54 // your design are read in the correct order.
55 `define PICORV32_V
56
57 /
58 /
59 * picorv32
60
61 *****
62 module picorv32 #(

```

```

63     parameter [0:0] ENABLE_COUNTERS = 1,
64     parameter [0:0] ENABLE_COUNTERS64 = 1,
65     parameter [0:0] ENABLE_REGS_16_31 = 1,
66     parameter [0:0] ENABLE_REGS_DUALPORT = 1,
67     parameter [0:0] LATCHED_MEM_RDATA = 0,
68     parameter [0:0] TWO_STAGE_SHIFT = 1,
69     parameter [0:0] BARREL_SHIFTER = 0,
70     parameter [0:0] TWO_CYCLE_COMPARE = 0,
71     parameter [0:0] TWO_CYCLE_ALU = 0,
72     parameter [0:0] COMPRESSED_ISA = 0,
73     parameter [0:0] CATCH_MISALIGN = 1,
74     parameter [0:0] CATCH_ILLINSN = 1,
75     parameter [0:0] ENABLE_PCPPI = 0,
76     parameter [0:0] ENABLE_MUL = 0,
77     parameter [0:0] ENABLE_FAST_MUL = 0,
78     parameter [0:0] ENABLE_DIV = 0,
79     parameter [0:0] ENABLE_IRQ = 0,
80     parameter [0:0] ENABLE_IRQ_QREGS = 1,
81     parameter [0:0] ENABLE_IRQ_TIMER = 1,
82     parameter [0:0] ENABLE_TRACE = 0,
83     parameter [0:0] REGS_INIT_ZERO = 0,
84     parameter [31:0] MASKED_IRQ = 32'h 0000_0000,
85     parameter [31:0] LATCHED_IRQ = 32'h ffff_ffff,
86     parameter [31:0] PROGADDR_RESET = 32'h 0000_0000,
87     parameter [31:0] PROGADDR_IRQ = 32'h 0000_0010,
88     parameter [31:0] STACKADDR = 32'h ffff_ffff
89 ) (
90     input clk, resetn,
91     output reg trap,
92
93     output reg mem_valid,
94     output reg mem_instr,
95     input mem_ready,
96
97     output reg [31:0] mem_addr,
98     output reg [31:0] mem_wdata,
99     output reg [3:0] mem_wstrb,
100    input [31:0] mem_rdata,
101
102    // Look-Ahead Interface
103    output mem_la_read,
104    output mem_la_write,
105    output [31:0] mem_la_addr,
106    output reg [31:0] mem_la_wdata,
107    output reg [3:0] mem_la_wstrb,
108
109    // Pico Co-Processor Interface (PCPI)
110    output reg pcpi_valid,
111    output reg [31:0] pcpi_insn,
112    output [31:0] pcpi_rs1,
113    output [31:0] pcpi_rs2,
114    input pcpi_wr,
115    input [31:0] pcpi_rd,
116    input pcpi_wait,
117    input pcpi_ready,
118
119    // IRQ Interface
120    input [31:0] irq,
121    output reg [31:0] eoi,
122
123    `ifdef RISC_V_FORMAL
124    output reg rvfi_valid,
125    output reg [63:0] rvfi_order,
126    output reg [31:0] rvfi_insn,
127    output reg rvfi_trap,
128    output reg rvfi_halt,
129    output reg rvfi_intr,
130    output reg [1:0] rvfi_mode,
131    output reg [1:0] rvfi_ixl,
132    output reg [4:0] rvfi_rs1_addr,
133    output reg [4:0] rvfi_rs2_addr,
134    output reg [31:0] rvfi_rs1_rdata,
135    output reg [31:0] rvfi_rs2_rdata,
136    output reg [4:0] rvfi_rd_addr,
137    output reg [31:0] rvfi_rd_wdata,
138    output reg [31:0] rvfi_pc_rdata,
139    output reg [31:0] rvfi_pc_wdata,
140    output reg [31:0] rvfi_mem_addr,
141    output reg [3:0] rvfi_mem_rmask,

```

```

142     output reg [3:0] rvfi_mem_wmask,
143     output reg [31:0] rvfi_mem_rdata,
144     output reg [31:0] rvfi_mem_wdata,
145
146     output reg [63:0] rvfi_csr_mcycle_rmask,
147     output reg [63:0] rvfi_csr_mcycle_wmask,
148     output reg [63:0] rvfi_csr_mcycle_rdata,
149     output reg [63:0] rvfi_csr_mcycle_wdata,
150
151     output reg [63:0] rvfi_csr_minstret_rmask,
152     output reg [63:0] rvfi_csr_minstret_wmask,
153     output reg [63:0] rvfi_csr_minstret_rdata,
154     output reg [63:0] rvfi_csr_minstret_wdata,
155 `endif
156
157     // Trace Interface
158     output reg trace_valid,
159     output reg [35:0] trace_data
160 );
161     localparam integer irq_timer = 0;
162     localparam integer irq_ebreak = 1;
163     localparam integer irq_buserror = 2;
164
165     localparam integer irqregs_offset = ENABLE_REGS_16_31 ?
32 : 16;
166     localparam integer regfile_size = (ENABLE_REGS_16_31 ?
32 : 16) + 4*ENABLE_IRQ*ENABLE_IRQ_QREGS;
167     localparam integer regindex_bits = (ENABLE_REGS_16_31 ?
5 : 4) + ENABLE_IRQ*ENABLE_IRQ_QREGS;
168
169     localparam WITH_PCPPI = ENABLE_PCPPI || ENABLE_MUL || EN-
ABLE_FAST_MUL || ENABLE_DIV;
170
171     localparam [35:0] TRACE_BRANCH = {4'b 0001, 32'b 0};
172     localparam [35:0] TRACE_ADDR = {4'b 0010, 32'b 0};
173     localparam [35:0] TRACE_IRQ = {4'b 1000, 32'b 0};
174
175     reg [63:0] count_cycle, count_instr;
176     reg [31:0] reg_pc, reg_next_pc, reg_op1, reg_op2,
reg_out;
177     reg [4:0] reg_sh;
178
179     reg [31:0] next_insn_opcode;
180     reg [31:0] dbg_insn_opcode;
181     reg [31:0] dbg_insn_addr;
182
183     wire dbg_mem_valid = mem_valid;
184     wire dbg_mem_instr = mem_instr;
185     wire dbg_mem_ready = mem_ready;
186     wire [31:0] dbg_mem_addr = mem_addr;
187     wire [31:0] dbg_mem_wdata = mem_wdata;
188     wire [3:0] dbg_mem_wstrb = mem_wstrb;
189     wire [31:0] dbg_mem_rdata = mem_rdata;
190
191     assign pcpi_rs1 = reg_op1;
192     assign pcpi_rs2 = reg_op2;
193
194     wire [31:0] next_pc;
195
196     reg irq_delay;
197     reg irq_active;
198     reg [31:0] irq_mask;
199     reg [31:0] irq_pending;
200     reg [31:0] timer;
201
202 `ifndef PICORV32_REGS
203     reg [31:0] cpuregs [0:regfile_size-1];
204
205     integer i;
206     initial begin
207         if (REGS_INIT_ZERO) begin
208             for (i = 0; i < regfile_size; i = i+1)
209                 cpuregs[i] = 0;
210         end
211     end
212 `endif
213
214     task empty_statement;
215         // This task is used by the `assert directive
in non-formal mode to
216         // avoid empty statement (which are unsupported
by plain Verilog syntax).
217         begin end
218     endtask
219
220 `ifndef DEBUGREGS
221     wire [31:0] dbg_reg_x0 = 0;
222     wire [31:0] dbg_reg_x1 = cpuregs[1];
223     wire [31:0] dbg_reg_x2 = cpuregs[2];
224     wire [31:0] dbg_reg_x3 = cpuregs[3];
225     wire [31:0] dbg_reg_x4 = cpuregs[4];
226     wire [31:0] dbg_reg_x5 = cpuregs[5];
227     wire [31:0] dbg_reg_x6 = cpuregs[6];
228     wire [31:0] dbg_reg_x7 = cpuregs[7];
229     wire [31:0] dbg_reg_x8 = cpuregs[8];
230     wire [31:0] dbg_reg_x9 = cpuregs[9];
231     wire [31:0] dbg_reg_x10 = cpuregs[10];
232     wire [31:0] dbg_reg_x11 = cpuregs[11];

```

```

233     wire [31:0] dbg_reg_x12 = cpuregs[12];
234     wire [31:0] dbg_reg_x13 = cpuregs[13];
235     wire [31:0] dbg_reg_x14 = cpuregs[14];
236     wire [31:0] dbg_reg_x15 = cpuregs[15];
237     wire [31:0] dbg_reg_x16 = cpuregs[16];
238     wire [31:0] dbg_reg_x17 = cpuregs[17];
239     wire [31:0] dbg_reg_x18 = cpuregs[18];
240     wire [31:0] dbg_reg_x19 = cpuregs[19];
241     wire [31:0] dbg_reg_x20 = cpuregs[20];
242     wire [31:0] dbg_reg_x21 = cpuregs[21];
243     wire [31:0] dbg_reg_x22 = cpuregs[22];
244     wire [31:0] dbg_reg_x23 = cpuregs[23];
245     wire [31:0] dbg_reg_x24 = cpuregs[24];
246     wire [31:0] dbg_reg_x25 = cpuregs[25];
247     wire [31:0] dbg_reg_x26 = cpuregs[26];
248     wire [31:0] dbg_reg_x27 = cpuregs[27];
249     wire [31:0] dbg_reg_x28 = cpuregs[28];
250     wire [31:0] dbg_reg_x29 = cpuregs[29];
251     wire [31:0] dbg_reg_x30 = cpuregs[30];
252     wire [31:0] dbg_reg_x31 = cpuregs[31];
253 `endif
254
255     // Internal PCPI Cores
256
257     wire pcpi_mul_wr;
258     wire [31:0] pcpi_mul_rd;
259     wire pcpi_mul_wait;
260     wire pcpi_mul_ready;
261
262     wire pcpi_div_wr;
263     wire [31:0] pcpi_div_rd;
264     wire pcpi_div_wait;
265     wire pcpi_div_ready;
266
267     reg pcpi_int_wr;
268     reg [31:0] pcpi_int_rd;
269     reg pcpi_int_wait;
270     reg pcpi_int_ready;
271
272     generate if (ENABLE_FAST_MUL) begin
273         picorv32_pcpi_fast_mul pcpi_mul (
274             .clk (clk),
275             .resetn (resetn),
276             .pcpi_valid (pcpi_valid),
277             .pcpi_insn (pcpi_insn),
278             .pcpi_rs1 (pcpi_rs1),
279             .pcpi_rs2 (pcpi_rs2),
280             .pcpi_wr (pcpi_mul_wr),
281             .pcpi_rd (pcpi_mul_rd),
282             .pcpi_wait (pcpi_mul_wait),
283             .pcpi_ready (pcpi_mul_ready)
284         );
285     end else if (ENABLE_MUL) begin
286         picorv32_pcpi_mul pcpi_mul (
287             .clk (clk),
288             .resetn (resetn),
289             .pcpi_valid (pcpi_valid),
290             .pcpi_insn (pcpi_insn),
291             .pcpi_rs1 (pcpi_rs1),
292             .pcpi_rs2 (pcpi_rs2),
293             .pcpi_wr (pcpi_mul_wr),
294             .pcpi_rd (pcpi_mul_rd),
295             .pcpi_wait (pcpi_mul_wait),
296             .pcpi_ready (pcpi_mul_ready)
297         );
298     end else begin
299         assign pcpi_mul_wr = 0;
300         assign pcpi_mul_rd = 32'bx;
301         assign pcpi_mul_wait = 0;
302         assign pcpi_mul_ready = 0;
303     end endgenerate
304
305     generate if (ENABLE_DIV) begin
306         picorv32_pcpi_div pcpi_div (
307             .clk (clk),
308             .resetn (resetn),
309             .pcpi_valid (pcpi_valid),
310             .pcpi_insn (pcpi_insn),
311             .pcpi_rs1 (pcpi_rs1),
312             .pcpi_rs2 (pcpi_rs2),
313             .pcpi_wr (pcpi_div_wr),
314             .pcpi_rd (pcpi_div_rd),
315             .pcpi_wait (pcpi_div_wait),
316             .pcpi_ready (pcpi_div_ready)
317         );
318     end else begin
319         assign pcpi_div_wr = 0;
320         assign pcpi_div_rd = 32'bx;
321         assign pcpi_div_wait = 0;
322         assign pcpi_div_ready = 0;
323     end endgenerate
324
325     always @* begin
326         pcpi_int_wr = 0;
327         pcpi_int_rd = 32'bx;
328         pcpi_int_wait = |(ENABLE_PCPPI && pcpi_wait,
(ENABLE_MUL || ENABLE_FAST_MUL) && pcpi_mul_wait, ENABLE_DIV &&
pcpi_div_wait);
329         pcpi_int_ready = |(ENABLE_PCPPI && pcpi_ready,
(ENABLE_MUL || ENABLE_FAST_MUL) && pcpi_mul_ready, ENABLE_DIV &&
pcpi_div_ready);

```



```

330
331      (* parallel_case *)
332      case (1'b1)
333          ENABLE_PCPI && pcpi_ready: begin
334              pcpi_int_wr = ENABLE_PCPI ?
pcpi_wr : 0;
335              pcpi_int_rd = ENABLE_PCPI ?
pcpi_rd : 0;
336          end
337          (ENABLE_MUL || ENABLE_FAST_MUL) &&
pcpi_mul_ready: begin
338              pcpi_int_wr = pcpi_mul_wr;
339              pcpi_int_rd = pcpi_mul_rd;
340          end
341          ENABLE_DIV && pcpi_div_ready: begin
342              pcpi_int_wr = pcpi_div_wr;
343              pcpi_int_rd = pcpi_div_rd;
344          end
345      endcase
346  end
347
348  // Memory Interface
349
350  reg [1:0] mem_state;
351  reg [1:0] mem_wordsize;
352  reg [31:0] mem_rdata_word;
353  reg [31:0] mem_rdata_q;
354  reg mem_do_prefetch;
355  reg mem_do_rinst;
356  reg mem_do_rdata;
357  reg mem_do_wdata;
358
359  wire mem_xfer;
360  reg mem_la_secondword, mem_la_firstword_reg,
last_mem_valid;
361  wire mem_la_firstword = COMPRESSED_ISA &&
(mem_do_prefetch || mem_do_rinst) && next_pc[1] && !mem_la_second-
word;
362
363  wire mem_la_firstword_xfer = COMPRESSED_ISA && mem_xfer
&& (!last_mem_valid ? mem_la_firstword : mem_la_firstword_reg);
364
365  reg prefetched_high_word;
366  reg clear_prefetched_high_word;
367  reg [15:0] mem_16bit_buffer;
368
369  wire [31:0] mem_rdata_latched_noshuffle;
370  wire [31:0] mem_rdata_latched;
371
372  wire mem_la_use_prefetched_high_word = COMPRESSED_ISA
&& mem_la_firstword && prefetched_high_word && !
clear_prefetched_high_word;
373  assign mem_xfer = (mem_valid && mem_ready) ||
(mem_la_use_prefetched_high_word && mem_do_rinst);
374
375  wire mem_busy = !(mem_do_prefetch, mem_do_rinst,
mem_do_rdata, mem_do_wdata);
376  wire mem_done = resetn && ((mem_xfer && !mem_state &&
(mem_do_rinst || mem_do_rdata || mem_do_wdata)) || (&mem_state &&
mem_do_rinst)) &&
377  (!mem_la_firstword ||
(~&mem_rdata_latched[1:0] && mem_xfer));
378
379  assign mem_la_write = resetn && !mem_state &&
mem_do_wdata;
380  assign mem_la_read = resetn && (!
mem_la_use_prefetched_high_word && !mem_state && (mem_do_rinst ||
mem_do_prefetch || mem_do_rdata)) ||
381  (COMPRESSED_ISA && mem_xfer && (!
last_mem_valid ? mem_la_firstword : mem_la_firstword_reg) && !
mem_la_secondword && &mem_rdata_latched[1:0]));
382  assign mem_la_addr = (mem_do_prefetch ||
mem_do_rinst) ? {next_pc[31:2] + mem_la_firstword_xfer, 2'b00} :
{reg_op1[31:2], 2'b00};
383
384  assign mem_rdata_latched_noshuffle = (mem_xfer ||
LATCHED_MEM_RDATA) ? mem_rdata : mem_rdata_q;
385
386  assign mem_rdata_latched = COMPRESSED_ISA &&
mem_la_use_prefetched_high_word ? {16'bx, mem_16bit_buffer} :
387  COMPRESSED_ISA && mem_la_secondword ?
{mem_rdata_latched_noshuffle[15:0], mem_16bit_buffer} :
388  COMPRESSED_ISA && mem_la_firstword ?
{16'bx, mem_rdata_latched_noshuffle[31:16]} :
mem_rdata_latched_noshuffle;
389
390  always @(posedge clk) begin
391      if (!resetn) begin
392          mem_la_firstword_reg <= 0;
393          last_mem_valid <= 0;
394      end else begin
395          if (!last_mem_valid)
396              mem_la_firstword_reg <=
mem_la_firstword;
397          last_mem_valid <= mem_valid && !
mem_ready;
398      end
399  end
400
401  always @* begin

```

```

402      (* full_case *)
403      case (mem_wordsize)
404          0: begin
405              mem_la_wdata = reg_op2;
406              mem_la_wstrb = 4'b1111;
407              mem_rdata_word = mem_rdata;
408          end
409          1: begin
410              mem_la_wdata =
{2{reg_op2[15:0]}};
411              mem_la_wstrb = reg_op1[1] ?
4'b1100 : 4'b0011;
412              case (reg_op1[1])
413                  1'b0: mem_rdata_word =
{16'b0, mem_rdata[15: 0]};
414                  1'b1: mem_rdata_word =
{16'b0, mem_rdata[31:16]};
415              endcase
416          end
417          2: begin
418              mem_la_wdata =
{4{reg_op2[7:0]}};
419              mem_la_wstrb = 4'b0001 <<
reg_op1[1:0];
420              case (reg_op1[1:0])
421                  2'b00: mem_rdata_word =
{24'b0, mem_rdata[ 7: 0]};
422                  2'b01: mem_rdata_word =
{24'b0, mem_rdata[15: 8]};
423                  2'b10: mem_rdata_word =
{24'b0, mem_rdata[23:16]};
424                  2'b11: mem_rdata_word =
{24'b0, mem_rdata[31:24]};
425              endcase
426          end
427      endcase
428  end
429
430  always @(posedge clk) begin
431      if (mem_xfer) begin
432          mem_rdata_q <= COMPRESSED_ISA ?
mem_rdata_latched : mem_rdata;
433          next_insn_opcode <= COMPRESSED_ISA ?
mem_rdata_latched : mem_rdata;
434      end
435
436      if (COMPRESSED_ISA && mem_done &&
(mem_do_prefetch || mem_do_rinst)) begin
437          case (mem_rdata_latched[1:0])
438              2'b00: begin // Quadrant 0
439                  case
440                      3'b000: be-
gin // C.ADDI4SPN
441                      mem_rdata_q[14:12] <= 3'b000;
442                      mem_rdata_q[31:20] <= {2'b0, mem_rdata_latched[10:7],
mem_rdata_latched[12:11], mem_rdata_latched[5],
mem_rdata_latched[6], 2'b00};
443                  end
444                      3'b010: be-
gin // C.LW
445                      mem_rdata_q[31:20] <= {5'b0, mem_rdata_latched[5],
mem_rdata_latched[12:10], mem_rdata_latched[6], 2'b00};
446                      mem_rdata_q[14:12] <= 3'b 010;
447                  end
448                      3'b 110: be-
gin // C.SW
449                      {mem_rdata_q[31:25], mem_rdata_q[11:7]} <= {5'b0,
mem_rdata_latched[5], mem_rdata_latched[12:10],
mem_rdata_latched[6], 2'b00};
450                      mem_rdata_q[14:12] <= 3'b 010;
451                  end
452              endcase
453          end
454          2'b01: begin // Quadrant 1
455              case
456                  3'b 000: be-
gin // C.ADDI
457                  mem_rdata_q[14:12] <= 3'b000;
458                  mem_rdata_q[31:20] <= $signed({mem_rdata_latched[12],
mem_rdata_latched[6:2]});
459              end
460                  3'b 010: be-
gin // C.LI
461                  mem_rdata_q[14:12] <= 3'b000;
462                  mem_rdata_q[31:20] <= $signed({mem_rdata_latched[12],

```

```

mem_rdata_latched[6:2]);
463                                     end
464 3'b 011: begin
465     if
(mem_rdata_latched[11:7] == 2) begin // C.ADDI16SP
466
mem_rdata_q[14:12] <= 3'b000;
467
mem_rdata_q[31:20] <= $signed({mem_rdata_latched[12],
mem_rdata_latched[4:3]},
468
mem_rdata_latched[5], mem_rdata_latched[2], mem_rdata_latched[6],
4'b 0000});
469                                     end
else begin // C.LUI
470
mem_rdata_q[31:12] <= $signed({mem_rdata_latched[12],
mem_rdata_latched[6:2]});
471                                     end
472                                     end
473 3'b100: begin
474     if
(mem_rdata_latched[11:10] == 2'b00) begin // C.SRLI
475
mem_rdata_q[31:25] <= 7'b00000000;
476
mem_rdata_q[14:12] <= 3'b 101;
477                                     end
478     if
(mem_rdata_latched[11:10] == 2'b01) begin // C.SRAI
479
mem_rdata_q[31:25] <= 7'b01000000;
480
mem_rdata_q[14:12] <= 3'b 101;
481                                     end
482     if
(mem_rdata_latched[11:10] == 2'b10) begin // C.ANDI
483
mem_rdata_q[14:12] <= 3'b111;
484
mem_rdata_q[31:20] <= $signed({mem_rdata_latched[12],
mem_rdata_latched[6:2]});
485                                     end
486     if
(mem_rdata_latched[12:10] == 3'b011) begin // C.SUB, C.XOR, C.OR,
C.AND
487
if (mem_rdata_latched[6:5] == 2'b00) mem_rdata_q[14:12] <= 3'b000;
488
if (mem_rdata_latched[6:5] == 2'b01) mem_rdata_q[14:12] <= 3'b100;
489
if (mem_rdata_latched[6:5] == 2'b10) mem_rdata_q[14:12] <= 3'b110;
490
if (mem_rdata_latched[6:5] == 2'b11) mem_rdata_q[14:12] <= 3'b111;
491
mem_rdata_q[31:25] <= mem_rdata_latched[6:5] == 2'b00 ? 7'b01000000 :
7'b00000000;
492                                     end
493                                     end
494 3'b 110: be-
gin // C.BEQZ
495
mem_rdata_q[14:12] <= 3'b000;
496
{ mem_rdata_q[31], mem_rdata_q[7], mem_rdata_q[30:25],
mem_rdata_q[11:8] } <=
497
$signed({mem_rdata_latched[12], mem_rdata_latched[6:5],
mem_rdata_latched[2],
498
mem_rdata_latched[11:10], mem_rdata_latched[4:3]});
499                                     end
500 3'b 111: be-
gin // C.BNEZ
501
mem_rdata_q[14:12] <= 3'b001;
502
{ mem_rdata_q[31], mem_rdata_q[7], mem_rdata_q[30:25],
mem_rdata_q[11:8] } <=
503
$signed({mem_rdata_latched[12], mem_rdata_latched[6:5],
mem_rdata_latched[2],

```

```

504
mem_rdata_latched[11:10], mem_rdata_latched[4:3]});
505                                     end
506                                     endcase
507                                     end
508 2'b10: begin // Quadrant 2
509     case
(mem_rdata_latched[15:13])
510                                     3'b000: be-
gin // C.SLLI
511
mem_rdata_q[31:25] <= 7'b00000000;
512
mem_rdata_q[14:12] <= 3'b 001;
513                                     end
514 3'b010: be-
gin // C.LWSP
515
mem_rdata_q[31:20] <= {4'b0, mem_rdata_latched[3:2],
mem_rdata_latched[12], mem_rdata_latched[6:4], 2'b00};
516
mem_rdata_q[14:12] <= 3'b 010;
517                                     end
518 3'b100: begin
519     if
(mem_rdata_latched[12] == 0 && mem_rdata_latched[6:2] == 0) begin //
C.JR
520
mem_rdata_q[14:12] <= 3'b000;
521
mem_rdata_q[31:20] <= 12'b0;
522                                     end
523     if
(mem_rdata_latched[12] == 0 && mem_rdata_latched[6:2] != 0) begin //
C.MV
524
mem_rdata_q[14:12] <= 3'b000;
525
mem_rdata_q[31:25] <= 7'b00000000;
526                                     end
527     if
(mem_rdata_latched[12] != 0 && mem_rdata_latched[11:7] != 0 &&
mem_rdata_latched[6:2] == 0) begin // C.JALR
528
mem_rdata_q[14:12] <= 3'b000;
529
mem_rdata_q[31:20] <= 12'b0;
530                                     end
531     if
(mem_rdata_latched[12] != 0 && mem_rdata_latched[6:2] != 0) begin //
C.ADD
532
mem_rdata_q[14:12] <= 3'b000;
533
mem_rdata_q[31:25] <= 7'b00000000;
534                                     end
535                                     end
536 3'b110: be-
gin // C.SWSP
537
{mem_rdata_q[31:25], mem_rdata_q[11:7]} <= {4'b0,
mem_rdata_latched[8:7], mem_rdata_latched[12:9], 2'b00};
538
mem_rdata_q[14:12] <= 3'b 010;
539                                     end
540                                     endcase
541                                     end
542     endcase
543     end
544     end
545
always @(posedge clk) begin
546     if (resetn && !trap) begin
547         if (mem_do_prefetch || mem_do_rinst ||
mem_do_rdata)
548             `assert(!mem_do_wdata);
549
550         if (mem_do_prefetch || mem_do_rinst)
551             `assert(!mem_do_rdata);
552
553         if (mem_do_rdata)
554             `assert(!mem_do_prefetch && !
mem_do_rinst);
555
556         if (mem_do_wdata)
557             `assert(!(mem_do_prefetch ||
mem_do_rinst || mem_do_rdata));
558
559         if (mem_state == 2 || mem_state == 3)
560             `assert(mem_valid ||
mem_do_prefetch);
561
562     end
563     end

```

```

564
565     always @(posedge clk) begin
566         if (!resetn || trap) begin
567             if (!resetn)
568                 mem_state <= 0;
569             if (!resetn || mem_ready)
570                 mem_valid <= 0;
571             mem_la_secondword <= 0;
572             prefetched_high_word <= 0;
573         end else begin
574             if (mem_la_read || mem_la_write) begin
575                 mem_addr <= mem_la_addr;
576                 mem_wstrb <= mem_la_wstrb &
577                 {4{mem_la_write}};
578             end
579             if (mem_la_write) begin
580                 mem_wdata <= mem_la_wdata;
581             end
582             case (mem_state)
583                 0: begin
584                     if (mem_do_prefetch ||
585                     mem_do_rinst || mem_do_rdata) begin
586                         mem_valid <= !
587                         mem_la_use_prefetched_high_word;
588                         mem_instr <=
589                         mem_do_prefetch || mem_do_rinst;
590                         mem_wstrb <= 0;
591                         mem_state <= 1;
592                     end
593                     if (mem_do_wdata) begin
594                         mem_valid <= 1;
595                         mem_instr <= 0;
596                         mem_state <= 2;
597                     end
598                     end
599                     1: begin
600                         `assert(mem_wstrb ==
601                         0);
602                         `assert(mem_do_prefetch
603                         || mem_do_rinst || mem_do_rdata);
604                         `assert(mem_valid == !
605                         mem_la_use_prefetched_high_word);
606                         `assert(mem_instr ==
607                         (mem_do_prefetch || mem_do_rinst));
608                         if (mem_xfer) begin
609                             if (COM-
610                             PRESSED_ISA && mem_la_read) begin
611                                 mem_valid <= 1;
612                                 mem_la_secondword <= 1;
613                                 mem_la_use_prefetched_high_word)
614                                     if (!
615                                     0);
616                                 mem_16bit_buffer <= mem_rdata[31:16];
617                                 end else begin
618                                     mem_valid <= 0;
619                                     mem_la_secondword <= 0;
620                                     if
621                                     (COMPRESSED_ISA && !mem_do_rdata) begin
622                                         if (~&mem_rdata[1:0] || mem_la_secondword) begin
623                                             mem_16bit_buffer <= mem_rdata[31:16];
624                                             prefetched_high_word <= 1;
625                                         end else begin
626                                             prefetched_high_word <= 0;
627                                         end
628                                     end
629                                     mem_state <= mem_do_rinst || mem_do_rdata ? 0 : 3;
630                                     end
631                                     end
632                                     2: begin
633                                         `assert(mem_wstrb !=
634                                         0);
635                                         `assert(mem_do_wdata);
636                                         if (mem_xfer) begin
637                                             mem_valid <= 0;
638                                             mem_state <= 0;
639                                         end
640                                     end
641                                     3: begin
642                                         `assert(mem_wstrb ==
643                                         0);
644                                         `as-
645                                         sert(mem_do_prefetch);
646                                         if (mem_do_rinst) begin
647                                             mem_state <= 0;
648                                         end
649                                     end
650                                     endcase
651                                     if (clear_prefetched_high_word)
652                                         prefetched_high_word <= 0;
653                                     end
654                                     // Instruction Decoder
655                                     reg instr_lui, instr_auiipc, instr_jal, instr_jalr;
656                                     reg instr_beq, instr_bne, instr_blt, instr_bge, in-
657                                     str_bltu, instr_bgeu;
658                                     reg instr_lb, instr_lh, instr_lw, instr_lbu, instr_lhu,
659                                     instr_sb, instr_sh, instr_sw;
660                                     reg instr_addi, instr_slti, instr_sltiu, instr_xori,
661                                     instr_ori, instr_andi, instr_slli, instr_srli, instr_srai;
662                                     reg instr_add, instr_sub, instr_sll, instr_slt, in-
663                                     str_sltu, instr_xor, instr_srl, instr_sra, instr_or, instr_and;
664                                     reg instr_rdcycle, instr_rdcycleh, instr_rdinstr, in-
665                                     str_rdinstrh, instr_ecall_ebreak, instr_fence;
666                                     reg instr_getq, instr_setq, instr_retirq, in-
667                                     str_maskirq, instr_waitirq, instr_timer;
668                                     wire instr_trap;
669                                     reg [regindex_bits-1:0] decoded_rd, decoded_rs1;
670                                     reg [4:0] decoded_rs2;
671                                     reg [31:0] decoded_imm, decoded_imm_j;
672                                     reg decoder_trigger;
673                                     reg decoder_trigger_q;
674                                     reg decoder_pseudo_trigger;
675                                     reg decoder_pseudo_trigger_q;
676                                     reg compressed_instr;
677                                     reg is_lui_auiipc_jal;
678                                     reg is_lb_lh_lw_lbu_lhu;
679                                     reg is_slli_srli_srai;
680                                     reg is_jalr_addi_slti_sltiu_xori_ori_andi;
681                                     reg is_sb_sh_sw;
682                                     reg is_sll_srl_sra;
683                                     reg is_lui_auiipc_jal_jalr_addi_add_sub;
684                                     reg is_slti_blt_slt;
685                                     reg is_sltiu_bltu_sltu;
686                                     reg is_beq_bne_blt_bge_bltu_bgeu;
687                                     reg is_lbu_lhu_lw;
688                                     reg is_alu_reg_imm;
689                                     reg is_alu_reg_reg;
690                                     reg is_compare;
691                                     assign instr_trap = (CATCH_ILLINSN || WITH_PCPPI) && !
692                                     {instr_lui, instr_auiipc, instr_jal, instr_jalr,
693                                     instr_beq, instr_bne, instr_blt, in-
694                                     str_bge, instr_bltu, instr_bgeu,
695                                     instr_lb, instr_lh, instr_lw, in-
696                                     str_lbu, instr_lhu, instr_sb, instr_sh, instr_sw,
697                                     instr_addi, instr_slti, instr_sltiu,
698                                     instr_xori, instr_ori, instr_andi, instr_slli, instr_srli, in-
699                                     str_srai,
700                                     instr_add, instr_sub, instr_sll, in-
701                                     str_slt, instr_sltu, instr_xor, instr_srl, instr_sra, instr_or, in-
702                                     str_and,
703                                     instr_rdcycle, instr_rdcycleh, in-
704                                     str_rdinstr, instr_rdinstrh, instr_fence,
705                                     instr_getq, instr_setq, instr_retirq,
706                                     instr_maskirq, instr_waitirq, instr_timer};
707                                     wire is_rdcycle_rdcycleh_rdinstr_rdinstrh;
708                                     assign is_rdcycle_rdcycleh_rdinstr_rdinstrh = |{in-
709                                     str_rdcycle, instr_rdcycleh, instr_rdinstr, instr_rdinstrh};
710                                     reg [63:0] new_ascii_instr;
711                                     `FORMAL_KEEP reg [63:0] dbg_ascii_instr;
712                                     `FORMAL_KEEP reg [31:0] dbg_insn_imm;
713                                     `FORMAL_KEEP reg [4:0] dbg_insn_rs1;
714                                     `FORMAL_KEEP reg [4:0] dbg_insn_rs2;
715                                     `FORMAL_KEEP reg [4:0] dbg_insn_rd;
716                                     `FORMAL_KEEP reg [31:0] dbg_rs1val;
717                                     `FORMAL_KEEP reg [31:0] dbg_rs2val;
718                                     `FORMAL_KEEP reg dbg_rs1val_valid;
719                                     `FORMAL_KEEP reg dbg_rs2val_valid;
720                                     always @* begin
721                                         new_ascii_instr = "";
722                                         if (instr_lui)
723                                             new_ascii_instr = "lui";
724                                         if (instr_auiipc)
725                                             new_ascii_instr = "auiipc";
726                                         if (instr_jal)
727                                             new_ascii_instr = "jal";
728                                         if (instr_jalr)
729                                             new_ascii_instr = "jalr";
730                                         if (instr_beq)
731                                             new_ascii_instr = "beq";
732                                         if (instr_bne)
733                                             new_ascii_instr = "bne";
734                                         if (instr_blt)
735                                             new_ascii_instr = "blt";
736                                         if (instr_bge)
737                                             new_ascii_instr = "bge";
738                                         if (instr_bltu)
739                                             new_ascii_instr = "bltu";
740                                         if (instr_bgeu)
741                                             new_ascii_instr = "bgeu";

```

```

715
716         if (instr_lb)      new_ascii_instr = "lb";
717         if (instr_lh)      new_ascii_instr = "lh";
718         if (instr_lw)      new_ascii_instr = "lw";
719         if (instr_lbu)     new_ascii_instr = "lbu";
720         if (instr_lhu)     new_ascii_instr = "lhu";
721         if (instr_sb)      new_ascii_instr = "sb";
722         if (instr_sh)      new_ascii_instr = "sh";
723         if (instr_sw)      new_ascii_instr = "sw";
724
725         if (instr_addi)     new_ascii_instr = "addi";
726         if (instr_slti)     new_ascii_instr = "slti";
727         if (instr_sltiu)    new_ascii_instr = "sltiu";
728         if (instr_xori)     new_ascii_instr = "xori";
729         if (instr_ori)      new_ascii_instr = "ori";
730         if (instr_andi)     new_ascii_instr = "andi";
731         if (instr_slli)     new_ascii_instr = "slli";
732         if (instr_srli)     new_ascii_instr = "srli";
733         if (instr_srai)     new_ascii_instr = "srai";
734
735         if (instr_add)      new_ascii_instr = "add";
736         if (instr_sub)      new_ascii_instr = "sub";
737         if (instr_sll)      new_ascii_instr = "sll";
738         if (instr_slt)      new_ascii_instr = "slt";
739         if (instr_sltu)     new_ascii_instr = "sltu";
740         if (instr_xor)      new_ascii_instr = "xor";
741         if (instr_srl)      new_ascii_instr = "srl";
742         if (instr_sra)      new_ascii_instr = "sra";
743         if (instr_or)       new_ascii_instr = "or";
744         if (instr_and)      new_ascii_instr = "and";
745
746         if (instr_rdcycle)  new_ascii_instr = "rd-
cycle";
747         if (instr_rdcycleh) new_ascii_instr = "rd-
cycleh";
748         if (instr_rdinstr)  new_ascii_instr = "rdin-
str";
749         if (instr_rdinstrh) new_ascii_instr = "rdin-
strh";
750         if (instr_fence)    new_ascii_instr = "fence";
751
752         if (instr_getq)     new_ascii_instr = "getq";
753         if (instr_setq)     new_ascii_instr = "setq";
754         if (instr_retirq)   new_ascii_instr = "retirq";
755         if (instr_maskirq)  new_ascii_instr = "maskirq";
756         if (instr_waitirq)  new_ascii_instr = "waitirq";
757         if (instr_timer)    new_ascii_instr = "timer";
758     end
759
760     reg [63:0] q_ascii_instr;
761     reg [31:0] q_insn_imm;
762     reg [31:0] q_insn_opcode;
763     reg [4:0] q_insn_rs1;
764     reg [4:0] q_insn_rs2;
765     reg [4:0] q_insn_rd;
766     reg dbg_next;
767
768     wire launch_next_insn;
769     reg dbg_valid_insn;
770
771     reg [63:0] cached_ascii_instr;
772     reg [31:0] cached_insn_imm;
773     reg [31:0] cached_insn_opcode;
774     reg [4:0] cached_insn_rs1;
775     reg [4:0] cached_insn_rs2;
776     reg [4:0] cached_insn_rd;
777
778     always @(posedge clk) begin
779         q_ascii_instr <= dbg_ascii_instr;
780         q_insn_imm <= dbg_insn_imm;
781         q_insn_opcode <= dbg_insn_opcode;
782         q_insn_rs1 <= dbg_insn_rs1;
783         q_insn_rs2 <= dbg_insn_rs2;
784         q_insn_rd <= dbg_insn_rd;
785         dbg_next <= launch_next_insn;
786
787         if (!resetn || trap)
788             dbg_valid_insn <= 0;
789         else if (launch_next_insn)
790             dbg_valid_insn <= 1;
791
792         if (decoder_trigger_q) begin
793             cached_ascii_instr <= new_ascii_instr;
794             cached_insn_imm <= decoded_imm;
795             if (&next_insn_opcode[1:0])
796                 cached_insn_opcode <= next_in-
sn_opcode;
797             else
798                 cached_insn_opcode <= {16'b0,
next_insn_opcode[15:0]};
799             cached_insn_rs1 <= decoded_rs1;
800             cached_insn_rs2 <= decoded_rs2;
801             cached_insn_rd <= decoded_rd;
802         end
803
804         if (launch_next_insn) begin
805             dbg_insn_addr <= next_pc;
806         end
807     end
808
809     always @* begin
810         dbg_ascii_instr = q_ascii_instr;
811         dbg_insn_imm = q_insn_imm;
812         dbg_insn_opcode = q_insn_opcode;
813         dbg_insn_rs1 = q_insn_rs1;
814         dbg_insn_rs2 = q_insn_rs2;
815         dbg_insn_rd = q_insn_rd;
816
817         if (dbg_next) begin
818             if (decoder_pseudo_trigger_q) begin
819                 dbg_ascii_instr =
cached_ascii_instr;
820                 dbg_insn_imm = cached_insn_imm;
821                 dbg_insn_opcode =
cached_insn_opcode;
822                 dbg_insn_rs1 = cached_insn_rs1;
823                 dbg_insn_rs2 = cached_insn_rs2;
824                 dbg_insn_rd = cached_insn_rd;
825             end else begin
826                 dbg_ascii_instr = new_as-
cii_instr;
827                 if (&next_insn_opcode[1:0])
828                     dbg_insn_opcode =
next_insn_opcode;
829                 else
830                     dbg_insn_opcode =
{16'b0, next_insn_opcode[15:0]};
831                 dbg_insn_imm = decoded_imm;
832                 dbg_insn_rs1 = decoded_rs1;
833                 dbg_insn_rs2 = decoded_rs2;
834                 dbg_insn_rd = decoded_rd;
835             end
836         end
837     end
838
839     `ifdef DEBUGASM
840     always @(posedge clk) begin
841         if (dbg_next) begin
842             $display("debugasm %x %x %s", dbg_in-
sn_addr, dbg_insn_opcode, dbg_ascii_instr ? dbg_ascii_instr : "");
843         end
844     end
845     `endif
846
847     `ifdef DEBUG
848     always @(posedge clk) begin
849         if (dbg_next) begin
850             if (&dbg_insn_opcode[1:0])
851                 $display("DECODE: 0x%08x 0x%08x
%-0s", dbg_insn_addr, dbg_insn_opcode, dbg_ascii_instr ? dbg_as-
cii_instr : "UNKNOWN");
852             else
853                 $display("DECODE: 0x%08x
0x%04x %-0s", dbg_insn_addr, dbg_insn_opcode[15:0], dbg_as-
cii_instr ? dbg_ascii_instr : "UNKNOWN");
854             end
855         end
856     end
857     `endif
858
859     always @(posedge clk) begin
860         is_lui_auipc_jal <= |{instr_lui, instr_auipc,
instr_jal};
861         is_lui_auipc_jal_jalr_addi_add_sub <= |{in-
str_lui, instr_auipc, instr_jal, instr_jalr, instr_addi, instr_add,
instr_sub};
862         is_slti_blt_slt <= |{instr_slti, instr_blt, in-
str_slt};
863         is_sltiu_bltu_sltu <= |{instr_sltiu, in-
str_bltu, instr_sltu};
864         is_lbu_lhu_lw <= |{instr_lbu, instr_lhu, in-
str_lw};
865         is_compare <= |{is_beq_bne_blt_bge_bltu_bgeu,
instr_slti, instr_slt, instr_sltiu, instr_sltu};
866         if (mem_do_rinst && mem_done) begin
867             instr_lui <= mem_rdata_latched[6:0]
== 7'b0110111;
868             instr_auipc <= mem_rdata_latched[6:0]
== 7'b0010111;
869             instr_jal <= mem_rdata_latched[6:0]
== 7'b1101111;
870             instr_jalr <= mem_rdata_latched[6:0]
== 7'b1100111 && mem_rdata_latched[14:12] == 3'b000;
871             instr_retirq <= mem_rdata_latched[6:0]
== 7'b0001011 && mem_rdata_latched[31:25] == 7'b0000010 && EN-
ABLE_IRQ;
872             instr_waitirq <= mem_rdata_latched[6:0]
== 7'b0001011 && mem_rdata_latched[31:25] == 7'b0000010 && EN-
ABLE_IRQ;
873             is_beq_bne_blt_bge_bltu_bgeu <=
mem_rdata_latched[6:0] == 7'b1100011;
874             is_lb_lw_lhu_lhu <=

```

```

mem_rdata_latched[6:0] == 7'b0000011;
876         is_sb_sh_sw          <=
mem_rdata_latched[6:0] == 7'b0100011;
877         is_alu_reg_imm       <=
mem_rdata_latched[6:0] == 7'b0010011;
878         is_alu_reg_reg       <=
mem_rdata_latched[6:0] == 7'b0110011;
879
880         { decoded_imm_j[31:20], de-
coded_imm_j[10:1], decoded_imm_j[11], decoded_imm_j[19:12], de-
coded_imm_j[0] } <= $signed({mem_rdata_latched[31:12], 1'b0});
881
882         decoded_rd <= mem_rdata_latched[11:7];
883         decoded_rs1 <=
mem_rdata_latched[19:15];
884         decoded_rs2 <=
mem_rdata_latched[24:20];
885
886         if (mem_rdata_latched[6:0] ==
7'b0001011 && mem_rdata_latched[31:25] == 7'b0000000 && ENABLE_IRQ
&& ENABLE_IRQ_QREGS)
887             decoded_rs1[regindex_bits-1] <=
1; // instr_getq
888
889         if (mem_rdata_latched[6:0] ==
7'b0001011 && mem_rdata_latched[31:25] == 7'b0000010 && ENABLE_IRQ)
890             decoded_rs1 <= EN-
ABLE_IRQ_QREGS ? irqregs_offset : 3; // instr_retirq
891
892         compressed_instr <= 0;
893         if (COMPRESSED_ISA &&
mem_rdata_latched[1:0] != 2'b11) begin
894             compressed_instr <= 1;
895             decoded_rd <= 0;
896             decoded_rs1 <= 0;
897             decoded_rs2 <= 0;
898
899             { decoded_imm_j[31:11], de-
coded_imm_j[4], decoded_imm_j[9:8], decoded_imm_j[10], de-
coded_imm_j[6],
900             decoded_imm_j[7], de-
coded_imm_j[3:1], decoded_imm_j[5], decoded_imm_j[0] } <=
$signed({mem_rdata_latched[12:2], 1'b0});
901
902             case (mem_rdata_latched[1:0])
903                 2'b00: begin // Quad-
rant 0
904                     case
(mem_rdata_latched[15:13])
905                 3'b000: begin // C.ADDI4SPN
906
907             is_alu_reg_imm <= |mem_rdata_latched[12:5];
908
909             decoded_rs1 <= 2;
910
911             decoded_rd <= 8 + mem_rdata_latched[4:2];
912
913             decoded_rs1 <= 8 + mem_rdata_latched[9:7];
914
915             decoded_rd <= 8 + mem_rdata_latched[4:2];
916
917             decoded_rs1 <= 8 + mem_rdata_latched[9:7];
918
919             decoded_rs2 <= 8 + mem_rdata_latched[4:2];
920
921             endcase
922             2'b01: begin // Quad-
rant 1
923                 case
(mem_rdata_latched[15:13])
924             3'b000: begin // C.NOP / C.ADDI
925
926             is_alu_reg_imm <= 1;
927
928             decoded_rd <= mem_rdata_latched[11:7];

```

```

927
928         decoded_rs1 <= mem_rdata_latched[11:7];
929
930         3'b001: begin // C.JAL
931
932         instr_jal <= 1;
933
934         decoded_rd <= 1;
935
936         010: begin // C.LI
937
938         is_alu_reg_imm <= 1;
939
940         decoded_rd <= mem_rdata_latched[11:7];
941
942         decoded_rs1 <= 0;
943
944         011: begin
945
946         if (mem_rdata_latched[12] || mem_rdata_latched[6:2]) begin
947
948         if (mem_rdata_latched[11:7] == 2) begin // C.ADDI16SP
949
950         is_alu_reg_imm <= 1;
951
952         decoded_rd <= mem_rdata_latched[11:7];
953
954         decoded_rs1 <= mem_rdata_latched[11:7];
955
956         end else begin // C.LUI
957
958         instr_lui <= 1;
959
960         decoded_rd <= mem_rdata_latched[11:7];
961
962         decoded_rs1 <= 0;
963
964         end
965
966         3'b100: begin
967
968         if (!mem_rdata_latched[11] && !mem_rdata_latched[12]) begin //
C.SRLI, C.SRAI
969
970         is_alu_reg_imm <= 1;
971
972         decoded_rd <= 8 + mem_rdata_latched[9:7];
973
974         decoded_rs1 <= 8 + mem_rdata_latched[9:7];
975
976         decoded_rs2 <= {mem_rdata_latched[12], mem_rdata_latched[6:2]};
977
978         end
979
980         if (mem_rdata_latched[11:10] == 2'b10) begin // C.ANDI
981
982         is_alu_reg_imm <= 1;
983
984         decoded_rd <= 8 + mem_rdata_latched[9:7];
985
986         decoded_rs1 <= 8 + mem_rdata_latched[9:7];
987
988         end
989
990         if (mem_rdata_latched[12:10] == 3'b011) begin // C.SUB, C.XOR, C.OR,
C.AND

```

```

964 is_alu_reg_reg <= 1;
965 decoded_rd <= 8 + mem_rdata_latched[9:7];
966 decoded_rs1 <= 8 + mem_rdata_latched[9:7];
967 decoded_rs2 <= 8 + mem_rdata_latched[4:2];
968 end
969                                     end
970 3'b101: begin // C.J
971 instr_jal <= 1;
972                                     end
973 3'b110: begin // C.BEQZ
974 is_beq_bne_blt_bge_bltu_bgeu <= 1;
975 decoded_rs1 <= 8 + mem_rdata_latched[9:7];
976 decoded_rs2 <= 0;
977                                     end
978 3'b111: begin // C.BNEZ
979 is_beq_bne_blt_bge_bltu_bgeu <= 1;
980 decoded_rs1 <= 8 + mem_rdata_latched[9:7];
981 decoded_rs2 <= 0;
982                                     end
983                                     endcase
984                                     end
985 2'b10: begin // Quad-
986                                     case
987 (mem_rdata_latched[15:13])
988 3'b000: begin // C.SLLI
989 if (!mem_rdata_latched[12]) begin
990 is_alu_reg_imm <= 1;
991 decoded_rd <= mem_rdata_latched[11:7];
992 decoded_rs1 <= mem_rdata_latched[11:7];
993 decoded_rs2 <= {mem_rdata_latched[12], mem_rdata_latched[6:2]};
994 end
995 3'b010: begin // C.LWSP
996 if (mem_rdata_latched[11:7]) begin
997 is_lb_lh_lw_lbu_lhu <= 1;
998 decoded_rd <= mem_rdata_latched[11:7];
999 decoded_rs1 <= 2;
1000 end
1001                                     end
1002 3'b100: begin
1003 if (mem_rdata_latched[12] == 0 && mem_rdata_latched[11:7] != 0 &&
mem_rdata_latched[6:2] == 0) begin // C.JR
1004 instr_jalr <= 1;
1005
1006 decoded_rd <= 0;
1007 end
1008 if (mem_rdata_latched[12] == 0 && mem_rdata_latched[6:2] != 0) be-
gin // C.MV
1009 is_alu_reg_reg <= 1;
1010 decoded_rd <= mem_rdata_latched[11:7];
1011 decoded_rs1 <= 0;
1012 decoded_rs2 <= mem_rdata_latched[6:2];
1013 end
1014 if (mem_rdata_latched[12] != 0 && mem_rdata_latched[11:7] != 0 &&
mem_rdata_latched[6:2] == 0) begin // C.JALR
1015 instr_jalr <= 1;
1016 decoded_rd <= 1;
1017 decoded_rs1 <= mem_rdata_latched[11:7];
1018 end
1019 if (mem_rdata_latched[12] != 0 && mem_rdata_latched[6:2] != 0) be-
gin // C.ADD
1020 is_alu_reg_reg <= 1;
1021 decoded_rd <= mem_rdata_latched[11:7];
1022 decoded_rs1 <= mem_rdata_latched[11:7];
1023 decoded_rs2 <= mem_rdata_latched[6:2];
1024 end
1025                                     end
1026 3'b110: begin // C.SWSP
1027 is_sb_sh_sw <= 1;
1028 decoded_rs1 <= 2;
1029 decoded_rs2 <= mem_rdata_latched[6:2];
1030                                     end
1031                                     endcase
1032                                     end
1033                                     endcase
1034                                     end
1035                                     end
1036
1037 if (decoder_trigger && !decoder_pseudo_trigger)
begin
1038 pcpi_insn <= WITH_PCPI ? mem_rdata_q :
'bx;
1039
1040 instr_beq <=
is_beq_bne_blt_bge_bltu_bgeu && mem_rdata_q[14:12] == 3'b000;
1041 instr_bne <=
is_beq_bne_blt_bge_bltu_bgeu && mem_rdata_q[14:12] == 3'b001;
1042 instr_blt <=
is_beq_bne_blt_bge_bltu_bgeu && mem_rdata_q[14:12] == 3'b100;
1043 instr_bge <=
is_beq_bne_blt_bge_bltu_bgeu && mem_rdata_q[14:12] == 3'b101;
1044 instr_bltu <=
is_beq_bne_blt_bge_bltu_bgeu && mem_rdata_q[14:12] == 3'b110;
1045 instr_bgeu <=
is_beq_bne_blt_bge_bltu_bgeu && mem_rdata_q[14:12] == 3'b111;
1046
1047 instr_lb <= is_lb_lh_lw_lbu_lhu &&
mem_rdata_q[14:12] == 3'b000;

```



```

1048 instr_lh      <= is_lb_lh_lw_lbu_lhu &&
mem_rdata_q[14:12] == 3'b001;
1049 instr_lw      <= is_lb_lh_lw_lbu_lhu &&
mem_rdata_q[14:12] == 3'b010;
1050 instr_lbu      <= is_lb_lh_lw_lbu_lhu &&
mem_rdata_q[14:12] == 3'b100;
1051 instr_lhu      <= is_lb_lh_lw_lbu_lhu &&
mem_rdata_q[14:12] == 3'b101;
1052
1053 instr_sb      <= is_sb_sh_sw &&
mem_rdata_q[14:12] == 3'b000;
1054 instr_sh      <= is_sb_sh_sw &&
mem_rdata_q[14:12] == 3'b001;
1055 instr_sw      <= is_sb_sh_sw &&
mem_rdata_q[14:12] == 3'b010;
1056
1057 instr_addi     <= is_alu_reg_imm &&
mem_rdata_q[14:12] == 3'b000;
1058 instr_slti     <= is_alu_reg_imm &&
mem_rdata_q[14:12] == 3'b010;
1059 instr_sltiu    <= is_alu_reg_imm &&
mem_rdata_q[14:12] == 3'b011;
1060 instr_xori     <= is_alu_reg_imm &&
mem_rdata_q[14:12] == 3'b100;
1061 instr_ori      <= is_alu_reg_imm &&
mem_rdata_q[14:12] == 3'b110;
1062 instr_andi     <= is_alu_reg_imm &&
mem_rdata_q[14:12] == 3'b111;
1063
1064 instr_slli     <= is_alu_reg_imm &&
mem_rdata_q[14:12] == 3'b001 && mem_rdata_q[31:25] == 7'b00000000;
1065 instr_srli     <= is_alu_reg_imm &&
mem_rdata_q[14:12] == 3'b101 && mem_rdata_q[31:25] == 7'b00000000;
1066 instr_srai     <= is_alu_reg_imm &&
mem_rdata_q[14:12] == 3'b101 && mem_rdata_q[31:25] == 7'b01000000;
1067
1068 instr_add      <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b000 && mem_rdata_q[31:25] == 7'b00000000;
1069 instr_sub      <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b000 && mem_rdata_q[31:25] == 7'b01000000;
1070 instr_sll      <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b001 && mem_rdata_q[31:25] == 7'b00000000;
1071 instr_slt      <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b010 && mem_rdata_q[31:25] == 7'b00000000;
1072 instr_sltu     <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b011 && mem_rdata_q[31:25] == 7'b00000000;
1073 instr_xor      <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b100 && mem_rdata_q[31:25] == 7'b00000000;
1074 instr_srl      <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b101 && mem_rdata_q[31:25] == 7'b00000000;
1075 instr_sra      <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b101 && mem_rdata_q[31:25] == 7'b01000000;
1076 instr_or       <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b110 && mem_rdata_q[31:25] == 7'b00000000;
1077 instr_and      <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b111 && mem_rdata_q[31:25] == 7'b00000000;
1078
1079 instr_rdcycle  <= ((mem_rdata_q[6:0] ==
7'b1110011 && mem_rdata_q[31:12] == 'b1100000000000000010) ||
(mem_rdata_q[6:0] ==
7'b1110011 && mem_rdata_q[31:12] == 'b11000000000100000010)) && EN-
ABLE_COUNTERS;
1080
1081 instr_rdcycleh <= ((mem_rdata_q[6:0] ==
7'b1110011 && mem_rdata_q[31:12] == 'b1100100000000000010) ||
(mem_rdata_q[6:0] ==
7'b1110011 && mem_rdata_q[31:12] == 'b11001000000100000010)) && EN-
ABLE_COUNTERS && ENABLE_COUNTERS64;
1082
1083 instr_rdinstr  <= (mem_rdata_q[6:0] ==
7'b1110011 && mem_rdata_q[31:12] == 'b1100000000100000010) && EN-
ABLE_COUNTERS;
1084
1085 instr_rdinstrh <= (mem_rdata_q[6:0] ==
7'b1110011 && mem_rdata_q[31:12] == 'b1100100000100000010) && EN-
ABLE_COUNTERS && ENABLE_COUNTERS64;
1086
1087 instr_ecall_ebreak <=
((mem_rdata_q[6:0] == 7'b1110011 && !mem_rdata_q[31:21] && !
mem_rdata_q[19:7]) ||
(COMPRESSED_ISA &&
mem_rdata_q[15:0] == 16'h9002));
1088 instr_fence   <= (mem_rdata_q[6:0] ==
7'b0001111 && !mem_rdata_q[14:12]);
1089
1090 instr_getq     <= mem_rdata_q[6:0] ==
7'b0001011 && mem_rdata_q[31:25] == 7'b00000000 && ENABLE_IRQ && EN-
ABLE_IRQ_QREGS;
1091 instr_setq     <= mem_rdata_q[6:0] ==
7'b0001011 && mem_rdata_q[31:25] == 7'b00000001 && ENABLE_IRQ && EN-
ABLE_IRQ_QREGS;
1092 instr_maskirq <= mem_rdata_q[6:0] ==
7'b0001011 && mem_rdata_q[31:25] == 7'b00000011 && ENABLE_IRQ;
1093 instr_timer   <= mem_rdata_q[6:0] ==
7'b0001011 && mem_rdata_q[31:25] == 7'b00000101 && ENABLE_IRQ && EN-
ABLE_IRQ_TIMER;
1094
1095 is_slli_srli_srai <= is_alu_reg_imm &&

```

```

|{
1096 mem_rdata_q[31:25] == 7'b00000000, mem_rdata_q[14:12] == 3'b001 &&
1097 mem_rdata_q[31:25] == 7'b00000000, mem_rdata_q[14:12] == 3'b101 &&
1098 mem_rdata_q[31:25] == 7'b01000000, mem_rdata_q[14:12] == 3'b101 &&
1099 };
1100 is_jalr_addi_slti_sltiu_xori_ori_andi
1101 <= instr_jalr || is_alu_reg_imm && |{
1102 mem_rdata_q[14:12] == 3'b000,
1103 mem_rdata_q[14:12] == 3'b010,
1104 mem_rdata_q[14:12] == 3'b011,
1105 mem_rdata_q[14:12] == 3'b100,
1106 mem_rdata_q[14:12] == 3'b110,
1107 mem_rdata_q[14:12] == 3'b111
1108 };
1109 is_sll_srl_sra <= is_alu_reg_reg && |{
1110 mem_rdata_q[14:12] == 3'b001 &&
1111 mem_rdata_q[31:25] == 7'b00000000, mem_rdata_q[14:12] == 3'b101 &&
1112 mem_rdata_q[31:25] == 7'b00000000, mem_rdata_q[14:12] == 3'b101 &&
1113 mem_rdata_q[31:25] == 7'b01000000, mem_rdata_q[14:12] == 3'b101 &&
1114 };
1115 is_lui_auiopc_jal_jalr_addi_add_sub <=
0;
1117 is_compare <= 0;
1118 (* parallel_case *)
1119 case (1'b1)
1120 instr_jal:
1121 decoded_imm <= de-
coded_imm_j;
1123 |{instr_lui, instr_auiopc}:
1124 decoded_imm <=
mem_rdata_q[31:12] << 12;
1125 |{instr_jalr,
is_lb_lh_lw_lbu_lhu, is_alu_reg_imm}:
1126 decoded_imm <=
$signed(mem_rdata_q[31:20]);
1127 is_beq_bne_blt_bge_bltu_bgeu:
1128 decoded_imm <=
$signed({mem_rdata_q[31], mem_rdata_q[7], mem_rdata_q[30:25],
mem_rdata_q[11:8], 1'b0});
1129 is_sb_sh_sw:
1130 decoded_imm <=
$signed({mem_rdata_q[31:25], mem_rdata_q[11:7]});
1131 default:
1132 decoded_imm <= 1'bx;
1133
1134 endcase
1135
1136 if (!resetn) begin
1137 is_beq_bne_blt_bge_bltu_bgeu <= 0;
1138 is_compare <= 0;
1139
1140 instr_beq <= 0;
1141 instr_bne <= 0;
1142 instr_blt <= 0;
1143 instr_bge <= 0;
1144 instr_bltu <= 0;
1145 instr_bgeu <= 0;
1146
1147 instr_addi <= 0;
1148 instr_slti <= 0;
1149 instr_sltiu <= 0;
1150 instr_xori <= 0;
1151 instr_ori <= 0;
1152 instr_andi <= 0;
1153
1154 instr_add <= 0;
1155 instr_sub <= 0;
1156 instr_sll <= 0;
1157 instr_slt <= 0;
1158 instr_sltu <= 0;
1159 instr_xor <= 0;
1160 instr_srl <= 0;
1161 instr_sra <= 0;
1162 instr_or <= 0;
1163 instr_and <= 0;
1164
1165 instr_fence <= 0;
1166
1167 end
1168
1169 // Main State Machine
1170
1171 localparam cpu_state_trap = 8'b10000000;
1172 localparam cpu_state_fetch = 8'b01000000;
1173 localparam cpu_state_ld_rs1 = 8'b00100000;
1174

```



```

1175 localparam cpu_state_ld_rs2 = 8'b000010000;
1176 localparam cpu_state_exec = 8'b00001000;
1177 localparam cpu_state_shift = 8'b00000100;
1178 localparam cpu_state_stmem = 8'b00000010;
1179 localparam cpu_state_ldmem = 8'b00000001;
1180
1181 reg [7:0] cpu_state;
1182 reg [1:0] irq_state;
1183
1184 `FORMAL_KEEP reg [127:0] dbg_ascii_state;
1185
1186 always @* begin
1187   dbg_ascii_state = "";
1188   if (cpu_state == cpu_state_trap) dbg_as-
cii_state = "trap";
1189   if (cpu_state == cpu_state_fetch) dbg_as-
cii_state = "fetch";
1190   if (cpu_state == cpu_state_ld_rs1) dbg_as-
cii_state = "ld_rs1";
1191   if (cpu_state == cpu_state_ld_rs2) dbg_as-
cii_state = "ld_rs2";
1192   if (cpu_state == cpu_state_exec) dbg_as-
cii_state = "exec";
1193   if (cpu_state == cpu_state_shift) dbg_as-
cii_state = "shift";
1194   if (cpu_state == cpu_state_stmem) dbg_as-
cii_state = "stmem";
1195   if (cpu_state == cpu_state_ldmem) dbg_as-
cii_state = "ldmem";
1196 end
1197
1198 reg set_mem_do_rinst;
1199 reg set_mem_do_rdata;
1200 reg set_mem_do_wdata;
1201
1202 reg latched_store;
1203 reg latched_stalu;
1204 reg latched_branch;
1205 reg latched_compr;
1206 reg latched_trace;
1207 reg latched_is_lu;
1208 reg latched_is_lh;
1209 reg latched_is_lb;
1210 reg [regindex_bits-1:0] latched_rd;
1211
1212 reg [31:0] current_pc;
1213 assign next_pc = latched_store && latched_branch ?
reg_out & ~1 : reg_next_pc;
1214
1215 reg [3:0] pcpi_timeout_counter;
1216 reg pcpi_timeout;
1217
1218 reg [31:0] next_irq_pending;
1219 reg do_waitirq;
1220
1221 reg [31:0] alu_out, alu_out_q;
1222 reg alu_out_0, alu_out_0_q;
1223 reg alu_wait, alu_wait_2;
1224
1225 reg [31:0] alu_add_sub;
1226 reg [31:0] alu_shl, alu_shr;
1227 reg alu_eq, alu_ltu, alu_lts;
1228
1229 generate if (TWO_CYCLE_ALU) begin
1230   always @(posedge clk) begin
1231     alu_add_sub <= instr_sub ? reg_op1 -
reg_op2 : reg_op1 + reg_op2;
1232     alu_eq <= reg_op1 == reg_op2;
1233     alu_lts <= $signed(reg_op1) <
$signed(reg_op2);
1234     alu_ltu <= reg_op1 < reg_op2;
1235     alu_shl <= reg_op1 << reg_op2[4:0];
1236     alu_shr <= $signed({instr_sra || in-
str_srai ? reg_op1[31] : 1'b0, reg_op1}) >>> reg_op2[4:0];
1237   end
1238   end else begin
1239     always @* begin
1240       alu_add_sub = instr_sub ? reg_op1 -
reg_op2 : reg_op1 + reg_op2;
1241       alu_eq = reg_op1 == reg_op2;
1242       alu_lts = $signed(reg_op1) <
$signed(reg_op2);
1243       alu_ltu = reg_op1 < reg_op2;
1244       alu_shl = reg_op1 << reg_op2[4:0];
1245       alu_shr = $signed({instr_sra || in-
str_srai ? reg_op1[31] : 1'b0, reg_op1}) >>> reg_op2[4:0];
1246     end
1247   end endgenerate
1248
1249 always @* begin
1250   alu_out_0 = 'bx;
1251   (* parallel_case, full_case *)
1252   case (1'b1)
1253     instr_beq:
1254       alu_out_0 = alu_eq;
1255     instr_bne:
1256       alu_out_0 = !alu_eq;
1257     instr_bge:
1258       alu_out_0 = !alu_lts;
1259     instr_bgeu:
1260       alu_out_0 = !alu_ltu;

```

```

1261     is_slti_blt_slt && (!TWO_CYCLE_COMPARE
|| !{instr_beq, instr_bne, instr_bge, instr_bgeu}):
1262       alu_out_0 = alu_lts;
1263     is_sltiu_bltu_sltu && (!
TWO_CYCLE_COMPARE || !{instr_beq, instr_bne, instr_bge, instr_bgeu}):
1264       alu_out_0 = alu_ltu;
1265   endcase
1266
1267   alu_out = 'bx;
1268   (* parallel_case, full_case *)
1269   case (1'b1)
1270     is_lui_auiopc_jal_jalr_addi_add_sub:
1271       alu_out = alu_add_sub;
1272     is_compare:
1273       alu_out = alu_out_0;
1274     instr_xori || instr_xor:
1275       alu_out = reg_op1 ^ reg_op2;
1276     instr_ori || instr_or:
1277       alu_out = reg_op1 | reg_op2;
1278     instr_andi || instr_and:
1279       alu_out = reg_op1 & reg_op2;
1280     BARREL_SHIFTER && (instr_sll || in-
str_slli):
1281       alu_out = alu_shl;
1282     BARREL_SHIFTER && (instr_srl || in-
str_srli || instr_sra || instr_srai):
1283       alu_out = alu_shr;
1284   endcase
1285
1286 `ifdef RISC_V_FORMAL_BLACKBOX_ALU
1287   alu_out_0 = $anyseq;
1288   alu_out = $anyseq;
1289 `endif
1290 end
1291
1292 reg clear_prefetched_high_word_q;
1293 always @(posedge clk) clear_prefetched_high_word_q <=
clear_prefetched_high_word;
1294
1295 always @* begin
1296   clear_prefetched_high_word =
clear_prefetched_high_word_q;
1297   if (!prefetched_high_word)
1298     clear_prefetched_high_word = 0;
1299   if (latched_branch || irq_state || !resetn)
1300     clear_prefetched_high_word = COM-
PRESSED_ISA;
1301 end
1302
1303 reg cpuregs_write;
1304 reg [31:0] cpuregs_wrdata;
1305 reg [31:0] cpuregs_rs1;
1306 reg [31:0] cpuregs_rs2;
1307 reg [regindex_bits-1:0] decoded_rs;
1308
1309 always @* begin
1310   cpuregs_write = 0;
1311   cpuregs_wrdata = 'bx;
1312
1313   if (cpu_state == cpu_state_fetch) begin
1314     (* parallel_case *)
1315     case (1'b1)
1316       latched_branch: begin
1317         cpuregs_wrdata = reg_pc
+ (latched_compr ? 2 : 4);
1318       end
1319       cpuregs_write = 1;
1320       latched_store && !
latched_branch: begin
1321         cpuregs_wrdata =
reg_next_pc | latched_compr;
1322         cpuregs_write = 1;
1323       end
1324       ENABLE_IRQ && irq_state[0]: be-
gin
1325         cpuregs_wrdata =
reg_op2;
1326         cpuregs_write = 1;
1327       end
1328       ENABLE_IRQ && irq_state[1]: be-
gin
1329         cpuregs_wrdata =
reg_op1;
1330         cpuregs_write = 1;
1331       end
1332     endcase
1333   end
1334 end
1335
1336 `ifndef PICORV32_REGS
1337   always @(posedge clk) begin
1338     if (resetn && cpuregs_write && latched_rd)
1339       `ifdef PICORV32_TESTBUG_001
1340         cpuregs[latched_rd ^ 1] <=
cpuregs_wrdata;
1341       `else
1342         cpuregs[latched_rd] <= cpuregs_wrdata ^
1;
1343       `else

```

```

1344         cpuregs[latched_rd] <= cpuregs_wrdata;
1345     `endif
1346     end
1347
1348     always @* begin
1349         decoded_rs = 'bx;
1350         if (ENABLE_REGS_DUALPORT) begin
1351             `ifndef RISC_V_FORMAL_BLACKBOX_REGS
1352                 cpuregs_rs1 = decoded_rs1 ?
cpuregs[decoded_rs1] : 0;
1353                 cpuregs_rs2 = decoded_rs2 ?
cpuregs[decoded_rs2] : 0;
1354             `else
1355                 cpuregs_rs1 = decoded_rs1 ? $anyseq :
0;
1356                 cpuregs_rs2 = decoded_rs2 ? $anyseq :
0;
1357             `endif
1358             end else begin
1359                 decoded_rs = (cpu_state ==
cpu_state_ld_rs2) ? decoded_rs2 : decoded_rs1;
1360             `ifndef RISC_V_FORMAL_BLACKBOX_REGS
1361                 cpuregs_rs1 = decoded_rs ?
cpuregs[decoded_rs] : 0;
1362             `else
1363                 cpuregs_rs1 = decoded_rs ? $anyseq : 0;
1364             `endif
1365                 cpuregs_rs2 = cpuregs_rs1;
1366             end
1367         `else
1368             wire[31:0] cpuregs_rdata1;
1369             wire[31:0] cpuregs_rdata2;
1370             wire [5:0] cpuregs_waddr = latched_rd;
1371             wire [5:0] cpuregs_raddr1 = ENABLE_REGS_DUALPORT ? de-
coded_rs1 : decoded_rs;
1372             wire [5:0] cpuregs_raddr2 = ENABLE_REGS_DUALPORT ? de-
coded_rs2 : 0;
1373             `PICO RV32_REGS cpuregs (
1374                 .clk(clk),
1375                 .wen(resetn && cpuregs_write && latched_rd),
1376                 .waddr(cpuregs_waddr),
1377                 .raddr1(cpuregs_raddr1),
1378                 .raddr2(cpuregs_raddr2),
1379                 .wdata(cpuregs_wrdata),
1380                 .rdata1(cpuregs_rdata1),
1381                 .rdata2(cpuregs_rdata2)
1382             );
1383             always @* begin
1384                 decoded_rs = 'bx;
1385                 if (ENABLE_REGS_DUALPORT) begin
1386                     cpuregs_rs1 = decoded_rs1 ?
cpuregs_rdata1 : 0;
1387                     cpuregs_rs2 = decoded_rs2 ?
cpuregs_rdata2 : 0;
1388                 `end else begin
1389                     decoded_rs = (cpu_state ==
cpu_state_ld_rs2) ? decoded_rs2 : decoded_rs1;
1390                     cpuregs_rs1 = decoded_rs ?
cpuregs_rdata1 : 0;
1391                     cpuregs_rs2 = cpuregs_rs1;
1392                 `end
1393             end
1394         `endif
1395         assign launch_next_insn = cpu_state == cpu_state_fetch
&& decoder_trigger && (!ENABLE_IRQ || irq_delay || irq_active || !
(irq_pending & ~irq_mask));
1401         always @(posedge clk) begin
1402             trap <= 0;
1403             reg_sh <= 'bx;
1404             reg_out <= 'bx;
1405             set_mem_do_rinst = 0;
1406             set_mem_do_rdata = 0;
1407             set_mem_do_wdata = 0;
1408             alu_out_0_q <= alu_out_0;
1409             alu_out_q <= alu_out;
1410             alu_wait <= 0;
1411             alu_wait_2 <= 0;
1412             if (launch_next_insn) begin
1413                 dbg_rs1val <= 'bx;
1414                 dbg_rs2val <= 'bx;
1415                 dbg_rs1val_valid <= 0;
1416                 dbg_rs2val_valid <= 0;
1417             end
1418             if (WITH_PCPI && CATCH_ILLSINS) begin
1419                 if (resetn && pcpi_valid && !
pcpi_int_wait) begin
1420                     if (pcpi_timeout_counter)
1421                         pcpi_timeout_counter <=
pcpi_timeout_counter - 1;
1422                 end else

```

```

1428         pcpi_timeout_counter <= ~0;
1429         pcpi_timeout <= !pcpi_timeout_counter;
1430     end
1431
1432     if (ENABLE_COUNTERS) begin
1433         count_cycle <= resetn ? count_cycle +
1 : 0;
1434         if (!ENABLE_COUNTERS64)
count_cycle[63:32] <= 0;
1435     end else begin
1436         count_cycle <= 'bx;
1437         count_instr <= 'bx;
1438     end
1439     next_irq_pending = ENABLE_IRQ ? irq_pending &
LATCHED_IRQ : 'bx;
1440
1441     if (ENABLE_IRQ && ENABLE_IRQ_TIMER && timer)
begin
1442         timer <= timer - 1;
1443     end
1444
1445     decoder_trigger <= mem_do_rinst && mem_done;
1446     decoder_trigger_q <= decoder_trigger;
1447     decoder_pseudo_trigger <= 0;
1448     decoder_pseudo_trigger_q <= de-
coder_pseudo_trigger;
1449     do_waitirq <= 0;
1450
1451     trace_valid <= 0;
1452
1453     if (!ENABLE_TRACE)
trace_data <= 'bx;
1454
1455     if (!resetn) begin
1456         reg_pc <= PROGADDR_RESET;
1457         reg_next_pc <= PROGADDR_RESET;
1458         if (ENABLE_COUNTERS)
count_instr <= 0;
1459         latched_store <= 0;
1460         latched_stalu <= 0;
1461         latched_branch <= 0;
1462         latched_trace <= 0;
1463         latched_is_lu <= 0;
1464         latched_is_lh <= 0;
1465         latched_is_lb <= 0;
1466         pcpi_valid <= 0;
1467         pcpi_timeout <= 0;
1468         irq_active <= 0;
1469         irq_delay <= 0;
1470         irq_mask <= ~0;
1471         next_irq_pending = 0;
1472         irq_state <= 0;
1473         eoi <= 0;
1474         timer <= 0;
1475         if (~STACKADDR) begin
1476             latched_store <= 1;
1477             latched_rd <= 2;
1478             reg_out <= STACKADDR;
1479         end
1480         cpu_state <= cpu_state_fetch;
1481     end else
(* parallel_case, full_case *)
case (cpu_state)
1482         cpu_state_trap: begin
1483             trap <= 1;
1484         end
1485         cpu_state_fetch: begin
1486             mem_do_rinst <= !de-
coder_trigger && !do_waitirq;
1487             mem_wordsize <= 0;
1488             current_pc = reg_next_pc;
1489             (* parallel_case *)
case (1'b1)
1490                 latched_branch: begin
1491                     current_pc =
latched_store ? (latched_stalu ? alu_out_q : reg_out) & ~1 :
reg_next_pc;
1492                 `de-
bug($display("ST_RD: %2d 0x%08x", latched_rd, reg_pc
+ (latched_compr ? 2 : 4), current_pc));
1493             end
1494                 latched_store && !
latched_branch: begin
1495                 `de-
bug($display("ST_RD: %2d 0x%08x", latched_rd, latched_stalu ?
alu_out_q : reg_out));
1496             end
1497             ENABLE_IRQ &&
irq_state[0]: begin
1498                 current_pc =
PROGADDR_IRQ;
1499                 irq_active <=
1;
1500
1501                 mem_do_rinst <=
1;
1502             end

```

```

1511                                     ENABLE_IRQ &&
irq_state[1]: begin
1512                                     eoi <=
1513                                     next_irq_pending & ~irq_mask;
1514                                     end
1515                                     endcase
1516                                     if (ENABLE_TRACE &&
1517                                     latched_trace) begin
1518                                     latched_trace <= 0;
1519                                     trace_valid <= 1;
1520                                     if (latched_branch)
1521                                     trace_data <=
1522                                     (irq_active ? TRACE_IRQ : 0) | TRACE_BRANCH | (current_pc &
1523                                     32'hfffffffe);
1524                                     else
1525                                     trace_data <=
1526                                     (irq_active ? TRACE_IRQ : 0) | (latched_stalu ? alu_out_q :
1527                                     reg_out);
1528                                     end
1529                                     reg_pc <= current_pc;
1530                                     reg_next_pc <= current_pc;
1531                                     latched_store <= 0;
1532                                     latched_stalu <= 0;
1533                                     latched_branch <= 0;
1534                                     latched_is_lu <= 0;
1535                                     latched_is_lh <= 0;
1536                                     latched_is_lb <= 0;
1537                                     latched_rd <= decoded_rd;
1538                                     latched_compr <= com-
1539                                     pressed_instr;
1540                                     if (ENABLE_IRQ && ((de-
1541                                     coder_trigger && !irq_active && !irq_delay && !(irq_pending &
1542                                     ~irq_mask)) || irq_state)) begin
1543                                     irq_state <=
1544                                     irq_state ==
1545                                     2'b00 ? 2'b01 :
1546                                     irq_state ==
1547                                     2'b01 ? 2'b10 : 2'b00;
1548                                     latched_compr <=
1549                                     latched_compr;
1550                                     if (ENABLE_IRQ_QREGS)
1551                                     latched_rd <=
1552                                     irqregs_offset | irq_state[0];
1553                                     else
1554                                     latched_rd <=
1555                                     irq_state[0] ? 4 : 3;
1556                                     end else
1557                                     if (ENABLE_IRQ && (de-
1558                                     coder_trigger || do_waitirq) && instr_waitirq) begin
1559                                     if (irq_pending) begin
1560                                     latched_store
1561                                     <= 1;
1562                                     reg_out <=
1563                                     irq_pending;
1564                                     reg_next_pc <=
1565                                     current_pc + (compressed_instr ? 2 : 4);
1566                                     mem_do_rinst <=
1567                                     1;
1568                                     end else
1569                                     do_waitirq <=
1570                                     1;
1571                                     end else
1572                                     if (decoder_trigger) begin
1573                                     `debug($display("--
1574                                     %0t", $time));
1575                                     irq_delay <= irq_act-
1576                                     ive;
1577                                     reg_next_pc <= cur-
1578                                     rent_pc + (compressed_instr ? 2 : 4);
1579                                     if (ENABLE_TRACE)
1580                                     latched_trace
1581                                     <= 1;
1582                                     if (ENABLE_COUNTERS)
1583                                     count_instr <=
1584                                     count_instr + 1;
1585                                     if (!EN-
1586                                     ABLE_COUNTERS64) count_instr[63:32] <= 0;
1587                                     end
1588                                     if (instr_jal) begin
1589                                     mem_do_rinst <=
1590                                     1;
1591                                     reg_next_pc <=
1592                                     current_pc + decoded_imm_j;
1593                                     latched_branch
1594                                     <= 1;
1595                                     end else begin
1596                                     mem_do_rinst <=
1597                                     0;
1598                                     mem_do_prefetch
1599                                     <= !instr_jalr && !instr_retirq;

```

```

1574                                     cpu_state <=
cpu_state_ld_rs1;
1575                                     end
1576                                     end
1577                                     end
1578                                     cpu_state_ld_rs1: begin
1579                                     reg_op1 <= 'bx;
1580                                     reg_op2 <= 'bx;
1581                                     (* parallel_case *)
1582                                     case (1'b1)
1583                                     (CATCH_ILLINSN ||
1584                                     WITH_PCPI) && instr_trap: begin
1585                                     if (WITH_PCPI)
1586                                     begin
1587                                     `de-
1588                                     bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1));
1589                                     reg_op1 <= cpuregs_rs1;
1590                                     sival <= cpuregs_rs1;
1591                                     sival_valid <= 1;
1592                                     if
1593                                     (ENABLE_REGS_DUALPORT) begin
1594                                     pcpi_valid <= 1;
1595                                     `debug($display("LD_RS2: %2d 0x%08x", decoded_rs2, cpuregs_rs2));
1596                                     reg_sh <= cpuregs_rs2;
1597                                     reg_op2 <= cpuregs_rs2;
1598                                     dbg_rs2val <= cpuregs_rs2;
1599                                     dbg_rs2val_valid <= 1;
1600                                     if (pcpi_int_ready) begin
1601                                     mem_do_rinst <= 1;
1602                                     pcpi_valid <= 0;
1603                                     reg_out <= pcpi_int_rd;
1604                                     latched_store <= pcpi_int_wr;
1605                                     cpu_state <= cpu_state_fetch;
1606                                     end else
1607                                     if (CATCH_ILLINSN && (pcpi_timeout || instr_ecall_ebreak)) begin
1608                                     pcpi_valid <= 0;
1609                                     `debug($display("EBREAK OR UNSUPPORTED INSN AT 0x%08x", reg_pc));
1610                                     if (ENABLE_IRQ && !irq_mask[irq_ebreak] && !irq_active) begin
1611                                     next_irq_pending[irq_ebreak] = 1;
1612                                     cpu_state <= cpu_state_fetch;
1613                                     end else
1614                                     begin
1615                                     cpu_state <= cpu_state_ld_rs2;
1616                                     end else begin
1617                                     `de-
1618                                     bug($display("EBREAK OR UNSUPPORTED INSN AT 0x%08x", reg_pc));

```

```

1619                                     if
(ENABLE_IRQ && !irq_mask[irq_ebreak] && !irq_active) begin
1620 next_irq_pending[irq_ebreak] = 1;
1621
1622                                     end
else
1623
1624                                     end
1625                                     end
1626                                     ENABLE_COUNTERS &&
is_rdcycle_rdcycleh_rdinstr_rdinstrh: begin
1627                                     (* paral-
lel_case, full_case *)
1628                                     case (1'b1)
1629                                     in-
str_rdcycle:
1630
1631                                     in-
reg_out <= count_cycle[31:0];
1632                                     in-
str_rdcycleh && ENABLE_COUNTERS64:
1633                                     in-
reg_out <= count_cycle[63:32];
1634                                     in-
str_rdinstr:
1635                                     in-
reg_out <= count_instr[31:0];
1636                                     in-
str_rdinstrh && ENABLE_COUNTERS64:
1637                                     in-
reg_out <= count_instr[63:32];
1638                                     endcase
1639                                     latched_store
<= 1;
1640                                     cpu_state <=
cpu_state_fetch;
1641                                     end
1642                                     is_lui_auiopc_jal: begin
1643                                     reg_op1 <= in-
str_lui ? 0 : reg_pc;
1644                                     reg_op2 <= de-
coded_imm;
1645                                     if
(TWO_CYCLE_ALU)
1646                                     else
alu_wait <= 1;
1647                                     else
mem_do_rinst <= mem_do_prefetch;
1648                                     cpu_state <=
cpu_state_exec;
1649                                     end
1650                                     ENABLE_IRQ && EN-
ABLE_IRQ_QREGS && instr_getq: begin
1651                                     `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1);)
1652                                     reg_out <=
cpuregs_rs1;
1653                                     dbg_rsival <=
cpuregs_rs1;
1654                                     dbg_rsiv-
al_valid <= 1;
1655                                     latched_store
<= 1;
1656                                     cpu_state <=
cpu_state_fetch;
1657                                     end
1658                                     ENABLE_IRQ && EN-
ABLE_IRQ_QREGS && instr_setq: begin
1659                                     `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1);)
1660                                     reg_out <=
cpuregs_rs1;
1661                                     dbg_rsival <=
cpuregs_rs1;
1662                                     dbg_rsiv-
al_valid <= 1;
1663                                     latched_rd <=
latched_rd | irqregs_offset;
1664                                     latched_store
<= 1;
1665                                     cpu_state <=
cpu_state_fetch;
1666                                     end
1667                                     ENABLE_IRQ && in-
str_retirq: begin
1668                                     eoi <= 0;
1669                                     irq_active <=
0;
1670                                     latched_branch
<= 1;
1671                                     latched_store

```

```

1672                                     `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1));
1673                                     reg_out <=
CATCH_MISALIGN ? (cpuregs_rs1 & 32'h ffffffff) : cpuregs_rs1;
1674                                     dbg_rs1val <=
cpuregs_rs1;
1675                                     dbg_rs1v-
al_valid <= 1;
1676                                     cpu_state <=
cpu_state_fetch;
1677                                     end
1678                                     ENABLE_IRQ && in-
str_maskirq: begin
1679                                     latched_store
<= 1;
1680                                     reg_out <=
irq_mask;
1681                                     `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1));
1682                                     irq_mask <=
cpuregs_rs1 | MASKED_IRQ;
1683                                     dbg_rs1val <=
cpuregs_rs1;
1684                                     dbg_rs1v-
al_valid <= 1;
1685                                     cpu_state <=
cpu_state_fetch;
1686                                     end
1687                                     ENABLE_IRQ && EN-
ABLE_IRQ_TIMER && instr_timer: begin
1688                                     latched_store
<= 1;
1689                                     reg_out <=
timer;
1690                                     `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1));
1691                                     timer <=
cpuregs_rs1;
1692                                     dbg_rs1val <=
cpuregs_rs1;
1693                                     dbg_rs1v-
al_valid <= 1;
1694                                     cpu_state <=
cpu_state_fetch;
1695                                     end
1696                                     is_lb_lh_lw_lbu_lhu
&& !instr_trap: begin
1697                                     `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1));
1698                                     reg_op1 <=
cpuregs_rs1;
1699                                     dbg_rs1val <=
cpuregs_rs1;
1700                                     dbg_rs1v-
al_valid <= 1;
1701                                     cpu_state <=
cpu_state_ldmem;
1702                                     mem_do_rinst <=
1;
1703                                     end
1704                                     is_slli_srli_srai && !
BARREL_SHIFTER: begin
1705                                     `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1));
1706                                     reg_op1 <=
cpuregs_rs1;
1707                                     dbg_rs1val <=
cpuregs_rs1;
1708                                     dbg_rs1v-
al_valid <= 1;
1709                                     reg_sh <= de-
coded_rs2;
1710                                     cpu_state <=
cpu_state_shift;
1711                                     end
1712                                     is_jalr_addi_slti_sltiu_xori_ori_andi, is_slli_srli_srai && BAR-
REL_SHIFTER: begin
1713                                     `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1));
1714                                     reg_op1 <=
cpuregs_rs1;
1715                                     dbg_rs1val <=
cpuregs_rs1;
1716                                     dbg_rs1v-
al_valid <= 1;
1717                                     reg_op2 <=
is_slli_srli_srai && BARREL_SHIFTER ? decoded_rs2 : decoded_imm;
1718                                     if
(TWO_CYCLE_ALU)
1719                                     alu_wait <= 1;
1720                                     else
1721                                     mem_do_rinst <= mem_do_prefetch;
1722                                     cpu_state <=
cpu_state_exec;
1723                                     end
1724                                     default: begin

```

```

1725 `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1));
1726 reg_op1 <=
cpuregs_rs1;
1727 dbg_rs1val <=
cpuregs_rs1;
1728 dbg_rs1v-
al_valid <= 1;
1729 if (EN-
ABLE_REGS_DUALPORT) begin
1730 `de-
bug($display("LD_RS2: %2d 0x%08x", decoded_rs2, cpuregs_rs2));
1731 reg_sh
<= cpuregs_rs2;
1732 reg_op2 <= cpuregs_rs2;
1733 dbg_r-
s2val <= cpuregs_rs2;
1734 dbg_r-
s2val_valid <= 1;
1735 (*
parallel_case *)
1736 case
(1'b1)
1737 is_sb_sh_sw: begin
1738 cpu_state <= cpu_state_stmem;
1739 mem_do_rinst <= 1;
1740 end
1741 is_sll_srl_sra && !BARREL_SHIFTER: begin
1742 cpu_state <= cpu_state_shift;
1743 end
1744 default: begin
1745 if (TWO_CYCLE_ALU || (TWO_CYCLE_COMPARE &&
is_beq_bne_blt_bge_bltu_bgeu)) begin
1746 alu_wait_2 <= TWO_CYCLE_ALU && (TWO_CYCLE_COMPARE &&
is_beq_bne_blt_bge_bltu_bgeu);
1747 alu_wait <= 1;
1748 end else
1749 mem_do_rinst <= mem_do_prefetch;
1750 cpu_state <= cpu_state_exec;
1751 end
1752 end-
case
1753 end else
1754 cpu_state <= cpu_state_ld_rs2;
1755 end
1756 endcase
1757 end
1758 cpu_state_ld_rs2: begin
1759 `debug($display("LD_RS2: %2d
0x%08x", decoded_rs2, cpuregs_rs2));
1760 reg_sh <= cpuregs_rs2;
1761 reg_op2 <= cpuregs_rs2;
1762 dbg_rs2val <= cpuregs_rs2;
1763 dbg_rs2val_valid <= 1;
1764 (* parallel_case *)
1765 case (1'b1)
1766 WITH_PCPI && in-
str_trap: begin
1769 pcpi_valid <=
1;
1770 (pcpi_int_ready) begin
1771 mem_do_rinst <= 1;
1772 pcpi_valid <= 0;

```

```

1773 reg_out <= pcpi_int_rd;
1774 latched_store <= pcpi_int_wr;
1775 cpu_state <= cpu_state_fetch;
1776 end else
1777 if
(CATCH_ILLINSN && (pcpi_timeout || instr_ecall_ebreak)) begin
1778 pcpi_valid <= 0;
1779 `de-
bug($display("EBREAK OR UNSUPPORTED INSN AT 0x%08x", reg_pc));
1780 if
(ENABLE_IRQ && !irq_mask[irq_ebreak] && !irq_active) begin
1781 next_irq_pending[irq_ebreak] = 1;
1782 cpu_state <= cpu_state_fetch;
1783 end
1784 else
1785 cpu_state <= cpu_state_trap;
1786 end
1787 is_sb_sh_sw: begin
1788 cpu_state <=
mem_do_rinst <=
1;
1789 end
1790 is_sll_srl_sra && !BAR-
REL_SHIFTER: begin
1791 cpu_state <=
cpu_state_shift;
1792 end
1793 default: begin
1794 if
(TWO_CYCLE_ALU || (TWO_CYCLE_COMPARE &&
is_beq_bne_blt_bge_bltu_bgeu)) begin
1795 alu_wait_2 <= TWO_CYCLE_ALU && (TWO_CYCLE_COMPARE &&
is_beq_bne_blt_bge_bltu_bgeu);
1796 alu_wait <= 1;
1797 end else
1798 mem_do_rinst <= mem_do_prefetch;
1799 cpu_state <=
cpu_state_exec;
1800 end
1801 endcase
1802 end
1803 end
1804 cpu_state_exec: begin
1805 reg_out <= reg_pc + de-
coded_imm;
1806 if ((TWO_CYCLE_ALU ||
TWO_CYCLE_COMPARE) && (alu_wait || alu_wait_2)) begin
1807 mem_do_rinst <=
alu_wait <= alu_wait_2;
1808 end else
1809 if
(is_beq_bne_blt_bge_bltu_bgeu) begin
1810 latched_rd <= 0;
1811 latched_store <=
TWO_CYCLE_COMPARE ? alu_out_0_q : alu_out_0;
1812 latched_branch <=
TWO_CYCLE_COMPARE ? alu_out_0_q : alu_out_0;
1813 if (mem_done)
1814 cpu_state <=
1815 if (TWO_CYCLE_COMPARE ?
alu_out_0_q : alu_out_0) begin
1816 decoder_trigger
<= 0;
1817 set_mem_do_rinst = 1;
1818 end
1819 end else begin
1820 latched_branch <= in-
str_jalr;
1821 latched_store <= 1;
1822 latched_stalu <= 1;
1823 cpu_state <=
1824 end
1825 end
1826 end
1827 end
1828 cpu_state_shift: begin
1829 latched_store <= 1;
1830 if (reg_sh == 0) begin
1831 reg_out <= reg_op1;
1832 mem_do_rinst <=

```

```

mem_do_prefetch;
1834          cpu_state <=
cpu_state_fetch;

1835          end else if (TWO_STAGE_SHIFT &&
reg_sh >= 4) begin
1836              (* parallel_case,
full_case *)
1837              case (1'b1)
1838                  instr_slli ||
1839                  instr_srli ||
1840                  instr_srai ||
1841                  endcase
1842              reg_sh <= reg_sh - 4;
1843          end else begin
1844              (* parallel_case,
full_case *)
1845              case (1'b1)
1846                  instr_slli ||
1847                  instr_srli ||
1848                  instr_srai ||
1849                  endcase
1850              reg_sh <= reg_sh - 1;
1851          end
1852      end
1853      cpu_state_stmem: begin
1854          if (ENABLE_TRACE)
1855              reg_out <= reg_op2;
1856          if (!mem_do_prefetch ||
mem_done) begin
1857              if (!mem_do_wdata) be-
gin
1858                  (* paral-
lel_case, full_case *)
1859                  case (1'b1)
1860                      in-
1861                      in-
1862                      in-
1863                      endcase
1864                  if (EN-
ABLE_TRACE) begin
1865                      trace_valid <= 1;
1866                      trace_data <= (irq_active ? TRACE_IRQ : 0) | TRACE_ADDR | ((reg_op1
+ decoded_imm) & 32'hffffffff);
1867                      end
1868                      reg_op1 <=
reg_op1 + decoded_imm;
1869                      set_mem_do_wdata = 1;
1870                      end
1871                  if (!mem_do_prefetch &&
mem_done) begin
1872                      cpu_state <=
cpu_state_fetch;
1873                      decoder_trigger
<= 1;
1874                      coder_pseudo_trigger <= 1;
1875                      end
1876                      end
1877                      end
1878                      end
1879                      cpu_state_ldmem: begin
1880                          latched_store <= 1;
1881                          if (!mem_do_prefetch ||
mem_done) begin
1882                              if (!mem_do_rdata) be-
gin
1883                                  (* paral-
lel_case, full_case *)
1884                                  case (1'b1)
1885                                      in-
1886                                      in-
1887                                      in-
1888                                      endcase
1889                                  latched_is_lu
<= is_lbu_lhu_lw;
1890                                  latched_is_lh
<= instr_lh;
1891                                  latched_is_lb
<= instr_lb;
1892                                  if (EN-
ABLE_TRACE) begin
1893

```

```

trace_valid <= 1;
1895          trace_data <= (irq_active ? TRACE_IRQ : 0) | TRACE_ADDR | ((reg_op1
+ decoded_imm) & 32'hffffffff);
1896          end
1897          reg_op1 <=
reg_op1 + decoded_imm;
1898          set_mem_do_rdata = 1;
1899          end
1900          if (!mem_do_prefetch &&
mem_done) begin
1901              (* paral-
lel_case, full_case *)
1902              case (1'b1)
1903                  latched_is_lu: reg_out <= mem_rdata_word;
1904                  latched_is_lh: reg_out <= $signed(mem_rdata_word[15:0]);
1905                  latched_is_lb: reg_out <= $signed(mem_rdata_word[7:0]);
1906                  endcase
1907                  decoder_trigger
<= 1;
1908                  de-
coder_pseudo_trigger <= 1;
1909                  cpu_state <=
cpu_state_fetch;
1910                  end
1911                  end
1912                  endcase
1913                  if (ENABLE_IRQ) begin
1914                      next_irq_pending = next_irq_pending |
irq;
1915                      if (ENABLE_IRQ_TIMER && timer)
1916                          if (timer - 1 == 0)
1917                              next_irq_pend-
ing[irq_timer] = 1;
1918                      end
1919                      if (CATCH_MISALIGN && reseth && (mem_do_rdata
|| mem_do_wdata)) begin
1920                          if (mem_wordsize == 0 && reg_op1[1:0] !
= 0) begin
1921                              `debug($display("MISALIGNED
WORD: 0x%08x", reg_op1));
1922                          if (ENABLE_IRQ && !
irq_mask[irq_buserror] && !irq_active) begin
1923                              next_irq_pend-
ing[irq_buserror] = 1;
1924                          end else
1925                              cpu_state <=
cpu_state_trap;
1926                          end
1927                          if (mem_wordsize == 1 && reg_op1[0] !=
0) begin
1928                              `debug($display("MISALIGNED
HALFWORD: 0x%08x", reg_op1));
1929                          if (ENABLE_IRQ && !
irq_mask[irq_buserror] && !irq_active) begin
1930                              next_irq_pend-
ing[irq_buserror] = 1;
1931                          end else
1932                              cpu_state <=
cpu_state_trap;
1933                          end
1934                          end
1935                          if (CATCH_MISALIGN && reseth && mem_do_rinst &&
(COMPRESSED_ISA ? reg_pc[0] : |reg_pc[1:0])) begin
1936                              `debug($display("MISALIGNED INSTRUC-
TION: 0x%08x", reg_pc));
1937                          if (ENABLE_IRQ && !
irq_mask[irq_buserror] && !irq_active) begin
1938                              next_irq_pending[irq_buserror]
= 1;
1939                          end else
1940                              cpu_state <= cpu_state_trap;
1941                          end
1942                          if (!CATCH_ILLINSN && decoder_trigger_q && !de-
coder_pseudo_trigger_q && instr_ecall_ebreak) begin
1943                              cpu_state <= cpu_state_trap;
1944                          end
1945                          if (!reseth || mem_done) begin
1946                              mem_do_prefetch <= 0;
1947                              mem_do_rinst <= 0;
1948                              mem_do_rdata <= 0;
1949                              mem_do_wdata <= 0;
1950                          end
1951                          if (set_mem_do_rinst)
1952                              mem_do_rinst <= 1;
1953                          if (set_mem_do_rdata)
1954                              mem_do_rdata <= 1;
1955                          if (set_mem_do_wdata)
1956                              mem_do_wdata <= 1;
1957                      end

```



```

1962         irq_pending <= next_irq_pending & ~MASKED_IRQ;
1963     if (!CATCH_MISALIGN) begin
1964         if (COMPRESSED_ISA) begin
1965             reg_pc[0] <= 0;
1966             reg_next_pc[0] <= 0;
1967         end else begin
1968             reg_pc[1:0] <= 0;
1969             reg_next_pc[1:0] <= 0;
1970         end
1971     end
1972     end
1973     current_pc = 'bx;
1974 end
1975
1976 `ifdef RISC_V_FORMAL
1977     reg_dbg_irq_call;
1978     reg_dbg_irq_enter;
1979     reg [31:0] dbg_irq_ret;
1980     always @(posedge clk) begin
1981         rvfi_valid <= resetn && (launch_next_insn ||
1982 trap) && dbg_valid_insn;
1983         rvfi_order <= resetn ? rvfi_order + rvfi_val-
1984 id : 0;
1985         rvfi_insn <= dbg_insn_opcode;
1986         rvfi_rs1_addr <= dbg_rs1val_valid ? dbg_in-
1987 sn_rs1 : 0;
1988         rvfi_rs2_addr <= dbg_rs2val_valid ? dbg_in-
1989 sn_rs2 : 0;
1990         rvfi_pc_rdata <= dbg_insn_addr;
1991         rvfi_rs1_rdata <= dbg_rs1val_valid ? dbg_rs1v-
1992 al : 0;
1993         rvfi_rs2_rdata <= dbg_rs2val_valid ? dbg_rs2v-
1994 al : 0;
1995         rvfi_trap <= trap;
1996         rvfi_halt <= trap;
1997         rvfi_intr <= dbg_irq_enter;
1998         rvfi_mode <= 3;
1999         rvfi_ixl <= 1;
2000     if (!resetn) begin
2001         dbg_irq_call <= 0;
2002         dbg_irq_enter <= 0;
2003     end else
2004     if (rvfi_valid) begin
2005         dbg_irq_call <= 0;
2006         dbg_irq_enter <= dbg_irq_call;
2007     end else
2008     if (irq_state == 1) begin
2009         dbg_irq_call <= 1;
2010         dbg_irq_ret <= next_pc;
2011     end
2012     if (!resetn) begin
2013         rvfi_rd_addr <= 0;
2014         rvfi_rd_wdata <= 0;
2015     end else
2016     if (cpuregs_write && !irq_state) begin
2017         rvfi_rd_addr <= latched_rd ^ 1;
2018     else
2019         rvfi_rd_addr <= latched_rd;
2020     `endif
2021     rvfi_rd_wdata <= latched_rd ?
2022 cpuregs_wrddata ^ 1 : 0;
2023     else
2024         rvfi_rd_wdata <= latched_rd ?
2025 cpuregs_wrddata : 0;
2026     `endif
2027     end else
2028     if (rvfi_valid) begin
2029         rvfi_rd_addr <= 0;
2030         rvfi_rd_wdata <= 0;
2031     end
2032     casez (dbg_insn_opcode)
2033         32'b 0000000_????_000??_????_????
2034 _0001011: begin // getq
2035             rvfi_rs1_addr <= 0;
2036             rvfi_rs1_rdata <= 0;
2037         end
2038         32'b 0000001_????_????_???_000??
2039 _0001011: begin // setq
2040             rvfi_rd_addr <= 0;
2041             rvfi_rd_wdata <= 0;
2042         end
2043         32'b 0000010_????_00000_???
2044 _00000_0001011: begin // retirq
2045             rvfi_rs1_addr <= 0;
2046             rvfi_rs1_rdata <= 0;
2047         end
2048     endcase
2049     if (!dbg_irq_call) begin
2050         if (dbg_mem_instr) begin
2051             rvfi_mem_addr <= 0;
2052             rvfi_mem_rmask <= 0;
2053             rvfi_mem_wmask <= 0;
2054             rvfi_mem_rdata <= 0;

```

```

2055             rvfi_mem_wdata <= 0;
2056         end else
2057         if (dbg_mem_valid && dbg_mem_ready) be-
2058 gin
2059             rvfi_mem_addr <= dbg_mem_addr;
2060             rvfi_mem_rmask <=
2061 dbg_mem_wstrb ? 0 : ~0;
2062             rvfi_mem_wmask <=
2063 dbg_mem_wstrb;
2064             rvfi_mem_rdata <=
2065 dbg_mem_rdata;
2066             rvfi_mem_wdata <=
2067 dbg_mem_wdata;
2068         end
2069     end
2070     always @* begin
2071         `ifdef PICORV32_TESTBUG_005
2072             rvfi_pc_wdata = (dbg_irq_call ? dbg_irq_ret :
2073 dbg_insn_addr) ^ 4;
2074         `else
2075             rvfi_pc_wdata = dbg_irq_call ? dbg_irq_ret :
2076 dbg_insn_addr;
2077         `endif
2078         rvfi_csr_mcycle_rmask = 0;
2079         rvfi_csr_mcycle_wmask = 0;
2080         rvfi_csr_mcycle_rdata = 0;
2081         rvfi_csr_mcycle_wdata = 0;
2082         rvfi_csr_minstret_rmask = 0;
2083         rvfi_csr_minstret_wmask = 0;
2084         rvfi_csr_minstret_rdata = 0;
2085         rvfi_csr_minstret_wdata = 0;
2086     if (rvfi_valid && rvfi_insn[6:0] == 7'b 1110011
2087 && rvfi_insn[13:12] == 3'b 010) begin
2088         if (rvfi_insn[31:20] == 12'h C00) begin
2089             rvfi_csr_mcycle_rmask = 64'h
2090 0000_0000_FFFF_FFFF;
2091             rvfi_csr_mcycle_rdata = {32'h
2092 0000_0000, rvfi_rd_wdata};
2093         end
2094         if (rvfi_insn[31:20] == 12'h C80) begin
2095             rvfi_csr_mcycle_rmask = 64'h
2096 FFFF_FFFF_0000_0000;
2097             rvfi_csr_mcycle_rdata =
2098 {rvfi_rd_wdata, 32'h 0000_0000};
2099         end
2100         if (rvfi_insn[31:20] == 12'h C02) begin
2101             rvfi_csr_minstret_rmask = 64'h
2102 0000_0000_FFFF_FFFF;
2103             rvfi_csr_minstret_rdata = {32'h
2104 0000_0000, rvfi_rd_wdata};
2105         end
2106         if (rvfi_insn[31:20] == 12'h C82) begin
2107             rvfi_csr_minstret_rmask = 64'h
2108 FFFF_FFFF_0000_0000;
2109             rvfi_csr_minstret_rdata =
2110 {rvfi_rd_wdata, 32'h 0000_0000};
2111         end
2112     end
2113     `endif
2114     // Formal Verification
2115     `ifdef FORMAL
2116         reg [3:0] last_mem_nowait;
2117         always @(posedge clk)
2118             last_mem_nowait <= {last_mem_nowait, mem_ready
2119 || !mem_valid};
2120     end
2121     // stall the memory interface for max 4 cycles
2122     restrict property (!last_mem_nowait || mem_ready || !
2123 mem_valid);
2124     // resetn low in first cycle, after that resetn high
2125     restrict property (resetn != $initstate);
2126     // this just makes it much easier to read traces. un-
2127     comment as needed.
2128     // assume property (mem_valid || !mem_ready);
2129     reg ok;
2130     always @* begin
2131         if (resetn) begin
2132             // instruction fetches are read-only
2133             if (mem_valid && mem_instr)
2134                 assert (mem_wstrb == 0);
2135             // cpu_state must be valid
2136             ok = 0;
2137         if (cpu_state == cpu_state_trap) ok =
2138 1;
2139         if (cpu_state == cpu_state_fetch) ok =
2140 1;

```



```

2128             if (cpu_state == cpu_state_ld_rs1) ok =
1;
2129             if (cpu_state == cpu_state_ld_rs2) ok
= !ENABLE_REGS_DUALPORT;

2130             if (cpu_state == cpu_state_exec) ok =
1;

2131             if (cpu_state == cpu_state_shift) ok =
1;

2132             if (cpu_state == cpu_state_stmem) ok =
1;

2133             if (cpu_state == cpu_state_ldmem) ok =
1;
2134             assert (ok);
2135         end
2136     end
2137
2138     reg last_mem_la_read = 0;
2139     reg last_mem_la_write = 0;
2140     reg [31:0] last_mem_la_addr;
2141     reg [31:0] last_mem_la_wdata;
2142     reg [3:0] last_mem_la_wstrb = 0;
2143
2144     always @(posedge clk) begin
2145         last_mem_la_read <= mem_la_read;
2146         last_mem_la_write <= mem_la_write;
2147         last_mem_la_addr <= mem_la_addr;
2148         last_mem_la_wdata <= mem_la_wdata;
2149         last_mem_la_wstrb <= mem_la_wstrb;
2150
2151         if (last_mem_la_read) begin
2152             assert(mem_valid);
2153             assert(mem_addr == last_mem_la_addr);
2154             assert(mem_wstrb == 0);
2155         end
2156         if (last_mem_la_write) begin
2157             assert(mem_valid);
2158             assert(mem_addr == last_mem_la_addr);
2159             assert(mem_wdata == last_mem_la_wdata);
2160             assert(mem_wstrb == last_mem_la_wstrb);
2161         end
2162         if (mem_la_read || mem_la_write) begin
2163             assert(!mem_valid || mem_ready);
2164         end
2165     end
2166 `endif
2167 endmodule
2168
2169 // This is a simple example implementation of PICORV32_REGS.
2170 // Use the PICORV32_REGS mechanism if you want to use custom
2171 // memory resources to implement the processor register file.
2172 // Note that your implementation must match the requirements
of
2173 // the PicoRV32 configuration. (e.g. QREGS, etc)
2174 module picorv32_regs (
2175     input clk, wen,
2176     input [5:0] waddr,
2177     input [5:0] raddr1,
2178     input [5:0] raddr2,
2179     input [31:0] wdata,
2180     output [31:0] rdata1,
2181     output [31:0] rdata2
2182 );
2183     reg [31:0] regs [0:30];
2184
2185     always @(posedge clk)
2186         if (wen) regs[waddr[4:0]] <= wdata;
2187
2188     assign rdata1 = regs[raddr1[4:0]];
2189     assign rdata2 = regs[raddr2[4:0]];
2190 endmodule
2191
2192 /
2193 *
2194 * picorv32_pcpi_mul
2195 *
2196 *****/
2197 module picorv32_pcpi_mul #(
2198     parameter STEPS_AT_ONCE = 1,
2199     parameter CARRY_CHAIN = 4
2200 ) (
2201     input clk, resetn,
2202
2203     input pcpi_valid,
2204     input [31:0] pcpi_insn,
2205     input [31:0] pcpi_rs1,
2206     input [31:0] pcpi_rs2,
2207     output reg pcpi_wr,
2208     output reg [31:0] pcpi_rd,
2209     output reg pcpi_wait,
2210     output reg pcpi_ready
2211 );
2212     reg instr_mul, instr_mulh, instr_mulhsu, instr_mulhu;
2213     wire instr_any_mul = |{instr_mul, instr_mulh, in-

```

```

2214     str_mulhu};
2215     wire instr_rs1_signed = |{instr_mulh, instr_mulhsu};
2216     wire instr_rs2_signed = |{instr_mulh};
2217
2218     reg pcpi_wait_q;
2219     wire mul_start = pcpi_wait && !pcpi_wait_q;
2220
2221     always @(posedge clk) begin
2222         instr_mul <= 0;
2223         instr_mulh <= 0;
2224         instr_mulhsu <= 0;
2225         instr_mulhu <= 0;
2226
2227         if (resetn && pcpi_valid && pcpi_insn[6:0] ==
7'b0110011 && pcpi_insn[31:25] == 7'b0000001) begin
2228             case (pcpi_insn[14:12])
2229                 3'b000: instr_mul <= 1;
2230                 3'b001: instr_mulh <= 1;
2231                 3'b010: instr_mulhsu <= 1;
2232                 3'b011: instr_mulhu <= 1;
2233             endcase
2234         end
2235
2236         pcpi_wait <= instr_any_mul;
2237         pcpi_wait_q <= pcpi_wait;
2238     end
2239
2240     reg [63:0] rs1, rs2, rd, rdx;
2241     reg [63:0] next_rs1, next_rs2, this_rs2;
2242     reg [63:0] next_rdx, next_rdx, next_rdt;
2243     reg [6:0] mul_counter;
2244     reg mul_waiting;
2245     reg mul_finish;
2246     integer i, j;
2247
2248     // carry save accumulator
2249     always @* begin
2250         next_rd = rd;
2251         next_rdx = rdx;
2252         next_rs1 = rs1;
2253         next_rs2 = rs2;
2254
2255         for (i = 0; i < STEPS_AT_ONCE; i=i+1) begin
2256             this_rs2 = next_rs1[0] ? next_rs2 : 0;
2257             if (CARRY_CHAIN == 0) begin
2258                 next_rdt = next_rd ^ next_rdx ^
this_rs2;
2259                 next_rdx = ((next_rd &
next_rdx) | (next_rd & this_rs2) | (next_rdx & this_rs2)) << 1;
2260                 next_rd = next_rdt;
2261             end else begin
2262                 next_rdt = 0;
2263                 for (j = 0; j < 64; j = j +
CARRY_CHAIN)
2264                     {next_rdt[j+CARRY_CHAIN-1], next_rd[j +: CARRY_CHAIN]} =
next_rd[j +: CARRY_CHAIN] + next_rdx[j +: CARRY_CHAIN] + this_rs2[j
+: CARRY_CHAIN];
2265                 next_rdx = next_rdt << 1;
2266             end
2267             next_rs1 = next_rs1 >> 1;
2268             next_rs2 = next_rs2 << 1;
2269         end
2270     end
2271
2272     always @(posedge clk) begin
2273         mul_finish <= 0;
2274         if (!resetn) begin
2275             mul_waiting <= 1;
2276         end else
2277             if (mul_waiting) begin
2278                 if (instr_rs1_signed)
2279                     rs1 <= $signed(pcpi_rs1);
2280                 else
2281                     rs1 <= $unsigned(pcpi_rs1);
2282
2283                 if (instr_rs2_signed)
2284                     rs2 <= $signed(pcpi_rs2);
2285                 else
2286                     rs2 <= $unsigned(pcpi_rs2);
2287
2288                 rd <= 0;
2289                 rdx <= 0;
2290                 mul_counter <= (instr_any_mulh ? 63 -
STEPS_AT_ONCE : 31 - STEPS_AT_ONCE);
2291                 mul_waiting <= !mul_start;
2292             end else begin
2293                 rd <= next_rd;
2294                 rdx <= next_rdx;
2295                 rs1 <= next_rs1;
2296                 rs2 <= next_rs2;
2297
2298                 mul_counter <= mul_counter -
STEPS_AT_ONCE;
2299                 if (mul_counter[6]) begin
2300                     mul_finish <= 1;
2301                     mul_waiting <= 1;
2302                 end
2303             end

```

```

2304         end
2305     end
2306
2307     always @(posedge clk) begin
2308         pcpi_wr <= 0;
2309         pcpi_ready <= 0;
2310         if (mul_finish && resetn) begin
2311             pcpi_wr <= 1;
2312             pcpi_ready <= 1;
2313             pcpi_rd <= instr_any_mulh ? rd >> 32 :
rd;
2314         end
2315     end
2316 endmodule
2317
2318 module picorv32_pcpi_fast_mul #(
2319     parameter EXTRA_MUL_FFS = 0,
2320     parameter EXTRA_INSN_FFS = 0,
2321     parameter MUL_CLKGATE = 0
2322 ) (
2323     input clk, resetn,
2324
2325     input          pcpi_valid,
2326     input [31:0]  pcpi_insn,
2327     input [31:0]  pcpi_rs1,
2328     input [31:0]  pcpi_rs2,
2329     output         pcpi_wr,
2330     output [31:0] pcpi_rd,
2331     output         pcpi_wait,
2332     output         pcpi_ready
2333 );
2334     reg instr_mul, instr_mulh, instr_mulhsu, instr_mulhu;
2335     wire instr_any_mul = |{instr_mul, instr_mulh, in-
str_mulhsu, instr_mulhu};
2336     wire instr_any_mulh = |{instr_mulh, instr_mulhsu, in-
str_mulhu};
2337     wire instr_rs1_signed = |{instr_mulh, instr_mulhsu};
2338     wire instr_rs2_signed = |{instr_mulh};
2339
2340     reg shift_out;
2341     reg [3:0] active;
2342     reg [32:0] rs1, rs2, rs1_q, rs2_q;
2343     reg [63:0] rd, rd_q;
2344
2345     wire pcpi_insn_valid = pcpi_valid && pcpi_insn[6:0] ==
7'b0110011 && pcpi_insn[31:25] == 7'b0000001;
2346     reg pcpi_insn_valid_q;
2347
2348     always @* begin
2349         instr_mul = 0;
2350         instr_mulh = 0;
2351         instr_mulhsu = 0;
2352         instr_mulhu = 0;
2353
2354         if (resetn && (EXTRA_INSN_FFS ? pcpi_in-
sn_valid_q : pcpi_insn_valid)) begin
2355             case (pcpi_insn[14:12])
2356                 3'b000: instr_mul = 1;
2357                 3'b001: instr_mulh = 1;
2358                 3'b010: instr_mulhsu = 1;
2359                 3'b011: instr_mulhu = 1;
2360             endcase
2361         end
2362     end
2363
2364     always @(posedge clk) begin
2365         pcpi_insn_valid_q <= pcpi_insn_valid;
2366         if (!MUL_CLKGATE || active[0]) begin
2367             rs1_q <= rs1;
2368             rs2_q <= rs2;
2369         end
2370         if (!MUL_CLKGATE || active[1]) begin
2371             rd <= $signed(EXTRA_MUL_FFS ? rs1_q :
rs1) * $signed(EXTRA_MUL_FFS ? rs2_q : rs2);
2372         end
2373         if (!MUL_CLKGATE || active[2]) begin
2374             rd_q <= rd;
2375         end
2376     end
2377
2378     always @(posedge clk) begin
2379         if (instr_any_mul && !(EXTRA_MUL_FFS ? act-
ive[3:0] : active[1:0])) begin
2380             if (instr_rs1_signed)
2381                 rs1 <= $signed(pcpi_rs1);
2382             else
2383                 rs1 <= $unsigned(pcpi_rs1);
2384
2385             if (instr_rs2_signed)
2386                 rs2 <= $signed(pcpi_rs2);
2387             else
2388                 rs2 <= $unsigned(pcpi_rs2);
2389             active[0] <= 1;
2390         end else begin
2391             active[0] <= 0;
2392         end
2393
2394         active[3:1] <= active;
2395         shift_out <= instr_any_mulh;
2396
2397         if (!resetn)

```

```

2398         active <= 0;
2399     end
2400
2401     assign pcpi_wr = active[EXTRA_MUL_FFS ? 3 : 1];
2402     assign pcpi_wait = 0;
2403     assign pcpi_ready = active[EXTRA_MUL_FFS ? 3 : 1];
2404     `ifdef RISC_V_FORMAL_ALTOPS
2405         assign pcpi_rd =
2406             instr_mul    ? (pcpi_rs1 + pcpi_rs2) ^
32'h5876063e :
2407             instr_mulh   ? (pcpi_rs1 + pcpi_rs2) ^
32'hf6583fb7 :
2408             instr_mulhsu ? (pcpi_rs1 - pcpi_rs2) ^
32'hecfbc137 :
2409             instr_mulhu  ? (pcpi_rs1 + pcpi_rs2) ^
32'h949ce5e8 : 1'bxx;
2410     `else
2411         assign pcpi_rd = shift_out ? (EXTRA_MUL_FFS ? rd_q :
rd) >> 32 : (EXTRA_MUL_FFS ? rd_q : rd);
2412     `endif
2413 endmodule
2414
2415
2416 /
*****
2417 * picorv32_pcpi_div
2418 *****/
2419
2420 module picorv32_pcpi_div (
2421     input clk, resetn,
2422
2423     input          pcpi_valid,
2424     input [31:0]  pcpi_insn,
2425     input [31:0]  pcpi_rs1,
2426     input [31:0]  pcpi_rs2,
2427     output reg     pcpi_wr,
2428     output reg [31:0] pcpi_rd,
2429     output reg     pcpi_wait,
2430     output reg     pcpi_ready
2431 );
2432     reg instr_div, instr_divu, instr_rem, instr_remu;
2433     wire instr_any_div_rem = |{instr_div, instr_divu, in-
str_rem, instr_remu};
2434
2435     reg pcpi_wait_q;
2436     wire start = pcpi_wait && !pcpi_wait_q;
2437
2438     always @(posedge clk) begin
2439         instr_div <= 0;
2440         instr_divu <= 0;
2441         instr_rem <= 0;
2442         instr_remu <= 0;
2443
2444         if (resetn && pcpi_valid && !pcpi_ready &&
pcpi_insn[6:0] == 7'b0110011 && pcpi_insn[31:25] == 7'b0000001) be-
gin
2445             case (pcpi_insn[14:12])
2446                 3'b100: instr_div <= 1;
2447                 3'b101: instr_divu <= 1;
2448                 3'b110: instr_rem <= 1;
2449                 3'b111: instr_remu <= 1;
2450             endcase
2451         end
2452
2453         pcpi_wait <= instr_any_div_rem && resetn;
2454         pcpi_wait_q <= pcpi_wait && resetn;
2455     end
2456
2457     reg [31:0] dividend;
2458     reg [62:0] divisor;
2459     reg [31:0] quotient;
2460     reg [31:0] quotient_msk;
2461     reg running;
2462     reg outsign;
2463
2464     always @(posedge clk) begin
2465         pcpi_ready <= 0;
2466         pcpi_wr <= 0;
2467         pcpi_rd <= 'bx;
2468
2469         if (!resetn) begin
2470             running <= 0;
2471         end else
2472             if (start) begin
2473                 running <= 1;
2474
2475                 dividend <= (instr_div || instr_rem) &&
pcpi_rs1[31] ? -pcpi_rs1 : pcpi_rs1;
2476
2477                 divisor <= ((instr_div || instr_rem) &&
pcpi_rs2[31] ? -pcpi_rs2 : pcpi_rs2) << 31;
2478                 outsign <= (instr_div &&
(pcpi_rs1[31] != pcpi_rs2[31]) && !pcpi_rs2) || (instr_rem &&
pcpi_rs1[31]);
2479                 quotient <= 0;
2480                 quotient_msk <= 1 << 31;
2481             end else
2482                 if (!quotient_msk && running) begin
2483                     running <= 0;
2484                     pcpi_ready <= 1;

```

```

2483         pcpi_wr <= 1;
2484 `ifdef RISC_V Formal_ALTOPS
2485         case (1)
2486             instr_div: pcpi_rd <=
2487 (pcpi_rs1 - pcpi_rs2) ^ 32'h7f8529ec;
2488             instr_divu: pcpi_rd <=
2489 (pcpi_rs1 - pcpi_rs2) ^ 32'h10e8fd70;
2490             instr_rem: pcpi_rd <=
2491 (pcpi_rs1 - pcpi_rs2) ^ 32'h8da68fa5;
2492             instr_remu: pcpi_rd <=
2493 (pcpi_rs1 - pcpi_rs2) ^ 32'h3138d0e1;
2494         endcase
2495 `else
2496         if (instr_div || instr_divu)
2497             pcpi_rd <= outsign ? -quo-
2498             else
2499                 pcpi_rd <= outsign ? -di-
2500             end
2501         end else begin
2502             if (divisor <= dividend) begin
2503                 dividend <= dividend - divisor;
2504                 quotient <= quotient | quo-
2505             end
2506             divisor <= divisor >> 1;
2507         end
2508         quotient_msk <= quotient_msk >> 5;
2509         quotient_msk <= quotient_msk >> 1;
2510     end
2511 endmodule
2512
2513 /
2514 * picorv32_axi
2515
2516 ****
2517 module picorv32_axi #(
2518     parameter [0:0] ENABLE_COUNTERS = 1,
2519     parameter [0:0] ENABLE_COUNTERS64 = 1,
2520     parameter [0:0] ENABLE_REGS_16_31 = 1,
2521     parameter [0:0] ENABLE_REGS_DUALPORT = 1,
2522     parameter [0:0] TWO_STAGE_SHIFT = 1,
2523     parameter [0:0] BARREL_SHIFTER = 0,
2524     parameter [0:0] TWO_CYCLE_COMPARE = 0,
2525     parameter [0:0] TWO_CYCLE_ALU = 0,
2526     parameter [0:0] COMPRESSED_ISA = 0,
2527     parameter [0:0] CATCH_MISALIGN = 1,
2528     parameter [0:0] CATCH_ILLINSN = 1,
2529     parameter [0:0] ENABLE_PCPI = 0,
2530     parameter [0:0] ENABLE_MUL = 0,
2531     parameter [0:0] ENABLE_FAST_MUL = 0,
2532     parameter [0:0] ENABLE_DIV = 0,
2533     parameter [0:0] ENABLE_IRQ = 0,
2534     parameter [0:0] ENABLE_IRQ_QREGS = 1,
2535     parameter [0:0] ENABLE_IRQ_TIMER = 1,
2536     parameter [0:0] ENABLE_TRACE = 0,
2537     parameter [0:0] REGS_INIT_ZERO = 0,
2538     parameter [31:0] MASKED_IRQ = 32'h 0000_0000,
2539     parameter [31:0] LATCHED_IRQ = 32'h ffff_ffff,
2540     parameter [31:0] PROGADDR_RESET = 32'h 0000_0000,
2541     parameter [31:0] PROGADDR_IRQ = 32'h 0000_0010,
2542     parameter [31:0] STACKADDR = 32'h ffff_ffff
2543 ) (
2544     input clk, resetn,
2545     output trap,
2546
2547     // AXI4-lite master memory interface
2548
2549     output mem_axi_avalid,
2550     input mem_axi_awready,
2551     output [31:0] mem_axi_awaddr,
2552     output [2:0] mem_axi_awprot,
2553
2554     output mem_axi_wvalid,
2555     input mem_axi_wready,
2556     output [31:0] mem_axi_wdata,
2557     output [3:0] mem_axi_wstrb,
2558
2559     input mem_axi_bvalid,
2560     output mem_axi_bready,
2561
2562     output mem_axi_rvalid,
2563     input mem_axi_rready,
2564     output [31:0] mem_axi_rdata,
2565     output [2:0] mem_axi_rprot,
2566
2567     input mem_axi_rvalid,
2568     output mem_axi_rready,
2569     input [31:0] mem_axi_rdata,
2570
2571     // Pico Co-Processor Interface (PCPI)
2572     output pcpi_valid,
2573     output [31:0] pcpi_insn,
2574     output [31:0] pcpi_rs1,

```

```

2575     output [31:0] pcpi_rs2,
2576     input pcpi_wr,
2577     input [31:0] pcpi_rd,
2578     input pcpi_wait,
2579     input pcpi_ready,
2580
2581     // IRQ interface
2582     input [31:0] irq,
2583     output [31:0] eoi,
2584
2585 `ifdef RISC_V Formal
2586     output rvfi_valid,
2587     output [63:0] rvfi_order,
2588     output [31:0] rvfi_insn,
2589     output rvfi_trap,
2590     output rvfi_halt,
2591     output rvfi_intr,
2592     output [4:0] rvfi_rs1_addr,
2593     output [4:0] rvfi_rs2_addr,
2594     output [31:0] rvfi_rs1_rdata,
2595     output [31:0] rvfi_rs2_rdata,
2596     output [4:0] rvfi_rd_addr,
2597     output [31:0] rvfi_rd_wdata,
2598     output [31:0] rvfi_pc_rdata,
2599     output [31:0] rvfi_pc_wdata,
2600     output [31:0] rvfi_mem_addr,
2601     output [3:0] rvfi_mem_rmask,
2602     output [3:0] rvfi_mem_wmask,
2603     output [31:0] rvfi_mem_rdata,
2604     output [31:0] rvfi_mem_wdata,
2605 `endif
2606
2607 // Trace Interface
2608 output trace_valid,
2609 output [35:0] trace_data
2610 );
2611
2612 wire mem_valid;
2613 wire [31:0] mem_addr;
2614 wire [31:0] mem_wdata;
2615 wire [3:0] mem_wstrb;
2616 wire mem_instr;
2617 wire mem_ready;
2618 wire [31:0] mem_rdata;
2619
2620 picorv32_axi_adapter axi_adapter (
2621     .clk (clk),
2622     .resetn (resetn),
2623     .mem_axi_avalid (mem_axi_avalid),
2624     .mem_axi_awready (mem_axi_awready),
2625     .mem_axi_awaddr (mem_axi_awaddr),
2626     .mem_axi_awprot (mem_axi_awprot),
2627     .mem_axi_wvalid (mem_axi_wvalid),
2628     .mem_axi_wready (mem_axi_wready),
2629     .mem_axi_wdata (mem_axi_wdata),
2630     .mem_axi_wstrb (mem_axi_wstrb),
2631     .mem_axi_bvalid (mem_axi_bvalid),
2632     .mem_axi_bready (mem_axi_bready),
2633     .mem_axi_rvalid (mem_axi_rvalid),
2634     .mem_axi_rready (mem_axi_rready),
2635     .mem_axi_rdata (mem_axi_rdata),
2636     .mem_valid (mem_valid),
2637     .mem_instr (mem_instr),
2638     .mem_ready (mem_ready),
2639     .mem_addr (mem_addr),
2640     .mem_wdata (mem_wdata),
2641     .mem_wstrb (mem_wstrb),
2642     .mem_rdata (mem_rdata)
2643 );
2644
2645 picorv32 #(
2646     .ENABLE_COUNTERS (ENABLE_COUNTERS),
2647     .ENABLE_COUNTERS64 (ENABLE_COUNTERS64),
2648     .ENABLE_REGS_16_31 (ENABLE_REGS_16_31),
2649     .ENABLE_REGS_DUALPORT (ENABLE_REGS_DUALPORT),
2650     .TWO_STAGE_SHIFT (TWO_STAGE_SHIFT),
2651     .BARREL_SHIFTER (BARREL_SHIFTER),
2652     .TWO_CYCLE_COMPARE (TWO_CYCLE_COMPARE),
2653     .TWO_CYCLE_ALU (TWO_CYCLE_ALU),
2654     .COMPRESSED_ISA (COMPRESSED_ISA),
2655     .CATCH_MISALIGN (CATCH_MISALIGN),
2656     .CATCH_ILLINSN (CATCH_ILLINSN),
2657     .ENABLE_PCPI (ENABLE_PCPI),
2658     .ENABLE_MUL (ENABLE_MUL),
2659     .ENABLE_FAST_MUL (ENABLE_FAST_MUL),
2660     .ENABLE_DIV (ENABLE_DIV),
2661     .ENABLE_IRQ (ENABLE_IRQ),
2662     .ENABLE_IRQ_QREGS (ENABLE_IRQ_QREGS),
2663     .ENABLE_IRQ_TIMER (ENABLE_IRQ_TIMER),
2664     .ENABLE_TRACE (ENABLE_TRACE),
2665     .REGS_INIT_ZERO (REGS_INIT_ZERO),
2666     .MASKED_IRQ (MASKED_IRQ),
2667     .LATCHED_IRQ (LATCHED_IRQ),
2668     .PROGADDR_RESET (PROGADDR_RESET),
2669     .PROGADDR_IRQ (PROGADDR_IRQ),
2670     .STACKADDR (STACKADDR)
2671 ) picorv32_core (
2672     .clk (clk),

```

```

2676         .resetsn (resetsn),
2677         .trap     (trap ),
2678
2679         .mem_valid(mem_valid),
2680         .mem_addr (mem_addr ),
2681         .mem_wdata(mem_wdata),
2682         .mem_wstrb(mem_wstrb),
2683         .mem_instr(mem_instr),
2684         .mem_ready(mem_ready),
2685         .mem_rdata(mem_rdata),
2686
2687         .pcpi_valid(pcpi_valid),
2688         .pcpi_insn (pcpi_insn ),
2689         .pcpi_rs1  (pcpi_rs1  ),
2690         .pcpi_rs2  (pcpi_rs2  ),
2691         .pcpi_wr   (pcpi_wr   ),
2692         .pcpi_rd   (pcpi_rd   ),
2693         .pcpi_wait (pcpi_wait ),
2694         .pcpi_ready(pcpi_ready),
2695
2696         .irq(irq),
2697         .eoi(eoi),
2698
2699 `ifdef RISC_V_FORMAL
2700         .rvfi_valid (rvfi_valid ),
2701         .rvfi_order (rvfi_order ),
2702         .rvfi_insn  (rvfi_insn  ),
2703         .rvfi_trap  (rvfi_trap  ),
2704         .rvfi_halt  (rvfi_halt  ),
2705         .rvfi_intr  (rvfi_intr  ),
2706         .rvfi_rs1_addr (rvfi_rs1_addr ),
2707         .rvfi_rs2_addr (rvfi_rs2_addr ),
2708         .rvfi_rs1_rdata(rvfi_rs1_rdata),
2709         .rvfi_rs2_rdata(rvfi_rs2_rdata),
2710         .rvfi_rd_addr (rvfi_rd_addr ),
2711         .rvfi_rd_wdata (rvfi_rd_wdata ),
2712         .rvfi_pc_rdata (rvfi_pc_rdata ),
2713         .rvfi_pc_wdata (rvfi_pc_wdata ),
2714         .rvfi_mem_addr (rvfi_mem_addr ),
2715         .rvfi_mem_rmask(rvfi_mem_rmask),
2716         .rvfi_mem_wmask(rvfi_mem_wmask),
2717         .rvfi_mem_rdata(rvfi_mem_rdata),
2718         .rvfi_mem_wdata(rvfi_mem_wdata),
2719 `endif
2720
2721         .trace_valid(trace_valid),
2722         .trace_data (trace_data)
2723     );
2724 endmodule
2725
2726 /
2727 *****
2728 * picorv32_axi_adapter
2729 *****
2730 /
2731 module picorv32_axi_adapter (
2732     input clk, resetsn,
2733
2734     // AXI4-lite master memory interface
2735
2736     output mem_axi_awvalid,
2737     input  mem_axi_awready,
2738     output [31:0] mem_axi_awaddr,
2739     output [2:0] mem_axi_awprot,
2740
2741     output mem_axi_wvalid,
2742     input  mem_axi_wready,
2743     output [31:0] mem_axi_wdata,
2744     output [3:0] mem_axi_wstrb,
2745
2746     input  mem_axi_bvalid,
2747     output mem_axi_bready,
2748
2749     output mem_axi_arvalid,
2750     input  mem_axi_arready,
2751     output [31:0] mem_axi_araddr,
2752     output [2:0] mem_axi_arprot,
2753
2754     input  mem_axi_rvalid,
2755     output mem_axi_rready,
2756     input  [31:0] mem_axi_rdata,
2757
2758     // Native PicoRV32 memory interface
2759
2760     input  mem_valid,
2761     input  mem_instr,
2762     output mem_ready,
2763     input  [31:0] mem_addr,
2764     input  [31:0] mem_wdata,
2765     input  [3:0] mem_wstrb,
2766     output [31:0] mem_rdata
2767 );
2768     reg ack_awvalid;
2769     reg ack_arvalid;
2770     reg ack_wvalid;
2771     reg xfer_done;
2772
2773     assign mem_axi_awvalid = mem_valid && !mem_wstrb && !
ack_awvalid;
2774
2775     assign mem_axi_awaddr = mem_addr;
2776     assign mem_axi_awprot = 0;
2777
2778     assign mem_axi_arvalid = mem_valid && !mem_wstrb && !
ack_arvalid;
2779     assign mem_axi_araddr = mem_addr;
2780     assign mem_axi_arprot = mem_instr ? 3'b100 : 3'b000;
2781
2782     assign mem_axi_wvalid = mem_valid && !mem_wstrb && !
ack_wvalid;
2783     assign mem_axi_wdata = mem_wdata;
2784     assign mem_axi_wstrb = mem_wstrb;
2785
2786     assign mem_ready = mem_axi_bvalid || mem_axi_rvalid;
2787     assign mem_axi_bready = mem_valid && !mem_wstrb;
2788     assign mem_axi_rready = mem_valid && !mem_wstrb;
2789     assign mem_rdata = mem_axi_rdata;
2790
2791     always @(posedge clk) begin
2792         if (!resetsn) begin
2793             ack_awvalid <= 0;
2794             ack_arvalid <= 0;
2795             ack_wvalid <= 0;
2796             xfer_done <= 0;
2797         end else begin
2798             if (mem_axi_arready && mem_axi_arvalid)
2799                 ack_arvalid <= 1;
2800             if (mem_axi_wready && mem_axi_wvalid)
2801                 ack_wvalid <= 1;
2802             if (xfer_done || !mem_valid) begin
2803                 ack_awvalid <= 0;
2804                 ack_arvalid <= 0;
2805                 ack_wvalid <= 0;
2806             end
2807             xfer_done <= mem_axi_bready && mem_axi_bvalid;
2808         end
2809     end
2810 endmodule
2811
2812 *****
2813 * picorv32_wb
2814 *****
2815 /
2816 module picorv32_wb #(
2817     parameter [0:0] ENABLE_COUNTERS = 1,
2818     parameter [0:0] ENABLE_COUNTERS64 = 1,
2819     parameter [0:0] ENABLE_REGS_16_31 = 1,
2820     parameter [0:0] ENABLE_REGS_DUALPORT = 1,
2821     parameter [0:0] TWO_STAGE_SHIFT = 1,
2822     parameter [0:0] BARREL_SHIFTER = 0,
2823     parameter [0:0] TWO_CYCLE_COMPARE = 0,
2824     parameter [0:0] TWO_CYCLE_ALU = 0,
2825     parameter [0:0] COMPRESSED_ISA = 0,
2826     parameter [0:0] CATCH_MISALIGN = 1,
2827     parameter [0:0] CATCH_ILLINSN = 1,
2828     parameter [0:0] ENABLE_PCPI = 0,
2829     parameter [0:0] ENABLE_MUL = 0,
2830     parameter [0:0] ENABLE_FAST_MUL = 0,
2831     parameter [0:0] ENABLE_DIV = 0,
2832     parameter [0:0] ENABLE_IRQ = 0,
2833     parameter [0:0] ENABLE_IRQ_QREGS = 1,
2834     parameter [0:0] ENABLE_IRQ_TIMER = 1,
2835     parameter [0:0] ENABLE_TRACE = 0,
2836     parameter [0:0] REGS_INIT_ZERO = 0,
2837     parameter [31:0] MASKED_IRQ = 32'h 0000_0000,
2838     parameter [31:0] LATCHED_IRQ = 32'h ffff_ffff,
2839     parameter [31:0] PROGADDR_RESET = 32'h 0000_0000,
2840     parameter [31:0] PROGADDR_IRQ = 32'h 0000_0010,
2841     parameter [31:0] STACKADDR = 32'h ffff_ffff
2842 ) (
2843     output trap,
2844
2845     // Wishbone interfaces
2846     input wb_rst_i,
2847     input wb_clk_i,
2848
2849     output reg [31:0] wbm_adr_o,
2850     output reg [31:0] wbm_dat_o,
2851     input [31:0] wbm_dat_i,
2852     output reg wbm_we_o,
2853     output reg [3:0] wbm_sel_o,
2854     output reg wbm_stb_o,
2855     input wbm_ack_i,
2856     output reg wbm_cyc_o,
2857
2858     // Pico Co-Processor Interface (PCPI)
2859     output pcpi_valid,
2860     output [31:0] pcpi_insn,
2861     output [31:0] pcpi_rs1,
2862     output [31:0] pcpi_rs2,
2863     input [31:0] pcpi_wr,
2864     input [31:0] pcpi_rd,
2865     input pcpi_wait,
2866     input pcpi_ready,
2867
2868     // IRQ interface
2869     input [31:0] irq,
2870     output [31:0] eoi,
2871
2872     *****

```

```

2871 `ifdef RISC_V Formal
2872 output rvfi_valid,
2873 output [63:0] rvfi_order,
2874 output [31:0] rvfi_insn,
2875 output rvfi_trap,
2876 output rvfi_halt,
2877 output rvfi_intr,
2878 output [ 4:0] rvfi_rs1_addr,
2879 output [ 4:0] rvfi_rs2_addr,
2880 output [31:0] rvfi_rs1_rdata,
2881 output [31:0] rvfi_rs2_rdata,
2882 output [ 4:0] rvfi_rd_addr,
2883 output [31:0] rvfi_rd_wdata,
2884 output [31:0] rvfi_pc_rdata,
2885 output [31:0] rvfi_pc_wdata,
2886 output [31:0] rvfi_mem_addr,
2887 output [ 3:0] rvfi_mem_rmask,
2888 output [ 3:0] rvfi_mem_wmask,
2889 output [31:0] rvfi_mem_rdata,
2890 output [31:0] rvfi_mem_wdata,
2891 `endif
2892
2893 // Trace Interface
2894 output trace_valid,
2895 output [35:0] trace_data,
2896
2897 output mem_instr
2898 );
2899 wire mem_valid;
2900 wire [31:0] mem_addr;
2901 wire [31:0] mem_wdata;
2902 wire [ 3:0] mem_wstrb;
2903 reg mem_ready;
2904 reg [31:0] mem_rdata;
2905
2906 wire clk;
2907 wire resetn;
2908
2909 assign clk = wb_clk_i;
2910 assign resetn = ~wb_rst_i;
2911
2912 picorv32 #(
2913     .ENABLE_COUNTERS (ENABLE_COUNTERS ),
2914     .ENABLE_COUNTERS64 (ENABLE_COUNTERS64 ),
2915     .ENABLE_REGS_16_31 (ENABLE_REGS_16_31 ),
2916     .ENABLE_REGS_DUALPORT (ENABLE_REGS_DUALPORT ),
2917     .TWO_STAGE_SHIFT (TWO_STAGE_SHIFT ),
2918     .BARREL_SHIFTER (BARREL_SHIFTER ),
2919     .TWO_CYCLE_COMPARE (TWO_CYCLE_COMPARE ),
2920     .TWO_CYCLE_ALU (TWO_CYCLE_ALU ),
2921     .COMPRESSED_ISA (COMPRESSED_ISA ),
2922     .CATCH_MISALIGN (CATCH_MISALIGN ),
2923     .CATCH_ILLEGAL (CATCH_ILLEGAL ),
2924     .ENABLE_PCPI (ENABLE_PCPI ),
2925     .ENABLE_MUL (ENABLE_MUL ),
2926     .ENABLE_FAST_MUL (ENABLE_FAST_MUL ),
2927     .ENABLE_DIV (ENABLE_DIV ),
2928     .ENABLE_IRQ (ENABLE_IRQ ),
2929     .ENABLE_IRQ_QREGS (ENABLE_IRQ_QREGS ),
2930     .ENABLE_IRQ_TIMER (ENABLE_IRQ_TIMER ),
2931     .ENABLE_TRACE (ENABLE_TRACE ),
2932     .REGS_INIT_ZERO (REGS_INIT_ZERO ),
2933     .MASKED_IRQ (MASKED_IRQ ),
2934     .LATCHED_IRQ (LATCHED_IRQ ),
2935     .PROGADDR_RESET (PROGADDR_RESET ),
2936     .PROGADDR_IRQ (PROGADDR_IRQ ),
2937     .STACKADDR (STACKADDR )
2938 ) picorv32_core (
2939     .clk (clk ),
2940     .resetn (resetn),
2941     .trap (trap ),
2942
2943     .mem_valid(mem_valid),
2944     .mem_addr (mem_addr ),
2945     .mem_wdata(mem_wdata),
2946     .mem_wstrb(mem_wstrb),
2947     .mem_instr(mem_instr),
2948     .mem_ready(mem_ready),
2949     .mem_rdata(mem_rdata),
2950
2951     .pcpi_valid(pcpi_valid),
2952     .pcpi_insn (pcpi_insn ),
2953     .pcpi_rs1 (pcpi_rs1 ),
2954     .pcpi_rs2 (pcpi_rs2 ),
2955     .pcpi_wr (pcpi_wr ),
2956     .pcpi_rd (pcpi_rd ),
2957     .pcpi_wait (pcpi_wait ),
2958     .pcpi_ready(pcpi_ready),
2959
2960     .irq(irq),
2961     .eoi(eoi),
2962 `ifdef RISC_V Formal
2963     .rvfi_valid (rvfi_valid ),
2964     .rvfi_order (rvfi_order ),
2965     .rvfi_insn (rvfi_insn ),
2966     .rvfi_trap (rvfi_trap ),
2967     .rvfi_halt (rvfi_halt ),
2968     .rvfi_intr (rvfi_intr ),
2969     .rvfi_rs1_addr (rvfi_rs1_addr ),
2970     .rvfi_rs2_addr (rvfi_rs2_addr ),
2971
2972     .rvfi_rs1_rdata(rvfi_rs1_rdata),
2973     .rvfi_rs2_rdata(rvfi_rs2_rdata),
2974     .rvfi_rd_addr (rvfi_rd_addr ),
2975     .rvfi_rd_wdata (rvfi_rd_wdata ),
2976     .rvfi_pc_rdata (rvfi_pc_rdata ),
2977     .rvfi_pc_wdata (rvfi_pc_wdata ),
2978     .rvfi_mem_addr (rvfi_mem_addr ),
2979     .rvfi_mem_rmask(rvfi_mem_rmask),
2980     .rvfi_mem_wmask(rvfi_mem_wmask),
2981     .rvfi_mem_rdata(rvfi_mem_rdata),
2982     .rvfi_mem_wdata(rvfi_mem_wdata),
2983 `endif
2984     .trace_valid(trace_valid),
2985     .trace_data (trace_data)
2986 );
2987
2988 localparam IDLE = 2'b00;
2989 localparam WBSTART = 2'b01;
2990 localparam WBEND = 2'b10;
2991
2992 reg [1:0] state;
2993
2994 wire we;
2995
2996 assign we = (mem_wstrb[0] | mem_wstrb[1] | mem_wstrb[2]
2997 | mem_wstrb[3]);
2998
2999 always @(posedge wb_clk_i) begin
3000     if (wb_rst_i) begin
3001         wbm_adr_o <= 0;
3002         wbm_dat_o <= 0;
3003         wbm_we_o <= 0;
3004         wbm_sel_o <= 0;
3005         wbm_stb_o <= 0;
3006         wbm_cyc_o <= 0;
3007         state <= IDLE;
3008     end else begin
3009         case (state)
3010             IDLE: begin
3011                 if (mem_valid) begin
3012                     wbm_adr_o <=
3013                         mem_addr;
3014                     wbm_dat_o <=
3015                         mem_wdata;
3016                     wbm_we_o <= we;
3017                     wbm_sel_o <=
3018                         mem_wstrb;
3019                     wbm_stb_o <=
3020                         1'b1;
3021                     wbm_cyc_o <=
3022                         1'b1;
3023                     state <=
3024                         WBSTART;
3025                 end else begin
3026                     mem_ready <=
3027                         1'b0;
3028                     wbm_stb_o <=
3029                         1'b0;
3030                     wbm_cyc_o <=
3031                         1'b0;
3032                     wbm_we_o <=
3033                         1'b0;
3034                     end
3035                 end
3036             end
3037             WBSTART: begin
3038                 if (wbm_ack_i) begin
3039                     mem_rdata <=
3040                         wbm_dat_i;
3041                     mem_ready <=
3042                         1'b1;
3043                     state <= WBEND;
3044                     wbm_stb_o <=
3045                         1'b0;
3046                     wbm_cyc_o <=
3047                         1'b0;
3048                     wbm_we_o <=
3049                         1'b0;
3050                 end
3051             end
3052             WBEND: begin
3053                 mem_ready <= 1'b0;
3054                 state <= IDLE;
3055             end
3056             default:
3057                 state <= IDLE;
3058         endcase
3059     end
3060 end
3061 endmodule

```

2. YosysHQ_picorv32/testbench.v (479 lines)

1 // This is free and unencumbered software released into the public domain.


```

2 //
3 // Anyone is free to copy, modify, publish, use, compile,
sell, or
4 // distribute this software, either in source code form or as
a compiled
5 // binary, for any purpose, commercial or non-commercial, and
by any
6 // means.
7
8 `timescale 1 ns / 1 ps
9
10 `ifndef VERILATOR
11 module testbench #(
12     parameter AXI_TEST = 0,
13     parameter VERBOSE = 0
14 );
15     reg clk = 1;
16     reg resetn = 0;
17     wire trap;
18
19     always #5 clk = ~clk;
20
21     initial begin
22         repeat (100) @(posedge clk);
23         resetn <= 1;
24     end
25
26     initial begin
27         if ($test$plusargs("vcd")) begin
28             $dumpfile("testbench.vcd");
29             $dumpvars(0, testbench);
30         end
31         repeat (1000000) @(posedge clk);
32         $display("TIMEOUT");
33         $finish;
34     end
35
36     wire trace_valid;
37     wire [35:0] trace_data;
38     integer trace_file;
39
40     initial begin
41         if ($test$plusargs("trace")) begin
42             trace_file = $fopen("testbench.trace",
"w");
43             repeat (10) @(posedge clk);
44             while (!trap) begin
45                 @(posedge clk);
46                 if (trace_valid)
47                     $fwrite(trace_file,
"%x\n", trace_data);
48             end
49             $fclose(trace_file);
50             $display("Finished writing test-
bench.trace.");
51         end
52     end
53
54     picorv32_wrapper #(
55         .AXI_TEST (AXI_TEST),
56         .VERBOSE (VERBOSE)
57     ) top (
58         .clk(clk),
59         .resetn(resetn),
60         .trap(trap),
61         .trace_valid(trace_valid),
62         .trace_data(trace_data)
63     );
64 endmodule
65 `endif
66
67 module picorv32_wrapper #(
68     parameter AXI_TEST = 0,
69     parameter VERBOSE = 0
70 ) (
71     input clk,
72     input resetn,
73     output trap,
74     output trace_valid,
75     output [35:0] trace_data
76 );
77     wire tests_passed;
78     reg [31:0] irq = 0;
79
80     reg [15:0] count_cycle = 0;
81     always @(posedge clk) count_cycle <= resetn ?
count_cycle + 1 : 0;
82
83     always @* begin
84         irq = 0;
85         irq[4] = &count_cycle[12:0];
86         irq[5] = &count_cycle[15:0];
87     end
88
89     wire mem_axi_awvalid;
90     wire mem_axi_awready;
91     wire [31:0] mem_axi_awaddr;
92     wire [ 2:0] mem_axi_awprot;
93
94     wire mem_axi_wvalid;
95     wire mem_axi_wready;

```

```

96     wire [31:0] mem_axi_wdata;
97     wire [ 3:0] mem_axi_wstrb;
98
99     wire mem_axi_bvalid;
100    wire mem_axi_bready;
101
102    wire mem_axi_rvalid;
103    wire mem_axi_rready;
104    wire [31:0] mem_axi_raddr;
105    wire [ 2:0] mem_axi_rprot;
106
107    wire mem_axi_rvalid;
108    wire mem_axi_rready;
109    wire [31:0] mem_axi_rdata;
110
111    axi4_memory #(
112        .AXI_TEST (AXI_TEST),
113        .VERBOSE (VERBOSE)
114    ) mem (
115        .clk (clk),
116        .mem_axi_awvalid (mem_axi_awvalid),
117        .mem_axi_awready (mem_axi_awready),
118        .mem_axi_awaddr (mem_axi_awaddr),
119        .mem_axi_awprot (mem_axi_awprot),
120
121        .mem_axi_wvalid (mem_axi_wvalid),
122        .mem_axi_wready (mem_axi_wready),
123        .mem_axi_wdata (mem_axi_wdata),
124        .mem_axi_wstrb (mem_axi_wstrb),
125
126        .mem_axi_bvalid (mem_axi_bvalid),
127        .mem_axi_bready (mem_axi_bready),
128
129        .mem_axi_rvalid (mem_axi_rvalid),
130        .mem_axi_rready (mem_axi_rready),
131        .mem_axi_raddr (mem_axi_raddr),
132        .mem_axi_rprot (mem_axi_rprot),
133
134        .mem_axi_rvalid (mem_axi_rvalid),
135        .mem_axi_rready (mem_axi_rready),
136        .mem_axi_rdata (mem_axi_rdata),
137
138        .tests_passed (tests_passed)
139    );
140
141    `ifndef RISC_V_FORMAL
142    wire rvfi_valid;
143    wire [63:0] rvfi_order;
144    wire [31:0] rvfi_insn;
145    wire rvfi_trap;
146    wire rvfi_halt;
147    wire rvfi_intr;
148    wire [4:0] rvfi_rs1_addr;
149    wire [4:0] rvfi_rs2_addr;
150    wire [31:0] rvfi_rs1_rdata;
151    wire [31:0] rvfi_rs2_rdata;
152    wire [4:0] rvfi_rd_addr;
153    wire [31:0] rvfi_rd_wdata;
154    wire [31:0] rvfi_pc_rdata;
155    wire [31:0] rvfi_pc_wdata;
156    wire [31:0] rvfi_mem_addr;
157    wire [3:0] rvfi_mem_rmask;
158    wire [3:0] rvfi_mem_wmask;
159    wire [31:0] rvfi_mem_rdata;
160    wire [31:0] rvfi_mem_wdata;
161    `endif
162
163    picorv32_axi #(
164        `ifndef SYNTH_TEST
165        `ifdef SP_TEST
166            .ENABLE_REGS_DUALPORT(0),
167        `endif
168        `ifdef COMPRESSED_ISA
169            .COMPRESSED_ISA(1),
170        `endif
171            .ENABLE_MUL(1),
172            .ENABLE_DIV(1),
173            .ENABLE_IRQ(1),
174            .ENABLE_TRACE(1)
175        `endif
176    ) uut (
177        .clk (clk),
178        .resetn (resetn),
179        .trap (trap),
180        .mem_axi_awvalid(mem_axi_awvalid),
181        .mem_axi_awready(mem_axi_awready),
182        .mem_axi_awaddr (mem_axi_awaddr),
183        .mem_axi_awprot (mem_axi_awprot),
184        .mem_axi_wvalid (mem_axi_wvalid),
185        .mem_axi_wready (mem_axi_wready),
186        .mem_axi_wdata (mem_axi_wdata),
187        .mem_axi_wstrb (mem_axi_wstrb),
188        .mem_axi_bvalid (mem_axi_bvalid),
189        .mem_axi_bready (mem_axi_bready),
190        .mem_axi_rvalid(mem_axi_rvalid),
191        .mem_axi_rready(mem_axi_rready),
192        .mem_axi_raddr (mem_axi_raddr),
193        .mem_axi_rprot (mem_axi_rprot),
194        .mem_axi_rvalid (mem_axi_rvalid),
195        .mem_axi_rready (mem_axi_rready),
196        .mem_axi_rdata (mem_axi_rdata),

```

```

197         .irq            (irq            ),
198 `ifdef RISC_V Formal
199         .rvfi_valid      (rvfi_valid      ),
200         .rvfi_order      (rvfi_order      ),
201         .rvfi_insn       (rvfi_insn       ),
202         .rvfi_trap       (rvfi_trap       ),
203         .rvfi_halt       (rvfi_halt       ),
204         .rvfi_intr       (rvfi_intr       ),
205         .rvfi_rs1_addr   (rvfi_rs1_addr   ),
206         .rvfi_rs2_addr   (rvfi_rs2_addr   ),
207         .rvfi_rs1_rdata   (rvfi_rs1_rdata ),
208         .rvfi_rs2_rdata   (rvfi_rs2_rdata ),
209         .rvfi_rd_addr     (rvfi_rd_addr     ),
210         .rvfi_rd_wdata    (rvfi_rd_wdata    ),
211         .rvfi_pc_rdata    (rvfi_pc_rdata    ),
212         .rvfi_pc_wdata    (rvfi_pc_wdata    ),
213         .rvfi_mem_addr    (rvfi_mem_addr    ),
214         .rvfi_mem_rmask   (rvfi_mem_rmask   ),
215         .rvfi_mem_wmask   (rvfi_mem_wmask   ),
216         .rvfi_mem_rdata   (rvfi_mem_rdata   ),
217         .rvfi_mem_wdata   (rvfi_mem_wdata   ),
218 `endif
219         .trace_valid     (trace_valid     ),
220         .trace_data      (trace_data      )
221     );
222
223 `ifdef RISC_V Formal
224     picorv32_rvfimon rvfi_monitor (
225         .clock            (clk            ),
226         .reset            (!resetsn       ),
227         .rvfi_valid      (rvfi_valid      ),
228         .rvfi_order      (rvfi_order      ),
229         .rvfi_insn       (rvfi_insn       ),
230         .rvfi_trap       (rvfi_trap       ),
231         .rvfi_halt       (rvfi_halt       ),
232         .rvfi_intr       (rvfi_intr       ),
233         .rvfi_rs1_addr   (rvfi_rs1_addr   ),
234         .rvfi_rs2_addr   (rvfi_rs2_addr   ),
235         .rvfi_rs1_rdata   (rvfi_rs1_rdata ),
236         .rvfi_rs2_rdata   (rvfi_rs2_rdata ),
237         .rvfi_rd_addr     (rvfi_rd_addr     ),
238         .rvfi_rd_wdata    (rvfi_rd_wdata    ),
239         .rvfi_pc_rdata    (rvfi_pc_rdata    ),
240         .rvfi_pc_wdata    (rvfi_pc_wdata    ),
241         .rvfi_mem_addr    (rvfi_mem_addr    ),
242         .rvfi_mem_rmask   (rvfi_mem_rmask   ),
243         .rvfi_mem_wmask   (rvfi_mem_wmask   ),
244         .rvfi_mem_rdata   (rvfi_mem_rdata   ),
245         .rvfi_mem_wdata   (rvfi_mem_wdata   )
246     );
247 `endif
248
249     reg [1023:0] firmware_file;
250     initial begin
251         if (!$value$plusargs("firmware=%s", firm-
ware_file))
252             firmware_file = "firmware/firm-
ware.hex";
253         $readmemh(firmware_file, mem.memory);
254     end
255
256     integer cycle_counter;
257     always @(posedge clk) begin
258         cycle_counter <= resetsn ? cycle_counter + 1 :
0;
259         if (resetsn && trap) begin
260 `ifndef VERILATOR
261             repeat (10) @(posedge clk);
262 `endif
263             $display("TRAP after %1d clock cycles",
cycle_counter);
264             if (tests_passed) begin
265                 $display("ALL TESTS PASSED.");
266                 $finish;
267             end else begin
268                 $display("ERROR!");
269                 if ($test$plusargs("noerror"))
270                     $finish;
271                 $stop;
272             end
273         end
274     end
275 endmodule
276
277 module axi4_memory #(
278     parameter AXI_TEST = 0,
279     parameter VERBOSE = 0
280 ) (
281     /* verilator lint_off MULTIDRIVEN */
282
283     input          clk,
284     input          mem_axi_awvalid,
285     output reg     mem_axi_awready,
286     input [31:0]   mem_axi_awaddr,
287     input [ 2:0]   mem_axi_awprot,
288
289     input          mem_axi_wvalid,
290     output reg     mem_axi_wready,
291     input [31:0]   mem_axi_wdata,
292     input [ 3:0]   mem_axi_wstrb,

```

```

293
294     output reg     mem_axi_bvalid,
295     input          mem_axi_bready,
296
297     input          mem_axi_arvalid,
298     output reg     mem_axi_arready,
299     input [31:0]   mem_axi_araddr,
300     input [ 2:0]   mem_axi_arprot,
301
302     output reg     mem_axi_rvalid,
303     input          mem_axi_rready,
304     output reg [31:0] mem_axi_rdata,
305
306     output reg     tests_passed
307 );
308
309 reg [31:0] memory [0:128*1024/4-1] /* verilator pub-
lic */;
310
311 reg verbose;
312 initial verbose = $test$plusargs("verbose") || VERBOSE;
313
314 reg axi_test;
315 initial axi_test = $test$plusargs("axi_test") ||
AXI_TEST;
316
317 initial begin
318     mem_axi_awready = 0;
319     mem_axi_wready = 0;
320     mem_axi_bvalid = 0;
321     mem_axi_arready = 0;
322     mem_axi_rvalid = 0;
323     tests_passed = 0;
324 end
325
326 reg [63:0] xorshift64_state = 64'd88172645463325252;
327
328 task xorshift64_next;
329     begin
330         // see page 4 of Marsaglia, George
331         (July 2003). "Xorshift RNGs". Journal of Statistical Software 8
332         (14).
333         xorshift64_state = xorshift64_state ^
(xorshift64_state << 13);
334         xorshift64_state = xorshift64_state ^
(xorshift64_state >> 7);
335         xorshift64_state = xorshift64_state ^
(xorshift64_state << 17);
336     end
337 endtask
338
339 reg [2:0] fast_axi_transaction = ~0;
340 reg [4:0] async_axi_transaction = ~0;
341 reg [4:0] delay_axi_transaction = 0;
342
343 always @(posedge clk) begin
344     if (axi_test) begin
345         xorshift64_next;
346         {fast_axi_transaction,
347         async_axi_transaction, delay_axi_transaction} <= xorshift64_state;
348     end
349 end
350
351 reg latched_raddr_en = 0;
352 reg latched_waddr_en = 0;
353 reg latched_wdata_en = 0;
354
355 reg fast_raddr = 0;
356 reg fast_waddr = 0;
357 reg fast_wdata = 0;
358
359 reg [31:0] latched_raddr;
360 reg [31:0] latched_waddr;
361 reg [31:0] latched_wdata;
362 reg [ 3:0] latched_wstrb;
363 reg         latched_rinsn;
364
365 task handle_axi_arvalid; begin
366     mem_axi_arready <= 1;
367     latched_raddr = mem_axi_araddr;
368     latched_rinsn = mem_axi_arprot[2];
369     latched_raddr_en = 1;
370     fast_raddr <= 1;
371 end endtask
372
373 task handle_axi_wvalid; begin
374     mem_axi_wready <= 1;
375     latched_wdata = mem_axi_wdata;
376     latched_wstrb = mem_axi_wstrb;
377     latched_wdata_en = 1;
378     fast_wdata <= 1;
379 end endtask
380
381 task handle_axi_rvalid; begin
382     if (verbose)
383         $display("RD: ADDR=%08x DATA=%08x%",

```



```

latched_raddr, memory[latched_raddr >> 2], latched_rinsn ? " INSN" :
""));
386         if (latched_raddr < 128*1024) begin
387             mem_axi_rdata <= memory[latched_raddr
>> 2];
388             mem_axi_rvalid <= 1;
389             latched_raddr_en = 0;
390         end else begin
391             $display("OUT-OF-BOUNDS MEMORY READ
FROM %08x", latched_raddr);
392             $finish;
393         end
394     end endtask
395
396     task handle_axi_bvalid; begin
397         if (verbose)
398             $display("WR: ADDR=%08x DATA=%08x
STRB=%04b", latched_waddr, latched_wdata, latched_wstrb);
399         if (latched_waddr < 128*1024) begin
400             if (latched_wstrb[0])
memory[latched_waddr >> 2][ 7: 0] <= latched_wdata[ 7: 0];
401             if (latched_wstrb[1])
memory[latched_waddr >> 2][15: 8] <= latched_wdata[15: 8];
402             if (latched_wstrb[2])
memory[latched_waddr >> 2][23:16] <= latched_wdata[23:16];
403             if (latched_wstrb[3])
memory[latched_waddr >> 2][31:24] <= latched_wdata[31:24];
404         end else
405             if (latched_waddr == 32'h1000_0000) begin
406                 if (verbose) begin
407                     if (32 <= latched_wdata &&
latched_wdata < 128)
408                         $display("OUT: '%c'",
latched_wdata[7:0]);
409                     else
410                         $display("OUT: %3d",
latched_wdata);
411                 end else begin
412                     $write("%c",
latched_wdata[7:0]);
413                 `ifndef VERILATOR
414                     $fflush();
415                 `endif
416             end
417         end else
418             if (latched_waddr == 32'h2000_0000) begin
419                 if (latched_wdata == 123456789)
420                     tests_passed = 1;
421             end else begin
422                 $display("OUT-OF-BOUNDS MEMORY WRITE TO
%08x", latched_waddr);
423                 $finish;
424             end
425             mem_axi_bvalid <= 1;
426             latched_waddr_en = 0;
427             latched_wdata_en = 0;
428         end endtask
429
430         always @(negedge clk) begin
431             if (mem_axi_arvalid && !(latched_raddr_en ||
fast_raddr) && async_axi_transaction[0]) handle_axi_arvalid;
432             if (mem_axi_awvalid && !(latched_waddr_en ||
fast_waddr) && async_axi_transaction[1]) handle_axi_awvalid;
433             if (mem_axi_wvalid && !(latched_wdata_en ||
fast_wdata) && async_axi_transaction[2]) handle_axi_wvalid;
434             if (!mem_axi_rvalid && latched_raddr_en &&
async_axi_transaction[3]) handle_axi_rvalid;
435             if (!mem_axi_bvalid && latched_waddr_en &&
latched_wdata_en && async_axi_transaction[4]) handle_axi_bvalid;
436         end
437
438         always @(posedge clk) begin
439             mem_axi_arready <= 0;
440             mem_axi_awready <= 0;
441             mem_axi_wready <= 0;
442
443             fast_raddr <= 0;
444             fast_waddr <= 0;
445             fast_wdata <= 0;
446
447             if (mem_axi_rvalid && mem_axi_rready) begin
448                 mem_axi_rvalid <= 0;
449             end
450
451             if (mem_axi_bvalid && mem_axi_bready) begin
452                 mem_axi_bvalid <= 0;
453             end
454
455             if (mem_axi_arvalid && mem_axi_arready && !
fast_raddr) begin
456                 latched_raddr = mem_axi_araddr;
457                 latched_rinsn = mem_axi_arprot[2];
458                 latched_raddr_en = 1;
459             end
460
461             if (mem_axi_awvalid && mem_axi_awready && !
fast_waddr) begin
462                 latched_waddr = mem_axi_awaddr;
463                 latched_waddr_en = 1;
464             end

```

```

465
466         if (mem_axi_wvalid && mem_axi_wready && !
fast_wdata) begin
467             latched_wdata = mem_axi_wdata;
468             latched_wstrb = mem_axi_wstrb;
469             latched_wdata_en = 1;
470         end
471
472         if (mem_axi_arvalid && !(latched_raddr_en ||
fast_raddr) && !delay_axi_transaction[0]) handle_axi_arvalid;
473         if (mem_axi_awvalid && !(latched_waddr_en ||
fast_waddr) && !delay_axi_transaction[1]) handle_axi_awvalid;
474         if (mem_axi_wvalid && !(latched_wdata_en ||
fast_wdata) && !delay_axi_transaction[2]) handle_axi_wvalid;
475
476         if (!mem_axi_rvalid && latched_raddr_en && !
delay_axi_transaction[3]) handle_axi_rvalid;
477         if (!mem_axi_bvalid && latched_waddr_en &&
latched_wdata_en && !delay_axi_transaction[4]) handle_axi_bvalid;
478     end
479 endmodule

```

3. YosysHQ_picorv32/testbench_ez.v (86 lines)

```

1 // This is free and unencumbered software released into the
public domain.
2 //
3 // Anyone is free to copy, modify, publish, use, compile,
sell, or
4 // distribute this software, either in source code form or as
a compiled
5 // binary, for any purpose, commercial or non-commercial, and
by any
6 // means.
7
8 `timescale 1 ns / 1 ps
9
10 module testbench;
11     reg clk = 1;
12     reg resetn = 0;
13     wire trap;
14
15     always #5 clk = ~clk;
16
17     initial begin
18         if ($test$plusargs("vcd")) begin
19             $dumpfile("testbench.vcd");
20             $dumpvars(0, testbench);
21         end
22         repeat (100) @(posedge clk);
23         resetn <= 1;
24         repeat (1000) @(posedge clk);
25         $finish;
26     end
27
28     wire mem_valid;
29     wire mem_instr;
30     reg mem_ready;
31     wire [31:0] mem_addr;
32     wire [31:0] mem_wdata;
33     wire [3:0] mem_wstrb;
34     reg [31:0] mem_rdata;
35
36     always @(posedge clk) begin
37         if (mem_valid && mem_ready) begin
38             if (mem_instr)
39                 $display("ifetch 0x%08x:
0x%08x", mem_addr, mem_rdata);
40             else if (mem_wstrb)
41                 $display("write 0x%08x: 0x%08x
(wstrb=%b)", mem_addr, mem_wdata, mem_wstrb);
42             else
43                 $display("read 0x%08x:
0x%08x", mem_addr, mem_rdata);
44         end
45     end
46
47     picorv32 #(
48         ) uut (
49         .clk          (clk          ),
50         .resetn       (resetn      ),
51         .trap         (trap        ),
52         .mem_valid    (mem_valid    ),
53         .mem_instr    (mem_instr    ),
54         .mem_ready    (mem_ready    ),
55         .mem_addr     (mem_addr     ),
56         .mem_wdata    (mem_wdata    ),
57         .mem_wstrb    (mem_wstrb    ),
58         .mem_rdata    (mem_rdata    )
59     );
60
61     reg [31:0] memory [0:255];
62
63     initial begin
64         memory[0] = 32'h 3fc00093; //      li
x1,1020
65         memory[1] = 32'h 0000a023; //      sw
x0,0(x1)
66         memory[2] = 32'h 0000a103; // loop: lw

```

```

x2,0(x1)
67          memory[3] = 32'h 00110113; //      addi
x2,x2,1
68          memory[4] = 32'h 0020a023; //      sw
x2,0(x1)
69          memory[5] = 32'h ff5ff06f; //      j
<loop>
70      end
71
72      always @(posedge clk) begin
73          mem_ready <= 0;
74          if (mem_valid && !mem_ready) begin
75              if (mem_addr < 1024) begin
76                  mem_ready <= 1;
77
78                  mem_rdata <= memory[mem_addr >>
2];
79                  if (mem_wstrb[0])
memory[mem_addr >> 2][ 7: 0] <= mem_wdata[ 7: 0];
80                  if (mem_wstrb[1])
memory[mem_addr >> 2][15: 8] <= mem_wdata[15: 8];
81                  if (mem_wstrb[2])
memory[mem_addr >> 2][23:16] <= mem_wdata[23:16];
82                  if (mem_wstrb[3])
memory[mem_addr >> 2][31:24] <= mem_wdata[31:24];
83                  end
84                  /* add memory-mapped IO here */
85      end
86 endmodule

```

4. YosysHQ_picorv32/testbench_wb.v (293 lines)

```

1 `timescale 1 ns / 1 ps
2
3 `ifndef VERILATOR
4 module testbench #(
5     parameter VERBOSE = 0
6 );
7     reg clk = 1;
8     reg resetn = 1;
9     wire trap;
10
11     always #5 clk = ~clk;
12
13     initial begin
14         repeat (100) @(posedge clk);
15         resetn <= 0;
16     end
17
18     initial begin
19         if ($test$plusargs("vcd")) begin
20             $dumpfile("testbench.vcd");
21             $dumpvars(0, testbench);
22         end
23         repeat (1000000) @(posedge clk);
24         $display("TIMEOUT");
25         $finish;
26     end
27
28     wire trace_valid;
29     wire [35:0] trace_data;
30     integer trace_file;
31
32     initial begin
33         if ($test$plusargs("trace")) begin
34             trace_file = $fopen("testbench.trace",
"w");
35             repeat (10) @(posedge clk);
36             while (!trap) begin
37                 @(posedge clk);
38                 if (trace_valid)
39                     $fwrite(trace_file,
"%x\n", trace_data);
40             end
41             $fclose(trace_file);
42             $display("Finished writing test-
bench.trace.");
43         end
44     end
45
46     picorv32_wrapper #(
47         .VERBOSE (VERBOSE)
48     ) top (
49         .wb_clk(clk),
50         .wb_rst(resetn),
51         .trap(trap),
52         .trace_valid(trace_valid),
53         .trace_data(trace_data)
54     );
55 endmodule
56 `endif
57
58 module picorv32_wrapper #(
59     parameter VERBOSE = 0
60 ) (
61     input wb_clk,
62     input wb_rst,
63     output trap,
64     output trace_valid,

```

```

65     output [35:0] trace_data
66 );
67     wire tests_passed;
68     reg [31:0] irq = 0;
69     wire mem_instr;
70
71     reg [15:0] count_cycle = 0;
72     always @(posedge wb_clk) count_cycle <= !wb_rst ?
count_cycle + 1 : 0;
73
74     always @* begin
75         irq = 0;
76         irq[4] = &count_cycle[12:0];
77         irq[5] = &count_cycle[15:0];
78     end
79
80     wire [31:0] wb_m2s_adr;
81     wire [31:0] wb_m2s_dat;
82     wire [3:0] wb_m2s_sel;
83     wire wb_m2s_we;
84     wire wb_m2s_cyc;
85     wire wb_m2s_stb;
86     wire [31:0] wb_s2m_dat;
87     wire wb_s2m_ack;
88
89     wb_ram #(
90         .depth (128*1024),
91         .VERBOSE (VERBOSE)
92     ) ram ( // Wishbone interface
93         .wb_clk_i(wb_clk),
94         .wb_rst_i(wb_rst),
95
96         .wb_adr_i(wb_m2s_adr),
97         .wb_dat_i(wb_m2s_dat),
98         .wb_stb_i(wb_m2s_stb),
99         .wb_cyc_i(wb_m2s_cyc),
100        .wb_dat_o(wb_s2m_dat),
101        .wb_ack_o(wb_s2m_ack),
102        .wb_sel_i(wb_m2s_sel),
103        .wb_we_i(wb_m2s_we),
104
105        .mem_instr(mem_instr),
106        .tests_passed(tests_passed)
107    );
108
109    picorv32_wb #(
110        `ifndef SYNTH_TEST
111        `ifdef SP_TEST
112            .ENABLE_REGS_DUALPORT(0),
113        `endif
114        `ifndef COMPRESSED_ISA
115            .COMPRESSED_ISA(1),
116        `endif
117            .ENABLE_MUL(1),
118            .ENABLE_DIV(1),
119            .ENABLE_IRQ(1),
120            .ENABLE_TRACE(1)
121        `endif
122    ) uut (
123        .trap(trap),
124        .irq(irq),
125        .trace_valid(trace_valid),
126        .trace_data(trace_data),
127        .mem_instr(mem_instr),
128
129        .wb_clk_i(wb_clk),
130        .wb_rst_i(wb_rst),
131
132        .wbm_adr_o(wb_m2s_adr),
133        .wbm_dat_i(wb_s2m_dat),
134        .wbm_stb_o(wb_m2s_stb),
135        .wbm_ack_i(wb_s2m_ack),
136        .wbm_cyc_o(wb_m2s_cyc),
137        .wbm_dat_o(wb_m2s_dat),
138        .wbm_we_o(wb_m2s_we),
139        .wbm_sel_o(wb_m2s_sel)
140    );
141
142    reg [1023:0] firmware_file;
143    initial begin
144        if (!$value$plusargs("firmware=%s", firm-
ware_file))
145            firmware_file = "firmware/firm-
ware.hex";
146        $readmemh(firmware_file, ram.mem);
147    end
148
149    integer cycle_counter;
150    always @(posedge wb_clk) begin
151        cycle_counter <= !wb_rst ? cycle_counter + 1 :
0;
152    end
153    `ifndef VERILATOR
154        repeat (10) @(posedge wb_clk);
155    `endif
156
157    $display("TRAP after %1d clock cycles",
cycle_counter);
158
159    if (tests_passed) begin
160        $display("ALL TESTS PASSED.");
161        $finish;
162    end

```

```

160         end else begin
161             $display("ERROR!");
162             if ($test$plusargs("noerror"))
163                 $finish;
164             $stop;
165         end
166     end
167 endmodule
168
169 /* ISC License
170 *
171 * Verilog on-chip RAM with Wishbone interface
172 *
173 * Copyright (C) 2014, 2016 Olof Kindgren <olof.kind-
174 gren@gmail.com>
175 *
176 * Permission to use, copy, modify, and/or distribute this
177 software for any
178 * purpose with or without fee is hereby granted, provided
179 that the above
180 * copyright notice and this permission notice appear in all
181 copies.
182 *
183 * THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS
184 ALL WARRANTIES
185 * WITH REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WAR-
186 RANTIES OF
187 * MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL THE AUTHOR
188 BE LIABLE FOR
189 * ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL DAMAGES OR
190 ANY DAMAGES
191 * WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS,
192 WHETHER IN AN
193 * ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION,
194 ARISING OUT OF
195 * OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFT-
196 WARE.
197 */
198
199 module wb_ram #(
200     parameter depth = 256,
201     parameter memfile = "",
202     parameter VERBOSE = 0
203 ) (
204     input wb_clk_i,
205     input wb_rst_i,
206
207     input [31:0] wb_adr_i,
208     input [31:0] wb_dat_i,
209     input [3:0] wb_sel_i,
210     input wb_we_i,
211     input wb_cyc_i,
212     input wb_stb_i,
213
214     output reg wb_ack_o,
215     output reg [31:0] wb_dat_o,
216
217     input mem_instr,
218     output reg tests_passed
219 );
220
221 reg verbose;
222 initial verbose = $test$plusargs("verbose") || VERBOSE;
223
224 initial tests_passed = 0;
225
226 reg [31:0] adr_r;
227 wire valid = wb_cyc_i & wb_stb_i;
228
229 always @(posedge wb_clk_i) begin
230     adr_r <= wb_adr_i;
231     // Ack generation
232     wb_ack_o <= valid & !wb_ack_o;
233     if (wb_rst_i)
234         begin
235             adr_r <= {32{1'b0}};
236             wb_ack_o <= 1'b0;
237         end
238     end
239
240 wire ram_we = wb_we_i & valid & wb_ack_o;
241
242 wire [31:0] waddr = adr_r[31:2];
243 wire [31:0] raddr = wb_adr_i[31:2];
244 wire [3:0] we = {4{ram_we}} & wb_sel_i;
245
246 wire [$clog2(depth/4)-1:0] raddr2 = raddr[$clog2(depth/
4)-1:0];
247 wire [$clog2(depth/4)-1:0] waddr2 = waddr[$clog2(depth/
4)-1:0];
248
249 reg [31:0] mem [0:depth/4-1] /* verilator public */;
250
251 always @(posedge wb_clk_i) begin
252     if (ram_we) begin
253         if (verbose)
254             $display("WR: ADDR=%08x
DATA=%08x STRB=%04b",
255                     adr_r, wb_dat_i, we);

```

```

247         if (adr_r[31:0] == 32'h1000_0000)
248             if (verbose) begin
249                 if (32 <= wb_dat_i[7:0]
&& wb_dat_i[7:0] < 128)
250                     $display("OUT:
'%c'", wb_dat_i[7:0]);
251                 else
252                     $display("OUT:
%3d", wb_dat_i[7:0]);
253             end else begin
254                 $write("%c",
wb_dat_i[7:0]);
255             `ifndef VERILATOR
256                 $fflush();
257             `endif
258         end
259     else
260         if (adr_r[31:0] == 32'h2000_0000)
261             if (wb_dat_i[31:0] ==
123456789)
262                 tests_passed = 1;
263         end
264     end
265
266     always @(posedge wb_clk_i) begin
267         if (waddr2 < 128 * 1024 / 4) begin
268             if (we[0])
269                 mem[waddr2][7:0] <=
wb_dat_i[7:0];
270             if (we[1])
271                 mem[waddr2][15:8] <=
wb_dat_i[15:8];
272             if (we[2])
273                 mem[waddr2][23:16] <=
wb_dat_i[23:16];
274             if (we[3])
275                 mem[waddr2][31:24] <=
wb_dat_i[31:24];
276         end
277     end
278
279     if (valid & wb_ack_o & !ram_we)
280         if (verbose)
281             $display("RD: ADDR=%08x
DATA=%08x%s", adr_r, mem[raddr2], mem_instr ? " INSN" : "");
282         wb_dat_o <= mem[raddr2];
283     end
284
285     initial begin
286         if (memfile != "")
287             $readmemh(memfile, mem);
288     end
289 endmodule

```

5. YosysHQ_picorv32/scripts/smtbmc/axicheck.v (210 lines)

```

1 module testbench (
2     input      clk,
3     output     trap,
4
5     output     mem_axi_awvalid,
6     input     mem_axi_awready,
7     output [31:0] mem_axi_awaddr,
8     output [ 2:0] mem_axi_awprot,
9
10    output     mem_axi_wvalid,
11    input     mem_axi_wready,
12    output [31:0] mem_axi_wdata,
13    output [ 3:0] mem_axi_wstrb,
14
15    input     mem_axi_bvalid,
16    output    mem_axi_bready,
17
18    output     mem_axi_rvalid,
19    input     mem_axi_rready,
20    output [31:0] mem_axi_raddr,
21    output [ 2:0] mem_axi_rprot,
22
23    input     mem_axi_rvalid,
24    output    mem_axi_rready,
25    input [31:0] mem_axi_rdata
26 );
27
28 reg resetn = 0;
29
30 always @(posedge clk)
31     resetn <= 1;
32
33 picorv32_axi #(
34     .ENABLE_COUNTERS(1),
35     .ENABLE_COUNTERS64(1),
36     .ENABLE_REGS_16_31(1),
37     .ENABLE_REGS_DUALPORT(1),
38     .BARREL_SHIFTER(1),
39     .TWO_CYCLE_COMPARE(0),
40     .TWO_CYCLE_ALU(0),

```

```

40      .COMPRESSED_ISA(0),
41      .CATCH_MISALIGN(1),
42      .CATCH_ILLINSN(1)
43  ) uut (
44      .clk          (clk          ),
45      .resetn       (resetn       ),
46      .trap         (trap         ),
47      .mem_axi_awvalid (mem_axi_awvalid),
48      .mem_axi_awready (mem_axi_awready),
49      .mem_axi_awaddr  (mem_axi_awaddr ),
50      .mem_axi_awprot  (mem_axi_awprot ),
51      .mem_axi_wvalid  (mem_axi_wvalid ),
52      .mem_axi_wready  (mem_axi_wready ),
53      .mem_axi_wdata   (mem_axi_wdata  ),
54      .mem_axi_wstrb   (mem_axi_wstrb  ),
55      .mem_axi_bvalid  (mem_axi_bvalid ),
56      .mem_axi_bready  (mem_axi_bready ),
57      .mem_axi_arvalid (mem_axi_arvalid),
58      .mem_axi_arready (mem_axi_arready),
59      .mem_axi_araddr  (mem_axi_araddr ),
60      .mem_axi_arprot  (mem_axi_arprot ),
61      .mem_axi_rvalid  (mem_axi_rvalid ),
62      .mem_axi_rready  (mem_axi_rready ),
63      .mem_axi_rdata   (mem_axi_rdata  )
64  );
65
66  reg expect_bvalid_aw = 0;
67  reg expect_bvalid_w  = 0;
68  reg expect_rvalid    = 0;
69
70  reg [3:0] timeout_aw = 0;
71  reg [3:0] timeout_w  = 0;
72  reg [3:0] timeout_b  = 0;
73  reg [3:0] timeout_ar = 0;
74  reg [3:0] timeout_r  = 0;
75  reg [3:0] timeout_ex = 0;
76
77  always @(posedge clk) begin
78      timeout_aw <= !mem_axi_awvalid ||
mem_axi_awready ? 0 : timeout_aw + 1;
79      timeout_w  <= !mem_axi_wvalid ||
mem_axi_wready ? 0 : timeout_w  + 1;
80      timeout_b  <= !mem_axi_bvalid ||
mem_axi_bready ? 0 : timeout_b  + 1;
81      timeout_ar <= !mem_axi_arvalid || mem_axi_ar-
ready ? 0 : timeout_ar + 1;
82      timeout_r  <= !mem_axi_rvalid || mem_axi_r-
ready ? 0 : timeout_r  + 1;
83      timeout_ex <= !(expect_bvalid_aw, ex-
pect_bvalid_w, expect_rvalid) ? 0 : timeout_ex + 1;
84      restrict(timeout_aw != 15);
85      restrict(timeout_w  != 15);
86      restrict(timeout_b  != 15);
87      restrict(timeout_ar != 15);
88      restrict(timeout_r  != 15);
89      restrict(timeout_ex != 15);
90      restrict(!trap);
91
92  end
93
94  always @(posedge clk) begin
95      if (resetn) begin
96          if (!$past(resetn)) begin
97              assert(!mem_axi_awvalid);
98              assert(!mem_axi_wvalid);
99              assume(!mem_axi_bvalid);
100             assert(!mem_axi_arvalid);
101             assume(!mem_axi_rvalid);
102         end else begin
103             // Only one read/write transac-
tion in flight at each point in time
104
105             if (expect_bvalid_aw) begin
106                 assert(!mem_axi_awval-
id);
107             end
108
109             if (expect_bvalid_w) begin
110                 assert(!mem_axi_wval-
id);
111             end
112
113             if (expect_rvalid) begin
114                 assert(!mem_axi_arval-
id);
115             end
116
117             expect_bvalid_aw = ex-
pect_bvalid_w || (mem_axi_awvalid && mem_axi_awready);
118             expect_bvalid_w  = ex-
pect_bvalid_w || (mem_axi_wvalid && mem_axi_wready);
119             expect_rvalid    = ex-
pect_rvalid || (mem_axi_arvalid && mem_axi_arready);
120
121             if (expect_bvalid_aw || ex-
pect_bvalid_w) begin
122                 assert(!expect_rvalid);
123             end
124
125             if (expect_rvalid) begin
126                 assert(!ex-
pect_bvalid_w);
127                 assert(!ex-
pect_bvalid_w);
128             end
129
130             if (!expect_bvalid_w || !ex-
pect_bvalid_w) begin
131                 assume(!mem_axi_bval-
id);
132             end
133
134             if (!expect_rvalid) begin
135                 assume(!mem_axi_rval-
id);
136             end
137
138             if (mem_axi_bvalid &&
expect_bvalid_aw = 0;
expect_bvalid_w = 0;
139
140             if (mem_axi_rvalid &&
expect_rvalid = 0;
141
142             // Check AXI Master Streams
143
144             if ($past(mem_axi_awvalid && !
as-
145             sert(mem_axi_awvalid);
146             as-
147             sert($stable(mem_axi_awaddr));
148             as-
149             sert($stable(mem_axi_awprot));
150
151             end
152             if ($fell(mem_axi_awvalid)) be-
gin
153                 as-
154                 sert($past(mem_axi_awready));
155             end
156             if ($fell(mem_axi_awready)) be-
gin
157                 as-
158                 sune($past(mem_axi_awvalid));
159             end
160             if ($past(mem_axi_arvalid && !
as-
161             sert(mem_axi_arvalid);
162             as-
163             sert($stable(mem_axi_araddr));
164             as-
165             sert($stable(mem_axi_arprot));
166
167             end
168             if ($fell(mem_axi_arvalid)) be-
gin
169                 as-
170                 sert($past(mem_axi_arready));
171             end
172             if ($fell(mem_axi_arready)) be-
gin
173                 as-
174                 sune($past(mem_axi_arvalid));
175             end
176             if ($past(mem_axi_wvalid && !
assert(mem_axi_wvalid);
177             as-
178             sert($stable(mem_axi_wdata));
179             as-
180             sert($stable(mem_axi_wstrb));
181
182             end
183             if ($fell(mem_axi_wvalid)) be-
gin
184                 as-
185                 sert($past(mem_axi_wready));
186             end
187             if ($fell(mem_axi_wready)) be-
gin
188                 as-
189                 sune($past(mem_axi_wvalid));
190             end
191             // Check AXI Slave Streams
192
193             if ($past(mem_axi_bvalid && !
assume(mem_axi_bvalid);
194             end
195             if ($fell(mem_axi_bvalid)) be-
gin
196                 as-
197                 sert($past(mem_axi_bready));
198             end
199             if ($fell(mem_axi_bready)) be-
gin
200

```

```

194                                     as-
sert($past(mem_axi_bvalid));
195                                     end
196                                     if ($past(mem_axi_rvalid && !
197 mem_axi_rready)) begin
198                                     assume(mem_axi_rvalid);
199                                     as-
sume($stable(mem_axi_rdata));
200                                     end
201                                     if ($fell(mem_axi_rvalid)) be-
gin
202                                     as-
sume($past(mem_axi_rready));
203                                     end
204                                     if ($fell(mem_axi_rready)) be-
gin
205                                     as-
sert($past(mem_axi_rvalid));
206                                     end
207                                     end
208                                     end
209                                     end
210 endmodule

```

6. YosysHQ_picorv32/scripts/smtbmc/axicheck2.v (147 lines)

```

1 module testbench (
2     input      clk,
3     input      resetn,
4     output     trap_0,
5     output     trap_1,
6
7     output     mem_axi_awvalid_0,
8     input      mem_axi_awready_0,
9     output [31:0] mem_axi_awaddr_0,
10    output [ 2:0] mem_axi_awprot_0,
11
12    output     mem_axi_awvalid_1,
13    input      mem_axi_awready_1,
14    output [31:0] mem_axi_awaddr_1,
15    output [ 2:0] mem_axi_awprot_1,
16
17    output     mem_axi_wvalid_0,
18    input      mem_axi_wready_0,
19    output [31:0] mem_axi_wdata_0,
20    output [ 3:0] mem_axi_wstrb_0,
21
22    output     mem_axi_wvalid_1,
23    input      mem_axi_wready_1,
24    output [31:0] mem_axi_wdata_1,
25    output [ 3:0] mem_axi_wstrb_1,
26
27    input      mem_axi_bvalid,
28    output     mem_axi_bready_0,
29    output     mem_axi_bready_1,
30
31    output     mem_axi_arvalid_0,
32    input      mem_axi_arready_0,
33    output [31:0] mem_axi_araddr_0,
34    output [ 2:0] mem_axi_arprot_0,
35
36    output     mem_axi_arvalid_1,
37    input      mem_axi_arready_1,
38    output [31:0] mem_axi_araddr_1,
39    output [ 2:0] mem_axi_arprot_1,
40
41    input      mem_axi_rvalid,
42    output     mem_axi_rready_0,
43    output     mem_axi_rready_1,
44    input [31:0] mem_axi_rdata
45 );
46 picorv32_axi #(
47     .ENABLE_COUNTERS(1),
48     .ENABLE_COUNTERS64(1),
49     .ENABLE_REGS_16_31(1),
50     .ENABLE_REGS_DUALPORT(1),
51     .BARREL_SHIFTER(1),
52     .TWO_CYCLE_COMPARE(0),
53     .TWO_CYCLE_ALU(0),
54     .COMPRESSED_ISA(0),
55     .CATCH_MISALIGN(1),
56     .CATCH_ILLINSN(1)
57 ) uut_0 (
58     .clk          (clk),
59     .resetn       (resetn),
60     .trap         (trap_0),
61     .mem_axi_awvalid (mem_axi_awvalid_0),
62     .mem_axi_awready (mem_axi_awready_0),
63     .mem_axi_awaddr  (mem_axi_awaddr_0),
64     .mem_axi_awprot  (mem_axi_awprot_0),
65     .mem_axi_wvalid  (mem_axi_wvalid_0),
66     .mem_axi_wready  (mem_axi_wready_0),
67     .mem_axi_wdata   (mem_axi_wdata_0),
68     .mem_axi_wstrb   (mem_axi_wstrb_0),
69     .mem_axi_bvalid  (mem_axi_bvalid),
70     .mem_axi_bready  (mem_axi_bready_0),
71     .mem_axi_arvalid (mem_axi_arvalid_0),
72     .mem_axi_arready (mem_axi_arready_0),
73     .mem_axi_araddr  (mem_axi_araddr_0),

```

```

74     .mem_axi_arprot  (mem_axi_arprot_0),
75     .mem_axi_rvalid  (mem_axi_rvalid),
76     .mem_axi_rready  (mem_axi_rready_0),
77     .mem_axi_rdata   (mem_axi_rdata)
78 );
79
80 picorv32_axi #(
81     .ENABLE_COUNTERS(1),
82     .ENABLE_COUNTERS64(1),
83     .ENABLE_REGS_16_31(1),
84     .ENABLE_REGS_DUALPORT(1),
85     .BARREL_SHIFTER(1),
86     .TWO_CYCLE_COMPARE(0),
87     .TWO_CYCLE_ALU(0),
88     .COMPRESSED_ISA(0),
89     .CATCH_MISALIGN(1),
90     .CATCH_ILLINSN(1)
91 ) uut_1 (
92     .clk          (clk),
93     .resetn       (resetn),
94     .trap         (trap_1),
95     .mem_axi_awvalid (mem_axi_awvalid_1),
96     .mem_axi_awready (mem_axi_awready_1),
97     .mem_axi_awaddr  (mem_axi_awaddr_1),
98     .mem_axi_awprot  (mem_axi_awprot_1),
99     .mem_axi_wvalid  (mem_axi_wvalid_1),
100    .mem_axi_wready  (mem_axi_wready_1),
101    .mem_axi_wdata   (mem_axi_wdata_1),
102    .mem_axi_wstrb   (mem_axi_wstrb_1),
103    .mem_axi_bvalid  (mem_axi_bvalid),
104    .mem_axi_bready  (mem_axi_bready_1),
105    .mem_axi_arvalid (mem_axi_arvalid_1),
106    .mem_axi_arready (mem_axi_arready_1),
107    .mem_axi_araddr  (mem_axi_araddr_1),
108    .mem_axi_arprot  (mem_axi_arprot_1),
109    .mem_axi_rvalid  (mem_axi_rvalid),
110    .mem_axi_rready  (mem_axi_rready_1),
111    .mem_axi_rdata   (mem_axi_rdata)
112 );
113
114 always @(posedge clk) begin
115     if (resetn && $past(resetn)) begin
116         assert(trap_0) ==
117         trap_1 );
118         assert(mem_axi_awvalid_0 ==
119         mem_axi_awvalid_1);
120         assert(mem_axi_awaddr_0 ==
121         mem_axi_awaddr_1 );
122         assert(mem_axi_awprot_0 == mem_axi_aw-
123         prot_1 );
124         assert(mem_axi_wvalid_0 ==
125         mem_axi_wvalid_1 );
126         assert(mem_axi_wdata_0 ==
127         mem_axi_wdata_1 );
128         assert(mem_axi_wstrb_0 ==
129         mem_axi_wstrb_1 );
130         assert(mem_axi_bready_0 ==
131         mem_axi_bready_1 );
132         assert(mem_axi_arvalid_0 == mem_axi_ar-
133         valid_1);
134         assert(mem_axi_araddr_0 ==
135         mem_axi_araddr_1 );
136         assert(mem_axi_arprot_0 == mem_axi_ar-
137         prot_1 );
138         assert(mem_axi_rready_0 == mem_axi_r-
139         ready_1 );
140         if (mem_axi_awvalid_0) as-
141         sume(mem_axi_awready_0 == mem_axi_awready_1);
142         if (mem_axi_wvalid_0 ) as-
143         sume(mem_axi_wready_0 == mem_axi_wready_1 );
144         if (mem_axi_arvalid_0) as-
145         sume(mem_axi_arready_0 == mem_axi_arready_1);
146         if ($fell(mem_axi_awready_0)) as-
147         sume($past(mem_axi_awvalid_0));
148         if ($fell(mem_axi_wready_0 )) as-
149         sume($past(mem_axi_wvalid_0 ));
150         if ($fell(mem_axi_arready_0)) as-
151         sume($past(mem_axi_arvalid_0));
152         if ($fell(mem_axi_awready_1)) as-
153         sume($past(mem_axi_awvalid_1));
154         if ($fell(mem_axi_wready_1 )) as-
155         sume($past(mem_axi_wvalid_1 ));
156         if ($fell(mem_axi_arready_1)) as-
157         sume($past(mem_axi_arvalid_1));
158         if ($fell(mem_axi_bvalid)) as-
159         sume($past(mem_axi_bready_0));
160         if ($fell(mem_axi_rvalid)) as-
161         sume($past(mem_axi_rready_0));
162         if (mem_axi_rvalid && $past(mem_axi_r-
163         valid)) assume($stable(mem_axi_rdata));
164     end
165 end
166 endmodule

```

7. YosysHQ_picorv32/scripts/smtbmc/mulcmp.v (87 lines)

```

1 module testbench(input clk, mem_ready_0, mem_ready_1);
2
3     reg resetn = 0;
4
5     always @(posedge clk)
6         resetn <= 1;
7
8     reg          pcpi_valid_0 = 1;
9     reg          pcpi_valid_1 = 1;
10
11     wire [31:0] pcpi_insn = $anyconst;
12     wire [31:0] pcpi_rs1 = $anyconst;
13     wire [31:0] pcpi_rs2 = $anyconst;
14
15     wire          pcpi_wr_0;
16     wire [31:0]  pcpi_rd_0;
17     wire          pcpi_wait_0;
18     wire          pcpi_ready_0;
19
20     wire          pcpi_wr_1;
21     wire [31:0]  pcpi_rd_1;
22     wire          pcpi_wait_1;
23     wire          pcpi_ready_1;
24
25     reg          pcpi_wr_ref;
26     reg [31:0]  pcpi_rd_ref;
27     reg          pcpi_ready_ref = 0;
28
29     picorv32_pcpi_mul mul_0 (
30         .clk          (clk          ),
31         .resetn       (resetn       ),
32         .pcpi_valid   (pcpi_valid_0),
33         .pcpi_insn    (pcpi_insn    ),
34         .pcpi_rs1     (pcpi_rs1     ),
35         .pcpi_rs2     (pcpi_rs2     ),
36         .pcpi_wr      (pcpi_wr_0    ),
37         .pcpi_rd      (pcpi_rd_0    ),
38         .pcpi_wait    (pcpi_wait_0  ),
39         .pcpi_ready   (pcpi_ready_0 ),
40     );
41
42     picorv32_pcpi_fast_mul mul_1 (
43         .clk          (clk          ),
44         .resetn       (resetn       ),
45         .pcpi_valid   (pcpi_valid_1),
46         .pcpi_insn    (pcpi_insn    ),
47         .pcpi_rs1     (pcpi_rs1     ),
48         .pcpi_rs2     (pcpi_rs2     ),
49         .pcpi_wr      (pcpi_wr_1    ),
50         .pcpi_rd      (pcpi_rd_1    ),
51         .pcpi_wait    (pcpi_wait_1  ),
52         .pcpi_ready   (pcpi_ready_1 ),
53     );
54
55     );
56
57     always @(posedge clk) begin
58         if (resetn) begin
59             if (pcpi_ready_0 && pcpi_ready_1) begin
60                 assert(pcpi_wr_0 == pcpi_wr_1);
61                 assert(pcpi_rd_0 == pcpi_rd_1);
62             end
63
64             if (pcpi_ready_0) begin
65                 pcpi_valid_0 <= 0;
66                 pcpi_wr_ref <= pcpi_wr_0;
67                 pcpi_rd_ref <= pcpi_rd_0;
68                 pcpi_ready_ref <= 1;
69                 if (pcpi_ready_ref) begin
70                     assert(pcpi_wr_0 ==
71                         pcpi_wr_ref);
72                     assert(pcpi_rd_0 ==
73                         pcpi_rd_ref);
74                 end
75             end
76
77             if (pcpi_ready_1) begin
78                 pcpi_valid_1 <= 0;
79                 pcpi_wr_ref <= pcpi_wr_1;
80                 pcpi_rd_ref <= pcpi_rd_1;
81                 pcpi_ready_ref <= 1;
82                 if (pcpi_ready_ref) begin
83                     assert(pcpi_wr_1 ==
84                         pcpi_wr_ref);
85                     assert(pcpi_rd_1 ==
86                         pcpi_rd_ref);
87                 end
88             end
89         end
90     end
91 endmodule

```

8. YosysHQ_picorv32/scripts/smtbmc/notrap_validop.v (67 lines)

```

1 module testbench(input clk, mem_ready);
2     `include "opcode.v"
3
4     reg resetn = 0;
5     always @(posedge clk) resetn <= 1;
6
7     (* keep *) wire trap, mem_valid, mem_instr;
8     (* keep *) wire [3:0] mem_wstrb;
9     (* keep *) wire [31:0] mem_addr, mem_wdata, mem_rdata;
10    (* keep *) wire [35:0] trace_data;
11
12    reg [31:0] mem [0:2**30-1];
13
14    assign mem_rdata = mem[mem_addr >> 2];
15
16    always @(posedge clk) begin
17        if (resetn && mem_valid && mem_ready) begin
18            if (mem_wstrb[3]) mem[mem_addr >> 2]
19                [31:24] <= mem_wdata[31:24];
20            if (mem_wstrb[2]) mem[mem_addr >> 2]
21                [23:16] <= mem_wdata[23:16];
22            if (mem_wstrb[1]) mem[mem_addr >> 2]
23                [15: 8] <= mem_wdata[15: 8];
24            if (mem_wstrb[0]) mem[mem_addr >> 2]
25                [ 7: 0] <= mem_wdata[ 7: 0];
26        end
27    end
28
29    reg [1:0] mem_ready_stall = 0;
30
31    always @(posedge clk) begin
32        mem_ready_stall <= {mem_ready_stall, mem_valid
33            && !mem_ready};
34        restrict(&mem_ready_stall == 0);
35
36        if (mem_instr && mem_ready && mem_valid) begin
37            assume(opcode_valid(mem_rdata));
38            assume(!opcode_branch(mem_rdata));
39            assume(!opcode_load(mem_rdata));
40            assume(!opcode_store(mem_rdata));
41        end
42
43        if (!mem_valid)
44            assume(!mem_ready);
45
46        if (resetn)
47            assert(!trap);
48    end
49
50    picorv32 #(
51        // change this settings as you like
52        .ENABLE_REGS_DUALPORT(1),
53        .TWO_STAGE_SHIFT(1),
54        .BARREL_SHIFTER(0),
55        .TWO_CYCLE_COMPARE(0),
56        .TWO_CYCLE_ALU(0),
57        .COMPRESSED_ISA(0),
58        .ENABLE_MUL(0),
59        .ENABLE_DIV(0)
60    ) cpu (
61        .clk          (clk          ),
62        .resetn       (resetn       ),
63        .trap         (trap         ),
64        .mem_valid    (mem_valid    ),
65        .mem_instr    (mem_instr    ),
66        .mem_ready    (mem_ready    ),
67        .mem_addr     (mem_addr     ),
68        .mem_wdata    (mem_wdata    ),
69        .mem_wstrb    (mem_wstrb    ),
70        .mem_rdata    (mem_rdata    )
71    );
72 endmodule

```

9. YosysHQ_picorv32/scripts/smtbmc/opcode.v (104 lines)

```

1 function opcode_jump;
2     input [31:0] opcode;
3     begin
4         opcode_jump = 0;
5         if (opcode[6:0] == 7'b1101111) opcode_jump =
6             1; // JAL
7         if (opcode[14:12] == 3'b000 && opcode[6:0] ==
8             7'b1100111) opcode_jump = 1; // JALR
9     end
10 endfunction
11
12 function opcode_branch;
13     input [31:0] opcode;
14     begin
15         opcode_branch = 0;
16         if (opcode[14:12] == 3'b000 && opcode[6:0] ==
17             7'b1100011) opcode_branch = 1; // BEQ
18         if (opcode[14:12] == 3'b001 && opcode[6:0] ==
19             7'b1100011) opcode_branch = 1; // BNE
20         if (opcode[14:12] == 3'b100 && opcode[6:0] ==

```



```

7'b1100011) opcode_branch = 1; // BLT
17   if (opcode[14:12] == 3'b101 && opcode[6:0] ==
7'b1100011) opcode_branch = 1; // BGE
18   if (opcode[14:12] == 3'b110 && opcode[6:0] ==
7'b1100011) opcode_branch = 1; // BLTU
19   if (opcode[14:12] == 3'b111 && opcode[6:0] ==
7'b1100011) opcode_branch = 1; // BGEU
20   end
21 endfunction
22
23 function opcode_load;
24   input [31:0] opcode;
25   begin
26     opcode_load = 0;
27     if (opcode[14:12] == 3'b000 && opcode[6:0] ==
7'b0000011) opcode_load = 1; // LB
28     if (opcode[14:12] == 3'b001 && opcode[6:0] ==
7'b0000011) opcode_load = 1; // LH
29     if (opcode[14:12] == 3'b010 && opcode[6:0] ==
7'b0000011) opcode_load = 1; // LW
30     if (opcode[14:12] == 3'b100 && opcode[6:0] ==
7'b0000011) opcode_load = 1; // LBU
31     if (opcode[14:12] == 3'b101 && opcode[6:0] ==
7'b0000011) opcode_load = 1; // LHU
32   end
33 endfunction
34
35 function opcode_store;
36   input [31:0] opcode;
37   begin
38     opcode_store = 0;
39     if (opcode[14:12] == 3'b000 && opcode[6:0] ==
7'b0100011) opcode_store = 1; // SB
40     if (opcode[14:12] == 3'b001 && opcode[6:0] ==
7'b0100011) opcode_store = 1; // SH
41     if (opcode[14:12] == 3'b010 && opcode[6:0] ==
7'b0100011) opcode_store = 1; // SW
42   end
43 endfunction
44
45 function opcode_alui;
46   input [31:0] opcode;
47   begin
48     opcode_alui = 0;
49     if (opcode[14:12] == 3'b000 && opcode[6:0] ==
7'b0010011) opcode_alui = 1; // ADDI
50     if (opcode[14:12] == 3'b010 && opcode[6:0] ==
7'b0010011) opcode_alui = 1; // SLTI
51     if (opcode[14:12] == 3'b011 && opcode[6:0] ==
7'b0010011) opcode_alui = 1; // SLTIU
52     if (opcode[14:12] == 3'b100 && opcode[6:0] ==
7'b0010011) opcode_alui = 1; // XORI
53     if (opcode[14:12] == 3'b110 && opcode[6:0] ==
7'b0010011) opcode_alui = 1; // ORI
54     if (opcode[14:12] == 3'b111 && opcode[6:0] ==
7'b0010011) opcode_alui = 1; // ANDI
55     if (opcode[31:25] == 7'b0000000 && op-
code[14:12] == 3'b001 && opcode[6:0] == 7'b0010011) opcode_alui =
1; // SLLI
56     if (opcode[31:25] == 7'b0000000 && op-
code[14:12] == 3'b101 && opcode[6:0] == 7'b0010011) opcode_alui =
1; // SRLI
57     if (opcode[31:25] == 7'b0100000 && op-
code[14:12] == 3'b101 && opcode[6:0] == 7'b0010011) opcode_alui =
1; // SRAI
58   end
59 endfunction
60
61 function opcode_alu;
62   input [31:0] opcode;
63   begin
64     opcode_alu = 0;
65     if (opcode[31:25] == 7'b0000000 && op-
code[14:12] == 3'b000 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // ADD
66     if (opcode[31:25] == 7'b0100000 && op-
code[14:12] == 3'b000 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // SUB
67     if (opcode[31:25] == 7'b0000000 && op-
code[14:12] == 3'b001 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // SLL
68     if (opcode[31:25] == 7'b0000000 && op-
code[14:12] == 3'b010 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // SLT
69     if (opcode[31:25] == 7'b0000000 && op-
code[14:12] == 3'b011 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // SLTU
70     if (opcode[31:25] == 7'b0000000 && op-
code[14:12] == 3'b100 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // XOR
71     if (opcode[31:25] == 7'b0000000 && op-
code[14:12] == 3'b101 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // SRL
72     if (opcode[31:25] == 7'b0100000 && op-
code[14:12] == 3'b101 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // SRA
73     if (opcode[31:25] == 7'b0000000 && op-
code[14:12] == 3'b110 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // OR
74     if (opcode[31:25] == 7'b0000000 && op-
code[14:12] == 3'b111 && opcode[6:0] == 7'b0110011) opcode_alu =

```

```

1; // AND
75   end
76 endfunction
77
78 function opcode_sys;
79   input [31:0] opcode;
80   begin
81     opcode_sys = 0;
82     if (opcode[31:20] == 12'hC00 && opcode[19:12]
== 3'b010 && opcode[6:0] == 7'b1110011) opcode_sys = 1; // RDCYCLE
83     if (opcode[31:20] == 12'hC01 && opcode[19:12]
== 3'b010 && opcode[6:0] == 7'b1110011) opcode_sys = 1; // RDTIME
84     if (opcode[31:20] == 12'hC02 && opcode[19:12]
== 3'b010 && opcode[6:0] == 7'b1110011) opcode_sys = 1; // RDINSTRET
85     if (opcode[31:20] == 12'hC80 && opcode[19:12]
== 3'b010 && opcode[6:0] == 7'b1110011) opcode_sys = 1; // RDCYCLEH
86     if (opcode[31:20] == 12'hC81 && opcode[19:12]
== 3'b010 && opcode[6:0] == 7'b1110011) opcode_sys = 1; // RDTIMEH
87     if (opcode[31:20] == 12'hC82 && opcode[19:12]
== 3'b010 && opcode[6:0] == 7'b1110011) opcode_sys = 1; // RDIN-
STRETH
88   end
89
90 endfunction
91
92 function opcode_valid;
93   input [31:0] opcode;
94   begin
95     opcode_valid = 0;
96     if (opcode_jump (opcode)) opcode_valid = 1;
97     if (opcode_branch(opcode)) opcode_valid = 1;
98     if (opcode_load (opcode)) opcode_valid = 1;
99     if (opcode_store (opcode)) opcode_valid = 1;
100    if (opcode_alui (opcode)) opcode_valid = 1;
101    if (opcode_alu (opcode)) opcode_valid = 1;
102    if (opcode_sys (opcode)) opcode_valid = 1;
103   end
104 endfunction

```

10. YosysHQ_picorv32/scripts/smtbmc/tracecmp.v (109 lines)

```

1 module testbench(input clk, mem_ready_0, mem_ready_1);
2   // set this to 1 to test generation of counter examples
3   localparam ENABLE_COUNTERS = 0;
4
5   reg resetn = 0;
6   always @(posedge clk) resetn <= 1;
7
8   (* keep *) wire trap_0, trace_valid_0, mem_valid_0,
mem_instr_0;
9   (* keep *) wire [3:0] mem_wstrb_0;
10  (* keep *) wire [31:0] mem_addr_0, mem_wdata_0,
mem_rdata_0;
11  (* keep *) wire [35:0] trace_data_0;
12
13  (* keep *) wire trap_1, trace_valid_1, mem_valid_1,
mem_instr_1;
14  (* keep *) wire [3:0] mem_wstrb_1;
15  (* keep *) wire [31:0] mem_addr_1, mem_wdata_1,
mem_rdata_1;
16  (* keep *) wire [35:0] trace_data_1;
17
18  reg [31:0] mem_0 [0:2**30-1];
19  reg [31:0] mem_1 [0:2**30-1];
20
21  assign mem_rdata_0 = mem_0[mem_addr_0 >> 2];
22  assign mem_rdata_1 = mem_1[mem_addr_1 >> 2];
23
24  always @(posedge clk) begin
25    if (resetn && mem_valid_0 && mem_ready_0) begin
26      if (mem_wstrb_0[3]) mem_0[mem_addr_0 >> 2]
[31:24] <= mem_wdata_0[31:24];
27      if (mem_wstrb_0[2]) mem_0[mem_addr_0 >> 2]
[23:16] <= mem_wdata_0[23:16];
28      if (mem_wstrb_0[1]) mem_0[mem_addr_0 >> 2]
[15: 8] <= mem_wdata_0[15: 8];
29      if (mem_wstrb_0[0]) mem_0[mem_addr_0 >> 2]
[ 7: 0] <= mem_wdata_0[ 7: 0];
30    end
31    if (resetn && mem_valid_1 && mem_ready_1) begin
32      if (mem_wstrb_1[3]) mem_1[mem_addr_1 >> 2]
[31:24] <= mem_wdata_1[31:24];
33      if (mem_wstrb_1[2]) mem_1[mem_addr_1 >> 2]
[23:16] <= mem_wdata_1[23:16];
34      if (mem_wstrb_1[1]) mem_1[mem_addr_1 >> 2]
[15: 8] <= mem_wdata_1[15: 8];
35      if (mem_wstrb_1[0]) mem_1[mem_addr_1 >> 2]
[ 7: 0] <= mem_wdata_1[ 7: 0];
36    end
37    end
38
39    (* keep *) reg [7:0] trace_balance;

```



```

40     reg [7:0] trace_balance_q;
41
42     always @* begin
43         trace_balance = trace_balance_q;
44         if (trace_valid_0) trace_balance =
trace_balance + 1;
45         if (trace_valid_1) trace_balance =
trace_balance - 1;
46     end
47
48     always @(posedge clk) begin
49         trace_balance_q <= reseth ? trace_balance : 0;
50     end
51
52     picorv32 #(
53         // do not change this settings
54         .ENABLE_COUNTERS(ENABLE_COUNTERS),
55         .ENABLE_TRACE(1),
56
57         // change this settings as you like
58         .ENABLE_REGS_DUALPORT(1),
59         .TWO_STAGE_SHIFT(1),
60         .BARREL_SHIFTER(0),
61         .TWO_CYCLE_COMPARE(0),
62         .TWO_CYCLE_ALU(0),
63         .COMPRESSED_ISA(0),
64         .ENABLE_MUL(0),
65         .ENABLE_DIV(0)
66     ) cpu_0 (
67         .clk          (clk          ),
68         .reseth       (reseth       ),
69         .trap         (trap_0       ),
70         .mem_valid    (mem_valid_0  ),
71         .mem_instr    (mem_instr_0  ),
72         .mem_ready    (mem_ready_0  ),
73         .mem_addr     (mem_addr_0   ),
74         .mem_wdata    (mem_wdata_0  ),
75         .mem_wstrb    (mem_wstrb_0  ),
76         .mem_rdata    (mem_rdata_0  ),
77         .trace_valid  (trace_valid_0),
78         .trace_data   (trace_data_0 )
79     );
80
81     picorv32 #(
82         // do not change this settings
83         .ENABLE_COUNTERS(ENABLE_COUNTERS),
84         .ENABLE_TRACE(1),
85
86         // change this settings as you like
87         .ENABLE_REGS_DUALPORT(1),
88         .TWO_STAGE_SHIFT(1),
89         .BARREL_SHIFTER(0),
90         .TWO_CYCLE_COMPARE(0),
91         .TWO_CYCLE_ALU(0),
92         .COMPRESSED_ISA(0),
93         .ENABLE_MUL(0),
94         .ENABLE_DIV(0)
95     ) cpu_1 (
96         .clk          (clk          ),
97         .reseth       (reseth       ),
98         .trap         (trap_1       ),
99         .mem_valid    (mem_valid_1  ),
100        .mem_instr    (mem_instr_1  ),
101        .mem_ready    (mem_ready_1  ),
102        .mem_addr     (mem_addr_1   ),
103        .mem_wdata    (mem_wdata_1  ),
104        .mem_wstrb    (mem_wstrb_1  ),
105        .mem_rdata    (mem_rdata_1  ),
106        .trace_valid  (trace_valid_1),
107        .trace_data   (trace_data_1 )
108    );
109 endmodule

```

11. YosysHQ_picorv32/scripts/smtbmc/tracemp2.v (196 lines)

```

1 module testbench(
2     input clk, mem_ready_0, mem_ready_1,
3     input [31:0] mem_rdata
4 );
5     // set this to 1 to test generation of counterexamples
6     localparam ENABLE_COUNTERS = 0;
7
8     reg reseth = 0;
9     always @(posedge clk) reseth <= 1;
10
11     (* keep *) wire trap_0, trace_valid_0, mem_valid_0,
mem_instr_0;
12     (* keep *) wire [3:0] mem_wstrb_0;
13     (* keep *) wire [31:0] mem_addr_0, mem_wdata_0,
mem_rdata_0;
14     (* keep *) wire [35:0] trace_data_0;
15
16     (* keep *) wire trap_1, trace_valid_1, mem_valid_1,
mem_instr_1;
17     (* keep *) wire [3:0] mem_wstrb_1;
18     (* keep *) wire [31:0] mem_addr_1, mem_wdata_1,
mem_rdata_1;
19     (* keep *) wire [35:0] trace_data_1;
20
21     reg [31:0] last_mem_rdata;

```

```

22
23     assign mem_rdata_0 = mem_rdata;
24     assign mem_rdata_1 = mem_rdata;
25
26     wire mem_xfer_0 = reseth && mem_valid_0 && mem_ready_0;
27     wire mem_xfer_1 = reseth && mem_valid_1 && mem_ready_1;
28
29     reg [1:0] cmp_mem_state = 0;
30     reg [31:0] cmp_mem_addr;
31     reg [31:0] cmp_mem_wdata;
32     reg [3:0] cmp_mem_wstrb;
33
34     always @* begin
35         if (mem_valid_0 == 0)
36             assume(!mem_ready_0 == 0);
37         if (mem_valid_1 == 0)
38             assume(mem_ready_1 == 0);
39     end
40
41     always @(posedge clk) begin
42         if (cmp_mem_state)
43             assume(last_mem_rdata == mem_rdata);
44         last_mem_rdata <= mem_rdata;
45     end
46
47     always @(posedge clk) begin
48         case (cmp_mem_state)
49             2'b 00: begin
50                 case ({mem_xfer_1, mem_xfer_0})
51                     2'b 11: begin
52                         as-
53                         as-
54                         if
sert(mem_addr_0 == mem_addr_1);
55                         if
sert(mem_wstrb_0 == mem_wstrb_1);
56                         if
(mem_wstrb_0[3]) assert(mem_wdata_0[31:24] == mem_wdata_1[31:24]);
57                         if
(mem_wstrb_0[2]) assert(mem_wdata_0[23:16] == mem_wdata_1[23:16]);
58                         if
(mem_wstrb_0[1]) assert(mem_wdata_0[15: 8] == mem_wdata_1[15: 8]);
59                         if
(mem_wstrb_0[0]) assert(mem_wdata_0[ 7: 0] == mem_wdata_1[ 7: 0]);
60                     end
61                     2'b 01: begin
62                         cmp_mem_state
63                         <= 2'b 01;
64
65                         cmp_mem_addr <=
mem_addr_0;
66                         cmp_mem_wdata
67                         <= mem_wdata_0;
68                         cmp_mem_wstrb
69                         <= mem_wstrb_0;
70                     end
71                     2'b 10: begin
72                         cmp_mem_state
73                         <= 2'b 10;
74
75                         cmp_mem_addr <=
mem_addr_1;
76                         cmp_mem_wdata
77                         <= mem_wdata_1;
78                         cmp_mem_wstrb
79                         <= mem_wstrb_1;
80                     end
81                 endcase
82             end
83             2'b 01: begin
84                 assume(!mem_xfer_0);
85                 if (mem_xfer_1) begin
86                     cmp_mem_state <= 2'b
00;
87                     assert(cmp_mem_addr ==
mem_addr_1);
88
89                     assert(cmp_mem_wstrb ==
mem_wstrb_1);
90                     if (cmp_mem_wstrb[3])
assert(cmp_mem_wdata[31:24] == mem_wdata_1[31:24]);
91                     if (cmp_mem_wstrb[2])
assert(cmp_mem_wdata[23:16] == mem_wdata_1[23:16]);
92                     if (cmp_mem_wstrb[1])
assert(cmp_mem_wdata[15: 8] == mem_wdata_1[15: 8]);
93                     if (cmp_mem_wstrb[0])
assert(cmp_mem_wdata[ 7: 0] == mem_wdata_1[ 7: 0]);
94                 end
95                 2'b 10: begin
96                     assume(!mem_xfer_1);
97                     if (mem_xfer_0) begin
98                         cmp_mem_state <= 2'b
00;
99                         assert(cmp_mem_addr ==
mem_addr_0);
100
101                         assert(cmp_mem_wstrb ==
mem_wstrb_0);
102                         if (cmp_mem_wstrb[3])
assert(cmp_mem_wdata[31:24] == mem_wdata_0[31:24]);
103                         if (cmp_mem_wstrb[2])
assert(cmp_mem_wdata[23:16] == mem_wdata_0[23:16]);

```

```

93         if (cmp_mem_wstrb[1])
assert(cmp_mem_wdata[15: 8] == mem_wdata_0[15: 8]);
94         if (cmp_mem_wstrb[0])
assert(cmp_mem_wdata[ 7: 0] == mem_wdata_0[ 7: 0]);
95         end
96     end
97 endcase
98 end
99
100 reg [1:0] cmp_trace_state = 0;
101 reg [35:0] cmp_trace_data;
102
103 always @(posedge clk) begin
104     if (resetn) begin
105         case (cmp_trace_state)
106             2'b 00: begin
107                 case ({trace_valid_1,
trace_valid_0})
108                     2'b 11: begin
109                         as-
110                     end
111                     2'b 01: begin
112
113 cmp_trace_state <= 2'b 01;
114 cmp_trace_data <= trace_data_0;
115
116 end
117                 2'b 10: begin
118 cmp_trace_state <= 2'b 10;
119 cmp_trace_data <= trace_data_1;
120
121 end
122             endcase
123             2'b 01: begin
124                 assume(!trace_valid_0);
125                 if (trace_valid_1) be-
gin
126 cmp_trace_state
127 <= 2'b 00;
128 as-
129 ssert(cmp_trace_data == trace_data_1);
130
131 end
132             2'b 10: begin
133                 assume(!trace_valid_1);
134                 if (trace_valid_0) be-
gin
135 cmp_trace_state
136 <= 2'b 00;
137 as-
138 ssert(cmp_trace_data == trace_data_0);
139
140 end
141             endcase
142         end
143     end
144
145 picorv32 #(
146     // do not change this settings
147     .ENABLE_COUNTERS(ENABLE_COUNTERS),
148     .ENABLE_TRACE(1),
149
150     // change this settings as you like
151     .ENABLE_REGS_DUALPORT(1),
152     .TWO_STAGE_SHIFT(0),
153     .BARREL_SHIFTER(0),
154     .TWO_CYCLE_COMPARE(0),
155     .TWO_CYCLE_ALU(0),
156     .COMPRESSED_ISA(1),
157     .ENABLE_MUL(0),
158     .ENABLE_DIV(0)
159 ) cpu_0 (
160     .clk          (clk          ),
161     .resetn       (resetn       ),
162     .trap         (trap_0       ),
163     .mem_valid    (mem_valid_0   ),
164     .mem_instr    (mem_instr_0   ),
165     .mem_ready    (mem_ready_0   ),
166     .mem_addr     (mem_addr_0    ),
167     .mem_wdata    (mem_wdata_0   ),
168     .mem_wstrb    (mem_wstrb_0   ),
169     .mem_rdata    (mem_rdata_0   ),
170     .trace_valid  (trace_valid_0),
171     .trace_data   (trace_data_0 )
172 );
173
174 picorv32 #(
175     // do not change this settings
176     .ENABLE_COUNTERS(ENABLE_COUNTERS),
177     .ENABLE_TRACE(1),
178
179     // change this settings as you like
180     .ENABLE_REGS_DUALPORT(1),
181     .TWO_STAGE_SHIFT(1),
182     .BARREL_SHIFTER(0),
183     .TWO_CYCLE_COMPARE(0),

```

```

178     .TWO_CYCLE_ALU(0),
179     .COMPRESSED_ISA(1),
180     .ENABLE_MUL(0),
181     .ENABLE_DIV(0)
182 ) cpu_1 (
183     .clk          (clk          ),
184     .resetn       (resetn       ),
185     .trap         (trap_1       ),
186     .mem_valid    (mem_valid_1   ),
187     .mem_instr    (mem_instr_1   ),
188     .mem_ready    (mem_ready_1   ),
189     .mem_addr     (mem_addr_1    ),
190     .mem_wdata    (mem_wdata_1   ),
191     .mem_wstrb    (mem_wstrb_1   ),
192     .mem_rdata    (mem_rdata_1   ),
193     .trace_valid  (trace_valid_1),
194     .trace_data   (trace_data_1 )
195 );
196 endmodule

```

12. YosysHQ_picorv32/scripts/smtbmc/tracecmp3.v (135 lines)

```

1 // Based on the benchmark from 2016 Yosys-SMTBMC presentation,
which in turn is
2 // based on the tracecmp2 test from this directory.
3
4 module testbench (
5     input clk,
6     input [31:0] mem_rdata_in,
7
8     input          pcpi_wr,
9     input [31:0] pcpi_rd,
10    input          pcpi_wait,
11    input          pcpi_ready
12 );
13
14 reg resetn = 0;
15
16 always @(posedge clk)
17     resetn <= 1;
18
19 wire      cpu0_trap;
20 wire      cpu0_mem_valid;
21 wire      cpu0_mem_instr;
22 wire      cpu0_mem_ready;
23 wire [31:0] cpu0_mem_addr;
24 wire [31:0] cpu0_mem_wdata;
25 wire [3:0]  cpu0_mem_wstrb;
26 wire [31:0] cpu0_mem_rdata;
27 wire      cpu0_trace_valid;
28 wire [35:0] cpu0_trace_data;
29
30 wire      cpu1_trap;
31 wire      cpu1_mem_valid;
32 wire      cpu1_mem_instr;
33 wire      cpu1_mem_ready;
34 wire [31:0] cpu1_mem_addr;
35 wire [31:0] cpu1_mem_wdata;
36 wire [3:0]  cpu1_mem_wstrb;
37 wire [31:0] cpu1_mem_rdata;
38 wire      cpu1_trace_valid;
39 wire [35:0] cpu1_trace_data;
40
41 wire      mem_ready;
42 wire [31:0] mem_rdata;
43
44 assign mem_ready = cpu0_mem_valid && cpu1_mem_valid;
45 assign mem_rdata = mem_rdata_in;
46
47 assign cpu0_mem_ready = mem_ready;
48 assign cpu0_mem_rdata = mem_rdata;
49
50 assign cpu1_mem_ready = mem_ready;
51 assign cpu1_mem_rdata = mem_rdata;
52
53 reg [ 2:0] trace_balance = 3'b010;
54 reg [35:0] trace_buffer_cpu0 = 0, trace_buffer_cpu1 = 0;
55
56 always @(posedge clk) begin
57     if (resetn) begin
58         if (cpu0_trace_valid)
59             trace_buffer_cpu0 <=
cpu0_trace_data;
60         if (cpu1_trace_valid)
61             trace_buffer_cpu1 <=
cpu1_trace_data;
62
63         if (cpu0_trace_valid && !
cpu1_trace_valid)
64             trace_balance <= trace_balance
65 << 1;
66         if (!cpu0_trace_valid &&
cpu1_trace_valid)
67             trace_balance <= trace_balance
68 >> 1;
69     end

```

```

70
71     always @* begin
72         if (resetrn && cpu0_mem_ready) begin
73             assert(cpu0_mem_addr == cpu1_mem_addr);
74             assert(cpu0_mem_wstrb ==
cpu1_mem_wstrb);
75             if (cpu0_mem_wstrb[3]) as-
sert(cpu0_mem_wdata[31:24] == cpu1_mem_wdata[31:24]);
76             if (cpu0_mem_wstrb[2]) as-
sert(cpu0_mem_wdata[23:16] == cpu1_mem_wdata[23:16]);
77             if (cpu0_mem_wstrb[1]) as-
sert(cpu0_mem_wdata[15: 8] == cpu1_mem_wdata[15: 8]);
78             if (cpu0_mem_wstrb[0]) as-
sert(cpu0_mem_wdata[ 7: 0] == cpu1_mem_wdata[ 7: 0]);
79         end
80         if (trace_balance == 3'b010) begin
81             assert(trace_buffer_cpu0 ==
trace_buffer_cpu1);
82         end
83     end
84
85     picorv32 #(
86         .ENABLE_COUNTERS(0),
87         .REGS_INIT_ZERO(1),
88         .COMPRESSED_ISA(1),
89         .ENABLE_TRACE(1),
90
91         .TWO_STAGE_SHIFT(0),
92         .ENABLE_PCP1(1)
93     ) cpu0 (
94         .clk          (clk          ),
95         .resetrn      (resetrn      ),
96         .trap         (cpu0_trap    ),
97         .mem_valid    (cpu0_mem_valid),
98         .mem_instr    (cpu0_mem_instr),
99         .mem_ready    (cpu0_mem_ready),
100        .mem_addr     (cpu0_mem_addr ),
101        .mem_wdata     (cpu0_mem_wdata),
102        .mem_wstrb     (cpu0_mem_wstrb),
103        .mem_rdata     (cpu0_mem_rdata),
104        .pcpi_wr       (pcpi_wr      ),
105        .pcpi_rd       (pcpi_rd      ),
106        .pcpi_wait     (pcpi_wait    ),
107        .pcpi_ready    (pcpi_ready   ),
108        .trace_valid   (cpu0_trace_valid),
109        .trace_data    (cpu0_trace_data)
110    );
111
112     picorv32 #(
113         .ENABLE_COUNTERS(0),
114         .REGS_INIT_ZERO(1),
115         .COMPRESSED_ISA(1),
116         .ENABLE_TRACE(1),
117
118         .TWO_STAGE_SHIFT(1),
119         .TWO_CYCLE_COMPARE(1),
120         .TWO_CYCLE_ALU(1)
121     ) cpu1 (
122         .clk          (clk          ),
123         .resetrn      (resetrn      ),
124         .trap         (cpu1_trap    ),
125         .mem_valid    (cpu1_mem_valid),
126         .mem_instr    (cpu1_mem_instr),
127         .mem_ready    (cpu1_mem_ready),
128         .mem_addr     (cpu1_mem_addr ),
129         .mem_wdata     (cpu1_mem_wdata),
130         .mem_wstrb     (cpu1_mem_wstrb),
131         .mem_rdata     (cpu1_mem_rdata),
132         .trace_valid   (cpu1_trace_valid),
133         .trace_data    (cpu1_trace_data)
134     );
135 endmodule

```

13. YosysHQ_picorv32/scripts/cxxdemo/testbench.v (114 lines)

```

1 `timescale 1 ns / 1 ps
2 `undef VERBOSE_MEM
3 `undef WRITE_VCD
4 `undef MEM8BIT
5
6 module testbench;
7     reg clk = 1;
8     reg resetrn = 0;
9     wire trap;
10
11     always #5 clk = ~clk;
12
13     initial begin
14         repeat (100) @(posedge clk);
15         resetrn <= 1;
16     end
17
18     wire mem_valid;
19     wire mem_instr;
20     reg mem_ready;
21     wire [31:0] mem_addr;
22     wire [31:0] mem_wdata;
23     wire [3:0] mem_wstrb;
24     reg [31:0] mem_rdata;
25

```

```

26     picorv32 #(
27         .COMPRESSED_ISA(1)
28     ) uut (
29         .clk          (clk          ),
30         .resetrn      (resetrn      ),
31         .trap         (trap         ),
32         .mem_valid    (mem_valid    ),
33         .mem_instr    (mem_instr    ),
34         .mem_ready    (mem_ready    ),
35         .mem_addr     (mem_addr     ),
36         .mem_wdata     (mem_wdata   ),
37         .mem_wstrb     (mem_wstrb   ),
38         .mem_rdata     (mem_rdata   )
39     );
40
41     localparam MEM_SIZE = 4*1024*1024;
42     `ifdef MEM8BIT
43         reg [7:0] memory [0:MEM_SIZE-1];
44         initial $readmemh("firmware.hex", memory);
45     `else
46         reg [31:0] memory [0:MEM_SIZE/4-1];
47         initial $readmemh("firmware32.hex", memory);
48     `endif
49
50     always @(posedge clk) begin
51         mem_ready <= 0;
52         if (mem_valid && !mem_ready) begin
53             mem_ready <= 1;
54             mem_rdata <= 'bx;
55             case (1)
56                 mem_addr < MEM_SIZE: begin
57                     `ifdef MEM8BIT
58                         if (!mem_wstrb) begin
59                             if
(mem_wstrb[0]) memory[mem_addr + 0] <= mem_wdata[ 7: 0];
60                             if
(mem_wstrb[1]) memory[mem_addr + 1] <= mem_wdata[15: 8];
61                             if
(mem_wstrb[2]) memory[mem_addr + 2] <= mem_wdata[23:16];
62                             if
(mem_wstrb[3]) memory[mem_addr + 3] <= mem_wdata[31:24];
63                             end else begin
64                                 mem_rdata <=
{memory[mem_addr+3], memory[mem_addr+2], memory[mem_addr+1],
memory[mem_addr]};
65                             end
66                         `else
67                             if (!mem_wstrb) begin
68                                 if
(mem_wstrb[0]) memory[mem_addr >> 2][ 7: 0] <= mem_wdata[ 7: 0];
69                                 if
(mem_wstrb[1]) memory[mem_addr >> 2][15: 8] <= mem_wdata[15: 8];
70                                 if
(mem_wstrb[2]) memory[mem_addr >> 2][23:16] <= mem_wdata[23:16];
71                                 if
(mem_wstrb[3]) memory[mem_addr >> 2][31:24] <= mem_wdata[31:24];
72                                 end else begin
73                                     mem_rdata <=
memory[mem_addr >> 2];
74                                 end
75                             `endif
76                             end
77                             mem_addr == 32'h 1000_0000: be-
gin
78                                 $write("%c",
mem_wdata[7:0]);
79                             end
80                             endcase
81                             end
82                             if (mem_valid && mem_ready) begin
83                                 `ifdef VERBOSE_MEM
84                                     if (!mem_wstrb)
85                                         $display("WR: ADDR=%x DATA=%x
MASK=%b", mem_addr, mem_wdata, mem_wstrb);
86                                     else
87                                         $display("RD: ADDR=%x
DATA=%x%s", mem_addr, mem_rdata, mem_instr ? " INSN" : "");
88                                 `endif
89                                     if (^mem_addr === 1'bx ||
90                                         (mem_wstrb[0] &&
91                                             (mem_wstrb[1] &&
92                                                 (mem_wstrb[2] &&
93                                                     (mem_wstrb[3] &&
94                                                         ^mem_wdata[31:24] == 1'bx))) begin
95                                     $display("CRITICAL UNDEF MEM
TRANSACTION");
96                                     $finish;
97                                 end
98                             end
99
100         `ifdef WRITE_VCD
101             initial begin
102                 $dumpfile("testbench.vcd");
103                 $dumpvars(0, testbench);
104             end
105         `endif
106     end

```

```
107 always @(posedge clk) begin
108     if (resetn && trap) begin
109         repeat (10) @(posedge clk);
110         $display("TRAP");
111     end
112     $finish;
113 end
114 endmodule
```

SERV -- 비트직렬 RISC-V

What this is. A bit-serial RISC-V core — among the smallest in existence.

Why it exists. Because it demonstrates the extreme of the area–time trade. Processing one bit per cycle makes the datapath almost disappear, at 32× the cycle count.

Undergraduate review · EECS concepts. Bit-serial arithmetic; area–time product; extreme resource constraint.

Graduate · Practice note. Worth reading as an existence proof: many datapaths can be serialised when area matters more than latency, and the technique is under-used.

18 files • 3,562 lines

API — serv_aligner.v

Table 1737. olofk_serv/rtl/serv_aligner.v — module / interface / package (1개)

이름	종류
serv_aligner	module

Table 1738. olofk_serv/rtl/serv_aligner.v — 포트 (10개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
i_ibus_adr	input	wire [31:0]
i_ibus_cyc	input	wire
o_ibus_rdt	output	wire [31:0]
o_ibus_ack	output	wire
o_wb_ibus_adr	output	wire [31:0]
o_wb_ibus_cyc	output	wire
i_wb_ibus_rdt	input	wire [31:0]
i_wb_ibus_ack	input	wire

API — serv_alu.v

Table 1739. olofk_serv/rtl/serv_alu.v — module / interface / package (1개)

이름	종류
serv_alu	module

Table 1740. olofk_serv/rtl/serv_alu.v — parameter (2개)

파라미터	형/폭	기본값
W		1

파라미터	형/폭	기본값
B		W-1

Table 1741. olofk_serv/rtl/serv_alu.v — 포트 (13개)

포트	방향	형/폭
clk	input	wire
i_en	input	wire
i_cnt0	input	wire
o_cmp	output	wire
i_sub	input	wire
i_bool_op	input	wire [1:0]
i_cmp_eq	input	wire
i_cmp_sig	input	wire
i_rd_sel	input	wire [2:0]
i_rs1	input	wire [B:0]
i_op_b	input	wire [B:0]
i_buf	input	wire [B:0]
o_rd	output	wire [B:0]

API — serv_bufreg.v**Table 1742. olofk_serv/rtl/serv_bufreg.v — module / interface / package (1개)**

이름	종류
serv_bufreg	module

Table 1743. olofk_serv/rtl/serv_bufreg.v — parameter (3개)

파라미터	형/폭	기본값
MDU	[0:0]	0
W		1
B		W-1

Table 1744. olofk_serv/rtl/serv_bufreg.v — 포트 (20개)

포트	방향	형/폭
i_clk	input	wire
i_cnt0	input	wire
i_cnt1	input	wire
i_cnt_done	input	wire
i_en	input	wire
i_init	input	wire

포트	방향	형/폭
i_mdu_op	input	wire
o_lsb	output	wire [1:0]
i_rs1_en	input	wire
i_imm_en	input	wire
i_clr_lsb	input	wire
i_shift_op	input	wire
i_right_shift_op	input	wire
i_shamt	input	wire [2:0]
i_sh_signed	input	wire
i_rs1	input	wire [B:0]
i_imm	input	wire [B:0]
o_q	output	wire [B:0]
o_dbus_adr	output	wire [31:0]
o_ext_rs1	output	wire [31:0]

API — serv_bufreg2.v

Table 1745. olofk_serv/rtl/serv_bufreg2.v — module / interface / package (1개)

이름	종류
serv_bufreg2	module

Table 1746. olofk_serv/rtl/serv_bufreg2.v — parameter (1개)

파라미터	형/폭	기본값
B		W-1)

Table 1747. olofk_serv/rtl/serv_bufreg2.v — 포트 (18개)

포트	방향	형/폭
i_clk	input	wire
i_en	input	wire
i_init	input	wire
i_cnt7	input	wire
i_cnt_done	input	wire
i_sh_right	input	wire
i_lsb	input	wire [1:0]
i_bytecnt	input	wire [1:0]
o_sh_done	output	wire
i_op_b_sel	input	wire

포트	방향	형/폭
i_shift_op	input	wire
i_rs2	input	wire [B:0]
i_imm	input	wire [B:0]
o_op_b	output	wire [B:0]
o_q	output	wire [B:0]
o_dat	output	wire [31:0]
i_load	input	wire
i_dat	input	wire [31:0]

API — serv_compdec.v

Table 1748. olofk_serv/rtl/serv_compdec.v — module / interface / package (1개)

이름	종류
serv_compdec	module

Table 1749. olofk_serv/rtl/serv_compdec.v — localparam (8개)

상수	형/폭	값
OPCODE_LOAD		7'h03
OPCODE_OP_IMM		7'h13
OPCODE_STORE		7'h23
OPCODE_OP		7'h33
OPCODE_LUI		7'h37
OPCODE_BRANCH		7'h63
OPCODE_JALR		7'h67
OPCODE_JAL		7'h6f

Table 1750. olofk_serv/rtl/serv_compdec.v — 포트 (5개)

포트	방향	형/폭
i_clk	input	wire
i_instr	input	wire [31:0]
i_ack	input	wire
o_instr	output	wire [31:0]
o_iscomp	output	reg

API — serv_csr.v

Table 1751. olofk_serv/rtl/serv_csr.v — module / interface / package (1개)

이름	종류
serv_csr	module

Table 1752. olofk_serv/rtl/serv_csr.v — parameter (3개)

파라미터	형/폭	기본값
RESET_STRATEGY		"MINI"
W		1
B		W-1

Table 1753. olofk_serv/rtl/serv_csr.v — localparam (1개)

상수	형/폭	값
CSR_SOURCE_CSR	[1:0]	2'b00

Table 1754. olofk_serv/rtl/serv_csr.v — 포트 (28개)

포트	방향	형/폭
i_clk	input	wire
i_rst	input	wire
i_trig_irq	input	wire
i_en	input	wire
i_cnt0to3	input	wire
i_cnt3	input	wire
i_cnt7	input	wire
i_cnt11	input	wire
i_cnt12	input	wire
i_cnt_done	input	wire
i_mem_op	input	wire
i_mtip	input	wire
i_trap	input	wire
o_new_irq	output	reg
i_e_op	input	wire
i_ebreak	input	wire
i_mem_cmd	input	wire
i_mstatus_en	input	wire
i_mie_en	input	wire
i_mcause_en	input	wire
i_csr_source	input	wire [1:0]
i_mret	input	wire
i_csr_d_sel	input	wire
i_rf_csr_out	input	wire [B:0]
o_csr_in	output	wire [B:0]

포트	방향	형/폭
i_csr_imm	input	wire [B:0]
i_rs1	input	wire [B:0]
o_q	output	wire [B:0]

API — serv_ctrl.v

Table 1755. olofk_serv/rtl/serv_ctrl.v — module / interface / package (1개)

이름	종류
serv_ctrl	module

Table 1756. olofk_serv/rtl/serv_ctrl.v — parameter (4개)

파라미터	형/폭	기본값
RESET_PC		32'd0
WITH_CSR		1
W		1
B		W-1

Table 1757. olofk_serv/rtl/serv_ctrl.v — 포트 (19개)

포트	방향	형/폭
clk	input	wire
i_rst	input	wire
i_pc_en	input	wire
i_cnt12to31	input	wire
i_cnt0	input	wire
i_cnt1	input	wire
i_cnt2	input	wire
i_jump	input	wire
i_jal_or_jalr	input	wire
i_utype	input	wire
i_pc_rel	input	wire
i_trap	input	wire
i_iscomp	input	wire
i_imm	input	wire [B:0]
i_buf	input	wire [B:0]
i_csr_pc	input	wire [B:0]
o_rd	output	wire [B:0]
o_bad_pc	output	wire [B:0]

포트	방향	형/폭
o_ibus_adr	output	reg [31:0]

API — serv_debug.v

Table 1758. olofk_serv/rtl/serv_debug.v — module / interface / package (1개)

이름	종류
serv_debug	module

Table 1759. olofk_serv/rtl/serv_debug.v — parameter (2개)

파라미터	형/폭	기본값
RESET_PC		0
B		W-1)

Table 1760. olofk_serv/rtl/serv_debug.v — 포트 (55개)

포트	방향	형/폭
b0	output	reg rvfi_valid = 1'
d0	output	reg [63:0] rvfi_order = 64'
d0	output	reg [31:0] rvfi_insn = 32'
b0	output	reg rvfi_trap = 1'
b0	output	reg rvfi_halt = 1'
b0	output	reg rvfi_intr = 1'
b11	output	reg [1:0] rvfi_mode = 2'
b01	output	reg [1:0] rvfi_ixl = 2'
rvfi_rs1_addr	output	reg [4:0]
rvfi_rs2_addr	output	reg [4:0]
rvfi_rs1_rdata	output	reg [31:0]
rvfi_rs2_rdata	output	reg [31:0]
rvfi_rd_addr	output	reg [4:0]
rvfi_rd_wdata	output	wire [31:0]
rvfi_pc_rdata	output	reg [31:0]
rvfi_pc_wdata	output	wire [31:0]
rvfi_mem_addr	output	reg [31:0]
rvfi_mem_rmask	output	reg [3:0]
rvfi_mem_wmask	output	reg [3:0]
rvfi_mem_rdata	output	reg [31:0]
rvfi_mem_wdata	output	reg [31:0]
i_dbus_adr	input	wire [31:0]

포트	방향	형/폭
i_dbus_dat	input	wire [31:0]
i_dbus_sel	input	wire [3:0]
i_dbus_we	input	wire
i_dbus_rdt	input	wire [31:0]
i_dbus_ack	input	wire
i_ctrl_pc_en	input	wire
rs1	input	wire [B:0]
rs2	input	wire [B:0]
rs1_addr	input	wire [4:0]
rs2_addr	input	wire [4:0]
immdec_en	input	wire [3:0]
rd_en	input	wire
trap	input	wire
i_rf_ready	input	wire
i_ibus_cyc	input	wire
two_stage_op	input	wire
init	input	wire
i_ibus_adr	input	wire [31:0]
i_clk	input	wire
i_rst	input	wire
i_ibus_rdt	input	wire [31:0]
i_ibus_ack	input	wire
i_rd_addr	input	wire [4:0]
i_cnt_en	input	wire
i_csr_in	input	wire [B:0]
i_csr_mstatus_en	input	wire
i_csr_mie_en	input	wire
i_csr_mcause_en	input	wire
i_csr_en	input	wire
i_csr_addr	input	wire [1:0]
i_wen0	input	wire
i_wdata0	input	wire [B:0]
i_cnt_done	input	wire

API — serv_decode.v**Table 1761. olofk_serv/rtl/serv_decode.v — module / interface / package (1개)**

이름	종류
serv_decode	module

Table 1762. olofk_serv/rtl/serv_decode.v — parameter (1개)

파라미터	형/폭	기본값
MDU	[0:0]	0)

Table 1763. olofk_serv/rtl/serv_decode.v — 포트 (47개)

포트	방향	형/폭
clk	input	wire
i_wb_rdt	input	wire [31:2]
i_wb_en	input	wire
o_sh_right	output	reg
o_bne_or_bge	output	reg
o_cond_branch	output	reg
o_e_op	output	reg
o_ebreak	output	reg
o_branch_op	output	reg
o_shift_op	output	reg
o_rd_op	output	reg
o_two_stage_op	output	reg
o_dbus_en	output	reg
o_mdu_op	output	reg
o_ext_funct3	output	reg [2:0]
o_bufreg_rs1_en	output	reg
o_bufreg_imm_en	output	reg
o_bufreg_clr_lsb	output	reg
o_bufreg_sh_signed	output	reg
o_ctrl_jal_or_jalr	output	reg
o_ctrl_ctype	output	reg
o_ctrl_pc_rel	output	reg
o_ctrl_mret	output	reg
o_alu_sub	output	reg
o_alu_bool_op	output	reg [1:0]
o_alu_cmp_eq	output	reg

포트	방향	형/폭
o_alu_cmp_sig	output	reg
o_alu_rd_sel	output	reg [2:0]
o_mem_signed	output	reg
o_mem_word	output	reg
o_mem_half	output	reg
o_mem_cmd	output	reg
o_csr_en	output	reg
o_csr_addr	output	reg [1:0]
o_csr_mstatus_en	output	reg
o_csr_mie_en	output	reg
o_csr_mcause_en	output	reg
o_csr_source	output	reg [1:0]
o_csr_d_sel	output	reg
o_csr_imm_en	output	reg
o_mtval_pc	output	reg
o_immdec_ctrl	output	reg [3:0]
o_immdec_en	output	reg [3:0]
o_op_b_source	output	reg
o_rd_mem_en	output	reg
o_rd_csr_en	output	reg
o_rd_alu_en	output	reg

API — serv_immdec.v

Table 1764. olofk_serv/rtl/serv_immdec.v — module / interface / package (1개)

이름	종류
serv_immdec	module

Table 1765. olofk_serv/rtl/serv_immdec.v — parameter (1개)

파라미터	형/폭	기본값
W		1)

Table 1766. olofk_serv/rtl/serv_immdec.v — 포트 (13개)

포트	방향	형/폭
i_clk	input	wire
i_cnt_en	input	wire
i_cnt_done	input	wire

포트	방향	형/폭
i_immdec_en	input	wire [3:0]
i_csr_imm_en	input	wire
i_ctrl	input	wire [3:0]
o_rd_addr	output	wire [4:0]
o_rs1_addr	output	wire [4:0]
o_rs2_addr	output	wire [4:0]
o_csr_imm	output	wire [W-1:0]
o_imm	output	wire [W-1:0]
i_wb_en	input	wire
i_wb_rdt	input	wire [31:7]

API — serv_mem_if.v

Table 1767. olofk_serv/rtl/serv_mem_if.v — module / interface / package (1개)

이름	종류
serv_mem_if	module

Table 1768. olofk_serv/rtl/serv_mem_if.v — parameter (3개)

파라미터	형/폭	기본값
WITH_CSR	[0:0]	1
W		1
B		W-1

Table 1769. olofk_serv/rtl/serv_mem_if.v — 포트 (11개)

포트	방향	형/폭
i_clk	input	wire
i_bytecnt	input	wire [1:0]
i_lsb	input	wire [1:0]
o_misalign	output	wire
i_signed	input	wire
i_word	input	wire
i_half	input	wire
i_mdu_op	input	wire
i_bufreg2_q	input	wire [B:0]
o_rd	output	wire [B:0]
o_wb_sel	output	wire [3:0]

API — serv_rf_if.v**Table 1770. olofk_serv/rtl/serv_rf_if.v — module / interface / package (1개)**

이름	종류
serv_rf_if	module

Table 1771. olofk_serv/rtl/serv_rf_if.v — parameter (2개)

파라미터	형/폭	기본값
W		1
B		W-1

Table 1772. olofk_serv/rtl/serv_rf_if.v — 포트 (35개)

포트	방향	형/폭
i_cnt_en	input	wire
o_wreg0	output	wire [4+WITH_CSR:0]
o_wreg1	output	wire [4+WITH_CSR:0]
o_wen0	output	wire
o_wen1	output	wire
o_wdata0	output	wire [B:0]
o_wdata1	output	wire [B:0]
o_rreg0	output	wire [4+WITH_CSR:0]
o_rreg1	output	wire [4+WITH_CSR:0]
i_rdata0	input	wire [B:0]
i_rdata1	input	wire [B:0]
i_trap	input	wire
i_mret	input	wire
i_mepc	input	wire [B:0]
i_mtval_pc	input	wire
i_bufreg_q	input	wire [B:0]
i_bad_pc	input	wire [B:0]
o_csr_pc	output	wire [B:0]
i_csr_en	input	wire
i_csr_addr	input	wire [1:0]
i_csr	input	wire [B:0]
o_csr	output	wire [B:0]
i_rd_wen	input	wire
i_rd_waddr	input	wire [4:0]
i_ctrl_rd	input	wire [B:0]

포트	방향	형/폭
i_alu_rd	input	wire [B:0]
i_rd_alu_en	input	wire
i_csr_rd	input	wire [B:0]
i_rd_csr_en	input	wire
i_mem_rd	input	wire [B:0]
i_rd_mem_en	input	wire
i_rs1_raddr	input	wire [4:0]
o_rs1	output	wire [B:0]
i_rs2_raddr	input	wire [4:0]
o_rs2	output	wire [B:0]

API — serv_rf_ram.v

Table 1773. olofk_serv/rtl/serv_rf_ram.v — module / interface / package (1개)

이름	종류
serv_rf_ram	module

Table 1774. olofk_serv/rtl/serv_rf_ram.v — parameter (2개)

파라미터	형/폭	기본값
csr_regs		4
depth		32*(32+csr_regs)/width)

Table 1775. olofk_serv/rtl/serv_rf_ram.v — 포트 (6개)

포트	방향	형/폭
depth	input	wire [\$clog2(
i_wdata	input	wire [width-1:0]
i_wen	input	wire
depth	input	wire [\$clog2(
i_ren	input	wire
o_rdata	output	wire [width-1:0]

API — serv_rf_ram_if.v

Table 1776. olofk_serv/rtl/serv_rf_ram_if.v — module / interface / package (1개)

이름	종류
serv_rf_ram_if	module

Table 1777. olofk_serv/rtl/serv_rf_ram_if.v — parameter (8개)

파라미터	형/폭	기본값
width		8
W		1
reset_strategy		"MINI"
csr_regs		4
B		W-1
raw		$\$clog2(32+csr_regs)$
l2w		$\$clog2(width)$
aw		$5+raw-l2w$ //Address width

Table 1778. olofk_serv/rtl/serv_rf_ram_if.v — localparam (3개)

상수	형/폭	값
ratio		width/W
CMSB		$4-\$clog2(W)$
l2r		$\$clog2(ratio)$

Table 1779. olofk_serv/rtl/serv_rf_ram_if.v — 포트 (21개)

포트	방향	형/폭
i_clk	input	wire
i_rst	input	wire
i_wreq	input	wire
i_rreq	input	wire
o_ready	output	wire
i_wreg0	input	wire [raw-1:0]
i_wreg1	input	wire [raw-1:0]
i_wen0	input	wire
i_wen1	input	wire
i_wdata0	input	wire [B:0]
i_wdata1	input	wire [B:0]
i_rreg0	input	wire [raw-1:0]
i_rreg1	input	wire [raw-1:0]
o_rdata0	output	wire [B:0]
o_rdata1	output	wire [B:0]
o_waddr	output	wire [aw-1:0]
o_wdata	output	wire [width-1:0]
o_wen	output	wire
o_raddr	output	wire [aw-1:0]

포트	방향	형/폭
o_ren	output	wire
i_rdata	input	wire [width-1:0]

API — serv_rf_top.v

Table 1780. olofk_serv/rtl/serv_rf_top.v — module / interface / package (1개)

이름	종류
serv_rf_top	module

Table 1781. olofk_serv/rtl/serv_rf_top.v — parameter (10개)

파라미터	형/폭	기본값
COMPRESSED	[0:0]	0
ALIGN	[0:0]	COMPRESSED
MDU	[0:0]	0
PRE_REGISTER		1
RESET_STRATEGY		"MINI"
DEBUG	[0:0]	1'b0
WITH_CSR		1
W		1
RF_WIDTH		W * 2
RF_L2D		$\$clog2((32+(WITH_CSR*4))*32/RF_WIDTH))$

Table 1782. olofk_serv/rtl/serv_rf_top.v — localparam (1개)

상수	형/폭	값
CSR_REGS		WITH_CSR*4

Table 1783. olofk_serv/rtl/serv_rf_top.v — 포트 (41개)

포트	방향	형/폭
clk	input	wire
i_rst	input	wire
i_timer_irq	input	wire
rvfi_valid	output	wire
rvfi_order	output	wire [63:0]
rvfi_insn	output	wire [31:0]
rvfi_trap	output	wire
rvfi_halt	output	wire
rvfi_intr	output	wire
rvfi_mode	output	wire [1:0]

포트	방향	형/폭
rvfi_ixl	output	wire [1:0]
rvfi_rs1_addr	output	wire [4:0]
rvfi_rs2_addr	output	wire [4:0]
rvfi_rs1_rdata	output	wire [31:0]
rvfi_rs2_rdata	output	wire [31:0]
rvfi_rd_addr	output	wire [4:0]
rvfi_rd_wdata	output	wire [31:0]
rvfi_pc_rdata	output	wire [31:0]
rvfi_pc_wdata	output	wire [31:0]
rvfi_mem_addr	output	wire [31:0]
rvfi_mem_rmask	output	wire [3:0]
rvfi_mem_wmask	output	wire [3:0]
rvfi_mem_rdata	output	wire [31:0]
rvfi_mem_wdata	output	wire [31:0]
o_ibus_adr	output	wire [31:0]
o_ibus_cyc	output	wire
i_ibus_rdt	input	wire [31:0]
i_ibus_ack	input	wire
o_dbus_adr	output	wire [31:0]
o_dbus_dat	output	wire [31:0]
o_dbus_sel	output	wire [3:0]
o_dbus_we	output	wire
o_dbus_cyc	output	wire
i_dbus_rdt	input	wire [31:0]
i_dbus_ack	input	wire
o_ext_rs1	output	wire [31:0]
o_ext_rs2	output	wire [31:0]
o_ext_funct3	output	wire [2:0]
i_ext_rd	input	wire [31:0]
i_ext_ready	input	wire
o_mdu_valid	output	wire

API — serv_state.v

Table 1784. olofk_serv/rtl/serv_state.v — module / interface / package (1개)

이름	종류
serv_state	module

Table 1785. olofk_serv/rtl/serv_state.v — parameter (4개)

파라미터	형/폭	기본값
WITH_CSR	[0:0]	1
ALIGN	[0:0]	0
MDU	[0:0]	0
W		1

Table 1786. olofk_serv/rtl/serv_state.v — 포트 (46개)

포트	방향	형/폭
i_clk	input	wire
i_rst	input	wire
i_new_irq	input	wire
i_alu_cmp	input	wire
o_init	output	wire
o_cnt_en	output	wire
o_cnt0to3	output	wire
o_cnt12to31	output	wire
o_cnt0	output	wire
o_cnt1	output	wire
o_cnt2	output	wire
o_cnt3	output	wire
o_cnt7	output	wire
o_cnt11	output	wire
o_cnt12	output	wire
o_cnt_done	output	wire
o_bufreg_en	output	wire
o_ctrl_pc_en	output	wire
o_ctrl_jump	output	reg
o_ctrl_trap	output	wire
i_ctrl_misalign	input	wire
i_sh_done	input	wire
o_mem_bytecnt	output	wire [1:0]
i_mem_misalign	input	wire
i_bne_or_bge	input	wire

포트	방향	형/폭
i_cond_branch	input	wire
i_dbus_en	input	wire
i_two_stage_op	input	wire
i_branch_op	input	wire
i_shift_op	input	wire
i_sh_right	input	wire
i_alu_rd_sel1	input	wire
i_rd_alu_en	input	wire
i_e_op	input	wire
i_rd_op	input	wire
i_mdu_op	input	wire
o_mdu_valid	output	wire
i_mdu_ready	input	wire
o_dbus_cyc	output	wire
i_dbus_ack	input	wire
o_ibus_cyc	output	wire
i_ibus_ack	input	wire
o_rf_rreq	output	wire
o_rf_wreq	output	wire
i_rf_ready	input	wire
o_rf_rd_en	output	wire

API — serv_synth_wrapper.v

Table 1787. olofk_serv/rtl/serv_synth_wrapper.v — module / interface / package (1개)

이름	종류
serv_synth_wrapper	module

Table 1788. olofk_serv/rtl/serv_synth_wrapper.v — parameter (5개)

파라미터	형/폭	기본값
PRE_REGISTER		1
RESET_STRATEGY		"MINI"
WITH_CSR		1
RF_WIDTH		2
RF_L2D		$\$clog2((32+(WITH_CSR*4))*32/RF_WIDTH))$

Table 1789. olofk_serv/rtl/serv_synth_wrapper.v — localparam (1개)

상수	형/폭	값
CSR_REGS		WITH_CSR*4

Table 1790. olofk_serv/rtl/serv_synth_wrapper.v — 포트 (19개)

포트	방향	형/폭
clk	input	wire
i_rst	input	wire
i_timer_irq	input	wire
o_ibus_adr	output	wire [31:0]
o_ibus_cyc	output	wire
i_ibus_rdt	input	wire [31:0]
i_ibus_ack	input	wire
o_dbus_adr	output	wire [31:0]
o_dbus_dat	output	wire [31:0]
o_dbus_sel	output	wire [3:0]
o_dbus_we	output	wire
o_dbus_cyc	output	wire
i_dbus_rdt	input	wire [31:0]
i_dbus_ack	input	wire
o_waddr	output	wire [RF_L2D-1:0]
o_wdata	output	wire [RF_WIDTH-1:0]
o_wen	output	wire
o_raddr	output	wire [RF_L2D-1:0]
i_rdata	input	wire [RF_WIDTH-1:0]

API — serv_top.v**Table 1791. olofk_serv/rtl/serv_top.v — module / interface / package (1개)**

이름	종류
serv_top	module

Table 1792. olofk_serv/rtl/serv_top.v — parameter (9개)

파라미터	형/폭	기본값
W		1
B		W-1
PRE_REGISTER		1
RESET_STRATEGY		"MINI"
RESET_PC		32'd0

파라미터	형/폭	기본값
DEBUG	[0:0]	1'b0
MDU	[0:0]	1'b0
COMPRESSED	[0:0]	0
ALIGN	[0:0]	COMPRESSED)

Table 1793. olofk_serv/rtl/serv_top.v — 포트 (54개)

포트	방향	형/폭
clk	input	wire
i_rst	input	wire
i_timer_irq	input	wire
rvfi_valid	output	wire
rvfi_order	output	wire [63:0]
rvfi_insn	output	wire [31:0]
rvfi_trap	output	wire
rvfi_halt	output	wire
rvfi_intr	output	wire
rvfi_mode	output	wire [1:0]
rvfi_ixl	output	wire [1:0]
rvfi_rs1_addr	output	wire [4:0]
rvfi_rs2_addr	output	wire [4:0]
rvfi_rs1_rdata	output	wire [31:0]
rvfi_rs2_rdata	output	wire [31:0]
rvfi_rd_addr	output	wire [4:0]
rvfi_rd_wdata	output	wire [31:0]
rvfi_pc_rdata	output	wire [31:0]
rvfi_pc_wdata	output	wire [31:0]
rvfi_mem_addr	output	wire [31:0]
rvfi_mem_rmask	output	wire [3:0]
rvfi_mem_wmask	output	wire [3:0]
rvfi_mem_rdata	output	wire [31:0]
rvfi_mem_wdata	output	wire [31:0]
o_rf_rreq	output	wire
o_rf_wreq	output	wire
i_rf_ready	input	wire
o_wreg0	output	wire [4+WITH_CSR:0]

포트	방향	형/폭
o_wreg1	output	wire [4+WITH_CSR:0]
o_wen0	output	wire
o_wen1	output	wire
o_wdata0	output	wire [B:0]
o_wdata1	output	wire [B:0]
o_rreg0	output	wire [4+WITH_CSR:0]
o_rreg1	output	wire [4+WITH_CSR:0]
i_rdata0	input	wire [B:0]
i_rdata1	input	wire [B:0]
o_ibus_adr	output	wire [31:0]
o_ibus_cyc	output	wire
i_ibus_rdt	input	wire [31:0]
i_ibus_ack	input	wire
o_dbus_adr	output	wire [31:0]
o_dbus_dat	output	wire [31:0]
o_dbus_sel	output	wire [3:0]
o_dbus_we	output	wire
o_dbus_cyc	output	wire
i_dbus_rdt	input	wire [31:0]
i_dbus_ack	input	wire
o_ext_func3	output	wire [2:0]
i_ext_ready	input	wire
i_ext_rd	input	wire [31:0]
o_ext_rs1	output	wire [31:0]
o_ext_rs2	output	wire [31:0]
o_mdu_valid	output	wire

1. olofk_serv/rtl/serv_aligner.v (73 lines)

```

1  /*
2  * serv_aligner.v : Realign a misaligned 32-bit word fetched
from memory
3  *
4  * SPDX-FileCopyrightText: 2022 Olof Kindgren <olof@award-win-
ning.me>
5  * SPDX-License-Identifier: ISC
6  */
7  module serv_aligner
8  (
9      input wire clk,
10     input wire rst,
11     // serv_top
12     input wire [31:0] i_ibus_adr,
13     input wire      i_ibus_cyc,
14     output wire [31:0] o_ibus_rdt,
15     output wire      o_ibus_ack,
16     // serv_rf_top
17     output wire [31:0] o_wb_ibus_adr,
18     output wire      o_wb_ibus_cyc,

```

```

19     input wire [31:0] i_wb_ibus_rdt,
20     input wire      i_wb_ibus_ack);
21
22     wire [31:0] ibus_rdt_concat;
23     wire      ack_en;
24
25     reg [15:0] lower_hw;
26     reg      ctrl_misal ;
27
28     /* From SERV core to Memory
29
30     o_wb_ibus_adr: Carries address of instruction to memory.
In case of misaligned access,
31     which is caused by pc+2 due to compressed instruction,
next instruction is fetched
32     by pc+4 and concatenation is done to make the instruction
aligned.
33
34     o_wb_ibus_cyc: Simply forwarded from SERV to Memory and is
only altered by memory or SERV core.
35     */

```

```

36 assign o_wb_ibus_adr = ctrl_misal ? (i_ibus_adr+32'b100) :
i_ibus_adr;
37 assign o_wb_ibus_cyc = i_ibus_cyc;
38
39 /* From Memory to SERV core
40
41 o_ibus_ack: Instruction bus acknowledge is send to
SERV only when the aligned instruction,
42 either compressed or un-compressed, is ready to dis-
patch.
43
44 o_ibus_rdt: Carries the instruction from memory to
SERV core. It can be either aligned
45 instruction coming from memory or made aligned by two
bus transactions and concatenation.
46 */
47 assign o_ibus_ack = i_wb_ibus_ack & ack_en;
48 assign o_ibus_rdt = ctrl_misal ? ibus_rdt_concat :
i_wb_ibus_rdt;
49
50 /* 16-bit register used to hold the upper half word of the
current instruction in-case
51 concatenation will be required with the upper half word of
upcoming instruction
52 */
53 always @(posedge clk) begin
54 if(i_wb_ibus_ack)begin
55 lower_hw <= i_wb_ibus_rdt[31:16];
56 end
57 end
58
59 assign ibus_rdt_concat = {i_wb_ibus_rdt[15:0], lower_hw};
60
61 /* Two control signals: ack_en, ctrl_misal are set to con-
trol the bus transactions between
62 SERV core and the memory
63 */
64 assign ack_en = !(i_ibus_adr[1] & !ctrl_misal);
65
66 always @(posedge clk ) begin
67 if(rst)
68 ctrl_misal <= 0;
69 else if(i_wb_ibus_ack & i_ibus_adr[1])
70 ctrl_misal <= !ctrl_misal;
71 end
72
73 endmodule

```

2. olof_serv/rtl/serv_alu.v (87 lines)

```

1 /*
2 * serv_alu.v : SERV Arithmetic Logic Unit
3 *
4 * SPDX-FileCopyrightText: 2018 Olof Kindgren <olof@award-win-
ning.me>
5 * SPDX-License-Identifier: ISC
6 */
7 `default_nettype none
8 module serv_alu
9 #
10 parameter W = 1,
11 parameter B = W-1
12 )
13 (
14 input wire clk,
15 //State
16 input wire i_en,
17 input wire i_cnt0,
18 output wire o_cmp,
19 //Control
20 input wire i_sub,
21 input wire [1:0] i_bool_op,
22 input wire i_cmp_eq,
23 input wire i_cmp_sig,
24 input wire [2:0] i_rd_sel,
25 //Data
26 input wire [B:0] i_rs1,
27 input wire [B:0] i_op_b,
28 input wire [B:0] i_buf,
29 output wire [B:0] o_rd;
30
31 wire [B:0] result_add;
32 wire [B:0] result_slt;
33
34 reg cmp_r;
35
36 wire add_cy;
37 reg [B:0] add_cy_r;
38
39 //Sign-extended operands
40 wire rs1_sx = i_rs1[B] & i_cmp_sig;
41 wire op_b_sx = i_op_b[B] & i_cmp_sig;
42
43 wire [B:0] add_b = i_op_b^i_sub;
44
45 assign {add_cy,result_add} = i_rs1+add_b+add_cy_r;
46
47 wire result_lt = rs1_sx + ~op_b_sx + add_cy;

```

```

48
49 wire result_eq = !(result_add) & (cmp_r | i_cnt0);
50
51 assign o_cmp = i_cmp_eq ? result_eq : result_lt;
52
53 /*
54 The result_bool expression implements the following opera-
tions between
55 i_rs1 and i_op_b depending on the value of i_bool_op
56
57 00 xor
58 01 0
59 10 or
60 11 and
61
62 i_bool_op will be 01 during shift operations, so by out-
putting zero under
63 this condition we can safely or result_bool with i_buf
64 */
65 wire [B:0] result_bool = ((i_rs1 ^ i_op_b) &
~{i_bool_op[0]}) | ({i_bool_op[1]} & i_op_b & i_rs1);
66
67 assign result_slt[0] = cmp_r & i_cnt0;
68 generate
69 if (W>1) begin : gen_w_gt_1
70 assign result_slt[B:1] = {B{1'b0}};
71 end
72 endgenerate
73
74 assign o_rd = i_buf |
({i_rd_sel[0]} & result_add) |
({i_rd_sel[1]} & result_slt) |
({i_rd_sel[2]} & result_bool);
75
76 always @(posedge clk) begin
77 add_cy_r <= {W{1'b0}};
78 add_cy_r[0] <= i_en ? add_cy : i_sub;
79
80 if (i_en)
81 cmp_r <= o_cmp;
82 end
83
84 endmodule

```

3. olof_serv/rtl/serv_bufreg.v (101 lines)

```

1 /*
2 * serv_bufreg.v : SERV buffer register for load/store address and
shift data
3 *
4 * SPDX-FileCopyrightText: 2019 Olof Kindgren <olof@award-win-
ning.me>
5 * SPDX-License-Identifier: ISC
6 */
7 module serv_bufreg #(
8 parameter [0:0] MDU = 0,
9 parameter W = 1,
10 parameter B = W-1
11 )
12 (
13 input wire i_clk,
14 //State
15 input wire i_cnt0,
16 input wire i_cnt1,
17 input wire i_cnt_done,
18 input wire i_en,
19 input wire i_init,
20 input wire i_mdu_op,
21 output wire [1:0] o_lsb,
22 //Control
23 input wire i_rs1_en,
24 input wire i_imm_en,
25 input wire i_clr_lsb,
26 input wire i_shift_op,
27 input wire i_right_shift_op,
28 input wire [2:0] i_shamt,
29 input wire i_sh_signed,
30 //Data
31 input wire [B:0] i_rs1,
32 input wire [B:0] i_imm,
33 output wire [B:0] o_q,
34 //External
35 output wire [31:0] o_dbus_adr,
36 //Extension
37 output wire [31:0] o_ext_rs1;
38
39 wire c;
40 wire [B:0] q;
41 reg [B:0] c_r;
42 reg [31:0] data;
43 wire [B:0] clr_lsb;
44
45 assign clr_lsb[0] = i_cnt0 & i_clr_lsb;
46
47 generate
48 if (W > 1) begin : gen_clr_lsb_w_gt_1
49 assign clr_lsb[B:1] = {B{1'b0}};
50 end
51 endgenerate

```

```

52  assign {c,q} = {1'b0,(i_rs1 & {W{i_rs1_en}})} + {1'b0,
(i_imm & {W{i_imm_en}} & ~clr_lsb)} + c_r;
53
54  always @(posedge i_clk) begin
55      //Make sure carry is cleared before loading new data
56      c_r <= {W{1'b0}};
57      c_r[0] <= c & i_en;
58  end
59
60  generate
61      if (W == 1) begin : gen_w_eq_1
62          always @(posedge i_clk) begin
63              if (i_en)
64                  data[31:2] <= {i_init ? q : {W{data[31] &
i_sh_signed}}, data[31:3]};
65
66                  if (i_init ? (i_cnt0 | i_cnt1) : i_en)
67                      data[1:0] <= {i_init ? q : data[2], data[1]};
68              end
69              assign o_lsb = (MDU & i_mdu_op) ? 2'b00 : data[1:0];
70              assign o_q = data[0] & {W{i_en}};
71          end else if (W == 4) begin : gen_lsb_w_4
72              reg [1:0] lsb;
73              reg [W-2:0] data_tail;
74
75              wire [2:0] shift_amount
76              = !i_shift_op ? 3'd3 :
77                i_right_shift_op ? (3'd3+{1'b0,i_shamt[1:0]}) :
78                ({1'b0,~i_shamt[1:0]});
79
80              always @(posedge i_clk) begin
81                  if (i_en)
82                      if (i_cnt0) lsb <= q[1:0];
83                      if (i_en)
84                          data <= {i_init ? q : {W{i_sh_signed &
data[31]}}, data[31:W]};
85                      if (i_en)
86                          data_tail <= data[B:1] & {B{~i_cnt_done}};
87                  end
88
89                  wire [2*W+B-2:0] muxdata = {data[W+B-1:0],data_tail};
90                  wire [B:0] muxout = muxdata[{1'b0,shift_amount}
+:W];
91
92                  assign o_lsb = (MDU & i_mdu_op) ? 2'b00 : lsb;
93                  assign o_q = i_en ? muxout : {W{1'b0}};
94              end
95          endgenerate
96
97          assign o_dbus_adr = {data[31:2], 2'b00};
98          assign o_ext_rs1 = data;
99
100      endmodule
101

```

4. olofk_serv/rtl/serv_bufreg2.v (105 lines)

```

1  /*
2  * serv_bufreg2.v : SERV buffer register for load/store data
and shift amount
3  *
4  * SPDX-FileCopyrightText: 2022 Olof Kindgren <olof@award-win-
ning.me>
5  * SPDX-License-Identifier: ISC
6  */
7  module serv_bufreg2
8      #(parameter W = 1,
9        //Internally calculated. Do not touch
10         parameter B=W-1)
11  (
12      input wire        i_clk,
13      //State
14      input wire        i_en,
15      input wire        i_init,
16      input wire        i_cnt7,
17      input wire        i_cnt_done,
18      input wire        i_sh_right,
19      input wire [1:0]  i_lsb,
20      input wire [1:0]  i_bytecnt,
21      output wire       o_sh_done,
22      //Control
23      input wire        i_op_b_sel,
24      input wire        i_shift_op,
25      //Data
26      input wire [B:0]  i_rs2,
27      input wire [B:0]  i_imm,
28      output wire [B:0] o_op_b,
29      output wire [B:0] o_q,
30      //External
31      output wire [31:0] o_dat,
32      input wire        i_load,
33      input wire [31:0] i_dat);
34
35      // High and low data words form a 32-bit word
36      reg [7:0] dhi;
37      reg [23:0] dlo;
38
39      /*
40      Before a store operation, the data to be written needs to
be shifted into

```

```

41      place. Depending on the address alignment, we need to
shift different
42      amounts. One formula for calculating this is to say that
we shift when
43      i_lsb + i_bytecnt < 4. Unfortunately, the synthesis tools
don't seem to be
44      clever enough so the hideous expression below is used to
achieve the same
45      thing in a more optimal way.
46      */
47      wire byte_valid
48      = (!i_lsb[0] & !i_lsb[1]) |
49        (!i_bytecnt[0] & !i_bytecnt[1]) |
50        (!i_bytecnt[1] & !i_lsb[1]) |
51        (!i_bytecnt[1] & !i_lsb[0]) |
52        (!i_bytecnt[0] & !i_lsb[1]);
53
54      assign o_op_b = i_op_b_sel ? i_rs2 : i_imm;
55
56      wire        shift_en = i_shift_op ? (i_en & i_init &
(i_bytecnt == 2'b00)) : (i_en & byte_valid);
57
58      wire        cnt_en = (i_shift_op & (!i_init | (i_cnt_done
& i_sh_right)));
59
60      /* The dat register has three different use cases for
store, load and
61      shift operations.
62      store : Data to be written is shifted to the correct posi-
tion in dat during
63      init by shift_en and is presented on the data bus
as o_wb_dat
64      load  : Data from the bus gets latched into dat during
i_wb_ack and is then
65      shifted out at the appropriate time to end up in
the correct
66      position in rd
67      shift : Data is shifted in during init. After that, the
six LSB are used as
68
a downcounter (with bit 5 initially set to 0) that
trigger
69      o_sh_done when they wrap around to indicate that
the requested number of shifts have been performed
70
71      */
72
73      wire [7:0] cnt_next;
74      generate
75          if (W == 1) begin : gen_cnt_w_eq_1
76              assign cnt_next = {o_op_b, dhi[7], dhi[5:0]-6'd1};
77          end else if (W == 4) begin : gen_cnt_w_eq_4
78              assign cnt_next = {o_op_b[3:2], dhi[5:0]-6'd4};
79          end
80      endgenerate
81
82      wire [7:0] dat_shamt = cnt_en ?
83      //Down counter mode
84      cnt_next :
85      //Shift reg mode
86      {o_op_b, dhi[7:W]};
87
88      assign o_sh_done = dat_shamt[5];
89
90      assign o_q =
91          ({W{(i_lsb == 2'd3)}} & o_dat[W+23:24]) |
92          ({W{(i_lsb == 2'd2)}} & o_dat[W+15:16]) |
93          ({W{(i_lsb == 2'd1)}} & o_dat[W+7:8]) |
94          ({W{(i_lsb == 2'd0)}} & o_dat[W-1:0]);
95
96      assign o_dat = {dhi,dlo};
97
98      always @(posedge i_clk) begin
99          if (shift_en | cnt_en | i_load)
100              dhi <= i_load ? i_dat[31:24] : dat_shamt & {2'b11, !
(i_shift_op & i_cnt7 & !cnt_en), 5'b11111};
101          if (shift_en | i_load)
102              dlo <= i_load ? i_dat[23:0] : {dhi[B:0], dlo[23:W]};
103      end
104
105  endmodule

```

5. olofk_serv/rtl/serv_compdec.v (232 lines)

```

1  /* Copyright lowRISC contributors.
2  Copyright 2018 ETH Zurich and University of Bologna, see also
CREDTIS.md.
3  Licensed under the Apache License, Version 2.0, see LICENSE
for details.
4  SPDX-License-Identifier: Apache-2.0
5
6  * Adapted to SERV by @Abdulwadoodd as part of the project un-
der spring '22 LFX Mentorship program */
7
8  /* Decodes RISC-V compressed instructions into their RV32i
equivalent. */
9
10 module serv_compdec
11 (
12     input wire i_clk,
13     input wire [31:0] i_instr,

```

```

14  input wire i_ack,
15  output wire [31:0] o_instr,
16  output reg o_iscomp;
17
18  localparam OPCODE_LOAD    = 7'h03;
19  localparam OPCODE_OP_IMM  = 7'h13;
20  localparam OPCODE_STORE   = 7'h23;
21  localparam OPCODE_OP      = 7'h33;
22  localparam OPCODE_LUI     = 7'h37;
23  localparam OPCODE_BRANCH  = 7'h63;
24  localparam OPCODE_JALR    = 7'h67;
25  localparam OPCODE_JAL     = 7'h6f;
26
27  reg [31:0] comp_instr;
28  reg illegal_instr;
29
30  assign o_instr = illegal_instr ? i_instr : comp_instr;
31
32  always @(posedge i_clk) begin
33    if(i_ack)
34      o_iscomp <= !illegal_instr;
35  end
36
37  always @ (*) begin
38    // By default, forward incoming instruction, mark it as
39    legal.
40    comp_instr = i_instr;
41    illegal_instr = 1'b0;
42
43    // Check if incoming instruction is compressed.
44    case (i_instr[1:0])
45      // C0
46      2'b00: begin
47        case (i_instr[15:14])
48          2'b00: begin
49            // c.addi4spn -> addi rd', x2, imm
50            comp_instr = {2'b0, i_instr[10:7], i_instr[12:11],
51            i_instr[5],
52
53            i_instr[6], 2'b00, 5'h02, 3'b000, 2'b01,
54            i_instr[4:2], {OPCODE_OP_IMM}};
55            end
56            2'b01: begin
57              // c.lw -> lw rd', imm(rs1')
58              comp_instr = {5'b0, i_instr[5], i_instr[12:10],
59              i_instr[6],
60              2'b00, 2'b01, i_instr[9:7], 3'b010,
61              2'b01, i_instr[4:2], {OPCODE_LOAD}};
62              end
63              2'b11: begin
64                // c.sw -> sw rs2', imm(rs1')
65                comp_instr = {5'b0, i_instr[5], i_instr[12],
66                2'b01, i_instr[4:2],
67                2'b01, i_instr[9:7], 3'b010, i_in-
68                str[11:10], i_instr[6],
69                2'b00, {OPCODE_STORE}};
70                end
71                2'b10: begin
72                  illegal_instr = 1'b1;
73                end
74              endcase
75            end
76            // C1
77            // Register address checks for RV32E are performed in
78            the regular instruction decoder.
79
80            // If this check fails, an illegal instruction exception is
81            triggered and the controller
82            // writes the actual faulting instruction to mtval.
83            2'b01: begin
84              case (i_instr[15:13])
85                3'b000: begin
86                  // c.addi -> addi rd, rd, nzimm
87                  // c.nop
88                  comp_instr = {{6 {i_instr[12]}}, i_instr[12],
89                  i_instr[6:2],
90                  i_instr[11:7], 3'b0, i_instr[11:7], {OP-
91                  CODE_OP_IMM}};
92                  end
93                  3'b001, 3'b101: begin
94                    // 001: c.jal -> jal x1, imm
95                    // 101: c.j -> jal x0, imm
96                    comp_instr = {i_instr[12], i_instr[8], i_in-
97                    str[10:9], i_instr[6],
98                    i_instr[7], i_instr[2], i_instr[11],
99                    i_instr[5:3],
100                   {9 {i_instr[12]}}, 4'b0, ~i_instr[15],
101                   {OPCODE_JAL}};
102                   end
103                   3'b010: begin
104                     // c.li -> addi rd, x0, nzimm
105                     // (c.li hints are translated into an addi hint)

```

```

98     comp_instr = {{6 {i_instr[12]}}, i_instr[12],
99     i_instr[6:2], 5'b0,
100     3'b0, i_instr[11:7], {OPCODE_OP_IMM}};
101   end
102   3'b011: begin
103     // c.lui -> lui rd, imm
104     // (c.lui hints are translated into a lui hint)
105     comp_instr = {{15 {i_instr[12]}}, i_instr[6:2],
106     i_instr[11:7], {OPCODE_LUI}};
107   end
108   if (i_instr[11:7] == 5'h02) begin
109     // c.addi16sp -> addi x2, x2, nzimm
110     comp_instr = {{3 {i_instr[12]}}, i_instr[4:3],
111     i_instr[5], i_instr[2],
112     i_instr[6], 4'b0, 5'h02, 3'b000,
113     5'h02, {OPCODE_OP_IMM}};
114   end
115   end
116   3'b100: begin
117     case (i_instr[11:10])
118       2'b00,
119       2'b01: begin
120         // 00: c.srli -> srli rd, rd, shamt
121         // 01: c.srai -> srai rd, rd, shamt
122         // (c.srli/c.srai hints are translated into a
123         srli/srai hint)
124         comp_instr = {1'b0, i_instr[10], 5'b0, i_in-
125         str[6:2], 2'b01, i_instr[9:7],
126         3'b101, 2'b01, i_instr[9:7], {OP-
127         CODE_OP_IMM}};
128       end
129       2'b10: begin
130         // c.andi -> andi rd, rd, imm
131         comp_instr = {{6 {i_instr[12]}}, i_instr[12],
132         i_instr[6:2], 2'b01, i_instr[9:7],
133         3'b111, 2'b01, i_instr[9:7], {OP-
134         CODE_OP_IMM}};
135       end
136       2'b11: begin
137         case (i_instr[6:5])
138           2'b00: begin
139             // c.sub -> sub rd', rd', rs2'
140             comp_instr = {2'b01, 5'b0, 2'b01, i_in-
141             str[4:2], 2'b01, i_instr[9:7],
142             3'b000, 2'b01, i_instr[9:7],
143             {OPCODE_OP}};
144           end
145           2'b01: begin
146             // c.xor -> xor rd', rd', rs2'
147             comp_instr = {7'b0, 2'b01, i_instr[4:2],
148             2'b01, i_instr[9:7], 3'b100,
149             2'b01, i_instr[9:7], {OP-
150             CODE_OP}};
151           end
152           2'b10: begin
153             // c.or -> or rd', rd', rs2'
154             comp_instr = {7'b0, 2'b01, i_instr[4:2],
155             2'b01, i_instr[9:7], 3'b110,
156             2'b01, i_instr[9:7], {OP-
157             CODE_OP}};
158           end
159           2'b11: begin
160             // c.and -> and rd', rd', rs2'
161             comp_instr = {7'b0, 2'b01, i_instr[4:2],
162             2'b01, i_instr[9:7], 3'b111,
163             2'b01, i_instr[9:7], {OP-
164             CODE_OP}};
165           end
166           3'b100, 3'b111: begin
167             // 0: c.beqz -> beq rs1', x0, imm
168             // 1: c.bnez -> bne rs1', x0, imm
169             comp_instr = {{4 {i_instr[12]}}, i_instr[6:5],
170             i_instr[2], 5'b0, 2'b01,
171             i_instr[9:7], 2'b00, i_instr[13], i_in-
172             str[11:10], i_instr[4:3],
173             i_instr[12], {OPCODE_BRANCH}};
174           end
175         endcase
176       end
177     // C2
178     // Register address checks for RV32E are performed in
179     the regular instruction decoder.
180
181     // If this check fails, an illegal instruction exception
182     is triggered and the controller

```



```

176 // writes the actual faulting instruction to mtval.
177 2'b10: begin
178   case (i_instr[15:14])
179     2'b00: begin
180       // c.slli -> slli rd, rd, shamt
181       // (c.slli hints are translated into a slli hint)
182       comp_instr = {7'b0, i_instr[6:2], i_instr[11:7],
3'b001, i_instr[11:7], {OPCODE_OP_IMM}};
183     end
184     2'b01: begin
185       // c.lwsp -> lw rd, imm(x2)
186       comp_instr = {4'b0, i_instr[3:2], i_instr[12],
i_instr[6:4], 2'b00, 5'h02,
187       3'b010, i_instr[11:7], OPCODE_LOAD};
188     end
189     2'b10: begin
190       if (i_instr[12] == 1'b0) begin
191         if (i_instr[6:2] != 5'b0) begin
192           // c.mv -> add rd/rs1, x0, rs2
193           // (c.mv hints are translated into an add
hint)
194           comp_instr = {7'b0, i_instr[6:2], 5'b0, 3'b0,
i_instr[11:7], {OPCODE_OP}};
195         end else begin
196           // c.jr -> jalr x0, rd/rs1, 0
197           comp_instr = {12'b0, i_instr[11:7], 3'b0,
5'b0, {OPCODE_JALR}};
198         end
199       end else begin
200         if (i_instr[6:2] != 5'b0) begin
201           // c.add -> add rd, rd, rs2
202           // (c.add hints are translated into an add
hint)
203           comp_instr = {7'b0, i_instr[6:2], i_in-
str[11:7], 3'b0, i_instr[11:7], {OPCODE_OP}};
204         end else begin
205           if (i_instr[11:7] == 5'b0) begin
206             // c.ebreak -> ebreak
207             comp_instr = {32'h00_10_00_73};
208           end else begin
209             // c.jalr -> jalr x1, rs1, 0
210             comp_instr = {12'b0, i_instr[11:7], 3'b000,
5'b00001, {OPCODE_JALR}};
211           end
212         end
213       end
214     end
215   endcase
216 end
217
218 2'b11: begin
219   // c.swsp -> sw rs2, imm(x2)
220   comp_instr = {4'b0, i_instr[8:7], i_instr[12],
i_instr[6:2], 5'h02, 3'b010,
221   i_instr[11:9], 2'b00, {OPCODE_STORE}};
222 end
223 endcase
224 end
225
226 // Incoming instruction is not compressed.
227 2'b11: illegal_instr = 1'b1;
228
229 endcase
230 end
231
232 endmodule

```

6. olofk_serv/rtl/serv_csr.v (164 lines)

```

1 /*
2  * serv_csr.v : SERV module for handling CSR registers
3  *
4  * SPDX-FileCopyrightText: 2018 Olof Kindgren <olof@award-win-
ning.me>
5  * SPDX-License-Identifier: ISC
6  */
7 `default_nettype none
8 module serv_csr
9   #(
10     parameter RESET_STRATEGY = "MINI",
11     parameter W = 1,
12     parameter B = W-1
13   )
14   (
15     input wire i_clk,
16     input wire i_rst,
17     //State
18     input wire i_trig_irq,
19     input wire i_en,
20     input wire i_cnt0to3,
21     input wire i_cnt3,
22     input wire i_cnt7,
23     input wire i_cnt11,
24     input wire i_cnt12,
25     input wire i_cnt_done,
26     input wire i_mem_op,
27     input wire i_mtip,
28     input wire i_trap,
29     output reg o_new_irq,
30     //Control

```

```

31     input wire i_e_op,
32     input wire i_ebreak,
33     input wire i_mem_cmd,
34     input wire i_mstatus_en,
35     input wire i_mie_en,
36     input wire i_mcause_en,
37     input wire [1:0] i_csr_source,
38     input wire i_mret,
39     input wire i_csr_d_sel,
40     //Data
41     input wire [B:0] i_rf_csr_out,
42     output wire [B:0] o_csr_in,
43     input wire [B:0] i_csr_imm,
44     input wire [B:0] i_rs1,
45     output wire [B:0] o_q;
46
47     localparam [1:0]
48       CSR_SOURCE_CSR = 2'b00,
49       CSR_SOURCE_EXT = 2'b01,
50       CSR_SOURCE_SET = 2'b10,
51       CSR_SOURCE_CLR = 2'b11;
52
53     reg mstatus_mie;
54     reg mstatus_mpie;
55     reg mie_mtie;
56
57     reg mcause31;
58     reg [3:0] mcause3_0;
59     wire [B:0] mcause;
60
61     wire [B:0] csr_in;
62     wire [B:0] csr_out;
63
64     reg timer_irq_r;
65
66     wire [B:0] d = i_csr_d_sel ? i_csr_imm : i_rs1;
67
68     assign csr_in = (i_csr_source == CSR_SOURCE_EXT) ? d :
69
70     (i_csr_source == CSR_SOURCE_SET) ? csr_out |
d :
71
72     (i_csr_source == CSR_SOURCE_CLR) ? csr_out &
~d :
73
74     (i_csr_source == CSR_SOURCE_CSR) ? csr_out :
{W{1'b0}};
75
76     wire [B:0] mstatus;
77
78     generate
79       if (W==1) begin : gen_mstatus_w1
80         assign mstatus = ((mstatus_mie & i_cnt3) | (i_cnt11 |
i_cnt12));
81       end else if (W==4) begin : gen_mstatus_w4
82         assign mstatus = {i_cnt11 | (mstatus_mie & i_cnt3),
2'b00, i_cnt12};
83       end
84     endgenerate
85
86     assign csr_out = ({W{i_mstatus_en & i_en}} & mstatus) |
i_rf_csr_out |
87     ({W{i_mcause_en & i_en}} & mcause);
88
89     assign o_q = csr_out;
90
91     wire timer_irq = i_mtip & mstatus_mie & mie_mtie;
92
93     assign mcause = i_cnt0to3 ? mcause3_0[B:0] : {[3:0]
i_cnt_done ? {mcause31, {B{1'b0}}} }/[31]
94     : {W{1'b0}};
95
96     assign o_csr_in = csr_in;
97
98     always @(posedge i_clk) begin
99       if (i_trig_irq) begin
100         timer_irq_r <= timer_irq;
101         o_new_irq <= timer_irq & !timer_irq_r;
102       end
103
104       if (i_mie_en & i_cnt7)
105         mie_mtie <= csr_in[B];
106
107       /*
108        * The mie bit in mstatus gets updated under three condi-
tions
109        *
110        * When a trap is taken, the bit is cleared
111        * During an mret instruction, the bit is restored from
mpie
112        * During a mstatus CSR access instruction it's assigned
when
113        * bit 3 gets updated
114        *
115        * These conditions are all mutually exclusive
116        */
117       if ((i_trap & i_cnt_done) | i_mstatus_en & i_cnt3 & i_en |
i_mret)
118         mstatus_mie <= !i_trap & (i_mret ? mstatus_mpie :
csr_in[B]);
119

```

```

120      /*
121      Note: To save resources mstatus_mpie (mstatus bit 7) is
not
122      readable or writable from sw
123      */
124      if (i_trap & i_cnt_done)
125          mstatus_mpie <= mstatus_mie;
126
127      /*
128      The four lowest bits in mcause hold the exception code
129
130      These bits get updated under three conditions
131
132      During an mcause CSR access function, they are assigned
when
133      bits 0 to 3 gets updated
134
135      During an external interrupt the exception code is set
to
136      7, since SERV only support timer interrupts
137
138      During an exception, the exception code is assigned to
indicate
139      if it was caused by an ebreak instruction (3),
140
141      ecall instruction (11), misaligned load (4), misaligned
store (6)
142      or misaligned jump (0)
143
144      The expressions below are derived from the following
truth table
145      irq => 0111 (timer=7)
146      e_op => x011 (ebreak=3, ecall=11)
147      mem => 01x0 (store=6, load=4)
148      ctrl => 0000 (jump=0)
149      */
150      if (i_mcause_en & i_en & i_cnt0to3 | (i_trap &
i_cnt_done)) begin
151          mcause3_0[3] <= (i_e_op & !i_ebreak) | (!i_trap &
csr_in[B]);
152          mcause3_0[2] <= o_new_irq | i_mem_op | (!i_trap & ((W
== 1) ? mcause3_0[3] : csr_in[(W == 1) ? 0 : 2]));
153          mcause3_0[1] <= o_new_irq | i_e_op | (i_mem_op &
i_mem_cmd) | (!i_trap & ((W == 1) ? mcause3_0[2] : csr_in[(W == 1) ?
0 : 1]));
154          mcause3_0[0] <= o_new_irq | i_e_op | (!i_trap & ((W ==
1) ? mcause3_0[1] : csr_in[0]));
155      end
156      if (i_mcause_en & i_cnt_done | i_trap)
157          mcause31 <= i_trap ? o_new_irq : csr_in[B];
158      if (i_rst)
159          if (RESET_STRATEGY != "NONE") begin
160              o_new_irq <= 1'b0;
161              mie_mtie <= 1'b0;
162          end
163      end
164  endmodule

```

7. olofk_serv/rtl/serv_ctrl.v (115 lines)

```

1  /*
2  * serv_ctrl.v : SERV module for updating program counter
3  *
4  * SPDX-FileCopyrightText: 2018 Olof Kindgren <olof@award-win-
ning.me>
5  * SPDX-License-Identifier: ISC
6  */
7  `default_nettype none
8  module serv_ctrl
9  #(parameter RESET_STRATEGY = "MINI",
10     parameter RESET_PC = 32'd0,
11     parameter WITH_CSR = 1,
12     parameter W = 1,
13     parameter B = W-1
14 )
15 (
16     input wire      clk,
17     input wire      i_rst,
18     //State
19     input wire      i_pc_en,
20     input wire      i_cnt12to31,
21     input wire      i_cnt0,
22     input wire      i_cnt1,
23     input wire      i_cnt2,
24     //Control
25     input wire      i_jump,
26     input wire      i_jal_or_jalr,
27     input wire      i_ctype,
28     input wire      i_pc_rel,
29     input wire      i_trap,
30     input wire      i_iscomp,
31     //Data
32     input wire [B:0] i_imm,
33     input wire [B:0] i_buf,
34     input wire [B:0] i_csr_pc,
35     output wire [B:0] o_rd,

```

```

36     output wire [B:0] o_bad_pc,
37     //External
38     output reg [31:0] o_ibus_adr;
39
40     wire [B:0] pc_plus_4;
41     wire      pc_plus_4_cy;
42     reg       pc_plus_4_cy_r;
43     wire [B:0] pc_plus_4_cy_r_w;
44     wire [B:0] pc_plus_offset;
45     wire      pc_plus_offset_cy;
46     reg       pc_plus_offset_cy_r;
47     wire [B:0] pc_plus_offset_cy_r_w;
48     wire [B:0] pc_plus_offset_aligned;
49     wire [B:0] plus_4;
50
51     wire [B:0] pc = o_ibus_adr[B:0];
52
53     wire [B:0] new_pc;
54
55     wire [B:0] offset_a;
56     wire [B:0] offset_b;
57
58     /* If i_iscomp=1: increment pc by 2 else increment pc by 4 */
59     generate
60     if (W == 1) begin : gen_plus_4_w_eq_1
61         assign plus_4 = i_iscomp ? i_cnt1 : i_cnt2;
62     end else if (W == 4) begin : gen_plus_4_w_eq_4
63         assign plus_4 = (i_cnt0 | i_cnt1) ? (i_iscomp ? 2 :
4) : 0;
64     end
65     endgenerate
66
67     assign o_bad_pc = pc_plus_offset_aligned;
68
69     assign {pc_plus_4_cy, pc_plus_4} =
pc_plus_4 + pc_plus_4_cy_r_w;
70
71     generate
72     if (!WITH_CSR) begin : gen_csr
73         if (W == 1) begin : gen_new_pc_w_eq_1
74             assign new_pc = i_trap ? (i_csr_pc & !(i_cnt0 |
i_cnt1)) : i_jump ? pc_plus_offset_aligned : pc_plus_4;
75         end else if (W == 4) begin : gen_new_pc_w_eq_4
76             assign new_pc = i_trap ? (i_csr_pc & ((i_cnt0 |
i_cnt1) ? 4'b1100 : 4'b1111)) : i_jump ? pc_plus_offset_aligned :
pc_plus_4;
77         end
78     end else begin : gen_no_csr
79         assign new_pc = i_jump ? pc_plus_offset_aligned :
pc_plus_4;
80     end
81     endgenerate
82
83     assign o_rd = ({W{i_ctype}} & pc_plus_offset_aligned) |
(pc_plus_4 & {W{i_jal_or_jalr}});
84
85     assign offset_a = {W{i_pc_rel}} & pc;
86     assign offset_b = i_ctype ? (i_imm & {W{i_cnt12to31}}) :
i_buf;
87
88     assign {pc_plus_offset_cy, pc_plus_offset} = off-
set_a + offset_b + pc_plus_offset_cy_r_w;
89
90     generate
91     if (W > 1) begin : gen_w_gt_1
92         assign pc_plus_offset_aligned[B:1] = pc_plus_offset[B:
1];
93         assign pc_plus_offset_cy_r_w[B:1] = {B{1'b0}};
94         assign pc_plus_4_cy_r_w[B:1] = {B{1'b0}};
95     end
96     endgenerate
97
98     assign pc_plus_offset_aligned[0] = pc_plus_offset[0] & !
i_cnt0;
99     assign pc_plus_offset_cy_r_w[0] = pc_plus_offset_cy_r;
100    assign pc_plus_4_cy_r_w[0] = pc_plus_4_cy_r;
101
102    initial if (RESET_STRATEGY == "NONE") o_ibus_adr = RE-
SET_PC;
103
104    always @(posedge clk) begin
105        pc_plus_4_cy_r <= i_pc_en & pc_plus_4_cy;
106        pc_plus_offset_cy_r <= i_pc_en & pc_plus_offset_cy;
107
108        if (RESET_STRATEGY == "NONE") begin
109            if (i_pc_en)
110                o_ibus_adr <= {new_pc, o_ibus_adr[31:W]};
111        end else begin
112            if (i_pc_en | i_rst)
113                o_ibus_adr <= i_rst ? RESET_PC : {new_pc,
o_ibus_adr[31:W]};
114        end
115    endmodule

```

8. olofk_serv/rtl/serv_debug.v (351 lines)

```

1  /*
2  * serv_debug.v : SERV module for introspecting CPU/RF state
during simulations

```

```

3 *
4 * SPDX-FileCopyrightText: 2024 Olof Kindgren <olof@award-win-
ning.me>
5 * SPDX-License-Identifier: ISC
6 */
7 module serv_debug
8 # (parameter W = 1,
9   parameter RESET_PC = 0,
10  // Internally calculated. Do not touch
11   parameter B=W-1)
12 (
13   `ifdef RISC_V_FORMAL
14     output reg rvfi_valid = 1'b0,
15     output reg [63:0] rvfi_order = 64'd0,
16     output reg [31:0] rvfi_insn = 32'd0,
17     output reg rvfi_trap = 1'b0,
18     output reg rvfi_halt = 1'b0, // Not used
19     output reg rvfi_intr = 1'b0, // Not used
20     output reg [1:0] rvfi_mode = 2'b11, // Not used
21     output reg [1:0] rvfi_ixl = 2'b01, // Not used
22     output reg [4:0] rvfi_rs1_addr,
23     output reg [4:0] rvfi_rs2_addr,
24     output reg [31:0] rvfi_rs1_rdata,
25     output reg [31:0] rvfi_rs2_rdata,
26     output reg [4:0] rvfi_rd_addr,
27     output wire [31:0] rvfi_rd_wdata,
28     output reg [31:0] rvfi_pc_rdata,
29     output wire [31:0] rvfi_pc_wdata,
30     output reg [31:0] rvfi_mem_addr,
31     output reg [3:0] rvfi_mem_rmask,
32     output reg [3:0] rvfi_mem_wmask,
33     output reg [31:0] rvfi_mem_rdata,
34     output reg [31:0] rvfi_mem_wdata,
35     input wire [31:0] i_dbus_adr,
36     input wire [31:0] i_dbus_dat,
37     input wire [3:0] i_dbus_sel,
38     input wire i_dbus_we,
39     input wire [31:0] i_dbus_rdt,
40     input wire i_dbus_ack,
41     input wire i_ctrl_pc_en,
42     input wire [B:0] rs1,
43     input wire [B:0] rs2,
44     input wire [4:0] rs1_addr,
45     input wire [4:0] rs2_addr,
46     input wire [3:0] immdec_en,
47     input wire rd_en,
48     input wire trap,
49     input wire i_rf_ready,
50     input wire i_ibus_cyc,
51     input wire two_stage_op,
52     input wire init,
53     input wire [31:0] i_ibus_adr,
54   `endif
55   input wire i_clk,
56   input wire i_rst,
57   input wire [31:0] i_ibus_rdt,
58   input wire i_ibus_ack,
59   input wire [4:0] i_rd_addr,
60   input wire i_cnt_en,
61   input wire [B:0] i_csr_in,
62   input wire i_csr_mstatus_en,
63   input wire i_csr_mie_en,
64   input wire i_csr_mcause_en,
65   input wire i_csr_en,
66   input wire [1:0] i_csr_addr,
67   input wire i_wen0,
68   input wire [B:0] i_wdata0,
69   input wire i_cnt_done);
70
71   reg update_rd = 1'b0;
72   reg update_mscratch;
73   reg update_mtvec;
74   reg update_mepc;
75   reg update_mtval;
76   reg update_mstatus;
77   reg update_mie;
78   reg update_mcause;
79
80   reg [31:0] dbg_rd = 32'hxxxxxxxx;
81   reg [31:0] dbg_csr = 32'hxxxxxxxx;
82   reg [31:0] dbg_mstatus = 32'hxxxxxxxx;
83   reg [31:0] dbg_mie = 32'hxxxxxxxx;
84   reg [31:0] dbg_mcause = 32'hxxxxxxxx;
85   reg [31:0] dbg_mscratch = 32'hxxxxxxxx;
86   reg [31:0] dbg_mtvec = 32'hxxxxxxxx;
87   reg [31:0] dbg_mepc = 32'hxxxxxxxx;
88   reg [31:0] dbg_mtval = 32'hxxxxxxxx;
89   reg [31:0] x1 = 32'hxxxxxxxx;
90   reg [31:0] x2 = 32'hxxxxxxxx;
91   reg [31:0] x3 = 32'hxxxxxxxx;
92   reg [31:0] x4 = 32'hxxxxxxxx;
93   reg [31:0] x5 = 32'hxxxxxxxx;
94   reg [31:0] x6 = 32'hxxxxxxxx;
95   reg [31:0] x7 = 32'hxxxxxxxx;
96   reg [31:0] x8 = 32'hxxxxxxxx;
97   reg [31:0] x9 = 32'hxxxxxxxx;
98   reg [31:0] x10 = 32'hxxxxxxxx;
99   reg [31:0] x11 = 32'hxxxxxxxx;
100  reg [31:0] x12 = 32'hxxxxxxxx;
101  reg [31:0] x13 = 32'hxxxxxxxx;
102  reg [31:0] x14 = 32'hxxxxxxxx;

```

```

103  reg [31:0] x15 = 32'hxxxxxxxx;
104  reg [31:0] x16 = 32'hxxxxxxxx;
105  reg [31:0] x17 = 32'hxxxxxxxx;
106  reg [31:0] x18 = 32'hxxxxxxxx;
107  reg [31:0] x19 = 32'hxxxxxxxx;
108  reg [31:0] x20 = 32'hxxxxxxxx;
109  reg [31:0] x21 = 32'hxxxxxxxx;
110  reg [31:0] x22 = 32'hxxxxxxxx;
111  reg [31:0] x23 = 32'hxxxxxxxx;
112  reg [31:0] x24 = 32'hxxxxxxxx;
113  reg [31:0] x25 = 32'hxxxxxxxx;
114  reg [31:0] x26 = 32'hxxxxxxxx;
115  reg [31:0] x27 = 32'hxxxxxxxx;
116  reg [31:0] x28 = 32'hxxxxxxxx;
117  reg [31:0] x29 = 32'hxxxxxxxx;
118  reg [31:0] x30 = 32'hxxxxxxxx;
119  reg [31:0] x31 = 32'hxxxxxxxx;
120
121  always @(posedge i_clk) begin
122    update_rd <= i_cnt_done & i_wen0;
123
124    if (i_wen0)
125      dbg_rd <= {i_wdata0, dbg_rd[31:W]};
126
127    // End of instruction that writes to RF
128    if (update_rd) begin
129      case (i_rd_addr)
130        5'd1 : x1 <= dbg_rd;
131        5'd2 : x2 <= dbg_rd;
132        5'd3 : x3 <= dbg_rd;
133        5'd4 : x4 <= dbg_rd;
134        5'd5 : x5 <= dbg_rd;
135        5'd6 : x6 <= dbg_rd;
136        5'd7 : x7 <= dbg_rd;
137        5'd8 : x8 <= dbg_rd;
138        5'd9 : x9 <= dbg_rd;
139        5'd10 : x10 <= dbg_rd;
140        5'd11 : x11 <= dbg_rd;
141        5'd12 : x12 <= dbg_rd;
142        5'd13 : x13 <= dbg_rd;
143        5'd14 : x14 <= dbg_rd;
144        5'd15 : x15 <= dbg_rd;
145        5'd16 : x16 <= dbg_rd;
146        5'd17 : x17 <= dbg_rd;
147        5'd18 : x18 <= dbg_rd;
148        5'd19 : x19 <= dbg_rd;
149        5'd20 : x20 <= dbg_rd;
150        5'd21 : x21 <= dbg_rd;
151        5'd22 : x22 <= dbg_rd;
152        5'd23 : x23 <= dbg_rd;
153        5'd24 : x24 <= dbg_rd;
154        5'd25 : x25 <= dbg_rd;
155        5'd26 : x26 <= dbg_rd;
156        5'd27 : x27 <= dbg_rd;
157        5'd28 : x28 <= dbg_rd;
158        5'd29 : x29 <= dbg_rd;
159        5'd30 : x30 <= dbg_rd;
160        5'd31 : x31 <= dbg_rd;
161        default : ;
162      endcase
163    end
164
165    update_mscratch <= i_cnt_done & i_csr_en & (i_csr_addr
166    == 2'b00);
167    update_mtvec <= i_cnt_done & i_csr_en & (i_csr_addr
168    == 2'b01);
169    update_mepc <= i_cnt_done & i_csr_en & (i_csr_addr
170    == 2'b10);
171    update_mtval <= i_cnt_done & i_csr_en & (i_csr_addr
172    == 2'b11);
173    update_mstatus <= i_cnt_done & i_csr_mstatus_en;
174    update_mie <= i_cnt_done & i_csr_mie_en;
175    update_mcause <= i_cnt_done & i_csr_mcause_en;
176
177    if (i_cnt_en)
178      dbg_csr <= {i_csr_in, dbg_csr[31:W]};
179
180    if (update_mscratch) dbg_mscratch <= dbg_csr;
181    if (update_mtvec) dbg_mtvec <= dbg_csr;
182    if (update_mepc) dbg_mepc <= dbg_csr;
183    if (update_mtval) dbg_mtval <= dbg_csr;
184    if (update_mstatus) dbg_mstatus <= dbg_csr;
185    if (update_mie) dbg_mie <= dbg_csr;
186    if (update_mcause) dbg_mcause <= dbg_csr;
187  end
188
189  reg LUI, AUIPC, JAL, JALR, BEQ, BNE, BLT, BGE, BLTU, BGEU,
190  LB, LH, LW, LBU, SB, SH, SW, ADDI, SLTI, SLTIU, XORI, ORI,
191  ANDI, SLLI, SRLI, SRAI, ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR,
192  AND, FENCE, ECALL, EBREAK;
193  reg CSRWR, CSRRS, CSRRC, CSRRI, CSRRII, CSRRII, CSRRII;
194  reg OTHER;
195
196  always @(posedge i_clk) begin
197    if (i_ibus_ack) begin
198      LUI <= 1'b0;
199      AUIPC <= 1'b0;
200      JAL <= 1'b0;
201      JALR <= 1'b0;
202      BEQ <= 1'b0;
203      BNE <= 1'b0;

```

```

197     BLT    <= 1'b0;
198     BGE    <= 1'b0;
199     BLTU   <= 1'b0;
200     BGEU   <= 1'b0;
201     LB     <= 1'b0;
202     LH     <= 1'b0;
203     LW     <= 1'b0;
204     LBU    <= 1'b0;
205     LHU    <= 1'b0;
206     SB     <= 1'b0;
207     SH     <= 1'b0;
208     SW     <= 1'b0;
209     ADDI   <= 1'b0;
210     SLTI   <= 1'b0;
211     SLTIU  <= 1'b0;
212     XORI   <= 1'b0;
213     ORI    <= 1'b0;
214     ANDI   <= 1'b0;
215     SLLI   <= 1'b0;
216     SRLI   <= 1'b0;
217     SRAI   <= 1'b0;
218     ADD     <= 1'b0;
219     SUB     <= 1'b0;
220     SLL     <= 1'b0;
221     SLT     <= 1'b0;
222     SLTU    <= 1'b0;
223     XOR     <= 1'b0;
224     SRL     <= 1'b0;
225     SRA     <= 1'b0;
226     OR      <= 1'b0;
227     AND     <= 1'b0;
228     FENCE   <= 1'b0;
229     ECALL   <= 1'b0;
230     EBREAK  <= 1'b0;
231     CSRRW   <= 1'b0;
232     CSRRS   <= 1'b0;
233     CSRRC   <= 1'b0;
234     CSRRWI  <= 1'b0;
235     CSRRSI  <= 1'b0;
236     CSRRCI  <= 1'b0;
237     OTHER   <= 1'b0;
238
239     casez(i_ibus_rdt)
240         // 3322222_22222 11111_111 11
241         // 1098765_43210 98765_432 10987_65432_10
242         32'b????????_?????_?????_???_?????_01101_11 : LUI
243     <= 1'b1;
244         32'b????????_?????_?????_???_?????_00101_11 : AUIPC
245     <= 1'b1;
246         32'b????????_?????_?????_???_?????_11011_11 : JAL
247     <= 1'b1;
248         32'b????????_?????_?????_000_?????_11001_11 : JALR
249     <= 1'b1;
250         32'b????????_?????_?????_000_?????_11000_11 : BEQ
251     <= 1'b1;
252         32'b????????_?????_?????_001_?????_11000_11 : BNE
253     <= 1'b1;
254         32'b????????_?????_?????_100_?????_11000_11 : BLT
255     <= 1'b1;
256         32'b????????_?????_?????_101_?????_11000_11 : BGE
257     <= 1'b1;
258         32'b????????_?????_?????_110_?????_11000_11 : BLTU
259     <= 1'b1;
260         32'b????????_?????_?????_111_?????_11000_11 : BGEU
261     <= 1'b1;
262         32'b????????_?????_?????_000_?????_00000_11 : LB
263     <= 1'b1;
264         32'b????????_?????_?????_001_?????_00000_11 : LH
265     <= 1'b1;
266         32'b????????_?????_?????_010_?????_00000_11 : LW
267     <= 1'b1;
268         32'b????????_?????_?????_100_?????_00000_11 : LBU
269     <= 1'b1;
270         32'b????????_?????_?????_101_?????_00000_11 : LHU
271     <= 1'b1;
272         32'b????????_?????_?????_000_?????_01000_11 : SB
273     <= 1'b1;
274         32'b????????_?????_?????_001_?????_01000_11 : SH
275     <= 1'b1;
276         32'b????????_?????_?????_010_?????_01000_11 : SW
277     <= 1'b1;
278         32'b????????_?????_?????_000_?????_00100_11 : ADDI
279     <= 1'b1;
280         32'b????????_?????_?????_010_?????_00100_11 : SLTI
281     <= 1'b1;
282         32'b????????_?????_?????_011_?????_00100_11 : SLTIU
283     <= 1'b1;
284         32'b????????_?????_?????_100_?????_00100_11 : XORI
285     <= 1'b1;
286         32'b????????_?????_?????_110_?????_00100_11 : ORI
287     <= 1'b1;
288         32'b????????_?????_?????_111_?????_00100_11 : ANDI
289     <= 1'b1;
290         32'b00000000_?????_?????_001_?????_00100_11 : SLLI
291     <= 1'b1;
292         32'b00000000_?????_?????_101_?????_00100_11 : SRLI
293     <= 1'b1;
294         32'b0100000_?????_?????_101_?????_00100_11 : SRAI
295     <= 1'b1;
296         32'b00000000_?????_?????_000_?????_01100_11 : ADD
297
298     32'b0100000_?????_?????_000_?????_01100_11 : SUB
299     <= 1'b1;
300     32'b00000000_?????_?????_001_?????_01100_11 : SLL
301     <= 1'b1;
302     32'b00000000_?????_?????_010_?????_01100_11 : SLT
303     <= 1'b1;
304     32'b00000000_?????_?????_011_?????_01100_11 : SLTU
305     <= 1'b1;
306     32'b????????_?????_?????_100_?????_01100_11 : XOR
307     <= 1'b1;
308     32'b00000000_?????_?????_101_?????_01100_11 : SRL
309     <= 1'b1;
310     32'b0100000_?????_?????_101_?????_01100_11 : SRA
311     <= 1'b1;
312     32'b????????_?????_?????_110_?????_01100_11 : OR
313     <= 1'b1;
314     32'b????????_?????_?????_111_?????_01100_11 : AND
315     <= 1'b1;
316     32'b????????_?????_?????_000_?????_00011_11 : FENCE
317     <= 1'b1;
318     32'b00000000_00000_00000_000_00000_11100_11 : ECALL
319     <= 1'b1;
320     32'b00000000_00001_00000_000_00000_11100_11 : EBREAK
321     <= 1'b1;
322     32'b????????_?????_?????_001_?????_11100_11 : CSRRW
323     <= 1'b1;
324     32'b????????_?????_?????_010_?????_11100_11 : CSRRS
325     <= 1'b1;
326     32'b????????_?????_?????_011_?????_11100_11 : CSRRC
327     <= 1'b1;
328     32'b????????_?????_?????_101_?????_11100_11 : CSRRWI
329     <= 1'b1;
330     32'b????????_?????_?????_110_?????_11100_11 : CSRRSI
331     <= 1'b1;
332     32'b????????_?????_?????_111_?????_11100_11 : CSRRCI
333
334     default : OTHER <= 1'b1;
335 endcase
336 end
337 end
338
339 `ifdef RISCV_FORMAL
340     reg [31:0] pc = RESET_PC;
341
342     wire rs_en = two_stage_op ? init : i_ctrl_pc_en;
343
344     assign rvfi_rd_wdata = update_rd ? dbg_rd : 32'd0;
345
346     always @(posedge i_clk) begin
347         /* End of instruction */
348         rvfi_valid <= i_cnt_done & i_ctrl_pc_en & !i_rst;
349         rvfi_order <= rvfi_order + {63'd0,rvfi_valid};
350
351         /* Get instruction word when it's fetched from ibus */
352         if (i_ibus_cyc & i_ibus_ack)
353             rvfi_insn <= i_ibus_rdt;
354
355         if (i_cnt_done & i_ctrl_pc_en) begin
356             rvfi_pc_rdata <= pc;
357             if (!(rd_en & (!i_rd_addr))) begin
358                 rvfi_rd_addr <= 5'd0;
359             end
360         end
361         rvfi_trap <= trap;
362         if (rvfi_valid) begin
363             rvfi_trap <= 1'b0;
364             pc <= rvfi_pc_wdata;
365         end
366     end
367
368     /* RS1 not valid during J, U instructions (immdec_en[1]) */
369     /*
370     323     /* RS2 not valid during I, J, U instructions (imm-
371     dec_en[2]) */
372     324     if (i_rf_ready) begin
373     325         rvfi_rs1_addr <= !immdec_en[1] ? rs1_addr : 5'd0;
374     326         rvfi_rs2_addr <= !immdec_en[2] /*rs2_valid*/ ?
375     rs2_addr : 5'd0;
376     327         rvfi_rd_addr <= i_rd_addr;
377     328     end
378     329     if (rs_en) begin
379     330         rvfi_rs1_rdata <= {(!immdec_en[1] ? rs1 :
380     {W{1'b0}}),rvfi_rs1_rdata[31:W]};
381     331         rvfi_rs2_rdata <= {(!immdec_en[2] ? rs2 :
382     {W{1'b0}}),rvfi_rs2_rdata[31:W]};
383     332     end
384     333
385     334     if (i_dbus_ack) begin
386     335         rvfi_mem_addr <= i_dbus_addr;
387     336         rvfi_mem_rmask <= i_dbus_we ? 4'b0000 : i_dbus_sel;
388     337         rvfi_mem_wmask <= i_dbus_we ? i_dbus_sel : 4'b0000;
389     338         rvfi_mem_rdata <= i_dbus_rdt;
390     339         rvfi_mem_wdata <= i_dbus_dat;
391     340     end
392     341     if (i_ibus_ack) begin
393     342         rvfi_mem_rmask <= 4'b0000;
394     343         rvfi_mem_wmask <= 4'b0000;
395     344     end
396     345     end
397     346

```

```

347 assign rvfi_pc_wdata = i_ibus_adr;
348
349 `endif
350
351 endmodule

```

9. olof_serv_rtl/serv_decode.v (367 lines)

```

1 /*
2  * serv_decode.v : SERV module decoding instruction word into
3  * control signals
4  *
5  * SPDX-FileCopyrightText: 2018 Olof Kindgren <olof@award-win-
6  * ning.me>
7  *
8  * SPDX-License-Identifier: ISC
9  */
10 `default_nettype none
11 module serv_decode
12     #(parameter [0:0] PRE_REGISTER = 1,
13       parameter [0:0] MDU = 0)
14 (
15     input wire      clk,
16     //Input
17     input wire [31:2] i_wb_rdt,
18     input wire      i_wb_en,
19     //To state
20     output reg      o_sh_right,
21     output reg      o_bne_or_bge,
22     output reg      o_cond_branch,
23     output reg      o_e_op,
24     output reg      o_ebreak,
25     output reg      o_branch_op,
26     output reg      o_shift_op,
27     output reg      o_rd_op,
28     output reg      o_two_stage_op,
29     output reg      o_dbus_en,
30     //MDU
31     output reg      o_mdu_op,
32     //Extension
33     output reg [2:0] o_ext_func3,
34     //To bufreg
35     output reg      o_bufreg_rs1_en,
36     output reg      o_bufreg_imm_en,
37     output reg      o_bufreg_clr_lsb,
38     output reg      o_bufreg_sh_signed,
39     //To ctrl
40     output reg      o_ctrl_jal_or_jalr,
41     output reg      o_ctrl_ctype,
42     output reg      o_ctrl_pc_rel,
43     output reg      o_ctrl_mret,
44     //To alu
45     output reg      o_alu_sub,
46     output reg [1:0] o_alu_bool_op,
47     output reg      o_alu_cmp_eq,
48     output reg      o_alu_cmp_sig,
49     output reg [2:0] o_alu_rd_sel,
50     //To mem IF
51     output reg      o_mem_signed,
52     output reg      o_mem_word,
53     output reg      o_mem_half,
54     output reg      o_mem_cmd,
55     //To CSR
56     output reg      o_csr_en,
57     output reg [1:0] o_csr_addr,
58     output reg      o_csr_mstatus_en,
59     output reg      o_csr_mie_en,
60     output reg      o_csr_mcause_en,
61     output reg [1:0] o_csr_source,
62     output reg      o_csr_d_sel,
63     output reg      o_csr_imm_en,
64     output reg      o_mtval_pc,
65     //To top
66     output reg [3:0] o_immdec_ctrl,
67     output reg [3:0] o_immdec_en,
68     output reg      o_op_b_source,
69     //To RF IF
70     output reg      o_rd_mem_en,
71     output reg      o_rd_csr_en,
72     output reg      o_rd_alu_en;
73
74     reg [4:0] opcode;
75     reg [2:0] funct3;
76     reg      op20;
77     reg      op21;
78     reg      op22;
79     reg      op26;
80
81     reg      imm25;
82     reg      imm30;
83
84     wire co_mdu_op      = MDU & (opcode == 5'b01100) & imm25;
85
86     wire co_two_stage_op =
87         ~opcode[2] | (funct3[0] & ~funct3[1] & ~opcode[0] &
88         ~opcode[4]) |
89         (funct3[1] & ~funct3[2] & ~opcode[0] & ~opcode[4]) |
90         co_mdu_op;
91     wire co_shift_op = (opcode[2] & ~funct3[1]) & !co_mdu_op;
92     wire co_branch_op = opcode[4];
93     wire co_dbus_en   = ~opcode[2] & ~opcode[4];

```

```

89     wire co_mtval_pc  = opcode[4];
90     wire co_mem_word  = funct3[1];
91     wire co_rd_alu_en = !opcode[0] & opcode[2] & !opcode[4]
92     & !co_mdu_op;
93     wire co_rd_mem_en = (!opcode[2] & !opcode[0]) | co_mdu_op;
94     wire [2:0] co_ext_func3 = funct3;
95
96     //jal,branch = imm
97     //jalr      = rs1+imm
98     //mem       = rs1+imm
99     //shift     = rs1
100     wire co_bufreg_rs1_en = !opcode[4] | (!opcode[1] & op-
101     code[0]);
102     wire co_bufreg_imm_en = !opcode[2];
103
104     //Clear LSB of immediate for BRANCH and JAL ops
105     //True for BRANCH and JAL
106     //False for JALR/LOAD/STORE/OP/OPIMM?
107     wire co_bufreg_clr_lsb = opcode[4] & ((opcode[1:0] ==
108     2'b00) | (opcode[1:0] == 2'b11));
109
110     //Conditional branch
111     //True for BRANCH
112     //False for JAL/JALR
113     wire co_cond_branch = !opcode[0];
114
115     wire co_ctrl_ctype = !opcode[4] & opcode[2] & op-
116     code[0];
117     wire co_ctrl_jal_or_jalr = opcode[4] & opcode[0];
118
119     //PC-relative operations
120     //True for jal, b* auipc, ebreak
121     //False for jalr, lui
122     wire co_ctrl_pc_rel = (opcode[2:0] == 3'b000) |
123         (opcode[1:0] == 2'b11) |
124         (opcode[4] & opcode[2]) & op20 |
125         (opcode[4:3] == 2'b00);
126
127     //Write to RD
128     //True for OP-IMM, AUIPC, OP, LUI, SYSTEM, JALR, JAL, LOAD
129     //False for STORE, BRANCH, MISC-MEM
130     wire co_rd_op = (opcode[2] |
131         (!opcode[2] & opcode[4] & opcode[0]) |
132         (!opcode[2] & !opcode[3] & !opcode[0]));
133
134     //
135     //funct3
136     //
137     wire co_sh_right = funct3[2];
138     wire co_bne_or_bge = funct3[0];
139
140     //Matches system ops except ecall/ebreak/mret
141     wire csr_op = opcode[4] & opcode[2] & (!funct3);
142
143     //op20
144     wire co_ebreak = op20;
145
146     //opcode & funct3 & op21
147     wire co_ctrl_mret = opcode[4] & opcode[2] & op21 & !(func-
148     t3);
149     //Matches system opcodes except CSR accesses (funct3 == 0)
150     //and mret (!op21)
151     wire co_e_op = opcode[4] & opcode[2] & !op21 & !(funct3);
152
153     //opcode & funct3 & imm30
154     wire co_bufreg_sh_signed = imm30;
155
156     /*
157     True for sub, b*, slt*
158     False for add*
159     op      opcode f3 i30
160     b*      11000 xxx x t
161     addi    00100 000 x f
162     slt*    0x100 01x x t
163     add     01100 000 0 f
164     sub     01100 000 1 t
165     */
166     wire co_alu_sub = funct3[1] | funct3[0] | (opcode[3] & im-
167     m30) | opcode[4];
168
169     /*
170     Bits 26, 22, 21 and 20 are enough to uniquely identify the
171     eight supported CSR regs
172     mtime, mscratch, mepc and mtval are stored externally
173     (normally in the RF) and are
174     treated differently from mstatus, mie and mcause which are
175     stored in serv_csr.
176
177     The former get a 2-bit address as seen below while the
178     latter get a
179     one-hot enable signal each.
180
181     Hex|2 222|Reg      |csr
182     adr|6 210|name     |addr
183     ---|-----|-----|-----

```



```

178 300|0_000|mstatus | xx
179 304|0_100|mie | xx
180 305|0_101|mtvec | 01
181 340|1_000|mscratch| 00
182 341|1_001|mepc | 10
183 342|1_010|mcause | xx
184 343|1_011|mtval | 11
185
186 */
187
188 //true for mtvec,mscratch,mepc and mtval
189 //false for mstatus, mie, mcause
190 wire csr_valid = op20 | (op26 & !op21);
191
192 wire co_rd_csr_en = csr_op;
193
194 wire co_csr_en = csr_op & csr_valid;
195 wire co_csr_mstatus_en = csr_op & !op26 & !op22 & !op20;
196 wire co_csr_mie_en = csr_op & !op26 & op22 & !op20;
197 wire co_csr_mcause_en = csr_op & op21 & !op20;
198
199 wire [1:0] co_csr_source = funct3[1:0];
200 wire co_csr_d_sel = funct3[2];
201 wire co_csr_imm_en = opcode[4] & opcode[2] & funct3[2];
202 wire [1:0] co_csr_addr = {op26 & op20, !op26 | op21};
203
204 wire co_alu_cmp_eq = funct3[2:1] == 2'b00;
205
206 wire co_alu_cmp_sig = ~(funct3[0] & funct3[1]) | (funct3[1] & funct3[2]);
207
208 wire co_mem_cmd = opcode[3];
209 wire co_mem_signed = ~funct3[2];
210 wire co_mem_half = funct3[0];
211
212 wire [1:0] co_alu_bool_op = funct3[1:0];
213
214 wire [3:0] co_immdec_ctrl;
215 //True for S (STORE) or B (BRANCH) type instructions
216 //False for J type instructions
217 assign co_immdec_ctrl[0] = opcode[3:0] == 4'b1000;
218 //True for OP-IMM, LOAD, STORE, JALR (I S)
219 //False for LUI, AUIPC, JAL (U J)
220 assign co_immdec_ctrl[1] = (opcode[1:0] == 2'b00) | (opcode[2:1] == 2'b00);
221 assign co_immdec_ctrl[2] = opcode[4] & !opcode[0];
222 assign co_immdec_ctrl[3] = opcode[4];
223
224 wire [3:0] co_immdec_en;
225 assign co_immdec_en[3] = opcode[4] | opcode[3] | opcode[2] | !opcode[0];
226 //B I J S U
227 assign co_immdec_en[2] = (opcode[4] & opcode[2]) | !opcode[3] | opcode[0];
228 //I J U
229 assign co_immdec_en[1] = (opcode[2:1] == 2'b01) | (opcode[2] & opcode[0]) | co_csr_imm_en;
230 //J U
231 assign co_immdec_en[0] = ~co_rd_op;
232
233 B S
234
235 wire [2:0] co_alu_rd_sel;
236 assign co_alu_rd_sel[0] = (funct3 == 3'b000); // Add/sub
237 assign co_alu_rd_sel[1] = (funct3[2:1] == 2'b01); //SLT*
238 assign co_alu_rd_sel[2] = funct3[2]; //Bool
239
240 //0 (OP_B_SOURCE_IMM) when OPIMM
241 //1 (OP_B_SOURCE_RS2) when BRANCH or OP
242 wire co_op_b_source = opcode[3];
243
244 generate
245 if (PRE_REGISTER) begin : gen_pre_register
246
247 always @(posedge clk) begin
248 if (i_wb_en) begin
249 funct3 <= i_wb_rdt[14:12];
250 imm30 <= i_wb_rdt[30];
251 imm25 <= i_wb_rdt[25];
252 opcode <= i_wb_rdt[6:2];
253 op20 <= i_wb_rdt[20];
254 op21 <= i_wb_rdt[21];
255 op22 <= i_wb_rdt[22];
256 op26 <= i_wb_rdt[26];
257 end
258 end
259
260 always @(*) begin
261 o_sh_right = co_sh_right;
262 o_bne_or_bge = co_bne_or_bge;
263 o_cond_branch = co_cond_branch;
264 o_dbus_en = co_dbus_en;
265 o_mtval_pc = co_mtval_pc;
266 o_two_stage_op = co_two_stage_op;
267 o_e_op = co_e_op;
268 o_ebreak = co_ebreak;
269 o_branch_op = co_branch_op;
270 o_shift_op = co_shift_op;
271 o_rd_op = co_rd_op;
272 o_mdu_op = co_mdu_op;
273 o_ext_funct3 = co_ext_funct3;
274 o_bufreg_rs1_en = co_bufreg_rs1_en;
275 o_bufreg_imm_en = co_bufreg_imm_en;
276 o_bufreg_clr_lsb = co_bufreg_clr_lsb;

```

```

277 o_bufreg_sh_signed = co_bufreg_sh_signed;
278 o_ctrl_jal_or_jalr = co_ctrl_jal_or_jalr;
279 o_ctrl_utype = co_ctrl_utype;
280 o_ctrl_pc_rel = co_ctrl_pc_rel;
281 o_ctrl_mret = co_ctrl_mret;
282 o_alu_sub = co_alu_sub;
283 o_alu_bool_op = co_alu_bool_op;
284 o_alu_cmp_eq = co_alu_cmp_eq;
285 o_alu_cmp_sig = co_alu_cmp_sig;
286 o_alu_rd_sel = co_alu_rd_sel;
287 o_mem_signed = co_mem_signed;
288 o_mem_word = co_mem_word;
289 o_mem_half = co_mem_half;
290 o_mem_cmd = co_mem_cmd;
291 o_csr_en = co_csr_en;
292 o_csr_addr = co_csr_addr;
293 o_csr_mstatus_en = co_csr_mstatus_en;
294 o_csr_mie_en = co_csr_mie_en;
295 o_csr_mcause_en = co_csr_mcause_en;
296 o_csr_source = co_csr_source;
297 o_csr_d_sel = co_csr_d_sel;
298 o_csr_imm_en = co_csr_imm_en;
299 o_immdec_ctrl = co_immdec_ctrl;
300 o_immdec_en = co_immdec_en;
301 o_op_b_source = co_op_b_source;
302 o_rd_csr_en = co_rd_csr_en;
303 o_rd_alu_en = co_rd_alu_en;
304 o_rd_mem_en = co_rd_mem_en;
305 end
306
307 end else begin : gen_post_register
308
309 always @(*) begin
310 funct3 = i_wb_rdt[14:12];
311 imm30 = i_wb_rdt[30];
312 imm25 = i_wb_rdt[25];
313 opcode = i_wb_rdt[6:2];
314 op20 = i_wb_rdt[20];
315 op21 = i_wb_rdt[21];
316 op22 = i_wb_rdt[22];
317 op26 = i_wb_rdt[26];
318 end
319
320 always @(posedge clk) begin
321 if (i_wb_en) begin
322 o_sh_right <= co_sh_right;
323 o_bne_or_bge <= co_bne_or_bge;
324 o_cond_branch <= co_cond_branch;
325 o_e_op <= co_e_op;
326 o_ebreak <= co_ebreak;
327 o_two_stage_op <= co_two_stage_op;
328 o_dbus_en <= co_dbus_en;
329 o_mtval_pc <= co_mtval_pc;
330 o_branch_op <= co_branch_op;
331 o_shift_op <= co_shift_op;
332 o_rd_op <= co_rd_op;
333 o_mdu_op <= co_mdu_op;
334 o_ext_funct3 <= co_ext_funct3;
335 o_bufreg_rs1_en <= co_bufreg_rs1_en;
336 o_bufreg_imm_en <= co_bufreg_imm_en;
337 o_bufreg_clr_lsb <= co_bufreg_clr_lsb;
338 o_bufreg_sh_signed <= co_bufreg_sh_signed;
339 o_ctrl_jal_or_jalr <= co_ctrl_jal_or_jalr;
340 o_ctrl_utype <= co_ctrl_utype;
341 o_ctrl_pc_rel <= co_ctrl_pc_rel;
342 o_ctrl_mret <= co_ctrl_mret;
343 o_alu_sub <= co_alu_sub;
344 o_alu_bool_op <= co_alu_bool_op;
345 o_alu_cmp_eq <= co_alu_cmp_eq;
346 o_alu_cmp_sig <= co_alu_cmp_sig;
347 o_alu_rd_sel <= co_alu_rd_sel;
348 o_mem_signed <= co_mem_signed;
349 o_mem_word <= co_mem_word;
350 o_mem_half <= co_mem_half;
351 o_mem_cmd <= co_mem_cmd;
352 o_csr_en <= co_csr_en;
353 o_csr_addr <= co_csr_addr;
354 o_csr_mstatus_en <= co_csr_mstatus_en;
355 o_csr_mie_en <= co_csr_mie_en;
356 o_csr_mcause_en <= co_csr_mcause_en;
357 o_csr_source <= co_csr_source;
358 o_csr_d_sel <= co_csr_d_sel;
359 o_csr_imm_en <= co_csr_imm_en;
360 o_immdec_ctrl <= co_immdec_ctrl;
361 o_immdec_en <= co_immdec_en;
362 o_op_b_source <= co_op_b_source;
363 o_rd_csr_en <= co_rd_csr_en;
364 o_rd_alu_en <= co_rd_alu_en;
365 o_rd_mem_en <= co_rd_mem_en;
366 end
367 end
368
369 endgenerate
370 endmodule

```

10. olofk_serv/rtl/serv_immdec.v (234 lines)

```

1 /*
2  * serv_immdec.v : SERV module for decoding immediates from
3  * instruction words
4  * SPDX-FileCopyrightText: 2020 Olof Kindgren <olof@award-win-
5  * ning.me>
6  * SPDX-License-Identifier: ISC
7  */
8 `default_nettype none
9 module serv_immdec
10     #(parameter SHARED_RFADDR_IMM_REGS = 1,
11       parameter W = 1)
12     (
13         input wire      i_clk,
14         //State
15         input wire      i_cnt_en,
16         input wire      i_cnt_done,
17         //Control
18         input wire [3:0] i_immdec_en,
19         input wire      i_csr_imm_en,
20         input wire [3:0] i_ctrl,
21         output wire [4:0] o_rd_addr,
22         output wire [4:0] o_rs1_addr,
23         output wire [4:0] o_rs2_addr,
24         //Data
25         output wire [W-1:0] o_csr_imm,
26         output wire [W-1:0] o_imm,
27         //External
28         input wire      i_wb_en,
29         input wire [31:7] i_wb_rdt;
30     )
31     generate
32         if (W == 1) begin : gen_immdec_w_eq_1
33             reg
34                 imm31;
35             reg [8:0] imm19_12_20;
36             reg [5:0] imm30_25;
37             reg [4:0] imm24_20;
38             reg [4:0] imm11_7;
39             assign o_csr_imm = imm19_12_20[4];
40             wire
41                 signbit = imm31 & !i_csr_imm_en;
42             if (SHARED_RFADDR_IMM_REGS) begin : gen_shared_imm_regs
43                 assign o_rs1_addr = imm19_12_20[8:4];
44                 assign o_rs2_addr = imm24_20;
45                 assign o_rd_addr = imm11_7;
46             end
47             always @(posedge i_clk) begin
48                 if (i_wb_en) begin
49                     /* CSR immediates are always zero-extended,
50                     hence clear the signbit */
51                     imm31 <= i_wb_rdt[31];
52                     end
53                     if (i_wb_en | (i_cnt_en & i_immdec_en[1]))
54                         imm19_12_20 <= i_wb_en ?
55                         {i_wb_rdt[19:12], i_wb_rdt[20]} : {i_ctrl[3] ? signbit : imm24_20[0],
56                         imm19_12_20[8:1]};
57                     if (i_wb_en | (i_cnt_en))
58                         imm7 <= i_wb_en ?
59                         : signbit;
60                     if (i_wb_en | (i_cnt_en & i_immdec_en[3]))
61                         imm30_25 <= i_wb_en ? i_wb_rdt[30:25] : {i_c-
62                         trl[2] ? imm7 : i_ctrl[1] ? signbit : imm19_12_20[0], im-
63                         m30_25[5:1]};
64                     if (i_wb_en | (i_cnt_en & i_immdec_en[2]))
65                         imm24_20 <= i_wb_en ? i_wb_rdt[24:20] : {im-
66                         m30_25[0], imm24_20[4:1]};
67                     if (i_wb_en | (i_cnt_en & i_immdec_en[0]))
68                         imm11_7 <= i_wb_en ? i_wb_rdt[11:7] : {im-
69                         m30_25[0], imm11_7[4:1]};
70                     end
71                     end else begin : gen_separate_imm_regs
72                         reg [4:0] rd_addr;
73                         reg [4:0] rs1_addr;
74                         reg [4:0] rs2_addr;
75                         assign o_rd_addr = rd_addr;
76                         assign o_rs1_addr = rs1_addr;
77                         assign o_rs2_addr = rs2_addr;
78                         always @(posedge i_clk) begin
79                             if (i_wb_en) begin
80                                 /* CSR immediates are always zero-extended,
81                                 hence clear the signbit */
82                                 imm31 <= i_wb_rdt[31];
83                                 imm19_12_20 <= {i_wb_rdt[19:12], i_wb_rdt[20]};
84                                 imm7 <= i_wb_rdt[7];
85                                 imm30_25 <= i_wb_rdt[30:25];
86                                 imm24_20 <= i_wb_rdt[24:20];
87                                 imm11_7 <= i_wb_rdt[11:7];
88                                 rd_addr <= i_wb_rdt[11:7];
89                                 rs1_addr <= i_wb_rdt[19:15];
90                                 rs2_addr <= i_wb_rdt[24:20];
91                             end
92                             if (i_cnt_en) begin
93                                 imm19_12_20 <= {i_ctrl[3] ? signbit : im-
94                                 m24_20[0], imm19_12_20[8:1]};
95                                 imm7 <= signbit;
96                                 imm30_25 <= {i_ctrl[2] ? imm7 : i_ctrl[1] ?
97                                 signbit : imm19_12_20[0], imm30_25[5:1]};
98                                 imm24_20 <= {imm30_25[0], imm24_20[4:1]};
99                                 imm11_7 <= {imm30_25[0], imm11_7[4:1]};
100                             end
101                             end
102                             assign o_imm = i_cnt_done ? signbit : i_ctrl[0] ? im-
103                             m11_7[0] : imm24_20[0];
104                             end else begin : gen_immdec_w_eq_4
105                                 reg [4:0] rd_addr;
106                                 reg [4:0] rs1_addr;
107                                 reg [4:0] rs2_addr;
108                                 reg
109                                     i31;
110                                 reg
111                                     i30;
112                                 reg
113                                     i29;
114                                 reg
115                                     i28;
116                                 reg
117                                     i27;
118                                 reg
119                                     i26;
120                                 reg
121                                     i25;
122                                 reg
123                                     i24;
124                                 reg
125                                     i23;
126                                 reg
127                                     i22;
128                                 reg
129                                     i21;
130                                 reg
131                                     i20;
132                                 reg
133                                     i19;
134                                 reg
135                                     i18;
136                                 reg
137                                     i17;
138                                 reg
139                                     i16;
140                                 reg
141                                     i15;
142                                 reg
143                                     i14;
144                                 reg
145                                     i13;
146                                 reg
147                                     i12;
148                                 reg
149                                     i11;
150                                 reg
151                                     i10;
152                                 reg
153                                     i9;
154                                 reg
155                                     i8;
156                                 reg
157                                     i7;
158                                 reg
159                                     i7_2;
160                                 reg
161                                     i20_2;
162                                 wire
163                                     signbit = i31 & !i_csr_imm_en;
164                                 assign o_csr_imm[3] = i18;
165                                 assign o_csr_imm[2] = i17;
166                                 assign o_csr_imm[1] = i16;
167                                 assign o_csr_imm[0] = i15;
168                                 assign o_rd_addr = rd_addr;
169                                 assign o_rs1_addr = rs1_addr;
170                                 assign o_rs2_addr = rs2_addr;
171                                 always @(posedge i_clk) begin
172                                     if (i_wb_en) begin
173                                         //Common
174                                         i31 <= i_wb_rdt[31];
175                                         //Bit lane 3
176                                         i19 <= i_wb_rdt[19];
177                                         i15 <= i_wb_rdt[15];
178                                         i20 <= i_wb_rdt[20];
179                                         i7 <= i_wb_rdt[7];
180                                         i27 <= i_wb_rdt[27];
181                                         i23 <= i_wb_rdt[23];
182                                         i10 <= i_wb_rdt[10];
183                                         //Bit lane 2
184                                         i22 <= i_wb_rdt[22];
185                                         i9 <= i_wb_rdt[9];
186                                         i26 <= i_wb_rdt[26];
187                                         i30 <= i_wb_rdt[30];
188                                         i14 <= i_wb_rdt[14];
189                                         i18 <= i_wb_rdt[18];
190                                         //Bit lane 1
191                                         i21 <= i_wb_rdt[21];
192                                         i8 <= i_wb_rdt[8];
193                                         i25 <= i_wb_rdt[25];
194                                         i29 <= i_wb_rdt[29];
195                                         i13 <= i_wb_rdt[13];
196                                         i17 <= i_wb_rdt[17];
197                                         //Bit lane 0
198                                         i11 <= i_wb_rdt[11];
199                                         i7_2 <= i_wb_rdt[7];
200                                         i20_2 <= i_wb_rdt[20];
201                                         i24 <= i_wb_rdt[24];
202                                         i28 <= i_wb_rdt[28];
203                                         i12 <= i_wb_rdt[12];
204                                         i16 <= i_wb_rdt[16];
205                                         rd_addr <= i_wb_rdt[11:7];

```



```

185     rs1_addr <= i_wb_rdt[19:15];
186     rs2_addr <= i_wb_rdt[24:20];
187   end
188   if (i_cnt_en) begin
189     //Bit lane 3
190     i10 <= i27;
191     i23 <= i27;
192     i27 <= i_ctrl[2] ? i7 : i_ctrl[1] ? signbit : i20;
193     i7 <= signbit;
194     i20 <= i15;
195     i15 <= i19;
196     i19 <= i_ctrl[3] ? signbit : i23;
197
198     //Bit lane 2
199     i22 <= i26;
200     i9 <= i26;
201     i26 <= i30;
202     i30 <= (i_ctrl[1] | i_ctrl[2]) ? signbit : i14;
203     i14 <= i18;
204     i18 <= i_ctrl[3] ? signbit : i22;
205
206     //Bit lane 1
207     i21 <= i25;
208     i8 <= i25;
209     i25 <= i29;
210     i29 <= (i_ctrl[1] | i_ctrl[2]) ? signbit : i13;
211     i13 <= i17;
212     i17 <= i_ctrl[3] ? signbit : i21;
213
214     //Bit lane 0
215     i7_2 <= i11;
216     i11 <= i28;
217     i20_2 <= i24;
218     i24 <= i28;
219     i28 <= (i_ctrl[1] | i_ctrl[2]) ? signbit : i12;
220     i12 <= i16;
221     i16 <= i_ctrl[3] ? signbit : i20_2;
222
223   end
224 end
225
226 assign o_imm[3] = (i_cnt_done ? signbit : (i_ctrl[0] ?
i10 : i23));
227 assign o_imm[2] = i_ctrl[0] ? i9 : i22;
228 assign o_imm[1] = i_ctrl[0] ? i8 : i21;
229 assign o_imm[0] = i_ctrl[0] ? i7_2 : i20_2;
230
231 end
232 endgenerate
233
234 endmodule

```

11. olofk_serv/rtl/serv_mem_if.v (59 lines)

```

1 /*
2  * serv_mem_if.v : SERV memory interface
3  *
4  * SPDX-FileCopyrightText: 2018 Olof Kindgren <olof@award-win-
ning.me>
5  * SPDX-License-Identifier: ISC
6  */
7 `default_nettype none
8 module serv_mem_if
9 # (
10     parameter [0:0] WITH_CSR = 1,
11     parameter W = 1,
12     parameter B = W-1
13 )
14 (
15     input wire i_clk,
16     //State
17     input wire [1:0] i_bytecnt,
18     input wire [1:0] i_lsb,
19     output wire o_misalign,
20     //Control
21     input wire i_signed,
22     input wire i_word,
23     input wire i_half,
24     //MDU
25     input wire i_mdu_op,
26     //Data
27     input wire [B:0] i_bufreg2_q,
28     output wire [B:0] o_rd,
29     //External interface
30     output wire [3:0] o_wb_sel;
31
32     reg signbit;
33
34     wire dat_valid =
35         i_mdu_op |
36         i_word |
37         (i_bytecnt == 2'b00) |
38         (i_half & i_bytecnt[1]);
39
40     assign o_rd = dat_valid ? i_bufreg2_q : {W{i_signed & sign-
bit}};
41
42     assign o_wb_sel[3] = (i_lsb == 2'b11) | i_word | (i_half &
i_lsb[1]);
43     assign o_wb_sel[2] = (i_lsb == 2'b10) | i_word;

```

```

44     assign o_wb_sel[1] = (i_lsb == 2'b01) | i_word | (i_half
& i_lsb[1]);
45     assign o_wb_sel[0] = (i_lsb == 2'b00);
46
47     always @(posedge i_clk) begin
48         if (dat_valid)
49             signbit <= i_bufreg2_q[B];
50     end
51
52     /*
53     mem_misalign is checked after the init stage to decide
whether to do a data
54     bus transaction or go to the trap state. It is only guar-
anteed to be correct
55     at this time
56     */
57     assign o_misalign = WITH_CSR & ((i_lsb[0] & (i_word |
i_half)) | (i_lsb[1] & i_word));
58
59 endmodule

```

12. olofk_serv/rtl/serv_rf_if.v (159 lines)

```

1 /*
2  * serv_rf_if.v : SERV register file interface
3  *
4  * SPDX-FileCopyrightText: 2019 Olof Kindgren <olof@award-win-
ning.me>
5  * SPDX-License-Identifier: ISC
6  */
7 `default_nettype none
8 module serv_rf_if
9 # (parameter WITH_CSR = 1,
10     parameter W = 1,
11     parameter B = W-1
12 )
13 (//RF Interface
14     input wire i_cnt_en,
15     output wire [4+WITH_CSR:0] o_wreg0,
16     output wire [4+WITH_CSR:0] o_wreg1,
17     output wire o_wen0,
18     output wire o_wen1,
19     output wire [B:0] o_wdata0,
20     output wire [B:0] o_wdata1,
21     output wire [4+WITH_CSR:0] o_rreg0,
22     output wire [4+WITH_CSR:0] o_rreg1,
23     input wire [B:0] i_rdata0,
24     input wire [B:0] i_rdata1,
25
26     //Trap interface
27     input wire i_trap,
28     input wire i_mret,
29     input wire [B:0] i_mepc,
30     input wire i_mtval_pc,
31     input wire [B:0] i_bufreg_q,
32     input wire [B:0] i_bad_pc,
33     output wire [B:0] o_csr_pc,
34     //CSR interface
35     input wire i_csr_en,
36     input wire [1:0] i_csr_addr,
37     input wire [B:0] i_csr,
38     output wire [B:0] o_csr,
39     //RD write port
40     input wire i_rd_wen,
41     input wire [4:0] i_rd_waddr,
42     input wire [B:0] i_ctrl_rd,
43     input wire [B:0] i_alu_rd,
44     input wire i_rd_alu_en,
45     input wire [B:0] i_csr_rd,
46     input wire i_rd_csr_en,
47     input wire [B:0] i_mem_rd,
48     input wire i_rd_mem_en,
49
50     //RS1 read port
51     input wire [4:0] i_rs1_raddr,
52     output wire [B:0] o_rs1,
53     //RS2 read port
54     input wire [4:0] i_rs2_raddr,
55     output wire [B:0] o_rs2;
56
57     /*
58     ***** Write side *****
59     */
60
61     wire rd_wen = i_rd_wen & (!i_rd_waddr);
62
63     generate
64     if (!WITH_CSR) begin : gen_csr
65     wire [B:0] rd =
66         {W{i_rd_alu_en}} & i_alu_rd |
67         {W{i_rd_csr_en}} & i_csr_rd |
68         {W{i_rd_mem_en}} & i_mem_rd |
69         i_ctrl_rd;
70
71     wire [B:0] mtval = i_mtval_pc ? i_bad_pc : i_bufreg_q;
72
73     assign o_wdata0 = i_trap ? mtval : rd;
74     assign o_wdata1 = i_trap ? i_mepc : i_csr;
75
76

```

```

77 /* Port 0 handles writes to mtval during traps and rd oth-
erwise
78 * Port 1 handles writes to mepc during traps and csr ac-
cesses otherwise
79 *
80 * GPR registers are mapped to address 0-31 (bits 0xxxxx).
81 * Following that are four CSR registers
82 * mscratch 100000
83 * mtvec 100001
84 * mepc 100010
85 * mtval 100011
86 */
87
88 assign o_wreg0 = i_trap ? {6'b100011} : {1'b0,i_rd_waddr};
89 assign o_wreg1 = i_trap ? {6'b100010} :
{4'b1000,i_csr_addr};
90
91 assign o_wen0 = i_cnt_en & (i_trap | rd_wen);
92 assign o_wen1 = i_cnt_en & (i_trap | i_csr_en);
93
94 /*
95 ***** Read side *****
96 */
97
98 //0 : RS1
99 //1 : RS2 / CSR
100
101 assign o_rreg0 = {1'b0, i_rs1_raddr};
102
103 /*
104 The address of the second read port (o_rreg1) can get as-
signed from four
105 different sources
106
107 Normal operations : i_rs2_raddr
108 CSR access : i_csr_addr
109 trap : MTVEC
110 mret : MEPC
111
112 Address 0-31 in the RF are assigned to the GPRs. After
that follows the four
113 CSRs on addresses 32-35
114
115 32 MSCRATCH
116 33 MTVEC
117 34 MEPC
118 35 MTVAL
119
120 The expression below is an optimized version of this logic
121 */
122 wire sel_rs2 = !(i_trap | i_mret | i_csr_en);
123 assign o_rreg1 = {~sel_rs2,
i_rs2_raddr[4:2] & {3{sel_rs2}},
{1'b0,i_trap} | {i_mret,1'b0} |
{2{i_csr_en}} & i_csr_addr} | ({2{sel_rs2}} & i_rs2_raddr[1:0]);
124
125 assign o_rs1 = i_rdata0;
126 assign o_rs2 = i_rdata1;
127 assign o_csr = i_rdata1 & {W{i_csr_en}};
128 assign o_csr_pc = i_rdata1;
129
130 end else begin : gen_no_csr
131 wire [B:0] rd = (i_ctrl_rd |
i_alu_rd & {W{i_rd_alu_en}} |
i_mem_rd & {W{i_rd_mem_en}});
132
133 assign o_wdata0 = rd;
134 assign o_wdata1 = {W{1'b0}};
135
136 assign o_wreg0 = i_rd_waddr;
137 assign o_wreg1 = 5'd0;
138
139 assign o_wen0 = i_cnt_en & rd_wen;
140 assign o_wen1 = 1'b0;
141
142 /*
143 ***** Read side *****
144 */
145
146 assign o_rreg0 = i_rs1_raddr;
147 assign o_rreg1 = i_rs2_raddr;
148
149 assign o_rs1 = i_rdata0;
150 assign o_rs2 = i_rdata1;
151 assign o_csr = {W{1'b0}};
152 assign o_csr_pc = {W{1'b0}};
153 end // else: !if(WITH_CSR)
154 endgenerate
155 endmodule

```

13. olof_serv/rtl/serv_rf_ram.v (51 lines)

```

1 /*
2 * serv_rf_ram.v : SRAM-based RF storage for SERV
3 *
4 * SPDX-FileCopyrightText: 2019 Olof Kindgren <olof@award-win-
ning.me>
5 * SPDX-License-Identifier: ISC
6 */
7 module serv_rf_ram

```

```

8 #(parameter width=0,
9 parameter csr_regs=4,
10 parameter depth=32*(32+csr_regs)/width)
11 (input wire i_clk,
12 input wire [$clog2(depth)-1:0] i_waddr,
13 input wire [width-1:0] i_wdata,
14 input wire i_wen,
15 input wire [$clog2(depth)-1:0] i_raddr,
16 input wire i_ren,
17 output wire [width-1:0] o_rdata);
18
19 reg [width-1:0] memory [0:depth-1];
20 reg [width-1:0] rdata ;
21
22 always @(posedge i_clk) begin
23 if (i_wen)
24 memory[i_waddr] <= i_wdata;
25 rdata <= i_ren ? memory[i_raddr] : {width{1'bx}};
26 end
27
28 /* Reads from reg x0 needs to return 0
29 Check that the part of the read address corresponding to
the register
30 is zero and gate the output
31 width LSB of reg index $clog2(width)
32 2 4 1
33 4 3 2
34 8 2 3
35 16 1 4
36 32 0 5
37 */
38 reg regzero;
39
40 always @(posedge i_clk)
41 regzero <= !(|i_raddr[$clog2(depth)-1:5-$clog2(width)]);
42
43 assign o_rdata = rdata & ~{width{regzero}};
44
45 `ifdef SERV_CLEAR_RAM
46 integer i;
47 initial
48 for (i=0;i<depth;i=i+1)
49 memory[i] = {width{1'd0}};
50 `endif
51 endmodule

```

14. olof_serv/rtl/serv_rf_ram_if.v (186 lines)

```

1 /*
2 * serv_rf_ram_if.v : Interface between SERV and SRAM-based RF
storage
3 *
4 * SPDX-FileCopyrightText: 2019 Olof Kindgren <olof@award-win-
ning.me>
5 * SPDX-License-Identifier: ISC
6 */
7 `default_nettype none
8 module serv_rf_ram_if
9 #(/*Data width. Adjust to preferred width of SRAM data in-
terface
10 parameter width=8,
11
12 parameter W = 1,
13 //Select reset strategy.
14 // "MINI" for resetting minimally required FFs
15 // "NONE" for relying on FFs having a defined value on
startup
16 parameter reset_strategy="MINI",
17
18 //Number of CSR registers. These are allocated after the
normal
19 // GPR registers in the RAM.
20 parameter csr_regs=4,
21
22 //Internal parameters calculated from above values. Do not
change
23 parameter B=W-1,
24 parameter raw=$clog2(32+csr_regs), //Register address
width
25 parameter l2w=$clog2(width), //log2 of width
26 parameter aw=5+raw-l2w //Address width
27 )
28 //SERV side
29 input wire i_clk,
30 input wire i_rst,
31 input wire i_wreq,
32 input wire i_rreq,
33 output wire o_ready,
34 input wire [raw-1:0] i_wreg0,
35 input wire [raw-1:0] i_wreg1,
36 input wire i_wen0,
37 input wire i_wen1,
38 input wire [B:0] i_wdata0,
39 input wire [B:0] i_wdata1,
40 input wire [raw-1:0] i_rreg0,
41 input wire [raw-1:0] i_rreg1,
42 output wire [B:0] o_rdata0,
43 output wire [B:0] o_rdata1,

```

```

44 //RAM side
45 output wire [aw-1:0] o_waddr,
46 output wire [width-1:0] o_wdata,
47 output wire o_wen,
48 output wire [aw-1:0] o_raddr,
49 output wire o_ren,
50 input wire [width-1:0] i_rdata;
51
52 localparam ratio = width/W;
53 localparam CMSB = 4-$clog2(W); //Counter MSB
54 localparam l2r = $clog2(ratio);
55
56 reg rgnt;
57 assign o_ready = rgnt | i_wreq;
58 reg [CMSB:0] rcnt;
59
60 reg rtrig1;
61 /*
62 ***** Write side *****
63 */
64
65 wire [CMSB:0] wcnt;
66
67 reg [width-1:0] wdata0_r;
68 reg [width+W-1:0] wdata1_r;
69
70 reg wen0_r;
71 reg wen1_r;
72 wire wtrig0;
73 wire wtrig1;
74
75 assign wtrig0 = rtrig1;
76
77 generate if (ratio == 2) begin : gen_wtrig_ratio_eq_2
78   assign wtrig1 = wcnt[0];
79 end else begin : gen_wtrig_ratio_neq_2
80   reg wtrig0_r;
81   always @(posedge i_clk) wtrig0_r <= wtrig0;
82   assign wtrig1 = wtrig0_r;
83 end
84 endgenerate
85
86 assign o_wdata = wtrig1 ?
87   wdata1_r[width-1:0] :
88   wdata0_r;
89
90 wire [raw:1:0] wreg = wtrig1 ? i_wreg1 : i_wreg0;
91 generate if (width == 32) begin : gen_w_eq_32
92   assign o_waddr = wreg;
93 end else begin : gen_w_neq_32
94   assign o_waddr = {wreg, wcnt[CMSB:l2r]};
95 end
96 endgenerate
97
98 assign o_wen = (wtrig0 & wen0_r) | (wtrig1 & wen1_r);
99
100 assign wcnt = rcnt-4;
101
102 always @(posedge i_clk) begin
103   if (wcnt[0]) begin
104     wen0_r <= i_wen0;
105     wen1_r <= i_wen1;
106   end
107
108   wdata0_r <= {i_wdata0, wdata0_r[width-1:W]};
109   wdata1_r <= {i_wdata1, wdata1_r[width+W-1:W]};
110 end
111
112 /*
113 ***** Read side *****
114 */
115
116
117 wire rtrig0;
118
119 wire [raw:1:0] rreg = rtrig0 ? i_rreg1 : i_rreg0;
120 generate if (width == 32) begin : gen_rreg_eq_32
121   assign o_raddr = rreg;
122 end else begin : gen_rreg_neq_32
123   assign o_raddr = {rreg, rcnt[CMSB:l2r]};
124 end
125 endgenerate
126
127
128 reg [width-1:0] rdata0;
129 reg [width-1+W:0] rdata1;
130
131 reg rgate;
132
133 assign o_rdata0 = rdata0[B:0];
134 assign o_rdata1 = rtrig1 ? i_rdata[B:0] : rdata1[B:0];
135
136 assign rtrig0 = (rcnt[l2r-1:0] == 1);
137
138 generate if (ratio == 2) begin : gen_ren_w_eq_2
139   assign o_ren = rgate;
140 end else begin : gen_ren_w_neq_2
141   assign o_ren = rgate & (rcnt[l2r-1:1] == 0);
142 end
143 endgenerate
144

```

```

145 reg rreq_r;
146
147 generate if (ratio > 2) begin : gen_rdata1_w_neq_2
148   always @(posedge i_clk) begin
149     rdata1 <= {{W{1'b0}}, rdata1[width-W-1:W]};
150     if (rtrig1)
151       rdata1[width-W-1:0] <= i_rdata[width-1:W];
152   end
153 end else begin : gen_rdata1_w_eq_2
154   always @(posedge i_clk) if (rtrig1) rdata1 <=
i_rdata[W*2-1:W];
155 end
156 endgenerate
157
158 always @(posedge i_clk) begin
159   if (&rcnt | i_rreq)
160     rgate <= i_rreq;
161
162   rtrig1 <= rtrig0;
163   rcnt <= rcnt+{{CMSB{1'b0}}, 1'b1};
164   if (i_rreq | i_wreq)
165     rcnt <= {{CMSB-1{1'b0}}, i_wreq, 1'b0};
166
167   rreq_r <= i_rreq;
168   rgnt <= rreq_r;
169
170   rdata0 <= {{W{1'b0}}, rdata0[width-1:W]};
171   if (rtrig0)
172     rdata0 <= i_rdata;
173
174   if (i_rst) begin
175     if (reset_strategy != "NONE") begin
176       rgate <= 1'b0;
177       rgnt <= 1'b0;
178       rreq_r <= 1'b0;
179       rcnt <= {{CMSB+1{1'b0}}};
180     end
181   end
182 end
183
184
185
186 endmodule

```

15. olofk_serv/rtl/serv_rf_top.v (227 lines)

```

1 /*
2  * serv_rf_top.v : Toplevel including SERV and SRAM-based RF
storage
3  *
4  * SPDX-FileCopyrightText: 2019 Olof Kindgren <olof@award-win-
ning.me>
5  * SPDX-License-Identifier: ISC
6  */
7 `default_nettype none
8
9 module serv_rf_top
10   #(parameter RESET_PC = 32'd0,
11     /* COMPRESSED=1: Enable the compressed decoder and al-
lowed misaligned jump of pc
12     COMPRESSED=0: Disable the compressed decoder and does
not allow the misaligned jump of pc
13     */
14     parameter [0:0] COMPRESSED = 0,
15     /*
16     ALIGN = 1: Fetch the aligned instruction by making two
bus transactions if the misaligned address
17     is given to the instruction bus.
18     */
19     parameter [0:0] ALIGN = COMPRESSED,
20     /* Multiplication and Division Unit
21     This parameter enables the interface for connecting
SERV and MDU
22     */
23     parameter [0:0] MDU = 0,
24     /* Register signals before or after the decoder
25     0 : Register after the decoder. Faster but uses more
resources
26     1 : (default) Register before the decoder. Slower but
uses less resources
27     */
28     parameter PRE_REGISTER = 1,
29     /* Amount of reset applied to design
30     "NONE" : No reset at all. Relies on a POR to set cor-
rect initialization
31     values and that core isn't reset during
runtime
32     "MINI" : Standard setting. Resets the minimal amount of
FFs needed to
33     restart execution from the instruction at RE-
SET_PC
34     */
35     parameter RESET_STRATEGY = "MINI",
36     parameter [0:0] DEBUG = 1'b0,
37     parameter WITH_CSR = 1,
38     parameter W = 1,
39     parameter RF_WIDTH = W * 2,
40     parameter RF_L2D = $clog2((32+(WITH_CSR*4))*32/
RF_WIDTH))

```

```

41 (
42   input wire      clk,
43   input wire      i_rst,
44   input wire      i_timer_irq,
45   `ifdef RISC_V_FORMAL
46     output wire    rvfi_valid,
47     output wire [63:0] rvfi_order,
48     output wire [31:0] rvfi_insn,
49     output wire    rvfi_trap,
50     output wire    rvfi_halt,
51     output wire    rvfi_intr,
52     output wire [1:0] rvfi_mode,
53     output wire [1:0] rvfi_ixl,
54     output wire [4:0] rvfi_rs1_addr,
55     output wire [4:0] rvfi_rs2_addr,
56     output wire [31:0] rvfi_rs1_rdata,
57     output wire [31:0] rvfi_rs2_rdata,
58     output wire [4:0] rvfi_rd_addr,
59     output wire [31:0] rvfi_rd_wdata,
60     output wire [31:0] rvfi_pc_rdata,
61     output wire [31:0] rvfi_pc_wdata,
62     output wire [31:0] rvfi_mem_addr,
63     output wire [3:0] rvfi_mem_rmask,
64     output wire [3:0] rvfi_mem_wmask,
65     output wire [31:0] rvfi_mem_rdata,
66     output wire [31:0] rvfi_mem_wdata,
67   `endif
68   output wire [31:0] o_ibus_adr,
69   output wire    o_ibus_cyc,
70   input wire [31:0] i_ibus_rdt,
71   input wire    i_ibus_ack,
72   output wire [31:0] o_dbus_adr,
73   output wire [31:0] o_dbus_dat,
74   output wire [3:0] o_dbus_sel,
75   output wire    o_dbus_we,
76   output wire    o_dbus_cyc,
77   input wire [31:0] i_dbus_rdt,
78   input wire    i_dbus_ack,
79
80   // Extension
81   output wire [31:0] o_ext_rs1,
82   output wire [31:0] o_ext_rs2,
83   output wire [2:0] o_ext_funct3,
84   input wire [31:0] i_ext_rd,
85   input wire    i_ext_ready,
86   // MDU
87   output wire    o_mdu_valid);
88
89   localparam CSR_REGS = WITH_CSR*4;
90
91   wire      rf_wreq;
92   wire      rf_rreq;
93   wire [4+WITH_CSR:0] wreg0;
94   wire [4+WITH_CSR:0] wreg1;
95   wire      wen0;
96   wire      wen1;
97   wire [W-1:0] wdata0;
98   wire [W-1:0] wdata1;
99   wire [4+WITH_CSR:0] rreg0;
100  wire [4+WITH_CSR:0] rreg1;
101  wire      rf_ready;
102  wire [W-1:0] rdata0;
103  wire [W-1:0] rdata1;
104
105  wire [RF_L2D-1:0] waddr;
106  wire [RF_WIDTH-1:0] wdata;
107  wire      wen;
108  wire [RF_L2D-1:0] raddr;
109  wire      ren;
110  wire [RF_WIDTH-1:0] rdata;
111
112  serv_rf_ram_if
113  #(.width (RF_WIDTH),
114    .reset_strategy (RESET_STRATEGY),
115    .csr_regs (CSR_REGS),
116    .W(W))
117  rf_ram_if
118  (.i_clk (clk),
119   .i_rst (i_rst),
120   .i_wreq (rf_wreq),
121   .i_rreq (rf_rreq),
122   .o_ready (rf_ready),
123   .i_wreg0 (wreg0),
124   .i_wreg1 (wreg1),
125   .i_wen0 (wen0),
126   .i_wen1 (wen1),
127   .i_wdata0 (wdata0),
128   .i_wdata1 (wdata1),
129   .i_rreg0 (rreg0),
130   .i_rreg1 (rreg1),
131   .o_rdata0 (rdata0),
132   .o_rdata1 (rdata1),
133   .o_waddr (waddr),
134   .o_wdata (wdata),
135   .o_wen (wen),
136   .o_raddr (raddr),
137   .o_ren (ren),
138   .i_rdata (rdata));
139
140  serv_rf_ram
141  #(.width (RF_WIDTH),

```

```

142    .csr_regs (CSR_REGS))
143  rf_ram
144  (.i_clk (clk),
145   .i_waddr (waddr),
146   .i_wdata (wdata),
147   .i_wen (wen),
148   .i_raddr (raddr),
149   .i_ren (ren),
150   .o_rdata (rdata));
151
152  serv_top
153  #(.RESET_PC (RESET_PC),
154   .PRE_REGISTER (PRE_REGISTER),
155   .RESET_STRATEGY (RESET_STRATEGY),
156   .WITH_CSR (WITH_CSR),
157   .DEBUG (DEBUG),
158   .MDU (MDU),
159   .COMPRESSED (COMPRESSED),
160   .ALIGN (ALIGN),
161   .W(W))
162  cpu
163  (
164   .clk (clk),
165   .i_rst (i_rst),
166   .i_timer_irq (i_timer_irq),
167   `ifdef RISC_V_FORMAL
168     .rvfi_valid (rvfi_valid),
169     .rvfi_order (rvfi_order),
170     .rvfi_insn (rvfi_insn),
171     .rvfi_trap (rvfi_trap),
172     .rvfi_halt (rvfi_halt),
173     .rvfi_intr (rvfi_intr),
174     .rvfi_mode (rvfi_mode),
175     .rvfi_ixl (rvfi_ixl),
176     .rvfi_rs1_addr (rvfi_rs1_addr),
177     .rvfi_rs2_addr (rvfi_rs2_addr),
178     .rvfi_rs1_rdata (rvfi_rs1_rdata),
179     .rvfi_rs2_rdata (rvfi_rs2_rdata),
180     .rvfi_rd_addr (rvfi_rd_addr),
181     .rvfi_rd_wdata (rvfi_rd_wdata),
182     .rvfi_pc_rdata (rvfi_pc_rdata),
183     .rvfi_pc_wdata (rvfi_pc_wdata),
184     .rvfi_mem_addr (rvfi_mem_addr),
185     .rvfi_mem_rmask (rvfi_mem_rmask),
186     .rvfi_mem_wmask (rvfi_mem_wmask),
187     .rvfi_mem_rdata (rvfi_mem_rdata),
188     .rvfi_mem_wdata (rvfi_mem_wdata),
189   `endif
190   .o_rf_rreq (rf_rreq),
191   .o_rf_wreq (rf_wreq),
192   .i_rf_ready (rf_ready),
193   .o_wreg0 (wreg0),
194   .o_wreg1 (wreg1),
195   .o_wen0 (wen0),
196   .o_wen1 (wen1),
197   .o_wdata0 (wdata0),
198   .o_wdata1 (wdata1),
199   .o_rreg0 (rreg0),
200   .o_rreg1 (rreg1),
201   .i_rdata0 (rdata0),
202   .i_rdata1 (rdata1),
203
204   .o_ibus_adr (o_ibus_adr),
205   .o_ibus_cyc (o_ibus_cyc),
206   .i_ibus_rdt (i_ibus_rdt),
207   .i_ibus_ack (i_ibus_ack),
208
209   .o_dbus_adr (o_dbus_adr),
210   .o_dbus_dat (o_dbus_dat),
211   .o_dbus_sel (o_dbus_sel),
212   .o_dbus_we (o_dbus_we),
213   .o_dbus_cyc (o_dbus_cyc),
214   .i_dbus_rdt (i_dbus_rdt),
215   .i_dbus_ack (i_dbus_ack),
216
217   //Extension
218   .o_ext_funct3 (o_ext_funct3),
219   .i_ext_ready (i_ext_ready),
220   .i_ext_rd (i_ext_rd),
221   .o_ext_rs1 (o_ext_rs1),
222   .o_ext_rs2 (o_ext_rs2),
223   //MDU
224   .o_mdu_valid (o_mdu_valid));
225
226  endmodule
227  `default_nettype wire

```

16. olofk_serv/rtl/serv_state.v (239 lines)

```

1 /*
2  * serv_state.v : SERV module for handling internal state dur-
ing instructions
3  *
4  * SPDX-FileCopyrightText: 2019 Olof Kindgren <olof@award-win-
ning.me>
5  * SPDX-License-Identifier: ISC
6  */
7 module serv_state
8   #(.parameter RESET_STRATEGY = "MINI",
9     .parameter [0:0] WITH_CSR = 1,

```

```

10  parameter [0:0] ALIGN =0,
11  parameter [0:0] MDU = 0,
12  parameter      W = 1
13  )
14  (
15  input wire      i_clk,
16  input wire      i_rst,
17  //State
18  input wire      i_new_irq,
19  input wire      i_alu_cmp,
20  output wire      o_init,
21  output wire      o_cnt_en,
22  output wire      o_cnt0to3,
23  output wire      o_cnt12to31,
24  output wire      o_cnt0,
25  output wire      o_cnt1,
26  output wire      o_cnt2,
27  output wire      o_cnt3,
28  output wire      o_cnt7,
29  output wire      o_cnt11,
30  output wire      o_cnt12,
31  output wire      o_cnt_done,
32  output wire      o_bufreg_en,
33  output wire      o_ctrl_pc_en,
34  output reg       o_ctrl_jump,
35  output wire      o_ctrl_trap,
36  input wire      i_ctrl_misalign,
37  input wire      i_sh_done,
38  output wire [1:0] o_mem_bytecnt,
39  input wire      i_mem_misalign,
40  //Control
41  input wire      i_bne_or_bge,
42  input wire      i_cond_branch,
43  input wire      i_dbus_en,
44  input wire      i_two_stage_op,
45  input wire      i_branch_op,
46  input wire      i_shift_op,
47  input wire      i_sh_right,
48  input wire      i_alu_rd_sel1,
49  input wire      i_rd_alu_en,
50  input wire      i_e_op,
51  input wire      i_rd_op,
52  //MDU
53  input wire      i_mdu_op,
54  output wire      o_mdu_valid,
55  //Extension
56  input wire      i_mdu_ready,
57  //External
58  output wire      o_dbus_cyc,
59  input wire      i_dbus_ack,
60  output wire      o_ibus_cyc,
61  input wire      i_ibus_ack,
62  //RF Interface
63  output wire      o_rf_rreq,
64  output wire      o_rf_wreq,
65  input wire      i_rf_ready,
66  output wire      o_rf_rd_en);
67
68  reg init_done;
69  wire misalign_trap_sync;
70
71  reg [4:2] o_cnt;
72  wire [3:0] cnt_r;
73
74  reg      ibus_cyc;
75  //Update PC in RUN or TRAP states
76  assign o_ctrl_pc_en = o_cnt_en & !o_init;
77
78  assign o_mem_bytecnt = o_cnt[4:3];
79
80  assign o_cnt0to3 = (o_cnt[4:2] == 3'd0);
81  assign o_cnt12to31 = (o_cnt[4] | (o_cnt[3:2] == 2'b11));
82  assign o_cnt0 = (o_cnt[4:2] == 3'd0) & cnt_r[0];
83  assign o_cnt1 = (o_cnt[4:2] == 3'd0) & cnt_r[1];
84  assign o_cnt2 = (o_cnt[4:2] == 3'd0) & cnt_r[2];
85  assign o_cnt3 = (o_cnt[4:2] == 3'd0) & cnt_r[3];
86  assign o_cnt7 = (o_cnt[4:2] == 3'd1) & cnt_r[3];
87  assign o_cnt11 = (o_cnt[4:2] == 3'd2) & cnt_r[3];
88  assign o_cnt12 = (o_cnt[4:2] == 3'd3) & cnt_r[0];
89
90  //Take branch for jump or branch instructions (opcode ==
1x0xx) if
91  //a) It's an unconditional branch (opcode[0] == 1)
92  //b) It's a conditional branch (opcode[0] == 0) of type
beq,blt,bltu (funct3[0] == 0) and ALU compare is true
93  //c) It's a conditional branch (opcode[0] == 0) of type
bne,bge,bgeu (funct3[0] == 1) and ALU compare is false
94  //Only valid during the last cycle of INIT, when the branch
condition has
95  //been calculated.
96  wire      take_branch = i_branch_op & (!i_cond_branch |
(i_alu_cmp^i_bne_or_bge));
97
98  wire last_init = o_cnt_done & o_init;
99
100 //valid signal for mdu
101 assign o_mdu_valid = MDU & !o_cnt_en & init_done &
i_mdu_op;
102
103 //trap_pending is only guaranteed to have correct value

```

```

during the
104 // last cycle of the init stage
105 wiretrap_pending = WITH_CSR & ((take_branch & i_c-
trl_misalign & !ALIGN) |
106                                     (i_dbus_en &
i_mem_misalign));
107
108 //Prepare RF for writes when everything is ready to enter
stage two
109 // and the first stage didn't cause a misalign exception
110 //Left shifts, SLT & Branch ops. First cycle after init
111 //Right shift. o_sh_done
112 //Mem ops. i_dbus_ack
113 //MDU ops. i_mdu_ready

114 assign o_rf_wreq = (i_shift_op & (i_sh_right ? (i_sh_done &
(last_init | !o_cnt_en & init_done)) : last_init)) |
115                                     i_dbus_ack | (MDU & i_mdu_ready) |
116                                     (i_branch_op & (last_init & !trap_pend-
ing))) |
117                                     (i_rd_alu_en & i_alu_rd_sel1 & last-in-
it);
118
119 assign o_dbus_cyc = !o_cnt_en & init_done & i_dbus_en & !
i_mem_misalign;
120
121 //Prepare RF for reads when a new instruction is fetched
122 // or when stage one caused an exception (rreq implies a
write request too)
123 assign o_rf_rreq = i_ibus_ack | (trap_pending & last_init);
124
125 assign o_rf_rd_en = i_rd_op & !o_init;
126
127 /*
128  bufreg is used during mem, branch, and shift operations
129
130  mem : bufreg is used for dbus address. Shift in data dur-
ing phase 1.

131  Shift out during phase 2 if there was a misalignment
exception.
132
133  branch : Shift in during phase 1. Shift out during phase 2
134
135  shift : Shift in during phase 1. Continue shifting between
phases (except
136          for the first cycle after init). Shift out during
phase 2
137  */
138
139 assign o_bufreg_en = (o_cnt_en & (o_init | ((o_ctrl_trap |
i_branch_op & i_two_stage_op))) | (i_shift_op & init_done &
(i_sh_right | i_sh_done)));
140
141 assign o_ibus_cyc = ibus_cyc & !i_rst;
142
143 assign o_init = i_two_stage_op & !i_new_irq & !init_done;
144
145 assign o_cnt_done = (o_cnt[4:2] == 3'b111) & cnt_r[3];
146
147 always @(posedge i_clk) begin
148  //ibus_cyc changes on three conditions.
149  //1. i_rst is asserted. Together with the async gating
above, o_ibus_cyc
150  // will be asserted as soon as the reset is released.
This is how the
151  // first instruction is fetched
152  //2. o_cnt_done and o_ctrl_pc_en are asserted. This
means that SERV just
153  // finished updating the PC, is done with the current
instruction and
154  // o_ibus_cyc gets asserted to fetch a new instruction
155  //3. When i_ibus_ack, a new instruction is fetched and
o_ibus_cyc gets
156  // deasserted to finish the transaction
157  if (i_ibus_ack | o_cnt_done | i_rst)
158    ibus_cyc <= o_ctrl_pc_en | i_rst;
159
160  if (o_cnt_done) begin
161    init_done <= o_init & !init_done;
162    o_ctrl_jump <= o_init & take_branch;
163  end
164
165  if (i_rst) begin
166    if (RESET_STRATEGY != "NONE") begin
167      init_done <= 1'b0;
168      o_ctrl_jump <= 1'b0;
169    end
170  end
171 end
172
173 generate
174  /*
175  Because SERV is 32-bit bit-serial we need a counter
than can count 0-31
176
177  to keep track of which bit we are currently processing.
o_cnt and cnt_r
178  are used together to create such a counter.
The top three bits (o_cnt) are implemented as a normal

```

counter, but

```

179      instead of the two LSB, cnt_r is a 4-bit shift register
which loops 0-3
180      When cnt_r[3] is 1, o_cnt will be increased.
181
182      The counting starts when the core is idle and the
i_rf_ready signal
183      comes in from the RF module by shifting in the
i_rf_ready bit as LSB of
184      the shift register. Counting is stopped by using
o_cnt_done to block the
185      bit that was supposed to be shifted into bit 0 of
cnt_r.
186
187      There are two benefit of doing the counter this way
188
189      1. We only need to check four bits instead of five when
we want to check
190      if the counter is at a certain value. For 4-LUT archi-
tectures this means
191      we only need one LUT instead of two for each comparis-
on.
192
193      2. We don't need a separate enable signal to turn on
and off the counter
194      between stages, which saves an extra FF and a unique
control signal. We
195      just need to check if cnt_r is not zero to see if the
counter is
196      currently running
197      */
198      if (W == 1) begin : gen_cnt_w_eq_1
199          reg [3:0] cnt_lsb;
200          always @(posedge i_clk) begin
201              o_cnt <= o_cnt + {2'd0, cnt_r[3]};
202              cnt_lsb <= {cnt_lsb[2:0], (cnt_lsb[3] & !
o_cnt_done) | i_rf_ready};
203              if (i_rst & (RESET_STRATEGY != "NONE")) begin
204                  o_cnt <= 3'd0;
205                  cnt_lsb <= 4'b0000;
206              end
207          end
208          assign cnt_r = cnt_lsb;
209          assign o_cnt_en = |cnt_lsb;
210      end else if (W == 4) begin : gen_cnt_w_eq_4
211          reg cnt_en;
212          always @(posedge i_clk) begin
213              if (i_rf_ready) cnt_en <= 1; else
214              if (o_cnt_done) cnt_en <= 0;
215              o_cnt <= o_cnt + { 2'd0, cnt_en };
216              if (i_rst & (RESET_STRATEGY != "NONE")) begin
217                  o_cnt <= 3'd0;
218                  cnt_en <= 1'b0;
219              end
220          end
221          assign cnt_r = 4'b1111;
222          assign o_cnt_en = cnt_en;
223      end
224      endgenerate
225
226      assign o_ctrl_trap = WITH_CSR & (i_e_op | i_new_irq | mis-
align_trap_sync);
227
228      generate
229      if (WITH_CSR) begin : gen_csr
230          reg misalign_trap_sync_r;
231
232          always @(posedge i_clk) begin
233              if (i_ibus_ack | o_cnt_done | i_rst)
234              misalign_trap_sync_r <= !(i_ibus_ack | i_rst) &
((trap_pending & o_init) | misalign_trap_sync_r);
235              end
236              assign misalign_trap_sync = misalign_trap_sync_r;
237          end else begin : gen_no_csr
238              assign misalign_trap_sync = 1'b0;
239          end
240      endgenerate
241      endmodule

```

17. olofk_serv/rtl/serv_synth_wrapper.v (138 lines)

```

1 /*
2  * serv_synth_wrapper.v : SERV wrapper for synthesis
3  *
4  * SPDX-FileCopyrightText: 2021 Olof Kindgren <olof@award-win-
ning.me>
5  * SPDX-License-Identifier: ISC
6  */
7 `default_nettype none
8
9 module serv_synth_wrapper
10     #(
11         /* Register signals before or after the decoder
12         0 : Register after the decoder. Faster but uses more re-
sources
13         1 : (default) Register before the decoder. Slower but
uses less resources
14         */
15         parameter PRE_REGISTER = 1,
16         /* Amount of reset applied to design

```

```

17         "NONE" : No reset at all. Relies on a POR to set cor-
rect initialization
18         values and that core isn't reset during
runtime
19         "MINI" : Standard setting. Resets the minimal amount of
FFs needed to
20         restart execution from the instruction at RE-
SET_PC
21     */
22     parameter RESET_STRATEGY = "MINI",
23     parameter WITH_CSR = 1,
24     parameter RF_WIDTH = 2,
25     parameter RF_L2D = $clog2((32+(WITH_CSR*4))*32/
RF_WIDTH))
26     (
27         input wire          clk,
28         input wire          i_rst,
29         input wire          i_timer_irq,
30         output wire [31:0]  o_ibus_adr,
31         output wire         o_ibus_cyc,
32         input wire [31:0]   i_ibus_rdt,
33         input wire          i_ibus_ack,
34         output wire [31:0]  o_ibus_adr,
35         output wire [31:0]  o_dbus_dat,
36         output wire [3:0]   o_dbus_sel,
37         output wire         o_dbus_we ,
38         output wire         o_dbus_cyc,
39         input wire [31:0]   i_dbus_rdt,
40         input wire          i_dbus_ack,
41
42         output wire [RF_L2D-1:0] o_waddr,
43         output wire [RF_WIDTH-1:0] o_wdata,
44         output wire             o_wen,
45         output wire [RF_L2D-1:0] o_raddr,
46         input wire [RF_WIDTH-1:0] i_rdata;
47
48         localparam CSR_REGS = WITH_CSR*4;
49
50         wire          rf_wreq;
51         wire          rf_rreq;
52         wire [4+WITH_CSR:0] wreg0;
53         wire [4+WITH_CSR:0] wreg1;
54         wire             wen0;
55         wire             wen1;
56         wire             wdata0;
57         wire             wdata1;
58         wire [4+WITH_CSR:0] rreg0;
59         wire [4+WITH_CSR:0] rreg1;
60         wire             rf_ready;
61         wire             rdata0;
62         wire             rdata1;
63
64         serv_rf_ram_if
65         #(.width (RF_WIDTH),
66           .reset_strategy (RESET_STRATEGY),
67           .csr_regs (CSR_REGS))
68         rf_ram_if
69         (.i_clk (clk),
70          .i_rst (i_rst),
71          .i_wreq (rf_wreq),
72          .i_rreq (rf_rreq),
73          .o_ready (rf_ready),
74          .i_wreg0 (wreg0),
75          .i_wreg1 (wreg1),
76          .i_wen0 (wen0),
77          .i_wen1 (wen1),
78          .i_wdata0 (wdata0),
79          .i_wdata1 (wdata1),
80          .i_rreg0 (rreg0),
81          .i_rreg1 (rreg1),
82          .o_rdata0 (rdata0),
83          .o_rdata1 (rdata1),
84          .o_waddr (o_waddr),
85          .o_wdata (o_wdata),
86          .o_wen (o_wen),
87          .o_raddr (o_raddr),
88          .i_rdata (i_rdata));
89
90         serv_top
91         #(.RESET_PC (32'd0),
92           .PRE_REGISTER (PRE_REGISTER),
93           .RESET_STRATEGY (RESET_STRATEGY),
94           .WITH_CSR (WITH_CSR),
95           .MDU(1'b0))
96         cpu
97         (
98             .clk (clk),
99             .i_rst (i_rst),
100             .i_timer_irq (i_timer_irq),
101             .o_rf_rreq (rf_rreq),
102             .o_rf_wreq (rf_wreq),
103             .i_rf_ready (rf_ready),
104             .o_wreg0 (wreg0),
105             .o_wreg1 (wreg1),
106             .o_wen0 (wen0),
107             .o_wen1 (wen1),
108             .o_wdata0 (wdata0),
109             .o_wdata1 (wdata1),
110             .o_rreg0 (rreg0),
111             .o_rreg1 (rreg1),

```



```

112 .i_rdata0 (rdata0),
113 .i_rdata1 (rdata1),
114
115 .o_ibus_adr (o_ibus_adr),
116 .o_ibus_cyc (o_ibus_cyc),
117 .i_ibus_rdt (i_ibus_rdt),
118 .i_ibus_ack (i_ibus_ack),
119
120 .o_dbus_adr (o_dbus_adr),
121 .o_dbus_dat (o_dbus_dat),
122 .o_dbus_sel (o_dbus_sel),
123 .o_dbus_we (o_dbus_we),
124 .o_dbus_cyc (o_dbus_cyc),
125 .i_dbus_rdt (i_dbus_rdt),
126 .i_dbus_ack (i_dbus_ack),
127
128 //Extension
129 .o_ext_func3 (),
130 .i_ext_ready (1'b0),
131 .i_ext_rd (32'd0),
132 .o_ext_rs1 (),
133 .o_ext_rs2 (),
134 //MDU
135 .o_mdu_valid ();
136
137 endmodule
138 `default_nettype wire

```

18. olofk_serv/rtl/serv_top.v (674 lines)

```

1 /*
2  * serv_top.v : SERV toplevel
3  *
4  * SPDX-FileCopyrightText: 2018 Olof Kindgren <olof@award-win-
ning.me>
5  * SPDX-License-Identifier: ISC
6  */
7 `default_nettype none
8
9 module serv_top
10     #(parameter WITH_CSR = 1,
11       parameter W = 1,
12       parameter B = W-1,
13       parameter PRE_REGISTER = 1,
14       parameter RESET_STRATEGY = "MINI",
15       parameter RESET_PC = 32'd0,
16       parameter [0:0] DEBUG = 1'b0,
17       parameter [0:0] MDU = 1'b0,
18       parameter [0:0] COMPRESSED=0,
19       parameter [0:0] ALIGN = COMPRESSED)
20     (
21       input wire clk,
22       input wire i_rst,
23       input wire i_timer_irq,
24       `ifdef RISC_V Formal
25       output wire rvfi_valid,
26       output wire [63:0] rvfi_order,
27       output wire [31:0] rvfi_insn,
28       output wire rvfi_trap,
29       output wire rvfi_halt,
30       output wire rvfi_intr,
31       output wire [1:0] rvfi_mode,
32       output wire [1:0] rvfi_ixl,
33       output wire [4:0] rvfi_rs1_addr,
34       output wire [4:0] rvfi_rs2_addr,
35       output wire [31:0] rvfi_rs1_rdata,
36       output wire [31:0] rvfi_rs2_rdata,
37       output wire [4:0] rvfi_rd_addr,
38       output wire [31:0] rvfi_rd_wdata,
39       output wire [31:0] rvfi_pc_rdata,
40       output wire [31:0] rvfi_pc_wdata,
41       output wire [31:0] rvfi_mem_addr,
42       output wire [3:0] rvfi_mem_rmask,
43       output wire [3:0] rvfi_mem_wmask,
44       output wire [31:0] rvfi_mem_rdata,
45       output wire [31:0] rvfi_mem_wdata,
46       `endif
47       //RF Interface
48       output wire o_rf_rreq,
49       output wire o_rf_wreq,
50       input wire i_rf_ready,
51       output wire [4+WITH_CSR:0] o_wreg0,
52       output wire [4+WITH_CSR:0] o_wreg1,
53       output wire o_wen0,
54       output wire o_wen1,
55       output wire [B:0] o_wdata0,
56       output wire [B:0] o_wdata1,
57       output wire [4+WITH_CSR:0] o_rreg0,
58       output wire [4+WITH_CSR:0] o_rreg1,
59       input wire [B:0] i_rdata0,
60       input wire [B:0] i_rdata1,
61
62       output wire [31:0] o_ibus_adr,
63       output wire o_ibus_cyc,
64       input wire [31:0] i_ibus_rdt,
65       input wire i_ibus_ack,
66       output wire [31:0] o_dbus_adr,
67       output wire [31:0] o_dbus_dat,
68       output wire [3:0] o_dbus_sel,
69       output wire o_dbus_we,

```

```

70 output wire o_dbus_cyc,
71 input wire [31:0] i_dbus_rdt,
72 input wire i_dbus_ack,
73 //Extension
74 output wire [2:0] o_ext_func3,
75 input wire i_ext_ready,
76 input wire [31:0] i_ext_rd,
77 output wire [31:0] o_ext_rs1,
78 output wire [31:0] o_ext_rs2,
79 //MDU
80 output wire o_mdu_valid);
81
82 wire [4:0] rd_addr;
83 wire [4:0] rs1_addr;
84 wire [4:0] rs2_addr;
85
86 wire [3:0] immdec_ctrl;
87 wire [3:0] immdec_en;
88
89 wire sh_right;
90 wire bne_or_bge;
91 wire cond_branch;
92 wire two_stage_op;
93 wire e_op;
94 wire ebreak;
95 wire branch_op;
96 wire shift_op;
97 wire rd_op;
98 wire mdu_op;
99
100 wire rd_alu_en;
101 wire rd_csr_en;
102 wire rd_mem_en;
103 wire [B:0] ctrl_rd;
104 wire [B:0] alu_rd;
105 wire [B:0] mem_rd;
106 wire [B:0] csr_rd;
107 wire mtval_pc;
108
109 wire ctrl_pc_en;
110 wire jump;
111 wire jal_or_jalr;
112 wire utype;
113 wire mret;
114 wire [B:0] imm;
115 wire trap;
116 wire pc_rel;
117 wire iscomp;
118
119 wire init;
120 wire cnt_en;
121 wire cnt0to3;
122 wire cnt12to31;
123 wire cnt0;
124 wire cnt1;
125 wire cnt2;
126 wire cnt3;
127 wire cnt7;
128 wire cnt11;
129 wire cnt12;
130
131 wire cnt_done;
132
133 wire bufreg_en;
134 wire bufreg_sh_signed;
135 wire bufreg_rs1_en;
136 wire bufreg_imm_en;
137 wire bufreg_clr_lsb;
138 wire [B:0] bufreg_q;
139 wire [B:0] bufreg2_q;
140 wire [31:0] dbus_rdt;
141 wire dbus_ack;
142
143 wire alu_sub;
144 wire [1:0] alu_bool_op;
145 wire alu_cmp_eq;
146 wire alu_cmp_sig;
147 wire alu_cmp;
148 wire [2:0] alu_rd_sel;
149
150 wire [B:0] rs1;
151 wire [B:0] rs2;
152 wire rd_en;
153
154 wire [B:0] op_b;
155 wire op_b_sel;
156
157 wire mem_signed;
158 wire mem_word;
159 wire mem_half;
160 wire [1:0] mem_bytecnt;
161 wire sh_done;
162
163 wire mem_misalign;
164
165 wire [B:0] bad_pc;
166
167 wire csr_mstatus_en;
168 wire csr_mie_en;
169 wire csr_mcause_en;
170 wire [1:0] csr_source;

```



```

171 wire[B:0]    csr_imm;
172 wire        csr_d_sel;
173 wire        csr_en;
174 wire [1:0]   csr_addr;
175 wire [B:0]   csr_pc;
176 wire        csr_imm_en;
177 wire [B:0]   csr_in;
178 wire [B:0]   rf_csr_out;
179 wire        dbus_en;
180
181 wire         new_irq;
182
183 wire [1:0]    lsb;
184
185 wire [31:0]   i_wb_rdt;
186
187 wire [31:0]   wb_ibus_addr;
188 wire         wb_ibus_cyc;
189 wire [31:0]   wb_ibus_rdt;
190 wire         wb_ibus_ack;
191
192 generate
193   if (ALIGN) begin : gen_align
194     serv_aligner align
195     (
196       .clk(clk),
197       .rst(i_rst),
198       // serv_rf_top
199       .i_ibus_addr(wb_ibus_addr),
200       .i_ibus_cyc(wb_ibus_cyc),
201       .o_ibus_rdt(wb_ibus_rdt),
202       .o_ibus_ack(wb_ibus_ack),
203       // servant_arbiter
204       .o_wb_ibus_addr(o_ibus_addr),
205       .o_wb_ibus_cyc(o_ibus_cyc),
206       .i_wb_ibus_rdt(i_ibus_rdt),
207       .i_wb_ibus_ack(i_ibus_ack));
208   end else begin : gen_no_align
209     assign o_ibus_addr = wb_ibus_addr;
210     assign o_ibus_cyc = wb_ibus_cyc;
211     assign wb_ibus_rdt = i_ibus_rdt;
212     assign wb_ibus_ack = i_ibus_ack;
213   end
214 endgenerate
215
216 generate
217   if (COMPRESSED) begin : gen_compressed
218     serv_compdec compdec
219     (
220       .i_clk(clk),
221       .i_instr(wb_ibus_rdt),
222       .i_ack(wb_ibus_ack),
223       .o_instr(i_wb_rdt),
224       .o_iscomp(iscomp));
225   end else begin : gen_no_compressed
226     assign i_wb_rdt = wb_ibus_rdt;
227     assign iscomp = 1'b0;
228   end
229 endgenerate
230
231 serv_state
232   #(.RESET_STRATEGY (RESET_STRATEGY),
233     .WITH_CSR (WITH_CSR[0:0]),
234     .MDU(MDU),
235     .ALIGN(ALIGN),
236     .W(W))
237 state
238   (
239     .i_clk (clk),
240     .i_rst (i_rst),
241     //State
242     .i_new_irq (new_irq),
243     .i_alu_cmp (alu_cmp),
244     .o_init (init),
245     .o_cnt_en (cnt_en),
246     .o_cnt0to3 (cnt0to3),
247     .o_cnt12to31 (cnt12to31),
248     .o_cnt0 (cnt0),
249     .o_cnt1 (cnt1),
250     .o_cnt2 (cnt2),
251     .o_cnt3 (cnt3),
252     .o_cnt7 (cnt7),
253     .o_cnt11 (cnt11),
254     .o_cnt12 (cnt12),
255     .o_cnt_done (cnt_done),
256     .o_bufreg_en (bufreg_en),
257     .o_ctrl_pc_en (ctrl_pc_en),
258     .o_ctrl_jump (jump),
259     .o_ctrl_trap (trap),
260     .i_ctrl_misalign(lsb[1]),
261     .i_sh_done (sh_done),
262     .o_mem_bytecnt (mem_bytecnt),
263     .i_mem_misalign (mem_misalign),
264     //Control
265     .i_bne_or_bge (bne_or_bge),
266     .i_cond_branch (cond_branch),
267     .i_dbus_en (dbus_en),
268     .i_two_stage_op (two_stage_op),
269     .i_branch_op (branch_op),
270     .i_shift_op (shift_op),
271     .i_sh_right (sh_right),
272     .i_alu_rd_sel1 (alu_rd_sel[1]),
273     .i_rd_alu_en (rd_alu_en),
274     .i_e_op (e_op),
275     .i_rd_op (rd_op),
276     //MDU
277     .i_mdu_op (mdu_op),
278     .o_mdu_valid (o_mdu_valid),
279     //Extension
280     .i_mdu_ready (i_ext_ready),
281     //External
282     .o_dbus_cyc (o_dbus_cyc),
283     .i_dbus_ack (i_dbus_ack),
284     .o_ibus_cyc (wb_ibus_cyc),
285     .i_ibus_ack (wb_ibus_ack),
286     //RF Interface
287     .o_rf_rreq (o_rf_rreq),
288     .o_rf_wreq (o_rf_wreq),
289     .i_rf_ready (i_rf_ready),
290     .o_rf_rd_en (rd_en));
291
292 serv_decode
293   #(.PRE_REGISTER (PRE_REGISTER),
294     .MDU(MDU))
295 decode
296   (
297     .clk (clk),
298     //Input
299     .i_wb_rdt (i_wb_rdt[31:2]),
300     .i_wb_en (wb_ibus_ack),
301     //To state
302     .o_bne_or_bge (bne_or_bge),
303     .o_cond_branch (cond_branch),
304     .o_dbus_en (dbus_en),
305     .o_e_op (e_op),
306     .o_ebreak (ebreak),
307     .o_branch_op (branch_op),
308     .o_shift_op (shift_op),
309     .o_rd_op (rd_op),
310     .o_sh_right (sh_right),
311     .o_mdu_op (mdu_op),
312     .o_two_stage_op (two_stage_op),
313     //Extension
314     .o_ext_funct3 (o_ext_funct3),
315
316     //To bufreg
317     .o_bufreg_rs1_en (bufreg_rs1_en),
318     .o_bufreg_imm_en (bufreg_imm_en),
319     .o_bufreg_clr_lsb (bufreg_clr_lsb),
320     .o_bufreg_sh_signed (bufreg_sh_signed),
321     //To bufreg2
322     .o_op_b_source (op_b_sel),
323     //To ctrl
324     .o_ctrl_jal_or_jalr (jal_or_jalr),
325     .o_ctrl_utyte (utyte),
326     .o_ctrl_pc_rel (pc_rel),
327     .o_ctrl_mret (mret),
328     //To alu
329     .o_alu_sub (alu_sub),
330     .o_alu_bool_op (alu_bool_op),
331     .o_alu_cmp_eq (alu_cmp_eq),
332     .o_alu_cmp_sig (alu_cmp_sig),
333     .o_alu_rd_sel (alu_rd_sel),
334     //To mem IF
335     .o_mem_cmd (o_dbus_we),
336     .o_mem_signed (mem_signed),
337     .o_mem_word (mem_word),
338     .o_mem_half (mem_half),
339     //To CSR
340     .o_csr_en (csr_en),
341     .o_csr_addr (csr_addr),
342     .o_csr_mstatus_en (csr_mstatus_en),
343     .o_csr_mie_en (csr_mie_en),
344     .o_csr_mcause_en (csr_mcause_en),
345     .o_csr_source (csr_source),
346     .o_csr_d_sel (csr_d_sel),
347     .o_csr_imm_en (csr_imm_en),
348     .o_mtval_pc (mtval_pc),
349     //To top
350     .o_immdec_ctrl (immdec_ctrl),
351     .o_immdec_en (immdec_en),
352     //To RF IF
353     .o_rd_mem_en (rd_mem_en),
354     .o_rd_csr_en (rd_csr_en),
355     .o_rd_alu_en (rd_alu_en);
356
357 serv_immdec #(.W (W)) immdec
358   (
359     .i_clk (clk),
360     //State
361     .i_cnt_en (cnt_en),
362     .i_cnt_done (cnt_done),
363     //Control
364     .i_immdec_en (immdec_en),
365     .i_csr_imm_en (csr_imm_en),
366     .i_ctrl (immdec_ctrl),
367     .o_rd_addr (rd_addr),
368     .o_rs1_addr (rs1_addr),
369     .o_rs2_addr (rs2_addr),
370     //Data
371     .o_csr_imm (csr_imm),
372     .o_imm (imm),

```

```

373 //External
374 .i_wb_en      (wb_ibus_ack),
375 .i_wb_rdt     (i_wb_rdt[31:7]);
376
377 serv_bufreg
378 #(.MDU(MDU),
379  .W(W))
380 bufreg
381 (
382  .i_clk      (clk),
383  //State
384  .i_cnt0     (cnt0),
385  .i_cnt1     (cnt1),
386  .i_cnt_done (cnt_done),
387  .i_en       (bufreg_en),
388  .i_init     (init),
389  .i_mdu_op   (mdu_op),
390  .o_lsb      (lsb),
391  //Control
392  .i_sh_signed (bufreg_sh_signed),
393  .i_rs1_en    (bufreg_rs1_en),
394  .i_imm_en    (bufreg_imm_en),
395  .i_clr_lsb   (bufreg_clr_lsb),
396  .i_shift_op  (shift_op),
397  .i_right_shift_op (sh_right),
398  .i_shamt     (o_dbus_dat[26:24]),
399  //Data
400  .i_rs1       (rs1),
401  .i_imm       (imm),
402  .o_q         (bufreg_q),
403  //External
404  .o_dbus_addr (o_dbus_addr),
405  .o_ext_rs1   (o_ext_rs1));
406
407 serv_bufreg2 #(.W(W)) bufreg2
408 (
409  .i_clk      (clk),
410  //State
411  .i_en       (cnt_en),
412  .i_init     (init),
413  .i_cnt7     (cnt7),
414  .i_cnt_done (cnt_done),
415  .i_sh_right (sh_right),
416  .i_lsb      (lsb),
417  .i_bytecnt  (mem_bytecnt),
418  .o_sh_done  (sh_done),
419  //Control
420  .i_op_b_sel (op_b_sel),
421  .i_shift_op (shift_op),
422  //Data
423  .i_rs2      (rs2),
424  .i_imm      (imm),
425  .o_op_b     (op_b),
426  .o_q        (bufreg2_q),
427  //External
428  .o_dat      (o_dbus_dat),
429  .i_load     (dbus_ack),
430  .i_dat      (dbus_rdt));
431
432 serv_ctrl
433 #(.RESET_PC (RESET_PC),
434  .RESET_STRATEGY (RESET_STRATEGY),
435  .WITH_CSR (WITH_CSR),
436  .W(W))
437 ctrl
438 (
439  .clk      (clk),
440  .i_rst    (i_rst),
441  //State
442  .i_pc_en  (ctrl_pc_en),
443  .i_cnt12to31 (cnt12to31),
444  .i_cnt0   (cnt0),
445  .i_cnt1   (cnt1),
446  .i_cnt2   (cnt2),
447  //Control
448  .i_jump   (jump),
449  .i_jal_or_jalr (jal_or_jalr),
450  .i_utyte  (utyte),
451  .i_pc_rel (pc_rel),
452  .i_trap   (trap | mret),
453  .i_iscomp (iscomp),
454  //Data
455  .i_imm    (imm),
456  .i_buf    (bufreg_q),
457  .i_csr_pc (csr_pc),
458  .o_rd     (ctrl_rd),
459  .o_bad_pc (bad_pc),
460  //External
461  .o_ibus_addr (wb_ibus_addr));
462
463 serv_alu #(.W(W)) alu
464 (
465  .clk      (clk),
466  //State
467  .i_en     (cnt_en),
468  .i_cnt0   (cnt0),
469  .o_cmp    (alu_cmp),
470  //Control
471  .i_sub    (alu_sub),
472  .i_bool_op (alu_bool_op),
473  .i_cmp_eq (alu_cmp_eq),
474  .i_cmp_sig (alu_cmp_sig),
475  .i_rd_sel  (alu_rd_sel),
476  //Data
477  .i_rs1     (rs1),
478  .i_op_b    (op_b),
479  .i_buf     (bufreg_q),
480  .o_rd      (alu_rd));
481
482 serv_rf_if
483 #(.WITH_CSR (WITH_CSR), .W(W))
484 rf_if
485 (//RF interface
486  .i_cnt_en  (cnt_en),
487  .o_wreg0   (o_wreg0),
488  .o_wreg1   (o_wreg1),
489  .o_wen0    (o_wen0),
490  .o_wen1    (o_wen1),
491  .o_wdata0  (o_wdata0),
492  .o_wdata1  (o_wdata1),
493  .o_rreg0   (o_rreg0),
494  .o_rreg1   (o_rreg1),
495  .i_rdata0  (i_rdata0),
496  .i_rdata1  (i_rdata1),
497  //Trap interface
498  .i_trap    (trap),
499  .i_mret    (mret),
500  .i_mepc    (wb_ibus_addr[B:0]),
501  .i_mtval_pc (mtval_pc),
502  .i_bufreg_q (bufreg_q),
503  .i_bad_pc   (bad_pc),
504  .o_csr_pc   (csr_pc),
505  //CSR write port
506  .i_csr_en   (csr_en),
507  .i_csr_addr (csr_addr),
508  .i_csr      (csr_in),
509  //RD write port
510  .i_rd_wen   (rd_en),
511  .i_rd_waddr (rd_addr),
512  .i_ctrl_rd  (ctrl_rd),
513  .i_alu_rd   (alu_rd),
514  .i_rd_alu_en (rd_alu_en),
515  .i_csr_rd   (csr_rd),
516  .i_rd_csr_en (rd_csr_en),
517  .i_mem_rd   (mem_rd),
518  .i_rd_mem_en (rd_mem_en),
519  //RS1 read port
520  .i_rs1_raddr (rs1_addr),
521  .o_rs1       (rs1),
522  //RS2 read port
523  .i_rs2_raddr (rs2_addr),
524  .o_rs2       (rs2),
525  //CSR read port
526  .o_csr       (rf_csr_out));
527
528 serv_mem_if
529 #(.WITH_CSR (WITH_CSR[0:0]),
530  .W(W))
531 mem_if
532 (
533  .i_clk      (clk),
534  //State
535  .i_bytecnt  (mem_bytecnt),
536  .i_lsb      (lsb),
537  .o_misalign (mem_misalign),
538  //Control
539  .i_mdu_op   (mdu_op),
540  .i_signed   (mem_signed),
541  .i_word     (mem_word),
542  .i_half     (mem_half),
543  //Data
544  .i_bufreg2_q (bufreg2_q),
545  .o_rd        (mem_rd),
546  //External interface
547  .o_wb_sel   (o_dbus_sel));
548
549 generate
550 if (!WITH_CSR) begin : gen_csr
551   serv_csr
552   #(.RESET_STRATEGY (RESET_STRATEGY),
553    .W(W))
554   csr
555   (
556    .i_clk      (clk),
557    .i_rst      (i_rst),
558    //State
559    .i_trig_irq (wb_ibus_ack),
560    .i_en       (cnt_en),
561    .i_cnt0to3  (cnt0to3),
562    .i_cnt3     (cnt3),
563    .i_cnt7     (cnt7),
564    .i_cnt11    (cnt11),
565    .i_cnt12    (cnt12),
566    .i_cnt_done (cnt_done),
567    .i_mem_op   (!mtval_pc),
568    .i_mtip     (i_timer_irq),
569    .i_trap     (trap),
570    .o_new_irq  (new_irq),
571    //Control

```

```

575     .i_e_op      (e_op),
576     .i_ebreak   (ebreak),
577     .i_mem_cmd   (o_dbus_we),
578     .i_mstatus_en (csr_mstatus_en),
579     .i_mie_en    (csr_mie_en ),
580     .i_mcause_en (csr_mcause_en ),
581     .i_csr_source (csr_source),
582     .i_mret      (mret),
583     .i_csr_d_sel  (csr_d_sel),
584     //Data
585     .i_rf_csr_out (rf_csr_out),
586     .o_csr_in     (csr_in),
587     .i_csr_imm    (csr_imm),
588     .i_rs1        (rs1),
589     .o_q          (csr_rd));
590 end else begin : gen_no_csr
591     assign csr_in = {W{1'b0}};
592     assign csr_rd = {W{1'b0}};
593     assign new_irq = 1'b0;
594 end
595 endgenerate
596
597 generate
598     if (DEBUG) begin : gen_debug
599         serv_debug #(.W (W), .RESET_PC (RESET_PC)) debug
600             (
601 `ifdef RISC_V_FORMAL
602         .rvfi_valid      (rvfi_valid      ),
603         .rvfi_order      (rvfi_order      ),
604         .rvfi_insn       (rvfi_insn       ),
605         .rvfi_trap       (rvfi_trap       ),
606         .rvfi_halt       (rvfi_halt       ),
607         .rvfi_intr       (rvfi_intr       ),
608         .rvfi_mode       (rvfi_mode       ),
609         .rvfi_ixl        (rvfi_ixl        ),
610         .rvfi_rs1_addr   (rvfi_rs1_addr   ),
611         .rvfi_rs2_addr   (rvfi_rs2_addr   ),
612         .rvfi_rs1_rdata  (rvfi_rs1_rdata  ),
613         .rvfi_rs2_rdata  (rvfi_rs2_rdata  ),
614         .rvfi_rd_addr    (rvfi_rd_addr    ),
615         .rvfi_rd_wdata   (rvfi_rd_wdata   ),
616         .rvfi_pc_rdata   (rvfi_pc_rdata   ),
617         .rvfi_pc_wdata   (rvfi_pc_wdata   ),
618         .rvfi_mem_addr   (rvfi_mem_addr   ),
619         .rvfi_mem_rmask  (rvfi_mem_rmask  ),
620         .rvfi_mem_wmask  (rvfi_mem_wmask  ),
621         .rvfi_mem_rdata  (rvfi_mem_rdata  ),
622         .rvfi_mem_wdata  (rvfi_mem_wdata  ),
623         .i_dbus_adr      (o_dbus_adr),
624         .i_dbus_dat      (o_dbus_dat),

```

```

625         .i_dbus_sel      (o_dbus_sel),
626         .i_dbus_we       (o_dbus_we ),
627         .i_dbus_rdt      (i_dbus_rdt),
628         .i_dbus_ack      (i_dbus_ack),
629         .i_ctrl_pc_en    (ctrl_pc_en),
630         .rs1             (rs1),
631         .rs2             (rs2),
632         .rs1_addr        (rs1_addr),
633         .rs2_addr        (rs2_addr),
634         .immedec_en      (immedec_en),
635         .rd_en           (rd_en),
636         .trap            (trap),
637         .i_rf_ready      (i_rf_ready),
638         .i_ibus_cyc      (o_ibus_cyc),
639         .two_stage_op    (two_stage_op),
640         .init            (init),
641         .i_ibus_adr      (o_ibus_adr),
642 `endif
643         .i_clk           (clk),
644         .i_rst           (i_rst),
645         .i_ibus_rdt      (i_ibus_rdt),
646         .i_ibus_ack      (i_ibus_ack),
647         .i_rd_addr       (rd_addr      ),
648         .i_cnt_en        (cnt_en       ),
649         .i_csr_in        (csr_in       ),
650         .i_csr_mstatus_en (csr_mstatus_en),
651         .i_csr_mie_en    (csr_mie_en   ),
652         .i_csr_mcause_en (csr_mcause_en ),
653         .i_csr_en        (csr_en       ),
654         .i_csr_addr      (csr_addr),
655         .i_wen0          (o_wen0),
656         .i_wdata0        (o_wdata0),
657         .i_cnt_done      (cnt_done));
658     end
659 endgenerate
660
661 generate
662     if (MDU) begin: gen_mdu
663         assign dbus_rdt = i_ext_ready ? i_ext_rd:i_dbus_rdt;
664         assign dbus_ack = i_dbus_ack | i_ext_ready;
665     end else begin : gen_no_mdu
666         assign dbus_rdt = i_dbus_rdt;
667         assign dbus_ack = i_dbus_ack;
668     end
669     assign o_ext_rs2 = o_dbus_dat;
670 endgenerate
671
672 endmodule
673
674 `default_nettype wire

```

형식검증된 AXI/Wishbone 브리지

What this is. A bridge between Wishbone and AXI, developed with formal verification throughout.

Why it exists. Because a bus bridge is where protocol violations are introduced, and because protocol conformance is exactly the kind of local, bounded property that formal methods settle completely.

Undergraduate review · EECS concepts. Formal property verification; protocol assertions; induction and bounded model checking.

Graduate · Practice note. Read the assertions before the implementation. They are a precise, executable statement of the AXI rules and are more instructive than the protocol specification's prose.

6 files • 4,175 lines

API — addrdecode.v

Table 1794. ZipCPU_wb2axip/rtl/addrdecode.v — module / interface / package (1개)

이름	종류
addrdecode	module

Table 1795. ZipCPU_wb2axip/rtl/addrdecode.v — parameter (5개)

파라미터	형/폭	기본값
NS		8
AW		32
ACCESS_ALLOWED	[NS-1:0]	-1
OPT_REGISTERED	[0:0]	0
OPT_LOWPOWER	[0:0]	0

Table 1796. ZipCPU_wb2axip/rtl/addrdecode.v — localparam (1개)

상수	형/폭	값
OPT_NONESEL	[0:0]	(!ACCESS_ALLOWED[0])

Table 1797. ZipCPU_wb2axip/rtl/addrdecode.v — 포트 (10개)

포트	방향	형/폭
i_clk	input	wire
i_valid	input	wire
o_stall	output	reg
i_addr	input	wire [AW-1:0]
i_data	input	wire [DW-1:0]
o_valid	output	reg

포트	방향	형/폭
i_stall	input	wire
o_decode	output	reg [NS:0]
o_addr	output	reg [AW-1:0]
o_data	output	reg [DW-1:0]

API — afifo.v

Table 1798. ZipCPU_wb2axip/rtl/afifo.v — module / interface / package (1개)

이름	종류
afifo	module

Table 1799. ZipCPU_wb2axip/rtl/afifo.v — parameter (5개)

파라미터	형/폭	기본값
LGFIFO		3
WIDTH		16
NFF		2
WRITE_ON_POSEDGE	[0:0]	1'b1
OPT_REGISTER_READS	[0:0]	1'b1

Table 1800. ZipCPU_wb2axip/rtl/afifo.v — localparam (1개)

상수	형/폭	값
MSB		LGFIFO

Table 1801. ZipCPU_wb2axip/rtl/afifo.v — 포트 (6개)

포트	방향	형/폭
i_wclk	input	wire
i_wr_data	input	wire [WIDTH-1:0]
o_wr_full	output	reg
i_rclk	input	wire
o_rd_data	output	reg [WIDTH-1:0]
o_rd_empty	output	reg

Table 1802. ZipCPU_wb2axip/rtl/afifo.v — #define / `define 매크로 (4개)

매크로	인자	정의
ASSERT		assert
ASSUME		assume
ASSERT		assert
ASSUME		assert

API — apbslave.v**Table 1803. ZipCPU_wb2axip/rtl/apbslave.v — module / interface / package (1개)**

이름	종류
apbslave	module

Table 1804. ZipCPU_wb2axip/rtl/apbslave.v — parameter (2개)

파라미터	형/폭	기본값
C_APB_ADDR_WIDTH		12
C_APB_DATA_WIDTH		32

Table 1805. ZipCPU_wb2axip/rtl/apbslave.v — localparam (3개)

상수	형/폭	값
AW		C_APB_ADDR_WIDTH
DW		C_APB_DATA_WIDTH
APBLSB		$\$clog2(C_APB_DATA_WIDTH)-3$

Table 1806. ZipCPU_wb2axip/rtl/apbslave.v — 포트 (11개)

포트	방향	형/폭
PCLK	input	wire
PSEL	input	wire
PENABLE	input	wire
PREADY	output	reg
PADDR	input	wire [AW-1:0]
PWRITE	input	wire
PWDATA	input	wire [DW-1:0]
PWSTRB	input	wire [DW/8-1:0]
PPROT	input	wire [2:0]
PRDATA	output	reg [DW-1:0]
PSLVERR	output	wire

API — apbxclock.v**Table 1807. ZipCPU_wb2axip/rtl/apbxclock.v — module / interface / package (1개)**

이름	종류
apbxclock	module

Table 1808. ZipCPU_wb2axip/rtl/apbxclock.v — parameter (3개)

파라미터	형/폭	기본값
C_APB_ADDR_WIDTH		12
C_APB_DATA_WIDTH		32

파라미터	형/폭	기본값
OPT_REGISTERED	[0:0]	1'b0

Table 1809. ZipCPU_wb2axip/rtl/apbxclock.v — localparam (3개)

상수	형/폭	값
AW		C_APB_ADDR_WIDTH
DW		C_APB_DATA_WIDTH
F_NCLK		5

Table 1810. ZipCPU_wb2axip/rtl/apbxclock.v — 포트 (23개)

포트	방향	형/폭
S_APB_PCLK	input	wire
S_APB_PSEL	input	wire
S_APB_PENABLE	input	wire
S_APB_PREADY	output	reg
S_APB_PADDR	input	wire [AW-1:0]
S_APB_PWRITE	input	wire
S_APB_PWDATA	input	wire [DW-1:0]
S_APB_PWSTRB	input	wire [DW/8-1:0]
S_APB_PPROT	input	wire [2:0]
S_APB_PRDATA	output	wire [DW-1:0]
S_APB_PSLVERR	output	wire
M_APB_PCLK	input	wire
M_PRESETn	output	reg
M_APB_PSEL	output	reg
M_APB_PENABLE	output	reg
M_APB_PREADY	input	wire
M_APB_PADDR	output	wire [AW-1:0]
M_APB_PWRITE	output	wire
M_APB_PWDATA	output	wire [DW-1:0]
M_APB_PWSTRB	output	wire [DW/8-1:0]
M_APB_PPROT	output	wire [2:0]
M_APB_PRDATA	input	wire [DW-1:0]
M_APB_PSLVERR	input	wire

API — axi2axi3.v**Table 1811. ZipCPU_wb2axip/rtl/axi2axi3.v — module / interface / package (17개)**

이름	종류
axi2axi3	module

Table 1812. ZipCPU_wb2axip/rtl/axi2axi3.v — parameter (4개)

파라미터	형/폭	기본값
C_AXI_ID_WIDTH		1
C_AXI_ADDR_WIDTH		32
C_AXI_DATA_WIDTH		32
LGWFIFO		8

Table 1813. ZipCPU_wb2axip/rtl/axi2axi3.v — localparam (4개)

상수	형/폭	값
ADDRLSB		$\$clog2(C_AXI_DATA_WIDTH)-3$
OPT_LOWPOWER	[0:0]	1'b0
LGWFIFO		4
NID		$(1 << C_AXI_ID_WIDTH)$

Table 1814. ZipCPU_wb2axip/rtl/axi2axi3.v — 포트 (77개) — 앞 60개만 표시(전체 77개, 본문 소스 참조)

포트	방향	형/폭
S_AXI_ACLK	input	wire
S_AXI_ARESETN	input	wire
S_AXI_AWVALID	input	reg
S_AXI_AWREADY	output	wire
S_AXI_AWID	input	reg [C_AXI_ID_WIDTH-1:0]
S_AXI_AWADDR	input	reg [C_AXI_ADDR_WIDTH-1:0]
S_AXI_AWLEN	input	reg [7:0]
S_AXI_AWSIZE	input	reg [2:0]
S_AXI_AWBURST	input	reg [1:0]
S_AXI_AWLOCK	input	reg
S_AXI_AWCACHE	input	reg [3:0]
S_AXI_AWPROT	input	reg [2:0]
S_AXI_AWQOS	input	reg [3:0]
S_AXI_WVALID	input	wire
S_AXI_WREADY	output	wire
S_AXI_WDATA	input	wire [C_AXI_DATA_WIDTH-1:0]
S_AXI_WSTRB	input	wire [C_AXI_DATA_WIDTH/8-1:0]
S_AXI_WLAST	input	wire

포트	방향	형/폭
S_AXI_BVALID	output	wire
S_AXI_BREADY	input	wire
S_AXI_BID	output	wire [C_AXI_ID_WIDTH-1:0]
S_AXI_BRESP	output	wire [1:0]
S_AXI_ARVALID	input	wire
S_AXI_ARREADY	output	wire
S_AXI_ARID	input	wire [C_AXI_ID_WIDTH-1:0]
S_AXI_ARADDR	input	wire [C_AXI_ADDR_WIDTH-1:0]
S_AXI_ARLEN	input	wire [7:0]
S_AXI_ARSIZE	input	wire [2:0]
S_AXI_ARBURST	input	wire [1:0]
S_AXI_ARLOCK	input	wire
S_AXI_ARCACHE	input	wire [3:0]
S_AXI_ARPROT	input	wire [2:0]
S_AXI_ARQOS	input	wire [3:0]
S_AXI_RVALID	output	wire
S_AXI_RREADY	input	wire
S_AXI_RID	output	wire [C_AXI_ID_WIDTH-1:0]
S_AXI_RDATA	output	wire [C_AXI_DATA_WIDTH-1:0]
S_AXI_RLAST	output	wire
S_AXI_RRESP	output	wire [1:0]
M_AXI_AWVALID	output	wire
M_AXI_AWREADY	input	wire
M_AXI_AWID	output	wire [C_AXI_ID_WIDTH-1:0]
M_AXI_AWADDR	output	wire [C_AXI_ADDR_WIDTH-1:0]
M_AXI_AWLEN	output	wire [3:0]
M_AXI_AWSIZE	output	wire [2:0]
M_AXI_AWBURST	output	wire [1:0]
M_AXI_AWLOCK	output	wire [1:0]
M_AXI_AWCACHE	output	wire [3:0]
M_AXI_AWPROT	output	wire [2:0]
M_AXI_AWQOS	output	wire [3:0]
M_AXI_WVALID	output	wire
M_AXI_WREADY	input	wire
M_AXI_WID	output	wire [C_AXI_ID_WIDTH-1:0]

포트	방향	형/폭
M_AXI_WDATA	output	wire [C_AXI_DATA_WIDTH-1:0]
M_AXI_WSTRB	output	wire [C_AXI_DATA_WIDTH/8-1:0]
M_AXI_WLAST	output	wire
M_AXI_BVALID	input	wire
M_AXI_BREADY	output	wire
M_AXI_BID	input	wire [C_AXI_ID_WIDTH-1:0]
M_AXI_BRESP	input	wire [1:0]

API — axi2axilite.v

Table 1815. ZipCPU_wb2axip/rtl/axi2axilite.v — module / interface / package (1개)

이름	종류
axi2axilite	module

Table 1816. ZipCPU_wb2axip/rtl/axi2axilite.v — parameter (7개)

파라미터	형/폭	기본값
C_AXI_ID_WIDTH	integer	2
C_AXI_DATA_WIDTH	integer	32
C_AXI_ADDR_WIDTH	integer	6
OPT_WRITES	[0:0]	1
OPT_READS	[0:0]	1
OPT_LOWPOWER	[0:0]	0
LGFIFO		4

Table 1817. ZipCPU_wb2axip/rtl/axi2axilite.v — localparam (5개)

상수	형/폭	값
SLVERR	[1:0]	2'b10
AW		C_AXI_ADDR_WIDTH
DW		C_AXI_DATA_WIDTH
IW		C_AXI_ID_WIDTH
F_LGDEPTH		LGFIFO+1+8

Table 1818. ZipCPU_wb2axip/rtl/axi2axilite.v — 포트 (58개)

포트	방향	형/폭
S_AXI_ACLK	input	wire
S_AXI_ARESETN	input	wire
S_AXI_AWVALID	input	wire
S_AXI_AWREADY	output	wire

포트	방향	형/폭
S_AXI_AWID	input	wire [C_AXI_ID_WIDTH-1:0]
S_AXI_AWADDR	input	wire [C_AXI_ADDR_WIDTH-1:0]
S_AXI_AWLEN	input	wire [7:0]
S_AXI_AWSIZE	input	wire [2:0]
S_AXI_AWBURST	input	wire [1:0]
S_AXI_AWLOCK	input	wire
S_AXI_AWCACHE	input	wire [3:0]
S_AXI_AWPROT	input	wire [2:0]
S_AXI_AWQOS	input	wire [3:0]
S_AXI_WVALID	input	wire
S_AXI_WREADY	output	wire
S_AXI_WDATA	input	wire [C_AXI_DATA_WIDTH-1:0]
8	input	wire [(C_AXI_DATA_WIDTH/
S_AXI_WLAST	input	wire
S_AXI_BVALID	output	wire
S_AXI_BREADY	input	wire
S_AXI_BID	output	wire [C_AXI_ID_WIDTH-1:0]
S_AXI_BRESP	output	wire [1:0]
S_AXI_ARVALID	input	wire
S_AXI_ARREADY	output	wire
S_AXI_ARID	input	wire [C_AXI_ID_WIDTH-1:0]
S_AXI_ARADDR	input	wire [C_AXI_ADDR_WIDTH-1:0]
S_AXI_ARLEN	input	wire [7:0]
S_AXI_ARSIZE	input	wire [2:0]
S_AXI_ARBURST	input	wire [1:0]
S_AXI_ARLOCK	input	wire
S_AXI_ARCACHE	input	wire [3:0]
S_AXI_ARPROT	input	wire [2:0]
S_AXI_ARQOS	input	wire [3:0]
S_AXI_RVALID	output	wire
S_AXI_RREADY	input	wire
S_AXI_RID	output	wire [C_AXI_ID_WIDTH-1:0]
S_AXI_RDATA	output	wire [C_AXI_DATA_WIDTH-1:0]
S_AXI_RRESP	output	wire [1:0]
S_AXI_RLAST	output	wire

포트	방향	형/폭
M_AXI_AWADDR	output	wire [C_AXI_ADDR_WIDTH-1:0]
M_AXI_AWPROT	output	wire [2 : 0]
M_AXI_AWVALID	output	wire
M_AXI_AWREADY	input	wire
M_AXI_WDATA	output	wire [C_AXI_DATA_WIDTH-1:0]
8	output	wire [(C_AXI_DATA_WIDTH/
M_AXI_WVALID	output	wire
M_AXI_WREADY	input	wire
M_AXI_BRESP	input	wire [1 : 0]
M_AXI_BVALID	input	wire
M_AXI_BREADY	output	wire
M_AXI_ARADDR	output	wire [C_AXI_ADDR_WIDTH-1:0]
M_AXI_ARPROT	output	wire [2:0]
M_AXI_ARVALID	output	wire
M_AXI_ARREADY	input	wire
M_AXI_RVALID	input	wire
M_AXI_RREADY	output	wire
M_AXI_RDATA	input	wire [C_AXI_DATA_WIDTH-1 : 0]
M_AXI_RRESP	input	wire [1 : 0]

1. ZipCPU_wb2axip/rtl/addrdecode.v (467 lines)

```

1 ///////////////////////////////////////////////////////////////////
2 //
3 // Filename:    rtl/addrdecode.v
4 // {{{
5 // Project:    WB2AXIPSP: bus bridges and other odds and ends
6 //
7 // Purpose:    Supports bus crossbars by answering the ques-
tion, which slave
8 //
9 // does the current address need to be routed to? Re-
quests are
10 // pipelined using valid!/stall handshaking. For those
familiar with
11 // AXI, READY!=STALL. The outgoing stream is identical to
the incoming
12 // one, save for the (new) o_decode field. This is a bit-
mask containing
13 // one bit for each slave that the request might be routed
to, and one
14 // extra bit to indicate no slaves matched.
15 //
16 // The keys to the operation of this module are found in
the two
17 // parameters, SLAVE_ADDR and SLAVE_MASK.
18 //
19 // SLAVE_ADDR specifies the address of the slave in ques-
tion. It's a large
20 // array, with one address (i.e. one set of AW
bits) for each
21 // potential slave address region.
22 //
23 // SLAVE_MASK specifies which of the bits in SLAVE_ADDR
need to match in
24 // order to route a request to a that slave.
25 //
26 // It's important to guarantee that no two slaves will
ever map to the

```

```

26 // same address, and likewise any given slave may only map to
a single
27 // address region.
28 //
29 // Incidentally, the algorithm forces all slaves to have
an address
30 // aligned with their memory size. Hence a 2GB memory
must have an
31 // address (range) of either 0-2GB, 2GB-4GB, 4GB-6GB,
etc. However, a
32 // second slave having only 8kB of memory may be placed at
0-8kB,
33 // 8-16kB, 16-24kB, etc. For logic minimization purposes, it
is often to
34 // the advantage of the bus compositor to minimize the
number of mask
35 // bits, and hence 8kB slaves may be aliased to many
places in memory.
36 // Bus composition and address assignment, however, are
both outside of
37 // the scope of the operation of this module.
38 //
39 //
40 // Creator:    Dan Gisselquist, Ph.D.
41 //             Gisselquist Technology, LLC
42 //
43 ///////////////////////////////////////////////////////////////////
44 // }}}
45 // Copyright (C) 2019-2025, Gisselquist Technology, LLC
46 // {{{
47 // This file is part of the WB2AXIP project.
48 //
49 // The WB2AXIP project contains free software and gateware,
licensed under the
50 // Apache License, Version 2.0 (the "License"). You may not
use this project,
51 // or this file, except in compliance with the License. You
may obtain a copy
52 // of the License at

```

```

53 // }}}
54 // http://www.apache.org/licenses/LICENSE-2.0
55 // {{{
56 // Unless required by applicable law or agreed to in writing,
software
57 // distributed under the License is distributed on an "AS IS"
BASIS, WITHOUT
58 // WARRANTIES OR CONDITIONS OF ANY KIND, either express or im-
plied. See the

59 // License for the specific language governing permissions and
limitations
60 // under the License.
61 //
62 ///////////////////////////////////////////////////////////////////
63 //
64 `default_nettype none
65 // }}}
66 module addrdecode #(
67 // {{{
68     parameter NS=8,
69     parameter AW = 32, DW=32+32/8+1+1,
70 //
71 // SLAVE_ADDR contains address assignments for
each of the
72 // various slaves we are adjudicating between.
73     parameter [NS*AW-1:0] SLAVE_ADDR = {
74         { 3'b111, {(AW-3){1'b0}} },
75         { 3'b110, {(AW-3){1'b0}} },
76         { 3'b101, {(AW-3){1'b0}} },
77         { 3'b100, {(AW-3){1'b0}} },
78         { 3'b011, {(AW-3){1'b0}} },
79         { 3'b010, {(AW-3){1'b0}} },
80         { 4'b0010, {(AW-4){1'b0}} },
81         { 4'b0000, {(AW-4){1'b0}} },
82 //
83 // SLAVE_MASK contains a mask of those address
bits in
84 // SLAVE_ADDR which are relevant. It shall be
true that if
85 // !SLAVE_MASK[k] then !SLAVE_ADDR[k], for any
bits of k
86     parameter [NS*AW-1:0] SLAVE_MASK =
87         : { {(NS-2){ 3'b111, {(AW-3){1'b0}} }},
88             {(2){ 4'b1111, {(AW-4)
{1'b0}} } } },
89 //
90 // ACCESS_ALLOWED is a bit-wise mask indicating
which slaves
91 // may get access to the bus. If AC-
CESS_ALLOWED[slave] is true,
92 // then a master can connect to the slave via
this method. This
93 // parameter is primarily here to support AXI
(or other similar
94 // buses) which may have separate accesses for
both read and
95 // write. By using this, a read-only slave can be
connected,
96 // which would also naturally create an error
on any attempt to
97 // write to it.
98     parameter [NS-1:0]ACCESS_ALLOWED = -1,
99 //
100 // If OPT_REGISTERED is set, address decoding
will take an extra
101 // clock, and will register the results of the
decoding
102 // operation.
103     parameter [0:0] OPT_REGISTERED = 0,
104 //
105 // If OPT_LOWPPOWER is set, then whenever the
output is not
106 // valid, any respective data line will also
be forced to zero
107 // in an effort to minimize power.
108     parameter [0:0] OPT_LOWPPOWER = 0
109 // }}}
110 ) (
111 // {{{
112     input wire i_clk, i_reset,
113 //
114     input wire i_valid,
115     output reg o_stall,
116     input wire [AW-1:0]i_addr,
117     input wire [DW-1:0]i_data,
118 //
119     output reg o_valid,
120     input wire i_stall,
121     output reg [NS:0] o_decode,
122     output reg [AW-1:0]o_addr,
123     output reg [DW-1:0]o_data
124 // }}}
125 );
126
127 // Local declarations
128 // {{{

```

```

129 //
130 // OPT_NONESEL controls whether or not the address
lines are fully
131 // proscribed, or whether or not a "no-slave identi-
fied" slave should
132 // be created. To avoid a "no-slave selected" output,
slave zero must

133 // have no mask bits set (and therefore no address bits
set), and it
134 // must also allow access.
135     localparam [0:0] OPT_NONESEL = (!ACCESS_ALLOWED[0])
136 //
137 (SLAVE_MASK[AW-1:0] != 0);
138 //
139     wire [NS:0] request;
140     reg [NS-1:0]prerequest;
141     integer iM;
142 // }}}
143 // prerequest
144 // {{{
145     always @(*)
146     for(iM=0; iM<NS; iM=iM+1)
147         prerequest[iM] = (((i_addr ^ SLAVE_ADDR[iM*AW
+: AW])
148             &SLAVE_MASK[iM*AW +: AW])==0)
149             &&(ACCESS_ALLOWED[iM]);
150 // }}}
151 // request
152 // {{{
153     generate if (OPT_NONESEL)
154     begin : NO_DEFAULT_REQUEST
155         // {{{
156         reg [NS-1:0]r_request;
157 //
158 // Need to create a slave to describe when
nothing is selected
159 //
160         always @(*)
161         begin
162             for(iM=0; iM<NS; iM=iM+1)
163                 r_request[iM] = i_valid && pre-
request[iM];
164             if (!OPT_NONESEL && (NS > 1 && |pre-
request[NS-1:1]))
165                 r_request[0] = 1'b0;
166         end
167         assign request[NS-1:0] = r_request;
168 // }}}
169     end else if (NS == 1)
170     begin : SINGLE_SLAVE
171         // {{{
172         assign request[0] = i_valid;
173 //
174 // Verilator lint_off UNUSED
175     wire unused;
176     assign unused = &{ 1'b0, prerequest };
177 // Verilator lint_on UNUSED
178 // }}}
179     end else begin : GENERAL_CASE
180         // {{{
181         reg [NS-1:0]r_request;
182 //
183         always @(*)
184         begin
185             for(iM=0; iM<NS; iM=iM+1)
186                 r_request[iM] = i_valid && pre-
request[iM];
187             if (!OPT_NONESEL && (NS > 1 && |pre-
request[NS-1:1]))
188                 r_request[0] = 1'b0;
189         end
190         assign request[NS-1:0] = r_request;
191 // }}}
192     end endgenerate
193 // }}}
194 // request[NS]
195 // {{{
196     generate if (OPT_NONESEL)
197     begin : GENERATE_NONESEL_SLAVE
198         reg r_request_NS, r_none_sel;
199 //
200         always @(*)
201         begin
202             // Let's assume nothing's been selec-
ted, and then check
203             // to prove ourselves wrong.
204             //
205             // Note that none_sel will be con-
sidered an error
206             // condition in the follow-on pro-
cessing. Therefore
207             // it's important to clear it if no re-
quest is pending.
208             r_none_sel = i_valid && (prerequest ==
0);

```

```

213 //
214 // request[NS] indicates a request for
a non-existent
215 // slave. A request that should (even-
tually) return a
216 // bus error
217 //
218 r_request_NS = r_none_sel;
219 end
220
221 assign request[NS] = r_request_NS;
222 end else begin : NO_NONESEL_SLAVE
223 assign request[NS] = 1'b0;
224 end endgenerate
225 // }}}
226
227 // o_valid, o_addr, o_data, o_decode, o_stall
228 // {{{
229 generate if (OPT_REGISTERED)
230 begin : GEN_REG_OUTPUTS
231
232 // o_valid
233 // {{{
234 initial o_valid = 0;
235 always @(posedge i_clk)
236 if (i_reset)
237 o_valid <= 0;
238 else if (!o_stall)
239 o_valid <= i_valid;
240 // }}}
241
242 // o_addr, o_data
243 // {{{
244 initial o_addr = 0;
245 initial o_data = 0;
246 always @(posedge i_clk)
247 if (i_reset && OPT_LOWPPOWER)
248 begin
249 o_addr <= 0;
250 o_data <= 0;
251 end else if ((!o_valid || !i_stall)
252 && (i_valid || !OPT_LOWPPOWER))
253 begin
254 o_addr <= i_addr;
255 o_data <= i_data;
256 end else if (OPT_LOWPPOWER && !i_stall)
257 begin
258 o_addr <= 0;
259 o_data <= 0;
260 end
261 // }}}
262
263 // o_decode
264 // {{{
265 initial o_decode = 0;
266 always @(posedge i_clk)
267 if (i_reset)
268 o_decode <= 0;
269 else if ((!o_valid || !i_stall)
270 && (i_valid || !OPT_LOWPPOWER))
271 o_decode <= request;
272 else if (OPT_LOWPPOWER && !i_stall)
273 o_decode <= 0;
274 // }}}
275
276 // o_stall
277 // {{{
278 always @(*)
279 o_stall = (o_valid && i_stall);
280 // }}}
281 end else begin : GEN_COMBINATORIAL_OUTPUTS
282
283 always @(*)
284 begin
285 o_valid = i_valid;
286 o_stall = i_stall;
287 o_addr = i_addr;
288 o_data = i_data;
289
290 o_decode = request;
291 end
292
293 // Make Verilator happy
294 // {{{
295 // verilator coverage_off
296 // verilator lint_off UNUSED
297 wire unused;
298 assign unused = &{ 1'b0,
299 `ifdef VERILATOR
300 // Can't declare the clock as
unused for formal,
301 // lest it not be recognized as
302 i_clk,
303 `endif
304 i_reset };
305 // verilator lint_on UNUSED
306 // verilator coverage_on
307 // }}}
308 end endgenerate

```

```

309 // }}}
310 ///////////////////////////////////////////////////
311 ///////////////////////////////////////////////////
312 ///////////////////////////////////////////////////
313 //
314 // Formal properties
315 // {{{
316 ///////////////////////////////////////////////////
317 ///////////////////////////////////////////////////
318 ///////////////////////////////////////////////////
319 `ifdef FORMAL
320 reg f_past_valid;
321 initial f_past_valid = 0;
322 always @(posedge i_clk)
323 f_past_valid <= 1;
324
325 reg [AW+DW-1:0] f_idata;
326 always @(*)
327 f_idata = { i_addr, i_data };
328
329 `ifdef ADDRDECODE
330 always @(posedge i_clk)
331 if (!f_past_valid)
332 assume(i_reset);
333 `else
334 always @(posedge i_clk)
335 if (!f_past_valid)
336 assert(i_reset);
337 `endif // ADDRDECODE
338 always @(posedge i_clk)
339 if (OPT_REGISTERED && (!f_past_valid || $past(i_re-
340 set)))
341 begin
342 assert(!o_valid);
343 assert(o_decode == 0);
344 end else if ($past(o_valid && i_stall) && OPT_RE-
345 GISTERED)
346 begin
347 assert($stable(o_addr));
348 assert($stable(o_decode));
349 assert($stable(o_data));
350 end
351 // If the output is ever valid, there must be at least
one
352 // decoded output
353 always @(*)
354 assert(o_valid == (o_decode != 0));
355
356 always @(*)
357 for(iM=0; iM<NS; iM=iM+1)
358 if (o_decode[iM])
359 begin
360 // The address must match
361 assert((((o_addr ^ SLAVE_ADDR[iM*AW +: AW])
362 & SLAVE_MASK[iM*AW +: AW])==0)
363 && ACCESS_ALLOWED[iM]);
364 //
365 // And nothing else must match
366 assert(o_decode == (1<iM));
367 end
368
369 always @(*)
370 for(iM=0; iM<NS; iM=iM+1)
371 if (!ACCESS_ALLOWED[iM])
372 assert(!o_decode[iM]);
373
374 // LOWPOWER check
375 // {{{
376 generate if (OPT_LOWPPOWER && OPT_REGISTERED)
377 begin
378 always @(*)
379 if (!o_valid)
380 begin
381 assert(o_addr == 0);
382 assert(o_decode == 0);
383 assert(o_data == 0);
384 end
385 end endgenerate
386 // }}}
387
388 //
389 // The output decoded value may only ever have one
value high,
390 // never more--i.e. $onehot0
391 // {{{
392 `ifdef VERIFIC
393 always @(*)
394 assert($onehot0(request));
395 `else
396 reg onehot_request;
397 always @(*)
398 begin
399 onehot_request = 0;

```



```

400         for(iM=0; iM<NS+1; iM=iM+1)
401             if ((request ^ (1<<iM))==0)
402                 onehot_request = 1;
403         end
404
405         always @(*)
406             if (request != 0)
407                 assert(onehot_request);
408     `endif
409     // }}}
410
411     ///////////////////////////////////////////////////////////////////
412     //
413     // Cover properties
414     // {{{
415     ///////////////////////////////////////////////////////////////////
416     //
417     //
418     //
419     // Make sure all addresses are reachable
420     //
421     reg    [NS:0]  f_reached;
422
423     always @(posedge i_clk)
424         cover(i_valid);
425
426     always @(posedge i_clk)
427         cover(o_valid);
428
429     always @(posedge i_clk)
430         cover(o_valid && !i_stall);
431
432     initial f_reached = 0;
433     always @(posedge i_clk)
434         if (i_reset)
435             f_reached = 0;
436         else if (o_valid)
437             f_reached = f_reached | o_decode;
438
439     generate if (!OPT_NONESEL && ACCESS_ALLOWED[0]
440                 && SLAVE_MASK == 0 && NS == 1)
441     begin
442
443         always @(*)
444             cover(f_reached[0]);
445
446         always @(posedge i_clk)
447             if (f_past_valid && $stable(o_valid))
448                 assert($stable(o_decode));
449
450     end else begin
451
452         always @(*)
453             cover(&f_reached);
454
455         always @(posedge i_clk)
456             if (f_past_valid && $stable(o_valid))
457                 cover($changed(o_decode));
458
459     end endgenerate
460     // }}}
461 `endif // FORMAL
462 // }}}
463 endmodule
464
465 `ifndef Y0SYS
466 `default_nettype wire
467 `endif

```

2. ZipCPU_wb2axip/rtl/afifo.v (804 lines)

```

1 ///////////////////////////////////////////////////////////////////
2 //
3 // Filename:    rtl/afifo.v
4 // {{{
5 // Project:     WB2AXIPSP: bus bridges and other odds and ends
6 //
7 // Purpose:     A basic asynchronous FIFO.
8 //
9 // Creator:     Dan Gisselquist, Ph.D.
10 //             Gisselquist Technology, LLC
11 //
12 ///////////////////////////////////////////////////////////////////
13 // }}}
14 // Copyright (C) 2019-2025, Gisselquist Technology, LLC
15 // {{{
16 // This file is part of the WB2AXIP project.
17 //
18 // The WB2AXIP project contains free software and gateware,
19 // licensed under the
20 // Apache License, Version 2.0 (the "License"). You may not
21 // use this project,
22 // or this file, except in compliance with the License. You
23 // may obtain a copy
24 // of the License at
25 //
26 // }}}

```

```

23 //      http://www.apache.org/licenses/LICENSE-2.0
24 // {{{
25 // Unless required by applicable law or agreed to in writing,
26 // software
27 // distributed under the License is distributed on an "AS IS"
28 // BASIS, WITHOUT
29 // WARRANTIES OR CONDITIONS OF ANY KIND, either express or im-
30 // plied. See the
31 //
32 // License for the specific language governing permissions and
33 // limitations
34 // under the License.
35 //
36 ///////////////////////////////////////////////////////////////////
37 //
38 //
39 // `default_nettype    none
40 // {{{
41 module afifo #(
42     // LGFIFO is the log based-two of the number of
43     entries
44     // in the FIFO, log_2(fifo size)
45     parameter    LGFIFO = 3,
46     // WIDTH is the number of data bits in each
47     entry
48     parameter    WIDTH = 16,
49     // NFF is the number of flip flops used to
50     cross clock domains.
51     // 2 is a minimum. Some applications appreci-
52     ate the better
53     parameter    NFF = 2,
54     // This core can either write on the positive
55     edge of the clock
56     // or the negative edge. Set WRITE_ON_POSEDGE
57     (the default)
58     // to write on the positive edge of the clock.
59     parameter [0:0] WRITE_ON_POSEDGE = 1'b1,
60     //
61     // Many logic elements can read from memory
62     asynchronously.
63     // This capability is sometimes known as "Dis-
64     tributed RAM" or
65     // "LUT RAM". Doing so, however, burdens the
66     timing of any
67     // logic to follow. By setting OPT_RE-
68     GISTER_READS, we force
69     // all reads to be synchronous and not burdened by
70     any logic.
71     // This tends to work with more hardware. If
72     your hardware does
73     // support "Distributed RAM", then you may be
74     able to spare a
75     // clock of latency by clearing this parameter.
76     parameter [0:0] OPT_REGISTER_READS = 1'b1
77 `ifdef FORMAL
78     // F_OPT_DATA_STB
79     // {{{
80     // In the formal proof, F_OPT_DATA_STB includes a
81     series of
82     // assumptions associated with a data strobe I/
83     O pin--things
84     // like a discontinuous clock--just to make
85     sure the core still
86     // works in those circumstances
87     , parameter [0:0]    F_OPT_DATA_STB = 1'b1
88     // }}}
89 `endif
90     ) (
91         // {{{
92         // The (incoming) write data interface
93         input    wire    i_wclk,
94         i_wr_reset_n, i_wr,
95         input    wire    [WIDTH-1:0]    i_wr_data,
96         output    reg    o_wr_full,
97         //
98         // The (incoming) write data interface
99         input    wire    i_rclk,
100         i_rd_reset_n, i_rd,
101         output    reg    [WIDTH-1:0]    o_rd_data,
102         output    reg    o_rd_empty
103         , output reg    [LGFIFO:0]    f_fill
104         // }}}
105     );
106
107     // Register/net declarations
108     // {{{
109     // MSB = most significant bit of the FIFO address vec-
110     tor. It's

```

```

95      // just short-hand for LGFIFO, and won't work any other
way.
96      localparam      MSB = LGFIFO;
97      //
98      reg      [WIDTH-1:0]      mem
99      [(1<<LGFIFO)-1:0];
100      reg      [LGFIFO:0]      rd_addr, wr_addr,
101      rd_wgray, wr_rgray;
102      wire      [LGFIFO:0]      next_rd_addr,
103      next_wr_addr;
104      reg      [LGFIFO:0]      rgray, wgray;
105      (* ASYNC_REG = "TRUE" *) reg
106      [(LGFIFO+1)*(NFF-1)-1:0]      rgray_cross,
107      wgray_cross;
108      wire      wclk;
109      reg      [WIDTH-1:0]      lcl_rd_data;
110      reg      [WIDTH-1:0]      lcl_read, lcl_rd_empty;
111      // }}}
112      // wclk - Write clock generation
113      // {{{
114      generate if (WRITE_ON_POSEDGE)
115      begin : GEN_POSEDGE_WRITES
116
117          assign wclk = i_wclk;
118
119      end else begin : GEN_NEGEDGE_WRITES
120
121          assign wclk = !i_wclk;
122
123      end endgenerate
124      // }}}
125      //
126      // Write to and read from memory
127      // {{{
128      //
129      //
130      //
131      // wr_addr, wgray
132      // {{{
133      assign next_wr_addr = wr_addr + 1;
134      always @(posedge wclk or negedge i_wr_reset_n)
135      if (!i_wr_reset_n)
136      begin
137          wr_addr <= 0;
138          wgray <= 0;
139      end else if (i_wr && !o_wr_full)
140      begin
141          wr_addr <= next_wr_addr;
142          wgray <= next_wr_addr ^ (next_wr_addr >> 1);
143      end
144      // }}}
145      // Write to memory
146      // {{{
147      always @(posedge wclk)
148      if (i_wr && !o_wr_full)
149      mem[wr_addr[LGFIFO-1:0]] <= i_wr_data;
150      // }}}
151      // rd_addr, rgray
152      // {{{
153      assign next_rd_addr = rd_addr + 1;
154      always @(posedge i_rclk or negedge i_rd_reset_n)
155      if (!i_rd_reset_n)
156      begin
157          rd_addr <= 0;
158          rgray <= 0;
159      end else if (lcl_read && !lcl_rd_empty)
160      begin
161          rd_addr <= next_rd_addr;
162          rgray <= next_rd_addr ^ (next_rd_addr >> 1);
163      end
164      // }}}
165      // Read from memory
166      // {{{
167      always @(*)
168      lcl_rd_data = mem[rd_addr[LGFIFO-1:0]];
169      // }}}
170      // }}}
171      //
172      // Cross clock domains
173      // {{{
174      //
175      //
176      //
177      // read pointer -> wr_rgray
178      // {{{
179      always @(posedge wclk or negedge i_wr_reset_n)
180      if (!i_wr_reset_n)

```

```

187      { wr_rgray, rgray_cross } <= 0;
188      else
189      { wr_rgray, rgray_cross } <= { rgray_cross,
rgray };
190      // }}}
191      // write pointer -> rd_wgray
192      // {{{
193      always @(posedge i_rclk or negedge i_rd_reset_n)
194      if (!i_rd_reset_n)
195      { rd_wgray, wgray_cross } <= 0;
196      else
197      { rd_wgray, wgray_cross } <= { wgray_cross,
wgray };
198      // }}}
199      // }}}
200      // }}}
201      // }}}
202      //
203      //
204      // Flag generation
205      // {{{
206      //
207      //
208      //
209      //
210      always @(*)
211      o_wr_full = (wr_rgray == { ~wgray[MSB:MSB-1],
wgray[MSB-2:0] });
212      //
213      always @(*)
214      lcl_rd_empty = (rd_wgray == rgray);
215      //
216      // o_rd_empty, o_rd_data
217      // {{{
218      generate if (OPT_REGISTER_READS)
219      begin : GEN_REGISTERED_READ
220      // {{{
221      always @(*)
222      lcl_read = (o_rd_empty || i_rd);
223      //
224      always @(posedge i_rclk or negedge i_rd_re-
set_n)
225      if (!i_rd_reset_n)
226      o_rd_empty <= 1'b1;
227      else if (lcl_read)
228      o_rd_empty <= lcl_rd_empty;
229      //
230      always @(posedge i_rclk)
231      if (lcl_read)
232      o_rd_data <= lcl_rd_data;
233      // }}}
234      end else begin : GEN_COMBINATORIAL_FLAGS
235      // {{{
236      always @(*)
237      lcl_read = i_rd;
238      //
239      always @(*)
240      o_rd_empty = lcl_rd_empty;
241      //
242      always @(*)
243      o_rd_data = lcl_rd_data;
244      // }}}
245      end endgenerate
246      // }}}
247      // }}}
248      //
249      //
250      //
251      //
252      // Formal properties
253      // {{{
254      //
255      //
256      //
257      `ifdef FORMAL
258      // Start out with some register/net/macro declarations,
f_past_valid,etc
259      // {{{
260      `ifndef AFIFO
261      `define ASSERT assert
262      `define ASSUME assume
263      `else
264      `define ASSERT assert
265      `define ASSUME assert
266      `endif
267      //
268      (* gclk *)      reg      gbl_clk;
269      reg      f_past_valid_gbl,
f_past_valid_rd,
270      f_rd_in_reset, f_wr_in_reset;
271      reg      [WIDTH-1:0]      past_rd_data, past_wr_data;
272      reg      past_wr_reset_n, past_rd_re-

```

```

set_n,
273                                past_rd_empty, past_wclk,
past_rclk, past_rd;
274     reg [(LGFIFO+1)*(NFF-1)-1:0] f_wcross, f_rcross;
275     reg [LGFIFO:0] f_rd_waddr, f_wr_raddr;
276     reg [LGFIFO:0] f_rdcross_fill [NFF-1:0];
277     reg [LGFIFO:0] f_wrcross_fill [NFF-1:0];
278
279
280     initial f_past_valid_gbl = 1'b0;
281     always @(posedge gbl_clk)
282         f_past_valid_gbl <= 1'b1;
283
284     initial f_past_valid_rd = 1'b0;
285     always @(posedge i_rclk)
286         f_past_valid_rd <= 1'b1;
287     // }}}
288     ///////////////////////////////////////////////////////////////////
289     //
290     // Reset checks
291     // {{{
292     ///////////////////////////////////////////////////////////////////
293     //
294     //
295     initial f_wr_in_reset = 1'b1;
296     always @(posedge wclk or negedge i_wr_reset_n)
297         if (!i_wr_reset_n)
298             f_wr_in_reset <= 1'b1;
299     else
300         f_wr_in_reset <= 1'b0;
301
302     initial f_rd_in_reset = 1'b1;
303     always @(posedge i_rclk or negedge i_rd_reset_n)
304         if (!i_rd_reset_n)
305             f_rd_in_reset <= 1'b1;
306     else
307         f_rd_in_reset <= 1'b0;
308
309     //
310     // Resets are ...
311     // 1. Asserted always initially, and ...
312     always @(*)
313         if (!f_past_valid_gbl)
314             begin
315                 `ASSUME(!i_wr_reset_n);
316                 `ASSUME(!i_rd_reset_n);
317             end
318
319     // 2. They only ever become active together
320     always @(*)
321         if (past_wr_reset_n && !i_wr_reset_n)
322             `ASSUME(!i_rd_reset_n);
323
324     always @(*)
325         if (past_rd_reset_n && !i_rd_reset_n)
326             `ASSUME(!i_wr_reset_n);
327
328     // }}}
329     ///////////////////////////////////////////////////////////////////
330     //
331     // Synchronous signal assumptions
332     // {{{
333     ///////////////////////////////////////////////////////////////////
334     //
335     //
336
337     always @(posedge gbl_clk)
338         begin
339             past_wr_reset_n <= i_wr_reset_n;
340             past_rd_reset_n <= i_rd_reset_n;
341
342             past_wclk <= wclk;
343             past_rclk <= i_rclk;
344
345             past_rd <= i_rd;
346             past_rd_data <= lcl_rd_data;
347             past_wr_data <= i_wr_data;
348
349             past_rd_empty <= lcl_rd_empty;
350         end
351
352     //
353     // Read side may be assumed to be synchronous
354     always @(*)
355         if (f_past_valid_gbl && i_rd_reset_n && (past_rclk || !
i_rclk))
356             // i.e. if (!$rose(i_rclk))
357             `ASSUME(i_rd == past_rd);
358
359     always @(*)
360         if (f_past_valid_rd && !f_rd_in_reset && !lcl_rd_empty
&& (past_rclk || !i_rclk))
361             begin
362                 `ASSERT(lcl_rd_data == past_rd_data);
363                 `ASSERT(lcl_rd_empty == past_rd_empty);
364             end
365
366

```

```

367
368     generate if (F_OPT_DATA_STB)
369         begin
370
371             always @(posedge gbl_clk)
372                 `ASSUME(!o_wr_full);
373
374             always @(posedge gbl_clk)
375                 if (!i_wr_reset_n)
376                     `ASSUME(!i_wclk);
377
378             always @(posedge gbl_clk)
379                 `ASSUME(i_wr == i_wr_reset_n);
380
381             always @(posedge gbl_clk)
382                 if ($changed(i_wr_reset_n))
383                     `ASSUME($stable(wclk));
384
385         end endgenerate
386     // }}}
387     ///////////////////////////////////////////////////////////////////
388     //
389     // Fill checks
390     // {{{
391     ///////////////////////////////////////////////////////////////////
392     //
393     //
394     always @(*)
395         f_fill = wr_addr - rd_addr;
396
397     always @(*)
398         if (!f_wr_in_reset)
399             `ASSERT(f_fill <= { 1'b1, {(MSB){1'b0}} });
400
401     always @(*)
402         if (wr_addr == rd_addr)
403             `ASSERT(lcl_rd_empty);
404
405     always @(*)
406         if (!(f_wr_in_reset && !f_rd_in_reset)
&& wr_addr == { ~rd_addr[MSB], rd_ad-
dr[MSB-1:0] })
407             `ASSERT(o_wr_full);
408
409     // }}}
410     ///////////////////////////////////////////////////////////////////
411     //
412     //
413     // Induction checks
414     // {{{
415     ///////////////////////////////////////////////////////////////////
416     //
417     //
418
419     // f_wr_in_reset -- write logic is in its reset state
420     // {{{
421     always @(*)
422         if (f_wr_in_reset)
423             begin
424                 `ASSERT(wr_addr == 0);
425                 `ASSERT(wgray_cross == 0);
426
427                 `ASSERT(rd_addr == 0);
428                 `ASSERT(rgray_cross == 0);
429                 `ASSERT(rd_wgray == 0);
430
431                 `ASSERT(lcl_rd_empty);
432                 `ASSERT(!o_wr_full);
433             end
434         // }}}
435
436     // f_rd_in_reset -- read logic is in its reset state
437     // {{{
438     always @(*)
439         if (f_rd_in_reset)
440             begin
441                 `ASSERT(rd_addr == 0);
442                 `ASSERT(rgray_cross == 0);
443                 `ASSERT(rd_wgray == 0);
444
445                 `ASSERT(lcl_rd_empty);
446             end
447         // }}}
448
449     // f_wr_raddr -- a read address to match the gray val-
ues
450     // {{{
451     always @(posedge wclk or negedge i_wr_reset_n)
452         if (!i_wr_reset_n)
453             { f_wr_raddr, f_rcross } <= 0;
454         else
455             { f_wr_raddr, f_rcross } <= { f_rcross, rd_ad-
dr };
456     // }}}
457
458     // f_rd_waddr -- a write address to match the gray val-
ues
459     // {{{

```

```

460     always @(posedge i_rclk or negedge i_rd_reset_n)
461     if (!i_rd_reset_n)
462     { f_rd_waddr, f_wcross } <= 0;
463     else
464     { f_rd_waddr, f_wcross } <= { f_wcross, wr_ad-
dr };
465     // }}}
466
467     integer k;
468
469     // wgray check
470     // {{{
471     always @(*)
472     `ASSERT((wr_addr ^ (wr_addr >> 1)) == wgray);
473     // }}}
474
475     // wgray_cross check
476     // {{{
477     always @(*)
478     for(k=0; k<NFF-1; k=k+1)
479     `ASSERT((f_wcross[k*(LGFIFO+1) +: LGFIFO+1]
480     ^ (f_wcross[k*(LGFIFO+1) +:
LGFIFO+1]>>1))
481     == wgray_cross[k*(LGFIFO+1) +:
LGFIFO+1]);
482     // }}}
483
484     // rgray check
485     // {{{
486     always @(*)
487     `ASSERT((rd_addr ^ (rd_addr >> 1)) == rgray);
488     // }}}
489
490     // rgray_cross check
491     // {{{
492     always @(*)
493     for(k=0; k<NFF-1; k=k+1)
494     `ASSERT((f_rcross[k*(LGFIFO+1) +: LGFIFO+1]
495     ^ (f_rcross[k*(LGFIFO+1) +:
LGFIFO+1]>>1))
496     == rgray_cross[k*(LGFIFO+1) +:
LGFIFO+1]);
497     // }}}
498
499     // wr_rgray
500     // {{{
501     always @(*)
502     `ASSERT((f_wr_raddr ^ (f_wr_raddr >> 1)) ==
wr_rgray);
503     // }}}
504
505     // rd_wgray
506     // {{{
507     always @(*)
508     `ASSERT((f_rd_waddr ^ (f_rd_waddr >> 1)) ==
rd_wgray);
509     // }}}
510
511     // f_rdcross_fill
512     // {{{
513     always @(*)
514     for(k=0; k<NFF-1; k=k+1)
515     f_rdcross_fill[k] = f_wcross[k*(LGFIFO+1) +:
LGFIFO+1]
516     - rd_addr;
517
518     always @(*)
519     f_rdcross_fill[NFF-1] = f_rd_waddr - rd_addr;
520
521     always @(*)
522     for(k=0; k<NFF; k=k+1)
523     `ASSERT(f_rdcross_fill[k] <= { 1'b1, {(LGFIFO)
{1'b0}} });
524
525     always @(*)
526     for(k=1; k<NFF; k=k+1)
527     `ASSERT(f_rdcross_fill[k] <= f_rd-
cross_fill[k-1]);
528     always @(*)
529     `ASSERT(f_rdcross_fill[0] <= f_fill);
530     // }}}
531
532     // f_wrcross_fill
533     // {{{
534     always @(*)
535     for(k=0; k<NFF-1; k=k+1)
536     f_wrcross_fill[k] = wr_addr -
f_rcross[k*(LGFIFO+1) +: LGFIFO+1];
537
538     always @(*)
539     f_wrcross_fill[NFF-1] = wr_addr - f_wr_raddr;
540
541     always @(*)
542     for(k=0; k<NFF; k=k+1)
543     `ASSERT(f_wrcross_fill[k] <= { 1'b1, {(LGFIFO)
{1'b0}} });
544
545     always @(*)
546     for(k=1; k<NFF; k=k+1)
547     `ASSERT(f_wrcross_fill[k] >= f_wr-
cross_fill[k-1]);

```

```

549     always @(*)
550     `ASSERT(f_wrcross_fill[0] >= f_fill);
551     // }}}
552
553     // }}}
554     ///////////////////////////////////////////////////
555     //
556     // Clock generation
557     // {{{
558     ///////////////////////////////////////////////////
559     //
560     //
561     // Here's the challenge: if we use $past with any of
our clocks, such
562     // as to determine stability or any such, the proof
takes forever, and
563
564     // we need to guarantee a minimum number of transitions
within the
565     // depth of the proof. If, on the other hand, we build
our own $past
566     // primitives--we can then finish much faster and be
successful on
567     // any depth of proof.
568
569     // pre_xclk is what the clock will become on the next
global clock
edge.
570     // By using it here, we can check things @(*) instead
of
571     // @(posedge gbl_clk). Further, we can check
$rose(pre_xclk) (or $fell)
572     // and essentially check things @(*) but while using
@(global_clk).
573     // In other words, we can transition on @(posedge
gbl_clk), but stay
574     // in sync with the data--rather than being behind by a
clock.
575     // now_xclk is what the clock is currently.
576     //
577     (* anyseq *) reg pre_wclk, pre_rclk;
578     reg now_wclk, now_rclk;
579     always @(posedge gbl_clk)
580     begin
581         now_wclk <= pre_wclk;
582         now_rclk <= pre_rclk;
583     end
584
585     always @(*)
586     begin
587         assume(i_wclk == now_wclk);
588         assume(i_rclk == now_rclk);
589     end
590
591     always @(posedge gbl_clk)
592     assume(i_rclk == $past(pre_rclk));
593
594     // Assume both clocks start idle
595     // {{{
596     always @(*)
597     if (!f_past_valid_gbl)
598     begin
599         assume(!pre_wclk && !wclk);
600         assume(!pre_rclk && !i_rclk);
601     end
602     // }}}
603
604     // }}}
605     ///////////////////////////////////////////////////
606     //
607     // Formal contract check --- the twin write test
608     // {{{
609     ///////////////////////////////////////////////////
610     //
611     //
612
613     // Tracking register declarations
614     // {{{
615     reg [WIDTH-1:0] f_first,
f_next;
616     (* anyconst *) reg [LGFIFO:0] f_addr;
617     reg [LGFIFO:0] f_next_addr;
618     reg f_first_in_fifo,
f_next_in_fifo;
619     reg [LGFIFO:0] f_to_first, f_to_next;
620     reg [1:0] f_state;
621
622     always @(*)
623     f_next_addr = f_addr + 1;
624     // }}}
625
626     // distance_to*, *_in_fifo
627     // {{{
628     always @(*)
629     begin

```

```

630      f_to_first = f_addr - rd_addr;
631
632      f_first_in_fifo = 1'b1;
633      if ((f_to_first >= f_fill) || (f_fill == 0))
634          f_first_in_fifo = 1'b0;
635
636      if (mem[f_addr] != f_first)
637          f_first_in_fifo = 1'b0;
638
639      //
640      // Check the second item
641      //
642
643      f_to_next = f_next_addr - rd_addr;
644      f_next_in_fifo = 1'b1;
645      if ((f_to_next >= f_fill) || (f_fill == 0))
646          f_next_in_fifo = 1'b0;
647
648      if (mem[f_next_addr] != f_next)
649          f_next_in_fifo = 1'b0;
650
651      end
652      // }}}
653      // f_state -- generate our state variable
654      // {{{
655      initial f_state = 0;
656      always @(posedge gbl_clk)
657      if (!i_wr_reset_n)
658          f_state <= 0;
659      else case(f_state)
660      2'b00: if (($rose(pre_wclk)) && i_wr && !o_wr_full
&&(wr_addr == f_addr))
661          begin
662              f_state <= 2'b01;
663              f_first <= i_wr_data;
664          end
665      2'b01: if ($rose(pre_rclk) && lcl_read && rd_addr ==
f_addr)
666          f_state <= 2'b00;
667      else if ($rose(pre_wclk) && i_wr && !
o_wr_full )
668          begin
669              f_state <= 2'b10;
670              f_next <= i_wr_data;
671          end
672
673      2'b10: if ($rose(pre_rclk) && lcl_read && !lcl_rd_empty
&& rd_addr == f_addr)
674          f_state <= 2'b11;
675
676      2'b11: if ($rose(pre_rclk) && lcl_read && !lcl_rd_empty
&& rd_addr == f_next_addr)
677          f_state <= 2'b00;
678      endcase
679      // }}}
680      // f_state invariants
681      // {{{
682      always @(*)
683      if (i_wr_reset_n) case(f_state)
684      2'b00: begin end
685      2'b01: begin
686          `ASSERT(f_first_in_fifo);
687          `ASSERT(wr_addr == f_next_addr);
688          `ASSERT(f_fill >= 1);
689      end
690      2'b10: begin
691          `ASSERT(f_first_in_fifo);
692          `ASSERT(f_next_in_fifo);
693          if (!lcl_rd_empty && (rd_addr == f_addr))
694              `ASSERT(lcl_rd_data == f_first);
695          `ASSERT(f_fill >= 2);
696      end
697      2'b11: begin
698          `ASSERT(rd_addr == f_next_addr);
699          `ASSERT(f_next_in_fifo);
700          `ASSERT(f_fill >= 1);
701          if (!lcl_rd_empty)
702              `ASSERT(lcl_rd_data == f_next);
703          end
704      endcase
705      // }}}
706      generate if (OPT_REGISTER_READS)
707      begin
708          reg      past_o_rd_empty;
709
710          always @(posedge gbl_clk)
711              past_o_rd_empty <= o_rd_empty;
712
713          always @(posedge gbl_clk)
714          if (f_past_valid_gbl && i_rd_reset_n)
715          begin
716              if ($past(!o_rd_empty && !i_rd &&
i_rd_reset_n))
717                  `ASSERT($stable(o_rd_data));
718          end
719
720          always @(posedge gbl_clk)
721          if (!f_rd_in_reset && i_rd_reset_n && i_rclk
&& !past_rclk)

```

```

722      begin
723          if (past_o_rd_empty)
724              `ASSERT(o_rd_data ==
past_rd_data);
725          if (past_rd)
726              `ASSERT(o_rd_data ==
past_rd_data);
727      end
728      end endgenerate
729      // }}}
730      // Cover checks
731      // {{{
732      // Cover checks
733      // {{{
734      // Cover checks
735      // {{{
736      //
737      //
738      // Prove that we can read and write the FIFO
739      // {{{
740      always @(*)
741      if (i_wr_reset_n && i_rd_reset_n)
742      begin
743          cover(o_rd_empty);
744          cover(!o_rd_empty);
745          cover(f_state == 2'b01);
746          cover(f_state == 2'b10);
747          cover(f_state == 2'b11);
748          cover(f_fill[MSB-1:0]);
749          cover(i_rd);
750          cover(i_rd && !o_rd_empty);
751      end
752      // }}}
753      `ifdef AFIFO
754      generate if (!F_OPT_DATA_STB)
755      begin : COVER_FULL
756          // {{{
757          reg      cvr_full;
758
759          initial cvr_full = 1'b0;
760          always @(posedge gbl_clk)
761          if (!i_wr_reset_n)
762              cvr_full <= 1'b0;
763          else if (o_wr_full)
764              cvr_full <= 1'b1;
765
766          always @(*)
767          if (f_past_valid_gbl && i_wr_reset_n)
768          begin
769              cover(o_wr_full);
770              cover(o_rd_empty && cvr_full);
771              cover(o_rd_empty && f_fill == 0 &&
cvr_full);
772          end
773          // }}}
774      end else begin : COVER_NEARLY_FULL
775          // {{{
776          reg      cvr_nearly_full;
777
778          initial cvr_nearly_full = 1'b0;
779          always @(posedge gbl_clk)
780          if (!i_wr_reset_n)
781              cvr_nearly_full <= 1'b0;
782          else if (f_fill == {1'b0, {(LGFIFO){1'b1}}})
783              cvr_nearly_full <= 1'b1;
784
785          always @(*)
786          if (f_past_valid_gbl && i_wr_reset_n)
787          begin
788              cover(f_fill == {1'b0, {(LGFIFO){1'b1}}});
789              cover(cvr_nearly_full && i_wr_reset_n);
790              cover(o_rd_empty && cvr_nearly_full);
791              cover(o_rd_empty && f_fill == 0 &&
cvr_nearly_full);
792          end
793          // }}}
794      end endgenerate
795      `endif
796      // }}}
797      // }}}
798      // }}}
799      endmodule

```

3. ZipCPU_wb2axip/rtl/apbslave.v (228 lines)

```

1 ///////////////////////////////////////////////////////////////////
2 //
3 // Filename:   rtl/apbslave.v
4 // {{{
5 // Project:   WB2AXIPSP: bus bridges and other odds and ends
6 //

```

```

7 // Purpose:      Just a simple demonstration APB slave
8 //
9 // Creator:      Dan Gisselquist, Ph.D.
10 //              Gisselquist Technology, LLC
11 //
12 ///////////////////////////////////////////////////////////////////
13 // }}}
14 // Copyright (C) 2019-2025, Gisselquist Technology, LLC
15 // {{{
16 // This file is part of the WB2AXIP project.
17 //
18 // The WB2AXIP project contains free software and gateware,
19 // licensed under the
20 // Apache License, Version 2.0 (the "License"). You may not
21 // use this project,
22 // or this file, except in compliance with the License. You
23 // may obtain a copy
24 // of the License at
25 // http://www.apache.org/licenses/LICENSE-2.0
26 // {{{
27 // Unless required by applicable law or agreed to in writing,
28 // software
29 // distributed under the License is distributed on an "AS IS"
30 // BASIS, WITHOUT
31 // WARRANTIES OR CONDITIONS OF ANY KIND, either express or im-
32 // plied. See the
33 //
34 // License for the specific language governing permissions and
35 // limitations
36 // under the License.
37 //
38 ///////////////////////////////////////////////////////////////////
39 //
40 // default_nettype      none
41 // }}}
42 module apbslave #(
43     // {{{
44     parameter          C_APB_ADDR_WIDTH = 12,
45     parameter          C_APB_DATA_WIDTH = 32,
46     localparam         AW = C_APB_ADDR_WIDTH,
47     localparam         DW = C_APB_DATA_WIDTH,
48     localparam         APBLSB = $clog2(C_APB_DATA_WIDTH)-3
49 ) (
50     // {{{
51     input  wire         PCLK, PRESETn,
52     input  wire         PSEL,
53     input  wire         PENABLE,
54     output reg          PREADY,
55     input  wire         [AW-1:0]PADDR,
56     input  wire         PWRITE,
57     input  wire         [DW-1:0]PDATA,
58     input  wire         [DW/8-1:0]PWSTRB,
59     input  wire         [2:0]PPROT,
60     output reg          [DW-1:0]PRDATA,
61     output wire         PSLVERR
62     // }}}
63 );
64
65 // Register declarations
66 // {{{
67 // Just our demonstration "memory" here
68 reg [DW-1:0]mem [0:(1<<(AW-APBLSB))-1];
69 integer      ik;
70 // }}}
71
72 // PREADY
73 // {{{
74 initial PREADY = 1'b0;
75 always @(posedge PCLK)
76 if (!PRESETn)
77     PREADY <= 1'b0;
78 else if (PSEL && !PENABLE)
79     PREADY <= 1'b1;
80 else
81     PREADY <= 1'b0;
82 // }}}
83
84 // mem writes
85 // {{{
86 always @(posedge PCLK)
87 if (PRESETn && PSEL && !PENABLE && PWRITE)
88 begin
89     for(ik=0; ik<DW/8; ik=ik+1)
90     if (PWSTRB[ik])
91         mem[PADDR[AW-1:APBLSB]][8*ik +: 8] <=
92 PDATA[8*ik +: 8];
93     end
94 // }}}
95
96 // PRDATA, memory reads
97 // {{{
98 always @(posedge PCLK)
99 if (PSEL && !PENABLE && !PWRITE)
100 PRDATA <= mem[PADDR[AW-1:APBLSB]];
101 // }}}

```

```

96 // PSLVERR -- unused in this design, and so kept at
97 zero
98 // {{{
99 assign PSLVERR = 1'b0;
100 // }}}
101 // Make Verilator happy
102 // {{{
103 // Verilator lint_off UNUSED
104 wire unused;
105 assign unused = &{ 1'b0, PADDR[APBLSB-1:0], PPROT };
106 // Verilator lint_on UNUSED
107 // }}}
108 ///////////////////////////////////////////////////////////////////
109 ///////////////////////////////////////////////////////////////////
110 ///////////////////////////////////////////////////////////////////
111 //
112 // Formal properties
113 // {{{
114 ///////////////////////////////////////////////////////////////////
115 ///////////////////////////////////////////////////////////////////
116 ///////////////////////////////////////////////////////////////////
117 `ifndef FORMAL
118 ///////////////////////////////////////////////////////////////////
119 //
120 //
121 // Interface properties
122 // {{{
123 ///////////////////////////////////////////////////////////////////
124 //
125 //
126 fapb_slave #(.AW(C_APB_AD-
127 DR_WIDTH), .DW(C_APB_DATA_WIDTH),
128 .F_OPT_MAXSTALL(1)
129 ) fapb (PCLK, PRESETn,
130 PSEL, PENABLE, PREADY, PADDR, PWRITE, PDATA,
131 PWSTRB,
132 PPROT, PRDATA, PSLVERR);
133 //
134 always @(*)
135 if (PRESETn && PSEL && PENABLE)
136     assert(PREADY);
137 // }}}
138 ///////////////////////////////////////////////////////////////////
139 //
140 // Contract check
141 // {{{
142 ///////////////////////////////////////////////////////////////////
143 //
144 (* anyconst *) reg [AW-1:0] f_addr;
145 reg [DW-1:0] f_data;
146 //
147 `ifndef FORMAL
148 initial for(ik=0; ik<(1<<AW); ik=ik+1)
149     mem[ik] = 0;
150 `endif
151 //
152 always @(*)
153 if (!PRESETn)
154     assume(mem[f_addr[AW-1:APBLSB]] == f_data);
155 //
156 initial f_data = 0;
157 always @(posedge PCLK)
158 if (PRESETn && PSEL && !PENABLE && PWRITE && PAD-
159 DR[AW-1:APBLSB] == f_addr[AW-1:APBLSB])
160 begin
161     for(ik=0; ik<DW/8; ik=ik+1)
162     if (PWSTRB[ik])
163         f_data[ik*8 +: 8] <= PDATA[ik*8 +: 8];
164     end
165 //
166 always @(posedge PCLK)
167 if (PSEL && PENABLE && PREADY && !PWRITE && PAD-
168 DR[AW-1:APBLSB] == f_addr[AW-1:APBLSB])
169     assert(PRDATA == f_data);
170 //
171 always @(*)
172     assert(f_data == mem[f_addr[AW-1:APBLSB]]);
173 // }}}
174 // {{{
175 ///////////////////////////////////////////////////////////////////
176 //
177 //
178 reg [2:0] cvr_reads, cvr_writes, cvr_seq;

```



```

180
181     initial cvr_writes = 0;
182     always @(posedge PCLK)
183     if (!PRESETn)
184         cvr_writes <= 0;
185     else if (PSEL && PENABLE && PREADY && PWRITE && !
cvr_writes[2])
186         cvr_writes <= cvr_writes + 1;
187
188     initial cvr_reads = 0;
189     always @(posedge PCLK)
190     if (!PRESETn)
191         cvr_reads <= 0;
192     else if (PSEL && PENABLE && PREADY && !PWRITE && !
cvr_reads[2])
193         cvr_reads <= cvr_reads + 1;
194
195     always @(*)
196         cover(cvr_writes[2]);
197
198     always @(*)
199         cover(cvr_reads[2]);
200
201     initial cvr_seq = 0;
202     always @(posedge PCLK)
203     if (!PRESETn)
204         cvr_seq <= 0;
205     else if (PSEL && PENABLE && PREADY && !PWRITE && PADDR
== f_addr)
206     begin
207         if (cvr_seq == 0 && PRDATA == 32'h12345678)
208             cvr_seq[0] <= 1'b1;
209         if (cvr_seq[0] && PRDATA == 32'h87654321)
210             cvr_seq[1] <= 1'b1;
211         if (cvr_seq[1] && PRDATA == 32'h0)
212             cvr_seq[2] <= 1'b1;
213     end
214
215     always @(*)
216     if (PRESETn)
217     begin
218         cover(cvr_seq[0]);
219         cover(cvr_seq[1]);
220         cover(cvr_seq[2]);
221     end
222
223     always @(*)
224         cover(PRESETn && !PSEL && !PENABLE &&
cvr_seq[2]);
225 // }}}
226 `endif
227 // }}}
228 endmodule

```

4. ZipCPU_wb2axip/rtl/apbxclock.v (609 lines)

```

1 ///////////////////////////////////////////////////////////////////
2 //
3 // Filename:    rtl/apbxclock.v
4 // {{{
5 // Project:    WB2AXIPSP: bus bridges and other odds and ends
6 //
7 // Purpose:
8 //
9 // Creator:    Dan Gisselquist, Ph.D.
10 //             Gisselquist Technology, LLC
11 //
12 ///////////////////////////////////////////////////////////////////
13 // }}}
14 // Copyright (C) 2019-2025, Gisselquist Technology, LLC
15 // {{{
16 // This file is part of the WB2AXIP project.
17 //
18 // The WB2AXIP project contains free software and gateware,
19 // licensed under the
20 // Apache License, Version 2.0 (the "License"). You may not
21 // use this project,
22 // or this file, except in compliance with the License. You
23 // may obtain a copy
24 // of the License at
25 // http://www.apache.org/licenses/LICENSE-2.0
26 // {{{
27 // Unless required by applicable law or agreed to in writing,
28 // software
29 // distributed under the License is distributed on an "AS IS"
30 // BASIS, WITHOUT
31 // WARRANTIES OR CONDITIONS OF ANY KIND, either express or im-
32 // plied. See the
33 //
34 // License for the specific language governing permissions and
35 // limitations
36 // under the License.
37 //
38 ///////////////////////////////////////////////////////////////////
39 //
40 // default_nettype    none

```

```

34 // }}}
35 module apbxclock #(
36     // {{{
37     parameter      C_APB_ADDR_WIDTH = 12,
38     parameter      C_APB_DATA_WIDTH = 32,
39     parameter [0:0] OPT_REGISTERED = 1'b0,
40     localparam     AW = C_APB_ADDR_WIDTH,
41     localparam     DW = C_APB_DATA_WIDTH
42     // }}}
43 ) (
44     // {{{
45     input  wire      S_APB_PCLK,
46     input  wire      S_APB_PSEL,
47     input  wire      S_APB_PENABLE,
48     output reg       S_APB_PREADY,
49     input  wire [AW-1:0] S_APB_PADDR,
50     input  wire      S_APB_PWRITE,
51     input  wire [DW-1:0] S_APB_PWDATA,
52     input  wire [DW/8-1:0] S_APB_PWSTRB,
53     input  wire [2:0] S_APB_PPROT,
54     output wire [DW-1:0] S_APB_PRDATA,
55     output wire      S_APB_PSLVERR,
56     //
57     input  wire      M_APB_PCLK,
58     output reg       M_PRESETn,
59     output reg       M_APB_PSEL,
60     output reg       M_APB_PENABLE,
61     input  wire      M_APB_PREADY,
62     output wire [AW-1:0] M_APB_PADDR,
63     output wire      M_APB_PWRITE,
64     output wire [DW-1:0] M_APB_PWDATA,
65     output wire [DW/8-1:0] M_APB_PWSTRB,
66     output wire [2:0] M_APB_PPROT,
67     input  wire [DW-1:0] M_APB_PRDATA,
68     input  wire      M_APB_PSLVERR
69     // }}}
70 );
71
72 // Local declarations
73 // {{{
74 reg      reset_pipe, full_reset_pipe,
full_reset;
75
76 reg      s_request, s_tfr_request;
77 reg      m_request, m_request_pipe;
78
79 reg      m_ack;
80 reg      s_ack, s_ack_pipe;
81 reg      [DW-1:0] m_prdata;
82 reg      m_pslverr;
83
84 // }}}
85 ///////////////////////////////////////////////////////////////////
86 //
87 // Synchronize resets
88 // {{{
89 always @(posedge M_APB_PCLK or negedge S_PRESETn)
90 if (!S_PRESETn)
91     { M_PRESETn, reset_pipe } <= 0;
92 else
93     { M_PRESETn, reset_pipe } <= { reset_pipe,
1'b1 };
94
95 always @(posedge S_APB_PCLK or negedge M_PRESETn)
96 if (!M_PRESETn)
97     { full_reset, full_reset_pipe } <= 0;
98 else
99     { full_reset, full_reset_pipe } <= { full_re-
set_pipe, 1'b1 };
100 // }}}
101 ///////////////////////////////////////////////////////////////////
102 //
103 // Move the request across clock domains
104 // {{{
105 // s_request
106 // {{{
107 // Step 1: Register the request
108 // Always register anything before moving cross
109 clock domains
110 always @(posedge S_APB_PCLK or negedge S_PRESETn)
111 if (!S_PRESETn)
112     s_request <= 0;
113 else if (S_APB_PREADY)
114     s_request <= 0;
115 else if (S_APB_PSEL && !S_APB_PENABLE)
116     s_request <= 1;
117 // }}}
118
119 // s_tfr_request
120 // {{{
121 // Step 2: Forward requests--but only when the down-
stream handshake
122 // is ready to accept them
123 always @(posedge S_APB_PCLK or negedge S_PRESETn)
124 if (!S_PRESETn)
125     s_tfr_request <= 0;
126 else if (S_APB_PREADY)

```



```

127     s_tfr_request <= 0;
128     else if ((S_APB_PSEL && !S_APB_PENABLE) || s_request)
129         s_tfr_request <= (full_reset && !s_ack);
130     // }}}
131
132     // m_request, m_request_pipe -- 3. Capture the request
in new clk domain
133     // {{{
134     always @(posedge M_APB_PCLK or negedge M_PRESETn)
135     if (!M_PRESETn)
136         { m_request, m_request_pipe } <= 0;
137     else
138         { m_request, m_request_pipe }
139         <= { m_request_pipe, s_tfr_re-
quest };
140     // }}}
141
142     // M_APB_PSEL, M_APB_PENABLE -- 4. Forward the request
downstream
143     // {{{
144     always @(posedge M_APB_PCLK or negedge M_PRESETn)
145     if (!M_PRESETn)
146     begin
147         M_APB_PSEL <= 0;
148         M_APB_PENABLE <= 1'b0;
149     end else begin
150         M_APB_PSEL <= m_request && !m_ack
151         && (!M_APB_PSEL || !M_APB_PEN-
ABLE || !M_APB_READY);
152         M_APB_PENABLE <= M_APB_PSEL && (!M_APB_PSEL
|| !M_APB_PENABLE || !M_APB_READY);
153     end
154     // }}}
155     // }}}
156     ///////////////////////////////////////////////////////////////////
157     //
158     // Move the response back across clock domains
159     // {{{
160
161     // m_ack
162     // {{{
163     always @(posedge M_APB_PCLK or negedge M_PRESETn)
164     if (!M_PRESETn)
165         m_ack <= 0;
166     else if (m_request && !m_ack && M_APB_PSEL &&
M_APB_READY && M_APB_PENABLE)
167         m_ack <= 1'b1;
168     else if (!m_request)
169         m_ack <= 1'b0;
170     // }}}
171
172     // s_ack, s_ack_pipe
173     // {{{
174     always @(posedge S_APB_PCLK or negedge S_PRESETn)
175     if (!S_PRESETn)
176         { s_ack, s_ack_pipe } <= 0;
177     else
178         { s_ack, s_ack_pipe } <= { s_ack_pipe, m_ack };
179     // }}}
180
181     // S_APB_READY
182     // {{{
183     always @(posedge S_APB_PCLK or negedge S_PRESETn)
184     if (!S_PRESETn)
185         S_APB_READY <= 0;
186     else
187         S_APB_READY <= !S_APB_READY && s_tfr_request
&& s_ack;
188     // }}}
189
190     // m_prdata
191     // {{{
192     always @(posedge M_APB_PCLK)
193     if (M_APB_PSEL && M_APB_READY)
194         m_prdata <= M_APB_PRDATA;
195     // }}}
196
197     // m_pslverr
198     // {{{
199     always @(posedge M_APB_PCLK or negedge M_PRESETn)
200     if (!M_PRESETn)
201         m_pslverr <= 0;
202     else if (M_APB_PSEL && M_APB_READY)
203         m_pslverr <= M_APB_PSLVERR;
204     // }}}
205     // }}}
206     ///////////////////////////////////////////////////////////////////
207     //
208     // Handle the ancillary (data bearing) signals
209     // {{{
210
211     generate if (OPT_REGISTERED)
212     begin : GEN_REGISTERS
213         // {{{
214         // Register all results into the new domain
215         // crossing clock domains
216         reg [AW-1:0] m_paddr;
217         reg

```

```

218         reg [DW-1:0] m_pwdata;
219         reg [DW/8-1:0] m_pwstrb;
220         reg [2:0] m_pprot;
221
222         reg [DW-1:0] s_prdata;
223         reg s_pslverr;
224
225         always @(posedge M_APB_PCLK)
226         if (m_request && !m_ack)
227         begin
228             m_paddr <= S_APB_PADDR;
229             m_pwrite <= S_APB_PWRITE;
230             m_pwdata <= S_APB_PWDATA;
231             m_pwstrb <= S_APB_PWSTRB;
232             m_pprot <= S_APB_PPROT;
233         end
234
235         assign M_APB_PADDR = m_paddr;
236         assign M_APB_PWRITE = m_pwrite;
237         assign M_APB_PWDATA = m_pwdata;
238         assign M_APB_PWSTRB = m_pwstrb;
239         assign M_APB_PPROT = m_pprot;
240
241         always @(posedge S_APB_PCLK or negedge S_PRE-
SETn)
242         if (!S_PRESETn)
243             s_pslverr <= 1'b0;
244         else if (s_tfr_request && s_ack
&& (!S_APB_READY || !
S_APB_PENABLE))
245             s_pslverr <= m_pslverr;
246         else
247             s_pslverr <= 0;
248
249         always @(posedge S_APB_PCLK)
250         if (s_request && s_ack)
251         begin
252             s_prdata <= m_prdata;
253         end else begin
254             s_prdata <= 0;
255         end
256
257         assign S_APB_PRDATA = s_prdata;
258         assign S_APB_PSLVERR = s_pslverr;
259         // }}}
260     end else begin : NO_REGISTERING
261         // {{{
262         // Results will be stable whenever PSEL is
true. There's
263         // really no *need* to register them
264
265         assign M_APB_PADDR = S_APB_PADDR;
266         assign M_APB_PWRITE = S_APB_PWRITE;
267         assign M_APB_PWDATA = S_APB_PWDATA;
268         assign M_APB_PWSTRB = S_APB_PWSTRB;
269         assign M_APB_PPROT = S_APB_PPROT;
270
271         assign S_APB_PRDATA = m_prdata;
272         assign S_APB_PSLVERR = m_pslverr &&
S_APB_READY;
273         // }}}
274     end endgenerate
275     // }}}
276     // Make Verilator happy
277     // {{{
278     // Verilator lint_off UNUSED
279     wire unused;
280     assign unused = &{ 1'b0 };
281     // Verilator lint_on UNUSED
282     // }}}
283     ///////////////////////////////////////////////////////////////////
284     //
285     // Formal properties
286     // {{{
287     ///////////////////////////////////////////////////////////////////
288     //
289     // Formal properties
290     // {{{
291     ///////////////////////////////////////////////////////////////////
292     ///////////////////////////////////////////////////////////////////
293     ///////////////////////////////////////////////////////////////////
294     `ifdef FORMAL
295         (* gclk *) reg gbl_clk;
296
297         ///////////////////////////////////////////////////////////////////
298         //
299         // Interface properties
300         // {{{
301         ///////////////////////////////////////////////////////////////////
302         //
303         //
304         fapb_slave #(
305             // {{{
306

```

```

307         .AW(C_APB_ADDR_WIDTH), .DW(C_APB_DATA_WIDTH),
308         .F_OPT_MAXSTALL(0), .F_OPT_ASYNC_RESET(1'b1),
309         .F_OPT_SLVERR(1'b1)
310         // }}}
311     ) fslv (
312         // {{{
313         .PCLK(S_APB_PCLK), .PRESETn(S_PRESETn),
314         .PSEL(S_APB_PSEL),
315         .PENABLE(S_APB_PENABLE),
316         .PREADY(S_APB_PREADY),
317         .PADDR(S_APB_PADDR),
318         .PWRITE(S_APB_PWRITE), .PDATA(S_APB_PDATA),
319         .PWSTRB(S_APB_PWSTRB),
320         .PPROT(S_APB_PPROT),
321         .PRDATA(S_APB_PRDATA), .PSLVERR(S_APB_PSLVERR)
322         // }}}
323     );
324
325     fapb_master #(
326         // {{{
327         .AW(C_APB_ADDR_WIDTH), .DW(C_APB_DATA_WIDTH),
328         .F_OPT_MAXSTALL(0), .F_OPT_ASYNC_RESET(1'b1),
329         .F_OPT_SLVERR(1'b1)
330         // }}}
331     ) fmas (
332         // {{{
333         .PCLK(M_APB_PCLK), .PRESETn(M_PRESETn),
334         .PSEL(M_APB_PSEL), .PENABLE(M_APB_PENABLE),
335         .PREADY(M_APB_PREADY), .PADDR(M_APB_PA
336         .PWRITE(M_APB_PWRITE), .PDATA(M_APB_PDATA),
337         .PWSTRB(M_APB_PWSTRB),
338         .PPROT(M_APB_PPROT),
339         .PRDATA(M_APB_PRDATA), .PSLVERR(M_APB_PSLVERR)
340         // }}}
341     );
342
343     // }}}
344     ///////////////////////////////////////////////////////////////////
345     //
346     // Clock stability
347     // {{{
348     ///////////////////////////////////////////////////////////////////
349     //
350     //
351     reg        f_past_valid_gbl;
352
353     initial f_past_valid_gbl <= 1'b0;
354     always @(posedge gbl_clk)
355         f_past_valid_gbl <= 1'b1;
356
357     always @(*)
358     if (!f_past_valid_gbl)
359         assume(!S_PRESETn);
360
361     always @(posedge gbl_clk)
362     if (!$rose(S_APB_PCLK))
363     begin
364         assume(!$rose(S_PRESETn));
365         assume($stable(S_APB_PSEL));
366         assume($stable(S_APB_PENABLE));
367         assume($stable(S_APB_PADDR));
368         assume($stable(S_APB_PWRITE));
369         assume($stable(S_APB_PDATA));
370         assume($stable(S_APB_PWSTRB));
371         assume($stable(S_APB_PPROT));
372         //
373         if (f_past_valid_gbl && S_PRESETn)
374         begin
375             assert($stable(S_APB_PREADY));
376             if (OPT_REGISTERED)
377             begin
378                 assert($stable(S_APB_PRDATA));
379                 assert($stable(S_APB_PSLVERR));
380             end else if ($past(S_APB_PREADY) &&
381             S_APB_PREADY)
382             begin
383                 assert($stable(S_APB_PRDATA));
384                 assert($stable(S_APB_PSLVERR));
385             end
386         end
387     end
388
389     always @(posedge gbl_clk)
390     if (!$rose(M_APB_PCLK))
391     begin
392         if (f_past_valid_gbl)
393         begin
394             assert(!$rose(M_PRESETn));
395         end
396         if (f_past_valid_gbl && M_PRESETn)
397         begin
398             assert($stable(M_APB_PSEL));
399             assert($stable(M_APB_PENABLE));
400             if (OPT_REGISTERED)
401             begin
402                 assert($stable(M_APB_PADDR));
403                 assert($stable(M_APB_PWRITE));
404                 assert($stable(M_APB_PDATA));
405                 assert($stable(M_APB_PWRITE));
406                 assert($stable(M_APB_PPROT));
407             end else if ($past(M_APB_PSEL) &&
408             M_APB_PSEL)
409             begin
410                 assert($stable(M_APB_PADDR));
411                 assert($stable(M_APB_PWRITE));
412                 assert($stable(M_APB_PDATA));
413                 assert($stable(M_APB_PWSTRB));
414                 assert($stable(M_APB_PPROT));
415             end
416             //
417             assume($stable(M_APB_PREADY));
418             assume($stable(M_APB_PRDATA));
419             assume($stable(M_APB_PSLVERR));
420         end
421         always @(posedge gbl_clk)
422         if ($past(S_APB_PSEL && !S_APB_PREADY) && $past(S_PRE-
423             SETn) && S_PRESETn)
424         begin
425             assume($stable(S_APB_PSEL));
426             assume($stable(S_APB_PADDR));
427             assume($stable(S_APB_PWRITE));
428             assume($stable(S_APB_PDATA));
429             assume($stable(S_APB_PWSTRB));
430             assume($stable(S_APB_PPROT));
431         end
432         // }}}
433         ///////////////////////////////////////////////////////////////////
434         //
435         // Induction invariants
436         // {{{
437         ///////////////////////////////////////////////////////////////////
438         //
439         //
440         //
441         //
442         always @(*)
443         case({ S_PRESETn, reset_pipe, M_PRESETn, full_re-
444             set_pipe, full_reset })
445             5'b0_00_00: begin end
446             5'b1_00_00: begin end
447             5'b1_10_00: begin end
448             5'b1_11_00: begin end
449             5'b1_11_10: begin end
450             5'b1_11_11: begin end
451             default: assert(0);
452         endcase
453
454         always @(*)
455         casez({ s_request, s_tfr_request, m_request_pipe, m_re-
456             quest, m_ack, s_ack_pipe, s_ack })
457             7'b00_00_000: begin end
458             7'b10_00_000: begin end
459             7'b11_00_000: begin end
460             7'b11_10_000: begin end
461             7'b11_11_000: begin end
462             7'b11_11_100: begin end
463             7'b11_11_110: begin end
464             7'b11_11_111: begin end
465             7'b?0_11_111: begin end
466             7'b?0_01_111: begin end
467             7'b?0_00_111: begin end
468             7'b?0_00_011: begin end
469             7'b?0_00_001: begin end
470             7'b?0_00_000: begin end
471             default: assert(0);
472         endcase
473
474         always @(posedge S_APB_PCLK)
475         if (s_request && S_PRESETn)
476             assert(S_APB_PSEL && S_APB_PENABLE);
477
478         always @(posedge gbl_clk)
479         if (!s_tfr_request && S_PRESETn)
480             assert(!M_APB_PSEL);
481
482         always @(posedge gbl_clk)
483         if (M_APB_PSEL)
484             assert(m_request && !m_ack);
485
486         always @(posedge gbl_clk)
487         if (S_PRESETn && (m_ack || s_ack_pipe || s_ack ||
488             S_APB_PREADY))
489             assert(!M_APB_PSEL);
490         // }}}
491         //
492         // Contract check
493         // {{{
494         ///////////////////////////////////////////////////////////////////
495         //

```

```

496 //
497
498 (* anyconst *) reg          fnvr_check;
499 (* anyconst *) reg          [3+DW/8+DW:0] fnvr_write;
500 (* anyconst *) reg          [AW-1:0]fnvr_addr;
501 (* anyconst *) reg          [1+DW-1:0]    fnvr_return;
502
503 always @(posedge gbl_clk)
504 if (S_APB_PSEL && fnvr_check)
505 begin
506     assume({ S_APB_PPROT, S_APB_PWSTRB, S_APB_P-
WDATA } != fnvr_write);
507     assume(S_APB_PADDR != fnvr_addr);
508 end
509
510 always @(posedge gbl_clk)
511 if (M_APB_PSEL && fnvr_check && M_PRESETn)
512 begin
513     assert({ M_APB_PPROT, M_APB_PWSTRB, M_APB_P-
WDATA } != fnvr_write);
514     assert(M_APB_PADDR != fnvr_addr);
515 end
516
517 always @(posedge gbl_clk)
518 if (M_APB_PSEL && M_PRESETn)
519 begin
520     assert(M_APB_PADDR == S_APB_PADDR);
521     assert(M_APB_PWRITE == S_APB_PWRITE);
522     assert(M_APB_PWDATA == S_APB_PWDATA);
523     assert(M_APB_PWSTRB == S_APB_PWSTRB);
524 end
525
526 always @(posedge gbl_clk)
527 if (fnvr_check && M_APB_PSEL && M_APB_PENABLE &&
M_APB_PREADY)
528     assume({ M_APB_PSLVERR, M_APB_PRDATA } !=
fnvr_return);
529
530 always @(posedge gbl_clk)
531 if (fnvr_check && m_ack)
532     assert({ m_pslverr, m_prdata } != fnvr_return);
533
534 always @(posedge gbl_clk)
535 if (fnvr_check && S_APB_PSEL && S_APB_PENABLE &&
S_APB_PREADY)
536     assert({ S_APB_PSLVERR, S_APB_PRDATA } !=
fnvr_return);
537
538 // }}}
539 // }}}
540 // }}}
541 //
542 // Cover checks
543 // {{{
544 // }}}
545 //
546 //
547
548 reg          [2:0]    cvr_reads, cvr_writes;
549
550 initial cvr_writes = 0;
551 always @(posedge S_APB_PCLK)
552 if (!S_PRESETn)
553     cvr_writes <= 0;
554
555 else if (S_APB_PSEL && S_APB_PENABLE && S_APB_PREADY &&
S_APB_PWRITE
556         && !cvr_writes[2])
557     cvr_writes <= cvr_writes + 1;
558
559 initial cvr_reads = 0;
560 always @(posedge S_APB_PCLK)
561 if (!S_PRESETn)
562     cvr_reads <= 0;
563 else if (S_APB_PSEL && S_APB_PENABLE && S_APB_PREADY
&& !S_APB_PWRITE
564         && !cvr_reads[2])
565     cvr_reads <= cvr_reads + 1;
566
567 always @(*)
568 if (S_PRESETn)
569 begin
570     cover(cvr_writes >= 2);
571     cover(cvr_reads >= 2);
572
573     cover(cvr_writes >= 3);
574     cover(cvr_reads >= 3);
575 end
576
577 // }}}
578 // }}}
579 // "Careless" assumptions
580 // {{{
581 // }}}
582 //
583 //

```

```

584 localparam      F_NCLK = 5;
585 reg          [F_NCLK-1:0]    gbl_sclk, gbl_mclk;
586
587 always @(posedge gbl_clk)
588     gbl_sclk <= { gbl_sclk[F_NCLK-2:0], S_APB_P-
CLK };
589
590 always @(posedge gbl_clk)
591     gbl_mclk <= { gbl_mclk[F_NCLK-2:0], M_APB_P-
CLK };
592
593 always @(posedge gbl_clk)
594 if (gbl_sclk == 0)
595     assume(S_APB_PCLK);
596 else if (&gbl_sclk)
597     assume(!S_APB_PCLK);
598
599 always @(posedge gbl_clk)
600 if (gbl_mclk == 0)
601     assume(M_APB_PCLK);
602 else if (&gbl_mclk)
603     assume(!M_APB_PCLK);
604
605 // }}}
606 `endif
607 // }}}
608 endmodule

```

5. ZipCPU_wb2axip/rtl/axi2axi3.v (895 lines)

```

1 ///////////////////////////////////////////////////////////////////
2 //
3 // Filename:      rtl/axi2axi3.v
4 // {{{
5 // Project:      WB2AXIPSP: bus bridges and other odds and ends
6 //
7 // Purpose:      Bridge from an AXI4 slave to an AXI3 master
8 //
9 // The goal here is to not lose bus resolution, capacity
or capability,
10 // while bridging from AXI4 to AXI3. The biggest problem
with such
11 // a bridge is that we'll need to break large requests
(AxLEN>15) up
12 // into smaller packets. After that, everything should
work as normal
13 // with only minor modifications for AxCACHE.
14 //
15 // Opportunity:
16 // The cost of this core is very much determined by the
number of ID's
17 // supported. It should be possible, with little extra
cost or effort,
18 // to reduce the ID space herein. This should be a topic
for future
19 // exploration.
20 //
21 // Status:
22 // This core is an unverified first draft. It has past
neither formal
23 // nor simulation testing. Therefore, it is almost cer-
tain to have bugs
24 // within it. Use it at your own risk.
25 //
26 // Creator:      Dan Gisselquist, Ph.D.
27 // Gisselquist Technology, LLC
28 //
29 ///////////////////////////////////////////////////////////////////
30 // }}}
31 // Copyright (C) 2019-2025, Gisselquist Technology, LLC
32 // {{{
33 // This file is part of the WB2AXIP project.
34 //
35 // The WB2AXIP project contains free software and gateware,
licensed under the
36 // Apache License, Version 2.0 (the "License"). You may not
use this project,
37 // or this file, except in compliance with the License. You
may obtain a copy
38 // of the License at
39 // {{{
40 // http://www.apache.org/licenses/LICENSE-2.0
41 // {{{
42 // Unless required by applicable law or agreed to in writing,
software
43 // distributed under the License is distributed on an "AS IS"
BASIS, WITHOUT
44 // WARRANTIES OR CONDITIONS OF ANY KIND, either express or im-
plied. See the
45 // License for the specific language governing permissions and
limitations
46 // under the License.
47 //
48 ///////////////////////////////////////////////////////////////////
49 //

```

```

50 `default_nettype none
51 // }}}
52 module axi2axi3 #(
53     // {{{
54     parameter C_AXI_ID_WIDTH = 1,
55     parameter C_AXI_ADDR_WIDTH = 32,
56     parameter C_AXI_DATA_WIDTH = 32
57     // }}}
58 ) (
59     // {{{
60     input wire
61     S_AXI_ACLK,
62     input wire
63     S_AXI_ARESETN,
64     //
65     // The AXI4 incoming/slave interface
66     input reg
67     S_AXI_AWVALID,
68     output wire
69     S_AXI_AWREADY,
70     input reg [C_AXI_ID_WIDTH-1:0]
71     S_AXI_AWID,
72     input reg [C_AXI_ADDR_WIDTH-1:0]
73     S_AXI_AWADDR,
74     input reg [7:0]
75     S_AXI_AWLEN,
76     input reg [2:0]
77     S_AXI_AWSIZE,
78     input reg [1:0]
79     S_AXI_AWBURST,
80     input reg
81     S_AXI_AWLOCK,
82     input reg [3:0]
83     S_AXI_AWCACHE,
84     input reg [2:0]
85     S_AXI_AWPROT,
86     input reg [3:0]
87     S_AXI_AWQOS,
88     //
89     //
90     input wire
91     S_AXI_WVALID,
92     output wire
93     S_AXI_WREADY,
94     input wire [C_AXI_DATA_WIDTH-1:0]
95     S_AXI_WDATA,
96     input wire [C_AXI_DATA_WIDTH/8-1:0]
97     S_AXI_WSTRB,
98     input wire
99     S_AXI_WLAST,
100     //
101     //
102     output wire
103     S_AXI_BVALID,
104     input wire
105     S_AXI_BREADY,
106     output wire [C_AXI_ID_WIDTH-1:0]
107     S_AXI_BID,
108     output wire [1:0]
109     S_AXI_BRESP,
110     //
111     //
112     input wire
113     S_AXI_ARVALID,
114     output wire
115     S_AXI_ARREADY,
116     input wire [C_AXI_ID_WIDTH-1:0]
117     S_AXI_ARID,
118     input wire [C_AXI_ADDR_WIDTH-1:0]
119     S_AXI_ARADDR,
120     input wire [7:0]
121     S_AXI_ARLEN,
122     input wire [2:0]
123     S_AXI_ARSIZE,
124     input wire [1:0]
125     S_AXI_ARBURST,
126     input wire
127     S_AXI_ARLOCK,
128     input wire [3:0]
129     S_AXI_ARCACHE,
130     input wire [2:0]
131     S_AXI_ARPROT,
132     input wire [3:0]
133     S_AXI_ARQOS,
134     //
135     output wire
136     S_AXI_RVALID,
137     input wire
138     S_AXI_RREADY,
139     output wire [C_AXI_ID_WIDTH-1:0]
140     S_AXI_RID,
141     output wire [C_AXI_DATA_WIDTH-1:0]
142     S_AXI_RDATA,
143     output wire
144     S_AXI_RLAST,
145     output wire [1:0]
146     S_AXI_RRESP,
147     //
148     //
149     output wire
150     // The AXI3 Master (outgoing) interface
151     output wire

```

```

M_AXI_AWVALID,
112     input wire
M_AXI_AWREADY,
113     output wire [C_AXI_ID_WIDTH-1:0]
M_AXI_AWID,
114     output wire [C_AXI_ADDR_WIDTH-1:0]
M_AXI_AWADDR,
115     output wire [3:0]
M_AXI_AWLEN,
116     output wire [2:0]
M_AXI_AWSIZE,
117     output wire [1:0]
M_AXI_AWBURST,
118     output wire [1:0]
M_AXI_AWLOCK,
119     output wire [3:0]
M_AXI_AWCACHE,
120     output wire [2:0]
M_AXI_AWPROT,
121     output wire [3:0]
M_AXI_AWQOS,
122     //
123     //
124     output wire
M_AXI_WVALID,
125     input wire
M_AXI_WREADY,
126     output wire [C_AXI_ID_WIDTH-1:0]
M_AXI_WID,
127     output wire [C_AXI_DATA_WIDTH-1:0]
M_AXI_WDATA,
128     output wire [C_AXI_DATA_WIDTH/8-1:0]
M_AXI_WSTRB,
129     output wire
M_AXI_WLAST,
130     //
131     //
132     input wire
M_AXI_BVALID,
133     output wire
M_AXI_BREADY,
134     input wire [C_AXI_ID_WIDTH-1:0]
M_AXI_BID,
135     input wire [1:0]
M_AXI_BRESP,
136     //
137     //
138     output wire
M_AXI_ARVALID,
139     input wire
M_AXI_ARREADY,
140     output wire [C_AXI_ID_WIDTH-1:0]
M_AXI_ARID,
141     output wire [C_AXI_ADDR_WIDTH-1:0]
M_AXI_ARADDR,
142     output wire [3:0]
M_AXI_ARLEN,
143     output wire [2:0]
M_AXI_ARSIZE,
144     output wire [1:0]
M_AXI_ARBURST,
145     output wire [1:0]
M_AXI_ARLOCK,
146     output wire [3:0]
M_AXI_ARCACHE,
147     output wire [2:0]
M_AXI_ARPROT,
148     output wire [3:0]
M_AXI_ARQOS,
149     //
150     input wire
M_AXI_RVALID,
151     output wire
M_AXI_RREADY,
152     input wire [C_AXI_ID_WIDTH-1:0]
M_AXI_RID,
153     input wire [C_AXI_DATA_WIDTH-1:0]
M_AXI_RDATA,
154     input wire
M_AXI_RLAST,
155     input wire [1:0]
M_AXI_RRESP,
156     // }}}
157     );
158
159     //
160     localparam
161     g2(C_AXI_DATA_WIDTH)-3;
162     localparam [0:0]
163     localparam
164     localparam
165     parameter
166     genvar ID;
167
168     ////////////////////////////////////////////////////
169     ////////////////////////////////////////////////////
170     //
171     // WRITE SIDE

```

```

172 // {{{
173 ///////////////////////////////////////////////////////////////////
174 ///////////////////////////////////////////////////////////////////
175 ///////////////////////////////////////////////////////////////////
176 ///////////////////////////////////////////////////////////////////
177 //
178 // Write address channel
179 // {{{
180 ///////////////////////////////////////////////////////////////////
181 //
182 //
183 reg [7:0] r_awlen;
184 reg [3:0] next_wlen;
185 wire awskd_valid;
186 reg awskd_ready;
187 wire [C_AXI_ID_WIDTH-1:0] awskd_id;
188 wire [C_AXI_ADDR_WIDTH-1:0] awskd_addr;
189 wire [7:0] awskd_len;
190 wire [2:0] awskd_size;
191 wire [1:0] awskd_burst;
192 wire awskd_lock;
193 wire [3:0] awskd_cache;
194 wire [2:0] awskd_prot;
195 wire [3:0] awskd_qos;
196 reg axi_awvalid;
197 reg [C_AXI_ID_WIDTH-1:0] axi_awid;
198 reg [C_AXI_ADDR_WIDTH-1:0] axi_awaddr;
199 reg [3:0] axi_awlen;
200 reg [2:0] axi_awsz;
201 reg [1:0] axi_awburst;
202 reg [1:0] axi_awlock;
203 reg [3:0] axi_awcache;
204 reg [2:0] axi_awprot;
205 reg [3:0] axi_awqos;
206
207 skidbuffer #(
208     .DW(C_AXI_ADDR_WIDTH + C_AXI_ID_WIDTH + 8 + 3 + 2
209 + 1+4+3+4),
210     .OPT_OUTREG(1'b0)
211 ) awskid (
212     .i_clk(S_AXI_ACLK), .i_reset(!S_AXI_ARESETN),
213     .i_val-
214 id(S_AXI_AWVALID), .o_ready(S_AXI_AWREADY),
215     .i_data({ S_AXI_AWID, S_AXI_AWADDR,
216 S_AXI_AWLEN,
217 S_AXI_AWLOCK,
218 S_AXI_AWCACHE, S_AXI_AWPROT,
219 S_AXI_AWQOS }},
220     .o_valid(awskd_valid), .i_ready(awskd_ready),
221     .o_data({ awskd_id, awskd_addr,
222 awskd_len,
223 awskd_lock,
224 awskd_size, awskd_burst,
225 awskd_cache, awskd_prot,
226 awskd_qos }
227 );
228
229 always @(*)
230 begin
231     awskd_ready = 1;
232     if (r_awlen > 0)
233         awskd_ready = 0;
234     if (M_AXI_AWVALID && M_AXI_AWREADY)
235         awskd_ready = 0;
236     if (!wfifo_full[LGWFIFO:LGWFIFO-1])
237         awskd_ready = 0;
238 end
239
240 initial axi_awvalid = 1'b0;
241 always @(posedge S_AXI_ACLK)
242 if (!S_AXI_ARESETN)
243     axi_awvalid <= 0;
244 else if (awskd_valid || r_awlen > 0)
245     axi_awvalid <= 1;
246 else if (M_AXI_AWREADY)
247     axi_awvalid <= 0;
248
249 initial r_awlen = 0;
250 initial axi_awlen = 0;
251 always @(posedge S_AXI_ACLK)
252 if (!S_AXI_ARESETN)
253     begin
254         r_awlen <= 0;
255         axi_awlen <= 0;
256     end
257 else if (!M_AXI_AWVALID || M_AXI_AWREADY)
258     begin
259         if (r_awlen > 15)
260             begin
261                 axi_awlen <= 4'd15;
262                 r_awlen <= r_awlen - 8'd16;
263             end
264         else if (r_awlen > 0)
265             begin
266                 axi_awlen[3:0] <= r_awlen[3:0] - 1;
267                 r_awlen <= 0;
268             end
269         else if (awskd_valid)

```

```

260 begin
261     if (awskd_len > 15)
262     begin
263         r_awlen <= awskd_len + 1'b1 -
264             8'd16;
265         axi_awlen <= 4'd15;
266     end
267     else begin
268         r_awlen <= 0;
269         axi_awlen <= awskd_len[3:0];
270     end
271 end
272
273 end
274
275 always @(posedge S_AXI_ACLK)
276 if (awskd_valid && awskd_ready)
277     axi_awaddr <= awskd_addr;
278 else if (M_AXI_AWVALID && M_AXI_AWREADY)
279     begin
280         // Verilator lint_off WIDTH
281         axi_awaddr <= axi_awaddr + ((M_AXI_AWLEN + 1)
282 << M_AXI_AWSIZE);
283         // Verilator lint_on WIDTH
284
285         case(M_AXI_AWSIZE)
286         3'b000: begin end
287         3'b001: axi_awaddr[ 0] <= 0;
288         3'b010: axi_awaddr[1:0] <= 0;
289         3'b011: axi_awaddr[2:0] <= 0;
290         3'b100: axi_awaddr[3:0] <= 0;
291         3'b101: axi_awaddr[4:0] <= 0;
292         3'b110: axi_awaddr[5:0] <= 0;
293         3'b111: axi_awaddr[6:0] <= 0;
294         endcase
295     end
296
297 initial M_AXI_AWSIZE = ADDR_LSB[2:0];
298 always @(posedge S_AXI_ACLK)
299 begin
300     if (awskd_valid && awskd_ready)
301     begin
302         axi_awid <= awskd_id;
303         axi_awsz <= awskd_size;
304         axi_awburst <= awskd_burst;
305         axi_awlock[0] <= awskd_lock;
306         axi_awcache <= awskd_cache;
307         axi_awprot <= awskd_prot;
308         axi_awqos <= awskd_qos;
309     end
310
311     if (C_AXI_DATA_WIDTH < 16)
312         axi_awsz <= 3'b000;
313     else if (C_AXI_DATA_WIDTH < 32)
314         axi_awsz[2:1] <= 2'b00;
315     else if (C_AXI_DATA_WIDTH < 128)
316         axi_awsz[2] <= 1'b0;
317
318     axi_awlock[1] <= 1'b0;
319 end
320
321 assign M_AXI_AWVALID = axi_awvalid;
322 assign M_AXI_AWID = axi_awid;
323 assign M_AXI_AWADDR = axi_awaddr;
324 assign M_AXI_AWLEN = axi_awlen;
325 assign M_AXI_AWSIZE = axi_awsz;
326 assign M_AXI_AWBURST = axi_awburst;
327 assign M_AXI_AWLOCK = axi_awlock;
328 assign M_AXI_AWCACHE = axi_awcache;
329 assign M_AXI_AWPROT = axi_awprot;
330 assign M_AXI_AWQOS = axi_awqos;
331 // }}}
332
333 ///////////////////////////////////////////////////////////////////
334 //
335 // Write data channel
336 // {{{
337 ///////////////////////////////////////////////////////////////////
338 //
339 //
340 wire wskd_valid;
341 reg wskd_ready, write_idle;
342 wire [C_AXI_DATA_WIDTH-1:0] wskd_data;
343 wire [C_AXI_DATA_WIDTH/8-1:0] wskd_strb;
344 wire wskd_last;
345 reg [4:0] r_wlen;
346 wire [C_AXI_ID_WIDTH-1:0] wfifo_id, raw_w-
347 fifo_id;
348 wire [3:0] wfifo_wlen, raw_w-
349 fifo_wlen;
350 reg next_wlast, r_wlast;
351 wire raw_wfifo_last;
352 wire wfifo_empty;
353
354 wire [LGWFIFO:0] wfifo_fill;
355 reg axi_wvalid;
356 reg [C_AXI_DATA_WIDTH-1:0] axi_wdata;

```

```

354 reg [C_AXI_DATA_WIDTH/8-1:0] axi_wstrb;
355 reg [C_AXI_ID_WIDTH-1:0] axi_wid;
356 reg axi_wlast;
357
358 skidbuffer #(
359     .DW(C_AXI_DATA_WIDTH + (C_AXI_DATA_WIDTH/8) +
1),
360     .OPT_OUTREG(1'b0)
361 ) wskid (
362     .i_clk(S_AXI_ACLK), .i_reset(!S_AXI_ARESETN),
363     .i_valid(S_AXI_WVALID), .o_ready(S_AXI_WREADY),
364     .i_data({ S_AXI_WDATA, S_AXI_WSTRB,
S_AXI_WLAST }),
365     .o_valid(wskd_valid), .i_ready(wskd_ready),
366     .o_data({ wskd_data, wskd_strb,
wskd_last })
367 );
368
369 always @(*)
370     wskd_ready = !M_AXI_WVALID || M_AXI_WREADY;
371
372 initial axi_wvalid = 0;
373 always @(posedge S_AXI_ACLK)
374     if (!S_AXI_ARESETN)
375         axi_wvalid <= 0;
376     else if (wskd_valid)
377         axi_wvalid <= 1;
378     else if (M_AXI_WREADY)
379         axi_wvalid <= 0;
380
381 initial write_idle = 0;
382 always @(posedge S_AXI_ACLK)
383     if (!S_AXI_ARESETN)
384         write_idle <= 1;
385     else if (M_AXI_WVALID && M_AXI_WREADY && M_AXI_WLAST
&& !wskd_valid)
386         write_idle <= 1;
387     else if (wskd_valid && wskd_ready)
388         write_idle <= 0;
389
390 always @(posedge S_AXI_ACLK)
391     if (!S_AXI_ARESETN && OPT_LOWPOWER)
392     begin
393         axi_wdata <= 0;
394         axi_wstrb <= 0;
395     end else if (!M_AXI_WVALID || M_AXI_WREADY)
396     begin
397         axi_wdata <= wskd_data;
398         axi_wstrb <= wskd_strb;
399
400         if (OPT_LOWPOWER && !wskd_valid)
401         begin
402             axi_wdata <= 0;
403             axi_wstrb <= 0;
404         end
405     end
406
407 always @(*)
408     begin
409         if (r_awlen[7:4] != 0)
410             next_wlen = 4'd15;
411         else if (r_awlen[3:0] != 0)
412             next_wlen = r_awlen[3:0];
413         else if (awskd_len[7:4] != 0)
414             next_wlen = 4'd15;
415         else
416             next_wlen = awskd_len[3:0];
417
418         if (r_awlen != 0)
419             next_wlast = (r_awlen[7:4] == 0);
420         else
421             next_wlast = (awskd_len[7:4] == 0);
422     end
423
424 sfifo #( .BW(C_AXI_ID_WIDTH+4+1), .LGFLN(LGW_FIFO))
425 wfifo(S_AXI_ACLK, !S_AXI_ARESETN,
426     (!M_AXI_WVALID ||
M_AXI_WREADY)&&(awskd_valid||(r_awlen > 0)))
427     && !write_idle,
428     // The length of the next burst
429     { ((r_awlen != 0) ? M_AXI_AWID : awskd_id),
430       next_wlen,
next_wlast },
431     wfifo_full, wfifo_fill,
432     //
433     (!M_AXI_WVALID || M_AXI_WREADY),
434     { raw_wfifo_id, raw_wfifo_wlen, raw_w-
fifo_last },
435     wfifo_empty);
436
437 assign wfifo_id = (wfifo_empty) ? awskd_id : raw_w-
fifo_id;
438 assign wfifo_wlen = (wfifo_empty) ? next_wlen : raw_w-
fifo_wlen;
439
440 initial r_wlen = 0;
441 always @(posedge S_AXI_ACLK)
442     if (!S_AXI_ARESETN && OPT_LOWPOWER)
443         r_wlen <= 0;
444     else if (!M_AXI_WVALID || M_AXI_WREADY)
445         begin

```

```

446         if (r_wlen > 1)
447             r_wlen <= r_wlen - 1;
448         else if (!wfifo_empty)
449             r_wlen <= raw_wfifo_wlen + 1;
450         else if (awskd_valid && awskd_ready)
451             r_wlen <= next_wlen + 1;
452     end
453
454 initial r_wlast = 0;
455 always @(posedge S_AXI_ACLK)
456     if (!S_AXI_ARESETN && OPT_LOWPOWER)
457         r_wlast <= 0;
458     else if (!M_AXI_WVALID || M_AXI_WREADY)
459     begin
460         if (r_wlen > 0)
461             r_wlast <= (r_wlen <= 1);
462         else
463             r_wlast <= raw_wfifo_last;
464     end
465
466 always @(posedge S_AXI_ACLK)
467     if (!S_AXI_ARESETN && OPT_LOWPOWER)
468     begin
469         axi_wid <= 0;
470         axi_wlast <= 0;
471     end else if (!M_AXI_WVALID || M_AXI_WREADY)
472     begin
473         axi_wid <= wfifo_id;
474         axi_wlast <= r_wlast;
475
476         if (OPT_LOWPOWER && !wskd_valid)
477         begin
478             axi_wid <= 0;
479             axi_wlast <= 0;
480         end
481     end
482
483 assign M_AXI_WVALID = axi_wvalid;
484 assign M_AXI_WDATA = axi_wdata;
485 assign M_AXI_WSTRB = axi_wstrb;
486 assign M_AXI_WID = axi_wid;
487 assign M_AXI_WLAST = axi_wlast;
488 // }}}
489 ///////////////////////////////////////////////////////////////////
490 //
491 // Write return channel
492 // {{{
493 ///////////////////////////////////////////////////////////////////
494 //
495 //
496 wire
497 reg
498 wire [C_AXI_ID_WIDTH-1:0]
499 wire [1:0]
500 reg [1:0]
501
502 [0:NID-1];
503 reg [NID-1:0]
504 reg [NID-1:0]
505 reg [C_AXI_ID_WIDTH-1:0]
506 reg [1:0]
507
508 skidbuffer #(.DW(C_AXI_ID_WIDTH + 2), .OPT_OUTREG(1'b0)
) bskid (
509     .i_clk(S_AXI_ACLK), .i_reset(!S_AXI_ARESETN),
510     .i_valid(M_AXI_BVALID), .o_ready(M_AXI_BREADY),
511     .i_data({ M_AXI_BID, M_AXI_BRESP }),
512     .o_valid(bskd_valid), .i_ready(bskd_ready),
513     .o_data({ bskd_id, bskd_resp })
514 );
515
516 generate for(ID=0; ID < NID; ID = ID + 1)
517     begin : WRITE_BURST_TRACKING
518         wire
519         wire [LG_FIFO:0]
520         wire [LG_FIFO:0]
521         wire [LG_FIFO:0]
522         wire [LG_FIFO:0]
523         wire [LG_FIFO:0]
524         wire [LG_FIFO:0]
525         wire [LG_FIFO:0]
526         wire [LG_FIFO:0]
527         wire [LG_FIFO:0]
528         wire [LG_FIFO:0]
529         wire [LG_FIFO:0]
530         wire [LG_FIFO:0]
531         wire [LG_FIFO:0]
532         wire [LG_FIFO:0]
533         wire [LG_FIFO:0]
534         wire [LG_FIFO:0]
535         wire [LG_FIFO:0]
536         wire [LG_FIFO:0]
537         wire [LG_FIFO:0]
538         wire [LG_FIFO:0]
539         wire [LG_FIFO:0]
540         wire [LG_FIFO:0]
541         wire [LG_FIFO:0]
542         wire [LG_FIFO:0]
543         wire [LG_FIFO:0]
544         wire [LG_FIFO:0]
545         wire [LG_FIFO:0]
546         wire [LG_FIFO:0]
547         wire [LG_FIFO:0]
548         wire [LG_FIFO:0]
549         wire [LG_FIFO:0]
550         wire [LG_FIFO:0]
551         wire [LG_FIFO:0]
552         wire [LG_FIFO:0]
553         wire [LG_FIFO:0]
554         wire [LG_FIFO:0]
555         wire [LG_FIFO:0]
556         wire [LG_FIFO:0]
557         wire [LG_FIFO:0]
558         wire [LG_FIFO:0]
559         wire [LG_FIFO:0]
560         wire [LG_FIFO:0]
561         wire [LG_FIFO:0]
562         wire [LG_FIFO:0]
563         wire [LG_FIFO:0]
564         wire [LG_FIFO:0]
565         wire [LG_FIFO:0]
566         wire [LG_FIFO:0]
567         wire [LG_FIFO:0]
568         wire [LG_FIFO:0]
569         wire [LG_FIFO:0]
570         wire [LG_FIFO:0]
571         wire [LG_FIFO:0]
572         wire [LG_FIFO:0]
573         wire [LG_FIFO:0]
574         wire [LG_FIFO:0]
575         wire [LG_FIFO:0]
576         wire [LG_FIFO:0]
577         wire [LG_FIFO:0]
578         wire [LG_FIFO:0]
579         wire [LG_FIFO:0]
580         wire [LG_FIFO:0]
581         wire [LG_FIFO:0]
582         wire [LG_FIFO:0]
583         wire [LG_FIFO:0]
584         wire [LG_FIFO:0]
585         wire [LG_FIFO:0]
586         wire [LG_FIFO:0]
587         wire [LG_FIFO:0]
588         wire [LG_FIFO:0]
589         wire [LG_FIFO:0]
590         wire [LG_FIFO:0]
591         wire [LG_FIFO:0]
592         wire [LG_FIFO:0]
593         wire [LG_FIFO:0]
594         wire [LG_FIFO:0]
595         wire [LG_FIFO:0]
596         wire [LG_FIFO:0]
597         wire [LG_FIFO:0]
598         wire [LG_FIFO:0]
599         wire [LG_FIFO:0]
600         wire [LG_FIFO:0]
601         wire [LG_FIFO:0]
602         wire [LG_FIFO:0]
603         wire [LG_FIFO:0]
604         wire [LG_FIFO:0]
605         wire [LG_FIFO:0]
606         wire [LG_FIFO:0]
607         wire [LG_FIFO:0]
608         wire [LG_FIFO:0]
609         wire [LG_FIFO:0]
610         wire [LG_FIFO:0]
611         wire [LG_FIFO:0]
612         wire [LG_FIFO:0]
613         wire [LG_FIFO:0]
614         wire [LG_FIFO:0]
615         wire [LG_FIFO:0]
616         wire [LG_FIFO:0]
617         wire [LG_FIFO:0]
618         wire [LG_FIFO:0]
619         wire [LG_FIFO:0]
620         wire [LG_FIFO:0]
621         wire [LG_FIFO:0]
622         wire [LG_FIFO:0]
623         wire [LG_FIFO:0]
624         wire [LG_FIFO:0]
625         wire [LG_FIFO:0]
626         wire [LG_FIFO:0]
627         wire [LG_FIFO:0]
628         wire [LG_FIFO:0]
629         wire [LG_FIFO:0]
630         wire [LG_FIFO:0]
631         wire [LG_FIFO:0]
632         wire [LG_FIFO:0]
633         wire [LG_FIFO:0]
634         wire [LG_FIFO:0]
635         wire [LG_FIFO:0]
636         wire [LG_FIFO:0]
637         wire [LG_FIFO:0]
638         wire [LG_FIFO:0]
639         wire [LG_FIFO:0]
640         wire [LG_FIFO:0]
641         wire [LG_FIFO:0]
642         wire [LG_FIFO:0]
643         wire [LG_FIFO:0]
644         wire [LG_FIFO:0]
645         wire [LG_FIFO:0]
646         wire [LG_FIFO:0]
647         wire [LG_FIFO:0]
648         wire [LG_FIFO:0]
649         wire [LG_FIFO:0]
650         wire [LG_FIFO:0]
651         wire [LG_FIFO:0]
652         wire [LG_FIFO:0]
653         wire [LG_FIFO:0]
654         wire [LG_FIFO:0]
655         wire [LG_FIFO:0]
656         wire [LG_FIFO:0]
657         wire [LG_FIFO:0]
658         wire [LG_FIFO:0]
659         wire [LG_FIFO:0]
660         wire [LG_FIFO:0]
661         wire [LG_FIFO:0]
662         wire [LG_FIFO:0]
663         wire [LG_FIFO:0]
664         wire [LG_FIFO:0]
665         wire [LG_FIFO:0]
666         wire [LG_FIFO:0]
667         wire [LG_FIFO:0]
668         wire [LG_FIFO:0]
669         wire [LG_FIFO:0]
670         wire [LG_FIFO:0]
671         wire [LG_FIFO:0]
672         wire [LG_FIFO:0]
673         wire [LG_FIFO:0]
674         wire [LG_FIFO:0]
675         wire [LG_FIFO:0]
676         wire [LG_FIFO:0]
677         wire [LG_FIFO:0]
678         wire [LG_FIFO:0]
679         wire [LG_FIFO:0]
680         wire [LG_FIFO:0]
681         wire [LG_FIFO:0]
682         wire [LG_FIFO:0]
683         wire [LG_FIFO:0]
684         wire [LG_FIFO:0]
685         wire [LG_FIFO:0]
686         wire [LG_FIFO:0]
687         wire [LG_FIFO:0]
688         wire [LG_FIFO:0]
689         wire [LG_FIFO:0]
690         wire [LG_FIFO:0]
691         wire [LG_FIFO:0]
692         wire [LG_FIFO:0]
693         wire [LG_FIFO:0]
694         wire [LG_FIFO:0]
695         wire [LG_FIFO:0]
696         wire [LG_FIFO:0]
697         wire [LG_FIFO:0]
698         wire [LG_FIFO:0]
699         wire [LG_FIFO:0]
700         wire [LG_FIFO:0]
701         wire [LG_FIFO:0]
702         wire [LG_FIFO:0]
703         wire [LG_FIFO:0]
704         wire [LG_FIFO:0]
705         wire [LG_FIFO:0]
706         wire [LG_FIFO:0]
707         wire [LG_FIFO:0]
708         wire [LG_FIFO:0]
709         wire [LG_FIFO:0]
710         wire [LG_FIFO:0]
711         wire [LG_FIFO:0]
712         wire [LG_FIFO:0]
713         wire [LG_FIFO:0]
714         wire [LG_FIFO:0]
715         wire [LG_FIFO:0]
716         wire [LG_FIFO:0]
717         wire [LG_FIFO:0]
718         wire [LG_FIFO:0]
719         wire [LG_FIFO:0]
720         wire [LG_FIFO:0]
721         wire [LG_FIFO:0]
722         wire [LG_FIFO:0]
723         wire [LG_FIFO:0]
724         wire [LG_FIFO:0]
725         wire [LG_FIFO:0]
726         wire [LG_FIFO:0]
727         wire [LG_FIFO:0]
728         wire [LG_FIFO:0]
729         wire [LG_FIFO:0]
730         wire [LG_FIFO:0]
731         wire [LG_FIFO:0]
732         wire [LG_FIFO:0]
733         wire [LG_FIFO:0]
734         wire [LG_FIFO:0]
735         wire [LG_FIFO:0]
736         wire [LG_FIFO:0]
737         wire [LG_FIFO:0]
738         wire [LG_FIFO:0]
739         wire [LG_FIFO:0]
740         wire [LG_FIFO:0]
741         wire [LG_FIFO:0]
742         wire [LG_FIFO:0]
743         wire [LG_FIFO:0]
744         wire [LG_FIFO:0]
745         wire [LG_FIFO:0]
746         wire [LG_FIFO:0]
747         wire [LG_FIFO:0]
748         wire [LG_FIFO:0]
749         wire [LG_FIFO:0]
750         wire [LG_FIFO:0]
751         wire [LG_FIFO:0]
752         wire [LG_FIFO:0]
753         wire [LG_FIFO:0]
754         wire [LG_FIFO:0]
755         wire [LG_FIFO:0]
756         wire [LG_FIFO:0]
757         wire [LG_FIFO:0]
758         wire [LG_FIFO:0]
759         wire [LG_FIFO:0]
760         wire [LG_FIFO:0]
761         wire [LG_FIFO:0]
762         wire [LG_FIFO:0]
763         wire [LG_FIFO:0]
764         wire [LG_FIFO:0]
765         wire [LG_FIFO:0]
766         wire [LG_FIFO:0]
767         wire [LG_FIFO:0]
768         wire [LG_FIFO:0]
769         wire [LG_FIFO:0]
770         wire [LG_FIFO:0]
771         wire [LG_FIFO:0]
772         wire [LG_FIFO:0]
773         wire [LG_FIFO:0]
774         wire [LG_FIFO:0]
775         wire [LG_FIFO:0]
776         wire [LG_FIFO:0]
777         wire [LG_FIFO:0]
778         wire [LG_FIFO:0]
779         wire [LG_FIFO:0]
780         wire [LG_FIFO:0]
781         wire [LG_FIFO:0]
782         wire [LG_FIFO:0]
783         wire [LG_FIFO:0]
784         wire [LG_FIFO:0]
785         wire [LG_FIFO:0]
786         wire [LG_FIFO:0]
787         wire [LG_FIFO:0]
788         wire [LG_FIFO:0]
789         wire [LG_FIFO:0]
790         wire [LG_FIFO:0]
791         wire [LG_FIFO:0]
792         wire [LG_FIFO:0]
793         wire [LG_FIFO:0]
794         wire [LG_FIFO:0]
795         wire [LG_FIFO:0]
796         wire [LG_FIFO:0]
797         wire [LG_FIFO:0]
798         wire [LG_FIFO:0]
799         wire [LG_FIFO:0]
800         wire [LG_FIFO:0]
801         wire [LG_FIFO:0]
802         wire [LG_FIFO:0]
803         wire [LG_FIFO:0]
804         wire [LG_FIFO:0]
805         wire [LG_FIFO:0]
806         wire [LG_FIFO:0]
807         wire [LG_FIFO:0]
808         wire [LG_FIFO:0]
809         wire [LG_FIFO:0]
810         wire [LG_FIFO:0]
811         wire [LG_FIFO:0]
812         wire [LG_FIFO:0]
813         wire [LG_FIFO:0]
814         wire [LG_FIFO:0]
815         wire [LG_FIFO:0]
816         wire [LG_FIFO:0]
817         wire [LG_FIFO:0]
818         wire [LG_FIFO:0]
819         wire [LG_FIFO:0]
820         wire [LG_FIFO:0]
821         wire [LG_FIFO:0]
822         wire [LG_FIFO:0]
823         wire [LG_FIFO:0]
824         wire [LG_FIFO:0]
825         wire [LG_FIFO:0]
826         wire [LG_FIFO:0]
827         wire [LG_FIFO:0]
828         wire [LG_FIFO:0]
829         wire [LG_FIFO:0]
830         wire [LG_FIFO:0]
831         wire [LG_FIFO:0]
832         wire [LG_FIFO:0]
833         wire [LG_FIFO:0]
834         wire [LG_FIFO:0]
835         wire [LG_FIFO:0]
836         wire [LG_FIFO:0]
837         wire [LG_FIFO:0]
838         wire [LG_FIFO:0]
839         wire [LG_FIFO:0]
840         wire [LG_FIFO:0]
841         wire [LG_FIFO:0]
842         wire [LG_FIFO:0]
843         wire [LG_FIFO:0]
844         wire [LG_FIFO:0]
845         wire [LG_FIFO:0]
846         wire [LG_FIFO:0]
847         wire [LG_FIFO:0]
848         wire [LG_FIFO:0]
849         wire [LG_FIFO:0]
850         wire [LG_FIFO:0]
851         wire [LG_FIFO:0]
852         wire [LG_FIFO:0]
853         wire [LG_FIFO:0]
854         wire [LG_FIFO:0]
855         wire [LG_FIFO:0]
856         wire [LG_FIFO:0]
857         wire [LG_FIFO:0]
858         wire [LG_FIFO:0]
859         wire [LG_FIFO:0]
860         wire [LG_FIFO:0]
861         wire [LG_FIFO:0]
862         wire [LG_FIFO:0]
863         wire [LG_FIFO:0]
864         wire [LG_FIFO:0]
865         wire [LG_FIFO:0]
866         wire [LG_FIFO:0]
867         wire [LG_FIFO:0]
868         wire [LG_FIFO:0]
869         wire [LG_FIFO:0]
870         wire [LG_FIFO:0]
871         wire [LG_FIFO:0]
872         wire [LG_FIFO:0]
873         wire [LG_FIFO:0]
874         wire [LG_FIFO:0]
875         wire [LG_FIFO:0]
876         wire [LG_FIFO:0]
877         wire [LG_FIFO:0]
878         wire [LG_FIFO:0]
879         wire [LG_FIFO:0]
880         wire [LG_FIFO:0]
881         wire [LG_FIFO:0]
882         wire [LG_FIFO:0]
883         wire [LG_FIFO:0]
884         wire [LG_FIFO:0]
885         wire [LG_FIFO:0]
886         wire [LG_FIFO:0]
887         wire [LG_FIFO:0]
888         wire [LG_FIFO:0]
889         wire [LG_FIFO:0]
890         wire [LG_FIFO:0]
891         wire [LG_FIFO:0]
892         wire [LG_FIFO:0]
893         wire [LG_FIFO:0]
894         wire [LG_FIFO:0]
895         wire [LG_FIFO:0]
896         wire [LG_FIFO:0]
897         wire [LG_FIFO:0]
898         wire [LG_FIFO:0]
899         wire [LG_FIFO:0]
900         wire [LG_FIFO:0]
901         wire [LG_FIFO:0]
902         wire [LG_FIFO:0]
903         wire [LG_FIFO:0]
904         wire [LG_FIFO:0]
905         wire [LG_FIFO:0]
906         wire [LG_FIFO:0]
907         wire [LG_FIFO:0]
908         wire [LG_FIFO:0]
909         wire [LG_FIFO:0]
910         wire [LG_FIFO:0]
911         wire [LG_FIFO:0]
912         wire [LG_FIFO:0]
913         wire [LG_FIFO:0]
914         wire [LG_FIFO:0]
915         wire [LG_FIFO:0]
916         wire [LG_FIFO:0]
917         wire [LG_FIFO:0]
918         wire [LG_FIFO:0]
919         wire [LG_FIFO:0]
920         wire [LG_FIFO:0]
921         wire [LG_FIFO:0]
922         wire [LG_FIFO:0]
923         wire [LG_FIFO:0]
924         wire [LG_FIFO:0]
925         wire [LG_FIFO:0]
926         wire [LG_FIFO:0]
927         wire [LG_FIFO:0]
928         wire [LG_FIFO:0]
929         wire [LG_FIFO:0]
930         wire [LG_FIFO:0]
931         wire [LG_FIFO:0]
932         wire [LG_FIFO:0]
933         wire [LG_FIFO:0]
934         wire [LG_FIFO:0]
935         wire [LG_FIFO:0]
936         wire [LG_FIFO:0]
937         wire [LG_FIFO:0]
938         wire [LG_FIFO:0]
939         wire [LG_FIFO:0]
940         wire [LG_FIFO:0]
941         wire [LG_FIFO:0]
942         wire [LG_FIFO:0]
943         wire [LG_FIFO:0]
944         wire [LG_FIFO:0]
945         wire [LG_FIFO:0]
946         wire [LG_FIFO:0]
947         wire [LG_FIFO:0]
948         wire [LG_FIFO:0]
949         wire [LG_FIFO:0]
950         wire [LG_FIFO:0]
951         wire [LG_FIFO:0]
952         wire [LG_FIFO:0]
953         wire [LG_FIFO:0]
954         wire [LG_FIFO:0]
955         wire [LG_FIFO:0]
956         wire [LG_FIFO:0]
957         wire [LG_FIFO:0]
958         wire [LG_FIFO:0]
959         wire [LG_FIFO:0]
960         wire [LG_FIFO:0]
961         wire [LG_FIFO:0]
962         wire [LG_FIFO:0]
963         wire [LG_FIFO:0]
964         wire [LG_FIFO:0]
965         wire [LG_FIFO:0]
966         wire [LG_FIFO:0]
967         wire [LG_FIFO:0]
968         wire [LG_FIFO:0]
969         wire [LG_FIFO:0]
970         wire [LG_FIFO:0]
971         wire [LG_FIFO:0]
972         wire [LG_FIFO:0]
973         wire [LG_FIFO:0]
974         wire [LG_FIFO:0]
975         wire [LG_FIFO:0]
976         wire [LG_FIFO:0]
977         wire [LG_FIFO:0]
978         wire [LG_FIFO:0]
979         wire [LG_FIFO:0]
980         wire [LG_FIFO:0]
981         wire [LG_FIFO:0]
982         wire [LG_FIFO:0]
983         wire [LG_FIFO:0]
984         wire [LG_FIFO:0]
985         wire [LG_FIFO:0]
986         wire [LG_FIFO:0]
987         wire [LG_FIFO:0]
988         wire [LG_FIFO:0]
989         wire [LG_FIFO:0]
990         wire [LG_FIFO:0]
991         wire [LG_FIFO:0]
992         wire [LG_FIFO:0]
993         wire [LG_FIFO:0]
994         wire [LG_FIFO:0]
995         wire [LG_FIFO:0]
996         wire [LG_FIFO:0]
997         wire [LG_FIFO:0]
998         wire [LG_FIFO:0]
999         wire [LG_FIFO:0]
1000        wire [LG_FIFO:0]

```



```

540      );
541
542      // Verilator lint_off UNUSED
543      wire unused;
544      assign unused = &{ 1'b0, wbfifo_full,
wbfifo_fill };
545      // Verilator lint_on UNUSED
546
547      end endgenerate
548
549      always @(*)
550          bskd_ready = (!S_AXI_BVALID || S_AXI_BREADY);
551
552      initial axi_bvalid = 1'b0;
553      always @(posedge S_AXI_ACLK)
554          if (!S_AXI_ARESETN)
555              axi_bvalid <= 1'b0;
556          else if (bskd_valid && bskd_ready)
557              axi_bvalid <= wbfifo_last[bskd_id];
558          else if (M_AXI_BREADY)
559              axi_bvalid <= 1'b0;
560
561      generate for(ID=0; ID < NID; ID = ID + 1)
562          begin : WRITE_ERR_BY_ID
563
564              initial bburst_err[ID] = 2'b0;
565              always @(posedge S_AXI_ACLK)
566                  if (!S_AXI_ARESETN)
567                      bburst_err[ID] <= 2'b0;
568                  else if (S_AXI_BVALID && S_AXI_BREADY &&
S_AXI_BID == ID
569                      && wbfifo_last[ID])
570                      bburst_err[ID] <= 2'b0;
571                  else if (bskd_valid && bskd_id == ID)
572                      bburst_err[ID] <= bburst_err[ID] |
bskd_resp;
573
574              end endgenerate
575
576              initial axi_bid = 0;
577              initial axi_bresp = 0;
578              always @(posedge S_AXI_ACLK)
579                  if (OPT_LOWPPOWER && !S_AXI_ARESETN)
580                      begin
581                          axi_bid <= 0;
582                          axi_bresp <= 0;
583                      end else if (!S_AXI_BVALID || S_AXI_BREADY)
584                      begin
585                          axi_bid <= bskd_id;
586                          axi_bresp <= bskd_resp | bburst_err[bskd_id];
587
588                          if (OPT_LOWPPOWER && !bskd_valid)
589                              begin
590                                  axi_bid <= 0;
591                                  axi_bresp <= 0;
592                              end
593                      end
594
595              assign S_AXI_BVALID = axi_bvalid;
596              assign S_AXI_BID = axi_bid;
597              assign S_AXI_BRESP = axi_bresp;
598              // }}}
599              // }}}
600              ///////////////////////////////////////////////////////////////////
601              ///////////////////////////////////////////////////////////////////
602              //
603              // READ SIDE
604              // {{{
605              ///////////////////////////////////////////////////////////////////
606              ///////////////////////////////////////////////////////////////////
607              //
608              ///////////////////////////////////////////////////////////////////
609              //
610              // Read address channel
611              // {{{
612              ///////////////////////////////////////////////////////////////////
613              //
614              //
615              reg [7:0] r_arlen;
616              wire arskd_valid;
617              reg arskd_ready;
618              wire [C_AXI_ID_WIDTH-1:0] arskd_id;
619              wire [C_AXI_ADDR_WIDTH-1:0] arskd_addr;
620              wire [7:0] arskd_len;
621              wire [2:0] arskd_size;
622              wire [1:0] arskd_burst;
623              wire arskd_lock;
624              wire [3:0] arskd_cache;
625              wire [2:0] arskd_prot;
626              wire [3:0] arskd_qos;
627              reg axi_arvalid;
628              reg [C_AXI_ID_WIDTH-1:0] axi_arid;
629              reg [C_AXI_ADDR_WIDTH-1:0] axi_araddr;
630              reg [3:0] axi_arlen;
631              reg [2:0] axi_arsize;

```

```

632      reg [1:0] axi_arburst;
633      reg [1:0] axi_arlock;
634      reg [3:0] axi_arcache;
635      reg [2:0] axi_arprot;
636      reg [3:0] axi_arqos;
637
638      skidbuffer #(
639          .DW(C_AXI_ADDR_WIDTH + C_AXI_ID_WIDTH + 8 + 3 + 2
+ 1+4+3+4),
640          .OPT_OUTREG(1'b0)
641      ) arskid (
642          .i_clk(S_AXI_ACLK), .i_reset(!S_AXI_ARESETN),
643          .i_val-
id(S_AXI_ARVALID), .o_ready(S_AXI_ARREADY),
644          .i_data({ S_AXI_ARID, S_AXI_ARADDR,
S_AXI_ARLEN,
645                  S_AXI_ARSIZE, S_AXI_ARBURST,
S_AXI_ARLOCK,
646                  S_AXI_ARCACHE, S_AXI_ARPROT,
S_AXI_ARQOS })),
647          .o_valid(arskd_valid), .i_ready(arskd_ready),
648          .o_data({ arskd_id, arskd_addr,
arskd_len,
649                  arskd_size, arskd_burst,
arskd_lock,
650                  arskd_cache, arskd_prot,
arskd_qos }));
651
652      always @(*)
653          begin
654              arskd_ready = 1;
655              if (r_arlen > 0)
656                  arskd_ready = 0;
657              if (M_AXI_ARVALID && M_AXI_ARREADY)
658                  arskd_ready = 0;
659
660          end
661
662          initial axi_arvalid = 1'b0;
663          always @(posedge S_AXI_ACLK)
664              if (!S_AXI_ARESETN)
665                  axi_arvalid <= 1'b0;
666              else if (r_arlen > 0)
667                  axi_arvalid <= 1'b1;
668              else if (arskd_ready)
669                  axi_arvalid <= 1'b1;
670              else if (M_AXI_ARREADY)
671                  axi_arvalid <= 1'b0;
672
673          initial r_arlen = 0;
674          initial M_AXI_ARLEN = 0;
675          always @(posedge S_AXI_ACLK)
676              if (!S_AXI_ARESETN)
677                  begin
678                      r_arlen <= 0;
679                      axi_arlen <= 0;
680                  end else if (!M_AXI_ARVALID || M_AXI_ARREADY)
681                  begin
682                      if (r_arlen[7:4] != 0)
683                          begin
684                              axi_arlen <= 4'd15;
685                              r_arlen <= r_arlen - 16;
686                          end else if (r_arlen[3:0] != 0)
687                          begin
688                              axi_arlen <= r_arlen[3:0] - 1;
689                              r_arlen <= 0;
690                          end else if (arskd_valid)
691                          begin
692                              if (arskd_len[7:4] != 0)
693                                  begin
694                                      r_arlen <= arskd_len + 1 - 16;
695                                      axi_arlen <= 4'd15;
696                                  end else begin
697                                      r_arlen <= 0;
698                                      axi_arlen <= arskd_len[3:0];
699                                  end
700                          end else begin
701                              r_arlen <= 0;
702                              axi_arlen <= 0;
703                          end
704                      end
705
706          always @(posedge S_AXI_ACLK)
707              if (arskd_valid && arskd_ready)
708                  axi_araddr <= arskd_addr;
709              else if (M_AXI_ARVALID && M_AXI_ARREADY)
710                  begin
711                      // Verilator lint_off WIDTH
712                      axi_araddr <= axi_araddr + ((M_AXI_ARLEN + 1)
<< M_AXI_ARSIZE);
713                      // Verilator lint_on WIDTH
714
715                      case(M_AXI_AWSIZE)
716                      3'b000: begin end
717                      3'b001: axi_araddr[ 0] <= 0;
718                      3'b010: axi_araddr[1:0] <= 0;
719                      3'b011: axi_araddr[2:0] <= 0;
720                      3'b100: axi_araddr[3:0] <= 0;
721                      3'b101: axi_araddr[4:0] <= 0;
722

```



```

723      3'b110: axi_araddr[5:0] <= 0;
724      3'b111: axi_araddr[6:0] <= 0;
725      endcase
726  end
727
728  initial axi_arsize = ADDR_LSB[2:0];
729  always @(posedge S_AXI_ACLK)
730  begin
731      if (arskd_valid && arskd_ready)
732      begin
733          axi_arid      <= arskd_id;
734          axi_arsize    <= arskd_size;
735          axi_arburst    <= arskd_burst;
736          axi_arlock[0] <= arskd_lock;
737          axi_arcache    <= arskd_cache;
738          axi_arprot    <= arskd_prot;
739          axi_arqos      <= arskd_qos;
740      end
741
742      // Propagate constants, to help the optimizer
743
744  out out a touch
745  axi_arlock[1] <= 1'b0;
746
747  if (C_AXI_DATA_WIDTH < 16)
748      axi_arsize <= 3'b000;
749  else if (C_AXI_DATA_WIDTH < 32)
750      axi_arsize[2:1] <= 2'b00;
751  else if (C_AXI_DATA_WIDTH < 128)
752      axi_arsize[2] <= 1'b0;
753  end
754
755  generate for(ID=0; ID < NID; ID = ID + 1)
756  begin : READ_BURST_TRACKING
757      wire                                rbfifo_empty, rb-
758  fifo_full;
759      wire [LGFIFO:0]                    rbfifo_fill;
760
761      initial rbfifo_valid[ID] = 0;
762      always @(posedge S_AXI_ACLK)
763      if (!S_AXI_ARESETN)
764          rbfifo_valid[ID] <= 0;
765      else if (!rbfifo_valid[ID] && !rbfifo_empty)
766          rbfifo_valid[ID] <= 1;
767      else if (rskd_valid && rskd_ready && rskd_last
768  && rskd_id==ID)
769          rbfifo_valid[ID] <= !rbfifo_empty;
770
771      sfifo #(BW(1), .LGFLN(LGFIFO))
772      rbfifo(S_AXI_ACLK, !S_AXI_ARESETN,
773  (!M_AXI_ARVALID || M_AXI_ARREADY)
774  && ((r_ar-
775  len>0)&&(M_AXI_ARID==ID))
776  || (arskd_valid &&
777  arskd_id == ID)),
778  ((arskd_valid && arskd_ready &&
779  arskd_len <= 15)
780  || (r_arlen <= 15)) ? 1'b1 :
781  1'b0,
782  rbfifo_full, rbfifo_fill,
783  //
784  (!rbfifo_valid[ID] || (M_AXI_RVALID &&
785  M_AXI_RREADY
786  && M_AXI_RID == ID &&
787  M_AXI_RLAST))),
788  rid_last[ID], rbfifo_empty
789  );
790
791  // Verilator lint_off UNUSED
792  wire unused;
793  assign unused = &{ 1'b0, rbfifo_full, rb-
794  fifo_fill };
795  // Verilator lint_on UNUSED
796  end endgenerate
797
798  assign M_AXI_ARVALID= axi_arvalid;
799  assign M_AXI_ARID = axi_arid;
800  assign M_AXI_ARADDR = axi_araddr;
801  assign M_AXI_ARLEN = axi_arlen;
802  assign M_AXI_ARSIZE = axi_arsize;
803  assign M_AXI_ARBURST= axi_arburst;
804  assign M_AXI_ARLOCK = axi_arlock;
805  assign M_AXI_ARCACHE= axi_arcache;
806  assign M_AXI_ARPROT = axi_arprot;
807  assign M_AXI_ARQOS = axi_arqos;
808  // }}}
809  // }}}
810  // Read data channel
811  // {{{
812  // {{{
813  // }}}
814  // }}}
815  wire                                rskd_valid;
816  reg                                 rskd_ready;
817  wire [C_AXI_ID_WIDTH-1:0]          rskd_id;
818  wire [C_AXI_DATA_WIDTH-1:0]        rskd_data;
819  wire                                rskd_last;
820  wire [1:0]                          rskd_resp;
821  reg [1:0]                          rburst_err

```

```

[0:NID-1];
812  reg [NID-1:0]                      rbfifo_valid;
813  reg [NID-1:0]                      rid_last;
814  reg                                axi_rvalid;
815  reg [C_AXI_ID_WIDTH-1:0]          axi_rid;
816  reg [C_AXI_DATA_WIDTH-1:0]        axi_rdata;
817  reg                                axi_rlast;
818  reg                                axi_rresp;
819
820  skidbuffer #(
821      .DW(C_AXI_DATA_WIDTH + C_AXI_ID_WIDTH + 3),
822      .OPT_OUTREG(1'b0)
823  ) rskid (
824      .i_clk(S_AXI_ACLK), .i_reset(!S_AXI_ARESETN),
825      .i_valid(M_AXI_RVALID), .o_ready(M_AXI_RREADY),
826      .i_data({ M_AXI_RID, M_AXI_RDATA,
827  M_AXI_RLAST,
828      M_AXI_RRESP }},
829      .o_valid(rskd_valid), .i_ready(rskd_ready),
830      .o_data({ rskd_id, rskd_data,
831  rskd_last, rskd_resp }));
832
833  always @(*)
834      rskd_ready = !S_AXI_RVALID || S_AXI_RREADY;
835
836  initial axi_rvalid = 1'b0;
837  always @(posedge S_AXI_ACLK)
838  if (!S_AXI_ARESETN)
839      axi_rvalid <= 1'b0;
840  else if (rskd_valid && rskd_ready)
841      axi_rvalid <= 1'b1;
842  else if (S_AXI_RREADY)
843      axi_rvalid <= 1'b0;
844
845  generate for(ID=0; ID < NID; ID = ID + 1)
846  begin : READ_ERR_BY_ID
847
848      initial rburst_err[ID] = 2'b0;
849      always @(posedge S_AXI_ACLK)
850      if (!S_AXI_ARESETN)
851          rburst_err[ID] <= 2'b0;
852      else if (S_AXI_RVALID && S_AXI_RREADY &&
853  && S_AXI_RID == ID)
854          rburst_err[ID] <= 2'b0;
855      else if (rskd_valid && rskd_id == ID)
856          rburst_err[ID] <= rburst_err[ID] |
857  S_AXI_RLAST
858  );
859
860  end endgenerate
861
862  always @(posedge S_AXI_ACLK)
863  if (!S_AXI_RVALID || S_AXI_RREADY)
864  begin
865      axi_rid <= rskd_id;
866      axi_rdata <= rskd_data;
867      axi_rresp <= rskd_resp | rburst_err[rskd_id];
868      axi_rlast <= rid_last[rskd_id] && rskd_last;
869  end
870
871  assign S_AXI_RVALID= axi_rvalid;
872  assign S_AXI_RID = axi_rid;
873  assign S_AXI_RDATA = axi_rdata;
874  assign S_AXI_RRESP = axi_rresp;
875  assign S_AXI_RLAST = axi_rlast;
876  // }}}
877  // }}}
878  // Make Verilator happy
879  // {{{
880  // Verilator lint_off UNUSED
881  wire unused;
882  assign unused = &{ 1'b0, wfifo_fill[LGFIFO-2:0],
883  wskd_last,
884      wfifo_wlen, wfifo_full };
885  // Verilator lint_on UNUSED
886  // }}}
887  // }}}
888  // Formal property section
889  //
890  `ifdef FORMAL
891  //
892  // This design has not been formally verified.
893  //
894  `endif
895  endmodule

```

6. ZipCPU_wb2axip/rtl/axi2axilite.v (1,172 lines)

```

1 //
2 //

```

```

3 // Filename:    rtl/axi2axilite.v
4 // {{{
5 // Project:    WB2AXIPSP: bus bridges and other odds and ends
6 //
7 // Purpose:    Convert from AXI to AXI-lite with no performance loss.
8 //
9 // Performance: The goal of this converter is to convert from AXI to AXI-lite
10 //              while still maintaining the one-clock per transaction speed
11 //              of AXI. It currently achieves this goal. The design needs very little
12 //              configuration to be useful, but you might wish to resize the FIFOs
13 //              within depending upon the length of your slave's data path. The current
14 //              FIFO length, LGFIFO=4, is sufficient to maintain full speed. If the
15 //              slave, however, can maintain full speed but requires a longer
16 //              processing cycle, then you may need longer FIFOs.
17 //
18 //              The AXI specification does require an additional 2 clocks per
19 //              transaction when using this core, so your latency will go up.
20 //
21 // Related:    There's a related axidoube.v core in the same repository as
22 //              well. That can be used to convert the AXI protocol to something
23 //              simpler (even simpler than AXI-lite), but it can also do so for multiple
24 //              downstream slaves at the same time.
25 //
26 // Creator:    Dan Gisselquist, Ph.D.
27 //              Gisselquist Technology, LLC
28 //
29 ///////////////////////////////////////////////////////////////////
30 // }}}
31 // Copyright (C) 2019-2025, Gisselquist Technology, LLC
32 // {{{
33 // This file is part of the WB2AXIP project.
34 //
35 // The WB2AXIP project contains free software and gatewayware, licensed under the
36 // Apache License, Version 2.0 (the "License"). You may not use this project,
37 // or this file, except in compliance with the License. You may obtain a copy
38 // of the License at
39 // }}}
40 // http://www.apache.org/licenses/LICENSE-2.0
41 // {{{
42 // Unless required by applicable law or agreed to in writing, software
43 // distributed under the License is distributed on an "AS IS" BASIS, WITHOUT
44 // WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the
45 // License for the specific language governing permissions and limitations
46 // under the License.
47 //
48 ///////////////////////////////////////////////////////////////////
49 //
50 `default_nettype      none
51 // }}}
52 module axi2axilite #(
53     // {{{
54     parameter integer C_AXI_ID_WIDTH= 2,
55     parameter integer C_AXI_DATA_WIDTH = 32,
56     parameter integer C_AXI_ADDR_WIDTH = 6,
57     parameter [0:0] OPT_WRITES = 1,
58     parameter [0:0] OPT_READS = 1,
59     parameter [0:0] OPT_LOWPOWER = 0,
60     // Log (based two) of the maximum number of
61     outstanding AXI
62     // (not AXI-lite) transactions. If you multiply 2^LGFIPO * 256,
63     // you'll get the maximum number of outstanding
64     // transactions
65     parameter LGFIPO = 4
66     // }}}
67 ) (
68     // {{{
69     input wire S_AXI_ACLK,
70     input wire S_AXI_ARESETN,
71     // AXI4 slave interface
72     // {{{
73     // Write address channel
74     input wire
75     S_AXI_AWVALID,

```

```

S_AXI_AWREADY,
76     input wire [C_AXI_ID_WIDTH-1:0]
S_AXI_AWID,
77     input wire [C_AXI_ADDR_WIDTH-1:0]
S_AXI_AWADDR,
78     input wire [7:0]
S_AXI_AWLEN,
79     input wire [2:0]
S_AXI_AWSIZE,
80     input wire [1:0]
S_AXI_AWBURST,
81     input wire
S_AXI_AWLOCK,
82     input wire [3:0]
S_AXI_AWCACHE,
83     input wire [2:0]
S_AXI_AWPROT,
84     input wire [3:0]
S_AXI_AWQOS,
85     // }}}
86     // Write data channel
87     // {{{
88     input wire
S_AXI_WVALID,
89     output wire
S_AXI_WREADY,
90     input wire [C_AXI_DATA_WIDTH-1:0]
S_AXI_WDATA,
91     input wire [(C_AXI_DATA_WIDTH/8)-1:0]
S_AXI_WSTRB,
92     input wire
S_AXI_WLAST,
93     // }}}
94     // Write return channel
95     // {{{
96     output wire
S_AXI_BVALID,
97     input wire
S_AXI_BREADY,
98     output wire [C_AXI_ID_WIDTH-1:0]
S_AXI_BID,
99     output wire [1:0]
S_AXI_BRESP,
100     // }}}
101     // Read address channel
102     // {{{
103     input wire
S_AXI_ARVALID,
104     output wire
S_AXI_ARREADY,
105     input wire [C_AXI_ID_WIDTH-1:0]
S_AXI_ARID,
106     input wire [C_AXI_ADDR_WIDTH-1:0]
S_AXI_ARADDR,
107     input wire [7:0]
S_AXI_ARLEN,
108     input wire [2:0]
S_AXI_ARSIZE,
109     input wire [1:0]
S_AXI_ARBURST,
110     input wire
S_AXI_ARLOCK,
111     input wire [3:0]
S_AXI_ARCACHE,
112     input wire [2:0]
S_AXI_ARPROT,
113     input wire [3:0]
S_AXI_ARQOS,
114     // }}}
115     // Read data channel
116     // {{{
117     output wire
S_AXI_RVALID,
118     input wire
S_AXI_RREADY,
119     output wire [C_AXI_ID_WIDTH-1:0] S_AXI_RID,
120     output wire [C_AXI_DATA_WIDTH-1:0]
S_AXI_RDATA,
121     output wire [1:0]
S_AXI_RRESP,
122     output wire
S_AXI_RLAST,
123     // }}}
124     // }}}
125     // AXI-lite master interface
126     // {{{
127     // AXI-lite Write interface
128     // {{{
129     output wire [C_AXI_ADDR_WIDTH-1:0]
M_AXI_AWADDR,
130     output wire [2 : 0]
M_AXI_AWPROT,
131     output wire
M_AXI_AWVALID,
132     input wire
M_AXI_AWREADY,
133     output wire [C_AXI_DATA_WIDTH-1:0]
M_AXI_WDATA,
134     output wire [(C_AXI_DATA_WIDTH/8)-1:0]
M_AXI_WSTRB,
135     output wire

```

```

M_AXI_WVALID,
136         input    wire
M_AXI_WREADY,
137         input    wire    [1 : 0]
M_AXI_BRESP,
138         input    wire
M_AXI_BVALID,
139         output   wire
M_AXI_BREADY,
140         // }}}
141         // AXI-lite read interface
142         // {{{
143         output    wire    [C_AXI_ADDR_WIDTH-1:0]
M_AXI_ARADDR,
144         output    wire    [2:0]
M_AXI_ARPROT,
145         output    wire
M_AXI_ARVALID,
146         input    wire
M_AXI_ARREADY,
147         //
148         input    wire
M_AXI_RVALID,
149         output    wire
M_AXI_RREADY,
150         input    wire    [C_AXI_DATA_WIDTH-1 : 0]
M_AXI_RDATA,
151         input    wire    [1 : 0]
M_AXI_RRESP
152         // }}}
153         // }}}
154         // }}}
155     );
156
157     // Local parameters, register, and net declarations
158     // {{{
159     localparam [1:0] SLVERR = 2'b10;
160     // localparam [1:0] OKAY = 2'b00,
161     //                  EXOKAY = 2'b01,
162     //                  DECERR = 2'b10;
163     localparam      AW = C_AXI_ADDR_WIDTH;
164     localparam      DW = C_AXI_DATA_WIDTH;
165     localparam      IW = C_AXI_ID_WIDTH;
166     // }}}
167     // Register declarations
168     // {{{
169     //
170     // Write registers
171     reg          m_axi_awvalid,
s_axi_wready;
172     reg          [C_AXI_ADDR_WIDTH-1:0] axi_awaddr;
173     reg          [7:0] axi_awlen, axi_blen;
174     reg          [1:0] axi_awburst;
175     reg          [2:0] axi_awsz;
176     wire         [C_AXI_ADDR_WIDTH-1:0] next_write_addr;
177     wire         [4:0] wfifo_count;
178     wire         wfifo_full;
179     wire         wfifo_empty;
180     wire         [7:0] wfifo_bcount;
181     wire         [IW-1:0] wfifo_bid;
182     reg          [8:0] bcounts;
183     reg          [C_AXI_ID_WIDTH-1:0] axi_bid, bid;
184     reg          [1:0] axi_bresp;
185     reg          s_axi_bvalid;
186     wire         read_from_wrfifo;
187     //
188     // Read register
189     reg          m_axi_arvalid;
190     wire         [4:0] rfifo_count;
191     wire         rfifo_full;
192     wire         rfifo_empty;
193     wire         [7:0] rfifo_rcount;
194     reg          s_axi_rvalid;
195     reg          s_axi_rresp;
196     reg          [8:0] rcounts;
197     reg          [C_AXI_ADDR_WIDTH-1:0] axi_araddr;
198     reg          [7:0] axi_arlen, axi_rlen;
199     reg          [1:0] axi_arburst;
200     reg          [2:0] axi_arsz;
201     wire         [C_AXI_ADDR_WIDTH-1:0] next_read_addr;
202     reg          [C_AXI_ID_WIDTH-1:0] s_axi_rid;
203     wire         [C_AXI_ID_WIDTH-1:0] rfifo_rid;
204     reg          [C_AXI_DATA_WIDTH-1:0] s_axi_rdata;
205     reg          s_axi_rlast;
206     reg          [IW-1:0] rid;
207     wire         read_from_rdfifo;
208
209     //
210     // S_AXI_AW* skid buffer
211     wire         skids_awvalid, skids_awready;
212     wire         [IW-1:0] skids_awid;
213     wire         [AW-1:0] skids_awaddr;
214     wire         [7:0] skids_awlen;
215     wire         [2:0] skids_awsz;
216     wire         [1:0] skids_awburst;
217     //
218     // S_AXI_W* skid buffer
219     wire         skids_wvalid, skids_wready,
skids_wlast;
220     wire         [DW-1:0] skids_wdata;
221     wire         [DW/8-1:0] skids_wstrb;

```

```

222     //
223     // S_AXI_B* skid buffer isn't needed
224     //
225     // M_AXI_AW* skid buffer isn't needed
226     //
227     // M_AXI_W* skid buffer
228     wire         skidm_wvalid, skidm_wready;
229     wire         [DW-1:0] skidm_wdata;
230     wire         [DW/8-1:0] skidm_wstrb;
231     //
232     // M_AXI_B* skid buffer
233     wire         skidm_bvalid, skidm_bready;
234     wire         [1:0] skidm_bresp;
235     //
236     //
237     //
238     // S_AXI_AR* skid buffer
239     wire         skids_arvalid, skids_arready;
240     wire         [IW-1:0] skids_arid;
241     wire         [AW-1:0] skids_araddr;
242     wire         [7:0] skids_arlen;
243     wire         [2:0] skids_arsz;
244     wire         [1:0] skids_arburst;
245     //
246     // S_AXI_R* skid buffer isn't needed
247     //
248     // M_AXI_AR* skid buffer isn't needed
249     // M_AXI_R* skid buffer
250     wire         skidm_rvalid, skidm_rready;
251     wire         [DW-1:0] skidm_rdata;
252     wire         [1:0] skidm_rresp;
253     // }}}
254     ///////////////////////////////////////////////////
255     //
256     // Write logic
257     // {{{
258     ///////////////////////////////////////////////////
259     //
260     //
261     generate if (OPT_WRITES)
262     begin : IMPLEMENT_WRITES
263         // {{{
264         // The write address channel's skid buffer
265         // {{{
266         skidbuffer #(
267             // {{{
268             .DW(IW+AW+8+3+2), .OPT_LOW-
POWER(0), .OPT_OUTREG(0)
269             // }}}
270         ) awskid(
271             // {{{
272             S_AXI_ACLK, !S_AXI_ARESETN,
273             S_AXI_AWVALID, S_AXI_AWREADY,
274             { S_AXI_AWID, S_AXI_AWADDR,
S_AXI_AWLEN, S_AXI_AWSZ,
275             S_AXI_AWBURST },
276             skids_awvalid, skids_awready,
277             { skids_awid, skids_awaddr,
skids_awlen, skids_awsz,
278             skids_awburst }
279             // }}}
280         );
281         // }}}
282         //
283         // The write data channel's skid buffer
284         (S_AXI_W*)
285         // {{{
286         skidbuffer #(
287             // {{{
288             .DW(DW+DW/8+1), .OPT_LOW-
POWER(0), .OPT_OUTREG(0)
289             // }}}
290         ) wskid(
291             // {{{
292             S_AXI_ACLK, !S_AXI_ARESETN,
293             S_AXI_WVALID, S_AXI_WREADY,
294             { S_AXI_WDATA, S_AXI_WSTRB, S_AXI_W-
LAST },
295             skids_wvalid, skids_wready,
296             { skids_wdata, skids_wstrb,
skids_wlast }
297             // }}}
298         );
299         // }}}
300         // The downstream AXI-lite write data
301         (M_AXI_W*) skid buffer
302         // {{{
303         skidbuffer #(
304             // {{{
305             .DW(DW+DW/8), .OPT_LOW-
POWER(0), .OPT_OUTREG(1)
306             // }}}
307         ) mwskid(
308             // {{{
309             S_AXI_ACLK, !S_AXI_ARESETN,
310             skidm_wvalid, skidm_wready,
{ skidm_wdata, skidm_wstrb },
311             M_AXI_WVALID, M_AXI_WREADY,

```

```

{ M_AXI_WDATA, M_AXI_WSTRB }
311 // }}}
312 );
313 // }}}
314 //
315 // The downstream AXI-lite response (M_AXI_B*)
skid buffer
316 // {{{
317 skidbuffer #(
318 // {{{
319 .DW(2), .OPT_LOW-
POWER(0), .OPT_OUTREG(0)
320 // }}}
321 ) bskid(
322 // {{{
323 S_AXI_ACLK, !S_AXI_ARESETN,
324 M_AXI_BVALID, M_AXI_BREADY,
{ M_AXI_BRESP },
325 skidm_bvalid, skidm_bready,
{ skidm_bresp }
326 // }}}
327 );
328 // }}}
329
330 // m_axi_awvalid
331 // {{{
332 initial m_axi_awvalid = 0;
333 always @(posedge S_AXI_ACLK)
334 if (!S_AXI_ARESETN)
335     m_axi_awvalid <= 0;
336 else if (skids_awvalid & skids_awready)
337     m_axi_awvalid <= 1;
338 else if (M_AXI_AWREADY && axi_awlen == 0)
339     m_axi_awvalid <= 0;
340
341 assign M_AXI_AWVALID = m_axi_awvalid;
342 // }}}
343
344 // skids_awready
345 // {{{
346 assign skids_awready = (!M_AXI_AWVALID
347 || ((axi_awlen ==
0)&&M_AXI_AWREADY))
348 && !wfifo_full
349 &&(!s_axi_wready ||
(skids_wvalid && skids_wlast && skids_wready));
350 // }}}
351
352 // Address processing
353 // {{{
354 always @(posedge S_AXI_ACLK)
355 if (skids_awvalid && skids_awready)
356 begin
357     axi_awaddr <= skids_awaddr;
358     axi_blen <= skids_awlen;
359     axi_awburst <= skids_awburst;
360     axi_awsz <= skids_awsz;
361 end else if (M_AXI_AWVALID && M_AXI_AWREADY)
362     axi_awaddr <= next_write_addr;
363 // }}}
364
365 // axi_awlen
366 // {{{
367 initial axi_awlen = 0;
368 always @(posedge S_AXI_ACLK)
369 if (!S_AXI_ARESETN)
370     axi_awlen <= 0;
371 else if (skids_awvalid && skids_awready)
372     axi_awlen <= skids_awlen;
373 else if (M_AXI_AWVALID && M_AXI_AWREADY &&
axi_awlen > 0)
374     axi_awlen <= axi_awlen - 1;
375 // }}}
376
377 // axi_addr
378 // {{{
379 axi_addr #(
380 // {{{
381 .AW(C_AXI_AD-
DR_WIDTH), .DW(C_AXI_DATA_WIDTH)
382 // }}}
383 ) calcwaddr(
384 // {{{
385 axi_awaddr, axi_awsz, axi_awburst,
386 axi_blen, next_write_addr
387 // }}}
388 );
389 // }}}
390
391 // s_axi_wready
392 // {{{
393 // We really don't need to do anything special
to the write
394 // channel.
395 initial s_axi_wready = 0;
396 always @(posedge S_AXI_ACLK)
397 if (!S_AXI_ARESETN)
398     s_axi_wready <= 0;
399 else if (skids_awvalid && skids_awready)
400     s_axi_wready <= 1;
401 else if (skids_wvalid && skids_wready &&

```

```

skids_wlast)
402 s_axi_wready <= 0;
403 // }}}
404
405 // skidm*, and read_from_wrfifo
406 // {{{
407 assign skidm_wdata = skids_wdata;
408 assign skidm_wstrb = skids_wstrb;
409 assign skidm_wvalid = skids_wvalid &&
s_axi_wready;
410 assign skids_wready = s_axi_wready &&
skidm_wready;
411
412 assign read_from_wrfifo = (bcounts <= 1)&&(!
wfifo_empty)
413 &&(skidm_bvalid && skidm_bready);
414 // }}}
415
416 // BFIFO
417 // {{{
418 sfifo #(
419 .BW(C_AXI_ID_WIDTH+8), .LGFLN(LGFIFO)
420 ) bidlnfifo(
421 S_AXI_ACLK, !S_AXI_ARESETN,
422 skids_awvalid && skids_awready,
423 { skids_awid, skids_awlen },
424 wfifo_full, wfifo_count,
425 read_from_wrfifo,
426 { wfifo_bid, wfifo_bcount },
wfifo_empty);
427 // }}}
428
429 // bcounts
430 // {{{
431 // Return counts
432 initial bcounts = 0;
433 always @(posedge S_AXI_ACLK)
434 if (!S_AXI_ARESETN)
435     bcounts <= 0;
436 else if (read_from_wrfifo)
437 begin
438     bcounts <= wfifo_bcount + bcounts;
439 end else if (skidm_bvalid && skidm_bready)
440     bcounts <= bcounts - 1;
441 // }}}
442
443 // bid
444 // {{{
445 always @(posedge S_AXI_ACLK)
446 if (read_from_wrfifo)
447     bid <= wfifo_bid;
448
449 always @(posedge S_AXI_ACLK)
450 if (!S_AXI_BVALID || S_AXI_BREADY)
451     axi_bid <= (read_from_wrfifo &&
bcounts==0) ? wfifo_bid : bid;
452 // }}}
453
454 // s_axi_bvalid
455 // {{{
456 initial s_axi_bvalid = 0;
457 always @(posedge S_AXI_ACLK)
458 if (!S_AXI_ARESETN)
459     s_axi_bvalid <= 0;
460 else if (skidm_bvalid && skidm_bready)
461     s_axi_bvalid <= (bcounts == 1)
462 ||((bcounts == 0) && (!
wfifo_empty) && (wfifo_bcount == 0));
463 else if (S_AXI_BREADY)
464     s_axi_bvalid <= 0;
465 // }}}
466
467 // axi_bresp
468 // {{{
469 initial axi_bresp = 0;
470 always @(posedge S_AXI_ACLK)
471 if (!S_AXI_ARESETN)
472     axi_bresp <= 0;
473 else if (S_AXI_BVALID && S_AXI_BREADY)
474 begin
475     if (skidm_bvalid && skidm_bready)
476         axi_bresp <= skidm_bresp;
477     else
478         axi_bresp <= 0;
479 end else if (skidm_bvalid && skidm_bready)
480 begin
481     // Let SLVERR take priority over DECERR
482     casez({ S_AXI_BRESP, skidm_bresp })
483     4'b??0?: axi_bresp <= S_AXI_BRESP;
484     4'b0?1?: axi_bresp <= skidm_bresp;
485     4'b1?10: axi_bresp <= SLVERR;
486     4'b1011: axi_bresp <= SLVERR;
487     4'b1111: axi_bresp <= skidm_bresp;
488     endcase
489 end
490 // /}}}
491
492 // M_AXI_AW*
493 // {{{
494 assign M_AXI_AWVALID = m_axi_awvalid;
495 assign M_AXI_AWADDR = axi_awaddr;

```

```

496         assign M_AXI_AWPROT = 0;
497         // }}}
498
499         // skidm_bready, S_AXI_B*
500         // {{{
501         assign skidm_bready = ((bcounts > 0) || (!
wfifo_empty)) && (!S_AXI_BVALID | S_AXI_BREADY);
502         assign S_AXI_BID = axi_bid;
503         assign S_AXI_BRESP = axi_bresp;
504         assign S_AXI_BVALID = s_axi_bvalid;
505         // }}}
506         // }}}
507     end else begin : NO_WRITE_SUPPORT
508         // {{{
509         assign S_AXI_AWREADY = 0;
510         assign S_AXI_WREADY = 0;
511         assign S_AXI_BID = 0;
512         assign S_AXI_BRESP = 2'b11;
513         assign S_AXI_BVALID = 0;
514         assign S_AXI_BID = 0;
515
516         //
517         assign M_AXI_AWVALID = 0;
518         assign M_AXI_AWADDR = 0;
519         assign M_AXI_AWPROT = 0;
520         //
521         assign M_AXI_WVALID = 0;
522         assign M_AXI_WDATA = 0;
523         assign M_AXI_WSTRB = 0;
524         //
525         assign M_AXI_BREADY = 0;
526
527         //
528         // S_AXI_AW* skid buffer
529         assign skids_awvalid = 0;
530         assign skids_awready = 0;
531         assign skids_awid = 0;
532         assign skids_awaddr = 0;
533         assign skids_awlen = 0;
534         assign skids_awsz = 0;
535         assign skids_awburst = 0;
536         //
537         // S_AXI_W* skid buffer
538         assign skids_wvalid = S_AXI_WVALID;
539         assign skids_wready = S_AXI_WREADY;
540         assign skids_wdata = S_AXI_WDATA;
541         assign skids_wstrb = S_AXI_WSTRB;
542         assign skids_wlast = S_AXI_WLAST;
543         //
544         // S_AXI_B* skid buffer isn't needed
545         //
546         // M_AXI_AW* skid buffer isn't needed
547         //
548         // M_AXI_W* skid buffer
549         assign skidm_wvalid = M_AXI_WVALID;
550         assign skidm_wready = M_AXI_WREADY;
551         assign skidm_wdata = M_AXI_WDATA;
552         assign skidm_wstrb = M_AXI_WSTRB;
553         //
554         // M_AXI_B* skid buffer
555         assign skidm_bvalid = M_AXI_BVALID;
556         assign skidm_bready = M_AXI_BREADY;
557         assign skidm_bresp = M_AXI_BRESP;
558         //
559         //
560         always @(*)
561         begin
562             s_axi_wready = 0;
563
564             axi_awlen = 0;
565             bcounts = 0;
566             bid = 0;
567             axi_bresp = 0;
568             axi_bid = 0;
569
570         end
571
572         assign wfifo_full = 0;
573         assign wfifo_empty = 1;
574         assign wfifo_count = 0;
575         assign read_from_wrfifo = 0;
576
577         // Make Verilator happy
578         // {{{
579         // Verilator lint_off UNUSED
580         wire unused_write_signals;
581         assign unused_write_signals = &{ 1'b0,
M_AXI_AWREADY,
582             M_AXI_WREADY, S_AXI_BREADY };
583         // Verilator lint_on UNUSED
584         // }}}
585         // }}}
586     end endgenerate
587     // }}}
588     // }}}
589     ///////////////////////////////////////////////////////////////////
590     //
591     // Read logic
592     // {{{
593     ///////////////////////////////////////////////////////////////////

```

```

///////////////////////////////////////////////////////////////////
594     //
595     //
596     generate if (OPT_READS)
597     begin : IMPLEMENT_READS
598         // {{{
599         // S_AXI_AR* skid buffer
600         // {{{
601         skidbuffer #(
602             // {{{
603             .DW(IW+AW+8+3+2), .OPT_LOW-
POWER(0), .OPT_OUTREG(0)
604             // }}}
605             ) arskid(
606             // {{{
607             S_AXI_ACLK, !S_AXI_ARESETN,
608             S_AXI_ARVALID, S_AXI_ARREADY,
609             { S_AXI_ARID, S_AXI_ARADDR, S_AXI_AR-
LEN, S_AXI_ARSIZE,
610             S_AXI_ARBURST },
611             skids_arvalid, skids_arready,
612             { skids_arid, skids_araddr,
skids_arburst }
613             // }}}
614             );
615         // }}}
616         // M_AXI_R* skid buffer
617         // {{{
618         skidbuffer #(
619             // {{{
620             .DW(DW+2), .OPT_LOW-
POWER(0), .OPT_OUTREG(0)
621             // }}}
622             ) rskid(
623             // {{{
624             S_AXI_ACLK, !S_AXI_ARESETN,
625             M_AXI_RVALID, M_AXI_RREADY,
626             { M_AXI_RDATA, M_AXI_RRESP },
627             skidm_rvalid, skidm_rready,
628             { skidm_rdata, skidm_rresp }
629             // }}}
630             );
631         // }}}
632         // m_axi_arvalid
633         // {{{
634         initial m_axi_arvalid = 0;
635         always @(posedge S_AXI_ACLK)
636         if (!S_AXI_ARESETN)
637             m_axi_arvalid <= 0;
638         else if (skids_arvalid && skids_arready)
639             m_axi_arvalid <= 1;
640         else if (M_AXI_ARREADY && axi_arlen == 0)
641             m_axi_arvalid <= 0;
642         // }}}
643         // Read address processing
644         // {{{
645         always @(posedge S_AXI_ACLK)
646         if (skids_arvalid && skids_arready)
647         begin
648             axi_araddr <= skids_araddr;
649             axi_arburst <= skids_arburst;
650             axi_arsize <= skids_arsize;
651             axi_rlen <= skids_arlen;
652         end else if (M_AXI_ARREADY)
653         begin
654             axi_araddr <= next_read_addr;
655             if (OPT_LOWPPOWER && axi_arlen == 0)
656                 axi_araddr <= 0;
657         end
658
659         axi_addr #(
660             // {{{
661             .AW(C_AXI_AD-
DR_WIDTH), .DW(C_AXI_DATA_WIDTH)
662             // }}}
663             ) calcrdaddr(
664             // {{{
665             axi_araddr, axi_arsize, axi_arburst,
666             axi_rlen, next_read_addr
667             // }}}
668             );
669         // }}}
670
671         // axi_arlen, Read length processing
672         // {{{
673         initial axi_arlen = 0;
674         always @(posedge S_AXI_ACLK)
675         if (!S_AXI_ARESETN)
676             axi_arlen <= 0;
677         else if (skids_arvalid && skids_arready)
678             axi_arlen <= skids_arlen;
679         else if (M_AXI_ARVALID && M_AXI_ARREADY &&
axi_arlen > 0)
680             axi_arlen <= axi_arlen - 1;
681         // }}}
682
683         assign skids_arready = (!M_AXI_ARVALID ||
((axi_arlen == 0) && M_AXI_AR-
READY))

```

```

685          && !rfifo_full;
686
687      assign read_from_rdfifo = skidm_rvalid &&
skidm_rready
688          && (rcounts <= 1) && !
rfifo_empty;
689
690      // Read ID FIFO
691      // {{{
692      sfifo #(
693          // {{{
694          .BW(C_AXI_ID_WIDTH+8), .LGFLN(LGFIFO)
695          // }}}
696      ) ridInfifo(
697          // {{{
698          S_AXI_ACLK, !S_AXI_ARESETN,
699          skids_arvalid && skids_arready,
700          { skids_arid, skids_arlen },
701          rfifo_full, rfifo_count,
702          read_from_rdfifo,
703          { rfifo_rid, rfifo_rcount },
rfifo_empty
704          // }}}
705      );
706      // }}}
707
708      assign skidm_rready = (!S_AXI_RVALID ||
S_AXI_RREADY);
709
710      // s_axi_rvalid
711      // {{{
712      initial s_axi_rvalid = 0;
713      always @(posedge S_AXI_ACLK)
714      if (!S_AXI_ARESETN)
715          s_axi_rvalid <= 0;
716      else if (skidm_rvalid && skidm_rready)
717          s_axi_rvalid <= 1;
718      else if (S_AXI_RREADY)
719          s_axi_rvalid <= 0;
720      // }}}
721
722      // s_axi_rresp, s_axi_rdata
723      // {{{
724      always @(posedge S_AXI_ACLK)
725      if (skidm_rvalid && skidm_rready)
726      begin
727          s_axi_rresp <= skidm_rresp;
728          s_axi_rdata <= skidm_rdata;
729      end else if (S_AXI_RREADY)
730      begin
731          s_axi_rresp <= 0;
732          s_axi_rdata <= 0;
733      end
734      // }}}
735
736      // rcounts, Return counts
737      // {{{
738      initial rcounts = 0;
739      always @(posedge S_AXI_ACLK)
740      if (!S_AXI_ARESETN)
741          rcounts <= 0;
742      else if (read_from_rdfifo)
743          rcounts <= rfifo_rcount + rcounts;
744      else if (skidm_rvalid && skidm_rready)
745          rcounts <= rcounts - 1;
746      // }}}
747
748      // rid
749      // {{{
750      initial rid = 0;
751      always @(posedge S_AXI_ACLK)
752      if (read_from_rdfifo)
753          rid <= rfifo_rid;
754      // }}}
755
756      // s_axi_rlast
757      // {{{
758      always @(posedge S_AXI_ACLK)
759      if (!S_AXI_RVALID || S_AXI_RREADY)
760      begin
761          // if (rcounts == 1) s_axi_rlast <= 1;
else
762          if (read_from_rdfifo)
763              s_axi_rlast <= (rfifo_rcount ==
0);
764          else
765              s_axi_rlast <= 0;
766
767          if (rcounts == 1)
768              s_axi_rlast <= 1;
769      end
770      // }}}
771
772      // s_axi_rid
773      // {{{
774      initial s_axi_rid = 0;
775      always @(posedge S_AXI_ACLK)
776      if ((S_AXI_RVALID && S_AXI_RREADY && S_AXI_R-
LAST)
777          || (!S_AXI_RVALID && rcounts ==

```

```

0))
778          s_axi_rid <=
(read_from_rdfifo)&&(rcounts == 0)?rfifo_rid : rid;
779          // }}}
780
781          // M_AXI_AR*
782          // {{{
783          assign M_AXI_ARVALID= m_axi_arvalid;
784          assign M_AXI_ARADDR = axi_araddr;
785          assign M_AXI_ARPROT = 0;
786          // }}}
787          // S_AXI_R*
788          // {{{
789          assign S_AXI_RVALID = s_axi_rvalid;
790          assign S_AXI_RDATA = s_axi_rdata;
791          assign S_AXI_RRESP = s_axi_rresp;
792          assign S_AXI_RLAST = s_axi_rlast;
793          assign S_AXI_RID = s_axi_rid;
794          // }}}
795          // }}}
796      end else begin : NO_READ_SUPPORT // if (!OPT_READS)
797          // {{{
798          assign M_AXI_ARVALID= 0;
799          assign M_AXI_ARADDR = 0;
800          assign M_AXI_ARPROT = 0;
801          assign M_AXI_RREADY = 0;
802          //
803          assign S_AXI_ARREADY= 0;
804          assign S_AXI_RVALID = 0;
805          assign S_AXI_RDATA = 0;
806          assign S_AXI_RRESP = 0;
807          assign S_AXI_RLAST = 0;
808          assign S_AXI_RID = 0;
809
810          //
811          assign skids_arvalid = S_AXI_ARVALID;
812          assign skids_arready = S_AXI_ARREADY;
813          assign skids_arid = S_AXI_ARID;
814          assign skids_araddr = S_AXI_ARADDR;
815          assign skids_arlen = S_AXI_ARLEN;
816          assign skids_arsize = S_AXI_ARSIZE;
817          assign skids_arburst = S_AXI_ARBURST;
818          //
819          assign skidm_rvalid = M_AXI_RVALID;
820          assign skidm_rready = M_AXI_RREADY;
821          assign skidm_rdata = M_AXI_RDATA;
822          assign skidm_rresp = M_AXI_RRESP;
823          //
824          //
825          always @(*)
826          begin
827              axi_arlen = 0;
828
829              rcounts = 0;
830              rid = 0;
831
832          end
833          assign rfifo_empty = 1;
834          assign rfifo_full = 0;
835          assign rfifo_count = 0;
836
837          // Make Verilator happy
838          // {{{
839          // Verilator lint_off UNUSED
840          wire unused_read_signals;
841          assign unused_read_signals = &{ 1'b0,
M_AXI_ARREADY,
842              S_AXI_RREADY };
843          // Verilator lint_on UNUSED
844          // }}}
845
846          // }}}
847      end endgenerate
848      // }}}
849      // Make Verilator happy
850      // {{{
851      // Verilator lint_off UNUSED
852      wire [35-1:0]unused;
853      assign unused = {
854          S_AXI_AWLOCK, S_AXI_AWCACHE, S_AXI_AWPROT,
S_AXI_AWQOS,
855          skids_wlast, wfifo_count,
856          S_AXI_ARLOCK, S_AXI_ARCACHE, S_AXI_ARPROT,
S_AXI_ARQOS,
857          rfifo_count };
858          // Verilator lint_on UNUSED
859          // }}}
860          // }}}
861          // }}}
862          // }}}
863          //
864          // Formal properties
865          // {{{
866          // The following are a subset of the formal properties used to
verify this core
867          // }}}

```



```

868 //////////////////////////////////////
869 //////////////////////////////////////
870 `ifdef FORMAL
871     localparam      F_LGDEPTH = LGFIFO+1+8;
872
873     //
874     // ...
875     //
876
877 //////////////////////////////////////
878 //
879 // AXI channel properties
880 // {{{
881 //////////////////////////////////////
882 //
883 //
884 faxi_slave #(.C_AXI_ID_WIDTH(IW),
885             .C_AXI_DATA_WIDTH(DW),
886             .C_AXI_ADDR_WIDTH(AW),
887             )
888     faxi(.i_clk(S_AXI_ACLK),
889         .i_axi_reset_n(S_AXI_ARESETN),
890         // Write address
891         .i_axi_awready(skids_awready),
892         .i_axi_awid( skids_awid),
893         .i_axi_awaddr( skids_awaddr),
894         .i_axi_awlen( skids_awlen),
895         .i_axi_awsz( skids_awsz),
896         .i_axi_awburst(skids_awburst),
897         .i_axi_awlock( 0),
898         .i_axi_awcache(0),
899         .i_axi_awprot( 0),
900         .i_axi_awqos( 0),
901         .i_axi_awvalid(skids_awvalid),
902         // Write data
903         .i_axi_wready( skids_wready),
904         .i_axi_wdata( skids_wdata),
905         .i_axi_wstrb( skids_wstrb),
906         .i_axi_wlast( skids_wlast),
907         .i_axi_wvalid( skids_wvalid),
908         // Write return response
909         .i_axi_bid( S_AXI_BID),
910         .i_axi_bresp( S_AXI_BRESP),
911         .i_axi_bvalid( S_AXI_BVALID),
912         .i_axi_bready( S_AXI_BREADY),
913         // Read address
914         .i_axi_arready(skids_arready),
915         .i_axi_arid( skids_arid),
916         .i_axi_araddr( skids_araddr),
917         .i_axi_arlen( skids_arlen),
918         .i_axi_arsize( skids_arsize),
919         .i_axi_arburst(skids_arburst),
920         .i_axi_arlock( 0),
921         .i_axi_arcache(0),
922         .i_axi_arprot( 0),
923         .i_axi_arqos( 0),
924         .i_axi_arvalid(skids_arvalid),
925         // Read response
926         .i_axi_rid( S_AXI_RID),
927         .i_axi_rresp( S_AXI_RRESP),
928         .i_axi_rvalid( S_AXI_RVALID),
929         .i_axi_rdata( S_AXI_RDATA),
930         .i_axi_rlast( S_AXI_RLAST),
931         .i_axi_rready( S_AXI_RREADY),
932         //
933         // Formal property data
934         .f_axi_awr_n-
935 bursts( faxi_awr_nbursts),
936         .f_axi_wr_pending( faxi_wr_pending),
937         .f_axi_rd_nbursts( faxi_rd_nbursts),
938         .f_axi_rd_outstand-
939 ing(faxi_rd_outstanding),
940         //
941         // ...
942         );
943     // }}}
944 //////////////////////////////////////
945 //
946 // AXI-lite properties
947 // {{{
948 //////////////////////////////////////
949 //
950 //
951 faxil_master
952 #(.C_AXI_DATA_WIDTH(DW), .C_AXI_ADDR_WIDTH(AW),
953     .F_OPT_NO_RESET(1),
954     .F_AXI_MAXWAIT(5),
955     .F_AXI_MAXDELAY(4),
956     .F_AXI_MAXRSTALL(0),
957     .F_OPT_WRITE_ONLY(OPT_WRITES && !
958 OPT_READS),
959     .F_OPT_READ_ONLY(!OPT_WRITES &&
960 OPT_READS),
961     .F_LGDEPTH(F_AXIL_LGDEPTH))

```

```

958         faxil(.i_clk(S_AXI_ACLK),
959             .i_axi_reset_n(S_AXI_ARESETN),
960             // Write address channel
961             .i_axi_awvalid(M_AXI_AWVALID),
962             .i_axi_awready(M_AXI_AWREADY),
963             .i_axi_aawaddr( M_AXI_AWADDR),
964             .i_axi_awprot( M_AXI_ARPROT),
965             // Write data
966             .i_axi_wready( skidm_wready),
967             .i_axi_wdata( skidm_wdata),
968             .i_axi_wstrb( skidm_wstrb),
969             .i_axi_wvalid( skidm_wvalid),
970             // Write response
971             .i_axi_bresp( skidm_bresp),
972             .i_axi_bvalid( skidm_bvalid),
973             .i_axi_bready( skidm_bready),
974             // Read address
975             .i_axi_arvalid(M_AXI_ARVALID),
976             .i_axi_arready(M_AXI_ARREADY),
977             .i_axi_araddr( M_AXI_ARADDR),
978             .i_axi_arprot( M_AXI_ARPROT),
979             // Read data return
980             .i_axi_rvalid( skidm_rvalid),
981             .i_axi_rready( skidm_rready),
982             .i_axi_rdata( skidm_rdata),
983             .i_axi_rresp( skidm_rresp),
984             //
985             // Formal check variables
986             .f_axi_rd_outstand-
ing(faxil_rd_outstanding),
987             .f_axi_wr_outstand-
ing(faxil_wr_outstanding),
988             .f_axi_aws_outstand-
ing(faxil_aws_outstanding));
989     // }}}
990     ////////////////////////////////////////
991     //
992     // Assume that the two write channels stay within an
appropriate
993     // distance of each other. This is to make certain
that the property
994     // file features are not violated, although not neces-
sary true for
995     // actual operation
996     //
997     always @(*)
998         assert(s_axi_wready == (OPT_WRITES &&
faxi_wr_pending > 0));
999
1000     ////////////////////////////////////////
1001     //
1002     // Write induction properties
1003     // {{{
1004     ////////////////////////////////////////
1005     //
1006     // These are extra properties necessary to pass write
induction
1007     //
1008
1009     always @(*)
1010     if ((bcounts == 0)&&!read_from_wrfifo))
1011         assert(!skidm_bvalid || !skidm_bready);
1012
1013     always @(*)
1014     if (axi_awlen > 0)
1015     begin
1016         assert(m_axi_awvalid);
1017         if (axi_awlen > 1)
1018             begin
1019                 assert(!skids_awready);
1020             end else if (wfifo_full)
1021             begin
1022                 assert(!skids_awready);
1023             end else if (M_AXI_AWVALID && !M_AXI_AWREADY)
1024                 assert(!skids_awready);
1025     end
1026
1027
1028     always @(*)
1029     assert(axi_bresp != EXOKAY);
1030
1031     reg [F_LGDEPTH-1:0] f_wfifo_bursts, f_w-
fifobursts_minus_one,
1032         f_wfifo_within,
1033         f_fwid_bursts, f_w-
fiid_bursts_minus_one;
1034     reg [IW-1:0]f_await;
1035     always @(posedge S_AXI_ACLK)
1036     if (skids_awvalid && skids_awready)
1037         f_await = skids_await;
1038     // }}}
1039     ////////////////////////////////////////
1040     //
1041     // Read induction properties
1042     // {{{
1043     ////////////////////////////////////////

```



```

1044 //
1045 //
1046
1047 always @(*)
1048 if (!S_AXI_RVALID && rcounts > 0)
1049     assert(rid == S_AXI_RID);
1050
1051 always @(*)
1052 if (S_AXI_RVALID && !S_AXI_RLAST)
1053     assert(rid == S_AXI_RID);
1054
1055 always @(*)
1056 if ((rcounts == 0)&&(!read_from_rdfifo))
1057     assert(!skidm_rvalid || !skidm_rready);
1058
1059 always @(*)
1060 if (axi_arlen > 0)
1061 begin
1062     assert(m_axi_arvalid);
1063     assert(!skids_arready);
1064 end
1065
1066 //
1067 // ...
1068 //
1069 // }}}
1070 ///////////////////////////////////////////////////////////////////
1071 //
1072 // Select only write or only read operation
1073 // {{{
1074 ///////////////////////////////////////////////////////////////////
1075 //
1076 //
1077 generate if (!OPT_WRITES)
1078 begin
1079     always @(*)
1080     begin
1081         assume(!skids_awvalid);
1082         assume(!skids_wvalid);
1083         assert(M_AXI_AWVALID == 0);
1084         assert(faxil_awr_outstanding == 0);
1085         assert(faxil_wr_outstanding == 0);
1086         assert(!skidm_bvalid);
1087         assert(!S_AXI_BVALID);
1088     end
1089 end endgenerate
1090
1091 generate if (!OPT_READS)
1092 begin
1093     always @(*)
1094     begin
1095         assume(!S_AXI_ARVALID);
1096         assert(M_AXI_ARVALID == 0);
1097         assert(faxil_rd_outstanding == 0);
1098     end
1099 end endgenerate
1100
1101 end endgenerate
1102 // }}}
1103 ///////////////////////////////////////////////////////////////////
1104 //
1105 // Cover statements, to show performance
1106 // {{{
1107 ///////////////////////////////////////////////////////////////////
1108 //
1109 //
1110 generate if (OPT_WRITES)
1111 begin
1112     // {{{
1113     reg [3:0] cvr_write_count,

```

```

cvr_write_count_simple;
1114
1115     initial cvr_write_count = 0;
1116     always @(posedge S_AXI_ACLK)
1117     if (!S_AXI_ARESETN)
1118         cvr_write_count_simple <= 0;
1119     else if (S_AXI_AWVALID && S_AXI_AWREADY &&
S_AXI_AWLEN == 0)
1120         cvr_write_count_simple <=
cvr_write_count_simple + 1;
1121
1122     initial cvr_write_count = 0;
1123     always @(posedge S_AXI_ACLK)
1124     if (!S_AXI_ARESETN)
1125         cvr_write_count <= 0;
1126     else if (S_AXI_AWVALID && S_AXI_AWREADY &&
S_AXI_AWLEN > 2)
1127         cvr_write_count <= cvr_write_count + 1;
1128
1129     always @(*)
1130         cover(cvr_write_count_simple > 6 && /
* ... */ !S_AXI_BVALID);
1131     always @(*)
1132         cover(cvr_write_count > 2 && /* ...
*/ !S_AXI_BVALID);
1133     // }}}
1134 end endgenerate
1135
1136 generate if (OPT_READS)
1137 begin
1138     // {{{
1139     reg [3:0] cvr_read_count,
cvr_read_count_simple;
1140
1141     initial cvr_read_count_simple = 0;
1142     always @(posedge S_AXI_ACLK)
1143     if (!S_AXI_ARESETN)
1144         cvr_read_count_simple <= 0;
1145     else if (S_AXI_ARVALID && S_AXI_ARREADY &&
S_AXI_ARLEN == 0)
1146         cvr_read_count_simple <=
cvr_read_count_simple + 1;
1147
1148     initial cvr_read_count = 0;
1149     always @(posedge S_AXI_ACLK)
1150     if (!S_AXI_ARESETN)
1151         cvr_read_count <= 0;
1152     else if (S_AXI_ARVALID && S_AXI_ARREADY &&
S_AXI_ARLEN > 2)
1153         cvr_read_count <= cvr_read_count + 1;
1154
1155     always @(*)
1156         cover(cvr_read_count_simple > 6 && /
* ... */ !S_AXI_RVALID);
1157     always @(*)
1158         cover(cvr_read_count > 2 && /* ... */ !
S_AXI_RVALID);
1159     // }}}
1160 end endgenerate
1161 // }}}
1162 ///////////////////////////////////////////////////////////////////
1163 //
1164 // ...
1165 // {{{
1166 ///////////////////////////////////////////////////////////////////
1167 //
1168 // }}}
1169 `undef BMC_ASSERT
1170 `endif
1171 // }}}
1172 endmodule

```

PCIe 코어

What this is. PCIe interface logic: transaction layer, data link layer, DMA.

Why it exists. Because **PCIe is the dominant system interconnect**, and because its layering shows how ordering rules, credit flow control and replay interact.

Undergraduate review · EECS concepts. Layered protocols; credit-based flow control; packet ordering rules; retry mechanisms.

Graduate · Practice note. PCIe ordering rules (posted, non-posted, completion) are a frequent source of subtle bugs. They are worth studying as a specification-reading exercise in their own right.

9 files • 5,198 lines

API — arbiter.v

Table 1819. alexforencich_verilog-pcie/rtl/arbiter.v — module / interface / package (1개)

이름	종류
arbiter	module

Table 1820. alexforencich_verilog-pcie/rtl/arbiter.v — parameter (5개)

파라미터	형/폭	기본값
PORTS		4
ARB_TYPE_ROUND_ROBIN		0
ARB_BLOCK		0
ARB_BLOCK_ACK		1
ARB_LSB_HIGH_PRIORITY		0

Table 1821. alexforencich_verilog-pcie/rtl/arbiter.v — 포트 (7개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
request	input	wire [PORTS-1:0]
acknowledge	input	wire [PORTS-1:0]
grant	output	wire [PORTS-1:0]
grant_valid	output	wire
PORTS	output	wire [\$clog2(

API — axis_arb_mux.v

Table 1822. alexforencich_verilog-pcie/rtl/axis_arb_mux.v — module / interface / package (1개)

이름	종류
axis_arb_mux	module

Table 1823. alexforencich_verilog-pcie/rtl/axis_arb_mux.v — parameter (17개)

파라미터	형/폭	기본값
S_COUNT		4
DATA_WIDTH		8
KEEP_ENABLE		(DATA_WIDTH>8)
KEEP_WIDTH		((DATA_WIDTH+7)/8)
ID_ENABLE		0
S_ID_WIDTH		8
M_ID_WIDTH		S_ID_WIDTH+\$clog2(S_COUNT)
DEST_ENABLE		0
DEST_WIDTH		8
USER_ENABLE		1
USER_WIDTH		1
LAST_ENABLE		1
UPDATE_TID		0
ARB_TYPE_ROUND_ROBIN		0
ARB_LSB_HIGH_PRIORITY		1
CL_S_COUNT		\$clog2(S_COUNT)
S_ID_WIDTH_INT		S_ID_WIDTH > 0 ? S_ID_WIDTH : 1

Table 1824. alexforencich_verilog-pcie/rtl/axis_arb_mux.v — 포트 (18개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_axis_tdata	input	wire [S_COUNT*DATA_WIDTH-1:0]
s_axis_tkeep	input	wire [S_COUNT*KEEP_WIDTH-1:0]
s_axis_tvalid	input	wire [S_COUNT-1:0]
s_axis_tready	output	wire [S_COUNT-1:0]
s_axis_tlast	input	wire [S_COUNT-1:0]
s_axis_tid	input	wire [S_COUNT*S_ID_WIDTH-1:0]
s_axis_tdest	input	wire [S_COUNT*DEST_WIDTH-1:0]
s_axis_tuser	input	wire [S_COUNT*USER_WIDTH-1:0]
m_axis_tdata	output	wire [DATA_WIDTH-1:0]
m_axis_tkeep	output	wire [KEEP_WIDTH-1:0]

포트	방향	형/폭
m_axis_tvalid	output	wire
m_axis_tready	input	wire
m_axis_tlast	output	wire
m_axis_tid	output	wire [M_ID_WIDTH-1:0]
m_axis_tdest	output	wire [DEST_WIDTH-1:0]
m_axis_tuser	output	wire [USER_WIDTH-1:0]

API — dma_client_axis_sink.v

Table 1825. alexforencich_verilog-pcie/rtl/dma_client_axis_sink.v — module / interface / package (1개)

이름	종류
dma_client_axis_sink	module

Table 1826. alexforencich_verilog-pcie/rtl/dma_client_axis_sink.v — parameter (33개)

파라미터	형/폭	기본값
RAM_ADDR_WIDTH		16
SEG_COUNT		2
SEG_DATA_WIDTH		64
SEG_BE_WIDTH		SEG_DATA_WIDTH/8
SEG_ADDR_WIDTH		RAM_ADDR_WIDTH-\$clog2(SEG_COUNT*SEG_BE_WIDTH)
AXIS_DATA_WIDTH		SEG_DATA_WIDTH*SEG_COUNT/2
AXIS_KEEP_ENABLE		(AXIS_DATA_WIDTH>8)
AXIS_KEEP_WIDTH		(AXIS_DATA_WIDTH/8)
AXIS_LAST_ENABLE		1
AXIS_ID_ENABLE		0
AXIS_ID_WIDTH		8
AXIS_DEST_ENABLE		0
AXIS_DEST_WIDTH		8
AXIS_USER_ENABLE		1
AXIS_USER_WIDTH		1
LEN_WIDTH		16
TAG_WIDTH		8
RAM_WORD_WIDTH		SEG_BE_WIDTH
RAM_WORD_SIZE		SEG_DATA_WIDTH/RAM_WORD_WIDTH
AXIS_KEEP_WIDTH_INT		AXIS_KEEP_ENABLE ? AXIS_KEEP_WIDTH : 1
AXIS_WORD_WIDTH		AXIS_KEEP_WIDTH_INT
AXIS_WORD_SIZE		AXIS_DATA_WIDTH/AXIS_WORD_WIDTH

파라미터	형/폭	기본값
PART_COUNT		SEG_COUNT*SEG_BE_WIDTH / AXIS_KEEP_WIDTH_INT
PART_COUNT_WIDTH		PART_COUNT > 1 ? \$clog2(PART_COUNT) : 1
PART_OFFSET_WIDTH		AXIS_KEEP_WIDTH_INT > 1 ? \$clog2(AXIS_KEEP_WIDTH_INT) : 1
PARTS_PER_SEG		(SEG_BE_WIDTH + AXIS_KEEP_WIDTH_INT - 1) / AXIS_KEEP_WIDTH_I...
SEGS_PER_PART		(AXIS_KEEP_WIDTH_INT + SEG_BE_WIDTH - 1) / SEG_BE_WIDTH
OFFSET_WIDTH		AXIS_KEEP_WIDTH_INT > 1 ? \$clog2(AXIS_KEEP_WIDTH_INT) : 1
OFFSET_MASK		AXIS_KEEP_WIDTH_INT > 1 ? {OFFSET_WIDTH{1'b1}} : 0
ADDR_MASK		{RAM_ADDR_WIDTH{1'b1}} << \$clog2(AXIS_KEEP_WIDTH_INT)
CYCLE_COUNT_WIDTH		LEN_WIDTH - \$clog2(AXIS_KEEP_WIDTH_INT) + 1
STATUS_FIFO_ADDR_WIDTH		5
OUTPUT_FIFO_ADDR_WIDTH		5

Table 1827. alexforencich_verilog-pcie/rtl/dma_client_axis_sink.v — localparam (1개)

상수	형/폭	값
STATE_IDLE	[1:0]	2'd0

Table 1828. alexforencich_verilog-pcie/rtl/dma_client_axis_sink.v — 포트 (30개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_axis_write_desc_ram_addr	input	wire [RAM_ADDR_WIDTH-1:0]
s_axis_write_desc_len	input	wire [LEN_WIDTH-1:0]
s_axis_write_desc_tag	input	wire [TAG_WIDTH-1:0]
s_axis_write_desc_valid	input	wire
s_axis_write_desc_ready	output	wire
m_axis_write_desc_status_len	output	wire [LEN_WIDTH-1:0]
m_axis_write_desc_status_tag	output	wire [TAG_WIDTH-1:0]
m_axis_write_desc_status_id	output	wire [AXIS_ID_WIDTH-1:0]
m_axis_write_desc_status_dest	output	wire [AXIS_DEST_WIDTH-1:0]
m_axis_write_desc_status_user	output	wire [AXIS_USER_WIDTH-1:0]
m_axis_write_desc_status_error	output	wire [3:0]
m_axis_write_desc_status_valid	output	wire
s_axis_write_data_tdata	input	wire [AXIS_DATA_WIDTH-1:0]
s_axis_write_data_tkeep	input	wire [AXIS_KEEP_WIDTH-1:0]
s_axis_write_data_tvalid	input	wire
s_axis_write_data_tready	output	wire

포트	방향	형/폭
s_axis_write_data_tlast	input	wire
s_axis_write_data_tid	input	wire [AXIS_ID_WIDTH-1:0]
s_axis_write_data_tdest	input	wire [AXIS_DEST_WIDTH-1:0]
s_axis_write_data_tuser	input	wire [AXIS_USER_WIDTH-1:0]
ram_wr_cmd_be	output	wire [SEG_COUNT*SEG_BE_WIDTH-1:0]
ram_wr_cmd_addr	output	wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0]
ram_wr_cmd_data	output	wire [SEG_COUNT*SEG_DATA_WIDTH-1:0]
ram_wr_cmd_valid	output	wire [SEG_COUNT-1:0]
ram_wr_cmd_ready	input	wire [SEG_COUNT-1:0]
ram_wr_done	input	wire [SEG_COUNT-1:0]
enable	input	wire
abort	input	wire

API — dma_client_axis_source.v

Table 1829. alexforencich_verilog-pcie/rtl/dma_client_axis_source.v — module / interface / package (1개)

이름	종류
dma_client_axis_source	module

Table 1830. alexforencich_verilog-pcie/rtl/dma_client_axis_source.v — parameter (32개)

파라미터	형/폭	기본값
RAM_ADDR_WIDTH		16
SEG_COUNT		2
SEG_DATA_WIDTH		64
SEG_BE_WIDTH		SEG_DATA_WIDTH/8
SEG_ADDR_WIDTH		RAM_ADDR_WIDTH-\$clog2(SEG_COUNT*SEG_BE_WIDTH)
AXIS_DATA_WIDTH		SEG_DATA_WIDTH*SEG_COUNT/2
AXIS_KEEP_ENABLE		(AXIS_DATA_WIDTH>8)
AXIS_KEEP_WIDTH		(AXIS_DATA_WIDTH/8)
AXIS_LAST_ENABLE		1
AXIS_ID_ENABLE		0
AXIS_ID_WIDTH		8
AXIS_DEST_ENABLE		0
AXIS_DEST_WIDTH		8
AXIS_USER_ENABLE		1
AXIS_USER_WIDTH		1
LEN_WIDTH		16

파라미터	형/폭	기본값
TAG_WIDTH		8
RAM_WORD_WIDTH		SEG_BE_WIDTH
RAM_WORD_SIZE		SEG_DATA_WIDTH/RAM_WORD_WIDTH
AXIS_KEEP_WIDTH_INT		AXIS_KEEP_ENABLE ? AXIS_KEEP_WIDTH : 1
AXIS_WORD_WIDTH		AXIS_KEEP_WIDTH_INT
AXIS_WORD_SIZE		AXIS_DATA_WIDTH/AXIS_WORD_WIDTH
PART_COUNT		SEG_COUNT*SEG_BE_WIDTH / AXIS_KEEP_WIDTH_INT
PART_COUNT_WIDTH		PART_COUNT > 1 ? \$clog2(PART_COUNT) : 1
PART_OFFSET_WIDTH		AXIS_KEEP_WIDTH_INT > 1 ? \$clog2(AXIS_KEEP_WIDTH_INT) : 1
PARTS_PER_SEG		(SEG_BE_WIDTH + AXIS_KEEP_WIDTH_INT - 1) / AXIS_KEEP_WIDTH_I...
SEGS_PER_PART		(AXIS_KEEP_WIDTH_INT + SEG_BE_WIDTH - 1) / SEG_BE_WIDTH
OFFSET_WIDTH		AXIS_KEEP_WIDTH_INT > 1 ? \$clog2(AXIS_KEEP_WIDTH_INT) : 1
OFFSET_MASK		AXIS_KEEP_WIDTH_INT > 1 ? {OFFSET_WIDTH{1'b1}} : 0
ADDR_MASK		{RAM_ADDR_WIDTH{1'b1}} << \$clog2(AXIS_KEEP_WIDTH_INT)
CYCLE_COUNT_WIDTH		LEN_WIDTH - \$clog2(AXIS_KEEP_WIDTH_INT) + 1
OUTPUT_FIFO_ADDR_WIDTH		5

Table 1831. alexforencich_verilog-pcie/rtl/dma_client_axis_source.v — localparam (2개)

상수	형/폭	값
READ_STATE_IDLE	[0:0]	1'd0
AXIS_STATE_IDLE	[0:0]	1'd0

Table 1832. alexforencich_verilog-pcie/rtl/dma_client_axis_source.v — 포트 (28개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_axis_read_desc_ram_addr	input	wire [RAM_ADDR_WIDTH-1:0]
s_axis_read_desc_len	input	wire [LEN_WIDTH-1:0]
s_axis_read_desc_tag	input	wire [TAG_WIDTH-1:0]
s_axis_read_desc_id	input	wire [AXIS_ID_WIDTH-1:0]
s_axis_read_desc_dest	input	wire [AXIS_DEST_WIDTH-1:0]
s_axis_read_desc_user	input	wire [AXIS_USER_WIDTH-1:0]
s_axis_read_desc_valid	input	wire
s_axis_read_desc_ready	output	wire
m_axis_read_desc_status_tag	output	wire [TAG_WIDTH-1:0]
m_axis_read_desc_status_error	output	wire [3:0]

포트	방향	형/폭
m_axis_read_desc_status_valid	output	wire
m_axis_read_data_tdata	output	wire [AXIS_DATA_WIDTH-1:0]
m_axis_read_data_tkeep	output	wire [AXIS_KEEP_WIDTH-1:0]
m_axis_read_data_tvalid	output	wire
m_axis_read_data_tready	input	wire
m_axis_read_data_tlast	output	wire
m_axis_read_data_tid	output	wire [AXIS_ID_WIDTH-1:0]
m_axis_read_data_tdest	output	wire [AXIS_DEST_WIDTH-1:0]
m_axis_read_data_tuser	output	wire [AXIS_USER_WIDTH-1:0]
ram_rd_cmd_addr	output	wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0]
ram_rd_cmd_valid	output	wire [SEG_COUNT-1:0]
ram_rd_cmd_ready	input	wire [SEG_COUNT-1:0]
ram_rd_resp_data	input	wire [SEG_COUNT*SEG_DATA_WIDTH-1:0]
ram_rd_resp_valid	input	wire [SEG_COUNT-1:0]
ram_rd_resp_ready	output	wire [SEG_COUNT-1:0]
enable	input	wire

API — dma_if_axi.v

Table 1833. alexforencich_verilog-pcie/rtl/dma_if_axi.v — module / interface / package (17개)

이름	종류
dma_if_axi	module

Table 1834. alexforencich_verilog-pcie/rtl/dma_if_axi.v — parameter (19개)

파라미터	형/폭	기본값
AXI_DATA_WIDTH		32
AXI_ADDR_WIDTH		16
AXI_STRB_WIDTH		(AXI_DATA_WIDTH/8)
AXI_ID_WIDTH		8
AXI_MAX_BURST_LEN		256
RAM_SEL_WIDTH		2
RAM_ADDR_WIDTH		16
RAM_SEG_COUNT		2
RAM_SEG_DATA_WIDTH		AXI_DATA_WIDTH*2/RAM_SEG_COUNT
RAM_SEG_BE_WIDTH		RAM_SEG_DATA_WIDTH/8
RAM_SEG_ADDR_WIDTH		RAM_ADDR_WIDTH-\$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH)
IMM_ENABLE		0

파라미터	형/폭	기본값
IMM_WIDTH		32
LEN_WIDTH		16
TAG_WIDTH		8
READ_OP_TABLE_SIZE		2**(AXI_ID_WIDTH)
WRITE_OP_TABLE_SIZE		2**(AXI_ID_WIDTH)
READ_USE_AXI_ID		0
WRITE_USE_AXI_ID		1

Table 1835. alexforenrich_verilog-pcie/rtl/dma_if_axi.v — 포트 (105개) — 앞 60개만 표시(전체 105개, 본문 소스 참조)

포트	방향	형/폭
clk	input	wire
rst	input	wire
m_axi_awid	output	wire [AXI_ID_WIDTH-1:0]
m_axi_awaddr	output	wire [AXI_ADDR_WIDTH-1:0]
m_axi_awlen	output	wire [7:0]
m_axi_awsz	output	wire [2:0]
m_axi_awburst	output	wire [1:0]
m_axi_awlock	output	wire
m_axi_awcache	output	wire [3:0]
m_axi_awprot	output	wire [2:0]
m_axi_awvalid	output	wire
m_axi_awready	input	wire
m_axi_wdata	output	wire [AXI_DATA_WIDTH-1:0]
m_axi_wstrb	output	wire [AXI_STRB_WIDTH-1:0]
m_axi_wlast	output	wire
m_axi_wvalid	output	wire
m_axi_wready	input	wire
m_axi_bid	input	wire [AXI_ID_WIDTH-1:0]
m_axi_bresp	input	wire [1:0]
m_axi_bvalid	input	wire
m_axi_bready	output	wire
m_axi_arid	output	wire [AXI_ID_WIDTH-1:0]
m_axi_araddr	output	wire [AXI_ADDR_WIDTH-1:0]
m_axi_arlen	output	wire [7:0]

포트	방향	형/폭
m_axi_arsize	output	wire [2:0]
m_axi_arburst	output	wire [1:0]
m_axi_arlock	output	wire
m_axi_arcache	output	wire [3:0]
m_axi_arprot	output	wire [2:0]
m_axi_arvalid	output	wire
m_axi_arready	input	wire
m_axi_rid	input	wire [AXI_ID_WIDTH-1:0]
m_axi_rdata	input	wire [AXI_DATA_WIDTH-1:0]
m_axi_rresp	input	wire [1:0]
m_axi_rlast	input	wire
m_axi_rvalid	input	wire
m_axi_rready	output	wire
s_axis_read_desc_axi_addr	input	wire [AXI_ADDR_WIDTH-1:0]
s_axis_read_desc_ram_sel	input	wire [RAM_SEL_WIDTH-1:0]
s_axis_read_desc_ram_addr	input	wire [RAM_ADDR_WIDTH-1:0]
s_axis_read_desc_len	input	wire [LEN_WIDTH-1:0]
s_axis_read_desc_tag	input	wire [TAG_WIDTH-1:0]
s_axis_read_desc_valid	input	wire
s_axis_read_desc_ready	output	wire
m_axis_read_desc_status_tag	output	wire [TAG_WIDTH-1:0]
m_axis_read_desc_status_error	output	wire [3:0]
m_axis_read_desc_status_valid	output	wire
s_axis_write_desc_axi_addr	input	wire [AXI_ADDR_WIDTH-1:0]
s_axis_write_desc_ram_sel	input	wire [RAM_SEL_WIDTH-1:0]
s_axis_write_desc_ram_addr	input	wire [RAM_ADDR_WIDTH-1:0]
s_axis_write_desc_imm	input	wire [IMM_WIDTH-1:0]
s_axis_write_desc_imm_en	input	wire
s_axis_write_desc_len	input	wire [LEN_WIDTH-1:0]
s_axis_write_desc_tag	input	wire [TAG_WIDTH-1:0]
s_axis_write_desc_valid	input	wire
s_axis_write_desc_ready	output	wire
m_axis_write_desc_status_tag	output	wire [TAG_WIDTH-1:0]
m_axis_write_desc_status_error	output	wire [3:0]
m_axis_write_desc_status_valid	output	wire

포트	방향	형/폭
ram_wr_cmd_sel	output	wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]

API — dma_if_axi_rd.v

Table 1836. alexforencich_verilog-pcie/rtl/dma_if_axi_rd.v — module / interface / package (1개)

이름	종류
dma_if_axi_rd	module

Table 1837. alexforencich_verilog-pcie/rtl/dma_if_axi_rd.v — parameter (32개)

파라미터	형/폭	기본값
AXI_DATA_WIDTH		32
AXI_ADDR_WIDTH		16
AXI_STRB_WIDTH		(AXI_DATA_WIDTH/8)
AXI_ID_WIDTH		8
AXI_MAX_BURST_LEN		256
RAM_SEL_WIDTH		2
RAM_ADDR_WIDTH		16
RAM_SEG_COUNT		2
RAM_SEG_DATA_WIDTH		AXI_DATA_WIDTH*2/RAM_SEG_COUNT
RAM_SEG_BE_WIDTH		RAM_SEG_DATA_WIDTH/8
RAM_SEG_ADDR_WIDTH		RAM_ADDR_WIDTH-\$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH)
LEN_WIDTH		16
TAG_WIDTH		8
OP_TABLE_SIZE		2**(AXI_ID_WIDTH)
USE_AXI_ID		0
RAM_DATA_WIDTH		RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH
RAM_WORD_WIDTH		RAM_SEG_BE_WIDTH
RAM_WORD_SIZE		RAM_SEG_DATA_WIDTH/RAM_WORD_WIDTH
AXI_WORD_WIDTH		AXI_STRB_WIDTH
AXI_WORD_SIZE		AXI_DATA_WIDTH/AXI_WORD_WIDTH
AXI_BURST_SIZE		\$clog2(AXI_STRB_WIDTH)
AXI_MAX_BURST_SIZE		AXI_MAX_BURST_LEN << AXI_BURST_SIZE
OFFSET_WIDTH		AXI_STRB_WIDTH > 1 ? \$clog2(AXI_STRB_WIDTH) : 1
OFFSET_MASK		AXI_STRB_WIDTH > 1 ? {OFFSET_WIDTH{1'b1}} : 0
RAM_OFFSET_WIDTH		\$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH)
ADDR_MASK		{AXI_ADDR_WIDTH{1'b1}} << \$clog2(AXI_STRB_WIDTH)
CYCLE_COUNT_WIDTH		LEN_WIDTH - AXI_BURST_SIZE + 1

파라미터	형/폭	기본값
OP_TAG_WIDTH		$\$clog2(OP_TABLE_SIZE)$
OP_TABLE_READ_COUNT_WIDTH		AXI_ID_WIDTH+1
OP_TABLE_WRITE_COUNT_WIDTH		LEN_WIDTH
STATUS_FIFO_ADDR_WIDTH		5
OUTPUT_FIFO_ADDR_WIDTH		5

Table 1838. alexforencich_verilog-pcie/rtl/dma_if_axi_rd.v — localparam (4개)

상수	형/폭	값
AXI_RESP_OKAY	[1:0]	2'b00
DMA_ERROR_NONE	[3:0]	4'd0
REQ_STATE_IDLE	[0:0]	1'd0
AXI_STATE_IDLE	[0:0]	1'd0

Table 1839. alexforencich_verilog-pcie/rtl/dma_if_axi_rd.v — 포트 (51개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
m_axi_arid	output	wire [AXI_ID_WIDTH-1:0]
m_axi_araddr	output	wire [AXI_ADDR_WIDTH-1:0]
m_axi_arlen	output	wire [7:0]
m_axi_arsize	output	wire [2:0]
m_axi_arburst	output	wire [1:0]
m_axi_arlock	output	wire
m_axi_arcache	output	wire [3:0]
m_axi_arprot	output	wire [2:0]
m_axi_arvalid	output	wire
m_axi_arready	input	wire
m_axi_rid	input	wire [AXI_ID_WIDTH-1:0]
m_axi_rdata	input	wire [AXI_DATA_WIDTH-1:0]
m_axi_rresp	input	wire [1:0]
m_axi_rlast	input	wire
m_axi_rvalid	input	wire
m_axi_rready	output	wire
s_axis_read_desc_axi_addr	input	wire [AXI_ADDR_WIDTH-1:0]
s_axis_read_desc_ram_sel	input	wire [RAM_SEL_WIDTH-1:0]
s_axis_read_desc_ram_addr	input	wire [RAM_ADDR_WIDTH-1:0]

포트	방향	형/폭
s_axis_read_desc_len	input	wire [LEN_WIDTH-1:0]
s_axis_read_desc_tag	input	wire [TAG_WIDTH-1:0]
s_axis_read_desc_valid	input	wire
s_axis_read_desc_ready	output	wire
m_axis_read_desc_status_tag	output	wire [TAG_WIDTH-1:0]
m_axis_read_desc_status_error	output	wire [3:0]
m_axis_read_desc_status_valid	output	wire
ram_wr_cmd_sel	output	wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]
ram_wr_cmd_be	output	wire [RAM_SEG_COUNT*RAM_SEG_BE_WIDTH-1:0]
ram_wr_cmd_addr	output	wire [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0]
ram_wr_cmd_data	output	wire [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0]
ram_wr_cmd_valid	output	wire [RAM_SEG_COUNT-1:0]
ram_wr_cmd_ready	input	wire [RAM_SEG_COUNT-1:0]
ram_wr_done	input	wire [RAM_SEG_COUNT-1:0]
enable	input	wire
status_busy	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_rd_op_start_len	output	wire [LEN_WIDTH-1:0]
stat_rd_op_start_valid	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_rd_op_finish_status	output	wire [3:0]
stat_rd_op_finish_valid	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_rd_req_start_len	output	wire [12:0]
stat_rd_req_start_valid	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_rd_req_finish_status	output	wire [3:0]
stat_rd_req_finish_valid	output	wire
stat_rd_op_table_full	output	wire
stat_rd_tx_stall	output	wire

API — dma_if_axi_wr.v

Table 1840. alexforenrich_verilog-pcie/rtl/dma_if_axi_wr.v — module / interface / package (1개)

이름	종류
dma_if_axi_wr	module

Table 1841. alexforencich_verilog-pcie/rtl/dma_if_axi_wr.v — parameter (32개)

파라미터	형/폭	기본값
AXI_DATA_WIDTH		32
AXI_ADDR_WIDTH		16
AXI_STRB_WIDTH		(AXI_DATA_WIDTH/8)
AXI_ID_WIDTH		8
AXI_MAX_BURST_LEN		256
RAM_SEL_WIDTH		2
RAM_ADDR_WIDTH		16
RAM_SEG_COUNT		2
RAM_SEG_DATA_WIDTH		AXI_DATA_WIDTH*2/RAM_SEG_COUNT
RAM_SEG_BE_WIDTH		RAM_SEG_DATA_WIDTH/8
RAM_SEG_ADDR_WIDTH		RAM_ADDR_WIDTH-\$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH)
IMM_ENABLE		0
IMM_WIDTH		32
LEN_WIDTH		16
TAG_WIDTH		8
OP_TABLE_SIZE		2**(AXI_ID_WIDTH)
USE_AXI_ID		1
RAM_DATA_WIDTH		RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH
RAM_WORD_WIDTH		RAM_SEG_BE_WIDTH
RAM_WORD_SIZE		RAM_SEG_DATA_WIDTH/RAM_WORD_WIDTH
AXI_WORD_WIDTH		AXI_STRB_WIDTH
AXI_WORD_SIZE		AXI_DATA_WIDTH/AXI_WORD_WIDTH
AXI_BURST_SIZE		\$clog2(AXI_STRB_WIDTH)
AXI_MAX_BURST_SIZE		AXI_MAX_BURST_LEN << AXI_BURST_SIZE
OFFSET_WIDTH		AXI_STRB_WIDTH > 1 ? \$clog2(AXI_STRB_WIDTH) : 1
OFFSET_MASK		AXI_STRB_WIDTH > 1 ? {OFFSET_WIDTH{1'b1}} : 0
RAM_OFFSET_WIDTH		\$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH)
ADDR_MASK		{AXI_ADDR_WIDTH{1'b1}} << \$clog2(AXI_STRB_WIDTH)
CYCLE_COUNT_WIDTH		LEN_WIDTH - AXI_BURST_SIZE + 1
MASK_FIFO_ADDR_WIDTH		\$clog2(OP_TABLE_SIZE)+1
OP_TAG_WIDTH		\$clog2(OP_TABLE_SIZE)
OUTPUT_FIFO_ADDR_WIDTH		5

Table 1842. alexforencich_verilog-pcie/rtl/dma_if_axi_wr.v — localparam (5개)

상수	형/폭	값
AXI_RESP_OKAY	[1:0]	2'b00
DMA_ERROR_NONE	[3:0]	4'd0
REQ_STATE_IDLE	[0:0]	1'd0
READ_STATE_IDLE	[0:0]	1'd0
AXI_STATE_IDLE	[0:0]	1'd0

Table 1843. alexforencich_verilog-pcie/rtl/dma_if_axi_wr.v — 포트 (56개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
m_axi_awid	output	wire [AXI_ID_WIDTH-1:0]
m_axi_awaddr	output	wire [AXI_ADDR_WIDTH-1:0]
m_axi_awlen	output	wire [7:0]
m_axi_awsz	output	wire [2:0]
m_axi_awburst	output	wire [1:0]
m_axi_awlock	output	wire
m_axi_awcache	output	wire [3:0]
m_axi_awprot	output	wire [2:0]
m_axi_awvalid	output	wire
m_axi_awready	input	wire
m_axi_wdata	output	wire [AXI_DATA_WIDTH-1:0]
m_axi_wstrb	output	wire [AXI_STRB_WIDTH-1:0]
m_axi_wlast	output	wire
m_axi_wvalid	output	wire
m_axi_wready	input	wire
m_axi_bid	input	wire [AXI_ID_WIDTH-1:0]
m_axi_bresp	input	wire [1:0]
m_axi_bvalid	input	wire
m_axi_bready	output	wire
s_axis_write_desc_axi_addr	input	wire [AXI_ADDR_WIDTH-1:0]
s_axis_write_desc_ram_sel	input	wire [RAM_SEL_WIDTH-1:0]
s_axis_write_desc_ram_addr	input	wire [RAM_ADDR_WIDTH-1:0]
s_axis_write_desc_imm	input	wire [IMM_WIDTH-1:0]
s_axis_write_desc_imm_en	input	wire
s_axis_write_desc_len	input	wire [LEN_WIDTH-1:0]

포트	방향	형/폭
s_axis_write_desc_tag	input	wire [TAG_WIDTH-1:0]
s_axis_write_desc_valid	input	wire
s_axis_write_desc_ready	output	wire
m_axis_write_desc_status_tag	output	wire [TAG_WIDTH-1:0]
m_axis_write_desc_status_error	output	wire [3:0]
m_axis_write_desc_status_valid	output	wire
ram_rd_cmd_sel	output	wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]
ram_rd_cmd_addr	output	wire [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0]
ram_rd_cmd_valid	output	wire [RAM_SEG_COUNT-1:0]
ram_rd_cmd_ready	input	wire [RAM_SEG_COUNT-1:0]
ram_rd_resp_data	input	wire [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0]
ram_rd_resp_valid	input	wire [RAM_SEG_COUNT-1:0]
ram_rd_resp_ready	output	wire [RAM_SEG_COUNT-1:0]
enable	input	wire
status_busy	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_wr_op_start_len	output	wire [LEN_WIDTH-1:0]
stat_wr_op_start_valid	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_wr_op_finish_status	output	wire [3:0]
stat_wr_op_finish_valid	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_wr_req_start_len	output	wire [12:0]
stat_wr_req_start_valid	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_wr_req_finish_status	output	wire [3:0]
stat_wr_req_finish_valid	output	wire
stat_wr_op_table_full	output	wire
stat_wr_tx_stall	output	wire

API — dma_if_desc_mux.v

Table 1844. alexforencich_verilog-pcie/rtl/dma_if_desc_mux.v — module / interface / package (1개)

이름	종류
dma_if_desc_mux	module

Table 1845. alexforencich_verilog-pcie/rtl/dma_if_desc_mux.v — parameter (16개)

파라미터	형/폭	기본값
PORTS		2
EXTEND_RAM_SEL		0
S_RAM_SEL_WIDTH		2
M_RAM_SEL_WIDTH		S_RAM_SEL_WIDTH+(EXTEND_RAM_SEL ? \$clog2(PORTS) : 0)
RAM_ADDR_WIDTH		16
DMA_ADDR_WIDTH		64
IMM_ENABLE		0
IMM_WIDTH		32
LEN_WIDTH		16
S_TAG_WIDTH		8
M_TAG_WIDTH		S_TAG_WIDTH+\$clog2(PORTS)
ARB_TYPE_ROUND_ROBIN		0
ARB_LSB_HIGH_PRIORITY		1
CL_PORTS		\$clog2(PORTS)
S_RAM_SEL_WIDTH_INT		S_RAM_SEL_WIDTH > 0 ? S_RAM_SEL_WIDTH : 1
FIFO_ADDR_WIDTH		5

Table 1846. alexforencich_verilog-pcie/rtl/dma_if_desc_mux.v — 포트 (26개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
m_axis_desc_dma_addr	output	wire [DMA_ADDR_WIDTH-1:0]
m_axis_desc_ram_sel	output	wire [M_RAM_SEL_WIDTH-1:0]
m_axis_desc_ram_addr	output	wire [RAM_ADDR_WIDTH-1:0]
m_axis_desc_imm	output	wire [IMM_WIDTH-1:0]
m_axis_desc_imm_en	output	wire
m_axis_desc_len	output	wire [LEN_WIDTH-1:0]
m_axis_desc_tag	output	wire [M_TAG_WIDTH-1:0]
m_axis_desc_valid	output	wire
m_axis_desc_ready	input	wire
s_axis_desc_status_tag	input	wire [M_TAG_WIDTH-1:0]
s_axis_desc_status_error	input	wire [3:0]
s_axis_desc_status_valid	input	wire
s_axis_desc_dma_addr	input	wire [PORTS*DMA_ADDR_WIDTH-1:0]
s_axis_desc_ram_sel	input	wire [PORTS*S_RAM_SEL_WIDTH-1:0]

포트	방향	형/폭
s_axis_desc_ram_addr	input	wire [PORTS*RAM_ADDR_WIDTH-1:0]
s_axis_desc_imm	input	wire [PORTS*IMM_WIDTH-1:0]
s_axis_desc_imm_en	input	wire [PORTS-1:0]
s_axis_desc_len	input	wire [PORTS*LEN_WIDTH-1:0]
s_axis_desc_tag	input	wire [PORTS*S_TAG_WIDTH-1:0]
s_axis_desc_valid	input	wire [PORTS-1:0]
s_axis_desc_ready	output	wire [PORTS-1:0]
m_axis_desc_status_tag	output	wire [PORTS*S_TAG_WIDTH-1:0]
m_axis_desc_status_error	output	wire [PORTS*4-1:0]
m_axis_desc_status_valid	output	wire [PORTS-1:0]

API — dma_if_mux.v

Table 1847. alexforencich_verilog-pcie/rtl/dma_if_mux.v — module / interface / package (17개)

이름	종류
dma_if_mux	module

Table 1848. alexforencich_verilog-pcie/rtl/dma_if_mux.v — parameter (16개)

파라미터	형/폭	기본값
PORTS		2
SEG_COUNT		2
SEG_DATA_WIDTH		64
SEG_ADDR_WIDTH		8
SEG_BE_WIDTH		SEG_DATA_WIDTH/8
S_RAM_SEL_WIDTH		2
M_RAM_SEL_WIDTH		S_RAM_SEL_WIDTH+\$clog2(PORTS)
RAM_ADDR_WIDTH		SEG_ADDR_WIDTH+\$clog2(SEG_COUNT)+\$clog2(SEG_BE_WIDTH)
DMA_ADDR_WIDTH		64
IMM_ENABLE		0
IMM_WIDTH		32
LEN_WIDTH		16
S_TAG_WIDTH		8
M_TAG_WIDTH		S_TAG_WIDTH+\$clog2(PORTS)
ARB_TYPE_ROUND_ROBIN		0
ARB_LSB_HIGH_PRIORITY		1

Table 1849. alexforencich_verilog-pcie/rtl/dma_if_mux.v — 포트 (74개) — 앞 60개만 표시(전체 74개, 본문 소스 참조)

포트	방향	형/폭
clk	input	wire
rst	input	wire
m_axis_read_desc_dma_addr	output	wire [DMA_ADDR_WIDTH-1:0]
m_axis_read_desc_ram_sel	output	wire [M_RAM_SEL_WIDTH-1:0]
m_axis_read_desc_ram_addr	output	wire [RAM_ADDR_WIDTH-1:0]
m_axis_read_desc_len	output	wire [LEN_WIDTH-1:0]
m_axis_read_desc_tag	output	wire [M_TAG_WIDTH-1:0]
m_axis_read_desc_valid	output	wire
m_axis_read_desc_ready	input	wire
s_axis_read_desc_status_tag	input	wire [M_TAG_WIDTH-1:0]
s_axis_read_desc_status_error	input	wire [3:0]
s_axis_read_desc_status_valid	input	wire
m_axis_write_desc_dma_addr	output	wire [DMA_ADDR_WIDTH-1:0]
m_axis_write_desc_ram_sel	output	wire [M_RAM_SEL_WIDTH-1:0]
m_axis_write_desc_ram_addr	output	wire [RAM_ADDR_WIDTH-1:0]
m_axis_write_desc_imm	output	wire [IMM_WIDTH-1:0]
m_axis_write_desc_imm_en	output	wire
m_axis_write_desc_len	output	wire [LEN_WIDTH-1:0]
m_axis_write_desc_tag	output	wire [M_TAG_WIDTH-1:0]
m_axis_write_desc_valid	output	wire
m_axis_write_desc_ready	input	wire
s_axis_write_desc_status_tag	input	wire [M_TAG_WIDTH-1:0]
s_axis_write_desc_status_error	input	wire [3:0]
s_axis_write_desc_status_valid	input	wire
s_axis_read_desc_dma_addr	input	wire [PORTS*DMA_ADDR_WIDTH-1:0]
s_axis_read_desc_ram_sel	input	wire [PORTS*S_RAM_SEL_WIDTH-1:0]
s_axis_read_desc_ram_addr	input	wire [PORTS*RAM_ADDR_WIDTH-1:0]
s_axis_read_desc_len	input	wire [PORTS*LEN_WIDTH-1:0]
s_axis_read_desc_tag	input	wire [PORTS*S_TAG_WIDTH-1:0]
s_axis_read_desc_valid	input	wire [PORTS-1:0]
s_axis_read_desc_ready	output	wire [PORTS-1:0]
m_axis_read_desc_status_tag	output	wire [PORTS*S_TAG_WIDTH-1:0]
m_axis_read_desc_status_error	output	wire [PORTS*4-1:0]
m_axis_read_desc_status_valid	output	wire [PORTS-1:0]
s_axis_write_desc_dma_addr	input	wire [PORTS*DMA_ADDR_WIDTH-1:0]

포트	방향	형/폭
s_axis_write_desc_ram_sel	input	wire [PORTS*S_RAM_SEL_WIDTH-1:0]
s_axis_write_desc_ram_addr	input	wire [PORTS*RAM_ADDR_WIDTH-1:0]
s_axis_write_desc_imm	input	wire [PORTS*IMM_WIDTH-1:0]
s_axis_write_desc_imm_en	input	wire [PORTS-1:0]
s_axis_write_desc_len	input	wire [PORTS*LEN_WIDTH-1:0]
s_axis_write_desc_tag	input	wire [PORTS*S_TAG_WIDTH-1:0]
s_axis_write_desc_valid	input	wire [PORTS-1:0]
s_axis_write_desc_ready	output	wire [PORTS-1:0]
m_axis_write_desc_status_tag	output	wire [PORTS*S_TAG_WIDTH-1:0]
m_axis_write_desc_status_error	output	wire [PORTS*4-1:0]
m_axis_write_desc_status_valid	output	wire [PORTS-1:0]
if_ram_wr_cmd_sel	input	wire [SEG_COUNT*M_RAM_SEL_WIDTH-1:0]
if_ram_wr_cmd_be	input	wire [SEG_COUNT*SEG_BE_WIDTH-1:0]
if_ram_wr_cmd_addr	input	wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0]
if_ram_wr_cmd_data	input	wire [SEG_COUNT*SEG_DATA_WIDTH-1:0]
if_ram_wr_cmd_valid	input	wire [SEG_COUNT-1:0]
if_ram_wr_cmd_ready	output	wire [SEG_COUNT-1:0]
if_ram_wr_done	output	wire [SEG_COUNT-1:0]
if_ram_rd_cmd_sel	input	wire [SEG_COUNT*M_RAM_SEL_WIDTH-1:0]
if_ram_rd_cmd_addr	input	wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0]
if_ram_rd_cmd_valid	input	wire [SEG_COUNT-1:0]
if_ram_rd_cmd_ready	output	wire [SEG_COUNT-1:0]
if_ram_rd_resp_data	output	wire [SEG_COUNT*SEG_DATA_WIDTH-1:0]
if_ram_rd_resp_valid	output	wire [SEG_COUNT-1:0]
if_ram_rd_resp_ready	input	wire [SEG_COUNT-1:0]

1. alexforenchich_verilog-pcie/rtl/arbiter.v (159 lines)

```

1 /*
2
3 Copyright (c) 2014-2021 Alex Forenchich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR

```

```

16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * Arbiter module
33 */
34 module arbiter #
35 (
36     parameter PORTS = 4,

```

```

37 // select round robin arbitration
38 parameter ARB_TYPE_ROUND_ROBIN = 0,
39 // blocking arbiter enable
40 parameter ARB_BLOCK = 0,
41 // block on acknowledge assert when nonzero, request deas-
sert when 0
42 parameter ARB_BLOCK_ACK = 1,
43 // LSB priority selection
44 parameter ARB_LSB_HIGH_PRIORITY = 0
45 )
46 (
47     input wire clk,
48     input wire rst,
49     input wire [PORTS-1:0] request,
50     input wire [PORTS-1:0] acknowledge,
51     output wire [PORTS-1:0] grant,
52     output wire grant_valid,
53     output wire [$clog2(PORTS)-1:0] grant_encoded
54 );
55 reg [PORTS-1:0] grant_reg = 0, grant_next;
56 reg grant_valid_reg = 0, grant_valid_next;
57 reg [$clog2(PORTS)-1:0] grant_encoded_reg = 0,
grant_encoded_next;
58 assign grant_valid = grant_valid_reg;
59 assign grant = grant_reg;
60 assign grant_encoded = grant_encoded_reg;
61
62 wire request_valid;
63 wire [$clog2(PORTS)-1:0] request_index;
64 wire [PORTS-1:0] request_mask;
65
66 priority_encoder #(
67     .WIDTH(PORTS),
68     .LSB_HIGH_PRIORITY(ARB_LSB_HIGH_PRIORITY)
69 )
70 priority_encoder_inst (
71     .input_unencoded(request),
72     .output_valid(request_valid),
73     .output_encoded(request_index),
74     .output_unencoded(request_mask)
75 );
76 reg [PORTS-1:0] mask_reg = 0, mask_next;
77
78 wire masked_request_valid;
79 wire [$clog2(PORTS)-1:0] masked_request_index;
80 wire [PORTS-1:0] masked_request_mask;
81
82 priority_encoder #(
83     .WIDTH(PORTS),
84     .LSB_HIGH_PRIORITY(ARB_LSB_HIGH_PRIORITY)
85 )
86 priority_encoder_masked (
87     .input_unencoded(request & mask_reg),
88     .output_valid(masked_request_valid),
89     .output_encoded(masked_request_index),
90     .output_unencoded(masked_request_mask)
91 );
92
93 always @* begin
94     grant_next = 0;
95     grant_valid_next = 0;
96     grant_encoded_next = 0;
97     mask_next = mask_reg;
98
99     if (ARB_BLOCK && !ARB_BLOCK_ACK && grant_reg & request)
100 begin
101     // granted request still asserted; hold it
102     grant_valid_next = grant_valid_reg;
103     grant_next = grant_reg;
104     grant_encoded_next = grant_encoded_reg;
105 end else if (ARB_BLOCK && ARB_BLOCK_ACK && grant_valid
&& !(grant_reg & acknowledge)) begin
106     // granted request not yet acknowledged; hold it
107     grant_valid_next = grant_valid_reg;
108     grant_next = grant_reg;
109     grant_encoded_next = grant_encoded_reg;
110 end else if (request_valid) begin
111     if (ARB_TYPE_ROUND_ROBIN) begin
112         if (masked_request_valid) begin
113             grant_valid_next = 1;
114             grant_next = masked_request_mask;
115             grant_encoded_next = masked_request_index;
116             if (ARB_LSB_HIGH_PRIORITY) begin
117                 mask_next = {PORTS{1'b1}} <<
(masked_request_index + 1);
118             end else begin
119                 mask_next = {PORTS{1'b1}} >> (PORTS -
masked_request_index);
120             end
121         end else begin
122             grant_valid_next = 1;
123             grant_next = request_mask;
124             grant_encoded_next = request_index;
125             if (ARB_LSB_HIGH_PRIORITY) begin
126                 mask_next = {PORTS{1'b1}} << (re-
quest_index + 1);

```

```

131     end else begin
132         mask_next = {PORTS{1'b1}} >> (PORTS - re-
quest_index);
133     end
134 end
135 end else begin
136     grant_valid_next = 1;
137     grant_next = request_mask;
138     grant_encoded_next = request_index;
139 end
140 end
141 end
142
143 always @(posedge clk) begin
144     if (rst) begin
145         grant_reg <= 0;
146         grant_valid_reg <= 0;
147         grant_encoded_reg <= 0;
148         mask_reg <= 0;
149     end else begin
150         grant_reg <= grant_next;
151         grant_valid_reg <= grant_valid_next;
152         grant_encoded_reg <= grant_encoded_next;
153         mask_reg <= mask_next;
154     end
155 end
156
157 endmodule
158
159 `resetall

```

2. alexforenrich_verilog-pcie/rtl/axis_arb_mux.v (310 lines)

```

1 /*
2
3 Copyright (c) 2014-2018 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * AXI4-Stream arbitrated multiplexer
33 */
34 module axis_arb_mux #
35 (
36     // Number of AXI stream inputs
37     parameter S_COUNT = 4,
38     // Width of AXI stream interfaces in bits
39     parameter DATA_WIDTH = 8,
40     // Propagate tkeep signal
41     parameter KEEP_ENABLE = (DATA_WIDTH>8),
42     // tkeep signal width (words per cycle)
43     parameter KEEP_WIDTH = ((DATA_WIDTH+7)/8),
44     // Propagate tid signal
45     parameter ID_ENABLE = 0,
46     // input tid signal width
47     parameter S_ID_WIDTH = 8,
48     // output tid signal width
49     parameter M_ID_WIDTH = S_ID_WIDTH+$clog2(S_COUNT),
50     // Propagate tdest signal
51     parameter DEST_ENABLE = 0,
52     // tdest signal width
53     parameter DEST_WIDTH = 8,
54     // Propagate tuser signal

```



```

55 parameter USER_ENABLE = 1,
56 // tuser signal width
57 parameter USER_WIDTH = 1,
58 // Propagate tlast signal
59 parameter LAST_ENABLE = 1,
60 // Update tid with routing information
61 parameter UPDATE_TID = 0,
62 // select round robin arbitration
63 parameter ARB_TYPE_ROUND_ROBIN = 0,
64 // LSB priority selection
65 parameter ARB_LSB_HIGH_PRIORITY = 1
66 )
67 (
68   input wire clk,
69   input wire rst,
70
71   /*
72    * AXI Stream inputs
73    */
74   input wire [S_COUNT*DATA_WIDTH-1:0] s_axis_tdata,
75   input wire [S_COUNT*KEEP_WIDTH-1:0] s_axis_tkeep,
76   input wire [S_COUNT-1:0] s_axis_tvalid,
77   output wire [S_COUNT-1:0] s_axis_tready,
78   input wire [S_COUNT-1:0] s_axis_tlast,
79   input wire [S_COUNT*S_ID_WIDTH-1:0] s_axis_tid,
80   input wire [S_COUNT*DEST_WIDTH-1:0] s_axis_tdest,
81   input wire [S_COUNT*USER_WIDTH-1:0] s_axis_tuser,
82
83   /*
84    * AXI Stream output
85    */
86   output wire [DATA_WIDTH-1:0] m_axis_tdata,
87   output wire [KEEP_WIDTH-1:0] m_axis_tkeep,
88   output wire m_axis_tvalid,
89   input wire m_axis_tready,
90   output wire m_axis_tlast,
91   output wire [M_ID_WIDTH-1:0] m_axis_tid,
92   output wire [DEST_WIDTH-1:0] m_axis_tdest,
93   output wire [USER_WIDTH-1:0] m_axis_tuser
94 );
95
96 parameter CL_S_COUNT = $clog2(S_COUNT);
97
98 parameter S_ID_WIDTH_INT = S_ID_WIDTH > 0 ? S_ID_WIDTH : 1;
99
100 // check configuration
101 initial begin
102   if (UPDATE_TID) begin
103     if (!ID_ENABLE) begin
104       $error("Error: UPDATE_TID set requires ID_ENABLE");
105     end
106   end
107
108   if (M_ID_WIDTH < CL_S_COUNT) begin
109     $error("Error: M_ID_WIDTH too small for port count (instance %m)");
110   end
111   $finish;
112 end
113
114 wire [S_COUNT-1:0] request;
115 wire [S_COUNT-1:0] acknowledge;
116 wire [S_COUNT-1:0] grant;
117 wire grant_valid;
118 wire [CL_S_COUNT-1:0] grant_encoded;
119
120 // input registers to pipeline arbitration delay
121 reg [S_COUNT*DATA_WIDTH-1:0] s_axis_tdata_reg = 0;
122 reg [S_COUNT*KEEP_WIDTH-1:0] s_axis_tkeep_reg = 0;
123 reg [S_COUNT-1:0] s_axis_tvalid_reg = 0;
124 reg [S_COUNT-1:0] s_axis_tlast_reg = 0;
125 reg [S_COUNT*S_ID_WIDTH-1:0] s_axis_tid_reg = 0;
126 reg [S_COUNT*DEST_WIDTH-1:0] s_axis_tdest_reg = 0;
127 reg [S_COUNT*USER_WIDTH-1:0] s_axis_tuser_reg = 0;
128
129 // internal datapath
130 reg [DATA_WIDTH-1:0] m_axis_tdata_int;
131 reg [KEEP_WIDTH-1:0] m_axis_tkeep_int;
132 reg m_axis_tvalid_int;
133 reg m_axis_tready_int_reg = 1'b0;
134 reg m_axis_tlast_int;
135 reg [M_ID_WIDTH-1:0] m_axis_tid_int;
136 reg [DEST_WIDTH-1:0] m_axis_tdest_int;
137 reg [USER_WIDTH-1:0] m_axis_tuser_int;
138 wire m_axis_tready_int_early;
139
140 assign s_axis_tready = ~s_axis_tvalid_reg |
({S_COUNT{m_axis_tready_int_reg}} & grant);
141
142 // mux for incoming packet
143 wire [DATA_WIDTH-1:0] current_s_tdata = s_ax-
is_tdata_reg[grant_encoded*DATA_WIDTH+: DATA_WIDTH];
144 wire [KEEP_WIDTH-1:0] current_s_tkeep = s_ax-
is_tkeep_reg[grant_encoded*KEEP_WIDTH+: KEEP_WIDTH];
145 wire current_s_tvalid = s_ax-
is_tvalid_reg[grant_encoded];
146 wire current_s_tready = s_ax-
is_tready[grant_encoded];

```

```

147 wire current_s_tlast = s_ax-
is_tlast_reg[grant_encoded];
148 wire [S_ID_WIDTH-1:0] current_s_tid = s_ax-
is_tid_reg[grant_encoded*S_ID_WIDTH+: S_ID_WIDTH_INT];
149 wire [DEST_WIDTH-1:0] current_s_tdest = s_ax-
is_tdest_reg[grant_encoded*DEST_WIDTH+: DEST_WIDTH];
150 wire [USER_WIDTH-1:0] current_s_tuser = s_ax-
is_tuser_reg[grant_encoded*USER_WIDTH+: USER_WIDTH];
151
152 // arbiter instance
153 arbiter #(
154   .PORTS(S_COUNT),
155   .ARB_TYPE_ROUND_ROBIN(ARB_TYPE_ROUND_ROBIN),
156   .ARB_BLOCK(1),
157   .ARB_BLOCK_ACK(1),
158   .ARB_LSB_HIGH_PRIORITY(ARB_LSB_HIGH_PRIORITY)
159 )
160 arb_inst (
161   .clk(clk),
162   .rst(rst),
163   .request(request),
164   .acknowledge(acknowledge),
165   .grant(grant),
166   .grant_valid(grant_valid),
167   .grant_encoded(grant_encoded)
168 );
169
170
171 assign request = (s_axis_tvalid_reg & ~grant) | (s_axis_tvalid &
grant);
172 assign acknowledge = grant & s_axis_tvalid_reg &
{S_COUNT{m_axis_tready_int_reg}} & (LAST_ENABLE ? s_axis_tlast_reg :
{S_COUNT{1'b1}});
173
174 always @* begin
175   // pass through selected packet data
176   m_axis_tdata_int = current_s_tdata;
177   m_axis_tkeep_int = current_s_tkeep;
178   m_axis_tvalid_int = current_s_tvalid && m_ax-
is_tready_int_reg && grant_valid;
179   m_axis_tlast_int = current_s_tlast;
180   m_axis_tid_int = current_s_tid;
181   if (UPDATE_TID && S_COUNT > 1) begin
182     m_axis_tid_int[M_ID_WIDTH-1:M_ID_WIDTH-CL_S_COUNT] =
grant_encoded;
183   end
184   m_axis_tdest_int = current_s_tdest;
185   m_axis_tuser_int = current_s_tuser;
186 end
187
188 integer i;
189
190 always @(posedge clk) begin
191   // register inputs
192   for (i = 0; i < S_COUNT; i = i + 1) begin
193     if (s_axis_tready[i]) begin
194       s_axis_tdata_reg[i*DATA_WIDTH+: DATA_WIDTH] <=
s_axis_tdata[i*DATA_WIDTH+: DATA_WIDTH];
195       s_axis_tkeep_reg[i*KEEP_WIDTH+: KEEP_WIDTH] <=
s_axis_tkeep[i*KEEP_WIDTH+: KEEP_WIDTH];
196       s_axis_tvalid_reg[i] <= s_axis_tvalid[i];
197       s_axis_tlast_reg[i] <= s_axis_tlast[i];
198       s_axis_tid_reg[i*S_ID_WIDTH+: S_ID_WIDTH_INT] <=
s_axis_tid[i*S_ID_WIDTH+: S_ID_WIDTH_INT];
199       s_axis_tdest_reg[i*DEST_WIDTH+: DEST_WIDTH] <=
s_axis_tdest[i*DEST_WIDTH+: DEST_WIDTH];
200       s_axis_tuser_reg[i*USER_WIDTH+: USER_WIDTH] <=
s_axis_tuser[i*USER_WIDTH+: USER_WIDTH];
201     end
202   end
203
204   if (rst) begin
205     s_axis_tvalid_reg <= 0;
206   end
207 end
208
209 // output datapath logic
210 reg [DATA_WIDTH-1:0] m_axis_tdata_reg = {DATA_WIDTH{1'b0}};
211 reg [KEEP_WIDTH-1:0] m_axis_tkeep_reg = {KEEP_WIDTH{1'b0}};
212 reg m_axis_tvalid_reg = 1'b0, m_ax-
is_tvalid_next;
213 reg m_axis_tlast_reg = 1'b0;
214 reg [M_ID_WIDTH-1:0] m_axis_tid_reg = {M_ID_WIDTH{1'b0}};
215 reg [DEST_WIDTH-1:0] m_axis_tdest_reg = {DEST_WIDTH{1'b0}};
216 reg [USER_WIDTH-1:0] m_axis_tuser_reg = {USER_WIDTH{1'b0}};
217
218 reg [DATA_WIDTH-1:0] temp_m_axis_tdata_reg =
{DATA_WIDTH{1'b0}};
219 reg [KEEP_WIDTH-1:0] temp_m_axis_tkeep_reg =
{KEEP_WIDTH{1'b0}};
220 reg temp_m_axis_tvalid_reg = 1'b0, temp_m_ax-
is_tvalid_next;
221 reg temp_m_axis_tlast_reg = 1'b0;
222 reg [M_ID_WIDTH-1:0] temp_m_axis_tid_reg =
{M_ID_WIDTH{1'b0}};
223 reg [DEST_WIDTH-1:0] temp_m_axis_tdest_reg =
{DEST_WIDTH{1'b0}};
224 reg [USER_WIDTH-1:0] temp_m_axis_tuser_reg =
{USER_WIDTH{1'b0}};
225
226 // datapath control

```

```

227 reg store_axis_int_to_output;
228 reg store_axis_int_to_temp;
229 reg store_axis_temp_to_output;
230
231 assign m_axis_tdata = m_axis_tdata_reg;
232 assign m_axis_tkeep = KEEP_ENABLE ? m_axis_tkeep_reg :
{KEEP_WIDTH{1'b1}};
233 assign m_axis_tvalid = m_axis_tvalid_reg;
234 assign m_axis_tlast = LAST_ENABLE ? m_axis_tlast_reg : 1'b1;
235 assign m_axis_tid = ID_ENABLE ? m_axis_tid_reg :
{M_ID_WIDTH{1'b0}};
236 assign m_axis_tdest = DEST_ENABLE ? m_axis_tdest_reg :
{DEST_WIDTH{1'b0}};
237 assign m_axis_tuser = USER_ENABLE ? m_axis_tuser_reg :
{USER_WIDTH{1'b0}};
238
239 // enable ready input next cycle if output is ready or if both
output registers are empty
240 assign m_axis_tready_int_early = m_axis_tready || (!temp_m_ax-
is_tvalid_reg && !m_axis_tvalid_reg);
241
242 always @* begin
243 // transfer sink ready state to source
244 m_axis_tvalid_next = m_axis_tvalid_reg;
245 temp_m_axis_tvalid_next = temp_m_axis_tvalid_reg;
246
247 store_axis_int_to_output = 1'b0;
248 store_axis_int_to_temp = 1'b0;
249 store_axis_temp_to_output = 1'b0;
250
251 if (m_axis_tready_int_reg) begin
252 // input is ready
253 if (m_axis_tready || !m_axis_tvalid_reg) begin
254 // output is ready or currently not valid, trans-
fer data to output
255 m_axis_tvalid_next = m_axis_tvalid_int;
256 store_axis_int_to_output = 1'b1;
257 end else begin
258 // output is not ready, store input in temp
259 temp_m_axis_tvalid_next = m_axis_tvalid_int;
260 store_axis_int_to_temp = 1'b1;
261 end
262 end else if (m_axis_tready) begin
263 // input is not ready, but output is ready
264 m_axis_tvalid_next = temp_m_axis_tvalid_reg;
265 temp_m_axis_tvalid_next = 1'b0;
266 store_axis_temp_to_output = 1'b1;
267 end
268 end
269
270 always @(posedge clk) begin
271 m_axis_tvalid_reg <= m_axis_tvalid_next;
272 m_axis_tready_int_reg <= m_axis_tready_int_early;
273 temp_m_axis_tvalid_reg <= temp_m_axis_tvalid_next;
274
275 // datapath
276 if (store_axis_int_to_output) begin
277 m_axis_tdata_reg <= m_axis_tdata_int;
278 m_axis_tkeep_reg <= m_axis_tkeep_int;
279 m_axis_tlast_reg <= m_axis_tlast_int;
280 m_axis_tid_reg <= m_axis_tid_int;
281 m_axis_tdest_reg <= m_axis_tdest_int;
282 m_axis_tuser_reg <= m_axis_tuser_int;
283 end else if (store_axis_temp_to_output) begin
284 m_axis_tdata_reg <= temp_m_axis_tdata_reg;
285 m_axis_tkeep_reg <= temp_m_axis_tkeep_reg;
286 m_axis_tlast_reg <= temp_m_axis_tlast_reg;
287 m_axis_tid_reg <= temp_m_axis_tid_reg;
288 m_axis_tdest_reg <= temp_m_axis_tdest_reg;
289 m_axis_tuser_reg <= temp_m_axis_tuser_reg;
290 end
291
292 if (store_axis_int_to_temp) begin
293 temp_m_axis_tdata_reg <= m_axis_tdata_int;
294 temp_m_axis_tkeep_reg <= m_axis_tkeep_int;
295 temp_m_axis_tlast_reg <= m_axis_tlast_int;
296 temp_m_axis_tid_reg <= m_axis_tid_int;
297 temp_m_axis_tdest_reg <= m_axis_tdest_int;
298 temp_m_axis_tuser_reg <= m_axis_tuser_int;
299 end
300
301 if (rst) begin
302 m_axis_tvalid_reg <= 1'b0;
303 m_axis_tready_int_reg <= 1'b0;
304 temp_m_axis_tvalid_reg <= 1'b0;
305 end
306 end
307 endmodule
308
309 `resetall
310

```

3. alexforenrich_verilog-pcie/rtl/dma_client_axis_sink.v (635 lines)

```

1 /*
2
3 Copyright (c) 2019-2021 Alex Forenrich
4

```

```

5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * AXI stream sink DMA client
33 */
34 module dma_client_axis_sink #
35 (
36 // RAM address width
37 parameter RAM_ADDR_WIDTH = 16,
38 // RAM segment count
39 parameter SEG_COUNT = 2,
40 // RAM segment data width
41 parameter SEG_DATA_WIDTH = 64,
42 // RAM segment byte enable width
43 parameter SEG_BE_WIDTH = SEG_DATA_WIDTH/8,
44 // RAM segment address width
45 parameter SEG_ADDR_WIDTH = RAM_ADDR_WIDTH-$clo-
g2(SEG_COUNT*SEG_BE_WIDTH),
46 // Width of AXI stream interfaces in bits
47 parameter AXIS_DATA_WIDTH = SEG_DATA_WIDTH*SEG_COUNT/2,
48 // Use AXI stream tkeep signal
49 parameter AXIS_KEEP_ENABLE = (AXIS_DATA_WIDTH>8),
50 // AXI stream tkeep signal width (words per cycle)
51 parameter AXIS_KEEP_WIDTH = (AXIS_DATA_WIDTH/8),
52 // Use AXI stream tlast signal
53 parameter AXIS_LAST_ENABLE = 1,
54 // Propagate AXI stream tid signal
55 parameter AXIS_ID_ENABLE = 0,
56 // AXI stream tid signal width
57 parameter AXIS_ID_WIDTH = 8,
58 // Propagate AXI stream tdest signal
59 parameter AXIS_DEST_ENABLE = 0,
60 // AXI stream tdest signal width
61 parameter AXIS_DEST_WIDTH = 8,
62 // Propagate AXI stream tuser signal
63 parameter AXIS_USER_ENABLE = 1,
64 // AXI stream tuser signal width
65 parameter AXIS_USER_WIDTH = 1,
66 // Width of length field
67 parameter LEN_WIDTH = 16,
68 // Width of tag field
69 parameter TAG_WIDTH = 8
70 )
71 (
72 input wire clk,
73 input wire rst,
74
75 /*
76 * AXI write descriptor input
77 */
78 input wire [RAM_ADDR_WIDTH-1:0] s_ax-
is_write_desc_ram_addr,
79 input wire [LEN_WIDTH-1:0] s_ax-
is_write_desc_len,
80 input wire [TAG_WIDTH-1:0] s_ax-
is_write_desc_tag,
81 input wire s_ax-
is_write_desc_valid,
82 output wire s_ax-
is_write_desc_ready,
83
84 /*
85 * AXI write descriptor status output
86 */

```

```

87   output wire [LEN_WIDTH-1:0]          m_ax-
is_write_desc_status_len,                m_ax-
88   output wire [TAG_WIDTH-1:0]          m_ax-
is_write_desc_status_tag,                m_ax-
89   output wire [AXIS_ID_WIDTH-1:0]      m_ax-
is_write_desc_status_id,                 m_ax-
90   output wire [AXIS_DEST_WIDTH-1:0]    m_ax-
is_write_desc_status_dest,               m_ax-
91   output wire [AXIS_USER_WIDTH-1:0]    m_ax-
is_write_desc_status_user,               m_ax-
92   output wire [3:0]                    m_ax-
is_write_desc_status_error,              m_ax-
93   output wire                           m_ax-
is_write_desc_status_valid,
94
95   /*
96   * AXI stream write data input
97   */
98   input wire [AXIS_DATA_WIDTH-1:0]      s_ax-
is_write_data_tdata,                    s_ax-
99   input wire [AXIS_KEEP_WIDTH-1:0]      s_ax-
is_write_data_tkeep,                    s_ax-
100  input wire                             s_ax-
is_write_data_tvalid,                   s_ax-
101  output wire                             s_ax-
is_write_data_tready,                   s_ax-
102  input wire                             s_ax-
is_write_data_tlast,                    s_ax-
103  input wire [AXIS_ID_WIDTH-1:0]        s_ax-
is_write_data_tid,                      s_ax-
104  input wire [AXIS_DEST_WIDTH-1:0]      s_ax-
is_write_data_tdest,                    s_ax-
105  input wire [AXIS_USER_WIDTH-1:0]      s_ax-
is_write_data_tuser,
106
107  /*
108  * RAM interface
109  */
110  output wire [SEG_COUNT*SEG_BE_WIDTH-1:0] ram_wr_cmd_be,
111  output wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0] ram_wr_cmd_ad-
dr,
112  output wire [SEG_COUNT*SEG_DATA_WIDTH-1:0] ram_wr_cmd_data,
113  output wire [SEG_COUNT-1:0] ram_wr_cmd_valid,
114  input wire [SEG_COUNT-1:0] ram_wr_cmd_ready,
115  input wire [SEG_COUNT-1:0] ram_wr_done,
116
117  /*
118  * Configuration
119  */
120  input wire enable,
121  input wire abort
122 );
123
124 parameter RAM_WORD_WIDTH = SEG_BE_WIDTH;
125 parameter RAM_WORD_SIZE = SEG_DATA_WIDTH/RAM_WORD_WIDTH;
126
127 parameter AXIS_KEEP_WIDTH_INT = AXIS_KEEP_ENABLE ?
AXIS_KEEP_WIDTH : 1;
128 parameter AXIS_WORD_WIDTH = AXIS_KEEP_WIDTH_INT;
129 parameter AXIS_WORD_SIZE = AXIS_DATA_WIDTH/AXIS_WORD_WIDTH;
130
131 parameter PART_COUNT = SEG_COUNT*SEG_BE_WIDTH /
AXIS_KEEP_WIDTH_INT;
132 parameter PART_COUNT_WIDTH = PART_COUNT > 1 ? $clog-
2(PART_COUNT) : 1;
133 parameter PART_OFFSET_WIDTH = AXIS_KEEP_WIDTH_INT > 1 ? $clog-
2(AXIS_KEEP_WIDTH_INT) : 1;
134 parameter PARTS_PER_SEG = (SEG_BE_WIDTH + AXIS_KEEP_WIDTH_INT
- 1) / AXIS_KEEP_WIDTH_INT;
135 parameter SEG_PER_PART = (AXIS_KEEP_WIDTH_INT + SEG_BE_WIDTH
- 1) / SEG_BE_WIDTH;
136
137 parameter OFFSET_WIDTH = AXIS_KEEP_WIDTH_INT > 1 ? $clog-
2(AXIS_KEEP_WIDTH_INT) : 1;
138 parameter OFFSET_MASK = AXIS_KEEP_WIDTH_INT > 1 ? {OFF-
SET_WIDTH{1'b1}} : 0;
139 parameter ADDR_MASK = {RAM_ADDR_WIDTH{1'b1}} << $clog-
2(AXIS_KEEP_WIDTH_INT);
140 parameter CYCLE_COUNT_WIDTH = LEN_WIDTH - $clog-
2(AXIS_KEEP_WIDTH_INT) + 1;
141
142 parameter STATUS_FIFO_ADDR_WIDTH = 5;
143 parameter OUTPUT_FIFO_ADDR_WIDTH = 5;
144
145 // bus width assertions
146 initial begin
147   if (RAM_WORD_SIZE * SEG_BE_WIDTH != SEG_DATA_WIDTH) begin
148     $error("Error: RAM data width not evenly divisble (in-
stance %m)");
149   $finish;
150   end
151
152   if (AXIS_WORD_SIZE * AXIS_KEEP_WIDTH_INT !=
AXIS_DATA_WIDTH) begin
153     $error("Error: AXI stream data width not evenly divis-
ble (instance %m)");
154   $finish;
155   end

```

```

156
157   if (RAM_WORD_SIZE != AXIS_WORD_SIZE) begin
158     $error("Error: word size mismatch (instance %m)");
159   $finish;
160   end
161
162   if (2**$clog2(RAM_WORD_WIDTH) != RAM_WORD_WIDTH) begin
163     $error("Error: RAM word width must be even power of
two (instance %m)");
164   $finish;
165   end
166
167   if (RAM_ADDR_WIDTH != SEG_ADDR_WIDTH+$clog2(SEG_COUNT)+
$clog2(SEG_BE_WIDTH)) begin
168     $error("Error: RAM_ADDR_WIDTH does not match RAM con-
figuration (instance %m)");
169   $finish;
170   end
171
172   if (AXIS_DATA_WIDTH > SEG_COUNT*SEG_DATA_WIDTH) begin
173     $error("Error: AXI stream interface width must not be
wider than RAM interface width (instance %m)");
174   $finish;
175   end
176
177   if (AXIS_DATA_WIDTH*2**$clog2(PART_COUNT) !=
SEG_COUNT*SEG_DATA_WIDTH) begin
178     $error("Error: AXI stream interface width must be a
power of two fraction of RAM interface width (instance %m)");
179   $finish;
180   end
181 end
182
183 localparam [1:0]
184   STATE_IDLE = 2'd0,
185   STATE_WRITE = 2'd1,
186   STATE_DROP_DATA = 2'd2;
187
188 reg [1:0] state_reg = STATE_IDLE, state_next;
189
190 integer i;
191 reg [OFFSET_WIDTH:0] cycle_size;
192
193 reg [RAM_ADDR_WIDTH-1:0] addr_reg = {RAM_ADDR_WIDTH{1'b0}},
addr_next;
194 reg [AXIS_KEEP_WIDTH_INT-1:0] keep_mask_reg =
{AXIS_KEEP_WIDTH_INT{1'b0}}, keep_mask_next;
195 reg [OFFSET_WIDTH-1:0] last_cycle_offset_reg = {OFF-
SET_WIDTH{1'b0}}, last_cycle_offset_next;
196 reg [LEN_WIDTH-1:0] length_reg = {LEN_WIDTH{1'b0}},
length_next;
197 reg [CYCLE_COUNT_WIDTH-1:0] cycle_count_reg =
{CYCLE_COUNT_WIDTH{1'b0}}, cycle_count_next;
198 reg last_cycle_reg = 1'b0, last_cycle_next;
199
200 reg [TAG_WIDTH-1:0] tag_reg = {TAG_WIDTH{1'b0}}, tag_next;
201
202 reg [STATUS_FIFO_ADDR_WIDTH+1-1:0] status_fifo_wr_ptr_reg = 0;
203 reg [STATUS_FIFO_ADDR_WIDTH+1-1:0] status_fifo_rd_ptr_reg = 0,
status_fifo_rd_ptr_next;
204 reg [LEN_WIDTH-1:0] status_fifo_len[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
205 reg [TAG_WIDTH-1:0] status_fifo_tag[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
206 reg [AXIS_ID_WIDTH-1:0] status_fifo_id[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
207 reg [AXIS_DEST_WIDTH-1:0] status_fifo_dest[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
208 reg [AXIS_USER_WIDTH-1:0] status_fifo_user[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
209 reg [SEG_COUNT-1:0] status_fifo_mask[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
210 reg status_fifo_last[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
211 reg [LEN_WIDTH-1:0] status_fifo_wr_len;
212 reg [TAG_WIDTH-1:0] status_fifo_wr_tag;
213 reg [AXIS_ID_WIDTH-1:0] status_fifo_wr_id;
214 reg [AXIS_DEST_WIDTH-1:0] status_fifo_wr_dest;
215 reg [AXIS_USER_WIDTH-1:0] status_fifo_wr_user;
216 reg [SEG_COUNT-1:0] status_fifo_wr_mask;
217 reg status_fifo_wr_last;
218 reg status_fifo_we = 1'b0;
219 reg status_fifo_half_full_reg = 1'b0;
220
221 reg [STATUS_FIFO_ADDR_WIDTH+1-1:0] active_count_reg = 0;
222 reg active_count_av_reg = 1'b1;
223 reg inc_active;
224 reg dec_active;
225
226 reg s_axis_write_desc_ready_reg = 1'b0, s_ax-
is_write_desc_ready_next;
227
228 reg [LEN_WIDTH-1:0] m_axis_write_desc_status_len_reg =
{LEN_WIDTH{1'b0}}, m_axis_write_desc_status_len_next;
229 reg [TAG_WIDTH-1:0] m_axis_write_desc_status_tag_reg =
{TAG_WIDTH{1'b0}}, m_axis_write_desc_status_tag_next;
230 reg [AXIS_ID_WIDTH-1:0] m_axis_write_desc_status_id_reg =
{AXIS_ID_WIDTH{1'b0}}, m_axis_write_desc_status_id_next;
231 reg [AXIS_DEST_WIDTH-1:0] m_axis_write_desc_status_dest_reg =
{AXIS_DEST_WIDTH{1'b0}}, m_axis_write_desc_status_dest_next;
232 reg [AXIS_USER_WIDTH-1:0] m_axis_write_desc_status_user_reg =

```

```

[AXIS_USER_WIDTH{1'b0}], m_axis_write_desc_status_user_next;
233 reg m_axis_write_desc_status_valid_reg = 1'b0, m_ax-
is_write_desc_status_valid_next;
234
235 reg s_axis_write_data_tready_reg = 1'b0, s_ax-
is_write_data_tready_next;
236
237 // internal datapath
238 reg [SEG_COUNT*SEG_BE_WIDTH-1:0] ram_wr_cmd_be_int;
239 reg [SEG_COUNT*SEG_ADDR_WIDTH-1:0] ram_wr_cmd_addr_int;
240 reg [SEG_COUNT*SEG_DATA_WIDTH-1:0] ram_wr_cmd_data_int;
241 reg [SEG_COUNT-1:0] ram_wr_cmd_valid_int;
242 wire [SEG_COUNT-1:0] ram_wr_cmd_ready_int;
243
244 reg [SEG_COUNT-1:0] ram_wr_cmd_mask;
245
246 wire [SEG_COUNT-1:0] out_done;
247 reg [SEG_COUNT-1:0] out_done_ack;
248
249 assign s_axis_write_desc_ready = s_axis_write_desc_ready_reg;
250
251 assign m_axis_write_desc_status_len = m_ax-
is_write_desc_status_len_reg;
252 assign m_axis_write_desc_status_tag = m_ax-
is_write_desc_status_tag_reg;
253 assign m_axis_write_desc_status_id = m_ax-
is_write_desc_status_id_reg;
254 assign m_axis_write_desc_status_dest = m_ax-
is_write_desc_status_dest_reg;
255 assign m_axis_write_desc_status_user = m_ax-
is_write_desc_status_user_reg;
256 assign m_axis_write_desc_status_error = 4'd0;
257 assign m_axis_write_desc_status_valid = m_ax-
is_write_desc_status_valid_reg;
258
259 assign s_axis_write_data_tready = s_ax-
is_write_data_tready_reg;
260
261 always @* begin
262     state_next = STATE_IDLE;
263
264     s_axis_write_desc_ready_next = 1'b0;
265
266     m_axis_write_desc_status_len_next = m_ax-
is_write_desc_status_len_reg;
267     m_axis_write_desc_status_tag_next = m_ax-
is_write_desc_status_tag_reg;
268     m_axis_write_desc_status_id_next = m_ax-
is_write_desc_status_id_reg;
269     m_axis_write_desc_status_dest_next = m_ax-
is_write_desc_status_dest_reg;
270     m_axis_write_desc_status_user_next = m_ax-
is_write_desc_status_user_reg;
271     m_axis_write_desc_status_valid_next = 1'b0;
272
273     s_axis_write_data_tready_next = 1'b0;
274
275     if (PART_COUNT > 1) begin
276         ram_wr_cmd_be_int = (s_axis_write_data_tkeep &
keep_mask_reg) << (addr_reg & ({PART_COUNT_WIDTH{1'b1}} << PART_OFF-
SET_WIDTH));
277     end else begin
278         ram_wr_cmd_be_int = s_axis_write_data_tkeep &
keep_mask_reg;
279     end
280     ram_wr_cmd_addr_int =
{PART_COUNT{addr_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-SEG_AD-
DR_WIDTH]}};
281     ram_wr_cmd_data_int =
{PART_COUNT{s_axis_write_data_tdata}};
282     ram_wr_cmd_valid_int = {SEG_COUNT{1'b0}};
283     for (i = 0; i < SEG_COUNT; i = i + 1) begin
284         ram_wr_cmd_mask[i] = ram_wr_cmd_be_int[i*SEG_BE_WIDTH
+: SEG_BE_WIDTH] != 0;
285     end
286
287     cycle_size = AXIS_KEEP_WIDTH_INT;
288
289     addr_next = addr_reg;
290     keep_mask_next = keep_mask_reg;
291     last_cycle_offset_next = last_cycle_offset_reg;
292     length_next = length_reg;
293     cycle_count_next = cycle_count_reg;
294     last_cycle_next = last_cycle_reg;
295
296     tag_next = tag_reg;
297
298     status_fifo_rd_ptr_next = status_fifo_rd_ptr_reg;
299
300     status_fifo_wr_len = 0;
301     status_fifo_wr_tag = tag_reg;
302     status_fifo_wr_id = s_axis_write_data_tid;
303     status_fifo_wr_dest = s_axis_write_data_tdest;
304     status_fifo_wr_user = s_axis_write_data_tuser;
305     status_fifo_wr_mask = ram_wr_cmd_mask;
306     status_fifo_wr_last = 1'b0;
307     status_fifo_we = 1'b0;
308
309     inc_active = 1'b0;
310     dec_active = 1'b0;
311

```

```

312     out_done_ack = {SEG_COUNT{1'b0}};
313
314     case (state_reg)
315     STATE_IDLE: begin
316         // idle state - load new descriptor to start oper-
ation
317         s_axis_write_desc_ready_next = enable && act-
ive_count_av_reg;
318
319         addr_next = s_axis_write_desc_ram_addr &
ADDR_MASK;
320         last_cycle_offset_next = s_axis_write_desc_len &
OFFSET_MASK;
321
322         tag_next = s_axis_write_desc_tag;
323
324         length_next = 0;
325
326         cycle_count_next = (s_axis_write_desc_len - 1) >>
$log2(AXIS_KEEP_WIDTH_INT);
327         last_cycle_next = cycle_count_next == 0;
328         if (cycle_count_next == 0 &&
last_cycle_offset_next != 0) begin
329             keep_mask_next = {AXIS_KEEP_WIDTH_INT{1'b1}}
>> (AXIS_KEEP_WIDTH_INT - last_cycle_offset_next);
330         end else begin
331             keep_mask_next = {AXIS_KEEP_WIDTH_INT{1'b1}};
332         end
333
334         if (s_axis_write_desc_ready && s_ax-
is_write_desc_valid) begin
335             s_axis_write_desc_ready_next = 1'b0;
336             s_axis_write_data_tready_next =
&ram_wr_cmd_ready_int && !status_fifo_half_full_reg;
337
338             inc_active = 1'b1;
339
340             state_next = STATE_WRITE;
341         end else begin
342             state_next = STATE_IDLE;
343         end
344     end
345     STATE_WRITE: begin
346         // write state - generate write operations
347         s_axis_write_data_tready_next =
&ram_wr_cmd_ready_int && !status_fifo_half_full_reg;
348
349         if (s_axis_write_data_tready && s_ax-
is_write_data_tvalid) begin
350
351             // update counters
352             addr_next = addr_reg + AXIS_KEEP_WIDTH_INT;
353             length_next = length_reg +
AXIS_KEEP_WIDTH_INT;
354             cycle_count_next = cycle_count_reg - 1;
355             last_cycle_next = cycle_count_next == 0;
356             if (cycle_count_next == 0 &&
last_cycle_offset_reg != 0) begin
357                 keep_mask_next =
{AXIS_KEEP_WIDTH_INT{1'b1}} >> (AXIS_KEEP_WIDTH_INT -
last_cycle_offset_reg);
358             end else begin
359                 keep_mask_next =
{AXIS_KEEP_WIDTH_INT{1'b1}};
360             end
361
362             if (PART_COUNT > 1) begin
363                 ram_wr_cmd_be_int = (s_ax-
is_write_data_tkeep & keep_mask_reg) << (addr_reg &
({PART_COUNT_WIDTH{1'b1}} << PART_OFFSET_WIDTH));
364             end else begin
365                 ram_wr_cmd_be_int = s_ax-
is_write_data_tkeep & keep_mask_reg;
366             end
367             ram_wr_cmd_addr_int =
{SEG_COUNT{addr_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-SEG_AD-
DR_WIDTH]}};
368             ram_wr_cmd_data_int =
{PART_COUNT{s_axis_write_data_tdata}};
369             ram_wr_cmd_valid_int = ram_wr_cmd_mask;
370
371             // enqueue status FIFO entry for write comple-
tion
372             status_fifo_wr_len = length_next;
373             status_fifo_wr_tag = tag_reg;
374             status_fifo_wr_id = s_axis_write_data_tid;
375             status_fifo_wr_dest = s_axis_write_data_tdest;
376             status_fifo_wr_user = s_axis_write_data_tuser;
377             status_fifo_wr_mask = ram_wr_cmd_mask;
378             status_fifo_wr_last = 1'b0;
379             status_fifo_we = 1'b1;
380
381             if (AXIS_LAST_ENABLE && s_ax-
is_write_data_tlast) begin
382                 if (AXIS_KEEP_ENABLE) begin
383                     cycle_size = AXIS_KEEP_WIDTH_INT;
384                     for (i = AXIS_KEEP_WIDTH_INT-1; i >=
0; i = i - 1) begin
385                         if (~(s_axis_write_data_tkeep &
keep_mask_reg) & (1 << i)) begin
386                             cycle_size = i;

```

```

387         end
388     end
389 end else begin
390     cycle_size = AXIS_KEEP_WIDTH_INT;
391 end
392
393 // no more data to transfer, finish operation
394 if (last_cycle_reg &&
last_cycle_offset_reg > 0) begin
395     if (AXIS_KEEP_ENABLE && !(s_ax-
is_write_data_tkeep & keep_mask_reg & ~(AXIS_KEEP_WIDTH_INT{1'b1}}
>> (AXIS_KEEP_WIDTH_INT - last_cycle_offset_reg)))) begin
396         length_next = length_reg +
cycle_size;
397     end else begin
398         length_next = length_reg +
last_cycle_offset_reg;
399     end
400 end else begin
401     if (AXIS_KEEP_ENABLE) begin
402         length_next = length_reg +
cycle_size;
403     end
404 end
405
406 // enqueue status FIFO entry for write
completion
407     status_fifo_wr_len = length_next;
408     status_fifo_wr_tag = tag_reg;
409     status_fifo_wr_id = s_axis_write_data_tid;
410     status_fifo_wr_dest = s_ax-
is_write_data_tdest;
411     status_fifo_wr_user = s_ax-
is_write_data_tuser;
412     status_fifo_wr_mask = ram_wr_cmd_mask;
413     status_fifo_wr_last = 1'b1;
414     status_fifo_we = 1'b1;
415
416     s_axis_write_data_tready_next = 1'b0;
417     s_axis_write_desc_ready_next = enable &&
active_count_av_reg;
418     state_next = STATE_IDLE;
419 end else if (last_cycle_reg) begin
420     if (last_cycle_offset_reg > 0) begin
421         length_next = length_reg +
last_cycle_offset_reg;
422     end
423
424     // enqueue status FIFO entry for write
completion
425     status_fifo_wr_len = length_next;
426     status_fifo_wr_tag = tag_reg;
427     status_fifo_wr_id = s_axis_write_data_tid;
428     status_fifo_wr_dest = s_ax-
is_write_data_tdest;
429     status_fifo_wr_user = s_ax-
is_write_data_tuser;
430     status_fifo_wr_mask = ram_wr_cmd_mask;
431     status_fifo_wr_last = 1'b1;
432     status_fifo_we = 1'b1;
433
434     if (AXIS_LAST_ENABLE) begin
435         s_axis_write_data_tready_next = 1'b1;
436         state_next = STATE_DROP_DATA;
437     end else begin
438         s_axis_write_data_tready_next = 1'b0;
439         s_axis_write_desc_ready_next = enable
&& active_count_av_reg;
440         state_next = STATE_IDLE;
441     end
442 end else begin
443     state_next = STATE_WRITE;
444 end
445 end else begin
446     state_next = STATE_WRITE;
447 end
448 end
449 STATE_DROP_DATA: begin
450     // drop excess AXI stream data
451     s_axis_write_data_tready_next = 1'b1;
452
453     if (s_axis_write_data_tready && s_ax-
is_write_data_tvalid) begin
454         if (s_axis_write_data_tlast) begin
455             s_axis_write_data_tready_next = 1'b0;
456             s_axis_write_desc_ready_next = enable &&
active_count_av_reg;
457             state_next = STATE_IDLE;
458         end else begin
459             state_next = STATE_DROP_DATA;
460         end
461     end else begin
462         state_next = STATE_DROP_DATA;
463     end
464 end
465 endcase
466
467 m_axis_write_desc_status_len_next =
status_fifo_len[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]];
468 m_axis_write_desc_status_tag_next =

```

```

status_fifo_tag[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]];
469 m_axis_write_desc_status_id_next =
status_fifo_id[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]];
470 m_axis_write_desc_status_dest_next =
status_fifo_dest[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
;
471 m_axis_write_desc_status_user_next =
status_fifo_user[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
;
472 m_axis_write_desc_status_valid_next = 1'b0;
473
474 if (status_fifo_rd_ptr_reg != status_fifo_wr_ptr_reg) be-
gin
475     // status FIFO not empty
476     if
((status_fifo_mask[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]] & ~out_done) == 0) begin
477         // got write completion, pop and return status
478         status_fifo_rd_ptr_next = status_fifo_rd_ptr_reg +
1;
479
480         out_done_ack =
status_fifo_mask[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
;
481
482         if
(status_fifo_last[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]]) begin
483             m_axis_write_desc_status_valid_next = 1'b1;
484
485             dec_active = 1'b1;
486         end
487     end
488 end
489 end
490
491 always @(posedge clk) begin
492     state_reg <= state_next;
493
494     s_axis_write_desc_ready_reg <= s_ax-
is_write_desc_ready_next;
495
496     m_axis_write_desc_status_len_reg <= m_ax-
is_write_desc_status_len_next;
497     m_axis_write_desc_status_tag_reg <= m_ax-
is_write_desc_status_tag_next;
498     m_axis_write_desc_status_id_reg <= m_ax-
is_write_desc_status_id_next;
499     m_axis_write_desc_status_dest_reg <= m_ax-
is_write_desc_status_dest_next;
500     m_axis_write_desc_status_user_reg <= m_ax-
is_write_desc_status_user_next;
501     m_axis_write_desc_status_valid_reg <= m_ax-
is_write_desc_status_valid_next;
502
503     s_axis_write_data_tready_reg <= s_ax-
is_write_data_tready_next;
504
505     addr_reg <= addr_next;
506     keep_mask_reg <= keep_mask_next;
507     last_cycle_offset_reg <= last_cycle_offset_next;
508     length_reg <= length_next;
509     cycle_count_reg <= cycle_count_next;
510     last_cycle_reg <= last_cycle_next;
511
512     tag_reg <= tag_next;
513
514     if (status_fifo_we) begin
515
516         status_fifo_len[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_len;
517
518         status_fifo_tag[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_tag;
519
520         status_fifo_id[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_id;
521
522         status_fifo_dest[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_dest;
523
524         status_fifo_user[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_user;
525
526         status_fifo_mask[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_mask;
527
528         status_fifo_last[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_last;
529
530         status_fifo_wr_ptr_reg <= status_fifo_wr_ptr_reg + 1;
531
532         status_fifo_rd_ptr_reg <= status_fifo_rd_ptr_next;
533
534         status_fifo_half_full_reg <= $un-
signed(status_fifo_wr_ptr_reg - status_fifo_rd_ptr_reg) >=
2** (STATUS_FIFO_ADDR_WIDTH-1);
535
536         if (active_count_reg < 2**STATUS_FIFO_ADDR_WIDTH &&
inc_active && !dec_active) begin
537             active_count_reg <= active_count_reg + 1;

```



```

530     active_count_av_reg <= active_count_reg <
(2**STATUS_FIFO_ADDR_WIDTH-1);
531     end else if (active_count_reg > 0 && !inc_active &&
dec_active) begin
532         active_count_reg <= active_count_reg - 1;
533         active_count_av_reg <= 1'b1;
534     end else begin
535         active_count_av_reg <= active_count_reg <
2**STATUS_FIFO_ADDR_WIDTH;
536     end
537
538     if (rst) begin
539         state_reg <= STATE_IDLE;
540
541         s_axis_write_desc_ready_reg <= 1'b0;
542         m_axis_write_desc_status_valid_reg <= 1'b0;
543
544         s_axis_write_data_tready_reg <= 1'b0;
545
546         status_fifo_wr_ptr_reg <= 0;
547         status_fifo_rd_ptr_reg <= 0;
548
549         active_count_reg <= 0;
550         active_count_av_reg <= 1'b1;
551     end
552 end
553
554 // output datapath logic (write data)
555 generate
556     genvar n;
557     for (n = 0; n < SEG_COUNT; n = n + 1) begin
558
559         reg [SEG_BE_WIDTH-1:0] ram_wr_cmd_be_reg =
{SEG_BE_WIDTH{1'b0}};
560
561         reg [SEG_ADDR_WIDTH-1:0] ram_wr_cmd_addr_reg = {SEG_AD-
DR_WIDTH{1'b0}};
562
563         reg [SEG_DATA_WIDTH-1:0] ram_wr_cmd_data_reg =
{SEG_DATA_WIDTH{1'b0}};
564
565         reg ram_wr_cmd_valid_reg = 1'b0;
566
567         reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] out_fifo_wr_ptr_reg =
0;
568         reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] out_fifo_rd_ptr_reg =
0;
569
570         reg out_fifo_half_full_reg = 1'b0;
571
572         wire out_fifo_full = out_fifo_wr_ptr_reg ==
(out_fifo_rd_ptr_reg ^ {1'b1, {OUTPUT_FIFO_ADDR_WIDTH{1'b0}}});
573         wire out_fifo_empty = out_fifo_wr_ptr_reg ==
out_fifo_rd_ptr_reg;
574
575         (* ram_style = "distributed", ramstyle = "no_rw_check,
mlab" *)
576         reg [SEG_BE_WIDTH-1:0]
out_fifo_wr_cmd_be[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
577         (* ram_style = "distributed", ramstyle = "no_rw_check,
mlab" *)
578         reg [SEG_ADDR_WIDTH-1:0]
out_fifo_wr_cmd_addr[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
579         (* ram_style = "distributed", ramstyle = "no_rw_check,
mlab" *)
580         reg [SEG_DATA_WIDTH-1:0]
out_fifo_wr_cmd_data[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
581
582         reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] done_count_reg = 0;
583         reg done_reg = 1'b0;
584
585         assign ram_wr_cmd_ready_int[n +: 1] = !
out_fifo_half_full_reg;
586
587         assign ram_wr_cmd_be[n*SEG_BE_WIDTH +: SEG_BE_WIDTH] =
ram_wr_cmd_be_reg;
588
589         assign ram_wr_cmd_addr[n*SEG_ADDR_WIDTH +: SEG_ADDR_WIDTH] =
ram_wr_cmd_addr_reg;
590
591         assign ram_wr_cmd_data[n*SEG_DATA_WIDTH +: SEG_DATA_WIDTH] =
ram_wr_cmd_data_reg;
592
593         assign ram_wr_cmd_valid[n +: 1] = ram_wr_cmd_valid_reg;
594
595         assign out_done[n] = done_reg;
596
597         always @(posedge clk) begin
598             ram_wr_cmd_valid_reg <= ram_wr_cmd_valid_reg && !
ram_wr_cmd_ready[n +: 1];
599
600             out_fifo_half_full_reg <= $un-
signed(out_fifo_wr_ptr_reg - out_fifo_rd_ptr_reg) >= 2**(OUT-
PUT_FIFO_ADDR_WIDTH-1);
601
602             if (!out_fifo_full && ram_wr_cmd_valid_int[n +: 1])
begin
603                 out_fifo_wr_cmd_be[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
<= ram_wr_cmd_be_int[n*SEG_BE_WIDTH +: SEG_BE_WIDTH];
604
605                 out_fifo_wr_cmd_addr[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
<= ram_wr_cmd_addr_int[n*SEG_ADDR_WIDTH +: SEG_ADDR_WIDTH];
606
607                 out_fifo_wr_cmd_data[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
<= ram_wr_cmd_data_int[n*SEG_DATA_WIDTH +: SEG_DATA_WIDTH];
608
609                 ram_wr_cmd_valid_reg <= 1'b1;
610                 out_fifo_rd_ptr_reg <= out_fifo_rd_ptr_reg + 1;
611             end
612
613             if (done_count_reg < 2**OUTPUT_FIFO_ADDR_WIDTH &&
ram_wr_done[n] && !out_done_ack[n]) begin
614                 done_count_reg <= done_count_reg + 1;
615                 done_reg <= 1;
616             end else if (done_count_reg > 0 && !ram_wr_done[n] &&
out_done_ack[n]) begin
617                 done_count_reg <= done_count_reg - 1;
618                 done_reg <= done_count_reg > 1;
619             end
620
621             if (rst) begin
622                 out_fifo_wr_ptr_reg <= 0;
623                 out_fifo_rd_ptr_reg <= 0;
624                 ram_wr_cmd_valid_reg <= 1'b0;
625                 done_count_reg <= 0;
626                 done_reg <= 1'b0;
627             end
628         end
629     endgenerate
630 endmodule
631
632 `resetall
633
634 `resetall
635

```

```

out_fifo_wr_cmd_data[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
] <= ram_wr_cmd_data_int[n*SEG_DATA_WIDTH +: SEG_DATA_WIDTH];
601     out_fifo_wr_ptr_reg <= out_fifo_wr_ptr_reg + 1;
602     end
603
604     if (!out_fifo_empty && (!ram_wr_cmd_valid_reg ||
ram_wr_cmd_ready[n +: 1])) begin
605         ram_wr_cmd_be_reg <=
out_fifo_wr_cmd_be[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
606         ram_wr_cmd_addr_reg <=
out_fifo_wr_cmd_addr[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
];
607         ram_wr_cmd_data_reg <=
out_fifo_wr_cmd_data[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
];
608         ram_wr_cmd_valid_reg <= 1'b1;
609         out_fifo_rd_ptr_reg <= out_fifo_rd_ptr_reg + 1;
610     end
611
612     if (done_count_reg < 2**OUTPUT_FIFO_ADDR_WIDTH &&
ram_wr_done[n] && !out_done_ack[n]) begin
613         done_count_reg <= done_count_reg + 1;
614         done_reg <= 1;
615     end else if (done_count_reg > 0 && !ram_wr_done[n] &&
out_done_ack[n]) begin
616         done_count_reg <= done_count_reg - 1;
617         done_reg <= done_count_reg > 1;
618     end
619
620     if (rst) begin
621         out_fifo_wr_ptr_reg <= 0;
622         out_fifo_rd_ptr_reg <= 0;
623         ram_wr_cmd_valid_reg <= 1'b0;
624         done_count_reg <= 0;
625         done_reg <= 1'b0;
626     end
627 end
628
629 end
630
631 endgenerate
632
633 endmodule
634
635 `resetall

```

4. alexforench_verilog-pcie/rtl/dma_client_axis_source.v

(568 lines)

```

1 /*
2
3 Copyright (c) 2019-2021 Alex Forench
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * AXI stream source DMA client
33 */
34 module dma_client_axis_source #
35 (
36     // RAM address width
37     parameter RAM_ADDR_WIDTH = 16,
38     // RAM segment count

```

```

39 parameter SEG_COUNT = 2,
40 // RAM segment data width
41 parameter SEG_DATA_WIDTH = 64,
42 // RAM segment byte enable width
43 parameter SEG_BE_WIDTH = SEG_DATA_WIDTH/8,
44 // RAM segment address width
45 parameter SEG_ADDR_WIDTH = RAM_ADDR_WIDTH-$clog2(
g2(SEG_COUNT*SEG_BE_WIDTH)),
46 // Width of AXI stream interfaces in bits
47 parameter AXIS_DATA_WIDTH = SEG_DATA_WIDTH*SEG_COUNT/2,
48 // Use AXI stream tkeep signal
49 parameter AXIS_KEEP_ENABLE = (AXIS_DATA_WIDTH>8),
50 // AXI stream tkeep signal width (words per cycle)
51 parameter AXIS_KEEP_WIDTH = (AXIS_DATA_WIDTH/8),
52 // Use AXI stream tlast signal
53 parameter AXIS_LAST_ENABLE = 1,
54 // Propagate AXI stream tid signal
55 parameter AXIS_ID_ENABLE = 0,
56 // AXI stream tid signal width
57 parameter AXIS_ID_WIDTH = 8,
58 // Propagate AXI stream tdest signal
59 parameter AXIS_DEST_ENABLE = 0,
60 // AXI stream tdest signal width
61 parameter AXIS_DEST_WIDTH = 8,
62 // Propagate AXI stream tuser signal
63 parameter AXIS_USER_ENABLE = 1,
64 // AXI stream tuser signal width
65 parameter AXIS_USER_WIDTH = 1,
66 // Width of length field
67 parameter LEN_WIDTH = 16,
68 // Width of tag field
69 parameter TAG_WIDTH = 8
70 )
71 (
72 input wire clk,
73 input wire rst,
74
75 /*
76 * AXI read descriptor input
77 */
78 input wire [RAM_ADDR_WIDTH-1:0] s_ax-
is_read_desc_ram_addr,
79 input wire [LEN_WIDTH-1:0] s_ax-
is_read_desc_len,
80 input wire [TAG_WIDTH-1:0] s_ax-
is_read_desc_tag,
81 input wire [AXIS_ID_WIDTH-1:0] s_ax-
is_read_desc_id,
82 input wire [AXIS_DEST_WIDTH-1:0] s_ax-
is_read_desc_dest,
83 input wire [AXIS_USER_WIDTH-1:0] s_ax-
is_read_desc_user,
84 input wire s_ax-
is_read_desc_valid,
85 output wire s_ax-
is_read_desc_ready,
86
87 /*
88 * AXI read descriptor status output
89 */
90 output wire [TAG_WIDTH-1:0] m_ax-
is_read_desc_status_tag,
91 output wire [3:0] m_ax-
is_read_desc_status_error,
92 output wire m_ax-
is_read_desc_status_valid,
93
94 /*
95 * AXI stream read data output
96 */
97 output wire [AXIS_DATA_WIDTH-1:0] m_ax-
is_read_data_tdata,
98 output wire [AXIS_KEEP_WIDTH-1:0] m_ax-
is_read_data_tkeep,
99 output wire m_ax-
is_read_data_tvalid,
100 input wire m_ax-
is_read_data_tready,
101 output wire m_ax-
is_read_data_tlast,
102 output wire [AXIS_ID_WIDTH-1:0] m_ax-
is_read_data_tid,
103 output wire [AXIS_DEST_WIDTH-1:0] m_ax-
is_read_data_tdest,
104 output wire [AXIS_USER_WIDTH-1:0] m_ax-
is_read_data_tuser,
105
106 /*
107 * RAM interface
108 */
109 output wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0] ram_rd_cmd_ad-
dr,
110 output wire [SEG_COUNT-1:0] ram_rd_cmd_valid,
111 input wire [SEG_COUNT-1:0] ram_rd_cmd_ready,
112 input wire [SEG_COUNT*SEG_DATA_WIDTH-1:0] ram_rd_resp_data,
113 input wire [SEG_COUNT-1:0] ram_rd_resp_valid,
114 output wire [SEG_COUNT-1:0]

```

```

ram_rd_resp_ready,
115
116 /*
117 * Configuration
118 */
119 input wire enable
120 );
121
122 parameter RAM_WORD_WIDTH = SEG_BE_WIDTH;
123 parameter RAM_WORD_SIZE = SEG_DATA_WIDTH/RAM_WORD_WIDTH;
124
125 parameter AXIS_KEEP_WIDTH_INT = AXIS_KEEP_ENABLE ?
AXIS_KEEP_WIDTH : 1;
126 parameter AXIS_WORD_WIDTH = AXIS_KEEP_WIDTH_INT;
127 parameter AXIS_WORD_SIZE = AXIS_DATA_WIDTH/AXIS_WORD_WIDTH;
128
129 parameter PART_COUNT = SEG_COUNT*SEG_BE_WIDTH /
AXIS_KEEP_WIDTH_INT;
130 parameter PART_COUNT_WIDTH = PART_COUNT > 1 ? $clog2-
g2(PART_COUNT) : 1;
131 parameter PART_OFFSET_WIDTH = AXIS_KEEP_WIDTH_INT > 1 ? $clog2-
g2(AXIS_KEEP_WIDTH_INT) : 1;
132 parameter PARTS_PER_SEG = (SEG_BE_WIDTH + AXIS_KEEP_WIDTH_INT
- 1) / AXIS_KEEP_WIDTH_INT;
133 parameter SEG_SIZE_PER_PART = (AXIS_KEEP_WIDTH_INT + SEG_BE_WIDTH
- 1) / SEG_BE_WIDTH;
134
135 parameter OFFSET_WIDTH = AXIS_KEEP_WIDTH_INT > 1 ? $clog2-
g2(AXIS_KEEP_WIDTH_INT) : 1;
136 parameter OFFSET_MASK = AXIS_KEEP_WIDTH_INT > 1 ? {OFF-
SET_WIDTH{1'b1}} : 0;
137 parameter ADDR_MASK = {RAM_ADDR_WIDTH{1'b1}} << $clog2-
g2(AXIS_KEEP_WIDTH_INT);
138 parameter CYCLE_COUNT_WIDTH = LEN_WIDTH - $clog2-
g2(AXIS_KEEP_WIDTH_INT) + 1;
139
140 parameter OUTPUT_FIFO_ADDR_WIDTH = 5;
141
142 // bus width assertions
143 initial begin
144 if (RAM_WORD_SIZE * SEG_BE_WIDTH != SEG_DATA_WIDTH) begin
145 Error("Error: RAM data width not evenly divisble (in-
stance %m)");
146 $finish;
147 end
148
149 if (AXIS_WORD_SIZE * AXIS_KEEP_WIDTH_INT !=
AXIS_DATA_WIDTH) begin
150 Error("Error: AXI stream data width not evenly divis-
ble (instance %m)");
151 $finish;
152 end
153
154 if (RAM_WORD_SIZE != AXIS_WORD_SIZE) begin
155 Error("Error: word size mismatch (instance %m)");
156 $finish;
157 end
158
159 if (2**$clog2(RAM_WORD_WIDTH) != RAM_WORD_WIDTH) begin
160 Error("Error: RAM word width must be even power of
two (instance %m)");
161 $finish;
162 end
163
164 if (RAM_ADDR_WIDTH != SEG_ADDR_WIDTH+$clog2(SEG_COUNT)+
$clog2(SEG_BE_WIDTH)) begin
165 Error("Error: RAM_ADDR_WIDTH does not match RAM con-
figuration (instance %m)");
166 $finish;
167 end
168
169 if (AXIS_DATA_WIDTH > SEG_COUNT*SEG_DATA_WIDTH) begin
170 Error("Error: AXI stream interface width must not be
wider than RAM interface width (instance %m)");
171 $finish;
172 end
173
174 if (AXIS_DATA_WIDTH*2**$clog2(PART_COUNT) !=
SEG_COUNT*SEG_DATA_WIDTH) begin
175 Error("Error: AXI stream interface width must be a
power of two fraction of RAM interface width (instance %m)");
176 $finish;
177 end
178 end
179
180 localparam [0:0]
181 READ_STATE_IDLE = 1'd0,
182 READ_STATE_READ = 1'd1;
183
184 reg [0:0] read_state_reg = READ_STATE_IDLE, read_state_next;
185
186 localparam [0:0]
187 AXIS_STATE_IDLE = 1'd0,
188 AXIS_STATE_READ = 1'd1;
189
190 reg [0:0] axis_state_reg = AXIS_STATE_IDLE, axis_state_next;
191
192 // datapath control signals
193 reg axis_cmd_ready;
194
195 integer i;

```



```

196
197 reg [RAM_ADDR_WIDTH-1:0] read_addr_reg = {RAM_AD-
DR_WIDTH{1'b0}}, read_addr_next;
198 reg [SEG_COUNT-1:0] read_ram_mask_reg = 0, read_ram_mask_next;
199 reg [CYCLE_COUNT_WIDTH-1:0] read_cycle_count_reg =
{CYCLE_COUNT_WIDTH{1'b0}}, read_cycle_count_next;
200
201 reg [RAM_ADDR_WIDTH-1:0] axis_cmd_addr_reg = {RAM_AD-
DR_WIDTH{1'b0}}, axis_cmd_addr_next;
202 reg [OFFSET_WIDTH-1:0] axis_cmd_last_cycle_offset_reg = {OFF-
SET_WIDTH{1'b0}}, axis_cmd_last_cycle_offset_next;
203 reg [CYCLE_COUNT_WIDTH-1:0] axis_cmd_cycle_count_reg =
{CYCLE_COUNT_WIDTH{1'b0}}, axis_cmd_cycle_count_next;
204 reg [TAG_WIDTH-1:0] axis_cmd_tag_reg = {TAG_WIDTH{1'b0}},
axis_cmd_tag_next;
205 reg [AXIS_ID_WIDTH-1:0] axis_cmd_axis_id_reg =
{AXIS_ID_WIDTH{1'b0}}, axis_cmd_axis_id_next;
206 reg [AXIS_DEST_WIDTH-1:0] axis_cmd_axis_dest_reg = {AXIS_D-
EST_WIDTH{1'b0}}, axis_cmd_axis_dest_next;
207 reg [AXIS_USER_WIDTH-1:0] axis_cmd_axis_user_reg =
{AXIS_USER_WIDTH{1'b0}}, axis_cmd_axis_user_next;
208 reg axis_cmd_valid_reg = 1'b0, axis_cmd_valid_next;
209
210 reg [RAM_ADDR_WIDTH-1:0] addr_reg = {RAM_ADDR_WIDTH{1'b0}},
addr_next;
211 reg [SEG_COUNT-1:0] ram_mask_reg = 0, ram_mask_next;
212 reg [OFFSET_WIDTH-1:0] last_cycle_offset_reg = {OFF-
SET_WIDTH{1'b0}}, last_cycle_offset_next;
213 reg [CYCLE_COUNT_WIDTH-1:0] cycle_count_reg =
{CYCLE_COUNT_WIDTH{1'b0}}, cycle_count_next;
214 reg last_cycle_reg = 1'b0, last_cycle_next;
215
216 reg [AXIS_ID_WIDTH-1:0] axis_id_reg = {AXIS_ID_WIDTH{1'b0}},
axis_id_next;
217 reg [AXIS_DEST_WIDTH-1:0] axis_dest_reg = {AXIS_D-
EST_WIDTH{1'b0}}, axis_dest_next;
218 reg [AXIS_USER_WIDTH-1:0] axis_user_reg =
{AXIS_USER_WIDTH{1'b0}}, axis_user_next;
219
220 reg s_axis_read_desc_ready_reg = 1'b0, s_ax-
is_read_desc_ready_next;
221
222 reg [TAG_WIDTH-1:0] m_axis_read_desc_status_tag_reg =
{TAG_WIDTH{1'b0}}, m_axis_read_desc_status_tag_next;
223 reg m_axis_read_desc_status_valid_reg = 1'b0, m_ax-
is_read_desc_status_valid_next;
224
225 reg [SEG_COUNT*SEG_ADDR_WIDTH-1:0] ram_rd_cmd_addr_reg = 0,
ram_rd_cmd_addr_next;
226 reg [SEG_COUNT-1:0] ram_rd_cmd_valid_reg = 0, ram_rd_cmd_val-
id_next;
227 reg [SEG_COUNT-1:0] ram_rd_resp_ready_cmb;
228
229 // internal datapath
230 reg [AXIS_DATA_WIDTH-1:0] m_axis_read_data_tdata_int;
231 reg [AXIS_KEEP_WIDTH-1:0] m_axis_read_data_tkeep_int;
232 reg [AXIS_DATA_WIDTH-1:0] m_axis_read_data_tvalid_int;
233 wire [AXIS_DATA_WIDTH-1:0] m_axis_read_data_tready_int;
234 reg [AXIS_DATA_WIDTH-1:0] m_axis_read_data_tlast_int;
235 reg [AXIS_ID_WIDTH-1:0] m_axis_read_data_tid_int;
236 reg [AXIS_DEST_WIDTH-1:0] m_axis_read_data_tdest_int;
237 reg [AXIS_USER_WIDTH-1:0] m_axis_read_data_tuser_int;
238
239 assign s_axis_read_desc_ready = s_axis_read_desc_ready_reg;
240
241 assign m_axis_read_desc_status_tag = m_ax-
is_read_desc_status_tag_reg;
242 assign m_axis_read_desc_status_error = 4'd0;
243 assign m_axis_read_desc_status_valid = m_ax-
is_read_desc_status_valid_reg;
244
245 assign ram_rd_cmd_addr = ram_rd_cmd_addr_reg;
246 assign ram_rd_cmd_valid = ram_rd_cmd_valid_reg;
247 assign ram_rd_resp_ready = ram_rd_resp_ready_cmb;
248
249 always @* begin
250     read_state_next = READ_STATE_IDLE;
251
252     s_axis_read_desc_ready_next = 1'b0;
253
254     ram_rd_cmd_addr_next = ram_rd_cmd_addr_reg;
255     ram_rd_cmd_valid_next = ram_rd_cmd_valid_reg &
~ram_rd_cmd_ready;
256
257     read_addr_next = read_addr_reg;
258     read_ram_mask_next = read_ram_mask_reg;
259     read_cycle_count_next = read_cycle_count_reg;
260
261     axis_cmd_addr_next = axis_cmd_addr_reg;
262     axis_cmd_last_cycle_offset_next =
axis_cmd_last_cycle_offset_reg;
263     axis_cmd_cycle_count_next = axis_cmd_cycle_count_reg;
264     axis_cmd_tag_next = axis_cmd_tag_reg;
265     axis_cmd_axis_id_next = axis_cmd_axis_id_reg;
266     axis_cmd_axis_dest_next = axis_cmd_axis_dest_reg;
267     axis_cmd_axis_user_next = axis_cmd_axis_user_reg;
268     axis_cmd_valid_next = axis_cmd_valid_reg && !
axis_cmd_ready;
269
270     case (read_state_reg)
271         READ_STATE_IDLE: begin

```

```

272         // idle state - load new descriptor to start oper-
ation
273         s_axis_read_desc_ready_next = !axis_cmd_valid_reg
&& enable;
274
275         if (s_axis_read_desc_ready && s_ax-
is_read_desc_valid) begin
276             read_addr_next = s_axis_read_desc_ram_addr &
ADDR_MASK;
277
278             if (PART_COUNT > 1) begin
279
280                 read_ram_mask_next = {SEGS_PER_PART{1'b1}}
<< (((read_addr_next >> PART_OFFSET_WIDTH) &
({PART_COUNT_WIDTH{1'b1}})) / PARTS_PER_SEG) * SEGS_PER_PART);
281                 end else begin
282                     read_ram_mask_next = {SEG_COUNT{1'b1}};
283                 end
284
285                 axis_cmd_addr_next = s_axis_read_desc_ram_addr &
ADDR_MASK;
286                 axis_cmd_last_cycle_offset_next = s_ax-
is_read_desc_len & OFFSET_MASK;
287
288                 axis_cmd_tag_next = s_axis_read_desc_tag;
289
290                 axis_cmd_axis_id_next = s_axis_read_desc_id;
291                 axis_cmd_axis_dest_next = s_ax-
is_read_desc_dest;
292                 axis_cmd_axis_user_next = s_ax-
is_read_desc_user;
293
294                 axis_cmd_cycle_count_next = (s_ax-
is_read_desc_len - 1) >> $clog2(AXIS_KEEP_WIDTH_INT);
295                 read_cycle_count_next = (s_axis_read_desc_len
- 1) >> $clog2(AXIS_KEEP_WIDTH_INT);
296
297                 axis_cmd_valid_next = 1'b1;
298
299                 s_axis_read_desc_ready_next = 1'b0;
300                 read_state_next = READ_STATE_READ;
301             end else begin
302                 read_state_next = READ_STATE_IDLE;
303             end
304         end
305         READ_STATE_READ: begin
306             // read state - start new read operations
307
308             if (!(ram_rd_cmd_valid & ~ram_rd_cmd_ready &
read_ram_mask_reg)) begin
309
310                 // update counters
311                 read_addr_next = read_addr_reg +
AXIS_KEEP_WIDTH_INT;
312
313                 read_cycle_count_next = read_cycle_count_reg -
1;
314
315                 if (PART_COUNT > 1) begin
316
317                     read_ram_mask_next = {SEGS_PER_PART{1'b1}}
<< (((read_addr_next >> PART_OFFSET_WIDTH) &
({PART_COUNT_WIDTH{1'b1}})) / PARTS_PER_SEG) * SEGS_PER_PART);
318                     end else begin
319                         read_ram_mask_next = {SEG_COUNT{1'b1}};
320                     end
321
322                     for (i = 0; i < SEG_COUNT; i = i + 1) begin
323                         if (read_ram_mask_reg[i]) begin
324                             ram_rd_cmd_addr_next[i*SEG_ADDR_WIDTH
+: SEG_ADDR_WIDTH] = read_addr_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-
SEG_ADDR_WIDTH];
325                             ram_rd_cmd_valid_next[i] = 1'b1;
326                         end
327                     end
328
329                     if (read_cycle_count_reg == 0) begin
330                         s_axis_read_desc_ready_next = !
axis_cmd_valid_reg && enable;
331                         read_state_next = READ_STATE_IDLE;
332                     end else begin
333                         read_state_next = READ_STATE_READ;
334                     end
335                 end
336             end
337         endcase
338     end
339
340     always @* begin
341         axis_state_next = AXIS_STATE_IDLE;
342
343         m_axis_read_desc_status_tag_next = m_ax-
is_read_desc_status_tag_reg;
344         m_axis_read_desc_status_valid_next = 1'b0;
345
346         if (PART_COUNT > 1) begin
347             m_axis_read_data_rdata_int = ram_rd_resp_data >>

```

```

(((addr_reg >> PART_OFFSET_WIDTH) & {PART_COUNT_WIDTH{1'b1}}) *
AXIS_DATA_WIDTH);
348 end else begin
349     m_axis_read_data_tdata_int = ram_rd_resp_data;
350 end
351 m_axis_read_data_tkeep_int = {AXIS_KEEP_WIDTH{1'b1}};
352 m_axis_read_data_tlast_int = 1'b0;
353 m_axis_read_data_tvalid_int = 1'b0;
354 m_axis_read_data_tid_int = axis_id_reg;
355 m_axis_read_data_tdest_int = axis_dest_reg;
356 m_axis_read_data_tuser_int = axis_user_reg;
357
358 ram_rd_resp_ready_cmb = {SEG_COUNT{1'b0}};
359
360 axis_cmd_ready = 1'b0;
361
362 addr_next = addr_reg;
363 ram_mask_next = ram_mask_reg;
364 last_cycle_offset_next = last_cycle_offset_reg;
365 cycle_count_next = cycle_count_reg;
366 last_cycle_next = last_cycle_reg;
367
368 axis_id_next = axis_id_reg;
369 axis_dest_next = axis_dest_reg;
370 axis_user_next = axis_user_reg;
371
372 case (axis_state_reg)
373     AXIS_STATE_IDLE: begin
374         // idle state - load new descriptor to start oper-
375         ation
376         // store transfer parameters
377         addr_next = axis_cmd_addr_reg;
378         last_cycle_offset_next =
axis_cmd_last_cycle_offset_reg;
379         cycle_count_next = axis_cmd_cycle_count_reg;
380         last_cycle_next = axis_cmd_cycle_count_reg == 0;
381
382         if (PART_COUNT > 1) begin
383             ram_mask_next = {SEGS_PER_PART{1'b1}} <<
(((addr_next >> PART_OFFSET_WIDTH) & ({PART_COUNT_WIDTH{1'b1}})) /
PARTS_PER_SEG) * SEGS_PER_PART;
384         end else begin
385             ram_mask_next = {SEG_COUNT{1'b1}};
386         end
387
388         m_axis_read_desc_status_tag_next =
axis_cmd_tag_reg;
389         axis_id_next = axis_cmd_axis_id_reg;
390         axis_dest_next = axis_cmd_axis_dest_reg;
391         axis_user_next = axis_cmd_axis_user_reg;
392
393         if (axis_cmd_valid_reg) begin
394             axis_cmd_ready = 1'b1;
395             axis_state_next = AXIS_STATE_READ;
396         end
397     end
398     AXIS_STATE_READ: begin
399         // handle read data
400         ram_rd_resp_ready_cmb = {SEG_COUNT{1'b0}};
401
402         if (!(ram_mask_reg & ~ram_rd_resp_valid) && m_ax-
is_read_data_tready_int) begin
403             // transfer in read data
404             ram_rd_resp_ready_cmb = ram_mask_reg;
405
406             // update counters
407             addr_next = addr_reg + AXIS_KEEP_WIDTH_INT;
408             cycle_count_next = cycle_count_reg - 1;
409             last_cycle_next = cycle_count_next == 0;
410
411             if (PART_COUNT > 1) begin
412                 ram_mask_next = {SEGS_PER_PART{1'b1}} <<
(((addr_next >> PART_OFFSET_WIDTH) & ({PART_COUNT_WIDTH{1'b1}})) /
PARTS_PER_SEG) * SEGS_PER_PART;
413             end else begin
414                 ram_mask_next = {SEG_COUNT{1'b1}};
415             end
416
417             if (PART_COUNT > 1) begin
418                 m_axis_read_data_tdata_int =
ram_rd_resp_data >> (((addr_reg >> PART_OFFSET_WIDTH) &
{PART_COUNT_WIDTH{1'b1}}) * AXIS_DATA_WIDTH);
419             end else begin
420                 m_axis_read_data_tdata_int =
ram_rd_resp_data;
421             end
422             m_axis_read_data_tkeep_int =
{AXIS_KEEP_WIDTH_INT{1'b1}};
423             m_axis_read_data_tvalid_int = 1'b1;
424
425             if (last_cycle_reg) begin
426                 // no more data to transfer, finish opera-
427                 tion
428                 if (last_cycle_offset_reg > 0) begin
429                     m_axis_read_data_tkeep_int =
{AXIS_KEEP_WIDTH_INT{1'b1}} >> (AXIS_KEEP_WIDTH_INT -
last_cycle_offset_reg);
430                 end
431                 m_axis_read_data_tlast_int = 1'b1;
432
433                 m_axis_read_desc_status_valid_next = 1'b1;
434
435                 axis_state_next = AXIS_STATE_IDLE;
436             end else begin
437                 // more cycles in AXI transfer
438                 axis_state_next = AXIS_STATE_READ;
439             end
440         end else begin
441             axis_state_next = AXIS_STATE_READ;
442         end
443     endcase
444 end
445
446 always @(posedge clk) begin
447     read_state_reg <= read_state_next;
448     axis_state_reg <= axis_state_next;
449
450     s_axis_read_desc_ready_reg <= s_axis_read_desc_ready_next;
451
452     m_axis_read_desc_status_tag_reg <= m_ax-
is_read_desc_status_tag_next;
453     m_axis_read_desc_status_valid_reg <= m_ax-
is_read_desc_status_valid_next;
454
455     ram_rd_cmd_addr_reg <= ram_rd_cmd_addr_next;
456     ram_rd_cmd_valid_reg <= ram_rd_cmd_valid_next;
457
458     read_addr_reg <= read_addr_next;
459     read_ram_mask_reg <= read_ram_mask_next;
460     read_cycle_count_reg <= read_cycle_count_next;
461
462     axis_cmd_addr_reg <= axis_cmd_addr_next;
463     axis_cmd_last_cycle_offset_reg <=
axis_cmd_last_cycle_offset_next;
464     axis_cmd_cycle_count_reg <= axis_cmd_cycle_count_next;
465     axis_cmd_tag_reg <= axis_cmd_tag_next;
466     axis_cmd_axis_id_reg <= axis_cmd_axis_id_next;
467     axis_cmd_axis_dest_reg <= axis_cmd_axis_dest_next;
468     axis_cmd_axis_user_reg <= axis_cmd_axis_user_next;
469     axis_cmd_valid_reg <= axis_cmd_valid_next;
470
471     addr_reg <= addr_next;
472     ram_mask_reg <= ram_mask_next;
473     last_cycle_offset_reg <= last_cycle_offset_next;
474     cycle_count_reg <= cycle_count_next;
475     last_cycle_reg <= last_cycle_next;
476
477     axis_id_reg <= axis_id_next;
478     axis_dest_reg <= axis_dest_next;
479     axis_user_reg <= axis_user_next;
480
481     if (rst) begin
482         read_state_reg <= READ_STATE_IDLE;
483         axis_state_reg <= AXIS_STATE_IDLE;
484
485         axis_cmd_valid_reg <= 1'b0;
486
487         s_axis_read_desc_ready_reg <= 1'b0;
488         m_axis_read_desc_status_valid_reg <= 1'b0;
489
490         ram_rd_cmd_valid_reg <= {SEG_COUNT{1'b0}};
491     end
492 end
493
494 // output datapath logic
495 reg [AXIS_DATA_WIDTH-1:0] m_axis_read_data_tdata_reg =
{AXIS_DATA_WIDTH{1'b0}};
496 reg [AXIS_KEEP_WIDTH-1:0] m_axis_read_data_tkeep_reg =
{AXIS_KEEP_WIDTH{1'b0}};
497 reg [AXIS_DATA_WIDTH-1:0] m_axis_read_data_tvalid_reg = 1'b0;
498 reg [AXIS_DATA_WIDTH-1:0] m_axis_read_data_tlast_reg = 1'b0;
499 reg [AXIS_ID_WIDTH-1:0] m_axis_read_data_tid_reg =
{AXIS_ID_WIDTH{1'b0}};
500 reg [AXIS_DEST_WIDTH-1:0] m_axis_read_data_tdest_reg =
{AXIS_DEST_WIDTH{1'b0}};
501 reg [AXIS_USER_WIDTH-1:0] m_axis_read_data_tuser_reg =
{AXIS_USER_WIDTH{1'b0}};
502
503 reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] out_fifo_wr_ptr_reg = 0;
504 reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] out_fifo_rd_ptr_reg = 0;
505 reg out_fifo_half_full_reg = 1'b0;
506
507 wire out_fifo_full = out_fifo_wr_ptr_reg ==
(out_fifo_rd_ptr_reg ^ {1'b1, {OUTPUT_FIFO_ADDR_WIDTH{1'b0}}});
508 wire out_fifo_empty = out_fifo_wr_ptr_reg ==
out_fifo_rd_ptr_reg;
509
510 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
511 reg [AXIS_DATA_WIDTH-1:0]
out_fifo_tdata[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
512 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
513 reg [AXIS_KEEP_WIDTH-1:0]
out_fifo_tkeep[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
514 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
515 reg
out_fifo_tlast[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
516 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"

```

```

*)
517 reg [AXIS_ID_WIDTH-1:0]
out_fifo_tid[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
518 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
519 reg [AXIS_DEST_WIDTH-1:0]
out_fifo_tdest[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
520 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
521 reg [AXIS_USER_WIDTH-1:0]
out_fifo_tuser[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
522
523 assign m_axis_read_data_tready_int = !out_fifo_half_full_reg;
524
525 assign m_axis_read_data_tdata = m_axis_read_data_tdata_reg;
526 assign m_axis_read_data_tkeep = AXIS_KEEP_ENABLE ? m_ax-
is_read_data_tkeep_reg : {AXIS_KEEP_WIDTH{1'b1}};
527 assign m_axis_read_data_tvalid = m_axis_read_data_tvalid_reg;
528 assign m_axis_read_data_tlast = AXIS_LAST_ENABLE ? m_ax-
is_read_data_tlast_reg : 1'b1;
529 assign m_axis_read_data_tid = AXIS_ID_ENABLE ? m_ax-
is_read_data_tid_reg : {AXIS_ID_WIDTH{1'b0}};
530 assign m_axis_read_data_tdest = AXIS_DEST_ENABLE ? m_ax-
is_read_data_tdest_reg : {AXIS_DEST_WIDTH{1'b0}};
531 assign m_axis_read_data_tuser = AXIS_USER_ENABLE ? m_ax-
is_read_data_tuser_reg : {AXIS_USER_WIDTH{1'b0}};
532
533 always @(posedge clk) begin
534     m_axis_read_data_tvalid_reg <= m_axis_read_data_tvalid_reg
&& !m_axis_read_data_tready;
535
536     out_fifo_half_full_reg <= $unsigned(out_fifo_wr_ptr_reg -
out_fifo_rd_ptr_reg) >= 2**((OUTPUT_FIFO_ADDR_WIDTH-1);
537
538     if (!out_fifo_full && m_axis_read_data_tvalid_int) begin
539
out_fifo_tdata[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axis_read_data_tdata_int;
540
out_fifo_tkeep[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axis_read_data_tkeep_int;
541
out_fifo_tlast[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axis_read_data_tlast_int;
542
out_fifo_tid[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axis_read_data_tid_int;
543
out_fifo_tdest[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axis_read_data_tdest_int;
544
out_fifo_tuser[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axis_read_data_tuser_int;
545         out_fifo_wr_ptr_reg <= out_fifo_wr_ptr_reg + 1;
546     end
547
548     if (!out_fifo_empty && (!m_axis_read_data_tvalid_reg ||
m_axis_read_data_tready)) begin
549         m_axis_read_data_tdata_reg <=
out_fifo_tdata[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
550         m_axis_read_data_tkeep_reg <=
out_fifo_tkeep[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
551         m_axis_read_data_tvalid_reg <= 1'b1;
552         m_axis_read_data_tlast_reg <=
out_fifo_tlast[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
553         m_axis_read_data_tid_reg <=
out_fifo_tid[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
554         m_axis_read_data_tdest_reg <=
out_fifo_tdest[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
555         m_axis_read_data_tuser_reg <=
out_fifo_tuser[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
556         out_fifo_rd_ptr_reg <= out_fifo_rd_ptr_reg + 1;
557     end
558
559     if (rst) begin
560         out_fifo_wr_ptr_reg <= 0;
561         out_fifo_rd_ptr_reg <= 0;
562         m_axis_read_data_tvalid_reg <= 1'b0;
563     end
564 end
565 endmodule
566
567 `resetall
568

```

5. alexforenrich_verilog-pcie/rtl/dma_if_axi.v (426 lines)

```

1 /*
2
3 Copyright (c) 2021 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell

```

```

9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * AXI DMA interface
33 */
34 module dma_if_axi #
35 (
36     // Width of AXI data bus in bits
37     parameter AXI_DATA_WIDTH = 32,
38     // Width of AXI address bus in bits
39     parameter AXI_ADDR_WIDTH = 16,
40     // Width of AXI wstrb (width of data bus in words)
41     parameter AXI_STRB_WIDTH = (AXI_DATA_WIDTH/8),
42     // Width of AXI ID signal
43     parameter AXI_ID_WIDTH = 8,
44     // Maximum AXI burst length to generate
45     parameter AXI_MAX_BURST_LEN = 256,
46     // RAM select width
47     parameter RAM_SEL_WIDTH = 2,
48     // RAM address width
49     parameter RAM_ADDR_WIDTH = 16,
50     // RAM segment count
51     parameter RAM_SEG_COUNT = 2,
52     // RAM segment data width
53     parameter RAM_SEG_DATA_WIDTH = AXI_DATA_WIDTH*2/
RAM_SEG_COUNT,
54     // RAM segment byte enable width
55     parameter RAM_SEG_BE_WIDTH = RAM_SEG_DATA_WIDTH/8,
56     // RAM segment address width
57     parameter RAM_SEG_ADDR_WIDTH = RAM_ADDR_WIDTH-$clo-
g2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH),
58     // Immediate enable
59     parameter IMM_ENABLE = 0,
60     // Immediate width
61     parameter IMM_WIDTH = 32,
62     // Length field width
63     parameter LEN_WIDTH = 16,
64     // Tag field width
65     parameter TAG_WIDTH = 8,
66     // Operation table size (read)
67     parameter READ_OP_TABLE_SIZE = 2**(AXI_ID_WIDTH),
68     // Operation table size (write)
69     parameter WRITE_OP_TABLE_SIZE = 2**(AXI_ID_WIDTH),
70     // Use AXI ID signals (read)
71     parameter READ_USE_AXI_ID = 0,
72     // Use AXI ID signals (write)
73     parameter WRITE_USE_AXI_ID = 1
74 )
75 (
76     input wire clk,
77     input wire rst,
78
79     /*
80     * AXI master interface
81     */
82     output wire [AXI_ID_WIDTH-1:0]
m_axi_awid,
83     output wire [AXI_ADDR_WIDTH-1:0]
m_axi_awaddr,
84     output wire [7:0]
m_axi_awlen,
85     output wire [2:0]
m_axi_awsiz,
86     output wire [1:0]
m_axi_awburst,
87     output wire
m_axi_awlock,
88     output wire [3:0]
m_axi_awcache,
89     output wire [2:0]
m_axi_awprot,
90     output wire

```

```

m_axi_awvalid,
91   input wire
m_axi_awready,
92   output wire [AXI_DATA_WIDTH-1:0]
m_axi_wdata,
93   output wire [AXI_STRB_WIDTH-1:0]
m_axi_wstrb,
94   output wire
m_axi_wlast,
95   output wire
m_axi_wvalid,
96   input wire
m_axi_wready,
97   input wire [AXI_ID_WIDTH-1:0]
m_axi_bid,
98   input wire [1:0]
m_axi_bresp,
99   input wire
m_axi_bvalid,
100  output wire
m_axi_bready,
101  output wire [AXI_ID_WIDTH-1:0]
m_axi_arid,
102  output wire [AXI_ADDR_WIDTH-1:0]
m_axi_araddr,
103  output wire [7:0]
m_axi_arlen,
104  output wire [2:0]
m_axi_arsize,
105  output wire [1:0]
m_axi_arburst,
106  output wire
m_axi_arlock,
107  output wire [3:0]
m_axi_arsize,
108  output wire [2:0]
m_axi_arprot,
109  output wire
m_axi_arvalid,
110  input wire
m_axi_arready,
111  input wire [AXI_ID_WIDTH-1:0]
m_axi_rid,
112  input wire [AXI_DATA_WIDTH-1:0]
m_axi_rdata,
113  input wire [1:0]
m_axi_rresp,
114  input wire
m_axi_rlast,
115  input wire
m_axi_rvalid,
116  output wire
m_axi_rready,
117
118  /*
119   * AXI read descriptor input
120   */
121  input wire [AXI_ADDR_WIDTH-1:0]
is_read_desc_axi_addr,
122  input wire [RAM_SEL_WIDTH-1:0]
is_read_desc_ram_sel,
123  input wire [RAM_ADDR_WIDTH-1:0]
is_read_desc_ram_addr,
124  input wire [LEN_WIDTH-1:0]
is_read_desc_len,
125  input wire [TAG_WIDTH-1:0]
is_read_desc_tag,
126  input wire
is_read_desc_valid,
127  output wire
is_read_desc_ready,
128
129  /*
130   * AXI read descriptor status output
131   */
132  output wire [TAG_WIDTH-1:0]
is_read_desc_status_tag,
133  output wire [3:0]
is_read_desc_status_error,
134  output wire
is_read_desc_status_valid,
135
136  /*
137   * AXI write descriptor input
138   */
139  input wire [AXI_ADDR_WIDTH-1:0]
is_write_desc_axi_addr,
140  input wire [RAM_SEL_WIDTH-1:0]
is_write_desc_ram_sel,
141  input wire [RAM_ADDR_WIDTH-1:0]
is_write_desc_ram_addr,
142  input wire [IMM_WIDTH-1:0]
is_write_desc_imm,
143  input wire
is_write_desc_imm_en,
144  input wire [LEN_WIDTH-1:0]
is_write_desc_len,
145  input wire [TAG_WIDTH-1:0]
is_write_desc_tag,
146  input wire
is_write_desc_valid,

```

```

147  output wire
is_write_desc_ready,
148
149  /*
150   * AXI write descriptor status output
151   */
152  output wire [TAG_WIDTH-1:0]
is_write_desc_status_tag,
153  output wire [3:0]
is_write_desc_status_error,
154  output wire
is_write_desc_status_valid,
155
156  /*
157   * RAM interface
158   */
159  output wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]
ram_wr_cmd_sel,
160  output wire [RAM_SEG_COUNT*RAM_SEG_BE_WIDTH-1:0]
ram_wr_cmd_be,
161  output wire [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0]
ram_wr_cmd_addr,
162  output wire [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0]
ram_wr_cmd_data,
163  output wire [RAM_SEG_COUNT-1:0]
ram_wr_cmd_valid,
164  input wire [RAM_SEG_COUNT-1:0]
ram_wr_cmd_ready,
165  input wire [RAM_SEG_COUNT-1:0]
ram_wr_done,
166  output wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]
ram_rd_cmd_sel,
167  output wire [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0]
ram_rd_cmd_addr,
168  output wire [RAM_SEG_COUNT-1:0]
ram_rd_cmd_valid,
169  input wire [RAM_SEG_COUNT-1:0]
ram_rd_cmd_ready,
170  input wire [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0]
ram_rd_resp_data,
171  input wire [RAM_SEG_COUNT-1:0]
ram_rd_resp_valid,
172  output wire [RAM_SEG_COUNT-1:0]
ram_rd_resp_ready,
173
174  /*
175   * Configuration
176   */
177  input wire
read_enable,
178  input wire
write_enable,
179
180  /*
181   * Status
182   */
183  output wire
status_rd_busy,
184  output wire
status_wr_busy,
185
186  /*
187   * Statistics
188   */
189  output wire [$clog2(READ_OP_TABLE_SIZE)-1:0]
stat_rd_op_start_tag,
190  output wire [LEN_WIDTH-1:0]
stat_rd_op_start_len,
191  output wire
stat_rd_op_start_valid,
192  output wire [$clog2(READ_OP_TABLE_SIZE)-1:0]
stat_rd_op_finish_tag,
193  output wire [3:0]
stat_rd_op_finish_status,
194  output wire
stat_rd_op_finish_valid,
195  output wire [$clog2(READ_OP_TABLE_SIZE)-1:0]
stat_rd_req_start_tag,
196  output wire [12:0]
stat_rd_req_start_len,
197  output wire
stat_rd_req_start_valid,
198  output wire [$clog2(READ_OP_TABLE_SIZE)-1:0]
stat_rd_req_finish_tag,
199  output wire [3:0]
stat_rd_req_finish_status,
200  output wire
stat_rd_req_finish_valid,
201  output wire
stat_rd_op_table_full,
202  output wire
stat_rd_tx_stall,
203  output wire [$clog2(WRITE_OP_TABLE_SIZE)-1:0]
stat_wr_op_start_tag,
204  output wire [LEN_WIDTH-1:0]
stat_wr_op_start_len,
205  output wire
stat_wr_op_start_valid,
206  output wire [$clog2(WRITE_OP_TABLE_SIZE)-1:0]
stat_wr_op_finish_tag,
207  output wire [3:0]

```

```

stat_wr_op_finish_status,
208     output wire
stat_wr_op_finish_valid,
209     output wire [$clog2(WRITE_OP_TABLE_SIZE)-1:0]
stat_wr_req_start_tag,
210     output wire [12:0]
stat_wr_req_start_len,
211     output wire
stat_wr_req_start_valid,
212     output wire [$clog2(WRITE_OP_TABLE_SIZE)-1:0]
stat_wr_req_finish_tag,
213     output wire [3:0]
stat_wr_req_finish_status,
214     output wire
stat_wr_req_finish_valid,
215     output wire
stat_wr_op_table_full,
216     output wire
stat_wr_tx_stall
217 );
218
219 dma_if_axi_rd #(
220     .AXI_DATA_WIDTH(AXI_DATA_WIDTH),
221     .AXI_ADDR_WIDTH(AXI_ADDR_WIDTH),
222     .AXI_STRB_WIDTH(AXI_STRB_WIDTH),
223     .AXI_ID_WIDTH(AXI_ID_WIDTH),
224     .AXI_MAX_BURST_LEN(AXI_MAX_BURST_LEN),
225     .RAM_SEL_WIDTH(RAM_SEL_WIDTH),
226     .RAM_ADDR_WIDTH(RAM_ADDR_WIDTH),
227     .RAM_SEG_COUNT(RAM_SEG_COUNT),
228     .RAM_SEG_DATA_WIDTH(RAM_SEG_DATA_WIDTH),
229     .RAM_SEG_BE_WIDTH(RAM_SEG_BE_WIDTH),
230     .RAM_SEG_ADDR_WIDTH(RAM_SEG_ADDR_WIDTH),
231     .LEN_WIDTH(LEN_WIDTH),
232     .TAG_WIDTH(TAG_WIDTH),
233     .OP_TABLE_SIZE(READ_OP_TABLE_SIZE),
234     .USE_AXI_ID(READ_USE_AXI_ID)
235 )
236 dma_if_axi_rd_inst (
237     .clk(clk),
238     .rst(rst),
239
240     /*
241      * AXI master interface
242      */
243     .m_axi_arid(m_axi_arid),
244     .m_axi_araddr(m_axi_araddr),
245     .m_axi_arlen(m_axi_arlen),
246     .m_axi_arsize(m_axi_arsize),
247     .m_axi_arburst(m_axi_arburst),
248     .m_axi_arlock(m_axi_arlock),
249     .m_axi_arcache(m_axi_arcache),
250     .m_axi_arprot(m_axi_arprot),
251     .m_axi_arvalid(m_axi_arvalid),
252     .m_axi_arready(m_axi_arready),
253     .m_axi_rid(m_axi_rid),
254     .m_axi_rdata(m_axi_rdata),
255     .m_axi_rresp(m_axi_rresp),
256     .m_axi_rlast(m_axi_rlast),
257     .m_axi_rvalid(m_axi_rvalid),
258     .m_axi_rready(m_axi_rready),
259
260     /*
261      * AXI read descriptor input
262      */
263     .s_axis_read_desc_axi_addr(s_axis_read_desc_axi_addr),
264     .s_axis_read_desc_ram_sel(s_axis_read_desc_ram_sel),
265     .s_axis_read_desc_ram_addr(s_axis_read_desc_ram_addr),
266     .s_axis_read_desc_len(s_axis_read_desc_len),
267     .s_axis_read_desc_tag(s_axis_read_desc_tag),
268     .s_axis_read_desc_valid(s_axis_read_desc_valid),
269     .s_axis_read_desc_ready(s_axis_read_desc_ready),
270
271     /*
272      * AXI read descriptor status output
273      */
274     .m_axis_read_desc_status_tag(m_axis_read_desc_status_tag),
275     .m_ax-
is_read_desc_status_error(m_axis_read_desc_status_error),
276     .m_ax-
is_read_desc_status_valid(m_axis_read_desc_status_valid),
277
278     /*
279      * RAM interface
280      */
281     .ram_wr_cmd_sel(ram_wr_cmd_sel),
282     .ram_wr_cmd_be(ram_wr_cmd_be),
283     .ram_wr_cmd_addr(ram_wr_cmd_addr),
284     .ram_wr_cmd_data(ram_wr_cmd_data),
285     .ram_wr_cmd_valid(ram_wr_cmd_valid),
286     .ram_wr_cmd_ready(ram_wr_cmd_ready),
287     .ram_wr_done(ram_wr_done),
288
289     /*
290      * Configuration
291      */
292     .enable(read_enable),
293
294     /*
295      * Status
296      */

```

```

297     .status_busy(status_rd_busy),
298
299     /*
300      * Statistics
301      */
302     .stat_rd_op_start_tag(stat_rd_op_start_tag),
303     .stat_rd_op_start_len(stat_rd_op_start_len),
304     .stat_rd_op_start_valid(stat_rd_op_start_valid),
305     .stat_rd_op_finish_tag(stat_rd_op_finish_tag),
306     .stat_rd_op_finish_status(stat_rd_op_finish_status),
307     .stat_rd_op_finish_valid(stat_rd_op_finish_valid),
308     .stat_rd_req_start_tag(stat_rd_req_start_tag),
309     .stat_rd_req_start_len(stat_rd_req_start_len),
310     .stat_rd_req_start_valid(stat_rd_req_start_valid),
311     .stat_rd_req_finish_tag(stat_rd_req_finish_tag),
312     .stat_rd_req_finish_status(stat_rd_req_finish_status),
313     .stat_rd_req_finish_valid(stat_rd_req_finish_valid),
314     .stat_rd_op_table_full(stat_rd_op_table_full),
315     .stat_rd_tx_stall(stat_rd_tx_stall)
316 );
317
318 dma_if_axi_wr #(
319     .AXI_DATA_WIDTH(AXI_DATA_WIDTH),
320     .AXI_ADDR_WIDTH(AXI_ADDR_WIDTH),
321     .AXI_STRB_WIDTH(AXI_STRB_WIDTH),
322     .AXI_ID_WIDTH(AXI_ID_WIDTH),
323     .AXI_MAX_BURST_LEN(AXI_MAX_BURST_LEN),
324     .RAM_SEL_WIDTH(RAM_SEL_WIDTH),
325     .RAM_ADDR_WIDTH(RAM_ADDR_WIDTH),
326     .RAM_SEG_COUNT(RAM_SEG_COUNT),
327     .RAM_SEG_DATA_WIDTH(RAM_SEG_DATA_WIDTH),
328     .RAM_SEG_BE_WIDTH(RAM_SEG_BE_WIDTH),
329     .RAM_SEG_ADDR_WIDTH(RAM_SEG_ADDR_WIDTH),
330     .IMM_ENABLE(IMM_ENABLE),
331     .IMM_WIDTH(IMM_WIDTH),
332     .LEN_WIDTH(LEN_WIDTH),
333     .TAG_WIDTH(TAG_WIDTH),
334     .OP_TABLE_SIZE(WRITE_OP_TABLE_SIZE),
335     .USE_AXI_ID(WRITE_USE_AXI_ID)
336 )
337 dma_if_axi_wr_inst (
338     .clk(clk),
339     .rst(rst),
340
341     /*
342      * AXI master interface
343      */
344     .m_axi_awid(m_axi_awid),
345     .m_axi_awaddr(m_axi_awaddr),
346     .m_axi_awlen(m_axi_awlen),
347     .m_axi_awsz(m_axi_awsz),
348     .m_axi_awburst(m_axi_awburst),
349     .m_axi_awlock(m_axi_awlock),
350     .m_axi_awcache(m_axi_awcache),
351     .m_axi_awprot(m_axi_awprot),
352     .m_axi_awvalid(m_axi_awvalid),
353     .m_axi_awready(m_axi_awready),
354     .m_axi_wdata(m_axi_wdata),
355     .m_axi_wstrb(m_axi_wstrb),
356     .m_axi_wlast(m_axi_wlast),
357     .m_axi_wvalid(m_axi_wvalid),
358     .m_axi_wready(m_axi_wready),
359     .m_axi_bid(m_axi_bid),
360     .m_axi_bresp(m_axi_bresp),
361     .m_axi_bvalid(m_axi_bvalid),
362     .m_axi_bready(m_axi_bready),
363
364     /*
365      * AXI write descriptor input
366      */
367     .s_axis_write_desc_axi_addr(s_axis_write_desc_axi_addr),
368     .s_axis_write_desc_ram_sel(s_axis_write_desc_ram_sel),
369     .s_axis_write_desc_ram_addr(s_axis_write_desc_ram_addr),
370     .s_axis_write_desc_imm(s_axis_write_desc_imm),
371     .s_axis_write_desc_imm_en(s_axis_write_desc_imm_en),
372     .s_axis_write_desc_len(s_axis_write_desc_len),
373     .s_axis_write_desc_tag(s_axis_write_desc_tag),
374     .s_axis_write_desc_valid(s_axis_write_desc_valid),
375     .s_axis_write_desc_ready(s_axis_write_desc_ready),
376
377     /*
378      * AXI write descriptor status output
379      */
380     .m_ax-
is_write_desc_status_tag(m_axis_write_desc_status_tag),
381     .m_ax-
is_write_desc_status_error(m_axis_write_desc_status_error),
382     .m_ax-
is_write_desc_status_valid(m_axis_write_desc_status_valid),
383
384     /*
385      * RAM interface
386      */
387     .ram_rd_cmd_sel(ram_rd_cmd_sel),
388     .ram_rd_cmd_addr(ram_rd_cmd_addr),
389     .ram_rd_cmd_valid(ram_rd_cmd_valid),
390     .ram_rd_cmd_ready(ram_rd_cmd_ready),
391     .ram_rd_resp_data(ram_rd_resp_data),
392     .ram_rd_resp_valid(ram_rd_resp_valid),
393     .ram_rd_resp_ready(ram_rd_resp_ready),
394

```



```

395  /*
396  * Configuration
397  */
398  .enable(write_enable),
399
400  /*
401  * Status
402  */
403  .status_busy(status_wr_busy),
404
405  /*
406  * Statistics
407  */
408  .stat_wr_op_start_tag(stat_wr_op_start_tag),
409  .stat_wr_op_start_len(stat_wr_op_start_len),
410  .stat_wr_op_start_valid(stat_wr_op_start_valid),
411  .stat_wr_op_finish_tag(stat_wr_op_finish_tag),
412  .stat_wr_op_finish_status(stat_wr_op_finish_status),
413  .stat_wr_op_finish_valid(stat_wr_op_finish_valid),
414  .stat_wr_req_start_tag(stat_wr_req_start_tag),
415  .stat_wr_req_start_len(stat_wr_req_start_len),
416  .stat_wr_req_start_valid(stat_wr_req_start_valid),
417  .stat_wr_req_finish_tag(stat_wr_req_finish_tag),
418  .stat_wr_req_finish_status(stat_wr_req_finish_status),
419  .stat_wr_req_finish_valid(stat_wr_req_finish_valid),
420  .stat_wr_op_table_full(stat_wr_op_table_full),
421  .stat_wr_tx_stall(stat_wr_tx_stall)
422  );
423
424 endmodule
425
426 `resetall

```

6. alexforenrich_verilog-pcie/rtl/dma_if_axi_rd.v (1,132 lines)

```

1  /*
2
3  Copyright (c) 2021 Alex Forenrich
4
5  Permission is hereby granted, free of charge, to any person
6  obtaining a copy
7  of this software and associated documentation files (the
8  "Software"), to deal
9  in the Software without restriction, including without limita-
10 tion the rights
11 to use, copy, modify, merge, publish, distribute, sublicense,
12 and/or sell
13 copies of the Software, and to permit persons to whom the
14 Software is
15 furnished to do so, subject to the following conditions:
16
17 The above copyright notice and this permission notice shall be
18 included in
19 all copies or substantial portions of the Software.
20
21 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
22 KIND, EXPRESS OR
23 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
24 CHANTABILITY
25 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
26 EVENT SHALL THE
27 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
28 OR OTHER
29 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
30 WISE, ARISING FROM,
31 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
32 DEALINGS IN
33 THE SOFTWARE.
34
35 */
36 // Language: Verilog 2001
37
38 `resetall
39 `timescale 1ns / 1ps
40 `default_nettype none
41
42 /*
43 * AXI DMA read interface
44 */
45 module dma_if_axi_rd #
46 (
47     // Width of AXI data bus in bits
48     parameter AXI_DATA_WIDTH = 32,
49     // Width of AXI address bus in bits
50     parameter AXI_ADDR_WIDTH = 16,
51     // Width of AXI wstrb (width of data bus in words)
52     parameter AXI_STRB_WIDTH = (AXI_DATA_WIDTH/8),
53     // Width of AXI ID signal
54     parameter AXI_ID_WIDTH = 8,
55     // Maximum AXI burst length to generate
56     parameter AXI_MAX_BURST_LEN = 256,
57     // RAM select width
58     parameter RAM_SEL_WIDTH = 2,
59     // RAM address width
60     parameter RAM_ADDR_WIDTH = 16,
61     // RAM segment count
62     parameter RAM_SEG_COUNT = 2,
63     // RAM segment data width

```

```

53     parameter RAM_SEG_DATA_WIDTH = AXI_DATA_WIDTH*2/
RAM_SEG_COUNT,
54     // RAM segment byte enable width
55     parameter RAM_SEG_BE_WIDTH = RAM_SEG_DATA_WIDTH/8,
56     // RAM segment address width
57     parameter RAM_SEG_ADDR_WIDTH = RAM_ADDR_WIDTH-$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH),
58     // Length field width
59     parameter LEN_WIDTH = 16,
60     // Tag field width
61     parameter TAG_WIDTH = 8,
62     // Operation table size
63     parameter OP_TABLE_SIZE = 2** (AXI_ID_WIDTH),
64     // Use AXI ID signals
65     parameter USE_AXI_ID = 0
66 )
67 (
68     input wire clk,
69     input wire rst,
70
71     /*
72     * AXI master interface
73     */
74     output wire [AXI_ID_WIDTH-1:0] m_axi_arid,
75     output wire [AXI_ADDR_WIDTH-1:0] m_axi_araddr,
76     output wire [7:0] m_axi_arlen,
77     output wire [2:0] m_axi_arsize,
78     output wire [1:0] m_axi_arburst,
79     output wire m_axi_arlock,
80     output wire [3:0] m_axi_arcache,
81     output wire [2:0] m_axi_arprot,
82     output wire m_axi_arvalid,
83     input wire m_axi_arready,
84     input wire [AXI_ID_WIDTH-1:0] m_axi_rid,
85     input wire [AXI_DATA_WIDTH-1:0] m_axi_rdata,
86     input wire [1:0] m_axi_rresp,
87     input wire m_axi_rlast,
88     input wire m_axi_rvalid,
89     output wire m_axi_rready,
90
91     /*
92     * AXI read descriptor input
93     */
94     input wire [AXI_ADDR_WIDTH-1:0] is_read_desc_axi_addr,
95     input wire [RAM_SEL_WIDTH-1:0] is_read_desc_ram_sel,
96     input wire [RAM_ADDR_WIDTH-1:0] is_read_desc_ram_addr,
97     input wire [LEN_WIDTH-1:0] is_read_desc_len,
98     input wire [TAG_WIDTH-1:0] is_read_desc_tag,
99     input wire is_read_desc_valid,
100     output wire [3:0] is_read_desc_status_error,
101     output wire is_read_desc_status_valid,
102
103     /*
104     * AXI read descriptor status output
105     */
106     output wire [TAG_WIDTH-1:0] m_ax-
107     is_read_desc_status_tag,
108     output wire [3:0] m_ax-
109     is_read_desc_status_error,
110     output wire m_ax-
111     is_read_desc_status_valid,
112
113     /*
114     * RAM interface
115     */
116     output wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0] ram_wr_cmd_sel,
117     output wire [RAM_SEG_COUNT*RAM_SEG_BE_WIDTH-1:0] ram_wr_cmd_be,
118     output wire [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0] ram_wr_cmd_addr,
119     output wire [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0] ram_wr_cmd_data,
120     output wire [RAM_SEG_COUNT-1:0] ram_wr_cmd_valid,
121     input wire [RAM_SEG_COUNT-1:0] ram_wr_cmd_ready,
122     input wire [RAM_SEG_COUNT-1:0] ram_wr_done,

```

```

119
120 /*
121  * Configuration
122  */
123 input wire en-
able,
124
125 /*
126  * Status
127  */
128 output wire
status_busy,
129
130 /*
131  * Statistics
132  */
133 output wire [$clog2(OP_TABLE_SIZE)-1:0]
stat_rd_op_start_tag,
134 output wire [LEN_WIDTH-1:0]
stat_rd_op_start_len,
135 output wire
stat_rd_op_start_valid,
136 output wire [$clog2(OP_TABLE_SIZE)-1:0]
stat_rd_op_finish_tag,
137 output wire [3:0]
stat_rd_op_finish_status,
138 output wire
stat_rd_op_finish_valid,
139 output wire [$clog2(OP_TABLE_SIZE)-1:0]
stat_rd_req_start_tag,
140 output wire [12:0]
stat_rd_req_start_len,
141 output wire
stat_rd_req_start_valid,
142 output wire [$clog2(OP_TABLE_SIZE)-1:0]
stat_rd_req_finish_tag,
143 output wire [3:0]
stat_rd_req_finish_status,
144 output wire
stat_rd_req_finish_valid,
145 output wire
stat_rd_op_table_full,
146 output wire
stat_rd_tx_stall
147 );
148
149 parameter RAM_DATA_WIDTH = RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH;
150 parameter RAM_WORD_WIDTH = RAM_SEG_BE_WIDTH;
151 parameter RAM_WORD_SIZE = RAM_SEG_DATA_WIDTH/RAM_WORD_WIDTH;
152
153 parameter AXI_WORD_WIDTH = AXI_STRB_WIDTH;
154 parameter AXI_WORD_SIZE = AXI_DATA_WIDTH/AXI_WORD_WIDTH;
155 parameter AXI_BURST_SIZE = $clog2(AXI_STRB_WIDTH);
156 parameter AXI_MAX_BURST_SIZE = AXI_MAX_BURST_LEN <<
AXI_BURST_SIZE;
157
158 parameter OFFSET_WIDTH = AXI_STRB_WIDTH > 1 ? $clog-
2(AXI_STRB_WIDTH) : 1;
159 parameter OFFSET_MASK = AXI_STRB_WIDTH > 1 ? {OFF-
SET_WIDTH{1'b1}} : 0;
160 parameter RAM_OFFSET_WIDTH = $clog-
2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH);
161 parameter ADDR_MASK = {AXI_ADDR_WIDTH{1'b1}} << $clog-
2(AXI_STRB_WIDTH);
162 parameter CYCLE_COUNT_WIDTH = LEN_WIDTH - AXI_BURST_SIZE + 1;
163
164 parameter OP_TAG_WIDTH = $clog2(OP_TABLE_SIZE);
165 parameter OP_TABLE_READ_COUNT_WIDTH = AXI_ID_WIDTH+1;
166 parameter OP_TABLE_WRITE_COUNT_WIDTH = LEN_WIDTH;
167
168 parameter STATUS_FIFO_ADDR_WIDTH = 5;
169 parameter OUTPUT_FIFO_ADDR_WIDTH = 5;
170
171 // bus width assertions
172 initial begin
173     if (AXI_WORD_SIZE * AXI_STRB_WIDTH != AXI_DATA_WIDTH) be-
gin
174         $error("Error: AXI data width not evenly divisble (in-
stance %m)");
175     $finish;
176 end
177
178 if (AXI_WORD_SIZE != RAM_WORD_SIZE) begin
179     $error("Error: word size mismatch (instance %m)");
180     $finish;
181 end
182
183 if (2**$clog2(AXI_WORD_WIDTH) != AXI_WORD_WIDTH) begin
184     $error("Error: AXI word width must be even power of
two (instance %m)");
185     $finish;
186 end
187
188 if (AXI_MAX_BURST_LEN < 1 || AXI_MAX_BURST_LEN > 256) be-
gin
189     $error("Error: AXI_MAX_BURST_LEN must be between 1 and
256 (instance %m)");
190     $finish;
191 end
192

```

```

193     if (RAM_SEG_COUNT < 2) begin
194         $error("Error: RAM interface requires at least 2 seg-
ments (instance %m)");
195         $finish;
196     end
197
198     if (RAM_DATA_WIDTH != AXI_DATA_WIDTH*2) begin
199         $error("Error: RAM interface width must be double the
AXI interface width (instance %m)");
200         $finish;
201     end
202
203     if (2**$clog2(RAM_WORD_WIDTH) != RAM_WORD_WIDTH) begin
204         $error("Error: RAM word width must be even power of
two (instance %m)");
205         $finish;
206     end
207
208     if (RAM_ADDR_WIDTH != RAM_SEG_ADDR_WIDTH+$clog-
2(RAM_SEG_COUNT)+$clog2(RAM_SEG_BE_WIDTH)) begin
209         $error("Error: RAM_ADDR_WIDTH does not match RAM con-
figuration (instance %m)");
210         $finish;
211     end
212
213     if (OP_TABLE_SIZE > 2**AXI_ID_WIDTH) begin
214         $error("Error: AXI_ID_WIDTH insufficient for requested
OP_TABLE_SIZE (instance %m)");
215         $finish;
216     end
217 end
218
219 localparam [1:0]
220     AXI_RESP_OKAY = 2'b00,
221     AXI_RESP_EXOKAY = 2'b01,
222     AXI_RESP_SLVERR = 2'b10,
223     AXI_RESP_DECERR = 2'b11;
224
225 localparam [3:0]
226     DMA_ERROR_NONE = 4'd0,
227     DMA_ERROR_TIMEOUT = 4'd1,
228     DMA_ERROR_PARITY = 4'd2,
229     DMA_ERROR_AXI_RD_SLVERR = 4'd4,
230     DMA_ERROR_AXI_RD_DECERR = 4'd5,
231     DMA_ERROR_AXI_WR_SLVERR = 4'd6,
232     DMA_ERROR_AXI_WR_DECERR = 4'd7,
233     DMA_ERROR_PCIE_FLR = 4'd8,
234     DMA_ERROR_PCIE_CPL_POISONED = 4'd9,
235     DMA_ERROR_PCIE_CPL_STATUS_UR = 4'd10,
236     DMA_ERROR_PCIE_CPL_STATUS_CA = 4'd11;
237
238 localparam [0:0]
239     REQ_STATE_IDLE = 1'd0,
240     REQ_STATE_START = 1'd1;
241
242 reg [0:0] req_state_reg = REQ_STATE_IDLE, req_state_next;
243
244 localparam [0:0]
245     AXI_STATE_IDLE = 1'd0,
246     AXI_STATE_WRITE = 1'd1;
247
248 reg [0:0] axi_state_reg = AXI_STATE_IDLE, axi_state_next;
249
250 reg [AXI_ADDR_WIDTH-1:0] req_axi_addr_reg = {AXI_AD-
DR_WIDTH{1'b0}}, req_axi_addr_next;
251 reg [RAM_SEL_WIDTH-1:0] req_ram_sel_reg =
{RAM_SEL_WIDTH{1'b0}}, req_ram_sel_next;
252 reg [RAM_ADDR_WIDTH-1:0] req_ram_addr_reg = {RAM_AD-
DR_WIDTH{1'b0}}, req_ram_addr_next;
253 reg [LEN_WIDTH-1:0] req_op_count_reg = {LEN_WIDTH{1'b0}},
req_op_count_next;
254 reg [LEN_WIDTH-1:0] req_tr_count_reg = {LEN_WIDTH{1'b0}},
req_tr_count_next;
255 reg req_zero_len_reg = 1'b0, req_zero_len_next;
256 reg [TAG_WIDTH-1:0] req_tag_reg = {TAG_WIDTH{1'b0}},
req_tag_next;
257
258 reg [RAM_SEL_WIDTH-1:0] ram_sel_reg = {RAM_SEL_WIDTH{1'b0}},
ram_sel_next;
259 reg [RAM_ADDR_WIDTH-1:0] addr_reg = {RAM_ADDR_WIDTH{1'b0}},
addr_next;
260 reg [RAM_ADDR_WIDTH-1:0] addr_delay_reg = {RAM_AD-
DR_WIDTH{1'b0}}, addr_delay_next;
261 reg [12:0] op_count_reg = 13'd0, op_count_next;
262 reg zero_len_reg = 1'b0, zero_len_next;
263 reg [RAM_SEG_COUNT-1:0] ram_mask_reg = {RAM_SEG_COUNT{1'b0}},
ram_mask_next;
264 reg [RAM_SEG_COUNT-1:0] ram_mask_0_reg =
{RAM_SEG_COUNT{1'b0}}, ram_mask_0_next;
265 reg [RAM_SEG_COUNT-1:0] ram_mask_1_reg =
{RAM_SEG_COUNT{1'b0}}, ram_mask_1_next;
266 reg ram_wrap_reg = 1'b0, ram_wrap_next;
267 reg [OFFSET_WIDTH+1-1:0] cycle_byte_count_reg = {OFF-
SET_WIDTH+1{1'b0}}, cycle_byte_count_next;
268 reg [RAM_OFFSET_WIDTH-1:0] start_offset_reg = {RAM_OFF-
SET_WIDTH{1'b0}}, start_offset_next;
269 reg [RAM_OFFSET_WIDTH-1:0] end_offset_reg = {RAM_OFF-
SET_WIDTH{1'b0}}, end_offset_next;
270 reg [OFFSET_WIDTH-1:0] offset_reg = {OFFSET_WIDTH{1'b0}}, off-
set_next;

```



```

271 reg [OP_TAG_WIDTH-1:0] op_tag_reg = {OP_TAG_WIDTH{1'b0}},
op_tag_next;
272
273 reg [STATUS_FIFO_ADDR_WIDTH+1-1:0] status_fifo_wr_ptr_reg = 0;

274 reg [STATUS_FIFO_ADDR_WIDTH+1-1:0] status_fifo_rd_ptr_reg = 0,
status_fifo_rd_ptr_next;
275 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
276 reg [OP_TAG_WIDTH-1:0]
status_fifo_op_tag[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
277 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
278 reg [RAM_SEG_COUNT-1:0]
status_fifo_mask[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
279 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
280 reg status_fifo_finish[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
281 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
282 reg [3:0] status_fifo_error[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
283 reg [OP_TAG_WIDTH-1:0] status_fifo_wr_op_tag;
284 reg [RAM_SEG_COUNT-1:0] status_fifo_wr_mask;
285 reg status_fifo_wr_finish;
286 reg [3:0] status_fifo_wr_error;
287 reg status_fifo_we;
288 reg status_fifo_mask_reg = 1'b0, status_fifo_mask_next;
289 reg status_fifo_finish_reg = 1'b0, status_fifo_finish_next;
290 reg [3:0] status_fifo_error_reg = 4'd0,
status_fifo_error_next;
291 reg status_fifo_we_reg = 1'b0, status_fifo_we_next;
292 reg status_fifo_half_full_reg = 1'b0;
293 reg [OP_TAG_WIDTH-1:0] status_fifo_rd_op_tag_reg = 0,
status_fifo_rd_op_tag_next;
294 reg [RAM_SEG_COUNT-1:0] status_fifo_rd_mask_reg = 0,
status_fifo_rd_mask_next;
295 reg status_fifo_rd_finish_reg = 1'b0,
status_fifo_rd_finish_next;
296 reg [3:0] status_fifo_rd_error_reg = 4'd0,
status_fifo_rd_error_next;
297 reg status_fifo_rd_valid_reg = 1'b0,
status_fifo_rd_valid_next;
298
299 reg [OP_TAG_WIDTH+1-1:0] active_op_count_reg = 0;
300 reg inc_active_op;
301 reg dec_active_op;
302
303 reg [AXI_DATA_WIDTH-1:0] m_axi_rdata_int_reg =
{AXI_DATA_WIDTH{1'b0}}, m_axi_rdata_int_next;
304 reg m_axi_rvalid_int_reg = 1'b0, m_axi_rvalid_int_next;
305
306 reg [AXI_ID_WIDTH-1:0] m_axi_arid_reg = {AXI_ID_WIDTH{1'b0}},
m_axi_arid_next;
307 reg [AXI_ADDR_WIDTH-1:0] m_axi_araddr_reg = {AXI_AD-
DR_WIDTH{1'b0}}, m_axi_araddr_next;
308 reg [7:0] m_axi_arlen_reg = 8'd0, m_axi_arlen_next;
309 reg m_axi_arvalid_reg = 1'b0, m_axi_arvalid_next;
310 reg m_axi_rready_reg = 1'b0, m_axi_rready_next;
311
312 reg s_axis_read_desc_ready_reg = 1'b0, s_ax-
is_read_desc_ready_next;
313
314 reg [TAG_WIDTH-1:0] m_axis_read_desc_status_tag_reg =
{TAG_WIDTH{1'b0}}, m_axis_read_desc_status_tag_next;
315 reg [3:0] m_axis_read_desc_status_error_reg = 4'd0, m_ax-
is_read_desc_status_error_next;
316 reg m_axis_read_desc_status_valid_reg = 1'b0, m_ax-
is_read_desc_status_valid_next;
317
318 reg status_busy_reg = 1'b0;
319
320 reg [OP_TAG_WIDTH-1:0] stat_rd_op_start_tag_reg = 0,
stat_rd_op_start_tag_next;
321 reg [LEN_WIDTH-1:0] stat_rd_op_start_len_reg = 0,
stat_rd_op_start_len_next;
322 reg stat_rd_op_start_valid_reg = 1'b0,
stat_rd_op_start_valid_next;
323 reg [OP_TAG_WIDTH-1:0] stat_rd_op_finish_tag_reg = 0,
stat_rd_op_finish_tag_next;
324 reg [3:0] stat_rd_op_finish_status_reg = 4'd0, stat_rd_op_fin-
ish_status_next;
325 reg stat_rd_op_finish_valid_reg = 1'b0, stat_rd_op_fin-
ish_valid_next;
326 reg [OP_TAG_WIDTH-1:0] stat_rd_req_start_tag_reg = 0,
stat_rd_req_start_tag_next;
327 reg [12:0] stat_rd_req_start_len_reg = 13'd0,
stat_rd_req_start_len_next;
328 reg stat_rd_req_start_valid_reg = 1'b0,
stat_rd_req_start_valid_next;
329 reg [OP_TAG_WIDTH-1:0] stat_rd_req_finish_tag_reg = 0,
stat_rd_req_finish_tag_next;
330 reg [3:0] stat_rd_req_finish_status_reg = 4'd0,
stat_rd_req_finish_status_next;
331 reg stat_rd_req_finish_valid_reg = 1'b0, stat_rd_req_fin-
ish_valid_next;
332 reg stat_rd_op_table_full_reg = 1'b0, stat_rd_op_t-
able_full_next;
333 reg stat_rd_tx_stall_reg = 1'b0, stat_rd_tx_stall_next;
334
335 // internal datapath
336 reg [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]

```

```

ram_wr_cmd_sel_int;
337 reg [RAM_SEG_COUNT*RAM_SEG_BE_WIDTH-1:0] ram_wr_cmd_be_int;
338 reg [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0] ram_wr_cmd_ad-
dr_int;
339 reg [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0]
ram_wr_cmd_data_int;
340 reg [RAM_SEG_COUNT-1:0] ram_wr_cmd_val-
id_int;
341 wire [RAM_SEG_COUNT-1:0]
ram_wr_cmd_ready_int;
342
343 wire [RAM_SEG_COUNT-1:0] out_done;
344 reg [RAM_SEG_COUNT-1:0] out_done_ack;
345
346 assign m_axi_arid = USE_AXI_ID ? m_axi_arid_reg :
{AXI_ID_WIDTH{1'b0}};
347 assign m_axi_araddr = m_axi_araddr_reg;
348 assign m_axi_arlen = m_axi_arlen_reg;
349 assign m_axi_arsize = AXI_BURST_SIZE;
350 assign m_axi_arburst = 2'b01;
351 assign m_axi_arlock = 1'b0;
352 assign m_axi_arcache = 4'b0011;
353 assign m_axi_arprot = 3'b010;
354 assign m_axi_arvalid = m_axi_arvalid_reg;
355 assign m_axi_rready = m_axi_rready_reg;
356
357 assign s_axis_read_desc_ready = s_axis_read_desc_ready_reg;
358
359 assign m_axis_read_desc_status_tag = m_ax-
is_read_desc_status_tag_reg;
360 assign m_axis_read_desc_status_error = m_ax-
is_read_desc_status_error_reg;
361 assign m_axis_read_desc_status_valid = m_ax-
is_read_desc_status_valid_reg;
362
363 assign status_busy = status_busy_reg;
364
365 assign stat_rd_op_start_tag = stat_rd_op_start_tag_reg;
366 assign stat_rd_op_start_len = stat_rd_op_start_len_reg;
367 assign stat_rd_op_start_valid = stat_rd_op_start_valid_reg;
368 assign stat_rd_op_finish_tag = stat_rd_op_finish_tag_reg;
369 assign stat_rd_op_finish_status = stat_rd_op_fin-
ish_status_reg;
370 assign stat_rd_op_finish_valid = stat_rd_op_finish_valid_reg;
371 assign stat_rd_req_start_tag = stat_rd_req_start_tag_reg;
372 assign stat_rd_req_start_len = stat_rd_req_start_len_reg;
373 assign stat_rd_req_start_valid = stat_rd_req_start_valid_reg;
374 assign stat_rd_req_finish_tag = stat_rd_req_finish_tag_reg;
375 assign stat_rd_req_finish_status = stat_rd_req_fin-
ish_status_reg;
376 assign stat_rd_req_finish_valid = stat_rd_req_fin-
ish_valid_reg;
377 assign stat_rd_op_table_full = stat_rd_op_table_full_reg;
378 assign stat_rd_tx_stall = stat_rd_tx_stall_reg;
379
380 // operation tag management
381 reg [OP_TAG_WIDTH+1-1:0] op_table_start_ptr_reg = 0;
382 reg [AXI_ADDR_WIDTH-1:0] op_table_start_axi_addr;
383 reg [RAM_SEL_WIDTH-1:0] op_table_start_ram_sel;
384 reg [RAM_ADDR_WIDTH-1:0] op_table_start_ram_addr;
385 reg [11:0] op_table_start_len;
386 reg op_table_start_zero_len;
387 reg [CYCLE_COUNT_WIDTH-1:0] op_table_start_cycle_count;
388 reg [TAG_WIDTH-1:0] op_table_start_tag;
389 reg op_table_start_last;
390 reg op_table_start_en;
391 reg [OP_TAG_WIDTH+1-1:0] op_table_read_complete_ptr_reg = 0;
392 reg op_table_read_complete_en;
393 reg [OP_TAG_WIDTH-1:0] op_table_update_status_ptr;
394 reg [3:0] op_table_update_status_error;
395 reg op_table_update_status_en;
396 reg [OP_TAG_WIDTH-1:0] op_table_write_complete_ptr;
397 reg op_table_write_complete_en;
398 reg [OP_TAG_WIDTH+1-1:0] op_table_finish_ptr_reg = 0;
399 reg op_table_finish_en;
400
401 reg [2**OP_TAG_WIDTH-1:0] op_table_active = 0;
402 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
403 reg [AXI_ADDR_WIDTH-1:0] op_table_axi_addr
[2**OP_TAG_WIDTH-1:0];
404 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
405 reg [RAM_SEL_WIDTH-1:0] op_table_ram_sel
[2**OP_TAG_WIDTH-1:0];
406 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
407 reg [RAM_ADDR_WIDTH-1:0] op_table_ram_addr
[2**OP_TAG_WIDTH-1:0];
408 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
409 reg [11:0] op_table_len[2**OP_TAG_WIDTH-1:0];
410 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
411 reg op_table_zero_len[2**OP_TAG_WIDTH-1:0];
412 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
413 reg [CYCLE_COUNT_WIDTH-1:0] op_t-
able_cycle_count[2**OP_TAG_WIDTH-1:0];
414 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)

```

```

415 reg [TAG_WIDTH-1:0] op_table_tag[2**OP_TAG_WIDTH-1:0];
416 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
417 reg op_table_last[2**OP_TAG_WIDTH-1:0];
418 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
419 reg op_table_write_complete[2**OP_TAG_WIDTH-1:0];
420 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
421 reg op_table_error_a [2**OP_TAG_WIDTH-1:0];
422 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
423 reg op_table_error_b [2**OP_TAG_WIDTH-1:0];
424 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
425 reg [3:0] op_table_error_code [2**OP_TAG_WIDTH-1:0];
426
427 integer i;
428
429 initial begin
430   for (i = 0; i < 2**OP_TAG_WIDTH; i = i + 1) begin
431     op_table_axi_addr[i] = 0;
432     op_table_ram_sel[i] = 0;
433     op_table_ram_addr[i] = 0;
434     op_table_len[i] = 0;
435     op_table_zero_len[i] = 1'b0;
436     op_table_cycle_count[i] = 0;
437     op_table_tag[i] = 0;
438     op_table_last[i] = 0;
439     op_table_write_complete[i] = 0;
440     op_table_error_a[i] = 0;
441     op_table_error_b[i] = 0;
442     op_table_error_code[i] = 0;
443   end
444 end
445
446 always @* begin
447   req_state_next = REQ_STATE_IDLE;
448
449   s_axis_read_desc_ready_next = 1'b0;
450
451   stat_rd_op_start_tag_next = stat_rd_op_start_tag_reg;
452   stat_rd_op_start_len_next = stat_rd_op_start_len_reg;
453   stat_rd_op_start_valid_next = 1'b0;
454   stat_rd_req_start_tag_next = stat_rd_req_start_tag_reg;
455   stat_rd_req_start_len_next = stat_rd_req_start_len_reg;
456   stat_rd_req_start_valid_next = 1'b0;
457   stat_rd_op_table_full_next = !(op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2**OP_TAG_WIDTH));
458   stat_rd_tx_stall_next = m_axi_arvalid_reg && !m_axi_ar-
ready;
459
460   req_axi_addr_next = req_axi_addr_reg;
461   req_ram_sel_next = req_ram_sel_reg;
462   req_ram_addr_next = req_ram_addr_reg;
463   req_op_count_next = req_op_count_reg;
464   req_tr_count_next = req_tr_count_reg;
465   req_zero_len_next = req_zero_len_reg;
466   req_tag_next = req_tag_reg;
467
468   m_axi_arid_next = m_axi_arid_reg;
469   m_axi_araddr_next = m_axi_araddr_reg;
470   m_axi_arlen_next = m_axi_arlen_reg;
471   m_axi_arvalid_next = m_axi_arvalid_reg && !m_axi_arready;
472
473   op_table_start_axi_addr = req_axi_addr_reg;
474   op_table_start_ram_sel = req_ram_sel_reg;
475   op_table_start_ram_addr = req_ram_addr_reg;
476   op_table_start_len = 0;
477   op_table_start_zero_len = req_zero_len_reg;
478   op_table_start_tag = req_tag_reg;
479   op_table_start_cycle_count = 0;
480   op_table_start_last = 0;
481   op_table_start_en = 1'b0;
482
483   inc_active_op = 1'b0;
484
485   // segmentation and request generation
486   case (req_state_reg)
487     REQ_STATE_IDLE: begin
488       s_axis_read_desc_ready_next = !op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2**OP_TAG_WIDTH) && enable;
489
490       req_axi_addr_next = s_axis_read_desc_axi_addr;
491       req_ram_sel_next = s_axis_read_desc_ram_sel;
492       req_ram_addr_next = s_axis_read_desc_ram_addr;
493       if (s_axis_read_desc_len == 0) begin
494         // zero-length operation
495         req_op_count_next = 1;
496         req_zero_len_next = 1'b1;
497       end else begin
498         req_op_count_next = s_axis_read_desc_len;
499         req_zero_len_next = 1'b0;
500       end
501       req_tag_next = s_axis_read_desc_tag;
502
503       if (req_op_count_next <= AXI_MAX_BURST_SIZE -

```

```

(req_axi_addr_next & OFFSET_MASK) || AXI_MAX_BURST_SIZE >= 4096) be-
gin
504       // packet smaller than max burst size
505       if (((req_axi_addr_next & 12'hfff) +
(req_op_count_next & 12'hfff)) >> 12 != 0 || req_op_count_next >>
12 != 0) begin
506         // crosses 4k boundary
507         req_tr_count_next = 13'h1000 - req_axi_ad-
dr_next[11:0];
508       end else begin
509         // does not cross 4k boundary
510         req_tr_count_next = req_op_count_next;
511       end
512     end else begin
513       // packet larger than max burst size
514       if (((req_axi_addr_next & 12'hfff) +
AXI_MAX_BURST_SIZE) >> 12 != 0) begin
515         // crosses 4k boundary
516         req_tr_count_next = 13'h1000 - req_axi_ad-
dr_next[11:0];
517       end else begin
518         // does not cross 4k boundary
519         req_tr_count_next = AXI_MAX_BURST_SIZE -
(req_axi_addr_next & OFFSET_MASK);
520       end
521     end
522
523     if (s_axis_read_desc_ready && s_ax-
is_read_desc_valid) begin
524       s_axis_read_desc_ready_next = 1'b0;
525
526       stat_rd_op_start_tag_next =
stat_rd_op_start_tag_reg+1;
527       stat_rd_op_start_len_next = s_ax-
is_read_desc_len;
528       stat_rd_op_start_valid_next = 1'b1;
529
530       req_state_next = REQ_STATE_START;
531     end else begin
532       req_state_next = REQ_STATE_IDLE;
533     end
534   end
535   REQ_STATE_START: begin
536     if (!op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2**OP_TAG_WIDTH) && (!m_axi_arvalid || m_axi_arready)) begin
537       req_axi_addr_next = req_axi_addr_reg +
req_tr_count_reg;
538       req_ram_addr_next = req_ram_addr_reg +
req_tr_count_reg;
539       req_op_count_next = req_op_count_reg -
req_tr_count_reg;
540
541       op_table_start_axi_addr = req_axi_addr_reg;
542       op_table_start_ram_sel = req_ram_sel_reg;
543       op_table_start_ram_addr = req_ram_addr_reg;
544       op_table_start_len = req_tr_count_next;
545       op_table_start_zero_len = req_zero_len_reg;
546       op_table_start_tag = req_tag_reg;
547       op_table_start_cycle_count =
(req_tr_count_next + (req_axi_addr_reg & OFFSET_MASK) - 1) >>
AXI_BURST_SIZE;
548       op_table_start_last = req_op_count_reg ==
req_tr_count_next;
549       op_table_start_en = 1'b1;
550       inc_active_op = 1'b1;
551
552       stat_rd_req_start_tag_next = op_t-
able_start_ptr_reg[OP_TAG_WIDTH-1:0];
553       stat_rd_req_start_len_next =
req_zero_len_reg ? 0 : req_tr_count_reg;
554       stat_rd_req_start_valid_next = 1'b1;
555
556       m_axi_arid_next = op_t-
able_start_ptr_reg[OP_TAG_WIDTH-1:0];
557       m_axi_araddr_next = req_axi_addr_reg;
558       m_axi_arlen_next = op_table_start_cycle_count;
559       m_axi_arvalid_next = 1'b1;
560
561       if (req_op_count_next <= AXI_MAX_BURST_SIZE -
(req_axi_addr_next & OFFSET_MASK) || AXI_MAX_BURST_SIZE >= 4096) be-
gin
562         // packet smaller than max burst size
563         if (((req_axi_addr_next & 12'hfff) +
(req_op_count_next & 12'hfff)) >> 12 != 0 || req_op_count_next >>
12 != 0) begin
564           // crosses 4k boundary
565           req_tr_count_next = 13'h1000 -
req_axi_addr_next[11:0];
566         end else begin
567           // does not cross 4k boundary
568           req_tr_count_next = req_op_count_next;
569         end
570       end else begin
571         // packet larger than max burst size
572         if (((req_axi_addr_next & 12'hfff) +
AXI_MAX_BURST_SIZE) >> 12 != 0) begin
573           // crosses 4k boundary
574           req_tr_count_next = 13'h1000 -
req_axi_addr_next[11:0];

```

```

575         end else begin
576             // does not cross 4k boundary
577             req_tr_count_next = AXI_MAX_BURST_SIZE -
(req_axi_addr_next & OFFSET_MASK);
578             end
579         end
580     end
581     if (!op_table_start_last) begin
582         req_state_next = REQ_STATE_START;
583     end else begin
584         s_axis_read_desc_ready_next = !op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2*OP_TAG_WIDTH) && enable;
585         req_state_next = REQ_STATE_IDLE;
586     end
587     end else begin
588         req_state_next = REQ_STATE_START;
589     end
590 end
591 endcase
592 end
593
594 always @* begin
595     axi_state_next = AXI_STATE_IDLE;
596
597     m_axi_rready_next = 1'b0;
598
599     stat_rd_op_finish_tag_next = stat_rd_op_finish_tag_reg;
600     stat_rd_op_finish_status_next = stat_rd_op_fin-
ish_status_reg;
601     stat_rd_op_finish_valid_next = 1'b0;
602     stat_rd_req_finish_tag_next = stat_rd_req_finish_tag_reg;
603     stat_rd_req_finish_status_next = stat_rd_req_fin-
ish_status_reg;
604     stat_rd_req_finish_valid_next = 1'b0;
605
606     ram_sel_next = ram_sel_reg;
607     addr_next = addr_reg;
608     addr_delay_next = addr_delay_reg;
609     op_count_next = op_count_reg;
610     zero_len_next = zero_len_reg;
611     ram_mask_next = ram_mask_reg;
612     ram_mask_0_next = ram_mask_0_reg;
613     ram_mask_1_next = ram_mask_1_reg;
614     ram_wrap_next = ram_wrap_reg;
615     cycle_byte_count_next = cycle_byte_count_reg;
616     start_offset_next = start_offset_reg;
617     end_offset_next = end_offset_reg;
618     offset_next = offset_reg;
619     op_tag_next = op_tag_reg;
620
621     op_table_read_complete_en = 1'b0;
622
623     m_axi_rdata_int_next = m_axi_rdata_int_reg;
624     m_axi_rvalid_int_next = 1'b0;
625
626     status_fifo_mask_next = 1'b1;
627     status_fifo_finish_next = 1'b0;
628     status_fifo_error_next = DMA_ERROR_NONE;
629     status_fifo_we_next = 1'b0;
630
631     out_done_ack = {RAM_SEG_COUNT{1'b0}};
632
633     // Write generation
634     ram_wr_cmd_sel_int = {RAM_SEG_COUNT{ram_sel_reg}};
635     if (!ram_wrap_reg) begin
636         ram_wr_cmd_be_int =
({RAM_SEG_COUNT*RAM_SEG_BE_WIDTH{1'b1}} << start_offset_reg) &
({RAM_SEG_COUNT*RAM_SEG_BE_WIDTH{1'b1}} >>
(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH-1-end_offset_reg));
637     end else begin
638         ram_wr_cmd_be_int =
({RAM_SEG_COUNT*RAM_SEG_BE_WIDTH{1'b1}} << start_offset_reg) |
({RAM_SEG_COUNT*RAM_SEG_BE_WIDTH{1'b1}} >>
(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH-1-end_offset_reg));
639     end
640     for (i = 0; i < RAM_SEG_COUNT; i = i + 1) begin
641         ram_wr_cmd_addr_int[i*RAM_SEG_ADDR_WIDTH +:
RAM_SEG_ADDR_WIDTH] =
addr_delay_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-RAM_SEG_ADDR_WIDTH];
642         if (ram_mask_1_reg[i]) begin
643             ram_wr_cmd_addr_int[i*RAM_SEG_ADDR_WIDTH +:
RAM_SEG_ADDR_WIDTH] =
addr_delay_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-RAM_SEG_ADDR_WIDTH]
+1;
644         end
645     end
646     ram_wr_cmd_data_int = {3{m_axi_rdata_int_reg}} >>
(AXI_DATA_WIDTH - offset_reg*AXI_WORD_SIZE);
647     ram_wr_cmd_valid_int = {RAM_SEG_COUNT{1'b0}};
648
649     if (m_axi_rvalid_int_reg) begin
650         ram_wr_cmd_valid_int = ram_mask_reg;
651     end
652
653     // AXI read response handling
654     case (axi_state_reg)
655         AXI_STATE_IDLE: begin
656             // idle state, wait for read data

```

```

657         m_axi_rready_next = &ram_wr_cmd_ready_int && !
status_fifo_half_full_reg;
658
659         if (USE_AXI_ID) begin
660             op_tag_next = m_axi_rid[OP_TAG_WIDTH-1:0];
661         end else begin
662             op_tag_next = op_table_read_complete_ptr_reg;
663         end
664         ram_sel_next = op_table_ram_sel[op_tag_next];
665         addr_next = op_table_ram_addr[op_tag_next];
666         op_count_next = op_table_len[op_tag_next];
667         zero_len_next = op_table_zero_len[op_tag_next];
668         offset_next = op_table_ram_addr[op_tag_next]
[RAM_OFFSET_WIDTH-1:0]-(op_table_axi_addr[op_tag_next] & OFF-
SET_MASK);
669
670         if (m_axi_rready && m_axi_rvalid) begin
671             if (op_count_next > AXI_WORD_WIDTH-(op_t-
able_axi_addr[op_tag_next] & OFFSET_MASK)) begin
672                 cycle_byte_count_next = AXI_WORD_WIDTH-
(op_table_axi_addr[op_tag_next] & OFFSET_MASK);
673             end else begin
674                 cycle_byte_count_next = op_count_next;
675             end
676             start_offset_next = addr_next;
677             {ram_wrap_next, end_offset_next} =
start_offset_next+cycle_byte_count_next-1;
678
679             ram_mask_0_next = {RAM_SEG_COUNT{1'b1}} <<
(start_offset_next >> $clog2(RAM_SEG_BE_WIDTH));
680             ram_mask_1_next = {RAM_SEG_COUNT{1'b1}} >>
(RAM_SEG_COUNT-1-(end_offset_next >> $clog2(RAM_SEG_BE_WIDTH)));
681
682             if (!ram_wrap_next) begin
683                 ram_mask_next = ram_mask_0_next &
ram_mask_1_next;
684                 ram_mask_0_next = ram_mask_0_next &
ram_mask_1_next;
685                 ram_mask_1_next = 0;
686             end else begin
687                 ram_mask_next = ram_mask_0_next |
ram_mask_1_next;
688             end
689
690             addr_delay_next = addr_next;
691             addr_next = addr_next + cycle_byte_count_next;
692             op_count_next = op_count_next -
cycle_byte_count_next;
693
694             m_axi_rdata_int_next = m_axi_rdata;
695             m_axi_rvalid_int_next = 1'b1;
696
697             status_fifo_mask_next = 1'b1;
698             status_fifo_finish_next = 1'b0;
699             status_fifo_error_next = DMA_ERROR_NONE;
700             status_fifo_we_next = 1'b1;
701
702             if (zero_len_next) begin
703                 m_axi_rvalid_int_next = 1'b0;
704                 status_fifo_mask_next = 1'b0;
705             end
706
707             if (m_axi_rresp == AXI_RESP_SLVERR) begin
708                 m_axi_rvalid_int_next = 1'b0;
709                 status_fifo_mask_next = 1'b0;
710                 status_fifo_error_next = DMA_ER-
ROR_AXI_RD_SLVERR;
711             end else if (m_axi_rresp == AXI_RESP_DECERR)
begin
712                 m_axi_rvalid_int_next = 1'b0;
713                 status_fifo_mask_next = 1'b0;
714                 status_fifo_error_next = DMA_ER-
ROR_AXI_RD_DECERR;
715             end
716
717             stat_rd_req_finish_tag_next = op_tag_next;
718             stat_rd_req_finish_status_next =
status_fifo_error_next;
719             stat_rd_req_finish_valid_next = 1'b0;
720
721             if (!USE_AXI_ID) begin
722                 op_table_read_complete_en = 1'b1;
723             end
724
725             if (m_axi_rlast) begin
726                 status_fifo_finish_next = 1'b1;
727                 stat_rd_req_finish_valid_next = 1'b1;
728                 axi_state_next = AXI_STATE_IDLE;
729             end else begin
730                 axi_state_next = AXI_STATE_WRITE;
731             end
732             end else begin
733                 axi_state_next = AXI_STATE_IDLE;
734             end
735         end
736         AXI_STATE_WRITE: begin
737             // write state - generate write operations
738             m_axi_rready_next = &ram_wr_cmd_ready_int && !
status_fifo_half_full_reg;
739
740             if (m_axi_rready && m_axi_rvalid) begin

```

```

741
742     if (op_count_next > AXI_WORD_WIDTH) begin
743         cycle_byte_count_next = AXI_WORD_WIDTH;
744     end else begin
745         cycle_byte_count_next = op_count_next;
746     end
747     start_offset_next = addr_next;
748     {ram_wrap_next, end_offset_next} =
start_offset_next+cycle_byte_count_next-1;
749
750     ram_mask_0_next = {RAM_SEG_COUNT{1'b1}} <<
(start_offset_next >> $clog2(RAM_SEG_BE_WIDTH));
751     ram_mask_1_next = {RAM_SEG_COUNT{1'b1}} >>
(RAM_SEG_COUNT-1-(end_offset_next >> $clog2(RAM_SEG_BE_WIDTH)));
752
753     if (!ram_wrap_next) begin
754         ram_mask_next = ram_mask_0_next &
ram_mask_1_next;
755         ram_mask_0_next = ram_mask_0_next &
ram_mask_1_next;
756         ram_mask_1_next = 0;
757     end else begin
758         ram_mask_next = ram_mask_0_next |
ram_mask_1_next;
759     end
760
761     addr_delay_next = addr_next;
762     addr_next = addr_next + cycle_byte_count_next;
763     op_count_next = op_count_next -
cycle_byte_count_next;
764
765     m_axi_rdata_int_next = m_axi_rdata;
766     m_axi_rvalid_int_next = 1'b1;
767
768     status_fifo_mask_next = 1'b1;
769     status_fifo_finish_next = 1'b0;
770     status_fifo_error_next = DMA_ERROR_NONE;
771     status_fifo_we_next = 1'b1;
772
773     if (m_axi_rresp == AXI_RESP_SLVERR) begin
774         m_axi_rvalid_int_next = 1'b0;
775         status_fifo_mask_next = 1'b0;
776         status_fifo_error_next = DMA_ER-
ROR_AXI_RD_SLVERR;
777     end else if (m_axi_rresp == AXI_RESP_DECERR)
begin
778         m_axi_rvalid_int_next = 1'b0;
779         status_fifo_mask_next = 1'b0;
780         status_fifo_error_next = DMA_ER-
ROR_AXI_RD_DECERR;
781     end
782
783     stat_rd_req_finish_tag_next = op_tag_next;
784     stat_rd_req_finish_status_next =
status_fifo_error_next;
785     stat_rd_req_finish_valid_next = 1'b0;
786
787     if (m_axi_rlast) begin
788         status_fifo_finish_next = 1'b1;
789         stat_rd_req_finish_valid_next = 1'b1;
790         axi_state_next = AXI_STATE_IDLE;
791     end else begin
792         axi_state_next = AXI_STATE_WRITE;
793     end
794     end else begin
795         axi_state_next = AXI_STATE_WRITE;
796     end
797 end
798 endcase
799
800 status_fifo_rd_ptr_next = status_fifo_rd_ptr_reg;
801
802 status_fifo_wr_op_tag = op_tag_reg;
803 status_fifo_wr_mask = status_fifo_mask_reg ?
ram_mask_reg : 0;
804 status_fifo_wr_finish = status_fifo_finish_reg;
805 status_fifo_wr_error = status_fifo_error_reg;
806 status_fifo_we = 1'b0;
807
808 if (status_fifo_we_reg) begin
809     status_fifo_wr_op_tag = op_tag_reg;
810     status_fifo_wr_mask = status_fifo_mask_reg ?
ram_mask_reg : 0;
811     status_fifo_wr_finish = status_fifo_finish_reg;
812     status_fifo_wr_error = status_fifo_error_reg;
813     status_fifo_we = 1'b1;
814 end
815
816 status_fifo_rd_op_tag_next = status_fifo_rd_op_tag_reg;
817 status_fifo_rd_mask_next = status_fifo_rd_mask_reg;
818 status_fifo_rd_finish_next = status_fifo_rd_finish_reg;
819 status_fifo_rd_valid_next = status_fifo_rd_valid_reg;
820 status_fifo_rd_error_next = status_fifo_rd_error_reg;
821
822 op_table_update_status_ptr = status_fifo_rd_op_tag_reg;
823 op_table_update_status_error = status_fifo_rd_error_reg;
824 op_table_update_status_en = 1'b0;
825
826 op_table_write_complete_ptr = status_fifo_rd_op_tag_reg;
827 op_table_write_complete_en = 1'b0;
828

```

```

829     if (status_fifo_rd_valid_reg && (status_fifo_rd_mask_reg &
~out_done) == 0) begin
830         // got write completion, pop and return status
831         status_fifo_rd_valid_next = 1'b0;
832         op_table_update_status_en = 1'b1;
833
834         out_done_ack = status_fifo_rd_mask_reg;
835
836         if (status_fifo_rd_finish_reg) begin
837             // mark done
838             op_table_write_complete_ptr =
status_fifo_rd_op_tag_reg;
839             op_table_write_complete_en = 1'b1;
840         end
841     end
842
843     if (!status_fifo_rd_valid_next && status_fifo_rd_ptr_reg !
= status_fifo_wr_ptr_reg) begin
844         // status FIFO not empty
845         status_fifo_rd_op_tag_next =
status_fifo_op_tag[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]];
846         status_fifo_rd_mask_next =
status_fifo_mask[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
;
847         status_fifo_rd_finish_next =
status_fifo_finish[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]];
848         status_fifo_rd_error_next =
status_fifo_error[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]];
849         status_fifo_rd_valid_next = 1'b1;
850         status_fifo_rd_ptr_next = status_fifo_rd_ptr_reg + 1;
851     end
852
853     // commit operations in-order
854     op_table_finish_en = 1'b0;
855     dec_active_op = 1'b0;
856
857     if (m_axis_read_desc_status_valid_reg) begin
858         m_axis_read_desc_status_error_next = DMA_ERROR_NONE;
859     end else begin
860         m_axis_read_desc_status_error_next = m_ax-
is_read_desc_status_error_reg;
861     end
862
863     m_axis_read_desc_status_tag_next = op_t-
able_tag[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
864     m_axis_read_desc_status_valid_next = 1'b0;
865
866     stat_rd_op_finish_tag_next = stat_rd_op_finish_tag_reg;
867     stat_rd_op_finish_status_next = m_ax-
is_read_desc_status_error_next;
868     stat_rd_op_finish_valid_next = 1'b0;
869
870     if (op_t-
able_active[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] && op_t-
able_write_complete[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] &&
op_table_finish_ptr_reg != op_table_start_ptr_reg) begin
871         op_table_finish_en = 1'b1;
872         dec_active_op = 1'b1;
873
874         if (op_t-
able_error_a[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] != op_t-
able_error_b[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]]) begin
875             m_axis_read_desc_status_error_next = op_t-
able_error_code[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
876         end
877
878         stat_rd_op_finish_status_next = m_ax-
is_read_desc_status_error_next;
879
880         if (op_t-
able_last[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]]) begin
881             m_axis_read_desc_status_tag_next = op_t-
able_tag[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
882             m_axis_read_desc_status_valid_next = 1'b1;
883             stat_rd_op_finish_tag_next = stat_rd_op_fin-
ish_tag_reg + 1;
884             stat_rd_op_finish_valid_next = 1'b1;
885         end
886     end
887 end
888
889 always @(posedge clk) begin
890     req_state_reg <= req_state_next;
891     axi_state_reg <= axi_state_next;
892
893     req_axi_addr_reg <= req_axi_addr_next;
894     req_ram_sel_reg <= req_ram_sel_next;
895     req_ram_addr_reg <= req_ram_addr_next;
896     req_op_count_reg <= req_op_count_next;
897     req_tr_count_reg <= req_tr_count_next;
898     req_zero_len_reg <= req_zero_len_next;
899     req_tag_reg <= req_tag_next;
900
901     ram_sel_reg <= ram_sel_next;
902     addr_reg <= addr_next;
903     addr_delay_reg <= addr_delay_next;
904     op_count_reg <= op_count_next;

```

```

905 zero_len_reg <= zero_len_next;
906 ram_mask_reg <= ram_mask_next;
907 ram_mask_0_reg <= ram_mask_0_next;
908 ram_mask_1_reg <= ram_mask_1_next;
909 ram_wrap_reg <= ram_wrap_next;
910 cycle_byte_count_reg <= cycle_byte_count_next;
911 start_offset_reg <= start_offset_next;
912 end_offset_reg <= end_offset_next;
913 offset_reg <= offset_next;
914 op_tag_reg <= op_tag_next;
915
916 m_axi_rdata_int_reg <= m_axi_rdata_int_next;
917 m_axi_rvalid_int_reg <= m_axi_rvalid_int_next;
918
919 m_axi_arid_reg <= m_axi_arid_next;
920 m_axi_araddr_reg <= m_axi_araddr_next;
921 m_axi_arlen_reg <= m_axi_arlen_next;
922 m_axi_arvalid_reg <= m_axi_arvalid_next;
923 m_axi_rready_reg <= m_axi_rready_next;
924
925 s_axis_read_desc_ready_reg <= s_axis_read_desc_ready_next;
926
927 m_axis_read_desc_status_tag_reg <= m_ax-
is_read_desc_status_tag_next;
928 m_axis_read_desc_status_error_reg <= m_ax-
is_read_desc_status_error_next;
929 m_axis_read_desc_status_valid_reg <= m_ax-
is_read_desc_status_valid_next;
930
931 status_busy_reg <= active_op_count_reg != 0;
932
933 stat_rd_op_start_tag_reg <= stat_rd_op_start_tag_next;
934 stat_rd_op_start_len_reg <= stat_rd_op_start_len_next;
935 stat_rd_op_start_valid_reg <= stat_rd_op_start_valid_next;
936 stat_rd_op_finish_tag_reg <= stat_rd_op_finish_tag_next;
937 stat_rd_op_finish_status_reg <= stat_rd_op_fin-
ish_status_next;
938 stat_rd_op_finish_valid_reg <= stat_rd_op_fin-
ish_valid_next;
939 stat_rd_req_start_tag_reg <= stat_rd_req_start_tag_next;
940 stat_rd_req_start_len_reg <= stat_rd_req_start_len_next;
941 stat_rd_req_start_valid_reg <=
stat_rd_req_start_valid_next;
942 stat_rd_req_finish_tag_reg <= stat_rd_req_finish_tag_next;
943 stat_rd_req_finish_status_reg <= stat_rd_req_fin-
ish_status_next;
944 stat_rd_req_finish_valid_reg <= stat_rd_req_fin-
ish_valid_next;
945 stat_rd_op_table_full_reg <= stat_rd_op_table_full_next;
946 stat_rd_tx_stall_reg <= stat_rd_tx_stall_next;
947
948 if (status_fifo_we) begin
949
status_fifo_op_tag[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]] <= status_fifo_wr_op_tag;
950
status_fifo_mask[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_mask;
951
status_fifo_finish[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]] <= status_fifo_wr_finish;
952
status_fifo_error[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]
] <= status_fifo_wr_error;
953
status_fifo_wr_ptr_reg <= status_fifo_wr_ptr_reg + 1;
954
end
955
status_fifo_rd_ptr_reg <= status_fifo_rd_ptr_next;
956
status_fifo_mask_reg <= status_fifo_mask_next;
957
status_fifo_finish_reg <= status_fifo_finish_next;
958
status_fifo_error_reg <= status_fifo_error_next;
959
status_fifo_we_reg <= status_fifo_we_next;
960
961
status_fifo_rd_op_tag_reg <= status_fifo_rd_op_tag_next;
962
status_fifo_rd_mask_reg <= status_fifo_rd_mask_next;
963
status_fifo_rd_finish_reg <= status_fifo_rd_finish_next;
964
status_fifo_rd_error_reg <= status_fifo_rd_error_next;
965
status_fifo_rd_valid_reg <= status_fifo_rd_valid_next;
966
967
968
status_fifo_half_full_reg <= $un-
signed(status_fifo_wr_ptr_reg - status_fifo_rd_ptr_reg) >=
2*(STATUS_FIFO_ADDR_WIDTH-1);
969
970 active_op_count_reg <= active_op_count_reg + inc_active_op -
dec_active_op;
971
972 if (op_table_start_en) begin
973   op_table_start_ptr_reg <= op_table_start_ptr_reg + 1;
974   op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= 1'b1;
975   op_t-
able_axi_addr[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_axi_addr;
976   op_t-
able_ram_sel[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_ram_sel;
977   op_t-
able_ram_addr[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_ram_addr;

```

```

978   op_table_len[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]]
<= op_table_start_len;
979   op_t-
able_zero_len[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_zero_len;
980   op_t-
able_cycle_count[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_cycle_count;
981
982   op_table_tag[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]]
<= op_table_start_tag;
983   op_t-
able_last[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_last;
984   op_t-
able_write_complete[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <=
1'b0;
985   op_t-
able_error_a[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_error_b[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]];
986   end
987   if (!USE_AXI_ID && op_table_read_complete_en) begin
988     op_table_read_complete_ptr_reg <= op_t-
able_read_complete_ptr_reg + 1;
989   end
990
991   if (op_table_update_status_en) begin
992     if (op_table_update_status_error != 0) begin
993       op_table_error_code[op_table_update_status_ptr] <=
op_table_update_status_error;
994       op_table_error_b[op_table_update_status_ptr] <= !
op_table_error_a[op_table_update_status_ptr];
995     end
996   end
997
998   if (op_table_write_complete_en) begin
999     op_table_write_complete[op_table_write_complete_ptr]
<= 1'b1;
1000   end
1001
1002   if (op_table_finish_en) begin
1003     op_table_finish_ptr_reg <= op_table_finish_ptr_reg +
1;
1004     op_t-
able_active[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] <= 1'b0;
1005   end
1006
1007   if (rst) begin
1008     req_state_reg <= REQ_STATE_IDLE;
1009     axi_state_reg <= AXI_STATE_IDLE;
1010
m_axi_rdata_int_reg <= 1'b0;
1011
m_axi_arvalid_reg <= 1'b0;
1012
m_axi_rready_reg <= 1'b0;
1013
s_axis_read_desc_ready_reg <= 1'b0;
1014
m_axis_read_desc_status_error_reg = 4'd0;
1015
m_axis_read_desc_status_valid_reg <= 1'b0;
1016
status_busy_reg <= 1'b0;
1017
stat_rd_op_start_tag_reg <= 0;
1018
stat_rd_op_start_valid_reg <= 1'b0;
1019
stat_rd_op_finish_tag_reg <= 0;
1020
stat_rd_op_finish_valid_reg <= 1'b0;
1021
stat_rd_req_start_valid_reg <= 1'b0;
1022
stat_rd_req_finish_valid_reg <= 1'b0;
1023
stat_rd_op_table_full_reg <= 1'b0;
1024
stat_rd_tx_stall_reg <= 1'b0;
1025
status_fifo_wr_ptr_reg <= 0;
1026
status_fifo_rd_ptr_reg <= 0;
1027
status_fifo_we_reg <= 1'b0;
1028
status_fifo_rd_valid_reg <= 1'b0;
1029
active_op_count_reg <= 0;
1030
op_table_start_ptr_reg <= 0;
1031
op_table_read_complete_ptr_reg <= 0;
1032
op_table_finish_ptr_reg <= 0;
1033
op_table_active <= 0;
1034
end
1035
end
1036
// output datapath logic (write data)
1037 generate
1038   genvar n;
1039   for (n = 0; n < RAM_SEG_COUNT; n = n + 1) begin
1040     reg [RAM_SEL_WIDTH-1:0] ram_wr_cmd_sel_reg =
{RAM_SEL_WIDTH{1'b0}};
1041     reg [RAM_SEG_BE_WIDTH-1:0] ram_wr_cmd_be_reg =
{RAM_SEG_BE_WIDTH{1'b0}};
1042     reg [RAM_SEG_ADDR_WIDTH-1:0] ram_wr_cmd_addr_reg =
{RAM_SEG_ADDR_WIDTH{1'b0}};
1043     reg [RAM_SEG_DATA_WIDTH-1:0] ram_wr_cmd_data_reg =

```



```

(RAM_SEG_DATA_WIDTH{1'b0});
1056 reg ram_wr_cmd_valid_reg = 1'b0;
1057
1058 reg [OUTPUT_FIFO_ADDR_WIDTH-1:0] out_fifo_wr_ptr_reg = 0;
1059 reg [OUTPUT_FIFO_ADDR_WIDTH-1:0] out_fifo_rd_ptr_reg = 0;
1060 reg out_fifo_half_full_reg = 1'b0;
1061
1062 wire out_fifo_full = out_fifo_wr_ptr_reg ==
(out_fifo_rd_ptr_reg ^ {1'b1, {OUTPUT_FIFO_ADDR_WIDTH{1'b0}}});
1063 wire out_fifo_empty = out_fifo_wr_ptr_reg ==
out_fifo_rd_ptr_reg;
1064
1065 (* ram_style = "distributed", ramstyle = "no_rw_check,
mlab" *)
1066 reg [RAM_SEL_WIDTH-1:0]
out_fifo_wr_cmd_sel[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
1067 (* ram_style = "distributed", ramstyle = "no_rw_check,
mlab" *)
1068 reg [RAM_SEG_BE_WIDTH-1:0]
out_fifo_wr_cmd_be[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
1069 (* ram_style = "distributed", ramstyle = "no_rw_check,
mlab" *)
1070 reg [RAM_SEG_ADDR_WIDTH-1:0]
out_fifo_wr_cmd_addr[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
1071 (* ram_style = "distributed", ramstyle = "no_rw_check,
mlab" *)
1072 reg [RAM_SEG_DATA_WIDTH-1:0]
out_fifo_wr_cmd_data[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
1073
1074 reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] done_count_reg = 0;
1075 reg done_reg = 1'b0;
1076
1077 assign ram_wr_cmd_ready_int[n+:1] = !
out_fifo_half_full_reg;
1078
1079 assign ram_wr_cmd_sel[n*RAM_SEL_WIDTH+:RAM_SEL_WIDTH] =
ram_wr_cmd_sel_reg;
1080 assign ram_wr_cmd_be[n*RAM_SEG_BE_WIDTH+:
RAM_SEG_BE_WIDTH] = ram_wr_cmd_be_reg;
1081 assign ram_wr_cmd_addr[n*RAM_SEG_ADDR_WIDTH+:RAM_SEG_AD-
DR_WIDTH] = ram_wr_cmd_addr_reg;
1082 assign ram_wr_cmd_data[n*RAM_SEG_DATA_WIDTH+:
RAM_SEG_DATA_WIDTH] = ram_wr_cmd_data_reg;
1083 assign ram_wr_cmd_valid[n+:1] = ram_wr_cmd_valid_reg;
1084
1085 assign out_done[n] = done_reg;
1086
1087 always @(posedge clk) begin
1088 ram_wr_cmd_valid_reg <= ram_wr_cmd_valid_reg && !
ram_wr_cmd_ready[n+:1];
1089
1090 out_fifo_half_full_reg <= $un-
signed(out_fifo_wr_ptr_reg - out_fifo_rd_ptr_reg) >= 2** (OUT-
PUT_FIFO_ADDR_WIDTH-1);
1091
1092 if (!out_fifo_full && ram_wr_cmd_valid_int[n+:1])
begin
1093
out_fifo_wr_cmd_sel[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
<= ram_wr_cmd_sel_int[n*RAM_SEL_WIDTH+:RAM_SEL_WIDTH];
1094
out_fifo_wr_cmd_be[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
<= ram_wr_cmd_be_int[n*RAM_SEG_BE_WIDTH+:RAM_SEG_BE_WIDTH];
1095
out_fifo_wr_cmd_addr[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]
] <= ram_wr_cmd_addr_int[n*RAM_SEG_ADDR_WIDTH+:RAM_SEG_AD-
DR_WIDTH];
1096
out_fifo_wr_cmd_data[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]
] <= ram_wr_cmd_data_int[n*RAM_SEG_DATA_WIDTH+:
RAM_SEG_DATA_WIDTH];
1097 out_fifo_wr_ptr_reg <= out_fifo_wr_ptr_reg + 1;
1098 end
1099
1100 if (!out_fifo_empty && (!ram_wr_cmd_valid_reg ||
ram_wr_cmd_ready[n+:1])) begin
1101 ram_wr_cmd_sel_reg <=
out_fifo_wr_cmd_sel[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
;
1102 ram_wr_cmd_be_reg <=
out_fifo_wr_cmd_be[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
;
1103 ram_wr_cmd_addr_reg <=
out_fifo_wr_cmd_addr[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]
];
1104 ram_wr_cmd_data_reg <=
out_fifo_wr_cmd_data[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]
];
1105 ram_wr_cmd_valid_reg <= 1'b1;
1106 out_fifo_rd_ptr_reg <= out_fifo_rd_ptr_reg + 1;
1107 end
1108
1109 if (done_count_reg < 2**OUTPUT_FIFO_ADDR_WIDTH &&
ram_wr_done[n] && !out_done_ack[n]) begin
1110 done_count_reg <= done_count_reg + 1;
1111 done_reg <= 1;
1112 end else if (done_count_reg > 0 && !ram_wr_done[n] &&
out_done_ack[n]) begin
1113 done_count_reg <= done_count_reg - 1;
1114 done_reg <= done_count_reg > 1;
1115 end
1116

```

```

1117 if (rst) begin
1118 out_fifo_wr_ptr_reg <= 0;
1119 out_fifo_rd_ptr_reg <= 0;
1120 ram_wr_cmd_valid_reg <= 1'b0;
1121 done_count_reg <= 0;
1122 done_reg <= 1'b0;
1123 end
1124 end
1125
1126 end
1127
1128 endgenerate
1129
1130 endmodule
1131
1132 `resetall

```

7. alexforenrich_verilog-pcie/rtl/dma_if_axi_wr.v (1,249 lines)

```

1 /*
2
3 Copyright (c) 2021 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * AXI DMA write interface
33 */
34 module dma_if_axi_wr #
35 (
36 // Width of AXI data bus in bits
37 parameter AXI_DATA_WIDTH = 32,
38 // Width of AXI address bus in bits
39 parameter AXI_ADDR_WIDTH = 16,
40 // Width of AXI wstrb (width of data bus in words)
41 parameter AXI_STRB_WIDTH = (AXI_DATA_WIDTH/8),
42 // Width of AXI ID signal
43 parameter AXI_ID_WIDTH = 8,
44 // Maximum AXI burst length to generate
45 parameter AXI_MAX_BURST_LEN = 256,
46 // RAM select width
47 parameter RAM_SEL_WIDTH = 2,
48 // RAM address width
49 parameter RAM_ADDR_WIDTH = 16,
50 // RAM segment count
51 parameter RAM_SEG_COUNT = 2,
52 // RAM segment data width
53 parameter RAM_SEG_DATA_WIDTH = AXI_DATA_WIDTH*2/
RAM_SEG_COUNT,
54 // RAM segment byte enable width
55 parameter RAM_SEG_BE_WIDTH = RAM_SEG_DATA_WIDTH/8,
56 // RAM segment address width
57 parameter RAM_SEG_ADDR_WIDTH = RAM_ADDR_WIDTH-$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH),
58 // Immediate enable
59 parameter IMM_ENABLE = 0,
60 // Immediate width
61 parameter IMM_WIDTH = 32,
62 // Length field width
63 parameter LEN_WIDTH = 16,
64 // Tag field width
65 parameter TAG_WIDTH = 8,
66 // Operation table size

```

```

67     parameter OP_TABLE_SIZE = 2**(AXI_ID_WIDTH),
68     // Use AXI ID signals
69     parameter USE_AXI_ID = 1
70 )
71 (
72     input wire          clk,
73     input wire          rst,
74
75     /*
76      * AXI master interface
77      */
78     output wire [AXI_ID_WIDTH-1:0] m_axi_awid,
79     output wire [AXI_ADDR_WIDTH-1:0] m_axi_awaddr,
80     output wire [7:0] m_axi_awlen,
81     output wire [2:0] m_axi_awsz,
82     output wire [1:0] m_axi_awburst,
83     output wire m_axi_awlock,
84     output wire [3:0] m_axi_awcache,
85     output wire [2:0] m_axi_awprot,
86     output wire m_axi_awvalid,
87     input wire m_axi_awready,
88     output wire [AXI_DATA_WIDTH-1:0] m_axi_wdata,
89     output wire [AXI_STRB_WIDTH-1:0] m_axi_wstrb,
90     output wire m_axi_wlast,
91     output wire m_axi_wvalid,
92     input wire m_axi_wready,
93     input wire [AXI_ID_WIDTH-1:0] m_axi_bid,
94     input wire [1:0] m_axi_bresp,
95     input wire m_axi_bvalid,
96     output wire m_axi_bready,
97
98     /*
99      * AXI write descriptor input
100     */
101     input wire [AXI_ADDR_WIDTH-1:0] is_write_desc_axi_addr,
102     input wire [RAM_SEL_WIDTH-1:0] is_write_desc_ram_sel,
103     input wire [RAM_ADDR_WIDTH-1:0] is_write_desc_ram_addr,
104     input wire [IMM_WIDTH-1:0] is_write_desc_imm,
105     input wire is_write_desc_imm_en,
106     input wire [LEN_WIDTH-1:0] is_write_desc_len,
107     input wire [TAG_WIDTH-1:0] is_write_desc_tag,
108     input wire is_write_desc_valid,
109     output wire is_write_desc_ready,
110
111     /*
112      * AXI write descriptor status output
113      */
114     output wire [TAG_WIDTH-1:0] is_write_desc_status_tag,
115     output wire [3:0] is_write_desc_status_error,
116     output wire is_write_desc_status_valid,
117
118     /*
119      * RAM interface
120      */
121     output wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0] ram_rd_cmd_sel,
122     output wire [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0] ram_rd_cmd_addr,
123     output wire [RAM_SEG_COUNT-1:0] ram_rd_cmd_valid,
124     input wire [RAM_SEG_COUNT-1:0] ram_rd_cmd_ready,
125     input wire [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0] ram_rd_resp_data,
126     input wire [RAM_SEG_COUNT-1:0] ram_rd_resp_valid,
127     output wire [RAM_SEG_COUNT-1:0] ram_rd_resp_ready,
128
129     /*

```

```

130     * Configuration
131     */
132     input wire          en-
133
134     /*
135      * Status
136      */
137     output wire status_busy,
138
139     /*
140      * Statistics
141      */
142     output wire [$clog2(OP_TABLE_SIZE)-1:0] stat_wr_op_start_tag,
143     output wire [LEN_WIDTH-1:0] stat_wr_op_start_len,
144     output wire stat_wr_op_start_valid,
145     output wire [$clog2(OP_TABLE_SIZE)-1:0] stat_wr_op_finish_tag,
146     output wire [3:0] stat_wr_op_finish_status,
147     output wire stat_wr_op_finish_valid,
148     output wire [$clog2(OP_TABLE_SIZE)-1:0] stat_wr_req_start_tag,
149     output wire [12:0] stat_wr_req_start_len,
150     output wire stat_wr_req_start_valid,
151     output wire [$clog2(OP_TABLE_SIZE)-1:0] stat_wr_req_finish_tag,
152     output wire [3:0] stat_wr_req_finish_status,
153     output wire stat_wr_req_finish_valid,
154     output wire stat_wr_op_table_full,
155     output wire stat_wr_tx_stall
156 );
157
158 parameter RAM_DATA_WIDTH = RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH;
159 parameter RAM_WORD_WIDTH = RAM_SEG_BE_WIDTH;
160 parameter RAM_WORD_SIZE = RAM_SEG_DATA_WIDTH/RAM_WORD_WIDTH;
161
162 parameter AXI_WORD_WIDTH = AXI_STRB_WIDTH;
163 parameter AXI_WORD_SIZE = AXI_DATA_WIDTH/AXI_WORD_WIDTH;
164 parameter AXI_BURST_SIZE = $clog2(AXI_STRB_WIDTH);
165 parameter AXI_MAX_BURST_SIZE = AXI_MAX_BURST_LEN <<
AXI_BURST_SIZE;
166
167 parameter OFFSET_WIDTH = AXI_STRB_WIDTH > 1 ? $clog2(AXI_STRB_WIDTH) : 1;
168 parameter OFFSET_MASK = AXI_STRB_WIDTH > 1 ? {OFF-
SET_WIDTH{1'b1}} : 0;
169 parameter RAM_OFFSET_WIDTH = $clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH);
170 parameter ADDR_MASK = {AXI_ADDR_WIDTH{1'b1}} << $clog2(AXI_STRB_WIDTH);
171 parameter CYCLE_COUNT_WIDTH = LEN_WIDTH - AXI_BURST_SIZE + 1;
172
173 parameter MASK_FIFO_ADDR_WIDTH = $clog2(OP_TABLE_SIZE)+1;
174
175 parameter OP_TAG_WIDTH = $clog2(OP_TABLE_SIZE);
176
177 parameter OUTPUT_FIFO_ADDR_WIDTH = 5;
178
179 // bus width assertions
180 initial begin
181     if (AXI_WORD_SIZE * AXI_STRB_WIDTH != AXI_DATA_WIDTH) be-
gin
182         $error("Error: AXI data width not evenly divisble (in-
stance %m)");
183         $finish;
184     end
185
186     if (AXI_WORD_SIZE != RAM_WORD_SIZE) begin
187         $error("Error: word size mismatch (instance %m)");
188         $finish;
189     end
190
191     if (2**$clog2(AXI_WORD_WIDTH) != AXI_WORD_WIDTH) begin
192         $error("Error: AXI word width must be even power of
two (instance %m)");
193         $finish;
194     end
195
196     if (AXI_MAX_BURST_LEN < 1 || AXI_MAX_BURST_LEN > 256) be-
gin
197         $error("Error: AXI_MAX_BURST_LEN must be between 1 and
256 (instance %m)");
198         $finish;
199     end
200
201     if (RAM_SEG_COUNT < 2) begin
202         $error("Error: RAM interface requires at least 2 seg-
ments (instance %m)");

```



```

203     $finish;
204 end
205
206 if (RAM_DATA_WIDTH != AXI_DATA_WIDTH*2) begin
207     $error("Error: RAM interface width must be double the
AXI interface width (instance %m)");
208     $finish;
209 end
210
211 if (2**$clog2(RAM_WORD_WIDTH) != RAM_WORD_WIDTH) begin
212     $error("Error: RAM word width must be even power of
two (instance %m)");
213     $finish;
214 end
215
216 if (RAM_ADDR_WIDTH != RAM_SEG_ADDR_WIDTH+$clog2(RAM_SEG_COUNT)+$clog2(RAM_SEG_BE_WIDTH)) begin
217     $error("Error: RAM_ADDR_WIDTH does not match RAM configuration (instance %m)");
218     $finish;
219 end
220
221 if (OP_TABLE_SIZE > 2**AXI_ID_WIDTH) begin
222     $error("Error: AXI_ID_WIDTH insufficient for requested
OP_TABLE_SIZE (instance %m)");
223     $finish;
224 end
225
226 if (IMM_ENABLE && IMM_WIDTH > AXI_DATA_WIDTH) begin
227     $error("Error: IMM_WIDTH must not be larger than the
AXI interface width (instance %m)");
228     $finish;
229 end
230 end
231
232 localparam [1:0]
233     AXI_RESP_OKAY = 2'b00,
234     AXI_RESP_EXOKAY = 2'b01,
235     AXI_RESP_SLVERR = 2'b10,
236     AXI_RESP_DECERR = 2'b11;
237
238 localparam [3:0]
239     DMA_ERROR_NONE = 4'd0,
240     DMA_ERROR_TIMEOUT = 4'd1,
241     DMA_ERROR_PARITY = 4'd2,
242     DMA_ERROR_AXI_RD_SLVERR = 4'd4,
243     DMA_ERROR_AXI_RD_DECERR = 4'd5,
244     DMA_ERROR_AXI_WR_SLVERR = 4'd6,
245     DMA_ERROR_AXI_WR_DECERR = 4'd7,
246     DMA_ERROR_PCIE_FLR = 4'd8,
247     DMA_ERROR_PCIE_CPL_POISONED = 4'd9,
248     DMA_ERROR_PCIE_CPL_STATUS_UR = 4'd10,
249     DMA_ERROR_PCIE_CPL_STATUS_CA = 4'd11;
250
251 localparam [0:0]
252     REQ_STATE_IDLE = 1'd0,
253     REQ_STATE_START = 1'd1;
254
255 reg [0:0] req_state_reg = REQ_STATE_IDLE, req_state_next;
256
257 localparam [0:0]
258     READ_STATE_IDLE = 1'd0,
259     READ_STATE_READ = 1'd1;
260
261 reg [0:0] read_state_reg = READ_STATE_IDLE, read_state_next;
262
263 localparam [0:0]
264     AXI_STATE_IDLE = 1'd0,
265     AXI_STATE_TRANSFER = 1'd1;
266
267 reg [0:0] axi_state_reg = AXI_STATE_IDLE, axi_state_next;
268
269 // datapath control signals
270 reg mask_fifo_we;
271
272 reg read_cmd_ready;
273
274 reg [AXI_ADDR_WIDTH-1:0] req_axi_addr_reg = {AXI_ADDR_WIDTH{1'b0}}, req_axi_addr_next;
275 reg [RAM_SEL_WIDTH-1:0] ram_sel_reg = {RAM_SEL_WIDTH{1'b0}}, ram_sel_next;
276 reg [RAM_ADDR_WIDTH-1:0] ram_addr_reg = {RAM_ADDR_WIDTH{1'b0}}, ram_addr_next;
277 reg [IMM_WIDTH-1:0] imm_reg = {IMM_WIDTH{1'b0}}, imm_next;
278 reg [imm_en_reg = 1'b0, imm_en_next;
279 reg [LEN_WIDTH-1:0] op_count_reg = {LEN_WIDTH{1'b0}}, op_count_next;
280 reg [zero_len_reg = 1'b0, zero_len_next;
281 reg [LEN_WIDTH-1:0] tr_count_reg = {LEN_WIDTH{1'b0}}, tr_count_next;
282 reg [12:0] tr_word_count_reg = 13'd0, tr_word_count_next;
283 reg [TAG_WIDTH-1:0] tag_reg = {TAG_WIDTH{1'b0}}, tag_next;
284
285 reg [AXI_ADDR_WIDTH-1:0] read_axi_addr_reg = {AXI_ADDR_WIDTH{1'b0}}, read_axi_addr_next;
286 reg [RAM_SEL_WIDTH-1:0] read_ram_sel_reg = {RAM_SEL_WIDTH{1'b0}}, read_ram_sel_next;
287 reg [RAM_ADDR_WIDTH-1:0] read_ram_addr_reg = {RAM_ADDR_WIDTH{1'b0}}, read_ram_addr_next;
288 reg [read_imm_en_reg = 1'b0, read_imm_en_next;

```

```

289 reg [LEN_WIDTH-1:0] read_len_reg = {LEN_WIDTH{1'b0}}, read_len_next;
290 reg [RAM_SEG_COUNT-1:0] read_ram_mask_reg = {RAM_SEG_COUNT{1'b0}}, read_ram_mask_next;
291 reg [RAM_SEG_COUNT-1:0] read_ram_mask_0_reg = {RAM_SEG_COUNT{1'b0}}, read_ram_mask_0_next;
292 reg [RAM_SEG_COUNT-1:0] read_ram_mask_1_reg = {RAM_SEG_COUNT{1'b0}}, read_ram_mask_1_next;
293 reg [ram_wrap_reg = 1'b0, ram_wrap_next;
294 reg [CYCLE_COUNT_WIDTH-1:0] read_cycle_count_reg = {CYCLE_COUNT_WIDTH{1'b0}}, read_cycle_count_next;
295 reg [read_last_cycle_reg = 1'b0, read_last_cycle_next;
296 reg [OFFSET_WIDTH+1-1:0] cycle_byte_count_reg = {OFFSET_WIDTH+1{1'b0}}, cycle_byte_count_next;
297 reg [RAM_OFFSET_WIDTH-1:0] start_offset_reg = {RAM_OFFSET_WIDTH{1'b0}}, start_offset_next;
298 reg [RAM_OFFSET_WIDTH-1:0] end_offset_reg = {RAM_OFFSET_WIDTH{1'b0}}, end_offset_next;
299
300 reg [AXI_ADDR_WIDTH-1:0] axi_addr_reg = {AXI_ADDR_WIDTH{1'b0}}, axi_addr_next;
301 reg [IMM_WIDTH-1:0] axi_imm_reg = {IMM_WIDTH{1'b0}}, axi_imm_next;
302 reg [axi_imm_en_reg = 1'b0, axi_imm_en_next;
303 reg [12:0] axi_len_reg = 13'd0, axi_len_next;
304 reg [axi_zero_len_reg = 1'b0, axi_zero_len_next;
305 reg [RAM_OFFSET_WIDTH-1:0] offset_reg = {RAM_OFFSET_WIDTH{1'b0}}, offset_next;
306 reg [AXI_STRB_WIDTH-1:0] strb_offset_mask_reg = {AXI_STRB_WIDTH{1'b1}}, strb_offset_mask_next;
307 reg [OFFSET_WIDTH-1:0] last_cycle_offset_reg = {OFFSET_WIDTH{1'b0}}, last_cycle_offset_next;
308 reg [RAM_SEG_COUNT-1:0] ram_mask_reg = {RAM_SEG_COUNT{1'b0}}, ram_mask_next;
309 reg [ram_mask_valid_reg = 1'b0, ram_mask_valid_next;
310 reg [CYCLE_COUNT_WIDTH-1:0] cycle_count_reg = {CYCLE_COUNT_WIDTH{1'b0}}, cycle_count_next;
311 reg [last_cycle_reg = 1'b0, last_cycle_next;
312
313 reg [AXI_ADDR_WIDTH-1:0] read_cmd_axi_addr_reg = {AXI_ADDR_WIDTH{1'b0}}, read_cmd_axi_addr_next;
314 reg [RAM_SEL_WIDTH-1:0] read_cmd_ram_sel_reg = {RAM_SEL_WIDTH{1'b0}}, read_cmd_ram_sel_next;
315 reg [RAM_ADDR_WIDTH-1:0] read_cmd_ram_addr_reg = {RAM_ADDR_WIDTH{1'b0}}, read_cmd_ram_addr_next;
316 reg [read_cmd_imm_en_reg = 1'b0, read_cmd_imm_en_next;
317 reg [12:0] read_cmd_len_reg = 13'd0, read_cmd_len_next;
318 reg [CYCLE_COUNT_WIDTH-1:0] read_cmd_cycle_count_reg = {CYCLE_COUNT_WIDTH{1'b0}}, read_cmd_cycle_count_next;
319 reg [read_cmd_last_cycle_reg = 1'b0, read_cmd_last_cycle_next;
320 reg [read_cmd_valid_reg = 1'b0, read_cmd_valid_next;
321
322 reg [MASK_FIFO_ADDR_WIDTH+1-1:0] mask_fifo_wr_ptr_reg = 0;
323 reg [MASK_FIFO_ADDR_WIDTH+1-1:0] mask_fifo_rd_ptr_reg = 0, mask_fifo_rd_ptr_next;
324 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab" *)
325 reg [RAM_SEG_COUNT-1:0]
mask_fifo_mask[(2**MASK_FIFO_ADDR_WIDTH)-1:0];
326 reg [RAM_SEG_COUNT-1:0] mask_fifo_wr_mask;
327
328 wire mask_fifo_empty = mask_fifo_wr_ptr_reg == mask_fifo_rd_ptr_reg;
329 wire mask_fifo_full = mask_fifo_wr_ptr_reg == (mask_fifo_rd_ptr_reg ^ (1 << MASK_FIFO_ADDR_WIDTH));
330
331 reg [OP_TAG_WIDTH+1-1:0] active_op_count_reg = 0;
332 reg [inc_active_op;
333 reg [dec_active_op;
334
335 reg [AXI_ID_WIDTH-1:0] m_axi_awid_reg = {AXI_ID_WIDTH{1'b0}}, m_axi_awid_next;
336 reg [AXI_ADDR_WIDTH-1:0] m_axi_awaddr_reg = {AXI_ADDR_WIDTH{1'b0}}, m_axi_awaddr_next;
337 reg [7:0] m_axi_awlen_reg = 8'd0, m_axi_awlen_next;
338 reg [m_axi_awvalid_reg = 1'b0, m_axi_awvalid_next;
339 reg [m_axi_bready_reg = 1'b0, m_axi_bready_next;
340
341 reg [s_axis_write_desc_ready_reg = 1'b0, s_axis_write_desc_ready_next;
342
343 reg [TAG_WIDTH-1:0] m_axis_write_desc_status_tag_reg = {TAG_WIDTH{1'b0}}, m_axis_write_desc_status_tag_next;
344 reg [3:0] m_axis_write_desc_status_error_reg = 4'd0, m_axis_write_desc_status_error_next;
345 reg [m_axis_write_desc_status_valid_reg = 1'b0, m_axis_write_desc_status_valid_next;
346
347 reg [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0] ram_rd_cmd_sel_reg = 0, ram_rd_cmd_sel_next;
348 reg [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0] ram_rd_cmd_addr_reg = 0, ram_rd_cmd_addr_next;
349 reg [RAM_SEG_COUNT-1:0] ram_rd_cmd_valid_reg = 0, ram_rd_cmd_valid_next;
350 reg [RAM_SEG_COUNT-1:0] ram_rd_resp_ready_cmb;
351
352 reg [status_busy_reg = 1'b0;
353
354 reg [OP_TAG_WIDTH-1:0] stat_wr_op_start_tag_reg = 0, stat_wr_op_start_tag_next;

```

```

355 reg [LEN_WIDTH-1:0] stat_wr_op_start_len_reg = 0,
stat_wr_op_start_len_next;
356 reg stat_wr_op_start_valid_reg = 1'b0,
stat_wr_op_start_valid_next;
357 reg [OP_TAG_WIDTH-1:0] stat_wr_op_finish_tag_reg = 0,
stat_wr_op_finish_tag_next;
358 reg [3:0] stat_wr_op_finish_status_reg = 0, stat_wr_op_fin-
ish_status_next;
359 reg stat_wr_op_finish_valid_reg = 1'b0, stat_wr_op_fin-
ish_valid_next;
360 reg [OP_TAG_WIDTH-1:0] stat_wr_req_start_tag_reg = 0,
stat_wr_req_start_tag_next;
361 reg [12:0] stat_wr_req_start_len_reg = 13'd0,
stat_wr_req_start_len_next;
362 reg stat_wr_req_start_valid_reg = 1'b0,
stat_wr_req_start_valid_next;
363 reg [OP_TAG_WIDTH-1:0] stat_wr_req_finish_tag_reg = 0,
stat_wr_req_finish_tag_next;
364 reg [3:0] stat_wr_req_finish_status_reg = 0, stat_wr_req_fin-
ish_status_next;
365 reg stat_wr_req_finish_valid_reg = 1'b0, stat_wr_req_fin-
ish_valid_next;
366 reg stat_wr_op_table_full_reg = 1'b0, stat_wr_op_t-
able_full_next;
367 reg stat_wr_tx_stall_reg = 1'b0, stat_wr_tx_stall_next;
368
369 // internal datapath
370 reg [AXI_DATA_WIDTH-1:0] m_axi_wdata_int;
371 reg [AXI_STRB_WIDTH-1:0] m_axi_wstrb_int;
372 reg m_axi_wlast_int;
373 reg m_axi_wvalid_int;
374 wire m_axi_wready_int;
375
376 assign m_axi_awid = USE_AXI_ID ? m_axi_awid_reg :
{AXI_ID_WIDTH{1'b0}};
377 assign m_axi_awaddr = m_axi_awaddr_reg;
378 assign m_axi_awlen = m_axi_awlen_reg;
379 assign m_axi_awsz = AXI_BURST_SIZE;
380 assign m_axi_awburst = 2'b01;
381 assign m_axi_awlock = 1'b0;
382 assign m_axi_awcache = 4'b0011;
383 assign m_axi_awprot = 3'b010;
384 assign m_axi_awvalid = m_axi_awvalid_reg;
385 assign m_axi_bready = m_axi_bready_reg;
386
387 assign s_axis_write_desc_ready = s_axis_write_desc_ready_reg;
388
389 assign m_axis_write_desc_status_tag = m_ax-
is_write_desc_status_tag_reg;
390 assign m_axis_write_desc_status_error = m_ax-
is_write_desc_status_error_reg;
391 assign m_axis_write_desc_status_valid = m_ax-
is_write_desc_status_valid_reg;
392
393 assign ram_rd_cmd_sel = ram_rd_cmd_sel_reg;
394 assign ram_rd_cmd_addr = ram_rd_cmd_addr_reg;
395 assign ram_rd_cmd_valid = ram_rd_cmd_valid_reg;
396 assign ram_rd_resp_ready = ram_rd_resp_ready_cmb;
397
398 assign status_busy = status_busy_reg;
399
400 assign stat_wr_op_start_tag = stat_wr_op_start_tag_reg;
401 assign stat_wr_op_start_len = stat_wr_op_start_len_reg;
402 assign stat_wr_op_start_valid = stat_wr_op_start_valid_reg;
403 assign stat_wr_op_finish_tag = stat_wr_op_finish_tag_reg;
404 assign stat_wr_op_finish_status = stat_wr_op_fin-
ish_status_reg;
405 assign stat_wr_op_finish_valid = stat_wr_op_finish_valid_reg;
406 assign stat_wr_req_start_tag = stat_wr_req_start_tag_reg;
407 assign stat_wr_req_start_len = stat_wr_req_start_len_reg;
408 assign stat_wr_req_start_valid = stat_wr_req_start_valid_reg;
409 assign stat_wr_req_finish_tag = stat_wr_req_finish_tag_reg;
410 assign stat_wr_req_finish_status = stat_wr_req_fin-
ish_status_reg;
411 assign stat_wr_req_finish_valid = stat_wr_req_fin-
ish_valid_reg;
412 assign stat_wr_op_table_full = stat_wr_op_table_full_reg;
413 assign stat_wr_tx_stall = stat_wr_tx_stall_reg;
414
415 // operation tag management
416 reg [OP_TAG_WIDTH+1-1:0] op_table_start_ptr_reg = 0;
417 reg [AXI_ADDR_WIDTH-1:0] op_table_start_axi_addr;
418 reg [IMM_WIDTH-1:0] op_table_start_imm;
419 reg op_table_start_imm_en;
420 reg [11:0] op_table_start_len;
421 reg op_table_start_zero_len;
422 reg [CYCLE_COUNT_WIDTH-1:0] op_table_start_cycle_count;
423 reg [RAM_OFFSET_WIDTH-1:0] op_table_start_offset;
424 reg [TAG_WIDTH-1:0] op_table_start_tag;
425 reg op_table_start_last;
426 reg op_table_start_en;
427 reg [OP_TAG_WIDTH+1-1:0] op_table_tx_start_ptr_reg = 0;
428 reg op_table_tx_start_en;
429 reg [OP_TAG_WIDTH+1-1:0] op_table_tx_finish_ptr_reg = 0;
430 reg op_table_tx_finish_en;
431 reg [OP_TAG_WIDTH-1:0] op_table_write_complete_ptr;
432 reg [3:0] op_table_write_complete_error;
433 reg op_table_write_complete_en;
434 reg [OP_TAG_WIDTH+1-1:0] op_table_finish_ptr_reg = 0;
435 reg op_table_finish_en;
436
437 reg [2**OP_TAG_WIDTH-1:0] op_table_active = 0;
438 reg [2**OP_TAG_WIDTH-1:0] op_table_write_complete = 0;
439 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
440 reg [AXI_ADDR_WIDTH-1:0] op_t-
able_axi_addr[2**OP_TAG_WIDTH-1:0];
441 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
442 reg [IMM_WIDTH-1:0] op_table_imm[2**OP_TAG_WIDTH-1:0];
443 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
444 reg op_table_imm_en[2**OP_TAG_WIDTH-1:0];
445 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
446 reg [11:0] op_table_len[2**OP_TAG_WIDTH-1:0];
447 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
448 reg op_table_zero_len[2**OP_TAG_WIDTH-1:0];
449 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
450 reg [CYCLE_COUNT_WIDTH-1:0] op_t-
able_cycle_count[2**OP_TAG_WIDTH-1:0];
451 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
452 reg [RAM_OFFSET_WIDTH-1:0] op_t-
able_offset[2**OP_TAG_WIDTH-1:0];
453 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
454 reg [TAG_WIDTH-1:0] op_table_tag[2**OP_TAG_WIDTH-1:0];
455 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
456 reg op_table_last[2**OP_TAG_WIDTH-1:0];
457 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
458 reg [3:0] op_table_error_code [2**OP_TAG_WIDTH-1:0];
459
460 integer i;
461
462 initial begin
463   for (i = 0; i < 2**OP_TAG_WIDTH; i = i + 1) begin
464     op_table_axi_addr[i] = 0;
465     op_table_imm[i] = 0;
466     op_table_imm_en[i] = 0;
467     op_table_len[i] = 0;
468     op_table_zero_len[i] = 1'b0;
469     op_table_cycle_count[i] = 0;
470     op_table_offset[i] = 0;
471     op_table_tag[i] = 0;
472     op_table_last[i] = 0;
473     op_table_error_code[i] = 0;
474   end
475 end
476
477 always @* begin
478   req_state_next = REQ_STATE_IDLE;
479
480   s_axis_write_desc_ready_next = 1'b0;
481
482   stat_wr_op_start_tag_next = stat_wr_op_start_tag_reg;
483   stat_wr_op_start_len_next = stat_wr_op_start_len_reg;
484   stat_wr_op_start_valid_next = 1'b0;
485   stat_wr_req_start_tag_next = stat_wr_req_start_tag_reg;
486   stat_wr_req_start_len_next = stat_wr_req_start_len_reg;
487   stat_wr_req_start_valid_next = 1'b0;
488   stat_wr_op_table_full_next = !(op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2**OP_TAG_WIDTH));
489   stat_wr_tx_stall_next = (m_axi_awvalid && !m_axi_wready)
|| (m_axi_wvalid && !m_axi_wready);
490
491   tag_next = tag_reg;
492   req_axi_addr_next = req_axi_addr_reg;
493   ram_sel_next = ram_sel_reg;
494   ram_addr_next = ram_addr_reg;
495   imm_next = imm_reg;
496   imm_en_next = imm_en_reg;
497   op_count_next = op_count_reg;
498   zero_len_next = zero_len_reg;
499   tr_count_next = tr_count_reg;
500   tr_word_count_next = tr_word_count_reg;
501
502   read_cmd_axi_addr_next = read_cmd_axi_addr_reg;
503   read_cmd_ram_sel_next = read_cmd_ram_sel_reg;
504   read_cmd_ram_addr_next = read_cmd_ram_addr_reg;
505   read_cmd_imm_en_next = read_cmd_imm_en_reg;
506   read_cmd_len_next = read_cmd_len_reg;
507   read_cmd_cycle_count_next = read_cmd_cycle_count_reg;
508   read_cmd_last_cycle_next = read_cmd_last_cycle_reg;
509   read_cmd_valid_next = read_cmd_valid_reg && !
read_cmd_ready;
510
511   op_table_start_axi_addr = req_axi_addr_reg;
512   op_table_start_imm = imm_reg;
513   op_table_start_imm_en = imm_en_reg;
514   op_table_start_len = 0;
515   op_table_start_zero_len = zero_len_reg;
516   op_table_start_cycle_count = 0;
517   op_table_start_offset = (req_axi_addr_reg & OFFSET_MASK)-
ram_addr_reg[RAM_OFFSET_WIDTH-1:0];
518   op_table_start_tag = tag_reg;

```

```

519 op_table_start_last = 0;
520 op_table_start_en = 1'b0;
521
522 inc_active_op = 1'b0;
523
524 // TLP segmentation
525 case (req_state_reg)
526     REQ_STATE_IDLE: begin
527         // idle state, wait for incoming descriptor
528         s_axis_write_desc_ready_next = !op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2*OP_TAG_WIDTH) && enable;
529
530         req_axi_addr_next = s_axis_write_desc_axi_addr;
531         if (IMM_ENABLE && s_axis_write_desc_imm_en) begin
532             ram_sel_next = 0;
533             ram_addr_next = 0;
534         end else begin
535             ram_sel_next = s_axis_write_desc_ram_sel;
536             ram_addr_next = s_axis_write_desc_ram_addr;
537         end
538         imm_next = s_axis_write_desc_imm;
539         imm_en_next = IMM_ENABLE && s_ax-
is_write_desc_imm_en;
540         if (s_axis_write_desc_len == 0) begin
541             // zero-length operation
542             op_count_next = 1;
543             zero_len_next = 1'b1;
544         end else begin
545             op_count_next = s_axis_write_desc_len;
546             zero_len_next = 1'b0;
547         end
548         tag_next = s_axis_write_desc_tag;
549
550         if (op_count_next <= AXI_MAX_BURST_SIZE -
(req_axi_addr_next & OFFSET_MASK) || AXI_MAX_BURST_SIZE >= 4096) be-
gin
551             // packet smaller than max burst size
552             if (((req_axi_addr_next & 12'hfff) +
(op_count_next & 12'hfff)) >> 12 != 0 || op_count_next >> 12 != 0)
begin
553                 // crosses 4k boundary
554                 tr_word_count_next = 13'h1000 -
req_axi_addr_next[11:0];
555             end else begin
556                 // does not cross 4k boundary
557                 tr_word_count_next = op_count_next;
558             end
559         end else begin
560             // packet larger than max burst size
561             if (((req_axi_addr_next & 12'hfff) +
AXI_MAX_BURST_SIZE) >> 12 != 0) begin
562                 // crosses 4k boundary
563                 tr_word_count_next = 13'h1000 -
req_axi_addr_next[11:0];
564             end else begin
565                 // does not cross 4k boundary
566                 tr_word_count_next = AXI_MAX_BURST_SIZE -
(req_axi_addr_next & OFFSET_MASK);
567             end
568         end
569
570         if (s_axis_write_desc_ready & s_ax-
is_write_desc_valid) begin
571             s_axis_write_desc_ready_next = 1'b0;
572
573             stat_wr_op_start_tag_next =
stat_wr_op_start_tag_reg+1;
574             stat_wr_op_start_len_next = s_ax-
is_write_desc_len;
575             stat_wr_op_start_valid_next = 1'b1;
576
577             req_state_next = REQ_STATE_START;
578         end else begin
579             req_state_next = REQ_STATE_IDLE;
580         end
581     end
582     REQ_STATE_START: begin
583         // start state, compute length
584         if (!op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2*OP_TAG_WIDTH) && (!ram_rd_cmd_valid_reg || ram_rd_cmd_ready) &&
(!read_cmd_valid_reg || read_cmd_ready)) begin
585             read_cmd_axi_addr_next = req_axi_addr_reg;
586             read_cmd_ram_sel_next = ram_sel_reg;
587             read_cmd_ram_addr_next = ram_addr_reg;
588             read_cmd_imm_en_next = imm_en_reg;
589             read_cmd_len_next = tr_word_count_next;
590             read_cmd_cycle_count_next =
(tr_word_count_next + (req_axi_addr_reg & OFFSET_MASK) - 1) >>
AXI_BURST_SIZE;
591             op_table_start_cycle_count =
read_cmd_cycle_count_next;
592             read_cmd_last_cycle_next =
read_cmd_cycle_count_next == 0;
593             read_cmd_valid_next = 1'b1;
594
595             req_axi_addr_next = req_axi_addr_reg +
tr_word_count_next;

```

```

596             ram_addr_next = ram_addr_reg +
tr_word_count_next;
597             op_count_next = op_count_reg -
tr_word_count_next;
598
599             op_table_start_axi_addr = req_axi_addr_reg;
600             op_table_start_imm = imm_reg;
601             op_table_start_imm_en = imm_en_reg;
602             op_table_start_len = tr_word_count_next;
603             op_table_start_zero_len = zero_len_reg;
604             op_table_start_offset = (req_axi_addr_reg &
OFFSET_MASK) - ram_addr_reg[RAM_OFFSET_WIDTH-1:0];
605             op_table_start_tag = tag_reg;
606             op_table_start_last = op_count_reg ==
tr_word_count_next;
607             op_table_start_en = 1'b1;
608             inc_active_op = 1'b1;
609
610             stat_wr_req_start_tag_next = op_t-
able_start_ptr_reg[OP_TAG_WIDTH-1:0];
611             stat_wr_req_start_len_next = zero_len_reg ?
0 : tr_word_count_next;
612             stat_wr_req_start_valid_next = 1'b1;
613
614             if (op_count_next <= AXI_MAX_BURST_SIZE -
(req_axi_addr_next & OFFSET_MASK) || AXI_MAX_BURST_SIZE >= 4096) be-
gin
615                 // packet smaller than max burst size
616                 if (((req_axi_addr_next & 12'hfff) +
(op_count_next & 12'hfff)) >> 12 != 0 || op_count_next >> 12 != 0)
begin
617                     // crosses 4k boundary
618                     tr_word_count_next = 13'h1000 -
req_axi_addr_next[11:0];
619                 end else begin
620                     // does not cross 4k boundary
621                     tr_word_count_next = op_count_next;
622                 end
623             end else begin
624                 // packet larger than max burst size
625                 if (((req_axi_addr_next & 12'hfff) +
AXI_MAX_BURST_SIZE) >> 12 != 0) begin
626                     // crosses 4k boundary
627                     tr_word_count_next = 13'h1000 -
req_axi_addr_next[11:0];
628                 end else begin
629                     // does not cross 4k boundary
630                     tr_word_count_next =
AXI_MAX_BURST_SIZE - (req_axi_addr_next & OFFSET_MASK);
631                 end
632             end
633
634             if (!op_table_start_last) begin
635                 req_state_next = REQ_STATE_START;
636             end else begin
637                 s_axis_write_desc_ready_next = !op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2*OP_TAG_WIDTH) && enable;
638                 req_state_next = REQ_STATE_IDLE;
639             end
640         end else begin
641             req_state_next = REQ_STATE_START;
642         end
643     end
644 endcase
645 end
646
647 always @* begin
648     read_state_next = READ_STATE_IDLE;
649
650     read_cmd_ready = 1'b0;
651
652     ram_rd_cmd_sel_next = ram_rd_cmd_sel_reg;
653     ram_rd_cmd_addr_next = ram_rd_cmd_addr_reg;
654     ram_rd_cmd_valid_next = ram_rd_cmd_valid_reg &
~ram_rd_cmd_ready;
655
656     read_axi_addr_next = read_axi_addr_reg;
657     read_ram_sel_next = read_ram_sel_reg;
658     read_ram_addr_next = read_ram_addr_reg;
659     read_imm_en_next = read_imm_en_reg;
660     read_len_next = read_len_reg;
661     read_ram_mask_next = read_ram_mask_reg;
662     read_ram_mask_0_next = read_ram_mask_0_reg;
663     read_ram_mask_1_next = read_ram_mask_1_reg;
664     ram_wrap_next = ram_wrap_reg;
665     read_cycle_count_next = read_cycle_count_reg;
666     read_last_cycle_next = read_last_cycle_reg;
667     cycle_byte_count_next = cycle_byte_count_reg;
668     start_offset_next = start_offset_reg;
669     end_offset_next = end_offset_reg;
670
671     mask_fifo_wr_mask = read_ram_mask_reg;
672     mask_fifo_we = 1'b0;
673
674     // Read request generation
675     case (read_state_reg)
676         READ_STATE_IDLE: begin
677             // idle state, wait for read command
678

```

```

679      read_axi_addr_next = read_cmd_axi_addr_reg;
680      read_ram_sel_next = read_cmd_ram_sel_reg;
681      read_ram_addr_next = read_cmd_ram_addr_reg;
682      read_imm_en_next = read_cmd_imm_en_reg;
683      read_len_next = read_cmd_len_reg;
684      read_cycle_count_next = read_cmd_cycle_count_reg;
685      read_last_cycle_next = read_cmd_last_cycle_reg;
686
687      if (read_len_next > AXI_STRB_WIDTH-(read_axi_ad-
dr_next & OFFSET_MASK)) begin
688          cycle_byte_count_next = AXI_STRB_WIDTH-
(read_axi_addr_next & OFFSET_MASK);
689      end else begin
690          cycle_byte_count_next = read_len_next;
691      end
692      start_offset_next = read_ram_addr_next;
693      {ram_wrap_next, end_offset_next} =
start_offset_next+cycle_byte_count_next-1;
694
695      read_ram_mask_0_next = {RAM_SEG_COUNT{1'b1}} <<
(start_offset_next >> $clog2(RAM_SEG_BE_WIDTH));
696      read_ram_mask_1_next = {RAM_SEG_COUNT{1'b1}} >>
(RAM_SEG_COUNT-1-(end_offset_next >> $clog2(RAM_SEG_BE_WIDTH)));
697
698      if (!ram_wrap_next) begin
699          read_ram_mask_next = read_ram_mask_0_next &
read_ram_mask_1_next;
700      end else begin
701          read_ram_mask_1_next = 0;
702      end
703      read_ram_mask_next = read_ram_mask_0_next |
read_ram_mask_1_next;
704      end
705
706      if (read_cmd_valid_reg) begin
707          read_cmd_ready = 1'b1;
708          read_state_next = READ_STATE_READ;
709      end else begin
710          read_state_next = READ_STATE_IDLE;
711      end
712      end
713      READ_STATE_READ: begin
714          // read state - start new read operations
715
716          if (!ram_rd_cmd_valid & ~ram_rd_cmd_ready &
read_ram_mask_reg) && !mask_fifo_full) begin
717
718              // update counters
719              read_ram_addr_next = read_ram_addr_reg +
cycle_byte_count_reg;
720              read_len_next = read_len_reg -
cycle_byte_count_reg;
721
722              read_cycle_count_next = read_cycle_count_reg -
1;
723              read_last_cycle_next = read_cycle_count_next
== 0;
724
725              for (i = 0; i < RAM_SEG_COUNT; i = i + 1) be-
gin
726                  if (read_ram_mask_reg[i]) begin
727                      ram_rd_cmd_sel_next[i*RAM_SEL_WIDTH +:
RAM_SEL_WIDTH] = read_ram_sel_reg;
728                      ram_rd_cmd_ad-
dr_next[i*RAM_SEG_ADDR_WIDTH +: RAM_SEG_ADDR_WIDTH] = read_ram_ad-
dr_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-RAM_SEG_ADDR_WIDTH];
729                      ram_rd_cmd_valid_next[i] = !(IMM_EN-
ABLE && read_imm_en_reg);
730                  end
731                  if (read_ram_mask_1_reg[i]) begin
732                      ram_rd_cmd_ad-
dr_next[i*RAM_SEG_ADDR_WIDTH +: RAM_SEG_ADDR_WIDTH] = read_ram_ad-
dr_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-RAM_SEG_ADDR_WIDTH]+1;
733                  end
734              end
735              mask_fifo_wr_mask = (IMM_ENABLE &&
read_imm_en_reg) ? 0 : read_ram_mask_reg;
736              mask_fifo_we = 1'b1;
737
738              if (read_len_next > AXI_STRB_WIDTH) begin
739                  cycle_byte_count_next = AXI_STRB_WIDTH;
740              end else begin
741                  cycle_byte_count_next = read_len_next;
742              end
743              start_offset_next = read_ram_addr_next;
744              {ram_wrap_next, end_offset_next} =
start_offset_next+cycle_byte_count_next-1;
745
746              read_ram_mask_0_next = {RAM_SEG_COUNT{1'b1}}
<< (start_offset_next >> $clog2(RAM_SEG_BE_WIDTH));
747              read_ram_mask_1_next = {RAM_SEG_COUNT{1'b1}}
>> (RAM_SEG_COUNT-1-(end_offset_next >> $clog2(RAM_SEG_BE_WIDTH)));
748
749              if (!ram_wrap_next) begin
750                  read_ram_mask_next = read_ram_mask_0_next
& read_ram_mask_1_next;
751              end else begin
752                  read_ram_mask_1_next = 0;
753              end
754              read_ram_mask_next = read_ram_mask_0_next |
read_ram_mask_1_next;

```

```

752      read_ram_mask_1_next = 0;
753      end else begin
754          read_ram_mask_next = read_ram_mask_0_next
| read_ram_mask_1_next;
755      end
756
757      if (!read_last_cycle_reg) begin
758          read_state_next = READ_STATE_READ;
759      end else if (read_cmd_valid_reg) begin
760
761          read_axi_addr_next = read_cmd_axi_ad-
dr_reg;
762          read_ram_sel_next = read_cmd_ram_sel_reg;
763          read_ram_addr_next = read_cmd_ram_ad-
dr_reg;
764          read_imm_en_next = read_cmd_imm_en_reg;
765          read_len_next = read_cmd_len_reg;
766          read_cycle_count_next =
read_cmd_cycle_count_reg;
767          read_last_cycle_next =
read_cmd_last_cycle_reg;
768
769          if (read_len_next > AXI_STRB_WIDTH-
(read_axi_addr_next & OFFSET_MASK)) begin
770              cycle_byte_count_next =
AXI_STRB_WIDTH-(read_axi_addr_next & OFFSET_MASK);
771          end else begin
772              cycle_byte_count_next = read_len_next;
773          end
774          start_offset_next = read_ram_addr_next;
775          {ram_wrap_next, end_offset_next} =
start_offset_next+cycle_byte_count_next-1;
776
777          read_ram_mask_0_next =
{RAM_SEG_COUNT{1'b1}} << (start_offset_next >> $clo-
g2(RAM_SEG_BE_WIDTH));
778          read_ram_mask_1_next =
{RAM_SEG_COUNT{1'b1}} >> (RAM_SEG_COUNT-1-(end_offset_next >> $clo-
g2(RAM_SEG_BE_WIDTH)));
779
780          if (!ram_wrap_next) begin
781              read_ram_mask_next =
read_ram_mask_0_next & read_ram_mask_1_next;
782          end else begin
783              read_ram_mask_1_next = 0;
784          end
785          read_ram_mask_next =
read_ram_mask_0_next | read_ram_mask_1_next;
786      end
787
788      read_cmd_ready = 1'b1;
789
790      read_state_next = READ_STATE_READ;
791      end else begin
792          read_state_next = READ_STATE_IDLE;
793      end
794      end else begin
795          read_state_next = READ_STATE_READ;
796      end
797      end
798      endcase
799  end
800
801  always @* begin
802      axi_state_next = AXI_STATE_IDLE;
803
804      ram_rd_resp_ready_cmb = {RAM_SEG_COUNT{1'b0}};
805
806      stat_wr_op_finish_tag_next = stat_wr_op_finish_tag_reg;
807      stat_wr_op_finish_status_next = stat_wr_op_fin-
ish_status_reg;
808      stat_wr_op_finish_valid_next = 1'b0;
809      stat_wr_req_finish_tag_next = stat_wr_req_finish_tag_reg;
810      stat_wr_req_finish_status_next = stat_wr_req_fin-
ish_status_reg;
811      stat_wr_req_finish_valid_next = 1'b0;
812
813      axi_addr_next = axi_addr_reg;
814      axi_imm_next = axi_imm_reg;
815      axi_imm_en_next = axi_imm_en_reg;
816      axi_len_next = axi_len_reg;
817      axi_zero_len_next = axi_zero_len_reg;
818      offset_next = offset_reg;
819      strb_offset_mask_next = strb_offset_mask_reg;
820      last_cycle_offset_next = last_cycle_offset_reg;
821      ram_mask_next = ram_mask_reg;
822      ram_mask_valid_next = ram_mask_valid_reg;
823      cycle_count_next = cycle_count_reg;
824      last_cycle_next = last_cycle_reg;
825
826      mask_fifo_rd_ptr_next = mask_fifo_rd_ptr_reg;
827
828      op_table_tx_start_en = 1'b0;
829      op_table_tx_finish_en = 1'b0;
830
831      m_axi_awid_next = m_axi_awid_reg;
832      m_axi_awaddr_next = m_axi_awaddr_reg;
833      m_axi_awlen_next = m_axi_awlen_reg;
834      m_axi_awvalid_next = m_axi_awvalid_reg && !m_axi_awready;
835      m_axi_bready_next = 1'b0;

```



```

836
837   m_axi_wdata_int = ((IMM_ENABLE && axi_imm_en_reg) ?
{2{RAM_DATA_WIDTH{1'b0}} | axi_imm_reg}} : {2{ram_rd_resp_data}})
>> (RAM_DATA_WIDTH-offset_reg*AXI_WORD_SIZE);
838   m_axi_wstrb_int = strb_offset_mask_reg;
839   m_axi_wlast_int = 1'b0;
840   m_axi_wvalid_int = 1'b0;
841
842   // read response processing and AXI write generation
843   case (axi_state_reg)
844     AXI_STATE_IDLE: begin
845       // idle state, wait for command
846       ram_rd_resp_ready_cmb = {RAM_SEG_COUNT{1'b0}};
847
848       axi_addr_next = op_t-
able_axi_addr[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
849       axi_imm_next = op_t-
able_imm[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
850       axi_imm_en_next = op_t-
able_imm_en[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
851       axi_len_next = op_t-
able_len[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
852       axi_zero_len_next = op_t-
able_zero_len[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
853       offset_next = op_t-
able_offset[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
854       strb_offset_mask_next = axi_zero_len_next ?
{AXI_STRB_WIDTH{1'b0}} : ({AXI_STRB_WIDTH{1'b1}} << (axi_addr_next &
OFFSET_MASK));
855       last_cycle_offset_next = axi_addr_next +
(axi_len_next & OFFSET_MASK);
856       cycle_count_next = op_t-
able_cycle_count[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
857       last_cycle_next = op_t-
able_cycle_count[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]] == 0;
858
859       if (op_t-
able_active[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]] && op_t-
able_tx_start_ptr_reg != op_table_start_ptr_reg && (!m_axi_awval-
id_reg || m_axi_awready)) begin
860         m_axi_awid_next = op_t-
able_tx_start_ptr_reg[OP_TAG_WIDTH-1:0];
861         m_axi_awaddr_next = axi_addr_next;
862         m_axi_awlen_next = cycle_count_next;
863         m_axi_awvalid_next = 1'b1;
864         op_table_tx_start_en = 1'b1;
865         axi_state_next = AXI_STATE_TRANSFER;
866       end else begin
867         axi_state_next = AXI_STATE_IDLE;
868       end
869     end
870     AXI_STATE_TRANSFER: begin
871       // transfer state, transfer data
872       ram_rd_resp_ready_cmb = {RAM_SEG_COUNT{1'b0}};
873
874       if (!(ram_mask_reg & ~ram_rd_resp_valid) &&
ram_mask_valid_reg && m_axi_wready_int) begin
875         // transfer in read data
876         ram_rd_resp_ready_cmb = ram_mask_reg;
877         ram_mask_valid_next = 1'b0;
878
879         // update counters
880         cycle_count_next = cycle_count_reg - 1;
881         last_cycle_next = cycle_count_next == 0;
882         offset_next = offset_reg + AXI_STRB_WIDTH;
883         strb_offset_mask_next =
{AXI_STRB_WIDTH{1'b1}};
884
885         m_axi_wdata_int = ((IMM_ENABLE &&
axi_imm_en_reg) ? {2{RAM_DATA_WIDTH{1'b0}} | axi_imm_reg}} :
{2{ram_rd_resp_data}}) >> (RAM_DATA_WIDTH-offset_reg*AXI_WORD_SIZE);
886         m_axi_wstrb_int = strb_offset_mask_reg;
887         m_axi_wlast_int = 1'b0;
888         m_axi_wvalid_int = 1'b1;
889
890         if (last_cycle_reg) begin
891           // no more data to transfer, finish opera-
tion
892             m_axi_wlast_int = 1'b1;
893             op_table_tx_finish_en = 1'b1;
894
895             if (last_cycle_offset_reg) begin
896               m_axi_wstrb_int = strb_offset_mask_reg &
{AXI_STRB_WIDTH{1'b1}} >> (AXI_STRB_WIDTH - last_cycle_offset_reg);
897             end
898
899             // skip idle state if possible
900             axi_addr_next = op_t-
able_axi_addr[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
901             axi_imm_next = op_t-
able_imm[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
902             axi_imm_en_next = op_t-
able_imm_en[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
903             axi_len_next = op_t-
able_len[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
904             axi_zero_len_next = op_t-
able_zero_len[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
905             offset_next = op_t-
able_offset[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
906             strb_offset_mask_next =

```

```

axi_zero_len_next ? {AXI_STRB_WIDTH{1'b0}} : ({AXI_STRB_WIDTH{1'b1}}
<< (axi_addr_next & OFFSET_MASK));
907       last_cycle_offset_next = axi_addr_next +
(axi_len_next & OFFSET_MASK);
908       cycle_count_next = op_t-
able_cycle_count[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
909       last_cycle_next = op_t-
able_cycle_count[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]] == 0;
910
911       if (op_t-
able_active[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]] && op_t-
able_tx_start_ptr_reg != op_table_start_ptr_reg && (!m_axi_awval-
id_reg || m_axi_awready)) begin
912         m_axi_awid_next = op_t-
able_tx_start_ptr_reg[OP_TAG_WIDTH-1:0];
913         m_axi_awaddr_next = axi_addr_next;
914         m_axi_awlen_next = cycle_count_next;
915         m_axi_awvalid_next = 1'b1;
916         op_table_tx_start_en = 1'b1;
917         axi_state_next = AXI_STATE_TRANSFER;
918       end else begin
919         axi_state_next = AXI_STATE_IDLE;
920       end
921     end else begin
922       axi_state_next = AXI_STATE_TRANSFER;
923     end
924   end else begin
925     axi_state_next = AXI_STATE_TRANSFER;
926   end
927 end
928 endcase
929
930 if (!ram_mask_valid_next && !mask_fifo_empty) begin
931   ram_mask_next =
mask_fifo_mask[mask_fifo_rd_ptr_reg[MASK_FIFO_ADDR_WIDTH-1:0]];
932   ram_mask_valid_next = 1'b1;
933   mask_fifo_rd_ptr_next = mask_fifo_rd_ptr_reg+1;
934 end
935
936 op_table_write_complete_ptr = m_axi_bid;
937 if (m_axi_bresp == AXI_RESP_SLVERR) begin
938   op_table_write_complete_error = DMA_ER-
ROR_AXI_WR_SLVERR;
939 end else if (m_axi_bresp == AXI_RESP_DECERR) begin
940   op_table_write_complete_error = DMA_ER-
ROR_AXI_WR_DECERR;
941 end else begin
942   op_table_write_complete_error = DMA_ERROR_NONE;
943 end
944 op_table_write_complete_en = 1'b0;
945
946 m_axis_write_desc_status_tag_next = op_t-
able_tag[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
947 if (m_axis_write_desc_status_valid_reg) begin
948   m_axis_write_desc_status_error_next = DMA_ERROR_NONE;
949 end else begin
950   m_axis_write_desc_status_error_next = m_ax-
is_write_desc_status_error_reg;
951 end
952 m_axis_write_desc_status_valid_next = 1'b0;
953
954 stat_wr_req_finish_status_next = op_t-
able_write_complete_error;
955 stat_wr_req_finish_valid_next = 1'b0;
956
957 stat_wr_op_finish_tag_next = stat_wr_op_finish_tag_reg;
958 stat_wr_op_finish_status_next = m_ax-
is_write_desc_status_error_next;
959 stat_wr_op_finish_valid_next = 1'b0;
960
961 if (USE_AXI_ID) begin
962   // accept write completions
963   stat_wr_req_finish_tag_next = m_axi_bid;
964
965   m_axi_bready_next = 1'b1;
966   if (m_axi_bready && m_axi_bvalid) begin
967     op_table_write_complete_ptr = m_axi_bid;
968     op_table_write_complete_en = 1'b1;
969     stat_wr_req_finish_valid_next = 1'b1;
970   end
971
972   // commit operations in-order
973   op_table_finish_en = 1'b0;
974   dec_active_op = 1'b0;
975
976   if (op_t-
able_active[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] && op_t-
able_write_complete[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] &&
op_table_finish_ptr_reg != op_table_tx_finish_ptr_reg) begin
977     op_table_finish_en = 1'b1;
978     dec_active_op = 1'b1;
979
980     if (op_t-
able_error_code[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] !=
DMA_ERROR_NONE) begin
981       m_axis_write_desc_status_error_next = op_t-
able_error_code[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
982     end
983
984     stat_wr_op_finish_status_next = m_ax-
is_write_desc_status_error_next;

```

```

985
986     if (op_t-
able_last[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]]) begin
987         m_axis_write_desc_status_tag_next = op_t-
able_tag[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
988         m_axis_write_desc_status_valid_next = 1'b1;
989         stat_wr_op_finish_tag_next = stat_wr_op_fin-
ish_tag_reg + 1;
990         stat_wr_op_finish_valid_next = 1'b1;
991     end
992 end
993 end else begin
994     // accept write completions
995     op_table_finish_en = 1'b0;
996     dec_active_op = 1'b0;
997
998     stat_wr_req_finish_tag_next = op_t-
able_finish_ptr_reg[OP_TAG_WIDTH-1:0];
999
1000     m_axi_bready_next = 1'b1;
1001     if (m_axi_bready && m_axi_bvalid) begin
1002         op_table_finish_en = 1'b1;
1003         dec_active_op = 1'b1;
1004         stat_wr_req_finish_valid_next = 1'b1;
1005
1006         if (m_axi_bresp == AXI_RESP_SLVERR) begin
1007             m_axis_write_desc_status_error_next = DMA_ER-
ROR_AXI_WL_SLVERR;
1008         end else if (m_axi_bresp == AXI_RESP_DECERR) begin
1009             m_axis_write_desc_status_error_next = DMA_ER-
ROR_AXI_WL_DECERR;
1010         end
1011
1012         stat_wr_op_finish_status_next = m_ax-
is_write_desc_status_error_next;
1013
1014         if (op_t-
able_last[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]]) begin
1015             m_axis_write_desc_status_tag_next = op_t-
able_tag[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
1016             m_axis_write_desc_status_valid_next = 1'b1;
1017             stat_wr_op_finish_tag_next = stat_wr_op_fin-
ish_tag_reg + 1;
1018             stat_wr_op_finish_valid_next = 1'b1;
1019         end
1020     end
1021 end
1022 end
1023
1024 always @(posedge clk) begin
1025     req_state_reg <= req_state_next;
1026     read_state_reg <= read_state_next;
1027     axi_state_reg <= axi_state_next;
1028
1029     req_axi_addr_reg <= req_axi_addr_next;
1030     ram_sel_reg <= ram_sel_next;
1031     ram_addr_reg <= ram_addr_next;
1032     imm_reg <= imm_next;
1033     imm_en_reg <= imm_en_next;
1034     op_count_reg <= op_count_next;
1035     zero_len_reg <= zero_len_next;
1036     tr_count_reg <= tr_count_next;
1037     tr_word_count_reg <= tr_word_count_next;
1038     tag_reg <= tag_next;
1039
1040     read_axi_addr_reg <= read_axi_addr_next;
1041     read_ram_sel_reg <= read_ram_sel_next;
1042     read_ram_addr_reg <= read_ram_addr_next;
1043     read_imm_en_reg <= read_imm_en_next;
1044     read_len_reg <= read_len_next;
1045     read_ram_mask_reg <= read_ram_mask_next;
1046     read_ram_mask_0_reg <= read_ram_mask_0_next;
1047     read_ram_mask_1_reg <= read_ram_mask_1_next;
1048     ram_wrap_reg <= ram_wrap_next;
1049     read_cycle_count_reg <= read_cycle_count_next;
1050     read_last_cycle_reg <= read_last_cycle_next;
1051     cycle_byte_count_reg <= cycle_byte_count_next;
1052     start_offset_reg <= start_offset_next;
1053     end_offset_reg <= end_offset_next;
1054
1055     axi_addr_reg <= axi_addr_next;
1056     axi_imm_reg <= axi_imm_next;
1057     axi_imm_en_reg <= axi_imm_en_next;
1058     axi_len_reg <= axi_len_next;
1059     axi_zero_len_reg <= axi_zero_len_next;
1060     offset_reg <= offset_next;
1061     strb_offset_mask_reg <= strb_offset_mask_next;
1062     last_cycle_offset_reg <= last_cycle_offset_next;
1063     ram_mask_reg <= ram_mask_next;
1064     ram_mask_valid_reg <= ram_mask_valid_next;
1065     cycle_count_reg <= cycle_count_next;
1066     last_cycle_reg <= last_cycle_next;
1067
1068     read_cmd_axi_addr_reg <= read_cmd_axi_addr_next;
1069     read_cmd_ram_sel_reg <= read_cmd_ram_sel_next;
1070     read_cmd_ram_addr_reg <= read_cmd_ram_addr_next;
1071     read_cmd_imm_en_reg <= read_cmd_imm_en_next;
1072     read_cmd_len_reg <= read_cmd_len_next;
1073     read_cmd_cycle_count_reg <= read_cmd_cycle_count_next;
1074     read_cmd_last_cycle_reg <= read_cmd_last_cycle_next;
1075     read_cmd_valid_reg <= read_cmd_valid_next;

```

```

1076
1077     m_axi_awid_reg <= m_axi_awid_next;
1078     m_axi_awaddr_reg <= m_axi_awaddr_next;
1079     m_axi_awlen_reg <= m_axi_awlen_next;
1080     m_axi_awvalid_reg <= m_axi_awvalid_next;
1081     m_axi_bready_reg <= m_axi_bready_next;
1082
1083     s_axis_write_desc_ready_reg <= s_ax-
is_write_desc_ready_next;
1084
1085     m_axis_write_desc_status_tag_reg <= m_ax-
is_write_desc_status_tag_next;
1086     m_axis_write_desc_status_error_reg <= m_ax-
is_write_desc_status_error_next;
1087     m_axis_write_desc_status_valid_reg <= m_ax-
is_write_desc_status_valid_next;
1088
1089     status_busy_reg <= active_op_count_reg != 0;
1090
1091     stat_wr_op_start_tag_reg <= stat_wr_op_start_tag_next;
1092     stat_wr_op_start_len_reg <= stat_wr_op_start_len_next;
1093     stat_wr_op_start_valid_reg <= stat_wr_op_start_valid_next;
1094     stat_wr_op_finish_tag_reg <= stat_wr_op_finish_tag_next;
1095     stat_wr_op_finish_status_reg <= stat_wr_op_fin-
ish_status_next;
1096     stat_wr_op_finish_valid_reg <= stat_wr_op_fin-
ish_valid_next;
1097     stat_wr_req_start_tag_reg <= stat_wr_req_start_tag_next;
1098     stat_wr_req_start_len_reg <= stat_wr_req_start_len_next;
1099     stat_wr_req_start_valid_reg <=
stat_wr_req_start_valid_next;
1100     stat_wr_req_finish_tag_reg <= stat_wr_req_finish_tag_next;
1101     stat_wr_req_finish_status_reg <= stat_wr_req_fin-
ish_status_next;
1102     stat_wr_req_finish_valid_reg <= stat_wr_req_fin-
ish_valid_next;
1103     stat_wr_op_table_full_reg <= stat_wr_op_table_full_next;
1104     stat_wr_tx_stall_reg <= stat_wr_tx_stall_next;
1105
1106     ram_rd_cmd_sel_reg <= ram_rd_cmd_sel_next;
1107     ram_rd_cmd_addr_reg <= ram_rd_cmd_addr_next;
1108     ram_rd_cmd_valid_reg <= ram_rd_cmd_valid_next;
1109
1110     active_op_count_reg <= active_op_count_reg + inc_active_op
- dec_active_op;
1111
1112     if (mask_fifo_we) begin
1113         mask_fifo_mask[mask_fifo_wr_ptr_reg[MASK_FIFO_ADDR_WIDTH-1:0]] <=
mask_fifo_wr_mask;
1114         mask_fifo_wr_ptr_reg <= mask_fifo_wr_ptr_reg + 1;
1115     end
1116     mask_fifo_rd_ptr_reg <= mask_fifo_rd_ptr_next;
1117
1118     if (op_table_start_en) begin
1119         op_table_start_ptr_reg <= op_table_start_ptr_reg + 1;
1120         op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= 1'b1;
1121         op_t-
able_write_complete[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <=
1'b0;
1122         op_t-
able_axi_addr[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_axi_addr;
1123         op_table_imm[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]]
<= op_table_start_imm;
1124         op_t-
able_imm_en[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_imm_en;
1125         op_table_len[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]]
<= op_table_start_len;
1126         op_t-
able_zero_len[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_zero_len;
1127         op_t-
able_cycle_count[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_cycle_count;
1128         op_t-
able_offset[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_offset;
1129         op_table_tag[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]]
<= op_table_start_tag;
1130         op_t-
able_last[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_last;
1131     end
1132
1133     if (op_table_tx_start_en) begin
1134         op_table_tx_start_ptr_reg <= op_table_tx_start_ptr_reg
+ 1;
1135     end
1136
1137     if (op_table_tx_finish_en) begin
1138         op_table_tx_finish_ptr_reg <= op_t-
able_tx_finish_ptr_reg + 1;
1139     end

```

```

1140
1141 if (USE_AXI_ID && op_table_write_complete_en) begin
1142     op_table_write_complete[op_table_write_complete_ptr]
<= 1'b1;
1143     op_table_error_code[op_table_write_complete_ptr] <=
op_table_write_complete_error;
1144 end
1145
1146 if (op_table_finish_en) begin
1147     op_table_finish_ptr_reg <= op_table_finish_ptr_reg +
1;
1148     op_t-
able_active[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] <= 1'b0;
1149 end
1150
1151 if (rst) begin
1152     req_state_reg <= REQ_STATE_IDLE;
1153     read_state_reg <= READ_STATE_IDLE;
1154     axi_state_reg <= AXI_STATE_IDLE;
1155
1156     read_cmd_valid_reg <= 1'b0;
1157
1158     ram_mask_valid_reg <= 1'b0;
1159
1160     m_axi_awvalid_reg <= 1'b0;
1161     m_axi_bready_reg <= 1'b0;
1162
1163     s_axis_write_desc_ready_reg <= 1'b0;
1164     m_axis_write_desc_status_error_reg <= 4'd0;
1165     m_axis_write_desc_status_valid_reg <= 1'b0;
1166
1167     status_busy_reg <= 1'b0;
1168
1169     stat_wr_op_start_tag_reg <= 0;
1170     stat_wr_op_start_valid_reg <= 1'b0;
1171     stat_wr_op_finish_tag_reg <= 0;
1172     stat_wr_op_finish_valid_reg <= 1'b0;
1173     stat_wr_req_start_valid_reg <= 1'b0;
1174     stat_wr_req_finish_valid_reg <= 1'b0;
1175     stat_wr_op_table_full_reg <= 1'b0;
1176     stat_wr_tx_stall_reg <= 1'b0;
1177
1178     ram_rd_cmd_valid_reg <= {RAM_SEG_COUNT{1'b0}};
1179
1180     active_op_count_reg <= 0;
1181
1182     mask_fifo_wr_ptr_reg <= 0;
1183     mask_fifo_rd_ptr_reg <= 0;
1184
1185     op_table_start_ptr_reg <= 0;
1186     op_table_tx_start_ptr_reg <= 0;
1187     op_table_tx_finish_ptr_reg <= 0;
1188     op_table_finish_ptr_reg <= 0;
1189     op_table_active <= 0;
1190 end
1191 end
1192
1193 // output datapath logic
1194 reg [AXI_DATA_WIDTH-1:0] m_axi_wdata_reg =
{AXI_DATA_WIDTH{1'b0}};
1195 reg [AXI_STRB_WIDTH-1:0] m_axi_wstrb_reg =
{AXI_STRB_WIDTH{1'b0}};
1196 reg m_axi_wlast_reg = 1'b0;
1197 reg m_axi_wvalid_reg = 1'b0;
1198
1199 reg [OUTPUT_FIFO_ADDR_WIDTH+1:0] out_fifo_wr_ptr_reg = 0;
1200 reg [OUTPUT_FIFO_ADDR_WIDTH+1:0] out_fifo_rd_ptr_reg = 0;
1201 reg out_fifo_half_full_reg = 1'b0;
1202
1203 wire out_fifo_full = out_fifo_wr_ptr_reg ==
(out_fifo_rd_ptr_reg ^ {1'b1, {OUTPUT_FIFO_ADDR_WIDTH{1'b0}}});
1204 wire out_fifo_empty = out_fifo_wr_ptr_reg ==
out_fifo_rd_ptr_reg;
1205
1206 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
1207 reg [AXI_DATA_WIDTH-1:0]
out_fifo_wdata[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
1208 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
1209 reg [AXI_STRB_WIDTH-1:0]
out_fifo_wstrb[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
1210 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
1211 reg
out_fifo_wlast[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
1212
1213 assign m_axi_wready_int = !out_fifo_half_full_reg;
1214
1215 assign m_axi_wdata = m_axi_wdata_reg;
1216 assign m_axi_wstrb = m_axi_wstrb_reg;
1217 assign m_axi_wvalid = m_axi_wvalid_reg;
1218 assign m_axi_wlast = m_axi_wlast_reg;
1219
1220 always @(posedge clk) begin
1221     m_axi_wvalid_reg <= m_axi_wvalid_reg && !m_axi_wready;
1222
1223     out_fifo_half_full_reg <= $unsigned(out_fifo_wr_ptr_reg -
out_fifo_rd_ptr_reg) >= 2**(OUTPUT_FIFO_ADDR_WIDTH-1);
1224
1225     if (!out_fifo_full && m_axi_wvalid_int) begin

```

```

1226
out_fifo_wdata[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axi_wdata_int;
1227
out_fifo_wstrb[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axi_wstrb_int;
1228
out_fifo_wlast[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axi_wlast_int;
1229     out_fifo_wr_ptr_reg <= out_fifo_wr_ptr_reg + 1;
1230 end
1231
1232 if (!out_fifo_empty && (!m_axi_wvalid_reg ||
m_axi_wready)) begin
1233     m_axi_wdata_reg <=
out_fifo_wdata[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
1234     m_axi_wstrb_reg <=
out_fifo_wstrb[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
1235     m_axi_wlast_reg <=
out_fifo_wlast[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
1236     m_axi_wvalid_reg <= 1'b1;
1237     out_fifo_rd_ptr_reg <= out_fifo_rd_ptr_reg + 1;
1238 end
1239
1240 if (rst) begin
1241     out_fifo_wr_ptr_reg <= 0;
1242     out_fifo_rd_ptr_reg <= 0;
1243     m_axi_wvalid_reg <= 1'b0;
1244 end
1245 end
1246
1247 endmodule
1248
1249 `resetall

```

8. alexforenrich_verilog-pcie/rtl/dma_if_desc_mux.v (365 lines)

```

1 /*
2
3 Copyright (c) 2019-2021 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * DMA interface descriptor mux
33 */
34 module dma_if_desc_mux #
35 (
36     // Number of ports
37     parameter PORTS = 2,
38     // Extend RAM select signal
39     parameter EXTEND_RAM_SEL = 0,
40     // Input RAM segment select width
41     parameter S_RAM_SEL_WIDTH = 2,
42     // Output RAM segment select width
43     // Additional bits required for response routing
44     parameter M_RAM_SEL_WIDTH = S_RAM_SEL_WIDTH+(EX-
TEND_RAM_SEL ? $clog2(PORTS) : 0),
45     // RAM address width
46     parameter RAM_ADDR_WIDTH = 16,
47     // DMA address width
48     parameter DMA_ADDR_WIDTH = 64,
49     // Immediate enable

```



```

50 parameter IMM_ENABLE = 0,
51 // Immediate width
52 parameter IMM_WIDTH = 32,
53 // Length field width
54 parameter LEN_WIDTH = 16,
55 // Input tag field width
56 parameter S_TAG_WIDTH = 8,
57 // Output tag field width (towards DMA interface module)
58 // Additional bits required for response routing
59 parameter M_TAG_WIDTH = S_TAG_WIDTH+$clog2(PORTS),
60 // select round robin arbitration
61 parameter ARB_TYPE_ROUND_ROBIN = 0,
62 // LSB priority selection
63 parameter ARB_LSB_HIGH_PRIORITY = 1
64 )
65 (
66     input wire          clk,
67     input wire          rst,
68
69     /*
70      * Read descriptor output (to DMA interface)
71      */
72     output wire [DMA_ADDR_WIDTH-1:0] m_ax-
is_desc_dma_addr,
73     output wire [M_RAM_SEL_WIDTH-1:0] m_ax-
is_desc_ram_sel,
74     output wire [RAM_ADDR_WIDTH-1:0] m_ax-
is_desc_ram_addr,
75     output wire [IMM_WIDTH-1:0] m_ax-
is_desc_imm,
76     output wire          m_ax-
is_desc_imm_en,
77     output wire [LEN_WIDTH-1:0] m_ax-
is_desc_len,
78     output wire [M_TAG_WIDTH-1:0] m_ax-
is_desc_tag,
79     output wire          m_ax-
is_desc_valid,
80     input wire          m_ax-
is_desc_ready,
81
82     /*
83      * Read descriptor status input (from DMA interface)
84      */
85     input wire [M_TAG_WIDTH-1:0] s_ax-
is_desc_status_tag,
86     input wire [3:0] s_ax-
is_desc_status_error,
87     input wire s_ax-
is_desc_status_valid,
88
89     /*
90      * Read descriptor input
91      */
92     input wire [PORTS*DMA_ADDR_WIDTH-1:0] s_ax-
is_desc_dma_addr,
93     input wire [PORTS*S_RAM_SEL_WIDTH-1:0] s_ax-
is_desc_ram_sel,
94     input wire [PORTS*RAM_ADDR_WIDTH-1:0] s_ax-
is_desc_ram_addr,
95     input wire [PORTS*IMM_WIDTH-1:0] s_ax-
is_desc_imm,
96     input wire [PORTS:1:0] s_ax-
is_desc_imm_en,
97     input wire [PORTS*LEN_WIDTH-1:0] s_ax-
is_desc_len,
98     input wire [PORTS*S_TAG_WIDTH-1:0] s_ax-
is_desc_tag,
99     input wire [PORTS:1:0] s_ax-
is_desc_valid,
100    output wire [PORTS:1:0] s_ax-
is_desc_ready,
101
102    /*
103     * Read descriptor status output
104     */
105    output wire [PORTS*S_TAG_WIDTH-1:0] m_ax-
is_desc_status_tag,
106    output wire [PORTS*4-1:0] m_ax-
is_desc_status_error,
107    output wire [PORTS:1:0] m_ax-
is_desc_status_valid
108 );
109 parameter CL_PORTS = $clog2(PORTS);
110
111 parameter S_RAM_SEL_WIDTH_INT = S_RAM_SEL_WIDTH > 0 ?
S_RAM_SEL_WIDTH : 1;
112 parameter FIFO_ADDR_WIDTH = 5;
113
114 // check configuration
115 initial begin
116     if (M_TAG_WIDTH < S_TAG_WIDTH+$clog2(PORTS)) begin
117         $error("Error: M_TAG_WIDTH must be at least $clog2(PORTS) larger than S_TAG_WIDTH (instance %m)");
118     $finish;
119     end
120 end
121
122 if (EXTEND_RAM_SEL) begin
123     if (M_RAM_SEL_WIDTH < S_RAM_SEL_WIDTH+$clog2(PORTS))

```

```

begin
125     $error("Error: M_RAM_SEL_WIDTH must be at least
$clog2(PORTS) larger than S_RAM_SEL_WIDTH (instance %m)");
126     $finish;
127 end
128 end else begin
129     if (M_RAM_SEL_WIDTH < S_RAM_SEL_WIDTH) begin
130         $error("Error: M_RAM_SEL_WIDTH must be no smaller
than S_RAM_SEL_WIDTH (instance %m)");
131     $finish;
132     end
133 end
134 end
135
136 // descriptor mux
137 wire [PORTS-1:0] request;
138 wire [PORTS-1:0] acknowledge;
139 wire [PORTS-1:0] grant;
140 wire grant_valid;
141 wire [CL_PORTS-1:0] grant_encoded;
142
143 // input registers to pipeline arbitration delay
144 reg [PORTS*DMA_ADDR_WIDTH-1:0] s_axis_desc_dma_addr_reg = 0;
145 reg [PORTS*S_RAM_SEL_WIDTH-1:0] s_axis_desc_ram_sel_reg = 0;
146 reg [PORTS*RAM_ADDR_WIDTH-1:0] s_axis_desc_ram_addr_reg = 0;
147 reg [PORTS*IMM_WIDTH-1:0] s_axis_desc_imm_reg = 0;
148 reg [PORTS:1:0] s_axis_desc_imm_en_reg = 0;
149 reg [PORTS*LEN_WIDTH-1:0] s_axis_desc_len_reg = 0;
150 reg [PORTS*S_TAG_WIDTH-1:0] s_axis_desc_tag_reg = 0;
151 reg [PORTS:1:0] s_axis_desc_valid_reg = 0;
152
153 // internal datapath
154 reg [DMA_ADDR_WIDTH-1:0] m_axis_desc_dma_addr_int;
155 reg [M_RAM_SEL_WIDTH-1:0] m_axis_desc_ram_sel_int;
156 reg [RAM_ADDR_WIDTH-1:0] m_axis_desc_ram_addr_int;
157 reg [IMM_WIDTH-1:0] m_axis_desc_imm_int;
158 reg [LEN_WIDTH-1:0] m_axis_desc_imm_en_int;
159 reg [LEN_WIDTH-1:0] m_axis_desc_len_int;
160 reg [M_TAG_WIDTH-1:0] m_axis_desc_tag_int;
161 reg m_axis_desc_valid_int;
162 reg m_axis_desc_ready_int_reg = 1'b0;
163 wire m_axis_desc_ready_int_early;
164
165 assign s_axis_desc_ready = ~s_axis_desc_valid_reg |
({PORTS{m_axis_desc_ready_int_reg}} & grant);
166
167 // mux for incoming packet
168 wire [DMA_ADDR_WIDTH-1:0] current_s_desc_dma_addr = s_ax-
is_desc_dma_addr_reg[grant_encoded*DMA_ADDR_WIDTH +: DMA_AD-
DR_WIDTH];
169 wire [S_RAM_SEL_WIDTH-1:0] current_s_desc_ram_sel = s_ax-
is_desc_ram_sel_reg[grant_encoded*S_RAM_SEL_WIDTH +:
S_RAM_SEL_WIDTH_INT];
170 wire [RAM_ADDR_WIDTH-1:0] current_s_desc_ram_addr = s_ax-
is_desc_ram_addr_reg[grant_encoded*RAM_ADDR_WIDTH +: RAM_AD-
DR_WIDTH];
171 wire [IMM_WIDTH-1:0] current_s_desc_imm = s_ax-
is_desc_imm_reg[grant_encoded*IMM_WIDTH +: IMM_WIDTH];
172 wire [LEN_WIDTH-1:0] current_s_desc_imm_en = s_ax-
is_desc_imm_en_reg[grant_encoded];
173 wire [LEN_WIDTH-1:0] current_s_desc_len = s_ax-
is_desc_len_reg[grant_encoded*LEN_WIDTH +: LEN_WIDTH];
174 wire [S_TAG_WIDTH-1:0] current_s_desc_tag = s_ax-
is_desc_tag_reg[grant_encoded*S_TAG_WIDTH +: S_TAG_WIDTH];
175 wire current_s_desc_valid = s_ax-
is_desc_valid_reg[grant_encoded];
176 wire current_s_desc_ready = s_ax-
is_desc_ready[grant_encoded];
177
178 // arbiter instance
179 arbiter #(
180     .PORTS(PORTS),
181     .ARB_TYPE_ROUND_ROBIN(ARB_TYPE_ROUND_ROBIN),
182     .ARB_BLOCK(1),
183     .ARB_BLOCK_ACK(1),
184     .ARB_LSB_HIGH_PRIORITY(ARB_LSB_HIGH_PRIORITY)
185 )
186 arb_inst (
187     .clk(clk),
188     .rst(rst),
189     .request(request),
190     .acknowledge(acknowledge),
191     .grant(grant),
192     .grant_valid(grant_valid),
193     .grant_encoded(grant_encoded)
194 );
195
196 assign request = (s_axis_desc_valid_reg & ~grant) | (s_ax-
is_desc_valid & grant);
197 assign acknowledge = grant & s_axis_desc_valid_reg &
{PORTS{m_axis_desc_ready_int_reg}};
198
199 always @* begin
200     // pass through selected packet data
201     m_axis_desc_dma_addr_int = current_s_desc_dma_addr;
202     m_axis_desc_ram_sel_int = current_s_desc_ram_sel;
203     if (EXTEND_RAM_SEL && PORTS > 1) begin
204         m_ax-
is_desc_ram_sel_int[M_RAM_SEL_WIDTH-1:M_RAM_SEL_WIDTH-CL_PORTS] =
grant_encoded;
205     end

```

```

206 m_axis_desc_ram_addr_int = current_s_desc_ram_addr;
207 m_axis_desc_imm_int      = current_s_desc_imm;
208 m_axis_desc_imm_en_int   = current_s_desc_imm_en;
209 m_axis_desc_len_int      = current_s_desc_len;
210 m_axis_desc_tag_int      = current_s_desc_tag;
211 if (PORTS > 1) begin
212     m_axis_desc_tag_int[M_TAG_WIDTH-1:M_TAG_WIDTH-
CL_PORTS] = grant_encoded;
213 end
214 m_axis_desc_valid_int    = current_s_desc_valid && m_ax-
is_desc_ready_int_reg && grant_valid;
215 end
216
217 integer i;
218
219 always @(posedge clk) begin
220     // register inputs
221     for (i = 0; i < PORTS; i = i + 1) begin
222         if (s_axis_desc_ready[i]) begin
223             s_axis_desc_dma_addr_reg[i*DMA_ADDR_WIDTH +:
DMA_ADDR_WIDTH] <= s_axis_desc_dma_addr[i*DMA_ADDR_WIDTH +: DMA_AD-
DR_WIDTH];
224             s_axis_desc_ram_sel_reg[i*S_RAM_SEL_WIDTH +:
S_RAM_SEL_WIDTH_INT] <= s_axis_desc_ram_sel[i*S_RAM_SEL_WIDTH +:
S_RAM_SEL_WIDTH_INT];
225             s_axis_desc_ram_addr_reg[i*RAM_ADDR_WIDTH +:
RAM_ADDR_WIDTH] <= s_axis_desc_ram_addr[i*RAM_ADDR_WIDTH +: RAM_AD-
DR_WIDTH];
226             s_axis_desc_imm_reg[i*IMM_WIDTH +: IMM_WIDTH] <=
s_axis_desc_imm[i*IMM_WIDTH +: IMM_WIDTH];
227             s_axis_desc_imm_en_reg[i] <= s_ax-
is_desc_imm_en[i];
228             s_axis_desc_len_reg[i*LEN_WIDTH +: LEN_WIDTH] <=
s_axis_desc_len[i*LEN_WIDTH +: LEN_WIDTH];
229             s_axis_desc_tag_reg[i*S_TAG_WIDTH +: S_TAG_WIDTH]
<= s_axis_desc_tag[i*S_TAG_WIDTH +: S_TAG_WIDTH];
230             s_axis_desc_valid_reg[i] <= s_axis_desc_valid[i];
231         end
232     end
233
234     if (rst) begin
235         s_axis_desc_valid_reg <= 0;
236     end
237 end
238
239 // output datapath logic
240 reg [DMA_ADDR_WIDTH-1:0] m_axis_desc_dma_addr_reg = {DMA_AD-
DR_WIDTH{1'b0}};
241 reg [M_RAM_SEL_WIDTH-1:0] m_axis_desc_ram_sel_reg =
{M_RAM_SEL_WIDTH{1'b0}};
242 reg [RAM_ADDR_WIDTH-1:0] m_axis_desc_ram_addr_reg = {RAM_AD-
DR_WIDTH{1'b0}};
243 reg [IMM_WIDTH-1:0] m_axis_desc_imm_reg =
{IMM_WIDTH{1'b0}};
244 reg m_axis_desc_imm_en_reg = 1'b0;
245 reg [LEN_WIDTH-1:0] m_axis_desc_len_reg =
{LEN_WIDTH{1'b0}};
246 reg [M_TAG_WIDTH-1:0] m_axis_desc_tag_reg =
{M_TAG_WIDTH{1'b0}};
247 reg m_axis_desc_valid_reg = 1'b0,
m_axis_desc_valid_next;
248
249 reg [DMA_ADDR_WIDTH-1:0] temp_m_axis_desc_dma_addr_reg =
{DMA_ADDR_WIDTH{1'b0}};
250 reg [M_RAM_SEL_WIDTH-1:0] temp_m_axis_desc_ram_sel_reg =
{M_RAM_SEL_WIDTH{1'b0}};
251 reg [RAM_ADDR_WIDTH-1:0] temp_m_axis_desc_ram_addr_reg =
{RAM_ADDR_WIDTH{1'b0}};
252 reg [IMM_WIDTH-1:0] temp_m_axis_desc_imm_reg =
{IMM_WIDTH{1'b0}};
253 reg temp_m_axis_desc_imm_en_reg =
1'b0;
254 reg [LEN_WIDTH-1:0] temp_m_axis_desc_len_reg =
{LEN_WIDTH{1'b0}};
255 reg [M_TAG_WIDTH-1:0] temp_m_axis_desc_tag_reg =
{M_TAG_WIDTH{1'b0}};
256 reg temp_m_axis_desc_valid_reg =
1'b0, temp_m_axis_desc_valid_next;
257
258 // datapath control
259 reg store_axis_int_to_output;
260 reg store_axis_int_to_temp;
261 reg store_axis_temp_to_output;
262
263 assign m_axis_desc_dma_addr = m_axis_desc_dma_addr_reg;
264 assign m_axis_desc_ram_sel = m_axis_desc_ram_sel_reg;
265 assign m_axis_desc_ram_addr = m_axis_desc_ram_addr_reg;
266 assign m_axis_desc_imm = IMM_ENABLE ? m_ax-
is_desc_imm_reg : {IMM_WIDTH{1'b0}};
267 assign m_axis_desc_imm_en = IMM_ENABLE ? m_ax-
is_desc_imm_en_reg : 1'b0;
268 assign m_axis_desc_len = m_axis_desc_len_reg;
269 assign m_axis_desc_tag = m_axis_desc_tag_reg;
270 assign m_axis_desc_valid = m_axis_desc_valid_reg;
271
272 // enable ready input next cycle if output is ready or if both
output registers are empty
273 assign m_axis_desc_ready_int_early = m_axis_desc_ready || (!
temp_m_axis_desc_valid_reg && !m_axis_desc_valid_reg);
274

```

```

275 always @* begin
276     // transfer sink ready state to source
277     m_axis_desc_valid_next = m_axis_desc_valid_reg;
278     temp_m_axis_desc_valid_next = temp_m_axis_desc_valid_reg;
279
280     store_axis_int_to_output = 1'b0;
281     store_axis_int_to_temp = 1'b0;
282     store_axis_temp_to_output = 1'b0;
283
284     if (m_axis_desc_ready_int_reg) begin
285         // input is ready
286         if (m_axis_desc_ready || !m_axis_desc_valid_reg) begin
287             // output is ready or currently not valid, trans-
fer data to output
288             m_axis_desc_valid_next = m_axis_desc_valid_int;
289             store_axis_int_to_output = 1'b1;
290         end else begin
291             // output is not ready, store input in temp
292             temp_m_axis_desc_valid_next = m_ax-
is_desc_valid_int;
293             store_axis_int_to_temp = 1'b1;
294         end
295     end else if (m_axis_desc_ready) begin
296         // input is not ready, but output is ready
297         m_axis_desc_valid_next = temp_m_axis_desc_valid_reg;
298         temp_m_axis_desc_valid_next = 1'b0;
299         store_axis_temp_to_output = 1'b1;
300     end
301 end
302
303 always @(posedge clk) begin
304     m_axis_desc_valid_reg <= m_axis_desc_valid_next;
305     m_axis_desc_ready_int_reg <= m_axis_desc_ready_int_early;
306     temp_m_axis_desc_valid_reg <= temp_m_axis_desc_valid_next;
307
308     // datapath
309     if (store_axis_int_to_output) begin
310         m_axis_desc_dma_addr_reg <= m_axis_desc_dma_addr_int;
311         m_axis_desc_ram_sel_reg <= m_axis_desc_ram_sel_int;
312         m_axis_desc_ram_addr_reg <= m_axis_desc_ram_addr_int;
313         m_axis_desc_imm_reg <= m_axis_desc_imm_int;
314         m_axis_desc_imm_en_reg <= m_axis_desc_imm_en_int;
315         m_axis_desc_len_reg <= m_axis_desc_len_int;
316         m_axis_desc_tag_reg <= m_axis_desc_tag_int;
317     end else if (store_axis_temp_to_output) begin
318         m_axis_desc_dma_addr_reg <= temp_m_ax-
is_desc_dma_addr_reg;
319         m_axis_desc_ram_sel_reg <= temp_m_ax-
is_desc_ram_sel_reg;
320         m_axis_desc_ram_addr_reg <= temp_m_ax-
is_desc_ram_addr_reg;
321         m_axis_desc_imm_reg <= temp_m_axis_desc_imm_reg;
322         m_axis_desc_imm_en_reg <= temp_m_axis_desc_imm_en_reg;
323         m_axis_desc_len_reg <= temp_m_axis_desc_len_reg;
324         m_axis_desc_tag_reg <= temp_m_axis_desc_tag_reg;
325     end
326
327     if (store_axis_int_to_temp) begin
328         temp_m_axis_desc_dma_addr_reg <= m_ax-
is_desc_dma_addr_int;
329         temp_m_axis_desc_ram_sel_reg <= m_ax-
is_desc_ram_sel_int;
330         temp_m_axis_desc_ram_addr_reg <= m_ax-
is_desc_ram_addr_int;
331         temp_m_axis_desc_imm_reg <= m_axis_desc_imm_int;
332         temp_m_axis_desc_imm_en_reg <= m_axis_desc_imm_en_int;
333         temp_m_axis_desc_len_reg <= m_axis_desc_len_int;
334         temp_m_axis_desc_tag_reg <= m_axis_desc_tag_int;
335     end
336
337     if (rst) begin
338         m_axis_desc_valid_reg <= 1'b0;
339         m_axis_desc_ready_int_reg <= 1'b0;
340         temp_m_axis_desc_valid_reg <= 1'b0;
341     end
342 end
343
344 // descriptor status demux
345 reg [S_TAG_WIDTH-1:0] m_axis_desc_status_tag_reg =
{S_TAG_WIDTH{1'b0}};
346 reg [3:0] m_axis_desc_status_error_reg = 4'd0;
347 reg [PORTS-1:0] m_axis_desc_status_valid_reg = {PORTS{1'b0}};
348
349 assign m_axis_desc_status_tag =
{PORTS{m_axis_desc_status_tag_reg}};
350 assign m_axis_desc_status_error =
{PORTS{m_axis_desc_status_error_reg}};
351 assign m_axis_desc_status_valid = m_ax-
is_desc_status_valid_reg;
352
353 always @(posedge clk) begin
354     m_axis_desc_status_tag_reg <= s_axis_desc_status_tag;
355     m_axis_desc_status_error_reg <= s_axis_desc_status_error;
356     m_axis_desc_status_valid_reg <= s_axis_desc_status_valid
<< (PORTS > 1 ? s_axis_desc_status_tag[M_TAG_WIDTH-1:M_TAG_WIDTH-
CL_PORTS] : 0);
357
358     if (rst) begin
359         m_axis_desc_status_valid_reg <= {PORTS{1'b0}};
360     end
361 end

```

```

362
363 endmodule
364
365 `resetall

```

9. alexforenrich_verilog-pcie/rtl/dma_if_mux.v (354 lines)

```

1 /*
2
3 Copyright (c) 2019-2021 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * DMA interface mux
33 */
34 module dma_if_mux #
35 (
36     // Number of ports
37     parameter PORTS = 2,
38     // RAM segment count
39     parameter SEG_COUNT = 2,
40     // RAM segment data width
41     parameter SEG_DATA_WIDTH = 64,
42     // RAM segment address width
43     parameter SEG_ADDR_WIDTH = 8,
44     // RAM segment byte enable width
45     parameter SEG_BE_WIDTH = SEG_DATA_WIDTH/8,
46     // Input RAM segment select width
47     parameter S_RAM_SEL_WIDTH = 2,
48     // Output RAM segment select width
49     // Additional bits required for response routing
50     parameter M_RAM_SEL_WIDTH = S_RAM_SEL_WIDTH+$clog2(PORTS),
51     // RAM address width
52     parameter RAM_ADDR_WIDTH = SEG_ADDR_WIDTH+$clog2-
g2(SEG_COUNT)+$clog2(SEG_BE_WIDTH),
53     // DMA address width
54     parameter DMA_ADDR_WIDTH = 64,
55     // Immediate enable
56     parameter IMM_ENABLE = 0,
57     // Immediate width
58     parameter IMM_WIDTH = 32,
59     // Length field width
60     parameter LEN_WIDTH = 16,
61     // Input tag field width
62     parameter S_TAG_WIDTH = 8,
63     // Output tag field width (towards DMA module)
64     // Additional bits required for response routing
65     parameter M_TAG_WIDTH = S_TAG_WIDTH+$clog2(PORTS),
66     // select round robin arbitration
67     parameter ARB_TYPE_ROUND_ROBIN = 0,
68     // LSB priority selection
69     parameter ARB_LSB_HIGH_PRIORITY = 1
70 )
71 (
72     input wire clk,
73     input wire rst,
74
75     /*
76     * Read descriptor output (to DMA interface)
77     */
78     output wire [DMA_ADDR_WIDTH-1:0] m_ax-
is_read_desc_dma_addr,

```

```

79     output wire [M_RAM_SEL_WIDTH-1:0] m_ax-
is_read_desc_ram_sel,
80     output wire [RAM_ADDR_WIDTH-1:0] m_ax-
is_read_desc_ram_addr,
81     output wire [LEN_WIDTH-1:0] m_ax-
is_read_desc_len,
82     output wire [M_TAG_WIDTH-1:0] m_ax-
is_read_desc_tag,
83     output wire m_ax-
is_read_desc_valid,
84     input wire m_ax-
is_read_desc_ready,
85
86     /*
87     * Read descriptor status input (from DMA interface)
88     */
89     input wire [M_TAG_WIDTH-1:0] s_ax-
is_read_desc_status_tag,
90     input wire [3:0] s_ax-
is_read_desc_status_error,
91     input wire s_ax-
is_read_desc_status_valid,
92
93     /*
94     * Write descriptor output (to DMA interface)
95     */
96     output wire [DMA_ADDR_WIDTH-1:0] m_ax-
is_write_desc_dma_addr,
97     output wire [M_RAM_SEL_WIDTH-1:0] m_ax-
is_write_desc_ram_sel,
98     output wire [RAM_ADDR_WIDTH-1:0] m_ax-
is_write_desc_ram_addr,
99     output wire [IMM_WIDTH-1:0] m_ax-
is_write_desc_imm,
100    output wire m_ax-
is_write_desc_imm_en,
101    output wire [LEN_WIDTH-1:0] m_ax-
is_write_desc_len,
102    output wire [M_TAG_WIDTH-1:0] m_ax-
is_write_desc_tag,
103    output wire m_ax-
is_write_desc_valid,
104    input wire m_ax-
is_write_desc_ready,
105
106    /*
107    * Write descriptor status input (from DMA interface)
108    */
109    input wire [M_TAG_WIDTH-1:0] s_ax-
is_write_desc_status_tag,
110    input wire [3:0] s_ax-
is_write_desc_status_error,
111    input wire s_ax-
is_write_desc_status_valid,
112
113    /*
114    * Read descriptor input
115    */
116    input wire [PORTS*DMA_ADDR_WIDTH-1:0] s_ax-
is_read_desc_dma_addr,
117    input wire [PORTS*S_RAM_SEL_WIDTH-1:0] s_ax-
is_read_desc_ram_sel,
118    input wire [PORTS*RAM_ADDR_WIDTH-1:0] s_ax-
is_read_desc_ram_addr,
119    input wire [PORTS*LEN_WIDTH-1:0] s_ax-
is_read_desc_len,
120    input wire [PORTS*S_TAG_WIDTH-1:0] s_ax-
is_read_desc_tag,
121    input wire [PORTS-1:0] s_ax-
is_read_desc_valid,
122    output wire [PORTS-1:0] s_ax-
is_read_desc_ready,
123
124    /*
125    * Read descriptor status output
126    */
127    output wire [PORTS*S_TAG_WIDTH-1:0] m_ax-
is_read_desc_status_tag,
128    output wire [PORTS*4-1:0] m_ax-
is_read_desc_status_error,
129    output wire [PORTS-1:0] m_ax-
is_read_desc_status_valid,
130
131    /*
132    * Write descriptor input
133    */
134    input wire [PORTS*DMA_ADDR_WIDTH-1:0] s_ax-
is_write_desc_dma_addr,
135    input wire [PORTS*S_RAM_SEL_WIDTH-1:0] s_ax-
is_write_desc_ram_sel,
136    input wire [PORTS*RAM_ADDR_WIDTH-1:0] s_ax-
is_write_desc_ram_addr,
137    input wire [PORTS*IMM_WIDTH-1:0] s_ax-
is_write_desc_imm,
138    input wire [PORTS-1:0] s_ax-
is_write_desc_imm_en,
139    input wire [PORTS*LEN_WIDTH-1:0] s_ax-
is_write_desc_len,
140    input wire [PORTS*S_TAG_WIDTH-1:0] s_ax-
is_write_desc_tag,
141    input wire [PORTS-1:0] s_ax-

```

```

is_write_desc_valid,
142     output wire [PORTS:1:0]
is_write_desc_ready,
143
144     /*
145     * Write descriptor status output
146     */
147     output wire [PORTS*S_TAG_WIDTH-1:0]
is_write_desc_status_tag,
148     output wire [PORTS*4-1:0]
is_write_desc_status_error,
149     output wire [PORTS:1:0]
is_write_desc_status_valid,
150
151     /*
152     * RAM interface (from DMA interface)
153     */
154     input wire [SEG_COUNT*M_RAM_SEL_WIDTH-1:0]
if_ram_wr_cmd_sel,
155     input wire [SEG_COUNT*SEG_BE_WIDTH-1:0]
if_ram_wr_cmd_be,
156     input wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0]
if_ram_wr_cmd_addr,
157     input wire [SEG_COUNT*SEG_DATA_WIDTH-1:0]
if_ram_wr_cmd_data,
158     input wire [SEG_COUNT-1:0]
if_ram_wr_cmd_valid,
159     output wire [SEG_COUNT-1:0]
if_ram_wr_cmd_ready,
160     output wire [SEG_COUNT-1:0]
if_ram_wr_done,
161     input wire [SEG_COUNT*M_RAM_SEL_WIDTH-1:0]
if_ram_rd_cmd_sel,
162     input wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0]
if_ram_rd_cmd_addr,
163     input wire [SEG_COUNT-1:0]
if_ram_rd_cmd_valid,
164     output wire [SEG_COUNT-1:0]
if_ram_rd_cmd_ready,
165     output wire [SEG_COUNT*SEG_DATA_WIDTH-1:0]
if_ram_rd_resp_data,
166     output wire [SEG_COUNT-1:0]
if_ram_rd_resp_valid,
167     input wire [SEG_COUNT-1:0]
if_ram_rd_resp_ready,
168
169     /*
170     * RAM interface (towards RAM)
171     */
172     output wire [PORTS*SEG_COUNT*S_RAM_SEL_WIDTH-1:0]
ram_wr_cmd_sel,
173     output wire [PORTS*SEG_COUNT*SEG_BE_WIDTH-1:0]
ram_wr_cmd_be,
174     output wire [PORTS*SEG_COUNT*SEG_ADDR_WIDTH-1:0]
ram_wr_cmd_addr,
175     output wire [PORTS*SEG_COUNT*SEG_DATA_WIDTH-1:0]
ram_wr_cmd_data,
176     output wire [PORTS*SEG_COUNT-1:0]
ram_wr_cmd_valid,
177     input wire [PORTS*SEG_COUNT-1:0]
ram_wr_cmd_ready,
178     input wire [PORTS*SEG_COUNT-1:0]
ram_wr_done,
179     output wire [PORTS*SEG_COUNT*S_RAM_SEL_WIDTH-1:0]
ram_rd_cmd_sel,
180     output wire [PORTS*SEG_COUNT*SEG_ADDR_WIDTH-1:0]
ram_rd_cmd_addr,
181     output wire [PORTS*SEG_COUNT-1:0]
ram_rd_cmd_valid,
182     input wire [PORTS*SEG_COUNT-1:0]
ram_rd_cmd_ready,
183     input wire [PORTS*SEG_COUNT*SEG_DATA_WIDTH-1:0]
ram_rd_resp_data,
184     input wire [PORTS*SEG_COUNT-1:0]
ram_rd_resp_valid,
185     output wire [PORTS*SEG_COUNT-1:0]
ram_rd_resp_ready
186 );
187
188 dma_if_mux_rd #(
189     .PORTS(PORTS),
190     .SEG_COUNT(SEG_COUNT),
191     .SEG_DATA_WIDTH(SEG_DATA_WIDTH),
192     .SEG_ADDR_WIDTH(SEG_ADDR_WIDTH),
193     .SEG_BE_WIDTH(SEG_BE_WIDTH),
194     .S_RAM_SEL_WIDTH(S_RAM_SEL_WIDTH),
195     .M_RAM_SEL_WIDTH(M_RAM_SEL_WIDTH),
196     .RAM_ADDR_WIDTH(RAM_ADDR_WIDTH),
197     .DMA_ADDR_WIDTH(DMA_ADDR_WIDTH),
198     .LEN_WIDTH(LEN_WIDTH),
199     .S_TAG_WIDTH(S_TAG_WIDTH),
200     .M_TAG_WIDTH(M_TAG_WIDTH),
201     .ARB_TYPE_ROUND_ROBIN(ARB_TYPE_ROUND_ROBIN),
202     .ARB_LSB_HIGH_PRIORITY(ARB_LSB_HIGH_PRIORITY)
203 )
204 dma_if_mux_rd_inst (
205     .clk(clk),
206     .rst(rst),
207
208     /*
209     * Descriptor output (to DMA interface)
210
211     */
212     .m_axis_read_desc_dma_addr(m_axis_read_desc_dma_addr),
213     .m_axis_read_desc_ram_sel(m_axis_read_desc_ram_sel),
214     .m_axis_read_desc_ram_addr(m_axis_read_desc_ram_addr),
215     .m_axis_read_desc_len(m_axis_read_desc_len),
216     .m_axis_read_desc_tag(m_axis_read_desc_tag),
217     .m_axis_read_desc_valid(m_axis_read_desc_valid),
218     .m_axis_read_desc_ready(m_axis_read_desc_ready),
219
220     /*
221     * Descriptor status input (from DMA interface)
222     */
223     .s_axis_read_desc_status_tag(s_axis_read_desc_status_tag),
224     .s_axis_read_desc_status_error(s_axis_read_desc_status_error),
225     .s_axis_read_desc_status_valid(s_axis_read_desc_status_valid),
226
227     /*
228     * Descriptor input
229     */
230     .s_axis_read_desc_dma_addr(s_axis_read_desc_dma_addr),
231     .s_axis_read_desc_ram_sel(s_axis_read_desc_ram_sel),
232     .s_axis_read_desc_ram_addr(s_axis_read_desc_ram_addr),
233     .s_axis_read_desc_len(s_axis_read_desc_len),
234     .s_axis_read_desc_tag(s_axis_read_desc_tag),
235     .s_axis_read_desc_valid(s_axis_read_desc_valid),
236     .s_axis_read_desc_ready(s_axis_read_desc_ready),
237
238     /*
239     * Descriptor status output
240     */
241     .m_axis_read_desc_status_tag(m_axis_read_desc_status_tag),
242     .m_axis_read_desc_status_error(m_axis_read_desc_status_error),
243     .m_axis_read_desc_status_valid(m_axis_read_desc_status_valid),
244
245     /*
246     * RAM interface (from DMA interface)
247     */
248     .if_ram_wr_cmd_sel(if_ram_wr_cmd_sel),
249     .if_ram_wr_cmd_be(if_ram_wr_cmd_be),
250     .if_ram_wr_cmd_addr(if_ram_wr_cmd_addr),
251     .if_ram_wr_cmd_data(if_ram_wr_cmd_data),
252     .if_ram_wr_cmd_valid(if_ram_wr_cmd_valid),
253     .if_ram_wr_cmd_ready(if_ram_wr_cmd_ready),
254     .if_ram_wr_done(if_ram_wr_done),
255
256     /*
257     * RAM interface (towards RAM)
258     */
259     .ram_wr_cmd_sel(ram_wr_cmd_sel),
260     .ram_wr_cmd_be(ram_wr_cmd_be),
261     .ram_wr_cmd_addr(ram_wr_cmd_addr),
262     .ram_wr_cmd_data(ram_wr_cmd_data),
263     .ram_wr_cmd_valid(ram_wr_cmd_valid),
264     .ram_wr_cmd_ready(ram_wr_cmd_ready),
265     .ram_wr_done(ram_wr_done)
266 );
267
268 dma_if_mux_wr #(
269     .PORTS(PORTS),
270     .SEG_COUNT(SEG_COUNT),
271     .SEG_DATA_WIDTH(SEG_DATA_WIDTH),
272     .SEG_ADDR_WIDTH(SEG_ADDR_WIDTH),
273     .SEG_BE_WIDTH(SEG_BE_WIDTH),
274     .S_RAM_SEL_WIDTH(S_RAM_SEL_WIDTH),
275     .M_RAM_SEL_WIDTH(M_RAM_SEL_WIDTH),
276     .RAM_ADDR_WIDTH(RAM_ADDR_WIDTH),
277     .DMA_ADDR_WIDTH(DMA_ADDR_WIDTH),
278     .IMM_ENABLE(IMM_ENABLE),
279     .IMM_WIDTH(IMM_WIDTH),
280     .LEN_WIDTH(LEN_WIDTH),
281     .S_TAG_WIDTH(S_TAG_WIDTH),
282     .M_TAG_WIDTH(M_TAG_WIDTH),
283     .ARB_TYPE_ROUND_ROBIN(ARB_TYPE_ROUND_ROBIN),
284     .ARB_LSB_HIGH_PRIORITY(ARB_LSB_HIGH_PRIORITY)
285 )
286 dma_if_mux_wr_inst (
287     .clk(clk),
288     .rst(rst),
289
290     /*
291     * Descriptor output (to DMA interface)
292     */
293     .m_axis_write_desc_dma_addr(m_axis_write_desc_dma_addr),
294     .m_axis_write_desc_ram_sel(m_axis_write_desc_ram_sel),
295     .m_axis_write_desc_ram_addr(m_axis_write_desc_ram_addr),
296     .m_axis_write_desc_imm(m_axis_write_desc_imm),
297     .m_axis_write_desc_imm_en(m_axis_write_desc_imm_en),
298     .m_axis_write_desc_len(m_axis_write_desc_len),
299     .m_axis_write_desc_tag(m_axis_write_desc_tag),
300     .m_axis_write_desc_valid(m_axis_write_desc_valid),
301     .m_axis_write_desc_ready(m_axis_write_desc_ready),
302
303     /*
304     * Descriptor status input (from DMA interface)
305     */
306     .s_axis_write_desc_status_tag(s_axis_write_desc_status_tag),

```

```

306      .s_ax-
is_write_desc_status_error(s_axis_write_desc_status_error),
307      .s_ax-
is_write_desc_status_valid(s_axis_write_desc_status_valid),
308
309      /*
310      * Descriptor input
311      */
312      .s_axis_write_desc_dma_addr(s_axis_write_desc_dma_addr),
313      .s_axis_write_desc_ram_sel(s_axis_write_desc_ram_sel),
314      .s_axis_write_desc_ram_addr(s_axis_write_desc_ram_addr),
315      .s_axis_write_desc_imm(s_axis_write_desc_imm),
316      .s_axis_write_desc_imm_en(s_axis_write_desc_imm_en),
317      .s_axis_write_desc_len(s_axis_write_desc_len),
318      .s_axis_write_desc_tag(s_axis_write_desc_tag),
319      .s_axis_write_desc_valid(s_axis_write_desc_valid),
320      .s_axis_write_desc_ready(s_axis_write_desc_ready),
321
322      /*
323      * Descriptor status output
324      */
325      .m_ax-
is_write_desc_status_tag(m_axis_write_desc_status_tag),
326      .m_ax-
is_write_desc_status_error(m_axis_write_desc_status_error),
327      .m_ax-
is_write_desc_status_valid(m_axis_write_desc_status_valid),

```

```

328
329      /*
330      * RAM interface (from DMA interface)
331      */
332      .if_ram_rd_cmd_sel(if_ram_rd_cmd_sel),
333      .if_ram_rd_cmd_addr(if_ram_rd_cmd_addr),
334      .if_ram_rd_cmd_valid(if_ram_rd_cmd_valid),
335      .if_ram_rd_cmd_ready(if_ram_rd_cmd_ready),
336      .if_ram_rd_resp_data(if_ram_rd_resp_data),
337      .if_ram_rd_resp_valid(if_ram_rd_resp_valid),
338      .if_ram_rd_resp_ready(if_ram_rd_resp_ready),
339
340      /*
341      * RAM interface (towards RAM)
342      */
343      .ram_rd_cmd_sel(ram_rd_cmd_sel),
344      .ram_rd_cmd_addr(ram_rd_cmd_addr),
345      .ram_rd_cmd_valid(ram_rd_cmd_valid),
346      .ram_rd_cmd_ready(ram_rd_cmd_ready),
347      .ram_rd_resp_data(ram_rd_resp_data),
348      .ram_rd_resp_valid(ram_rd_resp_valid),
349      .ram_rd_resp_ready(ram_rd_resp_ready)
350 );
351
352 endmodule
353
354 `resetall

```

RVFI UVM 에이전트

What this is. A UVM agent that monitors the RVFI interface and checks its properties.

Why it exists. Because the trace must be captured and checked, not merely emitted. The agent turns retirement events into transactions a scoreboard can consume.

Undergraduate review · EECS concepts. UVM agent structure (driver, monitor, sequencer); transaction-level modelling; assertions.

Graduate · Practice note. This is the bridge between the RTL's observability and the comparison logic. Studying it shows concretely how a verification environment is layered.

13 files • 1,666 lines

API — uvma_rvfi_agent.sv

Table 1850. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_agent.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMA_RVFI_AGENT_SV__		/**

API — uvma_rvfi_assert.sv

Table 1851. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_assert.sv — module / interface / package (1개)

이름	종류
uvma_rvfi_assert	module

Table 1852. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_assert.sv — parameter (2개)

파라미터	형/폭	기본값
ADDR_WIDTH	int	32
DATA_WIDTH	int	32

Table 1853. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_assert.sv — 포트 (11개)

포트	방향	형/폭
clk	input	
reset_n	input	
req	input	
gnt	input	
addr	input	[ADDR_WIDTH-1:0]
we	input	
be	input	[DATA_WIDTH/8-1:0]

포트	방향	형/폭
wdata	input	[DATA_WIDTH-1:0]
rdata	input	[DATA_WIDTH-1:0]
rvalid	input	
rready	input	

API — uvma_rvfi_cfg.sv

Table 1854. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_cfg.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMA_RVFI_CFG_SV__		/**

API — uvma_rvfi_cntxt.sv

Table 1855. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_cntxt.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMA_RVFI_CNTXT_SV__		/**

API — uvma_rvfi_constants.sv

Table 1856. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_constants.sv — localparam (24개)

상수	형/폭	값
ORDER_WL		64
MODE_WL		2
IXL_WL		2
TRAP_WL		12
GPR_ADDR_WL		5
RVFI_DBG_WL		3
RVFI_NMIP_WL		2
RVFI_INTR_WL		16
CYCLE_CNT_WL		32
TRAP_EXCP_LSB		0
TRAP_EXCP_WL		1
TRAP_NONDBG_ENTRY_LSB		1
TRAP_NONDBG_ENTRY_WL		1
TRAP_DBG_ENTRY_LSB		2
TRAP_DBG_ENTRY_WL		1
TRAP_CAUSE_LSB		3

상수	형/폭	값
TRAP_CAUSE_WL		6
TRAP_DBG_CAUSE_LSB		9
TRAP_DBG_CAUSE_WL		3
DEFAULT_ILEN		32
DEFAULT_XLEN		32
MAX_XLEN		64
DEFAULT_NRET		1
CSR_MAX_SIZE		4096

Table 1857. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_constants.sv — #define / `define 매크로 (2개)

매크로	인자	정의
__UVMA_RVFI_CONSTANTS_SV__		// RVFI field widths
FORMAT_INSTR_STR_MACRO		"%8s RVFI %8d %6d %8x %8s %s x%-2d %8x x%-2d %8x x%-2d %16x %s"

API — uvma_rvfi_csr_if.sv

Table 1858. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_csr_if.sv — module / interface / package (2개)

이름	종류
uvma_rvfi_csr_if	interface
uvma_rvfi_unified_csr_if	interface

Table 1859. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_csr_if.sv — 포트 (19개)

포트	방향	형/폭
clk	input	
reset_n	input	
rvfi_csr_rmask	input	[XLEN-1:0]
rvfi_csr_wmask	input	[XLEN-1:0]
rvfi_csr_rdata	input	[XLEN-1:0]
rvfi_csr_wdata	input	[XLEN-1:0]
1step	input	#
clk	input	
reset_n	input	
rvfi_named_csr_rmask	input	[MAX_CSR-1:0][XLEN-1:0]
rvfi_named_csr_wmask	input	[MAX_CSR-1:0][XLEN-1:0]
rvfi_named_csr_rdata	input	[MAX_CSR-1:0][XLEN-1:0]

포트	방향	형/폭
rvfi_named_csr_wdata	input	[MAX_CSR-1:0][XLEN-1:0]
rvfi_csr_addr	input	[XLEN-1:0]
rvfi_csr_rmask	input	[XLEN-1:0]
rvfi_csr_wmask	input	[XLEN-1:0]
rvfi_csr_rdata	input	[XLEN-1:0]
rvfi_csr_wdata	input	[XLEN-1:0]
1step	input	#

Table 1860. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_csr_if.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMA_RVFI_CSR_IF_SV__		/**

API — uvma_rvfi_instr_if.sv

Table 1861. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_instr_if.sv — module / interface / package (1개)

이름	종류
uvma_rvfi_instr_if	interface

Table 1862. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_instr_if.sv — 포트 (31개)

포트	방향	형/폭
clk	input	
reset_n	input	
rvfi_valid	input	
rvfi_order	input	[ORDER_WL-1:0]
rvfi_insn	input	[ILEN-1:0]
rvfi_trap	input	[TRAP_WL-1:0]
rvfi_halt	input	
rvfi_dbg	input	[RVFI_DBG_WL-1:0]
rvfi_dbg_mode	input	
rvfi_nmip	input	[RVFI_NMIP_WL-1:0]
rvfi_intr	input	[RVFI_INTR_WL-1:0]
rvfi_mode	input	[MODE_WL-1:0]
rvfi_ixl	input	[IXL_WL-1:0]
rvfi_pc_rdata	input	[XLEN-1:0]
rvfi_pc_wdata	input	[XLEN-1:0]
rvfi_rs1_addr	input	[GPR_ADDR_WL-1:0]

포트	방향	형/폭
rvfi_rs1_rdata	input	[XLEN-1:0]
rvfi_rs2_addr	input	[GPR_ADDR_WL-1:0]
rvfi_rs2_rdata	input	[XLEN-1:0]
rvfi_rs3_addr	input	[GPR_ADDR_WL-1:0]
rvfi_rs3_rdata	input	[XLEN-1:0]
rvfi_rd1_addr	input	[GPR_ADDR_WL-1:0]
rvfi_rd1_wdata	input	[XLEN-1:0]
rvfi_rd2_addr	input	[GPR_ADDR_WL-1:0]
rvfi_rd2_wdata	input	[XLEN-1:0]
rvfi_mem_addr	input	[XLEN-1:0]
rvfi_mem_rdata	input	[XLEN-1:0]
rvfi_mem_rmask	input	[XLEN/8-1:0]
rvfi_mem_wdata	input	[XLEN-1:0]
rvfi_mem_wmask	input	[XLEN/8-1:0]
1step	input	#

Table 1863. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_instr_if.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMA_RVFI_INSTR_IF_SV__		/**

API — uvma_rvfi_instr_mon.sv

Table 1864. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_instr_mon.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMA_RVFI_INSTR_MON_SV__		/**

API — uvma_rvfi_macros.sv

Table 1865. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_macros.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMA_RVFI_MACROS_SV__		`endif // __UVMA_RVFI_MACROS_SV__

API — uvma_rvfi_mon_trn_logger.sv

Table 1866. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_mon_trn_logger.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMA_RVFI_MON_TRN_LOGGER_SV__		/**

API — uvma_rvfi_pkg.sv**Table 1867. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_pkg.sv — module / interface / package (1개)**

이름	종류
uvma_rvfi_pkg	package

Table 1868. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_pkg.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMA_RVFI_PKG_SV__		// Pre-processor macros

API — uvma_rvfi_tdefs.sv**Table 1869. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_tdefs.sv — parameter (1개)**

파라미터	형/폭	기본값
ST_NUM_WORDS		(\$size(st_rvfi)/MAX_XLEN)

Table 1870. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_tdefs.sv — typedef / using (1개)

새 이름	원래 형
vector_rvfi	bit [ST_NUM_WORDS-1:0] [MAX_XLEN-1:0]

Table 1871. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_tdefs.sv — #define / `define 매크로 (2개)

매크로	인자	정의
__UVMA_RVFI_TDEFS_SV__		typedef enum bit[MODE_WL-1:0] {
ST_NUM_WORDS		(\$size(st_rvfi)/MAX_XLEN)

API — uvma_rvfi_csr_seq_item.sv**Table 1872. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/seq/uvma_rvfi_csr_seq_item.sv — #define / `define 매크로 (1개)**

매크로	인자	정의
__UVMA_RVFI_CSR_SEQ_ITEM_SV__		/**

1. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_agent.sv (239 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 // Copyright 2020 Silicon Labs, Inc.
5 //
6 // Licensed under the Solderpad Hardware Licence, Version 2.0
7 // (the "License");
8 // you may not use this file except in compliance with the Li-
9 // cense.
10 // You may obtain a copy of the License at
11 // https://solderpad.org/licenses/
12 // Unless required by applicable law or agreed to in writing,
13 // software distributed under the License is distributed on an "AS IS"
14 // BASIS,
15 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-

```

```

press or implied.
15 // See the License for the specific language governing permis-
16 // sions and
17 // limitations under the License.
18 //
19 //
20 `ifndef __UVMA_RVFI_AGENT_SV__
21 `define __UVMA_RVFI_AGENT_SV__
22
23 /**
24 * Top-level component that encapsulates, builds and connects
25 * all others.
26 * Capable of driving/monitoring Clock & Reset interface.
27 */
28 class uvma_rvfi_agent_c#(int ILEN=DEFAULT_ILEN,
29                          int XLEN=DEFAULT_XLEN) extends uvm_a-
30 gent;
31
32 // Objects
33 uvma_rvfi_cfg_c#(ILEN,XLEN) cfg;
34 uvma_rvfi_cntxt_c#(ILEN,XLEN) cntxt;

```

```

34 // Components
35 uvma_rvfi_instr_mon_c#(ILEN,XLEN) in-
str_monitor;
36 uvma_rvfi_mon_trn_logger_c#(ILEN,XLEN)
mon_trn_logger;
37
38 uvm_analysis_port#(uvma_rvfi_instr_seq_item_c#(ILEN,XLEN))
rvfi_core_ap;
39
40 `uvm_component_param_utils_begin(uvma_rvfi_agent_c)
41 `uvm_field_object(cfg, UVM_DEFAULT)
42 `uvm_field_object(cntxt, UVM_DEFAULT)
43 `uvm_component_utils_end
44
45 /**
46 * Default constructor.
47 */
48 extern function new(string name="uvma_rvfi_agent", uvm_com-
ponent parent=null);
49
50 /**
51 * 1. Ensures cfg & cntxt handles are not null
52 * 2. Builds all components
53 */
54 extern virtual function void build_phase(uvm_phase phase);
55
56 /**
57 * 1. Links agent's analysis ports to sub-components'
58 * 2. Connects coverage models and loggers
59 */
60 extern virtual function void connect_phase(uvm_phase
phase);
61
62 /**
63 * Uses uvm_config_db to retrieve cfg and hand out to sub-
components.
64 */
65 extern function void get_and_set_cfg();
66
67 /**
68 * Uses uvm_config_db to retrieve cntxt and hand out to
sub-components.
69 */
70 extern function void get_and_set_cntxt();
71
72 /**
73 * Uses uvm_config_db to retrieve the Virtual Interface
(vif) associated with this
74 * agent.
75 */
76 extern function void retrieve_vif();
77
78 /**
79 * Creates sub-components.
80 */
81 extern function void create_components();
82
83 /**
84 * Connects sequencer and driver's TLM port(s).
85 */
86 extern function void connect_sequencer_and_driver();
87
88 /**
89 * Connects agent's TLM ports to driver's and monitor's.
90 */
91 extern function void connect_analysis_ports();
92
93 /**
94 * Connects transaction loggers to monitor and driver's
analysis ports.
95 */
96 extern function void connect_trn_loggers();
97
98 endclass : uvma_rvfi_agent_c
99
100
101 function uvma_rvfi_agent_c::new(string name="uvma_rvfi_agent",
uvm_component parent=null);
102
103 super.new(name, parent);
104
105 endfunction : new
106
107
108 function void uvma_rvfi_agent_c::build_phase(uvm_phase phase);
109
110 super.build_phase(phase);
111
112 get_and_set_cfg ();
113 get_and_set_cntxt();
114 retrieve_vif ();
115 create_components();
116
117 endfunction : build_phase
118
119
120 function void uvma_rvfi_agent_c::connect_phase(uvm_phase
phase);
121
122 super.connect_phase(phase);

```

```

123
124 connect_sequencer_and_driver();
125 connect_analysis_ports();
126
127 if (cfg.trn_log_enabled) begin
128 connect_trn_loggers();
129 end
130
131 endfunction: connect_phase
132
133
134 function void uvma_rvfi_agent_c::get_and_set_cfg();
135
136 void'(uvm-con-
fig_db#(uvma_rvfi_cfg_c#(ILEN,XLEN))::get(this, "", "cfg", cfg));
137 if (!cfg) begin
138 `uvm_fatal("CFG", "Configuration handle is null")
139 end
140 else begin
141 `uvm_info("CFG", $sformatf("Found configuration handle:
\n%s", cfg.sprint()), UVM_DEBUG)
142 uvm_config_db#(uvma_rvfi_cfg_c#(ILEN,XLEN))::set(this,
"", "cfg", cfg);
143 end
144
145 endfunction : get_and_set_cfg
146
147
148 function void uvma_rvfi_agent_c::get_and_set_cntxt();
149
150 void'(uvm-con-
fig_db#(uvma_rvfi_cntxt_c#(ILEN,XLEN))::get(this, "", "cntxt",
cntxt));
151 if (!cntxt) begin
152 `uvm_info("CNTXT", "Context handle is null; creating.",
UVM_DEBUG)
153 cntxt = uvma_r-
vfi_cntxt_c#(ILEN,XLEN)::type_id::create("cntxt");
154 end
155 uvm_config_db#(uvma_rvfi_cntxt_c#(ILEN,XLEN))::set(this,
"", "cntxt", cntxt);
156
157 endfunction : get_and_set_cntxt
158
159
160 function void uvma_rvfi_agent_c::retrieve_vif();
161
162 // Retrieve instruction interface
163 cntxt.instr_vif = new[cfg.nret];
164 for (int i = 0; i < cfg.nret; i++) begin
165 if (!uvm_config_db#(virtual uvma_r-
vfi_instr_if#(ILEN,XLEN))::get(this, "", $sformatf("instr_vif%d",
i), cntxt.instr_vif[i])) begin
166 `uvm_fatal("VIF", $sformatf("Could not find vif
handle of type %s in uvm_config_db",
167 $styp-
name(cntxt.instr_vif[i])))
168 end
169 else begin
170 `uvm_info("VIF", $sformatf("Found vif handle of type
%s in uvm_config_db",
171 $styp-
name(cntxt.instr_vif[i])), UVM_DEBUG)
172 end
173 end
174
175 // Create virtual interface and fetch virtual interface for
each supported CSR
176 if (!cfg.core_cfg.disable_all_csr_checks) begin
177 string csrs[$];
178
179 cfg.core_cfg.get_supported_csrs_names(csrs);
180 if (!cfg.unified_csr_vif) begin
181 foreach (csrs[c]) begin
182 string csr = csrs[c].tolower();
183 cntxt.csr_vif[csr] = new[cfg.nret];
184
185 for (int i = 0; i < cfg.nret; i++) begin
186 if (!uvm_config_db#(virtual uvma_r-
vfi_csr_if#(XLEN))::get(this, "", $sformatf("csr_%s_vif%d", csr,
i), cntxt.csr_vif[csr][i])) begin
187 `uvm_fatal("VIF", $sformatf("Could not find
vif handle of type %s, csr [%s] in uvm_config_db",
188 $styp-
name(cntxt.csr_vif[csr][i]), csr))
189 end else begin
190 `uvm_info("VIF", $sformatf("Found vif handle
of type %s, csr [%s] in uvm_config_db",
191 $styp-
name(cntxt.csr_vif[csr][i]), csr), UVM_DEBUG)
192 end
193 end
194 end
195 end
196 else begin
197 cntxt.csr_unified_vif = new[cfg.nret];
198 for (int i = 0; i < cfg.nret; i++) begin
199 if (!uvm_config_db#(virtual uvma_r-
vfi_unified_csr_if#(4096, XLEN))::get(this, "", $sform-

```

```

atf("csr_vif%0d", i), cntxt.csr_unified_vif[i])) begin
200     `uvm_fatal("VIF", $sformatf("Could not find vif
handle of type %s, csr_vif in uvm_config_db",
201                                     $type-
name(cntxt.csr_unified_vif[i]))
202     end else begin
203         `uvm_info("VIF", $sformatf("Found vif handle of
type %s, csr_vif in uvm_config_db",
204                                     $type-
name(cntxt.csr_unified_vif[i])), UVM_DEBUG)
205     end
206     end
207     end
208     end
209 endfunction : retrieve_vif
210
211
212
213 function void uvma_rvfi_agent_c::create_components();
214
215     instr_monitor = uvma_r-
vfi_instr_mon_c#(ILEN,XLEN)::type_id::create("instr_monitor", this);
216     mon_trn_logger = uvma_r-
vfi_mon_trn_logger_c#(ILEN,XLEN)::type_id::create("mon_trn_logger" ,
this);
217 endfunction : create_components
218
219
220
221 function void uvma_r-
vfi_agent_c::connect_sequencer_and_driver();
222
223 endfunction : connect_sequencer_and_driver
224
225
226 function void uvma_rvfi_agent_c::connect_analysis_ports();
227
228     rvfi_core_ap = instr_monitor.ap;
229
230 endfunction : connect_analysis_ports
231
232
233 function void uvma_rvfi_agent_c::connect_trn_loggers();
234
235     rvfi_core_ap.connect(mon_trn_logger.instr_export);
236
237 endfunction : connect_trn_loggers
238
239 `endif // __UVMA_RVFI_AGENT_SV__

```

2. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_assert.sv (133 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 // Copyright 2020 Silicon Labs, Inc.
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
6 // you may not use this file except in compliance with the Li-
cense.
7 // You may obtain a copy of the License at
8 //
9 //     https://solderpad.org/licenses/
10 //
11 // Unless required by applicable law or agreed to in writing,
software
12 // distributed under the License is distributed on an "AS IS"
BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14 // See the License for the specific language governing permis-
sions and
15 // limitations under the License.
16
17 module uvma_rvfi_assert
18     import uvm_pkg::*;
19     #(
20         parameter int ADDR_WIDTH=32,
21         parameter int DATA_WIDTH=32
22     )
23     (
24         input                clk,
25         input                reset_n,
26
27         input                req,
28         input                gnt,
29         input [ADDR_WIDTH-1:0] addr,
30         input                we,
31         input [DATA_WIDTH/8-1:0] be,
32         input [DATA_WIDTH-1:0] wdata,
33         input [DATA_WIDTH-1:0] rdata,
34         input                rvalid,
35         input                rready
36     );
37
38 //
-----
39 // Local variables

```

```

40 //
-----
41 string info_tag = "CV32E40P_RVFI_ASSERT";
42
43 //
-----
44 // Clocking blocks
45 //
-----
46
47 // Single clock, single reset design, use default clocking
48 default clocking @(posedge clk); endclocking
49 default disable iff !(reset_n);
50
51 //
-----
52 // Begin module code
53 //
-----
54
55 // R-3.1.1 : Address phase signals stable during address
phase
56 property p_addr_signal_stable(sig);
57 req ##0 !gnt | => $stable(sig);
58 endproperty : p_addr_signal_stable
59
60 a_addr_stable: assert property(p_addr_signal_stable(addr))
61 else
62     `uvm_error(info_tag, "addr signal not stable in address
phase")
63
64 a_we_stable: assert property(p_addr_signal_stable(we))
65 else
66     `uvm_error(info_tag, "we signal not stable in address
phase")
67
68 a_wdata_stable: assert property(p_addr_signal_stable(wdata))
69 else
70     `uvm_error(info_tag, "wdata signal not stable in address
phase")
71
72 a_be_stable: assert property(p_addr_signal_stable(be))
73 else
74     `uvm_error(info_tag, "be signal not stable in address
phase")
75
76 // R-3.1.2 : Req may not deassert until the gnt is asserted
77 property p_req_until_gnt;
78 req ##0 !gnt | => req;
79 endproperty : p_req_until_gnt
80 a_req_until_gnt : assert property(p_req_until_gnt)
81 else
82     `uvm_error(info_tag, "req may not deassert until gnt as-
serted")
83
84 // R-7 At least one byte enable must be set
85 property p_be_not_zero;
86 req |-> be != 0;
87 endproperty : p_be_not_zero
88 a_be_not_zero : assert property(p_be_not_zero)
89 else
90     `uvm_error(info_tag, "be was zero during an address
cycle")
91
92 // R-7 All ones must be contiguous in writes
93 reg[3:0] contiguous_be[] = {
94     4'b0001,
95     4'b0011,
96     4'b0111,
97     4'b1111,
98     4'b0010,
99     4'b0110,
100    4'b1110,
101    4'b0100,
102    4'b1100,
103    4'b1000
104 };
105 bit be_inside_contiguous_be;
106 always_comb begin
107     be_inside_contiguous_be = be inside {contiguous_be};
108 end
109 property p_be_contiguous;
110 req |-> be_inside_contiguous_be;
111 endproperty : p_be_contiguous
112 a_be_contiguous : assert property(p_be_contiguous)
113 else
114     `uvm_error(info_tag, $sformatf("be of 0x%x was not con-
tiguous", $sampled(be)));
115
116 // R-8 Data address LSBs must be consistent with byte en-
ables on writes
117 function bit [1:0] get_addr_lsb(bit[3:0] be);
118     casex (be)
119         4'b???1: return 0;
120         4'b??10: return 1;
121         4'b?100: return 2;

```

```

122     4'b1000: return 3;
123   endcase
124 endfunction : get_addr_lsb
125
126 property p_addr_be_consistent;
127   req |-> addr[1:0] == get_addr_lsb(be);
128 endproperty : p_addr_be_consistent
129 a_addr_be_consistent: assert property(p_addr_be_consistent)
130 else
131   `uvm_error(info_tag, $sformatf("be of 0x%01x not consist-
132   ent with addr 0x%08x", $sampled(be), $sampled(addr)));
133 endmodule : uvma_rvfi_assert

```

3. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_cfg.sv (116 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 // Copyright 2020 Silicon Labs, Inc.
5 //
6 // Licensed under the Solderpad Hardware Licence, Version 2.0
7 // (the "License");
8 // you may not use this file except in compliance with the Li-
9 // cense.
10 // You may obtain a copy of the License at
11 //
12 // https://solderpad.org/licenses/
13 // Unless required by applicable law or agreed to in writing,
14 // software
15 // distributed under the License is distributed on an "AS IS"
16 // BASIS,
17 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
18 // press or implied.
19 // See the License for the specific language governing permis-
20 // sions and
21 // limitations under the License.
22 //
23
24 `ifndef __UVMA_RVFI_CFG_SV__
25 `define __UVMA_RVFI_CFG_SV__
26
27 /**
28  * Object encapsulating all parameters for creating, connect-
29  * ing and running all
30  * Clock & Reset agent (uvma_rvfi_agent_c) components.
31  */
32 class uvma_rvfi_cfg_c#(int ILEN=DEFAULT_ILEN,
33   int XLEN=DEFAULT_XLEN) extends uvm_ob-
34   ject;
35
36   // Core configuration (used to extract list of CSRs)
37   uvma_core_cntrl_cfg_c core_cfg;
38
39   // Common options
40   rand bit enabled;
41   rand uvm_active_passive_enum is_active;
42
43   rand bit unified_csr_vif;
44   rand bit cov_model_enabled;
45   rand bit trn_log_enabled;
46
47   // Number of instructions that may be retired in a single
48   cycle
49   // This number cannot be zero
50   rand int unsigned nret;
51
52   // Configuration of NMI faults
53   rand bit nmi_load_fault_enabled;
54   rand int unsigned nmi_load_fault_cause;
55   rand bit nmi_store_fault_enabled;
56   rand int unsigned nmi_store_fault_cause;
57
58   // Enable bus interrupt fault bit
59   rand bit insn_bus_fault_enabled;
60   rand int unsigned insn_bus_fault_cause;
61
62   // Name for the instruction retirement ports (nret of these)
63   string instr_name[int];
64
65   // Merge exceptions with the first instruction after excep-
66   tion
67   `uvm_object_utils_begin(uvma_rvfi_cfg_c)
68   `uvm_field_int ( , UVM_DEFAULT)
69   `uvm_field_int (nret , UVM_DEFAULT)
70   `uvm_field_enum(uvm_active_passive_enum, is_act-
71   ive , UVM_DEFAULT)
72   `uvm_field_int (cov_mod-
73   el_enabled , UVM_DEFAULT)
74   `uvm_field_int (trn_log_en-

```

```

abled , UVM_DEFAULT)
69   `uvm_field_int
70   (nmi_load_fault_enabled , UVM_DEFAULT)
71   `uvm_field_int
72   (nmi_load_fault_cause , UVM_DEFAULT)
73   `uvm_field_int
74   (nmi_store_fault_enabled , UVM_DEFAULT)
75   `uvm_field_int
76   (nmi_store_fault_cause , UVM_DEFAULT)
77   `uvm_field_int
78   (insn_bus_fault_enabled , UVM_DEFAULT)
79   `uvm_field_int
80   (insn_bus_fault_cause , UVM_DEFAULT)
81   `uvm_object_utils_end
82
83   constraint valid_nret {
84     nret != 0;
85   }
86
87   constraint valid_active_passive {
88     // Only designed to support passive mode
89     is_active == UVM_PASSIVE;
90   }
91
92   constraint defaults_cons {
93     soft enabled == 1;
94     soft is_active == UVM_PASSIVE;
95     soft cov_model_enabled == 0;
96     soft trn_log_enabled == 1;
97     soft nret == 1;
98     soft nmi_load_fault_enabled == 0;
99     soft nmi_load_fault_cause == 0;
100    soft nmi_store_fault_enabled == 0;
101    soft nmi_store_fault_cause == 0;
102    soft insn_bus_fault_enabled == 0;
103    soft insn_bus_fault_cause == 0;
104    soft unified_csr_vif == 0;
105  }
106
107   /**
108    * Default constructor.
109    */
110   extern function new(string name="uvma_rvfi_cfg");
111
112   endclass : uvma_rvfi_cfg_c
113
114   function uvma_rvfi_cfg_c::new(string name="uvma_rvfi_cfg");
115   super.new(name);
116   endfunction : new
117
118   `endif // __UVMA_RVFI_CFG_SV__
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

4. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_cntxt.sv (78 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 // Copyright 2020 Silicon Labs, Inc.
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
6 // (the "License");
7 // you may not use this file except in compliance with the Li-
8 // cense.
9 // You may obtain a copy of the License at
10 //
11 // https://solderpad.org/licenses/
12 // Unless required by applicable law or agreed to in writing,
13 // software
14 // distributed under the License is distributed on an "AS IS"
15 // BASIS,
16 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
17 // press or implied.
18 // See the License for the specific language governing permis-
19 // sions and
20 // limitations under the License.
21 //
22 `ifndef __UVMA_RVFI_CNTXT_SV__
23 `define __UVMA_RVFI_CNTXT_SV__
24
25 /**
26  * Object encapsulating all state variables for all Rvfi agent
27  * (uvma_rvfi_agent_c) components.
28  */
29 class uvma_rvfi_cntxt_c#(int ILEN=DEFAULT_ILEN,
30   int XLEN=DEFAULT_XLEN) extends
31   uvm_object;
32
33   // Handle to instruction retirement interface
34   // The number of interfaces will be equal to the _nret_
35   value in the configuration object
36   virtual uvma_rvfi_instr_if#(ILEN,XLEN) instr_vif[];
37
38   // Handle to CSR interfaces
39   virtual uvma_rvfi_csr_if#(XLEN) csr_vif[string][];

```



```

34 virtual uvma_rvfi_unified_csr_if#(4096,XLEN) csr_uni-
fied_vif[];
35
36 // Events
37 uvma_event sample_cfg_e;
38 uvma_event sample_cntxt_e;
39
40 `uvvm_object_utils_begin(uvma_rvfi_cntxt_c)
41 `uvvm_field_event(sample_cfg_e, UVM_DEFAULT)
42 `uvvm_field_event(sample_cntxt_e, UVM_DEFAULT)
43 `uvvm_object_utils_end
44
45 /**
46  * Builds events.
47  */
48 extern function new(string name="uvma_rvfi_cntxt");
49
50 /**
51  * TODO Describe uvma_rvfi_cntxt_c::reset()
52  */
53 extern function void reset();
54
55 endclass : uvma_rvfi_cntxt_c
56
57
58 `pragma protect begin
59
60
61 function uvma_rvfi_cntxt_c::new(string name="uvma-r-
vfi_cntxt");
62
63     super.new(name);
64
65     sample_cfg_e = new("sample_cfg_e");
66     sample_cntxt_e = new("sample_cntxt_e");
67
68 endfunction : new
69
70 function void uvma_rvfi_cntxt_c::reset();
71
72 endfunction : reset
73
74
75 `pragma protect end
76
77
78 `endif // __UVMA_RVFI_CNTXT_SV__

```

5. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_constants.sv (56 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 // Copyright 2020 Silicon Labs, Inc.
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
6 // (the "License");
7 // you may not use this file except in compliance with the Li-
8 // cense.
9 // You may obtain a copy of the License at
10 //
11 // https://solderpad.org/licenses/
12 // Unless required by applicable law or agreed to in writing,
13 // distributed under the License is distributed on an "AS IS"
14 // BASIS,
15 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
16 // press or implied.
17 // See the License for the specific language governing permis-
18 // sions and
19 // limitations under the License.
20
21 `ifndef __UVMA_RVFI_CONSTANTS_SV__
22 `define __UVMA_RVFI_CONSTANTS_SV__
23
24 // RVFI field widths
25 localparam ORDER_WL = 64;
26 localparam MODE_WL = 2;
27 localparam IXL_WL = 2;
28 localparam TRAP_WL = 12;
29 localparam GPR_ADDR_WL = 5;
30 localparam RVFI_DBG_WL = 3;
31 localparam RVFI_NMIP_WL = 2;
32 localparam RVFI_INTR_WL = 16;
33 localparam CYCLE_CNT_WL = 32;
34
35 // Fields within TRAP
36 localparam TRAP_EXCP_LSB = 0;
37 localparam TRAP_EXCP_WL = 1;
38 localparam TRAP_NONDBG_ENTRY_LSB = 1;
39 localparam TRAP_NONDBG_ENTRY_WL = 1;
40 localparam TRAP_DBG_ENTRY_LSB = 2;
41 localparam TRAP_DBG_ENTRY_WL = 1;
42 localparam TRAP_CAUSE_LSB = 3;
43 localparam TRAP_CAUSE_WL = 6;
44 localparam TRAP_DBG_CAUSE_LSB = 9;
45 localparam TRAP_DBG_CAUSE_WL = 3;
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

```

44 localparam DEFAULT_ILEN = 32;
45 localparam DEFAULT_XLEN = 32;
46 localparam MAX_XLEN = 64;
47 localparam DEFAULT_NRET = 1;
48
49 localparam CSR_MAX_SIZE = 4096;
50
51 const string format_header_str = "%8s | RVFI | %8s | %6s | %8s |
%8s | %s | %03s | %08s | %03s | %08s | %03s | %08s |
%08s | %s";
52
53 const string format_instr_str = "%8s | RVFI | %8d | %6d | %8x |
%8s | %s | x%-2d | %8x | x%-2d | %8x | x%-2d | %16x";
54
55 `define FORMAT_INSTR_STR_MACRO "%8s | RVFI | %8d | %6d | %8x |
%8s | %s | x%-2d | %8x | x%-2d | %8x | x%-2d | %16x | %s"
56 const string format_mem_str = "| %02s | %08x | %08s |";
57
58 `endif // __UVMA_RVFI_CONSTANTS_SV__

```

6. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_csr_if.sv (123 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 // Copyright 2020 Silicon Labs, Inc.
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
6 // (the "License");
7 // you may not use this file except in compliance with the Li-
8 // cense.
9 // You may obtain a copy of the License at
10 //
11 // https://solderpad.org/licenses/
12 // Unless required by applicable law or agreed to in writing,
13 // distributed under the License is distributed on an "AS IS"
14 // BASIS,
15 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
16 // press or implied.
17 // See the License for the specific language governing permis-
18 // sions and
19 // limitations under the License.
20
21 `ifndef __UVMA_RVFI_CSR_IF_SV__
22 `define __UVMA_RVFI_CSR_IF_SV__
23
24 /**
25  * Encapsulates all signals and clocking of RVFI csrruction in-
26  * terface. Used by
27  * monitor,
28  */
29 interface uvma_rvfi_csr_if
30     import uvma_rvfi_pkg::*;
31     #(int XLEN=DEFAULT_XLEN)
32     (
33         input clk,
34         input reset_n,
35         input [XLEN-1:0] rvfi_csr_rmask,
36         input [XLEN-1:0] rvfi_csr_wmask,
37         input [XLEN-1:0] rvfi_csr_rdata,
38         input [XLEN-1:0] rvfi_csr_wdata
39     );
40
41 // Local variables
42
43 // Begin module code
44
45
46 /**
47  * Used by target DUT.
48  */
49 clocking dut_cb @(posedge clk or reset_n);
50 endclocking : dut_cb
51
52 /**
53  * Used by uvma_rvfi_csr_mon.c.
54  */
55 clocking mon_cb @(posedge clk or reset_n);
56 input #1step
57     rvfi_csr_rmask,
58     rvfi_csr_wmask,
59     rvfi_csr_rdata,
60     rvfi_csr_wdata;
61 endclocking : mon_cb
62
63 modport passive_mp (clocking mon_cb);
64
65 endinterface : uvma_rvfi_csr_if

```

```

66
67 interface uvma_rvfi_unified_csr_if
68   import uvma_rvfi_pkg::*;
69   #( int MAX_CSR=4096,
70     int XLEN=DEFAULT_XLEN)
71   (
72     input          clk,
73     input          reset_n,
74
75     input [MAX_CSR-1:0][XLEN-1:0] rvfi_named_csr_rmask,
76     input [MAX_CSR-1:0][XLEN-1:0] rvfi_named_csr_wmask,
77     input [MAX_CSR-1:0][XLEN-1:0] rvfi_named_csr_rdata,
78     input [MAX_CSR-1:0][XLEN-1:0] rvfi_named_csr_wdata,
79
80     // Generic READ/WRITE values
81     input [XLEN-1:0] rvfi_csr_addr,
82     input [XLEN-1:0] rvfi_csr_rmask,
83     input [XLEN-1:0] rvfi_csr_wmask,
84     input [XLEN-1:0] rvfi_csr_rdata,
85     input [XLEN-1:0] rvfi_csr_wdata,
86   );
87
88   //
89   // Local variables
90   //
91
92   //
93   // Begin module code
94   //
95
96   /**
97    * Used by target DUT.
98    */
99   clocking dut_cb @(posedge clk or reset_n);
100  endclocking : dut_cb
101
102   /**
103    * Used by uvma_rvfi_csr_mon.c.
104    */
105   clocking mon_cb @(posedge clk or reset_n);
106   input #1step
107     rvfi_csr_addr,
108     rvfi_csr_rmask,
109     rvfi_csr_wmask,
110     rvfi_csr_rdata,
111     rvfi_csr_wdata,
112     rvfi_named_csr_rmask,
113     rvfi_named_csr_wmask,
114     rvfi_named_csr_rdata,
115     rvfi_named_csr_wdata;
116  endclocking : mon_cb
117
118  modport passive_mp (clocking mon_cb);
119
120  endinterface : uvma_rvfi_unified_csr_if
121
122
123  `endif // __UVMA_RVFI_CSR_IF_SV__

```

7. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_instr_if.sv (132 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 // Copyright 2020 Silicon Labs, Inc.
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
6 // (the "License");
7 // you may not use this file except in compliance with the Li-
8 // cense.
9 // You may obtain a copy of the License at
10 // https://solderpad.org/licenses/
11 // Unless required by applicable law or agreed to in writing,
12 // software
13 // distributed under the License is distributed on an "AS IS"
14 // BASIS,
15 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
16 // press or implied.
17 // See the License for the specific language governing permis-
18 // sions and
19 // limitations under the License.
20
21 `ifndef __UVMA_RVFI_INSTR_IF_SV__
22 `define __UVMA_RVFI_INSTR_IF_SV__
23
24 /**
25  * Encapsulates all signals and clocking of RVFI Instruction
26  * interface. Used by
27  * monitor,
28  */
29 interface uvma_rvfi_instr_if
30   import uvma_rvfi_pkg::*;
31   #(int ILEN=DEFAULT_ILEN,

```

```

28   int XLEN=DEFAULT_XLEN)
29   //import uvma_rvfi_pkg::*;
30   (
31     input          clk,
32     input          reset_n,
33
34     input          rvfi_valid,
35     input [ORDER_WL-1:0] rvfi_order,
36     input [ILEN-1:0] rvfi_insn,
37     input [TRAP_WL-1:0] rvfi_trap,
38     input          rvfi_halt,
39     input [RVFI_DBG_WL-1:0] rvfi_dbg,
40     input          rvfi_dbg_mode,
41     input [RVFI_NMIP_WL-1:0] rvfi_nmip,
42     input [RVFI_INTR_WL-1:0] rvfi_intr,
43     input [MODE_WL-1:0] rvfi_mode,
44     input [IXL_WL-1:0] rvfi_ixl,
45     input [XLEN-1:0] rvfi_pc_rdata,
46     input [XLEN-1:0] rvfi_pc_wdata,
47
48     input [GPR_ADDR_WL-1:0] rvfi_rs1_addr,
49     input [XLEN-1:0] rvfi_rs1_rdata,
50
51     input [GPR_ADDR_WL-1:0] rvfi_rs2_addr,
52     input [XLEN-1:0] rvfi_rs2_rdata,
53
54     input [GPR_ADDR_WL-1:0] rvfi_rs3_addr,
55     input [XLEN-1:0] rvfi_rs3_rdata,
56
57     input [GPR_ADDR_WL-1:0] rvfi_rd1_addr,
58     input [XLEN-1:0] rvfi_rd1_wdata,
59
60     input [GPR_ADDR_WL-1:0] rvfi_rd2_addr,
61     input [XLEN-1:0] rvfi_rd2_wdata,
62
63     input [XLEN-1:0] rvfi_mem_addr,
64     input [XLEN-1:0] rvfi_mem_rdata,
65     input [XLEN/8-1:0] rvfi_mem_rmask,
66     input [XLEN-1:0] rvfi_mem_wdata,
67     input [XLEN/8-1:0] rvfi_mem_wmask,
68
69   );
70
71   //
72   // Local variables
73   //
74
75   bit [CYCLE_CNT_WL-1:0] cycle_cnt = 0;
76
77   //
78   // Begin module code
79   //
80
81   always @(posedge clk) begin
82     cycle_cnt <= cycle_cnt + 1;
83   end
84
85   /**
86    * Used by target DUT.
87    */
88   clocking dut_cb @(posedge clk or reset_n);
89   endclocking : dut_cb
90
91   /**
92    * Used by uvma_rvfi_instr_mon.c.
93    */
94   clocking mon_cb @(posedge clk or reset_n);
95   input #1step
96     cycle_cnt,
97     rvfi_valid,
98     rvfi_order,
99     rvfi_insn,
100     rvfi_trap,
101     rvfi_halt,
102     rvfi_dbg,
103     rvfi_dbg_mode,
104     rvfi_nmip,
105     rvfi_intr,
106     rvfi_mode,
107     rvfi_ixl,
108     rvfi_pc_rdata,
109     rvfi_pc_wdata,
110     rvfi_rs1_addr,
111     rvfi_rs1_rdata,
112     rvfi_rs2_addr,
113     rvfi_rs2_rdata,
114     rvfi_rs3_addr,
115     rvfi_rs3_rdata,
116     rvfi_rd1_addr,
117     rvfi_rd1_wdata,
118     rvfi_rd2_addr,
119     rvfi_rd2_wdata,
120     rvfi_mem_addr,
121     rvfi_mem_rdata,
122     rvfi_mem_rmask,
123     rvfi_mem_wdata,
124     rvfi_mem_wmask;
125  endclocking : mon_cb

```

```

125
126 modport passive_mp    (clocking mon_cb);
127
128 endinterface : uvma_rvfi_instr_if
129
130
131 `endif // __UVMA_RVFI_INSTR_IF_SV__
132

```

8. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_instr_mon.sv (299 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 // Copyright 2020 Silicon Labs, Inc.
5 //
6 // Licensed under the Solderpad Hardware Licence, Version 2.0
7 // (the "License");
8 // you may not use this file except in compliance with the Li-
9 // cense.
10 // You may obtain a copy of the License at
11 //
12 // https://solderpad.org/licenses/
13 // Unless required by applicable law or agreed to in writing,
14 // software
15 // distributed under the License is distributed on an "AS IS"
16 // BASIS,
17 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
18 // press or implied.
19 // See the License for the specific language governing permis-
20 // sions and
21 // limitations under the License.
22 //
23
24 `ifndef __UVMA_RVFI_INSTR_MON_SV__
25 `define __UVMA_RVFI_INSTR_MON_SV__
26
27 /**
28 * Component sampling transactions from a Clock & Reset virtu-
29 al interface
30 * (uvma_rvfi_instr_if).
31 */
32 class uvma_rvfi_instr_mon_c#(int ILEN=DEFAULT_ILEN,
33                             int XLEN=DEFAULT_XLEN) extends
34 uvvm_monitor;
35
36 // Objects
37 uvma_rvfi_cfg_c    cfg;
38 uvma_rvfi_cntxt_c  cntxt;
39
40 // TLM
41 uvvm_analysis_port#(uvma_rvfi_instr_seq_item_c#(ILEN,XLEN))
42 ap;
43
44 // State of monitor
45 bit last_dcsr_nmip = 0;
46 bit [uvma_rvfi_csr_seq_item_c#(ILEN-1:0)] dcsr_ret_data;
47
48 longint unsigned supported_csrs_addrs [$];
49 string supported_csrs_names [$];
50
51 string log_tag = "RVFIMONLOG";
52
53 `uvm_component_utils_begin(uvma_rvfi_instr_mon_c)
54 `uvm_field_int(last_dcsr_nmip, UVM_DEFAULT)
55 `uvm_field_int(dcsr_ret_data, UVM_DEFAULT)
56 `uvm_field_object(cfg, UVM_DEFAULT | UVM_REFER-
57 ENCE)
58 `uvm_field_object(cntxt, UVM_DEFAULT | UVM_REFER-
59 ENCE)
60 `uvm_component_utils_end
61
62 /**
63 * Default constructor.
64 */
65 extern function new(string name="uvma_rvfi_instr_mon",
66 uvvm_component parent=null);
67
68 /**
69 * 1. Ensures cfg & cntxt handles are not null.
70 * 2. Builds ap.
71 */
72 extern virtual function void build_phase(uvm_phase phase);
73
74 /**
75 * Oversees monitoring via monitor_clk() and monit-
76 or_reset() tasks in parallel
77 * forks.
78 */
79 extern virtual task run_phase(uvm_phase phase);
80
81 /**
82 * Monitor the instruction interface
83 */
84 extern virtual task monitor_rvfi_instr();
85

```

```

76 endclass : uvma_rvfi_instr_mon_c
77
78 function uvma_rvfi_instr_mon_c::new(string name="uvma_r-
79 vfi_instr_mon", uvvm_component parent=null);
80
81 super.new(name, parent);
82
83 endfunction : new
84
85 function void uvma_rvfi_instr_mon_c::build_phase(uvm_phase
86 phase);
87
88 super.build_phase(phase);
89
90 if ($test$plusargs("log_rvfi"))
91     set_report_id_verbosity(log_tag, UVM_HIGH);
92
93 void'(uvm-con-
94 fig_db#(uvma_rvfi_cfg_c#(ILEN,XLEN))::get(this, "", "cfg", cfg));
95 if (!cfg) begin
96     `uvm_fatal("CFG", "Configuration handle is null")
97 end
98 void'(uvm-con-
99 fig_db#(uvma_rvfi_cntxt_c#(ILEN,XLEN))::get(this, "", "cntxt",
100 cntxt));
101 if (!cntxt) begin
102     `uvm_fatal("CNTXT", "Context handle is null")
103 end
104 ap = new("ap", this);
105
106 endfunction : build_phase
107
108 task uvma_rvfi_instr_mon_c::run_phase(uvm_phase phase);
109
110 super.run_phase(phase);
111
112 cfg.core_cfg.get_support-
113 ted_csrs_names(supported_csrs_names);
114 cfg.core_cfg.get_support-
115 ted_csrs_addrs(supported_csrs_addrs);
116
117 // TODO Make this prettier
118 if (cfg.enabled) begin
119     while (1) begin
120         wait (cntxt.instr_vif[0].reset_n === 1'b0);
121         wait (cntxt.instr_vif[0].reset_n === 1'b1);
122
123         fork
124             begin
125                 monitor_rvfi_instr();
126             end
127         join_any
128     end
129 endtask : run_phase
130
131 task uvma_rvfi_instr_mon_c::monitor_rvfi_instr();
132
133 while(1) begin
134     @(cntxt.instr_vif[0].mon_cb)
135
136     for(int i = 0; i < cfg.nret; i++)
137         begin
138             int nret_id;
139             bit monitor_condition;
140
141             nret_id = i;
142             monitor_condition = (!cfg.core_cfg.unified_traps) ?
143 (cntxt.instr_vif[nret_id].mon_cb.rvfi_valid ||
144 cntxt.instr_vif[nret_id].mon_cb.rvfi_trap[0]) :
145 (cntxt.instr_vif[nret_id].mon_cb.rvfi_valid);
146
147             if (monitor_condition) begin
148                 uvma_rvfi_instr_seq_item_c#(ILEN,XLEN) mon_trn;
149
150                 mon_trn = uvma_r-
151 vfi_instr_seq_item_c#(ILEN,XLEN)::type_id::create("mon_trn");
152
153                 mon_trn.nret_id = nret_id;
154                 mon_trn.cycle_cnt =
155 cntxt.instr_vif[nret_id].mon_cb.cycle_cnt;
156                 mon_trn.order =
157 cntxt.instr_vif[nret_id].mon_cb.rvfi_order;
158                 mon_trn.insn =
159 cntxt.instr_vif[nret_id].mon_cb.rvfi_insn;
160                 mon_trn.trap =
161 cntxt.instr_vif[nret_id].mon_cb.rvfi_trap;
162                 mon_trn.halt =
163 cntxt.instr_vif[nret_id].mon_cb.rvfi_halt;
164                 mon_trn.dbg =
165 cntxt.instr_vif[nret_id].mon_cb.rvfi_dbg;
166

```

```

159         mon_trn.dbg_mode =
cntxt.instr_vif[nret_id].mon_cb.rvfi_dbg_mode;
160         mon_trn.nmip =
cntxt.instr_vif[nret_id].mon_cb.rvfi_nmip;
161         mon_trn.intr =
cntxt.instr_vif[nret_id].mon_cb.rvfi_intr;
162         $cast(mon_trn.mode,
cntxt.instr_vif[nret_id].mon_cb.rvfi_mode);
163         mon_trn.ixl =
cntxt.instr_vif[nret_id].mon_cb.rvfi_ixl;
164         mon_trn.pc_rdata =
cntxt.instr_vif[nret_id].mon_cb.rvfi_pc_rdata;
165         mon_trn.pc_wdata =
cntxt.instr_vif[nret_id].mon_cb.rvfi_pc_wdata;
166         mon_trn.rs1_addr =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rs1_addr;
167         mon_trn.rs1_rdata =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rs1_rdata;
168         mon_trn.rs2_addr =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rs2_addr;
169         mon_trn.rs2_rdata =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rs2_rdata;
170         mon_trn.rs3_addr =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rs3_addr;
171         mon_trn.rs3_rdata =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rs3_rdata;
172         mon_trn.rd1_addr =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rd1_addr;
173         mon_trn.rd1_wdata =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rd1_wdata;
174         mon_trn.rd2_addr =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rd2_addr;
175         mon_trn.rd2_wdata =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rd2_wdata;
176         mon_trn.mem_addr =
cntxt.instr_vif[nret_id].mon_cb.rvfi_mem_addr;
177         mon_trn.mem_rdata =
cntxt.instr_vif[nret_id].mon_cb.rvfi_mem_rdata;
178         mon_trn.mem_rmask =
cntxt.instr_vif[nret_id].mon_cb.rvfi_mem_rmask;
179         mon_trn.mem_wdata =
cntxt.instr_vif[nret_id].mon_cb.rvfi_mem_wdata;
180         mon_trn.mem_wmask =
cntxt.instr_vif[nret_id].mon_cb.rvfi_mem_wmask;
181         // Get the CSRs
182         if (!cfg.core_cfg.disable_all_csr_checks) begin
183             if (!cfg.unified_csr_vif) begin
184                 foreach (cntxt.csr_vif[csr]) begin
185                     uvma_rvfi_csr_seq_item_c csr_trn =
uvma_rvfi_csr_seq_item_c#(XLEN)::type_id::create({csr, "_trn"});
186                     csr_trn.csr = csr;
187                     csr_trn.nret_id = nret_id;
188                     csr_trn.rmask = cntxt.csr_vif[csr]
[nret_id].mon_cb.rvfi_csr_rmask;
189                     csr_trn.wmask = cntxt.csr_vif[csr]
[nret_id].mon_cb.rvfi_csr_wmask;
190                     csr_trn.rdata = cntxt.csr_vif[csr]
[nret_id].mon_cb.rvfi_csr_rdata;
191                     csr_trn.wdata = cntxt.csr_vif[csr]
[nret_id].mon_cb.rvfi_csr_wdata;
192                     mon_trn.name_csrs[csr] = csr_trn;
193                 end
194             end
195             else begin
196                 uvma_rvfi_csr_seq_item_c csr_trn;
197                 longint unsigned addr;
198                 csr_trn = uvma_r-
vfi_csr_seq_item_c#(XLEN)::type_id::create({addr, "_trn"});
199                 addr =
cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_csr_addr;
200                 csr_trn.addr = addr;
201                 if
(cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_csr_wmask != 0) begin
202                     csr_trn.wmask =
cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_csr_wmask;
203                     csr_trn.wdata =
cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_csr_wdata;
204                     csr_trn.rmask =
cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_csr_rmask;
205                     csr_trn.rdata =
cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_csr_rdata;
206                     mon_trn.addr_csrs[addr] = csr_trn;
207                 end
208                 foreach(supported_csrs_addrs[i]) begin
209                     addr = supported_csrs_addrs[i];
210                     csr_trn = uvma_r-
vfi_csr_seq_item_c#(XLEN)::type_id::create({addr, "_trn"});
211                     csr_trn.addr = addr;
212                     csr_trn.wmask =
cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_named_csr_wmask[addr];
213                     csr_trn.wdata =

```

```

cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_named_csr_wdata[addr];
224         csr_trn.rmask =
cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_named_csr_rmask[addr];
225         csr_trn.rdata =
cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_named_csr_rdata[addr];
226         mon_trn.addr_csrs[addr] = csr_trn;
227     end
228 end
229
230 // Decode interrupts that need to be commu-
nicated to ISS (external or NMI bus faults)
231 if (mon_trn.intr) begin
232     // The cause of the interrupt should be
in the "rdata" field of the mcause CSR RVFI port
233     bit [XLEN-1:0] csr_mcause;
234
235     if (!cfg.unified_csr_vif)
236         csr_mcause = mon_trn.name_c-
srs["mcause"].rdata;
237     else
238         csr_mcause = mon_trn.addr_c-
srs[MCAUSE].rdata;
239
240     // External interrupt
241     if (csr_mcause[31]) begin
242         // NMI - Load fault
243         if (cfg.nmi_load_fault_enabled &&
csr_mcause[XLEN-2:0] == cfg.nmi_load_fault_cause) begin
244             mon_trn.insn_nmi_load_fault = 1;
245         end
246         // NMI - Store fault
247     else if (cfg.nmi_store_fault_enabled
&& csr_mcause[XLEN-2:0] == cfg.nmi_store_fault_cause) begin
248         mon_trn.insn_nmi_store_fault =
1;
249     end
250     // External interrupt
251     else begin
252         mon_trn.insn_interrupt = 1;
253         mon_trn.insn_interrupt_id =
{ 1'b0, csr_mcause[XLEN-2:0] };
254     end
255 end
256 end
257 if (!cfg.unified_csr_vif && mon_trn.name_c-
srs.exists("dcsr"))
258     dcsr_ret_data = mon_trn.name_c-
srs["dcsr"].get_csr_retirement_data();
259     else if (cfg.unified_csr_vif &&
mon_trn.addr_csrs.exists(DCSR))
260         dcsr_ret_data = mon_trn.addr_c-
srs[DCSR].get_csr_retirement_data();
261
262 // In debug mode, detect NMIP event for a
data bus error
263 if (mon_trn.dbg_mode &&
!last_dcsr_nmip &&
264     mon_trn.nmip[0] &&
265     dcsr_ret_data[3])
266     begin
267         `uvm_info("RVFIMON", $sformatf("Debug
NMIP"), UVM_LOW);
268
269         if (mon_trn.nmip[1] == 0) begin
270             mon_trn.insn_nmi_load_fault = 1;
271         end else begin
272             mon_trn.insn_nmi_store_fault = 1;
273         end
274     end
275
276 // Latch the last DCSR NMIP bit to detect
positive assertion of dcsr.nmip
277 last_dcsr_nmip = dcsr_ret_data[3];
278
279 // Detect instruction bus fault
280 if (cfg.insn_bus_fault_enabled &&
281     mon_trn.is_trap() &&
282     mon_trn.get_trap_cause() ==
cfg.insn_bus_fault_cause) begin
283     mon_trn.insn_bus_fault = 1;
284
285     `uvm_info("RVFIMON", $sformatf("Detected bus
fault"), UVM_LOW)
286 end
287
288 `uvm_info(log_tag, $sformatf("%s", mon_trn.con-
vert2string()), UVM_HIGH);
289
290 ap.write(mon_trn);
291 end
292 end
293 endtask : monitor_rvfi_instr
294
295 `endif // __UVM_RVFI_INSTR_MON_SV__
296

```

9. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_macros.sv (23 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 // Copyright 2020 Silicon Labs, Inc.
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
6 // (the "License");
7 // you may not use this file except in compliance with the Li-
8 // cense.
9 // You may obtain a copy of the License at
10 // https://solderpad.org/licenses/
11 // Unless required by applicable law or agreed to in writing,
12 // software distributed under the License is distributed on an "AS IS"
13 // BASIS,
14 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
15 // press or implied.
16 // See the License for the specific language governing permis-
17 // sions and
18 // limitations under the License.
19 `ifndef __UVMA_RVFI_MACROS_SV__
20 `define __UVMA_RVFI_MACROS_SV__
21
22
23 `endif // __UVMA_RVFI_MACROS_SV__

```

10. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_mon_trn_logger.sv (199 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 // Copyright 2020 Silicon Labs, Inc.
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
6 // (the "License");
7 // you may not use this file except in compliance with the Li-
8 // cense.
9 // You may obtain a copy of the License at
10 // https://solderpad.org/licenses/
11 // Unless required by applicable law or agreed to in writing,
12 // software distributed under the License is distributed on an "AS IS"
13 // BASIS,
14 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
15 // press or implied.
16 // See the License for the specific language governing permis-
17 // sions and
18 // limitations under the License.
19 `ifndef __UVMA_RVFI_MON_TRN_LOGGER_SV__
20 `define __UVMA_RVFI_MON_TRN_LOGGER_SV__
21
22 /**
23  * Component writing Rvfi monitor transactions rvfi data to
24  * disk as plain text.
25  */
26 class uvma_rvfi_mon_trn_logger_c#(int ILEN=DEFAULT_ILEN,
27                                   int XLEN=DEFAULT_XLEN) ex-
28 tends uvml_logs_mon_trn_logger_c#(
29     .T_TRN (uvml_trn_seq_item_c),
30     .T_CFG (uvma_rvfi_cfg_c ),
31     .T_CNTXT(uvma_rvfi_cntxt_c )
32 );
33
34 uvml_analys-
35 is_imp_rvfi_instr#(uvma_rvfi_instr_seq_item_c#(ILEN,XLEN), uvma_r-
36 vfi_mon_trn_logger_c) instr_export;
37
38 uvma_rvfi_instr_table_seq_item_c#(ILEN,XLEN) in-
39 str_table[bit[XLEN-1:0]];
40
41 `uvm_component_param_utils(uvma_rvfi_mon_trn_logger_c)
42
43 /**
44  * Default constructor.
45  */
46 function new(string name="uvma_rvfi_mon_trn_logger",
47 uvml_component parent=null);
48     super.new(name, parent);
49     instr_export = new("instr_export", this);
50 endfunction : new
51
52 /**
53  * Build phase - attempt to load a firmware itb file (in-
54  * struction table file)
55  */
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199

```

```

50 */
51 function void build_phase(uvm_phase phase);
52
53     read_itb_file();
54
55 endfunction : build_phase
56
57 /**
58  * Read the itb file and build an instruction library
59  */
60 function void read_itb_file();
61
62     string file;
63
64     if ($value$plusargs("itb_file=%s", file)) begin
65         int unsigned itb_lineno;
66         int fh;
67
68         fh = $fopen(file, "r");
69         if (fh == 0) begin
70
71             `uvm_fatal("RVFIMONLOG", $sformatf("Could not open
72 itb file: %s", file));
73         end
74
75         while (!$feof(fh)) begin
76             string str;
77
78             int unsigned num;
79             int unsigned c;
80             string src_code_q[$];
81             uvma_rvfi_instr_table_seq_item_c instr;
82
83             itb_lineno++;
84             c = $fscanf(fh, "%1s%d", str, num);
85             instr = uvma_r-
86 vfi_instr_table_seq_item_c#(ILEN,XLEN)::type_id::create("instr");
87
88             `uvm_info("RVFIMONLOG", $sformatf("Parsing
89 line_num: %0d with %0d lines of src", itb_lineno, num), UVM_HIGH)
90             if (c == -1) begin
91                 break;
92             end
93             else if (num == 0) begin
94                 src_code_q.push_back("N/A");
95             end
96             else begin
97                 // Accumulate the source code
98                 repeat (num) begin
99                     itb_lineno++;
100                     c = $fscanf(fh, "%1s", str);
101                     c = $fgets(str, fh);
102                 end
103                 src_code_q.push_back(str.substr(0,str.len()-1));
104             end
105             instr.src_code = new[src_code_q.size()]
106 (src_code_q);
107
108             // Parse the context of the source code
109             c = $fscanf(fh, "%d %s %s %d %s %d", instr.addr,
110 instr.src_function, instr.filename, instr.lineno, instr.mcode, num);
111             for (int i = 0; i < num; i++) begin
112                 string asm_str;
113
114                 c = $fscanf(fh, "%s", asm_str);
115                 if (i == 0)
116                     instr.asm = asm_str;
117                 else
118                     instr.asm = { instr.asm, " ", asm_str };
119             end
120             itb_lineno++;
121
122             // Fatal error if instruction already exists at
123 the respective address
124             if (instr_table.exists(instr.addr)) begin
125                 `uvm_info("RVFIMONLOG", $sformatf("%s", in-
126 str_table[instr.addr].sprint(), UVM_LOW);
127                 `uvm_info("RVFIMONLOG", $sformatf("%s", in-
128 str.sprint(), UVM_LOW);
129                 `uvm_fatal("RVFIMONLOG", $sformatf("%0d: In-
130 struction at address: 0x%08x already exists in instruction table",
131 itb_lineno, instr.addr));
132             end
133             instr_table[instr.addr] = instr;
134
135             `uvm_info("RVFIMONLOG", $sformatf("%s", in-
136 str.sprint(), UVM_DEBUG);
137         end
138     end
139 endfunction : read_itb_file
140
141 /**
142  * Writes contents of t to disk
143  */
144 virtual function void write(uvml_trn_seq_item_c t);
145 endfunction : write
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199

```

```

138 virtual function void
write_rvfi_instr(uvma_rvfi_instr_seq_item_c#(ILEN,XLEN) t);
139 string instr;
140
141 instr = $sformatf(format_instr_str, $sformatf("%t",
$time),
142 t.cycle_cnt,
143 t.order,
144 t.pc_rdata,
145 t.get_insn_word_str(),
146 get_mode_str(t.mode),
147 t.rs1_addr, t.rs1_rdata,
148 t.rs2_addr, t.rs2_rdata,
149 t.rd1_addr, t.rd1_wdata);
150
151 if (t.mem_wmask)
152 instr = $sformatf({"%s ", format_mem_str}, instr, "
WR", t.mem_addr, t.get_mem_data_string());
153 else if (t.mem_rmask)
154 instr = $sformatf({"%s ", format_mem_str}, instr, "
RD", t.mem_addr, t.get_mem_data_string());
155 else
156 instr = $sformatf("%s | -- | ----- | ----- |",
instr);
157
158 if (t.insn_interrupt)
159 instr = $sformatf("%s INTR %0d", instr, t.insn_inter-
rupt_id);
160 if (t.insn_nmi_load_fault)
161 instr = $sformatf("%s NMI LOAD", instr);
162 if (t.insn_nmi_store_fault)
163 instr = $sformatf("%s NMI STORE", instr);
164 if (t.dbg)
165 instr = $sformatf("%s DEBUG", instr);
166
167 if (instr_table.exists(t.pc_rdata)) begin
168 instr = $sformatf("%s %s %s %0d - %s",
169 instr,
170 in-
str_table[t.pc_rdata].src_function,
171 instr_table[t.pc_rdata].filename,
172 instr_table[t.pc_rdata].lineno,
173 instr_table[t.pc_rdata].asm);
174 end
175
176 fwrite(instr);
177
178 endfunction : write_rvfi_instr
179
180 /**
181 * Writes log header to disk
182 */
183 virtual function void print_header();
184
185 string banner = {160{"-"};
186
187 fwrite(banner);
188 fwrite($sformatf(format_header_str,
189 "TIME", "CYCLE", "ORDER", "PC", "IN-
STR", "M", "RS1", "RS1_DATA", "RS2", "RS2_DATA", "RD",
190 "RD_DATA", "MEM", "MEM_ADDR",
"MEM_DATA", "INSTRUCTION"));
191 fwrite(banner);
192
193 endfunction : print_header
194
195 endclass : uvma_rvfi_mon_trn_logger_c
196
197
198 `endif // __UVMA_RVFI_MON_TRN_LOGGER_SV__
199

```

11. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_pkg.sv (71 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 // Copyright 2020 Silicon Labs, Inc.
5 //
6 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
7 // you may not use this file except in compliance with the Li-
cense.
8 // You may obtain a copy of the License at
9 //
10 // https://solderpad.org/licenses/
11 //
12 // Unless required by applicable law or agreed to in writing,
software
13 // distributed under the License is distributed on an "AS IS"
BASIS,
14 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
15 // See the License for the specific language governing permis-
sions and
16 // limitations under the License.
17 //
18

```

```

19
20 `ifndef __UVMA_RVFI_PKG_SV__
21 `define __UVMA_RVFI_PKG_SV__
22
23
24 // Pre-processor macros
25 `include "uvvm_macros.svh"
26 `include "uvml_hrtbt_macros.sv"
27 `include "uvma_rvfi_macros.sv"
28
29 /**
30 * Encapsulates all the types needed for an UVM agent capable
of driving and/or
31 * monitoring Clock & Reset.
32 */
33 package uvma_rvfi_pkg;
34
35 import uvvm_pkg :::*;
36 import uvml_hrtbt_pkg :::*;
37 import uvml_trn_pkg :::*;
38 import uvml_logs_pkg :::*;
39 import uvma_core_cntrl_pkg :::*;
40
41 // Analysis implementation declarations
42 `uvvm_analysis_imp_decl(_rvfi_instr)
43
44 // Constants / Structs / Enums
45 `include "uvma_rvfi_constants.sv"
46 `include "uvma_rvfi_tdefs.sv"
47
48 // Objects
49 `include "uvma_rvfi_cfg.sv"
50 `include "uvma_rvfi_cntxt.sv"
51
52 // High-level transactions
53 `include "seq/uvma_rvfi_csr_seq_item.sv"
54 `include "seq/uvma_rvfi_instr_seq_item.sv"
55 `include "seq/uvma_rvfi_instr_table_seq_item.sv"
56
57 // Agent components
58 `include "uvma_rvfi_mon_trn_logger.sv"
59 `include "uvma_rvfi_instr_mon.sv"
60 `include "uvma_rvfi_agent.sv"
61
62 endpackage : uvma_rvfi_pkg
63
64 // Interface(s) / Module(s) / Checker(s)
65 `include "uvma_rvfi_instr_if.sv"
66 `include "uvma_rvfi_csr_if.sv"
67 `include "uvma_rvfi_assert.sv"
68
69 `endif // __UVMA_RVFI_PKG_SV__
70
71

```

12. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_tdefs.sv (109 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 // Copyright 2020 Silicon Labs, Inc.
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
6 // you may not use this file except in compliance with the Li-
cense.
7 // You may obtain a copy of the License at
8 //
9 // https://solderpad.org/licenses/
10 //
11 // Unless required by applicable law or agreed to in writing,
software
12 // distributed under the License is distributed on an "AS IS"
BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14 // See the License for the specific language governing permis-
sions and
15 // limitations under the License.
16
17
18 `ifndef __UVMA_RVFI_TDEFS_SV__
19 `define __UVMA_RVFI_TDEFS_SV__
20
21 typedef enum bit[MODE_WL-1:0] {
22 UVMA_RVFI_U_MODE = 0,
23 UVMA_RVFI_S_MODE = 1,
24 UVMA_RVFI_RESERVED_MODE = 2,
25 UVMA_RVFI_M_MODE = 3
26 } uvma_rvfi_mode;
27
28 typedef struct packed {
29 bit [MAX_XLEN-1:0] nret_id;
30 bit [MAX_XLEN-1:0] cycle_cnt;
31 bit [MAX_XLEN-1:0] order;
32 bit [MAX_XLEN-1:0] insn;
33 bit [MAX_XLEN-1:0] trap;
34 bit [MAX_XLEN-1:0] halt;
35 bit [MAX_XLEN-1:0] intr;
36 bit [MAX_XLEN-1:0] mode;

```



```

37 bit [MAX_XLEN-1:0] ixl;
38 bit [MAX_XLEN-1:0] dbg;
39 bit [MAX_XLEN-1:0] dbg_mode;
40 bit [MAX_XLEN-1:0] nmip;
41
42 bit [MAX_XLEN-1:0] insn_interrupt;
43 bit [MAX_XLEN-1:0] insn_interrupt_id;
44 bit [MAX_XLEN-1:0] insn_bus_fault;
45 bit [MAX_XLEN-1:0] insn_nmi_store_fault;
46 bit [MAX_XLEN-1:0] insn_nmi_load_fault;
47
48 bit [MAX_XLEN-1:0] pc_rdata;
49 bit [MAX_XLEN-1:0] pc_wdata;
50
51 bit [MAX_XLEN-1:0] rs1_addr;
52 bit [MAX_XLEN-1:0] rs1_rdata;
53
54 bit [MAX_XLEN-1:0] rs2_addr;
55 bit [MAX_XLEN-1:0] rs2_rdata;
56
57 bit [MAX_XLEN-1:0] rs3_addr;
58 bit [MAX_XLEN-1:0] rs3_rdata;
59
60 bit [MAX_XLEN-1:0] rd1_addr;
61 bit [MAX_XLEN-1:0] rd1_wdata;
62
63 bit [MAX_XLEN-1:0] rd2_addr;
64 bit [MAX_XLEN-1:0] rd2_wdata;
65
66 bit [MAX_XLEN-1:0] mem_addr;
67 bit [MAX_XLEN-1:0] mem_rdata;
68 bit [MAX_XLEN-1:0] mem_rmask;
69 bit [MAX_XLEN-1:0] mem_wdata;
70 bit [MAX_XLEN-1:0] mem_wmask;
71
72 bit [CSR_MAX_SIZE-1:0] [MAX_XLEN-1:0] csr_valid;
73 bit [CSR_MAX_SIZE-1:0] [MAX_XLEN-1:0] csr_addr;
74 bit [CSR_MAX_SIZE-1:0] [MAX_XLEN-1:0] csr_rdata;
75 bit [CSR_MAX_SIZE-1:0] [MAX_XLEN-1:0] csr_rmask;
76 bit [CSR_MAX_SIZE-1:0] [MAX_XLEN-1:0] csr_wdata;
77 bit [CSR_MAX_SIZE-1:0] [MAX_XLEN-1:0] csr_wmask;
78
79 } st_rvfi;
80
81 `define ST_NUM_WORDS ($size(st_rvfi)/MAX_XLEN)
82 parameter ST_NUM_WORDS = ($size(st_rvfi)/MAX_XLEN);
83
84 typedef bit [ST_NUM_WORDS-1:0] [MAX_XLEN-1:0] vector_rvfi;
85 typedef union {
86     st_rvfi rvfi;
87     vector_rvfi array;
88 } union_rvfi;
89
90 function string get_mode_str(uvma_rvfi_mode mode);
91 case (mode)
92     UVMA_RVFI_U_MODE: return "U";
93     UVMA_RVFI_M_MODE: return "M";
94     UVMA_RVFI_S_MODE: return "S";
95 endcase
96
97 return "?";
98
99 endfunction : get_mode_str
100
101 function bit is_csr_insn (st_rvfi rvfi);
102     return (rvfi.insn[6:0] == 'b1110011);
103 endfunction : is_csr_insn
104
105 function bit [11:0] get_insn_rs1 (st_rvfi rvfi);
106     return rvfi.insn[31:20];
107 endfunction
108
109 `endif // __UVMA_RVFI_TDEFS_SV__

```

13. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/seq/uvma_rvfi_csr_seq_item.sv (88 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 //
4 // Licensed under the Solderpad Hardware Licence, Version 2.0
5 // (the "License");
6 // you may not use this file except in compliance with the Li-
7 // cense.
8 // You may obtain a copy of the License at
9 //
10 // https://solderpad.org/licenses/
11 //
12 // Unless required by applicable law or agreed to in writing,
13 // software

```

```

11 // distributed under the License is distributed on an "AS IS"
BASIS,
12 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
13 // See the License for the specific language governing permis-
sions and
14 // limitations under the License.
15
16
17 `ifndef __UVMA_RVFI_CSR_SEQ_ITEM_SV__
18     `define __UVMA_RVFI_CSR_SEQ_ITEM_SV__
19
20
21 /**
22  * Object created by RVFI agent sequences extending uvma_r-
vfi_seq_base_c.
23  */
24
25 class uvma_rvfi_csr_seq_item_c#(int XLEN=DEFAULT_XLEN) extends
uvml_trn_seq_item_c;
26
27     rand int unsigned nret_id;
28
29     string csr;
30     rand bit [XLEN-1:0] addr;
31
32     rand bit [XLEN-1:0] rmask;
33     rand bit [XLEN-1:0] wmask;
34     rand bit [XLEN-1:0] rdata;
35     rand bit [XLEN-1:0] wdata;
36
37     static protected string _log_format_string = "0x%08x %s
0x%01x 0x%08x";
38
39     `uvm_object_param_utils_begin(uvma_rvfi_csr_seq_item_c)
40         `uvm_field_int(nret_id, UVM_DEFAULT)
41         `uvm_field_int(rmask, UVM_DEFAULT)
42         `uvm_field_int(wmask, UVM_DEFAULT)
43         `uvm_field_int(rdata, UVM_DEFAULT)
44         `uvm_field_int(wdata, UVM_DEFAULT)
45     `uvm_object_utils_end
46
47 /**
48  * Default constructor.
49  */
50 extern function new(string name="uvma_rvfi_csr_seq_item");
51
52 /**
53  * Fetch the value of the CSR at instruction retirement
54  * taking into account write data and write mask
55  */
56 extern virtual function bit[uvma_r-
vfi_csr_seq_item_c::XLEN-1:0] get_csr_retirement_data();
57
58 /**
59  * One-liner log message
60  */
61 extern function string convert2string();
62
63 endclass : uvma_rvfi_csr_seq_item_c
64
65 `pragma protect begin
66 function uvma_rvfi_csr_seq_item_c::new(string name="uvma_r-
vfi_csr_seq_item");
67     super.new(name);
68 endfunction : new
69
70
71 function bit [uvma_rvfi_csr_seq_item_c::XLEN-1:0] uvma_r-
vfi_csr_seq_item_c::get_csr_retirement_data();
72
73 // Any bits with wmask set should use the wdata
74 // All other bits should use the rdata
75 // rmask is not used
76 return (wmask & wdata) | (~wmask & rdata);
77
78 endfunction : get_csr_retirement_data
79
80 function string uvma_rvfi_csr_seq_item_c::convert2string();
81
82     convert2string = $sformatf("csr: %s rdata(mask) 0x%08x:
0x%08x, wdata(mask) 0x%08x:0x%08x",
83         csr, rdata, rmask, wdata,
wmask);
84
85 endfunction : convert2string
86
87 `pragma protect end
88 `endif // __UVMA_RVFI_CSR_SEQ_ITEM_SV__

```


step-compare 테스트벤치

What this is. The testbench that steps a reference ISS and the RTL core together and compares architectural state after every retired instruction.

Why it exists. Because it is the operational definition of a bug in a CPU project: model and design disagree. It compares the PC, all 32 general-purpose registers and the CSRs.

Undergraduate review · EECS concepts. Reference-model comparison; scoreboarding; masking of implementation-defined state.

Graduate · Practice note. Read the comments. One records that static register checks were disabled because bus stalls made them fail — a real engineering compromise between coverage and usability, documented in place. **That is what honest verification code looks like.**

1 files • 25 lines

API — uvmt_cv32e40p_constants.sv

Table 1873. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_constants.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMT_CV32E40P_CONSTANTS_SV__		`endif // __UVMT_CV32E40P_CONSTANTS_SV__

1. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_constants.sv (25 lines)

```
1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
6 // (the "License");
7 // you may not use this file except in compliance with the Li-
8 // cense.
9 // You may obtain a copy of the License at
10 // https://solderpad.org/licenses/
11 // Unless required by applicable law or agreed to in writing,
```

```
software
12 // distributed under the License is distributed on an "AS IS"
13 // BASIS,
14 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
15 // press or implied.
16 // See the License for the specific language governing permis-
17 // sions and
18 // limitations under the License.
19 //
20 `ifndef __UVMT_CV32E40P_CONSTANTS_SV__
21 `define __UVMT_CV32E40P_CONSTANTS_SV__
22
23
24
25 `endif // __UVMT_CV32E40P_CONSTANTS_SV__
```